

My Project

Generated by Doxygen 1.9.8

Chapter 1

Cellular Automata Viewer

This folder contains the first standalone cellular-automata tool for the Genesys Qt GUI.

Current scope:

- 2D demo built from the existing cellular-automata plugin classes
- UI selectors for rule, neighborhood, and state preset
- Forest Fire preset with four enumerated states
- mouse editing with a selectable paint state
- discrete stepping plus timer-driven auto-run
- JSON save/load for lattice size, time, and cell states

Known limitations in this first pass:

- the viewer is intentionally demo-focused, not a universal CA builder yet
- more advanced neighborhood/radius combinations remain plugin-side follow-up work

Chapter 2

Genesys MCP Connector

Python MCP server that exposes GenESyS simulation capabilities to AI assistants (Claude Desktop, Claude Code, and any MCP-compatible client).

2.1 Prerequisites

- Python 3.10+
- `genesys-worker` running (see [source/applications/worker/README.md](#))

2.2 Install

```
cd source/applications/mcp
pip install -e .
```

2.3 Run

```
# via entry point
genesys-mcp

# or via module
python -m genesys_mcp
```

The server communicates over stdio (default FastMCP transport). [Set](#) `GENESYS_URL` to point to a non-default worker app address:

```
GENESYS_URL=http://192.168.1.10:8080 genesys-mcp
```

2.4 Claude Desktop integration

Add to `~/ .config/claude/claude_desktop_config.json` (Linux) or `~/Library/Application Support/Claude/claude_desktop_config.json` (macOS):

```
{
  "mcpServers": {
    "genesys": {
      "command": "genesys-mcp",
      "env": {
        "GENESYS_URL": "http://localhost:8080"
      }
    }
  }
}
```

```
}
}
```

If `genesys-mcp` is not on `PATH` (e.g. installed in a `venv`), use the full path:

```
{
  "mcpServers": {
    "genesys": {
      "command": "/path/to/venv/bin/genesys-mcp",
      "env": {
        "GENESYS_URL": "http://localhost:8080"
      }
    }
  }
}
```

2.5 Claude Code integration

```
claude mcp add genesys -- genesys-mcp
```

Or with a custom URL:

```
claude mcp add genesys -e GENESYS_URL=http://localhost:8080 -- genesys-mcp
```

2.6 Workflow

All tools that interact with the simulator require a session token. Call `genesys_auth` once per conversation and pass the returned token to every subsequent tool.

```
1. genesys_auth          → { token: "..." }
2. genesys_new_model    (token)
3. genesys_import_model_language (token, model_text)
4. genesys_configure_simulation (token, replications=10, length=480.0)
5. genesys_run_simulation (token)
6. genesys_create_job    (token) → { jobId: "..." }
7. genesys_run_job       (token, jobId)
8. genesys_job_result    (token, jobId)
```

2.7 Available tools (18)

Tool	Auth required	Description
<code>genesys_health</code>	No	Check worker app reachability
<code>genesys_auth</code>	No	Create session, returns token
<code>genesys_info</code>	Yes	Simulator version and licence
<code>genesys_new_model</code>	Yes	Create empty model
<code>genesys_get_model</code>	Yes	Describe current model
<code>genesys_save_model</code>	Yes	Save model to workspace file
<code>genesys_load_model</code>	Yes	Load model from workspace file
<code>genesys_import_model_language</code>	Yes	Import model from text
<code>genesys_simulation_status</code>	Yes	Current simulation state
<code>genesys_configure_simulation</code>	Yes	Set replication/length/warmup params
<code>genesys_run_simulation</code>	Yes	Run simulation synchronously
<code>genesys_step_simulation</code>	Yes	Advance one event step
<code>genesys_worker_info</code>	No	Worker identity metadata
<code>genesys_worker_capabilities</code>	No	Worker capability flags
<code>genesys_create_job</code>	Yes	Create job from current model
<code>genesys_job_status</code>	Yes	Query job state
<code>genesys_run_job</code>	Yes	Execute job synchronously
<code>genesys_job_result</code>	Yes	Retrieve job result summary

Chapter 3

Genesys Worker Application

First functional implementation of the **Worker** application type for GenESyS.

3.1 Current scope

- CMake target: `genesys_worker_app` (enabled with `-DGENESYS_BUILD_WORKER_APPLICATION=ON`).
- Executable name: `genesys-worker`.
- Layered architecture for HTTP transport, API routing, session/token, and simulator service.
- Stateful session management with one [Simulator](#) per session and per-session workspace directories.
- Endpoints:
 - GET `/health`
 - POST `/api/v1/auth/session`
 - GET `/api/v1/simulator/info` (requires Authorization: Bearer <token>)
 - POST `/api/v1/models` (requires Authorization: Bearer <token>)
 - GET `/api/v1/models/current` (requires Authorization: Bearer <token>)
 - POST `/api/v1/models/save` (requires Authorization: Bearer <token>)
 - POST `/api/v1/models/load` (requires Authorization: Bearer <token>)
 - GET `/api/v1/simulation/status` (requires Authorization: Bearer <token>)
 - POST `/api/v1/simulation/config` (requires Authorization: Bearer <token>)
 - POST `/api/v1/simulation/run` (requires Authorization: Bearer <token>)
 - POST `/api/v1/simulation/step` (requires Authorization: Bearer <token>)

3.2 Worker cycle Stage 1 (new cycle)

The Worker application is now also being evolved as a **worker API** that can be discovered by a future distributed orchestrator application.

This stage adds two public worker discovery endpoints (no Bearer token required):

- GET `/api/v1/worker/info`
- GET `/api/v1/worker/capabilities`

These routes expose worker identity metadata and current implementation capability flags. They are intentionally limited to discovery only and do not include job submission, polling, or background execution in this stage.

3.3 Worker cycle Stage 2

This stage starts worker model-ingress support with one authenticated and session-scoped endpoint:

- `POST /api/v1/worker/models/import-language` (requires Authorization: Bearer <token>)

The request body must be plain text GenESyS model language/specification. The worker imports this text into the session simulator as the current model.

This stage still does **not** include multipart/binary upload, distributed job submission, polling, or background orchestration.

3.4 Worker cycle Stage 3

This stage introduces a minimal worker job abstraction with authenticated, session-scoped endpoints:

- `POST /api/v1/worker/jobs` (requires Authorization: Bearer <token>)
- `GET /api/v1/worker/jobs/{jobId}` (requires Authorization: Bearer <token>)

Behavior in this stage:

- job creation requires a valid current model in the session;
- each job is created in in-memory storage with state `queued`;
- job creation persists a session-workspace snapshot file (for example `job_<id>.gen`) to decouple future execution from live model mutations;
- job metadata can be queried by id from the same authenticated session.

This stage still does **not** execute jobs, provide result retrieval, add polling loops, or introduce background worker threads.

3.5 Worker cycle Stage 4

This stage adds explicit, synchronous execution of existing worker jobs:

- `POST /api/v1/worker/jobs/{jobId}/run` (requires Authorization: Bearer <token>)

Behavior in this stage:

- execution is triggered only for an existing job id;
- job state is updated in memory through `queued -> running -> finished|failed`;
- execution is synchronous and performed by the session simulator using the job snapshot file;
- `GET /api/v1/worker/jobs/{jobId}` remains the basic state inspection/polling mechanism after run requests.

This stage still does **not** include background execution, queue worker loops, cancellation, streaming updates, or a dedicated job result endpoint.

3.6 Worker cycle Stage 5

This stage adds terminal result retrieval for previously executed worker jobs:

- `GET /api/v1/worker/jobs/{jobId}/result` (requires Authorization: Bearer <token>)

Behavior in this stage:

- result retrieval is available only after the job reaches a terminal state (`finished` or `failed`);
- queued or running jobs return a conflict response because no terminal result exists yet;
- the endpoint returns a minimal structured simulation summary persisted at the end of synchronous execution;
- execution remains synchronous and non-background (`POST /run` still performs the run inline);
- cancellation is still not available;
- distributed orchestration remains out of scope.

3.7 Worker cycle Stage 6

This stage hardens HTTP transport behavior for existing worker and session APIs without changing route contracts:

- request reading is now incremental (`recv` loop) instead of single-shot reads;
- request assembly is bounded by a transport limit of **1 MiB** (larger payloads return `413 Payload Too Large`);
- socket receive/send timeouts are configured to **5 seconds** per client connection;
- when timeout happens before a complete request is assembled, the server returns `408 Request Timeout`;
- transport-level error payloads remain JSON-shaped and consistent with current server error responses.

This stage is explicitly operational hardening only. Application-layer endpoints and worker cycle stages 1-5 contracts remain unchanged.

3.8 How to run

```
cmake -S . -B build/worker-debug -G Ninja -DGENESYS_BUILD_WORKER_APPLICATION=ON
cmake --build build/worker-debug --target genesys_worker_app
./build/worker-debug/source/applications/worker/genesys-worker --port 8080
```

Optional parameters:

- `--port <N>`: HTTP port (default 8080).
- `--max-requests <N>`: exit after N requests (useful for scripted smoke tests).

3.9 Security notice

Session creation in Stage 1 is provisional and stateful in memory. Generated tokens are opaque random strings and **not** production-grade authentication.

For Stage 3 model persistence:

- Save/load operations are restricted to each session workspace directory.
- API requests accept only a `filename` `basename` (no paths).
- Filenames must match `[A-Za-z0-9._-]` and cannot include `..`, `/`, or `\`.

For Stage 4 simulation configuration:

- `POST /api/v1/simulation/config` currently uses a minimal contract parser without external JSON libraries.
- All configuration fields are required in the request body: `numberOfReplications`, `replicationLength`, `warmUpPeriod`, `pauseOnEvent`, `pauseOnReplication`, `initializeStatistics`, `initializeSystem`.

For Stage 5 simulation execution:

- `POST /api/v1/simulation/run` and `POST /api/v1/simulation/step` are synchronous wrappers over `ModelSimulation::start()` and `ModelSimulation::step()`.
- This stage intentionally does not introduce asynchronous/background simulation control endpoints.

Chapter 4

Customization Points

The custom directory is an injection point for custom user configurations.

4.1 Header `<gtest.h>`

4.1.1 The following macros can be defined:

- `GTEST_OS_STACK_TRACE_GETTER_` - The name of an implementation of `OsStackTraceGetterInterface`.
- `GTEST_CUSTOM_TEMPDIR_FUNCTION_` - An override for `testing::TempDir()`. See `testing::TempDir` for semantics and signature.

4.2 Header `<gtest-port.h>`

The following macros can be defined:

4.2.1 Logging:

- `GTEST_LOG_(severity)`
- `GTEST_CHECK_(condition)`
- Functions `LogToStderr()` and `FlushInfoLog()` have to be provided too.

4.2.2 Threading:

- `GTEST_HAS_NOTIFICATION_` - Enabled if Notification is already provided.
- `GTEST_HAS_MUTEX_AND_THREAD_LOCAL_` - Enabled if `Mutex` and `ThreadLocal` are already provided. Must also provide `GTEST_DECLARE_STATIC_MUTEX_(mutex)` and `GTEST_DEFINE_STATIC_MUTEX_(mutex)`
- `GTEST_EXCLUSIVE_LOCK_REQUIRED_(locks)`
- `GTEST_LOCK_EXCLUDED_(locks)`

4.2.3 Underlying library support features

- `GTEST_HAS_CXXABI_H_`

4.2.4 Exporting API symbols:

- `GTEST_API_` - Specifier for exported symbols.

4.3 Header `<gtest-printers.h>`

- See documentation at [gtest/gtest-printers.h](#) for details on how to define a custom printer.

Chapter 5

Análise técnica do parser (Bison/Flex)

5.1 Audit Status (WiP20261)

- Branch audited: `WiP20261`
- Audit scope: reauditoria do lexer/gramática e ações semânticas em `source/parser/parserBison↔Flex`
- Status legend: DONE, PARTIAL, OPEN, UNCERTAIN, SUPERSEDED
- Data desta reauditoria documental: 2026-04-10

5.2 Escopo

Arquivos revalidados:

- `source/parser/parserBisonFlex/bisonparser.yy`
- `source/parser/parserBisonFlex/lexerparser.ll`
- `source/parser/parserBisonFlex/bisonparser.report` (não encontrado no checkout atual)

5.3 Registro histórico (pré-reauditoria)

O texto original deste documento apontava conflitos LR elevados, erros de tokenização e lacunas semânticas. Parte desse diagnóstico foi superada por mudanças posteriores; parte ainda permanece relevante.

5.4 Reclassificação dos achados históricos

5.4.1 Conversão hexadecimal

5.4.2 Audit status

DONE

5.4.3 Evidence

- O lexer converte hex com `std::strtoll(yytext, &endPtr, 16)` e emite NUMH.

5.4.4 Remaining gaps

- Sem gap direto neste item.

5.4.5 Tokenização de `<tt>min</tt>` / `<tt>max</tt>`

5.4.6 Audit status

DONE

5.4.7 Evidence

- `min` retorna `mathMIN` e `max` retorna `mathMAX`.

5.4.8 Remaining gaps

- Sem gap direto neste item.

5.4.9 Regex de `<tt>EntitiesWIP</tt>`

5.4.10 Audit status

DONE

5.4.11 Evidence

- Regra atual usa padrão de palavra completa `[eE][nN][tT][iI][tT][iI][eE][sS][wW][iI][pP]`.

5.4.12 Remaining gaps

- Sem gap direto neste item.

5.4.13 Regra isolada `<tt>T</tt>` no lexer

5.4.14 Audit status

SUPERSEDED

5.4.15 Evidence

- A regra isolada não aparece mais no lexer atual.

5.4.16 Remaining gaps

- Sem gap direto neste item.

5.4.17 Token `<tt>ILLEGAL</tt>` antes inútil

5.4.18 Audit status

DONE

5.4.19 Evidence

- A gramática atual inclui `expression: ... | illegal { $$.valor = -1; }`, tornando o caminho alcançável.

5.4.20 Remaining gaps

- Avaliar qualidade de recuperação/diagnóstico de erro além da alcançabilidade.

5.4.21 Estrutura de precedência/associatividade da gramática

5.4.22 Audit status

DONE

5.4.23 Evidence

- Estrutura por níveis explícitos: `logicalOr -> logicalXor -> logicalAnd -> logical←
Not -> relational -> additive -> multiplicative -> power -> unary ->
primary.`
- Potência em `power: unary POWER power`, indicando associatividade à direita.

5.4.24 Remaining gaps

- Não foi possível revalidar numericamente conflitos LR no estado atual sem `bisonparser.report`.

5.5 Pontos ainda pendentes (ou parcialmente mitigados)

5.5.1 Funções semânticas não implementadas totalmente (`<tt>fDISC</tt>`, `<tt>fLASTINQ</tt>`, `<tt>fRESSEIZES</tt>`)

5.5.2 Audit status

OPEN

5.5.3 Evidence

- `fDISC` em `probFunction` contém TODO e retorno placeholder (`sampleDiscrete(0, 0)`).
- `fLASTINQ` possui ação vazia com comentário de não implementado.
- `fRESSEIZES` também está com comentário de não implementado.

5.5.4 Remaining gaps

- Implementar semântica real dessas funções e cobrir com testes de expressão.

5.5.5 Avaliação de `<tt>FORM</tt>`

5.5.6 Audit status

OPEN

5.5.7 Evidence

- Regras de `FORM` continuam comentando problema sério de reentrada e retornam `0 . 0`.

5.5.8 Remaining gaps

- Resolver avaliação de fórmula (reentrância/arquitetura de avaliação) para evitar retorno silencioso inválido.

5.5.9 Risco de null-deref em atribuições/leituras ligadas a evento atual

5.5.10 Audit status

PARTIAL

5.5.11 Evidence

- Há guardas em alguns caminhos (`if (getCurrentEvent() != nullptr && getCurrentEvent()->getEntity() != nullptr)`), mas ainda existem acessos diretos como `fIDENT` e outras ações usando `getCurrentEvent()->getEntity()` sem guarda uniforme.

5.5.12 Remaining gaps

- Padronizar guarda robusta em todos os pontos de acesso a evento/entidade corrente.

5.5.13 Conflitos LR atuais

5.5.14 Audit status

UNCERTAIN

5.5.15 Evidence

- `source/parser/parserBisonFlex/bisonparser.report` não estava disponível no checkout atual, então não houve recontagem atual de conflitos `shift/reduce` e `reduce/reduce`.

5.5.16 Remaining gaps

- Gerar novo report do Bison e atualizar números concretos de conflitos.

5.6 Recomendações arquiteturais que seguem válidas

1. Formalizar tratamento de funções não implementadas com erro explícito (em vez de `0/placeholder silencioso`).
2. Consolidar estratégia de null-safety para contexto de evento corrente.
3. Reintroduzir rotina de auditoria de conflitos LR no pipeline (geração e versionamento de report ou artefato equivalente).
4. Fortalecer testes de regressão de parser para operadores, funções de plugin e cenários de erro.

5.7 Remaining Work

- OPEN — Implementar semântica real de `fDISC`.
- OPEN — Implementar semântica real de `fLASTING`.
- OPEN — Implementar semântica real de `fRESSEIZES`.
- OPEN — Resolver avaliação de `FORM` sem retorno neutro silencioso.
- PARTIAL — Eliminar caminhos restantes de null-deref em `getCurrentEvent()->getEntity()`.
- UNCERTAIN — Recontar conflitos LR com novo `bisonparser.report`.

Chapter 6

Kernel Simulator Unit Tests - Build & Run Guide

This document describes a **ready-to-use CMake/CLion setup** for running all unit tests related to the [Kernel Simulator](#) package.

6.1 What to use (recommended)

Use these presets from `CMakePresets.json`:

- **Configure preset:** `tests-kernel-unit`
- **Build preset:** `tests-kernel-unit`
- **Build target (executed by the preset):** `genesys_kernel_unit_tests_run`

The `genesys_kernel_unit_tests_run` target:

1. Builds all kernel unit-test executables;
 2. Runs all of them automatically (including the generated method-inventory test executable).
-

6.2 CLion setup (single-click workflow)

1. Open **Settings/Preferences** > **Build, Execution, Deployment** > **CMake**.
2. Enable **Load CMake presets** (if not already enabled).
3. Select **Configure preset = tests-kernel-unit**.
4. In the run/build target selector, choose **Build preset = tests-kernel-unit**.
5. Build that preset.

Expected behavior: CLion builds everything required and then runs all kernel unit-test binaries automatically in sequence.

6.3 Command line equivalent

```
cmake --preset tests-kernel-unit  
cmake --build --preset tests-kernel-unit
```

This is equivalent to running the single target `genesys_kernel_unit_tests_run`.

6.4 Executables included in the run target

The run target executes the following unit-test executables:

- `genesys_test_util`
 - `genesys_test_simulator_support`
 - `genesys_test_simulator_runtime`
 - `genesys_test_support_modelmanager`
 - `genesys_test_support_persistence`
 - `genesys_test_runtime_pluginmanager`
 - `genesys_test_support_oneeventmanager`
 - `genesys_test_support_parsermanager`
 - `genesys_test_support_tracemanager`
 - `genesys_test_support_connectionmanager`
 - `genesys_test_support_simulationscenario`
 - `genesys_test_support_experimentmanager`
 - `genesys_test_kernel_simulator_method_inventory` (auto-generated inventory scaffold)
-

6.5 Notes

- The generated inventory test executable is built from:
 - **generator script:** `source/tests/unit/tools/generate_kernel_simulator_test_↵ stubs.py`
 - **generated source:** `source/tests/unit/generated/test_kernel_simulator_↵ method_inventory.generated.cpp`
- If headers in `source/kernel/simulator` change, the generated inventory test file is automatically regenerated during build.

Chapter 7

Architecture notes for <tt>source/tools</tt>

7.1 1. Architectural overview

The `tools` package provides domain-support services for statistical analysis and numerical procedures. It exposes interfaces used by higher-level workflows and retains legacy implementations for compatibility.

7.2 2. Current package responsibilities

- Dataset-oriented analysis orchestration ([DataAnalyser_if](#)).
- Simulation result dataset loading, including Genesys [Record](#) text output with replication metadata.
- Distribution fitting contracts ([Fitter_if](#)).
- Parametric hypothesis testing ([HypothesisTester_if](#), default impl 1).
- Distribution mathematical utilities and quantile inversion.
- Numerical integration and derivation utilities via legacy solver abstractions.
- Binding of abstractions to implementations through traits.

7.3 3. Approved target decomposition

7.3.1 Data analysis

- Keep façade role in [DataAnalyser_if](#).
- Clarify ownership and lifecycle of returned collaborating services.

7.3.2 Fitting

- Preserve legacy fitting API.
- Transition from dummy implementation to a concrete default implementation in phased manner (FITTER-1 delivered baseline behavior, FITTER-2 added functional Beta and Weibull fitting in [FitterDefaultImpl](#), and FITTER-3 promotes [TraitsTools<Fitter_if>](#) to [FitterDefaultImpl](#)).

7.3.3 Hypothesis testing

- Maintain current contract and refine implementation completeness, especially two-population methods.
- HYPTEST-1 alignment keeps the current pooled-vs-Welch heuristic, but documents it as a TODO while consolidating one-population p-value coherence and the two-proportion-difference confidence interval formula.

- HYPTEST-2 final alignment updates one-population proportion confidence intervals to the normal-approximation formulation (including finite-population correction when N is provided).

7.3.4 Distribution abstractions

- Introduce `Distribution_if` and specialized continuous/discrete interfaces.
- Keep legacy static façade while migration is in progress.

7.3.5 Numerical tools

- Split legacy solver responsibilities into dedicated interfaces:
 - `Quadrature_if`
 - `RootFinder_if`
 - `OdeSystem_if`
 - `OdeSolver_if`

7.3.6 Traits/binding

- Keep current traits stable.
- Expand only when safe concrete implementations exist.

7.4 4. Legacy compatibility strategy

- Do not break existing public signatures in this phase.
- Preserve compatibility by keeping legacy classes available while promoting only validated trait selections.
- Add new abstractions as header-only contracts until implementations are mature.

7.5 5. Planned migration order

1. Stabilize documentation and explicit contracts for existing headers.
2. Introduce new interface headers without forcing behavior changes.
3. Add concrete implementations incrementally behind existing traits.
4. Migrate trait bindings once implementations are production-safe.
5. Deprecate legacy conflated abstractions only after full replacement is validated.

7.6 6. AI Assistant component

7.6.1 Role

`AIAssistant_if` is a new high-level tool abstraction that will allow users to describe a simulation system in natural language and have the assistant construct, configure and run a GenESyS model automatically.

7.6.2 Stage 1 (complete) — Interface and implementation scaffold

- `AIAssistant_if`: abstract interface with enumerations (`AIProvider`, `ExecutionState`, `ReasoningEffort`), data structures (`AIAssistantConfiguration`, `AIAssistantRequest`, `AIAssistantResponse`, `GenesysKnowledgeItem`, `GenesysKnowledgeBaseSummary`) and method groups for configuration, security, `SimulatorFacade` integration, knowledge base, provider client, and workflow.
- `AIAssistantDefaultImpl`: concrete class implementing all interface methods.
- Security model: API key stored in memory only; no public getter; `clearApiKey()` overwrites the buffer before clearing; `getMaskedApiKey()` exposes only the last four characters. Permission flags (`allowNetworkAccess`, `allowModelMutation`, `allowSimulationExecution`, `allowFileSystemWrites`) default to false (deny-by-default).
- `TraitsTools<AIAssistant_if>` bound to `AIAssistantDefaultImpl`.

7.6.3 Stage 2 (complete) — Plugin introspection

- `refreshKnowledgeBaseFromSimulator()` populates the knowledge base with:
 - A static catalog of 15 well-known GenESyS building blocks (`Create`, `Dispose`, `Queue`, `Seize`, `Release`, `Process`, `Delay`, `Decide`, `Assign`, `Record`, `Resource`, `Entity`, `Statistics`, `Variable`).
 - Dynamic introspection via `SimulatorFacade::firstPlugin()` / `nextPlugin()`: for each loaded `Plugin`, reads `PluginInformation` fields (typename, isComponent, isSource/isSink, input/output topology, descriptionHelp, languageTemplate).
 - Duplicate entries between the static catalog and dynamic plugins are silently skipped.
- Returns `true` even when no facade is attached (static catalog is still useful).

7.6.4 Stage 3 (complete) — AI provider HTTP clients

- `AIProviderClient_if`: abstract interface for a single chat-completion HTTP call. Structures: `ChatMessage`, `ProviderRequest`, `ProviderResponse`.
- `HttpProviderClientBase`: abstract base implementing `popen+curl` transport. Security: API key written to a 0600 temp file (`mkstemp`), never on the command line. Shared JSON utilities: `_escapeJsonString`, `_extractJsonString`, `_extractJsonInt`, `_buildMessagesArray`, `_extractFirstArrayElement`.
- `OpenAIProviderClientImpl`: OpenAI chat completions; supports o-series reasoning models via `highReasoningMode`; optional `Organization` and `Project` headers.
- `AnthropicProviderClientImpl`: Anthropic Messages API; separates system message from conversation turns; extended-thinking budget for `highReasoningMode`.
- `LocalProviderClientImpl`: Ollama/LM Studio/llama.cpp; OpenAI-compatible format; no API key required.
- `AIAssistant_if` extended with `setProviderClient`, `getProviderClient`, `hasProviderClient`. `AIAssistantDefaultImpl` also exposes `createAndAttachProviderClient()` which creates the right client from configuration and takes ownership.

7.6.5 Stage 4 (complete) — Prompt analysis and model plan generation

- `analyzePrompt()`: builds a system prompt injecting the GenESyS knowledge base, calls the provider client, and returns the LLM's structured analysis.
- `createModelPlan()`: uses a structured system prompt requesting a Markdown model plan (entities, resources, queues, arrival, processing steps, statistics, simulation parameters) and returns it in `AIAssistantResponse::generatedPlan`.
- Both methods enforce `allowNetworkAccess == true` and check `checkReady()`.
- Diagnostics include provider name, model, input/output token counts, and HTTP status.

7.6.6 Stage 5 (complete) — Model construction

- `buildModel()`: makes an LLM call requesting a GenESyS `.gen` model language file. The system prompt includes the `.gen` format specification, a complete example, and the knowledge base. If `AIAssistantRequest::modelPlan` is non-empty it is included as the model specification. The generated `.gen` content is written to a 0600 temp file (`mkstemp`), loaded via `SimulatorFacade::loadModel()`, and then deleted. `AIAssistantResponse::generatedModelLanguage` holds the generated file text.
- `configureSimulation()`: after a model is loaded, makes a small deterministic (`temperature=0`) LLM call to extract simulation parameters as JSON from `AIAssistantRequest::experimentDescription`. Applies non-zero values to the current model via `simSetNumberOfReplications`, `simSetReplicationLength`, `simSetWarmUpPeriod`. Falls back gracefully when no provider is available (`.gen` parameters remain active).
- `HttpProviderClientBase` JSON utilities promoted to `public static` so that `AIAssistantDefaultImpl` can call `_extractJsonInt`, `_extractJsonDouble`, and `_extractJsonString` directly; `_extractJsonDouble` added for floating-point fields.
- `AIAssistantRequest::modelPlan` field added; `execute()` populates it from `createModelPlan()` output and passes the enriched request to `buildModel()`.

7.6.7 Stage 6 (complete) — Simulation execution and result collection

- `runSimulation()`: verifies `allowSimulationExecution` and non-null `SimulatorFacade`, checks not already running, then calls `SimulatorFacade::simStart()` (synchronous). Reports number of completed replications in the response.
- `collectResults()`: iterates `SimulatorFacade::modelGetResponses()` using `List<SimulationResponse*>::front()/next()` and renders a Markdown table of metric name $\rightarrow$ value.
- `execute()` updated to call steps in the correct order with proper plan propagation: `analyzePrompt  $\rightarrow$  createModelPlan  $\rightarrow$  buildModel (with enriched plan)  $\rightarrow$  configureSimulation  $\rightarrow$  runSimulation  $\rightarrow$  collectResults`.

7.6.8 Planned stages

- **Stage 7** — GUI integration: prompt panel, plan review, graphical model creation.
- **Stage 8** — Security and governance: OS secret store, sandbox, dry-run mode, audit log.

7.6.9 Dependency constraints

- `AIAssistant_if.h` uses only a forward declaration of `SimulatorFacade` to avoid pulling the full kernel chain into every including translation unit.
- `AIAssistantDefaultImpl.cpp` includes `kernel/simulator/SimulatorFacade.h` directly.
- No external libraries are introduced in Stage 1.

7.7 7. Stability notes and non-goals for this phase

- FITTER-2 consolidated additional fitting algorithms in `FitterDefaultImpl` (beta and weibull) on top of FITTER-1 families; FITTER-3 now switches the default trait binding so `TraitsTools<Fitter_if>` points to `FitterDefaultImpl`.
- `FitterDummyImpl` is intentionally preserved as a legacy/documental placeholder and is no longer the default fitter binding.
- No full completion of hypothesis-testing theory coverage.
- No numerical refactor of legacy solver internals.
- No cross-package behavior changes outside `source/tools`.

Chapter 8

tools package

8.1 1. Purpose of the tools package

The `source/tools` package hosts statistical and numerical support abstractions used by simulation and analysis flows. It provides interfaces and legacy implementations for fitting, hypothesis testing, probability utilities, and numerical solvers.

8.2 2. Current major abstractions

- `DataAnalyser_if`: high-level façade to orchestrate dataset-oriented analysis services.
- `SimulationResultsDatasetParser`: loader for numeric result files, including Genesys `Record` outputs split by replication and optional time/value columns.
- `Fitter_if`: fitting contract for distribution parameter inference from sample data.
- `HypothesisTester_if`: parametric inference API (confidence intervals and tests).
- `ProbabilityDistributionBase` / `ProbabilityDistribution`: static math façade for PDF/↔ PMF and inverse/quantile routines.
- `Solver_if`: legacy numerical contract mixing integration and derivation/advancement.
- `TraitsTools`: traits registry binding abstractions to concrete implementations.

8.3 3. Current limitations

- Fitting baseline was expanded in FITTER-2: `FitterDefaultImpl` now also provides functional Beta (scaled) and Weibull fitting, still preserving controlled-failure behavior when constraints are not met.
- `FitterDummyImpl` is preserved as a legacy placeholder/documental implementation, but it is no longer the default trait binding after FITTER-3.
- Some hypothesis-testing paths, especially two-population paths, remain partially consolidated.
- Distribution APIs are static utilities, not yet an OO hierarchy with reusable distribution objects.
- Solver abstraction conflates quadrature and ODE-like concerns.

8.4 4. Planned evolution

- Introduce cohesive interfaces for dataset, distributions, quadrature, root finding, and ODE solving.
- Continue hardening `FitterDefaultImpl` as the promoted default fitting implementation.
- Evolve traits coverage to include newly stabilized abstractions.
- Keep legacy interfaces during migration to avoid behavior breaks.

8.5 5. Relationship with kernel/statistics

`source/tools` consumes `kernel/statistics` contracts (collectors, samplers and data files) but this phase does not modify kernel or statistics code. The package remains a consumer and adapter layer over those existing contracts.

8.6 6. AI Assistant abstraction

`source/tools` includes an AI-assisted simulation model-building tool across four completed stages:

- `AIAssistant_if`: interface defining configuration, security, knowledge base, provider client injection, workflow and diagnostics contracts.
- `AIAssistantDefaultImpl`: concrete implementation covering all six complete stages:
 - Full configuration management and secure API key handling.
 - Knowledge base: static catalog of 15 well-known components plus dynamic plugin introspection.
 - `createAndAttachProviderClient()` factory for owned provider clients.
 - `analyzePrompt()/createModelPlan()` — LLM-backed prompt analysis and plan generation.
 - `buildModel()` — LLM generates a `.gen` file, loaded via `SimulatorFacade::loadModel()`.
 - `configureSimulation()` — LLM extracts simulation parameters; applies via `simSet*()`.
 - `runSimulation()` — calls `SimulatorFacade::simStart()` (synchronous).
 - `collectResults()` — formats `modelGetResponses()` as a Markdown results table.
 - `execute()` — orchestrates the full pipeline with plan propagation between steps.
- `AIProviderClient_if`: interface for a single HTTP chat-completion call.
- `HttpProviderClientBase`: abstract base with `open+curl` transport; API key stored in 0600 temp file, never on command line.
- `OpenAIProviderClientImpl`: OpenAI chat completions with o-series reasoning support.
- `AnthropicProviderClientImpl`: Anthropic Messages API with extended-thinking support.
- `LocalProviderClientImpl`: Ollama/LM Studio/llama.cpp — no API key required.
- `TraitsTools<AIAssistant_if>`: binds the abstraction to `AIAssistantDefaultImpl`.

See `source/tools/ARCHITECTURE_tools.md` for the full 8-stage roadmap.

8.7 7. Current status by topic

- **Fitting**: interface defined; `FitterDefaultImpl` is functional for uniform/triangular/normal/exponential/erlang/beta/weibull with binary dataset loading and SSE-CDF comparison and is now the default `TraitsTools<Fitter_if>` binding (FITTER-3). `FitterDummyImpl` remains available as legacy placeholder.
- **Hypothesis testing**: functional baseline exists in `HypothesisTesterDefaultImpl1`, with known partial areas.
 - HYPTEST-1 alignment update: proportion-difference CI now follows the classical two-proportion formula, and one-population average/variance tests now compute p-values with Student-t/chi-square-coherent CDF paths.
 - HYPTEST-2 final alignment update: one-population proportion confidence intervals (with and without finite-population correction) now use the large-sample normal quantile formulation.
- **Simulation result datasets**: `SimulationResultsDatasetParser` reads text datasets and Genesys `Record` files, preserving replication ids and optional time columns so GUI analysis can expose pooled and per-replication scopes.
- **Probability distributions**: mathematical static base and inverse façade available, with internal numeric dependencies.
- **Numerical solvers**: legacy `Solver_if` + `SolverDefaultImpl1` remain the compatible baseline.

Chapter 9

Todo List

Member [AIAssistant_if::refreshKnowledgeBaseFromSimulator](#) (std::string *message=nullptr)=0

(Stage 2): Implement full plugin introspection via [SimulatorFacade](#).

Class [AIAssistantDefaultImpl](#)

annotations.

Class [GraphicalConnection](#)

Evaluate decoupling model metadata from this item to keep this class purely visual.

Member [GraphicalConnection::GraphicalConnection](#) ([GraphicalComponentPort](#) *sourceGraphicalPort, [GraphicalComponentPort](#) *destinationGraphicalPort, unsigned int portSourceConnection=0, unsigned int portDestinationConnection=0, QColor color=Qt::black, QGraphicsItem *parent=nullptr)

Extrair para método dedicado `_bindPorts()` para legibilidade.

Member [ModelGraphicsScene::notifyGraphicalModelChange](#) (GraphicalModelEvent::EventType event↔Type, GraphicalModelEvent::EventObjectType eventObjectType, QGraphicsItem *item)

Consider broadcasting to all attached views when multi-view editing is supported.

Member [ModelGraphicsView::notifySceneGraphicalModelEventHandler](#) (const [GraphicalModelEvent](#) &modelGraphicsEvent)

Replace callback with signal/slot to improve composability.

actualize property editor?

Member [SimulationController::ensureReady](#) (bool checkModel, bool modelChecked, const std::function< bool()> &runModelCheck, const std::function< bool()> &syncModelFromText) const

Replace callback-based contract by dedicated interfaces to improve testability.

Chapter 10

Namespace Index

10.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

generate_kernel_simulator_test_stubs	89
genesys_mcp	89
genesys_mcp.__main__	89
genesys_mcp.client	90
genesys_mcp.server	90
GuiCrashDiagnostics	
Crash diagnostics helpers for the Qt GUI process	94
QCP	95

Chapter 11

Hierarchical Index

11.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

AIAssistant_if	114
AIAssistantDefaultImpl	127
AIAssistantConfiguration	125
AIAssistantRequest	138
AIAssistantResponse	139
AIAuditLog	140
AIConversationConfiguration	141
AIConversationResponse	142
AIConversationService	142
AIProviderClient_if	142
HttpProviderClientBase	781
AnthropicProviderClientImpl	182
LocalProviderClientImpl	827
OpenAIProviderClientImpl	1059
AISecretStore	149
AnimationQueue	173
ApiRouter	186
testing::internal::AssertHelper	188
Assignment	195
AuditEntry	201
BacteriaColony::BacteriumState	216
BacteriaColony::BarrierSegment	219
Base	
yy::genesyspp_parser::basic_symbol< Base >	223
BaseMVController	222
yy::genesyspp_parser::basic_symbol< by_state >	223
testing::internal::TemplateSel< Tmpl >::Bind< T >	232
BioKineticLawExpression	237
BioParameterSensitivityEntry	254
BioReactionRateSample	266
BioReactionRateTimeCourse	266
BioSBMLBridge	274
BioSBMLExportResult	274
BioSBMLImportResult	275
BioSensitivityScan	276
BioSimulationAnalysis	283
BioSimulationResult	283
BioSimulationSample	285
BioSimulationSpeciesAmount	285
BioSpeciesDerivative	295

BioSteadyStateCheck	310
BioStoichiometryMatrix	310
BoundaryCondition	315
Boundary_Closed	312
Boundary_Fixed	313
yy::genesyspp_parser::by_kind	324
yy::genesyspp_parser::basic_symbol< by_kind >	223
yy::genesyspp_parser::symbol_type	2431
QCPAxisPainterPrivate::CachedLabel	326
QCPLabelPainterPrivate::CachedLabel	326
ModelLifecycleController::Callbacks	326
SimulationEventController::Callbacks	327
testing::internal::CartesianProductHolder< Gen >	334
Cell	335
CellularAutomata	363
CellularAutomataBase	367
CellularAutomata_1DTimed	364
CellularAutomata_Classic	365
CellularAutomataDemoBuilder	375
CellularAutomataDemoModel	376
CellularAutomataDemoSettings	376
CellularAutomataEvent	377
CellularAutomataStateColorMap	380
ChatMessage	382
testing::internal::CodeLocation	390
Collector_if	390
CollectorDatafile_if	393
CollectorDatafileDefaultImpl1	395
CollectorDefaultImpl1	398
GroProgramRuntime::ColonyMutation	400
GroProgramIrr::Command	401
Util::CommandResult	402
CppCompiler::CompilationResult	409
CompilerThread	409
ComponentItem	410
HypothesisTester_if::ConfidenceInterval	413
Connection	414
ConnectionChannel	414
ConnectionManager	415
testing::internal::ConstCharPtr	419
Optimizer_if::ConstraintDefinition	420
testing::internal::ConstRef< T >	420
testing::internal::ConstRef< T & >	420
testing::internal::ContainerPrinter	420
yy::genesyspp_parser::context	420
testing::internal::ConvertibleToIntegerPrinter	430
testing::internal::ConvertibleToStringViewPrinter	430
CppModelExporter	454
SimulatorSessionService::CreateSessionResult	465
QtBoolPropertyManagerPrivate::Data	465
QtDatePropertyManagerPrivate::Data	466
QtDateTimePropertyManagerPrivate::Data	466
QtDoublePropertyManagerPrivate::Data	466
QtEnumPropertyManagerPrivate::Data	467
QtFlagPropertyManagerPrivate::Data	467
QtIntPropertyManagerPrivate::Data	467
QtPointFPropertyManagerPrivate::Data	467

QtRectFPropertyManagerPrivate::Data	468
QtRectPropertyManagerPrivate::Data	468
QtSizeFPropertyManagerPrivate::Data	468
QtSizePropertyManagerPrivate::Data	468
QStringPropertyManagerPrivate::Data	469
QTimePropertyManagerPrivate::Data	469
DataAnalyser_if	469
DataDefinitionItem	473
DataDefinitionRelationItem	473
ModelInspectorController::DataDefinitionSelection	474
AnimationPlot::DataSeries	474
DataSet_if	475
Delay::DEFAULT_VALUES	482
ModalModelDefault::DEFAULT_VALUES	482
Seize::DEFAULT_VALUES	482
SourceModelComponent::DEFAULT_VALUES	482
testing::internal::DefaultNameGenerator	485
DefaultNodeTransition	491
EFSMTransition	573
PetriTransition	1132
DialogUtilityController	515
Distribution_if	551
ContinuousDistribution_if	421
DiscreteDistribution_if	544
testing::internal::DoubleSequence< plus_one, T, sizeofT >	554
testing::internal::DoubleSequence< false, IndexSequence< I... >, sizeofT >	554
testing::internal::DoubleSequence< true, IndexSequence< I... >, sizeofT >	555
DrawingItem	555
EditCommandController	571
EditorFactoryPrivate< Editor >	572
EditorFactoryPrivate< QComboBox >	572
QtEnumEditorFactoryPrivate	1779
EditorFactoryPrivate< QKeySequenceEdit >	572
QtKeySequenceEditorFactoryPrivate	1811
EditorFactoryPrivate< QScrollBar >	572
QtScrollBarFactoryPrivate	1854
EditorFactoryPrivate< QSlider >	572
QtSliderFactoryPrivate	1873
EditorFactoryPrivate< QtBoolEdit >	572
QtCheckBoxFactoryPrivate	1734
EditorFactoryPrivate< QtCharEdit >	572
QtCharEditorFactoryPrivate	1728
EditorFactoryPrivate< QColorEditWidget >	572
QtColorEditorFactoryPrivate	1737
EditorFactoryPrivate< QtDateEdit >	572
QtDateEditFactoryPrivate	1753
EditorFactoryPrivate< QDateTimeEdit >	572
QtDateTimeEditFactoryPrivate	1763
EditorFactoryPrivate< QtDoubleEdit >	572
QtDoubleSpinBoxFactoryPrivate	1776
EditorFactoryPrivate< QFontEditWidget >	572
QtFontEditorFactoryPrivate	1792
EditorFactoryPrivate< QtIntEdit >	572
QtSpinBoxFactoryPrivate	1877

EditorFactoryPrivate< QStringEdit >	572
QtLineEditFactoryPrivate	1818
EditorFactoryPrivate< QtTimeEdit >	572
QtTimeEditFactoryPrivate	1886
testing::internal::ElemFromList< N, T >	574
testing::internal::ElemFromList< I, T... >	574
testing::internal::ElemFromListImpl< typename >	575
testing::internal::ElemFromListImpl< IndexSequence< I... > >	575
PersistenceRecord::Entry	602
testing::Environment	602
Event	610
InternalEvent	797
Exact	612
Exception	
genesys_mcp.client.GenesysClientError	687
CompartmentExchangeComponent::ExchangeRule	617
GroProgramRuntime::ExecutionResult	618
ExperimentManager	624
ExperimentManager_if	625
ExperimentManagerDefaultImpl1	627
ExpressionCategory	629
ExpressionParameter	629
ExpressionTemplate	629
FactorialDesign	630
testing::internal::FallbackPrinter	643
std::false_type	
testing::internal::IsRecursiveContainerImpl< C, false >	802
testing::internal::is_proxy_type_list< typename >	799
testing::internal::FindFirstPrinter< T, E, Printer, Printers >	649
testing::internal::FindFirstPrinter< T, decltype(Printer::PrintValue(std::declval< const T & >(), nullptr)), Printer, Printers... >	649
Fitter_if	650
FitterDefaultImpl	652
FitterDummyImpl	654
testing::internal::FlatTupleBase< Derived, Idx >	658
testing::internal::FlatTupleBase< FlatTuple< T... >, MakeIndexSequence< sizeof...(T)>::type >	658
testing::internal::FlatTuple< Ts... >	657
testing::internal::FlatTuple< T >	657
testing::internal::FlatTupleConstructTag	659
testing::internal::FlatTupleElemBase< Derived, I >	659
testing::internal::FlatTupleElemBase< FlatTuple< T... >, I >	660
testing::internal::FlatTupleElemBase< FlatTuple< T... >, Idx >	659
testing::internal::FlatTupleBase< FlatTuple< T... >, IndexSequence< Idx... > >	658
testing::internal::FloatingPoint< RawType >	660
testing::internal::FormatForComparison< ToPrint, OtherOperand >	661
testing::internal::FormatForComparison< ToPrint[N], OtherOperand >	661
testing::internal::FunctionPointerPrinter	683
ParserManager::GenerateNewParserResult	683
testing::internal::GenerateTypeList< T >	684
GenesysApplication_if	685
BaseGenesysTerminalApplication	219
AirportSecurityExample	146
AirportSecurityExampleExtended	148
Airport_Extended1	145
AnElectronicAssemblyAndTestSystem	162
Assembly_Line	187

Banking_Transactions	217
Book_Cap02_Example01	310
BufferFIFO	323
ContinuousModel	422
Example_BasicOrderShipping	613
Example_PortModel	614
Example_PublicTransport	616
FullSimulationOfComplexModel	682
GenesysShell	695
Half_Adder	773
Loja01	845
OperatingSystem02	1063
OperatingSystem03	1064
Rectifier	1943
Smart_AddingResource	2156
Smart_AlternatingEntityCreation	2157
Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime	2159
Smart_ArrivalsEntityTypeVsAttribute	2161
Smart_AssignExample	2163
Smart_AssignWriteSeizes	2165
Smart_Attribute_Variable	2166
Smart_AutomaticStatisticsCollection	2168
Smart_BacteriaColony_GRO	2170
Smart_BasicModeling	2172
Smart_BatchAndSeparate	2173
Smart_BatchSeparate	2175
Smart_BioKineticLawRegulation	2177
Smart_BioNetworkAnalysis	2178
Smart_BioReversibleKineticLaw	2180
Smart_BioReversibleMassAction	2182
Smart_Buffer	2183
Smart_CellularAutomata	2185
Smart_CellularAutomata1	2187
Smart_Clone	2188
Smart_ContinuousFlowEntities	2190
Smart_CppForG	2192
Smart_Create	2193
Smart_DecideNWayByChance	2195
Smart_DefaultModalModel	2197
Smart_DefiningAttributesAsStrings	2198
Smart_DefiningControlLogic	2200
Smart_DefiningResourceCapacity	2202
Smart_Delay	2203
Smart_DelayBasedOnReplication	2205
Smart_Dummy	2207
Smart_EntitiesProcessedByPriority	2209
Smart_EvaluatingConditionsBeforeEnteringQueue	2211
Smart_Expression	2213
Smart_FSMState	2216
Smart_Failures	2215
Smart_GroColonyGrowth	2218
Smart_GroColonyLifecycle	2220
Smart_HoldSearchRemove	2221
Smart_InventoryAndHoldingCosts	2223
Smart_LSODE	2225
Smart_MarkovChain	2227
Smart_MaxArrivalsField	2228
Smart_ModalModel	2230

Smart_ModalModelDefaultNetwork	2232
Smart_ModalModelFSM	2233
Smart_ModalModelPetriNet	2235
Smart_ModelInfoModelSimulation	2237
Smart_ModelRunUntil1000Parts	2238
Smart_ModuleDisplayVariables	2240
Smart_ODE	2242
Smart_OnEvent	2243
Smart_OverlappingResources	2246
Smart_ParallelProcessingOfEntities	2248
Smart_Parser	2250
Smart_ParserModelFunctions	2252
Smart_PetriPlace	2253
Smart_PlacingEntitiesInQueueSets	2255
Smart_Plugin	2257
Smart_PriorityExample	2258
Smart_Process	2260
Smart_ProcessArena	2262
Smart_ProcessSet	2263
Smart_PythonForG	2265
Smart_R_Simulator	2266
Smart_Record	2269
Smart_Record_Arena	2270
Smart_RemovingAndReorderingEntitiesInAQueue	2272
Smart_ResourceCosting	2274
Smart_ResourceScheduleCosting	2276
Smart_ResourceSets	2277
Smart_RouteStation	2279
Smart_SeizeDelayRelease	2281
Smart_SeizeDelayReleaseMany	2282
Smart_SeizeDelayReleaseNoDataDefs	2284
Smart_SeizingMultipleSimultaneously	2286
Smart_SelectingRouteBasedOnProbability	2288
Smart_SelectingShorterQueue	2289
Smart_Sequence	2291
Smart_SimulationControlResponse	2293
Smart_SynchronizingParallelEntities	2294
Smart_ValueAdded	2296
Smart_WaitForSignal	2298
Smart_WaitScanCondition	2299
Smart_WaitSignal	2301
StartAIAssistant	2343
WcmBioNetworkBridge	2543
WcmCompartmentTransportWholeCellMvp	2544
WcmFluxBalanceWholeCellMvp	2546
WcmGeneExpression	2548
WcmLifecycleStress	2550
WcmMgenitaliumKarr2012	2552
WcmPathwayStressWholeCellMvp	2555
WcmSimpleSsa	2556
WcmWholeCellMvp	??
WcmYeastDidacticWholeCellMvp	??
WcmYeastEukaryoticWholeCellMvp	??
BaseGenesysWebApplication	221
GenesysShell_if	697
genesys_mcp.client.GenesysClient	686
GenesysKnowledgeBaseSummary	688
GenesysKnowledgeItem	688

genesyspp_driver	689
yy::genesyspp_parser	690
GenesysPropertyDescriptor	694
GenesysPropertyIntrospection	695
GeneticCircuitSimulationSummary	716
GraphicalAssociation	731
GraphicalAutomaticPositioningStrategy	731
GraphicalAutomaticPositioningStrategyFactory	732
GraphicalConnectionStyle	737
GraphicalContextMenuController	737
GraphicalDataDefinitionLayout	738
GraphicalModelBuilder	740
GraphicalModelEvent	749
GraphicalModelItemRenderContext	749
GraphicalModelItemRenderer	750
GraphicalModelItemRenderStrategy	750
GraphicalModelSerializer	751
GraphvizModelExporter	752
ModelGraphicsScene::GRID	754
GroProgramAst	759
GroProgramCompiler	760
GroProgramIr	760
GroProgramParser	761
GroProgramRuntime	761
GroProgramRuntimeState	762
GroupItem	762
testing::internal::GTestFlagSaver	763
testing::internal::GTestLog	763
testing::internal::GTestMutexLock	764
GuiActionContribution	764
GuiDockContribution	764
GuiExtensionManager	767
GuiExtensionPlugin	768
BacteriaColonyViewerGuiExtensionPlugin	210
BioNetworkAwareGuiExtensionPlugin	243
BioNetworkEditorGuiExtensionPlugin	245
BioNetworkExecutionGuiExtensionPlugin	246
BioParameterManagerGuiExtensionPlugin	252
BioReactionKineticsGuiExtensionPlugin	265
BioReactionStoichiometryGuiExtensionPlugin	267
BioSBMLInteroperabilityGuiExtensionPlugin	275
BioSimulationResultsGuiExtensionPlugin	284
BioSpeciesManagerGuiExtensionPlugin	296
CellularAutomataGuiExtensionPlugin	378
CoreGuiExtensionPlugin	431
GeneticCircuitGuiExtensionPlugin	703
GuiExtensionConfigurationGuiExtensionPlugin	766
ModelPluginInspectorGuiExtensionPlugin	991
GuiExtensionPluginAutoRegistrar	769
GuiExtensionPluginCatalog	769
GuiExtensionPresentationMetadata	769
GuiExtensionRegistry	769
GuiExtensionRuntimeContext	770
GuiScopeTrace	770
GuiThemeManager	772
GuiWindowContribution	773
testing::internal::HasDebugStringAndShortDebugString< T >	776
HttpRequest	785

HttpResponse	786
HypothesisTester_if	787
HypothesisTesterDefaultImpl1	788
testing::internal::Ignore< size_t >	795
testing::internal::IgnoredValue	795
testing::internal::IndexSequence< Is >	795
testing::internal::IndexSequence<>	795
testing::internal::MakeIndexSequenceImpl< 0 >	856
std::integral_constant	
std::tuple_size< testing::internal::FlatTuple< Ts... > >	2475
testing::internal::IsHashTable< T >	800
testing::internal::IsRecursiveContainerImpl< C, bool >	801
testing::internal::IsRecursiveContainerImpl< C, true >	802
IssueReportDraft	803
IssueReportRelayService	803
IssueReportRuntimeContext	804
IssueReportSubmissionResult	804
Json	804
KernelAccessorProbe	808
QCPLabelPainterPrivate::LabelData	813
Lattice	813
LicenceManager	821
List< T >	823
ListObservable< T >	825
List< Assignment * >	823
List< ConstraintDefinition >	823
List< Counter * >	823
List< DefaultNode * >	823
List< DefaultNodeTransition * >	823
List< double >	823
List< Entity * >	823
List< Event * >	823
List< Failure * >	823
List< Model * >	823
List< ModelComponent * >	823
List< ModelDataDefinition * >	823
List< modelEventHandler >	823
List< modelEventHandlerMethod >	823
List< ObjectiveDefinition >	823
List< ODEfunction * >	823
List< PairSignalDataEventHandler * >	823
List< PickableStationItem * >	823
List< Plugin * >	823
List< PluginLoadIssue >	823
List< Queue * >	823
List< Resource * >	823
List< SchedulableItem * >	823
List< SeizableItem * >	823
List< SequenceStep * >	823
List< ShellCommand * >	823
List< SimulationControl * >	823
List< simulationEventHandler >	823
List< simulationEventHandlerMethod >	823
List< SimulationExperiment * >	823
List< SimulationResponse * >	823
List< SolutionSummary >	823
List< SortedResourceEventHandler * >	823

List< SparseValueStore * >	823
List< StatisticsCollector * >	823
List< std::string >	823
List< traceErrorListener >	823
List< traceErrorListenerMethod >	823
List< traceListener >	823
List< traceListenerMethod >	823
List< traceSimulationListener >	823
List< traceSimulationListenerMethod >	823
List< void * >	823
List< Waiting * >	823
LocalRule	831
LocalRule_CopyCurrentState	833
LocalRule_Elementary	834
LocalRule_FlorestalFire	835
LocalRule_GameOfLife	837
LocalRule_Growty	838
LocalRule_HppLatticeGas	839
LocalRule_SandRock_Student	841
LocalRule_SandRock_Student	841
LocalRule_ShowNeighbordhood	842
yy::location	843
testing::internal::internal_stream_operator_without_lexical_name_lookup::LookupBlocker	846
LSODEInitialState	853
LSODESample	853
testing::internal::MarkAsIgnored	857
testing::Message	871
MetabolicFluxBalanceSolver	878
generate_kernel_simulator_test_stubs.MethodEntry	902
Model	921
ModelChecker_if	930
ModelCheckerDefaultImpl1	932
ModelComponentManager	943
ModelDataDefinitionAssociations	960
ModelDataManager	961
ModelEvent	966
SimulatorSessionService::ModelImportResult	978
ModelInfo	979
SimulatorSessionService::ModelInfoResult	982
ModelInspectorController	982
ModelLanguageSynchronizer	983
ModelLifecycleController	984
ModelManager	985
SimulatorSessionService::ModelPersistenceResult	991
ModelSimulation	993
GroProgramRuntime::MotionMutation	999
testing::internal::Mutex	1000
dialogBreakpoint::MVCResult	1001
GroProgramAst::NamedProgram	1001
GroProgramIr::NamedProgramDefinition	1001
testing::internal::NameGeneratorSelector< Provided >	1001
testing::internal::NativeArray< Element >	1008
Neighborhood	1010
Neighborhood_Center	1011
Neighborhood_Moore	1013
Neighborhood_VonNeumann	1015
ParserManager::NewParser	1016

testing::internal::None	1023
obj_t	1036
Optimizer_if::ObjectiveDefinition	1036
ODEfunction	1045
OdeSolver_if	1051
DormandPrince54OdeSolver	553
RungeKutta4OdeSolver	1997
OdeSolverFactory	1052
OdeSystem_if	1052
HarmonicOscillator	775
DiffusionMethodOfLinesSystem	529
HarmonicOscillator	775
MassActionOdeSystem	863
OnEventManager	1057
Optimizer_if::OptimizationSettings	1066
Optimizer_if	1066
OptimizerDefaultImpl1	1072
testing::internal::OsStackTraceGetterInterface	1087
testing::internal::OsStackTraceGetter	1086
testing::internal::ParameterizedTestSuiteInfoBase	1090
testing::internal::ParameterizedTestSuiteInfo< TestSuite >	1089
testing::internal::ParameterizedTestSuiteRegistry	1091
testing::internal::ParamGenerator< T >	1091
testing::internal::ParamGeneratorInterface< T >	1092
testing::internal::RangeGenerator< T, IncrementT >	1918
testing::internal::ValuesInIteratorRangeGenerator< T >	2508
testing::internal::ParamGeneratorInterface< ParamType >	1092
testing::internal::ParamGeneratorInterface<::std::tuple< T... > >	1092
testing::internal::CartesianProductGenerator< T >	333
testing::internal::ParamIterator< T >	1092
testing::internal::ParamIteratorInterface< T >	1093
testing::internal::ParamIteratorInterface< ParamType >	1093
Parser_if	1093
ParserDefaultImpl1	1102
ParserDefaultImpl2	1104
ParserAttributeEventInjector	1095
ParserAttributeProcessObserver	1101
ParserChangesInformation	1101
ParserManager	1108
Persistence_if	1116
FakeModelPersistence	638
FakeModelPersistenceB	640
PersistenceDefaultImpl2	1119
PersistencePartialLoadImpl1	1121
PersistenceRecord	1122
PersistentObject_base	1124
ModelDataDefinition	946
AISupport	151
Attribute	195
ParserAttributeInternalEventOwner	1096
Variable	2509
BacteriaSignalGrid	211
BioCompartment	232
BioNetwork	238
BioParameter	247

BioReaction	254
BioSimulatorRunner	286
BioSpecies	291
Counter	432
CppCompiler	442
DiffusionField	524
DummyElement	567
Entity	584
EntityGroup	592
EntityType	597
Failure	633
File	644
Formula	662
GeneticCircuit	698
GeneticCircuitPart	705
GeneticRegulation	722
GroProgram	754
Label	808
MetabolicNetwork	878
MetabolicReaction	883
ModelComponent	934
AIAssistant	108
Access	101
Assign	188
BacteriaColony	202
Batch	225
BioRunnerCommand	268
BioSimulate	276
BioStateProjectionComponent	297
BioSteadyState	303
Buffer	316
CellCycleCheckpointComponent	336
CellDivisionEvent	343
CellFateDecisionComponent	350
CellGrowthComponent	356
CellularAutomataComp	369
Clone	383
CompartmentExchangeComponent	402
ContinuousSystemComponent	424
RecordingContinuousSystemComponent	1931
RecordingContinuousSystemComponent	1931
CppForG	448
Decide	475
DefaultNode	485
FSMState	669
PetriPlace	1126
Delay	493
DelayProbe	500
DelayProbe	500
DiffEquations	517
DiffusionSimulate	531
DropOff	555
DummyComponent	561
Enter	578
EukaryoticCellCycleComponent	602
Exit	618
FtsZPolymerizationComponent	675
GeneticCircuitSimulate	709

GeneticExpressionStep	716
LSODE	846
RecordingLSODE	1937
Leave	815
MarkovChain	858
Match	865
MetabolicFluxBalance	872
MetabolicStateProjectionComponent	888
MetabolicSubmodelComponent	895
ModalModelDefault	902
ModalModelFSM	910
ModalModelPetriNet	916
PathwayStressResponseComponent	1110
PickStation	1134
Process	1175
PythonForG	1191
RSimulator	1985
Record	1924
Release	1944
Remove	1951
PickUp	1141
ResourceAllocationComponent	1970
Route	1978
SPICECircuit	2320
AND	156
Capacitor	327
Diode	538
NAND	1002
NMOS	1017
NOR	1023
NOT	1030
OR	1080
PMOS	1162
Resistor	1958
Vpulse	2516
Vsine	2522
Vsource	2528
XNOR	??
XOR	??
SPICENode	2326
Search	2016
Seize	2024
Separate	2031
Signal	2056
SinkModelComponent	2149
Dispose	545
SourceModelComponent	2309
Create	457
Start	2337
StochasticReactionComponent	2379
StochasticTranscription	2390
StochasticTranslation	2397
Stop	2405
Store	2416
Submodel	2423
Unstore	2489
Wait	2534
Write	??

MolecularSpecies	994
ODESolver	1046
OLD_ODEelement	1053
OctaveRunner	1040
PythonRuntime	1197
Queue	1912
RSimulatorRunner	1991
Resource	1964
SPICERunner	2332
Schedule	2011
Sequence	2037
Set	2050
SignalData	2062
Station	2349
StationTestProbe	2354
StatisticsCollector	2362
StochasticReactionRule	2385
Storage	2411
WholeCellState	??
SequenceStep	2043
PickableStationItem	1133
Plugin	1147
PluginCatalogController	1150
PluginConnector_if	1151
PluginConnectorDummyImpl1	1153
PluginConnectorStaticImpl1	1155
PluginInformation	1158
PluginInsertionOptions	1159
PluginLoadIssue	1159
PluginManager	1160
testing::internal::PointerPrinter	1168
GroProgramRuntime::PopulationMutation	1168
yy::position	1168
testing::PrintToStringParamName	1169
ProbabilityDistributionBase	1172
ProbabilityDistribution	1169
MetabolicFluxBalanceSolver::Problem	1175
BioStateProjectionComponent::Projection	1183
MetabolicStateProjectionComponent::Projection	1183
PropertyEditorController	1184
PropertyEditorGenesys	1185
PropertyGenesysBase	1187
PropertyGenesys	1185
SimulationResponse	2127
SimulationControl	2071
SimulationControlBool	2074
SimulationControlDouble	2077
SimulationControlGeneric< T >	2080
SimulationControlGenericClass< T, M, C >	2083
SimulationControlGenericClassNotDC< T, M, C >	2087
SimulationControlGenericEnum< T, E >	2091
SimulationControlGenericList< T, M, C >	2094
SimulationControlGenericListPointer< T, M, C >	2098
SimulationControlInt	2102
SimulationControlSourceCodeString	2107
SimulationControlString	2110
SimulationControlTimeUnit	2113

SimulationControlUInt	2116
SimulationControlUShort	2119
TestSimulationControl	2451
SimulationResponseDouble	2129
TestSimulationResponse	2454
testing::internal::ProtobufPrinter	1189
ProviderRequest	1189
ProviderResponse	1189
testing::internal::ProxyTypeList< Ts >	1190
QCPAbstractPaintBuffer	1216
QCPPaintBufferPixmap	1546
QCPAxisPainterPrivate	1259
QCPAxisTicker	1276
QCPAxisTickerDateTime	1280
QCPAxisTickerFixed	1285
QCPAxisTickerLog	1288
QCPAxisTickerPi	1291
QCPAxisTickerText	1294
QCPAxisTickerTime	1298
QCPBarsData	1312
QCPColorGradient	1318
QCPColorMapData	1333
QCPCurveData	1364
QCPDataContainer< DataType >	1366
QCPDataRange	1373
QCPDataSelection	1377
QCPErrorBarsData	1393
QCPFinancialData	1405
QCPGraphData	1416
QCPItemAnchor	1422
QCPItemPosition	1451
QCPLabelPainterPrivate	1482
QCPLineEnding	1541
QCPPlottableInterface1D	1552
QCPAbstractPlottable1D< QCPBarsData >	1231
QCPBars	1302
QCPAbstractPlottable1D< QCPCurveData >	1231
QCPCurve	1354
QCPAbstractPlottable1D< QCPFinancialData >	1231
QCPFinancial	1394
QCPAbstractPlottable1D< QCPGraphData >	1231
QCPGraph	1407
QCPAbstractPlottable1D< QCPStatisticalBoxData >	1231
QCPStatisticalBox	1642
QCPAbstractPlottable1D< DataType >	1231
QCPErrorBars	1382
QCPRange	1618
QCPScatterStyle	1624
QCPSelectionDecorator	1630
QCPSelectionDecoratorBracket	1633
QCPStatisticalBoxData	1652
QCPVector2D	1663
QDialog	
DialogFind	507
DialogModelInformation	508
DialogPluginManager	509

DialogReportIssue	510
DialogSelectCounter	511
DialogSelectVariable	512
DialogSimulationConfigure	513
DialogSystemPreferences	514
DialogTimerConfigure	515
ExpressionBuilder	628
GuiExtensionConfigurationDialog	765
WebWorkerDialog	??
dialogBreakpoint	506
QGraphicsLineItem	
GraphicalDiagramConnection	739
QGraphicsObject	
GraphicalAnimateExpression	730
GraphicalComponentPort	733
GraphicalConnection	734
GraphicalModelDataDefinition	747
GraphicalModelComponent	742
QGraphicsPixmapItem	
GraphicalImageAnimation	740
QGraphicsRectItem	
AnimationCounter	165
AnimationPlaceholder	170
AnimationAttribute	164
AnimationEntity	166
AnimationEvent	168
AnimationExpression	169
AnimationPlot	171
AnimationQueueDisplay	173
AnimationResource	175
AnimationStation	176
AnimationStatistics	177
AnimationTimer	178
AnimationVariable	181
QGraphicsScene	
ModelGraphicsScene	966
QGraphicsView	
ModelGraphicsView	975
QItemDelegate	
QtPropertyEditorDelegate	1839
QLineEdit	
FindLineEdit	649
QMainWindow	
AIAssistantWindow	140
CellularAutomataViewerWindow	382
DataAnalyzerWindow	470
MainWindow	853
OptimizerWindow	1079
QObject	
CellularAutomataViewerController	380
ComboBoxEnum	400
DataComponentEditor	471
DataComponentProperty	472
QCPBarsGroup	1314
QCPLayer	1484
QCPLayerable	1488
QCPAbstractItem	1202
QCPIItemBracket	1424

QCPItemCurve	1430
QCPItemEllipse	1435
QCPItemLine	1440
QCPItemPixmap	1445
QCPItemRect	1457
QCPItemStraightLine	1462
QCPItemText	1467
QCPItemTracer	1474
QCPAbstractPlottable	1218
QCPAbstractPlottable1D< QCPBarsData >	1231
QCPAbstractPlottable1D< QCPCurveData >	1231
QCPAbstractPlottable1D< QCPFinancialData >	1231
QCPAbstractPlottable1D< QCPGraphData >	1231
QCPAbstractPlottable1D< QCPStatisticalBoxData >	1231
QCPAbstractPlottable1D< DataType >	1231
QCPColorMap	1324
QCPErrors	1382
QCPAxis	1238
QCPGrid	1418
QCPLayoutElement	1502
QCPAbstractLegendItem	1209
QCPPlottableLegendItem	1556
QCPPolarLegendItem	1613
QCPAxisRect	1261
QCPColorScaleAxisRectPrivate	1348
QCPColorScale	1339
QCPLayout	1495
QCPLayoutGrid	1509
QCPLegend	1529
QCPLayoutInset	1521
QCPPolarAxisAngular	1561
QCPTextElement	1654
QCPPolarAxisRadial	1582
QCPPolarGraph	1601
QCPPolarGrid	1610
QCPSelectionRect	1637
QCPMarginGroup	1544
QtAbstractEditorFactoryBase	1702
QtAbstractEditorFactory< QtCharPropertyManager >	1698
QtCharEditorFactory	1726
QtAbstractEditorFactory< QtBoolPropertyManager >	1698
QtCheckBoxFactory	1732
QtAbstractEditorFactory< QtColorPropertyManager >	1698
QtColorEditorFactory	1735
QtAbstractEditorFactory< QtCursorPropertyManager >	1698
QtCursorEditorFactory	1744
QtAbstractEditorFactory< QtDatePropertyManager >	1698
QtDateEditFactory	1751
QtAbstractEditorFactory< QDateTimePropertyManager >	1698
QtDateTimeEditFactory	1761
QtAbstractEditorFactory< QtDoublePropertyManager >	1698
QtDoubleSpinBoxFactory	1774
QtAbstractEditorFactory< QtEnumPropertyManager >	1698
QtEnumEditorFactory	1777
QtAbstractEditorFactory< QtFontPropertyManager >	1698
QtFontEditorFactory	1790

QtAbstractEditorFactory< QtKeySequencePropertyManager >	1698
QtKeySequenceEditorFactory	1809
QtAbstractEditorFactory< QtStringPropertyManager >	1698
QtLineEditFactory	1816
QtAbstractEditorFactory< QtIntPropertyManager >	1698
QtScrollBarFactory	1852
QtSliderFactory	1871
QtSpinBoxFactory	1875
QtAbstractEditorFactory< QtTimePropertyManager >	1698
QtTimeEditFactory	1884
QtAbstractEditorFactory< QtVariantPropertyManager >	1698
QtVariantEditorFactory	1897
QtAbstractEditorFactory< PropertyManager >	1698
QtAbstractPropertyManager	1710
QtBoolPropertyManager	1716
QtCharPropertyManager	1729
QtColorPropertyManager	1740
QtCursorPropertyManager	1747
QtDatePropertyManager	1755
QtDateTimePropertyManager	1764
QtDoublePropertyManager	1768
QtEnumPropertyManager	1781
QtFlagPropertyManager	1785
QtFontPropertyManager	1794
QtGroupPropertyManager	1801
QtIntPropertyManager	1804
QtKeySequencePropertyManager	1813
QtLocalePropertyManager	1819
QtPointFPropertyManager	1824
QtPointPropertyManager	1829
QtRectFPropertyManager	1842
QtRectPropertyManager	1847
QtSizeFPropertyManager	1856
QtSizePolicyPropertyManager	1862
QtSizePropertyManager	1866
QtStringPropertyManager	1879
QtTimePropertyManager	1887
QtVariantPropertyManager	1903
QtMetaEnumWrapper	1823
QPainter	
QCPPainter	1548
QPlainTextEdit	
CodeEditor	389
QtAbstractPropertyBrowserPrivate	1709
QtAbstractPropertyManagerPrivate	1715
QtBoolPropertyManagerPrivate	1719
QtBrowserItem	1719
QtBrowserItemPrivate	1720
QtButtonPropertyBrowserPrivate	1724
QtCharPropertyManagerPrivate	1731
QtColorPropertyManagerPrivate	1743
QtCursorDatabase	1744
QtCursorEditorFactoryPrivate	1746
QtCursorPropertyManagerPrivate	1750
QtDatePropertyManagerPrivate	1759
QtDateTimePropertyManagerPrivate	1766
QtDoublePropertyManagerPrivate	1773
QtEnumPropertyManagerPrivate	1785

QtEnumPropertyType	1785
QtFlagPropertyManagerPrivate	1789
QtFlagPropertyType	1789
QtFontPropertyManagerPrivate	1797
QtGroupBoxPropertyBrowserPrivate	1800
QtGroupPropertyType	1803
QtIntPropertyManagerPrivate	1808
QtKeySequencePropertyManagerPrivate	1815
QtLocalePropertyManagerPrivate	1822
QtMetaEnumProvider	1822
QtPointFPropertyManagerPrivate	1828
QtPointPropertyManagerPrivate	1832
QtProperty	1833
QtVariantProperty	1900
QtPropertyBrowserUtils	1838
QtPropertyPrivate	1841
QtRectFPropertyManagerPrivate	1846
QtRectPropertyManagerPrivate	1851
QTreeWidgetItem	
PropertyEditor	1183
QtPropertyEditorView	1840
QtSizeFPropertyManagerPrivate	1861
QtSizePolicyPropertyManagerPrivate	1865
QtSizePropertyManagerPrivate	1870
QtStringPropertyManagerPrivate	1882
QtTimePropertyManagerPrivate	1889
QtTreePropertyBrowserPrivate	1896
QtVariantEditorFactoryPrivate	1899
QtVariantPropertyManagerPrivate	1909
QtVariantPropertyPrivate	1911
Quadrature_if	1911
QueueableItem	1917
QUndoCommand	
AddUndoCommand	107
DeleteUndoCommand	505
GroupUndoCommand	763
MoveUndoCommand	1000
PasteUndoCommand	1109
UngroupUndoCommand	2478
QVariantAnimation	
AnimationTransition	179
QWidget	
CellularAutomataLatticeWidget	379
DataAnalyzerWindow::DoePreviewPlot	552
DataAnalyzerWindow::HistogramPreview	776
LineNumberArea	822
OptimizerWindow::ProgressPlot	1182
QCustomPlot	1668
QtAbstractPropertyBrowser	1704
QtButtonPropertyBrowser	1721
QtGroupBoxPropertyBrowser	1798
QtTreePropertyBrowser	1890
ObjectPropertyBrowser	1037
QtBoolEdit	1715
QtCharEdit	1725
QtColorEditWidget	1739
QtDateEdit	1750
QtDateTimeEdit	1759

QtDoubleEdit	1767
QtFontEditWidget	1793
QtIntEdit	1803
QStringEdit	1878
QTimeEdit	1882
testing::internal::Random	1918
testing::internal::RawBytesPrinter	1919
testing::internal::RE	1920
MassActionOdeSystem::Reaction	1920
testing::internal::RelationToSourceCopy	1944
testing::internal::RelationToSourceReference	1944
ResourceTestProbe	1976
GroProgramParser::Result	1977
Sampler_if::RNG_Parameters	1977
SamplerDefaultImpl1::DefaultImpl1RNG_Parameters	484
RootFinder_if	1978
std::runtime_error	
yy::genesyspp_parser::syntax_error	2434
Sample	1998
Sampler_if	1998
CountingSampler	438
SamplerDefaultImpl1	2003
ScenarioExperiment_if	2007
SceneToolController	2007
SchedulableItem	2011
testing::ScopedTrace	2016
SeizableItem	2023
Serializer	2044
CppSerializer	455
GenSerializer	727
JsonSerializer	805
XmlSerializer	??
SessionContext	2048
SessionManager	2048
GroProgramRuntime::SignalMutation	2067
SimpleHttpServer	2067
SimulatorSessionService::SimulationActionResult	2068
SimulationCommandController	2069
SimulatorSessionService::SimulationConfigInput	2070
SimulatorSessionService::SimulationConfigResult	2070
SimulationController	2105
SimulationEvent	2122
SimulationEventController	2123
SimulationExperiment	2124
SimulationReporter_if	2124
SimulationReporterDefaultImpl1	2125
SimulationResultsDataset	2131
SimulationResultsDatasetParser	2132
SimulationResultsObservation	2132
SimulationScenario	2132
SimulatorSessionService::SimulationStatusResult	2133
Simulator	2134
SimulatorFacade	2136
SimulatorSessionService::SimulatorInfoResult	2142
SimulatorSessionService	2142
yy::genesyspp_parser::stack< T, S >::slice	2155
WebWorkerRuntime::Snapshot	2302

MetabolicFluxBalanceSolver::Solution	2303
Optimizer_if::SolutionSummary	2303
Solver_if	2304
SolverDefaultImpl1	2305
SortFile	2308
SourceCodeString	2309
SparseValueStore	2319
MassActionOdeSystem::Species	2320
State	2345
CompositeState	410
HppLatticeGasState	777
RealState	1922
GroProgramAst::Statement	2346
StateSet	2346
CompositeStateSet	412
HppLatticeGasStateSet	779
IntegerStateSet	795
NaturalStateSet	1008
RealIntervalStateSet	1920
StateSet_Enumerable	2347
Statistics_if	2358
StatisticsDatafile_if	2367
StatisticsDatafileDefaultImpl1	2371
StatisticsDefaultImpl1	2376
BioReaction::StoichiometricTerm	2404
MassActionOdeSystem::StoichiometricTerm	2404
MetabolicReaction::StoichiometricTerm	2404
StochasticReactionRule::StoichiometricTerm	2405
testing::internal::internal_stream_operator_without_lexical_name_lookup::StreamPrinter	2422
testing::internal::String	2422
yy::genesyspp_parser::symbol_kind	2430
SystemCommandExecutor_if	2435
FakeSystemCommandExecutor	642
SystemShellCommandExecutor	2442
SystemCommandResult	2435
SystemDependency	2436
SystemDependencyCheckEntry	2436
SystemDependencyCheckResult	2437
SystemDependencyInstallEntry	2438
SystemDependencyInstallResult	2438
SystemDependencyResolver	2439
SystemPreferences	2439
T	
testing::internal::SuiteApiResolver< T >	2429
testing::internal::Templates< Head_, Tail_ >	2443
testing::internal::Templates< Head_ >	2443
testing::internal::TemplateSel< Tmpl >	2443
testing::Test	2443
ParserExpressionsTest	1107
testing::TestWithParam< T >	2456
testing::TestEventListener	2444
testing::EmptyTestEventListener	575
testing::TestEventListeners	2445
testing::internal::TestFactoryBase	2445
testing::internal::ParameterizedTestFactory< TestClass >	1088
testing::internal::TestFactoryImpl< TestClass >	2446

testing::TestInfo	2447
testing::internal::TestMetaFactoryBase< ParamType >	2448
testing::internal::TestMetaFactoryBase< TestSuite::ParamType >	2448
testing::internal::TestMetaFactory< TestSuite >	2447
testing::TestParamInfo< ParamType >	2448
TestPartResultReporterInterface	
testing::internal::DefaultGlobalTestPartResultReporter	483
testing::internal::DefaultPerThreadTestPartResultReporter	492
testing::TestProperty	2449
testing::internal::TestPropertyKeys	2449
HypothesisTester_if::TestResult	2449
testing::TestResult	2450
testing::internal::TestResultAccessor	2450
testing::TestSuite	2455
testing::internal::ThreadLocal< T >	2457
testing::internal::ThreadLocal< std::vector< testing::internal::TraceInfo > >	2457
testing::internal::ThreadLocal< TestPartResultReporterInterface * >	2457
QCPAxisPainterPrivate::TickLabelData	2457
Token	2458
yy::genesyspp_parser::token	2458
TokenService	2459
TraceConsoleController	2459
TraceEvent	2461
TraceErrorEvent	2460
TraceSimulationEvent	2466
TraceSimulationProcess	2466
testing::internal::TraceInfo	2461
TraceManager	2461
TraitsGUI< T >	2467
TraitsGUI< GAssociation >	2468
TraitsGUI< GComponentPort >	2468
TraitsGUI< GConnection >	2468
TraitsGUI< GMainWindow >	2469
TraitsGUI< GModelComponent >	2469
TraitsGUI< GModelDataDefinition >	2469
TraitsGUI< GModelGraphicPic >	2470
TraitsGUI< GScene >	2470
TraitsGUI< GView >	2470
TraitsKernel< T >	2470
TraitsKernel< Collector_if >	2471
TraitsKernel< Model >	2471
TraitsKernel< ModelChecker_if >	2471
TraitsKernel< ModelComponent >	2471
TraitsKernel< ModelDataDefinition >	2471
TraitsKernel< Parser_if >	2472
TraitsKernel< Persistence_if >	2472
TraitsKernel< PluginConnector_if >	2472
TraitsKernel< Sampler_if >	2472
TraitsKernel< SimulationReporter_if >	2472
TraitsKernel< Simulator >	2473
TraitsKernel< Statistics_if >	2473
TraitsKernel< StatisticsDatafile_if >	2473
TraitsTools< T >	2473
TraitsTools< AIAssistant_if >	2474
TraitsTools< Fitter_if >	2474
TraitsTools< HypothesisTester_if >	2474
TraitsTools< Solver_if >	2474
std::true_type	

testing::internal::is_proxy_type_list< ProxyTypeList< Ts... > >	800
testing::internal::TrueWithString	2475
DoubleSequence::type	
testing::internal::MakeIndexSequenceImpl< N >	855
IsRecursiveContainerImpl::type	
testing::internal::IsRecursiveContainer< C >	801
testing::internal::TypeIdHelper< T >	2476
testing::internal::TypeParameterizedTest< Fixture, TestSel, Types >	2476
testing::internal::TypeParameterizedTest< Fixture, TestSel, internal::None >	2476
testing::internal::TypeParameterizedTestSuite< Fixture, Tests, Types >	2477
testing::internal::TypeParameterizedTestSuite< Fixture, internal::None, Types >	2477
testing::internal::TypeParameterizedTestSuiteRegistry	2477
testing::internal::Types< Head_, Tail_ >	2477
testing::internal::Types< Head_ >	2477
testing::internal::TypeWithSize< size >	2478
testing::internal::TypeWithSize< 4 >	2478
testing::internal::TypeWithSize< 8 >	2478
testing::internal::TypeWithSize< sizeof(RawType)>	2478
testing::UnitTest	2479
testing::internal::UnitTestImpl	2480
testing::internal::UnitTestOptions	2482
testing::internal::UniversalPrinter< T >	2482
testing::internal::UniversalPrinter< const T >	2483
testing::internal::UniversalPrinter< T & >	2483
testing::internal::UniversalPrinter< T[N]>	2484
testing::internal::UniversalTersePrinter< T >	2484
testing::internal::UniversalTersePrinter< const char * >	2487
testing::internal::UniversalTersePrinter< char * >	2484
testing::internal::UniversalTersePrinter< const char16_t * >	2488
testing::internal::UniversalTersePrinter< char16_t * >	2485
testing::internal::UniversalTersePrinter< const char32_t * >	2488
testing::internal::UniversalTersePrinter< char32_t * >	2486
testing::internal::UniversalTersePrinter< T & >	2489
testing::internal::UniversalTersePrinter< T[N]>	2489
testing::internal::UniversalTersePrinter< wchar_t * >	2489
Util	2495
yy::genesyspp_parser::value_type	2506
testing::internal::ValueArray< Ts >	2507
Waiting	2541
WaitingResource	2542
WebWorkerRuntime	??
WholeCellParameterReader	??
QtButtonPropertyBrowserPrivate::WidgetItem	??
QtGroupBoxPropertyBrowserPrivate::WidgetItem	??
testing::WithParamInterface< T >	??
testing::TestWithParam< T >	2456
SimulatorSessionService::WorkerCapabilitiesResult	??
SimulatorSessionService::WorkerInfoResult	??
WorkerJob	??
SimulatorSessionService::WorkerJobCreationResult	??
SimulatorSessionService::WorkerJobInfoResult	??
WorkerJobManager	??
SimulatorSessionService::WorkerJobQueryResult	??
SimulatorSessionService::WorkerJobResultInfo	??
SimulatorSessionService::WorkerJobResultQueryResult	??
SimulatorSessionService::WorkerJobRunResult	??
WorkerJobTerminalResult	??

yy_buffer_state	??
yy_trans_info	??

Chapter 12

Class Index

12.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Access	101
AddUndoCommand	107
AIAssistant	108
AIAssistant_if	
Abstract interface for the GenESyS AI Assistant tool	114
AIAssistantConfiguration	
Full configuration record for the AI assistant	125
AIAssistantDefaultImpl	
Initial concrete scaffold for the GenESyS AI Assistant	127
AIAssistantRequest	
Input to a single AI assistant request	138
AIAssistantResponse	
Output produced by a single AI assistant operation	139
AIAssistantWindow	140
AIAuditLog	
Append-only audit log for AI assistant operations	140
AIConversationConfiguration	141
AIConversationResponse	142
AIConversationService	142
AIProviderClient_if	
Abstract interface for AI provider HTTP clients	142
Airport_Extended1	145
AirportSecurityExample	146
AirportSecurityExampleExtended	148
AISecretStore	
Static helper for storing API keys in the OS keyring	149
AISupport	151
AND	156
AnElectronicAssemblyAndTestSystem	162
AnimationAttribute	164
AnimationCounter	165
AnimationEntity	166
AnimationEvent	168
AnimationExpression	169
AnimationPlaceholder	170
AnimationPlot	171
AnimationQueue	173
AnimationQueueDisplay	173
AnimationResource	175
AnimationStation	176

AnimationStatistics	177
AnimationTimer	178
AnimationTransition	179
AnimationVariable	181
AnthropicProviderClientImpl	
AI provider client for the Anthropic Messages API	182
ApiRouter	
Routes HTTP requests to Web API operations	186
Assembly_Line	187
testing::internal::AssertHelper	188
Assign	188
Assignment	195
Attribute	195
AuditEntry	
One entry in the AI assistant audit log	201
BacteriaColony	
First GenESyS component for Gro-inspired bacteria colony simulation	202
BacteriaColonyViewerGuiExtensionPlugin	210
BacteriaSignalGrid	
Reusable discrete signal-field definition for bacteria colonies	211
BacteriaColony::BacteriumState	216
Banking_Transactions	217
BacteriaColony::BarrierSegment	219
BaseGenesysTerminalApplication	219
BaseGenesysWebApplication	
Entry point for the standalone Worker application flavor of Genesys	221
BaseMVController	
Compatibility base class that binds the Qt UI and the simulator model	222
yy::genesyspp_parser::basic_symbol< Base >	223
Batch	225
testing::internal::TemplateSel< Tmpl >::Bind< T >	232
BioCompartment	
Defines a biological compartment for biochemical and whole-cell models	232
BioKineticLawExpression	237
BioNetwork	238
BioNetworkAwareGuiExtensionPlugin	243
BioNetworkEditorGuiExtensionPlugin	245
BioNetworkExecutionGuiExtensionPlugin	246
BioParameter	247
BioParameterManagerGuiExtensionPlugin	252
BioParameterSensitivityEntry	254
BioReaction	254
BioReactionKineticsGuiExtensionPlugin	265
BioReactionRateSample	266
BioReactionRateTimeCourse	266
BioReactionStoichiometryGuiExtensionPlugin	267
BioRunnerCommand	
Component wrapper over BioSimulatorRunner command execution	268
BioSBMLBridge	274
BioSBMLExportResult	274
BioSBMLImportResult	275
BioSBMLInteroperabilityGuiExtensionPlugin	275
BioSensitivityScan	276
BioSimulate	
Component that triggers deterministic BioNetwork simulation	276
BioSimulationAnalysis	283
BioSimulationResult	283
BioSimulationResultsGuiExtensionPlugin	284

BioSimulationSample	285
BioSimulationSpeciesAmount	285
BioSimulatorRunner	286
BioSpecies	291
BioSpeciesDerivative	295
BioSpeciesManagerGuiExtensionPlugin	296
BioStateProjectionComponent	
Projects selected BioSpecies values into a WholeCellState	297
BioSteadyState	
Component that checks steady-state diagnostics over BioNetwork results	303
BioSteadyStateCheck	310
BioStoichiometryMatrix	310
Book_Cap02_Example01	310
Boundary_Closed	312
Boundary_Fixed	313
BoundaryCondition	315
Buffer	316
BufferFIFO	323
yy::genesyspp_parser::by_kind	
Type access provider for token (enum) based symbols	324
QCPAxisPainterPrivate::CachedLabel	326
QCPLabelPainterPrivate::CachedLabel	326
ModelLifecycleController::Callbacks	326
SimulationEventController::Callbacks	327
Capacitor	327
testing::internal::CartesianProductGenerator< T >	333
testing::internal::CartesianProductHolder< Gen >	334
Cell	335
CellCycleCheckpointComponent	
Evaluates lifecycle checkpoints and advances whole-cell step time	336
CellDivisionEvent	343
CellFateDecisionComponent	
Routes entities according to the current whole-cell lifecycle state	350
CellGrowthComponent	356
CellularAutomata	363
CellularAutomata_1DTimed	
Complete uniform CA specific for unidimensional lattices where 2nd dimension is time (usefull for elementary CA)	364
CellularAutomata_Classic	
Complete uniform CA	365
CellularAutomataBase	367
CellularAutomataComp	369
CellularAutomataDemoBuilder	375
CellularAutomataDemoModel	376
CellularAutomataDemoSettings	376
CellularAutomataEvent	377
CellularAutomataGuiExtensionPlugin	378
CellularAutomataLatticeWidget	379
CellularAutomataStateColorMap	380
CellularAutomataViewerController	380
CellularAutomataViewerWindow	382
ChatMessage	
One message in a multi-turn conversation with an LLM	382
Clone	383
CodeEditor	389
testing::internal::CodeLocation	390
Collector_if	
Interface responsible for collecting observations of a stochastic variable	390

CollectorDatafile_if	
Collector interface specialized in file-backed value storage	393
CollectorDatafileDefaultImpl1	
Default file-backed implementation of CollectorDatafile_if	395
CollectorDefaultImpl1	
Default in-memory implementation of Collector_if	398
GroProgramRuntime::ColonyMutation	400
ComboBoxEnum	400
GroProgramlr::Command	401
Util::CommandResult	402
CompartmentExchangeComponent	
Moves metabolite proxies between whole-cell regions	402
CppCompiler::CompilationResult	409
CompilerThread	409
ComponentItem	410
CompositeState	410
CompositeStateSet	412
HypothesisTester_if::ConfidenceInterval	
Confidence interval [inferior, superior] and its half-width e0	413
Connection	414
ConnectionChannel	
Defines an input or output port that connects a ModelComponent to another	414
ConnectionManager	
Maintains outgoing connections from a component to downstream components	415
testing::internal::ConstCharPtr	419
Optimizer_if::ConstraintDefinition	
Feasibility restriction represented as an expression	420
testing::internal::ConstRef< T >	420
testing::internal::ConstRef< T & >	420
testing::internal::ContainerPrinter	420
yy::genesyspp_parser::context	420
ContinuousDistribution_if	
Interface for continuous distributions	421
ContinuousModel	422
ContinuousSystemComponent	
Component that advances a continuous ODE system when entities arrive	424
testing::internal::ConvertibleToIntegerPrinter	430
testing::internal::ConvertibleToStringViewPrinter	430
CoreGuiExtensionPlugin	431
Counter	
Model datum used to accumulate event counts (or weighted counts)	432
CountingSampler	438
CppCompiler	442
CppForG	448
CppModelExporter	
Service responsible for C++ code representation of the current simulation model	454
CppSerializer	455
Create	457
SimulatorSessionService::CreateSessionResult	
Contains session creation identifiers returned to API clients	465
QtBoolPropertyManagerPrivate::Data	465
QtDatePropertyManagerPrivate::Data	466
QtDateTimePropertyManagerPrivate::Data	466
QtDoublePropertyManagerPrivate::Data	466
QtEnumPropertyManagerPrivate::Data	467
QtFlagPropertyManagerPrivate::Data	467
QtIntPropertyManagerPrivate::Data	467
QtPointFPropertyManagerPrivate::Data	467

QtRectFPropertyManagerPrivate::Data	468
QtRectPropertyManagerPrivate::Data	468
QtSizeFPropertyManagerPrivate::Data	468
QtSizePropertyManagerPrivate::Data	468
QStringPropertyManagerPrivate::Data	469
QTimePropertyManagerPrivate::Data	469
DataAnalyser_if	
High-level façade for applied statistical analysis over datasets	469
DataAnalyzerWindow	470
DataComponentEditor	471
DataComponentProperty	472
DataDefinitionItem	473
DataDefinitionRelationItem	473
ModelInspectorController::DataDefinitionSelection	474
AnimationPlot::DataSeries	474
DataSet_if	
Abstract view of a numeric dataset	475
Decide	475
Delay::DEFAULT_VALUES	482
ModalModelDefault::DEFAULT_VALUES	
Attributes that should be loaded or saved with this component (Persistent Fields)	482
Seize::DEFAULT_VALUES	482
SourceModelComponent::DEFAULT_VALUES	482
testing::internal::DefaultGlobalTestPartResultReporter	483
SamplerDefaultImpl1::DefaultImpl1RNG_Parameters	
Parameter bundle for the default RNG algorithm	484
testing::internal::DefaultNameGenerator	485
DefaultNode	485
DefaultNodeTransition	491
testing::internal::DefaultPerThreadTestPartResultReporter	492
Delay	493
DelayProbe	500
DeleteUndoCommand	505
dialogBreakpoint	506
DialogFind	507
DialogModelInformation	508
DialogPluginManager	509
DialogReportIssue	
Modal form used to collect a user-facing issue report before relay submission	510
DialogSelectCounter	511
DialogSelectVariable	512
DialogSimulationConfigure	513
DialogSystemPreferences	514
DialogTimerConfigure	515
DialogUtilityController	
Controller that orchestrates auxiliary dialogs and utility actions	515
DiffEquations	517
DiffusionField	524
DiffusionMethodOfLinesSystem	529
DiffusionSimulate	
Component that triggers a DiffusionField solve when entities arrive	531
Diode	538
DiscreteDistribution_if	
Interface for discrete distributions	544
Dispose	545
Distribution_if	
Base abstraction for a theoretical probability distribution	551
DataAnalyzerWindow::DoePreviewPlot	552

DormandPrince54OdeSolver	553
testing::internal::DoubleSequence< plus_one, T, sizeofT >	554
testing::internal::DoubleSequence< false, IndexSequence< I... >, sizeofT >	554
testing::internal::DoubleSequence< true, IndexSequence< I... >, sizeofT >	555
DrawingItem	555
DropOff	555
DummyComponent	561
DummyElement	567
EditCommandController	
Controller for edit/clipboard/grouping command orchestration in the scene	571
EditorFactoryPrivate< Editor >	572
EFSMTransition	573
testing::internal::ElemFromList< N, T >	574
testing::internal::ElemFromListImpl< typename >	575
testing::internal::ElemFromListImpl< IndexSequence< I... > >	575
testing::EmptyTestEventListener	575
Enter	578
Entity	584
EntityGroup	592
EntityType	597
PersistenceRecord::Entry	
"Raw" entry, users should treat this as an opaque type since payloads could be in any format	602
testing::Environment	602
EukaryoticCellCycleComponent	
Coordinates a didactic eukaryotic cell cycle over a shared WholeCellState	602
Event	610
Exact	612
Example_BasicOrderShipping	613
Example_PortModel	614
Example_PublicTransport	616
CompartmentExchangeComponent::ExchangeRule	617
GroProgramRuntime::ExecutionResult	618
Exit	618
ExperimentManager	
Manages higher-level simulation experiments	624
ExperimentManager_if	
Interface that coordinates execution of scenario-based simulation experiments	625
ExperimentManagerDefaultImpl1	627
ExpressionBuilder	628
ExpressionCategory	629
ExpressionParameter	629
ExpressionTemplate	629
FactorialDesign	630
Failure	633
FakeModelPersistence	638
FakeModelPersistenceB	640
FakeSystemCommandExecutor	642
testing::internal::FallbackPrinter	643
File	644
testing::internal::FindFirstPrinter< T, E, Printer, Printers >	649
testing::internal::FindFirstPrinter< T, decltype(Printer::PrintValue(std::declval< const T & >(), nullptr)), Printer, Printers... >	649
FindLineEdit	649
Fitter_if	
Interface for fitting theoretical distributions to real datasets	650
FitterDefaultImpl	
Functional implementation of Fitter_if consolidated through FITTER-2	652

FitterDummyImpl	
Placeholder implementation of Fitter_if	654
testing::internal::FlatTuple< T >	657
testing::internal::FlatTupleBase< Derived, Idx >	658
testing::internal::FlatTupleBase< FlatTuple< T... >, IndexSequence< Idx... > >	658
testing::internal::FlatTupleConstructTag	659
testing::internal::FlatTupleElemBase< Derived, I >	659
testing::internal::FlatTupleElemBase< FlatTuple< T... >, I >	660
testing::internal::FloatingPoint< RawType >	660
testing::internal::FormatForComparison< ToPrint, OtherOperand >	661
testing::internal::FormatForComparison< ToPrint[N], OtherOperand >	661
Formula	662
FSMState	669
FtsZPolymerizationComponent	675
FullSimulationOfComplexModel	682
testing::internal::FunctionPointerPrinter	683
ParserManager::GenerateNewParserResult	
Result bundle for parser generation requests	683
testing::internal::GenerateTypeList< T >	684
GenesysApplication_if	685
genesys_mcp.client.GenesysClient	686
genesys_mcp.client.GenesysClientError	687
GenesysKnowledgeBaseSummary	
Aggregate summary of the current knowledge base state	688
GenesysKnowledgeItem	
One entry in the GenESyS knowledge base used to inform the AI assistant	688
genesyspp_driver	689
yy::genesyspp_parser	
A Bison parser	690
GenesysPropertyDescriptor	694
GenesysPropertyIntrospection	695
GenesysShell	695
GenesysShell_if	697
GeneticCircuit	698
GeneticCircuitGuiExtensionPlugin	703
GeneticCircuitPart	705
GeneticCircuitSimulate	709
GeneticCircuitSimulationSummary	716
GeneticExpressionStep	716
GeneticRegulation	722
GenSerializer	727
GraphicalAnimateExpression	730
GraphicalAssociation	731
GraphicalAutomaticPositioningStrategy	731
GraphicalAutomaticPositioningStrategyFactory	
Factory for the automatic positioning strategy selected from GUI preferences	732
GraphicalComponentPort	733
GraphicalConnection	
Graphical edge that represents model-level component connections	734
GraphicalConnectionStyle	737
GraphicalContextMenuController	
Builds context menus for the graphical model view using existing main-window actions	737
GraphicalDataDefinitionLayout	
Pure helper that computes arc placement for graphical data definitions	738
GraphicalDiagramConnection	739
GraphicalImageAnimation	740
GraphicalModelBuilder	
Service that reconstructs graphical components/connections from kernel model data	740

GraphicalModelComponent	
Visual component item that extends a data definition with ports and internal data	742
GraphicalModelDataDefinition	
Visual representation of a model data definition in the scene	747
GraphicalModelEvent	
Lightweight event envelope used to notify GUI changes in the graphics scene	749
GraphicalModelItemRenderContext	749
GraphicalModelItemRenderer	750
GraphicalModelItemRenderStrategy	750
GraphicalModelSerializer	
Persistence service for saving/loading graphical and textual model representations	751
GraphvizModelExporter	
Service that builds DOT content and renders model images for the GUI	752
ModelGraphicsScene::GRID	754
GroProgram	
Reusable Gro source program attached to a GenESyS model	754
GroProgramAst	
Minimal parsed representation of a Gro source program	759
GroProgramCompiler	
Converts parsed Gro AST into the first non-executable semantic IR	760
GroProgramIr	
Initial semantic representation compiled from GroProgramAst	760
GroProgramParser	
Plugin-side parser boundary for reusable Gro source programs	761
GroProgramRuntime	
Executes the supported Gro IR commands without simulator binding	761
GroProgramRuntimeState	
Mutable state used by the plugin-side Gro runtime helper	762
GroupItem	762
GroupUndoCommand	763
testing::internal::GTestFlagSaver	763
testing::internal::GTestLog	763
testing::internal::GTestMutexLock	764
GuiActionContribution	764
GuiDockContribution	764
GuiExtensionConfigurationDialog	765
GuiExtensionConfigurationGuiExtensionPlugin	766
GuiExtensionManager	767
GuiExtensionPlugin	768
GuiExtensionPluginAutoRegistrar	769
GuiExtensionPluginCatalog	769
GuiExtensionPresentationMetadata	769
GuiExtensionRegistry	769
GuiExtensionRuntimeContext	770
GuiScopeTrace	
Emits a scoped trace entry for a GUI method and measures its runtime	770
GuiThemeManager	
Applies the visual theme used by the Qt application and model views	772
GuiWindowContribution	773
Half_Adder	773
HarmonicOscillator	775
testing::internal::HasDebugStringAndShortDebugString< T >	776
DataAnalyzerWindow::HistogramPreview	776
HppLatticeGasState	777
HppLatticeGasStateSet	779
HttpProviderClientBase	
Abstract base class for HTTP-based AI provider clients	781

HttpRequest	
Lightweight representation of one parsed HTTP request	785
HttpResponse	
Minimal HTTP response payload produced by the web API router	786
HypothesisTester_if	
Interface for classical parametric inference	787
HypothesisTesterDefaultImpl1	
Current default implementation of HypothesisTester_if	788
testing::internal::Ignore< size_t >	795
testing::internal::IgnoredValue	795
testing::internal::IndexSequence< Is >	795
IntegerStateSet	795
InternalEvent	797
testing::internal::is_proxy_type_list< typename >	799
testing::internal::is_proxy_type_list< ProxyTypeList< Ts... > >	800
testing::internal::IsHashTable< T >	800
testing::internal::IsRecursiveContainer< C >	801
testing::internal::IsRecursiveContainerImpl< C, bool >	801
testing::internal::IsRecursiveContainerImpl< C, false >	802
testing::internal::IsRecursiveContainerImpl< C, true >	802
IssueReportDraft	
User-authored issue report collected by the GUI form before transport	803
IssueReportRelayService	
HTTPS client service that sends issue reports to a secure relay endpoint	803
IssueReportRuntimeContext	
Runtime diagnostics captured by the GUI after the user approves submission	804
IssueReportSubmissionResult	
Relay submission outcome returned to the controller layer	804
Json	804
JsonSerializer	805
KernelAccessorProbe	808
Label	808
QCLabelPainterPrivate::LabelData	813
Lattice	
N-dimensional lattice	813
Leave	815
LicenceManager	821
LineNumberArea	822
List< T >	
Lightweight wrapper around <code>std::list</code> with convenience helpers used by the kernel	823
ListObservable< T >	
Observable-oriented list variant used by kernel structures	825
LocalProviderClientImpl	
AI provider client for local model servers (Ollama, LM Studio, etc.)	827
LocalRule	831
LocalRule_CopyCurrentState	833
LocalRule_Elementary	834
LocalRule_FlorestalFire	835
LocalRule_GameOfLife	837
LocalRule_Growty	838
LocalRule_HppLatticeGas	839
LocalRule_SandRock_Student	841
LocalRule_ShowNeighbordhood	842
yy::location	
Two points in a source file	843
Loja01	845
testing::internal::internal_stream_operator_without_lexical_name_lookup::LookupBlocker	846
LSODE	846

LSODEInitialState	853
LSODESample	853
MainWindow	
Main Qt window that composes controllers/services and preserves legacy GUI entry points	853
testing::internal::MakeIndexSequenceImpl< N >	855
testing::internal::MakeIndexSequenceImpl< 0 >	856
testing::internal::MarkAsIgnored	857
MarkovChain	858
MassActionOdeSystem	
ODE system generated from biochemical reactions using mass-action kinetics	863
Match	865
testing::Message	871
MetabolicFluxBalance	872
MetabolicFluxBalanceSolver	878
MetabolicNetwork	878
MetabolicReaction	883
MetabolicStateProjectionComponent	
Projects flux-balance outputs into a WholeCellState	888
MetabolicSubmodelComponent	895
generate_kernel_simulator_test_stubs.MethodEntry	902
ModalModelDefault	902
ModalModelFSM	910
ModalModelPetriNet	916
Model	921
ModelChecker_if	930
ModelCheckerDefaultImpl1	932
ModelComponent	934
ModelComponentManager	
Owns and organizes the set of ModelComponent objects in a model	943
ModelDataDefinition	946
ModelDataDefinitionAssociations	960
ModelDataManager	
Central registry for ModelDataDefinition instances owned by a model	961
ModelEvent	966
ModelGraphicsScene	
Scene that owns and manipulates the graphical representation of a model	966
ModelGraphicsView	975
SimulatorSessionService::ModelImportResult	
Contains model import output and error state	978
ModelInfo	979
SimulatorSessionService::ModelInfoResult	
Describes the currently active model information for a session	982
ModelInspectorController	
Controller for model inspection views and scene-selection synchronization	982
ModelLanguageSynchronizer	
Service layer that synchronizes textual model language with kernel model state	983
ModelLifecycleController	
Controller for model/application lifecycle flows delegated from MainWindow	984
ModelManager	
The ModelManager class	985
SimulatorSessionService::ModelPersistenceResult	
Contains save/load model operation output and error state	991
ModelPluginInspectorGuiExtensionPlugin	991
ModelSimulation	
Controls the mature, currently functional simulation-execution layer of a model	993
MolecularSpecies	994
GroProgramRuntime::MotionMutation	999
MoveUndoCommand	1000

testing::internal::Mutex	1000
dialogBreakpoint::MVCResult	1001
GroProgramAst::NamedProgram	1001
GroProgramIrr::NamedProgramDefinition	1001
testing::internal::NameGeneratorSelector< Provided >	1001
NAND	1002
testing::internal::NativeArray< Element >	1008
NaturalStateSet	1008
Neighborhood	
Defines the complete neighborhood of a cell	1010
Neighborhood_Center	1011
Neighborhood_Moore	1013
Neighborhood_VonNeumann	1015
ParserManager::NewParser	
Paths associated with a generated parser artifact set	1016
NMOS	1017
testing::internal::None	1023
NOR	1023
NOT	1030
obj_t	1036
Optimizer_if::ObjectiveDefinition	
Configurable objective definition	1036
ObjectPropertyBrowser	1037
OctaveRunner	
Model data definition that executes Octave code through Rscript	1040
ODEfunction	1045
ODESolver	1046
OdeSolver_if	
ODE solver abstraction	1051
OdeSolverFactory	1052
OdeSystem_if	
Interface for first-order ODE systems $dy/dt = f(t, y)$	1052
OLD_ODEelement	1053
OnEventManager	1057
OpenAIProviderClientImpl	
AI provider client for the OpenAI Chat Completions API	1059
OperatingSystem02	1063
OperatingSystem03	1064
Optimizer_if::OptimizationSettings	
Generic optimization settings independent of a specific algorithm	1066
Optimizer_if	
Interface for simulation-based optimization engines (OptQuest-like)	1066
OptimizerDefaultImpl1	
Initial concrete scaffold for an OptQuest-like optimizer	1072
OptimizerWindow	1079
OR	1080
testing::internal::OsStackTraceGetter	1086
testing::internal::OsStackTraceGetterInterface	1087
testing::internal::ParameterizedTestFactory< TestClass >	1088
testing::internal::ParameterizedTestSuiteInfo< TestSuite >	1089
testing::internal::ParameterizedTestSuiteInfoBase	1090
testing::internal::ParameterizedTestSuiteRegistry	1091
testing::internal::ParamGenerator< T >	1091
testing::internal::ParamGeneratorInterface< T >	1092
testing::internal::ParamIterator< T >	1092
testing::internal::ParamIteratorInterface< T >	1093
Parser_if	
Interface for evaluating model expressions and stochastic functions	1093

ParserAttributeEventInjector	1095
ParserAttributeInternalEventOwner	1096
ParserAttributeProcessObserver	1101
ParserChangesInformation	1101
ParserDefaultImpl1	1102
ParserDefaultImpl2	1104
ParserExpressionsTest	1107
ParserManager	1108
PasteUndoCommand	1109
PathwayStressResponseComponent	
Applies sustained pathway-collapse responses to a whole-cell state	1110
Persistence_if	
Interface for model serialization/deserialization services	1116
PersistenceDefaultImpl2	1119
PersistencePartialLoadImpl1	1121
PersistenceRecord	
Serialization/Deserialization structure which behaves like an associative array	1122
PersistentObject_base	
Dynamic serializable interface	1124
PetriPlace	1126
PetriTransition	1132
PickableStationItem	1133
PickStation	1134
PickUp	1141
Plugin	1147
PluginCatalogController	
Controller responsible for plugin catalog presentation and plugin palette behavior	1150
PluginConnector_if	
Interface for dynamic discovery/loading/unloading of plugin libraries	1151
PluginConnectorDummyImpl1	
In-memory plugin connector that resolves built-in plugin metadata	1153
PluginConnectorStaticImpl1	
Static plugin connector stub used by the kernel interface	1155
PluginInformation	1158
PluginInsertionOptions	
Optional policy hooks used while inserting a plugin	1159
PluginLoadIssue	
Diagnostic record for a plugin candidate that could not be loaded	1159
PluginManager	
Manages discovery, loading, lookup and instantiation of simulation plugins	1160
PMOS	1162
testing::internal::PointerPrinter	1168
GroProgramRuntime::PopulationMutation	1168
yy::position	
A point in a source file	1168
testing::PrintToStringParamName	1169
ProbabilityDistribution	
Static façade for inverse/CDF-related distribution utilities	1169
ProbabilityDistributionBase	
Mathematical base library for distribution-related functions	1172
MetabolicFluxBalanceSolver::Problem	1175
Process	1175
OptimizerWindow::ProgressPlot	1182
BioStateProjectionComponent::Projection	1183
MetabolicStateProjectionComponent::Projection	1183
PropertyEditor	1183
PropertyEditorController	
Controller that orchestrates property editor state and scene-selection coupling	1184

PropertyEditorGenesys	1185
PropertyGenesys	1185
PropertyGenesysBase	
Base metadata for kernel-side control/response abstractions	1187
testing::internal::ProtobufPrinter	1189
ProviderRequest	
Input to a single LLM chat-completion call	1189
ProviderResponse	
Output from a single LLM chat-completion call	1189
testing::internal::ProxyTypeList< Ts >	1190
PythonForG	1191
PythonRuntime	1197
QCPAbstractItem	
The abstract base class for all items in a plot	1202
QCPAbstractLegendItem	
The abstract base class for all entries in a QCPLegend	1209
QCPAbstractPaintBuffer	
The abstract base class for paint buffers, which define the rendering backend	1216
QCPAbstractPlottable	
The abstract base class for all data representing objects in a plot	1218
QCPAbstractPlottable1D< DataType >	
A template base class for plottables with one-dimensional data	1231
QCPAxis	
Manages a single axis inside a QCustomPlot	1238
QCPAxisPainterPrivate	1259
QCPAxisRect	
Holds multiple axes and arranges them in a rectangular shape	1261
QCPAxisTicker	
The base class tick generator used by QCPAxis to create tick positions and tick labels	1276
QCPAxisTickerDateTime	
Specialized axis ticker for calendar dates and times as axis ticks	1280
QCPAxisTickerFixed	
Specialized axis ticker with a fixed tick step	1285
QCPAxisTickerLog	
Specialized axis ticker suited for logarithmic axes	1288
QCPAxisTickerPi	
Specialized axis ticker to display ticks in units of an arbitrary constant, for example pi	1291
QCPAxisTickerText	
Specialized axis ticker which allows arbitrary labels at specified coordinates	1294
QCPAxisTickerTime	
Specialized axis ticker for time spans in units of milliseconds to days	1298
QCPBars	
A plottable representing a bar chart in a plot	1302
QCPBarsData	
Holds the data of one single data point (one bar) for QCPBars	1312
QCPBarsGroup	
Groups multiple QCPBars together so they appear side by side	1314
QCPColorGradient	
Defines a color gradient for use with e.g. QCPColorMap	1318
QCPColorMap	
A plottable representing a two-dimensional color map in a plot	1324
QCPColorMapData	
Holds the two-dimensional data of a QCPColorMap plottable	1333
QCPColorScale	
A color scale for use with color coding data such as QCPColorMap	1339
QCPColorScaleAxisRectPrivate	1348
QCPCurve	
A plottable representing a parametric curve in a plot	1354

QCPCurveData	
Holds the data of one single data point for QCPCurve	1364
QCPDataContainer< DataType >	
The generic data container for one-dimensional plottables	1366
QCPDataRange	
Describes a data range given by begin and end index	1373
QCPDataSelection	
Describes a data set by holding multiple QCPDataRange instances	1377
QCPErrorBars	
A plottable that adds a set of error bars to other plottables	1382
QCPErrorBarsData	
Holds the data of one single error bar for QCPErrorBars	1393
QCPFinancial	
A plottable representing a financial stock chart	1394
QCPFinancialData	
Holds the data of one single data point for QCPFinancial	1405
QCPGraph	
A plottable representing a graph in a plot	1407
QCPGraphData	
Holds the data of one single data point for QCPGraph	1416
QCPGrid	
Responsible for drawing the grid of a QCPAxis	1418
QCPItemAnchor	
An anchor of an item to which positions can be attached to	1422
QCPItemBracket	
A bracket for referencing/highlighting certain parts in the plot	1424
QCPItemCurve	
A curved line from one point to another	1430
QCPItemEllipse	
An ellipse	1435
QCPItemLine	
A line from one point to another	1440
QCPItemPixmap	
An arbitrary pixmap	1445
QCPItemPosition	
Manages the position of an item	1451
QCPItemRect	
A rectangle	1457
QCPItemStraightLine	
A straight line that spans infinitely in both directions	1462
QCPItemText	
A text label	1467
QCPItemTracer	
Item that sticks to QCPGraph data points	1474
QCPLabelPainterPrivate	1482
QCPLayer	
A layer that may contain objects, to control the rendering order	1484
QCPLayerable	
Base class for all drawable objects	1488
QCPLayout	
The abstract base class for layouts	1495
QCPLayoutElement	
The abstract base class for all objects that form the layout system	1502
QCPLayoutGrid	
A layout that arranges child elements in a grid	1509
QCPLayoutInset	
A layout that places child elements aligned to the border or arbitrarily positioned	1521

QCPLegend	Manages a legend inside a QCustomPlot	1529
QCPLineEnding	Handles the different ending decorations for line-like items	1541
QCPMarginGroup	A margin group allows synchronization of margin sides if working with multiple layout elements	1544
QCPPaintBufferPixmap	A paint buffer based on QPixmap, using software raster rendering	1546
QCPPainter	QPainter subclass used internally	1548
QCPPlottableInterface1D	Defines an abstract interface for one-dimensional plottables	1552
QCPPlottableLegendItem	A legend item representing a plottable with an icon and the plottable name	1556
QCPPolarAxisAngular	The main container for polar plots, representing the angular axis as a circle	1561
QCPPolarAxisRadial	The radial axis inside a radial plot	1582
QCPPolarGraph	A radial graph used to display data in polar plots	1601
QCPPolarGrid	The grid in both angular and radial dimensions for polar plots	1610
QCPPolarLegendItem	A legend item for polar plots	1613
QCPRange	Represents the range an axis is encompassing	1618
QCPScatterStyle	Represents the visual appearance of scatter points	1624
QCPSelectionDecorator	Controls how a plottable's data selection is drawn	1630
QCPSelectionDecoratorBracket	A selection decorator which draws brackets around each selected data segment	1633
QCPSelectionRect	Provides rect/rubber-band data selection and range zoom interaction	1637
QCPStatisticalBox	A plottable representing a single statistical box in a plot	1642
QCPStatisticalBoxData	Holds the data of one single data point for QCPStatisticalBox	1652
QCPTextElement	A layout element displaying a text	1654
QCPVector2D	Represents two doubles as a mathematical 2D vector	1663
QCustomPlot	The central class of the library. This is the QWidget which displays the plot and interacts with the user	1668
QtAbstractEditorFactory< PropertyManager >		1698
QtAbstractEditorFactoryBase		1702
QtAbstractPropertyBrowser		1704
QtAbstractPropertyBrowserPrivate		1709
QtAbstractPropertyManager		1710
QtAbstractPropertyManagerPrivate		1715
QtBoolEdit		1715
QtBoolPropertyManager		1716
QtBoolPropertyManagerPrivate		1719
QtBrowserItem		1719
QtBrowserItemPrivate		1720
QtButtonPropertyBrowser		1721
QtButtonPropertyBrowserPrivate		1724

QtCharEdit	1725
QtCharEditorFactory	1726
QtCharEditorFactoryPrivate	1728
QtCharPropertyManager	1729
QtCharPropertyManagerPrivate	1731
QtCheckBoxFactory	1732
QtCheckBoxFactoryPrivate	1734
QtColorEditorFactory	1735
QtColorEditorFactoryPrivate	1737
QtColorEditWidget	1739
QtColorPropertyManager	1740
QtColorPropertyManagerPrivate	1743
QtCursorDatabase	1744
QtCursorEditorFactory	1744
QtCursorEditorFactoryPrivate	1746
QtCursorPropertyManager	1747
QtCursorPropertyManagerPrivate	1750
QtDateEdit	1750
QtDateEditFactory	1751
QtDateEditFactoryPrivate	1753
QtDatePropertyManager	1755
QtDatePropertyManagerPrivate	1759
QtDateTimeEdit	1759
QtDateTimeEditFactory	1761
QtDateTimeEditFactoryPrivate	1763
QtDateTimePropertyManager	1764
QtDateTimePropertyManagerPrivate	1766
QtDoubleEdit	1767
QtDoublePropertyManager	1768
QtDoublePropertyManagerPrivate	1773
QtDoubleSpinBoxFactory	1774
QtDoubleSpinBoxFactoryPrivate	1776
QtEnumEditorFactory	1777
QtEnumEditorFactoryPrivate	1779
QtEnumPropertyManager	1781
QtEnumPropertyManagerPrivate	1785
QtEnumPropertyType	1785
QtFlagPropertyManager	1785
QtFlagPropertyManagerPrivate	1789
QtFlagPropertyType	1789
QtFontEditorFactory	1790
QtFontEditorFactoryPrivate	1792
QtFontEditWidget	1793
QtFontPropertyManager	1794
QtFontPropertyManagerPrivate	1797
QtGroupBoxPropertyBrowser	1798
QtGroupBoxPropertyBrowserPrivate	1800
QtGroupPropertyManager	1801
QtGroupPropertyType	1803
QtIntEdit	1803
QtIntPropertyManager	1804
QtIntPropertyManagerPrivate	1808
QtKeySequenceEditorFactory	1809
QtKeySequenceEditorFactoryPrivate	1811
QtKeySequencePropertyManager	1813
QtKeySequencePropertyManagerPrivate	1815
QtLineEditFactory	1816
QtLineEditFactoryPrivate	1818

QtLocalePropertyManager	1819
QtLocalePropertyManagerPrivate	1822
QtMetaEnumProvider	1822
QtMetaEnumWrapper	1823
QtPointFPropertyManager	1824
QtPointFPropertyManagerPrivate	1828
QtPointPropertyManager	1829
QtPointPropertyManagerPrivate	1832
QtProperty	1833
QtPropertyBrowserUtils	1838
QtPropertyEditorDelegate	1839
QtPropertyEditorView	1840
QtPropertyPrivate	1841
QtRectFPropertyManager	1842
QtRectFPropertyManagerPrivate	1846
QtRectPropertyManager	1847
QtRectPropertyManagerPrivate	1851
QtScrollBarFactory	1852
QtScrollBarFactoryPrivate	1854
QtSizeFPropertyManager	1856
QtSizeFPropertyManagerPrivate	1861
QtSizePolicyPropertyManager	1862
QtSizePolicyPropertyManagerPrivate	1865
QtSizePropertyManager	1866
QtSizePropertyManagerPrivate	1870
QtSliderFactory	1871
QtSliderFactoryPrivate	1873
QtSpinBoxFactory	1875
QtSpinBoxFactoryPrivate	1877
QtStringEdit	1878
QtStringPropertyManager	1879
QtStringPropertyManagerPrivate	1882
QtTimeEdit	1882
QtTimeEditFactory	1884
QtTimeEditFactoryPrivate	1886
QtTimePropertyManager	1887
QtTimePropertyManagerPrivate	1889
QtTreePropertyBrowser	1890
QtTreePropertyBrowserPrivate	1896
QtVariantEditorFactory	1897
QtVariantEditorFactoryPrivate	1899
QtVariantProperty	1900
QtVariantPropertyManager	1903
QtVariantPropertyManagerPrivate	1909
QtVariantPropertyPrivate	1911
Quadrature_if	
One-dimensional numerical quadrature abstraction	1911
Queue	1912
QueueableItem	1917
testing::internal::Random	1918
testing::internal::RangeGenerator< T, IncrementT >	1918
testing::internal::RawBytesPrinter	1919
testing::internal::RE	1920
MassActionOdeSystem::Reaction	1920
RealIntervalStateSet	1920
RealState	1922
Record	1924
RecordingContinuousSystemComponent	1931

RecordingLSODE	1937
Rectifier	1943
testing::internal::RelationToSourceCopy	1944
testing::internal::RelationToSourceReference	1944
Release	1944
Remove	1951
Resistor	1958
Resource	1964
ResourceAllocationComponent	1970
ResourceTestProbe	1976
GroProgramParser::Result	1977
Sampler_if::RNG_Parameters	
Encapsulates generator-specific configuration/state parameters	1977
RootFinder_if	
Root-finding abstraction for scalar equations	1978
Route	1978
RSimulator	
Component that sends configured R commands to an internal RSimulatorRunner	1985
RSimulatorRunner	
Model data definition that executes R code through Rscript	1991
RungeKutta4OdeSolver	
Fixed-step fourth-order Runge-Kutta solver for first-order ODE systems	1997
Sample	1998
Sampler_if	
Abstraction for random-number generation and probability-distribution sampling	1998
SamplerDefaultImpl1	
Default pseudo-random sampler implementation used by the kernel	2003
ScenarioExperiment_if	2007
SceneToolController	
Controller for scene visualization, drawing tools, alignment, and zoom orchestration	2007
SchedulableItem	2011
Schedule	2011
testing::ScopedTrace	2016
Search	2016
SeizableItem	2023
Seize	2024
Separate	2031
Sequence	2037
SequenceStep	2043
Serializer	
Serialization/Deserialization interface for simulation models	2044
SessionContext	
Owns the state that belongs to one authenticated web session	2048
SessionManager	
Creates, stores and resolves authenticated web sessions	2048
Set	2050
Signal	2056
SignalData	2062
GroProgramRuntime::SignalMutation	2067
SimpleHttpServer	
Minimal blocking HTTP/1.1 server for single-request-per-connection handling	2067
SimulatorSessionService::SimulationActionResult	
Contains the result of a simulation action such as run or step	2068
SimulationCommandController	
Controller that orchestrates UI simulation commands over SimulationController guards	2069
SimulatorSessionService::SimulationConfigInput	
Defines user-provided simulation parameters to apply to the current model	2070

SimulatorSessionService::SimulationConfigResult	
Contains the result of applying simulation configuration	2070
SimulationControl	
Read/write kernel-side simulation control abstraction	2071
SimulationControlBool	2074
SimulationControlDouble	2077
SimulationControlGeneric< T >	2080
SimulationControlGenericClass< T, M, C >	2083
SimulationControlGenericClassNotDC< T, M, C >	2087
SimulationControlGenericEnum< T, E >	2091
SimulationControlGenericList< T, M, C >	2094
SimulationControlGenericListPointer< T, M, C >	2098
SimulationControlInt	2102
SimulationController	
Controller that validates simulation command readiness for MainWindow wrappers	2105
SimulationControlSourceCodeString	2107
SimulationControlString	2110
SimulationControlTimeUnit	2113
SimulationControlUInt	2116
SimulationControlUShort	2119
SimulationEvent	2122
SimulationEventController	
Controller for simulation event callbacks, UI updates, and handler wiring	2123
SimulationExperiment	
Represents a higher-level simulation experiment definition	2124
SimulationReporter_if	
Interface for reporting simulation results and experiment-facing outputs	2124
SimulationReporterDefaultImpl1	2125
SimulationResponse	
Read-only kernel-side simulation response abstraction	2127
SimulationResponseDouble	2129
SimulationResultsDataset	2131
SimulationResultsDatasetParser	2132
SimulationResultsObservation	2132
SimulationScenario	
Represents a candidate simulation scenario within the unfinished experiment layer	2132
SimulatorSessionService::SimulationStatusResult	
Captures the current simulation execution state	2133
Simulator	2134
SimulatorFacade	
High-level non-owning facade over Simulator and its main composed services	2136
SimulatorSessionService::SimulatorInfoResult	
Describes simulator identity metadata for a token-scoped session	2142
SimulatorSessionService	
Encapsulates session-scoped simulator operations exposed by the Web API	2142
SinkModelComponent	2149
yy::genesyspp_parser::stack< T, S >::slice	
Present a slice of the top of a stack	2155
Smart_AddingResource	2156
Smart_AlternatingEntityCreation	2157
Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime	2159
Smart_ArrivalsEntityTypeVsAttribute	2161
Smart_AssignExample	2163
Smart_AssignWriteSeizes	2165
Smart_Attribute_Variable	2166
Smart_AutomaticStatisticsCollection	2168
Smart_BacteriaColony_GRO	2170
Smart_BasicModeling	2172

Smart_BatchAndSeparate	2173
Smart_BatchSeparate	2175
Smart_BioKineticLawRegulation	2177
Smart_BioNetworkAnalysis	2178
Smart_BioReversibleKineticLaw	2180
Smart_BioReversibleMassAction	2182
Smart_Buffer	2183
Smart_CellularAutomata	2185
Smart_CellularAutomata1	2187
Smart_Clone	2188
Smart_ContinuousFlowEntities	2190
Smart_CppForG	2192
Smart_Create	2193
Smart_DecideNWayByChance	2195
Smart_DefaultModalModel	2197
Smart_DefiningAttributesAsStrings	2198
Smart_DefiningControlLogic	2200
Smart_DefiningResourceCapacity	2202
Smart_Delay	2203
Smart_DelayBasedOnReplication	2205
Smart_Dummy	2207
Smart_EntitiesProcessedByPriority	2209
Smart_EvaluatingConditionsBeforeEnteringQueue	2211
Smart_Expression	2213
Smart_Failures	2215
Smart_FSMState	2216
Smart_GroColonyGrowth	2218
Smart_GroColonyLifecycle	2220
Smart_HoldSearchRemove	2221
Smart_InventoryAndHoldingCosts	2223
Smart_LSODE	2225
Smart_MarkovChain	2227
Smart_MaxArrivalsField	2228
Smart_ModalModel	2230
Smart_ModalModelDefaultNetwork	2232
Smart_ModalModelFSM	2233
Smart_ModalModelPetriNet	2235
Smart_ModelInfoModelSimulation	2237
Smart_ModelRunUntil1000Parts	2238
Smart_ModuleDisplayVariables	2240
Smart_ODE	2242
Smart_OnEvent	2243
Smart_OverlappingResources	2246
Smart_ParallelProcessingOfEntities	2248
Smart_Parser	2250
Smart_ParserModelFunctions	2252
Smart_PetriPlace	2253
Smart_PlacingEntitiesInQueueSets	2255
Smart_Plugin	2257
Smart_PriorityExample	2258
Smart_Process	2260
Smart_ProcessArena	2262
Smart_ProcessSet	2263
Smart_PythonForG	2265
Smart_R_Simulator	
Builds a minimal Create -> RSimulator -> Dispose model	2266
Smart_Record	2269
Smart_Record_Arena	2270

Smart_RemovingAndReorderingEntitiesInAQueue	2272
Smart_ResourceCosting	2274
Smart_ResourceScheduleCosting	2276
Smart_ResourceSets	2277
Smart_RouteStation	2279
Smart_SeizeDelayRelease	2281
Smart_SeizeDelayReleaseMany	2282
Smart_SeizeDelayReleaseNoDataDefs	2284
Smart_SeizingMultipleSimultaneously	2286
Smart_SelectingRouteBasedOnProbability	2288
Smart_SelectingShorterQueue	2289
Smart_Sequence	2291
Smart_SimulationControlResponse	2293
Smart_SynchronizingParallelEntities	2294
Smart_ValueAdded	2296
Smart_WaitForSignal	2298
Smart_WaitScanCondition	2299
Smart_WaitSignal	2301
WebWorkerRuntime::Snapshot	
Captures a thread-safe snapshot of the worker state	2302
MetabolicFluxBalanceSolver::Solution	2303
Optimizer_if::SolutionSummary	
Captures one candidate or best-known optimization solution	2303
Solver_if	
Legacy compatibility interface for numerical routines	2304
SolverDefaultImpl1	
Legacy default implementation of Solver_if	2305
SortFile	
Utility class that sorts numeric values persisted in a data file	2308
SourceCodeString	2309
SourceModelComponent	2309
SparseValueStore	
Sparse N-dimensional numeric store using GenESyS textual index keys	2319
MassActionOdeSystem::Species	2320
SPICECircuit	2320
SPICENode	2326
SPICERunner	2332
Start	2337
StartAIAssistant	2343
State	2345
GroProgramAst::Statement	2346
StateSet	2346
StateSet_Enumerable	2347
Station	2349
StationTestProbe	2354
Statistics_if	
Interface for statistical synthesis over values provided by a Collector_if	2358
StatisticsCollector	2362
StatisticsDatafile_if	
Extension of Statistics_if for metrics that require full data access	2367
StatisticsDatafileDefaultImpl1	
Default implementation of StatisticsDatafile_if for file-backed samples	2371
StatisticsDefaultImpl1	
Default online statistics implementation over a Collector_if source	2376
StochasticReactionComponent	2379
StochasticReactionRule	2385
StochasticTranscription	2390
StochasticTranslation	2397

BioReaction::StoichiometricTerm	2404
MassActionOdeSystem::StoichiometricTerm	2404
MetabolicReaction::StoichiometricTerm	2404
StochasticReactionRule::StoichiometricTerm	2405
Stop	2405
Storage	2411
Store	2416
testing::internal::internal_stream_operator_without_lexical_name_lookup::StreamPrinter	2422
testing::internal::String	2422
Submodel	2423
testing::internal::SuiteApiResolver< T >	2429
yy::genesyspp_parser::symbol_kind	
Symbol kinds	2430
yy::genesyspp_parser::symbol_type	
"External" symbols: returned by the scanner	2431
yy::genesyspp_parser::syntax_error	
Syntax errors thrown from user actions	2434
SystemCommandExecutor_if	
Minimal command execution interface used to make system dependency checks testable	2435
SystemCommandResult	
Captures the result of executing a system command used by dependency checks	2435
SystemDependency	2436
SystemDependencyCheckEntry	
One dependency entry produced by a system dependency preflight	2436
SystemDependencyCheckResult	
Full preflight result for the system dependencies declared by one plugin	2437
SystemDependencyInstallEntry	
One command result from an attempted system dependency installation	2438
SystemDependencyInstallResult	
Aggregates commands executed while trying to satisfy missing system dependencies	2438
SystemDependencyResolver	
Evaluates and optionally installs plugin system dependencies	2439
SystemPreferences	
Persistent application preferences used by the Qt GUI	2439
SystemShellCommandExecutor	
Shell-based executor used by production plugin dependency preflight	2442
testing::internal::Templates< Head_, Tail_ >	2443
testing::internal::Templates< Head_ >	2443
testing::internal::TemplateSel< Tmpl >	2443
testing::Test	2443
testing::TestEventListener	2444
testing::TestEventListeners	2445
testing::internal::TestFactoryBase	2445
testing::internal::TestFactoryImpl< TestClass >	2446
testing::TestInfo	2447
testing::internal::TestMetaFactory< TestSuite >	2447
testing::internal::TestMetaFactoryBase< ParamType >	2448
testing::TestParamInfo< ParamType >	2448
testing::TestProperty	2449
testing::internal::TestPropertyKeyls	2449
HypothesisTester_if::TestResult	
Result of a parametric hypothesis test	2449
testing::TestResult	2450
testing::internal::TestResultAccessor	2450
TestSimulationControl	2451
TestSimulationResponse	2454
testing::TestSuite	2455
testing::TestWithParam< T >	2456

testing::internal::ThreadLocal< T >	2457
QCPAxisPainterPrivate::TickLabelData	2457
Token	2458
yy::genesyspp_parser::token	
Token kinds	2458
TokenService	2459
TraceConsoleController	
Controller that routes simulator trace events to GUI text widgets	2459
TraceErrorEvent	2460
TraceEvent	2461
testing::internal::TraceInfo	2461
TraceManager	2461
TraceSimulationEvent	2466
TraceSimulationProcess	2466
TraitsGUI< T >	
GUI-specific trait constants used to tune default rendering behavior	2467
TraitsGUI< GAssociation >	2468
TraitsGUI< GComponentPort >	2468
TraitsGUI< GConnection >	2468
TraitsGUI< GMainWindow >	2469
TraitsGUI< GModelComponent >	2469
TraitsGUI< GModelDataDefinition >	2469
TraitsGUI< GModelGraphicPic >	2470
TraitsGUI< GScene >	2470
TraitsGUI< GView >	2470
TraitsKernel< T >	2470
TraitsKernel< Collector_if >	2471
TraitsKernel< Model >	2471
TraitsKernel< ModelChecker_if >	2471
TraitsKernel< ModelComponent >	2471
TraitsKernel< ModelDataDefinition >	2471
TraitsKernel< Parser_if >	2472
TraitsKernel< Persistence_if >	2472
TraitsKernel< PluginConnector_if >	2472
TraitsKernel< Sampler_if >	2472
TraitsKernel< SimulationReporter_if >	2472
TraitsKernel< Simulator >	2473
TraitsKernel< Statistics_if >	2473
TraitsKernel< StatisticsDatafile_if >	2473
TraitsTools< T >	
Registry-like traits to bind abstractions to concrete tools classes	2473
TraitsTools< AIAssistant_if >	2474
TraitsTools< Fitter_if >	2474
TraitsTools< HypothesisTester_if >	2474
TraitsTools< Solver_if >	2474
testing::internal::TrueWithString	2475
std::tuple_size< testing::internal::FlatTuple< Ts... > >	2475
testing::internal::TypedHelper< T >	2476
testing::internal::TypeParameterizedTest< Fixture, TestSel, Types >	2476
testing::internal::TypeParameterizedTest< Fixture, TestSel, internal::None >	2476
testing::internal::TypeParameterizedTestSuite< Fixture, Tests, Types >	2477
testing::internal::TypeParameterizedTestSuite< Fixture, internal::None, Types >	2477
testing::internal::TypeParameterizedTestSuiteRegistry	2477
testing::internal::Types< Head_, Tail_ >	2477
testing::internal::Types< Head_ >	2477
testing::internal::TypeWithSize< size >	2478
testing::internal::TypeWithSize< 4 >	2478
testing::internal::TypeWithSize< 8 >	2478

UngroupUndoCommand	2478
testing::UnitTest	2479
testing::internal::UnitTestImpl	2480
testing::internal::UnitTestOptions	2482
testing::internal::UniversalPrinter< T >	2482
testing::internal::UniversalPrinter< const T >	2483
testing::internal::UniversalPrinter< T & >	2483
testing::internal::UniversalPrinter< T[N]>	2484
testing::internal::UniversalTersePrinter< T >	2484
testing::internal::UniversalTersePrinter< char * >	2484
testing::internal::UniversalTersePrinter< char16_t * >	2485
testing::internal::UniversalTersePrinter< char32_t * >	2486
testing::internal::UniversalTersePrinter< const char * >	2487
testing::internal::UniversalTersePrinter< const char16_t * >	2488
testing::internal::UniversalTersePrinter< const char32_t * >	2488
testing::internal::UniversalTersePrinter< T & >	2489
testing::internal::UniversalTersePrinter< T[N]>	2489
testing::internal::UniversalTersePrinter< wchar_t * >	2489
Unstore	2489
Util	2495
yy::genesyspp_parser::value_type	2506
testing::internal::ValueArray< Ts >	2507
testing::internal::ValuesInIteratorRangeGenerator< T >	2508
Variable	2509
Vpulse	2516
Vsine	2522
Vsource	2528
Wait	2534
Waiting	2541
WaitingResource	2542
WcmBioNetworkBridge	
Demonstrates a minimal deterministic/stochastic whole-cell bridge	2543
WcmCompartmentTransportWholeCellMvp	
Whole-cell MVP with compartment-aware metabolic transport	2544
WcmFluxBalanceWholeCellMvp	
Whole-cell MVP driven by flux-balance metabolism	2546
WcmGeneExpression	2548
WcmLifecycleStress	
Demonstrates starvation and death routing in a whole-cell stress scenario	2550
WcmMgenitaliumKarr2012	2552
WcmPathwayStressWholeCellMvp	
Whole-cell stress example driven by sustained low pathway activity	2555
WcmSimpleSsa	2556
WcmWholeCellMvp	
Minimal whole-cell MVP combining metabolism, expression, growth, and division	??
WcmYeastDidacticWholeCellMvp	
Didactic yeast-oriented whole-cell MVP built on GEM-inspired metabolism	??
WcmYeastEukaryoticWholeCellMvp	
Didactic eukaryotic whole-cell MVP inspired by budding yeast	??
WebWorkerDialog	
Lightweight control window for the embedded GenESyS web worker	??
WebWorkerRuntime	
Owns the embedded worker runtime used by the GUI and the standalone worker app	??
WholeCellParameterReader	??
WholeCellState	??
QtButtonPropertyBrowserPrivate::WidgetItem	??
QtGroupBoxPropertyBrowserPrivate::WidgetItem	??
testing::WithParamInterface< T >	??

SimulatorSessionService::WorkerCapabilitiesResult	
Describes currently supported worker features	??
SimulatorSessionService::WorkerInfoResult	
Describes worker identity metadata for distributed discovery	??
WorkerJob	
Stores metadata for a worker job snapshot owned by the web worker	??
SimulatorSessionService::WorkerJobCreationResult	
Contains worker job creation output and error state	??
SimulatorSessionService::WorkerJobInfoResult	
Describes worker job metadata exposed by worker stage 3 endpoints	??
WorkerJobManager	
Owns worker job records in memory and assigns unique identifiers	??
SimulatorSessionService::WorkerJobQueryResult	
Contains worker job lookup output and error state	??
SimulatorSessionService::WorkerJobResultInfo	
Describes terminal worker job result data exposed by worker stage 5 endpoint	??
SimulatorSessionService::WorkerJobResultQueryResult	
Contains worker job terminal result lookup output and error state	??
SimulatorSessionService::WorkerJobRunResult	
Contains worker job run output and error state	??
WorkerJobTerminalResult	
Stores the terminal simulation summary persisted for a finished or failed worker job	??
Write	??
XmlSerializer	??
XNOR	??
XOR	??
yy_buffer_state	??
yy_trans_info	??

Chapter 13

File Index

13.1 File List

Here is a list of all documented files with brief descriptions:

applications/ BaseGenesysTerminalApplication.h	??
applications/ GenesysApplication_if.h	??
applications/gui/ai_assistant/ AIAssistantWindow.h	??
applications/gui/genesys/ BaseMVController.h	??
applications/gui/genesys/ GuiCrashDiagnostics.h	??
applications/gui/genesys/ GuiScopeTrace.h	??
applications/gui/genesys/ guithememanager.h	??
applications/gui/genesys/ mainwindow.cpp	
Composition-root partition of MainWindow implementation	??
applications/gui/genesys/ mainwindow.h	??
applications/gui/genesys/ mainwindow_controller.cpp	
Controller-delegation partition of MainWindow implementation	??
applications/gui/genesys/ mainwindow_modelrepresentations.cpp	
Model representation/persistence wrapper partition of MainWindow implementation	??
applications/gui/genesys/ mainwindow_scene.cpp	
Scene callback and scene synchronization partition of MainWindow implementation	??
applications/gui/genesys/ mainwindow_simulator.cpp	
Simulator trace/event wrapper partition of MainWindow implementation	??
applications/gui/genesys/ MovedGuiCompatibilityIncludes.h	??
applications/gui/genesys/ qcustomplot.h	??
applications/gui/genesys/ systempreferences.h	??
applications/gui/genesys/ TraitsGUI.h	??
applications/gui/genesys/ UtilGUI.h	??
applications/gui/genesys/actions/ AddUndoCommand.h	??
applications/gui/genesys/actions/ DeleteUndoCommand.h	??
applications/gui/genesys/actions/ GroupUndoCommand.h	??
applications/gui/genesys/actions/ MoveUndoCommand.h	??
applications/gui/genesys/actions/ PasteUndoCommand.h	??
applications/gui/genesys/actions/ UngroupUndoCommand.h	??
applications/gui/genesys/animations/ AnimationCounter.h	??
applications/gui/genesys/animations/ AnimationPlaceholder.h	??
applications/gui/genesys/animations/ AnimationQueue.h	??
applications/gui/genesys/animations/ AnimationTimer.h	??
applications/gui/genesys/animations/ AnimationTransition.h	??
applications/gui/genesys/animations/ AnimationVariable.h	??
applications/gui/genesys/cellular_automata/ CellularAutomataDemoBuilder.h	??
applications/gui/genesys/cellular_automata/ CellularAutomataLatticeWidget.h	??
applications/gui/genesys/cellular_automata/ CellularAutomataStateColorMap.h	??
applications/gui/genesys/cellular_automata/ CellularAutomataViewerController.h	??
applications/gui/genesys/cellular_automata/ CellularAutomataViewerWindow.h	??

applications/gui/genesys/codeeditor/CodeEditor.h	??
applications/gui/genesys/codeeditor/LineNumberArea.h	??
applications/gui/genesys/controllers/DialogUtilityController.h	??
applications/gui/genesys/controllers/EditCommandController.h	??
applications/gui/genesys/controllers/GraphicalContextMenuController.h	??
applications/gui/genesys/controllers/ModelInspectorController.h	??
applications/gui/genesys/controllers/ModelLifecycleController.h	??
applications/gui/genesys/controllers/PluginCatalogController.h	??
applications/gui/genesys/controllers/PropertyEditorController.h	??
applications/gui/genesys/controllers/SceneToolController.h	??
applications/gui/genesys/controllers/SimulationCommandController.h	??
applications/gui/genesys/controllers/SimulationController.h	??
applications/gui/genesys/controllers/SimulationEventController.h	??
applications/gui/genesys/controllers/TraceConsoleController.h	??
applications/gui/genesys/dialogs/dialogBreakpoint.h	??
applications/gui/genesys/dialogs/DialogFind.h	??
applications/gui/genesys/dialogs/Dialogmodelinformation.h	??
applications/gui/genesys/dialogs/dialogpluginmanager.h	??
applications/gui/genesys/dialogs/DialogReportIssue.h	??
applications/gui/genesys/dialogs/DialogSelectCounter.h	??
applications/gui/genesys/dialogs/DialogSelectVariable.h	??
applications/gui/genesys/dialogs/dialogsimulationconfigure.h	??
applications/gui/genesys/dialogs/dialogsystempreferences.h	??
applications/gui/genesys/dialogs/DialogTimerConfigure.h	??
applications/gui/genesys/extensions/GuiExtensionConfigurationDialog.h	??
applications/gui/genesys/extensions/GuiExtensionContracts.h	??
applications/gui/genesys/extensions/GuiExtensionManager.h	??
applications/gui/genesys/extensions/GuiExtensionPluginCatalog.h	??
applications/gui/genesys/graphics/GraphicalAnimateExpression.h	??
applications/gui/genesys/graphics/GraphicalAssociation.h	??
applications/gui/genesys/graphics/GraphicalComponentPort.h	??
applications/gui/genesys/graphics/GraphicalConnection.h	??
applications/gui/genesys/graphics/GraphicalConnectionStyle.h	??
applications/gui/genesys/graphics/GraphicalDiagramConnection.h	??
applications/gui/genesys/graphics/GraphicalImageAnimation.h	??
applications/gui/genesys/graphics/GraphicalModelComponent.h	??
applications/gui/genesys/graphics/GraphicalModelDataDefinition.h	??
applications/gui/genesys/graphics/GraphicalModelItemRenderStrategy.h	??
applications/gui/genesys/graphics/ModelGraphicsScene.h	??
applications/gui/genesys/graphics/ModelGraphicsView.h	??
applications/gui/genesys/propertyeditor/ComboBoxEnum.h	??
applications/gui/genesys/propertyeditor/DataComponentEditor.h	??
applications/gui/genesys/propertyeditor/DataComponentProperty.h	??
applications/gui/genesys/propertyeditor/ObjectPropertyBrowser.h	??
applications/gui/genesys/propertyeditor/PropertyEditor.h	??
applications/gui/genesys/propertyeditor/qtpropertybrowser/qtbuttonpropertybrowser.h	??
applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h	??
applications/gui/genesys/propertyeditor/qtpropertybrowser/qtgroupboxpropertybrowser.h	??
applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.h	??
applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils_p.h	??
applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h	??
applications/gui/genesys/propertyeditor/qtpropertybrowser/qtreepropertybrowser.h	??
applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.h	??
applications/gui/genesys/services/CppModelExporter.h	??
applications/gui/genesys/services/GraphicalAutomaticPositioningStrategy.h	??
applications/gui/genesys/services/GraphicalDataDefinitionLayout.h	??
applications/gui/genesys/services/GraphicalModelBuilder.h	??
applications/gui/genesys/services/GraphicalModelSerializer.h	??

applications/gui/genesys/services/GraphvizModelExporter.h	??
applications/gui/genesys/services/IssueReportRelayService.h	??
applications/gui/genesys/services/ModelLanguageSynchronizer.h	??
applications/gui/genesys/services/ToolLauncher.h	??
applications/gui/genesys/tools/dataanalyzer/DataAnalyzerWindow.h	??
applications/gui/genesys/tools/expressionbuilder/ExpressionBuilder.h	??
applications/gui/genesys/tools/optimizer/OptimizerWindow.h	??
applications/gui/httpworker/WebWorkerDialog.h	??
applications/modelSpecific/arenaExamples/Airport_Extended1.h	??
applications/modelSpecific/arenaExamples/AirportSecurityExample.h	??
applications/modelSpecific/arenaExamples/AirportSecurityExampleExtended.h	??
applications/modelSpecific/arenaExamples/Assembly_Line.h	??
applications/modelSpecific/arenaExamples/Banking_Transactions.h	??
applications/modelSpecific/arenaExamples/Example_BasicOrderShipping.h	??
applications/modelSpecific/arenaExamples/Example_PortModel.h	??
applications/modelSpecific/arenaExamples/Example_PublicTransport.h	??
applications/modelSpecific/arenaSmarts/Smart_AddingResource.h	??
applications/modelSpecific/arenaSmarts/Smart_AlternatingEntityCreation.h	??
applications/modelSpecific/arenaSmarts/Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime.h	??
applications/modelSpecific/arenaSmarts/Smart_ArrivalsEntityTypeVsAttribute.h	??
applications/modelSpecific/arenaSmarts/Smart_AssignExample.h	??
applications/modelSpecific/arenaSmarts/Smart_AutomaticStatisticsCollection.h	??
applications/modelSpecific/arenaSmarts/Smart_BasicModeling.h	??
applications/modelSpecific/arenaSmarts/Smart_BatchAndSeparate.h	??
applications/modelSpecific/arenaSmarts/Smart_ContinuousFlowEntities.h	??
applications/modelSpecific/arenaSmarts/Smart_Create.h	??
applications/modelSpecific/arenaSmarts/Smart_DecideNWayByChance.h	??
applications/modelSpecific/arenaSmarts/Smart_DefiningAttributesAsStrings.h	??
applications/modelSpecific/arenaSmarts/Smart_DefiningControlLogic.h	??
applications/modelSpecific/arenaSmarts/Smart_DefiningResourceCapacity.h	??
applications/modelSpecific/arenaSmarts/Smart_DelayBasedOnReplication.h	??
applications/modelSpecific/arenaSmarts/Smart_EntitiesProcessedByPriority.h	??
applications/modelSpecific/arenaSmarts/Smart_EvaluatingConditionsBeforeEnteringQueue.h	??
applications/modelSpecific/arenaSmarts/Smart_Expression.h	??
applications/modelSpecific/arenaSmarts/Smart_InventoryAndHoldingCosts.h	??
applications/modelSpecific/arenaSmarts/Smart_MaxArrivalsField.h	??
applications/modelSpecific/arenaSmarts/Smart_ModelRunUntil1000Parts.h	??
applications/modelSpecific/arenaSmarts/Smart_ModuleDisplayVariables.h	??
applications/modelSpecific/arenaSmarts/Smart_OverlappingResources.h	??
applications/modelSpecific/arenaSmarts/Smart_ParallelProcessingOfEntities.h	??
applications/modelSpecific/arenaSmarts/Smart_PlacingEntitiesInQueueSets.h	??
applications/modelSpecific/arenaSmarts/Smart_PriorityExample.h	??
applications/modelSpecific/arenaSmarts/Smart_ProcessArena.h	??
applications/modelSpecific/arenaSmarts/Smart_Record_Arena.h	??
applications/modelSpecific/arenaSmarts/Smart_RemovingAndReorderingEntitiesInAQueue.h	??
applications/modelSpecific/arenaSmarts/Smart_ResourceCosting.h	??
applications/modelSpecific/arenaSmarts/Smart_ResourceScheduleCosting.h	??
applications/modelSpecific/arenaSmarts/Smart_ResourceSets.h	??
applications/modelSpecific/arenaSmarts/Smart_SeizingMultipleSimultaneously.h	??
applications/modelSpecific/arenaSmarts/Smart_SelectingRouteBasedOnProbability.h	??
applications/modelSpecific/arenaSmarts/Smart_SelectingShorterQueue.h	??
applications/modelSpecific/arenaSmarts/Smart_SynchronizingParallelEntities.h	??
applications/modelSpecific/arenaSmarts/Smart_ValueAdded.h	??
applications/modelSpecific/arenaSmarts/Smart_WaitForSignal.h	??
applications/modelSpecific/book/Book_Cap02_Example01.h	??
applications/modelSpecific/smarts/AI/StartAIAssistant.h	??
applications/modelSpecific/smarts/AnalyticalModeling/Smart_MarkovChain.h	??
applications/modelSpecific/smarts/BiochemicalSimulation/Smart_BacteriaColony_GRO.h	??

applications/modelSpecific/smarts/BiochemicalSimulation/Smart_BioKineticLawRegulation.h	??
applications/modelSpecific/smarts/BiochemicalSimulation/Smart_BioNetworkAnalysis.h	??
applications/modelSpecific/smarts/BiochemicalSimulation/Smart_BioReversibleKineticLaw.h	??
applications/modelSpecific/smarts/BiochemicalSimulation/Smart_BioReversibleMassAction.h	??
applications/modelSpecific/smarts/BiochemicalSimulation/Smart_GroColonyGrowth.h	??
applications/modelSpecific/smarts/BiochemicalSimulation/Smart_GroColonyLifecycle.h	??
applications/modelSpecific/smarts/Continuous/Smart_LSODE.h	??
applications/modelSpecific/smarts/Continuous/Smart_ODE.h	??
applications/modelSpecific/smarts/DiscreteProcessing/Smart_Buffer.h	??
applications/modelSpecific/smarts/DiscreteProcessing/Smart_Clone.h	??
applications/modelSpecific/smarts/DiscreteProcessing/Smart_Delay.h	??
applications/modelSpecific/smarts/DiscreteProcessing/Smart_Failures.h	??
applications/modelSpecific/smarts/DiscreteProcessing/Smart_HoldSearchRemove.h	??
applications/modelSpecific/smarts/DiscreteProcessing/Smart_Process.h	??
applications/modelSpecific/smarts/DiscreteProcessing/Smart_ProcessSet.h	??
applications/modelSpecific/smarts/DiscreteProcessing/Smart_SeizeDelayRelease.h	??
applications/modelSpecific/smarts/DiscreteProcessing/Smart_SeizeDelayReleaseMany.h	??
applications/modelSpecific/smarts/DiscreteProcessing/Smart_SeizeDelayReleaseNoDataDefs.h	??
applications/modelSpecific/smarts/ExternalIntegration/Smart_CppForG.h	??
applications/modelSpecific/smarts/ExternalIntegration/Smart_PythonForG.h	??
applications/modelSpecific/smarts/ExternalIntegration/Smart_R_Simulator.h	??
applications/modelSpecific/smarts/Grouping/Smart_BatchSeparate.h	??
applications/modelSpecific/smarts/InputOutput/Smart_Record.h	??
applications/modelSpecific/smarts/Logic/Smart_AssignWriteSeizes.h	??
applications/modelSpecific/smarts/Logic/Smart_Attribute_Variable.h	??
applications/modelSpecific/smarts/Logic/Smart_ModelInfoModelSimulation.h	??
applications/modelSpecific/smarts/Logic/Smart_Parser.h	??
applications/modelSpecific/smarts/Logic/Smart_ParserModelFunctions.h	??
applications/modelSpecific/smarts/Logic/Smart_Plugin.h	??
applications/modelSpecific/smarts/MaterialHandling/Smart_RouteStation.h	??
applications/modelSpecific/smarts/MaterialHandling/Smart_Sequence.h	??
applications/modelSpecific/smarts/ModalModel/Smart_CellularAutomata.h	??
applications/modelSpecific/smarts/ModalModel/Smart_CellularAutomata1.h	??
applications/modelSpecific/smarts/ModalModel/Smart_DefaultModalModel.h	??
applications/modelSpecific/smarts/ModalModel/Smart_Dummy.h	??
applications/modelSpecific/smarts/ModalModel/Smart_FSMState.h	??
applications/modelSpecific/smarts/ModalModel/Smart_Modelmodel.h	??
applications/modelSpecific/smarts/ModalModel/Smart_ModelModelDefaultNetwork.h	??
applications/modelSpecific/smarts/ModalModel/Smart_ModelModelFSM.h	??
applications/modelSpecific/smarts/ModalModel/Smart_ModelModelPetriNet.h	??
applications/modelSpecific/smarts/ModalModel/Smart_PetriPlace.h	??
applications/modelSpecific/smarts/Synchronization/Smart_OnEvent.h	??
applications/modelSpecific/smarts/Synchronization/Smart_SimulationControlResponse.h	??
applications/modelSpecific/smarts/Synchronization/Smart_WaitScanCondition.h	??
applications/modelSpecific/smarts/Synchronization/Smart_WaitSignal.h	??
applications/modelSpecific/teaching/AnElectronicAssemblyAndTestSystem.h	??
applications/modelSpecific/teaching/BufferFIFO.h	??
applications/modelSpecific/teaching/ContinuousModel.h	??
applications/modelSpecific/teaching/FullSimulationOfComplexModel.h	??
applications/modelSpecific/teaching/Half_Adder.h	??
applications/modelSpecific/teaching/OperatingSystem02.h	??
applications/modelSpecific/teaching/OperatingSystem03.h	??
applications/modelSpecific/teaching/Rectifier.h	??
applications/modelSpecific/teaching/Loja/Loja01.h	??
applications/modelSpecific/wcm/WcmBioNetworkBridge.h	??
applications/modelSpecific/wcm/WcmCompartmentTransportWholeCellMvp.h	??
applications/modelSpecific/wcm/WcmFluxBalanceWholeCellMvp.h	??
applications/modelSpecific/wcm/WcmGeneExpression.h	??

applications/modelSpecific/wcm/WcmLifecycleStress.h	??
applications/modelSpecific/wcm/WcmMgenitaliumKarr2012.h	??
applications/modelSpecific/wcm/WcmPathwayStressWholeCellMvp.h	??
applications/modelSpecific/wcm/WcmSimpleSsa.h	??
applications/modelSpecific/wcm/WcmWholeCellMvp.h	??
applications/modelSpecific/wcm/WcmYeastDidacticWholeCellMvp.h	??
applications/modelSpecific/wcm/WcmYeastEukaryoticWholeCellMvp.h	??
applications/shell/GenesysShell.h	??
applications/shell/GenesysShell_if.h	??
applications/worker/BaseGenesysWebApplication.h	??
applications/worker/api/ApiRouter.h	??
applications/worker/auth/TokenService.h	??
applications/worker/http/HttpRequest.h	??
applications/worker/http/HttpResponse.h	??
applications/worker/http/SimpleHttpServer.h	??
applications/worker/service/SimulatorSessionService.h	??
applications/worker/service/WebWorkerRuntime.h	??
applications/worker/session/SessionContext.h	??
applications/worker/session/SessionManager.h	??
applications/worker/worker/WorkerJob.h	??
applications/worker/worker/WorkerJobManager.h	??
gtest/gtest/gtest-assertion-result.h	??
gtest/gtest/gtest-death-test.h	??
gtest/gtest/gtest-matchers.h	??
gtest/gtest/gtest-message.h	??
gtest/gtest/gtest-param-test.h	??
gtest/gtest/gtest-printers.h	??
gtest/gtest/gtest-spi.h	??
gtest/gtest/gtest-test-part.h	??
gtest/gtest/gtest-typed-test.h	??
gtest/gtest/gtest.h	??
gtest/gtest/gtest_pred_impl.h	??
gtest/gtest/gtest_prod.h	??
gtest/gtest/internal/gtest-death-test-internal.h	??
gtest/gtest/internal/gtest-filepath.h	??
gtest/gtest/internal/gtest-internal.h	??
gtest/gtest/internal/gtest-param-util.h	??
gtest/gtest/internal/gtest-port-arch.h	??
gtest/gtest/internal/gtest-port.h	??
gtest/gtest/internal/gtest-string.h	??
gtest/gtest/internal/gtest-type-util.h	??
gtest/gtest/internal/custom/gtest-port.h	??
gtest/gtest/internal/custom/gtest-printers.h	??
gtest/gtest/internal/custom/gtest.h	??
gtest/src/gtest-assertion-result.cc	??
gtest/src/gtest-death-test.cc	??
gtest/src/gtest-filepath.cc	??
gtest/src/gtest-internal-inl.h	??
gtest/src/gtest-matchers.cc	??
gtest/src/gtest-port.cc	??
gtest/src/gtest-printers.cc	??
gtest/src/gtest-test-part.cc	??
gtest/src/gtest-typed-test.cc	??
gtest/src/gtest.cc	??
kernel/TraitsKernel.h	??
kernel/simulator/ConnectionManager.h	??
kernel/simulator/DefineGetterSetter.h	??
kernel/simulator/Event.h	??

kernel/simulator/ExperimentManager.h	??
kernel/simulator/ExperimentManagerDefaultImpl1.h	??
kernel/simulator/ExperimentManager_if.h	??
kernel/simulator/GenesysPropertyIntrospection.h	??
kernel/simulator/LicenceManager.h	??
kernel/simulator/OnEventManager.h	??
kernel/simulator/Parser_if.h	??
kernel/simulator/ParserChangesInformation.h	??
kernel/simulator/ParserDefaultImpl1.h	??
kernel/simulator/ParserDefaultImpl2.h	??
kernel/simulator/ParserManager.h	??
kernel/simulator/Persistence.h	??
kernel/simulator/Plugin.h	??
kernel/simulator/PluginConnector_if.h	??
kernel/simulator/PluginInformation.h	??
kernel/simulator/PluginManager.h	??
kernel/simulator/PropertyGenesys.h	??
kernel/simulator/ScenarioExperiment_if.h	??
kernel/simulator/SimulationControlAndResponse.h	??
kernel/simulator/SimulationExperiment.h	??
kernel/simulator/SimulationReporter_if.h	??
kernel/simulator/SimulationReporterDefaultImpl1.h	??
kernel/simulator/SimulationScenario.h	??
kernel/simulator/Simulator.h	??
kernel/simulator/SimulatorFacade.h	??
kernel/simulator/SinkModelComponent.h	??
kernel/simulator/SourceModelComponent.h	??
kernel/simulator/SystemDependencyResolver.h	??
kernel/simulator/TraceManager.h	??
kernel/simulator/essentialPlugins/Attribute.h	??
kernel/simulator/essentialPlugins/Counter.h	??
kernel/simulator/essentialPlugins/Entity.h	??
kernel/simulator/essentialPlugins/EntityType.h	??
kernel/simulator/essentialPlugins/SparseValueStore.h	??
kernel/simulator/essentialPlugins/StatisticsCollector.h	??
kernel/simulator/model/Model.h	??
kernel/simulator/model/ModelChecker_if.h	??
kernel/simulator/model/ModelCheckerDefaultImpl1.h	??
kernel/simulator/model/ModelComponent.h	??
kernel/simulator/model/ModelComponentManager.h	??
kernel/simulator/model/ModelDataDefinition.h	??
kernel/simulator/model/ModelDataDefinitionAssociations.h	??
kernel/simulator/model/ModelDataManager.h	??
kernel/simulator/model/ModelInfo.h	??
kernel/simulator/model/ModelManager.h	??
kernel/simulator/model/ModelSimulation.h	??
kernel/simulator/persistence/CppSerializer.h	??
kernel/simulator/persistence/GenSerializer.h	??
kernel/simulator/persistence/JsonSerializer.h	??
kernel/simulator/persistence/Persistence_if.h	??
kernel/simulator/persistence/PersistenceDefaultImpl2.h	??
kernel/simulator/persistence/PersistencePartialLoadImpl1.h	??
kernel/simulator/persistence/Serializer.h	??
kernel/simulator/persistence/XmlSerializer.h	??
kernel/statistics/Collector_if.h	??
kernel/statistics/CollectorDatafile_if.h	??
kernel/statistics/CollectorDatafileDefaultImpl1.h	??
kernel/statistics/CollectorDefaultImpl1.h	??

kernel/statistics/ Sampler_if.h	??
kernel/statistics/ SamplerDefaultImpl1.h	??
kernel/statistics/ SorttFile.h	??
kernel/statistics/ Statistics_if.h	??
kernel/statistics/ StatisticsDataFile_if.h	??
kernel/statistics/ StatisticsDataFileDefaultImpl.h	??
kernel/statistics/ StatisticsDefaultImpl1.h	??
kernel/util/ Exact.h	??
kernel/util/ List.h	??
kernel/util/ ListObservable.h	??
kernel/util/ Util.h	??
parser/ Genesys++-driver.h	??
parser/ GenesysParser.h	??
parser/ location.hh	??
parser/ obj_t.h	??
parser/ position.hh	??
parser/ stack.hh	??
plugins/ PluginConnectorDummyImpl1.h	??
plugins/ PluginConnectorStaticImpl1.h	??
plugins/components/ AI/AIAssistant.h	??
plugins/components/ AnalyticalModeling/MarkovChain.h	??
plugins/components/ BiochemicalSimulation/BacteriaColony.h	??
plugins/components/ BiochemicalSimulation/BioRunnerCommand.h	??
plugins/components/ BiochemicalSimulation/BioSimulate.h	??
plugins/components/ BiochemicalSimulation/BioSteadyState.h	??
plugins/components/ BiochemicalSimulation/GeneticCircuitSimulate.h	??
plugins/components/ BiochemicalSimulation/GeneticExpressionStep.h	??
plugins/components/ BiochemicalSimulation/MetabolicFluxBalance.h	??
plugins/components/ Continuous/ContinuousSystemComponent.h	??
plugins/components/ Continuous/DiffEquations.h	??
plugins/components/ Continuous/DiffusionSimulate.h	??
plugins/components/ Continuous/LSODE.h	??
plugins/components/ Continuous/OLD_ODEelement.h	??
plugins/components/ Decisions/Decide.h	??
plugins/components/ Decisions/DropOff.h	??
plugins/components/ Decisions/PickUp.h	??
plugins/components/ Decisions/Remove.h	??
plugins/components/ Decisions/Search.h	??
plugins/components/ DiscreteProcessing/Buffer.h	??
plugins/components/ DiscreteProcessing/Clone.h	??
plugins/components/ DiscreteProcessing/Delay.h	??
plugins/components/ DiscreteProcessing/Process.h	??
plugins/components/ DiscreteProcessing/Release.h	??
plugins/components/ DiscreteProcessing/Seize.h	??
plugins/components/ DiscreteProcessing/auxiliar/QueueableItem.h	??
plugins/components/ DiscreteProcessing/auxiliar/SeizableItem.h	??
plugins/components/ ElectronicsSimulation/SPICECircuit.h	??
plugins/components/ ElectronicsSimulation/SPICENode.h	??
plugins/components/ ExternalIntegration/CppForG.h	??
plugins/components/ ExternalIntegration/PythonForG.h	??
plugins/components/ ExternalIntegration/RSimulator.h	??
plugins/components/ Grouping/Batch.h	??
plugins/components/ Grouping/Separate.h	??
plugins/components/ InputOutput/Record.h	??
plugins/components/ InputOutput/Write.h	??
plugins/components/ Logic/Assign.h	??
plugins/components/ Logic/Create.h	??
plugins/components/ Logic/Dispose.h	??

plugins/components/MaterialHandling/Access.h	??
plugins/components/MaterialHandling/Enter.h	??
plugins/components/MaterialHandling/Exit.h	??
plugins/components/MaterialHandling/Leave.h	??
plugins/components/MaterialHandling/PickableStationItem.h	??
plugins/components/MaterialHandling/PickStation.h	??
plugins/components/MaterialHandling/Route.h	??
plugins/components/MaterialHandling/Start.h	??
plugins/components/MaterialHandling/Stop.h	??
plugins/components/MaterialHandling/Store.h	??
plugins/components/MaterialHandling/Unstore.h	??
plugins/components/ModalModel/CellularAutomataComp.h	??
plugins/components/ModalModel/DefaultNode.h	??
plugins/components/ModalModel/DefaultTransitionExtensions.h	??
plugins/components/ModalModel/FSMState.h	??
plugins/components/ModalModel/ModalModelDefault.h	??
plugins/components/ModalModel/ModalModelFSM.h	??
plugins/components/ModalModel/ModalModelPetriNet.h	??
plugins/components/ModalModel/PetriPlace.h	??
plugins/components/ModalModel/Submodel.h	??
plugins/components/ModalModel/CellularAutomata/Boundary_Closed.h	??
plugins/components/ModalModel/CellularAutomata/Boundary_Fixed.h	??
plugins/components/ModalModel/CellularAutomata/BoundaryCondition.h	??
plugins/components/ModalModel/CellularAutomata/Cell.h	??
plugins/components/ModalModel/CellularAutomata/CellularAutomata.h	??
plugins/components/ModalModel/CellularAutomata/CellularAutomata_1DTimed.h	??
plugins/components/ModalModel/CellularAutomata/CellularAutomata_Classic.h	??
plugins/components/ModalModel/CellularAutomata/CellularAutomataAllHeaders.h	??
plugins/components/ModalModel/CellularAutomata/CellularAutomataBase.h	??
plugins/components/ModalModel/CellularAutomata/Lattice.h	??
plugins/components/ModalModel/CellularAutomata/LocalRule.h	??
plugins/components/ModalModel/CellularAutomata/LocalRule_Elementary.h	??
plugins/components/ModalModel/CellularAutomata/LocalRule_FlorestalFire.h	??
plugins/components/ModalModel/CellularAutomata/LocalRule_GameOfLife.h	??
plugins/components/ModalModel/CellularAutomata/LocalRule_Growthy.h	??
plugins/components/ModalModel/CellularAutomata/LocalRule_HppLatticeGas.h	??
plugins/components/ModalModel/CellularAutomata/LocalRule_SandRock.h	??
plugins/components/ModalModel/CellularAutomata/LocalRule_SandRock_Student.h	??
plugins/components/ModalModel/CellularAutomata/LocalRule_ShowNeighborhood.h	??
plugins/components/ModalModel/CellularAutomata/Neighborhood.h	??
plugins/components/ModalModel/CellularAutomata/Neighborhood_Center.h	??
plugins/components/ModalModel/CellularAutomata/Neighborhood_Moore.h	??
plugins/components/ModalModel/CellularAutomata/Neighborhood_VonNeumann.h	??
plugins/components/ModalModel/CellularAutomata/State.h	??
plugins/components/ModalModel/CellularAutomata/State_Numerical.h	??
plugins/components/ModalModel/CellularAutomata/StateSet.h	??
plugins/components/ModalModel/CellularAutomata/StateSet_Enumerable.h	??
plugins/components/Synchronization/Match.h	??
plugins/components/Synchronization/Signal.h	??
plugins/components/Synchronization/Wait.h	??
plugins/components/Template/DummyComponent.h	??
plugins/components/WholeCellModeling/BioStateProjectionComponent.h	??
plugins/components/WholeCellModeling/CellCycleCheckpointComponent.h	??
plugins/components/WholeCellModeling/CellDivisionEvent.h	??
plugins/components/WholeCellModeling/CellFateDecisionComponent.h	??
plugins/components/WholeCellModeling/CellGrowthComponent.h	??
plugins/components/WholeCellModeling/CompartmentExchangeComponent.h	??
plugins/components/WholeCellModeling/EukaryoticCellCycleComponent.h	??

plugins/components/WholeCellModeling/FtsZPolymerizationComponent.h	??
plugins/components/WholeCellModeling/MetabolicStateProjectionComponent.h	??
plugins/components/WholeCellModeling/MetabolicSubmodelComponent.h	??
plugins/components/WholeCellModeling/PathwayStressResponseComponent.h	??
plugins/components/WholeCellModeling/ResourceAllocationComponent.h	??
plugins/components/WholeCellModeling/StochasticReactionComponent.h	??
plugins/components/WholeCellModeling/StochasticTranscription.h	??
plugins/components/WholeCellModeling/StochasticTranslation.h	??
plugins/data/AI/AISupport.h	??
plugins/data/BiochemicalSimulation/BacteriaSignalGrid.h	??
plugins/data/BiochemicalSimulation/BioNetwork.h	??
plugins/data/BiochemicalSimulation/BioParameter.h	??
plugins/data/BiochemicalSimulation/BioReaction.h	??
plugins/data/BiochemicalSimulation/BioSBMLBridge.h	??
plugins/data/BiochemicalSimulation/BioSimulatorRunner.h	??
plugins/data/BiochemicalSimulation/BioSpecies.h	??
plugins/data/BiochemicalSimulation/GeneticCircuit.h	??
plugins/data/BiochemicalSimulation/GeneticCircuitPart.h	??
plugins/data/BiochemicalSimulation/GeneticRegulation.h	??
plugins/data/BiochemicalSimulation/GroProgram.h	??
plugins/data/BiochemicalSimulation/GroProgramAst.h	??
plugins/data/BiochemicalSimulation/GroProgramCompiler.h	??
plugins/data/BiochemicalSimulation/GroProgramIrr.h	??
plugins/data/BiochemicalSimulation/GroProgramParser.h	??
plugins/data/BiochemicalSimulation/GroProgramRuntime.h	??
plugins/data/BiochemicalSimulation/MetabolicNetwork.h	??
plugins/data/BiochemicalSimulation/MetabolicReaction.h	??
plugins/data/Continuous/DiffusionField.h	??
plugins/data/Continuous/ODESolver.h	??
plugins/data/DiscreteProcessing/Failure.h	??
plugins/data/DiscreteProcessing/Queue.h	??
plugins/data/DiscreteProcessing/Resource.h	??
plugins/data/DiscreteProcessing/Schedule.h	??
plugins/data/ExternalIntegration/CppCompiler.h	??
plugins/data/ExternalIntegration/OctaveRunner.h	??
plugins/data/ExternalIntegration/PythonRuntime.h	??
plugins/data/ExternalIntegration/RSimulatorRunner.h	??
plugins/data/ExternalIntegration/SPICERunner.h	??
plugins/data/Grouping/EntityGroup.h	??
plugins/data/InputOutput/File.h	??
plugins/data/Logic/AssignmentItem.h	??
plugins/data/Logic/Formula.h	??
plugins/data/Logic/Label.h	??
plugins/data/Logic/Set.h	??
plugins/data/Logic/Variable.h	??
plugins/data/MaterialHandling/Sequence.h	??
plugins/data/MaterialHandling/Station.h	??
plugins/data/MaterialHandling/Storage.h	??
plugins/data/Synchronization/SignalData.h	??
plugins/data/Template/DummyElement.h	??
plugins/data/WholeCellModeling/BioCompartment.h	??
plugins/data/WholeCellModeling/MolecularSpecies.h	??
plugins/data/WholeCellModeling/StochasticReactionRule.h	??
plugins/data/WholeCellModeling/WholeCellState.h	??
tools/TraitsTools.h	??
tools/AIAssistant/AIAssistant_if.h	??
tools/AIAssistant/AIAssistantDefaultImpl.h	??
tools/AIAssistant/AIAuditLog.h	??

tools/AIAssistant/AIConversationService.h	??
tools/AIAssistant/AIProviderClient_if.h	??
tools/AIAssistant/AISecretStore.h	??
tools/AIAssistant/AnthropicProviderClientImpl.h	??
tools/AIAssistant/HttpProviderClientBase.h	??
tools/AIAssistant/LocalProviderClientImpl.h	??
tools/AIAssistant/OpenAIProviderClientImpl.h	??
tools/Biochemical/BioKineticLawExpression.h	??
tools/Biochemical/BioSimulationAnalysis.h	??
tools/Biochemical/BioSimulationResult.h	??
tools/Biochemical/GlpkFluxBalanceSolver.h	??
tools/Biochemical/MassActionOdeSystem.h	??
tools/Biochemical/MetabolicFluxBalanceSolver.h	??
tools/Biochemical/WholeCellParameterReader.h	??
tools/Continuous/DiffusionMethodOfLinesSystem.h	??
tools/Continuous/DormandPrince54OdeSolver.h	??
tools/Continuous/OdeSolver_if.h	??
tools/Continuous/OdeSolverFactory.h	??
tools/Continuous/OdeSystem_if.h	??
tools/Continuous/Quadrature_if.h	??
tools/Continuous/RootFinder_if.h	??
tools/Continuous/RungeKutta4OdeSolver.h	??
tools/Continuous/Solver_if.h	??
tools/Continuous/SolverDefaultImpl1.h	??
tools/FactorialDesign/FactorialDesign.h	??
tools/Optimization/Optimizer_if.h	??
tools/Optimization/OptimizerDefaultImpl1.h	??
tools/Statistics/ContinuousDistribution_if.h	??
tools/Statistics/DataAnalyser_if.h	??
tools/Statistics/DataSet_if.h	??
tools/Statistics/DiscreteDistribution_if.h	??
tools/Statistics/Distribution_if.h	??
tools/Statistics/Fitter_if.h	??
tools/Statistics/FitterDefaultImpl.h	??
tools/Statistics/FitterDummyImpl.h	??
tools/Statistics/HypothesisTester_if.h	??
tools/Statistics/HypothesisTesterDefaultImpl1.h	??
tools/Statistics/ProbabilityDistribution.h	??
tools/Statistics/ProbabilityDistributionBase.h	??
tools/Statistics/SimulationResultsDataset.h	??

Chapter 14

Namespace Documentation

14.1 generate_kernel_simulator_test_stubs Namespace Reference

Classes

- class [MethodEntry](#)

Functions

- str **_sanitize_identifier** (str text)
- bool **_is_method_candidate** (str line)
- list[[MethodEntry](#)] **parse_header** (pathlib.Path path)
- str **generate_test_source** (list[[MethodEntry](#)] methods)
- None **main** ()

Variables

- **CLASS_RE** = re.compile(r"^\s*class\s+([A-Za-z_]\w*)\b")
- **METHOD_RE** = re.compile(r"(")

14.1.1 Detailed Description

Generate gtest inventory tests for every method declaration found in source/kernel/simulator headers.

The goal is to provide a scalable scaffold so each class/method has a corresponding unit-test entry that can be progressively implemented.

14.2 genesys_mcp Namespace Reference

Namespaces

- namespace [__main__](#)
- namespace [client](#)
- namespace [server](#)

14.2.1 Detailed Description

Genesys Simulator MCP Connector.

14.3 genesys_mcp.__main__ Namespace Reference

14.3.1 Detailed Description

Allow running the MCP server with: `python -m genesys_mcp`

14.4 genesys_mcp.client Namespace Reference

Classes

- class [GenesysClient](#)
- class [GenesysClientError](#)

14.4.1 Detailed Description

HTTP client wrapping the Genesys Worker Application REST API.

14.5 genesys_mcp.server Namespace Reference

Functions

- [GenesysClient](#) **_client** (str base_url, str|None token=None)
- dict[str, str] **_err** ([GenesysClientError](#) e)
- dict[str, str] **_conn_err** (httpx.RequestError e)
- dict[str, Any] [genesys_health](#) (str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_auth](#) (str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_info](#) (str token, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_new_model](#) (str token, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_get_model](#) (str token, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_save_model](#) (str token, str filename, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_load_model](#) (str token, str filename, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_import_model_language](#) (str token, str model_text, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_simulation_status](#) (str token, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_configure_simulation](#) (str token, int number_of_replications, float replication_length, float warm_up_period=0.0, bool pause_on_event=False, bool pause_on_replication=False, bool initialize_statistics=True, bool initialize_system=True, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_run_simulation](#) (str token, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_step_simulation](#) (str token, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_worker_info](#) (str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_worker_capabilities](#) (str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_create_job](#) (str token, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_job_status](#) (str token, str job_id, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_run_job](#) (str token, str job_id, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_job_result](#) (str token, str job_id, str base_url=_DEFAULT_URL)
- None **main** ()

Variables

- [mcp](#)
- **_DEFAULT_URL** = os.environ.get("GENESYS_URL", "http://localhost:8080")

14.5.1 Detailed Description

Genesys MCP Server -- exposes GenESyS simulation capabilities via MCP.

14.5.2 Function Documentation

14.5.2.1 genesys_auth()

```
dict[str, Any] genesys_mcp.server.genesys_auth (
    str base_url = _DEFAULT_URL )
```

Create a new Genesys session and return the bearer token.

Returns {"token": "<bearer-token>", ...}.

Pass the token to all subsequent tools as the 'token' parameter.

14.5.2.2 genesys_configure_simulation()

```
dict[str, Any] genesys_mcp.server.genesys_configure_simulation (
    str token,
    int number_of_replications,
    float replication_length,
    float warm_up_period = 0.0,
    bool pause_on_event = False,
    bool pause_on_replication = False,
    bool initialize_statistics = True,
    bool initialize_system = True,
    str base_url = _DEFAULT_URL )
```

Configure simulation parameters before running.

Args:

token: Bearer token from genesys_auth.
number_of_replications: How many independent replications to run.
replication_length: Simulated time length of each replication.
warm_up_period: Simulated time before statistics start being collected.
pause_on_event: Pause simulation after each event (step-by-step mode).
pause_on_replication: Pause between replications.
initialize_statistics: Reset statistics counters at start of each replication.
initialize_system: Reset system state at start of each replication.
base_url: Genesys worker app base URL.

14.5.2.3 genesys_create_job()

```
dict[str, Any] genesys_mcp.server.genesys_create_job (
    str token,
    str base_url = _DEFAULT_URL )
```

Create a worker job from the session's current model.

The job is created in 'queued' state. A snapshot of the model is persisted so later execution is decoupled from live model mutations. Returns {"jobId": "<id>", "state": "queued", ...}. Requires a valid session token from genesys_auth.

14.5.2.4 genesys_get_model()

```
dict[str, Any] genesys_mcp.server.genesys_get_model (
    str token,
    str base_url = _DEFAULT_URL )
```

Return a description of the session's current simulation model.

Requires a valid session token from genesys_auth.

14.5.2.5 genesys_health()

```
dict[str, Any] genesys_mcp.server.genesys_health (
    str base_url = _DEFAULT_URL )
```

Check that the Genesys worker application is reachable.

Returns the health status object from GET /health.
No authentication required.

14.5.2.6 genesys_import_model_language()

```
dict[str, Any] genesys_mcp.server.genesys_import_model_language (
    str token,
    str model_text,
    str base_url = _DEFAULT_URL )
```

Import a simulation model from GenESyS model language text.

'model_text' is the full plain-text GenESyS model specification.
The imported model becomes the session's current model.
Requires a valid session token from genesys_auth.

14.5.2.7 genesys_info()

```
dict[str, Any] genesys_mcp.server.genesys_info (
    str token,
    str base_url = _DEFAULT_URL )
```

Return simulator version, name, and licence information.

Requires a valid session token from genesys_auth.

14.5.2.8 genesys_job_result()

```
dict[str, Any] genesys_mcp.server.genesys_job_result (
    str token,
    str job_id,
    str base_url = _DEFAULT_URL )
```

Retrieve the simulation result summary for a finished or failed job.

Only available after the job reaches a terminal state.
Requires a valid session token from genesys_auth.

14.5.2.9 genesys_job_status()

```
dict[str, Any] genesys_mcp.server.genesys_job_status (
    str token,
    str job_id,
    str base_url = _DEFAULT_URL )
```

Return the current state of a worker job (queued/running/finished/failed).

Requires a valid session token from genesys_auth.

14.5.2.10 genesys_load_model()

```
dict[str, Any] genesys_mcp.server.genesys_load_model (
    str token,
    str filename,
    str base_url = _DEFAULT_URL )
```

Load a model from a file inside the session workspace.

'filename' must be a basename (no path separators).
Requires a valid session token from genesys_auth.

14.5.2.11 genesys_new_model()

```
dict[str, Any] genesys_mcp.server.genesys_new_model (
    str token,
    str base_url = _DEFAULT_URL )
```

Create a new empty simulation model in the current session.

The new model becomes the session's current model.
Requires a valid session token from genesys_auth.

14.5.2.12 genesys_run_job()

```
dict[str, Any] genesys_mcp.server.genesys_run_job (
    str token,
    str job_id,
    str base_url = _DEFAULT_URL )
```

Execute a worker job synchronously.

The job must be in 'queued' state. This call blocks until the simulation finishes. Poll genesys_job_status or call genesys_job_result afterwards.
Requires a valid session token from genesys_auth.

14.5.2.13 genesys_run_simulation()

```
dict[str, Any] genesys_mcp.server.genesys_run_simulation (
    str token,
    str base_url = _DEFAULT_URL )
```

Run the current simulation synchronously and return a summary.

Configure simulation parameters first with genesys_configure_simulation.
Requires a valid session token from genesys_auth.

14.5.2.14 genesys_save_model()

```
dict[str, Any] genesys_mcp.server.genesys_save_model (
    str token,
    str filename,
    str base_url = _DEFAULT_URL )
```

Save the current model to a file inside the session workspace.

'filename' must be a basename (no path separators). Only alphanumeric characters, dots, hyphens, and underscores are allowed.
Requires a valid session token from genesys_auth.

14.5.2.15 genesys_simulation_status()

```
dict[str, Any] genesys_mcp.server.genesys_simulation_status (
    str token,
    str base_url = _DEFAULT_URL )
```

Return the current simulation status (state, replication, clock, etc.).

Requires a valid session token from genesys_auth.

14.5.2.16 genesys_step_simulation()

```
dict[str, Any] genesys_mcp.server.genesys_step_simulation (
    str token,
    str base_url = _DEFAULT_URL )
```

Advance the simulation by one event step.

Useful for step-by-step inspection. Configure with `pause_on_event=True` first. Requires a valid session token from `genesys_auth`.

14.5.2.17 genesys_worker_capabilities()

```
dict[str, Any] genesys_mcp.server.genesys_worker_capabilities (
    str base_url = _DEFAULT_URL )
```

Return the current worker's capability flags.

Describes what simulation features this worker instance supports. No authentication required.

14.5.2.18 genesys_worker_info()

```
dict[str, Any] genesys_mcp.server.genesys_worker_info (
    str base_url = _DEFAULT_URL )
```

Return worker identity metadata (id, version, role).

No authentication required.

14.5.3 Variable Documentation

14.5.3.1 mcp

```
genesys_mcp.server.mcp
```

Initial value:

```
00001 = FastMCP(
00002     "Genesys Simulator",
00003     instructions=(
00004         "Use these tools to create, configure, and run GenESyS discrete-event "
00005         "simulation models. Always call genesys_auth first to obtain a session "
00006         "token before using any other tool."
00007     ),
00008 )
```

14.6 GuiCrashDiagnostics Namespace Reference

Crash diagnostics helpers for the Qt GUI process.

Functions

- **void installFatalSignalHandlers ()**
Installs temporary fatal-signal handlers that print a crash backtrace.

14.6.1 Detailed Description

Crash diagnostics helpers for the Qt GUI process.

This namespace keeps temporary signal-handling utilities isolated from the rest of the GUI runtime so they can be installed only when needed during crash investigation.

14.7 QCP Namespace Reference

Enumerations

- enum [ResolutionUnit](#) { [ruDotsPerMeter](#) , [ruDotsPerCentimeter](#) , [ruDotsPerInch](#) }
- enum [ExportPen](#) { [epNoCosmetic](#) , [epAllowCosmetic](#) }
- enum [SignDomain](#) { [sdNegative](#) , [sdBoth](#) , [sdPositive](#) }
- enum [MarginSide](#) {
[msLeft](#) = 0x01 , [msRight](#) = 0x02 , [msTop](#) = 0x04 , [msBottom](#) = 0x08 ,
[msAll](#) = 0xFF , [msNone](#) = 0x00 }
- enum [AntialiasedElement](#) {
[aeAxes](#) = 0x0001 , [aeGrid](#) = 0x0002 , [aeSubGrid](#) = 0x0004 , [aeLegend](#) = 0x0008 ,
[aeLegendItems](#) = 0x0010 , [aePlottables](#) = 0x0020 , [aeItems](#) = 0x0040 , [aeScatters](#) = 0x0080 ,
[aeFills](#) = 0x0100 , [aeZeroLine](#) = 0x0200 , [aeOther](#) = 0x8000 , [aeAll](#) = 0xFFFF ,
[aeNone](#) = 0x0000 }
- enum [PlottingHint](#) { [phNone](#) = 0x000 , [phFastPolylines](#) = 0x001 , [phImmediateRefresh](#) = 0x002 ,
[phCacheLabels](#) = 0x004 }
- enum [Interaction](#) {
[iNone](#) = 0x000 , [iRangeDrag](#) = 0x001 , [iRangeZoom](#) = 0x002 , [iMultiSelect](#) = 0x004 ,
[iSelectPlottables](#) = 0x008 , [iSelectAxes](#) = 0x010 , [iSelectLegend](#) = 0x020 , [iSelectItems](#) = 0x040 ,
[iSelectOther](#) = 0x080 , [iSelectPlottablesBeyondAxisRect](#) = 0x100 }
- enum [SelectionRectMode](#) { [srmNone](#) , [srmZoom](#) , [srmSelect](#) , [srmCustom](#) }
- enum [SelectionType](#) {
[stNone](#) , [stWhole](#) , [stSingleData](#) , [stDataRange](#) ,
[stMultipleDataRanges](#) }

Functions

- bool **isInvalidData** (double value)
- bool **isInvalidData** (double value1, double value2)
- void **setMarginValue** (QMargins &margins, [QCP::MarginSide](#) side, int value)
- int **getMarginValue** (const QMargins &margins, [QCP::MarginSide](#) side)

Variables

- const QMetaObject **staticMetaObject**

14.7.1 Detailed Description

The [QCP](#) Namespace contains general enums, QFlags and functions used throughout the [QCustomPlot](#) library. It provides QMetaObject-based reflection of its enums and flags via [QCP::staticMetaObject](#).

14.7.2 Enumeration Type Documentation

14.7.2.1 AntialiasedElement

enum [QCP::AntialiasedElement](#)

Defines what objects of a plot can be forcibly drawn antialiased/not antialiased. If an object is neither forcibly drawn antialiased nor forcibly drawn not antialiased, it is up to the respective element how it is drawn. Typically it provides a *setAntialiased* function for this.

[AntialiasedElements](#) is a flag of or-combined elements of this enum type.

See also

[QCustomPlot::setAntialiasedElements](#), [QCustomPlot::setNotAntialiasedElements](#)

Enumerator

aeAxes	0x0001 Axis base line and tick marks
------------------------	--------------------------------------

Enumerator

aeGrid	0x0002 Grid lines
aeSubGrid	0x0004 Sub grid lines
aeLegend	0x0008 Legend box
aeLegendItems	0x0010 Legend items
aePlottables	0x0020 Main lines of plottables
aeItems	0x0040 Main lines of items
aeScatters	0x0080 Scatter symbols of plottables (excluding scatter symbols of type ssPixmap)
aeFills	0x0100 Borders of fills (e.g. under or between graphs)
aeZeroLine	0x0200 Zero-lines, see QCPGrid::setZeroLinePen
aeOther	0x8000 Other elements that don't fit into any of the existing categories
aeAll	0xFFFF All elements
aeNone	0x0000 No elements

14.7.2.2 ExportPen

enum [QCP::ExportPen](#)

Defines how cosmetic pens (pens with numerical width 0) are handled during export.

See also

[QCustomPlot::savePdf](#)

Enumerator

epNoCosmetic	Cosmetic pens are converted to pens with pixel width 1 when exporting.
epAllowCosmetic	Cosmetic pens are exported normally (e.g. in PDF exports, cosmetic pens always appear as 1 pixel on screen, independent of viewer zoom level)

14.7.2.3 Interaction

enum [QCP::Interaction](#)

Defines the mouse interactions possible with [QCustomPlot](#).

`Interactions` is a flag of or-combined elements of this enum type.

See also

[QCustomPlot::setInteractions](#)

Enumerator

iNone	0x000 None of the interactions are possible
iRangeDrag	0x001 Axis ranges are draggable (see QCPAxisRect::setRangeDrag , QCPAxisRect::setRangeDragAxes)
iRangeZoom	0x002 Axis ranges are zoomable with the mouse wheel (see QCPAxisRect::setRangeZoom , QCPAxisRect::setRangeZoomAxes)
iMultiSelect	0x004 The user can select multiple objects by holding the modifier set by QCustomPlot::setMultiSelectModifier while clicking
iSelectPlottables	0x008 Plottables are selectable (e.g. graphs, curves, bars,... see QCPAbstractPlottable)
iSelectAxes	0x010 Axes are selectable (or parts of them, see QCPAxis::setSelectableParts)

Enumerator

iSelectLegend	0x020 Legends are selectable (or their child items, see QCPLegend::setSelectableParts)
iSelectItems	0x040 Items are selectable (Rectangles, Arrows, Textitems, etc. see QCPAbstractItem)
iSelectOther	0x080 All other objects are selectable (e.g. your own derived layerables, other layout elements,...)
iSelectPlottablesBeyondAxisRect	0x100 When performing plottable selection/hit tests, this flag extends the sensitive area beyond the axis rect

14.7.2.4 MarginSide

enum [QCP::MarginSide](#)

Defines the sides of a rectangular entity to which margins can be applied.

See also

[QCPLayoutElement::setAutoMargins](#), [QCPAxisRect::setAutoMargins](#)

Enumerator

msLeft	0x01 left margin
msRight	0x02 right margin
msTop	0x04 top margin
msBottom	0x08 bottom margin
msAll	0xFF all margins
msNone	0x00 no margin

14.7.2.5 PlottingHint

enum [QCP::PlottingHint](#)

Defines plotting hints that control various aspects of the quality and speed of plotting.

See also

[QCustomPlot::setPlottingHints](#)

Enumerator

phNone	0x000 No hints are set
phFastPolylines	joins, thus is most effective for pen sizes larger than 1. It is only used for solid line pens. 0x001 Graph/Curve lines are drawn with a faster method. This reduces the quality especially of the line segment
phImmediateRefresh	This is set by default to prevent the plot from freezing on fast consecutive replots (e.g. user drags ranges with mouse). 0x002 causes an immediate repaint() instead of a soft update() when QCustomPlot::replot() is called with parameter QCustomPlot::rpRefreshHint .
phCacheLabels	0x004 axis (tick) labels will be cached as pixmaps, increasing replot performance.

14.7.2.6 ResolutionUnit

enum [QCP::ResolutionUnit](#)

Defines the different units in which the image resolution can be specified in the export functions.

See also

[QCustomPlot::savePng](#), [QCustomPlot::saveJpg](#), [QCustomPlot::saveBmp](#), [QCustomPlot::saveRastered](#)

Enumerator

ruDotsPerMeter	Resolution is given in dots per meter (dpm)
ruDotsPerCentimeter	Resolution is given in dots per centimeter (dpcm)
ruDotsPerInch	Resolution is given in dots per inch (DPI/PPI)

14.7.2.7 SelectionRectMode

enum [QCP::SelectionMode](#)

Defines the behaviour of the selection rect.

See also

[QCustomPlot::setSelectionMode](#), [QCustomPlot::selectionRect](#), [QCPSelectionRect](#)

Enumerator

srmNone	The selection rect is disabled, and all mouse events are forwarded to the underlying objects, e.g. for axis range dragging.
srmZoom	When dragging the mouse, a selection rect becomes active. Upon releasing, the axes that are currently set as range zoom axes (QCPAxisRect::setRangeZoomAxes) will have their ranges zoomed accordingly.
srmSelect	When dragging the mouse, a selection rect becomes active. Upon releasing, plottable data points that were within the selection rect are selected, if the plottable's selectability setting permits. (See data selection mechanism for details.)
srmCustom	When dragging the mouse, a selection rect becomes active. It is the programmer's responsibility to connect according slots to the selection rect's signals (e.g. QCPSelectionRect::accepted) in order to process the user interaction.

14.7.2.8 SelectionType

enum [QCP::SelectionType](#)

Defines the different ways a plottable can be selected. These images show the effect of the different selection types, when the indicated selection rect was dragged:

td	td	td	td	td
----	----	----	----	----

See also

[QCPAbstractPlottable::setSelectable](#), [QCPDataSelection::enforceType](#)

Enumerator

stNone	The plottable is not selectable.
stWhole	Selection behaves like stMultipleDataRanges , but if there are any data points selected, the entire plottable is drawn as selected.

Enumerator

stSingleData	One individual data point can be selected at a time.
stDataRange	Multiple contiguous data points (a data range) can be selected.
stMultipleDataRanges	Any combination of data points/ranges can be selected.

14.7.2.9 SignDomain

enum [QCP::SignDomain](#)

Represents negative and positive sign domain, e.g. for passing to [QCPAbstractPlottable::getKeyRange](#) and [QCPAbstractPlottable::getValueRange](#).

This is primarily needed when working with logarithmic axis scales, since only one of the sign domains can be visible at a time.

Enumerator

sdNegative	The negative sign domain, i.e. numbers smaller than zero.
sdBoth	Both sign domains, including zero, i.e. all numbers.
sdPositive	The positive sign domain, i.e. numbers greater than zero.

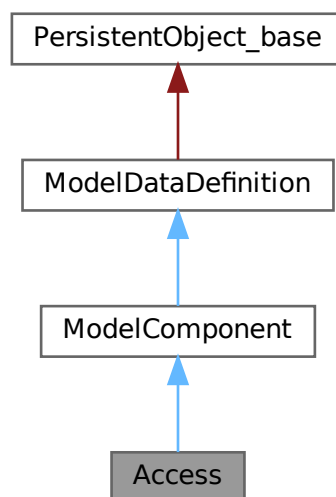
Chapter 15

Class Documentation

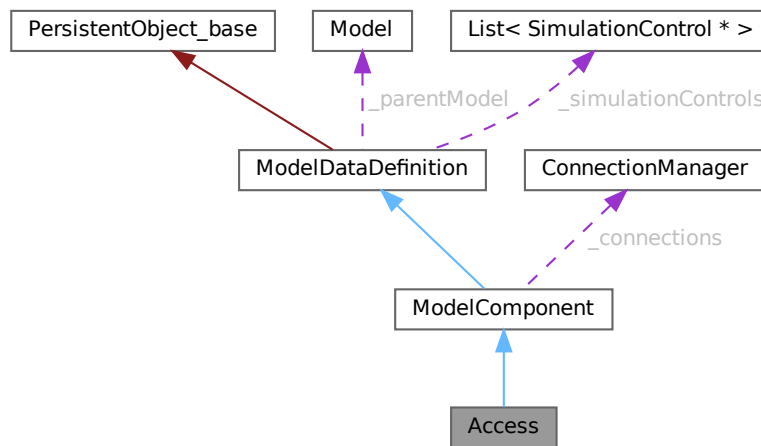
15.1 Access Class Reference

```
#include <plugins/components/MaterialHandling/Access.h>
```

Inheritance diagram for Access:



Collaboration diagram for Access:



Public Member Functions

- **Access** (**Model** *model, std::string name="")
 - virtual std::string **show** () override
- virtual public methods*

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↔ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.

- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- Model * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- ModelDataDefinition * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, ModelDataDefinition * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, ModelDataDefinition * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< SimulationControl * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) (TraceManager::Level level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static PluginInformation * [GetPluginInformation](#) ()
- static ModelComponent * [LoadInstance](#) (Model *model, PersistenceRecord *fields)
- static ModelDataDefinition * [NewInstance](#) (Model *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()

- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internaStatisticReportersClear** ()
- legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
- method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
- virtual protected methods that could be overridden by derived classes, if needed*

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel = 0`
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls = new List<SimulationControl*>()`
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled = false`

15.1.1 Detailed Description

[Access](#) module DESCRIPTION The [Access](#) module allocates one or more cells of a conveyor to an entity for movement from one station to another. Once the entity has control of the cells on the conveyor, it may then be conveyed to the next station. When an entity arrives at an [Access](#) module, it will wait until the appropriate number of contiguous cells on the conveyor are empty and aligned with the entity's station location. TYPICAL USES Parts accessing a conveyor to be sent to a paint booth Glass accessing a conveyor to be transferred to a cutting station PROMPTS Prompt Description Name Unique name of the module that will be displayed in the flowchart. Conveyor Name Name of the conveyor that the entity desires.

15.1.2 of Cells Number of contiguous conveyor cells the entity requires for

movement on the conveyor. [Queue](#) Type Determines the type of queue used to hold the entities, either an individual [Queue](#), a queue [Set](#), and Internal queue or an [Attribute](#) or Expression that evaluate to the queue name. [Queue](#) Name Name of the queue that will hold the entity until it accesses the conveyor. [Set](#) Name Name of the set of queues. [Set](#) Index Defines the index into the queue set. Note that this is the index into the set and not the name of the queue in the set. For example, the only valid entries for a queue set containing three members is an expression that evaluates to 1, 2, or 3. [Attribute](#) Name Defines the name of the attribute that stores the queue name to which entities will reside. Expression Defines the name of the expression that stores the queue name to which entities will reside.

15.1.3 Member Function Documentation

15.1.3.1 `_check()`

```
bool Access::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.1.3.2 `_loadInstance()`

```
bool Access::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [ModelComponent](#).

15.1.3.3 `_onDispatchEvent()`

```
void Access::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<code>entity</code>	Entity being processed.
---------------------	---

Parameters

<i>inputPortNumber</i>	Input port where the entity arrived.
------------------------	--------------------------------------

Implements [ModelComponent](#).

15.1.3.4 _saveInstance()

```
void Access::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.1.3.5 show()

```
std::string Access::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

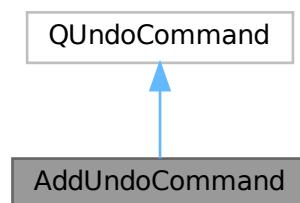
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

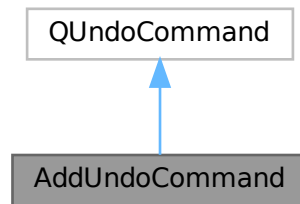
- plugins/components/MaterialHandling/Access.h
- plugins/components/MaterialHandling/Access.cpp

15.2 AddUndoCommand Class Reference

Inheritance diagram for AddUndoCommand:



Collaboration diagram for AddUndoCommand:



Public Member Functions

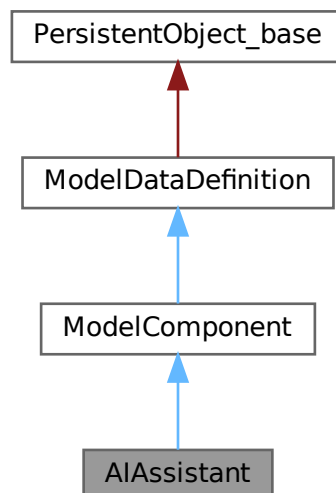
- **AddUndoCommand** (QGraphicsItem *item, [ModelGraphicsScene](#) *scene, QUndoCommand *parent=nullptr)
- void **undo** () override
- void **redo** () override

The documentation for this class was generated from the following files:

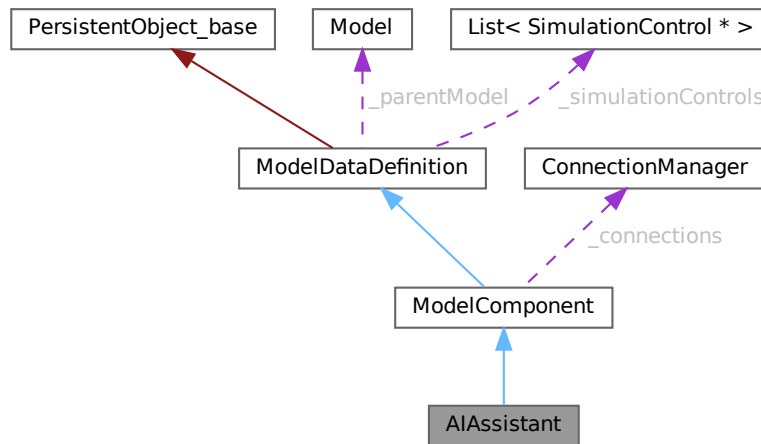
- applications/gui/genesys/actions/AddUndoCommand.h
- applications/gui/genesys/actions/AddUndoCommand.cpp

15.3 AIAssistant Class Reference

Inheritance diagram for AIAssistant:



Collaboration diagram for AIAssistant:



Public Member Functions

- **AIAssistant** ([Model](#) *model, std::string name="")
- void **setAISupport** ([AISupport](#) *support)
- [AISupport](#) * **getAISupport** () const
- void **setAISupportName** (std::string value)
- std::string **getAISupportName** () const
- void **setPromptTemplate** (std::string value)
- std::string **getPromptTemplate** () const
- void **setConversationScope** (std::string value)
- std::string **getConversationScope** () const
- void **setConversationKeyExpression** (std::string value)
- std::string **getConversationKeyExpression** () const
- void **setActionMappings** (std::string value)
- std::string **getActionMappings** () const
- void **setDelayExpression** (std::string value)
- std::string **getDelayExpression** () const
- void **setDefaultOutputPort** (unsigned int value)
- unsigned int **getDefaultOutputPort** () const
- void **setErrorOutputPort** (unsigned int value)
- unsigned int **getErrorOutputPort** () const
- void **setAllowDirectOutputPort** (bool value)
- bool **isAllowDirectOutputPort** () const
- void **setMinimumConfidence** (double value)
- double **getMinimumConfidence** () const
- void **setSaveConfidenceAttribute** (std::string value)
- std::string **getSaveConfidenceAttribute** () const
- void **setSaveSuccessAttribute** (std::string value)
- std::string **getSaveSuccessAttribute** () const
- std::string **getLastPrompt** () const
- std::string **getLastResponse** () const
- std::string **getLastError** () const
- bool **renderPrompt** (std::string &result, std::string &error)
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.

- void `setHasChanged` (bool `hasChanged`)
Updates the changed flag for this data definition.
- unsigned int `getLevel` () const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel` (unsigned int `_modelLevel`)
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls` () const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level` `getTraceLevelSpecific` () const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` `level`)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` `traceLevelSpecific`, bool `traceLevelSpecificEnabled=true`)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool `traceLevelSpecificEnabled`)
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation * GetPluginInformation` ()
- static `ModelComponent * LoadInstance` (`Model *model`, `PersistenceRecord *fields`)
- static `ModelDataDefinition * NewInstance` (`Model *model`, `std::string name=""`)

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent * LoadInstance` (`Model *model`, `PersistenceRecord *fields`)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check` (`ModelComponent *component`)
Validates a component instance.
- static void `CreateInternalData` (`ModelComponent *component`)
Creates internal data required by a component instance.
- static void `SaveInstance` (`PersistenceRecord *fields`, `ModelComponent *component`)
Serializes a component instance into a persistence record.
- static void `DispatchEvent` (`Event *event`)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition * LoadInstance` (`Model *model`, `PersistenceRecord *fields`, bool `insertIntoModel`)
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition * NewInstance` (`Model *model`, `std::string name=""`)
Creates a new instance of a data definition using the plugin registry.
- static void `SaveInstance` (`PersistenceRecord *fields`, `ModelDataDefinition *modeldatum`)
Serializes a data definition instance into a persistence record.
- static bool `Check` (`ModelDataDefinition *modeldatum`, `std::string &errorMessage`)
Validates a data definition instance.
- static void `InitBetweenReplications` (`ModelDataDefinition *modeldatum`)
Resets simulation-time state between replications.
- static void `CreateInternalData` (`ModelDataDefinition *modeldatum`)

Creates internal and attached data required by this instance.

- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)

Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()

- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.3.1 Member Function Documentation

15.3.1.1 **_check()**

```
bool AIAssistant::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
 Reimplemented from [ModelDataDefinition](#).

15.3.1.2 **_loadInstance()**

```
bool AIAssistant::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
 Reimplemented from [ModelComponent](#).

15.3.1.3 `_onDispatchEvent()`

```
void AIAssistant::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.3.1.4 `_saveInstance()`

```
void AIAssistant::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.3.1.5 `show()`

```
std::string AIAssistant::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

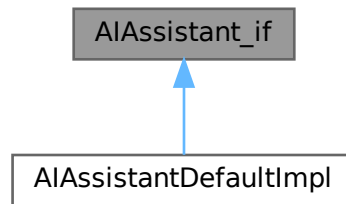
- `plugins/components/AI/AIAssistant.h`
- `plugins/components/AI/AIAssistant.cpp`

15.4 `AIAssistant_if` Class Reference

Abstract interface for the GenESyS AI Assistant tool.

```
#include <tools/AIAssistant/AIAssistant_if.h>
```


Inheritance diagram for AIAssistant_if:



Public Member Functions

- virtual void `setConfiguration` (const `AIAssistantConfiguration` &configuration)=0
Replaces the entire configuration record.
- virtual `AIAssistantConfiguration` `getConfiguration` () const =0
Returns a copy of the current configuration record.
- virtual void `setProvider` (AIProvider provider)=0
Sets the AI backend provider.
- virtual AIProvider `getProvider` () const =0
Returns the currently configured AI provider.
- virtual void `setProviderName` (const std::string &providerName)=0
Sets a human-readable provider label.
- virtual std::string `getProviderName` () const =0
Returns the human-readable provider label.
- virtual void `setModelName` (const std::string &modelName)=0
Sets the provider model identifier (e.g. "gpt-4o", "claude-opus-4-7").
- virtual std::string `getModelName` () const =0
Returns the provider model identifier.
- virtual void `setBaseUrl` (const std::string &baseUrl)=0
Sets the base URL; required when provider == CustomHttp.
- virtual std::string `getBaseUrl` () const =0
Returns the currently configured base URL.
- virtual void `setReasoningEffort` (ReasoningEffort effort)=0
Sets the desired reasoning effort hint.
- virtual ReasoningEffort `getReasoningEffort` () const =0
Returns the currently configured reasoning effort.
- virtual void `setTemperature` (double temperature)=0
Sets the sampling temperature (0.0 = deterministic, 1.0 = creative).
- virtual double `getTemperature` () const =0
Returns the configured sampling temperature.
- virtual void `setMaxOutputTokens` (unsigned int maxOutputTokens)=0
Sets the maximum number of output tokens to request.
- virtual unsigned int `getMaxOutputTokens` () const =0
Returns the maximum output tokens limit.
- virtual void `setTimeoutSeconds` (unsigned int timeoutSeconds)=0
Sets the HTTP request timeout in seconds.

- virtual unsigned int [getTimeoutSeconds](#) () const =0
Returns the HTTP request timeout in seconds.
- virtual void [setApiKey](#) (const std::string &apiKey)=0
Stores the API key in memory for the duration of the session.
- virtual void [clearApiKey](#) ()=0
Erases the in-memory API key.
- virtual bool [hasApiKey](#) () const =0
Returns true when a non-empty API key is held in memory.
- virtual std::string [getMaskedApiKey](#) () const =0
Returns a redacted representation of the current API key.
- virtual void [setApiKeyEnvironmentVariable](#) (const std::string &variableName)=0
Stores the name of the environment variable that holds the API key.
- virtual std::string [getApiKeyEnvironmentVariable](#) () const =0
Returns the configured environment-variable name.
- virtual bool [loadApiKeyFromEnvironment](#) ()=0
Reads the API key from the configured environment variable.
- virtual void [setSimulatorFacade](#) (SimulatorFacade *simulatorFacade)=0
Attaches a non-owning SimulatorFacade pointer.
- virtual SimulatorFacade * [getSimulatorFacade](#) () const =0
Returns the currently attached SimulatorFacade pointer, or nullptr.
- virtual bool [hasSimulatorFacade](#) () const =0
Returns true when a non-null SimulatorFacade is attached.
- virtual void [setProviderClient](#) (AIProviderClient_if *client)=0
Injects an external provider client (non-owning).
- virtual AIProviderClient_if * [getProviderClient](#) () const =0
Returns the currently attached provider client, or nullptr.
- virtual bool [hasProviderClient](#) () const =0
Returns true when a non-null provider client is attached.
- virtual void [clearKnowledgeBase](#) ()=0
Removes all items from the knowledge base.
- virtual void [addKnowledgeItem](#) (const GenesysKnowledgeItem &item)=0
Appends one item to the knowledge base.
- virtual std::vector< GenesysKnowledgeItem > [getKnowledgeItems](#) () const =0
Returns a copy of all registered knowledge items.
- virtual GenesysKnowledgeBaseSummary [getKnowledgeBaseSummary](#) () const =0
Returns an aggregate summary of the knowledge base.
- virtual std::string [exportKnowledgeBaseAsMarkdown](#) () const =0
Renders the knowledge base as a Markdown document.
- virtual bool [refreshKnowledgeBaseFromSimulator](#) (std::string *message=nullptr)=0
Populates the knowledge base by introspecting the attached SimulatorFacade.
- virtual bool [checkReady](#) (std::string *message=nullptr) const =0
Checks whether the assistant has enough configuration to operate.
- virtual AIAssistantResponse [analyzePrompt](#) (const AIAssistantRequest &request)=0
Parses and summarises a user prompt without calling any external service.
- virtual AIAssistantResponse [createModelPlan](#) (const AIAssistantRequest &request)=0
Produces a structured plan mapping the prompt to GenESyS components.
- virtual AIAssistantResponse [buildModel](#) (const AIAssistantRequest &request)=0
Constructs a GenESyS model via SimulatorFacade using an LLM-generated .gen file.
- virtual AIAssistantResponse [configureSimulation](#) (const AIAssistantRequest &request)=0
Overrides ModelSimulation parameters on the current model.
- virtual AIAssistantResponse [runSimulation](#) (const AIAssistantRequest &request)=0

- Executes the configured simulation synchronously via [SimulatorFacade](#).*
- virtual [AIAssistantResponse collectResults](#) (const [AIAssistantRequest](#) &request)=0
Collects and formats simulation results from the current model.
- virtual [AIAssistantResponse execute](#) (const [AIAssistantRequest](#) &request)=0
High-level dispatcher that orchestrates the full pipeline based on request flags.
- virtual [ExecutionState getState](#) () const =0
Returns the current lifecycle state.
- virtual std::string [getLastError](#) () const =0
Returns the error message from the most recent failed operation.
- virtual std::string [getLastDiagnostics](#) () const =0
Returns technical diagnostic output from the most recent operation.
- virtual void [clearDiagnostics](#) ()=0
Clears both the last error and diagnostics strings.
- virtual std::vector< [AuditEntry](#) > [getAuditEntries](#) (unsigned int maxEntries=200) const =0
*Returns up to *maxEntries* most-recent audit entries (oldest first).*
- virtual std::string [getAuditLogPath](#) () const =0
Returns the absolute path of the audit log file.
- virtual unsigned int [exportAuditLog](#) (const std::string &csvPath) const =0
Exports all audit entries to a CSV file.

15.4.1 Detailed Description

Abstract interface for the GenESyS AI Assistant tool.

The AI Assistant translates natural-language prompts into GenESyS simulation models, configures experiments and optionally executes them. This interface defines the full contract; concrete implementations supply the provider integration logic.

15.4.1.1 Security model

- API keys are accepted only through [setApiKey\(\)](#) or [loadApiKeyFromEnvironment\(\)](#).
- No method returns the raw key value ([getApiKey\(\)](#) does not exist by design).
- [getMaskedApiKey\(\)](#) exposes only the last four characters.
- All capability-affecting operations are guarded by explicit permission flags in [AIAssistantConfiguration](#). The deny-by-default posture means nothing is sent to external services, no model is mutated, and no simulation is run unless the caller has explicitly enabled each permission.

15.4.1.2 Planned evolution

See [source/tools/ARCHITECTURE_tools.md](#) for the multi-stage roadmap.

15.4.1.3 Thread safety

Implementations are not required to be thread-safe in this version.

15.4.2 Member Function Documentation

15.4.2.1 addKnowledgeItem()

```
virtual void AIAssistant_if::addKnowledgeItem (
    const GenesysKnowledgeItem & item ) [pure virtual]
```

Appends one item to the knowledge base.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.2 analyzePrompt()

```
virtual AIAssistantResponse AIAssistant_if::analyzePrompt (
    const AIAssistantRequest & request ) [pure virtual]
```

Parses and summarises a user prompt without calling any external service.
Stage 3 will implement actual LLM calls. Current stub returns NotImplemented.
Implemented in [AIAssistantDefaultImpl](#).

15.4.2.3 buildModel()

```
virtual AIAssistantResponse AIAssistant_if::buildModel (
    const AIAssistantRequest & request ) [pure virtual]
```

Constructs a GenESyS model via [SimulatorFacade](#) using an LLM-generated .gen file.
Requires `allowModelMutation == true`, `allowNetworkAccess == true`, a valid [SimulatorFacade](#), and an attached provider client.
Behaviour (Stage 5):

1. If `request.modelPlan` is non-empty, it is used as the model specification. Otherwise, the `request` description fields are used directly.
2. An LLM call is made requesting a GenESyS model language (.gen) file.
3. The generated .gen content is written to a temp file and loaded via `SimulatorFacade::loadModel()`. The temp file is deleted immediately after loading.
4. [AIAssistantResponse::generatedModelLanguage](#) holds the generated .gen text.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.4 checkReady()

```
virtual bool AIAssistant_if::checkReady (
    std::string * message = nullptr ) const [pure virtual]
```

Checks whether the assistant has enough configuration to operate.

Parameters

<i>message</i>	Optional; receives a description of the first unsatisfied condition, or a confirmation message when ready.
----------------	--

Returns

`true` when all minimum requirements are met for prompt analysis.

Checks performed (in order):

1. Provider is not `AIPProvider::Undefined`.
2. [Model](#) name is not empty.
3. For OpenAI/Anthropic/CustomHttp: an API key is held in memory or an environment-variable name is configured.
4. For CustomHttp: `baseUrl` is not empty.

Note: [SimulatorFacade](#) and permission flags are checked separately when model-mutation or simulation-execution operations are requested.
Implemented in [AIAssistantDefaultImpl](#).

15.4.2.5 clearApiKey()

```
virtual void AIAssistant_if::clearApiKey ( ) [pure virtual]
```

Erases the in-memory API key.

Implementations should overwrite the key buffer before clearing to reduce the window of exposure. Note that standard C++ provides no guarantee that the compiler will not optimise away the overwrite; platform-specific secure zeroing (`explicit_bzero`, `SecureZeroMemory`) should be used in security-critical contexts.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.6 clearDiagnostics()

```
virtual void AIAssistant_if::clearDiagnostics ( ) [pure virtual]
```

Clears both the last error and diagnostics strings.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.7 clearKnowledgeBase()

```
virtual void AIAssistant_if::clearKnowledgeBase ( ) [pure virtual]
```

Removes all items from the knowledge base.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.8 collectResults()

```
virtual AIAssistantResponse AIAssistant_if::collectResults (
    const AIAssistantRequest & request ) [pure virtual]
```

Collects and formats simulation results from the current model.

Iterates `SimulatorFacade::modelGetResponses()` and renders them as a Markdown table in `AIAssistantResponse::message`.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.9 configureSimulation()

```
virtual AIAssistantResponse AIAssistant_if::configureSimulation (
    const AIAssistantRequest & request ) [pure virtual]
```

Overrides [ModelSimulation](#) parameters on the current model.

Requires `allowModelMutation == true` and a valid [SimulatorFacade](#).

Behaviour (Stage 5):

1. Makes a small LLM call to extract structured simulation parameters from `request.experimentDescription` (replications, replication length, warm-up period, time units).
2. Applies the extracted values via `SimulatorFacade::simSet*()` methods.
3. Parameters already set by a loaded .gen file are overridden only when the LLM extraction yields non-zero values.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.10 createModelPlan()

```
virtual AIAssistantResponse AIAssistant_if::createModelPlan (
    const AIAssistantRequest & request ) [pure virtual]
```

Produces a structured plan mapping the prompt to GenESyS components.

Stage 4 will implement prompt-to-plan translation. Current stub returns `NotImplemented`.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.11 execute()

```
virtual AIAssistantResponse AIAssistant_if::execute (
    const AIAssistantRequest & request ) [pure virtual]
```

High-level dispatcher that orchestrates the full pipeline based on request flags.

Calls `analyzePrompt`, `createModelPlan`, `buildModel`, `configureSimulation`, `runSimulation` and `collectResults` in sequence according to the enabled flags in `request`. Stops at the first failure.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.12 exportAuditLog()

```
virtual unsigned int AIAssistant_if::exportAuditLog (
    const std::string & csvPath ) const [pure virtual]
```

Exports all audit entries to a CSV file.

Parameters

<i>csvPath</i>	Destination file path.
----------------	------------------------

Returns

Number of data rows written (excluding the header).

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.13 exportKnowledgeBaseAsMarkdown()

```
virtual std::string AIAssistant_if::exportKnowledgeBaseAsMarkdown ( ) const [pure virtual]
```

Renders the knowledge base as a Markdown document.

Intended to be injected into the system prompt so the AI model knows which GenESyS building blocks are available.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.14 getApiKeyEnvironmentVariable()

```
virtual std::string AIAssistant_if::getApiKeyEnvironmentVariable ( ) const [pure virtual]
```

Returns the configured environment-variable name.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.15 getAuditEntries()

```
virtual std::vector< AuditEntry > AIAssistant_if::getAuditEntries (
    unsigned int maxEntries = 200 ) const [pure virtual]
```

Returns up to `maxEntries` most-recent audit entries (oldest first).

Reads from the persistent audit log file. Returns an empty vector when the file does not yet exist or cannot be read.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.16 getAuditLogPath()

```
virtual std::string AIAssistant_if::getAuditLogPath ( ) const [pure virtual]
```

Returns the absolute path of the audit log file.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.17 getBaseUrl()

```
virtual std::string AIAssistant_if::getBaseUrl ( ) const [pure virtual]
```

Returns the currently configured base URL.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.18 getConfiguration()

```
virtual AIAssistantConfiguration AIAssistant_if::getConfiguration ( ) const [pure virtual]
```

Returns a copy of the current configuration record.

The returned struct never contains the API key value.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.19 getKnowledgeBaseSummary()

```
virtual GenesysKnowledgeBaseSummary AIAssistant_if::getKnowledgeBaseSummary ( ) const [pure virtual]
```

Returns an aggregate summary of the knowledge base.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.20 getKnowledgeItems()

```
virtual std::vector< GenesysKnowledgeItem > AIAssistant_if::getKnowledgeItems ( ) const [pure virtual]
```

Returns a copy of all registered knowledge items.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.21 getLastDiagnostics()

```
virtual std::string AIAssistant_if::getLastDiagnostics ( ) const [pure virtual]
```

Returns technical diagnostic output from the most recent operation.

Never contains API key values. Safe to log for debugging.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.22 getLastError()

```
virtual std::string AIAssistant_if::getLastError ( ) const [pure virtual]
```

Returns the error message from the most recent failed operation.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.23 getMaskedApiKey()

```
virtual std::string AIAssistant_if::getMaskedApiKey ( ) const [pure virtual]
```

Returns a redacted representation of the current API key.

Only the last four characters are exposed, preceded by "****". Returns an empty string when no key is set.

Example: "****abcd".

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.24 getMaxOutputTokens()

```
virtual unsigned int AIAssistant_if::getMaxOutputTokens ( ) const [pure virtual]
```

Returns the maximum output tokens limit.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.25 getModelName()

```
virtual std::string AIAssistant_if::getModelName ( ) const [pure virtual]
```

Returns the provider model identifier.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.26 getProvider()

```
virtual AIProvider AIAssistant_if::getProvider ( ) const [pure virtual]
```

Returns the currently configured AI provider.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.27 getProviderClient()

```
virtual AIProviderClient_if * AIAssistant_if::getProviderClient ( ) const [pure virtual]
```

Returns the currently attached provider client, or nullptr.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.28 `getProviderName()`

```
virtual std::string AIAssistant_if::getProviderName ( ) const [pure virtual]
```

Returns the human-readable provider label.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.29 `getReasoningEffort()`

```
virtual ReasoningEffort AIAssistant_if::getReasoningEffort ( ) const [pure virtual]
```

Returns the currently configured reasoning effort.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.30 `getSimulatorFacade()`

```
virtual SimulatorFacade * AIAssistant_if::getSimulatorFacade ( ) const [pure virtual]
```

Returns the currently attached [SimulatorFacade](#) pointer, or `nullptr`.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.31 `getState()`

```
virtual ExecutionState AIAssistant_if::getState ( ) const [pure virtual]
```

Returns the current lifecycle state.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.32 `getTemperature()`

```
virtual double AIAssistant_if::getTemperature ( ) const [pure virtual]
```

Returns the configured sampling temperature.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.33 `getTimeoutSeconds()`

```
virtual unsigned int AIAssistant_if::getTimeoutSeconds ( ) const [pure virtual]
```

Returns the HTTP request timeout in seconds.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.34 `hasApiKey()`

```
virtual bool AIAssistant_if::hasApiKey ( ) const [pure virtual]
```

Returns `true` when a non-empty API key is held in memory.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.35 `hasProviderClient()`

```
virtual bool AIAssistant_if::hasProviderClient ( ) const [pure virtual]
```

Returns `true` when a non-null provider client is attached.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.36 `hasSimulatorFacade()`

```
virtual bool AIAssistant_if::hasSimulatorFacade ( ) const [pure virtual]
```

Returns `true` when a non-null [SimulatorFacade](#) is attached.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.37 `loadApiKeyFromEnvironment()`

```
virtual bool AIAssistant_if::loadApiKeyFromEnvironment ( ) [pure virtual]
```

Reads the API key from the configured environment variable.

Returns

`true` when the variable exists and holds a non-empty value.

Does nothing and returns `false` when `apiKeyEnvironmentVariable` is empty or the variable is not defined in the process environment. The loaded value is kept only in memory; it is never written to disk.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.38 refreshKnowledgeBaseFromSimulator()

```
virtual bool AIAssistant_if::refreshKnowledgeBaseFromSimulator (
    std::string * message = nullptr ) [pure virtual]
```

Populates the knowledge base by introspecting the attached [SimulatorFacade](#).

Parameters

<code>message</code>	Optional; receives a human-readable status or error message.
----------------------	--

Returns

`true` when introspection succeeded and at least one item was added.

This method is a planned feature (Stage 2). In the current implementation it is a safe stub that returns `false` with a descriptive message.

Planned behaviour (Stage 2):

- Query `SimulatorFacade::getDataDefinitionPluginTypenames()` for registered types.
- Enumerate [ModelComponent](#) and [ModelDataDefinition](#) subtypes.
- Build [GenesysKnowledgeItem](#) entries with typenames, descriptions, and usage examples.

Todo (Stage 2): Implement full plugin introspection via [SimulatorFacade](#).

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.39 runSimulation()

```
virtual AIAssistantResponse AIAssistant_if::runSimulation (
    const AIAssistantRequest & request ) [pure virtual]
```

Executes the configured simulation synchronously via [SimulatorFacade](#).

Requires `allowSimulationExecution == true` and a valid [SimulatorFacade](#). Calls `SimulatorFacade::simStart()` (blocking) and returns after all replications complete. `AIAssistantResponse::message` reports the number of replications run.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.40 setApiKey()

```
virtual void AIAssistant_if::setApiKey (
    const std::string & apiKey ) [pure virtual]
```

Stores the API key in memory for the duration of the session.

The key is never written to disk or included in any log output. It is intentionally not retrievable via a getter.

Security guidance for production deployments:

- Prefer [loadApiKeyFromEnvironment\(\)](#) over embedding keys in source code.
- For long-running services, consider OS keyring / secret-store integration (planned for Stage 8; not implemented in this version).

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.41 setApiKeyEnvironmentVariable()

```
virtual void AIAssistant_if::setApiKeyEnvironmentVariable (
    const std::string & variableName ) [pure virtual]
```

Stores the name of the environment variable that holds the API key.

This is the variable *name*, not the value. Call [loadApiKeyFromEnvironment\(\)](#) to actually read the variable and load it into memory.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.42 setBaseUrl()

```
virtual void AIAssistant_if::setBaseUrl (
    const std::string & baseUrl ) [pure virtual]
```

Sets the base URL; required when provider == CustomHttp.

The URL should point to a chat-completions-compatible endpoint. Example: "http://localhost:11434/v1" for a local Ollama server.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.43 setConfiguration()

```
virtual void AIAssistant_if::setConfiguration (
    const AIAssistantConfiguration & configuration ) [pure virtual]
```

Replaces the entire configuration record.

Does not transfer the API key, which must be set separately via [setApiKey\(\)](#) or [loadApiKeyFromEnvironment\(\)](#).

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.44 setMaxOutputTokens()

```
virtual void AIAssistant_if::setMaxOutputTokens (
    unsigned int maxOutputTokens ) [pure virtual]
```

Sets the maximum number of output tokens to request.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.45 setModelName()

```
virtual void AIAssistant_if::setModelName (
    const std::string & modelName ) [pure virtual]
```

Sets the provider model identifier (e.g. "gpt-4o", "claude-opus-4-7").

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.46 setProvider()

```
virtual void AIAssistant_if::setProvider (
    AIProvider provider ) [pure virtual]
```

Sets the AI backend provider.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.47 setProviderClient()

```
virtual void AIAssistant_if::setProviderClient (
    AIProviderClient_if * client ) [pure virtual]
```

Injects an external provider client (non-owning).

The assistant does not take ownership; the caller is responsible for the lifetime of the injected client. The previously held client pointer (if any) is released without deletion.

To let the assistant create and own its own client based on the current [AIAssistantConfiguration](#), use [createAndAttachProviderClient\(\)](#) in [AIAssistantDefaultImpl](#) instead.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.48 setProviderName()

```
virtual void AIAssistant_if::setProviderName (
    const std::string & providerName ) [pure virtual]
```

Sets a human-readable provider label.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.49 setReasoningEffort()

```
virtual void AIAssistant_if::setReasoningEffort (
    ReasoningEffort effort ) [pure virtual]
```

Sets the desired reasoning effort hint.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.50 setSimulatorFacade()

```
virtual void AIAssistant_if::setSimulatorFacade (
    SimulatorFacade * simulatorFacade ) [pure virtual]
```

Attaches a non-owning [SimulatorFacade](#) pointer.

The assistant does not take ownership of the facade. The caller is responsible for keeping the facade alive for the duration of any operation that uses it.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.51 setTemperature()

```
virtual void AIAssistant_if::setTemperature (
    double temperature ) [pure virtual]
```

Sets the sampling temperature (0.0 = deterministic, 1.0 = creative).

Not all providers support this parameter; implementations may ignore it when the provider uses a fixed sampling scheme.

Implemented in [AIAssistantDefaultImpl](#).

15.4.2.52 setTimeoutSeconds()

```
virtual void AIAssistant_if::setTimeoutSeconds (
    unsigned int timeoutSeconds ) [pure virtual]
```

Sets the HTTP request timeout in seconds.

Implemented in [AIAssistantDefaultImpl](#).

The documentation for this class was generated from the following file:

- tools/AIAssistant/AIAssistant_if.h

15.5 AIAssistantConfiguration Struct Reference

Full configuration record for the AI assistant.

```
#include <tools/AIAssistant/AIAssistant_if.h>
```

Public Attributes

- AIProvider **provider** = AIProvider::Undefined
- std::string **providerName**
Human-readable label (e.g. "OpenAI GPT-4o").
- std::string **modelName**
Provider model identifier (e.g. "gpt-4o").
- std::string **baseUrl**
Required when provider == CustomHttp.
- std::string **apiKeyEnvironmentVariable**
Name of env-var holding the API key, not the key itself.

- `std::string organizationId`
Optional; passed in request headers when non-empty.
- `std::string projectId`
Optional; passed in request headers when non-empty.
- ReasoningEffort **reasoningEffort** = ReasoningEffort::Medium
- double **temperature** = 0.2
- unsigned int **maxOutputTokens** = 4096
- unsigned int **timeoutSeconds** = 120
- bool **allowNetworkAccess** = false
Permit outbound HTTP calls to the AI provider.
- bool **allowModelMutation** = false
Permit creation or modification of GenESyS models.
- bool **allowSimulationExecution** = false
Permit starting simulation replications.
- bool **allowFileSystemWrites** = false
Permit writing model/result files to disk.
- bool **sandboxEnabled** = false
When true, clears the current model before every buildModel() call.
- bool **dryRun** = false
When true, workflow methods simulate their actions without mutating the simulator state or consuming network quota beyond what is needed for planning.

15.5.1 Detailed Description

Full configuration record for the AI assistant.

Security contract:

- `apiKeyEnvironmentVariable` holds only the *name* of an environment variable, never the secret value itself.
- The actual API key must be supplied via `AIAssistant_if::setApiKey()` or loaded at runtime via `AIAssistant_if::loadApiKeyFromEnvironment()`.
- This struct must not be serialised to disk without first redacting the `providerName` field if it could be confused with a secret, and ensuring no key value has been stored here.

Permission flags default to `false` (deny-by-default). Each capability that touches external systems or mutates simulator state must be explicitly enabled before use.

15.5.2 Member Data Documentation

15.5.2.1 dryRun

```
bool AIAssistantConfiguration::dryRun = false
```

When `true`, workflow methods simulate their actions without mutating the simulator state or consuming network quota beyond what is needed for planning.

- `buildModel()` — generates the .gen text via LLM but does **NOT** load it.
- `configureSimulation()` — extracts parameters via LLM but does **NOT** apply them.
- `runSimulation()` — reports what would run without calling `simStart()`.
- `collectResults()` — reports the count of available responses without rendering.
- `analyzePrompt()` / `createModelPlan()` — always execute regardless of this flag.

Use dry-run to verify the LLM output before committing changes.

15.5.2.2 sandboxEnabled

```
bool AIAssistantConfiguration::sandboxEnabled = false
```

When `true`, clears the current model before every `buildModel()` call.

Sandbox mode provides a clean-room environment: any existing model elements are discarded so the LLM always starts from an empty canvas. This prevents the AI from accidentally inheriting components from a model the user was editing. The cleared state is permanent; no undo is provided.

The documentation for this struct was generated from the following file:

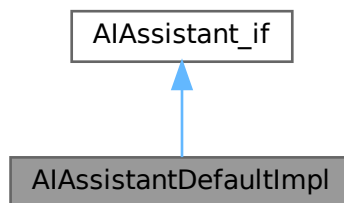
- `tools/AIAssistant/AIAssistant_if.h`

15.6 AIAssistantDefaultImpl Class Reference

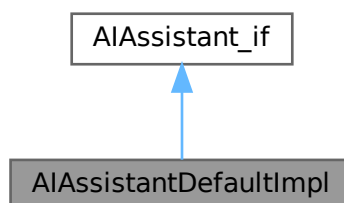
Initial concrete scaffold for the GenESyS AI Assistant.

```
#include <tools/AIAssistant/AIAssistantDefaultImpl.h>
```

Inheritance diagram for `AIAssistantDefaultImpl`:



Collaboration diagram for `AIAssistantDefaultImpl`:



Public Member Functions

- `AIAssistantDefaultImpl` (const `AIAssistantDefaultImpl` &)=delete
- `AIAssistantDefaultImpl` & `operator=` (const `AIAssistantDefaultImpl` &)=delete
- virtual void `setConfiguration` (const `AIAssistantConfiguration` &configuration) override
Replaces the entire configuration record.
- virtual `AIAssistantConfiguration` `getConfiguration` () const override
Returns a copy of the current configuration record.
- virtual void `setProvider` (AIProvider provider) override

- Sets the AI backend provider.*
- virtual AIProvider [getProvider](#) () const override
Returns the currently configured AI provider.
- virtual void [setProviderName](#) (const std::string &providerName) override
Sets a human-readable provider label.
- virtual std::string [getProviderName](#) () const override
Returns the human-readable provider label.
- virtual void [setModelName](#) (const std::string &modelName) override
Sets the provider model identifier (e.g. "gpt-4o", "claude-opus-4-7").
- virtual std::string [getModelName](#) () const override
Returns the provider model identifier.
- virtual void [setBaseUrl](#) (const std::string &baseUrl) override
Sets the base URL; required when provider == CustomHttp.
- virtual std::string [getBaseUrl](#) () const override
Returns the currently configured base URL.
- virtual void [setReasoningEffort](#) (ReasoningEffort effort) override
Sets the desired reasoning effort hint.
- virtual ReasoningEffort [getReasoningEffort](#) () const override
Returns the currently configured reasoning effort.
- virtual void [setTemperature](#) (double temperature) override
Sets the sampling temperature (0.0 = deterministic, 1.0 = creative).
- virtual double [getTemperature](#) () const override
Returns the configured sampling temperature.
- virtual void [setMaxOutputTokens](#) (unsigned int maxOutputTokens) override
Sets the maximum number of output tokens to request.
- virtual unsigned int [getMaxOutputTokens](#) () const override
Returns the maximum output tokens limit.
- virtual void [setTimeoutSeconds](#) (unsigned int timeoutSeconds) override
Sets the HTTP request timeout in seconds.
- virtual unsigned int [getTimeoutSeconds](#) () const override
Returns the HTTP request timeout in seconds.
- virtual void [setApiKey](#) (const std::string &apiKey) override
Stores the API key in memory for the duration of the session.
- virtual void [clearApiKey](#) () override
Erases the in-memory API key.
- virtual bool [hasApiKey](#) () const override
Returns true when a non-empty API key is held in memory.
- virtual std::string [getMaskedApiKey](#) () const override
Returns a redacted representation of the current API key.
- virtual void [setApiKeyEnvironmentVariable](#) (const std::string &variableName) override
Stores the name of the environment variable that holds the API key.
- virtual std::string [getApiKeyEnvironmentVariable](#) () const override
Returns the configured environment-variable name.
- virtual bool [loadApiKeyFromEnvironment](#) () override
Reads the API key from the configured environment variable.
- virtual void [setSimulatorFacade](#) (SimulatorFacade *simulatorFacade) override
Attaches a non-owning SimulatorFacade pointer.
- virtual SimulatorFacade * [getSimulatorFacade](#) () const override
Returns the currently attached SimulatorFacade pointer, or nullptr.
- virtual bool [hasSimulatorFacade](#) () const override
Returns true when a non-null SimulatorFacade is attached.

- virtual void [setProviderClient](#) ([AIProviderClient_if](#) *client) override
Injects an external provider client (non-owning).
- virtual [AIProviderClient_if](#) * [getProviderClient](#) () const override
Returns the currently attached provider client, or `nullptr`.
- virtual bool [hasProviderClient](#) () const override
Returns `true` when a non-null provider client is attached.
- bool [createAndAttachProviderClient](#) ()
Creates an owned provider client from the current configuration and attaches it, replacing any previously held client pointer.
- virtual void [clearKnowledgeBase](#) () override
Removes all items from the knowledge base.
- virtual void [addKnowledgeItem](#) (const [GenesysKnowledgeItem](#) &item) override
Appends one item to the knowledge base.
- virtual std::vector< [GenesysKnowledgeItem](#) > [getKnowledgeItems](#) () const override
Returns a copy of all registered knowledge items.
- virtual [GenesysKnowledgeBaseSummary](#) [getKnowledgeBaseSummary](#) () const override
Returns an aggregate summary of the knowledge base.
- virtual std::string [exportKnowledgeBaseAsMarkdown](#) () const override
Renders the knowledge base as a Markdown document.
- virtual bool [refreshKnowledgeBaseFromSimulator](#) (std::string *message=nullptr) override
Populates the knowledge base by introspecting the attached [SimulatorFacade](#).
- virtual bool [checkReady](#) (std::string *message=nullptr) const override
Checks whether the assistant has enough configuration to operate.
- virtual [AIAssistantResponse](#) [analyzePrompt](#) (const [AIAssistantRequest](#) &request) override
Parses and summarises a user prompt without calling any external service.
- virtual [AIAssistantResponse](#) [createModelPlan](#) (const [AIAssistantRequest](#) &request) override
Produces a structured plan mapping the prompt to GenESyS components.
- virtual [AIAssistantResponse](#) [buildModel](#) (const [AIAssistantRequest](#) &request) override
Constructs a GenESyS model via [SimulatorFacade](#) using an LLM-generated .gen file.
- virtual [AIAssistantResponse](#) [configureSimulation](#) (const [AIAssistantRequest](#) &request) override
Overrides [ModelSimulation](#) parameters on the current model.
- virtual [AIAssistantResponse](#) [runSimulation](#) (const [AIAssistantRequest](#) &request) override
Executes the configured simulation synchronously via [SimulatorFacade](#).
- virtual [AIAssistantResponse](#) [collectResults](#) (const [AIAssistantRequest](#) &request) override
Collects and formats simulation results from the current model.
- virtual [AIAssistantResponse](#) [execute](#) (const [AIAssistantRequest](#) &request) override
High-level dispatcher that orchestrates the full pipeline based on request flags.
- virtual [ExecutionState](#) [getState](#) () const override
Returns the current lifecycle state.
- virtual std::string [getLastError](#) () const override
Returns the error message from the most recent failed operation.
- virtual std::string [getLastDiagnostics](#) () const override
Returns technical diagnostic output from the most recent operation.
- virtual void [clearDiagnostics](#) () override
Clears both the last error and diagnostics strings.
- virtual std::vector< [AuditEntry](#) > [getAuditEntries](#) (unsigned int maxEntries) const override
Returns up to `maxEntries` most-recent audit entries (oldest first).
- virtual std::string [getAuditLogPath](#) () const override
Returns the absolute path of the audit log file.
- virtual unsigned int [exportAuditLog](#) (const std::string &csvPath) const override
Exports all audit entries to a CSV file.

15.6.1 Detailed Description

Initial concrete scaffold for the GenESyS AI Assistant.

This class implements [AIAssistant_if](#) and provides:

- Full configuration management including secure API key handling.
- A knowledge base backed by a simple in-memory vector.
- A [checkReady\(\)](#) implementation that validates minimum configuration.
- Safe stubs for all workflow methods (`analyzePrompt`, `buildModel`, etc.) that return `ExecutionState::NotImplemented` until the corresponding stage is implemented.

Current status:

- Stage 1: configuration, security, knowledge base, diagnostics — complete.
- Stage 2 and above: intentionally deferred; see [Todo](#) annotations.

15.6.1.1 Security model

- The API key is held in a private `std::string` member, never written to disk or included in trace output.
- No getter exposes the key value; only [getMaskedApiKey\(\)](#) is available.
- Copy construction and copy assignment are deleted to prevent accidental key propagation through object copying.

15.6.1.2 Ownership

- The [SimulatorFacade](#) pointer is non-owning; this class does not delete it.

15.6.2 Member Function Documentation

15.6.2.1 addKnowledgeItem()

```
void AIAssistantDefaultImpl::addKnowledgeItem (
    const GenesysKnowledgeItem & item ) [override], [virtual]
```

Appends one item to the knowledge base.

Implements [AIAssistant_if](#).

15.6.2.2 analyzePrompt()

```
AIAssistantResponse AIAssistantDefaultImpl::analyzePrompt (
    const AIAssistantRequest & request ) [override], [virtual]
```

Parses and summarises a user prompt without calling any external service.

Stage 3 will implement actual LLM calls. Current stub returns `NotImplemented`.

Implements [AIAssistant_if](#).

15.6.2.3 buildModel()

```
AIAssistantResponse AIAssistantDefaultImpl::buildModel (
    const AIAssistantRequest & request ) [override], [virtual]
```

Constructs a GenESyS model via [SimulatorFacade](#) using an LLM-generated `.gen` file.

Requires `allowModelMutation == true`, `allowNetworkAccess == true`, a valid [SimulatorFacade](#), and an attached provider client.

Behaviour (Stage 5):

1. If `request.modelPlan` is non-empty, it is used as the model specification. Otherwise, the `request` description fields are used directly.

2. An LLM call is made requesting a GenESyS model language (.gen) file.
3. The generated .gen content is written to a temp file and loaded via SimulatorFacade::loadModel(). The temp file is deleted immediately after loading.
4. [AIAssistantResponse::generatedModelLanguage](#) holds the generated .gen text.

Implements [AIAssistant_if](#).

15.6.2.4 checkReady()

```
bool AIAssistantDefaultImpl::checkReady (
    std::string * message = nullptr ) const [override], [virtual]
```

Checks whether the assistant has enough configuration to operate.

Parameters

<i>message</i>	Optional; receives a description of the first unsatisfied condition, or a confirmation message when ready.
----------------	--

Returns

`true` when all minimum requirements are met for prompt analysis.

Checks performed (in order):

1. Provider is not `AIPProvider::Undefined`.
2. [Model](#) name is not empty.
3. For OpenAI/Anthropic/CustomHttp: an API key is held in memory or an environment-variable name is configured.
4. For CustomHttp: baseUrl is not empty.

Note: [SimulatorFacade](#) and permission flags are checked separately when model-mutation or simulation-execution operations are requested.

Implements [AIAssistant_if](#).

15.6.2.5 clearApiKey()

```
void AIAssistantDefaultImpl::clearApiKey ( ) [override], [virtual]
```

Erases the in-memory API key.

Implementations should overwrite the key buffer before clearing to reduce the window of exposure. Note that standard C++ provides no guarantee that the compiler will not optimise away the overwrite; platform-specific secure zeroing (`explicit_bzero`, `SecureZeroMemory`) should be used in security-critical contexts.

Implements [AIAssistant_if](#).

15.6.2.6 clearDiagnostics()

```
void AIAssistantDefaultImpl::clearDiagnostics ( ) [override], [virtual]
```

Clears both the last error and diagnostics strings.

Implements [AIAssistant_if](#).

15.6.2.7 clearKnowledgeBase()

```
void AIAssistantDefaultImpl::clearKnowledgeBase ( ) [override], [virtual]
```

Removes all items from the knowledge base.

Implements [AIAssistant_if](#).

15.6.2.8 collectResults()

```
AIAssistantResponse AIAssistantDefaultImpl::collectResults (
    const AIAssistantRequest & request ) [override], [virtual]
```

Collects and formats simulation results from the current model.

Iterates SimulatorFacade::modelGetResponses() and renders them as a Markdown table in `AIAssistantResponse::message`.

Implements [AIAssistant_if](#).

15.6.2.9 configureSimulation()

```
AIAssistantResponse AIAssistantDefaultImpl::configureSimulation (
    const AIAssistantRequest & request ) [override], [virtual]
```

Overrides [ModelSimulation](#) parameters on the current model.

Requires `allowModelMutation == true` and a valid [SimulatorFacade](#).

Behaviour (Stage 5):

1. Makes a small LLM call to extract structured simulation parameters from `request.experimentDescription` (replications, replication length, warm-up period, time units).
2. Applies the extracted values via `SimulatorFacade::simSet*()` methods.
3. Parameters already set by a loaded .gen file are overridden only when the LLM extraction yields non-zero values.

Implements [AIAssistant_if](#).

15.6.2.10 createAndAttachProviderClient()

```
bool AIAssistantDefaultImpl::createAndAttachProviderClient ( )
```

Creates an owned provider client from the current configuration and attaches it, replacing any previously held client pointer.

Returns

`true` when a suitable client was created and attached.

The assistant takes ownership of the created client and deletes it in the destructor or on the next call to [createAndAttachProviderClient\(\)](#). Clients injected via [setProviderClient\(\)](#) are never deleted by the assistant.

Provider selection is based on `AIAssistantConfiguration::provider`:

- `AIProvider::OpenAI` → [OpenAIProviderClientImpl](#)
- `AIProvider::Anthropic` → [AnthropicProviderClientImpl](#)
- `AIProvider::Local` → [LocalProviderClientImpl](#)
- `AIProvider::CustomHttp` → [LocalProviderClientImpl](#) with custom base URL
- `AIProvider::Undefined` → returns false, no client created

15.6.2.11 createModelPlan()

```
AIAssistantResponse AIAssistantDefaultImpl::createModelPlan (
    const AIAssistantRequest & request ) [override], [virtual]
```

Produces a structured plan mapping the prompt to GenESyS components.

Stage 4 will implement prompt-to-plan translation. Current stub returns `NotImplemented`.

Implements [AIAssistant_if](#).

15.6.2.12 execute()

```
AIAssistantResponse AIAssistantDefaultImpl::execute (
    const AIAssistantRequest & request ) [override], [virtual]
```

High-level dispatcher that orchestrates the full pipeline based on request flags.

Calls `analyzePrompt`, `createModelPlan`, `buildModel`, `configureSimulation`, `runSimulation` and `collectResults` in sequence according to the enabled flags in `request`. Stops at the first failure.

Implements [AIAssistant_if](#).

15.6.2.13 exportAuditLog()

```
unsigned int AIAssistantDefaultImpl::exportAuditLog (
    const std::string & csvPath ) const [override], [virtual]
```

Exports all audit entries to a CSV file.

Parameters

<i>csvPath</i>	Destination file path.
----------------	------------------------

Returns

Number of data rows written (excluding the header).

Implements [AIAssistant_if](#).

15.6.2.14 exportKnowledgeBaseAsMarkdown()

```
std::string AIAssistantDefaultImpl::exportKnowledgeBaseAsMarkdown ( ) const [override], [virtual]
```

Renders the knowledge base as a Markdown document.

Intended to be injected into the system prompt so the AI model knows which GenESyS building blocks are available.

Implements [AIAssistant_if](#).

15.6.2.15 getApiKeyEnvironmentVariable()

```
std::string AIAssistantDefaultImpl::getApiKeyEnvironmentVariable ( ) const [override], [virtual]
```

Returns the configured environment-variable name.

Implements [AIAssistant_if](#).

15.6.2.16 getAuditEntries()

```
std::vector< AuditEntry > AIAssistantDefaultImpl::getAuditEntries (
    unsigned int maxEntries ) const [override], [virtual]
```

Returns up to `maxEntries` most-recent audit entries (oldest first).

Reads from the persistent audit log file. Returns an empty vector when the file does not yet exist or cannot be read.

Implements [AIAssistant_if](#).

15.6.2.17 getAuditLogPath()

```
std::string AIAssistantDefaultImpl::getAuditLogPath ( ) const [override], [virtual]
```

Returns the absolute path of the audit log file.

Implements [AIAssistant_if](#).

15.6.2.18 getBaseUrl()

```
std::string AIAssistantDefaultImpl::getBaseUrl ( ) const [override], [virtual]
```

Returns the currently configured base URL.

Implements [AIAssistant_if](#).

15.6.2.19 getConfiguration()

```
AIAssistantConfiguration AIAssistantDefaultImpl::getConfiguration ( ) const [override], [virtual]
```

Returns a copy of the current configuration record.

The returned struct never contains the API key value.

Implements [AIAssistant_if](#).

15.6.2.20 getKnowledgeBaseSummary()

```
GenesysKnowledgeBaseSummary AIAssistantDefaultImpl::getKnowledgeBaseSummary ( ) const [override], [virtual]
```

Returns an aggregate summary of the knowledge base.

Implements [AIAssistant_if](#).

15.6.2.21 getKnowledgeItems()

```
std::vector< GenesysKnowledgeItem > AIAssistantDefaultImpl::getKnowledgeItems ( ) const [override], [virtual]
```

Returns a copy of all registered knowledge items.

Implements [AIAssistant_if](#).

15.6.2.22 getLastDiagnostics()

```
std::string AIAssistantDefaultImpl::getLastDiagnostics ( ) const [override], [virtual]
```

Returns technical diagnostic output from the most recent operation.

Never contains API key values. Safe to log for debugging.

Implements [AIAssistant_if](#).

15.6.2.23 getLastError()

```
std::string AIAssistantDefaultImpl::getLastError ( ) const [override], [virtual]
```

Returns the error message from the most recent failed operation.

Implements [AIAssistant_if](#).

15.6.2.24 getMaskedApiKey()

```
std::string AIAssistantDefaultImpl::getMaskedApiKey ( ) const [override], [virtual]
```

Returns a redacted representation of the current API key.

Only the last four characters are exposed, preceded by "****". Returns an empty string when no key is set.

Example: "****abcd".

Implements [AIAssistant_if](#).

15.6.2.25 getMaxOutputTokens()

```
unsigned int AIAssistantDefaultImpl::getMaxOutputTokens ( ) const [override], [virtual]
```

Returns the maximum output tokens limit.

Implements [AIAssistant_if](#).

15.6.2.26 getModelName()

```
std::string AIAssistantDefaultImpl::getModelName ( ) const [override], [virtual]
```

Returns the provider model identifier.

Implements [AIAssistant_if](#).

15.6.2.27 getProvider()

```
AIProvider AIAssistantDefaultImpl::getProvider ( ) const [override], [virtual]
```

Returns the currently configured AI provider.

Implements [AIAssistant_if](#).

15.6.2.28 getProviderClient()

```
AIProviderClient_if * AIAssistantDefaultImpl::getProviderClient ( ) const [override], [virtual]
```

Returns the currently attached provider client, or nullptr.

Implements [AIAssistant_if](#).

15.6.2.29 getProviderName()

```
std::string AIAssistantDefaultImpl::getProviderName ( ) const [override], [virtual]
```

Returns the human-readable provider label.

Implements [AIAssistant_if](#).

15.6.2.30 getReasoningEffort()

```
ReasoningEffort AIAssistantDefaultImpl::getReasoningEffort ( ) const [override], [virtual]
```

Returns the currently configured reasoning effort.

Implements [AIAssistant_if](#).

15.6.2.31 getSimulatorFacade()

```
SimulatorFacade * AIAssistantDefaultImpl::getSimulatorFacade ( ) const [override], [virtual]
```

Returns the currently attached [SimulatorFacade](#) pointer, or nullptr.

Implements [AIAssistant_if](#).

15.6.2.32 getState()

```
ExecutionState AIAssistantDefaultImpl::getState ( ) const [override], [virtual]
```

Returns the current lifecycle state.

Implements [AIAssistant_if](#).

15.6.2.33 getTemperature()

```
double AIAssistantDefaultImpl::getTemperature ( ) const [override], [virtual]
```

Returns the configured sampling temperature.

Implements [AIAssistant_if](#).

15.6.2.34 getTimeoutSeconds()

```
unsigned int AIAssistantDefaultImpl::getTimeoutSeconds ( ) const [override], [virtual]
```

Returns the HTTP request timeout in seconds.

Implements [AIAssistant_if](#).

15.6.2.35 hasApiKey()

```
bool AIAssistantDefaultImpl::hasApiKey ( ) const [override], [virtual]
```

Returns true when a non-empty API key is held in memory.

Implements [AIAssistant_if](#).

15.6.2.36 hasProviderClient()

```
bool AIAssistantDefaultImpl::hasProviderClient ( ) const [override], [virtual]
```

Returns true when a non-null provider client is attached.

Implements [AIAssistant_if](#).

15.6.2.37 hasSimulatorFacade()

```
bool AIAssistantDefaultImpl::hasSimulatorFacade ( ) const [override], [virtual]
```

Returns true when a non-null [SimulatorFacade](#) is attached.

Implements [AIAssistant_if](#).

15.6.2.38 loadApiKeyFromEnvironment()

```
bool AIAssistantDefaultImpl::loadApiKeyFromEnvironment ( ) [override], [virtual]
```

Reads the API key from the configured environment variable.

Returns

`true` when the variable exists and holds a non-empty value.

Does nothing and returns `false` when `apiKeyEnvironmentVariable` is empty or the variable is not defined in the process environment. The loaded value is kept only in memory; it is never written to disk.

Implements [AIAssistant_if](#).

15.6.2.39 refreshKnowledgeBaseFromSimulator()

```
bool AIAssistantDefaultImpl::refreshKnowledgeBaseFromSimulator (
    std::string * message = nullptr ) [override], [virtual]
```

Populates the knowledge base by introspecting the attached [SimulatorFacade](#).

Parameters

<code>message</code>	Optional; receives a human-readable status or error message.
----------------------	--

Returns

`true` when introspection succeeded and at least one item was added.

This method is a planned feature (Stage 2). In the current implementation it is a safe stub that returns `false` with a descriptive message.

Planned behaviour (Stage 2):

- Query `SimulatorFacade::getDataDefinitionPluginTypenames()` for registered types.
- Enumerate [ModelComponent](#) and [ModelDataDefinition](#) subtypes.
- Build [GenesysKnowledgeItem](#) entries with typenames, descriptions, and usage examples.

Todo (Stage 2): Implement full plugin introspection via [SimulatorFacade](#).

Implements [AIAssistant_if](#).

15.6.2.40 runSimulation()

```
AIAssistantResponse AIAssistantDefaultImpl::runSimulation (
    const AIAssistantRequest & request ) [override], [virtual]
```

Executes the configured simulation synchronously via [SimulatorFacade](#).

Requires `allowSimulationExecution == true` and a valid [SimulatorFacade](#). Calls `SimulatorFacade::simStart()` (blocking) and returns after all replications complete. `AIAssistantResponse::message` reports the number of replications run.

Implements [AIAssistant_if](#).

15.6.2.41 setApiKey()

```
void AIAssistantDefaultImpl::setApiKey (
    const std::string & apiKey ) [override], [virtual]
```

Stores the API key in memory for the duration of the session.

The key is never written to disk or included in any log output. It is intentionally not retrievable via a getter.

Security guidance for production deployments:

- Prefer [loadApiKeyFromEnvironment\(\)](#) over embedding keys in source code.
- For long-running services, consider OS keyring / secret-store integration (planned for Stage 8; not implemented in this version).

Implements [AIAssistant_if](#).

15.6.2.42 setApiKeyEnvironmentVariable()

```
void AIAssistantDefaultImpl::setApiKeyEnvironmentVariable (
    const std::string & variableName ) [override], [virtual]
```

Stores the name of the environment variable that holds the API key.

This is the variable *name*, not the value. Call [loadApiKeyFromEnvironment\(\)](#) to actually read the variable and load it into memory.

Implements [AIAssistant_if](#).

15.6.2.43 setBaseUrl()

```
void AIAssistantDefaultImpl::setBaseUrl (
    const std::string & baseUrl ) [override], [virtual]
```

Sets the base URL; required when provider == CustomHttp.

The URL should point to a chat-completions-compatible endpoint. Example: "http://localhost:11434/v1" for a local Ollama server.

Implements [AIAssistant_if](#).

15.6.2.44 setConfiguration()

```
void AIAssistantDefaultImpl::setConfiguration (
    const AIAssistantConfiguration & configuration ) [override], [virtual]
```

Replaces the entire configuration record.

Does not transfer the API key, which must be set separately via [setApiKey\(\)](#) or [loadApiKeyFromEnvironment\(\)](#).

Implements [AIAssistant_if](#).

15.6.2.45 setMaxOutputTokens()

```
void AIAssistantDefaultImpl::setMaxOutputTokens (
    unsigned int maxOutputTokens ) [override], [virtual]
```

Sets the maximum number of output tokens to request.

Implements [AIAssistant_if](#).

15.6.2.46 setModelName()

```
void AIAssistantDefaultImpl::setModelName (
    const std::string & modelName ) [override], [virtual]
```

Sets the provider model identifier (e.g. "gpt-4o", "claude-opus-4-7").

Implements [AIAssistant_if](#).

15.6.2.47 setProvider()

```
void AIAssistantDefaultImpl::setProvider (
    AIProvider provider ) [override], [virtual]
```

Sets the AI backend provider.

Implements [AIAssistant_if](#).

15.6.2.48 setProviderClient()

```
void AIAssistantDefaultImpl::setProviderClient (
    AIProviderClient_if * client ) [override], [virtual]
```

Injects an external provider client (non-owning).

The assistant does not take ownership; the caller is responsible for the lifetime of the injected client. The previously held client pointer (if any) is released without deletion.

To let the assistant create and own its own client based on the current [AIAssistantConfiguration](#), use [createAndAttachProviderClient\(\)](#) in [AIAssistantDefaultImpl](#) instead.

Implements [AIAssistant_if](#).

15.6.2.49 setProviderName()

```
void AIAssistantDefaultImpl::setProviderName (
    const std::string & providerName ) [override], [virtual]
```

Sets a human-readable provider label.

Implements [AIAssistant_if](#).

15.6.2.50 setReasoningEffort()

```
void AIAssistantDefaultImpl::setReasoningEffort (
    ReasoningEffort effort ) [override], [virtual]
```

Sets the desired reasoning effort hint.

Implements [AIAssistant_if](#).

15.6.2.51 setSimulatorFacade()

```
void AIAssistantDefaultImpl::setSimulatorFacade (
    SimulatorFacade * simulatorFacade ) [override], [virtual]
```

Attaches a non-owning [SimulatorFacade](#) pointer.

The assistant does not take ownership of the facade. The caller is responsible for keeping the facade alive for the duration of any operation that uses it.

Implements [AIAssistant_if](#).

15.6.2.52 setTemperature()

```
void AIAssistantDefaultImpl::setTemperature (
    double temperature ) [override], [virtual]
```

Sets the sampling temperature (0.0 = deterministic, 1.0 = creative).

Not all providers support this parameter; implementations may ignore it when the provider uses a fixed sampling scheme.

Implements [AIAssistant_if](#).

15.6.2.53 setTimeoutSeconds()

```
void AIAssistantDefaultImpl::setTimeoutSeconds (
    unsigned int timeoutSeconds ) [override], [virtual]
```

Sets the HTTP request timeout in seconds.

Implements [AIAssistant_if](#).

The documentation for this class was generated from the following files:

- tools/AIAssistant/AIAssistantDefaultImpl.h
- tools/AIAssistant/AIAssistantDefaultImpl.cpp

15.7 AIAssistantRequest Struct Reference

Input to a single AI assistant request.

```
#include <tools/AIAssistant/AIAssistant_if.h>
```

Public Attributes

- std::string **userPrompt**
Free-text prompt from the user.
- std::string **systemContext**
Optional extra context to prepend to the system message.
- std::string **modelDescription**
Description of the system to model (may be empty).
- std::string **experimentDescription**
Description of the simulation experiment.

- `std::string` **modelPlan**
Structured model plan (Markdown) to guide model construction.
- `bool` **buildModel** = false
Request model construction via [SimulatorFacade](#).
- `bool` **configureSimulation** = false
Request [ModelSimulation](#) configuration.
- `bool` **runSimulation** = false
Request simulation execution.
- `bool` **collectResults** = false
Request result collection and reporting.

15.7.1 Detailed Description

Input to a single AI assistant request.

Not all fields need to be populated for every request. The assistant interprets the populated subset and ignores empty strings. Boolean flags signal intent; the assistant may reject them if the corresponding `AIAssistantConfiguration::allow*` flag is not set.

15.7.2 Member Data Documentation

15.7.2.1 modelPlan

```
std::string AIAssistantRequest::modelPlan
```

Structured model plan (Markdown) to guide model construction.

When non-empty, `buildModel()` uses this plan directly without calling the LLM a second time to produce a new plan. Populated automatically by `execute()` from the output of `createModelPlan()`. Callers may also supply a previously generated or manually written plan.

The documentation for this struct was generated from the following file:

- `tools/AIAssistant/AIAssistant_if.h`

15.8 AIAssistantResponse Struct Reference

Output produced by a single AI assistant operation.

```
#include <tools/AIAssistant/AIAssistant_if.h>
```

Public Attributes

- `bool` **success** = false
- `ExecutionState` **state** = `ExecutionState::NotImplemented`
- `std::string` **message**
Human-readable outcome summary or error description.
- `std::string` **generatedPlan**
Intermediate structured plan (Markdown or JSON fragment).
- `std::string` **generatedModelLanguage**
GenESyS model language text, when generated.
- `std::string` **diagnostics**
Technical detail for debugging; not shown to end users.

15.8.1 Detailed Description

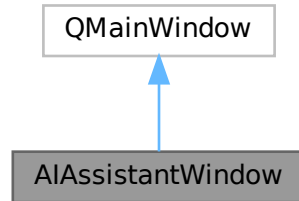
Output produced by a single AI assistant operation.

The documentation for this struct was generated from the following file:

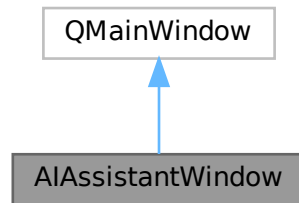
- `tools/AIAssistant/AIAssistant_if.h`

15.9 AIAssistantWindow Class Reference

Inheritance diagram for AIAssistantWindow:



Collaboration diagram for AIAssistantWindow:



Public Member Functions

- **AIAssistantWindow** ([Simulator](#) *simulator, QWidget *parent=nullptr)

The documentation for this class was generated from the following files:

- applications/gui/ai_assistant/AIAssistantWindow.h
- applications/gui/ai_assistant/AIAssistantWindow.cpp

15.10 AIAuditLog Class Reference

Append-only audit log for AI assistant operations.

```
#include <tools/AIAssistant/AIAuditLog.h>
```

Public Member Functions

- **AIAuditLog** (const std::string &logFilePath="")
- **AIAuditLog** (const [AIAuditLog](#) &)=delete
- **AIAuditLog** & **operator=** (const [AIAuditLog](#) &)=delete
- void **append** (const [AuditEntry](#) &entry)
Appends one entry to the log file (creates directory if needed).
- std::vector< [AuditEntry](#) > **loadRecentEntries** (unsigned int maxEntries=200) const

Reads up to `maxEntries` most-recent entries.

- `std::string logFilePath ()` const
Returns the absolute path of the log file.
- `unsigned int exportCsv (const std::string &csvPath)` const
Exports all log entries to a CSV file.

Static Public Member Functions

- `static std::string defaultLogFilePath ()`
Returns the platform-default log file path.

15.10.1 Detailed Description

Append-only audit log for AI assistant operations.

Entries are written in JSON Lines format to a file under `$XDG_DATA_HOME/genesys/` (typically `~/.local/share/genesys/ai-audit.log`). The log never contains API key values. The directory is created on first write.

15.10.2 Member Function Documentation

15.10.2.1 exportCsv()

```
unsigned int AIAuditLog::exportCsv (
    const std::string & csvPath ) const
```

Exports all log entries to a CSV file.

Returns

Number of rows written (excluding the header).

15.10.2.2 loadRecentEntries()

```
std::vector< AuditEntry > AIAuditLog::loadRecentEntries (
    unsigned int maxEntries = 200 ) const
```

Reads up to `maxEntries` most-recent entries.

Returns

Entries in chronological order (oldest first).

The documentation for this class was generated from the following files:

- `tools/AIAssistant/AIAuditLog.h`
- `tools/AIAssistant/AIAuditLog.cpp`

15.11 AIConversationConfiguration Struct Reference

Public Attributes

- `AIConversationProvider provider` = `AIConversationProvider::Local`
- `std::string model` = `"llama3.2"`
- `std::string baseUrl`
- `std::string apiKeyEnvironmentVariable`
- `std::string systemInstructions`
- `double temperature` = `0.2`
- `unsigned int maxOutputTokens` = `512`
- `unsigned int timeoutSeconds` = `120`
- `bool highReasoningMode` = `false`
- `bool conversationEnabled` = `true`
- `unsigned int maxHistoryMessages` = `20`

The documentation for this struct was generated from the following file:

- `tools/AIAssistant/AIConversationService.h`

15.12 AIConversationResponse Struct Reference

Public Attributes

- bool **success** = false
- std::string **content**
- std::string **errorMessage**
- int **httpStatusCode** = 0
- unsigned int **inputTokens** = 0
- unsigned int **outputTokens** = 0

The documentation for this struct was generated from the following file:

- tools/AIAssistant/AIConversationService.h

15.13 AIConversationService Class Reference

Public Member Functions

- void **setConfiguration** (const [AIConversationConfiguration](#) &configuration)
- const [AIConversationConfiguration](#) & **getConfiguration** () const
- bool **initialize** (std::string &errorMessage)
- void **setProviderClient** (std::unique_ptr< [AIProviderClient_if](#) > client)
- [AIConversationResponse](#) **send** (const std::string &conversationKey, const std::string &prompt)
- void **clearConversation** (const std::string &conversationKey)
- void **clearAllConversations** ()
- bool **isInitialized** () const

The documentation for this class was generated from the following files:

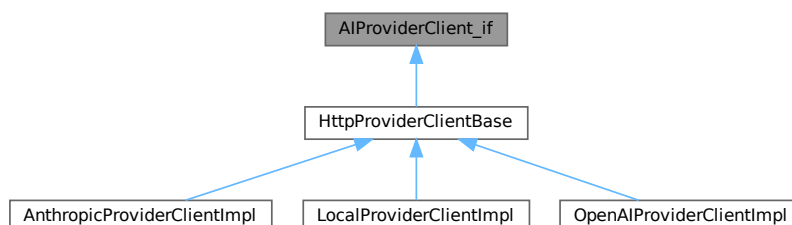
- tools/AIAssistant/AIConversationService.h
- tools/AIAssistant/AIConversationService.cpp

15.14 AIProviderClient_if Class Reference

Abstract interface for AI provider HTTP clients.

```
#include <tools/AIAssistant/AIProviderClient_if.h>
```

Inheritance diagram for AIProviderClient_if:



Public Member Functions

- virtual std::string **getProviderName** () const =0
Returns a human-readable provider name (e.g. "OpenAI", "Anthropic").
- virtual bool **isAvailable** () const =0

- Returns true when this client can attempt to send requests.*
- virtual void [setBaseUrl](#) (const std::string &url)=0
Sets the base URL for the provider API endpoint.
- virtual std::string [getBaseUrl](#) () const =0
Returns the currently configured base URL.
- virtual void [setTimeoutSeconds](#) (unsigned int seconds)=0
Sets the HTTP request timeout in seconds.
- virtual unsigned int [getTimeoutSeconds](#) () const =0
Returns the configured HTTP request timeout.
- virtual void [setApiKey](#) (const std::string &apiKey)=0
Stores the API key in memory.
- virtual void [clearApiKey](#) ()=0
Overwrites (best-effort) and clears the in-memory API key.
- virtual bool [hasApiKey](#) () const =0
Returns true when a non-empty API key is held.
- virtual [ProviderResponse](#) [send](#) (const [ProviderRequest](#) &request)=0
Sends a chat-completion request and returns the response.

15.14.1 Detailed Description

Abstract interface for AI provider HTTP clients.

A provider client wraps the HTTP call to one AI backend (OpenAI, Anthropic, a local Ollama server, etc.) and converts GenESys-internal request/response objects to and from the provider's wire format.

15.14.1.1 Security contract

- The API key is stored in-memory only; no getter exposes it.
- [clearApiKey\(\)](#) overwrites the buffer before releasing it.
- Implementations must not write the key to logs or trace output.

15.14.1.2 Ownership

Provider client objects do not own the assistants that use them. Callers that inject clients into [AIAssistant_if](#) are responsible for keeping the client alive for the duration of any ongoing request.

15.14.2 Member Function Documentation

15.14.2.1 clearApiKey()

```
virtual void AIProviderClient_if::clearApiKey ( ) [pure virtual]
```

Overwrites (best-effort) and clears the in-memory API key.
Implemented in [HttpProviderClientBase](#).

15.14.2.2 getBaseUrl()

```
virtual std::string AIProviderClient_if::getBaseUrl ( ) const [pure virtual]
```

Returns the currently configured base URL.
Implemented in [HttpProviderClientBase](#).

15.14.2.3 getProviderName()

```
virtual std::string AIProviderClient_if::getProviderName ( ) const [pure virtual]
```

Returns a human-readable provider name (e.g. "OpenAI", "Anthropic").
Implemented in [AnthropicProviderClientImpl](#), [LocalProviderClientImpl](#), and [OpenAIProviderClientImpl](#).

15.14.2.4 getTimeoutSeconds()

```
virtual unsigned int AIProviderClient_if::getTimeoutSeconds ( ) const [pure virtual]
```

Returns the configured HTTP request timeout.

Implemented in [HttpProviderClientBase](#).

15.14.2.5 hasApiKey()

```
virtual bool AIProviderClient_if::hasApiKey ( ) const [pure virtual]
```

Returns `true` when a non-empty API key is held.

Implemented in [HttpProviderClientBase](#).

15.14.2.6 isAvailable()

```
virtual bool AIProviderClient_if::isAvailable ( ) const [pure virtual]
```

Returns `true` when this client can attempt to send requests.

A client is considered available when it holds an API key (if required by the provider) and a reachable base URL.

Implemented in [AnthropicProviderClientImpl](#), [HttpProviderClientBase](#), [LocalProviderClientImpl](#), and [OpenAIProviderClientImpl](#).

15.14.2.7 send()

```
virtual ProviderResponse AIProviderClient_if::send (
    const ProviderRequest & request ) [pure virtual]
```

Sends a chat-completion request and returns the response.

This call is synchronous and may block for up to `timeoutSeconds`. On transport failure [ProviderResponse::success](#) is `false` and [ProviderResponse::errorMessage](#) is populated.

Implemented in [HttpProviderClientBase](#).

15.14.2.8 setApiKey()

```
virtual void AIProviderClient_if::setApiKey (
    const std::string & apiKey ) [pure virtual]
```

Stores the API key in memory.

Never write the key to disk or logs. Prefer loading from an environment variable via the owning [AIAssistant_if](#) rather than embedding keys in source.

Implemented in [HttpProviderClientBase](#).

15.14.2.9 setBaseUrl()

```
virtual void AIProviderClient_if::setBaseUrl (
    const std::string & url ) [pure virtual]
```

Sets the base URL for the provider API endpoint.

Implemented in [HttpProviderClientBase](#).

15.14.2.10 setTimeoutSeconds()

```
virtual void AIProviderClient_if::setTimeoutSeconds (
    unsigned int seconds ) [pure virtual]
```

Sets the HTTP request timeout in seconds.

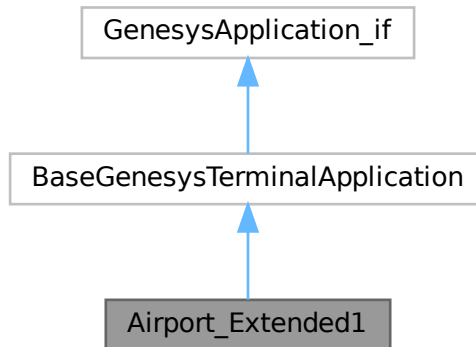
Implemented in [HttpProviderClientBase](#).

The documentation for this class was generated from the following file:

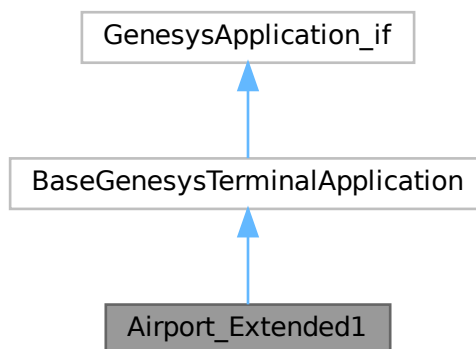
- `tools/AIAssistant/AIProviderClient_if.h`

15.15 Airport_Extended1 Class Reference

Inheritance diagram for Airport_Extended1:



Collaboration diagram for Airport_Extended1:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

15.15.1 Member Function Documentation

15.15.1.1 main()

```
int Airport_Extended1::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

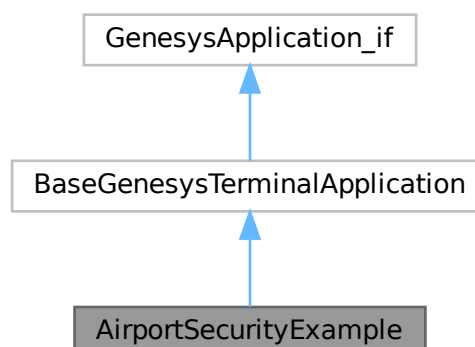
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

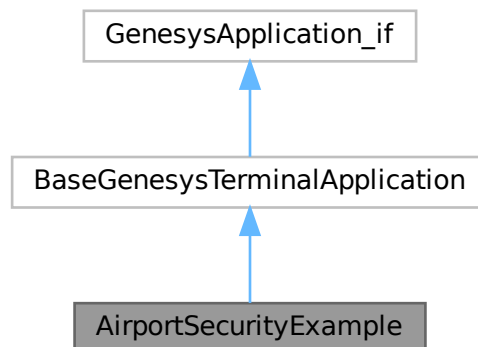
- applications/modelSpecific/arenaExamples/Airport_Extended1.h
- applications/modelSpecific/arenaExamples/Airport_Extended1.cpp

15.16 AirportSecurityExample Class Reference

Inheritance diagram for AirportSecurityExample:



Collaboration diagram for AirportSecurityExample:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

15.16.1 Member Function Documentation

15.16.1.1 main()

```
int AirportSecurityExample::main (
    int argc,
    char ** argv ) [override], [virtual]
```

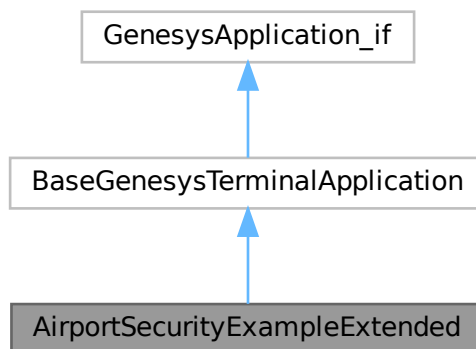
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

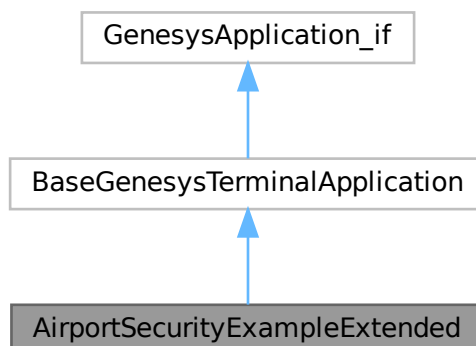
- applications/modelSpecific/arenaExamples/AirportSecurityExample.h
- applications/modelSpecific/arenaExamples/AirportSecurityExample.cpp

15.17 AirportSecurityExampleExtended Class Reference

Inheritance diagram for AirportSecurityExampleExtended:



Collaboration diagram for AirportSecurityExampleExtended:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members**Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)**

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

15.17.1 Member Function Documentation**15.17.1.1 main()**

```
int AirportSecurityExampleExtended::main (
    int argc,
    char ** argv ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/arenaExamples/AirportSecurityExampleExtended.h
- applications/modelSpecific/arenaExamples/AirportSecurityExampleExtended.cpp

15.18 AISecretStore Class Reference

Static helper for storing API keys in the OS keyring.

```
#include <tools/AIAssistant/AISecretStore.h>
```

Static Public Member Functions

- static bool [isAvailable](#) ()
Returns true if the keyring backend is reachable on this system.
- static bool [save](#) (const std::string &service, const std::string &account, const std::string &secret)
Saves secret under the given service and account labels.
- static std::optional< std::string > [load](#) (const std::string &service, const std::string &account)

Loads the secret stored under `service` and `account`.

- static bool `remove` (const std::string &service, const std::string &account)

Removes the stored secret.

15.18.1 Detailed Description

Static helper for storing API keys in the OS keyring.

On Linux, uses the `secret-tool` command-line utility (part of the `libsecret-tools` package) to access the GNOME Keyring / KWallet service via the freedesktop Secret Service protocol.

All methods return a failure indicator (`false` or `std::nullopt`) when `secret-tool` is not installed; callers should fall back to `env-var` loading.

API keys are never written to disk in plaintext; they live only in the session keyring managed by the OS.

15.18.1.1 Usage

```
const std::string svc = "genesys-ai-assistant";
const std::string acct = "openai";
if (AISecretStore::isAvailable()) {
    AISecretStore::save(svc, acct, rawKey);
    auto loaded = AISecretStore::load(svc, acct);
}
```

15.18.2 Member Function Documentation

15.18.2.1 isAvailable()

```
bool AISecretStore::isAvailable ( ) [static]
```

Returns `true` if the keyring backend is reachable on this system.

Checks that `secret-tool` is on `PATH` and that the D-Bus session bus is available (required by the Secret Service protocol).

15.18.2.2 load()

```
std::optional< std::string > AISecretStore::load (
    const std::string & service,
    const std::string & account ) [static]
```

Loads the secret stored under `service` and `account`.

Returns

The secret, or `std::nullopt` when not found or unavailable.

15.18.2.3 remove()

```
bool AISecretStore::remove (
    const std::string & service,
    const std::string & account ) [static]
```

Removes the stored secret.

Returns

`true` on success.

15.18.2.4 save()

```
bool AISecretStore::save (
    const std::string & service,
    const std::string & account,
    const std::string & secret ) [static]
```

Saves `secret` under the given `service` and `account` labels.

Returns

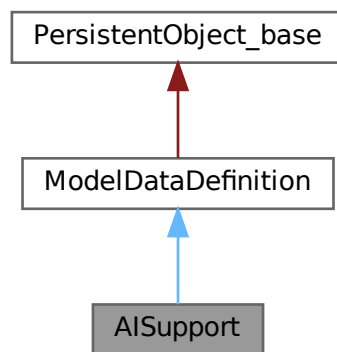
`true` on success, `false` if the keyring is unavailable or the operation failed.

The documentation for this class was generated from the following files:

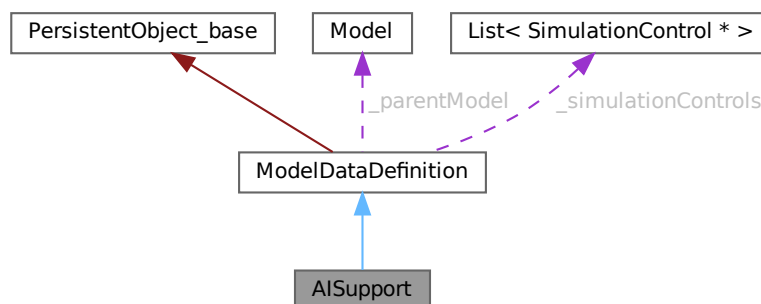
- tools/AIAssistant/AISecretStore.h
- tools/AIAssistant/AISecretStore.cpp

15.19 AISupport Class Reference

Inheritance diagram for AISupport:



Collaboration diagram for AISupport:

**Public Member Functions**

- **AISupport** ([Model](#) *model, std::string name="")
- [AIConversationResponse](#) **converse** (const std::string &conversationKey, const std::string &prompt)
- void **clearConversation** (const std::string &conversationKey)
- void **clearAllConversations** ()
- void **setProvider** (std::string value)
- std::string **getProvider** () const

- void **setModel** (std::string value)
- std::string **getModel** () const
- void **setBaseUrl** (std::string value)
- std::string **getBaseUrl** () const
- void **setApiKeyEnvironmentVariable** (std::string value)
- std::string **getApiKeyEnvironmentVariable** () const
- void **setSystemInstructions** (std::string value)
- std::string **getSystemInstructions** () const
- void **setTemperature** (double value)
- double **getTemperature** () const
- void **setMaxOutputTokens** (unsigned int value)
- unsigned int **getMaxOutputTokens** () const
- void **setTimeoutSeconds** (unsigned int value)
- unsigned int **getTimeoutSeconds** () const
- void **setHighReasoningMode** (bool value)
- bool **isHighReasoningMode** () const
- void **setConversationEnabled** (bool value)
- bool **isConversationEnabled** () const
- void **setMaxHistoryMessages** (unsigned int value)
- unsigned int **getMaxHistoryMessages** () const
- void **setClearHistoryBetweenReplications** (bool value)
- bool **isClearHistoryBetweenReplications** () const
- unsigned long **getCallCount** () const
- unsigned long **getFailureCount** () const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datatypeName, std::string name="", bool insert↵ IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- Releases resources owned by this data definition.*
- Util::identification **getId** () const
- Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
- Updates the logical name of this instance.*
- const std::string & **getName** () const
- Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)
- Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const
- Returns the user-visible label of this instance.*
- const std::string & **getClassname** () const
- Returns the runtime class name used by the plugin system.*
- **Model** * **getParentModel** () const
- Returns the parent model that owns this definition.*
- bool **isReportStatistics** () const
- Indicates whether this data definition contributes report statistics.*
- void **setReportStatistics** (bool reportStatistics)
- Enables or disables report-statistics generation for this definition.*
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

- Looks up a named internal child data definition.*
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↔ Enabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from ModelDataDefinition

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- `static void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.
- `static bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Validates a data definition instance.
- `static void InitBetweenReplications (ModelDataDefinition *modeldatum)`
Resets simulation-time state between replications.
- `static void CreateInternalData (ModelDataDefinition *modeldatum)`
Creates internal and attached data required by this instance.
- `static bool CreateRelatedDataElements (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_initBetweenReplications` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void `trace` (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))

not just an easy access to trace manager, but wrappers to check if specific trace level applies

- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

Traces text to the simulation output.

- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.19.1 Member Function Documentation

15.19.1.1 **_check()**

```
bool AISupport::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.19.1.2 **_initBetweenReplications()**

```
void AISupport::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.19.1.3 **_loadInstance()**

```
bool AISupport::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.19.1.4 **_saveInstance()**

```
void AISupport::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.19.1.5 show()

```
std::string AISupport::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

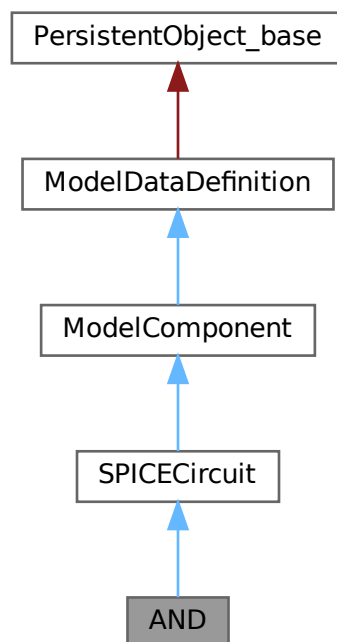
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

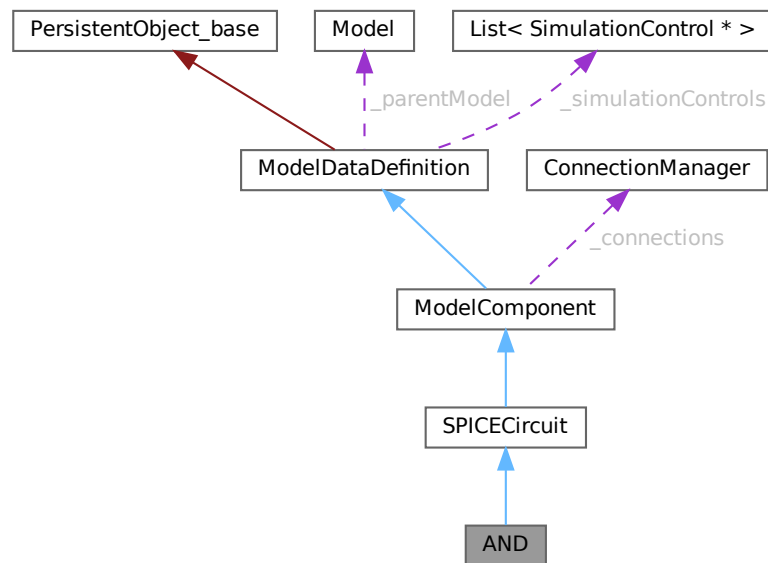
- plugins/data/AI/AISupport.h
- plugins/data/AI/AISupport.cpp

15.20 AND Class Reference

Inheritance diagram for AND:



Collaboration diagram for AND:



Public Member Functions

- **AND** ([Model](#) *model, std::string name="")
- void **Build** () override
- void **setPMOSWidth** (float PMOSWidth)
- void **setPMOSLength** (float length)
- void **setNMOSWidth** (float NMOSWidth)
- void **setNMOSLength** (float NMOSLength)
- void **setElectricalModel** (std::string electricalModel)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)

- Connects this component to another component.*

 - bool [hasBreakpointAt](#) ()
 - Indicates whether a simulation breakpoint is set on this component.*
 - void [setDescription](#) (std::string _description)
 - Updates the free-form description shown by the GUI.*
 - std::string [getDescription](#) () const
 - Returns the free-form description associated with this component.*
 - void [remove](#) ([ModelComponent](#) *comp)
 - Removes a connected component from this component.*

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- virtual ~[ModelDataDefinition](#) ()
- Releases resources owned by this data definition.*
- Util::identification [getId](#) () const
- Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)
- Updates the logical name of this instance.*
- const std::string & [getName](#) () const
- Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
- Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const
- Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const
- Returns the runtime class name used by the plugin system.*
- [Model](#) * [getParentModel](#) () const
- Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const
- Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)
- Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
- Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
- Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
- Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const
- Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))
- Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const
- Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int _modelLevel)
- Sets the model nesting level for this definition.*
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
- Returns the simulation controls owned by this data definition.*

- `TraceManager::Level` `getTraceLevelSpecific ()` const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` level)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` traceLevelspecific, bool traceLevelSpecific, bool Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled ()` const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation` * `GetPluginInformation ()`
- static `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")

Static Public Member Functions inherited from `SPICECircuit`

- static `PluginInformation` * `GetPluginInformation ()`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")
- static std::vector< std::string > `split` (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check` (`ModelComponent` *component)
Validates a component instance.
- static void `CreateInternalData` (`ModelComponent` *component)
Creates internal data required by a component instance.
- static void `SaveInstance` (`PersistenceRecord` *fields, `ModelComponent` *component)
Serializes a component instance into a persistence record.
- static void `DispatchEvent` (`Event` *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void `SaveInstance` (`PersistenceRecord` *fields, `ModelDataDefinition` *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool `Check` (`ModelDataDefinition` *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void `InitBetweenReplications` (`ModelDataDefinition` *modeldatum)

Resets simulation-time state between replications.

- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void [_createNonEditableDataDefinitions](#) () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)

- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.20.1 Member Function Documentation

15.20.1.1 `_loadInstance()`

```
bool AND::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

15.20.1.2 `_saveInstance()`

```
void AND::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

15.20.1.3 `Build()`

```
void AND::Build ( ) [override], [virtual]
```

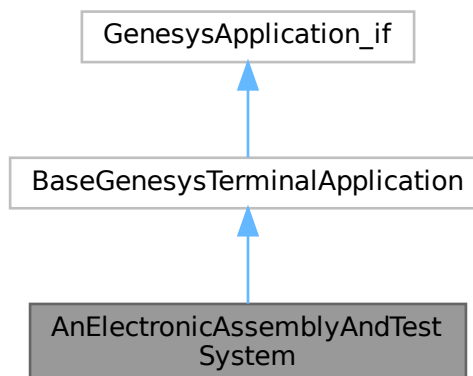
Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

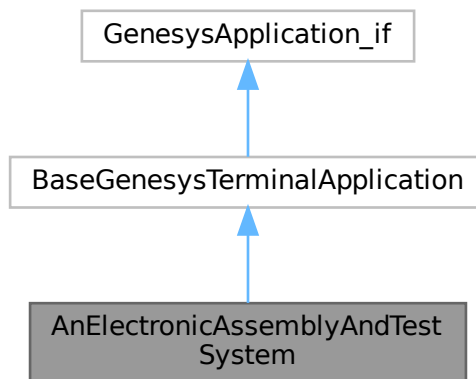
- `plugins/components/ElectronicsSimulation/SPICECircuit.h`
- `plugins/components/ElectronicsSimulation/SPICECircuit.cpp`

15.21 AnElectronicAssemblyAndTestSystem Class Reference

Inheritance diagram for AnElectronicAssemblyAndTestSystem:



Collaboration diagram for AnElectronicAssemblyAndTestSystem:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

15.21.1 Member Function Documentation

15.21.1.1 main()

```
int AnElectronicAssemblyAndTestSystem::main (
    int argc,
    char ** argv ) [override], [virtual]
```

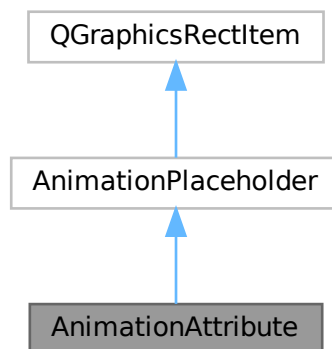
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

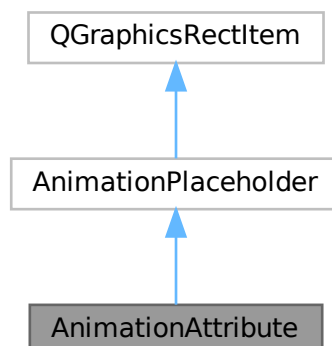
- applications/modelSpecific/teaching/AnElectronicAssemblyAndTestSystem.h
- applications/modelSpecific/teaching/AnElectronicAssemblyAndTestSystem.cpp

15.22 AnimationAttribute Class Reference

Inheritance diagram for AnimationAttribute:



Collaboration diagram for AnimationAttribute:



Additional Inherited Members**Public Member Functions inherited from [AnimationPlaceholder](#)**

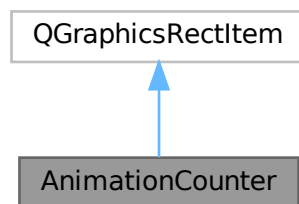
- **AnimationPlaceholder** (const QString &animationType)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- QString **getAnimationType** () const
- QString **getTargetName** () const
- void **setTargetName** (const QString &targetName)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** () const
- bool **isDrawingFinalized** () const

The documentation for this class was generated from the following files:

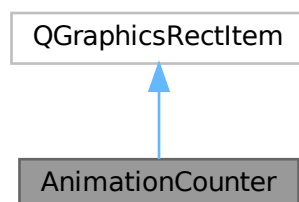
- applications/gui/genesys/animations/AnimationPlaceholder.h
- applications/gui/genesys/animations/AnimationPlaceholder.cpp

15.23 AnimationCounter Class Reference

Inheritance diagram for AnimationCounter:



Collaboration diagram for AnimationCounter:



Public Member Functions

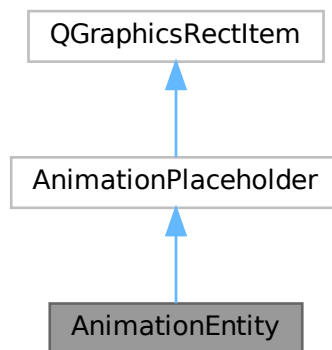
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- double **getValue** ()
- Counter * **getCounter** ()
- void **setValue** (double value)
- void **setCounter** (Counter *myCounter)
- void **setIdCounter** (int id)
- void **setNameCounter** (std::string name)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** ()
- bool **isDrawingFinalized** ()
- void **setWhenLoaded** (QList< Counter * > *counters)

The documentation for this class was generated from the following files:

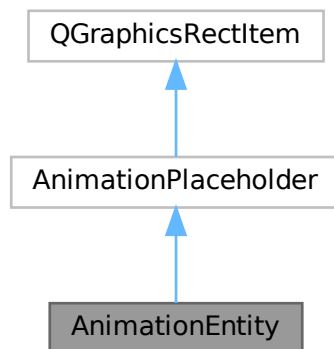
- applications/gui/genesys/animations/AnimationCounter.h
- applications/gui/genesys/animations/AnimationCounter.cpp

15.24 AnimationEntity Class Reference

Inheritance diagram for AnimationEntity:



Collaboration diagram for AnimationEntity:



Additional Inherited Members

Public Member Functions inherited from [AnimationPlaceholder](#)

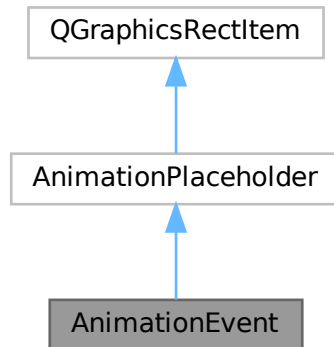
- **AnimationPlaceholder** (const QString &animationType)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- QString **getAnimationType** () const
- QString **getTargetName** () const
- void **setTargetName** (const QString &targetName)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** () const
- bool **isDrawingFinalized** () const

The documentation for this class was generated from the following files:

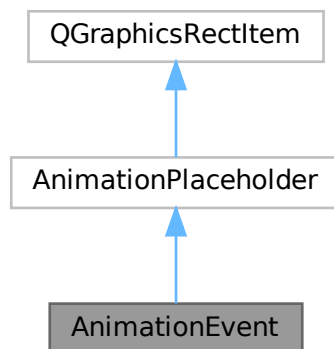
- applications/gui/genesys/animations/AnimationPlaceholder.h
- applications/gui/genesys/animations/AnimationPlaceholder.cpp

15.25 AnimationEvent Class Reference

Inheritance diagram for AnimationEvent:



Collaboration diagram for AnimationEvent:



Additional Inherited Members

Public Member Functions inherited from [AnimationPlaceholder](#)

- **AnimationPlaceholder** (const QString &animationType)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- QString **getAnimationType** () const
- QString **getTargetName** () const
- void **setTargetName** (const QString &targetName)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)

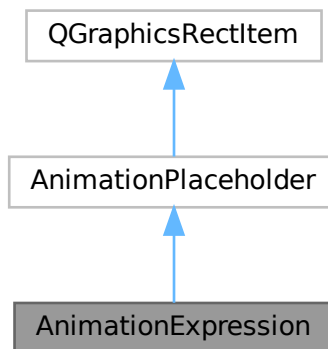
- bool **isDrawingInicialized** () const
- bool **isDrawingFinalized** () const

The documentation for this class was generated from the following files:

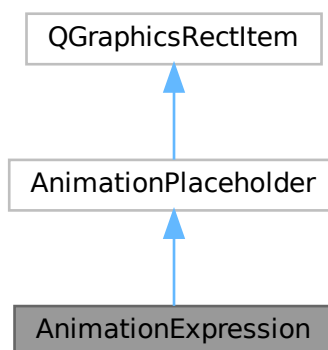
- applications/gui/genesys/animations/AnimationPlaceholder.h
- applications/gui/genesys/animations/AnimationPlaceholder.cpp

15.26 AnimationExpression Class Reference

Inheritance diagram for AnimationExpression:



Collaboration diagram for AnimationExpression:



Additional Inherited Members

Public Member Functions inherited from [AnimationPlaceholder](#)

- **AnimationPlaceholder** (const QString &animationType)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override

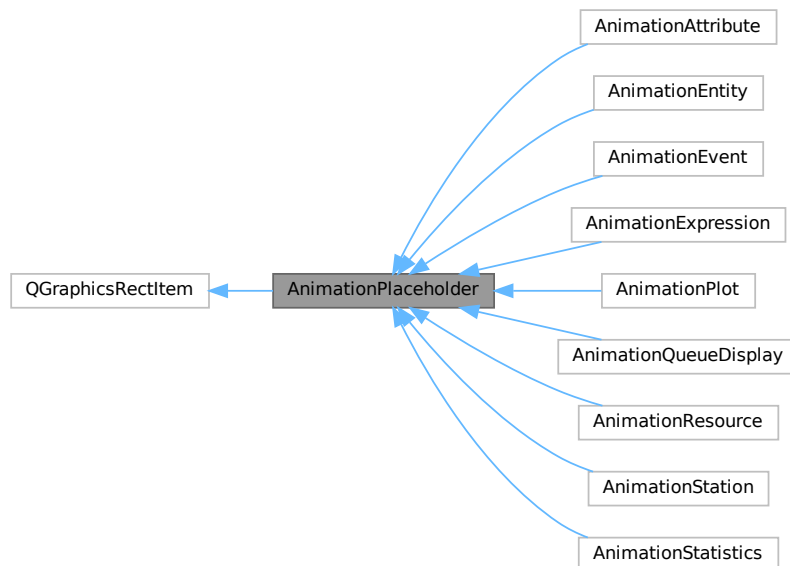
- QString **getAnimationType** () const
- QString **getTargetName** () const
- void **setTargetName** (const QString &targetName)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** () const
- bool **isDrawingFinalized** () const

The documentation for this class was generated from the following files:

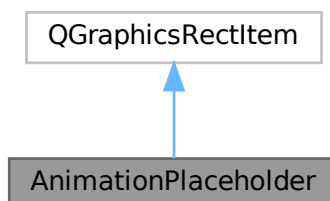
- applications/gui/genesys/animations/AnimationPlaceholder.h
- applications/gui/genesys/animations/AnimationPlaceholder.cpp

15.27 AnimationPlaceholder Class Reference

Inheritance diagram for AnimationPlaceholder:



Collaboration diagram for AnimationPlaceholder:



Public Member Functions

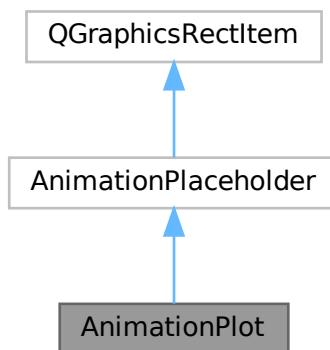
- **AnimationPlaceholder** (const QString &animationType)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- QString **getAnimationType** () const
- QString **getTargetName** () const
- void **setTargetName** (const QString &targetName)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** () const
- bool **isDrawingFinalized** () const

The documentation for this class was generated from the following files:

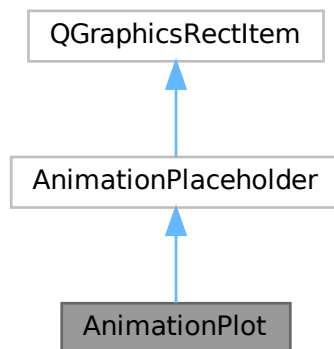
- applications/gui/genesys/animations/AnimationPlaceholder.h
- applications/gui/genesys/animations/AnimationPlaceholder.cpp

15.28 AnimationPlot Class Reference

Inheritance diagram for AnimationPlot:



Collaboration diagram for AnimationPlot:



Classes

- struct [DataSeries](#)

Public Types

- enum class **PlotType** { **Line** = 0 , **Bar** = 1 }

Public Member Functions

- QString **getTitle** () const
- void **setTitle** (const QString &title)
- QString **getYAxisTitle** () const
- void **setYAxisTitle** (const QString &title)
- QString **getXAxisTitle** () const
- void **setXAxisTitle** (const QString &title)
- PlotType **getPlotType** () const
- void **setPlotType** (PlotType type)
- QString **getDatasetsText** () const
- void **setDatasetsText** (const QString &datasetsText)
- bool **getShowGridLines** () const
- void **setShowGridLines** (bool show)
- bool **getShowTicks** () const
- void **setShowTicks** (bool show)
- double **getXAxisMin** () const
- void **setXAxisMin** (double value)
- double **getXAxisMax** () const
- void **setXAxisMax** (double value)
- double **getYAxisMin** () const
- void **setYAxisMin** (double value)
- double **getYAxisMax** () const
- void **setYAxisMax** (double value)
- void **clearValues** ()
- void **appendSample** ([Model](#) *model)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override

Public Member Functions inherited from AnimationPlaceholder

- **AnimationPlaceholder** (const QString &animationType)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- QString **getAnimationType** () const
- QString **getTargetName** () const
- void **setTargetName** (const QString &targetName)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** () const
- bool **isDrawingFinalized** () const

The documentation for this class was generated from the following files:

- applications/gui/genesys/animations/AnimationPlaceholder.h
- applications/gui/genesys/animations/AnimationPlaceholder.cpp

15.29 AnimationQueue Class Reference

Public Member Functions

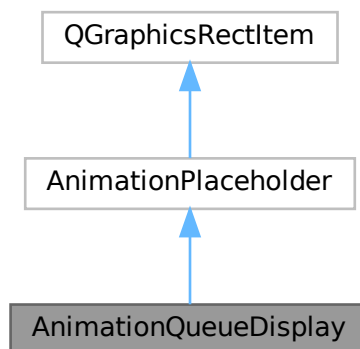
- **AnimationQueue** (ModelGraphicsScene *myScene, ModelComponent *component)
- GraphicalModelComponent * **getGraphicalComponent** () const
- void **addAnimationQueue** (bool visible)
- void **removeAnimationQueue** ()
- void **verifyAddAnimationQueue** (bool visible)
- void **verifyRemoveAnimationQueue** ()
- QPointF **calculatePositionImageQueue** (unsigned int indexQueue, unsigned int sizeQueue, unsigned int width, unsigned int height)

The documentation for this class was generated from the following files:

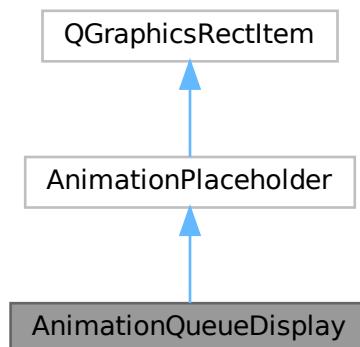
- applications/gui/genesys/animations/AnimationQueue.h
- applications/gui/genesys/animations/AnimationQueue.cpp

15.30 AnimationQueueDisplay Class Reference

Inheritance diagram for AnimationQueueDisplay:



Collaboration diagram for AnimationQueueDisplay:



Additional Inherited Members

Public Member Functions inherited from [AnimationPlaceholder](#)

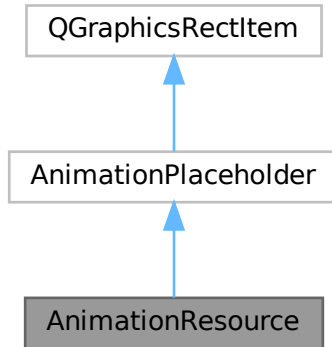
- **AnimationPlaceholder** (const QString &animationType)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- QString **getAnimationType** () const
- QString **getTargetName** () const
- void **setTargetName** (const QString &targetName)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** () const
- bool **isDrawingFinalized** () const

The documentation for this class was generated from the following files:

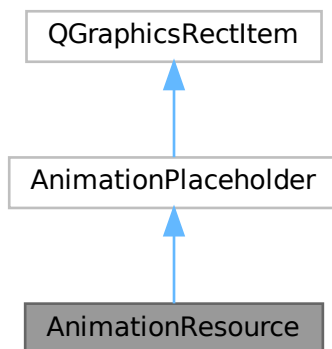
- applications/gui/genesys/animations/AnimationPlaceholder.h
- applications/gui/genesys/animations/AnimationPlaceholder.cpp

15.31 AnimationResource Class Reference

Inheritance diagram for AnimationResource:



Collaboration diagram for AnimationResource:



Additional Inherited Members

Public Member Functions inherited from [AnimationPlaceholder](#)

- **AnimationPlaceholder** (const QString &animationType)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- QString **getAnimationType** () const
- QString **getTargetName** () const
- void **setTargetName** (const QString &targetName)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)

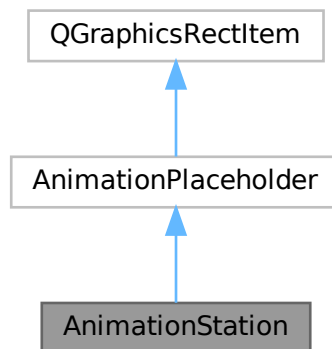
- bool **isDrawingInicialized** () const
- bool **isDrawingFinalized** () const

The documentation for this class was generated from the following files:

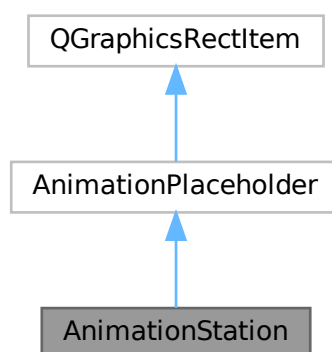
- applications/gui/genesys/animations/AnimationPlaceholder.h
- applications/gui/genesys/animations/AnimationPlaceholder.cpp

15.32 AnimationStation Class Reference

Inheritance diagram for AnimationStation:



Collaboration diagram for AnimationStation:



Additional Inherited Members

Public Member Functions inherited from [AnimationPlaceholder](#)

- **AnimationPlaceholder** (const QString &animationType)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override

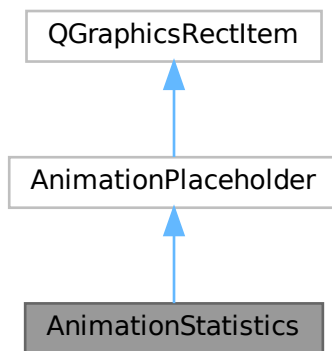
- QString **getAnimationType** () const
- QString **getTargetName** () const
- void **setTargetName** (const QString &targetName)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** () const
- bool **isDrawingFinalized** () const

The documentation for this class was generated from the following files:

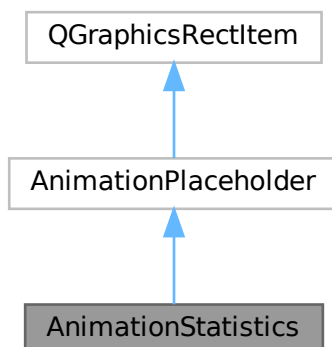
- applications/gui/genesys/animations/AnimationPlaceholder.h
- applications/gui/genesys/animations/AnimationPlaceholder.cpp

15.33 AnimationStatistics Class Reference

Inheritance diagram for AnimationStatistics:



Collaboration diagram for AnimationStatistics:



Additional Inherited Members

Public Member Functions inherited from [AnimationPlaceholder](#)

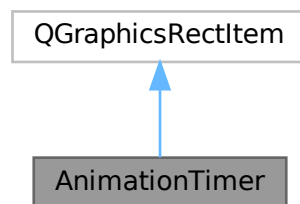
- **AnimationPlaceholder** (const QString &animationType)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- QString **getAnimationType** () const
- QString **getTargetName** () const
- void **setTargetName** (const QString &targetName)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** () const
- bool **isDrawingFinalized** () const

The documentation for this class was generated from the following files:

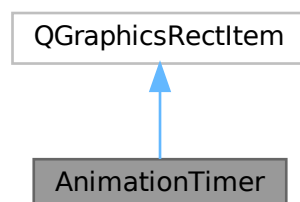
- applications/gui/genesys/animations/AnimationPlaceholder.h
- applications/gui/genesys/animations/AnimationPlaceholder.cpp

15.34 AnimationTimer Class Reference

Inheritance diagram for AnimationTimer:



Collaboration diagram for AnimationTimer:



Public Member Functions

- **AnimationTimer** ([ModelGraphicsScene](#) *myScene)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- double **getTime** ()
- unsigned int **getInitialHours** ()
- unsigned int **getInitialMinutes** ()
- unsigned int **getInitialSeconds** ()
- unsigned int **getHours** ()
- unsigned int **getMinutes** ()
- unsigned int **getSeconds** ()
- Util::TimeFormat **getTimeFormat** ()
- void **setTime** (double value)
- void **setInitialHours** (unsigned int initialHours)
- void **setInitialMinutes** (unsigned int initialMinutes)
- void **setInitialSeconds** (unsigned int initialSeconds)
- void **setHours** (unsigned int hours)
- void **setMinutes** (unsigned int minutes)
- void **setSeconds** (unsigned int seconds)
- void **setTimeFormat** (Util::TimeFormat timeFormat)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- void **convertSeconds** ()
- bool **isDrawingInialized** ()
- bool **isDrawingFinalized** ()

Static Public Member Functions

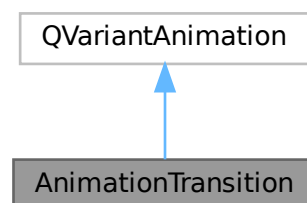
- static void **setConversionFactorToSeconds** (double factor)

The documentation for this class was generated from the following files:

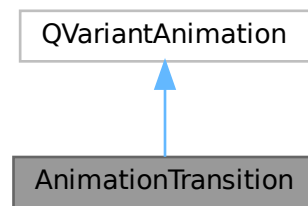
- applications/gui/genesys/animations/AnimationTimer.h
- applications/gui/genesys/animations/AnimationTimer.cpp

15.35 AnimationTransition Class Reference

Inheritance diagram for AnimationTransition:



Collaboration diagram for AnimationTransition:



Public Slots

- void **onAnimationValueChanged** (const QVariant &value)
- void **onAnimationFinished** ()

Public Member Functions

- **AnimationTransition** ([ModelGraphicsScene](#) *myScene, [ModelComponent](#) *graphicalStartComponent, [ModelComponent](#) *graphicalEndComponent, bool viewSimulation)
- [GraphicalModelComponent](#) * **getGraphicalStartComponent** () const
- [GraphicalModelComponent](#) * **getGraphicalEndComponent** () const
- [GraphicalConnection](#) * **getGraphicalConnection** () const
- QList< QPointF > **getPointsForAnimation** () const
- [GraphicalImageAnimation](#) * **getImageAnimation** () const
- unsigned int **getPortNumber** () const
- bool **isReadyToRun** () const
- void **setImageAnimation** ([GraphicalImageAnimation](#) *imageAnimation)
- void **startAnimation** ()
- void **stopAnimation** ()
- void **restartAnimation** ()
- void **configureAnimation** ()
- void **updateDurationIfNeeded** ()
- void **connectValueChangedSignal** ()
- void **connectFinishedSignal** ()

Static Public Member Functions

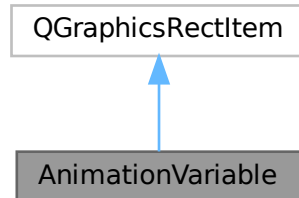
- static double **getTimeExecution** ()
- static void **setTimeExecution** (double timeExecution)
- static void **setPause** (bool pause)
- static void **setRunning** (bool running)

The documentation for this class was generated from the following files:

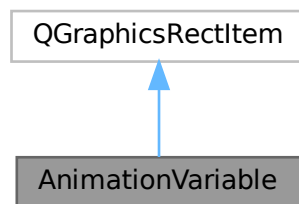
- applications/gui/genesys/animations/AnimationTransition.h
- applications/gui/genesys/animations/AnimationTransition.cpp

15.36 AnimationVariable Class Reference

Inheritance diagram for AnimationVariable:



Collaboration diagram for AnimationVariable:



Public Member Functions

- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- double **getValue** ()
- [ModelDataDefinition](#) * **getDataDefinition** () const
- [Variable](#) * **getVariable** ()
- [Attribute](#) * **getAttribute** ()
- QString **getTargetName** () const
- int **getTargetId** () const
- unsigned int **getDimensionCount** () const
- unsigned int **getDimensionSize** (unsigned int dimensionIndex) const
- unsigned int **getSliceIndex** (unsigned int dimensionIndex) const
- void **setValue** (double value)
- void **setDataDefinition** ([ModelDataDefinition](#) *definition)
- void **setVariable** ([Variable](#) *variable)
- void **setAttribute** ([Attribute](#) *attribute)
- void **setModel** ([Model](#) *model)
- void **setTargetName** (std::string name)
- void **setIdVariable** (int id)
- void **setSliceIndex** (unsigned int dimensionIndex, unsigned int value)
- void **setWhenLoaded** (QList< [ModelDataDefinition](#) * > *definitions)

- void **refreshValue** ([Entity](#) *entity=nullptr)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** ()
- bool **isDrawingFinalized** ()

The documentation for this class was generated from the following files:

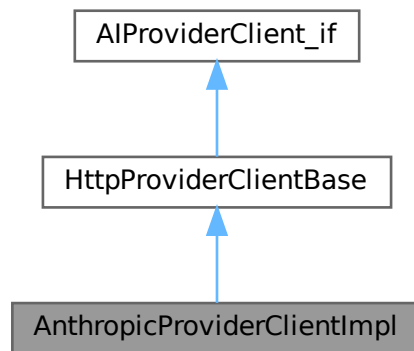
- applications/gui/genesys/animations/AnimationVariable.h
- applications/gui/genesys/animations/AnimationVariable.cpp

15.37 AnthropicProviderClientImpl Class Reference

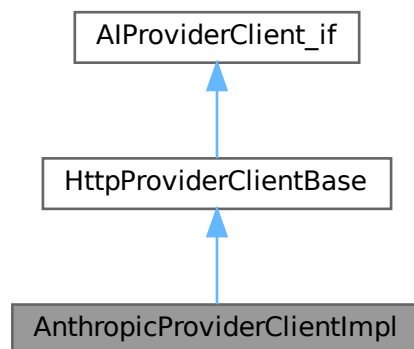
AI provider client for the Anthropic Messages API.

```
#include <tools/AIAssistant/AnthropicProviderClientImpl.h>
```

Inheritance diagram for AnthropicProviderClientImpl:



Collaboration diagram for AnthropicProviderClientImpl:



Public Member Functions

- **AnthropicProviderClientImpl** ()
Constructs the client with the standard Anthropic base URL.
- `std::string` **getProviderName** () const override
Returns a human-readable provider name (e.g. "OpenAI", "Anthropic").
- `bool` **isAvailable** () const override
Returns `true` when an API key is set and a base URL is configured.
- `void` **setApiVersion** (const `std::string` &version)
Sets the Anthropic API version header value.
- `std::string` **getApiVersion** () const
Returns the configured Anthropic API version string.
- `void` **setThinkingBudgetTokens** (unsigned int budget)
Sets the extended-thinking budget in tokens (default: 10000).
- unsigned int **getThinkingBudgetTokens** () const
Returns the thinking budget in tokens.

Public Member Functions inherited from [HttpProviderClientBase](#)

- **HttpProviderClientBase** (const [HttpProviderClientBase](#) &)=delete
- [HttpProviderClientBase](#) & **operator=** (const [HttpProviderClientBase](#) &)=delete
- `bool` **isAvailable** () const override
Returns `true` when this client can attempt to send requests.
- `void` **setBaseUrl** (const `std::string` &url) override
Sets the base URL for the provider API endpoint.
- `std::string` **getBaseUrl** () const override
Returns the currently configured base URL.
- `void` **setTimeoutSeconds** (unsigned int seconds) override
Sets the HTTP request timeout in seconds.
- unsigned int **getTimeoutSeconds** () const override
Returns the configured HTTP request timeout.
- `void` **setApiKey** (const `std::string` &apiKey) override
Stores the API key in memory.
- `void` **clearApiKey** () override
Overwrites (best-effort) and clears the in-memory API key.
- `bool` **hasApiKey** () const override
Returns `true` when a non-empty API key is held.
- [ProviderResponse](#) **send** (const [ProviderRequest](#) &request) override
Executes the HTTP request via curl and returns the provider response.

Protected Member Functions

- `std::string` **_buildRequestBody** (const [ProviderRequest](#) &request) const override
Serialises `request` to the provider's JSON wire format.
- [ProviderResponse](#) **_parseResponseBody** (const `std::string` &rawBody, int httpStatus) const override
Deserialises `rawBody` (the full curl stdout) into a [ProviderResponse](#).
- `std::string` **_authorizationHeader** () const override
Returns the Authorization header line without the "header = " prefix.
- `std::vector< std::string >` **_extraRequestHeaders** () const override
Returns additional HTTP headers to send with every request.

Additional Inherited Members

Static Public Member Functions inherited from [HttpProviderClientBase](#)

- static std::string **_escapeJsonString** (const std::string &s)
Escapes a plain string for safe embedding inside a JSON string value.
- static std::string **_extractJsonString** (const std::string &json, const std::string &key)
Extracts the value of a top-level JSON string field by key. Returns an empty string when not found or value is not a quoted string.
- static int **_extractJsonInt** (const std::string &json, const std::string &key)
Extracts the value of a top-level JSON integer field by key. Returns 0 when not found.
- static double **_extractJsonDouble** (const std::string &json, const std::string &key)
Extracts the value of a top-level JSON number field (int or float) by key. Returns 0.0 when not found.
- static std::string **_buildMessagesArray** (const std::vector< [ChatMessage](#) > &messages)
Serialises a vector of [ChatMessage](#) objects to a JSON array string.
- static std::string **_extractFirstArrayElement** (const std::string &json, const std::string &arrayKey)
Extracts the raw substring of the first element of a named JSON array. Returns an empty string when not found.

Protected Attributes inherited from [HttpProviderClientBase](#)

- std::string **_baseUrl**
- unsigned int **_timeoutSeconds** = 30
- std::string **_apiKey**
In-memory API key.

15.37.1 Detailed Description

AI provider client for the Anthropic Messages API.

Wraps the Anthropic `/v1/messages` endpoint (Claude 3/4 model family). Default base URL: <https://api.anthropic.com/v1/messages>

Wire format differences from the OpenAI schema:

- Auth header: `x-api-key: <key>` (not `Authorization: Bearer`).
- Required extra header: `anthropic-version: 2023-06-01`.
- Request body: `max_tokens` (required), `system` is a top-level field (not a message in the array), `messages` contains only user/assistant turns.
- Response: `content` is in `content[0].text`, `tokens` in `usage.input_tokens / usage.output_tokens`.
- Extended thinking: when `highReasoningMode` is true, a `thinking.type=enabled` budget block is added to the request and the text is extracted from the first content block whose `type` is "text".

15.37.1.1 API key

Sent as `x-api-key: <key>`. Written to the curl config file at mode 0600; never placed on the command line.

15.37.2 Member Function Documentation

15.37.2.1 `_authorizationHeader()`

```
std::string AnthropicProviderClientImpl::_authorizationHeader ( ) const [override], [protected], [virtual]
```

Returns the Authorization header line without the "header = " prefix.

Example return values:

- "Authorization: Bearer sk-..." (OpenAI, LocalProvider)

- "x-api-key: sk-ant-..." (Anthropic)

Called only from within [send\(\)](#); the value is written to a 0600 temp file and never logged.

Implements [HttpProviderClientBase](#).

15.37.2.2 _buildRequestBody()

```
std::string AnthropicProviderClientImpl::_buildRequestBody (
    const ProviderRequest & request ) const [override], [protected], [virtual]
```

Serialises `request` to the provider's JSON wire format.

Implements [HttpProviderClientBase](#).

15.37.2.3 _extraRequestHeaders()

```
std::vector< std::string > AnthropicProviderClientImpl::_extraRequestHeaders ( ) const [override],
[protected], [virtual]
```

Returns additional HTTP headers to send with every request.

Default implementation returns an empty vector. Anthropic overrides to add "anthropic-version: 2023-06-01".

Each element must be a complete header line, e.g. "anthropic-version: 2023-06-01".

Reimplemented from [HttpProviderClientBase](#).

15.37.2.4 _parseResponseBody()

```
ProviderResponse AnthropicProviderClientImpl::_parseResponseBody (
    const std::string & rawBody,
    int httpStatus ) const [override], [protected], [virtual]
```

Deserialises `rawBody` (the full curl stdout) into a [ProviderResponse](#).

Parameters

<i>rawBody</i>	Complete HTTP response body as received from curl.
<i>httpStatus</i>	HTTP status code captured from curl exit / response line, or 0 when unavailable.

Implements [HttpProviderClientBase](#).

15.37.2.5 getProviderName()

```
std::string AnthropicProviderClientImpl::getProviderName ( ) const [override], [virtual]
```

Returns a human-readable provider name (e.g. "OpenAI", "Anthropic").

Implements [AIProviderClient_if](#).

15.37.2.6 isAvailable()

```
bool AnthropicProviderClientImpl::isAvailable ( ) const [override], [virtual]
```

Returns `true` when an API key is set and a base URL is configured.

Implements [AIProviderClient_if](#).

15.37.2.7 setApiVersion()

```
void AnthropicProviderClientImpl::setApiVersion (
    const std::string & version )
```

Sets the Anthropic API version header value.

Defaults to "2023-06-01" which is the stable version as of mid-2026. Override only when Anthropic releases a newer stable version that changes the wire format.

15.37.2.8 setThinkingBudgetTokens()

```
void AnthropicProviderClientImpl::setThinkingBudgetTokens (
    unsigned int budget )
```

Sets the extended-thinking budget in tokens (default: 10000).

Used only when [ProviderRequest::highReasoningMode](#) is true. Must be less than [ProviderRequest::maxTokens](#).

The documentation for this class was generated from the following files:

- tools/AIAssistant/AnthropicProviderClientImpl.h
- tools/AIAssistant/AnthropicProviderClientImpl.cpp

15.38 ApiRouter Class Reference

Routes HTTP requests to Web API operations.

```
#include <applications/worker/api/ApiRouter.h>
```

Public Member Functions

- [ApiRouter](#) ([SimulatorSessionService](#) &simulatorService)
Creates an API router bound to the simulator session service.
- [HttpResponse](#) handle (const [HttpRequest](#) &request) const
Handles one HTTP request and returns the corresponding response.

15.38.1 Detailed Description

Routes HTTP requests to Web API operations.

15.38.2 Constructor & Destructor Documentation

15.38.2.1 ApiRouter()

```
ApiRouter::ApiRouter (
    SimulatorSessionService & simulatorService ) [explicit]
```

Creates an API router bound to the simulator session service.

Dispatches HTTP routes for the Genesys web API.

Parameters

<i>simulatorService</i>	Service used to execute API operations.
-------------------------	---

The router keeps request parsing, token extraction, payload decoding and JSON formatting in one place so the HTTP server can stay transport-only.

15.38.3 Member Function Documentation

15.38.3.1 handle()

```
HttpResponse ApiRouter::handle (
    const HttpRequest & request ) const
```

Handles one HTTP request and returns the corresponding response.

Resolves a request into one of the public API operations.

Parameters

<i>request</i>	HTTP request containing method, path, headers, and body.
----------------	--

Returns

HTTP response with status, content type, and JSON payload.

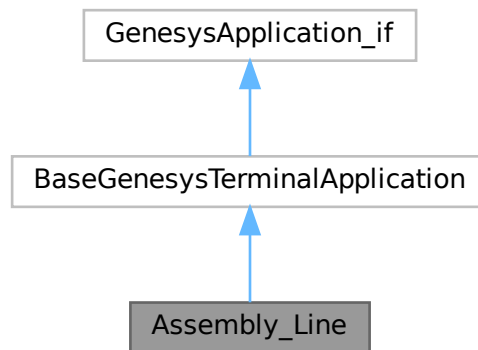
The route chain stays intentionally explicit so the generated PDF makes the supported endpoints and their validation rules easy to audit.

The documentation for this class was generated from the following files:

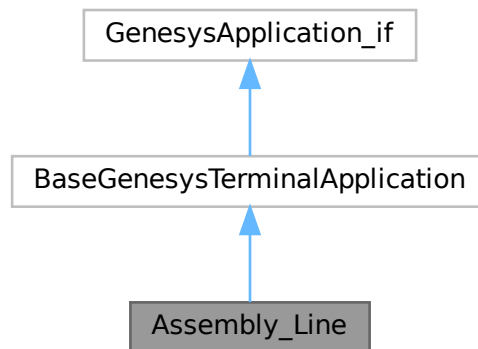
- applications/worker/api/ApiRouter.h
- applications/worker/api/ApiRouter.cpp

15.39 Assembly_Line Class Reference

Inheritance diagram for Assembly_Line:



Collaboration diagram for Assembly_Line:



Public Member Functions

- virtual int **main** (int argc, char **argv) override

Public Member Functions inherited from **BaseGenesysTerminalApplication**

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

15.39.1 Member Function Documentation

15.39.1.1 main()

```
int Assembly_Line::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/arenaExamples/Assembly_Line.h
- applications/modelSpecific/arenaExamples/Assembly_Line.cpp

15.40 testing::internal::AssertHelper Class Reference

Public Member Functions

- **AssertHelper** (TestPartResult::Type type, const char *file, int line, const char *message)
- void **operator=** (const [Message](#) &message) const

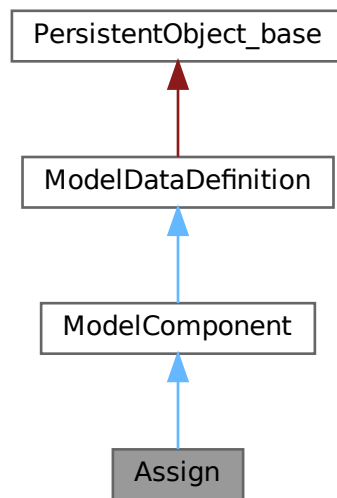
The documentation for this class was generated from the following files:

- gtest/gtest/gtest.h
- gtest/src/gtest.cc

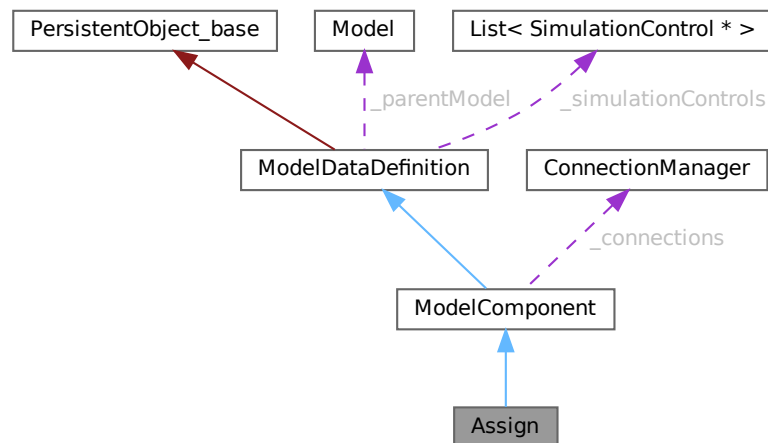
15.41 Assign Class Reference

```
#include <plugins/components/Logic/Assign.h>
```

Inheritance diagram for Assign:



Collaboration diagram for Assign:



Public Member Functions

- **Assign** (`Model *model`, `std::string name=""`)
- `virtual std::string show ()` override
- *virtual public methods*
- `List< Assignment * > *getAssignments ()` const
- `void addAssignment (Assignment *newAssignment)`
- `void removeAssignment (Assignment *assignment)`

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.

- void `setHasChanged` (bool `hasChanged`)
Updates the changed flag for this data definition.
- unsigned int `getLevel` () const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel` (unsigned int `_modelLevel`)
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls` () const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level` `getTraceLevelSpecific` () const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` `level`)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` `traceLevelSpecific`, bool `traceLevelSpecificEnabled`=true)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool `traceLevelSpecificEnabled`)
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation * GetPluginInformation` ()
- static `ModelComponent * LoadInstance` (`Model *model`, `PersistenceRecord *fields`)
- static `ModelDataDefinition * NewInstance` (`Model *model`, `std::string name=""`)

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent * LoadInstance` (`Model *model`, `PersistenceRecord *fields`)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check` (`ModelComponent *component`)
Validates a component instance.
- static void `CreateInternalData` (`ModelComponent *component`)
Creates internal data required by a component instance.
- static void `SaveInstance` (`PersistenceRecord *fields`, `ModelComponent *component`)
Serializes a component instance into a persistence record.
- static void `DispatchEvent` (`Event *event`)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition * LoadInstance` (`Model *model`, `PersistenceRecord *fields`, bool `insertIntoModel`)
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition * NewInstance` (`Model *model`, `std::string name=""`)
Creates a new instance of a data definition using the plugin registry.
- static void `SaveInstance` (`PersistenceRecord *fields`, `ModelDataDefinition *modeldatum`)
Serializes a data definition instance into a persistence record.
- static bool `Check` (`ModelDataDefinition *modeldatum`, `std::string &errorMessage`)
Validates a data definition instance.
- static void `InitBetweenReplications` (`ModelDataDefinition *modeldatum`)
Resets simulation-time state between replications.
- static void `CreateInternalData` (`ModelDataDefinition *modeldatum`)

Creates internal and attached data required by this instance.

- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)

Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()

- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.41.1 Detailed Description

[Assign](#) module DESCRIPTION This module is used for assigning new values to variables, entity attributes, entity types, entity pictures, or other system variables. Multiple assignments can be made with a single [Assign](#) module. TYPICAL USES Accumulate the number of subassemblies added to a part Change an entity's type to represent the customer copy of a multi-page form Establish a customer's priority PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Assignments Specifies the one or more assignments that will be made when an entity executes the module. Type Type of assignment to be made. Other can include system variables, such as resource capacity or simulation end time. [Variable](#) Name Name of the variable that will be assigned a new value when an entity enters the module. Applies only when Type is [Variable](#), [Variable](#) Array (1D), or [Variable](#) Array (2D). Row Specifies the row index for a variable array. Column Specifies the column index for a variable array. [Attribute](#) Name Name of the entity attribute that will be assigned a new value when the entity enters the module. Applies only when Type is [Attribute](#). [Entity](#) Type New entity type that will be assigned to the entity when the entity enters the module. Applies only when Type is [Entity](#) Type. [Entity](#) Picture New entity picture that will be assigned to the entity when the entity enters the module. Applies only when Type is [Entity](#) Picture. Other Identifies the special system variable that will be assigned a new value when an entity enters the module. Applies only when Type is Other. New Value [Assignment](#) value of the attribute, variable, or other system variable. Does not apply when Type is [Entity](#) Type or [Entity](#) Picture.

15.41.2 Member Function Documentation

15.41.2.1 `_check()`

```
bool Assign::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.41.2.2 `_createEditableDataDefinitions()`

```
void Assign::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDats. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dinamically linked to to simulator kernel to reflect the changes

Reimplemented from [ModelDataDefinition](#).

15.41.2.3 `_loadInstance()`

```
bool Assign::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.41.2.4 `_onDispatchEvent()`

```
void Assign::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.41.2.5 `_saveInstance()`

```
void Assign::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.41.2.6 `show()`

```
std::string Assign::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/Logic/Assign.h
- plugins/components/Logic/Assign.cpp

15.42 Assignment Class Reference

```
#include <plugins/data/Logic/AssignmentItem.h>
```

Public Member Functions

- **Assignment** ([Model](#) *model, std::string destination, std::string expression="1", bool isAttributeNotVariable=true)
- **Assignment** (std::string destination, std::string expression, bool isAttributeNotVariable=true)
- void **setDestination** (std::string _destination)
- std::string **getDestination** () const
- void **setExpression** (std::string _expression)
- std::string **getExpression** () const
- void **setAttributeNotVariable** (bool isAttributeNotVariable)
- bool **isAttributeNotVariable** () const
- void **setChangeCallback** (std::function< void([Assignment](#) &)> callback)
- std::string **getName** () const
- [List](#)< [SimulationControl](#) * > * **getSimulationControls** () const
- void **ensureSimulationControls** ([Model](#) *model)
- void **_addSimulationControl** ([SimulationControl](#) *control)
- std::string **getTypeDC** ()
- bool **loadInstance** ([PersistenceRecord](#) *fields, unsigned int parentIndex)
- void **saveInstance** ([PersistenceRecord](#) *fields, unsigned int parentIndex, bool saveDefault)

15.42.1 Detailed Description

While the assign ans sequence classes allows you to perform multiple assignments, the assignmentItem class defines an assignment itself.

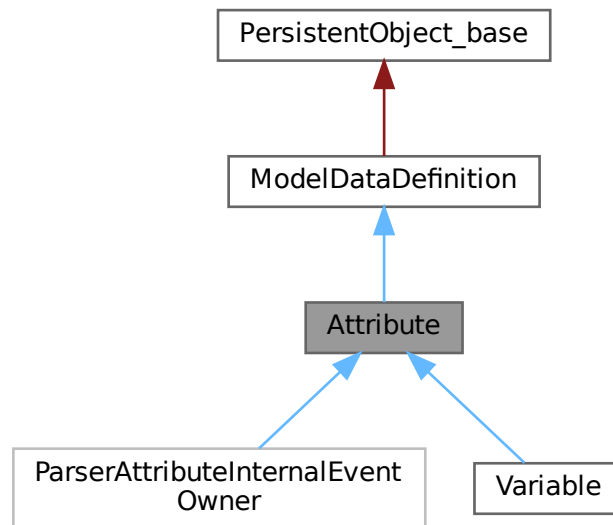
The documentation for this class was generated from the following files:

- plugins/data/Logic/AssignmentItem.h
- plugins/data/Logic/AssignmentItem.cpp

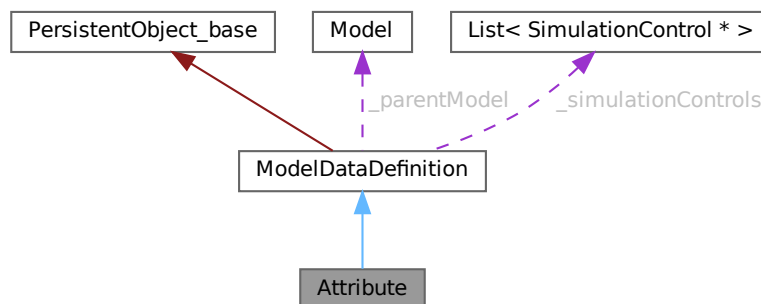
15.43 Attribute Class Reference

```
#include <kernel/simulator/essentialPlugins/Attribute.h>
```

Inheritance diagram for Attribute:



Collaboration diagram for Attribute:



Public Member Functions

- **Attribute** (`Model` *model, std::string name="", std::string dataDefinitionTypename=`Util::TypeOf< Attribute >()`)
Creates an entity attribute definition in the model.
- virtual **~Attribute** () override
Releases sparse initial values owned by this definition.
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- virtual std::string **getInitialValuesText** () const
Returns the initial values serialized in bracket notation.
- virtual void **setInitialValuesText** (std::string valuesText)

- Parses bracket notation and replaces the initial value store.*
- double **getInitialValue** (std::string index="")
Reads the initial sparse value, returning 0.0 when the index is absent.
- void **setInitialValue** (double value, std::string index="")
Writes the initial sparse value at the scalar or indexed position.
- void **setInitialValues** (const std::vector< std::pair< std::string, double > > values)
Replaces/extends initial sparse values from textual index/value pairs.
- void **copyInitialValuesFrom** (const [Attribute](#) &source)
Copies the full initial sparse store from another attribute.
- void **insertDimentionSize** (unsigned int size)
Appends one dimension size to the attribute definition.
- std::list< unsigned int > * **getDimensionSizes** () const
Returns dimension sizes for compatibility with existing callers.
- std::map< std::string, double > * **getInitialValues** () const
Returns mutable initial sparse values for compatibility with existing callers.
- [SparseValueStore](#) * **getInitialValueStore** ()
Returns the sparse initial value store used by this attribute.
- bool **isInitialValuesTextValid** () const
Returns whether the last textual initial value is syntactically valid.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.

- bool `hasChanged` () const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged` (bool `hasChanged`)
Updates the changed flag for this data definition.
- unsigned int `getLevel` () const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel` (unsigned int `_modelLevel`)
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls` () const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level` `getTraceLevelSpecific` () const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` `level`)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` `traceLevelSpecific`, bool `traceLevelSpecificEnabled`)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool `traceLevelSpecificEnabled`)
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation * GetPluginInformation` ()
- static `ModelDataDefinition * LoadInstance` (`Model *model`, `PersistenceRecord *fields`)
- static `ModelDataDefinition * NewInstance` (`Model *model`, `std::string name=""`)

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition * LoadInstance` (`Model *model`, `PersistenceRecord *fields`, bool `insertIntoModel`)
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition * NewInstance` (`Model *model`, `std::string name=""`)
Creates a new instance of a data definition using the plugin registry.
- static void `SaveInstance` (`PersistenceRecord *fields`, `ModelDataDefinition *modeldatum`)
Serializes a data definition instance into a persistence record.
- static bool `Check` (`ModelDataDefinition *modeldatum`, `std::string &errorMessage`)
Validates a data definition instance.
- static void `InitBetweenReplications` (`ModelDataDefinition *modeldatum`)
Resets simulation-time state between replications.
- static void `CreateInternalData` (`ModelDataDefinition *modeldatum`)
Creates internal and attached data required by this instance.
- static bool `CreateRelatedDataElements` (`ModelDataDefinition *modeldatum`, `std::string &errorMessage`)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` (`PersistenceRecord *fields`) override
must be overridden by derived classes
- virtual void `_saveInstance` (`PersistenceRecord *fields`, bool `saveDefaultValues`) override
Saves object fields into a serialization structure.
- virtual bool `_check` (`std::string &errorMessage`) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.43.1 Detailed Description

[Attribute](#) module DESCRIPTION This data module is used to define an attribute's dimension, data type and initial value(s). An attribute is a characteristic of all entities created, but with a specific value that can differ from one entity to another. Attributes can be referenced in other modules (for example, the [Decide](#) module), can be reassigned a new value with the [Assign](#) module, and can be used in any expression. [Attribute](#) values are unique for each entity, as compared to Variables which are global to the simulation module. There are three methods for manually editing the Initial Values of an [Attribute](#) module:

- Using the standard spreadsheet interface. In the module spreadsheet, right-click on the Initial Values cell and select the Edit via spreadsheet menu item. The values for two-dimensional arrays should be entered one column at a time. Array elements not explicitly assigned are assumed to have the last entered value.
- Using the module dialog box. In the module spreadsheet, right-click on any cell and select the Edit via dialog menu item. The values for two-dimensional arrays should be entered one column at a time. Array elements not explicitly assigned are assumed to have the last entered value.
- Using the two-dimensional (2-D) spreadsheet interface. In the module spreadsheet, click on the Initial Values cell. TYPICAL USES Due date of an order (entity) Priority of an order (entity) Color of a part (entity) PROMPTS Prompt Description Name The unique name of the attribute being defined. Rows Number of rows in a one- or two-dimensional attribute. Columns Number of columns in a two-dimensional attribute. Data Type The data type of the values stored in the attribute. Valid types are Real and String. The default type is Real. Initial Values Lists the initial value or values of the attribute. You can assign new values to the attribute by using the [Assign](#) module. Initial Value [Entity](#) attribute value when entity is created and enters the system.

15.43.2 Member Function Documentation

15.43.2.1 `_check()`

```
bool Attribute::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [Variable](#).

15.43.2.2 `_loadInstance()`

```
bool Attribute::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [Variable](#).

15.43.2.3 _saveInstance()

```
void Attribute::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [Variable](#).

15.43.2.4 getInitialValuesText()

```
std::string Attribute::getInitialValuesText ( ) const [virtual]
```

Returns the initial values serialized in bracket notation.

Reimplemented in [Variable](#).

15.43.2.5 setInitialValuesText()

```
void Attribute::setInitialValuesText (
    std::string valuesText ) [virtual]
```

Parses bracket notation and replaces the initial value store.

Reimplemented in [Variable](#).

15.43.2.6 show()

```
std::string Attribute::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [Variable](#).

The documentation for this class was generated from the following files:

- kernel/simulator/essentialPlugins/Attribute.h
- kernel/simulator/essentialPlugins/Attribute.cpp

15.44 AuditEntry Struct Reference

One entry in the AI assistant audit log.

```
#include <tools/AIAssistant/AIAuditLog.h>
```

Public Attributes

- std::chrono::system_clock::time_point **timestamp**
- std::string **operation**
e.g. "analyzePrompt", "buildModel"
- std::string **provider**
e.g. "OpenAI", "Anthropic", "Local"
- std::string **modelName**

LLM model identifier.

- `std::string` **promptPreview**

First 120 chars of user-visible input — no keys.

- `bool` **success** = false
- `bool` **dryRun** = false
- `long long` **durationMs** = 0
- `std::string` **errorSummary**

Empty on success; capped at 200 chars.

15.44.1 Detailed Description

One entry in the AI assistant audit log.

Never contains API key values. The `promptPreview` field is limited to the first 120 characters of the user-visible prompt text.

The documentation for this struct was generated from the following file:

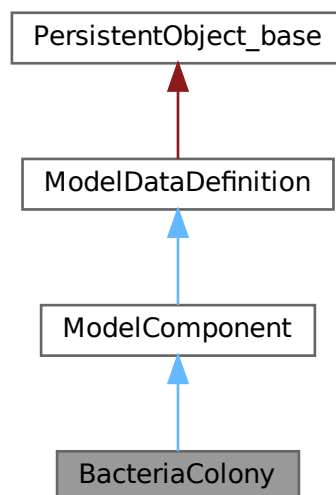
- `tools/AIAssistant/AIAuditLog.h`

15.45 BacteriaColony Class Reference

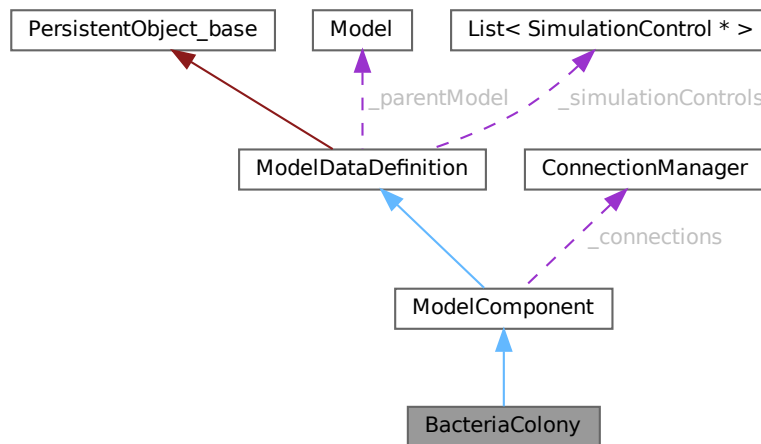
First GenESyS component for Gro-inspired bacteria colony simulation.

```
#include <plugins/components/BiochemicalSimulation/BacteriaColony.h>
```

Inheritance diagram for `BacteriaColony`:



Collaboration diagram for BacteriaColony:



Classes

- struct [BacteriumState](#)
- struct [BarrierSegment](#)

Public Member Functions

- **BacteriaColony** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
virtual public methods
- void **setGroProgram** ([GroProgram](#) *groProgram)
Sets the reusable Gro program used by this colony.
- [GroProgram](#) * **getGroProgram** () const
Returns the reusable Gro program used by this colony.
- void **setSimulationStep** (double simulationStep)
Sets the internal colony time increment used by one colony step.
- double **getSimulationStep** () const
Returns the internal colony time increment.
- void **setNumSteps** (unsigned int numSteps)
Sets the number of colony steps processed before forwarding one entity.
- unsigned int **getNumSteps** () const
Returns the number of colony steps processed before forwarding one entity.
- void **setColonyTimeUnit** ([Util::TimeUnit](#) colonyTimeUnit)
Sets the time unit used by the colony time-related attributes.
- [Util::TimeUnit](#) **getColonyTimeUnit** () const
Returns the time unit used by the colony time-related attributes.
- double **getColonyTime** () const
Returns the current colony time from the enclosing [ModelSimulation](#).
- void **setInitialPopulation** (unsigned int initialPopulation)
Sets the initial bacteria population summary for each replication.
- unsigned int **getInitialPopulation** () const
Returns the initial bacteria population summary.

- unsigned int **getPopulationSize** () const
Returns the current bacteria population summary.
- void **setBioNetwork** (BioNetwork *bioNetwork)
Sets the optional BioNetwork reused as the colony biochemical state.
- BioNetwork * **getBioNetwork** () const
Returns the optional BioNetwork reused as the colony biochemical state.
- void **setSignalGrid** (BacteriaSignalGrid *signalGrid)
Sets the reusable signal-grid definition used by this colony.
- BacteriaSignalGrid * **getSignalGrid** () const
Returns the reusable signal-grid definition used by this colony.
- std::size_t **getInternalBacteriaCount** () const
Returns the number of internal bacteria currently owned by the colony.
- const BacteriumState & **getBacteriumState** (std::size_t index) const
Returns one internal bacterium state by zero-based index.
- double **getBacteriumAge** (std::size_t index) const
Returns the age of one internal bacterium at the current colony time.
- double **getBacteriumPositionX** (std::size_t index) const
Returns the continuous X coordinate used by the viewer.
- double **getBacteriumPositionY** (std::size_t index) const
Returns the continuous Y coordinate used by the viewer.
- double **getBacteriumDirectionRadians** (std::size_t index) const
Returns the bacterium direction in radians.
- double **getBacteriumVolume** (std::size_t index) const
Returns the current bacterium volume.
- double **getBacteriumSize** (std::size_t index) const
Returns the current bacterium size used for rendering.
- double **getBacteriumGfp** (std::size_t index) const
Returns the current GFP amount used for rendering.
- double **getBacteriumRfp** (std::size_t index) const
Returns the current RFP amount used for rendering.
- double **getBacteriumRuntimeVariableValue** (std::size_t index, const std::string &variableName) const
Returns the last stored value for one bacterium-scoped runtime variable.
- bool **hasBacteriumRuntimeVariable** (std::size_t index, const std::string &variableName) const
Tells whether one bacterium currently stores one runtime scalar variable.
- double **getRuntimeVariableValue** (const std::string &variableName) const
Returns the last stored value for one runtime scalar variable.
- bool **hasRuntimeVariable** (const std::string &variableName) const
Tells whether the colony currently stores one runtime scalar variable.
- double **getSignalValueAt** (unsigned int x, unsigned int y) const
Returns the runtime signal value stored at one grid coordinate.
- std::vector< std::vector< double > > **getSignalMatrix** () const
Returns the current runtime signal field as a row-major matrix.
- std::string **getSignalMatrixDump** (std::size_t maxRows=0, std::size_t maxColumns=0) const
Returns a textual dump of the current runtime signal field.
- double **getBacteriumLocalSignal** (std::size_t index) const
Returns the runtime signal value seen by one bacterium at its current cell.
- void **setChemostatMode** (bool enabled)
Enables or disables the colony chemostat mode used by Gro microfluidics.
- bool **getChemostatMode** () const
Returns whether the colony chemostat mode is enabled.
- void **addBarrier** (double x1, double y1, double x2, double y2)

- Adds one barrier segment to the colony environment.*
- `const std::vector< BarrierSegment > & getBarriers () const`
Returns the current barrier segments used by the viewer.
- `const std::vector< double > & getMappedCellValues () const`
Returns the latest per-cell values produced by map_to_cells, if any.
- `const std::string & getMappedCellExpression () const`
Returns the expression used by the latest map_to_cells call, if any.
- `void setGridWidth (unsigned int gridWidth)`
Sets the discrete spatial grid width reserved for the first colony model.
- `unsigned int getGridWidth () const`
Returns the discrete spatial grid width.
- `void setGridHeight (unsigned int gridHeight)`
Sets the discrete spatial grid height reserved for the first colony model.
- `unsigned int getGridHeight () const`
Returns the discrete spatial grid height.
- `GroProgramRuntime::ExecutionResult executeGroProgram ()`
Executes the configured Gro program once against the current colony state.

Public Member Functions inherited from [ModelComponent](#)

- `ModelComponent (Model *model, std::string componentTypename, std::string name="")`
constructors
- `virtual ~ModelComponent ()`
Releases component-owned resources and unregisters it from the model.
- `ConnectionManager * getConnectionManager () const`
new public user methods for this component
- `void connectTo (ModelComponent *component, unsigned int inputPortNumber=0)`
Connects this component to another component.
- `bool hasBreakpointAt ()`
Indicates whether a simulation breakpoint is set on this component.
- `void setDescription (std::string _description)`
Updates the free-form description shown by the GUI.
- `std::string getDescription () const`
Returns the free-form description associated with this component.
- `void remove (ModelComponent *comp)`
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- `ModelDataDefinition (Model *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)`
Creates a model data definition and optionally inserts it into the model.
- `virtual ~ModelDataDefinition ()`
Releases resources owned by this data definition.
- `Util::identification getId () const`
Returns the unique identifier of this instance.
- `void setName (const std::string &name)`
Updates the logical name of this instance.
- `const std::string & getName () const`
Returns the logical name of this instance.
- `void setLabel (const std::string &label)`
Updates the user-visible label of this instance.

- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelspecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)

- Creates internal data required by a component instance.*
 - static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
 - Serializes a component instance into a persistence record.*
 - static void [DispatchEvent](#) ([Event](#) *event)
 - Dispatches a simulation event to the target component.*

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
 - Loads a model data definition instance from serialized fields.*
 - static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
 - Creates a new instance of a data definition using the plugin registry.*
 - static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
 - Serializes a data definition instance into a persistence record.*
 - static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
 - Validates a data definition instance.*
 - static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
 - Resets simulation-time state between replications.*
 - static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
 - Creates internal and attached data required by this instance.*
 - static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
 - Creates related data and validates the instance in a single call.*

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
 - virtual protected method that must be overridden*
 - virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
 - Saves object fields into a serialization structure.*
 - virtual bool [_check](#) (std::string &errorMessage) override
 - could be overridden by derived classes*
 - virtual void [_initBetweenReplications](#) () override
 - virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
 - Handles a dispatched event for this component type.*
 - virtual void [_createAttachedAttributes](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
 - virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
 - virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
 - virtual void [_createInternalAndAttachedData](#) ()
 - virtual void [_createInternalStatisticReporters](#) ()
 - virtual void [_createNonEditableDataDefinitions](#) ()
 - virtual void [_createEditableDataDefinitions](#) ()
 - void [_templateCreateInternalStatisticReporters](#) ()
 - void [_templateCreateAttachedAttributes](#) ()
 - void [_templateCreateNonEditableDataDefinitions](#) ()
 - void [_templateCreateEditableDataDefinitions](#) ()
 - void [_internalStatisticReportersClear](#) ()
 - legacy helper methods used by the granular create* hooks*
 - void [_internalDataClear](#) ()
 - void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)

- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
 - virtual protected methods that could be overridden by derived classes, if needed*

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.45.1 Detailed Description

First GenESyS component for Gro-inspired bacteria colony simulation.

[BacteriaColony](#) owns bacteria and signal-field runtime state while using the enclosing [ModelSimulation](#) time as the single colony clock. The component references a reusable [GroProgram](#) and executes the first supported runtime commands through plugin-side parser/runtime helpers.

15.45.2 Member Function Documentation

15.45.2.1 `_check()`

```
bool BacteriaColony::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.45.2.2 `_createAttachedAttributes()`

```
void BacteriaColony::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.45.2.3 `_initBetweenReplications()`

```
void BacteriaColony::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.45.2.4 `_loadInstance()`

```
bool BacteriaColony::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.45.2.5 `_onDispatchEvent()`

```
void BacteriaColony::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.45.2.6 `_saveInstance()`

```
void BacteriaColony::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.45.2.7 show()

```
std::string BacteriaColony::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

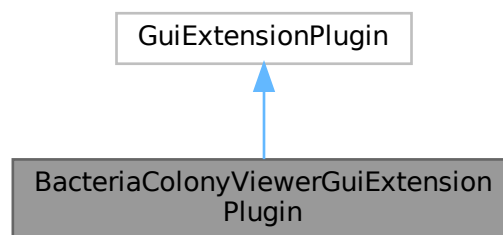
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

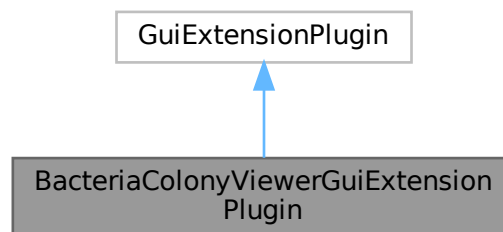
- plugins/components/BiochemicalSimulation/BacteriaColony.h
- plugins/components/BiochemicalSimulation/BacteriaColony.cpp

15.46 BacteriaColonyViewerGuiExtensionPlugin Class Reference

Inheritance diagram for BacteriaColonyViewerGuiExtensionPlugin:



Collaboration diagram for BacteriaColonyViewerGuiExtensionPlugin:



Public Member Functions

- std::string [extensionId](#) () const override
- std::string [displayName](#) () const override

- `std::string version ()` const override
- `std::vector< std::string > requiredModelPlugins ()` const override
- `void registerContributions (GuiExtensionRegistry *registry)` const override

15.46.1 Member Function Documentation

15.46.1.1 displayName()

```
std::string BacteriaColonyViewerGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.46.1.2 extensionId()

```
std::string BacteriaColonyViewerGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.46.1.3 registerContributions()

```
void BacteriaColonyViewerGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.46.1.4 requiredModelPlugins()

```
std::vector< std::string > BacteriaColonyViewerGuiExtensionPlugin::requiredModelPlugins ( )
const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.46.1.5 version()

```
std::string BacteriaColonyViewerGuiExtensionPlugin::version ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

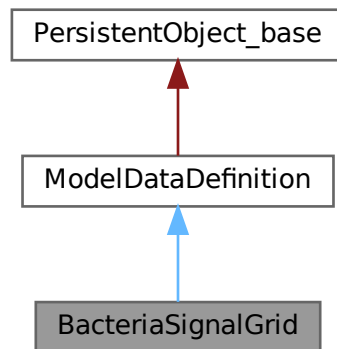
- `applications/gui/genesys/extensions/BacteriaColonyViewerGuiExtensionPlugin.cpp`

15.47 BacteriaSignalGrid Class Reference

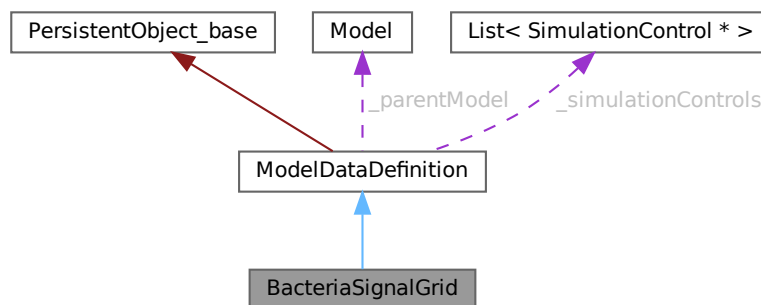
Reusable discrete signal-field definition for bacteria colonies.

```
#include <plugins/data/BiochemicalSimulation/BacteriaSignalGrid.h>
```

Inheritance diagram for BacteriaSignalGrid:



Collaboration diagram for BacteriaSignalGrid:



Public Member Functions

- **BacteriaSignalGrid** (**Model** *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- void **setWidth** (unsigned int width)
- unsigned int **getWidth** () const
- void **setHeight** (unsigned int height)
- unsigned int **getHeight** () const
- void **setInitialSignal** (double initialSignal)
- double **getInitialSignal** () const
- void **setDiffusionRate** (double diffusionRate)
- double **getDiffusionRate** () const
- void **setDecayRate** (double decayRate)
- double **getDecayRate** () const
- void **setInitialValues** (std::string initialValues)
- std::string **getInitialValues** () const
- unsigned int **getCellCount** () const
- bool **buildInitialField** (std::vector< double > &values, std::string &errorMessage) const

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool **hasChanged**)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- **List**< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool **_getSaveDefaultsOption** ()
- virtual [ParserChangesInformation](#) * **_getParserChangesInformation** ()
- virtual void **_initBetweenReplications** ()
- virtual void **_addSimulationControl** ([SimulationControl](#) *control)
- virtual void **_createInternalAndAttachedData** ()
- virtual void **_createInternalStatisticReporters** ()
- virtual void **_createAttachedAttributes** ()
- virtual void **_createNonEditableDataDefinitions** ()
- virtual void **_createEditableDataDefinitions** ()
- void **_templateCreateInternalStatisticReporters** ()
- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
legacy helper methods used by the granular create hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)

- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.47.1 Detailed Description

Reusable discrete signal-field definition for bacteria colonies.

The data definition persists the spatial configuration and initial scalar values of one signal field. Colonies copy this configuration into their runtime state so model configuration and simulation mutation stay separated.

15.47.2 Member Function Documentation

15.47.2.1 `_check()`

```
bool BacteriaSignalGrid::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.47.2.2 `_loadInstance()`

```
bool BacteriaSignalGrid::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.47.2.3 `_saveInstance()`

```
void BacteriaSignalGrid::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.47.2.4 `show()`

```
std::string BacteriaSignalGrid::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- plugins/data/BiochemicalSimulation/BacteriaSignalGrid.h
- plugins/data/BiochemicalSimulation/BacteriaSignalGrid.cpp

15.48 BacteriaColony::BacteriumState Struct Reference

Public Attributes

- unsigned int **id** = 0
- unsigned int **parentId** = 0
- unsigned int **generation** = 0
- std::string **programName** = ""
- std::vector< double > **programArguments**
- unsigned int **divisionCount** = 0
- double **birthTime** = 0.0
- double **lastUpdateTime** = 0.0
- double **lastDivisionTime** = 0.0

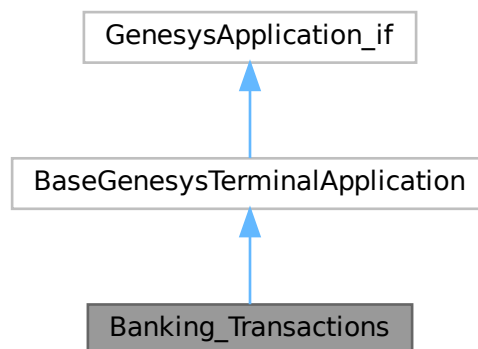
- double **positionX** = 0.0
- double **positionY** = 0.0
- double **directionRadians** = 0.0
- double **volume** = 1.0
- double **size** = 1.0
- double **gfp** = 0.0
- double **rfp** = 0.0
- double **yfp** = 0.0
- double **cfp** = 0.0
- double **speed** = 0.1
- unsigned int **tickCount** = 0
- unsigned int **gridX** = 0
- unsigned int **gridY** = 0
- bool **hasExplicitGridPosition** = false
- bool **justDivided** = false
- bool **daughter** = false
- bool **alive** = true
- std::map< std::string, double > **runtimeVariables**

The documentation for this struct was generated from the following file:

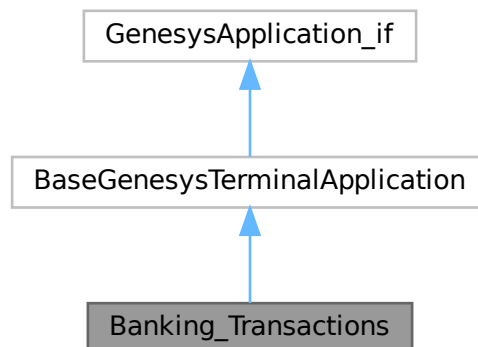
- plugins/components/BiochemicalSimulation/BacteriaColony.h

15.49 Banking_Transactions Class Reference

Inheritance diagram for Banking_Transactions:



Collaboration diagram for Banking_Transactions:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

15.49.1 Member Function Documentation

15.49.1.1 main()

```
int Banking_Transactions::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/arenaExamples/Banking_Transactions.h
- applications/modelSpecific/arenaExamples/Banking_Transactions.cpp

15.50 BacteriaColony::BarrierSegment Struct Reference

Public Attributes

- double **x1** = 0.0
- double **y1** = 0.0
- double **x2** = 0.0
- double **y2** = 0.0

The documentation for this struct was generated from the following file:

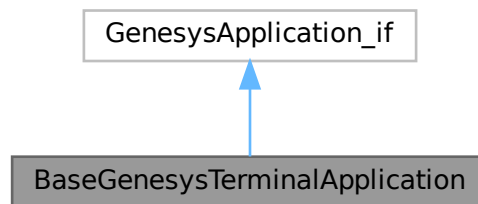
- plugins/components/BiochemicalSimulation/BacteriaColony.h

15.51 BaseGenesysTerminalApplication Class Reference

Inherits [GenesysApplication_if](#).

Inherited by [AirportSecurityExample](#), [AirportSecurityExampleExtended](#), [Airport_Extended1](#), [AnElectronicAssemblyAndTestSystem](#), [Assembly_Line](#), [Banking_Transactions](#), [Book_Cap02_Example01](#), [BufferFIFO](#), [ContinuousModel](#), [Example_BasicOrderShipping](#), [Example_PortModel](#), [Example_PublicTransport](#), [FullSimulationOfComplexModel](#), [GenesysShell](#), [Half_Adder](#), [Loja01](#), [OperatingSystem02](#), [OperatingSystem03](#), [Rectifier](#), [Smart_AddingResource](#), [Smart_AlternatingEntityCreation](#), [Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime](#), [Smart_ArrivalsEntityTypeVsAttribute](#), [Smart_AssignExample](#), [Smart_AssignWriteSeizes](#), [Smart_Attribute_Variable](#), [Smart_AutomaticStatisticsCollection](#), [Smart_BacteriaColony_GRO](#), [Smart_BasicModeling](#), [Smart_BatchAndSeparate](#), [Smart_BatchSeparate](#), [Smart_BioKineticLawRegulation](#), [Smart_BioNetworkAnalysis](#), [Smart_BioReversibleKineticLaw](#), [Smart_BioReversibleMassAction](#), [Smart_Buffer](#), [Smart_CellularAutomata](#), [Smart_CellularAutomata1](#), [Smart_Clone](#), [Smart_ContinuousFlowEntities](#), [Smart_CppForG](#), [Smart_Create](#), [Smart_DecideNWayByChance](#), [Smart_DefaultModalModel](#), [Smart_DefiningAttributesAsStrings](#), [Smart_DefiningControlLogic](#), [Smart_DefiningResourceCapacity](#), [Smart_Delay](#), [Smart_DelayBasedOnReplication](#), [Smart_Dummy](#), [Smart_EntitiesProcessedByPriority](#), [Smart_EvaluatingConditionsBeforeEnteringQueue](#), [Smart_Expression](#), [Smart_FSMState](#), [Smart_Failures](#), [Smart_GroColonyGrowth](#), [Smart_GroColonyLifecycle](#), [Smart_HoldSearchRemove](#), [Smart_InventoryAndHoldingCosts](#), [Smart_LSIDE](#), [Smart_MarkovChain](#), [Smart_MaxArrivalsField](#), [Smart_ModalModel](#), [Smart_ModalModelDefaultNetwork](#), [Smart_ModalModelFSM](#), [Smart_ModalModelPetriNet](#), [Smart_ModelInfoModelSimulation](#), [Smart_ModelRunUntil1000Parts](#), [Smart_ModuleDisplayVariables](#), [Smart_ODE](#), [Smart_OnEvent](#), [Smart_OverlappingResources](#), [Smart_ParallelProcessingOfEntities](#), [Smart_Parser](#), [Smart_ParserModelFunctions](#), [Smart_PetriPlace](#), [Smart_PlacingEntitiesInQueue](#), [Smart_Plugin](#), [Smart_PriorityExample](#), [Smart_Process](#), [Smart_ProcessArena](#), [Smart_ProcessSet](#), [Smart_PythonForG](#), [Smart_R_Simulator](#), [Smart_Record](#), [Smart_Record_Arena](#), [Smart_RemovingAndReorderingEntitiesInAQueue](#), [Smart_ResourceCosting](#), [Smart_ResourceScheduleCosting](#), [Smart_ResourceSets](#), [Smart_RouteStation](#), [Smart_SeizeDelayRelease](#), [Smart_SeizeDelayReleaseMany](#), [Smart_SeizeDelayReleaseNoDataDefs](#), [Smart_SeizingMultipleSimulta](#), [Smart_SelectingRouteBasedOnProbability](#), [Smart_SelectingShorterQueue](#), [Smart_Sequence](#), [Smart_SimulationControlResponse](#), [Smart_SynchronizingParallelEntities](#), [Smart_ValueAdded](#), [Smart_WaitForSignal](#), [Smart_WaitScanCondition](#), [Smart_WaitSignal](#), [StartAIAssistant](#), [WcmBioNetworkBridge](#), [WcmCompartmentTransportWholeCellMvp](#), [WcmFluxBalanceWholeCellMvp](#), [WcmGeneExpression](#), [WcmLifecycleStress](#), [WcmMgenitaliumKarr2012](#), [WcmPathwayStressWholeCellMvp](#), [WcmSimpleSsa](#), [WcmWholeCellMvp](#), [WcmYeastDidacticWholeCellMvp](#), and [WcmYeastEukaryoticWholeCellMvp](#).

Collaboration diagram for BaseGenesysTerminalApplication:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override
- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Protected Member Functions

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

15.51.1 Member Function Documentation

15.51.1.1 main()

```
int BaseGenesysTerminalApplication::main (
    int argc,
    char ** argv ) [override], [virtual]
```

Implements [GenesysApplication_if](#).

Reimplemented in [Airport_Extended1](#), [Assembly_Line](#), [Banking_Transactions](#), [Example_BasicOrderShipping](#), [Example_PortModel](#), [Example_PublicTransport](#), [Smart_AddingResource](#), [Smart_AlternatingEntityCreation](#), [Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime](#), [Smart_AutomaticStatisticsCollection](#), [Smart_ContinuousFlowEntities](#), [Smart_Create](#), [Smart_DecideNWayByChance](#), [Smart_DefiningAttributesAsStrings](#), [Smart_DefiningControlLogic](#), [Smart_DefiningResourceCapacity](#), [Smart_EntitiesProcessedByPriority](#), [Smart_EvaluatingConditionsBeforeEnteringQueue](#), [Smart_Expression](#), [Smart_MaxArrivalsField](#), [Smart_ModelRunUntil1000Parts](#), [Smart_ModuleDisplayVariables](#),

Smart_OverlappingResources, Smart_ParallelProcessingOfEntities, Smart_PlacingEntitiesInQueueSets, Smart_PriorityExample, Smart_ProcessArena, Smart_Record_Arena, Smart_RemovingAndReorderingEntitiesInAQueue, Smart_ResourceSets, Smart_SeizingMultipleSimultaneously, Smart_SelectingRouteBasedOnProbability, Smart_SelectingShorterQueue, Smart_SynchronizingParallelEntities, Smart_WaitForSignal, Smart_MarkovChain, Smart_Buffer, Smart_Clone, Smart_Delay, Smart_Failures, Smart_HoldSearchRemove, Smart_Process, Smart_ProcessSet, Smart_SeizeDelayRelease, Smart_SeizeDelayReleaseMany, Smart_SeizeDelayReleaseNoDataDefs, Smart_R_Simulator, Smart_Record, Smart_Attribute_Variable, Smart_ModelInfoModelSimulation, Smart_Parser, Smart_ParserModelFunctions, Smart_RouteStation, Smart_CellularAutomata, Smart_CellularAutomata1, Smart_DefaultModalModel, Smart_Dummy, Smart_ModalModel, Smart_OnEvent, Smart_WaitScanCondition, Smart_WaitSignal, FullSimulationOfComplexModel, Half_Adder, and Rectifier.

15.51.1.2 setDefaultEventHandlers()

```
void BaseGenesysTerminalApplication::setDefaultEventHandlers (
    OnEventManager * oem )
```

Add default method to handle simulator events so information is plotted to output screen
The documentation for this class was generated from the following files:

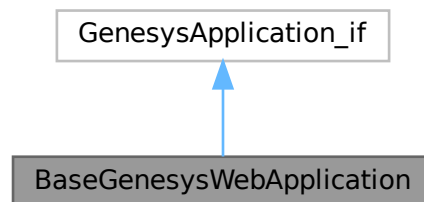
- applications/BaseGenesysTerminalApplication.h
- applications/BaseGenesysTerminalApplication.cpp

15.52 BaseGenesysWebApplication Class Reference

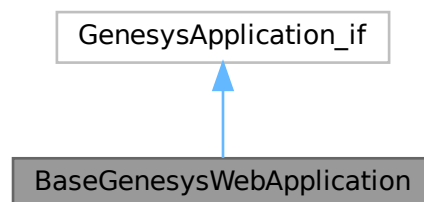
Entry point for the standalone Worker application flavor of Genesys.

```
#include <applications/worker/BaseGenesysWebApplication.h>
```

Inheritance diagram for BaseGenesysWebApplication:



Collaboration diagram for BaseGenesysWebApplication:



Public Member Functions

- `int main (int argc, char **argv)` override
Parses command-line arguments, starts the HTTP server and runs the worker app.

15.52.1 Detailed Description

Entry point for the standalone Worker application flavor of Genesys.

The worker variant bootstraps a session manager, HTTP server and router, then keeps processing requests until the configured request limit is reached or a transport error occurs.

15.52.2 Member Function Documentation

15.52.2.1 main()

```
int BaseGenesysWebApplication::main (
    int argc,
    char ** argv ) [override], [virtual]
```

Parses command-line arguments, starts the HTTP server and runs the worker app.

Boots the web stack and keeps the request loop alive until shutdown.

Parameters

<i>argc</i>	Number of command-line arguments.
<i>argv</i>	Command-line argument vector.

Returns

Zero on normal termination; non-zero on startup or server failure.

This entry point wires token management, session creation, worker-job storage, API routing and the embedded HTTP server into one runtime pipeline.

Implements [GenesysApplication_if](#).

The documentation for this class was generated from the following files:

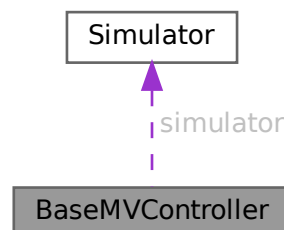
- applications/worker/BaseGenesysWebApplication.h
- applications/worker/BaseGenesysWebApplication.cpp

15.53 BaseMVController Class Reference

Compatibility base class that binds the Qt UI and the simulator model.

```
#include <applications/gui/genesys/BaseMVController.h>
```

Collaboration diagram for BaseMVController:



Protected Attributes

- Ui::MainWindow * **ui**
MainWindow UI built from the Qt Designer form.
- Simulator * **simulator**
Active simulator instance owned elsewhere by the application.

15.53.1 Detailed Description

Compatibility base class that binds the Qt UI and the simulator model.

Legacy GUI controllers in Genesys share the same two core pointers: the main window UI object and the active simulator instance. This base class keeps that pairing in one place so higher-level controllers can reuse it.

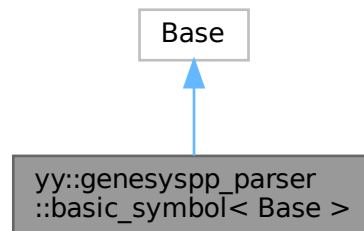
The documentation for this class was generated from the following files:

- applications/gui/genesys/BaseMVController.h
- applications/gui/genesys/BaseMVController.cpp

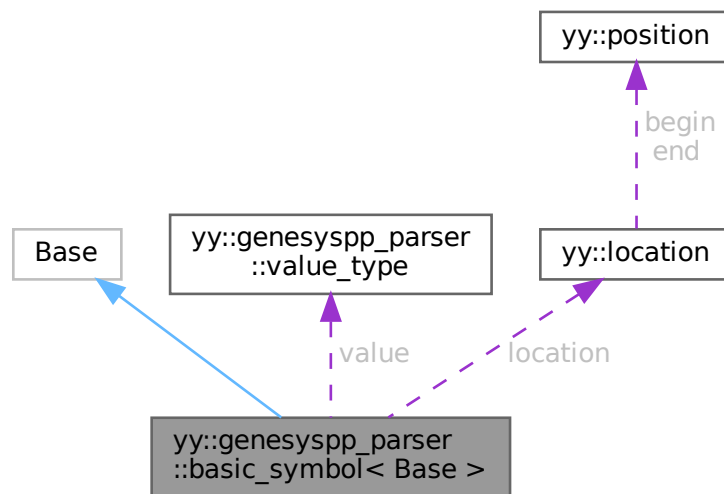
15.54 yy::genesyspp_parser::basic_symbol< Base > Struct Template Reference

```
#include <parser/GenesysParser.h>
```

Inheritance diagram for yy::genesyspp_parser::basic_symbol< Base >:



Collaboration diagram for `yy::genesyspp_parser::basic_symbol< Base >`:



Public Types

- typedef `Base` **super_type**

Alias to Base.

Public Member Functions

- **basic_symbol** () YY_NOEXCEPT
Default constructor.
- **basic_symbol** (const [basic_symbol](#) &that)
Copy constructor.
- **basic_symbol** (typename `Base::kind_type` t, const [location_type](#) &l)
Constructors for typed symbols.
- **basic_symbol** (typename `Base::kind_type` t, const [obj_t](#) &v, const [location_type](#) &l)
- **basic_symbol** (typename `Base::kind_type` t, const `std::string` &v, const [location_type](#) &l)
- **~basic_symbol** ()
Destroy the symbol.
- void **clear** () YY_NOEXCEPT
Destroy contents, and record that is empty.
- `std::string` **name** () const YY_NOEXCEPT
The user-facing name of this symbol.
- [symbol_kind_type](#) **type_get** () const YY_NOEXCEPT
Backward compatibility (Bison 3.6).
- bool **empty** () const YY_NOEXCEPT
Whether empty.
- void **move** ([basic_symbol](#) &s)
Destructive move, s is emptied into this.

Public Attributes

- [value_type](#) **value**
The semantic value.
- [location_type](#) **location**
The location.

15.54.1 Detailed Description

`template<typename Base>`

`struct yy::genesyspp_parser::basic_symbol< Base >`

A complete symbol.

Expects its Base type to provide access to the symbol kind via `kind ()`.

Provide access to semantic value and location.

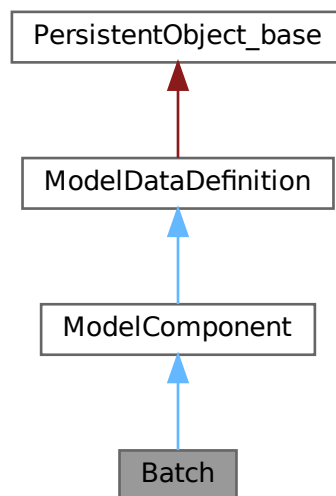
The documentation for this struct was generated from the following file:

- `parser/GenesysParser.h`

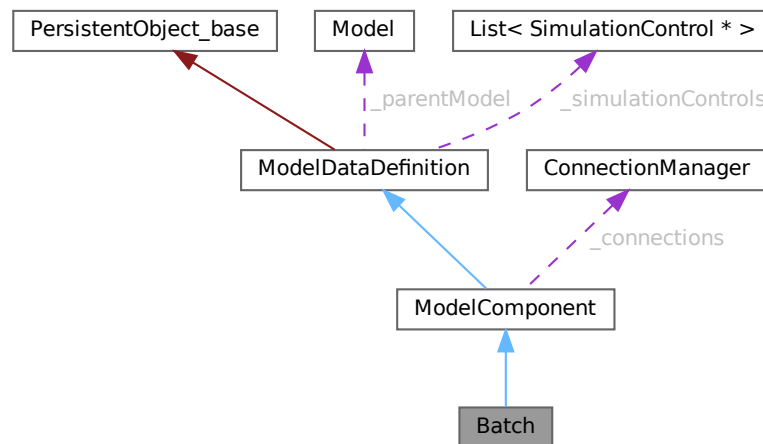
15.55 Batch Class Reference

```
#include <plugins/components/Grouping/Batch.h>
```

Inheritance diagram for Batch:



Collaboration diagram for Batch:



Public Types

- enum class **BatchType** : int { **Temporary** = 0 , **Permanent** = 1 , **num_elements** = 2 }
- enum class **Rule** : int { **Any** = 0 , **ByAttribute** = 1 , **num_elements** = 2 }
- enum class **GroupedAttribs** : int { **FirstEntity** = 0 , **LastEntity** = 1 , **SumAttributes** = 2 , **num_elements** = 3 }

Public Member Functions

- **Batch** (**Model** *model, std::string name="")
- virtual std::string **show** () override
- *virtual public methods*
- void **setGroupedEntityType** (**EntityType** *groupedEntityType)
- void **setGroupedEntityTypeName** (std::string groupedEntityTypeName)
- **EntityType** * **getGroupedEntityType** () const
- void **setBatchType** (Batch::BatchType batchType)
- Batch::BatchType **getBatchType** () const
- void **setAttributeName** (std::string attributeName)
- std::string **getAttributeName** () const
- void **setBatchSize** (std::string batchSize)
- std::string **getBatchSize** () const
- void **setRule** (Batch::Rule _rule)
- Batch::Rule **getRule** () const
- void **setGroupedAttributes** (Batch::GroupedAttribs _groupedAttributes)
- Batch::GroupedAttribs **getGroupedAttributes** () const

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
- *constructors*
- virtual ~**ModelComponent** ()
- *Releases component-owned resources and unregisters it from the model.*
- **ConnectionManager** * **getConnectionManager** () const

- new public user methods for this component*
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool **hasChanged**)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.

- [List< SimulationControl * > * getSimulationControls \(\)](#) const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level getTraceLevelSpecific \(\)](#) const
Returns the trace level that applies specifically to this definition.
- [void setTraceLevelSpecific \(TraceManager::Level level\)](#)
Updates the trace level used by this definition.
- [void defineTraceLevelSpecific \(TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true\)](#)
Defines the specific trace level and optionally enables it immediately.
- [bool isTraceLevelSpecificEnabled \(\)](#) const
Indicates whether the specific trace level is enabled.
- [void setTraceLevelSpecificEnabled \(bool traceLevelSpecificEnabled\)](#)
Enables or disables the specific trace level.

Static Public Member Functions

- [static std::string convertEnumToStr \(BatchType type\)](#)
- [static std::string convertEnumToStr \(Rule rule\)](#)
- [static std::string convertEnumToStr \(GroupedAttribs attribs\)](#)
- [static PluginInformation * GetPluginInformation \(\)](#)
- [static ModelComponent * LoadInstance \(Model *model, PersistenceRecord *fields\)](#)
- [static ModelDataDefinition * NewInstance \(Model *model, std::string name=""\)](#)

Static Public Member Functions inherited from [ModelComponent](#)

- [static ModelComponent * LoadInstance \(Model *model, PersistenceRecord *fields\)](#)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- [static bool Check \(ModelComponent *component\)](#)
Validates a component instance.
- [static void CreateInternalData \(ModelComponent *component\)](#)
Creates internal data required by a component instance.
- [static void SaveInstance \(PersistenceRecord *fields, ModelComponent *component\)](#)
Serializes a component instance into a persistence record.
- [static void DispatchEvent \(Event *event\)](#)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- [static ModelDataDefinition * LoadInstance \(Model *model, PersistenceRecord *fields, bool insertIntoModel\)](#)
Loads a model data definition instance from serialized fields.
- [static ModelDataDefinition * NewInstance \(Model *model, std::string name=""\)](#)
Creates a new instance of a data definition using the plugin registry.
- [static void SaveInstance \(PersistenceRecord *fields, ModelDataDefinition *modeldatum\)](#)
Serializes a data definition instance into a persistence record.
- [static bool Check \(ModelDataDefinition *modeldatum, std::string &errorMessage\)](#)
Validates a data definition instance.
- [static void InitBetweenReplications \(ModelDataDefinition *modeldatum\)](#)
Resets simulation-time state between replications.
- [static void CreateInternalData \(ModelDataDefinition *modeldatum\)](#)
Creates internal and attached data required by this instance.
- [static bool CreateRelatedDataElements \(ModelDataDefinition *modeldatum, std::string &errorMessage\)](#)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void [_createNonEditableDataDefinitions](#) () override
- virtual void [_createEditableDataDefinitions](#) () override
- virtual void [_createAttachedAttributes](#) () override
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_onDispatchEvent](#) (Entity *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) (PersistenceRecord *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) (PersistenceRecord *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual ParserChangesInformation * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) (SimulationControl *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.55.1 Detailed Description

[Batch](#) module DESCRIPTION This module is intended as the grouping mechanism within the simulation model. Batches can be permanently or temporarily grouped. Temporary batches must later be split using the [Separate](#) module. Batches may be made with any specified number of entering entities or may be matched together based on an attribute. Entities arriving at the [Batch](#) module are placed in a queue until the required number of entities has accumulated. Once accumulated, a new representative entity is created. TYPICAL USES Collect a number of parts before starting processing Reassemble previously separated copies of a form Bring together a patient and his record before commencing an appointment PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Type Method of batching entities together. [Batch](#) Size Number of entities to be batched. Save Criterion Method for assigning representative entity's user-defined attribute values. Rule Determines how incoming entities will be batched. Any [Entity](#) will take the first "Batch Size" number of entities and put them together. By [Attribute](#) signifies that the values of the specified attribute must match for entities to be grouped. For example, if [Attribute](#) Name is Color, all entities must have the same Color value to be grouped; otherwise, they will wait at the module for additional incoming entities. [Attribute](#) Name Name of the attribute whose value must match the value of the other incoming entities in order for a group to be made. Applies only when Rule is By [Attribute](#). Representative [Entity](#) The entity type for the representative entity.

15.55.2 Member Function Documentation

15.55.2.1 [_check\(\)](#)

```
bool Batch::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.55.2.2 `_createAttachedAttributes()`

```
void Batch::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.55.2.3 `_createEditableDataDefinitions()`

```
void Batch::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.55.2.4 `_createNonEditableDataDefinitions()`

```
void Batch::_createNonEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.55.2.5 `_loadInstance()`

```
bool Batch::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [ModelComponent](#).

15.55.2.6 `_onDispatchEvent()`

```
void Batch::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.55.2.7 `_saveInstance()`

```
void Batch::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.55.2.8 `show()`

```
std::string Batch::show ( ) [override], [virtual]
```

virtual public methods
Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- `plugins/components/Grouping/Batch.h`
- `plugins/components/Grouping/Batch.cpp`

15.56 `testing::internal::TemplateSel< Tmpl >::Bind< T > Struct` Template Reference

Public Types

- `typedef Tmpl< T > type`

The documentation for this struct was generated from the following file:

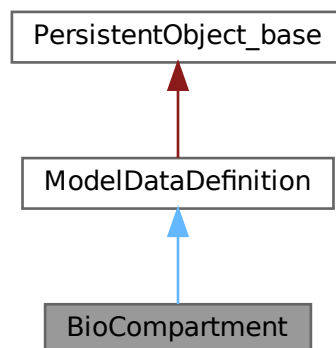
- `gtest/gtest/internal/gtest-type-util.h`

15.57 BioCompartment Class Reference

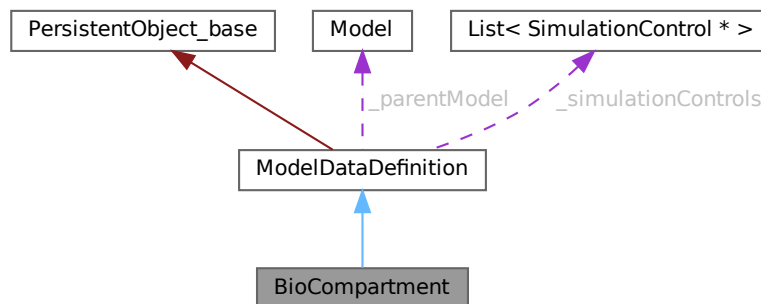
Defines a biological compartment for biochemical and whole-cell models.

```
#include <plugins/data/WholeCellModeling/BioCompartment.h>
```

Inheritance diagram for BioCompartment:



Collaboration diagram for BioCompartment:



Public Member Functions

- **BioCompartment** ([Model](#) *model, std::string name="")
- void **setCompartmentType** (std::string type)
- std::string **getCompartmentType** () const
- void **setParentCompartmentName** (std::string parentName)
- std::string **getParentCompartmentName** () const
- void **setMembraneBounded** (bool membraneBounded)
- bool **isMembraneBounded** () const
- void **setVolumeFraction** (double volumeFraction)
- double **getVolumeFraction** () const
- void **setCopyNumber** (unsigned int copyNumber)
- unsigned int **getCopyNumber** () const
- void **setRole** (std::string role)
- std::string **getRole** () const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- Releases resources owned by this data definition.*
- Util::identification [getId](#) () const
- Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)
- Updates the logical name of this instance.*
- const std::string & [getName](#) () const
- Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
- Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const
- Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const

- Returns the runtime class name used by the plugin system.*
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- **List**< **SimulationControl** * > * **getSimulationControls** () const
Returns the simulation controls owned by this data definition.
- **TraceManager::Level** **getTraceLevelSpecific** () const
Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (**TraceManager::Level** level)
Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (**TraceManager::Level** traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const
Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static **ModelDataDefinition** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
- static **PluginInformation** * **GetPluginInformation** ()
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")

Static Public Member Functions inherited from **ModelDataDefinition**

- static **ModelDataDefinition** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** (**PersistenceRecord** *fields, **ModelDataDefinition** *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** (**ModelDataDefinition** *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** (**ModelDataDefinition** *modeldatum)

Resets simulation-time state between replications.

- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()

- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.57.1 Detailed Description

Defines a biological compartment for biochemical and whole-cell models.

[BioCompartment](#) represents a named physical or logical cellular region such as cytosol, nucleus, mitochondrion, bud, membrane, or extracellular medium. It is a [ModelDataDefinition](#) and therefore stores structural metadata used by other plugins, but it does not execute simulation behavior itself.

The class is intentionally generic so it can support both prokaryotic and eukaryotic cells. Parent-child relationships allow hierarchical structures such as:

- extracellular -> cell surface
- cytosol -> nucleus
- cytosol -> mitochondrion
- cytosol -> bud

15.57.2 Member Function Documentation

15.57.2.1 **_check()**

```
bool BioCompartment::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.57.2.2 _createEditableDataDefinitions()

void BioCompartment::_createEditableDataDefinitions () [override], [protected], [virtual]
 Reimplemented from [ModelDataDefinition](#).

15.57.2.3 _loadInstance()

bool BioCompartment::_loadInstance (
 [PersistenceRecord](#) * fields) [override], [protected], [virtual]
 must be overridden by derived classes
 Reimplemented from [ModelDataDefinition](#).

15.57.2.4 _saveInstance()

void BioCompartment::_saveInstance (
 [PersistenceRecord](#) * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
 Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.57.2.5 show()

std::string BioCompartment::show () [override], [virtual]
 Returns a textual representation of this data definition.
 Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- plugins/data/WholeCellModeling/BioCompartment.h
- plugins/data/WholeCellModeling/BioCompartment.cpp

15.58 BioKineticLawExpression Class Reference

```
#include <tools/Biochemical/BioKineticLawExpression.h>
```

Public Types

- using **SymbolResolver** = std::function< bool(const std::string &name, double &value)>

Public Member Functions

- bool **evaluate** (const std::string &expression, SymbolResolver resolver, double &value, std::string &error← Message) const

15.58.1 Detailed Description

Small expression evaluator for biochemical kinetic laws.

The evaluator intentionally avoids the global GenESyS parser because the generated parser does not currently expose [BioSpecies](#) and [BioParameter](#) as kinetic-law symbols. Supported syntax is numeric literals, identifiers, parentheses, unary +/- operators, +, -, *, /, ^, and a small set of math functions.

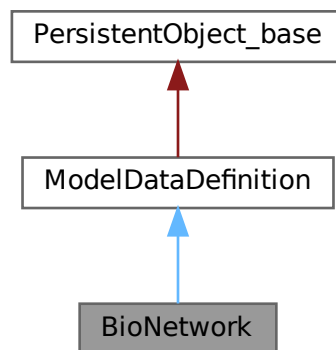
The documentation for this class was generated from the following file:

- tools/Biochemical/BioKineticLawExpression.h

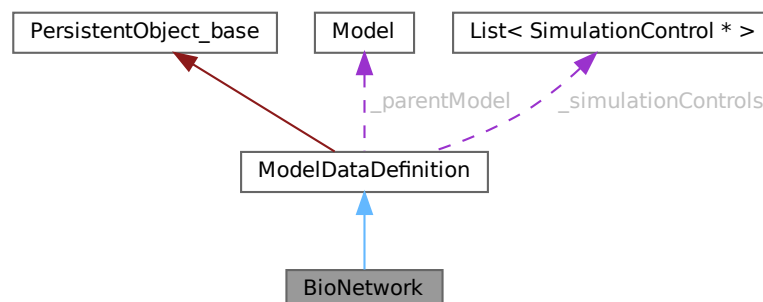
15.59 BioNetwork Class Reference

```
#include <plugins/data/BiochemicalSimulation/BioNetwork.h>
```

Inheritance diagram for BioNetwork:



Collaboration diagram for BioNetwork:



Public Member Functions

- **BioNetwork** ([Model](#) *model, std::string name="")
- void **setStartTime** (double startTime)
- double **getStartTime** () const
- void **setStopTime** (double stopTime)
- double **getStopTime** () const
- void **setStepSize** (double stepSize)

- double **getStepSize** () const
- void **setCurrentTime** (double currentTime)
- double **getCurrentTime** () const
- void **setAutoSchedule** (bool autoSchedule)
- bool **getAutoSchedule** () const
- void **setLastStatus** (std::string lastStatus)
- std::string **getLastStatus** () const
- void **setLastErrorMessage** (std::string lastErrorMessage)
- std::string **getLastErrorMessage** () const
- void **setLastResponsePayload** (std::string lastResponsePayload)
- std::string **getLastResponsePayload** () const
- void **addSpecies** (std::string speciesName)
- void **addReaction** (std::string reactionName)
- void **clearSpecies** ()
- void **clearReactions** ()
- const std::vector< std::string > & **getSpeciesNames** () const
- const std::vector< std::string > & **getReactionNames** () const
- const [BioSimulationResult](#) & **getLastSimulationResult** () const
- bool **getSpeciesTimeCourseDataset** (const std::string &speciesName, [SimulationResultsDataset](#) *dataset, std::string *errorMessage=nullptr) const
- bool **buildOdeSystemForAnalysis** ([MassActionOdeSystem](#) *system, std::string &errorMessage) const
- bool **getStoichiometryMatrix** ([BioStoichiometryMatrix](#) *matrix, std::string *errorMessage=nullptr) const
- bool **getReactionRateTimeCourse** ([BioReactionRateTimeCourse](#) *timeCourse, std::string *error← Message=nullptr) const
- bool **checkLastSampleSteadyState** (double tolerance, [BioSteadyStateCheck](#) *check, std::string *error← Message=nullptr) const
- bool **scanLocalParameterSensitivity** (double relativeStep, double absoluteStep, [BioSensitivityScan](#) *scan, std::string *errorMessage=nullptr) const
- bool **simulate** (std::string &errorMessage)
- bool **simulate** (double startTime, double stopTime, double stepSize, std::string &errorMessage)
- bool **advanceOneStep** (std::string &errorMessage)
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert← IntoModel=true)
- *Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- *Releases resources owned by this data definition.*
- Util::identification **getId** () const
- *Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
- *Updates the logical name of this instance.*
- const std::string & **getName** () const
- *Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)
- *Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const
- *Returns the user-visible label of this instance.*
- const std::string & **getClassname** () const
- *Returns the runtime class name used by the plugin system.*
- [Model](#) * **getParentModel** () const

- Returns the parent model that owns this definition.*
- bool `isReportStatistics ()` const
Indicates whether this data definition contributes report statistics.
- void `setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name)` const
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData ()` const
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData ()` const
Returns the full map of attached data definitions.
- bool `hasChanged ()` const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- unsigned int `getLevel ()` const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls ()` const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific ()` const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled ()` const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- static `ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
- static `PluginInformation * GetPluginInformation ()`
- static `ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- static void `SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.
- static bool `Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Validates a data definition instance.
- static void `InitBetweenReplications (ModelDataDefinition *modeldatum)`
Resets simulation-time state between replications.
- static void `CreateInternalData (ModelDataDefinition *modeldatum)`
Creates internal and attached data required by this instance.
- static bool `CreateRelatedDataElements (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` (`PersistenceRecord` *fields) override
must be overridden by derived classes
- virtual void `_saveInstance` (`PersistenceRecord` *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_initBetweenReplications` () override
- virtual void `_createEditableDataDefinitions` () override
Refreshes editable associations to referenced biochemical definitions.

Protected Member Functions inherited from `ModelDataDefinition`

- bool `_getSaveDefaultsOption` ()
- virtual `ParserChangesInformation` * `_getParserChangesInformation` ()
- virtual void `_addSimulationControl` (`SimulationControl` *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, `ModelDataDefinition` *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.59.1 Detailed Description

Native deterministic biochemical network runner.

[BioNetwork](#) advances [BioSpecies](#) with fixed-step RK4 over a mass-action ODE system. Networks may define explicit [BioSpecies](#) and [BioReaction](#) membership by name; when a membership list is empty, the runner falls back to discovering every registered definition of that type in the owning model for compatibility with existing models.

15.59.2 Member Function Documentation

15.59.2.1 [_check\(\)](#)

```
bool BioNetwork::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.59.2.2 [_createEditableDataDefinitions\(\)](#)

```
void BioNetwork::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Refreshes editable associations to referenced biochemical definitions.

[BioNetwork](#) stores model membership by name for persistence. During model checking these associations keep referenced species, reactions, and parameters from being pruned as orphan [ModelDataDefinitions](#).

Reimplemented from [ModelDataDefinition](#).

15.59.2.3 [_initBetweenReplications\(\)](#)

```
void BioNetwork::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.59.2.4 _loadInstance()

```
bool BioNetwork::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.59.2.5 _saveInstance()

```
void BioNetwork::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.59.2.6 show()

```
std::string BioNetwork::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

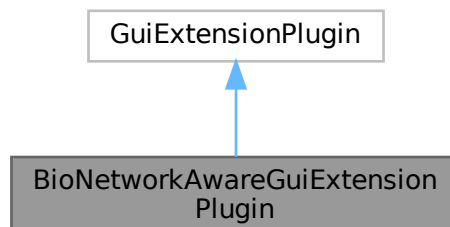
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

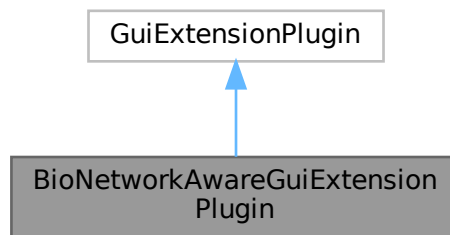
- plugins/data/BiochemicalSimulation/BioNetwork.h
- plugins/data/BiochemicalSimulation/BioNetwork.cpp

15.60 BioNetworkAwareGuiExtensionPlugin Class Reference

Inheritance diagram for BioNetworkAwareGuiExtensionPlugin:



Collaboration diagram for BioNetworkAwareGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId () const` override
- `std::string displayName () const` override
- `std::string version () const` override
- `std::vector< std::string > requiredModelPlugins () const` override
- `void registerContributions (GuiExtensionRegistry *registry) const` override

15.60.1 Member Function Documentation

15.60.1.1 displayName()

```
std::string BioNetworkAwareGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.60.1.2 extensionId()

```
std::string BioNetworkAwareGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.60.1.3 registerContributions()

```
void BioNetworkAwareGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.60.1.4 requiredModelPlugins()

```
std::vector< std::string > BioNetworkAwareGuiExtensionPlugin::requiredModelPlugins ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.60.1.5 version()

```
std::string BioNetworkAwareGuiExtensionPlugin::version ( ) const [inline], [override], [virtual]
```

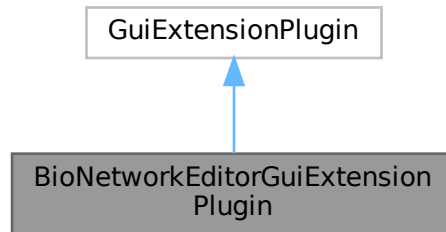
Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

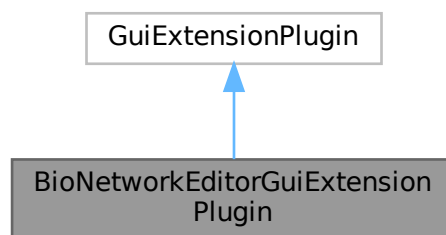
- `applications/gui/genesys/extensions/BioNetworkAwareGuiExtensionPlugin.cpp`

15.61 BioNetworkEditorGuiExtensionPlugin Class Reference

Inheritance diagram for BioNetworkEditorGuiExtensionPlugin:



Collaboration diagram for BioNetworkEditorGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId ()` const override
- `std::string displayName ()` const override
- `std::string version ()` const override
- `std::vector< std::string > requiredModelPlugins ()` const override
- `void registerContributions (GuiExtensionRegistry *registry)` const override

15.61.1 Member Function Documentation

15.61.1.1 displayName()

```
std::string BioNetworkEditorGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.61.1.2 extensionId()

```
std::string BioNetworkEditorGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.61.1.3 registerContributions()

```
void BioNetworkEditorGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.61.1.4 requiredModelPlugins()

```
std::vector< std::string > BioNetworkEditorGuiExtensionPlugin::requiredModelPlugins ( ) const
[inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.61.1.5 version()

```
std::string BioNetworkEditorGuiExtensionPlugin::version ( ) const [inline], [override], [virtual]
```

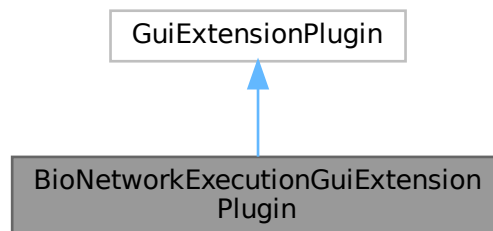
Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

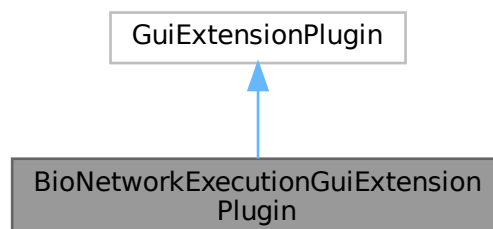
- applications/gui/genesys/extensions/BioNetworkEditorGuiExtensionPlugin.cpp

15.62 BioNetworkExecutionGuiExtensionPlugin Class Reference

Inheritance diagram for BioNetworkExecutionGuiExtensionPlugin:



Collaboration diagram for BioNetworkExecutionGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId ()` const override
- `std::string displayName ()` const override
- `std::string version ()` const override
- `std::vector< std::string > requiredModelPlugins ()` const override
- `void registerContributions (GuiExtensionRegistry *registry)` const override

15.62.1 Member Function Documentation**15.62.1.1 displayName()**

```
std::string BioNetworkExecutionGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.62.1.2 extensionId()

```
std::string BioNetworkExecutionGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.62.1.3 registerContributions()

```
void BioNetworkExecutionGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.62.1.4 requiredModelPlugins()

```
std::vector< std::string > BioNetworkExecutionGuiExtensionPlugin::requiredModelPlugins ( )
const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.62.1.5 version()

```
std::string BioNetworkExecutionGuiExtensionPlugin::version ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

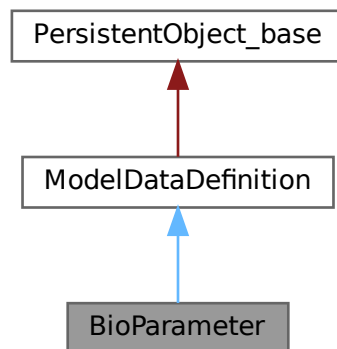
The documentation for this class was generated from the following file:

- `applications/gui/genesys/extensions/BioNetworkExecutionGuiExtensionPlugin.cpp`

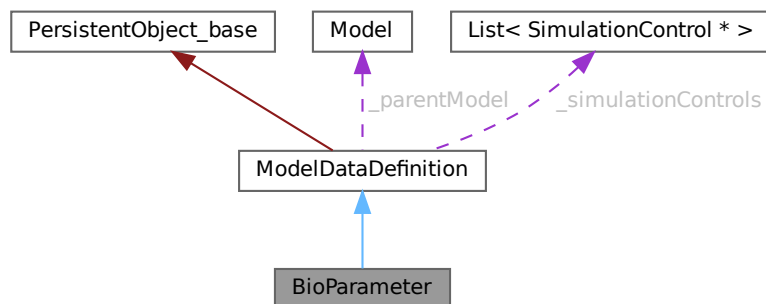
15.63 BioParameter Class Reference

```
#include <plugins/data/BiochemicalSimulation/BioParameter.h>
```

Inheritance diagram for BioParameter:



Collaboration diagram for BioParameter:



Public Member Functions

- **BioParameter** ([Model](#) *model, std::string name="")
- void **setValue** (double value)
- double **getValue** () const
- void **setUnit** (std::string unit)
- std::string **getUnit** () const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↔ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.

- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.

- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager](#)::Level [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager](#)::Level level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager](#)::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)

- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.63.1 Detailed Description

Scalar biochemical parameter, typically used as a kinetic constant.

[BioReaction](#) may reference a [BioParameter](#) by name. Parameter lookup remains a model-data responsibility, so missing references are reported by [BioReaction](#) validation rather than by this value object.

15.63.2 Member Function Documentation

15.63.2.1 **_check()**

```
bool BioParameter::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.63.2.2 `_loadInstance()`

```
bool BioParameter::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.63.2.3 `_saveInstance()`

```
void BioParameter::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.63.2.4 `show()`

```
std::string BioParameter::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

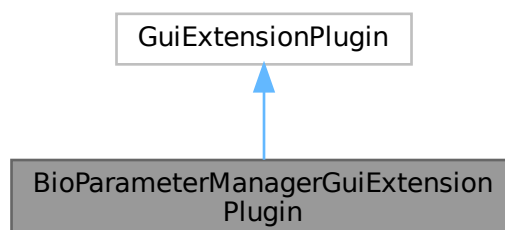
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

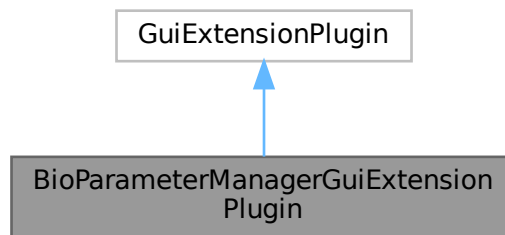
- plugins/data/BiochemicalSimulation/BioParameter.h
- plugins/data/BiochemicalSimulation/BioParameter.cpp

15.64 BioParameterManagerGuiExtensionPlugin Class Reference

Inheritance diagram for BioParameterManagerGuiExtensionPlugin:



Collaboration diagram for BioParameterManagerGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId () const` override
- `std::string displayName () const` override
- `std::string version () const` override
- `std::vector< std::string > requiredModelPlugins () const` override
- `void registerContributions (GuiExtensionRegistry *registry) const` override

15.64.1 Member Function Documentation

15.64.1.1 displayName()

```
std::string BioParameterManagerGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.64.1.2 extensionId()

```
std::string BioParameterManagerGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.64.1.3 registerContributions()

```
void BioParameterManagerGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.64.1.4 requiredModelPlugins()

```
std::vector< std::string > BioParameterManagerGuiExtensionPlugin::requiredModelPlugins ( )
const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.64.1.5 version()

```
std::string BioParameterManagerGuiExtensionPlugin::version ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

- `applications/gui/genesys/extensions/BioParameterManagerGuiExtensionPlugin.cpp`

15.65 BioParameterSensitivityEntry Struct Reference

Public Attributes

- std::string **parameterName**
- double **baseValue** = 0.0
- double **delta** = 0.0
- double **maxAbsoluteSensitivity** = 0.0
- std::vector< [BioSpeciesDerivative](#) > **derivativeSensitivities**

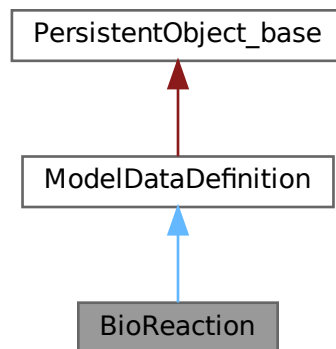
The documentation for this struct was generated from the following file:

- tools/Biochemical/BioSimulationAnalysis.h

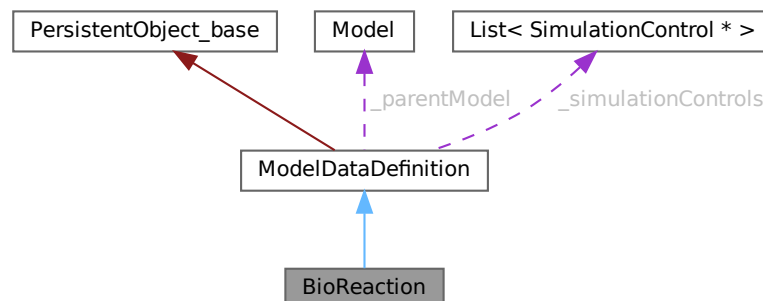
15.66 BioReaction Class Reference

```
#include <plugins/data/BiochemicalSimulation/BioReaction.h>
```

Inheritance diagram for BioReaction:



Collaboration diagram for BioReaction:



Classes

- struct [StoichiometricTerm](#)

Public Member Functions

- [BioReaction](#) ([Model](#) *model, std::string name="")
Creates an empty biochemical reaction definition.
- virtual ~**BioReaction** () override=default
Releases reaction-owned resources.
- void [addReactant](#) (std::string speciesName, double stoichiometry=1.0)
Adds a reactant species to the reaction.
- void [addProduct](#) (std::string speciesName, double stoichiometry=1.0)
Adds a product species to the reaction.
- void [addModifier](#) (std::string speciesName)
Adds a modifier species that influences the reaction rate.
- void **clearReactants** ()
Removes all reactants from the reaction.
- void **clearProducts** ()
Removes all products from the reaction.
- void **clearModifiers** ()
Removes all modifiers from the reaction.
- const std::vector< [StoichiometricTerm](#) > & [getReactants](#) () const
Returns the registered reactants.
- const std::vector< [StoichiometricTerm](#) > & [getProducts](#) () const
Returns the registered products.
- const std::vector< std::string > & [getModifiers](#) () const
Returns the registered modifiers.
- void [setRateConstant](#) (double rateConstant)
Sets the forward rate constant.
- double [getRateConstant](#) () const
Returns the forward rate constant.
- void [setRateConstantParameterName](#) (std::string rateConstantParameterName)
Stores the name of the parameter used for the forward rate constant.
- std::string [getRateConstantParameterName](#) () const
Returns the forward-rate parameter name.
- double [resolveRateConstant](#) () const
Resolves the effective forward rate constant.
- void [setReverseRateConstant](#) (double reverseRateConstant)
Sets the reverse rate constant.
- double [getReverseRateConstant](#) () const
Returns the reverse rate constant.
- void [setReverseRateConstantParameterName](#) (std::string reverseRateConstantParameterName)
Stores the name of the parameter used for the reverse rate constant.
- std::string [getReverseRateConstantParameterName](#) () const
Returns the reverse-rate parameter name.
- double [resolveReverseRateConstant](#) () const
Resolves the effective reverse rate constant.
- void [setKineticLawExpression](#) (std::string kineticLawExpression)
Sets the forward kinetic-law expression.
- std::string [getKineticLawExpression](#) () const
Returns the forward kinetic-law expression.
- void [setReverseKineticLawExpression](#) (std::string reverseKineticLawExpression)
Sets the reverse kinetic-law expression.
- std::string [getReverseKineticLawExpression](#) () const

- Returns the reverse kinetic-law expression.*
- void **setReversible** (bool reversible)
 - Marks the reaction as reversible or irreversible.*
- bool **isReversible** () const
 - Indicates whether the reaction is reversible.*
- virtual std::string **show** () override
 - Returns a textual representation of the reaction.*

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionType, std::string name="", bool insert↵ IntoModel=true)
 - Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
 - Releases resources owned by this data definition.*
- Util::identification **getId** () const
 - Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
 - Updates the logical name of this instance.*
- const std::string & **getName** () const
 - Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)
 - Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const
 - Returns the user-visible label of this instance.*
- const std::string & **getClassname** () const
 - Returns the runtime class name used by the plugin system.*
- **Model** * **getParentModel** () const
 - Returns the parent model that owns this definition.*
- bool **isReportStatistics** () const
 - Indicates whether this data definition contributes report statistics.*
- void **setReportStatistics** (bool reportStatistics)
 - Enables or disables report-statistics generation for this definition.*
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
 - Looks up a named internal child data definition.*
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
 - Returns the full map of internal child data definitions.*
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
 - Returns the full map of attached data definitions.*
- bool **hasChanged** () const
 - Indicates whether this instance has been modified since the last reset.*
- void **setHasChanged** (bool hasChanged)
 - Updates the changed flag for this data definition.*
- unsigned int **getLevel** () const
 - Returns the level in the model hierarchy where this definition lives.*
- void **setModelLevel** (unsigned int _modelLevel)
 - Sets the model nesting level for this definition.*
- List< **SimulationControl** * > * **getSimulationControls** () const
 - Returns the simulation controls owned by this data definition.*
- TraceManager::Level **getTraceLevelSpecific** () const
 - Returns the trace level that applies specifically to this definition.*

- void [setTraceLevelSpecific](#) (TraceManager::Level level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
Loads a biochemical reaction from serialized fields.
- static [PluginInformation](#) * [GetPluginInformation](#) ()
Returns plugin metadata for this reaction type.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new empty reaction instance.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createAttachedAttributes](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool _getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * _getParserChangesInformation ()`
- `virtual void _initBetweenReplications ()`
- `virtual void _addSimulationControl (SimulationControl *control)`
- `virtual void _createInternalAndAttachedData ()`
- `virtual void _createInternalStatisticReporters ()`
- `virtual void _createNonEditableDataDefinitions ()`
- `virtual void _createEditableDataDefinitions ()`
- `void _templateCreateInternalStatisticReporters ()`
- `void _templateCreateAttachedAttributes ()`
- `void _templateCreateNonEditableDataDefinitions ()`
- `void _templateCreateEditableDataDefinitions ()`
- `void _internalStatisticReportersClear ()`
legacy helper methods used by the granular create hooks*
- `void _internalDataClear ()`
- `void _internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void _internalDataRemove (const std::string &key)`
- `void _attachedDataClear ()`
- `void _attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void _attachedDataRemove (const std::string &key)`
- `void _attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void _statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void _statisticReporterRemove (const std::string &key)`
- `void _statisticReportersClear ()`
- `void _mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryAttachedAttributeRemove (const std::string &key)`
- `void _mandatoryAttachedAttributesClear ()`
- `void _mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryEditableDataDefinitionsClear ()`
- `void _optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalEditableDataDefinitionsClear ()`
- `void _mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryNonEditableDataDefinitionsClear ()`
- `void _optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalNonEditableDataDefinitionsClear ()`
- `void _checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
Traces text to the simulation output.
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.66.1 Detailed Description

Irreversible biochemical reaction definition.

Reactants, products, and modifiers are stored by species name so the reaction can be persisted independently from runtime pointers. Modifiers participate in kinetic laws without being consumed or produced. Reactions may omit reactants for synthesis or omit products for degradation, but must define at least one reactant or product. The current execution contract supports direct rate constants, a [BioParameter](#) reference, or an optional kinetic-law expression that resolves BioSpecies/BioParameter names at [BioNetwork](#) execution time. Reversible reactions use either a separate reverse kinetic-law expression or a reverse mass-action rate constant over the product side of the reaction.

15.66.2 Constructor & Destructor Documentation

15.66.2.1 BioReaction()

```
BioReaction::BioReaction (
    Model * model,
    std::string name = "" )
```

Creates an empty biochemical reaction definition.

Parameters

<i>model</i>	Parent model.
<i>name</i>	Initial reaction name.

15.66.3 Member Function Documentation

15.66.3.1 _check()

```
bool BioReaction::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.66.3.2 _createAttachedAttributes()

```
void BioReaction::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.66.3.3 _loadInstance()

```
bool BioReaction::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.66.3.4 `_saveInstance()`

```
void BioReaction::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.66.3.5 `addModifier()`

```
void BioReaction::addModifier (
    std::string speciesName )
```

Adds a modifier species that influences the reaction rate.

Parameters

<i>speciesName</i>	Modifier species name.
--------------------	------------------------

15.66.3.6 `addProduct()`

```
void BioReaction::addProduct (
    std::string speciesName,
    double stoichiometry = 1.0 )
```

Adds a product species to the reaction.

Parameters

<i>speciesName</i>	Product species name.
<i>stoichiometry</i>	Product stoichiometry.

15.66.3.7 `addReactant()`

```
void BioReaction::addReactant (
    std::string speciesName,
    double stoichiometry = 1.0 )
```

Adds a reactant species to the reaction.

Parameters

<i>speciesName</i>	Reactant species name.
<i>stoichiometry</i>	Reactant stoichiometry.

15.66.3.8 `getKineticLawExpression()`

```
std::string BioReaction::getKineticLawExpression ( ) const
```

Returns the forward kinetic-law expression.

Returns

Forward kinetic-law expression.

15.66.3.9 getModifiers()

```
const std::vector< std::string > & BioReaction::getModifiers ( ) const
```

Returns the registered modifiers.

Returns

Constant reference to the modifier names.

15.66.3.10 GetPluginInformation()

```
PluginInformation * BioReaction::GetPluginInformation ( ) [static]
```

Returns plugin metadata for this reaction type.

Returns

[Plugin](#) information block.

15.66.3.11 getProducts()

```
const std::vector< BioReaction::StoichiometricTerm > & BioReaction::getProducts ( ) const
```

Returns the registered products.

Returns

Constant reference to the product terms.

15.66.3.12 getRateConstant()

```
double BioReaction::getRateConstant ( ) const
```

Returns the forward rate constant.

Returns

Forward rate constant.

15.66.3.13 getRateConstantParameterName()

```
std::string BioReaction::getRateConstantParameterName ( ) const
```

Returns the forward-rate parameter name.

Returns

Parameter name used to resolve the forward rate constant.

15.66.3.14 getReactants()

```
const std::vector< BioReaction::StoichiometricTerm > & BioReaction::getReactants ( ) const
```

Returns the registered reactants.

Returns

Constant reference to the reactant terms.

15.66.3.15 getReverseKineticLawExpression()

```
std::string BioReaction::getReverseKineticLawExpression ( ) const
```

Returns the reverse kinetic-law expression.

Returns

Reverse kinetic-law expression.

15.66.3.16 getReverseRateConstant()

```
double BioReaction::getReverseRateConstant ( ) const
```

Returns the reverse rate constant.

Returns

Reverse rate constant.

15.66.3.17 getReverseRateConstantParameterName()

```
std::string BioReaction::getReverseRateConstantParameterName ( ) const
```

Returns the reverse-rate parameter name.

Returns

Parameter name used to resolve the reverse rate constant.

15.66.3.18 isReversible()

```
bool BioReaction::isReversible ( ) const
```

Indicates whether the reaction is reversible.

Returns

`true` when the reaction has a reverse branch.

15.66.3.19 LoadInstance()

```
ModelDataDefinition * BioReaction::LoadInstance (
    Model * model,
    PersistenceRecord * fields ) [static]
```

Loads a biochemical reaction from serialized fields.

Parameters

<i>model</i>	Parent model.
<i>fields</i>	Serialized fields.

Returns

Newly created reaction instance.

15.66.3.20 NewInstance()

```
ModelDataDefinition * BioReaction::NewInstance (
    Model * model,
    std::string name = "" ) [static]
```

Creates a new empty reaction instance.

Parameters

<i>model</i>	Parent model.
<i>name</i>	Initial reaction name.

Returns

Newly created reaction instance.

15.66.3.21 resolveRateConstant()

```
double BioReaction::resolveRateConstant ( ) const
```

Resolves the effective forward rate constant.

Returns

Forward rate constant after parameter resolution.

15.66.3.22 resolveReverseRateConstant()

```
double BioReaction::resolveReverseRateConstant ( ) const
```

Resolves the effective reverse rate constant.

Returns

Reverse rate constant after parameter resolution.

15.66.3.23 setKineticLawExpression()

```
void BioReaction::setKineticLawExpression (
    std::string kineticLawExpression )
```

Sets the forward kinetic-law expression.

Parameters

<i>kineticLawExpression</i>	Expression text.
-----------------------------	------------------

15.66.3.24 setRateConstant()

```
void BioReaction::setRateConstant (
    double rateConstant )
```

Sets the forward rate constant.

Parameters

<i>rateConstant</i>	Forward rate constant value.
---------------------	------------------------------

15.66.3.25 setRateConstantParameterName()

```
void BioReaction::setRateConstantParameterName (
    std::string rateConstantParameterName )
```

Stores the name of the parameter used for the forward rate constant.

Parameters

<i>rateConstantParameterName</i>	Parameter name.
----------------------------------	-----------------

15.66.3.26 setReverseKineticLawExpression()

```
void BioReaction::setReverseKineticLawExpression (
    std::string reverseKineticLawExpression )
```

Sets the reverse kinetic-law expression.

Parameters

<i>reverseKineticLawExpression</i>	Expression text.
------------------------------------	------------------

15.66.3.27 setReverseRateConstant()

```
void BioReaction::setReverseRateConstant (
    double reverseRateConstant )
```

Sets the reverse rate constant.

Parameters

<i>reverseRateConstant</i>	Reverse rate constant value.
----------------------------	------------------------------

15.66.3.28 setReverseRateConstantParameterName()

```
void BioReaction::setReverseRateConstantParameterName (
    std::string reverseRateConstantParameterName )
```

Stores the name of the parameter used for the reverse rate constant.

Parameters

<i>reverseRateConstantParameterName</i>	Parameter name.
---	-----------------

15.66.3.29 setReversible()

```
void BioReaction::setReversible (
    bool reversible )
```

Marks the reaction as reversible or irreversible.

Parameters

<i>reversible</i>	New reversible flag.
-------------------	----------------------

15.66.3.30 show()

```
std::string BioReaction::show ( ) [override], [virtual]
```

Returns a textual representation of the reaction.

Returns

Human-readable reaction summary.

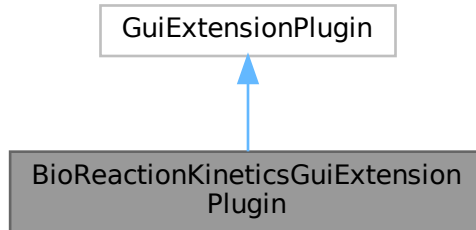
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

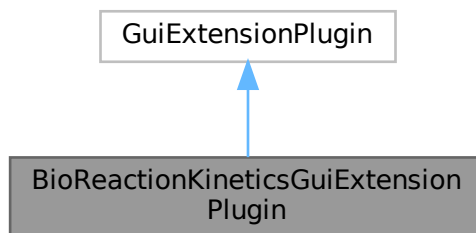
- plugins/data/BiochemicalSimulation/BioReaction.h
- plugins/data/BiochemicalSimulation/BioReaction.cpp

15.67 BioReactionKineticsGuiExtensionPlugin Class Reference

Inheritance diagram for BioReactionKineticsGuiExtensionPlugin:



Collaboration diagram for BioReactionKineticsGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId ()` const override
- `std::string displayName ()` const override
- `std::string version ()` const override
- `std::vector< std::string > requiredModelPlugins ()` const override
- `void registerContributions (GuiExtensionRegistry *registry)` const override

15.67.1 Member Function Documentation

15.67.1.1 displayName()

```
std::string BioReactionKineticsGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.67.1.2 extensionId()

```
std::string BioReactionKineticsGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.67.1.3 registerContributions()

```
void BioReactionKineticsGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.67.1.4 requiredModelPlugins()

```
std::vector< std::string > BioReactionKineticsGuiExtensionPlugin::requiredModelPlugins ( )
const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.67.1.5 version()

```
std::string BioReactionKineticsGuiExtensionPlugin::version ( ) const [inline], [override],
[virtual]
```

Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

- applications/gui/genesys/extensions/BioReactionKineticsGuiExtensionPlugin.cpp

15.68 BioReactionRateSample Struct Reference

Public Attributes

- double **time** = 0.0
- std::vector< double > **forwardRates**
- std::vector< double > **reverseRates**
- std::vector< double > **netRates**

The documentation for this struct was generated from the following file:

- tools/Biochemical/BioSimulationAnalysis.h

15.69 BioReactionRateTimeCourse Struct Reference

Public Member Functions

- bool **hasReaction** (const std::string &reactionName) const
- bool **toDataset** (const std::string &reactionName, BioReactionRateKind kind, [SimulationResultsDataset](#) *dataset, std::string *errorMessage=nullptr) const

Public Attributes

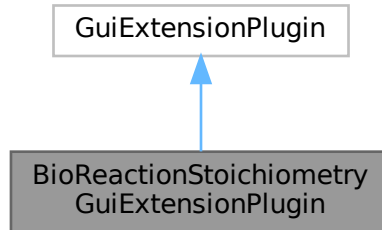
- std::vector< std::string > **reactionNames**
- std::vector< [BioReactionRateSample](#) > **samples**

The documentation for this struct was generated from the following file:

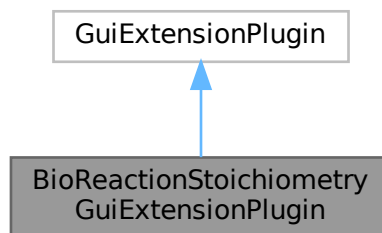
- tools/Biochemical/BioSimulationAnalysis.h

15.70 BioReactionStoichiometryGuiExtensionPlugin Class Reference

Inheritance diagram for BioReactionStoichiometryGuiExtensionPlugin:



Collaboration diagram for BioReactionStoichiometryGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId ()` const override
- `std::string displayName ()` const override
- `std::string version ()` const override
- `std::vector< std::string > requiredModelPlugins ()` const override
- `void registerContributions (GuiExtensionRegistry *registry)` const override

15.70.1 Member Function Documentation

15.70.1.1 displayName()

```
std::string BioReactionStoichiometryGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.70.1.2 extensionId()

```
std::string BioReactionStoichiometryGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.70.1.3 registerContributions()

```
void BioReactionStoichiometryGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.70.1.4 requiredModelPlugins()

```
std::vector< std::string > BioReactionStoichiometryGuiExtensionPlugin::requiredModelPlugins (
) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.70.1.5 version()

```
std::string BioReactionStoichiometryGuiExtensionPlugin::version ( ) const [inline], [override],
[virtual]
```

Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

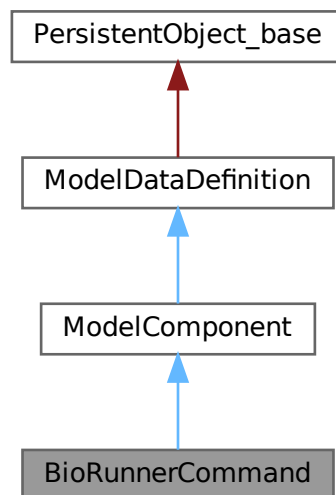
- applications/gui/genesys/extensions/BioReactionStoichiometryGuiExtensionPlugin.cpp

15.71 BioRunnerCommand Class Reference

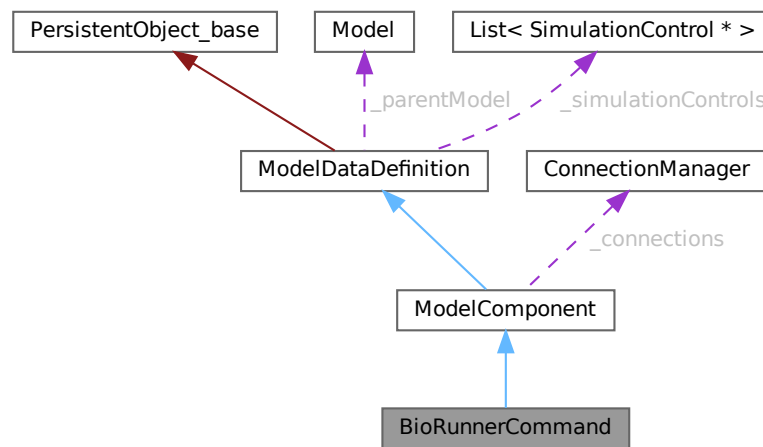
Component wrapper over [BioSimulatorRunner](#) command execution.

```
#include <plugins/components/BiochemicalSimulation/BioRunnerCommand.h>
```

Inheritance diagram for BioRunnerCommand:



Collaboration diagram for BioRunnerCommand:



Public Member Functions

- **BioRunnerCommand** ([Model](#) *model, std::string name="")
- void **setRunner** ([BioSimulatorRunner](#) *runner)
- [BioSimulatorRunner](#) * **getRunner** () const
- void **setCommand** (std::string command)
- std::string **getCommand** () const
- void **setLastSucceeded** (bool lastSucceeded)
- bool **getLastSucceeded** () const
- void **setLastStatus** (std::string lastStatus)
- std::string **getLastStatus** () const
- void **setLastMessage** (std::string lastMessage)
- std::string **getLastMessage** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual **~ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool **hasChanged**)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void **_createEditableDataDefinitions** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool _getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * _getParserChangesInformation ()`
- `virtual void _initBetweenReplications ()`
- `virtual void _addSimulationControl (SimulationControl *control)`
- `virtual void _createInternalAndAttachedData ()`
- `virtual void _createInternalStatisticReporters ()`
- `virtual void _createAttachedAttributes ()`
- `virtual void _createNonEditableDataDefinitions ()`
- `void _templateCreateInternalStatisticReporters ()`
- `void _templateCreateAttachedAttributes ()`
- `void _templateCreateNonEditableDataDefinitions ()`
- `void _templateCreateEditableDataDefinitions ()`
- `void _internalStatisticReportersClear ()`
 - legacy helper methods used by the granular create* hooks*
- `void _internalDataClear ()`
- `void _internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void _internalDataRemove (const std::string &key)`
- `void _attachedDataClear ()`
- `void _attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void _attachedDataRemove (const std::string &key)`
- `void _attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void _statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void _statisticReporterRemove (const std::string &key)`
- `void _statisticReportersClear ()`
- `void _mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryAttachedAttributeRemove (const std::string &key)`
- `void _mandatoryAttachedAttributesClear ()`
- `void _mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryEditableDataDefinitionsClear ()`
- `void _optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalEditableDataDefinitionsClear ()`
- `void _mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryNonEditableDataDefinitionsClear ()`
- `void _optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalNonEditableDataDefinitionsClear ()`
- `void _checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
 - Traces text to the simulation output.*
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

15.71.1 Detailed Description

Component wrapper over [BioSimulatorRunner](#) command execution.

[BioRunnerCommand](#) dispatches one configured command into a referenced [BioSimulatorRunner](#) whenever an entity arrives and stores the last runner status/payload metadata for downstream model logic.

15.71.2 Member Function Documentation

15.71.2.1 `_check()`

```
bool BioRunnerCommand::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.71.2.2 `_createEditableDataDefinitions()`

```
void BioRunnerCommand::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.71.2.3 `_loadInstance()`

```
bool BioRunnerCommand::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [ModelComponent](#).

15.71.2.4 `_onDispatchEvent()`

```
void BioRunnerCommand::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.71.2.5 `_saveInstance()`

```
void BioRunnerCommand::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.71.2.6 `show()`

```
std::string BioRunnerCommand::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/BiochemicalSimulation/BioRunnerCommand.h
- plugins/components/BiochemicalSimulation/BioRunnerCommand.cpp

15.72 BioSBMLBridge Class Reference

Static Public Member Functions

- static bool **importFromString** ([Model](#) *model, const std::string &sbmlText, const std::string &preferredNetworkName, [BioSBMLImportResult](#) *result, std::string &errorMessage)
- static bool **exportToString** ([Model](#) *model, const std::string &requestedNetworkName, std::string *sbmlText, [BioSBMLExportResult](#) *result, std::string &errorMessage)

The documentation for this class was generated from the following files:

- plugins/data/BiochemicalSimulation/BioSBMLBridge.h
- plugins/data/BiochemicalSimulation/BioSBMLBridge.cpp

15.73 BioSBMLExportResult Struct Reference

Public Attributes

- std::string **networkName**
- unsigned int **speciesExported** = 0
- unsigned int **parametersExported** = 0
- unsigned int **reactionsExported** = 0
- std::vector< std::string > **warnings**

The documentation for this struct was generated from the following file:

- plugins/data/BiochemicalSimulation/BioSBMLBridge.h

15.74 BioSBMLImportResult Struct Reference

Public Attributes

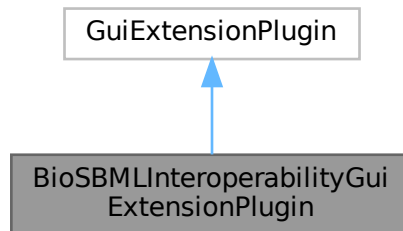
- std::string **networkName**
- unsigned int **speciesImported** = 0
- unsigned int **parametersImported** = 0
- unsigned int **reactionsImported** = 0
- std::vector< std::string > **warnings**

The documentation for this struct was generated from the following file:

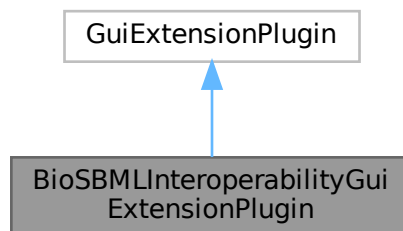
- plugins/data/BiochemicalSimulation/BioSBMLBridge.h

15.75 BioSBMLInteroperabilityGuiExtensionPlugin Class Reference

Inheritance diagram for BioSBMLInteroperabilityGuiExtensionPlugin:



Collaboration diagram for BioSBMLInteroperabilityGuiExtensionPlugin:



Public Member Functions

- std::string [extensionId](#) () const override
- std::string [displayName](#) () const override
- std::string [version](#) () const override
- std::vector< std::string > [requiredModelPlugins](#) () const override
- void [registerContributions](#) ([GuiExtensionRegistry](#) *registry) const override

15.75.1 Member Function Documentation

15.75.1.1 displayName()

```
std::string BioSBMLInteroperabilityGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.75.1.2 extensionId()

```
std::string BioSBMLInteroperabilityGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.75.1.3 registerContributions()

```
void BioSBMLInteroperabilityGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.75.1.4 requiredModelPlugins()

```
std::vector< std::string > BioSBMLInteroperabilityGuiExtensionPlugin::requiredModelPlugins ( )
const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.75.1.5 version()

```
std::string BioSBMLInteroperabilityGuiExtensionPlugin::version ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

- applications/gui/genesys/extensions/BioSBMLInteroperabilityGuiExtensionPlugin.cpp

15.76 BioSensitivityScan Struct Reference

Public Attributes

- double **time** = 0.0
- std::vector< std::string > **speciesNames**
- std::vector< [BioParameterSensitivityEntry](#) > **entries**

The documentation for this struct was generated from the following file:

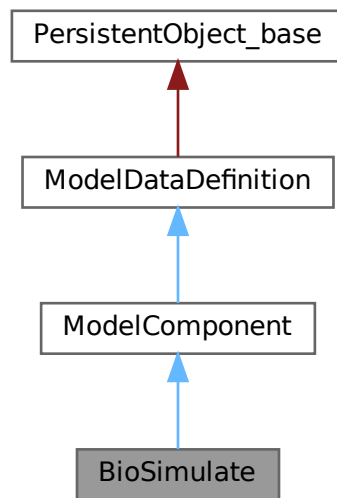
- tools/Biochemical/BioSimulationAnalysis.h

15.77 BioSimulate Class Reference

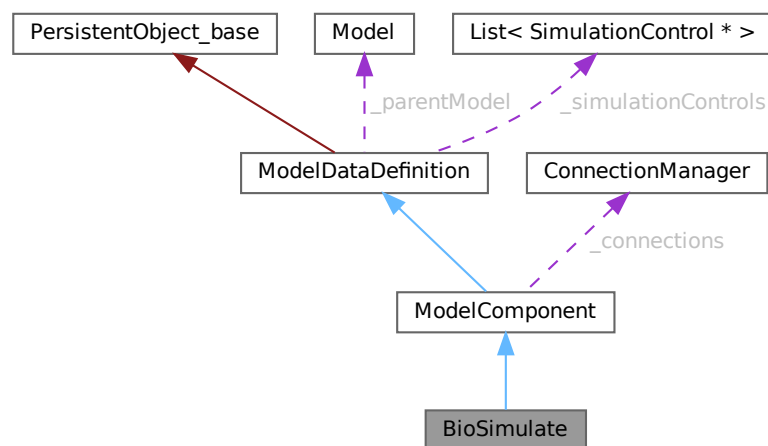
Component that triggers deterministic [BioNetwork](#) simulation.

```
#include <plugins/components/BiochemicalSimulation/BioSimulate.h>
```

Inheritance diagram for BioSimulate:



Collaboration diagram for BioSimulate:



Public Member Functions

- **BioSimulate** ([Model](#) *model, std::string name="")
- void **setBioNetwork** ([BioNetwork](#) *bioNetwork)
- [BioNetwork](#) * **getBioNetwork** () const
- void **setUseNetworkTimeWindow** (bool useNetworkTimeWindow)
- bool **getUseNetworkTimeWindow** () const
- void **setStartTime** (double startTime)
- double **getStartTime** () const

- void **setStopTime** (double stopTime)
- double **getStopTime** () const
- void **setStepSize** (double stepSize)
- double **getStepSize** () const
- void **setLastSucceeded** (bool lastSucceeded)
- bool **getLastSucceeded** () const
- void **setLastMessage** (std::string lastMessage)
- std::string **getLastMessage** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentName, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

- Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)
- Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
- Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
- Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
- Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const
- Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool hasChanged)
- Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const
- Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int _modelLevel)
- Sets the model nesting level for this definition.*
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
- Returns the simulation controls owned by this data definition.*
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
- Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
- Updates the trace level used by this definition.*
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
- Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const
- Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
- Enables or disables the specific trace level.*

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool [Check](#) ([ModelComponent](#) *component)
- Validates a component instance.*
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
- Creates internal data required by a component instance.*
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
- Serializes a component instance into a persistence record.*
- static void [DispatchEvent](#) ([Event](#) *event)
- Dispatches a simulation event to the target component.*

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)

- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.77.1 Detailed Description

Component that triggers deterministic [BioNetwork](#) simulation.

[BioSimulate](#) delegates simulation execution to a referenced [BioNetwork](#). Each arriving entity can either run the network with its own configured time window or with component-provided start/stop/step values.

15.77.2 Member Function Documentation

15.77.2.1 `_check()`

```
bool BioSimulate::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.77.2.2 `_createEditableDataDefinitions()`

```
void BioSimulate::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.77.2.3 `_loadInstance()`

```
bool BioSimulate::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.77.2.4 `_onDispatchEvent()`

```
void BioSimulate::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.77.2.5 `_saveInstance()`

```
void BioSimulate::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.77.2.6 `show()`

```
std::string BioSimulate::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/BiochemicalSimulation/BioSimulate.h
- plugins/components/BiochemicalSimulation/BioSimulate.cpp

15.78 BioSimulationAnalysis Class Reference

Static Public Member Functions

- static [BioStoichiometryMatrix](#) **buildStoichiometryMatrix** (const [MassActionOdeSystem](#) &system)
- static bool **buildReactionRateTimeCourse** (const [MassActionOdeSystem](#) &system, const [BioSimulationResult](#) &result, [BioReactionRateTimeCourse](#) *timeCourse, std::string *errorMessage=nullptr)
- static bool **checkSteadyState** (const [MassActionOdeSystem](#) &system, const [BioSimulationSample](#) &sample, double tolerance, [BioSteadyStateCheck](#) *check, std::string *errorMessage=nullptr)
- static bool **scanLocalParameterSensitivity** (const [MassActionOdeSystem](#) &system, const [BioSimulationSample](#) &sample, double relativeStep, double absoluteStep, [BioSensitivityScan](#) *scan, std::string *error←Message=nullptr)

The documentation for this class was generated from the following file:

- tools/Biochemical/BioSimulationAnalysis.h

15.79 BioSimulationResult Class Reference

Public Member Functions

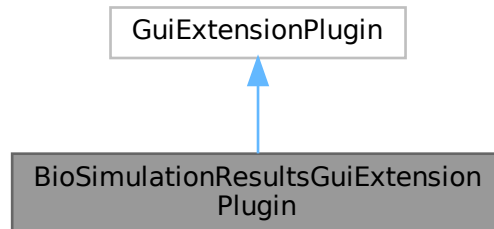
- void **clear** ()
- void **configure** (std::string networkName, double startTime, double stopTime, double stepSize, const std::vector< std::string > &speciesNames)
- void **appendSample** (double time, const std::vector< [BioSimulationSpeciesAmount](#) > &speciesAmounts)
- bool **empty** () const
- unsigned int **sampleCount** () const
- const std::string & **getNetworkName** () const
- double **getStartTime** () const
- double **getStopTime** () const
- double **getStepSize** () const
- const std::vector< std::string > & **getSpeciesNames** () const
- const std::vector< [BioSimulationSample](#) > & **getSamples** () const
- bool **hasSpecies** (const std::string &speciesName) const
- bool **toDataset** (const std::string &speciesName, [SimulationResultsDataset](#) *dataset, std::string *error←Message=nullptr) const
- std::vector< [SimulationResultsDataset](#) > **toDatasets** () const

The documentation for this class was generated from the following file:

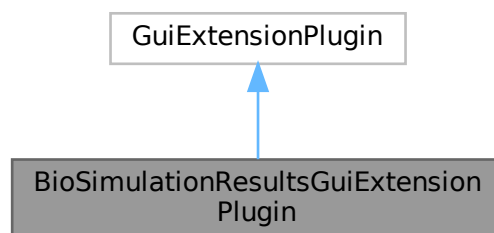
- tools/Biochemical/BioSimulationResult.h

15.80 BioSimulationResultsGuiExtensionPlugin Class Reference

Inheritance diagram for BioSimulationResultsGuiExtensionPlugin:



Collaboration diagram for BioSimulationResultsGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId ()` const override
- `std::string displayName ()` const override
- `std::string version ()` const override
- `std::vector< std::string > requiredModelPlugins ()` const override
- `void registerContributions (GuiExtensionRegistry *registry)` const override

15.80.1 Member Function Documentation

15.80.1.1 displayName()

```
std::string BioSimulationResultsGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.80.1.2 extensionId()

```
std::string BioSimulationResultsGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.80.1.3 registerContributions()

```
void BioSimulationResultsGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.80.1.4 requiredModelPlugins()

```
std::vector< std::string > BioSimulationResultsGuiExtensionPlugin::requiredModelPlugins ( )
const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.80.1.5 version()

```
std::string BioSimulationResultsGuiExtensionPlugin::version ( ) const [inline], [override],
[virtual]
```

Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

- applications/gui/genesys/extensions/BioSimulationResultsGuiExtensionPlugin.cpp

15.81 BioSimulationSample Struct Reference

Public Attributes

- double **time** = 0.0
- std::vector< [BioSimulationSpeciesAmount](#) > **species**

The documentation for this struct was generated from the following file:

- tools/Biochemical/BioSimulationResult.h

15.82 BioSimulationSpeciesAmount Struct Reference

Public Attributes

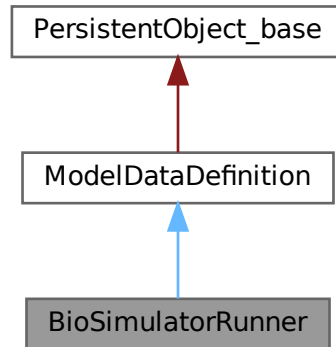
- std::string **speciesName**
- double **amount** = 0.0

The documentation for this struct was generated from the following file:

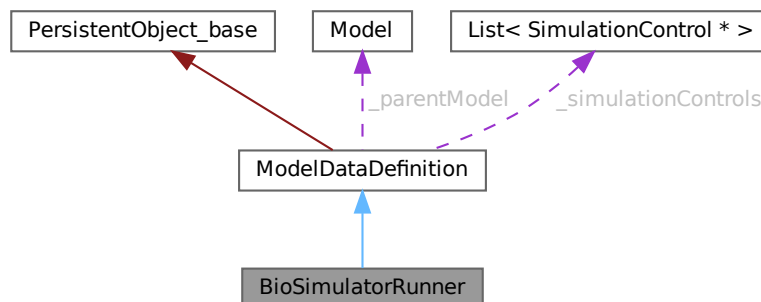
- tools/Biochemical/BioSimulationResult.h

15.83 BioSimulatorRunner Class Reference

Inheritance diagram for BioSimulatorRunner:



Collaboration diagram for BioSimulatorRunner:



Public Member Functions

- **BioSimulatorRunner** ([Model](#) *model, std::string name="")
- void **setBackend** (std::string backend)
- std::string **getBackend** () const
- void **setModelSourceType** (std::string modelSourceType)
- std::string **getModelSourceType** () const
- void **setModelSource** (std::string modelSource)
- std::string **getModelSource** () const
- void **setCommand** (std::string command)
- std::string **getCommand** () const
- void **setLastStatus** (std::string lastStatus)
- std::string **getLastStatus** () const
- void **setLastErrorMessage** (std::string lastErrorMessage)
- std::string **getLastErrorMessage** () const
- void **setLastResponsePayload** (std::string lastResponsePayload)

- `std::string getLastResponsePayload ()` const
- `void setLastResponseFilename (std::string lastResponseFilename)`
- `std::string getLastResponseFilename ()` const
- `void setTargetBioNetworkName (std::string targetBioNetworkName)`
- `std::string getTargetBioNetworkName ()` const
- `void setWorkingDirectory (std::string workingDirectory)`
- `std::string getWorkingDirectory ()` const
- `void setWorkingInputFilename (std::string workingInputFilename)`
- `std::string getWorkingInputFilename ()` const
- `void setWorkingOutputFilename (std::string workingOutputFilename)`
- `std::string getWorkingOutputFilename ()` const
- `void setEndpointOrLibrary (std::string endpointOrLibrary)`
- `std::string getEndpointOrLibrary ()` const
- `void setTimeoutSeconds (unsigned int timeoutSeconds)`
- `unsigned int getTimeoutSeconds ()` const
- `void setAutoValidateModel (bool autoValidateModel)`
- `bool getAutoValidateModel ()` const
- `bool executeCommand (std::string &errorMessage)`
- `virtual std::string show ()` override

Returns a textual representation of this data definition.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- `virtual ~ModelDataDefinition ()`
- Releases resources owned by this data definition.*
- `Util::identification getId ()` const
- Returns the unique identifier of this instance.*
- `void setName (const std::string &name)`
- Updates the logical name of this instance.*
- `const std::string & getName ()` const
- Returns the logical name of this instance.*
- `void setLabel (const std::string &label)`
- Updates the user-visible label of this instance.*
- `const std::string & getLabel ()` const
- Returns the user-visible label of this instance.*
- `const std::string & getClassname ()` const
- Returns the runtime class name used by the plugin system.*
- `Model * getParentModel ()` const
- Returns the parent model that owns this definition.*
- `bool isReportStatistics ()` const
- Indicates whether this data definition contributes report statistics.*
- `void setReportStatistics (bool reportStatistics)`
- Enables or disables report-statistics generation for this definition.*
- `ModelDataDefinition * getInternalData (const std::string &name)` const
- Looks up a named internal child data definition.*
- `std::map< std::string, ModelDataDefinition * > * getInternalData ()` const
- Returns the full map of internal child data definitions.*
- `std::map< std::string, ModelDataDefinition * > * getAttachedData ()` const
- Returns the full map of attached data definitions.*

- bool `hasChanged` () const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged` (bool `hasChanged`)
Updates the changed flag for this data definition.
- unsigned int `getLevel` () const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel` (unsigned int `_modelLevel`)
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls` () const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level` `getTraceLevelSpecific` () const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` `level`)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` `traceLevelSpecific`, bool `traceLevelSpecificEnabled`)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool `traceLevelSpecificEnabled`)
Enables or disables the specific trace level.

Static Public Member Functions

- static `ModelDataDefinition * LoadInstance` (`Model *model`, `PersistenceRecord *fields`)
- static `PluginInformation * GetPluginInformation` ()
- static `ModelDataDefinition * NewInstance` (`Model *model`, `std::string name=""`)

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition * LoadInstance` (`Model *model`, `PersistenceRecord *fields`, bool `insertIntoModel`)
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition * NewInstance` (`Model *model`, `std::string name=""`)
Creates a new instance of a data definition using the plugin registry.
- static void `SaveInstance` (`PersistenceRecord *fields`, `ModelDataDefinition *modeldatum`)
Serializes a data definition instance into a persistence record.
- static bool `Check` (`ModelDataDefinition *modeldatum`, `std::string &errorMessage`)
Validates a data definition instance.
- static void `InitBetweenReplications` (`ModelDataDefinition *modeldatum`)
Resets simulation-time state between replications.
- static void `CreateInternalData` (`ModelDataDefinition *modeldatum`)
Creates internal and attached data required by this instance.
- static bool `CreateRelatedDataElements` (`ModelDataDefinition *modeldatum`, `std::string &errorMessage`)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` (`PersistenceRecord *fields`) override
must be overridden by derived classes
- virtual void `_saveInstance` (`PersistenceRecord *fields`, bool `saveDefaultValues`) override
Saves object fields into a serialization structure.
- virtual bool `_check` (`std::string &errorMessage`) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.83.1 Member Function Documentation

15.83.1.1 `_check()`

```
bool BioSimulatorRunner::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.83.1.2 `_loadInstance()`

```
bool BioSimulatorRunner::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.83.1.3 `_saveInstance()`

```
void BioSimulatorRunner::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.83.1.4 `show()`

```
std::string BioSimulatorRunner::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

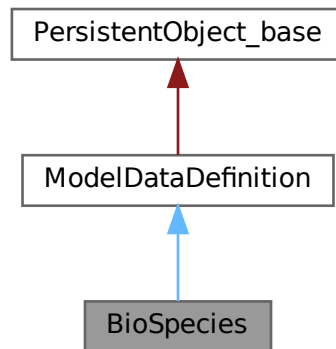
The documentation for this class was generated from the following files:

- plugins/data/BiochemicalSimulation/BioSimulatorRunner.h
- plugins/data/BiochemicalSimulation/BioSimulatorRunner.cpp

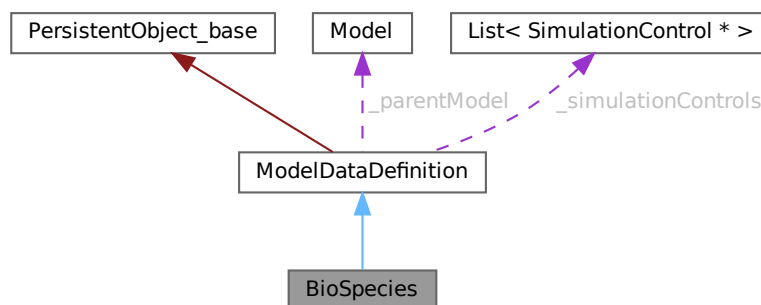
15.84 BioSpecies Class Reference

```
#include <plugins/data/BiochemicalSimulation/BioSpecies.h>
```

Inheritance diagram for BioSpecies:



Collaboration diagram for BioSpecies:



Public Member Functions

- **BioSpecies** ([Model](#) *model, std::string name="")
- void **setInitialAmount** (double initialAmount)
- double **getInitialAmount** () const
- void **setAmount** (double amount)
- double **getAmount** () const
- void **setConstant** (bool constant)
- bool **isConstant** () const
- void **setBoundaryCondition** (bool boundaryCondition)
- bool **isBoundaryCondition** () const
- void **setUnit** (std::string unit)
- std::string **getUnit** () const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool **hasChanged**)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_initBetweenReplications** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool **_getSaveDefaultsOption** ()
- virtual [ParserChangesInformation](#) * **_getParserChangesInformation** ()
- virtual void **_addSimulationControl** ([SimulationControl](#) *control)
- virtual void **_createInternalAndAttachedData** ()
- virtual void **_createInternalStatisticReporters** ()
- virtual void **_createAttachedAttributes** ()
- virtual void **_createNonEditableDataDefinitions** ()
- virtual void **_createEditableDataDefinitions** ()
- void **_templateCreateInternalStatisticReporters** ()
- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
legacy helper methods used by the granular create hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)

- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.84.1 Detailed Description

Biochemical state variable used by [BioReaction](#) and [BioNetwork](#).

A [BioSpecies](#) stores an initial amount for replication reset and a current amount for simulation. Constant and boundary-condition flags follow SBML-like semantics: [BioNetwork](#) still uses the current amount as kinetic input, but it must not overwrite the species amount while advancing the ODE state.

15.84.2 Member Function Documentation

15.84.2.1 `_check()`

```
bool BioSpecies::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.84.2.2 `_initBetweenReplications()`

```
void BioSpecies::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.84.2.3 `_loadInstance()`

```
bool BioSpecies::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.84.2.4 `_saveInstance()`

```
void BioSpecies::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.84.2.5 `show()`

```
std::string BioSpecies::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- plugins/data/BiochemicalSimulation/BioSpecies.h
- plugins/data/BiochemicalSimulation/BioSpecies.cpp

15.85 BioSpeciesDerivative Struct Reference

Public Attributes

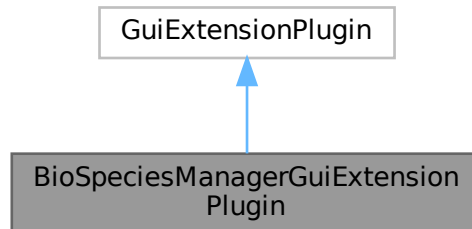
- std::string **speciesName**
- double **derivative** = 0.0

The documentation for this struct was generated from the following file:

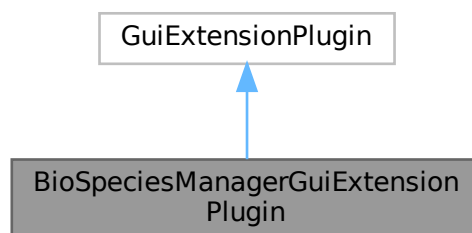
- tools/Biochemical/BioSimulationAnalysis.h

15.86 BioSpeciesManagerGuiExtensionPlugin Class Reference

Inheritance diagram for BioSpeciesManagerGuiExtensionPlugin:



Collaboration diagram for BioSpeciesManagerGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId ()` const override
- `std::string displayName ()` const override
- `std::string version ()` const override
- `std::vector< std::string > requiredModelPlugins ()` const override
- `void registerContributions (GuiExtensionRegistry *registry)` const override

15.86.1 Member Function Documentation

15.86.1.1 displayName()

```
std::string BioSpeciesManagerGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.86.1.2 extensionId()

```
std::string BioSpeciesManagerGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.86.1.3 registerContributions()

```
void BioSpeciesManagerGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.86.1.4 requiredModelPlugins()

```
std::vector< std::string > BioSpeciesManagerGuiExtensionPlugin::requiredModelPlugins ( ) const
[inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.86.1.5 version()

```
std::string BioSpeciesManagerGuiExtensionPlugin::version ( ) const [inline], [override],
[virtual]
```

Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

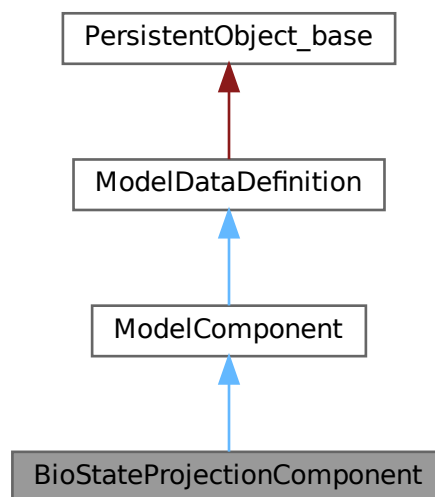
- applications/gui/genesys/extensions/BioSpeciesManagerGuiExtensionPlugin.cpp

15.87 BioStateProjectionComponent Class Reference

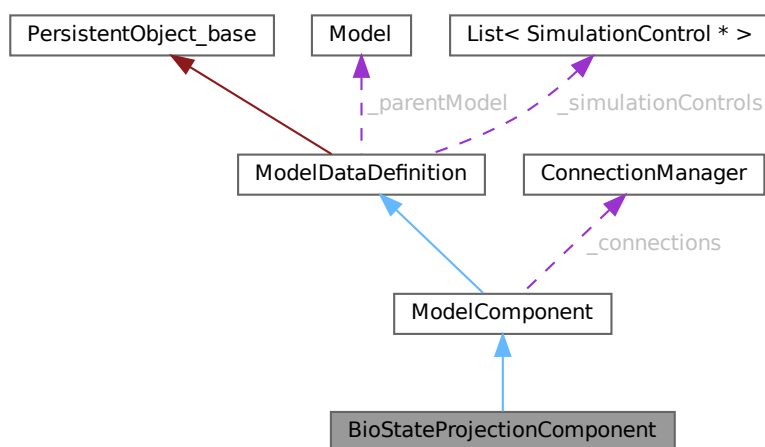
Projects selected [BioSpecies](#) values into a [WholeCellState](#).

```
#include <plugins/components/WholeCellModeling/BioStateProjectionComponent.h>
```

Inheritance diagram for BioStateProjectionComponent:



Collaboration diagram for BioStateProjectionComponent:



Classes

- struct [Projection](#)

Public Types

- enum class **ProjectionKind** { **MoleculeCount** , **MetaboliteAmount** }

Public Member Functions

- **BioStateProjectionComponent** ([Model](#) *model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **clearProjections** ()
- void **addMoleculeProjection** (const std::string &sourceSpeciesName, const std::string &targetKey="", double scale=1.0, double offset=0.0)
- void **addMetaboliteProjection** (const std::string &sourceSpeciesName, const std::string &targetKey="", double scale=1.0, double offset=0.0)
- unsigned int **getProjectionCount** () const
- void **setLastSucceeded** (bool lastSucceeded)
- bool **getLastSucceeded** () const
- void **setLastMessage** (std::string lastMessage)
- std::string **getLastMessage** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual **~ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component

- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

- Returns the simulation controls owned by this data definition.*
- `TraceManager::Level` `getTraceLevelSpecific` () const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` level)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` traceLevelspecific, bool traceLevelSpecific←
Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation` * `GetPluginInformation` ()
- static `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check` (`ModelComponent` *component)
Validates a component instance.
- static void `CreateInternalData` (`ModelComponent` *component)
Creates internal data required by a component instance.
- static void `SaveInstance` (`PersistenceRecord` *fields, `ModelComponent` *component)
Serializes a component instance into a persistence record.
- static void `DispatchEvent` (`Event` *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void `SaveInstance` (`PersistenceRecord` *fields, `ModelDataDefinition` *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool `Check` (`ModelDataDefinition` *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void `InitBetweenReplications` (`ModelDataDefinition` *modeldatum)
Resets simulation-time state between replications.
- static void `CreateInternalData` (`ModelDataDefinition` *modeldatum)
Creates internal and attached data required by this instance.
- static bool `CreateRelatedDataElements` (`ModelDataDefinition` *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_onDispatchEvent` ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void `_createEditableDataDefinitions` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * _connections = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification_id
- std::string _typename
- bool _reportStatistics
- bool _hasChanged
- unsigned int _modelLevel = 0
- [Model](#) * _parentModel
- [List](#)< [SimulationControl](#) * > * _simulationControls = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level _traceLevelSpecific
- bool _traceLevelSpecificEnabled = false

15.87.1 Detailed Description

Projects selected [BioSpecies](#) values into a [WholeCellState](#).

[BioStateProjectionComponent](#) bridges the biochemical and whole-cell layers by copying chosen [BioSpecies](#) amounts into [WholeCellState](#) molecule counts or metabolite pools at simulation event boundaries. It is intentionally small and explicit: the caller decides which source species feed which whole-cell fields and which scale/offset transform applies to each projection.

15.87.2 Member Function Documentation

15.87.2.1 _check()

```
bool BioStateProjectionComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.87.2.2 _createEditableDataDefinitions()

```
void BioStateProjectionComponent::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.87.2.3 _loadInstance()

```
bool BioStateProjectionComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.87.2.4 _onDispatchEvent()

```
void BioStateProjectionComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.87.2.5 _saveInstance()

```
void BioStateProjectionComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.87.2.6 show()

```
std::string BioStateProjectionComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

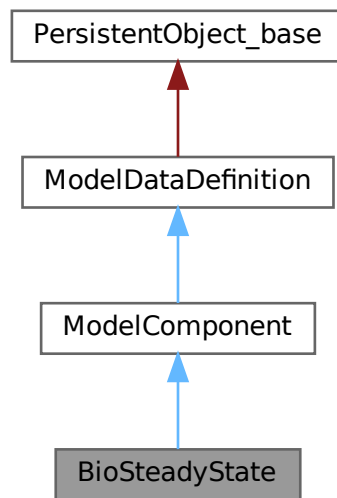
- plugins/components/WholeCellModeling/BioStateProjectionComponent.h
- plugins/components/WholeCellModeling/BioStateProjectionComponent.cpp

15.88 BioSteadyState Class Reference

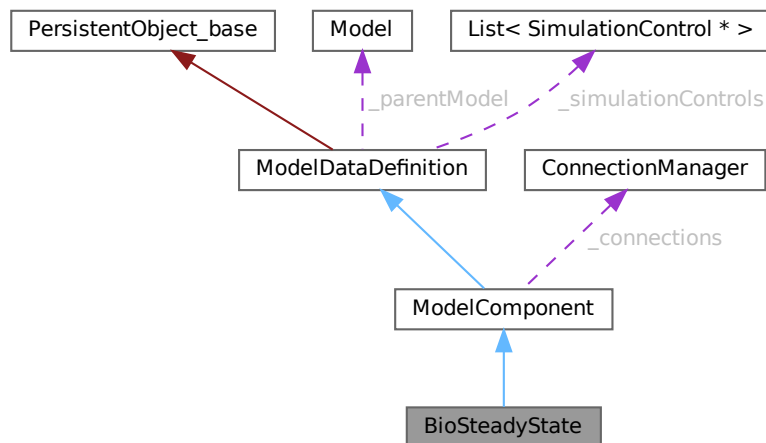
Component that checks steady-state diagnostics over [BioNetwork](#) results.

```
#include <plugins/components/BiochemicalSimulation/BioSteadyState.h>
```

Inheritance diagram for BioSteadyState:



Collaboration diagram for BioSteadyState:



Public Member Functions

- **BioSteadyState** ([Model](#) *model, std::string name="")
- void **setBioNetwork** ([BioNetwork](#) *bioNetwork)
- [BioNetwork](#) * **getBioNetwork** () const
- void **setTolerance** (double tolerance)
- double **getTolerance** () const
- void **setRunSimulationBeforeCheck** (bool runSimulationBeforeCheck)
- bool **getRunSimulationBeforeCheck** () const

- void **setLastSteady** (bool lastSteady)
- bool **getLastSteady** () const
- void **setLastMaxAbsoluteDerivative** (double lastMaxAbsoluteDerivative)
- double **getLastMaxAbsoluteDerivative** () const
- void **setLastMessage** (std::string lastMessage)
- std::string **getLastMessage** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentName, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

- Enables or disables report-statistics generation for this definition.*
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool **hasChanged**)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- **List**< **SimulationControl** * > * **getSimulationControls** () const
Returns the simulation controls owned by this data definition.
- **TraceManager::Level** **getTraceLevelSpecific** () const
Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (**TraceManager::Level** level)
Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (**TraceManager::Level** traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const
Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static **PluginInformation** * **GetPluginInformation** ()
- static **ModelComponent** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")

Static Public Member Functions inherited from **ModelComponent**

- static **ModelComponent** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** (**ModelComponent** *component)
Validates a component instance.
- static void **CreateInternalData** (**ModelComponent** *component)
Creates internal data required by a component instance.
- static void **SaveInstance** (**PersistenceRecord** *fields, **ModelComponent** *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** (**Event** *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createAttachedAttributes](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)

- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()

virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.88.1 Detailed Description

Component that checks steady-state diagnostics over [BioNetwork](#) results.

[BioSteadyState](#) can optionally trigger one simulation run before evaluating steady-state through [BioNetwork](#) analysis helpers.

15.88.2 Member Function Documentation

15.88.2.1 `_check()`

```
bool BioSteadyState::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.88.2.2 `_createAttachedAttributes()`

```
void BioSteadyState::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.88.2.3 `_loadInstance()`

```
bool BioSteadyState::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.88.2.4 `_onDispatchEvent()`

```
void BioSteadyState::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.88.2.5 `_saveInstance()`

```
void BioSteadyState::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.88.2.6 `show()`

```
std::string BioSteadyState::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/BiochemicalSimulation/BioSteadyState.h
- plugins/components/BiochemicalSimulation/BioSteadyState.cpp

15.89 BioSteadyStateCheck Struct Reference

Public Attributes

- bool **steady** = false
- double **maxAbsoluteDerivative** = 0.0
- double **tolerance** = 0.0
- std::vector< [BioSpeciesDerivative](#) > **derivatives**

The documentation for this struct was generated from the following file:

- tools/Biochemical/BioSimulationAnalysis.h

15.90 BioStoichiometryMatrix Struct Reference

Public Member Functions

- double **coefficient** (unsigned int speciesIndex, unsigned int reactionIndex) const

Public Attributes

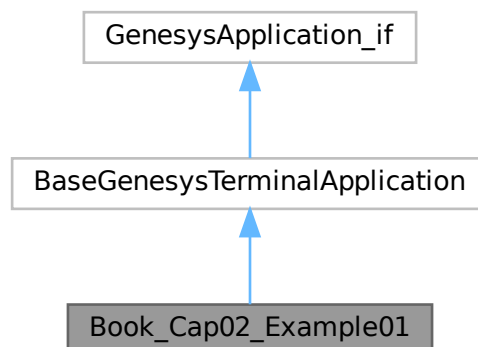
- std::vector< std::string > **speciesNames**
- std::vector< std::string > **reactionNames**
- std::vector< std::vector< double > > **coefficients**

The documentation for this struct was generated from the following file:

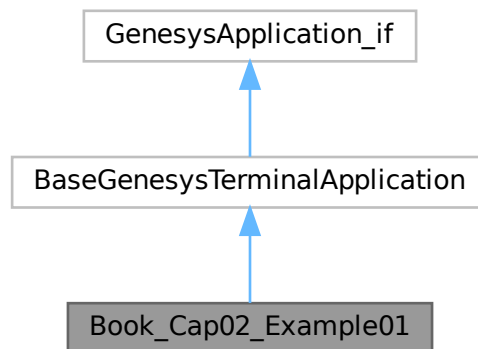
- tools/Biochemical/BioSimulationAnalysis.h

15.91 Book_Cap02_Example01 Class Reference

Inheritance diagram for Book_Cap02_Example01:



Collaboration diagram for Book_Cap02_Example01:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

15.91.1 Member Function Documentation

15.91.1.1 main()

```
int Book_Cap02_Example01::main (
    int argc,
    char ** argv ) [override], [virtual]
```

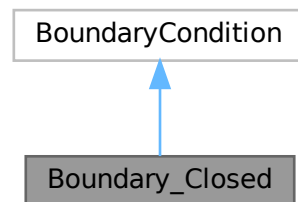
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

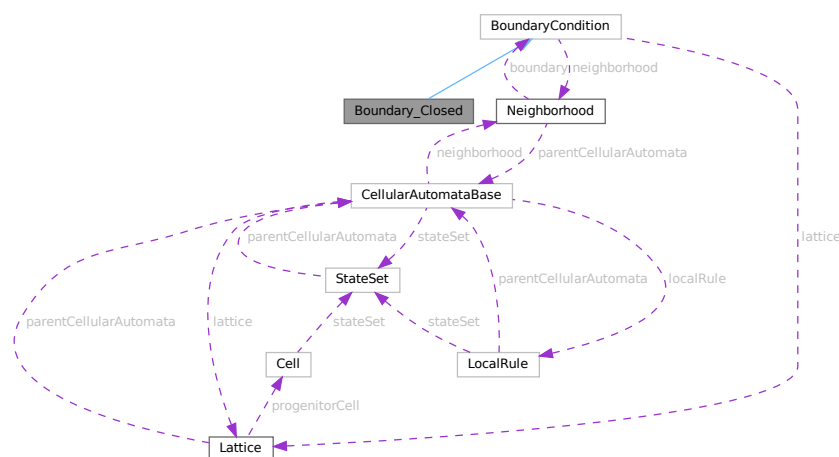
- applications/modelSpecific/book/Book_Cap02_Example01.h
- applications/modelSpecific/book/Book_Cap02_Example01.cpp

15.92 Boundary_Closed Class Reference

Inheritance diagram for Boundary_Closed:



Collaboration diagram for Boundary_Closed:



Public Member Functions

- **Boundary_Closed** ([Lattice](#) *lattice=nullptr, [Neighborhood](#) *neighborhood=nullptr)
- **Boundary_Closed** (const [Boundary_Closed](#) &orig)
- virtual [Cell](#) * [getNeighborCell](#) (std::vector< int > cellNDimPosition, std::vector< int > neighborNDimPosition) override

Pure virtual method that has to be overriden by derived class, responsible for defining the missing neighbor of a cell.

- void **setFixedState** (const [State](#) &fixedState)
- const [State](#) & **getFixedState** () const

Public Member Functions inherited from [BoundaryCondition](#)

- **BoundaryCondition** ([Lattice](#) *lattice=nullptr, [Neighborhood](#) *neighborhood=nullptr)
- **BoundaryCondition** (const [BoundaryCondition](#) &orig)
- virtual std::string **show** ()
- void **setNeighborhood** ([Neighborhood](#) *neighborhood)
- [Neighborhood](#) * **getNeighborhood** () const
- void **setLattice** ([Lattice](#) *lattice)
- [Lattice](#) * **getLattice** () const

Additional Inherited Members

Protected Attributes inherited from [BoundaryCondition](#)

- [Lattice](#) * **lattice**
- [Neighborhood](#) * **neighborhood**

15.92.1 Member Function Documentation

15.92.1.1 getNeighborCell()

```
virtual Cell * Boundary_Closed::getNeighborCell (
    std::vector< int > cellPosition,
    std::vector< int > neighborCellPosition ) [inline], [override], [virtual]
```

Pure virtual method that has to be overriden by derived class, responsible for defining the missing neighbor of a cell.

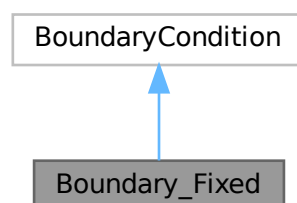
Implements [BoundaryCondition](#).

The documentation for this class was generated from the following file:

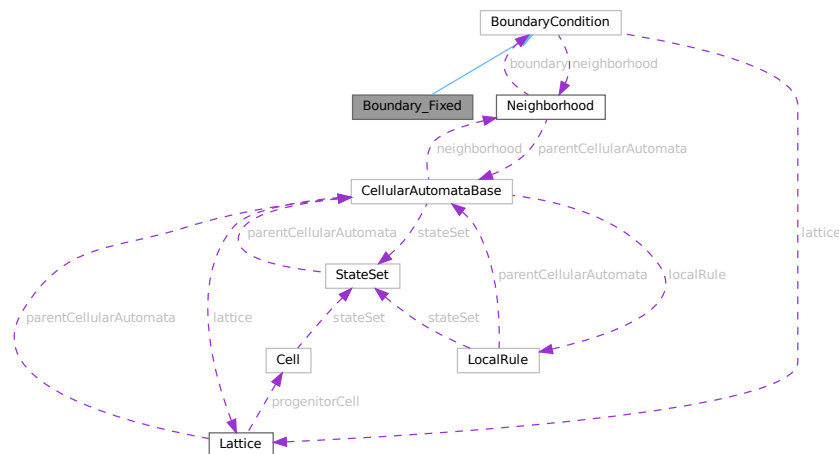
- plugins/components/ModalModel/CellularAutomata/Boundary_Closed.h

15.93 Boundary_Fixed Class Reference

Inheritance diagram for [Boundary_Fixed](#):



Collaboration diagram for `Boundary_Fixed`:



Public Member Functions

- **Boundary_Fixed** (`Lattice *lattice=nullptr`, `Neighborhood *neighborhood=nullptr`)
- **Boundary_Fixed** (`const Boundary_Fixed &orig`)
- virtual `Cell * getNeighborCell (std::vector< int > cellIndimPosition, std::vector< int > neighborNDimPosition)`
override
Pure virtual method that has to be overiden by derived class, responsible for defining the missing neighbor of a cell.
- void **setFixedState** (`const State &fixedState`)
- const `State & getFixedState () const`

Public Member Functions inherited from `BoundaryCondition`

- **BoundaryCondition** (`Lattice *lattice=nullptr`, `Neighborhood *neighborhood=nullptr`)
- **BoundaryCondition** (`const BoundaryCondition &orig`)
- virtual `std::string show ()`
- void **setNeighborhood** (`Neighborhood *neighborhood`)
- `Neighborhood * getNeighborhood () const`
- void **setLattice** (`Lattice *lattice`)
- `Lattice * getLattice () const`

Additional Inherited Members

Protected Attributes inherited from `BoundaryCondition`

- `Lattice * lattice`
- `Neighborhood * neighborhood`

15.93.1 Member Function Documentation

15.93.1.1 `getNeighborCell()`

```

virtual Cell * Boundary_Fixed::getNeighborCell (
    std::vector< int > cellPosition,
    std::vector< int > neighborCellPosition ) [inline], [override], [virtual]

```

Pure virtual method that has to be overiden by derived class, responsible for defining the missing neighbor of a cell.

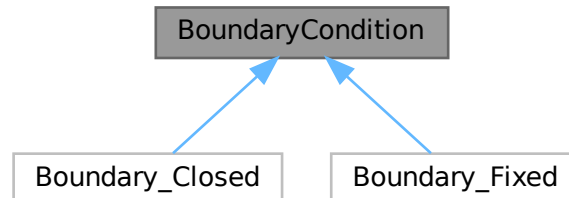
Implements `BoundaryCondition`.

The documentation for this class was generated from the following file:

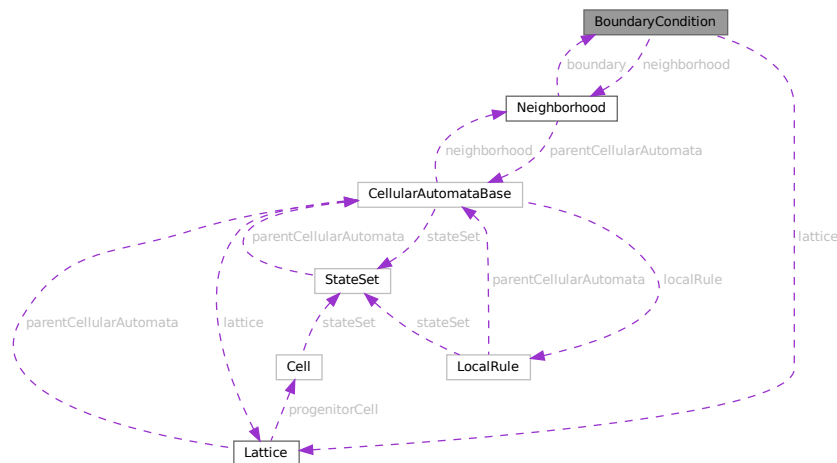
- plugins/components/ModalModel/CellularAutomata/Boundary_Fixed.h

15.94 BoundaryCondition Class Reference

Inheritance diagram for BoundaryCondition:



Collaboration diagram for BoundaryCondition:



Public Member Functions

- **BoundaryCondition** (**Lattice** *lattice=nullptr, **Neighborhood** *neighborhood=nullptr)
- **BoundaryCondition** (const **BoundaryCondition** &orig)
- virtual std::string **show** ()
- virtual **Cell** * **getNeighborCell** (std::vector< int > cellPosition, std::vector< int > neighborCellPosition)=0
Pure virtual method that has to be overriden by derived class, responsible for defining the missing neighbor of a cell.
- void **setNeighborhood** (**Neighborhood** *neighborhood)
- **Neighborhood** * **getNeighborhood** () const
- void **setLattice** (**Lattice** *lattice)
- **Lattice** * **getLattice** () const

Protected Attributes

- **Lattice** * **lattice**
- **Neighborhood** * **neighborhood**

15.94.1 Member Function Documentation

15.94.1.1 getNeighborCell()

```
virtual Cell * BoundaryCondition::getNeighborCell (
    std::vector< int > cellPosition,
    std::vector< int > neighborCellPosition ) [pure virtual]
```

Pure virtual method that has to be overriden by derived class, responsible for defining the missing neighbor of a cell.

Implemented in [Boundary_Closed](#), and [Boundary_Fixed](#).

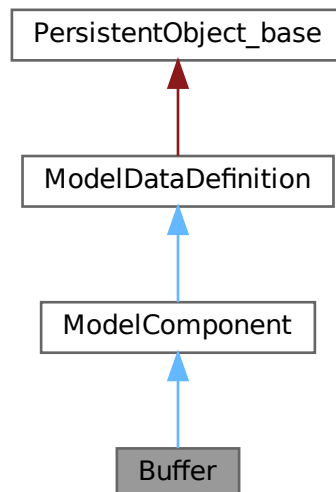
The documentation for this class was generated from the following file:

- `plugins/components/ModalModel/CellularAutomata/BoundaryCondition.h`

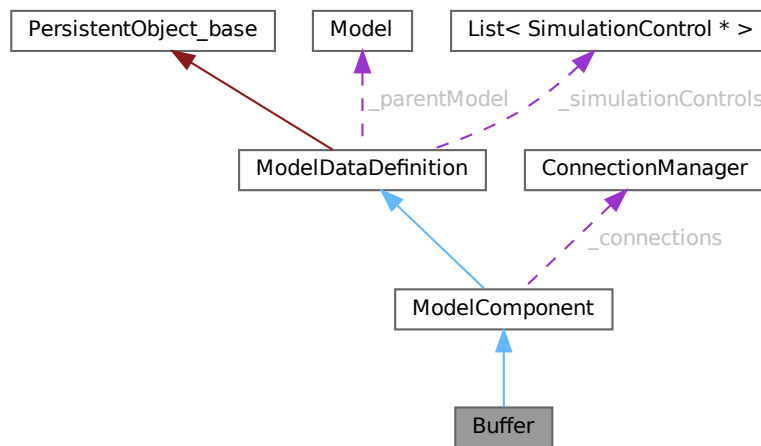
15.95 Buffer Class Reference

```
#include <plugins/components/DiscreteProcessing/Buffer.h>
```

Inheritance diagram for Buffer:



Collaboration diagram for Buffer:



Public Types

- enum class **AdvanceOn** : int { **NewArrivals** = 0 , **Signal** = 1 , **num_elements** = 2 }
- enum class **ArrivalOnFullBufferRule** : int { **Dispose** = 0 , **SendToBulkPort** = 1 , **ReplaceLastPosition** = 2 , **num_elements** = 3 }

Public Member Functions

- **Buffer** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
- *virtual public methods*
- Buffer::ArrivalOnFullBufferRule **getarrivalOnFullBufferRule** () const
- void **setArrivalOnFullBufferRule** (Buffer::ArrivalOnFullBufferRule newArrivalOnFullBufferRule)
- Buffer::AdvanceOn **getadvanceOn** () const
- void **setAdvanceOn** (Buffer::AdvanceOn newAdvanceOn)
- unsigned int **getcapacity** () const
- void **setCapacity** (unsigned int newCapacity)
- [SignalData](#) * **getsignal** () const
- void **setSignal** ([SignalData](#) *newSignal)

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentName, std::string name="")
- *constructors*
- virtual ~**ModelComponent** ()
- *Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * **getConnectionManager** () const
- *new public user methods for this component*
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
- *Connects this component to another component.*
- bool **hasBreakpointAt** ()
- *Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string _description)

Updates the free-form description shown by the GUI.

- `std::string getDescription () const`

Returns the free-form description associated with this component.

- `void remove (ModelComponent *comp)`

Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.

- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.

- Util::identification getId () const

Returns the unique identifier of this instance.

- void setName (const std::string &name)

Updates the logical name of this instance.

- const std::string & getName () const

Returns the logical name of this instance.

- void setLabel (const std::string &label)

Updates the user-visible label of this instance.

- const std::string & getLabel () const

Returns the user-visible label of this instance.

- const std::string & getClassname () const

Returns the runtime class name used by the plugin system.

- **Model** * getParentModel () const

Returns the parent model that owns this definition.

- bool isReportStatistics () const

Indicates whether this data definition contributes report statistics.

- void setReportStatistics (bool reportStatistics)

Enables or disables report-statistics generation for this definition.

- **ModelDataDefinition** * getInternalData (const std::string &name) const

Looks up a named internal child data definition.

- std::map< std::string, **ModelDataDefinition** * > * getInternalData () const

Returns the full map of internal child data definitions.

- std::map< std::string, **ModelDataDefinition** * > * getAttachedData () const

Returns the full map of attached data definitions.

- bool hasChanged () const

Indicates whether this instance has been modified since the last reset.

- void setHasChanged (bool hasChanged)

Updates the changed flag for this data definition.

- unsigned int getLevel () const

Returns the level in the model hierarchy where this definition lives.

- void setModelLevel (unsigned int _modelLevel)

Sets the model nesting level for this definition.

- List< **SimulationControl** * > * getSimulationControls () const

Returns the simulation controls owned by this data definition.

- TraceManager::Level getTraceLevelSpecific () const

Returns the trace level that applies specifically to this definition.

- void setTraceLevelSpecific (TraceManager::Level level)

Updates the trace level used by this definition.

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static std::string [convertEnumToStr](#) (AdvanceOn advance)
- static std::string [convertEnumToStr](#) (ArrivalOnFullBufferRule arrival)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void `_onDispatchEvent` ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` () override
- virtual void `_initBetweenReplications` () override
- virtual void `_addSimulationControl` ([SimulationControl](#) *property) override
- virtual void `_createEditableDataDefinitions` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.95.1 Detailed Description

This component ...

15.95.2 Member Function Documentation

15.95.2.1 [_addSimulationControl\(\)](#)

```
void Buffer::_addSimulationControl (
    SimulationControl * property ) [override], [protected], [virtual]
```

This method is necessary only for those components that instantiate internal elements that must exist before simulation starts and even before model checking. That's the case of components that have internal StatisticsCollectors, since others components may refer to them as expressions (as in "TVAG(ThisCSTAT)") and therefore the model datum must exist before checking such expression

Reimplemented from [ModelDataDefinition](#).

15.95.2.2 [_check\(\)](#)

```
bool Buffer::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.95.2.3 _createEditableDataDefinitions()

void Buffer::_createEditableDataDefinitions () [override], [protected], [virtual]
 Reimplemented from [ModelDataDefinition](#).

15.95.2.4 _getParserChangesInformation()

[ParserChangesInformation](#) * Buffer::_getParserChangesInformation () [override], [protected], [virtual]

This method returns all changes in the parser that are needed by plugins of this ModelData. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dynamically linked to to simulator kernel to reflect the changes
 Reimplemented from [ModelDataDefinition](#).

15.95.2.5 _initBetweenReplications()

void Buffer::_initBetweenReplications () [override], [protected], [virtual]
 Reimplemented from [ModelDataDefinition](#).

15.95.2.6 _loadInstance()

bool Buffer::_loadInstance (
 [PersistenceRecord](#) * fields) [override], [protected], [virtual]
 virtual protected method that must be overridden
 Reimplemented from [ModelComponent](#).

15.95.2.7 _onDispatchEvent()

void Buffer::_onDispatchEvent (
 [Entity](#) * entity,
 unsigned int inputPortNumber) [override], [protected], [virtual]

Handles a dispatched event for this component type.
 This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.95.2.8 _saveInstance()

void Buffer::_saveInstance (
 [PersistenceRecord](#) * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
 Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.95.2.9 show()

std::string Buffer::show () [override], [virtual]
 virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

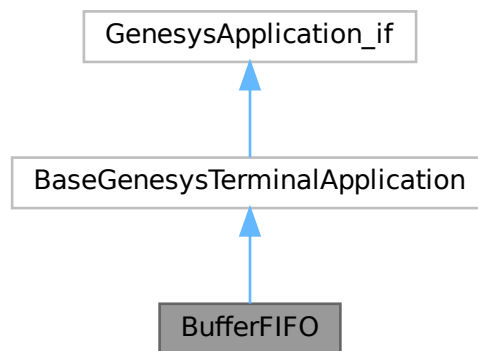
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

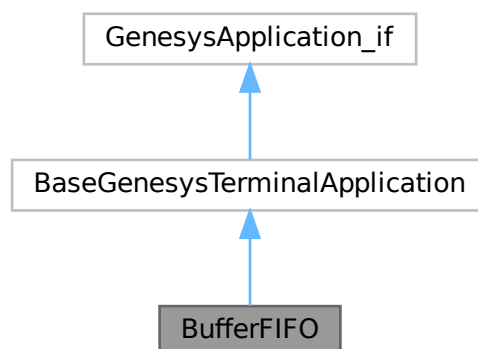
- `plugins/components/DiscreteProcessing/Buffer.h`
- `plugins/components/DiscreteProcessing/Buffer.cpp`

15.96 BufferFIFO Class Reference

Inheritance diagram for BufferFIFO:



Collaboration diagram for BufferFIFO:



Public Member Functions

- virtual int [main](#) (int argc, char **argv)

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

15.96.1 Member Function Documentation

15.96.1.1 main()

```
virtual int BufferFIFO::main (
    int argc,
    char ** argv ) [inline], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following file:

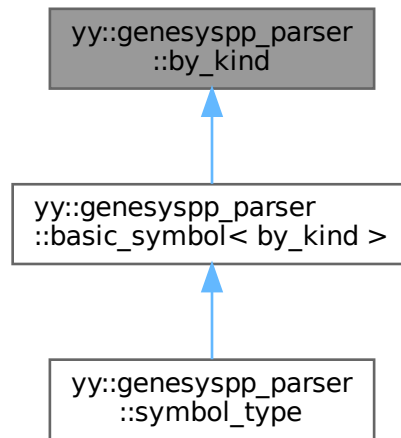
- applications/modelSpecific/teaching/BufferFIFO.h

15.97 yy::genesyspp_parser::by_kind Struct Reference

Type access provider for token (enum) based symbols.

```
#include <parser/GenesysParser.h>
```

Inheritance diagram for yy::genesyspp_parser::by_kind:



Public Types

- typedef [token_kind_type](#) **kind_type**
The symbol kind as needed by the constructor.

Public Member Functions

- **by_kind** () YY_NOEXCEPT
Default constructor.
- **by_kind** (const [by_kind](#) &that) YY_NOEXCEPT
Copy constructor.
- **by_kind** ([kind_type](#) t) YY_NOEXCEPT
Constructor from (external) token numbers.
- void **clear** () YY_NOEXCEPT
[Record](#) that this symbol is empty.
- void **move** ([by_kind](#) &that)
Steal the symbol kind from that.
- [symbol_kind_type](#) **kind** () const YY_NOEXCEPT
- [symbol_kind_type](#) **type_get** () const YY_NOEXCEPT
Backward compatibility (Bison 3.6).

Public Attributes

- [symbol_kind_type](#) **kind_**

15.97.1 Detailed Description

Type access provider for token (enum) based symbols.

15.97.2 Member Function Documentation

15.97.2.1 kind()

`genesyspp_parser::symbol_kind_type yy::genesyspp_parser::by_kind::kind ( ) const [inline]`

The (internal) type number (corresponding to *type*). *empty* when empty.

15.97.3 Member Data Documentation

15.97.3.1 kind_

`symbol_kind_type yy::genesyspp_parser::by_kind::kind_`

The symbol kind. *S_YYEMPTY* when empty.

The documentation for this struct was generated from the following file:

- parser/[GenesysParser.h](#)

15.98 QCPAxisPainterPrivate::CachedLabel Struct Reference

Public Attributes

- QPointF **offset**
- QPixmap **pixmap**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/qcustomplot.h

15.99 QCPLabelPainterPrivate::CachedLabel Struct Reference

Public Attributes

- QPoint **offset**
- QPixmap **pixmap**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/qcustomplot.h

15.100 ModelLifecycleController::Callbacks Struct Reference

Public Attributes

- std::function< void(const std::string &)> **insertCommandInConsole**
- std::function< void([Model](#) *)> **initUiForNewModel**
- std::function< void()> **actualizeActions**
- std::function< void()> **actualizeTabPanels**
- std::function< void(bool)> **actualizeModelTextHasChanged**
- std::function< bool()> **checkModel**
- std::function< bool()> **setSimulationModelBasedOnText**
- std::function< void()> **clearModelEditors**
- std::function< bool(QString)> **saveGraphicalModel**
- std::function< bool(QFile *, QString)> **saveTextModel**
- std::function< [Model](#) *(std::string)> **loadGraphicalModel**
- std::function< void()> **connectSceneSignals**
- std::function< void(const char *)> **disconnectSceneSignals**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/controllers/ModelLifecycleController.h

15.101 SimulationEventController::Callbacks Struct Reference

Public Attributes

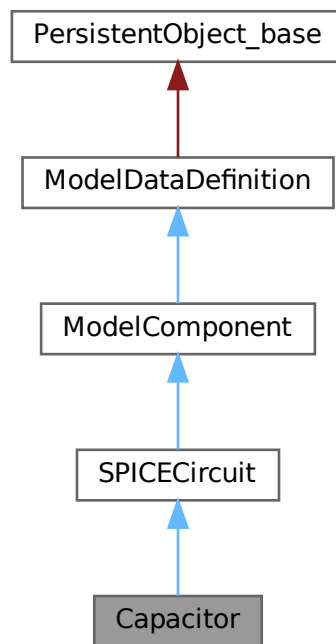
- `std::function< void()>` **actualizeActions**
- `std::function< void(SimulationEvent *)>` **actualizeSimulationEvents**
- `std::function< void(bool)>` **actualizeDebugEntities**
- `std::function< void(bool)>` **actualizeDebugVariables**
- `std::function< void(SimulationEvent *)>` **actualizeGraphicalModel**

The documentation for this struct was generated from the following file:

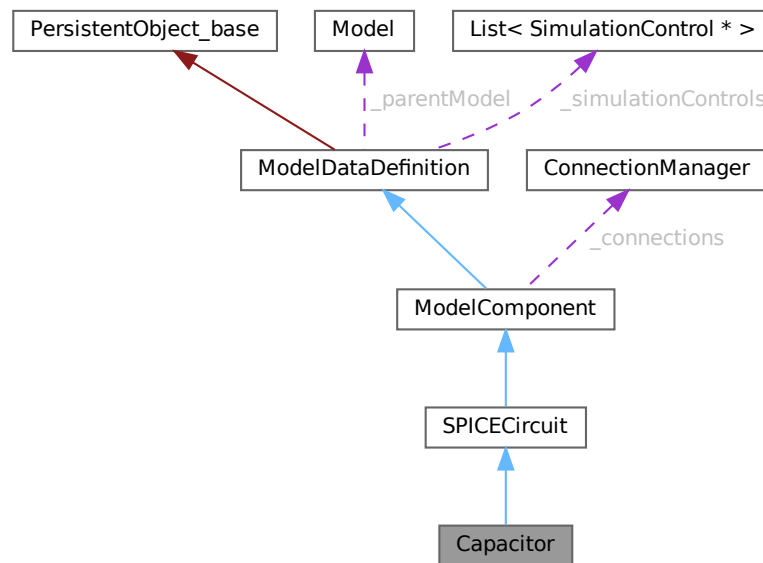
- `applications/gui/genesys/controllers/SimulationEventController.h`

15.102 Capacitor Class Reference

Inheritance diagram for Capacitor:



Collaboration diagram for Capacitor:



Public Member Functions

- **Capacitor** ([Model](#) *model, std::string name="")
- void **setCapacitance** (float capacitance)
- void **Build** () override

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)

Updates the free-form description shown by the GUI.

- `std::string getDescription () const`

Returns the free-form description associated with this component.

- `void remove (ModelComponent *comp)`

Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- `ModelDataDefinition (Model *model, std::string datadefinitionTypename, std::string name="", bool insertIntoModel=true)`

Creates a model data definition and optionally inserts it into the model.

- `virtual ~ModelDataDefinition ()`

Releases resources owned by this data definition.

- `Util::identification getId () const`

Returns the unique identifier of this instance.

- `void setName (const std::string &name)`

Updates the logical name of this instance.

- `const std::string & getName () const`

Returns the logical name of this instance.

- `void setLabel (const std::string &label)`

Updates the user-visible label of this instance.

- `const std::string & getLabel () const`

Returns the user-visible label of this instance.

- `const std::string & getClassname () const`

Returns the runtime class name used by the plugin system.

- `Model * getParentModel () const`

Returns the parent model that owns this definition.

- `bool isReportStatistics () const`

Indicates whether this data definition contributes report statistics.

- `void setReportStatistics (bool reportStatistics)`

Enables or disables report-statistics generation for this definition.

- `ModelDataDefinition * getInternalData (const std::string &name) const`

Looks up a named internal child data definition.

- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`

Returns the full map of internal child data definitions.

- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`

Returns the full map of attached data definitions.

- `bool hasChanged () const`

Indicates whether this instance has been modified since the last reset.

- `void setHasChanged (bool hasChanged)`

Updates the changed flag for this data definition.

- `unsigned int getLevel () const`

Returns the level in the model hierarchy where this definition lives.

- `void setModelLevel (unsigned int _modelLevel)`

Sets the model nesting level for this definition.

- `List< SimulationControl * > * getSimulationControls () const`

Returns the simulation controls owned by this data definition.

- `TraceManager::Level getTraceLevelSpecific () const`

Returns the trace level that applies specifically to this definition.

- `void setTraceLevelSpecific (TraceManager::Level level)`

Updates the trace level used by this definition.

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation](#) * [GetPluginInformation](#) ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
- static std::vector< std::string > [split](#) (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [SPICECircuit](#)

- void `BuildCircuit` (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void `UpdateConnections` ()
- virtual void `_onDispatchEvent` ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void `_createNonEditableDataDefinitions` () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual bool `_check` (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)

- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.102.1 Member Function Documentation

15.102.1.1 **_loadInstance()**

```
bool Capacitor::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

15.102.1.2 _saveInstance()

```
void Capacitor::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

15.102.1.3 Build()

```
void Capacitor::Build ( ) [override], [virtual]
```

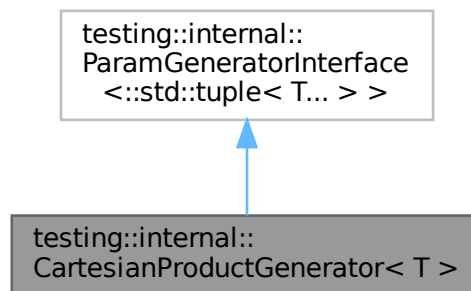
Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

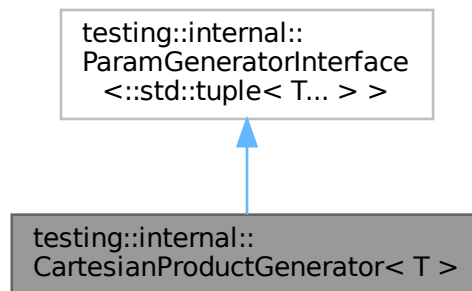
- plugins/components/ElectronicsSimulation/SPICECircuit.h
- plugins/components/ElectronicsSimulation/SPICECircuit.cpp

15.103 testing::internal::CartesianProductGenerator< T > Class Template Reference

Inheritance diagram for testing::internal::CartesianProductGenerator< T >:



Collaboration diagram for `testing::internal::CartesianProductGenerator< T >`:



Public Types

- `typedef ::std::tuple< T... > ParamType`

Public Types inherited from

[testing::internal::ParamGeneratorInterface<::std::tuple< T... > >](#)

- `typedef ::std::tuple< T... > ParamType`

Public Member Functions

- **CartesianProductGenerator** (const `std::tuple< ParamGenerator< T >... > &g`)
- [ParamIteratorInterface< ParamType > * Begin](#) () const override
- [ParamIteratorInterface< ParamType > * End](#) () const override

15.103.1 Member Function Documentation

15.103.1.1 Begin()

```

template<typename... T>
ParamIteratorInterface< ParamType > \* testing::internal::CartesianProductGenerator< T >::↔
Begin ( ) const [inline], [override], [virtual]
Implements testing::internal::ParamGeneratorInterface<::std::tuple< T... > >.
  
```

15.103.1.2 End()

```

template<typename... T>
ParamIteratorInterface< ParamType > \* testing::internal::CartesianProductGenerator< T >::End
( ) const [inline], [override], [virtual]
Implements testing::internal::ParamGeneratorInterface<::std::tuple< T... > >.
  
```

The documentation for this class was generated from the following file:

- `gtest/gtest/internal/gtest-param-util.h`

15.104 testing::internal::CartesianProductHolder< Gen > Class Template Reference

Public Member Functions

- **CartesianProductHolder** (const `Gen &... g`)

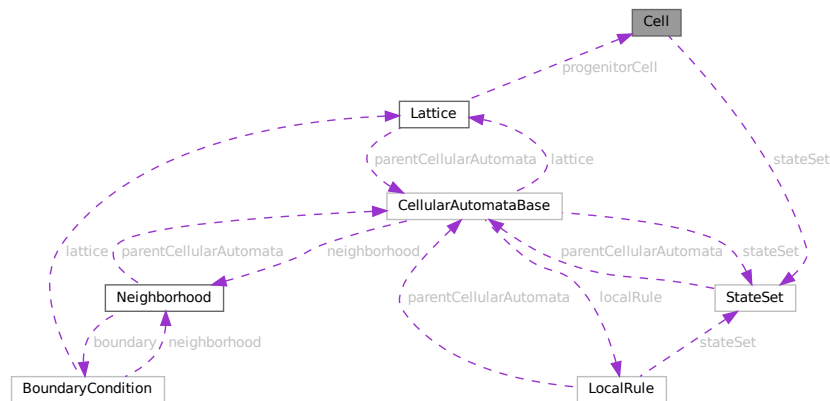
- `template<typename... T>`
operator ParamGenerator<::std::tuple< T... > > () const

The documentation for this class was generated from the following file:

- `gtest/gtest/internal/gtest-param-util.h`

15.105 Cell Class Reference

Collaboration diagram for Cell:



Public Member Functions

- **Cell** ([StateSet](#) *stateSet=nullptr, long cellNum=0, std::vector< int > position={})
- **Cell** (long cellNum, std::vector< int > position)
- **Cell** (const [Cell](#) &orig)
- [Cell](#) & **operator=** (const [Cell](#) &orig)
- virtual std::string **show** ()
- bool **updateState** ()
update cell state (currentState = nextState)
- bool **isUpdatePending** ()
true if nextState differs from currentState
- virtual void **draw** ()
virtual method that can be override by derived classes
- void **setCellNumber** (long cellNum)
cell rank position in the lattice vector
- long **getCellNumber** () const
- void **setPosition** (std::vector< int > position)
- std::vector< int > **getPosition** () const
n-dimensional position
- void **setStateSet** ([StateSet](#) *stateSet)
- [StateSet](#) * **getStateSet** () const
- const [State](#) & **getPreviousState** () const
- bool **setCurrentState** (const [State](#) ¤tState)
- const [State](#) & **getCurrentState** () const
- bool **setNextState** (const [State](#) &nextState)
- const [State](#) & **getNextState** () const
- void **setNeighbors** (std::vector< [Cell](#) * > neighbors)
- std::vector< [Cell](#) * > **getNeighbors** () const

Protected Member Functions

- bool **acceptsState** (const [State](#) &state) const

Protected Attributes

- std::unique_ptr< [State](#) > **previousState**
- std::unique_ptr< [State](#) > **currentState**
- std::unique_ptr< [State](#) > **nextState**
- [StateSet](#) * **stateSet** = nullptr
- bool **updatePending**
- std::vector< int > **position**
- long **cellNum**
- std::vector< [Cell](#) * > **neighbors**

The documentation for this class was generated from the following files:

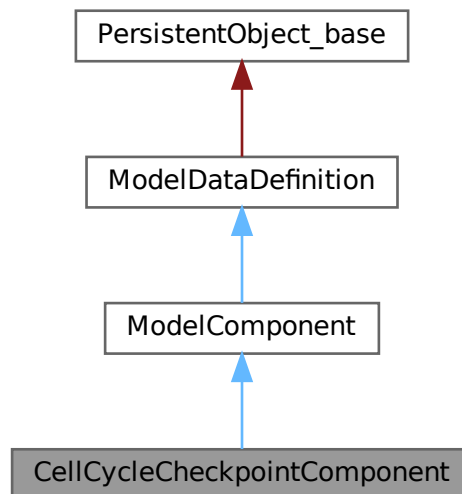
- plugins/components/ModalModel/CellularAutomata/Cell.h
- plugins/components/ModalModel/CellularAutomata/Cell.cpp

15.106 CellCycleCheckpointComponent Class Reference

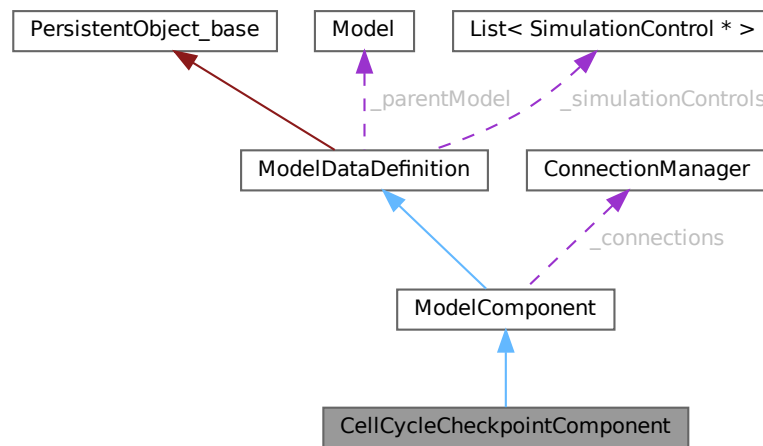
Evaluates lifecycle checkpoints and advances whole-cell step time.

```
#include <plugins/components/WholeCellModeling/CellCycleCheckpointComponent.h>
```

Inheritance diagram for CellCycleCheckpointComponent:



Collaboration diagram for CellCycleCheckpointComponent:



Public Member Functions

- **CellCycleCheckpointComponent** ([Model](#) *model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **setDeltaT** (double deltaT)
- double **getDeltaT** () const
- void **setAdvanceWholeCellClock** (bool advance)
- bool **getAdvanceWholeCellClock** () const
- void **setEnergyMetaboliteKey** (std::string key)
- std::string **getEnergyMetaboliteKey** () const
- void **setCompartmentEnergyRegion** (std::string region)
- std::string **getCompartmentEnergyRegion** () const
- void **setCompartmentEnergyMetaboliteKey** (std::string key)
- std::string **getCompartmentEnergyMetaboliteKey** () const
- void **setStarvationAtpThreshold** (double threshold)
- double **getStarvationAtpThreshold** () const
- void **setCompartmentStarvationThreshold** (double threshold)
- double **getCompartmentStarvationThreshold** () const
- void **setLethalStarvationSteps** (unsigned int steps)
- unsigned int **getLethalStarvationSteps** () const
- void **setDivisionMassThreshold** (double threshold)
- double **getDivisionMassThreshold** () const
- void **setFtsZRingKey** (std::string key)
- std::string **getFtsZRingKey** () const
- void **setFtsZThreshold** (double threshold)
- double **getFtsZThreshold** () const
- void **setCriticalPathwayActivityKey** (std::string key)
- std::string **getCriticalPathwayActivityKey** () const
- void **setCriticalPathwayActivityThreshold** (double threshold)
- double **getCriticalPathwayActivityThreshold** () const
- unsigned int **getStarvationStreak** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.

- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int [_modelLevel](#))
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) [traceLevelspecific](#), bool [traceLevelSpecificEnabled](#)=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool [traceLevelSpecificEnabled](#))
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)

Creates internal and attached data required by this instance.

- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)

Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()

- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.106.1 Detailed Description

Evaluates lifecycle checkpoints and advances whole-cell step time.

[CellCycleCheckpointComponent](#) centralizes two concerns that otherwise become scattered across individual submodels:

- explicit advancement of [WholeCellState](#) current time and step count; and
- annotation of a coarse lifecycle phase based on energy availability, division readiness, and viability.

The component does not execute metabolism or division itself. It only reads [WholeCellState](#) and writes back lifecycle metadata such as:

- LifecyclePhase = newborn, growth, division_ready, starved, dead
- Viable

This keeps [WholeCellState](#) as a passive data container and concentrates the event-boundary checkpoint logic in a dedicated [ModelComponent](#).

15.106.2 Member Function Documentation

15.106.2.1 `_check()`

```
bool CellCycleCheckpointComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.106.2.2 `_createEditableDataDefinitions()`

```
void CellCycleCheckpointComponent::_createEditableDataDefinitions ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.106.2.3 `_initBetweenReplications()`

```
void CellCycleCheckpointComponent::_initBetweenReplications ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.106.2.4 `_loadInstance()`

```
bool CellCycleCheckpointComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.106.2.5 `_onDispatchEvent()`

```
void CellCycleCheckpointComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.106.2.6 `_saveInstance()`

```
void CellCycleCheckpointComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.106.2.7 show()

```
std::string CellCycleCheckpointComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

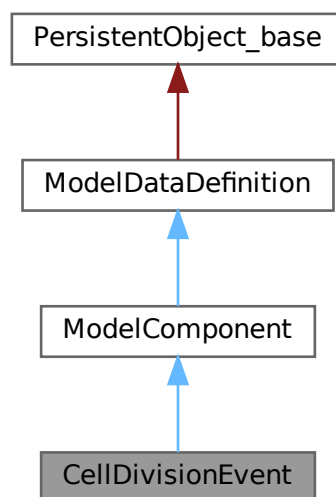
The documentation for this class was generated from the following files:

- plugins/components/WholeCellModeling/CellCycleCheckpointComponent.h
- plugins/components/WholeCellModeling/CellCycleCheckpointComponent.cpp

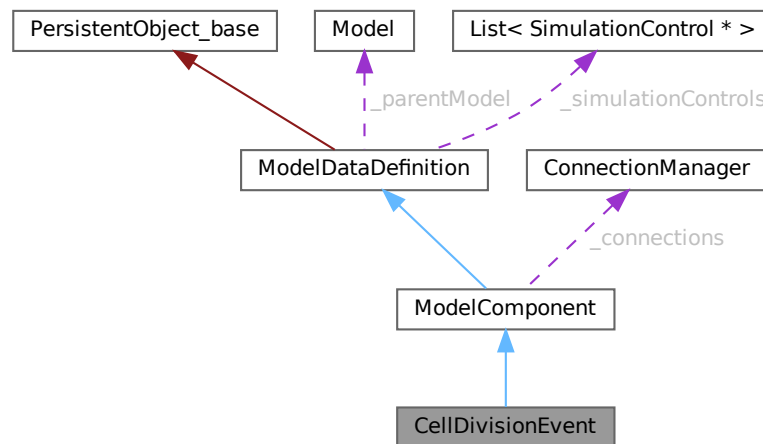
15.107 CellDivisionEvent Class Reference

```
#include <plugins/components/WholeCellModeling/CellDivisionEvent.h>
```

Inheritance diagram for CellDivisionEvent:



Collaboration diagram for CellDivisionEvent:



Public Member Functions

- **CellDivisionEvent** ([Model](#) *model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **setDivisionMassThreshold** (double threshold)
- double **getDivisionMassThreshold** () const
- void **setFtsZThreshold** (double threshold)
- double **getFtsZThreshold** () const
- void **setFtsZRingKey** (std::string key)
- std::string **getFtsZRingKey** () const
- void **setRandomSeed** (unsigned int seed)
- unsigned int **getRandomSeed** () const
- bool **getLastDivided** () const
- int **getDivisionCount** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.

- `std::string getDescription () const`
Returns the free-form description associated with this component.
- `void remove (ModelComponent *comp)`
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const
Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)
Updates the trace level used by this definition.

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

- virtual void `_initBetweenReplications ()` override
- virtual void `_onDispatchEvent (Entity *entity, unsigned int inputPortNumber)` override
Handles a dispatched event for this component type.
- virtual void `_createEditableDataDefinitions ()` override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption ()`
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation ()`
- virtual void `_addSimulationControl (SimulationControl *control)`
- virtual void `_createInternalAndAttachedData ()`
- virtual void `_createInternalStatisticReporters ()`
- virtual void `_createAttachedAttributes ()`
- virtual void `_createNonEditableDataDefinitions ()`
- void `_templateCreateInternalStatisticReporters ()`
- void `_templateCreateAttachedAttributes ()`
- void `_templateCreateNonEditableDataDefinitions ()`
- void `_templateCreateEditableDataDefinitions ()`
- void `_internalStatisticReportersClear ()`
legacy helper methods used by the granular create hooks*
- void `_internalDataClear ()`
- void `_internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- void `_internalDataRemove (const std::string &key)`
- void `_attachedDataClear ()`
- void `_attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- void `_attachedDataRemove (const std::string &key)`
- void `_attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- void `_statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- void `_statisticReporterRemove (const std::string &key)`
- void `_statisticReportersClear ()`
- void `_mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- void `_mandatoryAttachedAttributeRemove (const std::string &key)`
- void `_mandatoryAttachedAttributesClear ()`
- void `_mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- void `_mandatoryEditableDataDefinitionRemove (const std::string &key)`
- void `_mandatoryEditableDataDefinitionsClear ()`
- void `_optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- void `_optionalEditableDataDefinitionRemove (const std::string &key)`
- void `_optionalEditableDataDefinitionsClear ()`
- void `_mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- void `_mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- void `_mandatoryNonEditableDataDefinitionsClear ()`
- void `_optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- void `_optionalNonEditableDataDefinitionRemove (const std::string &key)`
- void `_optionalNonEditableDataDefinitionsClear ()`
- void `_checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void `trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void `traceError (const std::string &text, const std::exception &e)`
- void `traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)`
- void `traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)`

- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.107.1 Detailed Description

Discrete cell division event for whole-cell model simulations.

Implements the M. genitalium CellDivision process (Karr et al. 2012): Trigger: cell mass $\geq$ divisionMassThreshold (default $2 \times$ cellInitialDryWeight) **AND** (ftsZRingCompletion key in [WholeCellState](#) $\geq$ ftsZThreshold **OR** ftsZThreshold == 0, i.e., mass-only trigger)

On division:

- Each integer molecule count is binomially partitioned between daughters (each copy independently assigned to daughter 1 with $p=0.5$)
- This simulated cell becomes daughter 1
- [Cell](#) mass and volume are set to half the pre-division values
- Simulation step count is reset to 0 (new generation)
- [WholeCellState](#) lifecycle metadata is updated to reflect the new generation

Output ports: Port 0: entity forwarded when no division occurred this step (normal flow) Port 1: entity forwarded immediately after a division event (optional handler)

The division mass threshold is read from [WholeCellState](#) metabolite pool entry "param.cellDivisionMassThreshold" when set by loadParameters(). Override with the divisionMassThreshold property for manual configuration.

FtsZ ring completion is read from [WholeCellState](#) molecule count key ftsZRingKey (default "FtsZ_ring_completion", expected range 0–1000 as integer per-mille value; divide by 1000 for fraction).

15.107.2 Member Function Documentation

15.107.2.1 [_check\(\)](#)

```
bool CellDivisionEvent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.107.2.2 _createEditableDataDefinitions()

void CellDivisionEvent::_createEditableDataDefinitions () [override], [protected], [virtual]
 Reimplemented from [ModelDataDefinition](#).

15.107.2.3 _initBetweenReplications()

void CellDivisionEvent::_initBetweenReplications () [override], [protected], [virtual]
 Reimplemented from [ModelDataDefinition](#).

15.107.2.4 _loadInstance()

bool CellDivisionEvent::_loadInstance (
 [PersistenceRecord](#) * fields) [override], [protected], [virtual]
 virtual protected method that must be overridden
 Reimplemented from [ModelComponent](#).

15.107.2.5 _onDispatchEvent()

void CellDivisionEvent::_onDispatchEvent (
 [Entity](#) * entity,
 unsigned int inputPortNumber) [override], [protected], [virtual]

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.107.2.6 _saveInstance()

void CellDivisionEvent::_saveInstance (
 [PersistenceRecord](#) * fields,
 bool saveDefaultValues) [override], [protected], [virtual]

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.107.2.7 show()

std::string CellDivisionEvent::show () [override], [virtual]

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

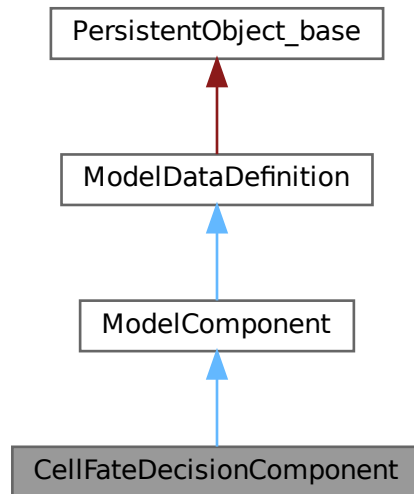
- plugins/components/WholeCellModeling/CellDivisionEvent.h
- plugins/components/WholeCellModeling/CellDivisionEvent.cpp

15.108 CellFateDecisionComponent Class Reference

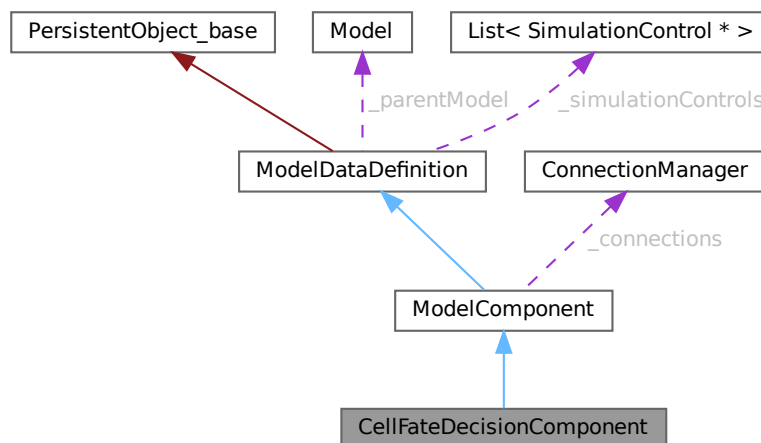
Routes entities according to the current whole-cell lifecycle state.

```
#include <plugins/components/WholeCellModeling/CellFateDecisionComponent.h>
```

Inheritance diagram for CellFateDecisionComponent:



Collaboration diagram for CellFateDecisionComponent:



Public Member Functions

- **CellFateDecisionComponent** ([Model](#) *model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const

- void **setDivisionReadyPhase** (std::string phase)
- std::string **getDivisionReadyPhase** () const
- void **setStarvedPhase** (std::string phase)
- std::string **getStarvedPhase** () const
- void **setArrestedPhase** (std::string phase)
- std::string **getArrestedPhase** () const
- void **setDeadPhase** (std::string phase)
- std::string **getDeadPhase** () const
- unsigned int **getLastRoutedPort** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.

- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)

- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()

virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.108.1 Detailed Description

Routes entities according to the current whole-cell lifecycle state.

[CellFateDecisionComponent](#) converts the coarse lifecycle annotation stored in [WholeCellState](#) into explicit simulation branching:

- port 0: viable default flow
- port 1: division-ready flow
- port 2: starved-but-still-viable flow
- port 3: dead or non-viable flow

The component does not mutate biological state. It only reads viability and lifecycle phase from [WholeCellState](#) and routes the entity to the appropriate downstream component.

15.108.2 Member Function Documentation

15.108.2.1 `_check()`

```
bool CellFateDecisionComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.108.2.2 `_createEditableDataDefinitions()`

```
void CellFateDecisionComponent::_createEditableDataDefinitions ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.108.2.3 `_initBetweenReplications()`

```
void CellFateDecisionComponent::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.108.2.4 `_loadInstance()`

```
bool CellFateDecisionComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.108.2.5 `_onDispatchEvent()`

```
void CellFateDecisionComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.108.2.6 `_saveInstance()`

```
void CellFateDecisionComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.108.2.7 `show()`

```
std::string CellFateDecisionComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

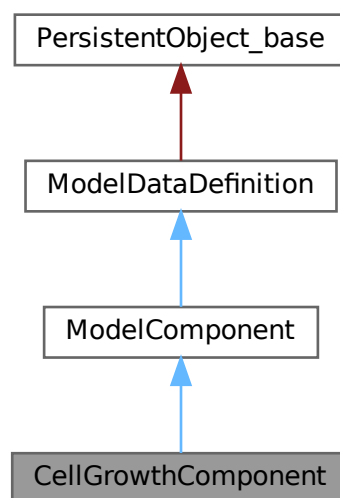
The documentation for this class was generated from the following files:

- `plugins/components/WholeCellModeling/CellFateDecisionComponent.h`
- `plugins/components/WholeCellModeling/CellFateDecisionComponent.cpp`

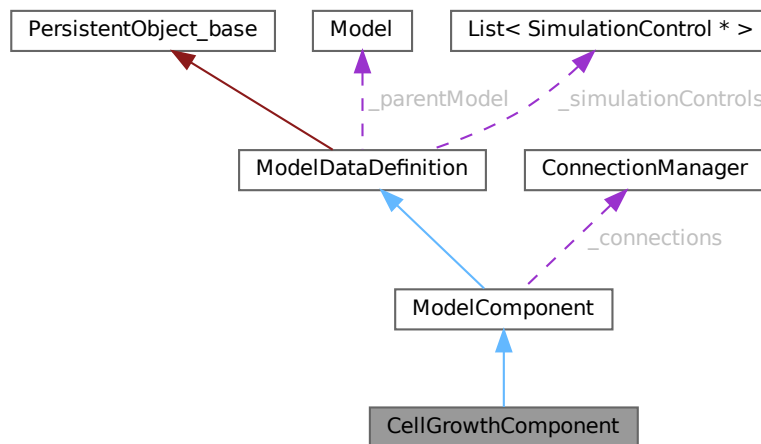
15.109 CellGrowthComponent Class Reference

```
#include <plugins/components/WholeCellModeling/CellGrowthComponent.h>
```

Inheritance diagram for CellGrowthComponent:



Collaboration diagram for CellGrowthComponent:



Public Member Functions

- **CellGrowthComponent** ([Model](#) *model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **setGrowthRate** (double rate)
- double **getGrowthRate** () const
- void **setEnergyMetaboliteKey** (std::string key)
- std::string **getEnergyMetaboliteKey** () const
- void **setEnergyHalfSaturation** (double value)
- double **getEnergyHalfSaturation** () const
- void **setDensity** (double density)
- double **getDensity** () const
- void **setDeltaT** (double dt)
- double **getDeltaT** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.

- `std::string getDescription () const`
Returns the free-form description associated with this component.
- `void remove (ModelComponent *comp)`
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const
Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)
Updates the trace level used by this definition.

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

- virtual void `_initBetweenReplications ()` override
- virtual void `_onDispatchEvent (Entity *entity, unsigned int inputPortNumber)` override
Handles a dispatched event for this component type.
- virtual void `_createEditableDataDefinitions ()` override

Protected Member Functions inherited from `ModelDataDefinition`

- bool `_getSaveDefaultsOption ()`
- virtual `ParserChangesInformation * _getParserChangesInformation ()`
- virtual void `_addSimulationControl (SimulationControl *control)`
- virtual void `_createInternalAndAttachedData ()`
- virtual void `_createInternalStatisticReporters ()`
- virtual void `_createAttachedAttributes ()`
- virtual void `_createNonEditableDataDefinitions ()`
- void `_templateCreateInternalStatisticReporters ()`
- void `_templateCreateAttachedAttributes ()`
- void `_templateCreateNonEditableDataDefinitions ()`
- void `_templateCreateEditableDataDefinitions ()`
- void `_internalStatisticReportersClear ()`
legacy helper methods used by the granular create hooks*
- void `_internalDataClear ()`
- void `_internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- void `_internalDataRemove (const std::string &key)`
- void `_attachedDataClear ()`
- void `_attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- void `_attachedDataRemove (const std::string &key)`
- void `_attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- void `_statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- void `_statisticReporterRemove (const std::string &key)`
- void `_statisticReportersClear ()`
- void `_mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- void `_mandatoryAttachedAttributeRemove (const std::string &key)`
- void `_mandatoryAttachedAttributesClear ()`
- void `_mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- void `_mandatoryEditableDataDefinitionRemove (const std::string &key)`
- void `_mandatoryEditableDataDefinitionsClear ()`
- void `_optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- void `_optionalEditableDataDefinitionRemove (const std::string &key)`
- void `_optionalEditableDataDefinitionsClear ()`
- void `_mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- void `_mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- void `_mandatoryNonEditableDataDefinitionsClear ()`
- void `_optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- void `_optionalNonEditableDataDefinitionRemove (const std::string &key)`
- void `_optionalNonEditableDataDefinitionsClear ()`
- void `_checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void `trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void `traceError (const std::string &text, const std::exception &e)`
- void `traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)`
- void `traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)`

- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.109.1 Detailed Description

Continuous cell growth applied at each simulation step.

Implements exponential mass growth: $\text{mass}(t+\Delta t) = \text{mass}(t) \times (1 + \text{growthRate} \times \Delta t)$ $\text{volume}(fL) = (\text{mass}_g \times 1e-3 / \text{density_kg_m3}) \times 1e15$

Parameter defaults from CovertLab/WholeCell parameters.json (MIT): $\text{growthRate} = 2.1393e-05$ /s (states.[↩](#)MetabolicReaction.meanInitialGrowthRate) $\text{density} = 1100.0$ kg/m³ (states.Geometry.density) $\text{deltaT} = 60.0$ s (must match simulation clock step)

15.109.2 Member Function Documentation

15.109.2.1 [_check\(\)](#)

```
bool CellGrowthComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.109.2.2 [_createEditableDataDefinitions\(\)](#)

```
void CellGrowthComponent::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.109.2.3 [_initBetweenReplications\(\)](#)

```
void CellGrowthComponent::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.109.2.4 [_loadInstance\(\)](#)

```
bool CellGrowthComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.109.2.5 `_onDispatchEvent()`

```
void CellGrowthComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.109.2.6 `_saveInstance()`

```
void CellGrowthComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.109.2.7 `show()`

```
std::string CellGrowthComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

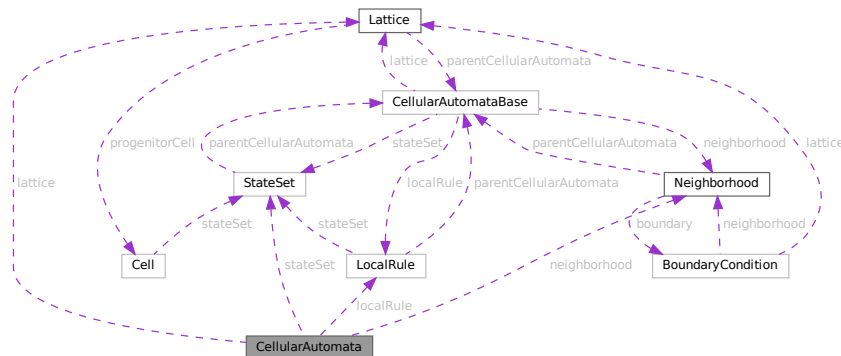
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- `plugins/components/WholeCellModeling/CellGrowthComponent.h`
- `plugins/components/WholeCellModeling/CellGrowthComponent.cpp`

15.110 CellularAutomata Class Reference

Collaboration diagram for CellularAutomata:



Public Member Functions

- **CellularAutomata** (std::string name="", [Lattice](#) *lattice=nullptr, [StateSet](#) *stateSet=nullptr, [Neighborhood](#) *neighborhood=nullptr, [LocalRule](#) *localRule=nullptr)
- **CellularAutomata** (const [CellularAutomata](#) &orig)
- virtual bool **init** ()
Init lattice and cells.
- virtual void **step** ()
Create next configuration $c(t+1) = G(c(t))$
- virtual void **start** ()
Run (step) until end of simulation.
- virtual void **pause** ()
Pause the "run".
- template<typename Class >
void **addObserver** (Class *object, void(Class::*function)([CellularAutomataEvent](#)))
- std::string **getName** () const
- void **setName** (const std::string name)
- void **setLocalRule** ([LocalRule](#) *localRule)
- [LocalRule](#) * **getLocalRule** () const
- void **setNeighborhood** ([Neighborhood](#) *neighborhood)
- [Neighborhood](#) * **getNeighborhood** () const
- void **setStateSet** ([StateSet](#) *stateSet)
- [StateSet](#) * **getStateSet** () const
- void **setLattice** ([Lattice](#) *lattice)
- [Lattice](#) * **getLattice** () const
- void **setSimulationLength** (unsigned int simulationLength)
- unsigned int **getSimulationLength** () const
- unsigned int **getSimulatedTime** () const
- bool **isRunning** () const

Protected Member Functions

- void **notifyObservers** ([CellularAutomataEvent](#) event)
Invoke observers then something (an event) happens.
- virtual void **applyLocalRule** ()=0
Pure virtual method that has to be overridden by derived classes.

Protected Attributes

- [Lattice](#) * **lattice**
- [StateSet](#) * **stateSet**
- [Neighborhood](#) * **neighborhood**
- [LocalRule](#) * **localRule**
- std::string **name**
- unsigned int **simulatedTime**
- unsigned int **simulationLength**
- bool **running**

The documentation for this class was generated from the following files:

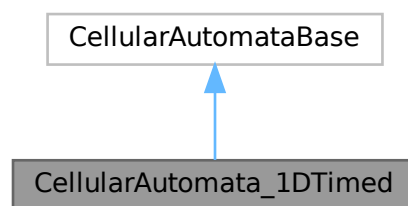
- plugins/components/ModalModel/CellularAutomata/CellularAutomata.h
- plugins/components/ModalModel/CellularAutomata/CellularAutomata.cpp

15.111 CellularAutomata_1DTimed Class Reference

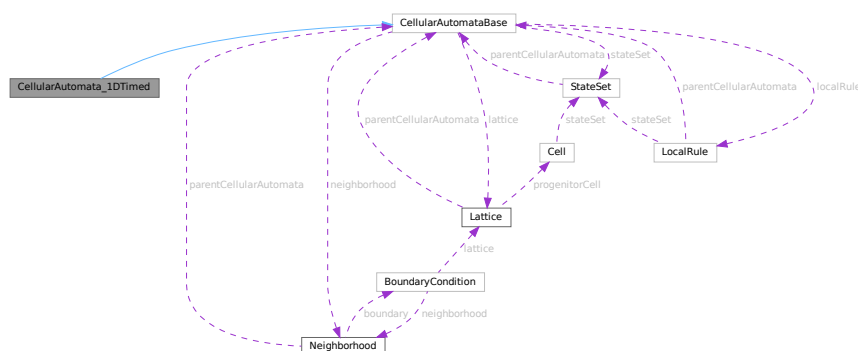
Complete uniform CA specific for unidimensional lattices where 2nd dimension is time (usefull for elementary CA)

```
#include <plugins/components/ModalModel/CellularAutomata/CellularAutomata_1DTimed.h>
```

Inheritance diagram for CellularAutomata_1DTimed:



Collaboration diagram for CellularAutomata_1DTimed:



Public Member Functions

- **CellularAutomata_1DTimed** ([Lattice](#) *lattice=nullptr, [StateSet](#) *stateSet=nullptr, [Neighborhood](#) *neighborhood=nullptr, [LocalRule](#) *localRule=nullptr)
- **CellularAutomata_1DTimed** (const [CellularAutomata_1DTimed](#) &orig)

Public Member Functions inherited from CellularAutomataBase

- **CellularAutomataBase** ([Lattice](#) *lattice=nullptr, [StateSet](#) *stateSet=nullptr, [Neighborhood](#) *neighborhood=nullptr, [LocalRule](#) *localRule=nullptr)
- **CellularAutomataBase** (const [CellularAutomataBase](#) &orig)
- virtual bool **init** ()
Init lattice and cells.
- virtual void **step** ()
Create next configuration $c(t+1) = G(c(t))$
- void **setLocalRule** ([LocalRule](#) *localRule)
- [LocalRule](#) * **getLocalRule** () const
- void **setNeighborhood** ([Neighborhood](#) *neighborhood)
- [Neighborhood](#) * **getNeighborhood** () const
- void **setStateSet** ([StateSet](#) *stateSet)
- [StateSet](#) * **getStateSet** () const
- void **setLattice** ([Lattice](#) *lattice)
- [Lattice](#) * **getLattice** () const

Protected Member Functions

- virtual void **applyLocalRule** () override
Pure virtual method that has to be overridden by derived classes.

Additional Inherited Members

Protected Attributes inherited from CellularAutomataBase

- [Lattice](#) * **lattice**
- [StateSet](#) * **stateSet**
- [Neighborhood](#) * **neighborhood**
- [LocalRule](#) * **localRule**

15.111.1 Detailed Description

Complete uniform CA specific for unidimensional lattices where 2nd dimension is time (usefull for elementary CA)

15.111.2 Member Function Documentation

15.111.2.1 applyLocalRule()

```
virtual void CellularAutomata_1DTimed::applyLocalRule ( ) [inline], [override], [protected], [virtual]
```

Pure virtual method that has to be overridden by derived classes.

Implements [CellularAutomataBase](#).

The documentation for this class was generated from the following file:

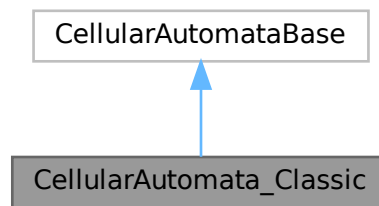
- plugins/components/ModalModel/CellularAutomata/CellularAutomata_1DTimed.h

15.112 CellularAutomata_Classic Class Reference

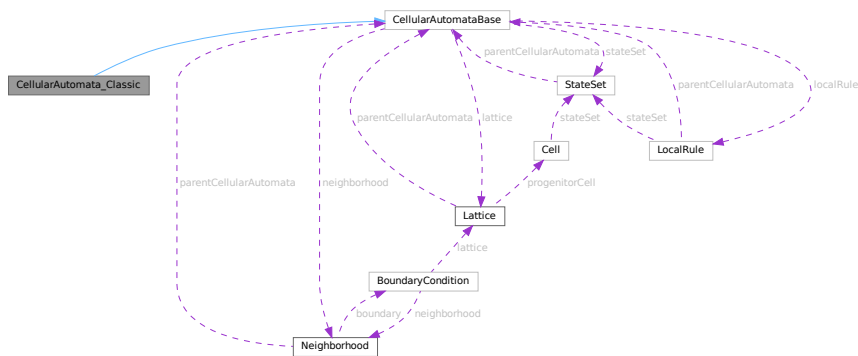
Complete uniform CA.

```
#include <plugins/components/ModalModel/CellularAutomata/CellularAutomata_↵
Classic.h>
```

Inheritance diagram for CellularAutomata_Classic:



Collaboration diagram for CellularAutomata_Classic:



Public Member Functions

- **CellularAutomata_Classic** ([Lattice](#) *lattice=nullptr, [StateSet](#) *stateSet=nullptr, [Neighborhood](#) *neighborhood=nullptr, [LocalRule](#) *localRule=nullptr)
- **CellularAutomata_Classic** (const [CellularAutomata_Classic](#) &orig)

Public Member Functions inherited from CellularAutomataBase

- **CellularAutomataBase** ([Lattice](#) *lattice=nullptr, [StateSet](#) *stateSet=nullptr, [Neighborhood](#) *neighborhood=nullptr, [LocalRule](#) *localRule=nullptr)
- **CellularAutomataBase** (const [CellularAutomataBase](#) &orig)
- virtual bool **init** ()
Init lattice and cells.
- virtual void **step** ()
Create next configuration $c(t+1) = G(c(t))$
- void **setLocalRule** ([LocalRule](#) *localRule)
- [LocalRule](#) * **getLocalRule** () const
- void **setNeighborhood** ([Neighborhood](#) *neighborhood)
- [Neighborhood](#) * **getNeighborhood** () const
- void **setStateSet** ([StateSet](#) *stateSet)
- [StateSet](#) * **getStateSet** () const
- void **setLattice** ([Lattice](#) *lattice)
- [Lattice](#) * **getLattice** () const

Protected Member Functions

- virtual void [applyLocalRule](#) () override
Pure virtual method that has to be overridden by derived classes.

Additional Inherited Members

Protected Attributes inherited from [CellularAutomataBase](#)

- [Lattice](#) * **lattice**
- [StateSet](#) * **stateSet**
- [Neighborhood](#) * **neighborhood**
- [LocalRule](#) * **localRule**

15.112.1 Detailed Description

Complete uniform CA.

15.112.2 Member Function Documentation

15.112.2.1 [applyLocalRule](#)()

```
virtual void CellularAutomata_Classic::applyLocalRule ( ) [inline], [override], [protected],  
[virtual]
```

Pure virtual method that has to be overridden by derived classes.

< overrides pure virtual method

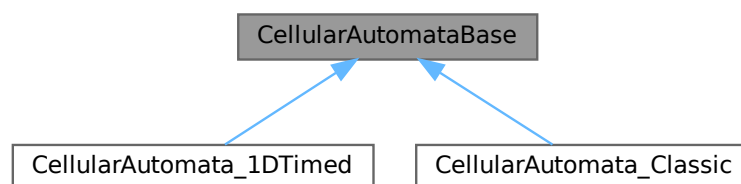
Implements [CellularAutomataBase](#).

The documentation for this class was generated from the following file:

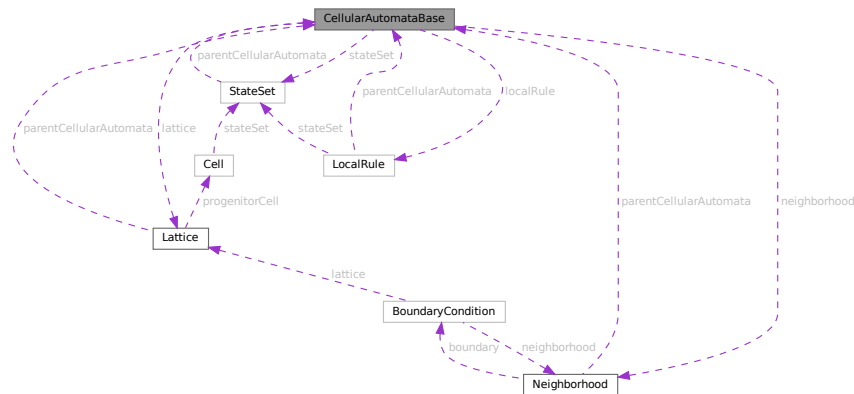
- plugins/components/ModalModel/CellularAutomata/CellularAutomata_Classic.h

15.113 CellularAutomataBase Class Reference

Inheritance diagram for CellularAutomataBase:



Collaboration diagram for CellularAutomataBase:



Public Member Functions

- **CellularAutomataBase** ([Lattice](#) *lattice=nullptr, [StateSet](#) *stateSet=nullptr, [Neighborhood](#) *neighborhood=nullptr, [LocalRule](#) *localRule=nullptr)
- **CellularAutomataBase** (const [CellularAutomataBase](#) &orig)
- virtual bool **init** ()
Init lattice and cells.
- virtual void **step** ()
Create next configuration $c(t+1) = G(c(t))$
- void **setLocalRule** ([LocalRule](#) *localRule)
- [LocalRule](#) * **getLocalRule** () const
- void **setNeighborhood** ([Neighborhood](#) *neighborhood)
- [Neighborhood](#) * **getNeighborhood** () const
- void **setStateSet** ([StateSet](#) *stateSet)
- [StateSet](#) * **getStateSet** () const
- void **setLattice** ([Lattice](#) *lattice)
- [Lattice](#) * **getLattice** () const

Protected Member Functions

- virtual void **applyLocalRule** ()=0
Pure virtual method that has to be overridden by derived classes.

Protected Attributes

- [Lattice](#) * **lattice**
- [StateSet](#) * **stateSet**
- [Neighborhood](#) * **neighborhood**
- [LocalRule](#) * **localRule**

15.113.1 Member Function Documentation

15.113.1.1 applyLocalRule()

```
void CellularAutomataBase::applyLocalRule ( ) [protected], [pure virtual]
```

Pure virtual method that has to be overridden by derived classes.

Implemented in [CellularAutomata_1DTimed](#), and [CellularAutomata_Classic](#).

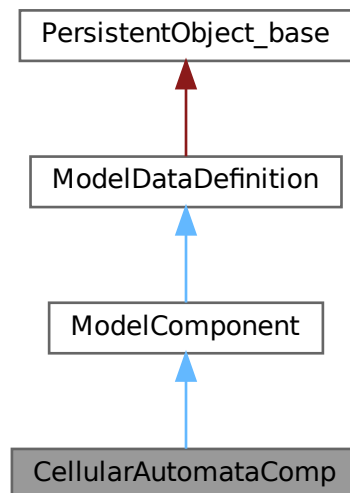
The documentation for this class was generated from the following files:

- plugins/components/ModalModel/CellularAutomata/CellularAutomataBase.h
- plugins/components/ModalModel/CellularAutomata/CellularAutomataBase.cpp

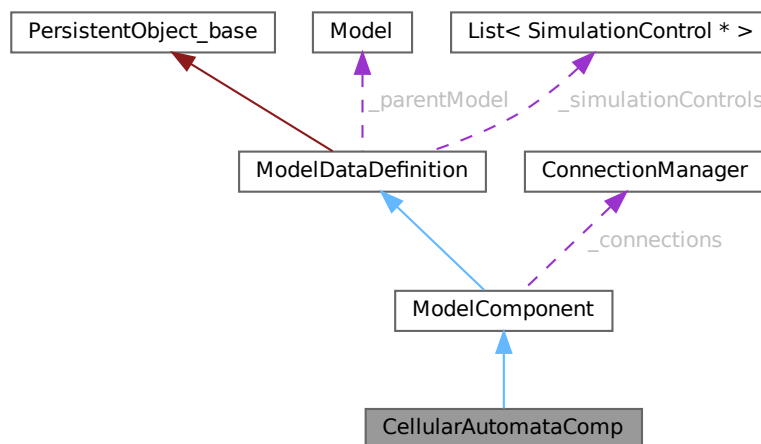
15.114 CellularAutomataComp Class Reference

```
#include <plugins/components/ModalModel/CellularAutomataComp.h>
```

Inheritance diagram for CellularAutomataComp:



Collaboration diagram for CellularAutomataComp:



Public Types

- enum class [CellularAutomataType](#) : int {
CLASSIC = 1 , **TIMED_1D** = 2 , **ASYNCHRONOUS** = 3 , **NONUNIFORMRULE** = 4 ,
NONUNIFORMNEIGHBOOR = 5 , **USERDEFINED** = 6 }

enums

- enum class **LatticeType** : int {
 RETICULAR = 1 , **TRIANGULAR** = 2 , **HEXAGONAL** = 3 , **NETWORK** = 4 ,
 USERDEFINED = 5 }
- enum class **NeighborhoodType** : int {
 CENTERED = 1 , **BACKWARD** = 2 , **FORWARD** = 3 , **VONNEUMANN** = 4 ,
 MOORE = 5 , **USERDEFINED** = 6 }
- enum class **BoundaryType** : int {
 FIXED = 1 , **CLOSED** = 2 , **REFLEXIVE** = 3 , **ADIABATIC** = 4 ,
 USERDEFINED = 5 }
- enum class **StateSetType** : int {
 ENUMERATED = 1 , **INTEGERBASED** = 2 , **BITBASED** = 3 , **DOUBLEBASED** = 4 ,
 USERDEFINED = 5 }
- enum class **LocalRuleType** : int {
 ELEMENTAR_CA = 1 , **GAME_OF_LIFE** = 2 , **BIASED_COMPETITION** = 3 , **HPP** = 4 ,
 SAND_ROCK = 5 , **USERDEFINED** = 99 }
- enum class **AutomataType** : int { **Temporary** = 0 , **Permanent** = 1 }

Public Member Functions

- **CellularAutomataComp** ([Model](#) *model, std::string name="")
 constructors
- [CellularAutomataComp::CellularAutomataType](#) **getCellularAutomataType** () const
 new public user methods for this component
- void **setCellularAutomataType** ([CellularAutomataComp::CellularAutomataType](#) newCellularAutomataType)
- [CellularAutomataComp::LatticeType](#) **getLatticeType** () const
- void **setLatticeType** ([CellularAutomataComp::LatticeType](#) newLatticeStructure)
- [CellularAutomataComp::NeighborhoodType](#) **getNeighborhoodType** () const
- void **setNeighborhoodType** ([CellularAutomataComp::NeighborhoodType](#) newNeighborhood)
- [CellularAutomataComp::BoundaryType](#) **geBoundaryType** () const
- void **setBoundaryType** ([CellularAutomataComp::BoundaryType](#) newBoundary)
- [CellularAutomataComp::StateSetType](#) **getStateSetType** () const
- void **setStateSetType** ([CellularAutomataComp::StateSetType](#) newStateSetType)
- [Lattice](#) * **getlattice** () const
- [Neighborhood](#) * **getNeighborhood** () const
- [StateSet](#) * **getStateSet** () const
- virtual std::string **show** () override
 virtual public methods
- [CellularAutomataComp::LocalRuleType](#) **getlocalRuleType** () const
- void **setLocalRuleType** ([CellularAutomataComp::LocalRuleType](#) newLocalRuleType)
- [LocalRule](#) * **getlocalRule** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
 constructors
- virtual ~**ModelComponent** ()
 Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
 new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
 Connects this component to another component.
- bool **hasBreakpointAt** ()
 Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)

Updates the free-form description shown by the GUI.

- `std::string getDescription () const`

Returns the free-form description associated with this component.

- `void remove (ModelComponent *comp)`

Removes a connected component from this component.

Public Member Functions inherited from `ModelDataDefinition`

- `ModelDataDefinition (Model *model, std::string datadefinitionTypename, std::string name="", bool insertIntoModel=true)`

Creates a model data definition and optionally inserts it into the model.

- `virtual ~ModelDataDefinition ()`

Releases resources owned by this data definition.

- `Util::identification getId () const`

Returns the unique identifier of this instance.

- `void setName (const std::string &name)`

Updates the logical name of this instance.

- `const std::string & getName () const`

Returns the logical name of this instance.

- `void setLabel (const std::string &label)`

Updates the user-visible label of this instance.

- `const std::string & getLabel () const`

Returns the user-visible label of this instance.

- `const std::string & getClassname () const`

Returns the runtime class name used by the plugin system.

- `Model * getParentModel () const`

Returns the parent model that owns this definition.

- `bool isReportStatistics () const`

Indicates whether this data definition contributes report statistics.

- `void setReportStatistics (bool reportStatistics)`

Enables or disables report-statistics generation for this definition.

- `ModelDataDefinition * getInternalData (const std::string &name) const`

Looks up a named internal child data definition.

- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`

Returns the full map of internal child data definitions.

- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`

Returns the full map of attached data definitions.

- `bool hasChanged () const`

Indicates whether this instance has been modified since the last reset.

- `void setHasChanged (bool hasChanged)`

Updates the changed flag for this data definition.

- `unsigned int getLevel () const`

Returns the level in the model hierarchy where this definition lives.

- `void setModelLevel (unsigned int _modelLevel)`

Sets the model nesting level for this definition.

- `List< SimulationControl * > * getSimulationControls () const`

Returns the simulation controls owned by this data definition.

- `TraceManager::Level getTraceLevelSpecific () const`

Returns the trace level that applies specifically to this definition.

- `void setTraceLevelSpecific (TraceManager::Level level)`

Updates the trace level used by this definition.

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void `_onDispatchEvent` ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual bool `_check` (std::string *errorMessage)
virtual protected methods that could be overridden by derived classes, if needed
- virtual void `_initBetweenReplications` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual bool `_check` (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)

- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.114.1 Detailed Description

This component ...

15.114.2 Member Function Documentation

15.114.2.1 **_check()**

```
bool CellularAutomataComp::_check (
    std::string * errorMessage ) [protected], [virtual]
```

virtual protected methods that could be overridden by derived classes, if needed

This method is called by ModelChecker during model check. The component should check itself to verify if user parameters are ok (ex: correct syntax for the parser) and everithing in its parameters allow the model too run without errors in this component

15.114.2.2 **_initBetweenReplications()**

```
void CellularAutomataComp::_initBetweenReplications ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDatas. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dynamically linked to to simulator kernel to reflect the changes

This method is called by [ModelSimulation](#) when initiating the replication. The model should set all value for a new replication (Ex: setting back to 0 any internal counter, clearing lists, etc.

Reimplemented from [ModelDataDefinition](#).

15.114.2.3 _loadInstance()

```
bool CellularAutomataComp::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.114.2.4 _onDispatchEvent()

```
void CellularAutomataComp::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

This method is only for ModelComponents, not ModelDataElements.

Implements [ModelComponent](#).

15.114.2.5 _saveInstance()

```
void CellularAutomataComp::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.114.2.6 show()

```
std::string CellularAutomataComp::show ( ) [override], [virtual]
```

virtual public methods

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/ModalModel/CellularAutomataComp.h
- plugins/components/ModalModel/CellularAutomataComp.cpp

15.115 CellularAutomataDemoBuilder Class Reference**Static Public Member Functions**

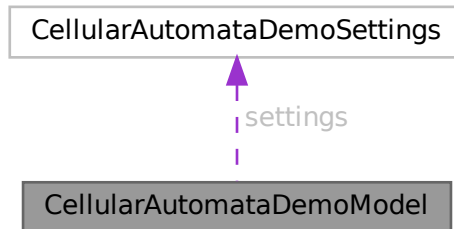
- static std::unique_ptr< [CellularAutomataDemoModel](#) > **buildDemo** ([CellularAutomataDemoSettings](#) settings={})

The documentation for this class was generated from the following files:

- applications/gui/genesys/cellular_automata/CellularAutomataDemoBuilder.h
- applications/gui/genesys/cellular_automata/CellularAutomataDemoBuilder.cpp

15.116 CellularAutomataDemoModel Struct Reference

Collaboration diagram for CellularAutomataDemoModel:



Public Member Functions

- **CellularAutomataDemoModel** ([CellularAutomataDemoSettings](#) settings={})
- int **columns** () const
- int **rows** () const
- bool **isValidPosition** (int x, int y) const
- [Cell](#) * **cellAt** (int x, int y) const
- long **stateAt** (int x, int y) const
- QString **stateTextAt** (int x, int y) const
- bool **setStateAt** (int x, int y, long value)
- void **fill** (long value)
- void **seedDefaultPattern** ()
- std::vector< long > **availablePaintStates** () const

Public Attributes

- [CellularAutomataDemoSettings](#) **settings**
- std::vector< [State](#) > **ownedStates**
- std::vector< [State](#) * > **statePointers**
- std::unique_ptr< [CellularAutomata_Classic](#) > **automaton**
- std::unique_ptr< [Lattice](#) > **lattice**
- std::unique_ptr< [StateSet](#) > **stateSet**
- std::unique_ptr< [BoundaryCondition](#) > **boundary**
- std::unique_ptr< [Neighborhood](#) > **neighborhood**
- std::unique_ptr< [LocalRule](#) > **localRule**
- std::unique_ptr< [LocalRule_CopyCurrentState](#) > **identityLocalRule**

The documentation for this struct was generated from the following files:

- applications/gui/genesys/cellular_automata/CellularAutomataDemoBuilder.h
- applications/gui/genesys/cellular_automata/CellularAutomataDemoBuilder.cpp

15.117 CellularAutomataDemoSettings Struct Reference

Public Attributes

- QSize **latticeSize** = QSize{100, 100}
- CellularAutomataRulePreset **rulePreset** = CellularAutomataRulePreset::SandRock

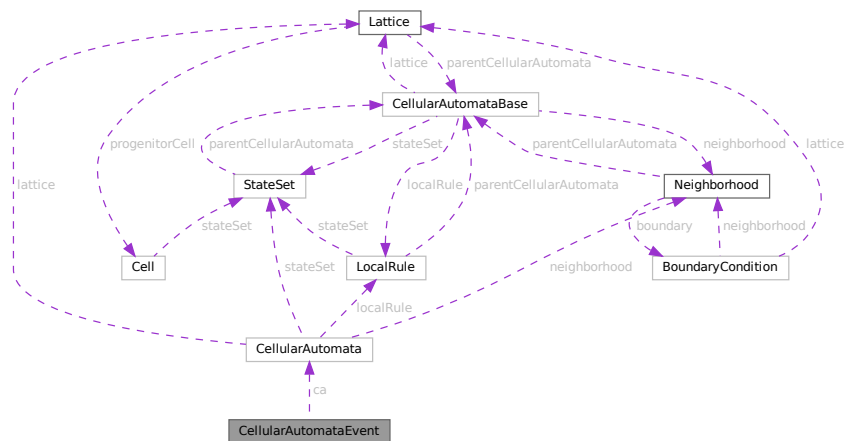
- CellularAutomataNeighborhoodPreset **neighborhoodPreset** = CellularAutomataNeighborhoodPreset::↔ Moore
- CellularAutomataBoundaryPreset **boundaryPreset** = CellularAutomataBoundaryPreset::Fixed
- CellularAutomataStatePreset **statePreset** = CellularAutomataStatePreset::Enumerated

The documentation for this struct was generated from the following file:

- applications/gui/genesys/cellular_automata/CellularAutomataDemoBuilder.h

15.118 CellularAutomataEvent Class Reference

Collaboration diagram for CellularAutomataEvent:



Public Member Functions

- CellularAutomataEvent (CellularAutomata *ca)

Public Attributes

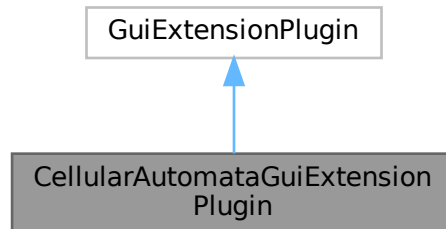
- CellularAutomata * ca

The documentation for this class was generated from the following file:

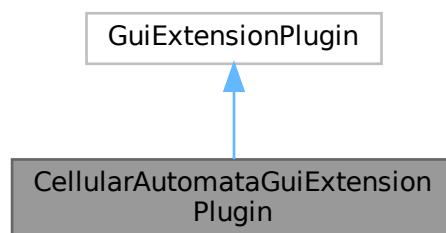
- plugins/components/ModalModel/CellularAutomata/CellularAutomata.h

15.119 CellularAutomataGuiExtensionPlugin Class Reference

Inheritance diagram for CellularAutomataGuiExtensionPlugin:



Collaboration diagram for CellularAutomataGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId () const` override
- `std::string displayName () const` override
- `std::string version () const` override
- `std::vector< std::string > requiredModelPlugins () const` override
- `void registerContributions (GuiExtensionRegistry *registry) const` override

15.119.1 Member Function Documentation

15.119.1.1 displayName()

```
std::string CellularAutomataGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.119.1.2 extensionId()

```
std::string CellularAutomataGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.119.1.3 registerContributions()

```
void CellularAutomataGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.119.1.4 requiredModelPlugins()

```
std::vector< std::string > CellularAutomataGuiExtensionPlugin::requiredModelPlugins ( ) const
[inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.119.1.5 version()

```
std::string CellularAutomataGuiExtensionPlugin::version ( ) const [inline], [override], [virtual]
```

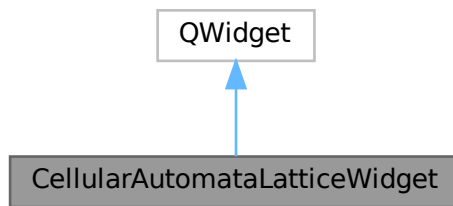
Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

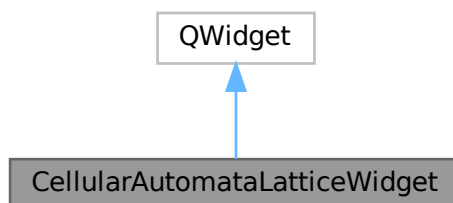
- applications/gui/genesys/extensions/CellularAutomataGuiExtensionPlugin.cpp

15.120 CellularAutomataLatticeWidget Class Reference

Inheritance diagram for CellularAutomataLatticeWidget:



Collaboration diagram for CellularAutomataLatticeWidget:



Public Member Functions

- **CellularAutomataLatticeWidget** (QWidget *parent=nullptr)
- void **setController** ([CellularAutomataViewerController](#) *controller)

Protected Member Functions

- void **paintEvent** (QPaintEvent *event) override
- void **mousePressEvent** (QMouseEvent *event) override
- void **mouseMoveEvent** (QMouseEvent *event) override
- void **mouseReleaseEvent** (QMouseEvent *event) override
- void **contextMenuEvent** (QContextMenuEvent *event) override
- QSize **minimumSizeHint** () const override

The documentation for this class was generated from the following files:

- applications/gui/genesys/cellular_automata/CellularAutomataLatticeWidget.h
- applications/gui/genesys/cellular_automata/CellularAutomataLatticeWidget.cpp

15.121 CellularAutomataStateColorMap Class Reference

Static Public Member Functions

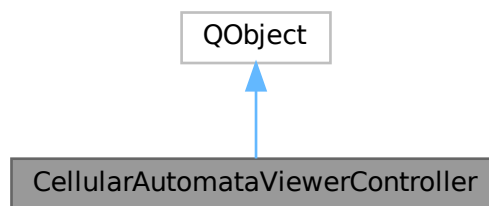
- static QColor **colorForState** (long state)
- static QColor **colorForStateText** (const QString &stateText, long fallbackState)
- static QColor **backgroundColor** ()
- static QColor **gridLineColor** ()
- static QColor **emptyTextColor** ()

The documentation for this class was generated from the following file:

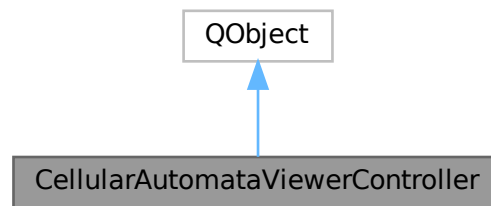
- applications/gui/genesys/cellular_automata/CellularAutomataStateColorMap.h

15.122 CellularAutomataViewerController Class Reference

Inheritance diagram for CellularAutomataViewerController:



Collaboration diagram for CellularAutomataViewerController:



Signals

- void **modelChanged** ()
- void **settingsChanged** ()
- void **timeChanged** (unsigned int time)
- void **runningChanged** (bool running)
- void **paintStateChanged** ()
- void **statusMessage** (const QString &message)

Public Member Functions

- **CellularAutomataViewerController** (QObject *parent=nullptr)
- [CellularAutomataDemoModel](#) * **model** () const
- [CellularAutomataDemoSettings](#) **settings** () const
- QSize **latticeSize** () const
- bool **hasDemo** () const
- unsigned int **simulatedTime** () const
- bool **isRunning** () const
- QString **configurationSummary** () const
- std::vector< long > **availablePaintStates** () const
- long **cellState** (int x, int y) const
- bool **setCellState** (int x, int y, long value)
- void **fill** (long value)
- bool **hasSelectedPaintState** () const
- std::optional< long > **selectedPaintState** () const
- QString **selectedPaintStateText** () const
- void **setSelectedPaintState** (long state)
- void **clearSelectedPaintState** ()
- int **autoStepIntervalMs** () const
- void **setAutoStepIntervalMs** (int intervalMs)
- void **rebuildDemo** ([CellularAutomataDemoSettings](#) settings={})
- void **setSettings** ([CellularAutomataDemoSettings](#) settings)
- void **setRulePreset** (CellularAutomataRulePreset preset)
- void **setNeighborhoodPreset** (CellularAutomataNeighborhoodPreset preset)
- void **setBoundaryPreset** (CellularAutomataBoundaryPreset preset)
- void **setStatePreset** (CellularAutomataStatePreset preset)
- void **resetToInitialState** ()
- void **step** ()
- void **start** ()

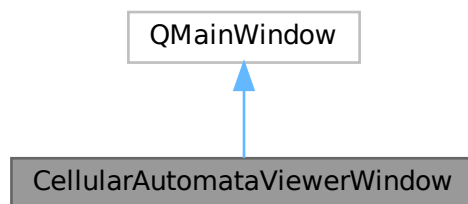
- void **pause** ()
- void **stop** ()
- bool **saveConfiguration** (const QString &filePath, QString *errorMessage=nullptr) const
- bool **loadConfiguration** (const QString &filePath, QString *errorMessage=nullptr)

The documentation for this class was generated from the following files:

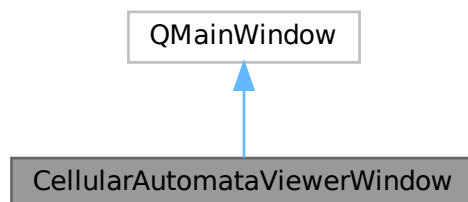
- applications/gui/genesys/cellular_automata/CellularAutomataViewerController.h
- applications/gui/genesys/cellular_automata/CellularAutomataViewerController.cpp

15.123 CellularAutomataViewerWindow Class Reference

Inheritance diagram for CellularAutomataViewerWindow:



Collaboration diagram for CellularAutomataViewerWindow:



Public Member Functions

- **CellularAutomataViewerWindow** (QWidget *parent=nullptr)

The documentation for this class was generated from the following files:

- applications/gui/genesys/cellular_automata/CellularAutomataViewerWindow.h
- applications/gui/genesys/cellular_automata/CellularAutomataViewerWindow.cpp

15.124 ChatMessage Struct Reference

One message in a multi-turn conversation with an LLM.

```
#include <tools/AIAssistant/AIProviderClient_if.h>
```

Public Attributes

- `std::string role`
"system", "user", or "assistant".
- `std::string content`
Text content of the message.

15.124.1 Detailed Description

One message in a multi-turn conversation with an LLM.

Role conventions follow the OpenAI and Anthropic chat-completion APIs:

- "system" — instructions/persona injected before the conversation.
- "user" — message from the human turn.
- "assistant" — previous model replies (for multi-turn context).

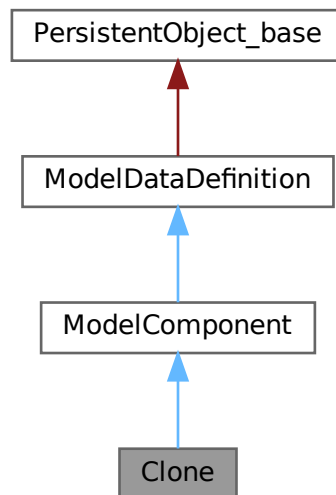
The documentation for this struct was generated from the following file:

- `tools/IAAssistant/AIProviderClient_if.h`

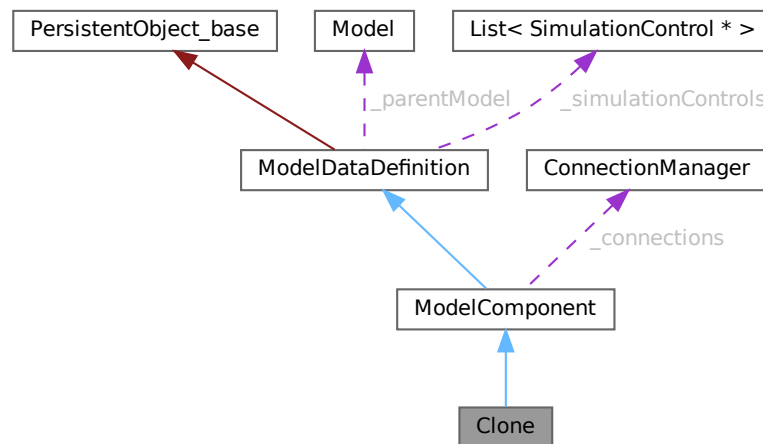
15.125 Clone Class Reference

```
#include <plugins/components/DiscreteProcessing/Clone.h>
```

Inheritance diagram for Clone:



Collaboration diagram for Clone:



Public Member Functions

- **Clone** ([Model](#) *model, std::string name="")
- virtual std::string [show](#) () override
virtual public methods
- void **setNumClonesExpression** (std::string numClones)
- std::string **getNumClonesExpression** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert←IntoModel=true)
Creates a model data definition and optionally inserts it into the model.

- virtual **~ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool **hasChanged**)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- **List**< **SimulationControl** * > * **getSimulationControls** () const
Returns the simulation controls owned by this data definition.
- **TraceManager::Level** **getTraceLevelSpecific** () const
Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (**TraceManager::Level** level)
Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (**TraceManager::Level** traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const
Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static **PluginInformation** * **GetPluginInformation** ()
- static **ModelComponent** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_createInternalStatisticReporters](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()

- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internaStatisticReportersClear** ()
- legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
- method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
- virtual protected methods that could be overridden by derived classes, if needed*

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.125.1 Detailed Description

This component ...

15.125.2 Member Function Documentation

15.125.2.1 **_check()**

```
bool Clone::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.125.2.2 **_createInternalStatisticReporters()**

```
void Clone::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.125.2.3 **_initBetweenReplications()**

```
void Clone::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.125.2.4 **_loadInstance()**

```
bool Clone::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.125.2.5 **_onDispatchEvent()**

```
void Clone::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.125.2.6 _saveInstance()

```
void Clone::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.125.2.7 show()

```
std::string Clone::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

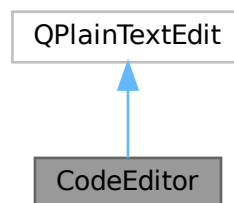
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

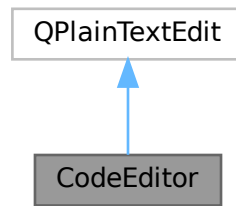
- plugins/components/DiscreteProcessing/Clone.h
- plugins/components/DiscreteProcessing/Clone.cpp

15.126 CodeEditor Class Reference

Inheritance diagram for CodeEditor:



Collaboration diagram for CodeEditor:



Public Member Functions

- **CodeEditor** (QWidget *parent=nullptr)
- void **lineNumberAreaPaintEvent** (QPaintEvent *event)
- int **lineNumberAreaWidth** ()

Protected Member Functions

- void **resizeEvent** (QResizeEvent *event) override

The documentation for this class was generated from the following files:

- applications/gui/genesys/codeeditor/CodeEditor.h
- applications/gui/genesys/codeeditor/CodeEditor.cpp

15.127 testing::internal::CodeLocation Struct Reference

Public Member Functions

- **CodeLocation** (const std::string &a_file, int a_line)

Public Attributes

- std::string **file**
- int **line**

The documentation for this struct was generated from the following file:

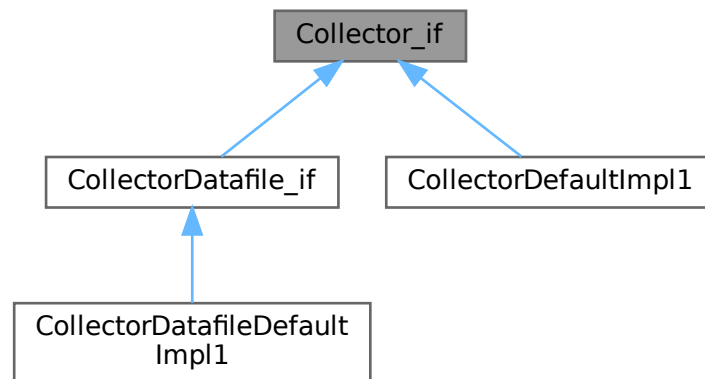
- gtest/gtest/internal/gtest-internal.h

15.128 Collector_if Class Reference

Interface responsible for collecting observations of a stochastic variable.

```
#include <kernel/statistics/Collector_if.h>
```

Inheritance diagram for Collector_if:



Public Member Functions

- virtual void `clear` ()=0
clear
- virtual void `addValue` (double value, double weight=1)=0
addValue
- virtual double `getLastValue` ()=0
getLastValue
- virtual unsigned long `numElements` ()=0
numElements
- virtual void `setAddValueHandler` (CollectorAddValueHandler addValueHandler)=0
setAddValueHandler
- virtual void `setClearHandler` (CollectorClearHandler clearHandler)=0
setClearHandler

15.128.1 Detailed Description

Interface responsible for collecting observations of a stochastic variable.

A collector stores values generated during simulation and optionally supports callbacks for integration with online statistics, tracing or UI updates.

15.128.2 Member Function Documentation

15.128.2.1 `addValue()`

```
virtual void Collector_if::addValue (
    double value,
    double weight = 1 ) [pure virtual]
```

`addValue`

Parameters

<i>value</i>	
<i>weight</i>	

Adds a new observed value (with optional weight).

Implemented in [CollectorDatafileDefaultImpl1](#), and [CollectorDefaultImpl1](#).

15.128.2.2 clear()

```
virtual void Collector_if::clear ( ) [pure virtual]
```

clear

Clears all collected values and resets internal state.

Implemented in [CollectorDatafileDefaultImpl1](#), and [CollectorDefaultImpl1](#).

15.128.2.3 getLastValue()

```
virtual double Collector_if::getLastValue ( ) [pure virtual]
```

getLastValue

Returns

Returns the last collected value.

Implemented in [CollectorDatafileDefaultImpl1](#), and [CollectorDefaultImpl1](#).

15.128.2.4 numElements()

```
virtual unsigned long Collector_if::numElements ( ) [pure virtual]
```

numElements

Returns

Returns the number of stored observations.

Implemented in [CollectorDatafileDefaultImpl1](#), and [CollectorDefaultImpl1](#).

15.128.2.5 setAddValueHandler()

```
virtual void Collector_if::setAddValueHandler (
    CollectorAddValueHandler addValueHandler ) [pure virtual]
```

setAddValueHandler

Parameters

<i>addValueHandler</i>	
------------------------	--

Registers a callback triggered after each added value.

Implemented in [CollectorDatafileDefaultImpl1](#), and [CollectorDefaultImpl1](#).

15.128.2.6 setClearHandler()

```
virtual void Collector_if::setClearHandler (
    CollectorClearHandler clearHandler ) [pure virtual]
```

setClearHandler

Parameters

<i>clearHandler</i>	
---------------------	--

Registers a callback triggered when the collector is cleared.

Implemented in [CollectorDatafileDefaultImpl1](#), and [CollectorDefaultImpl1](#).

The documentation for this class was generated from the following file:

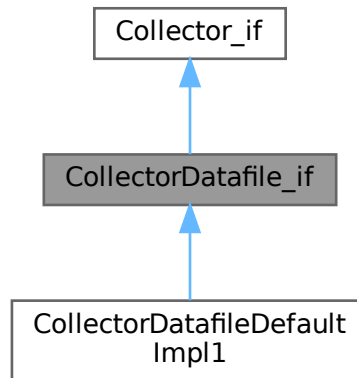
- kernel/statistics/Collector_if.h

15.129 CollectorDatafile_if Class Reference

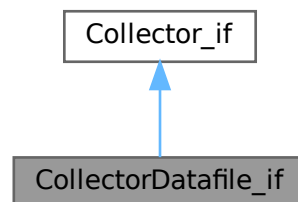
Collector interface specialized in file-backed value storage.

```
#include <kernel/statistics/CollectorDatafile_if.h>
```

Inheritance diagram for CollectorDatafile_if:



Collaboration diagram for CollectorDatafile_if:



Public Member Functions

- virtual double [getValue](#) (unsigned int rank)=0
getValue
- virtual void [seekFirstValue](#) ()=0
seekFirstValue
- virtual double [getNextValue](#) ()=0
getNextValue
- virtual std::string [getDataFilename](#) ()=0
getDataFilename
- virtual void [setDataFilename](#) (std::string filename)=0
setDataFilename

Public Member Functions inherited from [Collector_if](#)

- virtual void [clear](#) ()=0
clear
- virtual void [addValue](#) (double value, double weight=1)=0
addValue
- virtual double [getLastValue](#) ()=0
getLastValue
- virtual unsigned long [numElements](#) ()=0
numElements
- virtual void [setAddValueHandler](#) (CollectorAddValueHandler addValueHandler)=0
setAddValueHandler
- virtual void [setClearHandler](#) (CollectorClearHandler clearHandler)=0
setClearHandler

15.129.1 Detailed Description

Collector interface specialized in file-backed value storage.

Besides standard collection operations, implementations provide sequential/random access to persisted samples and filename management for the underlying dataset.

15.129.2 Member Function Documentation

15.129.2.1 [getDataFilename\(\)](#)

```
virtual std::string CollectorDatafile_if::getDataFilename ( ) [pure virtual]
getDataFilename
```

Returns

Returns the filename used to persist collected values.
Implemented in [CollectorDatafileDefaultImpl1](#).

15.129.2.2 [getNextValue\(\)](#)

```
virtual double CollectorDatafile_if::getNextValue ( ) [pure virtual]
getNextValue
```

Returns

Get the next value in the file and advances the pointer

Implemented in [CollectorDatafileDefaultImpl1](#).

15.129.2.3 [getValue\(\)](#)

```
virtual double CollectorDatafile_if::getValue (
    unsigned int rank ) [pure virtual]
getValue
```

Parameters

<i>rank</i>	
-------------	--

Returns

Get a value from a specific position

Implemented in [CollectorDatafileDefaultImpl1](#).

15.129.2.4 seekFirstValue()

```
virtual void CollectorDatafile_if::seekFirstValue ( ) [pure virtual]
```

seekFirstValue

Set the pointer to the first value in the file

Implemented in [CollectorDatafileDefaultImpl1](#).

15.129.2.5 setDataFilename()

```
virtual void CollectorDatafile_if::setDataFilename (
    std::string filename ) [pure virtual]
```

setDataFilename

Parameters

<i>filename</i>	
-----------------	--

Sets/updates the filename used to store collected data.

Implemented in [CollectorDatafileDefaultImpl1](#).

The documentation for this class was generated from the following file:

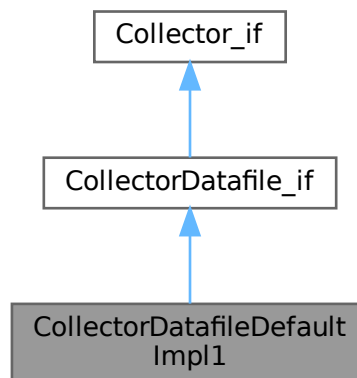
- kernel/statistics/CollectorDatafile_if.h

15.130 CollectorDatafileDefaultImpl1 Class Reference

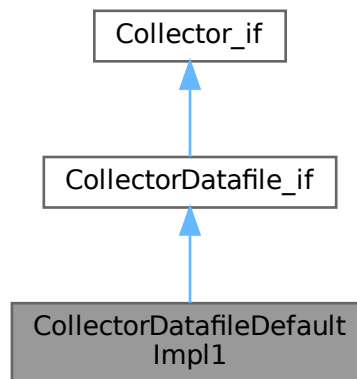
Default file-backed implementation of [CollectorDatafile_if](#).

```
#include <kernel/statistics/CollectorDatafileDefaultImpl1.h>
```

Inheritance diagram for CollectorDatafileDefaultImpl1:



Collaboration diagram for CollectorDatafileDefaultImpl1:



Public Member Functions

- **CollectorDatafileDefaultImpl1 ()**
Creates an empty file-backed collector instance.
- virtual void **clear ()** override
Clears collector state and resets file cursor/content as implemented.
- virtual void **addValue** (double value, double weight=1) override
Appends a value (and optional weight) to the underlying storage.
- virtual double **getLastValue ()** override
Returns the most recently collected value.
- virtual unsigned long **numElements ()** override
Returns the number of collected elements.
- virtual double **getValue** (unsigned int num) override
Returns the value stored at ordinal position num.
- virtual double **getNextValue ()** override
Returns the next value from current file cursor and advances cursor.
- virtual void **seekFirstValue ()** override
Repositions internal cursor to the first stored value.
- virtual std::string **getDataFilename ()** override
Returns the filename currently used as data backing store.
- virtual void **setDataFilename** (std::string filename) override
Sets the filename used as data backing store.
- virtual void **setAddValueHandler** (CollectorAddValueHandler addValueHandler) override
Registers callback invoked whenever a value is added.
- virtual void **setClearHandler** (CollectorClearHandler clearHandler) override
Registers callback invoked whenever the collector is cleared.

15.130.1 Detailed Description

Default file-backed implementation of [CollectorDatafile_if](#).

Stores collected values in an associated data file and keeps lightweight counters/cache for last value and number of elements.

15.130.2 Member Function Documentation

15.130.2.1 addValue()

```
void CollectorDatafileDefaultImpl1::addValue (
    double value,
    double weight = 1 ) [override], [virtual]
```

Appends a value (and optional weight) to the underlying storage.

Implements [Collector_if](#).

15.130.2.2 clear()

```
void CollectorDatafileDefaultImpl1::clear ( ) [override], [virtual]
```

Clears collector state and resets file cursor/content as implemented.

Implements [Collector_if](#).

15.130.2.3 getDataFilename()

```
std::string CollectorDatafileDefaultImpl1::getDataFilename ( ) [override], [virtual]
```

Returns the filename currently used as data backing store.

Implements [CollectorDatafile_if](#).

15.130.2.4 getLastValue()

```
double CollectorDatafileDefaultImpl1::getLastValue ( ) [override], [virtual]
```

Returns the most recently collected value.

Implements [Collector_if](#).

15.130.2.5 getNextValue()

```
double CollectorDatafileDefaultImpl1::getNextValue ( ) [override], [virtual]
```

Returns the next value from current file cursor and advances cursor.

Implements [CollectorDatafile_if](#).

15.130.2.6 getValue()

```
double CollectorDatafileDefaultImpl1::getValue (
    unsigned int num ) [override], [virtual]
```

Returns the value stored at ordinal position num.

Implements [CollectorDatafile_if](#).

15.130.2.7 numElements()

```
unsigned long CollectorDatafileDefaultImpl1::numElements ( ) [override], [virtual]
```

Returns the number of collected elements.

Implements [Collector_if](#).

15.130.2.8 seekFirstValue()

```
void CollectorDatafileDefaultImpl1::seekFirstValue ( ) [override], [virtual]
```

Repositions internal cursor to the first stored value.

Implements [CollectorDatafile_if](#).

15.130.2.9 setAddValueHandler()

```
void CollectorDatafileDefaultImpl1::setAddValueHandler (
    CollectorAddValueHandler addValueHandler ) [override], [virtual]
```

Registers callback invoked whenever a value is added.

Implements [Collector_if](#).

15.130.2.10 setClearHandler()

```
void CollectorDatafileDefaultImpl1::setClearHandler (
    CollectorClearHandler clearHandler ) [override], [virtual]
```

Registers callback invoked whenever the collector is cleared.

Implements [Collector_if](#).

15.130.2.11 setDataFilename()

```
void CollectorDatafileDefaultImpl1::setDataFilename (
    std::string filename ) [override], [virtual]
```

Sets the filename used as data backing store.

Implements [CollectorDatafile_if](#).

The documentation for this class was generated from the following files:

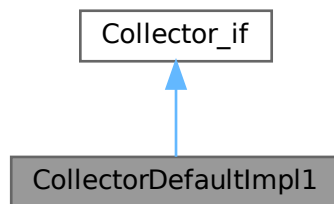
- kernel/statistics/CollectorDatafileDefaultImpl1.h
- kernel/statistics/CollectorDatafileDefaultImpl1.cpp

15.131 CollectorDefaultImpl1 Class Reference

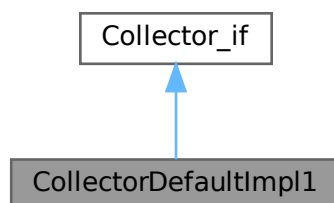
Default in-memory implementation of [Collector_if](#).

```
#include <kernel/statistics/CollectorDefaultImpl1.h>
```

Inheritance diagram for CollectorDefaultImpl1:



Collaboration diagram for CollectorDefaultImpl1:



Public Member Functions

- [CollectorDefaultImpl1](#) ()

- Creates an empty collector.*
- virtual void [clear](#) () override
 - Clears all collected data and resets counters.*
- virtual void [addValue](#) (double value, double weight=1) override
 - Adds a new observation to the collector.*
- virtual double [getLastValue](#) () override
 - Returns the latest observed value.*
- virtual unsigned long [numElements](#) () override
 - Returns total number of observed values.*
- virtual void [setAddValueHandler](#) (CollectorAddValueHandler addValueHandler) override
 - Registers callback notified after each addValue call.*
- virtual void [setClearHandler](#) (CollectorClearHandler clearHandler) override
 - Registers callback notified when clear is invoked.*

15.131.1 Detailed Description

Default in-memory implementation of [Collector_if](#).

Maintains minimal aggregate state (last value and count) and supports optional callbacks for add/clear events.

15.131.2 Member Function Documentation

15.131.2.1 addValue()

```
void CollectorDefaultImpl1::addValue (
    double value,
    double weight = 1 ) [override], [virtual]
```

Adds a new observation to the collector.

Implements [Collector_if](#).

15.131.2.2 clear()

```
void CollectorDefaultImpl1::clear ( ) [override], [virtual]
```

Clears all collected data and resets counters.

Implements [Collector_if](#).

15.131.2.3 getLastValue()

```
double CollectorDefaultImpl1::getLastValue ( ) [override], [virtual]
```

Returns the latest observed value.

Implements [Collector_if](#).

15.131.2.4 numElements()

```
unsigned long CollectorDefaultImpl1::numElements ( ) [override], [virtual]
```

Returns total number of observed values.

Implements [Collector_if](#).

15.131.2.5 setAddValueHandler()

```
void CollectorDefaultImpl1::setAddValueHandler (
    CollectorAddValueHandler addValueHandler ) [override], [virtual]
```

Registers callback notified after each addValue call.

Implements [Collector_if](#).

15.131.2.6 setClearHandler()

```
void CollectorDefaultImpl1::setClearHandler (
    CollectorClearHandler clearHandler ) [override], [virtual]
```

Registers callback notified when `clear` is invoked.

Implements [Collector_if](#).

The documentation for this class was generated from the following files:

- kernel/statistics/CollectorDefaultImpl1.h
- kernel/statistics/CollectorDefaultImpl1.cpp

15.132 GroProgramRuntime::ColonyMutation Struct Reference

Public Types

- enum class **Type** {
Reset , **SpawnSeed** , **SetChemostatMode** , **AddBarrier** ,
MapToCells , **SetSignalAt** , **SetSignalRect** , **SetSignalGridWidth** ,
SetSignalGridHeight , **GetSignalMatrix** , **DumpSignalField** }

Public Attributes

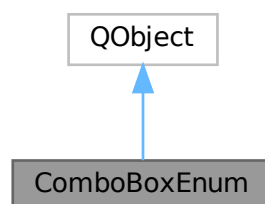
- Type **type** = Type::Reset
- std::vector< std::string > **arguments**
- std::vector< double > **numericArguments**
- std::string **expressionText**
- std::size_t **previewRows** = 0
- std::size_t **previewColumns** = 0

The documentation for this struct was generated from the following file:

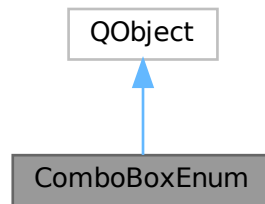
- plugins/data/BiochemicalSimulation/GroProgramRuntime.h

15.133 ComboBoxEnum Class Reference

Inheritance diagram for ComboBoxEnum:



Collaboration diagram for ComboBoxEnum:



Public Types

- using **AfterChange** = `std::function< void()>`

Public Member Functions

- **ComboBoxEnum** ([PropertyEditorGenesys](#) *editor, [SimulationControl](#) *property, AfterChange afterChange=())
- void **open_box** ()
- void **configure_options** ([SimulationControl](#) *property)

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/ComboBoxEnum.h
- applications/gui/genesys/propertyeditor/ComboBoxEnum.cpp

15.134 GroProgramlr::Command Struct Reference

Public Types

- enum class **Kind** { [RawStatement](#) , [FunctionCall](#) , [Assignment](#) , [IfStatement](#) }

Public Member Functions

- bool **isFunctionCall** () const
- bool **isAssignment** () const
- bool **isIfStatement** () const

Public Attributes

- Kind **kind** = `Kind::RawStatement`
- std::string **sourceText** = ""
- std::string **functionName** = ""
- std::vector< std::string > **arguments**
- std::string **assignmentTarget** = ""
- std::string **expressionText** = ""
- bool **assignmentOnlyIfUnset** = false
- std::vector< [Command](#) > **thenCommands**
- std::vector< [Command](#) > **elseCommands**

The documentation for this struct was generated from the following file:

- plugins/data/BiochemicalSimulation/GroProgramlr.h

15.135 Util::CommandResult Class Reference

Public Attributes

- bool **success** = false
- std::string **commandStdOutput** = ""
- std::string **commandErrOutput** = ""
- std::string **workingDirectory** = "./"
- std::string **destinationPath** = "./"

The documentation for this class was generated from the following file:

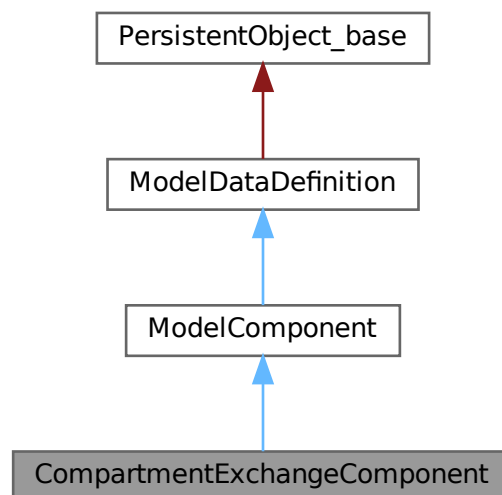
- kernel/util/Util.h

15.136 CompartmentExchangeComponent Class Reference

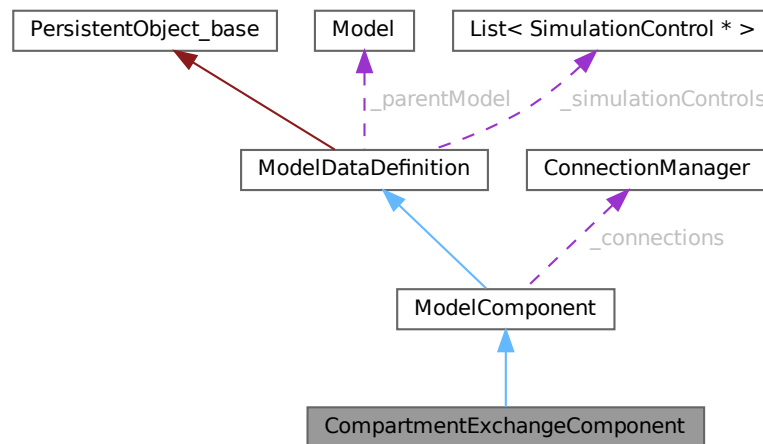
Moves metabolite proxies between whole-cell regions.

```
#include <plugins/components/WholeCellModeling/CompartmentExchangeComponent.h>
```

Inheritance diagram for CompartmentExchangeComponent:



Collaboration diagram for CompartmentExchangeComponent:



Classes

- struct [ExchangeRule](#)

Public Member Functions

- **CompartmentExchangeComponent** ([Model](#) *model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **setSourceRegion** (std::string region)
- std::string **getSourceRegion** () const
- void **setSourceMetaboliteKey** (std::string key)
- std::string **getSourceMetaboliteKey** () const
- void **setTargetRegion** (std::string region)
- std::string **getTargetRegion** () const
- void **setTargetMetaboliteKey** (std::string key)
- std::string **getTargetMetaboliteKey** () const
- void **setExchangeFraction** (double fraction)
- double **getExchangeFraction** () const
- void **setBaselineAmount** (double amount)
- double **getBaselineAmount** () const
- void **setDriverPathwayKey** (std::string key)
- std::string **getDriverPathwayKey** () const
- void **setDriverScale** (double scale)
- double **getDriverScale** () const
- void **setMaxTransferAmount** (double amount)
- double **getMaxTransferAmount** () const
- void **setConserveMass** (bool conserve)
- bool **getConserveMass** () const
- void **clearExchangeRules** ()
- void **addExchangeRule** (const std::string &label, const std::string &sourceRegion, const std::string &sourceMetaboliteKey, const std::string &targetRegion, const std::string &targetMetaboliteKey, double exchangeFraction, double baselineAmount=0.0, const std::string &driverPathwayKey="", double driverScale=1.0, double maxTransferAmount=0.0, bool conserveMass=true)

- unsigned int **getExchangeRuleCount** () const
- const std::vector< [ExchangeRule](#) > & **getExchangeRules** () const
- double **getLastTransferAmount** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- [Model](#) * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.

- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelComponent](#)

- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static bool Check (ModelComponent *component)`
Validates a component instance.
- `static void CreateInternalData (ModelComponent *component)`
Creates internal data required by a component instance.
- `static void SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
Serializes a component instance into a persistence record.
- `static void DispatchEvent (Event *event)`
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)

- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.136.1 Detailed Description

Moves metabolite proxies between whole-cell regions.

[CompartmentExchangeComponent](#) transfers a fraction of a source metabolite pool in [WholeCellState](#) into a target pool on each dispatched event. The component can represent didactic transport, exchange, or redistribution policies between global and compartment-scoped pools without embedding that logic directly into examples.

Source and target regions follow this convention:

- empty region name: global [WholeCellState](#) metabolite pool;
- non-empty region name: compartment-scoped pool.

15.136.2 Member Function Documentation

15.136.2.1 `_check()`

```
bool CompartmentExchangeComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.136.2.2 `_createEditableDataDefinitions()`

```
void CompartmentExchangeComponent::_createEditableDataDefinitions ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.136.2.3 `_initBetweenReplications()`

```
void CompartmentExchangeComponent::_initBetweenReplications ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.136.2.4 `_loadInstance()`

```
bool CompartmentExchangeComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.136.2.5 `_onDispatchEvent()`

```
void CompartmentExchangeComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.136.2.6 `_saveInstance()`

```
void CompartmentExchangeComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.136.2.7 show()

```
std::string CompartmentExchangeComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/WholeCellModeling/CompartmentExchangeComponent.h
- plugins/components/WholeCellModeling/CompartmentExchangeComponent.cpp

15.137 CppCompiler::CompilationResult Class Reference

Public Attributes

- bool **success** = false
- std::string **compilationStdOutput** = ""
- std::string **compilationErrOutput** = ""
- std::string **destinationPath** = ""
- unsigned int **lineNumber** = 0

The documentation for this class was generated from the following file:

- plugins/data/ExternalIntegration/CppCompiler.h

15.138 CompilerThread Class Reference

Public Member Functions

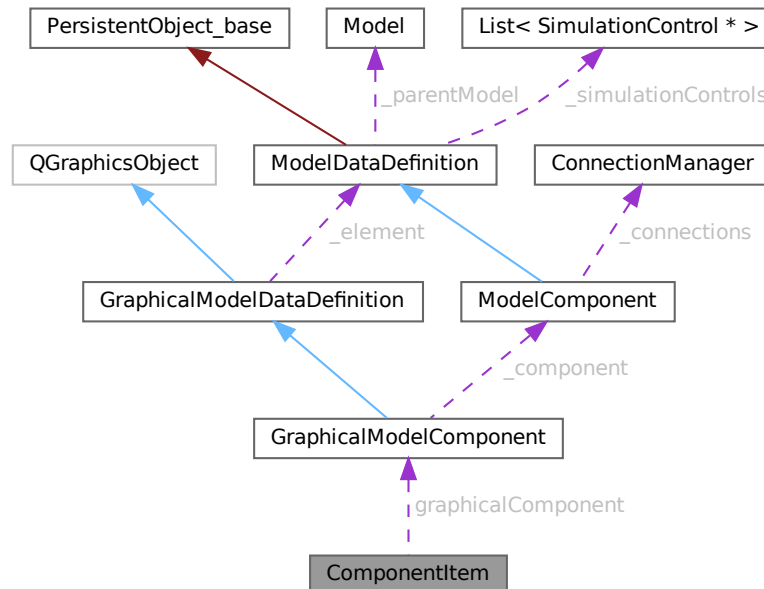
- void **operator()** ()

The documentation for this class was generated from the following file:

- plugins/data/ExternalIntegration/CppCompiler.h

15.139 ComponentItem Struct Reference

Collaboration diagram for ComponentItem:



Public Attributes

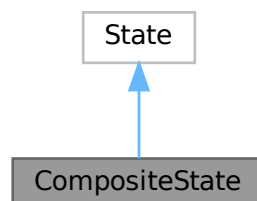
- [GraphicalModelComponent](#) * **graphicalComponent**
- QPointF **initialPosition**
- QList< [GraphicalConnection](#) * > **inputConnections**
- QList< [GraphicalConnection](#) * > **outputConnections**

The documentation for this struct was generated from the following file:

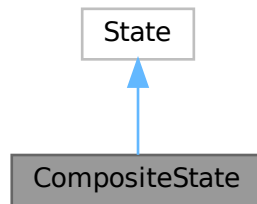
- applications/gui/genesys/actions/AddUndoCommand.h

15.140 CompositeState Class Reference

Inheritance diagram for CompositeState:



Collaboration diagram for CompositeState:



Public Types

- using **Field** = std::pair< std::string, std::unique_ptr< [State](#) > >

Public Member Functions

- **CompositeState** (const [CompositeState](#) &orig)
- [CompositeState](#) & **operator=** (const [CompositeState](#) &orig)
- std::unique_ptr< [State](#) > **clone** () const override
- bool **equals** (const [State](#) &other) const override
- std::string **toString** () const override
- std::string **typeName** () const override
- bool **addField** (std::string name, const [State](#) &state)
- const std::vector< Field > & **fields** () const

Public Member Functions inherited from [State](#)

- **State** (long value)
- **State** (const [State](#) &orig)
- virtual std::string **show** () const
- virtual long **getValue** () const
- virtual void **setValue** (long value)
- template<typename T >
const T * **as** () const noexcept
- template<typename T >
T * **as** () noexcept

Additional Inherited Members

Protected Attributes inherited from [State](#)

- long **_value**

15.140.1 Member Function Documentation

15.140.1.1 clone()

std::unique_ptr< [State](#) > CompositeState::clone () const [inline], [override], [virtual]
Reimplemented from [State](#).

- `std::unique_ptr< State > createDefaultState ()` const override
- `bool isComposite ()` const override
- `std::string name ()` const override

Public Member Functions inherited from [StateSet](#)

- **StateSet** ([CellularAutomataBase](#) *parentCellularAutomata)
- **StateSet** (const [StateSet](#) &orig)
- virtual `std::unique_ptr< State > parseState` (const `std::string &text`) const
- virtual `bool isFinite ()` const
- virtual `bool isDiscrete ()` const
- virtual `bool isContinuous ()` const
- virtual `std::vector< std::unique_ptr< State > > enumerateStates` () const

Additional Inherited Members

Protected Attributes inherited from [StateSet](#)

- [CellularAutomataBase](#) * parentCellularAutomata

15.141.1 Member Function Documentation

15.141.1.1 contains()

```
bool CompositeStateSet::contains (
    const State & state ) const    [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

15.141.1.2 createDefaultState()

```
std::unique_ptr< State > CompositeStateSet::createDefaultState ( ) const    [inline], [override],
[virtual]
```

Reimplemented from [StateSet](#).

15.141.1.3 isComposite()

```
bool CompositeStateSet::isComposite ( ) const    [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

15.141.1.4 name()

```
std::string CompositeStateSet::name ( ) const    [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

The documentation for this class was generated from the following file:

- `plugins/components/ModalModel/CellularAutomata/StateSet.h`

15.142 HypothesisTester_if::ConfidenceInterval Class Reference

Confidence interval [inferior, superior] and its half-width e0.

```
#include <tools/Statistics/HypothesisTester_if.h>
```

Public Member Functions

- **ConfidenceInterval** (double inferiorLimit, double superiorLimit, double e0)
- double **inferiorLimit** ()
- double **superiorLimit** ()
- double **halfWidth** ()

15.142.1 Detailed Description

Confidence interval [inferior, superior] and its half-width e0.

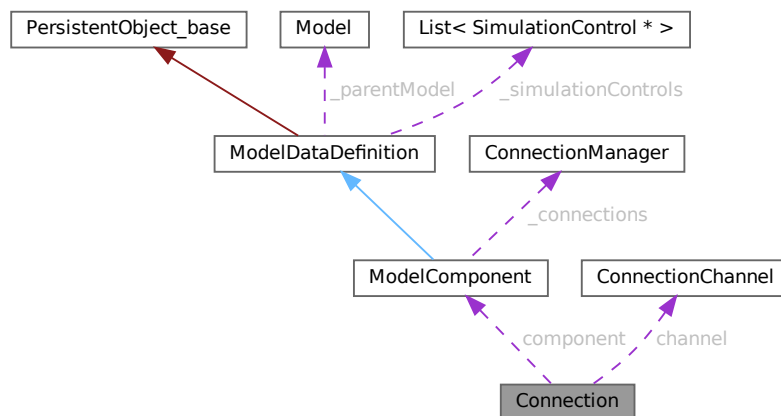
The documentation for this class was generated from the following file:

- tools/Statistics/HypothesisTester_if.h

15.143 Connection Struct Reference

```
#include <kernel/simulator/ConnectionManager.h>
```

Collaboration diagram for Connection:



Public Attributes

- [ModelComponent](#) * **component**
- [ConnectionChannel](#) **channel**

15.143.1 Detailed Description

Defines a connection between two or more [ModelComponents](#). A (receiver) component's inputPort may receive connection from one or more (sender) components' outputPorts. A (sender) component's outputPort is connected to one and only one (receiver) component's inputPort.

The documentation for this struct was generated from the following file:

- kernel/simulator/ConnectionManager.h

15.144 ConnectionChannel Struct Reference

The [ConnectionChannel](#) class defines an input or output port that connects a [ModelComponent](#) to another.

```
#include <kernel/simulator/ConnectionManager.h>
```

Public Attributes

- unsigned int **portNumber** = 0
- std::string **portDescription** = ""

15.144.1 Detailed Description

The [ConnectionChannel](#) class defines an input or output port that connects a [ModelComponent](#) to another. The documentation for this struct was generated from the following file:

- kernel/simulator/ConnectionManager.h

15.145 ConnectionManager Class Reference

Maintains outgoing connections from a component to downstream components.

```
#include <kernel/simulator/ConnectionManager.h>
```

Public Member Functions

- **ConnectionManager ()**
Creates an empty connection manager with zero constraints.
- unsigned int [size](#) ()
size
- [Connection](#) * [getFrontConnection](#) ()
getFrontConnection
- [Connection](#) * [getConnectionAtPort](#) (unsigned int rank)
getConnectionAtPort
- void [insert](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Inserts a new connection to a target component.
- void [insert](#) ([Connection](#) *connection)
insert
- void [insertAtPort](#) (unsigned int port, [Connection](#) *connection)
insertAtPort
- void [remove](#) ([Connection](#) *connection)
remove
- void [removeAtPort](#) (unsigned int port)
removeAtPort
- std::map< unsigned int, [Connection](#) * > * [connections](#) () const
Returns the internal output-port to connection map.
- unsigned int [getCurrentOutputConnectionsSize](#) () const
Returns the number of currently configured outgoing connections.
- void [setMaxOutputConnections](#) (unsigned int _maxOutputConnections)
Sets the maximum number of outgoing connections allowed.
- unsigned int [getMaxOutputConnections](#) () const
Returns the maximum number of outgoing connections allowed.
- void [setMinOutputConnections](#) (unsigned int _minOutputConnections)
Sets the minimum number of outgoing connections allowed.
- unsigned int [getMinOutputConnections](#) () const
Returns the minimum number of outgoing connections allowed.
- unsigned int [getCurrentInputConnectionsSize](#) () const
Returns the number of currently configured incoming connections.
- void [setMaxInputConnections](#) (unsigned int _maxInputConnections)
Sets the maximum number of incoming connections allowed.
- unsigned int [getMaxInputConnections](#) () const
Returns the maximum number of incoming connections allowed.
- void [setMinInputConnections](#) (unsigned int _minInputConnections)
Sets the minimum number of incoming connections allowed.
- unsigned int [getMinInputConnections](#) () const
Returns the minimum number of incoming connections allowed.

15.145.1 Detailed Description

Maintains outgoing connections from a component to downstream components.

A connection maps an output rank (port) to a destination component/input port. The manager also stores min/max input/output constraints used during validation.

15.145.2 Member Function Documentation

15.145.2.1 connections()

```
std::map< unsigned int, Connection * > * ConnectionManager::connections ( ) const
```

Returns the internal output-port to connection map.

Returns

Map of output ports to outgoing connections.

15.145.2.2 getConnectionAtPort()

```
Connection * ConnectionManager::getConnectionAtPort (
    unsigned int rank )
```

getConnectionAtPort

Parameters

<i>rank</i>	
-------------	--

Returns

[Connection](#) bound to output rank *rank*.

15.145.2.3 getCurrentInputConnectionsSize()

```
unsigned int ConnectionManager::getCurrentInputConnectionsSize ( ) const
```

Returns the number of currently configured incoming connections.

Returns

Current input connection count.

15.145.2.4 getCurrentOutputConnectionsSize()

```
unsigned int ConnectionManager::getCurrentOutputConnectionsSize ( ) const
```

Returns the number of currently configured outgoing connections.

Returns

Current output connection count.

15.145.2.5 getFrontConnection()

```
Connection * ConnectionManager::getFrontConnection ( )
```

getFrontConnection

Returns

[Connection](#) at the first available output rank.

15.145.2.6 getMaxInputConnections()

```
unsigned int ConnectionManager::getMaxInputConnections ( ) const
```

Returns the maximum number of incoming connections allowed.

Returns

Maximum incoming connection count.

15.145.2.7 getMaxOutputConnections()

```
unsigned int ConnectionManager::getMaxOutputConnections ( ) const
```

Returns the maximum number of outgoing connections allowed.

Returns

Maximum outgoing connection count.

15.145.2.8 getMinInputConnections()

```
unsigned int ConnectionManager::getMinInputConnections ( ) const
```

Returns the minimum number of incoming connections allowed.

Returns

Minimum incoming connection count.

15.145.2.9 getMinOutputConnections()

```
unsigned int ConnectionManager::getMinOutputConnections ( ) const
```

Returns the minimum number of outgoing connections allowed.

Returns

Minimum outgoing connection count.

15.145.2.10 insert() [1/2]

```
void ConnectionManager::insert (
    Connection * connection )
```

insert

Parameters

<i>connection</i>	
-------------------	--

Inserts a prebuilt connection object.

15.145.2.11 insert() [2/2]

```
void ConnectionManager::insert (
    ModelComponent * component,
    unsigned int inputPortNumber = 0 )
```

Inserts a new connection to a target component.

Creates and inserts a connection to *component* at input port *inputPortNumber*. When the port is not specified, the connection uses the default input port 0.

Parameters

<i>component</i>	Target component.
------------------	-------------------

Parameters

<i>inputPortNumber</i>	Target input port number.
------------------------	---------------------------

15.145.2.12 insertAtPort()

```
void ConnectionManager::insertAtPort (
    unsigned int port,
    Connection * connection )
```

insertAtPort

Parameters

<i>port</i>	
<i>connection</i>	

Forces insertion/replacement at a specific output port.

15.145.2.13 remove()

```
void ConnectionManager::remove (
    Connection * connection )
```

remove

Parameters

<i>connection</i>	
-------------------	--

Removes a connection by pointer match.

15.145.2.14 removeAtPort()

```
void ConnectionManager::removeAtPort (
    unsigned int port )
```

removeAtPort

Parameters

<i>port</i>	
-------------	--

Removes the connection associated with output port *port*.

15.145.2.15 setMaxInputConnections()

```
void ConnectionManager::setMaxInputConnections (
    unsigned int _maxInputConnections )
```

Sets the maximum number of incoming connections allowed.

Parameters

<i>_maxInputConnections</i>	Maximum incoming connection count.
-----------------------------	------------------------------------

15.145.2.16 setMaxOutputConnections()

```
void ConnectionManager::setMaxOutputConnections (
    unsigned int _maxOutputConnections )
```


Sets the maximum number of outgoing connections allowed.

Parameters

<code>_maxOutputConnections</code>	Maximum outgoing connection count.
------------------------------------	------------------------------------

15.145.2.17 setMinInputConnections()

```
void ConnectionManager::setMinInputConnections (
    unsigned int _minInputConnections )
```

Sets the minimum number of incoming connections allowed.

Parameters

<code>_minInputConnections</code>	Minimum incoming connection count.
-----------------------------------	------------------------------------

15.145.2.18 setMinOutputConnections()

```
void ConnectionManager::setMinOutputConnections (
    unsigned int _minOutputConnections )
```

Sets the minimum number of outgoing connections allowed.

Parameters

<code>_minOutputConnections</code>	Minimum outgoing connection count.
------------------------------------	------------------------------------

15.145.2.19 size()

```
unsigned int ConnectionManager::size ( )
size
```

Returns

Number of configured output connections.

The documentation for this class was generated from the following files:

- kernel/simulator/ConnectionManager.h
- kernel/simulator/ConnectionManager.cpp

15.146 testing::internal::ConstCharPtr Struct Reference

Public Member Functions

- **ConstCharPtr** (const char *str)
- **operator bool** () const

Public Attributes

- const char * **value**

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-internal.h

15.147 Optimizer_if::ConstraintDefinition Struct Reference

Feasibility restriction represented as an expression.

```
#include <tools/Optimization/Optimizer_if.h>
```

Public Attributes

- std::string **name**
- std::string **expression**
- bool **enabled** = true

15.147.1 Detailed Description

Feasibility restriction represented as an expression.

A constraint is considered satisfied when expression != 0 and violated when expression == 0.

The documentation for this struct was generated from the following file:

- tools/Optimization/Optimizer_if.h

15.148 testing::internal::ConstRef< T > Struct Template Reference

Public Types

- typedef const T & **type**

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-port.h

15.149 testing::internal::ConstRef< T & > Struct Template Reference

Public Types

- typedef T & **type**

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-port.h

15.150 testing::internal::ContainerPrinter Struct Reference

Static Public Member Functions

- template<typename T , typename = typename std::enable_if< (sizeof (IsContainerTest<T> (0)) == sizeof (IsContainer)) && !IsRecursiveContainer<T>::value>::type>
static void **PrintValue** (const T &container, std::ostream *os)

The documentation for this struct was generated from the following file:

- gtest/gtest/gtest-printers.h

15.151 yy::genesyspp_parser::context Class Reference

Public Member Functions

- **context** (const [genesyspp_parser](#) &yyparser, const [symbol_type](#) &yylla)
- const [symbol_type](#) & **lookahead** () const YY_NOEXCEPT
- [symbol_kind_type](#) **token** () const YY_NOEXCEPT
- const [location_type](#) & **location** () const YY_NOEXCEPT
- int **expected_tokens** ([symbol_kind_type](#) yyarg[], int yyargn) const

15.151.1 Member Function Documentation

15.151.1.1 expected_tokens()

```
int yy::genesyspp_parser::context::expected_tokens (
    symbol_kind_type yyarg[],
    int yyargn ) const
```

Put in YYARG at most YYARGN of the expected tokens, and return the number of tokens stored in YYARG. If YYARG is null, return the number of expected tokens (guaranteed to be less than YYNTOKENS).

The documentation for this class was generated from the following files:

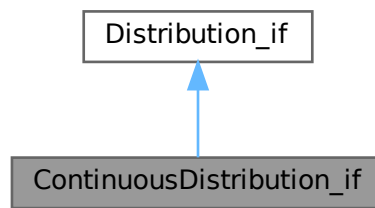
- parser/[GenesysParser.h](#)
- parser/GenesysParser.cpp

15.152 ContinuousDistribution_if Class Reference

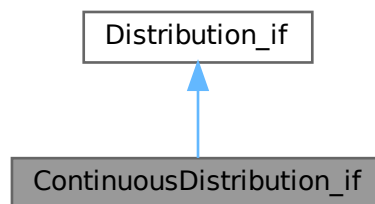
Interface for continuous distributions.

```
#include <tools/Statistics/ContinuousDistribution_if.h>
```

Inheritance diagram for ContinuousDistribution_if:



Collaboration diagram for ContinuousDistribution_if:



Public Member Functions

- virtual double **pdf** (double x) const =0
- virtual double **cdf** (double x) const =0
- virtual double **quantile** (double probability) const =0

Public Member Functions inherited from [Distribution_if](#)

- virtual std::string **name** () const =0

15.152.1 Detailed Description

Interface for continuous distributions.

Mathematical meaning:

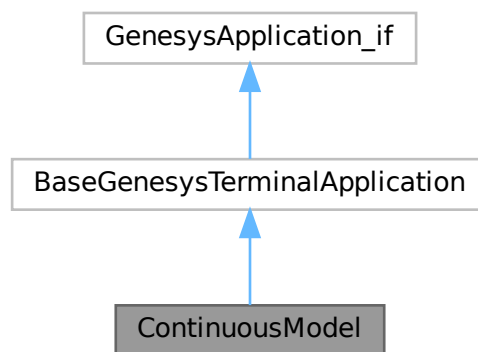
- pdf(x): probability density function value at x.
- cdf(x): cumulative probability $P(X \leq x)$.
- quantile(p): inverse CDF for p in [0, 1].

The documentation for this class was generated from the following file:

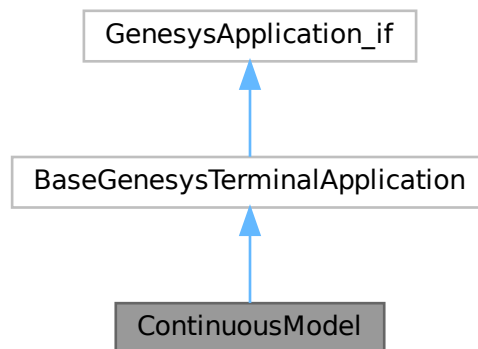
- tools/Statistics/ContinuousDistribution_if.h

15.153 ContinuousModel Class Reference

Inheritance diagram for ContinuousModel:



Collaboration diagram for ContinuousModel:



Public Member Functions

- virtual void [Simulate](#) ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual int [main](#) (int argc, char **argv) override
- void [setDefaultTraceHandlers](#) ([TraceManager](#) *tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void [traceHandler](#) ([TraceEvent](#) e)
- virtual void [traceErrorHandler](#) ([TraceErrorEvent](#) e)
- virtual void [traceReportHandler](#) ([TraceEvent](#) e)
- virtual void [traceSimulationHandler](#) ([TraceSimulationEvent](#) e)
- virtual void [onAfterProcessEventHandler](#) ([SimulationEvent](#) *re)
- virtual void [onBreakpointHandler](#) ([SimulationEvent](#) *re)
- virtual void [onEntityCreateHandler](#) ([SimulationEvent](#) *re)
- virtual void [onEntityMoveHandler](#) ([SimulationEvent](#) *re)
- virtual void [onSimulationStartHandler](#) ([SimulationEvent](#) *re)
- virtual void [onReplicationStepHandler](#) ([SimulationEvent](#) *re)
- virtual void [onReplicationStartHandler](#) ([SimulationEvent](#) *re)
- virtual void [onProcessEventHandler](#) ([SimulationEvent](#) *re)
- virtual void [onReplicationEndHandler](#) ([SimulationEvent](#) *re)
- virtual void [onSimulationEndHandler](#) ([SimulationEvent](#) *re)
- virtual void [onSimulationPausedHandler](#) ([SimulationEvent](#) *re)
- virtual void [onSimulationResumeHandler](#) ([SimulationEvent](#) *re)
- virtual void [onEntityRemoveHandler](#) ([SimulationEvent](#) *re)

15.153.1 Member Function Documentation

15.153.1.1 Simulate()

```
void ContinuousModel::Simulate (
    Simulator * genesys,
    Model * model,
    PluginManager * plugins ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

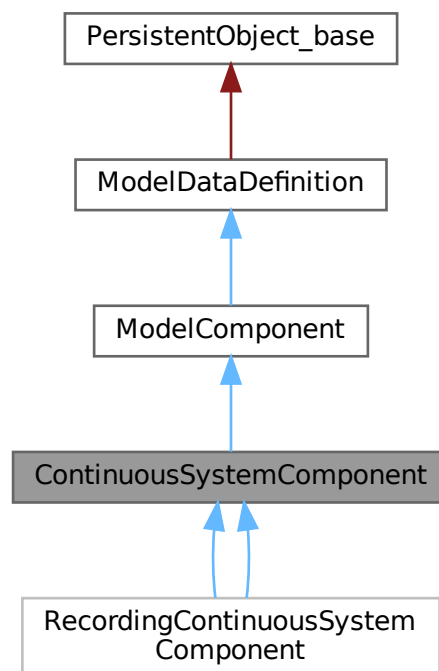
- applications/modelSpecific/teaching/ContinuousModel.h
- applications/modelSpecific/teaching/ContinuousModel.cpp

15.154 ContinuousSystemComponent Class Reference

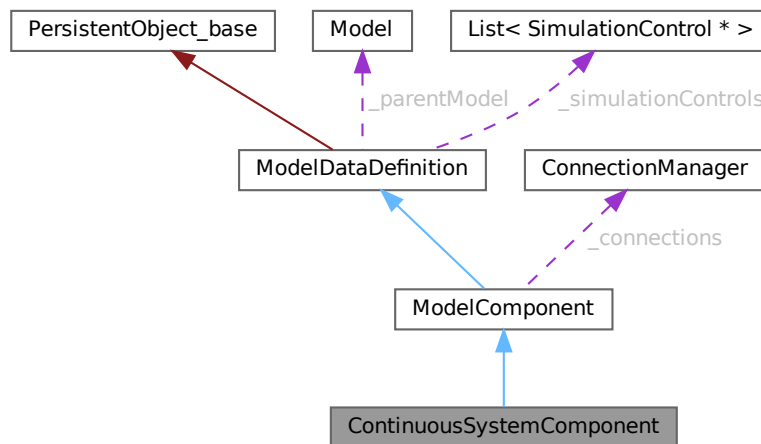
Component that advances a continuous ODE system when entities arrive.

```
#include <plugins/components/Continuous/ContinuousSystemComponent.h>
```

Inheritance diagram for ContinuousSystemComponent:



Collaboration diagram for ContinuousSystemComponent:



Public Member Functions

- **ContinuousSystemComponent** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
- *virtual public methods*
- void **setOdeSolver** ([ODESolver](#) *solver)
- [ODESolver](#) * **getOdeSolver** () const
- void **setOdeSolverName** (std::string name)
- std::string **getOdeSolverName** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
- *constructors*
- virtual ~**ModelComponent** ()
- *Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * **getConnectionManager** () const
- *new public user methods for this component*
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
- *Connects this component to another component.*
- bool **hasBreakpointAt** ()
- *Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string _description)
- *Updates the free-form description shown by the GUI.*
- std::string **getDescription** () const
- *Returns the free-form description associated with this component.*
- void **remove** ([ModelComponent](#) *comp)
- *Removes a connected component from this component.*

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool **hasChanged**)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- **List**< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
- static bool **Check** ([ModelComponent](#) *component)
- static void **CreateInternalData** ([ModelComponent](#) *component)
- static void **DispatchEvent** ([Event](#) *event)

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_createEditableDataDefinitions** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
 - legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

15.154.1 Detailed Description

Component that advances a continuous ODE system when entities arrive.

[ContinuousSystemComponent](#) holds a reference to an [ODESolver](#) data definition and triggers integration each time an entity enters through port 0. The solved state is written back to the [ODESolver](#) so downstream components can read the updated values. The entity is forwarded through output port 0 after integration completes.

15.154.2 Member Function Documentation

15.154.2.1 `_check()`

```
bool ContinuousSystemComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.154.2.2 `_createEditableDataDefinitions()`

```
void ContinuousSystemComponent::_createEditableDataDefinitions ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.154.2.3 `_loadInstance()`

```
bool ContinuousSystemComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.154.2.4 `_onDispatchEvent()`

```
void ContinuousSystemComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

Reimplemented in [RecordingContinuousSystemComponent](#), and [RecordingContinuousSystemComponent](#).

15.154.2.5 `_saveInstance()`

```
void ContinuousSystemComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.154.2.6 `show()`

```
std::string ContinuousSystemComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- `plugins/components/Continuous/ContinuousSystemComponent.h`
- `plugins/components/Continuous/ContinuousSystemComponent.cpp`

15.155 `testing::internal::ConvertibleToIntegerPrinter` Struct Reference

Static Public Member Functions

- static void **PrintValue** (`internal::BiggestInt` value, `::std::ostream *os`)

The documentation for this struct was generated from the following file:

- `gtest/gtest/gtest-printers.h`

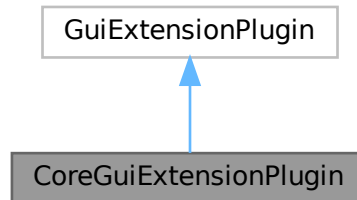
15.156 `testing::internal::ConvertibleToStringViewPrinter` Struct Reference

The documentation for this struct was generated from the following file:

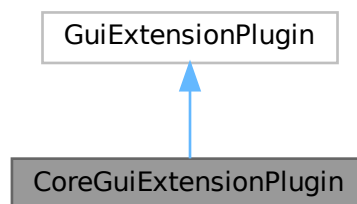
- `gtest/gtest/gtest-printers.h`

15.157 CoreGuiExtensionPlugin Class Reference

Inheritance diagram for CoreGuiExtensionPlugin:



Collaboration diagram for CoreGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId ()` const override
- `std::string displayName ()` const override
- `std::string version ()` const override
- `std::vector< std::string > requiredModelPlugins ()` const override
- `void registerContributions (GuiExtensionRegistry *registry)` const override

15.157.1 Member Function Documentation

15.157.1.1 displayName()

`std::string CoreGuiExtensionPlugin::displayName ( )` const [inline], [override], [virtual]
 Implements [GuiExtensionPlugin](#).

15.157.1.2 extensionId()

`std::string CoreGuiExtensionPlugin::extensionId ( )` const [inline], [override], [virtual]
 Implements [GuiExtensionPlugin](#).

15.157.1.3 registerContributions()

`void CoreGuiExtensionPlugin::registerContributions (`
 `GuiExtensionRegistry * registry )` const [inline], [override], [virtual]

Implements [GuiExtensionPlugin](#).

15.157.1.4 requiredModelPlugins()

```
std::vector< std::string > CoreGuiExtensionPlugin::requiredModelPlugins ( ) const [inline],
[override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.157.1.5 version()

```
std::string CoreGuiExtensionPlugin::version ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

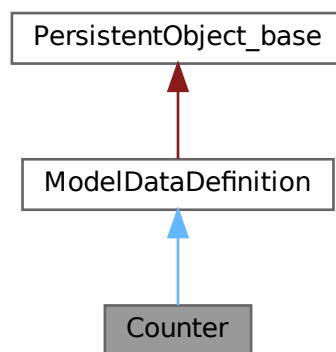
- applications/gui/genesys/extensions/CoreGuiExtensionPlugin.cpp

15.158 Counter Class Reference

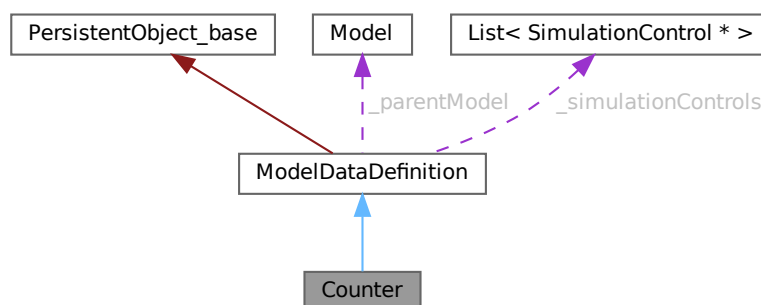
[Model](#) datum used to accumulate event counts (or weighted counts).

```
#include <kernel/simulator/essentialPlugins/Counter.h>
```

Inheritance diagram for Counter:



Collaboration diagram for Counter:



Public Member Functions

- **Counter** ([Model](#) *model, std::string name="", [ModelDataDefinition](#) *parent=nullptr)
Creates a counter model datum with optional parent relationship.
- virtual std::string [show](#) () override
Returns a textual representation of this data definition.
- void [clear](#) ()
clear
- void [incCountValue](#) (double value=1.0)
incCountValue
- double [getCountValue](#) () const
getCountValue
- [ModelDataDefinition](#) * [getParent](#) () const
getParent

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionType, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.

- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List< SimulationControl * > * getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelspecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool _getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * _getParserChangesInformation ()`
- `virtual void _addSimulationControl (SimulationControl *control)`
- `virtual void _createInternalAndAttachedData ()`
- `virtual void _createInternalStatisticReporters ()`
- `virtual void _createAttachedAttributes ()`
- `virtual void _createNonEditableDataDefinitions ()`
- `virtual void _createEditableDataDefinitions ()`
- `void _templateCreateInternalStatisticReporters ()`
- `void _templateCreateAttachedAttributes ()`
- `void _templateCreateNonEditableDataDefinitions ()`
- `void _templateCreateEditableDataDefinitions ()`
- `void _internalStatisticReportersClear ()`
legacy helper methods used by the granular create hooks*
- `void _internalDataClear ()`
- `void _internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void _internalDataRemove (const std::string &key)`
- `void _attachedDataClear ()`
- `void _attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void _attachedDataRemove (const std::string &key)`
- `void _attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void _statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void _statisticReporterRemove (const std::string &key)`
- `void _statisticReportersClear ()`
- `void _mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryAttachedAttributeRemove (const std::string &key)`
- `void _mandatoryAttachedAttributesClear ()`
- `void _mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryEditableDataDefinitionsClear ()`
- `void _optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalEditableDataDefinitionsClear ()`
- `void _mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryNonEditableDataDefinitionsClear ()`
- `void _optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalNonEditableDataDefinitionsClear ()`
- `void _checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
Traces text to the simulation output.
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.158.1 Detailed Description

[Model](#) datum used to accumulate event counts (or weighted counts).

The counter value starts cleared for each replication (depending on model lifecycle hooks) and can be incremented by arbitrary amounts.

15.158.2 Member Function Documentation

15.158.2.1 `_check()`

```
bool Counter::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.158.2.2 `_initBetweenReplications()`

```
void Counter::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.158.2.3 `_loadInstance()`

```
bool Counter::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.158.2.4 `_saveInstance()`

```
void Counter::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.158.2.5 `clear()`

```
void Counter::clear ( )
clear
```

Resets the internal count value to zero.

15.158.2.6 getCountValue()

```
double Counter::getCountValue ( ) const  
getCountValue
```

Returns

Current accumulated count value.

15.158.2.7 getParent()

```
ModelDataDefinition * Counter::getParent ( ) const  
getParent
```

Returns

Parent model datum associated with this counter (if any).

15.158.2.8 incCountValue()

```
void Counter::incCountValue (   
    double value = 1.0 )  
incCountValue
```

Parameters

<i>value</i>	
--------------	--

Adds *value* to the internal count (default increment is 1).

15.158.2.9 show()

```
std::string Counter::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

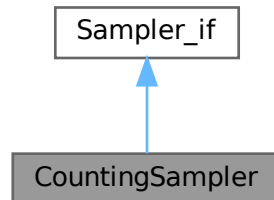
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

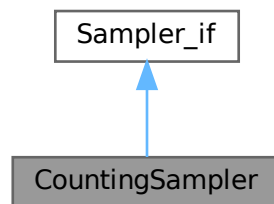
- kernel/simulator/essentialPlugins/Counter.h
- kernel/simulator/essentialPlugins/Counter.cpp

15.159 CountingSampler Class Reference

Inheritance diagram for CountingSampler:



Collaboration diagram for CountingSampler:



Public Member Functions

- **CountingSampler** (int *destroyedCounter, bool *destroyedFlag=nullptr)
- double [random](#) () override
Generates a pseudo-random value uniformly distributed in the range [0,1).
- double [sampleBeta](#) (double, double, double infLimit, double) override
Samples a Beta-distributed value scaled to an arbitrary output interval.
- double [sampleBeta](#) (double, double) override
Samples a standard Beta-distributed value in [0,1].
- double [sampleErlang](#) (double, int, double offset=0.0) override
Samples from an Erlang distribution using the provided mean and number of phases.
- double [sampleExponential](#) (double, double offset=0.0) override
Samples from an Exponential distribution with the provided mean.
- double [sampleGamma](#) (double, double, double offset=0.0) override
Samples from a Gamma distribution using the provided shape/scale parameters.
- double [sampleGumbell](#) (double mode, double) override
Samples from a Gumbel distribution using mode and scale.
- double [sampleLogNormal](#) (double, double, double offset=0.0) override
Samples from a Lognormal distribution parameterized by mean and standard deviation.
- double [sampleNormal](#) (double mean, double) override

- Samples from a Normal distribution.*
- double [sampleTriangular](#) (double min, double, double) override
Samples from a Triangular distribution.
- double [sampleUniform](#) (double min, double) override
Samples from a continuous Uniform distribution in [min, max].
- double [sampleWeibull](#) (double, double) override
Samples from a Weibull distribution.
- double [sampleBinomial](#) (int, double) override
Samples from a Binomial distribution with `trials` and success probability `p`.
- double [sampleBernoulli](#) (double) override
Samples from a Bernoulli distribution (0/1).
- double [sampleDiscrete](#) (double, double,...) override
Samples from a discrete distribution using a variadic list of probability/value pairs.
- double [sampleDiscrete](#) (double *, double *, int) override
Samples from a discrete distribution using probability/value arrays.
- double [sampleGeometric](#) (double) override
Samples from a Geometric distribution with success probability `p`.
- void [setRNGparameters](#) ([RNG_Parameters](#) *) override
Updates the pseudo-random generator internal parameters.
- [RNG_Parameters](#) * [getRNGparameters](#) () const override
Returns the current pseudo-random generator parameters.

15.159.1 Member Function Documentation

15.159.1.1 [getRNGparameters\(\)](#)

[RNG_Parameters](#) * CountingSampler::getRNGparameters () const [inline], [override], [virtual]

Returns the current pseudo-random generator parameters.

Implements [Sampler_if](#).

15.159.1.2 [random\(\)](#)

double CountingSampler::random () [inline], [override], [virtual]

Generates a pseudo-random value uniformly distributed in the range [0,1).

Returns

Base pseudo-random value used by the other distribution samplers.

Implements [Sampler_if](#).

15.159.1.3 [sampleBernoulli\(\)](#)

double CountingSampler::sampleBernoulli (
double p) [inline], [override], [virtual]

Samples from a Bernoulli distribution (0/1).

Implements [Sampler_if](#).

15.159.1.4 [sampleBeta\(\)](#) [1/2]

double CountingSampler::sampleBeta (
double alpha,
double beta) [inline], [override], [virtual]

Samples a standard Beta-distributed value in [0,1].

Implements [Sampler_if](#).

15.159.1.5 sampleBeta() [2/2]

```
double CountingSampler::sampleBeta (
    double alpha,
    double beta,
    double infLimit,
    double supLimit ) [inline], [override], [virtual]
```

Samples a Beta-distributed value scaled to an arbitrary output interval.

Parameters

<i>alpha</i>	Alpha shape parameter (>0).
<i>beta</i>	Beta shape parameter (>0).
<i>infLimit</i>	Lower bound of the output interval.
<i>supLimit</i>	Upper bound of the output interval.

Returns

Sampled value within [*infLimit*, *supLimit*].

Implements [Sampler_if](#).

15.159.1.6 sampleBinomial()

```
double CountingSampler::sampleBinomial (
    int trials,
    double p ) [inline], [override], [virtual]
```

Samples from a Binomial distribution with *trials* and success probability *p*.

Implements [Sampler_if](#).

15.159.1.7 sampleDiscrete() [1/2]

```
double CountingSampler::sampleDiscrete (
    double * prob,
    double * value,
    int size ) [inline], [override], [virtual]
```

Samples from a discrete distribution using probability/value arrays.

Implements [Sampler_if](#).

15.159.1.8 sampleDiscrete() [2/2]

```
double CountingSampler::sampleDiscrete (
    double prob,
    double value,
    ... ) [inline], [override], [virtual]
```

Samples from a discrete distribution using a variadic list of probability/value pairs.

Implements [Sampler_if](#).

15.159.1.9 sampleErlang()

```
double CountingSampler::sampleErlang (
    double mean,
    int M,
    double offset = 0.0 ) [inline], [override], [virtual]
```

Samples from an Erlang distribution using the provided mean and number of phases.

Implements [Sampler_if](#).

15.159.1.10 sampleExponential()

```
double CountingSampler::sampleExponential (
    double mean,
    double offset = 0.0 ) [inline], [override], [virtual]
```

Samples from an Exponential distribution with the provided mean.

Implements [Sampler_if](#).

15.159.1.11 sampleGamma()

```
double CountingSampler::sampleGamma (
    double alpha,
    double beta,
    double offset = 0.0 ) [inline], [override], [virtual]
```

Samples from a Gamma distribution using the provided shape/scale parameters.

Implements [Sampler_if](#).

15.159.1.12 sampleGeometric()

```
double CountingSampler::sampleGeometric (
    double p ) [inline], [override], [virtual]
```

Samples from a Geometric distribution with success probability p.

Implements [Sampler_if](#).

15.159.1.13 sampleGumbell()

```
double CountingSampler::sampleGumbell (
    double mode,
    double scale ) [inline], [override], [virtual]
```

Samples from a Gumbel distribution using mode and scale.

Implements [Sampler_if](#).

15.159.1.14 sampleLogNormal()

```
double CountingSampler::sampleLogNormal (
    double mean,
    double stddev,
    double offset = 0.0 ) [inline], [override], [virtual]
```

Samples from a Lognormal distribution parameterized by mean and standard deviation.

Implements [Sampler_if](#).

15.159.1.15 sampleNormal()

```
double CountingSampler::sampleNormal (
    double mean,
    double stddev ) [inline], [override], [virtual]
```

Samples from a Normal distribution.

Implements [Sampler_if](#).

15.159.1.16 sampleTriangular()

```
double CountingSampler::sampleTriangular (
    double min,
    double mode,
    double max ) [inline], [override], [virtual]
```

Samples from a Triangular distribution.

Implements [Sampler_if](#).

15.159.1.17 sampleUniform()

```
double CountingSampler::sampleUniform (
    double min,
    double max ) [inline], [override], [virtual]
```

Samples from a continuous Uniform distribution in [min, max].

Implements [Sampler_if](#).

15.159.1.18 sampleWeibull()

```
double CountingSampler::sampleWeibull (
    double alpha,
    double scale ) [inline], [override], [virtual]
```

Samples from a Weibull distribution.

Implements [Sampler_if](#).

15.159.1.19 setRNGparameters()

```
void CountingSampler::setRNGparameters (
    RNG_Parameters * param ) [inline], [override], [virtual]
```

Updates the pseudo-random generator internal parameters.

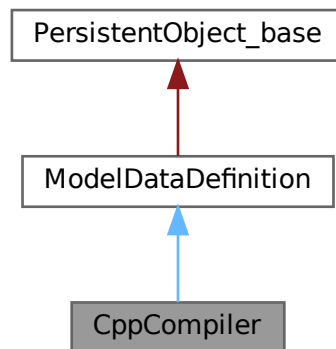
Implements [Sampler_if](#).

The documentation for this class was generated from the following file:

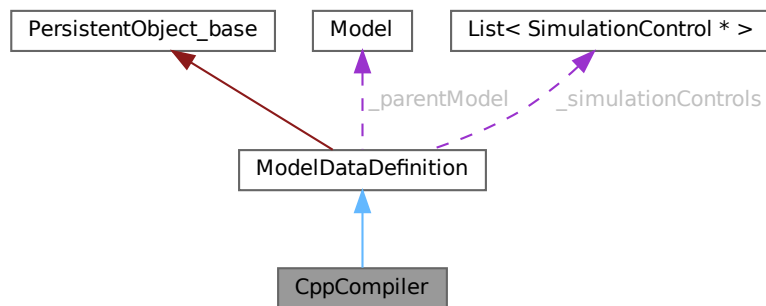
- tests/unit/test_parser_expressions.cpp

15.160 CppCompiler Class Reference

Inheritance diagram for CppCompiler:



Collaboration diagram for CppCompiler:



Classes

- class [CompilationResult](#)

Public Member Functions

- **CppCompiler** ([Model](#) *model, std::string name="")
- virtual std::string [show](#) () override
Returns a textual representation of this data definition.
- void **setSourceFilename** (std::string sourceFilename)
- std::string **getSourceFilename** () const
- bool **IsLibraryLoaded** () const
- void **setFlagsExecutable** (std::string _flagsExecutable)
- std::string **getFlagsExecutable** () const
- void **setLibraryLoaded** (bool libraryLoaded)
- void **setFlagsStaticLibrary** (std::string _flagsStaticLibrary)
- std::string **getFlagsStaticLibrary** () const
- void **setFlagsDynamicLibrary** (std::string _flagsDynamicLibrary)
- std::string **getFlagsDynamicLibrary** () const
- void **setFlagsGeneral** (std::string _flagsGeneral)
- std::string **getFlagsGeneral** () const
- void **setCompilerCommand** (std::string _compilerCommand)
- std::string **getCompilerCommand** () const
- void **setOutputDir** (std::string _outputDir)
- std::string **getOutputDir** () const
- void **setTempDir** (std::string _tempDir)
- std::string **getTempDir** () const
- void **setOutputFilename** (std::string _outputFilename)
- std::string **getOutputFilename** () const
- void **setObjectFiles** (std::string _objectFiles)
- std::string **getObjectFiles** () const
- [CompilationResult](#) **compileToExecutable** ()
- [CompilationResult](#) **compileToDynamicLibrary** ()
- [CompilationResult](#) **compileToStaticLibrary** ()
- bool **loadLibrary** (std::string &errorMessage)
- bool **unloadLibrary** ()
- void * **getDynamicLibraryHandler** () const

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool **hasChanged**)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- **List**< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- [CompilationResult](#) [_invokeCompiler](#) (std::string command)
- std::string [_read](#) (std::string filename)

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)

- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.160.1 Member Function Documentation

15.160.1.1 `_check()`

```
bool CppCompiler::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Validate minimal compiler configuration before use.

Reimplemented from [ModelDataDefinition](#).

15.160.1.2 `_initBetweenReplications()`

```
void CppCompiler::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.160.1.3 `_loadInstance()`

```
bool CppCompiler::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Load persisted compiler configuration.

The structure intentionally follows the same pattern used by other [ModelDataDefinition](#) implementations to simplify future maintenance.

Reimplemented from [ModelDataDefinition](#).

15.160.1.4 `_saveInstance()`

```
void CppCompiler::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Persist compiler configuration.

Keep field names symmetric to [_loadInstance](#) for predictable serialization behavior and easier backward-compatible migrations.

Reimplemented from [ModelDataDefinition](#).

15.160.1.5 `show()`

```
std::string CppCompiler::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

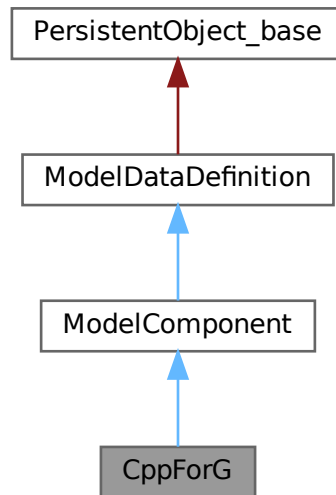
The documentation for this class was generated from the following files:

- `plugins/data/ExternalIntegration/CppCompiler.h`
- `plugins/data/ExternalIntegration/CppCompiler.cpp`

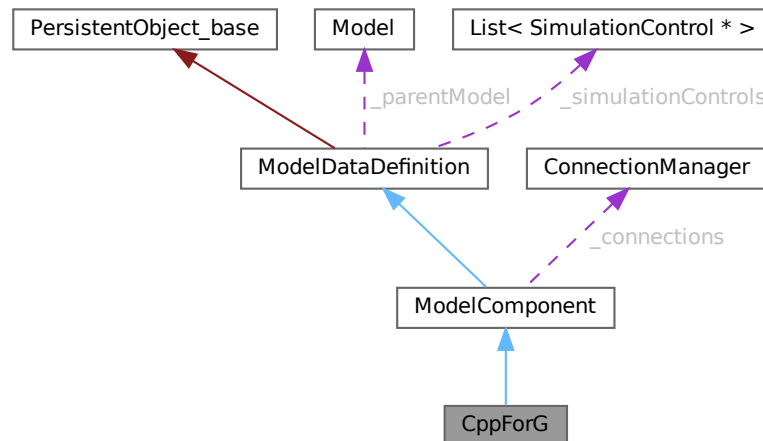
15.161 CppForG Class Reference

```
#include <plugins/components/ExternalIntegration/CppForG.h>
```

Inheritance diagram for CppForG:



Collaboration diagram for CppForG:



Public Member Functions

- **CppForG** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
virtual public methods
- void **setCppCompiler** ([CppCompiler](#) * _cppCompiler)

- [CppCompiler](#) * **getCppCompiler** () const
- void **setInitBetweenReplicationCode** (std::string _initBetweenReplicationCode)
- std::string **getInitBetweenReplicationCode** () const
- void **setOnDispatchEventCode** (std::string _onDispatchEventCode)
- std::string **getOnDispatchEventCode** () const
- void **setIncludesCode** (std::string _includesCode)
- std::string **getIncludesCode** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- [Model](#) * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.

- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) (TraceManager::Level level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_createNonEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)

- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()

virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.161.1 Detailed Description

This component ...

15.161.2 Member Function Documentation

15.161.2.1 `_check()`

```
bool CppForG::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.161.2.2 `_createNonEditableDataDefinitions()`

```
void CppForG::_createNonEditableDataDefinitions ( ) [override], [protected], [virtual]
```

This method is necessary only for those components that instantiate internal elements that must exist before simulation starts and even before model checking. That's the case of components that have internal `StatisticsCollectors`, since others components may refer to them as expressions (as in "TVAG(ThisCSTAT)") and therefore the model datum must exist before checking such expression

Reimplemented from [ModelDataDefinition](#).

15.161.2.3 `_initBetweenReplications()`

```
void CppForG::_initBetweenReplications ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this `ModelDats`. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dinamically linked to to simulator kernel to reflect the changes

Reimplemented from [ModelDataDefinition](#).

15.161.2.4 `_loadInstance()`

```
bool CppForG::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.161.2.5 `_onDispatchEvent()`

```
void CppForG::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for `ModelComponents`, not for `ModelDataElements`.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.161.2.6 `_saveInstance()`

```
void CppForG::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.161.2.7 show()

```
std::string CppForG::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/ExternalIntegration/CppForG.h
- plugins/components/ExternalIntegration/CppForG.cpp

15.162 CppModelExporter Class Reference

Service responsible for C++ code representation of the current simulation model.

```
#include <applications/gui/genesys/services/CppModelExporter.h>
```

Public Member Functions

- **CppModelExporter** ([Simulator](#) *simulator, QPlainTextEdit *cppCodeEditor)
Creates the exporter service for the C++ model-representation pane.
- std::string [addCppCodeLine](#) (const std::string &line, unsigned int indent=0) const
Formats one C++ output line with the requested indentation level.
- void **actualizeModelCppCode** () const
Rebuilds the full C++ model representation shown in the GUI editor pane.

15.162.1 Detailed Description

Service responsible for C++ code representation of the current simulation model.

The refactoring keeps C++ representation generation outside [MainWindow](#) by delegating this concern to a dedicated service. [MainWindow](#) wrappers call into this class when model or UI state changes require regeneration.

Responsibilities:

- format indented C++ output lines used by exporter routines;
- rebuild the full C++ representation shown in the GUI code pane.

Boundaries:

- it does not compile or execute generated code;
- it does not persist files directly;
- it does not manage lifecycle/simulation/scene controller flows.

15.162.2 Member Function Documentation

15.162.2.1 addCppCodeLine()

```
std::string CppModelExporter::addCppCodeLine (
    const std::string & line,
    unsigned int indent = 0 ) const
```

Formats one C++ output line with the requested indentation level.

Returns

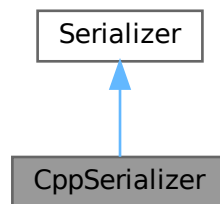
Formatted line that can be appended to the representation buffer.

The documentation for this class was generated from the following files:

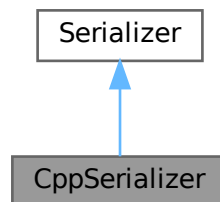
- applications/gui/genesys/services/CppModelExporter.h
- applications/gui/genesys/services/CppModelExporter.cpp

15.163 CppSerializer Class Reference

Inheritance diagram for CppSerializer:



Collaboration diagram for CppSerializer:



Public Member Functions

- **CppSerializer** ([Model](#) *model)
- [PersistenceRecord](#) * [newPersistenceRecord](#) () override
Creates an empty persistence record.
- bool [dump](#) (std::ostream &output) override
Writes formatted contents to a given output stream.
- bool [load](#) (std::istream &input) override
Attempts to load contents from a given input stream.
- bool [get](#) (const std::string &name, [PersistenceRecord](#) *entry) override
Tries to fetch an entry with the given label as a key.
- bool [put](#) (const std::string name, const std::string type, const Util::identification id, [PersistenceRecord](#) *fields) override

Upserts an entry into this structure.

- `int` [for_each](#) (`std::function< int(const std::string &) > delegate`) `override`

Iterates over this structure's entries, ordered by ID.

15.163.1 Member Function Documentation

15.163.1.1 dump()

```
bool CppSerializer::dump (
    std::ostream & output ) [override], [virtual]
```

Writes formatted contents to a given output stream.

Parameters

<i>output</i>	output stream to write to.
---------------	----------------------------

Returns

whether the current state of this structure's contents was deemed valid for serialization.

Implements [Serializer](#).

15.163.1.2 for_each()

```
int CppSerializer::for_each (
    std::function< int(const std::string &) > delegate ) [override], [virtual]
```

Iterates over this structure's entries, ordered by ID.

Parameters

<i>delegate</i>	delegate which is applied to every entry-identifying key, until it returns a non-zero value.
-----------------	--

Returns

either zero, or the first non-zero value returned by the delegate.

Implements [Serializer](#).

15.163.1.3 get()

```
bool CppSerializer::get (
    const std::string & name,
    PersistenceRecord * entry ) [override], [virtual]
```

Tries to fetch an entry with the given label as a key.

Parameters

<i>name</i>	a label which uniquely identifies an entry.
<i>entry</i>	(out-param) a pointer to a caller-owned entry, which is filled on success.

Returns

whether the entry was found and the out-param was actually written to.

Implements [Serializer](#).

15.163.1.4 load()

```
bool CppSerializer::load (
```

```
std::istream & input ) [override], [virtual]
```

Attempts to load contents from a given input stream.

Returns

whether the input was well-formatted and loading was successful.

Implements [Serializer](#).

15.163.1.5 newPersistenceRecord()

```
PersistenceRecord * CppSerializer::newPersistenceRecord ( ) [override], [virtual]
```

Creates an empty persistence record.

Although the returned record can be freed normally with `delete`, care must be taken to only use records which come from a implementation, when configuring that implementation's contents.

Implements [Serializer](#).

15.163.1.6 put()

```
bool CppSerializer::put (
    const std::string name,
    const std::string type,
    const Util::identification id,
    PersistenceRecord * fields ) [override], [virtual]
```

Upserts an entry into this structure.

Parameters

<i>name</i>	a label which uniquely identifies the entry.
<i>type</i>	a type name.
<i>id</i>	a component/datadef ID number.
<i>fields</i>	a persistence record for the entry, whose contents will be copied if valid.

Returns

whether the entry was valid and was actually copied into this structure.

Implements [Serializer](#).

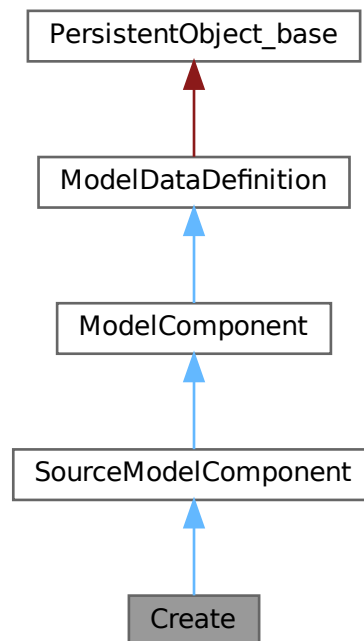
The documentation for this class was generated from the following files:

- kernel/simulator/persistence/CppSerializer.h
- kernel/simulator/persistence/CppSerializer.cpp

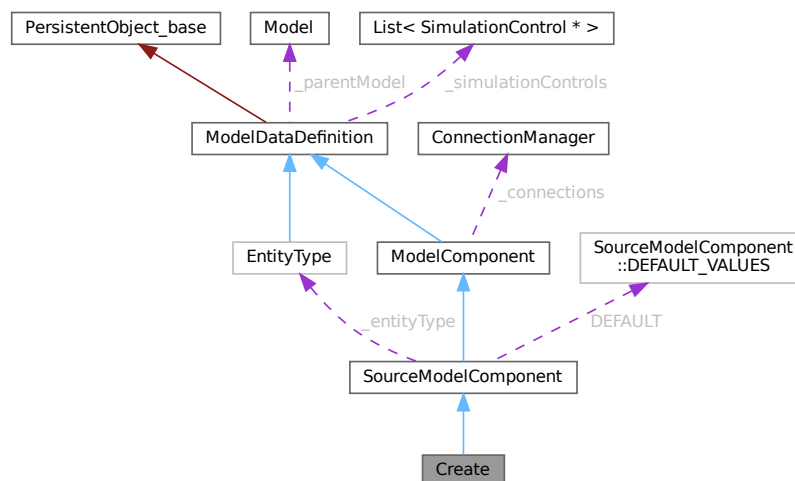
15.164 Create Class Reference

```
#include <plugins/components/Logic/Create.h>
```

Inheritance diagram for Create:



Collaboration diagram for Create:



Public Member Functions

- **Create** ([Model](#) *model, std::string name="")
 - virtual std::string [show](#) () override
- virtual public methods*

- void **setTimeBetweenCreationsFormula** ([Formula](#) * _timeBetweenCreationsFormula)
- [Formula](#) * **getTimeBetweenCreationsFormula** () const
- void **setTimeBetweenCreationsSchedule** ([Schedule](#) * _timeBetweenCreationsSchedule)
- [Schedule](#) * **getTimeBetweenCreationsSchedule** () const
- double **testePropertyCreateDouble** () const
- void **setTestePropertyCreateDouble** (double newTestePropertyCreateDouble)

Public Member Functions inherited from [SourceModelComponent](#)

- **SourceModelComponent** ([Model](#) *model, std::string componentTypename, std::string name="")
setFirstCreation
- void **setFirstCreation** (double _firstCreation)
getFirstCreation
- double **getFirstCreation** () const
getFirstCreation
- void **setEntityType** ([EntityType](#) * _entityType)
setEntityType
- void **setEntityType** (std::string entityType) *setEntityType*
- [EntityType](#) * **getEntityType** () const
getEntityType
- void **setTimeUnit** ([Util::TimeUnit](#) _timeUnit)
setTimeUnit
- [Util::TimeUnit](#) **getTimeUnit** () const
getTimeUnit
- void **setTimeBetweenCreationsExpression** (std::string _timeBetweenCreations, [Util::TimeUnit](#) _timeUnit=[Util::TimeUnit::unknown](#))
setTimeBetweenCreationsExpression
- std::string **getTimeBetweenCreationsExpression** () const
getTimeBetweenCreationsExpression
- void **setMaxCreations** (unsigned long _maxCreations)
setMaxCreations
- void **setMaxCreations** (std::string _maxCreationsExpression)
setMaxCreations
- std::string **getMaxCreations** () const
getMaxCreations
- unsigned int **getEntitiesCreated** () const
getEntitiesCreated
- void **setEntitiesCreated** (unsigned int _entitiesCreated)
setEntitiesCreated
- void **setEntitiesPerCreation** (unsigned int _entitiesPerCreation)
setEntitiesPerCreation
- unsigned int **getEntitiesPerCreation** () const
getEntitiesPerCreation

Public Member Functions inherited from [ModelComponent](#)

- **ModelComponent** ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual **~ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component

- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool **hasChanged**)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

- Returns the simulation controls owned by this data definition.*
- `TraceManager::Level` [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) (`TraceManager::Level` level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (`TraceManager::Level` traceLevelspecific, bool traceLevelSpecific←
Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void `_onDispatchEvent` (`Entity` *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool `_loadInstance` (`PersistenceRecord` *fields) override
virtual protected method that must be overridden
- virtual void `_initBetweenReplications` () override
- virtual void `_saveInstance` (`PersistenceRecord` *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_createInternalStatisticReporters` () override
- virtual void `_createEditableDataDefinitions` () override

Protected Member Functions inherited from `SourceModelComponent`

- virtual void `_createAttachedAttributes` ()

Protected Member Functions inherited from `ModelDataDefinition`

- bool `_getSaveDefaultsOption` ()
- virtual `ParserChangesInformation` * `_getParserChangesInformation` ()
- virtual void `_addSimulationControl` (`SimulationControl` *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createNonEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, `ModelDataDefinition` *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)

- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SourceModelComponent](#)

- [EntityType](#) * **_entityType** = nullptr
- const struct [SourceModelComponent::DEFAULT_VALUES](#) **DEFAULT**
- double **_firstCreation** = DEFAULT.firstCreation
- unsigned int **_entitiesPerCreation** = DEFAULT.entitiesPerCreation
- std::string **_maxCreationsExpression** = DEFAULT.maxCreationsExpression
- std::string **_timeBetweenCreationsExpression** = DEFAULT.timeBetweenCreationsExpression
- [Util::TimeUnit](#) **_timeBetweenCreationsTimeUnit** = DEFAULT.timeBetweenCreationsTimeUnit
- unsigned int **_entitiesCreatedSoFar** = 0

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- [Util::identification](#) **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.164.1 Detailed Description

[Create](#) is the most basic component to include the first entities into the model, and therefore is a source component (derived from [SourceModelComponent](#)) [Create](#) module DESCRIPTION This module is intended as the starting point for entities in a simulation model. Entities are created using a schedule or based on a time between arrivals. Entities then leave the module to begin processing through the system. The entity type is specified in this module. TYPICAL USES The start of a part's production in a manufacturing line A document's arrival (for example, order, check, application) into a business process A customer's arrival at a service process (for example, retail store, restaurant, information desk) PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. [Entity](#) Type Name of the entity type to be generated. Type Type of arrival stream to be generated. Types include

Random (uses an exponential distribution, user specifies mean), [Schedule](#) (uses an exponential distribution, mean determined from the specified [Schedule](#) module), Constant (user specifies constant value; for example, 100), or Expression (drop-down list of various distributions). Value Determines the mean of the exponential distribution (if Random is used) or the constant value (if Constant is used) for the time between arrivals. Applies only when Type is Random or Constant. [Schedule](#) Name Identifies the name of the schedule to be used. The schedule defines the arrival pattern for entities arriving to the system. Applies only when Type is [Schedule](#). Expression Any distribution or value specifying the time between arrivals. Applies only when Type is Expression. Units Time units used for interarrival and first creation times. Does not apply when Type is [Schedule](#). Entities per Arrival Number of entities that will enter the system at a given time with each arrival. Max Arrivals Maximum number of entities that this module will generate. When this value is reached, the creation of new entities by this module ceases. First Creation Starting time for the first entity to arrive into the system. Does not apply when Type is [Schedule](#).

15.164.2 Member Function Documentation

15.164.2.1 `_check()`

```
bool Create::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [SourceModelComponent](#).

15.164.2.2 `_createEditableDataDefinitions()`

```
void Create::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [SourceModelComponent](#).

15.164.2.3 `_createInternalStatisticReporters()`

```
void Create::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.164.2.4 `_initBetweenReplications()`

```
void Create::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [SourceModelComponent](#).

15.164.2.5 `_loadInstance()`

```
bool Create::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SourceModelComponent](#).

15.164.2.6 `_onDispatchEvent()`

```
void Create::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.164.2.7 _saveInstance()

```
void Create::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SourceModelComponent](#).

15.164.2.8 show()

```
std::string Create::show ( ) [override], [virtual]
virtual public methods
```

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [SourceModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/Logic/Create.h
- plugins/components/Logic/Create.cpp

15.165 SimulatorSessionService::CreateSessionResult Struct Reference

Contains session creation identifiers returned to API clients.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Public Attributes

- std::string **sessionId**
- std::string **accessToken**

15.165.1 Detailed Description

Contains session creation identifiers returned to API clients.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

15.166 QtBoolPropertyManagerPrivate::Data Struct Reference

Public Attributes

- bool **val**
- bool **initVal**
- bool **isInitialed**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.167 QtDatePropertyManagerPrivate::Data Struct Reference

Public Member Functions

- QDate **minimumValue** () const
- QDate **maximumValue** () const
- void **setMinimumValue** (QDate newMinVal)
- void **setMaximumValue** (QDate newMaxVal)

Public Attributes

- QDate **val** { QDate::currentDate() }
- QDate **minVal** { QDate(1752, 9, 14) }
- QDate **maxVal** { QDate(9999, 12, 31) }
- QDate **initVal** { QDate::currentDate() }
- bool **isInitialed**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.168 QDateTimePropertyManagerPrivate::Data Struct Reference

Public Attributes

- QDateTime **val** { QDateTime() }
- QDateTime **initVal** { QDateTime() }
- bool **isInitialed** { false }

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.169 QtDoublePropertyManagerPrivate::Data Struct Reference

Public Member Functions

- double **minimumValue** () const
- double **maximumValue** () const
- void **setMinimumValue** (double newMinVal)
- void **setMaximumValue** (double newMaxVal)

Public Attributes

- double **val** { 0 }
- double **initVal** { 0 }
- double **minVal** { -DBL_MAX }
- double **maxVal** { DBL_MAX }
- double **singleStep** { 1 }
- int **decimals** { 2 }
- bool **isInitialed** { false }

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.170 QtEnumPropertyManagerPrivate::Data Struct Reference

Public Attributes

- int **val** { -1 }
- QStringList **enumNames**
- QMap< int, QIcon > **enumIcons**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.171 QtFlagPropertyManagerPrivate::Data Struct Reference

Public Attributes

- int **val** { -1 }
- QStringList **flagNames**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.172 QtIntPropertyManagerPrivate::Data Struct Reference

Public Member Functions

- int **minimumValue** () const
- int **maximumValue** () const
- void **setMinimumValue** (int newMinVal)
- void **setMaximumValue** (int newMaxVal)

Public Attributes

- int **val** { 0 }
- int **initVal** { 0 }
- int **minVal** { -INT_MAX }
- int **maxVal** { INT_MAX }
- int **singleStep** { 1 }
- bool **isInitialed** { false }

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.173 QtPointFPropertyManagerPrivate::Data Struct Reference

Public Attributes

- QPointF **val**
- int **decimals** { 2 }

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.174 QtRectFPropertyManagerPrivate::Data Struct Reference

Public Attributes

- QRectF **val** { 0, 0, 0, 0 }
- QRectF **constraint**
- int **decimals** { 2 }

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.175 QtRectPropertyManagerPrivate::Data Struct Reference

Public Attributes

- QRect **val** { 0, 0, 0, 0 }
- QRect **constraint**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.176 QSizeFPropertyManagerPrivate::Data Struct Reference

Public Member Functions

- QSizeF **minimumValue** () const
- QSizeF **maximumValue** () const
- void **setMinimumValue** (const QSizeF &newMinVal)
- void **setMaximumValue** (const QSizeF &newMaxVal)

Public Attributes

- QSizeF **val** { 0, 0 }
- QSizeF **minVal** { 0, 0 }
- QSizeF **maxVal** { std::numeric_limits<qreal>::max(), std::numeric_limits<qreal>::max() }
- int **decimals** { 2 }

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.177 QSizePropertyManagerPrivate::Data Struct Reference

Public Member Functions

- QSize **minimumValue** () const
- QSize **maximumValue** () const
- void **setMinimumValue** (const QSize &newMinVal)
- void **setMaximumValue** (const QSize &newMaxVal)

Public Attributes

- QSize **val** { 0, 0 }
- QSize **minVal** { 0, 0 }
- QSize **maxVal** { INT_MAX, INT_MAX }

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.178 QStringPropertyManagerPrivate::Data Struct Reference

Public Attributes

- QString **val**
- QString **initVal**
- bool **isInitialed**
- QRegularExpression **regExp**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.179 QtTimePropertyManagerPrivate::Data Struct Reference

Public Attributes

- QTime **val** { QTime() }
- QTime **initVal** { QTime() }
- bool **isInitialed** { false }

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.180 DataAnalyser_if Class Reference

High-level façade for applied statistical analysis over datasets.

```
#include <tools/Statistics/DataAnalyser_if.h>
```

Public Member Functions

- virtual bool **loadDataSet** (std::string datafilename)=0
- virtual bool **saveDataSet** (std::string datasetname)=0
- virtual void **newDataSet** (std::string datasetname, std::string datafilename)=0
- virtual [Fitter_if](#) * **fitter** ()=0
- virtual [Sampler_if](#) * **sampler** ()=0
- virtual [ExperimentManager_if](#) * **experimenter** ()=0
- virtual [HypothesisTester_if](#) * **tester** ()=0

15.180.1 Detailed Description

High-level façade for applied statistical analysis over datasets.

Purpose:

- Coordinate dataset lifecycle and analysis services in a single entry point.

Architectural role:

- Facade that orchestrates fitting, sampling, experiment execution and hypothesis testing.

Ownership/lifetime:

- The returned pointers from `fitter()`, `sampler()`, `experimenter()` and `tester()` are non-owning views. Concrete implementations must document whether those services are internally owned singletons, members, or externally injected objects.

Implementation status:

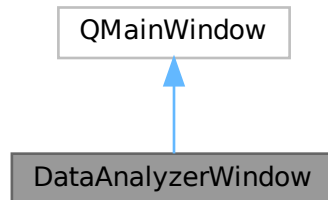
- Interface contract is stable and legacy-compatible.
- A consolidated default concrete implementation is still pending.

The documentation for this class was generated from the following file:

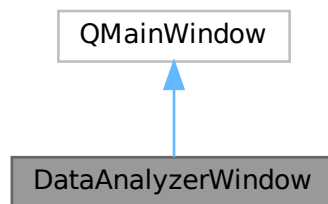
- tools/Statistics/DataAnalyser_if.h

15.181 DataAnalyzerWindow Class Reference

Inheritance diagram for DataAnalyzerWindow:



Collaboration diagram for DataAnalyzerWindow:



Classes

- class [DoePreviewPlot](#)
- class [HistogramPreview](#)

Public Member Functions

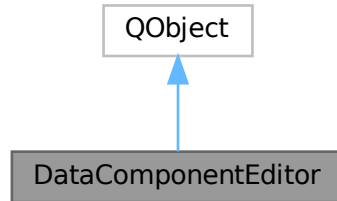
- **DataAnalyzerWindow** ([Simulator](#) *simulator, QString &lastDataAnalyzerPath, QWidget *parent=nullptr)

The documentation for this class was generated from the following files:

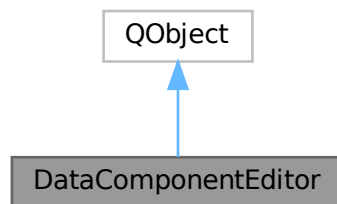
- applications/gui/genesys/tools/dataanalyzer/DataAnalyzerWindow.h
- applications/gui/genesys/tools/dataanalyzer/DataAnalyzerWindow.cpp

15.182 DataComponentEditor Class Reference

Inheritance diagram for DataComponentEditor:



Collaboration diagram for DataComponentEditor:



Public Types

- using **AfterChange** = std::function< void()>

Public Member Functions

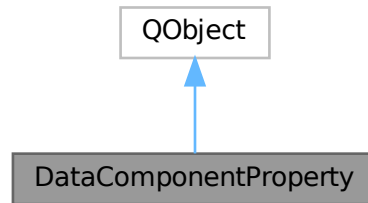
- **DataComponentEditor** ([PropertyEditorGenesys](#) *editor, [SimulationControl](#) *property, AfterChange after↵ Change=({})
- **DataComponentEditor** ([PropertyEditorGenesys](#) *editor, [List](#)< [SimulationControl](#) * > *properties, After↵ Change afterChange=({})
- void **open_window** ([SimulationControl](#) *property)
- void **open_window** ([List](#)< [SimulationControl](#) * > *properties)
- void **configure_properties** ([SimulationControl](#) *property)
- void **configure_properties** ([List](#)< [SimulationControl](#) * > *properties)

The documentation for this class was generated from the following files:

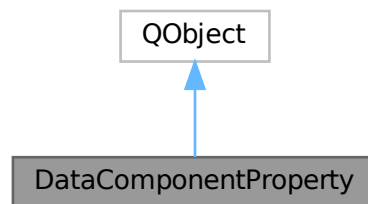
- applications/gui/genesys/propertyeditor/DataComponentEditor.h
- applications/gui/genesys/propertyeditor/DataComponentEditor.cpp

15.183 DataComponentProperty Class Reference

Inheritance diagram for DataComponentProperty:



Collaboration diagram for DataComponentProperty:



Public Types

- using **AfterChange** = std::function< void()>

Public Member Functions

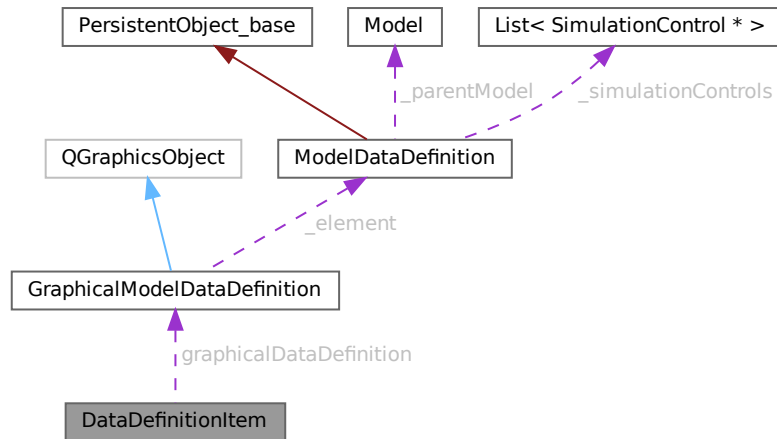
- **DataComponentProperty** ([PropertyEditorGenesys](#) *editor, [SimulationControl](#) *property, bool necessary, Config, AfterChange afterChange={})
- void **open_window** ()
- void **config_values** ()
- bool **isInList** (const std::string &value) const

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/DataComponentProperty.h
- applications/gui/genesys/propertyeditor/DataComponentProperty.cpp

15.184 DataDefinitionItem Struct Reference

Collaboration diagram for DataDefinitionItem:



Public Attributes

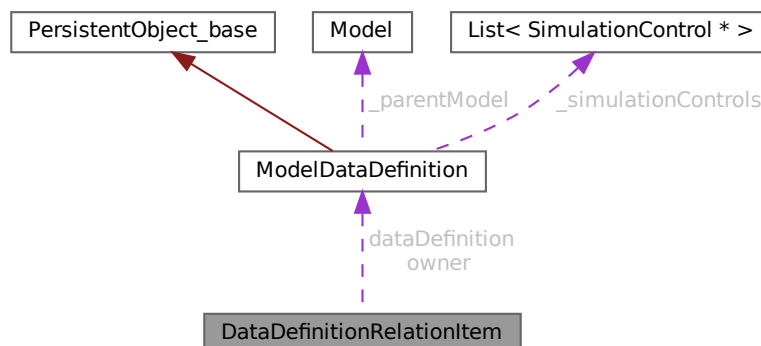
- [GraphicalModelDataDefinition](#) * **graphicalDataDefinition**
- `QPointF` **initialPosition**
- `bool` **initiallySelected**

The documentation for this struct was generated from the following file:

- `applications/gui/genesys/actions/DeleteUndoCommand.h`

15.185 DataDefinitionRelationItem Struct Reference

Collaboration diagram for DataDefinitionRelationItem:



Public Attributes

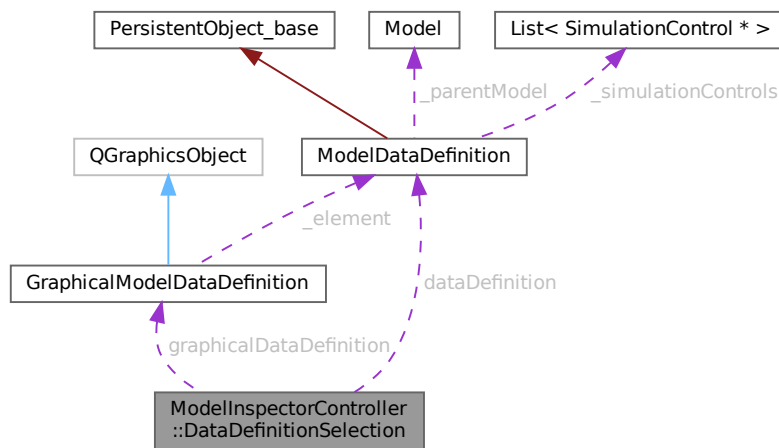
- [ModelDataDefinition](#) * **owner**
- [ModelDataDefinition](#) * **dataDefinition**
- std::string **key**
- bool **internal**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/actions/DeleteUndoCommand.h

15.186 ModelInspectorController::DataDefinitionSelection Struct Reference

Collaboration diagram for ModelInspectorController::DataDefinitionSelection:



Public Attributes

- bool **handled** = false
- [ModelDataDefinition](#) * **dataDefinition** = nullptr
- [GraphicalModelDataDefinition](#) * **graphicalDataDefinition** = nullptr

The documentation for this struct was generated from the following file:

- applications/gui/genesys/controllers/ModelInspectorController.h

15.187 AnimationPlot::DataSeries Struct Reference

Public Attributes

- QString **label**
- QString **expression**
- QColor **color** = QColor(31, 119, 180)
- qreal **lineWidth** = 2.0
- QVector< double > **values**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/animations/AnimationPlaceholder.h

15.188 DataSet_if Class Reference

Abstract view of a numeric dataset.

```
#include <tools/Statistics/DataSet_if.h>
```

Public Member Functions

- virtual unsigned int **size** () const =0
- virtual double **value** (unsigned int index) const =0
- virtual bool **isLoading** () const =0
- virtual std::string **sourceDescription** () const =0

15.188.1 Detailed Description

Abstract view of a numeric dataset.

Purpose:

- Represent sample data loaded in memory or abstracted from persistent source.

Ownership/lifetime:

- Implementations own storage/source handles; clients use this interface as a non-owning access contract.

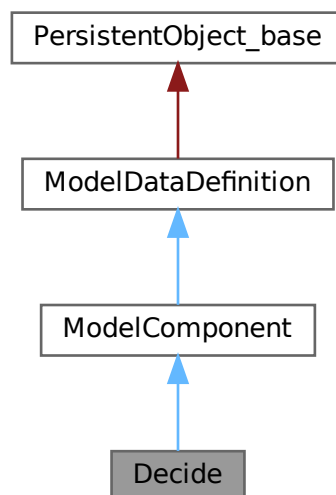
The documentation for this class was generated from the following file:

- tools/Statistics/DataSet_if.h

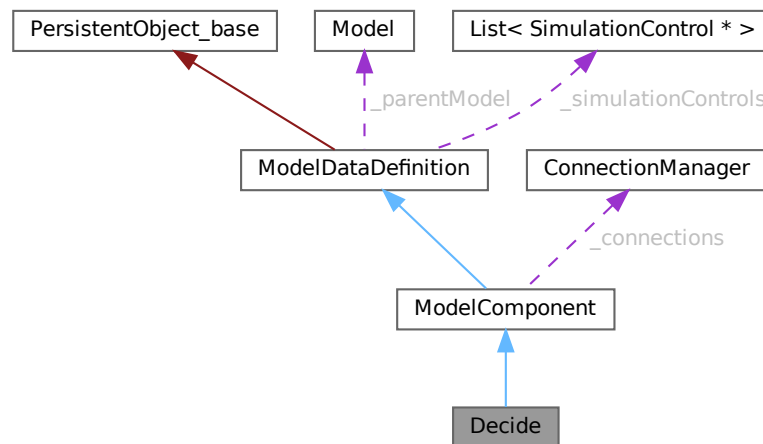
15.189 Decide Class Reference

```
#include <plugins/components/Decisions/Decide.h>
```

Inheritance diagram for Decide:



Collaboration diagram for Decide:



Public Member Functions

- **Decide** ([Model](#) *model, std::string name="")
- [List](#)< std::string > * **getConditions** () const
- void **addConditions** (std::string newCondition)
- void **removeConditions** (std::string condition)
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool **hasChanged**)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- **List**< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_createInternalStatisticReporters** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool _getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * _getParserChangesInformation ()`
- `virtual void _initBetweenReplications ()`
- `virtual void _addSimulationControl (SimulationControl *control)`
- `virtual void _createInternalAndAttachedData ()`
- `virtual void _createAttachedAttributes ()`
- `virtual void _createNonEditableDataDefinitions ()`
- `virtual void _createEditableDataDefinitions ()`
- `void _templateCreateInternalStatisticReporters ()`
- `void _templateCreateAttachedAttributes ()`
- `void _templateCreateNonEditableDataDefinitions ()`
- `void _templateCreateEditableDataDefinitions ()`
- `void _internalStatisticReportersClear ()`
legacy helper methods used by the granular create hooks*
- `void _internalDataClear ()`
- `void _internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void _internalDataRemove (const std::string &key)`
- `void _attachedDataClear ()`
- `void _attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void _attachedDataRemove (const std::string &key)`
- `void _attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void _statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void _statisticReporterRemove (const std::string &key)`
- `void _statisticReportersClear ()`
- `void _mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryAttachedAttributeRemove (const std::string &key)`
- `void _mandatoryAttachedAttributesClear ()`
- `void _mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryEditableDataDefinitionsClear ()`
- `void _optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalEditableDataDefinitionsClear ()`
- `void _mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryNonEditableDataDefinitionsClear ()`
- `void _optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalNonEditableDataDefinitionsClear ()`
- `void _checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
Traces text to the simulation output.
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

15.189.1 Detailed Description

[Decide](#) module DESCRIPTION This module allows for decision-making processes in the system. It includes options to make decisions based on one or more conditions (for example, if entity type is Gold Card) or based on one or more probabilities (for example, 75%, true; 25%, false). Conditions can be based on attribute values (for example, Priority), variable values (for example, Number Denied), the entity type, or an expression (for example, NQ(ProcessA.Queue)). There are two exit points out of the [Decide](#) module when its specified type is either 2-way by Chance or 2-way by Condition. There is one exit point for “true” entities and one for “false” entities. When the N-way by Chance or by Condition type is specified, multiple exit points are shown for each condition or probability and a single “else” exit. The number of entities that exit from each type (true/false) is displayed for 2-way by Chance or by Condition modules only. TYPICAL USES Dispatching a faulty part for rework Branching accepted vs. rejected checks Sending priority customers to a dedicated process Prompt Description Name Unique module identifier displayed on the module shape. Type Indicates whether the decision is based on a condition (if X>Y) or by chance/percentage (for example, 60%, yes; 40%, no). The type can be specified as either 2-way or N-way. 2-way allows for one condition or probability (plus the “false” exit). N-way allows for any number of conditions or probabilities to be specified as well as an “else” exit. Conditions Defines one or more conditions used to direct entities to different modules. Applies only when Type is N-way by Condition. Percentages Defines one or more percentages used to direct entities to different modules. Applies only when Type is N-way by Chance. Percent True Value that will be checked to determine the percentage of entities sent out a given True exit. If Types of conditions that are available for evaluation: [Variable](#), [Variable](#) Array (1D), [Variable](#) Array (2D), [Attribute](#), [Entity](#) Type, Expression. Named Specifies the name of the variable, attribute, or entity type that will be evaluated when an entity enters the module. Does not apply when Type is Expression. Is Evaluator for the condition. Applies only to [Attribute](#) and [Variable](#) conditions. Row Specifies the row index for a variable array. Applies only when Type is N-way by Condition or 2-way by Condition and [Variable](#) is Array 1-D or Array 2-D. Column Specifies the column index for a variable array. Applies only when Type is N-way by Condition or 2-way by Condition and [Variable](#) is Array 1-D or Array 2-D. Value Expression that will be either compared to an attribute or variable or that will be evaluated as a single expression to determine if it is true or false. Does not apply to [Entity](#) Type condition. If Type is Expression, this value must also include the evaluator (for example, Color<>Red).

15.189.2 Member Function Documentation

15.189.2.1 `_check()`

```
bool Decide::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.189.2.2 `_createInternalStatisticReporters()`

```
void Decide::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.189.2.3 `_loadInstance()`

```
bool Decide::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [ModelComponent](#).

15.189.2.4 `_onDispatchEvent()`

```
void Decide::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.189.2.5 `_saveInstance()`

```
void Decide::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.189.2.6 `show()`

```
std::string Decide::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- `plugins/components/Decisions/Decide.h`
- `plugins/components/Decisions/Decide.cpp`

15.190 Delay::DEFAULT_VALUES Struct Reference

Public Attributes

- const std::string **delayExpression** = "1.0"
- const Util::TimeUnit **delayTimeUnit** = Util::TimeUnit::second
- const Util::AllocationType **allocation** = Util::AllocationType::Wait

The documentation for this struct was generated from the following file:

- plugins/components/DiscreteProcessing/Delay.h

15.191 ModalModelDefault::DEFAULT_VALUES Struct Reference

Attributes that should be loaded or saved with this component (Persistent Fields)

```
#include <plugins/components/ModalModel/ModalModelDefault.h>
```

Public Attributes

- const std::string **timeDelayExpressionPerDispatch** = "0"
- const Util::TimeUnit **timeDelayPerDispatchTimeUnit** = Util::TimeUnit::second
- const unsigned int **maxTransitionsPerDispatch** = 1

15.191.1 Detailed Description

Attributes that should be loaded or saved with this component (Persistent Fields)

This method is called by ModelChecker and is necessary only for those components that instantiate internal elements that must exist before simulation starts and even before model checking. That's the case of components that have internal StatisticsCollectors, since others components may refer to them as expressions (as in "TVAG(This←CSTAT)") and therefore the modeldatum must exist before checking such expression

This method is not used yet. It should be usefull for new UIs Default values for the attributes. Used on initing, loading and saving

The documentation for this struct was generated from the following file:

- plugins/components/ModalModel/ModalModelDefault.h

15.192 Seize::DEFAULT_VALUES Struct Reference

Public Attributes

- const Util::AllocationType **allocationType** = Util::AllocationType::Others
- const unsigned short **priority** = 0
- const std::string **priorityExpression** = ""
- const unsigned int **seizeRequestSize** = 1

The documentation for this struct was generated from the following file:

- plugins/components/DiscreteProcessing/Seize.h

15.193 SourceModelComponent::DEFAULT_VALUES Struct Reference

Public Attributes

- const double **firstCreation** = 0.0
- const unsigned int **entitiesPerCreation** = 1
- const std::string **maxCreationsExpression** = std::to_string(std::numeric_limits<unsigned int>::max())
- const std::string **timeBetweenCreationsExpression** = "EXPO(1.0)"
- const Util::TimeUnit **timeBetweenCreationsTimeUnit** = Util::TimeUnit::second

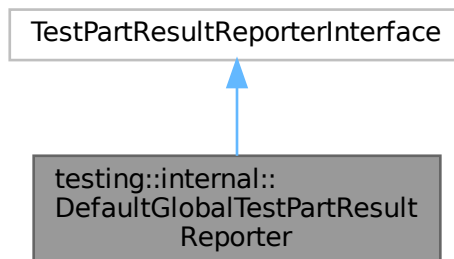
- `const std::string entityTypename = "entitytype"`

The documentation for this struct was generated from the following file:

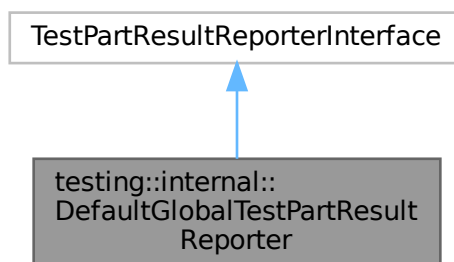
- `kernel/simulator/SourceModelComponent.h`

15.194 testing::internal::DefaultGlobalTestPartResultReporter Class Reference

Inheritance diagram for testing::internal::DefaultGlobalTestPartResultReporter:



Collaboration diagram for testing::internal::DefaultGlobalTestPartResultReporter:



Public Member Functions

- **DefaultGlobalTestPartResultReporter** ([UnitTestImpl](#) *unit_test)
- **ReportTestPartResult** (const TestPartResult &result) override

The documentation for this class was generated from the following files:

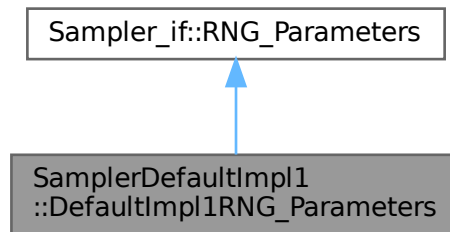
- `gtest/src/gtest-internal-inl.h`
- `gtest/src/gtest.cc`

15.195 SamplerDefaultImpl1::DefaultImpl1RNG_Parameters Struct Reference

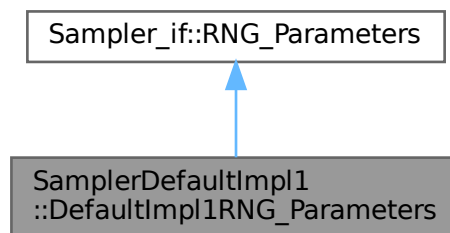
Parameter bundle for the default RNG algorithm.

```
#include <kernel/statistics/SamplerDefaultImpl1.h>
```

Inheritance diagram for SamplerDefaultImpl1::DefaultImpl1RNG_Parameters:



Collaboration diagram for SamplerDefaultImpl1::DefaultImpl1RNG_Parameters:



Public Attributes

- uint32_t **seed** = 16021974
Initial seed/state.
- uint32_t **a** = 279470273u
Multiplicative factor in the linear congruential generator.
- uint32_t **m** = 0xfffffff
Modulus used by the generator.
- uint32_t **c** = 0
Increment term used by the generator.

15.195.1 Detailed Description

Parameter bundle for the default RNG algorithm.

The documentation for this struct was generated from the following file:

- kernel/statistics/SamplerDefaultImpl1.h

15.196 testing::internal::DefaultNameGenerator Struct Reference

Static Public Member Functions

- `template<typename T >`
static std::string **GetName** (int i)

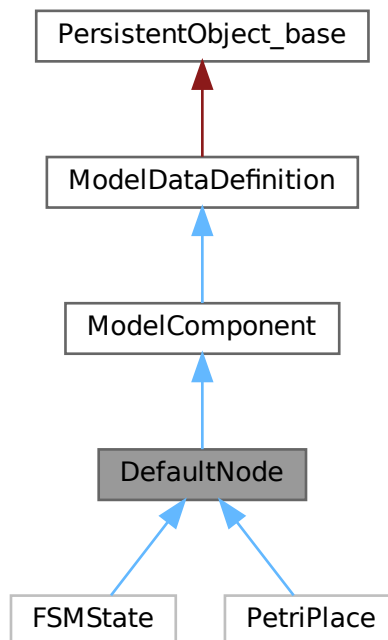
The documentation for this struct was generated from the following file:

- `gtest/gtest/internal/gtest-internal.h`

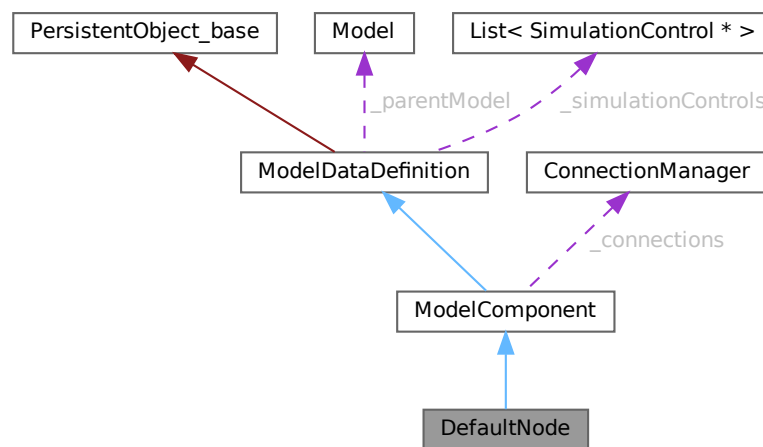
15.197 DefaultNode Class Reference

```
#include <plugins/components/ModalModel/DefaultNode.h>
```

Inheritance diagram for DefaultNode:



Collaboration diagram for DefaultNode:



Public Member Functions

- [DefaultNode](#) ([Model](#) *model, std::string componentTypename, std::string name)

constructors

- **DefaultNode** ([Model](#) *model, std::string name="")
- void **addTransition** ([DefaultNodeTransition](#) *transition)

new public user methods for this component

- void **removeTransition** ([DefaultNodeTransition](#) *transition)
- [List< DefaultNodeTransition * >](#) * **getTransitions** () const
- void **setInitialNode** (bool initialNode)
- bool **isInitialNode** () const
- void **setFinalNode** (bool finalNode)
- bool **isFinalNode** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")

constructors

- virtual **~ModelComponent** ()

Releases component-owned resources and unregisters it from the model.

- [ConnectionManager](#) * **getConnectionManager** () const

new public user methods for this component

- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)

Connects this component to another component.

- bool **hasBreakpointAt** ()

Indicates whether a simulation breakpoint is set on this component.

- void **setDescription** (std::string _description)

Updates the free-form description shown by the GUI.

- std::string **getDescription** () const

Returns the free-form description associated with this component.

- void **remove** ([ModelComponent](#) *comp)

Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()

Releases resources owned by this data definition.
- Util::identification [getId](#) () const

Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)

Updates the logical name of this instance.
- const std::string & [getName](#) () const

Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const

Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const

Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const

Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const

Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const

Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const

Returns the full map of attached data definitions.
- bool [hasChanged](#) () const

Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))

Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const

Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const

Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const

Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)

Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const

Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
 - could be overridden by derived classes*
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
 - legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
 - method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

15.197.1 Detailed Description

This component ...

15.197.2 Constructor & Destructor Documentation

15.197.2.1 DefaultNode()

```
DefaultNode::DefaultNode (
    Model * model,
    std::string componentTypename,
    std::string name )
```

constructors

Externalize function GetPluginInformation to be accessible through dynamic linked library.

15.197.3 Member Function Documentation

15.197.3.1 _loadInstance()

```
bool DefaultNode::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

Reimplemented in [FSMState](#), and [PetriPlace](#).

15.197.3.2 _onDispatchEvent()

```
void DefaultNode::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

This method is only for ModelComponents, not ModelDataElements.

Implements [ModelComponent](#).

15.197.3.3 _saveInstance()

```
void DefaultNode::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

Reimplemented in [FSMState](#), and [PetriPlace](#).

15.197.3.4 show()

```
std::string DefaultNode::show ( ) [override], [virtual]
```

virtual public methods

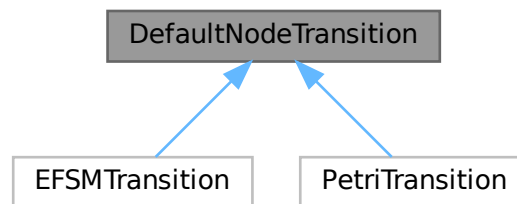
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/ModalModel/DefaultNode.h
- plugins/components/ModalModel/DefaultNode.cpp

15.198 DefaultNodeTransition Class Reference

Inheritance diagram for DefaultNodeTransition:

**Public Types**

- enum class **TransitionKind** : int { **DETERMINISTIC** = 0 , **PROBABILISTIC** = 1 }

Public Member Functions

- **DefaultNodeTransition** ([DefaultNode](#) *source, [DefaultNode](#) *destination, std::string name="")
- void **setSource** ([DefaultNode](#) *source)
- [DefaultNode](#) * **getSource** () const
- void **setDestination** ([DefaultNode](#) *destination)
- [DefaultNode](#) * **getDestination** () const
- void **setName** (std::string name)
- std::string **getName** () const
- void **setGuardExpression** (std::string guardExpression)
- std::string **getGuardExpression** () const
- void **setOutputExpression** (std::string outputExpression)
- std::string **getOutputExpression** () const
- void **setInputEvent** (std::string inputEvent)
- std::string **getInputEvent** () const
- void **setPriority** (unsigned int priority)

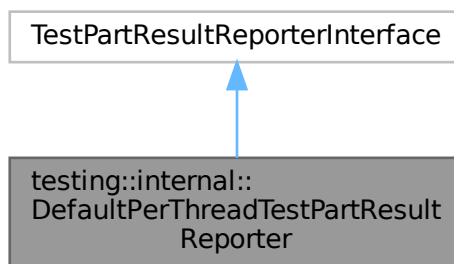
- unsigned int **getPriority** () const
- void **setProbability** (double probability)
- double **getProbability** () const
- void **setTransitionKind** (TransitionKind transitionKind)
- TransitionKind **getTransitionKind** () const
- virtual bool **canFire** (Model *model, Entity *entity) const
- virtual void **execute** (Model *model, Entity *entity) const

The documentation for this class was generated from the following files:

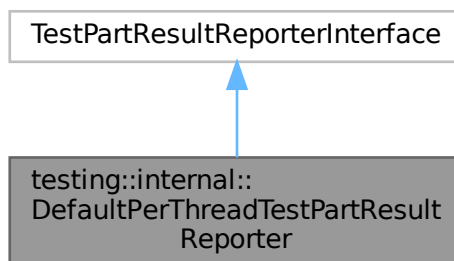
- plugins/components/ModalModel/DefaultNode.h
- plugins/components/ModalModel/DefaultNode.cpp

15.199 testing::internal::DefaultPerThreadTestPartResultReporter Class Reference

Inheritance diagram for testing::internal::DefaultPerThreadTestPartResultReporter:



Collaboration diagram for testing::internal::DefaultPerThreadTestPartResultReporter:



Public Member Functions

- **DefaultPerThreadTestPartResultReporter** (UnitTestImpl *unit_test)
- void **ReportTestPartResult** (const TestPartResult &result) override

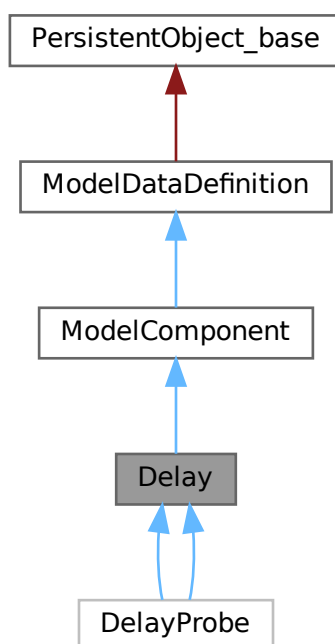
The documentation for this class was generated from the following files:

- gtest/src/gtest-internal-inl.h
- gtest/src/gtest.cc

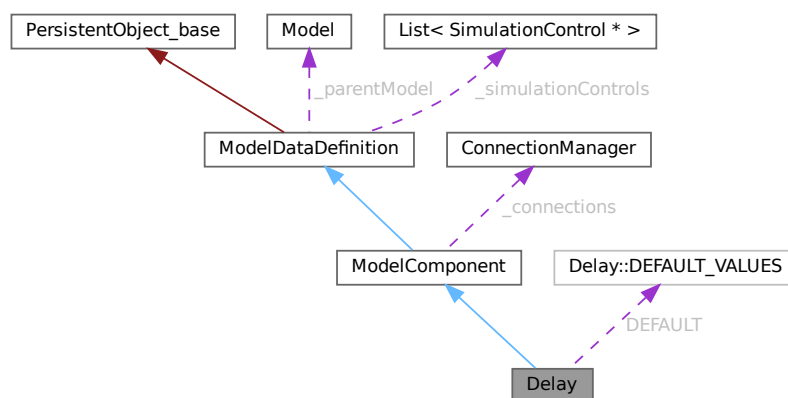
15.200 Delay Class Reference

```
#include <plugins/components/DiscreteProcessing/Delay.h>
```

Inheritance diagram for Delay:



Collaboration diagram for Delay:



Classes

- struct [DEFAULT_VALUES](#)

Public Member Functions

- **Delay** ([Model](#) *model, std::string name="")
 - void **setDelayExpression** (std::string _delayExpression, [Util::TimeUnit](#) _delayTimeUnit=[Util::TimeUnit::unknown](#))
 - std::string **delayExpression** () const
 - void **setDelay** (double delay)
 - double **delay** () const
 - void **setDelayTimeUnit** ([Util::TimeUnit](#) _delayTimeUnit)
 - [Util::TimeUnit](#) **delayTimeUnit** () const
 - void **setAllocation** ([Util::AllocationType](#) allocation)
 - [Util::AllocationType](#) **getAllocation** () const
 - virtual std::string **show** () override
- virtual public methods*

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentName, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionType, std::string name="", bool insertIntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- [Util::identification](#) **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

- Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.

- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Public Attributes

- const struct [Delay::DEFAULT_VALUES](#) **DEFAULT**

Protected Member Functions

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createInternalStatisticReporters](#) () override
- virtual void [_createAttachedAttributes](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internaStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*

- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Friends

- class **DelayProbe**

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.200.1 Detailed Description

[Delay](#) module DESCRIPTION The [Delay](#) module delays an entity by a specified amount of time. When an entity arrives at a [Delay](#) module, the time delay expression is evaluated and the entity remains in the module for the resulting time period. The time is then allocated to the entity's value-added, non-value added, transfer, wait, or other time. Associated costs are calculated and allocated as well. TYPICAL USES Processing a check at a bank Performing a setup on a machine Transferring a document to another department PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Allocation Type of category to which the entity's incurred delay time and cost will be added. [Delay](#) Time Determines the value of the delay for the entity. Units Time units used for the delay time.

15.200.2 Member Function Documentation

15.200.2.1 [_check\(\)](#)

```
bool Delay::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.200.2.2 [_createAttachedAttributes\(\)](#)

```
void Delay::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.200.2.3 [_createInternalStatisticReporters\(\)](#)

```
void Delay::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.200.2.4 [_loadInstance\(\)](#)

```
bool Delay::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [ModelComponent](#).

15.200.2.5 [_onDispatchEvent\(\)](#)

```
void Delay::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
---------------	---

Parameters

<i>inputPortNumber</i>	Input port where the entity arrived.
------------------------	--------------------------------------

Implements [ModelComponent](#).

15.200.2.6 _saveInstance()

```
void Delay::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.200.2.7 show()

```
std::string Delay::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

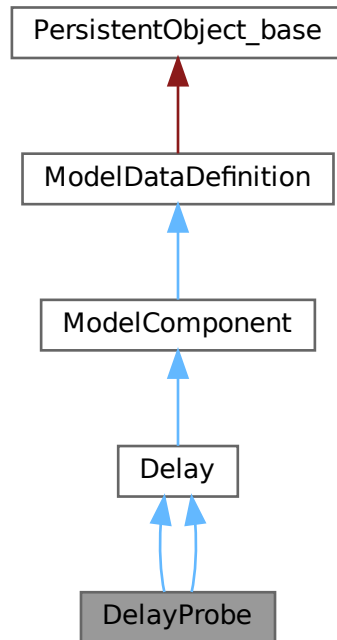
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

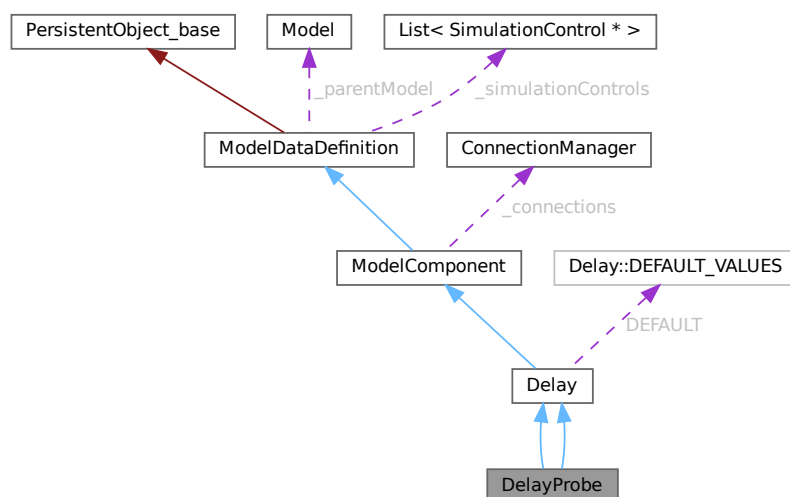
- plugins/components/DiscreteProcessing/Delay.h
- plugins/components/DiscreteProcessing/Delay.cpp

15.201 DelayProbe Class Reference

Inheritance diagram for DelayProbe:



Collaboration diagram for DelayProbe:



Public Member Functions

- **DelayProbe** ([Model](#) *model, const std::string &name="")
- void **CreateInternalAndAttachedDataProbe** ()
- [StatisticsCollector](#) * **WaitTimeStatisticsCollectorProbe** () const
- void **AddWaitTimeValueProbe** (double waitTime)
- bool **HasAttachedDataProbe** (const std::string &key) const

Public Member Functions inherited from [Delay](#)

- **Delay** ([Model](#) *model, std::string name="")
- void **setDelayExpression** (std::string _delayExpression, [Util::TimeUnit](#) _delayTimeUnit=[Util::TimeUnit](#)↵::unknown)
- std::string **delayExpression** () const
- void **setDelay** (double delay)
- double **delay** () const
- void **setDelayTimeUnit** ([Util::TimeUnit](#) _delayTimeUnit)
- [Util::TimeUnit](#) **delayTimeUnit** () const
- void **setAllocation** ([Util::AllocationType](#) allocation)
- [Util::AllocationType](#) **getAllocation** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentName, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- [Util::identification](#) **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const

- Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecific←
Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Additional Inherited Members

Static Public Member Functions inherited from [Delay](#)

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name= "")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Public Attributes inherited from [Delay](#)

- const struct [Delay::DEFAULT_VALUES](#) **DEFAULT**

Protected Member Functions inherited from [Delay](#)

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createInternalStatisticReporters](#) () override
- virtual void [_createAttachedAttributes](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool _getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * _getParserChangesInformation ()`
- `virtual void _initBetweenReplications ()`
- `virtual void _addSimulationControl (SimulationControl *control)`
- `virtual void _createInternalAndAttachedData ()`
- `virtual void _createNonEditableDataDefinitions ()`
- `virtual void _createEditableDataDefinitions ()`
- `void _templateCreateInternalStatisticReporters ()`
- `void _templateCreateAttachedAttributes ()`
- `void _templateCreateNonEditableDataDefinitions ()`
- `void _templateCreateEditableDataDefinitions ()`
- `void _internalStatisticReportersClear ()`
legacy helper methods used by the granular create hooks*
- `void _internalDataClear ()`
- `void _internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void _internalDataRemove (const std::string &key)`
- `void _attachedDataClear ()`
- `void _attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void _attachedDataRemove (const std::string &key)`
- `void _attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void _statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void _statisticReporterRemove (const std::string &key)`
- `void _statisticReportersClear ()`
- `void _mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryAttachedAttributeRemove (const std::string &key)`
- `void _mandatoryAttachedAttributesClear ()`
- `void _mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryEditableDataDefinitionsClear ()`
- `void _optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalEditableDataDefinitionsClear ()`
- `void _mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryNonEditableDataDefinitionsClear ()`
- `void _optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalNonEditableDataDefinitionsClear ()`
- `void _checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
Traces text to the simulation output.
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

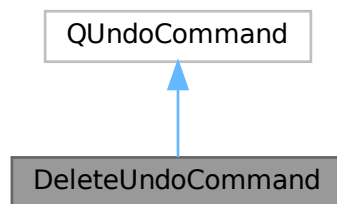
- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

The documentation for this class was generated from the following file:

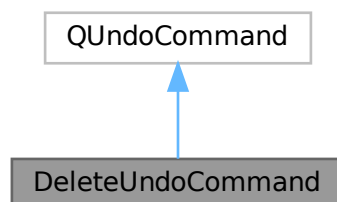
- tests/unit/test_simulator_runtime.cpp

15.202 DeleteUndoCommand Class Reference

Inheritance diagram for DeleteUndoCommand:



Collaboration diagram for DeleteUndoCommand:



Public Member Functions

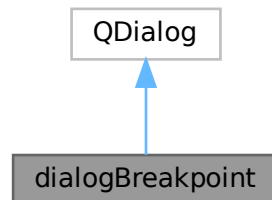
- **DeleteUndoCommand** (QList< QGraphicsItem * > items, [ModelGraphicsScene](#) *scene, QUndoCommand *parent=nullptr)
- void **undo** () override
- void **redo** () override

The documentation for this class was generated from the following files:

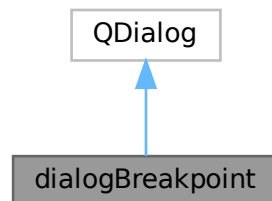
- applications/gui/genesys/actions/DeleteUndoCommand.h
- applications/gui/genesys/actions/DeleteUndoCommand.cpp

15.203 dialogBreakpoint Class Reference

Inheritance diagram for dialogBreakpoint:



Collaboration diagram for dialogBreakpoint:



Classes

- class [MVCResult](#)

Public Member Functions

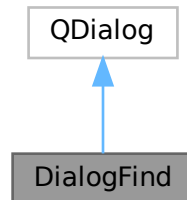
- void **setMVCModel** ([Simulator](#) *simulator)
- [dialogBreakpoint::MVCResult](#) * **getMVCResult** ()

The documentation for this class was generated from the following files:

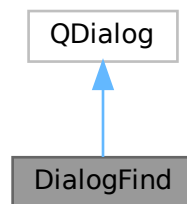
- applications/gui/genesys/dialogs/dialogBreakpoint.h
- applications/gui/genesys/dialogs/dialogBreakpoint.cpp

15.204 DialogFind Class Reference

Inheritance diagram for DialogFind:



Collaboration diagram for DialogFind:



Public Slots

- void **findClicked** ()
- void **nextClicked** ()
- void **previousClicked** ()

Public Member Functions

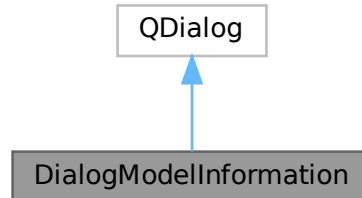
- **DialogFind** (QWidget *parent=nullptr, [ModelGraphicsScene](#) *scene=nullptr)
- QString **getFindText** ()

The documentation for this class was generated from the following files:

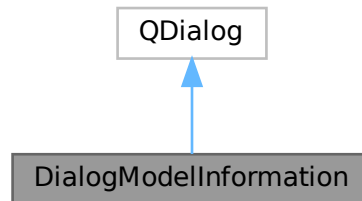
- applications/gui/genesys/dialogs/DialogFind.h
- applications/gui/genesys/dialogs/DialogFind.cpp

15.205 DialogModelInformation Class Reference

Inheritance diagram for DialogModelInformation:



Collaboration diagram for DialogModelInformation:



Public Slots

- void **accept** () override
- void **reject** () override

Public Member Functions

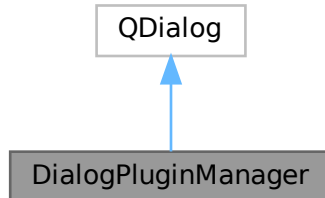
- **DialogModelInformation** (QWidget *parent=nullptr)
- void **setModelInfo** (ModelInfo *modelInfo)

The documentation for this class was generated from the following files:

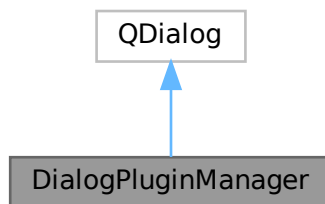
- applications/gui/genesys/dialogs/Dialogmodelinformation.h
- applications/gui/genesys/dialogs/Dialogmodelinformation.cpp

15.206 DialogPluginManager Class Reference

Inheritance diagram for DialogPluginManager:



Collaboration diagram for DialogPluginManager:



Public Member Functions

- [DialogPluginManager](#) (QWidget *parent=nullptr)
Builds the plugin manager dialog and prepares the loaded/problem tables.
- void **setSimulator** ([Simulator](#) *simulator)
- void **setPluginCatalogRefreshCallback** (std::function< void()> refreshCallback)
- void **showProblemPluginsTab** ()
Selects the problems tab when the dialog is opened for startup diagnostics.

15.206.1 Constructor & Destructor Documentation

15.206.1.1 DialogPluginManager()

```
DialogPluginManager::DialogPluginManager (
    QWidget * parent = nullptr ) [explicit]
```

Builds the plugin manager dialog and prepares the loaded/problem tables.

The dialog is responsible for plugin insertion/removal, dependency diagnostics, and recovery of blocked plugins that require system-package installation before they can load.

The documentation for this class was generated from the following files:

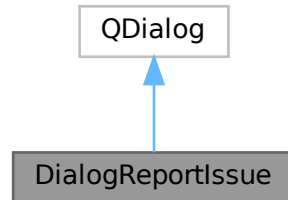
- applications/gui/genesys/dialogs/dialogpluginmanager.h
- applications/gui/genesys/dialogs/dialogpluginmanager.cpp

15.207 DialogReportIssue Class Reference

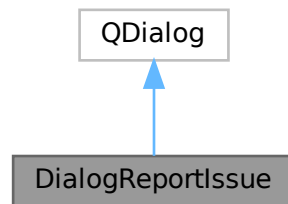
Modal form used to collect a user-facing issue report before relay submission.

```
#include <applications/gui/genesys/dialogs/DialogReportIssue.h>
```

Inheritance diagram for DialogReportIssue:



Collaboration diagram for DialogReportIssue:



Public Member Functions

- **DialogReportIssue** (QWidget *parent=nullptr)
- [IssueReportDraft](#) **draft** () const
Returns a normalized draft from the current form fields.
- void **setRelayConfigurationStatus** (const QString &endpoint, bool configured, const QString &detail)
Updates relay availability text and enables/disables the send action accordingly.

15.207.1 Detailed Description

Modal form used to collect a user-facing issue report before relay submission.

Responsibilities:

- collect category, title, description, and optional diagnostic consent flags;
- expose a validated [IssueReportDraft](#) for the controller layer;
- reflect whether the report relay is configured and available.

Boundaries:

- it does not perform HTTP transport;

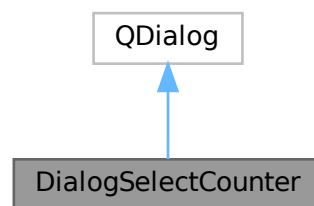
- it does not talk to GitHub directly;
- it does not inspect application runtime state on its own.

The documentation for this class was generated from the following files:

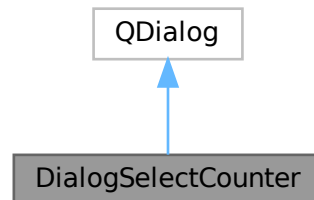
- applications/gui/genesys/dialogs/DialogReportIssue.h
- applications/gui/genesys/dialogs/DialogReportIssue.cpp

15.208 DialogSelectCounter Class Reference

Inheritance diagram for DialogSelectCounter:



Collaboration diagram for DialogSelectCounter:



Public Member Functions

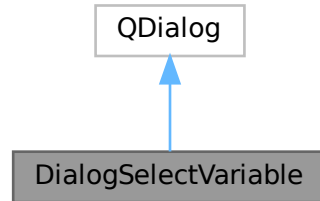
- **DialogSelectCounter** (QWidget *parent=nullptr)
- **Counter * selectedIndex** () const
- void **setCounterItems** (QList< Counter * > *counters, Counter *counter)

The documentation for this class was generated from the following files:

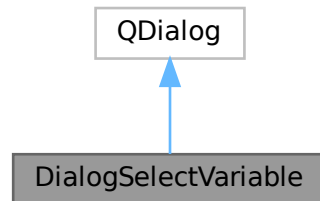
- applications/gui/genesys/dialogs/DialogSelectCounter.h
- applications/gui/genesys/dialogs/DialogSelectCounter.cpp

15.209 DialogSelectVariable Class Reference

Inheritance diagram for DialogSelectVariable:



Collaboration diagram for DialogSelectVariable:



Public Member Functions

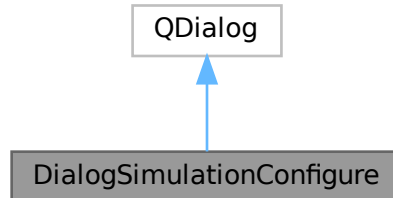
- **DialogSelectVariable** (QWidget *parent=nullptr)
- [ModelDataDefinition](#) * **selectedIndex** () const
- void **setVariableItems** (QList< [ModelDataDefinition](#) * > *definitions, [ModelDataDefinition](#) *definition)

The documentation for this class was generated from the following files:

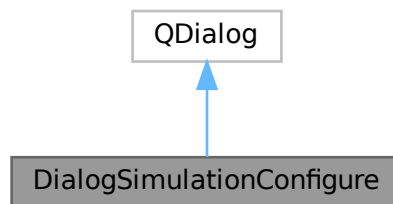
- applications/gui/genesys/dialogs/DialogSelectVariable.h
- applications/gui/genesys/dialogs/DialogSelectVariable.cpp

15.210 DialogSimulationConfigure Class Reference

Inheritance diagram for DialogSimulationConfigure:



Collaboration diagram for DialogSimulationConfigure:



Public Slots

- void [accept](#) () override
Applies the edited configuration back to the model only when the dialog is accepted.
- void [reject](#) () override
Asks for confirmation before discarding unsaved changes.

Public Member Functions

- [DialogSimulationConfigure](#) (QWidget *parent=nullptr)
Builds the simulation-configuration dialog and seeds the static combobox content.
- void **setModelSimulation** ([ModelSimulation](#) *modelSimulation)
- void **setExperimentManager** ([ExperimentManager](#) *experimentManager)
- void **setParallelizationSettings** (bool *enabled, int *localThreads, int *batchSize)

15.210.1 Constructor & Destructor Documentation

15.210.1.1 DialogSimulationConfigure()

```
DialogSimulationConfigure::DialogSimulationConfigure (
    QWidget * parent = nullptr ) [explicit]
```

Builds the simulation-configuration dialog and seeds the static combobox content.

The dialog currently owns a placeholder entry for distributed parallelization while the local simulation and parallelization settings are still exposed as editable GUI state.

15.210.2 Member Function Documentation

15.210.2.1 accept

```
void DialogSimulationConfigure::accept ( ) [override], [slot]
```

Applies the edited configuration back to the model only when the dialog is accepted.

This keeps the original values intact until the user confirms the dialog, while still supporting partial ownership of the parallelization fields when no model simulation is currently attached.

15.210.2.2 reject

```
void DialogSimulationConfigure::reject ( ) [override], [slot]
```

Asks for confirmation before discarding unsaved changes.

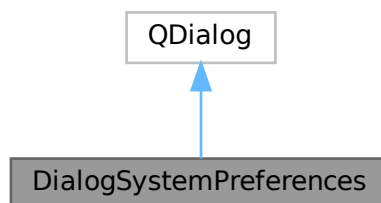
The dialog checks the current widget state against the snapshot captured at load time so the user does not lose pending configuration edits accidentally.

The documentation for this class was generated from the following files:

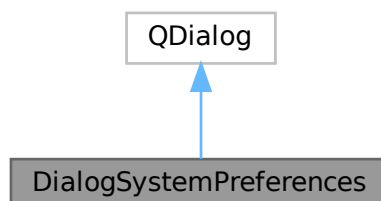
- applications/gui/genesys/dialogs/dialogsimulationconfigure.h
- applications/gui/genesys/dialogs/dialogsimulationconfigure.cpp

15.211 DialogSystemPreferences Class Reference

Inheritance diagram for DialogSystemPreferences:



Collaboration diagram for DialogSystemPreferences:



Public Member Functions

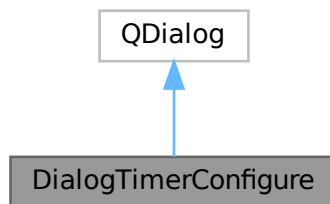
- **DialogSystemPreferences** ([Simulator](#) *simulator=nullptr, QWidget *parent=nullptr)

The documentation for this class was generated from the following files:

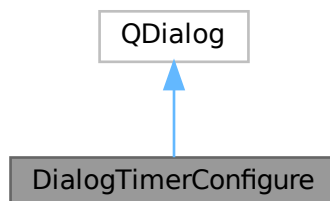
- applications/gui/genesys/dialogs/dialogsystempreferences.h
- applications/gui/genesys/dialogs/dialogsystempreferences.cpp

15.212 DialogTimerConfigure Class Reference

Inheritance diagram for DialogTimerConfigure:



Collaboration diagram for DialogTimerConfigure:

**Public Member Functions**

- **DialogTimerConfigure** (QWidget *parent=nullptr)
- void **setAnimationTimer** ([AnimationTimer](#) *animationTimer)
- void **updateAnimationTimer** ()

The documentation for this class was generated from the following files:

- applications/gui/genesys/dialogs/DialogTimerConfigure.h
- applications/gui/genesys/dialogs/DialogTimerConfigure.cpp

15.213 DialogUtilityController Class Reference

Controller that orchestrates auxiliary dialogs and utility actions.

```
#include <applications/gui/genesys/controllers/DialogUtilityController.h>
```

Public Member Functions

- **DialogUtilityController** ([MainWindow](#) *ownerWidget, [Simulator](#) *simulator, [Ui::MainWindow](#) *ui, [ModelGraphicsView](#) *graphicsView, std::function< void()> showMessageNotImplemented, std::function< void(bool)> actualizeDebugBreakpoints, std::function< bool()> createModelImage, std::function< void()> actualizeActions, std::function< void()> actualizeTabPanels, std::function< void()> reloadPluginCatalog, std::function< [ModelGraphicsScene](#) *()> currentScene, double &optimizerPrecision, unsigned int &optimizerMaxSteps, bool ¶llelizationEnabled, int ¶llelizationThreads, int ¶llelizationBatchSize, QString &lastDataAnalyzerPath)

Builds the utility controller that backs legacy [MainWindow](#) dialog wrappers.
- void **onActionAboutAboutTriggered** ()

Opens the About dialog while preserving legacy action routing.
- void **onActionAboutLicenceTriggered** ()

Opens the license dialog from the compatibility façade action slot.
- void **onActionAboutGetInvolvedTriggered** ()

Opens the get-involved/help contribution dialog flow.
- void **onActionAboutReportIssueTriggered** ()

Opens the report-issue workflow and submits it through the configured relay.
- void **onActionEditFindTriggered** ()

Runs find workflow for the currently focused text editor pane.
- void **onActionEditReplaceTriggered** ()

Runs replace workflow for the currently focused text editor pane.
- void **onActionToolsParserGrammarCheckerTriggered** ()

Opens parser grammar checker utility without altering model state.
- void **onActionToolsOptimizerTriggered** ()

Opens optimization settings dialog and stores lightweight preferences.
- void **onActionToolsAIAssistantTriggered** ()

Opens the AI Assistant panel for LLM-driven simulation model generation.
- void **onActionToolsDataAnalyzerTriggered** ()

Launches the Data Analyzer workstation using persisted last-path compatibility state.
- void **onActionViewConfigureTriggered** ()

Opens view configuration dialog and applies delegated scene/UI refreshes.
- void **onActionSimulatorPreferencesTriggered** ()

Opens simulator preferences utility dialog.
- void **onActionSimulatorsPluginManagerTriggered** (bool showProblemPlugins=false)

Opens plugin manager dialog from the compatibility action surface.
- void **onPushButtonBreakpointInsertClicked** ()

Inserts a breakpoint entry based on current scene selection.
- void **onPushButtonBreakpointRemoveClicked** ()

Removes selected breakpoint entries and refreshes debug breakpoint pane.
- void **onPushButtonExportClicked** ()

Exports current debug/trace context using legacy push-button route.
- void **onActionParallelizationTriggered** ()

Opens parallelization configuration and persists run-preference flags.

15.213.1 Detailed Description

Controller that orchestrates auxiliary dialogs and utility actions.

This Phase-11 controller gathers non-core menu/toolbar actions that were still in [MainWindow](#), such as About dialogs, breakpoint utilities, optimization/preferences dialogs, data analyzer launch flow, and parallelization toggles. Responsibilities:

- execute dialog-driven utility actions delegated from [MainWindow](#) wrappers;

- access shared UI/simulator state through narrow injected callbacks/references;
- keep compatibility behavior for debug breakpoints and auxiliary tooling actions.

Boundaries:

- it does not own simulator/model lifecycle;
- it does not manage scene command pipeline or property editor internals;
- it acts as a controller-level compatibility wrapper, not as a domain service.

15.213.2 Member Function Documentation

15.213.2.1 onActionAboutReportIssueTriggered()

```
void DialogUtilityController::onActionAboutReportIssueTriggered ( )
```

Opens the report-issue workflow and submits it through the configured relay.

Opens the GUI issue-report flow, captures runtime context, and submits the payload to the configured relay.

The report draft can include the current model text, console tail, simulation and report logs, the latest process log, and an optional screenshot. The submission result is then surfaced back to the user, including a browser link when the relay returns a created GitHub issue URL.

15.213.2.2 onActionEditReplaceTriggered()

```
void DialogUtilityController::onActionEditReplaceTriggered ( )
```

Runs replace workflow for the currently focused text editor pane.

Opens the editor replacement dialog and preserves the current find/replace workflow.

The dialog keeps the last search terms, supports case-sensitive matching, wraps the search to the start of the document, and performs single or bulk replacement directly on the code editor without detaching the current selection state.

15.213.2.3 onActionToolsParserGrammarCheckerTriggered()

```
void DialogUtilityController::onActionToolsParserGrammarCheckerTriggered ( )
```

Opens parser grammar checker utility without altering model state.

Builds the parser diagnostics window with grammar overview, DOT export, catalog, and console tabs.

The dialog combines a collapsible grammar map, a DOT text editor, a searchable function catalog, a parser console, and a lightweight model symbol browser so the user can inspect and exercise the expression language from one place.

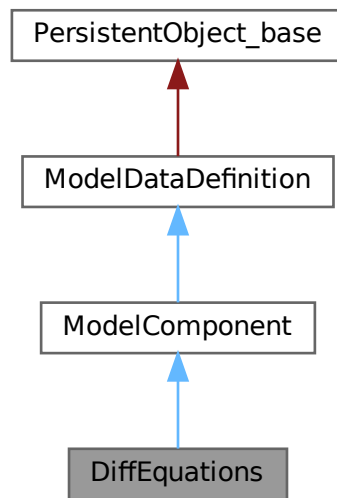
The documentation for this class was generated from the following files:

- applications/gui/genesys/controllers/DialogUtilityController.h
- applications/gui/genesys/controllers/DialogUtilityController.cpp

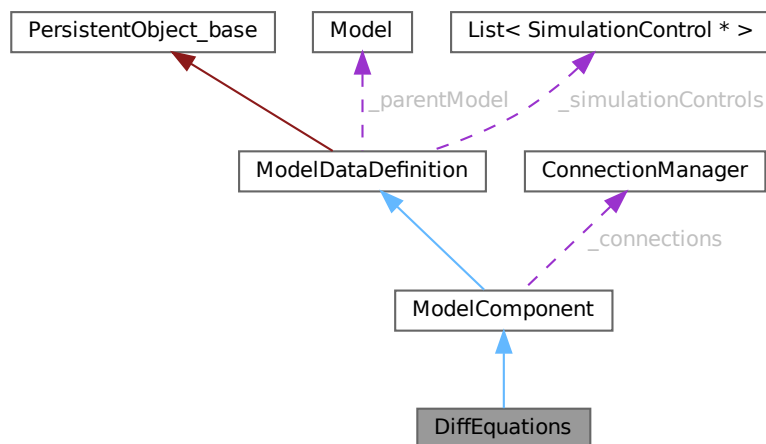
15.214 DiffEquations Class Reference

```
#include <plugins/components/Continuous/DiffEquations.h>
```

Inheritance diagram for DiffEquations:



Collaboration diagram for DiffEquations:



Public Member Functions

- **DiffEquations** (`Model` *model, `std::string` name="")
constructors
- `List< std::string > * getEquations ()` const
new public user methods for this component
- `List< std::string > * getStateVariableNames ()` const
- `double getprecision ()` const
- `void setPrecision (double newPrecision)`

- double **getmaxSteps** () const
- void **setMaxSteps** (double newMaxSteps)
- double **getfinalTime** () const
- void **setFinalTime** (double newFinalTime)
- [List](#)< double > * **getvaluesOnFinalTime** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- [Model](#) * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.

- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual bool [_check](#) (std::string &errorMessage) override
virtual protected methods that could be overridden by derived classes, if needed
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)

- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.214.1 Detailed Description

This component ...

15.214.2 Member Function Documentation

15.214.2.1 `_check()`

```
bool DiffEquations::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

virtual protected methods that could be overridden by derived classes, if needed

This method is called by ModelChecker during model check. The component should check itself to verify if user parameters are ok (ex: correct syntax for the parser) and everything in its parameters allow the model too run without errors in this component

Reimplemented from [ModelDataDefinition](#).

15.214.2.2 `_createEditableDataDefinitions()`

```
void DiffEquations::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDats. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dinamically linked to to simulator kernel to reflect the changes

This method is called by [ModelSimulation](#) when initiating the replication. The model should set all value for a new replication (Ex: setting back to 0 any internal counter, clearing lists, etc.

This method is not used yet. It should be usefull for new UIs

Reimplemented from [ModelDataDefinition](#).

15.214.2.3 `_loadInstance()`

```
bool DiffEquations::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.214.2.4 `_onDispatchEvent()`

```
void DiffEquations::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

This method is only for ModelComponents, not ModelDataElements.

Implements [ModelComponent](#).

15.214.2.5 `_saveInstance()`

```
void DiffEquations::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.214.2.6 `getEquations()`

```
List< std::string > * DiffEquations::getEquations ( ) const
```

new public user methods for this component

equations should be in terms of state variable names and t

15.214.2.7 show()

```
std::string DiffEquations::show ( ) [override], [virtual]
```

virtual public methods

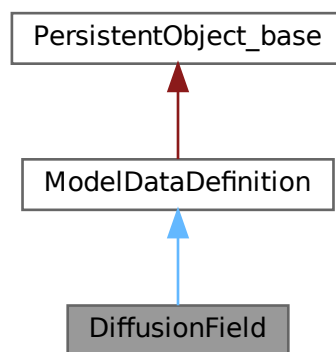
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

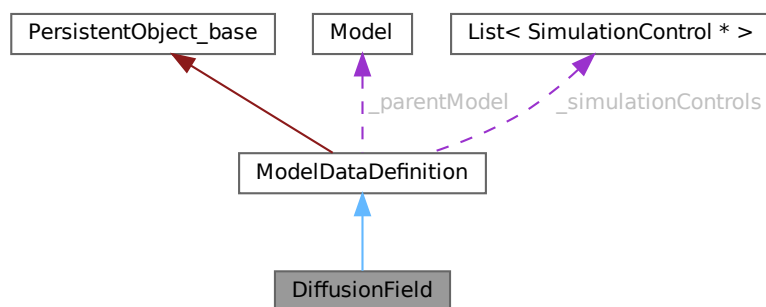
- plugins/components/Continuous/DiffEquations.h
- plugins/components/Continuous/DiffEquations.cpp

15.215 DiffusionField Class Reference

Inheritance diagram for DiffusionField:



Collaboration diagram for DiffusionField:



Public Member Functions

- **DiffusionField** ([Model](#) *model, std::string name="")
- void **setDimensions** (unsigned int dimensions)
- unsigned int **getDimensions** () const
- void **setPointsPerDimension** (unsigned int pointsPerDimension)

- unsigned int **getPointsPerDimension** () const
- void **setDomainLength** (double domainLength)
- double **getDomainLength** () const
- void **setDiffusionCoefficient** (double diffusionCoefficient)
- double **getDiffusionCoefficient** () const
- void **setBoundaryCondition** (std::string boundaryCondition)
- std::string **getBoundaryCondition** () const
- void **setInitialCondition** (std::string initialCondition)
- std::string **getInitialCondition** () const
- void **setInitialParameter** (double initialParameter)
- double **getInitialParameter** () const
- void **setStartTime** (double startTime)
- double **getStartTime** () const
- void **setStopTime** (double stopTime)
- double **getStopTime** () const
- void **setStepSize** (double stepSize)
- double **getStepSize** () const
- void **setCurrentTime** (double currentTime)
- double **getCurrentTime** () const
- void **setAutoSchedule** (bool autoSchedule)
- bool **getAutoSchedule** () const
- void **setOdeSolver** (std::string odeSolver)
- std::string **getOdeSolver** () const
- void **setLastStatus** (std::string lastStatus)
- std::string **getLastStatus** () const
- void **setLastErrorMessage** (std::string lastErrorMessage)
- std::string **getLastErrorMessage** () const
- double **getTotalMass** () const
- double **getMaxValue** () const
- double **getL2Norm** () const
- const std::vector< double > & **getField** () const
- double **getFieldValue** (const std::vector< unsigned int > &multiIndex) const
Field value at a multi-index (size must equal Dimensions); 0 if invalid.
- bool **simulate** (std::string &errorMessage)
- bool **simulate** (double startTime, double stopTime, double stepSize, std::string &errorMessage)
- bool **advanceOneStep** (std::string &errorMessage)
- std::string **showFieldAsAscii** () const
- virtual std::string **show** () override
Returns a textual representation of this data definition.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
 IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)

- Updates the user-visible label of this instance.*

 - `const std::string & getLabel () const`

Returns the user-visible label of this instance.
- `const std::string & getClassname () const`

Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`

Returns the parent model that owns this definition.
- `bool isReportStatistics () const`

Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`

Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`

Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`

Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`

Returns the full map of attached data definitions.
- `bool hasChanged () const`

Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`

Updates the changed flag for this data definition.
- `unsigned int getLevel () const`

Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`

Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`

Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`

Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`

Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`

Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`

Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`

Enables or disables the specific trace level.

Static Public Member Functions

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static PluginInformation * GetPluginInformation ()`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)

- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.215.1 Member Function Documentation

15.215.1.1 _check()

```
bool DiffusionField::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
 Reimplemented from [ModelDataDefinition](#).

15.215.1.2 _initBetweenReplications()

```
void DiffusionField::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.215.1.3 _loadInstance()

```
bool DiffusionField::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.215.1.4 _saveInstance()

```
void DiffusionField::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.215.1.5 show()

```
std::string DiffusionField::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

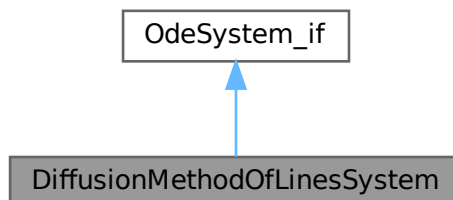
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

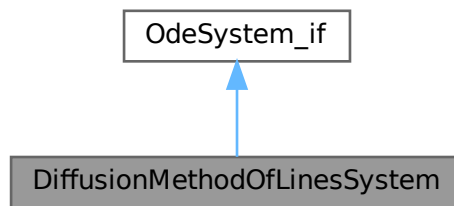
- plugins/data/Continuous/DiffusionField.h
- plugins/data/Continuous/DiffusionField.cpp

15.216 DiffusionMethodOfLinesSystem Class Reference

Inheritance diagram for DiffusionMethodOfLinesSystem:



Collaboration diagram for DiffusionMethodOfLinesSystem:



Public Types

- enum class **Boundary** { **Dirichlet** , **Neumann** }

Public Member Functions

- **DiffusionMethodOfLinesSystem** (std::vector< unsigned int > shape, std::vector< double > spacing, double diffusionCoefficient, Boundary boundary)
- unsigned int **dimension** () const override
- void **evaluate** (double, const double *y, double *dydt) const override
- bool **isValid** () const
- unsigned int **totalNodes** () const
- const std::vector< unsigned int > & **shape** () const
- const std::vector< double > & **spacing** () const
- double **diffusionCoefficient** () const
- Boundary **boundary** () const
- unsigned int **coordinate** (unsigned int node, unsigned int d) const
- unsigned int **nodeIndex** (const std::vector< unsigned int > &multiIndex) const
- bool **isBoundaryNode** (unsigned int node) const
- void **fillSineModes** (const std::vector< unsigned int > &modes, double amplitude, double *y) const
- void **fillGaussian** (double amplitude, double sigma, double *y) const
- double **sineModeDecayRate** (const std::vector< unsigned int > &modes) const
- double **totalMass** (const double *y) const
- double **l2Norm** (const double *y) const
- double **maxValue** (const double *y) const

Static Public Attributes

- static constexpr double **kPi** = 3.14159265358979323846

15.216.1 Member Function Documentation

15.216.1.1 dimension()

unsigned int DiffusionMethodOfLinesSystem::dimension () const [inline], [override], [virtual]
Implements [OdeSystem_if](#).

15.216.1.2 evaluate()

```
void DiffusionMethodOfLinesSystem::evaluate (
    double ,
    const double * y,
    double * dydt ) const [inline], [override], [virtual]
```

Implements [OdeSystem_if](#).

The documentation for this class was generated from the following file:

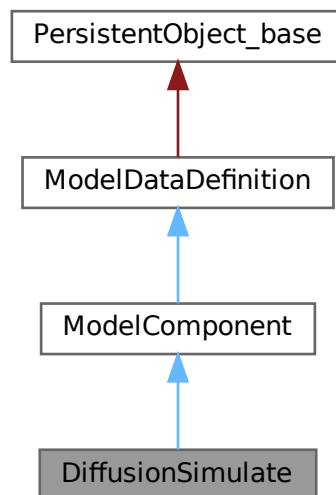
- tools/Continuous/DiffusionMethodOfLinesSystem.h

15.217 DiffusionSimulate Class Reference

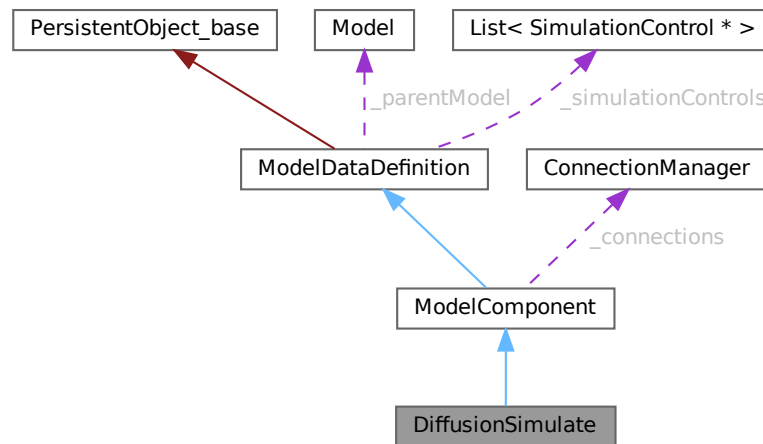
Component that triggers a [DiffusionField](#) solve when entities arrive.

```
#include <plugins/components/Continuous/DiffusionSimulate.h>
```

Inheritance diagram for DiffusionSimulate:



Collaboration diagram for DiffusionSimulate:



Public Member Functions

- **DiffusionSimulate** ([Model](#) *model, std::string name="")
- void **setDiffusionField** ([DiffusionField](#) *diffusionField)
- [DiffusionField](#) * **getDiffusionField** () const
- void **setUseFieldTimeWindow** (bool useFieldTimeWindow)
- bool **getUseFieldTimeWindow** () const
- void **setStartTime** (double startTime)
- double **getStartTime** () const
- void **setStopTime** (double stopTime)
- double **getStopTime** () const
- void **setStepSize** (double stepSize)
- double **getStepSize** () const
- void **setLastSucceeded** (bool lastSucceeded)
- bool **getLastSucceeded** () const
- void **setLastMessage** (std::string lastMessage)
- std::string **getLastMessage** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual **~ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)

Updates the free-form description shown by the GUI.

- `std::string getDescription () const`

Returns the free-form description associated with this component.

- `void remove (ModelComponent *comp)`

Removes a connected component from this component.

Public Member Functions inherited from `ModelDataDefinition`

- `ModelDataDefinition (Model *model, std::string datadefinitionTypename, std::string name="", bool insertIntoModel=true)`

Creates a model data definition and optionally inserts it into the model.

- `virtual ~ModelDataDefinition ()`

Releases resources owned by this data definition.

- `Util::identification getId () const`

Returns the unique identifier of this instance.

- `void setName (const std::string &name)`

Updates the logical name of this instance.

- `const std::string & getName () const`

Returns the logical name of this instance.

- `void setLabel (const std::string &label)`

Updates the user-visible label of this instance.

- `const std::string & getLabel () const`

Returns the user-visible label of this instance.

- `const std::string & getClassname () const`

Returns the runtime class name used by the plugin system.

- `Model * getParentModel () const`

Returns the parent model that owns this definition.

- `bool isReportStatistics () const`

Indicates whether this data definition contributes report statistics.

- `void setReportStatistics (bool reportStatistics)`

Enables or disables report-statistics generation for this definition.

- `ModelDataDefinition * getInternalData (const std::string &name) const`

Looks up a named internal child data definition.

- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`

Returns the full map of internal child data definitions.

- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`

Returns the full map of attached data definitions.

- `bool hasChanged () const`

Indicates whether this instance has been modified since the last reset.

- `void setHasChanged (bool hasChanged)`

Updates the changed flag for this data definition.

- `unsigned int getLevel () const`

Returns the level in the model hierarchy where this definition lives.

- `void setModelLevel (unsigned int _modelLevel)`

Sets the model nesting level for this definition.

- `List< SimulationControl * > * getSimulationControls () const`

Returns the simulation controls owned by this data definition.

- `TraceManager::Level getTraceLevelSpecific () const`

Returns the trace level that applies specifically to this definition.

- `void setTraceLevelSpecific (TraceManager::Level level)`

Updates the trace level used by this definition.

- void `defineTraceLevelSpecific` (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↔ Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation` * `GetPluginInformation` ()
- static `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check` (`ModelComponent` *component)
Validates a component instance.
- static void `CreateInternalData` (`ModelComponent` *component)
Creates internal data required by a component instance.
- static void `SaveInstance` (`PersistenceRecord` *fields, `ModelComponent` *component)
Serializes a component instance into a persistence record.
- static void `DispatchEvent` (`Event` *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void `SaveInstance` (`PersistenceRecord` *fields, `ModelDataDefinition` *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool `Check` (`ModelDataDefinition` *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void `InitBetweenReplications` (`ModelDataDefinition` *modeldatum)
Resets simulation-time state between replications.
- static void `CreateInternalData` (`ModelDataDefinition` *modeldatum)
Creates internal and attached data required by this instance.
- static bool `CreateRelatedDataElements` (`ModelDataDefinition` *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` (`PersistenceRecord` *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` (`PersistenceRecord` *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_onDispatchEvent` (`Entity` *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void `_createEditableDataDefinitions` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool` [_getSaveDefaultsOption](#) ()
- `virtual` [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- `virtual` `void` [_initBetweenReplications](#) ()
- `virtual` `void` [_addSimulationControl](#) ([SimulationControl](#) *control)
- `virtual` `void` [_createInternalAndAttachedData](#) ()
- `virtual` `void` [_createInternalStatisticReporters](#) ()
- `virtual` `void` [_createAttachedAttributes](#) ()
- `virtual` `void` [_createNonEditableDataDefinitions](#) ()
- `void` [_templateCreateInternalStatisticReporters](#) ()
- `void` [_templateCreateAttachedAttributes](#) ()
- `void` [_templateCreateNonEditableDataDefinitions](#) ()
- `void` [_templateCreateEditableDataDefinitions](#) ()
- `void` [_internalStatisticReportersClear](#) ()
 - legacy helper methods used by the granular create* hooks*
- `void` [_internalDataClear](#) ()
- `void` [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- `void` [_internalDataRemove](#) (const std::string &key)
- `void` [_attachedDataClear](#) ()
- `void` [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_attachedDataRemove](#) (const std::string &key)
- `void` [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- `void` [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_statisticReporterRemove](#) (const std::string &key)
- `void` [_statisticReportersClear](#) ()
- `void` [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- `void` [_mandatoryAttachedAttributesClear](#) ()
- `void` [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- `void` [_mandatoryEditableDataDefinitionsClear](#) ()
- `void` [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- `void` [_optionalEditableDataDefinitionsClear](#) ()
- `void` [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- `void` [_mandatoryNonEditableDataDefinitionsClear](#) ()
- `void` [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- `void` [_optionalNonEditableDataDefinitionsClear](#) ()
- `void` [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void` [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void` [traceError](#) (const std::string &text, const std::exception &e)
- `void` [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- `void` [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- `void` [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- `void` [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - Traces text to the simulation output.*
- `void` [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

15.217.1 Detailed Description

Component that triggers a [DiffusionField](#) solve when entities arrive.

[DiffusionSimulate](#) delegates the continuous diffusion solve to a referenced [DiffusionField](#) (a [ModelDataDefinition](#) in plugins/data/Continuous). Each arriving entity runs the field either with its own configured time window or with component-provided start/stop/step values, then forwards the entity. This is the discrete->continuous bridge: a discrete-event component drives the continuous PDE subsystem. For event-clock-synchronized advance, enable `AutoSchedule` on the [DiffusionField](#) instead, so it steps once per kernel [InternalEvent](#).

15.217.2 Member Function Documentation

15.217.2.1 `_check()`

```
bool DiffusionSimulate::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.217.2.2 `_createEditableDataDefinitions()`

```
void DiffusionSimulate::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.217.2.3 `_loadInstance()`

```
bool DiffusionSimulate::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [ModelComponent](#).

15.217.2.4 `_onDispatchEvent()`

```
void DiffusionSimulate::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for [ModelComponents](#), not for [ModelDataElements](#).

Parameters

<code>entity</code>	Entity being processed.
---------------------	---

Parameters

<i>inputPortNumber</i>	Input port where the entity arrived.
------------------------	--------------------------------------

Implements [ModelComponent](#).

15.217.2.5 _saveInstance()

```
void DiffusionSimulate::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.217.2.6 show()

```
std::string DiffusionSimulate::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

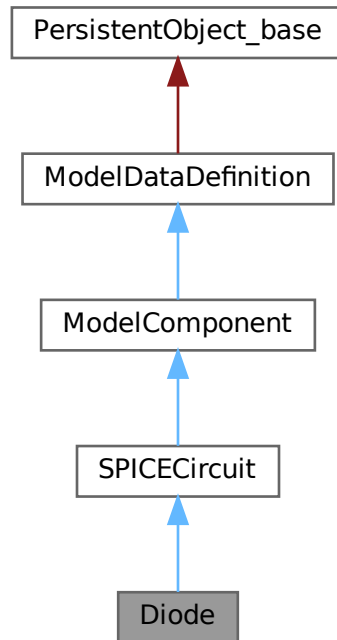
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

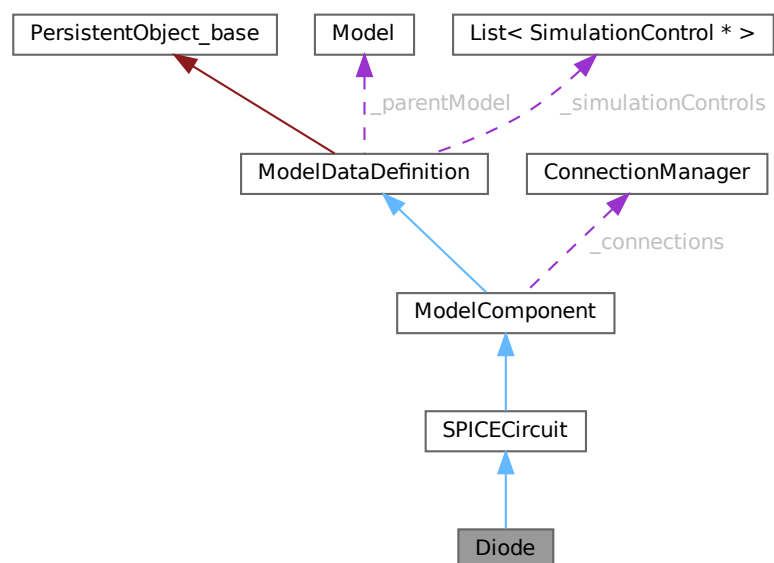
- plugins/components/Continuous/DiffusionSimulate.h
- plugins/components/Continuous/DiffusionSimulate.cpp

15.218 Diode Class Reference

Inheritance diagram for Diode:



Collaboration diagram for Diode:



Public Member Functions

- **Diode** ([Model](#) *model, std::string name="")
- void **Build** () override
- void **setElectricalModel** (std::string electricalModel)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentName, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

- Returns the runtime class name used by the plugin system.*
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- **List**< **SimulationControl** * > * **getSimulationControls** () const
Returns the simulation controls owned by this data definition.
- **TraceManager::Level** **getTraceLevelSpecific** () const
Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (**TraceManager::Level** level)
Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (**TraceManager::Level** traceLevelSpecific, bool traceLevelSpecific←Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const
Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static **PluginInformation** * **GetPluginInformation** ()
- static **ModelComponent** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")

Static Public Member Functions inherited from **SPICECircuit**

- static **PluginInformation** * **GetPluginInformation** ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static **ModelComponent** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")
- static std::vector< std::string > **split** (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from **ModelComponent**

- static **ModelComponent** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** (**ModelComponent** *component)
Validates a component instance.
- static void **CreateInternalData** (**ModelComponent** *component)
Creates internal data required by a component instance.
- static void **SaveInstance** (**PersistenceRecord** *fields, **ModelComponent** *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** (**Event** *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from **ModelDataDefinition**

- static **ModelDataDefinition** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** (**PersistenceRecord** *fields, **ModelDataDefinition** *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** (**ModelDataDefinition** *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** (**ModelDataDefinition** *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** (**ModelDataDefinition** *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** (**ModelDataDefinition** *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** (**PersistenceRecord** *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** (**PersistenceRecord** *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from **SPICECircuit**

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void **_onDispatchEvent** (**Entity** *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void **_createNonEditableDataDefinitions** () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.218.1 Member Function Documentation

15.218.1.1 [_loadInstance\(\)](#)

```
bool Diode::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [SPICECircuit](#).

15.218.1.2 [_saveInstance\(\)](#)

```
void Diode::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

15.218.1.3 [Build\(\)](#)

```
void Diode::Build ( ) [override], [virtual]
```

Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

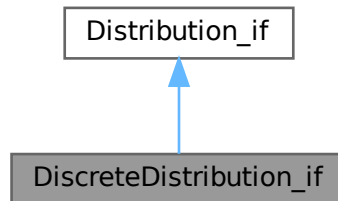
- plugins/components/ElectronicsSimulation/SPICECircuit.h
- plugins/components/ElectronicsSimulation/SPICECircuit.cpp

15.219 DiscreteDistribution_if Class Reference

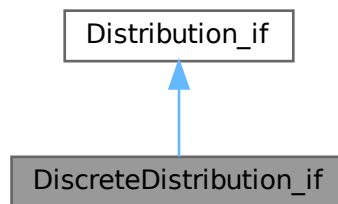
Interface for discrete distributions.

```
#include <tools/Statistics/DiscreteDistribution_if.h>
```

Inheritance diagram for DiscreteDistribution_if:



Collaboration diagram for DiscreteDistribution_if:



Public Member Functions

- virtual double **pmf** (unsigned int k) const =0
- virtual double **cdf** (unsigned int k) const =0
- virtual unsigned int **quantile** (double probability) const =0

Public Member Functions inherited from [Distribution_if](#)

- virtual std::string **name** () const =0

15.219.1 Detailed Description

Interface for discrete distributions.

Mathematical meaning:

- pmf(k): probability mass function value at k.
- cdf(k): cumulative probability $P(X \leq k)$.
- quantile(p): smallest k such that $CDF(k) \geq p$.

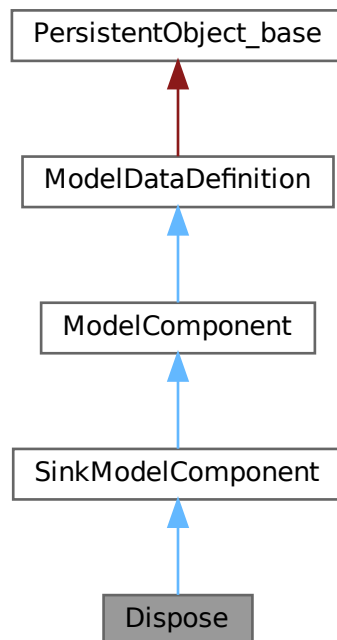
The documentation for this class was generated from the following file:

- tools/Statistics/DiscreteDistribution_if.h

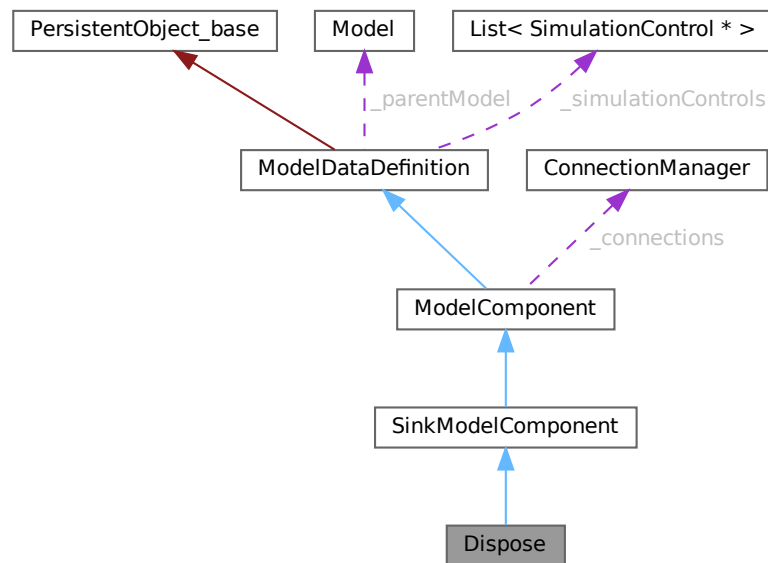
15.220 Dispose Class Reference

```
#include <plugins/components/Logic/Dispose.h>
```

Inheritance diagram for Dispose:



Collaboration diagram for Dispose:



Public Member Functions

- **Dispose** (`Model` *model, `std::string` name="")
 - virtual `std::string` **show** () override
- virtual public methods*

Public Member Functions inherited from SinkModelComponent

- **SinkModelComponent** (`Model` *model, `std::string` componentTypename, `std::string` name="")

Public Member Functions inherited from ModelComponent

- **ModelComponent** (`Model` *model, `std::string` componentTypename, `std::string` name="")
constructors
- virtual **~ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- `ConnectionManager` * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (`ModelComponent` *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (`std::string` _description)
Updates the free-form description shown by the GUI.
- `std::string` **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (`ModelComponent` *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool **hasChanged**)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void **_initBetweenReplications** () override
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_createInternalStatisticReporters** () override
- virtual void **_createAttachedAttributes** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- [bool](#) [_getSaveDefaultsOption](#) ()
- [virtual](#) [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- [virtual](#) [void](#) [_addSimulationControl](#) ([SimulationControl](#) *control)
- [virtual](#) [void](#) [_createInternalAndAttachedData](#) ()
- [virtual](#) [void](#) [_createNonEditableDataDefinitions](#) ()
- [virtual](#) [void](#) [_createEditableDataDefinitions](#) ()
- [void](#) [_templateCreateInternalStatisticReporters](#) ()
- [void](#) [_templateCreateAttachedAttributes](#) ()
- [void](#) [_templateCreateNonEditableDataDefinitions](#) ()
- [void](#) [_templateCreateEditableDataDefinitions](#) ()
- [void](#) [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- [void](#) [_internalDataClear](#) ()
- [void](#) [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- [void](#) [_internalDataRemove](#) (const std::string &key)
- [void](#) [_attachedDataClear](#) ()
- [void](#) [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_attachedDataRemove](#) (const std::string &key)
- [void](#) [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- [void](#) [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_statisticReporterRemove](#) (const std::string &key)
- [void](#) [_statisticReportersClear](#) ()
- [void](#) [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- [void](#) [_mandatoryAttachedAttributesClear](#) ()
- [void](#) [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- [void](#) [_mandatoryEditableDataDefinitionsClear](#) ()
- [void](#) [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- [void](#) [_optionalEditableDataDefinitionsClear](#) ()
- [void](#) [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- [void](#) [_mandatoryNonEditableDataDefinitionsClear](#) ()
- [void](#) [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- [void](#) [_optionalNonEditableDataDefinitionsClear](#) ()
- [void](#) [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- [void](#) [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- [void](#) [traceError](#) (const std::string &text, const std::exception &e)
- [void](#) [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- [void](#) [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- [void](#) [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- [void](#) [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- [void](#) [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

15.220.1 Detailed Description

[Dispose](#) module DESCRIPTION This module is intended as the ending point for entities in a simulation model. [Entity](#) statistics may be recorded before the entity is disposed of. TYPICAL USES Parts leaving the modeled facility The termination of a business process Customers departing from the store Prompt Description Name Unique module identifier displayed on the module shape. [Record Entity](#) Statistics Determines whether or not the incoming entity's statistics will be recorded. Statistics include value-added time, non-value-added time, wait time, transfer time, other time, total time, value-added cost, non-value-added cost, wait cost, transfer cost, other cost, and total cost.

15.220.2 Member Function Documentation

15.220.2.1 `_check()`

```
bool Dispose::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [SinkModelComponent](#).

15.220.2.2 `_createAttachedAttributes()`

```
void Dispose::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.220.2.3 `_createInternalStatisticReporters()`

```
void Dispose::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.220.2.4 `_initBetweenReplications()`

```
void Dispose::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.220.2.5 `_loadInstance()`

```
bool Dispose::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [SinkModelComponent](#).

15.220.2.6 _onDispatchEvent()

```
void Dispose::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.220.2.7 _saveInstance()

```
void Dispose::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SinkModelComponent](#).

15.220.2.8 show()

```
std::string Dispose::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

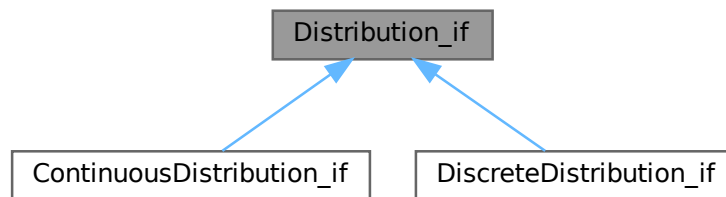
- plugins/components/Logic/Dispose.h
- plugins/components/Logic/Dispose.cpp

15.221 Distribution_if Class Reference

Base abstraction for a theoretical probability distribution.

```
#include <tools/Statistics/Distribution_if.h>
```

Inheritance diagram for Distribution_if:



Public Member Functions

- virtual std::string **name** () const =0

15.221.1 Detailed Description

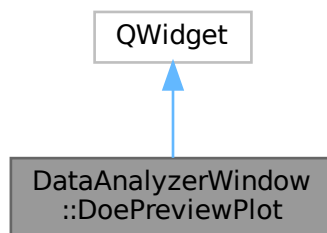
Base abstraction for a theoretical probability distribution.

The documentation for this class was generated from the following file:

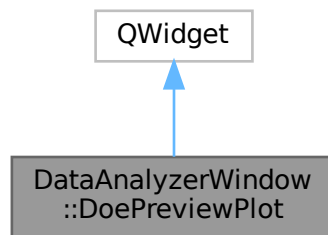
- tools/Statistics/Distribution_if.h

15.222 DataAnalyzerWindow::DoePreviewPlot Class Reference

Inheritance diagram for DataAnalyzerWindow::DoePreviewPlot:



Collaboration diagram for DataAnalyzerWindow::DoePreviewPlot:



Public Member Functions

- **DoePreviewPlot** (QWidget *parent=nullptr)
- void **setMode** (const QString &mode)

Protected Member Functions

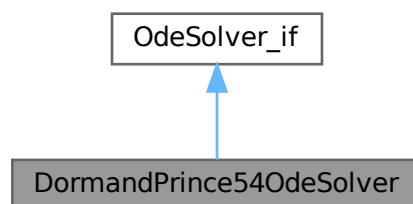
- void **paintEvent** (QPaintEvent *) override

The documentation for this class was generated from the following file:

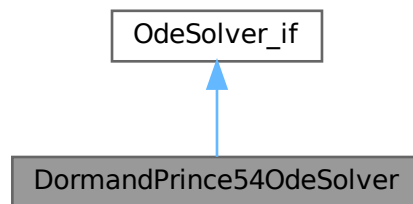
- applications/gui/genesys/tools/dataanalyzer/DataAnalyzerWindow.cpp

15.223 DormandPrince54OdeSolver Class Reference

Inheritance diagram for DormandPrince54OdeSolver:



Collaboration diagram for DormandPrince54OdeSolver:



Public Member Functions

- **DormandPrince54OdeSolver** (double relativeTolerance=1e-6, double absoluteTolerance=1e-9, double safetyFactor=0.9, double minScale=0.2, double maxScale=10.0, unsigned int maxSubsteps=100000u)
- double **getRelativeTolerance** () const
- double **getAbsoluteTolerance** () const
- bool **advance** (const [OdeSystem_if](#) &system, double t0, double dt, const double *y0, double *y1) const override

15.223.1 Member Function Documentation

15.223.1.1 advance()

```

bool DormandPrince54OdeSolver::advance (
    const OdeSystem\_if & system,
    double t0,
    double dt,
    const double * y0,
    double * y1 ) const [inline], [override], [virtual]
  
```

Implements [OdeSolver_if](#).

The documentation for this class was generated from the following file:

- tools/Continuous/DormandPrince54OdeSolver.h

15.224 testing::internal::DoubleSequence< plus_one, T, sizeofT > Struct Template Reference

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-internal.h

15.225 testing::internal::DoubleSequence< false, IndexSequence< I... >, sizeofT > Struct Template Reference

Public Types

- using **type** = [IndexSequence](#)< I..., (sizeofT+I)... >

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-internal.h

15.226 testing::internal::DoubleSequence< true, IndexSequence< I... >, sizeofT > Struct Template Reference

Public Types

- using **type** = [IndexSequence](#)< I..., (sizeofT+I)..., 2 *sizeofT >

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-internal.h

15.227 DrawingItem Struct Reference

Public Attributes

- QGraphicsItem * **myDrawingItem**
- QPointF **initialPosition**

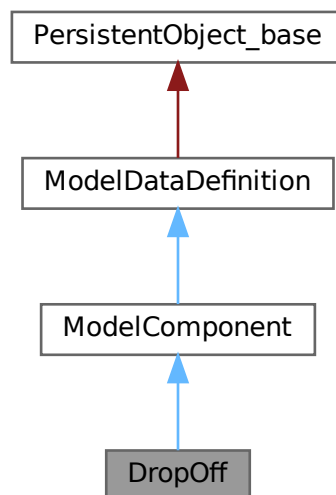
The documentation for this struct was generated from the following file:

- applications/gui/genesys/actions/AddUndoCommand.h

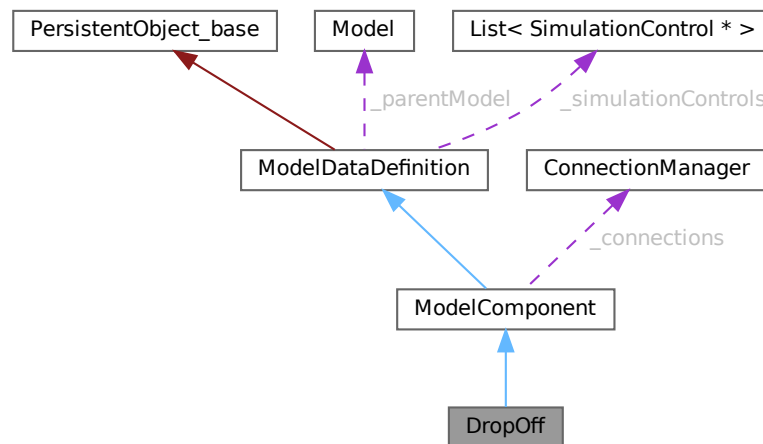
15.228 DropOff Class Reference

```
#include <plugins/components/Decisions/DropOff.h>
```

Inheritance diagram for DropOff:



Collaboration diagram for DropOff:



Public Member Functions

- **DropOff** ([Model](#) *model, std::string name="")
 - virtual std::string [show](#) () override
- virtual public methods*

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
- constructors*
- virtual ~**ModelComponent** ()
- Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * [getConnectionManager](#) () const
- new public user methods for this component*
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
- Connects this component to another component.*
- bool [hasBreakpointAt](#) ()
- Indicates whether a simulation breakpoint is set on this component.*
- void [setDescription](#) (std::string _description)
- Updates the free-form description shown by the GUI.*
- std::string [getDescription](#) () const
- Returns the free-form description associated with this component.*
- void [remove](#) ([ModelComponent](#) *comp)
- Removes a connected component from this component.*

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↔ IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- Releases resources owned by this data definition.*

- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- Model * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- ModelDataDefinition * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, ModelDataDefinition * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, ModelDataDefinition * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< SimulationControl * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) (TraceManager::Level level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static PluginInformation * [GetPluginInformation](#) ()
- static ModelComponent * [LoadInstance](#) (Model *model, PersistenceRecord *fields)
- static ModelDataDefinition * [NewInstance](#) (Model *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()

- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internaStatisticReportersClear** ()
- legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
- method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
- virtual protected methods that could be overridden by derived classes, if needed*

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

15.228.1 Detailed Description

Dropoff module DESCRIPTION The Dropoff module removes a specified number of entities from the entity's group and sends them to another module, as specified by a graphical connection. Group user-defined attribute value and internal attributes may be given to the dropped-off entities based on a specified rule. TYPICAL USES Loading shelves with product Separating a form for use in various departments PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Quantity Number of entities that will be dropped off from an incoming representative grouped entity. Starting Rank Starting rank of the entities to be dropped off, based on the entities in the group. Member Attributes Method of determining how to assign the representative entity attribute values (other than costs/times) to the dropped-off original entities. [Attribute](#) Name Name of representative entity attribute(s) assigned to droppedoff original entities of the group

15.228.2 Member Function Documentation

15.228.2.1 `_check()`

```
bool DropOff::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.228.2.2 `_loadInstance()`

```
bool DropOff::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [ModelComponent](#).

15.228.2.3 `_onDispatchEvent()`

```
void DropOff::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.
This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.228.2.4 `_saveInstance()`

```
void DropOff::_saveInstance (
    PersistenceRecord * fields,
```

```
bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.228.2.5 show()

```
std::string DropOff::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

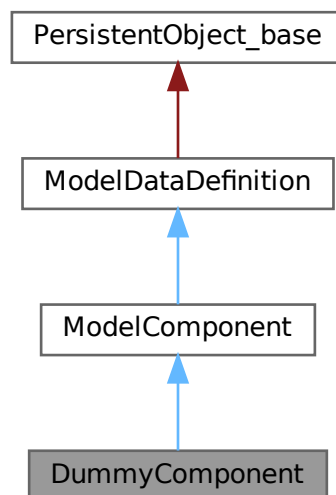
The documentation for this class was generated from the following files:

- plugins/components/Decisions/DropOff.h
- plugins/components/Decisions/DropOff.cpp

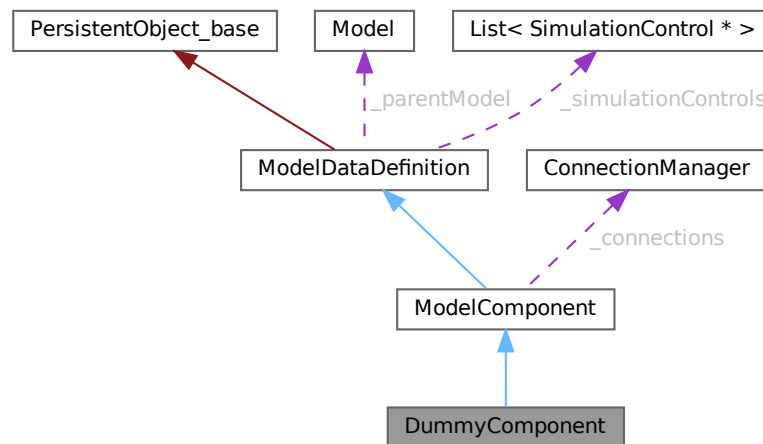
15.229 DummyComponent Class Reference

```
#include <plugins/components/Template/DummyComponent.h>
```

Inheritance diagram for DummyComponent:



Collaboration diagram for DummyComponent:



Public Member Functions

- [DummyComponent](#) ([Model](#) *model, std::string name="")

constructors

- virtual std::string [show](#) () override

new public user methods for this component

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")

constructors

- virtual ~[ModelComponent](#) ()

Releases component-owned resources and unregisters it from the model.

- [ConnectionManager](#) * [getConnectionManager](#) () const

new public user methods for this component

- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)

Connects this component to another component.

- bool [hasBreakpointAt](#) ()

Indicates whether a simulation breakpoint is set on this component.

- void [setDescription](#) (std::string _description)

Updates the free-form description shown by the GUI.

- std::string [getDescription](#) () const

Returns the free-form description associated with this component.

- void [remove](#) ([ModelComponent](#) *comp)

Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.

- virtual ~[ModelDataDefinition](#) ()

- Releases resources owned by this data definition.*

 - Util::identification **getId** () const
Returns the unique identifier of this instance.
 - void **setName** (const std::string &name)
Updates the logical name of this instance.
 - const std::string & **getName** () const
Returns the logical name of this instance.
 - void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
 - const std::string & **getLabel** () const
Returns the user-visible label of this instance.
 - const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
 - Model * **getParentModel** () const
Returns the parent model that owns this definition.
 - bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
 - void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
 - ModelDataDefinition * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
 - std::map< std::string, ModelDataDefinition * > * **getInternalData** () const
Returns the full map of internal child data definitions.
 - std::map< std::string, ModelDataDefinition * > * **getAttachedData** () const
Returns the full map of attached data definitions.
 - bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
 - void **setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
 - unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
 - void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
 - List< SimulationControl * > * **getSimulationControls** () const
Returns the simulation controls owned by this data definition.
 - TraceManager::Level **getTraceLevelSpecific** () const
Returns the trace level that applies specifically to this definition.
 - void **setTraceLevelSpecific** (TraceManager::Level level)
Updates the trace level used by this definition.
 - void **defineTraceLevelSpecific** (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
 - bool **isTraceLevelSpecificEnabled** () const
Indicates whether the specific trace level is enabled.
 - void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static PluginInformation * **GetPluginInformation** ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static ModelComponent * **LoadInstance** (Model *model, PersistenceRecord *fields)
- static ModelDataDefinition * **NewInstance** (Model *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()

- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internaStatisticReportersClear** ()
- legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
- method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
- virtual protected methods that could be overridden by derived classes, if needed*

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

15.229.1 Detailed Description

This component ...

15.229.2 Constructor & Destructor Documentation

15.229.2.1 DummyComponent()

```
DummyComponent::DummyComponent (
    Model * model,
    std::string name = "" )
```

constructors

Externalize function GetPluginInformation to be accessible through dynamic linked library.

15.229.3 Member Function Documentation

15.229.3.1 _loadInstance()

```
bool DummyComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.229.3.2 _onDispatchEvent()

```
void DummyComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

This method is only for ModelComponents, not ModelDataElements.

Implements [ModelComponent](#).

15.229.3.3 _saveInstance()

```
void DummyComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.229.3.4 show()

```
std::string DummyComponent::show ( ) [override], [virtual]
```

new public user methods for this component

virtual public methods

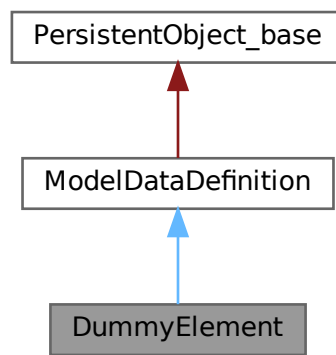
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

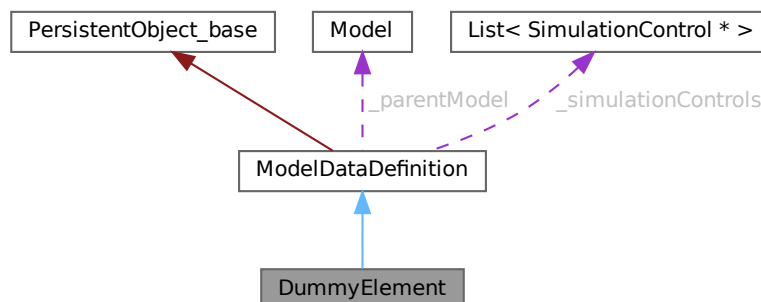
- plugins/components/Template/DummyComponent.h
- plugins/components/Template/DummyComponent.cpp

15.230 DummyElement Class Reference

Inheritance diagram for DummyElement:



Collaboration diagram for DummyElement:



Public Member Functions

- **DummyElement** ([Model](#) *model, std::string name="")
constructors
- void **setSomeString** (const std::string &someString)

new public user methods for this component

- std::string **getSomeString** () const
- void **setSomeUInt** (unsigned int someUInt)
- unsigned int **getSomeUInt** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool **hasChanged**)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)

Defines the specific trace level and optionally enables it immediately.

- bool [isTraceLevelSpecificEnabled](#) () const

Indicates whether the specific trace level is enabled.

- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()

static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)

Loads a model data definition instance from serialized fields.

- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Creates a new instance of a data definition using the plugin registry.

- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)

Serializes a data definition instance into a persistence record.

- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)

Validates a data definition instance.

- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)

Resets simulation-time state between replications.

- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)

Creates internal and attached data required by this instance.

- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)

Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override

virtual protected method that must be overridden

- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override

Saves object fields into a serialization structure.

- virtual bool [_check](#) (std::string &errorMessage) override

could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()

- void **_templateCreateInternalStatisticReporters** ()
- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
- legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
- method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0

- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- `TraceManager::Level` `_traceLevelSpecific`
- `bool` `_traceLevelSpecificEnabled` = false

15.230.1 Member Function Documentation

15.230.1.1 `_check()`

```
bool DummyElement::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.230.1.2 `_loadInstance()`

```
bool DummyElement::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [ModelDataDefinition](#).

15.230.1.3 `_saveInstance()`

```
void DummyElement::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.230.1.4 `show()`

```
std::string DummyElement::show ( ) [override], [virtual]
```

virtual public methods
Reimplemented from [ModelDataDefinition](#).
The documentation for this class was generated from the following files:

- `plugins/data/Template/DummyElement.h`
- `plugins/data/Template/DummyElement.cpp`

15.231 EditCommandController Class Reference

Controller for edit/clipboard/grouping command orchestration in the scene.

```
#include <applications/gui/genesys/controllers/EditCommandController.h>
```

Public Member Functions

- **EditCommandController** ([Simulator](#) *simulator, [ModelGraphicsView](#) *graphicsView, std::function< void()> actualizeActions, bool *cut, [QList](#)< [GraphicalModelComponent](#) * > **gmcCopies, [QList](#)< [GraphicalConnection](#) * > **portsCopies, [QList](#)< [QGraphicsItem](#) * > **drawCopy, [QList](#)< [QGraphicsItemGroup](#) * > **groupCopy)
Creates the edit-command controller used by [MainWindow](#) compatibility slots.
- void **onActionEditUndoTriggered** () const

- Delegates undo command handling without changing scene undo semantics.*
- void **onActionEditRedoTriggered** () const
- Delegates redo command handling without changing scene undo semantics.*
- void **onActionEditCutTriggered** () const
- Delegates cut command handling while preserving compatibility copy buffers.*
- void **onActionEditCopyTriggered** () const
- Delegates copy command handling while preserving compatibility copy buffers.*
- void **onActionEditPasteTriggered** () const
- Delegates paste command handling while preserving compatibility copy buffers.*
- void **onActionEditDeleteTriggered** () const
- Delegates delete command handling while preserving undo integration.*
- void **onActionEditGroupTriggered** () const
- Delegates group command handling through scene grouping APIs.*
- void **onActionEditUngroupTriggered** () const
- Delegates ungroup command handling through scene grouping APIs.*
- void **onActionViewGroupTriggered** () const
- Delegates view-group command handling through scene grouping APIs.*
- void **onActionViewUngroupTriggered** () const
- Delegates view-ungroup command handling through scene grouping APIs.*
- void **saveItemForCopy** (QList< [GraphicalModelComponent](#) * > *gmclist, QList< [GraphicalConnection](#) * > *connlist) const
- Stores eligible selected items in compatibility clipboard structures.*
- void **helpCopy** () const
- Performs deep-copy preparation for subsequent paste command execution.*

15.231.1 Detailed Description

Controller for edit/clipboard/grouping command orchestration in the scene.

This Phase-9 controller keeps [MainWindow](#) slots as compatibility wrappers while centralizing clipboard buffers, grouping operations, and undo-aware edit command dispatch against the current graphical scene.

Responsibilities:

- execute undo/redo/cut/copy/paste/delete/group/ungroup command flows;
- preserve legacy clipboard structures used by graphical components and drawings;
- trigger action refresh callbacks after command execution.

Boundaries:

- it does not own scene lifetime (resolved from injected graphics view);
- it does not persist clipboard state across sessions;
- it does not manage simulation events, model lifecycle, or property-editor synchronization.

The documentation for this class was generated from the following files:

- applications/gui/genesys/controllers/EditCommandController.h
- applications/gui/genesys/controllers/EditCommandController.cpp

15.232 EditorFactoryPrivate< Editor > Class Template Reference

Public Types

- typedef QList< Editor * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< Editor *, [QtProperty](#) * > **EditorToPropertyMap**

Public Member Functions

- Editor * **createEditor** (QtProperty *property, QWidget *parent)
- void **initializeEditor** (QtProperty *property, Editor *e)
- void **slotEditorDestroyed** (QObject *object)

Public Attributes

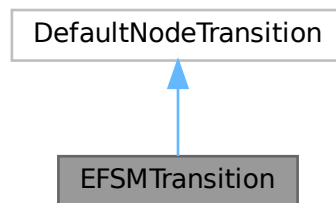
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

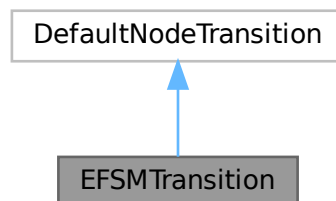
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.233 EFSMTransition Class Reference

Inheritance diagram for EFSMTransition:



Collaboration diagram for EFSMTransition:

**Public Member Functions**

- **EFSMTransition** (DefaultNode *source, DefaultNode *destination, std::string name="")
- void **setTriggerEvent** (std::string triggerEvent)
- std::string **getTriggerEvent** () const
- void **setProbabilityExpression** (std::string probabilityExpression)
- std::string **getProbabilityExpression** () const
- virtual bool **canFire** (Model *model, Entity *entity) const override
- virtual void **execute** (Model *model, Entity *entity) const override

Public Member Functions inherited from [DefaultNodeTransition](#)

- **DefaultNodeTransition** ([DefaultNode](#) *source, [DefaultNode](#) *destination, std::string name="")
- void **setSource** ([DefaultNode](#) *source)
- [DefaultNode](#) * **getSource** () const
- void **setDestination** ([DefaultNode](#) *destination)
- [DefaultNode](#) * **getDestination** () const
- void **setName** (std::string name)
- std::string **getName** () const
- void **setGuardExpression** (std::string guardExpression)
- std::string **getGuardExpression** () const
- void **setOutputExpression** (std::string outputExpression)
- std::string **getOutputExpression** () const
- void **setInputEvent** (std::string inputEvent)
- std::string **getInputEvent** () const
- void **setPriority** (unsigned int priority)
- unsigned int **getPriority** () const
- void **setProbability** (double probability)
- double **getProbability** () const
- void **setTransitionKind** (TransitionKind transitionKind)
- TransitionKind **getTransitionKind** () const

Additional Inherited Members

Public Types inherited from [DefaultNodeTransition](#)

- enum class **TransitionKind** : int { **DETERMINISTIC** = 0 , **PROBABILISTIC** = 1 }

15.233.1 Member Function Documentation

15.233.1.1 canFire()

```
bool EFSMTransition::canFire (
    Model * model,
    Entity * entity ) const [override], [virtual]
```

Reimplemented from [DefaultNodeTransition](#).

15.233.1.2 execute()

```
void EFSMTransition::execute (
    Model * model,
    Entity * entity ) const [override], [virtual]
```

Reimplemented from [DefaultNodeTransition](#).

The documentation for this class was generated from the following files:

- plugins/components/ModalModel/DefaultTransitionExtensions.h
- plugins/components/ModalModel/DefaultTransitionExtensions.cpp

15.234 testing::internal::ElemFromList< N, T > Struct Template Reference

Public Types

- using **type** = decltype([ElemFromListImpl](#)< typename MakeIndexSequence< N >::type >::Apply(static_↔ cast< T(*)>(>(nullptr)...))

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-internal.h

15.235 testing::internal::ElemFromListImpl< typename > Struct Template Reference

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-internal.h

15.236 testing::internal::ElemFromListImpl< IndexSequence< I... > > Struct Template Reference

Static Public Member Functions

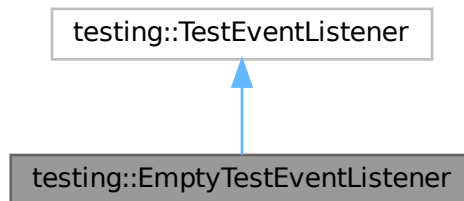
- template<typename R >
static R **Apply** ([Ignore](#)< 0 *I >..., R(*)(),...)

The documentation for this struct was generated from the following file:

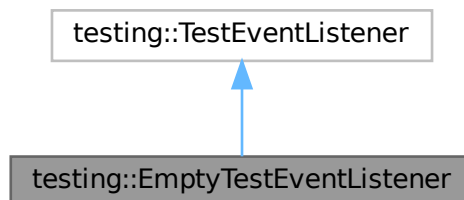
- gtest/gtest/internal/gtest-internal.h

15.237 testing::EmptyTestEventListener Class Reference

Inheritance diagram for testing::EmptyTestEventListener:



Collaboration diagram for testing::EmptyTestEventListener:



Public Member Functions

- void [OnTestProgramStart](#) (const [UnitTest](#) &) override

- void [OnTestIterationStart](#) (const [UnitTest](#) &, int) override
- void [OnEnvironmentsSetUpStart](#) (const [UnitTest](#) &) override
- void [OnEnvironmentsSetUpEnd](#) (const [UnitTest](#) &) override
- void [OnTestSuiteStart](#) (const [TestSuite](#) &) override
- void [OnTestCaseStart](#) (const [TestCase](#) &) override
- void [OnTestStart](#) (const [TestInfo](#) &) override
- void [OnTestDisabled](#) (const [TestInfo](#) &) override
- void [OnTestPartResult](#) (const [TestPartResult](#) &) override
- void [OnTestEnd](#) (const [TestInfo](#) &) override
- void [OnTestSuiteEnd](#) (const [TestSuite](#) &) override
- void [OnTestCaseEnd](#) (const [TestCase](#) &) override
- void [OnEnvironmentsTearDownStart](#) (const [UnitTest](#) &) override
- void [OnEnvironmentsTearDownEnd](#) (const [UnitTest](#) &) override
- void [OnTestIterationEnd](#) (const [UnitTest](#) &, int) override
- void [OnTestProgramEnd](#) (const [UnitTest](#) &) override

15.237.1 Member Function Documentation

15.237.1.1 OnEnvironmentsSetUpEnd()

```
void testing::EmptyTestEventListener::OnEnvironmentsSetUpEnd (
    const UnitTest & ) [inline], [override], [virtual]
```

Implements [testing::TestEventListener](#).

15.237.1.2 OnEnvironmentsSetUpStart()

```
void testing::EmptyTestEventListener::OnEnvironmentsSetUpStart (
    const UnitTest & ) [inline], [override], [virtual]
```

Implements [testing::TestEventListener](#).

15.237.1.3 OnEnvironmentsTearDownEnd()

```
void testing::EmptyTestEventListener::OnEnvironmentsTearDownEnd (
    const UnitTest & ) [inline], [override], [virtual]
```

Implements [testing::TestEventListener](#).

15.237.1.4 OnEnvironmentsTearDownStart()

```
void testing::EmptyTestEventListener::OnEnvironmentsTearDownStart (
    const UnitTest & ) [inline], [override], [virtual]
```

Implements [testing::TestEventListener](#).

15.237.1.5 OnTestCaseEnd()

```
void testing::EmptyTestEventListener::OnTestCaseEnd (
    const TestCase & ) [inline], [override], [virtual]
```

Reimplemented from [testing::TestEventListener](#).

15.237.1.6 OnTestCaseStart()

```
void testing::EmptyTestEventListener::OnTestCaseStart (
    const TestCase & ) [inline], [override], [virtual]
```

Reimplemented from [testing::TestEventListener](#).

15.237.1.7 OnTestDisabled()

```
void testing::EmptyTestEventListener::OnTestDisabled (
    const TestInfo & ) [inline], [override], [virtual]
```

Reimplemented from [testing::TestEventListener](#).

15.237.1.8 OnTestEnd()

```
void testing::EmptyTestEventListener::OnTestEnd (
    const TestInfo & ) [inline], [override], [virtual]
```

Implements [testing::TestEventListener](#).

15.237.1.9 OnTestIterationEnd()

```
void testing::EmptyTestEventListener::OnTestIterationEnd (
    const UnitTest & ,
    int ) [inline], [override], [virtual]
```

Implements [testing::TestEventListener](#).

15.237.1.10 OnTestIterationStart()

```
void testing::EmptyTestEventListener::OnTestIterationStart (
    const UnitTest & ,
    int ) [inline], [override], [virtual]
```

Implements [testing::TestEventListener](#).

15.237.1.11 OnTestPartResult()

```
void testing::EmptyTestEventListener::OnTestPartResult (
    const TestPartResult & ) [inline], [override], [virtual]
```

Implements [testing::TestEventListener](#).

15.237.1.12 OnTestProgramEnd()

```
void testing::EmptyTestEventListener::OnTestProgramEnd (
    const UnitTest & ) [inline], [override], [virtual]
```

Implements [testing::TestEventListener](#).

15.237.1.13 OnTestProgramStart()

```
void testing::EmptyTestEventListener::OnTestProgramStart (
    const UnitTest & ) [inline], [override], [virtual]
```

Implements [testing::TestEventListener](#).

15.237.1.14 OnTestStart()

```
void testing::EmptyTestEventListener::OnTestStart (
    const TestInfo & ) [inline], [override], [virtual]
```

Implements [testing::TestEventListener](#).

15.237.1.15 OnTestSuiteEnd()

```
void testing::EmptyTestEventListener::OnTestSuiteEnd (
    const TestSuite & ) [inline], [override], [virtual]
```

Reimplemented from [testing::TestEventListener](#).

15.237.1.16 OnTestSuiteStart()

```
void testing::EmptyTestEventListener::OnTestSuiteStart (
    const TestSuite & ) [inline], [override], [virtual]
```

Reimplemented from [testing::TestEventListener](#).

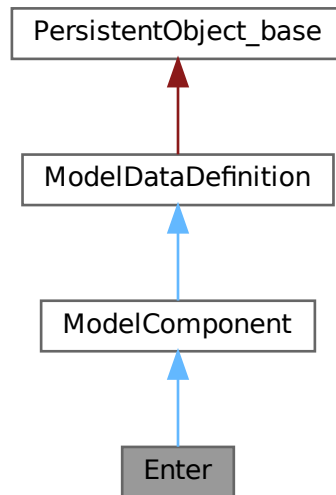
The documentation for this class was generated from the following file:

- `gtest/gtest/gtest.h`

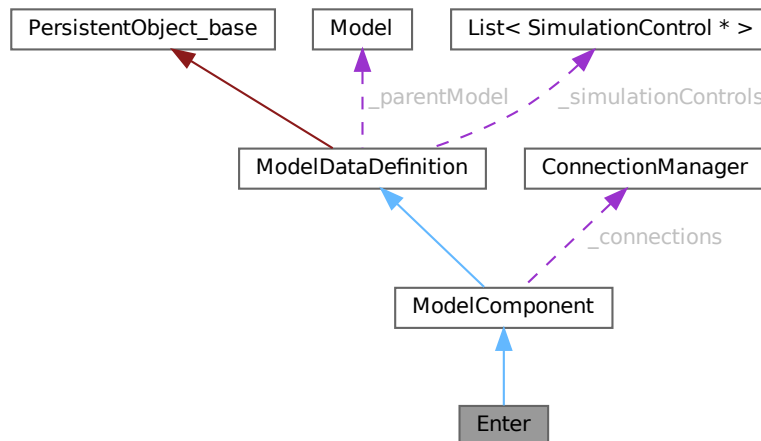
15.238 Enter Class Reference

```
#include <plugins/components/MaterialHandling/Enter.h>
```

Inheritance diagram for Enter:



Collaboration diagram for Enter:



Public Member Functions

- **Enter** ([Model](#) *model, std::string name="")
- virtual std::string [show](#) () override
virtual public methods
- void **setStation** ([Station](#) *_station)

- void **setStationName** (std::string stationName)
- [Station](#) * **getStation** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~[ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- [Model](#) * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * **getAttachedData** () const

- Returns the full map of attached data definitions.*
- bool `hasChanged` () const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged` (bool `hasChanged`)
Updates the changed flag for this data definition.
- unsigned int `getLevel` () const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel` (unsigned int `_modelLevel`)
Sets the model nesting level for this definition.
- List< `SimulationControl` * > * `getSimulationControls` () const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level` `getTraceLevelSpecific` () const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` `level`)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` `traceLevelSpecific`, bool `traceLevelSpecificEnabled`)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool `traceLevelSpecificEnabled`)
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation` * `GetPluginInformation` ()
- static `ModelComponent` * `LoadInstance` (`Model` *`model`, `PersistenceRecord` *`fields`)
- static `ModelDataDefinition` * `NewInstance` (`Model` *`model`, std::string `name`="")

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent` * `LoadInstance` (`Model` *`model`, `PersistenceRecord` *`fields`)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check` (`ModelComponent` *`component`)
Validates a component instance.
- static void `CreateInternalData` (`ModelComponent` *`component`)
Creates internal data required by a component instance.
- static void `SaveInstance` (`PersistenceRecord` *`fields`, `ModelComponent` *`component`)
Serializes a component instance into a persistence record.
- static void `DispatchEvent` (`Event` *`event`)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition` * `LoadInstance` (`Model` *`model`, `PersistenceRecord` *`fields`, bool `insertIntoModel`)
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition` * `NewInstance` (`Model` *`model`, std::string `name`="")
Creates a new instance of a data definition using the plugin registry.
- static void `SaveInstance` (`PersistenceRecord` *`fields`, `ModelDataDefinition` *`modeldatum`)
Serializes a data definition instance into a persistence record.
- static bool `Check` (`ModelDataDefinition` *`modeldatum`, std::string &`errorMessage`)
Validates a data definition instance.

- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createInternalStatisticReporters](#) () override
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()

- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()

virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.238.1 Detailed Description

[Enter](#) module DESCRIPTION The [Enter](#) module defines a station (or a set of stations) corresponding to a physical or logical location where processing occurs. When an entity arrives at an [Enter](#) module, an unloading delay may occur and any transfer device used to transfer the entity to the [Enter](#) module's station may be released. The station (or each station within the defined set) has a matching Activity Area that is used to report all times and costs accrued by the entities in this station. This Activity Area's name is the same as the station. If a parent Activity Area is defined, then it also accrues any times and costs by the entities in this station. TYPICAL USES The start of a part's production in a series of parallel processes where the part's forklift needs to be released The start of a document's processing after the document has been created where the mail clerk resource needs to be released PROMPTS Prompt Description Name Unique name of the module that will be displayed in the flowchart [Station](#) Type Type of station, either a single [Station](#) or station [Set](#). [Station](#) Name Name of the individual station. A given station can only exist once within a model. Parent Activity Area Name of the Activity Area's parent. Associated Intersection Name of the intersection associated with this station in a guided transporter network. Report Statistics Specifies whether or not statistics will automatically be collected and stored in the report database for this station and its corresponding activity area. [Set](#) Name Name of the station set. A given station set can only exist once

within a model. Save [Attribute](#) Specifies the attribute to be used to store the index into the station set for an entity entering this module. [Set Members](#) This repeat group permits you to define the individual stations that are to be members of the specified station set. A station set must have at least one member station. Active when [Station Type](#) is [Set](#). [Station Name](#) This field indicates the name of a station that is to be a member of this station set. A given station can only exist within a model once. Therefore, an individual station can only be the member of one station set, and that individual station may not be the name of a station in another module. [Parent Activity Area Name](#) of the Activity Area's parent for the station set member. [Associated Intersection Name](#) of the intersection associated with this station set in a guided transporter network. [Report Statistics](#) Specifies whether or not statistics will automatically be collected and stored in the report database for this station set member and its corresponding activity area. [Allocation Type](#) of category to which the entity's incurred delay time and cost will be added. [Delay](#) This field defines the delay that will be experienced by entities immediately upon arrival at the station. [Units Time](#) units used for the delay time. [Transfer In](#) If a resource, transporter, or conveyor was used to transfer the entity to this station, this can be used to release, free, or exit the device. If [Release Resource](#) is selected, the specified resource is released. If [Free Transporter](#) is selected, the specified transporter is freed. If [Exit Conveyor](#) is selected, the specified conveyor is exited. [Transporter Name](#) Name of the transporter to be freed upon arrival to the station. Active when [Transfer Name](#) is [Free Transporter](#). [Unit Number](#) Unit number of the transporter if the transporter is multicapacity. [Conveyor Name](#) Name of the conveyor to exit upon arrival to the station. [Resource Type](#) Type of allocation, either single [Resource](#) or resource [Set](#). [Resource Name](#) Name of the resource to release. Active when [Transfer Name](#) is [Release Resource](#). [Set Name](#) Name of the resource set from which the resource is to be released. [Release Rule](#) Determines which member of the set is to be released, either the Last Member Seized, First Member Seized, or Specific Member. [Set Index](#) Index into the set that determines which member of the set is to be released. [Attribute Name](#) Name of the attribute that determines the instance number of the resource to release. [Expression](#) Expression value that determines the instance number of the resource to release.

15.238.2 Member Function Documentation

15.238.2.1 `_check()`

```
bool Enter::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.238.2.2 `_createEditableDataDefinitions()`

```
void Enter::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.238.2.3 `_createInternalStatisticReporters()`

```
void Enter::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.238.2.4 `_loadInstance()`

```
bool Enter::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [ModelComponent](#).

15.238.2.5 `_onDispatchEvent()`

```
void Enter::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.238.2.6 _saveInstance()

```
void Enter::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.238.2.7 show()

```
std::string Enter::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

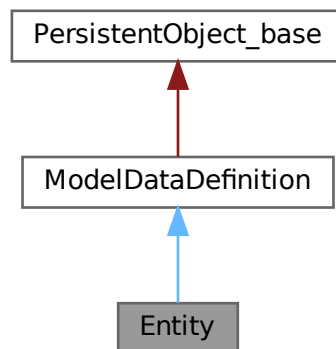
The documentation for this class was generated from the following files:

- plugins/components/MaterialHandling/Enter.h
- plugins/components/MaterialHandling/Enter.cpp

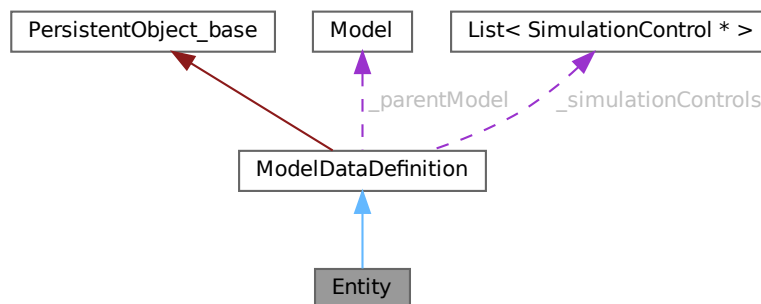
15.239 Entity Class Reference

```
#include <kernel/simulator/essentialPlugins/Entity.h>
```

Inheritance diagram for Entity:



Collaboration diagram for Entity:



Public Member Functions

- virtual std::string [show](#) () override
Returns a textual representation of this data definition.
- void [setEntityType](#) (std::string entityType)
setEntityType
- std::string [getEntityType](#) () const
getEntityType
- void [setEntityType](#) (EntityType *entityType)
setEntityType
- EntityType * [getEntityType](#) () const
getEntityType
- double [getAttributeValue](#) (std::string attributeName, std::string index="")
getAttributeValue
- double [getAttributeValue](#) (Util::identification attributeID, std::string index="")
getAttributeValue

- void `setAttributeValue` (std::string attributeName, double value, std::string index="", bool createIfNotFound=false)
setAttributeValue
- void `setAttributeValue` (Util::identification attributeID, double value, std::string index="")
setAttributeValue
- void `copyAttributeValues` (std::string destinationAttributeName, std::string sourceAttributeName)
Copies all sparse values from one attribute to another for this entity.
- Util::identification `entityNumber` () const
entityNumber

Public Member Functions inherited from `ModelDataDefinition`

- `ModelDataDefinition` (`Model` *model, std::string dataDefinitionTypeName, std::string name="", bool insertIntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~`ModelDataDefinition` ()
Releases resources owned by this data definition.
- Util::identification `getId` () const
Returns the unique identifier of this instance.
- void `setName` (const std::string &name)
Updates the logical name of this instance.
- const std::string & `getName` () const
Returns the logical name of this instance.
- void `setLabel` (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & `getLabel` () const
Returns the user-visible label of this instance.
- const std::string & `getClassname` () const
Returns the runtime class name used by the plugin system.
- `Model` * `getParentModel` () const
Returns the parent model that owns this definition.
- bool `isReportStatistics` () const
Indicates whether this data definition contributes report statistics.
- void `setReportStatistics` (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition` * `getInternalData` (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, `ModelDataDefinition` * > * `getInternalData` () const
Returns the full map of internal child data definitions.
- std::map< std::string, `ModelDataDefinition` * > * `getAttachedData` () const
Returns the full map of attached data definitions.
- bool `hasChanged` () const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged` (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int `getLevel` () const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel` (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< `SimulationControl` * > * `getSimulationControls` () const
Returns the simulation controls owned by this data definition.

- `TraceManager::Level` `getTraceLevelSpecific ()` const
Returns the trace level that applies specifically to this definition.
- `void` `setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void` `defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool` `isTraceLevelSpecificEnabled ()` const
Indicates whether the specific trace level is enabled.
- `void` `setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Protected Member Functions

- `virtual bool` `_loadInstance (PersistenceRecord *fields)` override
must be overridden by derived classes
- `virtual void` `_saveInstance (PersistenceRecord *fields, bool saveDefaultValues)` override
Saves object fields into a serialization structure.
- `virtual bool` `_check (std::string &errorMessage)` override
could be overridden by derived classes

Protected Member Functions inherited from `ModelDataDefinition`

- `bool` `_getSaveDefaultsOption ()`
- `virtual ParserChangesInformation *` `_getParserChangesInformation ()`
- `virtual void` `_initBetweenReplications ()`
- `virtual void` `_addSimulationControl (SimulationControl *control)`
- `virtual void` `_createInternalAndAttachedData ()`
- `virtual void` `_createInternalStatisticReporters ()`
- `virtual void` `_createAttachedAttributes ()`
- `virtual void` `_createNonEditableDataDefinitions ()`
- `virtual void` `_createEditableDataDefinitions ()`
- `void` `_templateCreateInternalStatisticReporters ()`
- `void` `_templateCreateAttachedAttributes ()`
- `void` `_templateCreateNonEditableDataDefinitions ()`
- `void` `_templateCreateEditableDataDefinitions ()`
- `void` `_internalStatisticReportersClear ()`
legacy helper methods used by the granular create hooks*
- `void` `_internalDataClear ()`
- `void` `_internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void` `_internalDataRemove (const std::string &key)`
- `void` `_attachedDataClear ()`
- `void` `_attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void` `_attachedDataRemove (const std::string &key)`
- `void` `_attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void` `_statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void` `_statisticReporterRemove (const std::string &key)`
- `void` `_statisticReportersClear ()`
- `void` `_mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void` `_mandatoryAttachedAttributeRemove (const std::string &key)`
- `void` `_mandatoryAttachedAttributesClear ()`
- `void` `_mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void` `_mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void` `_mandatoryEditableDataDefinitionsClear ()`

- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Friends

- class **Model**

Additional Inherited Members

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**

- `List< SimulationControl * > *_simulationControls = new List<SimulationControl*>()`
- `TraceManager::Level _traceLevelSpecific`
- `bool _traceLevelSpecificEnabled = false`

15.239.1 Detailed Description

Entity module DESCRIPTION This data module defines the various entity types and their initial picture values in a simulation. Initial costing information and holding costs are also defined for the entity. TYPICAL USES Items being produced or assembled (parts, pallets) Documents (forms, e-mails, faxes, reports) People moving through a process (customers, callers) PROMPTS Prompt Description Name The unique name of the attribute being defined. Rows Number of rows in a one- or two-dimensional attribute. Columns Number of columns in a two-dimensional attribute. Data Type The data type of the values stored in the attribute. Valid types are Real and String. The default type is Real. Initial Values Lists the initial value or values of the attribute. You can assign new values to the attribute by using the [Assign](#) module. Initial Value **Entity** attribute value when entity is created and enters the system. Prompt Description **Entity** Type The name of the entity type being defined. This name must be unique. Initial Picture Graphical representation of the entity at the start of the simulation. This value can be changed during the simulation using the [Assign](#) module. Holding Cost/Hour Hourly cost of processing the entity through the system. This cost is incurred when the entity is anywhere in the system. Initial VA Cost Initial cost value that will be assigned to the value-added cost attribute of the entity. This attribute accrues the costs incurred when an entity is spending time in a value-added activity. Initial NVA Cost Initial cost value that will be assigned to the non-value-added cost attribute of the entity. This attribute accrues the costs incurred when an entity is spending time in a non-value-added activity. Initial **Waiting** Cost Initial cost value that will be assigned to the waiting-cost attribute of the entity. This attribute accrues the costs incurred when an entity is spending time in a wait activity; for example, waiting to be batched or waiting for resource(s) at a [Process](#) module. Initial Transfer Cost Initial cost value that will be assigned to the transfer cost attribute of the entity. This attribute accrues the costs incurred when an entity is spending time in a transfer activity. Initial Other Cost Initial cost value that will be assigned to the other cost attribute of the entity. This attribute accrues the costs incurred when an entity is spending time in another activity. Report Statistics Specifies whether or not statistics will be collected automatically and stored in the report database for this entity type.

15.239.2 Member Function Documentation

15.239.2.1 `_check()`

```
bool Entity::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.239.2.2 `_loadInstance()`

```
bool Entity::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.239.2.3 `_saveInstance()`

```
void Entity::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.239.2.4 entityNumber()

```
Util::identification Entity::entityNumber ( ) const  
entityNumber
```

Returns

Unique runtime identifier for this entity instance.

15.239.2.5 getAttributeValue() [1/2]

```
double Entity::getAttributeValue (   
    std::string attributeName,  
    std::string index = "" )  
getAttributeValue
```

Parameters

<i>index</i>	
<i>attributeName</i>	

Returns

Current attribute value for the provided name/index.

15.239.2.6 getAttributeValue() [2/2]

```
double Entity::getAttributeValue (   
    Util::identification attributeID,  
    std::string index = "" )  
getAttributeValue
```

Parameters

<i>index</i>	
<i>attributeID</i>	

Returns

Current attribute value for the provided id/index.

15.239.2.7 getEntityType()

```
EntityType * Entity::getEntityType ( ) const  
getEntityType
```

Returns

Pointer to the associated entity type.

15.239.2.8 getEntityTypeName()

```
std::string Entity::getEntityTypeName ( ) const  
getEntityTypeName
```

Returns

Name of the associated entity type.

15.239.2.9 setAttributeValue() [1/2]

```
void Entity::setAttributeValue (
    std::string attributeName,
    double value,
    std::string index = "",
    bool createIfNotFound = false )
```

setAttributeValue

Parameters

<i>index</i>	
<i>attributeName</i>	
<i>value</i>	
<i>createIfNotFound</i>	

Assigns an attribute value by name and optionally creates missing attributes.

15.239.2.10 setAttributeValue() [2/2]

```
void Entity::setAttributeValue (
    Util::identification attributeID,
    double value,
    std::string index = "" )
```

setAttributeValue

Parameters

<i>index</i>	
<i>attributeID</i>	
<i>value</i>	

Assigns an attribute value by attribute identifier.

15.239.2.11 setEntityType()

```
void Entity::setEntityType (
    EntityType * entityType )
```

setEntityType

Parameters

<i>entityType</i>	
-------------------	--

Directly assigns the entity type pointer.

15.239.2.12 setEntityTypeName()

```
void Entity::setEntityTypeName (
    std::string entityTypeName )
```

setEntityTypeName

Parameters

<i>entityTypeName</i>	
-----------------------	--

Sets entity type by name (indirect lookup through model data manager).

15.239.2.13 show()

```
std::string Entity::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

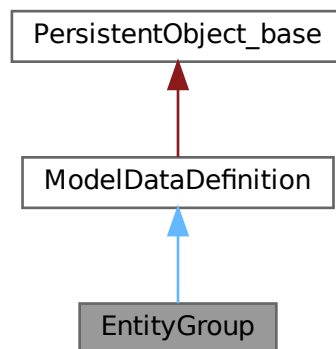
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

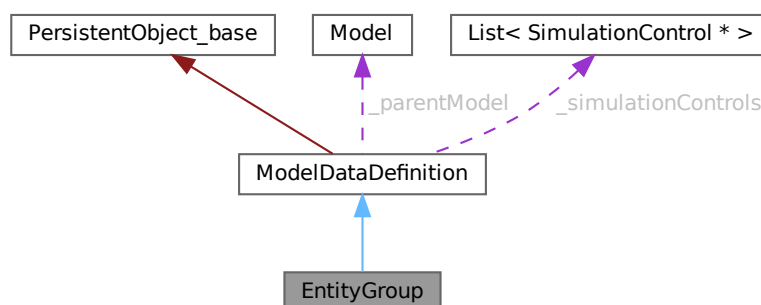
- kernel/simulator/essentialPlugins/Entity.h
- kernel/simulator/essentialPlugins/Entity.cpp

15.240 EntityGroup Class Reference

Inheritance diagram for EntityGroup:



Collaboration diagram for EntityGroup:



Public Member Functions

- **EntityGroup** ([Model](#) *model, std::string name="")
- virtual std::string [show](#) () override
Returns a textual representation of this data definition.
- void **insertElement** (unsigned int idKey, [Entity](#) *modeldatum)
- void **removeElement** (unsigned int idKey, [Entity](#) *modeldatum)
- [List](#)< [Entity](#) * > * **getGroup** (unsigned int idKey)

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager](#)::Level [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.

- void [setTraceLevelSpecific](#) (TraceManager::Level level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_createInternalStatisticReporters](#) () override
- virtual void [_createAttachedAttributes](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()

- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internaStatisticReportersClear** ()
 - legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.240.1 Member Function Documentation

15.240.1.1 `_check()`

```
bool EntityGroup::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.240.1.2 `_createAttachedAttributes()`

```
void EntityGroup::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.240.1.3 `_createInternalStatisticReporters()`

```
void EntityGroup::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.240.1.4 `_initBetweenReplications()`

```
void EntityGroup::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.240.1.5 `_loadInstance()`

```
bool EntityGroup::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.240.1.6 `_saveInstance()`

```
void EntityGroup::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.240.1.7 `show()`

```
std::string EntityGroup::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

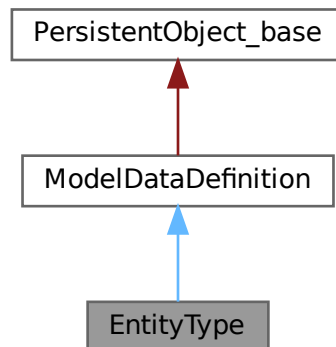
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

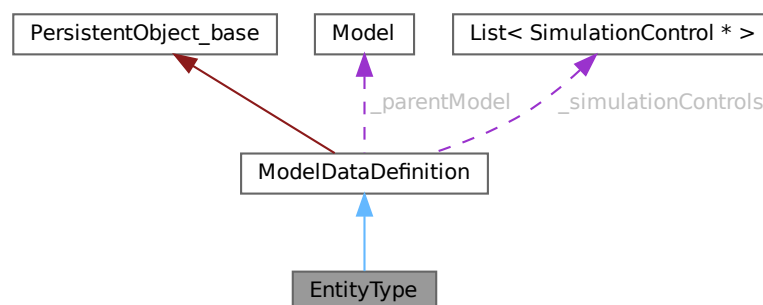
- `plugins/data/Grouping/EntityGroup.h`
- `plugins/data/Grouping/EntityGroup.cpp`

15.241 EntityType Class Reference

Inheritance diagram for EntityType:



Collaboration diagram for EntityType:



Public Member Functions

- **EntityType** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- void **setInitialWaitingCost** (double _initialWaitingCost)
- double **initialWaitingCost** () const
- void **setInitialOtherCost** (double _initialOtherCost)
- double **initialOtherCost** () const
- void **setInitialNVACost** (double _initialNVACost)
- double **initialNVACost** () const
- void **setInitialVACost** (double _initialVACost)
- double **initialVACost** () const
- void **setInitialPicture** (std::string _initialPicture)
- std::string **initialPicture** () const

- [StatisticsCollector](#) * [addGetStatisticsCollector](#) (std::string name)
addGetStatisticsCollector

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) (TraceManager::Level level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)
Defines the specific trace level and optionally enables it immediately.

- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_initBetweenReplications](#) () override
- virtual void [_createInternalStatisticReporters](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*

- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.241.1 Member Function Documentation

15.241.1.1 _createInternalStatisticReporters()

void EntityType::_createInternalStatisticReporters () [override], [protected], [virtual]
Reimplemented from [ModelDataDefinition](#).

15.241.1.2 _initBetweenReplications()

void EntityType::_initBetweenReplications () [override], [protected], [virtual]
Reimplemented from [ModelDataDefinition](#).

15.241.1.3 _loadInstance()

bool EntityType::_loadInstance (
 [PersistenceRecord](#) * *fields*) [override], [protected], [virtual]
must be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.241.1.4 _saveInstance()

void EntityType::_saveInstance (
 [PersistenceRecord](#) * *fields*,
 bool *saveDefaultValues*) [override], [protected], [virtual]
Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.241.1.5 addGetStatisticsCollector()

[StatisticsCollector](#) * EntityType::addGetStatisticsCollector (
 std::string *name*)
addGetStatisticsCollector

Parameters

<i>name</i>	
-------------	--

Returns

15.241.1.6 show()

std::string EntityType::show () [override], [virtual]
Returns a textual representation of this data definition.
Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- kernel/simulator/essentialPlugins/EntityType.h
- kernel/simulator/essentialPlugins/EntityType.cpp

15.242 PersistenceRecord::Entry Struct Reference

"Raw" entry, users should treat this as an opaque type since payloads could be in any format.

```
#include <kernel/simulator/Persistence.h>
```

Public Types

- enum **Kind** { **text** , **numeric** }

Public Attributes

- std::string **first**
- Payload **second**
- Kind **kind** {Kind::text}

15.242.1 Detailed Description

"Raw" entry, users should treat this as an opaque type since payloads could be in any format.

The documentation for this struct was generated from the following file:

- kernel/simulator/Persistence.h

15.243 testing::Environment Class Reference

Public Member Functions

- virtual void **SetUp** ()
- virtual void **TearDown** ()

The documentation for this class was generated from the following file:

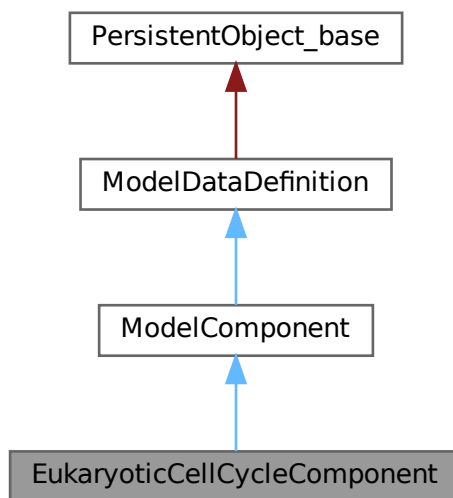
- gtest/gtest/gtest.h

15.244 EukaryoticCellCycleComponent Class Reference

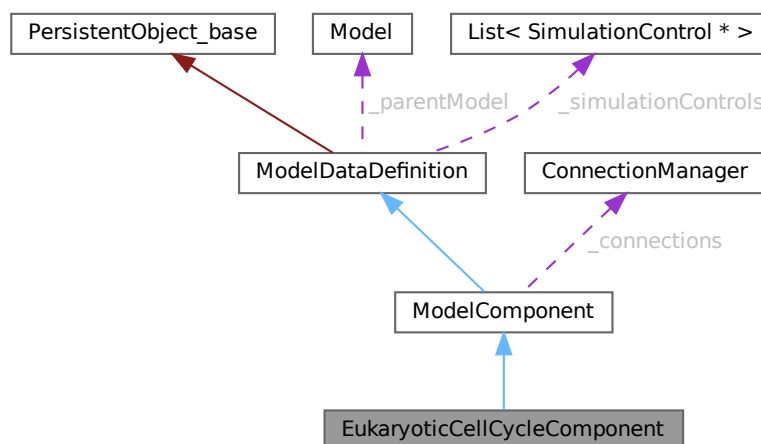
Coordinates a didactic eukaryotic cell cycle over a shared [WholeCellState](#).

```
#include <plugins/components/WholeCellModeling/EukaryoticCellCycleComponent.h>
```


Inheritance diagram for EukaryoticCellCycleComponent:



Collaboration diagram for EukaryoticCellCycleComponent:



Public Member Functions

- **EukaryoticCellCycleComponent** ([Model](#) *model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **setDeltaT** (double deltaT)
- double **getDeltaT** () const
- void **setAdvanceWholeCellClock** (bool advance)
- bool **getAdvanceWholeCellClock** () const

- void **setCytosolCompartmentName** (std::string name)
- std::string **getCytosolCompartmentName** () const
- void **setNucleusCompartmentName** (std::string name)
- std::string **getNucleusCompartmentName** () const
- void **setMitochondrionCompartmentName** (std::string name)
- std::string **getMitochondrionCompartmentName** () const
- void **setBudCompartmentName** (std::string name)
- std::string **getBudCompartmentName** () const
- void **setEnergyMetaboliteKey** (std::string key)
- std::string **getEnergyMetaboliteKey** () const
- void **setCytosolicEnergyMetaboliteKey** (std::string key)
- std::string **getCytosolicEnergyMetaboliteKey** () const
- void **setMitochondrialEnergyMetaboliteKey** (std::string key)
- std::string **getMitochondrialEnergyMetaboliteKey** () const
- void **setBudEnergyMetaboliteKey** (std::string key)
- std::string **getBudEnergyMetaboliteKey** () const
- void **setGrowthFluxKey** (std::string key)
- std::string **getGrowthFluxKey** () const
- void **setRespirationFluxKey** (std::string key)
- std::string **getRespirationFluxKey** () const
- void **setBudProgressKey** (std::string key)
- std::string **getBudProgressKey** () const
- void **setDnaReplicationProgressKey** (std::string key)
- std::string **getDnaReplicationProgressKey** () const
- void **setSpindleAssemblyProgressKey** (std::string key)
- std::string **getSpindleAssemblyProgressKey** () const
- void **setMitoticExitProgressKey** (std::string key)
- std::string **getMitoticExitProgressKey** () const
- void **setGlobalAtpThreshold** (double threshold)
- double **getGlobalAtpThreshold** () const
- void **setCytosolicAtpThreshold** (double threshold)
- double **getCytosolicAtpThreshold** () const
- void **setMitochondrialAtpThreshold** (double threshold)
- double **getMitochondrialAtpThreshold** () const
- void **setBudAtpThreshold** (double threshold)
- double **getBudAtpThreshold** () const
- void **setGrowthFluxThreshold** (double threshold)
- double **getGrowthFluxThreshold** () const
- void **setRespirationFluxThreshold** (double threshold)
- double **getRespirationFluxThreshold** () const
- void **setBudProgressRate** (double rate)
- double **getBudProgressRate** () const
- void **setDnaReplicationRate** (double rate)
- double **getDnaReplicationRate** () const
- void **setSpindleAssemblyRate** (double rate)
- double **getSpindleAssemblyRate** () const
- void **setMitoticExitRate** (double rate)
- double **getMitoticExitRate** () const
- void **setBudProgressThreshold** (double threshold)
- double **getBudProgressThreshold** () const
- void **setDnaReplicationThreshold** (double threshold)
- double **getDnaReplicationThreshold** () const
- void **setSpindleAssemblyThreshold** (double threshold)
- double **getSpindleAssemblyThreshold** () const
- void **setMitoticExitThreshold** (double threshold)
- double **getMitoticExitThreshold** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~[ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.

- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int [_modelLevel](#))
Sets the model nesting level for this definition.
- [List< SimulationControl * > * getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) [traceLevelspecific](#), bool [traceLevelSpecificEnabled](#)=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool [traceLevelSpecificEnabled](#))
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)

Creates internal and attached data required by this instance.

- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)

Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()

- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.244.1 Detailed Description

Coordinates a didactic eukaryotic cell cycle over a shared [WholeCellState](#).

[EukaryoticCellCycleComponent](#) provides an explicit, event-driven lifecycle layer for budding-yeast-like whole-cell models. It interprets shared state variables and compartment-specific energy proxies, then annotates [WholeCellState](#) with a coarse-grained eukaryotic cycle:

- newborn
- g1_budding
- s_phase
- g2_phase
- m_phase
- division_ready
- arrested
- dead

The component does not execute metabolism, transport, expression, or division itself. Instead, it consumes their outputs from [WholeCellState](#) and advances four progress variables stored as pathway activities:

- bud growth progress
- DNA replication progress
- spindle assembly progress
- mitotic exit progress

This design keeps structural biology in [BioCompartment](#), mutable system state in [WholeCellState](#), and dynamic checkpoint logic in a dedicated [ModelComponent](#).

15.244.2 Member Function Documentation

15.244.2.1 `_check()`

```
bool EukaryoticCellCycleComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.244.2.2 `_createEditableDataDefinitions()`

```
void EukaryoticCellCycleComponent::_createEditableDataDefinitions ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.244.2.3 `_initBetweenReplications()`

```
void EukaryoticCellCycleComponent::_initBetweenReplications ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.244.2.4 `_loadInstance()`

```
bool EukaryoticCellCycleComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.244.2.5 `_onDispatchEvent()`

```
void EukaryoticCellCycleComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.244.2.6 `_saveInstance()`

```
void EukaryoticCellCycleComponent::_saveInstance (
    PersistenceRecord * fields,
```

```
bool saveDefaultValues () [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.244.2.7 show()

```
std::string EukaryoticCellCycleComponent::show () [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

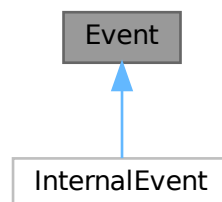
The documentation for this class was generated from the following files:

- plugins/components/WholeCellModeling/EukaryoticCellCycleComponent.h
- plugins/components/WholeCellModeling/EukaryoticCellCycleComponent.cpp

15.245 Event Class Reference

```
#include <kernel/simulator/Event.h>
```

Inheritance diagram for Event:



Public Member Functions

- **Event** (double time, [Entity](#) *entity, [ModelComponent](#) *component, unsigned int componentinputPort↵
Number=0)
- **Event** (double time, [Entity](#) *entity, [Connection](#) *connection)
- double [getTime](#) () const
getTime
- [ModelComponent](#) * [getComponent](#) () const
getComponent
- [Entity](#) * [getEntity](#) () const
getEntity
- unsigned int [getComponentinputPortNumber](#) () const

- getComponentinputPortNumber*
- void **setEntity** (**Entity** *_entity)
- setEntity*
- virtual std::string **show** ()

Protected Attributes

- double **_time**

15.245.1 Detailed Description

An instantaneous event, triggered at a certain moment by an entity upon reaching a component. The simulated time advances in discrete points in time and that are the instants that an event is triggered.

15.245.2 Member Function Documentation

15.245.2.1 getComponent()

```
ModelComponent * Event::getComponent ( ) const  
getComponent
```

Returns

15.245.2.2 getComponentinputPortNumber()

```
unsigned int Event::getComponentinputPortNumber ( ) const  
getComponentinputPortNumber
```

Returns

15.245.2.3 getEntity()

```
Entity * Event::getEntity ( ) const  
getEntity
```

Returns

15.245.2.4 getTime()

```
double Event::getTime ( ) const  
getTime
```

Returns

15.245.2.5 setEntity()

```
void Event::setEntity (  
    Entity * _entity )  
setEntity
```

Parameters

<code>_entity</code>	
----------------------	--

The documentation for this class was generated from the following files:

- kernel/simulator/Event.h
- kernel/simulator/Event.cpp

15.246 Exact Class Reference

Public Member Functions

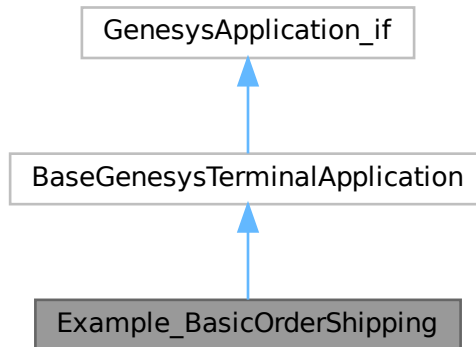
- **Exact** (int num, int den=1)
- const [Exact](#) & **operator+** (const [Exact](#) &right)
- const [Exact](#) & **operator+=** (const [Exact](#) &right)
- const [Exact](#) & **operator-=** (const [Exact](#) &right)
- const [Exact](#) & **operator*=** (const [Exact](#) &right)
- const [Exact](#) & **operator/=** (const [Exact](#) &right)
- const [Exact](#) & **operator=** (const [Exact](#) &right)
- const [Exact](#) & **operator=** (const int &right)
- bool **operator==** (const [Exact](#) &right)
- bool **operator==** (const double &right)
- bool **operator<** (const [Exact](#) &right)
- bool **operator<** (const double &right)
- bool **operator<=** (const [Exact](#) &right)
- bool **operator<=** (const double &right)
- bool **operator>** (const [Exact](#) &right)
- bool **operator>** (const double &right)
- bool **operator>=** (const [Exact](#) &right)
- bool **operator>=** (const double &right)
- void **Simplify** ()
- double **toFloat** ()

The documentation for this class was generated from the following file:

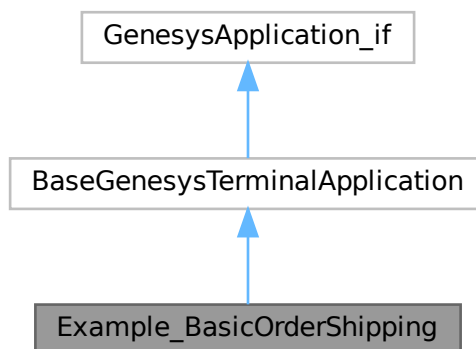
- kernel/util/Exact.h

15.247 Example_BasicOrderShipping Class Reference

Inheritance diagram for Example_BasicOrderShipping:



Collaboration diagram for Example_BasicOrderShipping:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

15.247.1 Member Function Documentation

15.247.1.1 main()

```
int Example_BasicOrderShipping::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

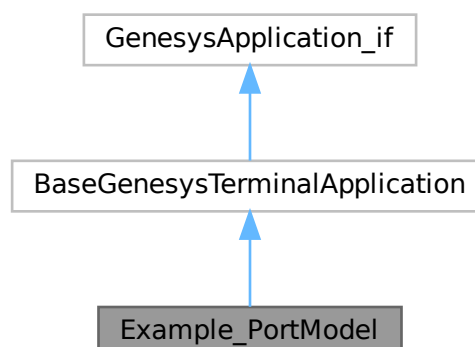
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

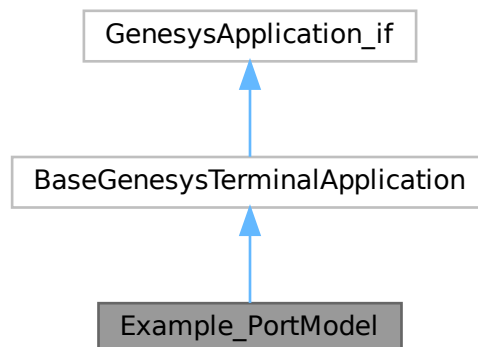
- applications/modelSpecific/arenaExamples/Example_BasicOrderShipping.h
- applications/modelSpecific/arenaExamples/Example_BasicOrderShipping.cpp

15.248 Example_PortModel Class Reference

Inheritance diagram for Example_PortModel:



Collaboration diagram for Example_PortModel:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

15.248.1 Member Function Documentation

15.248.1.1 main()

```
int Example_PortModel::main (  
    int argc,  
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

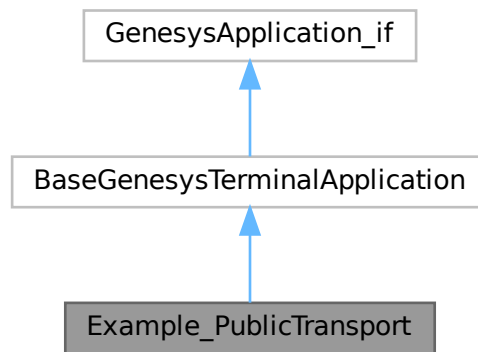
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

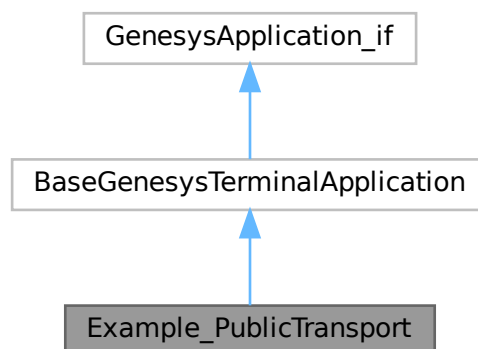
- applications/modelSpecific/arenaExamples/Example_PortModel.h
- applications/modelSpecific/arenaExamples/Example_PortModel.cpp

15.249 Example_PublicTransport Class Reference

Inheritance diagram for Example_PublicTransport:



Collaboration diagram for Example_PublicTransport:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

15.249.1 Member Function Documentation

15.249.1.1 main()

```
int Example_PublicTransport::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/arenaExamples/Example_PublicTransport.h
- applications/modelSpecific/arenaExamples/Example_PublicTransport.cpp

15.250 CompartmentExchangeComponent::ExchangeRule Struct Reference

Public Attributes

- std::string **label**
- std::string **sourceRegion**
- std::string **sourceMetaboliteKey**
- std::string **targetRegion**
- std::string **targetMetaboliteKey**

- double **exchangeFraction** = 0.0
- double **baselineAmount** = 0.0
- std::string **driverPathwayKey**
- double **driverScale** = 1.0
- double **maxTransferAmount** = 0.0
- bool **conserveMass** = true

The documentation for this struct was generated from the following file:

- `plugins/components/WholeCellModeling/CompartmentExchangeComponent.h`

15.251 GroProgramRuntime::ExecutionResult Struct Reference

Public Attributes

- bool **succeeded** = true
- std::string **errorMessage** = ""
- unsigned int **executedCommands** = 0
- std::vector< [PopulationMutation](#) > **populationMutations**
- std::vector< [SignalMutation](#) > **signalMutations**
- std::vector< [MotionMutation](#) > **motionMutations**
- std::vector< [ColonyMutation](#) > **colonyMutations**
- std::vector< double > **mappedCellValues**
- std::string **mappedCellExpression**
- std::vector< double > **signalMatrixValues**
- unsigned int **signalMatrixWidth** = 0
- unsigned int **signalMatrixHeight** = 0
- std::string **signalFieldDump**
- std::vector< std::string > **messages**
- std::map< std::string, double > **assignedVariables**
- std::vector< std::string > **unsupportedCommands**
- std::vector< std::string > **skippedRawStatements**

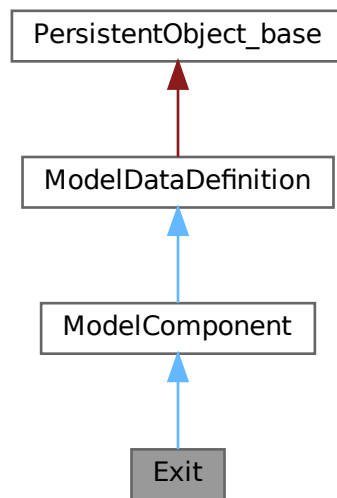
The documentation for this struct was generated from the following file:

- `plugins/data/BiochemicalSimulation/GroProgramRuntime.h`

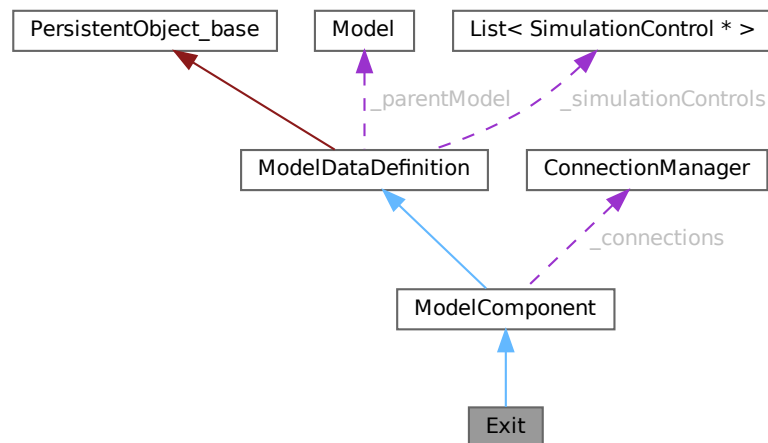
15.252 Exit Class Reference

```
#include <plugins/components/MaterialHandling/Exit.h>
```


Inheritance diagram for Exit:



Collaboration diagram for Exit:



Public Member Functions

- **Exit** (**Model** *model, std::string name="")
 - virtual std::string **show** () override
- virtual public methods*

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")

constructors

- virtual **~ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual **~ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.

- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecific←
Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void `_onDispatchEvent` (`Entity` *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool `_loadInstance` (`PersistenceRecord` *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` (`PersistenceRecord` *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from `ModelDataDefinition`

- bool `_getSaveDefaultsOption` ()
- virtual `ParserChangesInformation` * `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` (`SimulationControl` *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, `ModelDataDefinition` *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * [_connections](#) = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification [_id](#)
- std::string [_typename](#)
- bool [_reportStatistics](#)
- bool [_hasChanged](#)
- unsigned int [_modelLevel](#) = 0
- [Model](#) * [_parentModel](#)
- [List](#)< [SimulationControl](#) * > * [_simulationControls](#) = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level [_traceLevelSpecific](#)
- bool [_traceLevelSpecificEnabled](#) = false

15.252.1 Detailed Description

[Exit](#) module DESCRIPTION The [Exit](#) module releases the entity's cells on the specified conveyor. If another entity is waiting in queue for the conveyor at the same station when the cells are released, it will then access the conveyor. TYPICAL USES Cases exit a conveyor for packing Bad parts are removed from the conveyor and disposed Passengers remove luggage from the baggage claim conveyor PROMPTS Prompt Description Name Unique name of the module that will be displayed in the flowchart. Conveyor Name Name of the conveyor on which the entity will exit. If left blank, the previously accessed conveyor is assumed.

15.252.2 of Cells Number of contiguous conveyor cells the entity will relinquish.

15.252.3 Member Function Documentation

15.252.3.1 [_check\(\)](#)

```
bool Exit::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.252.3.2 [_loadInstance\(\)](#)

```
bool Exit::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [ModelComponent](#).

15.252.3.3 `_onDispatchEvent()`

```
void Exit::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.252.3.4 `_saveInstance()`

```
void Exit::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.252.3.5 `show()`

```
std::string Exit::show ( ) [override], [virtual]
virtual public methods
```

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- `plugins/components/MaterialHandling/Exit.h`
- `plugins/components/MaterialHandling/Exit.cpp`

15.253 ExperimentManager Class Reference

Manages higher-level simulation experiments.

```
#include <kernel/simulator/ExperimentManager.h>
```

Public Member Functions

- **ExperimentManager** ([Simulator](#) *simulator)
- [SimulationExperiment](#) * **newSimulationExperiment** ()
- void **insert** ([SimulationExperiment](#) *experiment)
- void **remove** ([SimulationExperiment](#) *experiment)
- void **setCurrent** ([SimulationExperiment](#) *experiment)
- bool **saveSimulationExperiment** (std::string filename)
- bool **loadSimulationExperiment** (std::string filename)

- unsigned int **size** ()
- [SimulationExperiment](#) * **front** ()
- [SimulationExperiment](#) * **current** ()
- [SimulationExperiment](#) * **next** ()
- [List](#)< [SimulationExperiment](#) * > * **getExperiments** () const

15.253.1 Detailed Description

Manages higher-level simulation experiments.

Historically, GenESyS initially supported only the basic experiment controls available through [ModelSimulation](#), especially the number of replications and the replication length.

This class represents a later attempt to move more advanced experiment design concerns into the simulation kernel, including higher-level experiment sets, scenario management and future [Process](#) Analyzer-like workflows.

This layer is still under development and should be treated as an incomplete experimental abstraction on top of the already functional [ModelSimulation](#) mechanism.

The documentation for this class was generated from the following files:

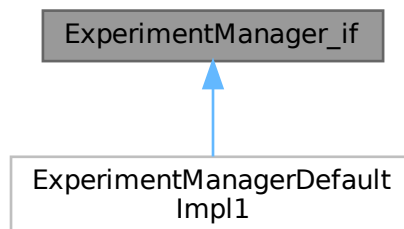
- kernel/simulator/ExperimentManager.h
- kernel/simulator/ExperimentManager.cpp

15.254 ExperimentManager_if Class Reference

Interface that coordinates execution of scenario-based simulation experiments.

```
#include <kernel/simulator/ExperimentManager_if.h>
```

Inheritance diagram for ExperimentManager_if:



Public Member Functions

- virtual [List](#)< [SimulationScenario](#) * > * **getScenarios** () const =0
Returns the list of scenarios that compose the experiment.
- virtual void **startSimulationOfScenario** ([SimulationScenario](#) *scenario)=0
Starts simulation for a specific scenario.
- virtual void **startExperiment** ()=0
Starts full experiment execution (all enabled scenarios).
- virtual void **stopExperiment** ()=0
Requests interruption of the currently running experiment.
- virtual void **addTraceSimulationHandler** (traceSimulationProcessListener traceSimulationProcessListener)=0
Registers a callback to receive simulation progress traces for the experiment.

15.254.1 Detailed Description

Interface that coordinates execution of scenario-based simulation experiments.

An experiment manager owns a set of scenarios, starts/stops scenario runs and exposes progress tracing hooks so higher-level tools can monitor experiment status.

15.254.2 Member Function Documentation

15.254.2.1 addTraceSimulationHandler()

```
virtual void ExperimentManager_if::addTraceSimulationHandler (
    traceSimulationProcessListener traceSimulationProcessListener ) [pure virtual]
```

Registers a callback to receive simulation progress traces for the experiment.

Implemented in [ExperimentManagerDefaultImpl1](#).

15.254.2.2 getScenarios()

```
virtual List< SimulationScenario * > * ExperimentManager_if::getScenarios ( ) const [pure
virtual]
```

Returns the list of scenarios that compose the experiment.

Implemented in [ExperimentManagerDefaultImpl1](#).

15.254.2.3 startExperiment()

```
virtual void ExperimentManager_if::startExperiment ( ) [pure virtual]
```

Starts full experiment execution (all enabled scenarios).

Implemented in [ExperimentManagerDefaultImpl1](#).

15.254.2.4 startSimulationOfScenario()

```
virtual void ExperimentManager_if::startSimulationOfScenario (
    SimulationScenario * scenario ) [pure virtual]
```

Starts simulation for a specific scenario.

Implemented in [ExperimentManagerDefaultImpl1](#).

15.254.2.5 stopExperiment()

```
virtual void ExperimentManager_if::stopExperiment ( ) [pure virtual]
```

Requests interruption of the currently running experiment.

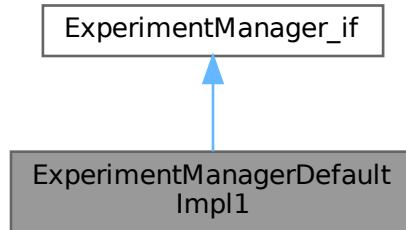
Implemented in [ExperimentManagerDefaultImpl1](#).

The documentation for this class was generated from the following file:

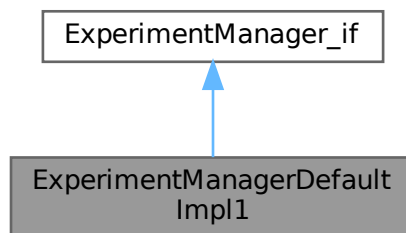
- kernel/simulator/ExperimetManager_if.h

15.255 ExperimentManagerDefaultImpl1 Class Reference

Inheritance diagram for ExperimentManagerDefaultImpl1:



Collaboration diagram for ExperimentManagerDefaultImpl1:



Public Member Functions

- virtual [List< SimulationScenario * > * getScenarios \(\)](#) const override
Returns the list of scenarios that compose the experiment.
- virtual void [startSimulationOfScenario \(SimulationScenario *scenario\)](#) override
Starts simulation for a specific scenario.
- virtual void [startExperiment \(\)](#) override
Starts full experiment execution (all enabled scenarios).
- virtual void [stopExperiment \(\)](#) override
Requests interruption of the currently running experiment.
- virtual void [addTraceSimulationHandler](#) (traceSimulationProcessListener traceSimulationProcessListener) override
Registers a callback to receive simulation progress traces for the experiment.

15.255.1 Member Function Documentation

15.255.1.1 addTraceSimulationHandler()

```
void ExperimentManagerDefaultImpl1::addTraceSimulationHandler (
    traceSimulationProcessListener traceSimulationProcessListener ) [override], [virtual]
```

Registers a callback to receive simulation progress traces for the experiment.
Implements [ExperimentManager_if](#).

15.255.1.2 getScenarios()

```
List< SimulationScenario * > * ExperimentManagerDefaultImpl1::getScenarios ( ) const [override], [virtual]
```

Returns the list of scenarios that compose the experiment.
Implements [ExperimentManager_if](#).

15.255.1.3 startExperiment()

```
void ExperimentManagerDefaultImpl1::startExperiment ( ) [override], [virtual]
```

Starts full experiment execution (all enabled scenarios).
Implements [ExperimentManager_if](#).

15.255.1.4 startSimulationOfScenario()

```
void ExperimentManagerDefaultImpl1::startSimulationOfScenario (
    SimulationScenario * scenario ) [override], [virtual]
```

Starts simulation for a specific scenario.
Implements [ExperimentManager_if](#).

15.255.1.5 stopExperiment()

```
void ExperimentManagerDefaultImpl1::stopExperiment ( ) [override], [virtual]
```

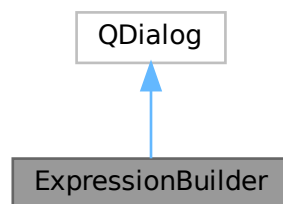
Requests interruption of the currently running experiment.
Implements [ExperimentManager_if](#).

The documentation for this class was generated from the following files:

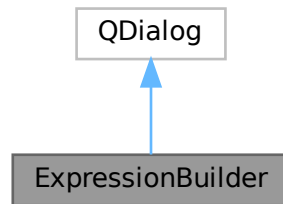
- kernel/simulator/ExperimentManagerDefaultImpl1.h
- kernel/simulator/ExperimentManagerDefaultImpl1.cpp

15.256 ExpressionBuilder Class Reference

Inheritance diagram for ExpressionBuilder:



Collaboration diagram for ExpressionBuilder:



Public Member Functions

- **ExpressionBuilder** (QWidget *parent=nullptr)

The documentation for this class was generated from the following files:

- applications/gui/genesys/tools/expressionbuilder/ExpressionBuilder.h
- applications/gui/genesys/tools/expressionbuilder/ExpressionBuilder.cpp

15.257 ExpressionCategory Struct Reference

Public Attributes

- QString **name**
- QList< [ExpressionTemplate](#) > **expressions**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/tools/expressionbuilder/ExpressionBuilder.h

15.258 ExpressionParameter Struct Reference

Public Attributes

- QString **name**
- QString **defaultValue**
- QString **description**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/tools/expressionbuilder/ExpressionBuilder.h

15.259 ExpressionTemplate Struct Reference

Public Attributes

- QString **name**
- QString **pattern**
- QString **description**
- QList< [ExpressionParameter](#) > **parameters**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/tools/expressionbuilder/ExpressionBuilder.h

15.260 FactorialDesign Class Reference

Public Member Functions

- **FactorialDesign** (int k, int p, int r)
- bool [checkFeasibility](#) ()
Checks the feasibility of the factorial design.
- void [generateIndexCombinations](#) ()
Generates combinations of indexes for the Factorial Design of Experiments.
- void [createColumnLabels](#) ()
Adds column labels for the sign table.
- void [generateInputs](#) ()
Generates input vectors of experiments to populate the sign table.
- void [calculateResultsMean](#) ()
Calculates the mean of results for each set of experiments from all replications.
- void [createSignTable](#) ()
Creates the sign table for a Factorial Design of Experiments.
- void [calculateStatistics](#) ()
Calculates statistical measures.
- double [linearRegression](#) (const std::vector< int > &xs)
Performs linear regression based on a set of input factors.
- void [showSignTable](#) ()
Displays the sign table.
- void [showImpacts](#) ()
Displays the impacts of factors, interaction of factors and experimental error.
- void [showLinearRegressionModel](#) ()
Displays the linear regression model.
- void [showResiduals](#) ()
Displays the residuals for the experimental results.
- const std::vector< std::vector< int > > & **getIndexCombinations** () const
- const std::vector< std::string > & **getColumnLabels** () const
- const std::vector< std::vector< int > > & **getInputs** () const
- const std::vector< double > & **getResultsMean** () const
- const std::vector< std::tuple< int, std::vector< int >, std::vector< int >, double > > & **getSignTable** () const
- const std::vector< double > & **getEffectSums** () const
- const std::vector< double > & **getEffectAverages** () const
- const std::vector< int > & **getSquareSums** () const
- double **getSquareSumError** () const
- double **getSquareSumTotal** () const
- const std::vector< double > & **getFactorVariations** () const
- double **getErrorVariation** () const
- void **setResultsMatrix** (const std::vector< std::vector< int > > &matrix)

Public Attributes

- int **k**
- int **p**
- int **r**
- int **NUM_FACTORS**
- int **NUM_ROWS**
- std::vector< std::vector< int > > **resultsMatrix**

15.260.1 Member Function Documentation

15.260.1.1 calculateResultsMean()

```
void FactorialDesign::calculateResultsMean ( )
```

Calculates the mean of results for each set of experiments from all replications.

This method computes the mean of results for each set of experiments from all replications, which are stored in the 'resultsMatrix'.

The resulting mean values are stored in the 'resultsMean' member variable.

15.260.1.2 calculateStatistics()

```
void FactorialDesign::calculateStatistics ( )
```

Calculates statistical measures.

This method computes various statistical measures, including effect sums, effect averages, square sums, square sum of errors, square sum total, factor variations, and error variation. These measures are essential for assessing the significance of experimental factors and understanding the variability in the experimental results. The calculated statistics are stored in corresponding member variables for further analysis.

15.260.1.3 checkFeasibility()

```
bool FactorialDesign::checkFeasibility ( )
```

Checks the feasibility of the factorial design.

This method evaluates the feasibility of a factorial design based on the number of interactions.

Returns

true if the factorial design is feasible, false otherwise.

15.260.1.4 createColumnLabels()

```
void FactorialDesign::createColumnLabels ( )
```

Adds column labels for the sign table.

This method creates labels for experimental factors (F1, F2, ..., FN), interaction terms, and results (R1, R2, ..., Rr, R~) based on the combinations of factor levels generated earlier. Additionally, it adds aliases for confounded factors and ensures appropriate labeling for each column in the table.

The resulting column labels are stored in the 'columnLabels' member variable.

15.260.1.5 createSignTable()

```
void FactorialDesign::createSignTable ( )
```

Creates the sign table for a Factorial Design of Experiments.

This method creates a sign table containing inputs and results of all experiments. It calculates the input for each interaction as the product of the inputs of its factors.

The resulting signTable is a vector of tuples, each representing a row in the sign table. Each tuple contains information about an experiment, facilitating further analysis and interpretation of the experimental results.

15.260.1.6 generateIndexCombinations()

```
void FactorialDesign::generateIndexCombinations ( )
```

Generates combinations of indexes for the Factorial Design of Experiments.

This method generates combinations of indexes with lengths ranging from 1 to NUM_FACTORS. The generated combinations are precomputed and utilized to access various data structures. These structures are essential for constructing the sign table and performing statistical calculations.

15.260.1.7 generateInputs()

```
void FactorialDesign::generateInputs ( )
```

Generates input vectors of experiments to populate the sign table.

This method creates input vectors representing the experimental conditions in a Factorial Design. Each input vector is a combination of factor levels, where the presence of a factor at a high level is represented by '1', whereas a low level is represented by '-1'. The generated input vectors cover all possible combinations of factor levels based on a binary counting pattern.

The resulting input vectors are stored in the 'inputs' member variable.

15.260.1.8 linearRegression()

```
double FactorialDesign::linearRegression (
    const std::vector< int > & xs )
```

Performs linear regression based on a set of input factors.

This method calculates the predicted result (y) using a linear regression model. The model is constructed using the effect averages for each factor in the Factorial Design, and the input factors provided in the 'xs' vector. The predicted result is the sum of the intercept (effect average for the first factor) and the product of each input factor with its corresponding effect average.

Parameters

xs	A vector representing the levels of input factors for which the result is predicted.
----	--

Returns

The predicted result (y) based on the linear regression model.

15.260.1.9 showImpacts()

```
void FactorialDesign::showImpacts ( )
```

Displays the impacts of factors, interaction of factors and experimental error.

This method prints the impacts of individual factors, interaction of factors and experimental error as calculated from the statistical measures. The impacts are represented as percentages of total variability, and provide insights into the significance of each factor and the contribution of experimental error to the overall variability.

15.260.1.10 showLinearRegressionModel()

```
void FactorialDesign::showLinearRegressionModel ( )
```

Displays the linear regression model.

This method prints the linear regression model for predicting the result variable based on the inputs of factors and interactions. The model is presented in the form of a mathematical expression, where coefficients are the intercept and the averages of effects for each factor, including interactions.

15.260.1.11 showResiduals()

```
void FactorialDesign::showResiduals ( )
```

Displays the residuals for the experimental results.

This method calculates and prints the residuals, which represent the differences between the observed experimental results and the values predicted by the linear regression model.

15.260.1.12 showSignTable()

```
void FactorialDesign::showSignTable ( )
```

Displays the sign table.

This method prints the sign table, organized in a tabular format, to the console. The table includes column labels, experiment numbers, input combinations and results. The output is formatted to align columns, providing a clear and readable representation of the experimental conditions and their corresponding information.

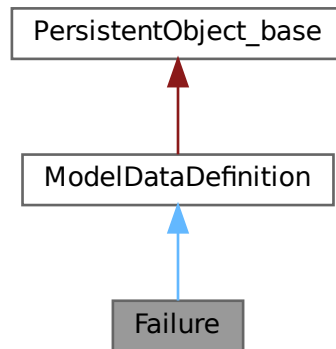
The documentation for this class was generated from the following files:

- tools/FactorialDesign/FactorialDesign.h
- tools/FactorialDesign/FactorialDesign.cpp

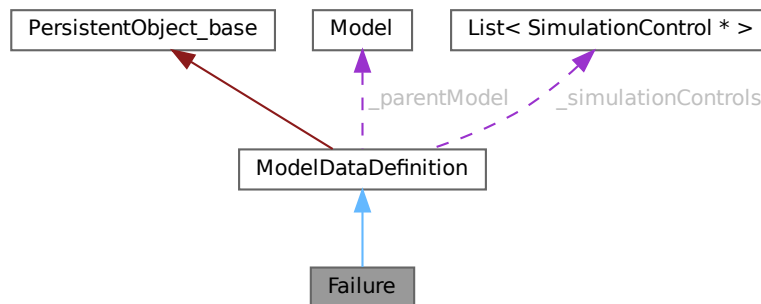
15.261 Failure Class Reference

```
#include <plugins/data/DiscreteProcessing/Failure.h>
```

Inheritance diagram for Failure:



Collaboration diagram for Failure:



Public Types

- enum class **FailureType** : int { **COUNT** = 0 , **TIME** = 1 , **num_elements** = 2 }
- enum class **FailureRule** : int { **IGNORE** = 0 , **PREEMPT** = 1 , **WAIT** = 2 , **num_elements** = 3 }

Public Member Functions

- **Failure** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- void **checkIsGoingToFailByCount** ([Resource](#) *resource)
- void **setFailureType** (FailureType _failureType)
- Failure::FailureType **getFailureType** () const
- void **setCountExpression** (std::string countExpression)
- std::string **getCountExpression** () const

- void **setDownTimeTimeUnit** ([Util::TimeUnit](#) downTimeTimeUnit)
- [Util::TimeUnit](#) **getDownTimeTimeUnit** () const
- void **setDownTimeExpression** (std::string downTimeExpression)
- std::string **getDownTimeExpression** () const
- void **setUpTimeTimeUnit** ([Util::TimeUnit](#) upTimeTimeUnit)
- [Util::TimeUnit](#) **getUpTimeTimeUnit** () const
- void **setUpTimeExpression** (std::string upTimeExpression)
- std::string **getUpTimeExpression** () const
- void **setFailureRule** (FailureRule _failureRule)
- FailureRule **getFailureRule** () const
- [List](#)< [Resource](#) * > * **falingResources** () const
- void **addResource** ([Resource](#) *newResource)
- void **removeResource** ([Resource](#) *resource)

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- [Model](#) * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

- Sets the model nesting level for this definition.*
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static std::string convertEnumToStr (FailureType type)`
- `static std::string convertEnumToStr (FailureRule rule)`
- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static PluginInformation * GetPluginInformation ()`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from ModelDataDefinition

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- `static void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.
- `static bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Validates a data definition instance.
- `static void InitBetweenReplications (ModelDataDefinition *modeldatum)`
Resets simulation-time state between replications.
- `static void CreateInternalData (ModelDataDefinition *modeldatum)`
Creates internal and attached data required by this instance.
- `static bool CreateRelatedDataElements (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Creates related data and validates the instance in a single call.

Protected Member Functions

- `virtual bool _loadInstance (PersistenceRecord *fields) override`
must be overridden by derived classes
- `virtual void _saveInstance (PersistenceRecord *fields, bool saveDefaultValues) override`
Saves object fields into a serialization structure.
- `virtual bool _check (std::string &errorMessage) override`
could be overridden by derived classes
- `virtual void _initBetweenReplications () override`

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
 - legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.261.1 Detailed Description

[Failure](#) module DESCRIPTION The [Failure](#) module is designed for use with resources. When a failure occurs, the entire resource (regardless of its capacity) is failed. Failures are designed to be used with single-capacity resources or with multiple-capacity resources whose individual resource units all fail at the same time. TYPICAL USES Break-down information for a machine Cash register tape refill every “x” customers Random computer shutdowns or restarts PROMPTS Recordset Name of the recordset in the specified file from which to read values. This field is available only if you specify a [File](#) Name with a file access type, path, and recordset. Arena uses the Rows and Columns properties to determine the amount of data to read from the recordset. A recordset is required for all file types except .xml. The recordset size must be equal to or greater than the number of rows and columns specified for the expression. Expression Values Lists the value or values of the expression. This property is not available if you specify a [File](#) Name from which to read expression values. Expression Value Expression value associated with the expression name. Prompt Description Name The name of the failure associated with one or more resources. Type Determines if the failure is time-based or count-based. Count Defines the number of resource releases for count-based failures. Valid when the Type is Count. Up Time Defines the time between failures for time-based failures. Valid when the Type is Time. Up Time Units Time units for the time between failures (Up Time) for timebased failures. Down Time Defines the duration of the failure. Down Time Units Time units for the duration of the failure (Down Time). Uptime in this [State](#) only Defines the state that should be considered for the time between failures (only for time-based failures). If state is not specified, then all states are considered (that is, the time between failures does not depend on the time spent in a specific state, but rather on the total simulation time). For example, you might want to define a failure to be based only on the state Busy, and therefore, the time between downtimes would be based on the amount of time that a resource is busy, not simulated clock time.

15.261.2 Member Function Documentation

15.261.2.1 [_check\(\)](#)

```
bool Failure::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Validate failure expressions according to selected failure mode.

Reimplemented from [ModelDataDefinition](#).

15.261.2.2 [_initBetweenReplications\(\)](#)

```
void Failure::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.261.2.3 [_loadInstance\(\)](#)

```
bool Failure::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Load [Failure](#) persistence fields.

This method mirrors [_saveInstance](#) field names to preserve a deterministic serialization contract.

Reimplemented from [ModelDataDefinition](#).

15.261.2.4 `_saveInstance()`

```
void Failure::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Persist [Failure](#) configuration fields.

Reimplemented from [ModelDataDefinition](#).

15.261.2.5 `show()`

```
std::string Failure::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

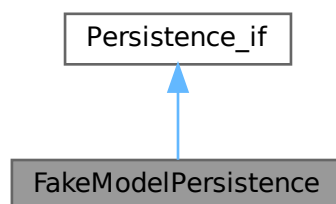
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

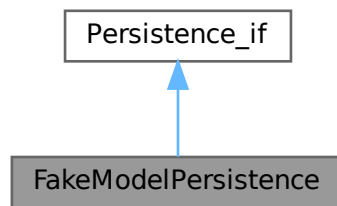
- `plugins/data/DiscreteProcessing/Failure.h`
- `plugins/data/DiscreteProcessing/Failure.cpp`

15.262 FakeModelPersistence Class Reference

Inheritance diagram for FakeModelPersistence:



Collaboration diagram for FakeModelPersistence:



Public Member Functions

- bool `save` (std::string) override
Serializes and saves the model to the provided filename.
- bool `load` (std::string) override
Loads and deserializes a model from file.
- bool `hasChanged` () override
Indicates whether persistent state changed since the last save/load.
- void `setHasChanged` (bool) override
Updates the persistent-state changed flag.
- bool `getOption` (Persistence_if::Options) override
Returns whether a persistence option is currently enabled.
- void `setOption` (Persistence_if::Options, bool) override
Enables or disables a persistence option.
- std::string `getFormattedField` (PersistenceRecord *) override
Formats a set of fields as text according to the persistence format.

Additional Inherited Members

Public Types inherited from Persistence_if

- enum class **Options** : int {
 SAVEDEFAULTS = 1 , **HIDEIDKEY** = 2 , **HIDETYPEKEY** = 4 , **HIDENAMEKEY** = 8 ,
 SORTALPHLY = 16 }

15.262.1 Member Function Documentation

15.262.1.1 getFormattedField()

```
std::string FakeModelPersistence::getFormattedField (
    PersistenceRecord * fields ) [inline], [override], [virtual]
```

Formats a set of fields as text according to the persistence format.

Implements [Persistence_if](#).

15.262.1.2 getOption()

```
bool FakeModelPersistence::getOption (
    Persistence_if::Options option ) [inline], [override], [virtual]
```

Returns whether a persistence option is currently enabled.

Implements [Persistence_if](#).

15.262.1.3 hasChanged()

```
bool FakeModelPersistence::hasChanged ( ) [inline], [override], [virtual]
```

Indicates whether persistent state changed since the last save/load.

Implements [Persistence_if](#).

15.262.1.4 load()

```
bool FakeModelPersistence::load (
    std::string filename ) [inline], [override], [virtual]
```

Loads and deserializes a model from file.

Implements [Persistence_if](#).

15.262.1.5 save()

```
bool FakeModelPersistence::save (
    std::string filename ) [inline], [override], [virtual]
```

Serializes and saves the model to the provided filename.

Implements [Persistence_if](#).

15.262.1.6 setHasChanged()

```
void FakeModelPersistence::setHasChanged (
    bool hasChanged ) [inline], [override], [virtual]
```

Updates the persistent-state changed flag.

Implements [Persistence_if](#).

15.262.1.7 setOption()

```
void FakeModelPersistence::setOption (
    Persistence_if::Options option,
    bool value ) [inline], [override], [virtual]
```

Enables or disables a persistence option.

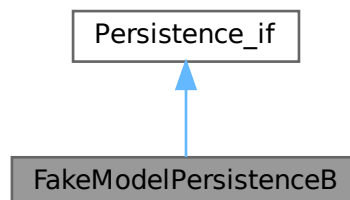
Implements [Persistence_if](#).

The documentation for this class was generated from the following file:

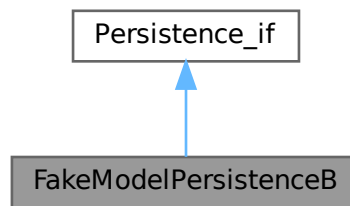
- tests/unit/test_simulator_support.cpp

15.263 FakeModelPersistenceB Class Reference

Inheritance diagram for FakeModelPersistenceB:



Collaboration diagram for FakeModelPersistenceB:



Public Member Functions

- bool `save` (std::string) override
Serializes and saves the model to the provided filename.
- bool `load` (std::string) override
Loads and deserializes a model from file.
- bool `hasChanged` () override
Indicates whether persistent state changed since the last save/load.
- void `setHasChanged` (bool) override
Updates the persistent-state changed flag.
- bool `getOption` (Persistence_if::Options) override
Returns whether a persistence option is currently enabled.
- void `setOption` (Persistence_if::Options, bool) override
Enables or disables a persistence option.
- std::string `getFormattedField` (PersistenceRecord *) override
Formats a set of fields as text according to the persistence format.

Additional Inherited Members

Public Types inherited from Persistence_if

- enum class **Options** : int {
 SAVEDEFAULTS = 1 , **HIDEIDKEY** = 2 , **HIDETYPEKEY** = 4 , **HIDENAMEKEY** = 8 ,
 SORTALPHLY = 16 }

15.263.1 Member Function Documentation

15.263.1.1 getFormattedField()

```
std::string FakeModelPersistenceB::getFormattedField (
    PersistenceRecord * fields ) [inline], [override], [virtual]
```

Formats a set of fields as text according to the persistence format.

Implements [Persistence_if](#).

15.263.1.2 getOption()

```
bool FakeModelPersistenceB::getOption (
    Persistence_if::Options option ) [inline], [override], [virtual]
```

Returns whether a persistence option is currently enabled.

Implements [Persistence_if](#).

15.263.1.3 hasChanged()

```
bool FakeModelPersistenceB::hasChanged ( ) [inline], [override], [virtual]
```

Indicates whether persistent state changed since the last save/load.

Implements [Persistence_if](#).

15.263.1.4 load()

```
bool FakeModelPersistenceB::load (
    std::string filename ) [inline], [override], [virtual]
```

Loads and deserializes a model from file.

Implements [Persistence_if](#).

15.263.1.5 save()

```
bool FakeModelPersistenceB::save (
    std::string filename ) [inline], [override], [virtual]
```

Serializes and saves the model to the provided filename.

Implements [Persistence_if](#).

15.263.1.6 setHasChanged()

```
void FakeModelPersistenceB::setHasChanged (
    bool hasChanged ) [inline], [override], [virtual]
```

Updates the persistent-state changed flag.

Implements [Persistence_if](#).

15.263.1.7 setOption()

```
void FakeModelPersistenceB::setOption (
    Persistence_if::Options option,
    bool value ) [inline], [override], [virtual]
```

Enables or disables a persistence option.

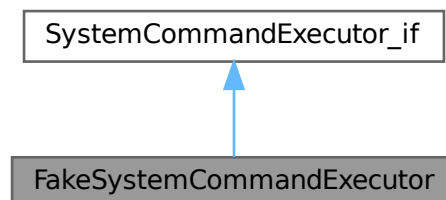
Implements [Persistence_if](#).

The documentation for this class was generated from the following file:

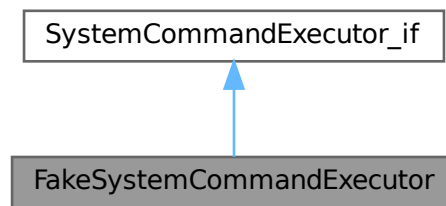
- tests/unit/test_support_persistence.cpp

15.264 FakeSystemCommandExecutor Class Reference

Inheritance diagram for FakeSystemCommandExecutor:



Collaboration diagram for FakeSystemCommandExecutor:



Public Member Functions

- [SystemCommandResult](#) `run` (const std::string &command) override
Runs a shell command and returns its observable result.

Public Member Functions inherited from [SystemCommandExecutor_if](#)

- virtual `~SystemCommandExecutor_if` ()=default
Destroys the command executor abstraction.

Public Attributes

- std::map< std::string, [SystemCommandResult](#) > `results`
- std::vector< std::string > `commands`

15.264.1 Member Function Documentation

15.264.1.1 `run()`

```
SystemCommandResult FakeSystemCommandExecutor::run (
    const std::string & command ) [inline], [override], [virtual]
```

Runs a shell command and returns its observable result.

Implements [SystemCommandExecutor_if](#).

The documentation for this class was generated from the following file:

- tests/unit/test_simulator_support.cpp

15.265 testing::internal::FallbackPrinter Struct Reference

Static Public Member Functions

- template<typename T >
static void **PrintValue** (const T &, ::std::ostream *os)

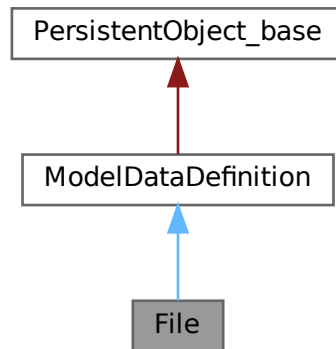
The documentation for this struct was generated from the following file:

- gtest/gtest/gtest-printers.h

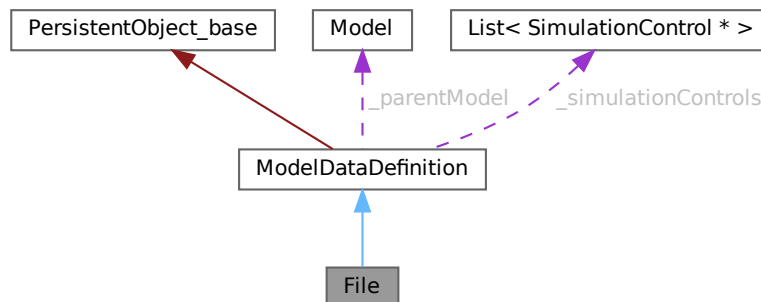
15.266 File Class Reference

```
#include <plugins/data/InputOutput/File.h>
```

Inheritance diagram for File:



Collaboration diagram for File:



Public Types

- enum class **AccessMode** : unsigned int { **Read** = 0 , **Write** = 1 , **Append** = 2 , **ReadWrite** = 3 }

Public Member Functions

- **File** ([Model](#) *model, std::string name="")
- void **setSystemFilename** (std::string systemFilename)
- std::string **getSystemFilename** () const
- std::string **getFilenameOnly** () const
- std::string **getPathOnly** () const
- void **setAccessMode** (AccessMode accessMode)
- AccessMode **getAccessMode** () const
- void **setAccessModeAsString** (std::string accessMode)
- std::string **getAccessModeAsString** () const

- virtual std::string **show** () override
Returns a textual representation of this data definition.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const
Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)
Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)
Defines the specific trace level and optionally enables it immediately.

- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internaStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*

- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.266.1 Detailed Description

File module DESCRIPTION Use the **File** module to access external files for the ReadWrite module, **Variable** module, and Expression module. The **File** module identifies the system file name and defines the access type and operational characteristics of the file. TYPICAL USES **File** containing predefined airline flight data **File** specifying customer order times and relevant information **File** to write user model configuration data from menu input PROMPTS Prompt Description Name The name of the file whose characteristics are being defined. This name must be unique. **Access** Type The file type. Operating System **File** Name Name of the actual file that is being read from or to which it is being written. Connecting String **Connection** string used to open ADO connection to the data source. Structure **File** structure, which can be unformatted, free format, or a specific C or FORTRAN format. End of **File** Action Type of action to occur if an end of file condition is reached. Initialize Option Action to be taken on file at beginning of each simulation replication. Comment Character Character indicating comment record. Recordset Name Name used to identify the recordset in the Expression, ReadWrite, and **Variable** modules. This name must be unique within the file. This field is available for Microsoft Excel, Microsoft Excel 2007, Microsoft **Access**, Microsoft **Access** 2007, and ActiveX Data Objects files. CommandText Text of the command that will be used to open the recordset (for example, SQL statement, procedure name, table name.) This field is available for ActiveX Data Object files only. CommandType Type of command entered in the CommandText. Named Range The named range in the Excel workbook to which the recordset refers. Table Name The name of the table in the **Access** database to which the recordset refers.

15.266.2 Member Function Documentation

15.266.2.1 _check()

```
bool File::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.266.2.2 _getParserChangesInformation()

```
ParserChangesInformation * File::_getParserChangesInformation ( ) [override], [protected],
[virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDatas. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dynamically linked to to simulator kernel to reflect the changes Return parser deltas required by [File](#).

[File](#) currently does not require parser extensions.

Reimplemented from [ModelDataDefinition](#).

15.266.2.3 _loadInstance()

```
bool File::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.266.2.4 _saveInstance()

```
void File::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.266.2.5 show()

```
std::string File::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- plugins/data/InOutOutput/File.h
- plugins/data/InOutOutput/File.cpp

15.267 testing::internal::FindFirstPrinter< T, E, Printer, Printers > Struct Template Reference

The documentation for this struct was generated from the following file:

- gtest/gtest/gtest-printers.h

15.268 testing::internal::FindFirstPrinter< T, decltype(Printer::PrintValue(std::declval< const T & >(), nullptr)), Printer, Printers... > Struct Template Reference

Public Types

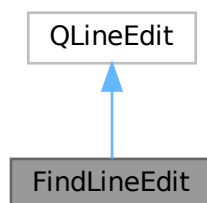
- using **type** = Printer

The documentation for this struct was generated from the following file:

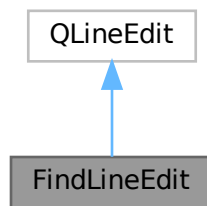
- gtest/gtest/gtest-printers.h

15.269 FindLineEdit Class Reference

Inheritance diagram for FindLineEdit:



Collaboration diagram for FindLineEdit:



Public Member Functions

- **FindLineEdit** (QWidget *parent=nullptr)

The documentation for this class was generated from the following file:

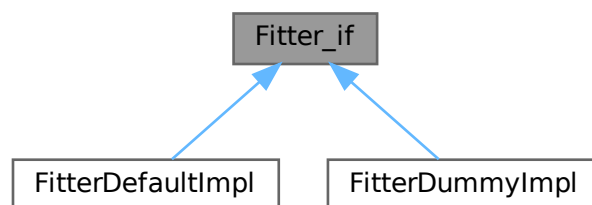
- applications/gui/genesys/dialogs/DialogFind.h

15.270 Fitter_if Class Reference

Interface for fitting theoretical distributions to real datasets.

```
#include <tools/Statistics/Fitter_if.h>
```

Inheritance diagram for Fitter_if:



Public Member Functions

- virtual bool **isNormalDistributed** (double confidencelevel)=0
- virtual void **fitUniform** (double *sqerror, double *min, double *max)=0
- virtual void **fitTriangular** (double *sqerror, double *min, double *mo, double *max)=0
- virtual void **fitNormal** (double *sqerror, double *avg, double *stddev)=0
- virtual void **fitExpo** (double *sqerror, double *avg1)=0
- virtual void **fitErlang** (double *sqerror, double *avg, double *m)=0
- virtual void **fitBeta** (double *sqerror, double *alpha, double *beta, double *infLimit, double *supLimit)=0
- virtual void **fitWeibull** (double *sqerror, double *alpha, double *scale)=0
- virtual void **fitAll** (double *sqerror, std::string *name)=0
- virtual void **setDataFilename** (std::string dataFilename)=0

Defines the dataset source filename used by fitting operations.

- virtual std::string [getDataFilename](#) ()=0

Retrieves the currently configured dataset source filename.

15.270.1 Detailed Description

Interface for fitting theoretical distributions to real datasets.

Purpose:

- Infer parameters of candidate probability distributions from sample data.

Architectural role:

- Core fitting abstraction consumed by analysis workflows.

Mathematical meaning:

- Each `fit*` method estimates family parameters and writes an adherence/error measure in `sqrerror` (typically squared-error-like criterion).
- `fitAll()` selects the best candidate according to implementation-defined criterion.
- `isNormalDistributed()` should be interpreted as a normality adherence check under the given confidence level.

Preconditions:

- Output pointers must be valid non-null addresses.
- Data source must be configured through [setDataFilename\(\)](#) before fitting.

Postconditions:

- Output pointers are filled with estimated values when operation succeeds.

Failure modes:

- Concrete implementations may return conservative defaults and/or report failure when insufficient data or unsupported models are provided.

Legacy compatibility:

- The pointer-based output API is intentionally preserved as a legacy public contract; it is not necessarily the final long-term shape.

15.270.2 Member Function Documentation

15.270.2.1 [getDataFilename\(\)](#)

```
virtual std::string Fitter_if::getDataFilename ( ) [pure virtual]
```

Retrieves the currently configured dataset source filename.

Implemented in [FitterDummyImpl](#), and [FitterDefaultImpl](#).

15.270.2.2 [setDataFilename\(\)](#)

```
virtual void Fitter_if::setDataFilename (
    std::string dataFilename ) [pure virtual]
```

Defines the dataset source filename used by fitting operations.

Implemented in [FitterDummyImpl](#), and [FitterDefaultImpl](#).

The documentation for this class was generated from the following file:

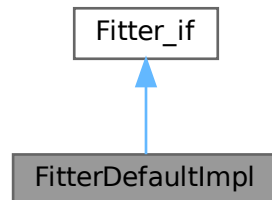
- tools/Statistics/Fitter_if.h

15.271 FitterDefaultImpl Class Reference

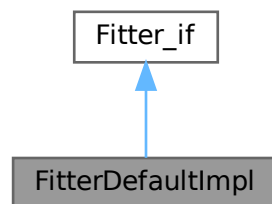
Functional implementation of [Fitter_if](#) consolidated through FITTER-2.

```
#include <tools/Statistics/FitterDefaultImpl.h>
```

Inheritance diagram for FitterDefaultImpl:



Collaboration diagram for FitterDefaultImpl:



Public Member Functions

- virtual bool [isNormalDistributed](#) (double confidencelevel) override
- virtual void [fitUniform](#) (double *sqerror, double *min, double *max) override
- virtual void [fitTriangular](#) (double *sqerror, double *min, double *mo, double *max) override
- virtual void [fitNormal](#) (double *sqerror, double *avg, double *stddev) override
- virtual void [fitExpo](#) (double *sqerror, double *avg1) override
- virtual void [fitErlang](#) (double *sqerror, double *avg, double *m) override
- virtual void [fitBeta](#) (double *sqerror, double *alpha, double *beta, double *infLimit, double *supLimit) override
- virtual void [fitWeibull](#) (double *sqerror, double *alpha, double *scale) override
- virtual void [fitAll](#) (double *sqerror, std::string *name) override
- virtual void [setDataFilename](#) (std::string dataFilename) override
- Defines the dataset source filename used by fitting operations.
- virtual std::string [getDataFilename](#) () override
- Retrieves the currently configured dataset source filename.

15.271.1 Detailed Description

Functional implementation of [Fitter_if](#) consolidated through FITTER-2.

Current scope:

- Supports real dataset loading from binary file with raw sequential doubles (compatible with [CollectorDatafileDefaultImpl1](#) persistence format).
- Implements fitting for Uniform, Triangular, Normal, Exponential, Erlang, Beta (scaled) and Weibull.

Error metric:

- Uses SSE between empirical CDF points $p_i = (i+0.5)/n$ and fitted model CDF.
- The same criterion is used consistently by `fit*` methods and `fitAll()`.

[Failure](#) convention:

- `sqrerror` = +infinity.
- returned parameters = `quiet_NaN()`.
- `fitAll` name = "no-valid-fit" or "invalid-dataset".
- `isNormalDistributed()` = false on invalid/insufficient data.

15.271.2 Member Function Documentation

15.271.2.1 `fitAll()`

```
virtual void FitterDefaultImpl::fitAll (
    double * sqrerror,
    std::string * name ) [inline], [override], [virtual]
```

Implements [Fitter_if](#).

15.271.2.2 `fitBeta()`

```
virtual void FitterDefaultImpl::fitBeta (
    double * sqrerror,
    double * alpha,
    double * beta,
    double * infLimit,
    double * supLimit ) [inline], [override], [virtual]
```

Implements [Fitter_if](#).

15.271.2.3 `fitErlang()`

```
virtual void FitterDefaultImpl::fitErlang (
    double * sqrerror,
    double * avg,
    double * m ) [inline], [override], [virtual]
```

Implements [Fitter_if](#).

15.271.2.4 `fitExpo()`

```
virtual void FitterDefaultImpl::fitExpo (
    double * sqrerror,
    double * avg1 ) [inline], [override], [virtual]
```

Implements [Fitter_if](#).

15.271.2.5 fitNormal()

```
virtual void FitterDefaultImpl::fitNormal (
    double * sqrerror,
    double * avg,
    double * stddev ) [inline], [override], [virtual]
```

Implements [Fitter_if](#).

15.271.2.6 fitTriangular()

```
virtual void FitterDefaultImpl::fitTriangular (
    double * sqrerror,
    double * min,
    double * mo,
    double * max ) [inline], [override], [virtual]
```

Implements [Fitter_if](#).

15.271.2.7 fitUniform()

```
virtual void FitterDefaultImpl::fitUniform (
    double * sqrerror,
    double * min,
    double * max ) [inline], [override], [virtual]
```

Implements [Fitter_if](#).

15.271.2.8 fitWeibull()

```
virtual void FitterDefaultImpl::fitWeibull (
    double * sqrerror,
    double * alpha,
    double * scale ) [inline], [override], [virtual]
```

Implements [Fitter_if](#).

15.271.2.9 getDataFilename()

```
virtual std::string FitterDefaultImpl::getDataFilename ( ) [inline], [override], [virtual]
```

Retrieves the currently configured dataset source filename.

Implements [Fitter_if](#).

15.271.2.10 isNormalDistributed()

```
virtual bool FitterDefaultImpl::isNormalDistributed (
    double confidencelevel ) [inline], [override], [virtual]
```

Implements [Fitter_if](#).

15.271.2.11 setDataFilename()

```
virtual void FitterDefaultImpl::setDataFilename (
    std::string dataFilename ) [inline], [override], [virtual]
```

Defines the dataset source filename used by fitting operations.

Implements [Fitter_if](#).

The documentation for this class was generated from the following file:

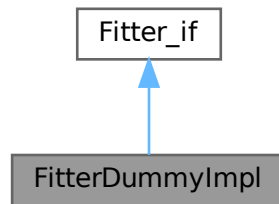
- tools/Statistics/FitterDefaultImpl.h

15.272 FitterDummyImpl Class Reference

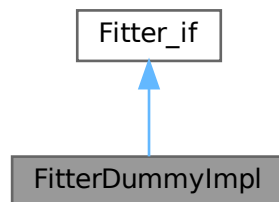
Placeholder implementation of [Fitter_if](#).

```
#include <tools/Statistics/FitterDummyImpl.h>
```

Inheritance diagram for FitterDummyImpl:



Collaboration diagram for FitterDummyImpl:



Public Member Functions

- **FitterDummyImpl** (const [FitterDummyImpl](#) &orig)
- bool [isNormalDistributed](#) (double confidencelevel)
- void [fitUniform](#) (double *sqerror, double *min, double *max)
- void [fitTriangular](#) (double *sqerror, double *min, double *mo, double *max)
- void [fitNormal](#) (double *sqerror, double *avg, double *stddev)
- void [fitExpo](#) (double *sqerror, double *avg1)
- void [fitErlang](#) (double *sqerror, double *avg, double *m)
- void [fitBeta](#) (double *sqerror, double *alpha, double *beta, double *infLimit, double *supLimit)
- void [fitWeibull](#) (double *sqerror, double *alpha, double *scale)
- void [fitAll](#) (double *sqerror, std::string *name)
- void [setDataFilename](#) (std::string dataFilename)
Defines the dataset source filename used by fitting operations.
- std::string [getDataFilename](#) ()
Retrieves the currently configured dataset source filename.

15.272.1 Detailed Description

Placeholder implementation of [Fitter_if](#).

Purpose:

- Keep binary/source compatibility while a functional fitter implementation is not fully available.

Implementation status:

- Dummy/stub component; not a functional fitting engine.

Planned evolution:

- To be replaced as default by [FitterDefaultImpl](#) when fitting logic is consolidated.

15.272.2 Member Function Documentation

15.272.2.1 `fitAll()`

```
void FitterDummyImpl::fitAll (
    double * sqrerror,
    std::string * name ) [virtual]
```

Implements [Fitter_if](#).

15.272.2.2 `fitBeta()`

```
void FitterDummyImpl::fitBeta (
    double * sqrerror,
    double * alpha,
    double * beta,
    double * inflimit,
    double * supLimit ) [virtual]
```

Implements [Fitter_if](#).

15.272.2.3 `fitErlang()`

```
void FitterDummyImpl::fitErlang (
    double * sqrerror,
    double * avg,
    double * m ) [virtual]
```

Implements [Fitter_if](#).

15.272.2.4 `fitExpo()`

```
void FitterDummyImpl::fitExpo (
    double * sqrerror,
    double * avg1 ) [virtual]
```

Implements [Fitter_if](#).

15.272.2.5 `fitNormal()`

```
void FitterDummyImpl::fitNormal (
    double * sqrerror,
    double * avg,
    double * stddev ) [virtual]
```

Implements [Fitter_if](#).

15.272.2.6 `fitTriangular()`

```
void FitterDummyImpl::fitTriangular (
    double * sqrerror,
    double * min,
    double * mo,
    double * max ) [virtual]
```

Implements [Fitter_if](#).

15.272.2.7 fitUniform()

```
void FitterDummyImpl::fitUniform (
    double * sqrerror,
    double * min,
    double * max ) [virtual]
```

Implements [Fitter_if](#).

15.272.2.8 fitWeibull()

```
void FitterDummyImpl::fitWeibull (
    double * sqrerror,
    double * alpha,
    double * scale ) [virtual]
```

Implements [Fitter_if](#).

15.272.2.9 getDataFilename()

```
std::string FitterDummyImpl::getDataFilename ( ) [virtual]
```

Retrieves the currently configured dataset source filename.

Implements [Fitter_if](#).

15.272.2.10 isNormalDistributed()

```
bool FitterDummyImpl::isNormalDistributed (
    double confidencelevel ) [virtual]
```

Implements [Fitter_if](#).

15.272.2.11 setDataFilename()

```
void FitterDummyImpl::setDataFilename (
    std::string dataFilename ) [virtual]
```

Defines the dataset source filename used by fitting operations.

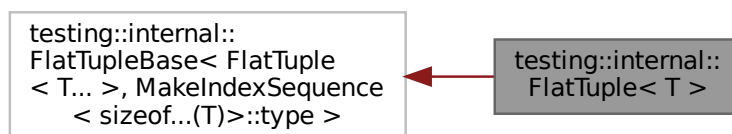
Implements [Fitter_if](#).

The documentation for this class was generated from the following files:

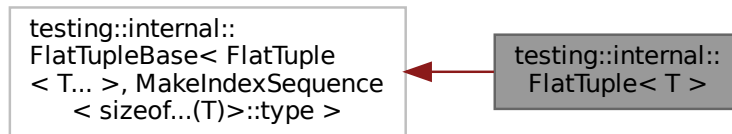
- tools/Statistics/FitterDummyImpl.h
- tools/Statistics/FitterDummyImpl.cpp

15.273 testing::internal::FlatTuple< T > Class Template Reference

Inheritance diagram for testing::internal::FlatTuple< T >:



Collaboration diagram for `testing::internal::FlatTuple< T >`:



Public Member Functions

- `template<typename... Args>`
FlatTuple ([FlatTupleConstructTag](#) tag, Args &&... args)

The documentation for this class was generated from the following file:

- `gtest/gtest/internal/gtest-internal.h`

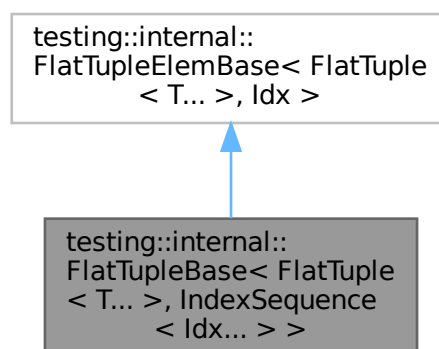
15.274 `testing::internal::FlatTupleBase< Derived, Idx >` Struct Template Reference

The documentation for this struct was generated from the following file:

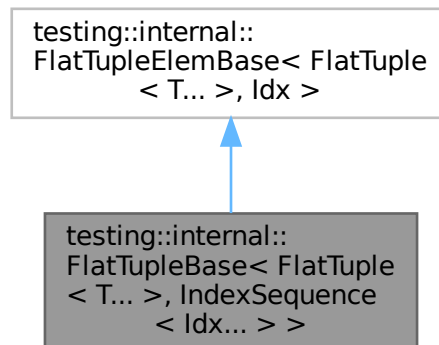
- `gtest/gtest/internal/gtest-internal.h`

15.275 `testing::internal::FlatTupleBase< FlatTuple< T... >, IndexSequence< Idx... > >` Struct Template Reference

Inheritance diagram for `testing::internal::FlatTupleBase< FlatTuple< T... >, IndexSequence< Idx... > >`:



Collaboration diagram for testing::internal::FlatTupleBase< FlatTuple< T... >, IndexSequence< Idx... > >:



Public Types

- using **Indices** = [IndexSequence](#)< Idx... >

Public Member Functions

- template<typename... Args>
FlatTupleBase ([FlatTupleConstructTag](#), Args &&... args)
- template<size_t I>
const [ElemFromList](#)< I, T... >::type & **Get** () const
- template<size_t I>
[ElemFromList](#)< I, T... >::type & **Get** ()
- template<typename F >
auto **Apply** (F &&f) -> decltype(std::forward< F >(f)(this->Get< Idx >()...))
- template<typename F >
auto **Apply** (F &&f) const -> decltype(std::forward< F >(f)(this->Get< Idx >()...))

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-internal.h

15.276 testing::internal::FlatTupleConstructTag Struct Reference

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-internal.h

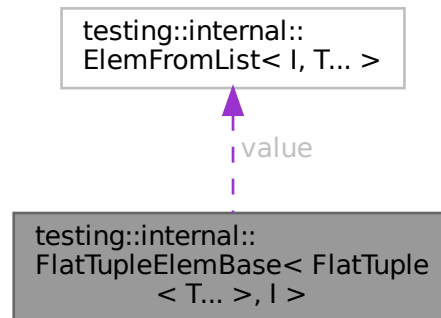
15.277 testing::internal::FlatTupleElemBase< Derived, I > Struct Template Reference

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-internal.h

15.278 testing::internal::FlatTupleElemBase< FlatTuple< T... >, I > Struct Template Reference

Collaboration diagram for testing::internal::FlatTupleElemBase< FlatTuple< T... >, I >:



Public Types

- using `value_type` = typename `ElemFromList< I, T... >::type`

Public Member Functions

- template<typename Arg >
`FlatTupleElemBase` (`FlatTupleConstructTag`, Arg &&t)

Public Attributes

- `value_type` `value`

The documentation for this struct was generated from the following file:

- `gtest/gtest/internal/gtest-internal.h`

15.279 testing::internal::FloatingPoint< RawType > Class Template Reference

Public Types

- typedef `TypeWithSize< sizeof(RawType)>::UInt` `Bits`

Public Member Functions

- `FloatingPoint` (const RawType &x)
- const Bits & `bits` () const
- Bits `exponent_bits` () const
- Bits `fraction_bits` () const
- Bits `sign_bit` () const
- bool `is_nan` () const
- bool `AlmostEquals` (const `FloatingPoint` &rhs) const
- float `Max` ()
- double `Max` ()

Static Public Member Functions

- static RawType **ReinterpretBits** (const Bits bits)
- static RawType **Infinity** ()
- static RawType **Max** ()

Static Public Attributes

- static const size_t **kBitCount** = 8 * sizeof(RawType)
- static const size_t **kFractionBitCount**
- static const size_t **kExponentBitCount** = kBitCount - 1 - kFractionBitCount
- static const Bits **kSignBitMask** = static_cast<Bits>(1) << (kBitCount - 1)
- static const Bits **kFractionBitMask**
- static const Bits **kExponentBitMask** = ~(kSignBitMask | kFractionBitMask)
- static const uint32_t **kMaxUlp** = 4

15.279.1 Member Data Documentation

15.279.1.1 kFractionBitCount

```
template<typename RawType >
const size_t testing::internal::FloatingPoint< RawType >::kFractionBitCount [static]
Initial value:
=
    std::numeric_limits<RawType>::digits - 1
```

15.279.1.2 kFractionBitMask

```
template<typename RawType >
const Bits testing::internal::FloatingPoint< RawType >::kFractionBitMask [static]
Initial value:
= ~static_cast<Bits>(0) >>
                                   (kExponentBitCount + 1)
```

The documentation for this class was generated from the following file:

- gtest/gtest/internal/gtest-internal.h

15.280 testing::internal::FormatForComparison< ToPrint, OtherOperand > Class Template Reference

Static Public Member Functions

- ::std::string **Format** (const ToPrint &value)

The documentation for this class was generated from the following file:

- gtest/gtest/gtest-printers.h

15.281 testing::internal::FormatForComparison< ToPrint[N], OtherOperand > Class Template Reference

Static Public Member Functions

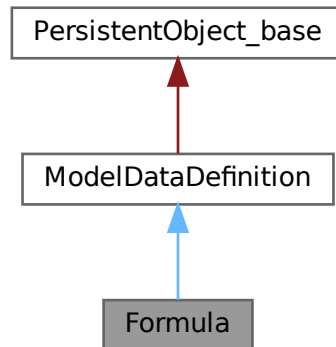
- ::std::string **Format** (const ToPrint *value)

The documentation for this class was generated from the following file:

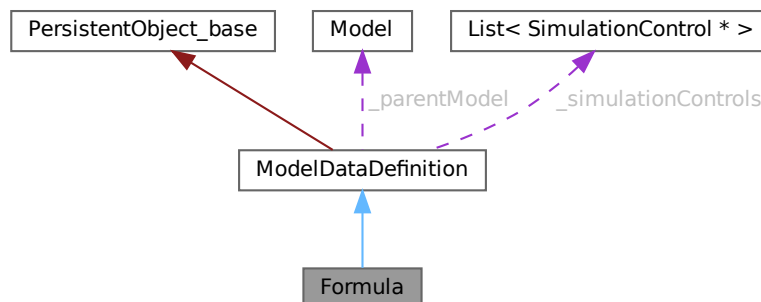
- gtest/gtest/gtest-printers.h

15.282 Formula Class Reference

Inheritance diagram for Formula:



Collaboration diagram for Formula:



Public Member Functions

- [Formula](#) ([Model](#) *model, std::string name="")
Creates an empty formula definition.
- virtual ~**Formula** () override
Releases formula-owned resources.
- virtual std::string [show](#) () override
Returns a textual representation of the formula.
- unsigned int [size](#) ()
Returns the number of indexed expressions stored in this formula.
- void [setExpression](#) (std::string formulaExpression, std::string index="")
Stores or updates the expression associated with an index.
- std::string [getExpression](#) (std::string index="")
Returns the expression associated with an index.
- double [getValue](#) (std::string index)
Evaluates the expression associated with an index.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool **hasChanged**)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- **List**< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
Returns plugin metadata for this formula type.
- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
Loads a formula from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new empty formula instance.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createAttachedAttributes](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)

- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.282.1 Constructor & Destructor Documentation

15.282.1.1 Formula()

```
Formula::Formula (
    Model * model,
    std::string name = "" )
```

Creates an empty formula definition.

Parameters

<i>model</i>	Parent model.
<i>name</i>	Initial formula name.

15.282.2 Member Function Documentation

15.282.2.1 _check()

```
bool Formula::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.282.2.2 _createAttachedAttributes()

```
void Formula::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDatas. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dynamically linked to simulator kernel to reflect the changes @ToDo: NOT AN ATTRIBUTE

Reimplemented from [ModelDataDefinition](#).

15.282.2.3 _loadInstance()

```
bool Formula::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Load formula expressions map.

Reimplemented from [ModelDataDefinition](#).

15.282.2.4 _saveInstance()

```
void Formula::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Persist formula expressions map.

Reimplemented from [ModelDataDefinition](#).

15.282.2.5 getExpression()

```
std::string Formula::getExpression (
    std::string index = "" )
```

Returns the expression associated with an index.

Parameters

<i>index</i>	Optional index key used for vectorized formulas.
--------------	--

Returns

Stored formula text, or an empty string when absent.

15.282.2.6 GetPluginInformation()

```
PluginInformation * Formula::GetPluginInformation ( ) [static]
```

Returns plugin metadata for this formula type.

Returns

Plugin information block.

15.282.2.7 getValue()

```
double Formula::getValue (
    std::string index )
```

Evaluates the expression associated with an index.

Parameters

<i>index</i>	Optional index key used for vectorized formulas.
--------------	--

Returns

Numeric value of the indexed expression.

15.282.2.8 LoadInstance()

```
ModelDataDefinition * Formula::LoadInstance (
    Model * model,
    PersistenceRecord * fields ) [static]
```

Loads a formula from serialized fields.

Parameters

<i>model</i>	Parent model.
<i>fields</i>	Serialized fields.

Returns

Newly created formula instance.

15.282.2.9 NewInstance()

```
ModelDataDefinition * Formula::NewInstance (
    Model * model,
    std::string name = "" ) [static]
```

Creates a new empty formula instance.

Parameters

<i>model</i>	Parent model.
--------------	---------------

Parameters

<i>name</i>	Initial formula name.
-------------	-----------------------

Returns

Newly created formula instance.

15.282.2.10 setExpression()

```
void Formula::setExpression (
    std::string formulaExpression,
    std::string index = "" )
```

Stores or updates the expression associated with an index.

Parameters

<i>formulaExpression</i>	Formula text.
<i>index</i>	Optional index key used for vectorized formulas.

15.282.2.11 show()

```
std::string Formula::show ( ) [override], [virtual]
```

Returns a textual representation of the formula.

Returns

Human-readable formula summary.

Reimplemented from [ModelDataDefinition](#).

15.282.2.12 size()

```
unsigned int Formula::size ( )
```

Returns the number of indexed expressions stored in this formula.

Returns

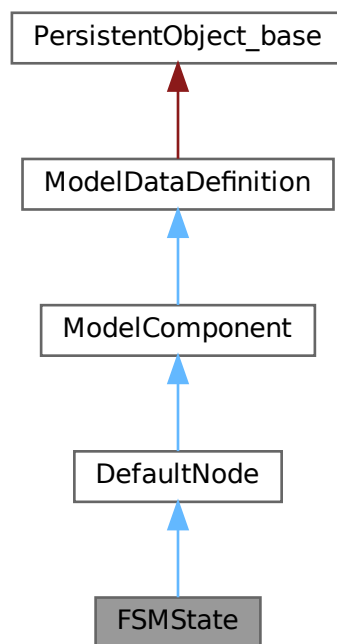
Number of expression slots.

The documentation for this class was generated from the following files:

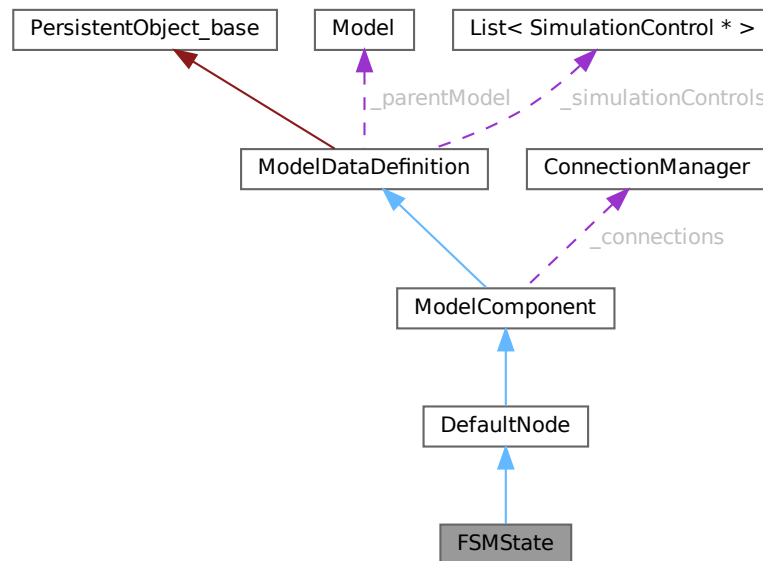
- plugins/data/Logic/Formula.h
- plugins/data/Logic/Formula.cpp

15.283 FSMState Class Reference

Inheritance diagram for FSMState:



Collaboration diagram for FSMState:



Public Member Functions

- **FSMState** ([Model](#) *model, std::string name="")
- void **setEntryActionExpression** (std::string expression)
- std::string **getEntryActionExpression** () const
- void **setExitActionExpression** (std::string expression)
- std::string **getExitActionExpression** () const

Public Member Functions inherited from [DefaultNode](#)

- [DefaultNode](#) ([Model](#) *model, std::string componentTypename, std::string name)
constructors
- **DefaultNode** ([Model](#) *model, std::string name="")
- void **addTransition** ([DefaultNodeTransition](#) *transition)
new public user methods for this component
- void **removeTransition** ([DefaultNodeTransition](#) *transition)
- [List](#)< [DefaultNodeTransition](#) * > * **getTransitions** () const
- void **setInitialNode** (bool initialNode)
- bool **isInitialNode** () const
- void **setFinalNode** (bool finalNode)
- bool **isFinalNode** () const
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual **~ModelComponent** ()
Releases component-owned resources and unregisters it from the model.

- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)

- Sets the model nesting level for this definition.*
 - `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
 - `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
 - `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
 - `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
 - `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
 - `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation * GetPluginInformation ()`
- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- static `ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from `DefaultNode`

- static `PluginInformation * GetPluginInformation ()`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- static `ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static `bool Check (ModelComponent *component)`
Validates a component instance.
- static `void CreateInternalData (ModelComponent *component)`
Creates internal data required by a component instance.
- static `void SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
Serializes a component instance into a persistence record.
- static `void DispatchEvent (Event *event)`
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- static `void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.
- static `bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Validates a data definition instance.

- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [DefaultNode](#)

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()

- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void **traceSimulation** (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager::Level](#) **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.283.1 Member Function Documentation

15.283.1.1 **_loadInstance()**

```
bool FSMState::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [DefaultNode](#).

15.283.1.2 **_saveInstance()**

```
void FSMState::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [DefaultNode](#).

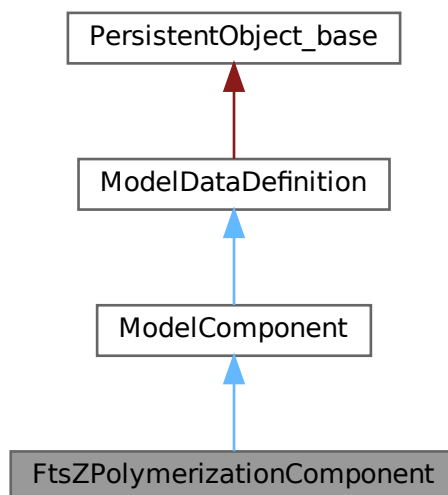
The documentation for this class was generated from the following files:

- plugins/components/ModalModel/FSMState.h
- plugins/components/ModalModel/FSMState.cpp

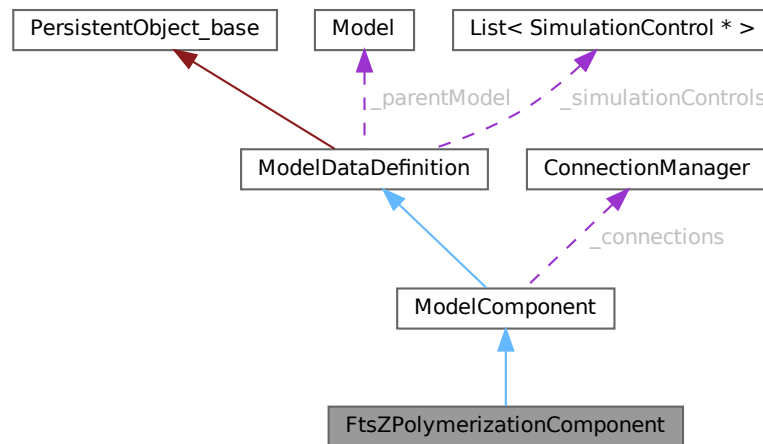
15.284 FtsZPolymerizationComponent Class Reference

```
#include <plugins/components/WholeCellModeling/FtsZPolymerizationComponent.h>
```

Inheritance diagram for FtsZPolymerizationComponent:



Collaboration diagram for FtsZPolymerizationComponent:



Public Member Functions

- **FtsZPolymerizationComponent** ([Model](#) *model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **setActivationFwd** (double rate)
- double **getActivationFwd** () const
- void **setActivationRev** (double rate)
- double **getActivationRev** () const
- void **setVolumeNorm** (double norm)
- double **getVolumeNorm** () const
- void **setDeltaT** (double dt)
- double **getDeltaT** () const
- void **setFtsZRingKey** (std::string key)
- std::string **getFtsZRingKey** () const
- void **setFtsZMonomerKey** (std::string key)
- std::string **getFtsZMonomerKey** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual **~ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)

Updates the free-form description shown by the GUI.

- `std::string getDescription () const`

Returns the free-form description associated with this component.

- `void remove (ModelComponent *comp)`

Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- `ModelDataDefinition (Model *model, std::string datadefinitionTypename, std::string name="", bool insertIntoModel=true)`

Creates a model data definition and optionally inserts it into the model.

- `virtual ~ModelDataDefinition ()`

Releases resources owned by this data definition.

- `Util::identification getId () const`

Returns the unique identifier of this instance.

- `void setName (const std::string &name)`

Updates the logical name of this instance.

- `const std::string & getName () const`

Returns the logical name of this instance.

- `void setLabel (const std::string &label)`

Updates the user-visible label of this instance.

- `const std::string & getLabel () const`

Returns the user-visible label of this instance.

- `const std::string & getClassname () const`

Returns the runtime class name used by the plugin system.

- `Model * getParentModel () const`

Returns the parent model that owns this definition.

- `bool isReportStatistics () const`

Indicates whether this data definition contributes report statistics.

- `void setReportStatistics (bool reportStatistics)`

Enables or disables report-statistics generation for this definition.

- `ModelDataDefinition * getInternalData (const std::string &name) const`

Looks up a named internal child data definition.

- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`

Returns the full map of internal child data definitions.

- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`

Returns the full map of attached data definitions.

- `bool hasChanged () const`

Indicates whether this instance has been modified since the last reset.

- `void setHasChanged (bool hasChanged)`

Updates the changed flag for this data definition.

- `unsigned int getLevel () const`

Returns the level in the model hierarchy where this definition lives.

- `void setModelLevel (unsigned int _modelLevel)`

Sets the model nesting level for this definition.

- `List< SimulationControl * > * getSimulationControls () const`

Returns the simulation controls owned by this data definition.

- `TraceManager::Level getTraceLevelSpecific () const`

Returns the trace level that applies specifically to this definition.

- `void setTraceLevelSpecific (TraceManager::Level level)`

Updates the trace level used by this definition.

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

- virtual void `_initBetweenReplications ()` override
- virtual void `_onDispatchEvent (Entity *entity, unsigned int inputPortNumber)` override
Handles a dispatched event for this component type.
- virtual void `_createEditableDataDefinitions ()` override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption ()`
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation ()`
- virtual void `_addSimulationControl (SimulationControl *control)`
- virtual void `_createInternalAndAttachedData ()`
- virtual void `_createInternalStatisticReporters ()`
- virtual void `_createAttachedAttributes ()`
- virtual void `_createNonEditableDataDefinitions ()`
- void `_templateCreateInternalStatisticReporters ()`
- void `_templateCreateAttachedAttributes ()`
- void `_templateCreateNonEditableDataDefinitions ()`
- void `_templateCreateEditableDataDefinitions ()`
- void `_internalStatisticReportersClear ()`
legacy helper methods used by the granular create hooks*
- void `_internalDataClear ()`
- void `_internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- void `_internalDataRemove (const std::string &key)`
- void `_attachedDataClear ()`
- void `_attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- void `_attachedDataRemove (const std::string &key)`
- void `_attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- void `_statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- void `_statisticReporterRemove (const std::string &key)`
- void `_statisticReportersClear ()`
- void `_mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- void `_mandatoryAttachedAttributeRemove (const std::string &key)`
- void `_mandatoryAttachedAttributesClear ()`
- void `_mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- void `_mandatoryEditableDataDefinitionRemove (const std::string &key)`
- void `_mandatoryEditableDataDefinitionsClear ()`
- void `_optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- void `_optionalEditableDataDefinitionRemove (const std::string &key)`
- void `_optionalEditableDataDefinitionsClear ()`
- void `_mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- void `_mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- void `_mandatoryNonEditableDataDefinitionsClear ()`
- void `_optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- void `_optionalNonEditableDataDefinitionRemove (const std::string &key)`
- void `_optionalNonEditableDataDefinitionsClear ()`
- void `_checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void `trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void `traceError (const std::string &text, const std::exception &e)`
- void `traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)`
- void `traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)`

- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()

virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.284.1 Detailed Description

FtsZ ring polymerization dynamics for whole-cell division timing.

Implements a simplified two-state Euler ODE for the FtsZ ring completion fraction $f \in [0,1]$, stored as a per-mille integer (0–1000) in [WholeCellState](#) under the key ftsZRingKey (default "FtsZ_ring_completion").

Monomer availability is read from [WholeCellState](#) under ftsZMonomerKey (default "FtsZ_monomer"). The reference count N_{ref} is captured on the first `_onDispatchEvent` call (N_{m} at $t=0$) and held constant; the fraction $N_{\text{m}}/N_{\text{ref}}$ scales the polymerization rate (lower monomer count = slower growth).

ODE (per step): $f_{\text{new}} = f + (k_{\text{poly}} * \phi * (1-f) - k_{\text{depoly}} * f) * \Delta T$ where $k_{\text{poly}} = \text{activationFwd} * \text{volumeNorm}$ [1/s] $k_{\text{depoly}} = \text{activationRev} * \text{volumeNorm}$ [1/s] $\phi = N_{\text{m}} / N_{\text{ref}}$ [dimensionless monomer availability]

Default $\text{volumeNorm} = 3.94\text{e-}5$ is calibrated so that $k_{\text{poly}} \approx 4.3\text{e-}5/\text{s}$, which gives $\sim 35\%$ ring completion at $t \approx 12\,000\text{ s}$ — consistent with the *M. genitalium* cell cycle. `activationFwd` and `activationRev` are read from [WholeCellState](#) "param.ftsZActivationFwd"/"param.ftsZActivationRev" when `loadParameters()` has been called; otherwise the component uses its own stored defaults.

15.284.2 Member Function Documentation

15.284.2.1 `_check()`

```
bool FtsZPolymerizationComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.284.2.2 `_createEditableDataDefinitions()`

```
void FtsZPolymerizationComponent::_createEditableDataDefinitions ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.284.2.3 _initBetweenReplications()

void FtsZPolymerizationComponent::_initBetweenReplications () [override], [protected], [virtual]
Reimplemented from [ModelDataDefinition](#).

15.284.2.4 _loadInstance()

bool FtsZPolymerizationComponent::_loadInstance (
 [PersistenceRecord](#) * fields) [override], [protected], [virtual]
virtual protected method that must be overridden
Reimplemented from [ModelComponent](#).

15.284.2.5 _onDispatchEvent()

void FtsZPolymerizationComponent::_onDispatchEvent (
 [Entity](#) * entity,
 unsigned int inputPortNumber) [override], [protected], [virtual]

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.284.2.6 _saveInstance()

void FtsZPolymerizationComponent::_saveInstance (
 [PersistenceRecord](#) * fields,
 bool saveDefaultValues) [override], [protected], [virtual]

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.284.2.7 show()

std::string FtsZPolymerizationComponent::show () [override], [virtual]
virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

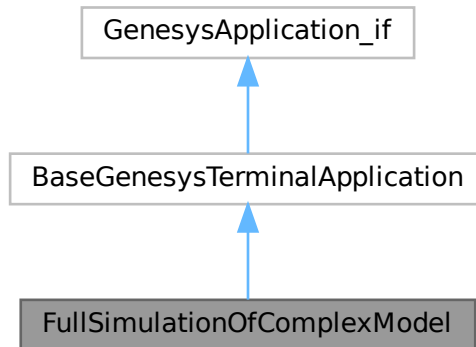
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

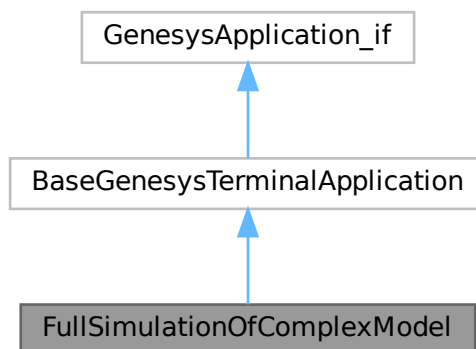
- plugins/components/WholeCellModeling/FtsZPolymerizationComponent.h
- plugins/components/WholeCellModeling/FtsZPolymerizationComponent.cpp

15.285 FullSimulationOfComplexModel Class Reference

Inheritance diagram for FullSimulationOfComplexModel:



Collaboration diagram for FullSimulationOfComplexModel:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

15.285.1 Member Function Documentation

15.285.1.1 main()

```
int FullSimulationOfComplexModel::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/teaching/FullSimulationOfComplexModel.h
- applications/modelSpecific/teaching/FullSimulationOfComplexModel.cpp

15.286 testing::internal::FunctionPointerPrinter Struct Reference

Static Public Member Functions

- `template<typename T , typename = typename std::enable_if< std::is_function<T>::value>::type>`
static void **PrintValue** (T *p, ::std::ostream *os)

The documentation for this struct was generated from the following file:

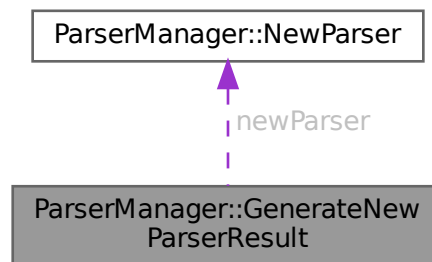
- gtest/gtest/gtest-printers.h

15.287 ParserManager::GenerateNewParserResult Struct Reference

Result bundle for parser generation requests.

```
#include <kernel/simulator/ParserManager.h>
```

Collaboration diagram for ParserManager::GenerateNewParserResult:



Public Attributes

- bool **result** = false
- std::string **bisonMessages**
- std::string **lexMessages**
- std::string **compilationMessages**
- [NewParser](#) **newParser**

15.287.1 Detailed Description

Result bundle for parser generation requests.

The documentation for this struct was generated from the following file:

- kernel/simulator/ParserManager.h

15.288 testing::internal::GenerateTypeList< T > Struct Template Reference

Public Types

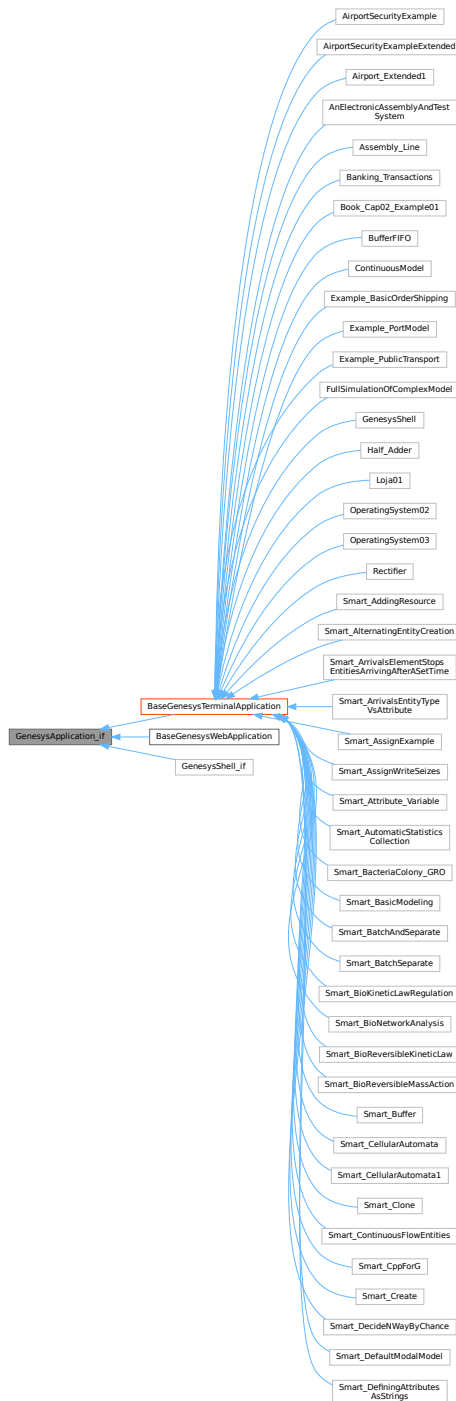
- using **type** = typename proxy::type

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-type-util.h

15.289 GenesysApplication_if Class Reference

Inheritance diagram for GenesysApplication_if:



Public Member Functions

- virtual int [main](#) (int argc, char **argv)=0

15.289.1 Member Function Documentation

15.289.1.1 main()

```
virtual int GenesysApplication_if::main (
    int argc,
    char ** argv ) [pure virtual]
```

Implemented in [Airport_Extended1](#), [Assembly_Line](#), [Banking_Transactions](#), [Example_BasicOrderShipping](#), [Example_PortModel](#), [Example_PublicTransport](#), [Smart_AddingResource](#), [Smart_AlternatingEntityCreation](#), [Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime](#), [Smart_AutomaticStatisticsCollection](#), [Smart_ContinuousFlowEntities](#), [Smart_Create](#), [Smart_DecideNWayByChance](#), [Smart_DefiningAttributesAsStrings](#), [Smart_DefiningControlLogic](#), [Smart_DefiningResourceCapacity](#), [Smart_EntitiesProcessedByPriority](#), [Smart_EvaluatingConditionsBeforeEnteringQueue](#), [Smart_Expression](#), [Smart_MaxArrivalsField](#), [Smart_ModelRunUntil1000Parts](#), [Smart_ModuleDisplayVariables](#), [Smart_OverlappingResources](#), [Smart_ParallelProcessingOfEntities](#), [Smart_PlacingEntitiesInQueueSets](#), [Smart_PriorityExample](#), [Smart_ProcessArena](#), [Smart_Record_Arena](#), [Smart_RemovingAndReorderingEntitiesInAQueue](#), [Smart_ResourceSets](#), [Smart_SeizingMultipleSimultaneously](#), [Smart_SelectingRouteBasedOnProbability](#), [Smart_SelectingShorterQueue](#), [Smart_SynchronizingParallelEntities](#), [Smart_WaitForSignal](#), [Smart_MarkovChain](#), [Smart_Buffer](#), [Smart_Clone](#), [Smart_Delay](#), [Smart_Failures](#), [Smart_HoldSearchRemove](#), [Smart_Process](#), [Smart_ProcessSet](#), [Smart_SeizeDelayRelease](#), [Smart_SeizeDelayReleaseMany](#), [Smart_SeizeDelayReleaseNoDataDefs](#), [Smart_R_Simulator](#), [Smart_Record](#), [Smart_Attribute_Variable](#), [Smart_ModelInfoModelSimulation](#), [Smart_Parser](#), [Smart_ParserModelFunctions](#), [Smart_RouteStation](#), [Smart_CellularAutomata](#), [Smart_CellularAutomata1](#), [Smart_DefaultModalModel](#), [Smart_Dummy](#), [Smart_ModalModel](#), [Smart_OnEvent](#), [Smart_WaitScanCondition](#), [Smart_WaitSignal](#), [FullSimulationOfComplexModel](#), [Half_Adder](#), [Rectifier](#), and [BaseGenesysWebApplication](#).

The documentation for this class was generated from the following file:

- applications/GenesysApplication_if.h

15.290 genesys_mcp.client.GenesysClient Class Reference

Public Member Functions

- None **__init__** (self, str base_url="http://localhost:8080", float timeout=30.0)
- dict[str, Any] **health** (self)
- dict[str, Any] **create_session** (self)
- dict[str, Any] **simulator_info** (self)
- dict[str, Any] **new_model** (self)
- dict[str, Any] **get_model** (self)
- dict[str, Any] **save_model** (self, str filename)
- dict[str, Any] **load_model** (self, str filename)
- dict[str, Any] **simulation_status** (self)
- dict[str, Any] **configure_simulation** (self, int number_of_replications, float replication_length, float warm←_up_period, bool pause_on_event=False, bool pause_on_replication=False, bool initialize_statistics=True, bool initialize_system=True)
- dict[str, Any] **run_simulation** (self)
- dict[str, Any] **step_simulation** (self)
- dict[str, Any] **worker_info** (self)
- dict[str, Any] **worker_capabilities** (self)
- dict[str, Any] **import_model_language** (self, str model_text)
- dict[str, Any] **create_job** (self)
- dict[str, Any] **get_job** (self, str job_id)
- dict[str, Any] **run_job** (self, str job_id)
- dict[str, Any] **get_job_result** (self, str job_id)
- None **close** (self)
- [GenesysClient](#) **__enter__** (self)
- None **__exit__** (self, *object _)

Protected Member Functions

- dict[str, str] **_headers** (self)
- Any **_get** (self, str path)
- Any **_post** (self, str path, Any body=None)
- Any **_post_text** (self, str path, str text)

Protected Attributes

- **_base_url**
- **_http**
- **_token**

15.290.1 Detailed Description

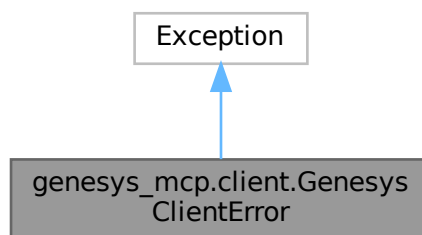
Stateful HTTP client for one Genesys worker-app session.

The documentation for this class was generated from the following file:

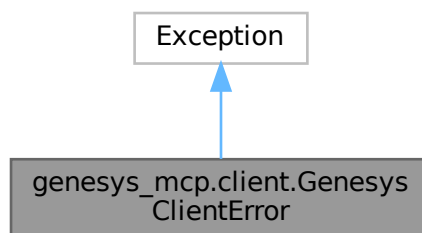
- applications/mcp/genesys_mcp/client.py

15.291 genesys_mcp.client.GenesysClientError Class Reference

Inheritance diagram for genesys_mcp.client.GenesysClientError:



Collaboration diagram for genesys_mcp.client.GenesysClientError:



Public Member Functions

- None `__init__` (self, int status_code, str message)

Public Attributes

- **status_code**

15.291.1 Detailed Description

Raised when the Genesys REST API returns an error.

The documentation for this class was generated from the following file:

- applications/mcp/genesys_mcp/client.py

15.292 GenesysKnowledgeBaseSummary Struct Reference

Aggregate summary of the current knowledge base state.

```
#include <tools/AIAssistant/AIAssistant_if.h>
```

Public Attributes

- unsigned int **numberOfPlugins** = 0
- unsigned int **numberOfComponents** = 0
- unsigned int **numberOfDataDefinitions** = 0
- std::string **summaryMarkdown**

Full knowledge base rendered as a Markdown table.

15.292.1 Detailed Description

Aggregate summary of the current knowledge base state.

The documentation for this struct was generated from the following file:

- tools/AIAssistant/AIAssistant_if.h

15.293 GenesysKnowledgeItem Struct Reference

One entry in the GenESyS knowledge base used to inform the AI assistant.

```
#include <tools/AIAssistant/AIAssistant_if.h>
```

Public Attributes

- std::string **name**
Short identifier (e.g. "Queue", "Create", "Entity").
- std::string **category**
Broad category: "Component", "DataDefinition", "Plugin", etc.
- std::string **typeName**
[Exact](#) C++ typename used by the kernel (e.g. "Queue").
- std::string **description**
One-paragraph description of purpose and behaviour.
- std::string **usage**
Typical usage pattern or construction example.
- std::string **constraints**
Known constraints, limitations, or required connection rules.

15.293.1 Detailed Description

One entry in the GenESyS knowledge base used to inform the AI assistant.

Knowledge items describe available simulation building blocks so that the assistant can map natural-language descriptions to concrete GenESyS types. Items are populated from:

- Static documentation embedded at build time.
- Dynamic introspection of connected plugins via [SimulatorFacade](#) (Stage 2, see [AIAssistant_if::refreshKnowledgeBaseFromSimu](#)

The documentation for this struct was generated from the following file:

- tools/AIAssistant/AIAssistant_if.h

15.294 genesyspp_driver Class Reference

Public Member Functions

- **genesyspp_driver** ([Model](#) *model, [Sampler_if](#) *sampler, bool throws=false)
- **genesyspp_driver** (const [genesyspp_driver](#) &other)
- [genesyspp_driver](#) & **operator=** (const [genesyspp_driver](#) &other)
- **genesyspp_driver** ([genesyspp_driver](#) &&other) noexcept
- [genesyspp_driver](#) & **operator=** ([genesyspp_driver](#) &&other) noexcept
- void **scan_begin_file** ()
- void **scan_end_file** ()
- void **scan_begin_str** ()
- void **scan_end_str** ()
- int **parse_file** (const std::string &f)
- int **parse_str** (const std::string &str)
- void **error** (const [yy::location](#) &l, const std::string &m)
- void **error** (const std::string &m)
- double **getResult** ()
- void **setResult** (double value)
- bool **getThrowsException** ()
- void **setThrowsException** (bool throws)
- void **setErrorMessage** (std::string message)
- std::string **getErrorMessage** ()
- [Model](#) * **getModel** ()
- std::string **getFile** ()
- void **setFile** (std::string f)
- std::string **getStrToParse** ()
- void **setStrToParse** (std::string str)
- void **setSampler** ([Sampler_if](#) *_sampler)
- [Sampler_if](#) * **getSampler** () const
- void **setRegisterReferredDataElements** (bool value)
- bool **getRegisterReferredDataElements** ()
- std::map< std::string, std::list< std::string > * > * **getReferredDataElements** ()
- void **clearReferredDataElements** ()
- void **addReferred** (std::pair< std::string, std::string > referredElement)
- [SimulationResponse](#) * **findSimulationResponse** (const std::string &name) const
- [SimulationControl](#) * **findSimulationControl** (const std::string &name) const
- double **getSimulationResponseValueAsDouble** (const std::string &name) const
- double **getSimulationControlValueAsDouble** (const std::string &name) const
- double **stringToDoubleOrWarn** (const std::string &sourceType, const std::string &symbolName, const std::string &valueText) const
- void **traceWarning** (const std::string &message) const

The documentation for this class was generated from the following files:

- parser/Genesys++-driver.h
- parser/Genesys++-driver.cpp
- parser/Genesys++-scanner.cpp

15.295 yy::genesyspp_parser Class Reference

A Bison parser.

```
#include <parser/GenesysParser.h>
```

Classes

- struct [basic_symbol](#)
- struct [by_kind](#)
Type access provider for token (enum) based symbols.
- class [context](#)
- struct [symbol_kind](#)
Symbol kinds.
- struct [symbol_type](#)
"External" symbols: returned by the scanner.
- struct [syntax_error](#)
Syntax errors thrown from user actions.
- struct [token](#)
Token kinds.
- class [value_type](#)

Public Types

- typedef [value_type](#) **semantic_type**
Backward compatibility (Bison 3.8).
- typedef [location](#) **location_type**
Symbol locations.
- typedef token::token_kind_type **token_kind_type**
Token kind, as returned by yylex.
- typedef [token_kind_type](#) **token_type**
Backward compatibility alias (Bison 3.6).
- typedef [symbol_kind::symbol_kind_type](#) **symbol_kind_type**
(Internal) symbol kind.
- typedef [by_kind](#) **by_type**
Backward compatibility for a private implementation detail (Bison 3.6).
- typedef int **debug_level_type**
Type for debugging levels.

Public Member Functions

- **genesyspp_parser** ([genesyspp_driver](#) &driver_yyarg)
Build a parser object.
- int **operator()** ()
- virtual int **parse** ()
- std::ostream & **debug_stream** () const YY_ATTRIBUTE_PURE
The current debugging stream.
- void **set_debug_stream** (std::ostream &)
Set the current debugging stream.
- [debug_level_type](#) **debug_level** () const YY_ATTRIBUTE_PURE
The current debugging level.
- void **set_debug_level** ([debug_level_type](#) l)
Set the current debugging level.
- virtual void **error** (const [location_type](#) &loc, const std::string &msg)
- void **error** (const [syntax_error](#) &err)
Report a syntax error.

Static Public Member Functions

- static std::string [symbol_name](#) ([symbol_kind_type](#) yysymbol)
- static [symbol_type](#) [make_END](#) (const [location_type](#) &l)
- static [symbol_type](#) [make_YYerror](#) (const [location_type](#) &l)
- static [symbol_type](#) [make_YYUNDEF](#) (const [location_type](#) &l)
- static [symbol_type](#) [make_NUMD](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_NUMH](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_CTEZERO](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_oLE](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_oGE](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_oEQ](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_oNE](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_oAND](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_oOR](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_oNAND](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_oXOR](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_oNOT](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fSIN](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fCOS](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fROUND](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fMOD](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fTRUNC](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fFRAC](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fEXP](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fSQRT](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fLOG](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fLN](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_mathMIN](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_mathMAX](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fVAL](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fEVAL](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fLENG](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_frND1](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fEXPO](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fNORM](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fUNIF](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fWEIB](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fLOGN](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fGAMM](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fERLA](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fTRIA](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fBETA](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fDISC](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fTNOW](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fTFIN](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fMAXREP](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fNUMREP](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_fIDENT](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_simulEntitiesWIP](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_cIF](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_cELSE](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_cFOR](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_cTO](#) (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) [make_cDO](#) (const [obj_t](#) &v, const [location_type](#) &l)

- static [symbol_type](#) `make_ATTRIB` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_CSTAT` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_COUNTER` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_SIMRESP` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_SIMCTRL` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_fTAVG` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_fCOUNT` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_ILLEGAL` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_RESOURCE` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_fNR` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_fMR` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_fIRF` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_fRESSEIZES` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_fSTATE` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_fSETSUM` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_fRESUTIL` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_QUEUE` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_fNQ` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_fFIRSTINQ` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_fLASTINQ` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_fSAQUE` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_fAQUE` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_fENTATRANK` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_SET` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_fNUMSET` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_VARI` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_FORM` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_fNUMGR` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_fATRGR` (const [obj_t](#) &v, const [location_type](#) &l)
- static [symbol_type](#) `make_LPAREN` (const [location_type](#) &l)
- static [symbol_type](#) `make_RPAREN` (const [location_type](#) &l)
- static [symbol_type](#) `make_LBRACKET` (const [location_type](#) &l)
- static [symbol_type](#) `make_RBRACKET` (const [location_type](#) &l)
- static [symbol_type](#) `make_PLUS` (const [location_type](#) &l)
- static [symbol_type](#) `make_MINUS` (const [location_type](#) &l)
- static [symbol_type](#) `make_STAR` (const [location_type](#) &l)
- static [symbol_type](#) `make_POWER` (const [location_type](#) &l)
- static [symbol_type](#) `make_SLASH` (const [location_type](#) &l)
- static [symbol_type](#) `make_LESS` (const [location_type](#) &l)
- static [symbol_type](#) `make_GREATER` (const [location_type](#) &l)
- static [symbol_type](#) `make_ASSIGN` (const [location_type](#) &l)
- static [symbol_type](#) `make_COMMA` (const [location_type](#) &l)

Static Public Attributes

- static const [symbol_kind_type](#) `YYNTOKENS` = [symbol_kind::YYNTOKENS](#)

The number of tokens.

15.295.1 Detailed Description

A Bison parser.

15.295.2 Member Function Documentation

15.295.2.1 error()

```
void yy::genesyspp_parser::error (
    const location\_type & loc,
    const std::string & msg ) [virtual]
```

Report a syntax error.

Parameters

<i>loc</i>	where the syntax error is found.
<i>msg</i>	a description of the syntax error.

15.295.2.2 operator>()

```
int yy::genesyspp_parser::operator() ( )
```

Parse. An alias for parse ().

Returns

0 iff parsing succeeded.

15.295.2.3 parse()

```
int yy::genesyspp_parser::parse ( ) [virtual]
```

Parse.

Returns

0 iff parsing succeeded.

Length of the RHS of the rule being reduced.

The lookahead symbol.

The locations where the error started and ended.

The return value of parse ().

15.295.2.4 symbol_name()

```
std::string yy::genesyspp_parser::symbol_name (
    symbol\_kind\_type yysymbol ) [static]
```

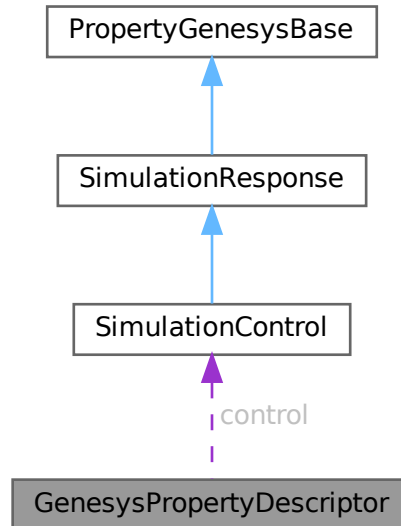
The user-facing name of the symbol whose (internal) number is YYSYMBOL. No bounds checking.

The documentation for this class was generated from the following files:

- parser/[GenesysParser.h](#)
- parser/GenesysParser.cpp

15.296 GenesysPropertyDescriptor Struct Reference

Collaboration diagram for GenesysPropertyDescriptor:



Public Attributes

- [SimulationControl](#) * **control** = nullptr
- std::string **ownerClassName**
- std::string **ownerElementName**
- std::string **displayName**
- std::string **technicalTypeName**
- GenesysPropertyKind **kind** = GenesysPropertyKind::Unknown
- bool **readOnly** = true
- bool **isList** = false
- bool **isClass** = false
- bool **isEnum** = false
- bool **isInlineObject** = false
- bool **isModelDataDefinitionReference** = false
- bool **supportsInlineExpansion** = false
- bool **supportsListEditor** = false
- bool **supportsExistingObjectSelection** = false
- bool **supportsObjectCreation** = false
- bool **supportsNewListElementCreation** = false
- SimulationControlEditorHint **editorHint** = SimulationControlEditorHint::Default
- std::string **currentValue**
- std::vector< std::string > **choices**
- std::string **currentReferenceType**
- std::vector< std::string > **creatableReferenceTypes**
- std::string **currentListElementType**
- std::vector< std::string > **creatableListElementTypes**

The documentation for this struct was generated from the following file:

- kernel/simulator/GenesysPropertyIntrospection.h

15.297 GenesysPropertyIntrospection Class Reference

Static Public Member Functions

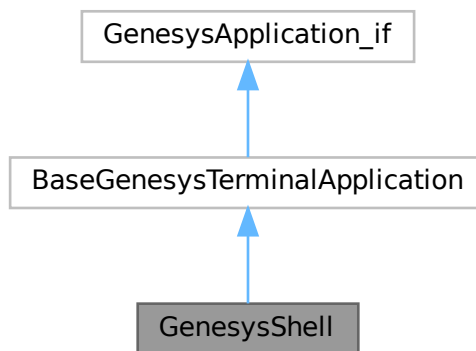
- static [GenesysPropertyDescriptor](#) **describe** ([SimulationControl](#) *control)
- static std::vector< [GenesysPropertyDescriptor](#) > **describe** (List< [SimulationControl](#) * > *controls)
- static GenesysPropertyKind **deduceKind** (const [SimulationControl](#) *control)
- static bool **setValue** ([SimulationControl](#) *control, const std::string &value, bool remove=false, std::string *errorMessage=nullptr)
- static std::vector< [SimulationControl](#) * > **children** ([SimulationControl](#) *control, int index=0)

The documentation for this class was generated from the following files:

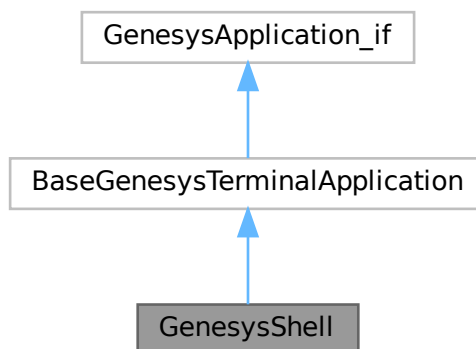
- kernel/simulator/GenesysPropertyIntrospection.h
- kernel/simulator/GenesysPropertyIntrospection.cpp

15.298 GenesysShell Class Reference

Inheritance diagram for GenesysShell:



Collaboration diagram for GenesysShell:



Public Member Functions

- **GenesysShell** (const [GenesysShell](#) &orig)
- virtual int **main** (int argc, char **argv) override
- void **cmdScript** ()
- void **cmdExecuteScript** ()
- void **cmdHelp** ()
- void **cmdQuit** ()
- void **cmdBash** ()
- void **cmdFacade** ()
- void **cmdLicence** ()
- void **cmdInfo** ()
- void **cmdSim** ()
- void **cmdData** ()
- void **cmdComponent** ()
- void **cmdExperiment** ()
- void **cmdParser** ()
- void **cmdTraceLevel** ()
- void **cmdPlugin** ()
- void **cmdSimulation** ()
- void **cmdReplication** ()
- void **cmdModel** ()
- void **cmdModelLoad** ()
- void **cmdModelSave** ()

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

15.298.1 Member Function Documentation

15.298.1.1 main()

```
int GenesysShell::main (
    int argc,
    char ** argv ) [override], [virtual]
```

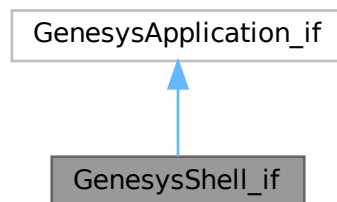
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

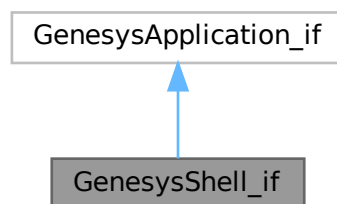
- applications/shell/GenesysShell.h
- applications/shell/GenesysShell.cpp

15.299 GenesysShell_if Class Reference

Inheritance diagram for GenesysShell_if:



Collaboration diagram for GenesysShell_if:



Public Member Functions

- virtual void **openModel** (std::string filename)=0
- virtual void **saveModelAs** (std::string filename)=0
- virtual void **saveModel** ()=0
- virtual void **listElements** ()=0
- virtual void **listComponents** ()=0
- virtual void **listHosts** ()=0
- virtual void **listPlugins** ()=0
- virtual void **deleteTraceFiles** ()=0

- virtual void **traceLevel** (TraceManager::TraceLevel tracelevel)=0
- virtual void **addPlugin** (std::string filename)=0
- virtual void **addFromFile** (std::string filename)=0
- virtual void **readCommandsFromFile** (std::string filename)=0
- virtual void **redirectTrace** (std::string trace, std::string dest, std::string filename)=0
- virtual void **closeModel** ()=0
- virtual void **createModel** ()=0
- virtual void **execLinuxCommand** (std::string command)=0
- virtual void **verboseMode** (bool on)=0
- virtual void **check** ()=0
- virtual void **getGenesysInfo** ()=0
- virtual void **getCommandLine** ()=0
- virtual void **sendFile** (std::string filename, std::string hostname, std::string portname)=0
- virtual void **setActivationCode** (std::string code)=0
- virtual void **receiveFile** (std::string filename)=0
- virtual void **startSimulation** ()=0
- virtual void **stepSimulation** ()=0
- virtual void **stopSimulation** ()=0
- virtual void **showInit** ()=0
- virtual void **showHelp** ()=0
- virtual void **showHostName** ()=0

Public Member Functions inherited from [GenesysApplication_if](#)

- virtual int [main](#) (int argc, char **argv)=0

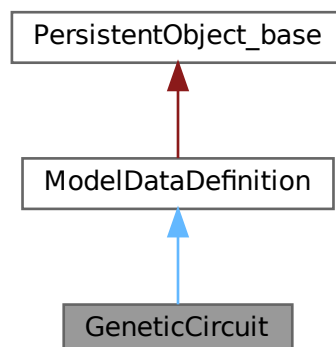
The documentation for this class was generated from the following file:

- applications/shell/GenesysShell_if.h

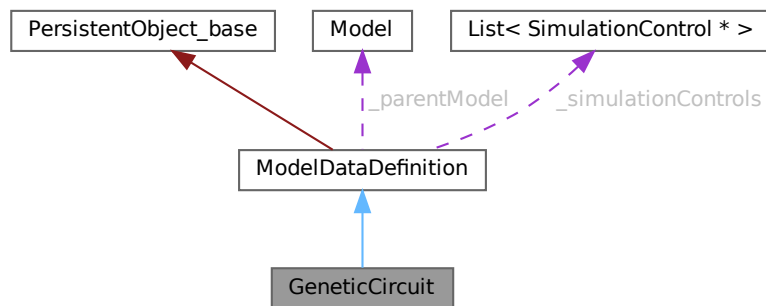
15.300 GeneticCircuit Class Reference

```
#include <plugins/data/BiochemicalSimulation/GeneticCircuit.h>
```

Inheritance diagram for GeneticCircuit:



Collaboration diagram for GeneticCircuit:



Public Member Functions

- **GeneticCircuit** ([Model](#) *model, std::string name="")
- void **addPart** (std::string partName)
- void **addRegulation** (std::string regulationName)
- void **clearParts** ()
- void **clearRegulations** ()
- const std::vector< std::string > & **getPartNames** () const
- const std::vector< std::string > & **getRegulationNames** () const
- void **setHostOrganism** (std::string hostOrganism)
- std::string **getHostOrganism** () const
- void **setCompartment** (std::string compartment)
- std::string **getCompartment** () const
- void **setEnabled** (bool enabled)
- bool **isEnabled** () const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
- *Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- *Releases resources owned by this data definition.*
- Util::identification [getId](#) () const
- *Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)
- *Updates the logical name of this instance.*
- const std::string & [getName](#) () const
- *Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
- *Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const
- *Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const

- Returns the runtime class name used by the plugin system.*
- **Model * getParentModel** () const
Returns the parent model that owns this definition.
- **bool isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- **void setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition * getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- **std::map< std::string, ModelDataDefinition * > * getInternalData** () const
Returns the full map of internal child data definitions.
- **std::map< std::string, ModelDataDefinition * > * getAttachedData** () const
Returns the full map of attached data definitions.
- **bool hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- **void setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
- **unsigned int getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- **void setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- **List< SimulationControl * > * getSimulationControls** () const
Returns the simulation controls owned by this data definition.
- **TraceManager::Level getTraceLevelSpecific** () const
Returns the trace level that applies specifically to this definition.
- **void setTraceLevelSpecific** (TraceManager::Level level)
Updates the trace level used by this definition.
- **void defineTraceLevelSpecific** (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific←Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- **bool isTraceLevelSpecificEnabled** () const
Indicates whether the specific trace level is enabled.
- **void setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- **static ModelDataDefinition * LoadInstance** (Model *model, PersistenceRecord *fields)
- **static PluginInformation * GetPluginInformation** ()
- **static ModelDataDefinition * NewInstance** (Model *model, std::string name="")

Static Public Member Functions inherited from ModelDataDefinition

- **static ModelDataDefinition * LoadInstance** (Model *model, PersistenceRecord *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- **static ModelDataDefinition * NewInstance** (Model *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- **static void SaveInstance** (PersistenceRecord *fields, ModelDataDefinition *modeldatum)
Serializes a data definition instance into a persistence record.
- **static bool Check** (ModelDataDefinition *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- **static void InitBetweenReplications** (ModelDataDefinition *modeldatum)

Resets simulation-time state between replications.

- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()

- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.300.1 Detailed Description

Aggregate genetic circuit definition.

A [GeneticCircuit](#) stores an ordered sequence of genetic parts and a set of regulatory relationships by name so it can be persisted independently from runtime object pointers.

15.300.2 Member Function Documentation

15.300.2.1 _check()

```
bool GeneticCircuit::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
 Reimplemented from [ModelDataDefinition](#).

15.300.2.2 _loadInstance()

```
bool GeneticCircuit::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes
 Reimplemented from [ModelDataDefinition](#).

15.300.2.3 _saveInstance()

```
void GeneticCircuit::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.300.2.4 show()

```
std::string GeneticCircuit::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

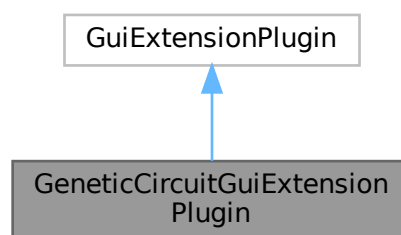
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

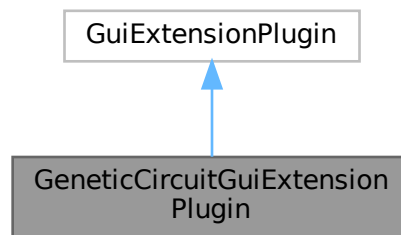
- plugins/data/BiochemicalSimulation/GeneticCircuit.h
- plugins/data/BiochemicalSimulation/GeneticCircuit.cpp

15.301 GeneticCircuitGuiExtensionPlugin Class Reference

Inheritance diagram for GeneticCircuitGuiExtensionPlugin:



Collaboration diagram for GeneticCircuitGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId () const override`
- `std::string displayName () const override`
- `std::string version () const override`
- `std::vector< std::string > requiredModelPlugins () const override`
- `void registerContributions (GuiExtensionRegistry *registry) const override`

15.301.1 Member Function Documentation

15.301.1.1 displayName()

```
std::string GeneticCircuitGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.301.1.2 extensionId()

```
std::string GeneticCircuitGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.301.1.3 registerContributions()

```
void GeneticCircuitGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.301.1.4 requiredModelPlugins()

```
std::vector< std::string > GeneticCircuitGuiExtensionPlugin::requiredModelPlugins ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.301.1.5 version()

```
std::string GeneticCircuitGuiExtensionPlugin::version ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

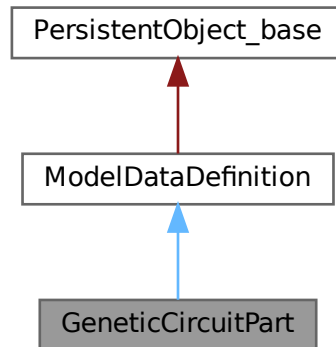
The documentation for this class was generated from the following file:

- `applications/gui/genesys/extensions/GeneticCircuitGuiExtensionPlugin.cpp`

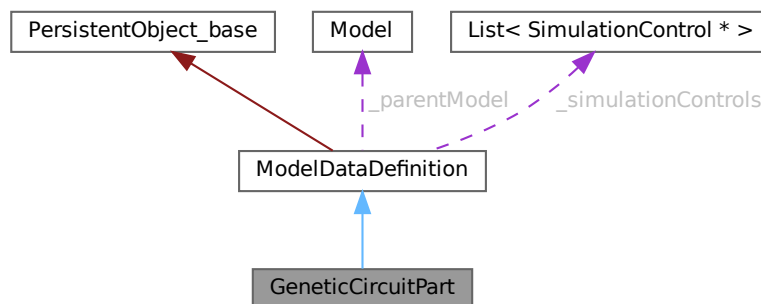
15.302 GeneticCircuitPart Class Reference

```
#include <plugins/data/BiochemicalSimulation/GeneticCircuitPart.h>
```

Inheritance diagram for GeneticCircuitPart:



Collaboration diagram for GeneticCircuitPart:



Public Member Functions

- **GeneticCircuitPart** ([Model](#) *model, std::string name="")
- void **setPartType** (std::string partType)
- std::string **getPartType** () const
- void **setSequence** (std::string sequence)
- std::string **getSequence** () const
- void **setProductSpeciesName** (std::string productSpeciesName)
- std::string **getProductSpeciesName** () const
- void **setCopyNumber** (double copyNumber)
- double **getCopyNumber** () const
- void **setBasalExpressionRate** (double basalExpressionRate)
- double **getBasalExpressionRate** () const
- void **setDegradationRate** (double degradationRate)
- double **getDegradationRate** () const

- void **setEnabled** (bool enabled)
- bool **isEnabled** () const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()

- void **_templateCreateEditableDataDefinitions** ()
- void **_internaStatisticReportersClear** ()
 - legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.302.1 Detailed Description

Genetic circuit structural part definition.

This data definition represents a reusable genetic part (promoter, RBS, CDS, terminator, etc.) and optionally links it to the [BioSpecies](#) produced by the part expression flow.

15.302.2 Member Function Documentation

15.302.2.1 `_check()`

```
bool GeneticCircuitPart::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.302.2.2 `_loadInstance()`

```
bool GeneticCircuitPart::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.302.2.3 `_saveInstance()`

```
void GeneticCircuitPart::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.302.2.4 `show()`

```
std::string GeneticCircuitPart::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

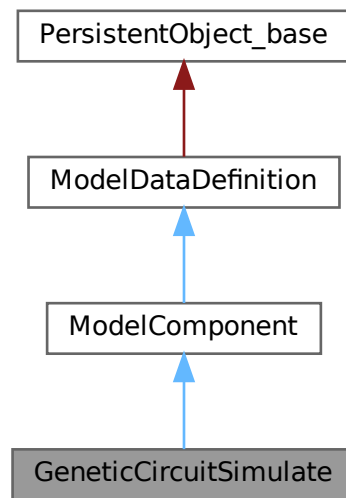
The documentation for this class was generated from the following files:

- plugins/data/BiochemicalSimulation/GeneticCircuitPart.h
- plugins/data/BiochemicalSimulation/GeneticCircuitPart.cpp

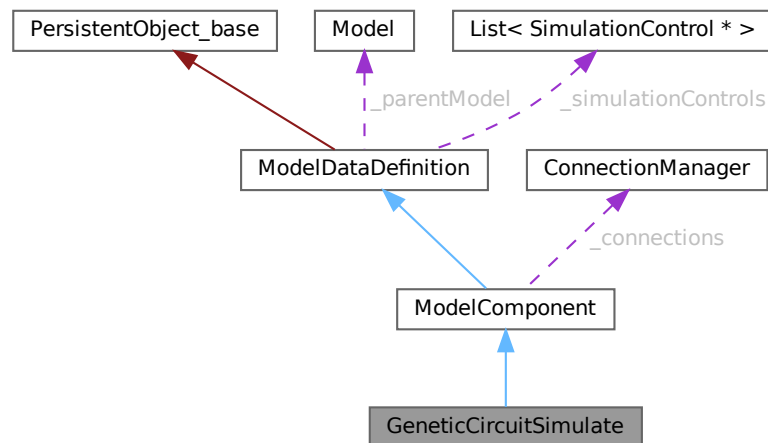
15.303 GeneticCircuitSimulate Class Reference

```
#include <plugins/components/BiochemicalSimulation/GeneticCircuitSimulate.h>
```

Inheritance diagram for GeneticCircuitSimulate:



Collaboration diagram for GeneticCircuitSimulate:



Public Member Functions

- **GeneticCircuitSimulate** ([Model](#) *model, std::string name="")
- void **setGeneticCircuit** ([GeneticCircuit](#) *geneticCircuit)
- [GeneticCircuit](#) * **getGeneticCircuit** () const
- void **setStartTime** (double startTime)
- double **getStartTime** () const
- void **setStopTime** (double stopTime)
- double **getStopTime** () const

- void **setStepSize** (double stepSize)
- double **getStepSize** () const
- void **setApplyRegulation** (bool applyRegulation)
- bool **getApplyRegulation** () const
- void **setLastSucceeded** (bool lastSucceeded)
- bool **getLastSucceeded** () const
- void **setLastSampleCount** (unsigned int lastSampleCount)
- unsigned int **getLastSampleCount** () const
- void **setLastTotalExpression** (double lastTotalExpression)
- double **getLastTotalExpression** () const
- void **setLastMessage** (std::string lastMessage)
- std::string **getLastMessage** () const
- bool **simulate** (std::string &errorMessage)
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentName, std::string name="")

constructors

- virtual ~[ModelComponent](#) ()

Releases component-owned resources and unregisters it from the model.

- [ConnectionManager](#) * [getConnectionManager](#) () const

new public user methods for this component

- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)

Connects this component to another component.

- bool [hasBreakpointAt](#) ()

Indicates whether a simulation breakpoint is set on this component.

- void [setDescription](#) (std::string _description)

Updates the free-form description shown by the GUI.

- std::string [getDescription](#) () const

Returns the free-form description associated with this component.

- void [remove](#) ([ModelComponent](#) *comp)

Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionType, std::string name="", bool insert↵
IntoModel=true)

Creates a model data definition and optionally inserts it into the model.

- virtual ~[ModelDataDefinition](#) ()

Releases resources owned by this data definition.

- Util::identification [getId](#) () const

Returns the unique identifier of this instance.

- void [setName](#) (const std::string &name)

Updates the logical name of this instance.

- const std::string & [getName](#) () const

Returns the logical name of this instance.

- void [setLabel](#) (const std::string &label)

Updates the user-visible label of this instance.

- const std::string & [getLabel](#) () const

Returns the user-visible label of this instance.

- const std::string & [getClassname](#) () const

- Returns the runtime class name used by the plugin system.*
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- **List**< **SimulationControl** * > * **getSimulationControls** () const
Returns the simulation controls owned by this data definition.
- **TraceManager::Level** **getTraceLevelSpecific** () const
Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (**TraceManager::Level** level)
Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (**TraceManager::Level** traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const
Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static **PluginInformation** * **GetPluginInformation** ()
- static **ModelComponent** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")
- static bool **simulateCircuit** (**Model** *model, **GeneticCircuit** *geneticCircuit, double startTime, double stopTime, double stepSize, bool applyRegulation, **GeneticCircuitSimulationSummary** *summary, std::string &errorMessage)

Static Public Member Functions inherited from **ModelComponent**

- static **ModelComponent** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** (**ModelComponent** *component)
Validates a component instance.
- static void **CreateInternalData** (**ModelComponent** *component)
Creates internal data required by a component instance.

- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)

- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.303.1 Detailed Description

Component that simulates a [GeneticCircuit](#) over a time window.

15.303.2 Member Function Documentation

15.303.2.1 `_check()`

```
bool GeneticCircuitSimulate::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.303.2.2 `_createEditableDataDefinitions()`

```
void GeneticCircuitSimulate::_createEditableDataDefinitions ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.303.2.3 `_loadInstance()`

```
bool GeneticCircuitSimulate::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.303.2.4 `_onDispatchEvent()`

```
void GeneticCircuitSimulate::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.303.2.5 `_saveInstance()`

```
void GeneticCircuitSimulate::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.303.2.6 `show()`

```
std::string GeneticCircuitSimulate::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/BiochemicalSimulation/GeneticCircuitSimulate.h
- plugins/components/BiochemicalSimulation/GeneticCircuitSimulate.cpp

15.304 GeneticCircuitSimulationSummary Struct Reference

Public Attributes

- bool **succeeded** = false
- unsigned int **sampleCount** = 0
- double **totalExpression** = 0.0
- std::string **message** = ""

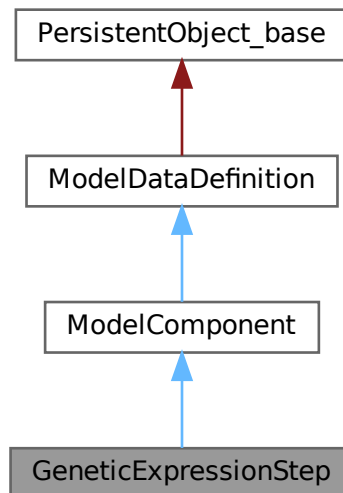
The documentation for this struct was generated from the following file:

- plugins/components/BiochemicalSimulation/GeneticCircuitSimulate.h

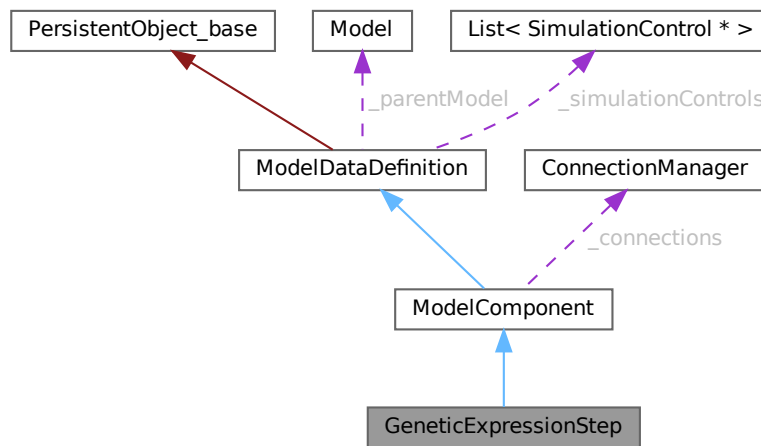
15.305 GeneticExpressionStep Class Reference

```
#include <plugins/components/BiochemicalSimulation/GeneticExpressionStep.h>
```

Inheritance diagram for GeneticExpressionStep:



Collaboration diagram for GeneticExpressionStep:



Public Member Functions

- **GeneticExpressionStep** ([Model](#) *model, std::string name="")
- void **setGeneticCircuit** ([GeneticCircuit](#) *geneticCircuit)
- [GeneticCircuit](#) * **getGeneticCircuit** () const
- void **setTimeStep** (double timeStep)
- double **getTimeStep** () const
- void **setApplyRegulation** (bool applyRegulation)
- bool **getApplyRegulation** () const
- void **setLastSucceeded** (bool lastSucceeded)
- bool **getLastSucceeded** () const
- void **setLastTotalExpression** (double lastTotalExpression)
- double **getLastTotalExpression** () const
- void **setLastMessage** (std::string lastMessage)
- std::string **getLastMessage** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.

- `std::string getDescription () const`
Returns the free-form description associated with this component.
- `void remove (ModelComponent *comp)`
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, `std::string` datadefinitionTypename, `std::string` name="", `bool` insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- `virtual ~ModelDataDefinition ()`
Releases resources owned by this data definition.
- `Util::identification getId () const`
Returns the unique identifier of this instance.
- `void setName (const std::string &name)`
Updates the logical name of this instance.
- `const std::string & getName () const`
Returns the logical name of this instance.
- `void setLabel (const std::string &label)`
Updates the user-visible label of this instance.
- `const std::string & getLabel () const`
Returns the user-visible label of this instance.
- `const std::string & getClassname () const`
Returns the runtime class name used by the plugin system.
- **Model** * `getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * `getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↔ Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
 - legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

15.305.1 Detailed Description

Component that executes one expression update step over a [GeneticCircuit](#).

15.305.2 Member Function Documentation

15.305.2.1 `_check()`

```
bool GeneticExpressionStep::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.305.2.2 `_createEditableDataDefinitions()`

```
void GeneticExpressionStep::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.305.2.3 `_loadInstance()`

```
bool GeneticExpressionStep::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [ModelComponent](#).

15.305.2.4 `_onDispatchEvent()`

```
void GeneticExpressionStep::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.305.2.5 _saveInstance()

```
void GeneticExpressionStep::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.305.2.6 show()

```
std::string GeneticExpressionStep::show ( ) [override], [virtual]
virtual public methods
```

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

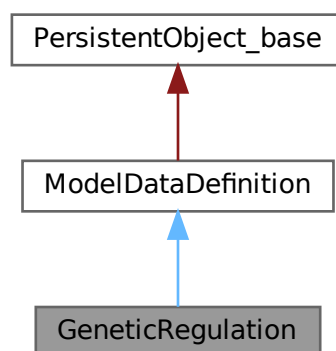
The documentation for this class was generated from the following files:

- plugins/components/BiochemicalSimulation/GeneticExpressionStep.h
- plugins/components/BiochemicalSimulation/GeneticExpressionStep.cpp

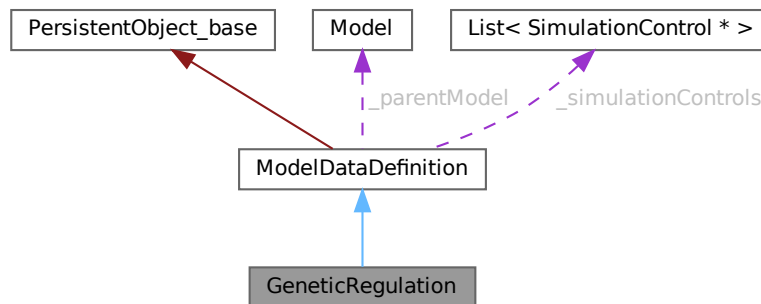
15.306 GeneticRegulation Class Reference

```
#include <plugins/data/BiochemicalSimulation/GeneticRegulation.h>
```

Inheritance diagram for GeneticRegulation:



Collaboration diagram for GeneticRegulation:



Public Member Functions

- **GeneticRegulation** ([Model](#) *model, std::string name="")
- void **setRegulatorSpeciesName** (std::string regulatorSpeciesName)
- std::string **getRegulatorSpeciesName** () const
- void **setTargetPartName** (std::string targetPartName)
- std::string **getTargetPartName** () const
- void **setRegulationType** (std::string regulationType)
- std::string **getRegulationType** () const
- void **setHillCoefficient** (double hillCoefficient)
- double **getHillCoefficient** () const
- void **setDissociationConstant** (double dissociationConstant)
- double **getDissociationConstant** () const
- void **setMaxFoldChange** (double maxFoldChange)
- double **getMaxFoldChange** () const
- void **setLeakiness** (double leakiness)
- double **getLeakiness** () const
- void **setEnabled** (bool enabled)
- bool **isEnabled** () const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
- *Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- *Releases resources owned by this data definition.*
- Util::identification **getId** () const
- *Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
- *Updates the logical name of this instance.*
- const std::string & **getName** () const
- *Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)
- *Updates the user-visible label of this instance.*

- `const std::string & getLabel () const`
Returns the user-visible label of this instance.
- `const std::string & getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static PluginInformation * GetPluginInformation ()`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from ModelDataDefinition

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- `static void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.

- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()

- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.306.1 Detailed Description

Transcriptional regulation relationship for genetic circuits.

15.306.2 Member Function Documentation

15.306.2.1 _check()

```
bool GeneticRegulation::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
 Reimplemented from [ModelDataDefinition](#).

15.306.2.2 _loadInstance()

```
bool GeneticRegulation::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes
 Reimplemented from [ModelDataDefinition](#).

15.306.2.3 _saveInstance()

```
void GeneticRegulation::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.306.2.4 show()

```
std::string GeneticRegulation::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

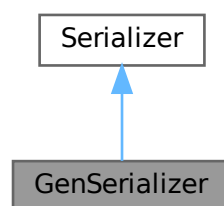
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

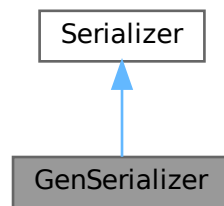
- plugins/data/BiochemicalSimulation/GeneticRegulation.h
- plugins/data/BiochemicalSimulation/GeneticRegulation.cpp

15.307 GenSerializer Class Reference

Inheritance diagram for GenSerializer:



Collaboration diagram for GenSerializer:



Public Member Functions

- **GenSerializer** ([Model](#) *model)
- [PersistenceRecord](#) * **newPersistenceRecord** () override
Creates an empty persistence record.
- bool **dump** (std::ostream &output) override
Writes formatted contents to a given output stream.
- bool **load** (std::istream &input) override
Attempts to load contents from a given input stream.
- bool **get** (const std::string &name, [PersistenceRecord](#) *entry) override
Tries to fetch an entry with the given label as a key.
- bool **put** (const std::string name, const std::string type, const Util::identification id, [PersistenceRecord](#) *fields) override
Upserts an entry into this structure.
- int **for_each** (std::function< int(const std::string &) > delegate) override
Iterates over this structure's entries, ordered by ID.

Friends

- class **PersistenceDefaultImpl2**

15.307.1 Member Function Documentation

15.307.1.1 dump()

```
bool GenSerializer::dump (
    std::ostream & output ) [override], [virtual]
```

Writes formatted contents to a given output stream.

Parameters

<i>output</i>	output stream to write to.
---------------	----------------------------

Returns

whether the current state of this structure's contents was deemed valid for serialization.

Implements [Serializer](#).

15.307.1.2 for_each()

```
int GenSerializer::for_each (
    std::function< int(const std::string &) > delegate ) [override], [virtual]
```

Iterates over this structure's entries, ordered by ID.

Parameters

<i>delegate</i>	delegate which is applied to every entry-identifying key, until it returns a non-zero value.
-----------------	--

Returns

either zero, or the first non-zero value returned by the delegate.

Implements [Serializer](#).

15.307.1.3 get()

```
bool GenSerializer::get (
    const std::string & name,
    PersistenceRecord * entry ) [override], [virtual]
```

Tries to fetch an entry with the given label as a key.

Parameters

<i>name</i>	a label which uniquely identifies an entry.
<i>entry</i>	(out-param) a pointer to a caller-owned entry, which is filled on success.

Returns

whether the entry was found and the out-param was actually written to.

Implements [Serializer](#).

15.307.1.4 load()

```
bool GenSerializer::load (
    std::istream & input ) [override], [virtual]
```

Attempts to load contents from a given input stream.

Returns

whether the input was well-formatted and loading was successful.

Implements [Serializer](#).

15.307.1.5 newPersistenceRecord()

```
PersistenceRecord * GenSerializer::newPersistenceRecord ( ) [override], [virtual]
```

Creates an empty persistence record.

Although the returned record can be freed normally with `delete`, care must be taken to only use records which come from a implementation, when configuring that implementation's contents.

Implements [Serializer](#).

15.307.1.6 put()

```
bool GenSerializer::put (
    const std::string name,
    const std::string type,
    const Util::identification id,
    PersistenceRecord * fields ) [override], [virtual]
```

Upserts an entry into this structure.

Parameters

<i>name</i>	a label which uniquely identifies the entry.
<i>type</i>	a type name.
<i>id</i>	a component/datadef ID number.
<i>fields</i>	a persistence record for the entry, whose contents will be copied if valid.

Returns

whether the entry was valid and was actually copied into this structure.

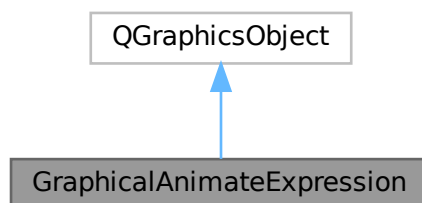
Implements [Serializer](#).

The documentation for this class was generated from the following files:

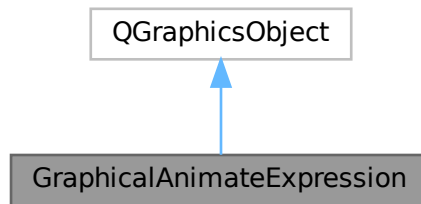
- kernel/simulator/persistence/GenSerializer.h
- kernel/simulator/persistence/GenSerializer.cpp

15.308 GraphicalAnimateExpression Class Reference

Inheritance diagram for GraphicalAnimateExpression:



Collaboration diagram for GraphicalAnimateExpression:



The documentation for this class was generated from the following files:

- applications/gui/genesys/graphics/GraphicalAnimateExpression.h
- applications/gui/genesys/graphics/GraphicalAnimateExpression.cpp

15.309 GraphicalAssociation Class Reference

The documentation for this class was generated from the following files:

- applications/gui/genesys/graphics/GraphicalAssociation.h
- applications/gui/genesys/graphics/GraphicalAssociation.cpp

15.310 GraphicalAutomaticPositioningStrategy Class Reference

Public Member Functions

- virtual [~GraphicalAutomaticPositioningStrategy](#) ()=default
Strategy interface for automatic placement of graphical data definitions.
- virtual QPointF **initialComponentLayoutOrigin** () const =0
Returns the origin used when laying out the first graphical component.
- virtual QPointF [dataDefinitionPosition](#) (const QRectF &anchorBounds, const QSizeF &childSize, int index, int count, bool upperArc, int radialLayer) const =0
Computes the position of one child item around an anchor rectangle.

15.310.1 Constructor & Destructor Documentation

15.310.1.1 ~GraphicalAutomaticPositioningStrategy()

```
virtual GraphicalAutomaticPositioningStrategy::~~GraphicalAutomaticPositioningStrategy ( )
[virtual], [default]
```

Strategy interface for automatic placement of graphical data definitions.

Implementations decide how child graphical items are laid out around an anchor item. The GUI uses this abstraction so layout policy can be selected from preferences without changing the reconstruction code that consumes the strategy.

15.310.2 Member Function Documentation

15.310.2.1 dataDefinitionPosition()

```
virtual QPointF GraphicalAutomaticPositioningStrategy::dataDefinitionPosition (
    const QRectF & anchorBounds,
```

```

    const QSizeF & childSize,
    int index,
    int count,
    bool upperArc,
    int radialLayer ) const [pure virtual]

```

Computes the position of one child item around an anchor rectangle.

Parameters

<i>anchorBounds</i>	Bounds of the item being decorated.
<i>childSize</i>	Size of the child item to place.
<i>index</i>	Zero-based child index in the current arc.
<i>count</i>	Number of children to place in the same arc.
<i>upperArc</i>	True when the child should be positioned above the anchor.
<i>radialLayer</i>	Additional radial layer for nested attachments.

Returns

Scene position where the child should be placed.

The documentation for this class was generated from the following file:

- applications/gui/genesys/services/GraphicalAutomaticPositioningStrategy.h

15.311 GraphicalAutomaticPositioningStrategyFactory Class Reference

Factory for the automatic positioning strategy selected from GUI preferences.

```
#include <applications/gui/genesys/services/GraphicalAutomaticPositioningStrategy.h>
```

Static Public Member Functions

- static std::unique_ptr< [GraphicalAutomaticPositioningStrategy](#) > **createFromPreferences** ()
Creates the preferred automatic positioning strategy instance.

15.311.1 Detailed Description

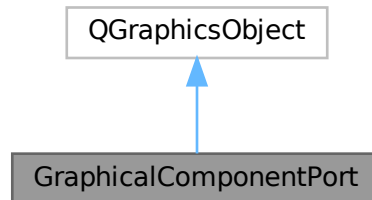
Factory for the automatic positioning strategy selected from GUI preferences.

The documentation for this class was generated from the following files:

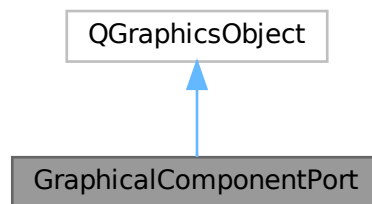
- applications/gui/genesys/services/GraphicalAutomaticPositioningStrategy.h
- applications/gui/genesys/services/GraphicalAutomaticPositioningStrategy.cpp

15.312 GraphicalComponentPort Class Reference

Inheritance diagram for GraphicalComponentPort:



Collaboration diagram for GraphicalComponentPort:



Public Member Functions

- `GraphicalComponentPort` (`GraphicalModelComponent` *componentGraph, bool `isInputPort`, unsigned int `portNum`=0, `QGraphicsItem` *parent=nullptr)
Visual port attached to a graphical component for connection routing.
- `QRectF` **boundingRect** () const override
Returns the local bounding rectangle used for hit testing and painting.
- void **paint** (`QPainter` *painter, const `QStyleOptionGraphicsItem` *option, `QWidget` *widget) override
Paints the port handle using the component and selection styling.
- qreal **width** () const
Returns the logical width used by the port geometry.
- qreal **height** () const
Returns the logical height used by the port geometry.
- void **addGraphicalConnection** (`GraphicalConnection` *connection)
Registers one connection attached to this port.
- void **removeGraphicalConnection** (`GraphicalConnection` *connection)
Unregisters one connection attached to this port.
- unsigned int **portNum** () const
Returns the port number within the owning component.
- bool **isInputPort** () const

Returns true when this is an input port.

- [GraphicalModelComponent](#) * **graphicalComponent** () const

Returns the owning graphical component.

- QList< [GraphicalConnection](#) * > * **getConnections** () const

Returns the list of connections currently attached to this port.

Protected Member Functions

- virtual QVariant **itemChange** (QGraphicsItem::GraphicsItemChange change, const QVariant &value) override

Keeps the port aligned when its parent component moves or is reparented.

15.312.1 Constructor & Destructor Documentation

15.312.1.1 GraphicalComponentPort()

```
GraphicalComponentPort::GraphicalComponentPort (
    GraphicalModelComponent * componentGraph,
    bool isInputPort,
    unsigned int portNum = 0,
    QGraphicsItem * parent = nullptr )
```

Visual port attached to a graphical component for connection routing.

Ports are the connection endpoints used by the scene to attach [GraphicalConnection](#) items to a component. The port item keeps track of connected edges and exposes enough geometry for the scene to render and route links consistently.

The documentation for this class was generated from the following files:

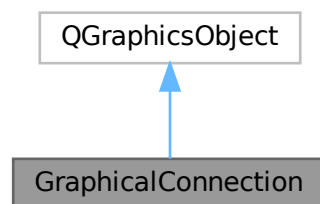
- applications/gui/genesys/graphics/GraphicalComponentPort.h
- applications/gui/genesys/graphics/GraphicalComponentPort.cpp

15.313 GraphicalConnection Class Reference

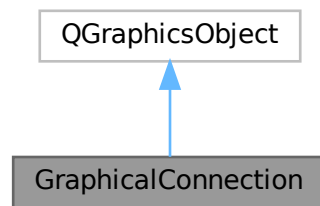
Graphical edge that represents model-level component connections.

```
#include <applications/gui/genesys/graphics/GraphicalConnection.h>
```

Inheritance diagram for GraphicalConnection:



Collaboration diagram for GraphicalConnection:



Public Types

- enum class **ConnectionType** : int { **HORIZONTAL** = 1 , **VERTICAL** = 2 , **DIRECT** = 3 , **USERDEFINED** = 4 }

Public Member Functions

- GraphicalConnection** (**GraphicalComponentPort** *sourceGraphicalPort, **GraphicalComponentPort** *destinationGraphicalPort, unsigned int portSourceConnection=0, unsigned int portDestinationConnection=0, **QColor** color=Qt::black, **QGraphicsItem** *parent=nullptr)

Creates a graphical connection between source and destination ports.
- GraphicalConnection** (const **GraphicalConnection** &orig)

Copy constructor used by restoration and clipboard flows.
- virtual ~**GraphicalConnection** ()

Releases the connection item and its auxiliary routing state.
- QRectF** **boundingRect** () const override

Returns item bounding rectangle for Qt painting system.
- QPainterPath** **shape** () const override

Returns the precise clickable shape around the visible connection path.
- void **paint** (**QPainter** *painter, const **QStyleOptionGraphicsItem** *option, **QWidget** *widget) override

Paints connection path and selection handles.
- Connection** * **getSource** () const

Returns auxiliary source Connection metadata.
- Connection** * **getDestination** () const

Returns auxiliary destination Connection metadata.
- GraphicalComponentPort** * **getSourceGraphicalPort** ()

Returns source graphical port.
- GraphicalComponentPort** * **getDestinationGraphicalPort** ()

Returns destination graphical port.
- GraphicalModelComponent** * **getSourceGraphicalComponent** () const

Returns source graphical component resolved from source port when available.
- GraphicalModelComponent** * **getDestinationGraphicalComponent** () const

Returns destination graphical component resolved from destination port when available.
- void **updateDimensionsAndPosition** ()

Recomputes local geometry based on current port positions.
- GraphicalConnection::ConnectionType** **connectionType** () const

Returns configured connection routing type.
- unsigned int **getPortSourceConnection** () const

- Returns source output port index associated with model connection.*
- unsigned int **getPortDestinationConnection** () const
Returns destination input port index associated with model connection.
- void **setConnectionType** (GraphicalConnection::ConnectionType newConnectionType)
Sets routing type used during painting.
- QList< QPointF > **getPoints** () const
Returns sampled scene points used for serialization/inspection.
- bool **usesCurvedStyle** () const
Returns true when the current visual style uses curved model connection paths.
- QPainterPath **animationPathForImage** (qreal imageWidth, qreal imageHeight) const
Returns the scene path followed by entity animation in modern connection style.

Protected Member Functions

- virtual bool **sceneEvent** (QEvent *event) override

15.313.1 Detailed Description

Graphical edge that represents model-level component connections.

[GraphicalConnection](#) links two [GraphicalComponentPort](#) objects and keeps auxiliary [Connection](#) metadata synchronized with the simulator model.

The item is also used for the diagram-level routing that the GUI derives from the model structure when rendering connections inside the scene.

Todo Evaluate decoupling model metadata from this item to keep this class purely visual.

15.313.2 Constructor & Destructor Documentation

15.313.2.1 GraphicalConnection()

```
GraphicalConnection::GraphicalConnection (
    GraphicalComponentPort * sourceGraphicalPort,
    GraphicalComponentPort * destinationGraphicalPort,
    unsigned int portSourceConnection = 0,
    unsigned int portDestinationConnection = 0,
    QColor color = Qt::black,
    QGraphicsItem * parent = nullptr )
```

Creates a graphical connection between source and destination ports.

Bloco 1: inicialização de metadados de conexão. Mantemos wrappers de [Connection](#) para facilitar sincronização com kernel.

Bloco 2: configuração visual/interativa do item.

Bloco 3: cálculo inicial de geometria da conexão.

Bloco 4: registro bidirecional nas portas para atualização quando houver movimento.

Todo Extrair para método dedicado `_bindPorts()` para legibilidade.

15.313.3 Member Function Documentation

15.313.3.1 boundingRect()

```
QRectF GraphicalConnection::boundingRect ( ) const [override]
```

Returns item bounding rectangle for Qt painting system.

Bloco 1: calcula margem extra baseada na porta para reduzir clipping visual.

Bloco 2: retorna retângulo local com margem.

15.313.3.2 paint()

```
void GraphicalConnection::paint (
    QPainter * painter,
    const QStyleOptionGraphicsItem * option,
    QWidget * widget ) [override]
```

Paints connection path and selection handles.

Bloco 2: normaliza coordenadas locais considerando orientação relativa.

Bloco 3: gera path de desenho com base no tipo de roteamento.

Bloco 4: renderiza o path final da conexão.

Bloco 5: desenha “handles” quando selecionado.

15.313.3.3 updateDimensionsAndPosition()

```
void GraphicalConnection::updateDimensionsAndPosition ( )
```

Recomputes local geometry based on current port positions.

Bloco 1: coleta de geometria absoluta de portas.

Bloco 2: projeta esta conexão para um sistema de coordenadas local mínimo.

Bloco 3: mantém apenas atualização geométrica local.

The documentation for this class was generated from the following files:

- applications/gui/genesys/graphics/GraphicalConnection.h
- applications/gui/genesys/graphics/GraphicalConnection.cpp

15.314 GraphicalConnectionStyle Class Reference

Public Types

- enum class **RouteType** { **Horizontal** , **Vertical** , **Direct** , **UserDefined** }

Static Public Member Functions

- static bool **usesModernCurves** ()
- static QPainterPath **modelConnectionPath** (const QPointF &source, const QPointF &destination, RouteType routeType, bool modernCurves)
- static QPainterPath **diagramConnectionPath** (const QPointF &source, const QPointF &destination, bool modernCurves)
- static QPointF **pointAtProgress** (const QPainterPath &path, qreal progress)

The documentation for this class was generated from the following files:

- applications/gui/genesys/graphics/GraphicalConnectionStyle.h
- applications/gui/genesys/graphics/GraphicalConnectionStyle.cpp

15.315 GraphicalContextMenuController Class Reference

Builds context menus for the graphical model view using existing main-window actions.

```
#include <applications/gui/genesys/controllers/GraphicalContextMenuController.h>
```

Public Member Functions

- **GraphicalContextMenuController** ([ModelGraphicsView](#) *graphicsView, Ui::MainWindow *ui, QAction *openSubmodelAction, std::function< [ModelGraphicsScene](#) *()> currentScene, std::function< void()> actualizeActions)
Creates the controller without taking ownership of UI or scene objects.
- void **handleGraphicsViewContextMenu** (QContextMenuEvent *event)
Shows the context menu matching the item or background under the mouse.

15.315.1 Detailed Description

Builds context menus for the graphical model view using existing main-window actions.

The controller intentionally reuses QAction instances from the main window so shortcuts, enabled state, icons, checked state, and command handlers remain centralized.

The documentation for this class was generated from the following files:

- applications/gui/genesys/controllers/GraphicalContextMenuController.h
- applications/gui/genesys/controllers/GraphicalContextMenuController.cpp

15.316 GraphicalDataDefinitionLayout Class Reference

Pure helper that computes arc placement for graphical data definitions.

```
#include <applications/gui/genesys/services/GraphicalDataDefinitionLayout.h>
```

Static Public Member Functions

- static QPointF [arcPosition](#) (const QRectF &anchorBounds, const QSizeF &childSize, int index, int count, bool upperArc, int radialLayer=0)

Computes a child position along the upper or lower arc around an anchor item.

15.316.1 Detailed Description

Pure helper that computes arc placement for graphical data definitions.

This helper keeps the geometry math independent from the GUI controllers and scene code so layout policies can be reused in both live rebuilds and persisted-state restoration.

15.316.2 Member Function Documentation

15.316.2.1 arcPosition()

```
QPointF GraphicalDataDefinitionLayout::arcPosition (
    const QRectF & anchorBounds,
    const QSizeF & childSize,
    int index,
    int count,
    bool upperArc,
    int radialLayer = 0 ) [static]
```

Computes a child position along the upper or lower arc around an anchor item.

Parameters

<i>anchorBounds</i>	Bounds of the anchor graphical item.
<i>childSize</i>	Size of the child item to place.
<i>index</i>	Zero-based child index.
<i>count</i>	Total number of children in the arc.
<i>upperArc</i>	True for the upper arc, false for the lower arc.
<i>radialLayer</i>	Additional radial offset used for nested attachment rings.

Returns

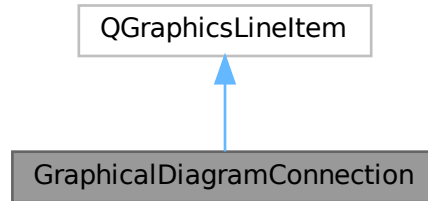
Scene position for the requested child.

The documentation for this class was generated from the following files:

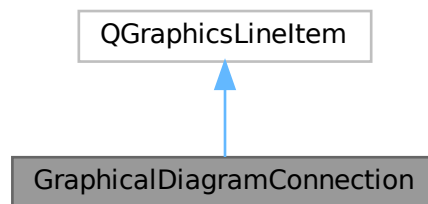
- applications/gui/genesys/services/GraphicalDataDefinitionLayout.h
- applications/gui/genesys/services/GraphicalDataDefinitionLayout.cpp

15.317 GraphicalDiagramConnection Class Reference

Inheritance diagram for GraphicalDiagramConnection:



Collaboration diagram for GraphicalDiagramConnection:



Public Types

- enum class **ConnectionType** { **ATTACHED** , **INTERNAL** }

Public Member Functions

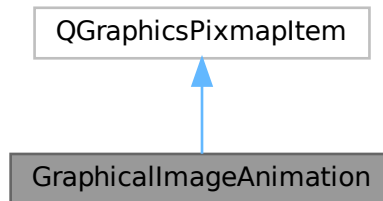
- **GraphicalDiagramConnection** (`QGraphicsItem *dataDefinition`, `QGraphicsItem *linkedTo`, `ConnectionType type`)
- void **refreshGeometry** ()
- `QRectF` **boundingRect** () const override
- `QPainterPath` **shape** () const override
- void **paint** (`QPainter *painter`, const `QStyleOptionGraphicsItem *option`, `QWidget *widget=nullptr`) override
- `QGraphicsItem *` **getDataDefinition** ()
- `QGraphicsItem *` **getLinkedDataDefinition** ()
- `ConnectionType` **getConnectionType** ()

The documentation for this class was generated from the following files:

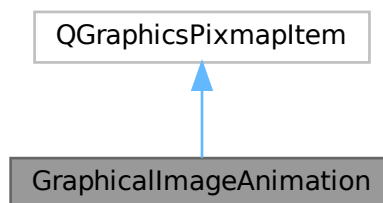
- `applications/gui/genesys/graphics/GraphicalDiagramConnection.h`
- `applications/gui/genesys/graphics/GraphicalDiagramConnection.cpp`

15.318 GraphicalImageAnimation Class Reference

Inheritance diagram for GraphicalImageAnimation:



Collaboration diagram for GraphicalImageAnimation:



Public Member Functions

- **GraphicalImageAnimation** (const QPointF startPoint, unsigned int width, unsigned int height, const QString imageName)
- QString **getImagePath** () const
- QString **getImageName** () const
- QPointF **getStartPoint** () const
- unsigned int **getWidth** () const
- unsigned int **getHeight** () const
- void **setImageName** (const QString &imageName)
- void **setStartPoint** (const QPointF &startPoint)
- void **setWidth** (unsigned int imageWidth)
- void **setHeight** (unsigned int imageHeight)
- void **updateImage** ()

The documentation for this class was generated from the following files:

- applications/gui/genesys/graphics/GraphicalImageAnimation.h
- applications/gui/genesys/graphics/GraphicalImageAnimation.cpp

15.319 GraphicalModelBuilder Class Reference

Service that reconstructs graphical components/connections from kernel model data.

```
#include <applications/gui/genesys/services/GraphicalModelBuilder.h>
```

Public Member Functions

- [GraphicalModelBuilder](#) ([Simulator](#) *simulator, [ModelGraphicsView](#) *graphicsView, [ModelGraphicsScene](#) *scene, std::map< std::string, QColor > *pluginCategoryColor, QTextEdit *console)
Creates the reconstruction service used by load/rebuild compatibility flows.
- void [setModelLevelFilter](#) (unsigned int modelLevel)
Restricts this builder and the target scene to one [ModelDataDefinition](#) level.
- void [clearModelLevelFilter](#) ()
Clears the level restriction and restores full-model rebuild semantics.
- void [recursivelyGenerateGraphicalModelFromModel](#) ([ModelComponent](#) *component, List< [ModelComponent](#) * > *visited, std::map< [ModelComponent](#) *, [GraphicalModelComponent](#) * > *map, int *x, int *y, int *ymax, int sequenceInLine)
Recursively rebuilds graphical items and links from one model-component branch.
- void [generateGraphicalModelFromModel](#) ()
Rebuilds the full scene model representation from kernel source components.

Static Public Member Functions

- static void [synchronizeGraphicalDataDefinitionsLayer](#) ([Simulator](#) *simulator, [ModelGraphicsScene](#) *scene)
Synchronizes only data-definition graphical layer with current model preserving existing layout.

15.319.1 Detailed Description

Service that reconstructs graphical components/connections from kernel model data.

This builder is used after model loading to recreate the scene representation while keeping [MainWindow](#) as a compatibility façade. It acts as a model-representation bridge between the kernel component graph and GUI graphical items.

Responsibilities:

- recursively create graphical nodes/connections for component branches;
- generate a full scene representation from model source components;
- apply plugin-category visual metadata needed during reconstruction.

Boundaries:

- it does not persist files or parse textual model language;
- it does not manage selection/property editor/simulation command flows;
- it operates as a reconstruction service, not a controller.

15.319.2 Constructor & Destructor Documentation

15.319.2.1 GraphicalModelBuilder()

```
GraphicalModelBuilder::GraphicalModelBuilder (
    Simulator * simulator,
    ModelGraphicsView * graphicsView,
    ModelGraphicsScene * scene,
    std::map< std::string, QColor > * pluginCategoryColor,
    QTextEdit * console )
```

Creates the reconstruction service used by load/rebuild compatibility flows.

Builds the graphical reconstruction service with explicit dependencies.

15.319.3 Member Function Documentation

15.319.3.1 generateGraphicalModelFromModel()

```
void GraphicalModelBuilder::generateGraphicalModelFromModel ( )
```

Rebuilds the full scene model representation from kernel source components.
Rebuilds the full scene representation from the current kernel model.

15.319.3.2 recursivelyGenerateGraphicalModelFromModel()

```
void GraphicalModelBuilder::recursivelyGenerateGraphicalModelFromModel (
    ModelComponent * component,
    List< ModelComponent * > * visited,
    std::map< ModelComponent *, GraphicalModelComponent * > * map,
    int * x,
    int * y,
    int * ymax,
    int sequenceInLine )
```

Recursively rebuilds graphical items and links from one model-component branch.

Recursively rebuilds graphical items and connections from one branch.

15.319.3.3 setModelLevelFilter()

```
void GraphicalModelBuilder::setModelLevelFilter (
    unsigned int modelLevel )
```

Restricts this builder and the target scene to one [ModelDataDefinition](#) level.

Root model tabs use level 0 today. [Submodel](#) tabs will use the owner component/data definition level, so rebuilds can be reused without mixing visual scopes.

15.319.3.4 synchronizeGraphicalDataDefinitionsLayer()

```
void GraphicalModelBuilder::synchronizeGraphicalDataDefinitionsLayer (
    Simulator * simulator,
    ModelGraphicsScene * scene ) [static]
```

Synchronizes only data-definition graphical layer with current model preserving existing layout.

Synchronizes the data-definition layer against the current kernel model.

The documentation for this class was generated from the following files:

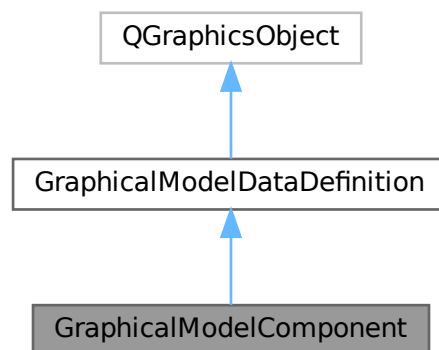
- applications/gui/genesys/services/GraphicalModelBuilder.h
- applications/gui/genesys/services/GraphicalModelBuilder.cpp

15.320 GraphicalModelComponent Class Reference

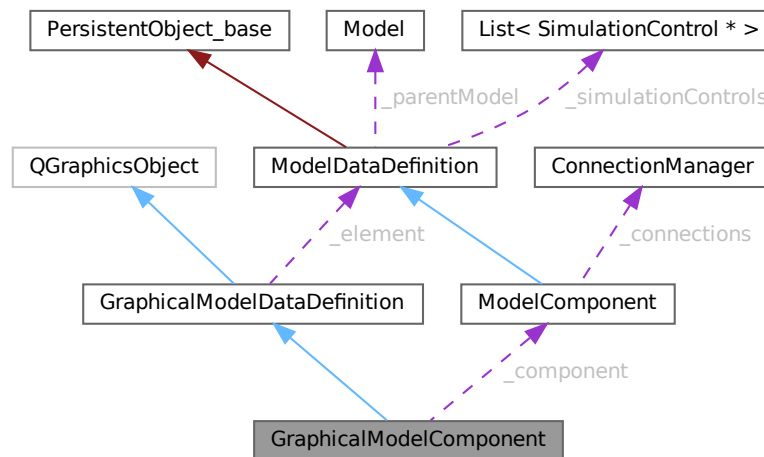
Visual component item that extends a data definition with ports and internal data.

```
#include <applications/gui/genesys/graphics/GraphicalModelComponent.h>
```

Inheritance diagram for GraphicalModelComponent:



Collaboration diagram for GraphicalModelComponent:



Public Member Functions

- **GraphicalModelComponent** ([Plugin](#) *plugin, [ModelComponent](#) *component, QPointF position, QColor color=Qt::blue, QGraphicsItem *parent=nullptr)
Creates a graphical component bound to a kernel component and plugin metadata.
- **GraphicalModelComponent** (const [GraphicalModelComponent](#) &orig)
Copy constructor used by clipboard and restoration flows.
- virtual ~**GraphicalModelComponent** ()
Releases the component item, including its generated port items and queues.
- QRectF **boundingRect** () const override
Returns the rectangle used for painting and hit testing the component body.
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget) override
Paints the component body, ports, and selection indicators.
- [ModelComponent](#) * **getComponent** () const
Returns the kernel component represented by this item.
- qreal **getHeight** () const
Returns the current item height derived from the render strategy.
- QPointF **getOldPosition** () const
Returns the previous scene position used by undo and restoration flows.
- void **setOldPosition** (QPointF oldPosition)
Stores the previous scene position used by undo and restoration flows.
- QList< [GraphicalComponentPort](#) * > **getGraphicalInputPorts** () const
Returns the input ports generated for this component.
- QList< [GraphicalComponentPort](#) * > **getGraphicalOutputPorts** () const
Returns the output ports generated for this component.
- QColor **getColor** () const
Returns the component color used by the renderer.
- void **setColor** (const QColor &color)
Updates the component color used by the renderer.
- QList< [ModelDataDefinition](#) * > * **getInternalData** () const
Returns the list of internal data definitions attached to the component.

- `QList< ModelDataDefinition * > * getAttachedData ()` const
Returns the list of externally attached data definitions.
- `EntityType * getEntityType ()` const
Returns the entity type synthesized or associated with the component.
- `void setEntityType (EntityType *entityType)`
Sets the entity type synthesized or associated with the component.
- `unsigned int getOccupiedInputPorts ()` const
Returns how many input ports are currently occupied.
- `unsigned int getOccupiedOutputPorts ()` const
Returns how many output ports are currently occupied.
- `void setOccupiedInputPorts (unsigned int value)`
Stores the number of occupied input ports tracked by the layout code.
- `void setOccupiedOutputPorts (unsigned int value)`
Stores the number of occupied output ports tracked by the layout code.
- `QString getAnimationImageName ()`
Returns the animation image identifier used for queue visualization.
- `void setAnimationImageName (QString name)`
Updates the animation image identifier used by the renderer.
- `QMap< Queue *, QPair< unsigned int, unsigned int > > * getMapQueue ()`
Returns the queue-to-metadata map used by the queue overlay renderer.
- `QList< QList< GraphicalImageAnimation * > * > * getImagesQueue ()`
Returns the grouped queue image lists used by the renderer.
- `void verifyQueue ()`
Verifies whether the component currently references a queue animation context.
- `bool hasQueue ()`
Returns true when a queue animation context is available.
- `unsigned int getIndexQueue (Queue *queue)`
Returns the queue index assigned during queue-overlay population.
- `unsigned int getSizeQueue (Queue *queue)`
Returns the queue size recorded during queue-overlay population.
- `void populateMapQueue (QList< Queue * > queues)`
Populates the queue overlay metadata from the supplied queue list.
- `bool insertImageQueue (Queue *queue, GraphicalImageAnimation *image)`
Inserts one animation image entry for the given queue.
- `QList< GraphicalImageAnimation * > * removeImageQueue (Queue *queue, unsigned int quantity↵ Removed)`
Removes a batch of animation images for the given queue.
- `bool insertImageQueue (GraphicalImageAnimation *image)`
Inserts one animation image using the current queue context.
- `GraphicalImageAnimation * removeImageQueue ()`
Removes the most recently inserted queue animation image.
- `void actualizeMapQueue (Queue *queue)`
Refreshes the queue metadata after the queue model changes.
- `void visivibleImageQueue (bool visivible)`
Shows or hides the queue image overlay.
- `void clearQueues ()`
Clears the cached queue overlay state.

Public Member Functions inherited from [GraphicalModelDataDefinition](#)

- **GraphicalModelDataDefinition** ([Plugin](#) *plugin, [ModelDataDefinition](#) *element, QPointF position, QColor color=Qt::blue, QGraphicsItem *parent=nullptr)
Creates one visual item bound to a kernel data definition.
- **GraphicalModelDataDefinition** (const [GraphicalModelDataDefinition](#) &orig)
Copy constructor used by scene-copy and restore flows.
- virtual ~**GraphicalModelDataDefinition** ()
Releases the graphical item and any owned helper state.
- QRectF **boundingRect** () const override
Returns the item bounds used by Qt for painting and hit testing.
- QPainterPath **shape** () const override
Returns the precise shape used for selection and collision checks.
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget) override
Paints the data-definition body, label, and selection frame.
- [ModelDataDefinition](#) * **getDataDefinition** () const
Returns the kernel data definition represented by this item.
- bool **isEditableInPropertyEditor** () const
Returns whether the property editor may edit this item in place.
- void **setEditableInPropertyEditor** (bool editable)
Marks the item as editable or read-only in the property editor.
- QPointF **getOldPosition** () const
Returns the last persisted scene position before a move operation.
- void **setOldPosition** (qreal x, qreal y)
Stores the old scene position used by undo and restoration flows.
- QColor **getColor** () const
Returns the color used to paint the item.
- void **setColor** (QColor newColor)
Sets the color used to paint the item.
- qreal **getHeight** () const
Returns the height used by the current render strategy.

Static Public Member Functions

- static bool **compareQueuesById** (const [Queue](#) *a, const [Queue](#) *b)
Compares queues by identifier so queue overlays can be ordered deterministically.

Protected Member Functions

- virtual bool [sceneEvent](#) (QEvent *event) override
- QVariant **itemChange** (GraphicsItemChange change, const QVariant &value) override
- [GraphicalModelItemRenderContext](#) **renderContext** () const override

Protected Member Functions inherited from [GraphicalModelDataDefinition](#)

- QColor **myrgba** (uint64_t color)

Protected Attributes

- qreal `_width` = [TraitsGUI](#)<GModelComponent>::width
- qreal `_height` = `_width` * [TraitsGUI](#)<GModelComponent>::heightProportion
- unsigned int `_margin` = [TraitsGUI](#)<GModelComponent>::margin
- unsigned int `_selWidth` = [TraitsGUI](#)<GModelComponent>::selectionWidth
- [ModelComponent](#) * `_component`
- QColor `_color`
- QPointF `_oldPosition`
- qreal `_stretchPosTop` = [TraitsGUI](#)<GModelComponent>::stretchPos
- qreal `_stretchPosBottom` = [TraitsGUI](#)<GModelComponent>::stretchPos
- qreal `_stretchPosLeft` = [TraitsGUI](#)<GModelComponent>::stretchPos
- qreal `_stretchPosRigth` = [TraitsGUI](#)<GModelComponent>::stretchPos
- qreal `_stretchRigth` = [TraitsGUI](#)<GModelComponent>::stretch
- qreal `_stretchLeft` = [TraitsGUI](#)<GModelComponent>::stretch
- qreal `_stretchRigthMiddle` = [TraitsGUI](#)<GModelComponent>::stretch
- qreal `_stretchLeftMiddle` = [TraitsGUI](#)<GModelComponent>::stretch
- qreal `_stretchTop` = [TraitsGUI](#)<GModelComponent>::stretch
- qreal `_stretchBottom` = [TraitsGUI](#)<GModelComponent>::stretch
- qreal `_stretchTopMiddle` = [TraitsGUI](#)<GModelComponent>::stretch
- qreal `_stretchBottomMiddle` = [TraitsGUI](#)<GModelComponent>::stretch

Protected Attributes inherited from [GraphicalModelDataDefinition](#)

- qreal `_width` = [TraitsGUI](#)<GModelDataDefinition>::width
- qreal `_height` = `_width` * [TraitsGUI](#)<GModelDataDefinition>::heightProportion
- unsigned int `_margin` = [TraitsGUI](#)<GModelDataDefinition>::margin
- unsigned int `_selWidth` = [TraitsGUI](#)<GModelDataDefinition>::selectionWidth
- [ModelDataDefinition](#) * `_element`
- bool `_editableInPropertyEditor` = false
- QColor `_color`
- QPointF `_oldPosition`
- qreal `_stretchPosTop` = [TraitsGUI](#)<GModelDataDefinition>::stretchPos
- qreal `_stretchPosBottom` = [TraitsGUI](#)<GModelDataDefinition>::stretchPos
- qreal `_stretchPosLeft` = [TraitsGUI](#)<GModelDataDefinition>::stretchPos
- qreal `_stretchPosRigth` = [TraitsGUI](#)<GModelDataDefinition>::stretchPos
- qreal `_stretchRigth` = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal `_stretchLeft` = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal `_stretchRigthMiddle` = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal `_stretchLeftMiddle` = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal `_stretchTop` = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal `_stretchBottom` = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal `_stretchTopMiddle` = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal `_stretchBottomMiddle` = [TraitsGUI](#)<GModelDataDefinition>::stretch

15.320.1 Detailed Description

Visual component item that extends a data definition with ports and internal data.

The scene uses this class for kernel model components that need both the visual body and the connection ports used to route edges between components.

15.320.2 Member Function Documentation

15.320.2.1 `renderContext()`

[GraphicalModelItemRenderContext](#) [GraphicalModelComponent](#)::`renderContext` () const [\[override\]](#), [\[protected\]](#), [\[virtual\]](#)

Reimplemented from [GraphicalModelDataDefinition](#).

15.320.2.2 sceneEvent()

```
bool GraphicalModelComponent::sceneEvent (
    QEvent * event ) [override], [protected], [virtual]
```

Reimplemented from [GraphicalModelDataDefinition](#).

The documentation for this class was generated from the following files:

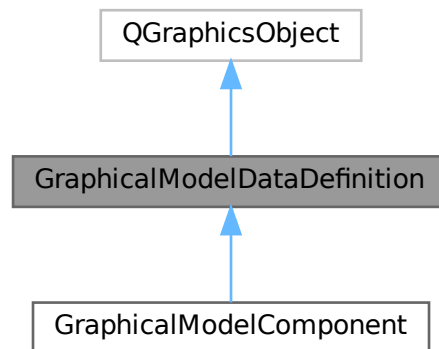
- applications/gui/genesys/graphics/GraphicalModelComponent.h
- applications/gui/genesys/graphics/GraphicalModelComponent.cpp

15.321 GraphicalModelDataDefinition Class Reference

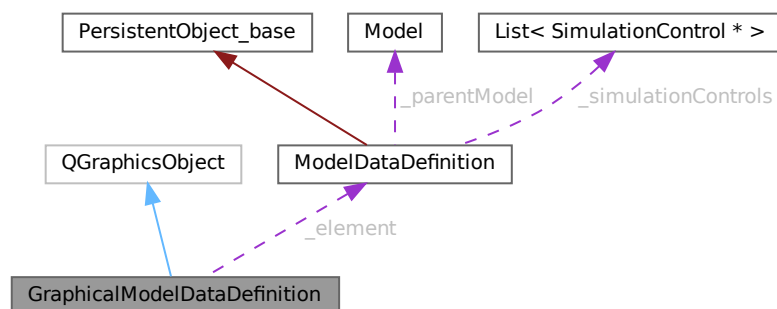
Visual representation of a model data definition in the scene.

```
#include <applications/gui/genesys/graphics/GraphicalModelDataDefinition.h>
```

Inheritance diagram for GraphicalModelDataDefinition:



Collaboration diagram for GraphicalModelDataDefinition:



Public Member Functions

- **GraphicalModelDataDefinition** ([Plugin](#) *plugin, [ModelDataDefinition](#) *element, `QPointF` position, `QColor` color=`Qt::blue`, `QGraphicsItem` *parent=`nullptr`)

- Creates one visual item bound to a kernel data definition.*
- **GraphicalModelDataDefinition** (const [GraphicalModelDataDefinition](#) &orig)
Copy constructor used by scene-copy and restore flows.
- virtual **~GraphicalModelDataDefinition** ()
Releases the graphical item and any owned helper state.
- QRectF **boundingRect** () const override
Returns the item bounds used by Qt for painting and hit testing.
- QPainterPath **shape** () const override
Returns the precise shape used for selection and collision checks.
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget) override
Paints the data-definition body, label, and selection frame.
- [ModelDataDefinition](#) * **getDataDefinition** () const
Returns the kernel data definition represented by this item.
- bool **isEditableInPropertyEditor** () const
Returns whether the property editor may edit this item in place.
- void **setEditableInPropertyEditor** (bool editable)
Marks the item as editable or read-only in the property editor.
- QPointF **getOldPosition** () const
Returns the last persisted scene position before a move operation.
- void **setOldPosition** (qreal x, qreal y)
Stores the old scene position used by undo and restoration flows.
- QColor **getColor** () const
Returns the color used to paint the item.
- void **setColor** (QColor newColor)
Sets the color used to paint the item.
- qreal **getHeight** () const
Returns the height used by the current render strategy.

Protected Member Functions

- QColor **myrgba** (uint64_t color)
- virtual [GraphicalModelItemRenderContext](#) **renderContext** () const
- virtual bool **sceneEvent** (QEvent *event) override

Protected Attributes

- qreal **_width** = [TraitsGUI](#)<GModelDataDefinition>::width
- qreal **_height** = _width * [TraitsGUI](#)<GModelDataDefinition>::heightProportion
- unsigned int **_margin** = [TraitsGUI](#)<GModelDataDefinition>::margin
- unsigned int **_selWidth** = [TraitsGUI](#)<GModelDataDefinition>::selectionWidth
- [ModelDataDefinition](#) * **_element**
- bool **_editableInPropertyEditor** = false
- QColor **_color**
- QPointF **_oldPosition**
- qreal **_stretchPosTop** = [TraitsGUI](#)<GModelDataDefinition>::stretchPos
- qreal **_stretchPosBottom** = [TraitsGUI](#)<GModelDataDefinition>::stretchPos
- qreal **_stretchPosLeft** = [TraitsGUI](#)<GModelDataDefinition>::stretchPos
- qreal **_stretchPosRigth** = [TraitsGUI](#)<GModelDataDefinition>::stretchPos
- qreal **_stretchRigth** = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal **_stretchLeft** = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal **_stretchRigthMiddle** = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal **_stretchLeftMiddle** = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal **_stretchTop** = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal **_stretchBottom** = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal **_stretchTopMiddle** = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal **_stretchBottomMiddle** = [TraitsGUI](#)<GModelDataDefinition>::stretch

15.321.1 Detailed Description

Visual representation of a model data definition in the scene.

This is the base graphical item used for non-component data definitions. Derived classes such as [GraphicalModelComponent](#) extend it with ports, internal data, and richer layout.

The documentation for this class was generated from the following files:

- applications/gui/genesys/graphics/GraphicalModelDataDefinition.h
- applications/gui/genesys/graphics/GraphicalModelDataDefinition.cpp

15.322 GraphicalModelEvent Class Reference

Lightweight event envelope used to notify GUI changes in the graphics scene.

```
#include <applications/gui/genesys/graphics/ModelGraphicsScene.h>
```

Public Types

- enum class **EventType** : int {
 CREATE = 1 , **REMOVE** = 2 , **EDIT** = 3 , **CLONE** = 4 ,
 OTHER = 9 }
- enum class **EventObjectType** : int {
 COMPONENT = 1 , **DATADEFINITION** = 2 , **CONNECTION** = 3 , **DRAWING** = 4 ,
 ANIMATION = 5 , **PLOT** = 6 , **OTHER** = 9 }

Public Member Functions

- **GraphicalModelEvent** (GraphicalModelEvent::EventType eventType, GraphicalModelEvent::EventObjectType eventObjectType, QGraphicsItem *item)
 Creates one event payload for the given item and event categories.

Public Attributes

- GraphicalModelEvent::EventType **eventType**
- GraphicalModelEvent::EventObjectType **eventObjectType**
- QGraphicsItem * **item**

15.322.1 Detailed Description

Lightweight event envelope used to notify GUI changes in the graphics scene.

The documentation for this class was generated from the following file:

- applications/gui/genesys/graphics/ModelGraphicsScene.h

15.323 GraphicalModelItemRenderContext Struct Reference

Public Types

- enum class **ItemKind** { **DataDefinition** , **Component** }

Public Attributes

- ItemKind **kind** = ItemKind::DataDefinition
- QRectF **bounds**
- QColor **fillColor**
- QString **primaryText**
- QString **secondaryText**
- QString **tertiaryText**
- QString **semanticClassName**

- QString **semanticSubtype**
- bool **selected** = false
- bool **breakpoint** = false
- qreal **width** = 0.0
- qreal **height** = 0.0
- int **penWidth** = 1
- int **raise** = 0
- int **margin** = 0
- int **selectionWidth** = 0
- qreal **stretchPosTop** = 0.5
- qreal **stretchPosBottom** = 0.5
- qreal **stretchPosLeft** = 0.5
- qreal **stretchPosRight** = 0.5
- qreal **stretchRight** = 0.0
- qreal **stretchLeft** = 0.0
- qreal **stretchRightMiddle** = 0.0
- qreal **stretchLeftMiddle** = 0.0
- qreal **stretchTop** = 0.0
- qreal **stretchBottom** = 0.0
- qreal **stretchTopMiddle** = 0.0
- qreal **stretchBottomMiddle** = 0.0
- uint64_t **borderColor** = 0
- uint64_t **raisedColor** = 0
- uint64_t **sunkenColor** = 0
- uint64_t **textColor** = 0
- uint64_t **textShadowColor** = 0
- uint64_t **selectionColor** = 0
- uint64_t **breakpointColor** = 0

The documentation for this struct was generated from the following file:

- applications/gui/genesys/graphics/GraphicalModelItemRenderStrategy.h

15.324 GraphicalModelItemRenderer Class Reference

Static Public Member Functions

- static void **paint** (QPainter *painter, const [GraphicalModelItemRenderContext](#) &context)
- static QPainterPath **shape** (const [GraphicalModelItemRenderContext](#) &context)
- static const [GraphicalModelItemRenderStrategy](#) & **currentStrategy** ()

The documentation for this class was generated from the following files:

- applications/gui/genesys/graphics/GraphicalModelItemRenderStrategy.h
- applications/gui/genesys/graphics/GraphicalModelItemRenderStrategy.cpp

15.325 GraphicalModelItemRenderStrategy Class Reference

Public Member Functions

- virtual const char * **name** () const =0
- virtual void **paint** (QPainter *painter, const [GraphicalModelItemRenderContext](#) &context) const =0
- virtual QPainterPath **shape** (const [GraphicalModelItemRenderContext](#) &context) const =0

The documentation for this class was generated from the following file:

- applications/gui/genesys/graphics/GraphicalModelItemRenderStrategy.h

15.326 GraphicalModelSerializer Class Reference

Persistence service for saving/loading graphical and textual model representations.

```
#include <applications/gui/genesys/services/GraphicalModelSerializer.h>
```

Public Member Functions

- [GraphicalModelSerializer](#) ([Simulator](#) *simulator, QWidget *ownerWidget, QPlainTextEdit *modelTextEditor, [ModelGraphicsView](#) *graphicsView, QSlider *zoomSlider, QAction *actionShowGrid, QAction *actionShowRule, QAction *actionShowSnap, QAction *actionShowGuides, QAction *actionShowInternalElements, QAction *actionShowEditableElements, QAction *actionShowAttachedElements, QAction *actionShowRecursiveElements, QTextEdit *console, QString *modelName, std::function< void()> clearModelEditors, std::function< void()> rebuildGraphicalModelFromModel, std::function< void()> applyShowInternalElements, std::function< void()> applyShowEditableElements, std::function< void()> applyShowAttachedElements)
- *Creates the persistence service that backs [MainWindow](#) lifecycle wrappers.*
- bool [saveTextModel](#) (QFile *saveFile, const QString &data) const
- *Saves textual model language using the established compatibility format.*
- bool [saveGraphicalModel](#) (const QString &filename) const
- *Saves full graphical .gui persistence sections without format changes.*
- [Model](#) * [loadGraphicalModel](#) (const std::string &filename) const
- *Loads .gui/.gen model files and restores persisted graphical/UI state.*

15.326.1 Detailed Description

Persistence service for saving/loading graphical and textual model representations.

This Phase-2 extraction removes serialization concerns from [MainWindow](#) while preserving file compatibility through wrapper delegation. It bridges GUI state (scene/view/actions/editors) and kernel model loading/saving workflows. Responsibilities:

- save textual model content with the existing line-based format;
- save full graphical .gui state sections used by legacy compatibility;
- load persisted model files and restore graphical/editor UI state via callbacks.

Boundaries:

- it does not own widgets or simulator lifetime;
- it does not run simulation commands or trace rendering;
- it coordinates persistence/serialization only.

15.326.2 Constructor & Destructor Documentation

15.326.2.1 GraphicalModelSerializer()

```
GraphicalModelSerializer::GraphicalModelSerializer (
    Simulator * simulator,
    QWidget * ownerWidget,
    QPlainTextEdit * modelTextEditor,
    ModelGraphicsView * graphicsView,
    QSlider * zoomSlider,
    QAction * actionShowGrid,
    QAction * actionShowRule,
    QAction * actionShowSnap,
    QAction * actionShowGuides,
    QAction * actionShowInternalElements,
    QAction * actionShowEditableElements,
```

```

    QAction * actionShowAttachedElements,
    QAction * actionShowRecursiveElements,
    QTextEdit * console,
    QString * modelFilename,
    std::function< void()> clearModelEditors,
    std::function< void()> rebuildGraphicalModelFromModel,
    std::function< void()> applyShowInternalElements,
    std::function< void()> applyShowEditableElements,
    std::function< void()> applyShowAttachedElements )

```

Creates the persistence service that backs [MainWindow](#) lifecycle wrappers.
Builds the serializer with explicit, narrow dependencies from [MainWindow](#).

15.326.3 Member Function Documentation

15.326.3.1 loadGraphicalModel()

```

Model * GraphicalModelSerializer::loadGraphicalModel (
    const std::string & filename ) const

```

Loads `.gui/.gen` model files and restores persisted graphical/UI state.
Loads `.gui` or `.gen` model files and restores persisted GUI state.

Returns

Loaded kernel model pointer or nullptr when loading fails.

15.326.3.2 saveGraphicalModel()

```

bool GraphicalModelSerializer::saveGraphicalModel (
    const QString & filename ) const

```

Saves full graphical `.gui` persistence sections without format changes.
Persists the complete graphical model, including view options and overlays.

Returns

true when serialization succeeds.

15.326.3.3 saveTextModel()

```

bool GraphicalModelSerializer::saveTextModel (
    QFile * saveFile,
    const QString & data ) const

```

Saves textual model language using the established compatibility format.
Writes model text line by line using the legacy persistence format.

Returns

true when write succeeds.

The documentation for this class was generated from the following files:

- applications/gui/genesys/services/GraphicalModelSerializer.h
- applications/gui/genesys/services/GraphicalModelSerializer.cpp

15.327 GraphvizModelExporter Class Reference

Service that builds DOT content and renders model images for the GUI.

```
#include <applications/gui/genesys/services/GraphvizModelExporter.h>
```

Public Member Functions

- **GraphvizModelExporter** ([Simulator](#) *simulator, QLabel *modelGraphicLabel, QCheckBox *showStatistics, QCheckBox *showEditable, QCheckBox *showShared, QCheckBox *showRecursive, QCheckBox *show↔ Levels, std::function< bool()> ensureModelSynchronized)
Creates the Graphviz exporter service used by [MainWindow](#) compatibility wrappers.
- std::string [adjustDotName](#) (std::string name) const
Normalizes identifiers to Graphviz-safe names.
- void **insertTextInDot** (std::string text, unsigned int compLevel, unsigned int compRank, std::map< unsigned int, std::map< unsigned int, std::list< std::string > * > * > *dotmap, bool isNode=false) const
Inserts a DOT line into the hierarchical rank/level map used for image generation.
- void **recursiveCreateModelGraphicPicture** ([ModelDataDefinition](#) *componentOrData, std::list< [ModelDataDefinition](#) * > *visited, std::map< unsigned int, std::map< unsigned int, std::list< std::string > * > * > *dotmap) const
Recursively traverses model/data nodes to build DOT content fragments.
- bool [createModelImage](#) () const
Generates and displays the model image after synchronization preconditions.

15.327.1 Detailed Description

Service that builds DOT content and renders model images for the GUI.

This service encapsulates Graphviz-oriented model representation logic extracted from [MainWindow](#). It keeps image-generation behavior stable while [MainWindow](#) exposes compatibility wrappers used by actions and tabs.

Responsibilities:

- normalize identifiers for DOT compatibility;
- build hierarchical DOT fragments from model/data-definition traversal;
- generate and display model images after ensuring text/model synchronization.

Boundaries:

- it does not own model synchronization policy (uses injected callback);
- it does not manage simulation commands, traces, or lifecycle actions;
- it provides export/representation service behavior, not controller orchestration.

15.327.2 Member Function Documentation

15.327.2.1 [adjustDotName\(\)](#)

```
std::string GraphvizModelExporter::adjustDotName (
    std::string name ) const
```

Normalizes identifiers to Graphviz-safe names.

Parameters

<i>name</i>	Raw label/name from model representation.
-------------	---

Returns

DOT-safe identifier preserving compatibility naming intent.

15.327.2.2 [createModelImage\(\)](#)

```
bool GraphvizModelExporter::createModelImage ( ) const
```

Generates and displays the model image after synchronization preconditions.

Returns

true when DOT generation and image rendering complete successfully.

The documentation for this class was generated from the following files:

- applications/gui/genesys/services/GraphvizModelExporter.h
- applications/gui/genesys/services/GraphvizModelExporter.cpp

15.328 ModelGraphicsScene::GRID Struct Reference

Public Member Functions

- void **clear** ()

Public Attributes

- unsigned int **interval**
- QPen **pen**
- std::list< QGraphicsLineItem * > * **lines**
- bool **visible**

The documentation for this struct was generated from the following files:

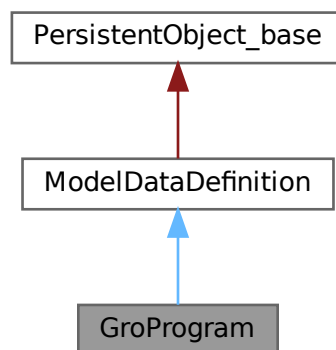
- applications/gui/genesys/graphics/ModelGraphicsScene.h
- applications/gui/genesys/graphics/ModelGraphicsScene.cpp

15.329 GroProgram Class Reference

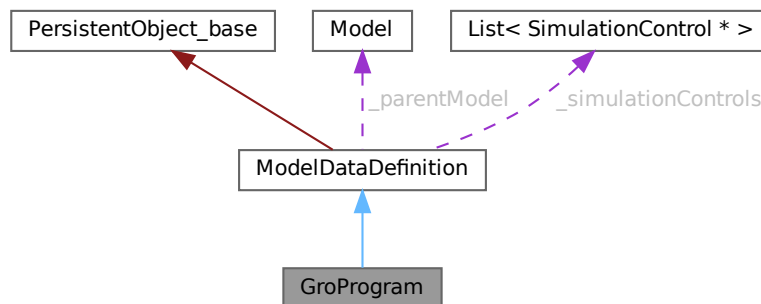
Reusable Gro source program attached to a GenESyS model.

```
#include <plugins/data/BiochemicalSimulation/GroProgram.h>
```

Inheritance diagram for GroProgram:



Collaboration diagram for GroProgram:



Public Member Functions

- **GroProgram** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- void **setSourceCode** (std::string sourceCode)
Replaces the stored Gro source text.
- void **setSourceCodeProperty** ([SourceCodeString](#) sourceCode)
Replaces the stored Gro source text through the property-editor wrapper type.
- std::string **getSourceCode** () const
Returns the stored Gro source text.
- [SourceCodeString](#) **getSourceCodeProperty** () const
Returns the stored Gro source text wrapped for the property editor.
- bool **createDefaultGroProgram** (const std::string &filename="")
Creates a small starter Gro program and optionally writes it to a file.
- bool **validateSyntax** (std::string &errorMessage) const
Performs a permissive lexical sanity check for the stored source.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.

- `const std::string & getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from ModelDataDefinition

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- `static void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.
- `static bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Validates a data definition instance.

- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)

- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.329.1 Detailed Description

Reusable Gro source program attached to a GenESyS model.

[GroProgram](#) is intentionally a [ModelDataDefinition](#) so several biological components can share the same Gro source text. This first implementation only stores the source and performs a small lexical sanity check; complete Gro parsing and executable semantics belong to later plugin-side helper classes.

15.329.2 Member Function Documentation

15.329.2.1 _check()

```
bool GroProgram::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.329.2.2 _loadInstance()

```
bool GroProgram::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.329.2.3 `_saveInstance()`

```
void GroProgram::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.329.2.4 `show()`

```
std::string GroProgram::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- `plugins/data/BiochemicalSimulation/GroProgram.h`
- `plugins/data/BiochemicalSimulation/GroProgram.cpp`

15.330 GroProgramAst Struct Reference

Minimal parsed representation of a Gro source program.

```
#include <plugins/data/BiochemicalSimulation/GroProgramAst.h>
```

Classes

- struct [NamedProgram](#)
- struct [Statement](#)

Public Types

- enum class **SourceForm** { **RawStatements** , **ProgramBlock** }

Public Member Functions

- bool **isProgramBlock** () const
- bool **hasNamedPrograms** () const

Public Attributes

- SourceForm **sourceForm** = SourceForm::RawStatements
- std::string **programName** = ""
- std::vector< std::string > **programParameters**
- std::string **bodySource** = ""
- std::vector< [Statement](#) > **statements**
- std::vector< [NamedProgram](#) > **namedPrograms**

15.330.1 Detailed Description

Minimal parsed representation of a Gro source program.

This AST intentionally models only the first integration target: either a recognized `program name() { ... }` block or a permissive raw statement list. Execution semantics and typed Gro constructs belong to later phases. The documentation for this struct was generated from the following file:

- `plugins/data/BiochemicalSimulation/GroProgramAst.h`

15.331 GroProgramCompiler Class Reference

Converts parsed Gro AST into the first non-executable semantic IR.

```
#include <plugins/data/BiochemicalSimulation/GroProgramCompiler.h>
```

Public Member Functions

- `GroProgramIr compile` (const [GroProgramAst](#) &ast) const
Builds a lightweight semantic IR from a parsed Gro AST.

15.331.1 Detailed Description

Converts parsed Gro AST into the first non-executable semantic IR.

The documentation for this class was generated from the following files:

- `plugins/data/BiochemicalSimulation/GroProgramCompiler.h`
- `plugins/data/BiochemicalSimulation/GroProgramCompiler.cpp`

15.332 GroProgramIr Struct Reference

Initial semantic representation compiled from [GroProgramAst](#).

```
#include <plugins/data/BiochemicalSimulation/GroProgramIr.h>
```

Classes

- struct [Command](#)
- struct [NamedProgramDefinition](#)

Public Member Functions

- bool `isProgramBlock` () const
- bool `hasNamedProgram` (const std::string &name) const

Public Attributes

- `GroProgramAst::SourceForm sourceForm` = `GroProgramAst::SourceForm::RawStatements`
- std::string `programName` = ""
- std::vector< [Command](#) > `commands`
- std::map< std::string, [NamedProgramDefinition](#) > `namedPrograms`

15.332.1 Detailed Description

Initial semantic representation compiled from [GroProgramAst](#).

The IR keeps the parsed program in a runtime-oriented shape. It can represent function calls, assignments, and basic conditional blocks without pushing any knowledge into the simulator kernel itself.

The documentation for this struct was generated from the following file:

- `plugins/data/BiochemicalSimulation/GroProgramIr.h`

15.333 GroProgramParser Class Reference

Plugin-side parser boundary for reusable Gro source programs.

```
#include <plugins/data/BiochemicalSimulation/GroProgramParser.h>
```

Classes

- struct [Result](#)

Public Member Functions

- [Result](#) **parse** (const std::string &sourceCode) const
Parses a Gro source string using the current lexical boundary.

15.333.1 Detailed Description

Plugin-side parser boundary for reusable Gro source programs.

This helper currently preserves the permissive lexical validation that used to live inside [GroProgram](#). It is intentionally small so later phases can grow a real grammar, AST, and semantic checks without changing [GroProgram](#) storage.

The documentation for this class was generated from the following files:

- plugins/data/BiochemicalSimulation/GroProgramParser.h
- plugins/data/BiochemicalSimulation/GroProgramParser.cpp

15.334 GroProgramRuntime Class Reference

Executes the supported Gro IR commands without simulator binding.

```
#include <plugins/data/BiochemicalSimulation/GroProgramRuntime.h>
```

Classes

- struct [ColonyMutation](#)
- struct [ExecutionResult](#)
- struct [MotionMutation](#)
- struct [PopulationMutation](#)
- struct [SignalMutation](#)

Public Types

- enum class **PopulationMutationType** { **Grow** , **Divide** , **Die** , **SetPopulation** }
- enum class **SignalMutationType** { **Emit** , **Consume** , **Set** }
- enum class **MotionMutationType** { **Run** , **Tumble** }

Public Member Functions

- [ExecutionResult](#) **execute** (const [GroProgramIr](#) &ir, [GroProgramRuntimeState](#) &state) const
Executes supported commands and reports unsupported commands.

Static Public Member Functions

- static bool **evaluateExpression** (const std::string &expressionText, const [GroProgramRuntimeState](#) &state, double &value, std::string &errorMessage)
Evaluates one scalar Gro expression against one runtime state.

15.334.1 Detailed Description

Executes the supported Gro IR commands without simulator binding.

This helper is intentionally independent from [BacteriaColony](#) and the GenESyS event scheduler. It now supports persistent scalar variables, arithmetic expressions, and basic `if/else` control flow while still keeping the runtime local to the biological plugins.

The documentation for this class was generated from the following files:

- `plugins/data/BiochemicalSimulation/GroProgramRuntime.h`
- `plugins/data/BiochemicalSimulation/GroProgramRuntime.cpp`

15.335 GroProgramRuntimeState Struct Reference

Mutable state used by the plugin-side Gro runtime helper.

```
#include <plugins/data/BiochemicalSimulation/GroProgramRuntime.h>
```

Public Attributes

- double **colonyTime** = 0.0
- double **simulationStep** = 1.0
- unsigned int **populationSize** = 1
- unsigned int **tickCount** = 0
- `std::map< std::string, double >` **contextVariables**
- `std::map< std::string, double >` **variables**

15.335.1 Detailed Description

Mutable state used by the plugin-side Gro runtime helper.

The documentation for this struct was generated from the following file:

- `plugins/data/BiochemicalSimulation/GroProgramRuntime.h`

15.336 GroupItem Struct Reference

Public Attributes

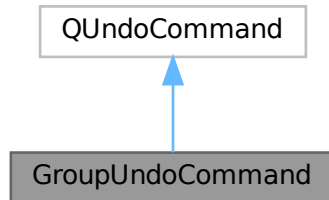
- `QGraphicsItemGroup *` **group**
- `QList< ComponentItem >` **myComponentItems**
- `QPointF` **initialPosition**

The documentation for this struct was generated from the following file:

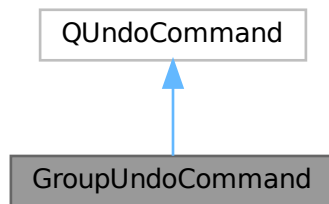
- `applications/gui/genesys/actions/AddUndoCommand.h`

15.337 GroupUndoCommand Class Reference

Inheritance diagram for GroupUndoCommand:



Collaboration diagram for GroupUndoCommand:



Public Member Functions

- **GroupUndoCommand** (QList< [GraphicalModelComponent](#) * > graphicalComponents, [ModelGraphicsScene](#) *scene, QUndoCommand *parent=nullptr)
- void **undo** () override
- void **redo** () override

The documentation for this class was generated from the following files:

- applications/gui/genesys/actions/GroupUndoCommand.h
- applications/gui/genesys/actions/GroupUndoCommand.cpp

15.338 testing::internal::GTestFlagSaver Class Reference

The documentation for this class was generated from the following file:

- gtest/src/gtest-internal-inl.h

15.339 testing::internal::GTestLog Class Reference

Public Member Functions

- **GTestLog** (GTestLogSeverity severity, const char *file, int line)

- `::std::ostream & GetStream ()`

The documentation for this class was generated from the following files:

- `gtest/gtest/internal/gtest-port.h`
- `gtest/src/gtest-port.cc`

15.340 `testing::internal::GTestMutexLock` Class Reference

Public Member Functions

- `GTestMutexLock (Mutex *)`

The documentation for this class was generated from the following file:

- `gtest/gtest/internal/gtest-port.h`

15.341 `GuiActionContribution` Struct Reference

Public Attributes

- `std::string id`
- `std::string text`
- `std::string menuPath`
- `std::string toolBarId`
- `std::string statusTip`
- `std::string shortcut`
- `std::function< bool(const GuiExtensionRuntimeContext &)> isVisible`
- `std::function< bool(const GuiExtensionRuntimeContext &)> isEnabled`
- `std::function< void(const GuiExtensionRuntimeContext &)> trigger`

The documentation for this struct was generated from the following file:

- `applications/gui/genesys/extensions/GuiExtensionContracts.h`

15.342 `GuiDockContribution` Struct Reference

Public Attributes

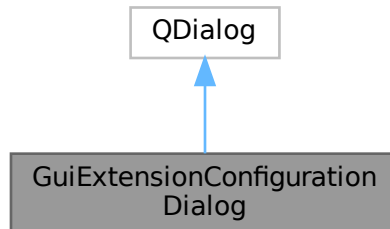
- `std::string id`
- `std::string title`
- `std::string areaHint`
- `std::function< bool(const GuiExtensionRuntimeContext &)> isVisible`
- `std::function< QWidget *(const GuiExtensionRuntimeContext &)> createWidget`

The documentation for this struct was generated from the following file:

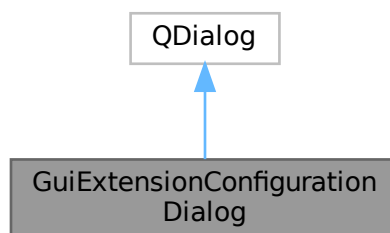
- `applications/gui/genesys/extensions/GuiExtensionContracts.h`

15.343 GuiExtensionConfigurationDialog Class Reference

Inheritance diagram for GuiExtensionConfigurationDialog:



Collaboration diagram for GuiExtensionConfigurationDialog:



Public Member Functions

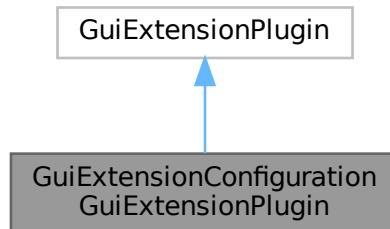
- **GuiExtensionConfigurationDialog** (QWidget *parent, std::vector< const [GuiExtensionPlugin](#) * > plugins, std::function< void()> onSaved={})

The documentation for this class was generated from the following files:

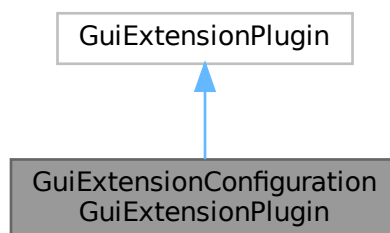
- applications/gui/genesys/extensions/GuiExtensionConfigurationDialog.h
- applications/gui/genesys/extensions/GuiExtensionConfigurationDialog.cpp

15.344 GuiExtensionConfigurationGuiExtensionPlugin Class Reference

Inheritance diagram for GuiExtensionConfigurationGuiExtensionPlugin:



Collaboration diagram for GuiExtensionConfigurationGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId ()` const override
- `std::string displayName ()` const override
- `std::string version ()` const override
- `std::vector< std::string > requiredModelPlugins ()` const override
- `void registerContributions (GuiExtensionRegistry *registry)` const override

15.344.1 Member Function Documentation

15.344.1.1 displayName()

```
std::string GuiExtensionConfigurationGuiExtensionPlugin::displayName ( ) const [inline],
[override], [virtual]
Implements GuiExtensionPlugin.
```

15.344.1.2 extensionId()

```
std::string GuiExtensionConfigurationGuiExtensionPlugin::extensionId ( ) const [inline],
[override], [virtual]
Implements GuiExtensionPlugin.
```

15.344.1.3 registerContributions()

```
void GuiExtensionConfigurationGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.344.1.4 requiredModelPlugins()

```
std::vector< std::string > GuiExtensionConfigurationGuiExtensionPlugin::requiredModelPlugins (
) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.344.1.5 version()

```
std::string GuiExtensionConfigurationGuiExtensionPlugin::version ( ) const [inline], [override],
[virtual]
```

Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

- applications/gui/genesys/extensions/GuiExtensionConfigurationGuiExtensionPlugin.cpp

15.345 GuiExtensionManager Class Reference

Public Member Functions

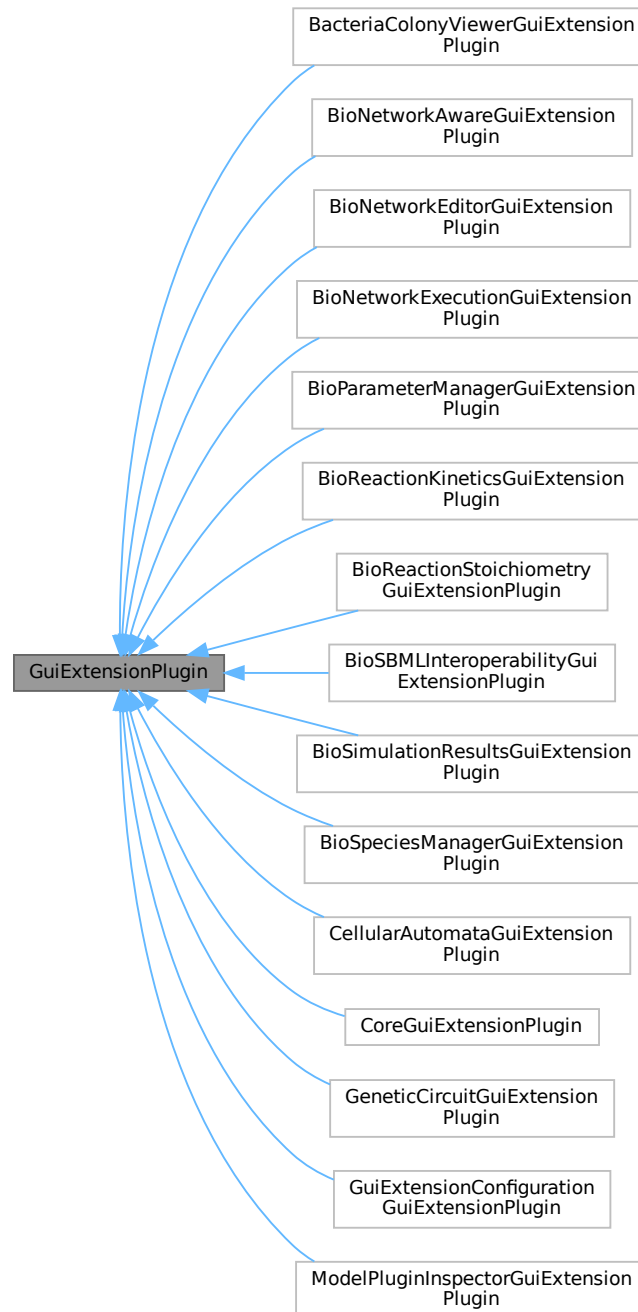
- **GuiExtensionManager** (QMainWindow *mainWindow)
- void **setPlugins** (std::vector< const [GuiExtensionPlugin](#) * > plugins)
- void **setLoadedModelPluginIds** (std::vector< std::string > modelPluginIds)
- void **rebuild** (const [GuiExtensionRuntimeContext](#) &context)
- void **clear** ()

The documentation for this class was generated from the following files:

- applications/gui/genesys/extensions/GuiExtensionManager.h
- applications/gui/genesys/extensions/GuiExtensionManager.cpp

15.346 GuiExtensionPlugin Class Reference

Inheritance diagram for GuiExtensionPlugin:



Public Member Functions

- virtual std::string **extensionId** () const =0
- virtual std::string **displayName** () const =0
- virtual std::string **version** () const =0
- virtual std::vector< std::string > **requiredModelPlugins** () const =0
- virtual void **registerContributions** ([GuiExtensionRegistry](#) *registry) const =0

The documentation for this class was generated from the following file:

- applications/gui/genesys/extensions/GuiExtensionContracts.h

15.347 GuiExtensionPluginAutoRegistrar Class Reference

Public Member Functions

- **GuiExtensionPluginAutoRegistrar** (const [GuiExtensionPlugin](#) *plugin)

The documentation for this class was generated from the following files:

- applications/gui/genesys/extensions/GuiExtensionPluginCatalog.h
- applications/gui/genesys/extensions/GuiExtensionPluginCatalog.cpp

15.348 GuiExtensionPluginCatalog Class Reference

Static Public Member Functions

- static const std::vector< const [GuiExtensionPlugin](#) * > & **plugins** ()
- static std::vector< const [GuiExtensionPlugin](#) * > **resolvedPlugins** ()
- static [GuiExtensionPresentationMetadata](#) **presentationMetadata** (const std::string &extensionId)
- static void **registerPlugin** (const [GuiExtensionPlugin](#) *plugin)

The documentation for this class was generated from the following files:

- applications/gui/genesys/extensions/GuiExtensionPluginCatalog.h
- applications/gui/genesys/extensions/GuiExtensionPluginCatalog.cpp

15.349 GuiExtensionPresentationMetadata Struct Reference

Public Attributes

- std::string **category**
- std::string **group**
- int **priority** = 0

The documentation for this struct was generated from the following file:

- applications/gui/genesys/extensions/GuiExtensionPluginCatalog.h

15.350 GuiExtensionRegistry Class Reference

Public Member Functions

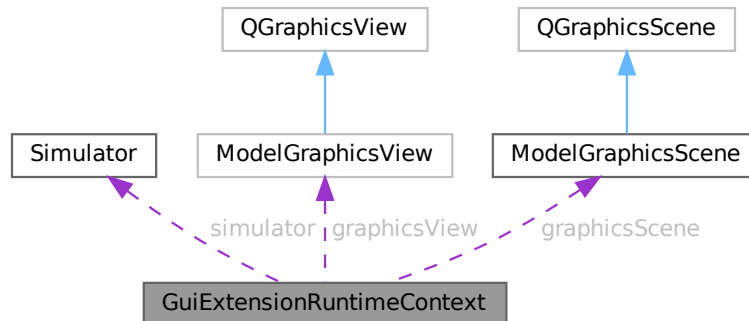
- void **addAction** ([GuiActionContribution](#) contribution)
- void **addDock** ([GuiDockContribution](#) contribution)
- void **addWindow** ([GuiWindowContribution](#) contribution)
- const std::vector< [GuiActionContribution](#) > & **actions** () const
- const std::vector< [GuiDockContribution](#) > & **docks** () const
- const std::vector< [GuiWindowContribution](#) > & **windows** () const
- void **clear** ()

The documentation for this class was generated from the following file:

- applications/gui/genesys/extensions/GuiExtensionContracts.h

15.351 GuiExtensionRuntimeContext Struct Reference

Collaboration diagram for GuiExtensionRuntimeContext:



Public Attributes

- [Simulator](#) * **simulator** = nullptr
- QMainWindow * **mainWindow** = nullptr
- Ui::MainWindow * **ui** = nullptr
- [ModelGraphicsView](#) * **graphicsView** = nullptr
- [ModelGraphicsScene](#) * **graphicsScene** = nullptr

The documentation for this struct was generated from the following file:

- applications/gui/genesys/extensions/GuiExtensionContracts.h

15.352 GuiScopeTrace Class Reference

Emits a scoped trace entry for a GUI method and measures its runtime.

```
#include <applications/gui/genesys/GuiScopeTrace.h>
```

Public Member Functions

- [GuiScopeTrace](#) (const char *methodName, const void *thisPtr)
Starts a scoped GUI trace entry for a critical method invocation.
- [~GuiScopeTrace](#) ()
Ends the scoped trace and logs the elapsed execution time.

15.352.1 Detailed Description

Emits a scoped trace entry for a GUI method and measures its runtime.

Instances are usually created on the stack at the beginning of a method so the constructor logs entry and the destructor logs the elapsed time when the scope exits.

15.352.2 Constructor & Destructor Documentation

15.352.2.1 GuiScopeTrace()

```
GuiScopeTrace::GuiScopeTrace (
    const char * methodName,
    const void * thisPtr )
```


Starts a scoped GUI trace entry for a critical method invocation.

Parameters

<i>methodName</i>	Name of the traced method.
<i>thisPtr</i>	Pointer to the current object, when available.

The documentation for this class was generated from the following files:

- applications/gui/genesys/GuiScopeTrace.h
- applications/gui/genesys/GuiScopeTrace.cpp

15.353 GuiThemeManager Class Reference

Applies the visual theme used by the Qt application and model views.

```
#include <applications/gui/genesys/guithememanager.h>
```

Static Public Member Functions

- static void [applyApplicationTheme](#) (QApplication *application)
Applies application-wide theme settings to the Qt application object.
- static void [applyModelGraphicsTheme](#) ([ModelGraphicsView](#) *graphicsView)
Applies the model-graphics theme to a specific graphics view.

15.353.1 Detailed Description

Applies the visual theme used by the Qt application and model views.

The theme manager centralizes palette-level adjustments so the application and graphics scene remain visually consistent when preferences change.

15.353.2 Member Function Documentation

15.353.2.1 [applyApplicationTheme\(\)](#)

```
void GuiThemeManager::applyApplicationTheme (
    QApplication * application ) [static]
```

Applies application-wide theme settings to the Qt application object.

Parameters

<i>application</i>	Target QApplication instance.
--------------------	-------------------------------

15.353.2.2 [applyModelGraphicsTheme\(\)](#)

```
void GuiThemeManager::applyModelGraphicsTheme (
    ModelGraphicsView * graphicsView ) [static]
```

Applies the model-graphics theme to a specific graphics view.

Parameters

<i>graphicsView</i>	Target graphics view.
---------------------	-----------------------

The documentation for this class was generated from the following files:

- applications/gui/genesys/guithememanager.h
- applications/gui/genesys/guithememanager.cpp

15.354 GuiWindowContribution Struct Reference

Public Attributes

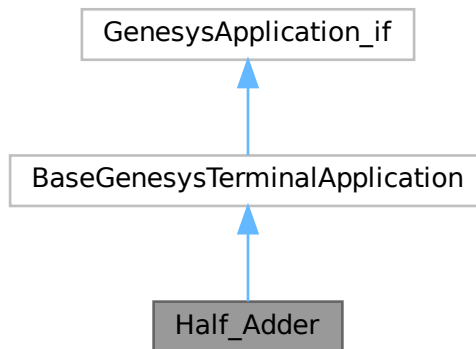
- `std::string` **id**
- `std::string` **menuPath**
- `std::string` **text**
- `std::function< bool(const GuiExtensionRuntimeContext &)>` **isVisible**
- `std::function< void(const GuiExtensionRuntimeContext &)>` **open**

The documentation for this struct was generated from the following file:

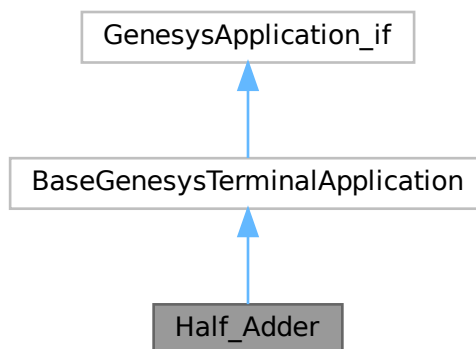
- `applications/gui/genesys/extensions/GuiExtensionContracts.h`

15.355 Half_Adder Class Reference

Inheritance diagram for `Half_Adder`:



Collaboration diagram for `Half_Adder`:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

15.355.1 Member Function Documentation

15.355.1.1 [main\(\)](#)

```
int Half_Adder::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

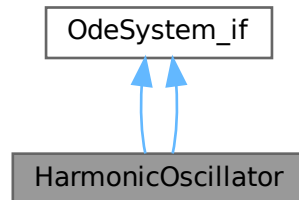
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

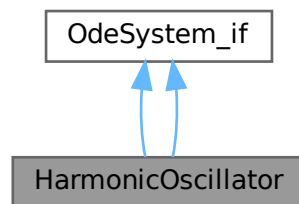
- applications/modelSpecific/teaching/Half_Adder.h
- applications/modelSpecific/teaching/Half_Adder.cpp

15.356 HarmonicOscillator Class Reference

Inheritance diagram for HarmonicOscillator:



Collaboration diagram for HarmonicOscillator:



Public Member Functions

- unsigned int `dimension` () const override
- void `evaluate` (double t, const double *y, double *dydt) const override

15.356.1 Member Function Documentation

15.356.1.1 `dimension()`

```
unsigned int HarmonicOscillator::dimension ( ) const [inline], [override], [virtual]
```

Implements [OdeSystem_if](#).

15.356.1.2 `evaluate()`

```
void HarmonicOscillator::evaluate (
    double t,
    const double * y,
    double * dydt ) const [inline], [override], [virtual]
```

Implements [OdeSystem_if](#).

The documentation for this class was generated from the following file:

- tests/test_ode_solver_numerical_validation.cpp

15.357 testing::internal::HasDebugStringAndShortDebugString< T > Class Template Reference

Static Public Attributes

- static constexpr bool [value](#)

15.357.1 Member Data Documentation

15.357.1.1 value

```
template<typename T >
constexpr bool testing::internal::HasDebugStringAndShortDebugString< T >::value [static],
[constexpr]
```

Initial value:

```
=
```

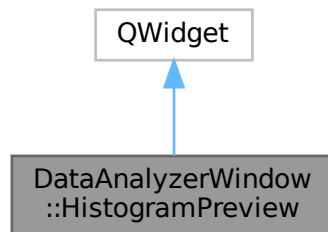
```
HasDebugStringType::value && HasShortDebugStringType::value
```

The documentation for this class was generated from the following file:

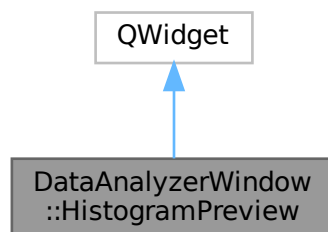
- gtest/gtest/internal/gtest-internal.h

15.358 DataAnalyzerWindow::HistogramPreview Class Reference

Inheritance diagram for DataAnalyzerWindow::HistogramPreview:



Collaboration diagram for DataAnalyzerWindow::HistogramPreview:



Public Member Functions

- **HistogramPreview** (QWidget *parent=nullptr)
- void **setValues** (const QList< double > &values)

Protected Member Functions

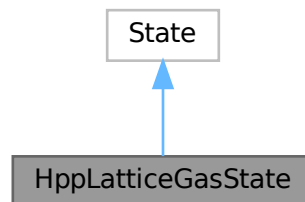
- void **paintEvent** (QPaintEvent *) override

The documentation for this class was generated from the following file:

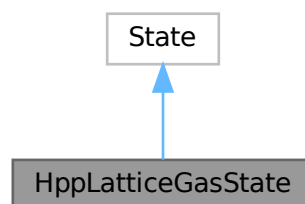
- applications/gui/genesys/tools/dataanalyzer/DataAnalyzerWindow.cpp

15.359 HppLatticeGasState Class Reference

Inheritance diagram for HppLatticeGasState:



Collaboration diagram for HppLatticeGasState:

**Public Types**

- enum **Direction** : size_t { **North** = 0 , **East** = 1 , **South** = 2 , **West** = 3 }

Public Member Functions

- **HppLatticeGasState** (std::array< bool, 4 > occupations)
- std::unique_ptr< [State](#) > **clone** () const override
- bool **equals** (const [State](#) &other) const override
- std::string **toString** () const override

- `std::string typeName ()` const override
- `long getValue ()` const override
- `void setValue (long value)` override
- `bool has (Direction direction)` const
- `void set (Direction direction, bool occupied)`
- `const std::array< bool, 4 > & occupations ()` const

Public Member Functions inherited from [State](#)

- **State** (long value)
- **State** (const [State](#) &orig)
- virtual `std::string show ()` const
- `template<typename T > const T * as ()` const noexcept
- `template<typename T > T * as ()` noexcept

Static Public Member Functions

- static `long maskFromOccupations (const std::array< bool, 4 > &occupations)`

Additional Inherited Members

Protected Attributes inherited from [State](#)

- `long _value`

15.359.1 Member Function Documentation

15.359.1.1 clone()

`std::unique_ptr< State > HppLatticeGasState::clone ()` const [inline], [override], [virtual]
Reimplemented from [State](#).

15.359.1.2 equals()

`bool HppLatticeGasState::equals (`
 `const State & other )` const [inline], [override], [virtual]
 Reimplemented from [State](#).

15.359.1.3 getValue()

`long HppLatticeGasState::getValue ()` const [inline], [override], [virtual]
Reimplemented from [State](#).

15.359.1.4 setValue()

`void HppLatticeGasState::setValue (`
 `long value )` [inline], [override], [virtual]
 Reimplemented from [State](#).

15.359.1.5 toString()

`std::string HppLatticeGasState::toString ()` const [inline], [override], [virtual]
Reimplemented from [State](#).

15.359.1.6 typeName()

```
std::string HppLatticeGasState::typeName ( ) const [inline], [override], [virtual]
```

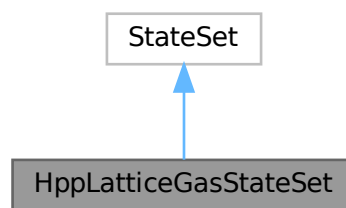
Reimplemented from [State](#).

The documentation for this class was generated from the following file:

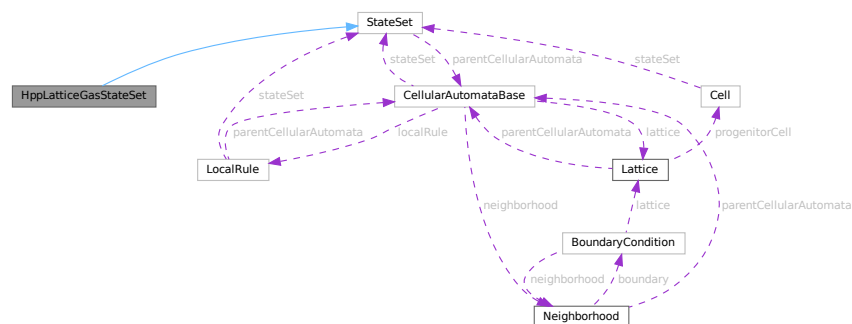
- plugins/components/ModalModel/CellularAutomata/State.h

15.360 HppLatticeGasStateSet Class Reference

Inheritance diagram for HppLatticeGasStateSet:



Collaboration diagram for HppLatticeGasStateSet:



Public Member Functions

- **HppLatticeGasStateSet** ([CellularAutomataBase](#) *parentCellularAutomata)
- bool [contains](#) (const [State](#) &state) const override
- std::unique_ptr< [State](#) > [createDefaultState](#) () const override
- std::unique_ptr< [State](#) > [parseState](#) (const std::string &text) const override
- bool [isFinite](#) () const override
- bool [isDiscrete](#) () const override
- bool [isComposite](#) () const override
- std::vector< std::unique_ptr< [State](#) > > [enumerateStates](#) () const override
- std::string [name](#) () const override

Public Member Functions inherited from [StateSet](#)

- **StateSet** ([CellularAutomataBase](#) *parentCellularAutomata)
- **StateSet** (const [StateSet](#) &orig)
- virtual bool **isContinuous** () const

Additional Inherited Members

Protected Attributes inherited from [StateSet](#)

- [CellularAutomataBase](#) * parentCellularAutomata

15.360.1 Member Function Documentation

15.360.1.1 contains()

```
bool HppLatticeGasStateSet::contains (
    const State & state ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

15.360.1.2 createDefaultState()

```
std::unique_ptr< State > HppLatticeGasStateSet::createDefaultState ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

15.360.1.3 enumerateStates()

```
std::vector< std::unique_ptr< State > > HppLatticeGasStateSet::enumerateStates ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

15.360.1.4 isComposite()

```
bool HppLatticeGasStateSet::isComposite ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

15.360.1.5 isDiscrete()

```
bool HppLatticeGasStateSet::isDiscrete ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

15.360.1.6 isFinite()

```
bool HppLatticeGasStateSet::isFinite ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

15.360.1.7 name()

```
std::string HppLatticeGasStateSet::name ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

15.360.1.8 parseState()

```
std::unique_ptr< State > HppLatticeGasStateSet::parseState (
    const std::string & text ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

The documentation for this class was generated from the following file:

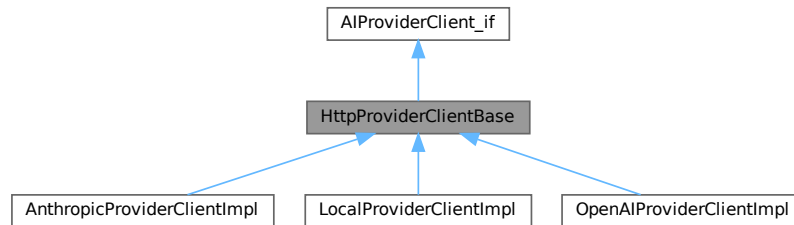
- plugins/components/ModalModel/CellularAutomata/StateSet.h

15.361 HttpProviderClientBase Class Reference

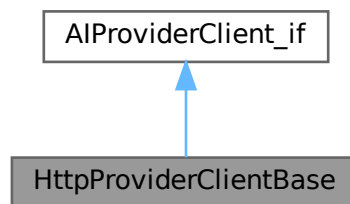
Abstract base class for HTTP-based AI provider clients.

```
#include <tools/AIAssistant/HttpProviderClientBase.h>
```

Inheritance diagram for HttpProviderClientBase:



Collaboration diagram for HttpProviderClientBase:



Public Member Functions

- **HttpProviderClientBase** (const [HttpProviderClientBase](#) &)=delete
- **HttpProviderClientBase & operator=** (const [HttpProviderClientBase](#) &)=delete
- bool [isAvailable](#) () const override
Returns true when this client can attempt to send requests.
- void [setBaseUrl](#) (const std::string &url) override
Sets the base URL for the provider API endpoint.
- std::string [getBaseUrl](#) () const override
Returns the currently configured base URL.
- void [setTimeoutSeconds](#) (unsigned int seconds) override
Sets the HTTP request timeout in seconds.
- unsigned int [getTimeoutSeconds](#) () const override
Returns the configured HTTP request timeout.
- void [setApiKey](#) (const std::string &apiKey) override
Stores the API key in memory.
- void [clearApiKey](#) () override
Overwrites (best-effort) and clears the in-memory API key.
- bool [hasApiKey](#) () const override
Returns true when a non-empty API key is held.

- [ProviderResponse](#) `send` (const [ProviderRequest](#) &request) override
Executes the HTTP request via curl and returns the provider response.

Public Member Functions inherited from [AIProviderClient_if](#)

- virtual std::string `getProviderName` () const =0
Returns a human-readable provider name (e.g. "OpenAI", "Anthropic").

Static Public Member Functions

- static std::string `_escapeJsonString` (const std::string &s)
Escapes a plain string for safe embedding inside a JSON string value.
- static std::string `_extractJsonString` (const std::string &json, const std::string &key)
Extracts the value of a top-level JSON string field by key. Returns an empty string when not found or value is not a quoted string.
- static int `_extractJsonInt` (const std::string &json, const std::string &key)
Extracts the value of a top-level JSON integer field by key. Returns 0 when not found.
- static double `_extractJsonDouble` (const std::string &json, const std::string &key)
Extracts the value of a top-level JSON number field (int or float) by key. Returns 0.0 when not found.
- static std::string `_buildMessagesArray` (const std::vector< [ChatMessage](#) > &messages)
Serialises a vector of [ChatMessage](#) objects to a JSON array string.
- static std::string `_extractFirstArrayElement` (const std::string &json, const std::string &arrayKey)
Extracts the raw substring of the first element of a named JSON array. Returns an empty string when not found.

Protected Member Functions

- virtual std::string `_buildRequestBody` (const [ProviderRequest](#) &request) const =0
Serialises `request` to the provider's JSON wire format.
- virtual [ProviderResponse](#) `_parseResponseBody` (const std::string &rawBody, int httpStatus) const =0
Deserialises `rawBody` (the full curl stdout) into a [ProviderResponse](#).
- virtual std::string `_authorizationHeader` () const =0
Returns the Authorization header line without the "header = " prefix.
- virtual std::vector< std::string > `_extraRequestHeaders` () const
Returns additional HTTP headers to send with every request.

Protected Attributes

- std::string `_baseUrl`
- unsigned int `_timeoutSeconds` = 30
- std::string `_apiKey`
In-memory API key.

15.361.1 Detailed Description

Abstract base class for HTTP-based AI provider clients.

Implements the common parts of [AIProviderClient_if](#) that are shared across all cloud and local provider backends:

- In-memory API key management with the same security contract as [AIAssistant_if](#).
- Base URL and timeout configuration.
- A synchronous `send()` that builds a curl command, writes the API key and request body to temporary files, invokes curl via `popen()`, and returns the raw response body to the subclass for parsing.

15.361.1.1 Transport design

The HTTP request is executed by shelling out to the system `curl` binary. This avoids any C++ HTTP library dependency while remaining portable. Security considerations:

- The API key is written to a temporary curl config file created with `mkstemp()` and restricted to mode 0600. The key is never passed on the command line, keeping it out of `/proc/<pid>/cmdline` and shell history.
- Both the body temp file and the config temp file are unlinked immediately after the curl invocation begins (unlink-before-pclose pattern).

15.361.1.2 Subclass responsibilities

Concrete implementations must override:

- `_buildRequestBody()` — serialise [ProviderRequest](#) to the provider's JSON format.
- `_parseResponseBody()` — deserialise the raw HTTP body to [ProviderResponse](#).
- `_authorizationHeader()` — return the full auth header line (e.g. "Authorization: Bearer <key>").
- `_extraRequestHeaders()` — return additional provider-specific headers (default: none).

15.361.1.3 curl dependency

curl must be installed and on PATH at runtime. If curl is unavailable, `send()` returns a [ProviderResponse](#) with `success=false` and a descriptive error message.

15.361.2 Member Function Documentation

15.361.2.1 _authorizationHeader()

```
virtual std::string HttpProviderClientBase::_authorizationHeader ( ) const [protected], [pure virtual]
```

Returns the Authorization header line without the "header = " prefix.

Example return values:

- "Authorization: Bearer sk-..." (OpenAI, LocalProvider)
- "x-api-key: sk-ant-..." (Anthropic)

Called only from within `send()`; the value is written to a 0600 temp file and never logged.

Implemented in [AnthropicProviderClientImpl](#), [LocalProviderClientImpl](#), and [OpenAIProviderClientImpl](#).

15.361.2.2 _buildRequestBody()

```
virtual std::string HttpProviderClientBase::_buildRequestBody (
    const ProviderRequest & request ) const [protected], [pure virtual]
```

Serialises `request` to the provider's JSON wire format.

Implemented in [AnthropicProviderClientImpl](#), [LocalProviderClientImpl](#), and [OpenAIProviderClientImpl](#).

15.361.2.3 _extraRequestHeaders()

```
std::vector< std::string > HttpProviderClientBase::_extraRequestHeaders ( ) const [protected], [virtual]
```

Returns additional HTTP headers to send with every request.

Default implementation returns an empty vector. Anthropic overrides to add "anthropic-version: 2023-06-01".

Each element must be a complete header line, e.g. "anthropic-version: 2023-06-01".

Reimplemented in [AnthropicProviderClientImpl](#), and [OpenAIProviderClientImpl](#).

15.361.2.4 _parseResponseBody()

```
virtual ProviderResponse HttpProviderClientBase::_parseResponseBody (
    const std::string & rawBody,
    int httpStatus ) const [protected], [pure virtual]
```

Deserialises `rawBody` (the full curl stdout) into a [ProviderResponse](#).

Parameters

<i>rawBody</i>	Complete HTTP response body as received from curl.
<i>httpStatus</i>	HTTP status code captured from curl exit / response line, or 0 when unavailable.

Implemented in [AnthropicProviderClientImpl](#), [LocalProviderClientImpl](#), and [OpenAIProviderClientImpl](#).

15.361.2.5 clearApiKey()

```
void HttpProviderClientBase::clearApiKey ( ) [override], [virtual]
```

Overwrites (best-effort) and clears the in-memory API key.

Implements [AIProviderClient_if](#).

15.361.2.6 getBaseUrl()

```
std::string HttpProviderClientBase::getBaseUrl ( ) const [override], [virtual]
```

Returns the currently configured base URL.

Implements [AIProviderClient_if](#).

15.361.2.7 getTimeoutSeconds()

```
unsigned int HttpProviderClientBase::getTimeoutSeconds ( ) const [override], [virtual]
```

Returns the configured HTTP request timeout.

Implements [AIProviderClient_if](#).

15.361.2.8 hasApiKey()

```
bool HttpProviderClientBase::hasApiKey ( ) const [override], [virtual]
```

Returns `true` when a non-empty API key is held.

Implements [AIProviderClient_if](#).

15.361.2.9 isAvailable()

```
bool HttpProviderClientBase::isAvailable ( ) const [override], [virtual]
```

Returns `true` when this client can attempt to send requests.

A client is considered available when it holds an API key (if required by the provider) and a reachable base URL.

Implements [AIProviderClient_if](#).

Reimplemented in [LocalProviderClientImpl](#), and [OpenAIProviderClientImpl](#).

15.361.2.10 send()

```
ProviderResponse HttpProviderClientBase::send (
    const ProviderRequest & request ) [override], [virtual]
```

Executes the HTTP request via curl and returns the provider response.

Steps:

1. Calls [_buildRequestBody\(\)](#) to serialise the request.
2. Writes the body JSON to a temp file.
3. Writes a curl config file with URL, headers (including auth), data reference, and timeout.
4. Invokes curl via `popen()`, capturing stdout.

5. Unlinks both temp files.
6. Calls `_parseResponseBody()` on the captured output.

Implements [AIProviderClient_if](#).

15.361.2.11 setApiKey()

```
void HttpProviderClientBase::setApiKey (
    const std::string & apiKey ) [override], [virtual]
```

Stores the API key in memory.

Never write the key to disk or logs. Prefer loading from an environment variable via the owning [AIAssistant_if](#) rather than embedding keys in source.

Implements [AIProviderClient_if](#).

15.361.2.12 setBaseUrl()

```
void HttpProviderClientBase::setBaseUrl (
    const std::string & url ) [override], [virtual]
```

Sets the base URL for the provider API endpoint.

Implements [AIProviderClient_if](#).

15.361.2.13 setTimeoutSeconds()

```
void HttpProviderClientBase::setTimeoutSeconds (
    unsigned int seconds ) [override], [virtual]
```

Sets the HTTP request timeout in seconds.

Implements [AIProviderClient_if](#).

15.361.3 Member Data Documentation

15.361.3.1 _apiKey

```
std::string HttpProviderClientBase::_apiKey [protected]
```

In-memory API key.

Security contract (matches [AIAssistant_if](#)):

- Never written to disk.
- Never included in log or trace output.
- Not accessible via a public getter.
- Overwritten (best-effort) before clearing in [clearApiKey\(\)](#).

The documentation for this class was generated from the following files:

- tools/AIAssistant/HttpProviderClientBase.h
- tools/AIAssistant/HttpProviderClientBase.cpp

15.362 HttpRequest Struct Reference

Lightweight representation of one parsed HTTP request.

```
#include <applications/worker/http/HttpRequest.h>
```

Public Member Functions

- `std::string header (const std::string &name) const`
Returns a header value using a case-insensitive lookup.

Public Attributes

- `std::string` **method**
HTTP method token as received from the request line.
- `std::string` **path**
Request target path used by the API router.
- `std::string` **body**
Raw body payload passed to JSON parsers and handlers.
- `std::unordered_map< std::string, std::string >` **headers**
Lowercase header map used for case-insensitive lookups.

15.362.1 Detailed Description

Lightweight representation of one parsed HTTP request.

The worker application uses a small transport surface, so the request keeps only the pieces needed by the router: method, path, raw body and normalized header lookup.

15.362.2 Member Function Documentation**15.362.2.1 header()**

```
std::string HttpRequest::header (
    const std::string & name ) const [inline]
```

Returns a header value using a case-insensitive lookup.

Parameters

<i>name</i>	Header name to search.
-------------	------------------------

Returns

Header value when present, or an empty string otherwise.

The documentation for this struct was generated from the following file:

- applications/worker/http/HttpRequest.h

15.363 HttpResponseMessage Struct Reference

Minimal HTTP response payload produced by the web API router.

```
#include <applications/worker/http/HttpResponse.h>
```

Public Attributes

- `int` **status** = 200
HTTP status code returned to the client.
- `std::string` **contentType** = "application/json"
MIME type advertised in the response.
- `std::string` **body**
Raw response body, usually JSON.
- `std::unordered_map< std::string, std::string >` **headers**
Arbitrary response headers written by the router or handlers.

15.363.1 Detailed Description

Minimal HTTP response payload produced by the web API router.

The response mirrors the small transport model used by the embedded server: status, content type, body, and a simple header map.

The documentation for this struct was generated from the following file:

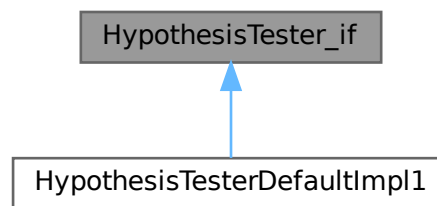
- applications/worker/http/HttpResponse.h

15.364 HypothesisTester_if Class Reference

Interface for classical parametric inference.

```
#include <tools/Statistics/HypothesisTester_if.h>
```

Inheritance diagram for HypothesisTester_if:



Classes

- class [ConfidenceInterval](#)
Confidence interval [inferior, superior] and its half-width e0.
- class [TestResult](#)
Result of a parametric hypothesis test.

Public Types

- enum [H1Comparition](#) { **DIFFERENT** = 1 , **LESS_THAN** = 2 , **GREATER_THAN** = 3 }
Alternative-hypothesis comparison mode. DIFFERENT: two-sided; LESS_THAN: left-tailed; GREATER_THAN: right-tailed.

Public Member Functions

- virtual [HypothesisTester_if::ConfidenceInterval](#) **averageConfidenceInterval** (double avg, double stddev, unsigned int n, double confidenceLevel)=0
- virtual [HypothesisTester_if::ConfidenceInterval](#) **proportionConfidenceInterval** (double prop, unsigned int n, double confidenceLevel)=0
- virtual [HypothesisTester_if::ConfidenceInterval](#) **proportionConfidenceInterval** (double prop, unsigned int n, int N, double confidenceLevel)=0
- virtual [HypothesisTester_if::ConfidenceInterval](#) **varianceConfidenceInterval** (double var, unsigned int n, double confidenceLevel)=0
- virtual [HypothesisTester_if::ConfidenceInterval](#) **averageDifferenceConfidenceInterval** (double avg1, double stddev1, unsigned int n1, double avg2, double stddev2, unsigned int n2, double confidenceLevel)=0
- virtual [HypothesisTester_if::ConfidenceInterval](#) **proportionDifferenceConfidenceInterval** (double avg1, double stddev1, unsigned int n1, double avg2, double stddev2, unsigned int n2, double confidenceLevel)=0
- virtual [HypothesisTester_if::ConfidenceInterval](#) **varianceRatioConfidenceInterval** (double var1, unsigned int n1, double var2, unsigned int n2, double confidenceLevel)=0
- virtual [HypothesisTester_if::ConfidenceInterval](#) **averageConfidenceInterval** (std::string sampleData←Filename, double confidenceLevel)=0

- virtual [HypothesisTester_if::ConfidenceInterval](#) **proportionConfidenceInterval** (std::string sampleData↵
Filename, checkProportionFunction function, double confidenceLevel)=0
- virtual [HypothesisTester_if::ConfidenceInterval](#) **proportionConfidenceInterval** (std::string sampleData↵
Filename, checkProportionFunction function, double N, double confidenceLevel)=0
- virtual [HypothesisTester_if::ConfidenceInterval](#) **varianceConfidenceInterval** (std::string sampleData↵
Filename, double confidenceLevel)=0
- virtual unsigned int **estimateSampleSize** (double avg, double stddev, double desiredE0, double confidence↵
Level)=0
- virtual [HypothesisTester_if::TestResult](#) **testAverage** (double avg, double stddev, unsigned int n, double avg↵
Sample, double confidenceLevel, [HypothesisTester_if::H1Comparison](#) comp)=0
- virtual [HypothesisTester_if::TestResult](#) **testProportion** (double prop, unsigned int n, double proptest, double
confidenceLevel, [HypothesisTester_if::H1Comparison](#) comp)=0
- virtual [HypothesisTester_if::TestResult](#) **testVariance** (double var, unsigned int n, double vartest, double
confidenceLevel, [HypothesisTester_if::H1Comparison](#) comp)=0
- virtual [HypothesisTester_if::TestResult](#) **testAverage** (double avg1, double stddev1, unsigned int n1, dou-
ble avg2, double stddev2, unsigned int n2, double confidenceLevel, [HypothesisTester_if::H1Comparison](#)
comp)=0
- virtual [HypothesisTester_if::TestResult](#) **testProportion** (double prop1, unsigned int n1, double prop2, un-
signed int n2, double confidenceLevel, [HypothesisTester_if::H1Comparison](#) comp)=0
- virtual [HypothesisTester_if::TestResult](#) **testVariance** (double var1, unsigned int n1, double var2, unsigned int
n2, double confidenceLevel, [HypothesisTester_if::H1Comparison](#) comp)=0
- virtual [HypothesisTester_if::TestResult](#) **testAverage** (std::string sampleDataFilename, double avgSample,
double confidenceLevel, [HypothesisTester_if::H1Comparison](#) comp)=0
- virtual [HypothesisTester_if::TestResult](#) **testProportion** (std::string sampleDataFilename, checkProportion↵
Function function, double proptest, double confidenceLevel, [HypothesisTester_if::H1Comparison](#) comp)=0
- virtual [HypothesisTester_if::TestResult](#) **testVariance** (std::string sampleDataFilename, double vartest, double
confidenceLevel, [HypothesisTester_if::H1Comparison](#) comp)=0
- virtual [HypothesisTester_if::TestResult](#) **testAverage** (std::string firstSampleDataFilename, std::string
secondSampleDataFilename, double confidenceLevel, [HypothesisTester_if::H1Comparison](#) comp)=0
- virtual [HypothesisTester_if::TestResult](#) **testProportion** (std::string firstSampleDataFilename, std↵
::string secondSampleDataFilename, checkProportionFunction function, double confidenceLevel,
[HypothesisTester_if::H1Comparison](#) comp)=0
- virtual [HypothesisTester_if::TestResult](#) **testVariance** (std::string firstSampleDataFilename, std::string
secondSampleDataFilename, double confidenceLevel, [HypothesisTester_if::H1Comparison](#) comp)=0

15.364.1 Detailed Description

Interface for classical parametric inference.

Purpose:

- Provide confidence intervals and hypothesis tests from parameters or dataset files.

Mathematical meaning:

- Contract follows classical statistical inference for one-population and two-population analyses.

Confidence level contract:

- All confidenceLevel parameters are expected to be probabilities in [0, 1].

The documentation for this class was generated from the following file:

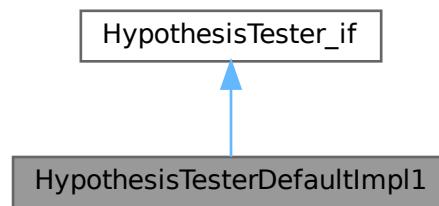
- tools/Statistics/HypothesisTester_if.h

15.365 HypothesisTesterDefaultImpl1 Class Reference

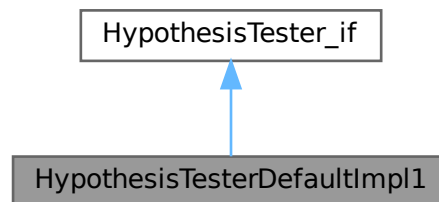
Current default implementation of [HypothesisTester_if](#).

```
#include <tools/Statistics/HypothesisTesterDefaultImpl1.h>
```

Inheritance diagram for HypothesisTesterDefaultImpl1:



Collaboration diagram for HypothesisTesterDefaultImpl1:



Public Member Functions

- [HypothesisTester_if::ConfidenceInterval averageConfidenceInterval](#) (double avg, double stddev, unsigned int n, double confidenceLevel) override
- [HypothesisTester_if::ConfidenceInterval proportionConfidenceInterval](#) (double prop, unsigned int n, double confidenceLevel) override
- [HypothesisTester_if::ConfidenceInterval proportionConfidenceInterval](#) (double prop, unsigned int n, int N, double confidenceLevel) override
- [HypothesisTester_if::ConfidenceInterval varianceConfidenceInterval](#) (double var, unsigned int n, double confidenceLevel) override
- [HypothesisTester_if::ConfidenceInterval averageDifferenceConfidenceInterval](#) (double avg1, double stddev1, unsigned int n1, double avg2, double stddev2, unsigned int n2, double confidenceLevel) override
- [HypothesisTester_if::ConfidenceInterval proportionDifferenceConfidenceInterval](#) (double avg1, double stddev1, unsigned int n1, double avg2, double stddev2, unsigned int n2, double confidenceLevel) override
- [HypothesisTester_if::ConfidenceInterval varianceRatioConfidenceInterval](#) (double var1, unsigned int n1, double var2, unsigned int n2, double confidenceLevel) override
- [HypothesisTester_if::ConfidenceInterval averageConfidenceInterval](#) (std::string sampleDataFilename, double confidenceLevel) override
- [HypothesisTester_if::ConfidenceInterval proportionConfidenceInterval](#) (std::string sampleDataFilename, checkProportionFunction function, double confidenceLevel) override
- [HypothesisTester_if::ConfidenceInterval proportionConfidenceInterval](#) (std::string sampleDataFilename, checkProportionFunction function, double N, double confidenceLevel) override
- [HypothesisTester_if::ConfidenceInterval varianceConfidenceInterval](#) (std::string sampleDataFilename, double confidenceLevel) override

- unsigned int `estimateSampleSize` (double avg, double stddev, double desiredE0, double confidenceLevel) override
- `HypothesisTester_if::TestResult testAverage` (double avg, double stddev, unsigned int n, double avgSample, double confidenceLevel, `HypothesisTester_if::H1Comparison` comp) override
- `HypothesisTester_if::TestResult testProportion` (double prop, unsigned int n, double proptest, double confidenceLevel, `HypothesisTester_if::H1Comparison` comp) override
- `HypothesisTester_if::TestResult testVariance` (double var, unsigned int n, double vartest, double confidenceLevel, `HypothesisTester_if::H1Comparison` comp) override
- `HypothesisTester_if::TestResult testAverage` (double avg1, double stddev1, unsigned int n1, double avg2, double stddev2, unsigned int n2, double confidenceLevel, `HypothesisTester_if::H1Comparison` comp) override
- `HypothesisTester_if::TestResult testProportion` (double prop1, unsigned int n1, double prop2, unsigned int n2, double confidenceLevel, `HypothesisTester_if::H1Comparison` comp) override
- `HypothesisTester_if::TestResult testVariance` (double var1, unsigned int n1, double var2, unsigned int n2, double confidenceLevel, `HypothesisTester_if::H1Comparison` comp) override
- `HypothesisTester_if::TestResult testAverage` (std::string sampleDataFilename, double avgSample, double confidenceLevel, `HypothesisTester_if::H1Comparison` comp) override
- `HypothesisTester_if::TestResult testProportion` (std::string sampleDataFilename, checkProportionFunction function, double proptest, double confidenceLevel, `HypothesisTester_if::H1Comparison` comp) override
- `HypothesisTester_if::TestResult testVariance` (std::string sampleDataFilename, double vartest, double confidenceLevel, `HypothesisTester_if::H1Comparison` comp) override
- `HypothesisTester_if::TestResult testAverage` (std::string firstSampleDataFilename, std::string secondSampleDataFilename, double confidenceLevel, `HypothesisTester_if::H1Comparison` comp) override
- `HypothesisTester_if::TestResult testProportion` (std::string firstSampleDataFilename, std::string secondSampleDataFilename, checkProportionFunction function, double confidenceLevel, `HypothesisTester_if::H1Comparison` comp) override
- `HypothesisTester_if::TestResult testVariance` (std::string firstSampleDataFilename, std::string secondSampleDataFilename, double confidenceLevel, `HypothesisTester_if::H1Comparison` comp) override

Additional Inherited Members

Public Types inherited from `HypothesisTester_if`

- enum `H1Comparison` { `DIFFERENT` = 1 , `LESS_THAN` = 2 , `GREATER_THAN` = 3 }
- Alternative-hypothesis comparison mode. DIFFERENT: two-sided; LESS_THAN: left-tailed; GREATER_THAN: right-tailed.*

15.365.1 Detailed Description

Current default implementation of `HypothesisTester_if`.

Architectural role:

- Best available legacy implementation in the tools package.

Implementation status:

- Partially consolidated. One-population methods are mostly available; two-population routines still require theoretical and implementation alignment.

Planned evolution:

- Continue converging to a fully consistent implementation based on formal classical hypothesis-testing theory.

15.365.2 Member Function Documentation

15.365.2.1 `averageConfidenceInterval()` [1/2]

```
HypothesisTester_if::ConfidenceInterval HypothesisTesterDefaultImpl1::averageConfidenceInterval (
    double avg,
```

```
double stddev,
unsigned int n,
double confidenceLevel ) [override], [virtual]
```

Implements [HypothesisTester_if](#).

15.365.2.2 averageConfidenceInterval() [2/2]

```
HypothesisTester\_if::ConfidenceInterval HypothesisTesterDefaultImpl1::averageConfidence←
Interval (
    std::string sampleDataFilename,
    double confidenceLevel ) [override], [virtual]
```

Implements [HypothesisTester_if](#).

15.365.2.3 averageDifferenceConfidenceInterval()

```
HypothesisTester\_if::ConfidenceInterval HypothesisTesterDefaultImpl1::averageDifference←
ConfidenceInterval (
    double avg1,
    double stddev1,
    unsigned int n1,
    double avg2,
    double stddev2,
    unsigned int n2,
    double confidenceLevel ) [override], [virtual]
```

Implements [HypothesisTester_if](#).

15.365.2.4 estimateSampleSize()

```
unsigned int HypothesisTesterDefaultImpl1::estimateSampleSize (
    double avg,
    double stddev,
    double desiredE0,
    double confidenceLevel ) [override], [virtual]
```

Implements [HypothesisTester_if](#).

15.365.2.5 proportionConfidenceInterval() [1/4]

```
HypothesisTester\_if::ConfidenceInterval HypothesisTesterDefaultImpl1::proportionConfidence←
Interval (
    double prop,
    unsigned int n,
    double confidenceLevel ) [override], [virtual]
```

Implements [HypothesisTester_if](#).

15.365.2.6 proportionConfidenceInterval() [2/4]

```
HypothesisTester\_if::ConfidenceInterval HypothesisTesterDefaultImpl1::proportionConfidence←
Interval (
    double prop,
    unsigned int n,
    int N,
    double confidenceLevel ) [override], [virtual]
```

Implements [HypothesisTester_if](#).

15.365.2.7 proportionConfidenceInterval() [3/4]

```
HypothesisTester\_if::ConfidenceInterval HypothesisTesterDefaultImpl1::proportionConfidence←
Interval (
    std::string sampleDataFilename,
```

```

        checkProportionFunction function,
        double confidenceLevel ) [override], [virtual]

```

Implements [HypothesisTester_if](#).

15.365.2.8 proportionConfidenceInterval() [4/4]

[HypothesisTester_if::ConfidenceInterval](#) [HypothesisTesterDefaultImpl1::proportionConfidenceInterval](#) (

```

    std::string sampleDataFilename,
    checkProportionFunction function,
    double N,
    double confidenceLevel ) [override], [virtual]

```

Implements [HypothesisTester_if](#).

15.365.2.9 proportionDifferenceConfidenceInterval()

[HypothesisTester_if::ConfidenceInterval](#) [HypothesisTesterDefaultImpl1::proportionDifferenceConfidenceInterval](#) (

```

    double avg1,
    double stddev1,
    unsigned int n1,
    double avg2,
    double stddev2,
    unsigned int n2,
    double confidenceLevel ) [override], [virtual]

```

Implements [HypothesisTester_if](#).

15.365.2.10 testAverage() [1/4]

[HypothesisTester_if::TestResult](#) [HypothesisTesterDefaultImpl1::testAverage](#) (

```

    double avg,
    double stddev,
    unsigned int n,
    double avgSample,
    double confidenceLevel,
    HypothesisTester\_if::H1Comparition comp ) [override], [virtual]

```

Implements [HypothesisTester_if](#).

15.365.2.11 testAverage() [2/4]

[HypothesisTester_if::TestResult](#) [HypothesisTesterDefaultImpl1::testAverage](#) (

```

    double avg1,
    double stddev1,
    unsigned int n1,
    double avg2,
    double stddev2,
    unsigned int n2,
    double confidenceLevel,
    HypothesisTester\_if::H1Comparition comp ) [override], [virtual]

```

Implements [HypothesisTester_if](#).

15.365.2.12 testAverage() [3/4]

[HypothesisTester_if::TestResult](#) [HypothesisTesterDefaultImpl1::testAverage](#) (

```

    std::string firstSampleDataFilename,
    std::string secondSampleDataFilename,
    double confidenceLevel,
    HypothesisTester\_if::H1Comparition comp ) [override], [virtual]

```

Implements [HypothesisTester_if](#).

15.365.2.13 testAverage() [4/4]

```
HypothesisTester_if::TestResult HypothesisTesterDefaultImpl1::testAverage (
    std::string sampleDataFilename,
    double avgSample,
    double confidenceLevel,
    HypothesisTester_if::H1Comparition comp ) [override], [virtual]
Implements HypothesisTester_if.
```

15.365.2.14 testProportion() [1/4]

```
HypothesisTester_if::TestResult HypothesisTesterDefaultImpl1::testProportion (
    double prop,
    unsigned int n,
    double proptest,
    double confidenceLevel,
    HypothesisTester_if::H1Comparition comp ) [override], [virtual]
Implements HypothesisTester_if.
```

15.365.2.15 testProportion() [2/4]

```
HypothesisTester_if::TestResult HypothesisTesterDefaultImpl1::testProportion (
    double prop1,
    unsigned int n1,
    double prop2,
    unsigned int n2,
    double confidenceLevel,
    HypothesisTester_if::H1Comparition comp ) [override], [virtual]
Implements HypothesisTester_if.
```

15.365.2.16 testProportion() [3/4]

```
HypothesisTester_if::TestResult HypothesisTesterDefaultImpl1::testProportion (
    std::string firstSampleDataFilename,
    std::string secondSampleDataFilename,
    checkProportionFunction function,
    double confidenceLevel,
    HypothesisTester_if::H1Comparition comp ) [override], [virtual]
Implements HypothesisTester_if.
```

15.365.2.17 testProportion() [4/4]

```
HypothesisTester_if::TestResult HypothesisTesterDefaultImpl1::testProportion (
    std::string sampleDataFilename,
    checkProportionFunction function,
    double proptest,
    double confidenceLevel,
    HypothesisTester_if::H1Comparition comp ) [override], [virtual]
Implements HypothesisTester_if.
```

15.365.2.18 testVariance() [1/4]

```
HypothesisTester_if::TestResult HypothesisTesterDefaultImpl1::testVariance (
    double var,
    unsigned int n,
    double vartest,
    double confidenceLevel,
    HypothesisTester_if::H1Comparition comp ) [override], [virtual]
Implements HypothesisTester_if.
```

15.365.2.19 testVariance() [2/4]

```
HypothesisTester_if::TestResult HypothesisTesterDefaultImpl1::testVariance (
    double var1,
    unsigned int n1,
    double var2,
    unsigned int n2,
    double confidenceLevel,
    HypothesisTester_if::H1Comparition comp ) [override], [virtual]
```

Implements [HypothesisTester_if](#).

15.365.2.20 testVariance() [3/4]

```
HypothesisTester_if::TestResult HypothesisTesterDefaultImpl1::testVariance (
    std::string firstSampleDataFilename,
    std::string secondSampleDataFilename,
    double confidenceLevel,
    HypothesisTester_if::H1Comparition comp ) [override], [virtual]
```

Implements [HypothesisTester_if](#).

15.365.2.21 testVariance() [4/4]

```
HypothesisTester_if::TestResult HypothesisTesterDefaultImpl1::testVariance (
    std::string sampleDataFilename,
    double vartest,
    double confidenceLevel,
    HypothesisTester_if::H1Comparition comp ) [override], [virtual]
```

Implements [HypothesisTester_if](#).

15.365.2.22 varianceConfidenceInterval() [1/2]

```
HypothesisTester_if::ConfidenceInterval HypothesisTesterDefaultImpl1::varianceConfidence↔
Interval (
    double var,
    unsigned int n,
    double confidenceLevel ) [override], [virtual]
```

Implements [HypothesisTester_if](#).

15.365.2.23 varianceConfidenceInterval() [2/2]

```
HypothesisTester_if::ConfidenceInterval HypothesisTesterDefaultImpl1::varianceConfidence↔
Interval (
    std::string sampleDataFilename,
    double confidenceLevel ) [override], [virtual]
```

Implements [HypothesisTester_if](#).

15.365.2.24 varianceRatioConfidenceInterval()

```
HypothesisTester_if::ConfidenceInterval HypothesisTesterDefaultImpl1::varianceRatioConfidence↔
Interval (
    double var1,
    unsigned int n1,
    double var2,
    unsigned int n2,
    double confidenceLevel ) [override], [virtual]
```

Implements [HypothesisTester_if](#).

The documentation for this class was generated from the following files:

- tools/Statistics/HypothesisTesterDefaultImpl1.h
- tools/Statistics/HypothesisTesterDefaultImpl1.cpp

15.366 testing::internal::Ignore< size_t > Struct Template Reference

Public Member Functions

- `Ignore (...)`

The documentation for this struct was generated from the following file:

- `gtest/gtest/internal/gtest-internal.h`

15.367 testing::internal::IgnoredValue Class Reference

Public Member Functions

- `template<typename T, typename std::enable_if<!std::is_convertible< T, Sink >::value, int >::type = 0> IgnoredValue (const T &)`

The documentation for this class was generated from the following file:

- `gtest/gtest/internal/gtest-internal.h`

15.368 testing::internal::IndexSequence< Is > Struct Template Reference

Public Types

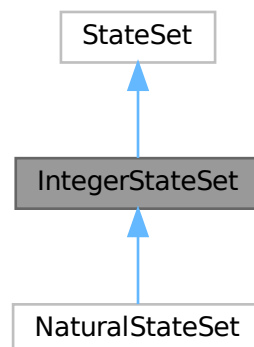
- using `type` = [IndexSequence](#)

The documentation for this struct was generated from the following file:

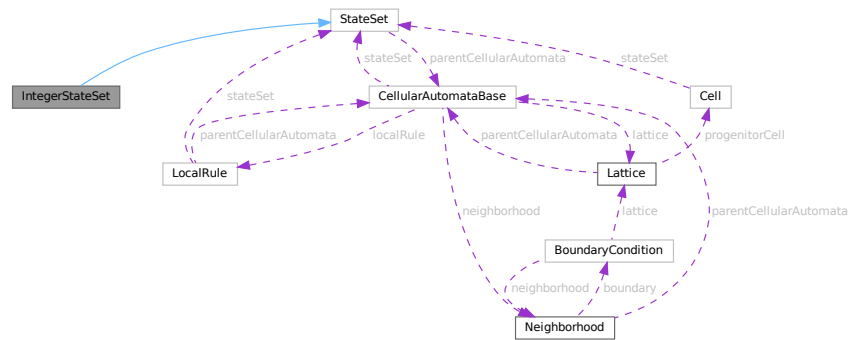
- `gtest/gtest/internal/gtest-internal.h`

15.369 IntegerStateSet Class Reference

Inheritance diagram for IntegerStateSet:



Collaboration diagram for IntegerStateSet:



Public Member Functions

- **IntegerStateSet** ([CellularAutomataBase](#) *parentCellularAutomata, std::optional< long > minimum=std::nullopt, std::optional< long > maximum=std::nullopt)
- bool **contains** (const [State](#) &state) const override
- std::unique_ptr< [State](#) > **createDefaultState** () const override
- std::unique_ptr< [State](#) > **parseState** (const std::string &text) const override
- bool **isFinite** () const override
- bool **isDiscrete** () const override
- std::vector< std::unique_ptr< [State](#) > > **enumerateStates** () const override
- std::string **name** () const override

Public Member Functions inherited from [StateSet](#)

- **StateSet** ([CellularAutomataBase](#) *parentCellularAutomata)
- **StateSet** (const [StateSet](#) &orig)
- virtual bool **isContinuous** () const
- virtual bool **isComposite** () const

Additional Inherited Members

Protected Attributes inherited from [StateSet](#)

- [CellularAutomataBase](#) * parentCellularAutomata

15.369.1 Member Function Documentation

15.369.1.1 contains()

```
bool IntegerStateSet::contains (
    const State & state ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

15.369.1.2 createDefaultState()

```
std::unique_ptr< State > IntegerStateSet::createDefaultState ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

15.369.1.3 enumerateStates()

```
std::vector< std::unique_ptr< State > > IntegerStateSet::enumerateStates ( ) const [inline],  
[override], [virtual]
```

Reimplemented from [StateSet](#).

15.369.1.4 isDiscrete()

```
bool IntegerStateSet::isDiscrete ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

15.369.1.5 isFinite()

```
bool IntegerStateSet::isFinite ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

15.369.1.6 name()

```
std::string IntegerStateSet::name ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

15.369.1.7 parseState()

```
std::unique_ptr< State > IntegerStateSet::parseState (  
    const std::string & text ) const [inline], [override], [virtual]
```

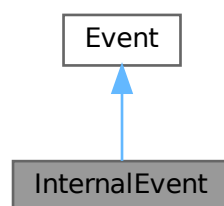
Reimplemented from [StateSet](#).

The documentation for this class was generated from the following file:

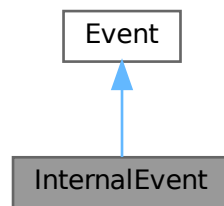
- [plugins/components/ModalModel/CellularAutomata/StateSet.h](#)

15.370 InternalEvent Class Reference

Inheritance diagram for InternalEvent:



Collaboration diagram for InternalEvent:



Public Member Functions

- **InternalEvent** (double time, std::string description)
- template<typename Class >
void **setEventHandler** (Class *object, void(Class::*function)(void *), void *parameter)
- virtual std::string **show** () override
- const InternalEventHandler & **eventHandler** () const
- const std::string & **description** () const
- void **dispatchEvent** () const
- void * **object** () const
- void * **parameter** () const

Public Member Functions inherited from [Event](#)

- **Event** (double time, [Entity](#) *entity, [ModelComponent](#) *component, unsigned int componentinputPortNumber=0)
- **Event** (double time, [Entity](#) *entity, [Connection](#) *connection)
- double **getTime** () const
getTime
- [ModelComponent](#) * **getComponent** () const
getComponent
- [Entity](#) * **getEntity** () const
getEntity
- unsigned int **getComponentinputPortNumber** () const
getComponentinputPortNumber
- void **setEntity** ([Entity](#) *_entity)
setEntity

Additional Inherited Members

Protected Attributes inherited from [Event](#)

- double **_time**

15.370.1 Member Function Documentation

15.370.1.1 show()

```
virtual std::string InternalEvent::show ( ) [inline], [override], [virtual]
```

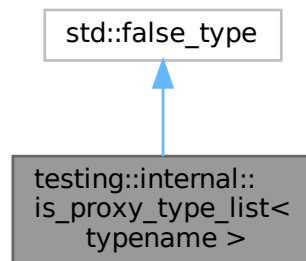
Reimplemented from [Event](#).

The documentation for this class was generated from the following file:

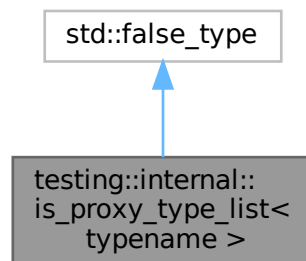
- kernel/simulator/Event.h

15.371 testing::internal::is_proxy_type_list< typename > Struct Template Reference

Inheritance diagram for testing::internal::is_proxy_type_list< typename >:



Collaboration diagram for testing::internal::is_proxy_type_list< typename >:

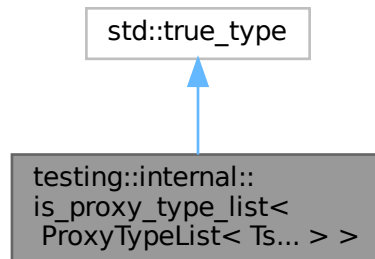


The documentation for this struct was generated from the following file:

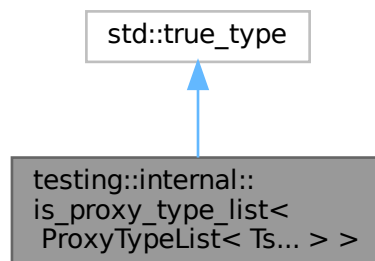
- gtest/gtest/internal/gtest-type-util.h

15.372 testing::internal::is_proxy_type_list< ProxyTypeList< Ts... > > Struct Template Reference

Inheritance diagram for testing::internal::is_proxy_type_list< ProxyTypeList< Ts... > >:



Collaboration diagram for testing::internal::is_proxy_type_list< ProxyTypeList< Ts... > >:



The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-type-util.h

15.373 testing::internal::IsHashTable< T > Struct Template Reference

Static Public Attributes

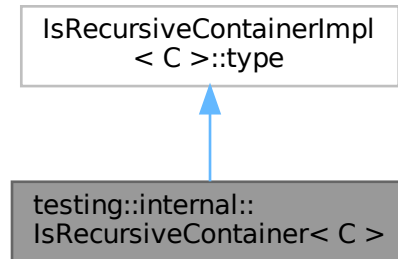
- static const bool **value** = sizeof(test<T>(nullptr, nullptr)) == sizeof(int)

The documentation for this struct was generated from the following file:

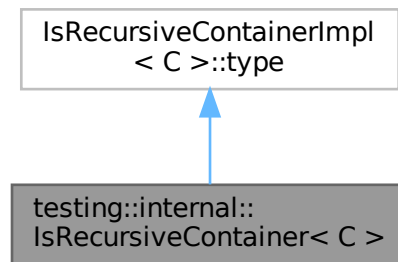
- gtest/gtest/internal/gtest-internal.h

15.374 testing::internal::IsRecursiveContainer< C > Struct Template Reference

Inheritance diagram for testing::internal::IsRecursiveContainer< C >:



Collaboration diagram for testing::internal::IsRecursiveContainer< C >:



The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-internal.h

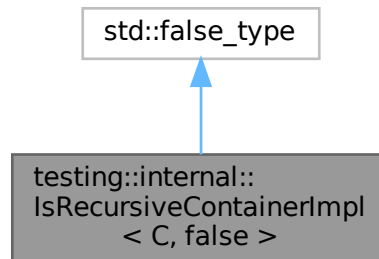
15.375 testing::internal::IsRecursiveContainerImpl< C, bool > Struct Template Reference

The documentation for this struct was generated from the following file:

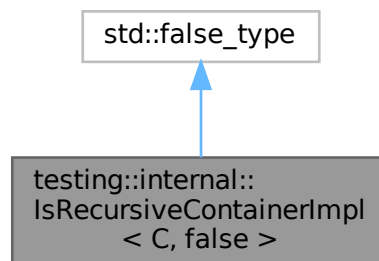
- gtest/gtest/internal/gtest-internal.h

15.376 testing::internal::IsRecursiveContainerImpl< C, false > Struct Template Reference

Inheritance diagram for testing::internal::IsRecursiveContainerImpl< C, false >:



Collaboration diagram for testing::internal::IsRecursiveContainerImpl< C, false >:



The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-internal.h

15.377 testing::internal::IsRecursiveContainerImpl< C, true > Struct Template Reference

Public Types

- using **value_type** = decltype(*std::declval< typename C::const_iterator >())
- using **type** = std::is_same< typename std::remove_const< typename std::remove_reference< value_type >::type >::type, C >

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-internal.h

15.378 IssueReportDraft Struct Reference

User-authored issue report collected by the GUI form before transport.

```
#include <applications/gui/genesys/services/IssueReportRelayService.h>
```

Public Attributes

- QString **categoryKey**
- QString **categoryLabel**
- QString **title**
- QString **summary**
- QString **reproductionSteps**
- QString **expectedBehavior**
- QString **observedBehavior**
- QString **additionalContext**
- QString **contactEmail**
- bool **includeScreenshot** = false
- bool **includeLogTail** = true
- bool **includeModelText** = false

15.378.1 Detailed Description

User-authored issue report collected by the GUI form before transport.

The documentation for this struct was generated from the following file:

- applications/gui/genesys/services/IssueReportRelayService.h

15.379 IssueReportRelayService Class Reference

HTTPS client service that sends issue reports to a secure relay endpoint.

```
#include <applications/gui/genesys/services/IssueReportRelayService.h>
```

Static Public Member Functions

- static QString **configuredEndpoint** ()
- static QString **optionalClientKey** ()
- static int **requestTimeoutMs** ()
- static bool **isEndpointConfigured** (QString *reason=nullptr)
- static [IssueReportSubmissionResult](#) **submit** (const [IssueReportDraft](#) &draft, const [IssueReportRuntimeContext](#) &runtimeContext)

15.379.1 Detailed Description

HTTPS client service that sends issue reports to a secure relay endpoint.

Responsibilities:

- validate relay configuration from environment;
- serialize the GUI report and runtime diagnostics into JSON;
- submit the payload via HTTPS and normalize the relay response.

Boundaries:

- it does not talk directly to GitHub with a secret embedded in the client;
- it does not own UI widgets or present dialogs;
- it assumes an external relay exists and enforces server-side policy.

The documentation for this class was generated from the following files:

- applications/gui/genesys/services/IssueReportRelayService.h
- applications/gui/genesys/services/IssueReportRelayService.cpp

15.380 IssueReportRuntimeContext Struct Reference

Runtime diagnostics captured by the GUI after the user approves submission.

```
#include <applications/gui/genesys/services/IssueReportRelayService.h>
```

Public Attributes

- QString **applicationName**
- QString **applicationVersion**
- QString **qtVersion**
- QString **operatingSystem**
- QString **currentModelName**
- QString **currentModelSimulanguage**
- QString **consoleTail**
- QString **simulationTail**
- QString **reportsTail**
- QString **processLogTail**
- QByteArray **screenshotPngBase64**

15.380.1 Detailed Description

Runtime diagnostics captured by the GUI after the user approves submission.

The documentation for this struct was generated from the following file:

- applications/gui/genesys/services/IssueReportRelayService.h

15.381 IssueReportSubmissionResult Struct Reference

Relay submission outcome returned to the controller layer.

```
#include <applications/gui/genesys/services/IssueReportRelayService.h>
```

Public Attributes

- bool **success** = false
- int **httpStatus** = 0
- int **issueNumber** = 0
- QString **issueUrl**
- QString **errorMessage**
- QString **responseBody**

15.381.1 Detailed Description

Relay submission outcome returned to the controller layer.

The documentation for this struct was generated from the following file:

- applications/gui/genesys/services/IssueReportRelayService.h

15.382 Json Struct Reference

Public Types

- enum **Kind** {
 null , **boolean** , **number** , **string** ,
 array , **object** }

Public Attributes

- enum `Json::Kind` **kind**
- double **value**
- `std::string` **text**
- `std::vector< std::unique_ptr< Json > >` **sequence**
- `std::unordered_map< std::string, std::unique_ptr< Json > >` **fields**

15.382.1 Member Data Documentation

15.382.1.1 fields

```
std::unordered_map<std::string, std::unique_ptr<Json> > Json::fields
```

Initial value:

```
{  
}
```

15.382.1.2 sequence

```
std::vector<std::unique_ptr<Json> > Json::sequence
```

Initial value:

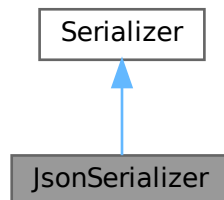
```
{  
}
```

The documentation for this struct was generated from the following file:

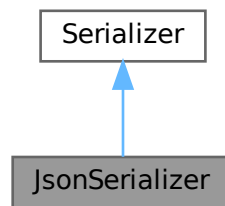
- `kernel/simulator/persistence/JsonSerializer.cpp`

15.383 JsonSerializer Class Reference

Inheritance diagram for JsonSerializer:



Collaboration diagram for JsonSerializer:



Public Member Functions

- **JsonSerializer** ([Model](#) *model)
- [PersistenceRecord](#) * **newPersistenceRecord** () override
Creates an empty persistence record.
- bool **dump** (std::ostream &output) override
Writes formatted contents to a given output stream.
- bool **load** (std::istream &input) override
Attempts to load contents from a given input stream.
- bool **get** (const std::string &name, [PersistenceRecord](#) *entry) override
Tries to fetch an entry with the given label as a key.
- bool **put** (const std::string name, const std::string type, const Util::identification id, [PersistenceRecord](#) *fields) override
Upserts an entry into this structure.
- int **for_each** (std::function< int(const std::string &) > delegate) override
Iterates over this structure's entries, ordered by ID.

15.383.1 Member Function Documentation

15.383.1.1 dump()

```
bool JsonSerializer::dump (
    std::ostream & output ) [override], [virtual]
```

Writes formatted contents to a given output stream.

Parameters

<i>output</i>	output stream to write to.
---------------	----------------------------

Returns

whether the current state of this structure's contents was deemed valid for serialization.

Implements [Serializer](#).

15.383.1.2 for_each()

```
int JsonSerializer::for_each (
    std::function< int(const std::string &) > delegate ) [override], [virtual]
```

Iterates over this structure's entries, ordered by ID.

Parameters

<i>delegate</i>	delegate which is applied to every entry-identifying key, until it returns a non-zero value.
-----------------	--

Returns

either zero, or the first non-zero value returned by the delegate.

Implements [Serializer](#).

15.383.1.3 get()

```
bool JsonSerializer::get (
    const std::string & name,
    PersistenceRecord * entry ) [override], [virtual]
```

Tries to fetch an entry with the given label as a key.

Parameters

<i>name</i>	a label which uniquely identifies an entry.
<i>entry</i>	(out-param) a pointer to a caller-owned entry, which is filled on success.

Returns

whether the entry was found and the out-param was actually written to.

Implements [Serializer](#).

15.383.1.4 load()

```
bool JsonSerializer::load (
    std::istream & input ) [override], [virtual]
```

Attempts to load contents from a given input stream.

Returns

whether the input was well-formatted and loading was successful.

Implements [Serializer](#).

15.383.1.5 newPersistenceRecord()

```
PersistenceRecord * JsonSerializer::newPersistenceRecord ( ) [override], [virtual]
```

Creates an empty persistence record.

Although the returned record can be freed normally with `delete`, care must be taken to only use records which come from a implementation, when configuring that implementation's contents.

Implements [Serializer](#).

15.383.1.6 put()

```
bool JsonSerializer::put (
    const std::string name,
    const std::string type,
    const Util::identification id,
    PersistenceRecord * fields ) [override], [virtual]
```

Upserts an entry into this structure.

Parameters

<i>name</i>	a label which uniquely identifies the entry.
-------------	--

Parameters

<i>type</i>	a type name.
<i>id</i>	a component/datadef ID number.
<i>fields</i>	a persistence record for the entry, whose contents will be copied if valid.

Returns

whether the entry was valid and was actually copied into this structure.

Implements [Serializer](#).

The documentation for this class was generated from the following files:

- kernel/simulator/persistence/JsonSerializer.h
- kernel/simulator/persistence/JsonSerializer.cpp

15.384 KernelAccessorProbe Struct Reference

Public Member Functions

- std::string **getName** () const
- void **setName** (std::string newName)

Public Attributes

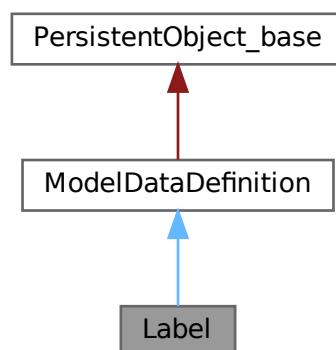
- std::string **name** = "alpha"

The documentation for this struct was generated from the following file:

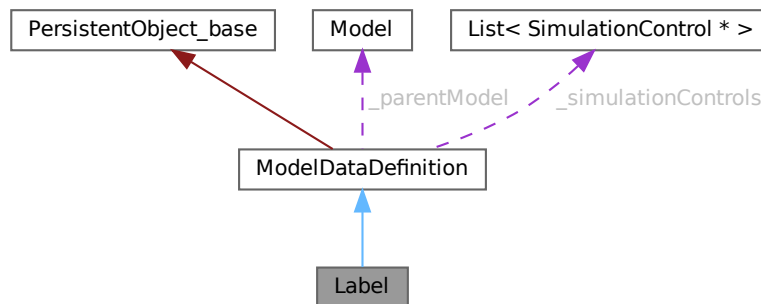
- tests/unit/test_simulator_support.cpp

15.385 Label Class Reference

Inheritance diagram for Label:



Collaboration diagram for Label:



Public Member Functions

- **Label** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- void **setLabel** (std::string _label)
- std::string **getLabel** () const
- void **setEnterIntoLabelComponent** ([ModelComponent](#) *enteringLabelComponent)
- [ModelComponent](#) * **getEnterIntoLabelComponent** () const
- void **sendEntityToLabelComponent** ([Entity](#) *entity, double timeDelay)

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- [Model](#) * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.

- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) (TraceManager::Level level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void `trace` (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))

not just an easy access to trace manager, but wrappers to check if specific trace level applies

- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

Traces text to the simulation output.

- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification_id
- std::string _typename
- bool _reportStatistics
- bool _hasChanged
- unsigned int _modelLevel = 0
- [Model](#) * _parentModel
- [List](#)< [SimulationControl](#) * > * _simulationControls = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level _traceLevelSpecific
- bool _traceLevelSpecificEnabled = false

15.385.1 Member Function Documentation

15.385.1.1 _check()

```
bool Label::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.385.1.2 _loadInstance()

```
bool Label::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.385.1.3 _saveInstance()

```
void Label::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.385.1.4 show()

```
std::string Label::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- plugins/data/Logic/Label.h
- plugins/data/Logic/Label.cpp

15.386 QCPLabelPainterPrivate::LabelData Struct Reference

Public Attributes

- [AnchorSide](#) **side**
- double **rotation**
- QTransform **transform**
- QString **basePart**
- QString **expPart**
- QString **suffixPart**
- QRect **baseBounds**
- QRect **expBounds**
- QRect **suffixBounds**
- QRect **totalBounds**
- QRect **rotatedTotalBounds**
- QFont **baseFont**
- QFont **expFont**
- QColor **color**

The documentation for this struct was generated from the following file:

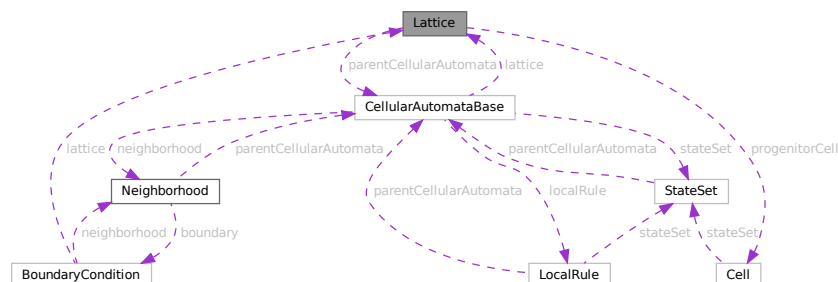
- applications/gui/genesys/qcustomplot.h

15.387 Lattice Class Reference

n-dimensional lattice

```
#include <plugins/components/ModalModel/CellularAutomata/Lattice.h>
```

Collaboration diagram for Lattice:



Public Member Functions

- **Lattice** ([CellularAutomataBase](#) *parentCellularAutomata, [Cell](#) *progenitorCell=nullptr, std::vector< unsigned short > dimensions={}, LatticeType latticeType=LatticeType::RETICULAR)
- **Lattice** (const [Lattice](#) &orig)
- virtual std::string **show** ()
- bool **init** ()
init all cells
- virtual long **cellNDimPosition2Number** (const std::vector< int > position)
Maps n-dimensional to cell number.
- virtual std::vector< int > **cellNumber2NDimPosition** (const long cellNumber)
Maps cell number to n-dimensional.
- [Cell](#) * **getCell** (const long cellNumber)
- [Cell](#) * **getCell** (const std::vector< int > position)
- void **setCell** (const std::vector< int > position, [Cell](#) *cell)
- bool **setCellState** (std::vector< int > position, [State](#) *state=nullptr, [Cell](#) *cell=nullptr)
- bool **setCellState** (long cellNumber, [State](#) *state=nullptr, [Cell](#) *cell=nullptr)
- void **setProgenitorCell** ([Cell](#) *progenitorCell)
set cell used to fill the lattice
- [Cell](#) * **getProgenitorCell** () const
- bool **setNumCells** (unsigned short dimension, unsigned int numCells, bool createDimensions=true)
- unsigned short **getNumDimensions** ()
- unsigned int **getNumCell** (unsigned short dimension)
- LatticeType **getLatticeType** () const
- unsigned long **getCellsSize** () const
- unsigned long **getTotalCells** () const
- unsigned long **calculateTotalCells** ()
- bool **isInit** () const
- void **setDimensions** (std::vector< unsigned short > dimensions)
- std::vector< unsigned short > **getDimensions** () const
- unsigned short **getDimension** (unsigned short dimensionNumber) const
- std::vector< [Cell](#) * > **getCells** () const

Protected Attributes

- [CellularAutomataBase](#) * **parentCellularAutomata**
- std::vector< [Cell](#) * > **cells**
- std::vector< unsigned short > **dimensions**
- [Cell](#) * **progenitorCell**
- LatticeType **latticeType**
- bool **hasBeenInit**
- unsigned long **totalCells**

15.387.1 Detailed Description

n-dimensional lattice

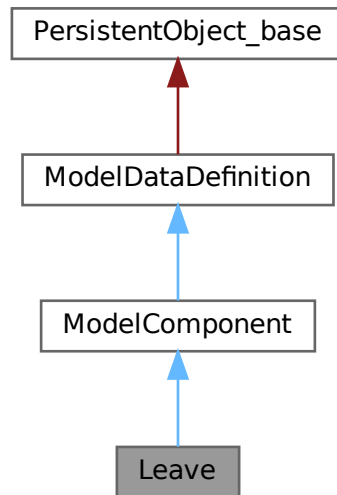
The documentation for this class was generated from the following files:

- plugins/components/ModalModel/CellularAutomata/Lattice.h
- plugins/components/ModalModel/CellularAutomata/Lattice.cpp

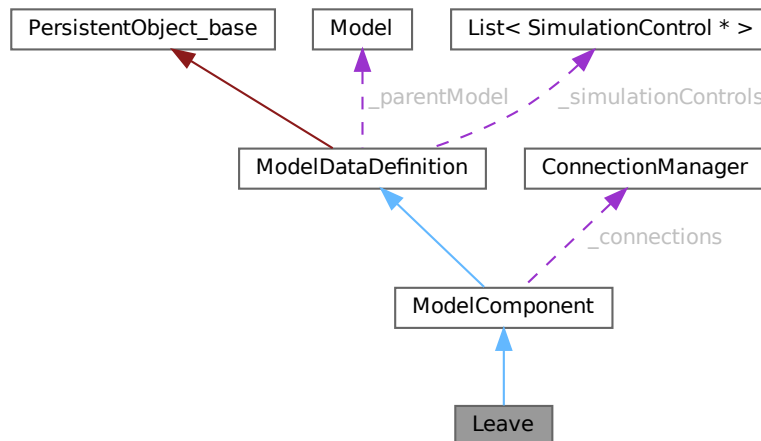
15.388 Leave Class Reference

```
#include <plugins/components/MaterialHandling/Leave.h>
```

Inheritance diagram for Leave:



Collaboration diagram for Leave:



Public Member Functions

- **Leave** (`Model` *model, std::string name="")
- virtual std::string **show** () override
virtual public methods
- void **setStation** (`Station` *_station)

- void **setStationName** (std::string stationName)
- [Station](#) * **getStation** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- [Model](#) * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * **getAttachedData** () const

- Returns the full map of attached data definitions.*
- bool `hasChanged` () const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged` (bool `hasChanged`)
Updates the changed flag for this data definition.
- unsigned int `getLevel` () const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel` (unsigned int `_modelLevel`)
Sets the model nesting level for this definition.
- List< `SimulationControl` * > * `getSimulationControls` () const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level` `getTraceLevelSpecific` () const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` `level`)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` `traceLevelSpecific`, bool `traceLevelSpecificEnabled`)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool `traceLevelSpecificEnabled`)
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation` * `GetPluginInformation` ()
- static `ModelComponent` * `LoadInstance` (`Model` *`model`, `PersistenceRecord` *`fields`)
- static `ModelDataDefinition` * `NewInstance` (`Model` *`model`, std::string `name`="")

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent` * `LoadInstance` (`Model` *`model`, `PersistenceRecord` *`fields`)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check` (`ModelComponent` *`component`)
Validates a component instance.
- static void `CreateInternalData` (`ModelComponent` *`component`)
Creates internal data required by a component instance.
- static void `SaveInstance` (`PersistenceRecord` *`fields`, `ModelComponent` *`component`)
Serializes a component instance into a persistence record.
- static void `DispatchEvent` (`Event` *`event`)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition` * `LoadInstance` (`Model` *`model`, `PersistenceRecord` *`fields`, bool `insertIntoModel`)
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition` * `NewInstance` (`Model` *`model`, std::string `name`="")
Creates a new instance of a data definition using the plugin registry.
- static void `SaveInstance` (`PersistenceRecord` *`fields`, `ModelDataDefinition` *`modeldatum`)
Serializes a data definition instance into a persistence record.
- static bool `Check` (`ModelDataDefinition` *`modeldatum`, std::string &`errorMessage`)
Validates a data definition instance.

- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_createInternalStatisticReporters](#) () override
- virtual void [_createEditableDataDefinitions](#) () override
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()

- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.388.1 Detailed Description

[Leave](#) module DESCRIPTION The [Leave](#) module is used to transfer an entity to a station or module. An entity may be transferred in two ways. It can be transferred to a module that defines a station by referencing the station and routing, conveying, or transporting to that station, or a graphical connection can be used to transfer an entity to another module. When an entity arrives at a [Leave](#) module, it may wait to obtain a transfer device (resource, transporter, or conveyor). When the transfer device has been obtained, the entity may experience a loading delay. Finally, the entity is transferred from this module to a destination module or station. TYPICAL USES The end of a part's production in a series of parallel processes where the part needs a forklift to be transferred to shipping PROMPTS Prompt Description Name Unique name of the module that will be displayed in the flowchart. Allocation Type of category to which the entity's incurred delay time and cost will be added. [Delay](#) Specifies a load time incurred after getting a transfer device. Units Time units used for the delay time. Transfer Out Determines whether a resource ([Seize Resource](#)), transporter (Request Transporter), or conveyor ([Access Conveyor](#)) is required prior to transferring the entity out of this module. Priority Indicates the priority of the module when either seizing a resource or requesting a transporter when there are entities waiting for that resource/transporter from other modules. This field is not visible when the Transfer Type is None or [Access Conveyor](#). Transporter Name Name of the transporter

to request. [Queue](#) Type Type of queue, either a single [Queue](#), queue [Set](#), Internal queue, [Attribute](#), or Expression. [Queue](#) Name Name of the individual queue. [Queue Set](#) Name Name of the queue set. [Set](#) Index Defines the index into the queue set. Note that this is the index into the set and not the name of the queue in the set. [Queue Attribute](#) Name The attribute name that will be evaluated to indicate which queue is to be used. [Queue](#) Expression The expression that will be evaluated to indicate which queue is to be used. Selection Rule Method of selecting among available transporters in a set. Cyclical will cycle through available members. Random will randomly select a member. Preferred Order will always select the first available member. Specific Member requires an input attribute value to specify which member of the set (previously saved in the Save [Attribute](#) field). Largest Distance selects the transporter farthest away, and Smallest Distance selects the closest transporter. Save [Attribute Attribute](#) name used to store the index number into the set of the member that is chosen. This attribute can later be referenced with the Specific Member selection rule. Active when Transfer Out is Request Transporter. Index [Set Attribute](#) name whose value identifies the index number into the set of the member requested. The entity must have a value for the attribute before utilizing this option. [Resource](#) Type Type of resource for seizing, either specifying a particular [Resource](#), selecting from a pool of resources (that is, a resource [Set](#)), [Attribute](#), or Expression. [Resource](#) Name Name of the resource to seize. Conveyor Name Name of the conveyor to access.

15.388.2 of Cells Number of contiguous cells the entity requires.

Connect Type Determines if the entity is to [Route](#), Convey, or Transport to another station or Connect to another module. Move Time Time to route from this module to the destination station. Units Time units used for the move time. [Station](#) Type The entity's destination station type either an individual [Station](#), a station based on an [Attribute](#) or Expression value, or By [Sequence](#). [Station](#) Name Name of the individual destination station. [Attribute](#) Name The attribute name that will be evaluated to indicate the station. Expression The expression that will be evaluated to indicate the station.

15.388.3 Member Function Documentation

15.388.3.1 `_check()`

```
bool Leave::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.388.3.2 `_createEditableDataDefinitions()`

```
void Leave::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.388.3.3 `_createInternalStatisticReporters()`

```
void Leave::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.388.3.4 `_loadInstance()`

```
bool Leave::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [ModelComponent](#).

15.388.3.5 `_onDispatchEvent()`

```
void Leave::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.388.3.6 _saveInstance()

```
void Leave::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.388.3.7 show()

```
std::string Leave::show ( ) [override], [virtual]
virtual public methods
```

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/MaterialHandling/Leave.h
- plugins/components/MaterialHandling/Leave.cpp

15.389 LicenceManager Class Reference

```
#include <kernel/simulator/LicenceManager.h>
```

Public Member Functions

- **LicenceManager** ([Simulator](#) *simulator)
- const std::string **showLicence** () const
- const std::string **showLimits** () const
- const std::string **showActivationCode** () const
- bool **lookforActivationCode** ()
- bool **insertActivationCode** ()
- void **removeActivationCode** ()
- unsigned int **getModelComponentsLimit** ()
- unsigned int **getModelDatasLimit** ()
- unsigned int **getEntityLimit** ()
- unsigned int **getHostsLimit** ()
- unsigned int **getThreadsLimit** ()

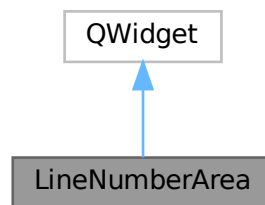
15.389.1 Detailed Description

[LicenceManager](#) just prints a licence agreement and checks for some resource limits.
The documentation for this class was generated from the following files:

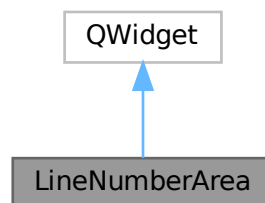
- kernel/simulator/LicenceManager.h
- kernel/simulator/LicenceManager.cpp

15.390 LineNumberArea Class Reference

Inheritance diagram for LineNumberArea:



Collaboration diagram for LineNumberArea:



Public Member Functions

- **LineNumberArea** ([CodeEditor](#) *editor)
- QSize **sizeHint** () const override

Protected Member Functions

- void **paintEvent** (QPaintEvent *event) override

The documentation for this class was generated from the following file:

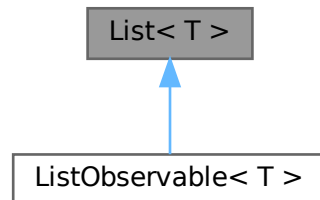
- applications/gui/genesys/codeeditor/LineNumberArea.h

15.391 List< T > Class Template Reference

Lightweight wrapper around `std::list` with convenience helpers used by the kernel.

```
#include <kernel/util/List.h>
```

Inheritance diagram for List< T >:



Public Types

- `using CompFunc` = `std::function< bool(const T, const T) >`

Public Member Functions

- **List** ()
Creates an empty list and positions the internal iterator at the beginning.
- **List** (const List< T > &origin)
Creates a copy of the source list.
- List< T > & **operator=** (const List< T > &origin)
- **unsigned int** **size** ()
Returns the number of stored elements.
- **bool** **empty** ()
Indicates whether the list is empty.
- **void** **clear** ()
Removes all elements from the list.
- **void** **pop_front** ()
Removes the first element from the list.
- template<class Compare >
void **sort** (Compare comp)
Sorts elements using the provided comparator.
- `std::list< T > * list () const`
Returns direct access to the encapsulated `std::list` structure.
- **T** **create** ()
Creates a new default element of type T.
- template<typename U >
T **create** (U arg)
Creates a new T element using a construction argument.
- `std::string` **show** ()
Generates a textual representation of elements for debugging.
- `std::list< T >::iterator` **find** (T element)
Searches for an element and returns an iterator to it (or `end()`).
- **void** **insert** (T element)

- Inserts an element preserving the current ordering policy.*
- **void remove** (T element)
Removes all occurrences of the provided element.
- **void setAtRank** (unsigned int rank, T element)
Replaces (or appends) an element at a specific rank.
- **T getAtRank** (unsigned int rank)
Returns the element at the provided rank.
- **T next** ()
Advances the internal iterator and returns the next element.
- **T front** ()
Moves to the beginning and returns the first element.
- **T last** ()
Moves to the end and returns the last element.
- **T previous** ()
Moves the internal iterator backward and returns the previous element.
- **T current** ()
Returns the element at the current internal iterator position.
- **void setSortFunc** (CompFunct _sortFunc)
Sets the comparison function used for ordered insertions.

Protected Attributes

- std::list< T > * **_list**
- CompFunct **_sortFunc**
- std::list< T >::iterator **_it**
Default function: insert at the end of the list.

15.391.1 Detailed Description

template<typename T>
class List< T >

Lightweight wrapper around `std::list` with convenience helpers used by the kernel.

The class centralizes insertion/removal, rank-based access and iterator-like navigation patterns heavily used in simulator internals and plugin infrastructure.

15.391.2 Member Data Documentation

15.391.2.1 _sortFunc

template<typename T >
CompFunct List< T >::_sortFunc [protected]

Initial value:

```
{[] (const T, const T) {
    return false;
}}
```

The documentation for this class was generated from the following files:

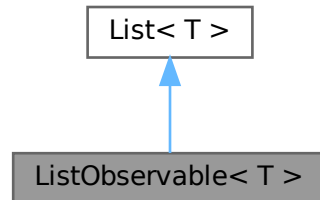
- applications/gui/genesys/services/GraphicalModelBuilder.h
- kernel/util/List.h

15.392 ListObservable< T > Class Template Reference

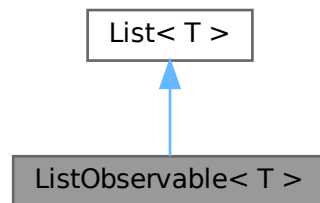
Observable-oriented list variant used by kernel structures.

```
#include <kernel/util/ListObservable.h>
```

Inheritance diagram for ListObservable< T >:



Collaboration diagram for ListObservable< T >:



Public Types

- `using CompFunc` = `std::function< bool(const T, const T) >`

Public Types inherited from List< T >

- `using CompFunc` = `std::function< bool(const T, const T) >`

Public Member Functions

- `ListObservable ()`
Initializes an empty observable list.
- `void clear ()`
Removes all elements from the observable list.
- `void pop_front ()`
Removes the first element from the observable list.
- `void insert (T element)`
Inserts an element respecting the configured ordering.
- `void remove (T element)`
Removes an element from the observable list.

- **void setAtRank** ([unsigned int](#) rank, T element)

Sets the element at a specific list index.

Public Member Functions inherited from [List< T >](#)

- **List** ()
Creates an empty list and positions the internal iterator at the beginning.
- **List** ([const List< T >](#) &[origin](#))
Creates a copy of the source list.
- [List< T >](#) & **operator=** ([const List< T >](#) &[origin](#))
- [unsigned int](#) **size** ()
Returns the number of stored elements.
- [bool](#) **empty** ()
Indicates whether the list is empty.
- **void clear** ()
Removes all elements from the list.
- **void pop_front** ()
Removes the first element from the list.
- [template<class Compare >](#)
void sort ([Compare comp](#))
Sorts elements using the provided comparator.
- [std::list< T > * list](#) () [const](#)
Returns direct access to the encapsulated `std::list` structure.
- T **create** ()
Creates a new default element of type T.
- [template<typename U >](#)
T **create** ([U arg](#))
Creates a new T element using a construction argument.
- [std::string](#) **show** ()
Generates a textual representation of elements for debugging.
- [std::list< T >::iterator](#) **find** (T element)
Searches for an element and returns an iterator to it (or `end()`).
- [void insert](#) (T element)
Inserts an element preserving the current ordering policy.
- [void remove](#) (T element)
Removes all occurrences of the provided element.
- [void setAtRank](#) ([unsigned int](#) rank, T element)
Replaces (or appends) an element at a specific rank.
- T **getAtRank** ([unsigned int](#) rank)
Returns the element at the provided rank.
- T **next** ()
Advances the internal iterator and returns the next element.
- T **front** ()
Moves to the beginning and returns the first element.
- T **last** ()
Moves to the end and returns the last element.
- T **previous** ()
Moves the internal iterator backward and returns the previous element.
- T **current** ()
Returns the element at the current internal iterator position.
- [void setSortFunc](#) ([CompFunct _sortFunc](#))
Sets the comparison function used for ordered insertions.

Additional Inherited Members**Protected Attributes inherited from [List< T >](#)**

- `std::list< T > * _list`
- `CompFunct _sortFunc`
- `std::list< T >::iterator _it`

Default function: insert at the end of the list.

15.392.1 Detailed Description

`template<typename T>`
`class ListObservable< T >`

Observable-oriented list variant used by kernel structures.

This template specializes [List](#) behavior for contexts where list mutations are relevant to external observers/managers, while preserving familiar list helpers.

The documentation for this class was generated from the following file:

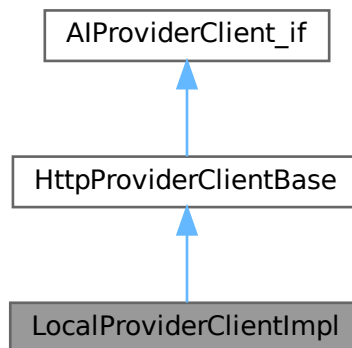
- `kernel/util/ListObservable.h`

15.393 LocalProviderClientImpl Class Reference

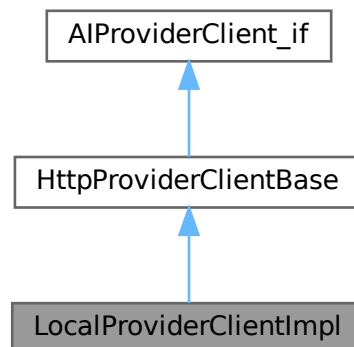
AI provider client for local model servers (Ollama, LM Studio, etc.).

```
#include <tools/AIAssistant/LocalProviderClientImpl.h>
```

Inheritance diagram for LocalProviderClientImpl:



Collaboration diagram for LocalProviderClientImpl:



Public Member Functions

- **LocalProviderClientImpl ()**
Constructs the client targeting the default Ollama endpoint.
- `std::string` **getProviderName ()** const override
Returns a human-readable provider name (e.g. "OpenAI", "Anthropic").
- `bool` **isAvailable ()** const override
Returns `true` when a base URL is configured.

Public Member Functions inherited from [HttpProviderClientBase](#)

- **HttpProviderClientBase** (const [HttpProviderClientBase](#) &)=delete
- [HttpProviderClientBase](#) & **operator=** (const [HttpProviderClientBase](#) &)=delete
- `void` **setBaseUrl** (const `std::string` &url) override
Sets the base URL for the provider API endpoint.
- `std::string` **getBaseUrl ()** const override
Returns the currently configured base URL.
- `void` **setTimeoutSeconds** (unsigned int seconds) override
Sets the HTTP request timeout in seconds.
- unsigned int **getTimeoutSeconds ()** const override
Returns the configured HTTP request timeout.
- `void` **setApiKey** (const `std::string` &apiKey) override
Stores the API key in memory.
- `void` **clearApiKey ()** override
Overwrites (best-effort) and clears the in-memory API key.
- `bool` **hasApiKey ()** const override
Returns `true` when a non-empty API key is held.
- [ProviderResponse](#) **send** (const [ProviderRequest](#) &request) override
Executes the HTTP request via curl and returns the provider response.

Protected Member Functions

- `std::string _buildRequestBody` (const [ProviderRequest](#) &request) const override
Serialises `request` to the provider's JSON wire format.
- [ProviderResponse](#) `_parseResponseBody` (const std::string &rawBody, int httpStatus) const override
Deserialises `rawBody` (the full curl stdout) into a [ProviderResponse](#).
- `std::string _authorizationHeader` () const override
Returns the Authorization header when an API key is set, empty otherwise.

Protected Member Functions inherited from [HttpProviderClientBase](#)

- virtual `std::vector< std::string > _extraRequestHeaders` () const
Returns additional HTTP headers to send with every request.

Additional Inherited Members

Static Public Member Functions inherited from [HttpProviderClientBase](#)

- static `std::string _escapeJsonString` (const std::string &s)
Escapes a plain string for safe embedding inside a JSON string value.
- static `std::string _extractJsonString` (const std::string &json, const std::string &key)
Extracts the value of a top-level JSON string field by key. Returns an empty string when not found or value is not a quoted string.
- static `int _extractJsonInt` (const std::string &json, const std::string &key)
Extracts the value of a top-level JSON integer field by key. Returns 0 when not found.
- static `double _extractJsonDouble` (const std::string &json, const std::string &key)
Extracts the value of a top-level JSON number field (int or float) by key. Returns 0.0 when not found.
- static `std::string _buildMessagesArray` (const std::vector< [ChatMessage](#) > &messages)
Serialises a vector of [ChatMessage](#) objects to a JSON array string.
- static `std::string _extractFirstArrayElement` (const std::string &json, const std::string &arrayKey)
Extracts the raw substring of the first element of a named JSON array. Returns an empty string when not found.

Protected Attributes inherited from [HttpProviderClientBase](#)

- `std::string _baseUrl`
- `unsigned int _timeoutSeconds = 30`
- `std::string _apiKey`
In-memory API key.

15.393.1 Detailed Description

AI provider client for local model servers (Ollama, LM Studio, etc.).

Targets any OpenAI-compatible chat completions endpoint reachable locally. Default base URL: <http://localhost:11434/v1/chat/completions> (Ollama default).

Differences from [OpenAIProviderClientImpl](#):

- No API key is required; `isAvailable()` returns true when the base URL is set.
- When a non-empty API key IS set (some local servers require a token), it is sent as `Authorization: Bearer <key>`. When no key is set, the Authorization header is omitted.
- Extended reasoning (highReasoningMode) adds `"stream": false` only; no `reasoning_effort` parameter (not supported by most local servers).
- The JSON request/response format otherwise mirrors OpenAI chat completions.

15.393.1.1 Tested server compatibility

- Ollama ($\geq 0.1.29$): set the base URL to <http://localhost:11434/v1/chat/completions>.
- LM Studio local server: set base URL to <http://localhost:1234/v1/chat/completions>.
- llama.cpp server: set base URL to <http://localhost:8080/v1/chat/completions>.

15.393.2 Member Function Documentation

15.393.2.1 `_authorizationHeader()`

```
std::string LocalProviderClientImpl::_authorizationHeader ( ) const [override], [protected], [virtual]
```

Returns the Authorization header when an API key is set, empty otherwise.

Implements [HttpProviderClientBase](#).

15.393.2.2 `_buildRequestBody()`

```
std::string LocalProviderClientImpl::_buildRequestBody (
    const ProviderRequest & request ) const [override], [protected], [virtual]
```

Serialises request to the provider's JSON wire format.

Implements [HttpProviderClientBase](#).

15.393.2.3 `_parseResponseBody()`

```
ProviderResponse LocalProviderClientImpl::_parseResponseBody (
    const std::string & rawBody,
    int httpStatus ) const [override], [protected], [virtual]
```

Deserialises rawBody (the full curl stdout) into a [ProviderResponse](#).

Parameters

<i>rawBody</i>	Complete HTTP response body as received from curl.
<i>httpStatus</i>	HTTP status code captured from curl exit / response line, or 0 when unavailable.

Implements [HttpProviderClientBase](#).

15.393.2.4 `getProviderName()`

```
std::string LocalProviderClientImpl::getProviderName ( ) const [override], [virtual]
```

Returns a human-readable provider name (e.g. "OpenAI", "Anthropic").

Implements [AIProviderClient_if](#).

15.393.2.5 `isAvailable()`

```
bool LocalProviderClientImpl::isAvailable ( ) const [override], [virtual]
```

Returns true when a base URL is configured.

Unlike cloud providers, a local server does not require an API key.

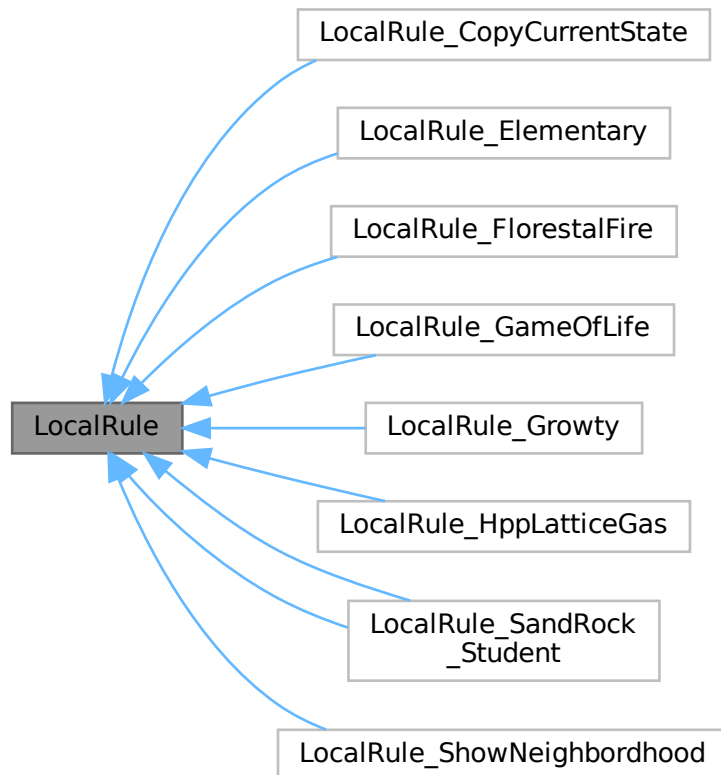
Reimplemented from [HttpProviderClientBase](#).

The documentation for this class was generated from the following files:

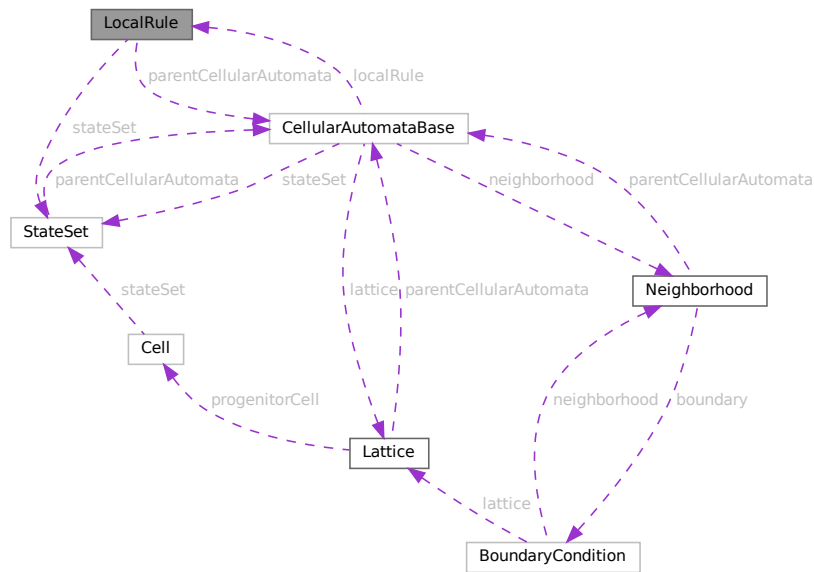
- tools/AIAssistant/LocalProviderClientImpl.h
- tools/AIAssistant/LocalProviderClientImpl.cpp

15.394 LocalRule Class Reference

Inheritance diagram for LocalRule:



Collaboration diagram for LocalRule:



Public Member Functions

- **LocalRule** ([CellularAutomataBase](#) *parentCellularAutomata, [StateSet](#) *stateSet=nullptr)
- **LocalRule** (const [LocalRule](#) &orig)
- virtual void **applyRule** ([Cell](#) *cell)=0
Pure virtual method that has to be overriden by derived class, responsible for applying the local rule to a cell.
- void **setStateSet** ([StateSet](#) *stateSet)

Protected Attributes

- [CellularAutomataBase](#) * **parentCellularAutomata**
- [StateSet](#) * **stateSet**

15.394.1 Member Function Documentation

15.394.1.1 applyRule()

```
virtual void LocalRule::applyRule (
    Cell * cell ) [pure virtual]
```

Pure virtual method that has to be overriden by derived class, responsible for applying the local rule to a cell.

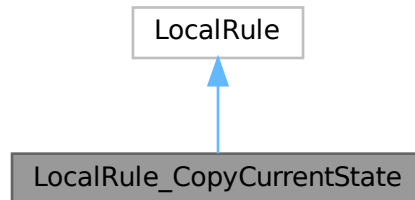
Implemented in [LocalRule_ShowNeighborhood](#), [LocalRule_CopyCurrentState](#), [LocalRule_Elementary](#), [LocalRule_FlorestalFire](#), [LocalRule_GameOfLife](#), [LocalRule_Growty](#), [LocalRule_HppLatticeGas](#), [LocalRule_SandRock_Student](#), and [LocalRule_SandRock_Student](#).

The documentation for this class was generated from the following file:

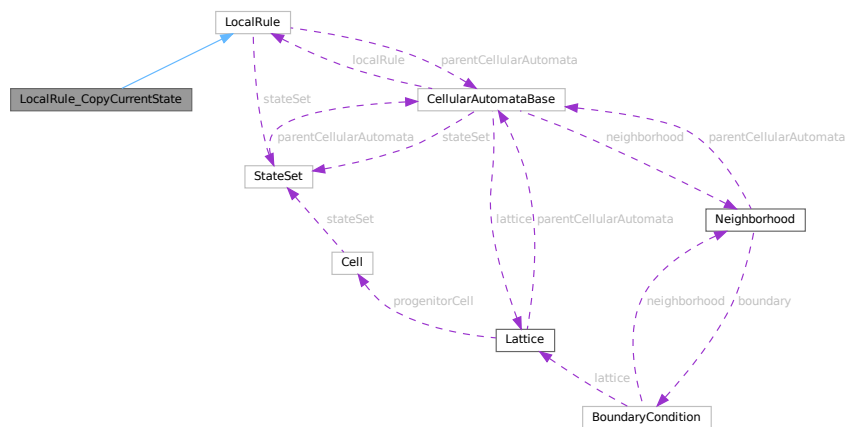
- plugins/components/ModalModel/CellularAutomata/LocalRule.h

15.395 LocalRule_CopyCurrentState Class Reference

Inheritance diagram for LocalRule_CopyCurrentState:



Collaboration diagram for LocalRule_CopyCurrentState:



Public Member Functions

- **LocalRule_CopyCurrentState** ([CellularAutomataBase](#) *parentCellularAutomata)
- void **applyRule** ([Cell](#) *cell) override

Pure virtual method that has to be overridden by derived class, responsible for applying the local rule to a cell.

Public Member Functions inherited from [LocalRule](#)

- **LocalRule** ([CellularAutomataBase](#) *parentCellularAutomata, [StateSet](#) *stateSet=nullptr)
- **LocalRule** (const [LocalRule](#) &orig)
- void **setStateSet** ([StateSet](#) *stateSet)

Additional Inherited Members

Protected Attributes inherited from [LocalRule](#)

- [CellularAutomataBase](#) * **parentCellularAutomata**
- [StateSet](#) * **stateSet**

15.395.1 Member Function Documentation

15.395.1.1 applyRule()

```
void LocalRule_CopyCurrentState::applyRule (
    Cell * cell ) [inline], [override], [virtual]
```

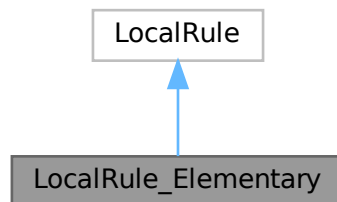
Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell. Implements [LocalRule](#).

The documentation for this class was generated from the following file:

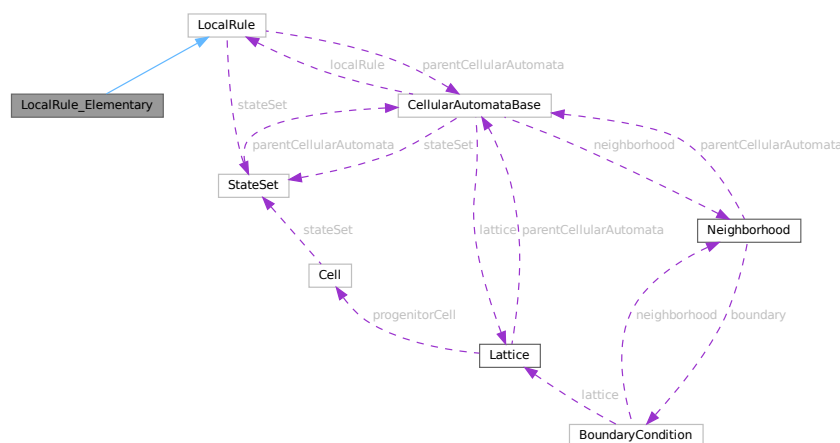
- applications/gui/genesys/cellular_automata/CellularAutomataDemoBuilder.h

15.396 LocalRule_Elementary Class Reference

Inheritance diagram for LocalRule_Elementary:



Collaboration diagram for LocalRule_Elementary:



Public Member Functions

- **LocalRule_Elementary** ([CellularAutomataBase](#) *parentCellularAutomata, uint8_t ruleNumber=30, [StateSet](#) *stateSet=nullptr)
- **LocalRule_Elementary** (const [LocalRule_Elementary](#) &orig)
- virtual void [applyRule](#) ([Cell](#) *cell) override

Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell.

- uint8_t **getRuleNumber** () const
- void **setRuleNumber** (const uint8_t ruleNumber)

Public Member Functions inherited from [LocalRule](#)

- **LocalRule** ([CellularAutomataBase](#) *parentCellularAutomata, [StateSet](#) *stateSet=nullptr)
- **LocalRule** (const [LocalRule](#) &orig)
- void **setStateSet** ([StateSet](#) *stateSet)

Additional Inherited Members

Protected Attributes inherited from [LocalRule](#)

- [CellularAutomataBase](#) * **parentCellularAutomata**
- [StateSet](#) * **stateSet**

15.396.1 Member Function Documentation

15.396.1.1 applyRule()

```
virtual void LocalRule_Elementary::applyRule (
    Cell * cell ) [inline], [override], [virtual]
```

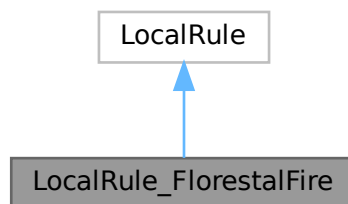
Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell. Implements [LocalRule](#).

The documentation for this class was generated from the following file:

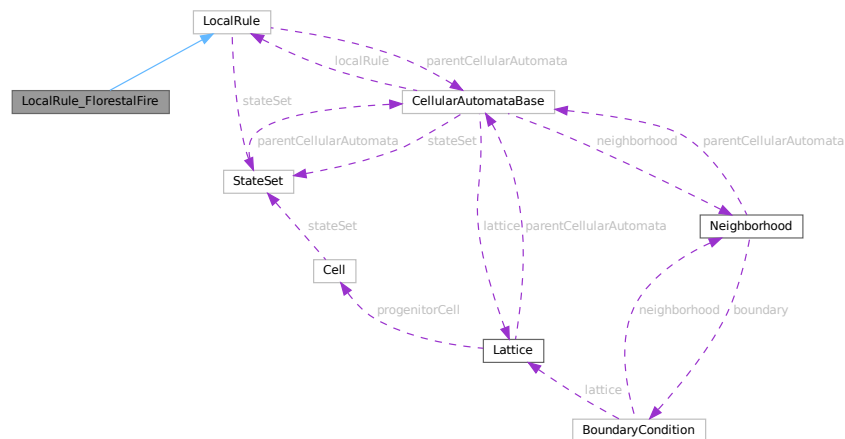
- plugins/components/ModalModel/CellularAutomata/LocalRule_Elementary.h

15.397 LocalRule_FlorestalFire Class Reference

Inheritance diagram for LocalRule_FlorestalFire:



Collaboration diagram for LocalRule_FlorestalFire:



Public Member Functions

- **LocalRule_FlorestalFire** ([CellularAutomataBase](#) *parentCellularAutomata)
- **LocalRule_FlorestalFire** (const [LocalRule_FlorestalFire](#) &orig)
- virtual void **applyRule** ([Cell](#) *cell) override

Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell.

Public Member Functions inherited from [LocalRule](#)

- **LocalRule** ([CellularAutomataBase](#) *parentCellularAutomata, [StateSet](#) *stateSet=nullptr)
- **LocalRule** (const [LocalRule](#) &orig)
- void **setStateSet** ([StateSet](#) *stateSet)

Additional Inherited Members

Protected Attributes inherited from [LocalRule](#)

- [CellularAutomataBase](#) * **parentCellularAutomata**
- [StateSet](#) * **stateSet**

15.397.1 Member Function Documentation

15.397.1.1 **applyRule()**

```
virtual void LocalRule_FlorestalFire::applyRule (
    Cell * cell ) [inline], [override], [virtual]
```

Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell.

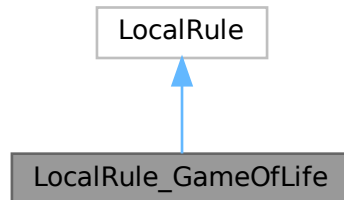
Implements [LocalRule](#).

The documentation for this class was generated from the following file:

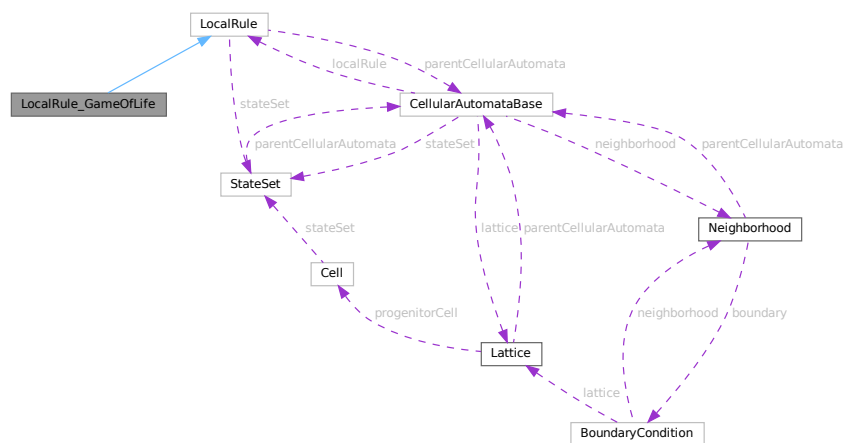
- plugins/components/ModalModel/CellularAutomata/LocalRule_FlorestalFire.h

15.398 LocalRule_GameOfLife Class Reference

Inheritance diagram for LocalRule_GameOfLife:



Collaboration diagram for LocalRule_GameOfLife:



Public Member Functions

- **LocalRule_GameOfLife** ([CellularAutomataBase](#) *parentCellularAutomata)
- **LocalRule_GameOfLife** (const [LocalRule_GameOfLife](#) &orig)
- virtual void **applyRule** ([Cell](#) *cell) override

Pure virtual method that has to be overriden by derived class, responsible for applying the local rule to a cell.

Public Member Functions inherited from [LocalRule](#)

- **LocalRule** ([CellularAutomataBase](#) *parentCellularAutomata, [StateSet](#) *stateSet=nullptr)
- **LocalRule** (const [LocalRule](#) &orig)
- void **setStateSet** ([StateSet](#) *stateSet)

Additional Inherited Members

Protected Attributes inherited from [LocalRule](#)

- [CellularAutomataBase](#) * **parentCellularAutomata**
- [StateSet](#) * **stateSet**

15.398.1 Member Function Documentation

15.398.1.1 applyRule()

```
virtual void LocalRule_GameOfLife::applyRule (
    Cell * cell ) [inline], [override], [virtual]
```

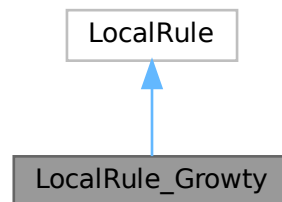
Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell. Implements [LocalRule](#).

The documentation for this class was generated from the following file:

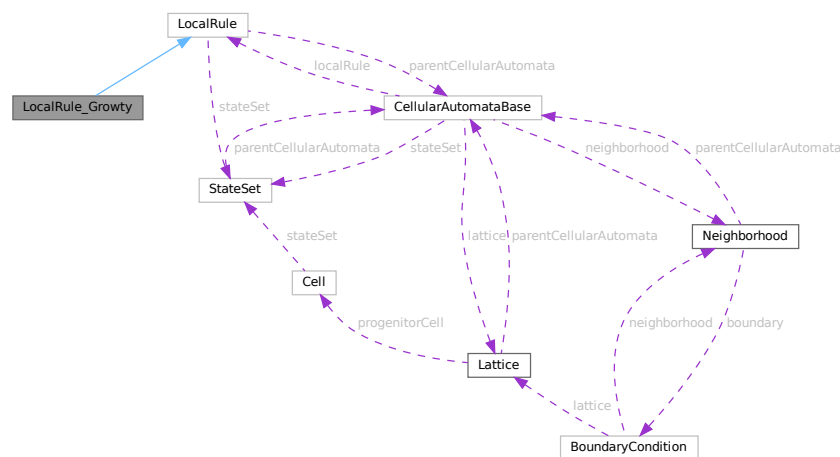
- plugins/components/ModalModel/CellularAutomata/LocalRule_GameOfLife.h

15.399 LocalRule_Growty Class Reference

Inheritance diagram for LocalRule_Growty:



Collaboration diagram for LocalRule_Growty:



Public Member Functions

- **LocalRule_Growty** ([CellularAutomataBase](#) *parentCellularAutomata)
- **LocalRule_Growty** (const [LocalRule_Growty](#) &orig)
- virtual void [applyRule](#) ([Cell](#) *cell) override

Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell.

Public Member Functions inherited from LocalRule

- **LocalRule** ([CellularAutomataBase](#) *parentCellularAutomata, [StateSet](#) *stateSet=nullptr)
- **LocalRule** (const [LocalRule](#) &orig)
- void **setStateSet** ([StateSet](#) *stateSet)

Additional Inherited Members

Protected Attributes inherited from LocalRule

- [CellularAutomataBase](#) * parentCellularAutomata
- [StateSet](#) * stateSet

15.399.1 Member Function Documentation

15.399.1.1 applyRule()

```
virtual void LocalRule_Growty::applyRule (
    Cell * cell ) [inline], [override], [virtual]
```

Pure virtual method that has to be overriden by derived class, responsible for applying the local rule to a cell.

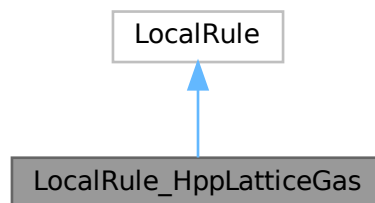
Implements [LocalRule](#).

The documentation for this class was generated from the following file:

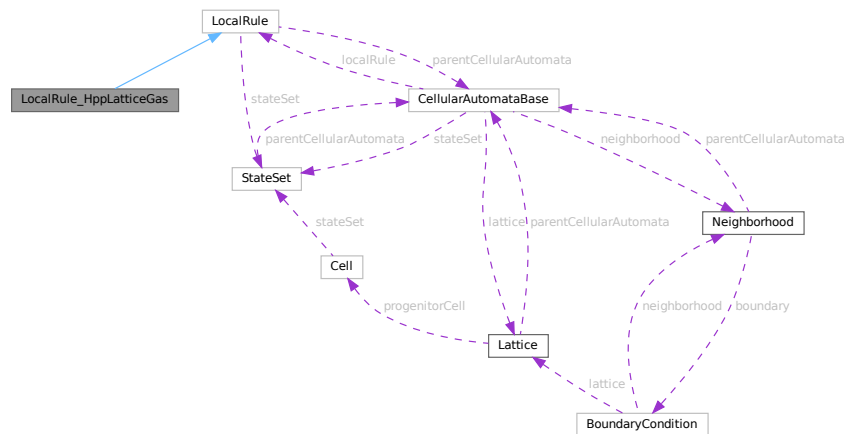
- plugins/components/ModalModel/CellularAutomata/LocalRule_Growty.h

15.400 LocalRule_HppLatticeGas Class Reference

Inheritance diagram for LocalRule_HppLatticeGas:



Collaboration diagram for LocalRule_HppLatticeGas:



Public Member Functions

- **LocalRule_HppLatticeGas** ([CellularAutomataBase](#) *parentCellularAutomata)
- void **applyRule** ([Cell](#) *cell) override

Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell.

Public Member Functions inherited from [LocalRule](#)

- **LocalRule** ([CellularAutomataBase](#) *parentCellularAutomata, [StateSet](#) *stateSet=nullptr)
- **LocalRule** (const [LocalRule](#) &orig)
- void **setStateSet** ([StateSet](#) *stateSet)

Additional Inherited Members

Protected Attributes inherited from [LocalRule](#)

- [CellularAutomataBase](#) * **parentCellularAutomata**
- [StateSet](#) * **stateSet**

15.400.1 Member Function Documentation

15.400.1.1 applyRule()

```
void LocalRule_HppLatticeGas::applyRule (
    Cell * cell ) [inline], [override], [virtual]
```

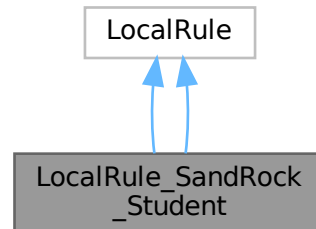
Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell.
Implements [LocalRule](#).

The documentation for this class was generated from the following file:

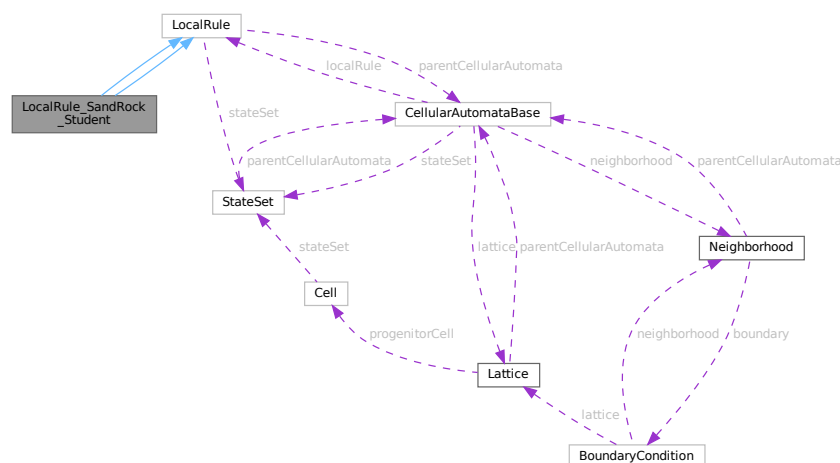
- plugins/components/ModalModel/CellularAutomata/LocalRule_HppLatticeGas.h

15.401 LocalRule_SandRock_Student Class Reference

Inheritance diagram for LocalRule_SandRock_Student:



Collaboration diagram for LocalRule_SandRock_Student:



Public Member Functions

- **LocalRule_SandRock_Student** ([CellularAutomataBase](#) *parentCellularAutomata)
- **LocalRule_SandRock_Student** (const [LocalRule_SandRock_Student](#) &orig)
- virtual void [applyRule](#) ([Cell](#) *cell) override
Pure virtual method that has to be overriden by derived class, responsable for applying the local rule to a cell.
- **LocalRule_SandRock_Student** ([CellularAutomataBase](#) *parentCellularAutomata)
- **LocalRule_SandRock_Student** (const [LocalRule_SandRock_Student](#) &orig)
- virtual void [applyRule](#) ([Cell](#) *cell) override
Pure virtual method that has to be overriden by derived class, responsable for applying the local rule to a cell.

Public Member Functions inherited from [LocalRule](#)

- **LocalRule** ([CellularAutomataBase](#) *parentCellularAutomata, [StateSet](#) *stateSet=nullptr)
- **LocalRule** (const [LocalRule](#) &orig)
- void **setStateSet** ([StateSet](#) *stateSet)

Static Public Attributes

- static constexpr long **EMPTY** = 0
- static constexpr long **SAND** = 1
- static constexpr long **ROCK** = 2

Additional Inherited Members

Protected Attributes inherited from [LocalRule](#)

- [CellularAutomataBase](#) * **parentCellularAutomata**
- [StateSet](#) * **stateSet**

15.401.1 Member Function Documentation

15.401.1.1 `applyRule()` [1/2]

```
virtual void LocalRule_SandRock_Student::applyRule (
    Cell * cell ) [inline], [override], [virtual]
```

Pure virtual method that has to be overiden by derived class, responsable for applying the local rule to a cell.
Implements [LocalRule](#).

15.401.1.2 `applyRule()` [2/2]

```
virtual void LocalRule_SandRock_Student::applyRule (
    Cell * cell ) [inline], [override], [virtual]
```

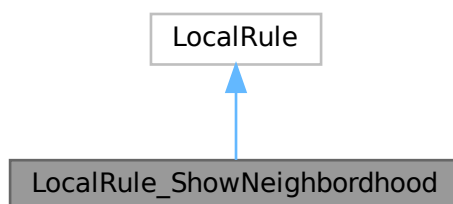
Pure virtual method that has to be overiden by derived class, responsable for applying the local rule to a cell.
Implements [LocalRule](#).

The documentation for this class was generated from the following files:

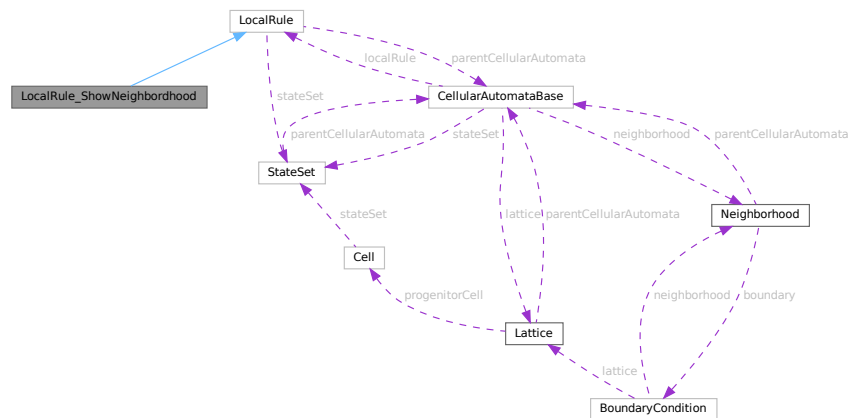
- plugins/components/ModalModel/CellularAutomata/LocalRule_SandRock.h
- plugins/components/ModalModel/CellularAutomata/LocalRule_SandRock_Student.h

15.402 [LocalRule_ShowNeighbordhood](#) Class Reference

Inheritance diagram for [LocalRule_ShowNeighbordhood](#):



Collaboration diagram for LocalRule_ShowNeighbordhood:



Public Member Functions

- **LocalRule_ShowNeighbordhood** ([CellularAutomataBase](#) *parentCellularAutomata)
- **LocalRule_ShowNeighbordhood** (const [LocalRule_ShowNeighbordhood](#) &orig)
- virtual void **applyRule** ([Cell](#) *cell)

Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell.

Public Member Functions inherited from [LocalRule](#)

- **LocalRule** ([CellularAutomataBase](#) *parentCellularAutomata, [StateSet](#) *stateSet=nullptr)
- **LocalRule** (const [LocalRule](#) &orig)
- void **setStateSet** ([StateSet](#) *stateSet)

Additional Inherited Members

Protected Attributes inherited from [LocalRule](#)

- [CellularAutomataBase](#) * **parentCellularAutomata**
- [StateSet](#) * **stateSet**

15.402.1 Member Function Documentation

15.402.1.1 **applyRule()**

```
virtual void LocalRule_ShowNeighbordhood::applyRule (
    Cell * cell ) [inline], [virtual]
```

Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell.

Implements [LocalRule](#).

The documentation for this class was generated from the following file:

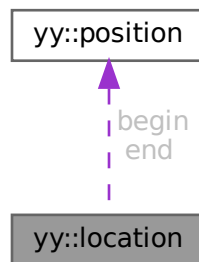
- plugins/components/ModalModel/CellularAutomata/LocalRule_ShowNeighbordhood.h

15.403 yy::location Class Reference

Two points in a source file.

```
#include <parser/location.hh>
```

Collaboration diagram for yy::location:



Public Types

- typedef [position::filename_type](#) **filename_type**
Type for file name.
- typedef [position::counter_type](#) **counter_type**
Type for line and column numbers.

Public Member Functions

- **location** (const [position](#) &b, const [position](#) &e)
Construct a location from b to e.
- **location** (const [position](#) &p=[position](#)())
Construct a 0-width location in p.
- **location** ([filename_type](#) *f, [counter_type](#) l=1, [counter_type](#) c=1)
Construct a 0-width location in f, l, c.
- void **initialize** ([filename_type](#) *f=YY_NULLPTR, [counter_type](#) l=1, [counter_type](#) c=1)
Initialization.

Line and Column related manipulators

- void **step** ()
Reset initial location to final location.
- void **columns** ([counter_type](#) count=1)
Extend the current location to the COUNT next columns.
- void **lines** ([counter_type](#) count=1)
Extend the current location to the COUNT next lines.

Public Attributes

- [position](#) **begin**
Beginning of the located region.
- [position](#) **end**
End of the located region.

15.403.1 Detailed Description

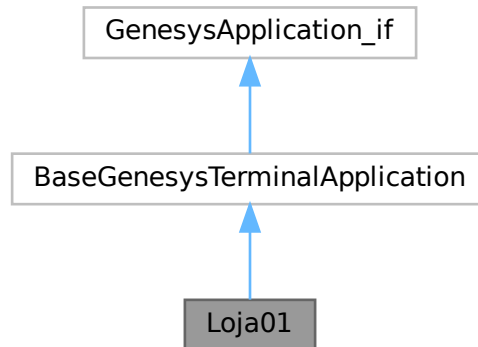
Two points in a source file.

The documentation for this class was generated from the following file:

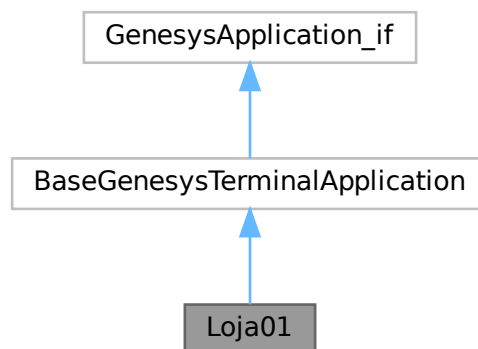
- parser/[location.hh](#)

15.404 Loja01 Class Reference

Inheritance diagram for Loja01:



Collaboration diagram for Loja01:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

15.404.1 Member Function Documentation

15.404.1.1 main()

```
int Loja01::main (
    int argc,
    char ** argv ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/teaching/Loja/Loja01.h
- applications/modelSpecific/teaching/Loja/Loja01.cpp

15.405 testing::internal::internal_stream_operator_without_lexical_↵ name_lookup::LookupBlocker Struct Reference

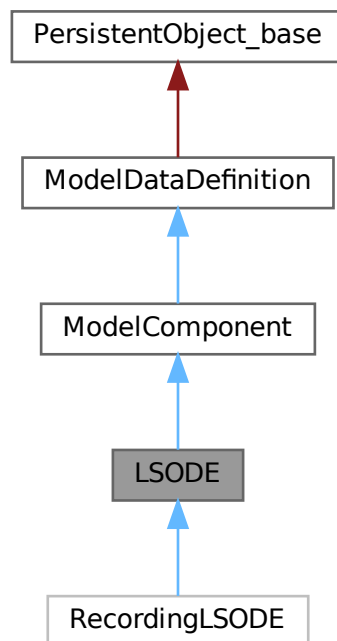
The documentation for this struct was generated from the following file:

- gtest/gtest/gtest-printers.h

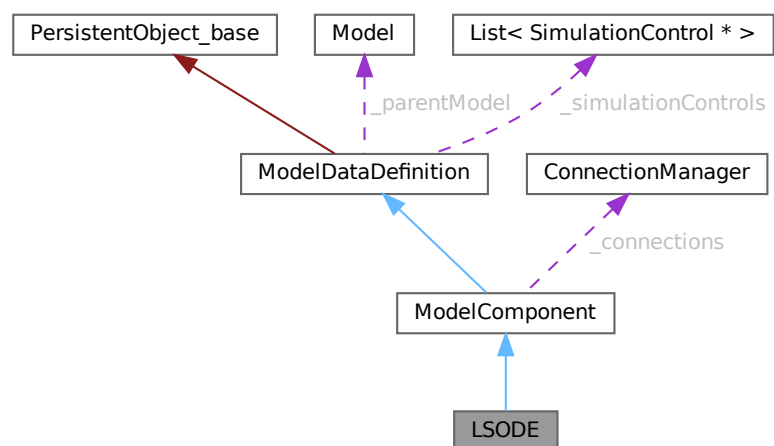
15.406 LSODE Class Reference

```
#include <plugins/components/Continuous/LSODE.h>
```

Inheritance diagram for LODE:



Collaboration diagram for LODE:



Public Member Functions

- **LSODE** (`Model` *model, std::string name="")
 - virtual std::string **show** () override
- virtual public methods*

- void **setTimeVariable** ([Variable](#) *_timeVariable)
- [Variable](#) * **getTimeVariable** () const
- void **setStep** (double _step)
- double **getStep** () const
- void **setVariable** ([Variable](#) *_variables)
- [Variable](#) * **getVariable** () const
- [List](#)< std::string > * **getDiffEquations** () const
- void **addDiffEquation** (std::string newDiffEquation)
- void **removeDiffEquation** (std::string diffEquation)
- void **setFilename** (std::string filename)
- std::string **getFileName** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- [Model](#) * **getParentModel** () const
Returns the parent model that owns this definition.

- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)

- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()

virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.406.1 Detailed Description

This component ...

15.406.2 Member Function Documentation

15.406.2.1 `_check()`

```
bool LSODe::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.406.2.2 `_loadInstance()`

```
bool LSODe::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.406.2.3 `_onDispatchEvent()`

```
void LSODe::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

Reimplemented in [RecordingLSODE](#).

15.406.2.4 `_saveInstance()`

```
void LSODe::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.406.2.5 `show()`

```
std::string LSODe::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

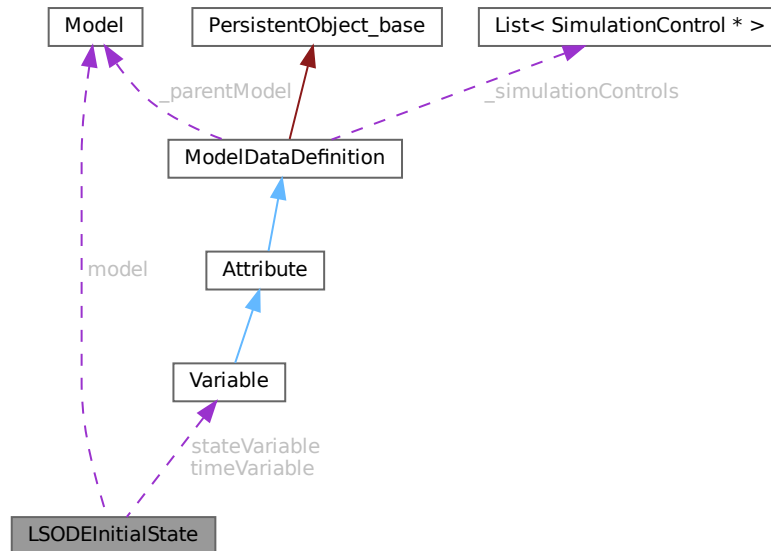
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- `plugins/components/Continuous/LSODE.h`
- `plugins/components/Continuous/LSODE.cpp`

15.407 LSODeInitialState Struct Reference

Collaboration diagram for LSODeInitialState:



Public Member Functions

- void **OnReplicationStart** ([SimulationEvent](#) *)

Public Attributes

- [Model](#) * **model**
- [Variable](#) * **timeVariable**
- [Variable](#) * **stateVariable**

The documentation for this struct was generated from the following file:

- tests/test_lsode.cpp

15.408 LSODeSample Struct Reference

Public Attributes

- double **t**
- double **x**
- double **v**
- double **energy**

The documentation for this struct was generated from the following file:

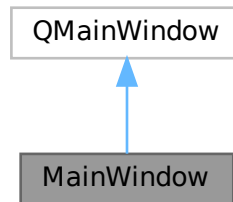
- tests/test_lsode.cpp

15.409 MainWindow Class Reference

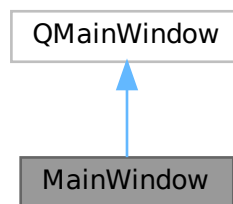
Main Qt window that composes controllers/services and preserves legacy GUI entry points.

```
#include <applications/gui/genesys/mainwindow.h>
```

Inheritance diagram for MainWindow:



Collaboration diagram for MainWindow:



Public Member Functions

- **MainWindow** (QWidget *parent=nullptr)
Builds the composition root, wiring controllers/services and legacy Qt surface.
- **~MainWindow** ()
Releases UI resources while preserving existing Qt ownership semantics.
- **ModelGraphicsScene * myScene** () const
Returns the current model graphics scene used by delegated controllers/services.
- bool **graphicalModelHasChanged** () const
Indicates whether graphical representation has unsaved modifications.
- void **setGraphicalModelHasChanged** (bool graphicalModelHasChanged)
Updates graphical-change flag used by persistence/action synchronization.
- void **unselectDrawIcons** ()
Compatibility wrapper that clears currently selected draw tool actions.
- bool **checkSelectedDrawIcons** ()
Compatibility wrapper that reports whether any draw tool action is selected.
- void **refreshGuiExtensions** ()
Public refresh hook for graphical extension contributions.
- void **refreshPluginCatalog** ()
Public refresh hook for plugin catalog tree.

Protected Member Functions

- void **closeEvent** (QCloseEvent *event) override

Friends

- class **SimulationEventController**

15.409.1 Detailed Description

Main Qt window that composes controllers/services and preserves legacy GUI entry points.

[MainWindow](#) remains the composition root of GenesysQtGUI: it owns the generated UI, wires extracted controllers/services, and exposes the original Qt slot surface expected by actions, widgets, and kernel callback registration. In the refactored architecture it primarily acts as a compatibility façade that delegates operational responsibilities to focused collaborators.

Responsibilities:

- bootstrap UI construction and dependency wiring for controllers/services;
- host compatibility wrappers used by menus/toolbars/scene callbacks;
- coordinate cross-cutting state that still spans multiple controllers.

Boundaries:

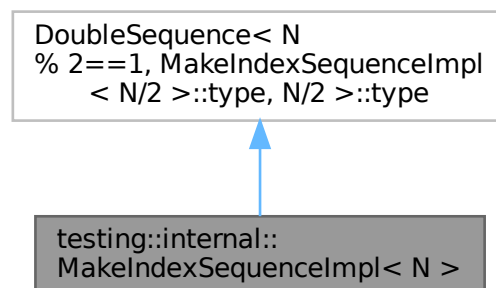
- detailed domain workflows (simulation commands, lifecycle, persistence, scene tools, property editing, trace/event rendering) are delegated to dedicated controller/service classes whenever extraction already exists;
- [MainWindow](#) does not replace kernel ownership semantics.

The documentation for this class was generated from the following files:

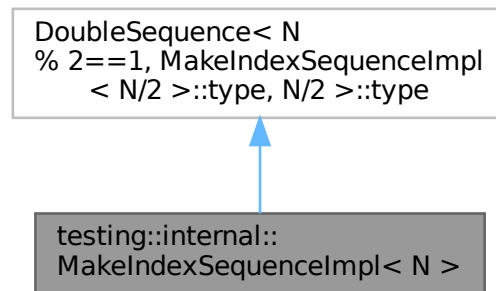
- applications/gui/genesys/mainwindow.h
- applications/gui/genesys/[mainwindow.cpp](#)
- applications/gui/genesys/[mainwindow_controller.cpp](#)
- applications/gui/genesys/[mainwindow_modelrepresentations.cpp](#)
- applications/gui/genesys/[mainwindow_scene.cpp](#)
- applications/gui/genesys/[mainwindow_simulator.cpp](#)

15.410 testing::internal::MakeIndexSequenceImpl< N > Struct Template Reference

Inheritance diagram for testing::internal::MakeIndexSequenceImpl< N >:



Collaboration diagram for `testing::internal::MakeIndexSequenceImpl< N >`:

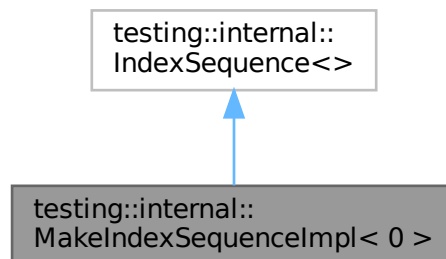


The documentation for this struct was generated from the following file:

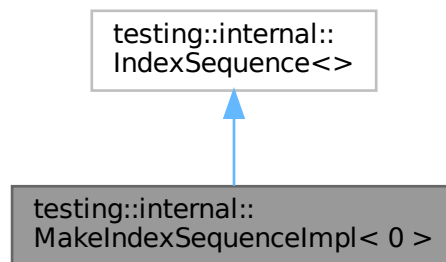
- `gtest/gtest/internal/gtest-internal.h`

15.411 `testing::internal::MakeIndexSequenceImpl< 0 >` Struct Reference

Inheritance diagram for `testing::internal::MakeIndexSequenceImpl< 0 >`:



Collaboration diagram for testing::internal::MakeIndexSequenceImpl< 0 >:



Additional Inherited Members

Public Types inherited from [testing::internal::IndexSequence<>](#)

- using **type** = [IndexSequence](#)

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-internal.h

15.412 testing::internal::MarkAsIgnored Struct Reference

Public Member Functions

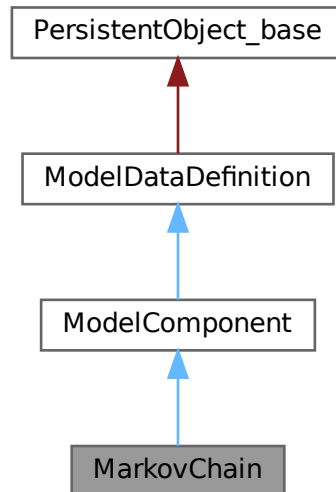
- **MarkAsIgnored** (const char *test_suite)

The documentation for this struct was generated from the following files:

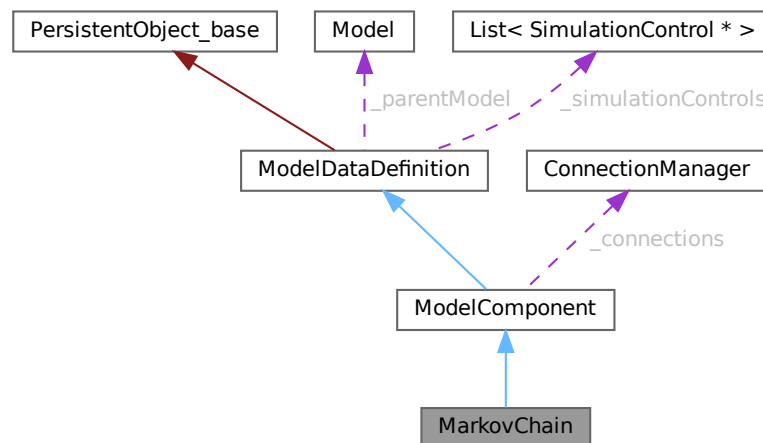
- gtest/gtest/internal/gtest-param-util.h
- gtest/src/gtest.cc

15.413 MarkovChain Class Reference

Inheritance diagram for MarkovChain:



Collaboration diagram for MarkovChain:



Public Member Functions

- **MarkovChain** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
- *virtual public methods*
- void **setTransitionProbabilityMatrix** ([ModelDataDefinition](#) *transitionMatrix)
- [ModelDataDefinition](#) * **getTransitionMatrix** () const

- void **setInitialDistribution** ([ModelDataDefinition](#) *initialDistribution)
- [ModelDataDefinition](#) * **getInitialDistribution** () const
- void **setCurrentState** ([ModelDataDefinition](#) *currentState)
- [ModelDataDefinition](#) * **getCurrentState** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentName, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionType, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- [Model](#) * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * **getInternalData** () const

- Returns the full map of internal child data definitions.*
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelspecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelComponent](#)

- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static bool Check (ModelComponent *component)`
Validates a component instance.
- `static void CreateInternalData (ModelComponent *component)`
Creates internal data required by a component instance.
- `static void SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
Serializes a component instance into a persistence record.
- `static void DispatchEvent (Event *event)`
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- `static void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.

- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_initBetweenReplications](#) () override
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)

- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.413.1 Member Function Documentation

15.413.1.1 **_check()**

```
bool MarkovChain::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
 Reimplemented from [ModelDataDefinition](#).

15.413.1.2 **_createEditableDataDefinitions()**

```
void MarkovChain::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.413.1.3 _initBetweenReplications()

void MarkovChain::_initBetweenReplications () [override], [protected], [virtual]
 Reimplemented from [ModelDataDefinition](#).

15.413.1.4 _loadInstance()

bool MarkovChain::_loadInstance (
 [PersistenceRecord](#) * *fields*) [override], [protected], [virtual]
 virtual protected method that must be overridden
 Reimplemented from [ModelComponent](#).

15.413.1.5 _onDispatchEvent()

void MarkovChain::_onDispatchEvent (
 [Entity](#) * *entity*,
 unsigned int *inputPortNumber*) [override], [protected], [virtual]
 Handles a dispatched event for this component type.
 This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.413.1.6 _saveInstance()

void MarkovChain::_saveInstance (
 [PersistenceRecord](#) * *fields*,
 bool *saveDefaultValues*) [override], [protected], [virtual]
 Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.413.1.7 show()

std::string MarkovChain::show () [override], [virtual]
 virtual public methods
 Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

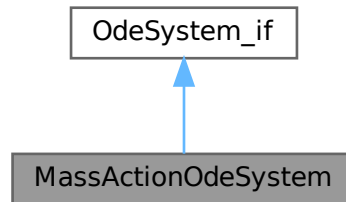
- plugins/components/AnalyticalModeling/MarkovChain.h
- plugins/components/AnalyticalModeling/MarkovChain.cpp

15.414 MassActionOdeSystem Class Reference

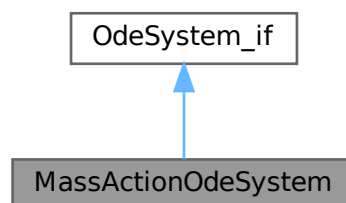
ODE system generated from biochemical reactions using mass-action kinetics.

```
#include <tools/Biochemical/MassActionOdeSystem.h>
```

Inheritance diagram for MassActionOdeSystem:



Collaboration diagram for MassActionOdeSystem:



Classes

- struct [Reaction](#)
- struct [Species](#)
- struct [StoichiometricTerm](#)

Public Member Functions

- **MassActionOdeSystem** (std::vector< [Species](#) > species, std::vector< [Reaction](#) > reactions)
- unsigned int [dimension](#) () const override
- void [evaluate](#) (double, const double *y, double *dydt) const override
- const std::vector< [Species](#) > & **species** () const
- const std::vector< [Reaction](#) > & **reactions** () const
- double **evaluateForwardRate** (unsigned int reactionIndex, const double *y) const
- double **evaluateReverseRate** (unsigned int reactionIndex, const double *y) const
- double **evaluateNetRate** (unsigned int reactionIndex, const double *y) const

15.414.1 Detailed Description

ODE system generated from biochemical reactions using mass-action kinetics.

15.414.2 Member Function Documentation

15.414.2.1 dimension()

```
unsigned int MassActionOdeSystem::dimension ( ) const [inline], [override], [virtual]
```

Implements [OdeSystem_if](#).

15.414.2.2 evaluate()

```
void MassActionOdeSystem::evaluate (
    double ,
    const double * y,
    double * dydt ) const [inline], [override], [virtual]
```

Implements [OdeSystem_if](#).

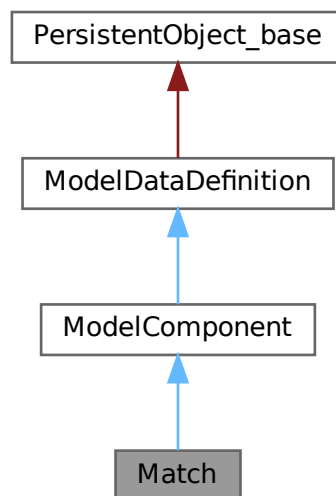
The documentation for this class was generated from the following file:

- tools/Biochemical/MassActionOdeSystem.h

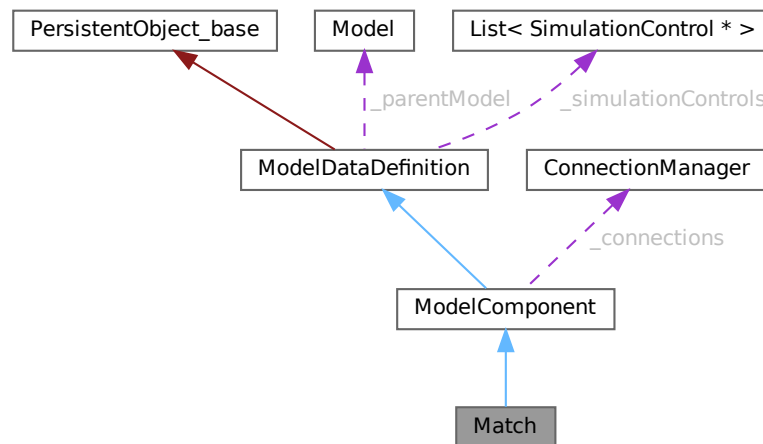
15.415 Match Class Reference

```
#include <plugins/components/Synchronization/Match.h>
```

Inheritance diagram for Match:



Collaboration diagram for Match:



Public Types

- enum class **Rule** : int { **Any** = 0 , **ByAttribute** = 1 , **num_elements** = 2 }

Public Member Functions

- Match** (**Model** *model, std::string name="")
- virtual std::string **show** () override
virtual public methods
- virtual void **_onDispatchEvent** (**Entity** *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_loadInstance** (**PersistenceRecord** *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** (**PersistenceRecord** *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- void **setRule** (Match::Rule _rule)
- Match::Rule **getRule** () const
- void **setAttributeName** (std::string _attributeName)
- std::string **getAttributeName** () const
- void **setMatchSize** (std::string _matchSize)
- std::string **getMatchSize** () const
- void **setNumberOfQueues** (unsigned int _numberOfQueues)
- unsigned int **getNumberOfQueues** () const

Public Member Functions inherited from **ModelComponent**

- ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)

- *Connects this component to another component.*
- bool `hasBreakpointAt` ()
Indicates whether a simulation breakpoint is set on this component.
- void `setDescription` (std::string _description)
Updates the free-form description shown by the GUI.
- std::string `getDescription` () const
Returns the free-form description associated with this component.
- void `remove` (ModelComponent *comp)
Removes a connected component from this component.

Public Member Functions inherited from `ModelDataDefinition`

- `ModelDataDefinition` (Model *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~`ModelDataDefinition` ()
Releases resources owned by this data definition.
- Util::identification `getId` () const
Returns the unique identifier of this instance.
- void `setName` (const std::string &name)
Updates the logical name of this instance.
- const std::string & `getName` () const
Returns the logical name of this instance.
- void `setLabel` (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & `getLabel` () const
Returns the user-visible label of this instance.
- const std::string & `getClassname` () const
Returns the runtime class name used by the plugin system.
- Model * `getParentModel` () const
Returns the parent model that owns this definition.
- bool `isReportStatistics` () const
Indicates whether this data definition contributes report statistics.
- void `setReportStatistics` (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition` * `getInternalData` (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, `ModelDataDefinition` * > * `getInternalData` () const
Returns the full map of internal child data definitions.
- std::map< std::string, `ModelDataDefinition` * > * `getAttachedData` () const
Returns the full map of attached data definitions.
- bool `hasChanged` () const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged` (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int `getLevel` () const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel` (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< `SimulationControl` * > * `getSimulationControls` () const
Returns the simulation controls owned by this data definition.

- `TraceManager::Level` `getTraceLevelSpecific ()` const
Returns the trace level that applies specifically to this definition.
- `void` `setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void` `defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool` `isTraceLevelSpecificEnabled ()` const
Indicates whether the specific trace level is enabled.
- `void` `setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static std::string` `convertEnumToStr (Rule rule)`
- `static PluginInformation *` `GetPluginInformation ()`
- `static ModelComponent *` `LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition *` `NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from `ModelComponent`

- `static ModelComponent *` `LoadInstance (Model *model, PersistenceRecord *fields)`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static bool` `Check (ModelComponent *component)`
Validates a component instance.
- `static void` `CreateInternalData (ModelComponent *component)`
Creates internal data required by a component instance.
- `static void` `SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
Serializes a component instance into a persistence record.
- `static void` `DispatchEvent (Event *event)`
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from `ModelDataDefinition`

- `static ModelDataDefinition *` `LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- `static ModelDataDefinition *` `NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- `static void` `SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.
- `static bool` `Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Validates a data definition instance.
- `static void` `InitBetweenReplications (ModelDataDefinition *modeldatum)`
Resets simulation-time state between replications.
- `static void` `CreateInternalData (ModelDataDefinition *modeldatum)`
Creates internal and attached data required by this instance.
- `static bool` `CreateRelatedDataElements (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Creates related data and validates the instance in a single call.

Protected Member Functions

- `virtual void` `_createEditableDataDefinitions ()` override
- `virtual bool` `_check (std::string &errorMessage)` override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
 - legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

15.415.1 Detailed Description

[Match](#) module DESCRIPTION The [Match](#) module brings together a specified number of entities waiting in different queues. The match may be accomplished when there is at least one entity in each of the desired queues. Additionally, an attribute may be specified such that the entities waiting in the queues must have the same attribute values before the match is initiated. When an entity arrives at the [Match](#) module, it is placed in one of up to five associated queues, based on the entry point to which it is connected. Entities will remain in their respective queues until a match exists. Once a match exists, one entity from each queue is released to be matched. The matched entities are then synchronized to depart from the module. TYPICAL USES Assembling a part Gathering various products for a customer order Synchronizing a customer exit with a filled order Prompt Description Name Unique module identifier displayed on the module shape. Number to [Match](#) Number of matching entities that must reside in different queues before a match may be completed. Type Method for matching the incoming entities. If Type is Any Entities, one entity must reside in each queue for a match to be made. If Type is Based on [Attribute](#), one entity must reside in each queue with the same attribute value. [Attribute](#) Name [Attribute](#) name that is used for identifying an arriving entity's match value. Applies only when Type is Based on [Attribute](#).

15.415.2 Member Function Documentation

15.415.2.1 `_check()`

```
bool Match::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.415.2.2 `_createEditableDataDefinitions()`

```
void Match::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.415.2.3 `_loadInstance()`

```
bool Match::_loadInstance (
    PersistenceRecord * fields ) [override], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [ModelComponent](#).

15.415.2.4 _onDispatchEvent()

```
void Match::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.415.2.5 _saveInstance()

```
void Match::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.415.2.6 show()

```
std::string Match::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/Synchronization/Match.h
- plugins/components/Synchronization/Match.cpp

15.416 testing::Message Class Reference

Public Member Functions

- **Message** (const [Message](#) &msg)
- **Message** (const char *str)
- template<typename T >
[Message](#) & **operator**<< (const T &val)
- template<typename T >
[Message](#) & **operator**<< (T *const &pointer)
- [Message](#) & **operator**<< (BasicNarrowIoManip val)
- [Message](#) & **operator**<< (bool b)
- [Message](#) & **operator**<< (const wchar_t *wide_c_str)
- [Message](#) & **operator**<< (wchar_t *wide_c_str)

- `std::string GetString () const`

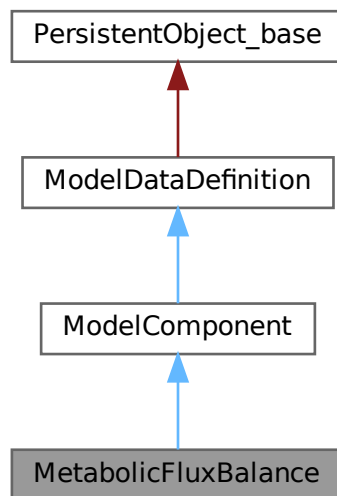
The documentation for this class was generated from the following files:

- `gtest/gtest/gtest-message.h`
- `gtest/src/gtest.cc`

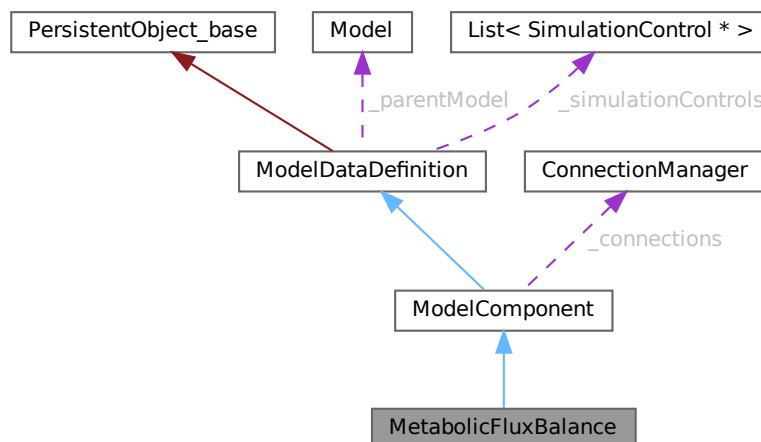
15.417 MetabolicFluxBalance Class Reference

```
#include <plugins/components/BiochemicalSimulation/MetabolicFluxBalance.h>
```

Inheritance diagram for MetabolicFluxBalance:



Collaboration diagram for MetabolicFluxBalance:



Public Member Functions

- **MetabolicFluxBalance** ([Model](#) *model, std::string name="")
- void **setMetabolicNetwork** ([MetabolicNetwork](#) *metabolicNetwork)
- [MetabolicNetwork](#) * **getMetabolicNetwork** () const
- void **setObjectiveReactionName** (std::string objectiveReactionName)
- std::string **getObjectiveReactionName** () const
- void **setLastSucceeded** (bool lastSucceeded)
- bool **getLastSucceeded** () const
- void **setLastObjectiveValue** (double lastObjectiveValue)
- double **getLastObjectiveValue** () const
- void **setLastMessage** (std::string lastMessage)
- std::string **getLastMessage** () const
- const std::map< std::string, double > & **getLastFluxes** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.

- `const std::string & getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from ModelComponent

- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static bool Check (ModelComponent *component)`
Validates a component instance.
- `static void CreateInternalData (ModelComponent *component)`
Creates internal data required by a component instance.
- `static void SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
Serializes a component instance into a persistence record.
- `static void DispatchEvent (Event *event)`
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)

- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()

virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.417.1 Detailed Description

Component that evaluates flux-balance analysis (FBA) over a [MetabolicNetwork](#).

15.417.1.1 LP solver dependency (GLPK)

[MetabolicFluxBalance](#) requires GLPK for metabolic networks with more than a few dozen reactions (e.g., M. genitalium: 641 reactions; yeast-GEM: 4131 reactions). Without GLPK, a built-in basis-enumeration fallback is used, which is only practical for small toy networks.

15.417.1.1.1 Installation

Ubuntu/Debian: `sudo apt-get install libglpk-dev` Fedora/RHEL: `sudo dnf install glpk-devel` macOS (Brew): `brew install glpk`

15.417.1.1.2 Re-configuration after installation

`cmake -preset terminal-app #` or any other preset `cmake -build build/terminal-app -target genesys_plugins_↔` components

CMake will print at configure time: GenESyS: GLPK found — [MetabolicFluxBalance](#) ... will use GLPK LP solver or: GenESyS: GLPK not found — ... (suitable for small models only; install libglpk-dev ...)

The preprocessor guard is `GENESYS_HAVE_GLPK`. The GLPK solver wrapper is in [source/tools/Biochemical/GlpkFluxBalanceSolver](#)

15.417.2 Member Function Documentation

15.417.2.1 _check()

```
bool MetabolicFluxBalance::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.417.2.2 _createEditableDataDefinitions()

```
void MetabolicFluxBalance::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.417.2.3 _loadInstance()

```
bool MetabolicFluxBalance::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.417.2.4 _onDispatchEvent()

```
void MetabolicFluxBalance::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.417.2.5 _saveInstance()

```
void MetabolicFluxBalance::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.417.2.6 show()

```
std::string MetabolicFluxBalance::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/BiochemicalSimulation/MetabolicFluxBalance.h
- plugins/components/BiochemicalSimulation/MetabolicFluxBalance.cpp

15.418 MetabolicFluxBalanceSolver Class Reference

```
#include <tools/Biochemical/MetabolicFluxBalanceSolver.h>
```

Classes

- struct [Problem](#)
- struct [Solution](#)

Static Public Member Functions

- static [Solution](#) **solve** (const [Problem](#) &problem)

15.418.1 Detailed Description

Small LP solver for flux-balance problems with equality constraints and variable bounds.

It solves: maximize/minimize $c^T v$ subject to $A v = 0$ $lb \leq v \leq ub$

using basis enumeration over bounded vertices. This is practical for the small metabolic models currently exercised by the runtime tests and provides a concrete solver-backed replacement for the previous stub objective logic.

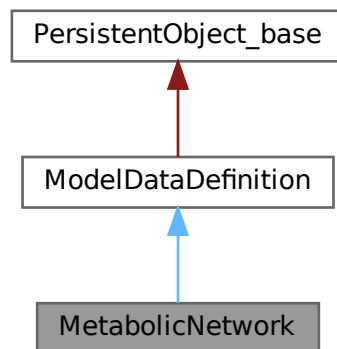
The documentation for this class was generated from the following file:

- tools/Biochemical/MetabolicFluxBalanceSolver.h

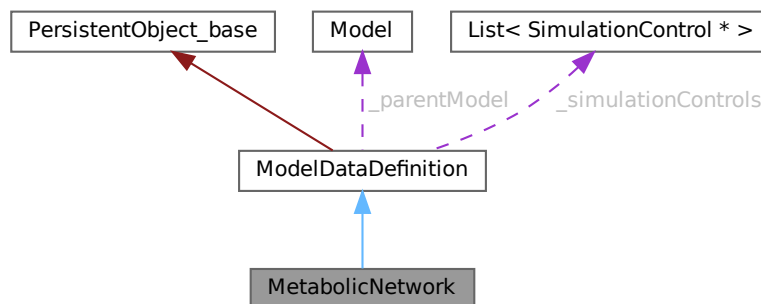
15.419 MetabolicNetwork Class Reference

```
#include <plugins/data/BiochemicalSimulation/MetabolicNetwork.h>
```

Inheritance diagram for MetabolicNetwork:



Collaboration diagram for MetabolicNetwork:



Public Member Functions

- **MetabolicNetwork** ([Model](#) *model, std::string name="")
- void **addReaction** (std::string reactionName)
- void **addExchangeSpecies** (std::string speciesName)
- void **clearReactions** ()
- void **clearExchangeSpecies** ()
- const std::vector< std::string > & **getReactionNames** () const
- const std::vector< std::string > & **getExchangeSpeciesNames** () const
- void **setObjectiveReactionName** (std::string objectiveReactionName)
- std::string **getObjectiveReactionName** () const
- void **setObjectiveSense** (std::string objectiveSense)
- std::string **getObjectiveSense** () const
- void **setCompartment** (std::string compartment)
- std::string **getCompartment** () const
- void **setEnabled** (bool enabled)
- bool **isEnabled** () const

- virtual std::string **show** () override
Returns a textual representation of this data definition.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const
Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)
Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)
Defines the specific trace level and optionally enables it immediately.

- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static `ModelDataDefinition` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
- static `PluginInformation` * `GetPluginInformation` ()
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void `SaveInstance` (`PersistenceRecord` *fields, `ModelDataDefinition` *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool `Check` (`ModelDataDefinition` *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void `InitBetweenReplications` (`ModelDataDefinition` *modeldatum)
Resets simulation-time state between replications.
- static void `CreateInternalData` (`ModelDataDefinition` *modeldatum)
Creates internal and attached data required by this instance.
- static bool `CreateRelatedDataElements` (`ModelDataDefinition` *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` (`PersistenceRecord` *fields) override
must be overridden by derived classes
- virtual void `_saveInstance` (`PersistenceRecord` *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_createEditableDataDefinitions` () override

Protected Member Functions inherited from `ModelDataDefinition`

- bool `_getSaveDefaultsOption` ()
- virtual `ParserChangesInformation` * `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` (`SimulationControl` *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*

- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.419.1 Detailed Description

Aggregate metabolic network definition.

A [MetabolicNetwork](#) stores the reaction membership and objective metadata for future flux-balance style components.

15.419.2 Member Function Documentation

15.419.2.1 `_check()`

```
bool MetabolicNetwork::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.419.2.2 `_createEditableDataDefinitions()`

```
void MetabolicNetwork::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.419.2.3 `_loadInstance()`

```
bool MetabolicNetwork::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.419.2.4 `_saveInstance()`

```
void MetabolicNetwork::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.419.2.5 `show()`

```
std::string MetabolicNetwork::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal [ModelDataDefinitions](#) such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

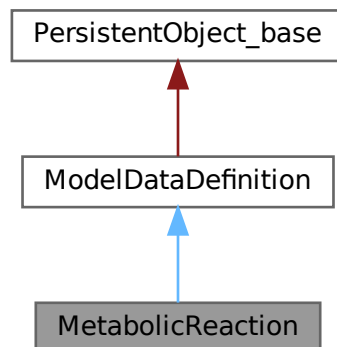
The documentation for this class was generated from the following files:

- `plugins/data/BiochemicalSimulation/MetabolicNetwork.h`
- `plugins/data/BiochemicalSimulation/MetabolicNetwork.cpp`

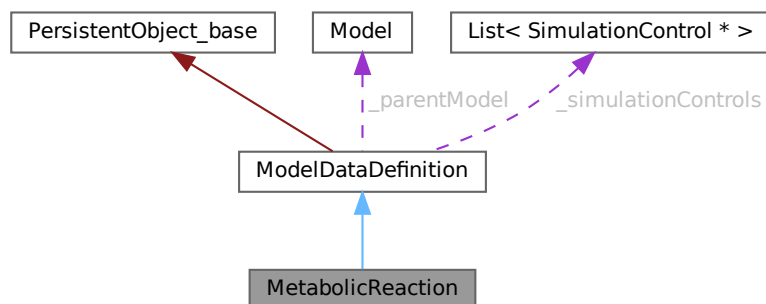
15.420 MetabolicReaction Class Reference

```
#include <plugins/data/BiochemicalSimulation/MetabolicReaction.h>
```

Inheritance diagram for MetabolicReaction:



Collaboration diagram for MetabolicReaction:



Classes

- struct [StoichiometricTerm](#)

Public Member Functions

- **MetabolicReaction** ([Model](#) *model, std::string name="")
- void **addReactant** (std::string speciesName, double stoichiometry=1.0)
- void **addProduct** (std::string speciesName, double stoichiometry=1.0)
- void **clearReactants** ()
- void **clearProducts** ()
- const std::vector< [StoichiometricTerm](#) > & **getReactants** () const
- const std::vector< [StoichiometricTerm](#) > & **getProducts** () const
- void **setLowerBound** (double lowerBound)
- double **getLowerBound** () const
- void **setUpperBound** (double upperBound)
- double **getUpperBound** () const
- void **setObjectiveCoefficient** (double objectiveCoefficient)

- double **getObjectiveCoefficient** () const
 - void **setReversible** (bool reversible)
 - bool **isReversible** () const
 - void **setGeneRule** (std::string geneRule)
 - std::string **getGeneRule** () const
 - virtual std::string **show** () override
- Returns a textual representation of this data definition.*

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- Releases resources owned by this data definition.*
- Util::identification **getId** () const
- Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
- Updates the logical name of this instance.*
- const std::string & **getName** () const
- Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)
- Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const
- Returns the user-visible label of this instance.*
- const std::string & **getClassname** () const
- Returns the runtime class name used by the plugin system.*
- **Model** * **getParentModel** () const
- Returns the parent model that owns this definition.*
- bool **isReportStatistics** () const
- Indicates whether this data definition contributes report statistics.*
- void **setReportStatistics** (bool reportStatistics)
- Enables or disables report-statistics generation for this definition.*
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
- Looks up a named internal child data definition.*
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
- Returns the full map of internal child data definitions.*
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
- Returns the full map of attached data definitions.*
- bool **hasChanged** () const
- Indicates whether this instance has been modified since the last reset.*
- void **setHasChanged** (bool **hasChanged**)
- Updates the changed flag for this data definition.*
- unsigned int **getLevel** () const
- Returns the level in the model hierarchy where this definition lives.*
- void **setModelLevel** (unsigned int _modelLevel)
- Sets the model nesting level for this definition.*
- List< **SimulationControl** * > * **getSimulationControls** () const
- Returns the simulation controls owned by this data definition.*
- TraceManager::Level **getTraceLevelSpecific** () const
- Returns the trace level that applies specifically to this definition.*
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↔ Enabled=true)

Defines the specific trace level and optionally enables it immediately.

- bool [isTraceLevelSpecificEnabled](#) () const

Indicates whether the specific trace level is enabled.

- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()

- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internaStatisticReportersClear** ()
legacy helper methods used by the granular create hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.420.1 Detailed Description

Constraint-based metabolic reaction definition.

15.420.2 Member Function Documentation

15.420.2.1 `_check()`

```
bool MetabolicReaction::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.420.2.2 `_createEditableDataDefinitions()`

```
void MetabolicReaction::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.420.2.3 `_loadInstance()`

```
bool MetabolicReaction::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.420.2.4 `_saveInstance()`

```
void MetabolicReaction::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.420.2.5 `show()`

```
std::string MetabolicReaction::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.
Returns the keys of internal [ModelDataDefinitions](#) such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- `plugins/data/BiochemicalSimulation/MetabolicReaction.h`
- `plugins/data/BiochemicalSimulation/MetabolicReaction.cpp`

15.421 MetabolicStateProjectionComponent Class Reference

Projects flux-balance outputs into a [WholeCellState](#).

```
#include <plugins/components/WholeCellModeling/MetabolicStateProjection↵
Component.h>
```


Public Member Functions

- **MetabolicStateProjectionComponent** ([Model](#) *model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **setFluxBalanceComponent** ([MetabolicFluxBalance](#) *component)
- [MetabolicFluxBalance](#) * **getFluxBalanceComponent** () const
- void **setFluxBalanceComponentName** (std::string componentName)
- std::string **getFluxBalanceComponentName** () const
- void **clearObjectiveProjection** ()
- void **setObjectiveAsMetabolite** (const std::string &targetKey, double scale=1.0, double offset=0.0, ProjectionUpdateMode updateMode=ProjectionUpdateMode::Overwrite, double turnoverFraction=0.0)
- void **setObjectiveAsCompartmentMetabolite** (const std::string &compartmentName, const std::string &targetKey, double scale=1.0, double offset=0.0, ProjectionUpdateMode updateMode=ProjectionUpdateMode::Overwrite, double turnoverFraction=0.0)
- void **setObjectiveAsPathwayActivity** (const std::string &targetKey, double scale=1.0, double offset=0.0, ProjectionUpdateMode updateMode=ProjectionUpdateMode::Overwrite, double turnoverFraction=0.0)
- bool **hasObjectiveProjection** () const
- void **clearFluxProjections** ()
- void **addMetaboliteProjection** (const std::string &reactionName, const std::string &targetKey, double scale=1.0, double offset=0.0, ProjectionUpdateMode updateMode=ProjectionUpdateMode::Overwrite, double turnoverFraction=0.0)
- void **addCompartmentMetaboliteProjection** (const std::string &reactionName, const std::string &compartmentName, const std::string &targetKey, double scale=1.0, double offset=0.0, ProjectionUpdateMode updateMode=ProjectionUpdateMode::Overwrite, double turnoverFraction=0.0)
- void **addPathwayProjection** (const std::string &reactionName, const std::string &targetKey, double scale=1.0, double offset=0.0, ProjectionUpdateMode updateMode=ProjectionUpdateMode::Overwrite, double turnoverFraction=0.0)
- unsigned int **getProjectionCount** () const
- void **setLastSucceeded** (bool lastSucceeded)
- bool **getLastSucceeded** () const
- void **setLastMessage** (std::string lastMessage)
- std::string **getLastMessage** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentName, std::string name="")
constructors
- virtual ~[ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool **hasChanged**)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void **_createEditableDataDefinitions** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

15.421.1 Detailed Description

Projects flux-balance outputs into a [WholeCellState](#).

[MetabolicStateProjectionComponent](#) bridges FBA-style submodels with the whole-cell shared state. It reads the latest objective value and reaction fluxes computed by a [MetabolicFluxBalance](#) component and writes selected projections into [WholeCellState](#) as:

- global metabolite proxies;
- compartment-specific metabolite proxies;
- pathway activity summaries.

This component intentionally does not solve the metabolic optimization itself. It only transfers selected results to the whole-cell layer so that growth, checkpoints, and gene-expression submodels can consume them.

15.421.2 Member Function Documentation

15.421.2.1 `_check()`

```
bool MetabolicStateProjectionComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.421.2.2 `_createEditableDataDefinitions()`

```
void MetabolicStateProjectionComponent::_createEditableDataDefinitions ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.421.2.3 `_loadInstance()`

```
bool MetabolicStateProjectionComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [ModelComponent](#).

15.421.2.4 _onDispatchEvent()

```
void MetabolicStateProjectionComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.421.2.5 _saveInstance()

```
void MetabolicStateProjectionComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.421.2.6 show()

```
std::string MetabolicStateProjectionComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

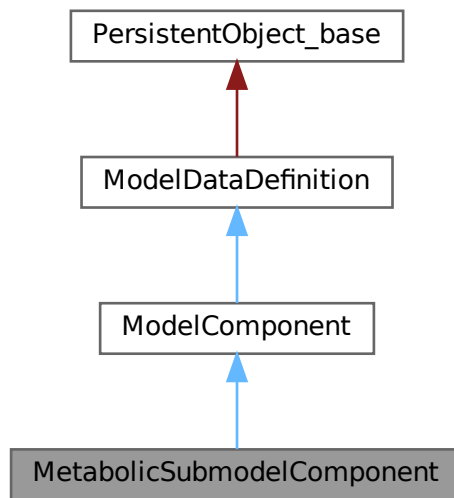
The documentation for this class was generated from the following files:

- plugins/components/WholeCellModeling/MetabolicStateProjectionComponent.h
- plugins/components/WholeCellModeling/MetabolicStateProjectionComponent.cpp

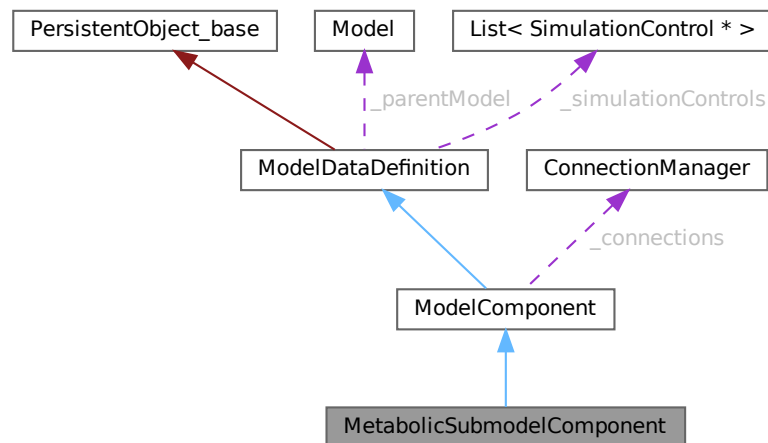
15.422 MetabolicSubmodelComponent Class Reference

```
#include <plugins/components/WholeCellModeling/MetabolicSubmodelComponent.h>
```

Inheritance diagram for MetabolicSubmodelComponent:



Collaboration diagram for MetabolicSubmodelComponent:



Public Member Functions

- **MetabolicSubmodelComponent** ([Model](#) *model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **setAtpYieldRate** (double rate)
- double **getAtpYieldRate** () const
- void **setConsumptionFraction** (double frac)
- double **getConsumptionFraction** () const

- void **setAtpPoolMax** (double maxPool)
- double **getAtpPoolMax** () const
- void **setDeltaT** (double dt)
- double **getDeltaT** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

- Looks up a named internal child data definition.*

 - `std::map< std::string, ModelDataDefinition * > * getInternalData () const`

Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`

Returns the full map of attached data definitions.
- `bool hasChanged () const`

Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`

Updates the changed flag for this data definition.
- `unsigned int getLevel () const`

Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`

Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`

Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`

Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`

Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↔ Enabled=true)`

Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`

Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`

Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelComponent](#)

- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`

static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static bool Check (ModelComponent *component)`

Validates a component instance.
- `static void CreateInternalData (ModelComponent *component)`

Creates internal data required by a component instance.
- `static void SaveInstance (PersistenceRecord *fields, ModelComponent *component)`

Serializes a component instance into a persistence record.
- `static void DispatchEvent (Event *event)`

Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)

- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.422.1 Detailed Description

Minimal metabolic submodel that produces ATP and NADPH each step.

Replaces a full FBA step with a volume-proportional energy production model calibrated for *M. genitalium* (Karr et al. 2012):

$\text{ATP_produced} = \text{atpYieldRate} \times \text{cellVolume(fL)} \times \text{deltaT [mmol]}$
 $\text{ATP_pool(new)} = \min(\text{ATP_pool} + \text{ATP_produced} - \text{ATP_consumed}, \text{atpPoolMax})$

ATP consumption is estimated as a fixed fraction of the current pool (consumption rate = consumptionFraction $\times$ ATP_pool).

Results are written to the [WholeCellState](#) metabolite pool under keys "ATP" and "ADP". Downstream components can read "ATP" to gate reactions.

For a full FBA integration, replace this component with a proper [MetabolicFluxBalance](#) + network wiring. This component is intentionally minimal so that Phases 2–5 can be validated without GLPK-specific setup.

15.422.2 Member Function Documentation

15.422.2.1 _check()

```
bool MetabolicSubmodelComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.422.2.2 _createEditableDataDefinitions()

```
void MetabolicSubmodelComponent::_createEditableDataDefinitions ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.422.2.3 _initBetweenReplications()

```
void MetabolicSubmodelComponent::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.422.2.4 _loadInstance()

```
bool MetabolicSubmodelComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.422.2.5 _onDispatchEvent()

```
void MetabolicSubmodelComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.422.2.6 _saveInstance()

```
void MetabolicSubmodelComponent::_saveInstance (
```

```
PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.422.2.7 show()

```
std::string MetabolicSubmodelComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/WholeCellModeling/MetabolicSubmodelComponent.h
- plugins/components/WholeCellModeling/MetabolicSubmodelComponent.cpp

15.423 generate_kernel_simulator_test_stubs.MethodEntry Class Reference

Static Public Attributes

- str **header**
- str **class_name**
- str **signature**

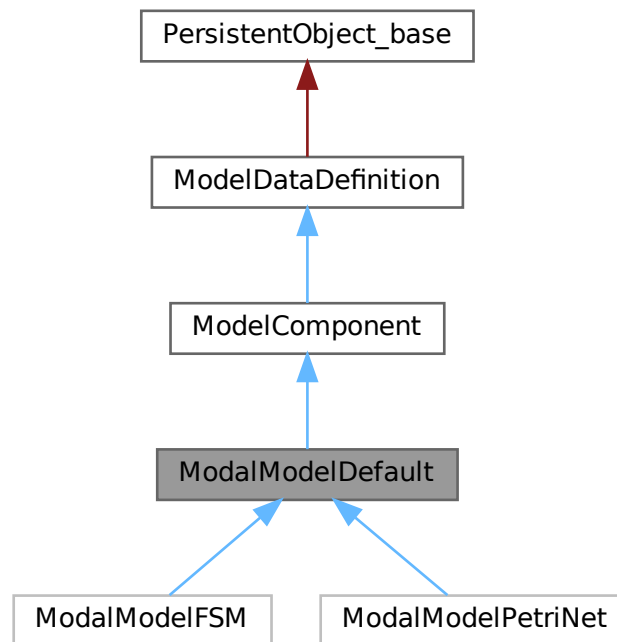
The documentation for this class was generated from the following file:

- tests/unit/tools/generate_kernel_simulator_test_stubs.py

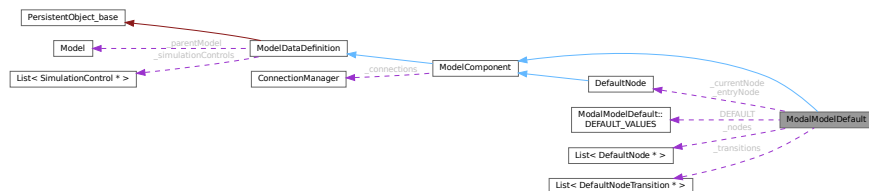
15.424 ModalModelDefault Class Reference

```
#include <plugins/components/ModalModel/ModalModelDefault.h>
```

Inheritance diagram for ModalModelDefault:



Collaboration diagram for ModalModelDefault:



Classes

- struct `DEFAULT_VALUES`

Attributes that should be loaded or saved with this component (Persistent Fields)

Public Member Functions

- `ModalModelDefault` (`Model` *model, std::string name="")
constructors
- virtual void **setMaxTransitionsPerDispatch** (unsigned int maxTransitionsPerDispatch)
- virtual unsigned int **getMaxTransitionsPerDispatch** () const
- virtual `List< DefaultNode * >` * **getNodes** () const
- virtual `List< DefaultNodeTransition * >` * **getTransitions** () const
- `DefaultNode` * **getEntryNode** ()
- void **setEntryNode** (`DefaultNode` *const entry_node)

- [DefaultNode](#) * **getCurrentNode** ()
- std::string **getTimeDelayExpressionPerDispatch** ()
- void **setTimeDelayExpressionPerDispatch** (const std::string time_delay_expression_per_dispatch)
- [Util::TimeUnit](#) **getTimeDelayPerDispatchTimeUnit** ()
- void **setTimeDelayPerDispatchTimeUnit** (const [Util::TimeUnit](#) time_delay_per_dispatch_time_unit)
- virtual void **addNode** ([DefaultNode](#) *node)
new public user methods for this component
- virtual void **removeNode** ([DefaultNode](#) *node)
- virtual void **addTransition** ([DefaultNodeTransition](#) *transition)
- virtual void **removeTransition** ([DefaultNodeTransition](#) *transition)
- virtual void **addOutputExpressionReference** ([ModelDataDefinition](#) *expressionReference)
- virtual void **removeOutputExpressionReference** ([DefaultNodeTransition](#) *expressionReference)
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

- Returns the runtime class name used by the plugin system.*
- **Model * getParentModel () const**
Returns the parent model that owns this definition.
- **bool isReportStatistics () const**
Indicates whether this data definition contributes report statistics.
- **void setReportStatistics (bool reportStatistics)**
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition * getInternalData (const std::string &name) const**
Looks up a named internal child data definition.
- **std::map< std::string, ModelDataDefinition * > * getInternalData () const**
Returns the full map of internal child data definitions.
- **std::map< std::string, ModelDataDefinition * > * getAttachedData () const**
Returns the full map of attached data definitions.
- **bool hasChanged () const**
Indicates whether this instance has been modified since the last reset.
- **void setHasChanged (bool hasChanged)**
Updates the changed flag for this data definition.
- **unsigned int getLevel () const**
Returns the level in the model hierarchy where this definition lives.
- **void setModelLevel (unsigned int _modelLevel)**
Sets the model nesting level for this definition.
- **List< SimulationControl * > * getSimulationControls () const**
Returns the simulation controls owned by this data definition.
- **TraceManager::Level getTraceLevelSpecific () const**
Returns the trace level that applies specifically to this definition.
- **void setTraceLevelSpecific (TraceManager::Level level)**
Updates the trace level used by this definition.
- **void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)**
Defines the specific trace level and optionally enables it immediately.
- **bool isTraceLevelSpecificEnabled () const**
Indicates whether the specific trace level is enabled.
- **void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)**
Enables or disables the specific trace level.

Static Public Member Functions

- **static PluginInformation * GetPluginInformation ()**
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- **static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)**
- **static ModelDataDefinition * NewInstance (Model *model, std::string name="")**

Static Public Member Functions inherited from ModelComponent

- **static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)**
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- **static bool Check (ModelComponent *component)**
Validates a component instance.
- **static void CreateInternalData (ModelComponent *component)**
Creates internal data required by a component instance.

- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual bool [_check](#) (std::string &errorMessage) override
virtual protected methods that could be overridden by derived classes, if needed
- virtual void [_initBetweenReplications](#) () override
- virtual void [_createAttachedAttributes](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)

- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Protected Attributes

- const struct [ModalModelDefault::DEFAULT_VALUES](#) **DEFAULT**
- unsigned int **_maxTransitionsPerDispatch** = DEFAULT.maxTransitionsPerDispatch
- std::string **_timeDelayExpressionPerDispatch** = DEFAULT.timeDelayExpressionPerDispatch
- [Util::TimeUnit](#) **_timeDelayPerDispatchTimeUnit** = DEFAULT.timeDelayPerDispatchTimeUnit
- [DefaultNode](#) * **_currentNode** = nullptr
- [DefaultNode](#) * **_entryNode** = nullptr
- [List](#)< [DefaultNode](#) * > * **_nodes** = new [List](#)<[DefaultNode](#)*>()
internal COMPONENTS (since it's a modal model / network) (Composition)
- [List](#)< [DefaultNodeTransition](#) * > * **_transitions** = new [List](#)<[DefaultNodeTransition](#)*>()

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.424.1 Detailed Description

This component ...

15.424.2 Constructor & Destructor Documentation

15.424.2.1 ModalModelDefault()

```
ModalModelDefault::ModalModelDefault (
    Model * model,
    std::string name = "" )
```

constructors

Externalize function GetPluginInformation to be accessible through dynamic linked library.

15.424.3 Member Function Documentation

15.424.3.1 _check()

```
bool ModalModelDefault::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

virtual protected methods that could be overridden by derived classes, if needed

This method is called by ModelChecker during model check. The component should check itself to verify if user parameters are ok (ex: correct syntax for the parser) and everything in its parameters allow the model to run without errors in this component

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [ModalModelFSM](#), and [ModalModelPetriNet](#).

15.424.3.2 _createAttachedAttributes()

```
void ModalModelDefault::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.424.3.3 _initBetweenReplications()

```
void ModalModelDefault::_initBetweenReplications ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelData. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dynamically linked to simulator kernel to reflect the changes

This method is called by [ModelSimulation](#) when initiating the replication. The model should set all value for a new replication (Ex: setting back to 0 any internal counter, clearing lists, etc.

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [ModalModelFSM](#).

15.424.3.4 `_loadInstance()`

```
bool ModalModelDefault::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.424.3.5 `_onDispatchEvent()`

```
void ModalModelDefault::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

This method is only for ModelComponents, not ModelDataElements.

Implements [ModelComponent](#).

15.424.3.6 `_saveInstance()`

```
void ModalModelDefault::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.424.3.7 `show()`

```
std::string ModalModelDefault::show ( ) [override], [virtual]
```

virtual public methods

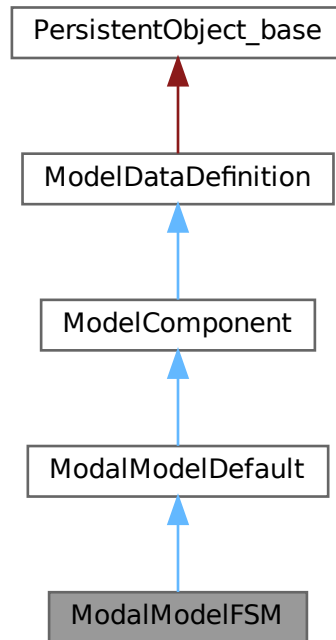
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

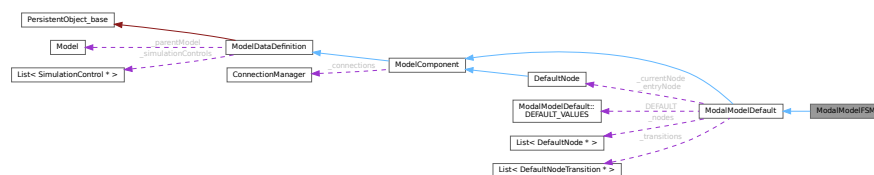
- `plugins/components/ModalModel/ModalModelDefault.h`
- `plugins/components/ModalModel/ModalModelDefault.cpp`

15.425 ModalModelFSM Class Reference

Inheritance diagram for ModalModelFSM:



Collaboration diagram for ModalModelFSM:



Public Member Functions

- **ModalModelFSM** ([Model](#) *model, std::string name="")

Public Member Functions inherited from [ModalModelDefault](#)

- [ModalModelDefault](#) ([Model](#) *model, std::string name="")
constructors
- virtual void **setMaxTransitionsPerDispatch** (unsigned int maxTransitionsPerDispatch)
- virtual unsigned int **getMaxTransitionsPerDispatch** () const
- virtual [List< \[DefaultNode\]\(#\) * >](#) * **getNodes** () const
- virtual [List< \[DefaultNodeTransition\]\(#\) * >](#) * **getTransitions** () const
- [DefaultNode](#) * **getEntryNode** ()
- void **setEntryNode** ([DefaultNode](#) *const entry_node)

- [DefaultNode](#) * **getCurrentNode** ()
- std::string **getTimeDelayExpressionPerDispatch** ()
- void **setTimeDelayExpressionPerDispatch** (const std::string time_delay_expression_per_dispatch)
- [Util::TimeUnit](#) **getTimeDelayPerDispatchTimeUnit** ()
- void **setTimeDelayPerDispatchTimeUnit** (const [Util::TimeUnit](#) time_delay_per_dispatch_time_unit)
- virtual void **addNode** ([DefaultNode](#) *node)
new public user methods for this component
- virtual void **removeNode** ([DefaultNode](#) *node)
- virtual void **addTransition** ([DefaultNodeTransition](#) *transition)
- virtual void **removeTransition** ([DefaultNodeTransition](#) *transition)
- virtual void **addOutputExpressionReference** ([ModelDataDefinition](#) *expressionReference)
- virtual void **removeOutputExpressionReference** ([DefaultNodeTransition](#) *expressionReference)
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~[ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

- Returns the runtime class name used by the plugin system.*
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const
Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)
Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific←Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const
Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static **PluginInformation** * **GetPluginInformation** ()
- static **ModelComponent** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")

Static Public Member Functions inherited from **ModalModelDefault**

- static **PluginInformation** * **GetPluginInformation** ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static **ModelComponent** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_check](#) (std::string &errorMessage) override
virtual protected methods that could be overridden by derived classes, if needed
- virtual void [_initBetweenReplications](#) ()

Protected Member Functions inherited from [ModalModelDefault](#)

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void [_createAttachedAttributes](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
 - legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModalModelDefault](#)

- const struct [ModalModelDefault::DEFAULT_VALUES](#) **DEFAULT**
- unsigned int **_maxTransitionsPerDispatch** = DEFAULT.maxTransitionsPerDispatch
- std::string **_timeDelayExpressionPerDispatch** = DEFAULT.timeDelayExpressionPerDispatch
- [Util::TimeUnit](#) **_timeDelayPerDispatchTimeUnit** = DEFAULT.timeDelayPerDispatchTimeUnit
- [DefaultNode](#) * **_currentNode** = nullptr
- [DefaultNode](#) * **_entryNode** = nullptr
- [List](#)< [DefaultNode](#) * > * **_nodes** = new [List](#)<[DefaultNode](#)*>()
internal COMPONENTS (since it's a modal model / network) (Composition)
- [List](#)< [DefaultNodeTransition](#) * > * **_transitions** = new [List](#)<[DefaultNodeTransition](#)*>()

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- [Util::identification](#) **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager::Level](#) **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.425.1 Member Function Documentation

15.425.1.1 [_check\(\)](#)

```
bool ModalModelFSM::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

virtual protected methods that could be overridden by derived classes, if needed

This method is called by [ModelChecker](#) during model check. The component should check itself to verify if user parameters are ok (ex: correct syntax for the parser) and everything in its parameters allow the model to run without errors in this component

Reimplemented from [ModalModelDefault](#).

15.425.1.2 [_initBetweenReplications\(\)](#)

```
void ModalModelFSM::_initBetweenReplications ( ) [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this [ModelData](#)s. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dynamically linked to to simulator kernel to reflect the changes

This method is called by [ModelSimulation](#) when initiating the replication. The model should set all value for a new replication (Ex: setting back to 0 any internal counter, clearing lists, etc.

Reimplemented from [ModalModelDefault](#).

The documentation for this class was generated from the following files:

- plugins/components/ModalModel/ModalModelFSM.h
- plugins/components/ModalModel/ModalModelFSM.cpp

- [DefaultNode](#) * **getCurrentNode** ()
- std::string **getTimeDelayExpressionPerDispatch** ()
- void **setTimeDelayExpressionPerDispatch** (const std::string time_delay_expression_per_dispatch)
- [Util::TimeUnit](#) **getTimeDelayPerDispatchTimeUnit** ()
- void **setTimeDelayPerDispatchTimeUnit** (const [Util::TimeUnit](#) time_delay_per_dispatch_time_unit)
- virtual void **addNode** ([DefaultNode](#) *node)
new public user methods for this component
- virtual void **removeNode** ([DefaultNode](#) *node)
- virtual void **addTransition** ([DefaultNodeTransition](#) *transition)
- virtual void **removeTransition** ([DefaultNodeTransition](#) *transition)
- virtual void **addOutputExpressionReference** ([ModelDataDefinition](#) *expressionReference)
- virtual void **removeOutputExpressionReference** ([DefaultNodeTransition](#) *expressionReference)
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~[ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

- Returns the runtime class name used by the plugin system.*
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const
Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)
Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific←Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const
Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static **PluginInformation** * **GetPluginInformation** ()
- static **ModelComponent** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")

Static Public Member Functions inherited from **ModalModelDefault**

- static **PluginInformation** * **GetPluginInformation** ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static **ModelComponent** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_check](#) (std::string &errorMessage) override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModalModelDefault](#)

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void [_initBetweenReplications](#) () override
- virtual void [_createAttachedAttributes](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModalModelDefault](#)

- const struct [ModalModelDefault::DEFAULT_VALUES](#) **DEFAULT**
- unsigned int **_maxTransitionsPerDispatch** = DEFAULT.maxTransitionsPerDispatch
- std::string **_timeDelayExpressionPerDispatch** = DEFAULT.timeDelayExpressionPerDispatch
- [Util::TimeUnit](#) **_timeDelayPerDispatchTimeUnit** = DEFAULT.timeDelayPerDispatchTimeUnit
- [DefaultNode](#) * **_currentNode** = nullptr
- [DefaultNode](#) * **_entryNode** = nullptr
- [List](#)< [DefaultNode](#) * > * **_nodes** = new [List](#)<[DefaultNode](#)*>()
internal COMPONENTS (since it's a modal model / network) (Composition)
- [List](#)< [DefaultNodeTransition](#) * > * **_transitions** = new [List](#)<[DefaultNodeTransition](#)*>()

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- [Util::identification](#) **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager::Level](#) **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.426.1 Member Function Documentation

15.426.1.1 `_check()`

```
bool ModalModelPetriNet::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

virtual protected methods that could be overridden by derived classes, if needed

This method is called by ModelChecker during model check. The component should check itself to verify if user parameters are ok (ex: correct syntax for the parser) and everything in its parameters allow the model too run without errors in this component

Reimplemented from [ModalModelDefault](#).

The documentation for this class was generated from the following files:

- plugins/components/ModalModel/ModalModelPetriNet.h
- plugins/components/ModalModel/ModalModelPetriNet.cpp

15.427 Model Class Reference

```
#include <kernel/simulator/model/Model.h>
```

Public Member Functions

- [Model](#) ([Simulator](#) *simulator, unsigned int level=0)
Creates a model bound to a simulator.
- virtual `~Model ()`
Releases model-owned runtime state.

- bool [save](#) (std::string filename)
Saves the model to a file.
- bool [load](#) (std::string filename)
Loads the model from a file.
- bool [check](#) ()
Validates the model structure and configuration.
- void [clear](#) ()
Clears the current model content.
- std::string [showLanguage](#) ()
Returns the model specification in the source language.
- void [show](#) ()
Displays the model contents to the configured trace output.
- bool [insert](#) ([ModelDataDefinition](#) *elemOrComp)
Inserts a model data definition or component into the model.
- void [remove](#) ([ModelDataDefinition](#) *elemOrComp)
Removes a model data definition or component from the model.
- std::list< [ModelDataDefinition](#) * > [collectDataDefinitionsRemovedWith](#) (const std::list< [ModelDataDefinition](#) * > &roots) const
Computes the data definitions that should be removed together with a set of roots.
- [Entity](#) * [createEntity](#) (std::string name, bool insertIntoModel=true)
Creates an entity owned by this model.
- void [removeEntity](#) ([Entity](#) *entity)
Removes an entity owned by this model.
- void [sendEntityToComponent](#) ([Entity](#) *entity, [Connection](#) *connection, double timeDelay=0.0)
Schedules an entity transfer through a specific connection.
- void [sendEntityToComponent](#) ([Entity](#) *entity, [ModelComponent](#) *component, double timeDelay=0.0, unsigned int componentinputPortNumber=0)
Schedules an entity transfer to a specific component and input port.
- double [parseExpression](#) (const std::string expression)
Parses and evaluates an expression.
- double [parseExpression](#) (const std::string expression, bool &success, std::string &errorMessage)
Parses and evaluates an expression while reporting success or failure.
- bool [checkExpression](#) (const std::string expression, const std::string expressionName, std::string &error← Message)
Checks whether an expression is syntactically valid.
- void [checkReferencesToDataDefinitions](#) (std::string expression, std::map< std::string, std::list< std::string > * > *referencedDataDefinitions)
Collects data-definition references found in an expression.
- Util::identification [getId](#) () const
Returns the unique model identifier.
- bool [hasChanged](#) () const
Indicates whether the model has pending changes.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for the model and its owned persistent objects.
- [OnEventManager](#) * [getOnEventManager](#) () const
Returns the event manager that coordinates model-level events.
- [ModelDataManager](#) * [getDataManager](#) () const
Returns the data manager for model data definitions.
- [ModelComponentManager](#) * [getComponentManager](#) () const
Returns the component manager for model components.
- [ModellInfo](#) * [getInfos](#) () const

- Returns the model metadata object.*
- [Simulator](#) * [getParentSimulator](#) () const
Returns the parent simulator that owns this model.
- [ModelSimulation](#) * [getSimulation](#) () const
Returns the simulation controller associated with this model.
- [List](#)< [Event](#) * > * [getFutureEvents](#) () const
Returns the future-event queue.
- [List](#)< [SimulationControl](#) * > * [getControls](#) () const
Returns the simulation controls exposed by the model.
- [List](#)< [SimulationResponse](#) * > * [getResponses](#) () const
Returns the simulation responses exposed by the model.
- void [setTracer](#) ([TraceManager](#) * _traceManager)
Sets the trace manager used by this model.
- [TraceManager](#) * [getTracer](#) () const
Returns the trace manager used by this model.
- [Persistence_if](#) * [getPersistence](#) () const
Returns the persistence interface used by this model.
- unsigned int [getLevel](#) () const
Returns the hierarchy level of the model.

15.427.1 Detailed Description

[Model](#) is probably the most important class of Genesys kernel. It represents a discrete event-driven simulation model. Each model is responsible for controlling its own simulation, ie, for sequentially processing events and collecting statistical results. A model is mainly represented by a collection of components ([ModelComponent](#)), adequately configured and connected, and a collection of under layered modeldatum ([ModelDataDefinition](#)).

15.427.2 Constructor & Destructor Documentation

15.427.2.1 Model()

```
Model::Model (
    Simulator * simulator,
    unsigned int level = 0 )
```

Creates a model bound to a simulator.

Parameters

<i>simulator</i>	Parent simulator that owns the model.
<i>level</i>	Hierarchy level of the model within the simulator.

15.427.3 Member Function Documentation

15.427.3.1 check()

```
bool Model::check ( )
```

Validates the model structure and configuration.

Checks the integrity and consistency of the model, may correct some inconsistencies, and returns whether the model is ready to be simulated.

Returns

`true` when the model is ready to simulate.

15.427.3.2 checkExpression()

```
bool Model::checkExpression (
    const std::string expression,
    const std::string expressionName,
    std::string & errorMessage )
```

Checks whether an expression is syntactically valid.

This method is invoked by ModelComponents and ModelDatas in their private `_check()` methods to validate user-defined expressions.

Parameters

<i>expression</i>	Expression text.
<i>expressionName</i>	Logical name used in diagnostics.
<i>errorMessage</i>	Output error message when validation fails.

Returns

`true` when the expression is valid.

15.427.3.3 checkReferencesToDataDefinitions()

```
void Model::checkReferencesToDataDefinitions (
    std::string expression,
    std::map< std::string, std::list< std::string > * > * referencedDataDefinitions
)
```

Collects data-definition references found in an expression.

Parameters

<i>expression</i>	Expression text.
<i>referencedDataDefinitions</i>	Output map of referenced data definitions by name.

15.427.3.4 collectDataDefinitionsRemovedWith()

```
std::list< ModelDataDefinition * > Model::collectDataDefinitionsRemovedWith (
    const std::list< ModelDataDefinition * > & roots ) const
```

Computes the data definitions that should be removed together with a set of roots.

Parameters

<i>roots</i>	Root objects selected for removal.
--------------	------------------------------------

Returns

Flattened list of dependent data definitions to remove as well.

15.427.3.5 createEntity()

```
Entity * Model::createEntity (
    std::string name,
    bool insertIntoModel = true )
```

Creates an entity owned by this model.

Parameters

<i>name</i>	Entity name.
<i>insertIntoModel</i>	If <code>true</code> , inserts the entity into the model immediately.

Returns

Newly created entity.

15.427.3.6 getComponentManager()

```
ModelComponentManager * Model::getComponentManager ( ) const
```

Returns the component manager for model components.

Returns

Component manager instance.

15.427.3.7 getControls()

```
List< SimulationControl * > * Model::getControls ( ) const
```

Returns the simulation controls exposed by the model.

These are the values that can be externally controlled and usually correspond to input parameters that must be changed for an experimental design.

Returns

[List](#) of writable simulation controls.

15.427.3.8 getDataManager()

```
ModelDataManager * Model::getDataManager ( ) const
```

Returns the data manager for model data definitions.

Provides access to the class that manages the basic elements of the simulation model, such as queues, resources, variables and similar model data definitions.

Returns

Data manager instance.

15.427.3.9 getFutureEvents()

```
List< Event * > * Model::getFutureEvents ( ) const
```

Returns the future-event queue.

The future events list is chronologically sorted. Events are scheduled by components while processing other events, and a replication evolves by sequentially processing the first event in this list. The list is initialized with the events first described by source components.

Returns

Chronologically ordered list of scheduled events.

15.427.3.10 getId()

```
Util::identification Model::getId ( ) const
```

Returns the unique model identifier.

Returns

[Model](#) identifier.

15.427.3.11 getInfos()

`ModelInfo * Model::getInfos ( ) const`

Returns the model metadata object.

Returns

`Model` info instance.

15.427.3.12 getLevel()

`unsigned int Model::getLevel ( ) const`

Returns the hierarchy level of the model.

Provides access to the class that performs the trace of simulation and replications.

Returns

`Model` level.

15.427.3.13 getOnEventManager()

`OnEventManager * Model::getOnEventManager ( ) const`

Returns the event manager that coordinates model-level events.

Provides access to the class that manages events generated by the model, such as the beginning of a new simulation or replication and the processing of an event.

Returns

On-event manager instance.

15.427.3.14 getParentSimulator()

`Simulator * Model::getParentSimulator ( ) const`

Returns the parent simulator that owns this model.

Returns

Parent simulator.

15.427.3.15 getPersistence()

`Persistence_if * Model::getPersistence ( ) const`

Returns the persistence interface used by this model.

Returns

Persistence interface instance.

15.427.3.16 getResponses()

`List< SimulationResponse * > * Model::getResponses ( ) const`

Returns the simulation responses exposed by the model.

These are the exits or simulation results that can be read externally and usually correspond to statistics gathered for an experimental design.

Returns

`List` of readable simulation responses.

15.427.3.17 getSimulation()

```
ModelSimulation * Model::getSimulation ( ) const
```

Returns the simulation controller associated with this model.

Provides access to the class that manages the model simulation.

Returns

Simulation controller instance.

15.427.3.18 getTracer()

```
TraceManager * Model::getTracer ( ) const
```

Returns the trace manager used by this model.

Returns

Trace manager instance.

15.427.3.19 hasChanged()

```
bool Model::hasChanged ( ) const
```

Indicates whether the model has pending changes.

Returns

`true` when the model or one of its owned objects changed.

15.427.3.20 insert()

```
bool Model::insert (
    ModelDataDefinition * elemOrComp )
```

Inserts a model data definition or component into the model.

This is a generic entry point for either `ComponentManager->insert()` or `ModelDataManager->insert()`.

Parameters

<i>elemOrComp</i>	Object to insert.
-------------------	-------------------

Returns

`true` when the object was accepted.

15.427.3.21 load()

```
bool Model::load (
    std::string filename )
```

Loads the model from a file.

Parameters

<i>filename</i>	Input filename.
-----------------	-----------------

Returns

`true` when loading succeeds.

15.427.3.22 parseExpression() [1/2]

```
double Model::parseExpression (
    const std::string expression )
```

Parses and evaluates an expression.

The parser always returns a double, even when the expression has syntax errors, in which case the result is 0.

Parameters

<i>expression</i>	Expression text.
-------------------	------------------

Returns

Numeric result of the expression, or 0 on syntax failure.

15.427.3.23 parseExpression() [2/2]

```
double Model::parseExpression (
    const std::string expression,
    bool & success,
    std::string & errorMessage )
```

Parses and evaluates an expression while reporting success or failure.

The parser always returns a double, even when the expression has syntax errors, in which case the result is 0. This overload also reports whether an error occurred.

Parameters

<i>expression</i>	Expression text.
<i>success</i>	Output flag that receives the parsing result.
<i>errorMessage</i>	Output error message when parsing fails.

Returns

Numeric result of the expression, or 0 on syntax failure.

15.427.3.24 remove()

```
void Model::remove (
    ModelDataDefinition * elemOrComp )
```

Removes a model data definition or component from the model.

This is a generic entry point for either `ComponentManager->remove()` or `ModelDataManager->remove()`.

Parameters

<i>elemOrComp</i>	Object to remove.
-------------------	-------------------

15.427.3.25 removeEntity()

```
void Model::removeEntity (
    Entity * entity )
```

Removes an entity owned by this model.

Parameters

<i>entity</i>	Entity to remove.
---------------	-----------------------------------

15.427.3.26 save()

```
bool Model::save (
    std::string filename )
```

Saves the model to a file.

Parameters

<i>filename</i>	Output filename.
-----------------	------------------

Returns

`true` when saving succeeds.

15.427.3.27 sendEntityToComponent() [1/2]

```
void Model::sendEntityToComponent (
    Entity * entity,
    Connection * connection,
    double timeDelay = 0.0 )
```

Schedules an entity transfer through a specific connection.

Used by components to send entities to the next connected component, or by the model itself while processing an event.

Parameters

<i>entity</i>	Entity to transfer.
<i>connection</i>	Destination connection.
<i>timeDelay</i>	Delay applied before the entity reaches the destination.

15.427.3.28 sendEntityToComponent() [2/2]

```
void Model::sendEntityToComponent (
    Entity * entity,
    ModelComponent * component,
    double timeDelay = 0.0,
    unsigned int componentinputPortNumber = 0 )
```

Schedules an entity transfer to a specific component and input port.

Used by components to send entities to the next connected component, or by the model itself while processing an event.

Parameters

<i>entity</i>	Entity to transfer.
<i>component</i>	Destination component.
<i>timeDelay</i>	Delay applied before the entity reaches the destination.
<i>componentinputPortNumber</i>	Destination input port number.

15.427.3.29 setHasChanged()

```
void Model::setHasChanged (
    bool hasChanged )
```

Updates the changed flag for the model and its owned persistent objects.

Parameters

<i>hasChanged</i>	New changed-state value.
-------------------	--------------------------

15.427.3.30 setTracer()

```
void Model::setTracer (
    TraceManager * _traceManager )
```

Sets the trace manager used by this model.

Parameters

<i>_traceManager</i>	Trace manager instance.
----------------------	-------------------------

15.427.3.31 showLanguage()

```
std::string Model::showLanguage ( )
```

Returns the model specification in the source language.

Returns

Textual model specification.

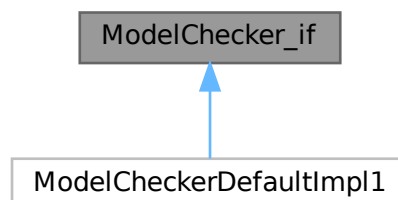
The documentation for this class was generated from the following files:

- kernel/simulator/model/Model.h
- kernel/simulator/model/Model.cpp
- tests/unit/test_simulator_support.cpp
- tests/unit/test_support_modelmanager.cpp

15.428 ModelChecker_if Class Reference

```
#include <kernel/simulator/model/ModelChecker_if.h>
```

Inheritance diagram for ModelChecker_if:



Public Member Functions

- virtual bool [checkAll](#) ()=0
checkAll
- virtual bool [checkConnected](#) ()=0
checkConnected
- virtual bool [checkSymbols](#) ()=0
checkSymbols
- virtual bool [checkActivationCode](#) ()=0
checkActivationCode
- virtual bool [checkLimits](#) ()=0
checkLimits

15.428.1 Detailed Description

The ModelChecker is responsible for verifying the model consistency, fixing inconsistencies whenever possible

15.428.2 Member Function Documentation

15.428.2.1 [checkActivationCode\(\)](#)

```
virtual bool ModelChecker_if::checkActivationCode ( ) [pure virtual]
checkActivationCode
```

Returns

Checks if the installed version has acquired a valid activation code for commercial use

Implemented in [ModelCheckerDefaultImpl1](#).

15.428.2.2 [checkAll\(\)](#)

```
virtual bool ModelChecker_if::checkAll ( ) [pure virtual]
checkAll
```

Returns

Invokes all other checks and returns true only if all of them returned true

Implemented in [ModelCheckerDefaultImpl1](#).

15.428.2.3 [checkConnected\(\)](#)

```
virtual bool ModelChecker_if::checkConnected ( ) [pure virtual]
checkConnected
```

Returns

Checks if components are consistently connected to other to form a valid process-oriented model, describing how entities proceed to the flow

Implemented in [ModelCheckerDefaultImpl1](#).

15.428.2.4 [checkLimits\(\)](#)

```
virtual bool ModelChecker_if::checkLimits ( ) [pure virtual]
checkLimits
```

Returns

Checks if the model (its components and elements) fits the limits based on the activation code

Implemented in [ModelCheckerDefaultImpl1](#).

15.428.2.5 checkSymbols()

```
virtual bool ModelChecker_if::checkSymbols ( ) [pure virtual]
checkSymbols
```

Returns

Checks if user-defined strings for symbols required by components, usually expressions or functions, are valid or references existing and valid elements.

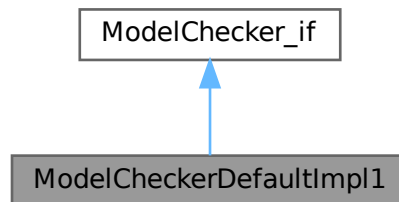
Implemented in [ModelCheckerDefaultImpl1](#).

The documentation for this class was generated from the following file:

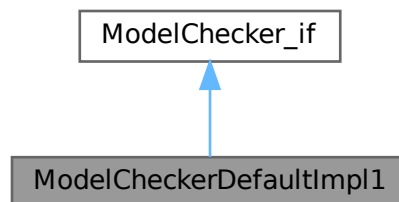
- kernel/simulator/model/ModelChecker_if.h

15.429 ModelCheckerDefaultImpl1 Class Reference

Inheritance diagram for ModelCheckerDefaultImpl1:



Collaboration diagram for ModelCheckerDefaultImpl1:



Public Member Functions

- **ModelCheckerDefaultImpl1** ([Model](#) *model)
- virtual bool [checkAll](#) () override
checkAll
- virtual bool [checkConnected](#) () override
checkConnected
- virtual bool [checkSymbols](#) () override

- checkSymbols*
- virtual bool [checkActivationCode](#) () override
- checkActivationCode*
- virtual bool [checkLimits](#) () override
- checkLimits*

15.429.1 Member Function Documentation

15.429.1.1 [checkActivationCode\(\)](#)

```
bool ModelCheckerDefaultImpl1::checkActivationCode ( ) [override], [virtual]
checkActivationCode
```

Returns

Checks if the installed version has acquired a valid activation code for commercial use

Implements [ModelChecker_if](#).

15.429.1.2 [checkAll\(\)](#)

```
bool ModelCheckerDefaultImpl1::checkAll ( ) [override], [virtual]
checkAll
```

Returns

Invokes all other checks and returns true only if all of them returned true

Implements [ModelChecker_if](#).

15.429.1.3 [checkConnected\(\)](#)

```
bool ModelCheckerDefaultImpl1::checkConnected ( ) [override], [virtual]
checkConnected
```

Returns

Checks if components are consistently connected to other to form a valid process-oriented model, describing how entities proceed to the flow

Implements [ModelChecker_if](#).

15.429.1.4 [checkLimits\(\)](#)

```
bool ModelCheckerDefaultImpl1::checkLimits ( ) [override], [virtual]
checkLimits
```

Returns

Checks if the model (its components and elements) fits the limits based on the activation code

Implements [ModelChecker_if](#).

15.429.1.5 [checkSymbols\(\)](#)

```
bool ModelCheckerDefaultImpl1::checkSymbols ( ) [override], [virtual]
checkSymbols
```

Returns

Checks if user-defined strings for symbols required by components, usually expressions or functions, are valid or references existing and valid elements.

Implements [ModelChecker_if](#).

The documentation for this class was generated from the following files:

- kernel/simulator/model/ModelCheckerDefaultImpl1.h
- kernel/simulator/model/ModelCheckerDefaultImpl1.cpp

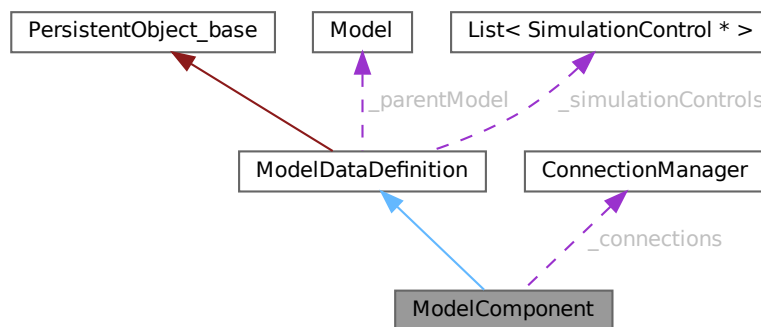
15.430 ModelComponent Class Reference

```
#include <kernel/simulator/model/ModelComponent.h>
```

Inherits [ModelDataDefinition](#).

Inherited by [AIAssistant](#), [Access](#), [Assign](#), [BacteriaColony](#), [Batch](#), [BioRunnerCommand](#), [BioSimulate](#), [BioStateProjectionComponent](#), [BioSteadyState](#), [Buffer](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CellularAutomataComp](#), [Clone](#), [CompartmentExchangeComponent](#), [ContinuousSystemComponent](#), [CppForG](#), [Decide](#), [DefaultNode](#), [Delay](#), [DiffEquations](#), [DiffusionSimulate](#), [DropOff](#), [DummyComponent](#), [Enter](#), [EukaryoticCellCycleComponent](#), [Exit](#), [FtsZPolymerizationComponent](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [LSODE](#), [Leave](#), [MarkovChain](#), [Match](#), [MetabolicFluxBalance](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelComponent](#), [ModalModelDefault](#), [PathwayStressResponseComponent](#), [PickStation](#), [Process](#), [PythonForG](#), [RSimulator](#), [Record](#), [Release](#), [Remove](#), [ResourceAllocationComponent](#), [Route](#), [SPICECircuit](#), [SPICENode](#), [Search](#), [Seize](#), [Separate](#), [Signal](#), [SinkModelComponent](#), [SourceModelComponent](#), [Start](#), [StochasticReactionComponent](#), [StochasticTranscription](#), [StochasticTranslation](#), [Stop](#), [Store](#), [Submodel](#), [Unstore](#), [Wait](#), and [Write](#).

Collaboration diagram for ModelComponent:



Public Member Functions

- [ModelComponent](#) ([Model](#) *model, std::string componentName, std::string name="")
constructors
- virtual ~[ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.
- virtual std::string [show](#) () override
virtual public methods

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()

Releases resources owned by this data definition.
- Util::identification [getId](#) () const

Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)

Updates the logical name of this instance.
- const std::string & [getName](#) () const

Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const

Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const

Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const

Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const

Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const

Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const

Returns the full map of attached data definitions.
- bool [hasChanged](#) () const

Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))

Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const

Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const

Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const

Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)

Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const

Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber)=0
Handles a dispatched event for this component type.

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()

- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
- legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
- method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Protected Attributes

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
- virtual protected methods that could be overridden by derived classes, if needed*

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.430.1 Detailed Description

A component of the model is a block that represents a specific behavior to be simulated. The behavior is triggered when an entity arrives at the component, which corresponds to the occurrence of an event. A simulation model corresponds to a set of interconnected components to form the process by which the entity is submitted.

Parameters

<i>model</i>	The model this component belongs to
--------------	-------------------------------------

15.430.2 Constructor & Destructor Documentation

15.430.2.1 ModelComponent()

```
ModelComponent::ModelComponent (
    Model * model,
    std::string componentTypename,
    std::string name = "" )
```

constructors

Creates a model component and registers it in the component manager.

Parameters

<i>model</i>	Parent model.
<i>componentTypename</i>	Runtime type name for the concrete component.
<i>name</i>	Initial component name.

15.430.3 Member Function Documentation

15.430.3.1 _loadInstance()

```
bool ModelComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [SinkModelComponent](#), [SourceModelComponent](#), [AIAssistant](#), [MarkovChain](#), [BacteriaColony](#), [BioRunnerCommand](#), [BioSimulate](#), [BioSteadyState](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [MetabolicFluxBalance](#), [ContinuousSystemComponent](#), [DiffEquations](#), [DiffusionSimulate](#), [LSODE](#), [Decide](#), [DropOff](#), [Remove](#), [Search](#), [Buffer](#), [Clone](#), [Delay](#), [Process](#), [Release](#), [Seize](#), [SPICECircuit](#), [Resistor](#), [Vsource](#), [Vpulse](#), [Vsine](#), [Capacitor](#), [Diode](#), [PMOS](#), [NMOS](#), [NOT](#), [NAND](#), [AND](#), [NOR](#), [OR](#), [XOR](#), [XNOR](#), [SPICENode](#), [CppForG](#), [PythonForG](#), [RSimulator](#), [Batch](#), [Separate](#), [Record](#), [Write](#), [Assign](#), [Create](#), [Dispose](#), [Access](#), [Enter](#), [Exit](#), [Leave](#), [PickStation](#), [Route](#), [Start](#), [Stop](#), [Store](#), [Unstore](#), [CellularAutomataComp](#), [DefaultNode](#), [FSMState](#), [ModalModelDefault](#), [PetriPlace](#), [Submodel](#), [Match](#), [Signal](#), [Wait](#), [DummyComponent](#), [BioStateProjectionComponent](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CompartmentExchangeComponent](#),

[EukaryoticCellCycleComponent](#), [FtsZPolymerizationComponent](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelComponent](#), [PathwayStressResponseComponent](#), [ResourceAllocationComponent](#), [StochasticReactionComponent](#), [StochasticTranscription](#), and [StochasticTranslation](#).

15.430.3.2 `_onDispatchEvent()`

```
virtual void ModelComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [protected], [pure virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implemented in [AIAssistant](#), [MarkovChain](#), [BacteriaColony](#), [BioRunnerCommand](#), [BioSimulate](#), [BioSteadyState](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [MetabolicFluxBalance](#), [ContinuousSystemComponent](#), [DiffEquations](#), [DiffusionSimulate](#), [LSODE](#), [Decide](#), [DropOff](#), [Remove](#), [Search](#), [Buffer](#), [Clone](#), [Delay](#), [Process](#), [Release](#), [Seize](#), [SPICECircuit](#), [SPICENode](#), [CppForG](#), [PythonForG](#), [RSimulator](#), [Batch](#), [Separate](#), [Record](#), [Write](#), [Assign](#), [Create](#), [Dispose](#), [Access](#), [Enter](#), [Exit](#), [Leave](#), [PickStation](#), [Route](#), [Start](#), [Stop](#), [Store](#), [Unstore](#), [CellularAutomataComp](#), [DefaultNode](#), [ModalModelDefault](#), [Submodel](#), [Match](#), [Signal](#), [Wait](#), [DummyComponent](#), [BioStateProjectionComponent](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CompartmentExchangeComponent](#), [EukaryoticCellCycleComponent](#), [FtsZPolymerizationComponent](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelComponent](#), [PathwayStressResponseComponent](#), [ResourceAllocationComponent](#), [StochasticReactionComponent](#), [StochasticTranscription](#), [StochasticTranslation](#), [RecordingContinuousSystemComponent](#), [RecordingContinuousSystemComponent](#), and [RecordingLSODE](#).

15.430.3.3 `_saveInstance()`

```
void ModelComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [SinkModelComponent](#), [SourceModelComponent](#), [AIAssistant](#), [MarkovChain](#), [BacteriaColony](#), [BioRunnerCommand](#), [BioSimulate](#), [BioSteadyState](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [MetabolicFluxBalance](#), [ContinuousSystemComponent](#), [DiffEquations](#), [DiffusionSimulate](#), [LSODE](#), [Decide](#), [DropOff](#), [Remove](#), [Search](#), [Buffer](#), [Clone](#), [Delay](#), [Process](#), [Release](#), [Seize](#), [SPICECircuit](#), [Resistor](#), [Vsource](#), [Vpulse](#), [Vsine](#), [Capacitor](#), [Diode](#), [PMOS](#), [NMOS](#), [NOT](#), [NAND](#), [AND](#), [NOR](#), [OR](#), [XOR](#), [XNOR](#), [SPICENode](#), [CppForG](#), [PythonForG](#), [RSimulator](#), [Batch](#), [Separate](#), [Record](#), [Write](#), [Assign](#), [Create](#), [Dispose](#), [Access](#), [Enter](#), [Exit](#), [Leave](#), [PickStation](#), [Route](#), [Start](#), [Stop](#), [Store](#), [Unstore](#), [CellularAutomataComp](#), [DefaultNode](#), [FSMState](#), [ModalModelDefault](#), [PetriPlace](#), [Submodel](#), [Match](#), [Signal](#), [Wait](#), [DummyComponent](#), [BioStateProjectionComponent](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CompartmentExchangeComponent](#), [EukaryoticCellCycleComponent](#), [FtsZPolymerizationComponent](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelComponent](#), [PathwayStressResponseComponent](#), [ResourceAllocationComponent](#), [StochasticReactionComponent](#), [StochasticTranscription](#), and [StochasticTranslation](#).

15.430.3.4 `Check()`

```
bool ModelComponent::Check (
```

```
ModelComponent * component ) [static]
```

Validates a component instance.

Parameters

<i>component</i>	Component to validate.
------------------	------------------------

Returns

true when the component is valid.

15.430.3.5 connectTo()

```
void ModelComponent::connectTo (
    ModelComponent * component,
    unsigned int inputPortNumber = 0 )
```

Connects this component to another component.

Parameters

<i>component</i>	Target component.
<i>inputPortNumber</i>	Input port number of the target component.

15.430.3.6 CreateInternalData()

```
void ModelComponent::CreateInternalData (
    ModelComponent * component ) [static]
```

Creates internal data required by a component instance.

Parameters

<i>component</i>	Component whose related data should be created.
------------------	---

15.430.3.7 DispatchEvent()

```
void ModelComponent::DispatchEvent (
    Event * event ) [static]
```

Dispatches a simulation event to the target component.

This method triggers the simulation behavior of the component. It is invoked when an event corresponding to this component is taken from the future events list or when an entity arrives through a connection.

Parameters

<i>event</i>	Event to process.
--------------	-------------------

15.430.3.8 getConnectionManager()

```
ConnectionManager * ModelComponent::getConnectionManager ( ) const
```

new public user methods for this component

Returns the connection manager owned by this component.

This manager stores the components directly connected to the output. Usually a component has a single output, but it may have none (such as [Dispose](#)) or more than one (such as [Decide](#)). Along with each connected component, the manager stores the input port number where the entity will be sent. Usually components have a single input, but they may have none (such as [Create](#)) or more than one (such as [Match](#)).

Returns

[Connection](#) manager that stores outgoing connections.

15.430.3.9 getDescription()

```
std::string ModelComponent::getDescription ( ) const
```

Returns the free-form description associated with this component.

Returns

Description text.

15.430.3.10 hasBreakpointAt()

```
bool ModelComponent::hasBreakpointAt ( )
```

Indicates whether a simulation breakpoint is set on this component.

Returns

`true` when the simulation breakpoint list contains this component.

15.430.3.11 LoadInstance()

```
static ModelComponent * ModelComponent::LoadInstance (
    Model * model,
    PersistenceRecord * fields ) [static]
```

static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component

Loads a component instance from serialized fields.

Parameters

<i>model</i>	Parent model that will own the component.
<i>fields</i>	Serialized fields for the component.

Returns

Newly created component instance.

15.430.3.12 remove()

```
void ModelComponent::remove (
    ModelComponent * comp )
```

Removes a connected component from this component.

Parameters

<i>comp</i>	Component to remove from the outgoing connection set.
-------------	---

15.430.3.13 SaveInstance()

```
void ModelComponent::SaveInstance (
    PersistenceRecord * fields,
    ModelComponent * component ) [static]
```

Serializes a component instance into a persistence record.

Parameters

<i>fields</i>	Output persistence record.
<i>component</i>	Component to serialize.

15.430.3.14 setDescription()

```
void ModelComponent::setDescription (
    std::string _description )
```

Updates the free-form description shown by the GUI.

Parameters

<i>_description</i>	New description text.
---------------------	-----------------------

15.430.3.15 show()

```
std::string ModelComponent::show ( ) [override], [virtual]
virtual public methods
```

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [SourceModelComponent](#), [AIAssistant](#), [MarkovChain](#), [BacteriaColony](#), [BioRunnerCommand](#), [BioSimulate](#), [BioSteadyState](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [MetabolicFluxBalance](#), [ContinuousSystemComponent](#), [DiffEquations](#), [DiffusionSimulate](#), [LSODE](#), [Decide](#), [DropOff](#), [Remove](#), [Search](#), [Buffer](#), [Clone](#), [Delay](#), [Process](#), [Release](#), [Seize](#), [SPICECircuit](#), [SPICENode](#), [CppForG](#), [PythonForG](#), [RSimulator](#), [Batch](#), [Separate](#), [Record](#), [Write](#), [Assign](#), [Create](#), [Dispose](#), [Access](#), [Enter](#), [Exit](#), [Leave](#), [PickStation](#), [Route](#), [Start](#), [Stop](#), [Store](#), [Unstore](#), [CellularAutomataComp](#), [DefaultNode](#), [ModalModelDefault](#), [Submodel](#), [Match](#), [Signal](#), [Wait](#), [DummyComponent](#), [BioStateProjectionComponent](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CompartmentExchangeComponent](#), [EukaryoticCellCycleComponent](#), [FtsZPolymerizationComponent](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelComponent](#), [PathwayStressResponseComponent](#), [ResourceAllocationComponent](#), [StochasticReactionComponent](#), [StochasticTranscription](#), and [StochasticTranslation](#).

15.430.4 Member Data Documentation**15.430.4.1 _connections**

```
ConnectionManager* ModelComponent::_connections = new ConnectionManager() [protected]
```

virtual protected methods that could be overridden by derived classes, if needed

This method is called by ModelChecker during model check. The component should check itself to verify if user parameters are ok (ex: correct syntax for the parser) and everything in its parameters allow the model to run without errors in this component

This method returns all changes in the parser that are needed by plugins of this ModelData. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dynamically linked to simulator kernel to reflect the changes

This method is called by [ModelSimulation](#) when initiating the replication. The model should set all value for a new replication (Ex: setting back to 0 any internal counter, clearing lists, etc).

Legacy compatibility hook. Prefer the granular create* methods and the composed association manager.

This method is not used yet. It should be useful for new UIs

The documentation for this class was generated from the following files:

- kernel/simulator/model/ModelComponent.h
- kernel/simulator/model/ModelComponent.cpp

15.431 ModelComponentManager Class Reference

Owns and organizes the set of [ModelComponent](#) objects in a model.

```
#include <kernel/simulator/model/ModelComponentManager.h>
```

Public Member Functions

- **ModelComponentManager** ([Model](#) *model)
Creates a component manager attached to a model.
- bool **insert** ([ModelComponent](#) *comp)
insert
- void **remove** ([ModelComponent](#) *comp)
remove
- [ModelComponent](#) * **find** (std::string name)
find
- [ModelComponent](#) * **find** (Util::identification id)
find
- void **clear** ()
clear
- unsigned int **getNumberOfComponents** ()
getNumberOfComponents
- std::list< [ModelComponent](#) * >::iterator **begin** ()
begin
- std::list< [ModelComponent](#) * >::iterator **end** ()
end
- [ModelComponent](#) * **front** ()
front
- [ModelComponent](#) * **next** ()
next
- bool **hasChanged** () const
hasChanged
- void **setHasChanged** (bool _hasChanged)
setHasChanged
- std::list< [SourceModelComponent](#) * > * **getSourceComponents** ()
getSourceComponents
- std::list< [ModelComponent](#) * > * **getTransferInComponents** ()
getTransferInComponents
- std::list< [ModelComponent](#) * > * **getAllComponents** () const
getAllComponents

15.431.1 Detailed Description

Owns and organizes the set of [ModelComponent](#) objects in a model.

Provides insertion/removal, lookup and iteration helpers while tracking structural changes relevant to model validation/persistence workflows.

15.431.2 Member Function Documentation

15.431.2.1 begin()

```
std::list< ModelComponent * >::iterator ModelComponentManager::begin ( )  
begin
```

Returns

Iterator to the first component.

15.431.2.2 clear()

```
void ModelComponentManager::clear ( )
```

clear

Removes all managed components from the model.

15.431.2.3 end()

```
std::list< ModelComponent * >::iterator ModelComponentManager::end ( )
```

end

Returns

Iterator past the last component.

15.431.2.4 find() [1/2]

```
ModelComponent * ModelComponentManager::find (
    std::string name )
```

find

Parameters

<i>name</i>	
-------------	--

Returns

Pointer to the component, or `nullptr` when not found.

15.431.2.5 find() [2/2]

```
ModelComponent * ModelComponentManager::find (
    Util::identification id )
```

find

Parameters

<i>id</i>	
-----------	--

Returns

Pointer to the component, or `nullptr` when not found.

15.431.2.6 front()

```
ModelComponent * ModelComponentManager::front ( )
```

front

Returns

First component in iteration order.

15.431.2.7 getAllComponents()

```
std::list< ModelComponent * > * ModelComponentManager::getAllComponents ( ) const
```

getAllComponents

Returns

15.431.2.8 getNumberOfComponents()

```
unsigned int ModelComponentManager::getNumberOfComponents ( )  
getNumberOfComponents
```

Returns

Number of managed components.

15.431.2.9 getSourceComponents()

```
std::list< SourceModelComponent * > * ModelComponentManager::getSourceComponents ( )  
getSourceComponents
```

Returns

15.431.2.10 getTransferInComponents()

```
std::list< ModelComponent * > * ModelComponentManager::getTransferInComponents ( )  
getTransferInComponents
```

Returns

15.431.2.11 hasChanged()

```
bool ModelComponentManager::hasChanged ( ) const  
hasChanged
```

Returns

15.431.2.12 insert()

```
bool ModelComponentManager::insert (   
    ModelComponent * comp )  
insert
```

Parameters

<i>comp</i>	
-------------	--

Returns

True when insertion succeeds.

15.431.2.13 next()

```
ModelComponent * ModelComponentManager::next ( )  
next
```

Returns

Next component in iteration order.

15.431.2.14 remove()

```
void ModelComponentManager::remove (
    ModelComponent * comp )
```

remove

Parameters

<i>comp</i>	
-------------	--

15.431.2.15 setHasChanged()

```
void ModelComponentManager::setHasChanged (
    bool _hasChanged )
```

setHasChanged

Parameters

<i>_hasChanged</i>	
--------------------	--

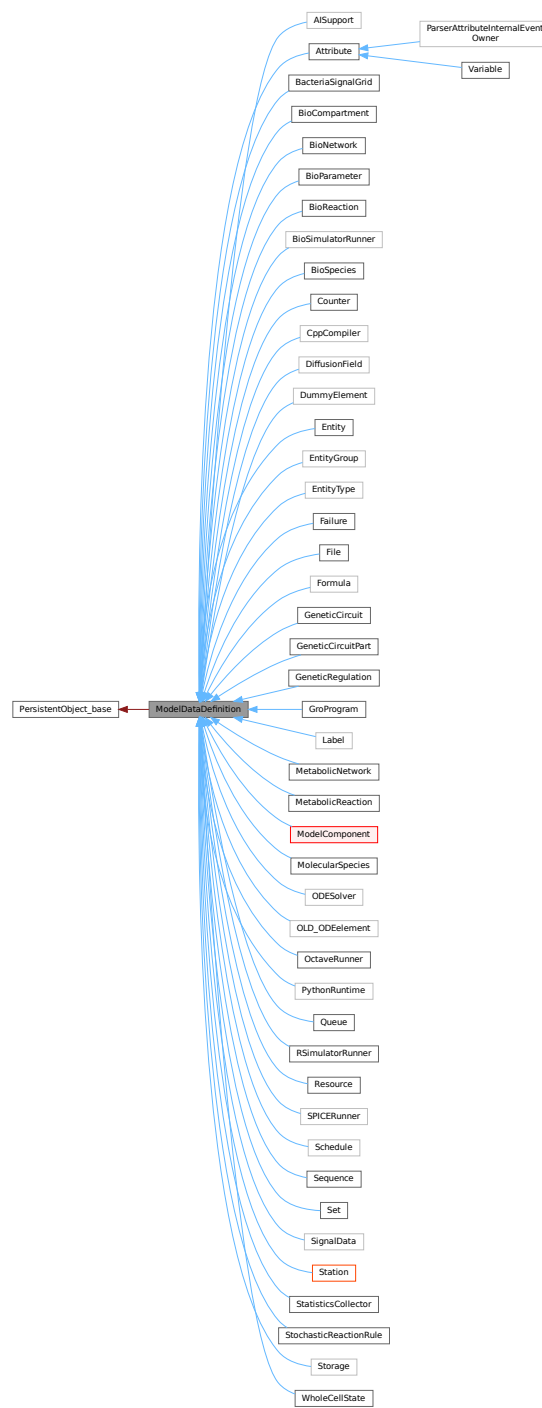
The documentation for this class was generated from the following files:

- kernel/simulator/model/ModelComponentManager.h
- kernel/simulator/model/ModelComponentManager.cpp

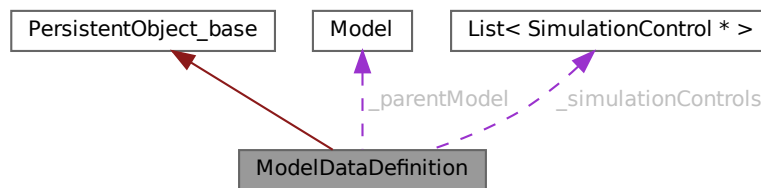
15.432 ModelDataDefinition Class Reference

```
#include <kernel/simulator/model/ModelDataDefinition.h>
```


Inheritance diagram for ModelDataDefinition:



Collaboration diagram for ModelDataDefinition:



Public Member Functions

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionType, std::string name="", bool insertIntoModel=true)
 - Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
 - Releases resources owned by this data definition.*
- Util::identification **getId** () const
 - Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
 - Updates the logical name of this instance.*
- const std::string & **getName** () const
 - Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)
 - Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const
 - Returns the user-visible label of this instance.*
- const std::string & **getClassname** () const
 - Returns the runtime class name used by the plugin system.*
- **Model** * **getParentModel** () const
 - Returns the parent model that owns this definition.*
- bool **isReportStatistics** () const
 - Indicates whether this data definition contributes report statistics.*
- void **setReportStatistics** (bool reportStatistics)
 - Enables or disables report-statistics generation for this definition.*
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
 - Looks up a named internal child data definition.*
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
 - Returns the full map of internal child data definitions.*
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
 - Returns the full map of attached data definitions.*
- bool **hasChanged** () const
 - Indicates whether this instance has been modified since the last reset.*
- void **setHasChanged** (bool hasChanged)
 - Updates the changed flag for this data definition.*
- unsigned int **getLevel** () const
 - Returns the level in the model hierarchy where this definition lives.*
- void **setModelLevel** (unsigned int _modelLevel)

- Sets the model nesting level for this definition.*
- [List< SimulationControl * > * getSimulationControls \(\)](#) const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level getTraceLevelSpecific \(\)](#) const
Returns the trace level that applies specifically to this definition.
- [void setTraceLevelSpecific \(TraceManager::Level level\)](#)
Updates the trace level used by this definition.
- [void defineTraceLevelSpecific \(TraceManager::Level traceLevelspecific, bool traceLevelSpecificEnabled=true\)](#)
Defines the specific trace level and optionally enables it immediately.
- [bool isTraceLevelSpecificEnabled \(\)](#) const
Indicates whether the specific trace level is enabled.
- [void setTraceLevelSpecificEnabled \(bool traceLevelSpecificEnabled\)](#)
Enables or disables the specific trace level.
- [virtual std::string show \(\)](#)
Returns a textual representation of this data definition.

Static Public Member Functions

- [static ModelDataDefinition * LoadInstance \(Model *model, PersistenceRecord *fields, bool insertIntoModel\)](#)
Loads a model data definition instance from serialized fields.
- [static ModelDataDefinition * NewInstance \(Model *model, std::string name=""\)](#)
Creates a new instance of a data definition using the plugin registry.
- [static void SaveInstance \(PersistenceRecord *fields, ModelDataDefinition *modeldatum\)](#)
Serializes a data definition instance into a persistence record.
- [static bool Check \(ModelDataDefinition *modeldatum, std::string &errorMessage\)](#)
Validates a data definition instance.
- [static void InitBetweenReplications \(ModelDataDefinition *modeldatum\)](#)
Resets simulation-time state between replications.
- [static void CreateInternalData \(ModelDataDefinition *modeldatum\)](#)
Creates internal and attached data required by this instance.
- [static bool CreateRelatedDataElements \(ModelDataDefinition *modeldatum, std::string &errorMessage\)](#)
Creates related data and validates the instance in a single call.

Protected Member Functions

- [bool _getSaveDefaultsOption \(\)](#)
- [virtual bool _loadInstance \(PersistenceRecord *fields\)](#) override
must be overridden by derived classes
- [virtual void _saveInstance \(PersistenceRecord *fields, bool saveDefaultValues\)](#) override
Saves object fields into a serialization structure.
- [virtual bool _check \(std::string &errorMessage\)](#)
could be overridden by derived classes
- [virtual ParserChangesInformation * _getParserChangesInformation \(\)](#)
- [virtual void _initBetweenReplications \(\)](#)
- [virtual void _addSimulationControl \(SimulationControl *control\)](#)
- [virtual void _createInternalAndAttachedData \(\)](#)
- [virtual void _createInternalStatisticReporters \(\)](#)
- [virtual void _createAttachedAttributes \(\)](#)
- [virtual void _createNonEditableDataDefinitions \(\)](#)
- [virtual void _createEditableDataDefinitions \(\)](#)
- [void _templateCreateInternalStatisticReporters \(\)](#)
- [void _templateCreateAttachedAttributes \(\)](#)

- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
 - legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Protected Attributes

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

Friends

- class **ModelDataDefinitionAssociations**

15.432.1 Detailed Description

This class is the basis for any modeldatum of the model (such as [Queue](#), [Resource](#), [Variable](#), etc.) and also for any component of the model. It has the infrastructure to read and write on file and to verify symbols.

15.432.2 Constructor & Destructor Documentation**15.432.2.1 ModelDataDefinition()**

```
ModelDataDefinition::ModelDataDefinition (
    Model * model,
    std::string datadefinitionTypename,
    std::string name = "",
    bool insertIntoModel = true )
```

Creates a model data definition and optionally inserts it into the model.

Parameters

<i>model</i>	Parent model that owns this data definition.
<i>datadefinitionTypename</i>	Runtime type name for the concrete data definition.
<i>name</i>	Initial object name.
<i>insertIntoModel</i>	If <code>true</code> , the new instance is registered in the model immediately.

15.432.3 Member Function Documentation**15.432.3.1 _addSimulationControl()**

```
void ModelDataDefinition::_addSimulationControl (
    SimulationControl * control ) [protected], [virtual]
```

Reimplemented in [Buffer](#), [PickStation](#), and [Schedule](#).

15.432.3.2 _check()

```
bool ModelDataDefinition::_check (
    std::string & errorMessage ) [protected], [virtual]
```

could be overridden by derived classes

Reimplemented in [Attribute](#), [Counter](#), [Entity](#), [StatisticsCollector](#), [SinkModelComponent](#), [SourceModelComponent](#), [AIAssistant](#), [MarkovChain](#), [BacteriaColony](#), [BioRunnerCommand](#), [BioSimulate](#), [BioSteadyState](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [MetabolicFluxBalance](#), [ContinuousSystemComponent](#), [DiffEquations](#), [DiffusionSimulate](#), [LSODE](#), [OLD_ODEelement](#), [Decide](#), [DropOff](#), [Remove](#), [Search](#), [Buffer](#), [Clone](#), [Delay](#), [Process](#), [Release](#), [Seize](#), [CppForG](#), [PythonForG](#), [RSimulator](#), [Batch](#), [Separate](#), [Record](#), [Write](#), [Assign](#), [Create](#), [Dispose](#), [Access](#), [Enter](#), [Exit](#), [Leave](#), [PickStation](#), [Route](#), [Start](#), [Stop](#), [Store](#), [Unstore](#), [ModalModelDefault](#), [ModalModelFSM](#), [ModalModelPetriNet](#), [Submodel](#), [Match](#), [Signal](#), [Wait](#), [BioStateProjectionComponent](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CompartmentExchangeComponent](#), [EukaryoticCellCycleComponent](#), [FtsZPolymerizationComponent](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelComponent](#), [PathwayStressResponseCom](#), [ResourceAllocationComponent](#), [StochasticReactionComponent](#), [StochasticTranscription](#), [StochasticTranslation](#), [AISupport](#), [BacteriaSignalGrid](#), [BioNetwork](#), [BioParameter](#), [BioReaction](#), [BioSimulatorRunner](#), [BioSpecies](#), [GeneticCircuit](#), [GeneticCircuitPart](#), [GeneticRegulation](#), [GroProgram](#), [MetabolicNetwork](#), [MetabolicReaction](#), [DiffusionField](#), [ODESolver](#), [Failure](#), [Queue](#), [Resource](#), [Schedule](#), [CppCompiler](#), [OctaveRunner](#), [PythonRuntime](#), [RSimulatorRunner](#), [SPICERunner](#), [EntityGroup](#), [File](#), [Formula](#), [Label](#), [Set](#), [Variable](#), [Sequence](#), [Station](#), [Storage](#), [SignalData](#), [DummyElement](#), [BioCompartment](#), [MolecularSpecies](#), [StochasticReactionRule](#), and [WholeCellState](#).

15.432.3.3 _createAttachedAttributes()

void ModelDataDefinition::_createAttachedAttributes () [protected], [virtual]
 Reimplemented in [Formula](#).

15.432.3.4 _createEditableDataDefinitions()

void ModelDataDefinition::_createEditableDataDefinitions () [protected], [virtual]
 Reimplemented in [DiffEquations](#), [Remove](#), [Search](#), [R Simulator](#), [Assign](#), and [BioNetwork](#).

15.432.3.5 _createInternalAndAttachedData()

void ModelDataDefinition::_createInternalAndAttachedData () [protected], [virtual]
 Legacy compatibility hook. Prefer the granular create* methods and the composed association manager.

15.432.3.6 _createNonEditableDataDefinitions()

void ModelDataDefinition::_createNonEditableDataDefinitions () [protected], [virtual]
 Reimplemented in [SPICECircuit](#), and [CppForG](#).

15.432.3.7 _getParserChangesInformation()

[ParserChangesInformation](#) * ModelDataDefinition::_getParserChangesInformation () [protected], [virtual]

This method returns all changes in the parser that are needed by plugins of this ModelDats. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dinamically linked to to simulator kernel to reflect the changes

Reimplemented in [Buffer](#), [PickStation](#), [Queue](#), [Schedule](#), [File](#), [Set](#), and [Storage](#).

15.432.3.8 _getSaveDefaultsOption()

bool ModelDataDefinition::_getSaveDefaultsOption () [protected]
 Returns a list of keys (names) of internal ModelDats, such as Counters, StatisticsCollectors and others. Children↵
 Elements are ModelDats used by this [ModelDataDefinition](#) thar are needed before model checking

15.432.3.9 _initBetweenReplications()

void ModelDataDefinition::_initBetweenReplications () [protected], [virtual]
 Reimplemented in [ModalModelFSM](#), [CppForG](#), [CellularAutomataComp](#), and [ModalModelDefault](#).

15.432.3.10 _loadInstance()

bool ModelDataDefinition::_loadInstance (
 [PersistenceRecord](#) * fields) [override], [protected], [virtual]

must be overridden by derived classes

Implements [PersistentObject_base](#).

Reimplemented in [Attribute](#), [Counter](#), [Entity](#), [EntityType](#), [StatisticsCollector](#), [ModelComponent](#), [SinkModelComponent](#), [SourceModelComponent](#), [AIAssistant](#), [MarkovChain](#), [BacteriaColony](#), [BioRunnerCommand](#), [BioSimulate](#), [BioSteadyState](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [MetabolicFluxBalance](#), [ContinuousSystemComponent](#), [DiffEquations](#), [DiffusionSimulate](#), [LSODE](#), [OLD_ODEelement](#), [Decide](#), [DropOff](#), [Remove](#), [Search](#), [Buffer](#), [Clone](#), [Delay](#), [Process](#), [Release](#), [Seize](#), [SPICECircuit](#), [Resistor](#), [Vsource](#), [Vpulse](#), [Vsine](#), [Capacitor](#), [Diode](#), [PMOS](#), [NMOS](#), [NOT](#), [NAND](#), [AND](#), [NOR](#), [OR](#), [XOR](#), [XNOR](#), [SPICENode](#), [CppForG](#), [PythonForG](#), [R Simulator](#), [Batch](#), [Separate](#), [Record](#), [Write](#), [Assign](#), [Create](#), [Dispose](#), [Access](#), [Enter](#), [Exit](#), [Leave](#), [PickStation](#), [Route](#), [Start](#), [Stop](#), [Store](#), [Unstore](#), [CellularAutomataComp](#), [DefaultNode](#), [FSMState](#), [ModalModelDefault](#), [PetriPlace](#), [Submodel](#), [Match](#), [Signal](#), [Wait](#), [DummyComponent](#), [BioStateProjectionComponent](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CompartmentExchangeComponent](#), [EukaryoticCellCycleComponent](#), [FtsZPolymerizationComponent](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelCompone](#), [PathwayStressResponseComponent](#), [ResourceAllocationComponent](#), [StochasticReactionComponent](#), [StochasticTranscription](#), [StochasticTranslation](#), [AISupport](#), [BacteriaSignalGrid](#), [BioNetwork](#), [BioParameter](#), [BioReaction](#), [BioSimulatorRunner](#), [BioSpecies](#), [GeneticCircuit](#), [GeneticCircuitPart](#), [GeneticRegulation](#), [GroProgram](#), [MetabolicNetwork](#), [MetabolicReaction](#),

[DiffusionField](#), [ODESolver](#), [Failure](#), [Queue](#), [Resource](#), [Schedule](#), [CppCompiler](#), [OctaveRunner](#), [PythonRuntime](#), [RSimulatorRunner](#), [SPICERunner](#), [EntityGroup](#), [File](#), [Formula](#), [Label](#), [Set](#), [Variable](#), [Sequence](#), [Station](#), [Storage](#), [SignalData](#), [DummyElement](#), [BioCompartment](#), [MolecularSpecies](#), [StochasticReactionRule](#), and [WholeCellState](#).

15.432.3.11 `_saveInstance()`

```
void ModelDataDefinition::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Implements [PersistentObject_base](#).

Reimplemented in [Attribute](#), [Counter](#), [Entity](#), [EntityType](#), [StatisticsCollector](#), [ModelComponent](#), [SinkModelComponent](#), [SourceModelComponent](#), [AIAssistant](#), [MarkovChain](#), [BacteriaColony](#), [BioRunnerCommand](#), [BioSimulate](#), [BioSteadyState](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [MetabolicFluxBalance](#), [ContinuousSystemComponent](#), [DiffEquations](#), [DiffusionSimulate](#), [LSODE](#), [OLD_ODEelement](#), [Decide](#), [DropOff](#), [Remove](#), [Search](#), [Buffer](#), [Clone](#), [Delay](#), [Process](#), [Release](#), [Seize](#), [SPICECircuit](#), [Resistor](#), [Vsource](#), [Vpulse](#), [Vsine](#), [Capacitor](#), [Diode](#), [PMOS](#), [NMOS](#), [NOT](#), [NAND](#), [AND](#), [NOR](#), [OR](#), [XOR](#), [XNOR](#), [SPICENode](#), [CppForG](#), [PythonForG](#), [RSimulator](#), [Batch](#), [Separate](#), [Record](#), [Write](#), [Assign](#), [Create](#), [Dispose](#), [Access](#), [Enter](#), [Exit](#), [Leave](#), [PickStation](#), [Route](#), [Start](#), [Stop](#), [Store](#), [Unstore](#), [CellularAutomataComp](#), [DefaultNode](#), [FSMState](#), [ModalModelDefault](#), [PetriPlace](#), [Submodel](#), [Match](#), [Signal](#), [Wait](#), [DummyComponent](#), [BioStateProjectionComponent](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CompartmentExchangeComponent](#), [EukaryoticCellCycleComponent](#), [FtsZPolymerizationComponent](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelComponent](#), [PathwayStressResponseComponent](#), [ResourceAllocationComponent](#), [StochasticReactionComponent](#), [StochasticTranscription](#), [StochasticTranslation](#), [AISupport](#), [BacteriaSignalGrid](#), [BioNetwork](#), [BioParameter](#), [BioReaction](#), [BioSimulatorRunner](#), [BioSpecies](#), [GeneticCircuit](#), [GeneticCircuitPart](#), [GeneticRegulation](#), [GroProgram](#), [MetabolicNetwork](#), [MetabolicReaction](#), [DiffusionField](#), [ODESolver](#), [Failure](#), [Queue](#), [Resource](#), [Schedule](#), [CppCompiler](#), [OctaveRunner](#), [PythonRuntime](#), [RSimulatorRunner](#), [SPICERunner](#), [EntityGroup](#), [File](#), [Formula](#), [Label](#), [Set](#), [Variable](#), [Sequence](#), [Station](#), [Storage](#), [SignalData](#), [DummyElement](#), [BioCompartment](#), [MolecularSpecies](#), [StochasticReactionRule](#), and [WholeCellState](#).

15.432.3.12 `Check()`

```
bool ModelDataDefinition::Check (
    ModelDataDefinition * modeldatum,
    std::string & errorMessage ) [static]
```

Validates a data definition instance.

This helper invokes the instance [_check\(\)](#) method to validate the object and any data it creates or references.

Parameters

<i>modeldatum</i>	Instance to validate.
<i>errorMessage</i>	Output error message when validation fails.

Returns

`true` when the instance is valid.

15.432.3.13 `CreateInternalData()`

```
void ModelDataDefinition::CreateInternalData (
    ModelDataDefinition * modeldatum ) [static]
```

Creates internal and attached data required by this instance.

This helper invokes the protected `_check()` path of the instance, which creates internal `ModelDataDefinitions` and other related data such as attributes or variables.

Parameters

<i>modeldatum</i>	Instance whose associated data should be created.
-------------------	---

15.432.3.14 CreateRelatedDataElements()

```
bool ModelDataDefinition::CreateRelatedDataElements (
    ModelDataDefinition * modeldatum,
    std::string & errorMessage ) [static]
```

Creates related data and validates the instance in a single call.

Materializes related data and runs local checks for the instance in a single step.

Parameters

<i>modeldatum</i>	Instance whose related data should be materialized.
<i>errorMessage</i>	Output error message when validation fails.

Returns

`true` when creation and validation succeed.

15.432.3.15 defineTraceLevelSpecific()

```
void ModelDataDefinition::defineTraceLevelSpecific (
    TraceManager::Level traceLevelspecific,
    bool traceLevelspecificEnabled = true )
```

Defines the specific trace level and optionally enables it immediately.

Parameters

<i>traceLevelspecific</i>	New trace level.
<i>traceLevelspecificEnabled</i>	Whether the specific trace level starts enabled.

15.432.3.16 getAttachedData()

```
std::map< std::string, ModelDataDefinition * > * ModelDataDefinition::getAttachedData ( )
const
```

Returns the full map of attached data definitions.

Returns

Map of attached data indexed by name.

15.432.3.17 getClassname()

```
const std::string & ModelDataDefinition::getClassname ( ) const
```

Returns the runtime class name used by the plugin system.

Returns

Concrete class name.

15.432.3.18 getId()

`Util::identification ModelDataDefinition::getId ( ) const`
Returns the unique identifier of this instance.

Returns

Instance identifier.

15.432.3.19 getInternalData() [1/2]

`std::map< std::string, ModelDataDefinition * > * ModelDataDefinition::getInternalData ( ) const`
Returns the full map of internal child data definitions.

Returns

Map of internal children indexed by name.

15.432.3.20 getInternalData() [2/2]

`ModelDataDefinition * ModelDataDefinition::getInternalData ( const std::string & name ) const`
Looks up a named internal child data definition.

Parameters

<i>name</i>	Child name to search for.
-------------	---------------------------

Returns

Matching internal child, or `nullptr` when not found.

15.432.3.21 getLabel()

`const std::string & ModelDataDefinition::getLabel ( ) const`
Returns the user-visible label of this instance.

Returns

Instance label.

15.432.3.22 getLevel()

`unsigned int ModelDataDefinition::getLevel ( ) const`
Returns the level in the model hierarchy where this definition lives.

Returns

[Model](#) nesting level.

15.432.3.23 getName()

`const std::string & ModelDataDefinition::getName ( ) const`
Returns the logical name of this instance.

Returns

Instance name.

15.432.3.24 getParentModel()

```
Model * ModelDataDefinition::getParentModel ( ) const
```

Returns the parent model that owns this definition.

Returns

Parent model pointer, or `nullptr` when detached.

15.432.3.25 getSimulationControls()

```
List< SimulationControl * > * ModelDataDefinition::getSimulationControls ( ) const
```

Returns the simulation controls owned by this data definition.

Preferred API for writable controls owned by this model data definition.

Returns

Writable control list managed by the definition.

15.432.3.26 getTraceLevelSpecific()

```
TraceManager::Level ModelDataDefinition::getTraceLevelSpecific ( ) const
```

Returns the trace level that applies specifically to this definition.

Returns

Explicit trace level for this instance.

15.432.3.27 hasChanged()

```
bool ModelDataDefinition::hasChanged ( ) const
```

Indicates whether this instance has been modified since the last reset.

Returns

`true` when the object has pending changes.

15.432.3.28 InitBetweenReplications()

```
void ModelDataDefinition::InitBetweenReplications (
    ModelDataDefinition * modeldatum ) [static]
```

Resets simulation-time state between replications.

Parameters

<i>modeldatum</i>	Instance to reset.
-------------------	--------------------

15.432.3.29 isReportStatistics()

```
bool ModelDataDefinition::isReportStatistics ( ) const
```

Indicates whether this data definition contributes report statistics.

Returns

`true` when this definition emits simulation statistics.

15.432.3.30 isTraceLevelSpecificEnabled()

```
bool ModelDataDefinition::isTraceLevelSpecificEnabled ( ) const
```

Indicates whether the specific trace level is enabled.

Returns

`true` when the instance-specific trace level is active.

15.432.3.31 LoadInstance()

```
ModelDataDefinition * ModelDataDefinition::LoadInstance (
    Model * model,
    PersistenceRecord * fields,
    bool insertIntoModel ) [static]
```

Loads a model data definition instance from serialized fields.

This helper receives a map of fields read from a file or other source, creates the instance, and invokes its protected `_loadInstance()` method. The instance can be automatically inserted into the simulation model when requested.

Parameters

<i>model</i>	Parent model that will own the loaded instance.
<i>fields</i>	Serialized fields to deserialize.
<i>insertIntoModel</i>	If <code>true</code> , registers the loaded instance in the model.

Returns

Newly created instance when loading succeeds.

15.432.3.32 NewInstance()

```
ModelDataDefinition * ModelDataDefinition::NewInstance (
    Model * model,
    std::string name = "" ) [static]
```

Creates a new instance of a data definition using the plugin registry.

This helper invokes the constructor and returns a new instance that usually requires a cast to the correct subclass. It is used when plugins are connected through dynamically loaded libraries.

Parameters

<i>model</i>	Parent model that will own the instance.
<i>name</i>	Initial object name.

Returns

Newly created data definition instance.

15.432.3.33 SaveInstance()

```
void ModelDataDefinition::SaveInstance (
    PersistenceRecord * fields,
    ModelDataDefinition * modeldatum ) [static]
```

Serializes a data definition instance into a persistence record.

This helper calls the instance `_saveInstance()` method and returns a field map that can be saved to a file or other destination.

Parameters

<i>fields</i>	Output record that receives the serialized fields.
<i>modeldatum</i>	Instance to serialize.

15.432.3.34 setHasChanged()

```
void ModelDataDefinition::setHasChanged (
    bool hasChanged )
```

Updates the changed flag for this data definition.

Parameters

<i>hasChanged</i>	New changed-state value.
-------------------	--------------------------

15.432.3.35 setLabel()

```
void ModelDataDefinition::setLabel (
    const std::string & label )
```

Updates the user-visible label of this instance.

Parameters

<i>label</i>	New label.
--------------	------------

15.432.3.36 setModelLevel()

```
void ModelDataDefinition::setModelLevel (
    unsigned int _modelLevel )
```

Sets the model nesting level for this definition.

Parameters

<i>_modelLevel</i>	New hierarchy level.
--------------------	----------------------

15.432.3.37 setName()

```
void ModelDataDefinition::setName (
    const std::string & name )
```

Updates the logical name of this instance.

Parameters

<i>name</i>	New name.
-------------	-----------

15.432.3.38 setReportStatistics()

```
void ModelDataDefinition::setReportStatistics (
    bool reportStatistics )
```

Enables or disables report-statistics generation for this definition.

Parameters

<i>reportStatistics</i>	New report-statistics flag.
-------------------------	-----------------------------

15.432.3.39 setTraceLevelSpecific()

```
void ModelDataDefinition::setTraceLevelSpecific (
```

```
TraceManager::Level level )
```

Updates the trace level used by this definition.

Parameters

<i>level</i>	New trace level.
--------------	------------------

15.432.3.40 setTraceLevelSpecificEnabled()

```
void ModelDataDefinition::setTraceLevelSpecificEnabled (
    bool traceLevelSpecificEnabled )
```

Enables or disables the specific trace level.

Parameters

<i>traceLevelSpecificEnabled</i>	New enabled-state value.
----------------------------------	--------------------------

15.432.3.41 show()

```
std::string ModelDataDefinition::show ( ) [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented in [Attribute](#), [Counter](#), [Entity](#), [EntityType](#), [StatisticsCollector](#), [ModelComponent](#), [SourceModelComponent](#), [AIAssistant](#), [MarkovChain](#), [BacteriaColony](#), [BioRunnerCommand](#), [BioSimulate](#), [BioSteadyState](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [MetabolicFluxBalance](#), [ContinuousSystemComponent](#), [DiffEquations](#), [DiffusionSimulate](#), [LSODE](#), [OLD_ODEelement](#), [Decide](#), [DropOff](#), [Remove](#), [Search](#), [Buffer](#), [Clone](#), [Delay](#), [Process](#), [Release](#), [Seize](#), [SPICECircuit](#), [SPICENode](#), [CppForG](#), [PythonForG](#), [RSimulator](#), [Batch](#), [Separate](#), [Record](#), [Write](#), [Assign](#), [Create](#), [Dispose](#), [Access](#), [Enter](#), [Exit](#), [Leave](#), [PickStation](#), [Route](#), [Start](#), [Stop](#), [Store](#), [Unstore](#), [CellularAutomataComp](#), [DefaultNode](#), [ModalModelDefault](#), [Submodel](#), [Match](#), [Signal](#), [Wait](#), [DummyComponent](#), [BioStateProjectionComponent](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CompartmentExchangeComponent](#), [EukaryoticCellCycleComponent](#), [FtsZPolymerizationComponent](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelComponent](#), [PathwayStressResponseComponent](#), [ResourceAllocationComponent](#), [StochasticReactionComponent](#), [StochasticTranscription](#), [StochasticTranslation](#), [AISupport](#), [BacteriaSignalGrid](#), [BioNetwork](#), [BioParameter](#), [BioReaction](#), [BioSimulatorRunner](#), [BioSpecies](#), [GeneticCircuit](#), [GeneticCircuitPart](#), [GeneticRegulation](#), [GroProgram](#), [MetabolicNetwork](#), [MetabolicReaction](#), [DiffusionField](#), [ODESolver](#), [Failure](#), [Queue](#), [Resource](#), [Schedule](#), [CppCompiler](#), [OctaveRunner](#), [PythonRuntime](#), [RSimulatorRunner](#), [SPICERunner](#), [EntityGroup](#), [File](#), [Formula](#), [Label](#), [Set](#), [Variable](#), [Sequence](#), [Station](#), [Storage](#), [SignalData](#), [DummyElement](#), [BioCompartment](#), [MolecularSpecies](#), [StochasticReactionRule](#), and [WholeCellState](#).

15.432.3.42 traceSimulation()

```
void ModelDataDefinition::traceSimulation (
    void * thisobject,
    const std::string & text,
    TraceManager::Level level = TraceManager::Level::L8_detailed ) [protected]
```

Traces text to the simulation output.

Used only while a simulation is running, for example when components or data elements need to inform something.

Parameters

<i>thisobject</i>	Caller object used for trace attribution.
<i>text</i>	Trace text to emit.
<i>level</i>	Trace severity level.

The documentation for this class was generated from the following files:

- kernel/simulator/model/ModelDataDefinition.h
- kernel/simulator/model/ModelDataDefinition.cpp
- tests/unit/test_simulator_support.cpp

15.433 ModelDataDefinitionAssociations Class Reference

```
#include <kernel/simulator/model/ModelDataDefinitionAssociations.h>
```

Public Member Functions

- **ModelDataDefinitionAssociations** ([ModelDataDefinition](#) *owner)
- std::map< std::string, [ModelDataDefinition](#) * > * **getInternalData** () const
- std::map< std::string, [ModelDataDefinition](#) * > * **getAttachedData** () const
- [ModelDataDefinition](#) * **getInternalData** (const std::string &key) const
- void **clearAll** ()
- void **clearInternalData** ()
- void **clearAttachedData** ()
- void **internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **internalDataRemove** (const std::string &key)
- void **attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **attachedDataRemove** (const std::string &key)
- void **attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
- void **createInternalAndAttachedData** ()
- void **createInternalStatisticReporters** ()
- void **createAttachedAttributes** ()
- void **createNonEditableDataDefinitions** ()
- void **createEditableDataDefinitions** ()
- void **insertStatisticReporter** (const std::string &key, [ModelDataDefinition](#) *data)
- void **removeStatisticReporter** (const std::string &key)
- void **clearStatisticReporters** ()
- void **insertMandatoryAttachedAttribute** (const std::string &key, [ModelDataDefinition](#) *data)
- void **removeMandatoryAttachedAttribute** (const std::string &key)
- void **clearMandatoryAttachedAttributes** ()
- void **insertMandatoryEditableDataDefinition** (const std::string &key, [ModelDataDefinition](#) *data)
- void **removeMandatoryEditableDataDefinition** (const std::string &key)
- void **clearMandatoryEditableDataDefinitions** ()
- void **insertOptionalEditableDataDefinition** (const std::string &key, [ModelDataDefinition](#) *data)
- void **removeOptionalEditableDataDefinition** (const std::string &key)
- void **clearOptionalEditableDataDefinitions** ()
- void **insertMandatoryNonEditableDataDefinition** (const std::string &key, [ModelDataDefinition](#) *data)
- void **removeMandatoryNonEditableDataDefinition** (const std::string &key)
- void **clearMandatoryNonEditableDataDefinitions** ()
- void **insertOptionalNonEditableDataDefinition** (const std::string &key, [ModelDataDefinition](#) *data)
- void **removeOptionalNonEditableDataDefinition** (const std::string &key)
- void **clearOptionalNonEditableDataDefinitions** ()

15.433.1 Detailed Description

Owns the internal/attached registries and the new lifecycle buckets for a [ModelDataDefinition](#).

The class starts as a compatibility layer around the current internal/attached storage so the kernel can move bucket by bucket without breaking direct callers that still inspect the legacy aggregate maps.

The documentation for this class was generated from the following files:

- kernel/simulator/model/ModelDataDefinitionAssociations.h
- kernel/simulator/model/ModelDataDefinitionAssociations.cpp

15.434 ModelDataManager Class Reference

Central registry for [ModelDataDefinition](#) instances owned by a model.

```
#include <kernel/simulator/model/ModelDataManager.h>
```

Public Member Functions

- **ModelDataManager** ([Model](#) *model)
Creates a data manager attached to model.
- bool **insert** ([ModelDataDefinition](#) *anElement)
insert
- void **remove** ([ModelDataDefinition](#) *anElement)
remove
- bool **insert** (std::string datadefinitionTypename, [ModelDataDefinition](#) *anElement)
insert
- void **remove** (std::string datadefinitionTypename, [ModelDataDefinition](#) *anElement)
remove
- bool **check** (std::string datadefinitionTypename, [ModelDataDefinition](#) *anElement, std::string expression↵
Name, std::string &errorMessage)
check
- bool **check** (std::string datadefinitionTypename, std::string elementName, std::string expressionName, bool
mandatory, std::string &errorMessage)
check
- void **clear** ()
clear
- [ModelDataDefinition](#) * **getDataDefinition** (std::string datadefinitionTypename, Util::identification id)
getDataDefinition
- [ModelDataDefinition](#) * **getDataDefinition** (std::string datadefinitionTypename, std::string name)
getDataDefinition
- unsigned int **getNumberOfDataDefinitions** (std::string datadefinitionTypename)
getNumberOfDataDefinitions
- unsigned int **getNumberOfDataDefinitions** ()
getNumberOfDataDefinitions
- int **getRankOf** (std::string datadefinitionTypename, std::string name)
getRankOf
- std::list< std::string > **getDataDefinitionClassnames** () const
getDataDefinitionClassnames
- [List](#)< [ModelDataDefinition](#) * > * **getDataDefinitionList** (std::string datadefinitionTypename) const
- void **show** ()
Prints a textual summary of registered model data definitions.
- [Model](#) * **getParentModel** () const
Returns the model that owns this data manager.
- bool **hasChanged** () const
Indicates whether managed data collections were modified.
- void **setHasChanged** (bool _hasChanged)
Updates the internal changed flag.

15.434.1 Detailed Description

Central registry for [ModelDataDefinition](#) instances owned by a model.

This manager groups model data by typename, supports insertion/removal/lookup operations and offers consistency checks used by components and model validation.

15.434.2 Member Function Documentation

15.434.2.1 check() [1/2]

```
bool ModelDataManager::check (
    std::string datadefinitionTypename,
    ModelDataDefinition * anElement,
    std::string expressionName,
    std::string & errorMessage )
```

check

Parameters

<i>datadefinitionTypename</i>	
<i>anElement</i>	
<i>expressionName</i>	
<i>errorMessage</i>	

Returns

15.434.2.2 check() [2/2]

```
bool ModelDataManager::check (
    std::string datadefinitionTypename,
    std::string elementName,
    std::string expressionName,
    bool mandatory,
    std::string & errorMessage )
```

check

Parameters

<i>datadefinitionTypename</i>	
<i>elementName</i>	
<i>expressionName</i>	
<i>mandatory</i>	
<i>errorMessage</i>	

Returns

15.434.2.3 getDataDefinition() [1/2]

```
ModelDataDefinition * ModelDataManager::getDataDefinition (
    std::string datadefinitionTypename,
    std::string name )
```

getDataDefinition

Parameters

<i>datadefinitionTypename</i>	
<i>name</i>	

Returns

15.434.2.4 getDataDefinition() [2/2]

```
ModelDataDefinition * ModelDataManager::getDataDefinition (
    std::string datadefinitionTypename,
    Util::identification id )
```

getDataDefinition

Parameters

<i>datadefinitionTypename</i>	
<i>id</i>	

Returns

15.434.2.5 getDataDefinitionClassnames()

```
std::list< std::string > ModelDataManager::getDataDefinitionClassnames ( ) const
getDataDefinitionClassnames
```

Returns

Snapshot list of current data-definition class names returned by value.

15.434.2.6 getNumberOfDataDefinitions() [1/2]

```
unsigned int ModelDataManager::getNumberOfDataDefinitions ( )
getNumberOfDataDefinitions
```

Returns

15.434.2.7 getNumberOfDataDefinitions() [2/2]

```
unsigned int ModelDataManager::getNumberOfDataDefinitions (
    std::string datadefinitionTypename )
```

getNumberOfDataDefinitions

Parameters

<i>datadefinitionTypename</i>	
-------------------------------	--

Returns

15.434.2.8 getRankOf()

```
int ModelDataManager::getRankOf (
    std::string datadefinitionTypename,
    std::string name )
```

getRankOf

Parameters

<i>datadefinitionTypename</i>	
<i>name</i>	

Returns

returns the position (1st position=0) of the modeldatum if found, or negative value if not found

15.434.2.9 insert() [1/2]

```
bool ModelDataManager::insert (  
    ModelDataDefinition * anElement )
```

insert

Parameters

<i>anElement</i>	
------------------	--

Returns

15.434.2.10 insert() [2/2]

```
bool ModelDataManager::insert (  
    std::string datadefinitionTypename,  
    ModelDataDefinition * anElement )
```

insert

Parameters

<i>datadefinitionTypename</i>	
<i>anElement</i>	

Returns

Deprected

15.434.2.11 remove() [1/2]

```
void ModelDataManager::remove (  
    ModelDataDefinition * anElement )
```

remove

Parameters

<i>anElement</i>	Deprected
------------------	-----------

15.434.2.12 remove() [2/2]

```
void ModelDataManager::remove (  
    std::string datadefinitionTypename,  
    ModelDataDefinition * anElement )
```

remove

Parameters

<i>datadefinitionTypename</i>	
<i>anElement</i>	

The documentation for this class was generated from the following files:

- kernel/simulator/model/ModelDataManager.h
- kernel/simulator/model/ModelDataManager.cpp

15.435 ModelEvent Class Reference

```
#include <kernel/simulator/OnEventManager.h>
```

Public Member Functions

- **ModelEvent** ([Model](#) *model)
- const [Model](#) * **getModel** ()

15.435.1 Detailed Description

Stores an event that happened on a specific model

The documentation for this class was generated from the following file:

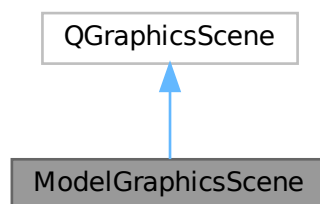
- kernel/simulator/OnEventManager.h

15.436 ModelGraphicsScene Class Reference

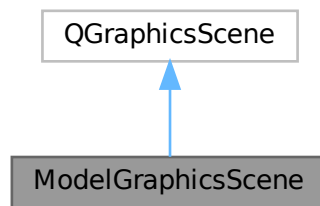
Scene that owns and manipulates the graphical representation of a model.

```
#include <applications/gui/genesys/graphics/ModelGraphicsScene.h>
```

Inheritance diagram for ModelGraphicsScene:



Collaboration diagram for ModelGraphicsScene:



Classes

- struct [GRID](#)

Public Types

- enum **DrawingMode** {
NONE , **LINE** , **TEXT** , **RECTANGLE** ,
ELLIPSE , **POLYGON** , **POLYGON_POINTS** , **POLYGON_FINISHED** ,
COUNTER , **VARIABLE** , **TIMER** , **ATTRIBUTE** ,
ENTITY , **EVENT** , **EXPRESSION** , **PLOT** ,
QUEUE_PLACEHOLDER , **RESOURCE** , **STATION** , **STATISTICS** }

Public Member Functions

- **ModelGraphicsScene** (qreal x, qreal y, qreal width, qreal height, QObject *parent=nullptr)
- **ModelGraphicsScene** (const [ModelGraphicsScene](#) &orig)=delete
- [ModelGraphicsScene](#) & **operator=** (const [ModelGraphicsScene](#) &other)=delete
- [GraphicalModelComponent](#) * **addGraphicalModelComponent** ([Plugin](#) *plugin, [ModelComponent](#) *component, QPointF position, QColor color=Qt::blue, bool notify=false, [GraphicalModelComponent](#) *auto↔ ConnectSource=nullptr)
Creates and inserts one graphical component into the scene.
- [GraphicalConnection](#) * **addGraphicalConnection** ([GraphicalComponentPort](#) *sourcePort, [GraphicalComponentPort](#) *destinationPort, unsigned int portSourceConnection, unsigned int portDestinationConnection, bool notify=false)
Creates and inserts one graphical connection between two ports.
- [GraphicalModelDataDefinition](#) * **addGraphicalModelDataDefinition** ([Plugin](#) *plugin, [ModelDataDefinition](#) *element, QPointF position, QColor color=Qt::blue)
Creates and inserts one graphical data definition into the scene.
- [GraphicalDiagramConnection](#) * **addGraphicalDiagramConnection** (QGraphicsItem *dataDefinition, QGraphicsItem *linkedTo, GraphicalDiagramConnection::ConnectionType type)
Creates or updates a diagram-level connection between data definitions.
- void **initializeAnimationDrawing** (QGraphicsSceneMouseEvent *mouseEvent)
Starts the interactive drawing workflow for animation-related items.
- void **continueAnimationDrawing** (QGraphicsSceneMouseEvent *mouseEvent)
Continues the interactive drawing workflow.
- void **finishAnimationDrawing** (QGraphicsSceneMouseEvent *mouseEvent)
Finalizes the interactive drawing workflow.
- void **startTextEditing** ()

- Opens inline text editing for the current text item.*

 - void **removeComponent** ([GraphicalModelComponent](#) *gmc, bool notify=false)

Removes one component and optionally emits the graphical change notification.
- void **clearConnectionsComponent** ([GraphicalModelComponent](#) *gmc)

Clears input and output connections attached to one component.
- void **clearInputConnectionsComponent** ([GraphicalModelComponent](#) *graphicalComponent)

Removes all incoming connections from one component.
- void **clearOutputConnectionsComponent** ([GraphicalModelComponent](#) *graphicalComponent)

Removes all outgoing connections from one component.
- void **clearPorts** ([GraphicalConnection](#) *connection, [GraphicalModelComponent](#) *source, [GraphicalModelComponent](#) *destination)

Clears the connection ports associated with one connection before removal.
- void **connectComponents** ([GraphicalConnection](#) *connection, [GraphicalModelComponent](#) *source=nullptr, [GraphicalModelComponent](#) *destination=nullptr, bool notify=false)

Connects one prepared connection to its source and destination items.
- bool **connectSource** ([GraphicalConnection](#) *connection, [GraphicalModelComponent](#) *source=nullptr)

Binds a source component to a connection while checking port constraints.
- bool **connectDestination** ([GraphicalConnection](#) *connection, [GraphicalModelComponent](#) *destination=nullptr)

Binds a destination component to a connection while checking port constraints.
- void **redoConnections** ([GraphicalModelComponent](#) *graphicalComponent, QList< [GraphicalConnection](#) * > *inputConnections, QList< [GraphicalConnection](#) * > *outputConnections)

Recreates all connection links for one graphical component.
- void **removeComponentInModel** ([GraphicalModelComponent](#) *gmc)

Removes one component from the kernel model as part of scene deletion.
- void **insertComponent** ([GraphicalModelComponent](#) *gmc, QList< [GraphicalConnection](#) * > *input↔Connections, QList< [GraphicalConnection](#) * > *outputConnections, bool addGMC=true, bool addAll↔GMC=true, bool notify=false)

Inserts an existing component back into the scene and restores its connections.
- void **removeGraphicalConnection** ([GraphicalConnection](#) *graphicalConnection, [GraphicalModelComponent](#) *source, [GraphicalModelComponent](#) *destination, bool notify=false)

Removes a graphical connection and updates the source/destination items.
- void **removeConnectionInModel** ([GraphicalConnection](#) *graphicalConnection, [GraphicalModelComponent](#) *source)

Removes the connection from the kernel model while keeping scene state consistent.
- void **addGeometry** (QPointF endPoint, bool moving)

Adds one freeform drawing geometry item to the scene.
- void **addDrawing** (QGraphicsItem *item, bool notify=false)

Adds one drawing item to the scene and optionally notifies listeners.
- bool **addDrawingGeometry** (QGraphicsItem *item)

Attempts to register one geometry drawing item.
- bool **addDrawingAnimation** (QGraphicsItem *item)

Attempts to register one animation drawing item.
- void **removeGraphicalModelDataDefinition** ([GraphicalModelDataDefinition](#) *gmdd)

Removes one graphical data definition from the scene.
- void **detachGraphicalModelDataDefinition** ([GraphicalModelDataDefinition](#) *gmdd)

Detaches one graphical data definition without deleting the underlying model data.
- void **restoreGraphicalModelDataDefinition** ([GraphicalModelDataDefinition](#) *gmdd)

Restores one previously detached graphical data definition.
- void **removeGraphicalDiagramConnection** ([GraphicalDiagramConnection](#) *connection)

Removes one diagram connection from the scene.
- void **clearGraphicalModelDataDefinitions** ()

Clears all graphical data definitions from the scene.

- void **clearGraphicalDiagramConnections** ()
Clears all diagram connections from the scene.
- void **sanitizeGraphicalDataDefinitionsBookkeeping** ()
Repairs bookkeeping used to track graphical data definitions.
- void **setDiagramLayerState** (bool diagramCreated, bool visible)
Shows or hides the diagram layer and its creation state.
- QList< QGraphicsItem * > **userOperableItems** (const QList< QGraphicsItem * > &items) const
Filters out scene items that the user cannot directly manipulate.
- QList< QGraphicsItem * > **userDeletableItems** (const QList< QGraphicsItem * > &items) const
Filters the selected items down to items that can be deleted.
- void **ensureInitialInternalDataDefinitionGrouping** (GraphicalModelDataDefinition *dataDefinition, GraphicalModelComponent *component)
Optionally groups newly created internal data definitions under their component.
- void **setPersistedGuiRestoreInProgress** (bool restoring)
Marks that persisted GUI state is being restored.
- bool **isPersistedGuiRestoreInProgress** () const
Returns whether persisted GUI state restoration is in progress.
- void **removeDrawing** (QGraphicsItem *item, bool notify=false)
Removes one freeform drawing item from the scene.
- bool **removeDrawingGeometry** (QGraphicsItem *item)
- bool **removeDrawingAnimation** (QGraphicsItem *item)
- void **removeGroup** (QGraphicsItemGroup *group, bool notify=false)
Removes one graphical group item from the scene.
- void **clearGraphicalModelComponents** ()
Clears all graphical component items from the scene.
- void **clearGraphicalModelConnections** ()
Clears all graphical connection items from the scene.
- void **groupComponents** (bool notify=false)
Groups the current selection into a scene group when possible.
- void **groupModelComponents** (QList< GraphicalModelComponent * > *graphicalComponents, QGraphicsItemGroup *group)
Groups the supplied graphical components into the provided group item.
- void **ungroupComponents** (bool notify=false)
Ungroups the current selection when it contains a valid group item.
- void **ungroupModelComponents** (QGraphicsItemGroup *group)
Ungroups one group item and restores its child components.
- void **notifyGraphicalModelChange** (GraphicalModelEvent::EventType eventType, GraphicalModelEvent::EventObjectType eventObjectType, QGraphicsItem *item)
Emits a scene-level change notification used by higher GUI layers.
- QList< GraphicalModelComponent * > **graphicalModelComponentItems** ()
Returns the list of graphical component items currently in the scene.
- GraphicalModelComponent * **findGraphicalModelComponent** (Util::identification id)
Finds one graphical component by kernel identification.
- GraphicalModelDataDefinition * **findGraphicalModelDataDefinition** (ModelDataDefinition *dataDefinition)
Finds one graphical data definition by kernel pointer.
- GraphicalModelComponent * **resolveSourceComponent** (GraphicalConnection *connection) const
Resolves the source component associated with one graphical connection.
- GraphicalModelComponent * **resolveDestinationComponent** (GraphicalConnection *connection) const
Resolves the destination component associated with one graphical connection.
- GRID * **grid** ()
Returns the current grid state owned by the scene.

- void **showGrid** ()
Shows the grid using the current grid configuration.
- void **setGridVisible** (bool visible)
Sets grid visibility without toggling the state implicitly.
- bool **isGridVisible** () const
Returns whether the grid is currently visible.
- void **snapItemsToGrid** ()
Snaps movable items to the active grid spacing.
- void **actualizeDiagramArrows** ()
Recomputes the direction arrows used by diagram connections.
- void **showDiagrams** ()
Shows all diagram-layer items.
- void **hideDiagrams** ()
Hides all diagram-layer items.
- bool **existDiagram** ()
Returns whether diagram-layer items exist.
- bool **visibleDiagram** ()
Returns whether diagram-layer items are currently visible.
- void **destroyDiagram** ()
Destroys the diagram layer and its cached items.
- void **createDiagrams** ()
Creates the diagram layer and its cached items.
- QUndoStack * **getUndoStack** ()
Returns the undo stack used by the scene.
- [Simulator](#) * **getSimulator** ()
Returns the simulator bound to this scene.
- void **setUndoStack** (QUndoStack *undo)
Replaces the undo stack used by the scene.
- void **beginConnection** ()
Starts a new component connection gesture.
- void **setSimulator** ([Simulator](#) *simulator)
Sets the simulator dependency used by scene logic.
- void **setPropertyEditor** ([PropertyEditorGenesys](#) *propEditor)
Sets the property editor dependency used by scene logic.
- void **setPropertyList** (std::map< [SimulationControl](#) *, [DataComponentProperty](#) * > *propList)
Sets the property list dependency used by scene logic.
- void **setPropertyEditorUI** (std::map< [SimulationControl](#) *, [DataComponentEditor](#) * > *propEditorUI)
Sets the property-editor widget map used by scene logic.
- void **setComboBox** (std::map< [SimulationControl](#) *, [ComboBoxEnum](#) * > *propCombo)
Sets the enum combo-box map used by scene logic.
- void **setObjectBeingDragged** (QTreeWidgetItem *objectBeingDragged)
Records which tree item is currently being dragged into the scene.
- void **setRestoringPersistedGuiLayout** (bool restoring)
Marks whether the scene is restoring persisted GUI layout.
- bool **isRestoringPersistedGuiLayout** () const
Returns whether the scene is restoring persisted GUI layout.
- void **setParentWidget** (QWidget *parentWidget)
Sets the parent widget used by transient UI interactions.
- unsigned short **connectingStep** () const
Returns the current step in the connection gesture.
- void **setConnectingStep** (unsigned short [connectingStep](#))

- Updates the current step in the connection gesture.*
- void **setSnapToGrid** (bool activated)
 - Enables or disables grid snapping.*
- bool **getSnapToGrid** ()
 - Returns whether grid snapping is enabled.*
- void **arranjeModels** (int direction)
 - Rearranges model items using the specified layout direction.*
- void **setDrawingMode** (DrawingMode drawingMode)
 - Sets the current drawing mode used by mouse gestures.*
- void **clearDrawingMode** ()
 - Clears the current drawing mode.*
- void **setGraphicalComponentPort** (GraphicalComponentPort *in)
 - Sets the current port used by connection gestures.*
- DrawingMode **getDrawingMode** ()
 - Returns the current drawing mode.*
- void **setAction** (QAction *action)
 - Sets the QAction that tracks the current scene tool.*
- QList< GraphicalModelComponent * > * **getAllComponents** ()
 - Returns all component items owned by the scene.*
- QList< GraphicalConnection * > * **getAllConnections** ()
 - Returns all connection items owned by the scene.*
- QList< GraphicalModelDataDefinition * > * **getAllDataDefinitions** ()
 - Returns all data-definition items owned by the scene.*
- QList< GraphicalDiagramConnection * > * **getAllGraphicalDiagramsConnections** ()
 - Returns all diagram-connection items owned by the scene.*
- QMap< QGraphicsItemGroup *, QList< GraphicalModelComponent * > > **getListComponentsGroup** ()
 - Returns the mapping used to restore grouped components.*
- void **insertComponentGroup** (QGraphicsItemGroup *group, QList< GraphicalModelComponent * > componentsGroup)
 - Stores the component list associated with one graphical group.*
- void **insertOldPositionItem** (QGraphicsItem *item, QPointF position)
 - Stores the previous scene position for one item.*
- QPointF **getOldPositionItem** (QGraphicsItem *item) const
 - Returns the previous scene position stored for one item.*
- QList< AnimationTransition * > * **getAnimationsTransition** ()
 - Returns the active transition animations tracked by the scene.*
- QList< AnimationCounter * > * **getAnimationsCounter** ()
 - Returns the active counter animations tracked by the scene.*
- QList< AnimationVariable * > * **getAnimationsVariable** ()
 - Returns the active variable animations tracked by the scene.*
- QList< AnimationTimer * > * **getAnimationsTimer** ()
 - Returns the active timer animations tracked by the scene.*
- QList< AnimationPlaceholder * > * **getAnimationsPlaceholder** ()
 - Returns the placeholder animation items tracked by the scene.*
- QMap< Event *, QList< AnimationTransition * > > * **getAnimationPaused** ()
 - Returns paused animations indexed by their simulation event.*
- bool **checkIgnoreEvent** ()
 - Returns true when the scene should suppress animation-driven event handling.*
- void **clearAnimations** ()
 - Clears all animation categories tracked by the scene.*
- void **clearAnimationsTransition** ()

- Clears transition animations tracked by the scene.*

 - void **clearAnimationsCounter** ()
- Clears counter animations tracked by the scene.*

 - void **clearAnimationsVariable** ()
- Clears variable animations tracked by the scene.*

 - void **clearAnimationsTimer** ()
- Clears timer animations tracked by the scene.*

 - void **clearAnimationsPlaceholder** ()
- Clears placeholder animations tracked by the scene.*

 - void **clearAnimationsQueue** ()
- Clears queue animation overlays tracked by the scene.*

 - void **animatePlot** ()
- Refreshes plot animation overlays for the current scene state.*

 - QList< QString > * **getImagesAnimation** ()
- Returns the list of animation image identifiers used by the scene.*

 - void **drawingCounter** ()
- Creates counter overlay geometry.*

 - void **drawingVariable** ()
- Creates variable overlay geometry.*

 - void **drawingTimer** ()
- Creates timer overlay geometry.*

 - void **drawingAttribute** ()
- Creates attribute overlay geometry.*

 - void **drawingEntity** ()
- Creates entity overlay geometry.*

 - void **drawingEvent** ()
- Creates event overlay geometry.*

 - void **drawingExpression** ()
- Creates expression overlay geometry.*

 - void **drawingPlot** ()
- Creates plot overlay geometry.*

 - void **drawingQueue** ()
- Creates queue overlay geometry.*

 - void **drawingResource** ()
- Creates resource overlay geometry.*

 - void **drawingStation** ()
- Creates station overlay geometry.*

 - void **drawingStatistics** ()
- Creates statistics overlay geometry.*

 - void **clearAnimationsValues** ()
- Clears animation-specific cached values.*

 - void **setCounters** ()
- Synchronizes the component counters used by animation overlays.*

 - void **setVariables** ()
- Synchronizes the variable counters used by animation overlays.*

 - void **insertRestoredDataDefinitions** (bool loaded)
- Restores data definitions removed by previous delete/check cycles.*

 - void **saveDataDefinitions** ()
- Caches data definitions before destructive scene operations.*

 - void **requestGraphicalDataDefinitionsSync** ()
- Requests an immediate data-definition synchronization pass.*

- void **scheduleGraphicalDataDefinitionsSync** ()
Schedules a deferred data-definition synchronization pass.
- bool **isGraphicalDataDefinitionsSyncInProgress** () const
Returns whether a data-definition sync is currently in progress.
- void **setShowStatisticsDataDefinitions** (bool show)
Shows or hides statistics-related data definitions.
- bool **showStatisticsDataDefinitions** () const
Returns whether statistics-related data definitions are visible.
- void **setShowEditableDataDefinitions** (bool show)
Shows or hides editable data definitions.
- bool **showEditableDataDefinitions** () const
Returns whether editable data definitions are visible.
- void **setShowSharedDataDefinitions** (bool show)
Shows or hides shared data definitions.
- bool **showSharedDataDefinitions** () const
Returns whether shared data definitions are visible.
- void **setShowRecursiveDataDefinitions** (bool show)
Shows or hides recursive data-definition expansion.
- bool **showRecursiveDataDefinitions** () const
Returns whether recursive data-definition expansion is enabled.
- void **setModelLevelFilter** (unsigned int modelLevel)
Restricts graphical rebuild/synchronization services to one kernel model level.
- void **clearModelLevelFilter** ()
Clears any level restriction and lets rebuild services render all model levels.
- bool **hasModelLevelFilter** () const
Returns true when this scene is scoped to a specific [ModelDataDefinition](#) level.
- unsigned int **modelLevelFilter** () const
Returns the active level used by rebuild services when the filter is enabled.
- void **setShowInternalDataDefinitions** (bool show)
Shows or hides internal data definitions.
- bool **showInternalDataDefinitions** () const
Returns whether internal data definitions are visible.
- void **setShowAttachedDataDefinitions** (bool show)
Shows or hides attached data definitions.
- bool **showAttachedDataDefinitions** () const
Returns whether attached data definitions are visible.
- void **setConnectionGeometryUpdatesBlocked** (bool blocked)
Temporarily blocks connection-geometry updates.
- bool **areConnectionGeometryUpdatesBlocked** () const
Returns whether connection-geometry updates are blocked.
- QList< QGraphicsItem * > * **getGraphicalModelDataDefinitions** () const
Returns the scene items classified as model data definitions.
- QList< QGraphicsItem * > * **getGraphicalModelComponents** () const
Returns the scene items classified as model components.
- QList< QGraphicsItem * > * **getGraphicalConnections** () const
Returns the scene items classified as connections.
- QList< QGraphicsItem * > * **getGraphicalGeometries** () const
Returns the scene items classified as geometry drawings.
- QList< QGraphicsItem * > * **getGraphicalAnimations** () const
Returns the scene items classified as animations.
- QList< QGraphicsItem * > * **getGraphicalEntities** () const

- Returns the scene items classified as entities.*
- `QList< QGraphicsItem * > * getGraphicalDiagramsConnections () const`
Returns the scene items classified as diagram connections.
- `QList< QGraphicsItemGroup * > * getGraphicalGroups () const`
Returns the groups currently tracked by the scene.
- `void animateTransition (ModelComponent *source, ModelComponent *destination, bool viewSimulation, Event *event)`
Starts a transition animation between two model components.
- `void runAnimateTransition (AnimationTransition *animationTransition, Event *event, bool restart=false)`
Runs one transition animation instance until completion.
- `void handleAnimationStateChanged (QAbstractAnimation::State newState, QEventLoop *loop, Event *event, AnimationTransition *animationTransition)`
Handles animation state changes for transition-driven event loops.
- `void animateQueueInsert (ModelComponent *component, bool visivable)`
Creates the queue-insert animation overlay for one component.
- `void animateQueueRemove (ModelComponent *component)`
Creates the queue-remove animation overlay for one component.
- `void animateCounter ()`
Updates the counter animation overlay.
- `void animateVariable (Entity *entity=nullptr)`
Updates the variable animation overlay.
- `void animateTimer (double time)`
Updates the timer animation overlay.

Protected Member Functions

- `virtual void contextMenuEvent (QGraphicsSceneContextMenuEvent *contextMenuEvent)`
- `virtual void dragEnterEvent (QGraphicsSceneDragDropEvent *event)`
- `virtual void dragLeaveEvent (QGraphicsSceneDragDropEvent *event)`
- `virtual void dragMoveEvent (QGraphicsSceneDragDropEvent *event)`
- `virtual void dropEvent (QGraphicsSceneDragDropEvent *event)`
- `virtual void focusInEvent (QFocusEvent *focusEvent)`
- `virtual void focusOutEvent (QFocusEvent *focusEvent)`
- `virtual void keyPressEvent (QKeyEvent *keyEvent)`
- `virtual void keyReleaseEvent (QKeyEvent *keyEvent)`
- `virtual void mouseDoubleClickEvent (QGraphicsSceneMouseEvent *mouseEvent)`
- `virtual void mouseMoveEvent (QGraphicsSceneMouseEvent *mouseEvent)`
- `virtual void mousePressEvent (QGraphicsSceneMouseEvent *mouseEvent)`
- `virtual void mouseReleaseEvent (QGraphicsSceneMouseEvent *mouseEvent)`
- `virtual void wheelEvent (QGraphicsSceneWheelEvent *wheelEvent)`

15.436.1 Detailed Description

Scene that owns and manipulates the graphical representation of a model.

The scene manages components, ports, connections, drawings, animations, and the auxiliary data-definition structures used by the GUI. It is the central editing surface for the graphical model and the main consumer of the view/controller services documented elsewhere.

15.436.2 Member Function Documentation

15.436.2.1 notifyGraphicalModelChange()

```
void ModelGraphicsScene::notifyGraphicalModelChange (
    GraphicalModelEvent::EventType eventType,
    GraphicalModelEvent::EventObjectType eventObjectType,
    QGraphicsItem * item )
```

Emits a scene-level change notification used by higher GUI layers.

Notifies view-level handlers about changes in graphical model.

[Event](#) object is stack-allocated and forwarded by const-reference to avoid heap ownership ambiguity.

Todo Consider broadcasting to all attached views when multi-view editing is supported.

15.436.2.2 setModelLevelFilter()

```
void ModelGraphicsScene::setModelLevelFilter (
    unsigned int modelLevel )
```

Restricts graphical rebuild/synchronization services to one kernel model level.

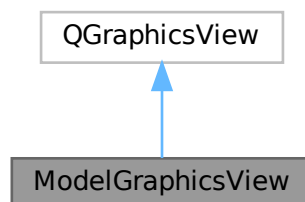
[ModelGraphicsScene](#) keeps this rendering context because some synchronization entry points are static services called from the property editor. Storing the active level in the scene lets those calls preserve the same root/submodel scope as the visible tab.

The documentation for this class was generated from the following files:

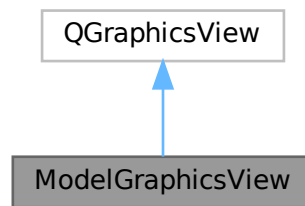
- applications/gui/genesys/graphics/ModelGraphicsScene.h
- applications/gui/genesys/graphics/ModelGraphicsScene.cpp

15.437 ModelGraphicsView Class Reference

Inheritance diagram for ModelGraphicsView:



Collaboration diagram for ModelGraphicsView:



Public Member Functions

- [ModelGraphicsView](#) (QWidget *parent=nullptr)
View wrapper that hosts the model graphics scene and delegates GUI events.
- **ModelGraphicsView** (const [ModelGraphicsView](#) &orig)
Copy constructor used by legacy compatibility paths.
- virtual ~**ModelGraphicsView** ()
Releases the Qt view and its compatibility state.
- [ModelGraphicsScene](#) * **getScene** ()
Returns the scene currently owned by the view.
- void **showGrid** ()
Requests grid visibility from the underlying scene.
- void **clear** ()
Clears the current scene contents and compatibility state.
- void **beginConnection** ()
Starts the connection-creation workflow in the scene.
- void **selectModelComponent** ([ModelComponent](#) *component)
Highlights the requested component in the scene and centers it when possible.
- void **setSimulator** ([Simulator](#) *simulator)
Injects the simulator dependency used by scene callbacks.
- void **setPropertyEditor** ([PropertyEditorGenesys](#) *propEditor)
Injects the property editor dependency used by scene callbacks.
- void **setPropertyList** (std::map< [SimulationControl](#) *, [DataComponentProperty](#) * > *propList)
Injects the property list map used by scene/editor synchronization.
- void **setPropertyEditorUI** (std::map< [SimulationControl](#) *, [DataComponentEditor](#) * > *propEditorUI)
Injects the property-editor widget map used by scene/editor synchronization.
- void **setComboBox** (std::map< [SimulationControl](#) *, [ComboBoxEnum](#) * > *propBox)
Injects the enum combo-box map used by scene/editor synchronization.
- void **setEnabled** (bool enabled)
Enables or disables interaction while keeping the scene wiring intact.
- QList< QGraphicsItem * > **selectedItems** ()
Returns the current selected items from the underlying scene.
- template<typename Class >
void **setSceneMouseEventHandler** (Class *object, void(Class::*function)(QGraphicsSceneMouseEvent *))
- template<typename Class >
void **setGraphicalModelEventHandler** (Class *object, void(Class::*function)(const [GraphicalModelEvent](#) &))

- `template<typename Class >`
`void setSceneWheelInEventHandler (Class *object, void(Class::*function)())`
- `template<typename Class >`
`void setSceneWheelOutEventHandler (Class *object, void(Class::*function)())`
- `template<typename Class >`
`void setContextMenuEventHandler (Class *object, void(Class::*function)(QContextMenuEvent *))`
- `void notifySceneMouseEventHandler (QGraphicsSceneMouseEvent *mouseEvent)`
Notifies the registered mouse-event handler, if any.
- `void notifySceneWheelInEventHandler ()`
Notifies the registered wheel-in handler, if any.
- `void notifySceneWheelOutEventHandler ()`
Notifies the registered wheel-out handler, if any.
- `void notifySceneGraphicalModelEventHandler (const GraphicalModelEvent &modelGraphicsEvent)`
Notifies the registered graphical-model-event handler, if any.
- `void setCanNotifyGraphicalModelEventHandlers (bool can)`
Enables or disables graphical-model event emission from the view.
- `void clearEventHandlers ()`
Clears all custom event handlers installed by the composition root.
- `void setParentWidget (QWidget *parentWidget)`
Sets the widget used as parent for dialogs and transient controls.
- `void setRuleVisible (bool visible)`
Enables or disables ruler rendering over the graphics viewport.
- `bool isRuleVisible () const`
Returns whether ruler rendering is active.
- `void setGuidesVisible (bool visible)`
Enables or disables guide line rendering over the graphics viewport.
- `bool isGuidesVisible () const`
Returns whether guide line rendering is active.

Protected Member Functions

- `void changed (const QList< QRectF > ®ion)`
- `void focusItemChanged (QGraphicsItem *newFocusItem, QGraphicsItem *oldFocusItem, Qt::FocusReason reason)`
- `void sceneRectChanged (const QRectF &rect)`
- `void selectionChanged ()`
- `virtual void contextMenuEvent (QContextMenuEvent *event) override`
- `virtual void dragEnterEvent (QDragEnterEvent *event) override`
- `virtual void keyPressEvent (QKeyEvent *event) override`
- `virtual void keyReleaseEvent (QKeyEvent *event) override`
- `virtual void wheelEvent (QWheelEvent *event) override`
- `virtual void drawForeground (QPainter *painter, const QRectF &rect) override`

15.437.1 Constructor & Destructor Documentation

15.437.1.1 ModelGraphicsView()

```
ModelGraphicsView::ModelGraphicsView (
    QWidget * parent = nullptr )
```

View wrapper that hosts the model graphics scene and delegates GUI events.

The view keeps the scene-specific input handlers outside the raw Qt widget layer so the [MainWindow](#) composition root can wire context menus, wheel behavior, and selection changes through injected callbacks.

15.437.2 Member Function Documentation

15.437.2.1 notifySceneGraphicalModelEventHandler()

```
void ModelGraphicsView::notifySceneGraphicalModelEventHandler (
    const GraphicalModelEvent & modelGraphicsEvent )
```

Notifies the registered graphical-model-event handler, if any.
Dispatches graphical model change event to registered callback.

Parameters

<i>modelGraphicsEvent</i>	Event payload from scene.
---------------------------	---------------------------

Todo Replace callback with signal/slot to improve composability.

Todo actualize property editor?

The documentation for this class was generated from the following files:

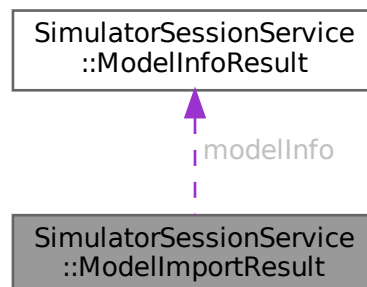
- applications/gui/genesys/graphicals/ModelGraphicsView.h
- applications/gui/genesys/graphicals/ModelGraphicsView.cpp

15.438 SimulatorSessionService::ModelImportResult Struct Reference

Contains model import output and error state.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Collaboration diagram for SimulatorSessionService::ModelImportResult:



Public Attributes

- bool **success** = false
- [ModelImportError](#) **error** = ModelImportError::None
- [ModelInfoResult](#) **modelInfo**

15.438.1 Detailed Description

Contains model import output and error state.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

15.439 ModelInfo Class Reference

```
#include <kernel/simulator/model/ModelInfo.h>
```

Public Member Functions

- `std::string show ()`
- `void setName (std::string _name)`
setName
- `std::string getName () const`
getName
- `void setAnalystName (std::string _analystName)`
setAnalystName
- `std::string getAnalystName () const`
getAnalystName
- `void setDescription (std::string _description)`
setDescription
- `std::string getDescription () const`
getDescription
- `void setProjectTitle (std::string _projectTitle)`
setProjectTitle
- `std::string getProjectTitle () const`
getProjectTitle
- `void setVersion (std::string _version)`
setVersion
- `std::string getVersion () const`
getVersion
- `void loadInstance (PersistenceRecord *fields)`
loadInstance
- `void saveInstance (PersistenceRecord *fields)`
saveInstance
- `bool hasChanged () const`
hasChanged
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for model metadata.

15.439.1 Detailed Description

[ModelInfo](#) stores basic model project information, such as the project and analyst names.

15.439.2 Member Function Documentation

15.439.2.1 getAnalystName()

```
std::string ModelInfo::getAnalystName ( ) const
getAnalystName
```

Returns

15.439.2.2 getDescription()

```
std::string ModelInfo::getDescription ( ) const  
getDescription
```

Returns

15.439.2.3 getName()

```
std::string ModelInfo::getName ( ) const  
getName
```

Returns

15.439.2.4 getProjectTitle()

```
std::string ModelInfo::getProjectTitle ( ) const  
getProjectTitle
```

Returns

15.439.2.5 getVersion()

```
std::string ModelInfo::getVersion ( ) const  
getVersion
```

Returns

15.439.2.6 hasChanged()

```
bool ModelInfo::hasChanged ( ) const  
hasChanged
```

Returns

15.439.2.7 loadInstance()

```
void ModelInfo::loadInstance (   
    PersistenceRecord * fields )  
loadInstance
```

Parameters

<i>fields</i>	
---------------	--

15.439.2.8 saveInstance()

```
void ModelInfo::saveInstance (   
    PersistenceRecord * fields )
```

saveInstance

Parameters

<i>fields</i>	
---------------	--

15.439.2.9 setAnalystName()

```
void ModelInfo::setAnalystName (
    std::string _analystName )
```

setAnalystName

Parameters

<i>_analystName</i>	
---------------------	--

15.439.2.10 setDescription()

```
void ModelInfo::setDescription (
    std::string _description )
```

setDescription

Parameters

<i>_description</i>	
---------------------	--

15.439.2.11 setName()

```
void ModelInfo::setName (
    std::string _name )
```

setName

Parameters

<i>_name</i>	
--------------	--

15.439.2.12 setProjectTitle()

```
void ModelInfo::setProjectTitle (
    std::string _projectTitle )
```

setProjectTitle

Parameters

<i>_projectTitle</i>	
----------------------	--

15.439.2.13 setVersion()

```
void ModelInfo::setVersion (
    std::string _version )
```

setVersion

Parameters

<code>_version</code>	
-----------------------	--

The documentation for this class was generated from the following files:

- kernel/simulator/model/ModelInfo.h
- kernel/simulator/model/ModelInfo.cpp

15.440 SimulatorSessionService::ModelInfoResult Struct Reference

Describes the currently active model information for a session.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Public Attributes

- bool **exists** = false
- unsigned int **modelId** = 0
- bool **hasChanged** = false
- unsigned int **level** = 0
- std::string **name**
- std::string **analystName**
- std::string **projectTitle**
- std::string **version**
- std::string **description**
- unsigned int **componentCount** = 0

15.440.1 Detailed Description

Describes the currently active model information for a session.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

15.441 ModelInspectorController Class Reference

Controller for model inspection views and scene-selection synchronization.

```
#include <applications/gui/genesys/controllers/ModelInspectorController.h>
```

Classes

- struct [DataDefinitionSelection](#)

Public Member Functions

- **ModelInspectorController** ([Simulator](#) *simulator, QTreeWidget *componentsTree, QTreeWidget *data↔DefinitionsTree, [ModelGraphicsView](#) *graphicsView)
Creates the model-inspector delegated controller.
- void **actualizeModelComponents** (bool force) const
Synchronizes the Components tree from the current model representation.
- void **actualizeModelDataDefinitions** (bool force) const
Synchronizes the Data Definitions tree from current model data definitions.
- void **beginDataDefinitionNameEdit** (QTreeWidgetItem *item, int column) const
Starts in-place rename editing for editable data-definition entries.
- void **applyDataDefinitionNameChange** (QTreeWidgetItem *item, int column) const
Commits edited data-definition names back to the kernel model.

- void **syncSelectedComponentTreeItemToScene** () const
Synchronizes tree selection with scene selection and viewport focus.
- [DataDefinitionSelection](#) **syncSelectedDataDefinitionTreeItemToScene** () const
Synchronizes Data Definitions tree selection with visible GMDDs, when any.

15.441.1 Detailed Description

Controller for model inspection views and scene-selection synchronization.

In the refactored GUI architecture, this controller receives delegated callbacks from [MainWindow](#) to keep model-inspector responsibilities outside the composition root while preserving the existing UI contract. It synchronizes Components/Data Definitions trees with the current kernel model and keeps tree-to-scene selection behavior consistent.

Responsibilities:

- refresh model component and data-definition trees from [Simulator](#) state;
- manage in-place rename flow for editable data-definition names;
- propagate current tree selection to the graphical scene and viewport.

Boundaries:

- it does not create/destroy models or files (model lifecycle remains elsewhere);
- it does not own scene command logic, undo/redo, or simulation commands;
- it does not act as a persistence or export service.

The documentation for this class was generated from the following files:

- applications/gui/genesys/controllers/ModelInspectorController.h
- applications/gui/genesys/controllers/ModelInspectorController.cpp

15.442 ModelLanguageSynchronizer Class Reference

Service layer that synchronizes textual model language with kernel model state.

```
#include <applications/gui/genesys/services/ModelLanguageSynchronizer.h>
```

Public Member Functions

- [ModelLanguageSynchronizer](#) ([Simulator](#) *simulator, QPlainTextEdit *modelTextEditor, bool *textModelHas↔ ChangedFlag, QWidget *ownerWidget, std::function< void()> onModelCreatedOrLoaded)
Creates the synchronization service used by [MainWindow](#) compatibility wrappers.
- void **actualizeModelSimLanguage** () const
Rebuilds textual model representation from the current kernel model.
- bool **setSimulationModelBasedOnText** () const
Parses text editor content and applies it to the simulator model.

15.442.1 Detailed Description

Service layer that synchronizes textual model language with kernel model state.

This service was extracted from [MainWindow](#) representation code to isolate text/model synchronization concerns. [MainWindow](#) remains the composition root and delegates wrapper methods to this class.

Responsibilities:

- generate/update the textual model representation from current [Simulator](#) model;
- parse/apply textual edits back into simulator model state;
- notify [MainWindow](#) via callback when model creation/loading side effects are required.

Boundaries:

- it does not own simulator or editor widgets;
- it does not handle graphical persistence, Graphviz export, or C++ code export;
- it is a synchronization service, not a controller.

15.442.2 Constructor & Destructor Documentation

15.442.2.1 ModelLanguageSynchronizer()

```
ModelLanguageSynchronizer::ModelLanguageSynchronizer (
    Simulator * simulator,
    QPlainTextEdit * modelTextEditor,
    bool * textModelHasChangedFlag,
    QWidget * ownerWidget,
    std::function< void()> onModelCreatedOrLoaded )
```

Creates the synchronization service used by [MainWindow](#) compatibility wrappers.
Stores explicit dependencies once to keep wrapper calls thin.

15.442.3 Member Function Documentation

15.442.3.1 actualizeModelSimLanguage()

```
void ModelLanguageSynchronizer::actualizeModelSimLanguage ( ) const
```

Rebuilds textual model representation from the current kernel model.

Regenerates the text editor content from the current kernel model.

This is part of the synchronization bridge used after lifecycle, property, and scene updates.

15.442.3.2 setSimulationModelBasedOnText()

```
bool ModelLanguageSynchronizer::setSimulationModelBasedOnText ( ) const
```

Parses text editor content and applies it to the simulator model.

Applies the text editor content to the current kernel model.

Returns

true when synchronization succeeds and model state is valid for downstream workflows.

The documentation for this class was generated from the following files:

- applications/gui/genesys/services/ModelLanguageSynchronizer.h
- applications/gui/genesys/services/ModelLanguageSynchronizer.cpp

15.443 ModelLifecycleController Class Reference

Controller for model/application lifecycle flows delegated from [MainWindow](#).

```
#include <applications/gui/genesys/controllers/ModelLifecycleController.h>
```

Classes

- struct [Callbacks](#)

Public Member Functions

- **ModelLifecycleController** (QWidget *ownerWidget, [Simulator](#) *simulator, Ui::MainWindow *ui, QString *modelFilename, bool *textModelHasChanged, bool *graphicalModelHasChanged, bool *closingApproved, bool *loaded, bool ¶llelizationEnabled, int ¶llelizationThreads, int ¶llelizationBatchSize, [Callbacks](#) callbacks)

Creates the lifecycle orchestration controller used by [MainWindow](#) wrappers.

- void **onActionModelNewTriggered** () const
Starts the new-model lifecycle flow and reinitializes GUI/model state.
- void **onActionModelOpenTriggered** () const
Starts model-open flow with persistence loading and UI synchronization.
- bool **openModelFile** (const QString &fileName) const
Opens a model from an explicit file path using the same lifecycle flow as the open action.
- void **onActionModelSaveTriggered** () const
Starts model-save flow for graphical/text representations.
- void **onActionModelCloseTriggered** () const
Closes current model after pending-change checks and cleanup orchestration.
- void **onActionModelInformationTriggered** () const
Opens model information dialog for the current model context.
- void **onActionModelCheckTriggered** () const
Runs model-check workflow as a delegated lifecycle operation.
- void **onActionSimulationConfigureTriggered** () const
Opens simulation configuration workflow anchored to lifecycle façade.
- void **onActionSimulatorExitTriggered** () const
Executes application-exit flow with pending-change confirmation semantics.
- bool **hasPendingModelChanges** () const
Indicates whether current model/text state has unsaved changes.
- bool **confirmApplicationExit** () const
Requests exit confirmation preserving compatibility prompt behavior.

15.443.1 Detailed Description

Controller for model/application lifecycle flows delegated from [MainWindow](#).

This controller centralizes lifecycle actions (new/open/save/close/check/configure/exit) while [MainWindow](#) remains the composition root and compatibility façade. It uses explicit callback injection to invoke existing UI update and persistence routines without changing signatures or ownership.

Responsibilities:

- orchestrate user-triggered lifecycle actions and guard pending-change confirmations;
- coordinate model load/save paths through injected serializer callbacks;
- trigger UI/tab/action refreshes required after lifecycle transitions.

Boundaries:

- it does not directly implement file formats (delegated to services);
- it does not own simulator/model objects;
- it does not manage scene tools, property editor internals, or simulation traces.

The documentation for this class was generated from the following files:

- applications/gui/genesys/controllers/ModelLifecycleController.h
- applications/gui/genesys/controllers/ModelLifecycleController.cpp

15.444 ModelManager Class Reference

The [ModelManager](#) class.

```
#include <kernel/simulator/model/ModelManager.h>
```

Public Member Functions

- [ModelManager](#) ([Simulator](#) *simulator)
Creates a model manager bound to a simulator instance.
- virtual [~ModelManager](#) ()
Releases the model manager and any owned bookkeeping structures.
- [Model](#) * [newModel](#) ()
Creates a new model, inserts it into the open-model list and makes it current.
- void [insert](#) ([Model](#) *model)
Inserts an existing model into the open-model list.
- void [remove](#) ([Model](#) *model)
Removes a model from the open-model list.
- bool [setCurrent](#) ([Model](#) *model)
Makes an open model the current one.
- bool [saveModel](#) (std::string filename)
Saves the current model to a file.
- [Model](#) * [loadModel](#) (std::string filename)
Loads a model from a file and opens it in the manager.
- [Model](#) * [createFromLanguage](#) (std::string modelSpecification)
Creates a model from a textual model specification.
- unsigned int [size](#) ()
Returns the number of open models.
- std::vector< [Model](#) * > [models](#) () const
Returns a stable snapshot of the currently open models in tab/navigation order.
- bool [hasModel](#) ([Model](#) *model) const
Checks whether a model is owned by this manager.
- int [indexOf](#) ([Model](#) *model) const
Returns the open-model index of a model.
- [Model](#) * [modelAt](#) (unsigned int index) const
Returns the model at a given open-model index.
- int [currentIndex](#) () const
Returns the index of the current model.
- [Model](#) * [front](#) ()
Returns the first open model.
- [Model](#) * [last](#) ()
Returns the last open model.
- [Model](#) * [current](#) ()
Returns the current model.
- [Model](#) * [next](#) ()
Advances to the next open model.
- [Model](#) * [previous](#) ()
Moves to the previous open model.
- bool [canGoNext](#) () const
Indicates whether a next model exists.
- bool [canGoPrevious](#) () const
Indicates whether a previous model exists.

15.444.1 Detailed Description

The [ModelManager](#) class.

15.444.2 Constructor & Destructor Documentation

15.444.2.1 ModelManager()

```
ModelManager::ModelManager (
    Simulator * simulator )
```

Creates a model manager bound to a simulator instance.

Parameters

<i>simulator</i>	Parent simulator that owns this manager.
------------------	--

15.444.3 Member Function Documentation

15.444.3.1 canGoNext()

```
bool ModelManager::canGoNext ( ) const
```

Indicates whether a next model exists.

Returns

true when a call to `next()` can advance.

15.444.3.2 canGoPrevious()

```
bool ModelManager::canGoPrevious ( ) const
```

Indicates whether a previous model exists.

Returns

true when a call to `previous()` can retreat.

15.444.3.3 createFromLanguage()

```
Model * ModelManager::createFromLanguage (
    std::string modelSpecification )
```

Creates a model from a textual model specification.

Parameters

<i>modelSpecification</i>	Source text in the model language.
---------------------------	------------------------------------

Returns

Newly created model instance, or `nullptr` on failure.

15.444.3.4 current()

```
Model * ModelManager::current ( )
```

Returns the current model.

Returns

Currently selected model, or `nullptr` when none is selected.

15.444.3.5 currentIndex()

```
int ModelManager::currentIndex ( ) const
```

Returns the index of the current model.

Returns

Zero-based current model index, or -1 when there is no current model.

15.444.3.6 front()

```
Model * ModelManager::front ( )
```

Returns the first open model.

Returns

First model in open-model order, or `nullptr` when empty.

15.444.3.7 hasModel()

```
bool ModelManager::hasModel (
    Model * model ) const
```

Checks whether a model is owned by this manager.

Parameters

<i>model</i>	Model pointer to test.
--------------	------------------------

Returns

True when model is in the open-model list.

15.444.3.8 indexOf()

```
int ModelManager::indexOf (
    Model * model ) const
```

Returns the open-model index of a model.

Parameters

<i>model</i>	Model pointer to locate.
--------------	--------------------------

Returns

Zero-based index, or -1 when not open.

15.444.3.9 insert()

```
void ModelManager::insert (
    Model * model )
```

Inserts an existing model into the open-model list.

Parameters

<i>model</i>	Model to register.
--------------	--------------------

15.444.3.10 last()

```
Model * ModelManager::last ( )
```

Returns the last open model.

Returns

Last model in open-model order, or `nullptr` when empty.

15.444.3.11 loadModel()

```
Model * ModelManager::loadModel (
    std::string filename )
```

Loads a model from a file and opens it in the manager.

Parameters

<i>filename</i>	Input filename.
-----------------	-----------------

Returns

Loaded model instance, or `nullptr` on failure.

15.444.3.12 modelAt()

```
Model * ModelManager::modelAt (
    unsigned int index ) const
```

Returns the model at a given open-model index.

Parameters

<i>index</i>	Zero-based model index.
--------------	-------------------------

Returns

`Model` pointer, or `nullptr` when index is out of range.

15.444.3.13 models()

```
std::vector< Model * > ModelManager::models ( ) const
```

Returns a stable snapshot of the currently open models in tab/navigation order.

Returns

Vector with the open model pointers owned by this manager.

15.444.3.14 newModel()

```
Model * ModelManager::newModel ( )
```

Creates a new model, inserts it into the open-model list and makes it current.

Returns

The newly opened current model.

15.444.3.15 next()

```
Model * ModelManager::next ( )
```

Advances to the next open model.

Returns

Next model in order, or `nullptr` when already at the end.

15.444.3.16 previous()

```
Model * ModelManager::previous ( )
```

Moves to the previous open model.

Returns

Previous model in order, or `nullptr` when already at the beginning.

15.444.3.17 remove()

```
void ModelManager::remove (
    Model * model )
```

Removes a model from the open-model list.

Parameters

<i>model</i>	Model to remove.
--------------	------------------

15.444.3.18 saveModel()

```
bool ModelManager::saveModel (
    std::string filename )
```

Saves the current model to a file.

Parameters

<i>filename</i>	Output filename.
-----------------	------------------

Returns

`true` when saving succeeds.

15.444.3.19 setCurrent()

```
bool ModelManager::setCurrent (
    Model * model )
```

Makes an open model the current one.

Parameters

<i>model</i>	Model to activate.
--------------	--------------------

Returns

`true` when the model was found and selected.

15.444.3.20 size()

```
unsigned int ModelManager::size ( )
```

Returns the number of open models.

Returns

Open-model count.

The documentation for this class was generated from the following files:

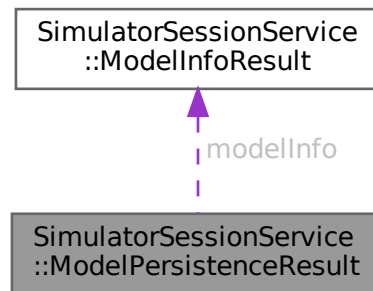
- kernel/simulator/model/ModelManager.h
- kernel/simulator/model/ModelManager.cpp

15.445 SimulatorSessionService::ModelPersistenceResult Struct Reference

Contains save/load model operation output and error state.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Collaboration diagram for SimulatorSessionService::ModelPersistenceResult:



Public Attributes

- bool **success** = false
- [PersistenceError](#) **error** = PersistenceError::None
- std::string **filename**
- [ModelInfoResult](#) **modelInfo**

15.445.1 Detailed Description

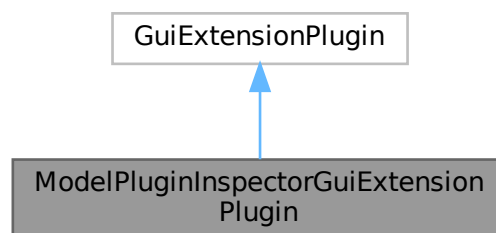
Contains save/load model operation output and error state.

The documentation for this struct was generated from the following file:

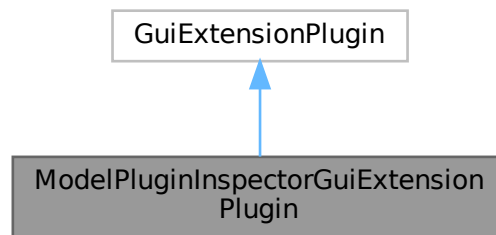
- applications/worker/service/SimulatorSessionService.h

15.446 ModelPluginInspectorGuiExtensionPlugin Class Reference

Inheritance diagram for ModelPluginInspectorGuiExtensionPlugin:



Collaboration diagram for ModelPluginInspectorGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId () const` override
- `std::string displayName () const` override
- `std::string version () const` override
- `std::vector< std::string > requiredModelPlugins () const` override
- `void registerContributions (GuiExtensionRegistry *registry) const` override

15.446.1 Member Function Documentation

15.446.1.1 displayName()

```
std::string ModelPluginInspectorGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.446.1.2 extensionId()

```
std::string ModelPluginInspectorGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.446.1.3 registerContributions()

```
void ModelPluginInspectorGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.446.1.4 requiredModelPlugins()

```
std::vector< std::string > ModelPluginInspectorGuiExtensionPlugin::requiredModelPlugins ( )
const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

15.446.1.5 version()

```
std::string ModelPluginInspectorGuiExtensionPlugin::version ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

- `applications/gui/genesys/extensions/ModelPluginInspectorGuiExtensionPlugin.cpp`

15.447 ModelSimulation Class Reference

Controls the mature, currently functional simulation-execution layer of a model.

```
#include <kernel/simulator/model/ModelSimulation.h>
```

Public Member Functions

- **ModelSimulation** ([Model](#) *model)
- `std::string show ()`
- `void start ()`
Starts a sequential execution of a simulation, ie, a set of replications of this model.
- `void pause ()`
- `void step ()`
Executes the processing of a single event, the next one in the future events list.
- `void stop ()`
- `void setNumberOfReplications (unsigned int _numberOfReplications)`
- `unsigned int getNumberOfReplications () const`
- `void setReplicationLength (double _replicationLength, Util::TimeUnit _replicationLengthTimeUnit=Util::TimeUnit::unknown)`
- `double getReplicationLength () const`
- `void setReplicationLengthTimeUnit (Util::TimeUnit _replicationLengthTimeUnit)`
- `Util::TimeUnit getReplicationLengthTimeUnit () const`
- `void setReplicationReportBaseTimeUnit (Util::TimeUnit _replicationReportBaseTimeUnit)`
- `Util::TimeUnit getReplicationBaseTimeUnit () const`
- `void setWarmUpPeriod (double warmUpPeriod, Util::TimeUnit warmUpPeriodTimeUnit=Util::TimeUnit::unknown)`
- `double getWarmUpPeriod () const`
- `void setWarmUpPeriodTimeUnit (Util::TimeUnit _warmUpPeriodTimeUnit)`
- `Util::TimeUnit getWarmUpPeriodTimeUnit () const`
- `void setTerminatingCondition (std::string _terminatingCondition)`
- `std::string getTerminatingCondition () const`
- `void setPauseOnEvent (bool _pauseOnEvent)`
- `bool isPauseOnEvent () const`
- `void setStepByStep (bool _stepByStep)`
- `bool isStepByStep () const`
- `void setInitializeStatistics (bool _initializeStatistics)`
- `bool isInitializeStatistics () const`
- `void setInitializeSystem (bool _initializeSystem)`
- `bool isInitializeSystem () const`
- `void setPauseOnReplication (bool _pauseBetweenReplications)`
- `bool isPauseOnReplication () const`
- `void setReporter (SimulationReporter\_if *_simulationReporter)`
- `SimulationReporter\_if * getReporter () const`
- `double getSimulatedTime () const`
- `bool isRunning () const`
- `bool isPaused () const`
- `unsigned int getCurrentReplicationNumber () const`
- `void setShowReportsAfterReplication (bool showReportsAfterReplication)`
- `bool isShowReportsAfterReplication () const`
- `void setShowReportsAfterSimulation (bool showReportsAfterSimulation)`
- `bool isShowReportsAfterSimulation () const`
- `List< double > * getBreakpointsOnTime () const`
- `List< Entity * > * getBreakpointsOnEntity () const`
- `List< ModelComponent * > * getBreakpointsOnComponent () const`
- `const List< ModelDataDefinition * > * getSimulationStatisticsAggregates () const`

- void **loadInstance** ([PersistenceRecord](#) *fields)
- void **saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaults)
- [Event](#) * **getCurrentEvent** () const
- void **setShowSimulationResponsesInReport** (bool _showSimulationResponsesInReport)
- bool **isShowSimulationResponsesInReport** () const
- void **setShowSimulationControlsInReport** (bool _showSimulationControlsInReport)
- bool **isShowSimulationControlsInReport** () const

Friends

- class **Model**

15.447.1 Detailed Description

Controls the mature, currently functional simulation-execution layer of a model.

The [ModelSimulation](#) controls the simulation of a model, allowing to start, pause, resume e stop a simulation, composed by a set of replications.

Historically, this class has been the original and operational experiment mechanism in GenESyS. It already supports the core execution workflow based on replications, replication length, warm-up configuration and event processing. Later experiment-oriented classes such as [SimulationExperiment](#) and [SimulationScenario](#) were introduced as an attempt to move richer design-of-experiments concerns into the kernel. Those newer classes are still under development, while [ModelSimulation](#) remains the stable execution core.

15.447.2 Member Function Documentation

15.447.2.1 isRunning()

```
bool ModelSimulation::isRunning ( ) const
```

The current time in the model being simulated, i.e., the instant when the current event was triggered

15.447.2.2 start()

```
void ModelSimulation::start ( )
```

Starts a sequential execution of a simulation, ie, a set of replications of this model.

Checks the model and if ok then initialize the simulation, execute repeatedly each replication and then show simulation statistics

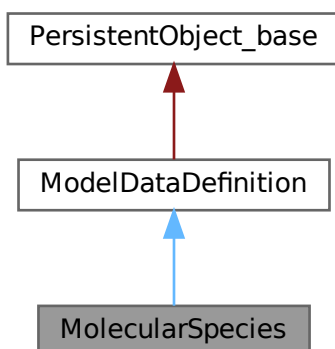
The documentation for this class was generated from the following files:

- kernel/simulator/model/ModelSimulation.h
- kernel/simulator/model/ModelSimulation.cpp

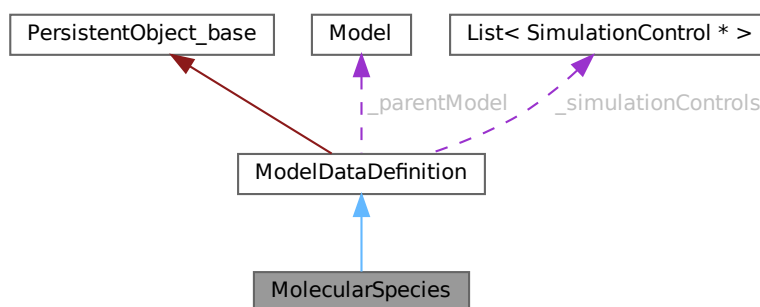
15.448 MolecularSpecies Class Reference

```
#include <plugins/data/WholeCellModeling/MolecularSpecies.h>
```


Inheritance diagram for MolecularSpecies:



Collaboration diagram for MolecularSpecies:



Public Member Functions

- **MolecularSpecies** ([Model](#) *model, std::string name="")
- void **setInitialCount** (int initialCount)
- int **getInitialCount** () const
- void **setCount** (int count)
- int **getCount** () const
- void **setCompartment** (std::string compartment)
- std::string **getCompartment** () const
- void **setUnit** (std::string unit)
- std::string **getUnit** () const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

- Creates a model data definition and optionally inserts it into the model.*

 - virtual **~ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- **List**< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- **TraceManager::Level** **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (**TraceManager::Level** level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (**TraceManager::Level** traceLevelspecific, bool traceLevelSpecific↔ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_initBetweenReplications** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool **_getSaveDefaultsOption** ()
- virtual [ParserChangesInformation](#) * **_getParserChangesInformation** ()
- virtual void **_addSimulationControl** ([SimulationControl](#) *control)
- virtual void **_createInternalAndAttachedData** ()
- virtual void **_createInternalStatisticReporters** ()
- virtual void **_createAttachedAttributes** ()
- virtual void **_createNonEditableDataDefinitions** ()
- virtual void **_createEditableDataDefinitions** ()
- void **_templateCreateInternalStatisticReporters** ()
- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
legacy helper methods used by the granular create hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)

- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.448.1 Detailed Description

Discrete molecular species with integer copy count for Gillespie SSA.

Unlike [BioSpecies](#) (continuous double amounts for ODE), [MolecularSpecies](#) tracks integer molecule counts. Compartment and unit annotate the biological context without affecting simulation logic.

15.448.2 Member Function Documentation

15.448.2.1 _check()

```
bool MolecularSpecies::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.448.2.2 _initBetweenReplications()

```
void MolecularSpecies::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.448.2.3 _loadInstance()

```
bool MolecularSpecies::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.448.2.4 _saveInstance()

```
void MolecularSpecies::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.448.2.5 show()

```
std::string MolecularSpecies::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- plugins/data/WholeCellModeling/MolecularSpecies.h
- plugins/data/WholeCellModeling/MolecularSpecies.cpp

15.449 GroProgramRuntime::MotionMutation Struct Reference

Public Attributes

- MotionMutationType **type** = MotionMutationType::Run
- double **value** = 0.0
- double **previousDirection** = 0.0
- double **resultingDirection** = 0.0

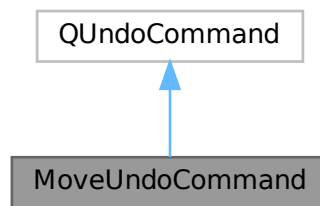
- double **previousSpeed** = 0.0
- double **resultingSpeed** = 0.0

The documentation for this struct was generated from the following file:

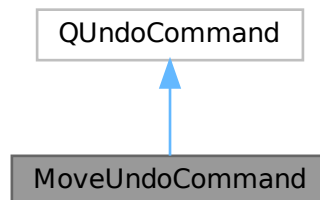
- plugins/data/BiochemicalSimulation/GroProgramRuntime.h

15.450 MoveUndoCommand Class Reference

Inheritance diagram for MoveUndoCommand:



Collaboration diagram for MoveUndoCommand:



Public Member Functions

- **MoveUndoCommand** (QList< QGraphicsItem * > item, [ModelGraphicsScene](#) *scene, QList< QPointF > &oldPos, QList< QPointF > &newPos, QUndoCommand *parent=nullptr)
- void **undo** () override
- void **redo** () override

The documentation for this class was generated from the following files:

- applications/gui/genesys/actions/MoveUndoCommand.h
- applications/gui/genesys/actions/MoveUndoCommand.cpp

15.451 testing::internal::Mutex Class Reference

Public Member Functions

- void **Lock** ()

- void **Unlock** ()
- void **AssertHeld** () const

The documentation for this class was generated from the following file:

- gtest/gtest/internal/gtest-port.h

15.452 dialogBreakpoint::MVCResult Class Reference

Public Member Functions

- **MVCResult** (std::string type, std::string on)

Public Attributes

- std::string **type**
- std::string **on**

The documentation for this class was generated from the following file:

- applications/gui/genesys/dialogs/dialogBreakpoint.h

15.453 GroProgramAst::NamedProgram Struct Reference

Public Attributes

- std::string **name** = ""
- std::vector< std::string > **parameters**
- std::string **bodySource** = ""
- std::vector< [Statement](#) > **statements**

The documentation for this struct was generated from the following file:

- plugins/data/BiochemicalSimulation/GroProgramAst.h

15.454 GroProgramIr::NamedProgramDefinition Struct Reference

Public Attributes

- std::vector< std::string > **parameters**
- std::string **sourceText** = ""
- std::vector< [Command](#) > **commands**

The documentation for this struct was generated from the following file:

- plugins/data/BiochemicalSimulation/GroProgramIr.h

15.455 testing::internal::NameGeneratorSelector< Provided > Struct Template Reference

Public Types

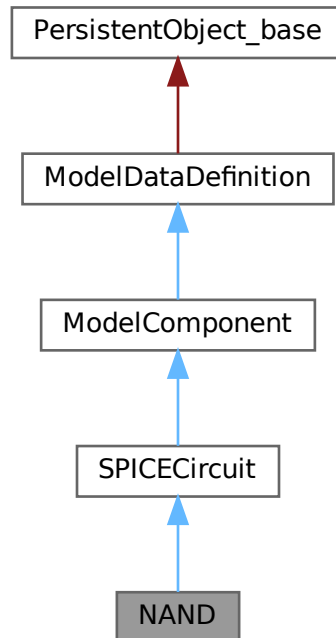
- typedef Provided **type**

The documentation for this struct was generated from the following file:

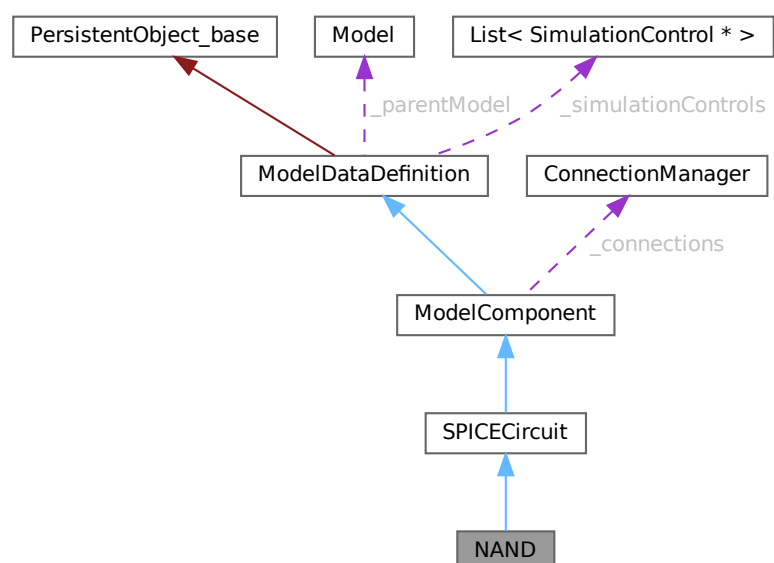
- gtest/gtest/internal/gtest-internal.h

15.456 NAND Class Reference

Inheritance diagram for NAND:



Collaboration diagram for NAND:



Public Member Functions

- **NAND** ([Model](#) *model, std::string name="")
- void **Build** () override
- void **setPMOSWidth** (float PMOSWidth)
- void **setPMOSLength** (float length)
- void **setNMOSWidth** (float NMOSWidth)
- void **setNMOSLength** (float NMOSLength)
- void **setElectricalModel** (std::string electricalModel)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")

constructors

- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")

constructors

- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

- Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation](#) * [GetPluginInformation](#) ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
- static std::vector< std::string > [split](#) (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from **ModelComponent**

- static **ModelComponent** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** (**ModelComponent** *component)
Validates a component instance.
- static void **CreateInternalData** (**ModelComponent** *component)
Creates internal data required by a component instance.
- static void **SaveInstance** (**PersistenceRecord** *fields, **ModelComponent** *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** (**Event** *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from **ModelDataDefinition**

- static **ModelDataDefinition** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** (**PersistenceRecord** *fields, **ModelDataDefinition** *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** (**ModelDataDefinition** *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** (**ModelDataDefinition** *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** (**ModelDataDefinition** *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** (**ModelDataDefinition** *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** (**PersistenceRecord** *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** (**PersistenceRecord** *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from **SPICECircuit**

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void **_onDispatchEvent** (**Entity** *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void **_createNonEditableDataDefinitions** () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
 - could be overridden by derived classes*
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
 - legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
 - method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.456.1 Member Function Documentation

15.456.1.1 [_loadInstance\(\)](#)

```
bool NAND::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [SPICECircuit](#).

15.456.1.2 [_saveInstance\(\)](#)

```
void NAND::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

15.456.1.3 [Build\(\)](#)

```
void NAND::Build ( ) [override], [virtual]
```

Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

- plugins/components/ElectronicsSimulation/SPICECircuit.h
- plugins/components/ElectronicsSimulation/SPICECircuit.cpp

15.457 testing::internal::NativeArray< Element > Class Template Reference

Public Types

- typedef Element **value_type**
- typedef Element * **iterator**
- typedef const Element * **const_iterator**

Public Member Functions

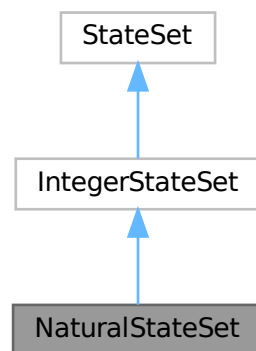
- **NativeArray** (const Element *array, size_t count, [RelationToSourceReference](#))
- **NativeArray** (const Element *array, size_t count, [RelationToSourceCopy](#))
- **NativeArray** (const [NativeArray](#) &rhs)
- size_t **size** () const
- const_iterator **begin** () const
- const_iterator **end** () const
- bool **operator==** (const [NativeArray](#) &rhs) const

The documentation for this class was generated from the following file:

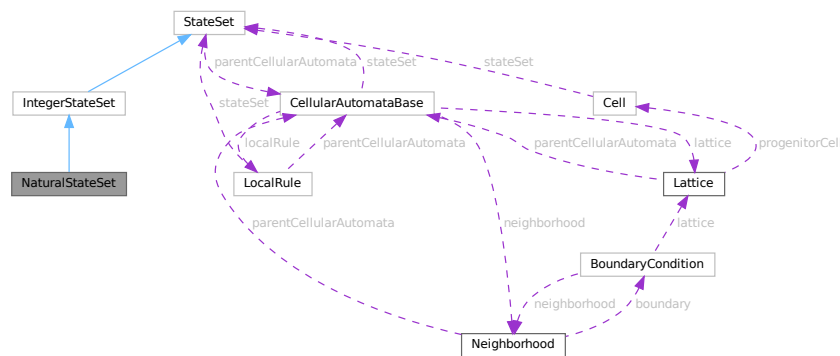
- gtest/gtest/internal/gtest-internal.h

15.458 NaturalStateSet Class Reference

Inheritance diagram for NaturalStateSet:



Collaboration diagram for NaturalStateSet:



Public Member Functions

- **NaturalStateSet** ([CellularAutomataBase](#) *parentCellularAutomata, std::optional< long > maximum=std::nullopt)
- std::string [name](#) () const override

Public Member Functions inherited from [IntegerStateSet](#)

- **IntegerStateSet** ([CellularAutomataBase](#) *parentCellularAutomata, std::optional< long > minimum=std::nullopt, std::optional< long > maximum=std::nullopt)
- bool [contains](#) (const [State](#) &state) const override
- std::unique_ptr< [State](#) > [createDefaultState](#) () const override
- std::unique_ptr< [State](#) > [parseState](#) (const std::string &text) const override
- bool [isFinite](#) () const override
- bool [isDiscrete](#) () const override
- std::vector< std::unique_ptr< [State](#) > > [enumerateStates](#) () const override

Public Member Functions inherited from [StateSet](#)

- **StateSet** ([CellularAutomataBase](#) *parentCellularAutomata)
- **StateSet** (const [StateSet](#) &orig)
- virtual bool [isContinuous](#) () const
- virtual bool [isComposite](#) () const

Additional Inherited Members

Protected Attributes inherited from [StateSet](#)

- [CellularAutomataBase](#) * [parentCellularAutomata](#)

15.458.1 Member Function Documentation

15.458.1.1 [name\(\)](#)

std::string NaturalStateSet::name () const [inline], [override], [virtual]

Reimplemented from [IntegerStateSet](#).

The documentation for this class was generated from the following file:

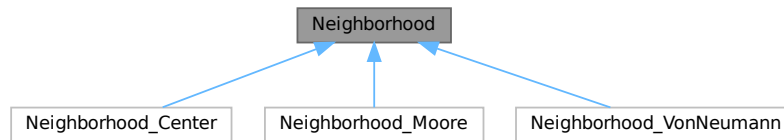
- plugins/components/ModalModel/CellularAutomata/StateSet.h

15.459 Neighborhood Class Reference

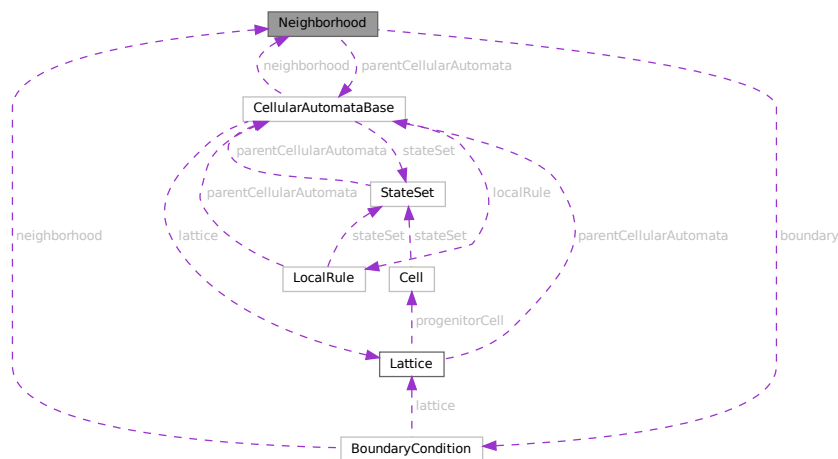
Defines the complete neighborhood of a cell.

```
#include <plugins/components/ModalModel/CellularAutomata/Neighborhood.h>
```

Inheritance diagram for Neighborhood:



Collaboration diagram for Neighborhood:



Public Member Functions

- **Neighborhood** ([CellularAutomataBase](#) *parentCellularAutomata, unsigned short radius=1, [BoundaryCondition](#) *boundary=nullptr, bool includeCellItself=false)
- **Neighborhood** (const [Neighborhood](#) &orig)
- virtual std::string **show** ()
- virtual std::vector< [Cell](#) * > **getNeighbors** ([Cell](#) *cell)=0
pure virtual method that has to be overridden by derived class, responsible for defining the neighborhood itself.
- void **setRadius** (unsigned short radius)
- unsigned short **getRadius** () const
- void **setBoundary** ([BoundaryCondition](#) *boundary)
- [BoundaryCondition](#) * **getBoundary** () const
- void **setIncludeCellItself** (bool includeCellItself)
- bool **getIncludeCellItself** () const

Protected Member Functions

- [Cell](#) * **getNeighborCell** (std::vector< int > cellPosition, std::vector< std::pair< unsigned short, int > > dimensionChanges)

Protected Attributes

- `std::string` **name**
- [CellularAutomataBase](#) * **parentCellularAutomata**
- [BoundaryCondition](#) * **boundary**
- unsigned short **radius**
- bool **includeCellItself**

15.459.1 Detailed Description

Defines the complete neighborhood of a cell.

15.459.2 Member Function Documentation

15.459.2.1 `getNeighbors()`

```
virtual std::vector< Cell * > Neighborhood::getNeighbors (
    Cell * cell ) [pure virtual]
```

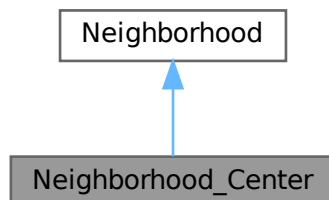
pure virtual method that has to be overridden by derived class, responsible for defining the neighborhood itself. Implemented in [Neighborhood_Center](#), [Neighborhood_Moore](#), and [Neighborhood_VonNeumann](#).

The documentation for this class was generated from the following file:

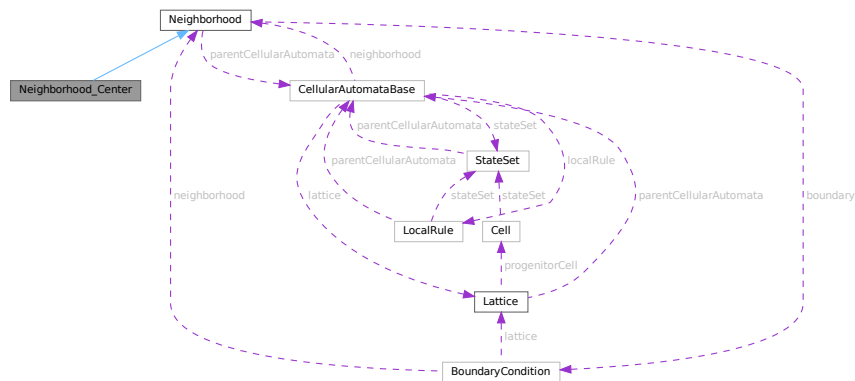
- `plugins/components/ModalModel/CellularAutomata/Neighborhood.h`

15.460 Neighborhood_Center Class Reference

Inheritance diagram for Neighborhood_Center:



Collaboration diagram for `Neighborhood_Center`:



Public Member Functions

- **Neighborhood_Center** ([CellularAutomataBase](#) *parentCellularAutomata, unsigned short radius=1, [BoundaryCondition](#) *boundary=nullptr, bool includeCellItself=false)
- **Neighborhood_Center** (const [Neighborhood_Center](#) &orig)
- virtual std::vector< [Cell](#) * > **getNeighbors** ([Cell](#) *cell) override
pure virtual method that has to be overridden by derived class, responsible for defining the neighborhood itself.

Public Member Functions inherited from [Neighborhood](#)

- **Neighborhood** ([CellularAutomataBase](#) *parentCellularAutomata, unsigned short radius=1, [BoundaryCondition](#) *boundary=nullptr, bool includeCellItself=false)
- **Neighborhood** (const [Neighborhood](#) &orig)
- virtual std::string **show** ()
- void **setRadius** (unsigned short radius)
- unsigned short **getRadius** () const
- void **setBoundary** ([BoundaryCondition](#) *boundary)
- [BoundaryCondition](#) * **getBoundary** () const
- void **setIncludeCellItself** (bool includeCellItself)
- bool **getIncludeCellItself** () const

Additional Inherited Members

Protected Member Functions inherited from [Neighborhood](#)

- [Cell](#) * **getNeighborCell** (std::vector< int > cellPosition, std::vector< std::pair< unsigned short, int > > dimensionChanges)

Protected Attributes inherited from [Neighborhood](#)

- std::string **name**
- [CellularAutomataBase](#) * **parentCellularAutomata**
- [BoundaryCondition](#) * **boundary**
- unsigned short **radius**
- bool **includeCellItself**

15.460.1 Member Function Documentation

15.460.1.1 getNeighbors()

```
virtual std::vector< Cell * > Neighborhood_Center::getNeighbors (
    Cell * cell ) [inline], [override], [virtual]
```

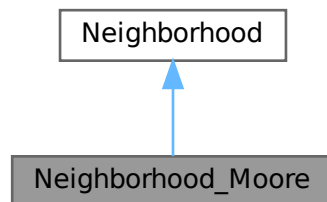
pure virtual method that has to be overridden by derived class, responsible for defining the neighborhood itself. Implements [Neighborhood](#).

The documentation for this class was generated from the following file:

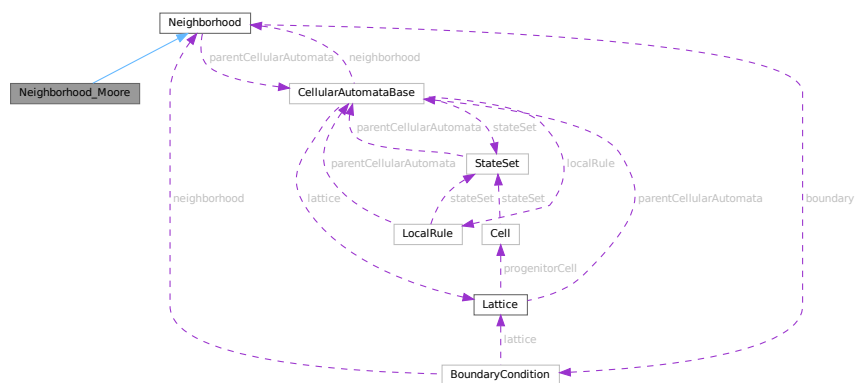
- plugins/components/ModalModel/CellularAutomata/Neighborhood_Center.h

15.461 Neighborhood_Moore Class Reference

Inheritance diagram for Neighborhood_Moore:



Collaboration diagram for Neighborhood_Moore:



Public Member Functions

- **Neighborhood_Moore** ([CellularAutomataBase](#) *parentCellularAutomata, unsigned short radius=1, [BoundaryCondition](#) *boundary=nullptr)
- **Neighborhood_Moore** (const [Neighborhood_Moore](#) &orig)
- virtual std::string [show](#) () override
- virtual std::vector< [Cell](#) * > [getNeighbors](#) ([Cell](#) *cell) override

pure virtual method that has to be overridden by derived class, responsible for defining the neighborhood itself.

Public Member Functions inherited from [Neighborhood](#)

- **Neighborhood** ([CellularAutomataBase](#) *parentCellularAutomata, unsigned short radius=1, [BoundaryCondition](#) *boundary=nullptr, bool includeCellItself=false)
- **Neighborhood** (const [Neighborhood](#) &orig)
- void **setRadius** (unsigned short radius)
- unsigned short **getRadius** () const
- void **setBoundary** ([BoundaryCondition](#) *boundary)
- [BoundaryCondition](#) * **getBoundary** () const
- void **setIncludeCellItself** (bool includeCellItself)
- bool **getIncludeCellItself** () const

Additional Inherited Members

Protected Member Functions inherited from [Neighborhood](#)

- [Cell](#) * **getNeighborCell** (std::vector< int > cellPosition, std::vector< std::pair< unsigned short, int > > dimensionChanges)

Protected Attributes inherited from [Neighborhood](#)

- std::string **name**
- [CellularAutomataBase](#) * **parentCellularAutomata**
- [BoundaryCondition](#) * **boundary**
- unsigned short **radius**
- bool **includeCellItself**

15.461.1 Member Function Documentation

15.461.1.1 getNeighbors()

```
virtual std::vector< Cell * > Neighborhood_Moore::getNeighbors (
    Cell * cell ) [inline], [override], [virtual]
```

pure virtual method that has to be overridden by derived class, responsible for defining the neighborhood itself. Implements [Neighborhood](#).

15.461.1.2 show()

```
virtual std::string Neighborhood_Moore::show ( ) [inline], [override], [virtual]
```

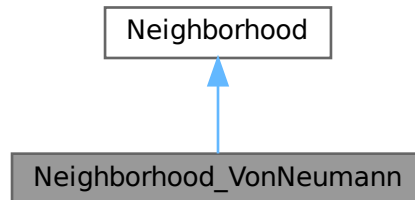
Reimplemented from [Neighborhood](#).

The documentation for this class was generated from the following file:

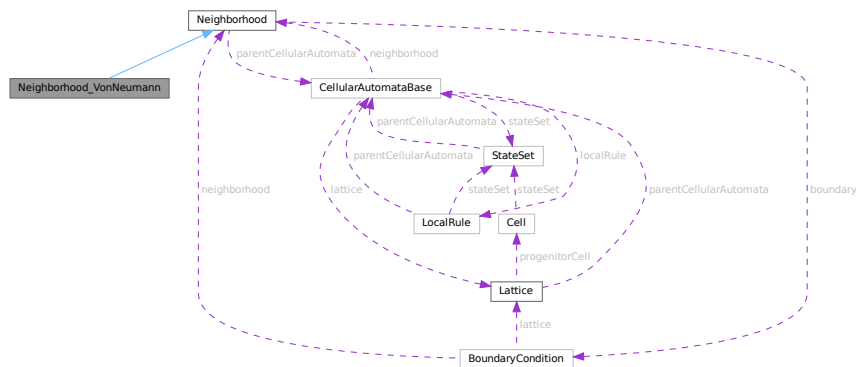
- plugins/components/ModalModel/CellularAutomata/Neighborhood_Moore.h

15.462 Neighborhood_VonNeumann Class Reference

Inheritance diagram for Neighborhood_VonNeumann:



Collaboration diagram for Neighborhood_VonNeumann:



Public Member Functions

- **Neighborhood_VonNeumann** ([CellularAutomataBase](#) *parentCellularAutomata, unsigned short radius=1, [BoundaryCondition](#) *boundary=nullptr)
 - **Neighborhood_VonNeumann** (const [Neighborhood_VonNeumann](#) &orig)
 - virtual std::string **show** () override
 - virtual std::vector< [Cell](#) * > **getNeighbors** ([Cell](#) *cell) override
- pure virtual method that has to be overridden by derived class, responsible for defining the neighborhood itself.*

Public Member Functions inherited from [Neighborhood](#)

- **Neighborhood** ([CellularAutomataBase](#) *parentCellularAutomata, unsigned short radius=1, [BoundaryCondition](#) *boundary=nullptr, bool includeCellItself=false)
- **Neighborhood** (const [Neighborhood](#) &orig)
- void **setRadius** (unsigned short radius)
- unsigned short **getRadius** () const
- void **setBoundary** ([BoundaryCondition](#) *boundary)
- [BoundaryCondition](#) * **getBoundary** () const
- void **setIncludeCellItself** (bool includeCellItself)
- bool **getIncludeCellItself** () const

Additional Inherited Members

Protected Member Functions inherited from [Neighborhood](#)

- [Cell](#) * **getNeighborCell** (std::vector< int > cellPosition, std::vector< std::pair< unsigned short, int > > dimensionChanges)

Protected Attributes inherited from [Neighborhood](#)

- std::string **name**
- [CellularAutomataBase](#) * **parentCellularAutomata**
- [BoundaryCondition](#) * **boundary**
- unsigned short **radius**
- bool **includeCellItself**

15.462.1 Member Function Documentation

15.462.1.1 getNeighbors()

```
virtual std::vector< Cell * > Neighborhood_VonNeumann::getNeighbors (
    Cell * cell ) [inline], [override], [virtual]
```

pure virtual method that has to be overridden by derived class, responsible for defining the neighborhood itself. Implements [Neighborhood](#).

15.462.1.2 show()

```
virtual std::string Neighborhood_VonNeumann::show ( ) [inline], [override], [virtual]
```

Reimplemented from [Neighborhood](#).

The documentation for this class was generated from the following file:

- plugins/components/ModalModel/CellularAutomata/Neighborhood_VonNeumann.h

15.463 ParserManager::NewParser Struct Reference

Paths associated with a generated parser artifact set.

```
#include <kernel/simulator/ParserManager.h>
```

Public Attributes

- std::string **bisonFilename**
- std::string **flexFilename**
- std::string **compiledParserFilename**

15.463.1 Detailed Description

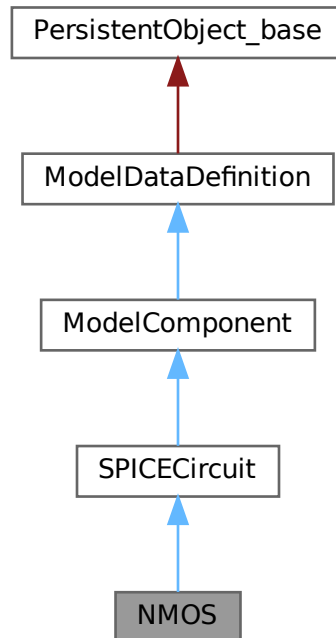
Paths associated with a generated parser artifact set.

The documentation for this struct was generated from the following file:

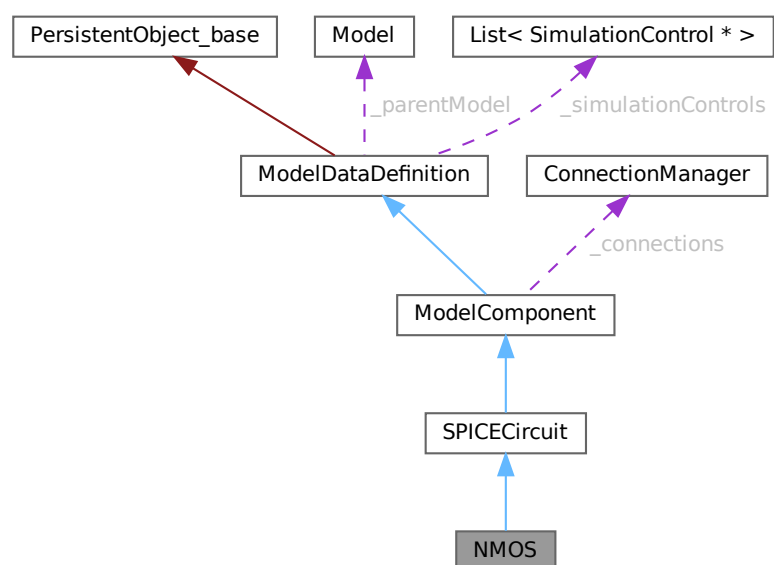
- kernel/simulator/ParserManager.h

15.464 NMOS Class Reference

Inheritance diagram for NMOS:



Collaboration diagram for NMOS:



Public Member Functions

- **NMOS** ([Model](#) *model, std::string name="")
- void **Build** () override
- void **setWidth** (float width)
- void **setLength** (float length)
- void **setElectricalModel** (std::string electricalModel)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentName, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionType, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

- Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation](#) * [GetPluginInformation](#) ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
- static std::vector< std::string > [split](#) (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from **ModelComponent**

- static **ModelComponent** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** (**ModelComponent** *component)
Validates a component instance.
- static void **CreateInternalData** (**ModelComponent** *component)
Creates internal data required by a component instance.
- static void **SaveInstance** (**PersistenceRecord** *fields, **ModelComponent** *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** (**Event** *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from **ModelDataDefinition**

- static **ModelDataDefinition** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** (**PersistenceRecord** *fields, **ModelDataDefinition** *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** (**ModelDataDefinition** *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** (**ModelDataDefinition** *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** (**ModelDataDefinition** *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** (**ModelDataDefinition** *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** (**PersistenceRecord** *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** (**PersistenceRecord** *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from **SPICECircuit**

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void **_onDispatchEvent** (**Entity** *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void **_createNonEditableDataDefinitions** () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
Traces text to the simulation output.
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.464.1 Member Function Documentation

15.464.1.1 [_loadInstance\(\)](#)

```
bool NMOS::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [SPICECircuit](#).

15.464.1.2 [_saveInstance\(\)](#)

```
void NMOS::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

15.464.1.3 [Build\(\)](#)

```
void NMOS::Build ( ) [override], [virtual]
```

Reimplemented from [SPICECircuit](#).
The documentation for this class was generated from the following files:

- plugins/components/ElectronicsSimulation/SPICECircuit.h
- plugins/components/ElectronicsSimulation/SPICECircuit.cpp

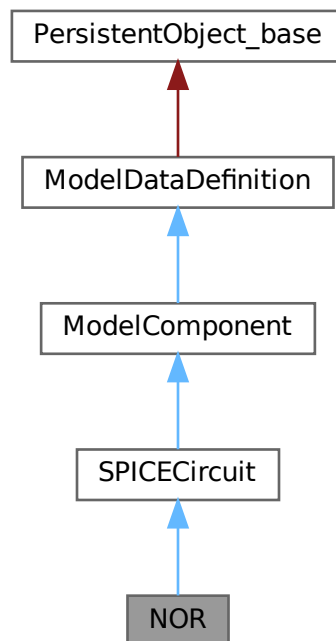
15.465 testing::internal::None Struct Reference

The documentation for this struct was generated from the following file:

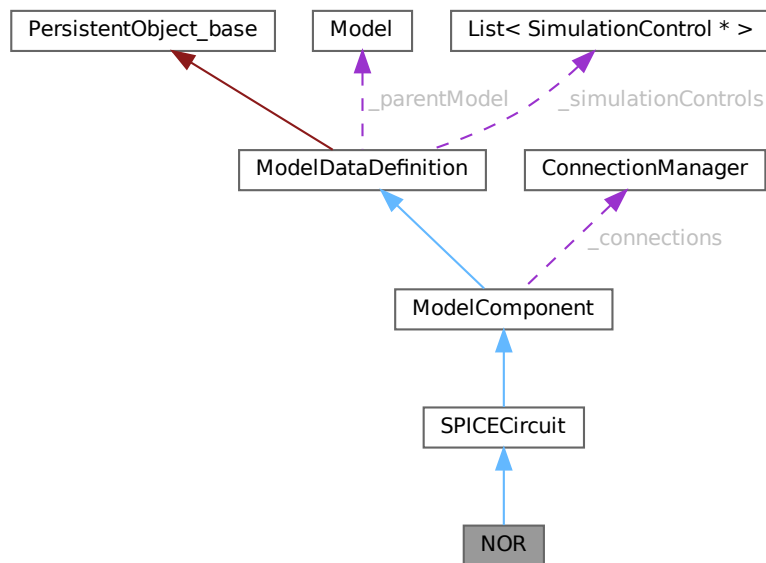
- gtest/gtest/internal/gtest-type-util.h

15.466 NOR Class Reference

Inheritance diagram for NOR:



Collaboration diagram for NOR:



Public Member Functions

- **NOR** ([Model](#) *model, std::string name="")
- void [Build](#) () override
- void **setPMOSWidth** (float PMOSWidth)
- void **setPMOSLength** (float length)
- void **setNMOSWidth** (float NMOSWidth)
- void **setNMOSLength** (float NMOSLength)
- void **setElectricalModel** (std::string electricalModel)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")

constructors

- void **setRunner** ([SPICERunner](#) *runner)

new public user methods for this components

- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")

constructors

- virtual **~ModelComponent** ()

Releases component-owned resources and unregisters it from the model.

- [ConnectionManager](#) * **getConnectionManager** () const

new public user methods for this component

- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)

- *Connects this component to another component.*
- bool `hasBreakpointAt` ()
Indicates whether a simulation breakpoint is set on this component.
- void `setDescription` (std::string _description)
Updates the free-form description shown by the GUI.
- std::string `getDescription` () const
Returns the free-form description associated with this component.
- void `remove` (ModelComponent *comp)
Removes a connected component from this component.

Public Member Functions inherited from `ModelDataDefinition`

- `ModelDataDefinition` (`Model` *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~`ModelDataDefinition` ()
Releases resources owned by this data definition.
- Util::identification `getId` () const
Returns the unique identifier of this instance.
- void `setName` (const std::string &name)
Updates the logical name of this instance.
- const std::string & `getName` () const
Returns the logical name of this instance.
- void `setLabel` (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & `getLabel` () const
Returns the user-visible label of this instance.
- const std::string & `getClassname` () const
Returns the runtime class name used by the plugin system.
- `Model` * `getParentModel` () const
Returns the parent model that owns this definition.
- bool `isReportStatistics` () const
Indicates whether this data definition contributes report statistics.
- void `setReportStatistics` (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition` * `getInternalData` (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, `ModelDataDefinition` * > * `getInternalData` () const
Returns the full map of internal child data definitions.
- std::map< std::string, `ModelDataDefinition` * > * `getAttachedData` () const
Returns the full map of attached data definitions.
- bool `hasChanged` () const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged` (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int `getLevel` () const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel` (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< `SimulationControl` * > * `getSimulationControls` () const
Returns the simulation controls owned by this data definition.

- `TraceManager::Level` `getTraceLevelSpecific ()` const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` level)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` traceLevelspecific, bool traceLevelSpecific↔ Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation` * `GetPluginInformation` ()
- static `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")

Static Public Member Functions inherited from `SPICECircuit`

- static `PluginInformation` * `GetPluginInformation` ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")
- static std::vector< std::string > `split` (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check` (`ModelComponent` *component)
Validates a component instance.
- static void `CreateInternalData` (`ModelComponent` *component)
Creates internal data required by a component instance.
- static void `SaveInstance` (`PersistenceRecord` *fields, `ModelComponent` *component)
Serializes a component instance into a persistence record.
- static void `DispatchEvent` (`Event` *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void `SaveInstance` (`PersistenceRecord` *fields, `ModelDataDefinition` *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool `Check` (`ModelDataDefinition` *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void `InitBetweenReplications` (`ModelDataDefinition` *modeldatum)

Resets simulation-time state between replications.

- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void [_createNonEditableDataDefinitions](#) () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)

- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.466.1 Member Function Documentation

15.466.1.1 `_loadInstance()`

```
bool NOR::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

15.466.1.2 `_saveInstance()`

```
void NOR::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

15.466.1.3 `Build()`

```
void NOR::Build ( ) [override], [virtual]
```

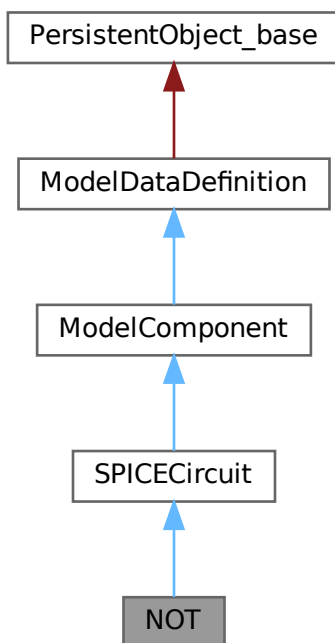
Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

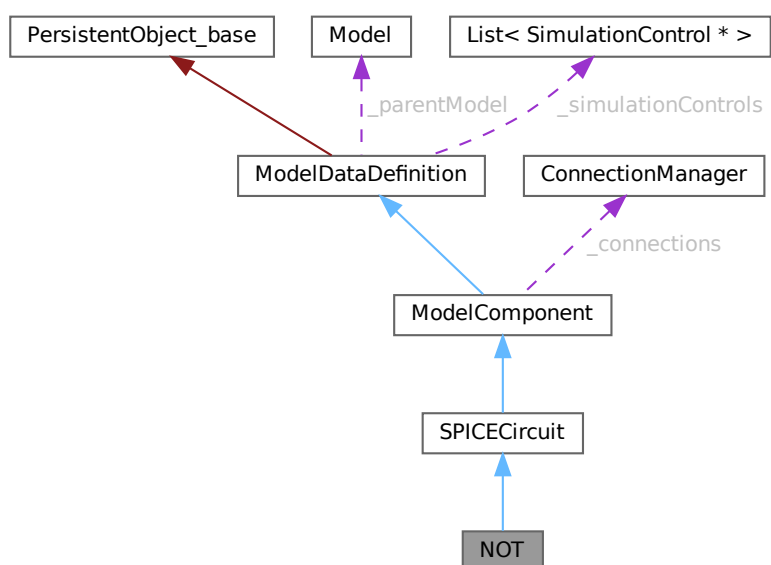
- `plugins/components/ElectronicsSimulation/SPICECircuit.h`
- `plugins/components/ElectronicsSimulation/SPICECircuit.cpp`

15.467 NOT Class Reference

Inheritance diagram for NOT:



Collaboration diagram for NOT:



Public Member Functions

- **NOT** ([Model](#) *model, std::string name="")
- void **Build** () override
- void **setPMOSWidth** (float PMOSWidth)
- void **setPMOSLength** (float length)
- void **setNMOSWidth** (float NMOSWidth)
- void **setNMOSLength** (float NMOSLength)
- void **setElectricalModel** (std::string electricalModel)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

- Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation](#) * [GetPluginInformation](#) ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
- static std::vector< std::string > [split](#) (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from **ModelComponent**

- static **ModelComponent** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** (**ModelComponent** *component)
Validates a component instance.
- static void **CreateInternalData** (**ModelComponent** *component)
Creates internal data required by a component instance.
- static void **SaveInstance** (**PersistenceRecord** *fields, **ModelComponent** *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** (**Event** *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from **ModelDataDefinition**

- static **ModelDataDefinition** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** (**PersistenceRecord** *fields, **ModelDataDefinition** *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** (**ModelDataDefinition** *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** (**ModelDataDefinition** *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** (**ModelDataDefinition** *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** (**ModelDataDefinition** *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** (**PersistenceRecord** *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** (**PersistenceRecord** *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from **SPICECircuit**

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void **_onDispatchEvent** (**Entity** *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void **_createNonEditableDataDefinitions** () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.467.1 Member Function Documentation

15.467.1.1 [_loadInstance\(\)](#)

```
bool NOT::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [SPICECircuit](#).

15.467.1.2 [_saveInstance\(\)](#)

```
void NOT::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

15.467.1.3 [Build\(\)](#)

```
void NOT::Build ( ) [override], [virtual]
```

Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

- plugins/components/ElectronicsSimulation/SPICECircuit.h
- plugins/components/ElectronicsSimulation/SPICECircuit.cpp

15.468 `obj_t` Class Reference

Public Member Functions

- `obj_t` (double v, std::string t)
- `obj_t` (double v, std::string t, unsigned long uid)

Public Attributes

- double **valor**
- std::string **tipo**
- unsigned long **id**
- bool **isReference**
- bool **isAttributeReference**

The documentation for this class was generated from the following files:

- parser/obj_t.h
- parser/obj_t.cpp

15.469 `Optimizer_if::ObjectiveDefinition` Struct Reference

Configurable objective definition.

```
#include <tools/Optimization/Optimizer_if.h>
```

Public Attributes

- std::string **name**
- [ObjectiveSense](#) **sense**
- std::string **expression**
- double **weight** = 1.0
- bool **enabled** = true

15.469.1 Detailed Description

Configurable objective definition.

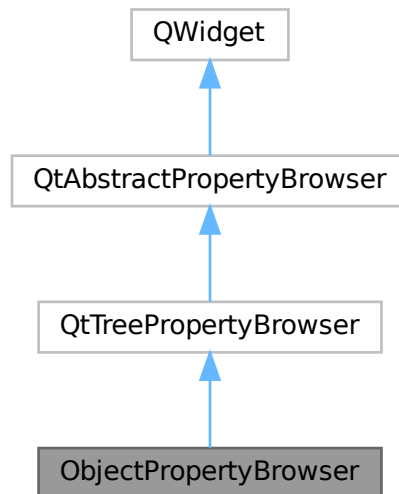
The expression is parsed by the model/parser context and can reference selected responses, controls, constants or user-defined expressions.

The documentation for this struct was generated from the following file:

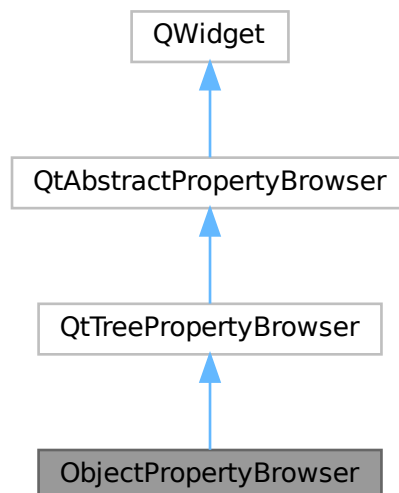
- tools/Optimization/Optimizer_if.h

15.470 ObjectPropertyBrowser Class Reference

Inheritance diagram for ObjectPropertyBrowser:



Collaboration diagram for ObjectPropertyBrowser:



Public Types

- using **ModelChangedCallback** = std::function< void()>

Public Types inherited from [QtTreePropertyBrowser](#)

- enum [ResizeMode](#) { [Interactive](#) , [Stretch](#) , [Fixed](#) , [ResizeToContents](#) }

Public Slots

- void **objectUpdated** ()

Public Slots inherited from [QtAbstractPropertyBrowser](#)

- [QtBrowserItem](#) * **addProperty** ([QtProperty](#) *property)
- [QtBrowserItem](#) * **insertProperty** ([QtProperty](#) *property, [QtProperty](#) *afterProperty)
- void **removeProperty** ([QtProperty](#) *property)

Public Member Functions

- **ObjectPropertyBrowser** (QWidget *parent=nullptr)
- void **setActiveObject** (QObject *obj, [ModelDataDefinition](#) *mdd=nullptr, const QSet< QString > &graphicallyRepresentedModelObjects={}, const QSet< QString > &editableModelObjects={}, [PropertyEditorGenesys](#) *peg=nullptr, std::map< [SimulationControl](#) *, [DataComponentProperty](#) * > *pl=nullptr, std::map< [SimulationControl](#) *, [DataComponentEditor](#) * > *peUI=nullptr, std::map< [SimulationControl](#) *, [ComboBoxEnum](#) * > *pb=nullptr)
- void **setActiveGraphicsItem** (QGraphicsItem *item)
- void **clearCurrentlyConnectedObject** ()
- void **setModelChangedCallback** (ModelChangedCallback callback)
- bool **isCommitPipelineBusy** () const

Public Member Functions inherited from [QtTreePropertyBrowser](#)

- [QtTreePropertyBrowser](#) (QWidget *parent=0)
- [~QtTreePropertyBrowser](#) ()
- int **indentation** () const
- void **setIndentation** (int i)
- bool **rootIsDecorated** () const
- void **setRootIsDecorated** (bool show)
- bool **alternatingRowColors** () const
- void **setAlternatingRowColors** (bool enable)
- bool **isHeaderVisible** () const
- void **setHeaderVisible** (bool visible)
- [ResizeMode](#) **resizeMode** () const
- void **setResizeMode** ([ResizeMode](#) mode)
- int **splitterPosition** () const
- void **setSplitterPosition** (int position)
- void **setExpanded** ([QtBrowserItem](#) *item, bool [expanded](#))
- bool **isExpanded** ([QtBrowserItem](#) *item) const
- bool **isItemVisible** ([QtBrowserItem](#) *item) const
- void **setItemVisible** ([QtBrowserItem](#) *item, bool visible)
- void **setBackgroundColor** ([QtBrowserItem](#) *item, const QColor &color)
- QColor **backgroundColor** ([QtBrowserItem](#) *item) const
- QColor **calculatedBackgroundColor** ([QtBrowserItem](#) *item) const
- void **setPropertiesWithoutValueMarked** (bool mark)
- bool **propertiesWithoutValueMarked** () const
- void **editItem** ([QtBrowserItem](#) *item)

Public Member Functions inherited from [QtAbstractPropertyBrowser](#)

- [QtAbstractPropertyBrowser](#) (QWidget *parent=0)
- [~QtAbstractPropertyBrowser](#) ()
- QList< [QtProperty](#) * > [properties](#) () const
- QList< [QtBrowserItem](#) * > [items](#) ([QtProperty](#) *property) const
- [QtBrowserItem](#) * [topLevelItem](#) ([QtProperty](#) *property) const
- QList< [QtBrowserItem](#) * > [topLevelItems](#) () const
- void [clear](#) ()
- template<class PropertyManager >
void [setFactoryForManager](#) (PropertyManager *manager, [QtAbstractEditorFactory](#)< PropertyManager > *factory)
- void [unsetFactoryForManager](#) ([QtAbstractPropertyManager](#) *manager)
- [QtBrowserItem](#) * [currentItem](#) () const
- void [setCurrentItem](#) ([QtBrowserItem](#) *)

Protected Member Functions

- void **keyPressEvent** (QKeyEvent *event) override
- void **contextMenuEvent** (QContextMenuEvent *event) override
- void **mouseDoubleClickEvent** (QMouseEvent *event) override

Protected Member Functions inherited from [QtTreePropertyBrowser](#)

- void [itemInserted](#) ([QtBrowserItem](#) *item, [QtBrowserItem](#) *afterItem) override
- void [itemRemoved](#) ([QtBrowserItem](#) *item) override
- void [itemChanged](#) ([QtBrowserItem](#) *item) override

Protected Member Functions inherited from [QtAbstractPropertyBrowser](#)

- virtual QWidget * [createEditor](#) ([QtProperty](#) *property, QWidget *parent)

Additional Inherited Members

Signals inherited from [QtTreePropertyBrowser](#)

- void [collapsed](#) ([QtBrowserItem](#) *item)
- void [expanded](#) ([QtBrowserItem](#) *item)

Signals inherited from [QtAbstractPropertyBrowser](#)

- void [currentItemChanged](#) ([QtBrowserItem](#) *)

Properties inherited from [QtTreePropertyBrowser](#)

- int **indentation**
indentation of the items in the tree view.
- bool **rootIsDecorated**
whether to show controls for expanding and collapsing root items.
- bool **alternatingRowColors**
whether to draw the background using alternating colors. By default this property is set to true.
- bool **headerVisible**
whether to show the header.
- [ResizeMode](#) **resizeMode**
the resize mode of sections in the header.
- int **splitterPosition**

the position of the splitter between the columns.

- bool `propertiesWithoutValueMarked`
whether to enable or disable marking properties without value.

The documentation for this class was generated from the following files:

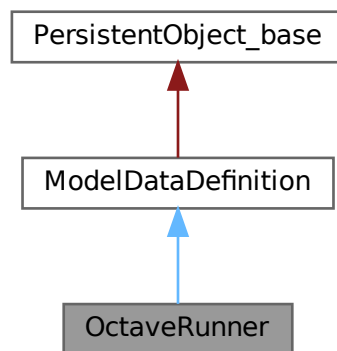
- applications/gui/genesys/propertyeditor/ObjectPropertyBrowser.h
- applications/gui/genesys/propertyeditor/ObjectPropertyBrowser.cpp

15.471 OctaveRunner Class Reference

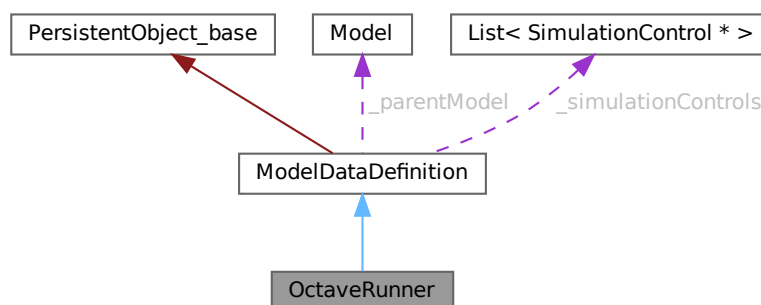
Model data definition that executes Octave code through Rscript.

```
#include <plugins/data/ExternalIntegration/OctaveRunner.h>
```

Inheritance diagram for OctaveRunner:



Collaboration diagram for OctaveRunner:



Public Member Functions

- **OctaveRunner** (**Model** *model, std::string name="")
Creates an Octave Runner bound to a model.
- virtual ~**OctaveRunner** ()=default

Destroys the Octave Runner.

- void **setOctaveExecutable** (std::string octaveExecutable)
Sets the Rscript executable name or path.
- std::string **getOctaveExecutable** () const
Returns the Rscript executable name or path.
- void **setWorkingDirectory** (std::string workingDirectory)
Sets the working directory used for generated scripts and command execution.
- std::string **getWorkingDirectory** () const
Returns the working directory used for generated scripts and command execution.
- void **setPreludeScript** (std::string preludeScript)
Sets optional Octave code prepended before the command block.
- std::string **getPreludeScript** () const
Returns optional Octave code prepended before the command block.
- void **setCommand** (std::string command)
Sets the Octave command/script body to execute.
- std::string **getCommand** () const
Returns the Octave command/script body to execute.
- void **setLastStatus** (std::string lastStatus)
Sets the last execution status.
- std::string **getLastStatus** () const
Returns the last execution status.
- void **setLastExitCode** (int lastExitCode)
Sets the last process exit code.
- int **getLastExitCode** () const
Returns the last process exit code.
- void **setLastStdout** (std::string lastStdout)
Sets stdout captured from the last Octave execution.
- std::string **getLastStdout** () const
Returns stdout captured from the last Octave execution.
- void **setLastStderr** (std::string lastStderr)
Sets stderr captured from the last Octave execution.
- std::string **getLastStderr** () const
Returns stderr captured from the last Octave execution.
- void **setLastResponsePayload** (std::string lastResponsePayload)
Sets the response payload exposed to other tools.
- std::string **getLastResponsePayload** () const
Returns the response payload exposed to other tools.
- void **setLastScriptFilename** (std::string lastScriptFilename)
Sets the generated Octave script filename from the last execution.
- std::string **getLastScriptFilename** () const
Returns the generated Octave script filename from the last execution.
- void **setLastResponseFilename** (std::string lastResponseFilename)
Sets the stdout file path from the last execution.
- std::string **getLastResponseFilename** () const
Returns the stdout file path from the last execution.
- bool **executeCommand** (std::string &errorMessage)
Executes the configured Octave code and updates last-execution fields atomically.
- virtual std::string **show** () override
Returns a compact human-readable description of the runner.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool **hasChanged**)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- **List**< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
Loads an Octave Runner from a persistence record.
- static [PluginInformation](#) * **GetPluginInformation** ()
Returns plugin metadata and system dependency declarations.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new Octave Runner instance for plugin construction.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
Loads persisted Octave Runner configuration and last-execution state.
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves Octave Runner configuration and last-execution state.
- virtual bool **_check** (std::string &errorMessage) override
Validates the runner configuration without executing Octave.

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool **_getSaveDefaultsOption** ()
- virtual [ParserChangesInformation](#) * **_getParserChangesInformation** ()
- virtual void **_initBetweenReplications** ()
- virtual void **_addSimulationControl** ([SimulationControl](#) *control)
- virtual void **_createInternalAndAttachedData** ()
- virtual void **_createInternalStatisticReporters** ()
- virtual void **_createAttachedAttributes** ()
- virtual void **_createNonEditableDataDefinitions** ()
- virtual void **_createEditableDataDefinitions** ()
- void **_templateCreateInternalStatisticReporters** ()
- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
legacy helper methods used by the granular create hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)

- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.471.1 Detailed Description

[Model](#) data definition that executes Octave code through Rscript.

The runner stores a compact Octave execution configuration, writes a temporary script, executes it with Rscript, and keeps status, exit code, stdout, stderr and response payload inside the model data definition for later inspection.

15.471.2 Member Function Documentation

15.471.2.1 `_check()`

```
bool OctaveRunner::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

Validates the runner configuration without executing Octave.

Reimplemented from [ModelDataDefinition](#).

15.471.2.2 `_loadInstance()`

```
bool OctaveRunner::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

Loads persisted Octave Runner configuration and last-execution state.

Reimplemented from [ModelDataDefinition](#).

15.471.2.3 `_saveInstance()`

```
void OctaveRunner::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves Octave Runner configuration and last-execution state.

Reimplemented from [ModelDataDefinition](#).

15.471.2.4 `show()`

```
std::string OctaveRunner::show ( ) [override], [virtual]
```

Returns a compact human-readable description of the runner.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- `plugins/data/ExternalIntegration/OctaveRunner.h`
- `plugins/data/ExternalIntegration/OctaveRunner.cpp`

15.472 ODEfunction Class Reference

Public Member Functions

- **ODEfunction** (std::string expression, double initialPoint, double initialValue)

Public Attributes

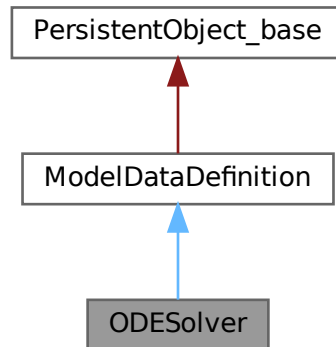
- std::string **expression**
- double **initialPoint**
- double **initialValue**

The documentation for this class was generated from the following file:

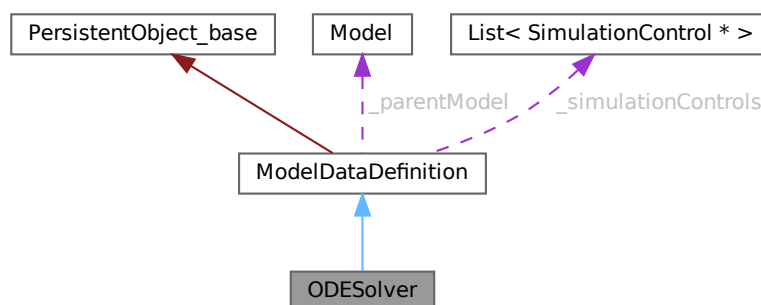
- `plugins/components/Continuous/OLD_ODEelement.h`

15.473 ODESolver Class Reference

Inheritance diagram for ODESolver:



Collaboration diagram for ODESolver:



Public Member Functions

- **ODESolver** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- void **setTimeVariableName** (std::string timeVariableName)
- std::string **getTimeVariableName** () const
- void **setStateVariableNames** (std::vector< std::string > stateVariableNames)
- std::vector< std::string > **getStateVariableNames** () const
- void **setEquationExpressions** (std::vector< std::string > equationExpressions)
- std::vector< std::string > **getEquationExpressions** () const
- void **setStep** (double step)
- double **getStep** () const
- void **setPrecision** (double precision)
- double **getPrecision** () const

- void **setMaxSteps** (int maxSteps)
- int **getMaxSteps** () const
- void **setCurrentTime** (double currentTime)
- double **getCurrentTime** () const
- void **setStateValues** (std::vector< double > stateValues)
- std::vector< double > **getStateValues** () const
- void **setInitialStateValues** (std::vector< double > initialStateValues)
- std::vector< double > **getInitialStateValues** () const
- void **setAutoAdvance** (bool autoAdvance)
- bool **isAutoAdvance** () const
- void **setOutputFile** (std::string outputFile)
- std::string **getOutputFile** () const
- void **integrate** (double targetTime)
- void **resetState** ()
- double **getStateValue** (unsigned int index) const
- void **setStateValue** (unsigned int index, double value)
- void **synchronizeStateFromVariables** ()
- void **writeStateToVariables** () const

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.

- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int [_modelLevel](#))
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) [traceLevelspecific](#), bool [traceLevelSpecificEnabled](#)=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool [traceLevelSpecificEnabled](#))
Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *dataDef)
- static bool [Check](#) ([ModelDataDefinition](#) *dataDef, std::string &errorMessage)
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *dataDef)
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *dataDef)

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_initBetweenReplications` () override
- virtual void `_createEditableDataDefinitions` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void `trace` (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))

not just an easy access to trace manager, but wrappers to check if specific trace level applies

- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

Traces text to the simulation output.

- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.473.1 Member Function Documentation

15.473.1.1 [_check\(\)](#)

```
bool ODESolver::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.473.1.2 [_createEditableDataDefinitions\(\)](#)

```
void ODESolver::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.473.1.3 [_initBetweenReplications\(\)](#)

```
void ODESolver::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.473.1.4 [_loadInstance\(\)](#)

```
bool ODESolver::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.473.1.5 [_saveInstance\(\)](#)

```
void ODESolver::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.473.1.6 show()

```
std::string ODESolver::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

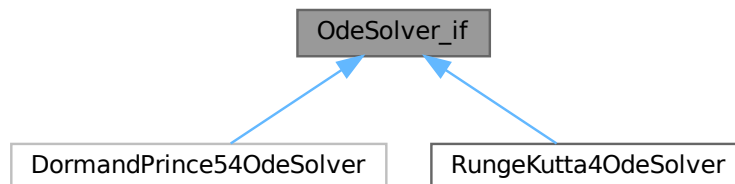
- plugins/data/Continuous/ODESolver.h
- plugins/data/Continuous/ODESolver.cpp

15.474 OdeSolver_if Class Reference

ODE solver abstraction.

```
#include <tools/Continuous/OdeSolver_if.h>
```

Inheritance diagram for OdeSolver_if:

**Public Member Functions**

- virtual bool **advance** (const [OdeSystem_if](#) &system, double t0, double dt, const double *y0, double *y1) const =0

15.474.1 Detailed Description

ODE solver abstraction.

Architectural role:

- Planned evolution of the "derivate" responsibility currently exposed by legacy [Solver_if](#).

The documentation for this class was generated from the following file:

- tools/Continuous/OdeSolver_if.h

15.475 OdeSolverFactory Class Reference

Public Types

- using **Creator** = std::function< std::unique_ptr< [OdeSolver_if](#) >()>

Static Public Member Functions

- static const std::string & **keyRungeKutta4** ()
- static const std::string & **keyDormandPrince54** ()
- static const std::string & **defaultKey** ()
- static std::unique_ptr< [OdeSolver_if](#) > **create** (OdeSolverType type)
- static std::unique_ptr< [OdeSolver_if](#) > **create** (const std::string &key)
- static bool **isRegistered** (const std::string &key)
- static std::vector< std::string > **availableKeys** ()
- static bool **registerCreator** (const std::string &key, Creator creator)
- static std::string **toKey** (OdeSolverType type)
- static bool **fromKey** (const std::string &key, OdeSolverType &out)

The documentation for this class was generated from the following file:

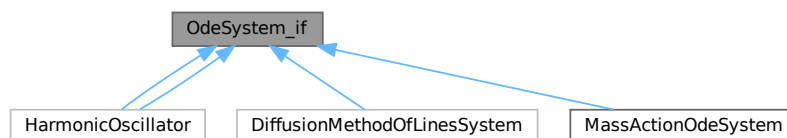
- tools/Continuous/OdeSolverFactory.h

15.476 OdeSystem_if Class Reference

Interface for first-order ODE systems $dy/dt = f(t, y)$.

```
#include <tools/Continuous/OdeSystem_if.h>
```

Inheritance diagram for OdeSystem_if:



Public Member Functions

- virtual unsigned int **dimension** () const =0
- virtual void **evaluate** (double t, const double *y, double *dydt) const =0

15.476.1 Detailed Description

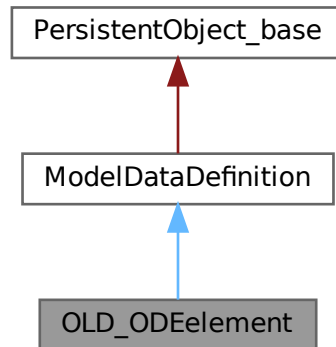
Interface for first-order ODE systems $dy/dt = f(t, y)$.

The documentation for this class was generated from the following file:

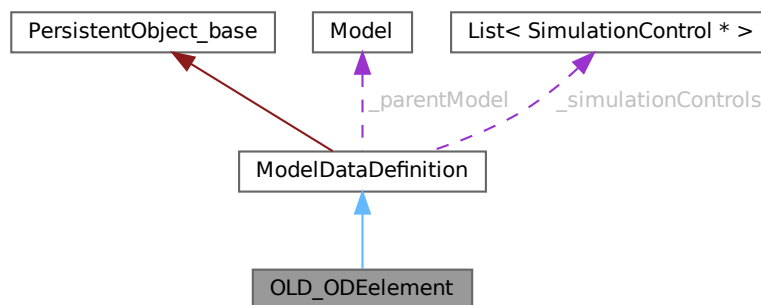
- tools/Continuous/OdeSystem_if.h

15.477 OLD_ODElement Class Reference

Inheritance diagram for OLD_ODElement:



Collaboration diagram for OLD_ODElement:



Public Member Functions

- **OLD_ODElement** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- double **solve** ()
- void **setStepH** (double _h)
- double **getStepH** () const
- void **setEndTime** (double _endTime)
- double **getEndTime** () const
- [List](#)< [ODEfunction](#) * > * **getODEfunctions** () const
- void **addODEfunction** ([ODEfunction](#) *newFunction)
- void **removeODEfunction** ([ODEfunction](#) *function)

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool **hasChanged**)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- **List**< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool **_getSaveDefaultsOption** ()
- virtual [ParserChangesInformation](#) * **_getParserChangesInformation** ()
- virtual void **_initBetweenReplications** ()
- virtual void **_addSimulationControl** ([SimulationControl](#) *control)
- virtual void **_createInternalAndAttachedData** ()
- virtual void **_createInternalStatisticReporters** ()
- virtual void **_createAttachedAttributes** ()
- virtual void **_createNonEditableDataDefinitions** ()
- virtual void **_createEditableDataDefinitions** ()
- void **_templateCreateInternalStatisticReporters** ()
- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
legacy helper methods used by the granular create hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)

- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.477.1 Member Function Documentation

15.477.1.1 _check()

```
bool OLD_ODElement::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.477.1.2 `_loadInstance()`

```
bool OLD_ODElement::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.477.1.3 `_saveInstance()`

```
void OLD_ODElement::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.477.1.4 `show()`

```
std::string OLD_ODElement::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- `plugins/components/Continuous/OLD_ODElement.h`
- `plugins/components/Continuous/OLD_ODElement.cpp`

15.478 OnEventManager Class Reference

```
#include <kernel/simulator/OnEventManager.h>
```

Public Member Functions

- void **addOnModelCheckSucessHandler** (modelEventHandler EventHandler)
- void **addOnModelLoadHandler** (modelEventHandler EventHandler)
- void **addOnModelSaveHandler** (modelEventHandler EventHandler)
- void **addOnReplicationStartHandler** (simulationEventHandler EventHandler)
- void **addOnReplicationStepHandler** (simulationEventHandler EventHandler)
- void **addOnReplicationEndHandler** (simulationEventHandler EventHandler)
- void **addOnProcessEventHandler** (simulationEventHandler EventHandler)
- void **addOnAfterProcessEventHandler** (simulationEventHandler EventHandler)
- void **addOnEntityCreateHandler** (simulationEventHandler EventHandler)
- void **addOnEntityMoveHandler** (simulationEventHandler EventHandler)
- void **addOnEntityRemoveHandler** (simulationEventHandler EventHandler)
- void **addOnSimulationStartHandler** (simulationEventHandler EventHandler)
- void **addOnSimulationPausedHandler** (simulationEventHandler EventHandler)
- void **addOnSimulationResumeHandler** (simulationEventHandler EventHandler)
- void **addOnSimulationEndHandler** (simulationEventHandler EventHandler)

- void **addOnBreakpointHandler** (simulationEventHandler EventHandler)
- template<typename Class >
void **addOnModelCheckSuccessHandler** (Class *object, void(Class::*function)(ModelEvent *))
- template<typename Class >
void **addOnModelLoadHandler** (Class *object, void(Class::*function)(ModelEvent *))
- template<typename Class >
void **addOnModelSaveHandler** (Class *object, void(Class::*function)(ModelEvent *))
- template<typename Class >
void **addOnReplicationStartHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnReplicationStepHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnReplicationEndHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnProcessEventHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnAfterProcessEventHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnEntityCreateHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnEntityMoveHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnEntityRemoveHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnSimulationStartHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnSimulationPausedHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnSimulationResumeHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnSimulationEndHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnBreakpointHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- void **NotifyModelCheckSuccessHandlers** (ModelEvent *se)
- void **NotifyModelLoadHandlers** (ModelEvent *se)
- void **NotifyModelSaveHandlers** (ModelEvent *se)
- void **NotifyReplicationStartHandlers** (SimulationEvent *se)
- void **NotifyReplicationStepHandlers** (SimulationEvent *se)
- void **NotifyReplicationEndHandlers** (SimulationEvent *se)
- void **NotifyProcessEventHandlers** (SimulationEvent *se)
- void **NotifyAfterProcessEventHandlers** (SimulationEvent *se)
- void **NotifyEntityCreateHandlers** (SimulationEvent *se)
- void **NotifyEntityMoveHandlers** (SimulationEvent *se)
- void **NotifyEntityRemoveHandlers** (SimulationEvent *se)
- void **NotifySimulationStartHandlers** (SimulationEvent *se)
- void **NotifySimulationPausedHandlers** (SimulationEvent *se)
- void **NotifySimulationResumeHandlers** (SimulationEvent *se)
- void **NotifySimulationEndHandlers** (SimulationEvent *se)
- void **NotifyBreakpointHandlers** (SimulationEvent *se)

15.478.1 Detailed Description

[OnEventManager](#) allows external methods to hook interval simulation events as listeners (or observers) of specific events. All methods added as listeners of an event will be invoked when that event is triggered.

The documentation for this class was generated from the following files:

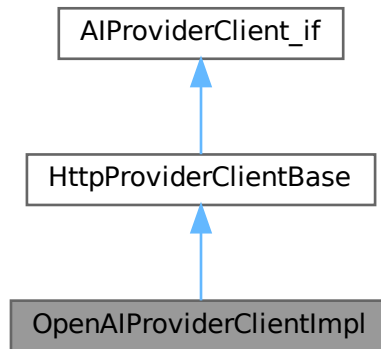
- kernel/simulator/OnEventManager.h
- kernel/simulator/OnEventManager.cpp

15.479 OpenAIProviderClientImpl Class Reference

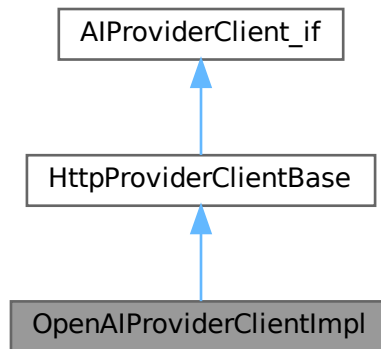
AI provider client for the OpenAI Chat Completions API.

```
#include <tools/AIAssistant/OpenAIProviderClientImpl.h>
```

Inheritance diagram for OpenAIProviderClientImpl:



Collaboration diagram for OpenAIProviderClientImpl:



Public Member Functions

- `OpenAIProviderClientImpl ()`
Constructs the client with the standard OpenAI base URL.
- `std::string getProviderName ()` const override
Returns a human-readable provider name (e.g. "OpenAI", "Anthropic").
- `bool isAvailable ()` const override
Returns `true` when an API key is set and a base URL is configured.
- `void setOrganizationId (const std::string &orgId)`
Sets the OpenAI organization identifier (sent as OpenAI-Organization header).

- `std::string getOrganizationId ()` const
Returns the configured organization identifier.
- `void setProjectId (const std::string &projectId)`
Sets the OpenAI project identifier (sent as OpenAI-Project header).
- `std::string getProjectId ()` const
Returns the configured project identifier.

Public Member Functions inherited from [HttpProviderClientBase](#)

- `HttpProviderClientBase (const HttpProviderClientBase &)=delete`
- `HttpProviderClientBase & operator= (const HttpProviderClientBase &)=delete`
- `void setBaseUrl (const std::string &url)` override
Sets the base URL for the provider API endpoint.
- `std::string getBaseUrl ()` const override
Returns the currently configured base URL.
- `void setTimeoutSeconds (unsigned int seconds)` override
Sets the HTTP request timeout in seconds.
- `unsigned int getTimeoutSeconds ()` const override
Returns the configured HTTP request timeout.
- `void setApiKey (const std::string &apiKey)` override
Stores the API key in memory.
- `void clearApiKey ()` override
Overwrites (best-effort) and clears the in-memory API key.
- `bool hasApiKey ()` const override
Returns `true` when a non-empty API key is held.
- `ProviderResponse send (const ProviderRequest &request)` override
Executes the HTTP request via curl and returns the provider response.

Protected Member Functions

- `std::string _buildRequestBody (const ProviderRequest &request)` const override
Serialises `request` to the provider's JSON wire format.
- `ProviderResponse _parseResponseBody (const std::string &rawBody, int httpStatus)` const override
Deserialises `rawBody` (the full curl stdout) into a [ProviderResponse](#).
- `std::string _authorizationHeader ()` const override
Returns the Authorization header line without the "header = " prefix.
- `std::vector< std::string > _extraRequestHeaders ()` const override
Returns additional HTTP headers to send with every request.

Additional Inherited Members

Static Public Member Functions inherited from [HttpProviderClientBase](#)

- `static std::string _escapeJsonString (const std::string &s)`
Escapes a plain string for safe embedding inside a JSON string value.
- `static std::string _extractJsonString (const std::string &json, const std::string &key)`
Extracts the value of a top-level JSON string field by key. Returns an empty string when not found or value is not a quoted string.
- `static int _extractJsonInt (const std::string &json, const std::string &key)`
Extracts the value of a top-level JSON integer field by key. Returns 0 when not found.
- `static double _extractJsonDouble (const std::string &json, const std::string &key)`
Extracts the value of a top-level JSON number field (int or float) by key. Returns 0.0 when not found.
- `static std::string _buildMessagesArray (const std::vector< ChatMessage > &messages)`

Serialises a vector of [ChatMessage](#) objects to a JSON array string.

- static std::string **_extractFirstArrayElement** (const std::string &json, const std::string &arrayKey)

Extracts the raw substring of the first element of a named JSON array. Returns an empty string when not found.

Protected Attributes inherited from [HttpProviderClientBase](#)

- std::string **_baseUrl**
- unsigned int **_timeoutSeconds** = 30
- std::string **_apiKey**

In-memory API key.

15.479.1 Detailed Description

AI provider client for the OpenAI Chat Completions API.

Wraps the OpenAI `/v1/chat/completions` endpoint. Default base URL: <https://api.openai.com/v1/chat/completions>

Supported features:

- Standard chat completions (gpt-4o, gpt-4-turbo, gpt-3.5-turbo, etc.).
- Reasoning models via `highReasoningMode` (o3, o4-mini): temperature is omitted and `reasoning_effort` is set to "high" in the request body.
- Organization and project headers when set via [setOrganizationId\(\)](#) / [setProjectId\(\)](#).

15.479.1.1 API key

Passed as `Authorization: Bearer <key>`. Stored in the parent's `_apiKey` member; never written to the command line or logs.

15.479.2 Constructor & Destructor Documentation

15.479.2.1 OpenAIProviderClientImpl()

```
OpenAIProviderClientImpl::OpenAIProviderClientImpl ( )
```

Constructs the client with the standard OpenAI base URL.

To use with Azure OpenAI or a compatible proxy, call [setBaseUrl\(\)](#) after construction.

15.479.3 Member Function Documentation

15.479.3.1 _authorizationHeader()

```
std::string OpenAIProviderClientImpl::_authorizationHeader ( ) const [override], [protected], [virtual]
```

Returns the Authorization header line without the "header = " prefix.

Example return values:

- "Authorization: Bearer sk-..." (OpenAI, LocalProvider)
- "x-api-key: sk-ant-..." (Anthropic)

Called only from within [send\(\)](#); the value is written to a 0600 temp file and never logged.

Implements [HttpProviderClientBase](#).

15.479.3.2 _buildRequestBody()

```
std::string OpenAIProviderClientImpl::_buildRequestBody (
    const ProviderRequest & request ) const [override], [protected], [virtual]
```

Serialises `request` to the provider's JSON wire format.

Implements [HttpProviderClientBase](#).

15.479.3.3 `_extraRequestHeaders()`

```
std::vector< std::string > OpenAIProviderClientImpl::_extraRequestHeaders ( ) const [override],  
[protected], [virtual]
```

Returns additional HTTP headers to send with every request.

Default implementation returns an empty vector. Anthropic overrides to add "anthropic-version: 2023-06-01".

Each element must be a complete header line, e.g. "anthropic-version: 2023-06-01".

Reimplemented from [HttpProviderClientBase](#).

15.479.3.4 `_parseResponseBody()`

```
ProviderResponse OpenAIProviderClientImpl::_parseResponseBody (   
    const std::string & rawBody,  
    int httpStatus ) const [override], [protected], [virtual]
```

Deserialises *rawBody* (the full curl stdout) into a [ProviderResponse](#).

Parameters

<i>rawBody</i>	Complete HTTP response body as received from curl.
<i>httpStatus</i>	HTTP status code captured from curl exit / response line, or 0 when unavailable.

Implements [HttpProviderClientBase](#).

15.479.3.5 `getProviderName()`

```
std::string OpenAIProviderClientImpl::getProviderName ( ) const [override], [virtual]
```

Returns a human-readable provider name (e.g. "OpenAI", "Anthropic").

Implements [AIProviderClient_if](#).

15.479.3.6 `isAvailable()`

```
bool OpenAIProviderClientImpl::isAvailable ( ) const [override], [virtual]
```

Returns `true` when an API key is set and a base URL is configured.

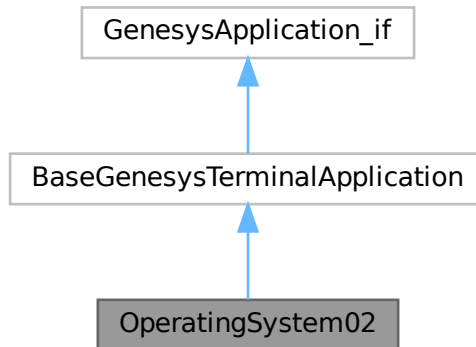
Reimplemented from [HttpProviderClientBase](#).

The documentation for this class was generated from the following files:

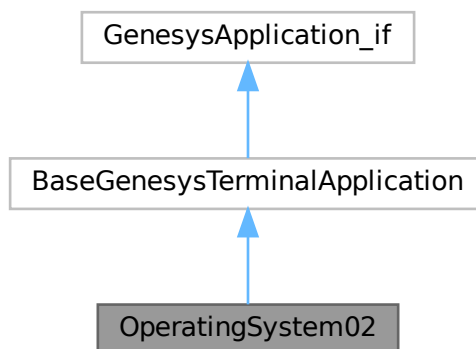
- tools/AIAssistant/OpenAIProviderClientImpl.h
- tools/AIAssistant/OpenAIProviderClientImpl.cpp

15.480 OperatingSystem02 Class Reference

Inheritance diagram for OperatingSystem02:



Collaboration diagram for OperatingSystem02:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

15.480.1 Member Function Documentation

15.480.1.1 main()

```
int OperatingSystem02::main (
    int argc,
    char ** argv ) [override], [virtual]
```

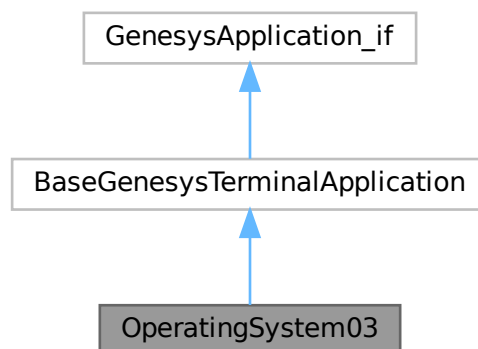
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

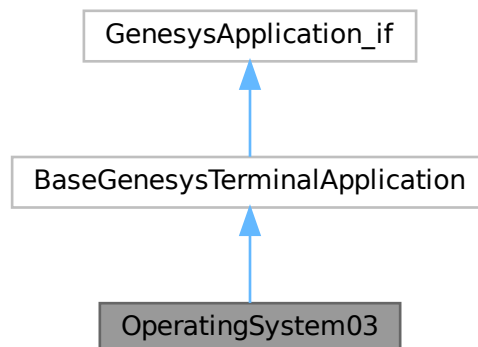
- applications/modelSpecific/teaching/OperatingSystem02.h
- applications/modelSpecific/teaching/OperatingSystem02.cpp

15.481 OperatingSystem03 Class Reference

Inheritance diagram for OperatingSystem03:



Collaboration diagram for OperatingSystem03:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

15.481.1 Member Function Documentation

15.481.1.1 main()

```
int OperatingSystem03::main (
    int argc,
    char ** argv ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/teaching/OperatingSystem03.h
- applications/modelSpecific/teaching/OperatingSystem03.cpp

15.482 Optimizer_if::OptimizationSettings Struct Reference

Generic optimization settings independent of a specific algorithm.

```
#include <tools/Optimization/Optimizer_if.h>
```

Public Attributes

- unsigned int **maxIterations** = 100
- unsigned int **maxSimulations** = 0
- unsigned int **replicationsPerSolution** = 1
- unsigned int **bestSolutionsToKeep** = 10
- unsigned int **randomSeed** = 0
- double **improvementTolerance** = 0.0
- double **timeLimitSeconds** = 0.0

15.482.1 Detailed Description

Generic optimization settings independent of a specific algorithm.

The documentation for this struct was generated from the following file:

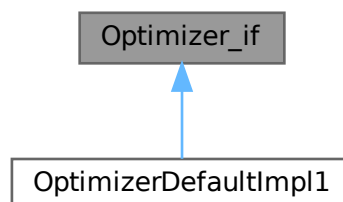
- tools/Optimization/Optimizer_if.h

15.483 Optimizer_if Class Reference

Interface for simulation-based optimization engines (OptQuest-like).

```
#include <tools/Optimization/Optimizer_if.h>
```

Inheritance diagram for Optimizer_if:



Classes

- struct [ConstraintDefinition](#)
Feasibility restriction represented as an expression.
- struct [ObjectiveDefinition](#)
Configurable objective definition.
- struct [OptimizationSettings](#)
Generic optimization settings independent of a specific algorithm.
- struct [SolutionSummary](#)
Captures one candidate or best-known optimization solution.

Public Types

- enum class [ObjectiveSense](#) { **MINIMIZE** , **MAXIMIZE** }
Direction used by an optimization objective.
- enum class [ExecutionState](#) {
NOT_READY , **READY** , **RUNNING** , **PAUSED** ,
STOPPED , **FINISHED** , **ERROR** }
Execution state of the optimization process.

Public Member Functions

- virtual void [setModel](#) ([Model](#) *model)=0
Associates an already instantiated model with this optimizer.
- virtual bool [setModelFilename](#) (const std::string &filename)=0
Loads/associates a model from a file supported by the simulator.
- virtual [Model](#) * [getModel](#) () const =0
Returns the model currently associated with the optimizer.
- virtual std::string [getModelFilename](#) () const =0
Returns the source filename previously configured for model loading.
- virtual [List](#)< [SimulationControl](#) * > * [getAvailableControls](#) () const =0
Retrieves available model controls discoverable from [Model](#).
- virtual [List](#)< [SimulationResponse](#) * > * [getAvailableResponses](#) () const =0
Retrieves available model responses discoverable from [Model](#).
- virtual void [setSelectedControls](#) ([List](#)< [SimulationControl](#) * > *controls)=0
Replaces the set of controls selected for optimization.
- virtual void [setSelectedResponses](#) ([List](#)< [SimulationResponse](#) * > *responses)=0
Replaces the set of responses selected for optimization.
- virtual [List](#)< [SimulationControl](#) * > * [getSelectedControls](#) () const =0
Returns currently selected controls used as decision variables.
- virtual [List](#)< [SimulationResponse](#) * > * [getSelectedResponses](#) () const =0
Returns currently selected responses monitored by optimization.
- virtual void [addObjective](#) (const [ObjectiveDefinition](#) &objective)=0
Adds one objective to the optimization model.
- virtual void [clearObjectives](#) ()=0
Clears all configured objectives.
- virtual [List](#)< [ObjectiveDefinition](#) > * [getObjectives](#) () const =0
Returns all configured optimization objectives.
- virtual void [addConstraint](#) (const [ConstraintDefinition](#) &constraint)=0
Adds one feasibility constraint expression.
- virtual void [clearConstraints](#) ()=0
Clears all configured constraints.
- virtual [List](#)< [ConstraintDefinition](#) > * [getConstraints](#) () const =0

- Returns all configured feasibility constraints.*
- virtual void `setSettings` (const `OptimizationSettings` &settings)=0
Applies algorithm settings such as iteration and retention limits.
- virtual `OptimizationSettings` `getSettings` () const =0
Returns currently active optimization settings.
- virtual bool `checkReady` (std::string *message=nullptr) const =0
Validates readiness before running optimization.
- virtual bool `start` ()=0
Starts a new optimization run.
- virtual bool `step` ()=0
Executes one optimization iteration (or one scenario decision step).
- virtual bool `pause` ()=0
Pauses an active optimization execution.
- virtual bool `resume` ()=0
Resumes optimization execution after a pause.
- virtual bool `stop` ()=0
Stops optimization execution and finalizes current run.
- virtual `ExecutionState` `getExecutionState` () const =0
Returns current lifecycle state of the optimizer.
- virtual unsigned int `getCurrentIteration` () const =0
Returns the number of optimization iterations already performed.
- virtual unsigned int `getTotalSimulations` () const =0
Returns total number of simulated scenarios evaluated so far.
- virtual `List< SolutionSummary >` * `getBestSolutions` () const =0
Returns all feasible solutions retained as "best" by the algorithm.
- virtual const `SolutionSummary` * `getCurrentBestSolution` () const =0
Returns the best-known feasible solution, or nullptr when absent.
- virtual void `resetResults` ()=0
Clears results from previous runs while preserving configuration.

15.483.1 Detailed Description

Interface for simulation-based optimization engines (OptQuest-like).

This contract defines how an optimization tool can be attached to a model, configured, and executed while managing:

- decision variables (selected `SimulationControl` objects),
- observed KPIs (selected `SimulationResponse` objects),
- objectives (minimize/maximize expressions), and
- constraints (boolean expressions evaluated as false==0 and true!=0).

Typical flow: 1) Associate a model (from file or direct instance). 2) Select controls and responses participating in the search. 3) Register optimization objectives and feasibility constraints. 4) Configure search parameters (iterations, retained best solutions, etc.). 5) Run lifecycle commands (start/step/pause/resume/stop). 6) Query current status, best solutions and progress metrics.

15.483.2 Member Function Documentation

15.483.2.1 addConstraint()

```
virtual void Optimizer_if::addConstraint (
    const ConstraintDefinition & constraint ) [pure virtual]
```

Adds one feasibility constraint expression.

Implemented in `OptimizerDefaultImpl1`.

15.483.2.2 addObjective()

```
virtual void Optimizer_if::addObjective (
    const ObjectiveDefinition & objective ) [pure virtual]
```

Adds one objective to the optimization model.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.3 checkReady()

```
virtual bool Optimizer_if::checkReady (
    std::string * message = nullptr ) const [pure virtual]
```

Validates readiness before running optimization.

Parameters

<i>message</i>	Receives validation details when non-null.
----------------	--

Returns

true when optimizer has enough data/configuration to start.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.4 clearConstraints()

```
virtual void Optimizer_if::clearConstraints ( ) [pure virtual]
```

Clears all configured constraints.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.5 clearObjectives()

```
virtual void Optimizer_if::clearObjectives ( ) [pure virtual]
```

Clears all configured objectives.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.6 getAvailableControls()

```
virtual List< SimulationControl * > * Optimizer_if::getAvailableControls ( ) const [pure virtual]
```

Retrieves available model controls discoverable from [Model](#).

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.7 getAvailableResponses()

```
virtual List< SimulationResponse * > * Optimizer_if::getAvailableResponses ( ) const [pure virtual]
```

Retrieves available model responses discoverable from [Model](#).

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.8 getBestSolutions()

```
virtual List< SolutionSummary > * Optimizer_if::getBestSolutions ( ) const [pure virtual]
```

Returns all feasible solutions retained as "best" by the algorithm.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.9 getConstraints()

```
virtual List< ConstraintDefinition > * Optimizer_if::getConstraints ( ) const [pure virtual]
```

Returns all configured feasibility constraints.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.10 getCurrentBestSolution()

```
virtual const SolutionSummary * Optimizer_if::getCurrentBestSolution ( ) const [pure virtual]
```

Returns the best-known feasible solution, or nullptr when absent.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.11 getCurrentIteration()

```
virtual unsigned int Optimizer_if::getCurrentIteration ( ) const [pure virtual]
```

Returns the number of optimization iterations already performed.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.12 getExecutionState()

```
virtual ExecutionState Optimizer_if::getExecutionState ( ) const [pure virtual]
```

Returns current lifecycle state of the optimizer.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.13 getModel()

```
virtual Model * Optimizer_if::getModel ( ) const [pure virtual]
```

Returns the model currently associated with the optimizer.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.14 getModelFilename()

```
virtual std::string Optimizer_if::getModelFilename ( ) const [pure virtual]
```

Returns the source filename previously configured for model loading.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.15 getObjectives()

```
virtual List< ObjectiveDefinition > * Optimizer_if::getObjectives ( ) const [pure virtual]
```

Returns all configured optimization objectives.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.16 getSelectedControls()

```
virtual List< SimulationControl * > * Optimizer_if::getSelectedControls ( ) const [pure virtual]
```

Returns currently selected controls used as decision variables.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.17 getSelectedResponses()

```
virtual List< SimulationResponse * > * Optimizer_if::getSelectedResponses ( ) const [pure virtual]
```

Returns currently selected responses monitored by optimization.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.18 getSettings()

```
virtual OptimizationSettings Optimizer_if::getSettings ( ) const [pure virtual]
```

Returns currently active optimization settings.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.19 getTotalSimulations()

```
virtual unsigned int Optimizer_if::getTotalSimulations ( ) const [pure virtual]
```

Returns total number of simulated scenarios evaluated so far.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.20 pause()

```
virtual bool Optimizer_if::pause ( ) [pure virtual]
```

Pauses an active optimization execution.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.21 resetResults()

```
virtual void Optimizer_if::resetResults ( ) [pure virtual]
```

Clears results from previous runs while preserving configuration.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.22 resume()

```
virtual bool Optimizer_if::resume ( ) [pure virtual]
```

Resumes optimization execution after a pause.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.23 setModel()

```
virtual void Optimizer_if::setModel (
    Model * model ) [pure virtual]
```

Associates an already instantiated model with this optimizer.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.24 setModelFilename()

```
virtual bool Optimizer_if::setModelFilename (
    const std::string & filename ) [pure virtual]
```

Loads/associates a model from a file supported by the simulator.

Parameters

<i>filename</i>	Model file path.
-----------------	----------------------------------

Returns

true when the model file was successfully loaded and linked.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.25 setSelectedControls()

```
virtual void Optimizer_if::setSelectedControls (
    List< SimulationControl * > * controls ) [pure virtual]
```

Replaces the set of controls selected for optimization.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.26 setSelectedResponses()

```
virtual void Optimizer_if::setSelectedResponses (
    List< SimulationResponse * > * responses ) [pure virtual]
```

Replaces the set of responses selected for optimization.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.27 setSettings()

```
virtual void Optimizer_if::setSettings (
    const OptimizationSettings & settings ) [pure virtual]
```

Applies algorithm settings such as iteration and retention limits.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.28 start()

```
virtual bool Optimizer_if::start ( ) [pure virtual]
```

Starts a new optimization run.

Returns

true on successful transition to RUNNING state.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.29 step()

```
virtual bool Optimizer_if::step ( ) [pure virtual]
```

Executes one optimization iteration (or one scenario decision step).

Returns

true when a step was executed.

Implemented in [OptimizerDefaultImpl1](#).

15.483.2.30 stop()

```
virtual bool Optimizer_if::stop ( ) [pure virtual]
```

Stops optimization execution and finalizes current run.

Implemented in [OptimizerDefaultImpl1](#).

The documentation for this class was generated from the following file:

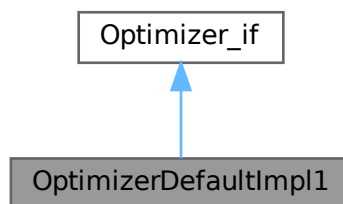
- tools/Optimization/Optimizer_if.h

15.484 OptimizerDefaultImpl1 Class Reference

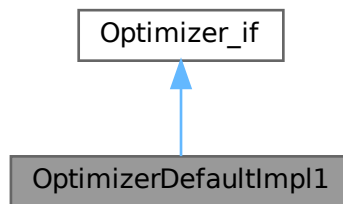
Initial concrete scaffold for an OptQuest-like optimizer.

```
#include <tools/Optimization/OptimizerDefaultImpl1.h>
```

Inheritance diagram for OptimizerDefaultImpl1:



Collaboration diagram for OptimizerDefaultImpl1:



Public Member Functions

- **OptimizerDefaultImpl1** (const [OptimizerDefaultImpl1](#) &)=delete
- [OptimizerDefaultImpl1](#) & **operator=** (const [OptimizerDefaultImpl1](#) &)=delete
- **OptimizerDefaultImpl1** ([OptimizerDefaultImpl1](#) &&)=delete
- [OptimizerDefaultImpl1](#) & **operator=** ([OptimizerDefaultImpl1](#) &&)=delete
- virtual void [setModel](#) ([Model](#) *model) override
Associates an already instantiated model with this optimizer.
- virtual bool [setModelFilename](#) (const std::string &filename) override
Loads/associates a model from a file supported by the simulator.
- virtual [Model](#) * [getModel](#) () const override
Returns the model currently associated with the optimizer.
- virtual std::string [getModelFilename](#) () const override
Returns the source filename previously configured for model loading.
- virtual [List](#)< [SimulationControl](#) * > * [getAvailableControls](#) () const override
Retrieves available model controls discoverable from [Model](#).
- virtual [List](#)< [SimulationResponse](#) * > * [getAvailableResponses](#) () const override
Retrieves available model responses discoverable from [Model](#).
- virtual void [setSelectedControls](#) ([List](#)< [SimulationControl](#) * > *controls) override
Replaces the set of controls selected for optimization.
- virtual void [setSelectedResponses](#) ([List](#)< [SimulationResponse](#) * > *responses) override
Replaces the set of responses selected for optimization.
- virtual [List](#)< [SimulationControl](#) * > * [getSelectedControls](#) () const override
Returns currently selected controls used as decision variables.
- virtual [List](#)< [SimulationResponse](#) * > * [getSelectedResponses](#) () const override
Returns currently selected responses monitored by optimization.
- virtual void [addObjective](#) (const [ObjectiveDefinition](#) &objective) override
Adds one objective to the optimization model.
- virtual void [clearObjectives](#) () override
Clears all configured objectives.
- virtual [List](#)< [ObjectiveDefinition](#) > * [getObjectives](#) () const override
Returns all configured optimization objectives.
- virtual void [addConstraint](#) (const [ConstraintDefinition](#) &constraint) override
Adds one feasibility constraint expression.
- virtual void [clearConstraints](#) () override
Clears all configured constraints.
- virtual [List](#)< [ConstraintDefinition](#) > * [getConstraints](#) () const override

- Returns all configured feasibility constraints.*
- virtual void [setSettings](#) (const [OptimizationSettings](#) &settings) override
Applies algorithm settings such as iteration and retention limits.
- virtual [OptimizationSettings](#) [getSettings](#) () const override
Returns currently active optimization settings.
- virtual bool [checkReady](#) (std::string *message=nullptr) const override
Validates readiness before running optimization.
- virtual bool [start](#) () override
Starts a new optimization run.
- virtual bool [step](#) () override
Executes one optimization iteration (or one scenario decision step).
- virtual bool [pause](#) () override
Pauses an active optimization execution.
- virtual bool [resume](#) () override
Resumes optimization execution after a pause.
- virtual bool [stop](#) () override
Stops optimization execution and finalizes current run.
- virtual [ExecutionState](#) [getExecutionState](#) () const override
Returns current lifecycle state of the optimizer.
- virtual unsigned int [getCurrentIteration](#) () const override
Returns the number of optimization iterations already performed.
- virtual unsigned int [getTotalSimulations](#) () const override
Returns total number of simulated scenarios evaluated so far.
- virtual [List](#)< [SolutionSummary](#) > * [getBestSolutions](#) () const override
Returns all feasible solutions retained as "best" by the algorithm.
- virtual const [SolutionSummary](#) * [getCurrentBestSolution](#) () const override
Returns the best-known feasible solution, or nullptr when absent.
- virtual void [resetResults](#) () override
Clears results from previous runs while preserving configuration.

Additional Inherited Members

Public Types inherited from [Optimizer_if](#)

- enum class [ObjectiveSense](#) { **MINIMIZE** , **MAXIMIZE** }
Direction used by an optimization objective.
- enum class [ExecutionState](#) {
NOT_READY , **READY** , **RUNNING** , **PAUSED** ,
STOPPED , **FINISHED** , **ERROR** }
Execution state of the optimization process.

15.484.1 Detailed Description

Initial concrete scaffold for an OptQuest-like optimizer.

This class intentionally provides only the public API declaration for the first default implementation.

Current status:

- interface contract declared;
- behavior/algorithm implementation intentionally deferred.

Planned behavior:

- discover controls/responses from an associated model,
- evaluate objective and constraint expressions through parser context,
- execute iterative search preserving top feasible solutions.

15.484.2 Member Function Documentation

15.484.2.1 addConstraint()

```
void OptimizerDefaultImpl1::addConstraint (
    const ConstraintDefinition & constraint ) [override], [virtual]
```

Adds one feasibility constraint expression.

Implements [Optimizer_if](#).

15.484.2.2 addObjective()

```
void OptimizerDefaultImpl1::addObjective (
    const ObjectiveDefinition & objective ) [override], [virtual]
```

Adds one objective to the optimization model.

Implements [Optimizer_if](#).

15.484.2.3 checkReady()

```
bool OptimizerDefaultImpl1::checkReady (
    std::string * message = nullptr ) const [override], [virtual]
```

Validates readiness before running optimization.

Parameters

<i>message</i>	Receives validation details when non-null.
----------------	--

Returns

true when optimizer has enough data/configuration to start.

Implements [Optimizer_if](#).

15.484.2.4 clearConstraints()

```
void OptimizerDefaultImpl1::clearConstraints ( ) [override], [virtual]
```

Clears all configured constraints.

Implements [Optimizer_if](#).

15.484.2.5 clearObjectives()

```
void OptimizerDefaultImpl1::clearObjectives ( ) [override], [virtual]
```

Clears all configured objectives.

Implements [Optimizer_if](#).

15.484.2.6 getAvailableControls()

```
List< SimulationControl * > * OptimizerDefaultImpl1::getAvailableControls ( ) const [override],
[virtual]
```

Retrieves available model controls discoverable from [Model](#).

Implements [Optimizer_if](#).

15.484.2.7 getAvailableResponses()

```
List< SimulationResponse * > * OptimizerDefaultImpl1::getAvailableResponses ( ) const [override],
[virtual]
```

Retrieves available model responses discoverable from [Model](#).

Implements [Optimizer_if](#).

15.484.2.8 getBestSolutions()

```
List< Optimizer_if::SolutionSummary > * OptimizerDefaultImpl1::getBestSolutions ( ) const
[override], [virtual]
```

Returns all feasible solutions retained as "best" by the algorithm.

Implements [Optimizer_if](#).

15.484.2.9 getConstraints()

```
List< Optimizer_if::ConstraintDefinition > * OptimizerDefaultImpl1::getConstraints ( ) const
[override], [virtual]
```

Returns all configured feasibility constraints.

Implements [Optimizer_if](#).

15.484.2.10 getCurrentBestSolution()

```
const Optimizer_if::SolutionSummary * OptimizerDefaultImpl1::getCurrentBestSolution ( ) const
[override], [virtual]
```

Returns the best-known feasible solution, or nullptr when absent.

Implements [Optimizer_if](#).

15.484.2.11 getCurrentIteration()

```
unsigned int OptimizerDefaultImpl1::getCurrentIteration ( ) const [override], [virtual]
```

Returns the number of optimization iterations already performed.

Implements [Optimizer_if](#).

15.484.2.12 getExecutionState()

```
Optimizer_if::ExecutionState OptimizerDefaultImpl1::getExecutionState ( ) const [override],
[virtual]
```

Returns current lifecycle state of the optimizer.

Implements [Optimizer_if](#).

15.484.2.13 getModel()

```
Model * OptimizerDefaultImpl1::getModel ( ) const [override], [virtual]
```

Returns the model currently associated with the optimizer.

Implements [Optimizer_if](#).

15.484.2.14 getModelFilename()

```
std::string OptimizerDefaultImpl1::getModelFilename ( ) const [override], [virtual]
```

Returns the source filename previously configured for model loading.

Implements [Optimizer_if](#).

15.484.2.15 getObjectives()

```
List< Optimizer_if::ObjectiveDefinition > * OptimizerDefaultImpl1::getObjectives ( ) const
[override], [virtual]
```

Returns all configured optimization objectives.

Implements [Optimizer_if](#).

15.484.2.16 getSelectedControls()

```
List< SimulationControl * > * OptimizerDefaultImpl1::getSelectedControls ( ) const [override],
[virtual]
```

Returns currently selected controls used as decision variables.

Implements [Optimizer_if](#).

15.484.2.17 getSelectedResponses()

```
List< SimulationResponse * > * OptimizerDefaultImpl1::getSelectedResponses ( ) const [override], [virtual]
```

Returns currently selected responses monitored by optimization.

Implements [Optimizer_if](#).

15.484.2.18 getSettings()

```
Optimizer_if::OptimizationSettings OptimizerDefaultImpl1::getSettings ( ) const [override], [virtual]
```

Returns currently active optimization settings.

Implements [Optimizer_if](#).

15.484.2.19 getTotalSimulations()

```
unsigned int OptimizerDefaultImpl1::getTotalSimulations ( ) const [override], [virtual]
```

Returns total number of simulated scenarios evaluated so far.

Implements [Optimizer_if](#).

15.484.2.20 pause()

```
bool OptimizerDefaultImpl1::pause ( ) [override], [virtual]
```

Pauses an active optimization execution.

Implements [Optimizer_if](#).

15.484.2.21 resetResults()

```
void OptimizerDefaultImpl1::resetResults ( ) [override], [virtual]
```

Clears results from previous runs while preserving configuration.

Implements [Optimizer_if](#).

15.484.2.22 resume()

```
bool OptimizerDefaultImpl1::resume ( ) [override], [virtual]
```

Resumes optimization execution after a pause.

Implements [Optimizer_if](#).

15.484.2.23 setModel()

```
void OptimizerDefaultImpl1::setModel (
    Model * model ) [override], [virtual]
```

Associates an already instantiated model with this optimizer.

Implements [Optimizer_if](#).

15.484.2.24 setModelFilename()

```
bool OptimizerDefaultImpl1::setModelFilename (
    const std::string & filename ) [override], [virtual]
```

Loads/associates a model from a file supported by the simulator.

Parameters

<i>filename</i>	Model file path.
-----------------	----------------------------------

Returns

true when the model file was successfully loaded and linked.

Implements [Optimizer_if](#).

15.484.2.25 setSelectedControls()

```
void OptimizerDefaultImpl1::setSelectedControls (
    List< SimulationControl * > * controls ) [override], [virtual]
```

Replaces the set of controls selected for optimization.

Implements [Optimizer_if](#).

15.484.2.26 setSelectedResponses()

```
void OptimizerDefaultImpl1::setSelectedResponses (
    List< SimulationResponse * > * responses ) [override], [virtual]
```

Replaces the set of responses selected for optimization.

Implements [Optimizer_if](#).

15.484.2.27 setSettings()

```
void OptimizerDefaultImpl1::setSettings (
    const OptimizationSettings & settings ) [override], [virtual]
```

Applies algorithm settings such as iteration and retention limits.

Implements [Optimizer_if](#).

15.484.2.28 start()

```
bool OptimizerDefaultImpl1::start ( ) [override], [virtual]
```

Starts a new optimization run.

Returns

true on successful transition to RUNNING state.

Implements [Optimizer_if](#).

15.484.2.29 step()

```
bool OptimizerDefaultImpl1::step ( ) [override], [virtual]
```

Executes one optimization iteration (or one scenario decision step).

Returns

true when a step was executed.

Implements [Optimizer_if](#).

15.484.2.30 stop()

```
bool OptimizerDefaultImpl1::stop ( ) [override], [virtual]
```

Stops optimization execution and finalizes current run.

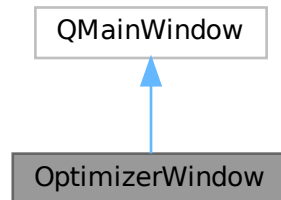
Implements [Optimizer_if](#).

The documentation for this class was generated from the following files:

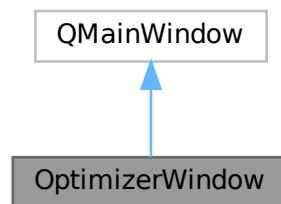
- tools/Optimization/OptimizerDefaultImpl1.h
- tools/Optimization/OptimizerDefaultImpl1.cpp

15.485 OptimizerWindow Class Reference

Inheritance diagram for OptimizerWindow:



Collaboration diagram for OptimizerWindow:



Classes

- class [ProgressPlot](#)

Public Member Functions

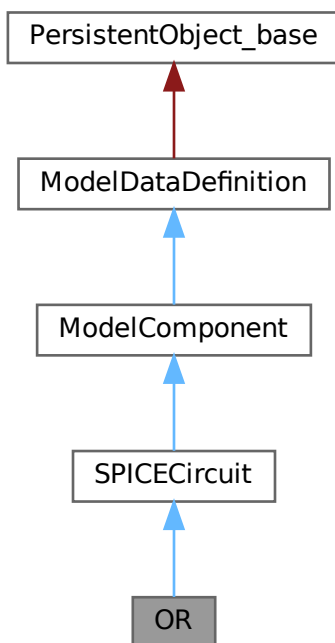
- **OptimizerWindow** ([Simulator](#) *simulator, QWidget *parent=nullptr)

The documentation for this class was generated from the following files:

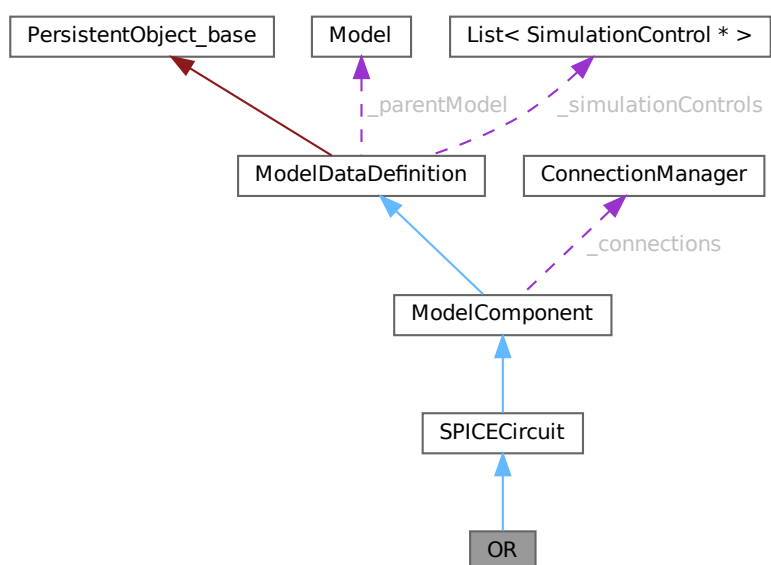
- applications/gui/genesys/tools/optimizer/OptimizerWindow.h
- applications/gui/genesys/tools/optimizer/OptimizerWindow.cpp

15.486 OR Class Reference

Inheritance diagram for OR:



Collaboration diagram for OR:



Public Member Functions

- **OR** ([Model](#) *model, std::string name="")
- void **Build** () override
- void **setPMOSWidth** (float PMOSWidth)
- void **setPMOSLength** (float length)
- void **setNMOSWidth** (float NMOSWidth)
- void **setNMOSLength** (float NMOSLength)
- void **setElectricalModel** (std::string electricalModel)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

- Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation](#) * [GetPluginInformation](#) ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
- static std::vector< std::string > [split](#) (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from **ModelComponent**

- static **ModelComponent** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** (**ModelComponent** *component)
Validates a component instance.
- static void **CreateInternalData** (**ModelComponent** *component)
Creates internal data required by a component instance.
- static void **SaveInstance** (**PersistenceRecord** *fields, **ModelComponent** *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** (**Event** *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from **ModelDataDefinition**

- static **ModelDataDefinition** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** (**PersistenceRecord** *fields, **ModelDataDefinition** *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** (**ModelDataDefinition** *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** (**ModelDataDefinition** *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** (**ModelDataDefinition** *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** (**ModelDataDefinition** *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** (**PersistenceRecord** *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** (**PersistenceRecord** *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from **SPICECircuit**

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void **_onDispatchEvent** (**Entity** *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void **_createNonEditableDataDefinitions** () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.486.1 Member Function Documentation

15.486.1.1 [_loadInstance\(\)](#)

```
bool OR::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [SPICECircuit](#).

15.486.1.2 [_saveInstance\(\)](#)

```
void OR::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

15.486.1.3 [Build\(\)](#)

```
void OR::Build ( ) [override], [virtual]
```

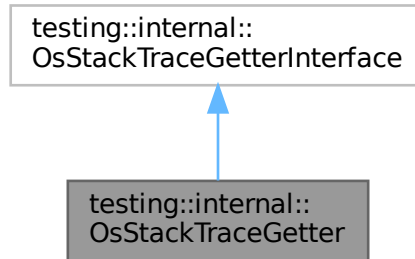
Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

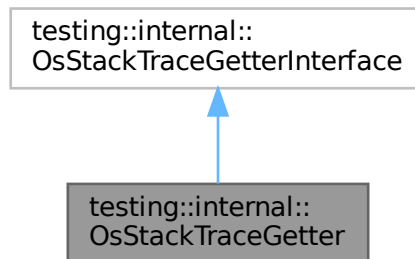
- plugins/components/ElectronicsSimulation/SPICECircuit.h
- plugins/components/ElectronicsSimulation/SPICECircuit.cpp

15.487 testing::internal::OsStackTraceGetter Class Reference

Inheritance diagram for testing::internal::OsStackTraceGetter:



Collaboration diagram for testing::internal::OsStackTraceGetter:



Public Member Functions

- std::string [CurrentStackTrace](#) (int max_depth, int skip_count) override
- void [UponLeavingGTest](#) () override

Additional Inherited Members

Static Public Attributes inherited from [testing::internal::OsStackTraceGetterInterface](#)

- static const char *const [kElidedFramesMarker](#)

15.487.1 Member Function Documentation

15.487.1.1 CurrentStackTrace()

```

std::string testing::internal::OsStackTraceGetter::CurrentStackTrace (
    int max_depth,
    int skip_count ) [override], [virtual]

```

Implements [testing::internal::OsStackTraceGetterInterface](#).

15.487.1.2 UponLeavingGTest()

void testing::internal::OsStackTraceGetter::UponLeavingGTest () [override], [virtual]

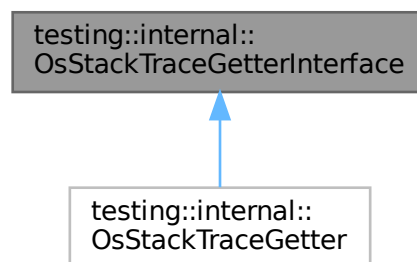
Implements [testing::internal::OsStackTraceGetterInterface](#).

The documentation for this class was generated from the following files:

- gtest/src/gtest-internal-inl.h
- gtest/src/gtest.cc

15.488 testing::internal::OsStackTraceGetterInterface Class Reference

Inheritance diagram for testing::internal::OsStackTraceGetterInterface:



Public Member Functions

- virtual std::string **CurrentStackTrace** (int max_depth, int skip_count)=0
- virtual void **UponLeavingGTest** ()=0

Static Public Attributes

- static const char *const [kElidedFramesMarker](#)

15.488.1 Member Data Documentation

15.488.1.1 kElidedFramesMarker

const char *const testing::internal::OsStackTraceGetterInterface::kElidedFramesMarker [static]

Initial value:

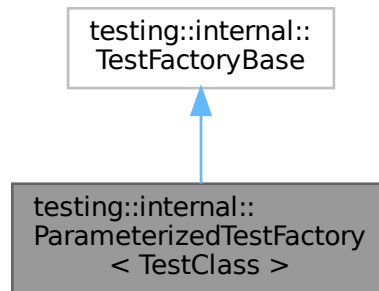
```
=
    "... " GTEST_NAME_ " internal frames ..."
```

The documentation for this class was generated from the following files:

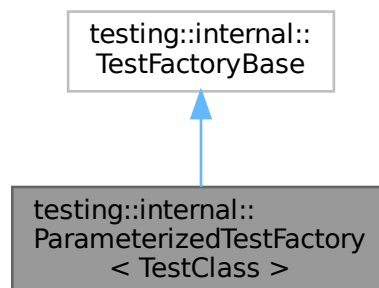
- gtest/src/gtest-internal-inl.h
- gtest/src/gtest.cc

15.489 testing::internal::ParameterizedTestFactory< TestClass > Class Template Reference

Inheritance diagram for testing::internal::ParameterizedTestFactory< TestClass >:



Collaboration diagram for testing::internal::ParameterizedTestFactory< TestClass >:



Public Types

- typedef TestClass::ParamType **ParamType**

Public Member Functions

- **ParameterizedTestFactory** (ParamType parameter)
- [Test](#) * [CreateTest](#) () override

15.489.1 Member Function Documentation

15.489.1.1 CreateTest()

```

template<class TestClass >
Test * testing::internal::ParameterizedTestFactory< TestClass >::CreateTest ( ) [inline],
[override], [virtual]
  
```

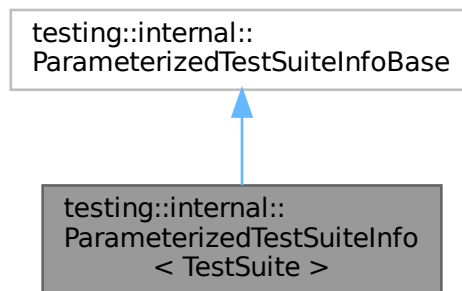
Implements [testing::internal::TestFactoryBase](#).

The documentation for this class was generated from the following file:

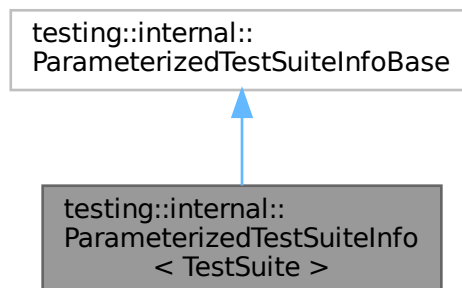
- gtest/gtest/internal/gtest-param-util.h

15.490 testing::internal::ParameterizedTestSuiteInfo< TestSuite > Class Template Reference

Inheritance diagram for testing::internal::ParameterizedTestSuiteInfo< TestSuite >:



Collaboration diagram for testing::internal::ParameterizedTestSuiteInfo< TestSuite >:



Public Types

- using **ParamType** = typename TestSuite::ParamType
- using **ParamNameGeneratorFunc** = std::string(const [TestParamInfo](#)< ParamType > &)

Public Member Functions

- typedef **ParamGenerator** (GeneratorCreationFunc)()
- **ParameterizedTestSuiteInfo** (const char *name, [CodeLocation](#) code_location)
- const std::string & [GetTestSuiteName](#) () const override

- Typeld [GetTestSuiteTypeId](#) () const override
- void **AddTestPattern** (const char *test_suite_name, const char *test_base_name, [TestMetaFactoryBase](#)< ParamType > *meta_factory, [CodeLocation](#) code_location)
- int **AddTestSuiteInstantiation** (const std::string &instantiation_name, GeneratorCreationFunc *func, ParamNameGeneratorFunc *name_func, const char *file, int line)
- void [RegisterTests](#) () override

15.490.1 Member Function Documentation

15.490.1.1 GetTestSuiteName()

```
template<class TestSuite >
const std::string & testing::internal::ParameterizedTestSuiteInfo< TestSuite >::GetTestSuiteName ( ) const [inline], [override], [virtual]
Implements testing::internal::ParameterizedTestSuiteInfoBase.
```

15.490.1.2 GetTestSuiteTypeId()

```
template<class TestSuite >
TypeId testing::internal::ParameterizedTestSuiteInfo< TestSuite >::GetTestSuiteTypeId ( ) const [inline], [override], [virtual]
Implements testing::internal::ParameterizedTestSuiteInfoBase.
```

15.490.1.3 RegisterTests()

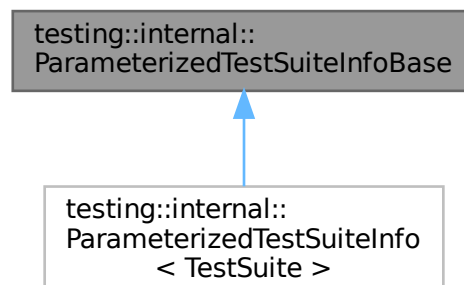
```
template<class TestSuite >
void testing::internal::ParameterizedTestSuiteInfo< TestSuite >::RegisterTests ( ) [inline], [override], [virtual]
Implements testing::internal::ParameterizedTestSuiteInfoBase.
```

The documentation for this class was generated from the following file:

- gtest/gtest/internal/gtest-param-util.h

15.491 testing::internal::ParameterizedTestSuiteInfoBase Class Reference

Inheritance diagram for testing::internal::ParameterizedTestSuiteInfoBase:



Public Member Functions

- virtual const std::string & **GetTestSuiteName** () const =0
- virtual TypeId **GetTestSuiteTypeId** () const =0
- virtual void **RegisterTests** ()=0

The documentation for this class was generated from the following file:

- gtest/gtest/internal/gtest-param-util.h

15.492 testing::internal::ParameterizedTestSuiteRegistry Class Reference

Public Member Functions

- template<class [TestSuite](#) >
[ParameterizedTestSuiteInfo](#)< [TestSuite](#) > * **GetTestSuitePatternHolder** (const char *test_suite_name, [CodeLocation](#) code_location)
- void **RegisterTests** ()
- template<class [TestCase](#) >
[ParameterizedTestCaseInfo](#)< [TestCase](#) > * **GetTestCasePatternHolder** (const char *test_case_name, [CodeLocation](#) code_location)

The documentation for this class was generated from the following file:

- gtest/gtest/internal/gtest-param-util.h

15.493 testing::internal::ParamGenerator< T > Class Template Reference

Public Types

- typedef [ParamIterator](#)< T > **iterator**

Public Member Functions

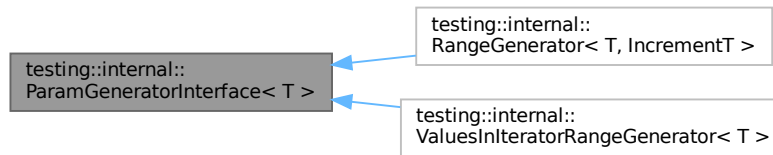
- **ParamGenerator** ([ParamGeneratorInterface](#)< T > *impl)
- **ParamGenerator** (const [ParamGenerator](#) &other)
- [ParamGenerator](#) & **operator=** (const [ParamGenerator](#) &other)
- [iterator](#) **begin** () const
- [iterator](#) **end** () const

The documentation for this class was generated from the following file:

- gtest/gtest/internal/gtest-param-util.h

15.494 testing::internal::ParamGeneratorInterface< T > Class Template Reference

Inheritance diagram for testing::internal::ParamGeneratorInterface< T >:



Public Types

- typedef T **ParamType**

Public Member Functions

- virtual [ParamIteratorInterface](#)< T > * **Begin** () const =0
- virtual [ParamIteratorInterface](#)< T > * **End** () const =0

The documentation for this class was generated from the following file:

- gtest/gtest/internal/gtest-param-util.h

15.495 testing::internal::ParamIterator< T > Class Template Reference

Public Types

- typedef T **value_type**
- typedef const T & **reference**
- typedef ptrdiff_t **difference_type**

Public Member Functions

- **ParamIterator** (const [ParamIterator](#) &other)
- [ParamIterator](#) & **operator=** (const [ParamIterator](#) &other)
- const T & **operator*** () const
- const T * **operator->** () const
- [ParamIterator](#) & **operator++** ()
- [ParamIterator](#) **operator++** (int)
- bool **operator==** (const [ParamIterator](#) &other) const
- bool **operator!=** (const [ParamIterator](#) &other) const

Friends

- class **ParamGenerator**< T >

The documentation for this class was generated from the following file:

- gtest/gtest/internal/gtest-param-util.h

15.496 testing::internal::ParamIteratorInterface< T > Class Template Reference

Public Member Functions

- virtual const [ParamGeneratorInterface](#)< T > * **BaseGenerator** () const =0
- virtual void **Advance** ()=0
- virtual [ParamIteratorInterface](#) * **Clone** () const =0
- virtual const T * **Current** () const =0
- virtual bool **Equals** (const [ParamIteratorInterface](#) &other) const =0

The documentation for this class was generated from the following file:

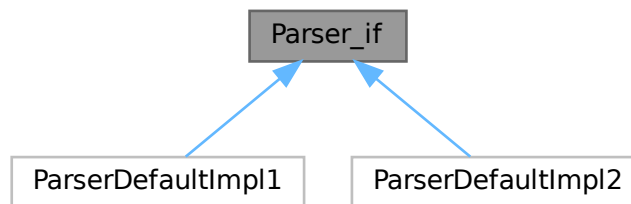
- gtest/gtest/internal/gtest-param-util.h

15.497 Parser_if Class Reference

Interface for evaluating model expressions and stochastic functions.

```
#include <kernel/simulator/Parser_if.h>
```

Inheritance diagram for Parser_if:



Public Member Functions

- virtual double [parse](#) (const std::string expression)=0
parse
- virtual double [parse](#) (const std::string expression, bool &success, std::string &errorMessage)=0
parse
- virtual std::string [getErrorMessage](#) ()=0
getErrorMessage
- virtual void [setSampler](#) ([Sampler_if](#) *sampler)=0
Sets the sampler used by stochastic language functions.
- virtual [Sampler_if](#) * [getSampler](#) () const =0
Returns the sampler currently associated with the parser.
- virtual [genesyspp_driver](#) [getParser](#) () const =0
getParser

15.497.1 Detailed Description

Interface for evaluating model expressions and stochastic functions.

Parser implementations are used by model components/data definitions to evaluate arithmetic/logical expressions, integrating deterministic symbols and stochastic calls backed by a [Sampler_if](#) instance.

15.497.2 Member Function Documentation

15.497.2.1 getErrorMessage()

```
virtual std::string Parser_if::getErrorMessage ( ) [pure virtual]
getErrorMessage
```

Returns

Returns the last error message produced by the parser.
Implemented in [ParserDefaultImpl1](#), and [ParserDefaultImpl2](#).

15.497.2.2 getParser()

```
virtual genesyspp_driver Parser_if::getParser ( ) const [pure virtual]
getParser
```

Returns

Exposes the internal parser driver for advanced integration scenarios.
Implemented in [ParserDefaultImpl2](#).

15.497.2.3 getSampler()

```
virtual Sampler_if * Parser_if::getSampler ( ) const [pure virtual]
```

Returns the sampler currently associated with the parser.
The returned pointer is non-owning. Callers must consult concrete parser documentation for lifetime/ownership details of the underlying instance.
Implemented in [ParserDefaultImpl2](#).

15.497.2.4 parse() [1/2]

```
virtual double Parser_if::parse (
    const std::string expression ) [pure virtual]
```

parse

Parameters

<i>expression</i>	
-------------------	--

Returns

Evaluates an expression and returns its numeric value; may throw on syntax/semantic errors.
Implemented in [ParserDefaultImpl1](#), and [ParserDefaultImpl2](#).

15.497.2.5 parse() [2/2]

```
virtual double Parser_if::parse (
    const std::string expression,
    bool & success,
    std::string & errorMessage ) [pure virtual]
```

parse

Parameters

<i>expression</i>	
-------------------	--

Parameters

<i>success</i>	
<i>errorMessage</i>	

Returns

Evaluates an expression without throwing and reports status via output parameters.
Implemented in [ParserDefaultImpl1](#), and [ParserDefaultImpl2](#).

15.497.2.6 setSampler()

```
virtual void Parser_if::setSampler (
    Sampler\_if * sampler ) [pure virtual]
```

Sets the sampler used by stochastic language functions.

Parameters

<i>sampler</i>	Pointer to sampler instance that must outlive parser usage unless an implementation-specific ownership transfer is explicitly documented.
----------------	---

This interface does not enforce ownership transfer. Concrete parser implementations may own a sampler created/provided at construction time, but samplers passed through `setSampler` are expected to be externally owned by default unless documented otherwise by the implementation.

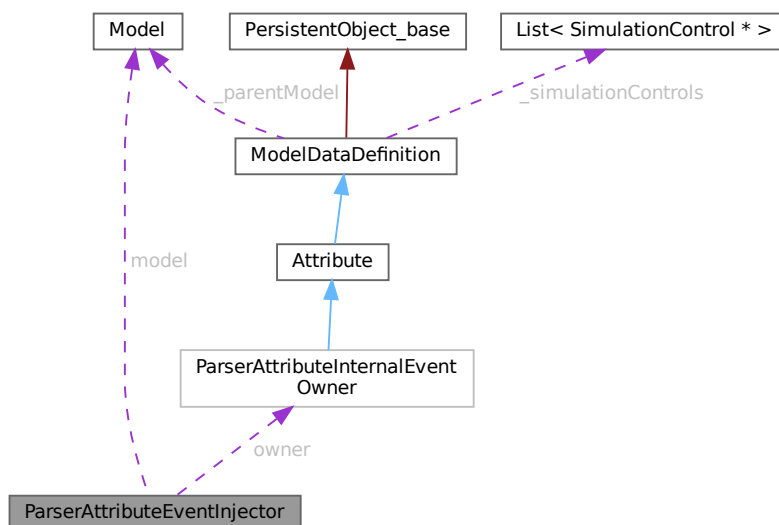
Implemented in [ParserDefaultImpl2](#).

The documentation for this class was generated from the following file:

- `kernel/simulator/Parser_if.h`

15.498 ParserAttributeEventInjector Struct Reference

Collaboration diagram for ParserAttributeEventInjector:



Public Member Functions

- void **OnReplicationStart** ([SimulationEvent](#) *)

Public Attributes

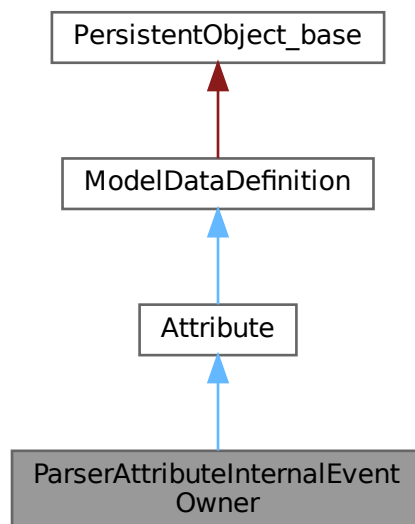
- [Model](#) * **model** = nullptr
- [ParserAttributeInternalEventOwner](#) * **owner** = nullptr
- std::string **attributeName** = "ParserAttrND"
- bool **inserted** = false

The documentation for this struct was generated from the following file:

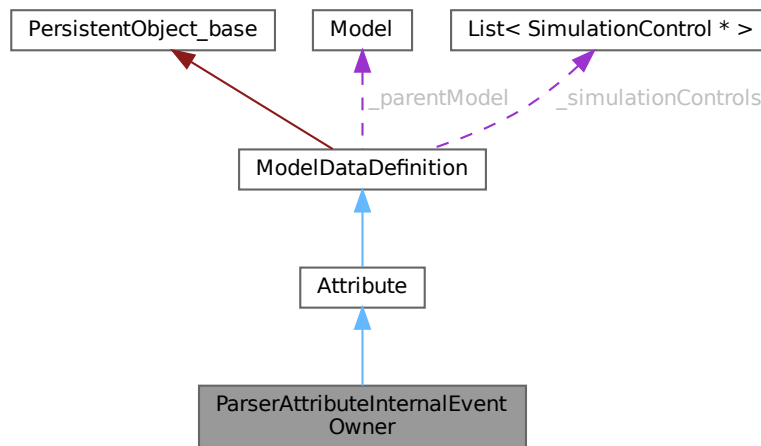
- tests/unit/test_parser_expressions.cpp

15.499 ParserAttributeInternalEventOwner Class Reference

Inheritance diagram for ParserAttributeInternalEventOwner:



Collaboration diagram for ParserAttributeInternalEventOwner:



Public Member Functions

- **ParserAttributeInternalEventOwner** ([Model](#) *model, const std::string &name="")
- void **Noop** (void *)

Public Member Functions inherited from [Attribute](#)

- **Attribute** ([Model](#) *model, std::string name="", std::string dataDefinitionType=Util::TypeOf< [Attribute](#) >())
Creates an entity attribute definition in the model.
- virtual ~**Attribute** () override
Releases sparse initial values owned by this definition.
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- virtual std::string **getInitialValuesText** () const
Returns the initial values serialized in bracket notation.
- virtual void **setInitialValuesText** (std::string valuesText)
Parses bracket notation and replaces the initial value store.
- double **getInitialValue** (std::string index="")
Reads the initial sparse value, returning 0.0 when the index is absent.
- void **setInitialValue** (double value, std::string index="")
Writes the initial sparse value at the scalar or indexed position.
- void **setInitialValues** (const std::vector< std::pair< std::string, double > > values)
Replaces/extends initial sparse values from textual index/value pairs.
- void **copyInitialValuesFrom** (const [Attribute](#) &source)
Copies the full initial sparse store from another attribute.
- void **insertDimentionSize** (unsigned int size)
Appends one dimension size to the attribute definition.
- std::list< unsigned int > * **getDimensionSizes** () const
Returns dimension sizes for compatibility with existing callers.
- std::map< std::string, double > * **getInitialValues** () const
Returns mutable initial sparse values for compatibility with existing callers.

- [SparseValueStore](#) * **getInitialValueStore** ()
Returns the sparse initial value store used by this attribute.
- bool **isInitialValuesTextValid** () const
Returns whether the last textual initial value is syntactically valid.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Additional Inherited Members

Static Public Member Functions inherited from [Attribute](#)

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions inherited from [Attribute](#)

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()

- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internaStatisticReportersClear** ()
- legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
- method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()

- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

The documentation for this class was generated from the following file:

- tests/unit/test_parser_expressions.cpp

15.500 ParserAttributeProcessObserver Struct Reference

Public Member Functions

- void **OnProcessEvent** ([SimulationEvent](#) *event)

Public Attributes

- std::function< void([Entity](#) *)> **onProcess**

The documentation for this struct was generated from the following file:

- tests/unit/test_parser_expressions.cpp

15.501 ParserChangesInformation Class Reference

Public Member Functions

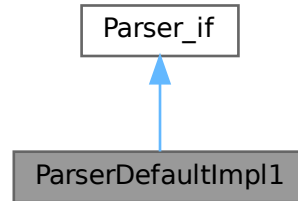
- std::string **getincludes** () const
- void **setincludes** (const std::string &newIncludes)
- std::string **gettokens** () const
- void **setTokens** (const std::string &newTokens)
- std::string **gettypeObjs** () const
- void **setTypeObjs** (const std::string &newTypeObjs)
- std::string **getexpressions** () const
- void **setExpressions** (const std::string &newExpressions)
- std::string **getexpressionProductions** () const
- void **setExpressionProductions** (const std::string &newExpressionProductions)
- std::string **getassignments** () const
- void **setAssignments** (const std::string &newAssignments)
- std::string **getfunctionProductions** () const
- void **setFunctionProductions** (const std::string &newFunctionProductions)

The documentation for this class was generated from the following files:

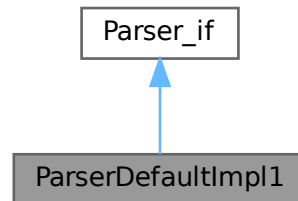
- kernel/simulator/ParserChangesInformation.h
- kernel/simulator/ParserChangesInformation.cpp

15.502 ParserDefaultImpl1 Class Reference

Inheritance diagram for ParserDefaultImpl1:



Collaboration diagram for ParserDefaultImpl1:



Public Member Functions

- **ParserDefaultImpl1** ([Model](#) *model)
- virtual double [parse](#) (const std::string expression) override
parse
- virtual double [parse](#) (const std::string expression, bool &success, std::string &errorMessage) override
parse
- virtual std::string [getErrorMessage](#) () override
getErrorMessage

Public Member Functions inherited from [Parser_if](#)

- virtual void [setSampler](#) ([Sampler_if](#) *sampler)=0
Sets the sampler used by stochastic language functions.
- virtual [Sampler_if](#) * [getSampler](#) () const =0
Returns the sampler currently associated with the parser.
- virtual [genesyspp_driver](#) [getParser](#) () const =0
getParser

15.502.1 Member Function Documentation

15.502.1.1 getErrorMessage()

```
std::string ParserDefaultImpl1::getErrorMessage ( ) [override], [virtual]  
getErrorMessage
```

Returns

Returns the last error message produced by the parser.
Implements [Parser_if](#).

15.502.1.2 parse() [1/2]

```
double ParserDefaultImpl1::parse (   
    const std::string expression ) [override], [virtual]  
parse
```

Parameters

<i>expression</i>	
-------------------	--

Returns

Evaluates an expression and returns its numeric value; may throw on syntax/semantic errors.
Implements [Parser_if](#).

15.502.1.3 parse() [2/2]

```
double ParserDefaultImpl1::parse (   
    const std::string expression,  
    bool & success,  
    std::string & errorMessage ) [override], [virtual]  
parse
```

Parameters

<i>expression</i>	
<i>success</i>	
<i>errorMessage</i>	

Returns

Evaluates an expression without throwing and reports status via output parameters.

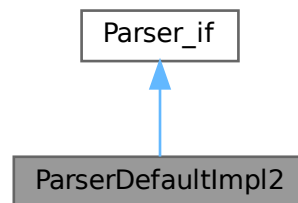
Implements [Parser_if](#).

The documentation for this class was generated from the following files:

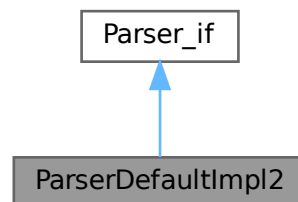
- kernel/simulator/ParserDefaultImpl1.h
- kernel/simulator/ParserDefaultImpl1.cpp

15.503 ParserDefaultImpl2 Class Reference

Inheritance diagram for ParserDefaultImpl2:



Collaboration diagram for ParserDefaultImpl2:



Public Member Functions

- [ParserDefaultImpl2](#) ([Model](#) *model, [Sampler_if](#) *sampler, bool throws=false)
Creates parser wrapper bound to a model and an initial sampler.
- virtual [~ParserDefaultImpl2](#) ()
Destroys parser wrapper and releases only parser-owned sampler.
- virtual double [parse](#) (const std::string expression) override
parse
- virtual double [parse](#) (const std::string expression, bool &success, std::string &errorMessage) override
parse
- virtual std::string [getErrorMessage](#) () override

getErrorMessage

- virtual void [setSampler](#) ([Sampler_if](#) * _sampler) override
Replaces parser sampler with an externally-owned sampler.
- virtual [Sampler_if](#) * [getSampler](#) () const override
Returns the sampler currently associated with the parser.
- virtual [genesyspp_driver](#) [getParser](#) () const override
getParser

15.503.1 Constructor & Destructor Documentation

15.503.1.1 ParserDefaultImpl2()

```
ParserDefaultImpl2::ParserDefaultImpl2 (
    Model * model,
    Sampler_if * sampler,
    bool throws = false )
```

Creates parser wrapper bound to a model and an initial sampler.

The constructor stores `sampler` in the internal driver and marks it as parser-owned when non-null. This preserves current model behavior where the parser starts with an internally managed sampler instance.

15.503.1.2 ~ParserDefaultImpl2()

```
ParserDefaultImpl2::~~ParserDefaultImpl2 ( ) [virtual]
```

Destroys parser wrapper and releases only parser-owned sampler.

The destructor deletes the current sampler only when ownership is flagged as internal. Samplers installed through `setSampler(...)` are non-owned by default and therefore not deleted at destruction time.

15.503.2 Member Function Documentation

15.503.2.1 getErrorMessage()

```
std::string ParserDefaultImpl2::getErrorMessage ( ) [override], [virtual]
getErrorMessage
```

Returns

Returns the last error message produced by the parser.

Implements [Parser_if](#).

15.503.2.2 getParser()

```
genesyspp\_driver ParserDefaultImpl2::getParser ( ) const [override], [virtual]
getParser
```

Returns

Exposes the internal parser driver for advanced integration scenarios.

Implements [Parser_if](#).

15.503.2.3 getSampler()

```
Sampler\_if * ParserDefaultImpl2::getSampler ( ) const [override], [virtual]
```

Returns the sampler currently associated with the parser.

The returned pointer is non-owning. Callers must consult concrete parser documentation for lifetime/ownership details of the underlying instance.

Implements [Parser_if](#).

15.503.2.4 parse() [1/2]

```
double ParserDefaultImpl2::parse (
    const std::string expression ) [override], [virtual]
```

parse

Parameters

<i>expression</i>	
-------------------	--

Returns

Evaluates an expression and returns its numeric value; may throw on syntax/semantic errors.
Implements [Parser_if](#).

15.503.2.5 parse() [2/2]

```
double ParserDefaultImpl2::parse (
    const std::string expression,
    bool & success,
    std::string & errorMessage ) [override], [virtual]
```

parse

Parameters

<i>expression</i>	
<i>success</i>	
<i>errorMessage</i>	

Returns

Evaluates an expression without throwing and reports status via output parameters.
Implements [Parser_if](#).

15.503.2.6 setSampler()

```
void ParserDefaultImpl2::setSampler (
    Sampler\_if * _sampler ) [override], [virtual]
```

Replaces parser sampler with an externally-owned sampler.

If the parser currently owns a previous sampler, that previous sampler is released before binding the new pointer.

The provided sampler pointer is then tracked as non-owned.

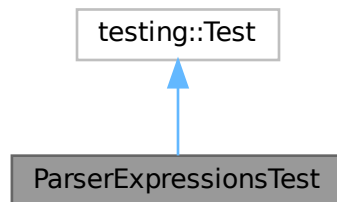
Implements [Parser_if](#).

The documentation for this class was generated from the following files:

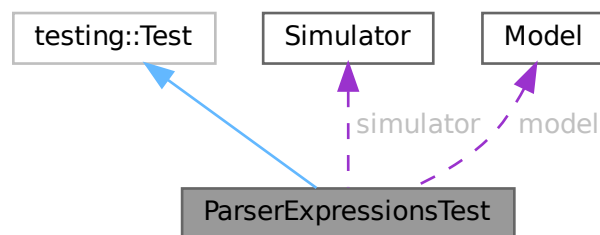
- kernel/simulator/ParserDefaultImpl2.h
- kernel/simulator/ParserDefaultImpl2.cpp

15.504 ParserExpressionsTest Class Reference

Inheritance diagram for ParserExpressionsTest:



Collaboration diagram for ParserExpressionsTest:



Protected Member Functions

- void [SetUp](#) () override

Protected Member Functions inherited from [testing::Test](#)

- virtual void **TearDown** ()

Protected Attributes

- [Simulator](#) **simulator**
- [Model](#) * **model** = nullptr

Additional Inherited Members

Static Public Member Functions inherited from [testing::Test](#)

- static void **SetUpTestSuite** ()
- static void **TearDownTestSuite** ()
- static void **TearDownTestCase** ()
- static void **SetUpTestCase** ()
- static bool **HasFatalFailure** ()

- static bool **HasNonfatalFailure** ()
- static bool **IsSkipped** ()
- static bool **HasFailure** ()
- static void **RecordProperty** (const std::string &key, const std::string &value)
- static void **RecordProperty** (const std::string &key, int value)

15.504.1 Member Function Documentation

15.504.1.1 SetUp()

void ParserExpressionsTest::SetUp () [inline], [override], [protected], [virtual]

Reimplemented from [testing::Test](#).

The documentation for this class was generated from the following file:

- tests/unit/test_parser_expressions.cpp

15.505 ParserManager Class Reference

Classes

- struct [GenerateNewParserResult](#)
Result bundle for parser generation requests.
- struct [NewParser](#)
Paths associated with a generated parser artifact set.

Public Member Functions

- [ParserManager::GenerateNewParserResult](#) [generateNewParser](#) ([ParserChangesInformation](#) *changes)
generateNewParser
- bool [connectNewParser](#) ([ParserManager::NewParser](#) newParser)
connectNewParser

15.505.1 Member Function Documentation

15.505.1.1 connectNewParser()

```
bool ParserManager::connectNewParser (
    ParserManager::NewParser newParser )
connectNewParser
```

Parameters

<i>newParser</i>	
------------------	--

Returns

15.505.1.2 generateNewParser()

```
ParserManager::GenerateNewParserResult ParserManager::generateNewParser (
    ParserChangesInformation * changes )
generateNewParser
```

Parameters

<i>changes</i>	
----------------	--

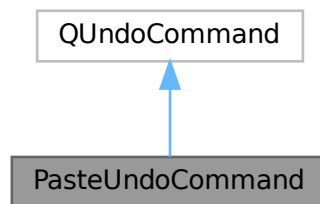
Returns

The documentation for this class was generated from the following files:

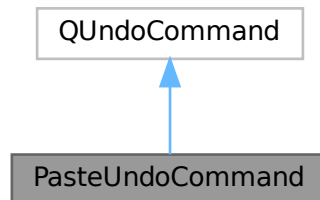
- kernel/simulator/ParserManager.h
- kernel/simulator/ParserManager.cpp

15.506 PasteUndoCommand Class Reference

Inheritance diagram for PasteUndoCommand:



Collaboration diagram for PasteUndoCommand:



Public Member Functions

- **PasteUndoCommand** (QList< [GraphicalModelComponent](#) * > *graphicalComponents, QList< [GraphicalConnection](#) * > *connections, QList< QGraphicsItemGroup * > *groups, QList< QGraphicsItem * > *drawing, [ModelGraphicsScene](#) *scene, QUndoCommand *parent=nullptr)
- void **undo** () override
- void **redo** () override

The documentation for this class was generated from the following files:

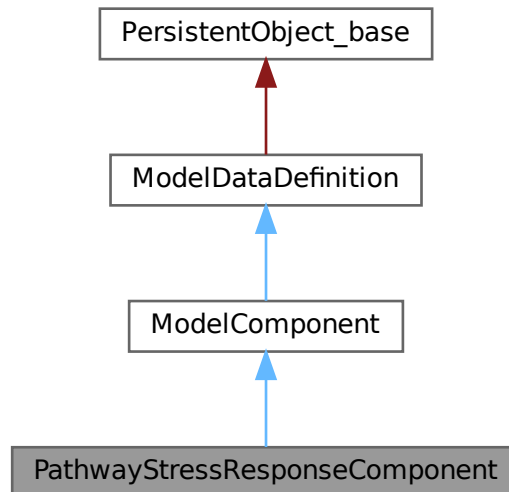
- applications/gui/genesys/actions/PasteUndoCommand.h
- applications/gui/genesys/actions/PasteUndoCommand.cpp

15.507 PathwayStressResponseComponent Class Reference

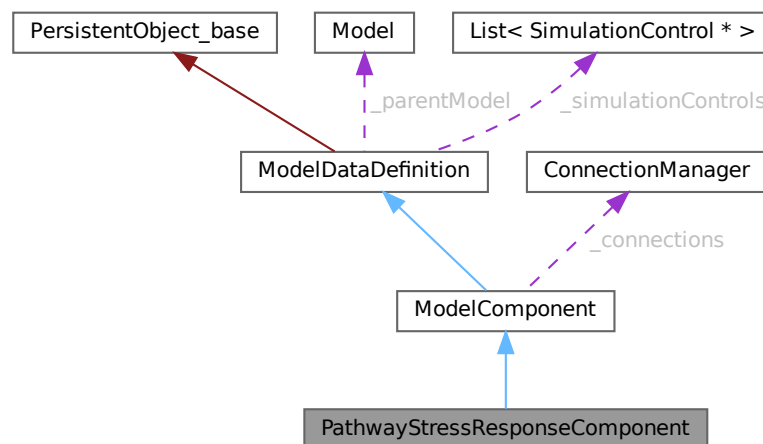
Applies sustained pathway-collapse responses to a whole-cell state.

```
#include <plugins/components/WholeCellModeling/PathwayStressResponseComponent.h>
```

Inheritance diagram for PathwayStressResponseComponent:



Collaboration diagram for PathwayStressResponseComponent:



Public Member Functions

- **PathwayStressResponseComponent** ([Model](#) *model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const

- void **setMonitoredPathwayKey** (std::string key)
- std::string **getMonitoredPathwayKey** () const
- void **setStressThreshold** (double threshold)
- double **getStressThreshold** () const
- void **setArrestAfterSteps** (unsigned int steps)
- unsigned int **getArrestAfterSteps** () const
- void **setDeathAfterSteps** (unsigned int steps)
- unsigned int **getDeathAfterSteps** () const
- void **setArrestPhase** (std::string phase)
- std::string **getArrestPhase** () const
- void **setDeadPhase** (std::string phase)
- std::string **getDeadPhase** () const
- void **setRecoveryPhase** (std::string phase)
- std::string **getRecoveryPhase** () const
- void **setResetStreakOnRecovery** (bool reset)
- bool **getResetStreakOnRecovery** () const
- unsigned int **getStressStreak** () const
- double **getLastObservedActivity** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~[ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.

- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()

- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
- legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
- method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
- virtual protected methods that could be overridden by derived classes, if needed*

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.507.1 Detailed Description

Applies sustained pathway-collapse responses to a whole-cell state.

[PathwayStressResponseComponent](#) watches one pathway activity proxy already stored in [WholeCellState](#) and converts repeated low-activity windows into higher-level lifecycle responses such as arrest and death.

The component is intentionally simple:

- it does not solve metabolism;
- it does not schedule time;
- it does not route entities.

Instead, it acts as a dedicated biological stress-response layer that can be placed between checkpointing and fate-routing components in hybrid whole-cell examples.

15.507.2 Member Function Documentation

15.507.2.1 `_check()`

```
bool PathwayStressResponseComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.507.2.2 `_createEditableDataDefinitions()`

```
void PathwayStressResponseComponent::_createEditableDataDefinitions ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.507.2.3 `_initBetweenReplications()`

```
void PathwayStressResponseComponent::_initBetweenReplications ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.507.2.4 `_loadInstance()`

```
bool PathwayStressResponseComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.507.2.5 _onDispatchEvent()

```
void PathwayStressResponseComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.507.2.6 _saveInstance()

```
void PathwayStressResponseComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.507.2.7 show()

```
std::string PathwayStressResponseComponent::show ( ) [override], [virtual]
virtual public methods
```

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

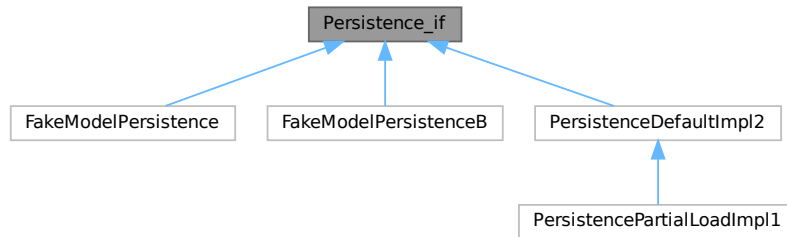
- plugins/components/WholeCellModeling/PathwayStressResponseComponent.h
- plugins/components/WholeCellModeling/PathwayStressResponseComponent.cpp

15.508 Persistence_if Class Reference

Interface for model serialization/deserialization services.

```
#include <kernel/simulator/persistence/Persistence_if.h>
```

Inheritance diagram for Persistence_if:



Public Types

- enum class **Options** : int {
SAVEDEFAULTS = 1 , **HIDEIDKEY** = 2 , **HIDETYPEKEY** = 4 , **HIDENAMEKEY** = 8 ,
SORTALPHLY = 16 }

Public Member Functions

- virtual bool **save** (std::string filename)=0
Serializes and saves the model to the provided filename.
- virtual bool **load** (std::string filename)=0
Loads and deserializes a model from file.
- virtual bool **hasChanged** ()=0
Indicates whether persistent state changed since the last save/load.
- virtual void **setHasChanged** (bool hasChanged)=0
Updates the persistent-state changed flag.
- virtual bool **getOption** (Persistence_if::Options option)=0
Returns whether a persistence option is currently enabled.
- virtual void **setOption** (Persistence_if::Options option, bool value)=0
Enables or disables a persistence option.
- virtual std::string **getFormattedField** (PersistenceRecord *fields)=0
Formats a set of fields as text according to the persistence format.

15.508.1 Detailed Description

Interface for model serialization/deserialization services.

Persistence implementations convert in-memory model structures to/from external representations (e.g., GEN/↔ XML/JSON-like formats) and expose formatting/options hooks used by the kernel persistence pipeline.

15.508.2 Member Function Documentation

15.508.2.1 getFormattedField()

```
virtual std::string Persistence_if::getFormattedField (
    PersistenceRecord * fields ) [pure virtual]
```

Formats a set of fields as text according to the persistence format.

Implemented in [FakeModelPersistence](#), [FakeModelPersistenceB](#), and [PersistenceDefaultImpl2](#).

15.508.2.2 `getOption()`

```
virtual bool Persistence_if::getOption (
    Persistence_if::Options option ) [pure virtual]
```

Returns whether a persistence option is currently enabled.

Implemented in [PersistenceDefaultImpl2](#), [FakeModelPersistence](#), and [FakeModelPersistenceB](#).

15.508.2.3 `hasChanged()`

```
virtual bool Persistence_if::hasChanged ( ) [pure virtual]
```

Indicates whether persistent state changed since the last save/load.

Implemented in [PersistenceDefaultImpl2](#), [FakeModelPersistence](#), and [FakeModelPersistenceB](#).

15.508.2.4 `load()`

```
virtual bool Persistence_if::load (
    std::string filename ) [pure virtual]
```

Loads and deserializes a model from file.

Implemented in [PersistenceDefaultImpl2](#), [PersistencePartialLoadImpl1](#), [FakeModelPersistence](#), and [FakeModelPersistenceB](#).

15.508.2.5 `save()`

```
virtual bool Persistence_if::save (
    std::string filename ) [pure virtual]
```

Serializes and saves the model to the provided filename.

Implemented in [PersistenceDefaultImpl2](#), [FakeModelPersistence](#), and [FakeModelPersistenceB](#).

15.508.2.6 `setHasChanged()`

```
virtual void Persistence_if::setHasChanged (
    bool hasChanged ) [pure virtual]
```

Updates the persistent-state changed flag.

Implemented in [PersistenceDefaultImpl2](#), [FakeModelPersistence](#), and [FakeModelPersistenceB](#).

15.508.2.7 `setOption()`

```
virtual void Persistence_if::setOption (
    Persistence_if::Options option,
    bool value ) [pure virtual]
```

Enables or disables a persistence option.

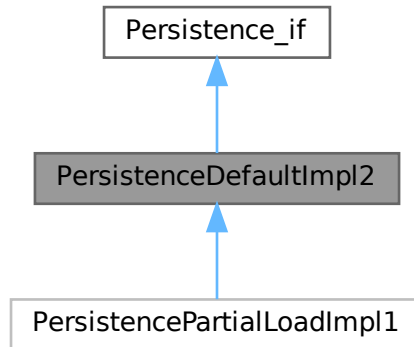
Implemented in [PersistenceDefaultImpl2](#), [FakeModelPersistence](#), and [FakeModelPersistenceB](#).

The documentation for this class was generated from the following file:

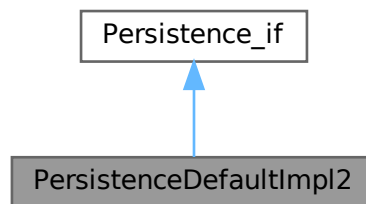
- `kernel/simulator/persistence/Persistence_if.h`

15.509 PersistenceDefaultImpl2 Class Reference

Inheritance diagram for PersistenceDefaultImpl2:



Collaboration diagram for PersistenceDefaultImpl2:



Public Member Functions

- **PersistenceDefaultImpl2** ([Model](#) *model)
- bool [save](#) (std::string filename) override
Serializes and saves the model to the provided filename.
- bool [load](#) (std::string filename) override
Loads and deserializes a model from file.
- bool [hasChanged](#) () override
Indicates whether persistent state changed since the last save/load.
- void [setHasChanged](#) (bool [hasChanged](#)) override
Updates the persistent-state changed flag.
- bool [getOption](#) (Persistence_if::Options option) override
Returns whether a persistence option is currently enabled.
- void [setOption](#) (Persistence_if::Options option, bool value) override
Enables or disables a persistence option.
- std::string [getFormattedField](#) ([PersistenceRecord](#) *fields) override
Formats a set of fields as text according to the persistence format.

Additional Inherited Members

Public Types inherited from [Persistence_if](#)

- enum class **Options** : int {
SAVEDEFAULTS = 1 , **HIDEIDKEY** = 2 , **HIDETYPEKEY** = 4 , **HIDENAMEKEY** = 8 ,
SORTALPHLY = 16 }

15.509.1 Member Function Documentation

15.509.1.1 getFormattedField()

```
std::string PersistenceDefaultImpl2::getFormattedField (
    PersistenceRecord * fields ) [override], [virtual]
```

Formats a set of fields as text according to the persistence format.

Implements [Persistence_if](#).

15.509.1.2 getOption()

```
bool PersistenceDefaultImpl2::getOption (
    Persistence_if::Options option ) [override], [virtual]
```

Returns whether a persistence option is currently enabled.

Implements [Persistence_if](#).

15.509.1.3 hasChanged()

```
bool PersistenceDefaultImpl2::hasChanged ( ) [override], [virtual]
```

Indicates whether persistent state changed since the last save/load.

Implements [Persistence_if](#).

15.509.1.4 load()

```
bool PersistenceDefaultImpl2::load (
    std::string filename ) [override], [virtual]
```

Loads and deserializes a model from file.

Implements [Persistence_if](#).

Reimplemented in [PersistencePartialLoadImpl1](#).

15.509.1.5 save()

```
bool PersistenceDefaultImpl2::save (
    std::string filename ) [override], [virtual]
```

Serializes and saves the model to the provided filename.

Implements [Persistence_if](#).

15.509.1.6 setHasChanged()

```
void PersistenceDefaultImpl2::setHasChanged (
    bool hasChanged ) [override], [virtual]
```

Updates the persistent-state changed flag.

Implements [Persistence_if](#).

15.509.1.7 setOption()

```
void PersistenceDefaultImpl2::setOption (
    Persistence_if::Options option,
    bool value ) [override], [virtual]
```

Enables or disables a persistence option.

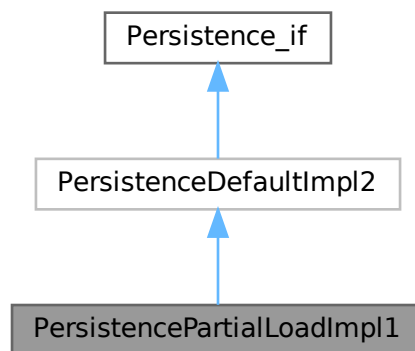
Implements [Persistence_if](#).

The documentation for this class was generated from the following files:

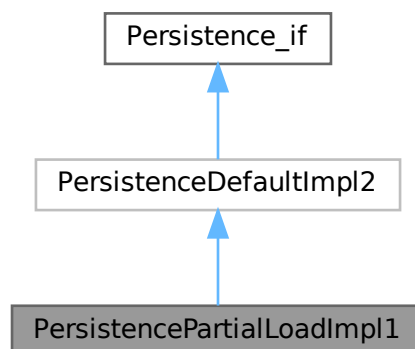
- kernel/simulator/persistence/PersistenceDefaultImpl2.h
- kernel/simulator/persistence/PersistenceDefaultImpl2.cpp

15.510 PersistencePartialLoadImpl1 Class Reference

Inheritance diagram for PersistencePartialLoadImpl1:



Collaboration diagram for PersistencePartialLoadImpl1:



Public Member Functions

- **PersistencePartialLoadImpl1** ([Model](#) *model)
- bool [load](#) (std::string filename) override
Lloads and deserializes a model from file.

Public Member Functions inherited from [PersistenceDefaultImpl2](#)

- **PersistenceDefaultImpl2** ([Model](#) *model)

- bool [save](#) (std::string filename) override
Serializes and saves the model to the provided filename.
- bool [hasChanged](#) () override
Indicates whether persistent state changed since the last save/load.
- void [setHasChanged](#) (bool [hasChanged](#)) override
Updates the persistent-state changed flag.
- bool [getOption](#) (Persistence_if::Options option) override
Returns whether a persistence option is currently enabled.
- void [setOption](#) (Persistence_if::Options option, bool value) override
Enables or disables a persistence option.
- std::string [getFormattedField](#) ([PersistenceRecord](#) *fields) override
Formats a set of fields as text according to the persistence format.

Additional Inherited Members

Public Types inherited from [Persistence_if](#)

- enum class **Options** : int {
 SAVEDEFAULTS = 1 , **HIDEIDKEY** = 2 , **HIDETYPEKEY** = 4 , **HIDENAMEKEY** = 8 ,
 SORTALPHLY = 16 }

15.510.1 Member Function Documentation

15.510.1.1 load()

```
bool PersistencePartialLoadImpl1::load (
    std::string filename ) [override], [virtual]
```

Loads and deserializes a model from file.

Reimplemented from [PersistenceDefaultImpl2](#).

The documentation for this class was generated from the following files:

- kernel/simulator/persistence/PersistencePartialLoadImpl1.h
- kernel/simulator/persistence/PersistencePartialLoadImpl1.cpp

15.511 PersistenceRecord Class Reference

Serialization/Deserialization structure which behaves like an associative array.

```
#include <kernel/simulator/Persistence.h>
```

Classes

- struct [Entry](#)
"Raw" entry, users should treat this as an opaque type since payloads could be in any format.

Public Types

- using **Payload** = std::string
- using **Iterator** = std::unordered_map< std::string, [Entry](#) >::iterator

Public Member Functions

- virtual [PersistenceRecord](#) * **newInstance** ()
- virtual std::string **loadField** (const std::string &key, const std::string &defaultValue="")
- virtual double **loadField** (const std::string &key, double defaultValue)
- virtual unsigned int **loadField** (const std::string &key, unsigned int defaultValue)
- virtual int **loadField** (const std::string &key, int defaultValue)
- virtual [Util::TimeUnit](#) **loadField** (const std::string &key, [Util::TimeUnit](#) defaultValue)

- virtual void **saveField** (const std::string &key, const std::string &value, const std::string &defaultValue, const bool saveIfDefault=false)
- virtual void **saveField** (const std::string &key, const std::string &value)
- virtual void **saveField** (const std::string &key, double value, const double defaultValue, const bool saveIfDefault=false)
- virtual void **saveField** (const std::string &key, unsigned int value, const unsigned int defaultValue, const bool saveIfDefault=false)
- virtual void **saveField** (const std::string &key, unsigned int value)
- virtual void **saveField** (const std::string &key, int value, const int defaultValue, const bool saveIfDefault=false)
- virtual void **saveField** (const std::string &key, Util::TimeUnit value, const Util::TimeUnit defaultValue, const bool saveIfDefault=false)
- **PersistenceRecord** ([Persistence_if](#) &config)
- std::size_t **size** () const
Returns the number of distinct entries in the record.
- void **insert** ([Entry](#) entry)
Inserts a raw entry in the record.
- void **insert** (Iterator it, Iterator end)
Inserts multiple entries from the given iterators.
- void **erase** (const std::string &key)
Removes a specific entry from the record.
- void **clear** ()
Removes all entries from this record.
- Iterator **find** (const std::string &key)
Returns an iterator over a specific entry, if found.
- Iterator **begin** ()
Returns an iterator over raw entries.
- Iterator **end** ()
Past-the-end sentinel iterator.

15.511.1 Detailed Description

Serialization/Deserialization structure which behaves like an associative array.

Simulation models/components/definitions use this as a "visitor", which is guided across their innards during ser/deserialization and filled with fields. See [PersistentObject_base](#).

When this structure reaches the main model persistence interface, it finishes the serialization process in a format-specific manner. See [Persistence_if](#) and [Serializer](#).

15.511.2 Member Function Documentation

15.511.2.1 insert()

```
void PersistenceRecord::insert (
    Iterator it,
    Iterator end )
```

Inserts multiple entries from the given iterators.

Insert the raw entry payload from the source iterator range. Skip entries that already exist to avoid overwriting without warning.

The documentation for this class was generated from the following files:

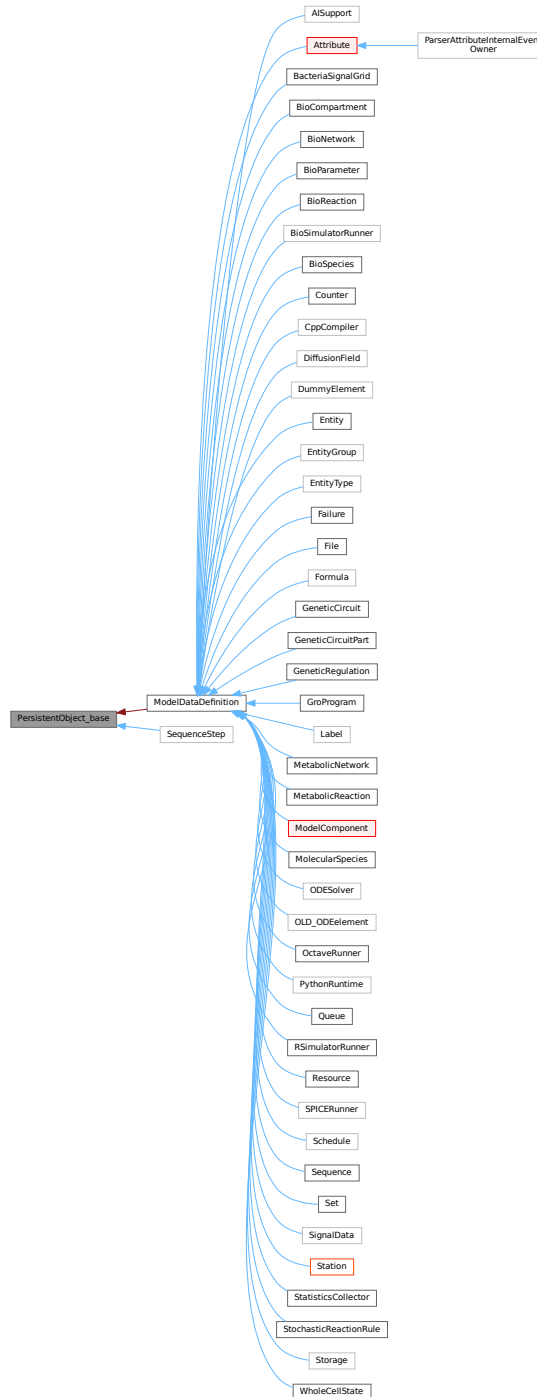
- kernel/simulator/Persistence.h
- kernel/simulator/Persistence.cpp

15.512 PersistentObject_base Class Reference

Dynamic serializable interface.

```
#include <kernel/simulator/Persistence.h>
```

Inheritance diagram for PersistentObject_base:



Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields)=0
Loads object fields from a deserialization structure.
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues=false)=0

Saves object fields into a serialization structure.

15.512.1 Detailed Description

Dynamic serializable interface.

Any object that's supposed to be saved to and/or loaded from a file must implement this interface.

15.512.2 Member Function Documentation

15.512.2.1 _loadInstance()

```
virtual bool PersistentObject_base::_loadInstance (
    PersistenceRecord * fields ) [protected], [pure virtual]
```

Loads object fields from a deserialization structure.

Parameters

<i>fields</i>	a table-like structure which can be queried for parsed fields.
---------------	--

Returns

a boolean which is true iff the load operation was successful.

Implemented in [Attribute](#), [Counter](#), [Entity](#), [EntityType](#), [StatisticsCollector](#), [ModelComponent](#), [ModelDataDefinition](#), [SinkModelComponent](#), [SourceModelComponent](#), [AIAssistant](#), [MarkovChain](#), [BacteriaColony](#), [BioRunnerCommand](#), [BioSimulate](#), [BioSteadyState](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [MetabolicFluxBalance](#), [ContinuousSystemComponent](#), [DiffEquations](#), [DiffusionSimulate](#), [LSODE](#), [OLD_ODEElement](#), [Decide](#), [DropOff](#), [Remove](#), [Search](#), [Buffer](#), [Clone](#), [Delay](#), [Process](#), [Release](#), [Seize](#), [SPICECircuit](#), [Resistor](#), [Vsource](#), [Vpulse](#), [Vsine](#), [Capacitor](#), [Diode](#), [PMOS](#), [NMOS](#), [NOT](#), [NAND](#), [AND](#), [NOR](#), [OR](#), [XOR](#), [XNOR](#), [SPICENode](#), [CppForG](#), [PythonForG](#), [RSimulator](#), [Batch](#), [Separate](#), [Record](#), [Write](#), [Assign](#), [Create](#), [Dispose](#), [Access](#), [Enter](#), [Exit](#), [Leave](#), [PickStation](#), [Route](#), [Start](#), [Stop](#), [Store](#), [Unstore](#), [CellularAutomataComp](#), [DefaultNode](#), [FSMState](#), [ModalModelDefault](#), [PetriPlace](#), [Submodel](#), [Match](#), [Signal](#), [Wait](#), [DummyComponent](#), [BioStateProjectionComponent](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CompartmentExchangeComponent](#), [EukaryoticCellCycleComponent](#), [FtsZPolymerizationComponent](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelComponent](#), [PathwayStressResponseComponent](#), [ResourceAllocationComponent](#), [StochasticReactionComponent](#), [StochasticTranscription](#), [StochasticTranslation](#), [AISupport](#), [BacteriaSignalGrid](#), [BioNetwork](#), [BioParameter](#), [BioReaction](#), [BioSimulatorRunner](#), [BioSpecies](#), [GeneticCircuit](#), [GeneticCircuitPart](#), [GeneticRegulation](#), [GroProgram](#), [MetabolicNetwork](#), [MetabolicReaction](#), [DiffusionField](#), [ODESolver](#), [Failure](#), [Queue](#), [Resource](#), [Schedule](#), [CppCompiler](#), [OctaveRunner](#), [PythonRuntime](#), [RSimulatorRunner](#), [SPICERunner](#), [EntityGroup](#), [File](#), [Formula](#), [Label](#), [Set](#), [Variable](#), [SequenceStep](#), [Sequence](#), [Station](#), [Storage](#), [SignalData](#), [DummyElement](#), [BioCompartment](#), [MolecularSpecies](#), [StochasticReactionRule](#), and [WholeCellState](#).

15.512.2.2 _saveInstance()

```
virtual void PersistentObject_base::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues = false ) [protected], [pure virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Implemented in [Attribute](#), [Counter](#), [Entity](#), [EntityType](#), [StatisticsCollector](#), [ModelComponent](#), [ModelDataDefinition](#), [SinkModelComponent](#), [SourceModelComponent](#), [AIAssistant](#), [MarkovChain](#), [BacteriaColony](#), [BioRunnerCommand](#), [BioSimulate](#), [BioSteadyState](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [MetabolicFluxBalance](#), [ContinuousSystemComponent](#), [DiffEquations](#), [DiffusionSimulate](#), [LSODE](#), [OLD_ODEElement](#), [Decide](#), [DropOff](#), [Remove](#), [Search](#), [Buffer](#), [Clone](#),

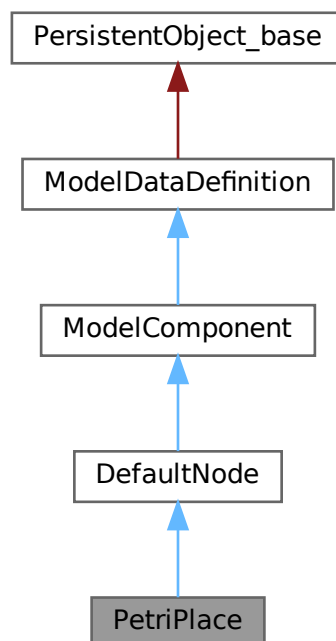
Delay, Process, Release, Seize, SPICECircuit, Resistor, Vsource, Vpulse, Vsine, Capacitor, Diode, PMOS, NMOS, NOT, NAND, AND, NOR, OR, XOR, XNOR, SPICENode, CppForG, PythonForG, RSimulator, Batch, Separate, Record, Write, Assign, Create, Dispose, Access, Enter, Exit, Leave, PickStation, Route, Start, Stop, Store, Unstore, CellularAutomataComp, DefaultNode, FSMState, ModalModelDefault, PetriPlace, Submodel, Match, Signal, Wait, DummyComponent, BioStateProjectionComponent, CellCycleCheckpointComponent, CellDivisionEvent, CellFateDecisionComponent, CellGrowthComponent, CompartmentExchangeComponent, EukaryoticCellCycleComponent, FtsZPolymerizationComponent, MetabolicStateProjectionComponent, MetabolicSubmodelComponent, PathwayStressResponseComponent, ResourceAllocationComponent, StochasticReactionComponent, StochasticTranscription, StochasticTranslation, AISupport, BacteriaSignalGrid, BioNetwork, BioParameter, BioReaction, BioSimulatorRunner, BioSpecies, GeneticCircuit, GeneticCircuitPart, GeneticRegulation, GroProgram, MetabolicNetwork, MetabolicReaction, DiffusionField, ODESolver, Failure, Queue, Resource, Schedule, CppCompiler, OctaveRunner, PythonRuntime, RSimulatorRunner, SPICERunner, EntityGroup, File, Formula, Label, Set, Variable, SequenceStep, Sequence, Station, Storage, SignalData, DummyElement, BioCompartment, MolecularSpecies, StochasticReactionRule, and WholeCellState.

The documentation for this class was generated from the following file:

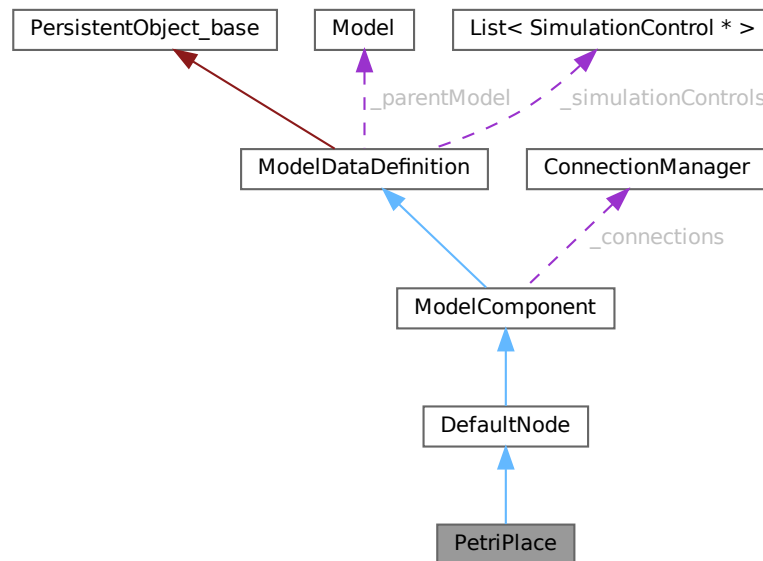
- kernel/simulator/Persistence.h

15.513 PetriPlace Class Reference

Inheritance diagram for PetriPlace:



Collaboration diagram for PetriPlace:



Public Member Functions

- **PetriPlace** ([Model](#) *model, std::string name="")
- unsigned int **getTokens** (std::string color="default") const
- void **addTokens** (unsigned int quantity, std::string color="default")
- bool **removeTokens** (unsigned int quantity, std::string color="default")
- std::map< std::string, unsigned int > * **getAllTokens** ()

Public Member Functions inherited from [DefaultNode](#)

- [DefaultNode](#) ([Model](#) *model, std::string componentName, std::string name)
constructors
- **DefaultNode** ([Model](#) *model, std::string name="")
- void **addTransition** ([DefaultNodeTransition](#) *transition)
new public user methods for this component
- void **removeTransition** ([DefaultNodeTransition](#) *transition)
- [List](#)< [DefaultNodeTransition](#) * > * **getTransitions** () const
- void **setInitialNode** (bool initialNode)
- bool **isInitialNode** () const
- void **setFinalNode** (bool finalNode)
- bool **isFinalNode** () const
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentName, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.

- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)

- Sets the model nesting level for this definition.*

 - `List< SimulationControl * > * getSimulationControls () const`

Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`

Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`

Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`

Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`

Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`

Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation * GetPluginInformation ()`
- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- static `ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from `DefaultNode`

- static `PluginInformation * GetPluginInformation ()`

static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- static `ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`

static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static `bool Check (ModelComponent *component)`

Validates a component instance.
- static `void CreateInternalData (ModelComponent *component)`

Creates internal data required by a component instance.
- static `void SaveInstance (PersistenceRecord *fields, ModelComponent *component)`

Serializes a component instance into a persistence record.
- static `void DispatchEvent (Event *event)`

Dispatches a simulation event to the target component.

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`

Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Creates a new instance of a data definition using the plugin registry.
- static `void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`

Serializes a data definition instance into a persistence record.
- static `bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`

Validates a data definition instance.

- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [DefaultNode](#)

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()

- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void **traceSimulation** (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager::Level](#) **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.513.1 Member Function Documentation

15.513.1.1 **_loadInstance()**

```
bool PetriPlace::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [DefaultNode](#).

15.513.1.2 **_saveInstance()**

```
void PetriPlace::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

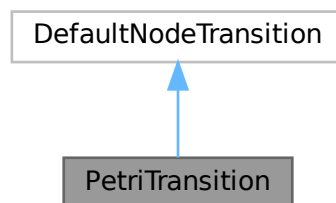
Reimplemented from [DefaultNode](#).

The documentation for this class was generated from the following files:

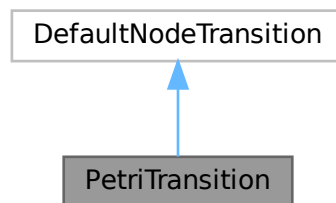
- plugins/components/ModalModel/PetriPlace.h
- plugins/components/ModalModel/PetriPlace.cpp

15.514 PetriTransition Class Reference

Inheritance diagram for PetriTransition:



Collaboration diagram for PetriTransition:



Public Member Functions

- **PetriTransition** ([DefaultNode](#) *source, [DefaultNode](#) *destination, std::string name="")
- void **setInputArcWeight** (std::string color, unsigned int weight)
- void **setOutputArcWeight** (std::string color, unsigned int weight)
- virtual bool **canFire** ([Model](#) *model, [Entity](#) *entity) const override
- virtual void **execute** ([Model](#) *model, [Entity](#) *entity) const override

Public Member Functions inherited from [DefaultNodeTransition](#)

- **DefaultNodeTransition** ([DefaultNode](#) *source, [DefaultNode](#) *destination, std::string name="")

- void **setSource** ([DefaultNode](#) *source)
- [DefaultNode](#) * **getSource** () const
- void **setDestination** ([DefaultNode](#) *destination)
- [DefaultNode](#) * **getDestination** () const
- void **setName** (std::string name)
- std::string **getName** () const
- void **setGuardExpression** (std::string guardExpression)
- std::string **getGuardExpression** () const
- void **setOutputExpression** (std::string outputExpression)
- std::string **getOutputExpression** () const
- void **setInputEvent** (std::string inputEvent)
- std::string **getInputEvent** () const
- void **setPriority** (unsigned int priority)
- unsigned int **getPriority** () const
- void **setProbability** (double probability)
- double **getProbability** () const
- void **setTransitionKind** (TransitionKind transitionKind)
- TransitionKind **getTransitionKind** () const

Additional Inherited Members

Public Types inherited from [DefaultNodeTransition](#)

- enum class **TransitionKind** : int { **DETERMINISTIC** = 0 , **PROBABILISTIC** = 1 }

15.514.1 Member Function Documentation

15.514.1.1 canFire()

```
bool PetriTransition::canFire (
    Model * model,
    Entity * entity ) const [override], [virtual]
```

Reimplemented from [DefaultNodeTransition](#).

15.514.1.2 execute()

```
void PetriTransition::execute (
    Model * model,
    Entity * entity ) const [override], [virtual]
```

Reimplemented from [DefaultNodeTransition](#).

The documentation for this class was generated from the following files:

- plugins/components/ModalModel/DefaultTransitionExtensions.h
- plugins/components/ModalModel/DefaultTransitionExtensions.cpp

15.515 PickableStationItem Class Reference

Public Member Functions

- **PickableStationItem** ([Model](#) *model, std::string stationName, std::string queueName="", std::string resourceName="")
- **PickableStationItem** ([Model](#) *model, std::string stationName, std::string expression)
- **PickableStationItem** ([Station](#) *station, [Queue](#) *queue)
- **PickableStationItem** ([Station](#) *station, [Resource](#) *resource)
- **PickableStationItem** ([Station](#) *station, std::string expression)
- **PickableStationItem** ([Station](#) *station)
- std::string **getExpression** () const

- [Queue](#) * **getQueue** () const
- [Resource](#) * **getResource** () const
- [Station](#) * **getStation** () const
- void **setExpression** (std::string _expression)
- void **setQueue** ([Queue](#) *_queue)
- void **setResource** ([Resource](#) *_resource)
- void **setStation** ([Station](#) *_station)
- std::string **getName** ()
- void **_addSimulationControl** ([SimulationControl](#) *control)
- [List](#)< [SimulationControl](#) * > * **getSimulationControls** () const
- std::string **getTypeDC** ()

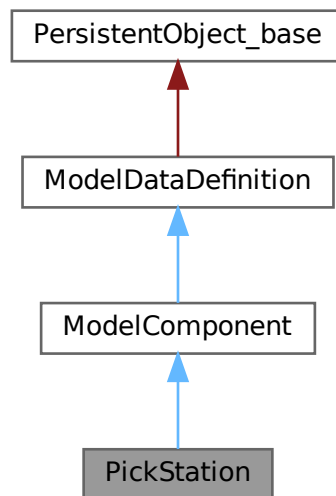
The documentation for this class was generated from the following files:

- plugins/components/MaterialHandling/PickableStationItem.h
- plugins/components/MaterialHandling/PickableStationItem.cpp

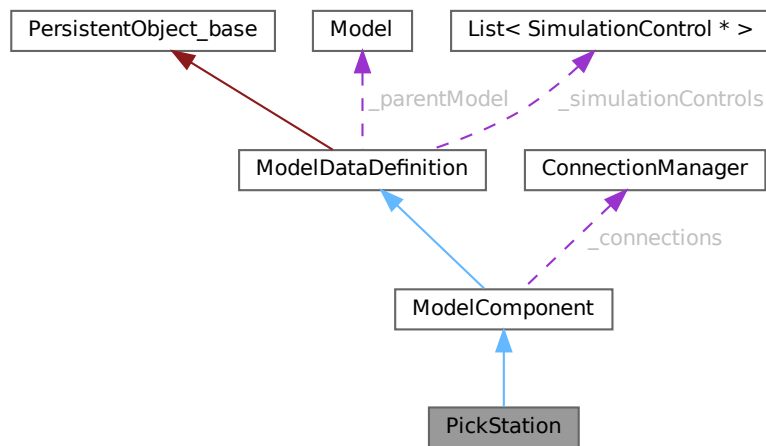
15.516 PickStation Class Reference

```
#include <plugins/components/MaterialHandling/PickStation.h>
```

Inheritance diagram for PickStation:



Collaboration diagram for PickStation:



Public Types

- enum class **TestCondition** : int { **MINIMUM** = 0 , **MAXIMUM** = 1 , **num_elements** = 2 }

Public Member Functions

- **PickStation** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
- virtual public methods*
- void **setTestCondition** (PickStation::TestCondition _testCondition)
- PickStation::TestCondition **getTestCondition** () const
- void **setSaveAttribute** (std::string _saveAttribute)
- std::string **getSaveAttribute** () const
- [List](#)< [PickableStationItem](#) * > * **getPickableStationItems** () const
- void **addPickableStationItem** ([PickableStationItem](#) *newItem)
- void **removePickableStationItem** ([PickableStationItem](#) *item)
- void **setPickConditionExpression** (bool _pickConditionExpression)
- bool **isPickConditionExpression** () const
- void **setPickConditionNumberInQueue** (bool _pickConditionNumberInQueue)
- bool **isPickConditionNumberInQueue** () const
- void **setPickConditionNumberBusyResource** (bool _pickConditionNumberBusyResource)
- bool **isPickConditionNumberBusyResource** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
- constructors*
- virtual ~**ModelComponent** ()
- Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * **getConnectionManager** () const
- new public user methods for this component*
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
- Connects this component to another component.*

- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) (ModelComponent *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level [getTraceLevelSpecific](#) () const

- Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) (TraceManager::Level level)
- Updates the trace level used by this definition.*
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelspecific, bool traceLevelSpecificEnabled=true)
- Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const
- Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
- Enables or disables the specific trace level.*

Static Public Member Functions

- static std::string [convertEnumToStr](#) (TestCondition condition)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool [Check](#) ([ModelComponent](#) *component)
- Validates a component instance.*
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
- Creates internal data required by a component instance.*
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
- Serializes a component instance into a persistence record.*
- static void [DispatchEvent](#) ([Event](#) *event)
- Dispatches a simulation event to the target component.*

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
- Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
- Creates a new instance of a data definition using the plugin registry.*
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
- Serializes a data definition instance into a persistence record.*
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
- Validates a data definition instance.*
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
- Resets simulation-time state between replications.*
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
- Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
- Creates related data and validates the instance in a single call.*

Protected Member Functions

- virtual bool `_loadInstance` (`PersistenceRecord` *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` (`PersistenceRecord` *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void `_onDispatchEvent` (`Entity` *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual `ParserChangesInformation` * `_getParserChangesInformation` () override
- virtual void `_initBetweenReplications` () override
- virtual void `_addSimulationControl` (`SimulationControl` *property) override
- virtual void `_createEditableDataDefinitions` () override

Protected Member Functions inherited from `ModelDataDefinition`

- bool `_getSaveDefaultsOption` ()
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, `ModelDataDefinition` *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.516.1 Detailed Description

[PickStation](#) module DESCRIPTION The [PickStation](#) module allows an entity to select a particular station from the multiple stations specified. This module picks among the group of stations based on the selection logic defined with the module. The entity may then route, transport, convey, or connect to the station specified. If the method chosen is connect, the selected station is assigned to an entity attribute. The station selection process is based on the minimum or maximum value of a variety of system variables and expressions. TYPICAL USES A part sent to a processing station based on machine's availability at each station A loan application sent to a set of loan officers based on the number sent to each officer A customer selecting among cashier lines based on the least number waiting in each line PROMPTS Prompt Description Name Unique name of the module that will be displayed in the flowchart. Test Condition Test condition to use for the station selection process, either Minimum or Maximum. Number En [Route](#) to [Station](#) The number of entities transferring to the station is considered in the station selection process. Number in [Queue](#) The number of entities in the queue at the station is considered in the station selection process. Number of Resources Busy The number of busy resources at the station is considered in the station selection process. Expression Determines if an additional user-defined expression is considered in the station selection process. Transfer Type Determines how an entity will be transferred out of this module to its next destination station—either [Route](#), Convey, Transport, or Connect. Save [Attribute](#) Defines the name of the attribute that will store the station name that is selected, visible when the transfer method is Connect. [Route](#) Time Move time of the entity from its current station to the station determined through this module. Units Time units for route-time parameters.

15.516.2 Member Function Documentation

15.516.2.1 `_addSimulationControl()`

```
void PickStation::_addSimulationControl (
    SimulationControl * property ) [override], [protected], [virtual]
```

This method is necessary only for those components that instantiate internal elements that must exist before simulation starts and even before model checking. That's the case of components that have internal `StatisticsCollectors`, since others components may refer to them as expressions (as in "TVAG(ThisCSTAT)") and therefore the model datum must exist before checking such expression

Reimplemented from [ModelDataDefinition](#).

15.516.2.2 `_check()`

```
bool PickStation::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.516.2.3 `_createEditableDataDefinitions()`

```
void PickStation::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.516.2.4 `_getParserChangesInformation()`

```
ParserChangesInformation * PickStation::_getParserChangesInformation ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDats. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dinamically linked to to simulator kernel to reflect the changes

Reimplemented from [ModelDataDefinition](#).

15.516.2.5 `_initBetweenReplications()`

```
void PickStation::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.516.2.6 `_loadInstance()`

```
bool PickStation::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.516.2.7 `_onDispatchEvent()`

```
void PickStation::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.516.2.8 _saveInstance()

```
void PickupStation::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.516.2.9 show()

```
std::string PickupStation::show ( ) [override], [virtual]
virtual public methods
```

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

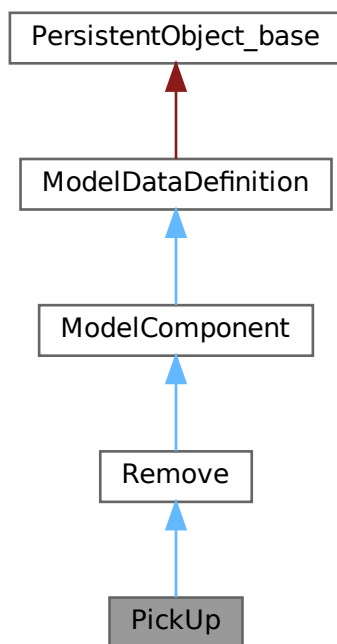
The documentation for this class was generated from the following files:

- plugins/components/MaterialHandling/PickStation.h
- plugins/components/MaterialHandling/PickStation.cpp

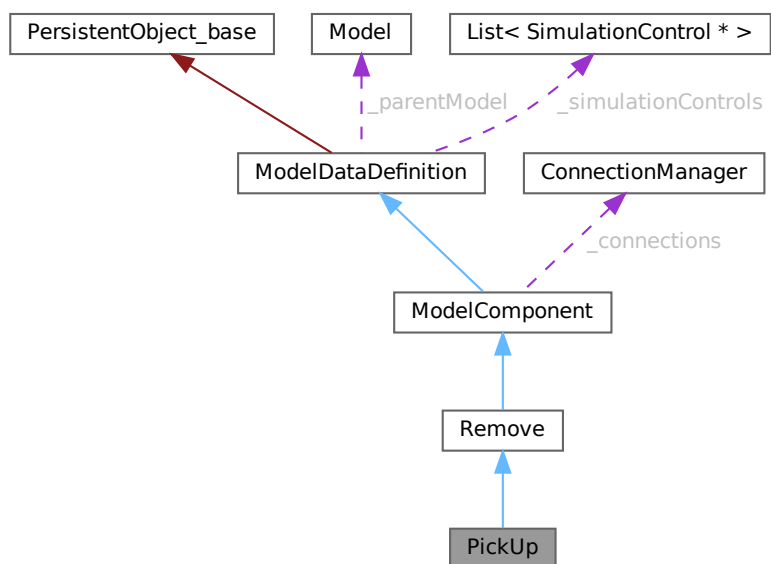
15.517 Pickup Class Reference

```
#include <plugins/components/Decisions/PickUp.h>
```

Inheritance diagram for Pickup:



Collaboration diagram for Pickup:



Public Member Functions

- **PickUp** ([Model](#) *model, std::string name="")

Public Member Functions inherited from [Remove](#)

- **Remove** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
- virtual public methods*
- void **setRemoveStartRank** (std::string _removeFromRank)
- std::string **getRemoveStartRank** () const
- void **setRemoveEndRank** (std::string _removeEndRank)
- std::string **getRemoveEndRank** () const
- void **setRemoveFromType** (Remove::RemoveFromType _removeFromType)
- Remove::RemoveFromType **getRemoveFromType** () const
- void **setRemoveFrom** ([ModelDataDefinition](#) *_removeFrom)
- [ModelDataDefinition](#) * **getRemoveFrom** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
- constructors*
- virtual ~**ModelComponent** ()
- Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * **getConnectionManager** () const
- new public user methods for this component*
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
- Connects this component to another component.*
- bool **hasBreakpointAt** ()
- Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string _description)
- Updates the free-form description shown by the GUI.*
- std::string **getDescription** () const
- Returns the free-form description associated with this component.*
- void **remove** ([ModelComponent](#) *comp)
- Removes a connected component from this component.*

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- Releases resources owned by this data definition.*
- Util::identification **getId** () const
- Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
- Updates the logical name of this instance.*
- const std::string & **getName** () const
- Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)
- Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const

- Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecific↔ Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [Remove](#)

- static std::string [convertEnumToStr](#) ([RemoveFromType](#) type)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Additional Inherited Members

Public Types inherited from [Remove](#)

- enum class [RemoveFromType](#) : int { [QUEUE](#) = 0 , [ENTITYGROUP](#) = 1 , [num_elements](#) = 2 }

Protected Member Functions inherited from [Remove](#)

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
 - legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

15.517.1 Detailed Description

Pickup module DESCRIPTION The Pickup module removes a number of consecutive entities from a given queue starting at a specified rank in the queue. The entities that are picked up are added to the end of the incoming entity's group. TYPICAL USES Gathering an order from various queue locations Gathering completed forms for an office order Picking up students at a bus stop for school PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Quantity Number of entities to pick up. [Queue](#) Name Name of the queue from which the entities will be picked up, starting at the specified rank. Starting Rank Starting rank of the entities to pick up from the queue, [Queue](#) Name.

The documentation for this class was generated from the following files:

- plugins/components/Decisions/PickUp.h
- plugins/components/Decisions/PickUp.cpp

15.518 Plugin Class Reference

```
#include <kernel/simulator/Plugin.h>
```

Public Member Functions

- [Plugin](#) (std::string filename_so_dll)
Creates a plugin wrapper for a dynamic library file.
- [Plugin](#) (StaticGetPluginInformation getInformation)
Creates a plugin wrapper from a compiled-in metadata callback.
- virtual ~[Plugin](#) ()
Releases metadata allocated for the wrapped plugin.
- std::string `show` ()
Returns a human-readable description of the plugin state.
- bool `isIsValidPlugin` () const
Indicates whether the plugin metadata was successfully initialized.
- [PluginInformation](#) * `getPluginInfo` () const
Returns the plugin metadata object.
- [ModelDataDefinition](#) * `loadNew` ([Model](#) *model, [PersistenceRecord](#) *fields)
Creates a new data definition instance from serialized fields.
- bool `loadAndInsertNew` ([Model](#) *model, [PersistenceRecord](#) *fields)
Loads a new instance and inserts it into the model in one step.
- [ModelDataDefinition](#) * `newInstance` ([Model](#) *model, std::string name="")
Creates a new in-memory instance of the plugin's primary type.

15.518.1 Detailed Description

A [Plugin](#) represents a dynamically linked component class ([ModelComponent](#)) or modeldatum class ([ModelDataDefinition](#)); It gives access to a [ModelComponent](#) so it can be used by the model. Classes like [Create](#), [Delay](#), and [Dispose](#) are examples of Plugins. It corresponds directly to the "Expansible" part (the capitalized 'E') of the GenESyS acronymous Plugins are [NOT](#) implemented yet

15.518.2 Constructor & Destructor Documentation

15.518.2.1 Plugin() [1/2]

```
Plugin::Plugin (
    std::string filename_so_dll )
```

Creates a plugin wrapper for a dynamic library file.

Parameters

<i>filename_so_dll</i>	Plugin library filename or path.
------------------------	--

15.518.2.2 Plugin() [2/2]

```
Plugin::Plugin (
    StaticGetPluginInformation getInformation )
```

Creates a plugin wrapper from a compiled-in metadata callback.

Parameters

<i>getInformation</i>	Callback that returns the plugin metadata.
-----------------------	--

15.518.3 Member Function Documentation

15.518.3.1 getPluginInfo()

```
PluginInformation * Plugin::getPluginInfo ( ) const
```

Returns the plugin metadata object.

Returns

[Plugin](#) information block, or `nullptr` when invalid.

15.518.3.2 isValidPlugin()

```
bool Plugin::isValidPlugin ( ) const
```

Indicates whether the plugin metadata was successfully initialized.

Returns

`true` when this wrapper contains a valid plugin.

15.518.3.3 loadAndInsertNew()

```
bool Plugin::loadAndInsertNew (
    Model * model,
    PersistenceRecord * fields )
```

Loads a new instance and inserts it into the model in one step.

Parameters

<i>model</i>	Parent model that will own the new instance.
<i>fields</i>	Serialized fields to deserialize.

Returns

`true` when loading and insertion succeed.

15.518.3.4 loadNew()

```
ModelDataDefinition * Plugin::loadNew (
    Model * model,
    PersistenceRecord * fields )
```

Creates a new data definition instance from serialized fields.

Creates a new [ModelDataDefinition](#) from fields loaded from a file or another persistence source.

Parameters

<i>model</i>	Parent model that will own the new instance.
<i>fields</i>	Serialized fields to deserialize.

Returns

New data definition instance, or `nullptr` on failure.

15.518.3.5 newInstance()

```
ModelDataDefinition * Plugin::newInstance (
    Model * model,
    std::string name = "" )
```

Creates a new in-memory instance of the plugin's primary type.

Parameters

<i>model</i>	Parent model that will own the instance.
<i>name</i>	Initial object name.

Returns

Newly created instance, or `nullptr` on failure.

15.518.3.6 show()

```
std::string Plugin::show ( )
```

Returns a human-readable description of the plugin state.

Returns

Textual summary used in traces and diagnostics.

responsibilities should remain in PluginConnector/PluginManager layers.

The documentation for this class was generated from the following files:

- kernel/simulator/Plugin.h
- kernel/simulator/Plugin.cpp

15.519 PluginCatalogController Class Reference

Controller responsible for plugin catalog presentation and plugin palette behavior.

```
#include <applications/gui/genesys/controllers/PluginCatalogController.h>
```

Public Member Functions

- **PluginCatalogController** ([Simulator](#) *simulator, QTreeWidget *pluginsTree, QPlainTextEdit *modelText↵ Editor, std::map< std::string, QColor > *pluginCategoryColor)
 - Creates the plugin-catalog controller used by [MainWindow](#) compatibility slots.*
- void **insertPluginUI** ([Plugin](#) *plugin) const
 - Inserts one plugin entry into the categorized plugin tree representation.*
- void **reloadFromPluginManager** () const
 - Rebuilds the categorized plugin tree from the current [PluginManager](#) state.*
- void **applyCategoryExpansionPolicy** () const
 - Applies the default expansion policy for plugin category roots.*
- void **showContextMenu** (const QPoint &pos) const
 - Shows the plugin-tree context menu used to reorder top-level categories.*
- void **insertFakePlugins** () const
 - Inserts compatibility fake plugins used by legacy GUI flows.*
- void **handlePluginItemDoubleClicked** (QTreeWidgetItem *item, int column) const
 - Handles plugin double-click by delegating insertion/selection behavior.*
- void **handlePluginItemClicked** (QTreeWidgetItem *item, int column) const
 - Handles plugin single-click behavior without mutating model lifecycle state.*
- void **moveTopLevelCategory** (QTreeWidgetItem *item, int delta) const
 - Moves a top-level category before or after its siblings.*

15.519.1 Detailed Description

Controller responsible for plugin catalog presentation and plugin palette behavior.

This controller encapsulates the plugin-tree behavior used by [MainWindow](#) wrappers, keeping plugin browsing and insertion affordances consistent while reducing direct widget logic in the [MainWindow](#) compatibility façade.

Responsibilities:

- populate plugin entries grouped by category with legacy color conventions;
- provide compatibility helpers for fake-plugin insertion paths;
- react to plugin-tree click/double-click events delegated by [MainWindow](#).

Boundaries:

- it does not load plugin binaries or manage kernel plugin lifecycle;
- it does not own textual model synchronization beyond delegated editor insertions;
- it is a UI controller layer, not a persistence/export service.

The documentation for this class was generated from the following files:

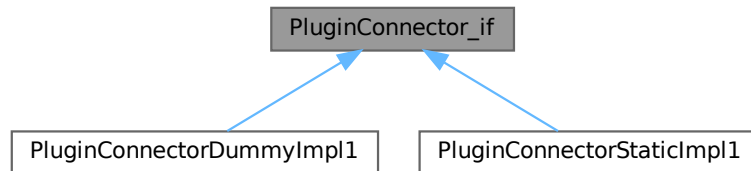
- applications/gui/genesys/controllers/PluginCatalogController.h
- applications/gui/genesys/controllers/PluginCatalogController.cpp

15.520 PluginConnector_if Class Reference

Interface for dynamic discovery/loading/unloading of plugin libraries.

```
#include <kernel/simulator/PluginConnector_if.h>
```

Inheritance diagram for PluginConnector_if:



Public Member Functions

- virtual [Plugin](#) * [check](#) (const std::string dynamicLibraryFilename)=0
check
- virtual [Plugin](#) * [connect](#) (const std::string dynamicLibraryFilename)=0
connect
- virtual [List](#)< std::string > * [find](#) ()=0
find dynamicLibraryFilenames
- virtual bool [disconnect](#) (const std::string dynamicLibraryFilename)=0
disconnect
- virtual bool [disconnect](#) ([Plugin](#) *plugin)=0
disconnect

15.520.1 Detailed Description

Interface for dynamic discovery/loading/unloading of plugin libraries.

Connector implementations bridge OS-level dynamic library handling and the kernel plugin abstraction, validating candidates and managing plugin lifecycles.

15.520.2 Member Function Documentation

15.520.2.1 check()

```
virtual Plugin * PluginConnector_if::check (
    const std::string dynamicLibraryFilename ) [pure virtual]
check
```

Parameters

<i>dynamicLibraryFilename</i>	
-------------------------------	--

Returns

Validates whether a dynamic library contains a compatible plugin without permanently connecting it.
Implemented in [PluginConnectorDummyImpl1](#), and [PluginConnectorStaticImpl1](#).

15.520.2.2 connect()

```
virtual Plugin * PluginConnector_if::connect (
    const std::string dynamicLibraryFilename ) [pure virtual]
connect
```

Parameters

<i>dynamicLibraryFilename</i>	
-------------------------------	--

Returns

Loads and connects a plugin from a dynamic library.
Implemented in [PluginConnectorDummyImpl1](#), and [PluginConnectorStaticImpl1](#).

15.520.2.3 disconnect() [1/2]

```
virtual bool PluginConnector_if::disconnect (
    const std::string dynamicLibraryFilename ) [pure virtual]
disconnect
```

Parameters

<i>dynamicLibraryFilename</i>	
-------------------------------	--

Returns

[List](#) of all Plugins found and connected

Disconnects the plugin associated with the provided library file.
Implemented in [PluginConnectorDummyImpl1](#), and [PluginConnectorStaticImpl1](#).

15.520.2.4 disconnect() [2/2]

```
virtual bool PluginConnector_if::disconnect (
    Plugin * plugin ) [pure virtual]
disconnect
```

Parameters

<i>plugin</i>	
---------------	--

Returns

Disconnects a previously connected plugin instance.
Implemented in [PluginConnectorDummyImpl1](#), and [PluginConnectorStaticImpl1](#).

15.520.2.5 find()

```
virtual List< std::string > * PluginConnector_if::find ( ) [pure virtual]
```

find dynamicLibraryFileNames

Parameters

--	--

return [List](#) of dynamicLibraryFileNames

Searches for candidate plugin libraries in the configured environment.

Implemented in [PluginConnectorDummyImpl1](#), and [PluginConnectorStaticImpl1](#).

The documentation for this class was generated from the following file:

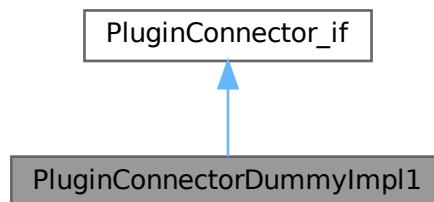
- kernel/simulator/PluginConnector_if.h

15.521 PluginConnectorDummyImpl1 Class Reference

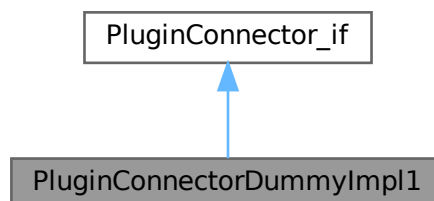
In-memory plugin connector that resolves built-in plugin metadata.

```
#include <plugins/PluginConnectorDummyImpl1.h>
```

Inheritance diagram for PluginConnectorDummyImpl1:



Collaboration diagram for PluginConnectorDummyImpl1:



Public Member Functions

- [Plugin](#) * [check](#) (const std::string dynamicLibraryFilename) override
Verifies whether a built-in plugin name can be resolved.
- [Plugin](#) * [connect](#) (const std::string dynamicLibraryFilename) override
Resolves a built-in plugin by filename alias.
- [List](#)< std::string > * [find](#) () override

Returns the list of built-in plugin filenames recognized by this connector.

- bool [disconnect](#) (const std::string dynamicLibraryFilename) override

Disconnects a plugin identified by filename.

- bool [disconnect](#) ([Plugin](#) *plugin) override

Disconnects a plugin instance.

15.521.1 Detailed Description

In-memory plugin connector that resolves built-in plugin metadata.

This connector does not load shared libraries. Instead, it maps known file names to compiled-in plugin classes and returns the corresponding metadata.

15.521.2 Member Function Documentation

15.521.2.1 [check\(\)](#)

```
Plugin * PluginConnectorDummyImpl1::check (
    const std::string dynamicLibraryFilename ) [override], [virtual]
```

Verifies whether a built-in plugin name can be resolved.

Parameters

<i>dynamicLibraryFilename</i>	Plugin library filename or alias.
-------------------------------	---

Returns

Metadata for the resolved plugin, or `nullptr` if unknown.

Implements [PluginConnector_if](#).

15.521.2.2 [connect\(\)](#)

```
Plugin * PluginConnectorDummyImpl1::connect (
    const std::string dynamicLibraryFilename ) [override], [virtual]
```

Resolves a built-in plugin by filename alias.

Parameters

<i>dynamicLibraryFilename</i>	Plugin library filename or alias.
-------------------------------	---

Returns

Metadata for the resolved plugin, or `nullptr` if unknown.

Implements [PluginConnector_if](#).

15.521.2.3 [disconnect\(\)](#) [1/2]

```
bool PluginConnectorDummyImpl1::disconnect (
    const std::string dynamicLibraryFilename ) [override], [virtual]
```

Disconnects a plugin identified by filename.

Parameters

<i>dynamicLibraryFilename</i>	Plugin library filename or alias.
-------------------------------	---

Returns

Always `true` in the current dummy implementation.

Implements [PluginConnector_if](#).

15.521.2.4 disconnect() [2/2]

```
bool PluginConnectorDummyImpl1::disconnect (
    Plugin * plugin ) [override], [virtual]
```

Disconnects a plugin instance.

Parameters

<i>plugin</i>	Plugin instance to disconnect.
---------------	--

Returns

Always `true` in the current dummy implementation.

Implements [PluginConnector_if](#).

15.521.2.5 find()

```
List< std::string > * PluginConnectorDummyImpl1::find ( ) [override], [virtual]
```

Returns the list of built-in plugin filenames recognized by this connector.

Returns

Newly allocated list with known plugin aliases.

Implements [PluginConnector_if](#).

The documentation for this class was generated from the following files:

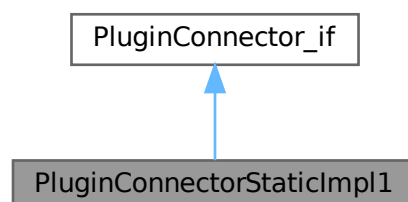
- `plugins/PluginConnectorDummyImpl1.h`
- `kernel/simulator/PluginConnectorDummyBootstrap.cpp`
- `plugins/PluginConnectorDummyImpl1.cpp`

15.522 PluginConnectorStaticImpl1 Class Reference

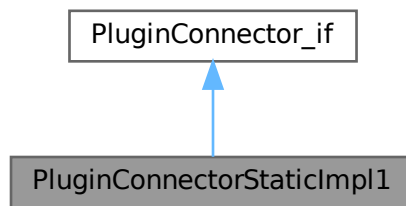
Static plugin connector stub used by the kernel interface.

```
#include <plugins/PluginConnectorStaticImpl1.h>
```

Inheritance diagram for `PluginConnectorStaticImpl1`:



Collaboration diagram for PluginConnectorStaticImpl1:



Public Member Functions

- `Plugin * check` (const std::string dynamicLibraryFilename) override
Checks whether a plugin library can be connected.
- `Plugin * connect` (const std::string dynamicLibraryFilename) override
Connects a plugin library.
- `List< std::string > * find` () override
Lists the available plugin libraries.
- `bool disconnect` (const std::string dynamicLibraryFilename) override
Disconnects a plugin by library filename.
- `bool disconnect` (Plugin *plugin) override
Disconnects a plugin instance.

15.522.1 Detailed Description

Static plugin connector stub used by the kernel interface.

This implementation currently provides a no-op connector that satisfies the plugin connector contract without loading dynamic libraries.

15.522.2 Member Function Documentation

15.522.2.1 check()

```
Plugin * PluginConnectorStaticImpl1::check (
    const std::string dynamicLibraryFilename ) [override], [virtual]
```

Checks whether a plugin library can be connected.

Parameters

<code>dynamicLibraryFilename</code>	Path to the plugin library.
-------------------------------------	-----------------------------

Returns

Always `nullptr` in the current static stub implementation.

Implements `PluginConnector_if`.

15.522.2.2 connect()

```
Plugin * PluginConnectorStaticImpl1::connect (
    const std::string dynamicLibraryFilename ) [override], [virtual]
```

Connects a plugin library.

Parameters

<i>dynamicLibraryFilename</i>	Path to the plugin library.
-------------------------------	-----------------------------

Returns

Always `nullptr` in the current static stub implementation.

Implements [PluginConnector_if](#).

15.522.2.3 disconnect() [1/2]

```
bool PluginConnectorStaticImpl1::disconnect (
    const std::string dynamicLibraryFilename ) [override], [virtual]
```

Disconnects a plugin by library filename.

Parameters

<i>dynamicLibraryFilename</i>	Path to the plugin library.
-------------------------------	-----------------------------

Returns

Always `true` in the current static stub implementation.

Implements [PluginConnector_if](#).

15.522.2.4 disconnect() [2/2]

```
bool PluginConnectorStaticImpl1::disconnect (
    Plugin * plugin ) [override], [virtual]
```

Disconnects a plugin instance.

Parameters

<i>plugin</i>	Plugin instance to disconnect.
---------------	--

Returns

Always `true` in the current static stub implementation.

Implements [PluginConnector_if](#).

15.522.2.5 find()

```
List< std::string > * PluginConnectorStaticImpl1::find ( ) [override], [virtual]
```

Lists the available plugin libraries.

Returns

Always `nullptr` in the current static stub implementation.

Implements [PluginConnector_if](#).

The documentation for this class was generated from the following files:

- `plugins/PluginConnectorStaticImpl1.h`
- `plugins/PluginConnectorStaticImpl1.cpp`

15.523 PluginInformation Class Reference

Public Member Functions

- **PluginInformation** (std::string pluginTypeName, StaticLoaderComponentInstance componentloader, StaticConstructorDataDefinitionInstance elementConstructor)
- **PluginInformation** (std::string pluginTypeName, StaticLoaderDataDefinitionInstance elementloader, StaticConstructorDataDefinitionInstance elementConstructor)
- StaticLoaderDataDefinitionInstance **getDataDefinitionLoader** () const
- StaticLoaderComponentInstance **GetComponentLoader** () const
- StaticConstructorDataDefinitionInstance **getDataDefinitionConstructor** () const
- bool **isGenerateReport** () const
- bool **isComponent** () const
- bool **isSendTransfer** () const
- bool **isReceiveTransfer** () const
- bool **isSink** () const
- bool **isSource** () const
- std::string **getObservation** () const
- std::string **getVersion** () const
- std::string **getDate** () const
- std::string **getAuthor** () const
- std::string **getPluginTypeName** () const
- void **insertDynamicLibFileDependence** (std::string filename)
- void **setDynamicLibFilenameDependencies** (std::list< std::string > *dynamicLibFilenameDependencies)
- std::list< std::string > * **getDynamicLibFilenameDependencies** () const
- void **insertSystemDependency** (const [SystemDependency](#) &dependency)
- const std::list< [SystemDependency](#) > * **getSystemDependencies** () const
- bool **hasSystemDependencies** () const
- void **setGenerateReport** (bool generateReport)
- void **setSendTransfer** (bool sendTransfer)
- void **setReceiveTransfer** (bool receiveTransfer)
- void **setSink** (bool Sink)
- void **setSource** (bool Source)
- void **setObservation** (std::string observation)
- void **setVersion** (std::string version)
- void **setDate** (std::string date)
- void **setAuthor** (std::string author)
- void **setMaximumOutputs** (unsigned short _maximumOutputs)
- unsigned short **getMaximumOutputs** () const
- void **setMinimumOutputs** (unsigned short _minimumOutputs)
- unsigned short **getMinimumOutputs** () const
- void **setMaximumInputs** (unsigned short _maximumInputs)
- unsigned short **getMaximumInputs** () const
- void **setMinimumInputs** (unsigned short _minimumInputs)
- unsigned short **getMinimumInputs** () const
- void **setDescriptionHelp** (std::string _descriptionHelp)
- std::string **getDescriptionHelp** () const
- void **setFields** (std::map< std::string, std::string > *_fields)
- std::map< std::string, std::string > * **getFields** () const
- void **setLanguageTemplate** (std::string _languageTemplate)
- std::string **getLanguageTemplate** () const
- void **setCategory** (std::string _category)
- std::string **getCategory** () const

Static Public Member Functions

- static std::string **categoryFolderName** (const std::string &category)

The documentation for this class was generated from the following files:

- kernel/simulator/PluginInformation.h
- kernel/simulator/PluginInformation.cpp

15.524 PluginInsertionOptions Struct Reference

Optional policy hooks used while inserting a plugin.

```
#include <kernel/simulator/PluginManager.h>
```

Public Attributes

- std::function< bool(const [SystemDependencyCheckResult](#) &)> **confirmSystemDependencyInstallation**
Called before running any install command for missing system dependencies.

15.524.1 Detailed Description

Optional policy hooks used while inserting a plugin.

The kernel reports dependency diagnostics through this structure and leaves user confirmation to the caller. An empty callback means non-interactive insertion: missing dependencies are reported through traces and no install command is executed.

The documentation for this struct was generated from the following file:

- kernel/simulator/PluginManager.h

15.525 PluginLoadIssue Class Reference

Diagnostic record for a plugin candidate that could not be loaded.

```
#include <kernel/simulator/PluginManager.h>
```

Public Types

- enum class [Reason](#) {
 InvalidPlugin , **MissingSystemDependency** , **DynamicDependencyFailure** , **ConnectionFailure** ,
 InsertionFailure , **Exception** }

High-level reason why a plugin candidate was not loaded.

Public Member Functions

- **PluginLoadIssue** ()
Creates an empty diagnostic record.
- **PluginLoadIssue** (std::string filename, std::string pluginTypename, [Reason](#) reason, std::string message, [SystemDependencyCheckResult](#) systemDependencyResult={}) noexcept
Creates a diagnostic record for a failed plugin load attempt.
- const std::string & **getFilename** () const
Returns the filename passed to the plugin connector.
- const std::string & **getPluginTypename** () const
Returns the plugin typename when it could be read from [PluginInformation](#).
- [Reason](#) **getReason** () const
Returns the high-level failure reason.
- const std::string & **getMessage** () const
Returns the human-readable diagnostic message captured by the manager.

- const [SystemDependencyCheckResult](#) & **getSystemDependencyResult** () const
Returns dependency diagnostics captured during system dependency preflight.
- bool **hasSystemDependencyResult** () const
Returns true when this issue includes system dependency diagnostics.
- std::string **diagnosticText** () const
Formats the issue for trace logs or GUI details.

Static Public Member Functions

- static std::string **reasonToString** ([Reason](#) reason)
Returns a stable textual label for a failure reason.

15.525.1 Detailed Description

Diagnostic record for a plugin candidate that could not be loaded.

The manager keeps these records so interactive front-ends can explain startup autoload failures after the application UI is ready. The kernel stores only technical diagnostics; GUI code decides how to present and resolve them.

The documentation for this class was generated from the following files:

- kernel/simulator/PluginManager.h
- kernel/simulator/PluginManager.cpp

15.526 PluginManager Class Reference

Manages discovery, loading, lookup and instantiation of simulation plugins.

```
#include <kernel/simulator/PluginManager.h>
```

Public Member Functions

- **PluginManager** ([Simulator](#) *simulator)
Creates a plugin manager bound to a simulator instance.
- **PluginManager** ([Simulator](#) *simulator, [PluginConnector_if](#) *pluginConnector, [SystemCommandExecutor_if](#) *systemCommandExecutor)
Creates a plugin manager with injectable connector and command executor.
- virtual ~**PluginManager** ()
Destroys owned plugin wrappers and connector resources.
- std::string **show** ()
Returns a human-readable summary of currently connected plugins.
- [List](#)< [Plugin](#) * > * **completePluginsFieldsAndTemplates** ()
Completes plugin metadata (fields/templates) after loading.
- bool **check** (const std::string &dynamicLibraryFilename)
Validates whether a dynamic library provides a compatible plugin.
- [SystemDependencyCheckResult](#) **checkSystemDependencies** (const std::string &dynamicLibraryFilename)
Evaluates system dependencies declared by a dynamic-library plugin without inserting it.
- [List](#)< std::string > * **discoverPluginFilenames** () const
Discovers plugin filenames through the configured connector without inserting them.
- [List](#)< [PluginLoadIssue](#) > * **getPluginLoadIssues** () const
Returns plugin candidates that failed to load during check/connect/preflight.
- void **clearPluginLoadIssues** ()
Clears all stored plugin load diagnostics.
- void **clearPluginLoadIssue** (const std::string &dynamicLibraryFilename)
Removes stored diagnostics for one plugin candidate filename.
- [Plugin](#) * **insert** (const std::string &dynamicLibraryFilename)

- Loads and inserts a plugin from a dynamic library file.*
- **Plugin * insert** (const std::string &dynamicLibraryFilename, const [PluginInsertionOptions](#) &options)
Loads and inserts a plugin, optionally asking the caller to authorize dependency installation.
- **bool remove** (const std::string &dynamicLibraryFilename)
Removes/disconnects a plugin by its dynamic library filename.
- **bool remove** ([Plugin](#) *plugin)
Removes/disconnects a plugin by pointer.
- **Plugin * find** (const std::string &pluginTypeName)
Finds a connected plugin by plugin type name.
- **std::vector< std::string >** [getDataDefinitionPluginTypeNames](#) () const
Returns connected plugin type names that can create [ModelDataDefinition](#) instances.
- **std::string sourceIncludePathFor** (const std::string &pluginTypeName)
Returns the source include path for a connected plugin type.
- **List< Plugin * > * autoInsertPlugins** (const std::string &pluginsListFilename, const bool lookForPluginsIf←
FilenameNotFound=true)
Auto-loads plugins listed in file (or discovered automatically as fallback).
- **List< Plugin * > * autoInsertPlugins** (const std::string &pluginsListFilename, const bool lookForPluginsIf←
FilenameNotFound, const [PluginInsertionOptions](#) &options)
Auto-loads plugins listed in file using insertion policy hooks.
- **List< Plugin * > * autoInsertPlugins** ()
Auto-loads plugins discovered automatically).
- **List< Plugin * > * autoInsertPlugins** (const [PluginInsertionOptions](#) &options)
Auto-loads plugins discovered automatically using insertion policy hooks.
- **Plugin * front** ()
Returns the first plugin in the internal plugin list.
- **Plugin * next** ()
Returns the next plugin using the internal iterator cursor.
- **Plugin * last** ()
Returns the last plugin in the internal plugin list.
- **unsigned int size** ()
Returns the number of connected plugins.
- **Plugin * getAtRank** (unsigned int rank)
Returns the plugin at a specific list rank.
- **ModelDataDefinition * newInstance** (const std::string &pluginType, [Model](#) *model, std::string name="")
Creates a new model data/component instance using a plugin typename.
- **template<typename T >**
T * newInstance ([Model](#) *model, std::string name="")

15.526.1 Detailed Description

Manages discovery, loading, lookup and instantiation of simulation plugins.

The plugin manager centralizes dynamic-library checks/connections through a [PluginConnector_if](#) and keeps the in-memory list of connected plugins used by the simulator to create model data/component instances.

15.526.2 Constructor & Destructor Documentation

15.526.2.1 PluginManager()

```
PluginManager::PluginManager (
    Simulator * simulator,
    PluginConnector\_if * pluginConnector,
    SystemCommandExecutor\_if * systemCommandExecutor )
```

Creates a plugin manager with injectable connector and command executor.

Ownership of both injected pointers is transferred to the manager. This constructor exists primarily for unit tests and narrow integration seams.

15.526.3 Member Function Documentation

15.526.3.1 `getDataDefinitionPluginTypeNames()`

```
std::vector< std::string > PluginManager::getDataDefinitionPluginTypeNames ( ) const
```

Returns connected plugin type names that can create [ModelDataDefinition](#) instances.

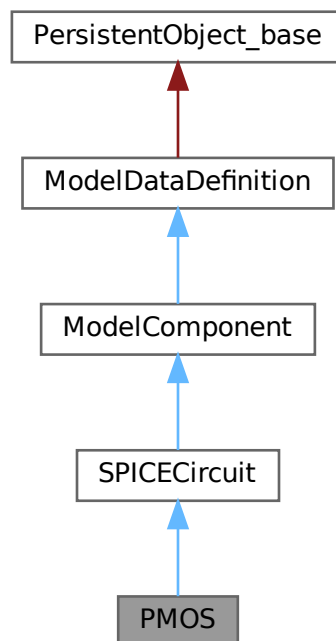
Polymorphic data structures such as [Set](#) use this snapshot to expose concrete, creatable element types to editors without hard-coding plugin names in the GUI.

The documentation for this class was generated from the following files:

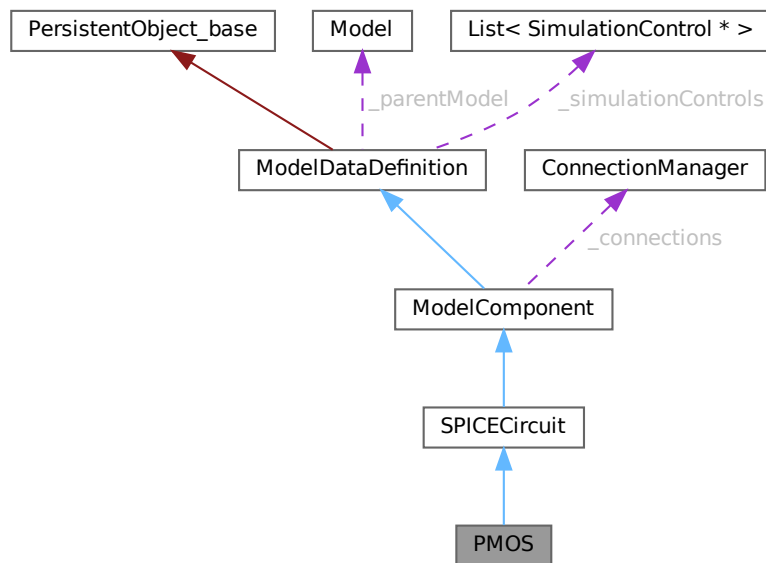
- `kernel/simulator/PluginManager.h`
- `kernel/simulator/PluginManager.cpp`

15.527 PMOS Class Reference

Inheritance diagram for PMOS:



Collaboration diagram for PMOS:



Public Member Functions

- **PMOS** ([Model](#) *model, std::string name="")
- void **Build** () override
- void **setWidth** (float width)
- void **setLength** (float length)
- void **setElectricalModel** (std::string electricalModel)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentName, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()

- Indicates whether a simulation breakpoint is set on this component.*
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) (ModelComponent *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) (Model *model, std::string datadefinitionType, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- Model * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.

- void [setTraceLevelSpecific](#) (TraceManager::Level level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation](#) * [GetPluginInformation](#) ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
- static std::vector< std::string > [split](#) (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void [_createNonEditableDataDefinitions](#) () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)

- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.527.1 Member Function Documentation

15.527.1.1 **_loadInstance()**

```
bool PMOS::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

15.527.1.2 `_saveInstance()`

```
void PMOS::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

15.527.1.3 `Build()`

```
void PMOS::Build ( ) [override], [virtual]
```

Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

- plugins/components/ElectronicsSimulation/SPICECircuit.h
- plugins/components/ElectronicsSimulation/SPICECircuit.cpp

15.528 `testing::internal::PointerPrinter` Struct Reference

Static Public Member Functions

- `template<typename T >`
static void **PrintValue** (T *p, ::std::ostream *os)

The documentation for this struct was generated from the following file:

- gtest/gtest/gtest-printers.h

15.529 `GroProgramRuntime::PopulationMutation` Struct Reference

Public Attributes

- PopulationMutationType **type** = PopulationMutationType::Grow
- unsigned int **value** = 0
- unsigned int **previousPopulationSize** = 0
- unsigned int **resultingPopulationSize** = 0

The documentation for this struct was generated from the following file:

- plugins/data/BiochemicalSimulation/GroProgramRuntime.h

15.530 `yy::position` Class Reference

A point in a source file.

```
#include <parser/location.hh>
```

Public Types

- typedef const std::string **filename_type**
Type for file name.
- typedef int **counter_type**
Type for line and column numbers.

Public Member Functions

- **position** ([filename_type](#) *f=YY_NULLPTR, [counter_type](#) l=1, [counter_type](#) c=1)
Construct a position.
- void **initialize** ([filename_type](#) *fn=YY_NULLPTR, [counter_type](#) l=1, [counter_type](#) c=1)
Initialization.

Line and Column related manipulators

- void **lines** ([counter_type](#) count=1)
(line related) Advance to the COUNT next lines.
- void **columns** ([counter_type](#) count=1)
(column related) Advance to the COUNT next columns.

Public Attributes

- [filename_type](#) * **filename**
File name to which this position refers.
- [counter_type](#) **line**
Current line number.
- [counter_type](#) **column**
Current column number.

15.530.1 Detailed Description

A point in a source file.

The documentation for this class was generated from the following file:

- parser/[location.hh](#)

15.531 testing::PrintToStringParamName Struct Reference

Public Member Functions

- template<class ParamType >
std::string **operator()** (const [TestParamInfo](#)< ParamType > &info) const

The documentation for this struct was generated from the following file:

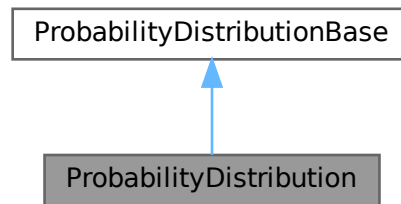
- gtest/gtest/internal/gtest-param-util.h

15.532 ProbabilityDistribution Class Reference

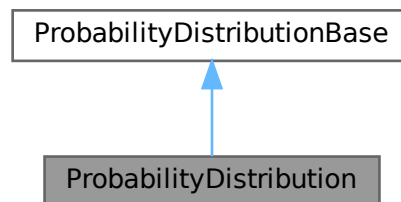
Static façade for inverse/CDF-related distribution utilities.

```
#include <tools/Statistics/ProbabilityDistribution.h>
```

Inheritance diagram for ProbabilityDistribution:



Collaboration diagram for ProbabilityDistribution:



Static Public Member Functions

- static double [inverseChi2](#) (double cumulativeProbability, double degreeFreedom)
- static double [inverseFFisherSnedecor](#) (double cumulativeProbability, double d1, double d2)
- static double [inverseNormal](#) (double cumulativeProbability, double mean, double stddev)
- static double [inverseTStudent](#) (double cumulativeProbability, double mean, double stddev, double degreeFreedom)

Static Public Member Functions inherited from [ProbabilityDistributionBase](#)

- static double [beta](#) (double x, double alpha, double beta)
- static double [chi2](#) (double x, double degreeFreedom)
- static double [erlang](#) (double x, double shape, double scale)
- static double [exponential](#) (double x, double mean)
- static double [fisherSnedecor](#) (double x, double d1, double d2)
- static double [gamma](#) (double x, double shape, double scale)
- static double [logNormal](#) (double x, double mean, double stddev)
- static double [normal](#) (double x, double mean, double stddev)
- static double [poisson](#) (double x, double mean)
- static double [triangular](#) (double x, double min, double mode, double max)
- static double [tStudent](#) (double x, double mean, double stddev, double degreeFreedom)
- static double [uniform](#) (double x, double min, double max)
- static double [weibull](#) (double x, double shape, double scale)

Additional Inherited Members

Static Protected Member Functions inherited from [ProbabilityDistributionBase](#)

- static double [_gammaFunction](#) (double x)
- static double [_betaFunction](#) (double x, double y)
- static double [_factorial](#) (double x)

15.532.1 Detailed Description

Static façade for inverse/CDF-related distribution utilities.

Architectural role:

- Exposes quantile/inverse computations used by hypothesis testing and confidence-interval routines.

Implementation notes:

- Uses numerical procedures and internal cache state.
- Current static design is legacy-oriented and not the final target model.

Planned evolution:

- Future versions may delegate responsibilities to [Distribution_if](#), [Quadrature_if](#) and [RootFinder_if](#) abstractions.

15.532.2 Member Function Documentation

15.532.2.1 `inverseChi2()`

```
double ProbabilityDistribution::inverseChi2 (
    double cumulativeProbability,
    double degreeFreedom ) [static]
```

@TODO: Is there a way to better determine the upper limit?

15.532.2.2 `inverseFFisherSnedecor()`

```
double ProbabilityDistribution::inverseFFisherSnedecor (
    double cumulativeProbability,
    double d1,
    double d2 ) [static]
```

@TODO: Is there a way to better determine the upper limit?

15.532.2.3 `inverseNormal()`

```
double ProbabilityDistribution::inverseNormal (
    double cumulativeProbability,
    double mean,
    double stddev ) [static]
```

@TODO: Could be better (separate into 1,2,3xstddev)

15.532.2.4 `inverseTStudent()`

```
double ProbabilityDistribution::inverseTStudent (
    double cumulativeProbability,
    double mean,
    double stddev,
    double degreeFreedom ) [static]
```

@TODO: Could be better

The documentation for this class was generated from the following files:

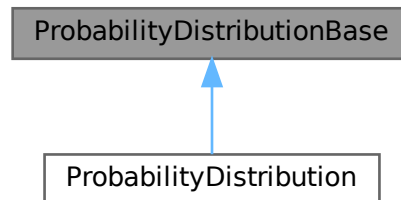
- tools/Statistics/ProbabilityDistribution.h
- tools/Statistics/ProbabilityDistribution.cpp

15.533 ProbabilityDistributionBase Class Reference

Mathematical base library for distribution-related functions.

```
#include <tools/Statistics/ProbabilityDistributionBase.h>
```

Inheritance diagram for ProbabilityDistributionBase:



Static Public Member Functions

- static double [beta](#) (double x, double alpha, double beta)
- static double [chi2](#) (double x, double degreeFreedom)
- static double [erlang](#) (double x, double shape, double scale)
- static double [exponential](#) (double x, double mean)
- static double [fisherSnedecor](#) (double x, double d1, double d2)
- static double [gamma](#) (double x, double shape, double scale)
- static double [logNormal](#) (double x, double mean, double stddev)
- static double [normal](#) (double x, double mean, double stddev)
- static double [poisson](#) (double x, double mean)
- static double [triangular](#) (double x, double min, double mode, double max)
- static double [tStudent](#) (double x, double mean, double stddev, double degreeFreedom)
- static double [uniform](#) (double x, double min, double max)
- static double [weibull](#) (double x, double shape, double scale)

Static Protected Member Functions

- static double [__gammaFunction](#) (double x)
- static double [__betaFunction](#) (double x, double y)
- static double [__factorial](#) (double x)

15.533.1 Detailed Description

Mathematical base library for distribution-related functions.

Architectural role:

- Provides static PDF/PMF and helper formulas used by higher-level tools.
- Not an OO hierarchy of distribution objects yet.

Mathematical meaning:

- Public methods represent density/mass functions or mathematically-related helpers for known distributions.

15.533.2 Member Function Documentation

15.533.2.1 `_betaFunction()`

```
double ProbabilityDistributionBase::_betaFunction (
    double x,
    double y ) [static], [protected]
```

Beta special function helper.

15.533.2.2 `_factorial()`

```
double ProbabilityDistributionBase::_factorial (
    double x ) [static], [protected]
```

Factorial helper, typically used by PMFs.

15.533.2.3 `_gammaFunction()`

```
double ProbabilityDistributionBase::_gammaFunction (
    double x ) [static], [protected]
```

Gamma special function approximation/helper. @TODO: Implement by myself

15.533.2.4 `beta()`

```
double ProbabilityDistributionBase::beta (
    double x,
    double alpha,
    double beta ) [static]
```

Beta distribution PDF.

15.533.2.5 `chi2()`

```
double ProbabilityDistributionBase::chi2 (
    double x,
    double degreeFreedom ) [static]
```

Chi-square distribution PDF.

15.533.2.6 `erlang()`

```
double ProbabilityDistributionBase::erlang (
    double x,
    double shape,
    double scale ) [static]
```

Erlang distribution PDF.

15.533.2.7 `exponential()`

```
double ProbabilityDistributionBase::exponential (
    double x,
    double mean ) [static]
```

Exponential distribution PDF.

15.533.2.8 `fisherSnedecor()`

```
double ProbabilityDistributionBase::fisherSnedecor (
    double x,
    double d1,
    double d2 ) [static]
```

Fisher-Snedecor F distribution PDF.

15.533.2.9 gamma()

```
double ProbabilityDistributionBase::gamma (
    double x,
    double shape,
    double scale ) [static]
```

Gamma distribution PDF.

15.533.2.10 logNormal()

```
double ProbabilityDistributionBase::logNormal (
    double x,
    double mean,
    double stddev ) [static]
```

Log-normal distribution PDF.

15.533.2.11 normal()

```
double ProbabilityDistributionBase::normal (
    double x,
    double mean,
    double stddev ) [static]
```

Normal distribution PDF.

15.533.2.12 poisson()

```
double ProbabilityDistributionBase::poisson (
    double x,
    double mean ) [static]
```

Poisson distribution PMF.

15.533.2.13 triangular()

```
double ProbabilityDistributionBase::triangular (
    double x,
    double min,
    double mode,
    double max ) [static]
```

Triangular distribution PDF.

15.533.2.14 tStudent()

```
double ProbabilityDistributionBase::tStudent (
    double x,
    double mean,
    double stddev,
    double degreeFreedom ) [static]
```

Student t distribution PDF.

15.533.2.15 uniform()

```
double ProbabilityDistributionBase::uniform (
    double x,
    double min,
    double max ) [static]
```

Uniform distribution PDF.

15.533.2.16 weibull()

```
double ProbabilityDistributionBase::weibull (
    double x,
    double shape,
    double scale ) [static]
```

Weibull distribution PDF.

The documentation for this class was generated from the following files:

- tools/Statistics/ProbabilityDistributionBase.h
- tools/Statistics/ProbabilityDistributionBase.cpp

15.534 MetabolicFluxBalanceSolver::Problem Struct Reference

Public Attributes

- `std::vector< std::vector< double > >` **stoichiometry**
- `std::vector< double >` **lowerBounds**
- `std::vector< double >` **upperBounds**
- `std::vector< double >` **objective**
- `bool` **maximize** = true

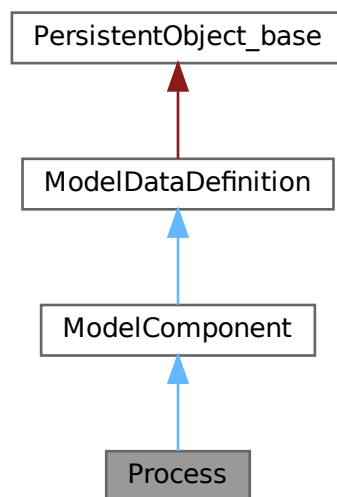
The documentation for this struct was generated from the following file:

- tools/Biochemical/MetabolicFluxBalanceSolver.h

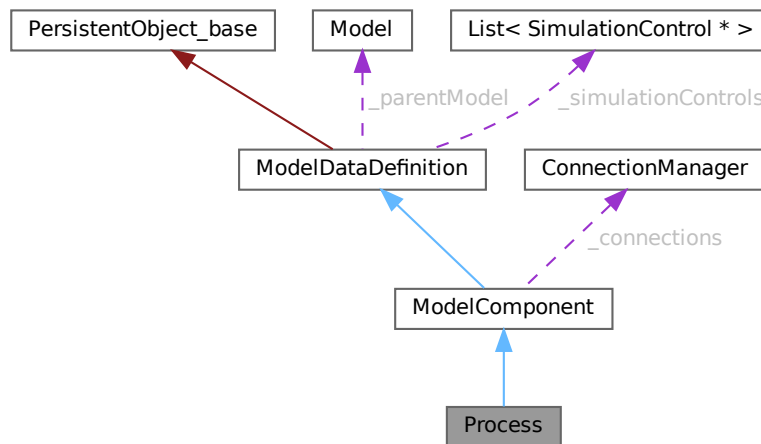
15.535 Process Class Reference

```
#include <plugins/components/DiscreteProcessing/Process.h>
```

Inheritance diagram for Process:



Collaboration diagram for Process:



Public Member Functions

- **Process** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
- *virtual public methods*
- void **setPriority** (unsigned short _priority)
- unsigned short **getPriority** () const
- void **setPriorityExpression** (std::string priority)
- std::string **getPriorityExpression** () const
- void **setAllocationType** (Util::AllocationType _allocationType)
- Util::AllocationType **getAllocationType** () const
- [List](#)< [SeizableItem](#) * > * **getSeizeRequests** () const
- void **addSeizeRequest** ([SeizableItem](#) *newRequest)
- void **removeSeizeRequest** ([SeizableItem](#) *request)
- void **setQueueableItem** ([QueueableItem](#) *_queueableItem)
- [QueueableItem](#) * **getQueueableItem** () const
- void **setDelayExpression** (std::string _delayExpression)
- void **setDelayExpression** (std::string _delayExpression, [Util::TimeUnit](#) _delayTimeUnit)
- std::string **delayExpression** () const
- void **setDelayTimeUnit** ([Util::TimeUnit](#) _delayTimeUnit)
- [Util::TimeUnit](#) **delayTimeUnit** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
- *constructors*
- virtual ~**ModelComponent** ()
- *Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * **getConnectionManager** () const
- *new public user methods for this component*
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
- *Connects this component to another component.*
- bool **hasBreakpointAt** ()

- Indicates whether a simulation breakpoint is set on this component.*
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) (ModelComponent *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) (Model *model, std::string datadefinitionType, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- Model * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.

- void [setTraceLevelSpecific](#) (TraceManager::Level level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void `_onDispatchEvent` (`Entity` *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool `_loadInstance` (`PersistenceRecord` *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` (`PersistenceRecord` *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_createNonEditableDataDefinitions` () override
- virtual void `_createEditableDataDefinitions` () override

Protected Member Functions inherited from `ModelDataDefinition`

- bool `_getSaveDefaultsOption` ()
- virtual `ParserChangesInformation` * `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` (`SimulationControl` *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, `ModelDataDefinition` *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.535.1 Detailed Description

This component ...

15.535.2 Member Function Documentation

15.535.2.1 [_check\(\)](#)

```
bool Process::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.535.2.2 [_createEditableDataDefinitions\(\)](#)

```
void Process::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.535.2.3 [_createNonEditableDataDefinitions\(\)](#)

```
void Process::_createNonEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.535.2.4 `_loadInstance()`

```
bool Process::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.535.2.5 `_onDispatchEvent()`

```
void Process::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.535.2.6 `_saveInstance()`

```
void Process::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.535.2.7 `show()`

```
std::string Process::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

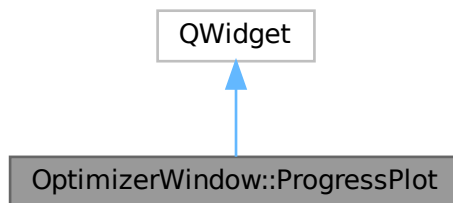
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

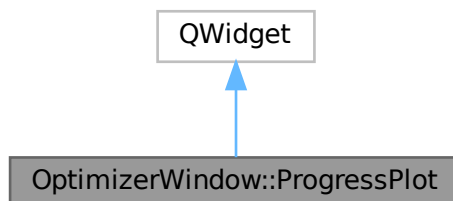
- plugins/components/DiscreteProcessing/Process.h
- plugins/components/DiscreteProcessing/Process.cpp

15.536 OptimizerWindow::ProgressPlot Class Reference

Inheritance diagram for OptimizerWindow::ProgressPlot:



Collaboration diagram for OptimizerWindow::ProgressPlot:



Public Member Functions

- **ProgressPlot** (`QWidget *parent=nullptr`)
- void **setValues** (`const QVector< double > &values`)

Protected Member Functions

- void **paintEvent** (`QPaintEvent *`) override

The documentation for this class was generated from the following file:

- applications/gui/genesys/tools/optimizer/OptimizerWindow.cpp

15.537 BioStateProjectionComponent::Projection Struct Reference

Public Attributes

- std::string **sourceSpeciesName**
- std::string **targetKey**
- ProjectionKind **kind** = ProjectionKind::MoleculeCount
- double **scale** = 1.0
- double **offset** = 0.0

The documentation for this struct was generated from the following file:

- plugins/components/WholeCellModeling/BioStateProjectionComponent.h

15.538 MetabolicStateProjectionComponent::Projection Struct Reference

Public Attributes

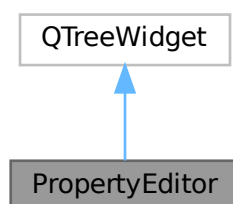
- std::string **reactionName**
- std::string **targetKey**
- ProjectionKind **kind** = ProjectionKind::MetaboliteAmount
- ProjectionUpdateMode **updateMode** = ProjectionUpdateMode::Overwrite
- double **scale** = 1.0
- double **offset** = 0.0
- double **turnoverFraction** = 0.0
- std::string **compartmentName**

The documentation for this struct was generated from the following file:

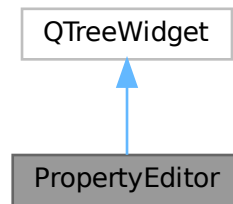
- plugins/components/WholeCellModeling/MetabolicStateProjectionComponent.h

15.539 PropertyEditor Class Reference

Inheritance diagram for PropertyEditor:



Collaboration diagram for PropertyEditor:



Public Member Functions

- **PropertyEditor** (QWidget *parent=nullptr)
- void **setModelBlock** (ModelDataDefinition *modelblock)

15.539.1 Member Function Documentation

15.539.1.1 setModelBlock()

```
void PropertyEditor::setModelBlock (
    ModelDataDefinition * modelblock )
```

!@TODO Era para ser getClassName());

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/PropertyEditor.h
- applications/gui/genesys/propertyeditor/PropertyEditor.cpp

15.540 PropertyEditorController Class Reference

Controller that orchestrates property editor state and scene-selection coupling.

```
#include <applications/gui/genesys/controllers/PropertyEditorController.h>
```

Public Member Functions

- **PropertyEditorController** (ObjectPropertyBrowser *propertyBrowser, ModelGraphicsView *graphicsView, PropertyEditorGenesys *propertyGenesys, std::map< SimulationControl *, DataComponentProperty * > *propertyList, std::map< SimulationControl *, DataComponentEditor * > *propertyEditorUI, std::map< SimulationControl *, ComboBoxEnum * > *propertyBox, std::function< void()> actualizeModelSimLanguage, std::function< void(bool)> actualizeModelComponents, std::function< void(bool)> actualizeModelDataDefinitions, std::function< void()> actualizeModelCppCode, std::function< bool()> createModelImage, std::function< void()> actualizeTabPanels, std::function< void()> actualizeActions)
- *Creates the property-editor controller and its synchronization callbacks.*
- void **sceneSelectionChanged** () const
- *Synchronizes current scene selection into property editor bindings.*
- void **onPropertyEditorModelChanged** () const
- *Executes post-commit synchronization across model representations and panes.*
- void **clearPropertyEditorSelection** () const
- *Clears property editor selection/bindings during lifecycle resets.*
- void **bindDataDefinitionFromInspector** (ModelDataDefinition *dataDefinition, QObject *graphicalObject=nullptr) const

Binds a DataDefinition selected from the inspector tree to the property editor.

- bool **isPostCommitPipelineActive** () const

Reports whether the deferred global refresh pipeline is active.

15.540.1 Detailed Description

Controller that orchestrates property editor state and scene-selection coupling.

This Phase-6 extraction isolates property editor wiring from [MainWindow](#) while preserving the existing compatibility façade methods. It coordinates selection-driven property binding and the deferred refresh cascade required after property commits.

Responsibilities:

- synchronize current scene selection into PropertyEditorGenesys/ObjectPropertyBrowser;
- execute post-edit refresh callbacks for model language, trees, C++ export, and tabs;
- coalesce heavy refresh requests to avoid re-entrant UI update storms.

Boundaries:

- it does not own property objects/editors lifetime (maps are injected);
- it does not alter undo/redo infrastructure or scene command orchestration;
- it does not persist models or execute simulation commands.

The documentation for this class was generated from the following files:

- applications/gui/genesys/controllers/PropertyEditorController.h
- applications/gui/genesys/controllers/PropertyEditorController.cpp

15.541 PropertyEditorGenesys Class Reference

Public Member Functions

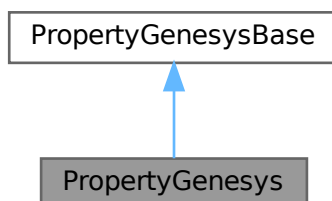
- std::list< [ModelComponent](#) * > **getElements** ()
- void **changeSimulationControl** ([SimulationControl](#) *control, const std::string &value, bool remove=false)
- [SimulationControl](#) * **findSimulationControl** (const std::string &id, const std::string &attribute)
- void **addElement** ([ModelComponent](#) *component)

The documentation for this class was generated from the following file:

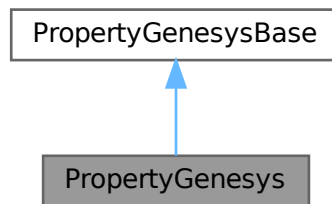
- kernel/simulator/PropertyGenesys.h

15.542 PropertyGenesys Class Reference

Inheritance diagram for PropertyGenesys:



Collaboration diagram for PropertyGenesys:



Public Member Functions

- **PropertyGenesys** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", std::string propertyId="", std::string toolTip="", std::string statusTip="")
- std::list< [PropertyGenesys](#) * > **subProperties** () const
- std::string **toolTip** () const
- std::string **statusTip** () const
- std::string **propertyName** () const
- std::string **propertyId** () const
- bool **isEnabled** () const
- bool **isModified** () const
- bool **isSubProperty** () const
- bool **hasValue** () const
- std::string **valueText** () const
- void **setToolTip** (const std::string text)
- void **setStatusTip** (const std::string text)
- void **setWhatsThis** (const std::string text)
- void **setPropertyName** (const std::string text)
- void **setPropertyId** (const std::string text)
- void **setEnabled** (bool enable)
- void **setModified** (bool modified)
- virtual bool **compare** ([PropertyGenesys](#) *otherProperty) const
- void **addSubProperty** ([PropertyGenesys](#) *property)
- void **insertSubProperty** ([PropertyGenesys](#) *property, [PropertyGenesys](#) *afterProperty)
- void **removeSubProperty** ([PropertyGenesys](#) *property)

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Protected Member Functions

- void **propertyChanged** ()

Protected Attributes

- std::string **_className**
- std::string **_propertyName**
- std::string **_whatsThis**
- std::string **_propertyId**
- std::string **_toolTip**
- std::string **_statusTip**
- std::list< [PropertyGenesys](#) * > **_subProperties**
- bool **_isEnabled**
- bool **_isModified**
- bool **_hasValue**
- bool **_isSubProperty**

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

The documentation for this class was generated from the following file:

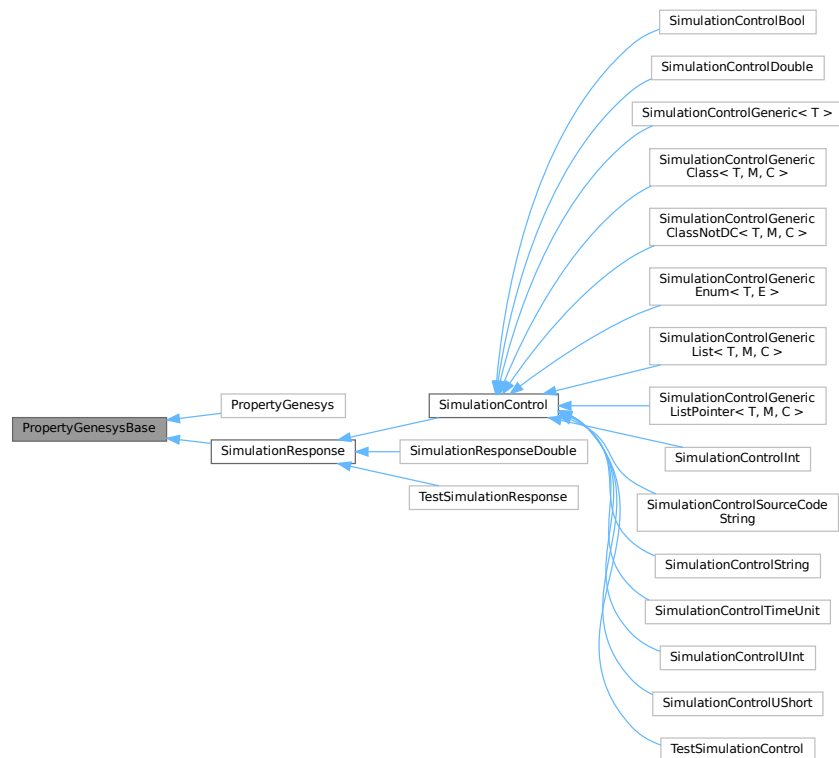
- kernel/simulator/SimulationControlAndResponse.h

15.543 PropertyGenesysBase Class Reference

Base metadata for kernel-side control/response abstractions.

```
#include <kernel/simulator/SimulationControlAndResponse.h>
```

Inheritance diagram for PropertyGenesysBase:



Public Member Functions

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Protected Attributes

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

15.543.1 Detailed Description

Base metadata for kernel-side control/response abstractions.

Historically, the experiment layer in GenESyS evolved from the basic [ModelSimulation](#) controls toward a more generic mechanism capable of exposing arbitrary getters and setters from model-related classes.

Despite the legacy name, this base currently lives in the simulation kernel and supports kernel-side experiment/control abstractions rather than a user-interface property editor.

The documentation for this class was generated from the following file:

- kernel/simulator/SimulationControlAndResponse.h

15.544 testing::internal::ProtobufPrinter Struct Reference

Static Public Member Functions

- `template<typename T, typename = typename std::enable_if< internal::HasDebugStringAndShortDebugString<T>::value>::type>`
static void **PrintValue** (const T &value, ::std::ostream *os)

Static Public Attributes

- static const size_t **kProtobufOneLinerMaxLength** = 50

The documentation for this struct was generated from the following file:

- gtest/gtest/gtest-printers.h

15.545 ProviderRequest Struct Reference

Input to a single LLM chat-completion call.

```
#include <tools/AIAssistant/AIProviderClient_if.h>
```

Public Attributes

- std::string **model**
Provider model identifier.
- std::vector< [ChatMessage](#) > **messages**
Conversation turns.
- double **temperature** = 0.2
Sampling temperature [0,2].
- unsigned int **maxTokens** = 4096
Maximum output tokens.
- bool **highReasoningMode** = false
Request extended reasoning (o-series / Claude thinking).

15.545.1 Detailed Description

Input to a single LLM chat-completion call.

The documentation for this struct was generated from the following file:

- tools/AIAssistant/AIProviderClient_if.h

15.546 ProviderResponse Struct Reference

Output from a single LLM chat-completion call.

```
#include <tools/AIAssistant/AIProviderClient_if.h>
```

Public Attributes

- bool **success** = false
true when the provider returned a valid completion.
- std::string **content**
Model reply text. Empty on failure.
- std::string **errorMessage**
Provider or transport error description.
- int **httpStatusCode** = 0
Raw HTTP status (0 when transport not used).
- unsigned int **inputTokens** = 0
Prompt tokens consumed (0 when unreported).
- unsigned int **outputTokens** = 0
Completion tokens produced (0 when unreported).

15.546.1 Detailed Description

Output from a single LLM chat-completion call.

The documentation for this struct was generated from the following file:

- tools/AIAssistant/AIProviderClient_if.h

15.547 testing::internal::ProxyTypeList< Ts > Struct Template Reference

Public Types

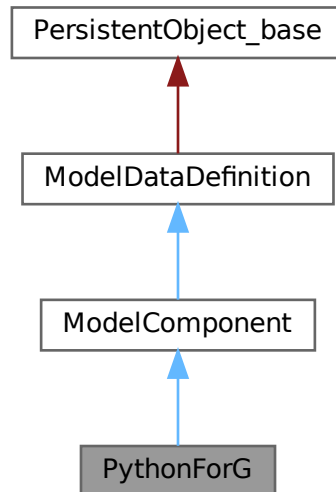
- using **type** = [Types](#)< Ts... >

The documentation for this struct was generated from the following file:

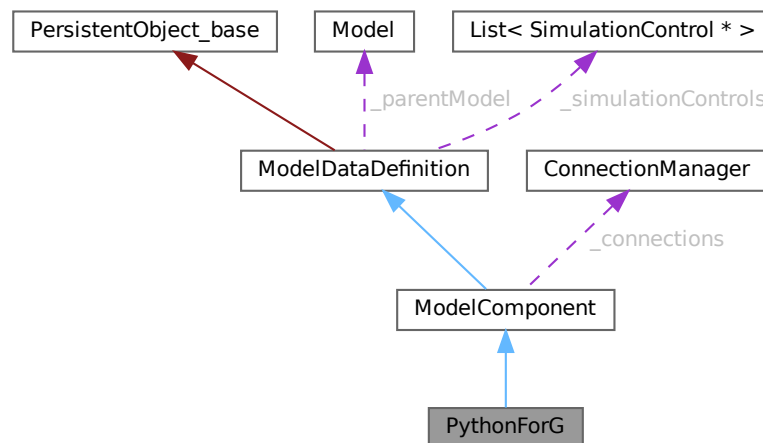
- gtest/gtest/internal/gtest-type-util.h

15.548 PythonForG Class Reference

Inheritance diagram for PythonForG:



Collaboration diagram for PythonForG:



Public Member Functions

- **PythonForG** ([Model](#) *model, std::string name="")
- void **setInitBetweenReplicationCode** (std::string initBetweenReplicationCode)
- std::string **getInitBetweenReplicationCode** () const
- void **setInitBetweenReplicationCodeProperty** ([SourceCodeString](#) initBetweenReplicationCode)
- [SourceCodeString](#) **getInitBetweenReplicationCodeProperty** () const

- void **setOnDispatchEventCode** (std::string onDispatchEventCode)
 - std::string **getOnDispatchEventCode** () const
 - void **setOnDispatchEventCodeProperty** (SourceCodeString onDispatchEventCode)
 - SourceCodeString **getOnDispatchEventCodeProperty** () const
 - void **setForwardEntityOnError** (bool forwardEntityOnError)
 - bool **isForwardEntityOnError** () const
 - PythonRuntime * **getPythonRuntime** () const
 - virtual std::string **show** () override
- virtual public methods*

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
- constructors*
- virtual ~**ModelComponent** ()
- Releases component-owned resources and unregisters it from the model.*
- **ConnectionManager** * **getConnectionManager** () const
- new public user methods for this component*
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
- Connects this component to another component.*
- bool **hasBreakpointAt** ()
- Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string _description)
- Updates the free-form description shown by the GUI.*
- std::string **getDescription** () const
- Returns the free-form description associated with this component.*
- void **remove** (**ModelComponent** *comp)
- Removes a connected component from this component.*

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- Releases resources owned by this data definition.*
- Util::identification **getId** () const
- Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
- Updates the logical name of this instance.*
- const std::string & **getName** () const
- Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)
- Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const
- Returns the user-visible label of this instance.*
- const std::string & **getClassname** () const
- Returns the runtime class name used by the plugin system.*
- **Model** * **getParentModel** () const
- Returns the parent model that owns this definition.*
- bool **isReportStatistics** () const
- Indicates whether this data definition contributes report statistics.*

- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelspecific, bool traceLevelSpecific↔ Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_createNonEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)

- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.548.1 Member Function Documentation

15.548.1.1 **_check()**

```
bool PythonForG::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.548.1.2 _createNonEditableDataDefinitions()

void PythonForG::_createNonEditableDataDefinitions () [override], [protected], [virtual]
 Reimplemented from [ModelDataDefinition](#).

15.548.1.3 _initBetweenReplications()

void PythonForG::_initBetweenReplications () [override], [protected], [virtual]
 Reimplemented from [ModelDataDefinition](#).

15.548.1.4 _loadInstance()

bool PythonForG::_loadInstance (
 [PersistenceRecord](#) * fields) [override], [protected], [virtual]
 virtual protected method that must be overridden
 Reimplemented from [ModelComponent](#).

15.548.1.5 _onDispatchEvent()

void PythonForG::_onDispatchEvent (
 [Entity](#) * entity,
 unsigned int inputPortNumber) [override], [protected], [virtual]

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.548.1.6 _saveInstance()

void PythonForG::_saveInstance (
 [PersistenceRecord](#) * fields,
 bool saveDefaultValues) [override], [protected], [virtual]

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.548.1.7 show()

std::string PythonForG::show () [override], [virtual]
 virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

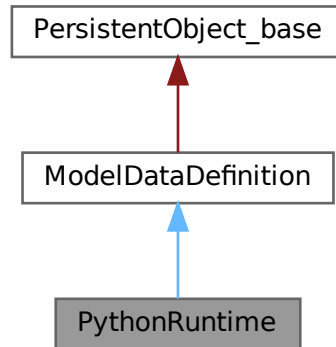
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

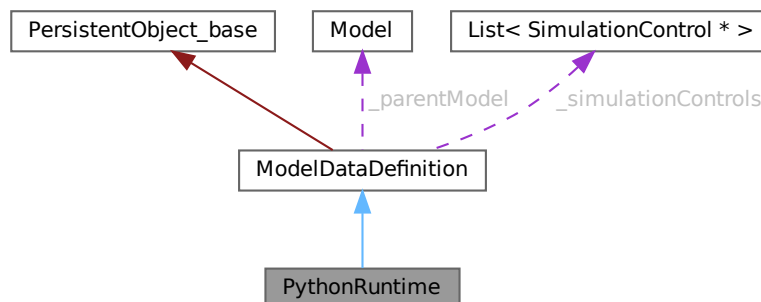
- plugins/components/ExternalIntegration/PythonForG.h
- plugins/components/ExternalIntegration/PythonForG.cpp

15.549 PythonRuntime Class Reference

Inheritance diagram for PythonRuntime:



Collaboration diagram for PythonRuntime:



Public Member Functions

- **PythonRuntime** ([Model](#) *model, std::string name="")
- void **setPythonExecutable** (std::string pythonExecutable)
- std::string **getPythonExecutable** () const
- void **setFacadeObjectName** (std::string facadeObjectName)
- std::string **getFacadeObjectName** () const
- void **setCaptureOutput** (bool captureOutput)
- bool **isCaptureOutput** () const
- void **setLastStatus** (std::string lastStatus)
- std::string **getLastStatus** () const
- void **setLastErrorMessage** (std::string lastErrorMessage)
- std::string **getLastErrorMessage** () const
- void **setLastStdout** (std::string lastStdout)
- std::string **getLastStdout** () const
- void **setLastStderr** (std::string lastStderr)

- `std::string getLastStderr ()` const
- `bool validateHooks` (const `std::string` &initCode, const `std::string` &onDispatchCode, `std::string` &error← Message) const
- `bool executelnitHook` ([ModelComponent](#) *component, const `std::string` &initCode, const `std::string` &on← DispatchCode, `std::string` &errorMessage)
- `bool executeOnDispatchHook` ([ModelComponent](#) *component, [Entity](#) *entity, const `std::string` &initCode, const `std::string` &onDispatchCode, `std::string` &errorMessage)
- `void resetExecutionState ()`
- `virtual std::string show ()` override
Returns a textual representation of this data definition.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, `std::string` datadefinitionTypename, `std::string` name="", bool insert← IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- `virtual ~ModelDataDefinition ()`
Releases resources owned by this data definition.
- `Util::identification getId ()` const
Returns the unique identifier of this instance.
- `void setName` (const `std::string` &name)
Updates the logical name of this instance.
- `const std::string & getName ()` const
Returns the logical name of this instance.
- `void setLabel` (const `std::string` &label)
Updates the user-visible label of this instance.
- `const std::string & getLabel ()` const
Returns the user-visible label of this instance.
- `const std::string & getClassname ()` const
Returns the runtime class name used by the plugin system.
- `Model * getParentModel ()` const
Returns the parent model that owns this definition.
- `bool isReportStatistics ()` const
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics` (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * `getInternalData` (const `std::string` &name) const
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData ()` const
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData ()` const
Returns the full map of attached data definitions.
- `bool hasChanged ()` const
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged` (bool hasChanged)
Updates the changed flag for this data definition.
- `unsigned int getLevel ()` const
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel` (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls ()` const
Returns the simulation controls owned by this data definition.

- `TraceManager::Level` `getTraceLevelSpecific ()` const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` level)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` traceLevelspecific, bool traceLevelSpecific↔ Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled ()` const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static `ModelDataDefinition` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
- static `PluginInformation` * `GetPluginInformation ()`
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void `SaveInstance` (`PersistenceRecord` *fields, `ModelDataDefinition` *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool `Check` (`ModelDataDefinition` *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void `InitBetweenReplications` (`ModelDataDefinition` *modeldatum)
Resets simulation-time state between replications.
- static void `CreateInternalData` (`ModelDataDefinition` *modeldatum)
Creates internal and attached data required by this instance.
- static bool `CreateRelatedDataElements` (`ModelDataDefinition` *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` (`PersistenceRecord` *fields) override
must be overridden by derived classes
- virtual void `_saveInstance` (`PersistenceRecord` *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from `ModelDataDefinition`

- bool `_getSaveDefaultsOption ()`
- virtual `ParserChangesInformation` * `_getParserChangesInformation ()`
- virtual void `_initBetweenReplications ()`
- virtual void `_addSimulationControl` (`SimulationControl` *control)
- virtual void `_createInternalAndAttachedData ()`
- virtual void `_createInternalStatisticReporters ()`
- virtual void `_createAttachedAttributes ()`
- virtual void `_createNonEditableDataDefinitions ()`

- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
 - legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void [traceReport](#) (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.549.1 Member Function Documentation

15.549.1.1 `_check()`

```
bool PythonRuntime::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.549.1.2 `_loadInstance()`

```
bool PythonRuntime::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.549.1.3 `_saveInstance()`

```
void PythonRuntime::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.549.1.4 `show()`

```
std::string PythonRuntime::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

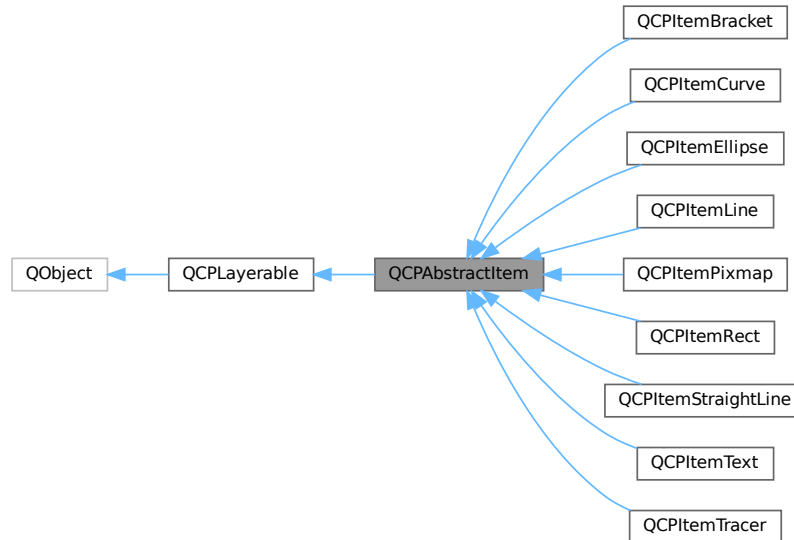
The documentation for this class was generated from the following files:

- plugins/data/ExternalIntegration/PythonRuntime.h
- plugins/data/ExternalIntegration/PythonRuntime.cpp

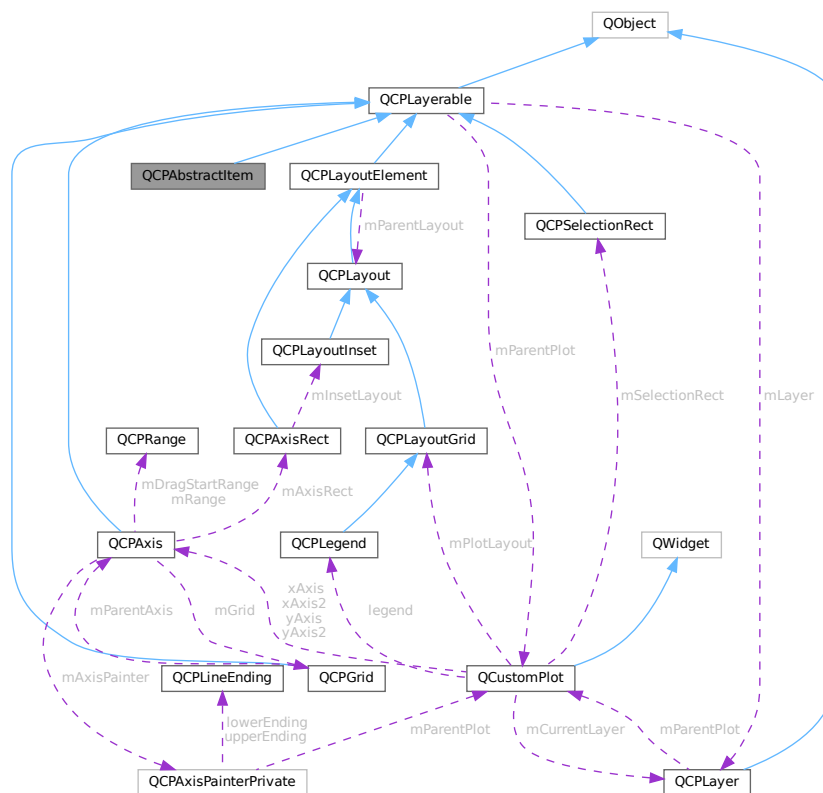
15.550 QCPAbstractItem Class Reference

The abstract base class for all items in a plot.

Inheritance diagram for QCPAbstractItem:



Collaboration diagram for QCPAbstractItem:



Signals

- void [selectionChanged](#) (bool selected)
- void [selectableChanged](#) (bool selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

Public Member Functions

- [QCPAbstractItem](#) ([QCustomPlot](#) *parentPlot)
- bool [clipToAxisRect](#) () const
- [QCPAxisRect](#) * [clipAxisRect](#) () const
- bool [selectable](#) () const
- bool [selected](#) () const
- void [setClipToAxisRect](#) (bool clip)
- void [setClipAxisRect](#) ([QCPAxisRect](#) *rect)
- Q_SLOT void [setSelectable](#) (bool selectable)
- Q_SLOT void [setSelected](#) (bool selected)
- virtual double [selectTest](#) (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const [Q_DECL_OVERRIDE](#)=0
- [QList](#)< [QCPItemPosition](#) * > [positions](#) () const
- [QList](#)< [QCPItemAnchor](#) * > [anchors](#) () const
- [QCPItemPosition](#) * [position](#) (const [QString](#) &name) const
- [QCPItemAnchor](#) * [anchor](#) (const [QString](#) &name) const
- bool [hasAnchor](#) (const [QString](#) &name) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool [visible](#) () const
- [QCustomPlot](#) * [parentPlot](#) () const
- [QCPLayerable](#) * [parentLayerable](#) () const
- [QCPLayer](#) * [layer](#) () const
- bool [antialiased](#) () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const [QString](#) &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual [QCP::Interaction](#) [selectionCategory](#) () const [Q_DECL_OVERRIDE](#)
- virtual [QRect](#) [clipRect](#) () const [Q_DECL_OVERRIDE](#)
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const [Q_DECL_OVERRIDE](#)
- virtual void [draw](#) ([QCPPainter](#) *painter) [Q_DECL_OVERRIDE](#)=0
- virtual void [selectEvent](#) ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionState↔ Changed) [Q_DECL_OVERRIDE](#)
- virtual void [deselectEvent](#) (bool *selectionStateChanged) [Q_DECL_OVERRIDE](#)
- virtual [QPointF](#) [anchorPixelPosition](#) (int anchorId) const
- double [rectDistance](#) (const [QRectF](#) &rect, const [QPointF](#) &pos, bool filledRect) const
- [QCPItemPosition](#) * [createPosition](#) (const [QString](#) &name)
- [QCPItemAnchor](#) * [createAnchor](#) (const [QString](#) &name, int anchorId)

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- bool **mClipToAxisRect**
- QPointer< [QCPAxisRect](#) > **mClipAxisRect**
- QList< [QCPItemPosition](#) * > **mPositions**
- QList< [QCPItemAnchor](#) * > **mAnchors**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCustomPlot](#)
- class [QCPItemAnchor](#)

15.550.1 Detailed Description

The abstract base class for all items in a plot.

In [QCustomPlot](#), items are supplemental graphical elements that are neither plottables ([QCPAbstractPlottable](#)) nor axes ([QCPAxis](#)). While plottables are always tied to two axes and thus plot coordinates, items can also be placed in absolute coordinates independent of any axes. Each specific item has at least one [QCPItemPosition](#) member which controls the positioning. Some items are defined by more than one coordinate and thus have two or more [QCPItemPosition](#) members (For example, [QCPItemRect](#) has *topLeft* and *bottomRight*).

This abstract base class defines a very basic interface like visibility and clipping. Since this class is abstract, it can't be instantiated. Use one of the subclasses or create a subclass yourself to create new items.

The built-in items are:

QCPItemLine	A line defined by a start and an end point. May have different ending styles on each side (e.g. arrows).
QCPItemStraightLine	A straight line defined by a start and a direction point. Unlike QCPItemLine , the straight line is infinitely long and has no endings.
QCPItemCurve	A curve defined by start, end and two intermediate control points. May have different ending styles on each side (e.g. arrows).
QCPItemRect	A rectangle
QCPItemEllipse	An ellipse
QCPItemPixmap	An arbitrary pixmap

QCPItemText	A text label
QCPItemBracket	A bracket which may be used to reference/highlight certain parts in the plot.
QCPItemTracer	An item that can be attached to a QCPGraph and sticks to its data points, given a key coordinate.

15.550.2 Clipping

Items are by default clipped to the main axis rect (they are only visible inside the axis rect). To make an item visible outside that axis rect, disable clipping via [setClipToAxisRect\(false\)](#).

On the other hand if you want the item to be clipped to a different axis rect, specify it via [setClipAxisRect](#). This clipAxisRect property of an item is only used for clipping behaviour, and in principle is independent of the coordinate axes the item might be tied to via its position members ([QCPItemPosition::setAxes](#)). However, it is common that the axis rect for clipping also contains the axes used for the item positions.

15.550.3 Using items

First you instantiate the item you want to use and add it to the plot:

by default, the positions of the item are bound to the x- and y-Axis of the plot. So we can just set the plot coordinates where the line should start/end:

If we don't want the line to be positioned in plot coordinates but a different coordinate system, e.g. absolute pixel positions on the [QCustomPlot](#) surface, we need to change the position type like this:

Then we can set the coordinates, this time in pixels:

and make the line visible on the entire [QCustomPlot](#), by disabling clipping to the axis rect:

For more advanced plots, it is even possible to set different types and parent anchors per X/Y coordinate of an item position, using for example [QCPItemPosition::setTypeX](#) or [QCPItemPosition::setParentAnchorX](#). For details, see the documentation of [QCPItemPosition](#).

15.550.4 Creating own items

To create an own item, you implement a subclass of [QCPAbstractItem](#). These are the pure virtual functions, you must implement:

- [selectTest](#)
- [draw](#)

See the documentation of those functions for what they need to do.

15.550.4.1 Allowing the item to be positioned

As mentioned, item positions are represented by [QCPItemPosition](#) members. Let's assume the new item shall have only one point as its position (as opposed to two like a rect or multiple like a polygon). You then add a public member of type [QCPItemPosition](#) like so:

```
QCPItemPosition * const myPosition;
```

the const makes sure the pointer itself can't be modified from the user of your new item (the [QCPItemPosition](#) instance it points to, can be modified, of course). The initialization of this pointer is made easy with the [createPosition](#) function. Just assign the return value of this function to each [QCPItemPosition](#) in the constructor of your item. [createPosition](#) takes a string which is the name of the position, typically this is identical to the variable name.

For example, the constructor of [QCPItemExample](#) could look like this:

```
QCPItemExample::QCPItemExample(QCustomPlot *parentPlot) :
    QCPAbstractItem(parentPlot),
    myPosition(createPosition("myPosition"))
{
    // other constructor code
}
```

15.550.4.2 The draw function

To give your item a visual representation, reimplement the draw function and use the passed [QCPPainter](#) to draw the item. You can retrieve the item position in pixel coordinates from the position member(s) via [QCPItemPosition::pixelPosition](#).

To optimize performance you should calculate a bounding rect first (don't forget to take the pen width into account), check whether it intersects the clipRect, and only draw the item at all if this is the case.

15.550.4.3 The selectTest function

Your implementation of the [selectTest](#) function may use the helpers [QCPVector2D::distanceSquaredToLine](#) and [rectDistance](#). With these, the implementation of the selection test becomes significantly simpler for most items. See the documentation of [selectTest](#) for what the function parameters mean and what the function should return.

15.550.4.4 Providing anchors

Providing anchors ([QCPItemAnchor](#)) starts off like adding a position. First you create a public member, e.g.

```
QCPItemAnchor * const bottom;
```

and create it in the constructor with the [createAnchor](#) function, assigning it a name and an anchor id (an integer enumerating all anchors on the item, you may create an own enum for this). Since anchors can be placed anywhere, relative to the item's position(s), your item needs to provide the position of every anchor with the reimplementation of the [anchorPixelPosition\(int anchorId\)](#) function.

In essence the [QCPItemAnchor](#) is merely an intermediary that itself asks your item for the pixel position when anything attached to the anchor needs to know the coordinates.

15.550.5 Constructor & Destructor Documentation

15.550.5.1 QCPAbstractItem()

```
QCPAbstractItem::QCPAbstractItem (
    QCustomPlot * parentPlot ) [explicit]
```

Base class constructor which initializes base class members.

15.550.6 Member Function Documentation

15.550.6.1 anchor()

```
QCPItemAnchor * QCPAbstractItem::anchor (
    const QString & name ) const
```

Returns the [QCPItemAnchor](#) with the specified *name*. If this item doesn't have an anchor by that name, returns `nullptr`.

This function provides an alternative way to access item anchors. Normally, you access anchors directly by their member pointers (which typically have the same variable name as *name*).

See also

[anchors](#), [position](#)

15.550.6.2 anchors()

```
QList< QCPItemAnchor * > QCPAbstractItem::anchors ( ) const [inline]
```

Returns all anchors of the item in a list. Note that since a position ([QCPItemPosition](#)) is always also an anchor, the list will also contain the positions of this item.

See also

[positions](#), [anchor](#)

15.550.6.3 applyDefaultAntialiasingHint()

```
void QCPAbstractItem::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Implements [QCPLayerable](#).

15.550.6.4 clipRect()

`QRect QCPAbstractItem::clipRect ( ) const [protected], [virtual]`
Reimplemented from [QCPLayerable](#).

15.550.6.5 deselectEvent()

`void QCPAbstractItem::deselectEvent (`
 `bool * selectionStateChanged ) [protected], [virtual]`
Reimplemented from [QCPLayerable](#).

15.550.6.6 draw()

`void QCPAbstractItem::draw (`
 `QCPPainter * painter ) [protected], [pure virtual]`
Implements [QCPLayerable](#).

15.550.6.7 hasAnchor()

`bool QCPAbstractItem::hasAnchor (`
 `const QString & name ) const`

Returns whether this item has an anchor with the specified *name*.

Note that you can check for positions with this function, too. This is because every position is also an anchor ([QCItemPosition](#) inherits from [QCItemAnchor](#)).

See also

[anchor](#), [position](#)

15.550.6.8 position()

`QCItemPosition * QCPAbstractItem::position (`
 `const QString & name ) const`

Returns the [QCItemPosition](#) with the specified *name*. If this item doesn't have a position by that name, returns `nullptr`.

This function provides an alternative way to access item positions. Normally, you access positions directly by their member pointers (which typically have the same variable name as *name*).

See also

[positions](#), [anchor](#)

15.550.6.9 positions()

`QList< QCItemPosition * > QCPAbstractItem::positions ( ) const [inline]`

Returns all positions of the item in a list.

See also

[anchors](#), [position](#)

15.550.6.10 selectEvent()

`void QCPAbstractItem::selectEvent (`
 `QMouseEvent * event,`
 `bool additive,`
 `const QVariant & details,`
 `bool * selectionStateChanged ) [protected], [virtual]`

Reimplemented from [QCPLayerable](#).

15.550.6.11 selectionCategory()

`QCP::Interaction` `QCPAbstractItem::selectionCategory ( ) const` [protected], [virtual]
 Reimplemented from [QCPLayerable](#).

15.550.6.12 selectionChanged

```
void QCPAbstractItem::selectionChanged (
    bool selected ) [signal]
```

This signal is emitted when the selection state of this item has changed, either by user interaction or by a direct call to [setSelected](#).

15.550.6.13 selectTest()

```
virtual double QCPAbstractItem::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [pure virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the `selectionTolerance` multiplied by 0.99). Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than $0.99 \cdot \text{selectionTolerance}$).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the `mouseReleaseEvent` occurs, and the finally selected object is notified via the `selectEvent/` `deselectEvent` methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to `selectEvent` when the parent [QCustomPlot](#) decides on the basis of this `selectTest` call, that the object was successfully selected. The subsequent call to `selectEvent` will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in `selectTest`. The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent `selectEvent`, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

`selectEvent`, `deselectEvent`, [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Reimplemented from [QCPLayerable](#).

Implemented in [QCPItemStraightLine](#), [QCPItemLine](#), [QCPItemCurve](#), [QCPItemRect](#), [QCPItemText](#), [QCPItemEllipse](#), [QCPItemPixmap](#), [QCPItemTracer](#), and [QCPItemBracket](#).

15.550.6.14 setClipAxisRect()

```
void QCPAbstractItem::setClipAxisRect (
    QCPAxisRect * rect )
```

Sets the clip axis rect. It defines the rect that will be used to clip the item when [setClipToAxisRect](#) is set to true.

See also

[setClipToAxisRect](#)

15.550.6.15 setClipToAxisRect()

```
void QCPAbstractItem::setClipToAxisRect (
    bool clip )
```

Sets whether the item shall be clipped to an axis rect or whether it shall be visible on the entire [QCustomPlot](#). The axis rect can be set with [setClipAxisRect](#).

See also

[setClipAxisRect](#)

15.550.6.16 setSelectable()

```
void QCPAbstractItem::setSelectable (
    bool selectable )
```

Sets whether the user can (de-)select this item by clicking on the [QCustomPlot](#) surface. (When [QCustomPlot::setInteractions](#) contains [QCustomPlot::iSelectItems](#).)

However, even when *selectable* was set to false, it is possible to set the selection manually, by calling [setSelected](#).

See also

[QCustomPlot::setInteractions](#), [setSelected](#)

15.550.6.17 setSelected()

```
void QCPAbstractItem::setSelected (
    bool selected )
```

Sets whether this item is selected or not. When selected, it might use a different visual appearance (e.g. pen and brush), this depends on the specific item though.

The entire selection mechanism for items is handled automatically when [QCustomPlot::setInteractions](#) contains [QCustomPlot::iSelectItems](#). You only need to call this function when you wish to change the selection state manually.

This function can change the selection state even when [setSelectable](#) was set to false.

emits the [selectionChanged](#) signal when *selected* is different from the previous selection state.

See also

[setSelectable](#), [selectTest](#)

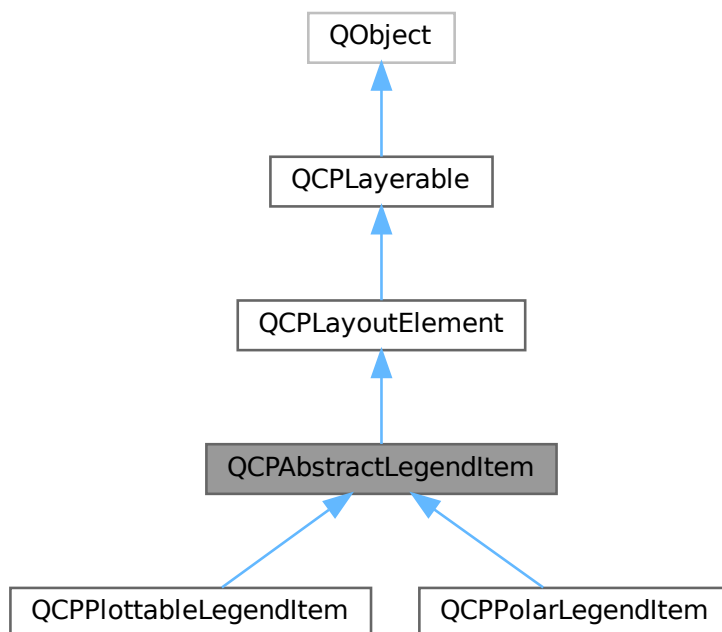
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

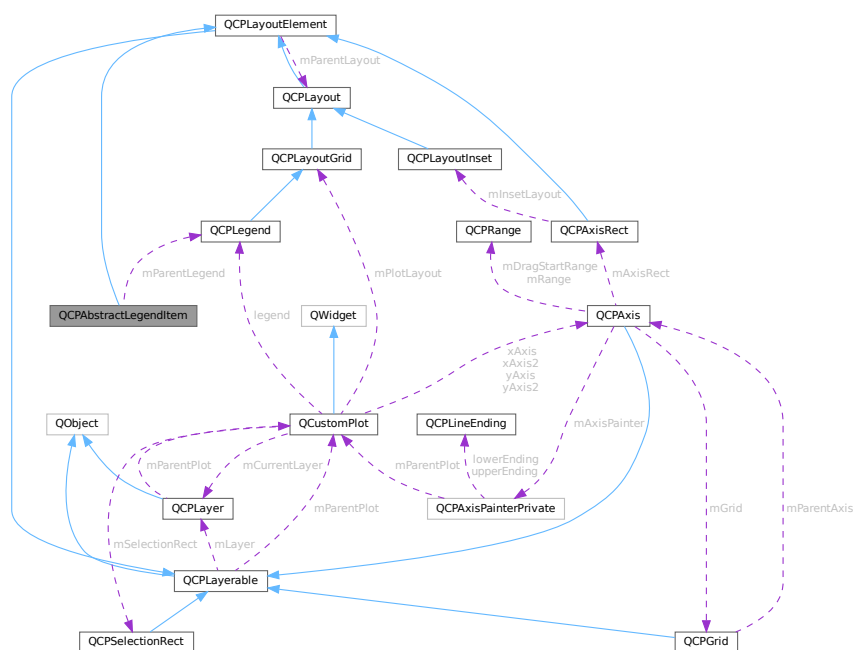
15.551 QCPAbstractLegendItem Class Reference

The abstract base class for all entries in a [QCPLegend](#).

Inheritance diagram for QCPAbstractLegendItem:



Collaboration diagram for QCPAbstractLegendItem:



Signals

- void [selectionChanged](#) (bool selected)
- void [selectableChanged](#) (bool selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

Public Member Functions

- [QCPAbstractLegendItem](#) ([QCPLegend](#) *parent)
- [QCPLegend](#) * [parentLegend](#) () const
- QFont [font](#) () const
- QColor [textColor](#) () const
- QFont [selectedFont](#) () const
- QColor [selectedTextColor](#) () const
- bool [selectable](#) () const
- bool [selected](#) () const
- void [setFont](#) (const QFont &font)
- void [setTextColor](#) (const QColor &color)
- void [setSelectedFont](#) (const QFont &font)
- void [setSelectedTextColor](#) (const QColor &color)
- Q_SLOT void [setSelectable](#) (bool selectable)
- Q_SLOT void [setSelected](#) (bool selected)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPLayoutElement](#)

- [QCPLayoutElement](#) ([QCustomPlot](#) *parentPlot=nullptr)
- [QCPLayout](#) * [layout](#) () const
- QRect [rect](#) () const
- QRect [outerRect](#) () const
- QMargins [margins](#) () const
- QMargins [minimumMargins](#) () const
- QCP::MarginSides [autoMargins](#) () const
- QSize [minimumSize](#) () const
- QSize [maximumSize](#) () const
- [SizeConstraintRect](#) [sizeConstraintRect](#) () const
- [QCPMarginGroup](#) * [marginGroup](#) ([QCP::MarginSide](#) side) const
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > [marginGroups](#) () const
- void [setOuterRect](#) (const QRect &rect)
- void [setMargins](#) (const QMargins &margins)
- void [setMinimumMargins](#) (const QMargins &margins)
- void [setAutoMargins](#) ([QCP::MarginSides](#) sides)
- void [setMinimumSize](#) (const QSize &size)
- void [setMinimumSize](#) (int width, int height)
- void [setMaximumSize](#) (const QSize &size)
- void [setMaximumSize](#) (int width, int height)
- void [setSizeConstraintRect](#) ([SizeConstraintRect](#) constraintRect)
- void [setMarginGroup](#) ([QCP::MarginSides](#) sides, [QCPMarginGroup](#) *group)
- virtual void [update](#) ([UpdatePhase](#) phase)
- virtual QSize [minimumOuterSizeHint](#) () const
- virtual QSize [maximumOuterSizeHint](#) () const
- virtual QList< [QCPLayoutElement](#) * > [elements](#) (bool recursive) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const [QString](#) &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Protected Member Functions

- virtual [QCP::Interaction](#) **selectionCategory** () const Q_DECL_OVERRIDE
- virtual void **applyDefaultAntialiasingHint** ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual [QRect](#) **clipRect** () const Q_DECL_OVERRIDE
- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE=0
- virtual void **selectEvent** ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionState↔ Changed) Q_DECL_OVERRIDE
- virtual void **deselectEvent** (bool *selectionStateChanged) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayoutElement](#)

- virtual int **calculateAutoMargin** ([QCP::MarginSide](#) side)
- virtual void **layoutChanged** ()
- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **mousePressEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **mouseMoveEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseReleaseEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseDoubleClickEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **wheelEvent** ([QWheelEvent](#) *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QCPLegend](#) * **mParentLegend**
- [QFont](#) **mFont**
- [QColor](#) **mTextColor**
- [QFont](#) **mSelectedFont**
- [QColor](#) **mSelectedTextColor**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from QCPLayoutElement

- [QCPLayout](#) * **mParentLayout**
- QSize **mMinimumSize**
- QSize **mMaximumSize**
- [SizeConstraintRect](#) **mSizeConstraintRect**
- QRect **mRect**
- QRect **mOuterRect**
- QMargins **mMargins**
- QMargins **mMinimumMargins**
- QCP::MarginSides **mAutoMargins**
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **mMarginGroups**

Protected Attributes inherited from QCPLayerable

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCPLegend](#)

Additional Inherited Members

Public Types inherited from QCPLayoutElement

- enum [UpdatePhase](#) { [upPreparation](#) , [upMargins](#) , [upLayout](#) }
- enum [SizeConstraintRect](#) { [scrInnerRect](#) , [scrOuterRect](#) }

15.551.1 Detailed Description

The abstract base class for all entries in a [QCPLegend](#).

It defines a very basic interface for entries in a [QCPLegend](#). For representing plottables in the legend, the subclass [QCPPlottableLegendItem](#) is more suitable.

Only derive directly from this class when you need absolute freedom (e.g. a custom legend entry that's not even associated with a plottable).

You must implement the following pure virtual functions:

- `draw` (from [QCPLayerable](#))

You inherit the following members you may use:

QCPLegend * mParentLegend	A pointer to the parent QCPLegend .
QFont mFont	The generic font of the item. You should use this font for all or at least the most prominent text of the item.

15.551.2 Constructor & Destructor Documentation

15.551.2.1 QCPLegendItem()

```
QCPLegendItem::QCPLegendItem (
    QCPLegend * parent ) [explicit]
```

Constructs a [QCPLegendItem](#) and associates it with the [QCPLegend](#) *parent*. This does not cause the item to be added to *parent*, so [QCPLegend::addItem](#) must be called separately.

15.551.3 Member Function Documentation

15.551.3.1 `applyDefaultAntialiasingHint()`

```
void QCPAbstractLegendItem::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

15.551.3.2 `clipRect()`

```
QRect QCPAbstractLegendItem::clipRect ( ) const [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.551.3.3 `deselectEvent()`

```
void QCPAbstractLegendItem::deselectEvent (
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.551.3.4 `draw()`

```
virtual void QCPAbstractLegendItem::draw (
    QCPPainter * painter ) [protected], [pure virtual]
```

Reimplemented from [QCPLayoutElement](#).

15.551.3.5 `selectEvent()`

```
void QCPAbstractLegendItem::selectEvent (
    QMouseEvent * event,
    bool additive,
    const QVariant & details,
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.551.3.6 `selectionCategory()`

```
QCP::Interaction QCPAbstractLegendItem::selectionCategory ( ) const [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.551.3.7 `selectionChanged`

```
void QCPAbstractLegendItem::selectionChanged (
    bool selected ) [signal]
```

This signal is emitted when the selection state of this legend item has changed, either by user interaction or by a direct call to [setSelected](#).

15.551.3.8 `selectTest()`

```
double QCPAbstractLegendItem::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Layout elements are sensitive to events inside their outer rect. If *pos* is within the outer rect, this method returns a value corresponding to 0.99 times the parent plot's selection tolerance. However, layout elements are not selectable by default. So if *onlySelectable* is true, -1.0 is returned.

See [QCPLayerable::selectTest](#) for a general explanation of this virtual method.

[QCPLayoutElement](#) subclasses may reimplement this method to provide more specific selection test behaviour.

Reimplemented from [QCPLayoutElement](#).

15.551.3.9 setFont()

```
void QCPAbstractLegendItem::setFont (
    const QFont & font )
```

Sets the default font of this specific legend item to *font*.

See also

[setTextColor](#), [QCPLegend::setFont](#)

15.551.3.10 setSelectable()

```
void QCPAbstractLegendItem::setSelectable (
    bool selectable )
```

Sets whether this specific legend item is selectable.

See also

[setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.551.3.11 setSelected()

```
void QCPAbstractLegendItem::setSelected (
    bool selected )
```

Sets whether this specific legend item is selected.

It is possible to set the selection state of this item by calling this function directly, even if `setSelectable` is set to false.

See also

[setSelectableParts](#), [QCustomPlot::setInteractions](#)

15.551.3.12 setSelectedFont()

```
void QCPAbstractLegendItem::setSelectedFont (
    const QFont & font )
```

When this legend item is selected, *font* is used to draw generic text, instead of the normal font set with [setFont](#).

See also

[setFont](#), [QCPLegend::setSelectedFont](#)

15.551.3.13 setSelectedTextColor()

```
void QCPAbstractLegendItem::setSelectedTextColor (
    const QColor & color )
```

When this legend item is selected, *color* is used to draw generic text, instead of the normal color set with [setTextColor](#).

See also

[setTextColor](#), [QCPLegend::setSelectedTextColor](#)

15.551.3.14 setTextColor()

```
void QCPAbstractLegendItem::setTextColor (
    const QColor & color )
```

Sets the default text color of this specific legend item to *color*.

See also

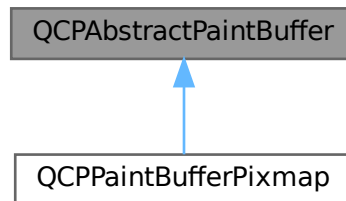
[setFont](#), [QCPLegend::setTextColor](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.552 QCPAbstractPaintBuffer Class Reference

The abstract base class for paint buffers, which define the rendering backend. Inheritance diagram for QCPAbstractPaintBuffer:



Public Member Functions

- [QCPAbstractPaintBuffer](#) (const QSize &size, double devicePixelRatio)
- QSize **size** () const
- bool **invalidated** () const
- double **devicePixelRatio** () const
- void [setSize](#) (const QSize &size)
- void [setInvalidated](#) (bool invalidated=true)
- void [setDevicePixelRatio](#) (double ratio)
- virtual [QCPPainter](#) * [startPainting](#) ()=0
- virtual void [donePainting](#) ()
- virtual void [draw](#) ([QCPPainter](#) *painter) const =0
- virtual void [clear](#) (const QColor &color)=0

Protected Member Functions

- virtual void [reallocateBuffer](#) ()=0

Protected Attributes

- QSize **mSize**
- double **mDevicePixelRatio**
- bool **mInvalidated**

15.552.1 Detailed Description

The abstract base class for paint buffers, which define the rendering backend.

This abstract base class defines the basic interface that a paint buffer needs to provide in order to be usable by [QCustomPlot](#).

A paint buffer manages both a surface to draw onto, and the matching paint device. The size of the surface can be changed via [setSize](#). External classes ([QCustomPlot](#) and [QCPLayer](#)) request a painter via [startPainting](#) and then perform the draw calls. Once the painting is complete, [donePainting](#) is called, so the paint buffer implementation can do clean up if necessary. Before rendering a frame, each paint buffer is usually filled with a color using [clear](#) (usually the color is `Qt::transparent`), to remove the contents of the previous frame.

The simplest paint buffer implementation is [QCPPaintBufferPixmap](#) which allows regular software rendering via the raster engine. Hardware accelerated rendering via pixel buffers and frame buffer objects is provided by [QCPPaintBufferGIPbuffer](#) and [QCPPaintBufferGIFbo](#). They are used automatically if [QCustomPlot::setOpenGl](#) is enabled.

15.552.2 Constructor & Destructor Documentation

15.552.2.1 QCPAbstractPaintBuffer()

```
QCPAbstractPaintBuffer::QCPAbstractPaintBuffer (
    const QSize & size,
    double devicePixelRatio ) [explicit]
```

Creates a paint buffer and initializes it with the provided *size* and *devicePixelRatio*.

Subclasses must call their [reallocateBuffer](#) implementation in their respective constructors.

15.552.3 Member Function Documentation

15.552.3.1 clear()

```
void QCPAbstractPaintBuffer::clear (
    const QColor & color ) [pure virtual]
```

Fills the entire buffer with the provided *color*. To have an empty transparent buffer, use the named color `Qt::transparent`.

This method must not be called if there is currently a painter (acquired with [startPainting](#)) active.

Implemented in [QCPPaintBufferPixmap](#).

15.552.3.2 donePainting()

```
void QCPAbstractPaintBuffer::donePainting ( ) [inline], [virtual]
```

If you have acquired a [QCPPainter](#) to paint onto this paint buffer via [startPainting](#), call this method as soon as you are done with the painting operations and have deleted the painter.

paint buffer subclasses may use this method to perform any type of cleanup that is necessary. The default implementation does nothing.

15.552.3.3 draw()

```
void QCPAbstractPaintBuffer::draw (
    QCPPainter * painter ) const [pure virtual]
```

Draws the contents of this buffer with the provided *painter*. This is the method that is used to finally join all paint buffers and draw them onto the screen.

Implemented in [QCPPaintBufferPixmap](#).

15.552.3.4 reallocateBuffer()

```
void QCPAbstractPaintBuffer::reallocateBuffer ( ) [protected], [pure virtual]
```

Reallocates the internal buffer with the currently configured size ([setSize](#)) and device pixel ratio, if applicable ([setDevicePixelRatio](#)). It is called as soon as any of those properties are changed on this paint buffer.

Note

Subclasses of [QCPAbstractPaintBuffer](#) must call their reimplementation of this method in their constructor, to perform the first allocation (this can not be done by the base class because calling pure virtual methods in base class constructors is not possible).

Implemented in [QCPPaintBufferPixmap](#).

15.552.3.5 setDevicePixelRatio()

```
void QCPAbstractPaintBuffer::setDevicePixelRatio (
    double ratio )
```

Sets the device pixel ratio to *ratio*. This is useful to render on high-DPI output devices. The ratio is automatically set to the device pixel ratio used by the parent [QCustomPlot](#) instance.

The buffer is reallocated (by calling [reallocateBuffer](#)), so any painters that were obtained by [startPainting](#) are invalidated and must not be used after calling this method.

Note

This method is only available for Qt versions 5.4 and higher.

15.552.3.6 setInvalidated()

```
void QCPAbstractPaintBuffer::setInvalidated (
    bool invalidated = true )
```

Sets the invalidated flag to *invalidated*.

This mechanism is used internally in conjunction with isolated replotting of [QCPLayer](#) instances (in [QCPLayer::lmBuffered](#) mode). If [QCPLayer::replot](#) is called on a buffered layer, i.e. an isolated repaint of only that layer (and its dedicated paint buffer) is requested, [QCustomPlot](#) will decide depending on the invalidated flags of other paint buffers whether it also replots them, instead of only the layer on which the replot was called.

The invalidated flag is set to true when [QCPLayer](#) association has changed, i.e. if layers were added or removed from this buffer, or if they were reordered. It is set to false as soon as all associated [QCPLayer](#) instances are drawn onto the buffer.

Under normal circumstances, it is not necessary to manually call this method.

15.552.3.7 setSize()

```
void QCPAbstractPaintBuffer::setSize (
    const QSize & size )
```

Sets the paint buffer size.

The buffer is reallocated (by calling [reallocateBuffer](#)), so any painters that were obtained by [startPainting](#) are invalidated and must not be used after calling this method.

If *size* is already the current buffer size, this method does nothing.

15.552.3.8 startPainting()

```
QCPPainter * QCPAbstractPaintBuffer::startPainting ( ) [pure virtual]
```

Returns a [QCPPainter](#) which is ready to draw to this buffer. The ownership and thus the responsibility to delete the painter after the painting operations are complete is given to the caller of this method.

Once you are done using the painter, delete the painter and call [donePainting](#).

While a painter generated with this method is active, you must not call [setSize](#), [setDevicePixelRatio](#) or [clear](#).

This method may return 0, if a painter couldn't be activated on the buffer. This usually indicates a problem with the respective painting backend.

Implemented in [QCPPaintBufferPixmap](#).

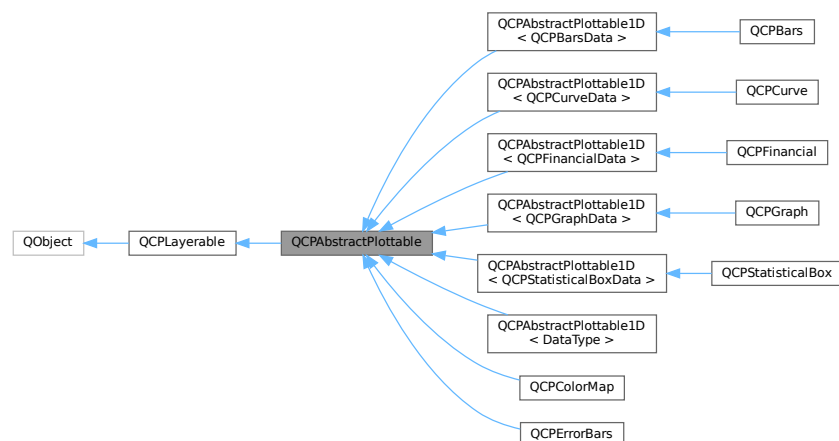
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

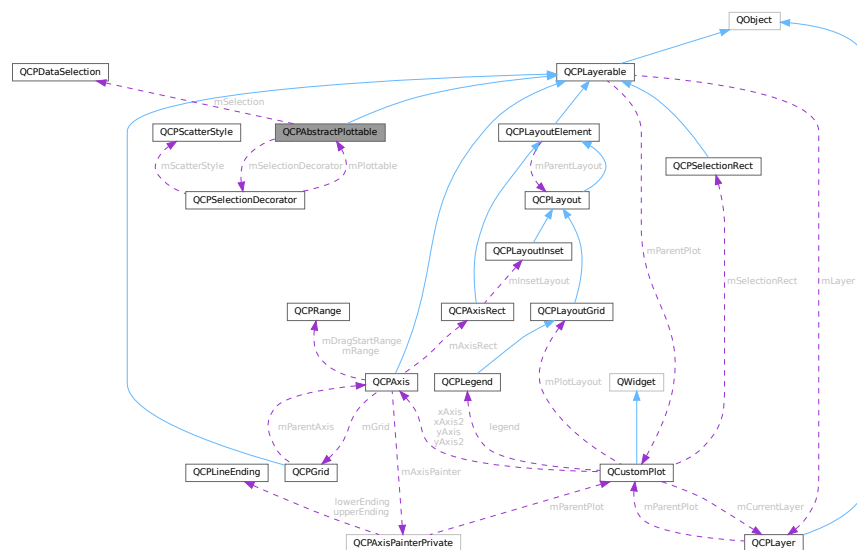
15.553 QCPAbstractPlottable Class Reference

The abstract base class for all data representing objects in a plot.

Inheritance diagram for QCPAbstractPlottable:



Collaboration diagram for QCPAbstractPlottable:



Signals

- void `selectionChanged` (bool `selected`)
- void `selectionChanged` (const `QCPDataSelection` &`selection`)
- void `selectableChanged` (`QCP::SelectionType` `selectable`)

Signals inherited from QCPLayerable

- void layerChanged (QCPLayer *newLayer)

Public Member Functions

- **QCPAbstractPlottable** (QCPAxis *keyAxis, QCPAxis *valueAxis)
- **QString name** () const

- bool **antialiasedFill** () const
- bool **antialiasedScatters** () const
- QPen **pen** () const
- QBrush **brush** () const
- QCPAxis * **keyAxis** () const
- QCPAxis * **valueAxis** () const
- QCP::SelectionType **selectable** () const
- bool **selected** () const
- QCPDataSelection **selection** () const
- QCPSelectionDecorator * **selectionDecorator** () const
- void **setName** (const QString &name)
- void **setAntialiasedFill** (bool enabled)
- void **setAntialiasedScatters** (bool enabled)
- void **setPen** (const QPen &pen)
- void **setBrush** (const QBrush &brush)
- void **setKeyAxis** (QCPAxis *axis)
- void **setValueAxis** (QCPAxis *axis)
- Q_SLOT void **setSelectable** (QCP::SelectionType selectable)
- Q_SLOT void **setSelection** (QCPDataSelection selection)
- void **setSelectionDecorator** (QCPSelectionDecorator *decorator)
- virtual double **selectTest** (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_
DECL_OVERRIDE=0
- virtual QCPPlottableInterface1D * **interface1D** ()
- virtual QCPRange **getKeyRange** (bool &foundRange, QCP::SignDomain inSignDomain=QCP::sdBoth) const
=0
- virtual QCPRange **getValueRange** (bool &foundRange, QCP::SignDomain inSignDomain=QCP::sdBoth,
const QCPRange &inKeyRange=QCPRange()) const =0
- void **coordsToPixels** (double key, double value, double &x, double &y) const
- const QPointF **coordsToPixels** (double key, double value) const
- void **pixelsToCoords** (double x, double y, double &key, double &value) const
- void **pixelsToCoords** (const QPointF &pixelPos, double &key, double &value) const
- void **rescaleAxes** (bool onlyEnlarge=false) const
- void **rescaleKeyAxis** (bool onlyEnlarge=false) const
- void **rescaleValueAxis** (bool onlyEnlarge=false, bool inKeyRange=false) const
- bool **addToLegend** (QCPLegend *legend)
- bool **addToLegend** ()
- bool **removeFromLegend** (QCPLegend *legend) const
- bool **removeFromLegend** () const

Public Member Functions inherited from QCPLayerable

- QCPLayerable (QCustomPlot *plot, QString targetLayer=QString(), QCPLayerable *parentLayerable=nullptr)
- bool **visible** () const
- QCustomPlot * **parentPlot** () const
- QCPLayerable * **parentLayerable** () const
- QCPLayer * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** (QCPLayer *layer)
- bool **setLayer** (const QString &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Protected Member Functions

- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual void [draw](#) (QCPPainter *painter) Q_DECL_OVERRIDE=0
- virtual QCP::Interaction [selectionCategory](#) () const Q_DECL_OVERRIDE
- void [applyDefaultAntialiasingHint](#) (QCPPainter *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [drawLegendIcon](#) (QCPPainter *painter, const QRectF &rect) const =0
- void [applyFillAntialiasingHint](#) (QCPPainter *painter) const
- void [applyScattersAntialiasingHint](#) (QCPPainter *painter) const

Protected Member Functions inherited from QCPLayerable

- virtual void [parentPlotInitialized](#) (QCustomPlot *parentPlot)
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void [initializeParentPlot](#) (QCustomPlot *parentPlot)
- void [setParentLayerable](#) (QCPLayerable *parentLayerable)
- bool [moveToLayer](#) (QCPLayer *layer, bool prepend)
- void [applyAntialiasingHint](#) (QCPPainter *painter, bool localAntialiased, QCP::AntialiasedElement overrideElement) const

Protected Attributes

- QString [mName](#)
- bool [mAntialiasedFill](#)
- bool [mAntialiasedScatters](#)
- QPen [mPen](#)
- QBrush [mBrush](#)
- QPointer< QCPAxis > [mKeyAxis](#)
- QPointer< QCPAxis > [mValueAxis](#)
- QCP::SelectionType [mSelectable](#)
- QCPDataSelection [mSelection](#)
- QCPSelectionDecorator * [mSelectionDecorator](#)

Protected Attributes inherited from QCPLayerable

- bool [mVisible](#)
- QCustomPlot * [mParentPlot](#)
- QPointer< QCPLayerable > [mParentLayerable](#)
- QCPLayer * [mLayer](#)
- bool [mAntialiased](#)

Friends

- class QCustomPlot
- class QCPAxis
- class QCPPlottableLegendItem

15.553.1 Detailed Description

The abstract base class for all data representing objects in a plot.

It defines a very basic interface like name, pen, brush, visibility etc. Since this class is abstract, it can't be instantiated. Use one of the subclasses or create a subclass yourself to create new ways of displaying data (see "Creating own plottables" below). Plottables that display one-dimensional data (i.e. data points have a single key dimension and one or multiple values at each key) are based off of the template subclass [QCPAbstractPlottable1D](#), see details there.

All further specifics are in the subclasses, for example:

- A normal graph with possibly a line and/or scatter points [QCPGraph](#) (typically created with [QCustomPlot::addGraph](#))
- A parametric curve: [QPCurve](#)
- A bar chart: [QCPBars](#)
- A statistical box plot: [QCPStatisticalBox](#)
- A color encoded two-dimensional map: [QCPColorMap](#)
- An OHLC/Candlestick chart: [QCPFinancial](#)

15.553.2 Creating own plottables

Subclassing directly from [QCPAbstractPlottable](#) is only recommended if you wish to display two-dimensional data like [QCPColorMap](#), i.e. two logical key dimensions and one (or more) data dimensions. If you want to display data with only one logical key dimension, you should rather derive from [QCPAbstractPlottable1D](#).

If subclassing [QCPAbstractPlottable](#) directly, these are the pure virtual functions you must implement:

- [selectTest](#)
- [draw](#)
- [drawLegendIcon](#)
- [getKeyRange](#)
- [getValueRange](#)

See the documentation of those functions for what they need to do.

For drawing your plot, you can use the [coordsToPixels](#) functions to translate a point in plot coordinates to pixel coordinates. This function is quite convenient, because it takes the orientation of the key and value axes into account for you (x and y are swapped when the key axis is vertical and the value axis horizontal). If you are worried about performance (i.e. you need to translate many points in a loop like [QCPGraph](#)), you can directly use [QCPAxis::coordToPixel](#). However, you must then take care about the orientation of the axis yourself.

Here are some important members you inherit from [QCPAbstractPlottable](#):

QCustomPlot *mParentPlot	A pointer to the parent QCustomPlot instance. The parent plot is inferred from the axes that are passed in the constructor.
QString mName	The name of the plottable.
QPen mPen	The generic pen of the plottable. You should use this pen for the most prominent data representing lines in the plottable (e.g. QCPGraph uses this pen for its graph lines and scatters)
QBrush mBrush	The generic brush of the plottable. You should use this brush for the most prominent fillable structures in the plottable (e.g. QCPGraph uses this brush to control filling under the graph)
QPointer< QCPAxis > mKeyAxis, mValueAxis	The key and value axes this plottable is attached to. Call their QCPAxis::coordToPixel functions to translate coordinates to pixels in either the key or value dimension. Make sure to check whether the pointer is <code>nullptr</code> before using it. If one of the axes is null, don't draw the plottable.

QCPSelectionDecorator mSelectionDecorator	The currently set selection decorator which specifies how selected data of the plottable shall be drawn and decorated. When drawing your data, you must consult this decorator for the appropriate pen/brush before drawing unselected/selected data segments. Finally, you should call its QCPSelectionDecorator::drawDecoration method at the end of your draw implementation.
QCP::SelectionType mSelectable	In which composition, if at all, this plottable's data may be selected. Enforcing this setting on the data selection is done by QCPAbstractPlottable automatically.
QCPDataSelection mSelection	Holds the current selection state of the plottable's data, i.e. the selected data ranges (QCPDataRange).

15.553.3 Constructor & Destructor Documentation

15.553.3.1 QCPAbstractPlottable()

```
QCPAbstractPlottable::QCPAbstractPlottable (
    QCPAxis * keyAxis,
    QCPAxis * valueAxis )
```

Constructs an abstract plottable which uses *keyAxis* as its key axis ("x") and *valueAxis* as its value axis ("y"). *keyAxis* and *valueAxis* must reside in the same [QCustomPlot](#) instance and have perpendicular orientations. If either of these restrictions is violated, a corresponding message is printed to the debug output (qDebug), the construction is not aborted, though.

Since [QCPAbstractPlottable](#) is an abstract class that defines the basic interface to plottables, it can't be directly instantiated.

You probably want one of the subclasses like [QCPGraph](#) or [QCPCurve](#) instead.

15.553.4 Member Function Documentation

15.553.4.1 addToLegend() [1/2]

```
bool QCPAbstractPlottable::addToLegend ( )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds this plottable to the legend of the parent [QCustomPlot](#) ([QCustomPlot::legend](#)).

See also

[removeFromLegend](#)

15.553.4.2 addToLegend() [2/2]

```
bool QCPAbstractPlottable::addToLegend (
    QCPLegend * legend )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds this plottable to the specified *legend*.

Creates a [QCPPlottableLegendItem](#) which is inserted into the legend. Returns true on success, i.e. when the legend exists and a legend item associated with this plottable isn't already in the legend.

If the plottable needs a more specialized representation in the legend, you can create a corresponding subclass of [QCPPlottableLegendItem](#) and add it to the legend manually instead of calling this method.

See also

[removeFromLegend](#), [QCPLegend::addItem](#)

15.553.4.3 applyDefaultAntialiasingHint()

```
void QCPAbstractPlottable::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Implements [QCPLayerable](#).

15.553.4.4 clipRect()

```
QRect QCPAbstractPlottable::clipRect ( ) const [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.553.4.5 coordsToPixels() [1/2]

```
const QPointF QCPAbstractPlottable::coordsToPixels (
    double key,
    double value ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Transforms the given *key* and *value* to pixel coordinates and returns them in a `QPointF`.

15.553.4.6 coordsToPixels() [2/2]

```
void QCPAbstractPlottable::coordsToPixels (
    double key,
    double value,
    double & x,
    double & y ) const
```

Convenience function for transforming a key/value pair to pixels on the [QCustomPlot](#) surface, taking the orientations of the axes associated with this plottable into account (e.g. whether *key* represents *x* or *y*).

key and *value* are transformed to the coordinates in pixels and are written to *x* and *y*.

See also

[pixelsToCoords](#), [QCPAxis::coordToPixel](#)

15.553.4.7 deselectEvent()

```
void QCPAbstractPlottable::deselectEvent (
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.553.4.8 draw()

```
virtual void QCPAbstractPlottable::draw (
    QCPPainter * painter ) [protected], [pure virtual]
```

Implements [QCPLayerable](#).

15.553.4.9 getKeyRange()

```
QCPRange QCPAbstractPlottable::getKeyRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth ) const [pure virtual]
```

Returns the coordinate range that all data in this plottable span in the key axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getValueRange](#)

Implemented in [QCPGraph](#), [QCPCurve](#), [QCPBars](#), [QCPStatisticalBox](#), [QCPColorMap](#), [QCPFinancial](#), and [QCPErrorBars](#).

15.553.4.10 getValueRange()

```
QCPRange QCPAbstractPlottable::getValueRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth,
    const QCPRange & inKeyRange = QCPRange() ) const [pure virtual]
```

Returns the coordinate range that the data points in the specified key range (*inKeyRange*) span in the value axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

If *inKeyRange* has both lower and upper bound set to zero (is equal to [QCPRange\(\)](#)), all data points are considered, without any restriction on the keys.

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getKeyRange](#)

Implemented in [QCPGraph](#), [QCPCurve](#), [QCPBars](#), [QCPStatisticalBox](#), [QCPColorMap](#), [QCPFinancial](#), and [QCPErrorBars](#).

15.553.4.11 interface1D()

```
QCPPlottableInterface1D * QCPAbstractPlottable::interface1D () [inline], [virtual]
```

If this plottable is a one-dimensional plottable, i.e. it implements the [QCPPlottableInterface1D](#), returns the *this* pointer with that type. Otherwise (e.g. in the case of a [QCPColorMap](#)) returns zero.

You can use this method to gain read access to data coordinates while holding a pointer to the abstract base class only.

Reimplemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPColorMapData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalBoxData >](#), and [QCPErrorBars](#).

15.553.4.12 pixelsToCoords() [1/2]

```
void QCPAbstractPlottable::pixelsToCoords (
    const QPointF & pixelPos,
    double & key,
    double & value ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns the pixel input *pixelPos* as plot coordinates *key* and *value*.

15.553.4.13 pixelsToCoords() [2/2]

```
void QCPAbstractPlottable::pixelsToCoords (
    double x,
    double y,
    double & key,
    double & value ) const
```

Convenience function for transforming a x/y pixel pair on the [QCustomPlot](#) surface to plot coordinates, taking the orientations of the axes associated with this plottable into account (e.g. whether key represents x or y). x and y are transformed to the plot coordinates and are written to *key* and *value*.

See also

[coordsToPixels](#), [QCPAxis::coordToPixel](#)

15.553.4.14 [removeFromLegend\(\)](#) [1/2]

```
bool QCPAbstractPlottable::removeFromLegend ( ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Removes the plottable from the legend of the parent [QCustomPlot](#).

See also

[addToLegend](#)

15.553.4.15 [removeFromLegend\(\)](#) [2/2]

```
bool QCPAbstractPlottable::removeFromLegend (
    QCPLegend * legend ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Removes the plottable from the specified *legend*. This means the [QCPPlottableLegendItem](#) that is associated with this plottable is removed.

Returns true on success, i.e. if the legend exists and a legend item associated with this plottable was found and removed.

See also

[addToLegend](#), [QCPLegend::removeItem](#)

15.553.4.16 [rescaleAxes\(\)](#)

```
void QCPAbstractPlottable::rescaleAxes (
    bool onlyEnlarge = false ) const
```

Rescales the key and value axes associated with this plottable to contain all displayed data, so the whole plottable is visible. If the scaling of an axis is logarithmic, [rescaleAxes](#) will make sure not to rescale to an illegal range i.e. a range containing different signs and/or zero. Instead it will stay in the current sign domain and ignore all parts of the plottable that lie outside of that domain.

onlyEnlarge makes sure the ranges are only expanded, never reduced. So it's possible to show multiple plottables in their entirety by multiple calls to [rescaleAxes](#) where the first call has *onlyEnlarge* set to false (the default), and all subsequent set to true.

See also

[rescaleKeyAxis](#), [rescaleValueAxis](#), [QCustomPlot::rescaleAxes](#), [QCPAxis::rescale](#)

15.553.4.17 [rescaleKeyAxis\(\)](#)

```
void QCPAbstractPlottable::rescaleKeyAxis (
    bool onlyEnlarge = false ) const
```

Rescales the key axis of the plottable so the whole plottable is visible.

See [rescaleAxes](#) for detailed behaviour.

15.553.4.18 rescaleValueAxis()

```
void QCPAbstractPlottable::rescaleValueAxis (
    bool onlyEnlarge = false,
    bool inKeyRange = false ) const
```

Rescales the value axis of the plottable so the whole plottable is visible. If *inKeyRange* is set to true, only the data points which are in the currently visible key axis range are considered.

Returns true if the axis was actually scaled. This might not be the case if this plottable has an invalid range, e.g. because it has no data points.

See [rescaleAxes](#) for detailed behaviour.

15.553.4.19 selectableChanged

```
void QCPAbstractPlottable::selectableChanged (
    QCP::SelectionType selectable ) [signal]
```

This signal is emitted when the selectability of this plottable has changed.

See also

[setSelectable](#)

15.553.4.20 selected()

```
bool QCPAbstractPlottable::selected ( ) const [inline]
```

Returns true if there are any data points of the plottable currently selected. Use [selection](#) to retrieve the current [QCPDataSelection](#).

15.553.4.21 selectEvent()

```
void QCPAbstractPlottable::selectEvent (
    QMouseEvent * event,
    bool additive,
    const QVariant & details,
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.553.4.22 selection()

```
QCPDataSelection QCPAbstractPlottable::selection ( ) const [inline]
```

Returns a [QCPDataSelection](#) encompassing all the data points that are currently selected on this plottable.

See also

[selected](#), [setSelection](#), [setSelectable](#)

15.553.4.23 selectionCategory()

```
QCP::Interaction QCPAbstractPlottable::selectionCategory ( ) const [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.553.4.24 selectionChanged [1/2]

```
void QCPAbstractPlottable::selectionChanged (
    bool selected ) [signal]
```

This signal is emitted when the selection state of this plottable has changed, either by user interaction or by a direct call to [setSelection](#). The parameter *selected* indicates whether there are any points selected or not.

See also

[selectionChanged\(const QCPDataSelection &selection\)](#)

15.553.4.25 selectionChanged [2/2]

```
void QCPAbstractPlottable::selectionChanged (
    const QCPDataSelection & selection ) [signal]
```

This signal is emitted when the selection state of this plottable has changed, either by user interaction or by a direct call to [setSelection](#). The parameter *selection* holds the currently selected data ranges.

See also

[selectionChanged\(bool selected\)](#)

15.553.4.26 selectionDecorator()

```
QCPSelectionDecorator * QCPAbstractPlottable::selectionDecorator ( ) const [inline]
```

Provides access to the selection decorator of this plottable. The selection decorator controls how selected data ranges are drawn (e.g. their pen color and fill), see [QCPSelectionDecorator](#) for details.

If you wish to use an own [QCPSelectionDecorator](#) subclass, pass an instance of it to [setSelectionDecorator](#).

15.553.4.27 selectTest()

```
virtual double QCPAbstractPlottable::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [pure virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the *selectionTolerance* multiplied by 0.99). Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than $0.99 \cdot \text{selectionTolerance}$).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the *mouseReleaseEvent* occurs, and the finally selected object is notified via the *selectEvent/* *deselectEvent* methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to *selectEvent* when the parent [QCustomPlot](#) decides on the basis of this *selectTest* call, that the object was successfully selected. The subsequent call to *selectEvent* will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in [selectTest](#). The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent *selectEvent*, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass *nullptr* as *details* to indicate that you are not interested in those selection details.

See also

[selectEvent](#), [deselectEvent](#), [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Reimplemented from [QCPLayerable](#).

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalBox >](#), [QCPGraph](#), [QCPCurve](#), [QCPBars](#), [QCPStatisticalBox](#), [QCPColorMap](#), [QCPFinancial](#), and [QCPErrorBars](#).

15.553.4.28 setAntialiasedFill()

```
void QCPAbstractPlottable::setAntialiasedFill (
    bool enabled )
```

Sets whether fills of this plottable are drawn antialiased or not.

Note that this setting may be overridden by [QCustomPlot::setAntialiasedElements](#) and [QCustomPlot::setNotAntialiasedElements](#).

15.553.4.29 setAntialiasedScatters()

```
void QCPAbstractPlottable::setAntialiasedScatters (
    bool enabled )
```

Sets whether the scatter symbols of this plottable are drawn antialiased or not.

Note that this setting may be overridden by [QCustomPlot::setAntialiasedElements](#) and [QCustomPlot::setNotAntialiasedElements](#).

15.553.4.30 setBrush()

```
void QCPAbstractPlottable::setBrush (
    const QBrush & brush )
```

The brush is used to draw basic fills of the plottable representation in the plot. The Fill can be a color, gradient or texture, see the usage of QBrush.

For example, the [QCPGraph](#) subclass draws the fill under the graph with this brush, when it's not set to Qt::NoBrush.

See also

[setPen](#)

15.553.4.31 setKeyAxis()

```
void QCPAbstractPlottable::setKeyAxis (
    QCPAxis * axis )
```

The key axis of a plottable can be set to any axis of a [QCustomPlot](#), as long as it is orthogonal to the plottable's value axis. This function performs no checks to make sure this is the case. The typical mathematical choice is to use the x-axis ([QCustomPlot::xAxis](#)) as key axis and the y-axis ([QCustomPlot::yAxis](#)) as value axis.

Normally, the key and value axes are set in the constructor of the plottable (or [QCustomPlot::addGraph](#) when working with QCPGraphs through the dedicated graph interface).

See also

[setValueAxis](#)

15.553.4.32 setName()

```
void QCPAbstractPlottable::setName (
    const QString & name )
```

The name is the textual representation of this plottable as it is displayed in the legend ([QCPLegend](#)). It may contain any UTF-8 characters, including newlines.

15.553.4.33 setPen()

```
void QCPAbstractPlottable::setPen (
    const QPen & pen )
```

The pen is used to draw basic lines that make up the plottable representation in the plot.

For example, the [QCPGraph](#) subclass draws its graph lines with this pen.

See also

[setBrush](#)

15.553.4.34 `setSelectable()`

```
void QCPAbstractPlottable::setSelectable (
    QCP::SelectionType selectable )
```

Sets whether and to which granularity this plottable can be selected.

A selection can happen by clicking on the [QCustomPlot](#) surface (When [QCustomPlot::setInteractions](#) contains [QCP::iSelectPlottables](#)), by dragging a selection rect (When [QCustomPlot::setSelectionRectMode](#) is [QCP::srmSelect](#)), or programmatically by calling [setSelection](#).

See also

[setSelection](#), [QCP::SelectionType](#)

15.553.4.35 `setSelection()`

```
void QCPAbstractPlottable::setSelection (
    QCPDataSelection selection )
```

Sets which data ranges of this plottable are selected. Selected data ranges are drawn differently (e.g. color) in the plot. This can be controlled via the selection decorator (see [selectionDecorator](#)).

The entire selection mechanism for plottables is handled automatically when [QCustomPlot::setInteractions](#) contains [iSelectPlottables](#). You only need to call this function when you wish to change the selection state programmatically. Using [setSelectable](#) you can further specify for each plottable whether and to which granularity it is selectable. If *selection* is not compatible with the current [QCP::SelectionType](#) set via [setSelectable](#), the resulting selection will be adjusted accordingly (see [QCPDataSelection::enforceType](#)).

emits the [selectionChanged](#) signal when *selected* is different from the previous selection state.

See also

[setSelectable](#), [selectTest](#)

15.553.4.36 `setSelectionDecorator()`

```
void QCPAbstractPlottable::setSelectionDecorator (
    QCPSelectionDecorator * decorator )
```

Use this method to set an own [QCPSelectionDecorator](#) (subclass) instance. This allows you to customize the visual representation of selected data ranges further than by using the default [QCPSelectionDecorator](#).

The plottable takes ownership of the *decorator*.

The currently set decorator can be accessed via [selectionDecorator](#).

15.553.4.37 `setValueAxis()`

```
void QCPAbstractPlottable::setValueAxis (
    QCPAxis * axis )
```

The value axis of a plottable can be set to any axis of a [QCustomPlot](#), as long as it is orthogonal to the plottable's key axis. This function performs no checks to make sure this is the case. The typical mathematical choice is to use the x-axis ([QCustomPlot::xAxis](#)) as key axis and the y-axis ([QCustomPlot::yAxis](#)) as value axis.

Normally, the key and value axes are set in the constructor of the plottable (or [QCustomPlot::addGraph](#) when working with [QCPGraphs](#) through the dedicated graph interface).

See also

[setKeyAxis](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

- virtual double [dataSortKey](#) (int index) const Q_DECL_OVERRIDE
- virtual double [dataMainValue](#) (int index) const Q_DECL_OVERRIDE
- virtual [QCPRange dataValueRange](#) (int index) const Q_DECL_OVERRIDE
- virtual QPointF [dataPixelPosition](#) (int index) const Q_DECL_OVERRIDE
- virtual bool [sortKeysMainKey](#) () const Q_DECL_OVERRIDE
- virtual [QCPDataSelection selectTestRect](#) (const QRectF &rect, bool onlySelectable) const Q_DECL_OVERRIDE
- virtual int [findBegin](#) (double sortKey, bool expandedRange=true) const Q_DECL_OVERRIDE
- virtual int [findEnd](#) (double sortKey, bool expandedRange=true) const Q_DECL_OVERRIDE
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE
- virtual [QCPPlottableInterface1D * interface1D](#) () Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractPlottable](#)

- [QCPAbstractPlottable](#) ([QCPAxis *keyAxis](#), [QCPAxis *valueAxis](#))
- QString [name](#) () const
- bool [antialiasedFill](#) () const
- bool [antialiasedScatters](#) () const
- QPen [pen](#) () const
- QBrush [brush](#) () const
- [QCPAxis * keyAxis](#) () const
- [QCPAxis * valueAxis](#) () const
- [QCP::SelectionType selectable](#) () const
- bool [selected](#) () const
- [QCPDataSelection selection](#) () const
- [QCPSelectionDecorator * selectionDecorator](#) () const
- void [setName](#) (const QString &name)
- void [setAntialiasedFill](#) (bool enabled)
- void [setAntialiasedScatters](#) (bool enabled)
- void [setPen](#) (const QPen &pen)
- void [setBrush](#) (const QBrush &brush)
- void [setKeyAxis](#) ([QCPAxis *axis](#))
- void [setValueAxis](#) ([QCPAxis *axis](#))
- Q_SLOT void [setSelectable](#) ([QCP::SelectionType selectable](#))
- Q_SLOT void [setSelection](#) ([QCPDataSelection selection](#))
- void [setSelectionDecorator](#) ([QCPSelectionDecorator *decorator](#))
- virtual [QCPRange getKeyRange](#) (bool &foundRange, [QCP::SignDomain inSignDomain=QCP::sdBoth](#)) const =0
- virtual [QCPRange getValueRange](#) (bool &foundRange, [QCP::SignDomain inSignDomain=QCP::sdBoth](#), const [QCPRange &inKeyRange=QCPRange\(\)](#)) const =0
- void [coordsToPixels](#) (double key, double value, double &x, double &y) const
- const QPointF [coordsToPixels](#) (double key, double value) const
- void [pixelsToCoords](#) (double x, double y, double &key, double &value) const
- void [pixelsToCoords](#) (const QPointF &pixelPos, double &key, double &value) const
- void [rescaleAxes](#) (bool onlyEnlarge=false) const
- void [rescaleKeyAxis](#) (bool onlyEnlarge=false) const
- void [rescaleValueAxis](#) (bool onlyEnlarge=false, bool inKeyRange=false) const
- bool [addToLegend](#) ([QCPLegend *legend](#))
- bool [addToLegend](#) ()
- bool [removeFromLegend](#) ([QCPLegend *legend](#)) const
- bool [removeFromLegend](#) () const

Public Member Functions inherited from QCPLayerable

- [QCPLayerable](#) (QCustomPlot *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- QCustomPlot * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const QString &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Protected Member Functions

- void **getDataSegments** (QList< [QCPDataRange](#) > &selectedSegments, QList< [QCPDataRange](#) > &unselectedSegments) const
- void **drawPolyline** ([QCPPainter](#) *painter, const QVector< QPointF > &lineData) const

Protected Member Functions inherited from QCPAbstractPlottable

- virtual QRect **clipRect** () const Q_DECL_OVERRIDE
- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE=0
- virtual [QCP::Interaction](#) **selectionCategory** () const Q_DECL_OVERRIDE
- void **applyDefaultAntialiasingHint** ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void **deselectEvent** (bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void **drawLegendIcon** ([QCPPainter](#) *painter, const QRectF &rect) const =0
- void **applyFillAntialiasingHint** ([QCPPainter](#) *painter) const
- void **applyScattersAntialiasingHint** ([QCPPainter](#) *painter) const

Protected Member Functions inherited from QCPLayerable

- virtual void **parentPlotInitialized** (QCustomPlot *parentPlot)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** (QCustomPlot *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QSharedPointer< [QCPDataContainer](#)< DataType > > **mDataContainer**

Protected Attributes inherited from [QCPAbstractPlottable](#)

- `QString mName`
- `bool mAntialiasedFill`
- `bool mAntialiasedScatters`
- `QPen mPen`
- `QBrush mBrush`
- `QPointer< QCPAxis > mKeyAxis`
- `QPointer< QCPAxis > mValueAxis`
- `QCP::SelectionType mSelectable`
- `QCPDataSelection mSelection`
- `QCPSelectionDecorator * mSelectionDecorator`

Protected Attributes inherited from [QCPLayerable](#)

- `bool mVisible`
- `QCustomPlot * mParentPlot`
- `QPointer< QCPLayerable > mParentLayerable`
- `QCPLayer * mLayer`
- `bool mAntialiased`

Additional Inherited Members

Signals inherited from [QCPAbstractPlottable](#)

- `void selectionChanged (bool selected)`
- `void selectionChanged (const QCPDataSelection &selection)`
- `void selectableChanged (QCP::SelectionType selectable)`

Signals inherited from [QCPLayerable](#)

- `void layerChanged (QCPLayer *newLayer)`

15.554.1 Detailed Description

`template<class DataType>`
`class QCPAbstractPlottable1D< DataType >`

A template base class for plottables with one-dimensional data.

This template class derives from [QCPAbstractPlottable](#) and from the abstract interface [QCPPlottableInterface1D](#). It serves as a base class for all one-dimensional data (i.e. data with one key dimension), such as [QCPGraph](#) and [QCPCurve](#).

The template parameter *DataType* is the type of the data points of this plottable (e.g. [QCPGraphData](#) or [QCPCurveData](#)). The main purpose of this base class is to provide the member *mDataContainer* (a shared pointer to a [QCPDataContainer<DataType>](#)) and implement the according virtual methods of the [QCPPlottableInterface1D](#), such that most subclassed plottables don't need to worry about this anymore.

Further, it provides a convenience method for retrieving selected/unselected data segments via [getDataSegments](#). This is useful when subclasses implement their draw method and need to draw selected segments with a different pen/brush than unselected segments (also see [QCPSelectionDecorator](#)).

This class implements basic functionality of [QCPAbstractPlottable::selectTest](#) and [QCPPlottableInterface1D::selectTestRect](#), assuming point-like data points, based on the 1D data interface. In spite of that, most plottable subclasses will want to reimplement those methods again, to provide a more accurate hit test based on their specific data visualization geometry.

15.554.2 Constructor & Destructor Documentation

15.554.2.1 QCPAbstractPlottable1D()

```
template<class DataType >
QCPAbstractPlottable1D< DataType >::QCPAbstractPlottable1D (
    QCPAxis * keyAxis,
    QCPAxis * valueAxis )
```

Forwards *keyAxis* and *valueAxis* to the [QCPAbstractPlottable](#) constructor and allocates the *mDataContainer*.

15.554.3 Member Function Documentation

15.554.3.1 dataCount()

```
template<class DataType >
int QCPAbstractPlottable1D< DataType >::dataCount ( ) const [virtual]
```

Returns the number of data points of the plottable.

Implements [QCPPlottableInterface1D](#).

15.554.3.2 dataMainKey()

```
template<class DataType >
double QCPAbstractPlottable1D< DataType >::dataMainKey (
    int index ) const [virtual]
```

Returns the main key of the data point at the given *index*.

What the main key is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implements [QCPPlottableInterface1D](#).

15.554.3.3 dataMainValue()

```
template<class DataType >
double QCPAbstractPlottable1D< DataType >::dataMainValue (
    int index ) const [virtual]
```

Returns the main value of the data point at the given *index*.

What the main value is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implements [QCPPlottableInterface1D](#).

15.554.3.4 dataPixelPosition()

```
template<class DataType >
QPointF QCPAbstractPlottable1D< DataType >::dataPixelPosition (
    int index ) const [virtual]
```

Returns the pixel position on the widget surface at which the data point at the given *index* appears.

Usually this corresponds to the point of *dataMainKey*/ *dataMainValue*, in pixel coordinates. However, depending on the plottable, this might be a different apparent position than just a coord-to-pixel transform of those values. For example, [QCPBars](#) apparent data values can be shifted depending on their stacking, bar grouping or configured base value.

Implements [QCPPlottableInterface1D](#).

Reimplemented in [QCPBars](#).

15.554.3.5 dataSortKey()

```
template<class DataType >
double QCPAbstractPlottable1D< DataType >::dataSortKey (
    int index ) const [virtual]
```

Returns the sort key of the data point at the given *index*.

What the sort key is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implements [QCPPlottableInterface1D](#).

15.554.3.6 dataValueRange()

```
template<class DataType >
QCPRange QCPAbstractPlottable1D< DataType >::dataValueRange (
    int index ) const [virtual]
```

Returns the value range of the data point at the given *index*.

What the value range is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implements [QCPPlottableInterface1D](#).

15.554.3.7 drawPolyline()

```
template<class DataType >
void QCPAbstractPlottable1D< DataType >::drawPolyline (
    QCPPainter * painter,
    const QVector< QPointF > & lineData ) const [protected]
```

A helper method which draws a line with the passed *painter*, according to the pixel data in *lineData*. NaN points create gaps in the line, as expected from [QCustomPlot](#)'s plottables (this is the main difference to [QPainter](#)'s regular [drawPolyline](#), which handles NaNs by lagging or crashing).

Further it uses a faster line drawing technique based on [QCPPainter::drawLine](#) rather than [QPainter::drawPolyline](#) if the configured [QCustomPlot::setPlottingHints\(\)](#) and *painter* style allows.

15.554.3.8 findBegin()

```
template<class DataType >
int QCPAbstractPlottable1D< DataType >::findBegin (
    double sortKey,
    bool expandedRange = true ) const [virtual]
```

Returns the index of the data point with a (sort-)key that is equal to, just below, or just above *sortKey*. If *expandedRange* is true, the data point just below *sortKey* will be considered, otherwise the one just above.

This can be used in conjunction with [findEnd](#) to iterate over data points within a given key range, including or excluding the bounding data points that are just beyond the specified range.

If *expandedRange* is true but there are no data points below *sortKey*, 0 is returned.

If the container is empty, returns 0 (in that case, [findEnd](#) will also return 0, so a loop using these methods will not iterate over the index 0).

See also

[findEnd](#), [QCPDataContainer::findBegin](#)

Implements [QCPPlottableInterface1D](#).

15.554.3.9 findEnd()

```
template<class DataType >
int QCPAbstractPlottable1D< DataType >::findEnd (
    double sortKey,
    bool expandedRange = true ) const [virtual]
```

Returns the index one after the data point with a (sort-)key that is equal to, just above, or just below *sortKey*. If *expandedRange* is true, the data point just above *sortKey* will be considered, otherwise the one just below.

This can be used in conjunction with [findBegin](#) to iterate over data points within a given key range, including the bounding data points that are just below and above the specified range.

If *expandedRange* is true but there are no data points above *sortKey*, the index just above the highest data point is returned.

If the container is empty, returns 0.

See also

[findBegin](#), [QCPDataContainer::findEnd](#)

Implements [QCPPlottableInterface1D](#).

15.554.3.10 getDataSegments()

```
template<class DataType >
void QCPAbstractPlottable1D< DataType >::getDataSegments (
    QList< QCPDataRange > & selectedSegments,
    QList< QCPDataRange > & unselectedSegments ) const [protected]
```

Splits all data into selected and unselected segments and outputs them via *selectedSegments* and *unselectedSegments*, respectively.

This is useful when subclasses implement their draw method and need to draw selected segments with a different pen/brush than unselected segments (also see [QCPSelectionDecorator](#)).

See also

[setSelection](#)

15.554.3.11 interface1D()

```
template<class DataType >
QCPPlottableInterface1D * QCPAbstractPlottable1D< DataType >::interface1D ( ) [inline], [virtual]
```

Returns a [QCPPlottableInterface1D](#) pointer to this plottable, providing access to its 1D interface.

\seebaseclassmethod

Reimplemented from [QCPAbstractPlottable](#).

15.554.3.12 selectTest()

```
template<class DataType >
double QCPAbstractPlottable1D< DataType >::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Implements a point-selection algorithm assuming the data (accessed via the 1D data interface) is point-like. Most subclasses will want to reimplement this method again, to provide a more accurate hit test based on the true data visualization geometry.

If *details* is not 0, it will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

\seebaseclassmethod

Implements [QCPAbstractPlottable](#).

Reimplemented in [QCPGraph](#), [QCPCurve](#), [QCPBars](#), [QCPStatisticalBox](#), and [QCPFinancial](#).

15.554.3.13 selectTestRect()

```
template<class DataType >
QCPDataSelection QCPAbstractPlottable1D< DataType >::selectTestRect (
    const QRectF & rect,
    bool onlySelectable ) const [virtual]
```

Implements a rect-selection algorithm assuming the data (accessed via the 1D data interface) is point-like. Most subclasses will want to reimplement this method again, to provide a more accurate hit test based on the true data visualization geometry.

\seebaseclassmethod

Implements [QCPPlottableInterface1D](#).

Reimplemented in [QCPBars](#), [QCPStatisticalBox](#), and [QCPFinancial](#).

15.554.3.14 sortKeysMainKey()

```
template<class DataType >
bool QCPAbstractPlottable1D< DataType >::sortKeyIsMainKey ( ) const [virtual]
```

Returns whether the sort key ([dataSortKey](#)) is identical to the main key ([dataMainKey](#)).

What the sort and main keys are, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implements [QCPPlottableInterface1D](#).

The documentation for this class was generated from the following file:

Signals

- void [rangeChanged](#) (const [QCPRange](#) &newRange)
- void [rangeChanged](#) (const [QCPRange](#) &newRange, const [QCPRange](#) &oldRange)
- void [scaleTypeChanged](#) ([QCPAxis::ScaleType](#) scaleType)
- void [selectionChanged](#) (const [QCPAxis::SelectableParts](#) &parts)
- void [selectableChanged](#) (const [QCPAxis::SelectableParts](#) &parts)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

Public Member Functions

- [QCPAxis](#) ([QCPAxisRect](#) *parent, [AxisType](#) type)
- [AxisType](#) **axisType** () const
- [QCPAxisRect](#) * **axisRect** () const
- [ScaleType](#) **scaleType** () const
- const [QCPRange](#) **range** () const
- bool **rangeReversed** () const
- QSharedPointer< [QCPAxisTicker](#) > **ticker** () const
- bool **ticks** () const
- bool **tickLabels** () const
- int **tickLabelPadding** () const
- QFont **tickLabelFont** () const
- QColor **tickLabelColor** () const
- double **tickLabelRotation** () const
- [LabelSide](#) **tickLabelSide** () const
- QString **numberFormat** () const
- int **numberPrecision** () const
- QVector< double > **tickVector** () const
- QVector< QString > **tickVectorLabels** () const
- int **tickLengthIn** () const
- int **tickLengthOut** () const
- bool **subTicks** () const
- int **subTickLengthIn** () const
- int **subTickLengthOut** () const
- QPen **basePen** () const
- QPen **tickPen** () const
- QPen **subTickPen** () const
- QFont **labelFont** () const
- QColor **labelColor** () const
- QString **label** () const
- int **labelPadding** () const
- int **padding** () const
- int **offset** () const
- SelectableParts **selectedParts** () const
- SelectableParts **selectableParts** () const
- QFont **selectedTickLabelFont** () const
- QFont **selectedLabelFont** () const
- QColor **selectedTickLabelColor** () const
- QColor **selectedLabelColor** () const
- QPen **selectedBasePen** () const
- QPen **selectedTickPen** () const
- QPen **selectedSubTickPen** () const
- [QCPLineEnding](#) **lowerEnding** () const

- [QCPLineEnding](#) **upperEnding** () const
- [QCPGrid](#) * **grid** () const
- Q_SLOT void **setScaleType** ([QCPAxis::ScaleType](#) type)
- Q_SLOT void **setRange** (const [QCPRange](#) &range)
- void **setRange** (double lower, double upper)
- void **setRange** (double position, double size, Qt::AlignmentFlag alignment)
- void **setRangeLower** (double lower)
- void **setRangeUpper** (double upper)
- void **setRangeReversed** (bool reversed)
- void **setTicker** (QSharedPointer< [QCPAxisTicker](#) > **ticker**)
- void **setTicks** (bool show)
- void **setTickLabels** (bool show)
- void **setTickLabelPadding** (int padding)
- void **setTickLabelFont** (const QFont &font)
- void **setTickLabelColor** (const QColor &color)
- void **setTickLabelRotation** (double degrees)
- void **setTickLabelSide** ([LabelSide](#) side)
- void **setNumberFormat** (const QString &formatCode)
- void **setNumberPrecision** (int precision)
- void **setTickLength** (int inside, int outside=0)
- void **setTickLengthIn** (int inside)
- void **setTickLengthOut** (int outside)
- void **setSubTicks** (bool show)
- void **setSubTickLength** (int inside, int outside=0)
- void **setSubTickLengthIn** (int inside)
- void **setSubTickLengthOut** (int outside)
- void **setBasePen** (const QPen &pen)
- void **setTickPen** (const QPen &pen)
- void **setSubTickPen** (const QPen &pen)
- void **setLabelFont** (const QFont &font)
- void **setLabelColor** (const QColor &color)
- void **setLabel** (const QString &str)
- void **setLabelPadding** (int padding)
- void **setPadding** (int padding)
- void **setOffset** (int offset)
- void **setSelectedTickLabelFont** (const QFont &font)
- void **setSelectedLabelFont** (const QFont &font)
- void **setSelectedTickLabelColor** (const QColor &color)
- void **setSelectedLabelColor** (const QColor &color)
- void **setSelectedBasePen** (const QPen &pen)
- void **setSelectedTickPen** (const QPen &pen)
- void **setSelectedSubTickPen** (const QPen &pen)
- Q_SLOT void **setSelectableParts** (const [QCPAxis::SelectableParts](#) &selectableParts)
- Q_SLOT void **setSelectedParts** (const [QCPAxis::SelectableParts](#) &selectedParts)
- void **setLowerEnding** (const [QCPLineEnding](#) &ending)
- void **setUpperEnding** (const [QCPLineEnding](#) &ending)
- virtual double **selectTest** (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const [Q_↔](#)
DECL_OVERRIDE
- Qt::Orientation **orientation** () const
- int **pixelOrientation** () const
- void **moveRange** (double diff)
- void **scaleRange** (double factor)
- void **scaleRange** (double factor, double center)
- void **setScaleRatio** (const [QCPAxis](#) *otherAxis, double ratio=1.0)
- void **rescale** (bool onlyVisiblePlottables=false)

- double [pixelToCoord](#) (double value) const
- double [coordToPixel](#) (double value) const
- [SelectablePart](#) [getPartAt](#) (const QPointF &pos) const
- QList< [QCPAbstractPlottable](#) * > [plottables](#) () const
- QList< [QCPGraph](#) * > [graphs](#) () const
- QList< [QCPAbstractItem](#) * > [items](#) () const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Static Public Member Functions

- static [AxisType](#) [marginSideToAxisType](#) ([QCP::MarginSide](#) side)
- static Qt::Orientation [orientation](#) ([AxisType](#) type)
- static [AxisType](#) [opposite](#) ([AxisType](#) type)

Protected Member Functions

- virtual int **calculateMargin** ()
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE
- virtual void [wheelEvent](#) (QWheelEvent *event) Q_DECL_OVERRIDE
- void **setupTickVectors** ()
- QPen **getBasePen** () const
- QPen **getTickPen** () const
- QPen **getSubTickPen** () const
- QFont **getTickLabelFont** () const
- QFont **getLabelFont** () const
- QColor **getTickLabelColor** () const
- QColor **getLabelColor** () const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual QRect **clipRect** () const
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [AxisType](#) **mAxisType**
- [QCPAxisRect](#) * **mAxisRect**
- int **mPadding**
- Qt::Orientation **mOrientation**
- SelectableParts **mSelectableParts**
- SelectableParts **mSelectedParts**
- QPen **mBasePen**
- QPen **mSelectedBasePen**
- QString **mLabel**
- QFont **mLabelFont**
- QFont **mSelectedLabelFont**
- QColor **mLabelColor**
- QColor **mSelectedLabelColor**
- bool **mTickLabels**
- QFont **mTickLabelFont**
- QFont **mSelectedTickLabelFont**
- QColor **mTickLabelColor**
- QColor **mSelectedTickLabelColor**
- int **mNumberPrecision**
- QLatin1Char **mNumberFormatChar**
- bool **mNumberBeautifulPowers**
- bool **mTicks**
- bool **mSubTicks**
- QPen **mTickPen**
- QPen **mSelectedTickPen**
- QPen **mSubTickPen**
- QPen **mSelectedSubTickPen**
- [QCPRange](#) **mRange**
- bool **mRangeReversed**
- [ScaleType](#) **mScaleType**
- [QCPGrid](#) * **mGrid**
- [QCPAxisPainterPrivate](#) * **mAxisPainter**
- QSharedPointer< [QCPAxisTicker](#) > **mTicker**
- QVector< double > **mTickVector**
- QVector< QString > **mTickVectorLabels**
- QVector< double > **mSubTickVector**
- bool **mCachedMarginValid**
- int **mCachedMargin**
- bool **mDragging**
- [QCPRange](#) **mDragStartRange**
- QCP::AntialiasedElements **mAADragBackup**
- QCP::AntialiasedElements **mNotAADragBackup**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class **QCustomPlot**
- class **QCPGrid**
- class **QCPAxisRect**

15.555.1 Detailed Description

Manages a single axis inside a [QCustomPlot](#).

Usually doesn't need to be instantiated externally. Access [QCustomPlot](#)'s default four axes via [QCustomPlot::xAxis](#) (bottom), [QCustomPlot::yAxis](#) (left), [QCustomPlot::xAxis2](#) (top) and [QCustomPlot::yAxis2](#) (right).

Axes are always part of an axis rect, see [QCPAxisRect](#).

Naming convention of axis parts

Overview of the spacings and paddings that define the geometry of an axis. The dashed gray line on the left represents the [QCustomPlot](#) widget border.

Each axis holds an instance of [QCPAxisTicker](#) which is used to generate the tick coordinates and tick labels. You can access the currently installed [ticker](#) or set a new one (possibly one of the specialized subclasses, or your own subclass) via [setTicker](#). For details, see the documentation of [QCPAxisTicker](#).

15.555.2 Member Enumeration Documentation**15.555.2.1 AxisType**

enum [QCPAxis::AxisType](#)

Defines at which side of the axis rect the axis will appear. This also affects how the tick marks are drawn, on which side the labels are placed etc.

Enumerator

atLeft	0x01 Axis is vertical and on the left side of the axis rect
atRight	0x02 Axis is vertical and on the right side of the axis rect
atTop	0x04 Axis is horizontal and on the top side of the axis rect
atBottom	0x08 Axis is horizontal and on the bottom side of the axis rect

15.555.2.2 LabelSide

enum [QCPAxis::LabelSide](#)

Defines on which side of the axis the tick labels (numbers) shall appear.

See also

[setTickLabelSide](#)

Enumerator

IsInside	Tick labels will be displayed inside the axis rect and clipped to the inner axis rect.
IsOutside	Tick labels will be displayed outside the axis rect.

15.555.2.3 ScaleType

enum [QCPAxis::ScaleType](#)

Defines the scale of an axis.

See also

[setScaleType](#)

Enumerator

stLinear	Linear scaling.
stLogarithmic	Logarithmic scaling with correspondingly transformed axis coordinates (possibly also setTicker to a QCPAxisTickerLog instance).

15.555.2.4 SelectablePart

enum [QCPAxis::SelectablePart](#)

Defines the selectable parts of an axis.

See also

[setSelectableParts](#), [setSelectedParts](#)

Enumerator

spNone	None of the selectable parts.
spAxis	The axis backbone and tick marks.
spTickLabels	Tick labels (numbers) of this axis (as a whole, not individually)
spAxisLabel	The axis label.

15.555.3 Constructor & Destructor Documentation

15.555.3.1 QCPAxis()

```
QCPAxis::QCPAxis (
    QCPAxisRect * parent,
    AxisType type ) [explicit]
```

Constructs an Axis instance of Type *type* for the axis rect *parent*.

Usually it isn't necessary to instantiate axes directly, because you can let [QCustomPlot](#) create them for you with [QCPAxisRect::addAxis](#). If you want to use own QCPAxis-subclasses however, create them manually and then inject them also via [QCPAxisRect::addAxis](#).

15.555.4 Member Function Documentation

15.555.4.1 applyDefaultAntialiasingHint()

```
void QCPAxis::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Implements [QCPLayerable](#).

15.555.4.2 coordToPixel()

```
double QCPAxis::coordToPixel (
    double value ) const
```

Transforms *value*, in coordinates of the axis, to pixel coordinates of the [QCustomPlot](#) widget.

15.555.4.3 deselectEvent()

```
void QCPAxis::deselectEvent (
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.555.4.4 draw()

```
void QCPAxis::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPLayerable](#).

15.555.4.5 getPartAt()

```
QCPAxis::SelectablePart QCPAxis::getPartAt (
    const QPointF & pos ) const
```

Returns the part of the axis that is hit by *pos* (in pixels). The return value of this function is independent of the user-selectable parts defined with [setSelectableParts](#). Further, this function does not change the current selection state of the axis.

If the axis is not visible ([setVisible](#)), this function always returns [spNone](#).

See also

[setSelectedParts](#), [setSelectableParts](#), [QCustomPlot::setInteractions](#)

15.555.4.6 graphs()

```
QList< QCPGraph * > QCPAxis::graphs ( ) const
```

Returns a list of all the graphs that have this axis as key or value axis.

See also

[plottables](#), [items](#)

15.555.4.7 grid()

```
QCPGrid * QCPAxis::grid ( ) const [inline]
```

Returns the [QCPGrid](#) instance belonging to this axis. [Access](#) it to set details about the way the grid is displayed.

15.555.4.8 items()

```
QList< QCPAbstractItem * > QCPAxis::items ( ) const
```

Returns a list of all the items that are associated with this axis. An item is considered associated with an axis if at least one of its positions uses the axis as key or value axis.

See also

[plottables](#), [graphs](#)

15.555.4.9 marginSideToAxisType()

```
QCPAxis::AxisType QCPAxis::marginSideToAxisType (
    QCP::MarginSide side ) [static]
```

Transforms a margin side to the logically corresponding axis type. ([QCP::msLeft](#) to [QCPAxis::atLeft](#), [QCP::msRight](#) to [QCPAxis::atRight](#), etc.)

15.555.4.10 mouseMoveEvent()

```
void QCPAxis::mouseMoveEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user moves the mouse while holding a mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *startPos* indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction. The default implementation does nothing.

See also

[mousePressEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

15.555.4.11 mousePressEvent()

```
void QCPAxis::mousePressEvent (
    QMouseEvent * event,
    const QVariant & details ) [protected], [virtual]
```

This event gets called when the user presses a mouse button while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *details* contains layerable-specific details about the hit, which were generated in the previous call to [selectTest](#). For example, One-dimensional plottables like [QCPGraph](#) or [QCPBars](#) convey the clicked data point in the *details* parameter, as [QCPDataSelection](#) packed as `QVariant`. Multi-part objects convey the specific `SelectablePart` that was hit (e.g. [QCPAxis::SelectablePart](#) in the case of axes).

[QCustomPlot](#) uses an event propagation system that works the same as Qt's system. If your layerable doesn't reimplement the [mousePressEvent](#) or explicitly calls `event->ignore()` in its reimplementation, the event will be propagated to the next layerable in the stacking order.

Once a layerable has accepted the [mousePressEvent](#), it is considered the mouse grabber and will receive all following calls to [mouseMoveEvent](#) or [mouseReleaseEvent](#) for this mouse interaction (a "mouse interaction" in this context ends with the release).

The default implementation does nothing except explicitly ignoring the event with `event->ignore()`.

See also

[mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

15.555.4.12 mouseReleaseEvent()

```
void QCPAxis::mouseReleaseEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user releases the mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *startPos* indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction. The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

15.555.4.13 moveRange()

```
void QCPAxis::moveRange (
    double diff )
```

If the scale type ([setScaleType](#)) is [stLinear](#), *diff* is added to the lower and upper bounds of the range. The range is simply moved by *diff*.

If the scale type is [stLogarithmic](#), the range bounds are multiplied by *diff*. This corresponds to an apparent "linear" move in logarithmic scaling by a distance of $\log(\text{diff})$.

15.555.4.14 opposite()

```
QCPAxis::AxisType QCPAxis::opposite (
    QCPAxis::AxisType type ) [static]
```

Returns the axis type that describes the opposite axis of an axis with the specified *type*.

15.555.4.15 orientation() [1/2]

```
Qt::Orientation QCPAxis::orientation ( ) const [inline]
```

Returns the orientation of this axis. The axis orientation (horizontal or vertical) is deduced from the axis type (left, top, right or bottom).

See also

[orientation\(AxisType type\)](#), [pixelOrientation](#)

15.555.4.16 orientation() [2/2]

```
static Qt::Orientation QCPAxis::orientation (
    AxisType type ) [inline], [static]
```

Returns the orientation of the specified axis type

See also

[orientation\(\)](#), [pixelOrientation](#)

15.555.4.17 pixelOrientation()

```
int QCPAxis::pixelOrientation ( ) const [inline]
```

Returns which direction points towards higher coordinate values/keys, in pixel space.

This method returns either 1 or -1. If it returns 1, then going in the positive direction along the orientation of the axis in pixels corresponds to going from lower to higher axis coordinates. On the other hand, if this method returns -1, going to smaller pixel values corresponds to going from lower to higher axis coordinates.

For example, this is useful to easily shift axis coordinates by a certain amount given in pixels, without having to care about reversed or vertically aligned axes:

```
double newKey = keyAxis->pixelToCoord(keyAxis->coordToPixel(oldKey)+10*keyAxis->pixelOrientation());
```

newKey will then contain a key that is ten pixels towards higher keys, starting from *oldKey*.

15.555.4.18 pixelToCoord()

```
double QCPAxis::pixelToCoord (
    double value ) const
```

Transforms *value*, in pixel coordinates of the [QCustomPlot](#) widget, to axis coordinates.

15.555.4.19 plottables()

```
QList< QCPAbstractPlottable * > QCPAxis::plottables ( ) const
```

Returns a list of all the plottables that have this axis as key or value axis.

If you are only interested in plottables of type [QCPGraph](#), see [graphs](#).

See also

[graphs](#), [items](#)

15.555.4.20 rangeChanged [1/2]

```
void QCPAxis::rangeChanged (
    const QCPRange & newRange ) [signal]
```

This signal is emitted when the range of this axis has changed. You can connect it to the [setRange](#) slot of another axis to communicate the new range to the other axis, in order for it to be synchronized.

You may also manipulate/correct the range with [setRange](#) in a slot connected to this signal. This is useful if for example a maximum range span shall not be exceeded, or if the lower/upper range shouldn't go beyond certain values (see [QCPRange::bounded](#)). For example, the following slot would limit the x axis to ranges between 0 and 10:

```
customPlot->xAxis->setRange(newRange.bounded(0, 10))
```

15.555.4.21 rangeChanged [2/2]

```
void QCPAxis::rangeChanged (
    const QCPRange & newRange,
    const QCPRange & oldRange ) [signal]
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Additionally to the new range, this signal also provides the previous range held by the axis as *oldRange*.

15.555.4.22 rescale()

```
void QCPAxis::rescale (
    bool onlyVisiblePlottables = false )
```

Changes the axis range such that all plottables associated with this axis are fully visible in that dimension.

See also

[QCPAbstractPlottable::rescaleAxes](#), [QCustomPlot::rescaleAxes](#)

15.555.4.23 scaleRange() [1/2]

```
void QCPAxis::scaleRange (
    double factor )
```

Scales the range of this axis by *factor* around the center of the current axis range. For example, if *factor* is 2.0, then the axis range will double its size, and the point at the axis range center won't have changed its position in the [QCustomPlot](#) widget (i.e. coordinates around the center will have moved symmetrically closer).

If you wish to scale around a different coordinate than the current axis range center, use the overload [scaleRange\(double factor, double center\)](#).

15.555.4.24 scaleRange() [2/2]

```
void QCPAxis::scaleRange (
    double factor,
    double center )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Scales the range of this axis by *factor* around the coordinate *center*. For example, if *factor* is 2.0, *center* is 1.0, then the axis range will double its size, and the point at coordinate 1.0 won't have changed its position in the [QCustomPlot](#) widget (i.e. coordinates around 1.0 will have moved symmetrically closer to 1.0).

See also

[scaleRange\(double factor\)](#)

15.555.4.25 scaleTypeChanged

```
void QCPAxis::scaleTypeChanged (
    QCPAxis::ScaleType scaleType ) [signal]
```

This signal is emitted when the scale type changes, by calls to [setScaleType](#)

15.555.4.26 selectableChanged

```
void QCPAxis::selectableChanged (
    const QCPAxis::SelectableParts & parts ) [signal]
```

This signal is emitted when the selectability changes, by calls to [setSelectableParts](#)

15.555.4.27 selectEvent()

```
void QCPAxis::selectEvent (
    QMouseEvent * event,
    bool additive,
    const QVariant & details,
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.555.4.28 selectionCategory()

```
QCP::Interaction QCPAxis::selectionCategory ( ) const [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.555.4.29 selectionChanged

```
void QCPAxis::selectionChanged (
    const QCPAxis::SelectableParts & parts ) [signal]
```

This signal is emitted when the selection state of this axis has changed, either by user interaction or by a direct call to [setSelectedParts](#).

15.555.4.30 selectTest()

```
double QCPAxis::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the *selectionTolerance* multiplied by 0.99). Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than $0.99 \cdot \text{selectionTolerance}$).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the *mouseReleaseEvent* occurs, and the finally selected object is notified via the *selectEvent/* *deselectEvent* methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to *selectEvent* when the parent [QCustomPlot](#) decides on the basis of this *selectTest* call, that the object was successfully selected. The subsequent call to *selectEvent* will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in [selectTest](#). The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent *selectEvent*, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass *nullptr* as *details* to indicate that you are not interested in those selection details.

See also

[selectEvent](#), [deselectEvent](#), [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Reimplemented from [QCPLayerable](#).

15.555.4.31 setBasePen()

```
void QCPAxis::setBasePen (
    const QPen & pen )
```

Sets the pen, the axis base line is drawn with.

See also

[setTickPen](#), [setSubTickPen](#)

15.555.4.32 **setLabel()**

```
void QCPAxis::setLabel (
    const QString & str )
```

Sets the text of the axis label that will be shown below/above or next to the axis, depending on its orientation. To disable axis labels, pass an empty string as *str*.

15.555.4.33 **setLabelColor()**

```
void QCPAxis::setLabelColor (
    const QColor & color )
```

Sets the color of the axis label.

See also

[setLabelFont](#)

15.555.4.34 **setLabelFont()**

```
void QCPAxis::setLabelFont (
    const QFont & font )
```

Sets the font of the axis label.

See also

[setLabelColor](#)

15.555.4.35 **setLabelPadding()**

```
void QCPAxis::setLabelPadding (
    int padding )
```

Sets the distance between the tick labels and the axis label.

See also

[setTickLabelPadding](#), [setPadding](#)

15.555.4.36 **setLowerEnding()**

```
void QCPAxis::setLowerEnding (
    const QCPLLineEnding & ending )
```

Sets the style for the lower axis ending. See the documentation of [QCPLLineEnding](#) for available styles.

For horizontal axes, this method refers to the left ending, for vertical axes the bottom ending. Note that this meaning does not change when the axis range is reversed with [setRangeReversed](#).

See also

[setUpperEnding](#)

15.555.4.37 **setNumberFormat()**

```
void QCPAxis::setNumberFormat (
    const QString & formatCode )
```

Sets the number format for the numbers in tick labels. This *formatCode* is an extended version of the format code used e.g. by `QString::number()` and `QLocale::toString()`. For reference about that, see the "Argument Formats" section in the detailed description of the `QString` class.

formatCode is a string of one, two or three characters.

The first character is identical to the normal format code used by Qt. In short, this means: 'e'/'E' scientific format, 'f' fixed format, 'g'/'G' scientific or fixed, whichever is shorter. For the 'e', 'E', and 'f' formats, the precision set by [setNumberPrecision](#) represents the number of digits after the decimal point. For the 'g' and 'G' formats, the precision represents the maximum number of significant digits, trailing zeroes are omitted.

The second and third characters are optional and specific to [QCustomPlot](#):

If the first char was 'e' or 'g', numbers are/might be displayed in the scientific format, e.g. "5.5e9", which is ugly in a plot. So when the second char of *formatCode* is set to 'b' (for "beautiful"), those exponential numbers are formatted in a more natural way, i.e. "5.5 [multiplication sign] 10 [superscript] 9". By default, the multiplication sign is a centered dot. If instead a cross should be shown (as is usual in the USA), the third char of *formatCode* can be set to 'c'. The inserted multiplication signs are the UTF-8 characters 215 (0xD7) for the cross and 183 (0xB7) for the dot.

Examples for *formatCode*:

- `g` normal format code behaviour. If number is small, fixed format is used, if number is large, normal scientific format is used
- `gb` If number is small, fixed format is used, if number is large, scientific format is used with beautifully typeset decimal powers and a dot as multiplication sign
- `ebc` All numbers are in scientific format with beautifully typeset decimal power and a cross as multiplication sign
- `fb` illegal format code, since fixed format doesn't support (or need) beautifully typeset decimal powers. Format code will be reduced to 'f'.
- `hello` illegal format code, since first char is not 'e', 'E', 'f', 'g' or 'G'. Current format code will not be changed.

15.555.4.38 setNumberPrecision()

```
void QCPAxis::setNumberPrecision (
    int precision )
```

Sets the precision of the tick label numbers. See [QLocale::toString\(double i, char f, int prec\)](#) for details. The effect of precisions are most notably for number Formats starting with 'e', see [setNumberFormat](#)

15.555.4.39 setOffset()

```
void QCPAxis::setOffset (
    int offset )
```

Sets the offset the axis has to its axis rect side.

If an axis rect side has multiple axes and automatic margin calculation is enabled for that side, only the offset of the inner most axis has meaning (even if it is set to be invisible). The offset of the other, outer axes is controlled automatically, to place them at appropriate positions.

15.555.4.40 setPadding()

```
void QCPAxis::setPadding (
    int padding )
```

Sets the padding of the axis.

When [QCPAxisRect::setAutoMargins](#) is enabled, the padding is the additional outer most space, that is left blank. The axis padding has no meaning if [QCPAxisRect::setAutoMargins](#) is disabled.

See also

[setLabelPadding](#), [setTickLabelPadding](#)

15.555.4.41 setRange() [1/3]

```
void QCPAxis::setRange (
    const QCPRange & range )
```

Sets the range of the axis.

This slot may be connected with the [rangeChanged](#) signal of another axis so this axis is always synchronized with the other axis range, when it changes.

To invert the direction of an axis, use [setRangeReversed](#).

15.555.4.42 setRange() [2/3]

```
void QCPAxis::setRange (
    double lower,
    double upper )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the lower and upper bound of the axis range.

To invert the direction of an axis, use [setRangeReversed](#).

There is also a slot to set a range, see [setRange\(const QCPRange &range\)](#).

15.555.4.43 setRange() [3/3]

```
void QCPAxis::setRange (
    double position,
    double size,
    Qt::AlignmentFlag alignment )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the range of the axis.

The *position* coordinate indicates together with the *alignment* parameter, where the new range will be positioned. *size* defines the size of the new axis range. *alignment* may be `Qt::AlignLeft`, `Qt::AlignRight` or `Qt::AlignCenter`. This will cause the left border, right border, or center of the range to be aligned with *position*. Any other values of *alignment* will default to `Qt::AlignCenter`.

15.555.4.44 setRangeLower()

```
void QCPAxis::setRangeLower (
    double lower )
```

Sets the lower bound of the axis range. The upper bound is not changed.

See also

[setRange](#)

15.555.4.45 setRangeReversed()

```
void QCPAxis::setRangeReversed (
    bool reversed )
```

Sets whether the axis range (direction) is displayed reversed. Normally, the values on horizontal axes increase left to right, on vertical axes bottom to top. When *reversed* is set to true, the direction of increasing values is inverted.

Note that the range and data interface stays the same for reversed axes, e.g. the *lower* part of the [setRange](#) interface will still reference the mathematically smaller number than the *upper* part.

15.555.4.46 setRangeUpper()

```
void QCPAxis::setRangeUpper (
    double upper )
```

Sets the upper bound of the axis range. The lower bound is not changed.

See also

[setRange](#)

15.555.4.47 setScaleRatio()

```
void QCPAxis::setScaleRatio (
    const QCPAxis * otherAxis,
    double ratio = 1.0 )
```

Scales the range of this axis to have a certain scale *ratio* to *otherAxis*. The scaling will be done around the center of the current axis range.

For example, if *ratio* is 1, this axis is the *yAxis* and *otherAxis* is *xAxis*, graphs plotted with those axes will appear in a 1:1 aspect ratio, independent of the aspect ratio the axis rect has.

This is an operation that changes the range of this axis once, it doesn't fix the scale ratio indefinitely. Note that calling this function in the constructor of the [QCustomPlot](#)'s parent won't have the desired effect, since the widget dimensions aren't defined yet, and a `resizeEvent` will follow.

15.555.4.48 setScaleType()

```
void QCPAxis::setScaleType (
    QCPAxis::ScaleType type )
```

Sets whether the axis uses a linear scale or a logarithmic scale.

Note that this method controls the coordinate transformation. For logarithmic scales, you will likely also want to use a logarithmic tick spacing and labeling, which can be achieved by setting the axis ticker to an instance of [QCPAxisTickerLog](#) :

See the documentation of [QCPAxisTickerLog](#) about the details of logarithmic axis tick creation.

[setNumberPrecision](#)

15.555.4.49 setSelectableParts()

```
void QCPAxis::setSelectableParts (
    const QCPAxis::SelectableParts & selectableParts )
```

Sets whether the user can (de-)select the parts in *selectable* by clicking on the [QCustomPlot](#) surface. (When [QCustomPlot::setInteractions](#) contains `iSelectAxes`.)

However, even when *selectable* is set to a value not allowing the selection of a specific part, it is still possible to set the selection of this part manually, by calling [setSelectedParts](#) directly.

See also

[SelectablePart](#), [setSelectedParts](#)

15.555.4.50 setSelectedBasePen()

```
void QCPAxis::setSelectedBasePen (
    const QPen & pen )
```

Sets the pen that is used to draw the axis base line when selected.

See also

[setBasePen](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.555.4.51 setSelectedLabelColor()

```
void QCPAxis::setSelectedLabelColor (
    const QColor & color )
```

Sets the color that is used for the axis label when it is selected.

See also

[setLabelColor](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.555.4.52 setSelectedLabelFont()

```
void QCPAxis::setSelectedLabelFont (
    const QFont & font )
```

Sets the font that is used for the axis label when it is selected.

See also

[setLabelFont](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.555.4.53 setSelectedParts()

```
void QCPAxis::setSelectedParts (
    const QCPAxis::SelectableParts & selectedParts )
```

Sets the selected state of the respective axis parts described by [SelectablePart](#). When a part is selected, it uses a different pen/font.

The entire selection mechanism for axes is handled automatically when [QCustomPlot::setInteractions](#) contains [iSelectAxes](#). You only need to call this function when you wish to change the selection state manually.

This function can change the selection state of a part, independent of the [setSelectableParts](#) setting.

emits the [selectionChanged](#) signal when *selected* is different from the previous selection state.

See also

[SelectablePart](#), [setSelectableParts](#), [selectTest](#), [setSelectedBasePen](#), [setSelectedTickPen](#), [setSelectedSubTickPen](#), [setSelectedTickLabelFont](#), [setSelectedLabelFont](#), [setSelectedTickLabelColor](#), [setSelectedLabelColor](#)

15.555.4.54 setSelectedSubTickPen()

```
void QCPAxis::setSelectedSubTickPen (
    const QPen & pen )
```

Sets the pen that is used to draw the subticks when selected.

See also

[setSubTickPen](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.555.4.55 setSelectedTickLabelColor()

```
void QCPAxis::setSelectedTickLabelColor (
    const QColor & color )
```

Sets the color that is used for tick labels when they are selected.

See also

[setTickLabelColor](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.555.4.56 setSelectedTickLabelFont()

```
void QCPAxis::setSelectedTickLabelFont (
    const QFont & font )
```

Sets the font that is used for tick labels when they are selected.

See also

[setTickLabelFont](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.555.4.57 setSelectedTickPen()

```
void QCPAxis::setSelectedTickPen (
    const QPen & pen )
```

Sets the pen that is used to draw the (major) ticks when selected.

See also

[setTickPen](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.555.4.58 setSubTickLength()

```
void QCPAxis::setSubTickLength (
    int inside,
    int outside = 0 )
```

Sets the length of the subticks in pixels. *inside* is the length the subticks will reach inside the plot and *outside* is the length they will reach outside the plot. If *outside* is greater than zero, the tick labels and axis label will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setTickLength](#), [setSubTickLengthIn](#), [setSubTickLengthOut](#)

15.555.4.59 setSubTickLengthIn()

```
void QCPAxis::setSubTickLengthIn (
    int inside )
```

Sets the length of the inward subticks in pixels. *inside* is the length the subticks will reach inside the plot.

See also

[setSubTickLengthOut](#), [setSubTickLength](#), [setTickLength](#)

15.555.4.60 setSubTickLengthOut()

```
void QCPAxis::setSubTickLengthOut (
    int outside )
```

Sets the length of the outward subticks in pixels. *outside* is the length the subticks will reach outside the plot. If *outside* is greater than zero, the tick labels will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setSubTickLengthIn](#), [setSubTickLength](#), [setTickLength](#)

15.555.4.61 setSubTickPen()

```
void QCPAxis::setSubTickPen (
    const QPen & pen )
```

Sets the pen, subtick marks will be drawn with.

See also

[setSubTickCount](#), [setSubTickLength](#), [setBasePen](#)

15.555.4.62 setSubTicks()

```
void QCPAxis::setSubTicks (
    bool show )
```

Sets whether sub tick marks are displayed.

Sub ticks are only potentially visible if (major) ticks are also visible (see [setTicks](#))

See also

[setTicks](#)

15.555.4.63 `setTicker()`

```
void QCPAxis::setTicker (
    QSharedPointer< QCPAxisTicker > ticker )
```

The axis ticker is responsible for generating the tick positions and tick labels. See the documentation of [QCPAxisTicker](#) for details on how to work with axis tickers.

You can change the tick positioning/labeling behaviour of this axis by setting a different [QCPAxisTicker](#) subclass using this method. If you only wish to modify the currently installed axis ticker, access it via [ticker](#).

Since the ticker is stored in the axis as a shared pointer, multiple axes may share the same axis ticker simply by passing the same shared pointer to multiple axes.

See also

[ticker](#)

15.555.4.64 `setTickLabelColor()`

```
void QCPAxis::setTickLabelColor (
    const QColor & color )
```

Sets the color of the tick labels.

See also

[setTickLabels](#), [setTickLabelFont](#)

15.555.4.65 `setTickLabelFont()`

```
void QCPAxis::setTickLabelFont (
    const QFont & font )
```

Sets the font of the tick labels.

See also

[setTickLabels](#), [setTickLabelColor](#)

15.555.4.66 `setTickLabelPadding()`

```
void QCPAxis::setTickLabelPadding (
    int padding )
```

Sets the distance between the axis base line (including any outward ticks) and the tick labels.

See also

[setLabelPadding](#), [setPadding](#)

15.555.4.67 `setTickLabelRotation()`

```
void QCPAxis::setTickLabelRotation (
    double degrees )
```

Sets the rotation of the tick labels. If *degrees* is zero, the labels are drawn normally. Else, the tick labels are drawn rotated by *degrees* clockwise. The specified angle is bound to values from -90 to 90 degrees.

If *degrees* is exactly -90, 0 or 90, the tick labels are centered on the tick coordinate. For other angles, the label is drawn with an offset such that it seems to point toward or away from the tick mark.

15.555.4.68 setTickLabels()

```
void QCPAxis::setTickLabels (
    bool show )
```

Sets whether tick labels are displayed. Tick labels are the numbers drawn next to tick marks.

15.555.4.69 setTickLabelSide()

```
void QCPAxis::setTickLabelSide (
    LabelSide side )
```

Sets whether the tick labels (numbers) shall appear inside or outside the axis rect.

The usual and default setting is [IsOutside](#). Very compact plots sometimes require tick labels to be inside the axis rect, to save space. If *side* is set to [IsInside](#), the tick labels appear on the inside are additionally clipped to the axis rect.

15.555.4.70 setTickLength()

```
void QCPAxis::setTickLength (
    int inside,
    int outside = 0 )
```

Sets the length of the ticks in pixels. *inside* is the length the ticks will reach inside the plot and *outside* is the length they will reach outside the plot. If *outside* is greater than zero, the tick labels and axis label will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setSubTickLength](#), [setTickLengthIn](#), [setTickLengthOut](#)

15.555.4.71 setTickLengthIn()

```
void QCPAxis::setTickLengthIn (
    int inside )
```

Sets the length of the inward ticks in pixels. *inside* is the length the ticks will reach inside the plot.

See also

[setTickLengthOut](#), [setTickLength](#), [setSubTickLength](#)

15.555.4.72 setTickLengthOut()

```
void QCPAxis::setTickLengthOut (
    int outside )
```

Sets the length of the outward ticks in pixels. *outside* is the length the ticks will reach outside the plot. If *outside* is greater than zero, the tick labels and axis label will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setTickLengthIn](#), [setTickLength](#), [setSubTickLength](#)

15.555.4.73 setTickPen()

```
void QCPAxis::setTickPen (
    const QPen & pen )
```

Sets the pen, tick marks will be drawn with.

See also

[setTickLength](#), [setBasePen](#)

15.555.4.74 setTicks()

```
void QCPAxis::setTicks (
    bool show )
```

Sets whether tick marks are displayed.

Note that setting *show* to false does not imply that tick labels are invisible, too. To achieve that, see [setTickLabels](#).

See also

[setSubTicks](#)

15.555.4.75 setUpperEnding()

```
void QCPAxis::setUpperEnding (
    const QCPLLineEnding & ending )
```

Sets the style for the upper axis ending. See the documentation of [QCPLLineEnding](#) for available styles.

For horizontal axes, this method refers to the right ending, for vertical axes the top ending. Note that this meaning does not change when the axis range is reversed with [setRangeReversed](#).

See also

[setLowerEnding](#)

15.555.4.76 ticker()

```
QSharedPointer< QCPAxisTicker > QCPAxis::ticker ( ) const [inline]
```

Returns a modifiable shared pointer to the currently installed axis ticker. The axis ticker is responsible for generating the tick positions and tick labels of this axis. You can access the [QCPAxisTicker](#) with this method and modify basic properties such as the approximate tick count ([QCPAxisTicker::setTickCount](#)).

You can gain more control over the axis ticks by setting a different [QCPAxisTicker](#) subclass, see the documentation there. A new axis ticker can be set with [setTicker](#).

Since the ticker is stored in the axis as a shared pointer, multiple axes may share the same axis ticker simply by passing the same shared pointer to multiple axes.

See also

[setTicker](#)

15.555.4.77 wheelEvent()

```
void QCPAxis::wheelEvent (
    QWheelEvent * event ) [protected], [virtual]
```

This event gets called when the user turns the mouse scroll wheel while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`.

The `event->angleDelta()` indicates how far the mouse wheel was turned, which is usually +/- 120 for single rotation steps. However, if the mouse wheel is turned rapidly, multiple steps may accumulate to one event, making the delta larger. On the other hand, if the wheel has very smooth steps or none at all, the delta may be smaller.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#)

Reimplemented from [QCPLayerable](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

Collaboration diagram for QCPAxisPainterPrivate:



- ## Public Member Functions

- ## Public Attributes

- Generated by Doxygen

- QString **label**
- int **tickLabelPadding**
- double **tickLabelRotation**
- [QCPAxis::LabelSide](#) **tickLabelSide**
- bool **substituteExponent**
- bool **numberMultiplyCross**
- int **tickLengthIn**
- int **tickLengthOut**
- int **subTickLengthIn**
- int **subTickLengthOut**
- QPen **tickPen**
- QPen **subTickPen**
- QFont **tickLabelFont**
- QColor **tickLabelColor**
- QRect **axisRect**
- QRect **viewportRect**
- int **offset**
- bool **abbreviateDecimalPowers**
- bool **reversedEndings**
- QVector< double > **subTickPositions**
- QVector< double > **tickPositions**
- QVector< QString > **tickLabels**

Protected Member Functions

- virtual QByteArray **generateLabelParameterHash** () const
- virtual void **placeTickLabel** ([QCPPainter](#) *painter, double position, int distanceToAxis, const QString &text, QSize *tickLabelsSize)
- virtual void **drawTickLabel** ([QCPPainter](#) *painter, double x, double y, const [TickLabelData](#) &labelData) const
- virtual [TickLabelData](#) **getTickLabelData** (const QFont &font, const QString &text) const
- virtual QPointF **getTickLabelDrawOffset** (const [TickLabelData](#) &labelData) const
- virtual void **getMaxTickLabelSize** (const QFont &font, const QString &text, QSize *tickLabelsSize) const

Protected Attributes

- [QCustomPlot](#) * **mParentPlot**
- QByteArray **mLabelParameterHash**
- QCache< QString, [CachedLabel](#) > **mLabelCache**
- QRect **mAxisSelectionBox**
- QRect **mTickLabelsSelectionBox**
- QRect **mLabelSelectionBox**

15.556.1 Constructor & Destructor Documentation

15.556.1.1 QCPAxisPainterPrivate()

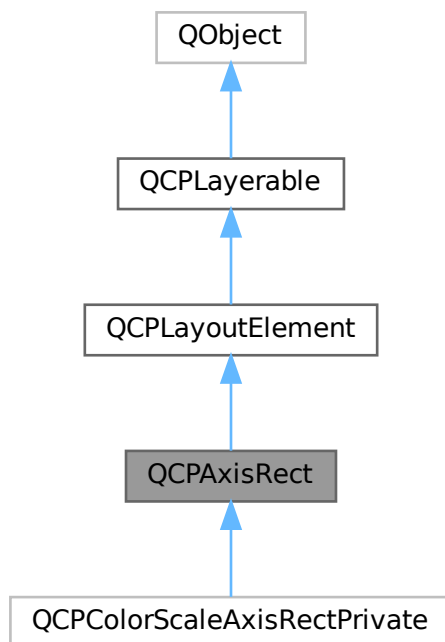
```
QCPAxisPainterPrivate::QCPAxisPainterPrivate (
    QCustomPlot * parentPlot ) [explicit]
```

Constructs a [QCPAxisPainterPrivate](#) instance. Make sure to not create a new instance on every redraw, to utilize the caching mechanisms.

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

Holds multiple axes and arranges them in a rectangular shape.
Inheritance diagram for QCPAxisRect:

[illegible]

- `QCPAxisRect (QCustomPlot *parentPlot, bool setupDefaultAxes=true)`

- QPixmap **background** () const
- QBrush **backgroundBrush** () const
- bool **backgroundScaled** () const
- Qt::AspectRatioMode **backgroundScaledMode** () const
- Qt::Orientations **rangeDrag** () const
- Qt::Orientations **rangeZoom** () const
- QCPAxis * **rangeDragAxis** (Qt::Orientation orientation)
- QCPAxis * **rangeZoomAxis** (Qt::Orientation orientation)
- QList< QCPAxis * > **rangeDragAxes** (Qt::Orientation orientation)
- QList< QCPAxis * > **rangeZoomAxes** (Qt::Orientation orientation)
- double **rangeZoomFactor** (Qt::Orientation orientation)
- void **setBackground** (const QPixmap &pm)
- void **setBackground** (const QPixmap &pm, bool scaled, Qt::AspectRatioMode mode=Qt::KeepAspectRatio←ByExpanding)
- void **setBackground** (const QBrush &brush)
- void **setBackgroundScaled** (bool scaled)
- void **setBackgroundScaledMode** (Qt::AspectRatioMode mode)
- void **setRangeDrag** (Qt::Orientations orientations)
- void **setRangeZoom** (Qt::Orientations orientations)
- void **setRangeDragAxes** (QCPAxis *horizontal, QCPAxis *vertical)
- void **setRangeDragAxes** (QList< QCPAxis * > axes)
- void **setRangeDragAxes** (QList< QCPAxis * > horizontal, QList< QCPAxis * > vertical)
- void **setRangeZoomAxes** (QCPAxis *horizontal, QCPAxis *vertical)
- void **setRangeZoomAxes** (QList< QCPAxis * > axes)
- void **setRangeZoomAxes** (QList< QCPAxis * > horizontal, QList< QCPAxis * > vertical)
- void **setRangeZoomFactor** (double horizontalFactor, double verticalFactor)
- void **setRangeZoomFactor** (double factor)
- int **axisCount** (QCPAxis::AxisType type) const
- QCPAxis * **axis** (QCPAxis::AxisType type, int index=0) const
- QList< QCPAxis * > **axes** (QCPAxis::AxisTypes types) const
- QList< QCPAxis * > **axes** () const
- QCPAxis * **addAxis** (QCPAxis::AxisType type, QCPAxis *axis=nullptr)
- QList< QCPAxis * > **addAxes** (QCPAxis::AxisTypes types)
- bool **removeAxis** (QCPAxis *axis)
- QCPLayoutInset * **insetLayout** () const
- void **zoom** (const QRectF &pixelRect)
- void **zoom** (const QRectF &pixelRect, const QList< QCPAxis * > &affectedAxes)
- void **setupFullAxesBox** (bool connectRanges=false)
- QList< QCPAbstractPlottable * > **plottables** () const
- QList< QCPGraph * > **graphs** () const
- QList< QCPAbstractItem * > **items** () const
- int **left** () const
- int **right** () const
- int **top** () const
- int **bottom** () const
- int **width** () const
- int **height** () const
- QSize **size** () const
- QPoint **topLeft** () const
- QPoint **topRight** () const
- QPoint **bottomLeft** () const
- QPoint **bottomRight** () const
- QPoint **center** () const
- virtual void **update** (UpdatePhase phase) Q_DECL_OVERRIDE
- virtual QList< QCPLayoutElement * > **elements** (bool recursive) const Q_DECL_OVERRIDE

Public Member Functions inherited from QCPLayoutElement

- [QCPLayoutElement](#) ([QCustomPlot](#) *parentPlot=nullptr)
- [QCPLayout](#) * [layout](#) () const
- [QRect](#) [rect](#) () const
- [QRect](#) [outerRect](#) () const
- [QMargins](#) [margins](#) () const
- [QMargins](#) [minimumMargins](#) () const
- [QCP::MarginSides](#) [autoMargins](#) () const
- [QSize](#) [minimumSize](#) () const
- [QSize](#) [maximumSize](#) () const
- [SizeConstraintRect](#) [sizeConstraintRect](#) () const
- [QCPMarginGroup](#) * [marginGroup](#) ([QCP::MarginSide](#) side) const
- [QHash](#)< [QCP::MarginSide](#), [QCPMarginGroup](#) * > [marginGroups](#) () const
- void [setOuterRect](#) (const [QRect](#) &[rect](#))
- void [setMargins](#) (const [QMargins](#) &margins)
- void [setMinimumMargins](#) (const [QMargins](#) &margins)
- void [setAutoMargins](#) ([QCP::MarginSides](#) sides)
- void [setMinimumSize](#) (const [QSize](#) &size)
- void [setMinimumSize](#) (int width, int height)
- void [setMaximumSize](#) (const [QSize](#) &size)
- void [setMaximumSize](#) (int width, int height)
- void [setSizeConstraintRect](#) ([SizeConstraintRect](#) constraintRect)
- void [setMarginGroup](#) ([QCP::MarginSides](#) sides, [QCPMarginGroup](#) *group)
- virtual [QSize](#) [minimumOuterSizeHint](#) () const
- virtual [QSize](#) [maximumOuterSizeHint](#) () const
- virtual double [selectTest](#) (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const [Q_DECL_OVERRIDE](#)

Public Member Functions inherited from QCPLayerable

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool [visible](#) () const
- [QCustomPlot](#) * [parentPlot](#) () const
- [QCPLayerable](#) * [parentLayerable](#) () const
- [QCPLayer](#) * [layer](#) () const
- bool [antialiased](#) () const
- void [setVisible](#) (bool on)
- [Q_SLOT](#) bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const [QString](#) &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const [Q_DECL_OVERRIDE](#)
- virtual void [draw](#) ([QCPPainter](#) *painter) [Q_DECL_OVERRIDE](#)
- virtual int [calculateAutoMargin](#) ([QCP::MarginSide](#) side) [Q_DECL_OVERRIDE](#)
- virtual void [layoutChanged](#) () [Q_DECL_OVERRIDE](#)
- virtual void [mousePressEvent](#) ([QMouseEvent](#) *event, const [QVariant](#) &details) [Q_DECL_OVERRIDE](#)
- virtual void [mouseMoveEvent](#) ([QMouseEvent](#) *event, const [QPointF](#) &startPos) [Q_DECL_OVERRIDE](#)
- virtual void [mouseReleaseEvent](#) ([QMouseEvent](#) *event, const [QPointF](#) &startPos) [Q_DECL_OVERRIDE](#)
- virtual void [wheelEvent](#) ([QWheelEvent](#) *event) [Q_DECL_OVERRIDE](#)
- void [drawBackground](#) ([QCPPainter](#) *painter)
- void [updateAxesOffset](#) ([QCPAxis::AxisType](#) type)

Protected Member Functions inherited from [QCPLayoutElement](#)

- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayerable](#)

- virtual [QCP::Interaction](#) [selectionCategory](#) () const
- virtual [QRect](#) [clipRect](#) () const
- virtual void [selectEvent](#) ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionState ← Changed)
- virtual void [deselectEvent](#) (bool *selectionStateChanged)
- virtual void [mouseDoubleClickEvent](#) ([QMouseEvent](#) *event, const [QVariant](#) &details)
- void [initializeParentPlot](#) ([QCustomPlot](#) *parentPlot)
- void [setParentLayerable](#) ([QCPLayerable](#) *parentLayerable)
- bool [moveToLayer](#) ([QCPLayer](#) *layer, bool prepend)
- void [applyAntialiasingHint](#) ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QBrush](#) [mBackgroundBrush](#)
- [QPixmap](#) [mBackgroundPixmap](#)
- [QPixmap](#) [mScaledBackgroundPixmap](#)
- bool [mBackgroundScaled](#)
- [Qt::AspectRatioMode](#) [mBackgroundScaledMode](#)
- [QCPLayoutInset](#) * [mInsetLayout](#)
- [Qt::Orientations](#) [mRangeDrag](#)
- [Qt::Orientations](#) [mRangeZoom](#)
- [QList](#)< [QPointer](#)< [QCPAxis](#) > > [mRangeDragHorzAxis](#)
- [QList](#)< [QPointer](#)< [QCPAxis](#) > > [mRangeDragVertAxis](#)
- [QList](#)< [QPointer](#)< [QCPAxis](#) > > [mRangeZoomHorzAxis](#)
- [QList](#)< [QPointer](#)< [QCPAxis](#) > > [mRangeZoomVertAxis](#)
- double [mRangeZoomFactorHorz](#)
- double [mRangeZoomFactorVert](#)
- [QList](#)< [QCPRange](#) > [mDragStartHorzRange](#)
- [QList](#)< [QCPRange](#) > [mDragStartVertRange](#)
- [QCP::AntialiasedElements](#) [mAADragBackup](#)
- [QCP::AntialiasedElements](#) [mNotAADragBackup](#)
- bool [mDragging](#)
- [QHash](#)< [QCPAxis::AxisType](#), [QList](#)< [QCPAxis](#) * > > [mAxes](#)

Protected Attributes inherited from [QCPLayoutElement](#)

- [QCPLayout](#) * [mParentLayout](#)
- [QSize](#) [mMinimumSize](#)
- [QSize](#) [mMaximumSize](#)
- [SizeConstraintRect](#) [mSizeConstraintRect](#)
- [QRect](#) [mRect](#)
- [QRect](#) [mOuterRect](#)
- [QMargins](#) [mMargins](#)
- [QMargins](#) [mMinimumMargins](#)
- [QCP::MarginSides](#) [mAutoMargins](#)
- [QHash](#)< [QCP::MarginSide](#), [QCPMarginGroup](#) * > [mMarginGroups](#)

Protected Attributes inherited from [QCPLayerable](#)

- bool `mVisible`
- [QCustomPlot](#) * `mParentPlot`
- [QPointer](#)< [QCPLayerable](#) > `mParentLayerable`
- [QCPLayer](#) * `mLayer`
- bool `mAntialiased`

Friends

- class [QCustomPlot](#)

Additional Inherited Members

Public Types inherited from [QCPLayoutElement](#)

- enum [UpdatePhase](#) { `upPreparation` , `upMargins` , `upLayout` }
- enum [SizeConstraintRect](#) { `scrInnerRect` , `scrOuterRect` }

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

15.557.1 Detailed Description

Holds multiple axes and arranges them in a rectangular shape.

This class represents an axis rect, a rectangular area that is bounded on all sides with an arbitrary number of axes. Initially [QCustomPlot](#) has one axis rect, accessible via [QCustomPlot::axisRect\(\)](#). However, the layout system allows to have multiple axis rects, e.g. arranged in a grid layout ([QCustomPlot::plotLayout](#)).

By default, [QCPAxisRect](#) comes with four axes, at bottom, top, left and right. They can be accessed via [axis](#) by providing the respective axis type ([QCPAxis::AxisType](#)) and index. If you need all axes in the axis rect, use [axes](#). The top and right axes are set to be invisible initially ([QCPAxis::setVisible](#)). To add more axes to a side, use [addAxis](#) or [addAxes](#). To remove an axis, use [removeAxis](#).

The axis rect layerable itself only draws a background pixmap or color, if specified ([setBackground](#)). It is placed on the "background" layer initially (see [QCPLayer](#) for an explanation of the [QCustomPlot](#) layer system). The axes that are held by the axis rect can be placed on other layers, independently of the axis rect.

Every axis rect has a child layout of type [QCPLayoutInset](#). It is accessible via [insetLayout](#) and can be used to have other layout elements (or even other layouts with multiple elements) hovering inside the axis rect.

If an axis rect is clicked and dragged, it processes this by moving certain axis ranges. The behaviour can be controlled with [setRangeDrag](#) and [setRangeDragAxes](#). If the mouse wheel is scrolled while the cursor is on the axis rect, certain axes are scaled. This is controllable via [setRangeZoom](#), [setRangeZoomAxes](#) and [setRangeZoomFactor](#). These interactions are only enabled if [QCustomPlot::setInteractions](#) contains [QCP::iRangeDrag](#) and [QCP::iRangeZoom](#).

Overview of the spacings and paddings that define the geometry of an axis. The dashed line on the far left indicates the viewport/widget border.

15.557.2 Constructor & Destructor Documentation

15.557.2.1 QCPAxisRect()

```
QCPAxisRect::QCPAxisRect (
    QCustomPlot * parentPlot,
    bool setupDefaultAxes = true ) [explicit]
```

Creates a [QCPAxisRect](#) instance and sets default values. An axis is added for each of the four sides, the top and right axes are set invisible initially.

15.557.3 Member Function Documentation

15.557.3.1 addAxes()

```
QList< QCPAxis * > QCPAxisRect::addAxes (
    QCPAxis::AxisTypes types )
```

Adds a new axis with [addAxis](#) to each axis rect side specified in *types*. This may be an `or`-combination of [QCPAxis::AxisType](#), so axes can be added to multiple sides at once.

Returns a list of the added axes.

See also

[addAxis](#), [setupFullAxesBox](#)

15.557.3.2 addAxis()

```
QCPAxis * QCPAxisRect::addAxis (
    QCPAxis::AxisType type,
    QCPAxis * axis = nullptr )
```

Adds a new axis to the axis rect side specified with *type*, and returns it. If *axis* is 0, a new [QCPAxis](#) instance is created internally. [QCustomPlot](#) owns the returned axis, so if you want to remove an axis, use [removeAxis](#) instead of deleting it manually.

You may inject [QCPAxis](#) instances (or subclasses of [QCPAxis](#)) by setting *axis* to an axis that was previously created outside [QCustomPlot](#). It is important to note that [QCustomPlot](#) takes ownership of the axis, so you may not delete it afterwards. Further, the *axis* must have been created with this axis rect as parent and with the same axis type as specified in *type*. If this is not the case, a debug output is generated, the axis is not added, and the method returns `nullptr`.

This method can not be used to move *axis* between axis rects. The same *axis* instance must not be added multiple times to the same or different axis rects.

If an axis rect side already contains one or more axes, the lower and upper endings of the new axis ([QCPAxis::setLowerEnding](#), [QCPAxis::setUpperEnding](#)) are set to [QCPLineEnding::esHalfBar](#).

See also

[addAxes](#), [setupFullAxesBox](#)

15.557.3.3 applyDefaultAntialiasingHint()

```
void QCPAxisRect::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

15.557.3.4 axes() [1/2]

```
QList< QCPAxis * > QCPAxisRect::axes ( ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns all axes of this axis rect.

15.557.3.5 axes() [2/2]

```
QList< QCPAxis * > QCPAxisRect::axes (
    QCPAxis::AxisTypes types ) const
```

Returns all axes on the axis rect sides specified with *types*.

types may be a single [QCPAxis::AxisType](#) or an `or`-combination, to get the axes of multiple sides.

See also

[axis](#)

15.557.3.6 axis()

```
QCPAxis * QCPAxisRect::axis (
    QCPAxis::AxisType type,
    int index = 0 ) const
```

Returns the axis with the given *index* on the axis rect side specified with *type*.

See also

[axisCount](#), [axes](#)

15.557.3.7 axisCount()

```
int QCPAxisRect::axisCount (
    QCPAxis::AxisType type ) const
```

Returns the number of axes on the axis rect side specified with *type*.

See also

[axis](#)

15.557.3.8 bottom()

```
int QCPAxisRect::bottom ( ) const [inline]
```

Returns the pixel position of the bottom border of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.557.3.9 bottomLeft()

```
QPoint QCPAxisRect::bottomLeft ( ) const [inline]
```

Returns the bottom left corner of this axis rect in pixels. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.557.3.10 bottomRight()

```
QPoint QCPAxisRect::bottomRight ( ) const [inline]
```

Returns the bottom right corner of this axis rect in pixels. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.557.3.11 calculateAutoMargin()

```
int QCPAxisRect::calculateAutoMargin (
    QCP::MarginSide side ) [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

15.557.3.12 center()

```
QPoint QCPAxisRect::center ( ) const [inline]
```

Returns the center of this axis rect in pixels. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.557.3.13 draw()

```
void QCPAxisRect::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

15.557.3.14 elements()

```
QList< QCPLayoutElement * > QCPAxisRect::elements (
    bool recursive ) const [virtual]
```

Returns a list of all child elements in this layout element. If *recursive* is true, all sub-child elements are included in the list, too.

Warning

There may be `nullptr` entries in the returned list. For example, [QCPLayoutGrid](#) may have empty cells which yield `nullptr` at the respective index.

Reimplemented from [QCPLayoutElement](#).

15.557.3.15 graphs()

```
QList< QCPLGraph * > QCPAxisRect::graphs ( ) const
```

Returns a list of all the graphs that are associated with this axis rect.

A graph is considered associated with an axis rect if its key or value axis (or both) is in this axis rect.

See also

[plottables](#), [items](#)

15.557.3.16 height()

```
int QCPAxisRect::height ( ) const [inline]
```

Returns the pixel height of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.557.3.17 insetLayout()

```
QCPLayoutInset * QCPAxisRect::insetLayout ( ) const [inline]
```

Returns the inset layout of this axis rect. It can be used to place other layout elements (or even layouts with multiple other elements) inside/on top of an axis rect.

See also

[QCPLayoutInset](#)

15.557.3.18 items()

```
QList< QCPLAbstractItem * > QCPAxisRect::items ( ) const
```

Returns a list of all the items that are associated with this axis rect.

An item is considered associated with an axis rect if any of its positions has key or value axis set to an axis that is in this axis rect, or if any of its positions has [QCPLItemPosition::setAxisRect](#) set to the axis rect, or if the clip axis rect ([QCPLAbstractItem::setClipAxisRect](#)) is set to this axis rect.

See also

[plottables](#), [graphs](#)

15.557.3.19 layoutChanged()

```
void QCPAxisRect::layoutChanged ( ) [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

15.557.3.20 left()

```
int QCPAxisRect::left ( ) const [inline]
```

Returns the pixel position of the left border of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.557.3.21 mouseMoveEvent()

```
void QCPAxisRect::mouseMoveEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user moves the mouse while holding a mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter `startPos` indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction. The default implementation does nothing.

See also

[mousePressEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

Reimplemented in [QCPColorScaleAxisRectPrivate](#).

15.557.3.22 mousePressEvent()

```
void QCPAxisRect::mousePressEvent (
    QMouseEvent * event,
    const QVariant & details ) [protected], [virtual]
```

This event gets called when the user presses a mouse button while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter `details` contains layerable-specific details about the hit, which were generated in the previous call to [selectTest](#). For example, One-dimensional plottables like [QCPGraph](#) or [QCPBars](#) convey the clicked data point in the `details` parameter, as [QCPDataSelection](#) packed as `QVariant`. Multi-part objects convey the specific `SelectablePart` that was hit (e.g. [QCPAxis::SelectablePart](#) in the case of axes).

[QCustomPlot](#) uses an event propagation system that works the same as Qt's system. If your layerable doesn't reimplement the [mousePressEvent](#) or explicitly calls `event->ignore()` in its reimplementation, the event will be propagated to the next layerable in the stacking order.

Once a layerable has accepted the [mousePressEvent](#), it is considered the mouse grabber and will receive all following calls to [mouseMoveEvent](#) or [mouseReleaseEvent](#) for this mouse interaction (a "mouse interaction" in this context ends with the release).

The default implementation does nothing except explicitly ignoring the event with `event->ignore()`.

See also

[mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

Reimplemented in [QCPColorScaleAxisRectPrivate](#).

15.557.3.23 mouseReleaseEvent()

```
void QCPAxisRect::mouseReleaseEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user releases the mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter `startPos` indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction. The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

Reimplemented in [QCPColorScaleAxisRectPrivate](#).

15.557.3.24 `plottables()`

```
QList< QCPAbstractPlottable * > QCPAxisRect::plottables ( ) const
```

Returns a list of all the plottables that are associated with this axis rect.

A plottable is considered associated with an axis rect if its key or value axis (or both) is in this axis rect.

See also

[graphs](#), [items](#)

15.557.3.25 `rangeDragAxes()`

```
QList< QCPAxis * > QCPAxisRect::rangeDragAxes (
    Qt::Orientation orientation )
```

Returns all range drag axes of the *orientation* provided.

See also

[rangeZoomAxis](#), [setRangeZoomAxes](#)

15.557.3.26 `rangeDragAxis()`

```
QCPAxis * QCPAxisRect::rangeDragAxis (
    Qt::Orientation orientation )
```

Returns the range drag axis of the *orientation* provided. If multiple axes were set, returns the first one (use [rangeDragAxes](#) to retrieve a list with all set axes).

See also

[setRangeDragAxes](#)

15.557.3.27 `rangeZoomAxes()`

```
QList< QCPAxis * > QCPAxisRect::rangeZoomAxes (
    Qt::Orientation orientation )
```

Returns all range zoom axes of the *orientation* provided.

See also

[rangeDragAxis](#), [setRangeDragAxes](#)

15.557.3.28 `rangeZoomAxis()`

```
QCPAxis * QCPAxisRect::rangeZoomAxis (
    Qt::Orientation orientation )
```

Returns the range zoom axis of the *orientation* provided. If multiple axes were set, returns the first one (use [rangeZoomAxes](#) to retrieve a list with all set axes).

See also

[setRangeZoomAxes](#)

15.557.3.29 `rangeZoomFactor()`

```
double QCPAxisRect::rangeZoomFactor (
    Qt::Orientation orientation )
```

Returns the range zoom factor of the *orientation* provided.

See also

[setRangeZoomFactor](#)

15.557.3.30 removeAxis()

```
bool QCPAxisRect::removeAxis (
    QCPAxis * axis )
```

Removes the specified *axis* from the axis rect and deletes it.
Returns true on success, i.e. if *axis* was a valid axis in this axis rect.

See also

[addAxis](#)

15.557.3.31 right()

```
int QCPAxisRect::right ( ) const [inline]
```

Returns the pixel position of the right border of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.557.3.32 setBackground() [1/3]

```
void QCPAxisRect::setBackground (
    const QBrush & brush )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets *brush* as the background brush. The axis rect background will be filled with this brush. Since axis rects place themselves on the "background" layer by default, the axis rect backgrounds are usually drawn below everything else.

The brush will be drawn before (under) any background pixmap, which may be specified with [setBackground\(const QPixmap &pm\)](#). To disable drawing of a background brush, set *brush* to `Qt::NoBrush`.

See also

[setBackground\(const QPixmap &pm\)](#)

15.557.3.33 setBackground() [2/3]

```
void QCPAxisRect::setBackground (
    const QPixmap & pm )
```

Sets *pm* as the axis background pixmap. The axis background pixmap will be drawn inside the axis rect. Since axis rects place themselves on the "background" layer by default, the axis rect backgrounds are usually drawn below everything else.

For cases where the provided pixmap doesn't have the same size as the axis rect, scaling can be enabled with [setBackgroundScaled](#) and the scaling mode (i.e. whether and how the aspect ratio is preserved) can be set with [setBackgroundScaledMode](#). To set all these options in one call, consider using the overloaded version of this function.

Below the pixmap, the axis rect may be optionally filled with a brush, if specified with [setBackground\(const QBrush &brush\)](#).

See also

[setBackgroundScaled](#), [setBackgroundScaledMode](#), [setBackground\(const QBrush &brush\)](#)

15.557.3.34 setBackground() [3/3]

```
void QCPAxisRect::setBackground (
    const QPixmap & pm,
    bool scaled,
    Qt::AspectRatioMode mode = Qt::KeepAspectRatioByExpanding )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Allows setting the background pixmap of the axis rect, whether it shall be scaled and how it shall be scaled in one call.

See also

[setBackground\(const QPixmap &pm\)](#), [setBackgroundScaled](#), [setBackgroundScaledMode](#)

15.557.3.35 setBackgroundScaled()

```
void QCPAxisRect::setBackgroundScaled (
    bool scaled )
```

Sets whether the axis background pixmap shall be scaled to fit the axis rect or not. If *scaled* is set to true, you may control whether and how the aspect ratio of the original pixmap is preserved with [setBackgroundScaledMode](#).

Note that the scaled version of the original pixmap is buffered, so there is no performance penalty on replots. (Except when the axis rect dimensions are changed continuously.)

See also

[setBackground](#), [setBackgroundScaledMode](#)

15.557.3.36 setBackgroundScaledMode()

```
void QCPAxisRect::setBackgroundScaledMode (
    Qt::AspectRatioMode mode )
```

If scaling of the axis background pixmap is enabled ([setBackgroundScaled](#)), use this function to define whether and how the aspect ratio of the original pixmap passed to [setBackground](#) is preserved.

See also

[setBackground](#), [setBackgroundScaled](#)

15.557.3.37 setRangeDrag()

```
void QCPAxisRect::setRangeDrag (
    Qt::Orientations orientations )
```

Sets which axis orientation may be range dragged by the user with mouse interaction. What orientation corresponds to which specific axis can be set with [setRangeDragAxes\(QCPAxis *horizontal, QCPAxis *vertical\)](#). By default, the horizontal axis is the bottom axis (xAxis) and the vertical axis is the left axis (yAxis).

To disable range dragging entirely, pass `nullptr` as *orientations* or remove [QCP::iRangeDrag](#) from [QCustomPlot::setInteractions](#). To enable range dragging for both directions, pass `Qt::Horizontal | Qt::Vertical` as *orientations*.

In addition to setting *orientations* to a non-zero value, make sure [QCustomPlot::setInteractions](#) contains [QCP::iRangeDrag](#) to enable the range dragging interaction.

See also

[setRangeZoom](#), [setRangeDragAxes](#), [QCustomPlot::setNoAntialiasingOnDrag](#)

15.557.3.38 setRangeDragAxes() [1/3]

```
void QCPAxisRect::setRangeDragAxes (
    QCPAxis * horizontal,
    QCPAxis * vertical )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the axes whose range will be dragged when [setRangeDrag](#) enables mouse range dragging on the [QCustomPlot](#) widget. Pass `nullptr` if no axis shall be dragged in the respective orientation.

Use the overload taking a list of axes, if multiple axes (more than one per orientation) shall react to dragging interactions.

See also

[setRangeZoomAxes](#)

15.557.3.39 setRangeDragAxes() [2/3]

```
void QCPAxisRect::setRangeDragAxes (
    QList< QCPAxis * > axes )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

This method allows to set up multiple axes to react to horizontal and vertical dragging. The drag orientation that the respective axis will react to is deduced from its orientation ([QCPAxis::orientation](#)).

In the unusual case that you wish to e.g. drag a vertically oriented axis with a horizontal drag motion, use the overload taking two separate lists for horizontal and vertical dragging.

15.557.3.40 setRangeDragAxes() [3/3]

```
void QCPAxisRect::setRangeDragAxes (
    QList< QCPAxis * > horizontal,
    QList< QCPAxis * > vertical )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

This method allows to set multiple axes up to react to horizontal and vertical dragging, and define specifically which axis reacts to which drag orientation (irrespective of the axis orientation).

15.557.3.41 setRangeZoom()

```
void QCPAxisRect::setRangeZoom (
    Qt::Orientations orientations )
```

Sets which axis orientation may be zoomed by the user with the mouse wheel. What orientation corresponds to which specific axis can be set with [setRangeZoomAxes\(QCPAxis *horizontal, QCPAxis *vertical\)](#). By default, the horizontal axis is the bottom axis (xAxis) and the vertical axis is the left axis (yAxis).

To disable range zooming entirely, pass `nullptr` as *orientations* or remove [QCP::iRangeZoom](#) from [QCustomPlot::setInteractions](#). To enable range zooming for both directions, pass `Qt::Horizontal | Qt::Vertical` as *orientations*.

In addition to setting *orientations* to a non-zero value, make sure [QCustomPlot::setInteractions](#) contains [QCP::iRangeZoom](#) to enable the range zooming interaction.

See also

[setRangeZoomFactor](#), [setRangeZoomAxes](#), [setRangeDrag](#)

15.557.3.42 setRangeZoomAxes() [1/3]

```
void QCPAxisRect::setRangeZoomAxes (
    QCPAxis * horizontal,
    QCPAxis * vertical )
```

Sets the axes whose range will be zoomed when [setRangeZoom](#) enables mouse wheel zooming on the [QCustomPlot](#) widget. Pass `nullptr` if no axis shall be zoomed in the respective orientation.

The two axes can be zoomed with different strengths, when different factors are passed to [setRangeZoomFactor\(double horizontalFactor, double verticalFactor\)](#). Use the overload taking a list of axes, if multiple axes (more than one per orientation) shall react to zooming interactions.

See also

[setRangeDragAxes](#)

15.557.3.43 setRangeZoomAxes() [2/3]

```
void QCPAxisRect::setRangeZoomAxes (
    QList< QCPAxis * > axes )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

This method allows to set up multiple axes to react to horizontal and vertical range zooming. The zoom orientation that the respective axis will react to is deduced from its orientation ([QCPAxis::orientation](#)). In the unusual case that you wish to e.g. zoom a vertically oriented axis with a horizontal zoom interaction, use the overload taking two separate lists for horizontal and vertical zooming.

15.557.3.44 `setRangeZoomAxes()` [3/3]

```
void QCPAxisRect::setRangeZoomAxes (
    QList< QCPAxis * > horizontal,
    QList< QCPAxis * > vertical )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

This method allows to set multiple axes up to react to horizontal and vertical zooming, and define specifically which axis reacts to which zoom orientation (irrespective of the axis orientation).

15.557.3.45 `setRangeZoomFactor()` [1/2]

```
void QCPAxisRect::setRangeZoomFactor (
    double factor )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets both the horizontal and vertical zoom *factor*.

15.557.3.46 `setRangeZoomFactor()` [2/2]

```
void QCPAxisRect::setRangeZoomFactor (
    double horizontalFactor,
    double verticalFactor )
```

Sets how strong one rotation step of the mouse wheel zooms, when range zoom was activated with [setRangeZoom](#). The two parameters *horizontalFactor* and *verticalFactor* provide a way to let the horizontal axis zoom at different rates than the vertical axis. Which axis is horizontal and which is vertical, can be set with [setRangeZoomAxes](#). When the zoom factor is greater than one, scrolling the mouse wheel backwards (towards the user) will zoom in (make the currently visible range smaller). For zoom factors smaller than one, the same scrolling direction will zoom out.

15.557.3.47 `setupFullAxesBox()`

```
void QCPAxisRect::setupFullAxesBox (
    bool connectRanges = false )
```

Convenience function to create an axis on each side that doesn't have any axes yet and set their visibility to true. Further, the top/right axes are assigned the following properties of the bottom/left axes:

- range ([QCPAxis::setRange](#))
- range reversed ([QCPAxis::setRangeReversed](#))
- scale type ([QCPAxis::setScaleType](#))
- tick visibility ([QCPAxis::setTicks](#))
- number format ([QCPAxis::setNumberFormat](#))
- number precision ([QCPAxis::setNumberPrecision](#))
- tick count of ticker ([QCPAxisTicker::setTickCount](#))
- tick origin of ticker ([QCPAxisTicker::setTickOrigin](#))

Tick label visibility ([QCPAxis::setTickLabels](#)) of the right and top axes are set to false.

If *connectRanges* is true, the [rangeChanged](#) signals of the bottom and left axes are connected to the [QCPAxis::setRange](#) slots of the top and right axes.

15.557.3.48 size()

```
QSize QCPAxisRect::size ( ) const [inline]
```

Returns the pixel size of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.557.3.49 top()

```
int QCPAxisRect::top ( ) const [inline]
```

Returns the pixel position of the top border of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.557.3.50 topLeft()

```
QPoint QCPAxisRect::topLeft ( ) const [inline]
```

Returns the top left corner of this axis rect in pixels. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.557.3.51 topRight()

```
QPoint QCPAxisRect::topRight ( ) const [inline]
```

Returns the top right corner of this axis rect in pixels. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.557.3.52 update()

```
void QCPAxisRect::update (
    UpdatePhase phase ) [virtual]
```

This method is called automatically upon replot and doesn't need to be called by users of [QCPAxisRect](#).

Calls the base class implementation to update the margins (see [QCPLayoutElement::update](#)), and finally passes the [rect](#) to the inset layout ([insetLayout](#)) and calls its [QCPIInsetLayout::update](#) function.

\see [baseclassmethod](#)

Reimplemented from [QCPLayoutElement](#).

Reimplemented in [QCPColorScaleAxisRectPrivate](#).

15.557.3.53 wheelEvent()

```
void QCPAxisRect::wheelEvent (
    QWheelEvent * event ) [protected], [virtual]
```

This event gets called when the user turns the mouse scroll wheel while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`.

The `event->angleDelta()` indicates how far the mouse wheel was turned, which is usually +/- 120 for single rotation steps. However, if the mouse wheel is turned rapidly, multiple steps may accumulate to one event, making the delta larger. On the other hand, if the wheel has very smooth steps or none at all, the delta may be smaller.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#)

Reimplemented from [QCPLayerable](#).

Reimplemented in [QCPColorScaleAxisRectPrivate](#).

15.557.3.54 width()

```
int QCPAxisRect::width ( ) const [inline]
```

Returns the pixel width of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.557.3.55 zoom() [1/2]

```
void QCPAxisRect::zoom (
    const QRectF & pixelRect )
```

Zooms in (or out) to the passed rectangular region *pixelRect*, given in pixel coordinates.

All axes of this axis rect will have their range zoomed accordingly. If you only wish to zoom specific axes, use the overloaded version of this method.

See also

[QCustomPlot::setSelectionRectMode](#)

15.557.3.56 zoom() [2/2]

```
void QCPAxisRect::zoom (
    const QRectF & pixelRect,
    const QList< QCPAxis * > & affectedAxes )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Zooms in (or out) to the passed rectangular region *pixelRect*, given in pixel coordinates.

Only the axes passed in *affectedAxes* will have their ranges zoomed accordingly.

See also

[QCustomPlot::setSelectionRectMode](#)

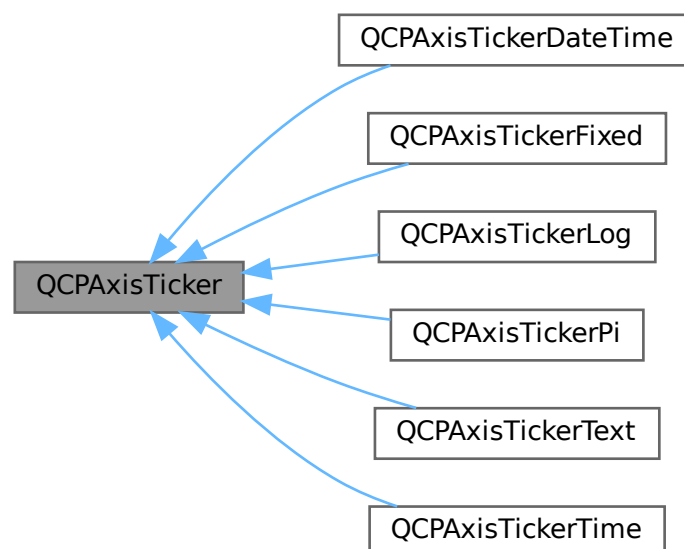
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.558 QCPAxisTicker Class Reference

The base class tick generator used by [QCPAxis](#) to create tick positions and tick labels.

Inheritance diagram for QCPAxisTicker:



Public Types

- enum [TickStepStrategy](#) { [tssReadability](#) , [tssMeetTickCount](#) }

Public Member Functions

- [QCPAxisTicker](#) ()
- [TickStepStrategy](#) [tickStepStrategy](#) () const
- int [tickCount](#) () const
- double [tickOrigin](#) () const
- void [setTickStepStrategy](#) ([TickStepStrategy](#) strategy)
- void [setTickCount](#) (int count)
- void [setTickOrigin](#) (double origin)
- virtual void [generate](#) (const [QCPRange](#) &range, const [QLocale](#) &locale, [QChar](#) formatChar, int precision, [QVector](#)< double > &ticks, [QVector](#)< double > *subTicks, [QVector](#)< [QString](#) > *tickLabels)

Protected Member Functions

- virtual double [getTickStep](#) (const [QCPRange](#) &range)
- virtual int [getSubTickCount](#) (double tickStep)
- virtual [QString](#) [getTickLabel](#) (double tick, const [QLocale](#) &locale, [QChar](#) formatChar, int precision)
- virtual [QVector](#)< double > [createTickVector](#) (double tickStep, const [QCPRange](#) &range)
- virtual [QVector](#)< double > [createSubTickVector](#) (int subTickCount, const [QVector](#)< double > &ticks)
- virtual [QVector](#)< [QString](#) > [createLabelVector](#) (const [QVector](#)< double > &ticks, const [QLocale](#) &locale, [QChar](#) formatChar, int precision)
- void [trimTicks](#) (const [QCPRange](#) &range, [QVector](#)< double > &ticks, bool keepOneOutlier) const
- double [pickClosest](#) (double target, const [QVector](#)< double > &candidates) const
- double [getMantissa](#) (double input, double *magnitude=nullptr) const
- double [cleanMantissa](#) (double input) const

Protected Attributes

- [TickStepStrategy](#) [mTickStepStrategy](#)
- int [mTickCount](#)
- double [mTickOrigin](#)

15.558.1 Detailed Description

The base class tick generator used by [QCPAxis](#) to create tick positions and tick labels.

Each [QCPAxis](#) has an internal [QCPAxisTicker](#) (or a subclass) in order to generate tick positions and tick labels for the current axis range. The ticker of an axis can be set via [QCPAxis::setTicker](#). Since that method takes a [QSharedPointer](#)<[QCPAxisTicker](#)>, multiple axes can share the same ticker instance.

This base class generates normal tick coordinates and numeric labels for linear axes. It picks a reasonable tick step (the separation between ticks) which results in readable tick labels. The number of ticks that should be approximately generated can be set via [setTickCount](#). Depending on the current tick step strategy ([setTickStepStrategy](#)), the algorithm either sacrifices readability to better match the specified tick count ([QCPAxisTicker::tssMeetTickCount](#)) or relaxes the tick count in favor of better tick steps ([QCPAxisTicker::tssReadability](#)), which is the default.

The following more specialized axis ticker subclasses are available, see details in the respective class documentation:

QCPAxisTickerFixed	td
QCPAxisTickerLog	td
QCPAxisTickerPi	td
QCPAxisTickerText	td
QCPAxisTickerDateTime	td
QCPAxisTickerTime	td

15.558.2 Creating own axis tickers

Creating own axis tickers can be achieved very easily by subclassing [QCPAxisTicker](#) and reimplementing some or all of the available virtual methods.

In the simplest case you might wish to just generate different tick steps than the other tickers, so you only reimplement the method `getTickStep`. If you additionally want control over the string that will be shown as tick label, reimplement `getTickLabel`.

If you wish to have complete control, you can generate the tick vectors and tick label vectors yourself by reimplementing `createTickVector` and `createLabelVector`. The default implementations use the previously mentioned virtual methods `getTickStep` and `getTickLabel`, but your reimplementations don't necessarily need to do so. For example in the case of unequal tick steps, the method `getTickStep` loses its usefulness and can be ignored.

The sub tick count between major ticks can be controlled with `getSubTickCount`. Full sub tick placement control is obtained by reimplementing `createSubTickVector`.

See the documentation of all these virtual methods in [QCPAxisTicker](#) for detailed information about the parameters and expected return values.

15.558.3 Member Enumeration Documentation

15.558.3.1 TickStepStrategy

enum [QCPAxisTicker::TickStepStrategy](#)

Defines the strategies that the axis ticker may follow when choosing the size of the tick step.

See also

[setTickStepStrategy](#)

Enumerator

tssReadability	A nicely readable tick step is prioritized over matching the requested number of ticks (see setTickCount)
tssMeetTickCount	Less readable tick steps are allowed which in turn facilitates getting closer to the requested tick count.

15.558.4 Constructor & Destructor Documentation

15.558.4.1 QCPAxisTicker()

`QCPAxisTicker::QCPAxisTicker ( )`

Constructs the ticker and sets reasonable default values. Axis tickers are commonly created managed by a `QSharedPointer`, which then can be passed to [QCPAxis::setTicker](#).

15.558.5 Member Function Documentation

15.558.5.1 createTickVector()

```
QVector< double > QCPAxisTicker::createTickVector (
    double tickStep,
    const QCPRange & range ) [protected], [virtual]
```

Reimplemented in [QCPAxisTickerText](#).

15.558.5.2 generate()

```
void QCPAxisTicker::generate (
    const QCPRange & range,
    const QLocale & locale,
```

```

    QChar formatChar,
    int precision,
    QVector< double > & ticks,
    QVector< double > * subTicks,
    QVector< QString > * tickLabels ) [virtual]

```

This is the method called by [QCPAxis](#) in order to actually generate tick coordinates (*ticks*), tick label strings (*tickLabels*) and sub tick coordinates (*subTicks*).

The ticks are generated for the specified *range*. The generated labels typically follow the specified *locale*, *formatChar* and number *precision*, however this might be different (or even irrelevant) for certain [QCPAxisTicker](#) subclasses.

The output parameter *ticks* is filled with the generated tick positions in axis coordinates. The output parameters *subTicks* and *tickLabels* are optional (set them to `nullptr` if not needed) and are respectively filled with sub tick coordinates, and tick label strings belonging to *ticks* by index.

15.558.5.3 getSubTickCount()

```

int QCPAxisTicker::getSubTickCount (
    double tickStep ) [protected], [virtual]

```

Reimplemented in [QCPAxisTickerText](#).

15.558.5.4 getTickLabel()

```

QString QCPAxisTicker::getTickLabel (
    double tick,
    const QLocale & locale,
    QChar formatChar,
    int precision ) [protected], [virtual]

```

Reimplemented in [QCPAxisTickerText](#).

15.558.5.5 getTickStep()

```

double QCPAxisTicker::getTickStep (
    const QCPRange & range ) [protected], [virtual]

```

Reimplemented in [QCPAxisTickerText](#).

15.558.5.6 setTickCount()

```

void QCPAxisTicker::setTickCount (
    int count )

```

Sets how many ticks this ticker shall aim to generate across the axis range. Note that *count* is not guaranteed to be matched exactly, as generating readable tick intervals may conflict with the requested number of ticks.

Whether the readability has priority over meeting the requested *count* can be specified with [setTickStepStrategy](#).

15.558.5.7 setTickOrigin()

```

void QCPAxisTicker::setTickOrigin (
    double origin )

```

Sets the mathematical coordinate (or "offset") of the zeroth tick. This tick coordinate is just a concept and doesn't need to be inside the currently visible axis range.

By default *origin* is zero, which for example yields ticks {-5, 0, 5, 10, 15,...} when the tick step is five. If *origin* is now set to 1 instead, the correspondingly generated ticks would be {-4, 1, 6, 11, 16,...}.

15.558.5.8 setTickStepStrategy()

```

void QCPAxisTicker::setTickStepStrategy (
    QCPAxisTicker::TickStepStrategy strategy )

```

Sets which strategy the axis ticker follows when choosing the size of the tick step. For the available strategies, see [TickStepStrategy](#).

The documentation for this class was generated from the following files:

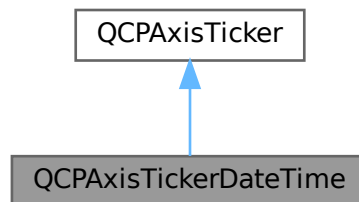
- applications/gui/genesys/qcustomplot.h

- applications/gui/genesys/qcustomplot.cpp

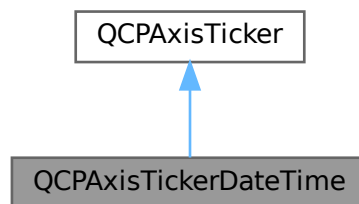
15.559 QCPAxisTickerDateTime Class Reference

Specialized axis ticker for calendar dates and times as axis ticks.

Inheritance diagram for QCPAxisTickerDateTime:



Collaboration diagram for QCPAxisTickerDateTime:



Public Member Functions

- [QCPAxisTickerDateTime](#) ()
- QString **dateTimeFormat** () const
- Qt::TimeSpec **dateTimeSpec** () const
- QTimeZone **timeZone** () const
- void [setDateTimeFormat](#) (const QString &format)
- void [setDateTimeSpec](#) (Qt::TimeSpec spec)
- void [setTimeZone](#) (const QTimeZone &zone)
- void [setTickOrigin](#) (double origin)
- void [setTickOrigin](#) (const QDateTime &origin)

Public Member Functions inherited from [QCPAxisTicker](#)

- [QCPAxisTicker](#) ()
- [TickStepStrategy](#) **tickStepStrategy** () const
- int **tickCount** () const
- double **tickOrigin** () const

- void [setTickStepStrategy](#) ([TickStepStrategy](#) strategy)
- void [setTickCount](#) (int count)
- void [setTickOrigin](#) (double origin)
- virtual void [generate](#) (const [QCPRange](#) &range, const QLocale &locale, QChar formatChar, int precision, QVector< double > &ticks, QVector< double > *subTicks, QVector< QString > *tickLabels)

Static Public Member Functions

- static QDateTime [keyToDateTime](#) (double key)
- static double [dateTimeToKey](#) (const QDateTime &dateTime)
- static double [dateTimeToKey](#) (const QDate &date, Qt::TimeSpec timeSpec=Qt::LocalTime)

Protected Types

- enum **DateStrategy** { **dsNone** , **dsUniformTimeInDay** , **dsUniformDayInMonth** }

Protected Member Functions

- virtual double [getTickStep](#) (const [QCPRange](#) &range) Q_DECL_OVERRIDE
- virtual int [getSubTickCount](#) (double tickStep) Q_DECL_OVERRIDE
- virtual QString [getTickLabel](#) (double tick, const QLocale &locale, QChar formatChar, int precision) Q_DECL_OVERRIDE
- virtual QVector< double > [createTickVector](#) (double tickStep, const [QCPRange](#) &range) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPAxisTicker](#)

- virtual QVector< double > **createSubTickVector** (int subTickCount, const QVector< double > &ticks)
- virtual QVector< QString > **createLabelVector** (const QVector< double > &ticks, const QLocale &locale, QChar formatChar, int precision)
- void **trimTicks** (const [QCPRange](#) &range, QVector< double > &ticks, bool keepOneOutlier) const
- double **pickClosest** (double target, const QVector< double > &candidates) const
- double **getMantissa** (double input, double *magnitude=NULLPTR) const
- double **cleanMantissa** (double input) const

Protected Attributes

- QString **mDateTimeFormat**
- Qt::TimeSpec **mDateTimeSpec**
- QTimeZone **mTimeZone**
- enum [QCPAxisTickerDateTime::DateStrategy](#) **mDateStrategy**

Protected Attributes inherited from [QCPAxisTicker](#)

- [TickStepStrategy](#) **mTickStepStrategy**
- int **mTickCount**
- double **mTickOrigin**

Additional Inherited Members

Public Types inherited from [QCPAxisTicker](#)

- enum [TickStepStrategy](#) { [tssReadability](#) , [tssMeetTickCount](#) }

15.559.1 Detailed Description

Specialized axis ticker for calendar dates and times as axis ticks.

This [QCPAxisTicker](#) subclass generates ticks that correspond to real calendar dates and times. The plot axis coordinate is interpreted as Unix Time, so seconds since Epoch (January 1, 1970, 00:00 UTC). This is also used for example by [QDateTime](#) in the `toTime_t()`/`setTime_t()` methods with a precision of one second. Since Qt 4.7, millisecond accuracy can be obtained from [QDateTime](#) by using `QDateTime::fromMSecsSinceEpoch()`/`toMSecsSinceEpoch()`. The static methods [dateTimeToKey](#) and [keyToDateTime](#) conveniently perform this conversion achieving a precision of one millisecond on all Qt versions.

The format of the date/time display in the tick labels is controlled with [setDateTimeFormat](#). If a different time spec or time zone shall be used for the tick label appearance, see [setDateTimeSpec](#) or [setTimeZone](#), respectively.

This ticker produces unequal tick spacing in order to provide intuitive date and time-of-day ticks. For example, if the axis range spans a few years such that there is one tick per year, ticks will be positioned on 1. January of every year. This is intuitive but, due to leap years, will result in slightly unequal tick intervals (visually unnoticeable). The same can be seen in the image above: even though the number of days varies month by month, this ticker generates ticks on the same day of each month.

If you would like to change the date/time that is used as a (mathematical) starting date for the ticks, use the [setTickOrigin\(const QDateTime &origin\)](#) method overload, which takes a [QDateTime](#). If you pass 15. July, 9:45 to this method, the yearly ticks will end up on 15. July at 9:45 of every year.

The ticker can be created and assigned to an axis like this:

Note

If you rather wish to display relative times in terms of days, hours, minutes, seconds and milliseconds, and are not interested in the intricacies of real calendar dates with months and (leap) years, have a look at [QCPAxisTickerTime](#) instead.

15.559.2 Constructor & Destructor Documentation

15.559.2.1 QCPAxisTickerDateTime()

```
QCPAxisTickerDateTime::QCPAxisTickerDateTime ( )
```

Constructs the ticker and sets reasonable default values. Axis tickers are commonly created managed by a [QSharedPointer](#), which then can be passed to [QCPAxis::setTicker](#).

15.559.3 Member Function Documentation

15.559.3.1 createTickVector()

```
QVector< double > QCPAxisTickerDateTime::createTickVector (
    double tickStep,
    const QCPRange & range ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

15.559.3.2 dateTimeToKey() [1/2]

```
double QCPAxisTickerDateTime::dateTimeToKey (
    const QDate & date,
    Qt::TimeSpec timeSpec = Qt::LocalTime ) [static]
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

A convenience method which turns a [QDate](#) object into a double value that corresponds to seconds since Epoch (1. Jan 1970, 00:00 UTC). This is the format used as axis coordinates by [QCPAxisTickerDateTime](#).

The returned value will be the start of the passed day of *date*, interpreted in the given *timeSpec*.

See also

[keyToDateTime](#)

15.559.3.3 dateTimeToKey() [2/2]

```
double QCPAxisTickerDateTime::dateTimeToKey (
    const QDateTime & dateTime ) [static]
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

A convenience method which turns a QDateTime object into a double value that corresponds to seconds since Epoch (1. Jan 1970, 00:00 UTC). This is the format used as axis coordinates by [QCPAxisTickerDateTime](#).

The accuracy achieved by this method is one millisecond, irrespective of the used Qt version (it works around the lack of a QDateTime::toMsecsSinceEpoch in Qt 4.6)

See also

[keyToDateTime](#)

15.559.3.4 getSubTickCount()

```
int QCPAxisTickerDateTime::getSubTickCount (
    double tickStep ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

15.559.3.5 getTickLabel()

```
QString QCPAxisTickerDateTime::getTickLabel (
    double tick,
    const QLocale & locale,
    QChar formatChar,
    int precision ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

15.559.3.6 getTickStep()

```
double QCPAxisTickerDateTime::getTickStep (
    const QCPRange & range ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

15.559.3.7 keyToDateTime()

```
QDateTime QCPAxisTickerDateTime::keyToDateTime (
    double key ) [static]
```

A convenience method which turns *key* (in seconds since Epoch 1. Jan 1970, 00:00 UTC) into a QDateTime object. This can be used to turn axis coordinates to actual QDateTime objects.

The accuracy achieved by this method is one millisecond, irrespective of the used Qt version (it works around the lack of a QDateTime::fromMsecsSinceEpoch in Qt 4.6)

See also

[dateTimeToKey](#)

15.559.3.8 setDateTimeFormat()

```
void QCPAxisTickerDateTime::setDateTimeFormat (
    const QString & format )
```

Sets the format in which dates and times are displayed as tick labels. For details about the *format* string, see the documentation of QDateTime::toString().

Typical expressions are

d	The day as a number without a leading zero (1 to 31)
dd	The day as a number with a leading zero (01 to 31)
ddd	The abbreviated localized day name (e.g. 'Mon' to 'Sun'). Uses the system locale to localize the name, i.e. <code>QLocale::system()</code> .
dddd	The long localized day name (e.g. 'Monday' to 'Sunday'). Uses the system locale to localize the name, i.e. <code>QLocale::system()</code> .
M	The month as a number without a leading zero (1 to 12)
MM	The month as a number with a leading zero (01 to 12)
MMM	The abbreviated localized month name (e.g. 'Jan' to 'Dec'). Uses the system locale to localize the name, i.e. <code>QLocale::system()</code> .
MMMM	The long localized month name (e.g. 'January' to 'December'). Uses the system locale to localize the name, i.e. <code>QLocale::system()</code> .
yy	The year as a two digit number (00 to 99)
yyyy	The year as a four digit number. If the year is negative, a minus sign is prepended, making five characters.
h	The hour without a leading zero (0 to 23 or 1 to 12 if AM/PM display)
hh	The hour with a leading zero (00 to 23 or 01 to 12 if AM/PM display)
H	The hour without a leading zero (0 to 23, even with AM/PM display)
HH	The hour with a leading zero (00 to 23, even with AM/PM display)
m	The minute without a leading zero (0 to 59)
mm	The minute with a leading zero (00 to 59)
s	The whole second, without any leading zero (0 to 59)
ss	The whole second, with a leading zero where applicable (00 to 59)
z	The fractional part of the second, to go after a decimal point, without trailing zeroes (0 to 999). Thus "s.z" reports the seconds to full available (millisecond) precision without trailing zeroes.
zzz	The fractional part of the second, to millisecond precision, including trailing zeroes where applicable (000 to 999).
AP or A	Use AM/PM display. A/AP will be replaced by an upper-case version of either <code>QLocale::amText()</code> or <code>QLocale::pmText()</code> .
ap or a	Use am/pm display. a/ap will be replaced by a lower-case version of either <code>QLocale::amText()</code> or <code>QLocale::pmText()</code> .
t	The timezone (for example "CEST")

Newlines can be inserted with `"\n"`, literal strings (even when containing above expressions) by encapsulating them using single-quotes. A literal single quote can be generated by using two consecutive single quotes in the format.

See also

[setDateTimeSpec](#), [setTimeZone](#)

15.559.3.9 setDateTimeSpec()

```
void QCPAxisTickerDateTime::setDateTimeSpec (
    Qt::TimeSpec spec )
```

Sets the time spec that is used for creating the tick labels from corresponding dates/times.

The default value of `QDateTime` objects (and also [QCPAxisTickerDateTime](#)) is `Qt::LocalTime`. However, if the displayed tick labels shall be given in UTC, set *spec* to `Qt::UTC`.

Tick labels corresponding to other time zones can be achieved with [setTimeZone](#) (which sets *spec* to `Qt::TimeZone` internally). Note that if *spec* is afterwards set to not be `Qt::TimeZone` again, the [setTimeZone](#) setting will be ignored accordingly.

See also

[setDateTimeFormat](#), [setTimeZone](#)

15.559.3.10 setTickOrigin() [1/2]

```
void QCPAxisTickerDateTime::setTickOrigin (
    const QDateTime & origin )
```

Sets the tick origin (see [QCPAxisTicker::setTickOrigin](#)) as a QDateTime *origin*.

This is useful to define the month/day/time recurring at greater tick interval steps. For example, If you pass 15. July, 9:45 to this method and the tick interval happens to be one tick per year, the ticks will end up on 15. July at 9:45 of every year.

15.559.3.11 setTickOrigin() [2/2]

```
void QCPAxisTickerDateTime::setTickOrigin (
    double origin )
```

Sets the tick origin (see [QCPAxisTicker::setTickOrigin](#)) in seconds since Epoch (1. Jan 1970, 00:00 UTC). For the date time ticker it might be more intuitive to use the overload which directly takes a QDateTime, see [setTickOrigin\(const QDateTime &origin\)](#).

This is useful to define the month/day/time recurring at greater tick interval steps. For example, If you pass 15. July, 9:45 to this method and the tick interval happens to be one tick per year, the ticks will end up on 15. July at 9:45 of every year.

15.559.3.12 setTimeZone()

```
void QCPAxisTickerDateTime::setTimeZone (
    const QTimeZone & zone )
```

Sets the time zone that is used for creating the tick labels from corresponding dates/times. The time spec ([setDateTimeSpec](#)) is set to `Qt::TimeZone`.

See also

[setDateTimeFormat](#), [setTimeZone](#)

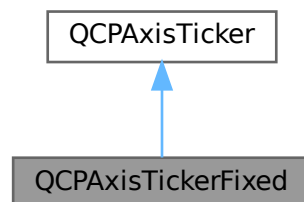
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

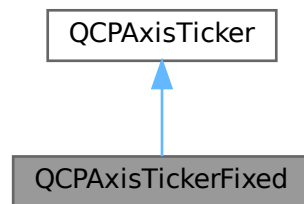
15.560 QCPAxisTickerFixed Class Reference

Specialized axis ticker with a fixed tick step.

Inheritance diagram for QCPAxisTickerFixed:



Collaboration diagram for QCPAxisTickerFixed:



Public Types

- enum [ScaleStrategy](#) { [ssNone](#) , [ssMultiples](#) , [ssPowers](#) }

Public Types inherited from [QCPAxisTicker](#)

- enum [TickStepStrategy](#) { [tssReadability](#) , [tssMeetTickCount](#) }

Public Member Functions

- [QCPAxisTickerFixed](#) ()
- double **tickStep** () const
- [ScaleStrategy](#) **scaleStrategy** () const
- void [setTickStep](#) (double step)
- void [setScaleStrategy](#) ([ScaleStrategy](#) strategy)

Public Member Functions inherited from [QCPAxisTicker](#)

- [QCPAxisTicker](#) ()
- [TickStepStrategy](#) **tickStepStrategy** () const
- int **tickCount** () const
- double **tickOrigin** () const
- void [setTickStepStrategy](#) ([TickStepStrategy](#) strategy)
- void [setTickCount](#) (int count)
- void [setTickOrigin](#) (double origin)
- virtual void [generate](#) (const [QCPRange](#) &range, const [QLocale](#) &locale, [QChar](#) formatChar, int precision, [QVector](#)< double > &ticks, [QVector](#)< double > *subTicks, [QVector](#)< [QString](#) > *tickLabels)

Protected Member Functions

- virtual double [getTickStep](#) (const [QCPRange](#) &range) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPAxisTicker](#)

- virtual int [getSubTickCount](#) (double tickStep)
- virtual [QString](#) [getTickLabel](#) (double tick, const [QLocale](#) &locale, [QChar](#) formatChar, int precision)
- virtual [QVector](#)< double > [createTickVector](#) (double tickStep, const [QCPRange](#) &range)
- virtual [QVector](#)< double > **createSubTickVector** (int subTickCount, const [QVector](#)< double > &ticks)
- virtual [QVector](#)< [QString](#) > **createLabelVector** (const [QVector](#)< double > &ticks, const [QLocale](#) &locale, [QChar](#) formatChar, int precision)
- void **trimTicks** (const [QCPRange](#) &range, [QVector](#)< double > &ticks, bool keepOneOutlier) const

- double **pickClosest** (double target, const QVector< double > &candidates) const
- double **getMantissa** (double input, double *magnitude=nullptr) const
- double **cleanMantissa** (double input) const

Protected Attributes

- double **mTickStep**
- [ScaleStrategy](#) **mScaleStrategy**

Protected Attributes inherited from [QCPAxisTicker](#)

- [TickStepStrategy](#) **mTickStepStrategy**
- int **mTickCount**
- double **mTickOrigin**

15.560.1 Detailed Description

Specialized axis ticker with a fixed tick step.

This [QCPAxisTicker](#) subclass generates ticks with a fixed tick step set with [setTickStep](#). It is also possible to allow integer multiples and integer powers of the specified tick step with [setScaleStrategy](#).

A typical application of this ticker is to make an axis only display integers, by setting the tick step of the ticker to 1.0 and the scale strategy to [ssMultiples](#).

Another case is when a certain number has a special meaning and axis ticks should only appear at multiples of that value. In this case you might also want to consider [QCPAxisTickerPi](#) because despite the name it is not limited to only pi symbols/values.

The ticker can be created and assigned to an axis like this:

15.560.2 Member Enumeration Documentation

15.560.2.1 ScaleStrategy

enum [QCPAxisTickerFixed::ScaleStrategy](#)

Defines how the axis ticker may modify the specified tick step ([setTickStep](#)) in order to control the number of ticks in the axis range.

See also

[setScaleStrategy](#)

Enumerator

ssNone	Modifications are not allowed, the specified tick step is absolutely fixed. This might cause a high tick density and overlapping labels if the axis range is zoomed out.
ssMultiples	An integer multiple of the specified tick step is allowed. The used factor follows the base class properties of setTickStepStrategy and setTickCount .
ssPowers	An integer power of the specified tick step is allowed.

15.560.3 Constructor & Destructor Documentation

15.560.3.1 QCPAxisTickerFixed()

[QCPAxisTickerFixed::QCPAxisTickerFixed](#) ()

Constructs the ticker and sets reasonable default values. Axis tickers are commonly created managed by a QSharedPointer, which then can be passed to [QCPAxis::setTicker](#).

15.560.4 Member Function Documentation

15.560.4.1 `getTickStep()`

```
double QCPAxisTickerFixed::getTickStep (
    const QCPRange & range ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

15.560.4.2 `setScaleStrategy()`

```
void QCPAxisTickerFixed::setScaleStrategy (
    QCPAxisTickerFixed::ScaleStrategy strategy )
```

Sets whether the specified tick step ([setTickStep](#)) is absolutely fixed or whether modifications may be applied to it before calculating the finally used tick step, such as permitting multiples or powers. See [ScaleStrategy](#) for details. The default strategy is `ssNone`, which means the tick step is absolutely fixed.

15.560.4.3 `setTickStep()`

```
void QCPAxisTickerFixed::setTickStep (
    double step )
```

Sets the fixed tick interval to *step*.

The axis ticker will only use this tick step when generating axis ticks. This might cause a very high tick density and overlapping labels if the axis range is zoomed out. Using [setScaleStrategy](#) it is possible to relax the fixed step and also allow multiples or powers of *step*. This will enable the ticker to reduce the number of ticks to a reasonable amount (see [setTickCount](#)).

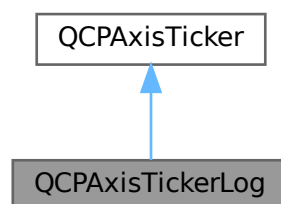
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

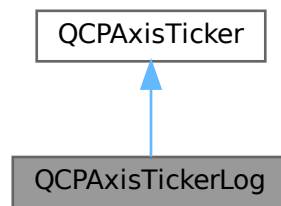
15.561 QCPAxisTickerLog Class Reference

Specialized axis ticker suited for logarithmic axes.

Inheritance diagram for QCPAxisTickerLog:



Collaboration diagram for QCPAxisTickerLog:



Public Member Functions

- [QCPAxisTickerLog](#) ()
- double **logBase** () const
- int **subTickCount** () const
- void [setLogBase](#) (double base)
- void [setSubTickCount](#) (int subTicks)

Public Member Functions inherited from [QCPAxisTicker](#)

- [QCPAxisTicker](#) ()
- [TickStepStrategy](#) **tickStepStrategy** () const
- int **tickCount** () const
- double **tickOrigin** () const
- void [setTickStepStrategy](#) ([TickStepStrategy](#) strategy)
- void [setTickCount](#) (int count)
- void [setTickOrigin](#) (double origin)
- virtual void [generate](#) (const [QCPRange](#) &range, const QLocale &locale, QChar formatChar, int precision, QVector< double > &ticks, QVector< double > *subTicks, QVector< QString > *tickLabels)

Protected Member Functions

- virtual int [getSubTickCount](#) (double tickStep) Q_DECL_OVERRIDE
- virtual QVector< double > [createTickVector](#) (double tickStep, const [QCPRange](#) &range) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPAxisTicker](#)

- virtual double [getTickStep](#) (const [QCPRange](#) &range)
- virtual QString [getTickLabel](#) (double tick, const QLocale &locale, QChar formatChar, int precision)
- virtual QVector< double > **createSubTickVector** (int subTickCount, const QVector< double > &ticks)
- virtual QVector< QString > **createLabelVector** (const QVector< double > &ticks, const QLocale &locale, QChar formatChar, int precision)
- void **trimTicks** (const [QCPRange](#) &range, QVector< double > &ticks, bool keepOneOutlier) const
- double **pickClosest** (double target, const QVector< double > &candidates) const
- double **getMantissa** (double input, double *magnitude=nullptr) const
- double **cleanMantissa** (double input) const

Protected Attributes

- double **mLogBase**
- int **mSubTickCount**
- double **mLogBaseLnInv**

Protected Attributes inherited from [QCPAxisTicker](#)

- [TickStepStrategy](#) **mTickStepStrategy**
- int **mTickCount**
- double **mTickOrigin**

Additional Inherited Members**Public Types inherited from [QCPAxisTicker](#)**

- enum [TickStepStrategy](#) { [tssReadability](#) , [tssMeetTickCount](#) }

15.561.1 Detailed Description

Specialized axis ticker suited for logarithmic axes.

This [QCPAxisTicker](#) subclass generates ticks with unequal tick intervals suited for logarithmic axis scales. The ticks are placed at powers of the specified log base ([setLogBase](#)).

Especially in the case of a log base equal to 10 (the default), it might be desirable to have tick labels in the form of powers of ten without mantissa display. To achieve this, set the number precision ([QCPAxis::setNumberPrecision](#)) to zero and the number format ([QCPAxis::setNumberFormat](#)) to scientific (exponential) display with beautifully typeset decimal powers, so a format string of "eb". This will result in the following axis tick labels:

The ticker can be created and assigned to an axis like this:

Note that the nature of logarithmic ticks imply that there exists a smallest possible tick step, corresponding to one multiplication by the log base. If the user zooms in further than that, no new ticks would appear, leading to very sparse or even no axis ticks on the axis. To prevent this situation, this ticker falls back to regular tick generation if the axis range would be covered by too few logarithmically placed ticks.

15.561.2 Constructor & Destructor Documentation**15.561.2.1 [QCPAxisTickerLog](#)()**

```
QCPAxisTickerLog::QCPAxisTickerLog ( )
```

Constructs the ticker and sets reasonable default values. Axis tickers are commonly created managed by a [QSharedPointer](#), which then can be passed to [QCPAxis::setTicker](#).

15.561.3 Member Function Documentation**15.561.3.1 [createTickVector](#)()**

```
QVector< double > QCPAxisTickerLog::createTickVector (
    double tickStep,
    const QCPRange & range ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

15.561.3.2 [getSubTickCount](#)()

```
int QCPAxisTickerLog::getSubTickCount (
    double tickStep ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

15.561.3.3 [setLogBase](#)()

```
void QCPAxisTickerLog::setLogBase (
    double base )
```

Sets the logarithm base used for tick coordinate generation. The ticks will be placed at integer powers of *base*.

15.561.3.4 setSubTickCount()

```
void QCPAxisTickerLog::setSubTickCount (
    int subTicks )
```

Sets the number of sub ticks in a tick interval. Within each interval, the sub ticks are spaced linearly to provide a better visual guide, so the sub tick density increases toward the higher tick.

Note that *subTicks* is the number of sub ticks (not sub intervals) in one tick interval. So in the case of logarithm base 10 an intuitive sub tick spacing would be achieved with eight sub ticks (the default). This means e.g. between the ticks 10 and 100 there will be eight ticks, namely at 20, 30, 40, 50, 60, 70, 80 and 90.

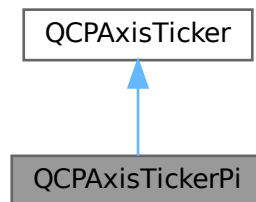
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

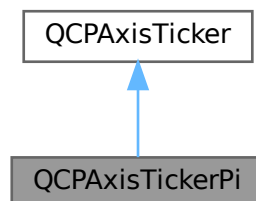
15.562 QCPAxisTickerPi Class Reference

Specialized axis ticker to display ticks in units of an arbitrary constant, for example pi.

Inheritance diagram for QCPAxisTickerPi:



Collaboration diagram for QCPAxisTickerPi:

**Public Types**

- enum [FractionStyle](#) { [fsFloatingPoint](#) , [fsAsciiFractions](#) , [fsUnicodeFractions](#) }

Public Types inherited from [QCPAxisTicker](#)

- enum [TickStepStrategy](#) { [tssReadability](#) , [tssMeetTickCount](#) }

Public Member Functions

- [QCPAxisTickerPi](#) ()
- QString **piSymbol** () const
- double **piValue** () const
- bool **periodicity** () const
- [FractionStyle](#) **fractionStyle** () const
- void **setPiSymbol** (QString symbol)
- void **setPiValue** (double pi)
- void **setPeriodicity** (int multiplesOfPi)
- void **setFractionStyle** ([FractionStyle](#) style)

Public Member Functions inherited from [QCPAxisTicker](#)

- [QCPAxisTicker](#) ()
- [TickStepStrategy](#) **tickStepStrategy** () const
- int **tickCount** () const
- double **tickOrigin** () const
- void **setTickStepStrategy** ([TickStepStrategy](#) strategy)
- void **setTickCount** (int count)
- void **setTickOrigin** (double origin)
- virtual void **generate** (const [QCPRange](#) &range, const QLocale &locale, QChar formatChar, int precision, QVector< double > &ticks, QVector< double > *subTicks, QVector< QString > *tickLabels)

Protected Member Functions

- virtual double **getTickStep** (const [QCPRange](#) &range) Q_DECL_OVERRIDE
- virtual int **getSubTickCount** (double tickStep) Q_DECL_OVERRIDE
- virtual QString **getTickLabel** (double tick, const QLocale &locale, QChar formatChar, int precision) Q_DECL_OVERRIDE
- void **simplifyFraction** (int &numerator, int &denominator) const
- QString **fractionToString** (int numerator, int denominator) const
- QString **unicodeFraction** (int numerator, int denominator) const
- QString **unicodeSuperscript** (int number) const
- QString **unicodeSubscript** (int number) const

Protected Member Functions inherited from [QCPAxisTicker](#)

- virtual QVector< double > **createTickVector** (double tickStep, const [QCPRange](#) &range)
- virtual QVector< double > **createSubTickVector** (int subTickCount, const QVector< double > &ticks)
- virtual QVector< QString > **createLabelVector** (const QVector< double > &ticks, const QLocale &locale, QChar formatChar, int precision)
- void **trimTicks** (const [QCPRange](#) &range, QVector< double > &ticks, bool keepOneOutlier) const
- double **pickClosest** (double target, const QVector< double > &candidates) const
- double **getMantissa** (double input, double *magnitude=nullptr) const
- double **cleanMantissa** (double input) const

Protected Attributes

- QString **mPiSymbol**
- double **mPiValue**
- int **mPeriodicity**
- [FractionStyle](#) **mFractionStyle**
- double **mPiTickStep**

Protected Attributes inherited from [QCPAxisTicker](#)

- [TickStepStrategy](#) `mTickStepStrategy`
- `int` `mTickCount`
- `double` `mTickOrigin`

15.562.1 Detailed Description

Specialized axis ticker to display ticks in units of an arbitrary constant, for example pi.

This [QCPAxisTicker](#) subclass generates ticks that are expressed with respect to a given symbolic constant with a numerical value specified with [setPiValue](#) and an appearance in the tick labels specified with [setPiSymbol](#).

Ticks may be generated at fractions of the symbolic constant. How these fractions appear in the tick label can be configured with [setFractionStyle](#).

The ticker can be created and assigned to an axis like this:

15.562.2 Member Enumeration Documentation

15.562.2.1 FractionStyle

enum [QCPAxisTickerPi::FractionStyle](#)

Defines how fractions should be displayed in tick labels.

See also

[setFractionStyle](#)

Enumerator

<code>fsFloatingPoint</code>	Fractions are displayed as regular decimal floating point numbers, e.g. "0.25" or "0.125".
<code>fsAsciiFractions</code>	Fractions are written as rationals using ASCII characters only, e.g. "1/4" or "1/8".
<code>fsUnicodeFractions</code>	Fractions are written using sub- and superscript UTF-8 digits and the fraction symbol.

15.562.3 Constructor & Destructor Documentation

15.562.3.1 QCPAxisTickerPi()

`QCPAxisTickerPi::QCPAxisTickerPi ( )`

Constructs the ticker and sets reasonable default values. Axis tickers are commonly created managed by a `QSharedPointer`, which then can be passed to [QCPAxis::setTicker](#).

15.562.4 Member Function Documentation

15.562.4.1 getSubTickCount()

```
int QCPAxisTickerPi::getSubTickCount (
    double tickStep ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

15.562.4.2 getTickLabel()

```
QString QCPAxisTickerPi::getTickLabel (
    double tick,
    const QLocale & locale,
    QChar formatChar,
    int precision ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

15.562.4.3 getTickStep()

```
double QCPAxisTickerPi::getTickStep (
    const QCPRange & range ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

15.562.4.4 setFractionStyle()

```
void QCPAxisTickerPi::setFractionStyle (
    QCPAxisTickerPi::FractionStyle style )
```

Sets how the numerical/fractional part preceding the symbolic constant is displayed in tick labels. See [FractionStyle](#) for the various options.

15.562.4.5 setPeriodicity()

```
void QCPAxisTickerPi::setPeriodicity (
    int multiplesOfPi )
```

Sets whether the axis labels shall appear periodically and if so, at which multiplicity of the symbolic constant. To disable periodicity, set *multiplesOfPi* to zero.

For example, an axis that identifies 0 with 2pi would set *multiplesOfPi* to two.

15.562.4.6 setPiSymbol()

```
void QCPAxisTickerPi::setPiSymbol (
    QString symbol )
```

Sets how the symbol part (which is always a suffix to the number) shall appear in the axis tick label.

If a space shall appear between the number and the symbol, make sure the space is contained in *symbol*.

15.562.4.7 setPiValue()

```
void QCPAxisTickerPi::setPiValue (
    double pi )
```

Sets the numerical value that the symbolic constant has.

This will be used to place the appropriate fractions of the symbol at the respective axis coordinates.

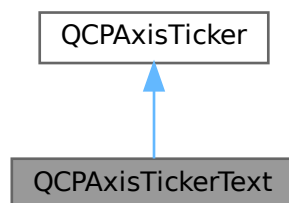
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

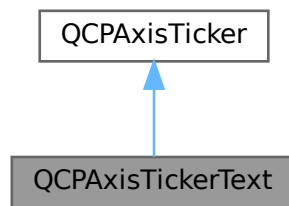
15.563 QCPAxisTickerText Class Reference

Specialized axis ticker which allows arbitrary labels at specified coordinates.

Inheritance diagram for QCPAxisTickerText:



Collaboration diagram for QCPAxisTickerText:



Public Member Functions

- [QCPAxisTickerText](#) ()
- [QMap](#)< double, [QString](#) > & [ticks](#) ()
- int [subTickCount](#) () const
- void [setTicks](#) (const [QMap](#)< double, [QString](#) > &[ticks](#))
- void [setTicks](#) (const [QVector](#)< double > &positions, const [QVector](#)< [QString](#) > &labels)
- void [setSubTickCount](#) (int subTicks)
- void [clear](#) ()
- void [addTick](#) (double position, const [QString](#) &label)
- void [addTicks](#) (const [QMap](#)< double, [QString](#) > &[ticks](#))
- void [addTicks](#) (const [QVector](#)< double > &positions, const [QVector](#)< [QString](#) > &labels)

Public Member Functions inherited from [QCPAxisTicker](#)

- [QCPAxisTicker](#) ()
- [TickStepStrategy](#) [tickStepStrategy](#) () const
- int [tickCount](#) () const
- double [tickOrigin](#) () const
- void [setTickStepStrategy](#) ([TickStepStrategy](#) strategy)
- void [setTickCount](#) (int count)
- void [setTickOrigin](#) (double origin)
- virtual void [generate](#) (const [QCPRange](#) &range, const [QLocale](#) &locale, [QChar](#) formatChar, int precision, [QVector](#)< double > &ticks, [QVector](#)< double > *subTicks, [QVector](#)< [QString](#) > *tickLabels)

Protected Member Functions

- virtual double [getTickStep](#) (const [QCPRange](#) &range) [Q_DECL_OVERRIDE](#)
- virtual int [getSubTickCount](#) (double tickStep) [Q_DECL_OVERRIDE](#)
- virtual [QString](#) [getTickLabel](#) (double tick, const [QLocale](#) &locale, [QChar](#) formatChar, int precision) [Q_DECL_OVERRIDE](#)
- virtual [QVector](#)< double > [createTickVector](#) (double tickStep, const [QCPRange](#) &range) [Q_DECL_OVERRIDE](#)

Protected Member Functions inherited from [QCPAxisTicker](#)

- virtual [QVector](#)< double > [createSubTickVector](#) (int subTickCount, const [QVector](#)< double > &ticks)
- virtual [QVector](#)< [QString](#) > [createLabelVector](#) (const [QVector](#)< double > &ticks, const [QLocale](#) &locale, [QChar](#) formatChar, int precision)
- void [trimTicks](#) (const [QCPRange](#) &range, [QVector](#)< double > &ticks, bool keepOneOutlier) const

- double **pickClosest** (double target, const QVector< double > &candidates) const
- double **getMantissa** (double input, double *magnitude=nullptr) const
- double **cleanMantissa** (double input) const

Protected Attributes

- QMap< double, QString > **mTicks**
- int **mSubTickCount**

Protected Attributes inherited from [QCPAxisTicker](#)

- [TickStepStrategy](#) **mTickStepStrategy**
- int **mTickCount**
- double **mTickOrigin**

Additional Inherited Members

Public Types inherited from [QCPAxisTicker](#)

- enum [TickStepStrategy](#) { [tssReadability](#) , [tssMeetTickCount](#) }

15.563.1 Detailed Description

Specialized axis ticker which allows arbitrary labels at specified coordinates.

This [QCPAxisTicker](#) subclass generates ticks which can be directly specified by the user as coordinates and associated strings. They can be passed as a whole with [setTicks](#) or one at a time with [addTick](#). Alternatively you can directly access the internal storage via [ticks](#) and modify the tick/label data there.

This is useful for cases where the axis represents categories rather than numerical values.

If you are updating the ticks of this ticker regularly and in a dynamic fashion (e.g. dependent on the axis range), it is a sign that you should probably create an own ticker by subclassing [QCPAxisTicker](#), instead of using this one.

The ticker can be created and assigned to an axis like this:

15.563.2 Constructor & Destructor Documentation

15.563.2.1 QCPAxisTickerText()

```
QCPAxisTickerText::QCPAxisTickerText ( )
```

Constructs the ticker and sets reasonable default values. Axis tickers are commonly created managed by a QSharedPointer, which then can be passed to [QCPAxis::setTicker](#).

15.563.3 Member Function Documentation

15.563.3.1 addTick()

```
void QCPAxisTickerText::addTick (
    double position,
    const QString & label )
```

Adds a single tick to the axis at the given axis coordinate *position*, with the provided tick *label*.

See also

[addTicks](#), [setTicks](#), [clear](#)

15.563.3.2 addTicks() [1/2]

```
void QCPAxisTickerText::addTicks (
    const QMap< double, QString > & ticks )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided *ticks* to the ones already existing. The map key of *ticks* corresponds to the axis coordinate, and the map value is the string that will appear as tick label.

An alternative to manipulate ticks is to directly access the internal storage with the [ticks](#) getter.

See also

[addTick](#), [setTicks](#), [clear](#)

15.563.3.3 addTicks() [2/2]

```
void QCPAxisTickerText::addTicks (
    const QVector< double > & positions,
    const QVector< QString > & labels )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided ticks to the ones already existing. The entries of *positions* correspond to the axis coordinates, and the entries of *labels* are the respective strings that will appear as tick labels.

An alternative to manipulate ticks is to directly access the internal storage with the [ticks](#) getter.

See also

[addTick](#), [setTicks](#), [clear](#)

15.563.3.4 clear()

```
void QCPAxisTickerText::clear ( )
```

Clears all ticks.

An alternative to manipulate ticks is to directly access the internal storage with the [ticks](#) getter.

See also

[setTicks](#), [addTicks](#), [addTick](#)

15.563.3.5 createTickVector()

```
QVector< double > QCPAxisTickerText::createTickVector (
    double tickStep,
    const QCPRange & range ) [protected], [virtual]
```

Returns the externally provided tick coordinates which are in the specified *range*. If available, one tick above and below the range is provided in addition, to allow possible sub tick calculation. The parameter *tickStep* is ignored.

\seebaseclassmethod

Reimplemented from [QCPAxisTicker](#).

15.563.3.6 getSubTickCount()

```
int QCPAxisTickerText::getSubTickCount (
    double tickStep ) [protected], [virtual]
```

Returns the sub tick count that was configured with [setSubTickCount](#).

\seebaseclassmethod

Reimplemented from [QCPAxisTicker](#).

15.563.3.7 getTickLabel()

```
QString QCPAxisTickerText::getTickLabel (
    double tick,
    const QLocale & locale,
    QChar formatChar,
    int precision ) [protected], [virtual]
```

Returns the tick label which corresponds to the key *tick* in the internal tick storage. Since the labels are provided externally, *locale*, *formatChar*, and *precision* are ignored.

\seebaseclassmethod

Reimplemented from [QCPAxisTicker](#).

15.563.3.8 `getTickStep()`

```
double QCPAxisTickerText::getTickStep (
    const QCPRange & range ) [protected], [virtual]
```

Since the tick coordinates are provided externally, this method implementation does nothing.

\see `baseclassmethod`

Reimplemented from [QCPAxisTicker](#).

15.563.3.9 `setSubTickCount()`

```
void QCPAxisTickerText::setSubTickCount (
    int subTicks )
```

Sets the number of sub ticks that shall appear between ticks. For [QCPAxisTickerText](#), there is no automatic sub tick count calculation. So if sub ticks are needed, they must be configured with this method.

15.563.3.10 `setTicks()` [1/2]

```
void QCPAxisTickerText::setTicks (
    const QMap< double, QString > & ticks )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the ticks that shall appear on the axis. The map key of *ticks* corresponds to the axis coordinate, and the map value is the string that will appear as tick label.

An alternative to manipulate ticks is to directly access the internal storage with the [ticks](#) getter.

See also

[addTicks](#), [addTick](#), [clear](#)

15.563.3.11 `setTicks()` [2/2]

```
void QCPAxisTickerText::setTicks (
    const QVector< double > & positions,
    const QVector< QString > & labels )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the ticks that shall appear on the axis. The entries of *positions* correspond to the axis coordinates, and the entries of *labels* are the respective strings that will appear as tick labels.

See also

[addTicks](#), [addTick](#), [clear](#)

15.563.3.12 `ticks()`

```
QMap< double, QString > & QCPAxisTickerText::ticks ( ) [inline]
```

Returns a non-const reference to the internal map which stores the tick coordinates and their labels.

You can access the map directly in order to add, remove or manipulate ticks, as an alternative to using the methods provided by [QCPAxisTickerText](#), such as [setTicks](#) and [addTick](#).

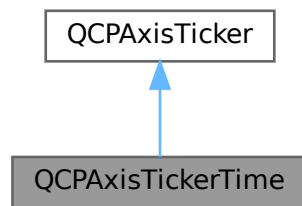
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

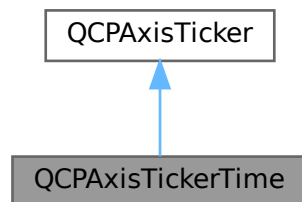
15.564 QCPAxisTickerTime Class Reference

Specialized axis ticker for time spans in units of milliseconds to days.

Inheritance diagram for QCPAxisTickerTime:



Collaboration diagram for QCPAxisTickerTime:



Public Types

- enum [TimeUnit](#) {
[tuMilliseconds](#) , [tuSeconds](#) , [tuMinutes](#) , [tuHours](#) ,
[tuDays](#) }

Public Types inherited from [QCPAxisTicker](#)

- enum [TickStepStrategy](#) { [tssReadability](#) , [tssMeetTickCount](#) }

Public Member Functions

- [QCPAxisTickerTime](#) ()
- QString [timeFormat](#) () const
- int [fieldWidth](#) ([TimeUnit](#) unit) const
- void [setTimeFormat](#) (const QString &format)
- void [setFieldWidth](#) ([TimeUnit](#) unit, int width)

Public Member Functions inherited from [QCPAxisTicker](#)

- [QCPAxisTicker](#) ()
- [TickStepStrategy](#) [tickStepStrategy](#) () const
- int [tickCount](#) () const
- double [tickOrigin](#) () const

- void [setTickStepStrategy](#) ([TickStepStrategy](#) strategy)
- void [setTickCount](#) (int count)
- void [setTickOrigin](#) (double origin)
- virtual void [generate](#) (const [QCPRange](#) &range, const [QLocale](#) &locale, [QChar](#) formatChar, int precision, [QVector](#)< double > &ticks, [QVector](#)< double > *subTicks, [QVector](#)< [QString](#) > *tickLabels)

Protected Member Functions

- virtual double [getTickStep](#) (const [QCPRange](#) &range) [Q_DECL_OVERRIDE](#)
- virtual int [getSubTickCount](#) (double tickStep) [Q_DECL_OVERRIDE](#)
- virtual [QString](#) [getTickLabel](#) (double tick, const [QLocale](#) &locale, [QChar](#) formatChar, int precision) [Q_DECL_OVERRIDE](#)
- void [replaceUnit](#) ([QString](#) &text, [TimeUnit](#) unit, int value) const

Protected Member Functions inherited from [QCPAxisTicker](#)

- virtual [QVector](#)< double > [createTickVector](#) (double tickStep, const [QCPRange](#) &range)
- virtual [QVector](#)< double > [createSubTickVector](#) (int subTickCount, const [QVector](#)< double > &ticks)
- virtual [QVector](#)< [QString](#) > [createLabelVector](#) (const [QVector](#)< double > &ticks, const [QLocale](#) &locale, [QChar](#) formatChar, int precision)
- void [trimTicks](#) (const [QCPRange](#) &range, [QVector](#)< double > &ticks, bool keepOneOutlier) const
- double [pickClosest](#) (double target, const [QVector](#)< double > &candidates) const
- double [getMantissa](#) (double input, double *magnitude=nullptr) const
- double [cleanMantissa](#) (double input) const

Protected Attributes

- [QString](#) [mTimeFormat](#)
- [QHash](#)< [TimeUnit](#), int > [mFieldWidth](#)
- [TimeUnit](#) [mSmallestUnit](#)
- [TimeUnit](#) [mBiggestUnit](#)
- [QHash](#)< [TimeUnit](#), [QString](#) > [mFormatPattern](#)

Protected Attributes inherited from [QCPAxisTicker](#)

- [TickStepStrategy](#) [mTickStepStrategy](#)
- int [mTickCount](#)
- double [mTickOrigin](#)

15.564.1 Detailed Description

Specialized axis ticker for time spans in units of milliseconds to days.

This [QCPAxisTicker](#) subclass generates ticks that corresponds to time intervals.

The format of the time display in the tick labels is controlled with [setTimeFormat](#) and [setFieldWidth](#). The time coordinate is in the unit of seconds with respect to the time coordinate zero. Unlike with [QCPAxisTickerDateTime](#), the ticks don't correspond to a specific calendar date and time.

The time can be displayed in milliseconds, seconds, minutes, hours and days. Depending on the largest available unit in the format specified with [setTimeFormat](#), any time spans above will be carried in that largest unit. So for example if the format string is "%m:%s" and a tick at coordinate value 7815 (being 2 hours, 10 minutes and 15 seconds) is created, the resulting tick label will show "130:15" (130 minutes, 15 seconds). If the format string is "%h:%m:%s", the hour unit will be used and the label will thus be "02:10:15". Negative times with respect to the axis zero will carry a leading minus sign.

The ticker can be created and assigned to an axis like this:

Here is an example of a time axis providing time information in days, hours and minutes. Due to the axis range spanning a few days and the wanted tick count ([setTickCount](#)), the ticker decided to use tick steps of 12 hours:

The format string for this example is

Note

If you rather wish to display calendar dates and times, have a look at [QCPAxisTickerDateTime](#) instead.

15.564.2 Member Enumeration Documentation**15.564.2.1 TimeUnit**

enum [QCPAxisTickerTime::TimeUnit](#)

Defines the logical units in which fractions of time spans can be expressed.

See also

[setFieldWidth](#), [setTimeFormat](#)

Enumerator

<code>tuMilliseconds</code>	Milliseconds, one thousandth of a second (%z in setTimeFormat)
<code>tuSeconds</code>	Seconds (%s in setTimeFormat)
<code>tuMinutes</code>	Minutes (%m in setTimeFormat)
<code>tuHours</code>	Hours (%h in setTimeFormat)
<code>tuDays</code>	Days (%d in setTimeFormat)

15.564.3 Constructor & Destructor Documentation**15.564.3.1 QCPAxisTickerTime()**

`QCPAxisTickerTime::QCPAxisTickerTime ( )`

Constructs the ticker and sets reasonable default values. Axis tickers are commonly created managed by a `QSharedPointer`, which then can be passed to [QCPAxis::setTicker](#).

15.564.4 Member Function Documentation**15.564.4.1 getSubTickCount()**

```
int QCPAxisTickerTime::getSubTickCount (
    double tickStep ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

15.564.4.2 getTickLabel()

```
QString QCPAxisTickerTime::getTickLabel (
    double tick,
    const QLocale & locale,
    QChar formatChar,
    int precision ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

15.564.4.3 getTickStep()

```
double QCPAxisTickerTime::getTickStep (
    const QCPRange & range ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

15.564.4.4 setFieldWidth()

```
void QCPAxisTickerTime::setFieldWidth (
    QCPAxisTickerTime::TimeUnit unit,
    int width )
```

Sets the field width of the specified *unit* to be *width* digits, when displayed in the tick label. If the number for the specific unit is shorter than *width*, it will be padded with an according number of zeros to the left in order to reach the field width.

See also

[setTimeFormat](#)

15.564.4.5 setTimeFormat()

```
void QCPAxisTickerTime::setTimeFormat (
    const QString & format )
```

Sets the format that will be used to display time in the tick labels.

The available patterns are:

- %z for milliseconds
- %s for seconds
- %m for minutes
- %h for hours
- %d for days

The field width (zero padding) can be controlled for each unit with [setFieldWidth](#).

The largest unit that appears in *format* will carry all the remaining time of a certain tick coordinate, even if it overflows the natural limit of the unit. For example, if %m is the largest unit it might become larger than 59 in order to consume larger time values. If on the other hand %h is available, the minutes will wrap around to zero after 59 and the time will carry to the hour digit.

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.565 QCPBars Class Reference

A plottable representing a bar chart in a plot.

Public Member Functions

- [QCPBars](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- double **width** () const
- [WidthType](#) **widthType** () const
- [QCPBarsGroup](#) * **barsGroup** () const
- double **baseValue** () const
- double **stackingGap** () const
- [QCPBars](#) * **barBelow** () const
- [QCPBars](#) * **barAbove** () const
- [QSharedPointer](#)< [QCPBarsDataContainer](#) > **data** () const
- void **setData** ([QSharedPointer](#)< [QCPBarsDataContainer](#) > data)
- void **setData** (const [QVector](#)< double > &keys, const [QVector](#)< double > &values, bool alreadySorted=false)
- void **setWidth** (double width)
- void **setWidthType** ([WidthType](#) widthType)
- void **setBarsGroup** ([QCPBarsGroup](#) *barsGroup)
- void **setBaseValue** (double baseValue)
- void **setStackingGap** (double pixels)
- void **addData** (const [QVector](#)< double > &keys, const [QVector](#)< double > &values, bool alreadySorted=false)
- void **addData** (double key, double value)
- void **moveBelow** ([QCPBars](#) *bars)
- void **moveAbove** ([QCPBars](#) *bars)
- virtual [QCPDataSelection](#) **selectTestRect** (const [QRectF](#) &rect, bool onlySelectable) const [Q_DECL_OVERRIDE](#)
- virtual double **selectTest** (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const [Q_DECL_OVERRIDE](#)
- virtual [QCPRange](#) **getKeyRange** (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#)) const [Q_DECL_OVERRIDE](#)
- virtual [QCPRange](#) **getValueRange** (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#), const [QCPRange](#) &inKeyRange=[QCPRange](#)()) const [Q_DECL_OVERRIDE](#)
- virtual [QPointF](#) **dataPixelPosition** (int index) const [Q_DECL_OVERRIDE](#)

Public Member Functions inherited from [QCPAbstractPlottable1D](#)< [QCPBarsData](#) >

- [QCPAbstractPlottable1D](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- virtual int **dataCount** () const [Q_DECL_OVERRIDE](#)
- virtual double **dataMainKey** (int index) const [Q_DECL_OVERRIDE](#)
- virtual double **dataSortKey** (int index) const [Q_DECL_OVERRIDE](#)
- virtual double **dataMainValue** (int index) const [Q_DECL_OVERRIDE](#)
- virtual [QCPRange](#) **dataValueRange** (int index) const [Q_DECL_OVERRIDE](#)
- virtual bool **sortKeysMainKey** () const [Q_DECL_OVERRIDE](#)
- virtual int **findBegin** (double sortKey, bool expandedRange=true) const [Q_DECL_OVERRIDE](#)
- virtual int **findEnd** (double sortKey, bool expandedRange=true) const [Q_DECL_OVERRIDE](#)
- virtual [QCPPlottableInterface1D](#) * **interface1D** () [Q_DECL_OVERRIDE](#)

Public Member Functions inherited from [QCPAbstractPlottable](#)

- [QCPAbstractPlottable](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- [QString](#) **name** () const
- bool **antialiasedFill** () const
- bool **antialiasedScatters** () const
- [QPen](#) **pen** () const
- [QBrush](#) **brush** () const
- [QCPAxis](#) * **keyAxis** () const
- [QCPAxis](#) * **valueAxis** () const

- [QCP::SelectionType](#) **selectable** () const
- bool **selected** () const
- [QCPDataSelection](#) **selection** () const
- [QCPSelectionDecorator](#) * **selectionDecorator** () const
- void **setName** (const QString &name)
- void **setAntialiasedFill** (bool enabled)
- void **setAntialiasedScatters** (bool enabled)
- void **setPen** (const QPen &pen)
- void **setBrush** (const QBrush &brush)
- void **setKeyAxis** ([QCPAxis](#) *axis)
- void **setValueAxis** ([QCPAxis](#) *axis)
- Q_SLOT void **setSelectable** ([QCP::SelectionType](#) selectable)
- Q_SLOT void **setSelection** ([QCPDataSelection](#) selection)
- void **setSelectionDecorator** ([QCPSelectionDecorator](#) *decorator)
- void **coordsToPixels** (double key, double value, double &x, double &y) const
- const QPointF **coordsToPixels** (double key, double value) const
- void **pixelsToCoords** (double x, double y, double &key, double &value) const
- void **pixelsToCoords** (const QPointF &pixelPos, double &key, double &value) const
- void **rescaleAxes** (bool onlyEnlarge=false) const
- void **rescaleKeyAxis** (bool onlyEnlarge=false) const
- void **rescaleValueAxis** (bool onlyEnlarge=false, bool inKeyRange=false) const
- bool **addToLegend** ([QCPLegend](#) *legend)
- bool **addToLegend** ()
- bool **removeFromLegend** ([QCPLegend](#) *legend) const
- bool **removeFromLegend** () const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const QString &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Protected Member Functions

- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void **drawLegendIcon** ([QCPPainter](#) *painter, const QRectF &rect) const Q_DECL_OVERRIDE
- void **getVisibleDataBounds** ([QCPBarsDataContainer::const_iterator](#) &begin, [QCPBarsDataContainer::const_iterator](#) &end) const
- QRectF **getBarRect** (double key, double value) const
- void **getPixelWidth** (double key, double &lower, double &upper) const
- double **getStackedBaseValue** (double key, bool positive) const

Protected Member Functions inherited from [QCPAbstractPlottable1D< QCPBarsData >](#)

- void **getDataSegments** (QList< [QCPDataRange](#) > &selectedSegments, QList< [QCPDataRange](#) > &unselectedSegments) const
- void **drawPolyline** ([QCPPainter](#) *painter, const QVector< QPointF > &lineData) const

Protected Member Functions inherited from [QCPAbstractPlottable](#)

- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- void [applyFillAntialiasingHint](#) ([QCPPainter](#) *painter) const
- void [applyScattersAntialiasingHint](#) ([QCPPainter](#) *painter) const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot)
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void [initializeParentPlot](#) ([QCustomPlot](#) *parentPlot)
- void [setParentLayerable](#) ([QCPLayerable](#) *parentLayerable)
- bool [moveToLayer](#) ([QCPLayer](#) *layer, bool prepend)
- void [applyAntialiasingHint](#) ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Static Protected Member Functions

- static void [connectBars](#) ([QCPBars](#) *lower, [QCPBars](#) *upper)

Protected Attributes

- double **mWidth**
- [WidthType](#) **mWidthType**
- [QCPBarsGroup](#) * **mBarsGroup**
- double **mBaseValue**
- double **mStackingGap**
- QPointer< [QCPBars](#) > **mBarBelow**
- QPointer< [QCPBars](#) > **mBarAbove**

Protected Attributes inherited from [QCPAbstractPlottable1D< QCPBarsData >](#)

- QSharedPointer< [QCPDataContainer](#)< [QCPBarsData](#) > > **mDataContainer**

Protected Attributes inherited from [QCPAbstractPlottable](#)

- QString **mName**
- bool **mAntialiasedFill**
- bool **mAntialiasedScatters**
- QPen **mPen**
- QBrush **mBrush**
- QPointer< [QCPAxis](#) > **mKeyAxis**
- QPointer< [QCPAxis](#) > **mValueAxis**
- [QCP::SelectionType](#) **mSelectable**
- [QCPDataSelection](#) **mSelection**
- [QCPSelectionDecorator](#) * **mSelectionDecorator**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- [QPointer](#)< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCustomPlot](#)
- class [QCPLegend](#)
- class [QCPBarsGroup](#)

Additional Inherited Members

Signals inherited from [QCPAbstractPlottable](#)

- void [selectionChanged](#) (bool [selected](#))
- void [selectionChanged](#) (const [QCPDataSelection](#) &[selection](#))
- void [selectableChanged](#) ([QCP::SelectionType](#) [selectable](#))

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *[newLayer](#))

15.565.1 Detailed Description

A plottable representing a bar chart in a plot.

To plot data, assign it with the [setData](#) or [addData](#) functions.

15.565.2 Changing the appearance

The appearance of the bars is determined by the pen and the brush ([setPen](#), [setBrush](#)). The width of the individual bars can be controlled with [setWidthType](#) and [setWidth](#).

Bar charts are stackable. This means, two [QCPBars](#) plottables can be placed on top of each other (see [QCPBars::moveAbove](#)). So when two bars are at the same key position, they will appear stacked.

If you would like to group multiple [QCPBars](#) plottables together so they appear side by side as shown below, use [QCPBarsGroup](#).

15.565.3 Usage

Like all data representing objects in [QCustomPlot](#), the [QCPBars](#) is a plottable ([QCPAbstractPlottable](#)). So the plottable-interface of [QCustomPlot](#) applies ([QCustomPlot::plottable](#), [QCustomPlot::removePlottable](#), etc.)

Usually, you first create an instance:

which registers it with the [QCustomPlot](#) instance of the passed axes. Note that this [QCustomPlot](#) instance takes ownership of the plottable, so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead. The newly created plottable can be modified, e.g.:

15.565.4 Member Enumeration Documentation

15.565.4.1 WidthType

```
enum QCPBars::WidthType
```

Defines the ways the width of the bar can be specified. Thus it defines what the number passed to [setWidth](#) actually means.

See also

[setWidthType](#), [setWidth](#)

Enumerator

wtAbsolute	Bar width is in absolute pixels.
wtAxisRectRatio	Bar width is given by a fraction of the axis rect size.
wtPlotCoords	Bar width is in key coordinates and thus scales with the key axis range.

15.565.5 Constructor & Destructor Documentation

15.565.5.1 QCPBars()

```
QCPBars::QCPBars (
    QCPAxis * keyAxis,
    QCPAxis * valueAxis ) [explicit]
```

Constructs a bar chart which uses *keyAxis* as its key axis ("x") and *valueAxis* as its value axis ("y"). *keyAxis* and *valueAxis* must reside in the same [QCustomPlot](#) instance and not have the same orientation. If either of these restrictions is violated, a corresponding message is printed to the debug output (qDebug), the construction is not aborted, though.

The created [QCPBars](#) is automatically registered with the [QCustomPlot](#) instance inferred from *keyAxis*. This [QCustomPlot](#) instance takes ownership of the [QCPBars](#), so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead.

15.565.6 Member Function Documentation

15.565.6.1 addData() [1/2]

```
void QCPBars::addData (
    const QVector< double > & keys,
    const QVector< double > & values,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided points in *keys* and *values* to the current data. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *keys* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

15.565.6.2 addData() [2/2]

```
void QCPBars::addData (
    double key,
    double value )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts. Adds the provided data point as *key* and *value* to the current data.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

15.565.6.3 barAbove()

```
QCPBars * QCPBars::barAbove ( ) const [inline]
```

Returns the bars plottable that is directly above this bars plottable. If there is no such plottable, returns `nullptr`.

See also

[barBelow](#), [moveBelow](#), [moveAbove](#)

15.565.6.4 barBelow()

```
QCPBars * QCPBars::barBelow ( ) const [inline]
```

Returns the bars plottable that is directly below this bars plottable. If there is no such plottable, returns `nullptr`.

See also

[barAbove](#), [moveBelow](#), [moveAbove](#)

15.565.6.5 data()

```
QSharedPointer< QCPBarsDataContainer > QCPBars::data ( ) const [inline]
```

Returns a shared pointer to the internal data storage of type `QCPBarsDataContainer`. You may use it to directly manipulate the data, which may be more convenient and faster than using the regular [setData](#) or [addData](#) methods.

15.565.6.6 dataPixelPosition()

```
QPointF QCPBars::dataPixelPosition (
    int index ) const [virtual]
```

Returns the pixel position on the widget surface at which the data point at the given *index* appears.

Usually this corresponds to the point of `dataMainKey/ dataMainValue`, in pixel coordinates. However, depending on the plottable, this might be a different apparent position than just a coord-to-pixel transform of those values. For example, [QCPBars](#) apparent data values can be shifted depending on their stacking, bar grouping or configured base value.

Reimplemented from [QCPAbstractPlottable1D< QCPBarsData >](#).

15.565.6.7 draw()

```
void QCPBars::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

15.565.6.8 drawLegendIcon()

```
void QCPBars::drawLegendIcon (
    QCPPainter * painter,
    const QRectF & rect ) const [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

15.565.6.9 getKeyRange()

```
QCPRange QCPBars::getKeyRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth ) const [virtual]
```

Returns the coordinate range that all data in this plottable span in the key axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getValueRange](#)

Implements [QCPAbstractPlottable](#).

15.565.6.10 getValueRange()

```
QCPRange QCPBars::getValueRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth,
    const QCPRange & inKeyRange = QCPRange() ) const [virtual]
```

Returns the coordinate range that the data points in the specified key range (*inKeyRange*) span in the value axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

If *inKeyRange* has both lower and upper bound set to zero (is equal to [QCPRange\(\)](#)), all data points are considered, without any restriction on the keys.

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getKeyRange](#)

Implements [QCPAbstractPlottable](#).

15.565.6.11 moveAbove()

```
void QCPBars::moveAbove (
    QCPBars * bars )
```

Moves this bars plottable above *bars*. In other words, the bars of this plottable will appear above the bars of *bars*. The move target *bars* must use the same key and value axis as this plottable.

Inserting into and removing from existing bar stacking is handled gracefully. If *bars* already has a bars object above itself, this bars object is inserted between the two. If this bars object is already between two other bars, the two other bars will be stacked on top of each other after the operation.

To remove this bars plottable from any stacking, set *bars* to `nullptr`.

See also

[moveBelow](#), [barBelow](#), [barAbove](#)

15.565.6.12 moveBelow()

```
void QCPBars::moveBelow (
    QCPBars * bars )
```

Moves this bars plottable below *bars*. In other words, the bars of this plottable will appear below the bars of *bars*. The move target *bars* must use the same key and value axis as this plottable.

Inserting into and removing from existing bar stacking is handled gracefully. If *bars* already has a bars object below itself, this bars object is inserted between the two. If this bars object is already between two other bars, the two other bars will be stacked on top of each other after the operation.

To remove this bars plottable from any stacking, set *bars* to `nullptr`.

See also

[moveBelow](#), [barAbove](#), [barBelow](#)

15.565.6.13 selectTest()

```
double QCPBars::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Implements a `selectTest` specific to this plottable's point geometry.

If *details* is not 0, it will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

\see [baseclassmethod QCPAbstractPlottable::selectTest](#)

Reimplemented from [QCPAbstractPlottable1D< QCPBarsData >](#).

15.565.6.14 selectTestRect()

```
QCPDataSelection QCPBars::selectTestRect (
    const QRectF & rect,
    bool onlySelectable ) const [virtual]
```

Returns a data selection containing all the data points of this plottable which are contained (or hit by) *rect*. This is used mainly in the selection rect interaction for data selection (data selection mechanism).

If *onlySelectable* is true, an empty [QCPDataSelection](#) is returned if this plottable is not selectable (i.e. if [QCPAbstractPlottable::setSelectable](#) is [QCP::stNone](#)).

Note

rect must be a normalized rect (positive or zero width and height). This is especially important when using the rect of [QCPSelectionRect::accepted](#), which is not necessarily normalized. Use `QRect::normalized()` when passing a rect which might not be normalized.

Reimplemented from [QCPAbstractPlottable1D< QCPBarsData >](#).

15.565.6.15 setBarsGroup()

```
void QCPBars::setBarsGroup (
    QCPBarsGroup * barsGroup )
```

Sets to which [QCPBarsGroup](#) this [QCPBars](#) instance belongs to. Alternatively, you can also use [QCPBarsGroup::append](#).

To remove this [QCPBars](#) from any group, set *barsGroup* to `nullptr`.

15.565.6.16 setBaseValue()

```
void QCPBars::setBaseValue (
    double baseValue )
```

Sets the base value of this bars plottable.

The base value defines where on the value coordinate the bars start. How far the bars extend from the base value is given by their individual value data. For example, if the base value is set to 1, a bar with data value 2 will have its lowest point at value coordinate 1 and highest point at 3.

For stacked bars, only the base value of the bottom-most [QCPBars](#) has meaning.

The default base value is 0.

15.565.6.17 setData() [1/2]

```
void QCPBars::setData (
    const QVector< double > & keys,
    const QVector< double > & values,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data with the provided points in *keys* and *values*. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *keys* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

See also

[addData](#)

15.565.6.18 setData() [2/2]

```
void QCPBars::setData (
    QSharedPointer< QCPBarsDataContainer > data )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data container with the provided *data* container.

Since a QSharedPointer is used, multiple [QCPBars](#) may share the same data container safely. Modifying the data in the container will then affect all bars that share the container. Sharing can be achieved by simply exchanging the data containers wrapped in shared pointers:

If you do not wish to share containers, but create a copy from an existing container, rather use the [QCPDataContainer<DataType>::set](#) method on the bar's data container directly:

See also

[addData](#)

15.565.6.19 setStackingGap()

```
void QCPBars::setStackingGap (
    double pixels )
```

If this bars plottable is stacked on top of another bars plottable ([moveAbove](#)), this method allows specifying a distance in *pixels*, by which the drawn bar rectangles will be separated by the bars below it.

15.565.6.20 setWidth()

```
void QCPBars::setWidth (
    double width )
```

Sets the width of the bars.

How the number passed as *width* is interpreted (e.g. screen pixels, plot coordinates,...), depends on the currently set width type, see [setWidthType](#) and [WidthType](#).

15.565.6.21 setWidthType()

```
void QCPBars::setWidthType (
    QCPBars::WidthType widthType )
```

Sets how the width of the bars is defined. See the documentation of [WidthType](#) for an explanation of the possible values for *widthType*.

The default value is [wtPlotCoords](#).

See also

[setWidth](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.566 QCPBarsData Class Reference

Holds the data of one single data point (one bar) for [QCPBars](#).

Public Member Functions

- [QCPBarsData](#) ()
- [QCPBarsData](#) (double key, double value)
- double [sortKey](#) () const
- double [mainKey](#) () const
- double [mainValue](#) () const
- [QCPRange](#) [valueRange](#) () const

Static Public Member Functions

- static [QCPBarsData fromSortKey](#) (double *sortKey*)
- static bool [sortKeysIsMainKey](#) ()

Public Attributes

- double **key**
- double **value**

15.566.1 Detailed Description

Holds the data of one single data point (one bar) for [QCPBars](#).

The stored data is:

- *key*: coordinate on the key axis of this bar (this is the *mainKey* and the *sortKey*)
- *value*: height coordinate on the value axis of this bar (this is the *mainValue*)

The container for storing multiple data points is [QCPBarsDataContainer](#). It is a typedef for [QCPDataContainer](#) with [QCPBarsData](#) as the `DataType` template parameter. See the documentation there for an explanation regarding the data type's generic methods.

See also

[QCPBarsDataContainer](#)

15.566.2 Constructor & Destructor Documentation

15.566.2.1 QCPBarsData() [1/2]

```
QCPBarsData::QCPBarsData ( )
```

Constructs a bar data point with key and value set to zero.

15.566.2.2 QCPBarsData() [2/2]

```
QCPBarsData::QCPBarsData (
    double key,
    double value )
```

Constructs a bar data point with the specified *key* and *value*.

15.566.3 Member Function Documentation

15.566.3.1 fromSortKey()

```
static QCPBarsData QCPBarsData::fromSortKey (
    double sortKey ) [inline], [static]
```

Returns a data point with the specified *sortKey*. All other members are set to zero.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

15.566.3.2 mainKey()

```
double QCPBarsData::mainKey ( ) const [inline]
```

Returns the *key* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

15.566.3.3 mainValue()

```
double QCPBarsData::mainValue ( ) const [inline]
```

Returns the *value* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

15.566.3.4 sortKey()

```
double QCPBarsData::sortKey ( ) const [inline]
```

Returns the *key* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

15.566.3.5 sortKeysMainKey()

```
static static bool QCPBarsData::sortKeyIsMainKey ( ) [inline], [static]
```

Since the member *key* is both the data point key coordinate and the data ordering parameter, this method returns true.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

15.566.3.6 valueRange()

```
QCPRange QCPBarsData::valueRange ( ) const [inline]
```

Returns a [QCPRange](#) with both lower and upper boundary set to *value* of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

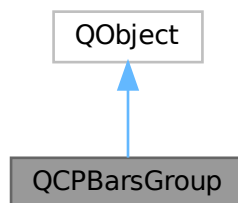
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

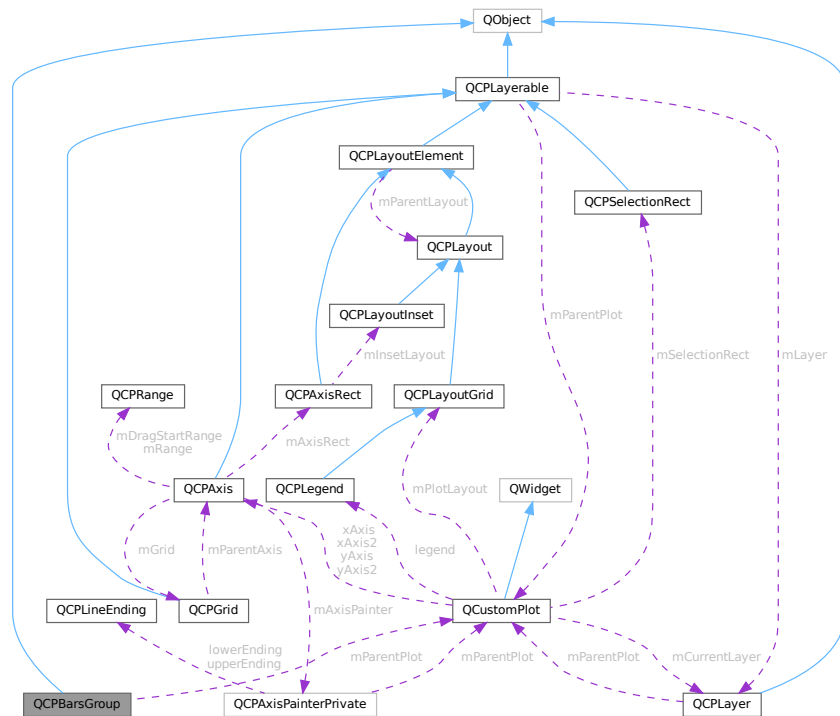
15.567 QCPBarsGroup Class Reference

Groups multiple [QCPBars](#) together so they appear side by side.

Inheritance diagram for QCPBarsGroup:



Collaboration diagram for QCPBarsGroup:



Public Types

- enum [SpacingType](#) { [stAbsolute](#) , [stAxisRectRatio](#) , [stPlotCoords](#) }

Public Member Functions

- [QCPBarsGroup](#) ([QCustomPlot](#) *parentPlot)
- [SpacingType](#) [spacingType](#) () const
- double [spacing](#) () const
- void [setSpacingType](#) ([SpacingType](#) spacingType)
- void [setSpacing](#) (double spacing)
- QList< [QCPBars](#) * > [bars](#) () const
- [QCPBars](#) * [bars](#) (int index) const
- int [size](#) () const
- bool [isEmpty](#) () const
- void [clear](#) ()
- bool [contains](#) ([QCPBars](#) *bars) const
- void [append](#) ([QCPBars](#) *bars)
- void [insert](#) (int i, [QCPBars](#) *bars)
- void [remove](#) ([QCPBars](#) *bars)

Protected Member Functions

- void [registerBars](#) ([QCPBars](#) *bars)
- void [unregisterBars](#) ([QCPBars](#) *bars)
- double [keyPixelOffset](#) (const [QCPBars](#) *bars, double keyCoord)
- double [getPixelSpacing](#) (const [QCPBars](#) *bars, double keyCoord)

Protected Attributes

- [QCustomPlot](#) * `mParentPlot`
- [SpacingType](#) `mSpacingType`
- double `mSpacing`
- `QList< QCPBars * >` `mBars`

Friends

- class `QCPBars`

15.567.1 Detailed Description

Groups multiple [QCPBars](#) together so they appear side by side.

When showing multiple [QCPBars](#) in one plot which have bars at identical keys, it may be desirable to have them appearing next to each other at each key. This is what adding the respective [QCPBars](#) plottables to a [QCPBarsGroup](#) achieves. (An alternative approach is to stack them on top of each other, see [QCPBars::moveAbove](#).)

15.567.2 Usage

To add a [QCPBars](#) plottable to the group, create a new group and then add the respective bars instances:

Alternatively to appending to the group like shown above, you can also set the group on the [QCPBars](#) plottable via [QCPBars::setBarsGroup](#).

The spacing between the bars can be configured via [setSpacingType](#) and [setSpacing](#). The bars in this group appear in the plot in the order they were appended. To insert a bars plottable at a certain index position, or to reposition a bars plottable which is already in the group, use [insert](#).

To remove specific bars from the group, use either [remove](#) or call [QCPBars::setBarsGroup\(0\)](#) on the respective bars plottable.

To clear the entire group, call [clear](#), or simply delete the group.

15.567.3 Example

The image above is generated with the following code:

15.567.4 Member Enumeration Documentation**15.567.4.1 SpacingType**

enum `QCPBarsGroup::SpacingType`

Defines the ways the spacing between bars in the group can be specified. Thus it defines what the number passed to [setSpacing](#) actually means.

See also

[setSpacingType](#), [setSpacing](#)

Enumerator

<code>stAbsolute</code>	Bar spacing is in absolute pixels.
<code>stAxisRectRatio</code>	Bar spacing is given by a fraction of the axis rect size.
<code>stPlotCoords</code>	Bar spacing is in key coordinates and thus scales with the key axis range.

15.567.5 Constructor & Destructor Documentation**15.567.5.1 QCPBarsGroup()**

```
QCPBarsGroup::QCPBarsGroup (
    QCustomPlot * parentPlot ) [explicit]
```

Constructs a new bars group for the specified [QCustomPlot](#) instance.

15.567.6 Member Function Documentation

15.567.6.1 `append()`

```
void QCPBarsGroup::append (
    QCPBars * bars )
```

Adds the specified *bars* plottable to this group. Alternatively, you can also use [QCPBars::setBarsGroup](#) on the *bars* instance.

See also

[insert](#), [remove](#)

15.567.6.2 `bars()` [1/2]

```
QList< QCPBars * > QCPBarsGroup::bars ( ) const [inline]
```

Returns all bars currently in this group.

See also

`bars(int index)`

15.567.6.3 `bars()` [2/2]

```
QCPBars * QCPBarsGroup::bars (
    int index ) const
```

Returns the [QCPBars](#) instance with the specified *index* in this group. If no such [QCPBars](#) exists, returns `nullptr`.

See also

`bars()`, `size`

15.567.6.4 `clear()`

```
void QCPBarsGroup::clear ( )
```

Removes all [QCPBars](#) plottables from this group.

See also

`isEmpty`

15.567.6.5 `contains()`

```
bool QCPBarsGroup::contains (
    QCPBars * bars ) const [inline]
```

Returns whether the specified *bars* plottable is part of this group.

15.567.6.6 `insert()`

```
void QCPBarsGroup::insert (
    int i,
    QCPBars * bars )
```

Inserts the specified *bars* plottable into this group at the specified index position *i*. This gives you full control over the ordering of the bars.

bars may already be part of this group. In that case, *bars* is just moved to the new index position.

See also

[append](#), [remove](#)

15.567.6.7 isEmpty()

```
bool QCPBarsGroup::isEmpty ( ) const [inline]
```

Returns whether this bars group is empty.

See also

[size](#)

15.567.6.8 remove()

```
void QCPBarsGroup::remove (
    QCPBars * bars )
```

Removes the specified *bars* plottable from this group.

See also

[contains](#), [clear](#)

15.567.6.9 setSpacing()

```
void QCPBarsGroup::setSpacing (
    double spacing )
```

Sets the spacing between adjacent bars. What the number passed as *spacing* actually means, is defined by the current [SpacingType](#), which can be set with [setSpacingType](#).

See also

[setSpacingType](#)

15.567.6.10 setSpacingType()

```
void QCPBarsGroup::setSpacingType (
    SpacingType spacingType )
```

Sets how the spacing between adjacent bars is interpreted. See [SpacingType](#). The actual spacing can then be specified with [setSpacing](#).

See also

[setSpacing](#)

15.567.6.11 size()

```
int QCPBarsGroup::size ( ) const [inline]
```

Returns the number of [QCPBars](#) plottables that are part of this group.

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.568 QCPCOLORGRADIENT Class Reference

Defines a color gradient for use with e.g. [QCPCOLORMAP](#).

Public Types

- enum [ColorInterpolation](#) { [ciRGB](#) , [ciHSV](#) }
- enum [NanHandling](#) {
[nhNone](#) , [nhLowestColor](#) , [nhHighestColor](#) , [nhTransparent](#) ,
[nhNaNColor](#) }
- enum [GradientPreset](#) {
[gpGrayscale](#) , [gpHot](#) , [gpCold](#) , [gpNight](#) ,
[gpCandy](#) , [gpGeography](#) , [gpIon](#) , [gpThermal](#) ,
[gpPolar](#) , [gpSpectrum](#) , [gpJet](#) , [gpHues](#) }

Public Member Functions

- [QCPColorGradient](#) ()
- [QCPColorGradient](#) ([GradientPreset](#) preset)
- bool **operator==** (const [QCPColorGradient](#) &other) const
- bool **operator!=** (const [QCPColorGradient](#) &other) const
- int **levelCount** () const
- QMap< double, QColor > **colorStops** () const
- [ColorInterpolation](#) **colorInterpolation** () const
- [NanHandling](#) **nanHandling** () const
- QColor **nanColor** () const
- bool **periodic** () const
- void **setLevelCount** (int n)
- void **setColorStops** (const QMap< double, QColor > &colorStops)
- void **setColorStopAt** (double position, const QColor &color)
- void **setColorInterpolation** ([ColorInterpolation](#) interpolation)
- void **setNanHandling** ([NanHandling](#) handling)
- void **setNanColor** (const QColor &color)
- void **setPeriodic** (bool enabled)
- void **colorize** (const double *data, const [QCPRange](#) &range, QRgb *scanLine, int n, int dataIndexFactor=1, bool logarithmic=false)
- void **colorize** (const double *data, const unsigned char *alpha, const [QCPRange](#) &range, QRgb *scanLine, int n, int dataIndexFactor=1, bool logarithmic=false)
- QRgb **color** (double position, const [QCPRange](#) &range, bool logarithmic=false)
- void **loadPreset** ([GradientPreset](#) preset)
- void **clearColorStops** ()
- [QCPColorGradient](#) **inverted** () const

Protected Member Functions

- bool **stopsUseAlpha** () const
- void **updateColorBuffer** ()

Protected Attributes

- int **mLevelCount**
- QMap< double, QColor > **mColorStops**
- [ColorInterpolation](#) **mColorInterpolation**
- [NanHandling](#) **mNanHandling**
- QColor **mNanColor**
- bool **mPeriodic**
- QVector< QRgb > **mColorBuffer**
- bool **mColorBufferInvalidated**

15.568.1 Detailed Description

Defines a color gradient for use with e.g. [QCPColorMap](#).

This class describes a color gradient which can be used to encode data with color. For example, [QCPColorMap](#) and [QCPColorScale](#) have [setGradient](#) methods which take an instance of this class. Colors are set with [setColorStopAt\(double position, const QColor &color\)](#) with a *position* from 0 to 1. In between these defined color positions, the color will be interpolated linearly either in RGB or HSV space, see [setColorInterpolation](#).

Alternatively, load one of the preset color gradients shown in the image below, with [loadPreset](#), or by directly specifying the preset in the constructor.

Apart from red, green and blue components, the gradient also interpolates the alpha values of the configured color stops. This allows to display some portions of the data range as transparent in the plot.

How NaN values are interpreted can be configured with [setNanHandling](#).

The constructor [QCPColorGradient\(GradientPreset preset\)](#) allows directly converting a [GradientPreset](#) to a [QCPColorGradient](#). This means that you can directly pass [GradientPreset](#) to all the *setGradient* methods, e.g.:

The total number of levels used in the gradient can be set with [setLevelCount](#). Whether the color gradient shall be applied periodically (wrapping around) to data values that lie outside the data range specified on the plottable instance can be controlled with [setPeriodic](#).

15.568.2 Member Enumeration Documentation

15.568.2.1 ColorInterpolation

enum [QCPColorGradient::ColorInterpolation](#)

Defines the color spaces in which color interpolation between gradient stops can be performed.

See also

[setColorInterpolation](#)

Enumerator

ciRGB	Color channels red, green and blue are linearly interpolated.
ciHSV	Color channels hue, saturation and value are linearly interpolated (The hue is interpolated over the shortest angle distance)

15.568.2.2 GradientPreset

enum [QCPColorGradient::GradientPreset](#)

Defines the available presets that can be loaded with [loadPreset](#). See the documentation there for an image of the presets.

Enumerator

gpGrayscale	Continuous lightness from black to white (suited for non-biased data representation)
gpHot	Continuous lightness from black over firey colors to white (suited for non-biased data representation)
gpCold	Continuous lightness from black over icy colors to white (suited for non-biased data representation)
gpNight	Continuous lightness from black over weak blueish colors to white (suited for non-biased data representation)
gpCandy	Blue over pink to white.
gpGeography	Colors suitable to represent different elevations on geographical maps.
gpIon	Half hue spectrum from black over purple to blue and finally green (creates banding illusion but allows more precise magnitude estimates)
gpThermal	Colors suitable for thermal imaging, ranging from dark blue over purple to orange, yellow and white.

Enumerator

gpPolar	Colors suitable to emphasize polarity around the center, with blue for negative, black in the middle and red for positive values.
gpSpectrum	An approximation of the visible light spectrum (creates banding illusion but allows more precise magnitude estimates)
gpJet	Hue variation similar to a spectrum, often used in numerical visualization (creates banding illusion but allows more precise magnitude estimates)
gpHues	Full hue cycle, with highest and lowest color red (suitable for periodic data, such as angles and phases, see setPeriodic)

15.568.2.3 NanHandling

enum [QCPColorGradient::NanHandling](#)

Defines how NaN data points shall appear in the plot.

See also

[setNanHandling](#), [setNanColor](#)

Enumerator

nhNone	NaN data points are not explicitly handled and shouldn't occur in the data (this gives slight performance improvement)
nhLowestColor	NaN data points appear as the lowest color defined in this QCPColorGradient .
nhHighestColor	NaN data points appear as the highest color defined in this QCPColorGradient .
nhTransparent	NaN data points appear transparent.
nhNanColor	NaN data points appear as the color defined with setNanColor .

15.568.3 Constructor & Destructor Documentation

15.568.3.1 QCPColorGradient() [1/2]

```
QCPColorGradient::QCPColorGradient ( )
```

Constructs a new, empty [QCPColorGradient](#) with no predefined color stops. You can add own color stops with [setColorStopAt](#).

The color level count is initialized to 350.

15.568.3.2 QCPColorGradient() [2/2]

```
QCPColorGradient::QCPColorGradient (
    GradientPreset preset )
```

Constructs a new [QCPColorGradient](#) initialized with the colors and color interpolation according to *preset*.

The color level count is initialized to 350.

15.568.4 Member Function Documentation

15.568.4.1 clearColorStops()

```
void QCPColorGradient::clearColorStops ( )
```

Clears all color stops.

See also

[setColorStops](#), [setColorStopAt](#)

15.568.4.2 `colorize()` [1/2]

```
void QCPColorGradient::colorize (
    const double * data,
    const QCPRange & range,
    QRgb * scanLine,
    int n,
    int dataIndexFactor = 1,
    bool logarithmic = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

This method is used to quickly convert a *data* array to colors. The colors will be output in the array *scanLine*. Both *data* and *scanLine* must have the length *n* when passed to this function. The data range that shall be used for mapping the data value to the gradient is passed in *range*. *logarithmic* indicates whether the data values shall be mapped to colors logarithmically.

if *data* actually contains 2D-data linearized via `[row*columnCount + column]`, you can set *dataIndexFactor* to `columnCount` to convert a column instead of a row of the data array, in *scanLine*. *scanLine* will remain a regular (1D) array. This works because *data* is addressed `data[i*dataIndexFactor]`.

Use the overloaded method to additionally provide alpha map data.

The QRgb values that are placed in *scanLine* have their r, g, and b components premultiplied with alpha (see `QImage::Format_ARGB32_Premultiplied`).

15.568.4.3 `colorize()` [2/2]

```
void QCPColorGradient::colorize (
    const double * data,
    const unsigned char * alpha,
    const QCPRange & range,
    QRgb * scanLine,
    int n,
    int dataIndexFactor = 1,
    bool logarithmic = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Additionally to the other overload of `colorize`, this method takes the array *alpha*, which has the same size and structure as *data* and encodes the alpha information per data point.

The QRgb values that are placed in *scanLine* have their r, g and b components premultiplied with alpha (see `QImage::Format_ARGB32_Premultiplied`).

15.568.4.4 `inverted()`

```
QCPColorGradient QCPColorGradient::inverted ( ) const
```

Returns an inverted gradient. The inverted gradient has all properties as this `QCPColorGradient`, but the order of the color stops is inverted.

See also

[setColorStops](#), [setColorStopAt](#)

15.568.4.5 `loadPreset()`

```
void QCPColorGradient::loadPreset (
    GradientPreset preset )
```

Clears the current color stops and loads the specified *preset*. A preset consists of predefined color stops and the corresponding color interpolation method.

The available presets are:

15.568.4.6 setColorInterpolation()

```
void QCPColorGradient::setColorInterpolation (
    QCPColorGradient::ColorInterpolation interpolation )
```

Sets whether the colors in between the configured color stops (see [setColorStopAt](#)) shall be interpolated linearly in RGB or in HSV color space.

For example, a sweep in RGB space from red to green will have a muddy brown intermediate color, whereas in HSV space the intermediate color is yellow.

15.568.4.7 setColorStopAt()

```
void QCPColorGradient::setColorStopAt (
    double position,
    const QColor & color )
```

Sets the *color* the gradient will have at the specified *position* (from 0 to 1). In between these color stops, the color is interpolated according to [setColorInterpolation](#).

See also

[setColorStops](#), [clearColorStops](#)

15.568.4.8 setColorStops()

```
void QCPColorGradient::setColorStops (
    const QMap< double, QColor > & colorStops )
```

Sets at which positions from 0 to 1 which color shall occur. The positions are the keys, the colors are the values of the passed QMap *colorStops*. In between these color stops, the color is interpolated according to [setColorInterpolation](#). A more convenient way to create a custom gradient may be to clear all color stops with [clearColorStops](#) (or creating a new, empty [QCPColorGradient](#)) and then adding them one by one with [setColorStopAt](#).

See also

[clearColorStops](#)

15.568.4.9 setLevelCount()

```
void QCPColorGradient::setLevelCount (
    int n )
```

Sets the number of discretization levels of the color gradient to *n*. The default is 350 which is typically enough to create a smooth appearance. The minimum number of levels is 2.

15.568.4.10 setNanColor()

```
void QCPColorGradient::setNanColor (
    const QColor & color )
```

Sets the color that NaN data is represented by, if [setNanHandling](#) is set to `ref nhNanColor`.

See also

[setNanHandling](#)

15.568.4.11 setNanHandling()

```
void QCPColorGradient::setNanHandling (
    QCPColorGradient::NanHandling handling )
```

Sets how NaNs in the data are displayed in the plot.

See also

[setNanColor](#)

15.568.4.12 setPeriodic()

```
void QCPColorGradient::setPeriodic (
    bool enabled )
```

Sets whether data points that are outside the configured data range (e.g. [QCPColorMap::setDataRange](#)) are colored by periodically repeating the color gradient or whether they all have the same color, corresponding to the respective gradient boundary color.

As shown in the image above, gradients that have the same start and end color are especially suitable for a periodic gradient mapping, since they produce smooth color transitions throughout the color map. A preset that has this property is [gpHues](#).

In practice, using periodic color gradients makes sense when the data corresponds to a periodic dimension, such as an angle or a phase. If this is not the case, the color encoding might become ambiguous, because multiple different data values are shown as the same color.

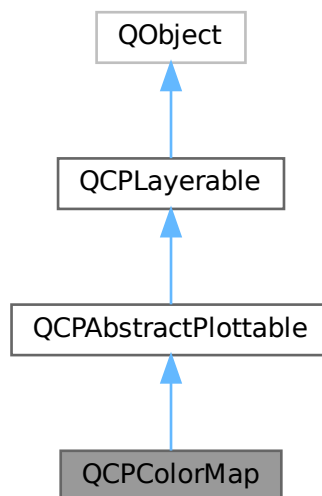
The documentation for this class was generated from the following files:

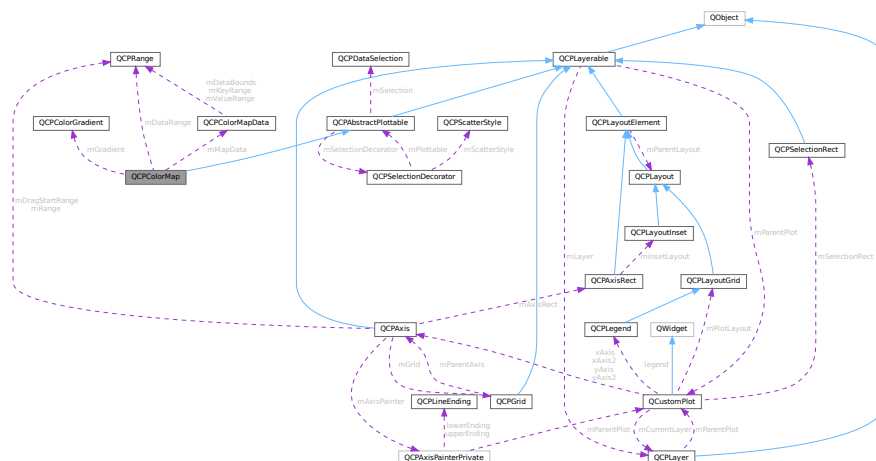
- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.569 QCPColorMap Class Reference

A plottable representing a two-dimensional color map in a plot.

Inheritance diagram for QCPColorMap:





- void **dataRangeChanged** (const **QCPRange** &newRange)
- void **dataScaleTypeChanged** (**QCPCAxis::ScaleType** scaleType)
- void **gradientChanged** (const **QCPCColorGradient** &newGradient)

- void `selectionChanged` (bool `selected`)
- void `selectionChanged` (const `QCPDataSelection` &`selection`)
- void `selectableChanged` (`QCP::SelectionType` `selectable`)

- void layerChanged (QCPLayer *newLayer)

- [QCPColorMap](#) ([QCPCAxis](#) *keyAxis, [QCPCAxis](#) *valueAxis)
- [QCPColorMapData](#) * [data](#) () const
- [QCPRange](#) **dataRange** () const
- [QCPCAxis::ScaleType](#) **dataScaleType** () const
- bool **interpolate** () const
- bool **tightBoundary** () const
- [QCPColorGradient](#) **gradient** () const
- [QCPColorScale](#) * **colorScale** () const
- void **setData** ([QCPColorMapData](#) *[data](#), bool copy=false)
- Q_SLOT void **setDataRange** (const [QCPRange](#) &dataRange)
- Q_SLOT void **setDataScaleType** ([QCPCAxis::ScaleType](#) scaleType)
- Q_SLOT void **setGradient** (const [QCPColorGradient](#) &gradient)
- void **setInterpolate** (bool enabled)
- void **setTightBoundary** (bool enabled)
- void **setColorScale** ([QCPColorScale](#) *colorScale)
- void **rescaleDataRange** (bool recalculateDataBounds=false)
- Q_SLOT void **updateLegendIcon** (Qt::TransformationMode transformMode=Qt::SmoothTransformation, const QSize &thumbSize=QSize(32, 18))

- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE
- virtual [QCPRange getKeyRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#)) const Q_DECL_OVERRIDE
- virtual [QCPRange getValueRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#), const [QCPRange](#) &inKeyRange=[QCPRange](#)()) const Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractPlottable](#)

- [QCPAbstractPlottable](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- QString **name** () const
- bool **antialiasedFill** () const
- bool **antialiasedScatters** () const
- QPen **pen** () const
- QBrush **brush** () const
- [QCPAxis](#) * **keyAxis** () const
- [QCPAxis](#) * **valueAxis** () const
- [QCP::SelectionType](#) **selectable** () const
- bool **selected** () const
- [QCPDataSelection](#) **selection** () const
- [QCPSelectionDecorator](#) * **selectionDecorator** () const
- void **setName** (const QString &name)
- void **setAntialiasedFill** (bool enabled)
- void **setAntialiasedScatters** (bool enabled)
- void **setPen** (const QPen &pen)
- void **setBrush** (const QBrush &brush)
- void **setKeyAxis** ([QCPAxis](#) *axis)
- void **setValueAxis** ([QCPAxis](#) *axis)
- Q_SLOT void **setSelectable** ([QCP::SelectionType](#) selectable)
- Q_SLOT void **setSelection** ([QCPDataSelection](#) selection)
- void **setSelectionDecorator** ([QCPSelectionDecorator](#) *decorator)
- virtual [QCPPlottableInterface1D](#) * **interface1D** ()
- void **coordsToPixels** (double key, double value, double &x, double &y) const
- const QPointF **coordsToPixels** (double key, double value) const
- void **pixelsToCoords** (double x, double y, double &key, double &value) const
- void **pixelsToCoords** (const QPointF &pixelPos, double &key, double &value) const
- void **rescaleAxes** (bool onlyEnlarge=false) const
- void **rescaleKeyAxis** (bool onlyEnlarge=false) const
- void **rescaleValueAxis** (bool onlyEnlarge=false, bool inKeyRange=false) const
- bool **addToLegend** ([QCPLegend](#) *legend)
- bool **addToLegend** ()
- bool **removeFromLegend** ([QCPLegend](#) *legend) const
- bool **removeFromLegend** () const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const QString &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Protected Member Functions

- virtual void **updateMapImage** ()
- virtual void **draw** (QCPPainter *painter) Q_DECL_OVERRIDE
- virtual void **drawLegendIcon** (QCPPainter *painter, const QRectF &rect) const Q_DECL_OVERRIDE

Protected Member Functions inherited from QCPAbstractPlottable

- virtual QRect **clipRect** () const Q_DECL_OVERRIDE
- virtual QCP::Interaction **selectionCategory** () const Q_DECL_OVERRIDE
- void **applyDefaultAntialiasingHint** (QCPPainter *painter) const Q_DECL_OVERRIDE
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void **deselectEvent** (bool *selectionStateChanged) Q_DECL_OVERRIDE
- void **applyFillAntialiasingHint** (QCPPainter *painter) const
- void **applyScattersAntialiasingHint** (QCPPainter *painter) const

Protected Member Functions inherited from QCPLayerable

- virtual void **parentPlotInitialized** (QCustomPlot *parentPlot)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** (QCustomPlot *parentPlot)
- void **setParentLayerable** (QCPLayerable *parentLayerable)
- bool **moveToLayer** (QCPLayer *layer, bool prepend)
- void **applyAntialiasingHint** (QCPPainter *painter, bool localAntialiased, QCP::AntialiasedElement overrideElement) const

Protected Attributes

- QCPRange **mDataRange**
- QCPAxis::ScaleType **mDataScaleType**
- QCPColorMapData * **mMapData**
- QCPColorGradient **mGradient**
- bool **mInterpolate**
- bool **mTightBoundary**
- QPointer< QCPColorScale > **mColorScale**
- QImage **mMapImage**
- QImage **mUndersampledMapImage**
- QPixmap **mLegendIcon**
- bool **mMapImageInvalidated**

Protected Attributes inherited from QCPAbstractPlottable

- QString **mName**
- bool **mAntialiasedFill**
- bool **mAntialiasedScatters**
- QPen **mPen**
- QBrush **mBrush**
- QPointer< QCPAxis > **mKeyAxis**
- QPointer< QCPAxis > **mValueAxis**
- QCP::SelectionType **mSelectable**
- QCPDataSelection **mSelection**
- QCPSelectionDecorator * **mSelectionDecorator**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- [QPointer](#)< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCustomPlot](#)
- class [QCPLegend](#)

15.569.1 Detailed Description

A plottable representing a two-dimensional color map in a plot.

The data is stored in the class [QCPCColorMapData](#), which can be accessed via the [data\(\)](#) method.

A color map has three dimensions to represent a data point: The *key* dimension, the *value* dimension and the *data* dimension. As with other plottables such as graphs, *key* and *value* correspond to two orthogonal axes on the [QCustomPlot](#) surface that you specify in the [QCPCColorMap](#) constructor. The *data* dimension however is encoded as the color of the point at (*key*, *value*).

[Set](#) the number of points (or *cells*) in the key/value dimension via [QCPCColorMapData::setSize](#). The plot coordinate range over which these points will be displayed is specified via [QCPCColorMapData::setRange](#). The first cell will be centered on the lower range boundary and the last cell will be centered on the upper range boundary. The data can be set by either accessing the cells directly with [QCPCColorMapData::setCell](#) or by addressing the cells via their plot coordinates with [QCPCColorMapData::setData](#). If possible, you should prefer [setCell](#), since it doesn't need to do any coordinate transformation and thus performs a bit better.

The cell with index (0, 0) is at the bottom left, if the color map uses normal (i.e. not reversed) key and value axes.

To show the user which colors correspond to which *data* values, a [QCPCColorScale](#) is typically placed to the right of the axis rect. See the documentation there for details on how to add and use a color scale.

15.569.2 Changing the appearance

Most important to the appearance is the color gradient, which can be specified via [setGradient](#). See the documentation of [QCPCColorGradient](#) for details on configuring a color gradient.

The *data* range that is mapped to the colors of the gradient can be specified with [setDataRange](#). To make the data range encompass the whole data set minimum to maximum, call [rescaleDataRange](#). If your data may contain NaN values, use [QCPCColorGradient::setNanHandling](#) to define how they are displayed.

15.569.3 Transparency

Transparency in color maps can be achieved by two mechanisms. On one hand, you can specify alpha values for color stops of the [QCPCColorGradient](#), via the regular [QColor](#) interface. This will cause the color map data which gets mapped to colors around those color stops to appear with the accordingly interpolated transparency.

On the other hand you can also directly apply an alpha value to each cell independent of its data, by using the alpha map feature of [QCPCColorMapData](#). The relevant methods are [QCPCColorMapData::setAlpha](#), [QCPCColorMapData::fillAlpha](#) and [QCPCColorMapData::clearAlpha\(\)](#).

The two transparencies will be joined together in the plot and otherwise not interfere with each other. They are mixed in a multiplicative matter, so an alpha of e.g. 50% (128/255) in both modes simultaneously, will result in a total transparency of 25% (64/255).

15.569.4 Usage

Like all data representing objects in [QCustomPlot](#), the [QCPCColorMap](#) is a plottable ([QCPCAbstractPlottable](#)). So the plottable-interface of [QCustomPlot](#) applies ([QCustomPlot::plottable](#), [QCustomPlot::removePlottable](#), etc.)

Usually, you first create an instance:

which registers it with the [QCustomPlot](#) instance of the passed axes. Note that this [QCustomPlot](#) instance takes ownership of the plottable, so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead. The newly created plottable can be modified, e.g.:

Note

The [QCPColorMap](#) always displays the data at equal key/value intervals, even if the key or value axis is set to a logarithmic scaling. If you want to use [QCPColorMap](#) with logarithmic axes, you shouldn't use the [QCPColorMapData::setData](#) method as it uses a linear transformation to determine the cell index. Rather directly access the cell index with [QCPColorMapData::setCell](#).

15.569.5 Constructor & Destructor Documentation**15.569.5.1 QCPColorMap()**

```
QCPColorMap::QCPColorMap (
    QCPAxis * keyAxis,
    QCPAxis * valueAxis ) [explicit]
```

Constructs a color map with the specified *keyAxis* and *valueAxis*.

The created [QCPColorMap](#) is automatically registered with the [QCustomPlot](#) instance inferred from *keyAxis*. This [QCustomPlot](#) instance takes ownership of the [QCPColorMap](#), so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead.

15.569.6 Member Function Documentation**15.569.6.1 data()**

```
QCPColorMapData * QCPColorMap::data ( ) const [inline]
```

Returns a pointer to the internal data storage of type [QCPColorMapData](#). Access this to modify data points (cells) and the color map key/value range.

See also

[setData](#)

15.569.6.2 dataRangeChanged

```
void QCPColorMap::dataRangeChanged (
    const QCPRange & newRange ) [signal]
```

This signal is emitted when the data range changes.

See also

[setDataRange](#)

15.569.6.3 dataScaleTypeChanged

```
void QCPColorMap::dataScaleTypeChanged (
    QCPAxis::ScaleType scaleType ) [signal]
```

This signal is emitted when the data scale type changes.

See also

[setDataScaleType](#)

15.569.6.4 draw()

```
void QCPColorMap::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

15.569.6.5 drawLegendIcon()

```
void QCPColorMap::drawLegendIcon (
    QCPPainter * painter,
    const QRectF & rect ) const [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

15.569.6.6 getKeyRange()

```
QCPRange QCPColorMap::getKeyRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth ) const [virtual]
```

Returns the coordinate range that all data in this plottable span in the key axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getValueRange](#)

Implements [QCPAbstractPlottable](#).

15.569.6.7 getValueRange()

```
QCPRange QCPColorMap::getValueRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth,
    const QCPRange & inKeyRange = QCPRange() ) const [virtual]
```

Returns the coordinate range that the data points in the specified key range (*inKeyRange*) span in the value axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

If *inKeyRange* has both lower and upper bound set to zero (is equal to [QCPRange\(\)](#)), all data points are considered, without any restriction on the keys.

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getKeyRange](#)

Implements [QCPAbstractPlottable](#).

15.569.6.8 gradientChanged

```
void QCPColorMap::gradientChanged (
    const QCPColorGradient & newGradient ) [signal]
```

This signal is emitted when the gradient changes.

See also

[setGradient](#)

15.569.6.9 rescaleDataRange()

```
void QCPColorMap::rescaleDataRange (
    bool recalculateDataBounds = false )
```

Sets the data range ([setDataRange](#)) to span the minimum and maximum values that occur in the current data set. This corresponds to the [rescaleKeyAxis](#) or [rescaleValueAxis](#) methods, only for the third data dimension of the color map.

The minimum and maximum values of the data set are buffered in the internal [QCPColorMapData](#) instance ([data](#)). As data is updated via its [QCPColorMapData::setCell](#) or [QCPColorMapData::setData](#), the buffered minimum and maximum values are updated, too. For performance reasons, however, they are only updated in an expanding fashion. So the buffered maximum can only increase and the buffered minimum can only decrease. In consequence, changes to the data that actually lower the maximum of the data set (by overwriting the cell holding the current maximum with a smaller value), aren't recognized and the buffered maximum overestimates the true maximum of the data set. The same happens for the buffered minimum. To recalculate the true minimum and maximum by explicitly looking at each cell, the method [QCPColorMapData::recalculateDataBounds](#) can be used. For convenience, setting the parameter *recalculateDataBounds* calls this method before setting the data range to the buffered minimum and maximum.

See also

[setDataRange](#)

15.569.6.10 selectTest()

```
double QCPColorMap::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the *selectionTolerance* multiplied by 0.99). Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than $0.99 \cdot \text{selectionTolerance}$).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the *mouseReleaseEvent* occurs, and the finally selected object is notified via the *selectEvent/* *deselectEvent* methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to *selectEvent* when the parent [QCustomPlot](#) decides on the basis of this *selectTest* call, that the object was successfully selected. The subsequent call to *selectEvent* will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in [selectTest](#). The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent *selectEvent*, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass *nullptr* as *details* to indicate that you are not interested in those selection details.

See also

[selectEvent](#), [deselectEvent](#), [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Implements [QCPAbstractPlottable](#).

15.569.6.11 setColorScale()

```
void QCPColorMap::setColorScale (
    QCPColorScale * colorScale )
```

Associates the color scale *colorScale* with this color map.

This means that both the color scale and the color map synchronize their gradient, data range and data scale type ([setGradient](#), [setDataRange](#), [setDataScaleType](#)). Multiple color maps can be associated with one single color scale. This causes the color maps to also synchronize those properties, via the mutual color scale.

This function causes the color map to adopt the current color gradient, data range and data scale type of *colorScale*. After this call, you may change these properties at either the color map or the color scale, and the setting will be applied to both.

Pass *nullptr* as *colorScale* to disconnect the color scale from this color map again.

15.569.6.12 setData()

```
void QCPColorMap::setData (
    QCPColorMapData * data,
    bool copy = false )
```

Replaces the current [data](#) with the provided *data*.

If *copy* is set to true, the *data* object will only be copied. If false, the color map takes ownership of the passed data and replaces the internal data pointer with it. This is significantly faster than copying for large datasets.

15.569.6.13 setDataRange()

```
void QCPColorMap::setDataRange (
    const QCPRange & dataRange )
```

Sets the data range of this color map to *dataRange*. The data range defines which data values are mapped to the color gradient.

To make the data range span the full range of the data set, use [rescaleDataRange](#).

See also

[QCPColorScale::setDataRange](#)

15.569.6.14 setDataScaleType()

```
void QCPColorMap::setDataScaleType (
    QCPAxis::ScaleType scaleType )
```

Sets whether the data is correlated with the color gradient linearly or logarithmically.

See also

[QCPColorScale::setDataScaleType](#)

15.569.6.15 setGradient()

```
void QCPColorMap::setGradient (
    const QCPColorGradient & gradient )
```

Sets the color gradient that is used to represent the data. For more details on how to create an own gradient or use one of the preset gradients, see [QCPColorGradient](#).

The colors defined by the gradient will be used to represent data values in the currently set data range, see [setDataRange](#). Data points that are outside this data range will either be colored uniformly with the respective gradient boundary color, or the gradient will repeat, depending on [QCPColorGradient::setPeriodic](#).

See also

[QCPColorScale::setGradient](#)

15.569.6.16 setInterpolate()

```
void QCPColorMap::setInterpolate (
    bool enabled )
```

Sets whether the color map image shall use bicubic interpolation when displaying the color map shrunk or expanded, and not at a 1:1 pixel-to-data scale.

15.569.6.17 setTightBoundary()

```
void QCPColorMap::setTightBoundary (
    bool enabled )
```

Sets whether the outer most data rows and columns are clipped to the specified key and value range (see [QCPColorMapData::setKeyRange](#), [QCPColorMapData::setValueRange](#)).

If *enabled* is set to false, the data points at the border of the color map are drawn with the same width and height as all other data points. Since the data points are represented by rectangles of one color centered on the data coordinate, this means that the shown color map extends by half a data point over the specified key/value range in each direction.

15.569.6.18 updateLegendIcon()

```
void QCPColormap::updateLegendIcon (
    Qt::TransformationMode transformMode = Qt::SmoothTransformation,
    const QSize & thumbSize = QSize(32, 18) )
```

Takes the current appearance of the color map and updates the legend icon, which is used to represent this color map in the legend (see [QCPLegend](#)).

The *transformMode* specifies whether the rescaling is done by a faster, low quality image scaling algorithm (Qt::FastTransformation) or by a slower, higher quality algorithm (Qt::SmoothTransformation).

The current color map appearance is scaled down to *thumbSize*. Ideally, this should be equal to the size of the legend icon (see [QCPLegend::setIconSize](#)). If it isn't exactly the configured legend icon size, the thumb will be rescaled during drawing of the legend item.

See also

[setDataRange](#)

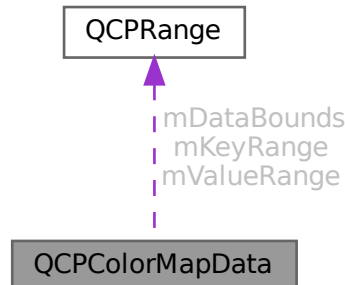
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.570 QCPColormapData Class Reference

Holds the two-dimensional data of a [QCPColormap](#) plottable.

Collaboration diagram for QCPColormapData:



Public Member Functions

- [QCPColormapData](#) (int keySize, int valueSize, const [QCPRange](#) &keyRange, const [QCPRange](#) &valueRange)
- [QCPColormapData](#) (const [QCPColormapData](#) &other)
- [QCPColormapData](#) & operator= (const [QCPColormapData](#) &other)
- int **keySize** () const
- int **valueSize** () const
- [QCPRange](#) **keyRange** () const
- [QCPRange](#) **valueRange** () const
- [QCPRange](#) **dataBounds** () const
- double **data** (double key, double value)
- double **cell** (int keyIndex, int valueIndex)
- unsigned char **alpha** (int keyIndex, int valueIndex)

- void [setSize](#) (int keySize, int valueSize)
- void [setKeySize](#) (int keySize)
- void [setValueSize](#) (int valueSize)
- void [setRange](#) (const [QCPRange](#) &keyRange, const [QCPRange](#) &valueRange)
- void [setKeyRange](#) (const [QCPRange](#) &keyRange)
- void [setValueRange](#) (const [QCPRange](#) &valueRange)
- void [setData](#) (double key, double value, double z)
- void [setCell](#) (int keyIndex, int valueIndex, double z)
- void [setAlpha](#) (int keyIndex, int valueIndex, unsigned char [alpha](#))
- void [recalculateDataBounds](#) ()
- void [clear](#) ()
- void [clearAlpha](#) ()
- void [fill](#) (double z)
- void [fillAlpha](#) (unsigned char [alpha](#))
- bool [isEmpty](#) () const
- void [coordToCell](#) (double key, double value, int *keyIndex, int *valueIndex) const
- void [cellToCoord](#) (int keyIndex, int valueIndex, double *key, double *value) const

Protected Member Functions

- bool [createAlpha](#) (bool initializeOpaque=true)

Protected Attributes

- int [mKeySize](#)
- int [mValueSize](#)
- [QCPRange](#) [mKeyRange](#)
- [QCPRange](#) [mValueRange](#)
- bool [mIsEmpty](#)
- double * [mData](#)
- unsigned char * [mAlpha](#)
- [QCPRange](#) [mDataBounds](#)
- bool [mDataModified](#)

Friends

- class [QCPColormap](#)

15.570.1 Detailed Description

Holds the two-dimensional data of a [QCPColormap](#) plottable.

This class is a data storage for [QCPColormap](#). It holds a two-dimensional array, which [QCPColormap](#) then displays as a 2D image in the plot, where the array values are represented by a color, depending on the value.

The size of the array can be controlled via [setSize](#) (or [setKeySize](#), [setValueSize](#)). Which plot coordinates these cells correspond to can be configured with [setRange](#) (or [setKeyRange](#), [setValueRange](#)).

The data cells can be accessed in two ways: They can be directly addressed by an integer index with [setCell](#). This is the fastest method. Alternatively, they can be addressed by their plot coordinate with [setData](#). plot coordinate to cell index transformations and vice versa are provided by the functions [coordToCell](#) and [cellToCoord](#).

A [QCPColormapData](#) also holds an on-demand two-dimensional array of alpha values which (if allocated) has the same size as the data map. It can be accessed via [setAlpha](#), [fillAlpha](#) and [clearAlpha](#). The memory for the alpha map is only allocated if needed, i.e. on the first call of [setAlpha](#). [clearAlpha](#) restores full opacity and frees the alpha map.

This class also buffers the minimum and maximum values that are in the data set, to provide [QCPColormap::rescaleDataRange](#) with the necessary information quickly. Setting a cell to a value that is greater than the current maximum increases this maximum to the new value. However, setting the cell that currently holds the maximum value to a smaller value doesn't decrease the maximum again, because finding the true new maximum would require going through the entire data array, which might be time consuming. The same holds for the data minimum. This functionality is

given by [recalculateDataBounds](#), such that you can decide when it is sensible to find the true current minimum and maximum. The method [QCPColormap::rescaleDataRange](#) offers a convenience parameter *recalculateDataBounds* which may be set to true to automatically call [recalculateDataBounds](#) internally.

15.570.2 Constructor & Destructor Documentation

15.570.2.1 QCPColormapData() [1/2]

```
QCPColormapData::QCPColormapData (
    int keySize,
    int valueSize,
    const QCPRange & keyRange,
    const QCPRange & valueRange )
```

Constructs a new [QCPColormapData](#) instance. The instance has *keySize* cells in the key direction and *valueSize* cells in the value direction. These cells will be displayed by the [QCPColormap](#) at the coordinates *keyRange* and *valueRange*.

See also

[setSize](#), [setKeySize](#), [setValueSize](#), [setRange](#), [setKeyRange](#), [setValueRange](#)

15.570.2.2 QCPColormapData() [2/2]

```
QCPColormapData::QCPColormapData (
    const QCPColormapData & other )
```

Constructs a new [QCPColormapData](#) instance copying the data and range of *other*.

15.570.3 Member Function Documentation

15.570.3.1 alpha()

```
unsigned char QCPColormapData::alpha (
    int keyIndex,
    int valueIndex )
```

Returns the alpha map value of the cell with the indices *keyIndex* and *valueIndex*.

If this color map data doesn't have an alpha map (because [setAlpha](#) was never called after creation or after a call to [clearAlpha](#)), returns 255, which corresponds to full opacity.

See also

[setAlpha](#)

15.570.3.2 cellToCoord()

```
void QCPColormapData::cellToCoord (
    int keyIndex,
    int valueIndex,
    double * key,
    double * value ) const
```

Transforms cell indices given by *keyIndex* and *valueIndex* to cell indices of this [QCPColormapData](#) instance. The resulting coordinates are returned via the output parameters *key* and *value*.

If you are only interested in a key or value coordinate, you may pass `nullptr` as *key* or *value*.

Note

The [QCPColormap](#) always displays the data at equal key/value intervals, even if the key or value axis is set to a logarithmic scaling. If you want to use [QCPColormap](#) with logarithmic axes, you shouldn't use the [QCPColormapData::cellToCoord](#) method as it uses a linear transformation to determine the cell index.

See also

[coordToCell](#), [QCPAxis::pixelToCoord](#)

15.570.3.3 clear()

```
void QCPColormapData::clear ( )
```

Frees the internal data memory.

This is equivalent to calling [setSize\(0, 0\)](#).

15.570.3.4 clearAlpha()

```
void QCPColormapData::clearAlpha ( )
```

Frees the internal alpha map. The color map will have full opacity again.

15.570.3.5 coordToCell()

```
void QCPColormapData::coordToCell (
    double key,
    double value,
    int * keyIndex,
    int * valueIndex ) const
```

Transforms plot coordinates given by *key* and *value* to cell indices of this [QCPColormapData](#) instance. The resulting cell indices are returned via the output parameters *keyIndex* and *valueIndex*.

The retrieved key/value cell indices can then be used for example with [setCell](#).

If you are only interested in a key or value index, you may pass `nullptr` as *valueIndex* or *keyIndex*.

Note

The [QCPColormap](#) always displays the data at equal key/value intervals, even if the key or value axis is set to a logarithmic scaling. If you want to use [QCPColormap](#) with logarithmic axes, you shouldn't use the [QCPColormapData::coordToCell](#) method as it uses a linear transformation to determine the cell index.

See also

[cellToCoord](#), [QCPColormap::coordToPixel](#)

15.570.3.6 fill()

```
void QCPColormapData::fill (
    double z )
```

Sets all cells to the value *z*.

15.570.3.7 fillAlpha()

```
void QCPColormapData::fillAlpha (
    unsigned char alpha )
```

Sets the opacity of all color map cells to *alpha*. A value of 0 for *alpha* results in a fully transparent color map, and a value of 255 results in a fully opaque color map.

If you wish to restore opacity to 100% and free any used memory for the alpha map, rather use [clearAlpha](#).

See also

[setAlpha](#)

15.570.3.8 isEmpty()

```
bool QCPColormapData::isEmpty ( ) const [inline]
```

Returns whether this instance carries no data. This is equivalent to having a size where at least one of the dimensions is 0 (see [setSize](#)).

15.570.3.9 operator=()

```
QCPColormapData & QCPColormapData::operator= (
    const QCPColormapData & other )
```

Overwrites this color map data instance with the data stored in *other*. The alpha map state is transferred, too.

15.570.3.10 recalculateDataBounds()

```
void QCPColormapData::recalculateDataBounds ( )
```

Goes through the data and updates the buffered minimum and maximum data values.

Calling this method is only advised if you are about to call [QCPColormap::rescaleDataRange](#) and can not guarantee that the cells holding the maximum or minimum data haven't been overwritten with a smaller or larger value respectively, since the buffered maximum/minimum values have been updated the last time. Why this is the case is explained in the class description ([QCPColormapData](#)).

Note that the method [QCPColormap::rescaleDataRange](#) provides a parameter *recalculateDataBounds* for convenience. Setting this to true will call this method for you, before doing the rescale.

15.570.3.11 setAlpha()

```
void QCPColormapData::setAlpha (
    int keyIndex,
    int valueIndex,
    unsigned char alpha )
```

Sets the alpha of the color map cell given by *keyIndex* and *valueIndex* to *alpha*. A value of 0 for *alpha* results in a fully transparent cell, and a value of 255 results in a fully opaque cell.

If an alpha map doesn't exist yet for this color map data, it will be created here. If you wish to restore full opacity and free any allocated memory of the alpha map, call [clearAlpha](#).

Note that the cell-wise alpha which can be configured here is independent of any alpha configured in the color map's gradient ([QCPColormapGradient](#)). If a cell is affected both by the cell-wise and gradient alpha, the alpha values will be blended accordingly during rendering of the color map.

See also

[fillAlpha](#), [clearAlpha](#)

15.570.3.12 setCell()

```
void QCPColormapData::setCell (
    int keyIndex,
    int valueIndex,
    double z )
```

Sets the data of the cell with indices *keyIndex* and *valueIndex* to *z*. The indices enumerate the cells starting from zero, up to the map's size-1 in the respective dimension (see [setSize](#)).

In the standard plot configuration (horizontal key axis and vertical value axis, both not range-reversed), the cell with indices (0, 0) is in the bottom left corner and the cell with indices (keySize-1, valueSize-1) is in the top right corner of the color map.

See also

[setData](#), [setSize](#)

15.570.3.13 setData()

```
void QCPColormapData::setData (
    double key,
    double value,
    double z )
```

Sets the data of the cell, which lies at the plot coordinates given by *key* and *value*, to *z*.

Note

The [QCPColormap](#) always displays the data at equal key/value intervals, even if the key or value axis is set to a logarithmic scaling. If you want to use [QCPColormap](#) with logarithmic axes, you shouldn't use the [QCPColormapData::setData](#) method as it uses a linear transformation to determine the cell index. Rather directly access the cell index with [QCPColormapData::setCell](#).

See also

[setCell](#), [setRange](#)

15.570.3.14 setKeyRange()

```
void QCPCoMapData::setKeyRange (
    const QCPCoRange & keyRange )
```

Sets the coordinate range the data shall be distributed over in the key dimension. Together with the value range, This defines the rectangular area covered by the color map in plot coordinates.

The outer cells will be centered on the range boundaries given to this function. For example, if the key size ([setKeySize](#)) is 3 and *keyRange* is set to [QCPCoRange](#) (2, 3) there will be cells centered on the key coordinates 2, 2.5 and 3.

See also

[setRange](#), [setValueRange](#), [setSize](#)

15.570.3.15 setKeySize()

```
void QCPCoMapData::setKeySize (
    int keySize )
```

Resizes the data array to have *keySize* cells in the key dimension.

The current data is discarded and the map cells are set to 0, unless the map had already the requested size.

Setting *keySize* to zero frees the internal data array and [isEmpty](#) returns true.

See also

[setKeyRange](#), [setSize](#), [setValueSize](#)

15.570.3.16 setRange()

```
void QCPCoMapData::setRange (
    const QCPCoRange & keyRange,
    const QCPCoRange & valueRange )
```

Sets the coordinate ranges the data shall be distributed over. This defines the rectangular area covered by the color map in plot coordinates.

The outer cells will be centered on the range boundaries given to this function. For example, if the key size ([setKeySize](#)) is 3 and *keyRange* is set to [QCPCoRange](#) (2, 3) there will be cells centered on the key coordinates 2, 2.5 and 3.

See also

[setSize](#)

15.570.3.17 setSize()

```
void QCPCoMapData::setSize (
    int keySize,
    int valueSize )
```

Resizes the data array to have *keySize* cells in the key dimension and *valueSize* cells in the value dimension.

The current data is discarded and the map cells are set to 0, unless the map had already the requested size.

Setting at least one of *keySize* or *valueSize* to zero frees the internal data array and [isEmpty](#) returns true.

See also

[setRange](#), [setKeySize](#), [setValueSize](#)

15.570.3.18 setValueRange()

```
void QCPCColorMapData::setValueRange (
    const QCPRange & valueRange )
```

Sets the coordinate range the data shall be distributed over in the value dimension. Together with the key range, This defines the rectangular area covered by the color map in plot coordinates.

The outer cells will be centered on the range boundaries given to this function. For example, if the value size ([setValueSize](#)) is 3 and *valueRange* is set to [QCPRange](#) (2, 3) there will be cells centered on the value coordinates 2, 2.5 and 3.

See also

[setRange](#), [setKeyRange](#), [setSize](#)

15.570.3.19 setValueSize()

```
void QCPCColorMapData::setValueSize (
    int valueSize )
```

Resizes the data array to have *valueSize* cells in the value dimension.

The current data is discarded and the map cells are set to 0, unless the map had already the requested size. Setting *valueSize* to zero frees the internal data array and [isEmpty](#) returns true.

See also

[setValueRange](#), [setSize](#), [setKeySize](#)

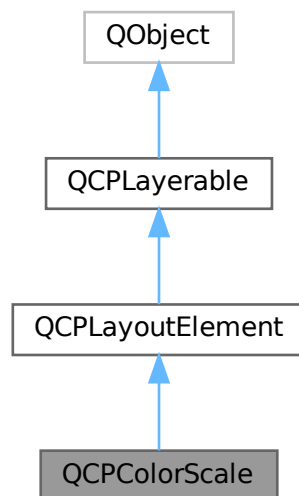
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.571 QCPCColorScale Class Reference

A color scale for use with color coding data such as [QCPCColorMap](#).

Inheritance diagram for QCPCColorScale:



- [QList< QCPColorMap * > colorMaps](#) () const
- void [rescaleDataRange](#) (bool onlyVisibleMaps)
- virtual void [update](#) ([UpdatePhase](#) phase) Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPLayoutElement](#)

- [QCPLayoutElement](#) ([QCustomPlot](#) *parentPlot=nullptr)
- [QCPLayout](#) * [layout](#) () const
- [QRect](#) [rect](#) () const
- [QRect](#) [outerRect](#) () const
- [QMargins](#) [margins](#) () const
- [QMargins](#) [minimumMargins](#) () const
- [QCP::MarginSides](#) [autoMargins](#) () const
- [QSize](#) [minimumSize](#) () const
- [QSize](#) [maximumSize](#) () const
- [SizeConstraintRect](#) [sizeConstraintRect](#) () const
- [QCPMarginGroup](#) * [marginGroup](#) ([QCP::MarginSide](#) side) const
- [QHash< QCP::MarginSide, QCPMarginGroup * >](#) [marginGroups](#) () const
- void [setOuterRect](#) (const [QRect](#) &rect)
- void [setMargins](#) (const [QMargins](#) &margins)
- void [setMinimumMargins](#) (const [QMargins](#) &margins)
- void [setAutoMargins](#) ([QCP::MarginSides](#) sides)
- void [setMinimumSize](#) (const [QSize](#) &size)
- void [setMinimumSize](#) (int width, int height)
- void [setMaximumSize](#) (const [QSize](#) &size)
- void [setMaximumSize](#) (int width, int height)
- void [setSizeConstraintRect](#) ([SizeConstraintRect](#) constraintRect)
- void [setMarginGroup](#) ([QCP::MarginSides](#) sides, [QCPMarginGroup](#) *group)
- virtual [QSize](#) [minimumOuterSizeHint](#) () const
- virtual [QSize](#) [maximumOuterSizeHint](#) () const
- virtual [QList< QCPLayoutElement * >](#) [elements](#) (bool recursive) const
- virtual double [selectTest](#) (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool [visible](#) () const
- [QCustomPlot](#) * [parentPlot](#) () const
- [QCPLayerable](#) * [parentLayerable](#) () const
- [QCPLayer](#) * [layer](#) () const
- bool [antialiased](#) () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const [QString](#) &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [mousePressEvent](#) ([QMouseEvent](#) *event, const [QVariant](#) &details) Q_DECL_OVERRIDE
- virtual void [mouseMoveEvent](#) ([QMouseEvent](#) *event, const [QPointF](#) &startPos) Q_DECL_OVERRIDE
- virtual void [mouseReleaseEvent](#) ([QMouseEvent](#) *event, const [QPointF](#) &startPos) Q_DECL_OVERRIDE
- virtual void [wheelEvent](#) ([QWheelEvent](#) *event) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayoutElement](#)

- virtual int **calculateAutoMargin** ([QCP::MarginSide](#) side)
- virtual void **layoutChanged** ()
- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayerable](#)

- virtual [QCP::Interaction](#) **selectionCategory** () const
- virtual QRect **clipRect** () const
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged↔)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QCPAxis::AxisType](#) mType
- [QCPRange](#) mDataRange
- [QCPAxis::ScaleType](#) mDataScaleType
- [QCPColorGradient](#) mGradient
- int mBarWidth
- QPointer< [QCPColorScaleAxisRectPrivate](#) > mAxisRect
- QPointer< [QCPAxis](#) > mColorAxis

Protected Attributes inherited from [QCPLayoutElement](#)

- [QCPLayout](#) * mParentLayout
- QSize mMinimumSize
- QSize mMaximumSize
- [SizeConstraintRect](#) mSizeConstraintRect
- QRect mRect
- QRect mOuterRect
- QMargins mMargins
- QMargins mMinimumMargins
- [QCP::MarginSides](#) mAutoMargins
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > mMarginGroups

Protected Attributes inherited from [QCPLayerable](#)

- bool mVisible
- [QCustomPlot](#) * mParentPlot
- QPointer< [QCPLayerable](#) > mParentLayerable
- [QCPLayer](#) * mLayer
- bool mAntialiased

Friends

- class [QCPColorScaleAxisRectPrivate](#)

Additional Inherited Members

Public Types inherited from QCPLayoutElement

- enum [UpdatePhase](#) { [upPreparation](#) , [upMargins](#) , [upLayout](#) }
- enum [SizeConstraintRect](#) { [scrInnerRect](#) , [scrOuterRect](#) }

15.571.1 Detailed Description

A color scale for use with color coding data such as [QCPCColorMap](#).

This layout element can be placed on the plot to correlate a color gradient with data values. It is usually used in combination with one or multiple [QCPCColorMaps](#).

The color scale can be either horizontal or vertical, as shown in the image above. The orientation and the side where the numbers appear is controlled with [setType](#).

Use [QCPCColorMap::setColorScale](#) to connect a color map with a color scale. Once they are connected, they share their gradient, data range and data scale type ([setGradient](#), [setDataRange](#), [setDataScaleType](#)). Multiple color maps may be associated with a single color scale, to make them all synchronize these properties.

To have finer control over the number display and axis behaviour, you can directly access the [axis](#). See the documentation of [QCPAxis](#) for details about configuring axes. For example, if you want to change the number of automatically generated ticks, call

Placing a color scale next to the main axis rect works like with any other layout element:

In this case we have placed it to the right of the default axis rect, so it wasn't necessary to call [setType](#), since [QCPAxis::atRight](#) is already the default. The text next to the color scale can be set with [setLabel](#).

For optimum appearance (like in the image above), it may be desirable to line up the axis rect and the borders of the color scale. Use a [QCPMarginGroup](#) to achieve this:

Color scales are initialized with a non-zero minimum top and bottom margin ([setMinimumMargins](#)), because vertical color scales are most common and the minimum top/bottom margin makes sure it keeps some distance to the top/bottom widget border. So if you change to a horizontal color scale by setting [setType](#) to [QCPAxis::atBottom](#) or [QCPAxis::atTop](#), you might want to also change the minimum margins accordingly, e.g. `setMinimumMargins(QMargins(6, 0, 6, 0))`.

15.571.2 Constructor & Destructor Documentation

15.571.2.1 QCPCColorScale()

```
QCPCColorScale::QCPCColorScale (
    QCustomPlot * parentPlot ) [explicit]
```

Constructs a new [QCPCColorScale](#).

15.571.3 Member Function Documentation

15.571.3.1 applyDefaultAntialiasingHint()

```
void QCPCColorScale::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

15.571.3.2 axis()

```
QCPAxis * QCPCColorScale::axis ( ) const [inline]
```

Returns the internal [QCPAxis](#) instance of this color scale. You can access it to alter the appearance and behaviour of the axis. [QCPCColorScale](#) duplicates some properties in its interface for convenience. Those are [setDataRange](#) ([QCPAxis::setRange](#)), [setDataScaleType](#) ([QCPAxis::setScaleType](#)), and the method [setLabel](#) ([QCPAxis::setLabel](#)). As they each are connected, it does not matter whether you use the method on the [QCPCColorScale](#) or on its [QCPAxis](#).

If the type of the color scale is changed with [setType](#), the axis returned by this method will change, too, to either the left, right, bottom or top axis, depending on which type was set.

15.571.3.3 colorMaps()

`QList< QCPColorMap * > QCPColorScale::colorMaps ( ) const`
Returns a list of all the color maps associated with this color scale.

15.571.3.4 dataRangeChanged

`void QCPColorScale::dataRangeChanged (
 const QCPRange & newRange) [signal]`

This signal is emitted when the data range changes.

See also

[setDataRange](#)

15.571.3.5 dataScaleTypeChanged

`void QCPColorScale::dataScaleTypeChanged (
 QCPAxis::ScaleType scaleType) [signal]`

This signal is emitted when the data scale type changes.

See also

[setDataScaleType](#)

15.571.3.6 gradientChanged

`void QCPColorScale::gradientChanged (
 const QCPColorGradient & newGradient) [signal]`

This signal is emitted when the gradient changes.

See also

[setGradient](#)

15.571.3.7 mouseMoveEvent()

`void QCPColorScale::mouseMoveEvent (
 QMouseEvent * event,
 const QPointF & startPos) [protected], [virtual]`

This event gets called when the user moves the mouse while holding a mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *startPos* indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction. The default implementation does nothing.

See also

[mousePressEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

15.571.3.8 mousePressEvent()

`void QCPColorScale::mousePressEvent (
 QMouseEvent * event,
 const QVariant & details) [protected], [virtual]`

This event gets called when the user presses a mouse button while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *details* contains layerable-specific details about the hit, which were generated in the previous call to [selectTest](#).

For example, One-dimensional plottables like [QCPGraph](#) or [QCPBars](#) convey the clicked data point in the *details* parameter, as [QCPDataSelection](#) packed as QVariant. Multi-part objects convey the specific `SelectablePart` that was hit (e.g. [QCPAxis::SelectablePart](#) in the case of axes).

[QCustomPlot](#) uses an event propagation system that works the same as Qt's system. If your layerable doesn't reimplement the [mousePressEvent](#) or explicitly calls `event->ignore()` in its reimplementation, the event will be propagated to the next layerable in the stacking order.

Once a layerable has accepted the [mousePressEvent](#), it is considered the mouse grabber and will receive all following calls to [mouseMoveEvent](#) or [mouseReleaseEvent](#) for this mouse interaction (a "mouse interaction" in this context ends with the release).

The default implementation does nothing except explicitly ignoring the event with `event->ignore()`.

See also

[mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

15.571.3.9 mouseReleaseEvent()

```
void QCPColorScale::mouseReleaseEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user releases the mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *startPos* indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction. The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

15.571.3.10 rescaleDataRange()

```
void QCPColorScale::rescaleDataRange (
    bool onlyVisibleMaps )
```

Changes the data range such that all color maps associated with this color scale are fully mapped to the gradient in the data dimension.

See also

[setDataRange](#)

15.571.3.11 setBarWidth()

```
void QCPColorScale::setBarWidth (
    int width )
```

Sets the width (or height, for horizontal color scales) the bar where the gradient is displayed will have.

15.571.3.12 setDataRange()

```
void QCPColorScale::setDataRange (
    const QCPRange & dataRange )
```

Sets the range spanned by the color gradient and that is shown by the axis in the color scale.

It is equivalent to calling [QCPColorMap::setDataRange](#) on any of the connected color maps. It is also equivalent to directly accessing the [axis](#) and setting its range with [QCPAxis::setRange](#).

See also

[setDataScaleType](#), [setGradient](#), [rescaleDataRange](#)

15.571.3.13 setDataScaleType()

```
void QCPColorScale::setDataScaleType (
    QCPAxis::ScaleType scaleType )
```

Sets the scale type of the color scale, i.e. whether values are associated with colors linearly or logarithmically. It is equivalent to calling [QCPColorMap::setDataScaleType](#) on any of the connected color maps. It is also equivalent to directly accessing the [axis](#) and setting its scale type with [QCPAxis::setScaleType](#).

Note that this method controls the coordinate transformation. For logarithmic scales, you will likely also want to use a logarithmic tick spacing and labeling, which can be achieved by setting the color scale's [axis](#) ticker to an instance of [QCPAxisTickerLog](#) :

See the documentation of [QCPAxisTickerLog](#) about the details of logarithmic axis tick creation.

See also

[setDataRange](#), [setGradient](#)

15.571.3.14 setGradient()

```
void QCPColorScale::setGradient (
    const QCPColorGradient & gradient )
```

Sets the color gradient that will be used to represent data values.

It is equivalent to calling [QCPColorMap::setGradient](#) on any of the connected color maps.

See also

[setDataRange](#), [setDataScaleType](#)

15.571.3.15 setLabel()

```
void QCPColorScale::setLabel (
    const QString & str )
```

Sets the axis label of the color scale. This is equivalent to calling [QCPAxis::setLabel](#) on the internal [axis](#).

15.571.3.16 setRangeDrag()

```
void QCPColorScale::setRangeDrag (
    bool enabled )
```

Sets whether the user can drag the data range ([setDataRange](#)).

Note that [QCP::iRangeDrag](#) must be in the [QCustomPlot](#)'s interactions ([QCustomPlot::setInteractions](#)) to allow range dragging.

15.571.3.17 setRangeZoom()

```
void QCPColorScale::setRangeZoom (
    bool enabled )
```

Sets whether the user can zoom the data range ([setDataRange](#)) by scrolling the mouse wheel.

Note that [QCP::iRangeZoom](#) must be in the [QCustomPlot](#)'s interactions ([QCustomPlot::setInteractions](#)) to allow range dragging.

15.571.3.18 setType()

```
void QCPColorScale::setType (
    QCPAxis::AxisType type )
```

Sets at which side of the color scale the axis is placed, and thus also its orientation.

Note that after setting *type* to a different value, the axis returned by [axis\(\)](#) will be a different one. The new axis will adopt the following properties from the previous axis: The range, scale type, label and ticker (the latter will be shared and not copied).

15.571.3.19 update()

```
void QCPCoiorScale::update (
    UpdatePhase phase ) [virtual]
```

Updates the layout element and sub-elements. This function is automatically called before every replot by the parent layout element. It is called multiple times, once for every [UpdatePhase](#). The phases are run through in the order of the enum values. For details about what happens at the different phases, see the documentation of [UpdatePhase](#). Layout elements that have child elements should call the [update](#) method of their child elements, and pass the current *phase* unchanged.

The default implementation executes the automatic margin mechanism in the [upMargins](#) phase. Subclasses should make sure to call the base class implementation.

Reimplemented from [QCPLayoutElement](#).

15.571.3.20 wheelEvent()

```
void QCPCoiorScale::wheelEvent (
    QWheelEvent * event ) [protected], [virtual]
```

This event gets called when the user turns the mouse scroll wheel while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`.

The `event->angleDelta()` indicates how far the mouse wheel was turned, which is usually +/- 120 for single rotation steps. However, if the mouse wheel is turned rapidly, multiple steps may accumulate to one event, making the delta larger. On the other hand, if the wheel has very smooth steps or none at all, the delta may be smaller.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#)

Reimplemented from [QCPLayerable](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

Public Member Functions inherited from **QCPAxisRect**

- **QCPAxisRect** (**QCustomPlot** *parentPlot, bool setupDefaultAxes=true)
- **QPixmap** **background** () const
- **QBrush** **backgroundBrush** () const
- bool **backgroundScaled** () const
- **Qt::AspectRatioMode** **backgroundScaledMode** () const
- **Qt::Orientations** **rangeDrag** () const
- **Qt::Orientations** **rangeZoom** () const
- **QCPAxis** * **rangeDragAxis** (**Qt::Orientation** orientation)
- **QCPAxis** * **rangeZoomAxis** (**Qt::Orientation** orientation)
- **QList**< **QCPAxis** * > **rangeDragAxes** (**Qt::Orientation** orientation)
- **QList**< **QCPAxis** * > **rangeZoomAxes** (**Qt::Orientation** orientation)
- double **rangeZoomFactor** (**Qt::Orientation** orientation)
- void **setBackground** (const **QPixmap** &pm)
- void **setBackground** (const **QPixmap** &pm, bool scaled, **Qt::AspectRatioMode** mode=**Qt::KeepAspectRatio**↵
ByExpanding)
- void **setBackground** (const **QBrush** &brush)
- void **setBackgroundScaled** (bool scaled)
- void **setBackgroundScaledMode** (**Qt::AspectRatioMode** mode)
- void **setRangeDrag** (**Qt::Orientations** orientations)
- void **setRangeZoom** (**Qt::Orientations** orientations)
- void **setRangeDragAxes** (**QCPAxis** *horizontal, **QCPAxis** *vertical)
- void **setRangeDragAxes** (**QList**< **QCPAxis** * > axes)
- void **setRangeDragAxes** (**QList**< **QCPAxis** * > horizontal, **QList**< **QCPAxis** * > vertical)
- void **setRangeZoomAxes** (**QCPAxis** *horizontal, **QCPAxis** *vertical)
- void **setRangeZoomAxes** (**QList**< **QCPAxis** * > axes)
- void **setRangeZoomAxes** (**QList**< **QCPAxis** * > horizontal, **QList**< **QCPAxis** * > vertical)
- void **setRangeZoomFactor** (double horizontalFactor, double verticalFactor)
- void **setRangeZoomFactor** (double factor)
- int **axisCount** (**QCPAxis::AxisType** type) const
- **QCPAxis** * **axis** (**QCPAxis::AxisType** type, int index=0) const
- **QList**< **QCPAxis** * > **axes** (**QCPAxis::AxisTypes** types) const
- **QList**< **QCPAxis** * > **axes** () const
- **QCPAxis** * **addAxis** (**QCPAxis::AxisType** type, **QCPAxis** *axis=nullptr)
- **QList**< **QCPAxis** * > **addAxes** (**QCPAxis::AxisTypes** types)
- bool **removeAxis** (**QCPAxis** *axis)
- **QCPLayoutInset** * **insetLayout** () const
- void **zoom** (const **QRectF** &pixelRect)
- void **zoom** (const **QRectF** &pixelRect, const **QList**< **QCPAxis** * > &affectedAxes)
- void **setupFullAxesBox** (bool connectRanges=false)
- **QList**< **QCPAbstractPlottable** * > **plottables** () const
- **QList**< **QCPGraph** * > **graphs** () const
- **QList**< **QCPAbstractItem** * > **items** () const
- int **left** () const
- int **right** () const
- int **top** () const
- int **bottom** () const
- int **width** () const
- int **height** () const
- **QSize** **size** () const
- **QPoint** **topLeft** () const
- **QPoint** **topRight** () const
- **QPoint** **bottomLeft** () const
- **QPoint** **bottomRight** () const
- **QPoint** **center** () const
- virtual **QList**< **QCPLayoutElement** * > **elements** (bool recursive) const **Q_DECL_OVERRIDE**

Public Member Functions inherited from [QCPLayoutElement](#)

- [QCPLayoutElement](#) ([QCustomPlot](#) *parentPlot=nullptr)
- [QCPLayout](#) * [layout](#) () const
- [QRect](#) [rect](#) () const
- [QRect](#) [outerRect](#) () const
- [QMargins](#) [margins](#) () const
- [QMargins](#) [minimumMargins](#) () const
- [QCP::MarginSides](#) [autoMargins](#) () const
- [QSize](#) [minimumSize](#) () const
- [QSize](#) [maximumSize](#) () const
- [SizeConstraintRect](#) [sizeConstraintRect](#) () const
- [QCPMarginGroup](#) * [marginGroup](#) ([QCP::MarginSide](#) side) const
- [QHash](#)< [QCP::MarginSide](#), [QCPMarginGroup](#) * > [marginGroups](#) () const
- void [setOuterRect](#) (const [QRect](#) &[rect](#))
- void [setMargins](#) (const [QMargins](#) &margins)
- void [setMinimumMargins](#) (const [QMargins](#) &margins)
- void [setAutoMargins](#) ([QCP::MarginSides](#) sides)
- void [setMinimumSize](#) (const [QSize](#) &size)
- void [setMinimumSize](#) (int width, int height)
- void [setMaximumSize](#) (const [QSize](#) &size)
- void [setMaximumSize](#) (int width, int height)
- void [setSizeConstraintRect](#) ([SizeConstraintRect](#) constraintRect)
- void [setMarginGroup](#) ([QCP::MarginSides](#) sides, [QCPMarginGroup](#) *group)
- virtual [QSize](#) [minimumOuterSizeHint](#) () const
- virtual [QSize](#) [maximumOuterSizeHint](#) () const
- virtual double [selectTest](#) (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const [Q_DECL_OVERRIDE](#)

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool [visible](#) () const
- [QCustomPlot](#) * [parentPlot](#) () const
- [QCPLayerable](#) * [parentLayerable](#) () const
- [QCPLayer](#) * [layer](#) () const
- bool [antialiased](#) () const
- void [setVisible](#) (bool on)
- [Q_SLOT](#) bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const [QString](#) &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void [draw](#) ([QCPPainter](#) *painter) [Q_DECL_OVERRIDE](#)
- void [updateGradientImage](#) ()
- [Q_SLOT](#) void [axisSelectionChanged](#) ([QCPAxis::SelectableParts](#) selectedParts)
- [Q_SLOT](#) void [axisSelectableChanged](#) ([QCPAxis::SelectableParts](#) selectableParts)
- virtual int [calculateAutoMargin](#) ([QCP::MarginSide](#) side) [Q_DECL_OVERRIDE](#)
- virtual void [mousePressEvent](#) ([QMouseEvent](#) *event, const [QVariant](#) &details) [Q_DECL_OVERRIDE](#)
- virtual void [mouseMoveEvent](#) ([QMouseEvent](#) *event, const [QPointF](#) &startPos) [Q_DECL_OVERRIDE](#)
- virtual void [mouseReleaseEvent](#) ([QMouseEvent](#) *event, const [QPointF](#) &startPos) [Q_DECL_OVERRIDE](#)
- virtual void [wheelEvent](#) ([QWheelEvent](#) *event) [Q_DECL_OVERRIDE](#)
- virtual void [update](#) ([UpdatePhase](#) phase) [Q_DECL_OVERRIDE](#)

Protected Member Functions inherited from [QCPAxisRect](#)

- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [layoutChanged](#) () Q_DECL_OVERRIDE
- void [drawBackground](#) ([QCPPainter](#) *painter)
- void [updateAxesOffset](#) ([QCPAxis::AxisType](#) type)

Protected Member Functions inherited from [QCPLayoutElement](#)

- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayerable](#)

- virtual [QCP::Interaction](#) [selectionCategory](#) () const
- virtual QRect [clipRect](#) () const
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged↔)
- virtual void [deselectEvent](#) (bool *selectionStateChanged)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- void [initializeParentPlot](#) ([QCustomPlot](#) *parentPlot)
- void [setParentLayerable](#) ([QCPLayerable](#) *parentLayerable)
- bool [moveToLayer](#) ([QCPLayer](#) *layer, bool prepend)
- void [applyAntialiasingHint](#) ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QCPColorScale](#) * [mParentColorScale](#)
- QImage [mGradientImage](#)
- bool [mGradientImageInvalidated](#)

Protected Attributes inherited from [QCPAxisRect](#)

- QBrush [mBackgroundBrush](#)
- QPixmap [mBackgroundPixmap](#)
- QPixmap [mScaledBackgroundPixmap](#)
- bool [mBackgroundScaled](#)
- Qt::AspectRatioMode [mBackgroundScaledMode](#)
- [QCPLayoutInset](#) * [mInsetLayout](#)
- Qt::Orientations [mRangeDrag](#)
- Qt::Orientations [mRangeZoom](#)
- QList< QPointer< [QCPAxis](#) > > [mRangeDragHorzAxis](#)
- QList< QPointer< [QCPAxis](#) > > [mRangeDragVertAxis](#)
- QList< QPointer< [QCPAxis](#) > > [mRangeZoomHorzAxis](#)
- QList< QPointer< [QCPAxis](#) > > [mRangeZoomVertAxis](#)
- double [mRangeZoomFactorHorz](#)
- double [mRangeZoomFactorVert](#)
- QList< [QCPRange](#) > [mDragStartHorzRange](#)
- QList< [QCPRange](#) > [mDragStartVertRange](#)
- QCP::AntialiasedElements [mAADragBackup](#)
- QCP::AntialiasedElements [mNotAADragBackup](#)
- bool [mDragging](#)
- QHash< [QCPAxis::AxisType](#), QList< [QCPAxis](#) * > > [mAxes](#)

Protected Attributes inherited from [QCPLayoutElement](#)

- [QCPLayout](#) * **mParentLayout**
- QSize **mMinimumSize**
- QSize **mMaximumSize**
- [SizeConstraintRect](#) **mSizeConstraintRect**
- QRect **mRect**
- QRect **mOuterRect**
- QMargins **mMargins**
- QMargins **mMinimumMargins**
- QCP::MarginSides **mAutoMargins**
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **mMarginGroups**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCPCColorScale](#)

Additional Inherited Members

Public Types inherited from [QCPLayoutElement](#)

- enum [UpdatePhase](#) { [upPreparation](#) , [upMargins](#) , [upLayout](#) }
- enum [SizeConstraintRect](#) { [scrInnerRect](#) , [scrOuterRect](#) }

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

15.572.1 Constructor & Destructor Documentation

15.572.1.1 [QCPCColorScaleAxisRectPrivate](#)()

`QCPCColorScaleAxisRectPrivate::QCPCColorScaleAxisRectPrivate ( QCPCColorScale * parentColorScale ) [explicit]`
 Creates a new instance, as a child of *parentColorScale*.

15.572.2 Member Function Documentation

15.572.2.1 [calculateAutoMargin](#)()

`int QCPAxisRect::calculateAutoMargin ( QCP::MarginSide side ) [protected], [virtual]`
 Reimplemented from [QCPAxisRect](#).

15.572.2.2 [draw](#)()

`void QCPCColorScaleAxisRectPrivate::draw ( QCPPainter * painter ) [protected], [virtual]`
 Reimplemented from [QCPAxisRect](#).

15.572.2.3 mouseMoveEvent()

```
void QCPAxisRect::mouseMoveEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user moves the mouse while holding a mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter `startPos` indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction. The default implementation does nothing.

See also

[mousePressEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPAxisRect](#).

15.572.2.4 mousePressEvent()

```
void QCPAxisRect::mousePressEvent (
    QMouseEvent * event,
    const QVariant & details ) [protected], [virtual]
```

This event gets called when the user presses a mouse button while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter `details` contains layerable-specific details about the hit, which were generated in the previous call to [selectTest](#). For example, One-dimensional plottables like [QCPGraph](#) or [QCPBars](#) convey the clicked data point in the `details` parameter, as [QCPDataSelection](#) packed as `QVariant`. Multi-part objects convey the specific `SelectablePart` that was hit (e.g. [QCPAxis::SelectablePart](#) in the case of axes).

[QCustomPlot](#) uses an event propagation system that works the same as Qt's system. If your layerable doesn't reimplement the [mousePressEvent](#) or explicitly calls `event->ignore()` in its reimplementation, the event will be propagated to the next layerable in the stacking order.

Once a layerable has accepted the [mousePressEvent](#), it is considered the mouse grabber and will receive all following calls to [mouseMoveEvent](#) or [mouseReleaseEvent](#) for this mouse interaction (a "mouse interaction" in this context ends with the release).

The default implementation does nothing except explicitly ignoring the event with `event->ignore()`.

See also

[mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPAxisRect](#).

15.572.2.5 mouseReleaseEvent()

```
void QCPAxisRect::mouseReleaseEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user releases the mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter `startPos` indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction. The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPAxisRect](#).

15.572.2.6 update()

```
void QCPAxisRect::update (
    UpdatePhase phase ) [protected], [virtual]
```

This method is called automatically upon replot and doesn't need to be called by users of [QCPAxisRect](#). Calls the base class implementation to update the margins (see [QCPLayoutElement::update](#)), and finally passes the [rect](#) to the inset layout ([insetLayout](#)) and calls its `QCPInsetLayout::update` function.

\see [baseclassmethod](#)

Reimplemented from [QCPAxisRect](#).

15.572.2.7 wheelEvent()

```
void QCPAxisRect::wheelEvent (
    QWheelEvent * event ) [protected], [virtual]
```

This event gets called when the user turns the mouse scroll wheel while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`.

The `event->angleDelta()` indicates how far the mouse wheel was turned, which is usually +/- 120 for single rotation steps. However, if the mouse wheel is turned rapidly, multiple steps may accumulate to one event, making the delta larger. On the other hand, if the wheel has very smooth steps or none at all, the delta may be smaller. The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#)

Reimplemented from [QCPAxisRect](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.573 QCPCurve Class Reference

A plottable representing a parametric curve in a plot.

Public Member Functions

- [QCPCurve](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- [QSharedPointer](#)< [QCPCurveDataContainer](#) > [data](#) () const
- [QCPScatterStyle](#) [scatterStyle](#) () const
- int [scatterSkip](#) () const
- [LineStyle](#) [lineStyle](#) () const
- void [setData](#) ([QSharedPointer](#)< [QCPCurveDataContainer](#) > [data](#))
- void [setData](#) (const [QVector](#)< double > &t, const [QVector](#)< double > &keys, const [QVector](#)< double > &values, bool alreadySorted=false)
- void [setData](#) (const [QVector](#)< double > &keys, const [QVector](#)< double > &values)
- void [setScatterStyle](#) (const [QCPScatterStyle](#) &style)
- void [setScatterSkip](#) (int skip)
- void [setLineStyle](#) ([LineStyle](#) style)
- void [addData](#) (const [QVector](#)< double > &t, const [QVector](#)< double > &keys, const [QVector](#)< double > &values, bool alreadySorted=false)
- void [addData](#) (const [QVector](#)< double > &keys, const [QVector](#)< double > &values)
- void [addData](#) (double t, double key, double value)
- void [addData](#) (double key, double value)
- virtual double [selectTest](#) (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const [Q_DECL_OVERRIDE](#)
- virtual [QCPRange](#) [getKeyRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#)) const [Q_DECL_OVERRIDE](#)
- virtual [QCPRange](#) [getValueRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#), const [QCPRange](#) &inKeyRange=[QCPRange](#)()) const [Q_DECL_OVERRIDE](#)

Public Member Functions inherited from [QCPAbstractPlottable1D](#)< [QCPCurveData](#) >

- [QCPAbstractPlottable1D](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- virtual int [dataCount](#) () const [Q_DECL_OVERRIDE](#)
- virtual double [dataMainKey](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual double [dataSortKey](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual double [dataMainValue](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual [QCPRange](#) [dataValueRange](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual [QPointF](#) [dataPixelPosition](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual bool [sortKeysIsMainKey](#) () const [Q_DECL_OVERRIDE](#)
- virtual [QCPDataSelection](#) [selectTestRect](#) (const [QRectF](#) &rect, bool onlySelectable) const [Q_DECL_OVERRIDE](#)
- virtual int [findBegin](#) (double sortKey, bool expandedRange=true) const [Q_DECL_OVERRIDE](#)
- virtual int [findEnd](#) (double sortKey, bool expandedRange=true) const [Q_DECL_OVERRIDE](#)
- virtual [QCPPlottableInterface1D](#) * [interface1D](#) () [Q_DECL_OVERRIDE](#)

Public Member Functions inherited from [QCPAbstractPlottable](#)

- [QCPAbstractPlottable](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- [QString](#) [name](#) () const
- bool [antialiasedFill](#) () const
- bool [antialiasedScatters](#) () const
- [QPen](#) [pen](#) () const
- [QBrush](#) [brush](#) () const
- [QCPAxis](#) * [keyAxis](#) () const
- [QCPAxis](#) * [valueAxis](#) () const
- [QCP::SelectionType](#) [selectable](#) () const
- bool [selected](#) () const
- [QCPDataSelection](#) [selection](#) () const
- [QCPSelectionDecorator](#) * [selectionDecorator](#) () const

- void [setName](#) (const QString &name)
- void [setAntialiasedFill](#) (bool enabled)
- void [setAntialiasedScatters](#) (bool enabled)
- void [setPen](#) (const QPen &pen)
- void [setBrush](#) (const QBrush &brush)
- void [setKeyAxis](#) (QCPAxis *axis)
- void [setValueAxis](#) (QCPAxis *axis)
- Q_SLOT void [setSelectable](#) (QCP::Selectable selectable)
- Q_SLOT void [setSelection](#) (QCPDataSelection selection)
- void [setSelectionDecorator](#) (QCPSelectionDecorator *decorator)
- void [coordsToPixels](#) (double key, double value, double &x, double &y) const
- const QPointF [coordsToPixels](#) (double key, double value) const
- void [pixelsToCoords](#) (double x, double y, double &key, double &value) const
- void [pixelsToCoords](#) (const QPointF &pixelPos, double &key, double &value) const
- void [rescaleAxes](#) (bool onlyEnlarge=false) const
- void [rescaleKeyAxis](#) (bool onlyEnlarge=false) const
- void [rescaleValueAxis](#) (bool onlyEnlarge=false, bool inKeyRange=false) const
- bool [addToLegend](#) (QCPLegend *legend)
- bool [addToLegend](#) ()
- bool [removeFromLegend](#) (QCPLegend *legend) const
- bool [removeFromLegend](#) () const

Public Member Functions inherited from QCPLayerable

- [QCPLayerable](#) (QCustomPlot *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- QCustomPlot * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void [drawLegendIcon](#) ([QCPPainter](#) *painter, const QRectF &rect) const Q_DECL_OVERRIDE
- virtual void [drawCurveLine](#) ([QCPPainter](#) *painter, const QVector< QPointF > &lines) const
- virtual void [drawScatterPlot](#) ([QCPPainter](#) *painter, const QVector< QPointF > &points, const [QCPScatterStyle](#) &style) const
- void [getCurveLines](#) (QVector< QPointF > *lines, const [QCPDataRange](#) &dataRange, double penWidth) const
- void [getScatters](#) (QVector< QPointF > *scatters, const [QCPDataRange](#) &dataRange, double scatterWidth) const
- int [getRegion](#) (double key, double value, double keyMin, double valueMax, double keyMax, double valueMin) const
- QPointF [getOptimizedPoint](#) (int otherRegion, double otherKey, double otherValue, double key, double value, double keyMin, double valueMax, double keyMax, double valueMin) const
- QVector< QPointF > [getOptimizedCornerPoints](#) (int prevRegion, int currentRegion, double prevKey, double prevValue, double key, double value, double keyMin, double valueMax, double keyMax, double valueMin) const
- bool [mayTraverse](#) (int prevRegion, int currentRegion) const

- bool **getTraverse** (double prevKey, double prevValue, double key, double value, double keyMin, double valueMax, double keyMax, double valueMin, QPointF &crossA, QPointF &crossB) const
- void **getTraverseCornerPoints** (int prevRegion, int currentRegion, double keyMin, double valueMax, double keyMax, double valueMin, QVector< QPointF > &beforeTraverse, QVector< QPointF > &afterTraverse) const
- double **pointDistance** (const QPointF &pixelPoint, QCPCurveDataContainer::const_iterator &closestData) const

Protected Member Functions inherited from [QCPAbstractPlottable1D](#)< [QCPCurveData](#) >

- void [getDataSegments](#) (QList< [QCPDataRange](#) > &selectedSegments, QList< [QCPDataRange](#) > &unselectedSegments) const
- void [drawPolyline](#) (QCPPainter *painter, const QVector< QPointF > &lineData) const

Protected Member Functions inherited from [QCPAbstractPlottable](#)

- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- void [applyDefaultAntialiasingHint](#) (QCPPainter *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- void [applyFillAntialiasingHint](#) (QCPPainter *painter) const
- void [applyScattersAntialiasingHint](#) (QCPPainter *painter) const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void [parentPlotInitialized](#) (QCustomPlot *parentPlot)
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void [initializeParentPlot](#) (QCustomPlot *parentPlot)
- void [setParentLayerable](#) (QCPLayerable *parentLayerable)
- bool [moveToLayer](#) (QCPLayer *layer, bool prepend)
- void [applyAntialiasingHint](#) (QCPPainter *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QCPScatterStyle](#) mScatterStyle
- int mScatterSkip
- [LineStyle](#) mLineStyle

Protected Attributes inherited from [QCPAbstractPlottable1D](#)< [QCPCurveData](#) >

- QSharedPointer< [QCPDataContainer](#)< [QCPCurveData](#) > > mDataContainer

Protected Attributes inherited from [QCPAbstractPlottable](#)

- QString mName
- bool mAntialiasedFill
- bool mAntialiasedScatters
- QPen mPen
- QBrush mBrush
- QPointer< [QCPAxis](#) > mKeyAxis

- `QPointer< QCPAxis > mValueAxis`
- `QCP::SelectionType mSelectable`
- `QCPDataSelection mSelection`
- `QCPSelectionDecorator * mSelectionDecorator`

Protected Attributes inherited from QCPLayerable

- `bool mVisible`
- `QCustomPlot * mParentPlot`
- `QPointer< QCPLayerable > mParentLayerable`
- `QCPLayer * mLayer`
- `bool mAntialiased`

Friends

- class `QCustomPlot`
- class `QCPLegend`

Additional Inherited Members

Signals inherited from QCPAbstractPlottable

- void `selectionChanged` (bool `selected`)
- void `selectionChanged` (const `QCPDataSelection` &`selection`)
- void `selectableChanged` (`QCP::SelectionType` `selectable`)

Signals inherited from QCPLayerable

- void `layerChanged` (`QCPLayer` *`newLayer`)

15.573.1 Detailed Description

A plottable representing a parametric curve in a plot.

Unlike `QCPGraph`, plottables of this type may have multiple points with the same key coordinate, so their visual representation can have *loops*. This is realized by introducing a third coordinate t , which defines the order of the points described by the other two coordinates x and y .

To plot data, assign it with the `setData` or `addData` functions. Alternatively, you can also access and modify the curve's data via the `data` method, which returns a pointer to the internal `QCPCurveDataContainer`.

Gaps in the curve can be created by adding data points with NaN as key and value (`qQNaN()` or `std::numeric_limits<double>::quiet_NaN()`) in between the two data points that shall be separated.

15.573.2 Changing the appearance

The appearance of the curve is determined by the pen and the brush (`setPen`, `setBrush`).

15.573.3 Usage

Like all data representing objects in `QCustomPlot`, the `QCPCurve` is a plottable (`QCPAbstractPlottable`). So the plottable-interface of `QCustomPlot` applies (`QCustomPlot::plottable`, `QCustomPlot::removePlottable`, etc.)

Usually, you first create an instance:

which registers it with the `QCustomPlot` instance of the passed axes. Note that this `QCustomPlot` instance takes ownership of the plottable, so do not delete it manually but use `QCustomPlot::removePlottable()` instead. The newly created plottable can be modified, e.g.:

15.573.4 Member Enumeration Documentation

15.573.4.1 LineStyle

enum [QCPCurve::LineStyle](#)

Defines how the curve's line is represented visually in the plot. The line is drawn with the current pen of the curve ([setPen](#)).

See also

[setLineStyle](#)

Enumerator

IsNone	No line is drawn between data points (e.g. only scatters)
IsLine	Data points are connected with a straight line.

15.573.5 Constructor & Destructor Documentation

15.573.5.1 QCPCurve()

```
QCPCurve::QCPCurve (
    QCPAxis * keyAxis,
    QCPAxis * valueAxis ) [explicit]
```

Constructs a curve which uses *keyAxis* as its key axis ("x") and *valueAxis* as its value axis ("y"). *keyAxis* and *valueAxis* must reside in the same [QCustomPlot](#) instance and not have the same orientation. If either of these restrictions is violated, a corresponding message is printed to the debug output (qDebug), the construction is not aborted, though.

The created [QCPCurve](#) is automatically registered with the [QCustomPlot](#) instance inferred from *keyAxis*. This [QCustomPlot](#) instance takes ownership of the [QCPCurve](#), so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead.

15.573.6 Member Function Documentation

15.573.6.1 addData() [1/4]

```
void QCPCurve::addData (
    const QVector< double > & keys,
    const QVector< double > & values )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided points in *keys* and *values* to the current data. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

The *t* parameter of each data point will be set to the integer index of the respective key/value pair.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

15.573.6.2 addData() [2/4]

```
void QCPCurve::addData (
    const QVector< double > & t,
    const QVector< double > & keys,
    const QVector< double > & values,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided points in *t*, *keys* and *values* to the current data. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *keys* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run. Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

15.573.6.3 addData() [3/4]

```
void QCPCurve::addData (
    double key,
    double value )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided data point as *key* and *value* to the current data.

The *t* parameter is generated automatically by increments of 1 for each point, starting at the highest *t* of previously existing data or 0, if the curve data is empty.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

15.573.6.4 addData() [4/4]

```
void QCPCurve::addData (
    double t,
    double key,
    double value )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts. Adds the provided data point as *t*, *key* and *value* to the current data.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

15.573.6.5 data()

```
QSharedPointer< QCPCurveDataContainer > QCPCurve::data ( ) const [inline]
```

Returns a shared pointer to the internal data storage of type `QCPCurveDataContainer`. You may use it to directly manipulate the data, which may be more convenient and faster than using the regular [setData](#) or [addData](#) methods.

15.573.6.6 draw()

```
void QCPCurve::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

15.573.6.7 drawLegendIcon()

```
void QCPCurve::drawLegendIcon (
    QCPPainter * painter,
    const QRectF & rect ) const [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

15.573.6.8 getKeyRange()

```
QCPRange QCPCurve::getKeyRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth ) const [virtual]
```

Returns the coordinate range that all data in this plottable span in the key axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange*

is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getValueRange](#)

Implements [QCPAbstractPlottable](#).

15.573.6.9 [getValueRange\(\)](#)

```
QCPRange QCPCurve::getValueRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth,
    const QCPRange & inKeyRange = QCPRange() ) const [virtual]
```

Returns the coordinate range that the data points in the specified key range (*inKeyRange*) span in the value axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

If *inKeyRange* has both lower and upper bound set to zero (is equal to [QCPRange\(\)](#)), all data points are considered, without any restriction on the keys.

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getKeyRange](#)

Implements [QCPAbstractPlottable](#).

15.573.6.10 [selectTest\(\)](#)

```
double QCPCurve::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Implements a *selectTest* specific to this plottable's point geometry.

If *details* is not 0, it will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

\see [baseclassmethod QCPAbstractPlottable::selectTest](#)

Reimplemented from [QCPAbstractPlottable1D< QCPCurveData >](#).

15.573.6.11 [setData\(\)](#) [1/3]

```
void QCPCurve::setData (
    const QVector< double > & keys,
    const QVector< double > & values )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data with the provided points in *keys* and *values*. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

The *t* parameter of each data point will be set to the integer index of the respective key/value pair.

See also

[addData](#)

15.573.6.12 setData() [2/3]

```
void QCPCurve::setData (
    const QVector< double > & t,
    const QVector< double > & keys,
    const QVector< double > & values,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data with the provided points in *t*, *keys* and *values*. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *t* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

See also

[addData](#)

15.573.6.13 setData() [3/3]

```
void QCPCurve::setData (
    QSharedPointer< QCPCurveDataContainer > data )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data container with the provided *data* container.

Since a QSharedPointer is used, multiple QCPCurves may share the same data container safely. Modifying the data in the container will then affect all curves that share the container. Sharing can be achieved by simply exchanging the data containers wrapped in shared pointers:

If you do not wish to share containers, but create a copy from an existing container, rather use the [QCPCurveDataContainer<DataType>::set](#) method on the curve's data container directly:

See also

[addData](#)

15.573.6.14 setLineStyle()

```
void QCPCurve::setLineStyle (
    QCPCurve::LineStyle style )
```

Sets how the single data points are connected in the plot or how they are represented visually apart from the scatter symbol. For scatter-only plots, set *style* to [IsNone](#) and [setScatterStyle](#) to the desired scatter style.

See also

[setScatterStyle](#)

15.573.6.15 setScatterSkip()

```
void QCPCurve::setScatterSkip (
    int skip )
```

If scatters are displayed (scatter style not [QCPCurve::ssNone](#)), *skip* number of scatter points are skipped/not drawn after every drawn scatter point.

This can be used to make the data appear sparser while for example still having a smooth line, and to improve performance for very high density plots.

If *skip* is set to 0 (default), all scatter points are drawn.

See also

[setScatterStyle](#)

15.573.6.16 setScatterStyle()

```
void QCPCurve::setScatterStyle (
    const QCPScatterStyle & style )
```

Sets the visual appearance of single data points in the plot. If set to [QCPScatterStyle::ssNone](#), no scatter points are drawn (e.g. for line-only plots with appropriate line style).

See also

[QCPScatterStyle](#), [setLineStyle](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.574 QCPCurveData Class Reference

Holds the data of one single data point for [QCPCurve](#).

Public Member Functions

- [QCPCurveData](#) ()
- [QCPCurveData](#) (double t, double key, double value)
- double [sortKey](#) () const
- double [mainKey](#) () const
- double [mainValue](#) () const
- [QCPRange](#) [valueRange](#) () const

Static Public Member Functions

- static [QCPCurveData](#) [fromSortKey](#) (double [sortKey](#))
- static bool [sortKeysIsMainKey](#) ()

Public Attributes

- double **t**
- double **key**
- double **value**

15.574.1 Detailed Description

Holds the data of one single data point for [QCPCurve](#).

The stored data is:

- *t*: the free ordering parameter of this curve point, like in the mathematical vector $(x(t), y(t))$. (This is the *sortKey*)
- *key*: coordinate on the key axis of this curve point (this is the *mainKey*)
- *value*: coordinate on the value axis of this curve point (this is the *mainValue*)

The container for storing multiple data points is [QCPCurveDataContainer](#). It is a typedef for [QCPCDataContainer](#) with [QCPCurveData](#) as the `DataType` template parameter. See the documentation there for an explanation regarding the data type's generic methods.

See also

[QCPCurveDataContainer](#)

15.574.2 Constructor & Destructor Documentation

15.574.2.1 QCPCurveData() [1/2]

```
QCPCurveData::QCPCurveData ( )
```

Constructs a curve data point with *t*, *key* and *value* set to zero.

15.574.2.2 QCPCurveData() [2/2]

```
QCPCurveData::QCPCurveData (
    double t,
    double key,
    double value )
```

Constructs a curve data point with the specified *t*, *key* and *value*.

15.574.3 Member Function Documentation

15.574.3.1 fromSortKey()

```
static QCPCurveData QCPCurveData::fromSortKey (
    double sortKey ) [inline], [static]
```

Returns a data point with the specified *sortKey* (assigned to the data point's *t* member). All other members are set to zero.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPCDataContainer](#).

15.574.3.2 mainKey()

```
double QCPCurveData::mainKey ( ) const [inline]
```

Returns the *key* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPCDataContainer](#).

15.574.3.3 mainValue()

```
double QCPCurveData::mainValue ( ) const [inline]
```

Returns the *value* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPCDataContainer](#).

15.574.3.4 sortKey()

```
double QCPCurveData::sortKey ( ) const [inline]
```

Returns the *t* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPCDataContainer](#).

15.574.3.5 sortKeyIsMainKey()

```
static static bool QCPCurveData::sortKeyIsMainKey ( ) [inline], [static]
```

Since the member *key* is the data point key coordinate and the member *t* is the data ordering parameter, this method returns false.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPCDataContainer](#).

15.574.3.6 valueRange()

```
QCPCRange QCPCurveData::valueRange ( ) const [inline]
```

Returns a [QCPCRange](#) with both lower and upper boundary set to *value* of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPCDataContainer](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.575 QCPDataContainer< DataType > Class Template Reference

The generic data container for one-dimensional plottables.

```
#include <applications/gui/genesys/qcustomplot.h>
```

Public Types

- typedef QVector< DataType >::const_iterator **const_iterator**
- typedef QVector< DataType >::iterator **iterator**

Public Member Functions

- [QCPDataContainer](#) ()
- int [size](#) () const
- bool [isEmpty](#) () const
- bool [autoSqueeze](#) () const
- void [setAutoSqueeze](#) (bool enabled)
- void [set](#) (const [QCPDataContainer](#)< DataType > &data)
- void [set](#) (const QVector< DataType > &data, bool alreadySorted=false)
- void [add](#) (const [QCPDataContainer](#)< DataType > &data)
- void [add](#) (const QVector< DataType > &data, bool alreadySorted=false)
- void [add](#) (const DataType &data)
- void [removeBefore](#) (double sortKey)
- void [removeAfter](#) (double sortKey)
- void [remove](#) (double sortKeyFrom, double sortKeyTo)
- void [remove](#) (double sortKey)
- void [clear](#) ()
- void [sort](#) ()
- void [squeeze](#) (bool preAllocation=true, bool postAllocation=true)
- const_iterator [constBegin](#) () const
- const_iterator [constEnd](#) () const
- iterator [begin](#) ()
- iterator [end](#) ()
- const_iterator [findBegin](#) (double sortKey, bool expandedRange=true) const
- const_iterator [findEnd](#) (double sortKey, bool expandedRange=true) const
- const_iterator [at](#) (int index) const
- [QCPRange](#) [keyRange](#) (bool &foundRange, [QCP::SignDomain](#) signDomain=[QCP::sdBoth](#))
- [QCPRange](#) [valueRange](#) (bool &foundRange, [QCP::SignDomain](#) signDomain=[QCP::sdBoth](#), const [QCPRange](#) &inKeyRange=[QCPRange](#)())
- [QCPDataRange](#) [dataRange](#) () const
- void [limitIteratorsToDataRange](#) (const_iterator &[begin](#), const_iterator &[end](#), const [QCPDataRange](#) &[dataRange](#)) const

Protected Member Functions

- void [preallocateGrow](#) (int minimumPreallocSize)
- void [performAutoSqueeze](#) ()

Protected Attributes

- bool **mAutoSqueeze**
- QVector< DataType > **mData**
- int **mPreallocSize**
- int **mPreallocIteration**

Related Symbols

(Note that these are not member symbols.)

- `template<class DataType >`
`bool qcplLessThanSortKey (const DataType &a, const DataType &b)`

15.575.1 Detailed Description

template<class DataType>
class QCPDataContainer< DataType >

The generic data container for one-dimensional plottables.

This class template provides a fast container for data storage of one-dimensional data. The data type is specified as template parameter (called *DataType* in the following) and must provide some methods as described in the [next section](#).

The data is stored in a sorted fashion, which allows very quick lookups by the sorted key as well as retrieval of ranges (see [findBegin](#), [findEnd](#), [keyRange](#)) using binary search. The container uses a preallocation and a postallocation scheme, such that appending and prepending data (with respect to the sort key) is very fast and minimizes reallocations. If data is added which needs to be inserted between existing keys, the merge usually can be done quickly too, using the fact that existing data is always sorted. The user can further improve performance by specifying that added data is already itself sorted by key, if he can guarantee that this is the case (see for example [add\(const QVector<DataType> &data, bool alreadySorted\)](#)).

The data can be accessed with the provided const iterators ([constBegin](#), [constEnd](#)). If it is necessary to alter existing data in-place, the non-const iterators can be used ([begin](#), [end](#)). Changing data members that are not the sort key (for most data types called *key*) is safe from the container's perspective.

Great care must be taken however if the sort key is modified through the non-const iterators. For performance reasons, the iterators don't automatically cause a re-sorting upon their manipulation. It is thus the responsibility of the user to leave the container in a sorted state when finished with the data manipulation, before calling any other methods on the container. A complete re-sort (e.g. after finishing all sort key manipulation) can be done by calling [sort](#). Failing to do so can not be detected by the container efficiently and will cause both rendering artifacts and potential data loss.

Implementing one-dimensional plottables that make use of a [QCPDataContainer<T>](#) is usually done by subclassing from [QCPAbstractPlottable1D<T>](#), which introduces an according *mDataContainer* member and some convenience methods.

15.575.2 Requirements for the DataType template parameter

The template parameter *DataType* is the type of the stored data points. It must be trivially copyable and have the following public methods, preferably inline:

- `double sortKey() const`
Returns the member variable of this data point that is the sort key, defining the ordering in the container. Often this variable is simply called *key*.
- `static DataType fromSortKey(double sortKey)`
Returns a new instance of the data type initialized with its sort key set to *sortKey*.
- `static bool sortKeyIsMainKey()`
Returns true if the sort key is equal to the main key (see method `mainKey` below). For most plottables this is the case. It is not the case for example for [QCPCurve](#), which uses *t* as sort key and *key* as main key. This is the reason why [QCPCurve](#) unlike [QCPGraph](#) can display parametric curves with loops.

- `double mainKey() const`
Returns the variable of this data point considered the main key. This is commonly the variable that is used as the coordinate of this data point on the key axis of the plottable. This method is used for example when determining the automatic axis rescaling of key axes ([QCPAxis::rescale](#)).
- `double mainValue() const`
Returns the variable of this data point considered the main value. This is commonly the variable that is used as the coordinate of this data point on the value axis of the plottable.
- [QCPRange](#) `valueRange() const`
Returns the range this data point spans in the value axis coordinate. If the data is single-valued (e.g. [QCPGraphData](#)), this is simply a range with both lower and upper set to the main data point value. However if the data points can represent multiple values at once (e.g. [QCPFinancialData](#) with its *high*, *low*, *open* and *close* values at each *key*) this method should return the range those values span. This method is used for example when determining the automatic axis rescaling of value axes ([QCPAxis::rescale](#)).

15.575.3 Constructor & Destructor Documentation

15.575.3.1 QCPDataContainer()

```
template<class DataType >
QCPDataContainer< DataType >::QCPDataContainer ( )
```

Constructs a [QCPDataContainer](#) used for plottable classes that represent a series of key-sorted data

15.575.4 Member Function Documentation

15.575.4.1 add() [1/3]

```
template<class DataType >
void QCPDataContainer< DataType >::add (
    const DataType & data )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided single data point to the current data.

See also

[remove](#)

15.575.4.2 add() [2/3]

```
template<class DataType >
void QCPDataContainer< DataType >::add (
    const QCPDataContainer< DataType > & data )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided *data* to the current data in this container.

See also

[set](#), [remove](#)

15.575.4.3 add() [3/3]

```
template<class DataType >
void QCPDataContainer< DataType >::add (
    const QVector< DataType > & data,
    bool alreadySorted = false )
```

Adds the provided data points in *data* to the current data.

If you can guarantee that the data points in *data* have ascending order with respect to the *DataType*'s sort key, set *alreadySorted* to true to avoid an unnecessary sorting run.

See also

[set](#), [remove](#)

15.575.4.4 at()

```
template<class DataType >
QCPDataContainer::const_iterator QCPDataContainer< DataType >::at (
    int index ) const [inline]
```

Returns a const iterator to the element with the specified *index*. If *index* points beyond the available elements in this container, returns [constEnd](#), i.e. an iterator past the last valid element.

You can use this method to easily obtain iterators from a [QCPDataRange](#), see the data selection page for an example.

15.575.4.5 begin()

```
template<class DataType >
QCPDataContainer::iterator QCPDataContainer< DataType >::begin ( ) [inline]
```

Returns a non-const iterator to the first data point in this container.

You can manipulate the data points in-place through the non-const iterators, but great care must be taken when manipulating the sort key of a data point, see [sort](#), or the detailed description of this class.

15.575.4.6 clear()

```
template<class DataType >
void QCPDataContainer< DataType >::clear ( )
```

Removes all data points.

See also

[remove](#), [removeAfter](#), [removeBefore](#)

15.575.4.7 constBegin()

```
template<class DataType >
QCPDataContainer::const_iterator QCPDataContainer< DataType >::constBegin ( ) const [inline]
```

Returns a const iterator to the first data point in this container.

15.575.4.8 constEnd()

```
template<class DataType >
QCPDataContainer::const_iterator QCPDataContainer< DataType >::constEnd ( ) const [inline]
```

Returns a const iterator to the element past the last data point in this container.

15.575.4.9 dataRange()

```
template<class DataType >
QCPDataRange QCPDataContainer< DataType >::dataRange ( ) const [inline]
```

Returns a [QCPDataRange](#) encompassing the entire data set of this container. This means the begin index of the returned range is 0, and the end index is [size](#).

15.575.4.10 end()

```
template<class DataType >
QCPDataContainer::iterator QCPDataContainer< DataType >::end ( ) [inline]
```

Returns a non-const iterator to the element past the last data point in this container.

You can manipulate the data points in-place through the non-const iterators, but great care must be taken when manipulating the sort key of a data point, see [sort](#), or the detailed description of this class.

15.575.4.11 findBegin()

```
template<class DataType >
QCPDataContainer< DataType >::const_iterator QCPDataContainer< DataType >::findBegin (
    double sortKey,
    bool expandedRange = true ) const
```

Returns an iterator to the data point with a (sort-)key that is equal to, just below, or just above *sortKey*. If *expandedRange* is true, the data point just below *sortKey* will be considered, otherwise the one just above.

This can be used in conjunction with [findEnd](#) to iterate over data points within a given key range, including or excluding the bounding data points that are just beyond the specified range.

If *expandedRange* is true but there are no data points below *sortKey*, [constBegin](#) is returned.

If the container is empty, returns [constEnd](#).

See also

[findEnd](#), [QCPPlottableInterface1D::findBegin](#)

15.575.4.12 findEnd()

```
template<class DataType >
QCPDataContainer< DataType >::const_iterator QCPDataContainer< DataType >::findEnd (
    double sortKey,
    bool expandedRange = true ) const
```

Returns an iterator to the element after the data point with a (sort-)key that is equal to, just above or just below *sortKey*. If *expandedRange* is true, the data point just above *sortKey* will be considered, otherwise the one just below.

This can be used in conjunction with [findBegin](#) to iterate over data points within a given key range, including the bounding data points that are just below and above the specified range.

If *expandedRange* is true but there are no data points above *sortKey*, [constEnd](#) is returned.

If the container is empty, [constEnd](#) is returned.

See also

[findBegin](#), [QCPPlottableInterface1D::findEnd](#)

15.575.4.13 isEmpty()

```
template<class DataType >
bool QCPDataContainer< DataType >::isEmpty ( ) const [inline]
```

Returns whether this container holds no data points.

15.575.4.14 keyRange()

```
template<class DataType >
QCPRange QCPDataContainer< DataType >::keyRange (
    bool & foundRange,
    QCP::SignDomain signDomain = QCP::sdBoth )
```

Returns the range encompassed by the (main-)key coordinate of all data points. The output parameter *foundRange* indicates whether a sensible range was found. If this is false, you should not use the returned [QCPRange](#) (e.g. the data container is empty or all points have the same key).

Use *signDomain* to control which sign of the key coordinates should be considered. This is relevant e.g. for logarithmic plots which can mathematically only display one sign domain at a time.

If the *DataType* reports that its main key is equal to the sort key (*sortKeyIsMainKey*), as is the case for most plottables, this method uses this fact and finds the range very quickly.

See also

[valueRange](#)

15.575.4.15 limitIteratorsToDataRange()

```
template<class DataType >
void QCPDataContainer< DataType >::limitIteratorsToDataRange (
    const_iterator & begin,
    const_iterator & end,
    const QCPDataRange & dataRange ) const
```

Makes sure *begin* and *end* mark a data range that is both within the bounds of this data container's data, as well as within the specified *dataRange*. The initial range described by the passed iterators *begin* and *end* is never expanded, only contracted if necessary.

This function doesn't require for *dataRange* to be within the bounds of this data container's valid range.

15.575.4.16 remove() [1/2]

```
template<class DataType >
void QCPDataContainer< DataType >::remove (
    double sortKey )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Removes a single data point at *sortKey*. If the position is not known with absolute (binary) precision, consider using [remove\(double sortKeyFrom, double sortKeyTo\)](#) with a small fuzziness interval around the suspected position, depending on the precision with which the (sort-)key is known.

See also

[removeBefore](#), [removeAfter](#), [clear](#)

15.575.4.17 remove() [2/2]

```
template<class DataType >
void QCPDataContainer< DataType >::remove (
    double sortKeyFrom,
    double sortKeyTo )
```

Removes all data points with (sort-)keys between *sortKeyFrom* and *sortKeyTo*. if *sortKeyFrom* is greater or equal to *sortKeyTo*, the function does nothing. To remove a single data point with known (sort-)key, use [remove\(double sortKey\)](#).

See also

[removeBefore](#), [removeAfter](#), [clear](#)

15.575.4.18 removeAfter()

```
template<class DataType >
void QCPDataContainer< DataType >::removeAfter (
    double sortKey )
```

Removes all data points with (sort-)keys greater than or equal to *sortKey*.

See also

[removeBefore](#), [remove](#), [clear](#)

15.575.4.19 removeBefore()

```
template<class DataType >
void QCPDataContainer< DataType >::removeBefore (
    double sortKey )
```

Removes all data points with (sort-)keys smaller than or equal to *sortKey*.

See also

[removeAfter](#), [remove](#), [clear](#)

15.575.4.20 set() [1/2]

```
template<class DataType >
void QCPDataContainer< DataType >::set (
    const QCPDataContainer< DataType > & data )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data in this container with the provided *data*.

See also

[add](#), [remove](#)

15.575.4.21 set() [2/2]

```
template<class DataType >
void QCPDataContainer< DataType >::set (
    const QVector< DataType > & data,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data in this container with the provided *data*

If you can guarantee that the data points in *data* have ascending order with respect to the *DataType*'s sort key, set *alreadySorted* to true to avoid an unnecessary sorting run.

See also

[add](#), [remove](#)

15.575.4.22 setAutoSqueeze()

```
template<class DataType >
void QCPDataContainer< DataType >::setAutoSqueeze (
    bool enabled )
```

Sets whether the container automatically decides when to release memory from its post- and preallocation pools when data points are removed. By default this is enabled and for typical applications shouldn't be changed.

If auto squeeze is disabled, you can manually decide when to release pre-/postallocation with [squeeze](#).

15.575.4.23 size()

```
template<class DataType >
int QCPDataContainer< DataType >::size ( ) const [inline]
```

Returns the number of data points in the container.

15.575.4.24 sort()

```
template<class DataType >
void QCPDataContainer< DataType >::sort ( )
```

Re-sorts all data points in the container by their sort key.

When setting, adding or removing points using the [QCPDataContainer](#) interface ([set](#), [add](#), [remove](#), etc.), the container makes sure to always stay in a sorted state such that a full resort is never necessary. However, if you choose to directly manipulate the sort key on data points by accessing and modifying it through the non-const iterators ([begin](#), [end](#)), it is your responsibility to bring the container back into a sorted state before any other methods are called on it. This can be achieved by calling this method immediately after finishing the sort key manipulation.

15.575.4.25 squeeze()

```
template<class DataType >
void QCPDataContainer< DataType >::squeeze (
```

```
bool preAllocation = true,
bool postAllocation = true )
```

Frees all unused memory that is currently in the preallocation and postallocation pools.

Note that [QCPDataContainer](#) automatically decides whether squeezing is necessary, if [setAutoSqueeze](#) is left enabled. It should thus not be necessary to use this method for typical applications.

The parameters *preAllocation* and *postAllocation* control whether pre- and/or post allocation should be freed, respectively.

15.575.4.26 valueRange()

```
template<class DataType >
QCPRange QCPDataContainer< DataType >::valueRange (
    bool & foundRange,
    QCP::SignDomain signDomain = QCP::sdBoth,
    const QCPRange & inKeyRange = QCPRange() )
```

Returns the range encompassed by the value coordinates of the data points in the specified key range (*inKeyRange*), using the full *DataType::valueRange* reported by the data points. The output parameter *foundRange* indicates whether a sensible range was found. If this is false, you should not use the returned [QCPRange](#) (e.g. the data container is empty or all points have the same value).

If *inKeyRange* has both lower and upper bound set to zero (is equal to [QCPRange\(\)](#)), all data points are considered, without any restriction on the keys.

Use *signDomain* to control which sign of the value coordinates should be considered. This is relevant e.g. for logarithmic plots which can mathematically only display one sign domain at a time.

See also

[keyRange](#)

15.575.5 Friends And Related Symbol Documentation

15.575.5.1 qcpLessThanSortKey()

```
template<class DataType >
bool qcpLessThanSortKey (
    const DataType & a,
    const DataType & b ) [related]
```

Returns whether the sort key of *a* is less than the sort key of *b*.

See also

[QCPDataContainer::sort](#)

The documentation for this class was generated from the following file:

- applications/gui/genesys/qcustomplot.h

15.576 QCPDataRange Class Reference

Describes a data range given by begin and end index.

Public Member Functions

- [QCPDataRange\(\)](#)
- [QCPDataRange\(int begin, int end\)](#)
- bool **operator==** (const [QCPDataRange](#) &other) const
- bool **operator!=** (const [QCPDataRange](#) &other) const
- int **begin** () const
- int **end** () const
- int **size** () const
- int **length** () const

- void [setBegin](#) (int begin)
- void [setEnd](#) (int end)
- bool [isValid](#) () const
- bool [isEmpty](#) () const
- [QCPDataRange](#) [bounded](#) (const [QCPDataRange](#) &other) const
- [QCPDataRange](#) [expanded](#) (const [QCPDataRange](#) &other) const
- [QCPDataRange](#) [intersection](#) (const [QCPDataRange](#) &other) const
- [QCPDataRange](#) [adjusted](#) (int changeBegin, int changeEnd) const
- bool [intersects](#) (const [QCPDataRange](#) &other) const
- bool [contains](#) (const [QCPDataRange](#) &other) const

Related Symbols

(Note that these are not member symbols.)

- QDebug [operator<<](#) (QDebug d, const [QCPDataRange](#) &dataRange)

15.576.1 Detailed Description

Describes a data range given by begin and end index.

[QCPDataRange](#) holds two integers describing the begin ([setBegin](#)) and end ([setEnd](#)) index of a contiguous set of data points. The *end* index corresponds to the data point just after the last data point of the data range, like in standard iterators.

Data Ranges are not bound to a certain plottable, thus they can be freely exchanged, created and modified. If a non-contiguous data set shall be described, the class [QCPDataSelection](#) is used, which holds and manages multiple instances of [QCPDataRange](#). In most situations, [QCPDataSelection](#) is thus used.

Both [QCPDataRange](#) and [QCPDataSelection](#) offer convenience methods to work with them, e.g. [bounded](#), [expanded](#), [intersects](#), [intersection](#), [adjusted](#), [contains](#). Further, addition and subtraction operators (defined in [QCPDataSelection](#)) can be used to join/subtract data ranges and data selections (or mixtures), to retrieve a corresponding [QCPDataSelection](#).

QCustomPlot's data selection mechanism is based on [QCPDataSelection](#) and [QCPDataRange](#).

Note

Do not confuse [QCPDataRange](#) with [QCPRange](#). A [QCPRange](#) describes an interval in floating point plot coordinates, e.g. the current axis range.

15.576.2 Constructor & Destructor Documentation

15.576.2.1 [QCPDataRange](#)() [1/2]

```
QCPDataRange::QCPDataRange ( )
```

Creates an empty [QCPDataRange](#), with begin and end set to 0.

15.576.2.2 [QCPDataRange](#)() [2/2]

```
QCPDataRange::QCPDataRange (
    int begin,
    int end )
```

Creates a [QCPDataRange](#), initialized with the specified *begin* and *end*.

No checks or corrections are made to ensure the resulting range is valid ([isValid](#)).

15.576.3 Member Function Documentation

15.576.3.1 [adjusted](#)()

```
QCPDataRange QCPDataRange::adjusted (
    int changeBegin,
    int changeEnd ) const [inline]
```

Returns a data range where *changeBegin* and *changeEnd* were added to the begin and end indices, respectively.

15.576.3.2 bounded()

```
QCPDataRange QCPDataRange::bounded (
    const QCPDataRange & other ) const
```

Returns a data range that matches this data range, except that parts exceeding *other* are excluded.

This method is very similar to [intersection](#), with one distinction: If this range and the *other* range share no intersection, the returned data range will be empty with begin and end set to the respective boundary side of *other*, at which this range is residing. ([intersection](#) would just return a range with begin and end set to 0.)

15.576.3.3 contains()

```
bool QCPDataRange::contains (
    const QCPDataRange & other ) const
```

Returns whether all data points of *other* are also contained inside this data range.

See also

[intersects](#)

15.576.3.4 expanded()

```
QCPDataRange QCPDataRange::expanded (
    const QCPDataRange & other ) const
```

Returns a data range that contains both this data range as well as *other*.

15.576.3.5 intersection()

```
QCPDataRange QCPDataRange::intersection (
    const QCPDataRange & other ) const
```

Returns the data range which is contained in both this data range and *other*.

This method is very similar to [bounded](#), with one distinction: If this range and the *other* range share no intersection, the returned data range will be empty with begin and end set to 0. ([bounded](#) would return a range with begin and end set to one of the boundaries of *other*, depending on which side this range is on.)

See also

[QCPDataSelection::intersection](#)

15.576.3.6 intersects()

```
bool QCPDataRange::intersects (
    const QCPDataRange & other ) const
```

Returns whether this data range and *other* share common data points.

See also

[intersection](#), [contains](#)

15.576.3.7 isEmpty()

```
bool QCPDataRange::isEmpty ( ) const [inline]
```

Returns whether this range is empty, i.e. whether its begin index equals its end index.

See also

[size](#), [length](#)

15.576.3.8 isValid()

```
bool QCPDataRange::isValid ( ) const [inline]
```

Returns whether this range is valid. A valid range has a begin index greater or equal to 0, and an end index greater or equal to the begin index.

Note

Invalid ranges should be avoided and are never the result of any of [QCustomPlot](#)'s methods (unless they are themselves fed with invalid ranges). Do not pass invalid ranges to [QCustomPlot](#)'s methods. The invalid range is not inherently prevented in [QCPDataRange](#), to allow temporary invalid begin/end values while manipulating the range. An invalid range is not necessarily empty ([isEmpty](#)), since its [length](#) can be negative and thus non-zero.

15.576.3.9 length()

```
int QCPDataRange::length ( ) const [inline]
```

Returns the number of data points described by this data range. Equivalent to [size](#).

15.576.3.10 setBegin()

```
void QCPDataRange::setBegin (
    int begin ) [inline]
```

Sets the begin of this data range. The *begin* index points to the first data point that is part of the data range. No checks or corrections are made to ensure the resulting range is valid ([isValid](#)).

See also

[setEnd](#)

15.576.3.11 setEnd()

```
void QCPDataRange::setEnd (
    int end ) [inline]
```

Sets the end of this data range. The *end* index points to the data point just after the last data point that is part of the data range.

No checks or corrections are made to ensure the resulting range is valid ([isValid](#)).

See also

[setBegin](#)

15.576.3.12 size()

```
int QCPDataRange::size ( ) const [inline]
```

Returns the number of data points described by this data range. This is equal to the end index minus the begin index.

See also

[length](#)

15.576.4 Friends And Related Symbol Documentation

15.576.4.1 operator<<()

```
QDebug operator<< (
    QDebug d,
    const QCPDataRange & dataRange ) [related]
```

Prints *dataRange* in a human readable format to the qDebug output.

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.577 QCPDataSelection Class Reference

Describes a data set by holding multiple [QCPDataRange](#) instances.

Public Member Functions

- [QCPDataSelection](#) ()
- [QCPDataSelection](#) (const [QCPDataRange](#) &range)
- bool [operator==](#) (const [QCPDataSelection](#) &other) const
- bool [operator!=](#) (const [QCPDataSelection](#) &other) const
- [QCPDataSelection](#) & [operator+=](#) (const [QCPDataSelection](#) &other)
- [QCPDataSelection](#) & [operator+=](#) (const [QCPDataRange](#) &other)
- [QCPDataSelection](#) & [operator-=](#) (const [QCPDataSelection](#) &other)
- [QCPDataSelection](#) & [operator-=](#) (const [QCPDataRange](#) &other)
- int [dataRangeCount](#) () const
- int [dataPointCount](#) () const
- [QCPDataRange](#) [dataRange](#) (int index=0) const
- QList< [QCPDataRange](#) > [dataRanges](#) () const
- [QCPDataRange](#) [span](#) () const
- void [addDataRange](#) (const [QCPDataRange](#) &dataRange, bool [simplify](#)=true)
- void [clear](#) ()
- bool [isEmpty](#) () const
- void [simplify](#) ()
- void [enforceType](#) ([QCP::SelectionType](#) type)
- bool [contains](#) (const [QCPDataSelection](#) &other) const
- [QCPDataSelection](#) [intersection](#) (const [QCPDataRange](#) &other) const
- [QCPDataSelection](#) [intersection](#) (const [QCPDataSelection](#) &other) const
- [QCPDataSelection](#) [inverse](#) (const [QCPDataRange](#) &outerRange) const

Friends

- const [QCPDataSelection](#) [operator+](#) (const [QCPDataSelection](#) &a, const [QCPDataSelection](#) &b)
- const [QCPDataSelection](#) [operator+](#) (const [QCPDataRange](#) &a, const [QCPDataSelection](#) &b)
- const [QCPDataSelection](#) [operator+](#) (const [QCPDataSelection](#) &a, const [QCPDataRange](#) &b)
- const [QCPDataSelection](#) [operator+](#) (const [QCPDataRange](#) &a, const [QCPDataRange](#) &b)
- const [QCPDataSelection](#) [operator-](#) (const [QCPDataSelection](#) &a, const [QCPDataSelection](#) &b)
- const [QCPDataSelection](#) [operator-](#) (const [QCPDataRange](#) &a, const [QCPDataSelection](#) &b)
- const [QCPDataSelection](#) [operator-](#) (const [QCPDataSelection](#) &a, const [QCPDataRange](#) &b)
- const [QCPDataSelection](#) [operator-](#) (const [QCPDataRange](#) &a, const [QCPDataRange](#) &b)

Related Symbols

(Note that these are not member symbols.)

- QDebug [operator<<](#) (QDebug d, const [QCPDataSelection](#) &selection)

15.577.1 Detailed Description

Describes a data set by holding multiple [QCPDataRange](#) instances.

[QCPDataSelection](#) manages multiple instances of [QCPDataRange](#) in order to represent any (possibly disjoint) set of data selection.

The data selection can be modified with addition and subtraction operators which take [QCPDataSelection](#) and [QCPDataRange](#) instances, as well as methods such as [addDataRange](#) and [clear](#). Read access is provided by [dataRange](#), [dataRanges](#), [dataRangeCount](#), etc.

The method [simplify](#) is used to join directly adjacent or even overlapping [QCPDataRange](#) instances. [QCPDataSelection](#) automatically simplifies when using the addition/subtraction operators. The only case when [simplify](#) is left to the user, is when calling [addDataRange](#), with the parameter [simplify](#) explicitly set to false. This is

useful if many data ranges will be added to the selection successively and the overhead for simplifying after each iteration shall be avoided. In this case, you should make sure to call [simplify](#) after completing the operation. Use [enforceType](#) to bring the data selection into a state complying with the constraints for selections defined in [QCP::SelectionType](#).

QCustomPlot's data selection mechanism is based on [QCPDataSelection](#) and [QCPDataRange](#).

15.577.2 Iterating over a data selection

As an example, the following code snippet calculates the average value of a graph's data [selection](#):

15.577.3 Constructor & Destructor Documentation

15.577.3.1 QCPDataSelection() [1/2]

```
QCPDataSelection::QCPDataSelection ( ) [explicit]
```

Creates an empty [QCPDataSelection](#).

15.577.3.2 QCPDataSelection() [2/2]

```
QCPDataSelection::QCPDataSelection (
    const QCPDataRange & range ) [explicit]
```

Creates a [QCPDataSelection](#) containing the provided *range*.

15.577.4 Member Function Documentation

15.577.4.1 addDataRange()

```
void QCPDataSelection::addDataRange (
    const QCPDataRange & dataRange,
    bool simplify = true )
```

Adds the given *dataRange* to this data selection. This is equivalent to the += operator but allows disabling immediate simplification by setting *simplify* to false. This can improve performance if adding a very large amount of data ranges successively. In this case, make sure to call [simplify](#) manually, after the operation.

15.577.4.2 clear()

```
void QCPDataSelection::clear ( )
```

Removes all data ranges. The data selection then contains no data points.

[isEmpty](#)

15.577.4.3 contains()

```
bool QCPDataSelection::contains (
    const QCPDataSelection & other ) const
```

Returns true if the data selection *other* is contained entirely in this data selection, i.e. all data point indices that are in *other* are also in this data selection.

See also

[QCPDataRange::contains](#)

15.577.4.4 dataPointCount()

```
int QCPDataSelection::dataPointCount ( ) const
```

Returns the total number of data points contained in all data ranges that make up this data selection.

15.577.4.5 dataRange()

```
QCPDataRange QCPDataSelection::dataRange (
    int index = 0 ) const
```

Returns the data range with the specified *index*.

If the data selection is simplified (the usual state of the selection, see [simplify](#)), the ranges are sorted by ascending data point index.

See also

[dataRangeCount](#)

15.577.4.6 dataRangeCount()

```
int QCPDataSelection::dataRangeCount ( ) const [inline]
```

Returns the number of ranges that make up the data selection. The ranges can be accessed by [dataRange](#) via their index.

See also

[dataRange](#), [dataPointCount](#)

15.577.4.7 dataRanges()

```
QList< QCPDataRange > QCPDataSelection::dataRanges ( ) const [inline]
```

Returns all data ranges that make up the data selection. If the data selection is simplified (the usual state of the selection, see [simplify](#)), the ranges are sorted by ascending data point index.

See also

[dataRange](#)

15.577.4.8 enforceType()

```
void QCPDataSelection::enforceType (
    QCP::SelectionType type )
```

Makes sure this data selection conforms to the specified *type* selection type. Before the type is enforced, [simplify](#) is called.

Depending on *type*, enforcing means adding new data points that were previously not part of the selection, or removing data points from the selection. If the current selection already conforms to *type*, the data selection is not changed.

See also

[QCP::SelectionType](#)

15.577.4.9 intersection() [1/2]

```
QCPDataSelection QCPDataSelection::intersection (
    const QCPDataRange & other ) const
```

Returns a data selection containing the points which are both in this data selection and in the data range *other*.

A common use case is to limit an unknown data selection to the valid range of a data container, using [QCPDataContainer::dataRange](#) as *other*. One can then safely iterate over the returned data selection without exceeding the data container's bounds.

15.577.4.10 intersection() [2/2]

```
QCPDataSelection QCPDataSelection::intersection (
    const QCPDataSelection & other ) const
```

Returns a data selection containing the points which are both in this data selection and in the data selection *other*.

15.577.4.11 inverse()

```
QCPDataSelection QCPDataSelection::inverse (
    const QCPDataRange & outerRange ) const
```

Returns a data selection which is the exact inverse of this data selection, with *outerRange* defining the base range on which to invert. If *outerRange* is smaller than the [span](#) of this data selection, it is expanded accordingly. For example, this method can be used to retrieve all unselected segments by setting *outerRange* to the full data range of the plottable, and calling this method on a data selection holding the selected segments.

15.577.4.12 isEmpty()

```
bool QCPDataSelection::isEmpty ( ) const [inline]
```

Returns true if there are no data ranges, and thus no data points, in this [QCPDataSelection](#) instance.

See also

[dataRangeCount](#)

15.577.4.13 operator+=() [1/2]

```
QCPDataSelection & QCPDataSelection::operator+= (
    const QCPDataRange & other )
```

Adds the data range *other* to this data selection, and then simplifies this data selection (see [simplify](#)).

15.577.4.14 operator+=() [2/2]

```
QCPDataSelection & QCPDataSelection::operator+= (
    const QCPDataSelection & other )
```

Adds the data selection of *other* to this data selection, and then simplifies this data selection (see [simplify](#)).

15.577.4.15 operator-=() [1/2]

```
QCPDataSelection & QCPDataSelection::operator-= (
    const QCPDataRange & other )
```

Removes all data point indices that are described by *other* from this data selection.

15.577.4.16 operator-=() [2/2]

```
QCPDataSelection & QCPDataSelection::operator-= (
    const QCPDataSelection & other )
```

Removes all data point indices that are described by *other* from this data selection.

15.577.4.17 operator==()

```
bool QCPDataSelection::operator==(
    const QCPDataSelection & other ) const
```

Returns true if this selection is identical (contains the same data ranges with the same begin and end indices) to *other*.

Note that both data selections must be in simplified state (the usual state of the selection, see [simplify](#)) for this operator to return correct results.

15.577.4.18 simplify()

```
void QCPDataSelection::simplify ( )
```

Sorts all data ranges by range begin index in ascending order, and then joins directly adjacent or overlapping ranges. This can reduce the number of individual data ranges in the selection, and prevents possible double-counting when iterating over the data points held by the data ranges.

This method is automatically called when using the addition/subtraction operators. The only case when [simplify](#) is left to the user, is when calling [addDataRange](#), with the parameter *simplify* explicitly set to false.

15.577.4.19 span()

[QCPDataRange](#) [QCPDataSelection::span](#) () const

Returns a [QCPDataRange](#) which spans the entire data selection, including possible intermediate segments which are not part of the original data selection.

15.577.5 Friends And Related Symbol Documentation**15.577.5.1 operator+ [1/4]**

```
const QCPDataSelection operator+ (
    const QCPDataRange & a,
    const QCPDataRange & b ) [friend]
```

Return a [QCPDataSelection](#) with the data points in *a* joined with the data points in *b*. The resulting data selection is already simplified (see [QCPDataSelection::simplify](#)).

15.577.5.2 operator+ [2/4]

```
const QCPDataSelection operator+ (
    const QCPDataRange & a,
    const QCPDataSelection & b ) [friend]
```

Return a [QCPDataSelection](#) with the data points in *a* joined with the data points in *b*. The resulting data selection is already simplified (see [QCPDataSelection::simplify](#)).

15.577.5.3 operator+ [3/4]

```
const QCPDataSelection operator+ (
    const QCPDataSelection & a,
    const QCPDataRange & b ) [friend]
```

Return a [QCPDataSelection](#) with the data points in *a* joined with the data points in *b*. The resulting data selection is already simplified (see [QCPDataSelection::simplify](#)).

15.577.5.4 operator+ [4/4]

```
const QCPDataSelection operator+ (
    const QCPDataSelection & a,
    const QCPDataSelection & b ) [friend]
```

Return a [QCPDataSelection](#) with the data points in *a* joined with the data points in *b*. The resulting data selection is already simplified (see [QCPDataSelection::simplify](#)).

15.577.5.5 operator- [1/4]

```
const QCPDataSelection operator- (
    const QCPDataRange & a,
    const QCPDataRange & b ) [friend]
```

Return a [QCPDataSelection](#) with the data points which are in *a* but not in *b*.

15.577.5.6 operator- [2/4]

```
const QCPDataSelection operator- (
    const QCPDataRange & a,
    const QCPDataSelection & b ) [friend]
```

Return a [QCPDataSelection](#) with the data points which are in *a* but not in *b*.

15.577.5.7 operator- [3/4]

```
const QCPDataSelection operator- (
    const QCPDataSelection & a,
    const QCPDataRange & b ) [friend]
```

Return a [QCPDataSelection](#) with the data points which are in *a* but not in *b*.

15.577.5.8 operator- [4/4]

```
const QCPDataSelection operator- (
    const QCPDataSelection & a,
    const QCPDataSelection & b ) [friend]
```

Return a [QCPDataSelection](#) with the data points which are in *a* but not in *b*.

15.577.5.9 operator<<()

```
QDebug operator<< (
    QDebug d,
    const QCPDataSelection & selection ) [related]
```

Prints *selection* in a human readable format to the qDebug output.

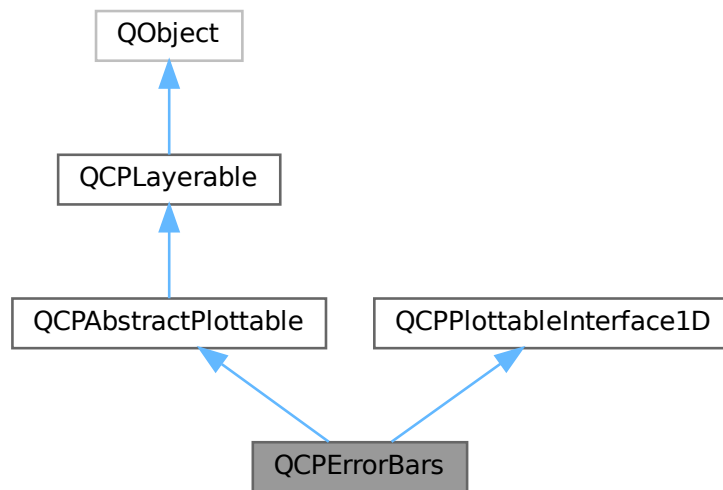
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

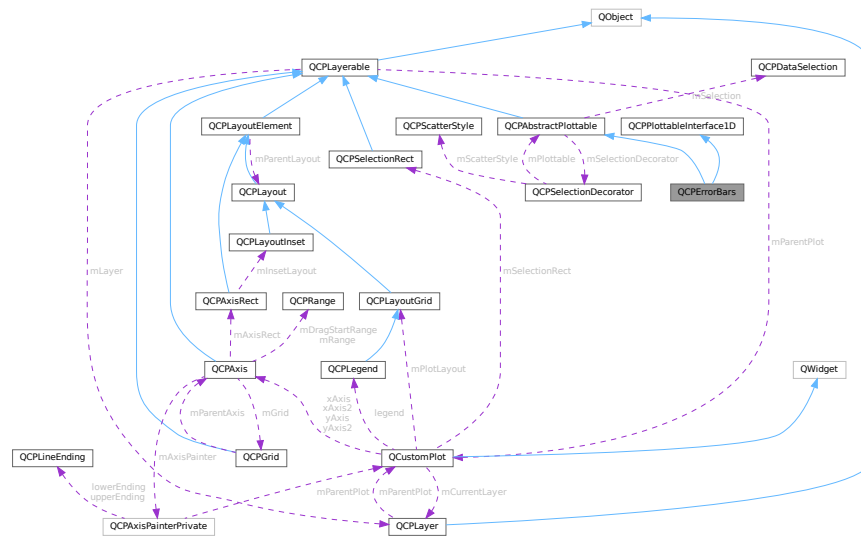
15.578 QCPErrorBars Class Reference

A plottable that adds a set of error bars to other plottables.

Inheritance diagram for QCPErrorBars:



Collaboration diagram for QCPErrors:



Public Types

- enum `ErrorType` { `etKeyError` , `etValueError` }

Public Member Functions

- `QCPEErrorBars (QCPEAxis *keyAxis, QCPEAxis *valueAxis)`
- `QSharedPointer< QCPEErrorBarsDataContainer > data () const`
- `QCPEAbstractPlottable * dataPlottable () const`
- `ErrorType errorType () const`
- `double whiskerWidth () const`
- `double symbolGap () const`
- `void setData (QSharedPointer< QCPEErrorBarsDataContainer > data)`
- `void setData (const QVector< double > &error)`
- `void setData (const QVector< double > &errorMinus, const QVector< double > &errorPlus)`
- `void setDataPlottable (QCPEAbstractPlottable *plottable)`
- `void setErrorType (ErrorType type)`
- `void setWhiskerWidth (double pixels)`
- `void setSymbolGap (double pixels)`
- `void addData (const QVector< double > &error)`
- `void addData (const QVector< double > &errorMinus, const QVector< double > &errorPlus)`
- `void addData (double error)`
- `void addData (double errorMinus, double errorPlus)`
- `virtual int dataCount () const Q_DECL_OVERRIDE`
- `virtual double dataMainKey (int index) const Q_DECL_OVERRIDE`
- `virtual double dataSortKey (int index) const Q_DECL_OVERRIDE`
- `virtual double dataMainValue (int index) const Q_DECL_OVERRIDE`
- `virtual QCPERange dataValueRange (int index) const Q_DECL_OVERRIDE`
- `virtual QPointF dataPixelPosition (int index) const Q_DECL_OVERRIDE`
- `virtual bool sortKeysMainKey () const Q_DECL_OVERRIDE`
- `virtual QCPEDataSelection selectTestRect (const QRectF &rect, bool onlySelectable) const Q_DECL_OVERRIDE`
- `virtual int findBegin (double sortKey, bool expandedRange=true) const Q_DECL_OVERRIDE`
- `virtual int findEnd (double sortKey, bool expandedRange=true) const Q_DECL_OVERRIDE`

- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_[↵](#)
DECL_OVERRIDE
- virtual [QCPPlottableInterface1D](#) * [interface1D](#) () Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractPlottable](#)

- [QCPAbstractPlottable](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- QString **name** () const
- bool **antialiasedFill** () const
- bool **antialiasedScatters** () const
- QPen **pen** () const
- QBrush **brush** () const
- [QCPAxis](#) * **keyAxis** () const
- [QCPAxis](#) * **valueAxis** () const
- [QCP::SelectionType](#) **selectable** () const
- bool **selected** () const
- [QCPDataSelection](#) **selection** () const
- [QCPSelectionDecorator](#) * **selectionDecorator** () const
- void **setName** (const QString &name)
- void **setAntialiasedFill** (bool enabled)
- void **setAntialiasedScatters** (bool enabled)
- void **setPen** (const QPen &pen)
- void **setBrush** (const QBrush &brush)
- void **setKeyAxis** ([QCPAxis](#) *axis)
- void **setValueAxis** ([QCPAxis](#) *axis)
- Q_SLOT void **setSelectable** ([QCP::SelectionType](#) selectable)
- Q_SLOT void **setSelection** ([QCPDataSelection](#) selection)
- void **setSelectionDecorator** ([QCPSelectionDecorator](#) *decorator)
- void **coordsToPixels** (double key, double value, double &x, double &y) const
- const QPointF **coordsToPixels** (double key, double value) const
- void **pixelsToCoords** (double x, double y, double &key, double &value) const
- void **pixelsToCoords** (const QPointF &pixelPos, double &key, double &value) const
- void **rescaleAxes** (bool onlyEnlarge=false) const
- void **rescaleKeyAxis** (bool onlyEnlarge=false) const
- void **rescaleValueAxis** (bool onlyEnlarge=false, bool inKeyRange=false) const
- bool **addToLegend** ([QCPLegend](#) *legend)
- bool **addToLegend** ()
- bool **removeFromLegend** ([QCPLegend](#) *legend) const
- bool **removeFromLegend** () const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const QString &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Protected Member Functions

- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void [drawLegendIcon](#) ([QCPPainter](#) *painter, const [QRectF](#) &rect) const Q_DECL_OVERRIDE
- virtual [QCPRange](#) [getKeyRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#)) const Q_DECL_OVERRIDE
- virtual [QCPRange](#) [getValueRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#), const [QCPRange](#) &inKeyRange=[QCPRange](#)()) const Q_DECL_OVERRIDE
- void [getErrorBarLines](#) ([QCPErrorsDataContainer::const_iterator](#) it, [QVector](#)< [QLineF](#) > &backbones, [QVector](#)< [QLineF](#) > &whiskers) const
- void [getVisibleDataBounds](#) ([QCPErrorsDataContainer::const_iterator](#) &begin, [QCPErrorsDataContainer::const_iterator](#) &end, const [QCPDataRange](#) &rangeRestriction) const
- double [pointDistance](#) (const [QPointF](#) &pixelPoint, [QCPErrorsDataContainer::const_iterator](#) &closestData) const
- void [getDataSegments](#) ([QList](#)< [QCPDataRange](#) > &selectedSegments, [QList](#)< [QCPDataRange](#) > &unselectedSegments) const
- bool [errorBarVisible](#) (int index) const
- bool [rectIntersectsLine](#) (const [QRectF](#) &pixelRect, const [QLineF](#) &line) const

Protected Member Functions inherited from [QCPAbstractPlottable](#)

- virtual [QRect](#) [clipRect](#) () const Q_DECL_OVERRIDE
- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- void [applyFillAntialiasingHint](#) ([QCPPainter](#) *painter) const
- void [applyScattersAntialiasingHint](#) ([QCPPainter](#) *painter) const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot)
- virtual void [mousePressEvent](#) ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void [mouseMoveEvent](#) ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void [mouseReleaseEvent](#) ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void [mouseDoubleClickEvent](#) ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void [wheelEvent](#) ([QWheelEvent](#) *event)
- void [initializeParentPlot](#) ([QCustomPlot](#) *parentPlot)
- void [setParentLayerable](#) ([QCPLayerable](#) *parentLayerable)
- bool [moveToLayer](#) ([QCPLayer](#) *layer, bool prepend)
- void [applyAntialiasingHint](#) ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QSharedPointer](#)< [QCPErrorsDataContainer](#) > **mDataContainer**
- [QPointer](#)< [QCPAbstractPlottable](#) > **mDataPlottable**
- [ErrorType](#) **mErrorType**
- double **mWhiskerWidth**
- double **mSymbolGap**

Protected Attributes inherited from [QCPAbstractPlottable](#)

- `QString mName`
- `bool mAntialiasedFill`
- `bool mAntialiasedScatters`
- `QPen mPen`
- `QBrush mBrush`
- `QPointer< QCPAxis > mKeyAxis`
- `QPointer< QCPAxis > mValueAxis`
- `QCP::SelectionType mSelectable`
- `QCPDataSelection mSelection`
- `QCPSelectionDecorator * mSelectionDecorator`

Protected Attributes inherited from [QCPLayerable](#)

- `bool mVisible`
- `QCustomPlot * mParentPlot`
- `QPointer< QCPLayerable > mParentLayerable`
- `QCPLayer * mLayer`
- `bool mAntialiased`

Friends

- `class QCustomPlot`
- `class QCPLegend`

Additional Inherited Members

Signals inherited from [QCPAbstractPlottable](#)

- `void selectionChanged (bool selected)`
- `void selectionChanged (const QCPDataSelection &selection)`
- `void selectableChanged (QCP::SelectionType selectable)`

Signals inherited from [QCPLayerable](#)

- `void layerChanged (QCPLayer *newLayer)`

15.578.1 Detailed Description

A plottable that adds a set of error bars to other plottables.

The [QCPErrorBars](#) plottable can be attached to other one-dimensional plottables (e.g. [QCPGraph](#), [QCPCurve](#), [QCPBars](#), etc.) and equips them with error bars.

Use [setDataPlottable](#) to define for which plottable the [QCPErrorBars](#) shall display the error bars. The orientation of the error bars can be controlled with [setErrorType](#).

By using [setData](#), you can supply the actual error data, either as symmetric error or plus/minus asymmetric errors. [QCPErrorBars](#) only stores the error data. The absolute key/value position of each error bar will be adopted from the configured data plottable. The error data of the [QCPErrorBars](#) are associated one-to-one via their index to the data points of the data plottable. You can directly access and manipulate the error bar data via [data](#).

[Set](#) either of the plus/minus errors to NaN (`qQNaN()` or `std::numeric_limits<double>::quiet_NaN()`) to not show the respective error bar on the data point at that index.

15.578.2 Changing the appearance

The appearance of the error bars is defined by the pen ([setPen](#)), and the width of the whiskers ([setWhiskerWidth](#)). Further, the error bar backbones may leave a gap around the data point center to prevent that error bars are drawn too close to or even through scatter points. This gap size can be controlled via [setSymbolGap](#).

15.578.3 Member Enumeration Documentation

15.578.3.1 ErrorType

enum [QCPErrors::ErrorType](#)

Defines in which orientation the error bars shall appear. If your data needs both error dimensions, create two [QCPErrors](#) with different [ErrorType](#).

See also

[setErrorType](#)

Enumerator

etKeyError	The errors are for the key dimension (bars appear parallel to the key axis)
etValueError	The errors are for the value dimension (bars appear parallel to the value axis)

15.578.4 Constructor & Destructor Documentation

15.578.4.1 QCPErrors()

```
QCPErrors::QCPErrors (
    QCPAxis * keyAxis,
    QCPAxis * valueAxis ) [explicit]
```

Constructs an error bars plottable which uses *keyAxis* as its key axis ("x") and *valueAxis* as its value axis ("y"). *keyAxis* and *valueAxis* must reside in the same [QCustomPlot](#) instance and not have the same orientation. If either of these restrictions is violated, a corresponding message is printed to the debug output (qDebug), the construction is not aborted, though.

It is also important that the *keyAxis* and *valueAxis* are the same for the error bars plottable and the data plottable that the error bars shall be drawn on ([setDataPlottable](#)).

The created [QCPErrors](#) is automatically registered with the [QCustomPlot](#) instance inferred from *keyAxis*. This [QCustomPlot](#) instance takes ownership of the [QCPErrors](#), so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead.

15.578.5 Member Function Documentation

15.578.5.1 addData() [1/4]

```
void QCPErrors::addData (
    const QVector< double > & error )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds symmetrical error values as specified in *error*. The errors will be associated one-to-one by the data point index to the associated data plottable ([setDataPlottable](#)).

You can directly access and manipulate the error bar data via [data](#).

See also

[setData](#)

15.578.5.2 addData() [2/4]

```
void QCPErrors::addData (
    const QVector< double > & errorMinus,
    const QVector< double > & errorPlus )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds asymmetrical errors as specified in *errorMinus* and *errorPlus*. The errors will be associated one-to-one by the data point index to the associated data plottable ([setDataPlottable](#)).

You can directly access and manipulate the error bar data via [data](#).

See also

[setData](#)

15.578.5.3 addData() [3/4]

```
void QCPErrors::addData (
    double error )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds a single symmetrical error bar as specified in *error*. The errors will be associated one-to-one by the data point index to the associated data plottable ([setDataPlottable](#)).

You can directly access and manipulate the error bar data via [data](#).

See also

[setData](#)

15.578.5.4 addData() [4/4]

```
void QCPErrors::addData (
    double errorMinus,
    double errorPlus )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds a single asymmetrical error bar as specified in *errorMinus* and *errorPlus*. The errors will be associated one-to-one by the data point index to the associated data plottable ([setDataPlottable](#)).

You can directly access and manipulate the error bar data via [data](#).

See also

[setData](#)

15.578.5.5 data()

```
QSharedPointer< QCPErrorsDataContainer > QCPErrors::data ( ) const [inline]
```

Returns a shared pointer to the internal data storage of type `QCPErrorsDataContainer`. You may use it to directly manipulate the error values, which may be more convenient and faster than using the regular [setData](#) methods.

15.578.5.6 dataCount()

```
int QCPErrors::dataCount ( ) const [virtual]
```

Returns the number of data points of the plottable.

Implements [QCPLottableInterface1D](#).

15.578.5.7 dataMainKey()

```
double QCPErrors::dataMainKey (
    int index ) const [virtual]
```

Returns the main key of the data point at the given *index*.

What the main key is, is defined by the plottable's data type. See the [QCPErrorsDataContainer DataType](#) documentation for details about this naming convention.

Implements [QCPLottableInterface1D](#).

15.578.5.8 dataMainValue()

```
double QCPErrors::dataMainValue (
    int index ) const [virtual]
```

Returns the main value of the data point at the given *index*.

What the main value is, is defined by the plottable's data type. See the [QCPErrorsDataContainer DataType](#) documentation for details about this naming convention.

Implements [QCPLottableInterface1D](#).

15.578.5.9 dataPixelPosition()

```
QPointF QCPErrors::dataPixelPosition (
    int index ) const [virtual]
```

Returns the pixel position on the widget surface at which the data point at the given *index* appears.

Usually this corresponds to the point of dataMainKey/ dataMainValue, in pixel coordinates. However, depending on the plottable, this might be a different apparent position than just a coord-to-pixel transform of those values. For example, [QCPBars](#) apparent data values can be shifted depending on their stacking, bar grouping or configured base value.

Implements [QCPPlottableInterface1D](#).

15.578.5.10 dataSortKey()

```
double QCPErrors::dataSortKey (
    int index ) const [virtual]
```

Returns the sort key of the data point at the given *index*.

What the sort key is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implements [QCPPlottableInterface1D](#).

15.578.5.11 dataValueRange()

```
QCPRange QCPErrors::dataValueRange (
    int index ) const [virtual]
```

Returns the value range of the data point at the given *index*.

What the value range is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implements [QCPPlottableInterface1D](#).

15.578.5.12 draw()

```
void QCPErrors::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

15.578.5.13 drawLegendIcon()

```
void QCPErrors::drawLegendIcon (
    QCPPainter * painter,
    const QRectF & rect ) const [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

15.578.5.14 findBegin()

```
int QCPErrors::findBegin (
    double sortKey,
    bool expandedRange = true ) const [virtual]
```

Returns the index of the data point with a (sort-)key that is equal to, just below, or just above *sortKey*. If *expandedRange* is true, the data point just below *sortKey* will be considered, otherwise the one just above.

This can be used in conjunction with [findEnd](#) to iterate over data points within a given key range, including or excluding the bounding data points that are just beyond the specified range.

If *expandedRange* is true but there are no data points below *sortKey*, 0 is returned.

If the container is empty, returns 0 (in that case, [findEnd](#) will also return 0, so a loop using these methods will not iterate over the index 0).

See also

[findEnd](#), [QCPDataContainer::findBegin](#)

Implements [QCPPlottableInterface1D](#).

15.578.5.15 findEnd()

```
int QCPErrorBars::findEnd (
    double sortKey,
    bool expandedRange = true ) const [virtual]
```

Returns the index one after the data point with a (sort-)key that is equal to, just above, or just below *sortKey*. If *expandedRange* is true, the data point just above *sortKey* will be considered, otherwise the one just below.

This can be used in conjunction with [findBegin](#) to iterate over data points within a given key range, including the bounding data points that are just below and above the specified range.

If *expandedRange* is true but there are no data points above *sortKey*, the index just above the highest data point is returned.

If the container is empty, returns 0.

See also

[findBegin](#), [QCPDataContainer::findEnd](#)

Implements [QCPPlottableInterface1D](#).

15.578.5.16 getKeyRange()

```
QCPRange QCPErrorBars::getKeyRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth ) const [protected], [virtual]
```

Returns the coordinate range that all data in this plottable span in the key axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getValueRange](#)

Implements [QCPAbstractPlottable](#).

15.578.5.17 getValueRange()

```
QCPRange QCPErrorBars::getValueRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth,
    const QCPRange & inKeyRange = QCPRange() ) const [protected], [virtual]
```

Returns the coordinate range that the data points in the specified key range (*inKeyRange*) span in the value axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

If *inKeyRange* has both lower and upper bound set to zero (is equal to [QCPRange\(\)](#)), all data points are considered, without any restriction on the keys.

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getKeyRange](#)

Implements [QCPAbstractPlottable](#).

15.578.5.18 interface1D()

```
virtual QCPPlottableInterface1D * QCPErrorBars::interface1D ( ) [inline], [virtual]
```

If this plottable is a one-dimensional plottable, i.e. it implements the [QCPPlottableInterface1D](#), returns the *this* pointer with that type. Otherwise (e.g. in the case of a [QCPColorMap](#)) returns zero.

You can use this method to gain read access to data coordinates while holding a pointer to the abstract base class only.

Reimplemented from [QCPAbstractPlottable](#).

15.578.5.19 selectTest()

```
double QCPErrorBars::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Implements a selectTest specific to this plottable's point geometry.

If *details* is not 0, it will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

\seebaseclassmethod [QCPAbstractPlottable::selectTest](#)

Implements [QCPAbstractPlottable](#).

15.578.5.20 selectTestRect()

```
QCPDataSelection QCPErrorBars::selectTestRect (
    const QRectF & rect,
    bool onlySelectable ) const [virtual]
```

Returns a data selection containing all the data points of this plottable which are contained (or hit by) *rect*. This is used mainly in the selection rect interaction for data selection (data selection mechanism).

If *onlySelectable* is true, an empty [QCPDataSelection](#) is returned if this plottable is not selectable (i.e. if [QCPAbstractPlottable::setSelectable](#) is [QCP::stNone](#)).

Note

rect must be a normalized rect (positive or zero width and height). This is especially important when using the rect of [QCPSelectionRect::accepted](#), which is not necessarily normalized. Use `QRect::normalized()` when passing a rect which might not be normalized.

Implements [QCPPlottableInterface1D](#).

15.578.5.21 setData() [1/3]

```
void QCPErrorBars::setData (
    const QVector< double > & error )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets symmetrical error values as specified in *error*. The errors will be associated one-to-one by the data point index to the associated data plottable ([setDataPlottable](#)).

You can directly access and manipulate the error bar data via [data](#).

See also

[addData](#)

15.578.5.22 setData() [2/3]

```
void QCPErrorBars::setData (
    const QVector< double > & errorMinus,
    const QVector< double > & errorPlus )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets asymmetrical errors as specified in *errorMinus* and *errorPlus*. The errors will be associated one-to-one by the data point index to the associated data plottable ([setDataPlottable](#)).

You can directly access and manipulate the error bar data via [data](#).

See also

[addData](#)

15.578.5.23 setData() [3/3]

```
void QCPErrorBars::setData (
    QSharedPointer< QCPErrorBarsDataContainer > data )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data container with the provided *data* container.

Since a QSharedPointer is used, multiple [QCPErrorBars](#) instances may share the same data container safely. Modifying the data in the container will then affect all [QCPErrorBars](#) instances that share the container. Sharing can be achieved by simply exchanging the data containers wrapped in shared pointers:

If you do not wish to share containers, but create a copy from an existing container, assign the data containers directly:

(This uses different notation compared with other plottables, because the [QCPErrorBars](#) uses a `QVector<QCPErrorBarsData>` as its data container, instead of a [QCPDataContainer](#).)

See also

[addData](#)

15.578.5.24 setDataPlottable()

```
void QCPErrorBars::setDataPlottable (
    QCPAbstractPlottable * plottable )
```

Sets the data plottable to which the error bars will be applied. The error values specified e.g. via [setData](#) will be associated one-to-one by the data point index to the data points of *plottable*. This means that the error bars will adopt the key/value coordinates of the data point with the same index.

The passed *plottable* must be a one-dimensional plottable, i.e. it must implement the [QCPPlottableInterface1D](#). Further, it must not be a [QCPErrorBars](#) instance itself. If either of these restrictions is violated, a corresponding qDebug output is generated, and the data plottable of this [QCPErrorBars](#) instance is set to zero.

For proper display, care must also be taken that the key and value axes of the *plottable* match those configured for this [QCPErrorBars](#) instance.

15.578.5.25 setErrorType()

```
void QCPErrorBars::setErrorType (
    ErrorType type )
```

Sets in which orientation the error bars shall appear on the data points. If your data needs both error dimensions, create two [QCPErrorBars](#) with different *type*.

15.578.5.26 setSymbolGap()

```
void QCPErrorBars::setSymbolGap (
    double pixels )
```

Sets the gap diameter around the data points that will be left out when drawing the error bar backbones. This gap prevents that error bars are drawn too close to or even through scatter points.

15.578.5.27 setWhiskerWidth()

```
void QCPErrorBars::setWhiskerWidth (
    double pixels )
```

Sets the width of the whiskers (the short bars at the end of the actual error bar backbones) to *pixels*.

15.578.5.28 sortKeysMainKey()

```
bool QCPErrorsData::sortKeyIsMainKey ( ) const [virtual]
```

Returns whether the sort key ([dataSortKey](#)) is identical to the main key ([dataMainKey](#)).

What the sort and main keys are, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implements [QCPPlottableInterface1D](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.579 QCPErrorsData Class Reference

Holds the data of one single error bar for [QCPErrorsData](#).

Public Member Functions

- [QCPErrorsData](#) ()
- [QCPErrorsData](#) (double error)
- [QCPErrorsData](#) (double errorMinus, double errorPlus)

Public Attributes

- double **errorMinus**
- double **errorPlus**

15.579.1 Detailed Description

Holds the data of one single error bar for [QCPErrorsData](#).

The stored data is:

- *errorMinus*: how much the error bar extends towards negative coordinates from the data point position
- *errorPlus*: how much the error bar extends towards positive coordinates from the data point position

The container for storing the error bar information is [QCPErrorsDataContainer](#). It is a typedef for [QVector<QCPErrorsData>](#).

See also

[QCPErrorsDataContainer](#)

15.579.2 Constructor & Destructor Documentation**15.579.2.1 QCPErrorsData() [1/3]**

```
QCPErrorsData::QCPErrorsData ( )
```

Constructs an error bar with errors set to zero.

15.579.2.2 QCPErrorsData() [2/3]

```
QCPErrorsData::QCPErrorsData (
    double error ) [explicit]
```

Constructs an error bar with equal *error* in both negative and positive direction.

15.579.2.3 QCPErrorsData() [3/3]

```
QCPErrorsData::QCPErrorsData (
    double errorMinus,
    double errorPlus )
```

Constructs an error bar with negative and positive errors set to *errorMinus* and *errorPlus*, respectively.

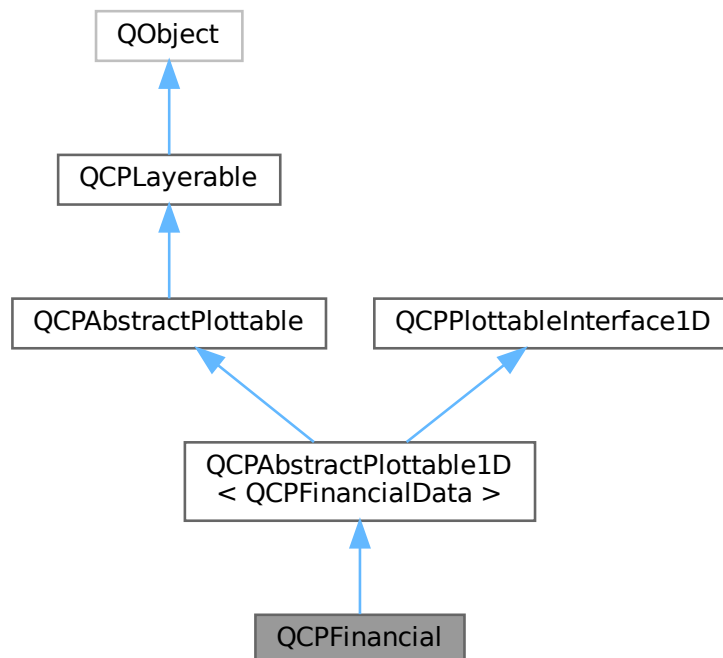
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.580 QCPFinancial Class Reference

A plottable representing a financial stock chart.

Inheritance diagram for QCPFinancial:



- enum **WidthType** { **wtAbsolute** , **wtAxisRectRatio** , **wtPlotCoords** }
- enum **ChartStyle** { **csOhlc** , **csCandlestick** }

- **QCPFinancial** (**QCPAxis** *keyAxis, **QCPAxis** *valueAxis)
- **QSharedPointer**< **QCPFinancialDataContainer** > **data** () const
- **ChartStyle** **chartStyle** () const
- double **width** () const
- **WidthType** **widthType** () const
- bool **twoColored** () const
- **QBrush** **brushPositive** () const
- **QBrush** **brushNegative** () const
- **QPen** **penPositive** () const
- **QPen** **penNegative** () const
- void **setData** (**QSharedPointer**< **QCPFinancialDataContainer** > **data**)
- void **setData** (const **QVector**< double > &keys, const **QVector**< double > &open, const **QVector**< double > &high, const **QVector**< double > &low, const **QVector**< double > &close, bool alreadySorted=false)
- void **setChartStyle** (**ChartStyle** style)
- void **setWidth** (double width)
- void **setWidthType** (**WidthType** widthType)
- void **setTwoColored** (bool twoColored)
- void **setBrushPositive** (const **QBrush** &brush)
- void **setBrushNegative** (const **QBrush** &brush)
- void **setPenPositive** (const **QPen** &pen)
- void **setPenNegative** (const **QPen** &pen)
- void **addData** (const **QVector**< double > &keys, const **QVector**< double > &open, const **QVector**< double > &high, const **QVector**< double > &low, const **QVector**< double > &close, bool alreadySorted=false)
- void **addData** (double key, double open, double high, double low, double close)
- virtual **QCPDataSelection** **selectTestRect** (const **QRectF** &rect, bool onlySelectable) const **Q_DECL_OVERRIDE**
- virtual double **selectTest** (const **QPointF** &pos, bool onlySelectable, **QVariant** *details=nullptr) const **Q_DECL_OVERRIDE**

- virtual [QCPRange getKeyRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#)) const Q_DECL_OVERRIDE
- virtual [QCPRange getValueRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#), const [QCPRange](#) &inKeyRange=[QCPRange](#)()) const Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractPlottable1D](#) < [QCPFinancialData](#) >

- [QCPAbstractPlottable1D](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- virtual int [dataCount](#) () const Q_DECL_OVERRIDE
- virtual double [dataMainKey](#) (int index) const Q_DECL_OVERRIDE
- virtual double [dataSortKey](#) (int index) const Q_DECL_OVERRIDE
- virtual double [dataMainValue](#) (int index) const Q_DECL_OVERRIDE
- virtual [QCPRange dataValueRange](#) (int index) const Q_DECL_OVERRIDE
- virtual [QPointF dataPixelPosition](#) (int index) const Q_DECL_OVERRIDE
- virtual bool [sortKeysMainKey](#) () const Q_DECL_OVERRIDE
- virtual int [findBegin](#) (double sortKey, bool expandedRange=true) const Q_DECL_OVERRIDE
- virtual int [findEnd](#) (double sortKey, bool expandedRange=true) const Q_DECL_OVERRIDE
- virtual [QCPPlottableInterface1D](#) * [interface1D](#) () Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractPlottable](#)

- [QCPAbstractPlottable](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- [QString name](#) () const
- bool [antialiasedFill](#) () const
- bool [antialiasedScatters](#) () const
- [QPen pen](#) () const
- [QBrush brush](#) () const
- [QCPAxis](#) * [keyAxis](#) () const
- [QCPAxis](#) * [valueAxis](#) () const
- [QCP::SelectionType selectable](#) () const
- bool [selected](#) () const
- [QCPDataSelection selection](#) () const
- [QCPSelectionDecorator](#) * [selectionDecorator](#) () const
- void [setName](#) (const [QString](#) &name)
- void [setAntialiasedFill](#) (bool enabled)
- void [setAntialiasedScatters](#) (bool enabled)
- void [setPen](#) (const [QPen](#) &pen)
- void [setBrush](#) (const [QBrush](#) &brush)
- void [setKeyAxis](#) ([QCPAxis](#) *axis)
- void [setValueAxis](#) ([QCPAxis](#) *axis)
- Q_SLOT void [setSelectable](#) ([QCP::SelectionType](#) selectable)
- Q_SLOT void [setSelection](#) ([QCPDataSelection](#) selection)
- void [setSelectionDecorator](#) ([QCPSelectionDecorator](#) *decorator)
- void [coordsToPixels](#) (double key, double value, double &x, double &y) const
- const [QPointF coordsToPixels](#) (double key, double value) const
- void [pixelsToCoords](#) (double x, double y, double &key, double &value) const
- void [pixelsToCoords](#) (const [QPointF](#) &pixelPos, double &key, double &value) const
- void [rescaleAxes](#) (bool onlyEnlarge=false) const
- void [rescaleKeyAxis](#) (bool onlyEnlarge=false) const
- void [rescaleValueAxis](#) (bool onlyEnlarge=false, bool inKeyRange=false) const
- bool [addToLegend](#) ([QCPLegend](#) *legend)
- bool [addToLegend](#) ()
- bool [removeFromLegend](#) ([QCPLegend](#) *legend) const
- bool [removeFromLegend](#) () const

Public Member Functions inherited from QCPLayerable

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const [QString](#) &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Static Public Member Functions

- static [QCPFinancialDataContainer](#) **timeSeriesToOhlc** (const [QVector](#)< double > &time, const [QVector](#)< double > &value, double timeBinSize, double timeBinOffset=0)

Protected Member Functions

- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void **drawLegendIcon** ([QCPPainter](#) *painter, const [QRectF](#) &rect) const Q_DECL_OVERRIDE
- void **drawOhlcPlot** ([QCPPainter](#) *painter, const [QCPFinancialDataContainer](#)::const_iterator &begin, const [QCPFinancialDataContainer](#)::const_iterator &end, bool isSelected)
- void **drawCandlestickPlot** ([QCPPainter](#) *painter, const [QCPFinancialDataContainer](#)::const_iterator &begin, const [QCPFinancialDataContainer](#)::const_iterator &end, bool isSelected)
- double **getPixelWidth** (double key, double keyPixel) const
- double **ohlcSelectTest** (const [QPointF](#) &pos, const [QCPFinancialDataContainer](#)::const_iterator &begin, const [QCPFinancialDataContainer](#)::const_iterator &end, [QCPFinancialDataContainer](#)::const_iterator &closestDataPoint) const
- double **candlestickSelectTest** (const [QPointF](#) &pos, const [QCPFinancialDataContainer](#)::const_iterator &begin, const [QCPFinancialDataContainer](#)::const_iterator &end, [QCPFinancialDataContainer](#)::const_iterator &closestDataPoint) const
- void **getVisibleDataBounds** ([QCPFinancialDataContainer](#)::const_iterator &begin, [QCPFinancialDataContainer](#)::const_iterator &end) const
- [QRectF](#) **selectionHitBox** ([QCPFinancialDataContainer](#)::const_iterator it) const

Protected Member Functions inherited from QCPAbstractPlottable1D< QCPFinancialData >

- void **getDataSegments** ([QList](#)< [QCPDataRange](#) > &selectedSegments, [QList](#)< [QCPDataRange](#) > &unselectedSegments) const
- void **drawPolyline** ([QCPPainter](#) *painter, const [QVector](#)< [QPointF](#) > &lineData) const

Protected Member Functions inherited from QCPAbstractPlottable

- virtual [QRect](#) **clipRect** () const Q_DECL_OVERRIDE
- virtual [QCP::Interaction](#) **selectionCategory** () const Q_DECL_OVERRIDE
- void **applyDefaultAntialiasingHint** ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void **selectEvent** ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void **deselectEvent** (bool *selectionStateChanged) Q_DECL_OVERRIDE
- void **applyFillAntialiasingHint** ([QCPPainter](#) *painter) const
- void **applyScattersAntialiasingHint** ([QCPPainter](#) *painter) const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual void **mousePressEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **mouseMoveEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseReleaseEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseDoubleClickEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **wheelEvent** ([QWheelEvent](#) *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [ChartStyle](#) **mChartStyle**
- double **mWidth**
- [WidthType](#) **mWidthType**
- bool **mTwoColored**
- [QBrush](#) **mBrushPositive**
- [QBrush](#) **mBrushNegative**
- [QPen](#) **mPenPositive**
- [QPen](#) **mPenNegative**

Protected Attributes inherited from [QCPAbstractPlottable1D](#)< [QCPFinancialData](#) >

- [QSharedPointer](#)< [QCPDataContainer](#)< [QCPFinancialData](#) > > **mDataContainer**

Protected Attributes inherited from [QCPAbstractPlottable](#)

- [QString](#) **mName**
- bool **mAntialiasedFill**
- bool **mAntialiasedScatters**
- [QPen](#) **mPen**
- [QBrush](#) **mBrush**
- [QPointer](#)< [QCPAxis](#) > **mKeyAxis**
- [QPointer](#)< [QCPAxis](#) > **mValueAxis**
- [QCP::SelectionType](#) **mSelectable**
- [QCPDataSelection](#) **mSelection**
- [QCPSelectionDecorator](#) * **mSelectionDecorator**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- [QPointer](#)< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCustomPlot](#)
- class [QCPLegend](#)

Additional Inherited Members

Signals inherited from [QCPAbstractPlottable](#)

- void [selectionChanged](#) (bool [selected](#))
- void [selectionChanged](#) (const [QCPDataSelection](#) &[selection](#))
- void [selectableChanged](#) ([QCP::SelectionType](#) [selectable](#))

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *[newLayer](#))

15.580.1 Detailed Description

A plottable representing a financial stock chart.

This plottable represents time series data binned to certain intervals, mainly used for stock charts. The two common representations OHLC (Open-High-Low-Close) bars and Candlesticks can be set via [setChartStyle](#).

The data is passed via [setData](#) as a set of open/high/low/close values at certain keys (typically times). This means the data must be already binned appropriately. If data is only available as a series of values (e.g. *price* against *time*), you can use the static convenience function [timeSeriesToOhlc](#) to generate binned OHLC-data which can then be passed to [setData](#).

The width of the OHLC bars/candlesticks can be controlled with [setWidth](#) and [setWidthType](#). A typical choice is to set the width type to [wtPlotCoords](#) (the default) and the width to (or slightly less than) one time bin interval width.

15.580.2 Changing the appearance

Charts can be either single- or two-colored ([setTwoColored](#)). If set to be single-colored, lines are drawn with the plottable's pen ([setPen](#)) and fills with the brush ([setBrush](#)).

If set to two-colored, positive changes of the value during an interval ($close \geq open$) are represented with a different pen and brush than negative changes ($close < open$). These can be configured with [setPenPositive](#), [setPenNegative](#), [setBrushPositive](#), and [setBrushNegative](#). In two-colored mode, the normal plottable pen/brush is ignored. Upon selection however, the normal selected pen/brush (provided by the [selectionDecorator](#)) is used, irrespective of whether the chart is single- or two-colored.

15.580.3 Usage

Like all data representing objects in [QCustomPlot](#), the [QCPFinancial](#) is a plottable ([QCPAbstractPlottable](#)). So the plottable-interface of [QCustomPlot](#) applies ([QCustomPlot::plottable](#), [QCustomPlot::removePlottable](#), etc.)

Usually, you first create an instance:

which registers it with the [QCustomPlot](#) instance of the passed axes. Note that this [QCustomPlot](#) instance takes ownership of the plottable, so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead. The newly created plottable can be modified, e.g.:

Here we have used the static helper method [timeSeriesToOhlc](#), to turn a time-price data series into a 24-hour binned open-high-low-close data series as [QCPFinancial](#) uses.

15.580.4 Member Enumeration Documentation

15.580.4.1 ChartStyle

enum [QCPFinancial::ChartStyle](#)

Defines the possible representations of OHLC data in the plot.

See also

[setChartStyle](#)

Enumerator

csOhlc	Open-High-Low-Close bar representation.
csCandlestick	Candlestick representation.

15.580.4.2 WidthType

enum [QCPFinancial::WidthType](#)

Defines the ways the width of the financial bar can be specified. Thus it defines what the number passed to [setWidth](#) actually means.

See also

[setWidthType](#), [setWidth](#)

Enumerator

<code>wtAbsolute</code>	width is in absolute pixels
<code>wtAxisRectRatio</code>	width is given by a fraction of the axis rect size
<code>wtPlotCoords</code>	width is in key coordinates and thus scales with the key axis range

15.580.5 Constructor & Destructor Documentation

15.580.5.1 QCPFinancial()

```
QCPFinancial::QCPFinancial (
    QCPAxis * keyAxis,
    QCPAxis * valueAxis ) [explicit]
```

Constructs a financial chart which uses *keyAxis* as its key axis ("x") and *valueAxis* as its value axis ("y"). *keyAxis* and *valueAxis* must reside in the same [QCustomPlot](#) instance and not have the same orientation. If either of these restrictions is violated, a corresponding message is printed to the debug output (qDebug), the construction is not aborted, though.

The created [QCPFinancial](#) is automatically registered with the [QCustomPlot](#) instance inferred from *keyAxis*. This [QCustomPlot](#) instance takes ownership of the [QCPFinancial](#), so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead.

15.580.6 Member Function Documentation

15.580.6.1 addData() [1/2]

```
void QCPFinancial::addData (
    const QVector< double > & keys,
    const QVector< double > & open,
    const QVector< double > & high,
    const QVector< double > & low,
    const QVector< double > & close,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided points in *keys*, *open*, *high*, *low* and *close* to the current data. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *keys* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

See also

[timeSeriesToOhlc](#)

15.580.6.2 addData() [2/2]

```
void QCPFinancial::addData (
    double key,
```

```
double open,
double high,
double low,
double close )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided data point as *key*, *open*, *high*, *low* and *close* to the current data.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

See also

[timeSeriesToOhlc](#)

15.580.6.3 data()

```
QCPFinancialDataContainer * QCPFinancial::data ( ) const [inline]
```

Returns a pointer to the internal data storage of type `QCPFinancialDataContainer`. You may use it to directly manipulate the data, which may be more convenient and faster than using the regular [setData](#) or [addData](#) methods, in certain situations.

15.580.6.4 draw()

```
void QCPFinancial::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

15.580.6.5 drawLegendIcon()

```
void QCPFinancial::drawLegendIcon (
    QCPPainter * painter,
    const QRectF & rect ) const [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

15.580.6.6 getKeyRange()

```
QCPRange QCPFinancial::getKeyRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth ) const [virtual]
```

Returns the coordinate range that all data in this plottable span in the key axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getValueRange](#)

Implements [QCPAbstractPlottable](#).

15.580.6.7 getValueRange()

```
QCPRange QCPFinancial::getValueRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth,
    const QCPRange & inKeyRange = QCPRange() ) const [virtual]
```

Returns the coordinate range that the data points in the specified key range (*inKeyRange*) span in the value axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

If *inKeyRange* has both lower and upper bound set to zero (is equal to [QCPRange\(\)](#)), all data points are considered, without any restriction on the keys.

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getKeyRange](#)

Implements [QCPAbstractPlottable](#).

15.580.6.8 selectTest()

```
double QCPFinancial::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Implements a selectTest specific to this plottable's point geometry.

If *details* is not 0, it will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

See baseclass method [QCPAbstractPlottable::selectTest](#)

Reimplemented from [QCPAbstractPlottable1D< QCPFinancialData >](#).

15.580.6.9 selectTestRect()

```
QCPDataSelection QCPFinancial::selectTestRect (
    const QRectF & rect,
    bool onlySelectable ) const [virtual]
```

Returns a data selection containing all the data points of this plottable which are contained (or hit by) *rect*. This is used mainly in the selection rect interaction for data selection (data selection mechanism).

If *onlySelectable* is true, an empty [QCPDataSelection](#) is returned if this plottable is not selectable (i.e. if [QCPAbstractPlottable::setSelectable](#) is [QCP::stNone](#)).

Note

rect must be a normalized rect (positive or zero width and height). This is especially important when using the rect of [QCPSelectionRect::accepted](#), which is not necessarily normalized. Use [QRect::normalized\(\)](#) when passing a rect which might not be normalized.

Reimplemented from [QCPAbstractPlottable1D< QCPFinancialData >](#).

15.580.6.10 setBrushNegative()

```
void QCPFinancial::setBrushNegative (
    const QBrush & brush )
```

If [setTwoColored](#) is set to true, this function controls the brush that is used to draw fills of data points with a negative trend (i.e. bars/candlesticks with close < open).

If *twoColored* is false, the normal plottable's pen and brush are used ([setPen](#), [setBrush](#)).

See also

[setBrushPositive](#), [setPenNegative](#), [setPenPositive](#)

15.580.6.11 setBrushPositive()

```
void QCPFinancial::setBrushPositive (
    const QBrush & brush )
```

If [setTwoColored](#) is set to true, this function controls the brush that is used to draw fills of data points with a positive trend (i.e. bars/candlesticks with close >= open).

If *twoColored* is false, the normal plottable's pen and brush are used ([setPen](#), [setBrush](#)).

See also

[setBrushNegative](#), [setPenPositive](#), [setPenNegative](#)

15.580.6.12 setChartStyle()

```
void QCPFinancial::setChartStyle (
    QCPFinancial::ChartStyle style )
```

Sets which representation style shall be used to display the OHLC data.

15.580.6.13 setData() [1/2]

```
void QCPFinancial::setData (
    const QVector< double > & keys,
    const QVector< double > & open,
    const QVector< double > & high,
    const QVector< double > & low,
    const QVector< double > & close,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data with the provided points in *keys*, *open*, *high*, *low* and *close*. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *keys* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

See also

[addData](#), [timeSeriesToOhlc](#)

15.580.6.14 setData() [2/2]

```
void QCPFinancial::setData (
    QSharedPointer< QCPFinancialDataContainer > data )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data container with the provided *data* container.

Since a QSharedPointer is used, multiple QCPFinancials may share the same data container safely. Modifying the data in the container will then affect all financials that share the container. Sharing can be achieved by simply exchanging the data containers wrapped in shared pointers:

If you do not wish to share containers, but create a copy from an existing container, rather use the [QCPDataContainer<DataType>::set](#) method on the financial's data container directly:

See also

[addData](#), [timeSeriesToOhlc](#)

15.580.6.15 setPenNegative()

```
void QCPFinancial::setPenNegative (
    const QPen & pen )
```

If [setTwoColored](#) is set to true, this function controls the pen that is used to draw outlines of data points with a negative trend (i.e. bars/candlesticks with close < open).

If *twoColored* is false, the normal plottable's pen and brush are used ([setPen](#), [setBrush](#)).

See also

[setPenPositive](#), [setBrushNegative](#), [setBrushPositive](#)

15.580.6.16 setPenPositive()

```
void QCPFinancial::setPenPositive (
    const QPen & pen )
```

If [setTwoColored](#) is set to true, this function controls the pen that is used to draw outlines of data points with a positive trend (i.e. bars/candlesticks with close $\geq$ open).

If *twoColored* is false, the normal plottable's pen and brush are used ([setPen](#), [setBrush](#)).

See also

[setPenNegative](#), [setBrushPositive](#), [setBrushNegative](#)

15.580.6.17 setTwoColored()

```
void QCPFinancial::setTwoColored (
    bool twoColored )
```

Sets whether this chart shall contrast positive from negative trends per data point by using two separate colors to draw the respective bars/candlesticks.

If *twoColored* is false, the normal plottable's pen and brush are used ([setPen](#), [setBrush](#)).

See also

[setPenPositive](#), [setPenNegative](#), [setBrushPositive](#), [setBrushNegative](#)

15.580.6.18 setWidth()

```
void QCPFinancial::setWidth (
    double width )
```

Sets the width of the individual bars/candlesticks to *width* in plot key coordinates.

A typical choice is to set it to (or slightly less than) one bin interval width.

15.580.6.19 setWidthType()

```
void QCPFinancial::setWidthType (
    QCPFinancial::WidthType widthType )
```

Sets how the width of the financial bars is defined. See the documentation of [WidthType](#) for an explanation of the possible values for *widthType*.

The default value is [wtPlotCoords](#).

See also

[setWidth](#)

15.580.6.20 timeSeriesToOhlc()

```
QCPFinancialDataContainer QCPFinancial::timeSeriesToOhlc (
    const QVector< double > & time,
    const QVector< double > & value,
    double timeBinSize,
    double timeBinOffset = 0 ) [static]
```

A convenience function that converts time series data (*value* against *time*) to OHLC binned data points. The return value can then be passed on to `QCPFinancialDataContainer::set(const QCPFinancialDataContainer&)`.

The size of the bins can be controlled with *timeBinSize* in the same units as *time* is given. For example, if the unit of *time* is seconds and single OHLC/Candlesticks should span an hour each, set *timeBinSize* to 3600.

timeBinOffset allows to control precisely at what *time* coordinate a bin should start. The value passed as *timeBinOffset* doesn't need to be in the range encompassed by the *time* keys. It merely defines the mathematical offset/phase of the bins that will be used to process the data.

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.581 QCPFinancialData Class Reference

Holds the data of one single data point for [QCPFinancial](#).

Public Member Functions

- [QCPFinancialData](#) ()
- [QCPFinancialData](#) (double key, double open, double high, double low, double close)
- double [sortKey](#) () const
- double [mainKey](#) () const
- double [mainValue](#) () const
- [QCPRange](#) [valueRange](#) () const

Static Public Member Functions

- static [QCPFinancialData](#) [fromSortKey](#) (double [sortKey](#))
- static bool [sortKeyIsMainKey](#) ()

Public Attributes

- double **key**
- double **open**
- double **high**
- double **low**
- double **close**

15.581.1 Detailed Description

Holds the data of one single data point for [QCPFinancial](#).

The stored data is:

- *key*: coordinate on the key axis of this data point (this is the *mainKey* and the *sortKey*)
- *open*: The opening value at the data point (this is the *mainValue*)
- *high*: The high/maximum value at the data point
- *low*: The low/minimum value at the data point
- *close*: The closing value at the data point

The container for storing multiple data points is [QCPFinancialDataContainer](#). It is a typedef for [QCPDataContainer](#) with [QCPFinancialData](#) as the `DataType` template parameter. See the documentation there for an explanation regarding the data type's generic methods.

See also

[QCPFinancialDataContainer](#)

15.581.2 Constructor & Destructor Documentation

15.581.2.1 QCPFinancialData() [1/2]

```
QCPFinancialData::QCPFinancialData ( )
```

Constructs a data point with key and all values set to zero.

15.581.2.2 QCPFinancialData() [2/2]

```
QCPFinancialData::QCPFinancialData (
    double key,
    double open,
    double high,
    double low,
    double close )
```

Constructs a data point with the specified *key* and OHLC values.

15.581.3 Member Function Documentation

15.581.3.1 fromSortKey()

```
static QCPFinancialData QCPFinancialData::fromSortKey (
    double sortKey ) [inline], [static]
```

Returns a data point with the specified *sortKey*. All other members are set to zero.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

15.581.3.2 mainKey()

```
double QCPFinancialData::mainKey ( ) const [inline]
```

Returns the *key* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

15.581.3.3 mainValue()

```
double QCPFinancialData::mainValue ( ) const [inline]
```

Returns the *open* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

15.581.3.4 sortKey()

```
double QCPFinancialData::sortKey ( ) const [inline]
```

Returns the *key* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

15.581.3.5 sortKeysMainKey()

```
static static bool QCPFinancialData::sortKeyIsMainKey ( ) [inline], [static]
```

Since the member *key* is both the data point key coordinate and the data ordering parameter, this method returns true.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

15.581.3.6 valueRange()

```
QCPRange QCPFinancialData::valueRange ( ) const [inline]
```

Returns a [QCPRange](#) spanning from the *low* to the *high* value of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

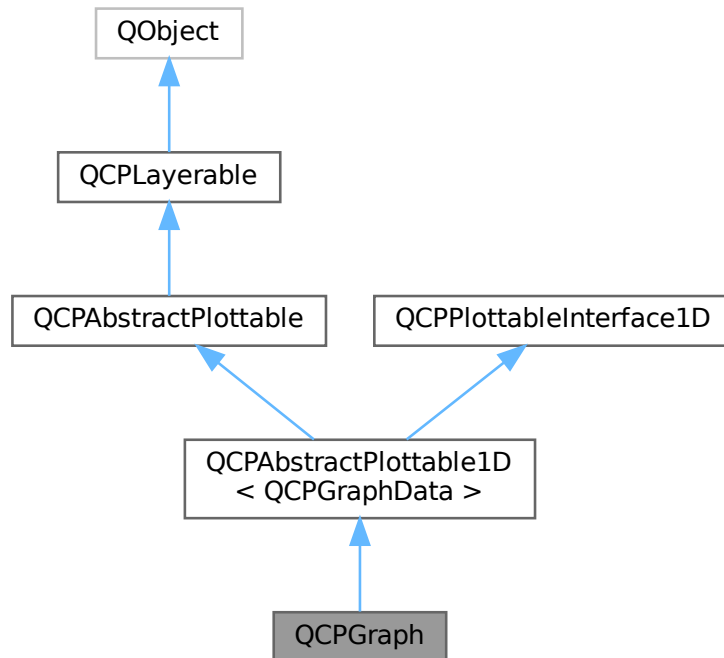
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

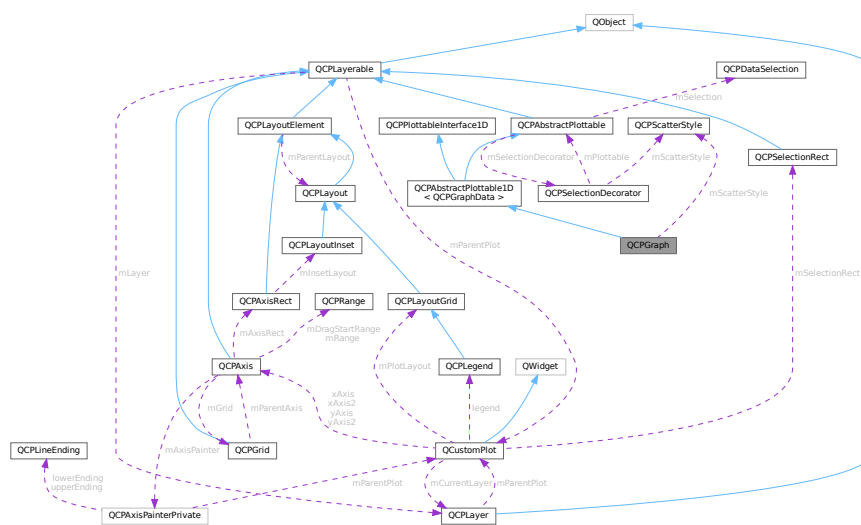
15.582 QCPGraph Class Reference

A plottable representing a graph in a plot.

Inheritance diagram for QCPGraph:



Collaboration diagram for QCPGraph:



Public Types

- enum [LineStyle](#) {
[IsNone](#) , [IsLine](#) , [IsStepLeft](#) , [IsStepRight](#) ,
[IsStepCenter](#) , [IsImpulse](#) }

Public Member Functions

- [QCPGraph](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- QSharedPointer< [QCPGraphDataContainer](#) > [data](#) () const
- [LineStyle](#) [lineStyle](#) () const
- [QCPScatterStyle](#) [scatterStyle](#) () const
- int [scatterSkip](#) () const
- [QCPGraph](#) * [channelFillGraph](#) () const
- bool [adaptiveSampling](#) () const
- void [setData](#) (QSharedPointer< [QCPGraphDataContainer](#) > [data](#))
- void [setData](#) (const QVector< double > &keys, const QVector< double > &values, bool alreadySorted=false)
- void [setLineStyle](#) ([LineStyle](#) ls)
- void [setScatterStyle](#) (const [QCPScatterStyle](#) &style)
- void [setScatterSkip](#) (int skip)
- void [setChannelFillGraph](#) ([QCPGraph](#) *targetGraph)
- void [setAdaptiveSampling](#) (bool enabled)
- void [addData](#) (const QVector< double > &keys, const QVector< double > &values, bool alreadySorted=false)
- void [addData](#) (double key, double value)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE
- virtual [QCPRange](#) [getKeyRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#)) const Q_DECL_OVERRIDE
- virtual [QCPRange](#) [getValueRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#), const [QCPRange](#) &inKeyRange=[QCPRange](#)()) const Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractPlottable1D](#)< [QCPGraphData](#) >

- [QCPAbstractPlottable1D](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- virtual int [dataCount](#) () const Q_DECL_OVERRIDE
- virtual double [dataMainKey](#) (int index) const Q_DECL_OVERRIDE
- virtual double [dataSortKey](#) (int index) const Q_DECL_OVERRIDE
- virtual double [dataMainValue](#) (int index) const Q_DECL_OVERRIDE
- virtual [QCPRange](#) [dataValueRange](#) (int index) const Q_DECL_OVERRIDE
- virtual QPointF [dataPixelPosition](#) (int index) const Q_DECL_OVERRIDE
- virtual bool [sortKeysIsMainKey](#) () const Q_DECL_OVERRIDE
- virtual [QCPDataSelection](#) [selectTestRect](#) (const QRectF &rect, bool onlySelectable) const Q_DECL_OVERRIDE
- virtual int [findBegin](#) (double sortKey, bool expandedRange=true) const Q_DECL_OVERRIDE
- virtual int [findEnd](#) (double sortKey, bool expandedRange=true) const Q_DECL_OVERRIDE
- virtual [QCPPlottableInterface1D](#) * [interface1D](#) () Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractPlottable](#)

- [QCPAbstractPlottable](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- QString [name](#) () const
- bool [antialiasedFill](#) () const
- bool [antialiasedScatters](#) () const
- QPen [pen](#) () const
- QBrush [brush](#) () const

- [QCPAxis](#) * **keyAxis** () const
- [QCPAxis](#) * **valueAxis** () const
- [QCP::SelectionType](#) **selectable** () const
- bool **selected** () const
- [QCPDataSelection](#) **selection** () const
- [QCPSelectionDecorator](#) * **selectionDecorator** () const
- void **setName** (const QString &name)
- void **setAntialiasedFill** (bool enabled)
- void **setAntialiasedScatters** (bool enabled)
- void **setPen** (const QPen &pen)
- void **setBrush** (const QBrush &brush)
- void **setKeyAxis** ([QCPAxis](#) *axis)
- void **setValueAxis** ([QCPAxis](#) *axis)
- Q_SLOT void **setSelectable** ([QCP::SelectionType](#) selectable)
- Q_SLOT void **setSelection** ([QCPDataSelection](#) selection)
- void **setSelectionDecorator** ([QCPSelectionDecorator](#) *decorator)
- void **coordsToPixels** (double key, double value, double &x, double &y) const
- const QPointF **coordsToPixels** (double key, double value) const
- void **pixelsToCoords** (double x, double y, double &key, double &value) const
- void **pixelsToCoords** (const QPointF &pixelPos, double &key, double &value) const
- void **rescaleAxes** (bool onlyEnlarge=false) const
- void **rescaleKeyAxis** (bool onlyEnlarge=false) const
- void **rescaleValueAxis** (bool onlyEnlarge=false, bool inKeyRange=false) const
- bool **addToLegend** ([QCPLegend](#) *legend)
- bool **addToLegend** ()
- bool **removeFromLegend** ([QCPLegend](#) *legend) const
- bool **removeFromLegend** () const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const QString &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Protected Member Functions

- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void **drawLegendIcon** ([QCPPainter](#) *painter, const QRectF &rect) const Q_DECL_OVERRIDE
- virtual void **drawFill** ([QCPPainter](#) *painter, QVector< QPointF > *lines) const
- virtual void **drawScatterPlot** ([QCPPainter](#) *painter, const QVector< QPointF > &scatters, const [QCPScatterStyle](#) &style) const
- virtual void **drawLinePlot** ([QCPPainter](#) *painter, const QVector< QPointF > &lines) const
- virtual void **drawImpulsePlot** ([QCPPainter](#) *painter, const QVector< QPointF > &lines) const
- virtual void **getOptimizedLineData** (QVector< [QCPGraphData](#) > *lineData, const [QCPGraphData](#)↔ Container::const_iterator &begin, const [QCPGraphData](#)↔ Container::const_iterator &end) const
- virtual void **getOptimizedScatterData** (QVector< [QCPGraphData](#) > *scatterData, [QCPGraphData](#)↔ Container::const_iterator begin, [QCPGraphData](#)↔ Container::const_iterator end) const

- void [setVisibleDataBounds](#) (QCPGraphDataContainer::const_iterator &begin, QCPGraphDataContainer::const_iterator &end, const [QCPDataRange](#) &rangeRestriction) const
- void **getLines** (QVector< QPointF > *lines, const [QCPDataRange](#) &dataRange) const
- void **getScatters** (QVector< QPointF > *scatters, const [QCPDataRange](#) &dataRange) const
- QVector< QPointF > **dataToLines** (const QVector< [QCPGraphData](#) > &data) const
- QVector< QPointF > **dataToStepLeftLines** (const QVector< [QCPGraphData](#) > &data) const
- QVector< QPointF > **dataToStepRightLines** (const QVector< [QCPGraphData](#) > &data) const
- QVector< QPointF > **dataToStepCenterLines** (const QVector< [QCPGraphData](#) > &data) const
- QVector< QPointF > **dataToImpulseLines** (const QVector< [QCPGraphData](#) > &data) const
- QVector< [QCPDataRange](#) > **getNonNaNSegments** (const QVector< QPointF > *lineData, Qt::Orientation keyOrientation) const
- QVector< QPair< [QCPDataRange](#), [QCPDataRange](#) > > **getOverlappingSegments** (QVector< [QCPDataRange](#) > thisSegments, const QVector< QPointF > *thisData, QVector< [QCPDataRange](#) > otherSegments, const QVector< QPointF > *otherData) const
- bool **segmentsIntersect** (double aLower, double aUpper, double bLower, double bUpper, int &bPrecedence) const
- QPointF **getFillBasePoint** (QPointF matchingDataPoint) const
- const QPolygonF **getFillPolygon** (const QVector< QPointF > *lineData, [QCPDataRange](#) segment) const
- const QPolygonF **getChannelFillPolygon** (const QVector< QPointF > *thisData, [QCPDataRange](#) thisSegment, const QVector< QPointF > *otherData, [QCPDataRange](#) otherSegment) const
- int **findIndexBelowX** (const QVector< QPointF > *data, double x) const
- int **findIndexAboveX** (const QVector< QPointF > *data, double x) const
- int **findIndexBelowY** (const QVector< QPointF > *data, double y) const
- int **findIndexAboveY** (const QVector< QPointF > *data, double y) const
- double **pointDistance** (const QPointF &pixelPoint, QCPGraphDataContainer::const_iterator &closestData) const

Protected Member Functions inherited from [QCPAbstractPlottable1D< QCPGraphData >](#)

- void [getDataSegments](#) (QList< [QCPDataRange](#) > &selectedSegments, QList< [QCPDataRange](#) > &unselectedSegments) const
- void [drawPolyline](#) (QCPPainter *painter, const QVector< QPointF > &lineData) const

Protected Member Functions inherited from [QCPAbstractPlottable](#)

- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- void [applyDefaultAntialiasingHint](#) (QCPPainter *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- void [applyFillAntialiasingHint](#) (QCPPainter *painter) const
- void [applyScattersAntialiasingHint](#) (QCPPainter *painter) const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** (QCustomPlot *parentPlot)
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void **initializeParentPlot** (QCustomPlot *parentPlot)
- void **setParentLayerable** (QCPLayerable *parentLayerable)
- bool **moveToLayer** (QCPLayer *layer, bool prepend)
- void **applyAntialiasingHint** (QCPPainter *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [LineStyle](#) **mLineStyle**
- [QCPScatterStyle](#) **mScatterStyle**
- int **mScatterSkip**
- [QPointer< QCPGraph >](#) **mChannelFillGraph**
- bool **mAdaptiveSampling**

Protected Attributes inherited from [QCPAbstractPlottable1D< QCPGraphData >](#)

- [QSharedPointer< QCPDataContainer< QCPGraphData > >](#) **mDataContainer**

Protected Attributes inherited from [QCPAbstractPlottable](#)

- [QString](#) **mName**
- bool **mAntialiasedFill**
- bool **mAntialiasedScatters**
- [QPen](#) **mPen**
- [QBrush](#) **mBrush**
- [QPointer< QCPAxis >](#) **mKeyAxis**
- [QPointer< QCPAxis >](#) **mValueAxis**
- [QCP::SelectionType](#) **mSelectable**
- [QCPDataSelection](#) **mSelection**
- [QCPSelectionDecorator](#) * **mSelectionDecorator**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- [QPointer< QCPLayerable >](#) **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCustomPlot](#)
- class [QCPLegend](#)

Additional Inherited Members

Signals inherited from [QCPAbstractPlottable](#)

- void [selectionChanged](#) (bool [selected](#))
- void [selectionChanged](#) (const [QCPDataSelection](#) &[selection](#))
- void [selectableChanged](#) ([QCP::SelectionType](#) [selectable](#))

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *[newLayer](#))

15.582.1 Detailed Description

A plottable representing a graph in a plot.

Usually you create new graphs by calling [QCustomPlot::addGraph](#). The resulting instance can be accessed via [QCustomPlot::graph](#).

To plot data, assign it with the [setData](#) or [addData](#) functions. Alternatively, you can also access and modify the data via the [data](#) method, which returns a pointer to the internal QCPGraphDataContainer.

Graphs are used to display single-valued data. Single-valued means that there should only be one data point per unique key coordinate. In other words, the graph can't have *loops*. If you do want to plot non-single-valued curves, rather use the [QCPCurve](#) plottable.

Gaps in the graph line can be created by adding data points with NaN as value (`qQNaN()` or `std::numeric_limits<double>::quiet_NaN()`) in between the two data points that shall be separated.

15.582.2 Changing the appearance

The appearance of the graph is mainly determined by the line style, scatter style, brush and pen of the graph ([setLineStyle](#), [setScatterStyle](#), [setBrush](#), [setPen](#)).

15.582.2.1 Filling under or between graphs

[QCPGraph](#) knows two types of fills: Normal graph fills towards the zero-value-line parallel to the key axis of the graph, and fills between two graphs, called channel fills. To enable a fill, just set a brush with [setBrush](#) which is neither Qt::NoBrush nor fully transparent.

By default, a normal fill towards the zero-value-line will be drawn. To set up a channel fill between this graph and another one, call [setChannelFillGraph](#) with the other graph as parameter.

See also

[QCustomPlot::addGraph](#), [QCustomPlot::graph](#)

15.582.3 Member Enumeration Documentation

15.582.3.1 LineStyle

enum [QCPGraph::LineStyle](#)

Defines how the graph's line is represented visually in the plot. The line is drawn with the current pen of the graph ([setPen](#)).

See also

[setLineStyle](#)

Enumerator

IsNone	with symbols according to the scatter style, see setScatterStyle) data points are not connected with any lines (e.g. data only represented
IsLine	data points are connected by a straight line
IsStepLeft	line is drawn as steps where the step height is the value of the left data point
IsStepRight	line is drawn as steps where the step height is the value of the right data point
IsStepCenter	line is drawn as steps where the step is in between two data points
IsImpulse	each data point is represented by a line parallel to the value axis, which reaches from the data point to the zero-value-line

15.582.4 Constructor & Destructor Documentation

15.582.4.1 QCPGraph()

[QCPGraph::QCPGraph](#) (

```
QCPAxis * keyAxis,
QCPAxis * valueAxis ) [explicit]
```

Constructs a graph which uses *keyAxis* as its key axis ("x") and *valueAxis* as its value axis ("y"). *keyAxis* and *valueAxis* must reside in the same [QCustomPlot](#) instance and not have the same orientation. If either of these restrictions is violated, a corresponding message is printed to the debug output (qDebug), the construction is not aborted, though.

The created [QCPGraph](#) is automatically registered with the [QCustomPlot](#) instance inferred from *keyAxis*. This [QCustomPlot](#) instance takes ownership of the [QCPGraph](#), so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead.

To directly create a graph inside a plot, you can also use the simpler [QCustomPlot::addGraph](#) function.

15.582.5 Member Function Documentation

15.582.5.1 addData() [1/2]

```
void QCPGraph::addData (
    const QVector< double > & keys,
    const QVector< double > & values,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided points in *keys* and *values* to the current data. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *keys* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

15.582.5.2 addData() [2/2]

```
void QCPGraph::addData (
    double key,
    double value )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided data point as *key* and *value* to the current data.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

15.582.5.3 data()

```
QSharedPointer< QCPGraphDataContainer > QCPGraph::data ( ) const [inline]
```

Returns a shared pointer to the internal data storage of type [QCPGraphDataContainer](#). You may use it to directly manipulate the data, which may be more convenient and faster than using the regular [setData](#) or [addData](#) methods.

15.582.5.4 draw()

```
void QCPGraph::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

15.582.5.5 drawLegendIcon()

```
void QCPGraph::drawLegendIcon (
    QCPPainter * painter,
    const QRectF & rect ) const [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

15.582.5.6 getKeyRange()

```
QCPRange QCPGraph::getKeyRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth ) const [virtual]
```

Returns the coordinate range that all data in this plottable span in the key axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getValueRange](#)

Implements [QCPAbstractPlottable](#).

15.582.5.7 getValueRange()

```
QCPRange QCPGraph::getValueRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth,
    const QCPRange & inKeyRange = QCPRange() ) const [virtual]
```

Returns the coordinate range that the data points in the specified key range (*inKeyRange*) span in the value axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

If *inKeyRange* has both lower and upper bound set to zero (is equal to [QCPRange\(\)](#)), all data points are considered, without any restriction on the keys.

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getKeyRange](#)

Implements [QCPAbstractPlottable](#).

15.582.5.8 getVisibleDataBounds()

```
void QCPGraph::getVisibleDataBounds (
    QCPGraphDataContainer::const_iterator & begin,
    QCPGraphDataContainer::const_iterator & end,
    const QCPDataRange & rangeRestriction ) const [protected]
```

This method outputs the currently visible data range via *begin* and *end*. The returned range will also never exceed *rangeRestriction*.

This method takes into account that the drawing of data lines at the axis rect border always requires the points just outside the visible axis range. So *begin* and *end* may actually indicate a range that contains one additional data point to the left and right of the visible axis range.

15.582.5.9 selectTest()

```
double QCPGraph::selectTest (
    const QPointF & pos,
```

```
bool onlySelectable,
QVariant * details = nullptr ) const [virtual]
```

Implements a selectTest specific to this plottable's point geometry.

If *details* is not 0, it will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

\see baseclass method [QCPAbstractPlottable::selectTest](#)

Reimplemented from [QCPAbstractPlottable1D< QCPGraphData >](#).

15.582.5.10 setAdaptiveSampling()

```
void QCPGraph::setAdaptiveSampling (
    bool enabled )
```

Sets whether adaptive sampling shall be used when plotting this graph. [QCustomPlot](#)'s adaptive sampling technique can drastically improve the replot performance for graphs with a larger number of points (e.g. above 10,000), without notably changing the appearance of the graph.

By default, adaptive sampling is enabled. Even if enabled, [QCustomPlot](#) decides whether adaptive sampling shall actually be used on a per-graph basis. So leaving adaptive sampling enabled has no disadvantage in almost all cases.

As can be seen, line plots experience no visual degradation from adaptive sampling. Outliers are reproduced reliably, as well as the overall shape of the data set. The replot time reduces dramatically though. This allows [QCustomPlot](#) to display large amounts of data in realtime.

Care must be taken when using high-density scatter plots in combination with adaptive sampling. The adaptive sampling algorithm treats scatter plots more carefully than line plots which still gives a significant reduction of replot times, but not quite as much as for line plots. This is because scatter plots inherently need more data points to be preserved in order to still resemble the original, non-adaptive-sampling plot. As shown above, the results still aren't quite identical, as banding occurs for the outer data points. This is in fact intentional, such that the boundaries of the data cloud stay visible to the viewer. How strong the banding appears, depends on the point density, i.e. the number of points in the plot.

For some situations with scatter plots it might thus be desirable to manually turn adaptive sampling off. For example, when saving the plot to disk. This can be achieved by setting *enabled* to false before issuing a command like [QCustomPlot::savePng](#), and setting *enabled* back to true afterwards.

15.582.5.11 setChannelFillGraph()

```
void QCPGraph::setChannelFillGraph (
    QCPGraph * targetGraph )
```

Sets the target graph for filling the area between this graph and *targetGraph* with the current brush ([setBrush](#)).

When *targetGraph* is set to 0, a normal graph fill to the zero-value-line will be shown. To disable any filling, set the brush to `Qt::NoBrush`.

See also

[setBrush](#)

15.582.5.12 setData() [1/2]

```
void QCPGraph::setData (
    const QVector< double > & keys,
    const QVector< double > & values,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data with the provided points in *keys* and *values*. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *keys* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

See also

[addData](#)

15.582.5.13 setData() [2/2]

```
void QCPGraph::setData (
    QSharedPointer< QCPGraphDataContainer > data )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data container with the provided *data* container.

Since a QSharedPointer is used, multiple QCPGraphs may share the same data container safely. Modifying the data in the container will then affect all graphs that share the container. Sharing can be achieved by simply exchanging the data containers wrapped in shared pointers:

If you do not wish to share containers, but create a copy from an existing container, rather use the [QCPDataContainer<DataType>::set](#) method on the graph's data container directly:

See also

[addData](#)

15.582.5.14 setLineStyle()

```
void QCPGraph::setLineStyle (
    LineStyle ls )
```

Sets how the single data points are connected in the plot. For scatter-only plots, set *ls* to [lsNone](#) and [setScatterStyle](#) to the desired scatter style.

See also

[setScatterStyle](#)

15.582.5.15 setScatterSkip()

```
void QCPGraph::setScatterSkip (
    int skip )
```

If scatters are displayed (scatter style not [QCPScatterStyle::ssNone](#)), *skip* number of scatter points are skipped/not drawn after every drawn scatter point.

This can be used to make the data appear sparser while for example still having a smooth line, and to improve performance for very high density plots.

If *skip* is set to 0 (default), all scatter points are drawn.

See also

[setScatterStyle](#)

15.582.5.16 setScatterStyle()

```
void QCPGraph::setScatterStyle (
    const QCPScatterStyle & style )
```

Sets the visual appearance of single data points in the plot. If set to [QCPScatterStyle::ssNone](#), no scatter points are drawn (e.g. for line-only-plots with appropriate line style).

See also

[QCPScatterStyle](#), [setLineStyle](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.583 QCPGraphData Class Reference

Holds the data of one single data point for [QCPGraph](#).

Public Member Functions

- [QCPGraphData](#) ()
- [QCPGraphData](#) (double key, double value)
- double [sortKey](#) () const
- double [mainKey](#) () const
- double [mainValue](#) () const
- [QCPRange](#) [valueRange](#) () const

Static Public Member Functions

- static [QCPGraphData](#) [fromSortKey](#) (double [sortKey](#))
- static bool [sortKeyIsMainKey](#) ()

Public Attributes

- double **key**
- double **value**

15.583.1 Detailed Description

Holds the data of one single data point for [QCPGraph](#).

The stored data is:

- *key*: coordinate on the key axis of this data point (this is the *mainKey* and the *sortKey*)
- *value*: coordinate on the value axis of this data point (this is the *mainValue*)

The container for storing multiple data points is [QCPGraphDataContainer](#). It is a typedef for [QCPDataContainer](#) with [QCPGraphData](#) as the `DataType` template parameter. See the documentation there for an explanation regarding the data type's generic methods.

See also

[QCPGraphDataContainer](#)

15.583.2 Constructor & Destructor Documentation

15.583.2.1 QCPGraphData() [1/2]

```
QCPGraphData::QCPGraphData ( )
```

Constructs a data point with key and value set to zero.

15.583.2.2 QCPGraphData() [2/2]

```
QCPGraphData::QCPGraphData (
    double key,
    double value )
```

Constructs a data point with the specified *key* and *value*.

15.583.3 Member Function Documentation

15.583.3.1 fromSortKey()

```
static QCPGraphData QCPGraphData::fromSortKey (
    double sortKey ) [inline], [static]
```

Returns a data point with the specified *sortKey*. All other members are set to zero.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

15.583.3.2 mainKey()

```
double QCPGraphData::mainKey ( ) const [inline]
```

Returns the *key* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

15.583.3.3 mainValue()

```
double QCPGraphData::mainValue ( ) const [inline]
```

Returns the *value* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

15.583.3.4 sortKey()

```
double QCPGraphData::sortKey ( ) const [inline]
```

Returns the *key* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

15.583.3.5 sortKeyIsMainKey()

```
static static bool QCPGraphData::sortKeyIsMainKey ( ) [inline], [static]
```

Since the member *key* is both the data point key coordinate and the data ordering parameter, this method returns true.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

15.583.3.6 valueRange()

```
QCPRange QCPGraphData::valueRange ( ) const [inline]
```

Returns a [QCPRange](#) with both lower and upper boundary set to *value* of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

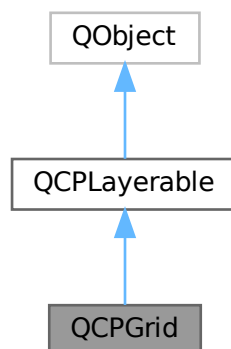
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

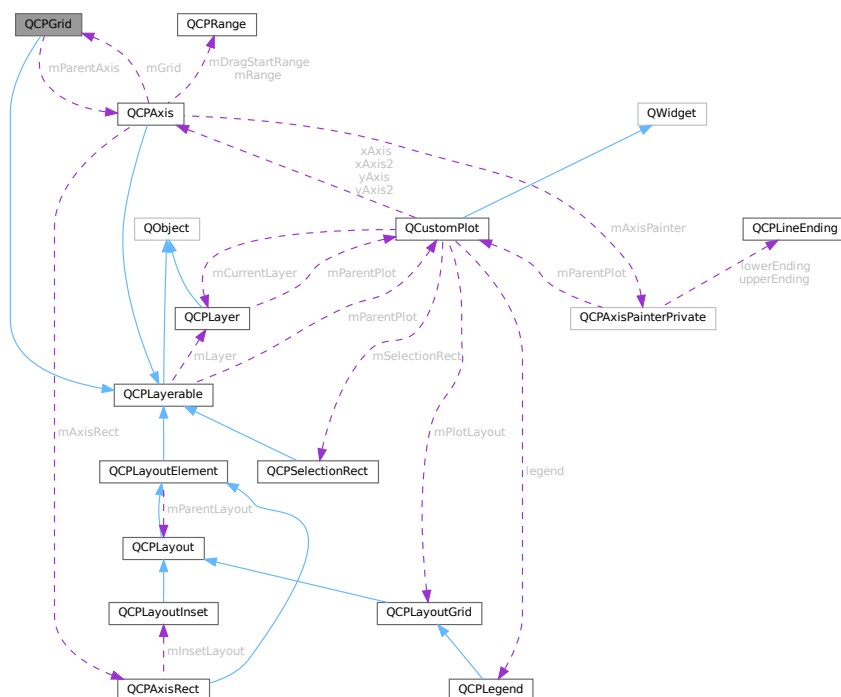
15.584 QCPGrid Class Reference

Responsible for drawing the grid of a [QCPAxis](#).

Inheritance diagram for QCPGrid:



Collaboration diagram for QCPGrid:



Public Member Functions

- [QCPGrid](#) ([QCPAxis](#) *parentAxis)
- bool **subGridVisible** () const
- bool **antialiasedSubGrid** () const
- bool **antialiasedZeroLine** () const
- QPen **pen** () const
- QPen **subGridPen** () const

- QPen **zeroLinePen** () const
- void **setSubGridVisible** (bool visible)
- void **setAntialiasedSubGrid** (bool enabled)
- void **setAntialiasedZeroLine** (bool enabled)
- void **setPen** (const QPen &pen)
- void **setSubGridPen** (const QPen &pen)
- void **setZeroLinePen** (const QPen &pen)

Public Member Functions inherited from QCPLayerable

- QCPLayerable (QCustomPlot *plot, QString targetLayer=QString(), QCPLayerable *parentLayerable=nullptr)
- bool **visible** () const
- QCustomPlot * **parentPlot** () const
- QCPLayerable * **parentLayerable** () const
- QCPLayer * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** (QCPLayer *layer)
- bool **setLayer** (const QString &layerName)
- void **setAntialiased** (bool enabled)
- virtual double **selectTest** (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const
- bool **realVisibility** () const

Protected Member Functions

- virtual void **applyDefaultAntialiasingHint** (QCPPainter *painter) const Q_DECL_OVERRIDE
- virtual void **draw** (QCPPainter *painter) Q_DECL_OVERRIDE
- void **drawGridLines** (QCPPainter *painter) const
- void **drawSubGridLines** (QCPPainter *painter) const

Protected Member Functions inherited from QCPLayerable

- virtual void **parentPlotInitialized** (QCustomPlot *parentPlot)
- virtual QCP::Interaction **selectionCategory** () const
- virtual QRect **clipRect** () const
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** (QCustomPlot *parentPlot)
- void **setParentLayerable** (QCPLayerable *parentLayerable)
- bool **moveToLayer** (QCPLayer *layer, bool prepend)
- void **applyAntialiasingHint** (QCPPainter *painter, bool localAntialiased, QCP::AntialiasedElement overrideElement) const

Protected Attributes

- bool **mSubGridVisible**
- bool **mAntialiasedSubGrid**
- bool **mAntialiasedZeroLine**
- QPen **mPen**
- QPen **mSubGridPen**
- QPen **mZeroLinePen**
- QCPAxis * **mParentAxis**

Protected Attributes inherited from [QCPLayerable](#)

- bool `mVisible`
- [QCustomPlot](#) * `mParentPlot`
- [QPointer](#)< [QCPLayerable](#) > `mParentLayerable`
- [QCPLayer](#) * `mLayer`
- bool `mAntialiased`

Friends

- class [QCPAxis](#)

Additional Inherited Members

Signals inherited from [QCPLayerable](#)

- void `layerChanged` ([QCPLayer](#) *newLayer)

15.584.1 Detailed Description

Responsible for drawing the grid of a [QCPAxis](#).

This class is tightly bound to [QCPAxis](#). Every axis owns a grid instance and uses it to draw the grid lines, sub grid lines and zero-line. You can interact with the grid of an axis via [QCPAxis::grid](#). Normally, you don't need to create an instance of [QCPGrid](#) yourself.

The axis and grid drawing was split into two classes to allow them to be placed on different layers (both [QCPAxis](#) and [QCPGrid](#) inherit from [QCPLayerable](#)). Thus it is possible to have the grid in the background and the axes in the foreground, and any plottables/items in between. This described situation is the default setup, see the [QCPLayer](#) documentation.

15.584.2 Constructor & Destructor Documentation

15.584.2.1 QCPGrid()

```
QCPGrid::QCPGrid (
    QCPAxis * parentAxis ) [explicit]
```

Creates a [QCPGrid](#) instance and sets default values.

You shouldn't instantiate grids on their own, since every [QCPAxis](#) brings its own [QCPGrid](#).

15.584.3 Member Function Documentation

15.584.3.1 applyDefaultAntialiasingHint()

```
void QCPGrid::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Implements [QCPLayerable](#).

15.584.3.2 draw()

```
void QCPGrid::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPLayerable](#).

15.584.3.3 setAntialiasedSubGrid()

```
void QCPGrid::setAntialiasedSubGrid (
    bool enabled )
```

Sets whether sub grid lines are drawn antialiased.

15.584.3.4 setAntialiasedZeroLine()

```
void QCPGrid::setAntialiasedZeroLine (
    bool enabled )
```

Sets whether zero lines are drawn antialiased.

15.584.3.5 setPen()

```
void QCPGrid::setPen (
    const QPen & pen )
```

Sets the pen with which (major) grid lines are drawn.

15.584.3.6 setSubGridPen()

```
void QCPGrid::setSubGridPen (
    const QPen & pen )
```

Sets the pen with which sub grid lines are drawn.

15.584.3.7 setSubGridVisible()

```
void QCPGrid::setSubGridVisible (
    bool visible )
```

Sets whether grid lines at sub tick marks are drawn.

See also

[setSubGridPen](#)

15.584.3.8 setZeroLinePen()

```
void QCPGrid::setZeroLinePen (
    const QPen & pen )
```

Sets the pen with which zero lines are drawn.

Zero lines are lines at value coordinate 0 which may be drawn with a different pen than other grid lines. To disable zero lines and just draw normal grid lines at zero, set *pen* to Qt::NoPen.

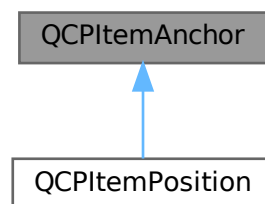
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

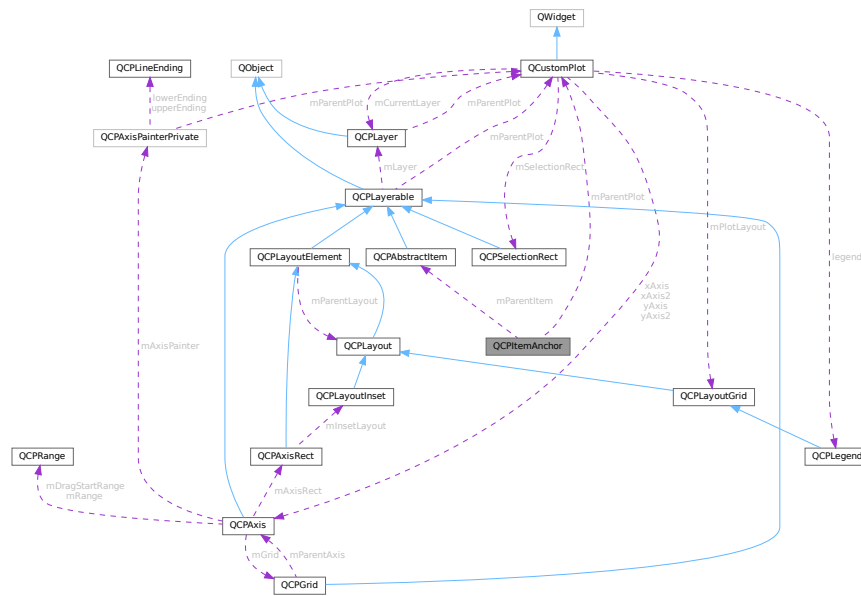
15.585 QCPItemAnchor Class Reference

An anchor of an item to which positions can be attached to.

Inheritance diagram for QCPItemAnchor:



Collaboration diagram for QCPIItemAnchor:



Public Member Functions

- `QCPIItemAnchor` (`QCustomPlot *parentPlot`, `QCPIAbstractItem *parentItem`, `const QString &name`, `int anchorId=-1`)
- `QString name` () const
- virtual `QPointF pixelPosition` () const

Protected Member Functions

- virtual **QCItemPosition** * **toQCItemPosition** ()
- void **addChildX** (**QCItemPosition** *pos)
- void **removeChildX** (**QCItemPosition** *pos)
- void **addChildY** (**QCItemPosition** *pos)
- void **removeChildY** (**QCItemPosition** *pos)

Protected Attributes

- QString **mName**
- QCustomPlot * **mParentPlot**
- QCPAbstractItem * **mParentItem**
- int **mAnchorId**
- QSet< QCPItemPosition * > **mChildrenX**
- QSet< QCPItemPosition * > **mChildrenY**

Friends

- class **QCPItemPosition**

15.585.1 Detailed Description

An anchor of an item to which positions can be attached to.

An item ([QCPAbstractItem](#)) may have one or more anchors. Unlike [QCPItemPosition](#), an anchor doesn't control anything on its item, but provides a way to tie other items via their positions to the anchor.

For example, a [QCItemRect](#) is defined by its positions *topLeft* and *bottomRight*. Additionally it has various anchors like *top*, *topRight* or *bottomLeft* etc. So you can attach the *start* (which is a [QCItemPosition](#)) of a [QCItemLine](#) to one of the anchors by calling [QCItemPosition::setParentAnchor](#) on *start*, passing the wanted anchor of the [QCItemRect](#). This way the start of the line will now always follow the respective anchor location on the rect item. Note that [QCItemPosition](#) derives from [QCItemAnchor](#), so every position can also serve as an anchor to other positions.

To learn how to provide anchors in your own item subclasses, see the subclassing section of the [QCPAbstractItem](#) documentation.

15.585.2 Constructor & Destructor Documentation

15.585.2.1 QCItemAnchor()

```
QCItemAnchor::QCItemAnchor (
    QCustomPlot * parentPlot,
    QCPAbstractItem * parentItem,
    const QString & name,
    int anchorId = -1 )
```

Creates a new [QCItemAnchor](#). You shouldn't create [QCItemAnchor](#) instances directly, even if you want to make a new item subclass. Use [QCPAbstractItem::createAnchor](#) instead, as explained in the subclassing section of the [QCPAbstractItem](#) documentation.

15.585.3 Member Function Documentation

15.585.3.1 pixelPosition()

```
QPointF QCItemAnchor::pixelPosition ( ) const [virtual]
```

Returns the final absolute pixel position of the [QCItemAnchor](#) on the [QCustomPlot](#) surface.

The pixel information is internally retrieved via [QCPAbstractItem::anchorPixelPosition](#) of the parent item, [QCItemAnchor](#) is just an intermediary.

Reimplemented in [QCItemPosition](#).

15.585.3.2 toQCItemPosition()

```
QCItemPosition * QCItemAnchor::toQCItemPosition ( ) [inline], [protected], [virtual]
```

Returns `nullptr` if this instance is merely a [QCItemAnchor](#), and a valid pointer of type `QCItemPosition*` if it actually is a [QCItemPosition](#) (which is a subclass of [QCItemAnchor](#)).

This safe downcast functionality could also be achieved with a `dynamic_cast`. However, [QCustomPlot](#) avoids `dynamic_cast` to work with projects that don't have RTTI support enabled (e.g. `-fno-rtti` flag with gcc compiler).

Reimplemented in [QCItemPosition](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.586 QCItemBracket Class Reference

A bracket for referencing/highlighting certain parts in the plot.

Public Member Functions

- [QCItemBracket](#) ([QCustomPlot](#) *parentPlot)
- `QPen pen () const`
- `QPen selectedPen () const`
- `double length () const`
- [BracketStyle](#) `style () const`
- `void setPen (const QPen &pen)`
- `void setSelectedPen (const QPen &pen)`
- `void setLength (double length)`
- `void setStyle (BracketStyle style)`
- `virtual double selectTest (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const` [Q_↔](#)
DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractItem](#)

- [QCPAbstractItem](#) ([QCustomPlot](#) *parentPlot)
- `bool clipToAxisRect () const`
- [QCPAxisRect](#) * `clipAxisRect () const`
- `bool selectable () const`
- `bool selected () const`
- `void setClipToAxisRect (bool clip)`
- `void setClipAxisRect (QCPAxisRect *rect)`
- `Q_SLOT void setSelectable (bool selectable)`
- `Q_SLOT void setSelected (bool selected)`
- `QList< QCItemPosition * > positions () const`
- `QList< QCItemAnchor * > anchors () const`
- [QCItemPosition](#) * `position (const QString &name) const`
- [QCItemAnchor](#) * `anchor (const QString &name) const`
- `bool hasAnchor (const QString &name) const`

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- `bool visible () const`
- [QCustomPlot](#) * `parentPlot () const`
- [QCPLayerable](#) * `parentLayerable () const`
- [QCPLayer](#) * `layer () const`
- `bool antialiased () const`
- `void setVisible (bool on)`
- `Q_SLOT bool setLayer (QCPLayer *layer)`
- `bool setLayer (const QString &layerName)`
- `void setAntialiased (bool enabled)`
- `bool realVisibility () const`

Public Attributes

- [QCItemPosition](#) *const `left`
- [QCItemPosition](#) *const `right`
- [QCItemAnchor](#) *const `center`

Protected Types

- `enum AnchorIndex { aiCenter }`

Protected Member Functions

- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual QPointF [anchorPixelPosition](#) (int anchorId) const Q_DECL_OVERRIDE
- QPen **mainPen** () const

Protected Member Functions inherited from [QCPAbstractItem](#)

- virtual [QCP::Interaction selectionCategory](#) () const Q_DECL_OVERRIDE
- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- double **rectDistance** (const QRectF &rect, const QPointF &pos, bool filledRect) const
- [QCItemPosition](#) * **createPosition** (const QString &name)
- [QCItemAnchor](#) * **createAnchor** (const QString &name, int anchorId)

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QPen **mPen**
- QPen **mSelectedPen**
- double **mLength**
- [BracketStyle](#) **mStyle**

Protected Attributes inherited from [QCPAbstractItem](#)

- bool **mClipToAxisRect**
- QPointer< [QCPAxisRect](#) > **mClipAxisRect**
- QList< [QCItemPosition](#) * > **mPositions**
- QList< [QCItemAnchor](#) * > **mAnchors**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members

Signals inherited from [QCPAbstractItem](#)

- void [selectionChanged](#) (bool selected)
- void [selectableChanged](#) (bool selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

15.586.1 Detailed Description

A bracket for referencing/highlighting certain parts in the plot.

It has two positions, *left* and *right*, which define the span of the bracket. If *left* is actually farther to the left than *right*, the bracket is opened to the bottom, as shown in the example image.

The bracket supports multiple styles via [setStyle](#). The length, i.e. how far the bracket stretches away from the embraced span, can be controlled with [setLength](#).

Demonstrating the effect of different values for [setLength](#), for styles [bsCalligraphic](#) and [bsSquare](#). Anchors and positions are displayed for reference.

It provides an anchor *center*, to allow connection of other items, e.g. an arrow ([QCPLItemLine](#) or [QCPLItemCurve](#)) or a text label ([QCPLItemText](#)), to the bracket.

15.586.2 Member Enumeration Documentation

15.586.2.1 BracketStyle

enum [QCPItemBracket::BracketStyle](#)

Defines the various visual shapes of the bracket item. The appearance can be further modified by [setLength](#) and [setPen](#).

See also

[setStyle](#)

Enumerator

bsSquare	A brace with angled edges.
bsRound	A brace with round edges.
bsCurly	A curly brace.
bsCalligraphic	A curly brace with varying stroke width giving a calligraphic impression.

15.586.3 Constructor & Destructor Documentation

15.586.3.1 QCPItemBracket()

```
QCPItemBracket::QCPItemBracket (
    QCustomPlot * parentPlot ) [explicit]
```

Creates a bracket item and sets default values.

The created item is automatically registered with *parentPlot*. This [QCustomPlot](#) instance takes ownership of the item, so do not delete it manually but use [QCustomPlot::removeItem\(\)](#) instead.

15.586.4 Member Function Documentation

15.586.4.1 anchorPixelPosition()

```
QPointF QCPItemBracket::anchorPixelPosition (
```

```
int anchorId ) const [protected], [virtual]
```

Reimplemented from [QCPAbstractItem](#).

15.586.4.2 draw()

```
void QCPItemBracket::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractItem](#).

15.586.4.3 selectTest()

```
double QCPItemBracket::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the *selectionTolerance* multiplied by 0.99). Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than $0.99 \cdot \text{selectionTolerance}$).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the *mouseReleaseEvent* occurs, and the finally selected object is notified via the *selectEvent/* *deselectEvent* methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to *selectEvent* when the parent [QCustomPlot](#) decides on the basis of this *selectTest* call, that the object was successfully selected. The subsequent call to *selectEvent* will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in [selectTest](#). The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent *selectEvent*, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass *nullptr* as *details* to indicate that you are not interested in those selection details.

See also

selectEvent, *deselectEvent*, [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Implements [QCPAbstractItem](#).

15.586.4.4 setLength()

```
void QCPItemBracket::setLength (
    double length )
```

Sets the *length* in pixels how far the bracket extends in the direction towards the embraced span of the bracket (i.e. perpendicular to the *left-right*-direction)

Demonstrating the effect of different values for [setLength](#), for styles [bsCalligraphic](#) and [bsSquare](#). Anchors and positions are displayed for reference.

15.586.4.5 setPen()

```
void QCPItemBracket::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the bracket.

Note that when the style is [bsCalligraphic](#), only the color will be taken from the pen, the stroke and width are ignored. To change the apparent stroke width of a calligraphic bracket, use [setLength](#), which has a similar effect.

See also

[setSelectedPen](#)

15.586.4.6 setSelectedPen()

```
void QCPItemBracket::setSelectedPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the bracket when selected

See also

[setPen](#), [setSelected](#)

15.586.4.7 setStyle()

```
void QCPItemBracket::setStyle (
    QCPItemBracket::BracketStyle style )
```

Sets the style of the bracket, i.e. the shape/visual appearance.

See also

[setPen](#)

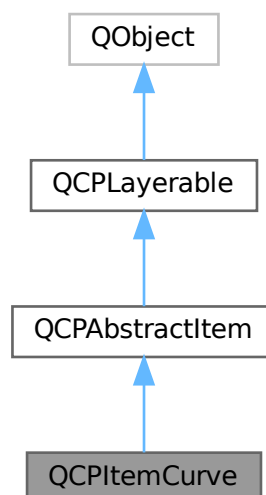
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

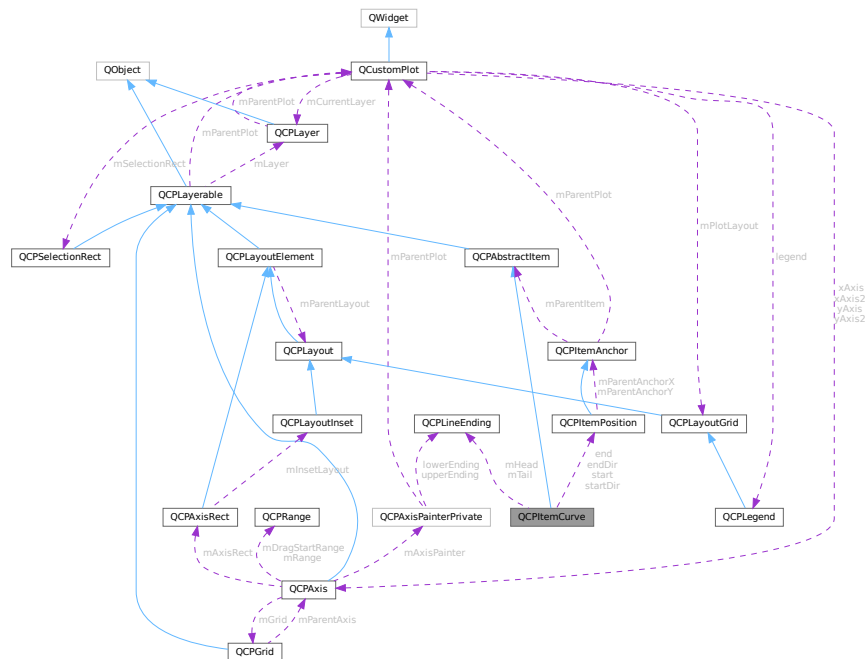
15.587 QCPItemCurve Class Reference

A curved line from one point to another.

Inheritance diagram for QCPItemCurve:



Collaboration diagram for QCPLItemCurve:



Public Member Functions

- [QCPLItemCurve](#) ([QCustomPlot](#) *parentPlot)
- [QPen](#) **pen** () const
- [QPen](#) **selectedPen** () const
- [QCPLLineEnding](#) **head** () const
- [QCPLLineEnding](#) **tail** () const
- void **setPen** (const [QPen](#) &pen)
- void **setSelectedPen** (const [QPen](#) &pen)
- void **setHead** (const [QCPLLineEnding](#) &head)
- void **setTail** (const [QCPLLineEnding](#) &tail)
- virtual double **selectTest** (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const [Q_↔](#) DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractItem](#)

- [QCPAbstractItem](#) ([QCustomPlot](#) *parentPlot)
- bool **clipToAxisRect** () const
- [QCPAxisRect](#) * **clipAxisRect** () const
- bool **selectable** () const
- bool **selected** () const
- void **setClipToAxisRect** (bool clip)
- void **setClipAxisRect** ([QCPAxisRect](#) *rect)
- Q_SLOT void **setSelectable** (bool selectable)
- Q_SLOT void **setSelected** (bool selected)
- [QList](#)< [QCPLItemPosition](#) * > **positions** () const
- [QList](#)< [QCPLItemAnchor](#) * > **anchors** () const
- [QCPLItemPosition](#) * **position** (const [QString](#) &name) const
- [QCPLItemAnchor](#) * **anchor** (const [QString](#) &name) const
- bool **hasAnchor** (const [QString](#) &name) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const [QString](#) &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Public Attributes

- [QCItemPosition](#) *const **start**
- [QCItemPosition](#) *const **startDir**
- [QCItemPosition](#) *const **endDir**
- [QCItemPosition](#) *const **end**

Protected Member Functions

- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- [QPen](#) **mainPen** () const

Protected Member Functions inherited from [QCPAbstractItem](#)

- virtual [QCP::Interaction](#) **selectionCategory** () const Q_DECL_OVERRIDE
- virtual [QRect](#) **clipRect** () const Q_DECL_OVERRIDE
- virtual void **applyDefaultAntialiasingHint** ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void **selectEvent** ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionState↔ Changed) Q_DECL_OVERRIDE
- virtual void **deselectEvent** (bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual [QPointF](#) **anchorPixelPosition** (int anchorId) const
- double **rectDistance** (const [QRectF](#) &rect, const [QPointF](#) &pos, bool filledRect) const
- [QCItemPosition](#) * **createPosition** (const [QString](#) &name)
- [QCItemAnchor](#) * **createAnchor** (const [QString](#) &name, int anchorId)

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual void **mousePressEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **mouseMoveEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseReleaseEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseDoubleClickEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **wheelEvent** ([QWheelEvent](#) *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QPen **mPen**
- QPen **mSelectedPen**
- [QCPLineEnding](#) **mHead**
- [QCPLineEnding](#) **mTail**

Protected Attributes inherited from [QCPAbstractItem](#)

- bool **mClipToAxisRect**
- QPointer< [QCPAxisRect](#) > **mClipAxisRect**
- QList< [QCPLItemPosition](#) * > **mPositions**
- QList< [QCPLItemAnchor](#) * > **mAnchors**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members**Signals inherited from [QCPAbstractItem](#)**

- void [selectionChanged](#) (bool selected)
- void [selectableChanged](#) (bool selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

15.587.1 Detailed Description

A curved line from one point to another.

It has four positions, *start* and *end*, which define the end points of the line, and two control points which define the direction the line exits from the start and the direction from which it approaches the end: *startDir* and *endDir*.

With [setHead](#) and [setTail](#) you may set different line ending styles, e.g. to create an arrow.

Often it is desirable for the control points to stay at fixed relative positions to the start/end point. This can be achieved by setting the parent anchor e.g. of *startDir* simply to *start*, and then specify the desired pixel offset with [QCPLItemPosition::setCoords](#) on *startDir*.

15.587.2 Constructor & Destructor Documentation**15.587.2.1 QCPLItemCurve()**

```
QCPLItemCurve::QCPLItemCurve (
    QCustomPlot * parentPlot ) [explicit]
```

Creates a curve item and sets default values.

The created item is automatically registered with *parentPlot*. This [QCustomPlot](#) instance takes ownership of the item, so do not delete it manually but use [QCustomPlot::removeItem\(\)](#) instead.

15.587.3 Member Function Documentation

15.587.3.1 draw()

```
void QCPItemCurve::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractItem](#).

15.587.3.2 selectTest()

```
double QCPItemCurve::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99). Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than 0.99*selectionTolerance).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the mouseReleaseEvent occurs, and the finally selected object is notified via the selectEvent/ deselectEvent methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to selectEvent when the parent [QCustomPlot](#) decides on the basis of this selectTest call, that the object was successfully selected. The subsequent call to selectEvent will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in [selectTest](#). The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent selectEvent, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

[selectEvent](#), [deselectEvent](#), [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Implements [QCPAbstractItem](#).

15.587.3.3 setHead()

```
void QCPItemCurve::setHead (
    const QCPLLineEnding & head )
```

Sets the line ending style of the head. The head corresponds to the *end* position.

Note that due to the overloaded [QCPLLineEnding](#) constructor, you may directly specify a [QCPLLineEnding::EndingStyle](#) here, e.g.

```
setHead(QCPLLineEnding::esSpikeArrow)
```

See also

[setTail](#)

15.587.3.4 setPen()

```
void QCPItemCurve::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line

See also

[setSelectedPen](#)

15.587.3.5 setSelectedPen()

```
void QCPItemCurve::setSelectedPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line when selected

See also

[setPen](#), [setSelected](#)

15.587.3.6 setTail()

```
void QCPItemCurve::setTail (
    const QCPLLineEnding & tail )
```

Sets the line ending style of the tail. The tail corresponds to the *start* position.

Note that due to the overloaded [QCPLLineEnding](#) constructor, you may directly specify a [QCPLLineEnding::EndingStyle](#) here, e.g.

```
setTail(QCPLLineEnding::esSpikeArrow)
```

See also

[setHead](#)

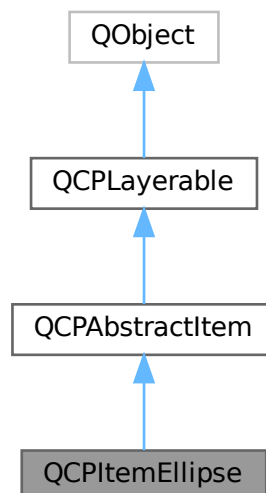
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

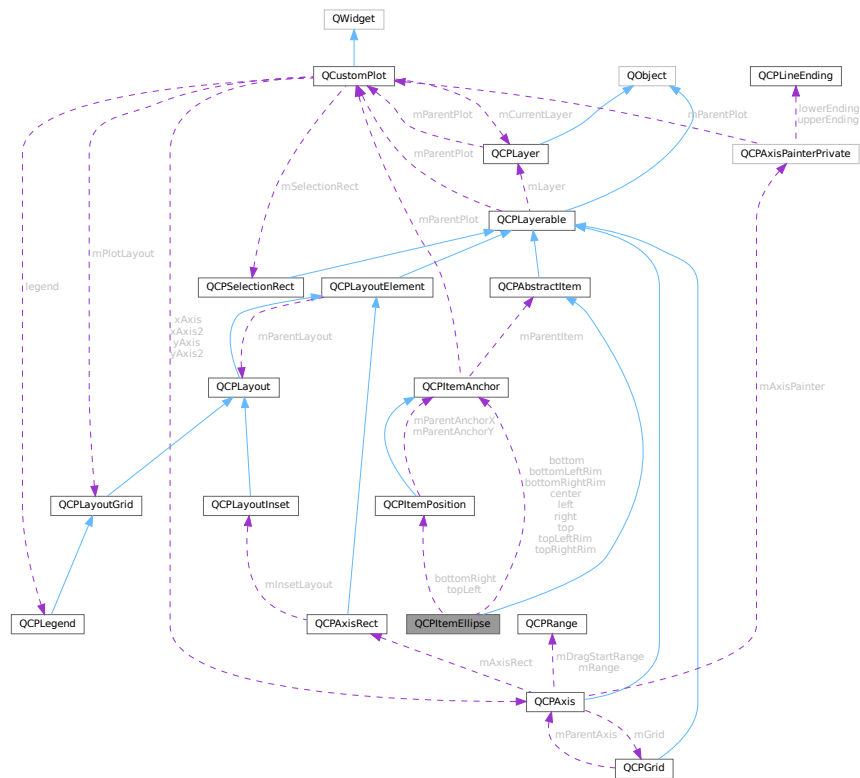
15.588 QCPItemEllipse Class Reference

An ellipse.

Inheritance diagram for QCPItemEllipse:



Collaboration diagram for QCPItemEllipse:



Public Member Functions

- [QCPItemEllipse](#) ([QCustomPlot](#) *parentPlot)
- QPen **pen** () const
- QPen **selectedPen** () const
- QBrush **brush** () const
- QBrush **selectedBrush** () const
- void **setPen** (const QPen &pen)
- void **setSelectedPen** (const QPen &pen)
- void **setBrush** (const QBrush &brush)
- void **setSelectedBrush** (const QBrush &brush)
- virtual double **selectTest** (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_
DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractItem](#)

- [QCPAbstractItem](#) ([QCustomPlot](#) *parentPlot)
- bool **clipToAxisRect** () const
- [QCPAxisRect](#) * **clipAxisRect** () const
- bool **selectable** () const
- bool **selected** () const
- void **setClipToAxisRect** (bool clip)
- void **setClipAxisRect** ([QCPAxisRect](#) *rect)
- Q_SLOT void **setSelectable** (bool selectable)
- Q_SLOT void **setSelected** (bool selected)
- QList< [QCPItemPosition](#) * > **positions** () const

- QList< QCPLItemAnchor * > **anchors** () const
- QCPLItemPosition * **position** (const QString &name) const
- QCPLItemAnchor * **anchor** (const QString &name) const
- bool **hasAnchor** (const QString &name) const

Public Member Functions inherited from QCPLayerable

- QCPLayerable (QCustomPlot *plot, QString targetLayer=QString(), QCPLayerable *parentLayerable=nullptr)
- bool **visible** () const
- QCustomPlot * **parentPlot** () const
- QCPLayerable * **parentLayerable** () const
- QCPLayer * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** (QCPLayer *layer)
- bool **setLayer** (const QString &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Public Attributes

- QCPLItemPosition *const **topLeft**
- QCPLItemPosition *const **bottomRight**
- QCPLItemAnchor *const **topLeftRim**
- QCPLItemAnchor *const **top**
- QCPLItemAnchor *const **topRightRim**
- QCPLItemAnchor *const **right**
- QCPLItemAnchor *const **bottomRightRim**
- QCPLItemAnchor *const **bottom**
- QCPLItemAnchor *const **bottomLeftRim**
- QCPLItemAnchor *const **left**
- QCPLItemAnchor *const **center**

Protected Types

- enum **AnchorIndex** {
aiTopLeftRim , **aiTop** , **aiTopRightRim** , **aiRight** ,
aiBottomRightRim , **aiBottom** , **aiBottomLeftRim** , **aiLeft** ,
aiCenter }

Protected Member Functions

- virtual void **draw** (QCPPainter *painter) Q_DECL_OVERRIDE
- virtual QPointF **anchorPixelPosition** (int anchorId) const Q_DECL_OVERRIDE
- QPen **mainPen** () const
- QBrush **mainBrush** () const

Protected Member Functions inherited from QCPLAbstractItem

- virtual QCPL::Interaction **selectionCategory** () const Q_DECL_OVERRIDE
- virtual QRect **clipRect** () const Q_DECL_OVERRIDE
- virtual void **applyDefaultAntialiasingHint** (QCPPainter *painter) const Q_DECL_OVERRIDE
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void **deselectEvent** (bool *selectionStateChanged) Q_DECL_OVERRIDE
- double **rectDistance** (const QRectF &rect, const QPointF &pos, bool filledRect) const
- QCPLItemPosition * **createPosition** (const QString &name)
- QCPLItemAnchor * **createAnchor** (const QString &name, int anchorId)

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual void **mousePressEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **mouseMoveEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseReleaseEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseDoubleClickEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **wheelEvent** ([QWheelEvent](#) *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QPen](#) **mPen**
- [QPen](#) **mSelectedPen**
- [QBrush](#) **mBrush**
- [QBrush](#) **mSelectedBrush**

Protected Attributes inherited from [QCPAbstractItem](#)

- bool **mClipToAxisRect**
- [QPointer](#)< [QCPAxisRect](#) > **mClipAxisRect**
- [QList](#)< [QCPItemPosition](#) * > **mPositions**
- [QList](#)< [QCPItemAnchor](#) * > **mAnchors**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- [QPointer](#)< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members

Signals inherited from [QCPAbstractItem](#)

- void **selectionChanged** (bool selected)
- void **selectableChanged** (bool selectable)

Signals inherited from [QCPLayerable](#)

- void **layerChanged** ([QCPLayer](#) *newLayer)

15.588.1 Detailed Description

An ellipse.

It has two positions, *topLeft* and *bottomRight*, which define the rect the ellipse will be drawn in.

15.588.2 Constructor & Destructor Documentation

15.588.2.1 QCPItemEllipse()

```
QCPItemEllipse::QCPItemEllipse (
    QCustomPlot * parentPlot ) [explicit]
```

Creates an ellipse item and sets default values.

The created item is automatically registered with *parentPlot*. This [QCustomPlot](#) instance takes ownership of the item, so do not delete it manually but use [QCustomPlot::removeItem\(\)](#) instead.

15.588.3 Member Function Documentation

15.588.3.1 anchorPixelPosition()

```
QPointF QCPItemEllipse::anchorPixelPosition (
    int anchorId ) const [protected], [virtual]
```

Reimplemented from [QCPAbstractItem](#).

15.588.3.2 draw()

```
void QCPItemEllipse::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractItem](#).

15.588.3.3 selectTest()

```
double QCPItemEllipse::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99). Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than 0.99*selectionTolerance).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the `mouseReleaseEvent` occurs, and the finally selected object is notified via the `selectEvent/` `deselectEvent` methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to `selectEvent` when the parent [QCustomPlot](#) decides on the basis of this `selectTest` call, that the object was successfully selected. The subsequent call to `selectEvent` will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in `selectTest`. The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent `selectEvent`, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

`selectEvent`, `deselectEvent`, `mousePressEvent`, `wheelEvent`, [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Implements [QCPAbstractItem](#).

15.588.3.4 setBrush()

```
void QCPItemEllipse::setBrush (
    const QBrush & brush )
```

Sets the brush that will be used to fill the ellipse. To disable filling, set *brush* to Qt::NoBrush.

See also

[setSelectedBrush](#), [setPen](#)

15.588.3.5 setPen()

```
void QCPItemEllipse::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line of the ellipse

See also

[setSelectedPen](#), [setBrush](#)

15.588.3.6 setSelectedBrush()

```
void QCPItemEllipse::setSelectedBrush (
    const QBrush & brush )
```

Sets the brush that will be used to fill the ellipse when selected. To disable filling, set *brush* to Qt::NoBrush.

See also

[setBrush](#)

15.588.3.7 setSelectedPen()

```
void QCPItemEllipse::setSelectedPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line of the ellipse when selected

See also

[setPen](#), [setSelected](#)

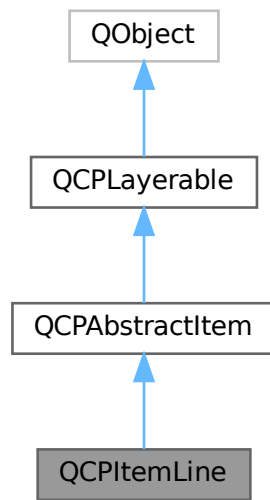
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

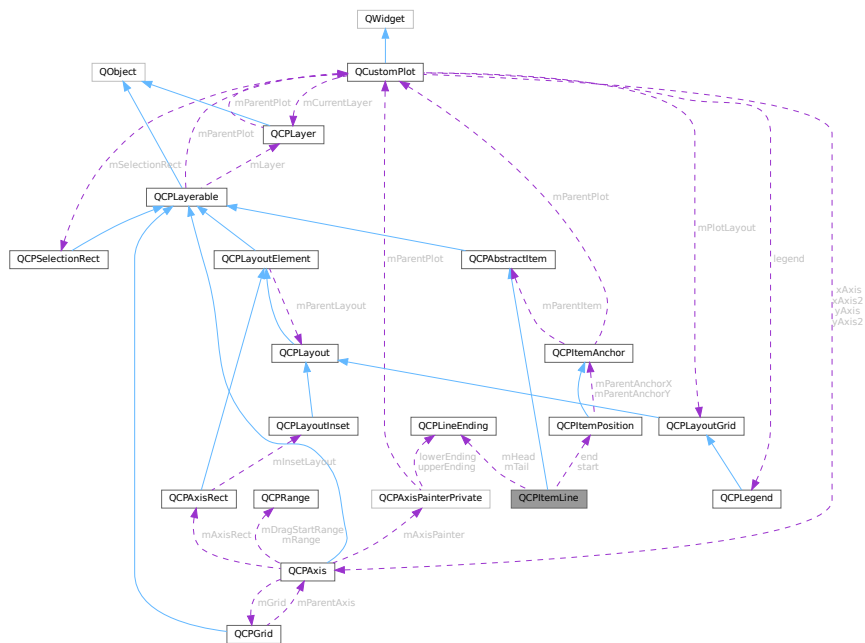
15.589 QCPItemLine Class Reference

A line from one point to another.

Inheritance diagram for QCPLItemLine:



Collaboration diagram for QCPLItemLine:



Public Member Functions

- [QCPLItemLine](#) ([QCustomPlot](#) *parentPlot)
- QPen **pen** () const
- QPen **selectedPen** () const
- [QCPLineEnding](#) **head** () const

- [QCPLineEnding](#) **tail** () const
- void [setPen](#) (const QPen &pen)
- void [setSelectedPen](#) (const QPen &pen)
- void [setHead](#) (const [QCPLineEnding](#) &head)
- void [setTail](#) (const [QCPLineEnding](#) &tail)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractItem](#)

- [QCPAbstractItem](#) ([QCustomPlot](#) *parentPlot)
- bool **clipToAxisRect** () const
- [QCPAxisRect](#) * **clipAxisRect** () const
- bool **selectable** () const
- bool **selected** () const
- void [setClipToAxisRect](#) (bool clip)
- void [setClipAxisRect](#) ([QCPAxisRect](#) *rect)
- Q_SLOT void [setSelectable](#) (bool selectable)
- Q_SLOT void [setSelected](#) (bool selected)
- QList< [QCPItemPosition](#) * > **positions** () const
- QList< [QCPItemAnchor](#) * > **anchors** () const
- [QCPItemPosition](#) * **position** (const QString &name) const
- [QCPItemAnchor](#) * **anchor** (const QString &name) const
- bool **hasAnchor** (const QString &name) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool **realVisibility** () const

Public Attributes

- [QCPItemPosition](#) *const **start**
- [QCPItemPosition](#) *const **end**

Protected Member Functions

- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- QLineF **getRectClippedLine** (const [QCPVector2D](#) &start, const [QCPVector2D](#) &end, const QRect &rect) const
- QPen **mainPen** () const

Protected Member Functions inherited from [QCPAbstractItem](#)

- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual QPointF [anchorPixelPosition](#) (int anchorId) const
- double [rectDistance](#) (const QRectF &rect, const QPointF &pos, bool filledRect) const
- [QCPLItemPosition](#) * [createPosition](#) (const QString &name)
- [QCPLItemAnchor](#) * [createAnchor](#) (const QString &name, int anchorId)

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot)
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void [initializeParentPlot](#) ([QCustomPlot](#) *parentPlot)
- void [setParentLayerable](#) ([QCPLayerable](#) *parentLayerable)
- bool [moveToLayer](#) ([QCPLayer](#) *layer, bool prepend)
- void [applyAntialiasingHint](#) ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QPen [mPen](#)
- QPen [mSelectedPen](#)
- [QCPLineEnding](#) [mHead](#)
- [QCPLineEnding](#) [mTail](#)

Protected Attributes inherited from [QCPAbstractItem](#)

- bool [mClipToAxisRect](#)
- QPointer< [QCPAxisRect](#) > [mClipAxisRect](#)
- QList< [QCPLItemPosition](#) * > [mPositions](#)
- QList< [QCPLItemAnchor](#) * > [mAnchors](#)
- bool [mSelectable](#)
- bool [mSelected](#)

Protected Attributes inherited from [QCPLayerable](#)

- bool [mVisible](#)
- [QCustomPlot](#) * [mParentPlot](#)
- QPointer< [QCPLayerable](#) > [mParentLayerable](#)
- [QCPLayer](#) * [mLayer](#)
- bool [mAntialiased](#)

Additional Inherited Members

Signals inherited from [QCPAbstractItem](#)

- void [selectionChanged](#) (bool selected)
- void [selectableChanged](#) (bool selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

15.589.1 Detailed Description

A line from one point to another.

It has two positions, *start* and *end*, which define the end points of the line.

With [setHead](#) and [setTail](#) you may set different line ending styles, e.g. to create an arrow.

15.589.2 Constructor & Destructor Documentation

15.589.2.1 QCPLItemLine()

```
QCPLItemLine::QCPLItemLine (
    QCustomPlot * parentPlot ) [explicit]
```

Creates a line item and sets default values.

The created item is automatically registered with *parentPlot*. This [QCustomPlot](#) instance takes ownership of the item, so do not delete it manually but use [QCustomPlot::removeItem\(\)](#) instead.

15.589.3 Member Function Documentation

15.589.3.1 draw()

```
void QCPLItemLine::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractItem](#).

15.589.3.2 selectTest()

```
double QCPLItemLine::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPLItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the *selectionTolerance* multiplied by 0.99). Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than 0.99**selectionTolerance*).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the *mouseReleaseEvent* occurs, and the finally selected object is notified via the *selectEvent/* *deselectEvent* methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to *selectEvent* when the parent [QCustomPlot](#) decides on the basis of this *selectTest* call, that the object was successfully selected. The subsequent call to *selectEvent* will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in *selectTest*. The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent *selectEvent*, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass *nullptr* as *details* to indicate that you are not interested in those selection details.

See also

[selectEvent](#), [deselectEvent](#), [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Implements [QCPAbstractItem](#).

15.589.3.3 setHead()

```
void QCPIItemLine::setHead (
    const QCPLLineEnding & head )
```

Sets the line ending style of the head. The head corresponds to the *end* position.

Note that due to the overloaded [QCPLLineEnding](#) constructor, you may directly specify a [QCPLLineEnding::EndingStyle](#) here, e.g.

```
setHead(QCPLLineEnding::esSpikeArrow)
```

See also

[setTail](#)

15.589.3.4 setPen()

```
void QCPIItemLine::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line

See also

[setSelectedPen](#)

15.589.3.5 setSelectedPen()

```
void QCPIItemLine::setSelectedPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line when selected

See also

[setPen](#), [setSelected](#)

15.589.3.6 setTail()

```
void QCPIItemLine::setTail (
    const QCPLLineEnding & tail )
```

Sets the line ending style of the tail. The tail corresponds to the *start* position.

Note that due to the overloaded [QCPLLineEnding](#) constructor, you may directly specify a [QCPLLineEnding::EndingStyle](#) here, e.g.

```
setTail(QCPLLineEnding::esSpikeArrow)
```

See also

[setHead](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.590 QCPIItemPixmap Class Reference

An arbitrary pixmap.

- QPixmap **pixmap** () const
- bool **scaled** () const
- Qt::AspectRatioMode **aspectRatioMode** () const
- Qt::TransformationMode **transformationMode** () const
- QPen **pen** () const
- QPen **selectedPen** () const
- void **setPixmap** (const QPixmap &pixmap)
- void **setScaled** (bool scaled, Qt::AspectRatioMode aspectRatioMode=Qt::KeepAspectRatio, Qt::TransformationMode transformationMode=Qt::SmoothTransformation)
- void **setPen** (const QPen &pen)
- void **setSelectedPen** (const QPen &pen)
- virtual double **selectTest** (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE

Public Member Functions inherited from QCAbstractItem

- QCAbstractItem (QCustomPlot *parentPlot)
- bool **clipToAxisRect** () const
- QCAxisRect * **clipAxisRect** () const
- bool **selectable** () const
- bool **selected** () const
- void **setClipToAxisRect** (bool clip)
- void **setClipAxisRect** (QCAxisRect *rect)
- Q_SLOT void **setSelectable** (bool selectable)
- Q_SLOT void **setSelected** (bool selected)
- QList< QCItemPosition * > **positions** () const
- QList< QCItemAnchor * > **anchors** () const
- QCItemPosition * **position** (const QString &name) const
- QCItemAnchor * **anchor** (const QString &name) const
- bool **hasAnchor** (const QString &name) const

Public Member Functions inherited from QCPLayerable

- QCPLayerable (QCustomPlot *plot, QString targetLayer=QString(), QCPLayerable *parentLayerable=nullptr)
- bool **visible** () const
- QCustomPlot * **parentPlot** () const
- QCPLayerable * **parentLayerable** () const
- QCPLayer * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** (QCPLayer *layer)
- bool **setLayer** (const QString &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Public Attributes

- QCItemPosition *const **topLeft**
- QCItemPosition *const **bottomRight**
- QCItemAnchor *const **top**
- QCItemAnchor *const **topRight**
- QCItemAnchor *const **right**
- QCItemAnchor *const **bottom**
- QCItemAnchor *const **bottomLeft**
- QCItemAnchor *const **left**

Protected Types

- enum **AnchorIndex** {
 aiTop , **aiTopRight** , **aiRight** , **aiBottom** ,
 aiBottomLeft , **aiLeft** }

Protected Member Functions

- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual QPointF **anchorPixelPosition** (int anchorId) const Q_DECL_OVERRIDE
- void **updateScaledPixmap** (QRect finalRect=QRect(), bool flipHorz=false, bool flipVert=false)
- QRect **getFinalRect** (bool *flippedHorz=nullptr, bool *flippedVert=nullptr) const
- QPen **mainPen** () const

Protected Member Functions inherited from [QCPAbstractItem](#)

- virtual [QCP::Interaction](#) **selectionCategory** () const Q_DECL_OVERRIDE
- virtual QRect **clipRect** () const Q_DECL_OVERRIDE
- virtual void **applyDefaultAntialiasingHint** ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void **deselectEvent** (bool *selectionStateChanged) Q_DECL_OVERRIDE
- double **rectDistance** (const QRectF &rect, const QPointF &pos, bool filledRect) const
- [QCPItemPosition](#) * **createPosition** (const QString &name)
- [QCPItemAnchor](#) * **createAnchor** (const QString &name, int anchorId)

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QPixmap **mPixmap**
- QPixmap **mScaledPixmap**
- bool **mScaled**
- bool **mScaledPixmapInvalidated**
- Qt::AspectRatioMode **mAspectRatioMode**
- Qt::TransformationMode **mTransformationMode**
- QPen **mPen**
- QPen **mSelectedPen**

Protected Attributes inherited from [QCPAbstractItem](#)

- bool **mClipToAxisRect**
- QPointer< [QCPAxisRect](#) > **mClipAxisRect**
- QList< [QCPItemPosition](#) * > **mPositions**
- QList< [QCPItemAnchor](#) * > **mAnchors**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from [QCPLayerable](#)

- bool `mVisible`
- [QCustomPlot](#) * `mParentPlot`
- `QPointer< QCPLayerable >` `mParentLayerable`
- [QCPLayer](#) * `mLayer`
- bool `mAntialiased`

Additional Inherited Members

Signals inherited from [QCPAbstractItem](#)

- void `selectionChanged` (bool selected)
- void `selectableChanged` (bool selectable)

Signals inherited from [QCPLayerable](#)

- void `layerChanged` ([QCPLayer](#) *newLayer)

15.590.1 Detailed Description

An arbitrary pixmap.

It has two positions, *topLeft* and *bottomRight*, which define the rectangle the pixmap will be drawn in. Depending on the scale setting ([setScaled](#)), the pixmap will be either scaled to fit the rectangle or be drawn aligned to the *topLeft* position.

If scaling is enabled and *topLeft* is further to the bottom/right than *bottomRight* (as shown on the right side of the example image), the pixmap will be flipped in the respective orientations.

15.590.2 Constructor & Destructor Documentation

15.590.2.1 QCPItemPixmap()

```
QCPItemPixmap::QCPItemPixmap (
    QCustomPlot * parentPlot ) [explicit]
```

Creates a rectangle item and sets default values.

The created item is automatically registered with *parentPlot*. This [QCustomPlot](#) instance takes ownership of the item, so do not delete it manually but use [QCustomPlot::removeItem\(\)](#) instead.

15.590.3 Member Function Documentation

15.590.3.1 anchorPixelPosition()

```
QPointF QCPItemPixmap::anchorPixelPosition (
    int anchorId ) const [protected], [virtual]
```

Reimplemented from [QCPAbstractItem](#).

15.590.3.2 draw()

```
void QCPItemPixmap::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractItem](#).

15.590.3.3 selectTest()

```
double QCPItemPixmap::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPIItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99). Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than 0.99*selectionTolerance).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the mouseReleaseEvent occurs, and the finally selected object is notified via the selectEvent/ deselectEvent methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to selectEvent when the parent [QCustomPlot](#) decides on the basis of this selectTest call, that the object was successfully selected. The subsequent call to selectEvent will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in [selectTest](#). The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent selectEvent, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

[selectEvent](#), [deselectEvent](#), [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Implements [QCPAbstractItem](#).

15.590.3.4 setPen()

```
void QCPItemPixmap::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw a border around the pixmap.

See also

[setSelectedPen](#), [setBrush](#)

15.590.3.5 setPixmap()

```
void QCPItemPixmap::setPixmap (
    const QPixmap & pixmap )
```

Sets the pixmap that will be displayed.

15.590.3.6 setScaled()

```
void QCPItemPixmap::setScaled (
    bool scaled,
    Qt::AspectRatioMode aspectRatioMode = Qt::KeepAspectRatio,
    Qt::TransformationMode transformationMode = Qt::SmoothTransformation )
```

Sets whether the pixmap will be scaled to fit the rectangle defined by the *topLeft* and *bottomRight* positions.

15.590.3.7 setSelectedPen()

```
void QCPItemPixmap::setSelectedPen (
    const QPen & pen )
```

Sets the pen that will be used to draw a border around the pixmap when selected

Public Member Functions

- [QCItemPosition](#) ([QCustomPlot](#) *parentPlot, [QCPAbstractItem](#) *parentItem, const QString &name)
- [PositionType](#) type () const
- [PositionType](#) typeX () const
- [PositionType](#) typeY () const
- [QCItemAnchor](#) * [parentAnchor](#) () const
- [QCItemAnchor](#) * [parentAnchorX](#) () const
- [QCItemAnchor](#) * [parentAnchorY](#) () const
- double [key](#) () const
- double [value](#) () const
- [QPointF](#) [coords](#) () const
- [QCPAxis](#) * [keyAxis](#) () const
- [QCPAxis](#) * [valueAxis](#) () const
- [QCPAxisRect](#) * [axisRect](#) () const
- virtual [QPointF](#) [pixelPosition](#) () const Q_DECL_OVERRIDE
- void [setType](#) ([PositionType](#) type)
- void [setTypeX](#) ([PositionType](#) type)
- void [setTypeY](#) ([PositionType](#) type)
- bool [setParentAnchor](#) ([QCItemAnchor](#) *parentAnchor, bool keepPixelPosition=false)
- bool [setParentAnchorX](#) ([QCItemAnchor](#) *parentAnchor, bool keepPixelPosition=false)
- bool [setParentAnchorY](#) ([QCItemAnchor](#) *parentAnchor, bool keepPixelPosition=false)
- void [setCoords](#) (double key, double value)
- void [setCoords](#) (const [QPointF](#) &pos)
- void [setAxes](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- void [setAxisRect](#) ([QCPAxisRect](#) *axisRect)
- void [setPixelPosition](#) (const [QPointF](#) &pixelPosition)

Public Member Functions inherited from [QCItemAnchor](#)

- [QCItemAnchor](#) ([QCustomPlot](#) *parentPlot, [QCPAbstractItem](#) *parentItem, const QString &name, int anchorId=-1)
- QString [name](#) () const

Protected Member Functions

- virtual [QCItemPosition](#) * [toQCItemPosition](#) () Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCItemAnchor](#)

- void [addChildX](#) ([QCItemPosition](#) *pos)
- void [removeChildX](#) ([QCItemPosition](#) *pos)
- void [addChildY](#) ([QCItemPosition](#) *pos)
- void [removeChildY](#) ([QCItemPosition](#) *pos)

Protected Attributes

- [PositionType](#) [mPositionTypeX](#)
- [PositionType](#) [mPositionTypeY](#)
- [QPointer](#)< [QCPAxis](#) > [mKeyAxis](#)
- [QPointer](#)< [QCPAxis](#) > [mValueAxis](#)
- [QPointer](#)< [QCPAxisRect](#) > [mAxisRect](#)
- double [mKey](#)
- double [mValue](#)
- [QCItemAnchor](#) * [mParentAnchorX](#)
- [QCItemAnchor](#) * [mParentAnchorY](#)

Protected Attributes inherited from [QCItemAnchor](#)

- `QString mName`
- `QCustomPlot * mParentPlot`
- `QCItemAbstractItem * mParentItem`
- `int mAnchorId`
- `QSet< QCItemPosition * > mChildrenX`
- `QSet< QCItemPosition * > mChildrenY`

15.591.1 Detailed Description

Manages the position of an item.

Every item has at least one public [QCItemPosition](#) member pointer which provides ways to position the item on the [QCustomPlot](#) surface. Some items have multiple positions, for example [QCItemRect](#) has two: *topLeft* and *bottomRight*.

[QCItemPosition](#) has a type ([PositionType](#)) that can be set with [setType](#). This type defines how coordinates passed to [setCoords](#) are to be interpreted, e.g. as absolute pixel coordinates, as plot coordinates of certain axes ([QCItemPosition::setAxes](#)), as fractions of the axis rect ([QCItemPosition::setAxisRect](#)), etc. For more advanced plots it is also possible to assign different types per X/Y coordinate of the position (see [setTypeX](#), [setTypeY](#)). This way an item could be positioned for example at a fixed pixel distance from the top in the Y direction, while following a plot coordinate in the X direction.

A [QCItemPosition](#) may have a parent [QCItemAnchor](#), see [setParentAnchor](#). This way you can tie multiple items together. If the [QCItemPosition](#) has a parent, its coordinates ([setCoords](#)) are considered to be absolute pixels in the reference frame of the parent anchor, where (0, 0) means directly ontop of the parent anchor. For example, You could attach the *start* position of a [QCItemLine](#) to the *bottom* anchor of a [QCItemText](#) to make the starting point of the line always be centered under the text label, no matter where the text is moved to. For more advanced plots, it is possible to assign different parent anchors per X/Y coordinate of the position, see [setParentAnchorX](#), [setParentAnchorY](#). This way an item could follow another item in the X direction but stay at a fixed position in the Y direction. Or even follow item A in X, and item B in Y.

Note that every [QCItemPosition](#) inherits from [QCItemAnchor](#) and thus can itself be used as parent anchor for other positions.

To set the apparent pixel position on the [QCustomPlot](#) surface directly, use [setPixelPosition](#). This works no matter what type this [QCItemPosition](#) is or what parent-child situation it is in, as [setPixelPosition](#) transforms the coordinates appropriately, to make the position appear at the specified pixel values.

15.591.2 Member Enumeration Documentation

15.591.2.1 PositionType

```
enum QCItemPosition::PositionType
```

Defines the ways an item position can be specified. Thus it defines what the numbers passed to [setCoords](#) actually mean.

See also

[setType](#)

Enumerator

<code>ptAbsolute</code>	Static positioning in pixels, starting from the top left corner of the viewport/widget.
<code>ptViewportRatio</code>	Static positioning given by a fraction of the viewport size. For example, if you call <code>setCoords(0, 0)</code> , the position will be at the top left corner of the viewport/widget. <code>setCoords(1, 1)</code> will be at the bottom right corner, <code>setCoords(0.5, 0)</code> will be horizontally centered and vertically at the top of the viewport/widget, etc.
<code>ptAxisRectRatio</code>	Static positioning given by a fraction of the axis rect size (see setAxisRect). For example, if you call <code>setCoords(0, 0)</code> , the position will be at the top left corner of the axis rect. <code>setCoords(1, 1)</code> will be at the bottom right corner, <code>setCoords(0.5, 0)</code> will be horizontally centered and vertically at the top of the axis rect, etc. You can also go beyond the axis rect by providing negative coordinates or coordinates larger than 1.
<code>ptPlotCoords</code>	Dynamic positioning at a plot coordinate defined by two axes (see setAxes).

15.591.3 Constructor & Destructor Documentation

15.591.3.1 QCPItemPosition()

```
QCPItemPosition::QCPItemPosition (
    QCustomPlot * parentPlot,
    QCPAbstractItem * parentItem,
    const QString & name )
```

Creates a new [QCPItemPosition](#). You shouldn't create [QCPItemPosition](#) instances directly, even if you want to make a new item subclass. Use [QCPAbstractItem::createPosition](#) instead, as explained in the subclassing section of the [QCPAbstractItem](#) documentation.

15.591.4 Member Function Documentation

15.591.4.1 parentAnchor()

```
QCPItemAnchor * QCPItemPosition::parentAnchor ( ) const [inline]
```

Returns the current parent anchor.

If different parent anchors were set for X and Y ([setParentAnchorX](#), [setParentAnchorY](#)), this method returns the parent anchor of the Y coordinate. In that case rather use [parentAnchorX\(\)](#) and [parentAnchorY\(\)](#).

See also

[setParentAnchor](#)

15.591.4.2 pixelPosition()

```
QPointF QCPItemPosition::pixelPosition ( ) const [virtual]
```

Returns the final absolute pixel position of the [QCPItemPosition](#) on the [QCustomPlot](#) surface. It includes all effects of type ([setType](#)) and possible parent anchors ([setParentAnchor](#)).

See also

[setPixelPosition](#)

Reimplemented from [QCPItemAnchor](#).

15.591.4.3 setAxes()

```
void QCPItemPosition::setAxes (
    QCPAxis * keyAxis,
    QCPAxis * valueAxis )
```

When [setType](#) is [ptPlotCoords](#), this function may be used to specify the axes the coordinates set with [setCoords](#) relate to. By default they are set to the initial [xAxis](#) and [yAxis](#) of the [QCustomPlot](#).

15.591.4.4 setAxisRect()

```
void QCPItemPosition::setAxisRect (
    QCPAxisRect * axisRect )
```

When [setType](#) is [ptAxisRectRatio](#), this function may be used to specify the axis rect the coordinates set with [setCoords](#) relate to. By default this is set to the main axis rect of the [QCustomPlot](#).

15.591.4.5 setCoords() [1/2]

```
void QCPItemPosition::setCoords (
    const QPointF & pos )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the coordinates as a [QPointF](#) *pos* where *pos.x* has the meaning of *key* and *pos.y* the meaning of *value* of the [setCoords\(double key, double value\)](#) method.

15.591.4.6 setCoords() [2/2]

```
void QCPItemPosition::setCoords (
    double key,
    double value )
```

Sets the coordinates of this [QCPItemPosition](#). What the coordinates mean, is defined by the type ([setType](#), [setTypeX](#), [setTypeY](#)).

For example, if the type is [ptAbsolute](#), *key* and *value* mean the x and y pixel position on the [QCustomPlot](#) surface. In that case the origin (0, 0) is in the top left corner of the [QCustomPlot](#) viewport. If the type is [ptPlotCoords](#), *key* and *value* mean a point in the plot coordinate system defined by the axes set by [setAxes](#). By default those are the [QCustomPlot](#)'s *xAxis* and *yAxis*. See the documentation of [setType](#) for other available coordinate types and their meaning.

If different types were configured for X and Y ([setTypeX](#), [setTypeY](#)), *key* and *value* must also be provided in the different coordinate systems. Here, the X type refers to *key*, and the Y type refers to *value*.

See also

[setPixelPosition](#)

15.591.4.7 setParentAnchor()

```
bool QCPItemPosition::setParentAnchor (
    QCPItemAnchor * parentAnchor,
    bool keepPixelPosition = false )
```

Sets the parent of this [QCPItemPosition](#) to *parentAnchor*. This means the position will now follow any position changes of the anchor. The local coordinate system of positions with a parent anchor always is absolute pixels, with (0, 0) being exactly on top of the parent anchor. (Hence the type shouldn't be set to [ptPlotCoords](#) for positions with parent anchors.)

if *keepPixelPosition* is true, the current pixel position of the [QCPItemPosition](#) is preserved during reparenting. If it's set to false, the coordinates are set to (0, 0), i.e. the position will be exactly on top of the parent anchor.

To remove this [QCPItemPosition](#) from any parent anchor, set *parentAnchor* to `nullptr`.

If the [QCPItemPosition](#) previously had no parent and the type is [ptPlotCoords](#), the type is set to [ptAbsolute](#), to keep the position in a valid state.

This method sets the parent anchor for both X and Y directions. It is also possible to set different parents for X and Y, see [setParentAnchorX](#), [setParentAnchorY](#).

15.591.4.8 setParentAnchorX()

```
bool QCPItemPosition::setParentAnchorX (
    QCPItemAnchor * parentAnchor,
    bool keepPixelPosition = false )
```

This method sets the parent anchor of the X coordinate to *parentAnchor*.

For a detailed description of what a parent anchor is, see the documentation of [setParentAnchor](#).

See also

[setParentAnchor](#), [setParentAnchorY](#)

15.591.4.9 setParentAnchorY()

```
bool QCPItemPosition::setParentAnchorY (
    QCPItemAnchor * parentAnchor,
    bool keepPixelPosition = false )
```

This method sets the parent anchor of the Y coordinate to *parentAnchor*.

For a detailed description of what a parent anchor is, see the documentation of [setParentAnchor](#).

See also

[setParentAnchor](#), [setParentAnchorX](#)

15.591.4.10 `setPixelPosition()`

```
void QCPItemPosition::setPixelPosition (
    const QPointF & pixelPosition )
```

Sets the apparent pixel position. This works no matter what type ([setType](#)) this [QCPItemPosition](#) is or what parent-child situation it is in, as coordinates are transformed appropriately, to make the position finally appear at the specified pixel values.

Only if the type is [ptAbsolute](#) and no parent anchor is set, this function's effect is identical to that of [setCoords](#).

See also

[pixelPosition](#), [setCoords](#)

15.591.4.11 `setType()`

```
void QCPItemPosition::setType (
    QCPItemPosition::PositionType type )
```

Sets the type of the position. The type defines how the coordinates passed to [setCoords](#) should be handled and how the [QCPItemPosition](#) should behave in the plot.

The possible values for *type* can be separated in two main categories:

- The position is regarded as a point in plot coordinates. This corresponds to [ptPlotCoords](#) and requires two axes that define the plot coordinate system. They can be specified with [setAxes](#). By default, the [QCustomPlot](#)'s x- and yAxis are used.
- The position is fixed on the [QCustomPlot](#) surface, i.e. independent of axis ranges. This corresponds to all other types, i.e. [ptAbsolute](#), [ptViewportRatio](#) and [ptAxisRectRatio](#). They differ only in the way the absolute position is described, see the documentation of [PositionType](#) for details. For [ptAxisRectRatio](#), note that you can specify the axis rect with [setAxisRect](#). By default this is set to the main axis rect.

Note that the position type [ptPlotCoords](#) is only available (and sensible) when the position has no parent anchor ([setParentAnchor](#)).

If the type is changed, the apparent pixel position on the plot is preserved. This means the coordinates as retrieved with [coords\(\)](#) and set with [setCoords](#) may change in the process.

This method sets the type for both X and Y directions. It is also possible to set different types for X and Y, see [setTypeX](#), [setTypeY](#).

15.591.4.12 `setTypeX()`

```
void QCPItemPosition::setTypeX (
    QCPItemPosition::PositionType type )
```

This method sets the position type of the X coordinate to *type*.

For a detailed description of what a position type is, see the documentation of [setType](#).

See also

[setType](#), [setTypeY](#)

15.591.4.13 `setTypeY()`

```
void QCPItemPosition::setTypeY (
    QCPItemPosition::PositionType type )
```

This method sets the position type of the Y coordinate to *type*.

For a detailed description of what a position type is, see the documentation of [setType](#).

See also

[setType](#), [setTypeX](#)

15.591.4.14 toQCItemPosition()

```
virtual QCItemPosition * QCItemPosition::toQCItemPosition ( ) [inline], [protected], [virtual]
```

Returns `nullptr` if this instance is merely a [QCItemAnchor](#), and a valid pointer of type `QCItemPosition*` if it actually is a [QCItemPosition](#) (which is a subclass of [QCItemAnchor](#)).

This safe downcast functionality could also be achieved with a `dynamic_cast`. However, [QCustomPlot](#) avoids `dynamic_cast` to work with projects that don't have RTTI support enabled (e.g. `-fno-rtti` flag with gcc compiler).

Reimplemented from [QCItemAnchor](#).

15.591.4.15 type()

```
QCItemPosition::PositionType * QCItemPosition::type ( ) const [inline]
```

Returns the current position type.

If different types were set for X and Y ([setTypeX](#), [setTypeY](#)), this method returns the type of the X coordinate. In that case rather use `typeX()` and `typeY()`.

See also

[setType](#)

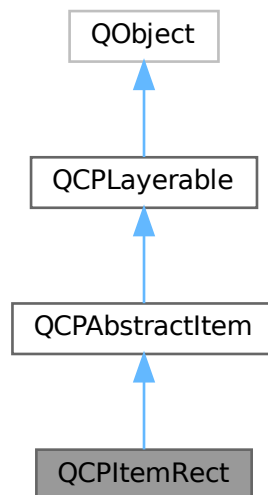
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.592 QCItemRect Class Reference

A rectangle.

Inheritance diagram for QCItemRect:



- [QCPLItemPosition](#) * [position](#) (const QString &name) const
- [QCPLItemAnchor](#) * [anchor](#) (const QString &name) const
- bool [hasAnchor](#) (const QString &name) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * [parentLayerable](#) () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Public Attributes

- [QCPLItemPosition](#) *const **topLeft**
- [QCPLItemPosition](#) *const **bottomRight**
- [QCPLItemAnchor](#) *const **top**
- [QCPLItemAnchor](#) *const **topRight**
- [QCPLItemAnchor](#) *const **right**
- [QCPLItemAnchor](#) *const **bottom**
- [QCPLItemAnchor](#) *const **bottomLeft**
- [QCPLItemAnchor](#) *const **left**

Protected Types

- enum **AnchorIndex** {
 aiTop , **aiTopRight** , **aiRight** , **aiBottom** ,
 aiBottomLeft , **aiLeft** }

Protected Member Functions

- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual QPointF [anchorPixelPosition](#) (int anchorId) const Q_DECL_OVERRIDE
- QPen **mainPen** () const
- QBrush **mainBrush** () const

Protected Member Functions inherited from [QCPAbstractItem](#)

- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- double **rectDistance** (const QRectF &rect, const QPointF &pos, bool filledRect) const
- [QCPLItemPosition](#) * **createPosition** (const QString &name)
- [QCPLItemAnchor](#) * **createAnchor** (const QString &name, int anchorId)

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual void **mousePressEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **mouseMoveEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseReleaseEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseDoubleClickEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **wheelEvent** ([QWheelEvent](#) *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QPen](#) **mPen**
- [QPen](#) **mSelectedPen**
- [QBrush](#) **mBrush**
- [QBrush](#) **mSelectedBrush**

Protected Attributes inherited from [QCPAbstractItem](#)

- bool **mClipToAxisRect**
- [QPointer](#)< [QCPAxisRect](#) > **mClipAxisRect**
- [QList](#)< [QCPItemPosition](#) * > **mPositions**
- [QList](#)< [QCPItemAnchor](#) * > **mAnchors**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- [QPointer](#)< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members

Signals inherited from [QCPAbstractItem](#)

- void **selectionChanged** (bool selected)
- void **selectableChanged** (bool selectable)

Signals inherited from [QCPLayerable](#)

- void **layerChanged** ([QCPLayer](#) *newLayer)

15.592.1 Detailed Description

A rectangle.

It has two positions, *topLeft* and *bottomRight*, which define the rectangle.

15.592.2 Constructor & Destructor Documentation

15.592.2.1 QCPItemRect()

```
QCPItemRect::QCPItemRect (
    QCustomPlot * parentPlot ) [explicit]
```

Creates a rectangle item and sets default values.

The created item is automatically registered with *parentPlot*. This [QCustomPlot](#) instance takes ownership of the item, so do not delete it manually but use [QCustomPlot::removeItem\(\)](#) instead.

15.592.3 Member Function Documentation

15.592.3.1 anchorPixelPosition()

```
QPointF QCPItemRect::anchorPixelPosition (
    int anchorId ) const [protected], [virtual]
```

Reimplemented from [QCPAbstractItem](#).

15.592.3.2 draw()

```
void QCPItemRect::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractItem](#).

15.592.3.3 selectTest()

```
double QCPItemRect::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99). Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than 0.99*selectionTolerance).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the `mouseReleaseEvent` occurs, and the finally selected object is notified via the `selectEvent/` `deselectEvent` methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to `selectEvent` when the parent [QCustomPlot](#) decides on the basis of this `selectTest` call, that the object was successfully selected. The subsequent call to `selectEvent` will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in `selectTest`. The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent `selectEvent`, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

`selectEvent`, `deselectEvent`, `mousePressEvent`, `wheelEvent`, [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Implements [QCPAbstractItem](#).

15.592.3.4 setBrush()

```
void QCPItemRect::setBrush (
    const QBrush & brush )
```

Sets the brush that will be used to fill the rectangle. To disable filling, set *brush* to Qt::NoBrush.

See also

[setSelectedBrush](#), [setPen](#)

15.592.3.5 setPen()

```
void QCPItemRect::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line of the rectangle

See also

[setSelectedPen](#), [setBrush](#)

15.592.3.6 setSelectedBrush()

```
void QCPItemRect::setSelectedBrush (
    const QBrush & brush )
```

Sets the brush that will be used to fill the rectangle when selected. To disable filling, set *brush* to Qt::NoBrush.

See also

[setBrush](#)

15.592.3.7 setSelectedPen()

```
void QCPItemRect::setSelectedPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line of the rectangle when selected

See also

[setPen](#), [setSelected](#)

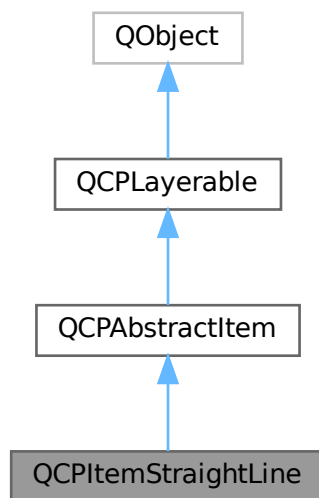
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

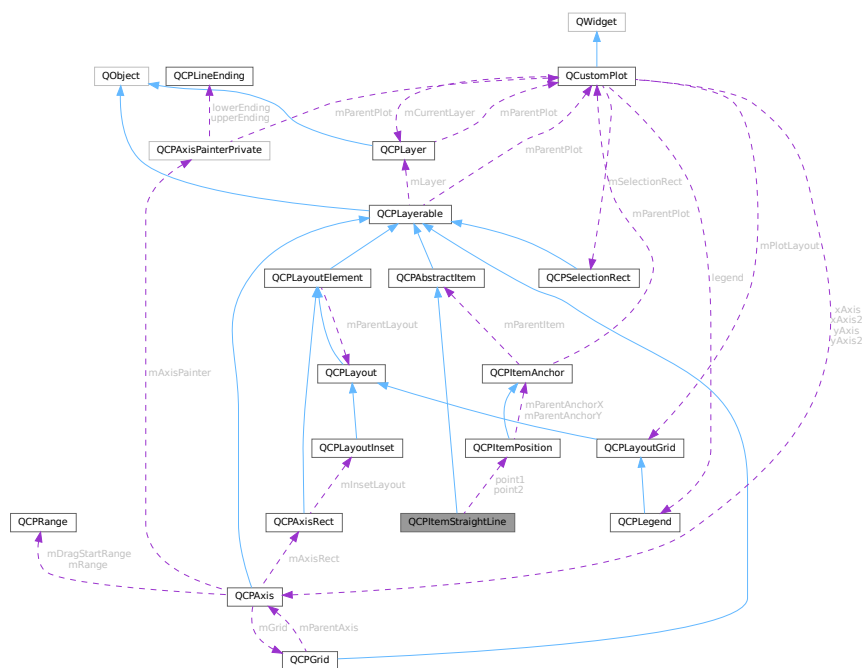
15.593 QCPItemStraightLine Class Reference

A straight line that spans infinitely in both directions.

Inheritance diagram for QCPItemStraightLine:



Collaboration diagram for QCPItemStraightLine:



Public Member Functions

- `QCPLItemStraightLine (QCustomPlot *parentPlot)`
- `QPen pen () const`
- `QPen selectedPen () const`

- void [setPen](#) (const QPen &pen)
- void [setSelectedPen](#) (const QPen &pen)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractItem](#)

- [QCPAbstractItem](#) ([QCustomPlot](#) *parentPlot)
- bool **clipToAxisRect** () const
- [QCPAxisRect](#) * **clipAxisRect** () const
- bool **selectable** () const
- bool **selected** () const
- void [setClipToAxisRect](#) (bool clip)
- void [setClipAxisRect](#) ([QCPAxisRect](#) *rect)
- Q_SLOT void [setSelectable](#) (bool selectable)
- Q_SLOT void [setSelected](#) (bool selected)
- QList< [QCPItemPosition](#) * > **positions** () const
- QList< [QCPItemAnchor](#) * > **anchors** () const
- [QCPItemPosition](#) * **position** (const QString &name) const
- [QCPItemAnchor](#) * **anchor** (const QString &name) const
- bool **hasAnchor** (const QString &name) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool **realVisibility** () const

Public Attributes

- [QCPItemPosition](#) *const **point1**
- [QCPItemPosition](#) *const **point2**

Protected Member Functions

- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- QLineF [getRectClippedStraightLine](#) (const [QCPVector2D](#) &base, const [QCPVector2D](#) &vec, const QRect &rect) const
- QPen **mainPen** () const

Protected Member Functions inherited from [QCPAbstractItem](#)

- virtual [QCP::Interaction](#) **selectionCategory** () const Q_DECL_OVERRIDE
- virtual QRect **clipRect** () const Q_DECL_OVERRIDE
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE

- virtual QPointF **anchorPixelPosition** (int anchorId) const
- double **rectDistance** (const QRectF &rect, const QPointF &pos, bool filledRect) const
- [QCPLItemPosition](#) * **createPosition** (const QString &name)
- [QCPLItemAnchor](#) * **createAnchor** (const QString &name, int anchorId)

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QPen **mPen**
- QPen **mSelectedPen**

Protected Attributes inherited from [QCPAbstractItem](#)

- bool **mClipToAxisRect**
- QPointer< [QCPAxisRect](#) > **mClipAxisRect**
- QList< [QCPLItemPosition](#) * > **mPositions**
- QList< [QCPLItemAnchor](#) * > **mAnchors**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members

Signals inherited from [QCPAbstractItem](#)

- void **selectionChanged** (bool selected)
- void **selectableChanged** (bool selectable)

Signals inherited from [QCPLayerable](#)

- void **layerChanged** ([QCPLayer](#) *newLayer)

15.593.1 Detailed Description

A straight line that spans infinitely in both directions.
It has two positions, *point1* and *point2*, which define the straight line.

15.593.2 Constructor & Destructor Documentation

15.593.2.1 QCItemStraightLine()

```
QCItemStraightLine::QCItemStraightLine (
    QCustomPlot * parentPlot ) [explicit]
```

Creates a straight line item and sets default values.

The created item is automatically registered with *parentPlot*. This [QCustomPlot](#) instance takes ownership of the item, so do not delete it manually but use [QCustomPlot::removeItem\(\)](#) instead.

15.593.3 Member Function Documentation

15.593.3.1 draw()

```
void QCItemStraightLine::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractItem](#).

15.593.3.2 selectTest()

```
double QCItemStraightLine::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99). Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than 0.99*selectionTolerance).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the [mouseReleaseEvent](#) occurs, and the finally selected object is notified via the [selectEvent/](#) [deselectEvent](#) methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to [selectEvent](#) when the parent [QCustomPlot](#) decides on the basis of this [selectTest](#) call, that the object was successfully selected. The subsequent call to [selectEvent](#) will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in [selectTest](#). The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent [selectEvent](#), the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

[selectEvent](#), [deselectEvent](#), [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Implements [QCPAbstractItem](#).

15.593.3.3 setPen()

```
void QCItemStraightLine::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line

See also

[setSelectedPen](#)

15.593.3.4 setSelectedPen()

```
void QCPItemStraightLine::setSelectedPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line when selected

See also

[setPen](#), [setSelected](#)

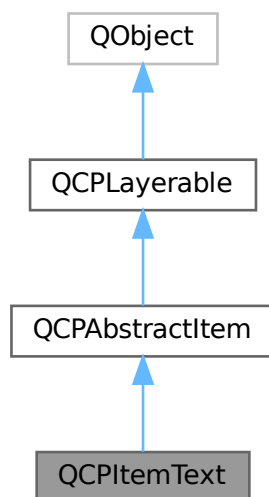
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.594 QCPItemText Class Reference

A text label.

Inheritance diagram for QCPItemText:



- void [setTextAlignment](#) (Qt::Alignment alignment)
- void [setRotation](#) (double degrees)
- void [setPadding](#) (const QMargins &padding)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_
DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractItem](#)

- [QCPAbstractItem](#) ([QCustomPlot](#) *parentPlot)
- bool **clipToAxisRect** () const
- [QCPAxisRect](#) * **clipAxisRect** () const
- bool **selectable** () const
- bool **selected** () const
- void [setClipToAxisRect](#) (bool clip)
- void [setClipAxisRect](#) ([QCPAxisRect](#) *rect)
- Q_SLOT void [setSelectable](#) (bool selectable)
- Q_SLOT void [setSelected](#) (bool selected)
- QList< [QCPItemPosition](#) * > **positions** () const
- QList< [QCPItemAnchor](#) * > **anchors** () const
- [QCPItemPosition](#) * **position** (const QString &name) const
- [QCPItemAnchor](#) * **anchor** (const QString &name) const
- bool **hasAnchor** (const QString &name) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool **realVisibility** () const

Public Attributes

- [QCPItemPosition](#) *const **position**
- [QCPItemAnchor](#) *const **topLeft**
- [QCPItemAnchor](#) *const **top**
- [QCPItemAnchor](#) *const **topRight**
- [QCPItemAnchor](#) *const **right**
- [QCPItemAnchor](#) *const **bottomRight**
- [QCPItemAnchor](#) *const **bottom**
- [QCPItemAnchor](#) *const **bottomLeft**
- [QCPItemAnchor](#) *const **left**

Protected Types

- enum **AnchorIndex** {
 aiTopLeft , **aiTop** , **aiTopRight** , **aiRight** ,
 aiBottomRight , **aiBottom** , **aiBottomLeft** , **aiLeft** }

Protected Member Functions

- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual QPointF [anchorPixelPosition](#) (int anchorId) const Q_DECL_OVERRIDE
- QPointF [getTextDrawPoint](#) (const QPointF &pos, const QRectF &rect, Qt::Alignment positionAlignment) const
- QFont [mainFont](#) () const
- QColor [mainColor](#) () const
- QPen [mainPen](#) () const
- QBrush [mainBrush](#) () const

Protected Member Functions inherited from [QCPAbstractItem](#)

- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- double [rectDistance](#) (const QRectF &rect, const QPointF &pos, bool filledRect) const
- [QCPItemPosition](#) * [createPosition](#) (const QString &name)
- [QCPItemAnchor](#) * [createAnchor](#) (const QString &name, int anchorId)

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot)
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void [initializeParentPlot](#) ([QCustomPlot](#) *parentPlot)
- void [setParentLayerable](#) ([QCPLayerable](#) *parentLayerable)
- bool [moveToLayer](#) ([QCPLayer](#) *layer, bool prepend)
- void [applyAntialiasingHint](#) ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QColor [mColor](#)
- QColor [mSelectedColor](#)
- QPen [mPen](#)
- QPen [mSelectedPen](#)
- QBrush [mBrush](#)
- QBrush [mSelectedBrush](#)
- QFont [mFont](#)
- QFont [mSelectedFont](#)
- QString [mText](#)
- Qt::Alignment [mPositionAlignment](#)
- Qt::Alignment [mTextAlignment](#)
- double [mRotation](#)
- QMargins [mPadding](#)

Protected Attributes inherited from [QCPAbstractItem](#)

- bool `mClipToAxisRect`
- [QPointer](#)< [QCPAxisRect](#) > `mClipAxisRect`
- [QList](#)< [QCPItemPosition](#) * > `mPositions`
- [QList](#)< [QCPItemAnchor](#) * > `mAnchors`
- bool `mSelectable`
- bool `mSelected`

Protected Attributes inherited from [QCPLayerable](#)

- bool `mVisible`
- [QCustomPlot](#) * `mParentPlot`
- [QPointer](#)< [QCPLayerable](#) > `mParentLayerable`
- [QCPLayer](#) * `mLayer`
- bool `mAntialiased`

Additional Inherited Members

Signals inherited from [QCPAbstractItem](#)

- void `selectionChanged` (bool selected)
- void `selectableChanged` (bool selectable)

Signals inherited from [QCPLayerable](#)

- void `layerChanged` ([QCPLayer](#) *newLayer)

15.594.1 Detailed Description

A text label.

Its position is defined by the member *position* and the setting of [setPositionAlignment](#). The latter controls which part of the text rect shall be aligned with *position*.

The text alignment itself (i.e. left, center, right) can be controlled with [setTextAlignment](#).

The text may be rotated around the *position* point with [setRotation](#).

15.594.2 Constructor & Destructor Documentation

15.594.2.1 QCPItemText()

```
QCPItemText::QCPItemText (
    QCustomPlot * parentPlot ) [explicit]
```

Creates a text item and sets default values.

The created item is automatically registered with *parentPlot*. This [QCustomPlot](#) instance takes ownership of the item, so do not delete it manually but use [QCustomPlot::removeItem\(\)](#) instead.

15.594.3 Member Function Documentation

15.594.3.1 anchorPixelPosition()

```
QPointF QCPItemText::anchorPixelPosition (
    int anchorId ) const [protected], [virtual]
```

Reimplemented from [QCPAbstractItem](#).

15.594.3.2 draw()

```
void QCPItemText::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractItem](#).

15.594.3.3 selectTest()

```
double QCPItemText::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99). Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than 0.99*selectionTolerance).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the mouseReleaseEvent occurs, and the finally selected object is notified via the selectEvent/ deselectEvent methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to selectEvent when the parent [QCustomPlot](#) decides on the basis of this selectTest call, that the object was successfully selected. The subsequent call to selectEvent will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in [selectTest](#). The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent selectEvent, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

[selectEvent](#), [deselectEvent](#), [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Implements [QCPAbstractItem](#).

15.594.3.4 setBrush()

```
void QCPItemText::setBrush (
    const QBrush & brush )
```

Sets the brush that will be used to fill the background of the text. To disable the background, set *brush* to `Qt::NoBrush`.

See also

[setSelectedBrush](#), [setPen](#), [setPadding](#)

15.594.3.5 setColor()

```
void QCPItemText::setColor (
    const QColor & color )
```

Sets the color of the text.

15.594.3.6 setFont()

```
void QCPItemText::setFont (
    const QFont & font )
```

Sets the font of the text.

See also

[setSelectedFont](#), [setColor](#)

15.594.3.7 setPadding()

```
void QCPIItemText::setPadding (
    const QMargins & padding )
```

Sets the distance between the border of the text rectangle and the text. The appearance (and visibility) of the text rectangle can be controlled with [setPen](#) and [setBrush](#).

15.594.3.8 setPen()

```
void QCPIItemText::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw a rectangular border around the text. To disable the border, set *pen* to `Qt::NoPen`.

See also

[setSelectedPen](#), [setBrush](#), [setPadding](#)

15.594.3.9 setPositionAlignment()

```
void QCPIItemText::setPositionAlignment (
    Qt::Alignment alignment )
```

Sets which point of the text rect shall be aligned with *position*.

Examples:

- If *alignment* is `Qt::AlignHCenter` | `Qt::AlignTop`, the text will be positioned such that the top of the text rect will be horizontally centered on *position*.
- If *alignment* is `Qt::AlignLeft` | `Qt::AlignBottom`, *position* will indicate the bottom left corner of the text rect.

If you want to control the alignment of (multi-lined) text within the text rect, use [setTextAlignment](#).

15.594.3.10 setRotation()

```
void QCPIItemText::setRotation (
    double degrees )
```

Sets the angle in degrees by which the text (and the text rectangle, if visible) will be rotated around *position*.

15.594.3.11 setSelectedBrush()

```
void QCPIItemText::setSelectedBrush (
    const QBrush & brush )
```

Sets the brush that will be used to fill the background of the text, when the item is selected. To disable the background, set *brush* to `Qt::NoBrush`.

See also

[setBrush](#)

15.594.3.12 setSelectedColor()

```
void QCPIItemText::setSelectedColor (
    const QColor & color )
```

Sets the color of the text that will be used when the item is selected.

15.594.3.13 setSelectedFont()

```
void QCPIItemText::setSelectedFont (
    const QFont & font )
```

Sets the font of the text that will be used when the item is selected.

See also

[setFont](#)

15.594.3.14 setSelectedPen()

```
void QCPItemText::setSelectedPen (
    const QPen & pen )
```

Sets the pen that will be used to draw a rectangular border around the text, when the item is selected. To disable the border, set *pen* to Qt::NoPen.

See also

[setPen](#)

15.594.3.15 setText()

```
void QCPItemText::setText (
    const QString & text )
```

Sets the text that will be displayed. Multi-line texts are supported by inserting a line break character, e.g. '
'.

See also

[setFont](#), [setColor](#), [setTextAlignment](#)

15.594.3.16 setTextAlignment()

```
void QCPItemText::setTextAlignment (
    Qt::Alignment alignment )
```

Controls how (multi-lined) text is aligned inside the text rect (typically Qt::AlignLeft, Qt::AlignCenter or Qt::Align↔Right).

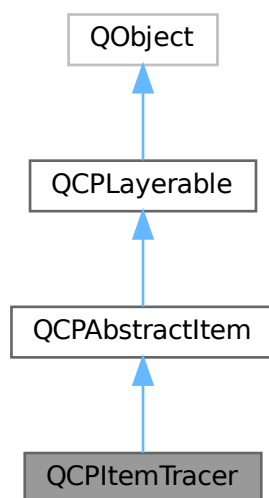
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

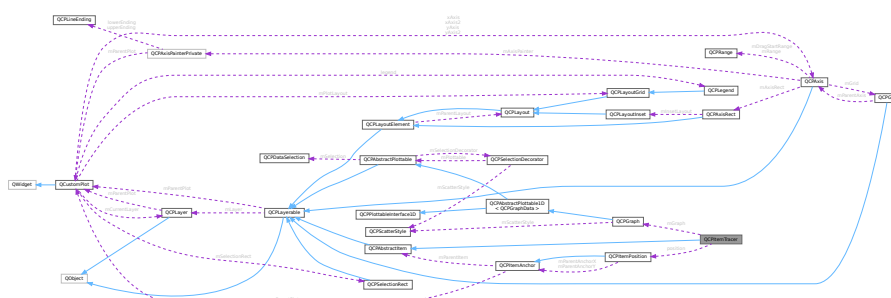
15.595 QCPItemTracer Class Reference

Item that sticks to [QCPGraph](#) data points.

Inheritance diagram for QCPItemTracer:



Collaboration diagram for QCPIItemTracer:



Public Types

- enum `TracerStyle` {
 `tsNone` , `tsPlus` , `tsCrosshair` , `tsCircle` ,
 `tsSquare` }

Public Member Functions

- **QCItemTracer** (**QCustomPlot** *parentPlot)
- QPen **pen** () const
- QPen **selectedPen** () const
- QBrush **brush** () const
- QBrush **selectedBrush** () const
- double **size** () const
- **TracerStyle** **style** () const
- **QCPGraph** * **graph** () const
- double **graphKey** () const
- bool **interpolating** () const

- void [setPen](#) (const QPen &pen)
- void [setSelectedPen](#) (const QPen &pen)
- void [setBrush](#) (const QBrush &brush)
- void [setSelectedBrush](#) (const QBrush &brush)
- void [setSize](#) (double size)
- void [setStyle](#) (TracerStyle style)
- void [setGraph](#) (QCPGraph *graph)
- void [setGraphKey](#) (double key)
- void [setInterpolating](#) (bool enabled)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE
- void [updatePosition](#) ()

Public Member Functions inherited from [QCPAbstractItem](#)

- [QCPAbstractItem](#) (QCustomPlot *parentPlot)
- bool [clipToAxisRect](#) () const
- [QCPAxisRect](#) * [clipAxisRect](#) () const
- bool [selectable](#) () const
- bool [selected](#) () const
- void [setClipToAxisRect](#) (bool clip)
- void [setClipAxisRect](#) (QCPAxisRect *rect)
- Q_SLOT void [setSelectable](#) (bool selectable)
- Q_SLOT void [setSelected](#) (bool selected)
- QList< [QCPItemPosition](#) * > [positions](#) () const
- QList< [QCPItemAnchor](#) * > [anchors](#) () const
- [QCPItemPosition](#) * [position](#) (const QString &name) const
- [QCPItemAnchor](#) * [anchor](#) (const QString &name) const
- bool [hasAnchor](#) (const QString &name) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) (QCustomPlot *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool [visible](#) () const
- QCustomPlot * [parentPlot](#) () const
- [QCPLayerable](#) * [parentLayerable](#) () const
- [QCPLayer](#) * [layer](#) () const
- bool [antialiased](#) () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Public Attributes

- [QCPItemPosition](#) *const [position](#)

Protected Member Functions

- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- QPen [mainPen](#) () const
- QBrush [mainBrush](#) () const

Protected Member Functions inherited from [QCPAbstractItem](#)

- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual QPointF [anchorPixelPosition](#) (int anchorId) const
- double [rectDistance](#) (const QRectF &rect, const QPointF &pos, bool filledRect) const
- [QCPLItemPosition](#) * [createPosition](#) (const QString &name)
- [QCPLItemAnchor](#) * [createAnchor](#) (const QString &name, int anchorId)

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot)
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void [initializeParentPlot](#) ([QCustomPlot](#) *parentPlot)
- void [setParentLayerable](#) ([QCPLayerable](#) *parentLayerable)
- bool [moveToLayer](#) ([QCPLayer](#) *layer, bool prepend)
- void [applyAntialiasingHint](#) ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QPen [mPen](#)
- QPen [mSelectedPen](#)
- QBrush [mBrush](#)
- QBrush [mSelectedBrush](#)
- double [mSize](#)
- [TracerStyle](#) [mStyle](#)
- [QCPGraph](#) * [mGraph](#)
- double [mGraphKey](#)
- bool [mInterpolating](#)

Protected Attributes inherited from [QCPAbstractItem](#)

- bool [mClipToAxisRect](#)
- QPointer< [QCPAxisRect](#) > [mClipAxisRect](#)
- QList< [QCPLItemPosition](#) * > [mPositions](#)
- QList< [QCPLItemAnchor](#) * > [mAnchors](#)
- bool [mSelectable](#)
- bool [mSelected](#)

Protected Attributes inherited from [QCPLayerable](#)

- bool [mVisible](#)
- [QCustomPlot](#) * [mParentPlot](#)
- QPointer< [QCPLayerable](#) > [mParentLayerable](#)
- [QCPLayer](#) * [mLayer](#)
- bool [mAntialiased](#)

Additional Inherited Members

Signals inherited from [QCPAbstractItem](#)

- void [selectionChanged](#) (bool selected)
- void [selectableChanged](#) (bool selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

15.595.1 Detailed Description

Item that sticks to [QCPGraph](#) data points.

The tracer can be connected with a [QCPGraph](#) via [setGraph](#). Then it will automatically adopt the coordinate axes of the graph and update its *position* to be on the graph's data. This means the key stays controllable via [setGraphKey](#), but the value will follow the graph data. If a [QCPGraph](#) is connected, note that setting the coordinates of the tracer item directly via *position* will have no effect because they will be overridden in the next redraw (this is when the coordinate update happens).

If the specified key in [setGraphKey](#) is outside the key bounds of the graph, the tracer will stay at the corresponding end of the graph.

With [setInterpolating](#) you may specify whether the tracer may only stay exactly on data points or whether it interpolates data points linearly, if given a key that lies between two data points of the graph.

The tracer has different visual styles, see [setStyle](#). It is also possible to make the tracer have no own visual appearance (set the style to [tsNone](#)), and just connect other item positions to the tracer *position* (used as an anchor) via [QCPItemPosition::setParentAnchor](#).

Note

The tracer position is only automatically updated upon redraws. So when the data of the graph changes and immediately afterwards (without a redraw) the position coordinates of the tracer are retrieved, they will not reflect the updated data of the graph. In this case [updatePosition](#) must be called manually, prior to reading the tracer coordinates.

15.595.2 Member Enumeration Documentation

15.595.2.1 TracerStyle

enum [QCPItemTracer::TracerStyle](#)

The different visual appearances a tracer item can have. Some styles size may be controlled with [setSize](#).

See also

[setStyle](#)

Enumerator

tsNone	The tracer is not visible.
tsPlus	A plus shaped crosshair with limited size.
tsCrosshair	A plus shaped crosshair which spans the complete axis rect.
tsCircle	A circle.
tsSquare	A square.

15.595.3 Constructor & Destructor Documentation

15.595.3.1 QCPItemTracer()

```
QCPItemTracer::QCPItemTracer (
    QCustomPlot * parentPlot ) [explicit]
```

Creates a tracer item and sets default values.

The created item is automatically registered with *parentPlot*. This [QCustomPlot](#) instance takes ownership of the item, so do not delete it manually but use [QCustomPlot::removeItem\(\)](#) instead.

15.595.4 Member Function Documentation

15.595.4.1 draw()

```
void QCPItemTracer::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractItem](#).

15.595.4.2 selectTest()

```
double QCPItemTracer::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99). Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than 0.99*selectionTolerance).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the mouseReleaseEvent occurs, and the finally selected object is notified via the selectEvent/ deselectEvent methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to selectEvent when the parent [QCustomPlot](#) decides on the basis of this selectTest call, that the object was successfully selected. The subsequent call to selectEvent will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in [selectTest](#). The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent selectEvent, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

[selectEvent](#), [deselectEvent](#), [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Implements [QCPAbstractItem](#).

15.595.4.3 setBrush()

```
void QCPItemTracer::setBrush (
    const QBrush & brush )
```

Sets the brush that will be used to draw any fills of the tracer

See also

[setSelectedBrush](#), [setPen](#)

15.595.4.4 setGraph()

```
void QCPItemTracer::setGraph (
    QCPGraph * graph )
```

Sets the [QCPGraph](#) this tracer sticks to. The tracer *position* will be set to type [QCPItemPosition::ptPlotCoords](#) and the axes will be set to the axes of *graph*.

To free the tracer from any graph, set *graph* to `nullptr`. The tracer *position* can then be placed freely like any other item position. This is the state the tracer will assume when its graph gets deleted while still attached to it.

See also

[setGraphKey](#)

15.595.4.5 setGraphKey()

```
void QCPItemTracer::setGraphKey (
    double key )
```

Sets the key of the graph's data point the tracer will be positioned at. This is the only free coordinate of a tracer when attached to a graph.

Depending on [setInterpolating](#), the tracer will be either positioned on the data point closest to *key*, or will stay exactly at *key* and interpolate the value linearly.

See also

[setGraph](#), [setInterpolating](#)

15.595.4.6 setInterpolating()

```
void QCPItemTracer::setInterpolating (
    bool enabled )
```

Sets whether the value of the graph's data points shall be interpolated, when positioning the tracer.

If *enabled* is set to false and a key is given with [setGraphKey](#), the tracer is placed on the data point of the graph which is closest to the key, but which is not necessarily exactly there. If *enabled* is true, the tracer will be positioned exactly at the specified key, and the appropriate value will be interpolated from the graph's data points linearly.

See also

[setGraph](#), [setGraphKey](#)

15.595.4.7 setPen()

```
void QCPItemTracer::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line of the tracer

See also

[setSelectedPen](#), [setBrush](#)

15.595.4.8 setSelectedBrush()

```
void QCPItemTracer::setSelectedBrush (
    const QBrush & brush )
```

Sets the brush that will be used to draw any fills of the tracer, when selected.

See also

[setBrush](#), [setSelected](#)

15.595.4.9 setSelectedPen()

```
void QCPItemTracer::setSelectedPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line of the tracer when selected

See also

[setPen](#), [setSelected](#)

15.595.4.10 setSize()

```
void QCPItemTracer::setSize (
    double size )
```

Sets the size of the tracer in pixels, if the style supports setting a size (e.g. [tsSquare](#) does, [tsCrosshair](#) does not).

15.595.4.11 setStyle()

```
void QCPItemTracer::setStyle (
    QCPItemTracer::TracerStyle style )
```

Sets the style/visual appearance of the tracer.

If you only want to use the tracer *position* as an anchor for other items, set *style* to [tsNone](#).

15.595.4.12 updatePosition()

```
void QCPItemTracer::updatePosition ( )
```

If the tracer is connected with a graph ([setGraph](#)), this function updates the tracer's *position* to reside on the graph data, depending on the configured key ([setGraphKey](#)).

It is called automatically on every redraw and normally doesn't need to be called manually. One exception is when you want to read the tracer coordinates via *position* and are not sure that the graph's data (or the tracer key with [setGraphKey](#)) hasn't changed since the last redraw. In that situation, call this function before accessing *position*, to make sure you don't get out-of-date coordinates.

If there is no graph set on this tracer, this function does nothing.

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

- void **setAbbreviateDecimalPowers** (bool enabled)
- void **setCacheSize** (int labelCount)
- [AnchorMode](#) **anchorMode** () const
- [AnchorSide](#) **anchorSide** () const
- QPointF **anchorReference** () const
- [AnchorReferenceType](#) **anchorReferenceType** () const
- QFont **font** () const
- QColor **color** () const
- int **padding** () const
- double **rotation** () const
- bool **substituteExponent** () const
- QChar **multiplicationSymbol** () const
- bool **abbreviateDecimalPowers** () const
- int **cacheSize** () const
- void **drawTickLabel** ([QCPPainter](#) *painter, const QPointF &tickPos, const QString &text)
- void **clearCache** ()

Static Public Attributes

- static const QChar **SymbolDot**
- static const QChar **SymbolCross**

Protected Member Functions

- virtual void **drawLabelMaybeCached** ([QCPPainter](#) *painter, const QFont &font, const QColor &color, const QPointF &pos, [AnchorSide](#) side, double rotation, const QString &text)
- virtual QByteArray **generateLabelParameterHash** () const
- QPointF **getAnchorPos** (const QPointF &tickPos)
- void **drawText** ([QCPPainter](#) *painter, const QPointF &pos, const [LabelData](#) &labelData) const
- [LabelData](#) **getTickLabelData** (const QFont &font, const QColor &color, double rotation, [AnchorSide](#) side, const QString &text) const
- void **applyAnchorTransform** ([LabelData](#) &labelData) const
- [CachedLabel](#) * **createCachedLabel** (const [LabelData](#) &labelData) const
- QByteArray **cacheKey** (const QString &text, const QColor &color, double rotation, [AnchorSide](#) side) const
- [AnchorSide](#) **skewedAnchorSide** (const QPointF &tickPos, double sideExpandHorz, double sideExpandVert) const
- [AnchorSide](#) **rotationCorrectedSide** ([AnchorSide](#) side, double rotation) const
- void **analyzeFontMetrics** ()

Protected Attributes

- [AnchorMode](#) **mAnchorMode**
- [AnchorSide](#) **mAnchorSide**
- QPointF **mAnchorReference**
- [AnchorReferenceType](#) **mAnchorReferenceType**
- QFont **mFont**
- QColor **mColor**
- int **mPadding**
- double **mRotation**
- bool **mSubstituteExponent**
- QChar **mMultiplicationSymbol**
- bool **mAbbreviateDecimalPowers**
- [QCustomPlot](#) * **mParentPlot**
- QByteArray **mLabelParameterHash**
- QCache< QString, [CachedLabel](#) > **mLabelCache**
- QRect **mAxisSelectionBox**

- QRect **mTickLabelsSelectionBox**
- QRect **mLabelSelectionBox**
- int **mLetterCapHeight**
- int **mLetterDescent**

15.596.1 Member Enumeration Documentation

15.596.1.1 AnchorMode

enum [QCPLabelPainterPrivate::AnchorMode](#)
 TODO

15.596.1.2 AnchorReferenceType

enum [QCPLabelPainterPrivate::AnchorReferenceType](#)
 TODO

15.596.1.3 AnchorSide

enum [QCPLabelPainterPrivate::AnchorSide](#)
 TODO

15.596.2 Constructor & Destructor Documentation

15.596.2.1 QCPLabelPainterPrivate()

```
QCPLabelPainterPrivate::QCPLabelPainterPrivate (
    QCustomPlot * parentPlot ) [explicit]
```

Constructs a [QCPLabelPainterPrivate](#) instance. Make sure to not create a new instance on every redraw, to utilize the caching mechanisms.

the *parentPlot* does not take ownership of the label painter. Make sure to delete it appropriately.

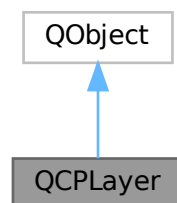
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

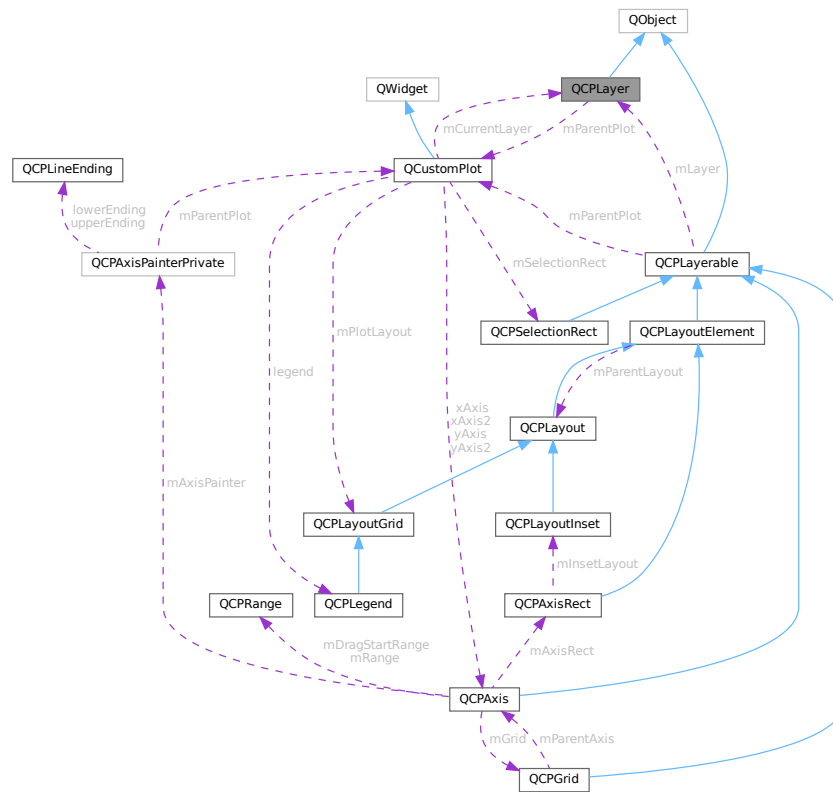
15.597 QCPLayer Class Reference

A layer that may contain objects, to control the rendering order.

Inheritance diagram for QCPLayer:



Collaboration diagram for QCPLayer:



Public Types

- enum [LayerMode](#) { [ImLogical](#) , [ImBuffered](#) }

Public Member Functions

- [QCPLayer](#) ([QCustomPlot](#) *parentPlot, const [QString](#) &layerName)
- [QCustomPlot](#) * **parentPlot** () const
- [QString](#) **name** () const
- int **index** () const
- [QList](#)< [QCPLayerable](#) * > **children** () const
- bool **visible** () const
- [LayerMode](#) **mode** () const
- void **setVisible** (bool visible)
- void **setMode** ([LayerMode](#) mode)
- void **replot** ()

Protected Member Functions

- void **draw** ([QCPPainter](#) *painter)
- void **drawToPaintBuffer** ()
- void **addChild** ([QCPLayerable](#) *layerable, bool prepend)
- void **removeChild** ([QCPLayerable](#) *layerable)

Protected Attributes

- [QCustomPlot](#) * **mParentPlot**
- [QString](#) **mName**
- [int](#) **mIndex**
- [QList](#)< [QCPLayerable](#) * > **mChildren**
- [bool](#) **mVisible**
- [LayerMode](#) **mMode**
- [QWeakPointer](#)< [QCPAbstractPaintBuffer](#) > **mPaintBuffer**

Friends

- class [QCustomPlot](#)
- class [QCPLayerable](#)

15.597.1 Detailed Description

A layer that may contain objects, to control the rendering order.

The Layering system of [QCustomPlot](#) is the mechanism to control the rendering order of the elements inside the plot.

It is based on the two classes [QCPLayer](#) and [QCPLayerable](#). [QCustomPlot](#) holds an ordered list of one or more instances of [QCPLayer](#) (see [QCustomPlot::addLayer](#), [QCustomPlot::layer](#), [QCustomPlot::moveLayer](#), etc.). When replotting, [QCustomPlot](#) goes through the list of layers bottom to top and successively draws the layerables of the layers into the paint buffer(s).

A [QCPLayer](#) contains an ordered list of [QCPLayerable](#) instances. [QCPLayerable](#) is an abstract base class from which almost all visible objects derive, like axes, grids, graphs, items, etc.

15.597.2 Default layers

Initially, [QCustomPlot](#) has six layers: "background", "grid", "main", "axes", "legend" and "overlay" (in that order). On top is the "overlay" layer, which only contains the [QCustomPlot](#)'s selection rect ([QCustomPlot::selectionRect](#)). The next two layers "axes" and "legend" contain the default axes and legend, so they will be drawn above plottables. In the middle, there is the "main" layer. It is initially empty and set as the current layer (see [QCustomPlot::setCurrentLayer](#)). This means, all new plottables, items etc. are created on this layer by default. Then comes the "grid" layer which contains the [QCPGrid](#) instances (which belong tightly to [QCPAxis](#), see [QCPAxis::grid](#)). The Axis rect background shall be drawn behind everything else, thus the default [QCPAxisRect](#) instance is placed on the "background" layer. Of course, the layer affiliation of the individual objects can be changed as required ([QCPLayerable::setLayer](#)).

15.597.3 Controlling the rendering order via layers

Controlling the ordering of layerables in the plot is easy: [Create](#) a new layer in the position you want the layerable to be in, e.g. above "main", with [QCustomPlot::addLayer](#). Then set the current layer with [QCustomPlot::setCurrentLayer](#) to that new layer and finally create the objects normally. They will be placed on the new layer automatically, due to the current layer setting. Alternatively you could have also ignored the current layer setting and just moved the objects with [QCPLayerable::setLayer](#) to the desired layer after creating them.

It is also possible to move whole layers. For example, If you want the grid to be shown in front of all plottables/items on the "main" layer, just move it above "main" with [QCustomPlot::moveLayer](#).

The rendering order within one layer is simply by order of creation or insertion. The item created last (or added last to the layer), is drawn on top of all other objects on that layer.

When a layer is deleted, the objects on it are not deleted with it, but fall on the layer below the deleted layer, see [QCustomPlot::removeLayer](#).

15.597.4 Replotting only a specific layer

If the layer mode ([setMode](#)) is set to [ImBuffered](#), you can replot only this specific layer by calling [replot](#). In certain situations this can provide better replot performance, compared with a full replot of all layers. Upon creation of a new layer, the layer mode is initialized to [ImLogical](#). The only layer that is set to [ImBuffered](#) in a new [QCustomPlot](#) instance is the "overlay" layer, containing the selection rect.

15.597.5 Member Enumeration Documentation

15.597.5.1 LayerMode

enum [QCPLayer::LayerMode](#)

Defines the different rendering modes of a layer. Depending on the mode, certain layers can be replotted individually, without the need to replot (possibly complex) layerables on other layers.

See also

[setMode](#)

Enumerator

ImLogical	Layer is used only for rendering order, and shares paint buffer with all other adjacent logical layers.
ImBuffered	Layer has its own paint buffer and may be replotted individually (see replot).

15.597.6 Constructor & Destructor Documentation

15.597.6.1 QCPLayer()

```
QCPLayer::QCPLayer (
    QCustomPlot * parentPlot,
    const QString & layerName )
```

Creates a new [QCPLayer](#) instance.

Normally you shouldn't directly instantiate layers, use [QCustomPlot::addLayer](#) instead.

Warning

It is not checked that *layerName* is actually a unique layer name in *parentPlot*. This check is only performed by [QCustomPlot::addLayer](#).

15.597.7 Member Function Documentation

15.597.7.1 children()

```
QList< QCPLayerable * > QCPLayer::children ( ) const [inline]
```

Returns a list of all layerables on this layer. The order corresponds to the rendering order: layerables with higher indices are drawn above layerables with lower indices.

15.597.7.2 index()

```
int QCPLayer::index ( ) const [inline]
```

Returns the index this layer has in the [QCustomPlot](#). The index is the integer number by which this layer can be accessed via [QCustomPlot::layer](#).

Layers with higher indices will be drawn above layers with lower indices.

15.597.7.3 replot()

```
void QCPLayer::replot ( )
```

If the layer mode ([setMode](#)) is set to [ImBuffered](#), this method allows replottting only the layerables on this specific layer, without the need to replot all other layers (as a call to [QCustomPlot::replot](#) would do).

[QCustomPlot](#) also makes sure to replot all layers instead of only this one, if the layer ordering or any layerable-layer-association has changed since the last full replot and any other paint buffers were thus invalidated.

If the layer mode is [ImLogical](#) however, this method simply calls [QCustomPlot::replot](#) on the parent [QCustomPlot](#) instance.

See also

[draw](#)

15.597.7.4 `setMode()`

```
void QCPLayer::setMode (
    QCPLayer::LayerMode mode )
```

Sets the rendering mode of this layer.

If *mode* is set to `ImBuffered` for a layer, it will be given a dedicated paint buffer by the parent `QCustomPlot` instance. This means it may be replotted individually by calling `QCPLayer::replot`, without needing to replot all other layers.

Layers which are set to `ImLogical` (the default) are used only to define the rendering order and can't be replotted individually.

Note that each layer which is set to `ImBuffered` requires additional paint buffers for the layers below, above and for the layer itself. This increases the memory consumption and (slightly) decreases the repainting speed because multiple paint buffers need to be joined. So you should carefully choose which layers benefit from having their own paint buffer. A typical example would be a layer which contains certain layerables (e.g. items) that need to be changed and thus replotted regularly, while all other layerables on other layers stay static. By default, only the topmost layer called "overlay" is in mode `ImBuffered`, and contains the selection rect.

See also

[replot](#)

15.597.7.5 `setVisible()`

```
void QCPLayer::setVisible (
    bool visible )
```

Sets whether this layer is visible or not. If *visible* is set to false, all layerables on this layer will be invisible.

This function doesn't change the visibility property of the layerables (`QCPLayerable::setVisible`), but the `QCPLayerable::realVisibility` of each layerable takes the visibility of the parent layer into account.

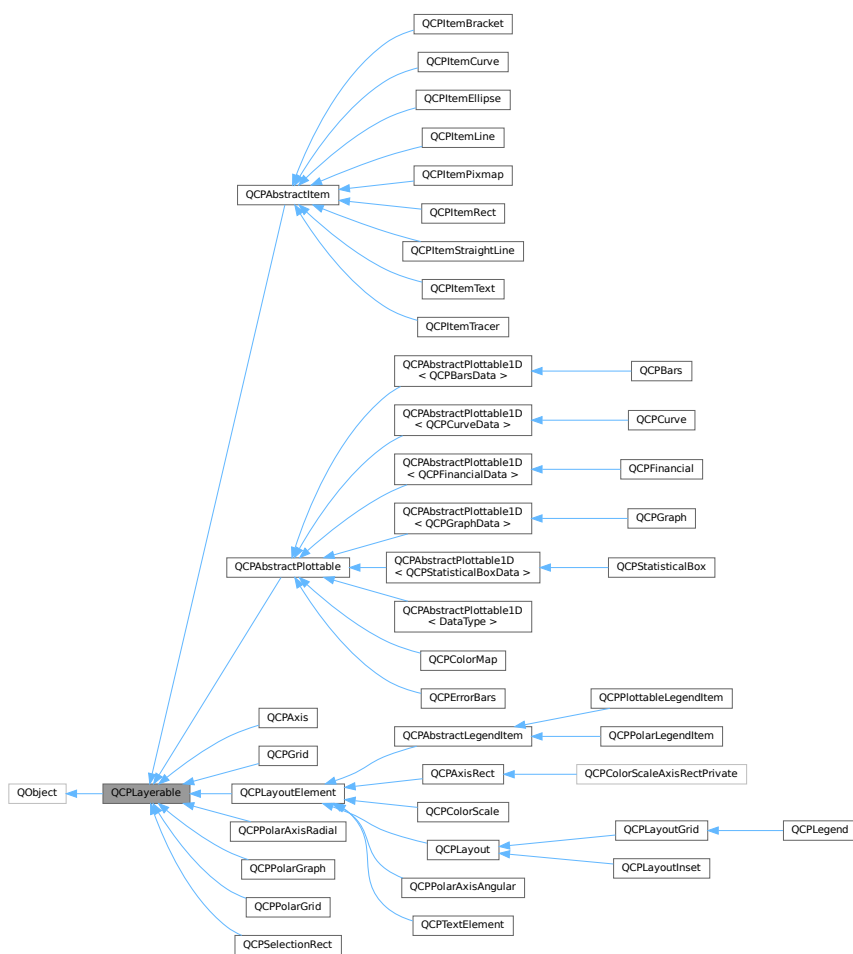
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.598 QCPLayerable Class Reference

Base class for all drawable objects.

Inheritance diagram for QCPLayerable:



- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionState↔ Changed)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** (QCustomPlot *parentPlot)
- void **setParentLayerable** (QCPLayerable *parentLayerable)
- bool **moveToLayer** (QCPLayer *layer, bool prepend)
- void **applyAntialiasingHint** (QCPPainter *painter, bool localAntialiased, QCP::AntialiasedElement overrideElement) const

Protected Attributes

- bool **mVisible**
- QCustomPlot * **mParentPlot**
- QPointer< QCPLayerable > **mParentLayerable**
- QCPLayer * **mLayer**
- bool **mAntialiased**

Friends

- class QCustomPlot
- class QCPLayer
- class QCPAxisRect

15.598.1 Detailed Description

Base class for all drawable objects.

This is the abstract base class most visible objects derive from, e.g. plottables, axes, grid etc.

Every layerable is on a layer (QCPLayer) which allows controlling the rendering order by stacking the layers accordingly.

For details about the layering mechanism, see the QCPLayer documentation.

15.598.2 Constructor & Destructor Documentation

15.598.2.1 QCPLayerable()

```
QCPLayerable::QCPLayerable (
    QCustomPlot * plot,
    QString targetLayer = QString(),
    QCPLayerable * parentLayerable = nullptr )
```

Creates a new QCPLayerable instance.

Since QCPLayerable is an abstract base class, it can't be instantiated directly. Use one of the derived classes.

If *plot* is provided, it automatically places itself on the layer named *targetLayer*. If *targetLayer* is an empty string, it places itself on the current layer of the plot (see QCustomPlot::setCurrentLayer).

It is possible to provide `nullptr` as *plot*. In that case, you should assign a parent plot at a later time with `initializeParentPlot`.

The layerable's parent layerable is set to *parentLayerable*, if provided. Direct layerable parents are mainly used to control visibility in a hierarchy of layerables. This means a layerable is only drawn, if all its ancestor layerables are also visible. Note that *parentLayerable* does not become the QObject-parent (for memory management) of this layerable, *plot* does. It is not uncommon to set the QObject-parent to something else in the constructors of QCPLayerable subclasses, to guarantee a working destruction hierarchy.

15.598.3 Member Function Documentation

15.598.3.1 layerChanged

```
void QCPLayerable::layerChanged (
    QCPLayer * newLayer ) [signal]
```

This signal is emitted when the layer of this layerable changes, i.e. this layerable is moved to a different layer.

See also

[setLayer](#)

15.598.3.2 mouseDoubleClickEvent()

```
void QCPLayerable::mouseDoubleClickEvent (
    QMouseEvent * event,
    const QVariant & details ) [protected], [virtual]
```

This event gets called when the user presses the mouse button a second time in a double-click, while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The [mouseDoubleClickEvent](#) is called instead of the second [mousePressEvent](#). So in the case of a double-click, the event succession is *pressEvent* – *releaseEvent* – *doubleClickEvent* – *releaseEvent*.

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *details* contains layerable-specific details about the hit, which were generated in the previous call to [selectTest](#). For example, One-dimensional plottables like [QCPGraph](#) or [QCPBars](#) convey the clicked data point in the *details* parameter, as [QCPDataSelection](#) packed as `QVariant`. Multi-part objects convey the specific `SelectablePart` that was hit (e.g. [QCPAxis::SelectablePart](#) in the case of axes).

Similarly to [mousePressEvent](#), once a layerable has accepted the [mouseDoubleClickEvent](#), it is considered the mouse grabber and will receive all following calls to [mouseMoveEvent](#) and [mouseReleaseEvent](#) for this mouse interaction (a "mouse interaction" in this context ends with the release).

The default implementation does nothing except explicitly ignoring the event with `event->ignore()`.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseReleaseEvent](#), [wheelEvent](#)

Reimplemented in [QCPLTextElement](#).

15.598.3.3 mouseMoveEvent()

```
void QCPLayerable::mouseMoveEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user moves the mouse while holding a mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *startPos* indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented in [QCPAxis](#), [QCPAxisRect](#), [QCPColorScaleAxisRectPrivate](#), [QCPColorScale](#), [QCPPolarAxisRadial](#), and [QCPPolarAxisAngular](#).

15.598.3.4 mousePressEvent()

```
void QCPLayerable::mousePressEvent (
    QMouseEvent * event,
    const QVariant & details ) [protected], [virtual]
```

This event gets called when the user presses a mouse button while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *details* contains layerable-specific details about the hit, which were generated in the previous call to [selectTest](#). For example, One-dimensional plottables like [QCPGraph](#) or [QCPBars](#) convey the clicked data point in the *details* parameter, as [QCPDataSelection](#) packed as `QVariant`. Multi-part objects convey the specific `SelectablePart` that was hit (e.g. [QCPAxis::SelectablePart](#) in the case of axes).

[QCustomPlot](#) uses an event propagation system that works the same as Qt's system. If your layerable doesn't reimplement the [mousePressEvent](#) or explicitly calls `event->ignore()` in its reimplementation, the event will be propagated to the next layerable in the stacking order.

Once a layerable has accepted the [mousePressEvent](#), it is considered the mouse grabber and will receive all following calls to [mouseMoveEvent](#) or [mouseReleaseEvent](#) for this mouse interaction (a "mouse interaction" in this context ends with the release).

The default implementation does nothing except explicitly ignoring the event with `event->ignore()`.

See also

[mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented in [QCPAxis](#), [QCPAxisRect](#), [QCPTextElement](#), [QCPColorScaleAxisRectPrivate](#), [QCPColorScale](#), [QCPPolarAxisRadial](#), and [QCPPolarAxisAngular](#).

15.598.3.5 mouseReleaseEvent()

```
void QCPLayerable::mouseReleaseEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user releases the mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *startPos* indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction. The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented in [QCPAxis](#), [QCPAxisRect](#), [QCPTextElement](#), [QCPColorScaleAxisRectPrivate](#), [QCPColorScale](#), [QCPPolarAxisRadial](#), and [QCPPolarAxisAngular](#).

15.598.3.6 parentLayerable()

```
QCPLayerable * QCPLayerable::parentLayerable ( ) const [inline]
```

Returns the parent layerable of this layerable. The parent layerable is used to provide visibility hierarchies in conjunction with the method [realVisibility](#). This way, layerables only get drawn if their parent layerables are visible, too.

Note that a parent layerable is not necessarily also the `QObject` parent for memory management. Further, a layerable doesn't always have a parent layerable, so this function may return `nullptr`.

A parent layerable is set implicitly when placed inside layout elements and doesn't need to be set manually by the user.

15.598.3.7 realVisibility()

```
bool QCPLayerable::realVisibility ( ) const
```

Returns whether this layerable is visible, taking the visibility of the layerable parent and the visibility of this layerable's layer into account. This is the method that is consulted to decide whether a layerable shall be drawn or not.

If this layerable has a direct layerable parent (usually set via hierarchies implemented in subclasses, like in the case of [QCPLayoutElement](#)), this function returns true only if this layerable has its visibility set to true and the parent layerable's [realVisibility](#) returns true.

15.598.3.8 selectTest()

```
double QCPLayerable::selectTest (
    const QPointF & pos,
```

```
bool onlySelectable,
QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99). Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than 0.99*selectionTolerance).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the `mouseReleaseEvent` occurs, and the finally selected object is notified via the `selectEvent/` `deselectEvent` methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to `selectEvent` when the parent [QCustomPlot](#) decides on the basis of this `selectTest` call, that the object was successfully selected. The subsequent call to `selectEvent` will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in `selectTest`. The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent `selectEvent`, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

`selectEvent`, `deselectEvent`, [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Reimplemented in [QCPPolarGraph](#), [QCPPolarAxisRadial](#), [QCPPolarAxisAngular](#), [QCPLayoutElement](#), [QCPLayoutInset](#), [QCPAxis](#), [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPCurve >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatBox >](#), [QCPAbstractLegendItem](#), [QCPLegend](#), [QCPTextElement](#), [QCPGraph](#), [QCPCurve](#), [QCPBars](#), [QCPStatisticalBox](#), [QCPColorMap](#), [QCPFinancial](#), [QCPErrorBars](#), [QCItemStraightLine](#), [QCItemLine](#), [QCItemCurve](#), [QCItemRect](#), [QCItemText](#), [QCItemEllipse](#), [QCItemPixmap](#), [QCItemTracer](#), [QCItemBracket](#), [QCPAbstractPlottable](#), and [QCPAbstractItem](#).

15.598.3.9 setAntialiased()

```
void QCPLayerable::setAntialiased (
    bool enabled )
```

Sets whether this object will be drawn antialiased or not.

Note that antialiasing settings may be overridden by [QCustomPlot::setAntialiasedElements](#) and [QCustomPlot::setNotAntialiasedElements](#).

15.598.3.10 setLayer() [1/2]

```
bool QCPLayerable::setLayer (
    const QString & layerName )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts. Sets the layer of this layerable object by name

Returns true on success, i.e. if *layerName* is a valid layer name.

15.598.3.11 setLayer() [2/2]

```
bool QCPLayerable::setLayer (
    QCPLayer * layer )
```

Sets the *layer* of this layerable object. The object will be placed on top of the other objects already on *layer*.

If *layer* is 0, this layerable will not be on any layer and thus not appear in the plot (or interact/receive events).

Returns true if the layer of this layerable was successfully changed to *layer*.

15.598.3.12 setVisible()

```
void QCPLayerable::setVisible (
    bool on )
```

Sets the visibility of this layerable object. If an object is not visible, it will not be drawn on the [QCustomPlot](#) surface, and user interaction with it (e.g. click and selection) is not possible.

15.598.3.13 wheelEvent()

```
void QCPLayerable::wheelEvent (
    QWheelEvent * event ) [protected], [virtual]
```

This event gets called when the user turns the mouse scroll wheel while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`.

The `event->angleDelta()` indicates how far the mouse wheel was turned, which is usually +/- 120 for single rotation steps. However, if the mouse wheel is turned rapidly, multiple steps may accumulate to one event, making the delta larger. On the other hand, if the wheel has very smooth steps or none at all, the delta may be smaller.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#)

Reimplemented in [QCPAxis](#), [QCPAxisRect](#), [QCPColorScaleAxisRectPrivate](#), [QCPColorScale](#), [QCPPolarAxisRadial](#), and [QCPPolarAxisAngular](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.599 QCPLayout Class Reference

The abstract base class for layouts.

Public Member Functions

- [QCPLLayout](#) ()
- virtual void [update](#) ([UpdatePhase](#) phase) Q_DECL_OVERRIDE
- virtual QList< [QCPLLayoutElement](#) * > [elements](#) (bool recursive) const Q_DECL_OVERRIDE
- virtual int [elementCount](#) () const =0
- virtual [QCPLLayoutElement](#) * [elementAt](#) (int index) const =0
- virtual [QCPLLayoutElement](#) * [takeAt](#) (int index)=0
- virtual bool [take](#) ([QCPLLayoutElement](#) *element)=0
- virtual void [simplify](#) ()
- bool [removeAt](#) (int index)
- bool [remove](#) ([QCPLLayoutElement](#) *element)
- void [clear](#) ()

Public Member Functions inherited from [QCPLLayoutElement](#)

- [QCPLLayoutElement](#) ([QCustomPlot](#) *parentPlot=nullptr)
- [QCPLLayout](#) * [layout](#) () const
- QRect [rect](#) () const
- QRect [outerRect](#) () const
- QMargins [margins](#) () const
- QMargins [minimumMargins](#) () const
- QCP::MarginSides [autoMargins](#) () const
- QSize [minimumSize](#) () const
- QSize [maximumSize](#) () const
- [SizeConstraintRect](#) [sizeConstraintRect](#) () const
- [QCPMarginGroup](#) * [marginGroup](#) ([QCP::MarginSide](#) side) const
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > [marginGroups](#) () const
- void [setOuterRect](#) (const QRect &rect)
- void [setMargins](#) (const QMargins &margins)
- void [setMinimumMargins](#) (const QMargins &margins)
- void [setAutoMargins](#) ([QCP::MarginSides](#) sides)
- void [setMinimumSize](#) (const QSize &size)
- void [setMinimumSize](#) (int width, int height)
- void [setMaximumSize](#) (const QSize &size)
- void [setMaximumSize](#) (int width, int height)
- void [setSizeConstraintRect](#) ([SizeConstraintRect](#) constraintRect)
- void [setMarginGroup](#) ([QCP::MarginSides](#) sides, [QCPMarginGroup](#) *group)
- virtual QSize [minimumOuterSizeHint](#) () const
- virtual QSize [maximumOuterSizeHint](#) () const
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool [visible](#) () const
- [QCustomPlot](#) * [parentPlot](#) () const
- [QCPLayerable](#) * [parentLayerable](#) () const
- [QCPLayer](#) * [layer](#) () const
- bool [antialiased](#) () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void **updateLayout** ()
- void **sizeConstraintsChanged** () const
- void **adoptElement** (QCPLayoutElement *el)
- void **releaseElement** (QCPLayoutElement *el)
- QVector< int > **getSectionSizes** (QVector< int > maxSizes, QVector< int > minSizes, QVector< double > stretchFactors, int totalSize) const

Protected Member Functions inherited from QCPLayoutElement

- virtual int **calculateAutoMargin** (QCP::MarginSide side)
- virtual void **layoutChanged** ()
- virtual void **applyDefaultAntialiasingHint** (QCPPainter *painter) const Q_DECL_OVERRIDE
- virtual void **draw** (QCPPainter *painter) Q_DECL_OVERRIDE
- virtual void **parentPlotInitialized** (QCustomPlot *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from QCPLayerable

- virtual QCP::Interaction **selectionCategory** () const
- virtual QRect **clipRect** () const
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** (QCustomPlot *parentPlot)
- void **setParentLayerable** (QCPLayerable *parentLayerable)
- bool **moveToLayer** (QCPLayer *layer, bool prepend)
- void **applyAntialiasingHint** (QCPPainter *painter, bool localAntialiased, QCP::AntialiasedElement overrideElement) const

Static Protected Member Functions

- static QSize **getFinalMinimumOuterSize** (const QCPLayoutElement *el)
- static QSize **getFinalMaximumOuterSize** (const QCPLayoutElement *el)

Friends

- class QCPLayoutElement

Additional Inherited Members**Public Types inherited from QCPLayoutElement**

- enum UpdatePhase { upPreparation , upMargins , upLayout }
- enum SizeConstraintRect { scrInnerRect , scrOuterRect }

Signals inherited from QCPLayerable

- void **layerChanged** (QCPLayer *newLayer)

Protected Attributes inherited from [QCPLayoutElement](#)

- [QCPLayout](#) * **mParentLayout**
- QSize **mMinimumSize**
- QSize **mMaximumSize**
- [SizeConstraintRect](#) **mSizeConstraintRect**
- QRect **mRect**
- QRect **mOuterRect**
- QMargins **mMargins**
- QMargins **mMinimumMargins**
- QCP::MarginSides **mAutoMargins**
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **mMarginGroups**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

15.599.1 Detailed Description

The abstract base class for layouts.

This is an abstract base class for layout elements whose main purpose is to define the position and size of other child layout elements. In most cases, layouts don't draw anything themselves (but there are exceptions to this, e.g. [QCPLegend](#)).

[QCPLayout](#) derives from [QCPLayoutElement](#), and thus can itself be nested in other layouts.

[QCPLayout](#) introduces a common interface for accessing and manipulating the child elements. Those functions are most notably [elementCount](#), [elementAt](#), [takeAt](#), [take](#), [simplify](#), [removeAt](#), [remove](#) and [clear](#). Individual subclasses may add more functions to this interface which are more specialized to the form of the layout. For example, [QCPLayoutGrid](#) adds functions that take row and column indices to access cells of the layout grid more conveniently. Since this is an abstract base class, you can't instantiate it directly. Rather use one of its subclasses like [QCPLayoutGrid](#) or [QCPLayoutInset](#).

For a general introduction to the layout system, see the dedicated documentation page [The Layout System](#).

15.599.2 Constructor & Destructor Documentation

15.599.2.1 [QCPLayout](#)()

```
QCPLayout::QCPLayout ( ) [explicit]
```

Creates an instance of [QCPLayout](#) and sets default values. Note that since [QCPLayout](#) is an abstract base class, it can't be instantiated directly.

15.599.3 Member Function Documentation

15.599.3.1 [clear](#)()

```
void QCPLayout::clear ( )
```

Removes and deletes all layout elements in this layout. Finally calls [simplify](#) to make sure all empty cells are collapsed.

See also

[remove](#), [removeAt](#)

15.599.3.2 `elementAt()`

```
QCPLayoutElement * QCPLayout::elementAt (
    int index ) const [pure virtual]
```

Returns the element in the cell with the given *index*. If *index* is invalid, returns `nullptr`.

Note that even if *index* is valid, the respective cell may be empty in some layouts (e.g. [QCPLayoutGrid](#)), so this function may return `nullptr` in those cases. You may use this function to check whether a cell is empty or not.

See also

[elements](#), [elementCount](#), [takeAt](#)

Implemented in [QCPLayoutGrid](#), and [QCPLayoutInset](#).

15.599.3.3 `elementCount()`

```
int QCPLayout::elementCount ( ) const [pure virtual]
```

Returns the number of elements/cells in the layout.

See also

[elements](#), [elementAt](#)

Implemented in [QCPLayoutGrid](#), and [QCPLayoutInset](#).

15.599.3.4 `elements()`

```
QList< QCPLayoutElement * > QCPLayout::elements (
    bool recursive ) const [virtual]
```

Returns a list of all child elements in this layout element. If *recursive* is true, all sub-child elements are included in the list, too.

Warning

There may be `nullptr` entries in the returned list. For example, [QCPLayoutGrid](#) may have empty cells which yield `nullptr` at the respective index.

Reimplemented from [QCPLayoutElement](#).

Reimplemented in [QCPLayoutGrid](#).

15.599.3.5 `remove()`

```
bool QCPLayout::remove (
    QCPLayoutElement * element )
```

Removes and deletes the provided *element*. Returns true on success. If *element* is not in the layout, returns false. This function internally uses [takeAt](#) to remove the element from the layout and then deletes the element. Note that some layouts don't remove the respective cell right away but leave an empty cell after successful removal of the layout element. To collapse empty cells, use [simplify](#).

See also

[removeAt](#), [take](#)

15.599.3.6 `removeAt()`

```
bool QCPLayout::removeAt (
    int index )
```

Removes and deletes the element at the provided *index*. Returns true on success. If *index* is invalid or points to an empty cell, returns false.

This function internally uses [takeAt](#) to remove the element from the layout and then deletes the returned element. Note that some layouts don't remove the respective cell right away but leave an empty cell after successful removal of the layout element. To collapse empty cells, use [simplify](#).

See also

[remove](#), [takeAt](#)

15.599.3.7 simplify()

```
void QCPLayout::simplify ( ) [virtual]
```

Simplifies the layout by collapsing empty cells. The exact behavior depends on subclasses, the default implementation does nothing.

Not all layouts need simplification. For example, [QCPLayoutInset](#) doesn't use explicit simplification while [QCPLayoutGrid](#) does.

Reimplemented in [QCPLayoutGrid](#), and [QCPLayoutInset](#).

15.599.3.8 sizeConstraintsChanged()

```
void QCPLayout::sizeConstraintsChanged ( ) const [protected]
```

Subclasses call this method to report changed (minimum/maximum) size constraints.

If the parent of this layout is again a [QCPLayout](#), forwards the call to the parent's [sizeConstraintsChanged](#). If the parent is a [QWidget](#) (i.e. is the [QCustomPlot::plotLayout](#) of [QCustomPlot](#)), calls [QWidget::updateGeometry](#), so if the [QCustomPlot](#) widget is inside a Qt [QLayout](#), it may update itself and resize cells accordingly.

15.599.3.9 take()

```
bool QCPLayout::take (
    QCPLayoutElement * element ) [pure virtual]
```

Removes the specified *element* from the layout and returns true on success.

If the *element* isn't in this layout, returns false.

Note that some layouts don't remove the respective cell right away but leave an empty cell after successful removal of the layout element. To collapse empty cells, use [simplify](#).

See also

[takeAt](#)

Implemented in [QCPLayoutGrid](#), and [QCPLayoutInset](#).

15.599.3.10 takeAt()

```
QCPLayoutElement * QCPLayout::takeAt (
    int index ) [pure virtual]
```

Removes the element with the given *index* from the layout and returns it.

If the *index* is invalid or the cell with that index is empty, returns `nullptr`.

Note that some layouts don't remove the respective cell right away but leave an empty cell after successful removal of the layout element. To collapse empty cells, use [simplify](#).

See also

[elementAt](#), [take](#)

Implemented in [QCPLayoutGrid](#), and [QCPLayoutInset](#).

15.599.3.11 update()

```
void QCPLayout::update (
    UpdatePhase phase ) [virtual]
```

If *phase* is [upLayout](#), calls [updateLayout](#), which subclasses may reimplement to reposition and resize their cells.

Finally, the call is propagated down to all child [QCPLayoutElements](#).

For details about this method and the update phases, see the documentation of [QCPLayoutElement::update](#).

Reimplemented from [QCPLayoutElement](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

- QSize **minimumSize** () const
- QSize **maximumSize** () const
- [SizeConstraintRect](#) **sizeConstraintRect** () const
- [QCPMarginGroup](#) * **marginGroup** ([QCP::MarginSide](#) side) const
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **marginGroups** () const
- void **setOuterRect** (const QRect &rect)
- void **setMargins** (const QMargins &margins)
- void **setMinimumMargins** (const QMargins &margins)
- void **setAutoMargins** ([QCP::MarginSides](#) sides)
- void **setMinimumSize** (const QSize &size)
- void **setMinimumSize** (int width, int height)
- void **setMaximumSize** (const QSize &size)
- void **setMaximumSize** (int width, int height)
- void **setSizeConstraintRect** ([SizeConstraintRect](#) constraintRect)
- void **setMarginGroup** ([QCP::MarginSides](#) sides, [QCPMarginGroup](#) *group)
- virtual void **update** ([UpdatePhase](#) phase)
- virtual QSize **minimumOuterSizeHint** () const
- virtual QSize **maximumOuterSizeHint** () const
- virtual QList< [QCPLayoutElement](#) * > **elements** (bool recursive) const
- virtual double **selectTest** (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_
DECL_OVERRIDE

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const QString &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Protected Member Functions

- virtual int **calculateAutoMargin** ([QCP::MarginSide](#) side)
- virtual void **layoutChanged** ()
- virtual void **applyDefaultAntialiasingHint** ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayerable](#)

- virtual [QCP::Interaction](#) **selectionCategory** () const
- virtual QRect **clipRect** () const
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)

- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QCPLayout](#) * **mParentLayout**
- [QSize](#) **mMinimumSize**
- [QSize](#) **mMaximumSize**
- [SizeConstraintRect](#) **mSizeConstraintRect**
- [QRect](#) **mRect**
- [QRect](#) **mOuterRect**
- [QMargins](#) **mMargins**
- [QMargins](#) **mMinimumMargins**
- [QCP::MarginSides](#) **mAutoMargins**
- [QHash](#)< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **mMarginGroups**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- [QPointer](#)< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCustomPlot](#)
- class [QCPLayout](#)
- class [QCPMarginGroup](#)

Additional Inherited Members

Signals inherited from [QCPLayerable](#)

- void **layerChanged** ([QCPLayer](#) *newLayer)

15.600.1 Detailed Description

The abstract base class for all objects that form the layout system.

This is an abstract base class. As such, it can't be instantiated directly, rather use one of its subclasses.

A Layout element is a rectangular object which can be placed in layouts. It has an outer rect ([QCPLayoutElement::outerRect](#)) and an inner rect ([QCPLayoutElement::rect](#)). The difference between outer and inner rect is called its margin. The margin can either be set to automatic or manual ([setAutoMargins](#)) on a per-side basis. If a side is set to manual, that margin can be set explicitly with [setMargins](#) and will stay fixed at that value. If it's set to automatic, the layout element subclass will control the value itself (via [calculateAutoMargin](#)).

Layout elements can be placed in layouts (base class [QCPLayout](#)) like [QCPLayoutGrid](#). The top level layout is reachable via [QCustomPlot::plotLayout](#), and is a [QCPLayoutGrid](#). Since [QCPLayout](#) itself derives from [QCPLayoutElement](#), layouts can be nested.

Thus in [QCustomPlot](#) one can divide layout elements into two categories: The ones that are invisible by themselves, because they don't draw anything. Their only purpose is to manage the position and size of other layout elements. This category of layout elements usually use [QCPLayout](#) as base class. Then there is the category of layout elements which actually draw something. For example, [QCPAxisRect](#), [QCPLegend](#) and [QCPTextElement](#) are of this category. This does not necessarily mean that the latter category can't have child layout elements. [QCPLegend](#) for instance, actually derives from [QCPLayoutGrid](#) and the individual legend items are child layout elements in the grid layout.

15.600.2 Member Enumeration Documentation

15.600.2.1 SizeConstraintRect

enum `QCPLayOutElement::SizeConstraintRect`

Defines to which rect of a layout element the size constraints that can be set via `setMinimumSize` and `setMaximumSize` apply. The outer rect (`outerRect`) includes the margins (e.g. in the case of a `QCPAxisRect` the axis labels), whereas the inner rect (`rect`) does not.

See also

[setSizeConstraintRect](#)

Enumerator

<code>scrInnerRect</code>	Minimum/Maximum size constraints apply to inner rect.
<code>scrOuterRect</code>	Minimum/Maximum size constraints apply to outer rect, thus include layout element margins.

15.600.2.2 UpdatePhase

enum `QCPLayOutElement::UpdatePhase`

Defines the phases of the update process, that happens just before a replot. At each phase, `update` is called with the according UpdatePhase value.

Enumerator

<code>upPreparation</code>	Phase used for any type of preparation that needs to be done before margin calculation and layout.
<code>upMargins</code>	Phase in which the margins are calculated and set.
<code>upLayout</code>	Final phase in which the layout system places the rects of the elements.

15.600.3 Constructor & Destructor Documentation

15.600.3.1 QCPLayOutElement()

```
QCPLayOutElement::QCPLayOutElement (
    QCustomPlot * parentPlot = nullptr ) [explicit]
```

Creates an instance of `QCPLayOutElement` and sets default values.

15.600.4 Member Function Documentation

15.600.4.1 applyDefaultAntialiasingHint()

```
virtual void QCPLayOutElement::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [inline], [protected], [virtual]
```

Implements `QCPLayerable`.

15.600.4.2 draw()

```
virtual void QCPLayOutElement::draw (
    QCPPainter * painter ) [inline], [protected], [virtual]
```

Implements `QCPLayerable`.

15.600.4.3 elements()

```
QList< QCPLayOutElement * > QCPLayOutElement::elements (
    bool recursive ) const [virtual]
```

Returns a list of all child elements in this layout element. If *recursive* is true, all sub-child elements are included in the list, too.

Warning

There may be `nullptr` entries in the returned list. For example, [QCPLayoutGrid](#) may have empty cells which yield `nullptr` at the respective index.

Reimplemented in [QCPLayout](#), [QCPLayoutGrid](#), [QCPAxisRect](#), and [QCPPolarAxisAngular](#).

15.600.4.4 layout()

```
QCPLayout * QCPLayoutElement::layout ( ) const [inline]
```

Returns the parent layout of this layout element.

15.600.4.5 maximumOuterSizeHint()

```
QSize QCPLayoutElement::maximumOuterSizeHint ( ) const [virtual]
```

Returns the suggested maximum size this layout element (the [outerRect](#)) may be expanded to, if no manual maximum size is set.

if a maximum size ([setMaximumSize](#)) was not set manually, parent layouts use the returned size (usually indirectly through [QCPLayout::getFinalMaximumOuterSize](#)) to determine the maximum allowed size of this layout element.

A manual maximum size is considered set if it is smaller than Qt's `QWIDGETSIZE_MAX`.

The default implementation simply returns `QWIDGETSIZE_MAX` for both width and height, implying no suggested maximum size. Reimplementations may use their detailed knowledge about the layout element's content to provide size hints.

Reimplemented in [QCPLayoutGrid](#), and [QCPTextElement](#).

15.600.4.6 minimumOuterSizeHint()

```
QSize QCPLayoutElement::minimumOuterSizeHint ( ) const [virtual]
```

Returns the suggested minimum size this layout element (the [outerRect](#)) may be compressed to, if no manual minimum size is set.

if a minimum size ([setMinimumSize](#)) was not set manually, parent layouts use the returned size (usually indirectly through [QCPLayout::getFinalMinimumOuterSize](#)) to determine the minimum allowed size of this layout element.

A manual minimum size is considered set if it is non-zero.

The default implementation simply returns the sum of the horizontal margins for the width and the sum of the vertical margins for the height. Reimplementations may use their detailed knowledge about the layout element's content to provide size hints.

Reimplemented in [QCPLayoutGrid](#), [QCPPlottableLegendItem](#), [QCPTextElement](#), and [QCPPolarLegendItem](#).

15.600.4.7 outerRect()

```
QRect QCPLayoutElement::outerRect ( ) const [inline]
```

Returns the outer rect of this layout element. The outer rect is the inner rect expanded by the margins ([setMargins](#), [setAutoMargins](#)). The outer rect is used (and set via [setOuterRect](#)) by the parent [QCPLayout](#) to control the size of this layout element.

See also

[rect](#)

15.600.4.8 parentPlotInitialized()

```
void QCPLayoutElement::parentPlotInitialized (
    QCustomPlot * parentPlot ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.600.4.9 rect()

```
QRect QCPLayoutElement::rect ( ) const [inline]
```

Returns the inner rect of this layout element. The inner rect is the outer rect ([outerRect](#), [setOuterRect](#)) shrunk by the margins ([setMargins](#), [setAutoMargins](#)).

In some cases, the area between outer and inner rect is left blank. In other cases the margin area is used to display peripheral graphics while the main content is in the inner rect. This is where automatic margin calculation becomes interesting because it allows the layout element to adapt the margins to the peripheral graphics it wants to draw. For example, [QCPAxisRect](#) draws the axis labels and tick labels in the margin area, thus needs to adjust the margins (if [setAutoMargins](#) is enabled) according to the space required by the labels of the axes.

See also

[outerRect](#)

15.600.4.10 selectTest()

```
double QCPLayoutElement::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Layout elements are sensitive to events inside their outer rect. If *pos* is within the outer rect, this method returns a value corresponding to 0.99 times the parent plot's selection tolerance. However, layout elements are not selectable by default. So if *onlySelectable* is true, -1.0 is returned.

See [QCPLayerable::selectTest](#) for a general explanation of this virtual method.

[QCPLayoutElement](#) subclasses may reimplement this method to provide more specific selection test behaviour.

Reimplemented from [QCPLayerable](#).

Reimplemented in [QCPPolarAxisAngular](#), [QCPLayoutInset](#), [QCPAbstractLegendItem](#), [QCPLegend](#), and [QCPTextElement](#).

15.600.4.11 setAutoMargins()

```
void QCPLayoutElement::setAutoMargins (
    QCP::MarginSides sides )
```

Sets on which sides the margin shall be calculated automatically. If a side is calculated automatically, a minimum margin value may be provided with [setMinimumMargins](#). If a side is set to be controlled manually, the value may be specified with [setMargins](#).

Margin sides that are under automatic control may participate in a [QCPMarginGroup](#) (see [setMarginGroup](#)), to synchronize (align) it with other layout elements in the plot.

See also

[setMinimumMargins](#), [setMargins](#), [QCP::MarginSide](#)

15.600.4.12 setMarginGroup()

```
void QCPLayoutElement::setMarginGroup (
    QCP::MarginSides sides,
    QCPMarginGroup * group )
```

Sets the margin *group* of the specified margin *sides*.

Margin groups allow synchronizing specified margins across layout elements, see the documentation of [QCPMarginGroup](#).

To unset the margin group of *sides*, set *group* to `nullptr`.

Note that margin groups only work for margin sides that are set to automatic ([setAutoMargins](#)).

See also

[QCP::MarginSide](#)

15.600.4.13 setMargins()

```
void QCPLayoutElement::setMargins (
    const QMargins & margins )
```

Sets the margins of this layout element. If [setAutoMargins](#) is disabled for some or all sides, this function is used to manually set the margin on those sides. Sides that are still set to be handled automatically are ignored and may have any value in *margins*.

The margin is the distance between the outer rect (controlled by the parent layout via [setOuterRect](#)) and the inner [rect](#) (which usually contains the main content of this layout element).

See also

[setAutoMargins](#)

15.600.4.14 setMaximumSize() [1/2]

```
void QCPLayoutElement::setMaximumSize (
    const QSize & size )
```

Sets the maximum size of this layout element. A parent layout tries to respect the *size* here by changing row/column sizes in the layout accordingly.

Whether this constraint applies to the inner or the outer rect can be specified with [setSizeConstraintRect](#) (see [rect](#) and [outerRect](#)).

15.600.4.15 setMaximumSize() [2/2]

```
void QCPLayoutElement::setMaximumSize (
    int width,
    int height )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the maximum size of this layout element.

Whether this constraint applies to the inner or the outer rect can be specified with [setSizeConstraintRect](#) (see [rect](#) and [outerRect](#)).

15.600.4.16 setMinimumMargins()

```
void QCPLayoutElement::setMinimumMargins (
    const QMargins & margins )
```

If [setAutoMargins](#) is enabled on some or all margins, this function is used to provide minimum values for those margins.

The minimum values are not enforced on margin sides that were set to be under manual control via [setAutoMargins](#).

See also

[setAutoMargins](#)

15.600.4.17 setMinimumSize() [1/2]

```
void QCPLayoutElement::setMinimumSize (
    const QSize & size )
```

Sets the minimum size of this layout element. A parent layout tries to respect the *size* here by changing row/column sizes in the layout accordingly.

If the parent layout size is not sufficient to satisfy all minimum size constraints of its child layout elements, the layout may set a size that is actually smaller than *size*. [QCustomPlot](#) propagates the layout's size constraints to the outside by setting its own minimum QWidget size accordingly, so violations of *size* should be exceptions.

Whether this constraint applies to the inner or the outer rect can be specified with [setSizeConstraintRect](#) (see [rect](#) and [outerRect](#)).

15.600.4.18 setMinimumSize() [2/2]

```
void QCPLayoutElement::setMinimumSize (
    int width,
    int height )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the minimum size of this layout element.

Whether this constraint applies to the inner or the outer rect can be specified with [setSizeConstraintRect](#) (see [rect](#) and [outerRect](#)).

15.600.4.19 setOuterRect()

```
void QCPLayoutElement::setOuterRect (
    const QRect & rect )
```

Sets the outer rect of this layout element. If the layout element is inside a layout, the layout sets the position and size of this layout element using this function.

Calling this function externally has no effect, since the layout will overwrite any changes to the outer rect upon the next replot.

The layout element will adapt its inner [rect](#) by applying the margins inward to the outer rect.

See also

[rect](#)

15.600.4.20 setSizeConstraintRect()

```
void QCPLayoutElement::setSizeConstraintRect (
    SizeConstraintRect constraintRect )
```

Sets to which rect of a layout element the size constraints apply. Size constraints can be set via [setMinimumSize](#) and [setMaximumSize](#).

The outer rect ([outerRect](#)) includes the margins (e.g. in the case of a [QCPAxisRect](#) the axis labels), whereas the inner rect ([rect](#)) does not.

See also

[setMinimumSize](#), [setMaximumSize](#)

15.600.4.21 update()

```
void QCPLayoutElement::update (
    UpdatePhase phase ) [virtual]
```

Updates the layout element and sub-elements. This function is automatically called before every replot by the parent layout element. It is called multiple times, once for every [UpdatePhase](#). The phases are run through in the order of the enum values. For details about what happens at the different phases, see the documentation of [UpdatePhase](#). Layout elements that have child elements should call the [update](#) method of their child elements, and pass the current *phase* unchanged.

The default implementation executes the automatic margin mechanism in the [upMargins](#) phase. Subclasses should make sure to call the base class implementation.

Reimplemented in [QCPLayout](#), [QCPAxisRect](#), [QCPColorScaleAxisRectPrivate](#), [QCPColorScale](#), and [QCPPolarAxisAngular](#).

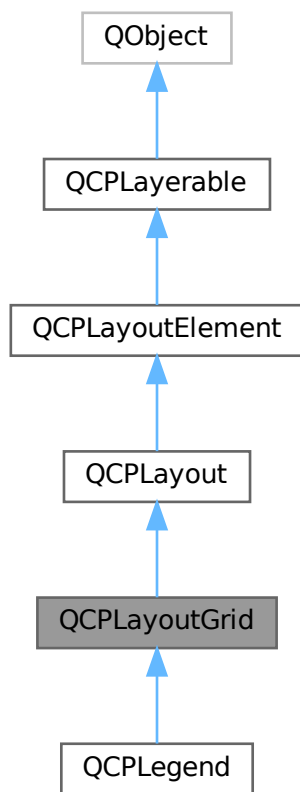
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

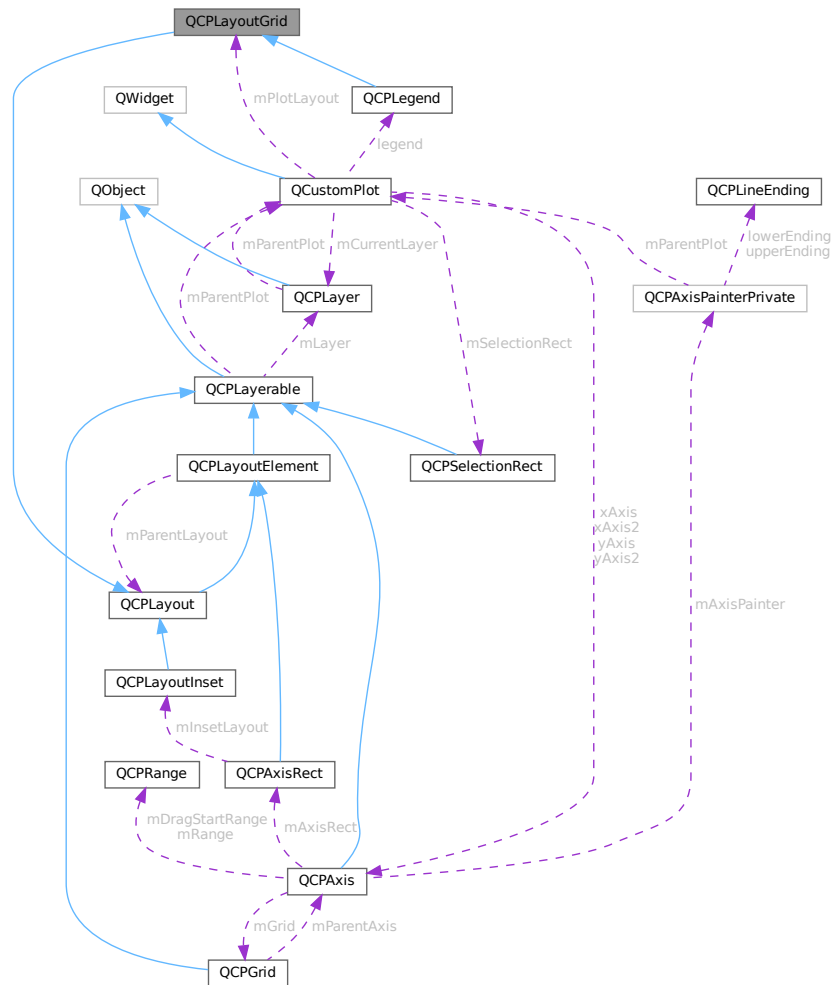
15.601 QCPLayoutGrid Class Reference

A layout that arranges child elements in a grid.

Inheritance diagram for QCPLayoutGrid:



Collaboration diagram for QCPLLayoutGrid:



Public Types

- enum [FillOrder](#) { [foRowsFirst](#) , [foColumnsFirst](#) }

Public Types inherited from [QCPLLayoutElement](#)

- enum [UpdatePhase](#) { [upPreparation](#) , [upMargins](#) , [upLayout](#) }
- enum [SizeConstraintRect](#) { [scrInnerRect](#) , [scrOuterRect](#) }

Public Member Functions

- [QCPLLayoutGrid](#) ()
- int [rowCount](#) () const
- int [columnCount](#) () const
- QList< double > [columnStretchFactors](#) () const
- QList< double > [rowStretchFactors](#) () const
- int [columnSpacing](#) () const
- int [rowSpacing](#) () const
- int [wrap](#) () const
- [FillOrder](#) [fillOrder](#) () const

- void [setColumnStretchFactor](#) (int column, double factor)
- void [setColumnStretchFactors](#) (const QList< double > &factors)
- void [setRowStretchFactor](#) (int row, double factor)
- void [setRowStretchFactors](#) (const QList< double > &factors)
- void [setColumnSpacing](#) (int pixels)
- void [setRowSpacing](#) (int pixels)
- void [setWrap](#) (int count)
- void [setFillOrder](#) ([FillOrder](#) order, bool rearrange=true)
- virtual void [updateLayout](#) () Q_DECL_OVERRIDE
- virtual int [elementCount](#) () const Q_DECL_OVERRIDE
- virtual [QCPLayoutElement](#) * [elementAt](#) (int index) const Q_DECL_OVERRIDE
- virtual [QCPLayoutElement](#) * [takeAt](#) (int index) Q_DECL_OVERRIDE
- virtual bool [take](#) ([QCPLayoutElement](#) *element) Q_DECL_OVERRIDE
- virtual QList< [QCPLayoutElement](#) * > [elements](#) (bool recursive) const Q_DECL_OVERRIDE
- virtual void [simplify](#) () Q_DECL_OVERRIDE
- virtual QSize [minimumOuterSizeHint](#) () const Q_DECL_OVERRIDE
- virtual QSize [maximumOuterSizeHint](#) () const Q_DECL_OVERRIDE
- [QCPLayoutElement](#) * [element](#) (int row, int column) const
- bool [addElement](#) (int row, int column, [QCPLayoutElement](#) *element)
- bool [addElement](#) ([QCPLayoutElement](#) *element)
- bool [hasElement](#) (int row, int column)
- void [expandTo](#) (int newRowCount, int newColumnCount)
- void [insertRow](#) (int newIndex)
- void [insertColumn](#) (int newIndex)
- int [rowColToIndex](#) (int row, int column) const
- void [indexToRowCol](#) (int index, int &row, int &column) const

Public Member Functions inherited from [QCPLayout](#)

- [QCPLayout](#) ()
- virtual void [update](#) ([UpdatePhase](#) phase) Q_DECL_OVERRIDE
- bool [removeAt](#) (int index)
- bool [remove](#) ([QCPLayoutElement](#) *element)
- void [clear](#) ()

Public Member Functions inherited from [QCPLayoutElement](#)

- [QCPLayoutElement](#) ([QCustomPlot](#) *parentPlot=nullptr)
- [QCPLayout](#) * [layout](#) () const
- QRect [rect](#) () const
- QRect [outerRect](#) () const
- QMargins [margins](#) () const
- QMargins [minimumMargins](#) () const
- QCP::MarginSides [autoMargins](#) () const
- QSize [minimumSize](#) () const
- QSize [maximumSize](#) () const
- [SizeConstraintRect](#) [sizeConstraintRect](#) () const
- [QCPMarginGroup](#) * [marginGroup](#) (QCP::MarginSide side) const
- QHash< QCP::MarginSide, [QCPMarginGroup](#) * > [marginGroups](#) () const
- void [setOuterRect](#) (const QRect &rect)
- void [setMargins](#) (const QMargins &margins)
- void [setMinimumMargins](#) (const QMargins &margins)
- void [setAutoMargins](#) (QCP::MarginSides sides)
- void [setMinimumSize](#) (const QSize &size)
- void [setMinimumSize](#) (int width, int height)

- void [setMaximumSize](#) (const QSize &size)
- void [setMaximumSize](#) (int width, int height)
- void [setSizeConstraintRect](#) ([SizeConstraintRect](#) constraintRect)
- void [setMarginGroup](#) (QCP::MarginSides sides, [QCPMarginGroup](#) *group)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- void [getMinimumRowColSizes](#) (QVector< int > *minColWidths, QVector< int > *minRowHeights) const
- void [getMaximumRowColSizes](#) (QVector< int > *maxColWidths, QVector< int > *maxRowHeights) const

Protected Member Functions inherited from [QCPLayout](#)

- void [sizeConstraintsChanged](#) () const
- void [adoptElement](#) ([QCPLayoutElement](#) *el)
- void [releaseElement](#) ([QCPLayoutElement](#) *el)
- QVector< int > [getSectionSizes](#) (QVector< int > maxSizes, QVector< int > minSizes, QVector< double > stretchFactors, int totalSize) const

Protected Member Functions inherited from [QCPLayoutElement](#)

- virtual int [calculateAutoMargin](#) ([QCP::MarginSide](#) side)
- virtual void [layoutChanged](#) ()
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayerable](#)

- virtual [QCP::Interaction](#) **selectionCategory** () const
- virtual QRect **clipRect** () const
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void [initializeParentPlot](#) ([QCustomPlot](#) *parentPlot)
- void [setParentLayerable](#) ([QCPLayerable](#) *parentLayerable)
- bool [moveToLayer](#) ([QCPLayer](#) *layer, bool prepend)
- void [applyAntialiasingHint](#) ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- `QList< QList< QCPLayoutElement * > > mElements`
- `QList< double > mColumnStretchFactors`
- `QList< double > mRowStretchFactors`
- `int mColumnSpacing`
- `int mRowSpacing`
- `int mWrap`
- `FillOrder mFillOrder`

Protected Attributes inherited from [QCPLayoutElement](#)

- `QCPLayout * mParentLayout`
- `QSize mMinimumSize`
- `QSize mMaximumSize`
- `SizeConstraintRect mSizeConstraintRect`
- `QRect mRect`
- `QRect mOuterRect`
- `QMargins mMargins`
- `QMargins mMinimumMargins`
- `QCP::MarginSides mAutoMargins`
- `QHash< QCP::MarginSide, QCPMarginGroup * > mMarginGroups`

Protected Attributes inherited from [QCPLayerable](#)

- `bool mVisible`
- `QCustomPlot * mParentPlot`
- `QPointer< QCPLayerable > mParentLayerable`
- `QCPLayer * mLayer`
- `bool mAntialiased`

Additional Inherited Members**Signals inherited from [QCPLayerable](#)**

- `void layerChanged (QCPLayer *newLayer)`

Static Protected Member Functions inherited from [QCPLayout](#)

- `static QSize getFinalMinimumOuterSize (const QCPLayoutElement *el)`
- `static QSize getFinalMaximumOuterSize (const QCPLayoutElement *el)`

15.601.1 Detailed Description

A layout that arranges child elements in a grid.

Elements are laid out in a grid with configurable stretch factors ([setColumnStretchFactor](#), [setRowStretchFactor](#)) and spacing ([setColumnSpacing](#), [setRowSpacing](#)).

Elements can be added to cells via [addElement](#). The grid is expanded if the specified row or column doesn't exist yet. Whether a cell contains a valid layout element can be checked with [hasElement](#), that element can be retrieved with [element](#). If rows and columns that only have empty cells shall be removed, call [simplify](#). Removal of elements is either done by just adding the element to a different layout or by using the [QCPLayout](#) interface [take](#) or [remove](#).

If you use [addElement\(QCPLayoutElement*\)](#) without explicit parameters for *row* and *column*, the grid layout will choose the position according to the current [setFillOrder](#) and the wrapping ([setWrap](#)).

Row and column insertion can be performed with [insertRow](#) and [insertColumn](#).

15.601.2 Member Enumeration Documentation

15.601.2.1 FillOrder

enum `QCPLayoutGrid::FillOrder`

Defines in which direction the grid is filled when using `addElement(QCPLayoutElement*)`. The column/row at which wrapping into the next row/column occurs can be specified with `setWrap`.

See also

[setFillOrder](#)

Enumerator

foRowsFirst	Rows are filled first, and a new element is wrapped to the next column if the row count would exceed setWrap .
foColumnsFirst	Columns are filled first, and a new element is wrapped to the next row if the column count would exceed setWrap .

15.601.3 Constructor & Destructor Documentation

15.601.3.1 QCPLayoutGrid()

`QCPLayoutGrid::QCPLayoutGrid ( ) [explicit]`

Creates an instance of [QCPLayoutGrid](#) and sets default values.

15.601.4 Member Function Documentation

15.601.4.1 addElement() [1/2]

```
bool QCPLayoutGrid::addElement (
    int row,
    int column,
    QCPLayoutElement * element )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the *element* to cell with *row* and *column*. If *element* is already in a layout, it is first removed from there. If *row* or *column* don't exist yet, the layout is expanded accordingly.

Returns true if the element was added successfully, i.e. if the cell at *row* and *column* didn't already have an element. Use the overload of this method without explicit row/column index to place the element according to the configured fill order and wrapping settings.

See also

[element](#), [hasElement](#), [take](#), [remove](#)

15.601.4.2 addElement() [2/2]

```
bool QCPLayoutGrid::addElement (
    QCPLayoutElement * element )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the *element* to the next empty cell according to the current fill order ([setFillOrder](#)) and wrapping ([setWrap](#)). If *element* is already in a layout, it is first removed from there. If necessary, the layout is expanded to hold the new element.

Returns true if the element was added successfully.

See also

[setFillOrder](#), [setWrap](#), [element](#), [hasElement](#), [take](#), [remove](#)

15.601.4.3 columnCount()

```
int QCPLayoutGrid::columnCount ( ) const [inline]
```

Returns the number of columns in the layout.

See also

[rowCount](#)

15.601.4.4 element()

```
QCPLayoutElement * QCPLayoutGrid::element (
    int row,
    int column ) const
```

Returns the element in the cell in *row* and *column*.

Returns `nullptr` if either the row/column is invalid or if the cell is empty. In those cases, a `qDebug` message is printed. To check whether a cell exists and isn't empty, use [hasElement](#).

See also

[addElement](#), [hasElement](#)

15.601.4.5 elementAt()

```
QCPLayoutElement * QCPLayoutGrid::elementAt (
    int index ) const [virtual]
```

\see `baseclassmethod`

Note that the association of the linear *index* to the row/column based cells depends on the current setting of [setFillOrder](#).

See also

[rowColToIndex](#)

Implements [QCPLayout](#).

15.601.4.6 elementCount()

```
virtual int QCPLayoutGrid::elementCount ( ) const [inline], [virtual]
```

Returns the number of elements/cells in the layout.

See also

[elements](#), [elementAt](#)

Implements [QCPLayout](#).

15.601.4.7 elements()

```
QList< QCPLayoutElement * > QCPLayoutGrid::elements (
    bool recursive ) const [virtual]
```

Returns a list of all child elements in this layout element. If *recursive* is true, all sub-child elements are included in the list, too.

Warning

There may be `nullptr` entries in the returned list. For example, [QCPLayoutGrid](#) may have empty cells which yield `nullptr` at the respective index.

Reimplemented from [QCPLayout](#).

15.601.4.8 expandTo()

```
void QCPLayoutGrid::expandTo (
    int newRowCount,
    int newColumnCount )
```

Expands the layout to have *newRowCount* rows and *newColumnCount* columns. So the last valid row index will be *newRowCount-1*, the last valid column index will be *newColumnCount-1*.

If the current column/row count is already larger or equal to *newColumnCount/newRowCount*, this function does nothing in that dimension.

Newly created cells are empty, new rows and columns have the stretch factor 1.

Note that upon a call to [addElement](#), the layout is expanded automatically to contain the specified row and column, using this function.

See also

[simplify](#)

15.601.4.9 hasElement()

```
bool QCPLayoutGrid::hasElement (
    int row,
    int column )
```

Returns whether the cell at *row* and *column* exists and contains a valid element, i.e. isn't empty.

See also

[element](#)

15.601.4.10 indexToRowCol()

```
void QCPLayoutGrid::indexToRowCol (
    int index,
    int & row,
    int & column ) const
```

Converts the linear index to row and column indices and writes the result to *row* and *column*.

The way the cells are indexed depends on [setFillOrder](#). If it is [foRowsFirst](#), the indices increase left to right and then top to bottom. If it is [foColumnsFirst](#), the indices increase top to bottom and then left to right.

If there are no cells (i.e. column or row count is zero), sets *row* and *column* to -1.

For the retrieved *row* and *column* to be valid, the passed *index* must be valid itself, i.e. greater or equal to zero and smaller than the current [elementCount](#).

See also

[rowColToIndex](#)

15.601.4.11 insertColumn()

```
void QCPLayoutGrid::insertColumn (
    int newIndex )
```

Inserts a new column with empty cells at the column index *newIndex*. Valid values for *newIndex* range from 0 (inserts a column at the left) to *columnCount* (appends a column at the right).

See also

[insertRow](#)

15.601.4.12 insertRow()

```
void QCPLayoutGrid::insertRow (
    int newIndex )
```

Inserts a new row with empty cells at the row index *newIndex*. Valid values for *newIndex* range from 0 (inserts a row at the top) to *rowCount* (appends a row at the bottom).

See also

[insertColumn](#)

15.601.4.13 maximumOuterSizeHint()

```
QSize QCPLayoutGrid::maximumOuterSizeHint ( ) const [virtual]
```

Returns the suggested maximum size this layout element (the [outerRect](#)) may be expanded to, if no manual maximum size is set.

if a maximum size ([setMaximumSize](#)) was not set manually, parent layouts use the returned size (usually indirectly through [QCPLayout::getFinalMaximumOuterSize](#)) to determine the maximum allowed size of this layout element.

A manual maximum size is considered set if it is smaller than Qt's `QWIDGETSIZE_MAX`.

The default implementation simply returns `QWIDGETSIZE_MAX` for both width and height, implying no suggested maximum size. Reimplementations may use their detailed knowledge about the layout element's content to provide size hints.

Reimplemented from [QCPLayoutElement](#).

15.601.4.14 minimumOuterSizeHint()

```
QSize QCPLayoutGrid::minimumOuterSizeHint ( ) const [virtual]
```

Returns the suggested minimum size this layout element (the [outerRect](#)) may be compressed to, if no manual minimum size is set.

if a minimum size ([setMinimumSize](#)) was not set manually, parent layouts use the returned size (usually indirectly through [QCPLayout::getFinalMinimumOuterSize](#)) to determine the minimum allowed size of this layout element.

A manual minimum size is considered set if it is non-zero.

The default implementation simply returns the sum of the horizontal margins for the width and the sum of the vertical margins for the height. Reimplementations may use their detailed knowledge about the layout element's content to provide size hints.

Reimplemented from [QCPLayoutElement](#).

15.601.4.15 rowColToIndex()

```
int QCPLayoutGrid::rowColToIndex (
    int row,
    int column ) const
```

Converts the given *row* and *column* to the linear index used by some methods of [QCPLayoutGrid](#) and [QCPLayout](#).

The way the cells are indexed depends on [setFillOrder](#). If it is [foRowsFirst](#), the indices increase left to right and then top to bottom. If it is [foColumnsFirst](#), the indices increase top to bottom and then left to right.

For the returned index to be valid, *row* and *column* must be valid indices themselves, i.e. greater or equal to zero and smaller than the current `rowCount/ columnCount`.

See also

[indexToRowCol](#)

15.601.4.16 rowCount()

```
int QCPLayoutGrid::rowCount ( ) const [inline]
```

Returns the number of rows in the layout.

See also

[columnCount](#)

15.601.4.17 setColumnSpacing()

```
void QCPLayoutGrid::setColumnSpacing (
    int pixels )
```

Sets the gap that is left blank between columns to *pixels*.

See also

[setRowSpacing](#)

15.601.4.18 setColumnStretchFactor()

```
void QCPLayoutGrid::setColumnStretchFactor (
    int column,
    double factor )
```

Sets the stretch *factor* of *column*.

Stretch factors control the relative sizes of rows and columns. Cells will not be resized beyond their minimum and maximum widths/heights, regardless of the stretch factor. (see [QCPLayoutElement::setMinimumSize](#), [QCPLayoutElement::setMaximumSize](#), [QCPLayoutElement::setSizeConstraintRect](#).)

The default stretch factor of newly created rows/columns is 1.

See also

[setColumnStretchFactors](#), [setRowStretchFactor](#)

15.601.4.19 setColumnStretchFactors()

```
void QCPLayoutGrid::setColumnStretchFactors (
    const QList< double > & factors )
```

Sets the stretch *factors* of all columns. *factors* must have the size [columnCount](#).

Stretch factors control the relative sizes of rows and columns. Cells will not be resized beyond their minimum and maximum widths/heights, regardless of the stretch factor. (see [QCPLayoutElement::setMinimumSize](#), [QCPLayoutElement::setMaximumSize](#), [QCPLayoutElement::setSizeConstraintRect](#).)

The default stretch factor of newly created rows/columns is 1.

See also

[setColumnStretchFactor](#), [setRowStretchFactors](#)

15.601.4.20 setFillOrder()

```
void QCPLayoutGrid::setFillOrder (
    FillOrder order,
    bool rearrange = true )
```

Sets the filling order and wrapping behaviour that is used when adding new elements with the method [addElement\(QCPLayoutElement*\)](#).

The specified *order* defines whether rows or columns are filled first. Using [setWrap](#), you can control at which row/column count wrapping into the next column/row will occur. If you set it to zero, no wrapping will ever occur. Changing the fill order also changes the meaning of the linear index used e.g. in [elementAt](#) and [takeAt](#). The default fill order for [QCPLayoutGrid](#) is [foColumnsFirst](#).

If you want to have all current elements arranged in the new order, set *rearrange* to true. The elements will be rearranged in a way that tries to preserve their linear index. However, empty cells are skipped during build-up of the new cell order, which shifts the succeeding element's index. The rearranging is performed even if the specified *order* is already the current fill order. Thus this method can be used to re-wrap the current elements.

If *rearrange* is false, the current element arrangement is not changed, which means the linear indexes change (because the linear index is dependent on the fill order).

Note that the method [addElement\(int row, int column, QCPLayoutElement *element\)](#) with explicitly stated row and column is not subject to wrapping and can place elements even beyond the specified wrapping point.

See also

[setWrap](#), [addElement\(QCPLayoutElement*\)](#)

15.601.4.21 setRowSpacing()

```
void QCPLayoutGrid::setRowSpacing (
    int pixels )
```

Sets the gap that is left blank between rows to *pixels*.

See also

[setColumnSpacing](#)

15.601.4.22 setRowStretchFactor()

```
void QCPLayoutGrid::setRowStretchFactor (
    int row,
    double factor )
```

Sets the stretch *factor* of *row*.

Stretch factors control the relative sizes of rows and columns. Cells will not be resized beyond their minimum and maximum widths/heights, regardless of the stretch factor. (see [QCPLayoutElement::setMinimumSize](#), [QCPLayoutElement::setMaximumSize](#), [QCPLayoutElement::setSizeConstraintRect](#).)

The default stretch factor of newly created rows/columns is 1.

See also

[setColumnStretchFactors](#), [setRowStretchFactor](#)

15.601.4.23 setRowStretchFactors()

```
void QCPLayoutGrid::setRowStretchFactors (
    const QList< double > & factors )
```

Sets the stretch *factors* of all rows. *factors* must have the size [rowCount](#).

Stretch factors control the relative sizes of rows and columns. Cells will not be resized beyond their minimum and maximum widths/heights, regardless of the stretch factor. (see [QCPLayoutElement::setMinimumSize](#), [QCPLayoutElement::setMaximumSize](#), [QCPLayoutElement::setSizeConstraintRect](#).)

The default stretch factor of newly created rows/columns is 1.

See also

[setRowStretchFactor](#), [setColumnStretchFactors](#)

15.601.4.24 setWrap()

```
void QCPLayoutGrid::setWrap (
    int count )
```

Sets the maximum number of columns or rows that are used, before new elements added with [addElement\(QCPLayoutElement*\)](#) will start to fill the next row or column, respectively. It depends on [setFillOrder](#), whether rows or columns are wrapped.

If *count* is set to zero, no wrapping will ever occur.

If you wish to re-wrap the elements currently in the layout, call [setFillOrder](#) with *rearrange* set to true (the actual fill order doesn't need to be changed for the rearranging to be done).

Note that the method [addElement\(int row, int column, QCPLayoutElement *element\)](#) with explicitly stated row and column is not subject to wrapping and can place elements even beyond the specified wrapping point.

See also

[setFillOrder](#)

15.601.4.25 simplify()

```
void QCPLayoutGrid::simplify ( ) [virtual]
```

Simplifies the layout by collapsing rows and columns which only contain empty cells.

Reimplemented from [QCPLayout](#).

15.601.4.26 take()

```
bool QCPLayoutGrid::take (
    QCPLayoutElement * element ) [virtual]
```

Removes the specified *element* from the layout and returns true on success.

If the *element* isn't in this layout, returns false.

Note that some layouts don't remove the respective cell right away but leave an empty cell after successful removal of the layout element. To collapse empty cells, use [simplify](#).

See also

[takeAt](#)

Implements [QCPLayout](#).

15.601.4.27 takeAt()

```
QCPLayoutElement * QCPLayoutGrid::takeAt (
    int index ) [virtual]
```

\see [baseclassmethod](#)

Note that the association of the linear *index* to the row/column based cells depends on the current setting of [setFillOrder](#).

See also

[rowColToIndex](#)

Implements [QCPLayout](#).

15.601.4.28 updateLayout()

```
void QCPLayoutGrid::updateLayout ( ) [virtual]
```

Reimplemented from [QCPLayout](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.602 QCPLayoutInset Class Reference

A layout that places child elements aligned to the border or arbitrarily positioned.

Public Types

- enum [InsetPlacement](#) { [ipFree](#) , [ipBorderAligned](#) }

Public Types inherited from [QCPLayoutElement](#)

- enum [UpdatePhase](#) { [upPreparation](#) , [upMargins](#) , [upLayout](#) }
- enum [SizeConstraintRect](#) { [scrInnerRect](#) , [scrOuterRect](#) }

Public Member Functions

- [QCPLayoutInset](#) ()
- [InsetPlacement](#) [insetPlacement](#) (int index) const
- Qt::Alignment [insetAlignment](#) (int index) const
- QRectF [insetRect](#) (int index) const
- void [setInsetPlacement](#) (int index, [InsetPlacement](#) placement)
- void [setInsetAlignment](#) (int index, Qt::Alignment alignment)
- void [setInsetRect](#) (int index, const QRectF &[rect](#))
- virtual void [updateLayout](#) () Q_DECL_OVERRIDE
- virtual int [elementCount](#) () const Q_DECL_OVERRIDE
- virtual [QCPLayoutElement](#) * [elementAt](#) (int index) const Q_DECL_OVERRIDE
- virtual [QCPLayoutElement](#) * [takeAt](#) (int index) Q_DECL_OVERRIDE
- virtual bool [take](#) ([QCPLayoutElement](#) *element) Q_DECL_OVERRIDE
- virtual void [simplify](#) () Q_DECL_OVERRIDE
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE
- void [addElement](#) ([QCPLayoutElement](#) *element, Qt::Alignment alignment)
- void [addElement](#) ([QCPLayoutElement](#) *element, const QRectF &[rect](#))

Public Member Functions inherited from [QCPLayout](#)

- [QCPLayout](#) ()
- virtual void [update](#) ([UpdatePhase](#) phase) Q_DECL_OVERRIDE
- virtual QList< [QCPLayoutElement](#) * > [elements](#) (bool recursive) const Q_DECL_OVERRIDE
- bool [removeAt](#) (int index)
- bool [remove](#) ([QCPLayoutElement](#) *element)
- void [clear](#) ()

Public Member Functions inherited from [QCPLayoutElement](#)

- [QCPLayoutElement](#) ([QCustomPlot](#) *parentPlot=nullptr)
- [QCPLayout](#) * [layout](#) () const
- QRect [rect](#) () const
- QRect [outerRect](#) () const
- QMargins [margins](#) () const
- QMargins [minimumMargins](#) () const
- QCP::MarginSides [autoMargins](#) () const
- QSize [minimumSize](#) () const
- QSize [maximumSize](#) () const
- [SizeConstraintRect](#) [sizeConstraintRect](#) () const
- [QCPMarginGroup](#) * [marginGroup](#) ([QCP::MarginSide](#) side) const
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > [marginGroups](#) () const
- void [setOuterRect](#) (const QRect &[rect](#))
- void [setMargins](#) (const QMargins &margins)
- void [setMinimumMargins](#) (const QMargins &margins)
- void [setAutoMargins](#) ([QCP::MarginSides](#) sides)

- void [setMinimumSize](#) (const QSize &size)
- void [setMinimumSize](#) (int width, int height)
- void [setMaximumSize](#) (const QSize &size)
- void [setMaximumSize](#) (int width, int height)
- void [setSizeConstraintRect](#) ([SizeConstraintRect](#) constraintRect)
- void [setMarginGroup](#) (QCP::MarginSides sides, [QCPMarginGroup](#) *group)
- virtual QSize [minimumOuterSizeHint](#) () const
- virtual QSize [maximumOuterSizeHint](#) () const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Attributes

- QList< [QCPLayoutElement](#) * > **mElements**
- QList< [InsetPlacement](#) > **mInsetPlacement**
- QList< Qt::Alignment > **mInsetAlignment**
- QList< QRectF > **mInsetRect**

Protected Attributes inherited from [QCPLayoutElement](#)

- [QCPLayout](#) * **mParentLayout**
- QSize **mMinimumSize**
- QSize **mMaximumSize**
- [SizeConstraintRect](#) **mSizeConstraintRect**
- QRect **mRect**
- QRect **mOuterRect**
- QMargins **mMargins**
- QMargins **mMinimumMargins**
- QCP::MarginSides **mAutoMargins**
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **mMarginGroups**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

Protected Member Functions inherited from QCPLayout

- void [sizeConstraintsChanged](#) () const
- void **adoptElement** ([QCPLayoutElement](#) *el)
- void **releaseElement** ([QCPLayoutElement](#) *el)
- QVector< int > **getSectionSizes** (QVector< int > maxSizes, QVector< int > minSizes, QVector< double > stretchFactors, int totalSize) const

Protected Member Functions inherited from QCPLayoutElement

- virtual int **calculateAutoMargin** ([QCP::MarginSide](#) side)
- virtual void **layoutChanged** ()
- virtual void **applyDefaultAntialiasingHint** ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from QCPLayerable

- virtual [QCP::Interaction](#) **selectionCategory** () const
- virtual QRect **clipRect** () const
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Static Protected Member Functions inherited from QCPLayout

- static QSize **getFinalMinimumOuterSize** (const [QCPLayoutElement](#) *el)
- static QSize **getFinalMaximumOuterSize** (const [QCPLayoutElement](#) *el)

15.602.1 Detailed Description

A layout that places child elements aligned to the border or arbitrarily positioned.

Elements are placed either aligned to the border or at arbitrary position in the area of the layout. Which placement applies is controlled with the [InsetPlacement](#) ([setInsetPlacement](#)).

Elements are added via [addElement\(QCPLayoutElement *element, Qt::Alignment alignment\)](#) or [addElement\(QCPLayoutElement *element, QRect rect\)](#).

If the first method is used, the inset placement will default to [ipBorderAligned](#) and the element will be aligned according to the *alignment* parameter. The second method defaults to [ipFree](#) and allows placing elements at arbitrary position and size, defined by *rect*.

The alignment or rect can be set via [setInsetAlignment](#) or [setInsetRect](#), respectively.

This is the layout that every [QCPAxisRect](#) has as [QCPAxisRect::insetLayout](#).

15.602.2 Member Enumeration Documentation

15.602.2.1 InsetPlacement

enum [QCPLayoutInset::InsetPlacement](#)

Defines how the placement and sizing is handled for a certain element in a [QCPLayoutInset](#).

Enumerator

ipFree	The element may be positioned/sized arbitrarily, see setInsetRect .
ipBorderAligned	The element is aligned to one of the layout sides, see setInsetAlignment .

15.602.3 Constructor & Destructor Documentation

15.602.3.1 QCPLayoutInset()

```
QCPLayoutInset::QCPLayoutInset ( ) [explicit]
```

Creates an instance of [QCPLayoutInset](#) and sets default values.

15.602.4 Member Function Documentation

15.602.4.1 addElement() [1/2]

```
void QCPLayoutInset::addElement (
    QCPLayoutElement * element,
    const QRectF & rect )
```

Adds the specified *element* to the layout as an inset with free positioning/sizing ([setInsetAlignment](#) is initialized with [ipFree](#)). The position and size is set to *rect*.

rect is given in fractions of the whole inset layout rect. So an inset with rect (0, 0, 1, 1) will span the entire layout. An inset with rect (0.6, 0.1, 0.35, 0.35) will be in the top right corner of the layout, with 35% width and height of the parent layout.

See also

[addElement\(QCPLayoutElement *element, Qt::Alignment alignment\)](#)

15.602.4.2 addElement() [2/2]

```
void QCPLayoutInset::addElement (
    QCPLayoutElement * element,
    Qt::Alignment alignment )
```

Adds the specified *element* to the layout as an inset aligned at the border ([setInsetAlignment](#) is initialized with [ipBorderAligned](#)). The alignment is set to *alignment*.

alignment is an or combination of the following alignment flags: Qt::AlignLeft, Qt::AlignHCenter, Qt::AlignRight, Qt::AlignTop, Qt::AlignVCenter, Qt::AlignBottom. Any other alignment flags will be ignored.

See also

[addElement\(QCPLayoutElement *element, const QRectF &rect\)](#)

15.602.4.3 elementAt()

```
QCPLayoutElement * QCPLayoutInset::elementAt (
    int index ) const [virtual]
```

Returns the element in the cell with the given *index*. If *index* is invalid, returns `nullptr`.

Note that even if *index* is valid, the respective cell may be empty in some layouts (e.g. [QCPLayoutGrid](#)), so this function may return `nullptr` in those cases. You may use this function to check whether a cell is empty or not.

See also

[elements](#), [elementCount](#), [takeAt](#)

Implements [QCPLayout](#).

15.602.4.4 elementCount()

```
int QCPLayoutInset::elementCount ( ) const [virtual]
```

Returns the number of elements/cells in the layout.

See also

[elements](#), [elementAt](#)

Implements [QCPLayout](#).

15.602.4.5 insetAlignment()

```
Qt::Alignment QCPLayoutInset::insetAlignment (
    int index ) const
```

Returns the alignment of the element with the specified *index*. The alignment only has a meaning, if the inset placement ([setInsetPlacement](#)) is [ipBorderAligned](#).

15.602.4.6 insetPlacement()

```
QCPLayoutInset::InsetPlacement QCPLayoutInset::insetPlacement (
    int index ) const
```

Returns the placement type of the element with the specified *index*.

15.602.4.7 insetRect()

```
QRectF QCPLayoutInset::insetRect (
    int index ) const
```

Returns the rect of the element with the specified *index*. The rect only has a meaning, if the inset placement ([setInsetPlacement](#)) is [ipFree](#).

15.602.4.8 selectTest()

```
double QCPLayoutInset::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

The inset layout is sensitive to events only at areas where its (visible) child elements are sensitive. If the `selectTest` method of any of the child elements returns a positive number for *pos*, this method returns a value corresponding to 0.99 times the parent plot's selection tolerance. The inset layout is not selectable itself by default. So if *onlySelectable* is true, -1.0 is returned.

See [QCPLayerable::selectTest](#) for a general explanation of this virtual method.

Reimplemented from [QCPLayoutElement](#).

15.602.4.9 setInsetAlignment()

```
void QCPLayoutInset::setInsetAlignment (
    int index,
    Qt::Alignment alignment )
```

If the inset placement ([setInsetPlacement](#)) is [ipBorderAligned](#), this function is used to set the alignment of the element with the specified *index* to *alignment*.

alignment is an or combination of the following alignment flags: `Qt::AlignLeft`, `Qt::AlignHCenter`, `Qt::AlignRight`, `Qt::AlignTop`, `Qt::AlignVCenter`, `Qt::AlignBottom`. Any other alignment flags will be ignored.

15.602.4.10 setInsetPlacement()

```
void QCPLayoutInset::setInsetPlacement (
    int index,
    QCPLayoutInset::InsetPlacement placement )
```

Sets the inset placement type of the element with the specified *index* to *placement*.

See also

[InsetPlacement](#)

15.602.4.11 setInsetRect()

```
void QCPLayoutInset::setInsetRect (
    int index,
    const QRectF & rect )
```

If the inset placement ([setInsetPlacement](#)) is [ipFree](#), this function is used to set the position and size of the element with the specified *index* to *rect*.

rect is given in fractions of the whole inset layout rect. So an inset with rect (0, 0, 1, 1) will span the entire layout. An inset with rect (0.6, 0.1, 0.35, 0.35) will be in the top right corner of the layout, with 35% width and height of the parent layout.

Note that the minimum and maximum sizes of the embedded element ([QCPLayoutElement::setMinimumSize](#), [QCPLayoutElement::setMaximumSize](#)) are enforced.

15.602.4.12 simplify()

```
void QCPLayoutInset::simplify ( ) [inline], [virtual]
```

The QCPLInsetLayout does not need simplification since it can never have empty cells due to its linear index structure. This method does nothing.

Reimplemented from [QCPLayout](#).

15.602.4.13 take()

```
bool QCPLayoutInset::take (
    QCPLayoutElement * element ) [virtual]
```

Removes the specified *element* from the layout and returns true on success.

If the *element* isn't in this layout, returns false.

Note that some layouts don't remove the respective cell right away but leave an empty cell after successful removal of the layout element. To collapse empty cells, use [simplify](#).

See also

[takeAt](#)

Implements [QCPLayout](#).

15.602.4.14 takeAt()

```
QCPLayoutElement * QCPLayoutInset::takeAt (
    int index ) [virtual]
```

Removes the element with the given *index* from the layout and returns it.

If the *index* is invalid or the cell with that index is empty, returns `nullptr`.

Note that some layouts don't remove the respective cell right away but leave an empty cell after successful removal of the layout element. To collapse empty cells, use [simplify](#).

See also

[elementAt](#), [take](#)

Implements [QCPLayout](#).

15.602.4.15 updateLayout()

```
void QCPLayoutInset::updateLayout ( ) [virtual]
```

Reimplemented from [QCPLayout](#).

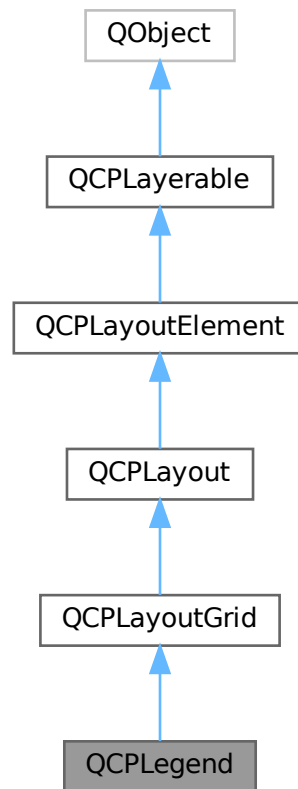
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.603 QCPLegend Class Reference

Manages a legend inside a [QCustomPlot](#).

Inheritance diagram for QCPLegend:



Public Member Functions

- [QCPLegend](#) ()
- QPen **borderPen** () const
- QBrush **brush** () const
- QFont **font** () const
- QColor **textColor** () const
- QSize **iconSize** () const
- int **iconTextPadding** () const
- QPen **iconBorderPen** () const
- SelectableParts **selectableParts** () const
- SelectableParts **selectedParts** () const
- QPen **selectedBorderPen** () const
- QPen **selectedIconBorderPen** () const
- QBrush **selectedBrush** () const
- QFont **selectedFont** () const
- QColor **selectedTextColor** () const
- void [setBorderPen](#) (const QPen &pen)
- void [setBrush](#) (const QBrush &brush)
- void [setFont](#) (const QFont &font)
- void [setTextColor](#) (const QColor &color)
- void [setIconSize](#) (const QSize &size)
- void [setIconSize](#) (int width, int height)
- void [setIconTextPadding](#) (int padding)
- void [setIconBorderPen](#) (const QPen &pen)
- Q_SLOT void [setSelectableParts](#) (const SelectableParts &selectableParts)
- Q_SLOT void [setSelectedParts](#) (const SelectableParts &selectedParts)
- void [setSelectedBorderPen](#) (const QPen &pen)
- void [setSelectedIconBorderPen](#) (const QPen &pen)
- void [setSelectedBrush](#) (const QBrush &brush)
- void [setSelectedFont](#) (const QFont &font)
- void [setSelectedTextColor](#) (const QColor &color)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q__←
DECL_OVERRIDE
- [QCPAbstractLegendItem](#) * [item](#) (int index) const
- [QCPPlottableLegendItem](#) * [itemWithPlottable](#) (const [QCPAbstractPlottable](#) *plottable) const
- int [itemCount](#) () const
- bool [hasItem](#) ([QCPAbstractLegendItem](#) *item) const
- bool [hasItemWithPlottable](#) (const [QCPAbstractPlottable](#) *plottable) const
- bool [addItem](#) ([QCPAbstractLegendItem](#) *item)
- bool [removeItem](#) (int index)
- bool [removeItem](#) ([QCPAbstractLegendItem](#) *item)
- void [clearItems](#) ()
- QList< [QCPAbstractLegendItem](#) * > [selectedItems](#) () const

Public Member Functions inherited from [QCPLayoutGrid](#)

- [QCPLayoutGrid](#) ()
- int [rowCount](#) () const
- int [columnCount](#) () const
- QList< double > **columnStretchFactors** () const
- QList< double > **rowStretchFactors** () const
- int **columnSpacing** () const
- int **rowSpacing** () const
- int **wrap** () const

- [FillOrder](#) **fillOrder** () const
- void [setColumnStretchFactor](#) (int column, double factor)
- void [setColumnStretchFactors](#) (const QList< double > &factors)
- void [setRowStretchFactor](#) (int row, double factor)
- void [setRowStretchFactors](#) (const QList< double > &factors)
- void [setColumnSpacing](#) (int pixels)
- void [setRowSpacing](#) (int pixels)
- void [setWrap](#) (int count)
- void [setFillOrder](#) ([FillOrder](#) order, bool rearrange=true)
- virtual void [updateLayout](#) () Q_DECL_OVERRIDE
- virtual int [elementCount](#) () const Q_DECL_OVERRIDE
- virtual [QCPLayoutElement](#) * [elementAt](#) (int index) const Q_DECL_OVERRIDE
- virtual [QCPLayoutElement](#) * [takeAt](#) (int index) Q_DECL_OVERRIDE
- virtual bool [take](#) ([QCPLayoutElement](#) *element) Q_DECL_OVERRIDE
- virtual QList< [QCPLayoutElement](#) * > [elements](#) (bool recursive) const Q_DECL_OVERRIDE
- virtual void [simplify](#) () Q_DECL_OVERRIDE
- virtual QSize [minimumOuterSizeHint](#) () const Q_DECL_OVERRIDE
- virtual QSize [maximumOuterSizeHint](#) () const Q_DECL_OVERRIDE
- [QCPLayoutElement](#) * [element](#) (int row, int column) const
- bool [addElement](#) (int row, int column, [QCPLayoutElement](#) *element)
- bool [addElement](#) ([QCPLayoutElement](#) *element)
- bool [hasElement](#) (int row, int column)
- void [expandTo](#) (int newRowCount, int newColumnCount)
- void [insertRow](#) (int newIndex)
- void [insertColumn](#) (int newIndex)
- int [rowColToIndex](#) (int row, int column) const
- void [indexToRowCol](#) (int index, int &row, int &column) const

Public Member Functions inherited from [QCPLayout](#)

- [QCPLayout](#) ()
- virtual void [update](#) ([UpdatePhase](#) phase) Q_DECL_OVERRIDE
- bool [removeAt](#) (int index)
- bool [remove](#) ([QCPLayoutElement](#) *element)
- void [clear](#) ()

Public Member Functions inherited from [QCPLayoutElement](#)

- [QCPLayoutElement](#) ([QCustomPlot](#) *parentPlot=nullptr)
- [QCPLayout](#) * [layout](#) () const
- [QRect](#) [rect](#) () const
- [QRect](#) [outerRect](#) () const
- [QMargins](#) [margins](#) () const
- [QMargins](#) [minimumMargins](#) () const
- [QCP::MarginSides](#) [autoMargins](#) () const
- [QSize](#) [minimumSize](#) () const
- [QSize](#) [maximumSize](#) () const
- [SizeConstraintRect](#) [sizeConstraintRect](#) () const
- [QCPMarginGroup](#) * [marginGroup](#) ([QCP::MarginSide](#) side) const
- [QHash](#)< [QCP::MarginSide](#), [QCPMarginGroup](#) * > [marginGroups](#) () const
- void [setOuterRect](#) (const [QRect](#) &rect)
- void [setMargins](#) (const [QMargins](#) &margins)
- void [setMinimumMargins](#) (const [QMargins](#) &margins)
- void [setAutoMargins](#) ([QCP::MarginSides](#) sides)
- void [setMinimumSize](#) (const [QSize](#) &size)

- void [setMinimumSize](#) (int width, int height)
- void [setMaximumSize](#) (const QSize &size)
- void [setMaximumSize](#) (int width, int height)
- void [setSizeConstraintRect](#) ([SizeConstraintRect](#) constraintRect)
- void [setMarginGroup](#) (QCP::MarginSides sides, [QCPMarginGroup](#) *group)

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE
- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionState← Changed) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- QPen **getBorderPen** () const
- QBrush **getBrush** () const

Protected Member Functions inherited from [QCPLayoutGrid](#)

- void **getMinimumRowColSizes** (QVector< int > *minColWidths, QVector< int > *minRowHeights) const
- void **getMaximumRowColSizes** (QVector< int > *maxColWidths, QVector< int > *maxRowHeights) const

Protected Member Functions inherited from [QCPLayout](#)

- void [sizeConstraintsChanged](#) () const
- void **adoptElement** ([QCPLayoutElement](#) *el)
- void **releaseElement** ([QCPLayoutElement](#) *el)
- QVector< int > **getSectionSizes** (QVector< int > maxSizes, QVector< int > minSizes, QVector< double > stretchFactors, int totalSize) const

Protected Member Functions inherited from [QCPLayoutElement](#)

- virtual int **calculateAutoMargin** ([QCP::MarginSide](#) side)
- virtual void **layoutChanged** ()

Protected Member Functions inherited from [QCPLayerable](#)

- virtual QRect **clipRect** () const
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void **initializeParentPlot** (QCustomPlot *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** (QCPPainter *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QPen **mBorderPen**
- QPen **mlconBorderPen**
- QBrush **mBrush**
- QFont **mFont**
- QColor **mTextColor**
- QSize **mlconSize**
- int **mlconTextPadding**
- SelectableParts **mSelectedParts**
- SelectableParts **mSelectableParts**
- QPen **mSelectedBorderPen**
- QPen **mSelectedIconBorderPen**
- QBrush **mSelectedBrush**
- QFont **mSelectedFont**
- QColor **mSelectedTextColor**

Protected Attributes inherited from [QCPLayoutGrid](#)

- QList< QList< [QCPLayoutElement](#) * > > **mElements**
- QList< double > **mColumnStretchFactors**
- QList< double > **mRowStretchFactors**
- int **mColumnSpacing**
- int **mRowSpacing**
- int **mWrap**
- [FillOrder](#) **mFillOrder**

Protected Attributes inherited from [QCPLayoutElement](#)

- [QCPLayout](#) * **mParentLayout**
- QSize **mMinimumSize**
- QSize **mMaximumSize**
- [SizeConstraintRect](#) **mSizeConstraintRect**
- QRect **mRect**
- QRect **mOuterRect**
- QMargins **mMargins**
- QMargins **mMinimumMargins**
- QCP::MarginSides **mAutoMargins**
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **mMarginGroups**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- [QPointer](#)< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCustomPlot](#)
- class [QCPAbstractLegendItem](#)

Additional Inherited Members

Static Protected Member Functions inherited from [QCPLayout](#)

- static [QSize](#) **getFinalMinimumOuterSize** (const [QCPLayoutElement](#) *el)
- static [QSize](#) **getFinalMaximumOuterSize** (const [QCPLayoutElement](#) *el)

15.603.1 Detailed Description

Manages a legend inside a [QCustomPlot](#).

A legend is a small box somewhere in the plot which lists plottables with their name and icon.

A legend is populated with legend items by calling [QCPAbstractPlottable::addToLegend](#) on the plottable, for which a legend item shall be created. In the case of the main legend ([QCustomPlot::legend](#)), simply adding plottables to the plot while [QCustomPlot::setAutoAddPlottableToLegend](#) is set to true (the default) creates corresponding legend items. The legend item associated with a certain plottable can be removed with [QCPAbstractPlottable::removeFromLegend](#). However, [QCPLegend](#) also offers an interface to add and manipulate legend items directly: [item](#), [itemWithPlottable](#), [itemCount](#), [addItem](#), [removeItem](#), etc.

Since [QCPLegend](#) derives from [QCPLayoutGrid](#), it can be placed in any position a [QCPLayoutElement](#) may be positioned. The legend items are themselves [QCPLayoutElements](#) which are placed in the grid layout of the legend. [QCPLegend](#) only adds an interface specialized for handling child elements of type [QCPAbstractLegendItem](#), as mentioned above. In principle, any other layout elements may also be added to a legend via the normal [QCPLayoutGrid](#) interface. See the special page about The Layout System for examples on how to add other elements to the legend and move it outside the axis rect.

Use the methods [setFillOrder](#) and [setWrap](#) inherited from [QCPLayoutGrid](#) to control in which order (column first or row first) the legend is filled up when calling [addItem](#), and at which column or row wrapping occurs. The default fill order for legends is [foRowsFirst](#).

By default, every [QCustomPlot](#) has one legend ([QCustomPlot::legend](#)) which is placed in the inset layout of the main axis rect ([QCPAxisRect::insetLayout](#)). To move the legend to another position inside the axis rect, use the methods of the [QCPLayoutInset](#). To move the legend outside of the axis rect, place it anywhere else with the [QCPLayout/QCPLayoutElement](#) interface.

15.603.2 Member Enumeration Documentation

15.603.2.1 SelectablePart

enum [QCPLegend::SelectablePart](#)

Defines the selectable parts of a legend

See also

[setSelectedParts](#), [setSelectableParts](#)

Enumerator

<code>spNone</code>	<code>0x000</code> None
<code>spLegendBox</code>	<code>0x001</code> The legend box (frame)
<code>spltems</code>	<code>0x002</code> Legend items individually (see selectedItems)

15.603.3 Constructor & Destructor Documentation

15.603.3.1 QCPLegend()

`QCPLegend::QCPLegend ( ) [explicit]`

Constructs a new [QCPLegend](#) instance with default values.

Note that by default, [QCustomPlot](#) already contains a legend ready to be used as [QCustomPlot::legend](#)

15.603.4 Member Function Documentation

15.603.4.1 addItem()

```
bool QCPLegend::addItem (
    QCPAbstractLegendItem * item )
```

Adds *item* to the legend, if it's not present already. The element is arranged according to the current fill order ([setFillOrder](#)) and wrapping ([setWrap](#)).

Returns true on success, i.e. if the item wasn't in the list already and has been successfully added.

The legend takes ownership of the item.

See also

[removeItem](#), [item](#), [hasItem](#)

15.603.4.2 applyDefaultAntialiasingHint()

```
void QCPLegend::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

15.603.4.3 clearItems()

```
void QCPLegend::clearItems ( )
```

Removes all items from the legend.

15.603.4.4 deselectEvent()

```
void QCPLegend::deselectEvent (
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.603.4.5 draw()

```
void QCPLegend::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

15.603.4.6 hasItem()

```
bool QCPLegend::hasItem (
    QCPAbstractLegendItem * item ) const
```

Returns whether the legend contains *item*.

See also

[hasItemWithPlottable](#)

15.603.4.7 hasItemWithPlottable()

```
bool QCPLegend::hasItemWithPlottable (
    const QCPAbstractPlottable * plottable ) const
```

Returns whether the legend contains a [QCPPlottableLegendItem](#) which is associated with *plottable* (e.g. a [QCPGraph](#)*). If such an item isn't in the legend, returns false.

See also

[itemWithPlottable](#)

15.603.4.8 item()

```
QCPLegend::item (
    int index ) const
```

Returns the item with index *i*. If non-legend items were added to the legend, and the element at the specified cell index is not a [QCPLegendItem](#), returns `nullptr`.

Note that the linear index depends on the current fill order ([setFillOrder](#)).

See also

[itemCount](#), [addItem](#), [itemWithPlottable](#)

15.603.4.9 itemCount()

```
int QCPLegend::itemCount ( ) const
```

Returns the number of items currently in the legend. It is identical to the base class [QCPLayoutGrid::elementCount\(\)](#), and unlike the other "item" interface methods of [QCPLegend](#), doesn't only address elements which can be cast to [QCPLegendItem](#).

Note that if empty cells are in the legend (e.g. by calling methods of the [QCPLayoutGrid](#) base class which allows creating empty cells), they are included in the returned count.

See also

[item](#)

15.603.4.10 itemWithPlottable()

```
QCPLegend::itemWithPlottable (
    const QCPLegendItem * plottable ) const
```

Returns the [QCPLegendItem](#) which is associated with *plottable* (e.g. a [QCPLegendItem](#)). If such an item isn't in the legend, returns `nullptr`.

See also

[hasItemWithPlottable](#)

15.603.4.11 parentPlotInitialized()

```
void QCPLegend::parentPlotInitialized (
    QCPLegend * parentPlot ) [protected], [virtual]
```

Reimplemented from [QCPLegendItem](#).

15.603.4.12 removeItem() [1/2]

```
bool QCPLegend::removeItem (
    int index )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Removes the item with the specified *index* from the legend and deletes it.

After successful removal, the legend is reordered according to the current fill order ([setFillOrder](#)) and wrapping ([setWrap](#)), so no empty cell remains where the removed *item* was. If you don't want this, rather use the raw element interface of [QCPLegendItem](#).

Returns true, if successful. Unlike [QCPLegendItem::removeAt](#), this method only removes elements derived from [QCPLegendItem](#).

See also

[itemCount](#), [clearItems](#)

15.603.4.13 removeItem() [2/2]

```
bool QCPLegend::removeItem (
    QCPAbstractLegendItem * item )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Removes *item* from the legend and deletes it.

After successful removal, the legend is reordered according to the current fill order ([setFillOrder](#)) and wrapping ([setWrap](#)), so no empty cell remains where the removed *item* was. If you don't want this, rather use the raw element interface of [QCPLayoutGrid](#).

Returns true, if successful.

See also

[clearItems](#)

15.603.4.14 selectedItems()

```
QList< QCPAbstractLegendItem * > QCPLegend::selectedItems ( ) const
```

Returns the legend items that are currently selected. If no items are selected, the list is empty.

See also

[QCPAbstractLegendItem::setSelected](#), [setSelectable](#)

15.603.4.15 selectEvent()

```
void QCPLegend::selectEvent (
    QMouseEvent * event,
    bool additive,
    const QVariant & details,
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.603.4.16 selectionCategory()

```
QCP::Interaction QCPLegend::selectionCategory ( ) const [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.603.4.17 selectionChanged

```
void QCPLegend::selectionChanged (
    QCPLegend::SelectableParts selection ) [signal]
```

This signal is emitted when the selection state of this legend has changed.

See also

[setSelectedParts](#), [setSelectableParts](#)

15.603.4.18 selectTest()

```
double QCPLegend::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Layout elements are sensitive to events inside their outer rect. If *pos* is within the outer rect, this method returns a value corresponding to 0.99 times the parent plot's selection tolerance. However, layout elements are not selectable by default. So if *onlySelectable* is true, -1.0 is returned.

See [QCPLayerable::selectTest](#) for a general explanation of this virtual method.

[QCPLayoutElement](#) subclasses may reimplement this method to provide more specific selection test behaviour.

Reimplemented from [QCPLayoutElement](#).

15.603.4.19 setBorderPen()

```
void QCPLegend::setBorderPen (
    const QPen & pen )
```

Sets the pen, the border of the entire legend is drawn with.

15.603.4.20 setBrush()

```
void QCPLegend::setBrush (
    const QBrush & brush )
```

Sets the brush of the legend background.

15.603.4.21 setFont()

```
void QCPLegend::setFont (
    const QFont & font )
```

Sets the default font of legend text. Legend items that draw text (e.g. the name of a graph) will use this font by default. However, a different font can be specified on a per-item-basis by accessing the specific legend item. This function will also set *font* on all already existing legend items.

See also

[QCPLegendItem::setFont](#)

15.603.4.22 setIconBorderPen()

```
void QCPLegend::setIconBorderPen (
    const QPen & pen )
```

Sets the pen used to draw a border around each legend icon. Legend items that draw an icon (e.g. a visual representation of the graph) will use this pen by default. If no border is wanted, set this to *Qt::NoPen*.

15.603.4.23 setIconSize() [1/2]

```
void QCPLegend::setIconSize (
    const QSize & size )
```

Sets the size of legend icons. Legend items that draw an icon (e.g. a visual representation of the graph) will use this size by default.

15.603.4.24 setIconSize() [2/2]

```
void QCPLegend::setIconSize (
    int width,
    int height )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

15.603.4.25 setIconTextPadding()

```
void QCPLegend::setIconTextPadding (
    int padding )
```

Sets the horizontal space in pixels between the legend icon and the text next to it. Legend items that draw an icon (e.g. a visual representation of the graph) and text (e.g. the name of the graph) will use this space by default.

15.603.4.26 setSelectableParts()

```
void QCPLegend::setSelectableParts (
    const SelectableParts & selectable )
```

Sets whether the user can (de-)select the parts in *selectable* by clicking on the [QCustomPlot](#) surface. (When [QCustomPlot::setInteractions](#) contains [QCP::iSelectLegend](#).)

However, even when *selectable* is set to a value not allowing the selection of a specific part, it is still possible to set the selection of this part manually, by calling [setSelectedParts](#) directly.

See also

[SelectablePart](#), [setSelectedParts](#)

15.603.4.27 setSelectedBorderPen()

```
void QCPLegend::setSelectedBorderPen (
    const QPen & pen )
```

When the legend box is selected, this pen is used to draw the border instead of the normal pen set via [setBorderPen](#).

See also

[setSelectedParts](#), [setSelectableParts](#), [setSelectedBrush](#)

15.603.4.28 setSelectedBrush()

```
void QCPLegend::setSelectedBrush (
    const QBrush & brush )
```

When the legend box is selected, this brush is used to draw the legend background instead of the normal brush set via [setBrush](#).

See also

[setSelectedParts](#), [setSelectableParts](#), [setSelectedBorderPen](#)

15.603.4.29 setSelectedFont()

```
void QCPLegend::setSelectedFont (
    const QFont & font )
```

Sets the default font that is used by legend items when they are selected. This function will also set *font* on all already existing legend items.

See also

[setFont](#), [QCPAbstractLegendItem::setSelectedFont](#)

15.603.4.30 setSelectedIconBorderPen()

```
void QCPLegend::setSelectedIconBorderPen (
    const QPen & pen )
```

Sets the pen legend items will use to draw their icon borders, when they are selected.

See also

[setSelectedParts](#), [setSelectableParts](#), [setSelectedFont](#)

15.603.4.31 setSelectedParts()

```
void QCPLegend::setSelectedParts (
    const SelectableParts & selected )
```

Sets the selected state of the respective legend parts described by [SelectablePart](#). When a part is selected, it uses a different pen/font and brush. If some legend items are selected and *selected* doesn't contain [splItems](#), those items become deselected.

The entire selection mechanism is handled automatically when [QCustomPlot::setInteractions](#) contains `iSelectLegend`. You only need to call this function when you wish to change the selection state manually.

This function can change the selection state of a part even when [setSelectableParts](#) was set to a value that actually excludes the part.

emits the [selectionChanged](#) signal when *selected* is different from the previous selection state.

Note that it doesn't make sense to set the selected state [splItems](#) here when it wasn't set before, because there's no way to specify which exact items to newly select. Do this by calling [QCPAbstractLegendItem::setSelected](#) directly on the legend item you wish to select.

See also

[SelectablePart](#), [setSelectableParts](#), [selectTest](#), [setSelectedBorderPen](#), [setSelectedIconBorderPen](#), [setSelectedBrush](#), [setSelectedFont](#)

15.603.4.32 setSelectedTextColor()

```
void QCPLegend::setSelectedTextColor (
    const QColor & color )
```

Sets the default text color that is used by legend items when they are selected. This function will also set *color* on all already existing legend items.

See also

[setTextColor](#), [QCPAbstractLegendItem::setSelectedTextColor](#)

15.603.4.33 setTextColor()

```
void QCPLegend::setTextColor (
    const QColor & color )
```

Sets the default color of legend text. Legend items that draw text (e.g. the name of a graph) will use this color by default. However, a different colors can be specified on a per-item-basis by accessing the specific legend item. This function will also set *color* on all already existing legend items.

See also

[QCPAbstractLegendItem::setTextColor](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.604 QCPLineEnding Class Reference

Handles the different ending decorations for line-like items.

Public Types

- enum [EndingStyle](#) {
[esNone](#) , [esFlatArrow](#) , [esSpikeArrow](#) , [esLineArrow](#) ,
[esDisc](#) , [esSquare](#) , [esDiamond](#) , [esBar](#) ,
[esHalfBar](#) , [esSkewedBar](#) }

Public Member Functions

- [QCPLineEnding](#) ()
- [QCPLineEnding](#) ([EndingStyle](#) style, double width=8, double length=10, bool inverted=false)
- [EndingStyle](#) **style** () const
- double **width** () const
- double **length** () const
- bool **inverted** () const
- void [setStyle](#) ([EndingStyle](#) style)
- void [setWidth](#) (double width)
- void [setLength](#) (double length)
- void [setInverted](#) (bool inverted)
- double **boundingDistance** () const
- double [realLength](#) () const
- void **draw** ([QCPPainter](#) *painter, const [QCPVector2D](#) &pos, const [QCPVector2D](#) &dir) const
- void **draw** ([QCPPainter](#) *painter, const [QCPVector2D](#) &pos, double angle) const

Protected Attributes

- [EndingStyle](#) `mStyle`
- double `mWidth`
- double `mLength`
- bool `mInverted`

15.604.1 Detailed Description

Handles the different ending decorations for line-like items.

For every ending a line-like item has, an instance of this class exists. For example, [QCPLItemLine](#) has two endings which can be set with [QCPLItemLine::setHead](#) and [QCPLItemLine::setTail](#).

The styles themselves are defined via the enum [QCPLineEnding::EndingStyle](#). Most decorations can be modified regarding width and length, see [setWidth](#) and [setLength](#). The direction of the ending decoration (e.g. direction an arrow is pointing) is controlled by the line-like item. For example, when both endings of a [QCPLItemLine](#) are set to be arrows, they will point to opposite directions, e.g. "outward". This can be changed by [setInverted](#), which would make the respective arrow point inward.

Note that due to the overloaded [QCPLineEnding](#) constructor, you may directly specify a [QCPLineEnding::EndingStyle](#) where actually a [QCPLineEnding](#) is expected, e.g.

15.604.2 Member Enumeration Documentation

15.604.2.1 EndingStyle

enum [QCPLineEnding::EndingStyle](#)

Defines the type of ending decoration for line-like items, e.g. an arrow.

The width and length of these decorations can be controlled with the functions [setWidth](#) and [setLength](#). Some decorations like [esDisc](#), [esSquare](#), [esDiamond](#) and [esBar](#) only support a width, the length property is ignored.

See also

[QCPLItemLine::setHead](#), [QCPLItemLine::setTail](#), [QCPLItemCurve::setHead](#), [QCPLItemCurve::setTail](#), [QCPAxis::setLowerEnding](#), [QCPAxis::setUpperEnding](#)

Enumerator

<code>esNone</code>	No ending decoration.
<code>esFlatArrow</code>	A filled arrow head with a straight/flat back (a triangle)
<code>esSpikeArrow</code>	A filled arrow head with an indented back.
<code>esLineArrow</code>	A non-filled arrow head with open back.
<code>esDisc</code>	A filled circle.
<code>esSquare</code>	A filled square.
<code>esDiamond</code>	A filled diamond (45 degrees rotated square)
<code>esBar</code>	A bar perpendicular to the line.
<code>esHalfBar</code>	A bar perpendicular to the line, pointing out to only one side (to which side can be changed with setInverted)
<code>esSkewedBar</code>	A bar that is skewed (skew controllable via setLength)

15.604.3 Constructor & Destructor Documentation

15.604.3.1 QCPLineEnding() [1/2]

`QCPLineEnding::QCPLineEnding ( )`

Creates a [QCPLineEnding](#) instance with default values (style [esNone](#)).

15.604.3.2 QCPLineEnding() [2/2]

```
QCPLineEnding::QCPLineEnding (
    QCPLineEnding::EndingStyle style,
    double width = 8,
    double length = 10,
    bool inverted = false )
```

Creates a [QCPLineEnding](#) instance with the specified values.

15.604.4 Member Function Documentation

15.604.4.1 realLength()

```
double QCPLineEnding::realLength ( ) const
```

Starting from the origin of this line ending (which is style specific), returns the length covered by the line ending symbol, in backward direction.

For example, the [esSpikeArrow](#) has a shorter real length than a [esFlatArrow](#), even if both have the same [setLength](#) value, because the spike arrow has an inward curved back, which reduces the length along its center axis (the drawing origin for arrows is at the tip).

This function is used for precise, style specific placement of line endings, for example in QCPAxes.

15.604.4.2 setInverted()

```
void QCPLineEnding::setInverted (
    bool inverted )
```

Sets whether the ending decoration shall be inverted. For example, an arrow decoration will point inward when *inverted* is set to true.

Note that also the *width* direction is inverted. For symmetrical ending styles like arrows or discs, this doesn't make a difference. However, asymmetric styles like [esHalfBar](#) are affected by it, which can be used to control to which side the half bar points to.

15.604.4.3 setLength()

```
void QCPLineEnding::setLength (
    double length )
```

Sets the length of the ending decoration, if the style supports it. On arrows, for example, the length defines the size in pointing direction.

See also

[setWidth](#)

15.604.4.4 setStyle()

```
void QCPLineEnding::setStyle (
    QCPLineEnding::EndingStyle style )
```

Sets the style of the ending decoration.

15.604.4.5 setWidth()

```
void QCPLineEnding::setWidth (
    double width )
```

Sets the width of the ending decoration, if the style supports it. On arrows, for example, the width defines the size perpendicular to the arrow's pointing direction.

See also

[setLength](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

- void **addChild** (QCP::MarginSide side, QCPLayoutElement *element)
- void **removeChild** (QCP::MarginSide side, QCPLayoutElement *element)

Protected Attributes

- QCustomPlot * **mParentPlot**
- QHash< QCP::MarginSide, QList< QCPLayoutElement * > > **mChildren**

Friends

- class QCPLayoutElement

15.605.1 Detailed Description

A margin group allows synchronization of margin sides if working with multiple layout elements.

[QCPMarginGroup](#) allows you to tie a margin side of two or more layout elements together, such that they will all have the same size, based on the largest required margin in the group.

In certain situations it is desirable that margins at specific sides are synchronized across layout elements. For example, if one [QCPAxisRect](#) is below another one in a grid layout, it will provide a cleaner look to the user if the left and right margins of the two axis rects are of the same size. The left axis of the top axis rect will then be at the same horizontal position as the left axis of the lower axis rect, making them appear aligned. The same applies for the right axes. This is what [QCPMarginGroup](#) makes possible.

To add/remove a specific side of a layout element to/from a margin group, use the [QCPLayoutElement::setMarginGroup](#) method. To completely break apart the margin group, either call [clear](#), or just delete the margin group.

15.605.2 Example

First create a margin group:

Then set this group on the layout element sides:

Here, we've used the first two axis rects of the plot and synchronized their left margins with each other and their right margins with each other.

15.605.3 Constructor & Destructor Documentation

15.605.3.1 QCPMarginGroup()

```
QCPMarginGroup::QCPMarginGroup (
    QCustomPlot * parentPlot ) [explicit]
```

Creates a new [QCPMarginGroup](#) instance in *parentPlot*.

15.605.4 Member Function Documentation

15.605.4.1 clear()

```
void QCPMarginGroup::clear ( )
```

Clears this margin group. The synchronization of the margin sides that use this margin group is lifted and they will use their individual margin sizes again.

15.605.4.2 elements()

```
QList< QCPLayoutElement * > QCPMarginGroup::elements (
    QCP::MarginSide side ) const [inline]
```

Returns a list of all layout elements that have their margin *side* associated with this margin group.

15.605.4.3 isEmpty()

```
bool QCPMarginGroup::isEmpty ( ) const
```

Returns whether this margin group is empty. If this function returns true, no layout elements use this margin group to synchronize margin sides.

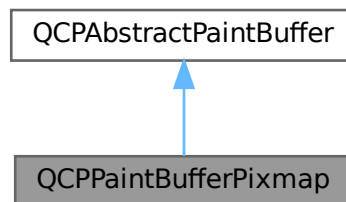
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

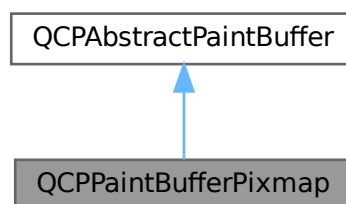
15.606 QCPPaintBufferPixmap Class Reference

A paint buffer based on QPixmap, using software raster rendering.

Inheritance diagram for QCPPaintBufferPixmap:



Collaboration diagram for QCPPaintBufferPixmap:



Public Member Functions

- [QCPPaintBufferPixmap](#) (const QSize &size, double devicePixelRatio)
- virtual [QCPPainter](#) * [startPainting](#) () Q_DECL_OVERRIDE
- virtual void [draw](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- void [clear](#) (const QColor &color) Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractPaintBuffer](#)

- [QCPAbstractPaintBuffer](#) (const QSize &size, double devicePixelRatio)
- QSize [size](#) () const

- bool **invalidated** () const
- double **devicePixelRatio** () const
- void **setSize** (const QSize &size)
- void **setInvalidated** (bool invalidated=true)
- void **setDevicePixelRatio** (double ratio)
- virtual void **donePainting** ()

Protected Member Functions

- virtual void **reallocateBuffer** () Q_DECL_OVERRIDE

Protected Attributes

- QPixmap **mBuffer**

Protected Attributes inherited from [QCPAbstractPaintBuffer](#)

- QSize **mSize**
- double **mDevicePixelRatio**
- bool **mInvalidated**

15.606.1 Detailed Description

A paint buffer based on QPixmap, using software raster rendering.

This paint buffer is the default and fall-back paint buffer which uses software rendering and QPixmap as internal buffer. It is used if [QCustomPlot::setOpenGl](#) is false.

15.606.2 Constructor & Destructor Documentation

15.606.2.1 QCPPaintBufferPixmap()

```
QCPPaintBufferPixmap::QCPPaintBufferPixmap (
    const QSize & size,
    double devicePixelRatio ) [explicit]
```

Creates a pixmap paint buffer instance with the specified *size* and *devicePixelRatio*, if applicable.

15.606.3 Member Function Documentation

15.606.3.1 clear()

```
void QCPPaintBufferPixmap::clear (
    const QColor & color ) [virtual]
```

Fills the entire buffer with the provided *color*. To have an empty transparent buffer, use the named color `Qt::transparent`.

This method must not be called if there is currently a painter (acquired with [startPainting](#)) active.

Implements [QCPAbstractPaintBuffer](#).

15.606.3.2 draw()

```
void QCPPaintBufferPixmap::draw (
    QCPPainter * painter ) const [virtual]
```

Draws the contents of this buffer with the provided *painter*. This is the method that is used to finally join all paint buffers and draw them onto the screen.

Implements [QCPAbstractPaintBuffer](#).

15.606.3.3 `reallocateBuffer()`

```
void QCPPaintBufferPixmap::reallocateBuffer ( ) [protected], [virtual]
```

Reallocates the internal buffer with the currently configured size ([setSize](#)) and device pixel ratio, if applicable ([setDevicePixelRatio](#)). It is called as soon as any of those properties are changed on this paint buffer.

Note

Subclasses of [QCPAbstractPaintBuffer](#) must call their reimplementations of this method in their constructor, to perform the first allocation (this can not be done by the base class because calling pure virtual methods in base class constructors is not possible).

Implements [QCPAbstractPaintBuffer](#).

15.606.3.4 `startPainting()`

```
QCPPainter * QCPPaintBufferPixmap::startPainting ( ) [virtual]
```

Returns a [QCPPainter](#) which is ready to draw to this buffer. The ownership and thus the responsibility to delete the painter after the painting operations are complete is given to the caller of this method.

Once you are done using the painter, delete the painter and call [donePainting](#).

While a painter generated with this method is active, you must not call [setSize](#), [setDevicePixelRatio](#) or [clear](#).

This method may return 0, if a painter couldn't be activated on the buffer. This usually indicates a problem with the respective painting backend.

Implements [QCPAbstractPaintBuffer](#).

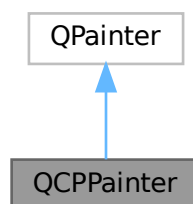
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

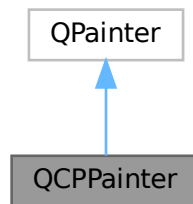
15.607 QCPPainter Class Reference

QPainter subclass used internally.

Inheritance diagram for QCPPainter:



Collaboration diagram for QCPPainter:



Public Types

- enum [PainterMode](#) { [pmDefault](#) = 0x00 , [pmVectorized](#) = 0x01 , [pmNoCaching](#) = 0x02 , [pmNonCosmetic](#) = 0x04 }

Public Member Functions

- [QCPPainter](#) ()
- [QCPPainter](#) (QPaintDevice *device)
- bool **antialiasing** () const
- PainterModes **modes** () const
- void [setAntialiasing](#) (bool enabled)
- void [setMode](#) ([PainterMode](#) mode, bool enabled=true)
- void [setModes](#) (PainterModes modes)
- bool [begin](#) (QPaintDevice *device)
- void [setPen](#) (const QPen &pen)
- void [setPen](#) (const QColor &color)
- void [setPen](#) (Qt::PenStyle penStyle)
- void [drawLine](#) (const QLineF &line)
- void **drawLine** (const QPointF &p1, const QPointF &p2)
- void [save](#) ()
- void [restore](#) ()
- void [makeNonCosmetic](#) ()

Protected Attributes

- PainterModes **mModes**
- bool **mlsAntialiasing**
- QStack< bool > **mAntialiasingStack**

15.607.1 Detailed Description

QPainter subclass used internally.

This QPainter subclass is used to provide some extended functionality e.g. for tweaking position consistency between antialiased and non-antialiased painting. Further it provides workarounds for QPainter quirks.

Warning

This class intentionally hides non-virtual functions of QPainter, e.g. [setPen](#), [save](#) and [restore](#). So while it is possible to pass a [QCPPainter](#) instance to a function that expects a QPainter pointer, some of the workarounds and tweaks will be unavailable to the function (because it will call the base class implementations of the functions actually hidden by [QCPPainter](#)).

15.607.2 Member Enumeration Documentation

15.607.2.1 PainterMode

enum [QCPPainter::PainterMode](#)

Defines special modes the painter can operate in. They disable or enable certain subsets of features/fixes/workarounds, depending on whether they are wanted on the respective output device.

Enumerator

<code>pmDefault</code>	<code>0x00</code> Default mode for painting on screen devices
<code>pmVectorized</code>	<code>0x01</code> Mode for vectorized painting (e.g. PDF export). For example, this prevents some antialiasing fixes.
<code>pmNoCaching</code>	<code>0x02</code> Mode for all sorts of exports (e.g. PNG, PDF,...). For example, this prevents using cached pixmap labels
<code>pmNonCosmetic</code>	<code>0x04</code> Turns pen widths 0 to 1, i.e. disables cosmetic pens. (A cosmetic pen is always drawn with width 1 pixel in the vector image/pdf viewer, independent of zoom.)

15.607.3 Constructor & Destructor Documentation

15.607.3.1 QCPPainter() [1/2]

`QCPPainter::QCPPainter ( )`

Creates a new [QCPPainter](#) instance and sets default values

15.607.3.2 QCPPainter() [2/2]

`QCPPainter::QCPPainter (
 QPaintDevice * device) [explicit]`

Creates a new [QCPPainter](#) instance on the specified paint *device* and sets default values. Just like the analogous QPainter constructor, begins painting on *device* immediately.

Like [begin](#), this method sets QPainter::NonCosmeticDefaultPen in Qt versions before Qt5.

15.607.4 Member Function Documentation

15.607.4.1 begin()

`bool QCPPainter::begin (
 QPaintDevice * device)`

Sets the QPainter::NonCosmeticDefaultPen in Qt versions before Qt5 after beginning painting on *device*. This is necessary to get cosmetic pen consistency across Qt versions, because since Qt5, all pens are non-cosmetic by default, and in Qt4 this render hint must be set to get that behaviour.

The Constructor [QCPPainter\(QPaintDevice *device\)](#) which directly starts painting also sets the render hint as appropriate.

Note

this function hides the non-virtual base class implementation.

15.607.4.2 drawLine()

`void QCPPainter::drawLine (
 const QLineF & line)`

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Works around a Qt bug introduced with Qt 4.8 which makes drawing QLineF unpredictable when antialiasing is disabled. Thus when antialiasing is disabled, it rounds the *line* to integer coordinates and then passes it to the original drawLine.

Note

this function hides the non-virtual base class implementation.

15.607.4.3 makeNonCosmetic()

```
void QCPPainter::makeNonCosmetic ( )
```

Changes the pen width to 1 if it currently is 0. This function is called in the [setPen](#) overrides when the [pmNonCosmetic](#) mode is set.

15.607.4.4 restore()

```
void QCPPainter::restore ( )
```

Restores the painter (see [QPainter::restore](#)). Since [QCPPainter](#) adds some new internal state to [QPainter](#), the save/restore functions are reimplemented to also save/restore those members.

Note

this function hides the non-virtual base class implementation.

See also

[save](#)

15.607.4.5 save()

```
void QCPPainter::save ( )
```

Saves the painter (see [QPainter::save](#)). Since [QCPPainter](#) adds some new internal state to [QPainter](#), the save/restore functions are reimplemented to also save/restore those members.

Note

this function hides the non-virtual base class implementation.

See also

[restore](#)

15.607.4.6 setAntialiasing()

```
void QCPPainter::setAntialiasing (
    bool enabled )
```

Sets whether painting uses antialiasing or not. Use this method instead of using [setRenderHint](#) with [QPainter::Antialiasing](#) directly, as it allows [QCPPainter](#) to regain pixel exactness between antialiased and non-antialiased painting (Since Qt < 5.0 uses slightly different coordinate systems for AA/Non-AA painting).

15.607.4.7 setMode()

```
void QCPPainter::setMode (
    QCPPainter::PainterMode mode,
    bool enabled = true )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the mode of the painter. This controls whether the painter shall adjust its fixes/workarounds optimized for certain output devices.

15.607.4.8 setModes()

```
void QCPPainter::setModes (
    PainterModes modes )
```

Sets the mode of the painter. This controls whether the painter shall adjust its fixes/workarounds optimized for certain output devices.

15.607.4.9 setPen() [1/3]

```
void QCPPainter::setPen (
    const QColor & color )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the pen (by color) of the painter and applies certain fixes to it, depending on the mode of this [QCPPainter](#).

Note

this function hides the non-virtual base class implementation.

15.607.4.10 setPen() [2/3]

```
void QCPPainter::setPen (
    const QPen & pen )
```

Sets the pen of the painter and applies certain fixes to it, depending on the mode of this [QCPPainter](#).

Note

this function hides the non-virtual base class implementation.

15.607.4.11 setPen() [3/3]

```
void QCPPainter::setPen (
    Qt::PenStyle penStyle )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the pen (by style) of the painter and applies certain fixes to it, depending on the mode of this [QCPPainter](#).

Note

this function hides the non-virtual base class implementation.

The documentation for this class was generated from the following files:

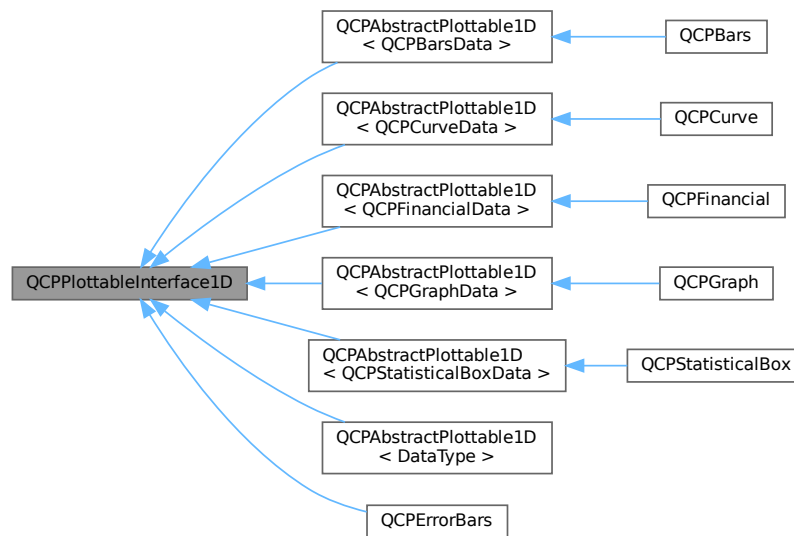
- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.608 QCPPloortableInterface1D Class Reference

Defines an abstract interface for one-dimensional plottables.

```
#include <applications/gui/genesys/qcustomplot.h>
```

Inheritance diagram for QCPPlottableInterface1D:



Public Member Functions

- virtual int [dataCount](#) () const =0
- virtual double [dataMainKey](#) (int index) const =0
- virtual double [dataSortKey](#) (int index) const =0
- virtual double [dataMainValue](#) (int index) const =0
- virtual [QCPRange](#) [dataValueRange](#) (int index) const =0
- virtual QPointF [dataPixelPosition](#) (int index) const =0
- virtual bool [sortKeysIsMainKey](#) () const =0
- virtual [QCPDataSelection](#) [selectTestRect](#) (const QRectF &rect, bool onlySelectable) const =0
- virtual int [findBegin](#) (double sortKey, bool expandedRange=true) const =0
- virtual int [findEnd](#) (double sortKey, bool expandedRange=true) const =0

15.608.1 Detailed Description

Defines an abstract interface for one-dimensional plottables.

This class contains only pure virtual methods which define a common interface to the data of one-dimensional plottables.

For example, it is implemented by the template class [QCPAbstractPlottable1D](#) (the preferred base class for one-dimensional plottables). So if you use that template class as base class of your one-dimensional plottable, you won't have to care about implementing the 1d interface yourself.

If your plottable doesn't derive from [QCPAbstractPlottable1D](#) but still wants to provide a 1d interface (e.g. like [QCPErrorBars](#) does), you should inherit from both [QCPAbstractPlottable](#) and [QCPPlottableInterface1D](#) and accordingly reimplement the pure virtual methods of the 1d interface, matching your data container. Also, reimplement [QCPAbstractPlottable::interface1D](#) to return the `this` pointer.

If you have a [QCPAbstractPlottable](#) pointer, you can check whether it implements this interface by calling [QCPAbstractPlottable::interface1D](#) and testing it for a non-zero return value. If it indeed implements this interface, you may use it to access the plottable's data without needing to know the exact type of the plottable or its data point type.

15.608.2 Member Function Documentation

15.608.2.1 dataCount()

```
int QCPPlottableInterface1D::dataCount ( ) const [pure virtual]
```

Returns the number of data points of the plottable.

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalData >](#), and [QCPErrorBars](#).

15.608.2.2 dataMainKey()

```
double QCPPlottableInterface1D::dataMainKey (
    int index ) const [pure virtual]
```

Returns the main key of the data point at the given *index*.

What the main key is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalData >](#), and [QCPErrorBars](#).

15.608.2.3 dataMainValue()

```
double QCPPlottableInterface1D::dataMainValue (
    int index ) const [pure virtual]
```

Returns the main value of the data point at the given *index*.

What the main value is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalData >](#), and [QCPErrorBars](#).

15.608.2.4 dataPixelPosition()

```
QPointF QCPPlottableInterface1D::dataPixelPosition (
    int index ) const [pure virtual]
```

Returns the pixel position on the widget surface at which the data point at the given *index* appears.

Usually this corresponds to the point of `dataMainKey/ dataMainValue`, in pixel coordinates. However, depending on the plottable, this might be a different apparent position than just a coord-to-pixel transform of those values. For example, [QCPBars](#) apparent data values can be shifted depending on their stacking, bar grouping or configured base value.

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalData >](#), [QCPBars](#), and [QCPErrorBars](#).

15.608.2.5 dataSortKey()

```
double QCPPlottableInterface1D::dataSortKey (
    int index ) const [pure virtual]
```

Returns the sort key of the data point at the given *index*.

What the sort key is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalData >](#), and [QCPErrorBars](#).

15.608.2.6 dataValueRange()

```
QCPRange QCPPlottableInterface1D::dataValueRange (
    int index ) const [pure virtual]
```


Returns the value range of the data point at the given *index*.

What the value range is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalData >](#) and [QCPErrorBars](#).

15.608.2.7 findBegin()

```
int QCPPlottableInterface1D::findBegin (
    double sortKey,
    bool expandedRange = true ) const [pure virtual]
```

Returns the index of the data point with a (sort-)key that is equal to, just below, or just above *sortKey*. If *expandedRange* is true, the data point just below *sortKey* will be considered, otherwise the one just above.

This can be used in conjunction with [findEnd](#) to iterate over data points within a given key range, including or excluding the bounding data points that are just beyond the specified range.

If *expandedRange* is true but there are no data points below *sortKey*, 0 is returned.

If the container is empty, returns 0 (in that case, [findEnd](#) will also return 0, so a loop using these methods will not iterate over the index 0).

See also

[findEnd](#), [QCPDataContainer::findBegin](#)

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalData >](#) and [QCPErrorBars](#).

15.608.2.8 findEnd()

```
int QCPPlottableInterface1D::findEnd (
    double sortKey,
    bool expandedRange = true ) const [pure virtual]
```

Returns the index one after the data point with a (sort-)key that is equal to, just above, or just below *sortKey*. If *expandedRange* is true, the data point just above *sortKey* will be considered, otherwise the one just below.

This can be used in conjunction with [findBegin](#) to iterate over data points within a given key range, including the bounding data points that are just below and above the specified range.

If *expandedRange* is true but there are no data points above *sortKey*, the index just above the highest data point is returned.

If the container is empty, returns 0.

See also

[findBegin](#), [QCPDataContainer::findEnd](#)

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalData >](#) and [QCPErrorBars](#).

15.608.2.9 selectTestRect()

```
QCPDataSelection QCPPlottableInterface1D::selectTestRect (
    const QRectF & rect,
    bool onlySelectable ) const [pure virtual]
```

Returns a data selection containing all the data points of this plottable which are contained (or hit by) *rect*. This is used mainly in the selection rect interaction for data selection (data selection mechanism).

If *onlySelectable* is true, an empty [QCPDataSelection](#) is returned if this plottable is not selectable (i.e. if [QCPAbstractPlottable::setSelectable](#) is [QCP::stNone](#)).

Note

rect must be a normalized rect (positive or zero width and height). This is especially important when using the rect of [QCPSelectionRect::accepted](#), which is not necessarily normalized. Use `QRect::normalized()` when passing a rect which might not be normalized.

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalBox >](#), [QCPBars](#), [QCPStatisticalBox](#), [QCPFinancial](#), and [QCPErrorBars](#).

15.608.2.10 sortKeysMainKey()

```
bool QCPPlottableInterface1D::sortKeyIsMainKey ( ) const [pure virtual]
```

Returns whether the sort key ([dataSortKey](#)) is identical to the main key ([dataMainKey](#)).

What the sort and main keys are, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalBox >](#), [QCPBars](#), [QCPStatisticalBox](#), [QCPFinancial](#), and [QCPErrorBars](#).

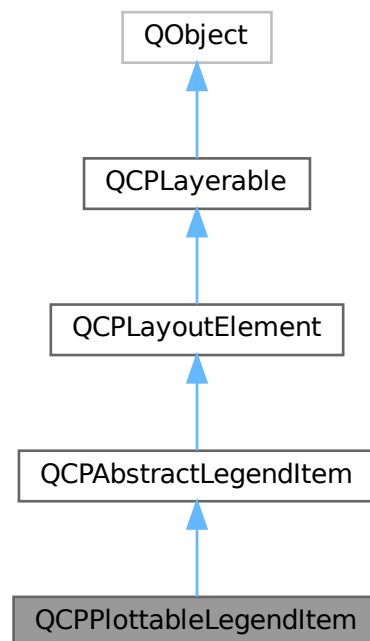
The documentation for this class was generated from the following file:

- applications/gui/genesys/qcustomplot.h

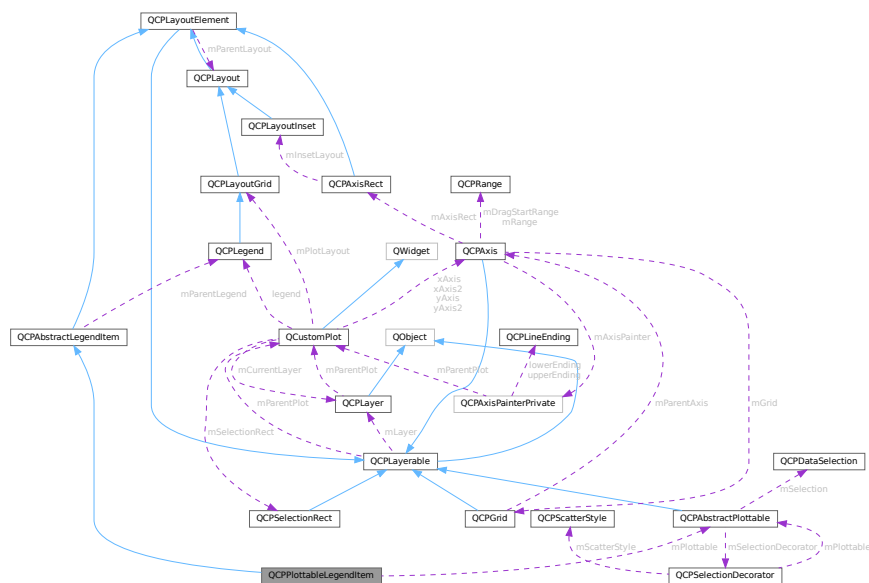
15.609 QCPPlottableLegendItem Class Reference

A legend item representing a plottable with an icon and the plottable name.

Inheritance diagram for QCPPlottableLegendItem:



Collaboration diagram for QCPPlottableLegendItem:



Public Member Functions

- `QCPlotLegendItem` (`QCPlotLegend *parent`, `QCAbstractPlottable *plottable`)
- `QCAbstractPlottable * plottable ()`

Public Member Functions inherited from QCPAbstractLegendItem

- QCPAbstractLegendItem (QCPLegend *parent)
 - QCPLegend * parentLegend () const
 - QFont font () const
 - QColor textColor () const
 - QFont selectedFont () const
 - QColor selectedTextColor () const
 - bool selectable () const
 - bool selected () const
 - void setFont (const QFont &font)
 - void setTextColor (const QColor &color)
 - void setSelectedFont (const QFont &font)
 - void setSelectedTextColor (const QColor &color)
 - Q_SLOT void setSelectable (bool selectable)
 - Q_SLOT void setSelected (bool selected)
 - virtual double selectTest (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_↔
- DECL OVERRIDE

Public Member Functions inherited from QCPLLayoutElement

- `QCPLayoutElement (QCustomPlot *parentPlot=nullptr)`
- `QCPLayout * layout () const`
- `QRect rect () const`
- `QRect outerRect () const`
- `QMargins margins () const`
- `QMargins minimumMargins () const`
- `QCP::MarginSides autoMargins () const`

- QSize **minimumSize** () const
- QSize **maximumSize** () const
- [SizeConstraintRect](#) **sizeConstraintRect** () const
- [QCPMarginGroup](#) * **marginGroup** ([QCP::MarginSide](#) side) const
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **marginGroups** () const
- void [setOuterRect](#) (const QRect &rect)
- void [setMargins](#) (const QMargins &margins)
- void [setMinimumMargins](#) (const QMargins &margins)
- void [setAutoMargins](#) ([QCP::MarginSides](#) sides)
- void [setMinimumSize](#) (const QSize &size)
- void [setMinimumSize](#) (int width, int height)
- void [setMaximumSize](#) (const QSize &size)
- void [setMaximumSize](#) (int width, int height)
- void [setSizeConstraintRect](#) ([SizeConstraintRect](#) constraintRect)
- void [setMarginGroup](#) ([QCP::MarginSides](#) sides, [QCPMarginGroup](#) *group)
- virtual void [update](#) ([UpdatePhase](#) phase)
- virtual QSize [maximumOuterSizeHint](#) () const
- virtual QList< [QCPLayoutElement](#) * > **elements** (bool recursive) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual QSize [minimumOuterSizeHint](#) () const Q_DECL_OVERRIDE
- QPen **getIconBorderPen** () const
- QColor **getTextColor** () const
- QFont **getFont** () const

Protected Member Functions inherited from [QCPAbstractLegendItem](#)

- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionState↔ Changed) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayoutElement](#)

- virtual int **calculateAutoMargin** ([QCP::MarginSide](#) side)
- virtual void **layoutChanged** ()
- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from QCPLayerable

- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void [initializeParentPlot](#) (QCustomPlot *parentPlot)
- void [setParentLayerable](#) (QCPLayerable *parentLayerable)
- bool [moveToLayer](#) (QCPLayer *layer, bool prepend)
- void [applyAntialiasingHint](#) (QCPPainter *painter, bool localAntialiased, QCP::AntialiasedElement overrideElement) const

Protected Attributes

- [QCPAbstractPlottable](#) * **mPlottable**

Protected Attributes inherited from QCPAbstractLegendItem

- [QCPLegend](#) * **mParentLegend**
- QFont **mFont**
- QColor **mTextColor**
- QFont **mSelectedFont**
- QColor **mSelectedTextColor**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from QCPLayoutElement

- [QCPLayout](#) * **mParentLayout**
- QSize **mMinimumSize**
- QSize **mMaximumSize**
- [SizeConstraintRect](#) **mSizeConstraintRect**
- QRect **mRect**
- QRect **mOuterRect**
- QMargins **mMargins**
- QMargins **mMinimumMargins**
- QCP::MarginSides **mAutoMargins**
- QHash< [QCP::MarginSide](#), QCPMarginGroup * > **mMarginGroups**

Protected Attributes inherited from QCPLayerable

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members**Public Types inherited from QCPLayoutElement**

- enum [UpdatePhase](#) { [upPreparation](#) , [upMargins](#) , [upLayout](#) }
- enum [SizeConstraintRect](#) { [scrInnerRect](#) , [scrOuterRect](#) }

Signals inherited from [QCPAbstractLegendItem](#)

- void [selectionChanged](#) (bool selected)
- void [selectableChanged](#) (bool selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

15.609.1 Detailed Description

A legend item representing a plottable with an icon and the plottable name.

This is the standard legend item for plottables. It displays an icon of the plottable next to the plottable name. The icon is drawn by the respective plottable itself ([QCPAbstractPlottable::drawLegendIcon](#)), and tries to give an intuitive symbol for the plottable. For example, the [QCPGraph](#) draws a centered horizontal line and/or a single scatter point in the middle.

Legend items of this type are always associated with one plottable (retrievable via the [plottable\(\)](#) function and settable with the constructor). You may change the font of the plottable name with [setFont](#). Icon padding and border pen is taken from the parent [QCPLegend](#), see [QCPLegend::setIconBorderPen](#) and [QCPLegend::setIconTextPadding](#). The function [QCPAbstractPlottable::addToLegend](#)/[QCPAbstractPlottable::removeFromLegend](#) creates/removes legend items of this type.

Since [QCPLegend](#) is based on [QCPLayoutGrid](#), a legend item itself is just a subclass of [QCPLayoutElement](#). While it could be added to a legend (or any other layout) via the normal layout interface, [QCPLegend](#) has specialized functions for handling legend items conveniently, see the documentation of [QCPLegend](#).

15.609.2 Constructor & Destructor Documentation

15.609.2.1 [QCPPlottableLegendItem\(\)](#)

```
QCPPlottableLegendItem::QCPPlottableLegendItem (
    QCPLegend * parent,
    QCPAbstractPlottable * plottable )
```

Creates a new legend item associated with *plottable*.

Once it's created, it can be added to the legend via [QCPLegend::addItem](#).

A more convenient way of adding/removing a plottable to/from the legend is via the functions [QCPAbstractPlottable::addToLegend](#) and [QCPAbstractPlottable::removeFromLegend](#).

15.609.3 Member Function Documentation

15.609.3.1 [draw\(\)](#)

```
void QCPPlottableLegendItem::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractLegendItem](#).

15.609.3.2 [minimumOuterSizeHint\(\)](#)

```
QSize QCPPlottableLegendItem::minimumOuterSizeHint ( ) const [protected], [virtual]
```

Returns the suggested minimum size this layout element (the [outerRect](#)) may be compressed to, if no manual minimum size is set.

if a minimum size ([setMinimumSize](#)) was not set manually, parent layouts use the returned size (usually indirectly through [QCPLayout::getFinalMinimumOuterSize](#)) to determine the minimum allowed size of this layout element.

A manual minimum size is considered set if it is non-zero.

The default implementation simply returns the sum of the horizontal margins for the width and the sum of the vertical margins for the height. Reimplementations may use their detailed knowledge about the layout element's content to provide size hints.

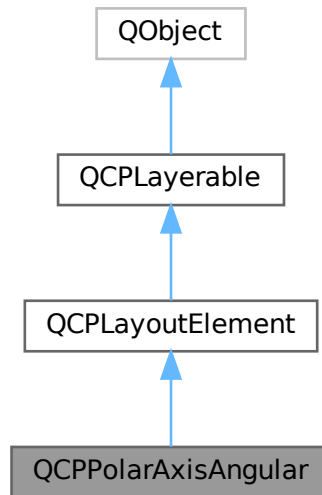
Reimplemented from [QCPLayoutElement](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

The main container for polar plots, representing the angular axis as a circle.

Inheritance diagram for QCPPolarAxisAngular:

[illegible]

- enum **SelectablePart** { **spNone** = 0, **spAxis** = 0x001, **spTickLabels** = 0x002, **spAxisLabel** = 0x004 }
- enum **LabelMode** { **ImUpright**, **ImRotated** }

Public Types inherited from [QCPLayoutElement](#)

- enum [UpdatePhase](#) { [upPreparation](#) , [upMargins](#) , [upLayout](#) }
- enum [SizeConstraintRect](#) { [scrInnerRect](#) , [scrOuterRect](#) }

Signals

- void **rangeChanged** (const [QCPRange](#) &newRange)
- void **rangeChanged** (const [QCPRange](#) &newRange, const [QCPRange](#) &oldRange)
- void **selectionChanged** (const QCPPolarAxisAngular::SelectableParts &parts)
- void **selectableChanged** (const QCPPolarAxisAngular::SelectableParts &parts)

Signals inherited from [QCPLayerable](#)

- void **layerChanged** ([QCPLayer](#) *newLayer)

Public Member Functions

- [QCPPolarAxisAngular](#) ([QCustomPlot](#) *parentPlot)
- QPixmap **background** () const
- QBrush **backgroundBrush** () const
- bool **backgroundScaled** () const
- Qt::AspectRatioMode **backgroundScaledMode** () const
- bool **rangeDrag** () const
- bool **rangeZoom** () const
- double **rangeZoomFactor** () const
- const [QCPRange](#) **range** () const
- bool **rangeReversed** () const
- double **angle** () const
- QSharedPointer< [QCPAxisTicker](#) > **ticker** () const
- bool **ticks** () const
- bool **tickLabels** () const
- int **tickLabelPadding** () const
- QFont **tickLabelFont** () const
- QColor **tickLabelColor** () const
- double **tickLabelRotation** () const
- [LabelMode](#) **tickLabelMode** () const
- QString **numberFormat** () const
- int **numberPrecision** () const
- QVector< double > **tickVector** () const
- QVector< QString > **tickVectorLabels** () const
- int **tickLengthIn** () const
- int **tickLengthOut** () const
- bool **subTicks** () const
- int **subTickLengthIn** () const
- int **subTickLengthOut** () const
- QPen **basePen** () const
- QPen **tickPen** () const
- QPen **subTickPen** () const
- QFont **labelFont** () const
- QColor **labelColor** () const
- QString **label** () const
- int **labelPadding** () const
- SelectableParts **selectedParts** () const
- SelectableParts **selectableParts** () const
- QFont **selectedTickLabelFont** () const

- QFont **selectedLabelFont** () const
- QColor **selectedTickLabelColor** () const
- QColor **selectedLabelColor** () const
- QPen **selectedBasePen** () const
- QPen **selectedTickPen** () const
- QPen **selectedSubTickPen** () const
- QCPPolarGrid * **grid** () const
- void **setBackground** (const QPixmap &pm)
- void **setBackground** (const QPixmap &pm, bool scaled, Qt::AspectRatioMode mode=Qt::KeepAspectRatio↵ByExpanding)
- void **setBackground** (const QBrush &brush)
- void **setBackgroundScaled** (bool scaled)
- void **setBackgroundScaledMode** (Qt::AspectRatioMode mode)
- void **setRangeDrag** (bool enabled)
- void **setRangeZoom** (bool enabled)
- void **setRangeZoomFactor** (double factor)
- Q_SLOT void **setRange** (const QCPRange &range)
- void **setRange** (double lower, double upper)
- void **setRange** (double position, double **size**, Qt::AlignmentFlag alignment)
- void **setRangeLower** (double lower)
- void **setRangeUpper** (double upper)
- void **setRangeReversed** (bool reversed)
- void **setAngle** (double degrees)
- void **setTicker** (QSharedPointer< QCPAxisTicker > ticker)
- void **setTicks** (bool show)
- void **setTickLabels** (bool show)
- void **setTickLabelPadding** (int padding)
- void **setTickLabelFont** (const QFont &font)
- void **setTickLabelColor** (const QColor &color)
- void **setTickLabelRotation** (double degrees)
- void **setTickLabelMode** (LabelMode mode)
- void **setNumberFormat** (const QString &formatCode)
- void **setNumberPrecision** (int precision)
- void **setTickLength** (int inside, int outside=0)
- void **setTickLengthIn** (int inside)
- void **setTickLengthOut** (int outside)
- void **setSubTicks** (bool show)
- void **setSubTickLength** (int inside, int outside=0)
- void **setSubTickLengthIn** (int inside)
- void **setSubTickLengthOut** (int outside)
- void **setBasePen** (const QPen &pen)
- void **setTickPen** (const QPen &pen)
- void **setSubTickPen** (const QPen &pen)
- void **setLabelFont** (const QFont &font)
- void **setLabelColor** (const QColor &color)
- void **setLabel** (const QString &str)
- void **setLabelPadding** (int padding)
- void **setLabelPosition** (Qt::AlignmentFlag position)
- void **setSelectedTickLabelFont** (const QFont &font)
- void **setSelectedLabelFont** (const QFont &font)
- void **setSelectedTickLabelColor** (const QColor &color)
- void **setSelectedLabelColor** (const QColor &color)
- void **setSelectedBasePen** (const QPen &pen)
- void **setSelectedTickPen** (const QPen &pen)
- void **setSelectedSubTickPen** (const QPen &pen)

- Q_SLOT void [setSelectableParts](#) (const QCPPolarAxisAngular::SelectableParts &selectableParts)
- Q_SLOT void [setSelectedParts](#) (const QCPPolarAxisAngular::SelectableParts &selectedParts)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=0) const Q_DECL_OVERRIDE
- virtual void [update](#) (UpdatePhase phase) Q_DECL_OVERRIDE
- virtual QList< [QCPLayoutElement](#) * > [elements](#) (bool recursive) const Q_DECL_OVERRIDE
- bool [removeGraph](#) ([QCPPolarGraph](#) *graph)
- int [radialAxisCount](#) () const
- [QCPPolarAxisRadial](#) * [radialAxis](#) (int index=0) const
- QList< [QCPPolarAxisRadial](#) * > [radialAxes](#) () const
- [QCPPolarAxisRadial](#) * [addRadialAxis](#) ([QCPPolarAxisRadial](#) *axis=0)
- bool [removeRadialAxis](#) ([QCPPolarAxisRadial](#) *axis)
- [QCPLayoutInset](#) * [insetLayout](#) () const
- QRegion [exactClipRegion](#) () const
- void [moveRange](#) (double diff)
- void [scaleRange](#) (double factor)
- void [scaleRange](#) (double factor, double [center](#))
- void [rescale](#) (bool onlyVisiblePlottables=false)
- double [coordToAngleRad](#) (double coord) const
- double [angleRadToCoord](#) (double angleRad) const
- void [pixelToCoord](#) (QPointF pixelPos, double &angleCoord, double &radiusCoord) const
- QPointF [coordToPixel](#) (double angleCoord, double radiusCoord) const
- [SelectablePart](#) [getPartAt](#) (const QPointF &pos) const
- int [left](#) () const
- int [right](#) () const
- int [top](#) () const
- int [bottom](#) () const
- int [width](#) () const
- int [height](#) () const
- QSize [size](#) () const
- QPoint [topLeft](#) () const
- QPoint [topRight](#) () const
- QPoint [bottomLeft](#) () const
- QPoint [bottomRight](#) () const
- QPointF [center](#) () const
- double [radius](#) () const

Public Member Functions inherited from [QCPLayoutElement](#)

- [QCPLayoutElement](#) ([QCustomPlot](#) *parentPlot=nullptr)
- [QCPLayout](#) * [layout](#) () const
- QRect [rect](#) () const
- QRect [outerRect](#) () const
- QMargins [margins](#) () const
- QMargins [minimumMargins](#) () const
- QCP::MarginSides [autoMargins](#) () const
- QSize [minimumSize](#) () const
- QSize [maximumSize](#) () const
- [SizeConstraintRect](#) [sizeConstraintRect](#) () const
- [QCPMarginGroup](#) * [marginGroup](#) ([QCP::MarginSide](#) side) const
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > [marginGroups](#) () const
- void [setOuterRect](#) (const QRect &rect)
- void [setMargins](#) (const QMargins &margins)
- void [setMinimumMargins](#) (const QMargins &margins)
- void [setAutoMargins](#) ([QCP::MarginSides](#) sides)

- void [setMinimumSize](#) (const QSize &size)
- void [setMinimumSize](#) (int width, int height)
- void [setMaximumSize](#) (const QSize &size)
- void [setMaximumSize](#) (int width, int height)
- void [setSizeConstraintRect](#) ([SizeConstraintRect](#) constraintRect)
- void [setMarginGroup](#) (QCP::MarginSides sides, [QCPMarginGroup](#) *group)
- virtual QSize [minimumOuterSizeHint](#) () const
- virtual QSize [maximumOuterSizeHint](#) () const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual [QCP::Interaction selectionCategory](#) () const Q_DECL_OVERRIDE
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE
- virtual void [wheelEvent](#) (QWheelEvent *event) Q_DECL_OVERRIDE
- bool [registerPolarGraph](#) ([QCPPolarGraph](#) *graph)
- void [drawBackground](#) ([QCPPainter](#) *painter, const QPointF &[center](#), double radius)
- void [setupTickVectors](#) ()
- QPen [getBasePen](#) () const
- QPen [getTickPen](#) () const
- QPen [getSubTickPen](#) () const
- QFont [getTickLabelFont](#) () const
- QFont [getLabelFont](#) () const
- QColor [getTickLabelColor](#) () const
- QColor [getLabelColor](#) () const

Protected Member Functions inherited from [QCPLayoutElement](#)

- virtual int [calculateAutoMargin](#) ([QCP::MarginSide](#) side)
- virtual void [layoutChanged](#) ()
- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayerable](#)

- virtual QRect **clipRect** () const
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionState↔ Changed)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- void **initializeParentPlot** (QCustomPlot *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QBrush **mBackgroundBrush**
- QPixmap **mBackgroundPixmap**
- QPixmap **mScaledBackgroundPixmap**
- bool **mBackgroundScaled**
- Qt::AspectRatioMode **mBackgroundScaledMode**
- [QCPLayoutInset](#) * **mInsetLayout**
- bool **mRangeDrag**
- bool **mRangeZoom**
- double **mRangeZoomFactor**
- double **mAngle**
- double **mAngleRad**
- SelectableParts **mSelectableParts**
- SelectableParts **mSelectedParts**
- QPen **mBasePen**
- QPen **mSelectedBasePen**
- int **mLabelPadding**
- QString **mLabel**
- QFont **mLabelFont**
- QFont **mSelectedLabelFont**
- QColor **mLabelColor**
- QColor **mSelectedLabelColor**
- bool **mTickLabels**
- QFont **mTickLabelFont**
- QFont **mSelectedTickLabelFont**
- QColor **mTickLabelColor**
- QColor **mSelectedTickLabelColor**
- int **mNumberPrecision**
- QLatin1Char **mNumberFormatChar**
- bool **mNumberBeautifulPowers**
- bool **mNumberMultiplyCross**
- bool **mTicks**
- bool **mSubTicks**
- int **mTickLengthIn**
- int **mTickLengthOut**
- int **mSubTickLengthIn**
- int **mSubTickLengthOut**
- QPen **mTickPen**
- QPen **mSelectedTickPen**
- QPen **mSubTickPen**
- QPen **mSelectedSubTickPen**
- [QCPRange](#) **mRange**

- bool **mRangeReversed**
- QPointF **mCenter**
- double **mRadius**
- QList< [QCPPolarAxisRadial](#) * > **mRadialAxes**
- [QCPPolarGrid](#) * **mGrid**
- QList< [QCPPolarGraph](#) * > **mGraphs**
- QSharedPointer< [QCPAxisTicker](#) > **mTicker**
- QVector< double > **mTickVector**
- QVector< QString > **mTickVectorLabels**
- QVector< QPointF > **mTickVectorCosSin**
- QVector< double > **mSubTickVector**
- QVector< QPointF > **mSubTickVectorCosSin**
- bool **mDragging**
- [QCPRange](#) **mDragAngularStart**
- QList< [QCPRange](#) > **mDragRadialStart**
- QCP::AntialiasedElements **mAADragBackup**
- QCP::AntialiasedElements **mNotAADragBackup**
- [QCPLabelPainterPrivate](#) **mLabelPainter**

Protected Attributes inherited from [QCPLayoutElement](#)

- [QCPLayout](#) * **mParentLayout**
- QSize **mMinimumSize**
- QSize **mMaximumSize**
- [SizeConstraintRect](#) **mSizeConstraintRect**
- QRect **mRect**
- QRect **mOuterRect**
- QMargins **mMargins**
- QMargins **mMinimumMargins**
- QCP::MarginSides **mAutoMargins**
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **mMarginGroups**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCustomPlot](#)
- class [QCPPolarGrid](#)
- class [QCPPolarGraph](#)

15.610.1 Detailed Description

The main container for polar plots, representing the angular axis as a circle.

Warning

In this [QCustomPlot](#) version, polar plots are a tech preview. Expect documentation and functionality to be incomplete, as well as changing public interfaces in the future.

15.610.2 Member Enumeration Documentation

15.610.2.1 LabelMode

enum [QCPPolarAxisAngular::LabelMode](#)
 TODO

15.610.2.2 SelectablePart

enum [QCPPolarAxisAngular::SelectablePart](#)
 Defines the selectable parts of an axis.

See also

[setSelectableParts](#), [setSelectedParts](#)

Enumerator

spNone	None of the selectable parts.
spAxis	The axis backbone and tick marks.
spTickLabels	Tick labels (numbers) of this axis (as a whole, not individually)
spAxisLabel	The axis label.

15.610.3 Constructor & Destructor Documentation

15.610.3.1 QCPPolarAxisAngular()

```
QCPPolarAxisAngular::QCPPolarAxisAngular (
    QCustomPlot * parentPlot ) [explicit]
```

Creates a QCPPolarAxis instance and sets default values. An axis is added for each of the four sides, the top and right axes are set invisible initially.

15.610.4 Member Function Documentation

15.610.4.1 addRadialAxis()

```
QCPPolarAxisRadial * QCPPolarAxisAngular::addRadialAxis (
    QCPPolarAxisRadial * axis = 0 )
```

Adds a new axis to the axis rect side specified with *type*, and returns it. If *axis* is 0, a new [QCPAxis](#) instance is created internally. [QCustomPlot](#) owns the returned axis, so if you want to remove an axis, use [removeAxis](#) instead of deleting it manually.

You may inject [QCPAxis](#) instances (or subclasses of [QCPAxis](#)) by setting *axis* to an axis that was previously created outside [QCustomPlot](#). It is important to note that [QCustomPlot](#) takes ownership of the axis, so you may not delete it afterwards. Further, the *axis* must have been created with this axis rect as parent and with the same axis type as specified in *type*. If this is not the case, a debug output is generated, the axis is not added, and the method returns 0.

This method can not be used to move *axis* between axis rects. The same *axis* instance must not be added multiple times to the same or different axis rects.

If an axis rect side already contains one or more axes, the lower and upper endings of the new axis ([QCPAxis::setLowerEnding](#), [QCPAxis::setUpperEnding](#)) are set to [QCPLineEnding::esHalfBar](#).

See also

[addAxes](#), [setupFullAxesBox](#)

15.610.4.2 applyDefaultAntialiasingHint()

```
void QCPPolarAxisAngular::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

15.610.4.3 bottom()

```
int QCPPolarAxisAngular::bottom ( ) const [inline]
```

Returns the pixel position of the bottom border of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.610.4.4 bottomLeft()

```
QPoint QCPPolarAxisAngular::bottomLeft ( ) const [inline]
```

Returns the bottom left corner of this axis rect in pixels. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.610.4.5 bottomRight()

```
QPoint QCPPolarAxisAngular::bottomRight ( ) const [inline]
```

Returns the bottom right corner of this axis rect in pixels. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.610.4.6 center()

```
QPoint QCPPolarAxisAngular::center ( ) const [inline]
```

Returns the center of this axis rect in pixels. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.610.4.7 coordToPixel()

```
QPointF QCPPolarAxisAngular::coordToPixel (
    double angleCoord,
    double radiusCoord ) const
```

Transforms *value*, in coordinates of the axis, to pixel coordinates of the [QCustomPlot](#) widget.

15.610.4.8 draw()

```
void QCPPolarAxisAngular::draw (
    QCPainter * painter ) [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

15.610.4.9 elements()

```
QList< QCPLayoutElement * > QCPPolarAxisAngular::elements (
    bool recursive ) const [virtual]
```

Returns a list of all child elements in this layout element. If *recursive* is true, all sub-child elements are included in the list, too.

Warning

There may be `nullptr` entries in the returned list. For example, [QCPLayoutGrid](#) may have empty cells which yield `nullptr` at the respective index.

Reimplemented from [QCPLayoutElement](#).

15.610.4.10 getPartAt()

```
QCPPolarAxisAngular::SelectablePart QCPPolarAxisAngular::getPartAt (
    const QPointF & pos ) const
```

Returns the part of the axis that is hit by *pos* (in pixels). The return value of this function is independent of the user-selectable parts defined with [setSelectableParts](#). Further, this function does not change the current selection state of the axis.

If the axis is not visible ([setVisible](#)), this function always returns [spNone](#).

See also

[setSelectedParts](#), [setSelectableParts](#), [QCustomPlot::setInteractions](#)

15.610.4.11 height()

```
int QCPPolarAxisAngular::height ( ) const [inline]
```

Returns the pixel height of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.610.4.12 insetLayout()

```
QCPLayoutInset * QCPPolarAxisAngular::insetLayout ( ) const [inline]
```

Returns the inset layout of this axis rect. It can be used to place other layout elements (or even layouts with multiple other elements) inside/on top of an axis rect.

See also

[QCPLayoutInset](#)

15.610.4.13 left()

```
int QCPPolarAxisAngular::left ( ) const [inline]
```

Returns the pixel position of the left border of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.610.4.14 mouseMoveEvent()

```
void QCPPolarAxisAngular::mouseMoveEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user moves the mouse while holding a mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *startPos* indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction. The default implementation does nothing.

See also

[mousePressEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

15.610.4.15 mousePressEvent()

```
void QCPPolarAxisAngular::mousePressEvent (
    QMouseEvent * event,
    const QVariant & details ) [protected], [virtual]
```

This event gets called when the user presses a mouse button while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *details* contains layerable-specific details about the hit, which were generated in the previous call to [selectTest](#). For example, One-dimensional plottables like [QCPGraph](#) or [QCPBars](#) convey the clicked data point in the *details* parameter, as [QCPDataSelection](#) packed as `QVariant`. Multi-part objects convey the specific `SelectablePart` that was hit (e.g. [QCPAxis::SelectablePart](#) in the case of axes).

[QCustomPlot](#) uses an event propagation system that works the same as Qt's system. If your layerable doesn't reimplement the [mousePressEvent](#) or explicitly calls `event->ignore()` in its reimplementation, the event will be propagated to the next layerable in the stacking order.

Once a layerable has accepted the [mousePressEvent](#), it is considered the mouse grabber and will receive all following calls to [mouseMoveEvent](#) or [mouseReleaseEvent](#) for this mouse interaction (a "mouse interaction" in this context ends with the release).

The default implementation does nothing except explicitly ignoring the event with `event->ignore()`.

See also

[mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

15.610.4.16 mouseReleaseEvent()

```
void QCPPolarAxisAngular::mouseReleaseEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user releases the mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *startPos* indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction. The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

15.610.4.17 moveRange()

```
void QCPPolarAxisAngular::moveRange (
    double diff )
```

If the scale type (setScaleType) is `stLinear`, *diff* is added to the lower and upper bounds of the range. The range is simply moved by *diff*.

If the scale type is `stLogarithmic`, the range bounds are multiplied by *diff*. This corresponds to an apparent "linear" move in logarithmic scaling by a distance of $\log(\text{diff})$.

15.610.4.18 pixelToCoord()

```
void QCPPolarAxisAngular::pixelToCoord (
    QPointF pixelPos,
    double & angleCoord,
    double & radiusCoord ) const
```

Transforms *value*, in pixel coordinates of the [QCustomPlot](#) widget, to axis coordinates.

15.610.4.19 radialAxes()

```
QList< QCPPolarAxisRadial * > QCPPolarAxisAngular::radialAxes ( ) const
```

Returns all axes on the axis rect sides specified with *types*.

types may be a single [QCPAxis::AxisType](#) or an `or`-combination, to get the axes of multiple sides.

See also

[axis](#)

15.610.4.20 radialAxis()

```
QCPPolarAxisRadial * QCPPolarAxisAngular::radialAxis (
    int index = 0 ) const
```

Returns the axis with the given *index* on the axis rect side specified with *type*.

See also

[axisCount](#), [axes](#)

15.610.4.21 radialAxisCount()

```
int QCPPolarAxisAngular::radialAxisCount ( ) const
```

Returns the number of axes on the axis rect side specified with *type*.

See also

[axis](#)

15.610.4.22 removeRadialAxis()

```
bool QCPPolarAxisAngular::removeRadialAxis (
    QCPPolarAxisRadial * radialAxis )
```

Removes the specified *axis* from the axis rect and deletes it.

Returns true on success, i.e. if *axis* was a valid axis in this axis rect.

See also

[addAxis](#)

15.610.4.23 rescale()

```
void QCPPolarAxisAngular::rescale (
    bool onlyVisiblePlottables = false )
```

Changes the axis range such that all plottables associated with this axis are fully visible in that dimension.

See also

[QCPAbstractPlottable::rescaleAxes](#), [QCustomPlot::rescaleAxes](#)

15.610.4.24 right()

```
int QCPPolarAxisAngular::right ( ) const [inline]
```

Returns the pixel position of the right border of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.610.4.25 scaleRange() [1/2]

```
void QCPPolarAxisAngular::scaleRange (
    double factor )
```

Scales the range of this axis by *factor* around the center of the current axis range. For example, if *factor* is 2.0, then the axis range will double its size, and the point at the axis range center won't have changed its position in the [QCustomPlot](#) widget (i.e. coordinates around the center will have moved symmetrically closer).

If you wish to scale around a different coordinate than the current axis range center, use the overload [scaleRange\(double factor, double center\)](#).

15.610.4.26 scaleRange() [2/2]

```
void QCPPolarAxisAngular::scaleRange (
    double factor,
    double center )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Scales the range of this axis by *factor* around the coordinate *center*. For example, if *factor* is 2.0, *center* is 1.0, then the axis range will double its size, and the point at coordinate 1.0 won't have changed its position in the [QCustomPlot](#) widget (i.e. coordinates around 1.0 will have moved symmetrically closer to 1.0).

See also

[scaleRange\(double factor\)](#)

15.610.4.27 selectionCategory()

`QCP::Interaction QCPPolarAxisAngular::selectionCategory ( ) const` [protected], [virtual]
 Reimplemented from [QCPLayerable](#).

15.610.4.28 selectTest()

```
double QCPPolarAxisAngular::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = 0 ) const [virtual]
```

Layout elements are sensitive to events inside their outer rect. If *pos* is within the outer rect, this method returns a value corresponding to 0.99 times the parent plot's selection tolerance. However, layout elements are not selectable by default. So if *onlySelectable* is true, -1.0 is returned.

See [QCPLayerable::selectTest](#) for a general explanation of this virtual method.

[QCPLayoutElement](#) subclasses may reimplement this method to provide more specific selection test behaviour.

Reimplemented from [QCPLayoutElement](#).

15.610.4.29 setBackground() [1/3]

```
void QCPPolarAxisAngular::setBackground (
    const QBrush & brush )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets *brush* as the background brush. The axis rect background will be filled with this brush. Since axis rects place themselves on the "background" layer by default, the axis rect backgrounds are usually drawn below everything else.

The brush will be drawn before (under) any background pixmap, which may be specified with [setBackground\(const QPixmap &pm\)](#). To disable drawing of a background brush, set *brush* to `Qt::NoBrush`.

See also

[setBackground\(const QPixmap &pm\)](#)

15.610.4.30 setBackground() [2/3]

```
void QCPPolarAxisAngular::setBackground (
    const QPixmap & pm )
```

Sets *pm* as the axis background pixmap. The axis background pixmap will be drawn inside the axis rect. Since axis rects place themselves on the "background" layer by default, the axis rect backgrounds are usually drawn below everything else.

For cases where the provided pixmap doesn't have the same size as the axis rect, scaling can be enabled with [setBackgroundScaled](#) and the scaling mode (i.e. whether and how the aspect ratio is preserved) can be set with [setBackgroundScaledMode](#). To set all these options in one call, consider using the overloaded version of this function.

Below the pixmap, the axis rect may be optionally filled with a brush, if specified with [setBackground\(const QBrush &brush\)](#).

See also

[setBackgroundScaled](#), [setBackgroundScaledMode](#), [setBackground\(const QBrush &brush\)](#)

15.610.4.31 setBackground() [3/3]

```
void QCPPolarAxisAngular::setBackground (
    const QPixmap & pm,
    bool scaled,
    Qt::AspectRatioMode mode = Qt::KeepAspectRatioByExpanding )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Allows setting the background pixmap of the axis rect, whether it shall be scaled and how it shall be scaled in one call.

See also

[setBackground\(const QPixmap &pm\)](#), [setBackgroundScaled](#), [setBackgroundScaledMode](#)

15.610.4.32 setBackgroundScaled()

```
void QCPPolarAxisAngular::setBackgroundScaled (
    bool scaled )
```

Sets whether the axis background pixmap shall be scaled to fit the axis rect or not. If *scaled* is set to true, you may control whether and how the aspect ratio of the original pixmap is preserved with [setBackgroundScaledMode](#).

Note that the scaled version of the original pixmap is buffered, so there is no performance penalty on replots. (Except when the axis rect dimensions are changed continuously.)

See also

[setBackground](#), [setBackgroundScaledMode](#)

15.610.4.33 setBackgroundScaledMode()

```
void QCPPolarAxisAngular::setBackgroundScaledMode (
    Qt::AspectRatioMode mode )
```

If scaling of the axis background pixmap is enabled ([setBackgroundScaled](#)), use this function to define whether and how the aspect ratio of the original pixmap passed to [setBackground](#) is preserved.

See also

[setBackground](#), [setBackgroundScaled](#)

15.610.4.34 setBasePen()

```
void QCPPolarAxisAngular::setBasePen (
    const QPen & pen )
```

Sets the pen, the axis base line is drawn with.

See also

[setTickPen](#), [setSubTickPen](#)

15.610.4.35 setLabel()

```
void QCPPolarAxisAngular::setLabel (
    const QString & str )
```

Sets the text of the axis label that will be shown below/above or next to the axis, depending on its orientation. To disable axis labels, pass an empty string as *str*.

15.610.4.36 setLabelColor()

```
void QCPPolarAxisAngular::setLabelColor (
    const QColor & color )
```

Sets the color of the axis label.

See also

[setLabelFont](#)

15.610.4.37 setLabelFont()

```
void QCPPolarAxisAngular::setLabelFont (
    const QFont & font )
```

Sets the font of the axis label.

See also

[setLabelColor](#)

15.610.4.38 setLabelPadding()

```
void QCPPolarAxisAngular::setLabelPadding (
    int padding )
```

Sets the distance between the tick labels and the axis label.

See also

[setTickLabelPadding](#), [setPadding](#)

15.610.4.39 setNumberFormat()

```
void QCPPolarAxisAngular::setNumberFormat (
    const QString & formatCode )
```

Sets the number format for the numbers in tick labels. This *formatCode* is an extended version of the format code used e.g. by `QString::number()` and `QLocale::toString()`. For reference about that, see the "Argument Formats" section in the detailed description of the `QString` class.

formatCode is a string of one, two or three characters. The first character is identical to the normal format code used by Qt. In short, this means: 'e'/'E' scientific format, 'f' fixed format, 'g'/'G' scientific or fixed, whichever is shorter.

The second and third characters are optional and specific to [QCustomPlot](#):

If the first char was 'e' or 'g', numbers are/might be displayed in the scientific format, e.g. "5.5e9", which might be visually unappealing in a plot. So when the second char of *formatCode* is set to 'b' (for "beautiful"), those exponential numbers are formatted in a more natural way, i.e. "5.5 [multiplication sign] 10 [superscript] 9". By default, the multiplication sign is a centered dot. If instead a cross should be shown (as is usual in the USA), the third char of *formatCode* can be set to 'c'. The inserted multiplication signs are the UTF-8 characters 215 (0xD7) for the cross and 183 (0xB7) for the dot.

Examples for *formatCode*:

- `g` normal format code behaviour. If number is small, fixed format is used, if number is large, normal scientific format is used
- `gb` If number is small, fixed format is used, if number is large, scientific format is used with beautifully typeset decimal powers and a dot as multiplication sign
- `ebc` All numbers are in scientific format with beautifully typeset decimal power and a cross as multiplication sign
- `fb` illegal format code, since fixed format doesn't support (or need) beautifully typeset decimal powers. Format code will be reduced to 'f'.
- `hello` illegal format code, since first char is not 'e', 'E', 'f', 'g' or 'G'. Current format code will not be changed.

15.610.4.40 setNumberPrecision()

```
void QCPPolarAxisAngular::setNumberPrecision (
    int precision )
```

Sets the precision of the tick label numbers. See `QLocale::toString(double i, char f, int prec)` for details. The effect of precisions are most notably for number Formats starting with 'e', see [setNumberFormat](#)

15.610.4.41 setRange() [1/3]

```
void QCPPolarAxisAngular::setRange (
    const QCPRange & range )
```

Sets the range of the axis.

This slot may be connected with the `rangeChanged` signal of another axis so this axis is always synchronized with the other axis range, when it changes.

To invert the direction of an axis, use [setRangeReversed](#).

15.610.4.42 setRange() [2/3]

```
void QCPPolarAxisAngular::setRange (
    double lower,
    double upper )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the lower and upper bound of the axis range.

To invert the direction of an axis, use [setRangeReversed](#).

There is also a slot to set a range, see [setRange\(const QCPRange &range\)](#).

15.610.4.43 setRange() [3/3]

```
void QCPPolarAxisAngular::setRange (
    double position,
    double size,
    Qt::AlignmentFlag alignment )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the range of the axis.

The *position* coordinate indicates together with the *alignment* parameter, where the new range will be positioned. *size* defines the size of the new axis range. *alignment* may be `Qt::AlignLeft`, `Qt::AlignRight` or `Qt::AlignCenter`. This will cause the left border, right border, or center of the range to be aligned with *position*. Any other values of *alignment* will default to `Qt::AlignCenter`.

15.610.4.44 setRangeLower()

```
void QCPPolarAxisAngular::setRangeLower (
    double lower )
```

Sets the lower bound of the axis range. The upper bound is not changed.

See also

[setRange](#)

15.610.4.45 setRangeReversed()

```
void QCPPolarAxisAngular::setRangeReversed (
    bool reversed )
```

Sets whether the axis range (direction) is displayed reversed. Normally, the values on horizontal axes increase left to right, on vertical axes bottom to top. When *reversed* is set to true, the direction of increasing values is inverted.

Note that the range and data interface stays the same for reversed axes, e.g. the *lower* part of the [setRange](#) interface will still reference the mathematically smaller number than the *upper* part.

15.610.4.46 setRangeUpper()

```
void QCPPolarAxisAngular::setRangeUpper (
    double upper )
```

Sets the upper bound of the axis range. The lower bound is not changed.

See also

[setRange](#)

15.610.4.47 setSelectableParts()

```
void QCPPolarAxisAngular::setSelectableParts (
    const QCPPolarAxisAngular::SelectableParts & selectableParts )
```

Sets whether the user can (de-)select the parts in *selectable* by clicking on the [QCustomPlot](#) surface. (When [QCustomPlot::setInteractions](#) contains `iSelectAxes`.)

However, even when *selectable* is set to a value not allowing the selection of a specific part, it is still possible to set the selection of this part manually, by calling [setSelectedParts](#) directly.

See also

[SelectablePart](#), [setSelectedParts](#)

15.610.4.48 setSelectedBasePen()

```
void QCPPolarAxisAngular::setSelectedBasePen (
    const QPen & pen )
```

Sets the pen that is used to draw the axis base line when selected.

See also

[setBasePen](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.610.4.49 setSelectedLabelColor()

```
void QCPPolarAxisAngular::setSelectedLabelColor (
    const QColor & color )
```

Sets the color that is used for the axis label when it is selected.

See also

[setLabelColor](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.610.4.50 setSelectedLabelFont()

```
void QCPPolarAxisAngular::setSelectedLabelFont (
    const QFont & font )
```

Sets the font that is used for the axis label when it is selected.

See also

[setLabelFont](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.610.4.51 setSelectedParts()

```
void QCPPolarAxisAngular::setSelectedParts (
    const QCPPolarAxisAngular::SelectableParts & selectedParts )
```

Sets the selected state of the respective axis parts described by [SelectablePart](#). When a part is selected, it uses a different pen/font.

The entire selection mechanism for axes is handled automatically when [QCustomPlot::setInteractions](#) contains `iSelectAxes`. You only need to call this function when you wish to change the selection state manually.

This function can change the selection state of a part, independent of the [setSelectableParts](#) setting.

emits the `selectionChanged` signal when *selected* is different from the previous selection state.

See also

[SelectablePart](#), [setSelectableParts](#), [selectTest](#), [setSelectedBasePen](#), [setSelectedTickPen](#), [setSelectedSubTickPen](#), [setSelectedTickLabelFont](#), [setLabelFont](#), [setSelectedTickLabelColor](#), [setLabelColor](#)

15.610.4.52 setSelectedSubTickPen()

```
void QCPPolarAxisAngular::setSelectedSubTickPen (
    const QPen & pen )
```

Sets the pen that is used to draw the subticks when selected.

See also

[setSubTickPen](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.610.4.53 setSelectedTickLabelColor()

```
void QCPPolarAxisAngular::setSelectedTickLabelColor (
    const QColor & color )
```

Sets the color that is used for tick labels when they are selected.

See also

[setTickLabelColor](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.610.4.54 setSelectedTickLabelFont()

```
void QCPPolarAxisAngular::setSelectedTickLabelFont (
    const QFont & font )
```

Sets the font that is used for tick labels when they are selected.

See also

[setTickLabelFont](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.610.4.55 setSelectedTickPen()

```
void QCPPolarAxisAngular::setSelectedTickPen (
    const QPen & pen )
```

Sets the pen that is used to draw the (major) ticks when selected.

See also

[setTickPen](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.610.4.56 setSubTickLength()

```
void QCPPolarAxisAngular::setSubTickLength (
    int inside,
    int outside = 0 )
```

Sets the length of the subticks in pixels. *inside* is the length the subticks will reach inside the plot and *outside* is the length they will reach outside the plot. If *outside* is greater than zero, the tick labels and axis label will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setTickLength](#), [setSubTickLengthIn](#), [setSubTickLengthOut](#)

15.610.4.57 setSubTickLengthIn()

```
void QCPPolarAxisAngular::setSubTickLengthIn (
    int inside )
```

Sets the length of the inward subticks in pixels. *inside* is the length the subticks will reach inside the plot.

See also

[setSubTickLengthOut](#), [setSubTickLength](#), [setTickLength](#)

15.610.4.58 setSubTickLengthOut()

```
void QCPPolarAxisAngular::setSubTickLengthOut (
    int outside )
```

Sets the length of the outward subticks in pixels. *outside* is the length the subticks will reach outside the plot. If *outside* is greater than zero, the tick labels will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setSubTickLengthIn](#), [setSubTickLength](#), [setTickLength](#)

15.610.4.59 setSubTickPen()

```
void QCPPolarAxisAngular::setSubTickPen (
    const QPen & pen )
```

Sets the pen, subtick marks will be drawn with.

See also

[setSubTickCount](#), [setSubTickLength](#), [setBasePen](#)

15.610.4.60 setSubTicks()

```
void QCPPolarAxisAngular::setSubTicks (
    bool show )
```

Sets whether sub tick marks are displayed.

Sub ticks are only potentially visible if (major) ticks are also visible (see [setTicks](#))

See also

[setTicks](#)

15.610.4.61 setTicker()

```
void QCPPolarAxisAngular::setTicker (
    QSharedPointer< QCPAxisTicker > ticker )
```

The axis ticker is responsible for generating the tick positions and tick labels. See the documentation of [QCPAxisTicker](#) for details on how to work with axis tickers.

You can change the tick positioning/labeling behaviour of this axis by setting a different [QCPAxisTicker](#) subclass using this method. If you only wish to modify the currently installed axis ticker, access it via `ticker`.

Since the ticker is stored in the axis as a shared pointer, multiple axes may share the same axis ticker simply by passing the same shared pointer to multiple axes.

See also

`ticker`

15.610.4.62 setTickLabelColor()

```
void QCPPolarAxisAngular::setTickLabelColor (
    const QColor & color )
```

Sets the color of the tick labels.

See also

[setTickLabels](#), [setTickLabelFont](#)

15.610.4.63 setTickLabelFont()

```
void QCPPolarAxisAngular::setTickLabelFont (
    const QFont & font )
```

Sets the font of the tick labels.

See also

[setTickLabels](#), [setTickLabelColor](#)

15.610.4.64 setTickLabelPadding()

```
void QCPPolarAxisAngular::setTickLabelPadding (
    int padding )
```

Sets the distance between the axis base line (including any outward ticks) and the tick labels.

See also

[setLabelPadding](#), [setPadding](#)

15.610.4.65 setTickLabelRotation()

```
void QCPPolarAxisAngular::setTickLabelRotation (
    double degrees )
```

Sets the rotation of the tick labels. If *degrees* is zero, the labels are drawn normally. Else, the tick labels are drawn rotated by *degrees* clockwise. The specified angle is bound to values from -90 to 90 degrees.

If *degrees* is exactly -90, 0 or 90, the tick labels are centered on the tick coordinate. For other angles, the label is drawn with an offset such that it seems to point toward or away from the tick mark.

15.610.4.66 setTickLabels()

```
void QCPPolarAxisAngular::setTickLabels (
    bool show )
```

Sets whether tick labels are displayed. Tick labels are the numbers drawn next to tick marks.

15.610.4.67 setTickLength()

```
void QCPPolarAxisAngular::setTickLength (
    int inside,
    int outside = 0 )
```

Sets the length of the ticks in pixels. *inside* is the length the ticks will reach inside the plot and *outside* is the length they will reach outside the plot. If *outside* is greater than zero, the tick labels and axis label will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setSubTickLength](#), [setTickLengthIn](#), [setTickLengthOut](#)

15.610.4.68 setTickLengthIn()

```
void QCPPolarAxisAngular::setTickLengthIn (
    int inside )
```

Sets the length of the inward ticks in pixels. *inside* is the length the ticks will reach inside the plot.

See also

[setTickLengthOut](#), [setTickLength](#), [setSubTickLength](#)

15.610.4.69 setTickLengthOut()

```
void QCPPolarAxisAngular::setTickLengthOut (
    int outside )
```

Sets the length of the outward ticks in pixels. *outside* is the length the ticks will reach outside the plot. If *outside* is greater than zero, the tick labels and axis label will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setTickLengthIn](#), [setTickLength](#), [setSubTickLength](#)

15.610.4.70 setTickPen()

```
void QCPPolarAxisAngular::setTickPen (
    const QPen & pen )
```

Sets the pen, tick marks will be drawn with.

See also

[setTickLength](#), [setBasePen](#)

15.610.4.71 setTicks()

```
void QCPPolarAxisAngular::setTicks (
    bool show )
```

Sets whether tick marks are displayed.

Note that setting *show* to false does not imply that tick labels are invisible, too. To achieve that, see [setTickLabels](#).

See also

[setSubTicks](#)

15.610.4.72 size()

```
QSize QCPPolarAxisAngular::size ( ) const [inline]
```

Returns the pixel size of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.610.4.73 top()

```
int QCPPolarAxisAngular::top ( ) const [inline]
```

Returns the pixel position of the top border of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.610.4.74 topLeft()

```
QPoint QCPPolarAxisAngular::topLeft ( ) const [inline]
```

Returns the top left corner of this axis rect in pixels. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.610.4.75 topRight()

```
QPoint QCPPolarAxisAngular::topRight ( ) const [inline]
```

Returns the top right corner of this axis rect in pixels. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

15.610.4.76 update()

```
void QCPPolarAxisAngular::update (
    UpdatePhase phase ) [virtual]
```

This method is called automatically upon replot and doesn't need to be called by users of [QCPPolarAxisAngular](#). Calls the base class implementation to update the margins (see [QCPLayoutElement::update](#)), and finally passes the [rect](#) to the inset layout ([insetLayout](#)) and calls its `QCPIInsetLayout::update` function.

\see [baseclassmethod](#)

Reimplemented from [QCPLayoutElement](#).

15.610.4.77 wheelEvent()

```
void QCPPolarAxisAngular::wheelEvent (
    QWheelEvent * event ) [protected], [virtual]
```

This event gets called when the user turns the mouse scroll wheel while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`.

The `event->angleDelta()` indicates how far the mouse wheel was turned, which is usually +/- 120 for single rotation steps. However, if the mouse wheel is turned rapidly, multiple steps may accumulate to one event, making the delta larger. On the other hand, if the wheel has very smooth steps or none at all, the delta may be smaller.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#)

Reimplemented from [QCPLayerable](#).

15.610.4.78 width()

```
int QCPPolarAxisAngular::width ( ) const [inline]
```

Returns the pixel width of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

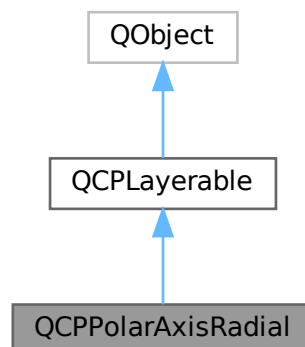
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

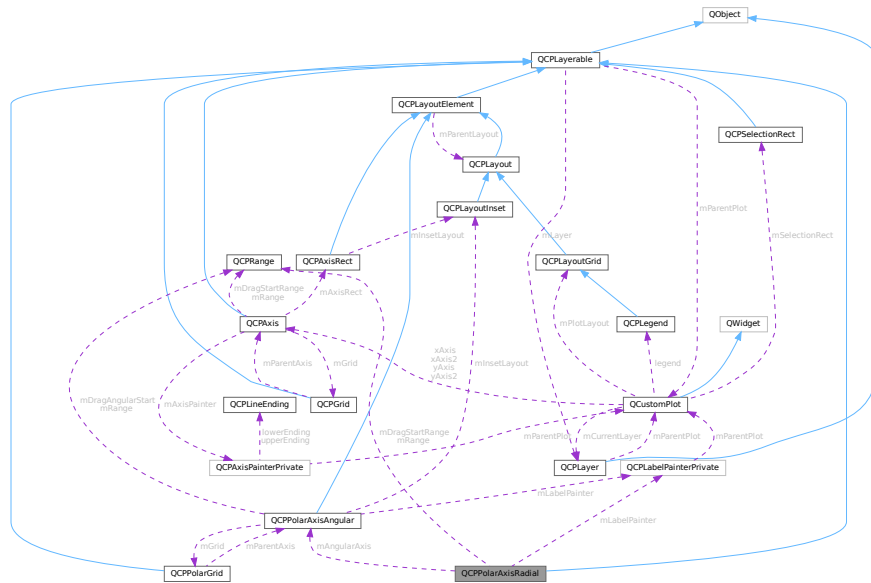
15.611 QCPPolarAxisRadial Class Reference

The radial axis inside a radial plot.

Inheritance diagram for [QCPPolarAxisRadial](#):



Collaboration diagram for QCPolarAxisRadial:



Public Types

- enum **AngleReference** { **arAbsolute** , **arAngularAxis** }
- enum **ScaleType** { **stLinear** , **stLogarithmic** }
- enum **SelectablePart** { **spNone** = 0 , **spAxis** = 0x001 , **spTickLabels** = 0x002 , **spAxisLabel** = 0x004 }
- enum **LabelMode** { **lmUpright** , **lmRotated** }

Signals

- void [rangeChanged](#) (const [QCPRange](#) &newRange)
- void [rangeChanged](#) (const [QCPRange](#) &newRange, const [QCPRange](#) &oldRange)
- void [scaleTypeChanged](#) ([QCPPolarAxisRadial::ScaleType](#) scaleType)
- void [selectionChanged](#) (const [QCPPolarAxisRadial::SelectableParts](#) &parts)
- void [selectableChanged](#) (const [QCPPolarAxisRadial::SelectableParts](#) &parts)

Signals inherited from QCPLayerable

- void layerChanged (QCPLayer *newLayer)

Public Member Functions

- **QCPPolarAxisRadial** (**QCPPolarAxisAngular** *parent)
- bool **rangeDrag** () const
- bool **rangeZoom** () const
- double **rangeZoomFactor** () const
- **QCPPolarAxisAngular** * **angularAxis** () const
- **ScaleType** **scaleType** () const
- const **QCPRange** **range** () const
- bool **rangeReversed** () const
- double **angle** () const
- **AngleReference** **angleReference** () const
- QSharedPointer< **QCPAxisTicker** > **ticker** () const
- bool **ticks** () const

- bool **tickLabels** () const
- int **tickLabelPadding** () const
- QFont **tickLabelFont** () const
- QColor **tickLabelColor** () const
- double **tickLabelRotation** () const
- LabelMode **tickLabelMode** () const
- QString **numberFormat** () const
- int **numberPrecision** () const
- QVector< double > **tickVector** () const
- QVector< double > **subTickVector** () const
- QVector< QString > **tickVectorLabels** () const
- int **tickLengthIn** () const
- int **tickLengthOut** () const
- bool **subTicks** () const
- int **subTickLengthIn** () const
- int **subTickLengthOut** () const
- QPen **basePen** () const
- QPen **tickPen** () const
- QPen **subTickPen** () const
- QFont **labelFont** () const
- QColor **labelColor** () const
- QString **label** () const
- int **labelPadding** () const
- SelectableParts **selectedParts** () const
- SelectableParts **selectableParts** () const
- QFont **selectedTickLabelFont** () const
- QFont **selectedLabelFont** () const
- QColor **selectedTickLabelColor** () const
- QColor **selectedLabelColor** () const
- QPen **selectedBasePen** () const
- QPen **selectedTickPen** () const
- QPen **selectedSubTickPen** () const
- void **setRangeDrag** (bool enabled)
- void **setRangeZoom** (bool enabled)
- void **setRangeZoomFactor** (double factor)
- Q_SLOT void **setScaleType** (QCPPolarAxisRadial::ScaleType type)
- Q_SLOT void **setRange** (const QCPRange &range)
- void **setRange** (double lower, double upper)
- void **setRange** (double position, double size, Qt::AlignmentFlag alignment)
- void **setRangeLower** (double lower)
- void **setRangeUpper** (double upper)
- void **setRangeReversed** (bool reversed)
- void **setAngle** (double degrees)
- void **setAngleReference** (AngleReference reference)
- void **setTicker** (QSharedPointer< QCPAxisTicker > ticker)
- void **setTicks** (bool show)
- void **setTickLabels** (bool show)
- void **setTickLabelPadding** (int padding)
- void **setTickLabelFont** (const QFont &font)
- void **setTickLabelColor** (const QColor &color)
- void **setTickLabelRotation** (double degrees)
- void **setTickLabelMode** (LabelMode mode)
- void **setNumberFormat** (const QString &formatCode)
- void **setNumberPrecision** (int precision)
- void **setTickLength** (int inside, int outside=0)

- void [setTickLengthIn](#) (int inside)
- void [setTickLengthOut](#) (int outside)
- void [setSubTicks](#) (bool show)
- void [setSubTickLength](#) (int inside, int outside=0)
- void [setSubTickLengthIn](#) (int inside)
- void [setSubTickLengthOut](#) (int outside)
- void [setBasePen](#) (const QPen &pen)
- void [setTickPen](#) (const QPen &pen)
- void [setSubTickPen](#) (const QPen &pen)
- void [setLabelFont](#) (const QFont &font)
- void [setLabelColor](#) (const QColor &color)
- void [setLabel](#) (const QString &str)
- void [setLabelPadding](#) (int padding)
- void [setSelectedTickLabelFont](#) (const QFont &font)
- void [setSelectedLabelFont](#) (const QFont &font)
- void [setSelectedTickLabelColor](#) (const QColor &color)
- void [setSelectedLabelColor](#) (const QColor &color)
- void [setSelectedBasePen](#) (const QPen &pen)
- void [setSelectedTickPen](#) (const QPen &pen)
- void [setSelectedSubTickPen](#) (const QPen &pen)
- Q_SLOT void [setSelectableParts](#) (const QCPPolarAxisRadial::SelectableParts &selectableParts)
- Q_SLOT void [setSelectedParts](#) (const QCPPolarAxisRadial::SelectableParts &selectedParts)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=0) const Q_DECL_OVERRIDE
- void [moveRange](#) (double diff)
- void [scaleRange](#) (double factor)
- void [scaleRange](#) (double factor, double center)
- void [rescale](#) (bool onlyVisiblePlottables=false)
- void [pixelToCoord](#) (QPointF pixelPos, double &angleCoord, double &radiusCoord) const
- QPointF [coordToPixel](#) (double angleCoord, double radiusCoord) const
- double [coordToRadius](#) (double coord) const
- double [radiusToCoord](#) (double radius) const
- [SelectablePart](#) [getPartAt](#) (const QPointF &pos) const

Public Member Functions inherited from QCPLayerable

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE
- virtual void [wheelEvent](#) (QWheelEvent *event) Q_DECL_OVERRIDE
- void [updateGeometry](#) (const QPointF ¢er, double radius)
- void [setupTickVectors](#) ()
- QPen [getBasePen](#) () const
- QPen [getTickPen](#) () const
- QPen [getSubTickPen](#) () const
- QFont [getTickLabelFont](#) () const
- QFont [getLabelFont](#) () const
- QColor [getTickLabelColor](#) () const
- QColor [getLabelColor](#) () const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot)
- virtual QRect [clipRect](#) () const
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- void [initializeParentPlot](#) ([QCustomPlot](#) *parentPlot)
- void [setParentLayerable](#) ([QCPLayerable](#) *parentLayerable)
- bool [moveToLayer](#) ([QCPLayer](#) *layer, bool prepend)
- void [applyAntialiasingHint](#) ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- bool [mRangeDrag](#)
- bool [mRangeZoom](#)
- double [mRangeZoomFactor](#)
- [QCPPolarAxisAngular](#) * [mAngularAxis](#)
- double [mAngle](#)
- [AngleReference](#) [mAngleReference](#)
- SelectableParts [mSelectableParts](#)
- SelectableParts [mSelectedParts](#)
- QPen [mBasePen](#)
- QPen [mSelectedBasePen](#)
- int [mLabelPadding](#)
- QString [mLabel](#)
- QFont [mLabelFont](#)
- QFont [mSelectedLabelFont](#)
- QColor [mLabelColor](#)
- QColor [mSelectedLabelColor](#)
- bool [mTickLabels](#)
- QFont [mTickLabelFont](#)
- QFont [mSelectedTickLabelFont](#)
- QColor [mTickLabelColor](#)
- QColor [mSelectedTickLabelColor](#)

- int **mNumberPrecision**
- QLatin1Char **mNumberFormatChar**
- bool **mNumberBeautifulPowers**
- bool **mNumberMultiplyCross**
- bool **mTicks**
- bool **mSubTicks**
- int **mTickLengthIn**
- int **mTickLengthOut**
- int **mSubTickLengthIn**
- int **mSubTickLengthOut**
- QPen **mTickPen**
- QPen **mSelectedTickPen**
- QPen **mSubTickPen**
- QPen **mSelectedSubTickPen**
- [QCPRange](#) **mRange**
- bool **mRangeReversed**
- [ScaleType](#) **mScaleType**
- QPointF **mCenter**
- double **mRadius**
- QSharedPointer< [QCPAxisTicker](#) > **mTicker**
- QVector< double > **mTickVector**
- QVector< QString > **mTickVectorLabels**
- QVector< double > **mSubTickVector**
- bool **mDragging**
- [QCPRange](#) **mDragStartRange**
- QCP::AntialiasedElements **mAADragBackup**
- QCP::AntialiasedElements **mNotAADragBackup**
- [QCPLabelPainterPrivate](#) **mLabelPainter**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCustomPlot](#)
- class [QCPPolarAxisAngular](#)

15.611.1 Detailed Description

The radial axis inside a radial plot.

Warning

In this [QCustomPlot](#) version, polar plots are a tech preview. Expect documentation and functionality to be incomplete, as well as changing public interfaces in the future.

Each axis holds an instance of [QCPAxisTicker](#) which is used to generate the tick coordinates and tick labels. You can access the currently installed [ticker](#) or set a new one (possibly one of the specialized subclasses, or your own subclass) via [setTicker](#). For details, see the documentation of [QCPAxisTicker](#).

15.611.2 Member Enumeration Documentation

15.611.2.1 AngleReference

enum `QCPPolarAxisRadial::AngleReference`

Defines the reference of the angle at which a radial axis is tilted (setAngle).

Enumerator

arAbsolute	The axis tilt is given in absolute degrees. The zero is to the right and positive angles are measured counter-clockwise.
arAngularAxis	The axis tilt is measured in the angular coordinate system given by the parent angular axis.

15.611.2.2 ScaleType

enum [QCPPolarAxisRadial::ScaleType](#)

Defines the scale of an axis.

See also

[setScaleType](#)

Enumerator

stLinear	Linear scaling.
stLogarithmic	Logarithmic scaling with correspondingly transformed axis coordinates (possibly also setTicker to a QCPAxisTickerLog instance).

15.611.2.3 SelectablePart

enum [QCPPolarAxisRadial::SelectablePart](#)

Defines the selectable parts of an axis.

See also

[setSelectableParts](#), [setSelectedParts](#)

Enumerator

spNone	None of the selectable parts.
spAxis	The axis backbone and tick marks.
spTickLabels	Tick labels (numbers) of this axis (as a whole, not individually)
spAxisLabel	The axis label.

15.611.3 Constructor & Destructor Documentation

15.611.3.1 QCPPolarAxisRadial()

```
QCPPolarAxisRadial::QCPPolarAxisRadial (
    QCPPolarAxisAngular * parent ) [explicit]
```

Constructs an Axis instance of Type *type* for the axis rect *parent*.

Usually it isn't necessary to instantiate axes directly, because you can let [QCustomPlot](#) create them for you with [QCPAxisRect::addAxis](#). If you want to use own QCPAxis-subclasses however, create them manually and then inject them also via [QCPAxisRect::addAxis](#).

15.611.4 Member Function Documentation

15.611.4.1 applyDefaultAntialiasingHint()

```
void QCPPolarAxisRadial::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Implements [QCPLayerable](#).

15.611.4.2 coordToPixel()

```
QPointF QCPPolarAxisRadial::coordToPixel (
    double angleCoord,
    double radiusCoord ) const
```

Transforms *value*, in coordinates of the axis, to pixel coordinates of the [QCustomPlot](#) widget.

15.611.4.3 deselectEvent()

```
void QCPPolarAxisRadial::deselectEvent (
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.611.4.4 draw()

```
void QCPPolarAxisRadial::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPLayerable](#).

15.611.4.5 getPartAt()

```
QCPPolarAxisRadial::SelectablePart QCPPolarAxisRadial::getPartAt (
    const QPointF & pos ) const
```

Returns the part of the axis that is hit by *pos* (in pixels). The return value of this function is independent of the user-selectable parts defined with [setSelectableParts](#). Further, this function does not change the current selection state of the axis.

If the axis is not visible ([setVisible](#)), this function always returns [spNone](#).

See also

[setSelectedParts](#), [setSelectableParts](#), [QCustomPlot::setInteractions](#)

15.611.4.6 mouseMoveEvent()

```
void QCPPolarAxisRadial::mouseMoveEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user moves the mouse while holding a mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *startPos* indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction. The default implementation does nothing.

See also

[mousePressEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

15.611.4.7 mousePressEvent()

```
void QCPPolarAxisRadial::mousePressEvent (
    QMouseEvent * event,
    const QVariant & details ) [protected], [virtual]
```

This event gets called when the user presses a mouse button while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *details* contains layerable-specific details about the hit, which were generated in the previous call to [selectTest](#). For example, One-dimensional plottables like [QCPGraph](#) or [QCPBars](#) convey the clicked data point in the *details* parameter, as [QCPDataSelection](#) packed as `QVariant`. Multi-part objects convey the specific `SelectablePart` that was hit (e.g. [QCPAxis::SelectablePart](#) in the case of axes).

[QCustomPlot](#) uses an event propagation system that works the same as Qt's system. If your layerable doesn't reimplement the [mousePressEvent](#) or explicitly calls `event->ignore()` in its reimplementation, the event will be propagated to the next layerable in the stacking order.

Once a layerable has accepted the [mousePressEvent](#), it is considered the mouse grabber and will receive all following calls to [mouseMoveEvent](#) or [mouseReleaseEvent](#) for this mouse interaction (a "mouse interaction" in this context ends with the release).

The default implementation does nothing except explicitly ignoring the event with `event->ignore()`.

See also

[mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

15.611.4.8 mouseReleaseEvent()

```
void QCPPolarAxisRadial::mouseReleaseEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user releases the mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *startPos* indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction. The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

15.611.4.9 moveRange()

```
void QCPPolarAxisRadial::moveRange (
    double diff )
```

If the scale type ([setScaleType](#)) is [stLinear](#), *diff* is added to the lower and upper bounds of the range. The range is simply moved by *diff*.

If the scale type is [stLogarithmic](#), the range bounds are multiplied by *diff*. This corresponds to an apparent "linear" move in logarithmic scaling by a distance of $\log(\text{diff})$.

15.611.4.10 pixelToCoord()

```
void QCPPolarAxisRadial::pixelToCoord (
    QPointF pixelPos,
    double & angleCoord,
    double & radiusCoord ) const
```

Transforms *value*, in pixel coordinates of the [QCustomPlot](#) widget, to axis coordinates.

15.611.4.11 rangeChanged [1/2]

```
void QCPPolarAxisRadial::rangeChanged (
    const QCPRange & newRange ) [signal]
```

This signal is emitted when the range of this axis has changed. You can connect it to the [setRange](#) slot of another axis to communicate the new range to the other axis, in order for it to be synchronized.

You may also manipulate/correct the range with [setRange](#) in a slot connected to this signal. This is useful if for example a maximum range span shall not be exceeded, or if the lower/upper range shouldn't go beyond certain values (see [QCPRange::bounded](#)). For example, the following slot would limit the x axis to ranges between 0 and 10:

```
customPlot->xAxis->setRange(newRange.bounded(0, 10))
```

15.611.4.12 rangeChanged [2/2]

```
void QCPPolarAxisRadial::rangeChanged (
    const QCPRange & newRange,
    const QCPRange & oldRange ) [signal]
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Additionally to the new range, this signal also provides the previous range held by the axis as *oldRange*.

15.611.4.13 rescale()

```
void QCPPolarAxisRadial::rescale (
    bool onlyVisiblePlottables = false )
```

Changes the axis range such that all plottables associated with this axis are fully visible in that dimension.

See also

[QCPAbstractPlottable::rescaleAxes](#), [QCustomPlot::rescaleAxes](#)

15.611.4.14 scaleRange() [1/2]

```
void QCPPolarAxisRadial::scaleRange (
    double factor )
```

Scales the range of this axis by *factor* around the center of the current axis range. For example, if *factor* is 2.0, then the axis range will double its size, and the point at the axis range center won't have changed its position in the [QCustomPlot](#) widget (i.e. coordinates around the center will have moved symmetrically closer).

If you wish to scale around a different coordinate than the current axis range center, use the overload [scaleRange\(double factor, double center\)](#).

15.611.4.15 scaleRange() [2/2]

```
void QCPPolarAxisRadial::scaleRange (
    double factor,
    double center )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Scales the range of this axis by *factor* around the coordinate *center*. For example, if *factor* is 2.0, *center* is 1.0, then the axis range will double its size, and the point at coordinate 1.0 won't have changed its position in the [QCustomPlot](#) widget (i.e. coordinates around 1.0 will have moved symmetrically closer to 1.0).

See also

[scaleRange\(double factor\)](#)

15.611.4.16 scaleTypeChanged

```
void QCPPolarAxisRadial::scaleTypeChanged (
    QCPPolarAxisRadial::ScaleType scaleType ) [signal]
```

This signal is emitted when the scale type changes, by calls to [setScaleType](#)

15.611.4.17 selectableChanged

```
void QCPPolarAxisRadial::selectableChanged (
    const QCPPolarAxisRadial::SelectableParts & parts ) [signal]
```

This signal is emitted when the selectability changes, by calls to [setSelectableParts](#)

15.611.4.18 selectEvent()

```
void QCPPolarAxisRadial::selectEvent (
    QMouseEvent * event,
    bool additive,
    const QVariant & details,
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.611.4.19 selectionCategory()

```
QCP::Interaction QCPPolarAxisRadial::selectionCategory ( ) const [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.611.4.20 selectionChanged

```
void QCPPolarAxisRadial::selectionChanged (
    const QCPPolarAxisRadial::SelectableParts & parts ) [signal]
```

This signal is emitted when the selection state of this axis has changed, either by user interaction or by a direct call to [setSelectedParts](#).

15.611.4.21 selectTest()

```
double QCPPolarAxisRadial::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = 0 ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the *selectionTolerance* multiplied by 0.99). Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than $0.99 \cdot \text{selectionTolerance}$).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the *mouseReleaseEvent* occurs, and the finally selected object is notified via the *selectEvent/* *deselectEvent* methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to *selectEvent* when the parent [QCustomPlot](#) decides on the basis of this *selectTest* call, that the object was successfully selected. The subsequent call to *selectEvent* will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in [selectTest](#). The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent *selectEvent*, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

[selectEvent](#), [deselectEvent](#), [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Reimplemented from [QCPLayerable](#).

15.611.4.22 setBasePen()

```
void QCPPolarAxisRadial::setBasePen (
    const QPen & pen )
```

Sets the pen, the axis base line is drawn with.

See also

[setTickPen](#), [setSubTickPen](#)

15.611.4.23 setLabel()

```
void QCPPolarAxisRadial::setLabel (
    const QString & str )
```

Sets the text of the axis label that will be shown below/above or next to the axis, depending on its orientation. To disable axis labels, pass an empty string as *str*.

15.611.4.24 setLabelColor()

```
void QCPPolarAxisRadial::setLabelColor (
    const QColor & color )
```

Sets the color of the axis label.

See also

[setLabelFont](#)

15.611.4.25 setLabelFont()

```
void QCPPolarAxisRadial::setLabelFont (
    const QFont & font )
```

Sets the font of the axis label.

See also

[setLabelColor](#)

15.611.4.26 setLabelPadding()

```
void QCPPolarAxisRadial::setLabelPadding (
    int padding )
```

Sets the distance between the tick labels and the axis label.

See also

[setTickLabelPadding](#), [setPadding](#)

15.611.4.27 setNumberFormat()

```
void QCPPolarAxisRadial::setNumberFormat (
    const QString & formatCode )
```

Sets the number format for the numbers in tick labels. This *formatCode* is an extended version of the format code used e.g. by `QString::number()` and `QLocale::toString()`. For reference about that, see the "Argument Formats" section in the detailed description of the `QString` class.

formatCode is a string of one, two or three characters. The first character is identical to the normal format code used by Qt. In short, this means: 'e'/'E' scientific format, 'f' fixed format, 'g'/'G' scientific or fixed, whichever is shorter.

The second and third characters are optional and specific to [QCustomPlot](#):

If the first char was 'e' or 'g', numbers are/might be displayed in the scientific format, e.g. "5.5e9", which is ugly in a plot. So when the second char of *formatCode* is set to 'b' (for "beautiful"), those exponential numbers are formatted in a more natural way, i.e. "5.5 [multiplication sign] 10 [superscript] 9". By default, the multiplication sign is a centered dot. If instead a cross should be shown (as is usual in the USA), the third char of *formatCode* can be set to 'c'. The inserted multiplication signs are the UTF-8 characters 215 (0xD7) for the cross and 183 (0xB7) for the dot.

Examples for *formatCode*:

- g normal format code behaviour. If number is small, fixed format is used, if number is large, normal scientific format is used

- `gb` If number is small, fixed format is used, if number is large, scientific format is used with beautifully typeset decimal powers and a dot as multiplication sign
- `ebc` All numbers are in scientific format with beautifully typeset decimal power and a cross as multiplication sign
- `fb` illegal format code, since fixed format doesn't support (or need) beautifully typeset decimal powers. Format code will be reduced to 'f'.
- `hello` illegal format code, since first char is not 'e', 'E', 'f', 'g' or 'G'. Current format code will not be changed.

15.611.4.28 `setNumberPrecision()`

```
void QCPPolarAxisRadial::setNumberPrecision (
    int precision )
```

Sets the precision of the tick label numbers. See `QLocale::toString(double i, char f, int prec)` for details. The effect of precisions are most notably for number Formats starting with 'e', see [setNumberFormat](#)

15.611.4.29 `setRange()` [1/3]

```
void QCPPolarAxisRadial::setRange (
    const QCPRange & range )
```

Sets the range of the axis.

This slot may be connected with the [rangeChanged](#) signal of another axis so this axis is always synchronized with the other axis range, when it changes.

To invert the direction of an axis, use [setRangeReversed](#).

15.611.4.30 `setRange()` [2/3]

```
void QCPPolarAxisRadial::setRange (
    double lower,
    double upper )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the lower and upper bound of the axis range.

To invert the direction of an axis, use [setRangeReversed](#).

There is also a slot to set a range, see [setRange\(const QCPRange &range\)](#).

15.611.4.31 `setRange()` [3/3]

```
void QCPPolarAxisRadial::setRange (
    double position,
    double size,
    Qt::AlignmentFlag alignment )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the range of the axis.

The *position* coordinate indicates together with the *alignment* parameter, where the new range will be positioned. *size* defines the size of the new axis range. *alignment* may be `Qt::AlignLeft`, `Qt::AlignRight` or `Qt::AlignCenter`. This will cause the left border, right border, or center of the range to be aligned with *position*. Any other values of *alignment* will default to `Qt::AlignCenter`.

15.611.4.32 `setRangeLower()`

```
void QCPPolarAxisRadial::setRangeLower (
    double lower )
```

Sets the lower bound of the axis range. The upper bound is not changed.

See also

[setRange](#)

15.611.4.33 `setRangeReversed()`

```
void QCPPolarAxisRadial::setRangeReversed (
    bool reversed )
```

Sets whether the axis range (direction) is displayed reversed. Normally, the values on horizontal axes increase left to right, on vertical axes bottom to top. When *reversed* is set to true, the direction of increasing values is inverted. Note that the range and data interface stays the same for reversed axes, e.g. the *lower* part of the [setRange](#) interface will still reference the mathematically smaller number than the *upper* part.

15.611.4.34 `setRangeUpper()`

```
void QCPPolarAxisRadial::setRangeUpper (
    double upper )
```

Sets the upper bound of the axis range. The lower bound is not changed.

See also

[setRange](#)

15.611.4.35 `setScaleType()`

```
void QCPPolarAxisRadial::setScaleType (
    QCPPolarAxisRadial::ScaleType type )
```

Sets whether the axis uses a linear scale or a logarithmic scale.

Note that this method controls the coordinate transformation. For logarithmic scales, you will likely also want to use a logarithmic tick spacing and labeling, which can be achieved by setting the axis ticker to an instance of [QCPAxisTickerLog](#) :

See the documentation of [QCPAxisTickerLog](#) about the details of logarithmic axis tick creation.

[setNumberPrecision](#)

15.611.4.36 `setSelectableParts()`

```
void QCPPolarAxisRadial::setSelectableParts (
    const QCPPolarAxisRadial::SelectableParts & selectableParts )
```

Sets whether the user can (de-)select the parts in *selectable* by clicking on the [QCustomPlot](#) surface. (When [QCustomPlot::setInteractions](#) contains `iSelectAxes`.)

However, even when *selectable* is set to a value not allowing the selection of a specific part, it is still possible to set the selection of this part manually, by calling [setSelectedParts](#) directly.

See also

[SelectablePart](#), [setSelectedParts](#)

15.611.4.37 `setSelectedBasePen()`

```
void QCPPolarAxisRadial::setSelectedBasePen (
    const QPen & pen )
```

Sets the pen that is used to draw the axis base line when selected.

See also

[setBasePen](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.611.4.38 setSelectedLabelColor()

```
void QCPPolarAxisRadial::setSelectedLabelColor (
    const QColor & color )
```

Sets the color that is used for the axis label when it is selected.

See also

[setLabelColor](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.611.4.39 setSelectedLabelFont()

```
void QCPPolarAxisRadial::setSelectedLabelFont (
    const QFont & font )
```

Sets the font that is used for the axis label when it is selected.

See also

[setLabelFont](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.611.4.40 setSelectedParts()

```
void QCPPolarAxisRadial::setSelectedParts (
    const QCPPolarAxisRadial::SelectableParts & selectedParts )
```

Sets the selected state of the respective axis parts described by [SelectablePart](#). When a part is selected, it uses a different pen/font.

The entire selection mechanism for axes is handled automatically when [QCustomPlot::setInteractions](#) contains [iSelectAxes](#). You only need to call this function when you wish to change the selection state manually.

This function can change the selection state of a part, independent of the [setSelectableParts](#) setting. emits the [selectionChanged](#) signal when *selected* is different from the previous selection state.

See also

[SelectablePart](#), [setSelectableParts](#), [selectTest](#), [setSelectedBasePen](#), [setSelectedTickPen](#), [setSelectedSubTickPen](#), [setSelectedTickLabelFont](#), [setSelectedLabelFont](#), [setSelectedTickLabelColor](#), [setSelectedLabelColor](#)

15.611.4.41 setSelectedSubTickPen()

```
void QCPPolarAxisRadial::setSelectedSubTickPen (
    const QPen & pen )
```

Sets the pen that is used to draw the subticks when selected.

See also

[setSubTickPen](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.611.4.42 setSelectedTickLabelColor()

```
void QCPPolarAxisRadial::setSelectedTickLabelColor (
    const QColor & color )
```

Sets the color that is used for tick labels when they are selected.

See also

[setTickLabelColor](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.611.4.43 setSelectedTickLabelFont()

```
void QCPPolarAxisRadial::setSelectedTickLabelFont (
    const QFont & font )
```

Sets the font that is used for tick labels when they are selected.

See also

[setTickLabelFont](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.611.4.44 setSelectedTickPen()

```
void QCPPolarAxisRadial::setSelectedTickPen (
    const QPen & pen )
```

Sets the pen that is used to draw the (major) ticks when selected.

See also

[setTickPen](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

15.611.4.45 setSubTickLength()

```
void QCPPolarAxisRadial::setSubTickLength (
    int inside,
    int outside = 0 )
```

Sets the length of the subticks in pixels. *inside* is the length the subticks will reach inside the plot and *outside* is the length they will reach outside the plot. If *outside* is greater than zero, the tick labels and axis label will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setTickLength](#), [setSubTickLengthIn](#), [setSubTickLengthOut](#)

15.611.4.46 setSubTickLengthIn()

```
void QCPPolarAxisRadial::setSubTickLengthIn (
    int inside )
```

Sets the length of the inward subticks in pixels. *inside* is the length the subticks will reach inside the plot.

See also

[setSubTickLengthOut](#), [setSubTickLength](#), [setTickLength](#)

15.611.4.47 setSubTickLengthOut()

```
void QCPPolarAxisRadial::setSubTickLengthOut (
    int outside )
```

Sets the length of the outward subticks in pixels. *outside* is the length the subticks will reach outside the plot. If *outside* is greater than zero, the tick labels will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setSubTickLengthIn](#), [setSubTickLength](#), [setTickLength](#)

15.611.4.48 setSubTickPen()

```
void QCPPolarAxisRadial::setSubTickPen (
    const QPen & pen )
```

Sets the pen, subtick marks will be drawn with.

See also

[setSubTickCount](#), [setSubTickLength](#), [setBasePen](#)

15.611.4.49 setSubTicks()

```
void QCPPolarAxisRadial::setSubTicks (
    bool show )
```

Sets whether sub tick marks are displayed.

Sub ticks are only potentially visible if (major) ticks are also visible (see [setTicks](#))

See also

[setTicks](#)

15.611.4.50 setTicker()

```
void QCPPolarAxisRadial::setTicker (
    QSharedPointer< QCPAxisTicker > ticker )
```

The axis ticker is responsible for generating the tick positions and tick labels. See the documentation of [QCPAxisTicker](#) for details on how to work with axis tickers.

You can change the tick positioning/labeling behaviour of this axis by setting a different [QCPAxisTicker](#) subclass using this method. If you only wish to modify the currently installed axis ticker, access it via [ticker](#).

Since the ticker is stored in the axis as a shared pointer, multiple axes may share the same axis ticker simply by passing the same shared pointer to multiple axes.

See also

[ticker](#)

15.611.4.51 setTickLabelColor()

```
void QCPPolarAxisRadial::setTickLabelColor (
    const QColor & color )
```

Sets the color of the tick labels.

See also

[setTickLabels](#), [setTickLabelFont](#)

15.611.4.52 setTickLabelFont()

```
void QCPPolarAxisRadial::setTickLabelFont (
    const QFont & font )
```

Sets the font of the tick labels.

See also

[setTickLabels](#), [setTickLabelColor](#)

15.611.4.53 setTickLabelPadding()

```
void QCPPolarAxisRadial::setTickLabelPadding (
    int padding )
```

Sets the distance between the axis base line (including any outward ticks) and the tick labels.

See also

[setLabelPadding](#), [setPadding](#)

15.611.4.54 setTickLabelRotation()

```
void QCPPolarAxisRadial::setTickLabelRotation (
    double degrees )
```

Sets the rotation of the tick labels. If *degrees* is zero, the labels are drawn normally. Else, the tick labels are drawn rotated by *degrees* clockwise. The specified angle is bound to values from -90 to 90 degrees.

If *degrees* is exactly -90, 0 or 90, the tick labels are centered on the tick coordinate. For other angles, the label is drawn with an offset such that it seems to point toward or away from the tick mark.

15.611.4.55 setTickLabels()

```
void QCPPolarAxisRadial::setTickLabels (
    bool show )
```

Sets whether tick labels are displayed. Tick labels are the numbers drawn next to tick marks.

15.611.4.56 setTickLength()

```
void QCPPolarAxisRadial::setTickLength (
    int inside,
    int outside = 0 )
```

Sets the length of the ticks in pixels. *inside* is the length the ticks will reach inside the plot and *outside* is the length they will reach outside the plot. If *outside* is greater than zero, the tick labels and axis label will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setSubTickLength](#), [setTickLengthIn](#), [setTickLengthOut](#)

15.611.4.57 setTickLengthIn()

```
void QCPPolarAxisRadial::setTickLengthIn (
    int inside )
```

Sets the length of the inward ticks in pixels. *inside* is the length the ticks will reach inside the plot.

See also

[setTickLengthOut](#), [setTickLength](#), [setSubTickLength](#)

15.611.4.58 setTickLengthOut()

```
void QCPPolarAxisRadial::setTickLengthOut (
    int outside )
```

Sets the length of the outward ticks in pixels. *outside* is the length the ticks will reach outside the plot. If *outside* is greater than zero, the tick labels and axis label will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setTickLengthIn](#), [setTickLength](#), [setSubTickLength](#)

15.611.4.59 setTickPen()

```
void QCPPolarAxisRadial::setTickPen (
    const QPen & pen )
```

Sets the pen, tick marks will be drawn with.

See also

[setTickLength](#), [setBasePen](#)

15.611.4.60 setTicks()

```
void QCPPolarAxisRadial::setTicks (
    bool show )
```

Sets whether tick marks are displayed.

Note that setting *show* to false does not imply that tick labels are invisible, too. To achieve that, see [setTickLabels](#).

See also

[setSubTicks](#)

15.611.4.61 ticker()

```
QSharedPointer< QCPAxisTicker > QCPPolarAxisRadial::ticker ( ) const [inline]
```

Returns a modifiable shared pointer to the currently installed axis ticker. The axis ticker is responsible for generating the tick positions and tick labels of this axis. You can access the [QCPAxisTicker](#) with this method and modify basic properties such as the approximate tick count ([QCPAxisTicker::setTickCount](#)).

You can gain more control over the axis ticks by setting a different [QCPAxisTicker](#) subclass, see the documentation there. A new axis ticker can be set with [setTicker](#).

Since the ticker is stored in the axis as a shared pointer, multiple axes may share the same axis ticker simply by passing the same shared pointer to multiple axes.

See also

[setTicker](#)

15.611.4.62 wheelEvent()

```
void QCPPolarAxisRadial::wheelEvent (
    QWheelEvent * event ) [protected], [virtual]
```

This event gets called when the user turns the mouse scroll wheel while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`.

The `event->angleDelta()` indicates how far the mouse wheel was turned, which is usually +/- 120 for single rotation steps. However, if the mouse wheel is turned rapidly, multiple steps may accumulate to one event, making the delta larger. On the other hand, if the wheel has very smooth steps or none at all, the delta may be smaller.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#)

Reimplemented from [QCPLayerable](#).

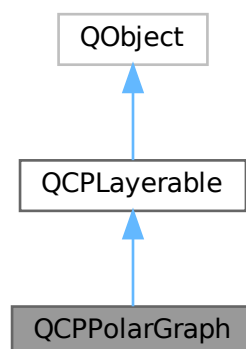
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.612 QCPPolarGraph Class Reference

A radial graph used to display data in polar plots.

Inheritance diagram for QCPPolarGraph:



- void [setAntialiasedScatters](#) (bool enabled)
- void [setPen](#) (const QPen &pen)
- void [setBrush](#) (const QBrush &brush)
- void [setPeriodic](#) (bool enabled)
- void [setKeyAxis](#) (QCPPolarAxisAngular *axis)
- void [setValueAxis](#) (QCPPolarAxisRadial *axis)
- Q_SLOT void [setSelectable](#) (QCP::Selectable selectable)
- Q_SLOT void [setSelection](#) (QCPDataSelection selection)
- void [setData](#) (QSharedPointer< QCPGraphDataContainer > data)
- void [setData](#) (const QVector< double > &keys, const QVector< double > &values, bool alreadySorted=false)
- void [setLineStyle](#) (LineStyle ls)
- void [setScatterStyle](#) (const QCPScatterStyle &style)
- void [addData](#) (const QVector< double > &keys, const QVector< double > &values, bool alreadySorted=false)
- void [addData](#) (double key, double value)
- void [coordsToPixels](#) (double key, double value, double &x, double &y) const
- const QPointF [coordsToPixels](#) (double key, double value) const
- void [pixelsToCoords](#) (double x, double y, double &key, double &value) const
- void [pixelsToCoords](#) (const QPointF &pixelPos, double &key, double &value) const
- void [rescaleAxes](#) (bool onlyEnlarge=false) const
- void [rescaleKeyAxis](#) (bool onlyEnlarge=false) const
- void [rescaleValueAxis](#) (bool onlyEnlarge=false, bool inKeyRange=false) const
- bool [addToLegend](#) (QCPLegend *legend)
- bool [addToLegend](#) ()
- bool [removeFromLegend](#) (QCPLegend *legend) const
- bool [removeFromLegend](#) () const
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=0) const
- virtual [QCPPloitableInterface1D](#) * [interface1D](#) ()
- virtual [QCPRange](#) [getKeyRange](#) (bool &foundRange, QCP::SignDomain inSignDomain=QCP::sdBoth) const
- virtual [QCPRange](#) [getValueRange](#) (bool &foundRange, QCP::SignDomain inSignDomain=QCP::sdBoth, const [QCPRange](#) &inKeyRange=[QCPRange](#)()) const

Public Member Functions inherited from QCPLayerable

- [QCPLayerable](#) (QCustomPlot *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool [visible](#) () const
- [QCustomPlot](#) * [parentPlot](#) () const
- [QCPLayerable](#) * [parentLayerable](#) () const
- [QCPLayer](#) * [layer](#) () const
- bool [antialiased](#) () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual QRect [clipRect](#) () const
- virtual void [draw](#) ([QCPPainter](#) *painter)
- virtual [QCP::Interaction](#) [selectionCategory](#) () const
- void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged)
- virtual void [deselectEvent](#) (bool *selectionStateChanged)

- virtual void **drawLinePlot** ([QCPPainter](#) *painter, const QVector< QPointF > &lines) const
- virtual void **drawFill** ([QCPPainter](#) *painter, QVector< QPointF > *lines) const
- virtual void **drawScatterPlot** ([QCPPainter](#) *painter, const QVector< QPointF > &scatters, const [QCPScatterStyle](#) &style) const
- virtual void **drawLegendIcon** ([QCPPainter](#) *painter, const QRectF &rect) const
- void **applyFillAntialiasingHint** ([QCPPainter](#) *painter) const
- void **applyScattersAntialiasingHint** ([QCPPainter](#) *painter) const
- double **pointDistance** (const QPointF &pixelPoint, QCPGraphDataContainer::const_iterator &closestData) const
- virtual int **dataCount** () const
- void **getDataSegments** (QList< [QCPDataRange](#) > &selectedSegments, QList< [QCPDataRange](#) > &unselectedSegments) const
- void **drawPolyline** ([QCPPainter](#) *painter, const QVector< QPointF > &lineData) const
- void **getVisibleDataBounds** (QCPGraphDataContainer::const_iterator &begin, QCPGraphDataContainer::const_iterator &end, const [QCPDataRange](#) &rangeRestriction) const
- void **getLines** (QVector< QPointF > *lines, const [QCPDataRange](#) &dataRange) const
- void **getScatters** (QVector< QPointF > *scatters, const [QCPDataRange](#) &dataRange) const
- void **getOptimizedLineData** (QVector< [QCPGraphData](#) > *lineData, const QCPGraphDataContainer::const_iterator &begin, const QCPGraphDataContainer::const_iterator &end) const
- void **getOptimizedScatterData** (QVector< [QCPGraphData](#) > *scatterData, QCPGraphDataContainer::const_iterator begin, QCPGraphDataContainer::const_iterator end) const
- QVector< QPointF > **dataToLines** (const QVector< [QCPGraphData](#) > &data) const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QSharedPointer< [QCPGraphDataContainer](#) > **mDataContainer**
- [LineStyle](#) **mLineStyle**
- [QCPScatterStyle](#) **mScatterStyle**
- QString **mName**
- bool **mAntialiasedFill**
- bool **mAntialiasedScatters**
- QPen **mPen**
- QBrush **mBrush**
- bool **mPeriodic**
- QPointer< [QCPPolarAxisAngular](#) > **mKeyAxis**
- QPointer< [QCPPolarAxisRadial](#) > **mValueAxis**
- [QCP::SelectionType](#) **mSelectable**
- [QCPDataSelection](#) **mSelection**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCPPolarLegendItem](#)

15.612.1 Detailed Description

A radial graph used to display data in polar plots.

Warning

In this [QCustomPlot](#) version, polar plots are a tech preview. Expect documentation and functionality to be incomplete, as well as changing public interfaces in the future.

15.612.2 Member Enumeration Documentation

15.612.2.1 LineStyle

enum [QCPPolarGraph::LineStyle](#)

Defines how the graph's line is represented visually in the plot. The line is drawn with the current pen of the graph ([setPen](#)).

See also

[setLineStyle](#)

Enumerator

IsNone	with symbols according to the scatter style, see setScatterStyle) data points are not connected with any lines (e.g. data only represented)
IsLine	data points are connected by a straight line

15.612.3 Constructor & Destructor Documentation

15.612.3.1 QCPPolarGraph()

```
QCPPolarGraph::QCPPolarGraph (
    QCPPolarAxisAngular * keyAxis,
    QCPPolarAxisRadial * valueAxis )
```

Constructs a graph which uses *keyAxis* as its angular and *valueAxis* as its radial axis. *keyAxis* and *valueAxis* must reside in the same [QCustomPlot](#), and the radial axis must be associated with the angular axis. If either of these restrictions is violated, a corresponding message is printed to the debug output (qDebug), the construction is not aborted, though.

The created [QCPPolarGraph](#) is automatically registered with the [QCustomPlot](#) instance inferred from *keyAxis*. This [QCustomPlot](#) instance takes ownership of the [QCPPolarGraph](#), so do not delete it manually but use [QCPPolarAxisAngular::removeGraph\(\)](#) instead.

To directly create a [QCPPolarGraph](#) inside a plot, you should use the [QCPPolarAxisAngular::addGraph](#) method.

15.612.4 Member Function Documentation

15.612.4.1 applyDefaultAntialiasingHint()

```
void QCPPolarGraph::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Implements [QCPLayerable](#).

15.612.4.2 clipRect()

```
QRect QCPPolarGraph::clipRect ( ) const [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.612.4.3 coordsToPixels()

```
void QCPPolarGraph::coordsToPixels (
    double key,
    double value,
    double & x,
    double & y ) const
```

Use this method to set an own [QCPSelectionDecorator](#) (subclass) instance. This allows you to customize the visual representation of selected data ranges further than by using the default [QCPSelectionDecorator](#).

The plottable takes ownership of the *decorator*.

The currently set decorator can be accessed via `selectionDecorator`.

15.612.4.4 deselectEvent()

```
void QCPPolarGraph::deselectEvent (
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.612.4.5 draw()

```
void QCPPolarGraph::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPLayerable](#).

15.612.4.6 selectEvent()

```
void QCPPolarGraph::selectEvent (
    QMouseEvent * event,
    bool additive,
    const QVariant & details,
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.612.4.7 selectionCategory()

```
QCP::Interaction QCPPolarGraph::selectionCategory ( ) const [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.612.4.8 selectTest()

```
double QCPPolarGraph::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = 0 ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99). Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than 0.99*selectionTolerance).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the `mouseReleaseEvent` occurs, and the finally selected object is notified via the `selectEvent/` `deselectEvent` methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to `selectEvent` when the parent [QCustomPlot](#) decides on the basis of this `selectTest` call, that the object was successfully selected. The subsequent call to `selectEvent` will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in `selectTest`. The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent `selectEvent`, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

`selectEvent`, `deselectEvent`, [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Reimplemented from [QCPLayerable](#).

15.612.4.9 setAntialiasedFill()

```
void QCPPolarGraph::setAntialiasedFill (
    bool enabled )
```

Sets whether fills of this plottable are drawn antialiased or not.

Note that this setting may be overridden by [QCustomPlot::setAntialiasedElements](#) and [QCustomPlot::setNotAntialiasedElements](#).

15.612.4.10 setAntialiasedScatters()

```
void QCPPolarGraph::setAntialiasedScatters (
    bool enabled )
```

Sets whether the scatter symbols of this plottable are drawn antialiased or not.

Note that this setting may be overridden by [QCustomPlot::setAntialiasedElements](#) and [QCustomPlot::setNotAntialiasedElements](#).

15.612.4.11 setBrush()

```
void QCPPolarGraph::setBrush (
    const QBrush & brush )
```

The brush is used to draw basic fills of the plottable representation in the plot. The Fill can be a color, gradient or texture, see the usage of `QBrush`.

For example, the [QCPGraph](#) subclass draws the fill under the graph with this brush, when it's not set to `Qt::NoBrush`.

See also

[setPen](#)

15.612.4.12 setData() [1/2]

```
void QCPPolarGraph::setData (
    const QVector< double > & keys,
```

```
const QVector< double > & values,
bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data with the provided points in *keys* and *values*. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *keys* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

See also

[addData](#)

15.612.4.13 setData() [2/2]

```
void QCPPolarGraph::setData (
    QSharedPointer< QCPGraphDataContainer > data )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data container with the provided *data* container.

Since a `QSharedPointer` is used, multiple `QCPPolarGraphs` may share the same data container safely. Modifying the data in the container will then affect all graphs that share the container. Sharing can be achieved by simply exchanging the data containers wrapped in shared pointers:

If you do not wish to share containers, but create a copy from an existing container, rather use the `QCPDataContainer<DataType>::set` method on the graph's data container directly:

See also

[addData](#)

15.612.4.14 setKeyAxis()

```
void QCPPolarGraph::setKeyAxis (
    QCPPolarAxisAngular * axis )
```

The key axis of a plottable can be set to any axis of a `QCustomPlot`, as long as it is orthogonal to the plottable's value axis. This function performs no checks to make sure this is the case. The typical mathematical choice is to use the x-axis (`QCustomPlot::xAxis`) as key axis and the y-axis (`QCustomPlot::yAxis`) as value axis.

Normally, the key and value axes are set in the constructor of the plottable (or `QCustomPlot::addGraph` when working with `QCPGraphs` through the dedicated graph interface).

See also

[setValueAxis](#)

15.612.4.15 setLineStyle()

```
void QCPPolarGraph::setLineStyle (
    LineStyle ls )
```

Sets how the single data points are connected in the plot. For scatter-only plots, set *ls* to `lsNone` and [setScatterStyle](#) to the desired scatter style.

See also

[setScatterStyle](#)

15.612.4.16 setName()

```
void QCPPolarGraph::setName (
    const QString & name )
```

The name is the textual representation of this plottable as it is displayed in the legend (`QCPLegend`). It may contain any UTF-8 characters, including newlines.

15.612.4.17 setPen()

```
void QCPPolarGraph::setPen (
    const QPen & pen )
```

The pen is used to draw basic lines that make up the plottable representation in the plot. For example, the [QCPGraph](#) subclass draws its graph lines with this pen.

See also

[setBrush](#)

15.612.4.18 setScatterStyle()

```
void QCPPolarGraph::setScatterStyle (
    const QCPScatterStyle & style )
```

Sets the visual appearance of single data points in the plot. If set to [QCPScatterStyle::ssNone](#), no scatter points are drawn (e.g. for line-only-plots with appropriate line style).

See also

[QCPScatterStyle](#), [setLineStyle](#)

15.612.4.19 setSelectable()

```
void QCPPolarGraph::setSelectable (
    QCP::SelectionType selectable )
```

Sets whether and to which granularity this plottable can be selected.

A selection can happen by clicking on the [QCustomPlot](#) surface (When [QCustomPlot::setInteractions](#) contains [QCP::iSelectPlottables](#)), by dragging a selection rect (When [QCustomPlot::setSelectionRectMode](#) is [QCP::srmSelect](#)), or programmatically by calling [setSelection](#).

See also

[setSelection](#), [QCP::SelectionType](#)

15.612.4.20 setSelection()

```
void QCPPolarGraph::setSelection (
    QCPDataSelection selection )
```

Sets which data ranges of this plottable are selected. Selected data ranges are drawn differently (e.g. color) in the plot. This can be controlled via the selection decorator (see [selectionDecorator](#)).

The entire selection mechanism for plottables is handled automatically when [QCustomPlot::setInteractions](#) contains [iSelectPlottables](#). You only need to call this function when you wish to change the selection state programmatically. Using [setSelectable](#) you can further specify for each plottable whether and to which granularity it is selectable. If *selection* is not compatible with the current [QCP::SelectionType](#) set via [setSelectable](#), the resulting selection will be adjusted accordingly (see [QCPDataSelection::enforceType](#)).

emits the `selectionChanged` signal when *selected* is different from the previous selection state.

See also

[setSelectable](#), [selectTest](#)

15.612.4.21 setValueAxis()

```
void QCPPolarGraph::setValueAxis (
    QCPPolarAxisRadial * axis )
```

The value axis of a plottable can be set to any axis of a [QCustomPlot](#), as long as it is orthogonal to the plottable's key axis. This function performs no checks to make sure this is the case. The typical mathematical choice is to use the x-axis ([QCustomPlot::xAxis](#)) as key axis and the y-axis ([QCustomPlot::yAxis](#)) as value axis.

Normally, the key and value axes are set in the constructor of the plottable (or [QCustomPlot::addGraph](#) when working with QCPGraphs through the dedicated graph interface).

- QPen **angularPen** () const
- QPen **angularSubGridPen** () const
- QPen **radialPen** () const
- QPen **radialSubGridPen** () const
- QPen **radialZeroLinePen** () const
- void **setRadialAxis** (QCPPolarAxisRadial *axis)
- void **setType** (GridTypes type)
- void **setSubGridType** (GridTypes type)
- void **setAntialiasedSubGrid** (bool enabled)
- void **setAntialiasedZeroLine** (bool enabled)
- void **setAngularPen** (const QPen &pen)
- void **setAngularSubGridPen** (const QPen &pen)
- void **setRadialPen** (const QPen &pen)
- void **setRadialSubGridPen** (const QPen &pen)
- void **setRadialZeroLinePen** (const QPen &pen)

Public Member Functions inherited from QCPLayerable

- QCPLayerable (QCustomPlot *plot, QString targetLayer=QString(), QCPLayerable *parentLayerable=nullptr)
- bool **visible** () const
- QCustomPlot * **parentPlot** () const
- QCPLayerable * **parentLayerable** () const
- QCPLayer * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** (QCPLayer *layer)
- bool **setLayer** (const QString &layerName)
- void **setAntialiased** (bool enabled)
- virtual double **selectTest** (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const
- bool **realVisibility** () const

Protected Member Functions

- virtual void **applyDefaultAntialiasingHint** (QCPPainter *painter) const Q_DECL_OVERRIDE
- virtual void **draw** (QCPPainter *painter) Q_DECL_OVERRIDE
- void **drawRadialGrid** (QCPPainter *painter, const QPointF ¢er, const QVector< double > &coords, const QPen &pen, const QPen &zeroPen=Qt::NoPen)
- void **drawAngularGrid** (QCPPainter *painter, const QPointF ¢er, double radius, const QVector< QPointF > &ticksCosSin, const QPen &pen)

Protected Member Functions inherited from QCPLayerable

- virtual void **parentPlotInitialized** (QCustomPlot *parentPlot)
- virtual QCP::Interaction **selectionCategory** () const
- virtual QRect **clipRect** () const
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** (QCustomPlot *parentPlot)
- void **setParentLayerable** (QCPLayerable *parentLayerable)
- bool **moveToLayer** (QCPLayer *layer, bool prepend)
- void **applyAntialiasingHint** (QCPPainter *painter, bool localAntialiased, QCP::AntialiasedElement overrideElement) const

Protected Attributes

- GridTypes **mType**
- GridTypes **mSubGridType**
- bool **mAntialiasedSubGrid**
- bool **mAntialiasedZeroLine**
- QPen **mAngularPen**
- QPen **mAngularSubGridPen**
- QPen **mRadialPen**
- QPen **mRadialSubGridPen**
- QPen **mRadialZeroLinePen**
- [QCPPolarAxisAngular](#) * **mParentAxis**
- QPointer< [QCPPolarAxisRadial](#) > **mRadialAxis**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members**Signals inherited from [QCPLayerable](#)**

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

15.613.1 Detailed Description

The grid in both angular and radial dimensions for polar plots.

Warning

In this [QCustomPlot](#) version, polar plots are a tech preview. Expect documentation and functionality to be incomplete, as well as changing public interfaces in the future.

15.613.2 Member Enumeration Documentation**15.613.2.1 GridType**

```
enum QCPPolarGrid::GridType
TODO
```

15.613.3 Constructor & Destructor Documentation**15.613.3.1 QCPPolarGrid()**

```
QCPPolarGrid::QCPPolarGrid (
    QCPPolarAxisAngular * parentAxis ) [explicit]
```

Creates a [QCPPolarGrid](#) instance and sets default values.

You shouldn't instantiate grids on their own, since every axis brings its own grid.

15.613.4 Member Function Documentation**15.613.4.1 applyDefaultAntialiasingHint()**

```
void QCPPolarGrid::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Implements [QCPLayerable](#).

15.613.4.2 draw()

```
void QCPPolarGrid::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPLayerable](#).

15.613.4.3 setAngularPen()

```
void QCPPolarGrid::setAngularPen (
    const QPen & pen )
```

Sets the pen with which (major) grid lines are drawn.

15.613.4.4 setAngularSubGridPen()

```
void QCPPolarGrid::setAngularSubGridPen (
    const QPen & pen )
```

Sets the pen with which sub grid lines are drawn.

15.613.4.5 setAntialiasedSubGrid()

```
void QCPPolarGrid::setAntialiasedSubGrid (
    bool enabled )
```

Sets whether sub grid lines are drawn antialiased.

15.613.4.6 setAntialiasedZeroLine()

```
void QCPPolarGrid::setAntialiasedZeroLine (
    bool enabled )
```

Sets whether zero lines are drawn antialiased.

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.614 QCPPolarLegendItem Class Reference

A legend item for polar plots.

- [QCPPolarGraph](#) * **polarGraph** ()

Public Member Functions inherited from [QCPAbstractLegendItem](#)

- [QCPAbstractLegendItem](#) ([QCPLegend](#) *parent)
- [QCPLegend](#) * **parentLegend** () const
- QFont **font** () const
- QColor **textColor** () const
- QFont **selectedFont** () const
- QColor **selectedTextColor** () const
- bool **selectable** () const
- bool **selected** () const
- void **setFont** (const QFont &font)
- void **setTextColor** (const QColor &color)
- void **setSelectedFont** (const QFont &font)
- void **setSelectedTextColor** (const QColor &color)
- Q_SLOT void **setSelectable** (bool selectable)
- Q_SLOT void **setSelected** (bool selected)
- virtual double **selectTest** (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_
DECL_OVERRIDE

Public Member Functions inherited from [QCPLayoutElement](#)

- [QCPLayoutElement](#) ([QCustomPlot](#) *parentPlot=nullptr)
- [QCPLayout](#) * **layout** () const
- QRect **rect** () const
- QRect **outerRect** () const
- QMargins **margins** () const
- QMargins **minimumMargins** () const
- QCP::MarginSides **autoMargins** () const
- QSize **minimumSize** () const
- QSize **maximumSize** () const
- [SizeConstraintRect](#) **sizeConstraintRect** () const
- [QCPMarginGroup](#) * **marginGroup** ([QCP::MarginSide](#) side) const
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **marginGroups** () const
- void **setOuterRect** (const QRect &rect)
- void **setMargins** (const QMargins &margins)
- void **setMinimumMargins** (const QMargins &margins)
- void **setAutoMargins** ([QCP::MarginSides](#) sides)
- void **setMinimumSize** (const QSize &size)
- void **setMinimumSize** (int width, int height)
- void **setMaximumSize** (const QSize &size)
- void **setMaximumSize** (int width, int height)
- void **setSizeConstraintRect** ([SizeConstraintRect](#) constraintRect)
- void **setMarginGroup** ([QCP::MarginSides](#) sides, [QCPMarginGroup](#) *group)
- virtual void **update** ([UpdatePhase](#) phase)
- virtual QSize **maximumOuterSizeHint** () const
- virtual QList< [QCPLayoutElement](#) * > **elements** (bool recursive) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const [QString](#) &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Protected Member Functions

- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual [QSize](#) **minimumOuterSizeHint** () const Q_DECL_OVERRIDE
- [QPen](#) **getIconBorderPen** () const
- [QColor](#) **getTextColor** () const
- [QFont](#) **getFont** () const

Protected Member Functions inherited from [QCPAbstractLegendItem](#)

- virtual [QCP::Interaction](#) **selectionCategory** () const Q_DECL_OVERRIDE
- virtual void **applyDefaultAntialiasingHint** ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual [QRect](#) **clipRect** () const Q_DECL_OVERRIDE
- virtual void **selectEvent** ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void **deselectEvent** (bool *selectionStateChanged) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayoutElement](#)

- virtual int **calculateAutoMargin** ([QCP::MarginSide](#) side)
- virtual void **layoutChanged** ()
- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **mousePressEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **mouseMoveEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseReleaseEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseDoubleClickEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **wheelEvent** ([QWheelEvent](#) *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QCPPolarGraph](#) * **mPolarGraph**

Protected Attributes inherited from [QCPAbstractLegendItem](#)

- [QCPLegend](#) * **mParentLegend**
- QFont **mFont**
- QColor **mTextColor**
- QFont **mSelectedFont**
- QColor **mSelectedTextColor**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from [QCPLayoutElement](#)

- [QCPLayout](#) * **mParentLayout**
- QSize **mMinimumSize**
- QSize **mMaximumSize**
- [SizeConstraintRect](#) **mSizeConstraintRect**
- QRect **mRect**
- QRect **mOuterRect**
- QMargins **mMargins**
- QMargins **mMinimumMargins**
- QCP::MarginSides **mAutoMargins**
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **mMarginGroups**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members**Public Types inherited from [QCPLayoutElement](#)**

- enum [UpdatePhase](#) { [upPreparation](#) , [upMargins](#) , [upLayout](#) }
- enum [SizeConstraintRect](#) { [scrInnerRect](#) , [scrOuterRect](#) }

Signals inherited from [QCPAbstractLegendItem](#)

- void [selectionChanged](#) (bool selected)
- void [selectableChanged](#) (bool selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

15.614.1 Detailed Description

A legend item for polar plots.

Warning

In this [QCustomPlot](#) version, polar plots are a tech preview. Expect documentation and functionality to be incomplete, as well as changing public interfaces in the future.

15.614.2 Member Function Documentation

15.614.2.1 draw()

```
void QCPPolarLegendItem::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractLegendItem](#).

15.614.2.2 minimumOuterSizeHint()

```
QSize QCPPolarLegendItem::minimumOuterSizeHint ( ) const [protected], [virtual]
```

Returns the suggested minimum size this layout element (the [outerRect](#)) may be compressed to, if no manual minimum size is set.

if a minimum size ([setMinimumSize](#)) was not set manually, parent layouts use the returned size (usually indirectly through [QCPLayout::getFinalMinimumOuterSize](#)) to determine the minimum allowed size of this layout element.

A manual minimum size is considered set if it is non-zero.

The default implementation simply returns the sum of the horizontal margins for the width and the sum of the vertical margins for the height. Reimplementations may use their detailed knowledge about the layout element's content to provide size hints.

Reimplemented from [QCPLayoutElement](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.615 QCPRange Class Reference

Represents the range an axis is encompassing.

Public Member Functions

- [QCPRange](#) ()
- [QCPRange](#) (double lower, double upper)
- bool **operator==** (const [QCPRange](#) &other) const
- bool **operator!=** (const [QCPRange](#) &other) const
- [QCPRange](#) & **operator+=** (const double &value)
- [QCPRange](#) & **operator-=** (const double &value)
- [QCPRange](#) & **operator*=** (const double &value)
- [QCPRange](#) & **operator/=** (const double &value)
- double [size](#) () const
- double [center](#) () const
- void [normalize](#) ()
- void [expand](#) (const [QCPRange](#) &otherRange)
- void [expand](#) (double includeCoord)
- [QCPRange](#) [expanded](#) (const [QCPRange](#) &otherRange) const
- [QCPRange](#) [expanded](#) (double includeCoord) const
- [QCPRange](#) [bounded](#) (double lowerBound, double upperBound) const
- [QCPRange](#) [sanitizedForLogScale](#) () const
- [QCPRange](#) [sanitizedForLinScale](#) () const
- bool [contains](#) (double value) const

Static Public Member Functions

- static bool [validRange](#) (double lower, double upper)
- static bool [validRange](#) (const [QCPRange](#) &range)

Public Attributes

- double **lower**
- double **upper**

Static Public Attributes

- static const double [minRange](#) = 1e-280
- static const double [maxRange](#) = 1e250

Friends

- const [QCPRange operator+](#) (const [QCPRange](#) &, double)
- const [QCPRange operator+](#) (double, const [QCPRange](#) &)
- const [QCPRange operator-](#) (const [QCPRange](#) &range, double value)
- const [QCPRange operator*](#) (const [QCPRange](#) &range, double value)
- const [QCPRange operator*](#) (double value, const [QCPRange](#) &range)
- const [QCPRange operator/](#) (const [QCPRange](#) &range, double value)

Related Symbols

(Note that these are not member symbols.)

- QDebug [operator<<](#) (QDebug d, const [QCPRange](#) &range)

15.615.1 Detailed Description

Represents the range an axis is encompassing.

contains a *lower* and *upper* double value and provides convenience input, output and modification functions.

See also

[QCPCAxis::setRange](#)

15.615.2 Constructor & Destructor Documentation

15.615.2.1 QCPRange() [1/2]

```
QCPRange::QCPRange ( )
```

Constructs a range with *lower* and *upper* set to zero.

15.615.2.2 QCPRange() [2/2]

```
QCPRange::QCPRange (
    double lower,
    double upper )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Constructs a range with the specified *lower* and *upper* values.

The resulting range will be normalized (see [normalize](#)), so if *lower* is not numerically smaller than *upper*, they will be swapped.

15.615.3 Member Function Documentation

15.615.3.1 bounded()

```
QCPRange QCPRange::bounded (
    double lowerBound,
    double upperBound ) const
```

Returns this range, possibly modified to not exceed the bounds provided as *lowerBound* and *upperBound*. If possible, the size of the current range is preserved in the process.

If the range shall only be bounded at the lower side, you can set *upperBound* to [QCPRange::maxRange](#). If it shall only be bounded at the upper side, set *lowerBound* to [-QCPRange::maxRange](#).

15.615.3.2 center()

```
double QCPRange::center ( ) const [inline]
```

Returns the center of the range, i.e. $(upper+lower)*0.5$

15.615.3.3 contains()

```
bool QCPRange::contains (
    double value ) const [inline]
```

Returns true when *value* lies within or exactly on the borders of the range.

15.615.3.4 expand() [1/2]

```
void QCPRange::expand (
    const QCPRange & otherRange )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Expands this range such that *otherRange* is contained in the new range. It is assumed that both this range and *otherRange* are normalized (see [normalize](#)).

If this range contains NaN as lower or upper bound, it will be replaced by the respective bound of *otherRange*.

If *otherRange* is already inside the current range, this function does nothing.

See also

[expanded](#)

15.615.3.5 expand() [2/2]

```
void QCPRange::expand (
    double includeCoord )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Expands this range such that *includeCoord* is contained in the new range. It is assumed that this range is normalized (see [normalize](#)).

If this range contains NaN as lower or upper bound, the respective bound will be set to *includeCoord*.

If *includeCoord* is already inside the current range, this function does nothing.

See also

[expand](#)

15.615.3.6 expanded() [1/2]

```
QCPRange QCPRange::expanded (
    const QCPRange & otherRange ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns an expanded range that contains this and *otherRange*. It is assumed that both this range and *otherRange* are normalized (see [normalize](#)).

If this range contains NaN as lower or upper bound, the returned range's bound will be taken from *otherRange*.

See also

[expand](#)

15.615.3.7 expanded() [2/2]

```
QCPRange QCPRange::expanded (
    double includeCoord ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns an expanded range that includes the specified *includeCoord*. It is assumed that this range is normalized (see [normalize](#)).

If this range contains NaN as lower or upper bound, the returned range's bound will be set to *includeCoord*.

See also

[expand](#)

15.615.3.8 normalize()

```
void QCPRange::normalize ( ) [inline]
```

Makes sure *lower* is numerically smaller than *upper*. If this is not the case, the values are swapped.

15.615.3.9 operator*=()

```
QCPRange & QCPRange::operator*= (
    const double & value ) [inline]
```

Multiplies both boundaries of the range by *value*.

15.615.3.10 operator+=()

```
QCPRange & QCPRange::operator+= (
    const double & value ) [inline]
```

Adds *value* to both boundaries of the range.

15.615.3.11 operator-=()

```
QCPRange & QCPRange::operator-= (
    const double & value ) [inline]
```

Subtracts *value* from both boundaries of the range.

15.615.3.12 operator/=()

```
QCPRange & QCPRange::operator/= (
    const double & value ) [inline]
```

Divides both boundaries of the range by *value*.

15.615.3.13 sanitizedForLinScale()

```
QCPRange QCPRange::sanitizedForLinScale ( ) const
```

Returns a sanitized version of the range. Sanitized means for linear scales, that *lower* will always be numerically smaller (or equal) to *upper*.

15.615.3.14 sanitizedForLogScale()

```
QCPRange QCPRange::sanitizedForLogScale ( ) const
```

Returns a sanitized version of the range. Sanitized means for logarithmic scales, that the range won't span the positive and negative sign domain, i.e. contain zero. Further *lower* will always be numerically smaller (or equal) to *upper*.

If the original range does span positive and negative sign domains or contains zero, the returned range will try to approximate the original range as good as possible. If the positive interval of the original range is wider than the negative interval, the returned range will only contain the positive interval, with lower bound set to *rangeFac* or *rangeFac * upper*, whichever is closer to zero. Same procedure is used if the negative interval is wider than the positive interval, this time by changing the *upper* bound.

15.615.3.15 size()

```
double QCPRange::size ( ) const [inline]
```

Returns the size of the range, i.e. *upper-lower*

15.615.3.16 validRange() [1/2]

```
bool QCPRange::validRange (
    const QCPRange & range ) [static]
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts. Checks, whether the specified range is within valid bounds, which are defined as [QCPRange::maxRange](#) and [QCPRange::minRange](#). A valid range means:

- range bounds within -maxRange and maxRange
- range size above minRange
- range size below maxRange

15.615.3.17 validRange() [2/2]

```
bool QCPRange::validRange (
    double lower,
    double upper ) [static]
```

Checks, whether the specified range is within valid bounds, which are defined as [QCPRange::maxRange](#) and [QCPRange::minRange](#). A valid range means:

- range bounds within -maxRange and maxRange
- range size above minRange
- range size below maxRange

15.615.4 Friends And Related Symbol Documentation**15.615.4.1 operator* [1/2]**

```
const QCPRange operator* (
    const QCPRange & range,
    double value ) [friend]
```

Multiplies both boundaries of the range by *value*.

15.615.4.2 operator* [2/2]

```
const QCPRange operator* (
    double value,
    const QCPRange & range ) [friend]
```

Multiplies both boundaries of the range by *value*.

15.615.4.3 operator+ [1/2]

```
const QCPRange operator+ (
    const QCPRange & range,
    double value ) [friend]
```

Adds *value* to both boundaries of the range.

15.615.4.4 operator+ [2/2]

```
const QCPRange operator+ (
    double value,
    const QCPRange & range ) [friend]
```

Adds *value* to both boundaries of the range.

15.615.4.5 operator-

```
const QCPRange operator- (
    const QCPRange & range,
    double value ) [friend]
```

Subtracts *value* from both boundaries of the range.

15.615.4.6 operator/

```
const QCPRange operator/ (
    const QCPRange & range,
    double value ) [friend]
```

Divides both boundaries of the range by *value*.

15.615.4.7 operator<<()

```
QDebug operator<< (
    QDebug d,
    const QCPRange & range ) [related]
```

Prints *range* in a human readable format to the qDebug output.

15.615.5 Member Data Documentation

15.615.5.1 maxRange

```
const double QCPRange::maxRange = 1e250 [static]
```

Maximum values (negative and positive) the range will accept in range-changing functions. Larger absolute values would cause errors due to the 11-bit exponent of double precision numbers, corresponding to a maximum magnitude of roughly 1e308.

Warning

Do not use this constant to indicate "arbitrarily large" values in plotting logic (as values that will appear in the plot)! It is intended only as a bound to compare against, e.g. to prevent axis ranges from obtaining overflowing ranges.

See also

[validRange](#), [minRange](#)

15.615.5.2 minRange

```
const double QCPRange::minRange = 1e-280 [static]
```

Minimum range size (*upper* - *lower*) the range changing functions will accept. Smaller intervals would cause errors due to the 11-bit exponent of double precision numbers, corresponding to a minimum magnitude of roughly 1e-308.

Warning

Do not use this constant to indicate "arbitrarily small" values in plotting logic (as values that will appear in the plot)! It is intended only as a bound to compare against, e.g. to prevent axis ranges from obtaining underflowing ranges.

See also

[validRange](#), [maxRange](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.616 QCPScatterStyle Class Reference

Represents the visual appearance of scatter points.

Public Types

- enum [ScatterProperty](#) {
[spNone](#) = 0x00 , [spPen](#) = 0x01 , [spBrush](#) = 0x02 , [spSize](#) = 0x04 ,
[spShape](#) = 0x08 , [spAll](#) = 0xFF }
- enum [ScatterShape](#) {
[ssNone](#) , [ssDot](#) , [ssCross](#) , [ssPlus](#) ,
[ssCircle](#) , [ssDisc](#) , [ssSquare](#) , [ssDiamond](#) ,
[ssStar](#) , [ssTriangle](#) , [ssTriangleInverted](#) , [ssCrossSquare](#) ,
[ssPlusSquare](#) , [ssCrossCircle](#) , [ssPlusCircle](#) , [ssPeace](#) ,
[ssPixmap](#) , [ssCustom](#) }

Public Member Functions

- [QCPScatterStyle](#) ()
- [QCPScatterStyle](#) ([ScatterShape](#) shape, double size=6)
- [QCPScatterStyle](#) ([ScatterShape](#) shape, const QColor &color, double size)
- [QCPScatterStyle](#) ([ScatterShape](#) shape, const QColor &color, const QColor &fill, double size)
- [QCPScatterStyle](#) ([ScatterShape](#) shape, const QPen &pen, const QBrush &brush, double size)
- [QCPScatterStyle](#) (const QPixmap &pixmap)
- [QCPScatterStyle](#) (const QPainterPath &customPath, const QPen &pen, const QBrush &brush=Qt::NoBrush, double size=6)
- double **size** () const
- [ScatterShape](#) **shape** () const
- QPen **pen** () const
- QBrush **brush** () const
- QPixmap **pixmap** () const
- QPainterPath **customPath** () const
- void [setFromOther](#) (const [QCPScatterStyle](#) &other, ScatterProperties properties)
- void [setSize](#) (double size)
- void [setShape](#) ([ScatterShape](#) shape)
- void [setPen](#) (const QPen &pen)
- void [setBrush](#) (const QBrush &brush)
- void [setPixmap](#) (const QPixmap &pixmap)
- void [setCustomPath](#) (const QPainterPath &customPath)
- bool [isNone](#) () const
- bool [isPenDefined](#) () const
- void [undefinePen](#) ()
- void [applyTo](#) ([QCPPainter](#) *painter, const QPen &defaultPen) const
- void [drawShape](#) ([QCPPainter](#) *painter, const QPointF &pos) const
- void [drawShape](#) ([QCPPainter](#) *painter, double x, double y) const

Protected Attributes

- double **mSize**
- [ScatterShape](#) **mShape**
- QPen **mPen**
- QBrush **mBrush**
- QPixmap **mPixmap**
- QPainterPath **mCustomPath**
- bool **mPenDefined**

15.616.1 Detailed Description

Represents the visual appearance of scatter points.

This class holds information about shape, color and size of scatter points. In plottables like [QCPGraph](#) it is used to store how scatter points shall be drawn. For example, [QCPGraph::setScatterStyle](#) takes a [QCPScatterStyle](#) instance.

A scatter style consists of a shape ([setShape](#)), a line color ([setPen](#)) and possibly a fill ([setBrush](#)), if the shape provides a fillable area. Further, the size of the shape can be controlled with [setSize](#).

15.616.2 Specifying a scatter style

You can set all these configurations either by calling the respective functions on an instance:

Or you can use one of the various constructors that take different parameter combinations, making it easy to specify a scatter style in a single call, like so:

15.616.3 Leaving the color/pen up to the plottable

There are two constructors which leave the pen undefined: [QCPScatterStyle\(\)](#) and [QCPScatterStyle\(ScatterShape shape, double size\)](#). If those constructors are used, a call to [isPenDefined](#) will return false. It leads to scatter points that inherit the pen from the plottable that uses the scatter style. Thus, if such a scatter style is passed to [QCPGraph](#), the line color of the graph ([QCPGraph::setPen](#)) will be used by the scatter points. This makes it very convenient to set up typical scatter settings:

Notice that it wasn't even necessary to explicitly call a [QCPScatterStyle](#) constructor. This works because [QCPScatterStyle](#) provides a constructor that can transform a [ScatterShape](#) directly into a [QCPScatterStyle](#) instance (that's the [QCPScatterStyle\(ScatterShape shape, double size\)](#) constructor with a default for *size*). In those cases, C++ allows directly supplying a [ScatterShape](#), where actually a [QCPScatterStyle](#) is expected.

15.616.4 Custom shapes and pixmaps

[QCPScatterStyle](#) supports drawing custom shapes and arbitrary pixmaps as scatter points.

For custom shapes, you can provide a [QPainterPath](#) with the desired shape to the [setCustomPath](#) function or call the constructor that takes a painter path. The scatter shape will automatically be set to [ssCustom](#).

For pixmaps, you call [setPixmap](#) with the desired [QPixmap](#). Alternatively you can use the constructor that takes a [QPixmap](#). The scatter shape will automatically be set to [ssPixmap](#). Note that [setSize](#) does not influence the appearance of the pixmap.

15.616.5 Member Enumeration Documentation

15.616.5.1 ScatterProperty

```
enum QCPScatterStyle::ScatterProperty
```

Represents the various properties of a scatter style instance. For example, this enum is used to specify which properties of [QCPSelectionDecorator::setScatterStyle](#) will be used when highlighting selected data points.

Specific scatter properties can be transferred between [QCPScatterStyle](#) instances via [setFromOther](#).

Enumerator

spNone	0x00 None
spPen	0x01 The pen property, see setPen
spBrush	0x02 The brush property, see setBrush
spSize	0x04 The size property, see setSize
spShape	0x08 The shape property, see setShape
spAll	0xFF All properties

15.616.5.2 ScatterShape

```
enum QCPScatterStyle::ScatterShape
```

Defines the shape used for scatter points.

On plottables/items that draw scatters, the sizes of these visualizations (with exception of [ssDot](#) and [ssPixmap](#)) can be controlled with the [setSize](#) function. Scatters are drawn with the pen and brush specified with [setPen](#) and [setBrush](#).

Enumerator

ssNone	no scatter symbols are drawn (e.g. in QCPGraph , data only represented with lines)
ssDot	<code>\enumimage{ssDot.png}</code> a single pixel (use ssDisc or ssCircle if you want a round shape with a certain radius)
ssCross	<code>\enumimage{ssCross.png}</code> a cross
ssPlus	<code>\enumimage{ssPlus.png}</code> a plus
ssCircle	<code>\enumimage{ssCircle.png}</code> a circle
ssDisc	<code>\enumimage{ssDisc.png}</code> a circle which is filled with the pen's color (not the brush as with ssCircle)
ssSquare	<code>\enumimage{ssSquare.png}</code> a square
ssDiamond	<code>\enumimage{ssDiamond.png}</code> a diamond
ssStar	<code>\enumimage{ssStar.png}</code> a star with eight arms, i.e. a combination of cross and plus
ssTriangle	<code>\enumimage{ssTriangle.png}</code> an equilateral triangle, standing on baseline
ssTriangleInverted	<code>\enumimage{ssTriangleInverted.png}</code> an equilateral triangle, standing on corner
ssCrossSquare	<code>\enumimage{ssCrossSquare.png}</code> a square with a cross inside
ssPlusSquare	<code>\enumimage{ssPlusSquare.png}</code> a square with a plus inside
ssCrossCircle	<code>\enumimage{ssCrossCircle.png}</code> a circle with a cross inside
ssPlusCircle	<code>\enumimage{ssPlusCircle.png}</code> a circle with a plus inside
ssPeace	<code>\enumimage{ssPeace.png}</code> a circle, with one vertical and two downward diagonal lines
ssPixmap	a custom pixmap specified by setPixmap , centered on the data point coordinates
ssCustom	custom painter operations are performed per scatter (As <code>QPainterPath</code> , see setCustomPath)

15.616.6 Constructor & Destructor Documentation

15.616.6.1 QCPScatterStyle() [1/7]

```
QCPScatterStyle::QCPScatterStyle ( )
```

Creates a new [QCPScatterStyle](#) instance with size set to 6. No shape, pen or brush is defined.

Since the pen is undefined ([isPenDefined](#) returns false), the scatter color will be inherited from the plottable that uses this scatter style.

15.616.6.2 QCPScatterStyle() [2/7]

```
QCPScatterStyle::QCPScatterStyle (
    ScatterShape shape,
    double size = 6 )
```

Creates a new [QCPScatterStyle](#) instance with shape set to *shape* and size to *size*. No pen or brush is defined.

Since the pen is undefined ([isPenDefined](#) returns false), the scatter color will be inherited from the plottable that uses this scatter style.

15.616.6.3 QCPScatterStyle() [3/7]

```
QCPScatterStyle::QCPScatterStyle (
    ScatterShape shape,
    const QColor & color,
    double size )
```


Creates a new [QCPScatterStyle](#) instance with shape set to *shape*, the pen color set to *color*, and size to *size*. No brush is defined, i.e. the scatter point will not be filled.

15.616.6.4 QCPScatterStyle() [4/7]

```
QCPScatterStyle::QCPScatterStyle (
    ScatterShape shape,
    const QColor & color,
    const QColor & fill,
    double size )
```

Creates a new [QCPScatterStyle](#) instance with shape set to *shape*, the pen color set to *color*, the brush color to *fill* (with a solid pattern), and size to *size*.

15.616.6.5 QCPScatterStyle() [5/7]

```
QCPScatterStyle::QCPScatterStyle (
    ScatterShape shape,
    const QPen & pen,
    const QBrush & brush,
    double size )
```

Creates a new [QCPScatterStyle](#) instance with shape set to *shape*, the pen set to *pen*, the brush to *brush*, and size to *size*.

Warning

In some cases it might be tempting to directly use a pen style like `Qt::NoPen` as *pen* and a color like `Qt::blue` as *brush*. Notice however, that the corresponding call `QCPScatterStyle(QCPScatterShape::ssCircle, Qt::NoPen, Qt::blue, 5)` doesn't necessarily lead C++ to use this constructor in some cases, but might mistake `Qt::NoPen` for a `QColor` and use the `QCPScatterStyle(ScatterShape shape, const QColor &color, const QColor &fill, double size)` constructor instead (which will lead to an unexpected look of the scatter points). To prevent this, be more explicit with the parameter types. For example, use `QBrush(Qt::blue)` instead of just `Qt::blue`, to clearly point out to the compiler that this constructor is wanted.

15.616.6.6 QCPScatterStyle() [6/7]

```
QCPScatterStyle::QCPScatterStyle (
    const QPixmap & pixmap )
```

Creates a new [QCPScatterStyle](#) instance which will show the specified *pixmap*. The scatter shape is set to [ssPixmap](#).

15.616.6.7 QCPScatterStyle() [7/7]

```
QCPScatterStyle::QCPScatterStyle (
    const QPainterPath & customPath,
    const QPen & pen,
    const QBrush & brush = Qt::NoBrush,
    double size = 6 )
```

Creates a new [QCPScatterStyle](#) instance with a custom shape that is defined via *customPath*. The scatter shape is set to [ssCustom](#).

The custom shape line will be drawn with *pen* and filled with *brush*. The size has a slightly different meaning than for built-in scatter points: The custom path will be drawn scaled by a factor of *size/6.0*. Since the default *size* is 6, the custom path will appear in its original size by default. To for example double the size of the path, set *size* to 12.

15.616.7 Member Function Documentation

15.616.7.1 applyTo()

```
void QCPScatterStyle::applyTo (
```

```
QCPPainter * painter,
const QPen & defaultPen ) const
```

Applies the pen and the brush of this scatter style to *painter*. If this scatter style has an undefined pen (*isPenDefined*), sets the pen of *painter* to *defaultPen* instead.

This function is used by plottables (or any class that wants to draw scatters) just before a number of scatters with this style shall be drawn with the *painter*.

See also

[drawShape](#)

15.616.7.2 drawShape() [1/2]

```
void QCPScatterStyle::drawShape (
    QCPPainter * painter,
    const QPointF & pos ) const
```

Draws the scatter shape with *painter* at position *pos*.

This function does not modify the pen or the brush on the painter, as [applyTo](#) is meant to be called before scatter points are drawn with [drawShape](#).

See also

[applyTo](#)

15.616.7.3 drawShape() [2/2]

```
void QCPScatterStyle::drawShape (
    QCPPainter * painter,
    double x,
    double y ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts. Draws the scatter shape with *painter* at position *x* and *y*.

15.616.7.4 isNone()

```
bool QCPScatterStyle::isNone ( ) const [inline]
```

Returns whether the scatter shape is [ssNone](#).

See also

[setShape](#)

15.616.7.5 isPenDefined()

```
bool QCPScatterStyle::isPenDefined ( ) const [inline]
```

Returns whether a pen has been defined for this scatter style.

The pen is undefined if a constructor is called that does not carry *pen* as parameter. Those are [QCPScatterStyle\(\)](#) and [QCPScatterStyle\(ScatterShape shape, double size\)](#). If the pen is undefined, the pen of the respective plottable will be used for drawing scatters.

If a pen was defined for this scatter style instance, and you now wish to undefine the pen, call [undefinePen](#).

See also

[setPen](#)

15.616.7.6 setBrush()

```
void QCPScatterStyle::setBrush (
    const QBrush & brush )
```

Sets the brush that will be used to fill scatter points to *brush*. Note that not all scatter shapes have fillable areas. For example, [ssPlus](#) does not while [ssCircle](#) does.

See also

[setPen](#)

15.616.7.7 setCustomPath()

```
void QCPScatterStyle::setCustomPath (
    const QPainterPath & customPath )
```

Sets the custom shape that will be drawn as scatter point to *customPath*. The scatter shape is automatically set to [ssCustom](#).

15.616.7.8 setFromOther()

```
void QCPScatterStyle::setFromOther (
    const QCPScatterStyle & other,
    ScatterProperties properties )
```

Copies the specified *properties* from the *other* scatter style to this scatter style.

15.616.7.9 setPen()

```
void QCPScatterStyle::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw scatter points to *pen*.

If the pen was previously undefined (see [isPenDefined](#)), the pen is considered defined after a call to this function, even if *pen* is `Qt::NoPen`. If you have defined a pen previously by calling this function and now wish to undefine the pen, call [undefinePen](#).

See also

[setBrush](#)

15.616.7.10 setPixmap()

```
void QCPScatterStyle::setPixmap (
    const QPixmap & pixmap )
```

Sets the pixmap that will be drawn as scatter point to *pixmap*.

Note that [setSize](#) does not influence the appearance of the pixmap.

The scatter shape is automatically set to [ssPixmap](#).

15.616.7.11 setShape()

```
void QCPScatterStyle::setShape (
    QCPScatterStyle::ScatterShape shape )
```

Sets the shape to *shape*.

Note that the calls [setPixmap](#) and [setCustomPath](#) automatically set the shape to [ssPixmap](#) and [ssCustom](#), respectively.

See also

[setSize](#)

15.616.7.12 setSize()

```
void QCPScatterStyle::setSize (
    double size )
```

Sets the size (pixel diameter) of the drawn scatter points to *size*.

See also

[setShape](#)

- void [setPen](#) (const QPen &pen)
- void [setBrush](#) (const QBrush &brush)
- void [setScatterStyle](#) (const [QCPScatterStyle](#) &scatterStyle, [QCPScatterStyle::ScatterProperties](#) used←→
Properties=[QCPScatterStyle::spPen](#))
- void [setUsedScatterProperties](#) (const [QCPScatterStyle::ScatterProperties](#) &properties)
- void [applyPen](#) ([QCPPainter](#) *painter) const
- void [applyBrush](#) ([QCPPainter](#) *painter) const
- [QCPScatterStyle](#) [getFinalScatterStyle](#) (const [QCPScatterStyle](#) &unselectedStyle) const
- virtual void [copyFrom](#) (const [QCPSelectionDecorator](#) *other)
- virtual void [drawDecoration](#) ([QCPPainter](#) *painter, [QCPDataSelection](#) selection)

Protected Member Functions

- virtual bool [registerWithPlottable](#) ([QCPAbstractPlottable](#) *plottable)

Protected Attributes

- QPen **mPen**
- QBrush **mBrush**
- [QCPScatterStyle](#) **mScatterStyle**
- [QCPScatterStyle::ScatterProperties](#) **mUsedScatterProperties**
- [QCPAbstractPlottable](#) * **mPlottable**

Friends

- class [QCPAbstractPlottable](#)

15.617.1 Detailed Description

Controls how a plottable's data selection is drawn.

Each [QCPAbstractPlottable](#) instance has one [QCPSelectionDecorator](#) (accessible via [QCPAbstractPlottable::selectionDecorator](#)) and uses it when drawing selected segments of its data.

The selection decorator controls both pen ([setPen](#)) and brush ([setBrush](#)), as well as the scatter style ([setScatterStyle](#)) if the plottable draws scatters. Since a [QCPScatterStyle](#) is itself composed of different properties such as color shape and size, the decorator allows specifying exactly which of those properties shall be used for the selected data point, via [setUsedScatterProperties](#).

A [QCPSelectionDecorator](#) subclass instance can be passed to a plottable via [QCPAbstractPlottable::setSelectionDecorator](#), allowing greater customizability of the appearance of selected segments.

Use [copyFrom](#) to easily transfer the settings of one decorator to another one. This is especially useful since plottables take ownership of the passed selection decorator, and thus the same decorator instance can not be passed to multiple plottables.

Selection decorators can also themselves perform drawing operations by reimplementing [drawDecoration](#), which is called by the plottable's draw method. The base class [QCPSelectionDecorator](#) does not make use of this however. For example, [QCPSelectionDecoratorBracket](#) draws brackets around selected data segments.

15.617.2 Constructor & Destructor Documentation

15.617.2.1 QCPSelectionDecorator()

```
QCPSelectionDecorator::QCPSelectionDecorator ( )
```

Creates a new [QCPSelectionDecorator](#) instance with default values

15.617.3 Member Function Documentation

15.617.3.1 applyBrush()

```
void QCPSelectionDecorator::applyBrush (
    QCPPainter * painter ) const
```

Sets the brush of *painter* to the brush of this selection decorator.

See also

[applyPen](#), [getFinalScatterStyle](#)

15.617.3.2 applyPen()

```
void QCPSelectionDecorator::applyPen (
    QCPPainter * painter ) const
```

Sets the pen of *painter* to the pen of this selection decorator.

See also

[applyBrush](#), [getFinalScatterStyle](#)

15.617.3.3 copyFrom()

```
void QCPSelectionDecorator::copyFrom (
    const QCPSelectionDecorator * other ) [virtual]
```

Copies all properties (e.g. color, fill, scatter style) of the *other* selection decorator to this selection decorator.

15.617.3.4 drawDecoration()

```
void QCPSelectionDecorator::drawDecoration (
    QCPPainter * painter,
    QCPCDataSelection selection ) [virtual]
```

This method is called by all plottables' draw methods to allow custom selection decorations to be drawn. Use the passed *painter* to perform the drawing operations. *selection* carries the data selection for which the decoration shall be drawn.

The default base class implementation of [QCPSelectionDecorator](#) has no special decoration, so this method does nothing.

Reimplemented in [QCPSelectionDecoratorBracket](#).

15.617.3.5 getFinalScatterStyle()

```
QCPScatterStyle QCPSelectionDecorator::getFinalScatterStyle (
    const QCPScatterStyle & unselectedStyle ) const
```

Returns the scatter style that the parent plottable shall use for selected scatter points. The plottable's original (unselected) scatter style must be passed as *unselectedStyle*. Depending on the setting of [setUsedScatterProperties](#), the returned scatter style is a mixture of this selection decorator's scatter style ([setScatterStyle](#)), and *unselectedStyle*.

See also

[applyPen](#), [applyBrush](#), [setScatterStyle](#)

15.617.3.6 setBrush()

```
void QCPSelectionDecorator::setBrush (
    const QBrush & brush )
```

Sets the brush that will be used by the parent plottable to draw selected data segments.

15.617.3.7 setPen()

```
void QCPSelectionDecorator::setPen (
    const QPen & pen )
```

Sets the pen that will be used by the parent plottable to draw selected data segments.

15.617.3.8 setScatterStyle()

```
void QCPSelectionDecorator::setScatterStyle (
    const QCPScatterStyle & scatterStyle,
    QCPScatterStyle::ScatterProperties usedProperties = QCPScatterStyle::spPen )
```

Sets the scatter style that will be used by the parent plottable to draw scatters in selected data segments. *usedProperties* specifies which parts of the passed *scatterStyle* will be used by the plottable. The used properties can also be changed via [setUsedScatterProperties](#).

15.617.3.9 setUsedScatterProperties()

```
void QCPSelectionDecorator::setUsedScatterProperties (
    const QCPScatterStyle::ScatterProperties & properties )
```

Use this method to define which properties of the scatter style (set via [setScatterStyle](#)) will be used for selected data segments. All properties of the scatter style that are not specified in *properties* will remain as specified in the plottable's original scatter style.

See also

[QCPScatterStyle::ScatterProperty](#)

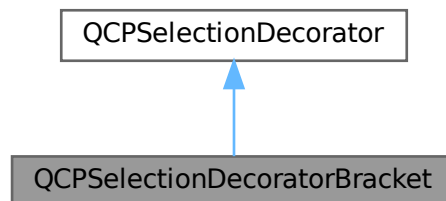
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.618 QCPSelectionDecoratorBracket Class Reference

A selection decorator which draws brackets around each selected data segment.

Inheritance diagram for QCPSelectionDecoratorBracket:



- void [setScatterStyle](#) (const [QCPScatterStyle](#) &scatterStyle, [QCPScatterStyle::ScatterProperties](#) used←→ Properties=[QCPScatterStyle::spPen](#))
- void [setUsedScatterProperties](#) (const [QCPScatterStyle::ScatterProperties](#) &properties)
- void [applyPen](#) ([QCPPainter](#) *painter) const
- void [applyBrush](#) ([QCPPainter](#) *painter) const
- [QCPScatterStyle](#) [getFinalScatterStyle](#) (const [QCPScatterStyle](#) &unselectedStyle) const
- virtual void [copyFrom](#) (const [QCPSelectionDecorator](#) *other)

Protected Member Functions

- double [getTangentAngle](#) (const [QCPPlottableInterface1D](#) *interface1d, int dataIndex, int direction) const
- [QPointF](#) [getPixelCoordinates](#) (const [QCPPlottableInterface1D](#) *interface1d, int dataIndex) const

Protected Member Functions inherited from [QCPSelectionDecorator](#)

- virtual bool [registerWithPlottable](#) ([QCPAbstractPlottable](#) *plottable)

Protected Attributes

- [QPen](#) [mBracketPen](#)
- [QBrush](#) [mBracketBrush](#)
- int [mBracketWidth](#)
- int [mBracketHeight](#)
- [BracketStyle](#) [mBracketStyle](#)
- bool [mTangentToData](#)
- int [mTangentAverage](#)

Protected Attributes inherited from [QCPSelectionDecorator](#)

- [QPen](#) [mPen](#)
- [QBrush](#) [mBrush](#)
- [QCPScatterStyle](#) [mScatterStyle](#)
- [QCPScatterStyle::ScatterProperties](#) [mUsedScatterProperties](#)
- [QCPAbstractPlottable](#) * [mPlottable](#)

15.618.1 Detailed Description

A selection decorator which draws brackets around each selected data segment.

Additionally to the regular highlighting of selected segments via color, fill and scatter style, this [QCPSelectionDecorator](#) subclass draws markers at the begin and end of each selected data segment of the plottable.

The shape of the markers can be controlled with [setBracketStyle](#), [setBracketWidth](#) and [setBracketHeight](#). The color/fill can be controlled with [setBracketPen](#) and [setBracketBrush](#).

To introduce custom bracket styles, it is only necessary to subclass [QCPSelectionDecoratorBracket](#) and reimplement [drawBracket](#). The rest will be managed by the base class.

15.618.2 Member Enumeration Documentation

15.618.2.1 BracketStyle

enum [QCPSelectionDecoratorBracket::BracketStyle](#)

Defines which shape is drawn at the boundaries of selected data ranges.

Some of the bracket styles further allow specifying a height and/or width, see [setBracketHeight](#) and [setBracketWidth](#).

Enumerator

bsSquareBracket	A square bracket is drawn.
bsHalfEllipse	A half ellipse is drawn. The size of the ellipse is given by the bracket width/height properties.
bsEllipse	An ellipse is drawn. The size of the ellipse is given by the bracket width/height properties.

Enumerator

bsPlus	A plus is drawn.
bsUserStyle	Start custom bracket styles at this index when subclassing and reimplementing drawBracket .

15.618.3 Constructor & Destructor Documentation

15.618.3.1 QCPSelectionDecoratorBracket()

`QCPSelectionDecoratorBracket::QCPSelectionDecoratorBracket ( )`

Creates a new [QCPSelectionDecoratorBracket](#) instance with default values.

15.618.4 Member Function Documentation

15.618.4.1 drawBracket()

```
void QCPSelectionDecoratorBracket::drawBracket (
    QCPPainter * painter,
    int direction ) const [virtual]
```

Draws the bracket shape with *painter*. The parameter *direction* is either -1 or 1 and indicates whether the bracket shall point to the left or the right (i.e. is a closing or opening bracket, respectively).

The passed *painter* already contains all transformations that are necessary to position and rotate the bracket appropriately. Painting operations can be performed as if drawing upright brackets on flat data with horizontal key axis, with (0, 0) being the center of the bracket.

If you wish to subclass [QCPSelectionDecoratorBracket](#) in order to provide custom bracket shapes (see [QCPSelectionDecoratorBracket::bsUserStyle](#)), this is the method you should reimplement.

15.618.4.2 drawDecoration()

```
void QCPSelectionDecoratorBracket::drawDecoration (
    QCPPainter * painter,
    QCPDataSelection selection ) [virtual]
```

Draws the bracket decoration on the data points at the begin and end of each selected data segment given in *selection*.

It uses the method [drawBracket](#) to actually draw the shapes.

\see [baseclassmethod](#)

Reimplemented from [QCPSelectionDecorator](#).

15.618.4.3 setBracketBrush()

```
void QCPSelectionDecoratorBracket::setBracketBrush (
    const QBrush & brush )
```

Sets the brush that will be used to draw the brackets at the beginning and end of each selected data segment.

15.618.4.4 setBracketHeight()

```
void QCPSelectionDecoratorBracket::setBracketHeight (
    int height )
```

Sets the height of the drawn bracket. The height dimension is always perpendicular to the key axis of the data, or the tangent direction of the current data slope, if [setTangentToData](#) is enabled.

15.618.4.5 setBracketPen()

```
void QCPSelectionDecoratorBracket::setBracketPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the brackets at the beginning and end of each selected data segment.

15.618.4.6 setBracketStyle()

```
void QCPSelectionDecoratorBracket::setBracketStyle (
    QCPSelectionDecoratorBracket::BracketStyle style )
```

Sets the shape that the bracket/marker will have.

See also

[setBracketWidth](#), [setBracketHeight](#)

15.618.4.7 setBracketWidth()

```
void QCPSelectionDecoratorBracket::setBracketWidth (
    int width )
```

Sets the width of the drawn bracket. The width dimension is always parallel to the key axis of the data, or the tangent direction of the current data slope, if [setTangentToData](#) is enabled.

15.618.4.8 setTangentAverage()

```
void QCPSelectionDecoratorBracket::setTangentAverage (
    int pointCount )
```

Controls over how many data points the slope shall be averaged, when brackets shall be aligned with the data (if [setTangentToData](#) is true).

From the position of the bracket, *pointCount* points towards the selected data range will be taken into account. The smallest value of *pointCount* is 1, which is effectively equivalent to disabling [setTangentToData](#).

15.618.4.9 setTangentToData()

```
void QCPSelectionDecoratorBracket::setTangentToData (
    bool enabled )
```

Sets whether the brackets will be rotated such that they align with the slope of the data at the position that they appear in.

For noisy data, it might be more visually appealing to average the slope over multiple data points. This can be configured via [setTangentAverage](#).

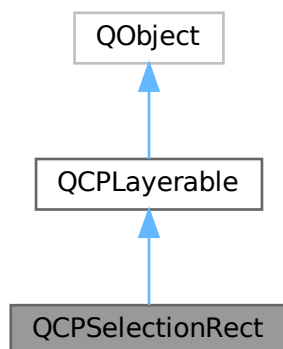
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

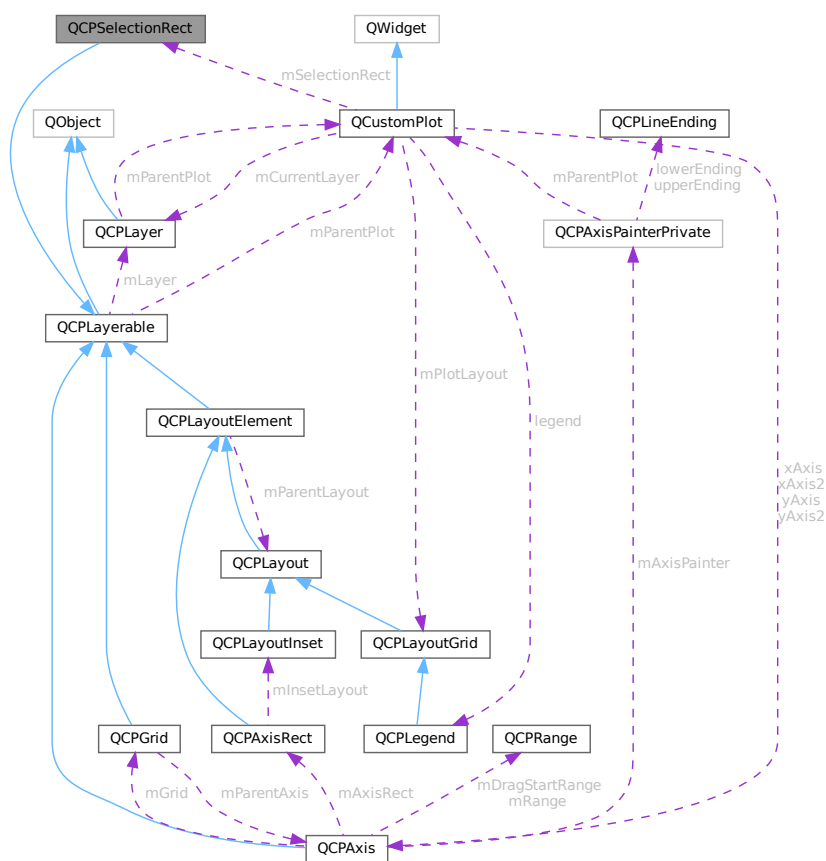
15.619 QCPSelectionRect Class Reference

Provides rect/rubber-band data selection and range zoom interaction.

Inheritance diagram for QCPSelectionRect:



Collaboration diagram for QCPSelectionRect:



Signals

- void **started** (QMouseEvent *event)

- void [changed](#) (const QRect &rect, QMouseEvent *event)
- void [canceled](#) (const QRect &rect, QInputEvent *event)
- void [accepted](#) (const QRect &rect, QMouseEvent *event)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

Public Member Functions

- [QCPSelectionRect](#) ([QCustomPlot](#) *parentPlot)
- QRect **rect** () const
- [QCPRange](#) **range** (const [QCPAxis](#) *axis) const
- QPen **pen** () const
- QBrush **brush** () const
- bool **isActive** () const
- void [setPen](#) (const QPen &pen)
- void [setBrush](#) (const QBrush &brush)
- Q_SLOT void [cancel](#) ()

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void **startSelection** (QMouseEvent *event)
- virtual void **moveSelection** (QMouseEvent *event)
- virtual void **endSelection** (QMouseEvent *event)
- virtual void **keyPressEvent** (QKeyEvent *event)
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual [QCP::Interaction](#) **selectionCategory** () const
- virtual QRect **clipRect** () const
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged↵ Changed)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)

- virtual void [wheelEvent](#) (QWheelEvent *event)
- void [initializeParentPlot](#) (QCustomPlot *parentPlot)
- void [setParentLayerable](#) (QCPLayerable *parentLayerable)
- bool [moveToLayer](#) (QCPLayer *layer, bool prepend)
- void [applyAntialiasingHint](#) (QCPPainter *painter, bool localAntialiased, QCP::AntialiasedElement overrideElement) const

Protected Attributes

- QRect [mRect](#)
- QPen [mPen](#)
- QBrush [mBrush](#)
- bool [mActive](#)

Protected Attributes inherited from [QCPLayerable](#)

- bool [mVisible](#)
- [QCustomPlot](#) * [mParentPlot](#)
- QPointer< [QCPLayerable](#) > [mParentLayerable](#)
- [QCPLayer](#) * [mLayer](#)
- bool [mAntialiased](#)

Friends

- class [QCustomPlot](#)

15.619.1 Detailed Description

Provides rect/rubber-band data selection and range zoom interaction.

[QCPSelectionRect](#) is used by [QCustomPlot](#) when the [QCustomPlot::setSelectionRectMode](#) is not [QCP::srmNone](#). When the user drags the mouse across the plot, the current selection rect instance ([QCustomPlot::setSelectionRect](#)) is forwarded these events and makes sure an according rect shape is drawn. At the begin, during, and after completion of the interaction, it emits the corresponding signals [started](#), [changed](#), [canceled](#), and [accepted](#). The [QCustomPlot](#) instance connects own slots to the current selection rect instance, in order to react to an accepted selection rect interaction accordingly.

[isActive](#) can be used to check whether the selection rect is currently active. An ongoing selection interaction can be cancelled programmatically via calling [cancel](#) at any time.

The appearance of the selection rect can be controlled via [setPen](#) and [setBrush](#).

If you wish to provide custom behaviour, e.g. a different visual representation of the selection rect ([QCPSelectionRect::draw](#)), you can subclass [QCPSelectionRect](#) and pass an instance of your subclass to [QCustomPlot::setSelectionRect](#).

15.619.2 Constructor & Destructor Documentation

15.619.2.1 QCPSelectionRect()

```
QCPSelectionRect::QCPSelectionRect (
    QCustomPlot * parentPlot ) [explicit]
```

Creates a new [QCPSelectionRect](#) instance. To make [QCustomPlot](#) use the selection rect instance, pass it to [QCustomPlot::setSelectionRect](#). *parentPlot* should be set to the same [QCustomPlot](#) widget.

15.619.3 Member Function Documentation

15.619.3.1 accepted

```
void QCPSelectionRect::accepted (
    const QRect & rect,
    QMouseEvent * event ) [signal]
```

This signal is emitted when the selection interaction was completed by the user releasing the mouse button.

Note that *rect* may have a negative width or height, if the selection is being dragged to the upper or left side of the selection rect origin.

15.619.3.2 applyDefaultAntialiasingHint()

```
void QCPSelectionRect::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Implements [QCPLayerable](#).

15.619.3.3 cancel()

```
void QCPSelectionRect::cancel ( )
```

If there is currently a selection interaction going on ([isActive](#)), the interaction is canceled. The selection rect will emit the [canceled](#) signal.

15.619.3.4 canceled

```
void QCPSelectionRect::canceled (
    const QRect & rect,
    QInputEvent * event ) [signal]
```

This signal is emitted when the selection interaction was cancelled. Note that *event* is `nullptr` if the selection interaction was cancelled programmatically, by a call to [cancel](#).

The user may cancel the selection interaction by pressing the escape key. In this case, *event* holds the respective input event.

Note that *rect* may have a negative width or height, if the selection is being dragged to the upper or left side of the selection rect origin.

15.619.3.5 changed

```
void QCPSelectionRect::changed (
    const QRect & rect,
    QMouseEvent * event ) [signal]
```

This signal is emitted while the selection rect interaction is ongoing and the *rect* has changed its size due to the user moving the mouse.

Note that *rect* may have a negative width or height, if the selection is being dragged to the upper or left side of the selection rect origin.

15.619.3.6 draw()

```
void QCPSelectionRect::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPLayerable](#).

15.619.3.7 isActive()

```
bool QCPSelectionRect::isActive ( ) const [inline]
```

Returns true if there is currently a selection going on, i.e. the user has started dragging a selection rect, but hasn't released the mouse button yet.

See also

[cancel](#)

15.619.3.8 range()

```
QCPRange QCPSelectionRect::range (
    const QCPAxis * axis ) const
```

A convenience function which returns the coordinate range of the provided *axis*, that this selection rect currently encompasses.

15.619.3.9 setBrush()

```
void QCPSelectionRect::setBrush (
    const QBrush & brush )
```

Sets the brush that will be used to fill the selection rect. By default the selection rect is not filled, i.e. *brush* is `Qt::NoBrush`.

See also

[setPen](#)

15.619.3.10 setPen()

```
void QCPSelectionRect::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the selection rect outline.

See also

[setBrush](#)

15.619.3.11 started

```
void QCPSelectionRect::started (
    QMouseEvent * event ) [signal]
```

This signal is emitted when a selection rect interaction was initiated, i.e. the user just started dragging the selection rect with the mouse.

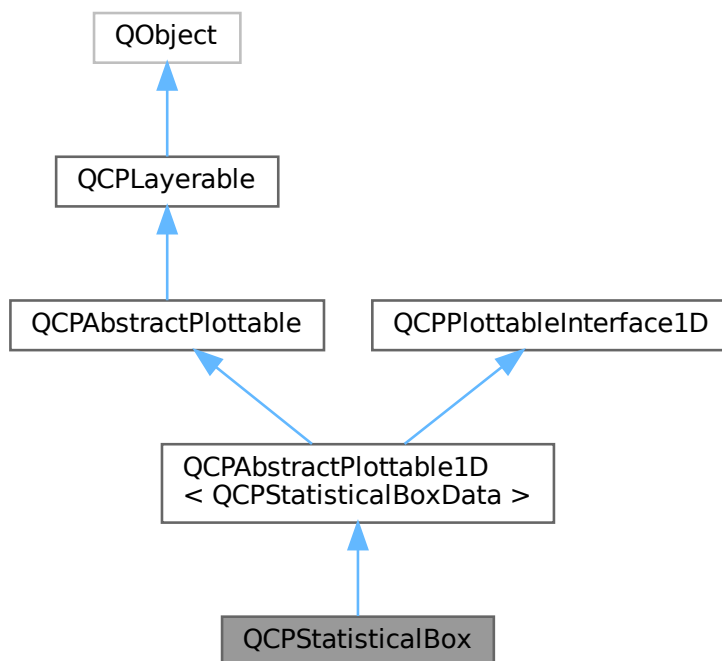
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

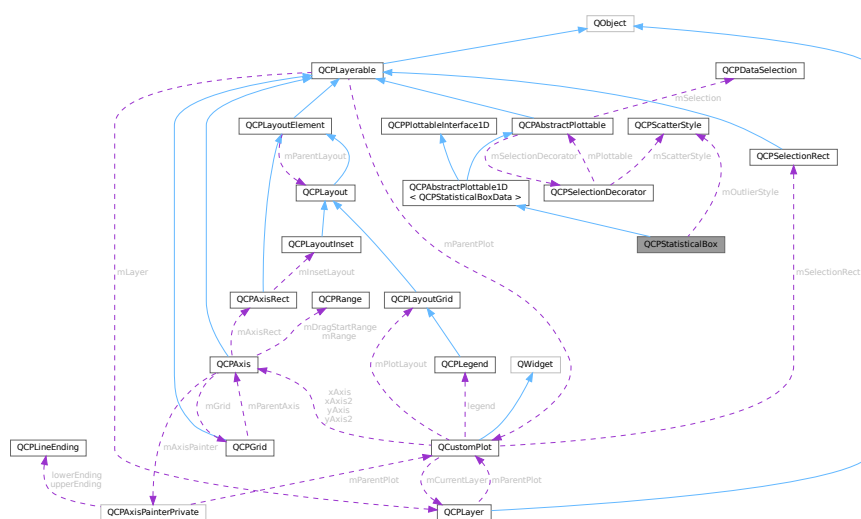
15.620 QCPStatisticalBox Class Reference

A plottable representing a single statistical box in a plot.

Inheritance diagram for QCPStatisticalBox:



Collaboration diagram for QCPStatisticalBox:



Public Member Functions

- `QCPStatisticalBox (QCPAxis *keyAxis, QCPAxis *valueAxis)`
- `QSharedPointer< QCPStatisticalBoxDataContainer > data () const`
- `double width () const`

- double **whiskerWidth** () const
- QPen **whiskerPen** () const
- QPen **whiskerBarPen** () const
- bool **whiskerAntialiased** () const
- QPen **medianPen** () const
- [QCPScatterStyle](#) **outlierStyle** () const
- void **setData** (QSharedPointer< [QCPStatisticalBoxDataContainer](#) > data)
- void **setData** (const QVector< double > &keys, const QVector< double > &minimum, const QVector< double > &lowerQuartile, const QVector< double > &median, const QVector< double > &upperQuartile, const QVector< double > &maximum, bool alreadySorted=false)
- void **setWidth** (double width)
- void **setWhiskerWidth** (double width)
- void **setWhiskerPen** (const QPen &pen)
- void **setWhiskerBarPen** (const QPen &pen)
- void **setWhiskerAntialiased** (bool enabled)
- void **setMedianPen** (const QPen &pen)
- void **setOutlierStyle** (const [QCPScatterStyle](#) &style)
- void **addData** (const QVector< double > &keys, const QVector< double > &minimum, const QVector< double > &lowerQuartile, const QVector< double > &median, const QVector< double > &upperQuartile, const QVector< double > &maximum, bool alreadySorted=false)
- void **addData** (double key, double minimum, double lowerQuartile, double median, double upperQuartile, double maximum, const QVector< double > &outliers=QVector< double >())
- virtual [QCPDataSelection](#) **selectTestRect** (const QRectF &rect, bool onlySelectable) const Q_DECL_OVERRIDE
- virtual double **selectTest** (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE
- virtual [QCPRange](#) **getKeyRange** (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#)) const Q_DECL_OVERRIDE
- virtual [QCPRange](#) **getValueRange** (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#), const [QCPRange](#) &inKeyRange=[QCPRange](#)()) const Q_DECL_OVERRIDE

Public Member Functions inherited from

[QCPAbstractPlottable1D](#)< [QCPStatisticalBoxData](#) >

- [QCPAbstractPlottable1D](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- virtual int **dataCount** () const Q_DECL_OVERRIDE
- virtual double **dataMainKey** (int index) const Q_DECL_OVERRIDE
- virtual double **dataSortKey** (int index) const Q_DECL_OVERRIDE
- virtual double **dataMainValue** (int index) const Q_DECL_OVERRIDE
- virtual [QCPRange](#) **dataValueRange** (int index) const Q_DECL_OVERRIDE
- virtual QPointF **dataPixelPosition** (int index) const Q_DECL_OVERRIDE
- virtual bool **sortKeysIsMainKey** () const Q_DECL_OVERRIDE
- virtual int **findBegin** (double sortKey, bool expandedRange=true) const Q_DECL_OVERRIDE
- virtual int **findEnd** (double sortKey, bool expandedRange=true) const Q_DECL_OVERRIDE
- virtual [QCPPlottableInterface1D](#) * **interface1D** () Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractPlottable](#)

- [QCPAbstractPlottable](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- QString **name** () const
- bool **antialiasedFill** () const
- bool **antialiasedScatters** () const
- QPen **pen** () const
- QBrush **brush** () const
- [QCPAxis](#) * **keyAxis** () const

- [QCPAxis](#) * **valueAxis** () const
- [QCP::SelectionType](#) **selectable** () const
- bool **selected** () const
- [QCPDataSelection](#) **selection** () const
- [QCPSelectionDecorator](#) * **selectionDecorator** () const
- void **setName** (const QString &name)
- void **setAntialiasedFill** (bool enabled)
- void **setAntialiasedScatters** (bool enabled)
- void **setPen** (const QPen &pen)
- void **setBrush** (const QBrush &brush)
- void **setKeyAxis** ([QCPAxis](#) *axis)
- void **setValueAxis** ([QCPAxis](#) *axis)
- Q_SLOT void **setSelectable** ([QCP::SelectionType](#) selectable)
- Q_SLOT void **setSelection** ([QCPDataSelection](#) selection)
- void **setSelectionDecorator** ([QCPSelectionDecorator](#) *decorator)
- void **coordsToPixels** (double key, double value, double &x, double &y) const
- const QPointF **coordsToPixels** (double key, double value) const
- void **pixelsToCoords** (double x, double y, double &key, double &value) const
- void **pixelsToCoords** (const QPointF &pixelPos, double &key, double &value) const
- void **rescaleAxes** (bool onlyEnlarge=false) const
- void **rescaleKeyAxis** (bool onlyEnlarge=false) const
- void **rescaleValueAxis** (bool onlyEnlarge=false, bool inKeyRange=false) const
- bool **addToLegend** ([QCPLegend](#) *legend)
- bool **addToLegend** ()
- bool **removeFromLegend** ([QCPLegend](#) *legend) const
- bool **removeFromLegend** () const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const QString &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Protected Member Functions

- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void **drawLegendIcon** ([QCPPainter](#) *painter, const QRectF &rect) const Q_DECL_OVERRIDE
- virtual void **drawStatisticalBox** ([QCPPainter](#) *painter, [QCPStatisticalBoxDataContainer::const_iterator](#) it, const [QCPScatterStyle](#) &outlierStyle) const
- void **getVisibleDataBounds** ([QCPStatisticalBoxDataContainer::const_iterator](#) &begin, [QCPStatisticalBoxDataContainer::const_iterator](#) &end) const
- QRectF **getQuartileBox** ([QCPStatisticalBoxDataContainer::const_iterator](#) it) const
- QVector< QLineF > **getWhiskerBackboneLines** ([QCPStatisticalBoxDataContainer::const_iterator](#) it) const
- QVector< QLineF > **getWhiskerBarLines** ([QCPStatisticalBoxDataContainer::const_iterator](#) it) const

Protected Member Functions inherited from [QCPAbstractPlottable1D](#)< [QCPStatisticalBoxData](#) >

- void [getDataSegments](#) (QList< [QCPDataRange](#) > &selectedSegments, QList< [QCPDataRange](#) > &unselectedSegments) const
- void [drawPolyline](#) ([QCPPainter](#) *painter, const QVector< QPointF > &lineData) const

Protected Member Functions inherited from [QCPAbstractPlottable](#)

- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- void [applyFillAntialiasingHint](#) ([QCPPainter](#) *painter) const
- void [applyScattersAntialiasingHint](#) ([QCPPainter](#) *painter) const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot)
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void [initializeParentPlot](#) ([QCustomPlot](#) *parentPlot)
- void [setParentLayerable](#) ([QCPLayerable](#) *parentLayerable)
- bool [moveToLayer](#) ([QCPLayer](#) *layer, bool prepend)
- void [applyAntialiasingHint](#) ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- double **mWidth**
- double **mWhiskerWidth**
- QPen **mWhiskerPen**
- QPen **mWhiskerBarPen**
- bool **mWhiskerAntialiased**
- QPen **mMedianPen**
- [QCPScatterStyle](#) **mOutlierStyle**

Protected Attributes inherited from [QCPAbstractPlottable1D](#)< [QCPStatisticalBoxData](#) >

- QSharedPointer< [QCPDataContainer](#)< [QCPStatisticalBoxData](#) > > **mDataContainer**

Protected Attributes inherited from [QCPAbstractPlottable](#)

- QString **mName**
- bool **mAntialiasedFill**
- bool **mAntialiasedScatters**
- QPen **mPen**
- QBrush **mBrush**
- QPointer< [QCPAxis](#) > **mKeyAxis**
- QPointer< [QCPAxis](#) > **mValueAxis**
- [QCP::SelectionType](#) **mSelectable**
- [QCPDataSelection](#) **mSelection**
- [QCPSelectionDecorator](#) * **mSelectionDecorator**

Protected Attributes inherited from [QCPLayerable](#)

- bool `mVisible`
- [QCustomPlot](#) * `mParentPlot`
- [QPointer](#)< [QCPLayerable](#) > `mParentLayerable`
- [QCPLayer](#) * `mLayer`
- bool `mAntialiased`

Friends

- class [QCustomPlot](#)
- class [QCPLegend](#)

Additional Inherited Members

Signals inherited from [QCPAbstractPlottable](#)

- void [selectionChanged](#) (bool `selected`)
- void [selectionChanged](#) (const [QCPDataSelection](#) &`selection`)
- void [selectableChanged](#) ([QCP::SelectionType](#) `selectable`)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *`newLayer`)

15.620.1 Detailed Description

A plottable representing a single statistical box in a plot.

To plot data, assign it with the [setData](#) or [addData](#) functions. Alternatively, you can also access and modify the data via the [data](#) method, which returns a pointer to the internal [QCPStatisticalBoxDataContainer](#).

Additionally each data point can itself have a list of outliers, drawn as scatter points at the key coordinate of the respective statistical box data point. They can either be set by using the respective [addData](#) method or accessing the individual data points through [data](#), and setting the `QVector<double> outliers` of the data points directly.

15.620.2 Changing the appearance

The appearance of each data point box, ranging from the lower to the upper quartile, is controlled via [setPen](#) and [setBrush](#). You may change the width of the boxes with [setWidth](#) in plot coordinates.

Each data point's visual representation also consists of two whiskers. Whiskers are the lines which reach from the upper quartile to the maximum, and from the lower quartile to the minimum. The appearance of the whiskers can be modified with: [setWhiskerPen](#), [setWhiskerBarPen](#), [setWhiskerWidth](#). The whisker width is the width of the bar perpendicular to the whisker at the top (for maximum) and bottom (for minimum). If the whisker pen is changed, make sure to set the `capStyle` to `Qt::FlatCap`. Otherwise the backbone line might exceed the whisker bars by a few pixels due to the pen cap being not perfectly flat.

The median indicator line inside the box has its own pen, [setMedianPen](#).

The outlier data points are drawn as normal scatter points. Their look can be controlled with [setOutlierStyle](#)

15.620.3 Usage

Like all data representing objects in [QCustomPlot](#), the [QCPStatisticalBox](#) is a plottable ([QCPAbstractPlottable](#)). So the plottable-interface of [QCustomPlot](#) applies ([QCustomPlot::plottable](#), [QCustomPlot::removePlottable](#), etc.)

Usually, you first create an instance:

which registers it with the [QCustomPlot](#) instance of the passed axes. Note that this [QCustomPlot](#) instance takes ownership of the plottable, so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead. The newly created plottable can be modified, e.g.:

15.620.4 Constructor & Destructor Documentation

15.620.4.1 QCPStatisticalBox()

```
QCPStatisticalBox::QCPStatisticalBox (
    QCPAxis * keyAxis,
    QCPAxis * valueAxis ) [explicit]
```

Constructs a statistical box which uses *keyAxis* as its key axis ("x") and *valueAxis* as its value axis ("y"). *keyAxis* and *valueAxis* must reside in the same [QCustomPlot](#) instance and not have the same orientation. If either of these restrictions is violated, a corresponding message is printed to the debug output (qDebug), the construction is not aborted, though.

The created [QCPStatisticalBox](#) is automatically registered with the [QCustomPlot](#) instance inferred from *keyAxis*. This [QCustomPlot](#) instance takes ownership of the [QCPStatisticalBox](#), so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead.

15.620.5 Member Function Documentation

15.620.5.1 addData() [1/2]

```
void QCPStatisticalBox::addData (
    const QVector< double > & keys,
    const QVector< double > & minimum,
    const QVector< double > & lowerQuartile,
    const QVector< double > & median,
    const QVector< double > & upperQuartile,
    const QVector< double > & maximum,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided points in *keys*, *minimum*, *lowerQuartile*, *median*, *upperQuartile* and *maximum* to the current data. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *keys* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

15.620.5.2 addData() [2/2]

```
void QCPStatisticalBox::addData (
    double key,
    double minimum,
    double lowerQuartile,
    double median,
    double upperQuartile,
    double maximum,
    const QVector< double > & outliers = QVector<double>() )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided data point as *key*, *minimum*, *lowerQuartile*, *median*, *upperQuartile* and *maximum* to the current data.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

15.620.5.3 data()

```
QSharedPointer< QCPStatisticalBoxDataContainer > QCPStatisticalBox::data ( ) const [inline]
```

Returns a shared pointer to the internal data storage of type [QCPStatisticalBoxDataContainer](#). You may use it to directly manipulate the data, which may be more convenient and faster than using the regular [setData](#) or [addData](#) methods.

15.620.5.4 draw()

```
void QCPStatisticalBox::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

15.620.5.5 drawLegendIcon()

```
void QCPStatisticalBox::drawLegendIcon (
    QCPPainter * painter,
    const QRectF & rect ) const [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

15.620.5.6 drawStatisticalBox()

```
void QCPStatisticalBox::drawStatisticalBox (
    QCPPainter * painter,
    QCPStatisticalBoxDataContainer::const_iterator it,
    const QCPScatterStyle & outlierStyle ) const [protected], [virtual]
```

Draws the graphical representation of a single statistical box with the data given by the iterator *it* with the provided *painter*.

If the statistical box has a set of outlier data points, they are drawn with *outlierStyle*.

See also

[getQuartileBox](#), [getWhiskerBackboneLines](#), [getWhiskerBarLines](#)

15.620.5.7 getKeyRange()

```
QCPRange QCPStatisticalBox::getKeyRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth ) const [virtual]
```

Returns the coordinate range that all data in this plottable span in the key axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getValueRange](#)

Implements [QCPAbstractPlottable](#).

15.620.5.8 getValueRange()

```
QCPRange QCPStatisticalBox::getValueRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth,
    const QCPRange & inKeyRange = QCPRange() ) const [virtual]
```

Returns the coordinate range that the data points in the specified key range (*inKeyRange*) span in the value axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

If *inKeyRange* has both lower and upper bound set to zero (is equal to [QCPRange\(\)](#)), all data points are considered, without any restriction on the keys.

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getKeyRange](#)

Implements [QCPAbstractPlottable](#).

15.620.5.9 selectTest()

```
double QCPStatisticalBox::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Implements a selectTest specific to this plottable's point geometry.

If *details* is not 0, it will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

\see baseclass method [QCPAbstractPlottable::selectTest](#)

Reimplemented from [QCPAbstractPlottable1D< QCPStatisticalBoxData >](#).

15.620.5.10 selectTestRect()

```
QCPDataSelection QCPStatisticalBox::selectTestRect (
    const QRectF & rect,
    bool onlySelectable ) const [virtual]
```

Returns a data selection containing all the data points of this plottable which are contained (or hit by) *rect*. This is used mainly in the selection rect interaction for data selection (data selection mechanism).

If *onlySelectable* is true, an empty [QCPDataSelection](#) is returned if this plottable is not selectable (i.e. if [QCPAbstractPlottable::setSelectable](#) is [QCP::stNone](#)).

Note

rect must be a normalized rect (positive or zero width and height). This is especially important when using the rect of [QCPSelectionRect::accepted](#), which is not necessarily normalized. Use [QRect::normalized\(\)](#) when passing a rect which might not be normalized.

Reimplemented from [QCPAbstractPlottable1D< QCPStatisticalBoxData >](#).

15.620.5.11 setData() [1/2]

```
void QCPStatisticalBox::setData (
    const QVector< double > & keys,
    const QVector< double > & minimum,
    const QVector< double > & lowerQuartile,
    const QVector< double > & median,
    const QVector< double > & upperQuartile,
    const QVector< double > & maximum,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data with the provided points in *keys*, *minimum*, *lowerQuartile*, *median*, *upperQuartile* and *maximum*. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *keys* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

See also

[addData](#)

15.620.5.12 setData() [2/2]

```
void QCPStatisticalBox::setData (
    QSharedPointer< QCPStatisticalBoxDataContainer > data )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data container with the provided *data* container.

Since a QSharedPointer is used, multiple QCPStatisticalBoxes may share the same data container safely. Modifying the data in the container will then affect all statistical boxes that share the container. Sharing can be achieved by simply exchanging the data containers wrapped in shared pointers:

If you do not wish to share containers, but create a copy from an existing container, rather use the [QCPDataContainer<DataType>::set](#) method on the statistical box data container directly:

See also

[addData](#)

15.620.5.13 setMedianPen()

```
void QCPStatisticalBox::setMedianPen (
    const QPen & pen )
```

Sets the pen used for drawing the median indicator line inside the statistical boxes.

15.620.5.14 setOutlierStyle()

```
void QCPStatisticalBox::setOutlierStyle (
    const QCPScatterStyle & style )
```

Sets the appearance of the outlier data points.

Outliers can be specified with the method [addData\(double key, double minimum, double lowerQuartile, double median, double upperC](#)

15.620.5.15 setWhiskerAntialiased()

```
void QCPStatisticalBox::setWhiskerAntialiased (
    bool enabled )
```

Sets whether the statistical boxes whiskers are drawn with antialiasing or not.

Note that antialiasing settings may be overridden by [QCustomPlot::setAntialiasedElements](#) and [QCustomPlot::setNotAntialiasedElement](#)

15.620.5.16 setWhiskerBarPen()

```
void QCPStatisticalBox::setWhiskerBarPen (
    const QPen & pen )
```

Sets the pen used for drawing the whisker bars. Those are the lines parallel to the key axis at each end of the whisker backbone.

Whiskers are the lines which reach from the upper quartile to the maximum, and from the lower quartile to the minimum.

See also

[setWhiskerPen](#)

15.620.5.17 setWhiskerPen()

```
void QCPStatisticalBox::setWhiskerPen (
    const QPen & pen )
```

Sets the pen used for drawing the whisker backbone.

Whiskers are the lines which reach from the upper quartile to the maximum, and from the lower quartile to the minimum.

Make sure to set the *capStyle* of the passed *pen* to `Qt::FlatCap`. Otherwise the backbone line might exceed the whisker bars by a few pixels due to the pen cap being not perfectly flat.

See also

[setWhiskerBarPen](#)

15.620.5.18 setWhiskerWidth()

```
void QCPStatisticalBox::setWhiskerWidth (
    double width )
```

Sets the width of the whiskers in key coordinates.

Whiskers are the lines which reach from the upper quartile to the maximum, and from the lower quartile to the minimum.

See also

[setWidth](#)

15.620.5.19 setWidth()

```
void QCPStatisticalBox::setWidth (
    double width )
```

Sets the width of the boxes in key coordinates.

See also

[setWhiskerWidth](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.621 QCPStatisticalBoxData Class Reference

Holds the data of one single data point for [QCPStatisticalBox](#).

Public Member Functions

- [QCPStatisticalBoxData](#) ()
- [QCPStatisticalBoxData](#) (double key, double minimum, double lowerQuartile, double median, double upperQuartile, double maximum, const QVector< double > &outliers=QVector< double >())
- double [sortKey](#) () const
- double [mainKey](#) () const
- double [mainValue](#) () const
- [QCPRange](#) [valueRange](#) () const

Static Public Member Functions

- static [QCPStatisticalBoxData](#) [fromSortKey](#) (double [sortKey](#))
- static bool [sortKeysIsMainKey](#) ()

Public Attributes

- double **key**
- double **minimum**
- double **lowerQuartile**
- double **median**
- double **upperQuartile**
- double **maximum**
- QVector< double > **outliers**

15.621.1 Detailed Description

Holds the data of one single data point for [QCPStatisticalBox](#).

The stored data is:

- *key*: coordinate on the key axis of this data point (this is the *mainKey* and the *sortKey*)
- *minimum*: the position of the lower whisker, typically the minimum measurement of the sample that's not considered an outlier.
- *lowerQuartile*: the lower end of the box. The lower and the upper quartiles are the two statistical quartiles around the median of the sample, they should contain 50% of the sample data.
- *median*: the value of the median mark inside the quartile box. The median separates the sample data in half (50% of the sample data is below/above the median). (This is the *mainValue*)
- *upperQuartile*: the upper end of the box. The lower and the upper quartiles are the two statistical quartiles around the median of the sample, they should contain 50% of the sample data.
- *maximum*: the position of the upper whisker, typically the maximum measurement of the sample that's not considered an outlier.
- *outliers*: a `QVector` of outlier values that will be drawn as scatter points at the *key* coordinate of this data point (see [QCPStatisticalBox::setOutlierStyle](#))

The container for storing multiple data points is `QCPStatisticalBoxDataContainer`. It is a typedef for [QCPDataContainer](#) with [QCPStatisticalBoxData](#) as the `DataType` template parameter. See the documentation there for an explanation regarding the data type's generic methods.

See also

`QCPStatisticalBoxDataContainer`

15.621.2 Constructor & Destructor Documentation

15.621.2.1 QCPStatisticalBoxData() [1/2]

```
QCPStatisticalBoxData::QCPStatisticalBoxData ( )
```

Constructs a data point with *key* and all values set to zero.

15.621.2.2 QCPStatisticalBoxData() [2/2]

```
QCPStatisticalBoxData::QCPStatisticalBoxData (
    double key,
    double minimum,
    double lowerQuartile,
    double median,
    double upperQuartile,
    double maximum,
    const QVector< double > & outliers = QVector<double>() )
```

Constructs a data point with the specified *key*, *minimum*, *lowerQuartile*, *median*, *upperQuartile*, *maximum* and optionally a number of *outliers*.

15.621.3 Member Function Documentation

15.621.3.1 fromSortKey()

```
static QCPStatisticalBoxData QCPStatisticalBoxData::fromSortKey (
    double sortKey ) [inline], [static]
```

Returns a data point with the specified *sortKey*. All other members are set to zero.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

15.621.3.2 mainKey()

```
double QCPStatisticalBoxData::mainKey ( ) const [inline]
```

Returns the *key* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

15.621.3.3 mainValue()

```
double QCPStatisticalBoxData::mainValue ( ) const [inline]
```

Returns the *median* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

15.621.3.4 sortKey()

```
double QCPStatisticalBoxData::sortKey ( ) const [inline]
```

Returns the *key* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

15.621.3.5 sortKeyIsMainKey()

```
static static bool QCPStatisticalBoxData::sortKeyIsMainKey ( ) [inline], [static]
```

Since the member *key* is both the data point key coordinate and the data ordering parameter, this method returns true.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

15.621.3.6 valueRange()

```
QCPRange QCPStatisticalBoxData::valueRange ( ) const [inline]
```

Returns a [QCPRange](#) spanning from the *minimum* to the *maximum* member of this statistical box data point, possibly further expanded by outliers.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.622 QCPTextElement Class Reference

A layout element displaying a text.

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

Public Member Functions

- [QCPTextElement](#) ([QCustomPlot](#) *parentPlot)
- [QCPTextElement](#) ([QCustomPlot](#) *parentPlot, const QString &text)
- [QCPTextElement](#) ([QCustomPlot](#) *parentPlot, const QString &text, double pointSize)
- [QCPTextElement](#) ([QCustomPlot](#) *parentPlot, const QString &text, const QString &fontFamily, double pointSize)
- [QCPTextElement](#) ([QCustomPlot](#) *parentPlot, const QString &text, const QFont &font)
- QString **text** () const
- int **textFlags** () const
- QFont **font** () const
- QColor **textColor** () const
- QFont **selectedFont** () const
- QColor **selectedTextColor** () const
- bool **selectable** () const
- bool **selected** () const
- void [setText](#) (const QString &text)
- void [setTextFlags](#) (int flags)
- void [setFont](#) (const QFont &font)
- void [setTextColor](#) (const QColor &color)
- void [setSelectedFont](#) (const QFont &font)
- void [setSelectedTextColor](#) (const QColor &color)
- Q_SLOT void [setSelectable](#) (bool selectable)
- Q_SLOT void [setSelected](#) (bool selected)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPLayoutElement](#)

- [QCPLayoutElement](#) ([QCustomPlot](#) *parentPlot=nullptr)
- [QCPLayout](#) * [layout](#) () const
- QRect **rect** () const
- QRect **outerRect** () const
- QMargins **margins** () const
- QMargins **minimumMargins** () const
- QCP::MarginSides **autoMargins** () const
- QSize **minimumSize** () const
- QSize **maximumSize** () const
- [SizeConstraintRect](#) **sizeConstraintRect** () const
- [QCPMarginGroup](#) * **marginGroup** ([QCP::MarginSide](#) side) const
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **marginGroups** () const
- void [setOuterRect](#) (const QRect &rect)
- void [setMargins](#) (const QMargins &margins)
- void [setMinimumMargins](#) (const QMargins &margins)
- void [setAutoMargins](#) ([QCP::MarginSides](#) sides)
- void [setMinimumSize](#) (const QSize &size)
- void [setMinimumSize](#) (int width, int height)
- void [setMaximumSize](#) (const QSize &size)
- void [setMaximumSize](#) (int width, int height)

- void [setSizeConstraintRect](#) ([SizeConstraintRect](#) constraintRect)
- void [setMarginGroup](#) (QCP::MarginSides sides, [QCPMarginGroup](#) *group)
- virtual void [update](#) ([UpdatePhase](#) phase)
- virtual QList< [QCPLayerable](#) * > [elements](#) (bool recursive) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual QSize [minimumOuterSizeHint](#) () const Q_DECL_OVERRIDE
- virtual QSize [maximumOuterSizeHint](#) () const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- QFont **mainFont** () const
- QColor **mainTextColor** () const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual int **calculateAutoMargin** ([QCP::MarginSide](#) side)
- virtual void **layoutChanged** ()
- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayerable](#)

- virtual [QCP::Interaction](#) **selectionCategory** () const
- virtual QRect **clipRect** () const
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QString **mText**
- int **mTextFlags**
- QFont **mFont**
- QColor **mTextColor**
- QFont **mSelectedFont**
- QColor **mSelectedTextColor**
- QRect **mTextBoundingRect**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from [QCPLayoutElement](#)

- [QCPLayout](#) * **mParentLayout**
- QSize **mMinimumSize**
- QSize **mMaximumSize**
- [SizeConstraintRect](#) **mSizeConstraintRect**
- QRect **mRect**
- QRect **mOuterRect**
- QMargins **mMargins**
- QMargins **mMinimumMargins**
- QCP::MarginSides **mAutoMargins**
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **mMarginGroups**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members**Public Types inherited from [QCPLayoutElement](#)**

- enum [UpdatePhase](#) { [upPreparation](#) , [upMargins](#) , [upLayout](#) }
- enum [SizeConstraintRect](#) { [scrInnerRect](#) , [scrOuterRect](#) }

15.622.1 Detailed Description

A layout element displaying a text.

The text may be specified with [setText](#), the formatting can be controlled with [setFont](#), [setTextColor](#), and [setTextFlags](#).

A text element can be added as follows:

15.622.2 Constructor & Destructor Documentation**15.622.2.1 [QCPTextElement](#)() [1/5]**

```
QCPTextElement::QCPTextElement (
    QCustomPlot * parentPlot ) [explicit]
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Creates a new [QCPTextElement](#) instance and sets default values. The initial text is empty ([setText](#)).

15.622.2.2 QCPTextElement() [2/5]

```
QCPTextElement::QCPTextElement (
    QCustomPlot * parentPlot,
    const QString & text )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Creates a new [QCPTextElement](#) instance and sets default values.

The initial text is set to *text*.

15.622.2.3 QCPTextElement() [3/5]

```
QCPTextElement::QCPTextElement (
    QCustomPlot * parentPlot,
    const QString & text,
    double pointSize )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Creates a new [QCPTextElement](#) instance and sets default values.

The initial text is set to *text* with *pointSize*.

15.622.2.4 QCPTextElement() [4/5]

```
QCPTextElement::QCPTextElement (
    QCustomPlot * parentPlot,
    const QString & text,
    const QString & fontFamily,
    double pointSize )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Creates a new [QCPTextElement](#) instance and sets default values.

The initial text is set to *text* with *pointSize* and the specified *fontFamily*.

15.622.2.5 QCPTextElement() [5/5]

```
QCPTextElement::QCPTextElement (
    QCustomPlot * parentPlot,
    const QString & text,
    const QFont & font )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Creates a new [QCPTextElement](#) instance and sets default values.

The initial text is set to *text* with the specified *font*.

15.622.3 Member Function Documentation**15.622.3.1 applyDefaultAntialiasingHint()**

```
void QCPTextElement::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

15.622.3.2 clicked

```
void QCPTextElement::clicked (
    QMouseEvent * event ) [signal]
```

This signal is emitted when the text element is clicked.

See also

[doubleClicked](#), [selectTest](#)

15.622.3.3 deselectEvent()

```
void QCPTextElement::deselectEvent (
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.622.3.4 doubleClicked

```
void QCPTextElement::doubleClicked (
    QMouseEvent * event ) [signal]
```

This signal is emitted when the text element is double clicked.

See also

[clicked](#), [selectTest](#)

15.622.3.5 draw()

```
void QCPTextElement::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

15.622.3.6 maximumOuterSizeHint()

```
QSize QCPTextElement::maximumOuterSizeHint ( ) const [protected], [virtual]
```

Returns the suggested maximum size this layout element (the [outerRect](#)) may be expanded to, if no manual maximum size is set.

if a maximum size ([setMaximumSize](#)) was not set manually, parent layouts use the returned size (usually indirectly through [QCPLayout::getFinalMaximumOuterSize](#)) to determine the maximum allowed size of this layout element.

A manual maximum size is considered set if it is smaller than Qt's `QWIDGETSIZE_MAX`.

The default implementation simply returns `QWIDGETSIZE_MAX` for both width and height, implying no suggested maximum size. Reimplementations may use their detailed knowledge about the layout element's content to provide size hints.

Reimplemented from [QCPLayoutElement](#).

15.622.3.7 minimumOuterSizeHint()

```
QSize QCPTextElement::minimumOuterSizeHint ( ) const [protected], [virtual]
```

Returns the suggested minimum size this layout element (the [outerRect](#)) may be compressed to, if no manual minimum size is set.

if a minimum size ([setMinimumSize](#)) was not set manually, parent layouts use the returned size (usually indirectly through [QCPLayout::getFinalMinimumOuterSize](#)) to determine the minimum allowed size of this layout element.

A manual minimum size is considered set if it is non-zero.

The default implementation simply returns the sum of the horizontal margins for the width and the sum of the vertical margins for the height. Reimplementations may use their detailed knowledge about the layout element's content to provide size hints.

Reimplemented from [QCPLayoutElement](#).

15.622.3.8 mouseDoubleClickEvent()

```
void QCPTextElement::mouseDoubleClickEvent (
    QMouseEvent * event,
    const QVariant & details ) [virtual]
```

Emits the [doubleClicked](#) signal.

\see [baseclassmethod](#)

Reimplemented from [QCPLayerable](#).

15.622.3.9 mousePressEvent()

```
void QCPTextElement::mousePressEvent (
    QMouseEvent * event,
    const QVariant & details ) [virtual]
```

Accepts the mouse event in order to emit the according click signal in the [mouseReleaseEvent](#).

\seebaseclassmethod

Reimplemented from [QCPLayerable](#).

15.622.3.10 mouseReleaseEvent()

```
void QCPTextElement::mouseReleaseEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [virtual]
```

Emits the [clicked](#) signal if the cursor hasn't moved by more than a few pixels since the [mousePressEvent](#).

\seebaseclassmethod

Reimplemented from [QCPLayerable](#).

15.622.3.11 selectEvent()

```
void QCPTextElement::selectEvent (
    QMouseEvent * event,
    bool additive,
    const QVariant & details,
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

15.622.3.12 selectionChanged

```
void QCPTextElement::selectionChanged (
    bool selected ) [signal]
```

This signal is emitted when the selection state has changed to *selected*, either by user interaction or by a direct call to [setSelected](#).

See also

[setSelected](#), [setSelectable](#)

15.622.3.13 selectTest()

```
double QCPTextElement::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Returns $0.99 * \text{selectionTolerance}$ (see [QCustomPlot::setSelectionTolerance](#)) when *pos* is within the bounding box of the text element's text. Note that this bounding box is updated in the draw call.

If *pos* is outside the text's bounding box or if *onlySelectable* is true and this text element is not selectable ([setSelectable](#)), returns -1.

\seebaseclassmethod

Reimplemented from [QCPLayoutElement](#).

15.622.3.14 setFont()

```
void QCPTextElement::setFont (
    const QFont & font )
```

Sets the *font* of the text.

See also

[setTextColor](#), [setSelectedFont](#)

15.622.3.15 setSelectable()

```
void QCPTextElement::setSelectable (
    bool selectable )
```

Sets whether the user may select this text element.

Note that even when *selectable* is set to `false`, the selection state may be changed programmatically via [setSelected](#).

15.622.3.16 setSelected()

```
void QCPTextElement::setSelected (
    bool selected )
```

Sets the selection state of this text element to *selected*. If the selection has changed, [selectionChanged](#) is emitted.

Note that this function can change the selection state independently of the current [setSelectable](#) state.

15.622.3.17 setSelectedFont()

```
void QCPTextElement::setSelectedFont (
    const QFont & font )
```

Sets the *font* of the text that will be used if the text element is selected ([setSelected](#)).

See also

[setFont](#)

15.622.3.18 setSelectedTextColor()

```
void QCPTextElement::setSelectedTextColor (
    const QColor & color )
```

Sets the *color* of the text that will be used if the text element is selected ([setSelected](#)).

See also

[setTextColor](#)

15.622.3.19 setText()

```
void QCPTextElement::setText (
    const QString & text )
```

Sets the text that will be displayed to *text*. Multiple lines can be created by insertion of `"\n"`.

See also

[setFont](#), [setTextColor](#), [setTextFlags](#)

15.622.3.20 setTextColor()

```
void QCPTextElement::setTextColor (
    const QColor & color )
```

Sets the *color* of the text.

See also

[setFont](#), [setSelectedTextColor](#)

15.622.3.21 setTextFlags()

```
void QCPTextElement::setTextFlags (
    int flags )
```

Sets options for text alignment and wrapping behaviour. *flags* is a bitwise OR-combination of `Qt::Alignment`↔`Flag` and `Qt::TextFlag` enums.

Possible enums are:

- `Qt::AlignLeft`
- `Qt::AlignRight`
- `Qt::AlignHCenter`
- `Qt::AlignJustify`
- `Qt::AlignTop`
- `Qt::AlignBottom`
- `Qt::AlignVCenter`
- `Qt::AlignCenter`
- `Qt::TextDontClip`
- `Qt::TextSingleLine`
- `Qt::TextExpandTabs`
- `Qt::TextShowMnemonic`
- `Qt::TextWordWrap`
- `Qt::TextIncludeTrailingSpaces`

The documentation for this class was generated from the following files:

- `applications/gui/genesys/qcustomplot.h`
- `applications/gui/genesys/qcustomplot.cpp`

15.623 QCPVector2D Class Reference

Represents two doubles as a mathematical 2D vector.

Public Member Functions

- [QCPVector2D](#) ()
- [QCPVector2D](#) (double x, double y)
- [QCPVector2D](#) (const QPoint &point)
- [QCPVector2D](#) (const QPointF &point)
- double **x** () const
- double **y** () const
- double & **rx** ()
- double & **ry** ()
- void [setX](#) (double x)
- void [setY](#) (double y)
- double [length](#) () const
- double [lengthSquared](#) () const
- double [angle](#) () const
- QPoint [toPoint](#) () const
- QPointF [toPointF](#) () const
- bool [isNull](#) () const

- void `normalize` ()
- `QCPVector2D` `normalized` () const
- `QCPVector2D` `perpendicular` () const
- double `dot` (const `QCPVector2D` &vec) const
- double `distanceSquaredToLine` (const `QCPVector2D` &start, const `QCPVector2D` &end) const
- double `distanceSquaredToLine` (const `QLineF` &line) const
- double `distanceToStraightLine` (const `QCPVector2D` &base, const `QCPVector2D` &direction) const
- `QCPVector2D` & `operator*=` (double factor)
- `QCPVector2D` & `operator/=` (double divisor)
- `QCPVector2D` & `operator+=` (const `QCPVector2D` &vector)
- `QCPVector2D` & `operator-=` (const `QCPVector2D` &vector)

Friends

- const `QCPVector2D` `operator*` (double factor, const `QCPVector2D` &vec)
- const `QCPVector2D` `operator*` (const `QCPVector2D` &vec, double factor)
- const `QCPVector2D` `operator/` (const `QCPVector2D` &vec, double divisor)
- const `QCPVector2D` `operator+` (const `QCPVector2D` &vec1, const `QCPVector2D` &vec2)
- const `QCPVector2D` `operator-` (const `QCPVector2D` &vec1, const `QCPVector2D` &vec2)
- const `QCPVector2D` `operator-` (const `QCPVector2D` &vec)

Related Symbols

(Note that these are not member symbols.)

- `QDebug` `operator<<` (`QDebug` d, const `QCPVector2D` &vec)

15.623.1 Detailed Description

Represents two doubles as a mathematical 2D vector.

This class acts as a replacement for `QVector2D` with the advantage of double precision instead of single, and some convenience methods tailored for the `QCustomPlot` library.

15.623.2 Constructor & Destructor Documentation

15.623.2.1 `QCPVector2D()` [1/4]

```
QCPVector2D::QCPVector2D ( )
```

Creates a `QCPVector2D` object and initializes the x and y coordinates to 0.

15.623.2.2 `QCPVector2D()` [2/4]

```
QCPVector2D::QCPVector2D (
    double x,
    double y )
```

Creates a `QCPVector2D` object and initializes the x and y coordinates with the specified values.

15.623.2.3 `QCPVector2D()` [3/4]

```
QCPVector2D::QCPVector2D (
    const QPoint & point )
```

Creates a `QCPVector2D` object and initializes the x and y coordinates respective coordinates of the specified *point*.

15.623.2.4 `QCPVector2D()` [4/4]

```
QCPVector2D::QCPVector2D (
    const QPointF & point )
```

Creates a `QCPVector2D` object and initializes the x and y coordinates respective coordinates of the specified *point*.

15.623.3 Member Function Documentation

15.623.3.1 angle()

```
double QCPVector2D::angle ( ) const [inline]
```

Returns the angle of the vector in radians. The angle is measured between the positive x line and the vector, counter-clockwise in a mathematical coordinate system (y axis upwards positive). In screen/widget coordinates where the y axis is inverted, the angle appears clockwise.

15.623.3.2 distanceSquaredToLine() [1/2]

```
double QCPVector2D::distanceSquaredToLine (
    const QCPVector2D & start,
    const QCPVector2D & end ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns the squared shortest distance of this vector (interpreted as a point) to the finite line segment given by *start* and *end*.

See also

[distanceToStraightLine](#)

15.623.3.3 distanceSquaredToLine() [2/2]

```
double QCPVector2D::distanceSquaredToLine (
    const QLineF & line ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns the squared shortest distance of this vector (interpreted as a point) to the finite line segment given by *line*.

See also

[distanceToStraightLine](#)

15.623.3.4 distanceToStraightLine()

```
double QCPVector2D::distanceToStraightLine (
    const QCPVector2D & base,
    const QCPVector2D & direction ) const
```

Returns the shortest distance of this vector (interpreted as a point) to the infinite straight line given by a *base* point and a *direction* vector.

See also

[distanceSquaredToLine](#)

15.623.3.5 dot()

```
double QCPVector2D::dot (
    const QCPVector2D & vec ) const [inline]
```

Returns the dot/scalar product of this vector with the specified vector *vec*.

15.623.3.6 isNull()

```
bool QCPVector2D::isNull ( ) const [inline]
```

Returns whether this vector is null. A vector is null if `isNull` returns true for both x and y coordinates, i.e. if both are binary equal to 0.

15.623.3.7 length()

```
double QCPVector2D::length ( ) const [inline]
```

Returns the length of this vector.

See also

[lengthSquared](#)

15.623.3.8 lengthSquared()

```
double QCPVector2D::lengthSquared ( ) const [inline]
```

Returns the squared length of this vector. In some situations, e.g. when just trying to find the shortest vector of a group, this is faster than calculating [length](#), because it avoids calculation of a square root.

See also

[length](#)

15.623.3.9 normalize()

```
void QCPVector2D::normalize ( )
```

Normalizes this vector. After this operation, the length of the vector is equal to 1.

If the vector has both entries set to zero, this method does nothing.

See also

[normalized](#), [length](#), [lengthSquared](#)

15.623.3.10 normalized()

```
QCPVector2D QCPVector2D::normalized ( ) const
```

Returns a normalized version of this vector. The length of the returned vector is equal to 1.

If the vector has both entries set to zero, this method returns the vector unmodified.

See also

[normalize](#), [length](#), [lengthSquared](#)

15.623.3.11 operator*=()

```
QCPVector2D & QCPVector2D::operator*= (
    double factor )
```

Scales this vector by the given *factor*, i.e. the x and y components are multiplied by *factor*.

15.623.3.12 operator+=()

```
QCPVector2D & QCPVector2D::operator+= (
    const QCPVector2D & vector )
```

Adds the given *vector* to this vector component-wise.

15.623.3.13 operator-=()

```
QCPVector2D & QCPVector2D::operator-= (
    const QCPVector2D & vector )
```

subtracts the given *vector* from this vector component-wise.

15.623.3.14 operator/=()

```
QCPVector2D & QCPVector2D::operator/= (
    double divisor )
```

Scales this vector by the given *divisor*, i.e. the x and y components are divided by *divisor*.

15.623.3.15 perpendicular()

`QCPVector2D QCPVector2D::perpendicular ( ) const [inline]`

Returns a vector perpendicular to this vector, with the same length.

15.623.3.16 setX()

```
void QCPVector2D::setX (
    double x ) [inline]
```

Sets the x coordinate of this vector to *x*.

See also

[setY](#)

15.623.3.17 setY()

```
void QCPVector2D::setY (
    double y ) [inline]
```

Sets the y coordinate of this vector to *y*.

See also

[setX](#)

15.623.3.18 toPoint()

`QPoint QCPVector2D::toPoint ( ) const [inline]`

Returns a QPoint which has the x and y coordinates of this vector, truncating any floating point information.

See also

[toPointF](#)

15.623.3.19 toPointF()

`QPointF QCPVector2D::toPointF ( ) const [inline]`

Returns a QPointF which has the x and y coordinates of this vector.

See also

[toPoint](#)

15.623.4 Friends And Related Symbol Documentation

15.623.4.1 operator<<()

```
QDebug operator<< (
    QDebug d,
    const QCPVector2D & vec ) [related]
```

Prints *vec* in a human readable format to the qDebug output.

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

- void [mouseRelease](#) (QMouseEvent *event)
- void [mouseWheel](#) (QWheelEvent *event)
- void [plottableClick](#) (QCPAbstractPlottable *plottable, int dataIndex, QMouseEvent *event)
- void [plottableDoubleClick](#) (QCPAbstractPlottable *plottable, int dataIndex, QMouseEvent *event)
- void [itemClick](#) (QCPAbstractItem *item, QMouseEvent *event)
- void [itemDoubleClick](#) (QCPAbstractItem *item, QMouseEvent *event)
- void [axisClick](#) (QCPAxis *axis, QCPAxis::SelectablePart part, QMouseEvent *event)
- void [axisDoubleClick](#) (QCPAxis *axis, QCPAxis::SelectablePart part, QMouseEvent *event)
- void [legendClick](#) (QCPLegend *legend, QCPAbstractLegendItem *item, QMouseEvent *event)
- void [legendDoubleClick](#) (QCPLegend *legend, QCPAbstractLegendItem *item, QMouseEvent *event)
- void [selectionChangedByUser](#) ()
- void [beforeReplot](#) ()
- void [afterLayout](#) ()
- void [afterReplot](#) ()

Public Member Functions

- [QCustomPlot](#) (QWidget *parent=nullptr)
- QRect **viewport** () const
- double **bufferDevicePixelRatio** () const
- QPixmap **background** () const
- bool **backgroundScaled** () const
- Qt::AspectRatioMode **backgroundScaledMode** () const
- QCPLayoutGrid * [plotLayout](#) () const
- QCP::AntialiasedElements **antialiasedElements** () const
- QCP::AntialiasedElements **notAntialiasedElements** () const
- bool **autoAddPlottableToLegend** () const
- const QCP::Interactions **interactions** () const
- int **selectionTolerance** () const
- bool **noAntialiasingOnDrag** () const
- QCP::PlottingHints **plottingHints** () const
- Qt::KeyboardModifier **multiSelectModifier** () const
- QCP::SelectionRectMode **selectionRectMode** () const
- QCPSelectionRect * [selectionRect](#) () const
- bool **openGL** () const
- void [setViewport](#) (const QRect &rect)
- void [setBufferDevicePixelRatio](#) (double ratio)
- void [setBackground](#) (const QPixmap &pm)
- void [setBackground](#) (const QPixmap &pm, bool scaled, Qt::AspectRatioMode mode=Qt::KeepAspectRatio$\leftrightarrow$ ByExpanding)
- void [setBackground](#) (const QBrush &brush)
- void [setBackgroundScaled](#) (bool scaled)
- void [setBackgroundScaledMode](#) (Qt::AspectRatioMode mode)
- void [setAntialiasedElements](#) (const QCP::AntialiasedElements &antialiasedElements)
- void [setAntialiasedElement](#) (QCP::AntialiasedElement antialiasedElement, bool enabled=true)
- void [setNotAntialiasedElements](#) (const QCP::AntialiasedElements ¬AntialiasedElements)
- void [setNotAntialiasedElement](#) (QCP::AntialiasedElement notAntialiasedElement, bool enabled=true)
- void [setAutoAddPlottableToLegend](#) (bool on)
- void [setInteractions](#) (const QCP::Interactions &interactions)
- void [setInteraction](#) (const QCP::Interaction &interaction, bool enabled=true)
- void [setSelectionTolerance](#) (int pixels)
- void [setNoAntialiasingOnDrag](#) (bool enabled)
- void [setPlottingHints](#) (const QCP::PlottingHints &hints)
- void [setPlottingHint](#) (QCP::PlottingHint hint, bool enabled=true)
- void [setMultiSelectModifier](#) (Qt::KeyboardModifier modifier)

- void [setSelectionMode](#) ([QCP::SelectionMode](#) mode)
- void [setSelectionRect](#) ([QCPSelectionRect](#) *selectionRect)
- void [setOpenGL](#) (bool enabled, int multisampling=16)
- [QCPAbstractPlottable](#) * [plottable](#) (int index)
- [QCPAbstractPlottable](#) * [plottable](#) ()
- bool [removePlottable](#) ([QCPAbstractPlottable](#) *plottable)
- bool [removePlottable](#) (int index)
- int [clearPlottables](#) ()
- int [plottableCount](#) () const
- [QList](#)< [QCPAbstractPlottable](#) * > [selectedPlottables](#) () const
- template<class PlottableType >
PlottableType * [plottableAt](#) (const [QPointF](#) &pos, bool onlySelectable=false, int *dataIndex=nullptr) const
- [QCPAbstractPlottable](#) * [plottableAt](#) (const [QPointF](#) &pos, bool onlySelectable=false, int *dataIndex=nullptr) const
- bool [hasPlottable](#) ([QCPAbstractPlottable](#) *plottable) const
- [QCPGraph](#) * [graph](#) (int index) const
- [QCPGraph](#) * [graph](#) () const
- [QCPGraph](#) * [addGraph](#) ([QCPAxis](#) *keyAxis=nullptr, [QCPAxis](#) *valueAxis=nullptr)
- bool [removeGraph](#) ([QCPGraph](#) *graph)
- bool [removeGraph](#) (int index)
- int [clearGraphs](#) ()
- int [graphCount](#) () const
- [QList](#)< [QCPGraph](#) * > [selectedGraphs](#) () const
- [QCPAbstractItem](#) * [item](#) (int index) const
- [QCPAbstractItem](#) * [item](#) () const
- bool [removeItem](#) ([QCPAbstractItem](#) *item)
- bool [removeItem](#) (int index)
- int [clearItems](#) ()
- int [itemCount](#) () const
- [QList](#)< [QCPAbstractItem](#) * > [selectedItems](#) () const
- template<class ItemType >
ItemType * [itemAt](#) (const [QPointF](#) &pos, bool onlySelectable=false) const
- [QCPAbstractItem](#) * [itemAt](#) (const [QPointF](#) &pos, bool onlySelectable=false) const
- bool [hasItem](#) ([QCPAbstractItem](#) *item) const
- [QCPLayer](#) * [layer](#) (const [QString](#) &name) const
- [QCPLayer](#) * [layer](#) (int index) const
- [QCPLayer](#) * [currentLayer](#) () const
- bool [setCurrentLayer](#) (const [QString](#) &name)
- bool [setCurrentLayer](#) ([QCPLayer](#) *layer)
- int [layerCount](#) () const
- bool [addLayer](#) (const [QString](#) &name, [QCPLayer](#) *otherLayer=nullptr, [LayerInsertMode](#) insertMode=[limAbove](#))
- bool [removeLayer](#) ([QCPLayer](#) *layer)
- bool [moveLayer](#) ([QCPLayer](#) *layer, [QCPLayer](#) *otherLayer, [LayerInsertMode](#) insertMode=[limAbove](#))
- int [axisRectCount](#) () const
- [QCPAxisRect](#) * [axisRect](#) (int index=0) const
- [QList](#)< [QCPAxisRect](#) * > [axisRects](#) () const
- [QCPLayoutElement](#) * [layoutElementAt](#) (const [QPointF](#) &pos) const
- [QCPAxisRect](#) * [axisRectAt](#) (const [QPointF](#) &pos) const
- Q_SLOT void [rescaleAxes](#) (bool onlyVisiblePlottables=false)
- [QList](#)< [QCPAxis](#) * > [selectedAxes](#) () const
- [QList](#)< [QCPLegend](#) * > [selectedLegends](#) () const
- Q_SLOT void [deselectAll](#) ()
- bool [savePdf](#) (const [QString](#) &fileName, int width=0, int height=0, [QCP::ExportPen](#) exportPen=[QCP::epAllowCosmetic](#), const [QString](#) &pdfCreator=[QString](#)(), const [QString](#) &pdfTitle=[QString](#)())

- bool [savePng](#) (const QString &fileName, int width=0, int height=0, double scale=1.0, int quality=-1, int resolution=96, [QCP::ResolutionUnit](#) resolutionUnit=[QCP::ruDotsPerInch](#))
- bool [saveJpg](#) (const QString &fileName, int width=0, int height=0, double scale=1.0, int quality=-1, int resolution=96, [QCP::ResolutionUnit](#) resolutionUnit=[QCP::ruDotsPerInch](#))
- bool [saveBmp](#) (const QString &fileName, int width=0, int height=0, double scale=1.0, int resolution=96, [QCP::ResolutionUnit](#) resolutionUnit=[QCP::ruDotsPerInch](#))
- bool [saveRastered](#) (const QString &fileName, int width, int height, double scale, const char *format, int quality=-1, int resolution=96, [QCP::ResolutionUnit](#) resolutionUnit=[QCP::ruDotsPerInch](#))
- QPixmap [toPixmap](#) (int width=0, int height=0, double scale=1.0)
- void [toPainter](#) ([QCPPainter](#) *painter, int width=0, int height=0)
- Q_SLOT void [replot](#) ([QCustomPlot::RefreshPriority](#) refreshPriority=[QCustomPlot::rpRefreshHint](#))
- double [replotTime](#) (bool average=false) const

Public Attributes

- [QCPAxis](#) * [xAxis](#)
- [QCPAxis](#) * [yAxis](#)
- [QCPAxis](#) * [xAxis2](#)
- [QCPAxis](#) * [yAxis2](#)
- [QCPLegend](#) * [legend](#)

Protected Member Functions

- virtual QSize **minimumSizeHint** () const Q_DECL_OVERRIDE
- virtual QSize **sizeHint** () const Q_DECL_OVERRIDE
- virtual void **paintEvent** (QPaintEvent *event) Q_DECL_OVERRIDE
- virtual void **resizeEvent** (QResizeEvent *event) Q_DECL_OVERRIDE
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event) Q_DECL_OVERRIDE
- virtual void **mousePressEvent** (QMouseEvent *event) Q_DECL_OVERRIDE
- virtual void **mouseMoveEvent** (QMouseEvent *event) Q_DECL_OVERRIDE
- virtual void **mouseReleaseEvent** (QMouseEvent *event) Q_DECL_OVERRIDE
- virtual void **wheelEvent** (QWheelEvent *event) Q_DECL_OVERRIDE
- virtual void **draw** ([QCPPainter](#) *painter)
- virtual void **updateLayout** ()
- virtual void **axisRemoved** ([QCPAxis](#) *axis)
- virtual void **legendRemoved** ([QCPLegend](#) *legend)
- virtual Q_SLOT void **processRectSelection** (QRect rect, QMouseEvent *event)
- virtual Q_SLOT void **processRectZoom** (QRect rect, QMouseEvent *event)
- virtual Q_SLOT void **processPointSelection** (QMouseEvent *event)
- bool **registerPlottable** ([QCPAbstractPlottable](#) *plottable)
- bool **registerGraph** ([QCPGraph](#) *graph)
- bool **registerItem** ([QCPAbstractItem](#) *item)
- void **updateLayerIndices** () const
- [QCPLayerable](#) * **layerableAt** (const QPointF &pos, bool onlySelectable, QVariant *selectionDetails=nullptr) const
- QList< [QCPLayerable](#) * > **layerableListAt** (const QPointF &pos, bool onlySelectable, QList< QVariant > *selectionDetails=nullptr) const
- void **drawBackground** ([QCPPainter](#) *painter)
- void **setupPaintBuffers** ()
- [QCPAbstractPaintBuffer](#) * **createPaintBuffer** ()
- bool **hasInvalidatedPaintBuffers** ()
- bool **setupOpenGL** ()
- void **freeOpenGL** ()

Protected Attributes

- QRect **mViewport**
- double **mBufferDevicePixelRatio**
- [QCPLayoutGrid](#) * **mPlotLayout**
- bool **mAutoAddPlottableToLegend**
- QList< [QCPAbstractPlottable](#) * > **mPlottables**
- QList< [QCPGraph](#) * > **mGraphs**
- QList< [QCPAbstractItem](#) * > **mItems**
- QList< [QCPLayer](#) * > **mLayers**
- QCP::AntialiasedElements **mAntialiasedElements**
- QCP::AntialiasedElements **mNotAntialiasedElements**
- QCP::Interactions **mInteractions**
- int **mSelectionTolerance**
- bool **mNoAntialiasingOnDrag**
- QBrush **mBackgroundBrush**
- QPixmap **mBackgroundPixmap**
- QPixmap **mScaledBackgroundPixmap**
- bool **mBackgroundScaled**
- Qt::AspectRatioMode **mBackgroundScaledMode**
- [QCPLayer](#) * **mCurrentLayer**
- QCP::PlottingHints **mPlottingHints**
- Qt::KeyboardModifier **mMultiSelectModifier**
- [QCP::SelectionRectMode](#) **mSelectionRectMode**
- [QCPSelectionRect](#) * **mSelectionRect**
- bool **mOpenGL**
- QList< QSharedPointer< [QCPAbstractPaintBuffer](#) > > **mPaintBuffers**
- QPoint **mMousePressPos**
- bool **mMouseHasMoved**
- QPointer< [QCPLayerable](#) > **mMouseEventLayerable**
- QPointer< [QCPLayerable](#) > **mMouseSignalLayerable**
- QVariant **mMouseEventLayerableDetails**
- QVariant **mMouseSignalLayerableDetails**
- bool **mReplotting**
- bool **mReplotQueued**
- double **mReplotTime**
- double **mReplotTimeAverage**
- int **mOpenGLMultisamples**
- QCP::AntialiasedElements **mOpenGLAntialiasedElementsBackup**
- bool **mOpenGLCacheLabelsBackup**

Friends

- class **QCPLegend**
- class **QCPAxis**
- class **QCPLayer**
- class **QCPAxisRect**
- class **QCPAbstractPlottable**
- class **QCPGraph**
- class **QCPAbstractItem**

15.624.1 Detailed Description

The central class of the library. This is the QWidget which displays the plot and interacts with the user.

For tutorials on how to use [QCustomPlot](#), see the website

<http://www.qcustomplot.com/>

15.624.2 Member Enumeration Documentation

15.624.2.1 LayerInsertMode

enum [QCustomPlot::LayerInsertMode](#)

Defines how a layer should be inserted relative to an other layer.

See also

[addLayer](#), [moveLayer](#)

Enumerator

limBelow	Layer is inserted below other layer.
limAbove	Layer is inserted above other layer.

15.624.2.2 RefreshPriority

enum [QCustomPlot::RefreshPriority](#)

Defines with what timing the [QCustomPlot](#) surface is refreshed after a replot.

See also

[replot](#)

Enumerator

rpImmediateRefresh	Replots immediately and repaints the widget immediately by calling <code>QWidget::repaint()</code> after the replot.
rpQueuedRefresh	Replots immediately, but queues the widget repaint, by calling <code>QWidget::update()</code> after the replot. This way multiple redundant widget repaints can be avoided.
rpRefreshHint	Whether to use immediate or queued refresh depends on whether the plotting hint QCP::phImmediateRefresh is set, see setPlottingHints .
rpQueuedReplot	Queues the entire replot for the next event loop iteration. This way multiple redundant replots can be avoided. The actual replot is then done with rpRefreshHint priority.

15.624.3 Constructor & Destructor Documentation

15.624.3.1 QCustomPlot()

```
QCustomPlot::QCustomPlot (
    QWidget * parent = nullptr ) [explicit]
```

Constructs a [QCustomPlot](#) and sets reasonable default values.

15.624.4 Member Function Documentation

15.624.4.1 addGraph()

```
QCPGraph * QCustomPlot::addGraph (
    QCPAxis * keyAxis = nullptr,
    QCPAxis * valueAxis = nullptr )
```

Creates a new graph inside the plot. If *keyAxis* and *valueAxis* are left unspecified (0), the bottom (xAxis) is used as key and the left (yAxis) is used as value axis. If specified, *keyAxis* and *valueAxis* must reside in this [QCustomPlot](#). *keyAxis* will be used as key axis (typically "x") and *valueAxis* as value axis (typically "y") for the graph.

Returns a pointer to the newly created graph, or `nullptr` if adding the graph failed.

See also

[graph](#), [graphCount](#), [removeGraph](#), [clearGraphs](#)

15.624.4.2 addLayer()

```
bool QCustomPlot::addLayer (
    const QString & name,
    QCPLayer * otherLayer = nullptr,
    QCustomPlot::LayerInsertMode insertMode = limAbove )
```

Adds a new layer to this [QCustomPlot](#) instance. The new layer will have the name *name*, which must be unique. Depending on *insertMode*, it is positioned either below or above *otherLayer*.

Returns true on success, i.e. if there is no other layer named *name* and *otherLayer* is a valid layer inside this [QCustomPlot](#).

If *otherLayer* is 0, the highest layer in the [QCustomPlot](#) will be used.

For an explanation of what layers are in [QCustomPlot](#), see the documentation of [QCPLayer](#).

See also

[layer](#), [moveLayer](#), [removeLayer](#)

15.624.4.3 afterLayout

```
void QCustomPlot::afterLayout ( ) [signal]
```

This signal is emitted immediately after the layout step has been completed, which occurs right before drawing the plot. This is typically during a call to [replot](#), and in such cases this signal is emitted in between the signals [beforeReplot](#) and [afterReplot](#). Unlike those signals however, this signal is also emitted during off-screen painting, such as when calling [toPixmap](#) or [savePdf](#).

The layout step queries all layouts and layout elements in the plot for their proposed size and arranges the objects accordingly as preparation for the subsequent drawing step. Through this signal, you have the opportunity to update certain things in your plot that depend crucially on the exact dimensions/positioning of layout elements such as axes and axis rects.

Warning

However, changing any parameters of this [QCustomPlot](#) instance which would normally affect the layouting (e.g. axis range order of magnitudes, tick label sizes, etc.) will not issue a second run of the layout step. It will propagate directly to the draw step and may cause graphical inconsistencies such as overlapping objects, if sizes or positions have changed.

See also

[updateLayout](#), [beforeReplot](#), [afterReplot](#)

15.624.4.4 afterReplot

```
void QCustomPlot::afterReplot ( ) [signal]
```

This signal is emitted immediately after a replot has taken place (caused by a call to the slot [replot](#)).

It is safe to mutually connect the replot slot with this signal on two [QCustomPlots](#) to make them replot synchronously, it won't cause an infinite recursion.

See also

[replot](#), [beforeReplot](#), [afterLayout](#)

15.624.4.5 axisClick

```
void QCustomPlot::axisClick (
    QCPAxis * axis,
    QCPAxis::SelectablePart part,
    QMouseEvent * event ) [signal]
```

This signal is emitted when an axis is clicked.

event is the mouse event that caused the click, *axis* is the axis that received the click and *part* indicates the part of the axis that was clicked.

See also

[axisDoubleClick](#)

15.624.4.6 axisDoubleClick

```
void QCustomPlot::axisDoubleClick (
    QCPAxis * axis,
    QCPAxis::SelectablePart part,
    QMouseEvent * event ) [signal]
```

This signal is emitted when an axis is double clicked.

event is the mouse event that caused the click, *axis* is the axis that received the click and *part* indicates the part of the axis that was clicked.

See also

[axisClick](#)

15.624.4.7 axisRect()

```
QCPAxisRect * QCustomPlot::axisRect (
    int index = 0 ) const
```

Returns the axis rect with *index*.

Initially, only one axis rect (with index 0) exists in the plot. If multiple axis rects were added, all of them may be accessed with this function in a linear fashion (even when they are nested in a layout hierarchy or inside other axis rects via [QCPAxisRect::insetLayout](#)).

The order of the axis rects is given by the fill order of the [QCPLLayout](#) that is holding them. For example, if the axis rects are in the top level grid layout (accessible via [QCustomPlot::plotLayout](#)), they are ordered from left to right, top to bottom, if the layout's default [setFillOrder](#) of [foColumnsFirst](#) wasn't changed.

If you want to access axis rects by their row and column index, use the layout interface. For example, use [QCPLLayoutGrid::element](#) of the top level grid layout, and `qobject_cast` the returned layout element to [QCPAxisRect](#). (See also [thelayoutsystem](#).)

See also

[axisRectCount](#), [axisRects](#), [QCPLLayoutGrid::setFillOrder](#)

15.624.4.8 axisRectAt()

```
QCPAxisRect * QCustomPlot::axisRectAt (
    const QPointF & pos ) const
```

Returns the layout element of type [QCPAxisRect](#) at pixel position *pos*. This method ignores other layout elements even if they are visually in front of the axis rect (e.g. a [QCPLegend](#)). If there is no axis rect at that position, returns `nullptr`.

Only visible axis rects are used. If [QCPLayoutElement::setVisible](#) on the axis rect itself or on any of its parent elements is set to false, it will not be considered.

See also

[layoutElementAt](#)

15.624.4.9 axisRectCount()

```
int QCustomPlot::axisRectCount ( ) const
```

Returns the number of axis rects in the plot.

All axis rects can be accessed via [QCustomPlot::axisRect\(\)](#).

Initially, only one axis rect exists in the plot.

See also

[axisRect](#), [axisRects](#)

15.624.4.10 axisRects()

```
QList< QCPAxisRect * > QCustomPlot::axisRects ( ) const
```

Returns all axis rects in the plot.

The order of the axis rects is given by the fill order of the [QCPLLayout](#) that is holding them. For example, if the axis rects are in the top level grid layout (accessible via [QCustomPlot::plotLayout](#)), they are ordered from left to right, top to bottom, if the layout's default [setFillOrder](#) of [foColumnsFirst](#) wasn't changed.

See also

[axisRectCount](#), [axisRect](#), [QCPLLayoutGrid::setFillOrder](#)

15.624.4.11 beforeReplot

```
void QCustomPlot::beforeReplot ( ) [signal]
```

This signal is emitted immediately before a replot takes place (caused by a call to the slot [replot](#)).

It is safe to mutually connect the replot slot with this signal on two QCustomPlots to make them replot synchronously, it won't cause an infinite recursion.

See also

[replot](#), [afterReplot](#), [afterLayout](#)

15.624.4.12 clearGraphs()

```
int QCustomPlot::clearGraphs ( )
```

Removes all graphs from the plot and deletes them. Corresponding legend items are also removed from the default legend ([QCustomPlot::legend](#)).

Returns the number of graphs removed.

See also

[removeGraph](#)

15.624.4.13 clearItems()

```
int QCustomPlot::clearItems ( )
```

Removes all items from the plot and deletes them.

Returns the number of items removed.

See also

[removeItem](#)

15.624.4.14 clearPlottables()

```
int QCustomPlot::clearPlottables ( )
```

Removes all plottables from the plot and deletes them. Corresponding legend items are also removed from the default legend ([QCustomPlot::legend](#)).

Returns the number of plottables removed.

See also

[removePlottable](#)

15.624.4.15 `currentLayer()`

```
QCPLayer * QCustomPlot::currentLayer ( ) const
```

Returns the layer that is set as current layer (see [setCurrentLayer](#)).

15.624.4.16 `deselectAll()`

```
void QCustomPlot::deselectAll ( )
```

Deselects all layerables (plottables, items, axes, legends,...) of the [QCustomPlot](#).

Since calling this function is not a user interaction, this does not emit the [selectionChangedByUser](#) signal. The individual selectionChanged signals are emitted though, if the objects were previously selected.

See also

[setInteractions](#), [selectedPlottables](#), [selectedItems](#), [selectedAxes](#), [selectedLegends](#)

15.624.4.17 `graph()` [1/2]

```
QCPGraph * QCustomPlot::graph ( ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns the last graph, that was created with [addGraph](#). If there are no graphs in the plot, returns `nullptr`.

See also

[graphCount](#), [addGraph](#)

15.624.4.18 `graph()` [2/2]

```
QCPGraph * QCustomPlot::graph (
    int index ) const
```

Returns the graph with *index*. If the index is invalid, returns `nullptr`.

There is an overloaded version of this function with no parameter which returns the last created graph, see [QCustomPlot::graph\(\)](#)

See also

[graphCount](#), [addGraph](#)

15.624.4.19 `graphCount()`

```
int QCustomPlot::graphCount ( ) const
```

Returns the number of currently existing graphs in the plot

See also

[graph](#), [addGraph](#)

15.624.4.20 `hasInvalidatedPaintBuffers()`

```
bool QCustomPlot::hasInvalidatedPaintBuffers ( ) [protected]
```

This method returns whether any of the paint buffers held by this [QCustomPlot](#) instance are invalidated.

If any buffer is invalidated, a partial replot ([QCPLayer::replot](#)) is not allowed and always causes a full replot ([QCustomPlot::replot](#)) of all layers. This is the case when for example the layer order has changed, new layers were added or removed, layer modes were changed ([QCPLayer::setMode](#)), or layerables were added or removed.

See also

[QCPAbstractPaintBuffer::setInvalidated](#)

15.624.4.21 hasItem()

```
bool QCustomPlot::hasItem (
    QCPAbstractItem * item ) const
```

Returns whether this [QCustomPlot](#) contains the *item*.

See also

[item](#)

15.624.4.22 hasPlottable()

```
bool QCustomPlot::hasPlottable (
    QCPAbstractPlottable * plottable ) const
```

Returns whether this [QCustomPlot](#) instance contains the *plottable*.

15.624.4.23 item() [1/2]

```
QCPAbstractItem * QCustomPlot::item ( ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns the last item that was added to this plot. If there are no items in the plot, returns `nullptr`.

See also

[itemCount](#)

15.624.4.24 item() [2/2]

```
QCPAbstractItem * QCustomPlot::item (
    int index ) const
```

Returns the item with *index*. If the index is invalid, returns `nullptr`.

There is an overloaded version of this function with no parameter which returns the last added item, see [QCustomPlot::item\(\)](#)

See also

[itemCount](#)

15.624.4.25 itemAt() [1/2]

```
template<class ItemType >
ItemType * QCustomPlot::itemAt (
    const QPointF & pos,
    bool onlySelectable = false ) const
```

Returns the item at the pixel position *pos*. The item type (a [QCPAbstractItem](#) subclass) that shall be taken into consideration can be specified via the template parameter. Items that only consist of single lines (e.g. [QCPLItemLine](#) or [QCPLItemCurve](#)) have a tolerance band around them, see [setSelectionTolerance](#). If multiple items come into consideration, the one closest to *pos* is returned.

If *onlySelectable* is true, only items that are selectable ([QCPAbstractItem::setSelectable](#)) are considered.

If there is no item at *pos*, returns `nullptr`.

See also

[plottableAt](#), [layoutElementAt](#)

15.624.4.26 itemAt() [2/2]

```
QCPAbstractItem * QCustomPlot::itemAt (
    const QPointF & pos,
    bool onlySelectable = false ) const
```

Returns the item at the pixel position *pos*. Since it can capture all items, the return type is the abstract base class of all items, [QCPAbstractItem](#).

For details, and if you wish to specify a certain item type (e.g. [QCPItemLine](#)), see the template method [itemAt<ItemType>\(\)](#)

See also

[itemAt<ItemType>\(\)](#), [plottableAt](#), [layoutElementAt](#)

15.624.4.27 itemClick

```
void QCustomPlot::itemClick (
    QCPAbstractItem * item,
    QMouseEvent * event ) [signal]
```

This signal is emitted when an item is clicked.

event is the mouse event that caused the click and *item* is the item that received the click.

See also

[itemDoubleClick](#)

15.624.4.28 itemCount()

```
int QCustomPlot::itemCount ( ) const
```

Returns the number of currently existing items in the plot

See also

[item](#)

15.624.4.29 itemDoubleClick

```
void QCustomPlot::itemDoubleClick (
    QCPAbstractItem * item,
    QMouseEvent * event ) [signal]
```

This signal is emitted when an item is double clicked.

event is the mouse event that caused the click and *item* is the item that received the click.

See also

[itemClick](#)

15.624.4.30 layer() [1/2]

```
QCPLayer * QCustomPlot::layer (
    const QString & name ) const
```

Returns the layer with the specified *name*. If there is no layer with the specified name, `nullptr` is returned. Layer names are case-sensitive.

See also

[addLayer](#), [moveLayer](#), [removeLayer](#)

15.624.4.31 layer() [2/2]

```
QCPLayer * QCustomPlot::layer (
    int index ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns the layer by *index*. If the index is invalid, `nullptr` is returned.

See also

[addLayer](#), [moveLayer](#), [removeLayer](#)

15.624.4.32 layerCount()

```
int QCustomPlot::layerCount ( ) const
```

Returns the number of currently existing layers in the plot

See also

[layer](#), [addLayer](#)

15.624.4.33 layoutElementAt()

```
QCPLayoutElement * QCustomPlot::layoutElementAt (
    const QPointF & pos ) const
```

Returns the layout element at pixel position *pos*. If there is no element at that position, returns `nullptr`.

Only visible elements are used. If [QCPLayoutElement::setVisible](#) on the element itself or on any of its parent elements is set to false, it will not be considered.

See also

[itemAt](#), [plottableAt](#)

15.624.4.34 legendClick

```
void QCustomPlot::legendClick (
    QCPLegend * legend,
    QCPLAbstractLegendItem * item,
    QMouseEvent * event ) [signal]
```

This signal is emitted when a legend (item) is clicked.

event is the mouse event that caused the click, *legend* is the legend that received the click and *item* is the legend item that received the click. If only the legend and no item is clicked, *item* is `nullptr`. This happens for a click inside the legend padding or the space between two items.

See also

[legendDoubleClick](#)

15.624.4.35 legendDoubleClick

```
void QCustomPlot::legendDoubleClick (
    QCPLegend * legend,
    QCPLAbstractLegendItem * item,
    QMouseEvent * event ) [signal]
```

This signal is emitted when a legend (item) is double clicked.

event is the mouse event that caused the click, *legend* is the legend that received the click and *item* is the legend item that received the click. If only the legend and no item is clicked, *item* is `nullptr`. This happens for a click inside the legend padding or the space between two items.

See also

[legendClick](#)

15.624.4.36 mouseClicked

```
void QCustomPlot::mouseDoubleClick (
    QMouseEvent * event ) [signal]
```

This signal is emitted when the [QCustomPlot](#) receives a mouse double click event.

15.624.4.37 mouseMove

```
void QCustomPlot::mouseMove (
    QMouseEvent * event ) [signal]
```

This signal is emitted when the [QCustomPlot](#) receives a mouse move event.

It is emitted before [QCustomPlot](#) handles any other mechanism like range dragging. So a slot connected to this signal can still influence the behaviour e.g. with [QCPAxisRect::setRangeDrag](#) or [QCPAxisRect::setRangeDragAxes](#).

Warning

It is discouraged to change the drag-axes with [QCPAxisRect::setRangeDragAxes](#) here, because the dragging starting point was saved the moment the mouse was pressed. Thus it only has a meaning for the range drag axes that were set at that moment. If you want to change the drag axes, consider doing this in the [mousePress](#) signal instead.

15.624.4.38 mousePress

```
void QCustomPlot::mousePress (
    QMouseEvent * event ) [signal]
```

This signal is emitted when the [QCustomPlot](#) receives a mouse press event.

It is emitted before [QCustomPlot](#) handles any other mechanism like range dragging. So a slot connected to this signal can still influence the behaviour e.g. with [QCPAxisRect::setRangeDrag](#) or [QCPAxisRect::setRangeDragAxes](#).

15.624.4.39 mouseRelease

```
void QCustomPlot::mouseRelease (
    QMouseEvent * event ) [signal]
```

This signal is emitted when the [QCustomPlot](#) receives a mouse release event.

It is emitted before [QCustomPlot](#) handles any other mechanisms like object selection. So a slot connected to this signal can still influence the behaviour e.g. with [setInteractions](#) or [QCPAbstractPlottable::setSelectable](#).

15.624.4.40 mouseWheel

```
void QCustomPlot::mouseWheel (
    QWheelEvent * event ) [signal]
```

This signal is emitted when the [QCustomPlot](#) receives a mouse wheel event.

It is emitted before [QCustomPlot](#) handles any other mechanisms like range zooming. So a slot connected to this signal can still influence the behaviour e.g. with [QCPAxisRect::setRangeZoom](#), [QCPAxisRect::setRangeZoomAxes](#) or [QCPAxisRect::setRangeZoomFactor](#).

15.624.4.41 moveLayer()

```
bool QCustomPlot::moveLayer (
    QCPLayer * layer,
    QCPLayer * otherLayer,
    QCustomPlot::LayerInsertMode insertMode = limAbove )
```

Moves the specified *layer* either above or below *otherLayer*. Whether it's placed above or below is controlled with *insertMode*.

Returns true on success, i.e. when both *layer* and *otherLayer* are valid layers in the [QCustomPlot](#).

See also

[layer](#), [addLayer](#), [moveLayer](#)

15.624.4.42 plotLayout()

```
QCPLayoutGrid * QCustomPlot::plotLayout ( ) const [inline]
```

Returns the top level layout of this [QCustomPlot](#) instance. It is a [QCPLayoutGrid](#), initially containing just one cell with the main [QCPAxisRect](#) inside.

15.624.4.43 plottable() [1/2]

```
QCPAbstractPlottable * QCustomPlot::plottable ( )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns the last plottable that was added to the plot. If there are no plottables in the plot, returns `nullptr`.

See also

[plottableCount](#)

15.624.4.44 plottable() [2/2]

```
QCPAbstractPlottable * QCustomPlot::plottable (
    int index )
```

Returns the plottable with *index*. If the index is invalid, returns `nullptr`.

There is an overloaded version of this function with no parameter which returns the last added plottable, see [QCustomPlot::plottable\(\)](#)

See also

[plottableCount](#)

15.624.4.45 plottableAt() [1/2]

```
template<class PlottableType >
PlottableType * QCustomPlot::plottableAt (
    const QPointF & pos,
    bool onlySelectable = false,
    int * dataIndex = nullptr ) const
```

Returns the plottable at the pixel position *pos*. The plottable type (a [QCPAbstractPlottable](#) subclass) that shall be taken into consideration can be specified via the template parameter.

Plottables that only consist of single lines (like graphs) have a tolerance band around them, see [setSelectionTolerance](#).

If multiple plottables come into consideration, the one closest to *pos* is returned.

If *onlySelectable* is true, only plottables that are selectable ([QCPAbstractPlottable::setSelectable](#)) are considered.

If *dataIndex* is non-null, it is set to the index of the plottable's data point that is closest to *pos*.

If there is no plottable of the specified type at *pos*, returns `nullptr`.

See also

[itemAt](#), [layoutElementAt](#)

15.624.4.46 plottableAt() [2/2]

```
QCPAbstractPlottable * QCustomPlot::plottableAt (
    const QPointF & pos,
    bool onlySelectable = false,
    int * dataIndex = nullptr ) const
```

Returns any plottable at the pixel position *pos*. Since it can capture all plottables, the return type is the abstract base class of all plottables, [QCPAbstractPlottable](#).

For details, and if you wish to specify a certain plottable type (e.g. [QCPGraph](#)), see the template method [plottableAt<PlottableType>\(\)](#)

See also

[plottableAt<PlottableType>\(\)](#), [itemAt](#), [layoutElementAt](#)

15.624.4.47 `plottableClick`

```
void QCustomPlot::plottableClick (
    QCPAbstractPlottable * plottable,
    int dataIndex,
    QMouseEvent * event ) [signal]
```

This signal is emitted when a plottable is clicked.

event is the mouse event that caused the click and *plottable* is the plottable that received the click. The parameter *dataIndex* indicates the data point that was closest to the click position.

See also

[plottableDoubleClick](#)

15.624.4.48 `plottableCount()`

```
int QCustomPlot::plottableCount ( ) const
```

Returns the number of currently existing plottables in the plot

See also

[plottable](#)

15.624.4.49 `plottableDoubleClick`

```
void QCustomPlot::plottableDoubleClick (
    QCPAbstractPlottable * plottable,
    int dataIndex,
    QMouseEvent * event ) [signal]
```

This signal is emitted when a plottable is double clicked.

event is the mouse event that caused the click and *plottable* is the plottable that received the click. The parameter *dataIndex* indicates the data point that was closest to the click position.

See also

[plottableClick](#)

15.624.4.50 `removeGraph()` [1/2]

```
bool QCustomPlot::removeGraph (
    int index )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Removes and deletes the graph by its *index*.

15.624.4.51 `removeGraph()` [2/2]

```
bool QCustomPlot::removeGraph (
    QCPGraph * graph )
```

Removes the specified *graph* from the plot and deletes it. If necessary, the corresponding legend item is also removed from the default legend ([QCustomPlot::legend](#)). If any other graphs in the plot have a channel fill set towards the removed graph, the channel fill property of those graphs is reset to `nullptr` (no channel fill).

Returns true on success.

See also

[clearGraphs](#)

15.624.4.52 removeItem() [1/2]

```
bool QCustomPlot::removeItem (
    int index )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Removes and deletes the item by its *index*.

15.624.4.53 removeItem() [2/2]

```
bool QCustomPlot::removeItem (
    QCPAbstractItem * item )
```

Removes the specified item from the plot and deletes it.

Returns true on success.

See also

[clearItems](#)

15.624.4.54 removeLayer()

```
bool QCustomPlot::removeLayer (
    QCPLayer * layer )
```

Removes the specified *layer* and returns true on success.

All layerables (e.g. plottables and items) on the removed layer will be moved to the layer below *layer*. If *layer* is the bottom layer, the layerables are moved to the layer above. In both cases, the total rendering order of all layerables in the [QCustomPlot](#) is preserved.

If *layer* is the current layer ([setCurrentLayer](#)), the layer below (or above, if bottom layer) becomes the new current layer.

It is not possible to remove the last layer of the plot.

See also

[layer](#), [addLayer](#), [moveLayer](#)

15.624.4.55 removePlottable() [1/2]

```
bool QCustomPlot::removePlottable (
    int index )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Removes and deletes the plottable by its *index*.

15.624.4.56 removePlottable() [2/2]

```
bool QCustomPlot::removePlottable (
    QCPAbstractPlottable * plottable )
```

Removes the specified plottable from the plot and deletes it. If necessary, the corresponding legend item is also removed from the default legend ([QCustomPlot::legend](#)).

Returns true on success.

See also

[clearPlottables](#)

15.624.4.57 replot()

```
void QCustomPlot::replot (
    QCustomPlot::RefreshPriority refreshPriority = QCustomPlot::rpRefreshHint )
```

Causes a complete replot into the internal paint buffer(s). Finally, the widget surface is refreshed with the new buffer contents. This is the method that must be called to make changes to the plot, e.g. on the axis ranges or data points of graphs, visible.

The parameter *refreshPriority* can be used to fine-tune the timing of the replot. For example if your application calls [replot](#) very quickly in succession (e.g. multiple independent functions change some aspects of the plot and each wants to make sure the change gets replotted), it is advisable to set *refreshPriority* to [QCustomPlot::rpQueuedReplot](#). This way, the actual replotting is deferred to the next event loop iteration. Multiple successive calls of [replot](#) with this priority will only cause a single replot, avoiding redundant replots and improving performance.

Under a few circumstances, [QCustomPlot](#) causes a replot by itself. Those are resize events of the [QCustomPlot](#) widget and user interactions (object selection and range dragging/zooming).

Before the replot happens, the signal [beforeReplot](#) is emitted. After the replot, [afterReplot](#) is emitted. It is safe to mutually connect the replot slot with any of those two signals on two [QCustomPlots](#) to make them replot synchronously, it won't cause an infinite recursion.

If a layer is in mode [QCPLayer::lmBuffered](#) ([QCPLayer::setMode](#)), it is also possible to replot only that specific layer via [QCPLayer::replot](#). See the documentation there for details.

See also

[replotTime](#)

15.624.4.58 replotTime()

```
double QCustomPlot::replotTime (
    bool average = false ) const
```

Returns the time in milliseconds that the last replot took. If *average* is set to true, an exponential moving average over the last couple of replots is returned.

See also

[replot](#)

15.624.4.59 rescaleAxes()

```
void QCustomPlot::rescaleAxes (
    bool onlyVisiblePlottables = false )
```

Rescales the axes such that all plottables (like graphs) in the plot are fully visible.

if *onlyVisiblePlottables* is set to true, only the plottables that have their visibility set to true ([QCPLayerable::setVisible](#)), will be used to rescale the axes.

See also

[QCPAbstractPlottable::rescaleAxes](#), [QCPAxis::rescale](#)

15.624.4.60 saveBmp()

```
bool QCustomPlot::saveBmp (
    const QString & fileName,
    int width = 0,
    int height = 0,
    double scale = 1.0,
    int resolution = 96,
    QCP::ResolutionUnit resolutionUnit = QCP::ruDotsPerInch )
```

Saves a BMP image file to *fileName* on disc. The output plot will have the dimensions *width* and *height* in pixels, multiplied by *scale*. If either *width* or *height* is zero, the current width and height of the [QCustomPlot](#) widget is used instead. Line widths and texts etc. are not scaled up when larger widths/heights are used. If you want that effect, use the *scale* parameter.

For example, if you set both *width* and *height* to 100 and *scale* to 2, you will end up with an image file of size 200*200 in which all graphical elements are scaled up by factor 2 (line widths, texts, etc.). This scaling is not done by stretching a 100*100 image, the result will have full 200*200 pixel resolution.

If you use a high scaling factor, it is recommended to enable antialiasing for all elements by temporarily setting [QCustomPlot::setAntialiasedElements](#) to [QCP::aeAll](#) as this allows [QCustomPlot](#) to place objects with sub-pixel accuracy.

The *resolution* will be written to the image file header and has no direct consequence for the quality or the pixel size. However, if opening the image with a tool which respects the metadata, it will be able to scale the image to match either a given size in real units of length (inch, centimeters, etc.), or the target display DPI. You can specify in which units *resolution* is given, by setting *resolutionUnit*. The *resolution* is converted to the format's expected resolution unit internally.

Returns true on success. If this function fails, most likely the BMP format isn't supported by the system, see Qt docs about [QImageWriter::supportedImageFormats\(\)](#).

The objects of the plot will appear in the current selection state. If you don't want any selected objects to be painted in their selected look, deselect everything with [deselectAll](#) before calling this function.

Warning

If calling this function inside the constructor of the parent of the [QCustomPlot](#) widget (i.e. the [MainWindow](#) constructor, if [QCustomPlot](#) is inside the [MainWindow](#)), always provide explicit non-zero widths and heights. If you leave *width* or *height* as 0 (default), this function uses the current width and height of the [QCustomPlot](#) widget. However, in Qt, these aren't defined yet inside the constructor, so you would get an image that has strange widths/heights.

See also

[savePdf](#), [savePng](#), [saveJpg](#), [saveRastered](#)

15.624.4.61 [saveJpg\(\)](#)

```
bool QCustomPlot::saveJpg (
    const QString & fileName,
    int width = 0,
    int height = 0,
    double scale = 1.0,
    int quality = -1,
    int resolution = 96,
    QCP::ResolutionUnit resolutionUnit = QCP::ruDotsPerInch )
```

Saves a JPEG image file to *fileName* on disc. The output plot will have the dimensions *width* and *height* in pixels, multiplied by *scale*. If either *width* or *height* is zero, the current width and height of the [QCustomPlot](#) widget is used instead. Line widths and texts etc. are not scaled up when larger widths/heights are used. If you want that effect, use the *scale* parameter.

For example, if you set both *width* and *height* to 100 and *scale* to 2, you will end up with an image file of size 200*200 in which all graphical elements are scaled up by factor 2 (line widths, texts, etc.). This scaling is not done by stretching a 100*100 image, the result will have full 200*200 pixel resolution.

If you use a high scaling factor, it is recommended to enable antialiasing for all elements by temporarily setting [QCustomPlot::setAntialiasedElements](#) to [QCP::aeAll](#) as this allows [QCustomPlot](#) to place objects with sub-pixel accuracy.

image compression can be controlled with the *quality* parameter which must be between 0 and 100 or -1 to use the default setting.

The *resolution* will be written to the image file header and has no direct consequence for the quality or the pixel size. However, if opening the image with a tool which respects the metadata, it will be able to scale the image to match either a given size in real units of length (inch, centimeters, etc.), or the target display DPI. You can specify in which units *resolution* is given, by setting *resolutionUnit*. The *resolution* is converted to the format's expected resolution unit internally.

Returns true on success. If this function fails, most likely the JPEG format isn't supported by the system, see Qt docs about [QImageWriter::supportedImageFormats\(\)](#).

The objects of the plot will appear in the current selection state. If you don't want any selected objects to be painted in their selected look, deselect everything with [deselectAll](#) before calling this function.

Warning

If calling this function inside the constructor of the parent of the [QCustomPlot](#) widget (i.e. the [MainWindow](#) constructor, if [QCustomPlot](#) is inside the [MainWindow](#)), always provide explicit non-zero widths and heights. If you leave *width* or *height* as 0 (default), this function uses the current width and height of the [QCustomPlot](#) widget. However, in Qt, these aren't defined yet inside the constructor, so you would get an image that has strange widths/heights.

See also

[savePdf](#), [savePng](#), [saveBmp](#), [saveRastered](#)

15.624.4.62 savePdf()

```
bool QCustomPlot::savePdf (
    const QString & fileName,
    int width = 0,
    int height = 0,
    QCP::ExportPen exportPen = QCP::epAllowCosmetic,
    const QString & pdfCreator = QString(),
    const QString & pdfTitle = QString() )
```

Saves a PDF with the vectorized plot to the file *fileName*. The axis ratio as well as the scale of texts and lines will be derived from the specified *width* and *height*. This means, the output will look like the normal on-screen output of a [QCustomPlot](#) widget with the corresponding pixel width and height. If either *width* or *height* is zero, the exported image will have the same dimensions as the [QCustomPlot](#) widget currently has.

Setting *exportPen* to [QCP::epNoCosmetic](#) allows to disable the use of cosmetic pens when drawing to the PDF file. Cosmetic pens are pens with numerical width 0, which are always drawn as a one pixel wide line, no matter what zoom factor is set in the PDF-Viewer. For more information about cosmetic pens, see the QPainter and QPen documentation.

The objects of the plot will appear in the current selection state. If you don't want any selected objects to be painted in their selected look, deselect everything with [deselectAll](#) before calling this function.

Returns true on success.

Warning

- If you plan on editing the exported PDF file with a vector graphics editor like Inkscape, it is advised to set *exportPen* to [QCP::epNoCosmetic](#) to avoid losing those cosmetic lines (which might be quite many, because cosmetic pens are the default for e.g. axes and tick marks).
- If calling this function inside the constructor of the parent of the [QCustomPlot](#) widget (i.e. the [MainWindow](#) constructor, if [QCustomPlot](#) is inside the [MainWindow](#)), always provide explicit non-zero widths and heights. If you leave *width* or *height* as 0 (default), this function uses the current width and height of the [QCustomPlot](#) widget. However, in Qt, these aren't defined yet inside the constructor, so you would get an image that has strange widths/heights.

pdfCreator and *pdfTitle* may be used to set the according metadata fields in the resulting PDF file.

Note

On Android systems, this method does nothing and issues an according qDebug warning message. This is also the case if for other reasons the define flag `QT_NO_PRINTER` is set.

See also

[savePng](#), [saveBmp](#), [saveJpg](#), [saveRastered](#)

15.624.4.63 savePng()

```
bool QCustomPlot::savePng (
    const QString & fileName,
```

```

int width = 0,
int height = 0,
double scale = 1.0,
int quality = -1,
int resolution = 96,
QCP::ResolutionUnit resolutionUnit = QCP::ruDotsPerInch )

```

Saves a PNG image file to *fileName* on disc. The output plot will have the dimensions *width* and *height* in pixels, multiplied by *scale*. If either *width* or *height* is zero, the current width and height of the [QCustomPlot](#) widget is used instead. Line widths and texts etc. are not scaled up when larger widths/heights are used. If you want that effect, use the *scale* parameter.

For example, if you set both *width* and *height* to 100 and *scale* to 2, you will end up with an image file of size 200*200 in which all graphical elements are scaled up by factor 2 (line widths, texts, etc.). This scaling is not done by stretching a 100*100 image, the result will have full 200*200 pixel resolution.

If you use a high scaling factor, it is recommended to enable antialiasing for all elements by temporarily setting [QCustomPlot::setAntialiasedElements](#) to [QCP::aeAll](#) as this allows [QCustomPlot](#) to place objects with sub-pixel accuracy.

image compression can be controlled with the *quality* parameter which must be between 0 and 100 or -1 to use the default setting.

The *resolution* will be written to the image file header and has no direct consequence for the quality or the pixel size. However, if opening the image with a tool which respects the metadata, it will be able to scale the image to match either a given size in real units of length (inch, centimeters, etc.), or the target display DPI. You can specify in which units *resolution* is given, by setting *resolutionUnit*. The *resolution* is converted to the format's expected resolution unit internally.

Returns true on success. If this function fails, most likely the PNG format isn't supported by the system, see Qt docs about [QImageWriter::supportedImageFormats\(\)](#).

The objects of the plot will appear in the current selection state. If you don't want any selected objects to be painted in their selected look, deselect everything with [deselectAll](#) before calling this function.

If you want the PNG to have a transparent background, call [setBackground\(const QBrush &brush\)](#) with no brush ([Qt::NoBrush](#)) or a transparent color ([Qt::transparent](#)), before saving.

Warning

If calling this function inside the constructor of the parent of the [QCustomPlot](#) widget (i.e. the [MainWindow](#) constructor, if [QCustomPlot](#) is inside the [MainWindow](#)), always provide explicit non-zero widths and heights. If you leave *width* or *height* as 0 (default), this function uses the current width and height of the [QCustomPlot](#) widget. However, in Qt, these aren't defined yet inside the constructor, so you would get an image that has strange widths/heights.

See also

[savePdf](#), [saveBmp](#), [saveJpg](#), [saveRastered](#)

15.624.4.64 saveRastered()

```

bool QCustomPlot::saveRastered (
    const QString & fileName,
    int width,
    int height,
    double scale,
    const char * format,
    int quality = -1,
    int resolution = 96,
    QCP::ResolutionUnit resolutionUnit = QCP::ruDotsPerInch )

```

Saves the plot to a rastered image file *fileName* in the image format *format*. The plot is sized to *width* and *height* in pixels and scaled with *scale*. (width 100 and scale 2.0 lead to a full resolution file with width 200.) If the *format* supports compression, *quality* may be between 0 and 100 to control it.

Returns true on success. If this function fails, most likely the given *format* isn't supported by the system, see Qt docs about [QImageWriter::supportedImageFormats\(\)](#).

The *resolution* will be written to the image file header (if the file format supports this) and has no direct consequence for the quality or the pixel size. However, if opening the image with a tool which respects the metadata, it will be able to scale the image to match either a given size in real units of length (inch, centimeters, etc.), or the target display DPI. You can specify in which units *resolution* is given, by setting *resolutionUnit*. The *resolution* is converted to the format's expected resolution unit internally.

See also

[saveBmp](#), [saveJpg](#), [savePng](#), [savePdf](#)

15.624.4.65 selectedAxes()

```
QList< QCPAxis * > QCustomPlot::selectedAxes ( ) const
```

Returns the axes that currently have selected parts, i.e. whose selection state is not [QCPAxis::spNone](#).

See also

[selectedPlottables](#), [selectedLegends](#), [setInteractions](#), [QCPAxis::setSelectedParts](#), [QCPAxis::setSelectableParts](#)

15.624.4.66 selectedGraphs()

```
QList< QCPGraph * > QCustomPlot::selectedGraphs ( ) const
```

Returns a list of the selected graphs. If no graphs are currently selected, the list is empty.

If you are not only interested in selected graphs but other plottables like [QCPCurve](#), [QCPBars](#), etc., use [selectedPlottables](#).

See also

[setInteractions](#), [selectedPlottables](#), [QCPAbstractPlottable::setSelectable](#), [QCPAbstractPlottable::setSelection](#)

15.624.4.67 selectedItems()

```
QList< QCPAbstractItem * > QCustomPlot::selectedItems ( ) const
```

Returns a list of the selected items. If no items are currently selected, the list is empty.

See also

[setInteractions](#), [QCPAbstractItem::setSelectable](#), [QCPAbstractItem::setSelected](#)

15.624.4.68 selectedLegends()

```
QList< QCPLegend * > QCustomPlot::selectedLegends ( ) const
```

Returns the legends that currently have selected parts, i.e. whose selection state is not [QCPLegend::spNone](#).

See also

[selectedPlottables](#), [selectedAxes](#), [setInteractions](#), [QCPLegend::setSelectedParts](#), [QCPLegend::setSelectableParts](#), [QCPLegend::selectedItems](#)

15.624.4.69 selectedPlottables()

```
QList< QCPAbstractPlottable * > QCustomPlot::selectedPlottables ( ) const
```

Returns a list of the selected plottables. If no plottables are currently selected, the list is empty.

There is a convenience function if you're only interested in selected graphs, see [selectedGraphs](#).

See also

[setInteractions](#), [QCPAbstractPlottable::setSelectable](#), [QCPAbstractPlottable::setSelection](#)

15.624.4.70 selectionChangedByUser

```
void QCustomPlot::selectionChangedByUser ( ) [signal]
```

This signal is emitted after the user has changed the selection in the [QCustomPlot](#), e.g. by clicking. It is not emitted when the selection state of an object has changed programmatically by a direct call to `setSelected()/setSelection()` on an object or by calling [deselectAll](#).

In addition to this signal, selectable objects also provide individual signals, for example [QCPAxis::selectionChanged](#) or [QCPAbstractPlottable::selectionChanged](#). Note that those signals are emitted even if the selection state is changed programmatically.

See the documentation of [setInteractions](#) for details about the selection mechanism.

See also

[selectedPlottables](#), [selectedGraphs](#), [selectedItems](#), [selectedAxes](#), [selectedLegends](#)

15.624.4.71 selectionRect()

```
QCPSelectionRect * QCustomPlot::selectionRect ( ) const [inline]
```

Allows access to the currently used [QCPSelectionRect](#) instance (or subclass thereof), that is used to handle and draw selection rect interactions (see [setSelectionRectMode](#)).

See also

[setSelectionRect](#)

15.624.4.72 setAntialiasedElement()

```
void QCustomPlot::setAntialiasedElement (
    QCP::AntialiasedElement antialiasedElement,
    bool enabled = true )
```

Sets whether the specified *antialiasedElement* is forcibly drawn antialiased.

See [setAntialiasedElements](#) for details.

See also

[setNotAntialiasedElement](#)

15.624.4.73 setAntialiasedElements()

```
void QCustomPlot::setAntialiasedElements (
    const QCP::AntialiasedElements & antialiasedElements )
```

Sets which elements are forcibly drawn antialiased as an *or* combination of [QCP::AntialiasedElement](#).

This overrides the antialiasing settings for whole element groups, normally controlled with the *setAntialiasing* function on the individual elements. If an element is neither specified in [setAntialiasedElements](#) nor in [setNotAntialiasedElements](#), the antialiasing setting on each individual element instance is used.

For example, if *antialiasedElements* contains [QCP::aePlottables](#), all plottables will be drawn antialiased, no matter what the specific [QCPAbstractPlottable::setAntialiased](#) value was set to.

If an element in *antialiasedElements* is already set in [setNotAntialiasedElements](#), it is removed from there.

See also

[setNotAntialiasedElements](#)

15.624.4.74 setAutoAddPlottableToLegend()

```
void QCustomPlot::setAutoAddPlottableToLegend (
    bool on )
```

If set to true, adding a plottable (e.g. a graph) to the [QCustomPlot](#) automatically also adds the plottable to the legend ([QCustomPlot::legend](#)).

See also

[addGraph](#), [QCPLegend::addItem](#)

15.624.4.75 setBackground() [1/3]

```
void QCustomPlot::setBackground (
    const QBrush & brush )
```

Sets the background brush of the viewport (see [setViewport](#)).

Before drawing everything else, the background is filled with *brush*. If a background pixmap was set with [setBackground\(const QPixmap &pm\)](#), this brush will be used to fill the viewport before the background pixmap is drawn. This can be useful for background pixmaps with translucent areas.

Set *brush* to Qt::NoBrush or Qt::Transparent to leave background transparent. This can be useful for exporting to image formats which support transparency, e.g. [savePng](#).

See also

[setBackgroundScaled](#), [setBackgroundScaledMode](#)

15.624.4.76 setBackground() [2/3]

```
void QCustomPlot::setBackground (
    const QPixmap & pm )
```

Sets *pm* as the viewport background pixmap (see [setViewport](#)). The pixmap is always drawn below all other objects in the plot.

For cases where the provided pixmap doesn't have the same size as the viewport, scaling can be enabled with [setBackgroundScaled](#) and the scaling mode (whether and how the aspect ratio is preserved) can be set with [setBackgroundScaledMode](#). To set all these options in one call, consider using the overloaded version of this function.

If a background brush was set with [setBackground\(const QBrush &brush\)](#), the viewport will first be filled with that brush, before drawing the background pixmap. This can be useful for background pixmaps with translucent areas.

See also

[setBackgroundScaled](#), [setBackgroundScaledMode](#)

15.624.4.77 setBackground() [3/3]

```
void QCustomPlot::setBackground (
    const QPixmap & pm,
    bool scaled,
    Qt::AspectRatioMode mode = Qt::KeepAspectRatioByExpanding )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Allows setting the background pixmap of the viewport, whether it shall be scaled and how it shall be scaled in one call.

See also

[setBackground\(const QPixmap &pm\)](#), [setBackgroundScaled](#), [setBackgroundScaledMode](#)

15.624.4.78 setBackgroundScaled()

```
void QCustomPlot::setBackgroundScaled (
    bool scaled )
```

Sets whether the viewport background pixmap shall be scaled to fit the viewport. If *scaled* is set to true, control whether and how the aspect ratio of the original pixmap is preserved with [setBackgroundScaledMode](#).

Note that the scaled version of the original pixmap is buffered, so there is no performance penalty on replots. (Except when the viewport dimensions are changed continuously.)

See also

[setBackground](#), [setBackgroundScaledMode](#)

15.624.4.79 setBackgroundScaledMode()

```
void QCustomPlot::setBackgroundScaledMode (
    Qt::AspectRatioMode mode )
```

If scaling of the viewport background pixmap is enabled ([setBackgroundScaled](#)), use this function to define whether and how the aspect ratio of the original pixmap is preserved.

See also

[setBackground](#), [setBackgroundScaled](#)

15.624.4.80 setBufferDevicePixelRatio()

```
void QCustomPlot::setBufferDevicePixelRatio (
    double ratio )
```

Sets the device pixel ratio used by the paint buffers of this [QCustomPlot](#) instance.

Normally, this doesn't need to be set manually, because it is initialized with the regular *QWidget::devicePixelRatio* which is configured by Qt to fit the display device (e.g. 1 for normal displays, 2 for High-DPI displays).

Device pixel ratios are supported by Qt only for Qt versions since 5.4. If this method is called when [QCustomPlot](#) is being used with older Qt versions, outputs an according qDebug message and leaves the internal buffer device pixel ratio at 1.0.

15.624.4.81 setCurrentLayer() [1/2]

```
bool QCustomPlot::setCurrentLayer (
    const QString & name )
```

Sets the layer with the specified *name* to be the current layer. All layerables ([QCPLayerable](#)), e.g. plottables and items, are created on the current layer.

Returns true on success, i.e. if there is a layer with the specified *name* in the [QCustomPlot](#).

Layer names are case-sensitive.

See also

[addLayer](#), [moveLayer](#), [removeLayer](#), [QCPLayerable::setLayer](#)

15.624.4.82 setCurrentLayer() [2/2]

```
bool QCustomPlot::setCurrentLayer (
    QCPLayer * layer )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the provided *layer* to be the current layer.

Returns true on success, i.e. when *layer* is a valid layer in the [QCustomPlot](#).

See also

[addLayer](#), [moveLayer](#), [removeLayer](#)

15.624.4.83 setInteraction()

```
void QCustomPlot::setInteraction (
    const QCP::Interaction & interaction,
    bool enabled = true )
```

Sets the single *interaction* of this [QCustomPlot](#) to *enabled*.

For details about the interaction system, see [setInteractions](#).

See also

[setInteractions](#)

15.624.4.84 setInteractions()

```
void QCustomPlot::setInteractions (
    const QCP::Interactions & interactions )
```

Sets the possible interactions of this [QCustomPlot](#) as an or-combination of [QCP::Interaction](#) enums. There are the following types of interactions:

Axis range manipulation is controlled via [QCP::iRangeDrag](#) and [QCP::iRangeZoom](#). When the respective interaction is enabled, the user may drag axes ranges and zoom with the mouse wheel. For details how to control which axes the user may drag/zoom and in what orientations, see [QCPAxisRect::setRangeDrag](#), [QCPAxisRect::setRangeZoom](#), [QCPAxisRect::setRangeDragAxes](#), [QCPAxisRect::setRangeZoomAxes](#).

Plottable data selection is controlled by [QCP::iSelectPlottables](#). If [QCP::iSelectPlottables](#) is set, the user may select plottables (graphs, curves, bars,...) and their data by clicking on them or in their vicinity ([setSelectionTolerance](#)). Whether the user can actually select a plottable and its data can further be restricted with the [QCPAbstractPlottable::setSelectable](#) method on the specific plottable. For details, see the special page about the data selection mechanism. To retrieve a list of all currently selected plottables, call [selectedPlottables](#). If you're only interested in QCPGraphs, you may use the convenience function [selectedGraphs](#).

Item selection is controlled by [QCP::iSelectItems](#). If [QCP::iSelectItems](#) is set, the user may select items ([QCPItemLine](#), [QCPItemText](#),...) by clicking on them or in their vicinity. To find out whether a specific item is selected, call [QCPAbstractItem::selected\(\)](#). To retrieve a list of all currently selected items, call [selectedItems](#).

Axis selection is controlled with [QCP::iSelectAxes](#). If [QCP::iSelectAxes](#) is set, the user may select parts of the axes by clicking on them. What parts exactly (e.g. Axis base line, tick labels, axis label) are selectable can be controlled via [QCPAxis::setSelectableParts](#) for each axis. To retrieve a list of all axes that currently contain selected parts, call [selectedAxes](#). Which parts of an axis are selected, can be retrieved with [QCPAxis::selectedParts\(\)](#).

Legend selection is controlled with [QCP::iSelectLegend](#). If this is set, the user may select the legend itself or individual items by clicking on them. What parts exactly are selectable can be controlled via [QCPLegend::setSelectableParts](#). To find out whether the legend or any of its child items are selected, check the value of [QCPLegend::selectedParts](#). To find out which child items are selected, call [QCPLegend::selectedItems](#).

All other selectable elements The selection of all other selectable objects (e.g. [QCPTextElement](#), or your own layerable subclasses) is controlled with [QCP::iSelectOther](#). If set, the user may select those objects by clicking on them. To find out which are currently selected, you need to check their selected state explicitly.

If the selection state has changed by user interaction, the [selectionChangedByUser](#) signal is emitted. Each selectable object additionally emits an individual [selectionChanged](#) signal whenever their selection state has changed, i.e. not only by user interaction.

To allow multiple objects to be selected by holding the selection modifier ([setMultiSelectModifier](#)), set the flag [QCP::iMultiSelect](#).

Note

In addition to the selection mechanism presented here, [QCustomPlot](#) always emits corresponding signals, when an object is clicked or double clicked. see [plottableClick](#) and [plottableDoubleClick](#) for example.

See also

[setInteraction](#), [setSelectionTolerance](#)

15.624.4.85 setMultiSelectModifier()

```
void QCustomPlot::setMultiSelectModifier (
    Qt::KeyboardModifier modifier )
```

Sets the keyboard modifier that will be recognized as multi-select-modifier.

If [QCP::iMultiSelect](#) is specified in [setInteractions](#), the user may select multiple objects (or data points) by clicking on them one after the other while holding down *modifier*.

By default the multi-select-modifier is set to [Qt::ControlModifier](#).

See also

[setInteractions](#)

15.624.4.86 setNoAntialiasingOnDrag()

```
void QCustomPlot::setNoAntialiasingOnDrag (
    bool enabled )
```

Sets whether antialiasing is disabled for this [QCustomPlot](#) while the user is dragging axes ranges. If many objects, especially plottables, are drawn antialiased, this greatly improves performance during dragging. Thus it creates a more responsive user experience. As soon as the user stops dragging, the last replot is done with normal antialiasing, to restore high image quality.

See also

[setAntialiasedElements](#), [setNotAntialiasedElements](#)

15.624.4.87 setNotAntialiasedElement()

```
void QCustomPlot::setNotAntialiasedElement (
    QCP::AntialiasedElement notAntialiasedElement,
    bool enabled = true )
```

Sets whether the specified *notAntialiasedElement* is forcibly drawn not antialiased.

See [setNotAntialiasedElements](#) for details.

See also

[setAntialiasedElement](#)

15.624.4.88 setNotAntialiasedElements()

```
void QCustomPlot::setNotAntialiasedElements (
    const QCP::AntialiasedElements & notAntialiasedElements )
```

Sets which elements are forcibly drawn not antialiased as an *or* combination of [QCP::AntialiasedElement](#).

This overrides the antialiasing settings for whole element groups, normally controlled with the *setAntialiasing* function on the individual elements. If an element is neither specified in [setAntialiasedElements](#) nor in [setNotAntialiasedElements](#), the antialiasing setting on each individual element instance is used.

For example, if *notAntialiasedElements* contains [QCP::aePlottables](#), no plottables will be drawn antialiased, no matter what the specific [QCPAbstractPlottable::setAntialiased](#) value was set to.

if an element in *notAntialiasedElements* is already set in [setAntialiasedElements](#), it is removed from there.

See also

[setAntialiasedElements](#)

15.624.4.89 setOpenGL()

```
void QCustomPlot::setOpenGL (
    bool enabled,
    int multisampling = 16 )
```

Warning

This is still an experimental feature and its performance depends on the system that it runs on. Having multiple [QCustomPlot](#) widgets in one application with enabled OpenGL rendering might cause context conflicts on some systems.

This method allows to enable OpenGL plot rendering, for increased plotting performance of graphically demanding plots (thick lines, translucent fills, etc.).

If *enabled* is set to true, [QCustomPlot](#) will try to initialize OpenGL and, if successful, continue plotting with hardware acceleration. The parameter *multisampling* controls how many samples will be used per pixel, it essentially controls the antialiasing quality. If *multisampling* is set too high for the current graphics hardware, the maximum allowed value will be used.

You can test whether switching to OpenGL rendering was successful by checking whether the according getter [QCustomPlot::openGl\(\)](#) returns true. If the OpenGL initialization fails, rendering continues with the regular software rasterizer, and an according qDebug output is generated.

If switching to OpenGL was successful, this method disables label caching (`setPlottingHint(QCP::phCacheLabels, false)`) and turns on [QCustomPlot](#)'s antialiasing override for all elements (`setAntialiasedElements(QCP::aeAll)`), leading to a higher quality output. The antialiasing override allows for pixel-grid aligned drawing in the OpenGL paint device. As stated before, in OpenGL rendering the actual antialiasing of the plot is controlled with *multisampling*. If *enabled* is set to false, the antialiasing/label caching settings are restored to what they were before OpenGL was enabled, if they weren't altered in the meantime.

Note

OpenGL support is only enabled if [QCustomPlot](#) is compiled with the macro `QCUSTOMPLOT_USE_OPENGL` defined. This define must be set before including the [QCustomPlot](#) header both during compilation of the [QCustomPlot](#) library as well as when compiling your application. It is best to just include the line `DEFINES += QCUSTOMPLOT_USE_OPENGL` in the respective qmake project files.

If you are using a Qt version before 5.0, you must also add the module "opengl" to your `QT` variable in the qmake project files. For Qt versions 5.0 and higher, [QCustomPlot](#) switches to a newer OpenGL interface which is already in the "gui" module.

15.624.4.90 setPlottingHint()

```
void QCustomPlot::setPlottingHint (
    QCP::PlottingHint hint,
    bool enabled = true )
```

Sets the specified plotting *hint* to *enabled*.

See also

[setPlottingHints](#)

15.624.4.91 setPlottingHints()

```
void QCustomPlot::setPlottingHints (
    const QCP::PlottingHints & hints )
```

Sets the plotting hints for this [QCustomPlot](#) instance as an *or* combination of [QCP::PlottingHint](#).

See also

[setPlottingHint](#)

15.624.4.92 setSelectionRect()

```
void QCustomPlot::setSelectionRect (
    QCPSelectionRect * selectionRect )
```

Sets the [QCPSelectionRect](#) instance that [QCustomPlot](#) will use if *mode* is not [QCP::srmNone](#) and the user performs a click-and-drag interaction. [QCustomPlot](#) takes ownership of the passed *selectionRect*. It can be accessed later via [selectionRect](#).

This method is useful if you wish to replace the default [QCPSelectionRect](#) instance with an instance of a [QCPSelectionRect](#) subclass, to introduce custom behaviour of the selection rect.

See also

[setSelectionRectMode](#)

15.624.4.93 setSelectionRectMode()

```
void QCustomPlot::setSelectionRectMode (
    QCP::SelectionMode mode )
```

Sets how [QCustomPlot](#) processes mouse click-and-drag interactions by the user.

If *mode* is [QCP::srmNone](#), the mouse drag is forwarded to the underlying objects. For example, [QCPAxisRect](#) may process a mouse drag by dragging axis ranges, see [QCPAxisRect::setRangeDrag](#). If *mode* is not [QCP::srmNone](#), the current selection rect ([selectionRect](#)) becomes activated and allows e.g. rect zooming and data point selection.

If you wish to provide your user both with axis range dragging and data selection/range zooming, use this method to switch between the modes just before the interaction is processed, e.g. in reaction to the [mousePress](#) or [mouseMove](#) signals. For example you could check whether the user is holding a certain keyboard modifier, and then decide which *mode* shall be set.

If a selection rect interaction is currently active, and *mode* is set to [QCP::srmNone](#), the interaction is canceled ([QCPSelectionRect::cancel](#)). Switching between any of the other modes will keep the selection rect active. Upon completion of the interaction, the behaviour is as defined by the currently set *mode*, not the mode that was set when the interaction started.

See also

[setInteractions](#), [setSelectionRect](#), [QCPSelectionRect](#)

15.624.4.94 setSelectionTolerance()

```
void QCustomPlot::setSelectionTolerance (
    int pixels )
```

Sets the tolerance that is used to decide whether a click selects an object (e.g. a plottable) or not.

If the user clicks in the vicinity of the line of e.g. a [QCPGraph](#), it's only regarded as a potential selection when the minimum distance between the click position and the graph line is smaller than *pixels*. Objects that are defined by an area (e.g. [QCPBars](#)) only react to clicks directly inside the area and ignore this selection tolerance. In other words, it only has meaning for parts of objects that are too thin to exactly hit with a click and thus need such a tolerance.

See also

[setInteractions](#), [QCPLayerable::selectTest](#)

15.624.4.95 setViewport()

```
void QCustomPlot::setViewport (
    const QRect & rect )
```

Sets the viewport of this [QCustomPlot](#). Usually users of [QCustomPlot](#) don't need to change the viewport manually. The viewport is the area in which the plot is drawn. All mechanisms, e.g. margin calculation take the viewport to be the outer border of the plot. The viewport normally is the `rect()` of the [QCustomPlot](#) widget, i.e. a rect with top left (0, 0) and size of the [QCustomPlot](#) widget.

Don't confuse the viewport with the axis rect ([QCustomPlot::axisRect](#)). An axis rect is typically an area enclosed by four axes, where the graphs/plottables are drawn in. The viewport is larger and contains also the axes themselves, their tick numbers, their labels, or even additional axis rects, color scales and other layout elements.

This function is used to allow arbitrary size exports with [toPixmap](#), [savePng](#), [savePdf](#), etc. by temporarily changing the viewport size.

15.624.4.96 toPainter()

```
void QCustomPlot::toPainter (
    QCPPainter * painter,
    int width = 0,
    int height = 0 )
```

Renders the plot using the passed *painter*.

The plot is sized to *width* and *height* in pixels. If the *painter's* scale is not 1.0, the resulting plot will appear scaled accordingly.

Note

If you are restricted to using a QPainter (instead of [QCPPainter](#)), create a temporary QPixmap and open a [QCPPainter](#) on it. Then call [toPainter](#) with this [QCPPainter](#). After ending the paint operation on the picture, draw it with the QPainter. This will reproduce the painter actions the [QCPPainter](#) took, with a QPainter.

See also

[toPixmap](#)

15.624.4.97 toPixmap()

```
QPixmap QCustomPlot::toPixmap (
    int width = 0,
    int height = 0,
    double scale = 1.0 )
```

Renders the plot to a pixmap and returns it.

The plot is sized to *width* and *height* in pixels and scaled with *scale*. (width 100 and scale 2.0 lead to a full resolution pixmap with width 200.)

See also

[toPainter](#), [saveRastered](#), [saveBmp](#), [savePng](#), [saveJpg](#), [savePdf](#)

15.624.5 Member Data Documentation

15.624.5.1 legend

```
QCPLegend * QCustomPlot::legend
```

A pointer to the default legend of the main axis rect. The legend is invisible by default. Use [QCPLegend::setVisible](#) to change this.

[QCustomPlot](#) offers convenient pointers to the axes ([xAxis](#), [yAxis](#), [xAxis2](#), [yAxis2](#)) and the [legend](#). They make it very easy working with plots that only have a single axis rect and at most one axis at each axis rect side. If you use the layout system to add multiple legends to the plot, use the layout system interface to access the new legend. For example, legends can be placed inside an axis rect's [inset layout](#), and must then also be accessed via the inset layout. If the default legend is removed due to manipulation of the layout system (e.g. by removing the main axis rect), the corresponding pointer becomes `nullptr`.

If an axis convenience pointer is currently `nullptr` and a new axis rect or a corresponding axis is added in the place of the main axis rect, [QCustomPlot](#) resets the convenience pointers to the according new axes. Similarly the [legend](#) convenience pointer will be reset if a legend is added after the main legend was removed before.

15.624.5.2 xAxis

```
QCPAxis * QCustomPlot::xAxis
```

A pointer to the primary x Axis (bottom) of the main axis rect of the plot.

[QCustomPlot](#) offers convenient pointers to the axes ([xAxis](#), [yAxis](#), [xAxis2](#), [yAxis2](#)) and the [legend](#). They make it very easy working with plots that only have a single axis rect and at most one axis at each axis rect side. If you use the layout system to add multiple axis rects or multiple axes to one side, use the [QCPAxisRect::axis](#) interface to access the new axes. If one of the four default axes or the default legend is removed due to manipulation of the layout system (e.g. by removing the main axis rect), the corresponding pointers become `nullptr`.

If an axis convenience pointer is currently `nullptr` and a new axis rect or a corresponding axis is added in the place of the main axis rect, [QCustomPlot](#) resets the convenience pointers to the according new axes. Similarly the [legend](#) convenience pointer will be reset if a legend is added after the main legend was removed before.

15.624.5.3 xAxis2

```
QCPAxis * QCustomPlot::xAxis2
```

A pointer to the secondary x Axis (top) of the main axis rect of the plot. Secondary axes are invisible by default. Use [QCPAxis::setVisible](#) to change this (or use [QCPAxisRect::setupFullAxesBox](#)).

[QCustomPlot](#) offers convenient pointers to the axes ([xAxis](#), [yAxis](#), [xAxis2](#), [yAxis2](#)) and the [legend](#). They make it very easy working with plots that only have a single axis rect and at most one axis at each axis rect side. If you use the layout system to add multiple axis rects or multiple axes to one side, use the [QCPAxisRect::axis](#) interface to access the new axes. If one of the four default axes or the default legend is removed due to manipulation of the layout system (e.g. by removing the main axis rect), the corresponding pointers become `nullptr`.

If an axis convenience pointer is currently `nullptr` and a new axis rect or a corresponding axis is added in the place of the main axis rect, [QCustomPlot](#) resets the convenience pointers to the according new axes. Similarly the [legend](#) convenience pointer will be reset if a legend is added after the main legend was removed before.

15.624.5.4 yAxis

```
QCPAxis * QCustomPlot::yAxis
```

A pointer to the primary y Axis (left) of the main axis rect of the plot.

[QCustomPlot](#) offers convenient pointers to the axes ([xAxis](#), [yAxis](#), [xAxis2](#), [yAxis2](#)) and the [legend](#). They make it very easy working with plots that only have a single axis rect and at most one axis at each axis rect side. If you use the layout system to add multiple axis rects or multiple axes to one side, use the [QCPAxisRect::axis](#) interface to access the new axes. If one of the four default axes or the default legend is removed due to manipulation of the layout system (e.g. by removing the main axis rect), the corresponding pointers become `nullptr`.

If an axis convenience pointer is currently `nullptr` and a new axis rect or a corresponding axis is added in the place of the main axis rect, [QCustomPlot](#) resets the convenience pointers to the according new axes. Similarly the [legend](#) convenience pointer will be reset if a legend is added after the main legend was removed before.

15.624.5.5 [yAxis2](#)

[QCPAxis](#) * [QCustomPlot::yAxis2](#)

A pointer to the secondary y Axis (right) of the main axis rect of the plot. Secondary axes are invisible by default. Use [QCPAxis::setVisible](#) to change this (or use [QCPAxisRect::setupFullAxesBox](#)).

[QCustomPlot](#) offers convenient pointers to the axes ([xAxis](#), [yAxis](#), [xAxis2](#), [yAxis2](#)) and the [legend](#). They make it very easy working with plots that only have a single axis rect and at most one axis at each axis rect side. If you use the layout system to add multiple axis rects or multiple axes to one side, use the [QCPAxisRect::axis](#) interface to access the new axes. If one of the four default axes or the default legend is removed due to manipulation of the layout system (e.g. by removing the main axis rect), the corresponding pointers become `nullptr`.

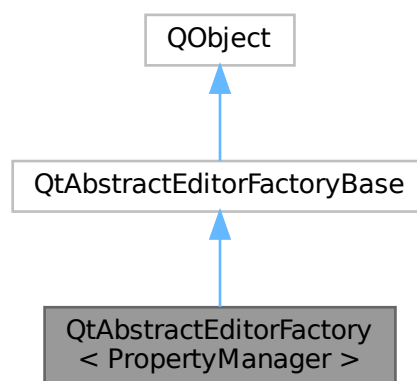
If an axis convenience pointer is currently `nullptr` and a new axis rect or a corresponding axis is added in the place of the main axis rect, [QCustomPlot](#) resets the convenience pointers to the according new axes. Similarly the [legend](#) convenience pointer will be reset if a legend is added after the main legend was removed before.

The documentation for this class was generated from the following files:

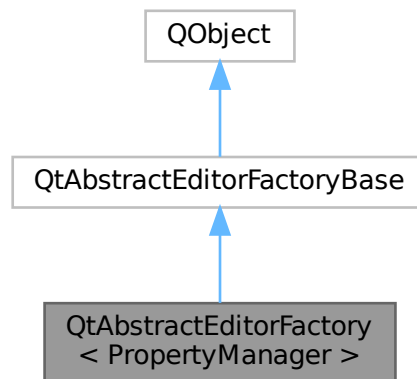
- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

15.625 [QtAbstractEditorFactory](#)< [PropertyManager](#) > Class Template Reference

Inheritance diagram for [QtAbstractEditorFactory](#)< [PropertyManager](#) >:



Collaboration diagram for QtAbstractEditorFactory< PropertyManager >:



Public Member Functions

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (PropertyManager *manager)
- void [removePropertyManager](#) (PropertyManager *manager)
- QSet< PropertyManager * > [propertyManagers](#) () const
- PropertyManager * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- virtual void [connectPropertyManager](#) (PropertyManager *manager)=0
- virtual QWidget * [createEditor](#) (PropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [disconnectPropertyManager](#) (PropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Friends

- class [QtAbstractPropertyEditor](#)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

15.625.1 Constructor & Destructor Documentation

15.625.1.1 QtAbstractEditorFactory()

```

template<class PropertyManager >
QtAbstractEditorFactory< PropertyManager >::QtAbstractEditorFactory (
    QObject * parent = 0 ) [inline], [explicit]

```

Creates an editor factory with the given *parent*.

See also

[addPropertyManager\(\)](#)

15.625.2 Member Function Documentation

15.625.2.1 addPropertyManager()

```
template<class PropertyManager >
void QtAbstractEditorFactory< PropertyManager >::addPropertyManager (
    PropertyManager * manager ) [inline]
```

Adds the given *manager* to this factory's set of managers, making this factory produce editing widgets for properties created by the given manager.

The PropertyManager type is a template argument class, and represents the chosen [QtAbstractPropertyManager](#) subclass.

See also

[propertyManagers\(\)](#), [removePropertyManager\(\)](#)

15.625.2.2 connectPropertyManager()

```
template<class PropertyManager >
void QtAbstractEditorFactory< PropertyManager >::connectPropertyManager (
    PropertyManager * manager ) [protected], [pure virtual]
```

Connects this factory to the given *manager's* signals. The PropertyManager type is a template argument class, and represents the chosen [QtAbstractPropertyManager](#) subclass.

This function is used internally by the [addPropertyManager\(\)](#) function, and makes it possible to update an editing widget when the associated property's data changes. This is typically done in custom slots responding to the signals emitted by the property's manager, e.g. [QtIntPropertyManager::valueChanged\(\)](#) and [QtIntPropertyManager::rangeChanged\(\)](#).

See also

[propertyManagers\(\)](#), [disconnectPropertyManager\(\)](#)

15.625.2.3 createEditor() [1/2]

```
template<class PropertyManager >
QWidget * QtAbstractEditorFactory< PropertyManager >::createEditor (
    PropertyManager * manager,
    QtProperty * property,
    QWidget * parent ) [protected], [pure virtual]
```

Creates an editing widget with the given *parent* for the specified *property* created by the given *manager*. The PropertyManager type is a template argument class, and represents the chosen [QtAbstractPropertyManager](#) subclass.

This function must be implemented in derived classes: It is recommended to store a pointer to the widget and map it to the given *property*, since the widget must be updated whenever the associated property's data changes. This is typically done in custom slots responding to the signals emitted by the property's manager, e.g. [QtIntPropertyManager::valueChanged\(\)](#) and [QtIntPropertyManager::rangeChanged\(\)](#).

See also

[connectPropertyManager\(\)](#)

15.625.2.4 createEditor() [2/2]

```
template<class PropertyManager >
QWidget * QtAbstractEditorFactory< PropertyManager >::createEditor (
    QtProperty * property,
    QWidget * parent ) [inline], [override], [virtual]
```

Creates an editing widget (with the given *parent*) for the given *property*.

Implements [QtAbstractEditorFactoryBase](#).

15.625.2.5 disconnectPropertyManager()

```
template<class PropertyManager >
void QtAbstractEditorFactory< PropertyManager >::disconnectPropertyManager (
    PropertyManager * manager ) [protected], [pure virtual]
```

Disconnects this factory from the given *manager's* signals. The PropertyManager type is a template argument class, and represents the chosen [QtAbstractPropertyManager](#) subclass.

This function is used internally by the [removePropertyManager\(\)](#) function.

See also

[propertyManagers\(\)](#), [connectPropertyManager\(\)](#)

15.625.2.6 managerDestroyed()

```
template<class PropertyManager >
void QtAbstractEditorFactory< PropertyManager >::managerDestroyed (
    QObject * manager ) [inline], [override], [protected], [virtual]
```

Implements [QtAbstractEditorFactoryBase](#).

15.625.2.7 propertyManager()

```
template<class PropertyManager >
PropertyManager * QtAbstractEditorFactory< PropertyManager >::propertyManager (
    QtProperty * property ) const [inline]
```

Returns the property manager for the given *property*, or 0 if the given *property* doesn't belong to any of this factory's registered managers.

The PropertyManager type is a template argument class, and represents the chosen [QtAbstractPropertyManager](#) subclass.

See also

[propertyManagers\(\)](#)

15.625.2.8 propertyManagers()

```
template<class PropertyManager >
QSet< PropertyManager * > QtAbstractEditorFactory< PropertyManager >::propertyManagers ( )
const [inline]
```

Returns the factory's set of associated managers. The PropertyManager type is a template argument class, and represents the chosen [QtAbstractPropertyManager](#) subclass.

See also

[addPropertyManager\(\)](#), [removePropertyManager\(\)](#)

15.625.2.9 removePropertyManager()

```
template<class PropertyManager >
void QtAbstractEditorFactory< PropertyManager >::removePropertyManager (
    PropertyManager * manager ) [inline]
```

Removes the given *manager* from this factory's set of managers. The PropertyManager type is a template argument class, and may be any [QtAbstractPropertyManager](#) subclass.

See also

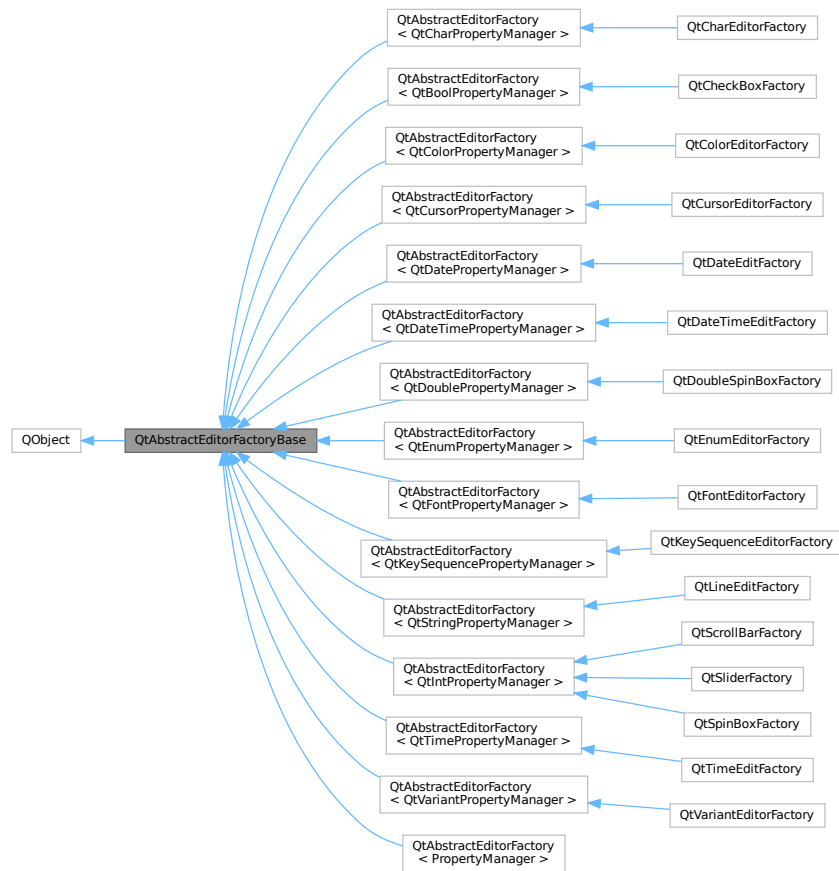
[propertyManagers\(\)](#), [addPropertyManager\(\)](#)

The documentation for this class was generated from the following files:

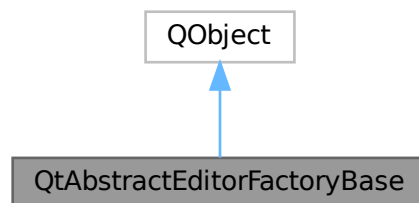
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.cpp

15.626 QtAbstractEditorFactoryBase Class Reference

Inheritance diagram for QtAbstractEditorFactoryBase:



Collaboration diagram for QtAbstractEditorFactoryBase:



Public Member Functions

- virtual QWidget * [createEditor](#) (QtProperty *property, QWidget *parent)=0

Protected Slots

- virtual void **managerDestroyed** (QObject *manager)=0

Protected Member Functions

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)
- virtual void **breakConnection** ([QtAbstractPropertyManager](#) *manager)=0

Friends

- class **QtAbstractPropertyBrowser**

15.626.1 Constructor & Destructor Documentation

15.626.1.1 QtAbstractEditorFactoryBase()

```
QtAbstractEditorFactoryBase::QtAbstractEditorFactoryBase (
    QObject * parent = 0 ) [inline], [explicit], [protected]
```

Creates an abstract editor factory with the given *parent*.

15.626.2 Member Function Documentation

15.626.2.1 createEditor()

```
QWidget * QtAbstractEditorFactoryBase::createEditor (
    QtProperty * property,
    QWidget * parent ) [pure virtual]
```

Creates an editing widget (with the given *parent*) for the given *property*.

This function is reimplemented in [QtAbstractEditorFactory](#) template class which also provides a pure virtual convenience overload of this function enabling access to the property's manager.

See also

[QtAbstractEditorFactory::createEditor\(\)](#)

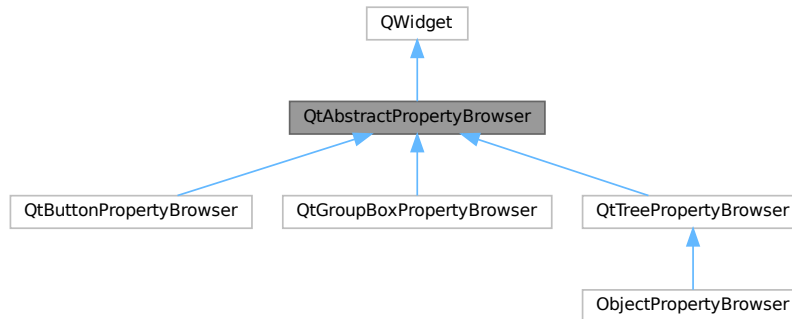
Implemented in [QtAbstractEditorFactory< PropertyManager >](#), [QtAbstractEditorFactory< QtBoolPropertyManager >](#), [QtAbstractEditorFactory< QtCharPropertyManager >](#), [QtAbstractEditorFactory< QtColorPropertyManager >](#), [QtAbstractEditorFactory< QtCursorPropertyManager >](#), [QtAbstractEditorFactory< QtDatePropertyManager >](#), [QtAbstractEditorFactory< QDateTimePropertyManager >](#), [QtAbstractEditorFactory< QtDoublePropertyManager >](#), [QtAbstractEditorFactory< QtEnumPropertyManager >](#), [QtAbstractEditorFactory< QFontPropertyManager >](#), [QtAbstractEditorFactory< QtIntPropertyManager >](#), [QtAbstractEditorFactory< QtKeySequencePropertyManager >](#), [QtAbstractEditorFactory< QStringPropertyManager >](#), [QtAbstractEditorFactory< QtTimePropertyManager >](#), and [QtAbstractEditorFactory< QtVariantPropertyManager >](#).

The documentation for this class was generated from the following files:

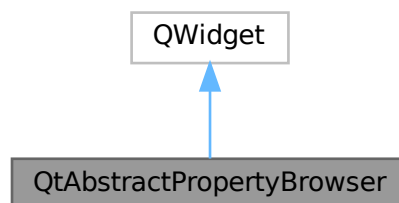
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.cpp

15.627 QtAbstractPropertyBrowser Class Reference

Inheritance diagram for QtAbstractPropertyBrowser:



Collaboration diagram for QtAbstractPropertyBrowser:



Public Slots

- [QtBrowserItem](#) * [addProperty](#) ([QtProperty](#) *property)
- [QtBrowserItem](#) * [insertProperty](#) ([QtProperty](#) *property, [QtProperty](#) *afterProperty)
- void [removeProperty](#) ([QtProperty](#) *property)

Signals

- void [currentItemChanged](#) ([QtBrowserItem](#) *)

Public Member Functions

- [QtAbstractPropertyBrowser](#) ([QWidget](#) *parent=0)
- [~QtAbstractPropertyBrowser](#) ()
- [QList](#)< [QtProperty](#) * > [properties](#) () const
- [QList](#)< [QtBrowserItem](#) * > [items](#) ([QtProperty](#) *property) const
- [QtBrowserItem](#) * [topLevelItem](#) ([QtProperty](#) *property) const
- [QList](#)< [QtBrowserItem](#) * > [topLevelItems](#) () const
- void [clear](#) ()
- template<class [PropertyManager](#) >
void [setFactoryForManager](#) ([PropertyManager](#) *manager, [QtAbstractEditorFactory](#)< [PropertyManager](#) > *factory)

- void [unsetFactoryForManager](#) ([QtAbstractPropertyManager](#) *manager)
- [QtBrowserItem](#) * [currentItem](#) () const
- void [setCurrentItem](#) ([QtBrowserItem](#) *)

Protected Member Functions

- virtual void [itemInserted](#) ([QtBrowserItem](#) *item, [QtBrowserItem](#) *afterItem)=0
- virtual void [itemRemoved](#) ([QtBrowserItem](#) *item)=0
- virtual void [itemChanged](#) ([QtBrowserItem](#) *item)=0
- virtual QWidget * [createEditor](#) ([QtProperty](#) *property, QWidget *parent)

15.627.1 Constructor & Destructor Documentation

15.627.1.1 QtAbstractPropertyBrowser()

```
QtAbstractPropertyBrowser::QtAbstractPropertyBrowser (
    QWidget * parent = 0 ) [explicit]
```

Creates an abstract property browser with the given *parent*.

15.627.1.2 ~QtAbstractPropertyBrowser()

```
QtAbstractPropertyBrowser::~QtAbstractPropertyBrowser ( )
```

Destroys the property browser, and destroys all the items that were created by this property browser.

Note that the properties that were displayed in the editor are not deleted since they still can be used in other editors. Neither does the destructor delete the property managers and editor factories that were used by this property browser widget unless this widget was their parent.

See also

[QtAbstractPropertyManager::~~QtAbstractPropertyManager\(\)](#)

15.627.2 Member Function Documentation

15.627.2.1 addProperty

```
QtBrowserItem * QtAbstractPropertyBrowser::addProperty (
    QtProperty * property ) [slot]
```

Appends the given *property* (and its subproperties) to the property browser's list of top level properties. Returns the item created by property browser which is associated with the *property*. In order to get all children items created by the property browser in this call, the returned item should be traversed.

If the specified *property* is already added, this function does nothing and returns 0.

See also

[insertProperty\(\)](#), [QtProperty::addSubProperty\(\)](#), [properties\(\)](#)

15.627.2.2 clear()

```
void QtAbstractPropertyBrowser::clear ( )
```

Removes all the properties from the editor, but does not delete them since they can still be used in other editors.

See also

[removeProperty\(\)](#), [QtAbstractPropertyManager::clear\(\)](#)

15.627.2.3 createEditor()

```
QWidget * QtAbstractPropertyBrowser::createEditor (
    QtProperty * property,
    QWidget * parent ) [protected], [virtual]
```

Creates an editing widget (with the given *parent*) for the given *property* according to the previously established associations between property managers and editor factories.

If the property is created by a property manager which was not associated with any of the existing factories in *this* property editor, the function returns 0.

To make a property editable in the property browser, the [createEditor\(\)](#) function must be called to provide the property with a suitable editing widget.

Reimplement this function to provide additional decoration for the editing widgets created by the installed factories.

See also

[setFactoryForManager\(\)](#)

15.627.2.4 currentItem()

```
QtBrowserItem * QtAbstractPropertyBrowser::currentItem ( ) const
```

Returns the current item in the property browser.

See also

[setCurrentItem\(\)](#)

15.627.2.5 currentItemChanged

```
void QtAbstractPropertyBrowser::currentItemChanged (
    QtBrowserItem * current ) [signal]
```

This signal is emitted when the current item changes. The current item is specified by *current*.

See also

[QtAbstractPropertyBrowser::setCurrentItem\(\)](#)

15.627.2.6 insertProperty

```
QtBrowserItem * QtAbstractPropertyBrowser::insertProperty (
    QtProperty * property,
    QtProperty * afterProperty ) [slot]
```

Inserts the given *property* (and its subproperties) after the specified *afterProperty* in the browser's list of top level properties. Returns item created by property browser which is associated with the *property*. In order to get all children items created by the property browser in this call returned item should be traversed.

If the specified *afterProperty* is 0, the given *property* is inserted at the beginning of the list. If *property* is already inserted, this function does nothing and returns 0.

See also

[addProperty\(\)](#), [QtProperty::insertSubProperty\(\)](#), [properties\(\)](#)

15.627.2.7 itemChanged()

```
void QtAbstractPropertyBrowser::itemChanged (
    QtBrowserItem * item ) [protected], [pure virtual]
```

This function is called whenever a property's data changes, passing a pointer to the *item* of property as parameter. This function must be reimplemented in derived classes in order to update the property browser widget whenever a property's name, tool tip, status tip, "what's this" text, value text or value icon changes.

Note that if the property browser contains several occurrences of the same property, this method will be called once for each occurrence (with a different item each time).

See also

[QtProperty](#), [items\(\)](#)

Implemented in [QtButtonPropertyBrowser](#), [QtGroupBoxPropertyBrowser](#), and [QtTreePropertyBrowser](#).

15.627.2.8 itemInserted()

```
void QtAbstractPropertyBrowser::itemInserted (
    QtBrowserItem * insertedItem,
    QtBrowserItem * precedingItem ) [protected], [pure virtual]
```

This function is called to update the widget whenever a property is inserted or added to the property browser, passing pointers to the *insertedItem* of property and the specified *precedingItem* as parameters.

If *precedingItem* is 0, the *insertedItem* was put at the beginning of its parent item's list of subproperties. If the parent of *insertedItem* is 0, the *insertedItem* was added as a top level property of *this* property browser.

This function must be reimplemented in derived classes. Note that if the *insertedItem*'s property has subproperties, this method will be called for those properties as soon as the current call is finished.

See also

[insertProperty\(\)](#), [addProperty\(\)](#)

Implemented in [QtButtonPropertyBrowser](#), [QtGroupBoxPropertyBrowser](#), and [QtTreePropertyBrowser](#).

15.627.2.9 itemRemoved()

```
void QtAbstractPropertyBrowser::itemRemoved (
    QtBrowserItem * item ) [protected], [pure virtual]
```

This function is called to update the widget whenever a property is removed from the property browser, passing the pointer to the *item* of the property as parameters. The passed *item* is deleted just after this call is finished.

If the the parent of *item* is 0, the removed *item* was a top level property in this editor.

This function must be reimplemented in derived classes. Note that if the removed *item*'s property has subproperties, this method will be called for those properties just before the current call is started.

See also

[removeProperty\(\)](#)

Implemented in [QtButtonPropertyBrowser](#), [QtGroupBoxPropertyBrowser](#), and [QtTreePropertyBrowser](#).

15.627.2.10 items()

```
QList< QtBrowserItem * > QtAbstractPropertyBrowser::items (
    QtProperty * property ) const
```

Returns the property browser's list of all items associated with the given *property*.

There is one item per instance of the property in the browser.

See also

[topLevelItem\(\)](#)

15.627.2.11 properties()

```
QList< QtProperty * > QtAbstractPropertyBrowser::properties ( ) const
```

Returns the property browser's list of top level properties.

To traverse the subproperties, use the [QtProperty::subProperties\(\)](#) function.

See also

[addProperty\(\)](#), [insertProperty\(\)](#), [removeProperty\(\)](#)

15.627.2.12 removeProperty

```
void QtAbstractPropertyBrowser::removeProperty (
    QtProperty * property ) [slot]
```

Removes the specified *property* (and its subproperties) from the property browser's list of top level properties. All items that were associated with the given *property* and its children are deleted.

Note that the properties are *not* deleted since they can still be used in other editors.

See also

[clear\(\)](#), [QtProperty::removeSubProperty\(\)](#), [properties\(\)](#)

15.627.2.13 setCurrentItem()

```
void QtAbstractPropertyBrowser::setCurrentItem (
    QtBrowserItem * item )
```

Sets the current item in the property browser to *item*.

See also

[currentItem\(\)](#), [currentItemChanged\(\)](#)

15.627.2.14 setFactoryForManager()

```
template<class PropertyManager >
void QtAbstractPropertyBrowser::setFactoryForManager (
    PropertyManager * manager,
    QtAbstractEditorFactory< PropertyManager > * factory ) [inline]
```

Connects the given *manager* to the given *factory*, ensuring that properties of the *manager*'s type will be displayed with an editing widget suitable for their value.

For example:

In this example the `myInteger` property's value is displayed with a `QSpinBox` widget, while the `myDouble` property's value is displayed with a `QDoubleSpinBox` widget.

Note that a factory can be associated with many managers, but a manager can only be associated with one single factory. If the given *manager* already is associated with another factory, the old association is broken before the new one established.

This function ensures that the given *manager* and the given *factory* are compatible, and it automatically calls the [QtAbstractEditorFactory::addPropertyManager\(\)](#) function if necessary.

See also

[unsetFactoryForManager\(\)](#)

15.627.2.15 topLevelItem()

```
QtBrowserItem * QtAbstractPropertyBrowser::topLevelItem (
    QtProperty * property ) const
```

Returns the top-level items associated with the given *property*.

Returns 0 if *property* wasn't inserted into this property browser or isn't a top-level one.

See also

[topLevelItems\(\)](#), [items\(\)](#)

15.627.2.16 topLevelItems()

```
QList< QtBrowserItem * > QtAbstractPropertyBrowser::topLevelItems ( ) const
```

Returns the list of top-level items.

See also

[topLevelItem\(\)](#)

15.627.2.17 unsetFactoryForManager()

```
void QtAbstractPropertyBrowser::unsetFactoryForManager (
    QtAbstractPropertyManager * manager )
```

Removes the association between the given *manager* and the factory bound to it, automatically calling the [QtAbstractEditorFactory::removePropertyManager\(\)](#) function if necessary.

See also

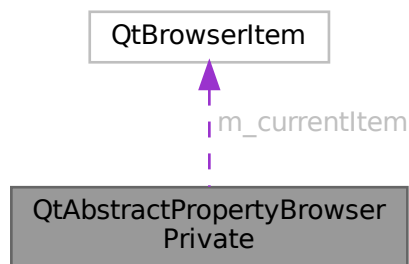
[setFactoryForManager\(\)](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.cpp

15.628 QtAbstractPropertyBrowserPrivate Class Reference

Collaboration diagram for QtAbstractPropertyBrowserPrivate:



Public Member Functions

- void **insertSubTree** ([QtProperty](#) *property, [QtProperty](#) *parentProperty)
- void **removeSubTree** ([QtProperty](#) *property, [QtProperty](#) *parentProperty)
- void **createBrowserIndexes** ([QtProperty](#) *property, [QtProperty](#) *parentProperty, [QtProperty](#) *afterProperty)
- void **removeBrowserIndexes** ([QtProperty](#) *property, [QtProperty](#) *parentProperty)
- [QtBrowserItem](#) * **createBrowserIndex** ([QtProperty](#) *property, [QtBrowserItem](#) *parentIndex, [QtBrowserItem](#) *afterIndex)
- void **removeBrowserIndex** ([QtBrowserItem](#) *index)
- void **clearIndex** ([QtBrowserItem](#) *index)
- void **slotPropertyInserted** ([QtProperty](#) *property, [QtProperty](#) *parentProperty, [QtProperty](#) *afterProperty)
- void **slotPropertyRemoved** ([QtProperty](#) *property, [QtProperty](#) *parentProperty)
- void **slotPropertyDestroyed** ([QtProperty](#) *property)
- void **slotPropertyDataChanged** ([QtProperty](#) *property)

Public Attributes

- [QList](#)< [QtProperty](#) * > **m_subItems**
- [QMap](#)< [QtAbstractPropertyManager](#) *, [QList](#)< [QtProperty](#) * > > **m_managerToProperties**
- [QMap](#)< [QtProperty](#) *, [QList](#)< [QtProperty](#) * > > **m_propertyToParents**
- [QMap](#)< [QtProperty](#) *, [QtBrowserItem](#) * > **m_topLevelPropertyToIndex**
- [QList](#)< [QtBrowserItem](#) * > **m_topLevelIndexes**

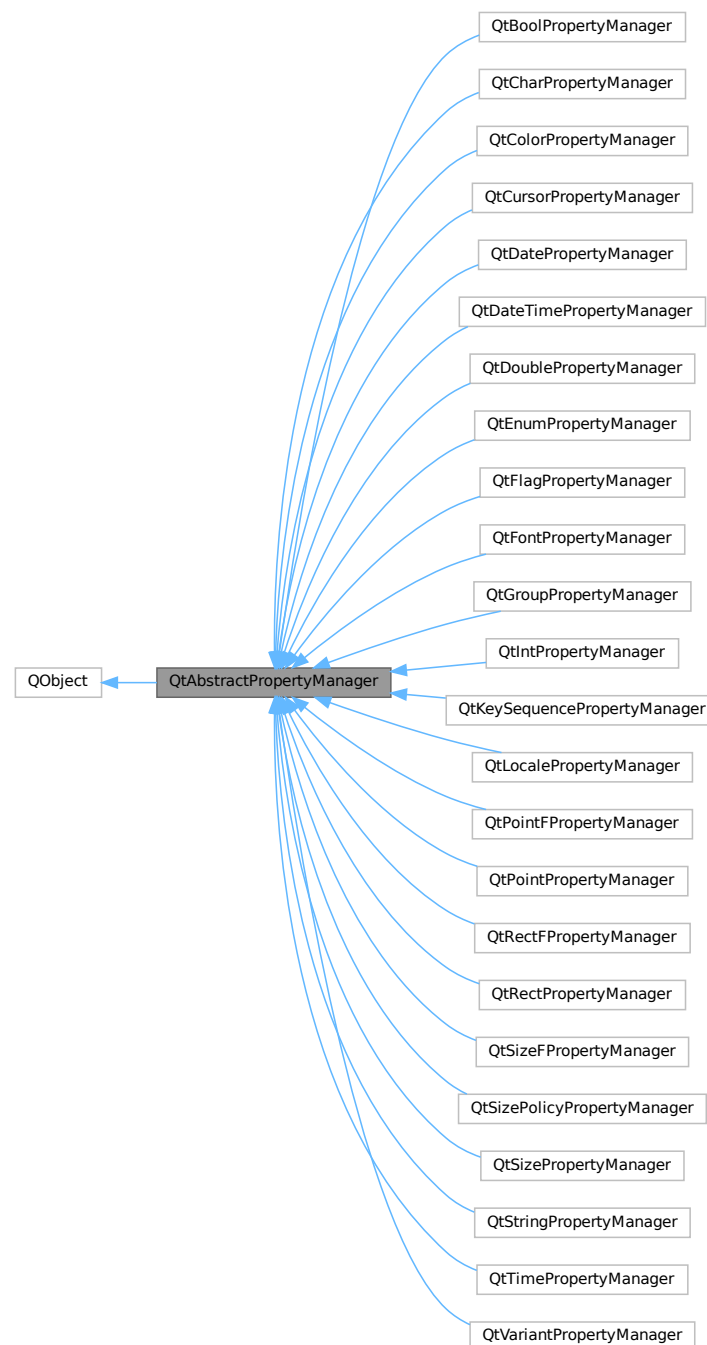
- `QMap< QtProperty *, QList< QtBrowserItem * > > m_propertyToIndexes`
- `QtBrowserItem * m_currentItem`

The documentation for this class was generated from the following file:

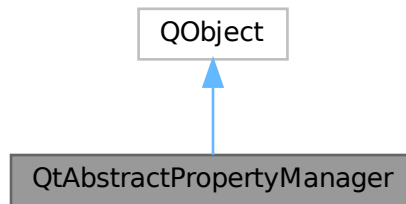
- `applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.cpp`

15.629 QtAbstractPropertyManager Class Reference

Inheritance diagram for QtAbstractPropertyManager:



Collaboration diagram for QtAbstractPropertyManager:



Signals

- void `propertyInserted` (`QtProperty` *property, `QtProperty` *parent, `QtProperty` *after)
- void `propertyChanged` (`QtProperty` *property)
- void `propertyRemoved` (`QtProperty` *property, `QtProperty` *parent)
- void `propertyDestroyed` (`QtProperty` *property)

Public Member Functions

- `QtAbstractPropertyManager` (`QObject` *parent=0)
- `~QtAbstractPropertyManager` ()
- `QSet< QtProperty * > properties` () const
- void `clear` () const
- `QtProperty` * `addProperty` (const `QString` &name=`QString`())

Protected Member Functions

- virtual bool `hasValue` (const `QtProperty` *property) const
- virtual `QIcon` `valueIcon` (const `QtProperty` *property) const
- virtual `QString` `valueText` (const `QtProperty` *property) const
- virtual void `initializeProperty` (`QtProperty` *property)=0
- virtual void `uninitializeProperty` (`QtProperty` *property)
- virtual `QtProperty` * `createProperty` ()

Friends

- class `QtProperty`

15.629.1 Constructor & Destructor Documentation

15.629.1.1 QtAbstractPropertyManager()

```
QtAbstractPropertyManager::QtAbstractPropertyManager (
    QObject * parent = 0 ) [explicit]
```

Creates an abstract property manager with the given *parent*.

15.629.1.2 ~QtAbstractPropertyManager()

```
QtAbstractPropertyManager::~QtAbstractPropertyManager ( )
```

Destroys the manager. All properties created by the manager are destroyed.

15.629.2 Member Function Documentation

15.629.2.1 addProperty()

```
QtProperty * QtAbstractPropertyManager::addProperty (
    const QString & name = QString() )
```

Creates a property with the given *name* which then is owned by this manager. Internally, this function calls the [createProperty\(\)](#) and [initializeProperty\(\)](#) functions.

See also

[initializeProperty\(\)](#), [properties\(\)](#)

15.629.2.2 clear()

```
void QtAbstractPropertyManager::clear ( ) const
```

Destroys all the properties that this manager has created.

See also

[propertyDestroyed\(\)](#), [uninitializeProperty\(\)](#)

15.629.2.3 createProperty()

```
QtProperty * QtAbstractPropertyManager::createProperty ( ) [protected], [virtual]
```

Creates a property.

The base implementation produce [QtProperty](#) instances; Reimplement this function to make this manager produce objects of a [QtProperty](#) subclass.

See also

[addProperty\(\)](#), [initializeProperty\(\)](#)

Reimplemented in [QtVariantPropertyManager](#).

15.629.2.4 hasValue()

```
bool QtAbstractPropertyManager::hasValue (
    const QtProperty * property ) const [protected], [virtual]
```

Returns whether the given *property* has a value.

The default implementation of this function returns true.

See also

[QtProperty::hasValue\(\)](#)

Reimplemented in [QtGroupPropertyManager](#), and [QtVariantPropertyManager](#).

15.629.2.5 initializeProperty()

```
void QtAbstractPropertyManager::initializeProperty (
    QtProperty * property ) [protected], [pure virtual]
```

This function is called whenever a new valid property pointer has been created, passing the pointer as parameter. The purpose is to let the manager know that the *property* has been created so that it can provide additional attributes for the new property, e.g. [QtIntPropertyManager](#) adds `\ {QtIntPropertyManager::value()}{value}`, `\ {QtIntPropertyManager::minimum()}{minimum}` and `\ {QtIntPropertyManager::maximum()}{maximum}` attributes. Since each manager subclass adds type specific attributes, this function is pure virtual and must be reimplemented when deriving from the [QtAbstractPropertyManager](#) class.

See also

[addProperty\(\)](#), [createProperty\(\)](#)

Implemented in [QtGroupPropertyManager](#), [QtIntPropertyManager](#), [QtBoolPropertyManager](#), [QtDoublePropertyManager](#), [QtStringPropertyManager](#), [QtDatePropertyManager](#), [QtTimePropertyManager](#), [QtDateTimePropertyManager](#), [QtKeySequencePropertyManager](#), [QtCharPropertyManager](#), [QtLocalePropertyManager](#), [QtPointPropertyManager](#), [QtPointFPropertyManager](#), [QtSizePropertyManager](#), [QtSizeFPropertyManager](#), [QtRectPropertyManager](#), [QtRectFPropertyManager](#), [QtEnumPropertyManager](#), [QtFlagPropertyManager](#), [QtSizePolicyPropertyManager](#), [QtFontPropertyManager](#), [QtColorPropertyManager](#), [QtCursorPropertyManager](#), and [QtVariantPropertyManager](#).

15.629.2.6 properties()

```
QSet< QtProperty * > QtAbstractPropertyManager::properties ( ) const
```

Returns the set of properties created by this manager.

See also

[addProperty\(\)](#)

15.629.2.7 propertyChanged

```
void QtAbstractPropertyManager::propertyChanged (
    QtProperty * property ) [signal]
```

This signal is emitted whenever a property's data changes, passing a pointer to the *property* as parameter. Note that signal is only emitted for properties that are created by this manager.

See also

[QtAbstractPropertyBrowser::itemChanged\(\)](#)

15.629.2.8 propertyDestroyed

```
void QtAbstractPropertyManager::propertyDestroyed (
    QtProperty * property ) [signal]
```

This signal is emitted when the specified *property* is about to be destroyed. Note that signal is only emitted for properties that are created by this manager.

See also

[clear\(\)](#), [uninitializeProperty\(\)](#)

15.629.2.9 propertyInserted

```
void QtAbstractPropertyManager::propertyInserted (
    QtProperty * newProperty,
    QtProperty * parentProperty,
    QtProperty * precedingProperty ) [signal]
```

This signal is emitted when a new subproperty is inserted into an existing property, passing pointers to the *newProperty*, *parentProperty* and *precedingProperty* as parameters.

If *precedingProperty* is 0, the *newProperty* was inserted at the beginning of the *parentProperty*'s subproperties list.

Note that signal is emitted only if the *parentProperty* is created by this manager.

See also

[QtAbstractPropertyBrowser::itemInserted\(\)](#)

15.629.2.10 propertyRemoved

```
void QtAbstractPropertyManager::propertyRemoved (
    QtProperty * property,
    QtProperty * parent ) [signal]
```

This signal is emitted when a subproperty is removed, passing pointers to the removed *property* and the *parent* property as parameters.

Note that signal is emitted only when the *parent* property is created by this manager.

See also

[QtAbstractPropertyBrowser::itemRemoved\(\)](#)

15.629.2.11 uninitializedProperty()

```
void QtAbstractPropertyManager::uninitializeProperty (
    QtProperty * property ) [protected], [virtual]
```

This function is called just before the specified *property* is destroyed.

The purpose is to let the property manager know that the *property* is being destroyed so that it can remove the property's additional attributes.

See also

[clear\(\)](#), [propertyDestroyed\(\)](#)

Reimplemented in [QtGroupPropertyManager](#), [QtIntPropertyManager](#), [QtBoolPropertyManager](#), [QtDoublePropertyManager](#), [QtStringPropertyManager](#), [QtDatePropertyManager](#), [QtTimePropertyManager](#), [QtDateTimePropertyManager](#), [QtKeySequencePropertyManager](#), [QtCharPropertyManager](#), [QtLocalePropertyManager](#), [QtPointPropertyManager](#), [QtPointFPropertyManager](#), [QtSizePropertyManager](#), [QtSizeFPropertyManager](#), [QtRectPropertyManager](#), [QtRectFPropertyManager](#), [QtEnumPropertyManager](#), [QtFlagPropertyManager](#), [QtSizePolicyPropertyManager](#), [QtFontPropertyManager](#), [QtColorPropertyManager](#), [QtCursorPropertyManager](#), and [QtVariantPropertyManager](#).

15.629.2.12 valueIcon()

```
QIcon QtAbstractPropertyManager::valueIcon (
    const QtProperty * property ) const [protected], [virtual]
```

Returns an icon representing the current state of the given *property*.

The default implementation of this function returns an invalid icon.

See also

[QtProperty::valueIcon\(\)](#)

Reimplemented in [QtBoolPropertyManager](#), [QtEnumPropertyManager](#), [QtFontPropertyManager](#), [QtColorPropertyManager](#), [QtCursorPropertyManager](#), and [QtVariantPropertyManager](#).

15.629.2.13 valueText()

```
QString QtAbstractPropertyManager::valueText (
    const QtProperty * property ) const [protected], [virtual]
```

Returns a string representing the current state of the given *property*.

The default implementation of this function returns an empty string.

See also

[QtProperty::valueText\(\)](#)

Reimplemented in [QtIntPropertyManager](#), [QtBoolPropertyManager](#), [QtDoublePropertyManager](#), [QtStringPropertyManager](#), [QtDatePropertyManager](#), [QtTimePropertyManager](#), [QtDateTimePropertyManager](#), [QtKeySequencePropertyManager](#), [QtCharPropertyManager](#), [QtLocalePropertyManager](#), [QtPointPropertyManager](#), [QtPointFPropertyManager](#), [QtSizePropertyManager](#), [QtSizeFPropertyManager](#), [QtRectPropertyManager](#), [QtRectFPropertyManager](#), [QtEnumPropertyManager](#), [QtFlagPropertyManager](#), [QtSizePolicyPropertyManager](#), [QtFontPropertyManager](#), [QtColorPropertyManager](#), [QtCursorPropertyManager](#), and [QtVariantPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.cpp

15.630 QtAbstractPropertyManagerPrivate Class Reference

Public Member Functions

- void **propertyDestroyed** (QtProperty *property)
- void **propertyChanged** (QtProperty *property) const
- void **propertyRemoved** (QtProperty *property, QtProperty *parentProperty) const
- void **propertyInserted** (QtProperty *property, QtProperty *parentProperty, QtProperty *afterProperty) const

Public Attributes

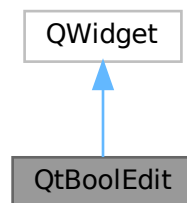
- QSet< QtProperty * > **m_properties**

The documentation for this class was generated from the following file:

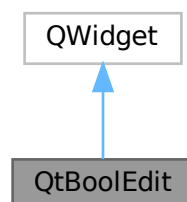
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.cpp

15.631 QtBoolEdit Class Reference

Inheritance diagram for QtBoolEdit:



Collaboration diagram for QtBoolEdit:



Signals

- void **toggled** (bool)

Public Member Functions

- **QtBoolEdit** (QWidget *parent=0)
- bool **textVisible** () const
- void **setTextVisible** (bool textVisible)
- Qt::CheckState **checkState** () const
- void **setCheckState** (Qt::CheckState state)
- bool **isChecked** () const
- void **setChecked** (bool c)
- void **setInitialChecked** (bool c)
- bool **blockCheckBoxSignals** (bool block)

Protected Member Functions

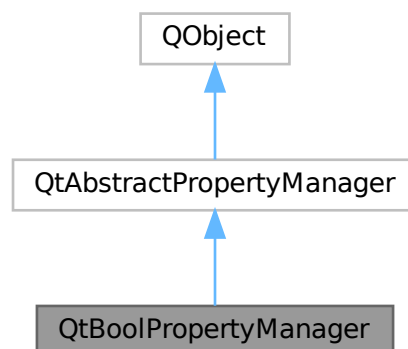
- void **mousePressEvent** (QMouseEvent *event) override

The documentation for this class was generated from the following files:

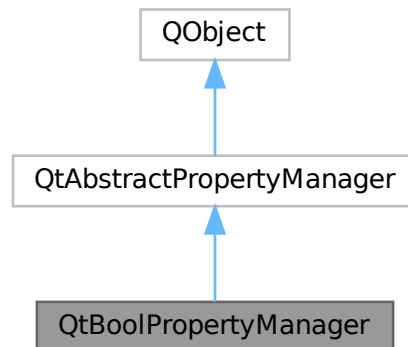
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils_p.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils.cpp

15.632 QtBoolPropertyManager Class Reference

Inheritance diagram for QtBoolPropertyManager:



Collaboration diagram for QtBoolPropertyManager:



Public Slots

- void [setValue](#) (QtProperty *property, bool val)
- void **setValueOnly** (QtProperty *property, bool val)

Signals

- void [valueChanged](#) (QtProperty *property, bool val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) (QtProperty *property, QtProperty *parent, QtProperty *after)
- void [propertyChanged](#) (QtProperty *property)
- void [propertyRemoved](#) (QtProperty *property, QtProperty *parent)
- void [propertyDestroyed](#) (QtProperty *property)

Public Member Functions

- [QtBoolPropertyManager](#) (QObject *parent=0)
- [~QtBoolPropertyManager](#) ()
- bool [value](#) (const QtProperty *property) const
- bool [initialValue](#) (const QtProperty *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< QtProperty * > [properties](#) () const
- void [clear](#) () const
- QtProperty * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const QtProperty *property) const override
- QIcon [valueIcon](#) (const QtProperty *property) const override
- void [initializeProperty](#) (QtProperty *property) override
- void [uninitializeProperty](#) (QtProperty *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.632.1 Constructor & Destructor Documentation

15.632.1.1 [QtBoolPropertyManager](#)()

```
QtBoolPropertyManager::QtBoolPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.632.1.2 [~QtBoolPropertyManager](#)()

```
QtBoolPropertyManager::~~QtBoolPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.632.2 Member Function Documentation

15.632.2.1 [initializeProperty](#)()

```
void QtBoolPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.632.2.2 [setValue](#)

```
void QtBoolPropertyManager::setValue (
    QtProperty * property,
    bool value ) [slot]
```

Sets the value of the given *property* to *value*.

See also

[value\(\)](#)

15.632.2.3 [uninitializeProperty](#)()

```
void QtBoolPropertyManager::uninitializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.632.2.4 [value](#)()

```
bool QtBoolPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by *this* manager, this function returns false.

See also

[setValue\(\)](#)

15.632.2.5 valueChanged

```
void QtBoolPropertyManager::valueChanged (
    QtProperty * property,
    bool value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

15.632.2.6 valueIcon()

```
QIcon QtBoolPropertyManager::valueIcon (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.632.2.7 valueText()

```
QString QtBoolPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.633 QtBoolPropertyManagerPrivate Class Reference**Classes**

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Attributes

- QMap< const [QtProperty](#) *, [Data](#) > **m_values**
- const QIcon **m_checkedIcon**
- const QIcon **m_uncheckedIcon**

The documentation for this class was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.634 QtBrowserItem Class Reference**Public Member Functions**

- [QtProperty](#) * **property** () const
- [QtBrowserItem](#) * **parent** () const
- QList< [QtBrowserItem](#) * > **children** () const
- [QtAbstractPropertyBrowser](#) * **browser** () const

Friends

- class **QtAbstractPropertyBrowserPrivate**

15.634.1 Member Function Documentation

15.634.1.1 browser()

`QtAbstractPropertyBrowser * QtBrowserItem::browser ( ) const`

Returns the property browser which owns *this* item.

15.634.1.2 children()

`QList< QtBrowserItem * > QtBrowserItem::children ( ) const`

Returns the children items of *this* item. The properties reproduced from children items are always the same as reproduced from associated property' children, for example:

The *childrenItems* list represents the same list as *childrenProperties*.

15.634.1.3 parent()

`QtBrowserItem * QtBrowserItem::parent ( ) const`

Returns the parent item of *this* item. Returns 0 if *this* item is associated with top-level property in item's property browser.

See also

[children\(\)](#)

15.634.1.4 property()

`QtProperty * QtBrowserItem::property ( ) const`

Returns the property which is accosiated with this item. Note that several items can be associated with the same property instance in the same property browser.

See also

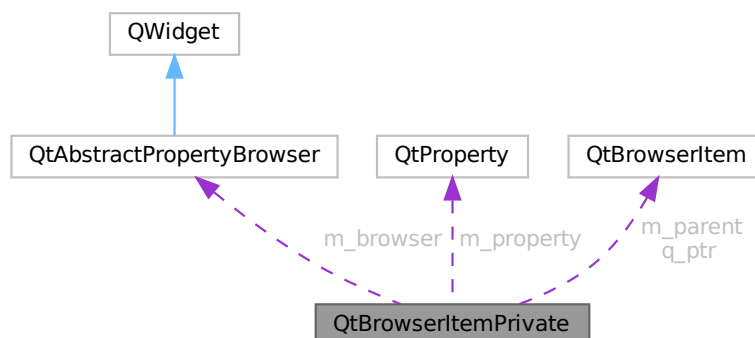
[QtAbstractPropertyBrowser::items\(\)](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.cpp

15.635 QtBrowserItemPrivate Class Reference

Collaboration diagram for QtBrowserItemPrivate:



Public Member Functions

- **QtBrowserItemPrivate** ([QtAbstractPropertyBrowser](#) *browser, [QtProperty](#) *property, [QtBrowserItem](#) *parent)
- void **addChild** ([QtBrowserItem](#) *index, [QtBrowserItem](#) *after)
- void **removeChild** ([QtBrowserItem](#) *index)

Public Attributes

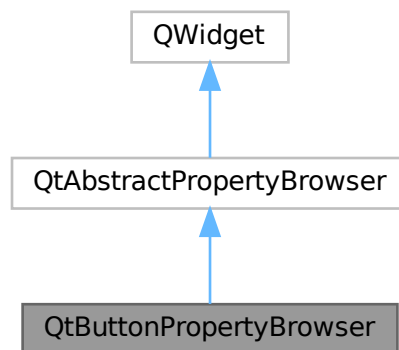
- [QtAbstractPropertyBrowser](#) *const **m_browser**
- [QtProperty](#) * **m_property**
- [QtBrowserItem](#) * **m_parent**
- [QtBrowserItem](#) * **q_ptr**
- QList< [QtBrowserItem](#) * > **m_children**

The documentation for this class was generated from the following file:

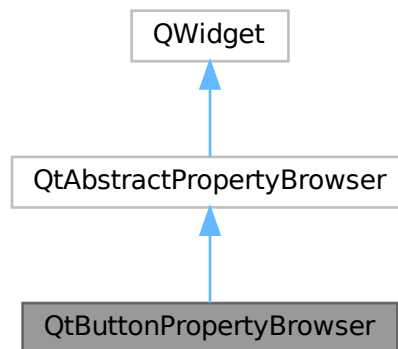
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.cpp

15.636 QPushButtonPropertyBrowser Class Reference

Inheritance diagram for QPushButtonPropertyBrowser:



Collaboration diagram for `QtButtonPropertyBrowser`:



Signals

- void `collapsed` (`QtBrowserItem *item`)
- void `expanded` (`QtBrowserItem *item`)

Signals inherited from `QtAbstractPropertyBrowser`

- void `currentItemChanged` (`QtBrowserItem *`)

Public Member Functions

- `QtButtonPropertyBrowser` (`QWidget *parent=0`)
- `~QtButtonPropertyBrowser` ()
- void `setExpanded` (`QtBrowserItem *item`, bool `expanded`)
- bool `isExpanded` (`QtBrowserItem *item`) const

Public Member Functions inherited from `QtAbstractPropertyBrowser`

- `QtAbstractPropertyBrowser` (`QWidget *parent=0`)
- `~QtAbstractPropertyBrowser` ()
- `QList< QtProperty * > properties` () const
- `QList< QtBrowserItem * > items` (`QtProperty *property`) const
- `QtBrowserItem * topLevelItem` (`QtProperty *property`) const
- `QList< QtBrowserItem * > topLevelItems` () const
- void `clear` ()
- template<class `PropertyManager` >
void `setFactoryForManager` (`PropertyManager *manager`, `QtAbstractEditorFactory< PropertyManager > *factory`)
- void `unsetFactoryForManager` (`QtAbstractPropertyManager *manager`)
- `QtBrowserItem * currentItem` () const
- void `setCurrentItem` (`QtBrowserItem *`)

Protected Member Functions

- void `itemInserted` (`QtBrowserItem *item`, `QtBrowserItem *afterItem`) override
- void `itemRemoved` (`QtBrowserItem *item`) override
- void `itemChanged` (`QtBrowserItem *item`) override

Protected Member Functions inherited from [QtAbstractPropertyBrowser](#)

- virtual QWidget * [createEditor](#) (QtProperty *property, QWidget *parent)

Additional Inherited Members

Public Slots inherited from [QtAbstractPropertyBrowser](#)

- [QtBrowserItem](#) * [addProperty](#) (QtProperty *property)
- [QtBrowserItem](#) * [insertProperty](#) (QtProperty *property, QtProperty *afterProperty)
- void [removeProperty](#) (QtProperty *property)

15.636.1 Constructor & Destructor Documentation

15.636.1.1 QPushButtonPropertyBrowser()

```
QPushButtonPropertyBrowser::QPushButtonPropertyBrowser (
    QWidget * parent = 0 )
```

Creates a property browser with the given *parent*.

15.636.1.2 ~QPushButtonPropertyBrowser()

```
QPushButtonPropertyBrowser::~~QPushButtonPropertyBrowser ( )
```

Destroys this property browser.

Note that the properties that were inserted into this browser are *not* destroyed since they may still be used in other browsers. The properties are owned by the manager that created them.

See also

[QtProperty](#), [QtAbstractPropertyManager](#)

15.636.2 Member Function Documentation

15.636.2.1 collapsed

```
void QPushButtonPropertyBrowser::collapsed (
    QtBrowserItem * item ) [signal]
```

This signal is emitted when the *item* is collapsed.

See also

[expanded\(\)](#), [setExpanded\(\)](#)

15.636.2.2 expanded

```
void QPushButtonPropertyBrowser::expanded (
    QtBrowserItem * item ) [signal]
```

This signal is emitted when the *item* is expanded.

See also

[collapsed\(\)](#), [setExpanded\(\)](#)

15.636.2.3 isExpanded()

```
bool QPushButtonPropertyBrowser::isExpanded (
    QtBrowserItem * item ) const
```

Returns true if the *item* is expanded; otherwise returns false.

See also

[setExpanded\(\)](#)

15.636.2.4 itemChanged()

```
void QtButtonPropertyBrowser::itemChanged (
    QtBrowserItem * item ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyBrowser](#).

15.636.2.5 itemInserted()

```
void QtButtonPropertyBrowser::itemInserted (
    QtBrowserItem * item,
    QtBrowserItem * afterItem ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyBrowser](#).

15.636.2.6 itemRemoved()

```
void QtButtonPropertyBrowser::itemRemoved (
    QtBrowserItem * item ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyBrowser](#).

15.636.2.7 setExpanded()

```
void QtButtonPropertyBrowser::setExpanded (
    QtBrowserItem * item,
    bool expanded )
```

Sets the *item* to either collapse or expanded, depending on the value of *expanded*.

See also

[isExpanded\(\)](#), [expanded\(\)](#), [collapsed\(\)](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtbuttonpropertybrowser.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtbuttonpropertybrowser.cpp

15.637 QtButtonPropertyBrowserPrivate Class Reference**Classes**

- struct [WidgetItem](#)

Public Member Functions

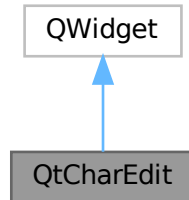
- void **init** (QWidget *parent)
- void **propertyInserted** ([QtBrowserItem](#) *index, [QtBrowserItem](#) *afterIndex)
- void **propertyRemoved** ([QtBrowserItem](#) *index)
- void **propertyChanged** ([QtBrowserItem](#) *index)
- QWidget * **createEditor** ([QtProperty](#) *property, QWidget *parent) const
- void **slotEditorDestroyed** ()
- void **slotUpdate** ()
- void **slotToggled** (bool checked)

The documentation for this class was generated from the following file:

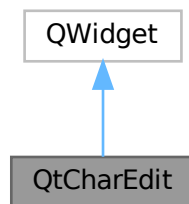
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtbuttonpropertybrowser.cpp

15.638 QtCharEdit Class Reference

Inheritance diagram for QtCharEdit:



Collaboration diagram for QtCharEdit:



Public Slots

- void **setValue** (const QChar &value)

Signals

- void **valueChanged** (const QChar &value)

Public Member Functions

- **QtCharEdit** (QWidget *parent=0)
- QChar **value** () const
- bool **eventFilter** (QObject *o, QEvent *e) override

Protected Member Functions

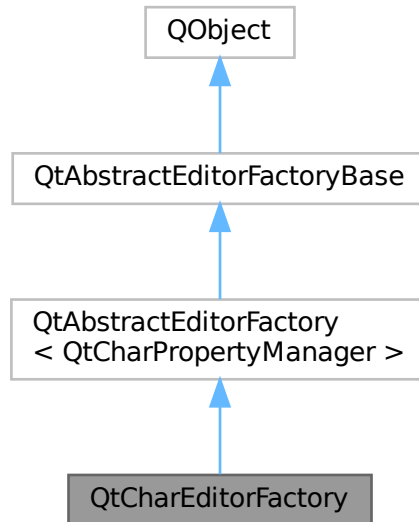
- void **focusInEvent** (QFocusEvent *e) override
- void **focusOutEvent** (QFocusEvent *e) override
- void **keyPressEvent** (QKeyEvent *e) override
- void **keyReleaseEvent** (QKeyEvent *e) override
- bool **event** (QEvent *e) override

The documentation for this class was generated from the following file:

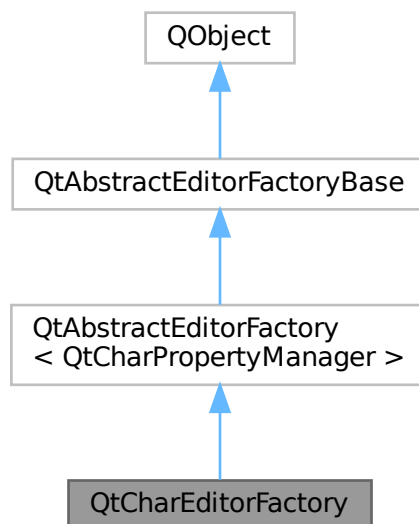
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.639 QtCharEditorFactory Class Reference

Inheritance diagram for QtCharEditorFactory:



Collaboration diagram for QtCharEditorFactory:



Public Member Functions

- [QtCharEditorFactory](#) (QObject *parent=0)

- [~QtCharEditorFactory\(\)](#)

Public Member Functions inherited from

[QtAbstractEditorFactory](#) < [QtCharPropertyManager](#) >

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtCharPropertyManager *manager)
- void [removePropertyManager](#) (QtCharPropertyManager *manager)
- QSet< [QtCharPropertyManager](#) * > [propertyManagers](#) () const
- [QtCharPropertyManager](#) * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtCharPropertyManager *manager) override
- QWidget * [createEditor](#) (QtCharPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtCharPropertyManager *manager) override

Protected Member Functions inherited from

[QtAbstractEditorFactory](#) < [QtCharPropertyManager](#) >

- virtual QWidget * [createEditor](#) (QtCharPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtCharPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtCharPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

15.639.1 Constructor & Destructor Documentation

15.639.1.1 QtCharEditorFactory()

```
QtCharEditorFactory::QtCharEditorFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

15.639.1.2 ~QtCharEditorFactory()

```
QtCharEditorFactory::~~QtCharEditorFactory ( )
```

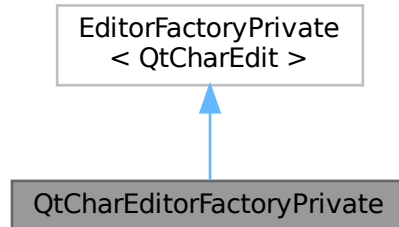
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

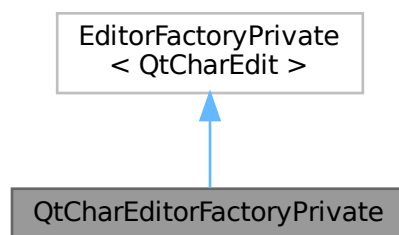
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.640 QtCharEditorFactoryPrivate Class Reference

Inheritance diagram for QtCharEditorFactoryPrivate:



Collaboration diagram for QtCharEditorFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, const QChar &value)
- void **slotSetValue** (const QChar &value)

Public Member Functions inherited from [EditorFactoryPrivate< QtCharEdit >](#)

- [QtCharEdit](#) * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, [QtCharEdit](#) *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QtCharEdit >](#)

- typedef QList< [QtCharEdit](#) * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< [QtCharEdit](#) *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate< QtCharEdit >](#)

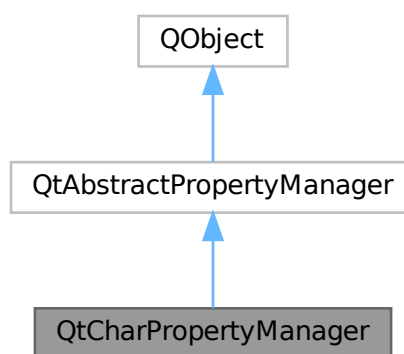
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

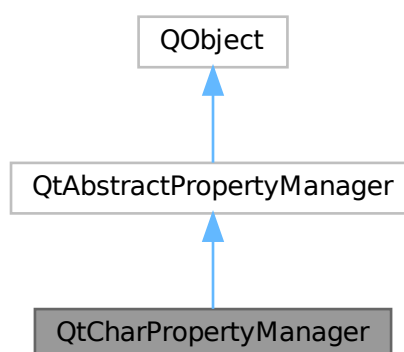
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.641 QtCharPropertyManager Class Reference

Inheritance diagram for QtCharPropertyManager:



Collaboration diagram for QtCharPropertyManager:

**Public Slots**

- void [setValue](#) ([QtProperty](#) *property, const QChar &val)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QChar &val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtCharPropertyManager](#) (QObject *parent=0)
- [~QtCharPropertyManager](#) ()
- QChar [value](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.641.1 Constructor & Destructor Documentation

15.641.1.1 [QtCharPropertyManager](#)()

```
QtCharPropertyManager::QtCharPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.641.1.2 [~QtCharPropertyManager](#)()

```
QtCharPropertyManager::~~QtCharPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.641.2 Member Function Documentation

15.641.2.1 [initializeProperty](#)()

```
void QtCharPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.641.2.2 setValue

```
void QtCharPropertyManager::setValue (
    QtProperty * property,
    const QChar & value ) [slot]
```

Sets the value of the given *property* to *value*.

See also

[value\(\)](#), [valueChanged\(\)](#)

15.641.2.3 uninitializedProperty()

```
void QtCharPropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.641.2.4 value()

```
QChar QtCharPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns an null QChar object.

See also

[setValue\(\)](#)

15.641.2.5 valueChanged

```
void QtCharPropertyManager::valueChanged (
    QtProperty * property,
    const QChar & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

15.641.2.6 valueText()

```
QString QtCharPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.642 QtCharPropertyManagerPrivate Class Reference**Public Types**

- typedef QMap< const [QtProperty](#) *, QChar > **PropertyValueMap**

Public Attributes

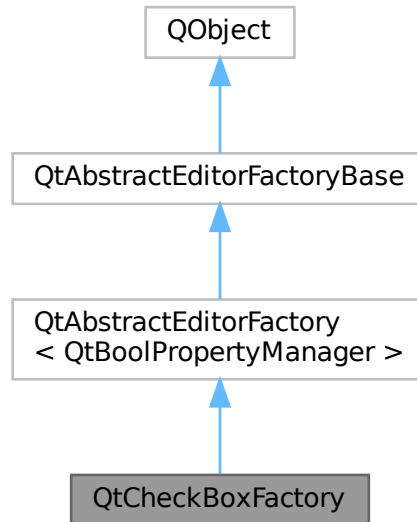
- PropertyValueMap **m_values**

The documentation for this class was generated from the following file:

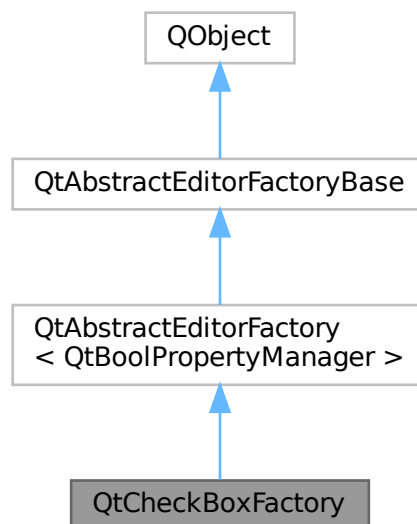
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.643 QtCheckBoxFactory Class Reference

Inheritance diagram for QtCheckBoxFactory:



Collaboration diagram for QtCheckBoxFactory:



Public Member Functions

- [QtCheckBoxFactory](#) (QObject *parent=0)

- [~QtCheckBoxFactory](#) ()

Public Member Functions inherited from

[QtAbstractEditorFactory](#) < [QtBoolPropertyManager](#) >

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtBoolPropertyManager *manager)
- void [removePropertyManager](#) (QtBoolPropertyManager *manager)
- QSet< [QtBoolPropertyManager](#) * > [propertyManagers](#) () const
- [QtBoolPropertyManager](#) * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtBoolPropertyManager *manager) override
- QWidget * [createEditor](#) (QtBoolPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtBoolPropertyManager *manager) override

Protected Member Functions inherited from

[QtAbstractEditorFactory](#) < [QtBoolPropertyManager](#) >

- virtual QWidget * [createEditor](#) (QtBoolPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtBoolPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtBoolPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

15.643.1 Constructor & Destructor Documentation

15.643.1.1 QtCheckBoxFactory()

```
QtCheckBoxFactory::QtCheckBoxFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

15.643.1.2 ~QtCheckBoxFactory()

```
QtCheckBoxFactory::~QtCheckBoxFactory ( )
```

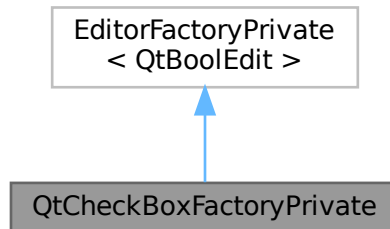
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

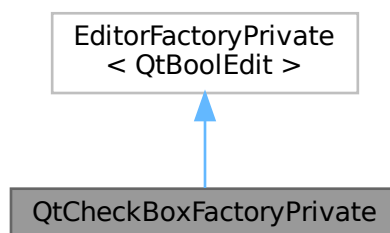
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.644 QtCheckBoxFactoryPrivate Class Reference

Inheritance diagram for QtCheckBoxFactoryPrivate:



Collaboration diagram for QtCheckBoxFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, bool value)
- void **slotSetValue** (bool value)

Public Member Functions inherited from [EditorFactoryPrivate< QtBoolEdit >](#)

- [QtBoolEdit](#) * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, [QtBoolEdit](#) *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QtBoolEdit >](#)

- typedef QList< [QtBoolEdit](#) * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< [QtBoolEdit](#) *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate](#)< [QtBoolEdit](#) >

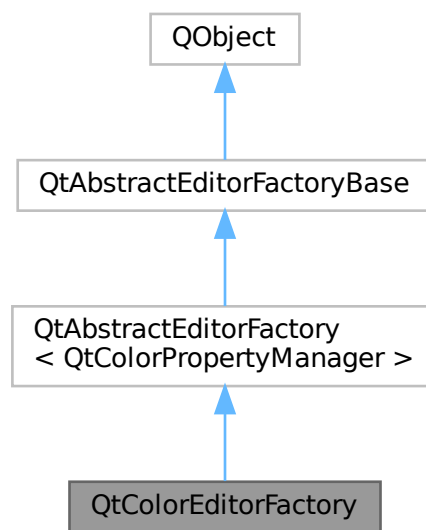
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

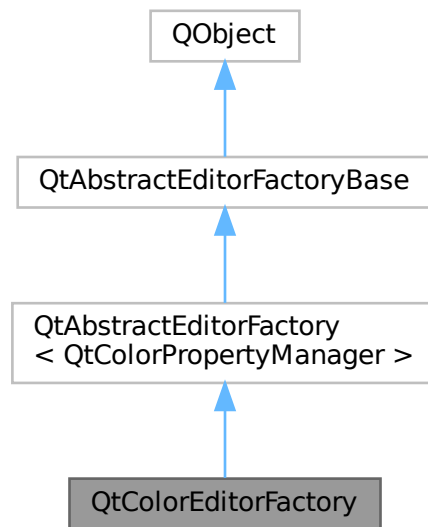
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.645 QtColorEditorFactory Class Reference

Inheritance diagram for QtColorEditorFactory:



Collaboration diagram for QColorEditorFactory:



Public Member Functions

- [QColorEditorFactory](#) (QObject *parent=0)
- [~QColorEditorFactory](#) ()

Public Member Functions inherited from [QtAbstractEditorFactory< QColorPropertyManager >](#)

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtColorPropertyManager *manager)
- void [removePropertyManager](#) (QtColorPropertyManager *manager)
- QSet< QtColorPropertyManager * > [propertyManagers](#) () const
- QtColorPropertyManager * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtColorPropertyManager *manager) override
- QWidget * [createEditor](#) (QtColorPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtColorPropertyManager *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactory< QColorPropertyManager >](#)

- virtual QWidget * [createEditor](#) (QtColorPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtColorPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtColorPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from QColorAbstractEditorFactoryBase

- QColorAbstractEditorFactoryBase (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from QColorAbstractEditorFactoryBase

15.645.1 Constructor & Destructor Documentation

15.645.1.1 QColorEditorFactory()

```
QColorEditorFactory::QColorEditorFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

15.645.1.2 ~QColorEditorFactory()

```
QColorEditorFactory::~~QColorEditorFactory ( )
```

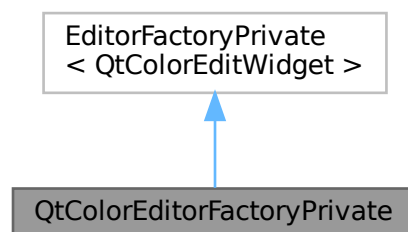
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

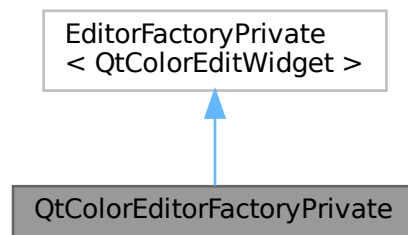
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.646 QColorEditorFactoryPrivate Class Reference

Inheritance diagram for QColorEditorFactoryPrivate:



Collaboration diagram for QColorEditorFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, const QColor &value)
- void **slotSetValue** (const QColor &value)

Public Member Functions inherited from [EditorFactoryPrivate< QColorEditWidget >](#)

- [QColorEditWidget](#) * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, [QColorEditWidget](#) *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QColorEditWidget >](#)

- typedef QList< [QColorEditWidget](#) * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< [QColorEditWidget](#) *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate< QColorEditWidget >](#)

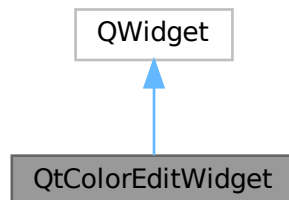
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

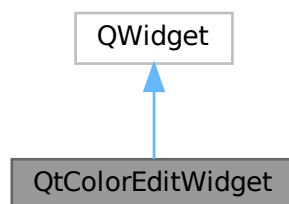
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.647 QColorEditWidget Class Reference

Inheritance diagram for QColorEditWidget:



Collaboration diagram for QColorEditWidget:



Public Slots

- void **setValue** (const QColor &value)

Signals

- void **valueChanged** (const QColor &value)

Public Member Functions

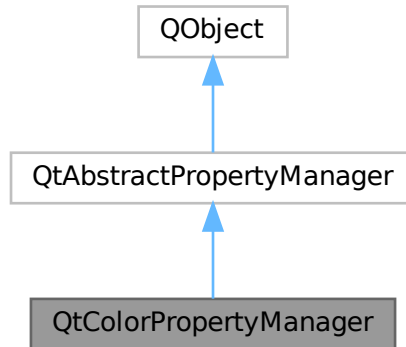
- **QColorEditWidget** (QWidget *parent)
- bool **eventFilter** (QObject *obj, QEvent *ev) override

The documentation for this class was generated from the following file:

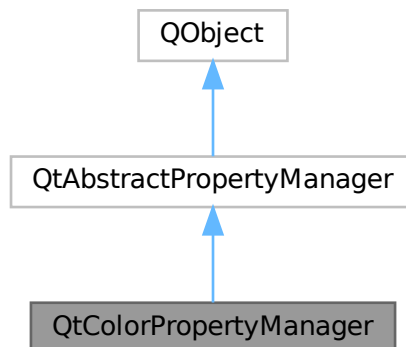
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.648 QColorPropertyManager Class Reference

Inheritance diagram for QColorPropertyManager:



Collaboration diagram for QColorPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, const QColor &val)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QColor &val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QColorPropertyManager](#) (QObject *parent=0)
- [~QColorPropertyManager](#) ()
- [QtIntPropertyManager](#) * [subIntPropertyManager](#) () const
- QColor [value](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- QIcon [valueIcon](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.648.1 Constructor & Destructor Documentation**15.648.1.1 QColorPropertyManager()**

```
QColorPropertyManager::QColorPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.648.1.2 ~QColorPropertyManager()

```
QColorPropertyManager::~~QColorPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.648.2 Member Function Documentation**15.648.2.1 initializeProperty()**

```
void QColorPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.648.2.2 setValue

```
void QColorPropertyManager::setValue (
    QtProperty * property,
    const QColor & value ) [slot]
```

Sets the value of the given *property* to *value*. Nested properties are updated automatically.

See also

[value\(\)](#), [valueChanged\(\)](#)

15.648.2.3 subIntPropertyManager()

`QtIntPropertyManager * QtColorPropertyManager::subIntPropertyManager ( ) const`

Returns the manager that produces the nested *red*, *green* and *blue* subproperties.

In order to provide editing widgets for the subproperties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

15.648.2.4 uninitializedProperty()

`void QtColorPropertyManager::uninitializedProperty (
 QtProperty * property) [override], [protected], [virtual]`

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.648.2.5 value()

`QColor QtColorPropertyManager::value (
 const QtProperty * property) const`

Returns the given *property*'s value.

If the given *property* is not managed by *this* manager, this function returns an invalid color.

See also

[setValue\(\)](#)

15.648.2.6 valueChanged

`void QtColorPropertyManager::valueChanged (
 QtProperty * property,
 const QColor & value) [signal]`

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

15.648.2.7 valueIcon()

`QIcon QtColorPropertyManager::valueIcon (
 const QtProperty * property) const [override], [protected], [virtual]`

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.648.2.8 valueText()

`QString QtColorPropertyManager::valueText (
 const QtProperty * property) const [override], [protected], [virtual]`

\reimp

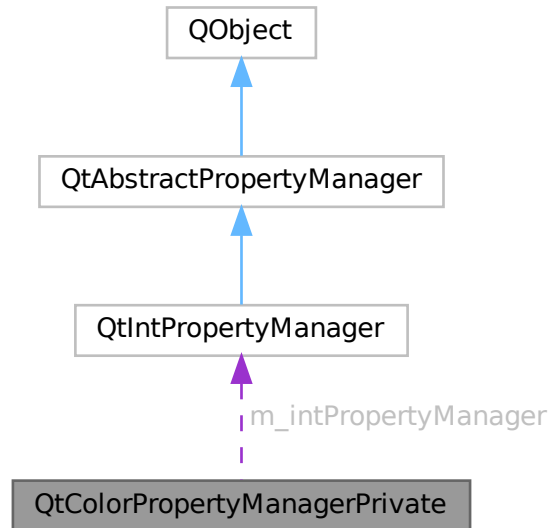
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.649 QColorPropertyManagerPrivate Class Reference

Collaboration diagram for QColorPropertyManagerPrivate:



Public Types

- typedef QMap< const QtProperty *, QColor > **PropertyValueMap**

Public Member Functions

- void **slotIntChanged** (QtProperty *property, int value)
- void **slotPropertyDestroyed** (QtProperty *property)

Public Attributes

- PropertyValueMap **m_values**
- QtIntPropertyManager * **m_intPropertyManager**
- QMap< const QtProperty *, QtProperty * > **m_propertyToR**
- QMap< const QtProperty *, QtProperty * > **m_propertyToG**
- QMap< const QtProperty *, QtProperty * > **m_propertyToB**
- QMap< const QtProperty *, QtProperty * > **m_propertyToA**
- QMap< const QtProperty *, QtProperty * > **m_rToProperty**
- QMap< const QtProperty *, QtProperty * > **m_gToProperty**
- QMap< const QtProperty *, QtProperty * > **m_bToProperty**
- QMap< const QtProperty *, QtProperty * > **m_aToProperty**

The documentation for this class was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.650 QtCursorDatabase Class Reference

Public Member Functions

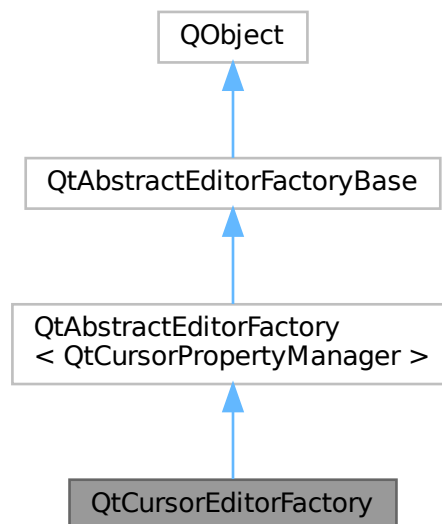
- void **clear** ()
- QStringList **cursorShapeNames** () const
- QMap< int, QIcon > **cursorShapelcons** () const
- QString **cursorToShapeName** (const QCursor &cursor) const
- QIcon **cursorToShapelcon** (const QCursor &cursor) const
- int **cursorToValue** (const QCursor &cursor) const
- QCursor **valueToCursor** (int value) const

The documentation for this class was generated from the following files:

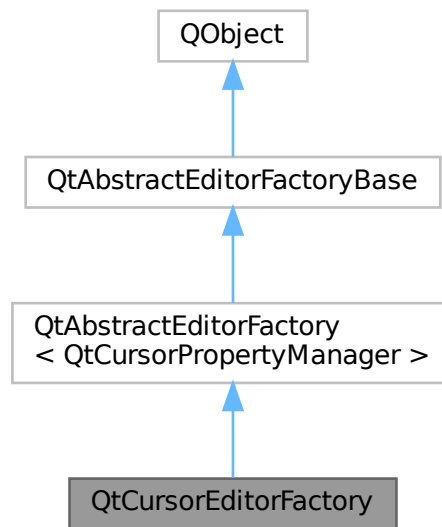
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils_p.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils.cpp

15.651 QtCursorEditorFactory Class Reference

Inheritance diagram for QtCursorEditorFactory:



Collaboration diagram for QtCursorEditorFactory:



Public Member Functions

- [QtCursorEditorFactory](#) (QObject *parent=0)
- [~QtCursorEditorFactory](#) ()

Public Member Functions inherited from [QtAbstractEditorFactory< QtCursorPropertyManager >](#)

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtCursorPropertyManager *manager)
- void [removePropertyManager](#) (QtCursorPropertyManager *manager)
- QSet< QtCursorPropertyManager * > [propertyManagers](#) () const
- QtCursorPropertyManager * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtCursorPropertyManager *manager) override
- QWidget * [createEditor](#) (QtCursorPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtCursorPropertyManager *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactory< QtCursorPropertyManager >](#)

- virtual QWidget * [createEditor](#) (QtCursorPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtCursorPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtCursorPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

15.651.1 Constructor & Destructor Documentation

15.651.1.1 QtCursorEditorFactory()

```
QtCursorEditorFactory::QtCursorEditorFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

15.651.1.2 ~QtCursorEditorFactory()

```
QtCursorEditorFactory::~~QtCursorEditorFactory ( )
```

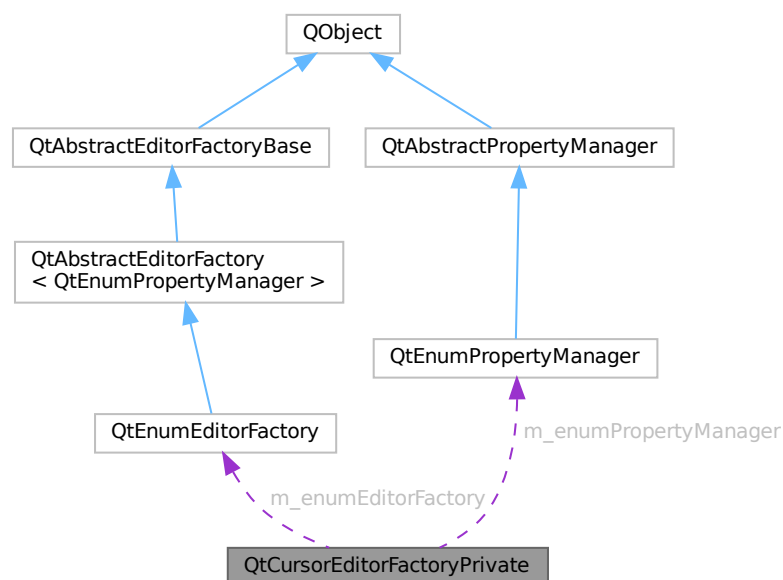
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.652 QtCursorEditorFactoryPrivate Class Reference

Collaboration diagram for QtCursorEditorFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, const QCursor &cursor)
- void **slotEnumChanged** ([QtProperty](#) *property, int value)
- void **slotEditorDestroyed** (QObject *object)

Public Attributes

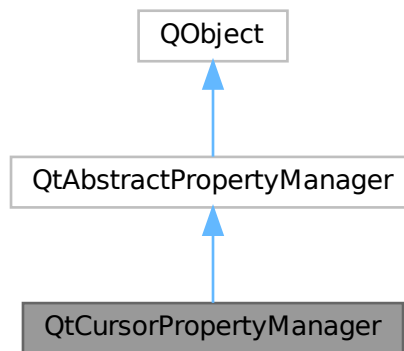
- [QtEnumEditorFactory](#) * **m_enumEditorFactory**
- [QtEnumPropertyManager](#) * **m_enumPropertyManager**
- QMap< [QtProperty](#) *, [QtProperty](#) * > **m_propertyToEnum**
- QMap< [QtProperty](#) *, [QtProperty](#) * > **m_enumToProperty**
- QMap< [QtProperty](#) *, QList > **m_enumToEditors**
- QMap< QWidget *, [QtProperty](#) * > **m_editorToEnum**
- bool **m_updatingEnum**

The documentation for this class was generated from the following file:

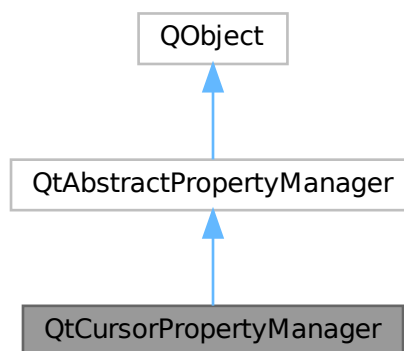
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.653 QtCursorPropertyManager Class Reference

Inheritance diagram for QtCursorPropertyManager:



Collaboration diagram for QtCursorPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, const [QCursor](#) &val)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const [QCursor](#) &val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtCursorPropertyManager](#) ([QObject](#) *parent=0)
- [~QtCursorPropertyManager](#) ()
- [QCursor](#) [value](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) ([QObject](#) *parent=0)
- [~QtAbstractPropertyManager](#) ()
- [QSet](#)< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const [QString](#) &name=[QString](#)())

Protected Member Functions

- [QString](#) [valueText](#) (const [QtProperty](#) *property) const override
- [QIcon](#) [valueIcon](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.653.1 Constructor & Destructor Documentation**15.653.1.1 [QtCursorPropertyManager](#)()**

```
QtCursorPropertyManager::QtCursorPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.653.1.2 [~QtCursorPropertyManager](#)()

```
QtCursorPropertyManager::~~QtCursorPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.653.2 Member Function Documentation

15.653.2.1 initializeProperty()

```
void QtCursorPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.653.2.2 setValue

```
void QtCursorPropertyManager::setValue (
    QtProperty * property,
    const QCursor & value ) [slot]
```

Sets the value of the given *property* to *value*.

See also

[value\(\)](#), [valueChanged\(\)](#)

15.653.2.3 uninitializedProperty()

```
void QtCursorPropertyManager::uninitializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.653.2.4 value()

```
QCursor QtCursorPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns a default QCursor object.

See also

[setValue\(\)](#)

15.653.2.5 valueChanged

```
void QtCursorPropertyManager::valueChanged (
    QtProperty * property,
    const QCursor & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

15.653.2.6 valueIcon()

```
QIcon QtCursorPropertyManager::valueIcon (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.653.2.7 valueText()

```
QString QtCursorPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.654 QtCursorPropertyManagerPrivate Class Reference

Public Types

- typedef QMap< const [QtProperty](#) *, QCursor > **PropertyValueMap**

Public Attributes

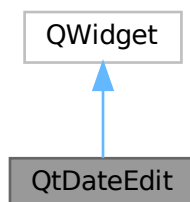
- PropertyValueMap **m_values**

The documentation for this class was generated from the following file:

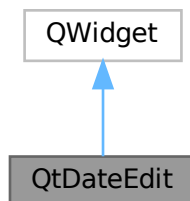
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.655 QtDateEdit Class Reference

Inheritance diagram for QtDateEdit:



Collaboration diagram for QtDateEdit:



Signals

- void **dateChanged** (QDate)

Public Member Functions

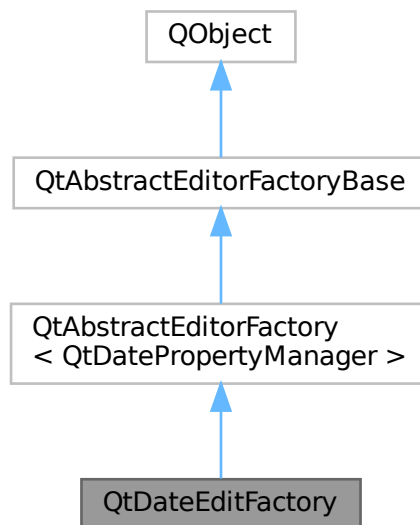
- **QtDateEdit** (QWidget *parent=Q_NULLPTR)
- QDate **date** ()
- void **setDate** (QDate)
- void **setInitialDate** (QDate)
- void **setDateRange** (QDate, QDate)
- void **setDisplayFormat** (QString)
- void **setCalendarPopup** (bool)

The documentation for this class was generated from the following files:

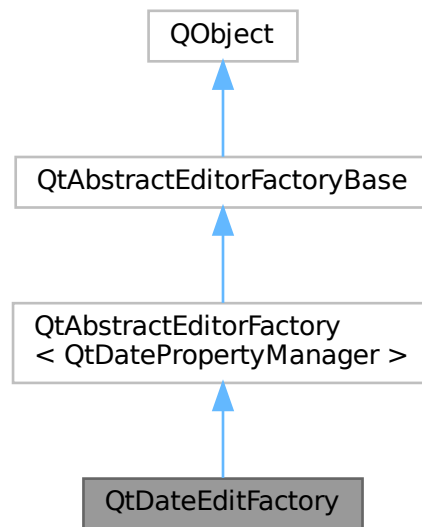
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils_p.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils.cpp

15.656 QtDateEditFactory Class Reference

Inheritance diagram for QtDateEditFactory:



Collaboration diagram for `QtDateEditFactory`:



Public Member Functions

- [QtDateEditFactory](#) (`QObject *parent=0`)
- [~QtDateEditFactory](#) ()

Public Member Functions inherited from [QtAbstractEditorFactory < QtDatePropertyManager >](#)

- [QtAbstractEditorFactory](#) (`QObject *parent`)
- `QWidget *` [createEditor](#) (`QtProperty *property`, `QWidget *parent`) override
- `void` [addPropertyManager](#) (`QtDatePropertyManager *manager`)
- `void` [removePropertyManager](#) (`QtDatePropertyManager *manager`)
- `QSet < QtDatePropertyManager * >` [propertyManagers](#) () const
- `QtDatePropertyManager *` [propertyManager](#) (`QtProperty *property`) const

Protected Member Functions

- `void` [connectPropertyManager](#) (`QtDatePropertyManager *manager`) override
- `QWidget *` [createEditor](#) (`QtDatePropertyManager *manager`, `QtProperty *property`, `QWidget *parent`) override
- `void` [disconnectPropertyManager](#) (`QtDatePropertyManager *manager`) override

Protected Member Functions inherited from [QtAbstractEditorFactory < QtDatePropertyManager >](#)

- `virtual QWidget *` [createEditor](#) (`QtDatePropertyManager *manager`, `QtProperty *property`, `QWidget *parent`)=0
- `virtual void` [connectPropertyManager](#) (`QtDatePropertyManager *manager`)=0
- `virtual void` [disconnectPropertyManager](#) (`QtDatePropertyManager *manager`)=0
- `void` [managerDestroyed](#) (`QObject *manager`) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

15.656.1 Constructor & Destructor Documentation

15.656.1.1 QtDateEditFactory()

```
QtDateEditFactory::QtDateEditFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

15.656.1.2 ~QtDateEditFactory()

```
QtDateEditFactory::~QtDateEditFactory ( )
```

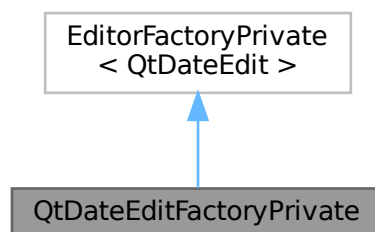
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

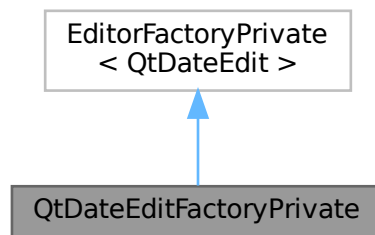
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.657 QtDateEditFactoryPrivate Class Reference

Inheritance diagram for QtDateEditFactoryPrivate:



Collaboration diagram for QtDateEditFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, QDate value)
- void **slotRangeChanged** ([QtProperty](#) *property, QDate min, QDate max)
- void **slotSetValue** (QDate value)

Public Member Functions inherited from [EditorFactoryPrivate< QtDateEdit >](#)

- [QtDateEdit](#) * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, [QtDateEdit](#) *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QtDateEdit >](#)

- typedef QList< [QtDateEdit](#) * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< [QtDateEdit](#) *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate< QtDateEdit >](#)

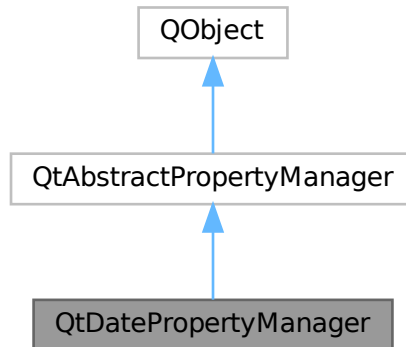
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

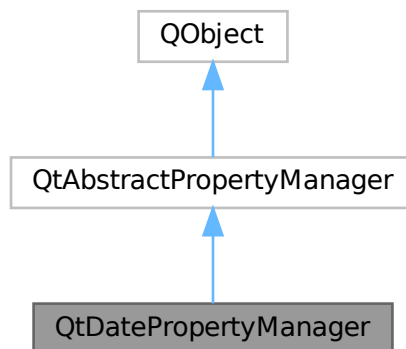
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.658 QDateTimePropertyManager Class Reference

Inheritance diagram for QDateTimePropertyManager:



Collaboration diagram for QDateTimePropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, [QDate](#) val)
- void **setValueOnly** ([QtProperty](#) *property, [QDate](#) val)
- void [setMinimum](#) ([QtProperty](#) *property, [QDate](#) minVal)
- void [setMaximum](#) ([QtProperty](#) *property, [QDate](#) maxVal)
- void [setRange](#) ([QtProperty](#) *property, [QDate](#) minVal, [QDate](#) maxVal)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, [QDate](#) val)
- void [rangeChanged](#) ([QtProperty](#) *property, [QDate](#) minVal, [QDate](#) maxVal)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtDatePropertyManager](#) (QObject *parent=0)
- [~QtDatePropertyManager](#) ()
- QDate [value](#) (const [QtProperty](#) *property) const
- QDate [initialValue](#) (const [QtProperty](#) *property) const
- QDate [minimum](#) (const [QtProperty](#) *property) const
- QDate [maximum](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.658.1 Constructor & Destructor Documentation

15.658.1.1 [QtDatePropertyManager](#)()

```
QtDatePropertyManager::QtDatePropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.658.1.2 [~QtDatePropertyManager](#)()

```
QtDatePropertyManager::~~QtDatePropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.658.2 Member Function Documentation

15.658.2.1 [initializeProperty](#)()

```
void QtDatePropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.658.2.2 maximum()

```
QDate QtDatePropertyManager::maximum (
    const QtProperty * property ) const
```

Returns the given *property*'s maximum date.

See also

[minimum\(\)](#), [setRange\(\)](#)

15.658.2.3 minimum()

```
QDate QtDatePropertyManager::minimum (
    const QtProperty * property ) const
```

Returns the given *property*'s minimum date.

See also

[maximum\(\)](#), [setRange\(\)](#)

15.658.2.4 rangeChanged

```
void QtDatePropertyManager::rangeChanged (
    QtProperty * property,
    QDate minimum,
    QDate maximum ) [signal]
```

This signal is emitted whenever a property created by this manager changes its range of valid dates, passing a pointer to the *property* and the new *minimum* and *maximum* dates.

See also

[setRange\(\)](#)

15.658.2.5 setMaximum

```
void QtDatePropertyManager::setMaximum (
    QtProperty * property,
    QDate maxVal ) [slot]
```

Sets the maximum value for the given *property* to *maxVal*.

When setting the maximum value, the minimum and current values are adjusted if necessary (ensuring that the range remains valid and that the current value is within in the range).

See also

[maximum\(\)](#), [setRange\(\)](#)

15.658.2.6 setMinimum

```
void QtDatePropertyManager::setMinimum (
    QtProperty * property,
    QDate minVal ) [slot]
```

Sets the minimum value for the given *property* to *minVal*.

When setting the minimum value, the maximum and current values are adjusted if necessary (ensuring that the range remains valid and that the current value is within in the range).

See also

[minimum\(\)](#), [setRange\(\)](#)

15.658.2.7 setRange

```
void QtDatePropertyManager::setRange (
    QtProperty * property,
    QDate minimum,
    QDate maximum ) [slot]
```

Sets the range of valid dates.

This is a convenience function defining the range of valid dates in one go; setting the *minimum* and *maximum* values for the given *property* with a single function call.

When setting a new date range, the current value is adjusted if necessary (ensuring that the value remains in date range).

See also

[setMinimum\(\)](#), [setMaximum\(\)](#), [rangeChanged\(\)](#)

15.658.2.8 setValue

```
void QtDatePropertyManager::setValue (
    QtProperty * property,
    QDate value ) [slot]
```

Sets the value of the given *property* to *value*.

If the specified *value* is not a valid date according to the given *property*'s range, the value is adjusted to the nearest valid value within the range.

See also

[value\(\)](#), [setRange\(\)](#), [valueChanged\(\)](#)

15.658.2.9 uninitializedProperty()

```
void QtDatePropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.658.2.10 value()

```
QDate QtDatePropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by *this* manager, this function returns an invalid date.

See also

[setValue\(\)](#)

15.658.2.11 valueChanged

```
void QtDatePropertyManager::valueChanged (
    QtProperty * property,
    QDate value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

15.658.2.12 valueText()

```
QString QtDatePropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.659 QtDatePropertyManagerPrivate Class Reference**Classes**

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Member Functions

- **QtDatePropertyManagerPrivate** ([QtDatePropertyManager](#) *q)

Public Attributes

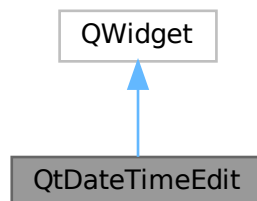
- QString **m_format**
- QMap< const [QtProperty](#) *, [Data](#) > **m_values**

The documentation for this class was generated from the following file:

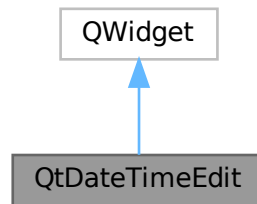
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.660 QtDateTimeEdit Class Reference

Inheritance diagram for QtDateTimeEdit:



Collaboration diagram for QDateTimeEdit:



Signals

- void **dateTimeChanged** (const QDateTime &)

Public Member Functions

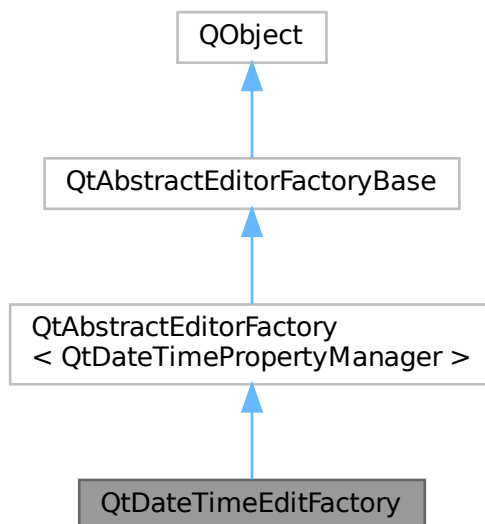
- **QDateTimeEdit** (QWidget *parent=Q_NULLPTR)
- QDateTime **dateTime** ()
- void **setDateTime** (const QDateTime &)
- void **setInitialDateTime** (const QDateTime &)
- void **setDateTimeRange** (const QDateTime &, const QDateTime &)
- void **setDisplayFormat** (const QString &)
- void **setCalendarPopup** (bool)

The documentation for this class was generated from the following files:

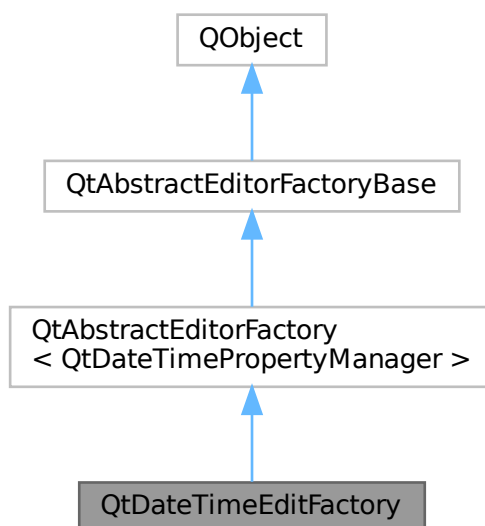
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils_p.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils.cpp

15.661 QDateTimeEditFactory Class Reference

Inheritance diagram for QDateTimeEditFactory:



Collaboration diagram for QDateTimeEditFactory:



Public Member Functions

- [QDateTimeEditFactory](#) (`QObject *parent=0`)

- [~QtDateTimeEditFactory](#) ()

Public Member Functions inherited from

[QtAbstractEditorFactory](#) < [QtDateTimePropertyManager](#) >

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtDateTimePropertyManager *manager)
- void [removePropertyManager](#) (QtDateTimePropertyManager *manager)
- QSet< [QtDateTimePropertyManager](#) * > [propertyManagers](#) () const
- [QtDateTimePropertyManager](#) * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtDateTimePropertyManager *manager) override
- QWidget * [createEditor](#) (QtDateTimePropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtDateTimePropertyManager *manager) override

Protected Member Functions inherited from

[QtAbstractEditorFactory](#) < [QtDateTimePropertyManager](#) >

- virtual QWidget * [createEditor](#) (QtDateTimePropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtDateTimePropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtDateTimePropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

15.661.1 Constructor & Destructor Documentation

15.661.1.1 [QtDateTimeEditFactory](#)()

```
QtDateTimeEditFactory::QtDateTimeEditFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

15.661.1.2 [~QtDateTimeEditFactory](#)()

```
QtDateTimeEditFactory::~~QtDateTimeEditFactory ( )
```

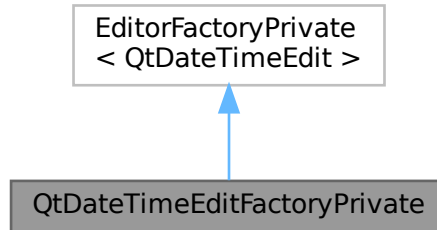
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

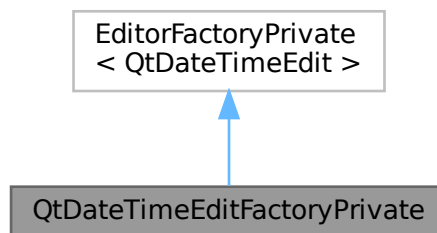
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.662 QDateTimeEditFactoryPrivate Class Reference

Inheritance diagram for QDateTimeEditFactoryPrivate:



Collaboration diagram for QDateTimeEditFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, const QDateTime &value)
- void **slotSetValue** (const QDateTime &value)

Public Member Functions inherited from [EditorFactoryPrivate< QDateTimeEdit >](#)

- [QDateTimeEdit](#) * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, [QDateTimeEdit](#) *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QDateTimeEdit >](#)

- typedef QList< [QDateTimeEdit](#) * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< [QDateTimeEdit](#) *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate](#)< [QtDateTimeEdit](#) >

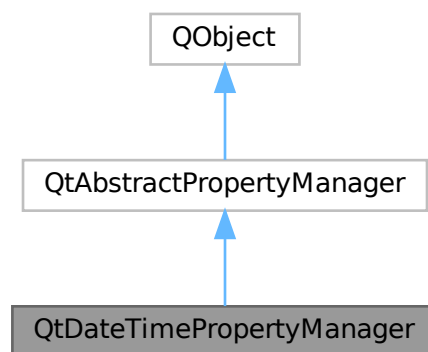
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

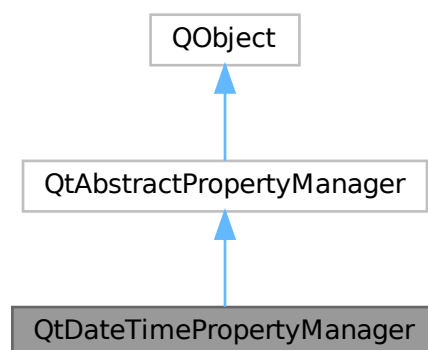
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.663 QtDateTimePropertyManager Class Reference

Inheritance diagram for QtDateTimePropertyManager:



Collaboration diagram for QtDateTimePropertyManager:

**Public Slots**

- void [setValue](#) ([QtProperty](#) *property, const QDateTime &val)
- void **setValueOnly** ([QtProperty](#) *property, const QDateTime &val)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QDateTime &val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QDateTimePropertyManager](#) (QObject *parent=0)
- [~QDateTimePropertyManager](#) ()
- QDateTime [value](#) (const [QtProperty](#) *property) const
- QDateTime [initialValue](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.663.1 Constructor & Destructor Documentation

15.663.1.1 QDateTimePropertyManager()

```
QDateTimePropertyManager::QDateTimePropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.663.1.2 ~QDateTimePropertyManager()

```
QDateTimePropertyManager::~~QDateTimePropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.663.2 Member Function Documentation

15.663.2.1 initializeProperty()

```
void QDateTimePropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.663.2.2 setValue

```
void QDateTimePropertyManager::setValue (
    QtProperty * property,
    const QDateTime & value ) [slot]
```

Sets the value of the given *property* to *value*.

See also

[value\(\)](#), [valueChanged\(\)](#)

15.663.2.3 uninitializedProperty()

```
void QDateTimePropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.663.2.4 value()

```
QDateTime QDateTimePropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns an invalid QDateTime object.

See also

[setValue\(\)](#)

15.663.2.5 valueChanged

```
void QDateTimePropertyManager::valueChanged (
    QtProperty * property,
    const QDateTime & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

15.663.2.6 valueText()

```
QString QDateTimePropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.664 QDateTimePropertyManagerPrivate Class Reference

Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Member Functions

- **QDateTimePropertyManagerPrivate** ([QDateTimePropertyManager](#) *q)

Public Attributes

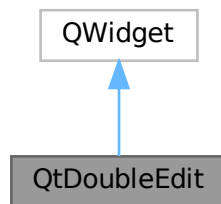
- const QString **m_format**
- PropertyValueMap **m_values**

The documentation for this class was generated from the following file:

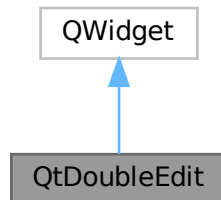
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.665 QtDoubleEdit Class Reference

Inheritance diagram for QtDoubleEdit:



Collaboration diagram for QtDoubleEdit:



Signals

- void **valueChanged** (double)

Public Member Functions

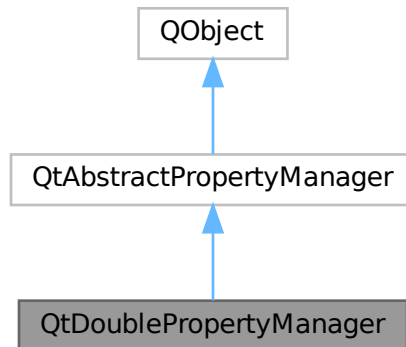
- **QtDoubleEdit** (QWidget *parent=Q_NULLPTR)
- int **value** ()
- void **setValue** (double)
- void **setInitialValue** (double)
- void **setDecimals** (double)
- void **setSingleStep** (double)
- void **setRange** (double, double)
- void **setKeyboardTracking** (bool)

The documentation for this class was generated from the following files:

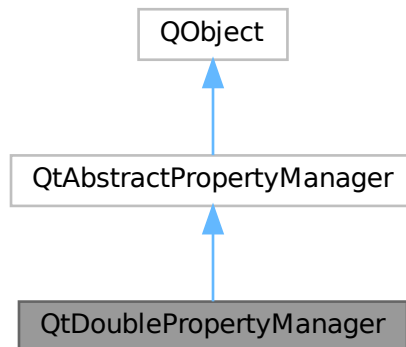
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils_p.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils.cpp

15.666 QtDoublePropertyManager Class Reference

Inheritance diagram for QtDoublePropertyManager:



Collaboration diagram for QtDoublePropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, double val)
- void **setValueOnly** ([QtProperty](#) *property, double val)
- void [setMinimum](#) ([QtProperty](#) *property, double minVal)
- void [setMaximum](#) ([QtProperty](#) *property, double maxVal)
- void [setRange](#) ([QtProperty](#) *property, double minVal, double maxVal)
- void [setSingleStep](#) ([QtProperty](#) *property, double step)
- void [setDecimals](#) ([QtProperty](#) *property, int prec)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, double val)
- void [rangeChanged](#) ([QtProperty](#) *property, double minVal, double maxVal)
- void [singleStepChanged](#) ([QtProperty](#) *property, double step)
- void [decimalsChanged](#) ([QtProperty](#) *property, int prec)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtDoublePropertyManager](#) (QObject *parent=0)
- [~QtDoublePropertyManager](#) ()
- double [value](#) (const [QtProperty](#) *property) const
- double [initialValue](#) (const [QtProperty](#) *property) const
- double [minimum](#) (const [QtProperty](#) *property) const
- double [maximum](#) (const [QtProperty](#) *property) const
- double [singleStep](#) (const [QtProperty](#) *property) const
- int [decimals](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.666.1 Constructor & Destructor Documentation

15.666.1.1 [QtDoublePropertyManager](#)()

```
QtDoublePropertyManager::QtDoublePropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.666.1.2 [~QtDoublePropertyManager](#)()

```
QtDoublePropertyManager::~~QtDoublePropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.666.2 Member Function Documentation

15.666.2.1 decimals()

```
int QtDoublePropertyManager::decimals (
    const QtProperty * property ) const
```

Returns the given *property*'s precision, in decimals.

See also

[setDecimals\(\)](#)

15.666.2.2 decimalsChanged

```
void QtDoublePropertyManager::decimalsChanged (
    QtProperty * property,
    int prec ) [signal]
```

This signal is emitted whenever a property created by this manager changes its precision of value, passing a pointer to the *property* and the new *prec* value

See also

[setDecimals\(\)](#)

15.666.2.3 initializeProperty()

```
void QtDoublePropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.666.2.4 maximum()

```
double QtDoublePropertyManager::maximum (
    const QtProperty * property ) const
```

Returns the given *property*'s maximum value.

See also

[minimum\(\)](#), [setRange\(\)](#)

15.666.2.5 minimum()

```
double QtDoublePropertyManager::minimum (
    const QtProperty * property ) const
```

Returns the given *property*'s minimum value.

See also

[maximum\(\)](#), [setRange\(\)](#)

15.666.2.6 rangeChanged

```
void QtDoublePropertyManager::rangeChanged (
    QtProperty * property,
    double minimum,
    double maximum ) [signal]
```

This signal is emitted whenever a property created by this manager changes its range of valid values, passing a pointer to the *property* and the new *minimum* and *maximum* values

See also

[setRange\(\)](#)

15.666.2.7 setDecimals

```
void QtDoublePropertyManager::setDecimals (
    QtProperty * property,
    int prec ) [slot]
```

Sets the precision of the given *property* to *prec*.
The valid decimal range is 0-13. The default is 2.

See also

[decimals\(\)](#)

15.666.2.8 setMaximum

```
void QtDoublePropertyManager::setMaximum (
    QtProperty * property,
    double maxVal ) [slot]
```

Sets the maximum value for the given *property* to *maxVal*.

When setting the maximum value, the minimum and current values are adjusted if necessary (ensuring that the range remains valid and that the current value is within in the range).

See also

[maximum\(\)](#), [setRange\(\)](#), [rangeChanged\(\)](#)

15.666.2.9 setMinimum

```
void QtDoublePropertyManager::setMinimum (
    QtProperty * property,
    double minVal ) [slot]
```

Sets the minimum value for the given *property* to *minVal*.

When setting the minimum value, the maximum and current values are adjusted if necessary (ensuring that the range remains valid and that the current value is within in the range).

See also

[minimum\(\)](#), [setRange\(\)](#), [rangeChanged\(\)](#)

15.666.2.10 setRange

```
void QtDoublePropertyManager::setRange (
    QtProperty * property,
    double minimum,
    double maximum ) [slot]
```

Sets the range of valid values.

This is a convenience function defining the range of valid values in one go; setting the *minimum* and *maximum* values for the given *property* with a single function call.

When setting a new range, the current value is adjusted if necessary (ensuring that the value remains within range).

See also

[setMinimum\(\)](#), [setMaximum\(\)](#), [rangeChanged\(\)](#)

15.666.2.11 setSingleStep

```
void QtDoublePropertyManager::setSingleStep (
    QtProperty * property,
    double step ) [slot]
```

Sets the step value for the given *property* to *step*.

The step is typically used to increment or decrement a property value while pressing an arrow key.

See also

[singleStep\(\)](#)

15.666.2.12 setValue

```
void QtDoublePropertyManager::setValue (
    QtProperty * property,
    double value ) [slot]
```

Sets the value of the given *property* to *value*.

If the specified *value* is not valid according to the given *property's* range, the *value* is adjusted to the nearest valid value within the range.

See also

[value\(\)](#), [setRange\(\)](#), [valueChanged\(\)](#)

15.666.2.13 singleStep()

```
double QtDoublePropertyManager::singleStep (
    const QtProperty * property ) const
```

Returns the given *property's* step value.

The step is typically used to increment or decrement a property value while pressing an arrow key.

See also

[setSingleStep\(\)](#)

15.666.2.14 singleStepChanged

```
void QtDoublePropertyManager::singleStepChanged (
    QtProperty * property,
    double step ) [signal]
```

This signal is emitted whenever a property created by this manager changes its single step property, passing a pointer to the *property* and the new *step* value

See also

[setSingleStep\(\)](#)

15.666.2.15 uninitializedProperty()

```
void QtDoublePropertyManager::uninitializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.666.2.16 value()

```
double QtDoublePropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property's* value.

If the given property is not managed by this manager, this function returns 0.

See also

[setValue\(\)](#)

15.666.2.17 valueChanged

```
void QtDoublePropertyManager::valueChanged (
    QtProperty * property,
    double value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

15.666.2.18 valueText()

```
QString QtDoublePropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
\reimp
```

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.667 QtDoublePropertyManagerPrivate Class Reference

Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Attributes

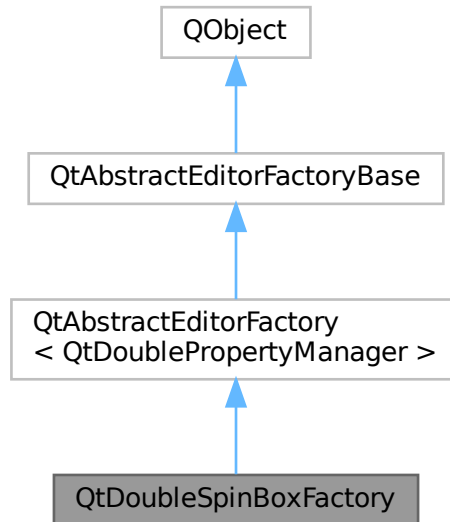
- PropertyValueMap **m_values**

The documentation for this class was generated from the following file:

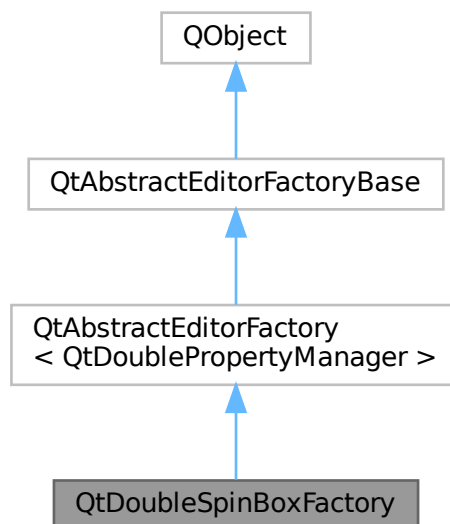
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.668 QtDoubleSpinBoxFactory Class Reference

Inheritance diagram for QtDoubleSpinBoxFactory:



Collaboration diagram for QtDoubleSpinBoxFactory:



Public Member Functions

- [QtDoubleSpinBoxFactory](#) (QObject *parent=0)

- [~QtDoubleSpinBoxFactory](#) ()

Public Member Functions inherited from

[QtAbstractEditorFactory](#) < [QtDoublePropertyManager](#) >

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtDoublePropertyManager *manager)
- void [removePropertyManager](#) (QtDoublePropertyManager *manager)
- QSet< [QtDoublePropertyManager](#) * > [propertyManagers](#) () const
- [QtDoublePropertyManager](#) * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtDoublePropertyManager *manager) override
- QWidget * [createEditor](#) (QtDoublePropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtDoublePropertyManager *manager) override

Protected Member Functions inherited from

[QtAbstractEditorFactory](#) < [QtDoublePropertyManager](#) >

- virtual QWidget * [createEditor](#) (QtDoublePropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtDoublePropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtDoublePropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

15.668.1 Constructor & Destructor Documentation

15.668.1.1 QtDoubleSpinBoxFactory()

```
QtDoubleSpinBoxFactory::QtDoubleSpinBoxFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

15.668.1.2 ~QtDoubleSpinBoxFactory()

```
QtDoubleSpinBoxFactory::~~QtDoubleSpinBoxFactory ( )
```

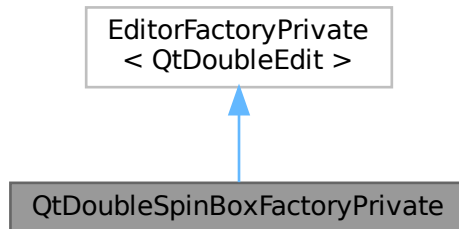
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

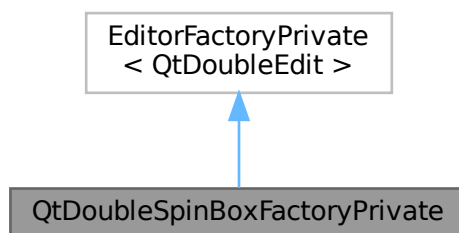
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.669 QtDoubleSpinBoxFactoryPrivate Class Reference

Inheritance diagram for QtDoubleSpinBoxFactoryPrivate:



Collaboration diagram for QtDoubleSpinBoxFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, double value)
- void **slotRangeChanged** ([QtProperty](#) *property, double min, double max)
- void **slotSingleStepChanged** ([QtProperty](#) *property, double step)
- void **slotDecimalsChanged** ([QtProperty](#) *property, int prec)
- void **slotSetValue** (double value)

Public Member Functions inherited from [EditorFactoryPrivate< QtDoubleEdit >](#)

- [QtDoubleEdit](#) * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, [QtDoubleEdit](#) *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QtDoubleEdit >](#)

- typedef QList< [QtDoubleEdit](#) * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< [QtDoubleEdit](#) *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate](#)< [QtDoubleEdit](#) >

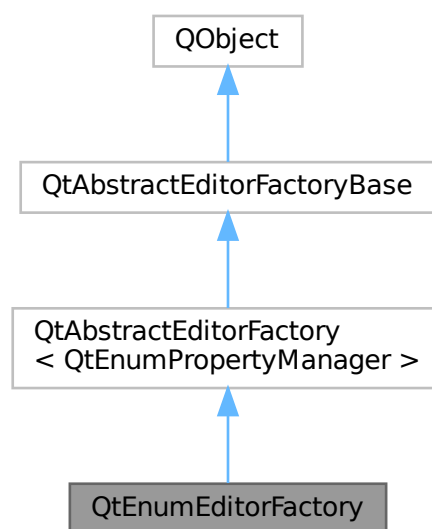
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

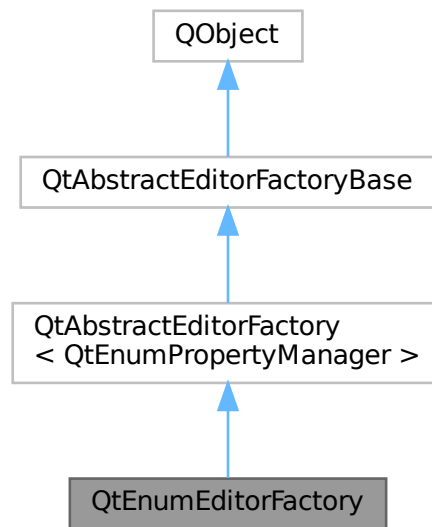
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.670 QtEnumEditorFactory Class Reference

Inheritance diagram for QtEnumEditorFactory:



Collaboration diagram for `QtEnumEditorFactory`:



Public Member Functions

- [QtEnumEditorFactory](#) (`QObject *parent=0`)
- [~QtEnumEditorFactory](#) ()

Public Member Functions inherited from [QtAbstractEditorFactory < QtEnumPropertyManager >](#)

- [QtAbstractEditorFactory](#) (`QObject *parent`)
- `QWidget *` [createEditor](#) (`QtProperty *property`, `QWidget *parent`) override
- `void` [addPropertyManager](#) (`QtEnumPropertyManager *manager`)
- `void` [removePropertyManager](#) (`QtEnumPropertyManager *manager`)
- `QSet < QtEnumPropertyManager * >` [propertyManagers](#) () const
- `QtEnumPropertyManager *` [propertyManager](#) (`QtProperty *property`) const

Protected Member Functions

- `void` [connectPropertyManager](#) (`QtEnumPropertyManager *manager`) override
- `QWidget *` [createEditor](#) (`QtEnumPropertyManager *manager`, `QtProperty *property`, `QWidget *parent`) override
- `void` [disconnectPropertyManager](#) (`QtEnumPropertyManager *manager`) override

Protected Member Functions inherited from [QtAbstractEditorFactory < QtEnumPropertyManager >](#)

- `virtual QWidget *` [createEditor](#) (`QtEnumPropertyManager *manager`, `QtProperty *property`, `QWidget *parent`)=0
- `virtual void` [connectPropertyManager](#) (`QtEnumPropertyManager *manager`)=0
- `virtual void` [disconnectPropertyManager](#) (`QtEnumPropertyManager *manager`)=0
- `void` [managerDestroyed](#) (`QObject *manager`) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

15.670.1 Constructor & Destructor Documentation

15.670.1.1 QtEnumEditorFactory()

```
QtEnumEditorFactory::QtEnumEditorFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

15.670.1.2 ~QtEnumEditorFactory()

```
QtEnumEditorFactory::~~QtEnumEditorFactory ( )
```

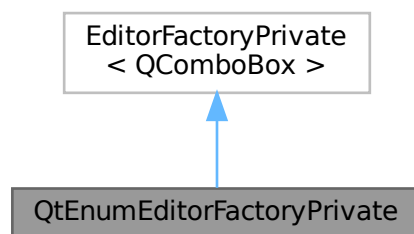
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

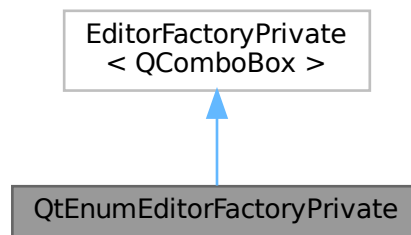
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.671 QtEnumEditorFactoryPrivate Class Reference

Inheritance diagram for QtEnumEditorFactoryPrivate:



Collaboration diagram for QtEnumEditorFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, int value)
- void **slotEnumNamesChanged** ([QtProperty](#) *property, const QStringList &)
- void **slotEnumIconsChanged** ([QtProperty](#) *property, const QMap< int, QIcon > &)
- void **slotSetValue** (int value)

Public Member Functions inherited from [EditorFactoryPrivate< QComboBox >](#)

- QComboBox * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, QComboBox *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QComboBox >](#)

- typedef QList< QComboBox * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< QComboBox *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate< QComboBox >](#)

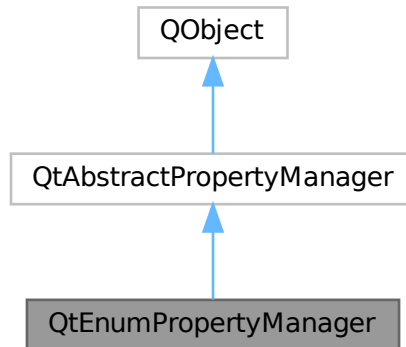
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

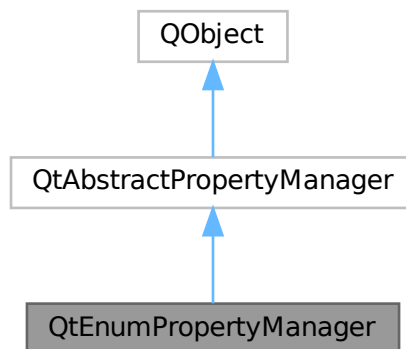
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.672 QtEnumPropertyManager Class Reference

Inheritance diagram for QtEnumPropertyManager:



Collaboration diagram for QtEnumPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, int val)
- void [setEnumNames](#) ([QtProperty](#) *property, const QStringList &names)
- void [setEnumIcons](#) ([QtProperty](#) *property, const QMap< int, QIcon > &icons)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, int val)
- void [enumNamesChanged](#) ([QtProperty](#) *property, const QStringList &names)
- void [enumIconsChanged](#) ([QtProperty](#) *property, const QMap< int, QIcon > &icons)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtEnumPropertyManager](#) (QObject *parent=0)
- [~QtEnumPropertyManager](#) ()
- int [value](#) (const [QtProperty](#) *property) const
- QStringList [enumNames](#) (const [QtProperty](#) *property) const
- QMap< int, QIcon > [enumIcons](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- QIcon [valueIcon](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.672.1 Constructor & Destructor Documentation

15.672.1.1 [QtEnumPropertyManager](#)()

```
QtEnumPropertyManager::QtEnumPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.672.1.2 [~QtEnumPropertyManager](#)()

```
QtEnumPropertyManager::~~QtEnumPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.672.2 Member Function Documentation

15.672.2.1 [enumIcons](#)()

```
QMap< int, QIcon > QtEnumPropertyManager::enumIcons (
    const QtProperty * property ) const
```

Returns the given *property*'s map of enum values to their icons.

See also

[value\(\)](#), [setEnumIcons\(\)](#)

15.672.2.2 enumIconsChanged

```
void QtEnumPropertyManager::enumIconsChanged (
    QtProperty * property,
    const QMap< int, QIcon > & icons ) [signal]
```

This signal is emitted whenever a property created by this manager changes its enum icons, passing a pointer to the *property* and the new mapping of values to *icons* as parameters.

See also

[setEnumIcons\(\)](#)

15.672.2.3 enumNames()

```
QStringList QtEnumPropertyManager::enumNames (
    const QtProperty * property ) const
```

Returns the given *property*'s list of enum names.

See also

[value\(\)](#), [setEnumNames\(\)](#)

15.672.2.4 enumNamesChanged

```
void QtEnumPropertyManager::enumNamesChanged (
    QtProperty * property,
    const QStringList & names ) [signal]
```

This signal is emitted whenever a property created by this manager changes its enum names, passing a pointer to the *property* and the new *names* as parameters.

See also

[setEnumNames\(\)](#)

15.672.2.5 initializeProperty()

```
void QtEnumPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.672.2.6 setEnumIcons

```
void QtEnumPropertyManager::setEnumIcons (
    QtProperty * property,
    const QMap< int, QIcon > & enumIcons ) [slot]
```

Sets the given *property*'s map of enum values to their icons to *enumIcons*.

Each enum value can have associated icon. This association is represented with passed *enumIcons* map.

See also

[enumNames\(\)](#), [enumNamesChanged\(\)](#)

15.672.2.7 setEnumNames

```
void QtEnumPropertyManager::setEnumNames (
    QtProperty * property,
    const QStringList & enumNames ) [slot]
```

Sets the given *property*'s list of enum names to *enumNames*. The *property*'s current value is reset to 0 indicating the first item of the list.

If the specified *enumNames* list is empty, the *property*'s current value is set to -1.

See also

[enumNames\(\)](#), [enumNamesChanged\(\)](#)

15.672.2.8 setValue

```
void QtEnumPropertyManager::setValue (
    QtProperty * property,
    int value ) [slot]
```

Sets the value of the given *property* to *value*.

The specified *value* must be less than the size of the given *property*'s [enumNames\(\)](#) list. Use -1 to leave the editor without a current selection.

See also

[value\(\)](#), [valueChanged\(\)](#)

15.672.2.9 uninitializedProperty()

```
void QtEnumPropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.672.2.10 value()

```
int QtEnumPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value which is an index in the list returned by [enumNames\(\)](#)

If the given property is not managed by this manager, this function returns -1.

See also

[enumNames\(\)](#), [setValue\(\)](#)

15.672.2.11 valueChanged

```
void QtEnumPropertyManager::valueChanged (
    QtProperty * property,
    int value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

15.672.2.12 valueIcon()

```
QIcon QtEnumPropertyManager::valueIcon (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.672.2.13 valueText()

```
QString QtEnumPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.673 QtEnumPropertyManagerPrivate Class Reference

Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Attributes

- PropertyValueMap **m_values**

The documentation for this class was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

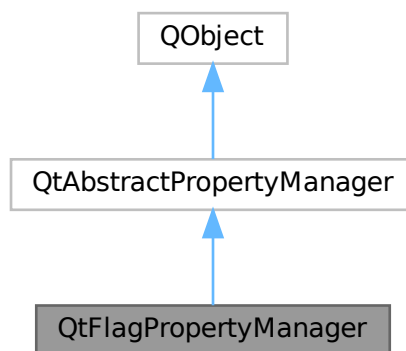
15.674 QtEnumPropertyType Class Reference

The documentation for this class was generated from the following file:

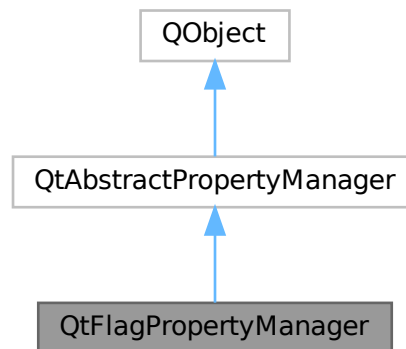
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.cpp

15.675 QtFlagPropertyManager Class Reference

Inheritance diagram for QtFlagPropertyManager:



Collaboration diagram for QtFlagPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, int val)
- void [setFlagNames](#) ([QtProperty](#) *property, const QStringList &names)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, int val)
- void [flagNamesChanged](#) ([QtProperty](#) *property, const QStringList &names)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtFlagPropertyManager](#) (QObject *parent=0)
- [~QtFlagPropertyManager](#) ()
- [QtBoolPropertyManager](#) * [subBoolPropertyManager](#) () const
- int [value](#) (const [QtProperty](#) *property) const
- QStringList [flagNames](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.675.1 Constructor & Destructor Documentation

15.675.1.1 QtFlagPropertyManager()

```
QtFlagPropertyManager::QtFlagPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.675.1.2 ~QtFlagPropertyManager()

```
QtFlagPropertyManager::~~QtFlagPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.675.2 Member Function Documentation

15.675.2.1 flagNames()

```
QStringList QtFlagPropertyManager::flagNames (
    const QtProperty * property ) const
```

Returns the given *property*'s list of flag names.

See also

[value\(\)](#), [setFlagNames\(\)](#)

15.675.2.2 flagNamesChanged

```
void QtFlagPropertyManager::flagNamesChanged (
    QtProperty * property,
    const QStringList & names ) [signal]
```

This signal is emitted whenever a property created by this manager changes its flag names, passing a pointer to the *property* and the new *names* as parameters.

See also

[setFlagNames\(\)](#)

15.675.2.3 initializeProperty()

```
void QtFlagPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.675.2.4 setFlagNames

```
void QtFlagPropertyManager::setFlagNames (
    QtProperty * property,
    const QStringList & flagNames ) [slot]
```

Sets the given *property*'s list of flag names to *flagNames*. The property's current value is reset to 0 indicating the first item of the list.

See also

[flagNames\(\)](#), [flagNamesChanged\(\)](#)

15.675.2.5 setValue

```
void QtFlagPropertyManager::setValue (
    QtProperty * property,
    int value ) [slot]
```

Sets the value of the given *property* to *value*. Nested properties are updated automatically.

The specified *value* must be less than the binary combination of the property's [flagNames\(\)](#) list size (i.e. less than $2^{\sup n}$, where n is the size of the list) and larger than (or equal to) 0.

See also

[value\(\)](#), [valueChanged\(\)](#)

15.675.2.6 subBoolPropertyManager()

```
QtBoolPropertyManager * QtFlagPropertyManager::subBoolPropertyManager ( ) const
```

Returns the manager that produces the nested boolean subproperties representing each flag.

In order to provide editing widgets for the subproperties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

15.675.2.7 uninitializedProperty()

```
void QtFlagPropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.675.2.8 value()

```
int QtFlagPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given property is not managed by this manager, this function returns 0.

See also

[flagNames\(\)](#), [setValue\(\)](#)

15.675.2.9 valueChanged

```
void QtFlagPropertyManager::valueChanged (
    QtProperty * property,
    int value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

15.675.2.10 valueText()

```
QString QtFlagPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

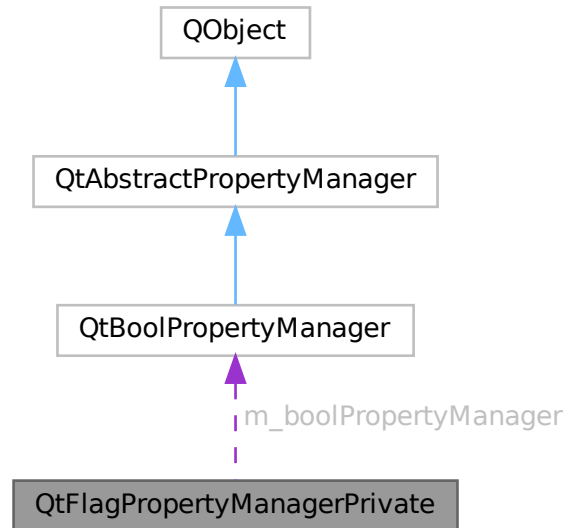
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.676 QtFlagPropertyManagerPrivate Class Reference

Collaboration diagram for QtFlagPropertyManagerPrivate:



Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Member Functions

- void **slotBoolChanged** ([QtProperty](#) *property, bool value)
- void **slotPropertyDestroyed** ([QtProperty](#) *property)

Public Attributes

- PropertyValueMap **m_values**
- [QtBoolPropertyManager](#) * **m_boolPropertyManager**
- QMap< const [QtProperty](#) *, QList< [QtProperty](#) * > > **m_propertyToFlags**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_flagToProperty**

The documentation for this class was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

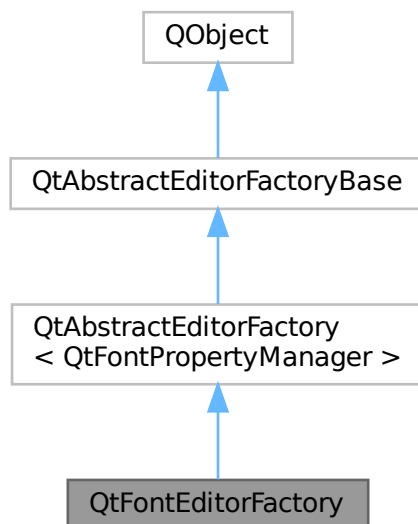
15.677 QtFlagPropertyType Class Reference

The documentation for this class was generated from the following file:

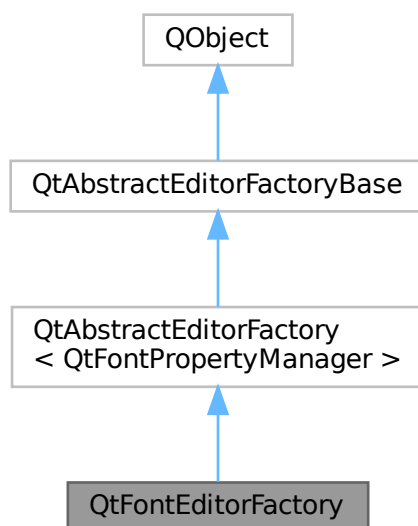
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.cpp

15.678 QFontEditorFactory Class Reference

Inheritance diagram for QFontEditorFactory:



Collaboration diagram for QFontEditorFactory:



Public Member Functions

- [QFontEditorFactory](#) (`QObject *parent=0`)

- [~QtFontEditorFactory\(\)](#)

Public Member Functions inherited from

[QtAbstractEditorFactory](#) < [QtFontPropertyManager](#) >

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtFontPropertyManager *manager)
- void [removePropertyManager](#) (QtFontPropertyManager *manager)
- QSet< [QtFontPropertyManager](#) * > [propertyManagers](#) () const
- [QtFontPropertyManager](#) * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtFontPropertyManager *manager) override
- QWidget * [createEditor](#) (QtFontPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtFontPropertyManager *manager) override

Protected Member Functions inherited from

[QtAbstractEditorFactory](#) < [QtFontPropertyManager](#) >

- virtual QWidget * [createEditor](#) (QtFontPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtFontPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtFontPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

15.678.1 Constructor & Destructor Documentation

15.678.1.1 QtFontEditorFactory()

```
QtFontEditorFactory::QtFontEditorFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

15.678.1.2 ~QtFontEditorFactory()

```
QtFontEditorFactory::~QtFontEditorFactory ( )
```

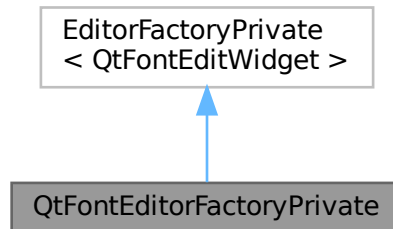
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

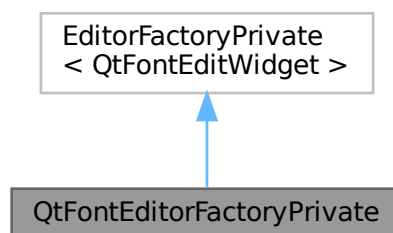
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.679 QFontEditorFactoryPrivate Class Reference

Inheritance diagram for QFontEditorFactoryPrivate:



Collaboration diagram for QFontEditorFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, const QFont &value)
- void **slotSetValue** (const QFont &value)

Public Member Functions inherited from [EditorFactoryPrivate< QFontEditWidget >](#)

- [QFontEditWidget](#) * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, [QFontEditWidget](#) *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QFontEditWidget >](#)

- typedef QList< [QFontEditWidget](#) * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< [QFontEditWidget](#) *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate](#)< [QFontEditWidget](#) >

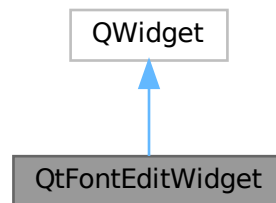
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

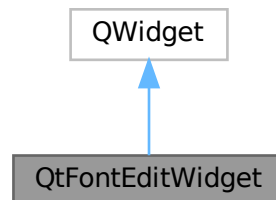
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.680 QFontEditWidget Class Reference

Inheritance diagram for QFontEditWidget:



Collaboration diagram for QFontEditWidget:

**Public Slots**

- void **setValue** (const QFont &value)

Signals

- void **valueChanged** (const QFont &value)

Public Member Functions

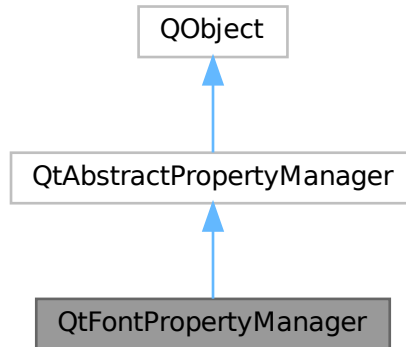
- **QFontEditWidget** (QWidget *parent)
- bool **eventFilter** (QObject *obj, QEvent *ev) override

The documentation for this class was generated from the following file:

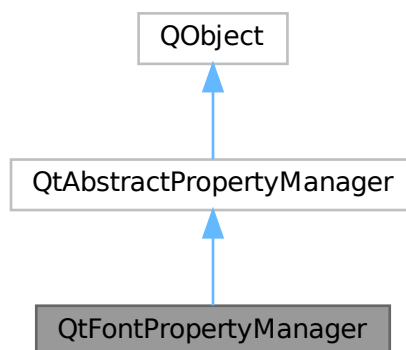
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.681 QFontPropertyManager Class Reference

Inheritance diagram for QFontPropertyManager:



Collaboration diagram for QFontPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, const QFont &val)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QFont &val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QFontPropertyManager](#) (QObject *parent=0)
- [~QFontPropertyManager](#) ()
- [QtIntPropertyManager](#) * [subIntPropertyManager](#) () const
- [QTypeEnumPropertyManager](#) * [subEnumPropertyManager](#) () const
- [QtBoolPropertyManager](#) * [subBoolPropertyManager](#) () const
- QFont [value](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- QIcon [valueIcon](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.681.1 Constructor & Destructor Documentation**15.681.1.1 QFontPropertyManager()**

```
QFontPropertyManager::QFontPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.681.1.2 ~QFontPropertyManager()

```
QFontPropertyManager::~~QFontPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.681.2 Member Function Documentation**15.681.2.1 initializeProperty()**

```
void QFontPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.681.2.2 setValue

```
void QFontPropertyManager::setValue (
    QtProperty * property,
    const QFont & value ) [slot]
```

Sets the value of the given *property* to *value*. Nested properties are updated automatically.

See also

[value\(\)](#), [valueChanged\(\)](#)

15.681.2.3 subBoolPropertyManager()

[QtBoolPropertyManager](#) * [QtFontPropertyManager::subBoolPropertyManager](#) () const

Returns the manager that creates the *bold*, *italic*, *underline*, *strikeOut* and *kerling* subproperties.

In order to provide editing widgets for the mentioned properties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

15.681.2.4 subEnumPropertyManager()

[QtEnumPropertyManager](#) * [QtFontPropertyManager::subEnumPropertyManager](#) () const

Returns the manager that create the *family* subproperty.

In order to provide editing widgets for the *family* property in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

15.681.2.5 subIntPropertyManager()

[QtIntPropertyManager](#) * [QtFontPropertyManager::subIntPropertyManager](#) () const

Returns the manager that creates the *pointSize* subproperty.

In order to provide editing widgets for the *pointSize* property in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

15.681.2.6 uninitializedProperty()

```
void QtFontPropertyManager::uninitializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.681.2.7 value()

```
QFont QtFontPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given property is not managed by this manager, this function returns a font object that uses the application's default font.

See also

[setValue\(\)](#)

15.681.2.8 valueChanged

```
void QFontPropertyManager::valueChanged (
    QtProperty * property,
    const QFont & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

15.681.2.9 valueIcon()

```
QIcon QFontPropertyManager::valueIcon (
    const QtProperty * property ) const [override], [protected], [virtual]
\reimp
```

Reimplemented from [QtAbstractPropertyManager](#).

15.681.2.10 valueText()

```
QString QFontPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
\reimp
```

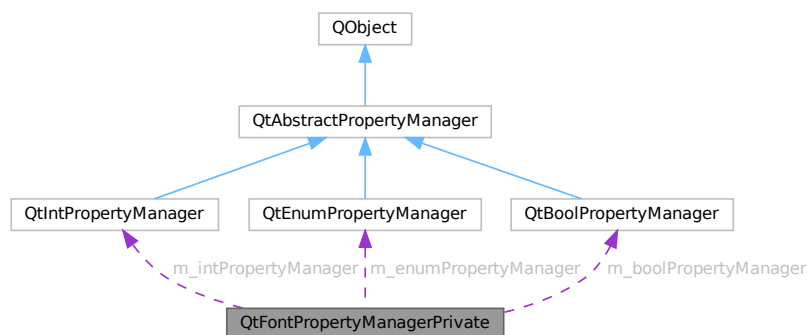
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.682 QFontPropertyManagerPrivate Class Reference

Collaboration diagram for QFontPropertyManagerPrivate:



Public Types

- typedef QMap< const [QtProperty](#) *, QFont > **PropertyValueMap**

Public Member Functions

- void **slotIntChanged** ([QtProperty](#) *property, int value)
- void **slotEnumChanged** ([QtProperty](#) *property, int value)
- void **slotBoolChanged** ([QtProperty](#) *property, bool value)

- void **slotPropertyDestroyed** (QtProperty *property)
- void **slotFontDatabaseChanged** ()
- void **slotFontDatabaseDelayedChange** ()

Public Attributes

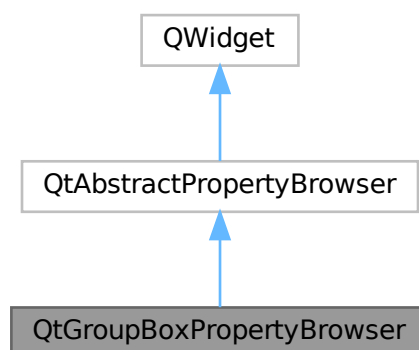
- QStringList **m_familyNames**
- PropertyValueMap **m_values**
- QtIntPropertyManager * **m_intPropertyManager**
- QtEnumPropertyManager * **m_enumPropertyManager**
- QtBoolPropertyManager * **m_boolPropertyManager**
- QMap< const QtProperty *, QtProperty * > **m_propertyToFamily**
- QMap< const QtProperty *, QtProperty * > **m_propertyToPointSize**
- QMap< const QtProperty *, QtProperty * > **m_propertyToBold**
- QMap< const QtProperty *, QtProperty * > **m_propertyToItalic**
- QMap< const QtProperty *, QtProperty * > **m_propertyToUnderline**
- QMap< const QtProperty *, QtProperty * > **m_propertyToStrikeOut**
- QMap< const QtProperty *, QtProperty * > **m_propertyToKerning**
- QMap< const QtProperty *, QtProperty * > **m_familyToProperty**
- QMap< const QtProperty *, QtProperty * > **m_pointSizeToProperty**
- QMap< const QtProperty *, QtProperty * > **m_boldToProperty**
- QMap< const QtProperty *, QtProperty * > **m_italicToProperty**
- QMap< const QtProperty *, QtProperty * > **m_underlineToProperty**
- QMap< const QtProperty *, QtProperty * > **m_strikeOutToProperty**
- QMap< const QtProperty *, QtProperty * > **m_kerningToProperty**
- bool **m_settingValue**
- QTimer * **m_fontDatabaseChangeTimer**

The documentation for this class was generated from the following file:

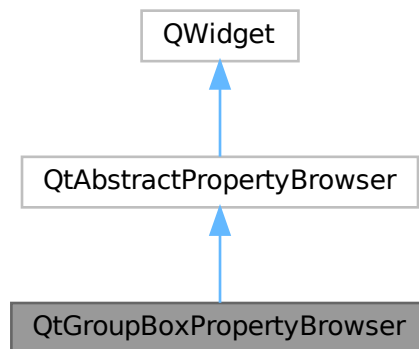
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.683 QtGroupBoxPropertyBrowser Class Reference

Inheritance diagram for QtGroupBoxPropertyBrowser:



Collaboration diagram for QtGroupBoxPropertyBrowser:



Public Member Functions

- [QtGroupBoxPropertyBrowser](#) (QWidget *parent=0)
- [~QtGroupBoxPropertyBrowser](#) ()

Public Member Functions inherited from [QtAbstractPropertyBrowser](#)

- [QtAbstractPropertyBrowser](#) (QWidget *parent=0)
- [~QtAbstractPropertyBrowser](#) ()
- [QList< QtProperty * > properties](#) () const
- [QList< QtBrowserItem * > items](#) (QtProperty *property) const
- [QtBrowserItem * topLevelItem](#) (QtProperty *property) const
- [QList< QtBrowserItem * > topLevelItems](#) () const
- void [clear](#) ()
- template<class PropertyManager >
void [setFactoryForManager](#) (PropertyManager *manager, [QtAbstractEditorFactory](#)< PropertyManager > *factory)
- void [unsetFactoryForManager](#) (QtAbstractPropertyManager *manager)
- [QtBrowserItem * currentItem](#) () const
- void [setCurrentItem](#) (QtBrowserItem *)

Protected Member Functions

- void [itemInserted](#) (QtBrowserItem *item, QtBrowserItem *afterItem) override
- void [itemRemoved](#) (QtBrowserItem *item) override
- void [itemChanged](#) (QtBrowserItem *item) override

Protected Member Functions inherited from [QtAbstractPropertyBrowser](#)

- virtual QWidget * [createEditor](#) (QtProperty *property, QWidget *parent)

Additional Inherited Members

Public Slots inherited from [QtAbstractPropertyBrowser](#)

- [QtBrowserItem * addProperty](#) (QtProperty *property)
- [QtBrowserItem * insertProperty](#) (QtProperty *property, QtProperty *afterProperty)
- void [removeProperty](#) (QtProperty *property)

Signals inherited from [QtAbstractPropertyBrowser](#)

- void [currentItemChanged](#) ([QtBrowserItem](#) *)

15.683.1 Constructor & Destructor Documentation

15.683.1.1 [QtGroupBoxPropertyBrowser\(\)](#)

```
QtGroupBoxPropertyBrowser::QtGroupBoxPropertyBrowser (
    QWidget * parent = 0 )
```

Creates a property browser with the given *parent*.

15.683.1.2 [~QtGroupBoxPropertyBrowser\(\)](#)

```
QtGroupBoxPropertyBrowser::~QtGroupBoxPropertyBrowser ( )
```

Destroys this property browser.

Note that the properties that were inserted into this browser are *not* destroyed since they may still be used in other browsers. The properties are owned by the manager that created them.

See also

[QtProperty](#), [QtAbstractPropertyManager](#)

15.683.2 Member Function Documentation

15.683.2.1 [itemChanged\(\)](#)

```
void QtGroupBoxPropertyBrowser::itemChanged (
    QtBrowserItem * item ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyBrowser](#).

15.683.2.2 [itemInserted\(\)](#)

```
void QtGroupBoxPropertyBrowser::itemInserted (
    QtBrowserItem * item,
    QtBrowserItem * afterItem ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyBrowser](#).

15.683.2.3 [itemRemoved\(\)](#)

```
void QtGroupBoxPropertyBrowser::itemRemoved (
    QtBrowserItem * item ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyBrowser](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtgroupboxpropertybrowser.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtgroupboxpropertybrowser.cpp

15.684 QtGroupBoxPropertyBrowserPrivate Class Reference

Classes

- struct [WidgetItem](#)

Public Member Functions

- void **init** (QWidget *parent)
- void **propertyInserted** ([QtBrowserItem](#) *index, [QtBrowserItem](#) *afterIndex)

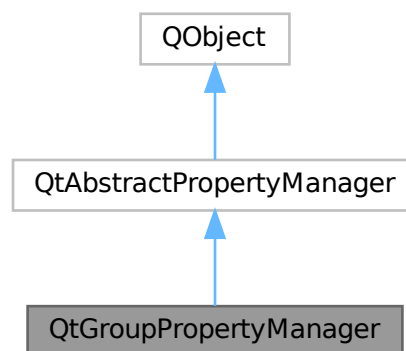
- void **propertyRemoved** ([QtBrowserItem](#) *index)
- void **propertyChanged** ([QtBrowserItem](#) *index)
- QWidget * **createEditor** ([QtProperty](#) *property, QWidget *parent) const
- void **slotEditorDestroyed** ()
- void **slotUpdate** ()

The documentation for this class was generated from the following file:

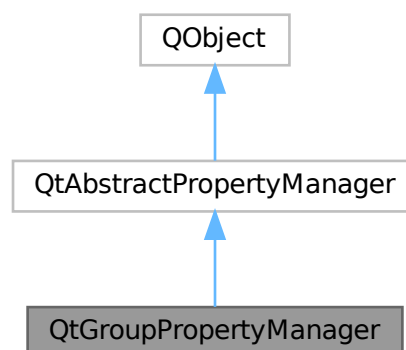
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtgroupboxpropertybrowser.cpp

15.685 QtGroupPropertyManager Class Reference

Inheritance diagram for QtGroupPropertyManager:



Collaboration diagram for QtGroupPropertyManager:



Public Member Functions

- [QtGroupPropertyManager](#) (QObject *parent=0)
- [~QtGroupPropertyManager](#) ()

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- bool [hasValue](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual QString [valueText](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

Additional Inherited Members

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

15.685.1 Constructor & Destructor Documentation

15.685.1.1 [QtGroupPropertyManager](#)()

```
QtGroupPropertyManager::QtGroupPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.685.1.2 [~QtGroupPropertyManager](#)()

```
QtGroupPropertyManager::~QtGroupPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.685.2 Member Function Documentation

15.685.2.1 [hasValue](#)()

```
bool QtGroupPropertyManager::hasValue (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.685.2.2 [initializeProperty](#)()

```
void QtGroupPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.685.2.3 uninitializedProperty()

```
void QtGroupPropertyManager::uninitializeProperty (  
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

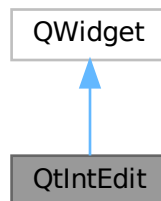
15.686 QtGroupPropertyType Class Reference

The documentation for this class was generated from the following file:

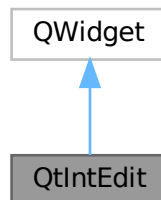
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.cpp

15.687 QtIntEdit Class Reference

Inheritance diagram for QtIntEdit:



Collaboration diagram for QtIntEdit:



Signals

- void **valueChanged** (int)

Public Member Functions

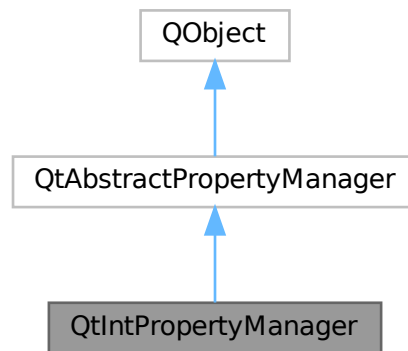
- **QtIntEdit** (QWidget *parent=Q_NULLPTR)
- int **value** ()
- void **setValue** (int)
- void **setInitialValue** (int)
- void **setSingleStep** (int)
- void **setRange** (int, int)
- void **setKeyboardTracking** (bool)

The documentation for this class was generated from the following files:

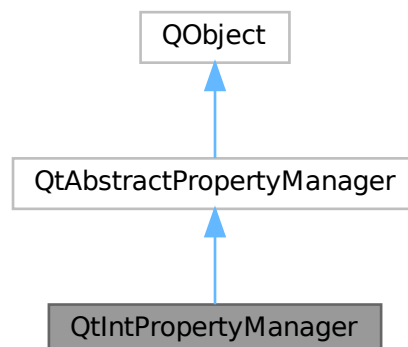
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils_p.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils.cpp

15.688 QtIntPropertyManager Class Reference

Inheritance diagram for QtIntPropertyManager:



Collaboration diagram for QtIntPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, int val)
- void [setValueOnly](#) ([QtProperty](#) *property, int val)
- void [setMinimum](#) ([QtProperty](#) *property, int minVal)
- void [setMaximum](#) ([QtProperty](#) *property, int maxVal)
- void [setRange](#) ([QtProperty](#) *property, int minVal, int maxVal)
- void [setSingleStep](#) ([QtProperty](#) *property, int step)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, int val)
- void [rangeChanged](#) ([QtProperty](#) *property, int minVal, int maxVal)
- void [singleStepChanged](#) ([QtProperty](#) *property, int step)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtIntPropertyManager](#) (QObject *parent=0)
- [~QtIntPropertyManager](#) ()
- int [value](#) (const [QtProperty](#) *property) const
- int [initialValue](#) (const [QtProperty](#) *property) const
- int [minimum](#) (const [QtProperty](#) *property) const
- int [maximum](#) (const [QtProperty](#) *property) const
- int [singleStep](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.688.1 Constructor & Destructor Documentation**15.688.1.1 [QtIntPropertyManager](#)()**

```
QtIntPropertyManager::QtIntPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.688.1.2 ~QtIntPropertyManager()

`QtIntPropertyManager::~QtIntPropertyManager ( )`
Destroys this manager, and all the properties it has created.

15.688.2 Member Function Documentation

15.688.2.1 initializeProperty()

```
void QtIntPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
\reimp
Implements QtAbstractPropertyManager.
```

15.688.2.2 maximum()

```
int QtIntPropertyManager::maximum (
    const QtProperty * property ) const
```

Returns the given *property*'s maximum value.

See also

[setMaximum\(\)](#), [minimum\(\)](#), [setRange\(\)](#)

15.688.2.3 minimum()

```
int QtIntPropertyManager::minimum (
    const QtProperty * property ) const
```

Returns the given *property*'s minimum value.

See also

[setMinimum\(\)](#), [maximum\(\)](#), [setRange\(\)](#)

15.688.2.4 rangeChanged

```
void QtIntPropertyManager::rangeChanged (
    QtProperty * property,
    int minimum,
    int maximum ) [signal]
```

This signal is emitted whenever a property created by this manager changes its range of valid values, passing a pointer to the *property* and the new *minimum* and *maximum* values.

See also

[setRange\(\)](#)

15.688.2.5 setMaximum

```
void QtIntPropertyManager::setMaximum (
    QtProperty * property,
    int maxVal ) [slot]
```

Sets the maximum value for the given *property* to *maxVal*.

When setting maximum value, the minimum and current values are adjusted if necessary (ensuring that the range remains valid and that the current value is within the range).

See also

[maximum\(\)](#), [setRange\(\)](#), [rangeChanged\(\)](#)

15.688.2.6 setMinimum

```
void QtIntPropertyManager::setMinimum (
    QtProperty * property,
    int minVal ) [slot]
```

Sets the minimum value for the given *property* to *minVal*.

When setting the minimum value, the maximum and current values are adjusted if necessary (ensuring that the range remains valid and that the current value is within the range).

See also

[minimum\(\)](#), [setRange\(\)](#), [rangeChanged\(\)](#)

15.688.2.7 setRange

```
void QtIntPropertyManager::setRange (
    QtProperty * property,
    int minimum,
    int maximum ) [slot]
```

Sets the range of valid values.

This is a convenience function defining the range of valid values in one go; setting the *minimum* and *maximum* values for the given *property* with a single function call.

When setting a new range, the current value is adjusted if necessary (ensuring that the value remains within range).

See also

[setMinimum\(\)](#), [setMaximum\(\)](#), [rangeChanged\(\)](#)

15.688.2.8 setSingleStep

```
void QtIntPropertyManager::setSingleStep (
    QtProperty * property,
    int step ) [slot]
```

Sets the step value for the given *property* to *step*.

The step is typically used to increment or decrement a property value while pressing an arrow key.

See also

[singleStep\(\)](#)

15.688.2.9 setValue

```
void QtIntPropertyManager::setValue (
    QtProperty * property,
    int value ) [slot]
```

Sets the value of the given *property* to *value*.

If the specified *value* is not valid according to the given *property's* range, the *value* is adjusted to the nearest valid value within the range.

See also

[value\(\)](#), [setRange\(\)](#), [valueChanged\(\)](#)

15.688.2.10 singleStep()

```
int QtIntPropertyManager::singleStep (
    const QtProperty * property ) const
```

Returns the given *property's* step value.

The step is typically used to increment or decrement a property value while pressing an arrow key.

See also

[setSingleStep\(\)](#)

15.688.2.11 singleStepChanged

```
void QtIntPropertyManager::singleStepChanged (
    QtProperty * property,
    int step ) [signal]
```

This signal is emitted whenever a property created by this manager changes its single step property, passing a pointer to the *property* and the new *step* value

See also

[setSingleStep\(\)](#)

15.688.2.12 uninitializedProperty()

```
void QtIntPropertyManager::uninitializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.688.2.13 value()

```
int QtIntPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given property is not managed by this manager, this function returns 0.

See also

[setValue\(\)](#)

15.688.2.14 valueChanged

```
void QtIntPropertyManager::valueChanged (
    QtProperty * property,
    int value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

15.688.2.15 valueText()

```
QString QtIntPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.689 QtIntPropertyManagerPrivate Class Reference

Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Attributes

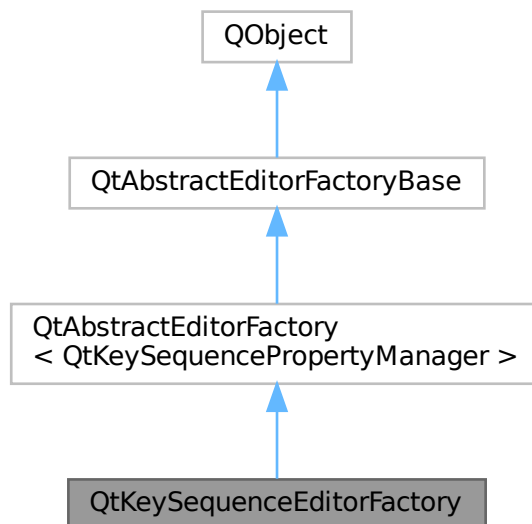
- PropertyValueMap **m_values**

The documentation for this class was generated from the following file:

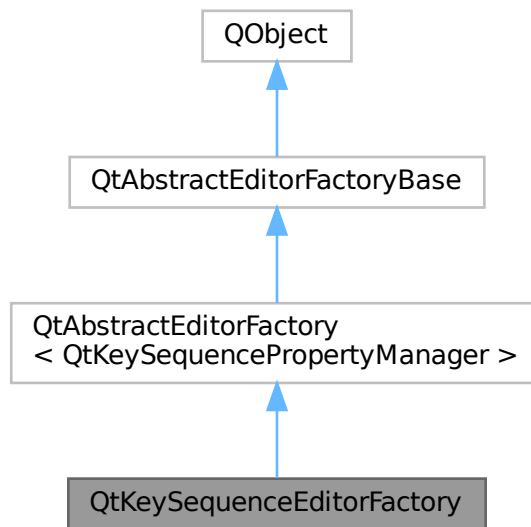
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.690 QtKeySequenceEditorFactory Class Reference

Inheritance diagram for QtKeySequenceEditorFactory:



Collaboration diagram for `QtKeySequenceEditorFactory`:



Public Member Functions

- [QtKeySequenceEditorFactory](#) (`QObject *parent=0`)
- [~QtKeySequenceEditorFactory](#) ()

Public Member Functions inherited from [QtAbstractEditorFactory < QtKeySequencePropertyManager >](#)

- [QtAbstractEditorFactory](#) (`QObject *parent`)
- `QWidget *` [createEditor](#) (`QtProperty *property`, `QWidget *parent`) override
- `void` [addPropertyManager](#) (`QtKeySequencePropertyManager *manager`)
- `void` [removePropertyManager](#) (`QtKeySequencePropertyManager *manager`)
- `QSet< QtKeySequencePropertyManager * >` [propertyManagers](#) () const
- `QtKeySequencePropertyManager *` [propertyManager](#) (`QtProperty *property`) const

Protected Member Functions

- `void` [connectPropertyManager](#) (`QtKeySequencePropertyManager *manager`) override
- `QWidget *` [createEditor](#) (`QtKeySequencePropertyManager *manager`, `QtProperty *property`, `QWidget *parent`) override
- `void` [disconnectPropertyManager](#) (`QtKeySequencePropertyManager *manager`) override

Protected Member Functions inherited from [QtAbstractEditorFactory < QtKeySequencePropertyManager >](#)

- `virtual QWidget *` [createEditor](#) (`QtKeySequencePropertyManager *manager`, `QtProperty *property`, `QWidget *parent`)=0
- `virtual void` [connectPropertyManager](#) (`QtKeySequencePropertyManager *manager`)=0
- `virtual void` [disconnectPropertyManager](#) (`QtKeySequencePropertyManager *manager`)=0
- `void` [managerDestroyed](#) (`QObject *manager`) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

15.690.1 Constructor & Destructor Documentation

15.690.1.1 QtKeySequenceEditorFactory()

```
QtKeySequenceEditorFactory::QtKeySequenceEditorFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

15.690.1.2 ~QtKeySequenceEditorFactory()

```
QtKeySequenceEditorFactory::~~QtKeySequenceEditorFactory ( )
```

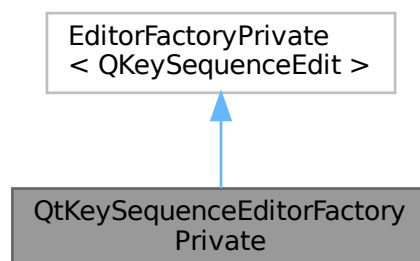
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

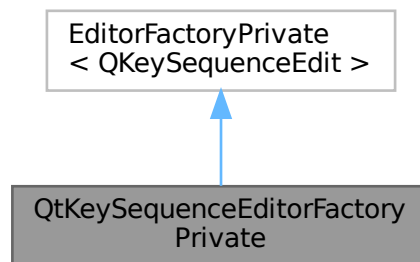
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.691 QtKeySequenceEditorFactoryPrivate Class Reference

Inheritance diagram for QtKeySequenceEditorFactoryPrivate:



Collaboration diagram for QtKeySequenceEditorFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, const QKeySequence &value)
- void **slotSetValue** (const QKeySequence &value)

Public Member Functions inherited from [EditorFactoryPrivate< QKeySequenceEdit >](#)

- QKeySequenceEdit * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, QKeySequenceEdit *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QKeySequenceEdit >](#)

- typedef QList< QKeySequenceEdit * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< QKeySequenceEdit *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate< QKeySequenceEdit >](#)

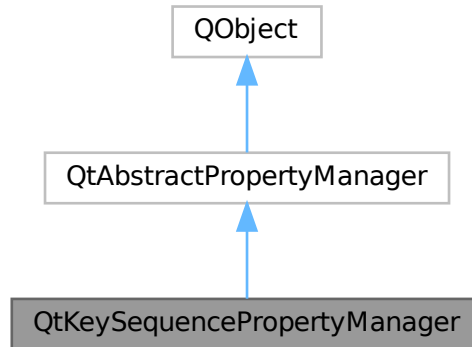
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

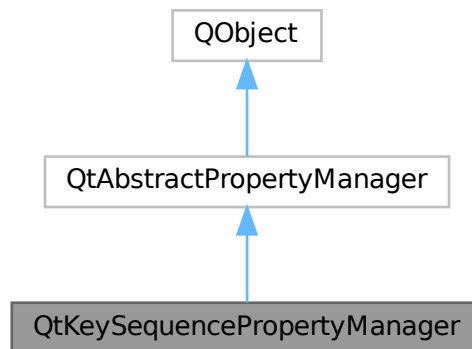
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.692 QtKeySequencePropertyManager Class Reference

Inheritance diagram for QtKeySequencePropertyManager:



Collaboration diagram for QtKeySequencePropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, const QKeySequence &val)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QKeySequence &val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtKeySequencePropertyManager](#) (QObject *parent=0)
- [~QtKeySequencePropertyManager](#) ()
- QKeySequence [value](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.692.1 Constructor & Destructor Documentation

15.692.1.1 [QtKeySequencePropertyManager](#)()

```
QtKeySequencePropertyManager::QtKeySequencePropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.692.1.2 [~QtKeySequencePropertyManager](#)()

```
QtKeySequencePropertyManager::~QtKeySequencePropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.692.2 Member Function Documentation

15.692.2.1 [initializeProperty](#)()

```
void QtKeySequencePropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.692.2.2 [setValue](#)

```
void QtKeySequencePropertyManager::setValue (
    QtProperty * property,
    const QKeySequence & value ) [slot]
```

Sets the value of the given *property* to *value*.

See also

[value\(\)](#), [valueChanged\(\)](#)

15.692.2.3 uninitializedProperty()

```
void QtKeySequencePropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.692.2.4 value()

```
QKeySequence QtKeySequencePropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns an empty QKeySequence object.

See also

[setValue\(\)](#)

15.692.2.5 valueChanged

```
void QtKeySequencePropertyManager::valueChanged (
    QtProperty * property,
    const QKeySequence & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

15.692.2.6 valueText()

```
QString QtKeySequencePropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.693 QtKeySequencePropertyManagerPrivate Class Reference

Public Types

- typedef QMap< const [QtProperty](#) *, QKeySequence > **PropertyValueMap**

Public Attributes

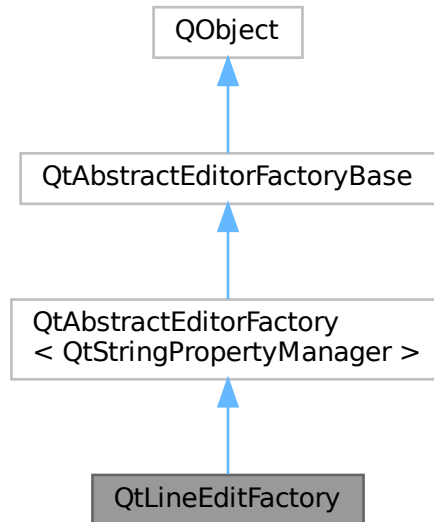
- QString **m_format**
- PropertyValueMap **m_values**

The documentation for this class was generated from the following file:

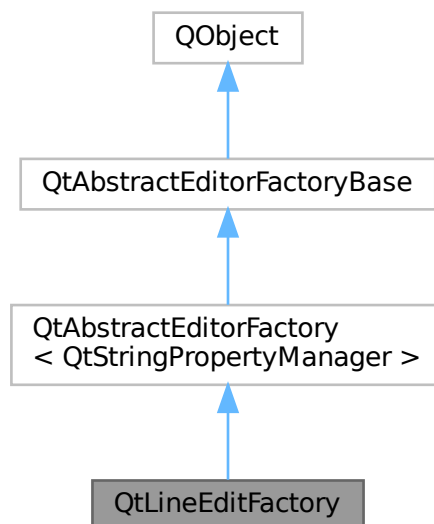
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.694 QtLineEditFactory Class Reference

Inheritance diagram for QtLineEditFactory:



Collaboration diagram for QtLineEditFactory:



Public Member Functions

- [QtLineEditFactory](#) (QObject *parent=0)

- [~QtLineEditFactory](#) ()

Public Member Functions inherited from

[QtAbstractEditorFactory](#) < [QtStringPropertyManager](#) >

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtStringPropertyManager *manager)
- void [removePropertyManager](#) (QtStringPropertyManager *manager)
- QSet< [QtStringPropertyManager](#) * > [propertyManagers](#) () const
- [QtStringPropertyManager](#) * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtStringPropertyManager *manager) override
- QWidget * [createEditor](#) (QtStringPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtStringPropertyManager *manager) override

Protected Member Functions inherited from

[QtAbstractEditorFactory](#) < [QtStringPropertyManager](#) >

- virtual QWidget * [createEditor](#) (QtStringPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtStringPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtStringPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

15.694.1 Constructor & Destructor Documentation

15.694.1.1 QtLineEditFactory()

```
QtLineEditFactory::QtLineEditFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

15.694.1.2 ~QtLineEditFactory()

```
QtLineEditFactory::~QtLineEditFactory ( )
```

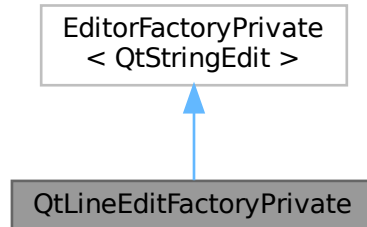
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

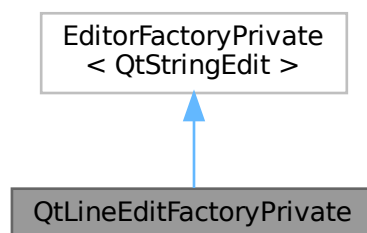
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.695 QtLineEditFactoryPrivate Class Reference

Inheritance diagram for QtLineEditFactoryPrivate:



Collaboration diagram for QtLineEditFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, const QString &value)
- void **slotRegExpChanged** ([QtProperty](#) *property, const QRegularExpression ®Exp)
- void **slotSetValue** (const QString &value)

Public Member Functions inherited from [EditorFactoryPrivate< QStringEdit >](#)

- [QStringEdit](#) * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, [QStringEdit](#) *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QStringEdit >](#)

- typedef QList< [QStringEdit](#) * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< [QStringEdit](#) *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate< QStringEdit >](#)

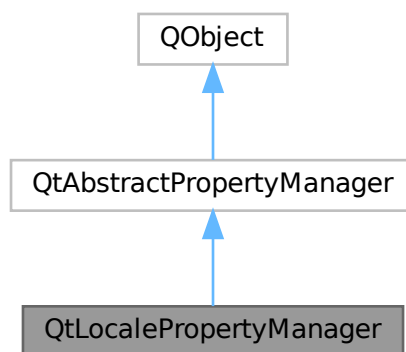
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

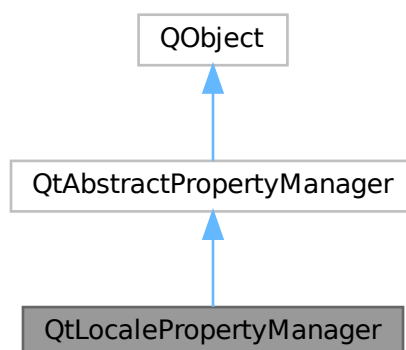
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.696 QtLocalePropertyManager Class Reference

Inheritance diagram for QtLocalePropertyManager:



Collaboration diagram for QtLocalePropertyManager:

**Public Slots**

- void [setValue](#) ([QtProperty](#) *property, const QLocale &val)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const [QLocale](#) &val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtLocalePropertyManager](#) ([QObject](#) *parent=0)
- [~QtLocalePropertyManager](#) ()
- [QtEnumPropertyManager](#) * [subEnumPropertyManager](#) () const
- [QLocale](#) [value](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) ([QObject](#) *parent=0)
- [~QtAbstractPropertyManager](#) ()
- [QSet](#)< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const [QString](#) &name=[QString](#)())

Protected Member Functions

- [QString](#) [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual [QIcon](#) [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.696.1 Constructor & Destructor Documentation

15.696.1.1 [QtLocalePropertyManager](#)()

```
QtLocalePropertyManager::QtLocalePropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.696.1.2 [~QtLocalePropertyManager](#)()

```
QtLocalePropertyManager::~~QtLocalePropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.696.2 Member Function Documentation

15.696.2.1 [initializeProperty](#)()

```
void QtLocalePropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.696.2.2 setValue

```
void QtLocalePropertyManager::setValue (
    QtProperty * property,
    const QLocale & value ) [slot]
```

Sets the value of the given *property* to *value*. Nested properties are updated automatically.

See also

[value\(\)](#), [valueChanged\(\)](#)

15.696.2.3 subEnumPropertyManager()

```
QtEnumPropertyManager * QtLocalePropertyManager::subEnumPropertyManager ( ) const
```

Returns the manager that creates the nested *language* and *country* subproperties.

In order to provide editing widgets for the mentioned subproperties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

15.696.2.4 uninitializedProperty()

```
void QtLocalePropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.696.2.5 value()

```
QLocale QtLocalePropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given property is not managed by this manager, this function returns the default locale.

See also

[setValue\(\)](#)

15.696.2.6 valueChanged

```
void QtLocalePropertyManager::valueChanged (
    QtProperty * property,
    const QLocale & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

15.696.2.7 valueText()

```
QString QtLocalePropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

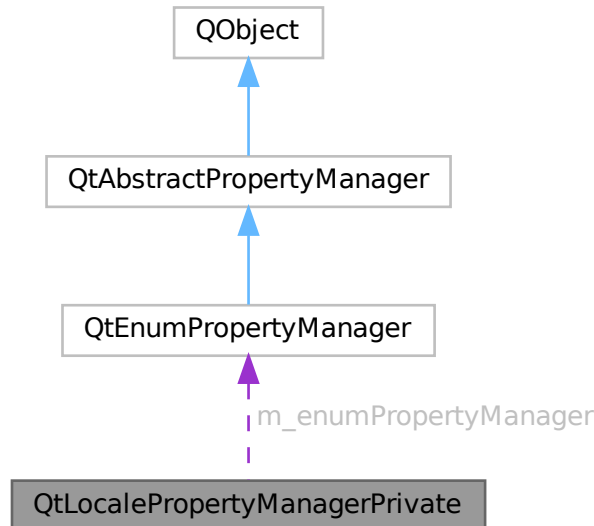
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.697 QtLocalePropertyManagerPrivate Class Reference

Collaboration diagram for QtLocalePropertyManagerPrivate:



Public Types

- typedef QMap< const [QtProperty](#) *, QLocale > **PropertyValueMap**

Public Member Functions

- void **slotEnumChanged** ([QtProperty](#) *property, int value)
- void **slotPropertyDestroyed** ([QtProperty](#) *property)

Public Attributes

- PropertyValueMap **m_values**
- [QtEnumPropertyManager](#) * **m_enumPropertyManager**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToLanguage**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToCountry**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_languageToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_countryToProperty**

The documentation for this class was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.698 QtMetaEnumProvider Class Reference

Public Member Functions

- QStringList **policyEnumNames** () const
- QStringList **languageEnumNames** () const
- QStringList **countryEnumNames** (QLocale::Language language) const

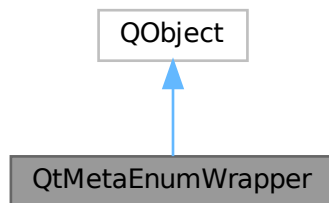
- QSizePolicy::Policy **indexToSizePolicy** (int index) const
- int **sizePolicyToIndex** (QSizePolicy::Policy policy) const
- void **indexToLocale** (int languageIndex, int countryIndex, QLocale::Language *language, QLocale::Country *country) const
- void **localeToIndex** (QLocale::Language language, QLocale::Country country, int *languageIndex, int *countryIndex) const

The documentation for this class was generated from the following file:

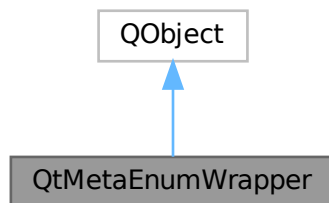
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.699 QtMetaEnumWrapper Class Reference

Inheritance diagram for QtMetaEnumWrapper:



Collaboration diagram for QtMetaEnumWrapper:



Public Member Functions

- QSizePolicy::Policy **policy** () const

Properties

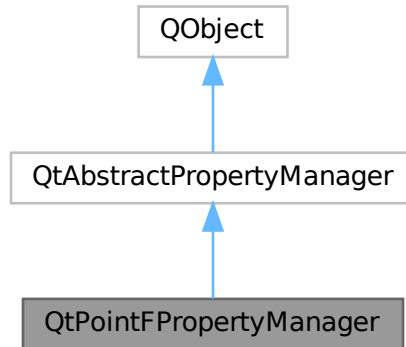
- QSizePolicy::Policy **policy**

The documentation for this class was generated from the following file:

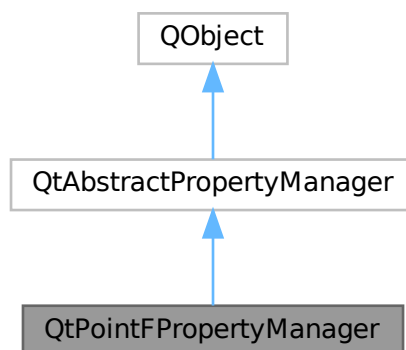
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.700 QtPointFPropertyManager Class Reference

Inheritance diagram for QtPointFPropertyManager:



Collaboration diagram for QtPointFPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, const QPointF &val)
- void [setDecimals](#) ([QtProperty](#) *property, int prec)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QPointF &val)
- void [decimalsChanged](#) ([QtProperty](#) *property, int prec)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)

- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtPointFPropertyManager](#) (QObject *parent=0)
- [~QtPointFPropertyManager](#) ()
- [QtDoublePropertyManager](#) * [subDoublePropertyManager](#) () const
- QPointF [value](#) (const [QtProperty](#) *property) const
- int [decimals](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.700.1 Constructor & Destructor Documentation

15.700.1.1 [QtPointFPropertyManager](#)()

```
QtPointFPropertyManager::QtPointFPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.700.1.2 [~QtPointFPropertyManager](#)()

```
QtPointFPropertyManager::~~QtPointFPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.700.2 Member Function Documentation

15.700.2.1 [decimals](#)()

```
int QtPointFPropertyManager::decimals (
    const QtProperty * property ) const
```

Returns the given *property*'s precision, in decimals.

See also

[setDecimals\(\)](#)

15.700.2.2 decimalsChanged

```
void QtPointFPropertyManager::decimalsChanged (
    QtProperty * property,
    int prec ) [signal]
```

This signal is emitted whenever a property created by this manager changes its precision of value, passing a pointer to the *property* and the new *prec* value

See also

[setDecimals\(\)](#)

15.700.2.3 initializeProperty()

```
void QtPointFPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.700.2.4 setDecimals

```
void QtPointFPropertyManager::setDecimals (
    QtProperty * property,
    int prec ) [slot]
```

Sets the precision of the given *property* to *prec*.

The valid decimal range is 0-13. The default is 2.

See also

[decimals\(\)](#)

15.700.2.5 setValue

```
void QtPointFPropertyManager::setValue (
    QtProperty * property,
    const QPointF & value ) [slot]
```

Sets the value of the given *property* to *value*. Nested properties are updated automatically.

See also

[value\(\)](#), [valueChanged\(\)](#)

15.700.2.6 subDoublePropertyManager()

```
QtDoublePropertyManager * QtPointFPropertyManager::subDoublePropertyManager ( ) const
```

Returns the manager that creates the nested *x* and *y* subproperties.

In order to provide editing widgets for the subproperties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

15.700.2.7 uninitializedProperty()

```
void QtPointFPropertyManager::uninitializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.700.2.8 value()

```
QPointF QtPointFPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns a point with coordinates (0, 0).

See also

[setValue\(\)](#)

15.700.2.9 valueChanged

```
void QtPointFPropertyManager::valueChanged (
    QtProperty * property,
    const QPointF & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

15.700.2.10 valueText()

```
QString QtPointFPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

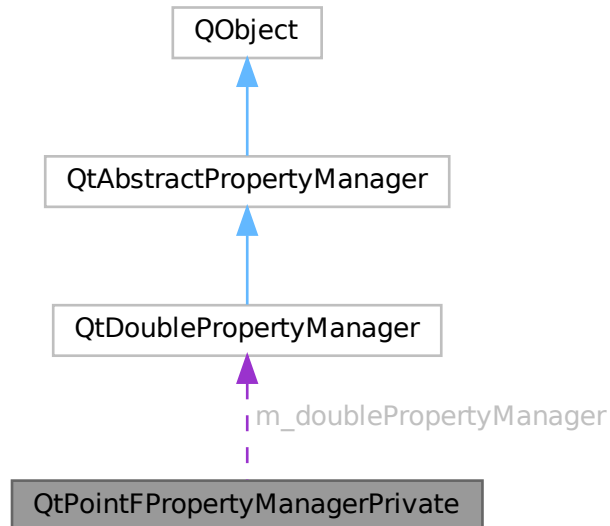
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.701 QtPointFPropertyManagerPrivate Class Reference

Collaboration diagram for QtPointFPropertyManagerPrivate:



Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Member Functions

- void **slotDoubleChanged** ([QtProperty](#) *property, double value)
- void **slotPropertyDestroyed** ([QtProperty](#) *property)

Public Attributes

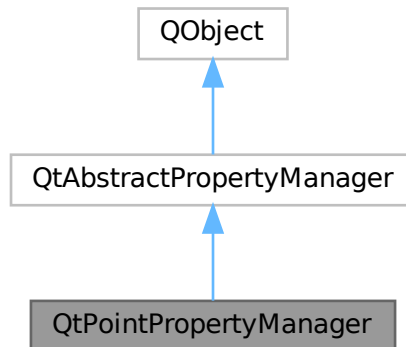
- PropertyValueMap **m_values**
- [QtDoublePropertyManager](#) * **m_doublePropertyManager**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToX**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToY**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_xToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_yToProperty**

The documentation for this class was generated from the following file:

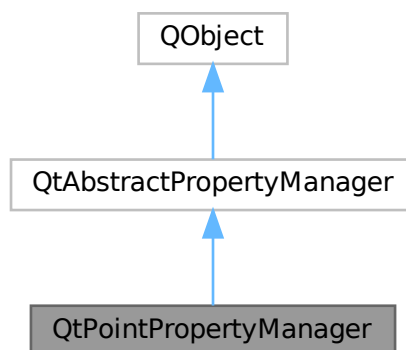
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.702 QtPointPropertyManager Class Reference

Inheritance diagram for QtPointPropertyManager:



Collaboration diagram for QtPointPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, const QPoint &val)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QPoint &val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtPointPropertyManager](#) (QObject *parent=0)
- [~QtPointPropertyManager](#) ()
- [QtIntPropertyManager](#) * [subIntPropertyManager](#) () const
- QPoint [value](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.702.1 Constructor & Destructor Documentation

15.702.1.1 QtPointPropertyManager()

```
QtPointPropertyManager::QtPointPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.702.1.2 ~QtPointPropertyManager()

```
QtPointPropertyManager::~~QtPointPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.702.2 Member Function Documentation

15.702.2.1 initializeProperty()

```
void QtPointPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.702.2.2 setValue

```
void QtPointPropertyManager::setValue (
    QtProperty * property,
    const QPoint & value ) [slot]
```

Sets the value of the given *property* to *value*. Nested properties are updated automatically.

See also

[value\(\)](#), [valueChanged\(\)](#)

15.702.2.3 subIntPropertyManager()

```
QtIntPropertyManager * QtPointPropertyManager::subIntPropertyManager ( ) const
```

Returns the manager that creates the nested *x* and *y* subproperties.

In order to provide editing widgets for the subproperties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

15.702.2.4 uninitializedProperty()

```
void QtPointPropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.702.2.5 value()

```
QPoint QtPointPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns a point with coordinates (0, 0).

See also

[setValue\(\)](#)

15.702.2.6 valueChanged

```
void QtPointPropertyManager::valueChanged (
    QtProperty * property,
    const QPoint & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

15.702.2.7 valueText()

```
QString QtPointPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

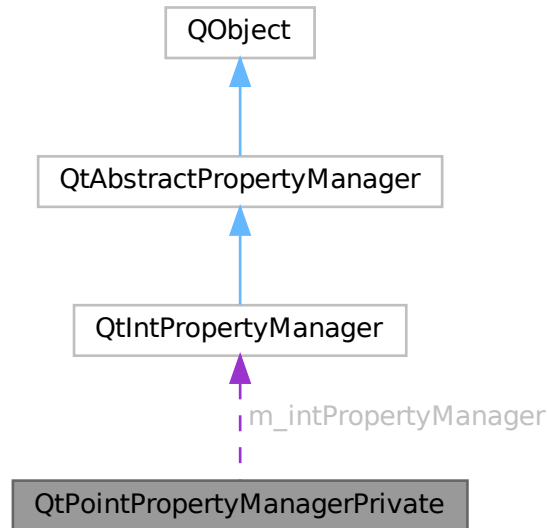
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.703 QPointPropertyManagerPrivate Class Reference

Collaboration diagram for QPointPropertyManagerPrivate:



Public Types

- typedef QMap< const [QtProperty](#) *, QPoint > **PropertyValueMap**

Public Member Functions

- void **slotIntChanged** ([QtProperty](#) *property, int value)
- void **slotPropertyDestroyed** ([QtProperty](#) *property)

Public Attributes

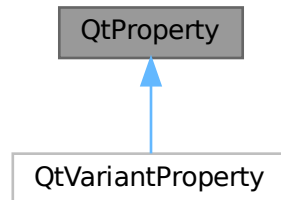
- PropertyValueMap **m_values**
- [QtIntPropertyManager](#) * **m_intPropertyManager**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToX**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToY**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_xToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_yToProperty**

The documentation for this class was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.704 QtProperty Class Reference

Inheritance diagram for QtProperty:



Public Member Functions

- virtual `~QtProperty` ()
- `QList< QtProperty * > subProperties` () const
- `QtAbstractPropertyManager * propertyManager` () const
- `QString toolTip` () const
- `QString valueToolTip` () const
- `QString descriptionToolTip` () const
- `QString statusTip` () const
- `QString whatsThis` () const
- `QString propertyName` () const
- `bool isEnabled` () const
- `bool isModified` () const
- `bool hasValue` () const
- `QIcon valueIcon` () const
- `QString valueText` () const
- void `setToolTip` (const `QString &text`)
- void `setValueToolTip` (const `QString &text`)
- void `setDescriptionToolTip` (const `QString &text`)
- void `setStatusTip` (const `QString &text`)
- void `setWhatsThis` (const `QString &text`)
- void `setPropertyName` (const `QString &text`)
- void `setEnabled` (bool enable)
- void `setModified` (bool modified)
- void `addSubProperty` (`QtProperty *property`)
- void `insertSubProperty` (`QtProperty *property`, `QtProperty *afterProperty`)
- void `removeSubProperty` (`QtProperty *property`)

Protected Member Functions

- `QtProperty` (`QtAbstractPropertyManager *manager`)
- void `propertyChanged` ()

Friends

- class `QtAbstractPropertyManager`

15.704.1 Constructor & Destructor Documentation

15.704.1.1 `~QtProperty()`

```
QtProperty::~~QtProperty ( ) [virtual]
```

Destroys this property.

Note that subproperties are detached but not destroyed, i.e. they can still be used in another context.

See also

[QtAbstractPropertyManager::clear\(\)](#)

15.704.1.2 `QtProperty()`

```
QtProperty::QtProperty (
    QtAbstractPropertyManager * manager ) [explicit], [protected]
```

Creates a property with the given *manager*.

This constructor is only useful when creating a custom [QtProperty](#) subclass (e.g. [QtVariantProperty](#)). To create a regular [QtProperty](#) object, use the [QtAbstractPropertyManager::addProperty\(\)](#) function instead.

See also

[QtAbstractPropertyManager::addProperty\(\)](#)

15.704.2 Member Function Documentation

15.704.2.1 `addSubProperty()`

```
void QtProperty::addSubProperty (
    QtProperty * property )
```

Appends the given *property* to this property's subproperties.

If the given *property* already is added, this function does nothing.

See also

[insertSubProperty\(\)](#), [removeSubProperty\(\)](#)

15.704.2.2 `descriptionToolTip()`

```
QString QtProperty::descriptionToolTip ( ) const
```

Returns the property description's tool tip.

This is suitable for tool tips over the description (label).

Since

5.6

See also

[setDescriptionToolTip\(\)](#)

15.704.2.3 `hasValue()`

```
bool QtProperty::hasValue ( ) const
```

Returns whether the property has a value.

See also

[QtAbstractPropertyManager::hasValue\(\)](#)

15.704.2.4 insertSubProperty()

```
void QtProperty::insertSubProperty (
    QtProperty * property,
    QtProperty * precedingProperty )
```

Inserts the given *property* after the specified *precedingProperty* into this property's list of subproperties. If *precedingProperty* is 0, the specified *property* is inserted at the beginning of the list. If the given *property* already is inserted, this function does nothing.

See also

[addSubProperty\(\)](#), [removeSubProperty\(\)](#)

15.704.2.5 isEnabled()

```
bool QtProperty::isEnabled ( ) const
```

Returns whether the property is enabled.

See also

[setEnabled\(\)](#)

15.704.2.6 isModified()

```
bool QtProperty::isModified ( ) const
```

Returns whether the property is modified.

See also

[setModified\(\)](#)

15.704.2.7 propertyManager()

```
QtAbstractPropertyManager * QtProperty::propertyManager ( ) const
```

Returns a pointer to the manager that owns this property.

15.704.2.8 propertyName()

```
QString QtProperty::propertyName ( ) const
```

Returns the property's name.

See also

[setPropertyName\(\)](#)

15.704.2.9 removeSubProperty()

```
void QtProperty::removeSubProperty (
    QtProperty * property )
```

Removes the given *property* from the list of subproperties without deleting it.

See also

[addSubProperty\(\)](#), [insertSubProperty\(\)](#)

15.704.2.10 setDescriptionToolTip()

```
void QtProperty::setDescriptionToolTip (
    const QString & text )
```

Sets the property description's tool tip to the given *text*.

Since

5.6

See also

[descriptionToolTip\(\)](#)

15.704.2.11 setEnabled()

```
void QtProperty::setEnabled (
    bool enable )
```

Enables or disables the property according to the passed *enable* value.

See also

[isEnabled\(\)](#)

15.704.2.12 setModified()

```
void QtProperty::setModified (
    bool modified )
```

Sets the property's modified state according to the passed *modified* value.

See also

[isModified\(\)](#)

15.704.2.13 setPropertyName()

```
void QtProperty::setPropertyName (
    const QString & name )
```

Sets the property's name to the given *name*.

See also

[propertyName\(\)](#)

15.704.2.14 setStatusTip()

```
void QtProperty::setStatusTip (
    const QString & text )
```

Sets the property's status tip to the given *text*.

See also

[statusTip\(\)](#)

15.704.2.15 setValueToolTip()

```
void QtProperty::setValueToolTip (
    const QString & text )
```

Sets the property value's tool tip to the given *text*.

Since

5.6

See also

[valueToolTip\(\)](#)

15.704.2.16 setWhatsThis()

```
void QtProperty::setWhatsThis (
    const QString & text )
```

Sets the property's "What's This" help text to the given *text*.

See also

[whatsThis\(\)](#)

15.704.2.17 statusTip()

```
QString QtProperty::statusTip ( ) const
```

Returns the property's status tip.

See also

[setStatusTip\(\)](#)

15.704.2.18 subProperties()

```
QList< QtProperty * > QtProperty::subProperties ( ) const
```

Returns the set of subproperties.

Note that subproperties are not owned by *this* property, but by the manager that created them.

See also

[insertSubProperty\(\)](#), [removeSubProperty\(\)](#)

15.704.2.19 valueIcon()

```
QIcon QtProperty::valueIcon ( ) const
```

Returns an icon representing the current state of this property.

If the given property type can not generate such an icon, this function returns an invalid icon.

See also

[QtAbstractPropertyManager::valueIcon\(\)](#)

15.704.2.20 valueText()

```
QString QtProperty::valueText ( ) const
```

Returns a string representing the current state of this property.

If the given property type can not generate such a string, this function returns an empty string.

See also

[QtAbstractPropertyManager::valueText\(\)](#)

15.704.2.21 valueToolTip()

```
QString QtProperty::valueToolTip ( ) const
```

Returns the property value's tool tip.

This is suitable for tool tips over the value (item delegate).

Since

5.6

See also

[setValueToolTip\(\)](#)

15.704.2.22 whatsThis()

```
QString QtProperty::whatsThis ( ) const
```

Returns the property's "What's This" help text.

See also

[setWhatsThis\(\)](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.cpp

15.705 QtPropertyBrowserUtils Class Reference

Static Public Member Functions

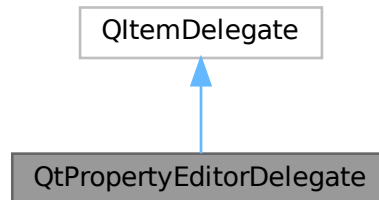
- static QPixmap **brushValuePixmap** (const QBrush &b)
- static QIcon **brushValueIcon** (const QBrush &b)
- static QString **colorValueText** (const QColor &c)
- static QPixmap **fontValuePixmap** (const QFont &f)
- static QIcon **fontValueIcon** (const QFont &f)
- static QString **fontValueText** (const QFont &f)
- static QString **dateFormat** ()
- static QString **timeFormat** ()
- static QString **dateTimeFormat** ()

The documentation for this class was generated from the following files:

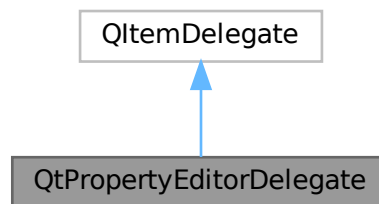
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils_p.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils.cpp

15.706 QtPropertyEditorDelegate Class Reference

Inheritance diagram for QtPropertyEditorDelegate:



Collaboration diagram for QtPropertyEditorDelegate:



Public Member Functions

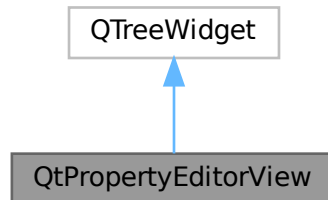
- **QtPropertyEditorDelegate** (QObject *parent=0)
- void **setEditorPrivate** ([QtTreePropertyBrowserPrivate](#) *editorPrivate)
- QWidget * **createEditor** (QWidget *parent, const QStyleOptionViewItem &option, const QModelIndex &index) const override
- void **updateEditorGeometry** (QWidget *editor, const QStyleOptionViewItem &option, const QModelIndex &index) const override
- void **paint** (QPainter *painter, const QStyleOptionViewItem &option, const QModelIndex &index) const override
- QSize **sizeHint** (const QStyleOptionViewItem &option, const QModelIndex &index) const override
- void **setModelData** (QWidget *, QAbstractItemModel *, const QModelIndex &) const override
- void **setEditorData** (QWidget *, const QModelIndex &) const override
- bool **eventFilter** (QObject *object, QEvent *event) override
- void **closeEditor** ([QtProperty](#) *property)
- QTreeWidgetItem * **editedItem** () const

The documentation for this class was generated from the following file:

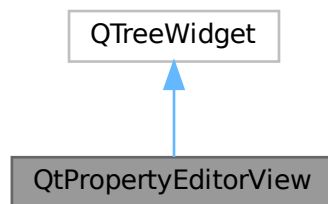
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qttreepropertybrowser.cpp

15.707 QtPropertyEditorView Class Reference

Inheritance diagram for QtPropertyEditorView:



Collaboration diagram for QtPropertyEditorView:



Public Member Functions

- **QtPropertyEditorView** (QWidget *parent=0)
- void **setEditorPrivate** ([QtTreePropertyBrowserPrivate](#) *editorPrivate)
- QTreeWidgetItem * **indexToItem** (const QModelIndex &index) const

Protected Member Functions

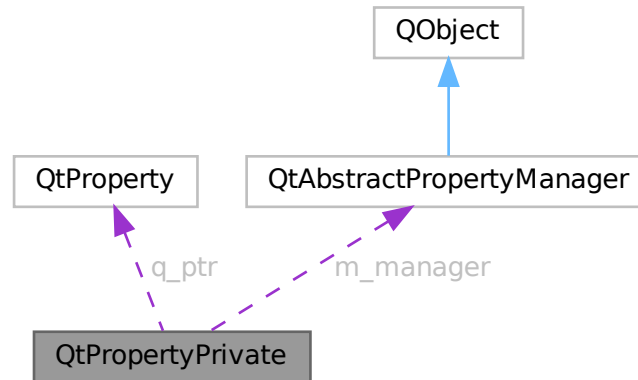
- void **keyPressEvent** (QKeyEvent *event) override
- void **mousePressEvent** (QMouseEvent *event) override
- void **drawRow** (QPainter *painter, const QStyleOptionViewItem &option, const QModelIndex &index) const override

The documentation for this class was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qttreepropertybrowser.cpp

15.708 QtPropertyPrivate Class Reference

Collaboration diagram for QtPropertyPrivate:



Public Member Functions

- **QtPropertyPrivate** ([QtAbstractPropertyManager](#) *manager)

Public Attributes

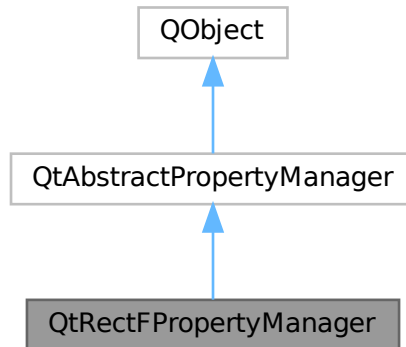
- [QtProperty](#) * **q_ptr**
- `QSet< QtProperty * >` **m_parentItems**
- `QList< QtProperty * >` **m_subItems**
- `QString` **m_valueToolTip**
- `QString` **m_descriptionToolTip**
- `QString` **m_statusTip**
- `QString` **m_whatsThis**
- `QString` **m_name**
- `bool` **m_enabled**
- `bool` **m_modified**
- [QtAbstractPropertyManager](#) *const **m_manager**

The documentation for this class was generated from the following file:

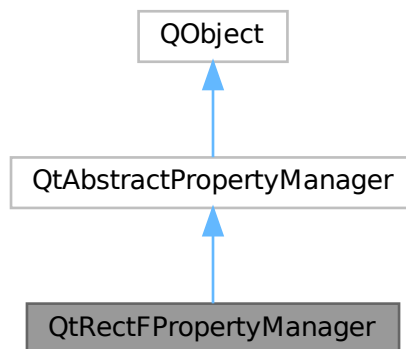
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.cpp

15.709 QtRectFPropertyManager Class Reference

Inheritance diagram for QtRectFPropertyManager:



Collaboration diagram for QtRectFPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, const QRectF &val)
- void [setConstraint](#) ([QtProperty](#) *property, const QRectF &[constraint](#))
- void [setDecimals](#) ([QtProperty](#) *property, int prec)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QRectF &val)
- void [constraintChanged](#) ([QtProperty](#) *property, const QRectF &[constraint](#))
- void [decimalsChanged](#) ([QtProperty](#) *property, int prec)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtRectFPropertyManager](#) (QObject *parent=0)
- [~QtRectFPropertyManager](#) ()
- [QtDoublePropertyManager](#) * [subDoublePropertyManager](#) () const
- QRectF [value](#) (const [QtProperty](#) *property) const
- QRectF [constraint](#) (const [QtProperty](#) *property) const
- int [decimals](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.709.1 Constructor & Destructor Documentation

15.709.1.1 [QtRectFPropertyManager](#)()

```
QtRectFPropertyManager::QtRectFPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.709.1.2 [~QtRectFPropertyManager](#)()

```
QtRectFPropertyManager::~~QtRectFPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.709.2 Member Function Documentation

15.709.2.1 [constraint](#)()

```
QRectF QtRectFPropertyManager::constraint (
    const QtProperty * property ) const
```

Returns the given *property*'s constraining rectangle. If returned value is null QRectF it means there is no constraint applied.

See also

[value\(\)](#), [setConstraint\(\)](#)

15.709.2.2 constraintChanged

```
void QtRectFPropertyManager::constraintChanged (
    QtProperty * property,
    const QRectF & constraint ) [signal]
```

This signal is emitted whenever property changes its constraint rectangle, passing a pointer to the *property* and the new *constraint* rectangle as parameters.

See also

[setConstraint\(\)](#)

15.709.2.3 decimals()

```
int QtRectFPropertyManager::decimals (
    const QtProperty * property ) const
```

Returns the given *property*'s precision, in decimals.

See also

[setDecimals\(\)](#)

15.709.2.4 decimalsChanged

```
void QtRectFPropertyManager::decimalsChanged (
    QtProperty * property,
    int prec ) [signal]
```

This signal is emitted whenever a property created by this manager changes its precision of value, passing a pointer to the *property* and the new *prec* value

See also

[setDecimals\(\)](#)

15.709.2.5 initializeProperty()

```
void QtRectFPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.709.2.6 setConstraint

```
void QtRectFPropertyManager::setConstraint (
    QtProperty * property,
    const QRectF & constraint ) [slot]
```

Sets the given *property*'s constraining rectangle to *constraint*.

When setting the constraint, the current value is adjusted if necessary (ensuring that the current rectangle value is inside the constraint). In order to reset the constraint pass a null QRectF value.

See also

[setValue\(\)](#), [constraint\(\)](#), [constraintChanged\(\)](#)

15.709.2.7 setDecimals

```
void QtRectFPropertyManager::setDecimals (
    QtProperty * property,
    int prec ) [slot]
```

Sets the precision of the given *property* to *prec*.
The valid decimal range is 0-13. The default is 2.

See also

[decimals\(\)](#)

15.709.2.8 setValue

```
void QtRectFPropertyManager::setValue (
    QtProperty * property,
    const QRectF & value ) [slot]
```

Sets the value of the given *property* to *value*. Nested properties are updated automatically.
If the specified *value* is not inside the given *property*'s constraining rectangle, the value is adjusted accordingly to fit within the constraint.

See also

[value\(\)](#), [setConstraint\(\)](#), [valueChanged\(\)](#)

15.709.2.9 subDoublePropertyManager()

```
QtDoublePropertyManager * QtRectFPropertyManager::subDoublePropertyManager ( ) const
```

Returns the manager that creates the nested *x*, *y*, *width* and *height* subproperties.

In order to provide editing widgets for the mentioned subproperties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

15.709.2.10 uninitializedProperty()

```
void QtRectFPropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.709.2.11 value()

```
QRectF QtRectFPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns an invalid rectangle.

See also

[setValue\(\)](#), [constraint\(\)](#)

15.709.2.12 valueChanged

```
void QtRectFPropertyManager::valueChanged (
    QtProperty * property,
    const QRectF & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

15.709.2.13 valueText()

```
QString QtRectFPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

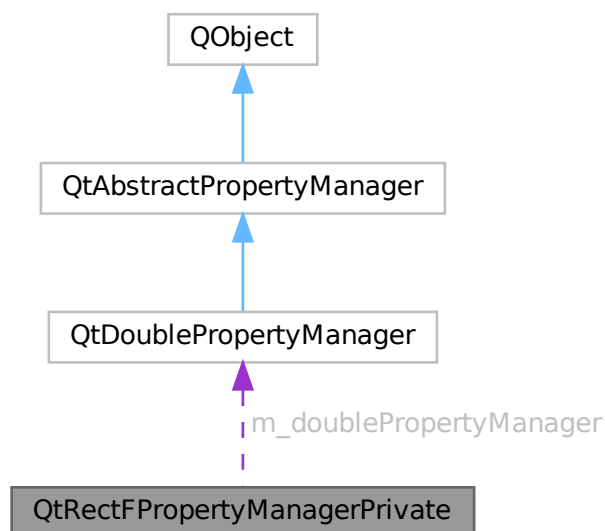
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.710 QtRectFPropertyManagerPrivate Class Reference

Collaboration diagram for QtRectFPropertyManagerPrivate:



Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Member Functions

- void **slotDoubleChanged** ([QtProperty](#) *property, double value)
- void **slotPropertyDestroyed** ([QtProperty](#) *property)
- void **setConstraint** ([QtProperty](#) *property, const QRectF &constraint, const QRectF &val)

Public Attributes

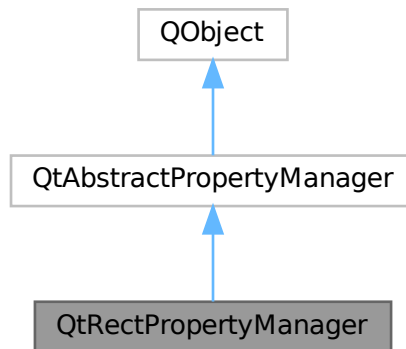
- PropertyValueMap **m_values**
- [QtDoublePropertyManager](#) * **m_doublePropertyManager**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToX**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToY**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToW**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToH**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_xToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_yToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_wToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_hToProperty**

The documentation for this class was generated from the following file:

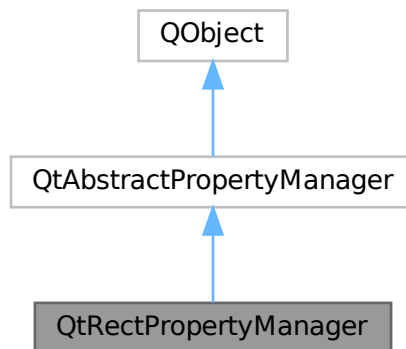
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.711 QtRectPropertyManager Class Reference

Inheritance diagram for QtRectPropertyManager:



Collaboration diagram for QtRectPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, const QRect &val)
- void [setConstraint](#) ([QtProperty](#) *property, const QRect &[constraint](#))

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QRect &val)
- void [constraintChanged](#) ([QtProperty](#) *property, const QRect &[constraint](#))

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtRectPropertyManager](#) (QObject *parent=0)
- [~QtRectPropertyManager](#) ()
- [QtIntPropertyManager](#) * [subIntPropertyManager](#) () const
- QRect [value](#) (const [QtProperty](#) *property) const
- QRect [constraint](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.711.1 Constructor & Destructor Documentation

15.711.1.1 QtRectPropertyManager()

```
QtRectPropertyManager::QtRectPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.711.1.2 ~QtRectPropertyManager()

```
QtRectPropertyManager::~~QtRectPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.711.2 Member Function Documentation

15.711.2.1 constraint()

```
QRect QtRectPropertyManager::constraint (
    const QtProperty * property ) const
```

Returns the given *property*'s constraining rectangle. If returned value is null QRect it means there is no constraint applied.

See also

[value\(\)](#), [setConstraint\(\)](#)

15.711.2.2 constraintChanged

```
void QtRectPropertyManager::constraintChanged (
    QtProperty * property,
    const QRect & constraint ) [signal]
```

This signal is emitted whenever property changes its constraint rectangle, passing a pointer to the *property* and the new *constraint* rectangle as parameters.

See also

[setConstraint\(\)](#)

15.711.2.3 initializeProperty()

```
void QtRectPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.711.2.4 setConstraint

```
void QtRectPropertyManager::setConstraint (
    QtProperty * property,
    const QRect & constraint ) [slot]
```

Sets the given *property*'s constraining rectangle to *constraint*.

When setting the constraint, the current value is adjusted if necessary (ensuring that the current rectangle value is inside the constraint). In order to reset the constraint pass a null QRect value.

See also

[setValue\(\)](#), [constraint\(\)](#), [constraintChanged\(\)](#)

15.711.2.5 setValue

```
void QtRectPropertyManager::setValue (
    QtProperty * property,
    const QRect & value ) [slot]
```

Sets the value of the given *property* to *value*. Nested properties are updated automatically.

If the specified *value* is not inside the given *property*'s constraining rectangle, the value is adjusted accordingly to fit within the constraint.

See also

[value\(\)](#), [setConstraint\(\)](#), [valueChanged\(\)](#)

15.711.2.6 subIntPropertyManager()

```
QtIntPropertyManager * QtRectPropertyManager::subIntPropertyManager ( ) const
```

Returns the manager that creates the nested *x*, *y*, *width* and *height* subproperties.

In order to provide editing widgets for the mentioned subproperties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

15.711.2.7 uninitializedProperty()

```
void QtRectPropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.711.2.8 value()

```
QRect QtRectPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns an invalid rectangle.

See also

[setValue\(\)](#), [constraint\(\)](#)

15.711.2.9 valueChanged

```
void QtRectPropertyManager::valueChanged (
    QtProperty * property,
    const QRect & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

15.711.2.10 valueText()

```
QString QtRectPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

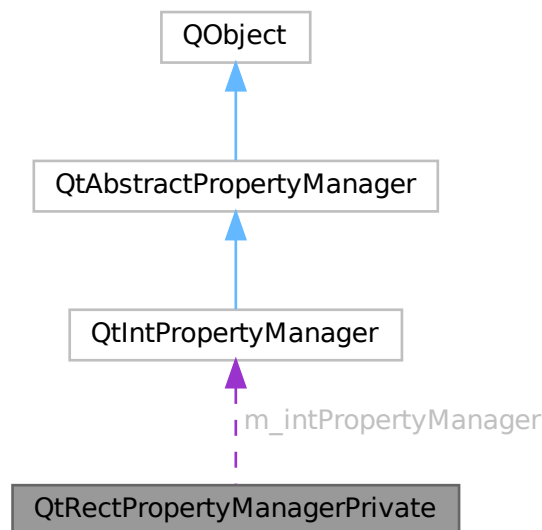
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.712 QtRectPropertyManagerPrivate Class Reference

Collaboration diagram for QtRectPropertyManagerPrivate:

**Classes**

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Member Functions

- void **slotIntChanged** ([QtProperty](#) *property, int value)
- void **slotPropertyDestroyed** ([QtProperty](#) *property)
- void **setConstraint** ([QtProperty](#) *property, const QRect &constraint, const QRect &val)

Public Attributes

- PropertyValueMap **m_values**
- [QtIntPropertyManager](#) * **m_intPropertyManager**

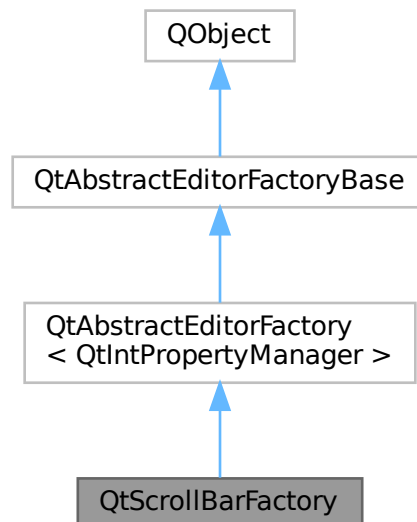
- QMap< const QtProperty *, QtProperty * > m_propertyToX
- QMap< const QtProperty *, QtProperty * > m_propertyToY
- QMap< const QtProperty *, QtProperty * > m_propertyToW
- QMap< const QtProperty *, QtProperty * > m_propertyToH
- QMap< const QtProperty *, QtProperty * > m_xToProperty
- QMap< const QtProperty *, QtProperty * > m_yToProperty
- QMap< const QtProperty *, QtProperty * > m_wToProperty
- QMap< const QtProperty *, QtProperty * > m_hToProperty

The documentation for this class was generated from the following file:

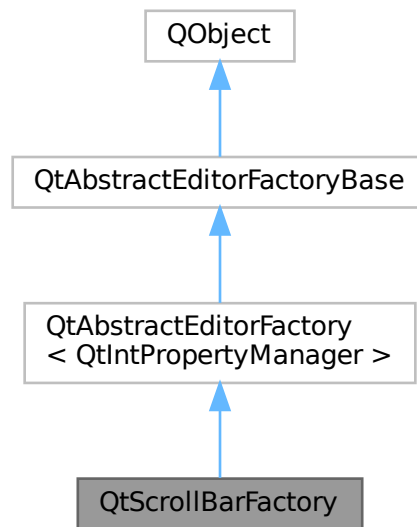
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.713 QtScrollBarFactory Class Reference

Inheritance diagram for QtScrollBarFactory:



Collaboration diagram for QtScrollBarFactory:



Public Member Functions

- [QtScrollBarFactory](#) (QObject *parent=0)
- [~QtScrollBarFactory](#) ()

Public Member Functions inherited from [QtAbstractEditorFactory < QtIntPropertyManager >](#)

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtIntPropertyManager *manager)
- void [removePropertyManager](#) (QtIntPropertyManager *manager)
- QSet< QtIntPropertyManager * > [propertyManagers](#) () const
- QtIntPropertyManager * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtIntPropertyManager *manager) override
- QWidget * [createEditor](#) (QtIntPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtIntPropertyManager *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactory < QtIntPropertyManager >](#)

- virtual QWidget * [createEditor](#) (QtIntPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtIntPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtIntPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

15.713.1 Constructor & Destructor Documentation

15.713.1.1 QtScrollBarFactory()

```
QtScrollBarFactory::QtScrollBarFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

15.713.1.2 ~QtScrollBarFactory()

```
QtScrollBarFactory::~QtScrollBarFactory ( )
```

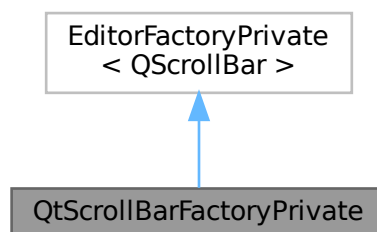
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

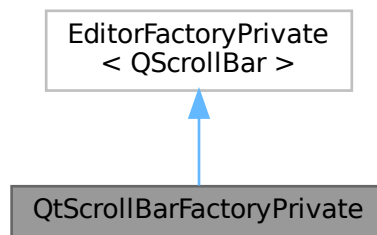
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.714 QtScrollBarFactoryPrivate Class Reference

Inheritance diagram for QtScrollBarFactoryPrivate:



Collaboration diagram for QtScrollBarFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, int value)
- void **slotRangeChanged** ([QtProperty](#) *property, int min, int max)
- void **slotSingleStepChanged** ([QtProperty](#) *property, int step)
- void **slotSetValue** (int value)

Public Member Functions inherited from [EditorFactoryPrivate< QScrollBar >](#)

- `QScrollBar *` **createEditor** ([QtProperty](#) *property, `QWidget *`parent)
- void **initializeEditor** ([QtProperty](#) *property, `QScrollBar *`e)
- void **slotEditorDestroyed** (`QObject *`object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QScrollBar >](#)

- typedef `QList< QScrollBar * >` **EditorList**
- typedef `QMap< QtProperty *, EditorList >` **PropertyToEditorListMap**
- typedef `QMap< QScrollBar *, QtProperty * >` **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate< QScrollBar >](#)

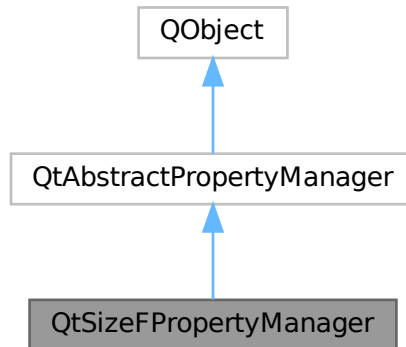
- `PropertyToEditorListMap` **m_createdEditors**
- `EditorToPropertyMap` **m_editorToProperty**

The documentation for this class was generated from the following file:

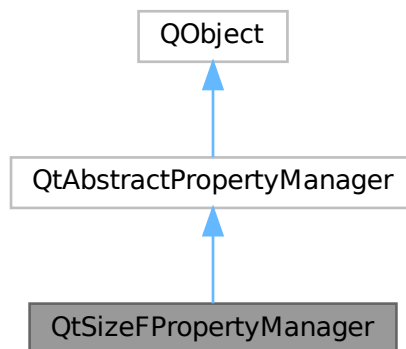
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.715 QSizeFPropertyManager Class Reference

Inheritance diagram for QSizeFPropertyManager:



Collaboration diagram for QSizeFPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, const QSizeF &val)
- void [setMinimum](#) ([QtProperty](#) *property, const QSizeF &minVal)
- void [setMaximum](#) ([QtProperty](#) *property, const QSizeF &maxVal)
- void [setRange](#) ([QtProperty](#) *property, const QSizeF &minVal, const QSizeF &maxVal)
- void [setDecimals](#) ([QtProperty](#) *property, int prec)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QSizeF &val)
- void [rangeChanged](#) ([QtProperty](#) *property, const QSizeF &minVal, const QSizeF &maxVal)
- void [decimalsChanged](#) ([QtProperty](#) *property, int prec)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtSizeFPropertyManager](#) (QObject *parent=0)
- [~QtSizeFPropertyManager](#) ()
- [QtDoublePropertyManager](#) * [subDoublePropertyManager](#) () const
- QSizeF [value](#) (const [QtProperty](#) *property) const
- QSizeF [minimum](#) (const [QtProperty](#) *property) const
- QSizeF [maximum](#) (const [QtProperty](#) *property) const
- int [decimals](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.715.1 Constructor & Destructor Documentation

15.715.1.1 [QtSizeFPropertyManager](#)()

```
QtSizeFPropertyManager::QtSizeFPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.715.1.2 [~QtSizeFPropertyManager](#)()

```
QtSizeFPropertyManager::~~QtSizeFPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.715.2 Member Function Documentation

15.715.2.1 [decimals](#)()

```
int QtSizeFPropertyManager::decimals (
    const QtProperty * property ) const
```

Returns the given *property*'s precision, in decimals.

See also

[setDecimals\(\)](#)

15.715.2.2 decimalsChanged

```
void QSizeFPropertyManager::decimalsChanged (
    QtProperty * property,
    int prec ) [signal]
```

This signal is emitted whenever a property created by this manager changes its precision of value, passing a pointer to the *property* and the new *prec* value

See also

[setDecimals\(\)](#)

15.715.2.3 initializeProperty()

```
void QSizeFPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.715.2.4 maximum()

```
QSizeF QSizeFPropertyManager::maximum (
    const QtProperty * property ) const
```

Returns the given *property*'s maximum size value.

See also

[setMaximum\(\)](#), [minimum\(\)](#), [setRange\(\)](#)

15.715.2.5 minimum()

```
QSizeF QSizeFPropertyManager::minimum (
    const QtProperty * property ) const
```

Returns the given *property*'s minimum size value.

See also

[setMinimum\(\)](#), [maximum\(\)](#), [setRange\(\)](#)

15.715.2.6 rangeChanged

```
void QSizeFPropertyManager::rangeChanged (
    QtProperty * property,
    const QSizeF & minimum,
    const QSizeF & maximum ) [signal]
```

This signal is emitted whenever a property created by this manager changes its range of valid sizes, passing a pointer to the *property* and the new *minimum* and *maximum* sizes.

See also

[setRange\(\)](#)

15.715.2.7 setDecimals

```
void QtSizeFPropertyManager::setDecimals (
    QtProperty * property,
    int prec ) [slot]
```

Sets the precision of the given *property* to *prec*.
The valid decimal range is 0-13. The default is 2.

See also

[decimals\(\)](#)

15.715.2.8 setMaximum

```
void QtSizeFPropertyManager::setMaximum (
    QtProperty * property,
    const QSizeF & maxVal ) [slot]
```

Sets the maximum size value for the given *property* to *maxVal*.

When setting the maximum size value, the minimum and current values are adjusted if necessary (ensuring that the size range remains valid and that the current value is within the range).

See also

[maximum\(\)](#), [setRange\(\)](#), [rangeChanged\(\)](#)

15.715.2.9 setMinimum

```
void QtSizeFPropertyManager::setMinimum (
    QtProperty * property,
    const QSizeF & minVal ) [slot]
```

Sets the minimum size value for the given *property* to *minVal*.

When setting the minimum size value, the maximum and current values are adjusted if necessary (ensuring that the size range remains valid and that the current value is within the range).

See also

[minimum\(\)](#), [setRange\(\)](#), [rangeChanged\(\)](#)

15.715.2.10 setRange

```
void QtSizeFPropertyManager::setRange (
    QtProperty * property,
    const QSizeF & minimum,
    const QSizeF & maximum ) [slot]
```

Sets the range of valid values.

This is a convenience function defining the range of valid values in one go; setting the *minimum* and *maximum* values for the given *property* with a single function call.

When setting a new range, the current value is adjusted if necessary (ensuring that the value remains within the range).

See also

[setMinimum\(\)](#), [setMaximum\(\)](#), [rangeChanged\(\)](#)

15.715.2.11 setValue

```
void QtSizeFPropertyManager::setValue (
    QtProperty * property,
    const QSizeF & value ) [slot]
```

Sets the value of the given *property* to *value*.

If the specified *value* is not valid according to the given *property's* size range, the *value* is adjusted to the nearest valid value within the size range.

See also

[value\(\)](#), [setRange\(\)](#), [valueChanged\(\)](#)

15.715.2.12 subDoublePropertyManager()

```
QtDoublePropertyManager * QtSizeFPropertyManager::subDoublePropertyManager ( ) const
```

Returns the manager that creates the nested *width* and *height* subproperties.

In order to provide editing widgets for the *width* and *height* properties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

15.715.2.13 uninitializedProperty()

```
void QtSizeFPropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.715.2.14 value()

```
QSizeF QtSizeFPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns an invalid size

See also

[setValue\(\)](#)

15.715.2.15 valueChanged

```
void QtSizeFPropertyManager::valueChanged (
    QtProperty * property,
    const QSizeF & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

15.715.2.16 valueText()

```
QString QtSizeFPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

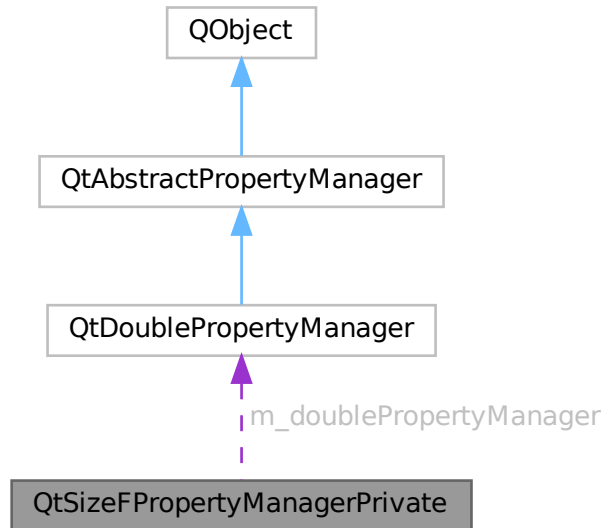
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.716 QtSizeFPropertyManagerPrivate Class Reference

Collaboration diagram for QtSizeFPropertyManagerPrivate:



Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Member Functions

- void **slotDoubleChanged** ([QtProperty](#) *property, double value)
- void **slotPropertyDestroyed** ([QtProperty](#) *property)
- void **setValue** ([QtProperty](#) *property, const QSizeF &val)
- void **setRange** ([QtProperty](#) *property, const QSizeF &minVal, const QSizeF &maxVal, const QSizeF &val)

Public Attributes

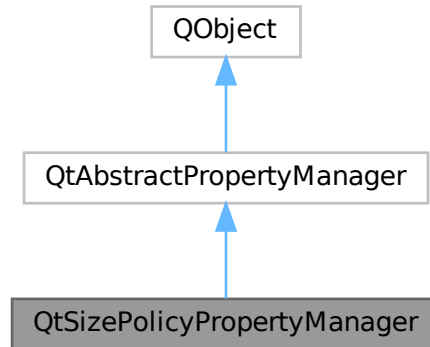
- PropertyValueMap **m_values**
- [QtDoublePropertyManager](#) * **m_doublePropertyManager**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToW**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToH**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_wToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_hToProperty**

The documentation for this class was generated from the following file:

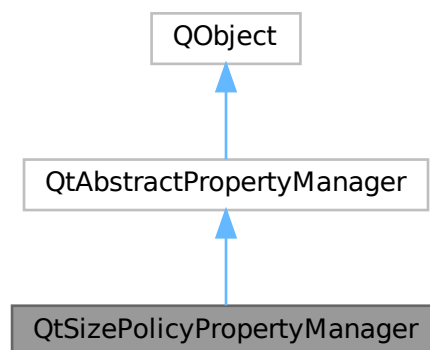
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.717 QSizePolicyPropertyManager Class Reference

Inheritance diagram for QSizePolicyPropertyManager:



Collaboration diagram for QSizePolicyPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, const QSizePolicy &val)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QSizePolicy &val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QSizePolicyPropertyManager](#) (QObject *parent=0)
- [~QSizePolicyPropertyManager](#) ()
- [QtIntPropertyManager](#) * [subIntPropertyManager](#) () const
- [QtEnumPropertyManager](#) * [subEnumPropertyManager](#) () const
- QSizePolicy [value](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.717.1 Constructor & Destructor Documentation**15.717.1.1 QSizePolicyPropertyManager()**

```
QSizePolicyPropertyManager::QSizePolicyPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.717.1.2 ~QSizePolicyPropertyManager()

```
QSizePolicyPropertyManager::~~QSizePolicyPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.717.2 Member Function Documentation**15.717.2.1 initializeProperty()**

```
void QSizePolicyPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.717.2.2 setValue

```
void QSizePolicyPropertyManager::setValue (
    QtProperty * property,
    const QSizePolicy & value ) [slot]
```

Sets the value of the given *property* to *value*. Nested properties are updated automatically.

See also

[value\(\)](#), [valueChanged\(\)](#)

15.717.2.3 subEnumPropertyManager()

`QtEnumPropertyManager * QtSizePolicyPropertyManager::subEnumPropertyManager ( ) const`

Returns the manager that creates the nested *horizontalPolicy* and *verticalPolicy* subproperties.

In order to provide editing widgets for the mentioned subproperties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

15.717.2.4 subIntPropertyManager()

`QtIntPropertyManager * QtSizePolicyPropertyManager::subIntPropertyManager ( ) const`

Returns the manager that creates the nested *horizontalStretch* and *verticalStretch* subproperties.

In order to provide editing widgets for the mentioned subproperties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

15.717.2.5 uninitializedProperty()

```
void QtSizePolicyPropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.717.2.6 value()

```
QSizePolicy QtSizePolicyPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property's* value.

If the given property is not managed by this manager, this function returns the default size policy.

See also

[setValue\(\)](#)

15.717.2.7 valueChanged

```
void QtSizePolicyPropertyManager::valueChanged (
    QtProperty * property,
    const QSizePolicy & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

15.717.2.8 valueText()

```
QString QtSizePolicyPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp Unknown size policy on reading invalid uic3 files

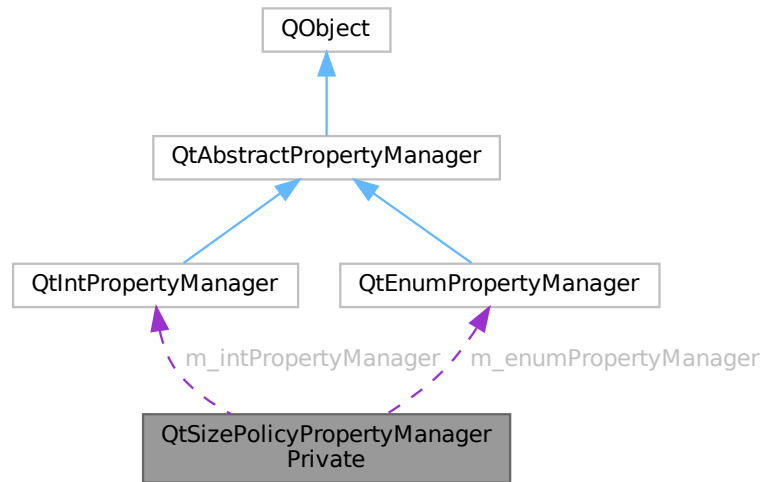
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.718 QtSizePolicyPropertyManagerPrivate Class Reference

Collaboration diagram for QtSizePolicyPropertyManagerPrivate:



Public Types

- typedef QMap< const [QtProperty](#) *, QSizePolicy > **PropertyValueMap**

Public Member Functions

- void **slotIntChanged** ([QtProperty](#) *property, int value)
- void **slotEnumChanged** ([QtProperty](#) *property, int value)
- void **slotPropertyDestroyed** ([QtProperty](#) *property)

Public Attributes

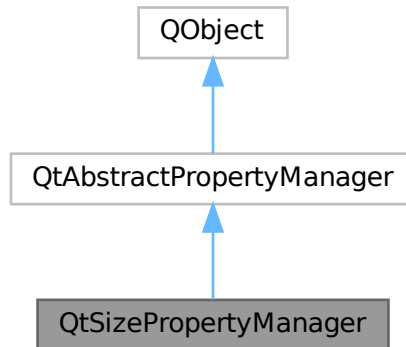
- PropertyValueMap **m_values**
- [QtIntPropertyManager](#) * **m_intPropertyManager**
- [QtEnumPropertyManager](#) * **m_enumPropertyManager**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToHPolicy**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToVPolicy**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToHStretch**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToVStretch**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_hPolicyToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_vPolicyToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_hStretchToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_vStretchToProperty**

The documentation for this class was generated from the following file:

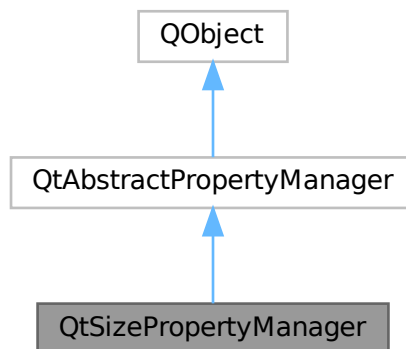
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.719 QSizePropertyManager Class Reference

Inheritance diagram for QSizePropertyManager:



Collaboration diagram for QSizePropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, const QSize &val)
- void [setMinimum](#) ([QtProperty](#) *property, const QSize &minVal)
- void [setMaximum](#) ([QtProperty](#) *property, const QSize &maxVal)
- void [setRange](#) ([QtProperty](#) *property, const QSize &minVal, const QSize &maxVal)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QSize &val)
- void [rangeChanged](#) ([QtProperty](#) *property, const QSize &minVal, const QSize &maxVal)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QSizePropertyManager](#) (QObject *parent=0)
- [~QSizePropertyManager](#) ()
- [QtIntPropertyManager](#) * [subIntPropertyManager](#) () const
- QSize [value](#) (const [QtProperty](#) *property) const
- QSize [minimum](#) (const [QtProperty](#) *property) const
- QSize [maximum](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.719.1 Constructor & Destructor Documentation

15.719.1.1 QSizePropertyManager()

```
QSizePropertyManager::QSizePropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.719.1.2 ~QSizePropertyManager()

```
QSizePropertyManager::~~QSizePropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.719.2 Member Function Documentation

15.719.2.1 initializeProperty()

```
void QSizePropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.719.2.2 maximum()

```
QSize QtSizePropertyManager::maximum (
    const QtProperty * property ) const
```

Returns the given *property*'s maximum size value.

See also

[setMaximum\(\)](#), [minimum\(\)](#), [setRange\(\)](#)

15.719.2.3 minimum()

```
QSize QtSizePropertyManager::minimum (
    const QtProperty * property ) const
```

Returns the given *property*'s minimum size value.

See also

[setMinimum\(\)](#), [maximum\(\)](#), [setRange\(\)](#)

15.719.2.4 rangeChanged

```
void QtSizePropertyManager::rangeChanged (
    QtProperty * property,
    const QSize & minimum,
    const QSize & maximum ) [signal]
```

This signal is emitted whenever a property created by this manager changes its range of valid sizes, passing a pointer to the *property* and the new *minimum* and *maximum* sizes.

See also

[setRange\(\)](#)

15.719.2.5 setMaximum

```
void QtSizePropertyManager::setMaximum (
    QtProperty * property,
    const QSize & maxVal ) [slot]
```

Sets the maximum size value for the given *property* to *maxVal*.

When setting the maximum size value, the minimum and current values are adjusted if necessary (ensuring that the size range remains valid and that the current value is within the range).

See also

[maximum\(\)](#), [setRange\(\)](#), [rangeChanged\(\)](#)

15.719.2.6 setMinimum

```
void QtSizePropertyManager::setMinimum (
    QtProperty * property,
    const QSize & minVal ) [slot]
```

Sets the minimum size value for the given *property* to *minVal*.

When setting the minimum size value, the maximum and current values are adjusted if necessary (ensuring that the size range remains valid and that the current value is within the range).

See also

[minimum\(\)](#), [setRange\(\)](#), [rangeChanged\(\)](#)

15.719.2.7 setRange

```
void QSizePropertyManager::setRange (
    QtProperty * property,
    const QSize & minimum,
    const QSize & maximum ) [slot]
```

Sets the range of valid values.

This is a convenience function defining the range of valid values in one go; setting the *minimum* and *maximum* values for the given *property* with a single function call.

When setting a new range, the current value is adjusted if necessary (ensuring that the value remains within the range).

See also

[setMinimum\(\)](#), [setMaximum\(\)](#), [rangeChanged\(\)](#)

15.719.2.8 setValue

```
void QSizePropertyManager::setValue (
    QtProperty * property,
    const QSize & value ) [slot]
```

Sets the value of the given *property* to *value*.

If the specified *value* is not valid according to the given *property's* size range, the *value* is adjusted to the nearest valid value within the size range.

See also

[value\(\)](#), [setRange\(\)](#), [valueChanged\(\)](#)

15.719.2.9 subIntPropertyManager()

```
QtIntPropertyManager * QSizePropertyManager::subIntPropertyManager ( ) const
```

Returns the manager that creates the nested *width* and *height* subproperties.

In order to provide editing widgets for the *width* and *height* properties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

15.719.2.10 uninitializedProperty()

```
void QSizePropertyManager::uninitializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.719.2.11 value()

```
QSize QSizePropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property's* value.

If the given *property* is not managed by this manager, this function returns an invalid size

See also

[setValue\(\)](#)

15.719.2.12 valueChanged

```
void QSizePropertyManager::valueChanged (
    QtProperty * property,
    const QSize & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

15.719.2.13 valueText()

```
QString QSizePropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

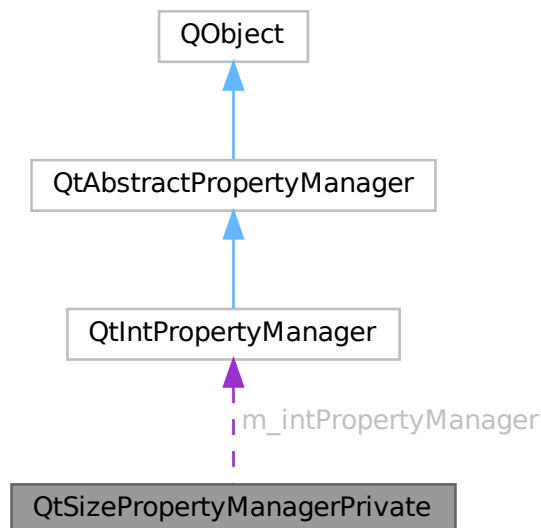
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.720 QSizePropertyManagerPrivate Class Reference

Collaboration diagram for QSizePropertyManagerPrivate:



Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Member Functions

- void **slotIntChanged** (QtProperty *property, int value)
- void **slotPropertyDestroyed** (QtProperty *property)
- void **setValue** (QtProperty *property, const QSize &val)
- void **setRange** (QtProperty *property, const QSize &minVal, const QSize &maxVal, const QSize &val)

Public Attributes

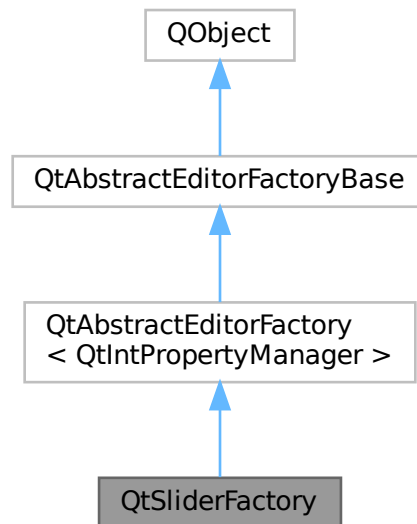
- PropertyValueMap **m_values**
- QtIntPropertyManager * **m_intPropertyManager**
- QMap< const QtProperty *, QtProperty * > **m_propertyToW**
- QMap< const QtProperty *, QtProperty * > **m_propertyToH**
- QMap< const QtProperty *, QtProperty * > **m_wToProperty**
- QMap< const QtProperty *, QtProperty * > **m_hToProperty**

The documentation for this class was generated from the following file:

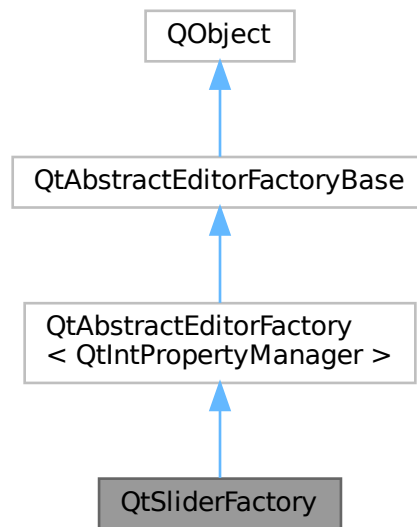
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.721 QtSliderFactory Class Reference

Inheritance diagram for QtSliderFactory:



Collaboration diagram for QtSliderFactory:



Public Member Functions

- [QtSliderFactory](#) (QObject *parent=0)
- [~QtSliderFactory](#) ()

Public Member Functions inherited from [QtAbstractEditorFactory < QtIntPropertyManager >](#)

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtIntPropertyManager *manager)
- void [removePropertyManager](#) (QtIntPropertyManager *manager)
- QSet< QtIntPropertyManager * > [propertyManagers](#) () const
- QtIntPropertyManager * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtIntPropertyManager *manager) override
- QWidget * [createEditor](#) (QtIntPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtIntPropertyManager *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactory < QtIntPropertyManager >](#)

- virtual QWidget * [createEditor](#) (QtIntPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtIntPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtIntPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

15.721.1 Constructor & Destructor Documentation

15.721.1.1 QtSliderFactory()

```
QtSliderFactory::QtSliderFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

15.721.1.2 ~QtSliderFactory()

```
QtSliderFactory::~QtSliderFactory ( )
```

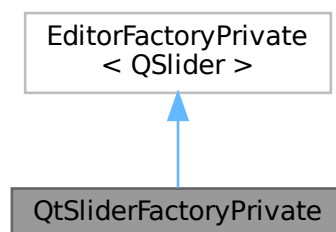
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

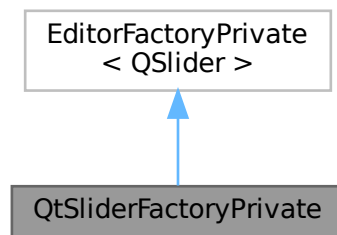
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.722 QtSliderFactoryPrivate Class Reference

Inheritance diagram for QtSliderFactoryPrivate:



Collaboration diagram for QtSliderFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, int value)
- void **slotRangeChanged** ([QtProperty](#) *property, int min, int max)
- void **slotSingleStepChanged** ([QtProperty](#) *property, int step)
- void **slotSetValue** (int value)

Public Member Functions inherited from [EditorFactoryPrivate< QSlider >](#)

- `QSlider *` **createEditor** ([QtProperty](#) *property, `QWidget *`parent)
- void **initializeEditor** ([QtProperty](#) *property, `QSlider *`e)
- void **slotEditorDestroyed** (`QObject *`object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QSlider >](#)

- typedef `QList< QSlider * >` **EditorList**
- typedef `QMap< QtProperty *, EditorList >` **PropertyToEditorListMap**
- typedef `QMap< QSlider *, QtProperty * >` **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate< QSlider >](#)

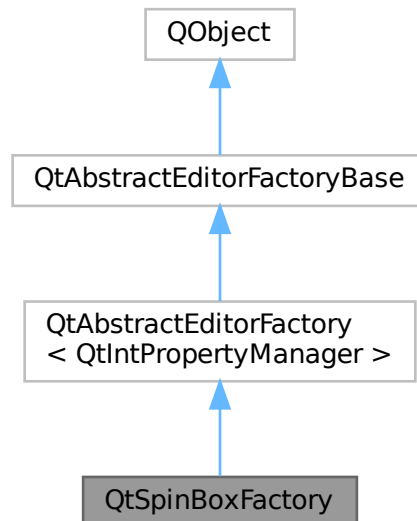
- `PropertyToEditorListMap` **m_createdEditors**
- `EditorToPropertyMap` **m_editorToProperty**

The documentation for this class was generated from the following file:

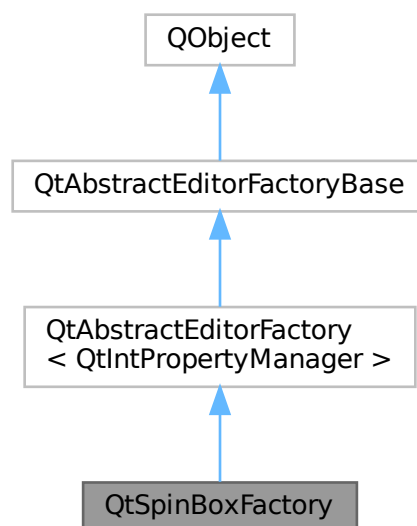
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.723 QtSpinBoxFactory Class Reference

Inheritance diagram for QtSpinBoxFactory:



Collaboration diagram for QtSpinBoxFactory:



Public Member Functions

- [QtSpinBoxFactory](#) (QObject *parent=0)

- [~QtSpinBoxFactory](#) ()

Public Member Functions inherited from [QtAbstractEditorFactory](#) < [QtIntPropertyManager](#) >

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtIntPropertyManager *manager)
- void [removePropertyManager](#) (QtIntPropertyManager *manager)
- QSet< [QtIntPropertyManager](#) * > [propertyManagers](#) () const
- [QtIntPropertyManager](#) * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtIntPropertyManager *manager) override
- QWidget * [createEditor](#) (QtIntPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtIntPropertyManager *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactory](#) < [QtIntPropertyManager](#) >

- virtual QWidget * [createEditor](#) (QtIntPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtIntPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtIntPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

15.723.1 Constructor & Destructor Documentation

15.723.1.1 QtSpinBoxFactory()

```
QtSpinBoxFactory::QtSpinBoxFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

15.723.1.2 ~QtSpinBoxFactory()

```
QtSpinBoxFactory::~~QtSpinBoxFactory ( )
```

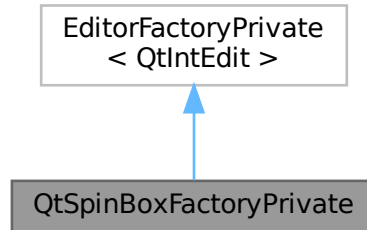
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

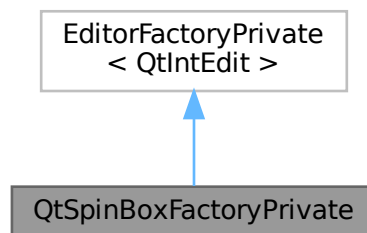
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.724 QtSpinBoxFactoryPrivate Class Reference

Inheritance diagram for QtSpinBoxFactoryPrivate:



Collaboration diagram for QtSpinBoxFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, int value)
- void **slotRangeChanged** ([QtProperty](#) *property, int min, int max)
- void **slotSingleStepChanged** ([QtProperty](#) *property, int step)
- void **slotSetValue** (int value)

Public Member Functions inherited from [EditorFactoryPrivate< QtIntEdit >](#)

- [QtIntEdit](#) * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, [QtIntEdit](#) *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QtIntEdit >](#)

- typedef QList< [QtIntEdit](#) * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< [QtIntEdit](#) *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate](#)< [QtIntEdit](#) >

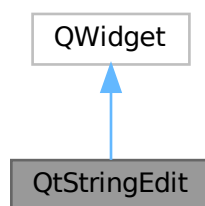
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

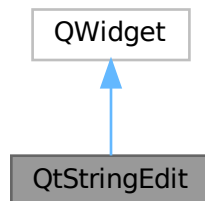
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.725 QtStringEdit Class Reference

Inheritance diagram for QtStringEdit:



Collaboration diagram for QtStringEdit:



Signals

- void **textChanged** (QString)
- void **textEdited** (QString)

Public Member Functions

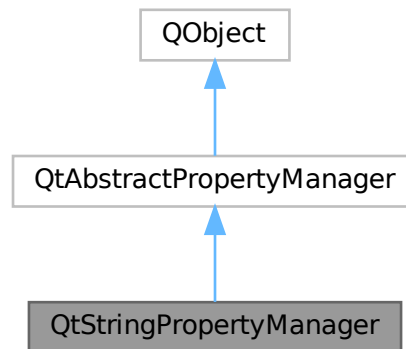
- **QtStringEdit** (QWidget *parent=Q_NULLPTR)
- QString **text** ()
- void **setText** (QString)
- void **setInitialText** (QString)
- const QValidator * **validator** ()
- void **setValidator** (QValidator *)

The documentation for this class was generated from the following files:

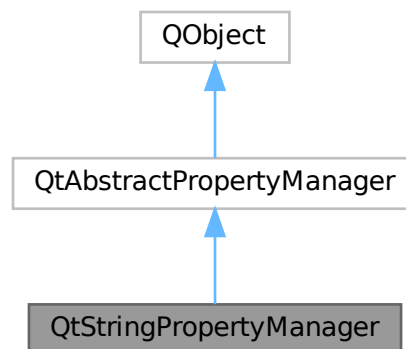
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils_p.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils.cpp

15.726 QtStringPropertyManager Class Reference

Inheritance diagram for QtStringPropertyManager:



Collaboration diagram for QtStringPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, const QString &val)
- void **setValueOnly** ([QtProperty](#) *property, const QString &val)
- void [setRegExp](#) ([QtProperty](#) *property, const QRegularExpression &[regExp](#))

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QString &val)

- void [regExpChanged](#) ([QtProperty](#) *property, const QRegularExpression &[regExp](#))

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtStringPropertyManager](#) (QObject *parent=0)
- [~QtStringPropertyManager](#) ()
- QString [value](#) (const [QtProperty](#) *property) const
- QString [initialValue](#) (const [QtProperty](#) *property) const
- QRegularExpression [regExp](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.726.1 Constructor & Destructor Documentation

15.726.1.1 [QtStringPropertyManager](#)()

```
QtStringPropertyManager::QtStringPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.726.1.2 [~QtStringPropertyManager](#)()

```
QtStringPropertyManager::~~QtStringPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.726.2 Member Function Documentation

15.726.2.1 [initializeProperty](#)()

```
void QtStringPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.726.2.2 regExp()

```
QRegularExpression QtStringPropertyManager::regExp (
    const QtProperty * property ) const
```

Returns the given *property*'s currently set regular expression.

If the given *property* is not managed by this manager, this function returns an empty expression.

See also

[setRegExp\(\)](#)

15.726.2.3 regExpChanged

```
void QtStringPropertyManager::regExpChanged (
    QtProperty * property,
    const QRegularExpression & regExp ) [signal]
```

This signal is emitted whenever a property created by this manager changes its currently set regular expression, passing a pointer to the *property* and the new *regExp* as parameters.

See also

[setRegExp\(\)](#)

15.726.2.4 setRegExp

```
void QtStringPropertyManager::setRegExp (
    QtProperty * property,
    const QRegularExpression & regExp ) [slot]
```

Sets the regular expression of the given *property* to *regExp*.

See also

[regExp\(\)](#), [setValue\(\)](#), [regExpChanged\(\)](#)

15.726.2.5 setValue

```
void QtStringPropertyManager::setValue (
    QtProperty * property,
    const QString & value ) [slot]
```

Sets the value of the given *property* to *value*.

If the specified *value* doesn't match the given *property*'s regular expression, this function does nothing.

See also

[value\(\)](#), [setRegExp\(\)](#), [valueChanged\(\)](#)

15.726.2.6 uninitializedProperty()

```
void QtStringPropertyManager::uninitializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.726.2.7 value()

```
QString QtStringPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given property is not managed by this manager, this function returns an empty string.

See also

[setValue\(\)](#)

15.726.2.8 valueChanged

```
void QtStringPropertyManager::valueChanged (
    QtProperty * property,
    const QString & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

15.726.2.9 valueText()

```
QString QtStringPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.727 QtStringPropertyManagerPrivate Class Reference

Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Attributes

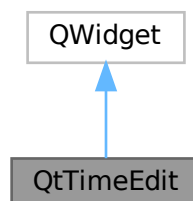
- QMap< const [QtProperty](#) *, [Data](#) > **m_values**

The documentation for this class was generated from the following file:

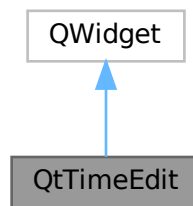
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.728 QtTimeEdit Class Reference

Inheritance diagram for QtTimeEdit:



Collaboration diagram for QtTimeEdit:



Signals

- void **timeChanged** (QTime)

Public Member Functions

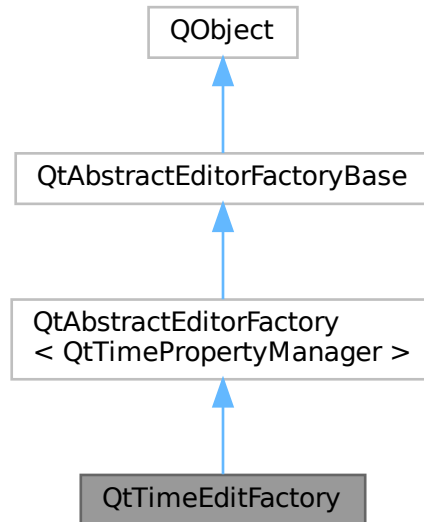
- **QtTimeEdit** (QWidget *parent=Q_NULLPTR)
- QTime **time** ()
- void **setTime** (QTime)
- void **setInitialTime** (QTime)
- void **setTimeRange** (QTime, QTime)
- void **setDisplayFormat** (QString)
- void **setCalendarPopup** (bool)

The documentation for this class was generated from the following files:

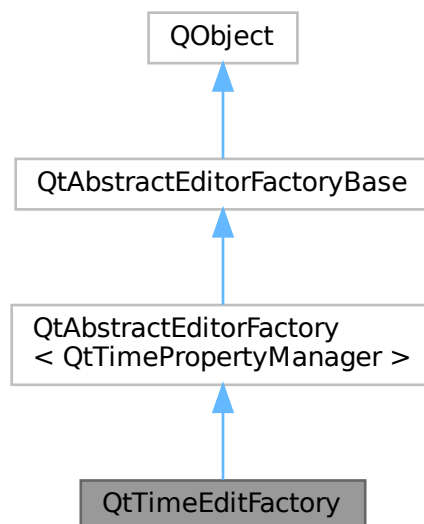
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils_p.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils.cpp

15.729 QtTimeEditFactory Class Reference

Inheritance diagram for QtTimeEditFactory:



Collaboration diagram for QtTimeEditFactory:



Public Member Functions

- [QtTimeEditFactory](#) (QObject *parent=0)

- [~QtTimeEditFactory](#) ()

Public Member Functions inherited from

[QtAbstractEditorFactory](#) < [QtTimePropertyManager](#) >

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtTimePropertyManager *manager)
- void [removePropertyManager](#) (QtTimePropertyManager *manager)
- QSet< [QtTimePropertyManager](#) * > [propertyManagers](#) () const
- [QtTimePropertyManager](#) * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtTimePropertyManager *manager) override
- QWidget * [createEditor](#) (QtTimePropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtTimePropertyManager *manager) override

Protected Member Functions inherited from

[QtAbstractEditorFactory](#) < [QtTimePropertyManager](#) >

- virtual QWidget * [createEditor](#) (QtTimePropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtTimePropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtTimePropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

15.729.1 Constructor & Destructor Documentation

15.729.1.1 QtTimeEditFactory()

```
QtTimeEditFactory::QtTimeEditFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

15.729.1.2 ~QtTimeEditFactory()

```
QtTimeEditFactory::~QtTimeEditFactory ( )
```

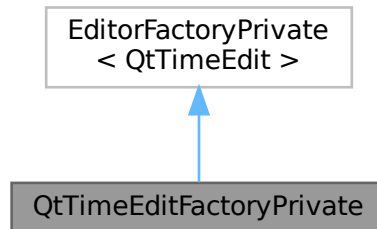
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

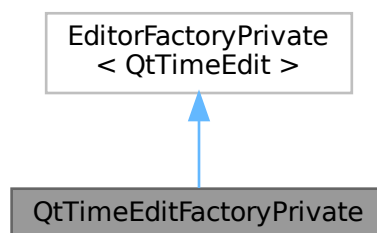
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.730 QtTimeEditFactoryPrivate Class Reference

Inheritance diagram for QtTimeEditFactoryPrivate:



Collaboration diagram for QtTimeEditFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, QTime value)
- void **slotSetValue** (QTime value)

Public Member Functions inherited from [EditorFactoryPrivate< QtTimeEdit >](#)

- [QtTimeEdit](#) * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, [QtTimeEdit](#) *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QtTimeEdit >](#)

- typedef QList< [QtTimeEdit](#) * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< [QtTimeEdit](#) *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate](#)< [QtTimeEdit](#) >

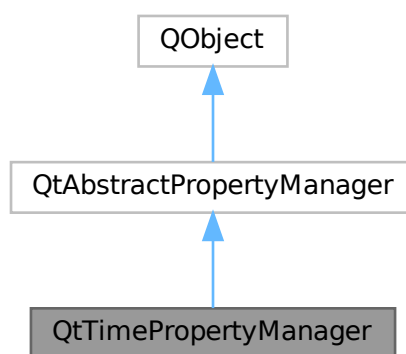
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

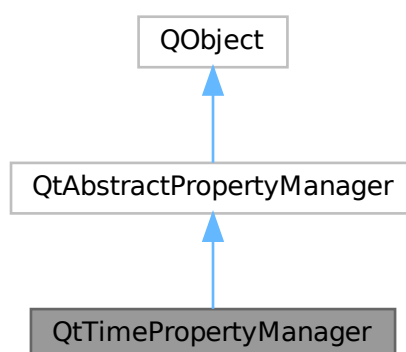
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

15.731 QtTimePropertyManager Class Reference

Inheritance diagram for QtTimePropertyManager:



Collaboration diagram for QtTimePropertyManager:

**Public Slots**

- void [setValue](#) ([QtProperty](#) *property, QTime val)
- void **setValueOnly** ([QtProperty](#) *property, QTime val)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, QTime val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtTimePropertyManager](#) (QObject *parent=0)
- [~QtTimePropertyManager](#) ()
- QTime [value](#) (const [QtProperty](#) *property) const
- QTime [initialValue](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

15.731.1 Constructor & Destructor Documentation

15.731.1.1 [QtTimePropertyManager](#)()

```
QtTimePropertyManager::QtTimePropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.731.1.2 [~QtTimePropertyManager](#)()

```
QtTimePropertyManager::~~QtTimePropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.731.2 Member Function Documentation

15.731.2.1 [initializeProperty](#)()

```
void QtTimePropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

15.731.2.2 setValue

```
void QtTimePropertyManager::setValue (
    QtProperty * property,
    QTime value ) [slot]
```

Sets the value of the given *property* to *value*.

See also

[value\(\)](#), [valueChanged\(\)](#)

15.731.2.3 uninitializedProperty()

```
void QtTimePropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

15.731.2.4 value()

```
QTime QtTimePropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given property is not managed by this manager, this function returns an invalid time object.

See also

[setValue\(\)](#)

15.731.2.5 valueChanged

```
void QtTimePropertyManager::valueChanged (
    QtProperty * property,
    QTime value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

15.731.2.6 valueText()

```
QString QtTimePropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.732 QtTimePropertyManagerPrivate Class Reference**Classes**

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Member Functions

- **QtTimePropertyManagerPrivate** ([QtTimePropertyManager](#) *q)

Public Attributes

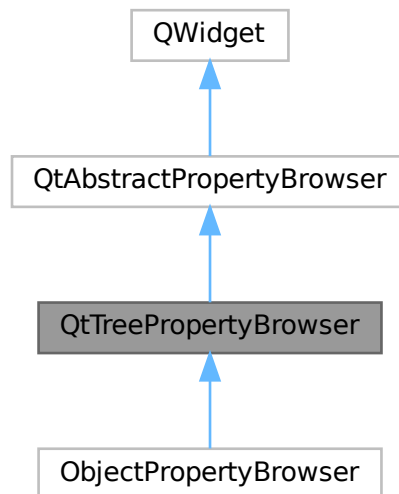
- const QString **m_format**
- PropertyValueMap **m_values**

The documentation for this class was generated from the following file:

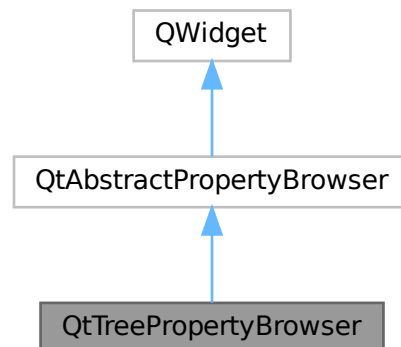
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

15.733 QtTreePropertyBrowser Class Reference

Inheritance diagram for QtTreePropertyBrowser:



Collaboration diagram for QtTreePropertyBrowser:



Public Types

- enum [ResizeMode](#) { **Interactive** , **Stretch** , **Fixed** , **ResizeToContents** }

Signals

- void [collapsed](#) ([QtBrowserItem](#) *item)
- void [expanded](#) ([QtBrowserItem](#) *item)

Signals inherited from [QtAbstractPropertyBrowser](#)

- void [currentItemChanged](#) ([QtBrowserItem](#) *)

Public Member Functions

- [QtTreePropertyBrowser](#) ([QWidget](#) *parent=0)
- [~QtTreePropertyBrowser](#) ()
- int **indentation** () const
- void **setIndentation** (int i)
- bool **rootsDecorated** () const
- void **setRootsDecorated** (bool show)
- bool **alternatingRowColors** () const
- void **setAlternatingRowColors** (bool enable)
- bool **isHeaderVisible** () const
- void **setHeaderVisible** (bool visible)
- [ResizeMode](#) **resizeMode** () const
- void **setResizeMode** ([ResizeMode](#) mode)
- int **splitterPosition** () const
- void **setSplitterPosition** (int position)
- void [setExpanded](#) ([QtBrowserItem](#) *item, bool [expanded](#))
- bool [isExpanded](#) ([QtBrowserItem](#) *item) const
- bool [isItemVisible](#) ([QtBrowserItem](#) *item) const
- void [setItemVisible](#) ([QtBrowserItem](#) *item, bool visible)
- void [setBackgroundColor](#) ([QtBrowserItem](#) *item, const [QColor](#) &color)
- [QColor](#) [backgroundColor](#) ([QtBrowserItem](#) *item) const
- [QColor](#) [calculatedBackgroundColor](#) ([QtBrowserItem](#) *item) const

- void **setPropertyWithoutValueMarked** (bool mark)
- bool **propertiesWithoutValueMarked** () const
- void **editItem** (QtBrowserItem *item)

Public Member Functions inherited from QtAbstractPropertyBrowser

- QtAbstractPropertyBrowser (QWidget *parent=0)
- ~QtAbstractPropertyBrowser ()
- QList< QtProperty * > **properties** () const
- QList< QtBrowserItem * > **items** (QtProperty *property) const
- QtBrowserItem * **topLevelItem** (QtProperty *property) const
- QList< QtBrowserItem * > **topLevelItems** () const
- void **clear** ()
- template<class PropertyManager >
void **setFactoryForManager** (PropertyManager *manager, QtAbstractEditorFactory< PropertyManager > *factory)
- void **unsetFactoryForManager** (QtAbstractPropertyManager *manager)
- QtBrowserItem * **currentItem** () const
- void **setCurrentItem** (QtBrowserItem *)

Protected Member Functions

- void **itemInserted** (QtBrowserItem *item, QtBrowserItem *afterItem) override
- void **itemRemoved** (QtBrowserItem *item) override
- void **itemChanged** (QtBrowserItem *item) override

Protected Member Functions inherited from QtAbstractPropertyBrowser

- virtual QWidget * **createEditor** (QtProperty *property, QWidget *parent)

Properties

- int **indentation**
indentation of the items in the tree view.
- bool **rootsDecorated**
whether to show controls for expanding and collapsing root items.
- bool **alternatingRowColors**
whether to draw the background using alternating colors. By default this property is set to true.
- bool **headerVisible**
whether to show the header.
- **SizeMode** **resizeMode**
the resize mode of sections in the header.
- int **splitterPosition**
the position of the splitter between the columns.
- bool **propertiesWithoutValueMarked**
whether to enable or disable marking properties without value.

Additional Inherited Members

Public Slots inherited from QtAbstractPropertyBrowser

- QtBrowserItem * **addProperty** (QtProperty *property)
- QtBrowserItem * **insertProperty** (QtProperty *property, QtProperty *afterProperty)
- void **removeProperty** (QtProperty *property)

15.733.1 Member Enumeration Documentation

15.733.1.1 ResizeMode

enum [QtTreePropertyBrowser::ResizeMode](#)

The resize mode specifies the behavior of the header sections.

\value Interactive The user can resize the sections. The sections can also be resized programmatically using [setSplitterPosition\(\)](#).

\value Fixed The user cannot resize the section. The section can only be resized programmatically using [setSplitterPosition\(\)](#).

\value Stretch QHeaderView will automatically resize the section to fill the available space. The size cannot be changed by the user or programmatically.

\value ResizeToContents QHeaderView will automatically resize the section to its optimal size based on the contents of the entire column. The size cannot be changed by the user or programmatically.

See also

[setResizeMode\(\)](#)

15.733.2 Constructor & Destructor Documentation

15.733.2.1 QtTreePropertyBrowser()

```
QtTreePropertyBrowser::QtTreePropertyBrowser (
    QWidget * parent = 0 )
```

Creates a property browser with the given *parent*.

15.733.2.2 ~QtTreePropertyBrowser()

```
QtTreePropertyBrowser::~QtTreePropertyBrowser ( )
```

Destroys this property browser.

Note that the properties that were inserted into this browser are *not* destroyed since they may still be used in other browsers. The properties are owned by the manager that created them.

See also

[QtProperty](#), [QtAbstractPropertyManager](#)

15.733.3 Member Function Documentation

15.733.3.1 backgroundColor()

```
QColor QtTreePropertyBrowser::backgroundColor (
    QtBrowserItem * item ) const
```

Returns the *item*'s color. If there is no color set for item it returns invalid color.

See also

[calculatedBackgroundColor\(\)](#), [setBackgroundColor\(\)](#)

15.733.3.2 calculatedBackgroundColor()

```
QColor QtTreePropertyBrowser::calculatedBackgroundColor (
    QtBrowserItem * item ) const
```

Returns the *item*'s color. If there is no color set for item it returns parent *item*'s color (if there is no color set for parent it returns grandparent's color and so on). In case the color is not set for *item* and it's top level item it returns invalid color.

See also

[backgroundColor\(\)](#), [setBackgroundColor\(\)](#)

15.733.3.3 collapsed

```
void QtTreePropertyBrowser::collapsed (
    QtBrowserItem * item ) [signal]
```

This signal is emitted when the *item* is collapsed.

See also

[expanded\(\)](#), [setExpanded\(\)](#)

15.733.3.4 editItem()

```
void QtTreePropertyBrowser::editItem (
    QtBrowserItem * item )
```

Sets the current item to *item* and opens the relevant editor for it.

15.733.3.5 expanded

```
void QtTreePropertyBrowser::expanded (
    QtBrowserItem * item ) [signal]
```

This signal is emitted when the *item* is expanded.

See also

[collapsed\(\)](#), [setExpanded\(\)](#)

15.733.3.6 isExpanded()

```
bool QtTreePropertyBrowser::isExpanded (
    QtBrowserItem * item ) const
```

Returns true if the *item* is expanded; otherwise returns false.

See also

[setExpanded\(\)](#)

15.733.3.7 isItemVisible()

```
bool QtTreePropertyBrowser::isItemVisible (
    QtBrowserItem * item ) const
```

Returns true if the *item* is visible; otherwise returns false.

See also

[setItemVisible\(\)](#)

Since

4.5

15.733.3.8 itemChanged()

```
void QtTreePropertyBrowser::itemChanged (
    QtBrowserItem * item ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyBrowser](#).

15.733.3.9 itemInserted()

```
void QtTreePropertyBrowser::itemInserted (
    QtBrowserItem * item,
    QtBrowserItem * afterItem ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyBrowser](#).

15.733.3.10 itemRemoved()

```
void QtTreePropertyBrowser::itemRemoved (
    QtBrowserItem * item ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyBrowser](#).

15.733.3.11 setBackgroundColor()

```
void QtTreePropertyBrowser::setBackgroundColor (
    QtBrowserItem * item,
    const QColor & color )
```

Sets the *item*'s background color to *color*. Note that while item's background is rendered every second row is being drawn with alternate color (which is a bit lighter than items *color*)

See also

[backgroundColor\(\)](#), [calculatedBackgroundColor\(\)](#)

15.733.3.12 setExpanded()

```
void QtTreePropertyBrowser::setExpanded (
    QtBrowserItem * item,
    bool expanded )
```

Sets the *item* to either collapse or expanded, depending on the value of *expanded*.

See also

[isExpanded\(\)](#), [expanded\(\)](#), [collapsed\(\)](#)

15.733.3.13 setItemVisible()

```
void QtTreePropertyBrowser::setItemVisible (
    QtBrowserItem * item,
    bool visible )
```

Sets the *item* to be visible, depending on the value of *visible*.

See also

[isItemVisible\(\)](#)

Since

4.5

15.733.4 Property Documentation

15.733.4.1 propertiesWithoutValueMarked

```
QtTreePropertyBrowser::propertiesWithoutValueMarked [read], [write]
```

whether to enable or disable marking properties without value.

When marking is enabled the item's background is rendered in dark color and item's foreground is rendered with light color.

See also

[propertiesWithoutValueMarked\(\)](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qttreepropertybrowser.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qttreepropertybrowser.cpp

15.734 QtTreePropertyBrowserPrivate Class Reference

Public Member Functions

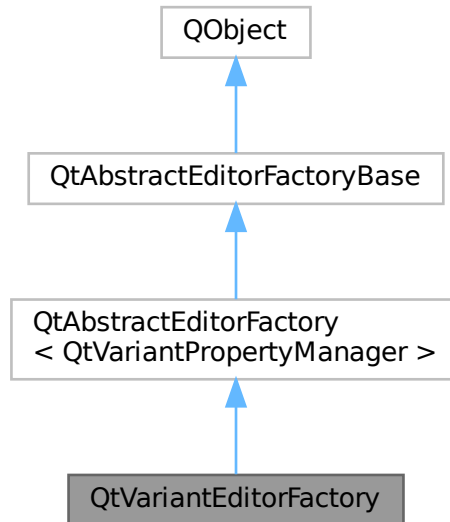
- void **init** (QWidget *parent)
- void **propertyInserted** (QtBrowserItem *index, QtBrowserItem *afterIndex)
- void **propertyRemoved** (QtBrowserItem *index)
- void **propertyChanged** (QtBrowserItem *index)
- QWidget * **createEditor** (QtProperty *property, QWidget *parent) const
- QtProperty * **indexToProperty** (const QModelIndex &index) const
- QTreeWidgetItem * **indexToItem** (const QModelIndex &index) const
- QtBrowserItem * **indexToBrowserItem** (const QModelIndex &index) const
- bool **lastColumn** (int column) const
- void **disableItem** (QTreeWidgetItem *item) const
- void **enableItem** (QTreeWidgetItem *item) const
- bool **hasValue** (QTreeWidgetItem *item) const
- void **slotCollapsed** (const QModelIndex &index)
- void **slotExpanded** (const QModelIndex &index)
- QColor **calculatedBackgroundColor** (QtBrowserItem *item) const
- QtPropertyEditorView * **treeWidget** () const
- bool **markPropertiesWithoutValue** () const
- QtBrowserItem * **currentItem** () const
- void **setCurrentItem** (QtBrowserItem *browserItem, bool block)
- void **editItem** (QtBrowserItem *browserItem)
- void **slotCurrentBrowserItemChanged** (QtBrowserItem *item)
- void **slotCurrentTreeItemChanged** (QTreeWidgetItem *newItem, QTreeWidgetItem *)
- QTreeWidgetItem * **editedItem** () const

The documentation for this class was generated from the following file:

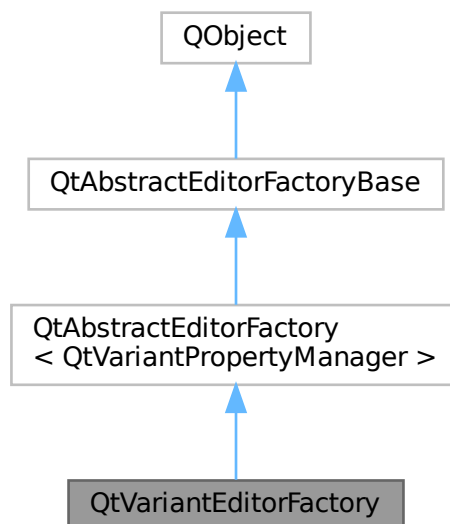
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qttreepropertybrowser.cpp

15.735 QtVariantEditorFactory Class Reference

Inheritance diagram for QtVariantEditorFactory:



Collaboration diagram for QtVariantEditorFactory:



Public Member Functions

- [QtVariantEditorFactory](#) (`QObject *parent=0`)

- [~QtVariantEditorFactory](#) ()

Public Member Functions inherited from

[QtAbstractEditorFactory](#) < [QtVariantPropertyManager](#) >

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtVariantPropertyManager *manager)
- void [removePropertyManager](#) (QtVariantPropertyManager *manager)
- QSet< [QtVariantPropertyManager](#) * > [propertyManagers](#) () const
- [QtVariantPropertyManager](#) * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtVariantPropertyManager *manager) override
- QWidget * [createEditor](#) (QtVariantPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtVariantPropertyManager *manager) override

Protected Member Functions inherited from

[QtAbstractEditorFactory](#) < [QtVariantPropertyManager](#) >

- virtual QWidget * [createEditor](#) (QtVariantPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtVariantPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtVariantPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

15.735.1 Constructor & Destructor Documentation

15.735.1.1 QtVariantEditorFactory()

```
QtVariantEditorFactory::QtVariantEditorFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

15.735.1.2 ~QtVariantEditorFactory()

```
QtVariantEditorFactory::~~QtVariantEditorFactory ( )
```

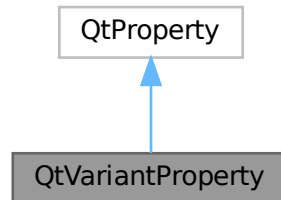
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

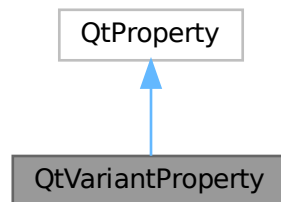
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.cpp

15.737 QtVariantProperty Class Reference

Inheritance diagram for QtVariantProperty:



Collaboration diagram for QtVariantProperty:



Public Member Functions

- [~QtVariantProperty](#) ()
- QVariant [value](#) () const
- QVariant [attributeValue](#) (const QString &attribute) const
- int [valueType](#) () const
- int [propertyType](#) () const
- void [setValue](#) (const QVariant &value)
- void [setAttribute](#) (const QString &attribute, const QVariant &value)

Public Member Functions inherited from [QtProperty](#)

- virtual [~QtProperty](#) ()
- QList< [QtProperty](#) * > [subProperties](#) () const
- [QtAbstractPropertyManager](#) * [propertyManager](#) () const
- QString [toolTip](#) () const
- QString [valueToolTip](#) () const
- QString [descriptionToolTip](#) () const
- QString [statusTip](#) () const
- QString [whatsThis](#) () const
- QString [propertyName](#) () const
- bool [isEnabled](#) () const

- bool [isModified](#) () const
- bool [hasValue](#) () const
- QIcon [valueIcon](#) () const
- QString [valueText](#) () const
- void [setToolTip](#) (const QString &text)
- void [setValueToolTip](#) (const QString &text)
- void [setDescriptionToolTip](#) (const QString &text)
- void [setStatusTip](#) (const QString &text)
- void [setWhatsThis](#) (const QString &text)
- void [setPropertyName](#) (const QString &text)
- void [setEnabled](#) (bool enable)
- void [setModified](#) (bool modified)
- void [addSubProperty](#) (QtProperty *property)
- void [insertSubProperty](#) (QtProperty *property, QtProperty *afterProperty)
- void [removeSubProperty](#) (QtProperty *property)

Protected Member Functions

- [QtVariantProperty](#) (QtVariantPropertyManager *manager)

Protected Member Functions inherited from [QtProperty](#)

- [QtProperty](#) (QtAbstractPropertyManager *manager)
- void [propertyChanged](#) ()

Friends

- class [QtVariantPropertyManager](#)

15.737.1 Constructor & Destructor Documentation

15.737.1.1 ~QtVariantProperty()

`QtVariantProperty::~QtVariantProperty ( )`

Destroys this property.

See also

[QtProperty::~QtProperty\(\)](#)

15.737.1.2 QtVariantProperty()

`QtVariantProperty::QtVariantProperty (
 QtVariantPropertyManager * manager) [protected]`

Creates a variant property using the given *manager*.

Do not use this constructor to create variant property instances; use the [QtVariantPropertyManager::addProperty\(\)](#) function instead. This constructor is used internally by the [QtVariantPropertyManager::createProperty\(\)](#) function.

See also

[QtVariantPropertyManager](#)

15.737.2 Member Function Documentation

15.737.2.1 attributeValue()

`QVariant QtVariantProperty::attributeValue (
 const QString & attribute) const`

Returns this property's value for the specified *attribute*.

[QtVariantPropertyManager](#) provides a couple of related functions: `\{QtVariantPropertyManager::attributes()\}` `attributes()` and `\{QtVariantPropertyManager::attributeType()\}` `attributeType()`.

See also

[setAttribute\(\)](#)

15.737.2.2 propertyType()

```
int QtVariantProperty::propertyType ( ) const
```

Returns this property's type.

[QtVariantPropertyManager](#) provides several related functions: [\{QtVariantPropertyManager::enumTypeId\(\)\}{enumTypeId\(\)↔}](#), [\{QtVariantPropertyManager::flagTypeId\(\)\}{flagTypeId\(\)↔}](#) and [\{QtVariantPropertyManager::groupTypeId\(\)\}{groupTypeId\(\)↔}](#).

See also

[valueType\(\)](#)

15.737.2.3 setAttribute()

```
void QtVariantProperty::setAttribute (
    const QString & attribute,
    const QVariant & value )
```

Sets the *attribute* of property to *value*.

[QtVariantPropertyManager](#) provides the related [\{QtVariantPropertyManager::setAttribute\(\)\}{setAttribute\(\)↔}](#) function.

See also

[attributeValue\(\)](#)

15.737.2.4 setValue()

```
void QtVariantProperty::setValue (
    const QVariant & value )
```

Sets the value of this property to *value*.

The specified *value* must be of the type returned by [valueType\(\)](#), or of a type that can be converted to [valueType\(\)](#) using the [QVariant::canConvert\(\)](#) function; otherwise this function does nothing.

See also

[value\(\)](#)

15.737.2.5 value()

```
QVariant QtVariantProperty::value ( ) const
```

Returns the property's current value.

See also

[valueType\(\)](#), [setValue\(\)](#)

15.737.2.6 valueType()

```
int QtVariantProperty::valueType ( ) const
```

Returns the type of this property's value.

See also

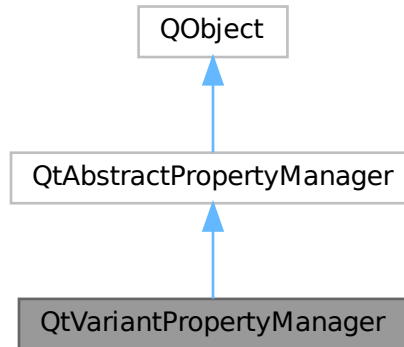
[propertyType\(\)](#)

The documentation for this class was generated from the following files:

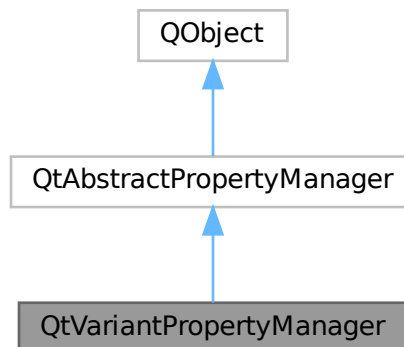
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.cpp

15.738 QtVariantPropertyManager Class Reference

Inheritance diagram for QtVariantPropertyManager:



Collaboration diagram for QtVariantPropertyManager:



Public Slots

- virtual void [setValue](#) ([QtProperty](#) *property, const QVariant &val)
- virtual void [setAttribute](#) ([QtProperty](#) *property, const QString &attribute, const QVariant &value)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QVariant &val)
- void [attributeChanged](#) ([QtProperty](#) *property, const QString &attribute, const QVariant &val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)

- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtVariantPropertyManager](#) (QObject *parent=0)
- [~QtVariantPropertyManager](#) ()
- virtual [QtVariantProperty](#) * [addProperty](#) (int [propertyType](#), const QString &name=QString())
- int [propertyType](#) (const [QtProperty](#) *property) const
- int [valueType](#) (const [QtProperty](#) *property) const
- [QtVariantProperty](#) * [variantProperty](#) (const [QtProperty](#) *property) const
- virtual bool [isPropertyTypeSupported](#) (int [propertyType](#)) const
- virtual int [valueType](#) (int [propertyType](#)) const
- virtual QStringList [attributes](#) (int [propertyType](#)) const
- virtual int [attributeType](#) (int [propertyType](#), const QString &attribute) const
- virtual QVariant [value](#) (const [QtProperty](#) *property) const
- virtual QVariant [attributeValue](#) (const [QtProperty](#) *property, const QString &attribute) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Static Public Member Functions

- static int [enumTypeId](#) ()
- static int [flagTypeId](#) ()
- static int [groupTypeId](#) ()
- static int [iconMapTypeId](#) ()

Protected Member Functions

- bool [hasValue](#) (const [QtProperty](#) *property) const override
- QString [valueText](#) (const [QtProperty](#) *property) const override
- QIcon [valueIcon](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override
- [QtProperty](#) * [createProperty](#) () override

15.738.1 Constructor & Destructor Documentation

15.738.1.1 [QtVariantPropertyManager](#)()

```
QtVariantPropertyManager::QtVariantPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

15.738.1.2 [~QtVariantPropertyManager](#)()

```
QtVariantPropertyManager::~~QtVariantPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

15.738.2 Member Function Documentation

15.738.2.1 addProperty()

```
QtVariantProperty * QtVariantPropertyManager::addProperty (
    int propertyType,
    const QString & name = QString() ) [virtual]
```

Creates and returns a variant property of the given *propertyType* with the given *name*.

If the specified *propertyType* is not supported by this variant manager, this function returns 0.

Do not use the inherited [QtAbstractPropertyManager::addProperty\(\)](#) function to create a variant property (that function will always return 0 since it will not be clear what type the property should have).

See also

[isPropertyTypeSupported\(\)](#)

15.738.2.2 attributeChanged

```
void QtVariantPropertyManager::attributeChanged (
    QtProperty * property,
    const QString & attribute,
    const QVariant & value ) [signal]
```

This signal is emitted whenever an attribute of a property created by this manager changes its value, passing a pointer to the *property*, the *attribute* and the new *value* as parameters.

See also

[setAttribute\(\)](#)

15.738.2.3 attributes()

```
QStringList QtVariantPropertyManager::attributes (
    int propertyType ) const [virtual]
```

Returns a list of the given *propertyType*'s attributes.

See also

[attributeValue\(\)](#), [attributeType\(\)](#)

15.738.2.4 attributeType()

```
int QtVariantPropertyManager::attributeType (
    int propertyType,
    const QString & attribute ) const [virtual]
```

Returns the type of the specified *attribute* of the given *propertyType*.

If the given *propertyType* is not supported by *this* manager, or if the given *propertyType* does not possess the specified *attribute*, this function returns `QMetaType::UnknownType`.

See also

[attributes\(\)](#), [valueType\(\)](#)

15.738.2.5 attributeValue()

```
QVariant QtVariantPropertyManager::attributeValue (
    const QtProperty * property,
    const QString & attribute ) const [virtual]
```

Returns the given *property*'s value for the specified *attribute*

If the given *property* was not created by *this* manager, or if the specified *attribute* does not exist, this function returns an invalid variant.

See also

[attributes\(\)](#), [attributeType\(\)](#), [setAttribute\(\)](#)

15.738.2.6 createProperty()

```
QtProperty * QtVariantPropertyManager::createProperty ( ) [override], [protected], [virtual]
```

Creates a property.

The base implementation produce [QtProperty](#) instances; Reimplement this function to make this manager produce objects of a [QtProperty](#) subclass.

See also

[addProperty\(\)](#), [initializeProperty\(\)](#)

Reimplemented from [QtAbstractPropertyManager](#).

15.738.2.7 enumTypeId()

```
QT_END_NAMESPACE QT_BEGIN_NAMESPACE int QtVariantPropertyManager::enumTypeId ( ) [static]
```

Returns the type id for an enum property.

Note that the property's value type can be retrieved using the [valueType\(\)](#) function (which is [QMetaType::Int](#) for the enum property type).

See also

[propertyType\(\)](#), [valueType\(\)](#)

15.738.2.8 flagTypeId()

```
int QtVariantPropertyManager::flagTypeId ( ) [static]
```

Returns the type id for a flag property.

Note that the property's value type can be retrieved using the [valueType\(\)](#) function (which is [QMetaType::Int](#) for the flag property type).

See also

[propertyType\(\)](#), [valueType\(\)](#)

15.738.2.9 groupTypeId()

```
int QtVariantPropertyManager::groupTypeId ( ) [static]
```

Returns the type id for a group property.

Note that the property's value type can be retrieved using the [valueType\(\)](#) function (which is [QMetaType::Unknown](#)↔[Type](#) for the group property type, since it doesn't provide any value).

See also

[propertyType\(\)](#), [valueType\(\)](#)

15.738.2.10 hasValue()

```
bool QtVariantPropertyManager::hasValue (
    const QtProperty * property ) const [override], [protected], [virtual]
```

Returns whether the given *property* has a value.

The default implementation of this function returns true.

See also

[QtProperty::hasValue\(\)](#)

Reimplemented from [QtAbstractPropertyManager](#).

15.738.2.11 iconMapTypeId()

```
int QtVariantPropertyManager::iconMapTypeId ( ) [static]
```

Returns the type id for a icon map attribute.

Note that the property's attribute type can be retrieved using the [attributeType\(\)](#) function.

See also

[attributeType\(\)](#), [QtEnumPropertyManager::enumIcons\(\)](#)

15.738.2.12 initializeProperty()

```
void QtVariantPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

This function is called whenever a new valid property pointer has been created, passing the pointer as parameter. The purpose is to let the manager know that the *property* has been created so that it can provide additional attributes for the new property, e.g. [QtIntPropertyManager](#) adds `\l {QtIntPropertyManager::value()}{value}`, `\l {QtIntPropertyManager::minimum()}{minimum}` and `\l {QtIntPropertyManager::maximum()}{maximum}` attributes. Since each manager subclass adds type specific attributes, this function is pure virtual and must be reimplemented when deriving from the [QtAbstractPropertyManager](#) class.

See also

[addProperty\(\)](#), [createProperty\(\)](#)

Implements [QtAbstractPropertyManager](#).

15.738.2.13 isPropertyTypeSupported()

```
bool QtVariantPropertyManager::isPropertyTypeSupported (
    int propertyType ) const [virtual]
```

Returns true if the given *propertyType* is supported by this variant manager; otherwise false.

See also

[propertyType\(\)](#)

15.738.2.14 propertyType()

```
int QtVariantPropertyManager::propertyType (
    const QtProperty * property ) const
```

Returns the given *property's* type.

See also

[valueType\(\)](#)

15.738.2.15 setAttribute

```
void QtVariantPropertyManager::setAttribute (
    QtProperty * property,
    const QString & attribute,
    const QVariant & value ) [virtual], [slot]
```

Sets the value of the specified *attribute* of the given *property*, to *value*.

The new *value's* type must be of the type returned by [attributeType\(\)](#), or of a type that can be converted to [attributeType\(\)](#) using the `QVariant::canConvert()` function, otherwise this function does nothing.

See also

[attributeValue\(\)](#), [QtVariantProperty::setAttribute\(\)](#), [attributeChanged\(\)](#)

15.738.2.16 setValue

```
void QtVariantPropertyManager::setValue (
    QtProperty * property,
    const QVariant & value ) [virtual], [slot]
```

Sets the value of the given *property* to *value*.

The specified *value* must be of a type returned by `valueType()`, or of type that can be converted to `valueType()` using the `QVariant::canConvert()` function, otherwise this function does nothing.

See also

[value\(\)](#), [QtVariantProperty::setValue\(\)](#), [valueChanged\(\)](#)

15.738.2.17 uninitializedProperty()

```
void QtVariantPropertyManager::uninitializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

This function is called just before the specified *property* is destroyed.

The purpose is to let the property manager know that the *property* is being destroyed so that it can remove the property's additional attributes.

See also

[clear\(\)](#), [propertyDestroyed\(\)](#)

Reimplemented from [QtAbstractPropertyManager](#).

15.738.2.18 value()

```
QVariant QtVariantPropertyManager::value (
    const QtProperty * property ) const [virtual]
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns an invalid variant.

See also

[setValue\(\)](#)

15.738.2.19 valueChanged

```
void QtVariantPropertyManager::valueChanged (
    QtProperty * property,
    const QVariant & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

15.738.2.20 valueIcon()

```
QIcon QtVariantPropertyManager::valueIcon (
    const QtProperty * property ) const [override], [protected], [virtual]
```

Returns an icon representing the current state of the given *property*.

The default implementation of this function returns an invalid icon.

See also

[QtProperty::valueIcon\(\)](#)

Reimplemented from [QtAbstractPropertyManager](#).

15.738.2.21 valueText()

```
QString QtVariantPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

Returns a string representing the current state of the given *property*.
The default implementation of this function returns an empty string.

See also

[QtProperty::valueText\(\)](#)

Reimplemented from [QtAbstractPropertyManager](#).

15.738.2.22 valueType() [1/2]

```
int QtVariantPropertyManager::valueType (
    const QtProperty * property ) const
```

Returns the given *property*'s value type.

See also

[propertyType\(\)](#)

15.738.2.23 valueType() [2/2]

```
int QtVariantPropertyManager::valueType (
    int propertyType ) const [virtual]
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns the value type associated with the given *propertyType*.

15.738.2.24 variantProperty()

```
QtVariantProperty * QtVariantPropertyManager::variantProperty (
    const QtProperty * property ) const
```

Returns the given *property* converted into a [QtVariantProperty](#).

If the *property* was not created by this variant manager, the function returns 0.

See also

[createProperty\(\)](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.cpp

15.739 QtVariantPropertyManagerPrivate Class Reference**Public Member Functions**

- void **slotValueChanged** ([QtProperty](#) *property, int val)
- void **slotRangeChanged** ([QtProperty](#) *property, int min, int max)
- void **slotSingleStepChanged** ([QtProperty](#) *property, int step)
- void **slotValueChanged** ([QtProperty](#) *property, double val)
- void **slotRangeChanged** ([QtProperty](#) *property, double min, double max)
- void **slotSingleStepChanged** ([QtProperty](#) *property, double step)
- void **slotDecimalsChanged** ([QtProperty](#) *property, int prec)
- void **slotValueChanged** ([QtProperty](#) *property, bool val)
- void **slotValueChanged** ([QtProperty](#) *property, const QString &val)
- void **slotRegExpChanged** ([QtProperty](#) *property, const QRegularExpression ®Exp)

- void **slotValueChanged** ([QtProperty](#) *property, QDate val)
- void **slotRangeChanged** ([QtProperty](#) *property, QDate min, QDate max)
- void **slotValueChanged** ([QtProperty](#) *property, QTime val)
- void **slotValueChanged** ([QtProperty](#) *property, const QDateTime &val)
- void **slotValueChanged** ([QtProperty](#) *property, const QKeySequence &val)
- void **slotValueChanged** ([QtProperty](#) *property, const QChar &val)
- void **slotValueChanged** ([QtProperty](#) *property, const QLocale &val)
- void **slotValueChanged** ([QtProperty](#) *property, const QPoint &val)
- void **slotValueChanged** ([QtProperty](#) *property, const QPointF &val)
- void **slotValueChanged** ([QtProperty](#) *property, const QSize &val)
- void **slotRangeChanged** ([QtProperty](#) *property, const QSize &min, const QSize &max)
- void **slotValueChanged** ([QtProperty](#) *property, const QSizeF &val)
- void **slotRangeChanged** ([QtProperty](#) *property, const QSizeF &min, const QSizeF &max)
- void **slotValueChanged** ([QtProperty](#) *property, const QRect &val)
- void **slotConstraintChanged** ([QtProperty](#) *property, const QRect &val)
- void **slotValueChanged** ([QtProperty](#) *property, const QRectF &val)
- void **slotConstraintChanged** ([QtProperty](#) *property, const QRectF &val)
- void **slotValueChanged** ([QtProperty](#) *property, const QColor &val)
- void **slotEnumChanged** ([QtProperty](#) *property, int val)
- void **slotEnumNamesChanged** ([QtProperty](#) *property, const QStringList &enumNames)
- void **slotEnumIconsChanged** ([QtProperty](#) *property, const QMap< int, QIcon > &enumIcons)
- void **slotValueChanged** ([QtProperty](#) *property, const QSizePolicy &val)
- void **slotValueChanged** ([QtProperty](#) *property, const QFont &val)
- void **slotValueChanged** ([QtProperty](#) *property, const QCursor &val)
- void **slotFlagChanged** ([QtProperty](#) *property, int val)
- void **slotFlagNamesChanged** ([QtProperty](#) *property, const QStringList &flagNames)
- void **slotPropertyInserted** ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void **slotPropertyRemoved** ([QtProperty](#) *property, [QtProperty](#) *parent)
- void **valueChanged** ([QtProperty](#) *property, const QVariant &val)
- int **internalPropertyToType** ([QtProperty](#) *property) const
- [QtVariantProperty](#) * **createSubProperty** ([QtVariantProperty](#) *parent, [QtVariantProperty](#) *after, [QtProperty](#) *internal)
- void **removeSubProperty** ([QtVariantProperty](#) *property)

Public Attributes

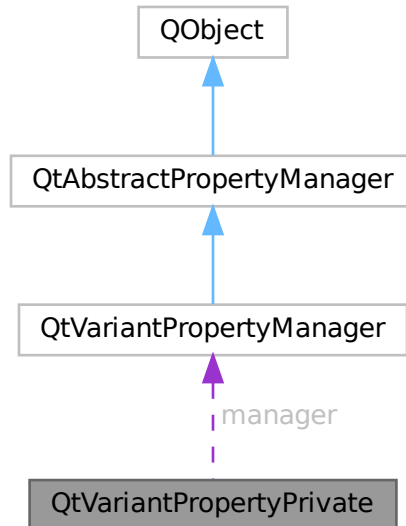
- bool **m_creatingProperty**
- bool **m_creatingSubProperties**
- bool **m_destroyingSubProperties**
- int **m_propertyType**
- QMap< int, [QtAbstractPropertyManager](#) * > **m_typeToPropertyManager**
- QMap< int, QMap< QString, int > > **m_typeToAttributeToAttributeType**
- QMap< const [QtProperty](#) *, QPair< [QtVariantProperty](#) *, int > > **m_propertyToType**
- QMap< int, int > **m_typeToValueType**
- QMap< [QtProperty](#) *, [QtVariantProperty](#) * > **m_internalToProperty**
- const QString **m_constraintAttribute**
- const QString **m_singleStepAttribute**
- const QString **m_decimalsAttribute**
- const QString **m_enumIconsAttribute**
- const QString **m_enumNamesAttribute**
- const QString **m_flagNamesAttribute**
- const QString **m_maximumAttribute**
- const QString **m_minimumAttribute**
- const QString **m_regExpAttribute**

The documentation for this class was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.cpp

15.740 QtVariantPropertyPrivate Class Reference

Collaboration diagram for QtVariantPropertyPrivate:



Public Member Functions

- **QtVariantPropertyPrivate** ([QtVariantPropertyManager](#) *m)

Public Attributes

- [QtVariantPropertyManager](#) * **manager**

The documentation for this class was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.cpp

15.741 Quadrature_if Class Reference

One-dimensional numerical quadrature abstraction.

```
#include <tools/Continuous/Quadrature_if.h>
```

Public Member Functions

- virtual double **integrate** (double min, double max, double(*f)(double), double precision) const =0

15.741.1 Detailed Description

One-dimensional numerical quadrature abstraction.

Architectural role:

- Planned extraction of integration responsibility from [Solver_if](#).

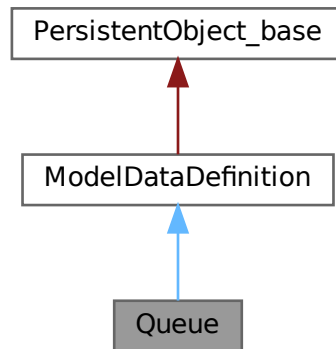
The documentation for this class was generated from the following file:

- tools/Continuous/Quadrature_if.h

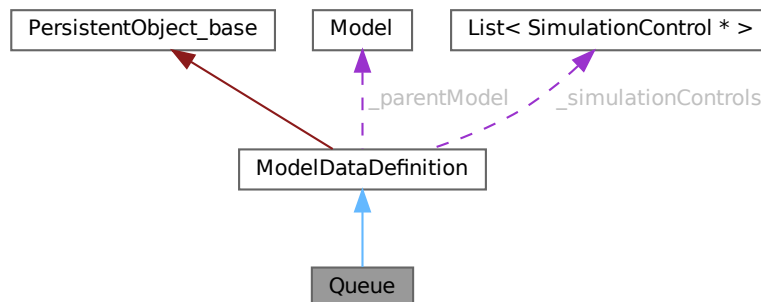
15.742 Queue Class Reference

```
#include <plugins/data/DiscreteProcessing/Queue.h>
```

Inheritance diagram for Queue:



Collaboration diagram for Queue:



Public Types

- enum class **OrderRule** : int {
FIFO = 0 , **LIFO** = 1 , **HIGHESTVALUE** = 2 , **SMALLESTVALUE** = 3 ,
num_elements = 4 }

Public Member Functions

- Queue** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- void **insertElement** ([Waiting](#) *modeldatum)
- void **removeElement** ([Waiting](#) *modeldatum)
- unsigned int **size** ()
- [Waiting](#) * **first** ()

- [Waiting](#) * **getAtRank** (unsigned int rank)
- void **setAttributeName** (std::string _attributeName)
- std::string **getAttributeName** () const
- void **setOrderRule** (OrderRule _orderRule)
- Queue::OrderRule **getOrderRule** () const
- void **setOrderRuleInt** (int orderRule)
- int **getOrderRuleInt** () const
- double **sumAttributesFromWaiting** (Util::identification attributeID)
- double **getAttributeFromWaitingRank** (unsigned int rank, Util::identification attributeID)

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- [Model](#) * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * **getSimulationControls** () const
Returns the simulation controls owned by this data definition.
- [TraceManager](#)::Level **getTraceLevelSpecific** () const

- *Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) (TraceManager::Level level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelspecific, bool traceLevelSpecific←Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static std::string [convertEnumToStr](#) (OrderRule rule)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) () override
- virtual void [_createInternalStatisticReporters](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- [bool _getSaveDefaultsOption \(\)](#)
- [virtual void _addSimulationControl \(SimulationControl *control\)](#)
- [virtual void _createInternalAndAttachedData \(\)](#)
- [virtual void _createAttachedAttributes \(\)](#)
- [virtual void _createNonEditableDataDefinitions \(\)](#)
- [virtual void _createEditableDataDefinitions \(\)](#)
- [void _templateCreateInternalStatisticReporters \(\)](#)
- [void _templateCreateAttachedAttributes \(\)](#)
- [void _templateCreateNonEditableDataDefinitions \(\)](#)
- [void _templateCreateEditableDataDefinitions \(\)](#)
- [void _internalStatisticReportersClear \(\)](#)
- *legacy helper methods used by the granular create* hooks*
- [void _internalDataClear \(\)](#)
- [void _internalDataInsert \(const std::string &key, \[ModelDataDefinition\]\(#\) *child\)](#)
- [void _internalDataRemove \(const std::string &key\)](#)
- [void _attachedDataClear \(\)](#)
- [void _attachedDataInsert \(const std::string &key, \[ModelDataDefinition\]\(#\) *data\)](#)
- [void _attachedDataRemove \(const std::string &key\)](#)
- [void _attachedAttributesInsert \(const std::vector< std::string > &neededNames\)](#)
- [void _statisticReporterInsert \(const std::string &key, \[ModelDataDefinition\]\(#\) *data\)](#)
- [void _statisticReporterRemove \(const std::string &key\)](#)
- [void _statisticReportersClear \(\)](#)
- [void _mandatoryAttachedAttributeInsert \(const std::string &key, \[ModelDataDefinition\]\(#\) *data\)](#)
- [void _mandatoryAttachedAttributeRemove \(const std::string &key\)](#)
- [void _mandatoryAttachedAttributesClear \(\)](#)
- [void _mandatoryEditableDataDefinitionInsert \(const std::string &key, \[ModelDataDefinition\]\(#\) *data\)](#)
- [void _mandatoryEditableDataDefinitionRemove \(const std::string &key\)](#)
- [void _mandatoryEditableDataDefinitionsClear \(\)](#)
- [void _optionalEditableDataDefinitionInsert \(const std::string &key, \[ModelDataDefinition\]\(#\) *data\)](#)
- [void _optionalEditableDataDefinitionRemove \(const std::string &key\)](#)
- [void _optionalEditableDataDefinitionsClear \(\)](#)
- [void _mandatoryNonEditableDataDefinitionInsert \(const std::string &key, \[ModelDataDefinition\]\(#\) *data\)](#)
- [void _mandatoryNonEditableDataDefinitionRemove \(const std::string &key\)](#)
- [void _mandatoryNonEditableDataDefinitionsClear \(\)](#)
- [void _optionalNonEditableDataDefinitionInsert \(const std::string &key, \[ModelDataDefinition\]\(#\) *data\)](#)
- [void _optionalNonEditableDataDefinitionRemove \(const std::string &key\)](#)
- [void _optionalNonEditableDataDefinitionsClear \(\)](#)
- [void _checkCreateAttachedReferencedDataDefinition \(const std::string &expression\)](#)
- *method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- [void trace \(const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed\)](#)
- *not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- [void traceError \(const std::string &text, const std::exception &e\)](#)
- [void traceError \(const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal\)](#)
- [void traceReport \(const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results\)](#)
- [void traceSimulation \(void *thisobject, double time, \[Entity\]\(#\) *entity, \[ModelComponent\]\(#\) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed\)](#)
- [void traceSimulation \(void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed\)](#)
- *Traces text to the simulation output.*
- [void traceSimulation \(void *thisobject, TraceManager::Level level, const std::string &text\)](#)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.742.1 Detailed Description

[Queue](#) module DESCRIPTION This data module may be utilized to change the ranking rule for a specified queue. The default ranking rule for all queues is First In, First Out unless otherwise specified in this module. There is an additional field that allows the queue to be defined as shared. TYPICAL USES Stack of work waiting for a resource at a [Process](#) module Holding area for documents waiting to be collated at a [Batch](#) module Prompt Description Name The name of the queue whose characteristics are being defined. This name must be unique. Type Ranking rule for the queue, which can be based on an attribute. Types include First In, First Out; Last In, First Out; Lowest [Attribute](#) Value (first); and Highest [Attribute](#) Value (first). A low attribute value would be 0 or 1, while a high value may be 200 or 300. [Attribute](#) Name [Attribute](#) that will be evaluated for the Lowest [Attribute](#) Value or Highest [Attribute](#) Value types. Entities with lowest or highest values of the attribute will be ranked first in the queue, with ties being broken using the First In, First Out rule. Shared Check box that determines whether a specific queue is used in multiple places within the simulation model. Shared queues can only be used for seizing resources (for example, with the [Seize](#) module from the Advanced [Process](#) panel). Report Statistics Specifies whether or not statistics will be collected automatically and stored in the report database for this queue.

15.742.2 Member Function Documentation

15.742.2.1 [_check\(\)](#)

```
bool Queue::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.742.2.2 [_createInternalStatisticReporters\(\)](#)

```
void Queue::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.742.2.3 [_getParserChangesInformation\(\)](#)

```
ParserChangesInformation * Queue::_getParserChangesInformation ( ) [override], [protected],
[virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDats. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dynamically linked to to simulator kernel to reflect the changes

Reimplemented from [ModelDataDefinition](#).

15.742.2.4 [_initBetweenReplications\(\)](#)

```
void Queue::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.742.2.5 _loadInstance()

```
bool Queue::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.742.2.6 _saveInstance()

```
void Queue::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.742.2.7 show()

```
std::string Queue::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- plugins/data/DiscreteProcessing/Queue.h
- plugins/data/DiscreteProcessing/Queue.cpp

15.743 QueueableItem Class Reference

Public Types

- enum class **QueueableType** : int { **QUEUE** = 0 , **SET** = 1 , **num_elements** = 2 }

Public Member Functions

- **QueueableItem** ([ModelDataDefinition](#) *queueOrSet, QueueableItem::QueueableType queueable↔ Type=QueueableItem::QueueableType::QUEUE, std::string index="0")
- **QueueableItem** ([Model](#) *model, std::string queueName)
- bool **loadInstance** ([PersistenceRecord](#) *fields)
- void **saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues)
- std::string **show** ()
- void **setIndex** (std::string index)
- std::string **getIndex** () const
- std::string **getQueueableName** () const
- std::string **getName** () const
- void **setQueue** ([Queue](#) *resource)
- [Queue](#) * **getQueue** () const
- void **setSet** ([Set](#) *set)

- [Set](#) * **getSet** () const
- void **setQueueableType** (QueueableType queueableType)
- QueueableType **getQueueableType** () const
- [ModelDataDefinition](#) * **getQueueable** () const
- void **setElementManager** ([ModelDataManager](#) *_modelDataManager)
- [List](#)< [SimulationControl](#) * > * **getSimulationControls** () const
- void **_addSimulationControl** ([SimulationControl](#) *control)
- std::string **getTypeDC** ()

Static Public Member Functions

- static std::string **convertEnumToStr** (QueueableType type)

The documentation for this class was generated from the following files:

- plugins/components/DiscreteProcessing/auxiliar/QueueableItem.h
- plugins/components/DiscreteProcessing/auxiliar/QueueableItem.cpp

15.744 testing::internal::Random Class Reference

Public Member Functions

- **Random** (uint32_t seed)
- void **Reseed** (uint32_t seed)
- uint32_t **Generate** (uint32_t range)

Static Public Attributes

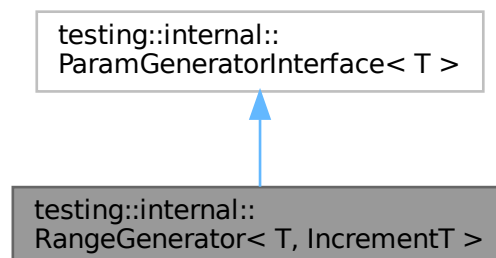
- static const uint32_t **kMaxRange** = 1u << 31

The documentation for this class was generated from the following files:

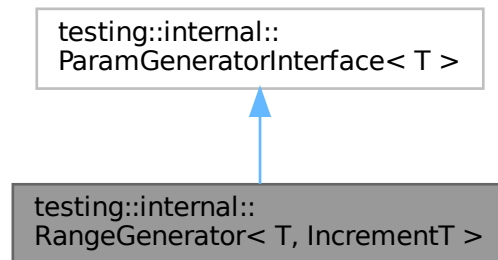
- gtest/gtest/internal/gtest-internal.h
- gtest/src/gtest.cc

15.745 testing::internal::RangeGenerator< T, IncrementT > Class Template Reference

Inheritance diagram for testing::internal::RangeGenerator< T, IncrementT >:



Collaboration diagram for testing::internal::RangeGenerator< T, IncrementT >:



Public Member Functions

- **RangeGenerator** (T begin, T end, IncrementT step)
- [ParamIteratorInterface](#)< T > * [Begin](#) () const override
- [ParamIteratorInterface](#)< T > * [End](#) () const override

Additional Inherited Members

Public Types inherited from [testing::internal::ParamGeneratorInterface](#)< T >

- typedef T **ParamType**

15.745.1 Member Function Documentation

15.745.1.1 Begin()

```
template<typename T , typename IncrementT >
ParamIteratorInterface< T > * testing::internal::RangeGenerator< T, IncrementT >::Begin ( )
const [inline], [override], [virtual]
Implements testing::internal::ParamGeneratorInterface< T >.
```

15.745.1.2 End()

```
template<typename T , typename IncrementT >
ParamIteratorInterface< T > * testing::internal::RangeGenerator< T, IncrementT >::End ( )
const [inline], [override], [virtual]
Implements testing::internal::ParamGeneratorInterface< T >.
```

The documentation for this class was generated from the following file:

- gtest/gtest/internal/gtest-param-util.h

15.746 testing::internal::RawBytesPrinter Struct Reference

Static Public Member Functions

- template<typename T , size_t = sizeof (T)>
static void **PrintValue** (const T &value, ::std::ostream *os)

The documentation for this struct was generated from the following file:

- gtest/gtest/gtest-printers.h

15.747 testing::internal::RE Class Reference

Public Member Functions

- **RE** (const [RE](#) &other)
- **RE** (const ::std::string ®ex)
- **RE** (const char *regex)
- const char * **pattern** () const

Static Public Member Functions

- static bool **FullMatch** (const ::std::string &str, const [RE](#) &re)
- static bool **PartialMatch** (const ::std::string &str, const [RE](#) &re)
- static bool **FullMatch** (const char *str, const [RE](#) &re)
- static bool **PartialMatch** (const char *str, const [RE](#) &re)

The documentation for this class was generated from the following file:

- gtest/gtest/internal/gtest-port.h

15.748 MassActionOdeSystem::Reaction Struct Reference

Public Attributes

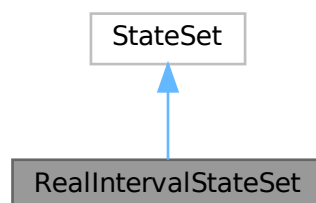
- std::string **name**
- double **rateConstant** = 0.0
- std::vector< [StoichiometricTerm](#) > **reactants**
- std::vector< [StoichiometricTerm](#) > **products**
- std::string **kineticLawExpression**
- std::vector< std::pair< std::string, double > > **parameters**
- std::string **reverseKineticLawExpression**
- double **reverseRateConstant** = 0.0
- bool **reversible** = false

The documentation for this struct was generated from the following file:

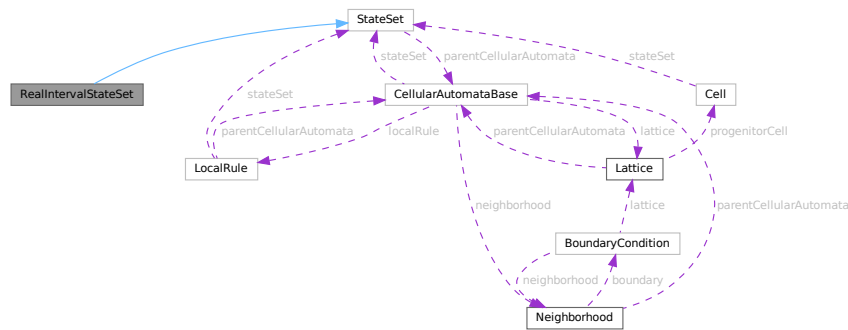
- tools/Biochemical/MassActionOdeSystem.h

15.749 RealIntervalStateSet Class Reference

Inheritance diagram for RealIntervalStateSet:



Collaboration diagram for RealIntervalStateSet:



Public Member Functions

- **RealIntervalStateSet** ([CellularAutomataBase](#) *parentCellularAutomata, std::optional< double > minimum=std::nullopt, std::optional< double > maximum=std::nullopt)
- bool **contains** (const [State](#) &state) const override
- std::unique_ptr< [State](#) > **createDefaultState** () const override
- std::unique_ptr< [State](#) > **parseState** (const std::string &text) const override
- bool **isContinuous** () const override
- std::string **name** () const override

Public Member Functions inherited from [StateSet](#)

- **StateSet** ([CellularAutomataBase](#) *parentCellularAutomata)
- **StateSet** (const [StateSet](#) &orig)
- virtual bool **isFinite** () const
- virtual bool **isDiscrete** () const
- virtual bool **isComposite** () const
- virtual std::vector< std::unique_ptr< [State](#) > > **enumerateStates** () const

Additional Inherited Members

Protected Attributes inherited from [StateSet](#)

- [CellularAutomataBase](#) * **parentCellularAutomata**

15.749.1 Member Function Documentation

15.749.1.1 contains()

```
bool RealIntervalStateSet::contains (
    const State & state ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

15.749.1.2 createDefaultState()

```
std::unique_ptr< State > RealIntervalStateSet::createDefaultState ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

15.749.1.3 isContinuous()

```
bool RealIntervalStateSet::isContinuous ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

15.749.1.4 name()

`std::string RealIntervalStateSet::name ( ) const [inline], [override], [virtual]`
 Reimplemented from [StateSet](#).

15.749.1.5 parseState()

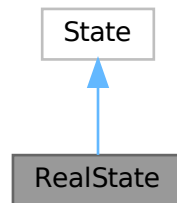
`std::unique_ptr< State > RealIntervalStateSet::parseState (`
 `const std::string & text ) const [inline], [override], [virtual]`
 Reimplemented from [StateSet](#).

The documentation for this class was generated from the following file:

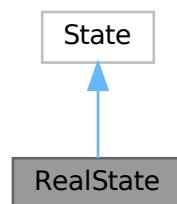
- `plugins/components/ModalModel/CellularAutomata/StateSet.h`

15.750 RealState Class Reference

Inheritance diagram for RealState:



Collaboration diagram for RealState:



Public Member Functions

- **RealState** (double value=0.0)
- `std::unique_ptr< State > clone ( )` const override
- `bool equals (const State &other)` const override
- `std::string toString ( )` const override
- `std::string typeName ( )` const override
- `long getValue ( )` const override
- `void setValue (long value)` override

- double **realValue** () const
- void **setRealValue** (double value)

Public Member Functions inherited from [State](#)

- **State** (long value)
- **State** (const [State](#) &orig)
- virtual std::string **show** () const
- template<typename T >
const T * **as** () const noexcept
- template<typename T >
T * **as** () noexcept

Additional Inherited Members

Protected Attributes inherited from [State](#)

- long **_value**

15.750.1 Member Function Documentation

15.750.1.1 clone()

std::unique_ptr< [State](#) > RealState::clone () const [inline], [override], [virtual]
Reimplemented from [State](#).

15.750.1.2 equals()

bool RealState::equals (
const [State](#) & other) const [inline], [override], [virtual]
Reimplemented from [State](#).

15.750.1.3 getValue()

long RealState::getValue () const [inline], [override], [virtual]
Reimplemented from [State](#).

15.750.1.4 setValue()

void RealState::setValue (
long value) [inline], [override], [virtual]
Reimplemented from [State](#).

15.750.1.5 toString()

std::string RealState::toString () const [inline], [override], [virtual]
Reimplemented from [State](#).

15.750.1.6 typeName()

std::string RealState::typeName () const [inline], [override], [virtual]
Reimplemented from [State](#).

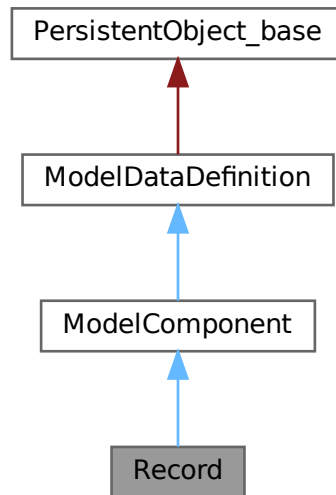
The documentation for this class was generated from the following file:

- plugins/components/ModalModel/CellularAutomata/State.h

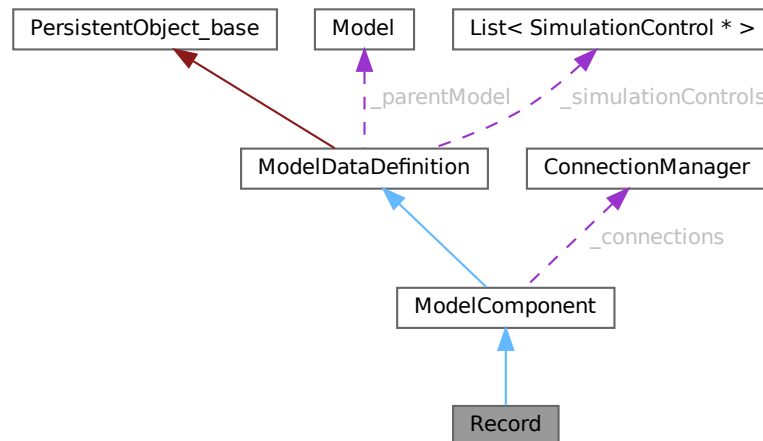
15.751 Record Class Reference

```
#include <plugins/components/InputOutput/Record.h>
```

Inheritance diagram for Record:



Collaboration diagram for Record:



Public Member Functions

- **Record** ([Model](#) *model, std::string name="")
- void **setFilename** (std::string filename)
- std::string **getFileName** () const
- void **setExpression** (const std::string expression)

- std::string **getExpression** () const
- void **setExpressionName** (std::string expressionName)
- std::string **getExpressionName** () const
- void **setDatasetName** (std::string datasetName)
- std::string **getDatasetName** () const
- void **setRandomVariableName** (std::string randomVariableName)
- std::string **getRandomVariableName** () const
- void **setVariableType** (std::string variableType)
- std::string **getVariableType** () const
- void **setDatasetDescription** (std::string description)
- std::string **getDatasetDescription** () const
- **StatisticsCollector** * **getCstatExpression** () const
- bool **getTimeDependent** () const
- void **setTimeDependent** (bool timeDependent)
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.

- `const std::string & getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from ModelComponent

- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static bool Check (ModelComponent *component)`
Validates a component instance.
- `static void CreateInternalData (ModelComponent *component)`
Creates internal data required by a component instance.
- `static void SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
Serializes a component instance into a persistence record.
- `static void DispatchEvent (Event *event)`
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_initBetweenReplications](#) () override
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createInternalStatisticReporters](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)

- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.751.1 Detailed Description

Record module DESCRIPTION This module is used to collect statistics in the simulation model. Various types of observational statistics are available, including time between exits through the module, entity statistics (such as time or costing), general observations, and interval statistics (from some time stamp to the current simulation time). A count type of statistic is available as well. Tally and **Counter** sets can also be specified. TYPICAL USES Collect the number of jobs completed each hour Count how many orders have been late being fulfilled **Record** the time spent by priority customers in the main check-out line PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Type Type of observational (tally) or count statistic to be generated. Count will increase or decrease the value of the named statistic by the specified value. **Entity** Statistics will generate general entity statistics, such as time and costing/duration information. Time Interval will calculate and record the difference between a specified attribute's value and current simulation time. Time Between will track and record the time between entities entering the module. Expression will record the value of the specified expression. **Attribute** Name Name of the attribute whose value will be used for the interval statistics. Applies only when Type is Interval. Value Value that will be recorded to the observational statistic when Type is Expression or added to the counter when Type is Count. Tally Name This field defines the symbol name of the tally into which the observation is to be recorded. Applies only when Type is Time Interval, Time Between, or Expression. **Counter** This field defines the symbol name of the counter to Name increment/decrement. Applies only when Type is **Counter**. **Record** into **Set** Check box to specify whether or not a tally or counter set will be used. Tally **Set** Name Name of the tally set that will be used to record the observational-type statistic. Applies only when Type is Time Interval, Time Between, or Expression. **Counter Set** Name Name of the counter set that will be used to record the count-type statistic. Applies only when Type is Count. **Set** Index Index into the tally or counter set.

15.751.2 Member Function Documentation

15.751.2.1 _check()

```
bool Record::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.751.2.2 _createInternalStatisticReporters()

```
void Record::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.751.2.3 _initBetweenReplications()

```
void Record::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.751.2.4 _loadInstance()

```
bool Record::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.751.2.5 _onDispatchEvent()

```
void Record::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
---------------	---

Parameters

<i>inputPortNumber</i>	Input port where the entity arrived.
------------------------	--------------------------------------

Implements [ModelComponent](#).

15.751.2.6 _saveInstance()

```
void Record::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.751.2.7 setDatasetDescription()

```
void Record::setDatasetDescription (
    std::string description )
```

Sets the free-text dataset description emitted in enriched [Record](#) file headers.

15.751.2.8 setDatasetName()

```
void Record::setDatasetName (
    std::string datasetName )
```

Sets the analytical dataset name emitted in enriched [Record](#) file headers.

15.751.2.9 setExpressionName()

```
void Record::setExpressionName (
    std::string expressionName )
```

Sets the legacy expression-name metadata kept for old [Record](#) datasets.

15.751.2.10 setRandomVariableName()

```
void Record::setRandomVariableName (
    std::string randomVariableName )
```

Sets the random variable name emitted in enriched [Record](#) file headers.

15.751.2.11 setVariableType()

```
void Record::setVariableType (
    std::string variableType )
```

Sets the canonical random variable type for enriched [Record](#) file headers.

15.751.2.12 show()

```
std::string Record::show ( ) [override], [virtual]
virtual public methods
```

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

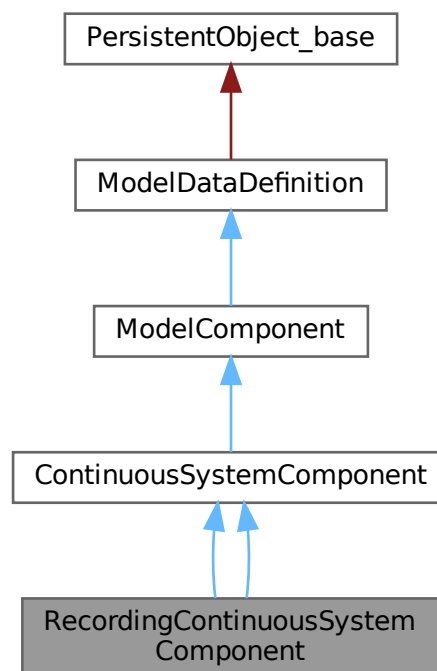
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

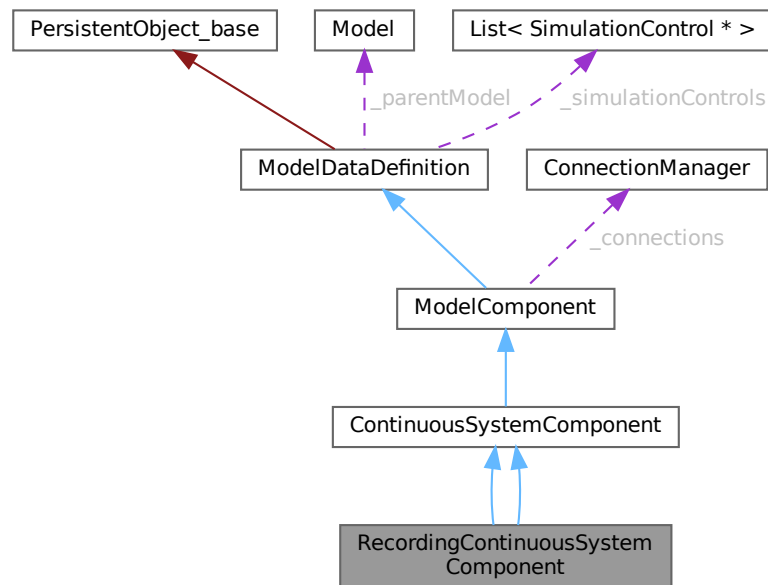
- plugins/components/InOutOutput/Record.h
- plugins/components/InOutOutput/Record.cpp

15.752 RecordingContinuousSystemComponent Class Reference

Inheritance diagram for RecordingContinuousSystemComponent:



Collaboration diagram for RecordingContinuousSystemComponent:



Public Member Functions

- **RecordingContinuousSystemComponent** ([Model](#) *model, [ODESolver](#) *solver, std::vector< [Sample](#) > *samples, std::string name="")
- **RecordingContinuousSystemComponent** ([Model](#) *model, [ODESolver](#) *solver, std::vector< [Sample](#) > *samples, std::string name="")

Public Member Functions inherited from [ContinuousSystemComponent](#)

- **ContinuousSystemComponent** ([Model](#) *model, std::string name="")
- virtual std::string [show](#) () override
- *virtual public methods*
- void **setOdeSolver** ([ODESolver](#) *solver)
- [ODESolver](#) * **getOdeSolver** () const
- void **setOdeSolverName** (std::string name)
- std::string **getOdeSolverName** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
- *constructors*
- virtual ~**ModelComponent** ()
- *Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * **getConnectionManager** () const
- *new public user methods for this component*
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
- *Connects this component to another component.*
- bool **hasBreakpointAt** ()
- *Indicates whether a simulation breakpoint is set on this component.*

- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) (TraceManager::Level level)

Updates the trace level used by this definition.

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific←Enabled=true)

Defines the specific trace level and optionally enables it immediately.

- bool [isTraceLevelSpecificEnabled](#) () const

Indicates whether the specific trace level is enabled.

- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Protected Member Functions

- void [_onDispatchEvent](#) (Entity *entity, unsigned int inputPortNumber) override

Handles a dispatched event for this component type.

- void [_onDispatchEvent](#) (Entity *entity, unsigned int inputPortNumber) override

Handles a dispatched event for this component type.

Protected Member Functions inherited from [ContinuousSystemComponent](#)

- virtual bool [_loadInstance](#) (PersistenceRecord *fields) override

virtual protected method that must be overridden

- virtual void [_saveInstance](#) (PersistenceRecord *fields, bool saveDefaultValues) override

Saves object fields into a serialization structure.

- virtual bool [_check](#) (std::string &errorMessage) override

could be overridden by derived classes

- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) (SimulationControl *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()

legacy helper methods used by the granular create hooks*

- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)

- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Static Public Member Functions inherited from [ContinuousSystemComponent](#)

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
- static bool **Check** ([ModelComponent](#) *component)
- static void **CreateInternalData** ([ModelComponent](#) *component)
- static void **DispatchEvent** ([Event](#) *event)

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * [_connections](#) = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification [_id](#)
- std::string [_typename](#)
- bool [_reportStatistics](#)
- bool [_hasChanged](#)
- unsigned int [_modelLevel](#) = 0
- [Model](#) * [_parentModel](#)
- [List](#)< [SimulationControl](#) * > * [_simulationControls](#) = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level [_traceLevelSpecific](#)
- bool [_traceLevelSpecificEnabled](#) = false

15.752.1 Member Function Documentation

15.752.1.1 [_onDispatchEvent\(\)](#) [1/2]

```
void RecordingContinuousSystemComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [inline], [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Reimplemented from [ContinuousSystemComponent](#).

15.752.1.2 [_onDispatchEvent\(\)](#) [2/2]

```
void RecordingContinuousSystemComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [inline], [override], [protected], [virtual]
```

Handles a dispatched event for this component type.
This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

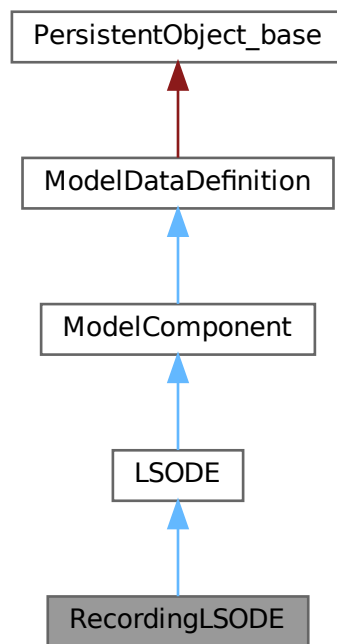
Reimplemented from [ContinuousSystemComponent](#).

The documentation for this class was generated from the following files:

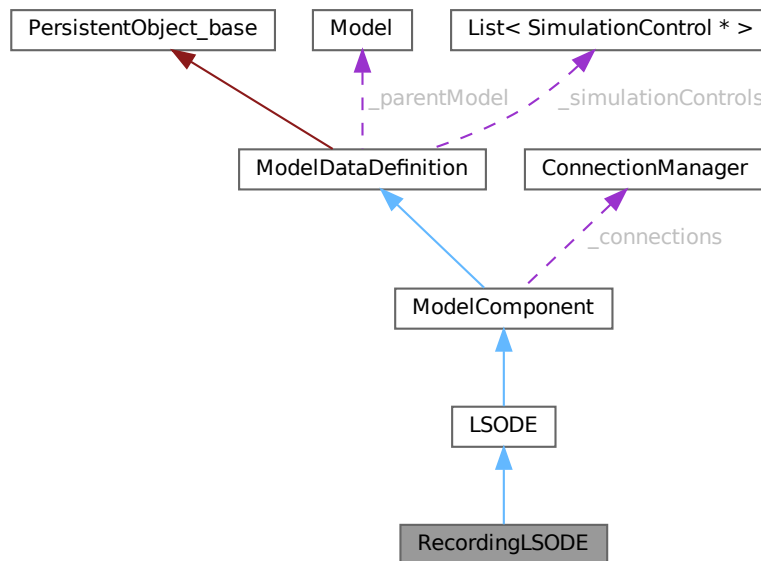
- tests/demo_continuous_trace.cpp
- tests/test_continuous_system.cpp

15.753 RecordingLSODE Class Reference

Inheritance diagram for RecordingLSODE:



Collaboration diagram for RecordingLSODE:



Public Member Functions

- **RecordingLSODE** ([Model](#) *model, [Variable](#) *timeVariable, [Variable](#) *stateVariable, std::vector< [LSODESample](#) > *samples, std::string name="")

Public Member Functions inherited from [LSODE](#)

- **LSODE** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
- *virtual public methods*
- void **setTimeVariable** ([Variable](#) *_timeVariable)
- [Variable](#) * **getTimeVariable** () const
- void **setStep** (double _step)
- double **getStep** () const
- void **setVariable** ([Variable](#) *_variables)
- [Variable](#) * **getVariable** () const
- [List](#)< std::string > * **getDiffEquations** () const
- void **addDiffEquation** (std::string newDiffEquation)
- void **removeDiffEquation** (std::string diffEquation)
- void **setFilename** (std::string filename)
- std::string **getFileName** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
- *constructors*
- virtual ~**ModelComponent** ()
- *Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * **getConnectionManager** () const
- *new public user methods for this component*

- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool **hasChanged**)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

- Returns the simulation controls owned by this data definition.*
- `TraceManager::Level` [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- `void` [setTraceLevelSpecific](#) (`TraceManager::Level` level)
Updates the trace level used by this definition.
- `void` [defineTraceLevelSpecific](#) (`TraceManager::Level` traceLevelspecific, `bool` traceLevelSpecific↔ Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- `bool` [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- `void` [setTraceLevelSpecificEnabled](#) (`bool` traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Protected Member Functions

- `void` [_onDispatchEvent](#) (`Entity` *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.

Protected Member Functions inherited from [LSODE](#)

- `virtual bool` [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- `virtual void` [_saveInstance](#) ([PersistenceRecord](#) *fields, `bool` saveDefaultValues) override
Saves object fields into a serialization structure.
- `virtual bool` [_check](#) (`std::string` &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool` [_getSaveDefaultsOption](#) ()
- `virtual` [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- `virtual void` [_initBetweenReplications](#) ()
- `virtual void` [_addSimulationControl](#) ([SimulationControl](#) *control)
- `virtual void` [_createInternalAndAttachedData](#) ()
- `virtual void` [_createInternalStatisticReporters](#) ()
- `virtual void` [_createAttachedAttributes](#) ()
- `virtual void` [_createNonEditableDataDefinitions](#) ()
- `virtual void` [_createEditableDataDefinitions](#) ()
- `void` [_templateCreateInternalStatisticReporters](#) ()
- `void` [_templateCreateAttachedAttributes](#) ()
- `void` [_templateCreateNonEditableDataDefinitions](#) ()
- `void` [_templateCreateEditableDataDefinitions](#) ()
- `void` [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- `void` [_internalDataClear](#) ()
- `void` [_internalDataInsert](#) (const `std::string` &key, [ModelDataDefinition](#) *child)
- `void` [_internalDataRemove](#) (const `std::string` &key)
- `void` [_attachedDataClear](#) ()
- `void` [_attachedDataInsert](#) (const `std::string` &key, [ModelDataDefinition](#) *data)
- `void` [_attachedDataRemove](#) (const `std::string` &key)
- `void` [_attachedAttributesInsert](#) (const `std::vector`< `std::string` > &neededNames)
- `void` [_statisticReporterInsert](#) (const `std::string` &key, [ModelDataDefinition](#) *data)
- `void` [_statisticReporterRemove](#) (const `std::string` &key)
- `void` [_statisticReportersClear](#) ()

- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void **traceSimulation** (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Static Public Member Functions inherited from [LSODE](#)

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * [_connections](#) = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification [_id](#)
- std::string [_typename](#)
- bool [_reportStatistics](#)
- bool [_hasChanged](#)
- unsigned int [_modelLevel](#) = 0
- [Model](#) * [_parentModel](#)
- [List](#)< [SimulationControl](#) * > * [_simulationControls](#) = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level [_traceLevelSpecific](#)
- bool [_traceLevelSpecificEnabled](#) = false

15.753.1 Member Function Documentation

15.753.1.1 [_onDispatchEvent\(\)](#)

```
void RecordingLSODE::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [inline], [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

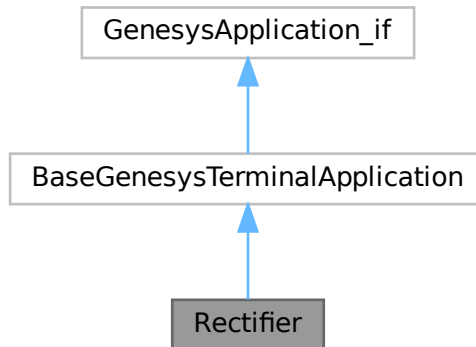
Reimplemented from [LSODE](#).

The documentation for this class was generated from the following file:

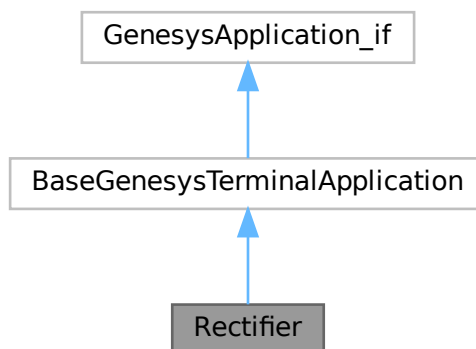
- tests/test_lsode.cpp

15.754 Rectifier Class Reference

Inheritance diagram for Rectifier:



Collaboration diagram for Rectifier:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

15.754.1 Member Function Documentation

15.754.1.1 main()

```
int Rectifier::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/teaching/Rectifier.h
- applications/modelSpecific/teaching/Rectifier.cpp

15.755 testing::internal::RelationToSourceCopy Struct Reference

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-internal.h

15.756 testing::internal::RelationToSourceReference Struct Reference

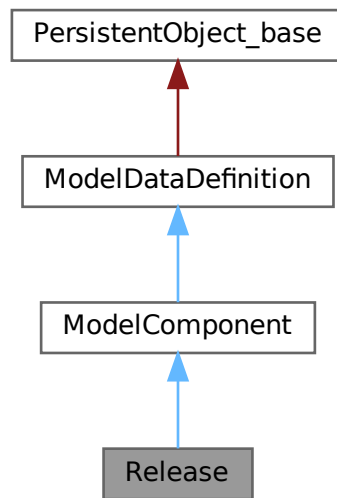
The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-internal.h

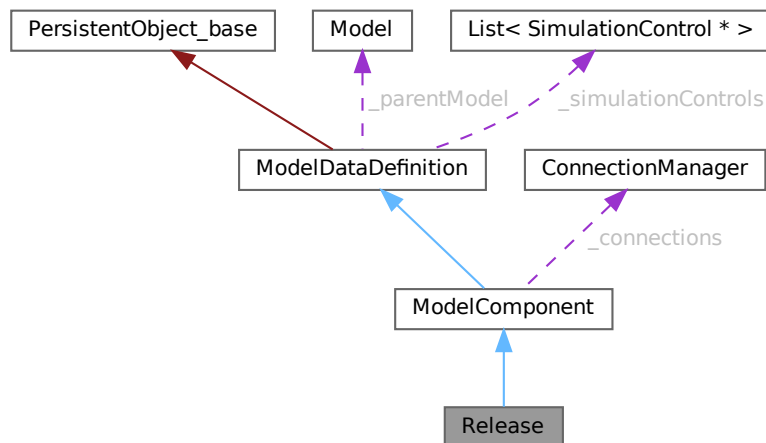
15.757 Release Class Reference

```
#include <plugins/components/DiscreteProcessing/Release.h>
```

Inheritance diagram for Release:



Collaboration diagram for Release:



Public Member Functions

- **Release** (`Model` *model, std::string name="")
- virtual std::string **show** () override
- *virtual public methods*
- void **setPriority** (unsigned short _priority)
- unsigned short **priority** () const
- List< `SeizableItem` * > * **getReleaseRequests** () const
- void **addReleaseRequests** (`SeizableItem` *newRequest)
- void **removeReleaseRequests** (`SeizableItem` *request)

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~[ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.

- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int [_modelLevel](#))
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) [traceLevelspecific](#), bool [traceLevelSpecificEnabled](#)=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool [traceLevelSpecificEnabled](#))
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)

Creates internal and attached data required by this instance.

- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)

Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_initBetweenReplications](#) () override
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()

- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void **traceSimulation** (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager::Level](#) **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.757.1 Detailed Description

[Release](#) module DESCRIPTION The [Release](#) module is used to release units of a resource that an entity previously has seized. This module may be used to release individual resources or may be used to release resources within a set. For each resource to be released, the name and quantity to release are specified. When the entity enters the [Release](#) module, it gives up control of the specified resource(s). Any entities waiting in queues for those resources will gain control of the resources immediately. TYPICAL USES Finishing a customer order (release the operator) Completing a tax return (release the accountant) Leaving the hospital (release the doctor, nurse, hospital room) PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Type Type of resource for releasing, either specifying a particular resource, or selecting from a pool of resources (that is, a resource set). The resource name may also be specified by an expression or attribute value. [Resource](#) Name Name of the resource that will be released. [Set](#) Name Name of the resource set from which a member will be released. [Attribute](#) Name Name of the attribute that specifies the resource name to be released. Expression Name of the expression that specifies the name of the resource to be released. Quantity Number of resources of a given name or from a given set that will be released. For sets, this value specifies only the number of a selected resource that will be released (based on the resource's capacity), not the number of members to be released within the set. [Release](#) Rule Method of determining which resource within a set to release. Last Member Seized and First Member Seized will release the last/first member from within the set that was seized. Specific member indicates that a member number or attribute (with a member number value) will be used to specify the member to release. [Set](#) Index Member index of the resource set that the entity will release.

15.757.2 Member Function Documentation

15.757.2.1 `_check()`

```
bool Release::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.757.2.2 `_createEditableDataDefinitions()`

```
void Release::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.757.2.3 `_initBetweenReplications()`

```
void Release::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.757.2.4 `_loadInstance()`

```
bool Release::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.757.2.5 `_onDispatchEvent()`

```
void Release::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.757.2.6 `_saveInstance()`

```
void Release::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.757.2.7 `show()`

```
std::string Release::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

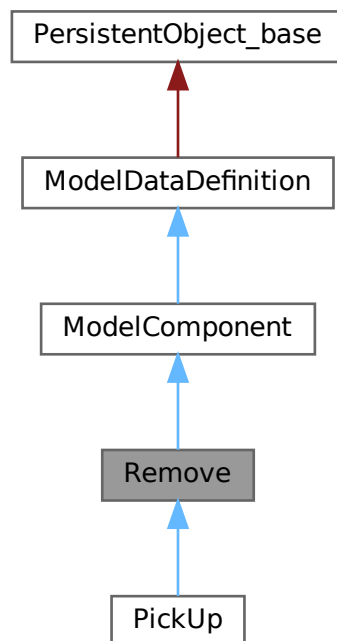
The documentation for this class was generated from the following files:

- `plugins/components/DiscreteProcessing/Release.h`
- `plugins/components/DiscreteProcessing/Release.cpp`

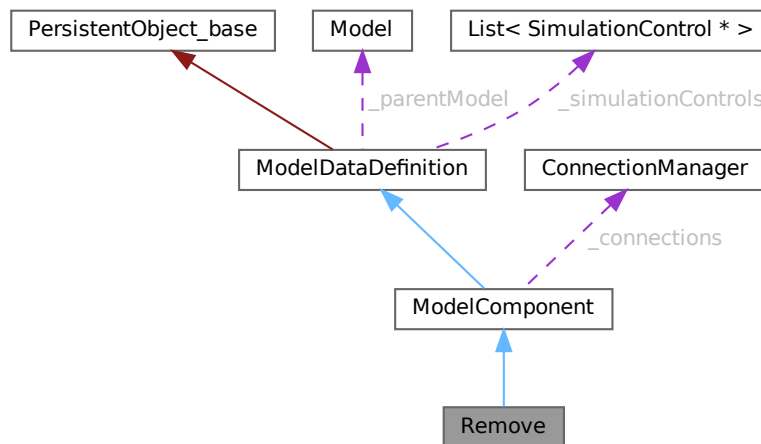
15.758 Remove Class Reference

```
#include <plugins/components/Decisions/Remove.h>
```

Inheritance diagram for Remove:



Collaboration diagram for Remove:



Public Types

- enum class **RemoveFromType** : int { QUEUE = 0 , ENTITYGROUP = 1 , num_elements = 2 }

Public Member Functions

- Remove** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
- virtual public methods*
- void **setRemoveStartRank** (std::string _removeFromRank)
- std::string **getRemoveStartRank** () const
- void **setRemoveEndRank** (std::string _removeEndRank)
- std::string **getRemoveEndRank** () const
- void **setRemoveFromType** (Remove::RemoveFromType _removeFromType)
- Remove::RemoveFromType **getRemoveFromType** () const
- void **setRemoveFrom** ([ModelDataDefinition](#) *_removeFrom)
- [ModelDataDefinition](#) * **getRemoveFrom** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
- constructors*
- virtual ~**ModelComponent** ()
- Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * **getConnectionManager** () const
- new public user methods for this component*
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
- Connects this component to another component.*
- bool **hasBreakpointAt** ()
- Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string _description)
- Updates the free-form description shown by the GUI.*
- std::string **getDescription** () const

Returns the free-form description associated with this component.

- void [remove](#) ([ModelComponent](#) *comp)

Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵IntoModel=true)

Creates a model data definition and optionally inserts it into the model.

- virtual ~[ModelDataDefinition](#) ()

Releases resources owned by this data definition.

- Util::identification [getId](#) () const

Returns the unique identifier of this instance.

- void [setName](#) (const std::string &name)

Updates the logical name of this instance.

- const std::string & [getName](#) () const

Returns the logical name of this instance.

- void [setLabel](#) (const std::string &label)

Updates the user-visible label of this instance.

- const std::string & [getLabel](#) () const

Returns the user-visible label of this instance.

- const std::string & [getClassname](#) () const

Returns the runtime class name used by the plugin system.

- [Model](#) * [getParentModel](#) () const

Returns the parent model that owns this definition.

- bool [isReportStatistics](#) () const

Indicates whether this data definition contributes report statistics.

- void [setReportStatistics](#) (bool reportStatistics)

Enables or disables report-statistics generation for this definition.

- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const

Looks up a named internal child data definition.

- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const

Returns the full map of internal child data definitions.

- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const

Returns the full map of attached data definitions.

- bool [hasChanged](#) () const

Indicates whether this instance has been modified since the last reset.

- void [setHasChanged](#) (bool [hasChanged](#))

Updates the changed flag for this data definition.

- unsigned int [getLevel](#) () const

Returns the level in the model hierarchy where this definition lives.

- void [setModelLevel](#) (unsigned int _modelLevel)

Sets the model nesting level for this definition.

- List< [SimulationControl](#) * > * [getSimulationControls](#) () const

Returns the simulation controls owned by this data definition.

- TraceManager::Level [getTraceLevelSpecific](#) () const

Returns the trace level that applies specifically to this definition.

- void [setTraceLevelSpecific](#) (TraceManager::Level level)

Updates the trace level used by this definition.

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵Enabled=true)

- Defines the specific trace level and optionally enables it immediately.*
 - bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
 - void `setTraceLevelSpecificEnabled` (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static std::string `convertEnumToStr` (RemoveFromType type)
- static `PluginInformation` * `GetPluginInformation` ()
- static `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check` (`ModelComponent` *component)
Validates a component instance.
- static void `CreateInternalData` (`ModelComponent` *component)
Creates internal data required by a component instance.
- static void `SaveInstance` (`PersistenceRecord` *fields, `ModelComponent` *component)
Serializes a component instance into a persistence record.
- static void `DispatchEvent` (`Event` *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void `SaveInstance` (`PersistenceRecord` *fields, `ModelDataDefinition` *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool `Check` (`ModelDataDefinition` *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void `InitBetweenReplications` (`ModelDataDefinition` *modeldatum)
Resets simulation-time state between replications.
- static void `CreateInternalData` (`ModelDataDefinition` *modeldatum)
Creates internal and attached data required by this instance.
- static bool `CreateRelatedDataElements` (`ModelDataDefinition` *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` (`PersistenceRecord` *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` (`PersistenceRecord` *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void `_onDispatchEvent` (`Entity` *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_createEditableDataDefinitions` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
 - legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

15.758.1 Detailed Description

[Remove](#) module DESCRIPTION The [Remove](#) module removes a single entity from a specified position in a queue and sends it to a designated module. When an entity arrives at a [Remove](#) module, it removes the entity from the specified queue and sends it to the connected module. The rank of the entity signifies the location of the entity within the queue. The entity that caused the removal proceeds to the next module specified and is processed before the removed entity. TYPICAL USES Removing an order from a queue that is due to be completed next Calling a patient from a waiting room for an examination Retrieving the next order to be processed from a pile of documents Prompt Description Name Unique module identifier displayed on the module shape. [Queue](#) Name Name of the queue from which the entity will be removed. Rank of [Entity](#) Rank of the entity to remove from within the queue.

15.758.2 Member Function Documentation

15.758.2.1 `_check()`

```
bool Remove::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.758.2.2 `_createEditableDataDefinitions()`

```
void Remove::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDatas. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dynamically linked to to simulator kernel to reflect the changes
Reimplemented from [ModelDataDefinition](#).

15.758.2.3 `_loadInstance()`

```
bool Remove::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [ModelComponent](#).

15.758.2.4 `_onDispatchEvent()`

```
void Remove::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.
This method is only for `ModelComponents`, not for `ModelDataElements`.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.758.2.5 `_saveInstance()`

```
void Remove::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.758.2.6 `show()`

```
std::string Remove::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

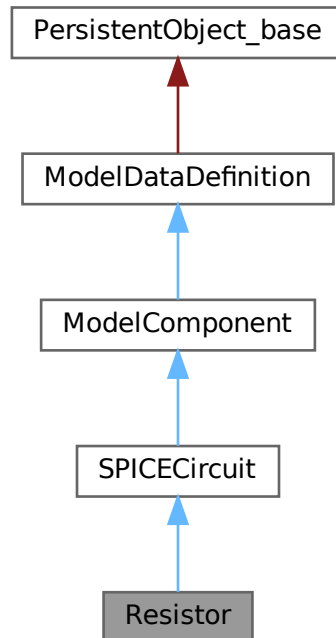
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

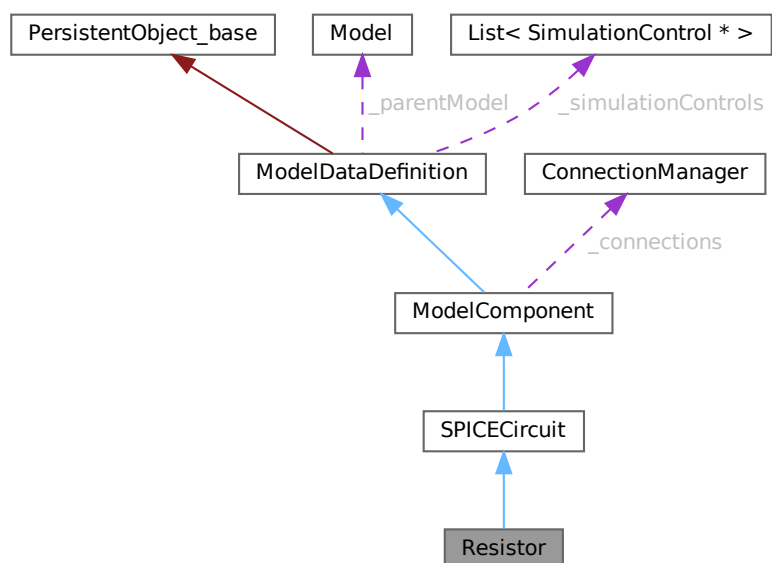
- `plugins/components/Decisions/Remove.h`
- `plugins/components/Decisions/Remove.cpp`

15.759 Resistor Class Reference

Inheritance diagram for Resistor:



Collaboration diagram for Resistor:



Public Member Functions

- **Resistor** ([Model](#) *model, std::string name="")
- void **Build** () override
- void **setResistance** (float resistance)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")

constructors

- void **setRunner** ([SPICERunner](#) *runner)

new public user methods for this components

- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- **ModelComponent** ([Model](#) *model, std::string componentName, std::string name="")

constructors

- virtual ~**ModelComponent** ()

Releases component-owned resources and unregisters it from the model.

- [ConnectionManager](#) * **getConnectionManager** () const

new public user methods for this component

- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)

Connects this component to another component.

- bool **hasBreakpointAt** ()

Indicates whether a simulation breakpoint is set on this component.

- void **setDescription** (std::string _description)

Updates the free-form description shown by the GUI.

- std::string **getDescription** () const

Returns the free-form description associated with this component.

- void **remove** ([ModelComponent](#) *comp)

Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- **ModelDataDefinition** ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)

Creates a model data definition and optionally inserts it into the model.

- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.

- Util::identification **getId** () const

Returns the unique identifier of this instance.

- void **setName** (const std::string &name)

Updates the logical name of this instance.

- const std::string & **getName** () const

Returns the logical name of this instance.

- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.

- const std::string & **getLabel** () const

Returns the user-visible label of this instance.

- const std::string & **getClassname** () const

- Returns the runtime class name used by the plugin system.*
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- **List**< **SimulationControl** * > * **getSimulationControls** () const
Returns the simulation controls owned by this data definition.
- **TraceManager::Level** **getTraceLevelSpecific** () const
Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (**TraceManager::Level** level)
Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (**TraceManager::Level** traceLevelSpecific, bool traceLevelSpecific←Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const
Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static **PluginInformation** * **GetPluginInformation** ()
- static **ModelComponent** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")

Static Public Member Functions inherited from **SPICECircuit**

- static **PluginInformation** * **GetPluginInformation** ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static **ModelComponent** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")
- static std::vector< std::string > **split** (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [SPICECircuit](#)

- void [BuildCircuit](#) (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void [UpdateConnections](#) ()
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void [_createNonEditableDataDefinitions](#) () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.759.1 Member Function Documentation

15.759.1.1 [_loadInstance\(\)](#)

```
bool Resistor::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [SPICECircuit](#).

15.759.1.2 [_saveInstance\(\)](#)

```
void Resistor::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

15.759.1.3 [Build\(\)](#)

```
void Resistor::Build ( ) [override], [virtual]
```

Reimplemented from [SPICECircuit](#).

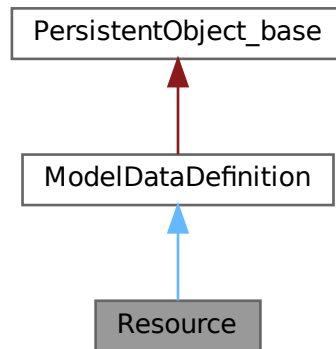
The documentation for this class was generated from the following files:

- plugins/components/ElectronicsSimulation/SPICECircuit.h
- plugins/components/ElectronicsSimulation/SPICECircuit.cpp

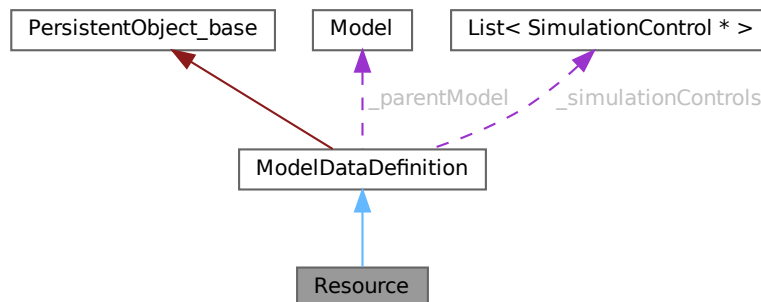
15.760 Resource Class Reference

```
#include <plugins/data/DiscreteProcessing/Resource.h>
```

Inheritance diagram for Resource:



Collaboration diagram for Resource:



Public Types

- enum class **ResourceState** : int {
IDLE = 0 , **BUSY** = 1 , **FAILED** = 2 , **INACTIVE** = 3 ,
OTHER = 4 , **num_elements** = 5 }
- typedef std::function< void([Resource](#) *) > **ResourceEventHandler**
- typedef std::pair< std::pair< ResourceEventHandler, [ModelComponent](#) * >, unsigned int > **SortedResourceEventHandler**

Public Member Functions

- [Resource](#) ([Model](#) *model, std::string name="")
- virtual std::string [show](#) () override
Returns a textual representation of this data definition.
- bool **seize** (unsigned int quantity, double priority=0)

- void **release** (unsigned int quantity)
- void **insertFailure** ([Failure](#) *failure)
- void **removeFailure** ([Failure](#) *failure)
- double **getInstantCapacityUtilization** () const
- double **getCapacityUtilization** () const
- double **getSeizedUtilization** () const
- double **getLastTimeSeized** () const
- void **addReleaseResourceEventHandler** (ResourceEventHandler eventHandler, [ModelComponent](#) *component, unsigned int priority)
- void **setResourceState** (ResourceState _resourceState)
- Resource::ResourceState **getResourceState** () const
- void **setCapacity** (unsigned int capacity)
- unsigned int **getCapacity** () const
- void **setCostBusyTimeUnit** (double _costBusyTimeUnit)
- double **getCostBusyTimeUnit** () const
- void **setCostIdleTimeUnit** (double _costIdleTimeUnit)
- double **getCostIdleTimeUnit** () const
- void **setCostPerUse** (double _costPerUse)
- double **getCostPerUse** () const
- void **setCapacitySchedule** ([Schedule](#) *_capacitySchedule)
- [Schedule](#) * **getCapacitySchedule** () const
- unsigned int **getNumberBusy** () const

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- [Model](#) * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * **getAttachedData** () const

- Returns the full map of attached data definitions.*
- bool **hasChanged** () const
 - Indicates whether this instance has been modified since the last reset.*
- void **setHasChanged** (bool **hasChanged**)
 - Updates the changed flag for this data definition.*
- unsigned int **getLevel** () const
 - Returns the level in the model hierarchy where this definition lives.*
- void **setModelLevel** (unsigned int **_modelLevel**)
 - Sets the model nesting level for this definition.*
- List< **SimulationControl** * > * **getSimulationControls** () const
 - Returns the simulation controls owned by this data definition.*
- TraceManager::Level **getTraceLevelSpecific** () const
 - Returns the trace level that applies specifically to this definition.*
- void **setTraceLevelSpecific** (TraceManager::Level level)
 - Updates the trace level used by this definition.*
- void **defineTraceLevelSpecific** (TraceManager::Level **traceLevelSpecific**, bool **traceLevelSpecificEnabled**=true)
 - Defines the specific trace level and optionally enables it immediately.*
- bool **isTraceLevelSpecificEnabled** () const
 - Indicates whether the specific trace level is enabled.*
- void **setTraceLevelSpecificEnabled** (bool **traceLevelSpecificEnabled**)
 - Enables or disables the specific trace level.*

Static Public Member Functions

- template<typename Class >
 static ResourceEventHandler **SetResourceEventHandler** (void(Class::*function)(Resource *), Class *object)
- static std::string **convertEnumToStr** (ResourceState state)
- static **PluginInformation** * **GetPluginInformation** ()
- static **ModelDataDefinition** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")

Static Public Member Functions inherited from **ModelDataDefinition**

- static **ModelDataDefinition** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields, bool insertIntoModel)
 - Loads a model data definition instance from serialized fields.*
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")
 - Creates a new instance of a data definition using the plugin registry.*
- static void **SaveInstance** (**PersistenceRecord** *fields, **ModelDataDefinition** *modeldatum)
 - Serializes a data definition instance into a persistence record.*
- static bool **Check** (**ModelDataDefinition** *modeldatum, std::string &errorMessage)
 - Validates a data definition instance.*
- static void **InitBetweenReplications** (**ModelDataDefinition** *modeldatum)
 - Resets simulation-time state between replications.*
- static void **CreateInternalData** (**ModelDataDefinition** *modeldatum)
 - Creates internal and attached data required by this instance.*
- static bool **CreateRelatedDataElements** (**ModelDataDefinition** *modeldatum, std::string &errorMessage)
 - Creates related data and validates the instance in a single call.*

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_initBetweenReplications` () override
- virtual void `_createAttachedAttributes` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void `trace` (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))

not just an easy access to trace manager, but wrappers to check if specific trace level applies

- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

Traces text to the simulation output.

- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Friends

- class **Failure**
- class **ResourceTestProbe**

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.760.1 Detailed Description

[Resource](#) module DESCRIPTION This data module defines the resources in the simulation system, including costing information and resource availability. Resources may have a fixed capacity that does not vary over the simulation run or may operate based on a schedule. [Resource](#) failures and states can also be specified in this module. TYPICAL USES Equipment (machinery, cash register, phone line) People (clerical, order processing, sales clerks, operators) PROMPTS Prompt Description Name The name of the resource whose characteristics are being defined. This name must be unique. Type Method for determining the capacity for a resource. Fixed Capacity will not change during the simulation run. Based on [Schedule](#) signifies that a [Schedule](#) module is used to specify the capacity and duration information for the resource. Capacity Number of resource units of a given name that are available to the system for processing. Applies only when Type is Fixed Capacity. [Schedule](#) Name Identifies the name of the schedule to be used by the resource. The schedule defines the capacity of a resource for a given period of time. Applies only when type is [Schedule](#). [Schedule](#) Rule Dictates when the actual capacity change is to occur when a decrease in capacity is required for a busy resource unit. Applies only when Type is [Schedule](#). Busy/TimeUnit Cost per timeUnit of a resource that is processing an entity. The resource becomes busy when it is originally allocated to an entity and becomes idle when it is released. During the time when it is busy, cost will accumulate based on the busy/timeUnit cost. The busy cost per timeUnit is automatically converted to the appropriate base time unit specified within the Replication Parameters page of the Run > Setup menu item. Idle/TimeUnit Cost per timeUnit of a resource that is idle. The resource is idle while it is not processing an entity. During the time when it is idle, cost will accumulate based on the idle/timeUnit cost. The idle cost per timeUnit is automatically converted to the appropriate base time unit specified within the Replication Parameters page of the Run > Setup menu item. Per Use Cost of a resource on a usage basis, regardless of the time for which it is used. Each time the resource is allocated to an entity, it will incur a per-use cost. [StateSet](#) Name Name of states that the resource may be assigned during the simulation run. Initial [State](#) Initial state of a resource. If specified, the name must be defined within the repeat group of state names. This field is shown only when a [StateSet](#) Name is defined. Failures Lists all failures that will be associated with the resource. [Failure](#) Name–Name of the failure associated with the resource. [Failure](#) Rule–Behavior that should occur when a failure is to occur for a busy resource unit. Report Statistics Specifies whether or not statistics will be collected automatically and stored in the report database for this resource.

15.760.2 Constructor & Destructor Documentation

15.760.2.1 Resource()

```
Resource::Resource (
    Model * model,
    std::string name = "" )
```

Handlers are sorted by priority

15.760.3 Member Function Documentation

15.760.3.1 _check()

```
bool Resource::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Validate resource basic semantic constraints.

Reimplemented from [ModelDataDefinition](#).

15.760.3.2 _createAttachedAttributes()

```
void Resource::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.760.3.3 _initBetweenReplications()

```
void Resource::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.760.3.4 _loadInstance()

```
bool Resource::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.760.3.5 _saveInstance()

```
void Resource::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.760.3.6 show()

```
std::string Resource::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

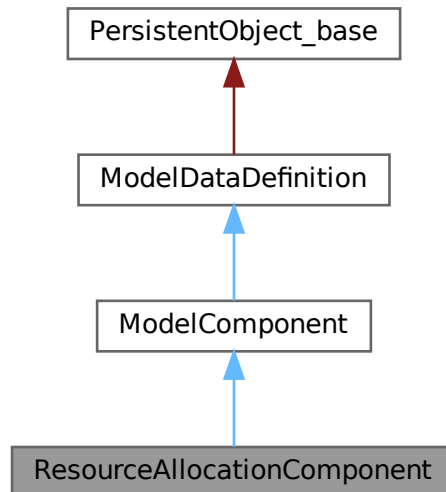
The documentation for this class was generated from the following files:

- `plugins/data/DiscreteProcessing/Resource.h`
- `plugins/data/DiscreteProcessing/Resource.cpp`

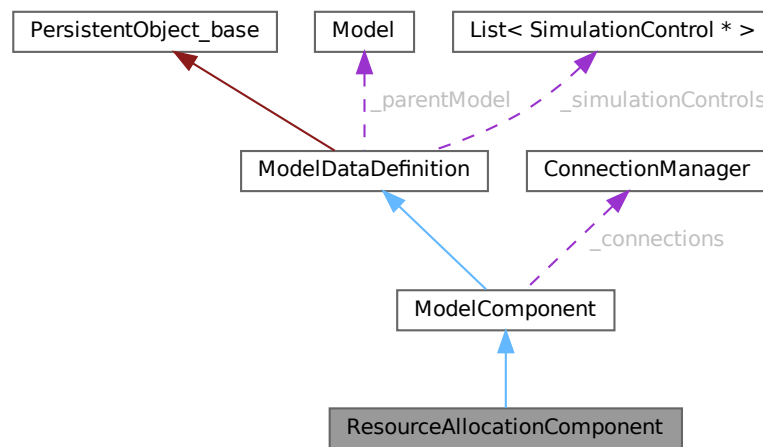
15.761 ResourceAllocationComponent Class Reference

```
#include <plugins/components/WholeCellModeling/ResourceAllocationComponent.h>
```

Inheritance diagram for ResourceAllocationComponent:



Collaboration diagram for ResourceAllocationComponent:



Public Member Functions

- **ResourceAllocationComponent** ([Model](#) *model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **setRnapCountKey** (std::string key)
- std::string **getRnapCountKey** () const
- void **setRibosomeCountKey** (std::string key)
- std::string **getRibosomeCountKey** () const
- void **setMRNAspeciesPrefix** (std::string prefix)
- std::string **getMRNAspeciesPrefix** () const
- void **setProteinSpeciesPrefix** (std::string prefix)
- std::string **getProteinSpeciesPrefix** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool **hasChanged**)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- **List**< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool _getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * _getParserChangesInformation ()`
- `virtual void _addSimulationControl (SimulationControl *control)`
- `virtual void _createInternalAndAttachedData ()`
- `virtual void _createInternalStatisticReporters ()`
- `virtual void _createAttachedAttributes ()`
- `virtual void _createNonEditableDataDefinitions ()`
- `void _templateCreateInternalStatisticReporters ()`
- `void _templateCreateAttachedAttributes ()`
- `void _templateCreateNonEditableDataDefinitions ()`
- `void _templateCreateEditableDataDefinitions ()`
- `void _internalStatisticReportersClear ()`
- *legacy helper methods used by the granular create* hooks*
- `void _internalDataClear ()`
- `void _internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void _internalDataRemove (const std::string &key)`
- `void _attachedDataClear ()`
- `void _attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void _attachedDataRemove (const std::string &key)`
- `void _attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void _statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void _statisticReporterRemove (const std::string &key)`
- `void _statisticReportersClear ()`
- `void _mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryAttachedAttributeRemove (const std::string &key)`
- `void _mandatoryAttachedAttributesClear ()`
- `void _mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryEditableDataDefinitionsClear ()`
- `void _optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalEditableDataDefinitionsClear ()`
- `void _mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryNonEditableDataDefinitionsClear ()`
- `void _optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalNonEditableDataDefinitionsClear ()`
- `void _checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
- *method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- *not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- *Traces text to the simulation output.*
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

15.761.1 Detailed Description

Fair allocation of molecular machines (RNAP, ribosomes) before each SSA step.

Implements the "ResourceAllocation" contract from Karr et al. (2012): available RNAP and ribosome counts are divided proportionally across active molecular species, then written to `WholeCellState::setResourceBudget()` so [StochasticTranscription](#) and [StochasticTranslation](#) can honour the budget.

Allocation logic:

- Count `N_mRNA` = number of keys in [WholeCellState](#) molecule counts whose name starts with `mRNA` ← `RNASpeciesPrefix` (default "mRNA_").
- Count `N_protein` = number of keys starting with `proteinSpeciesPrefix` (default "protein_").
- RNAP budget per mRNA gene = `floor(RNAP_free / max(N_mRNA, 1))`
- Ribosome budget per protein = `floor(ribosome_free / max(N_mRNA, 1))` (per active mRNA transcript, not per protein species)
- [Write](#) per-species budget to `WholeCellState::setResourceBudget()`.

This component must fire BEFORE [StochasticTranscription](#) and [StochasticTranslation](#) in the simulation pipeline.

15.761.2 Member Function Documentation

15.761.2.1 `_check()`

```
bool ResourceAllocationComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.761.2.2 `_createEditableDataDefinitions()`

```
void ResourceAllocationComponent::_createEditableDataDefinitions ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.761.2.3 `_initBetweenReplications()`

```
void ResourceAllocationComponent::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.761.2.4 _loadInstance()

```
bool ResourceAllocationComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.761.2.5 _onDispatchEvent()

```
void ResourceAllocationComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.761.2.6 _saveInstance()

```
void ResourceAllocationComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.761.2.7 show()

```
std::string ResourceAllocationComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/WholeCellModeling/ResourceAllocationComponent.h
- plugins/components/WholeCellModeling/ResourceAllocationComponent.cpp

15.762 ResourceTestProbe Class Reference**Static Public Member Functions**

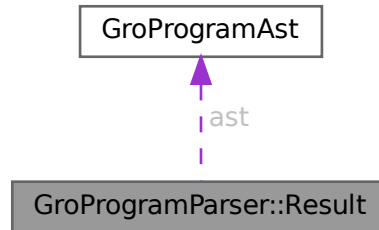
- static bool **HasStatisticsInternals** (const [Resource](#) &resource)
- static bool **HasFailure** (const [Resource](#) &resource, const [Failure](#) *failure)
- static size_t **FailureCount** (const [Resource](#) &resource)

The documentation for this class was generated from the following file:

- tests/unit/test_simulator_runtime.cpp

15.763 GroProgramParser::Result Struct Reference

Collaboration diagram for GroProgramParser::Result:



Public Attributes

- bool **accepted** = false
- std::string **errorMessage** = ""
- [GroProgramAst](#) **ast**

The documentation for this struct was generated from the following file:

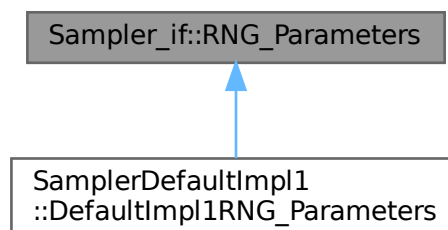
- plugins/data/BiochemicalSimulation/GroProgramParser.h

15.764 Sampler_if::RNG_Parameters Struct Reference

Encapsulates generator-specific configuration/state parameters.

```
#include <kernel/statistics/Sampler_if.h>
```

Inheritance diagram for Sampler_if::RNG_Parameters:



15.764.1 Detailed Description

Encapsulates generator-specific configuration/state parameters.

Implementations may derive this struct to expose seed, multiplier and other algorithm-specific controls required by a concrete RNG strategy.

The documentation for this struct was generated from the following file:

- kernel/statistics/Sampler_if.h

15.765 RootFinder_if Class Reference

Root-finding abstraction for scalar equations.

```
#include <tools/Continuous/RootFinder_if.h>
```

Public Member Functions

- virtual double **findRoot** (double min, double max, double(*f)(double), double tolerance) const =0

15.765.1 Detailed Description

Root-finding abstraction for scalar equations.

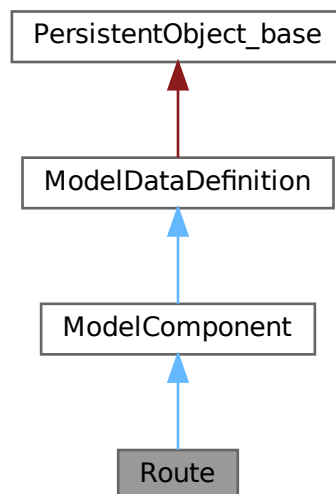
The documentation for this class was generated from the following file:

- tools/Continuous/RootFinder_if.h

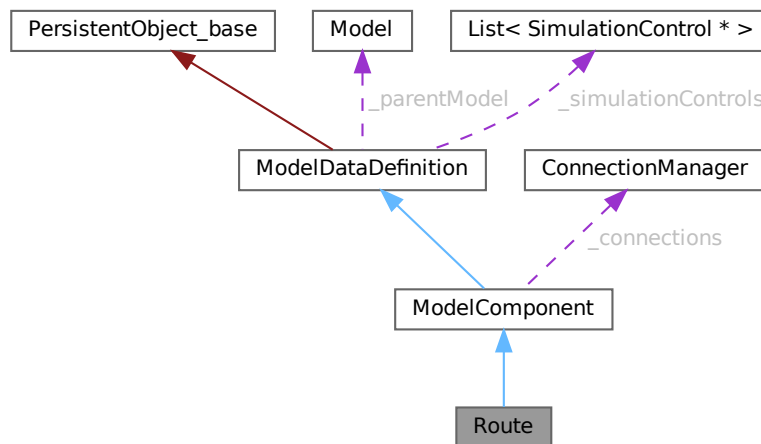
15.766 Route Class Reference

```
#include <plugins/components/MaterialHandling/Route.h>
```

Inheritance diagram for Route:



Collaboration diagram for Route:



Public Types

- enum class **DestinationType** : int { **Station** = 0 , **Sequence** = 1 , **Label** = 2 , **num_elements** = 3 }

Public Member Functions

- **Route** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
- *virtual public methods*
- void **setStationName** (std::string stationName)
- void **setStation** ([Station](#) *_station)
- [Station](#) * **getStation** () const
- void **setStationExpression** (std::string _stationExpression)
- std::string **getStationExpression** () const
- void **setRouteTimeExpression** (std::string _routeTimeExpression)
- void **setRouteTimeExpression** (std::string _routeTimeExpression, [Util::TimeUnit](#) _routeTimeTimeUnit)
- std::string **getRouteTimeExpression** () const
- void **setRouteTimeTimeUnit** ([Util::TimeUnit](#) _routeTimeTimeUnit)
- [Util::TimeUnit](#) **getRouteTimeTimeUnit** () const
- void **setRouteDestinationType** (DestinationType _routeDestinationType)
- Route::DestinationType **getRouteDestinationType** () const
- void **setLabel** ([Label](#) *_label)
- [Label](#) * **getLabel** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
- *constructors*
- virtual ~**ModelComponent** ()
- *Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * **getConnectionManager** () const
- *new public user methods for this component*
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)

- *Connects this component to another component.*
- bool `hasBreakpointAt ()`
Indicates whether a simulation breakpoint is set on this component.
- void `setDescription (std::string _description)`
Updates the free-form description shown by the GUI.
- std::string `getDescription () const`
Returns the free-form description associated with this component.
- void `remove (ModelComponent *comp)`
Removes a connected component from this component.

Public Member Functions inherited from `ModelDataDefinition`

- `ModelDataDefinition (Model *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)`
Creates a model data definition and optionally inserts it into the model.
- virtual `~ModelDataDefinition ()`
Releases resources owned by this data definition.
- Util::identification `getId () const`
Returns the unique identifier of this instance.
- void `setName (const std::string &name)`
Updates the logical name of this instance.
- const std::string & `getName () const`
Returns the logical name of this instance.
- void `setLabel (const std::string &label)`
Updates the user-visible label of this instance.
- const std::string & `getLabel () const`
Returns the user-visible label of this instance.
- const std::string & `getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- bool `isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- void `setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- std::map< std::string, `ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- std::map< std::string, `ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- bool `hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- unsigned int `getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.

- `TraceManager::Level` `getTraceLevelSpecific ()` const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` level)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` traceLevelspecific, bool traceLevelSpecific↔ Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static std::string `convertEnumToStr` (`DestinationType` type)
- static `PluginInformation` * `GetPluginInformation` ()
- static `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check` (`ModelComponent` *component)
Validates a component instance.
- static void `CreateInternalData` (`ModelComponent` *component)
Creates internal data required by a component instance.
- static void `SaveInstance` (`PersistenceRecord` *fields, `ModelComponent` *component)
Serializes a component instance into a persistence record.
- static void `DispatchEvent` (`Event` *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void `SaveInstance` (`PersistenceRecord` *fields, `ModelDataDefinition` *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool `Check` (`ModelDataDefinition` *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void `InitBetweenReplications` (`ModelDataDefinition` *modeldatum)
Resets simulation-time state between replications.
- static void `CreateInternalData` (`ModelDataDefinition` *modeldatum)
Creates internal and attached data required by this instance.
- static bool `CreateRelatedDataElements` (`ModelDataDefinition` *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void `_onDispatchEvent` (`Entity` *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool `_loadInstance` (`PersistenceRecord` *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` (`PersistenceRecord` *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_createInternalStatisticReporters` () override
- virtual void `_createEditableDataDefinitions` () override
- virtual void `_createAttachedAttributes` () override

Protected Member Functions inherited from `ModelDataDefinition`

- bool `_getSaveDefaultsOption` ()
- virtual `ParserChangesInformation` * `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` (`SimulationControl` *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createNonEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, `ModelDataDefinition` *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * _connections = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification _id
- std::string _typename
- bool _reportStatistics
- bool _hasChanged
- unsigned int _modelLevel = 0
- [Model](#) * _parentModel
- [List](#)< [SimulationControl](#) * > * _simulationControls = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level _traceLevelSpecific
- bool _traceLevelSpecificEnabled = false

15.766.1 Detailed Description

[Route](#) module DESCRIPTION The [Route](#) module transfers an entity to a specified station or the next station in the station visitation sequence defined for the entity. A delay time to transfer to the next station may be defined. When an entity enters the [Route](#) module, its [Station](#) attribute ([Entity](#).Station) is set to the destination station. The entity is then sent to the destination station, using the route time specified. If the station destination is entered as By [Sequence](#), the next station is determined by the entity's sequence and step within the set (defined by special-purpose attributes [Entity](#).Sequence and [Entity](#).Jobstep, respectively). TYPICAL USES Send a part to its next processing station based on its routing slip Send an account balance call to an account agent Send restaurant customers to a specific table PROMPTS Prompt Description Name Unique name of the module that will be displayed in the flowchart. [Route](#) Time Travel time from the entity's current location to the destination station. Units Time units for route-time parameters. Destination Type Method for determining the entity destination location. Selection of By [Sequence](#) requires that the entity has been assigned a sequence name and that the sequence itself has been defined. [Station](#) Name Name of the individual destination station. [Attribute](#) Name Name of the attribute that stores the station name to which entities will route. Expression Expression that is evaluated to the station name where entities will route.

15.766.2 Member Function Documentation

15.766.2.1 _check()

```
bool Route::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.766.2.2 _createAttachedAttributes()

void Route::_createAttachedAttributes () [override], [protected], [virtual]
 Reimplemented from [ModelDataDefinition](#).

15.766.2.3 _createEditableDataDefinitions()

void Route::_createEditableDataDefinitions () [override], [protected], [virtual]
 Reimplemented from [ModelDataDefinition](#).

15.766.2.4 _createInternalStatisticReporters()

void Route::_createInternalStatisticReporters () [override], [protected], [virtual]
 Reimplemented from [ModelDataDefinition](#).

15.766.2.5 _loadInstance()

bool Route::_loadInstance (
 [PersistenceRecord](#) * fields) [override], [protected], [virtual]
 virtual protected method that must be overridden
 Reimplemented from [ModelComponent](#).

15.766.2.6 _onDispatchEvent()

void Route::_onDispatchEvent (
 [Entity](#) * entity,
 unsigned int inputPortNumber) [override], [protected], [virtual]

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.766.2.7 _saveInstance()

void Route::_saveInstance (
 [PersistenceRecord](#) * fields,
 bool saveDefaultValues) [override], [protected], [virtual]

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.766.2.8 show()

std::string Route::show () [override], [virtual]
 virtual public methods
 Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

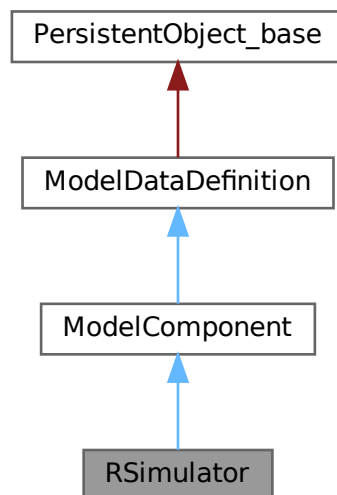
- `plugins/components/MaterialHandling/Route.h`
- `plugins/components/MaterialHandling/Route.cpp`

15.767 RSimulator Class Reference

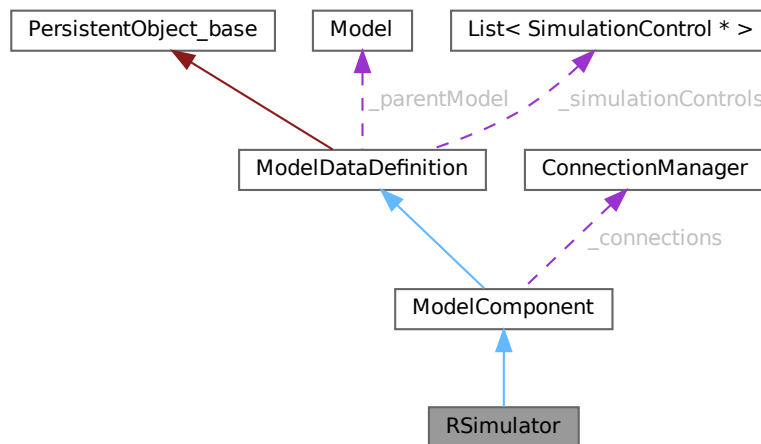
Component that sends configured R commands to an internal [RSimulatorRunner](#).

```
#include <plugins/components/ExternalIntegration/RSimulator.h>
```

Inheritance diagram for RSimulator:



Collaboration diagram for RSimulator:



Public Member Functions

- **RSimulator** (**Model** *model, std::string name="")
Creates an **RSimulator** component attached to a model.
- virtual ~**RSimulator** ()=default
Destroys the **RSimulator** component.
- void **setRExecutable** (std::string rExecutable)
Sets the Rscript executable name or path used by the internal runner.
- std::string **getRExecutable** () const
Returns the Rscript executable name or path used by the internal runner.
- void **setWorkingDirectory** (std::string workingDirectory)
Sets the working directory used by generated R scripts.
- std::string **getWorkingDirectory** () const
Returns the working directory used by generated R scripts.
- void **setPreludeScript** (std::string preludeScript)
Sets optional R code prepended to every command.
- std::string **getPreludeScript** () const
Returns optional R code prepended to every command.
- void **insertCommand** (std::string command)
Appends one R command to the execution list.
- void **setCommands** (std::list< std::string > commands)
Replaces the command list with the provided commands.
- **List**< std::string > * **getCommands** () const
Returns the command list used when entities arrive.
- void **setCommandList** (std::string commandList)
Replaces the command list using one command per non-empty line.
- std::string **getCommandList** () const
Returns commands serialized as one command per line for GUI editing.
- **RSimulatorRunner** * **getRunner** ()
Returns the internal runner, creating it first if necessary.
- virtual std::string **show** () override
Returns a compact human-readable description of the component.

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~[ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.

- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int [_modelLevel](#))
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) [traceLevelspecific](#), bool [traceLevelSpecificEnabled](#)=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool [traceLevelSpecificEnabled](#))
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
Returns plugin metadata for the [RSimulator](#) component.
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
Loads an [RSimulator](#) component from persistence.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new [RSimulator](#) component instance.

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)

Validates a data definition instance.

- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Executes R commands for the arriving entity and forwards it.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
Loads persisted RSimulator configuration.
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves RSimulator configuration.
- virtual bool [_check](#) (std::string &errorMessage) override
Validates the executable, working directory and command list.
- virtual void [_createEditableDataDefinitions](#) () override
Creates or updates the internal RSimulatorRunner.

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)

- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager::Level](#) **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.767.1 Detailed Description

Component that sends configured R commands to an internal [RSimulatorRunner](#).

Each arriving entity triggers the configured command list. Commands are executed sequentially by the internal [RSimulatorRunner](#), and the first version exposes results through simulation trace messages before forwarding the entity.

15.767.2 Member Function Documentation

15.767.2.1 **_check()**

```
bool RSimulator::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

Validates the executable, working directory and command list.

Reimplemented from [ModelDataDefinition](#).

15.767.2.2 _createEditableDataDefinitions()

```
void RSimulator::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Creates or updates the internal [RSimulatorRunner](#).

Reimplemented from [ModelDataDefinition](#).

15.767.2.3 _loadInstance()

```
bool RSimulator::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

Loads persisted [RSimulator](#) configuration.

Reimplemented from [ModelComponent](#).

15.767.2.4 _onDispatchEvent()

```
void RSimulator::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Executes R commands for the arriving entity and forwards it.

Implements [ModelComponent](#).

15.767.2.5 _saveInstance()

```
void RSimulator::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves [RSimulator](#) configuration.

Reimplemented from [ModelComponent](#).

15.767.2.6 show()

```
std::string RSimulator::show ( ) [override], [virtual]
```

Returns a compact human-readable description of the component.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

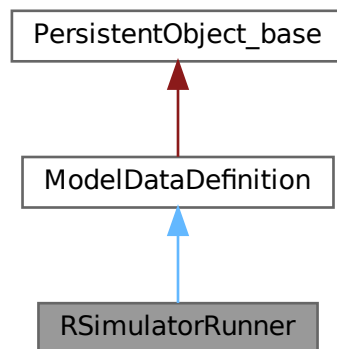
- plugins/components/ExternalIntegration/RSimulator.h
- plugins/components/ExternalIntegration/RSimulator.cpp

15.768 RSimulatorRunner Class Reference

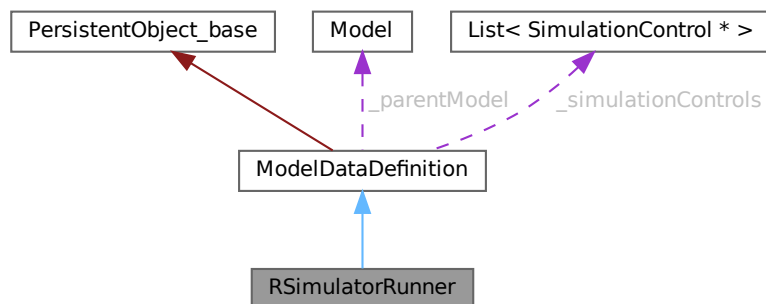
[Model](#) data definition that executes R code through Rscript.

```
#include <plugins/data/ExternalIntegration/RSimulatorRunner.h>
```

Inheritance diagram for RSimulatorRunner:



Collaboration diagram for RSimulatorRunner:



Public Member Functions

- **RSimulatorRunner** ([Model](#) *model, std::string name="")
Creates an R runner bound to a model.
- virtual ~**RSimulatorRunner** ()=default
Destroys the R runner.
- void **setRExecutable** (std::string rExecutable)
Sets the Rscript executable name or path.
- std::string **getRExecutable** () const
Returns the Rscript executable name or path.
- void **setWorkingDirectory** (std::string workingDirectory)
Sets the working directory used for generated scripts and command execution.
- std::string **getWorkingDirectory** () const
Returns the working directory used for generated scripts and command execution.
- void **setPreludeScript** (std::string preludeScript)
Sets optional R code prepended before the command block.

- `std::string getPreludeScript ()` const
Returns optional R code prepended before the command block.
- `void setCommand (std::string command)`
Sets the R command/script body to execute.
- `std::string getCommand ()` const
Returns the R command/script body to execute.
- `void setLastStatus (std::string lastStatus)`
Sets the last execution status.
- `std::string getLastStatus ()` const
Returns the last execution status.
- `void setLastExitCode (int lastExitCode)`
Sets the last process exit code.
- `int getLastExitCode ()` const
Returns the last process exit code.
- `void setLastStdout (std::string lastStdout)`
Sets stdout captured from the last R execution.
- `std::string getLastStdout ()` const
Returns stdout captured from the last R execution.
- `void setLastStderr (std::string lastStderr)`
Sets stderr captured from the last R execution.
- `std::string getLastStderr ()` const
Returns stderr captured from the last R execution.
- `void setLastResponsePayload (std::string lastResponsePayload)`
Sets the response payload exposed to other tools.
- `std::string getLastResponsePayload ()` const
Returns the response payload exposed to other tools.
- `void setLastScriptFilename (std::string lastScriptFilename)`
Sets the generated R script filename from the last execution.
- `std::string getLastScriptFilename ()` const
Returns the generated R script filename from the last execution.
- `void setLastResponseFilename (std::string lastResponseFilename)`
Sets the stdout file path from the last execution.
- `std::string getLastResponseFilename ()` const
Returns the stdout file path from the last execution.
- `bool executeCommand (std::string &errorMessage)`
Executes the configured R code and updates last-execution fields atomically.
- `virtual std::string show ()` override
Returns a compact human-readable description of the runner.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
- *Creates a model data definition and optionally inserts it into the model.*
- `virtual ~ModelDataDefinition ()`
Releases resources owned by this data definition.
- `Util::identification getId ()` const
Returns the unique identifier of this instance.
- `void setName (const std::string &name)`
Updates the logical name of this instance.
- `const std::string & getName ()` const

- Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecific←
Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
Loads an R runner from a persistence record.
- static [PluginInformation](#) * [GetPluginInformation](#) ()
Returns plugin metadata and system dependency declarations.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new R runner instance for plugin construction.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
Loads persisted R runner configuration and last-execution state.
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves R runner configuration and last-execution state.
- virtual bool [_check](#) (std::string &errorMessage) override
Validates the runner configuration without executing R.

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)

- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.768.1 Detailed Description

[Model](#) data definition that executes R code through Rscript.

The runner stores a compact R execution configuration, writes a temporary script, executes it with Rscript, and keeps status, exit code, stdout, stderr and response payload inside the model data definition for later inspection.

15.768.2 Member Function Documentation

15.768.2.1 **_check()**

```
bool RSimulatorRunner::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

Validates the runner configuration without executing R.

Reimplemented from [ModelDataDefinition](#).

15.768.2.2 `_loadInstance()`

```
bool RSimulatorRunner::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

Loads persisted R runner configuration and last-execution state.

Reimplemented from [ModelDataDefinition](#).

15.768.2.3 `_saveInstance()`

```
void RSimulatorRunner::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves R runner configuration and last-execution state.

Reimplemented from [ModelDataDefinition](#).

15.768.2.4 `show()`

```
std::string RSimulatorRunner::show ( ) [override], [virtual]
```

Returns a compact human-readable description of the runner.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

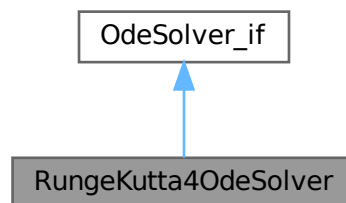
- `plugins/data/ExternalIntegration/RSimulatorRunner.h`
- `plugins/data/ExternalIntegration/RSimulatorRunner.cpp`

15.769 RungeKutta4OdeSolver Class Reference

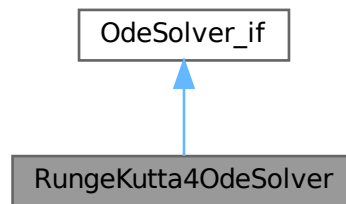
Fixed-step fourth-order Runge-Kutta solver for first-order ODE systems.

```
#include <tools/Continuous/RungeKutta4OdeSolver.h>
```

Inheritance diagram for RungeKutta4OdeSolver:



Collaboration diagram for RungeKutta4OdeSolver:



Public Member Functions

- bool [advance](#) (const [OdeSystem_if](#) &system, double t0, double dt, const double *y0, double *y1) const override

15.769.1 Detailed Description

Fixed-step fourth-order Runge-Kutta solver for first-order ODE systems.

15.769.2 Member Function Documentation

15.769.2.1 advance()

```
bool RungeKutta4OdeSolver::advance (
    const OdeSystem\_if & system,
    double t0,
    double dt,
    const double * y0,
    double * y1 ) const [inline], [override], [virtual]
```

Implements [OdeSolver_if](#).

The documentation for this class was generated from the following file:

- tools/Continuous/RungeKutta4OdeSolver.h

15.770 Sample Struct Reference

Public Attributes

- double **t**
- double **x**
- double **v**
- double **energy**

The documentation for this struct was generated from the following files:

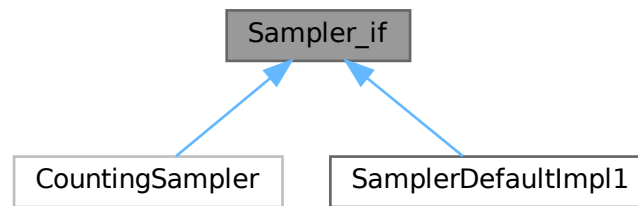
- tests/demo_continuous_trace.cpp
- tests/test_continuous_system.cpp

15.771 Sampler_if Class Reference

Abstraction for random-number generation and probability-distribution sampling.

```
#include <kernel/statistics/Sampler_if.h>
```

Inheritance diagram for Sampler_if:



Classes

- struct [RNG_Parameters](#)
Encapsulates generator-specific configuration/state parameters.

Public Member Functions

- virtual double [random](#) ()=0
Generates a pseudo-random value uniformly distributed in the range $[0, 1)$.
- virtual double [sampleBeta](#) (double alpha, double beta, double infLimit, double supLimit)=0
Samples a Beta-distributed value scaled to an arbitrary output interval.
- virtual double [sampleBeta](#) (double alpha, double beta)=0
Samples a standard Beta-distributed value in $[0, 1]$.
- virtual double [sampleErlang](#) (double mean, int M, double offset=0.0)=0
Samples from an Erlang distribution using the provided mean and number of phases.
- virtual double [sampleExponential](#) (double mean, double offset=0.0)=0
Samples from an Exponential distribution with the provided mean.
- virtual double [sampleGamma](#) (double alpha, double beta, double offset=0.0)=0
Samples from a Gamma distribution using the provided shape/scale parameters.
- virtual double [sampleGumbell](#) (double mode, double scale)=0
Samples from a Gumbel distribution using mode and scale.
- virtual double [sampleLogNormal](#) (double mean, double stddev, double offset=0.0)=0
Samples from a Lognormal distribution parameterized by mean and standard deviation.
- virtual double [sampleNormal](#) (double mean, double stddev)=0
Samples from a Normal distribution.
- virtual double [sampleTriangular](#) (double min, double mode, double max)=0
Samples from a Triangular distribution.
- virtual double [sampleUniform](#) (double min, double max)=0
Samples from a continuous Uniform distribution in $[min, max]$.
- virtual double [sampleWeibull](#) (double alpha, double scale)=0
Samples from a Weibull distribution.
- virtual double [sampleBinomial](#) (int trials, double p)=0
*Samples from a Binomial distribution with *trials* and success probability *p*.*
- virtual double [sampleBernoulli](#) (double p)=0
Samples from a Bernoulli distribution (0/1).
- virtual double [sampleDiscrete](#) (double prob, double value,...)=0
Samples from a discrete distribution using a variadic list of probability/value pairs.

- virtual double [sampleDiscrete](#) (double *prob, double *value, int size)=0
Samples from a discrete distribution using probability/value arrays.
- virtual double [sampleGeometric](#) (double p)=0
Samples from a Geometric distribution with success probability p .
- virtual void [setRNGparameters](#) ([RNG_Parameters](#) *param)=0
Updates the pseudo-random generator internal parameters.
- virtual [RNG_Parameters](#) * [getRNGparameters](#) () const =0
Returns the current pseudo-random generator parameters.

15.771.1 Detailed Description

Abstraction for random-number generation and probability-distribution sampling.

Concrete implementations provide a base uniform pseudo-random generator and helper methods to sample supported continuous and discrete probability distributions used across the simulation kernel (e.g., parser functions and stochastic model logic).

15.771.2 Member Function Documentation

15.771.2.1 [getRNGparameters\(\)](#)

```
virtual RNG\_Parameters * Sampler\_if::getRNGparameters ( ) const [pure virtual]
```

Returns the current pseudo-random generator parameters.

Implemented in [SamplerDefaultImpl1](#), and [CountingSampler](#).

15.771.2.2 [random\(\)](#)

```
virtual double Sampler\_if::random ( ) [pure virtual]
```

Generates a pseudo-random value uniformly distributed in the range [0,1).

Returns

Base pseudo-random value used by the other distribution samplers.

Implemented in [SamplerDefaultImpl1](#), and [CountingSampler](#).

15.771.2.3 [sampleBernoulli\(\)](#)

```
virtual double Sampler\_if::sampleBernoulli (
    double p ) [pure virtual]
```

Samples from a Bernoulli distribution (0/1).

Implemented in [SamplerDefaultImpl1](#), and [CountingSampler](#).

15.771.2.4 [sampleBeta\(\)](#) [1/2]

```
virtual double Sampler\_if::sampleBeta (
    double alpha,
    double beta ) [pure virtual]
```

Samples a standard Beta-distributed value in [0,1].

Implemented in [SamplerDefaultImpl1](#), and [CountingSampler](#).

15.771.2.5 [sampleBeta\(\)](#) [2/2]

```
virtual double Sampler\_if::sampleBeta (
    double alpha,
    double beta,
    double infLimit,
    double supLimit ) [pure virtual]
```

Samples a Beta-distributed value scaled to an arbitrary output interval.

Parameters

<i>alpha</i>	Alpha shape parameter (>0).
<i>beta</i>	Beta shape parameter (>0).
<i>infLimit</i>	Lower bound of the output interval.
<i>supLimit</i>	Upper bound of the output interval.

Returns

Sampled value within [infLimit, supLimit].

Implemented in [SamplerDefaultImpl1](#), and [CountingSampler](#).

15.771.2.6 sampleBinomial()

```
virtual double Sampler_if::sampleBinomial (
    int trials,
    double p ) [pure virtual]
```

Samples from a Binomial distribution with `trials` and success probability `p`.

Implemented in [SamplerDefaultImpl1](#), and [CountingSampler](#).

15.771.2.7 sampleDiscrete() [1/2]

```
virtual double Sampler_if::sampleDiscrete (
    double * prob,
    double * value,
    int size ) [pure virtual]
```

Samples from a discrete distribution using probability/value arrays.

Implemented in [CountingSampler](#), and [SamplerDefaultImpl1](#).

15.771.2.8 sampleDiscrete() [2/2]

```
virtual double Sampler_if::sampleDiscrete (
    double prob,
    double value,
    ... ) [pure virtual]
```

Samples from a discrete distribution using a variadic list of probability/value pairs.

Implemented in [SamplerDefaultImpl1](#), and [CountingSampler](#).

15.771.2.9 sampleErlang()

```
virtual double Sampler_if::sampleErlang (
    double mean,
    int M,
    double offset = 0.0 ) [pure virtual]
```

Samples from an Erlang distribution using the provided mean and number of phases.

Implemented in [SamplerDefaultImpl1](#), and [CountingSampler](#).

15.771.2.10 sampleExponential()

```
virtual double Sampler_if::sampleExponential (
    double mean,
    double offset = 0.0 ) [pure virtual]
```

Samples from an Exponential distribution with the provided mean.

Implemented in [SamplerDefaultImpl1](#), and [CountingSampler](#).

15.771.2.11 sampleGamma()

```
virtual double Sampler_if::sampleGamma (
    double alpha,
    double beta,
    double offset = 0.0 ) [pure virtual]
```

Samples from a Gamma distribution using the provided shape/scale parameters.
Implemented in [SamplerDefaultImpl1](#), and [CountingSampler](#).

15.771.2.12 sampleGeometric()

```
virtual double Sampler_if::sampleGeometric (
    double p ) [pure virtual]
```

Samples from a Geometric distribution with success probability *p*.
Implemented in [SamplerDefaultImpl1](#), and [CountingSampler](#).

15.771.2.13 sampleGumbell()

```
virtual double Sampler_if::sampleGumbell (
    double mode,
    double scale ) [pure virtual]
```

Samples from a Gumbel distribution using mode and scale.
Implemented in [SamplerDefaultImpl1](#), and [CountingSampler](#).

15.771.2.14 sampleLogNormal()

```
virtual double Sampler_if::sampleLogNormal (
    double mean,
    double stddev,
    double offset = 0.0 ) [pure virtual]
```

Samples from a Lognormal distribution parameterized by mean and standard deviation.
Implemented in [SamplerDefaultImpl1](#), and [CountingSampler](#).

15.771.2.15 sampleNormal()

```
virtual double Sampler_if::sampleNormal (
    double mean,
    double stddev ) [pure virtual]
```

Samples from a Normal distribution.
Implemented in [SamplerDefaultImpl1](#), and [CountingSampler](#).

15.771.2.16 sampleTriangular()

```
virtual double Sampler_if::sampleTriangular (
    double min,
    double mode,
    double max ) [pure virtual]
```

Samples from a Triangular distribution.
Implemented in [SamplerDefaultImpl1](#), and [CountingSampler](#).

15.771.2.17 sampleUniform()

```
virtual double Sampler_if::sampleUniform (
    double min,
    double max ) [pure virtual]
```

Samples from a continuous Uniform distribution in [*min*, *max*].
Implemented in [SamplerDefaultImpl1](#), and [CountingSampler](#).

15.771.2.18 sampleWeibull()

```
virtual double Sampler_if::sampleWeibull (
    double alpha,
    double scale ) [pure virtual]
```

Samples from a Weibull distribution.

Implemented in [SamplerDefaultImpl1](#), and [CountingSampler](#).

15.771.2.19 setRNGparameters()

```
virtual void Sampler_if::setRNGparameters (
    RNG_Parameters * param ) [pure virtual]
```

Updates the pseudo-random generator internal parameters.

Implemented in [CountingSampler](#), and [SamplerDefaultImpl1](#).

The documentation for this class was generated from the following file:

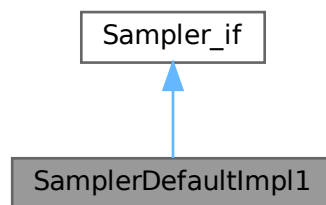
- kernel/statistics/Sampler_if.h

15.772 SamplerDefaultImpl1 Class Reference

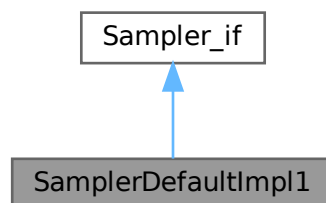
Default pseudo-random sampler implementation used by the kernel.

```
#include <kernel/statistics/SamplerDefaultImpl1.h>
```

Inheritance diagram for SamplerDefaultImpl1:



Collaboration diagram for SamplerDefaultImpl1:

**Classes**

- struct [DefaultImpl1RNG_Parameters](#)

Parameter bundle for the default RNG algorithm.

Public Member Functions

- **SamplerDefaultImpl1** ()
Creates the sampler with default RNG parameters.
- virtual ~**SamplerDefaultImpl1** ()
Destroys sampler and owned parameter objects when applicable.
- virtual double **random** () override
Generates one base uniform pseudo-random number in $[0, 1)$.
- virtual double **sampleBeta** (double alpha, double beta, double infLimit, double supLimit) override
Samples Beta in arbitrary interval $[\text{infLimit}, \text{supLimit}]$.
- virtual double **sampleBeta** (double alpha, double beta) override
Samples standard Beta in $[0, 1]$.
- virtual double **sampleErlang** (double mean, int M, double offset=0.0) override
Samples Erlang distribution.
- virtual double **sampleExponential** (double mean, double offset=0.0) override
Samples Exponential distribution.
- virtual double **sampleGamma** (double alpha, double beta, double offset=0.0) override
Samples Gamma distribution.
- virtual double **sampleGumbell** (double mode, double scale) override
Samples Gumbel distribution.
- virtual double **sampleLogNormal** (double mean, double stddev, double offset=0.0) override
Samples Lognormal distribution.
- virtual double **sampleNormal** (double mean, double stddev) override
Samples Normal distribution.
- virtual double **sampleTriangular** (double min, double mode, double max) override
Samples Triangular distribution.
- virtual double **sampleUniform** (double min, double max) override
Samples Uniform distribution in $[\text{min}, \text{max}]$.
- virtual double **sampleWeibull** (double alpha, double scale) override
Samples Weibull distribution.
- virtual double **sampleBinomial** (int trials, double p) override
Samples Binomial distribution.
- virtual double **sampleBernoulli** (double p) override
Samples Bernoulli distribution.
- virtual double **sampleDiscrete** (double prob, double value,...) override
Samples arbitrary discrete distribution (variadic interface).
- virtual double **sampleDiscrete** (double *prob, double *value, int size) override
Samples arbitrary discrete distribution (array interface).
- virtual double **sampleGeometric** (double p) override
Samples Geometric distribution.
- void **reset** ()
Reinitializes RNG state so the pseudo-random sequence restarts.
- virtual void **setRNGparameters** (RNG_Parameters *param) override
Updates the pseudo-random generator internal parameters.
- virtual RNG_Parameters * **getRNGparameters** () const override
Returns the current pseudo-random generator parameters.

15.772.1 Detailed Description

Default pseudo-random sampler implementation used by the kernel.

Implements a base RNG and multiple continuous/discrete distributions commonly used by simulation components and expression parser stochastic functions.

15.772.2 Member Function Documentation

15.772.2.1 getRNGparameters()

```
Sampler_if::RNG_Parameters * SamplerDefaultImpl1::getRNGparameters ( ) const [override], [virtual]
```

Returns the current pseudo-random generator parameters.

Implements [Sampler_if](#).

15.772.2.2 random()

```
double SamplerDefaultImpl1::random ( ) [override], [virtual]
```

Generates one base uniform pseudo-random number in [0,1).

Implements [Sampler_if](#).

15.772.2.3 sampleBernoulli()

```
double SamplerDefaultImpl1::sampleBernoulli (
    double p ) [override], [virtual]
```

Samples Bernoulli distribution.

Implements [Sampler_if](#).

15.772.2.4 sampleBeta() [1/2]

```
double SamplerDefaultImpl1::sampleBeta (
    double alpha,
    double beta ) [override], [virtual]
```

Samples standard Beta in [0,1].

Implements [Sampler_if](#).

15.772.2.5 sampleBeta() [2/2]

```
double SamplerDefaultImpl1::sampleBeta (
    double alpha,
    double beta,
    double infLimit,
    double supLimit ) [override], [virtual]
```

Samples Beta in arbitrary interval [infLimit, supLimit].

Implements [Sampler_if](#).

15.772.2.6 sampleBinomial()

```
double SamplerDefaultImpl1::sampleBinomial (
    int trials,
    double p ) [override], [virtual]
```

Samples Binomial distribution.

Implements [Sampler_if](#).

15.772.2.7 sampleDiscrete() [1/2]

```
double SamplerDefaultImpl1::sampleDiscrete (
    double * prob,
    double * value,
    int size ) [override], [virtual]
```

Samples arbitrary discrete distribution (array interface).

Implements [Sampler_if](#).

15.772.2.8 sampleDiscrete() [2/2]

```
double SamplerDefaultImpl1::sampleDiscrete (
    double prob,
    double value,
    ... ) [override], [virtual]
```

Samples arbitrary discrete distribution (variadic interface).

Implements [Sampler_if](#).

15.772.2.9 sampleErlang()

```
double SamplerDefaultImpl1::sampleErlang (
    double mean,
    int M,
    double offset = 0.0 ) [override], [virtual]
```

Samples Erlang distribution.

Implements [Sampler_if](#).

15.772.2.10 sampleExponential()

```
double SamplerDefaultImpl1::sampleExponential (
    double mean,
    double offset = 0.0 ) [override], [virtual]
```

Samples Exponential distribution.

Implements [Sampler_if](#).

15.772.2.11 sampleGamma()

```
double SamplerDefaultImpl1::sampleGamma (
    double alpha,
    double beta,
    double offset = 0.0 ) [override], [virtual]
```

Samples Gamma distribution.

Implements [Sampler_if](#).

15.772.2.12 sampleGeometric()

```
double SamplerDefaultImpl1::sampleGeometric (
    double p ) [override], [virtual]
```

Samples Geometric distribution.

Implements [Sampler_if](#).

15.772.2.13 sampleGumbell()

```
double SamplerDefaultImpl1::sampleGumbell (
    double mode,
    double scale ) [override], [virtual]
```

Samples Gumbel distribution.

Implements [Sampler_if](#).

15.772.2.14 sampleLogNormal()

```
double SamplerDefaultImpl1::sampleLogNormal (
    double mean,
    double stddev,
    double offset = 0.0 ) [override], [virtual]
```

Samples Lognormal distribution.

Implements [Sampler_if](#).

15.772.2.15 sampleNormal()

```
double SamplerDefaultImpl1::sampleNormal (
    double mean,
    double stddev ) [override], [virtual]
```

Samples Normal distribution.

Implements [Sampler_if](#).

15.772.2.16 sampleTriangular()

```
double SamplerDefaultImpl1::sampleTriangular (
    double min,
    double mode,
    double max ) [override], [virtual]
```

Samples Triangular distribution.

Implements [Sampler_if](#).

15.772.2.17 sampleUniform()

```
double SamplerDefaultImpl1::sampleUniform (
    double min,
    double max ) [override], [virtual]
```

Samples Uniform distribution in [min,max].

Implements [Sampler_if](#).

15.772.2.18 sampleWeibull()

```
double SamplerDefaultImpl1::sampleWeibull (
    double alpha,
    double scale ) [override], [virtual]
```

Samples Weibull distribution.

Implements [Sampler_if](#).

15.772.2.19 setRNGparameters()

```
void SamplerDefaultImpl1::setRNGparameters (
    RNG_Parameters * param ) [override], [virtual]
```

Updates the pseudo-random generator internal parameters.

Implements [Sampler_if](#).

The documentation for this class was generated from the following files:

- kernel/statistics/SamplerDefaultImpl1.h
- kernel/statistics/SamplerDefaultImpl1.cpp

15.773 ScenarioExperiment_if Class Reference

The documentation for this class was generated from the following file:

- kernel/simulator/ScenarioExperiment_if.h

15.774 SceneToolController Class Reference

Controller for scene visualization, drawing tools, alignment, and zoom orchestration.

```
#include <applications/gui/genesys/controllers/SceneToolController.h>
```

Public Member Functions

- [SceneToolController](#) ([ModelGraphicsView](#) *graphicsView, [Ui::MainWindow](#) *ui, [std::function](#)< [ModelGraphicsScene](#) *(> currentScene, [std::function](#)< [bool](#)()> createModelImage, [std::function](#)< [void](#)()> unselectDrawIcons, [std::function](#)< [bool](#)()> checkSelectedDrawIcons, [std::function](#)< [void](#)(double)> gentleZoom, [std::function](#)< [void](#)()> actualizeActions, [std::function](#)< [void](#)()> actualizeTabPanels, [int](#) &zoomValue, [bool](#) &firstClickShow↔ Connection)
Builds the scene-tool delegated controller used by [MainWindow](#) compatibility slots.
- void [onActionShowGridTriggered](#) ()
Toggles grid visibility and requests scene redraw updates.
- void [onActionShowRuleTriggered](#) ()
Toggles ruler visibility through the delegated scene callback path.
- void [onActionShowGuidesTriggered](#) ()
Toggles guide visibility and keeps tab/action synchronization.
- void [onActionShowSnapTriggered](#) ()
Toggles snap behavior on the current scene without changing tool wiring.
- void [onActionZoomInTriggered](#) ()
Applies incremental zoom-in and updates shared zoom state.
- void [onActionZoomOutTriggered](#) ()
Applies incremental zoom-out and updates shared zoom state.
- void [onActionZoomAllTriggered](#) ()
Fits the current graphical model in view and re-synchronizes zoom widgets.
- void [onActionZoomActualSizeTriggered](#) ()
Restores the graphical view to its default 1:1 zoom.
- void [syncZoomWidgetsForCurrentScale](#) ()
Synchronizes the zoom slider and label with the current view scale.
- void [onHorizontalSliderZoomGraphicalValueChanged](#) ([int](#) value)
Synchronizes slider-originated zoom changes with scene/view scale.
- void [onActionDrawLineTriggered](#) ()
Activates line drawing mode as a compatibility façade wrapper.
- void [onActionDrawRectangleTriggered](#) ()
Activates rectangle drawing mode as a compatibility façade wrapper.
- void [onActionDrawEllipseTriggered](#) ()
Activates ellipse drawing mode as a compatibility façade wrapper.
- void [onActionDrawTextTriggered](#) ()
Activates text drawing mode as a compatibility façade wrapper.
- void [onActionDrawPoligonTriggered](#) ()
Activates polygon drawing mode as a compatibility façade wrapper.
- void [onActionAnimateSimulatedTimeTriggered](#) ()
Toggles simulated-time animation overlays in the current scene.
- void [onActionAnimateVariableTriggered](#) ()
Toggles variable animation overlays in the current scene.
- void [onActionAnimateCounterTriggered](#) ()
Toggles counter animation overlays in the current scene.
- void [onActionAnimateAttributeTriggered](#) ()
Activates attribute animation placeholder drawing.
- void [onActionAnimateEntityTriggered](#) ()
Activates entity animation placeholder drawing.
- void [onActionAnimateEventTriggered](#) ()
Activates event animation placeholder drawing.
- void [onActionAnimateExpressionTriggered](#) ()
Activates expression animation placeholder drawing.

- void **onActionAnimatePlotTriggered** ()
Activates plot animation placeholder drawing.
- void **onActionAnimateQueueTriggered** ()
Activates queue animation placeholder drawing.
- void **onActionAnimateResourceTriggered** ()
Activates resource animation placeholder drawing.
- void **onActionAnimateStationTriggered** ()
Activates station animation placeholder drawing.
- void **onActionAnimateStatisticsTriggered** ()
Activates statistics animation placeholder drawing.
- void **onActionConnectTriggered** ()
Activates connection-creation mode and resets first-click compatibility state.
- void **onActionArrangeLeftTriggered** ()
Aligns selected graphical items to the left boundary.
- void **onActionArrangeRightTriggered** ()
Aligns selected graphical items to the right boundary.
- void **onActionArrangeTopTriggered** ()
Aligns selected graphical items to the top boundary.
- void **onActionArrangeBottomTriggered** ()
Aligns selected graphical items to the bottom boundary.
- void **onActionArrangeCenterTriggered** ()
Aligns selected graphical items to horizontal center.
- void **onActionArrangeMiddleTriggered** ()
Aligns selected graphical items to vertical middle.
- void **onActionActivateGraphicalSimulationTriggered** ()
Toggles graphical simulation visualization mode and action state.
- void **onHorizontalSliderAnimationSpeedValueChanged** (int value)
Applies animation-speed slider values to scene animation pacing.
- void **onActionSelectAllTriggered** ()
Selects all graphical items in the active scene context.
- void **onActionShowInternalElementsTriggered** ()
Toggles rendering of statistics data definitions and triggers representation refresh.
- void **onActionShowEditableElementsTriggered** ()
Toggles rendering of editable data definitions and triggers representation refresh.
- void **onActionShowAttachedElementsTriggered** ()
Toggles rendering of shared data definitions and triggers representation refresh.
- void **onCheckBoxShowElementsStateChanged** (int arg1)
Bridges checkbox state changes to the show-elements action wrapper.
- void **onCheckBoxShowInternalsStateChanged** (int arg1)
Bridges checkbox state changes to the show-internals action wrapper.
- void **onCheckBoxShowEditableElementsStateChanged** (int arg1)
Bridges checkbox state changes to the show-editable-elements action wrapper.
- void **onActionShowRecursiveElementsTriggered** ()
Toggles recursive data-definition expansion and triggers representation refresh.
- void **onCheckBoxShowRecursiveStateChanged** (int arg1)
Bridges recursive-render checkbox changes to model-image refresh workflow.
- void **onCheckBoxShowLevelsStateChanged** (int arg1)
Bridges level-annotation checkbox changes to model-image refresh workflow.
- void **onActionGModelShowConnectTriggered** ()
Toggles interactive connection-hint visibility in scene callback wiring.

15.774.1 Detailed Description

Controller for scene visualization, drawing tools, alignment, and zoom orchestration.

[MainWindow](#) delegates many menu/toolbar wrappers to this Phase-10 controller to keep the compatibility façade thin while concentrating scene-tool behavior in a dedicated component.

Responsibilities:

- handle grid/ruler/guides/snap toggles and zoom commands;
- orchestrate drawing/connection modes and alignment actions;
- coordinate animation toggles and scene-display filters through injected callbacks.

Boundaries:

- it does not own [MainWindow](#) or scene/view lifetime;
- it does not perform model persistence/export/simulation lifecycle work;
- it relies on injected references for shared UI state (zoom, first-click connection state).

15.774.2 Constructor & Destructor Documentation

15.774.2.1 SceneToolController()

```
SceneToolController::SceneToolController (
    ModelGraphicsView * graphicsView,
    Ui::MainWindow * ui,
    std::function< ModelGraphicsScene *()> currentScene,
    std::function< bool()> createModelImage,
    std::function< void()> unselectDrawIcons,
    std::function< bool()> checkSelectedDrawIcons,
    std::function< void(double)> gentleZoom,
    std::function< void()> actualizeActions,
    std::function< void()> actualizeTabPanels,
    int & zoomValue,
    bool & firstClickShowConnection )
```

Builds the scene-tool delegated controller used by [MainWindow](#) compatibility slots.
Stores only the narrow collaborators needed for scene-tool orchestration.

15.774.3 Member Function Documentation

15.774.3.1 onActionDrawLineTriggered()

```
void SceneToolController::onActionDrawLineTriggered ( )
```

Activates line drawing mode as a compatibility façade wrapper.

Activates the line drawing tool while preserving cursor semantics.

15.774.3.2 onActionShowGridTriggered()

```
void SceneToolController::onActionShowGridTriggered ( )
```

Toggles grid visibility and requests scene redraw updates.

Synchronizes the grid action with the current scene state.

15.774.3.3 onActionZoomActualSizeTriggered()

```
void SceneToolController::onActionZoomActualSizeTriggered ( )
```

Restores the graphical view to its default 1:1 zoom.

Restores the graphics view to the 1:1 zoom level and re-synchronizes the slider.

15.774.3.4 onActionZoomAllTriggered()

```
void SceneToolController::onActionZoomAllTriggered ( )
```

Fits the current graphical model in view and re-synchronizes zoom widgets.

Fits the scene to view and resets the zoom slider baseline.

15.774.3.5 onHorizontalSliderZoomGraphicalValueChanged()

```
void SceneToolController::onHorizontalSliderZoomGraphicalValueChanged (
    int value )
```

Synchronizes slider-originated zoom changes with scene/view scale.

Applies the zoom slider value to the graphics view transform.

The documentation for this class was generated from the following files:

- applications/gui/genesys/controllers/SceneToolController.h
- applications/gui/genesys/controllers/SceneToolController.cpp

15.775 SchedulableItem Class Reference**Public Types**

- enum class **Rule** : int { **IGNORE** = 1 , **PREEMPT** = 2 , **WAIT** = 3 }

Public Member Functions

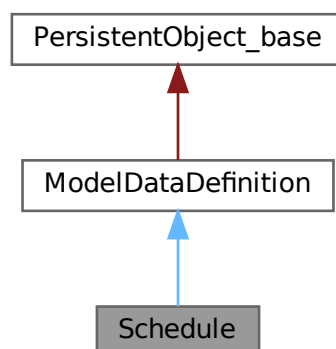
- **SchedulableItem** (std::string expression, double duration, SchedulableItem::Rule rule=SchedulableItem::Rule::IGNORE)
- void **setDuration** (double duration)
- double **getDuration** () const
- void **setExpression** (std::string expression)
- std::string **getExpression** () const
- void **setRule** (SchedulableItem::Rule rule)
- SchedulableItem::Rule **getRule** () const

The documentation for this class was generated from the following file:

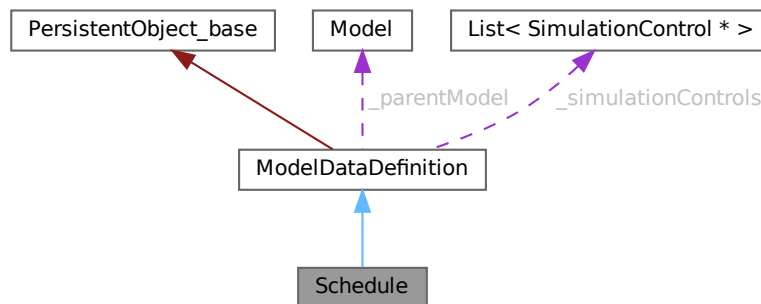
- plugins/data/DiscreteProcessing/Schedule.h

15.776 Schedule Class Reference

Inheritance diagram for Schedule:



Collaboration diagram for Schedule:



Public Member Functions

- **Schedule** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- std::string **getExpression** ()
- [List](#)< [SchedulableItem](#) * > * **getSchedulableItems** () const
- void **setRepeatAfterLast** (bool _repeatAfterLast)
- bool **isRepeatAfterLast** () const

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- [Model](#) * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * **getInternalData** (const std::string &name) const

- Looks up a named internal child data definition.*

 - `std::map< std::string, ModelDataDefinition * > * getInternalData () const`

Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`

Returns the full map of attached data definitions.
- `bool hasChanged () const`

Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`

Updates the changed flag for this data definition.
- `unsigned int getLevel () const`

Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`

Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`

Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`

Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`

Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↔ Enabled=true)`

Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`

Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`

Enables or disables the specific trace level.

Static Public Member Functions

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static PluginInformation * GetPluginInformation ()`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelDataDefinition](#)

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
- Loads a model data definition instance from serialized fields.*
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`
- Creates a new instance of a data definition using the plugin registry.*
- `static void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
- Serializes a data definition instance into a persistence record.*
- `static bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
- Validates a data definition instance.*
- `static void InitBetweenReplications (ModelDataDefinition *modeldatum)`
- Resets simulation-time state between replications.*
- `static void CreateInternalData (ModelDataDefinition *modeldatum)`
- Creates internal and attached data required by this instance.*
- `static bool CreateRelatedDataElements (ModelDataDefinition *modeldatum, std::string &errorMessage)`
- Creates related data and validates the instance in a single call.*

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` () override
- virtual void `_initBetweenReplications` () override
- virtual void `_addSimulationControl` ([SimulationControl](#) *property) override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void `trace` (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))

not just an easy access to trace manager, but wrappers to check if specific trace level applies

- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

Traces text to the simulation output.

- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.776.1 Member Function Documentation

15.776.1.1 [_addSimulationControl\(\)](#)

```
void Schedule::_addSimulationControl (
    SimulationControl * property ) [override], [protected], [virtual]
```

Keep local control registration aligned with the base model-data contract.

Reimplemented from [ModelDataDefinition](#).

15.776.1.2 [_check\(\)](#)

```
bool Schedule::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Validate that schedule has items and non-negative durations.

Reimplemented from [ModelDataDefinition](#).

15.776.1.3 [_getParserChangesInformation\(\)](#)

```
ParserChangesInformation * Schedule::_getParserChangesInformation ( ) [override], [protected],
[virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDatas. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dinamically linked to to simulator kernel to reflect the changes Return parser changes required by [Schedule](#).

No parser customization is currently necessary.

Reimplemented from [ModelDataDefinition](#).

15.776.1.4 [_initBetweenReplications\(\)](#)

```
void Schedule::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.776.1.5 `_loadInstance()`

```
bool Schedule::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Load schedulable items and repeat behavior.

Reimplemented from [ModelDataDefinition](#).

15.776.1.6 `_saveInstance()`

```
void Schedule::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Persist schedule items and repeat behavior.

Reimplemented from [ModelDataDefinition](#).

15.776.1.7 `show()`

```
std::string Schedule::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- `plugins/data/DiscreteProcessing/Schedule.h`
- `plugins/data/DiscreteProcessing/Schedule.cpp`

15.777 `testing::ScopedTrace` Class Reference

Public Member Functions

- `template<typename T>`
ScopedTrace (const char *file, int line, const T &message)
- **ScopedTrace** (const char *file, int line, const char *message)
- **ScopedTrace** (const char *file, int line, const std::string &message)

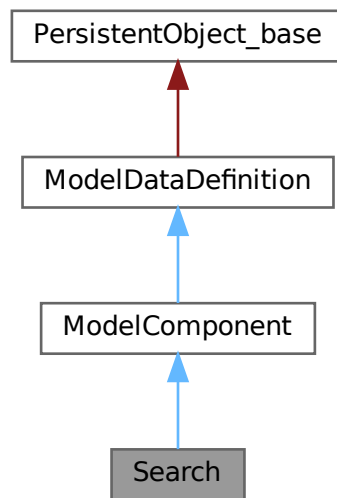
The documentation for this class was generated from the following files:

- `gtest/gtest/gtest.h`
- `gtest/src/gtest.cc`

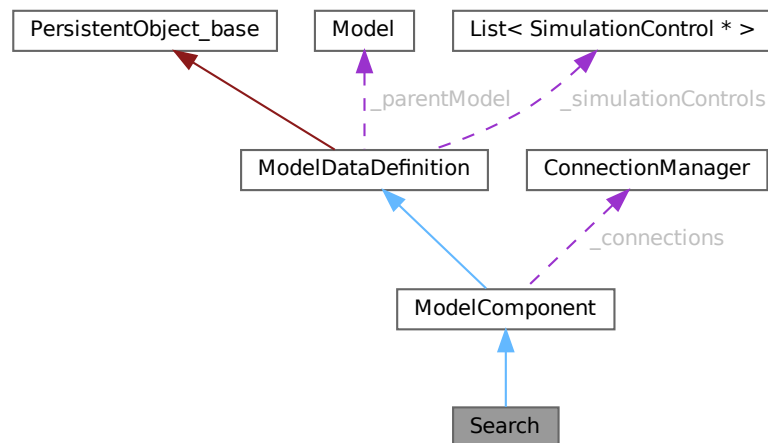
15.778 `Search` Class Reference

```
#include <plugins/components/Decisions/Search.h>
```

Inheritance diagram for Search:



Collaboration diagram for Search:



Public Types

- enum class **SearchInType** : int { **QUEUE** = 0 , **ENTITYGROUP** = 1 , **num_elements** = 2 }

Public Member Functions

- Search** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override

virtual public methods

- void **setSaveFounRankAttribute** (std::string _saveFounRankAttribute)
- std::string **getSaveFounRankAttribute** () const
- void **setSearchCondition** (std::string _searchCondition)
- std::string **getSearchCondition** () const
- void **setEndRank** (std::string _endRank)
- std::string **getEndRank** () const
- void **setStartRank** (std::string _startRank)
- std::string **getStartRank** () const
- void **setSearchInName** (std::string searchInName)
- std::string **getSearchInName** () const
- void **setSearchIn** (ModelDataDefinition * _searchIn)
- ModelDataDefinition * **getSearchIn** () const
- void **setSearchInType** (Search::SearchInType _searchInType)
- Search::SearchInType **getSearchInType** () const

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (Model *model, std::string componentTypename, std::string name="")
constructors
- virtual **~ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- ConnectionManager * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (ModelComponent *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (ModelComponent *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (Model *model, std::string datadefinitionTypename, std::string name="", bool insert↔IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual **~ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.

- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- **List**< **SimulationControl** * > * **getSimulationControls** () const
Returns the simulation controls owned by this data definition.
- **TraceManager::Level** **getTraceLevelSpecific** () const
Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (**TraceManager::Level** level)
Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (**TraceManager::Level** traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const
Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static std::string **convertEnumToStr** (**SearchInType** type)
- static **PluginInformation** * **GetPluginInformation** ()
- static **ModelComponent** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")

Static Public Member Functions inherited from **ModelComponent**

- static **ModelComponent** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** (**ModelComponent** *component)
Validates a component instance.
- static void **CreateInternalData** (**ModelComponent** *component)
Creates internal data required by a component instance.
- static void **SaveInstance** (**PersistenceRecord** *fields, **ModelComponent** *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** (**Event** *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)

- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()

virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.778.1 Detailed Description

Search module DESCRIPTION The **Search** module searches a queue, a group (batch), or an expression to find the entity rank (for entities in a queue or group) or the value of the global variable J that satisfies the specified search condition. When searching a queue or group, the value of the global system variable J is set to the rank of the first entity that satisfies **Search** Condition, or to 0 if **Search** Condition is not satisfied. When searching an expression, the global system variable J is set to the value of the first index value that satisfies the search condition or to zero if no value of J in the specified range satisfies the search condition. When an entity arrives at a **Search** module, the index J is set to the starting index and the search condition is then checked. If the search condition is satisfied, the search ends and the current value of J is retained. Otherwise, the value of J is increased or decreased and the condition is rechecked. This process repeats until the search condition is satisfied or the ending value is reached. If the condition is not met or there are no entities in the queue or group, J is set equal to 0. TYPICAL USES Looking for a particular order number in a queue Searching a group for a certain part type Determining which process to enter based on availability of resources (search an expression) Prompt Description Name Unique module identifier displayed on the module shape. Type Determination of what will be searched. **Search** options include entities in a queue, entities within a group (batch) or some expression(s). **Queue** Name Name of the queue that will be searched. Applies only when the Type is **Search** a **Queue**. Starting Value Starting rank in the queue or group or starting value for J in an expression. Ending Value Ending rank in the queue or group or ending value for J in an expression. **Search** Condition Condition containing the index J for searching expressions or containing an attribute name(s) for searching queues or batches.

15.778.2 Member Function Documentation

15.778.2.1 _check()

```
bool Search::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.778.2.2 _createEditableDataDefinitions()

```
void Search::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDats. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dynamically linked to to simulator kernel to reflect the changes

Reimplemented from [ModelDataDefinition](#).

15.778.2.3 _loadInstance()

```
bool Search::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.778.2.4 _onDispatchEvent()

```
void Search::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.778.2.5 _saveInstance()

```
void Search::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.778.2.6 show()

```
std::string Search::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/Decisions/Search.h
- plugins/components/Decisions/Search.cpp

15.779 SeizableItem Class Reference

Public Types

- enum class **SelectionRule** : int {
CYCLICAL = 0 , **RANDOM** = 1 , **SPECIFICMEMBER** = 2 , **LARGESTREMAININGCAPACITY** = 3 ,
SMALLESTNUMBERBUSY = 4 , **PREFEREDORDER** = 5 , **num_elements** = 6 }
- enum class **SeizableType** : int { **RESOURCE** = 0 , **SET** = 1 , **num_elements** = 2 }

Public Member Functions

- **SeizableItem** ([ModelDataDefinition](#) *resourceOrSet, std::string quantityExpression="1", SeizableItem↔
::SelectionRule selectionRule=SeizableItem::SelectionRule::LARGESTREMAININGCAPACITY, std::string
saveAttribute="", std::string index="0")
- **SeizableItem** ([Model](#) *model, std::string resourceName, std::string quantityExpression="1", SeizableItem↔
::SelectionRule selectionRule=SeizableItem::SelectionRule::LARGESTREMAININGCAPACITY, std::string
saveAttribute="", std::string index="0")
- **SeizableItem** ([SeizableItem](#) *original)
- bool **loadInstance** ([PersistenceRecord](#) *fields, unsigned int parentIndex)
- void **saveInstance** ([PersistenceRecord](#) *fields, unsigned int parentIndex, bool saveDefault)
- bool **loadInstance** ([PersistenceRecord](#) *fields)
- void **saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaults)
- std::string **show** ()
- void **setIndex** (std::string index)
- std::string **getIndex** () const
- void **setSaveAttribute** (std::string saveAttribute)
- std::string **getSaveAttribute** () const
- void **setSelectionRule** (SelectionRule selectionRule)
- SelectionRule **getSelectionRule** () const

- void **setQuantityExpression** (std::string quantityExpression)
- std::string **getQuantityExpression** () const
- std::string **getResourceName** () const
- std::string **getName** () const
- void **setResource** ([Resource](#) *resource)
- [Resource](#) * **getResource** () const
- void **setSet** ([Set](#) *set)
- [Set](#) * **getSet** () const
- SeizableType **getSeizableType** () const
- void **setLastMemberSeized** (unsigned int lastMemberSeized)
- unsigned int **getLastMemberSeized** () const
- [ModelDataDefinition](#) * **getSeizable** () const
- void **setElementManager** ([ModelDataManager](#) *modelDataManager)
- void **setLastPreferedOrder** (unsigned int lastPreferedOrder)
- unsigned int **getLastPreferedOrder** () const
- [List](#)< [SimulationControl](#) * > * **getSimulationControls** () const
- void **_addSimulationControl** ([SimulationControl](#) *control)
- std::string **getTypeDC** ()

Static Public Member Functions

- static std::string **convertEnumToStr** (SelectionRule rule)
- static std::string **convertEnumToStr** (SeizableType type)

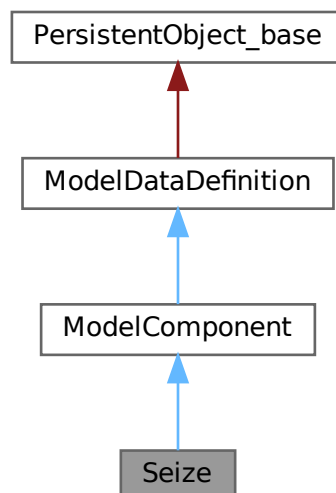
The documentation for this class was generated from the following files:

- plugins/components/DiscreteProcessing/auxiliar/SeizableItem.h
- plugins/components/DiscreteProcessing/auxiliar/SeizableItem.cpp

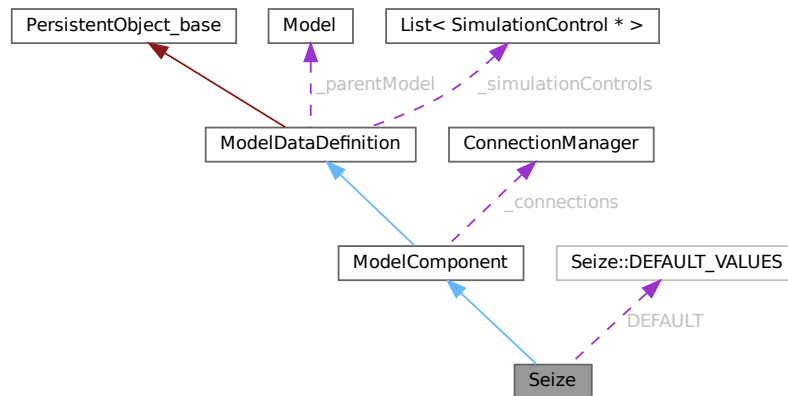
15.780 Seize Class Reference

```
#include <plugins/components/DiscreteProcessing/Seize.h>
```

Inheritance diagram for Seize:



Collaboration diagram for Seize:



Classes

- struct [DEFAULT_VALUES](#)

Public Member Functions

- **Seize** ([Model](#) *model, std::string name="")
- virtual std::string [show](#) () override
- *virtual public methods*
- void **setPriority** (unsigned short _priority)
- unsigned short **getPriority** () const
- void **setAllocationType** (Util::AllocationType _allocationType)
- Util::AllocationType **getAllocationType** () const
- void **setQueue** ([Queue](#) *queue)
- *Deprected.*
- [List](#)< [SeizableItem](#) * > * **getSeizeRequests** () const
- void **addRequest** ([SeizableItem](#) *newRequest)
- void **removeRequest** ([SeizableItem](#) *request)
- void **setQueueableItem** ([QueueableItem](#) * _queueableItem)
- [QueueableItem](#) * **getQueueableItem** () const
- void **setPriorityExpression** (std::string _priorityExpression)
- std::string **getPriorityExpression** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentName, std::string name="")
- *constructors*
- virtual ~**ModelComponent** ()
- *Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * **getConnectionManager** () const
- *new public user methods for this component*
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
- *Connects this component to another component.*
- bool **hasBreakpointAt** ()
- *Indicates whether a simulation breakpoint is set on this component.*

- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datatypeName, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) (TraceManager::Level level)

Updates the trace level used by this definition.

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific←Enabled=true)

Defines the specific trace level and optionally enables it immediately.

- bool [isTraceLevelSpecificEnabled](#) () const

Indicates whether the specific trace level is enabled.

- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Public Attributes

- const struct [Seize::DEFAULT_VALUES](#) **DEFAULT**

Protected Member Functions

- virtual void `_onDispatchEvent` (`Entity` *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool `_loadInstance` (`PersistenceRecord` *fields) override
virtual protected method that must be overridden
- virtual void `_initBetweenReplications` () override
- virtual void `_saveInstance` (`PersistenceRecord` *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_createEditableDataDefinitions` () override

Protected Member Functions inherited from `ModelDataDefinition`

- bool `_getSaveDefaultsOption` ()
- virtual `ParserChangesInformation` * `_getParserChangesInformation` ()
- virtual void `_addSimulationControl` (`SimulationControl` *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, `ModelDataDefinition` *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * _connections = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification_id
- std::string _typename
- bool _reportStatistics
- bool _hasChanged
- unsigned int _modelLevel = 0
- [Model](#) * _parentModel
- [List](#)< [SimulationControl](#) * > * _simulationControls = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level _traceLevelSpecific
- bool _traceLevelSpecificEnabled = false

15.780.1 Detailed Description

Seize module DESCRIPTION The [Seize](#) module allocates units of one or more resources to an entity. The [Seize](#) module may be used to seize units of a particular resource, a member of a resource set, or a resource as defined by an alternative method, such as an attribute or expression. When an entity enters this module, it waits in a queue (if specified) until all specified resources are available simultaneously. Allocation type for resource usage is also specified. TYPICAL USES Beginning a customer order (seize the operator) Starting a tax return (seize the accountant) Being admitted to hospital (seize the hospital room, nurse, doctor) PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Allocation Determines to which category the resource usage cost will be allocated for an entity going through the [Seize](#) module. Priority Priority value of the entity waiting at this module for the resource(s) specified if one or more entities from other modules are waiting for the same resource(s). Type Type of resource for seizing, either specifying a particular resource, or selecting from a pool of resources (that is, a resource set). The name of the resource may also be specified as an attribute value or within an expression. [Resource](#) Name Name of the resource that will be seized. [Set](#) Name Name of the resource set from which a member will be seized. [Attribute](#) Name Name of the attribute that stores the resource name to be seized. Expression Expression that evaluates to a resource name to be seized. Quantity Number of resources of a given name or from a given set that will be seized. For sets, this value specifies only the number of a selected resource that will be seized (based on the resource's capacity), not the number of members to be seized within the set. Selection Rule Method of selecting among available resources in a set. Cyclical will cycle through available members (for example, 1-2-3-1-2-3). Random will randomly select a member. Preferred Order will always select the first available member (for example, 1, if available; then 2, if available; then 3). Specific Member requires an input attribute value to specify which member of the set (previously saved in the Save [Attribute](#) field). Largest Remaining Capacity and Smallest Number Busy are used for resources with multiple capacity. Save [Attribute](#) [Attribute](#) name used to store the index number into the set of the member that is chosen. This attribute can later be referenced with the Specific Member

selection rule. [Set](#) Index Index value into the set that identifies the number into the set of the member requested. If an attribute name is used, the entity must have a value for the attribute before utilizing this option. [Resource State](#) of the resource that will be assigned after the resource is seized. The resource state must be defined with the [Resource](#) module. [Queue](#) Type Determines the type of queue used to hold the entities while waiting to seize the resource(s). If [Queue](#) is selected, the queue name is specified. If [Set](#) is selected, the queue set and member in the set are specified. If Internal is selected, an internal queue is used to hold all waiting entities. [Attribute](#) and [Expression](#) are additional methods for defining the queue to be used. [Queue](#) Name This field is visible only if [Queue](#) Type is [Queue](#), and it defines the symbol name of the queue. [Set](#) Name This field is visible only if [Queue](#) Type is [Set](#), and it defines the queue set that contains the queue being referenced. [Set](#) Index This field is visible only if [Queue](#) Type is [Set](#), and it defines the index into the queue set. Note that this is the index into the set and not the name of the queue in the set. For example, the only valid entries for a queue set containing three members is an expression that evaluates to 1, 2, or 3. [Attribute](#) This field is visible only if [Queue](#) Type is [Attribute](#). The attribute entered in this field will be evaluated to indicate which queue is to be used. [Expression](#) This field is visible only if [Queue](#) Type is [Expression](#). The expression entered in this field will be evaluated to indicate which queue is to be used.

15.780.2 Member Function Documentation

15.780.2.1 `_check()`

```
bool Seize::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.780.2.2 `_createEditableDataDefinitions()`

```
void Seize::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.780.2.3 `_initBetweenReplications()`

```
void Seize::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.780.2.4 `_loadInstance()`

```
bool Seize::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [ModelComponent](#).

15.780.2.5 `_onDispatchEvent()`

```
void Seize::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.780.2.6 _saveInstance()

```
void Seize::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.780.2.7 show()

```
std::string Seize::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

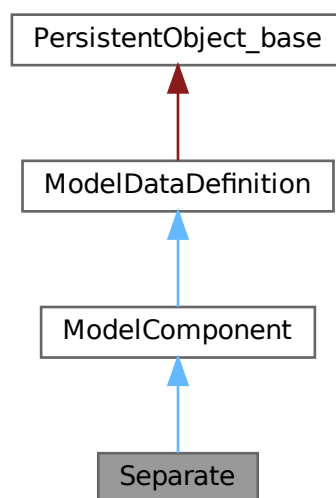
The documentation for this class was generated from the following files:

- plugins/components/DiscreteProcessing/Seize.h
- plugins/components/DiscreteProcessing/Seize.cpp

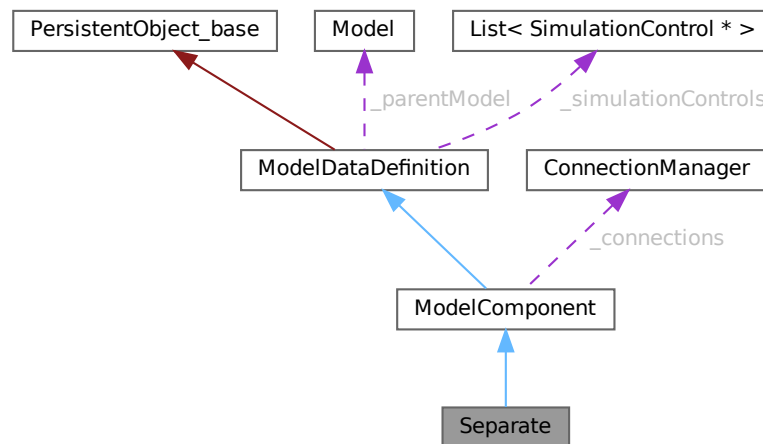
15.781 Separate Class Reference

```
#include <plugins/components/Grouping/Separate.h>
```

Inheritance diagram for Separate:



Collaboration diagram for Separate:



Public Member Functions

- **Separate** ([Model](#) *model, std::string name="")
 - virtual std::string **show** () override
- virtual public methods*

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
- *constructors*
- virtual ~**ModelComponent** ()
- *Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * **getConnectionManager** () const
- *new public user methods for this component*
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
- *Connects this component to another component.*
- bool **hasBreakpointAt** ()
- *Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string _description)
- *Updates the free-form description shown by the GUI.*
- std::string **getDescription** () const
- *Returns the free-form description associated with this component.*
- void **remove** ([ModelComponent](#) *comp)
- *Removes a connected component from this component.*

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↔ IntoModel=true)
- *Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- *Releases resources owned by this data definition.*

- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager](#)::Level [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager](#)::Level level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager](#)::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()

- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internaStatisticReportersClear** ()
- legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
- method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
- virtual protected methods that could be overridden by derived classes, if needed*

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel = 0`
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls = new List<SimulationControl*>()`
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled = false`

15.781.1 Detailed Description

[Separate](#) module DESCRIPTION This module can be used to either copy an incoming entity into multiple entities or to split a previously batched entity. Rules for allocating costs and times to the duplicate are also specified. Rules for attribute assignment to member entities are specified as well. When splitting existing batches, the temporary representative entity that was formed is disposed and the original entities that formed the group are recovered. The entities proceed sequentially from the module in the same order in which they originally were added to the batch. When duplicating entities, the specified number of copies is made and sent from the module. The original incoming entity also leaves the module. TYPICAL USES Send individual entities to represent boxes removed from a container Send an order both to fulfillment and billing for parallel processing [Separate](#) a previously batched set of documents PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Type Method of separating the incoming entity. Duplicate Original will simply take the original entity and make some number of identical duplicates. Split Existing [Batch](#) requires that the incoming entity be a temporarily batched entity using the [Batch](#) module. The original entities from the batch will be split. Percent Cost to Duplicates Allocation of costs and times of the incoming entity to the outgoing duplicates. This value is specified as a percentage of the original entity's costs and times (between 0-100). The percentage specified will be split evenly between the duplicates, while the original entity will retain any remaining cost/time percentage. Visible only when Type is Duplicate Original.

15.781.2 of Duplicates Number of outgoing entities that will leave the module, in

addition to the original incoming entity. Applies only when Type is Duplicate Original. Member Attributes Method of determining how to assign the representative entity attribute values to the original entities. These options relate to six of the special-purpose attributes (Entity.Type, Entity.Picture, Entity.Sequence, Entity.Station, Entity.Jobstep, and Entity.HoldCostRate) and all user-defined attributes. Applies only when Type is Split Existing [Batch](#). Attribute Name Name of representative entity attribute(s) that are assigned to original entities of the group. Applies only when Member Attributes is Take Specific Representative Values.

15.781.3 Member Function Documentation

15.781.3.1 `_check()`

```
bool Separate::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.781.3.2 `_loadInstance()`

```
bool Separate::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden
Reimplemented from [ModelComponent](#).

15.781.3.3 `_onDispatchEvent()`

```
void Separate::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.781.3.4 `_saveInstance()`

```
void Separate::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.781.3.5 `show()`

```
std::string Separate::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

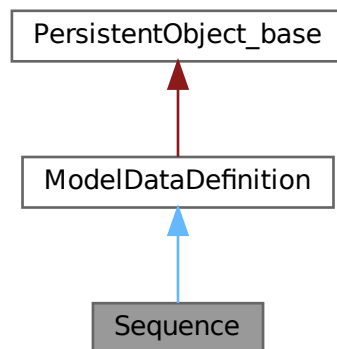
The documentation for this class was generated from the following files:

- `plugins/components/Grouping/Separate.h`
- `plugins/components/Grouping/Separate.cpp`

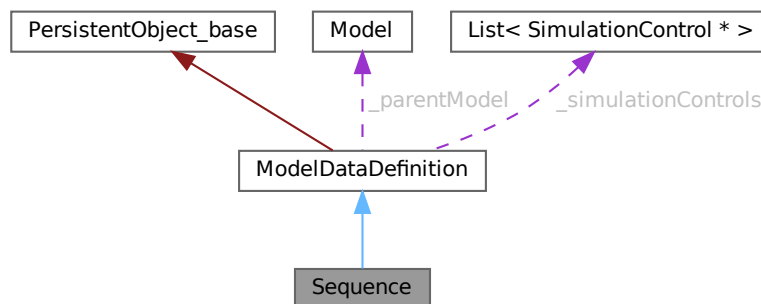
15.782 Sequence Class Reference

```
#include <plugins/data/MaterialHandling/Sequence.h>
```

Inheritance diagram for Sequence:



Collaboration diagram for Sequence:



Public Member Functions

- **Sequence** ([Model](#) *model, std::string name="")
- virtual std::string [show](#) () override
Returns a textual representation of this data definition.
- [List< SequenceStep * > * getSteps](#) () const

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert←IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)

- Updates the logical name of this instance.*

 - const std::string & [getName](#) () const

Returns the logical name of this instance.

- void [setLabel](#) (const std::string &label)

Updates the user-visible label of this instance.

- const std::string & [getLabel](#) () const

Returns the user-visible label of this instance.

- const std::string & [getClassname](#) () const

Returns the runtime class name used by the plugin system.

- [Model](#) * [getParentModel](#) () const

Returns the parent model that owns this definition.

- bool [isReportStatistics](#) () const

Indicates whether this data definition contributes report statistics.

- void [setReportStatistics](#) (bool reportStatistics)

Enables or disables report-statistics generation for this definition.

- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const

Looks up a named internal child data definition.

- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const

Returns the full map of internal child data definitions.

- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const

Returns the full map of attached data definitions.

- bool [hasChanged](#) () const

Indicates whether this instance has been modified since the last reset.

- void [setHasChanged](#) (bool [hasChanged](#))

Updates the changed flag for this data definition.

- unsigned int [getLevel](#) () const

Returns the level in the model hierarchy where this definition lives.

- void [setModelLevel](#) (unsigned int _modelLevel)

Sets the model nesting level for this definition.

- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const

Returns the simulation controls owned by this data definition.

- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const

Returns the trace level that applies specifically to this definition.

- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)

Updates the trace level used by this definition.

- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)

Defines the specific trace level and optionally enables it immediately.

- bool [isTraceLevelSpecificEnabled](#) () const

Indicates whether the specific trace level is enabled.

- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createAttachedAttributes](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)

- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void **traceSimulation** (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager::Level](#) **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.782.1 Detailed Description

[Sequence](#) module DESCRIPTION The [Sequence](#) module is used to define a sequence for entity flow through the model. A sequence consists of an ordered list of stations that an entity will visit. For each station in the visitation sequence, attributes and variables may be assigned values. Each station in the visitation sequence is referred to as a step (or jobstep) in the sequence. Three special-purpose attributes are provided for all entities. The [Sequence](#) attribute (Entity.Sequence) defines the sequence that an entity is to follow; a value of 0 indicates that the entity is not following any sequence. In order for an entity to follow a sequence, its [Sequence](#) attribute must be assigned a value (for example, in the [Assign](#) module). The Jobstep attribute (Entity.Jobstep) stores the entity's current step number in the sequence. This value is updated automatically each time an entity is transferred. You typically do not need to assign explicitly a value to Jobstep in the model. The PlannedStation attribute (Entity.PlannedStation) stores the number of the station associated with the next jobstep in the sequence. This attribute is not user-assignable. It is automatically updated whenever Entity.Sequence or Entity.JobStep changes, or whenever the entity enters a station. Jobstep names must be globally unique. TYPICAL USES Define a routing path for part processing Define a sequence of steps patients must take upon arrival at an emergency room

15.782.2 Member Function Documentation

15.782.2.1 `_check()`

```
bool Sequence::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.782.2.2 `_createAttachedAttributes()`

```
void Sequence::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.782.2.3 `_loadInstance()`

```
bool Sequence::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.782.2.4 `_saveInstance()`

```
void Sequence::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.782.2.5 `show()`

```
std::string Sequence::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.
Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

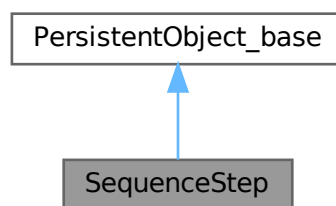
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

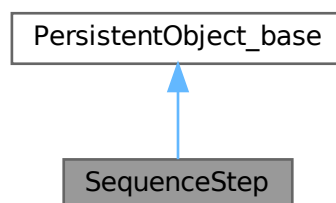
- `plugins/data/MaterialHandling/Sequence.h`
- `plugins/data/MaterialHandling/Sequence.cpp`

15.783 SequenceStep Class Reference

Inheritance diagram for SequenceStep:



Collaboration diagram for SequenceStep:



Public Member Functions

- **SequenceStep** ([Station](#) *station, std::list< [Assignment](#) * > *assignments=nullptr)
- **SequenceStep** ([Label](#) *label, std::list< [Assignment](#) * > *assignments=nullptr)
- **SequenceStep** ([Model](#) *model, std::string stationOrLabelName, bool isStation=true, std::list< [Assignment](#) * > *assignments=nullptr)
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields, unsigned int parentIndex)
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, unsigned int parentIndex, bool saveDefaultValues)
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
Loads object fields from a deserialization structure.
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

- `std::list< Assignment * > * getAssignments ()` const
- `void setStation (Station *_station)`
- `Station * getStation ()` const
- `void setElementManager (ModelDataManager *_modelDataManager)`
- `void setLabel (Label *_label)`
- `Label * getLabel ()` const

Additional Inherited Members

15.783.1 Member Function Documentation

15.783.1.1 `_loadInstance()`

```
bool SequenceStep::_loadInstance (
    PersistenceRecord * fields ) [override], [virtual]
```

Loads object fields from a deserialization structure.

Parameters

<i>fields</i>	a table-like structure which can be queried for parsed fields.
---------------	--

Returns

a boolean which is true iff the load operation was successful.

Implements [PersistentObject_base](#).

15.783.1.2 `_saveInstance()`

```
void SequenceStep::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Implements [PersistentObject_base](#).

The documentation for this class was generated from the following files:

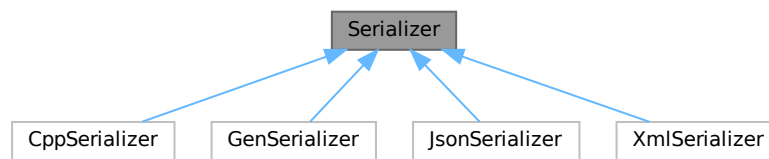
- `plugins/data/MaterialHandling/Sequence.h`
- `plugins/data/MaterialHandling/Sequence.cpp`

15.784 Serializer Class Reference

Serialization/Deserialization interface for simulation models.

```
#include <kernel/simulator/persistence/Serializer.h>
```

Inheritance diagram for Serializer:



Public Member Functions

- virtual [PersistenceRecord](#) * [newPersistenceRecord](#) ()=0
Creates an empty persistence record.
- virtual bool [dump](#) (std::ostream &output)=0
Writes formatted contents to a given output stream.
- virtual bool [load](#) (std::istream &input)=0
Attempts to load contents from a given input stream.
- virtual bool [get](#) (const std::string &name, [PersistenceRecord](#) *entry)=0
Tries to fetch an entry with the given label as a key.
- virtual bool [put](#) (const std::string name, const std::string type, const Util::identification id, [PersistenceRecord](#) *fields)=0
Upserts an entry into this structure.
- virtual int [for_each](#) (std::function< int(const std::string &) > delegate)=0
Iterates over this structure's entries, ordered by ID.

15.784.1 Detailed Description

Serialization/Deserialization interface for simulation models.

This can be seen as an associative array which knows how to ser/deserialize itself. The "contents" of a serializer are the set of entries it contains.

Its entries are identified by strings, which correspond to unique component/datadef names. Each entry has its own name, a type, an id, and an associated set of fields. All of these are contained within a [PersistenceRecord](#) instance.

Serializers also act as factories for [PersistenceRecords](#), which can then only be used with the serializer which originated them.

15.784.2 Member Function Documentation

15.784.2.1 dump()

```
virtual bool Serializer::dump (
    std::ostream & output ) [pure virtual]
```

Writes formatted contents to a given output stream.

Parameters

<i>output</i>	output stream to write to.
---------------	----------------------------

Returns

whether the current state of this structure's contents was deemed valid for serialization.

Implemented in [CppSerializer](#), [GenSerializer](#), [JsonSerializer](#), and [XmlSerializer](#).

15.784.2.2 for_each()

```
virtual int Serializer::for_each (
    std::function< int(const std::string &) > delegate ) [pure virtual]
```

Iterates over this structure's entries, ordered by ID.

Parameters

<i>delegate</i>	delegate which is applied to every entry-identifying key, until it returns a non-zero value.
-----------------	--

Returns

either zero, or the first non-zero value returned by the delegate.

Implemented in [CppSerializer](#), [GenSerializer](#), [JsonSerializer](#), and [XmlSerializer](#).

15.784.2.3 get()

```
virtual bool Serializer::get (
    const std::string & name,
    PersistenceRecord * entry ) [pure virtual]
```

Tries to fetch an entry with the given label as a key.

Parameters

<i>name</i>	a label which uniquely identifies an entry.
<i>entry</i>	(out-param) a pointer to a caller-owned entry, which is filled on success.

Returns

whether the entry was found and the out-param was actually written to.

Implemented in [CppSerializer](#), [GenSerializer](#), [JsonSerializer](#), and [XmlSerializer](#).

15.784.2.4 load()

```
virtual bool Serializer::load (
    std::istream & input ) [pure virtual]
```

Attempts to load contents from a given input stream.

Returns

whether the input was well-formatted and loading was successful.

Implemented in [CppSerializer](#), [GenSerializer](#), [JsonSerializer](#), and [XmlSerializer](#).

15.784.2.5 newPersistenceRecord()

```
virtual PersistenceRecord * Serializer::newPersistenceRecord ( ) [pure virtual]
```

Creates an empty persistence record.

Although the returned record can be freed normally with `delete`, care must be taken to only use records which come from a implementation, when configuring that implementation's contents.

Implemented in [CppSerializer](#), [GenSerializer](#), [JsonSerializer](#), and [XmlSerializer](#).

15.784.2.6 put()

```
virtual bool Serializer::put (
    const std::string name,
    const std::string type,
    const Util::identification id,
    PersistenceRecord * fields ) [pure virtual]
```

Upserts an entry into this structure.

Parameters

<i>name</i>	a label which uniquely identifies the entry.
<i>type</i>	a type name.
<i>id</i>	a component/datadef ID number.
<i>fields</i>	a persistence record for the entry, whose contents will be copied if valid.

Returns

whether the entry was valid and was actually copied into this structure.

Implemented in [CppSerializer](#), [GenSerializer](#), [JsonSerializer](#), and [XmlSerializer](#).

The documentation for this class was generated from the following files:

- kernel/simulator/persistence/Serializer.h
- kernel/simulator/persistence/Serializer.cpp

15.785 SessionContext Struct Reference

Owns the state that belongs to one authenticated web session.

```
#include <applications/worker/session/SessionContext.h>
```

Public Attributes

- std::string **sessionId**
Stable session identifier returned to the client.
- std::string **accessToken**
Bearer token used to authenticate API calls.
- std::filesystem::path **workspacePath**
Filesystem workspace reserved for the session.
- std::unique_ptr< [Simulator](#) > **simulator**
[Simulator](#) instance scoped to this session.
- std::mutex **mutex**
Guards session-local mutable state.

15.785.1 Detailed Description

Owns the state that belongs to one authenticated web session.

Each session binds one access token to one workspace directory and one simulator instance. The mutex protects session-local state during API calls and background job execution.

The documentation for this struct was generated from the following file:

- applications/worker/session/SessionContext.h

15.786 SessionManager Class Reference

Creates, stores and resolves authenticated web sessions.

```
#include <applications/worker/session/SessionManager.h>
```

Public Types

- using **SimulatorFactory** = std::function< std::unique_ptr< [Simulator](#) >()>
Factory used to build one simulator per session.

Public Member Functions

- [SessionManager](#) ([TokenService](#) &tokenService, [SimulatorFactory](#) simulatorFactory)
Builds a manager with the token service and simulator factory to use.
- [SessionContext](#) * [createSession](#) ()
Creates a new session, workspace and simulator instance.
- [SessionContext](#) * [getSessionByToken](#) (const std::string &token)
Resolves the session associated with a bearer token.

15.786.1 Detailed Description

Creates, stores and resolves authenticated web sessions.

[SessionManager](#) owns the token-to-session map used by the web API. It also creates per-session simulator instances and assigns each session an isolated workspace directory.

15.786.2 Constructor & Destructor Documentation

15.786.2.1 SessionManager()

```
SessionManager::SessionManager (
    TokenService & tokenService,
    SimulatorFactory simulatorFactory )
```

Builds a manager with the token service and simulator factory to use.

Builds the session registry and session workspace root.

Parameters

<i>tokenService</i>	Service responsible for generating and validating tokens.
<i>simulatorFactory</i>	Factory used to create a fresh simulator per session.

15.786.3 Member Function Documentation

15.786.3.1 createSession()

```
SessionContext * SessionManager::createSession ( )
```

Creates a new session, workspace and simulator instance.

Creates a session, allocates token, simulator instance and workspace.

Returns

Pointer to the stored session context.

15.786.3.2 getSessionByToken()

```
SessionContext * SessionManager::getSessionByToken (
    const std::string & token )
```

Resolves the session associated with a bearer token.

Resolves a session by bearer token.

Parameters

<i>token</i>	Bearer token presented by the client.
--------------	---------------------------------------

Returns

Pointer to the matching session or nullptr when not found.

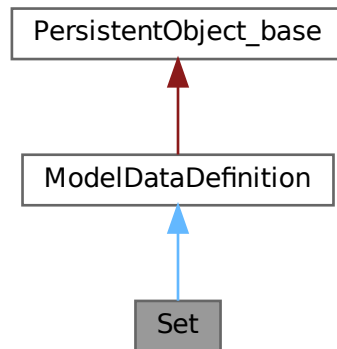
The documentation for this class was generated from the following files:

- applications/worker/session/SessionManager.h
- applications/worker/session/SessionManager.cpp

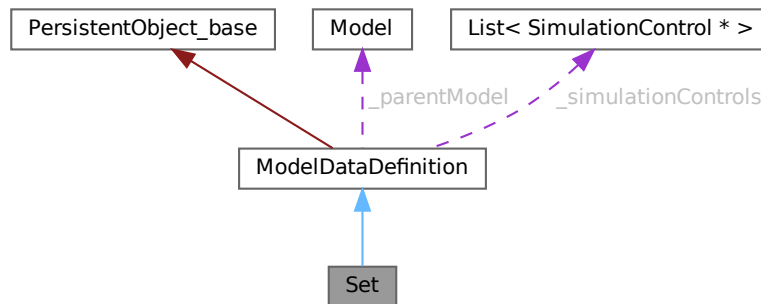
15.787 Set Class Reference

```
#include <plugins/data/Logic/Set.h>
```

Inheritance diagram for Set:



Collaboration diagram for Set:



Public Member Functions

- **Set** ([Model](#) *model, std::string name="", std::vector< std::string > allowedElementTypes={})
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- std::string **getSetOfType** () const
- bool **setSetOfType** (const std::string &setOfType)
Sets the concrete member type for this [Set](#) while it is still structurally compatible.
- bool **canChangeSetOfType** () const
Returns true while the selected concrete member type can still be changed.

- void [setAllowedElementTypes](#) (const std::vector< std::string > &allowedElementTypes)
Restricts which [ModelDataDefinition](#) subclasses may be created through this [Set](#).
- void **addAllowedElementType** (const std::string &allowedElementType)
Adds one accepted [ModelDataDefinition](#) subclass name to this [Set](#) creation contract.
- std::vector< std::string > **getAllowedElementTypes** () const
Returns the currently accepted concrete member types for GUI and tooling contracts.
- [List](#)< [ModelDataDefinition](#) * > * **getElementSet** () const
- void **addElementSet** ([ModelDataDefinition](#) *newElement)
- void **removeElementSet** ([ModelDataDefinition](#) *element)
- [ModelDataDefinition](#) * **createElementSetOfType** (const std::string &typeName, const std::string &name="")
Creates a new model data definition of the selected [Set](#) member type.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- [Model](#) * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.

- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) () override
- virtual void [_createAttachedAttributes](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.787.1 Detailed Description

[Set](#) module DESCRIPTION This data module defines various types of sets, including resource, counter, tally, entity type, and entity picture. [Resource](#) sets can be used in the [Process](#) modules (and [Seize](#), [Release](#), [Enter](#), and [Leave](#) of the Advanced [Process](#) and Advanced Transfer panels). [Counter](#) and Tally sets can be used in the [Record](#) module. [Queue](#) sets can be used with the [Seize](#), Hold, [Access](#), Request, [Leave](#), and Allocate modules of the Advanced [Process](#) and Advanced Transfer panels. TYPICAL USES Machines that can perform the same operations in a manufacturing facility Supervisors, check-out clerks in a store Shipping clerks, receptionists in an office [Set](#) of pictures corresponding to a set of entity types PROMPTS Prompt Description Name The unique name of the set being defined. Type Type of set being defined. Members Repeat group that specifies the resource members with the set. The order of listing the members within the repeat group is important when using selection rules such as Preferred Order and Cyclical. [Resource](#) Name Name of the resource to include in the resource set. Applies only when Type is [Resource](#). Tally Name Name of the tally within the tally set. Applies only when Type is Tally. [Counter](#) Name Name of the counter within the counter set. Applies only when Type is [Counter](#). [Entity](#) Type Name of the entity type within the entity type set. Applies only when Type is [Entity](#). Picture Name Name of the picture within the picture set. Applies only when Type is [Entity](#) Picture.

15.787.2 Member Function Documentation

15.787.2.1 [_check\(\)](#)

```
bool Set::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes
Reimplemented from [ModelDataDefinition](#).

15.787.2.2 [_createAttachedAttributes\(\)](#)

```
void Set::_createAttachedAttributes( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.787.2.3 [_getParserChangesInformation\(\)](#)

```
ParserChangesInformation * Set::_getParserChangesInformation ( ) [override], [protected],
[virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDats. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dynamically linked to to simulator kernel to reflect the changes

Reimplemented from [ModelDataDefinition](#).

15.787.2.4 [_loadInstance\(\)](#)

```
bool Set::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.787.2.5 _saveInstance()

```
void Set::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.787.2.6 createElementSetOfType()

```
ModelDataDefinition * Set::createElementSetOfType (
    const std::string & typeName,
    const std::string & name = "" )
```

Creates a new model data definition of the selected [Set](#) member type.

This method is the kernel-side factory used by generic property editors. The editor provides the intended type; [Set](#) validates the current type/allowed-type contract and delegates actual object construction to [PluginManager](#) so plugin registration rules stay centralized.

15.787.2.7 setAllowedElementTypes()

```
void Set::setAllowedElementTypes (
    const std::vector< std::string > & allowedElementTypes )
```

Restricts which [ModelDataDefinition](#) subclasses may be created through this [Set](#).

Empty means "use every creatable ModelDataDefinition plugin known by the simulator". Context owners such as [SeizableItem](#) and [QueueableItem](#) use this to expose Resource-only or Queue-only [Set](#) editing in the GUI without hard-coding those rules in the editor.

15.787.2.8 setSetOfType()

```
bool Set::setSetOfType (
    const std::string & setOfType )
```

Sets the concrete member type for this [Set](#) while it is still structurally compatible.

A [Set](#) is polymorphic only before its first member is inserted. Once members exist, all subsequent members must match this selected [ModelDataDefinition](#) subclass name.

15.787.2.9 show()

```
std::string Set::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

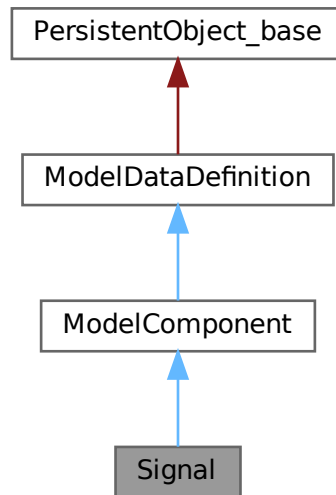
The documentation for this class was generated from the following files:

- plugins/data/Logic/Set.h
- plugins/data/Logic/Set.cpp

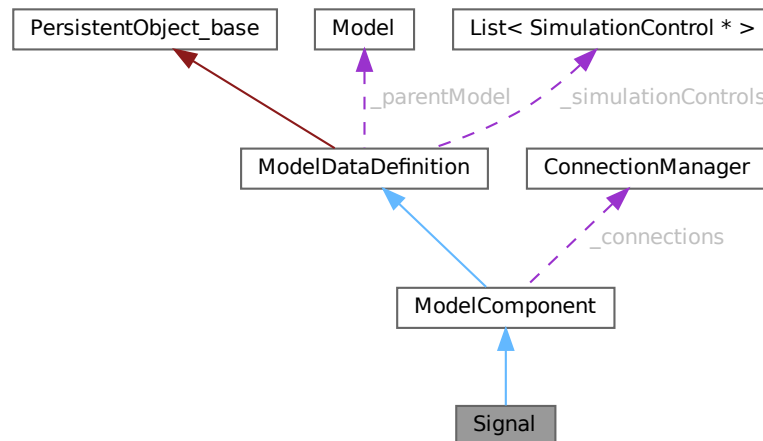
15.788 Signal Class Reference

```
#include <plugins/components/Synchronization/Signal.h>
```

Inheritance diagram for Signal:



Collaboration diagram for Signal:



Public Member Functions

- **Signal** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
virtual public methods
- void **setSignalData** ([SignalData](#) *signal)

- const std::string & **limitExpression** () const
- void **setLimitExpression** (const std::string &newLimitExpression)

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentName, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

- Returns the full map of attached data definitions.*
- bool `hasChanged` () const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged` (bool `hasChanged`)
Updates the changed flag for this data definition.
- unsigned int `getLevel` () const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel` (unsigned int `_modelLevel`)
Sets the model nesting level for this definition.
- List< `SimulationControl` * > * `getSimulationControls` () const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level` `getTraceLevelSpecific` () const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` `level`)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` `traceLevelSpecific`, bool `traceLevelSpecificEnabled`)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool `traceLevelSpecificEnabled`)
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation` * `GetPluginInformation` ()
- static `ModelComponent` * `LoadInstance` (`Model` *`model`, `PersistenceRecord` *`fields`)
- static `ModelDataDefinition` * `NewInstance` (`Model` *`model`, std::string `name`="")

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent` * `LoadInstance` (`Model` *`model`, `PersistenceRecord` *`fields`)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check` (`ModelComponent` *`component`)
Validates a component instance.
- static void `CreateInternalData` (`ModelComponent` *`component`)
Creates internal data required by a component instance.
- static void `SaveInstance` (`PersistenceRecord` *`fields`, `ModelComponent` *`component`)
Serializes a component instance into a persistence record.
- static void `DispatchEvent` (`Event` *`event`)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition` * `LoadInstance` (`Model` *`model`, `PersistenceRecord` *`fields`, bool `insertIntoModel`)
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition` * `NewInstance` (`Model` *`model`, std::string `name`="")
Creates a new instance of a data definition using the plugin registry.
- static void `SaveInstance` (`PersistenceRecord` *`fields`, `ModelDataDefinition` *`modeldatum`)
Serializes a data definition instance into a persistence record.
- static bool `Check` (`ModelDataDefinition` *`modeldatum`, std::string &`errorMessage`)
Validates a data definition instance.

- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()

- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.788.1 Detailed Description

[Signal](#) module DESCRIPTION The [Signal](#) module sends a signal value to each Hold module in the model set to [Wait](#) for [Signal](#) and releases the maximum specified number of entities. When an entity arrives at a [Signal](#) module, the signal is evaluated and the signal code is sent. At this time, entities at Hold modules that are waiting for the same signal are removed from their queues. The entity sending the signal continues processing until it encounters a delay, enters a queue, or is disposed. TYPICAL USES Analyzing traffic patterns at an intersection (signal when the light turns green) Signaling an operator to complete an order that was waiting for a component part PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. [Signal](#) Value Value of the signal to be sent to entities in Hold modules. Limit Maximum number of entities that are to be released from any Hold modules when the signal is received.

15.788.2 Member Function Documentation

15.788.2.1 `_check()`

```
bool Signal::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.788.2.2 `_createEditableDataDefinitions()`

```
void Signal::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.788.2.3 `_initBetweenReplications()`

```
void Signal::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.788.2.4 `_loadInstance()`

```
bool Signal::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

15.788.2.5 `_onDispatchEvent()`

```
void Signal::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

15.788.2.6 `_saveInstance()`

```
void Signal::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

15.788.2.7 `show()`

```
std::string Signal::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

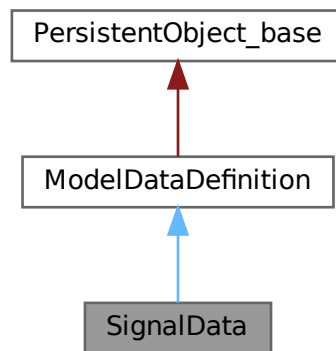
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

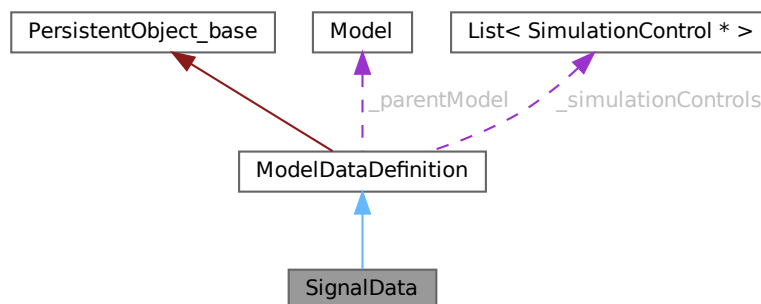
- plugins/components/Synchronization/Signal.h
- plugins/components/Synchronization/Signal.cpp

15.789 SignalData Class Reference

Inheritance diagram for SignalData:



Collaboration diagram for SignalData:



Public Types

- typedef std::function< unsigned int([SignalData](#) *) > **SignalDataEventHandler**
- typedef std::pair< SignalDataEventHandler, [ModelComponent](#) * > **PairSignalDataEventHandler**

Public Member Functions

- **SignalData** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- unsigned int **generateSignal** (double signalValue, unsigned int limit)
- void **addSignalDataEventHandler** (SignalDataEventHandler eventHandler, [ModelComponent](#) *component)
- void **removeSignalDataEventHandler** ([ModelComponent](#) *component)
- bool **hasSignalDataEventHandler** ([ModelComponent](#) *component) const
- unsigned int **remainsToLimit** () const
- void **decreaseRemainLimit** ()

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- [Model](#) * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool **hasChanged**)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * **getSimulationControls** () const

- Returns the simulation controls owned by this data definition.*
- `TraceManager::Level` [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- `void` [setTraceLevelSpecific](#) (`TraceManager::Level` level)
Updates the trace level used by this definition.
- `void` [defineTraceLevelSpecific](#) (`TraceManager::Level` traceLevelspecific, `bool` traceLevelSpecific←
Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- `bool` [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- `void` [setTraceLevelSpecificEnabled](#) (`bool` traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- `template<typename Class >`
`static` `SignalDataEventHandler` [SetSignalDataEventHandler](#) (`unsigned int`(`Class::*function`)([SignalData](#) *),
`Class` *object)
- `static` [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- `static` [PluginInformation](#) * [GetPluginInformation](#) ()
- `static` [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, `std::string` name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- `static` [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, `bool` insertIntoModel)
Loads a model data definition instance from serialized fields.
- `static` [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, `std::string` name="")
Creates a new instance of a data definition using the plugin registry.
- `static void` [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- `static bool` [Check](#) ([ModelDataDefinition](#) *modeldatum, `std::string` &errorMessage)
Validates a data definition instance.
- `static void` [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- `static void` [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- `static bool` [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, `std::string` &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- `virtual bool` [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- `virtual void` [_saveInstance](#) ([PersistenceRecord](#) *fields, `bool` saveDefaultValues) override
Saves object fields into a serialization structure.
- `virtual bool` [_check](#) (`std::string` &errorMessage) override
could be overridden by derived classes
- `virtual void` [_initBetweenReplications](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
 - legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

15.789.1 Member Function Documentation

15.789.1.1 [_check\(\)](#)

```
bool SignalData::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.789.1.2 [_initBetweenReplications\(\)](#)

```
void SignalData::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

15.789.1.3 [_loadInstance\(\)](#)

```
bool SignalData::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

15.789.1.4 [_saveInstance\(\)](#)

```
void SignalData::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

15.789.1.5 [show\(\)](#)

```
std::string SignalData::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- plugins/data/Synchronization/SignalData.h
- plugins/data/Synchronization/SignalData.cpp

15.790 GroProgramRuntime::SignalMutation Struct Reference**Public Attributes**

- SignalMutationType **type** = SignalMutationType::Emit
- double **value** = 0.0

The documentation for this struct was generated from the following file:

- plugins/data/BiochemicalSimulation/GroProgramRuntime.h

15.791 SimpleHttpServer Class Reference

Minimal blocking HTTP/1.1 server for single-request-per-connection handling.

```
#include <applications/worker/http/SimpleHttpServer.h>
```

Public Types

- using **RequestHandler** = std::function< [HttpResponse](#)(const [HttpRequest](#) &)>

Public Member Functions

- [SimpleHttpServer](#) (unsigned short port)
Construct a server bound to a TCP port.
- bool [serve](#) (const RequestHandler &handler, unsigned long maxRequests=0)
Serve HTTP requests until the optional request limit is reached.
- void [requestStop](#) ()
Requests the serving loop to stop as soon as it reaches a safe point.

15.791.1 Detailed Description

Minimal blocking HTTP/1.1 server for single-request-per-connection handling.

This server intentionally keeps the transport model simple (no keep-alive, no chunked transfer decoding) and delegates application routing to a request handler callback.

15.791.2 Constructor & Destructor Documentation**15.791.2.1 SimpleHttpServer()**

```
SimpleHttpServer::SimpleHttpServer (
    unsigned short port ) [explicit]
```

Construct a server bound to a TCP port.

Parameters

<i>port</i>	TCP port that the server will bind/listen on.
-------------	---

15.791.3 Member Function Documentation

15.791.3.1 `serve()`

```
bool SimpleHttpServer::serve (
    const RequestHandler & handler,
    unsigned long maxRequests = 0 )
```

Serve HTTP requests until the optional request limit is reached.

Serves one request at a time over a blocking TCP listener.

Parameters

<i>handler</i>	Callback used to transform a parsed request into a response.
<i>maxRequests</i>	Maximum number of client requests to process (0 means unlimited).

Returns

true when the serving loop ends normally; false on socket setup/runtime failure.

The loop accepts a connection, reads a bounded HTTP/1.1 request, delegates to the provided handler, serializes the response and closes the connection.

The documentation for this class was generated from the following files:

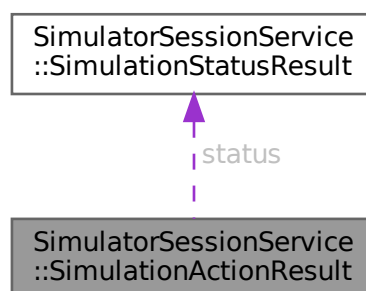
- applications/worker/http/SimpleHttpServer.h
- applications/worker/http/SimpleHttpServer.cpp

15.792 SimulatorSessionService::SimulationActionResult Struct Reference

Contains the result of a simulation action such as run or step.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Collaboration diagram for SimulatorSessionService::SimulationActionResult:



Public Attributes

- bool **success** = false
- bool **invalidToken** = false
- bool **missingCurrentModel** = false
- [SimulationStatusResult](#) **status**

15.792.1 Detailed Description

Contains the result of a simulation action such as run or step.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

15.793 SimulationCommandController Class Reference

Controller that orchestrates UI simulation commands over [SimulationController](#) guards.

```
#include <applications/gui/genesys/controllers/SimulationCommandController.h>
```

Public Member Functions

- **SimulationCommandController** ([SimulationController](#) *simulationController, const std::function< void(const std::string &)> &insertCommandInConsole, const std::function< void()> &actualizeActions, const std::function< bool()> &checkModel, const std::function< bool()> &setSimulationModelBasedOnText, const std::function< void()> &prepareNewSimulationOutputs)
Creates a command controller with narrow callbacks from [MainWindow](#).
- void **onActionSimulationStartTriggered** (bool modelChecked) const
Delegates the start simulation command flow.
- void **onActionSimulationStepTriggered** (bool modelChecked) const
Delegates the step simulation command flow.
- void **onActionSimulationPauseTriggered** () const
Delegates the pause simulation command flow.
- void **onActionSimulationResumeTriggered** (bool modelChecked) const
Delegates the resume simulation command flow.
- void **onActionSimulationStopTriggered** () const
Delegates the stop simulation command flow.

15.793.1 Detailed Description

Controller that orchestrates UI simulation commands over [SimulationController](#) guards.

This class sits between [MainWindow](#) compatibility slots and the low-level simulation readiness controller. It coordinates command intent (start/step/pause/resume/stop), command logging, and action refresh callbacks while preserving the existing UI trigger surface.

Responsibilities:

- execute simulation command flows delegated from [MainWindow](#);
- use [SimulationController](#) precondition checks before unsafe commands;
- invoke injected callbacks for console command logging and action-state refresh.

Boundaries:

- it does not own simulation state or kernel event wiring;
- it does not render trace/event widgets;
- it is a controller/orchestration layer, not a persistence or synchronization service.

The documentation for this class was generated from the following files:

- applications/gui/genesys/controllers/SimulationCommandController.h
- applications/gui/genesys/controllers/SimulationCommandController.cpp

15.794 SimulatorSessionService::SimulationConfigInput Struct Reference

Defines user-provided simulation parameters to apply to the current model.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Public Attributes

- unsigned int **numberOfReplications** = 0
- double **replicationLength** = 0.0
- double **warmUpPeriod** = 0.0
- bool **pauseOnEvent** = false
- bool **pauseOnReplication** = false
- bool **initializeStatistics** = false
- bool **initializeSystem** = false

15.794.1 Detailed Description

Defines user-provided simulation parameters to apply to the current model.

The documentation for this struct was generated from the following file:

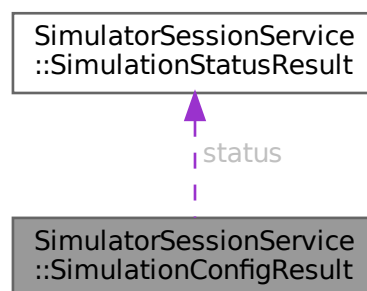
- applications/worker/service/SimulatorSessionService.h

15.795 SimulatorSessionService::SimulationConfigResult Struct Reference

Contains the result of applying simulation configuration.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Collaboration diagram for SimulatorSessionService::SimulationConfigResult:



Public Attributes

- bool **success** = false
- bool **invalidToken** = false
- bool **missingCurrentModel** = false
- [SimulationStatusResult](#) **status**

15.795.1 Detailed Description

Contains the result of applying simulation configuration.

The documentation for this struct was generated from the following file:

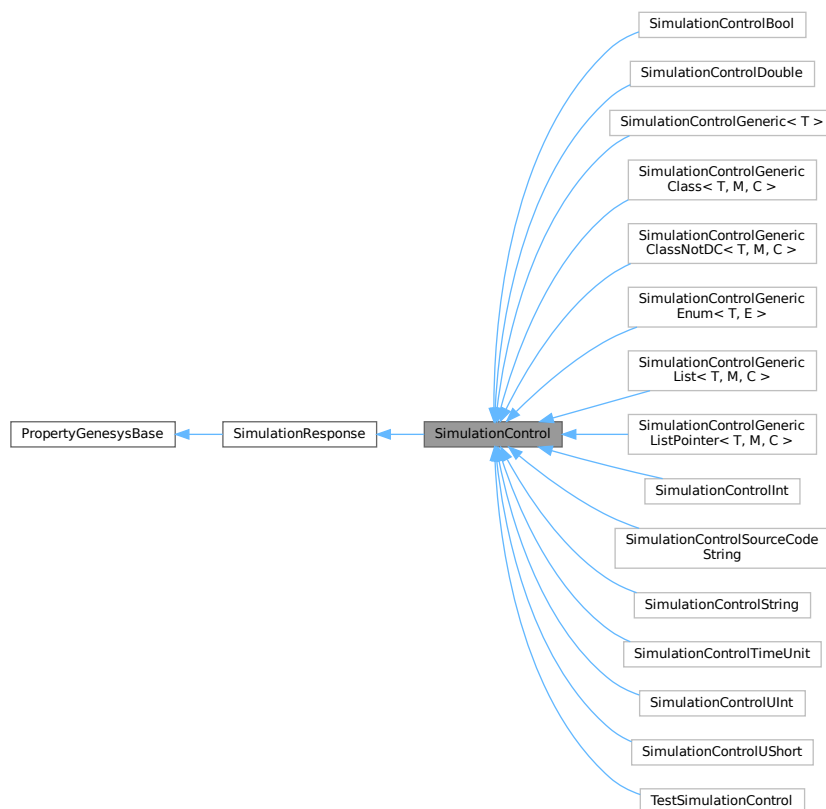
- applications/worker/service/SimulatorSessionService.h

15.796 SimulationControl Class Reference

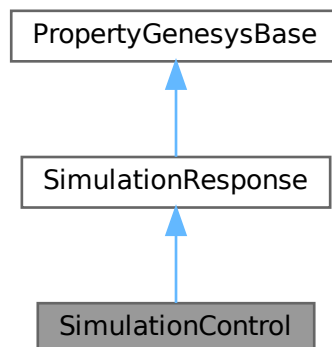
Read/write kernel-side simulation control abstraction.

```
#include <kernel/simulator/SimulationControlAndResponse.h>
```

Inheritance diagram for SimulationControl:



Collaboration diagram for SimulationControl:



Public Member Functions

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual void **setValue** (std::string value, bool remove=false)=0
- virtual [List](#)< [SimulationControl](#) * > * **getChildSimulationControls** (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool **hasObjectInstance** () const
- virtual bool **ensureObjectInstance** ()
- virtual bool **isModelDataDefinitionReference** () const
- virtual [ModelDataDefinition](#) * **getReferencedModelDataDefinition** () const
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const
- virtual SimulationControlEditorHint **preferredEditorHint** () const
- void **setPreferredEditorHint** (SimulationControlEditorHint hint)
- virtual [ModelDataDefinition](#) * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsInlineExpansion** () const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsExistingObjectSelection** () const
- virtual bool **supportsObjectCreation** () const
- virtual bool **supportsNewListElementCreation** () const
- virtual [List](#)< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual [List](#)< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual std::string **getValue** () const =0
- virtual [List](#)< std::string > * **getStrValues** ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Protected Member Functions

- void **_ensureWritable** (const char *operation) const

Protected Attributes

- bool **_readonly** = false
- SimulationControlEditorHint **_preferredEditorHint** = SimulationControlEditorHint::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

15.796.1 Detailed Description

Read/write kernel-side simulation control abstraction.

[SimulationControl](#) extends [SimulationResponse](#) with setter-based mutation and therefore represents writable kernel-side experiment controls.

15.796.2 Member Function Documentation

15.796.2.1 getChildSimulationControls()

```
virtual List< SimulationControl * > * SimulationControl::getChildSimulationControls (
    int index = 0 ) [inline], [virtual]
```

Returns nested SimulationControls exposed by this control.

This is the kernel-facing API for traversing child controls belonging to inline objects, references, or list elements represented by a control.

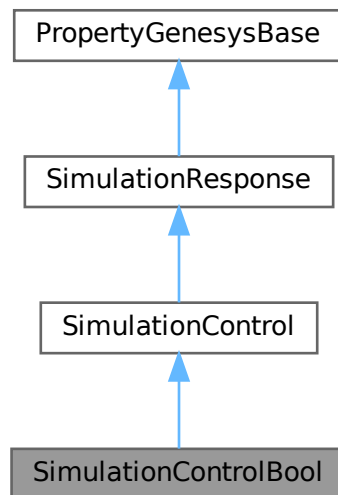
Reimplemented in [SimulationControlGenericClass< T, M, C >](#), [SimulationControlGenericClassNotDC< T, M, C >](#), and [SimulationControlGenericListPointer< T, M, C >](#).

The documentation for this class was generated from the following file:

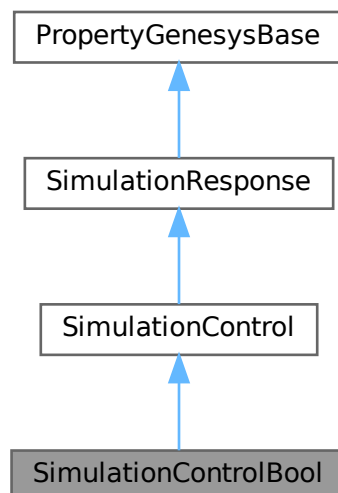
- kernel/simulator/SimulationControlAndResponse.h

15.797 SimulationControlBool Class Reference

Inheritance diagram for SimulationControlBool:



Collaboration diagram for SimulationControlBool:



Public Member Functions

- **SimulationControlBool** (GetterBool getter, SetterBool setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- virtual std::string **getValue** () const override
- virtual void **setValue** (std::string value, bool remove=false) override

Public Member Functions inherited from SimulationControl

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual [List](#)< [SimulationControl](#) * > * **getChildSimulationControls** (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool **hasObjectInstance** () const
- virtual bool **ensureObjectInstance** ()
- virtual bool **isModelDataDefinitionReference** () const
- virtual [ModelDataDefinition](#) * **getReferencedModelDataDefinition** () const
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const
- virtual SimulationControlEditorHint **preferredEditorHint** () const
- void **setPreferredEditorHint** (SimulationControlEditorHint hint)
- virtual [ModelDataDefinition](#) * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsInlineExpansion** () const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsExistingObjectSelection** () const
- virtual bool **supportsObjectCreation** () const
- virtual bool **supportsNewListElementCreation** () const
- virtual [List](#)< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual [List](#)< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from SimulationResponse

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual [List](#)< std::string > * **getStrValues** ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- SimulationControlEditorHint **_preferredEditorHint** = SimulationControlEditorHint::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

15.797.1 Member Function Documentation

15.797.1.1 **getValue()**

virtual std::string SimulationControlBool::getValue () const [inline], [override], [virtual]
Implements [SimulationResponse](#).

15.797.1.2 **setValue()**

```
virtual void SimulationControlBool::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

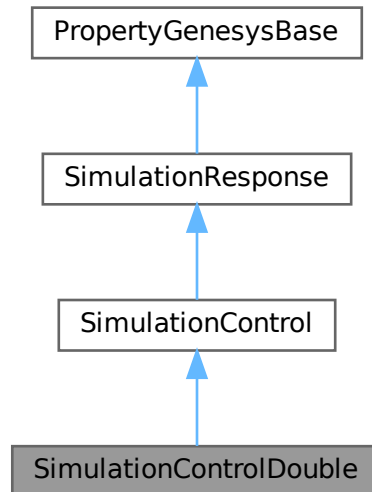
Implements [SimulationControl](#).

The documentation for this class was generated from the following file:

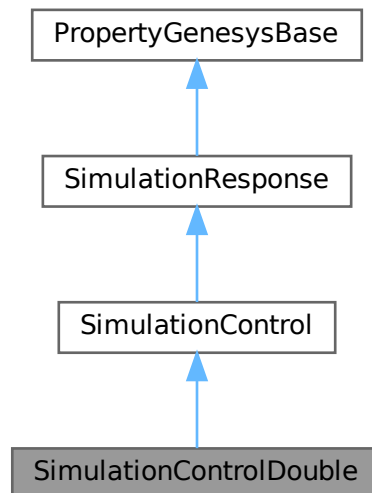
- kernel/simulator/SimulationControlAndResponse.h

15.798 SimulationControlDouble Class Reference

Inheritance diagram for SimulationControlDouble:



Collaboration diagram for SimulationControlDouble:



Public Member Functions

- **SimulationControlDouble** (GetterDouble getter, SetterDouble setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)

- virtual std::string [getValue](#) () const override
- virtual void [setValue](#) (std::string value, bool remove=false) override

Public Member Functions inherited from [SimulationControl](#)

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string [show](#) () const
- bool [isReadOnly](#) () const
- virtual [List](#)< [SimulationControl](#) * > * [getChildSimulationControls](#) (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool [hasObjectInstance](#) () const
- virtual bool [ensureObjectInstance](#) ()
- virtual bool [isModelDataDefinitionReference](#) () const
- virtual [ModelDataDefinition](#) * [getReferencedModelDataDefinition](#) () const
- virtual bool [validateProposedValue](#) (const std::string &value, std::string &errorMessage) const
- virtual [SimulationControlEditorHint](#) [preferredEditorHint](#) () const
- void [setPreferredEditorHint](#) ([SimulationControlEditorHint](#) hint)
- virtual [ModelDataDefinition](#) * [getListElementModelDataDefinition](#) (int index) const
- virtual bool [supportsInlineExpansion](#) () const
- virtual bool [supportsListEditor](#) () const
- virtual bool [supportsExistingObjectSelection](#) () const
- virtual bool [supportsObjectCreation](#) () const
- virtual bool [supportsNewListElementCreation](#) () const
- virtual [List](#)< std::string > * [getCreatableListElementTypes](#) ()
- virtual std::string [getCurrentListElementType](#) () const
- virtual bool [setCurrentListElementType](#) (const std::string &typeName)
- virtual bool [isInlineObjectProperty](#) () const
- virtual bool [createObjectInstance](#) (const std::string &value="")
- virtual [List](#)< std::string > * [getCreatableReferenceTypes](#) ()
- virtual std::string [getCurrentReferenceType](#) () const
- virtual bool [setCurrentReferenceType](#) (const std::string &typeName)
- virtual bool [createObjectInstanceOfType](#) (const std::string &typeName, const std::string &value="")
- virtual bool [createNewListElement](#) (const std::string &value="")
- virtual bool [createNewListElementOfType](#) (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * [getEditableChildSimulationControls](#) (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string [show](#) () const
- virtual [List](#)< std::string > * [getStrValues](#) ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string [whatsThis](#) () const
- std::string [getClassname](#) () const
- std::string [getElementName](#) () const
- std::string [getName](#) () const
- std::string [propertyType](#) () const
- bool [getIsList](#) () const
- bool [getIsClass](#) () const
- bool [getIsEnum](#) () const
- void [setWhatsThis](#) (const std::string text)
- void [setName](#) (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void `_ensureWritable` (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool `_readonly` = false
- SimulationControlEditorHint `_preferredEditorHint` = SimulationControlEditorHint::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string `_elementName`
- std::string `_className`
- std::string `_propertyName`
- std::string `_propertyType`
- std::string `_whatsThis`
- bool `_isList`
- bool `_isClass`
- bool `_isEnum`

15.798.1 Member Function Documentation

15.798.1.1 `getValue()`

virtual std::string SimulationControlDouble::getValue () const [inline], [override], [virtual]
Implements [SimulationResponse](#).

15.798.1.2 `setValue()`

```
virtual void SimulationControlDouble::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

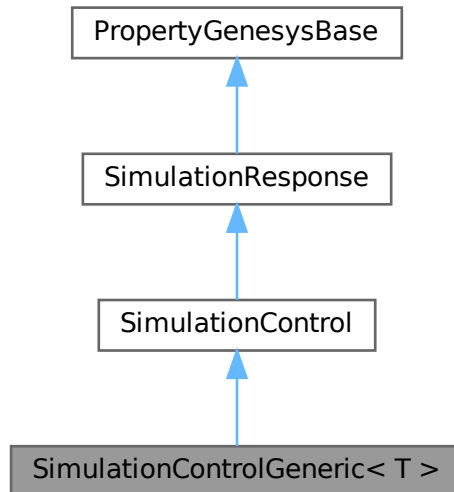
Implements [SimulationControl](#).

The documentation for this class was generated from the following file:

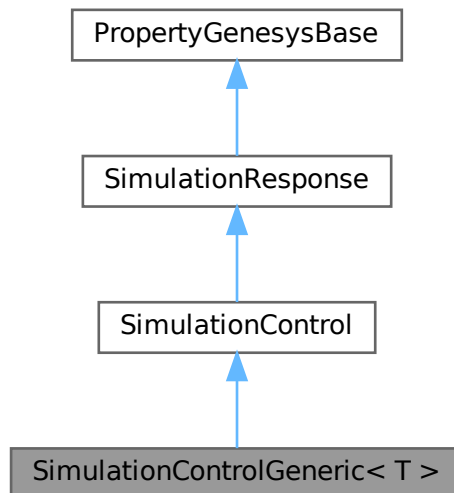
- kernel/simulator/SimulationControlAndResponse.h

15.799 SimulationControlGeneric< T > Class Template Reference

Inheritance diagram for SimulationControlGeneric< T >:



Collaboration diagram for SimulationControlGeneric< T >:



Public Member Functions

- **SimulationControlGeneric** (GetterGeneric< T > getter, SetterGeneric< T > setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)

- virtual std::string [getValue](#) () const override
- virtual void [setValue](#) (std::string value, bool remove=false) override

Public Member Functions inherited from [SimulationControl](#)

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string [show](#) () const
- bool [isReadOnly](#) () const
- virtual [List](#)< [SimulationControl](#) * > * [getChildSimulationControls](#) (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool [hasObjectInstance](#) () const
- virtual bool [ensureObjectInstance](#) ()
- virtual bool [isModelDataDefinitionReference](#) () const
- virtual [ModelDataDefinition](#) * [getReferencedModelDataDefinition](#) () const
- virtual bool [validateProposedValue](#) (const std::string &value, std::string &errorMessage) const
- virtual SimulationControlEditorHint [preferredEditorHint](#) () const
- void [setPreferredEditorHint](#) (SimulationControlEditorHint hint)
- virtual [ModelDataDefinition](#) * [getListElementModelDataDefinition](#) (int index) const
- virtual bool [supportsInlineExpansion](#) () const
- virtual bool [supportsListEditor](#) () const
- virtual bool [supportsExistingObjectSelection](#) () const
- virtual bool [supportsObjectCreation](#) () const
- virtual bool [supportsNewListElementCreation](#) () const
- virtual [List](#)< std::string > * [getCreatableListElementTypes](#) ()
- virtual std::string [getCurrentListElementType](#) () const
- virtual bool [setCurrentListElementType](#) (const std::string &typeName)
- virtual bool [isInlineObjectProperty](#) () const
- virtual bool [createObjectInstance](#) (const std::string &value="")
- virtual [List](#)< std::string > * [getCreatableReferenceTypes](#) ()
- virtual std::string [getCurrentReferenceType](#) () const
- virtual bool [setCurrentReferenceType](#) (const std::string &typeName)
- virtual bool [createObjectInstanceOfType](#) (const std::string &typeName, const std::string &value="")
- virtual bool [createNewListElement](#) (const std::string &value="")
- virtual bool [createNewListElementOfType](#) (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * [getEditableChildSimulationControls](#) (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string [show](#) () const
- virtual [List](#)< std::string > * [getStrValues](#) ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string [whatsThis](#) () const
- std::string [getClassname](#) () const
- std::string [getElementName](#) () const
- std::string [getName](#) () const
- std::string [propertyType](#) () const
- bool [getIsList](#) () const
- bool [getIsClass](#) () const
- bool [getIsEnum](#) () const
- void [setWhatsThis](#) (const std::string text)
- void [setName](#) (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void `_ensureWritable` (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool `_readonly` = false
- SimulationControlEditorHint `_preferredEditorHint` = SimulationControlEditorHint::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string `_elementName`
- std::string `_className`
- std::string `_propertyName`
- std::string `_propertyType`
- std::string `_whatsThis`
- bool `_isList`
- bool `_isClass`
- bool `_isEnum`

15.799.1 Member Function Documentation

15.799.1.1 `getValue()`

```
template<typename T >
virtual std::string SimulationControlGeneric< T >::getValue ( ) const [inline], [override],
[virtual]
Implements SimulationResponse.
```

15.799.1.2 `setValue()`

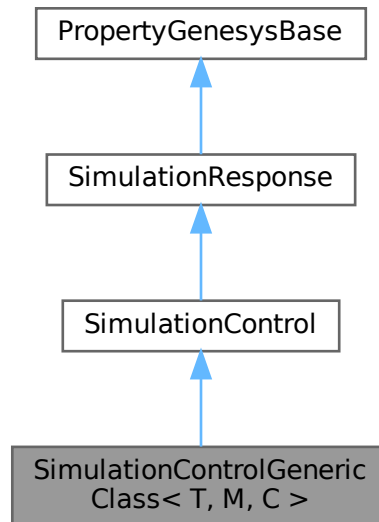
```
template<typename T >
virtual void SimulationControlGeneric< T >::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
Implements SimulationControl.
```

The documentation for this class was generated from the following file:

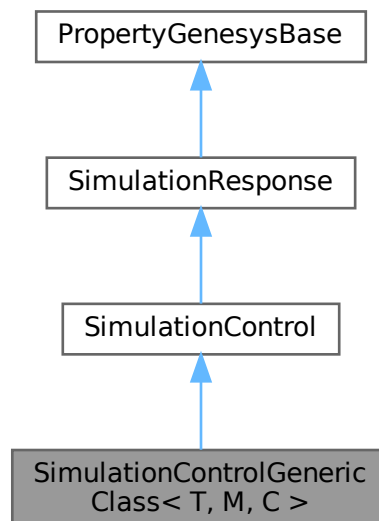
- kernel/simulator/SimulationControlAndResponse.h

15.800 SimulationControlGenericClass< T, M, C > Class Template Reference

Inheritance diagram for SimulationControlGenericClass< T, M, C >:



Collaboration diagram for SimulationControlGenericClass< T, M, C >:



Public Types

- using **Creator** = std::function< T(M, const std::string &)>

Public Member Functions

- **SimulationControlGenericClass** (M model, GetterGeneric< T > getter, SetterGeneric< T > setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=true, bool isEnum=false, Creator creator=nullptr)
- virtual std::string **getValue** () const override
- virtual void **setValue** (std::string value, bool remove=false) override
- virtual bool **isModelDataDefinitionReference** () const override
- virtual **ModelDataDefinition** * **getReferencedModelDataDefinition** () const override
- virtual bool **supportsInlineExpansion** () const override
- virtual bool **supportsExistingObjectSelection** () const override
- virtual bool **supportsObjectCreation** () const override
- virtual bool **isInlineObjectProperty** () const override
- virtual bool **hasObjectInstance** () const override
- virtual bool **ensureObjectInstance** () override
- virtual bool **createObjectInstance** (const std::string &value="") override
- virtual List< std::string > * **getStrValues** () override
- virtual List< **SimulationControl** * > * **getChildSimulationControls** (int index=0) override

Returns nested SimulationControls exposed by this control.

Public Member Functions inherited from **SimulationControl**

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const
- virtual SimulationControlEditorHint **preferredEditorHint** () const
- void **setPreferredEditorHint** (SimulationControlEditorHint hint)
- virtual **ModelDataDefinition** * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsNewListElementCreation** () const
- virtual List< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual List< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual List< **SimulationControl** * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from **SimulationResponse**

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members**Protected Member Functions inherited from [SimulationControl](#)**

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- SimulationControlEditorHint **_preferredEditorHint** = SimulationControlEditorHint::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

15.800.1 Member Function Documentation**15.800.1.1 createObjectInstance()**

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClass< T, M, C >::createObjectInstance (
    const std::string & value = "" ) [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

15.800.1.2 ensureObjectInstance()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClass< T, M, C >::ensureObjectInstance ( ) [inline],
[override], [virtual]
```

Reimplemented from [SimulationControl](#).

15.800.1.3 getChildSimulationControls()

```
template<typename T , typename M , typename C >
virtual List< SimulationControl * > * SimulationControlGenericClass< T, M, C >::getChildSimulationControls (
    int index = 0 ) [inline], [override], [virtual]
```

Returns nested SimulationControls exposed by this control.

This is the kernel-facing API for traversing child controls belonging to inline objects, references, or list elements represented by a control.

Reimplemented from [SimulationControl](#).

15.800.1.4 getReferencedModelDataDefinition()

```
template<typename T , typename M , typename C >
virtual ModelDataDefinition * SimulationControlGenericClass< T, M, C >::getReferencedModelDataDefinition ( ) const [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

15.800.1.5 getStrValues()

```
template<typename T , typename M , typename C >
virtual List< std::string > * SimulationControlGenericClass< T, M, C >::getStrValues ( )
[inline], [override], [virtual]
```

Reimplemented from [SimulationResponse](#).

15.800.1.6 getValue()

```
template<typename T , typename M , typename C >
virtual std::string SimulationControlGenericClass< T, M, C >::getValue ( ) const [inline],
[override], [virtual]
```

Implements [SimulationResponse](#).

15.800.1.7 hasObjectInstance()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClass< T, M, C >::hasObjectInstance ( ) const [inline],
[override], [virtual]
```

Reimplemented from [SimulationControl](#).

15.800.1.8 isInlineObjectProperty()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClass< T, M, C >::isInlineObjectProperty ( ) const [inline],
[override], [virtual]
```

Reimplemented from [SimulationControl](#).

15.800.1.9 isModelDataDefinitionReference()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClass< T, M, C >::isModelDataDefinitionReference ( )
const [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

15.800.1.10 setValue()

```
template<typename T , typename M , typename C >
virtual void SimulationControlGenericClass< T, M, C >::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

Implements [SimulationControl](#).

15.800.1.11 supportsExistingObjectSelection()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClass< T, M, C >::supportsExistingObjectSelection ( )
const [inline], [override], [virtual]
Reimplemented from SimulationControl.
```

15.800.1.12 supportsInlineExpansion()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClass< T, M, C >::supportsInlineExpansion ( ) const
[inline], [override], [virtual]
Reimplemented from SimulationControl.
```

15.800.1.13 supportsObjectCreation()

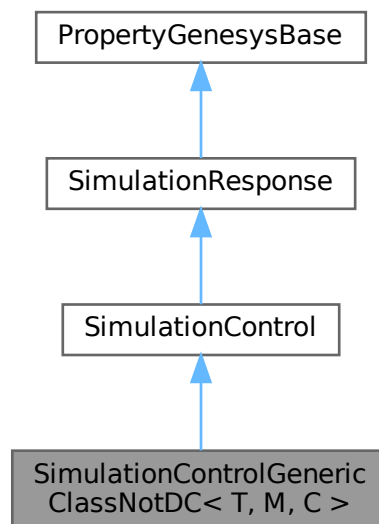
```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClass< T, M, C >::supportsObjectCreation ( ) const [inline],
[override], [virtual]
Reimplemented from SimulationControl.
```

The documentation for this class was generated from the following file:

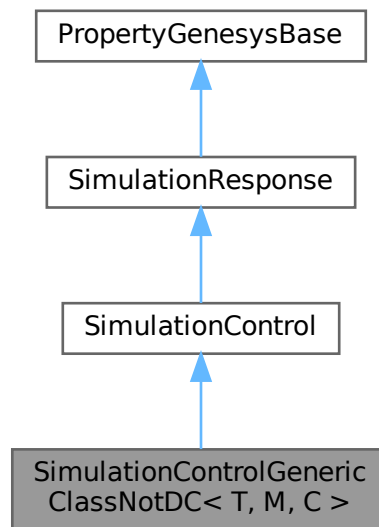
- kernel/simulator/SimulationControlAndResponse.h

15.801 SimulationControlGenericClassNotDC< T, M, C > Class Template Reference

Inheritance diagram for SimulationControlGenericClassNotDC< T, M, C >:



Collaboration diagram for `SimulationControlGenericClassNotDC< T, M, C >`:



Public Types

- using **Creator** = `std::function< T(M)>`
- using **NamedCreator** = `std::function< T(M, const std::string &)>`

Public Member Functions

- **SimulationControlGenericClassNotDC** (M model, GetterGeneric< T > getter, SetterGeneric< T > setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=true, bool isEnum=false, Creator creator=nullptr, NamedCreator namedCreator=nullptr) `Creator=nullptr`
- virtual std::string [getValue](#) () const override
- virtual void [setValue](#) (std::string value, bool remove=false) override
- virtual bool [hasObjectInstance](#) () const override
- virtual bool [supportsInlineExpansion](#) () const override
- virtual bool [supportsObjectCreation](#) () const override
- virtual bool [isInlineObjectProperty](#) () const override
- virtual bool [ensureObjectInstance](#) () override
- virtual bool [createObjectInstance](#) (const std::string &value="") override
- virtual [List](#)< [SimulationControl](#) * > * [getChildSimulationControls](#) (int index=0) override

Returns nested SimulationControls exposed by this control.

Public Member Functions inherited from [SimulationControl](#)

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string [show](#) () const
- bool [isReadOnly](#) () const
- virtual bool [isModelDataDefinitionReference](#) () const
- virtual [ModelDataDefinition](#) * [getReferencedModelDataDefinition](#) () const

- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const
- virtual SimulationControlEditorHint **preferredEditorHint** () const
- void **setPreferredEditorHint** (SimulationControlEditorHint hint)
- virtual [ModelDataDefinition](#) * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsExistingObjectSelection** () const
- virtual bool **supportsNewListElementCreation** () const
- virtual [List](#)< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual [List](#)< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual [List](#)< std::string > * **getStrValues** ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- SimulationControlEditorHint **_preferredEditorHint** = SimulationControlEditorHint::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- `std::string _elementName`
- `std::string _className`
- `std::string _propertyName`
- `std::string _propertyType`
- `std::string _whatsThis`
- `bool _isList`
- `bool _isClass`
- `bool _isEnum`

15.801.1 Member Function Documentation

15.801.1.1 `createObjectInstance()`

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClassNotDC< T, M, C >::createObjectInstance (
    const std::string & value = "" ) [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

15.801.1.2 `ensureObjectInstance()`

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClassNotDC< T, M, C >::ensureObjectInstance ( ) [inline],
[override], [virtual]
```

Reimplemented from [SimulationControl](#).

15.801.1.3 `getChildSimulationControls()`

```
template<typename T , typename M , typename C >
virtual List< SimulationControl * > * SimulationControlGenericClassNotDC< T, M, C >::get←
ChildSimulationControls (
    int index = 0 ) [inline], [override], [virtual]
```

Returns nested `SimulationControls` exposed by this control.

This is the kernel-facing API for traversing child controls belonging to inline objects, references, or list elements represented by a control.

Reimplemented from [SimulationControl](#).

15.801.1.4 `getValue()`

```
template<typename T , typename M , typename C >
virtual std::string SimulationControlGenericClassNotDC< T, M, C >::getValue ( ) const [inline],
[override], [virtual]
```

Implements [SimulationResponse](#).

15.801.1.5 `hasObjectInstance()`

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClassNotDC< T, M, C >::hasObjectInstance ( ) const [inline],
[override], [virtual]
```

Reimplemented from [SimulationControl](#).

15.801.1.6 `isInlineObjectProperty()`

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClassNotDC< T, M, C >::isInlineObjectProperty ( ) const
[inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

15.801.1.7 setValue()

```
template<typename T , typename M , typename C >
virtual void SimulationControlGenericClassNotDC< T, M, C >::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

Implements [SimulationControl](#).

15.801.1.8 supportsInlineExpansion()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClassNotDC< T, M, C >::supportsInlineExpansion ( ) const
[inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

15.801.1.9 supportsObjectCreation()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClassNotDC< T, M, C >::supportsObjectCreation ( ) const
[inline], [override], [virtual]
```

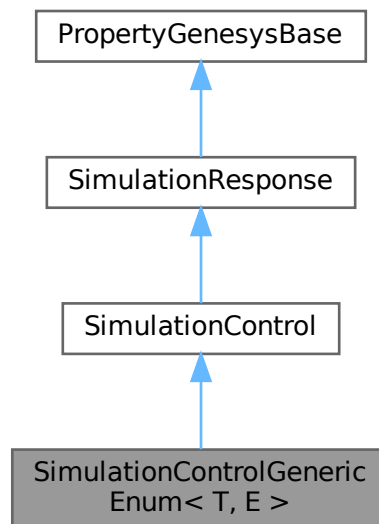
Reimplemented from [SimulationControl](#).

The documentation for this class was generated from the following file:

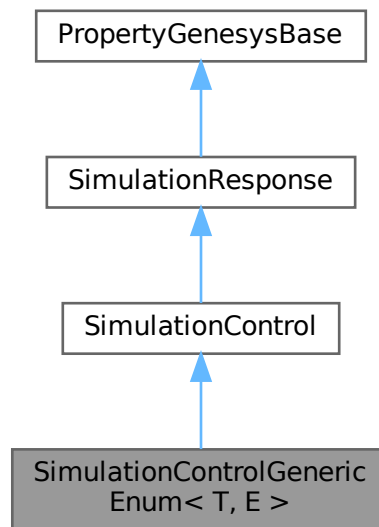
- kernel/simulator/SimulationControlAndResponse.h

15.802 SimulationControlGenericEnum< T, E > Class Template Reference

Inheritance diagram for SimulationControlGenericEnum< T, E >:



Collaboration diagram for `SimulationControlGenericEnum< T, E >`:



Public Member Functions

- **SimulationControlGenericEnum** (GetterGeneric< T > getter, SetterGeneric< T > setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=true)
- virtual std::string **getValue** () const override
- virtual void **setValue** (std::string value, bool remove=false) override
- virtual List< std::string > * **getStrValues** () override

Public Member Functions inherited from **SimulationControl**

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual List< **SimulationControl** * > * **getChildSimulationControls** (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool **hasObjectInstance** () const
- virtual bool **ensureObjectInstance** ()
- virtual bool **isModelDataDefinitionReference** () const
- virtual **ModelDataDefinition** * **getReferencedModelDataDefinition** () const
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const
- virtual **SimulationControlEditorHint** **preferredEditorHint** () const
- void **setPreferredEditorHint** (**SimulationControlEditorHint** hint)
- virtual **ModelDataDefinition** * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsInlineExpansion** () const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsExistingObjectSelection** () const
- virtual bool **supportsObjectCreation** () const
- virtual bool **supportsNewListElementCreation** () const

- virtual [List](#)< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual [List](#)< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- SimulationControlEditorHint **_preferredEditorHint** = SimulationControlEditorHint::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

15.802.1 Member Function Documentation

15.802.1.1 getStrValues()

```
template<typename T , typename E >
virtual List< std::string > * SimulationControlGenericEnum< T, E >::getStrValues ( ) [inline],
[override], [virtual]
```

Reimplemented from [SimulationResponse](#).

15.802.1.2 getValue()

```
template<typename T , typename E >
virtual std::string SimulationControlGenericEnum< T, E >::getValue ( ) const [inline], [override],
[virtual]
```

Implements [SimulationResponse](#).

15.802.1.3 setValue()

```
template<typename T , typename E >
virtual void SimulationControlGenericEnum< T, E >::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

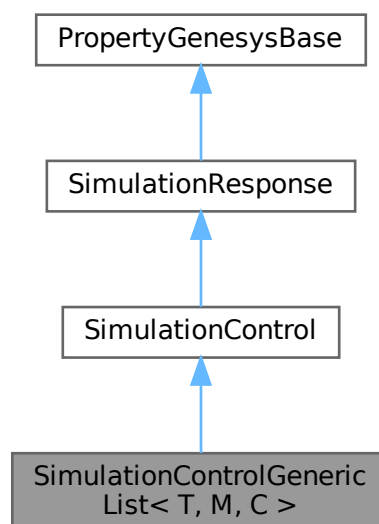
Implements [SimulationControl](#).

The documentation for this class was generated from the following file:

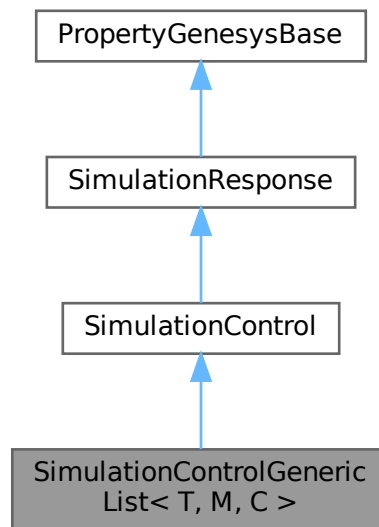
- kernel/simulator/SimulationControlAndResponse.h

15.803 SimulationControlGenericList< T, M, C > Class Template Reference

Inheritance diagram for SimulationControlGenericList< T, M, C >:



Collaboration diagram for SimulationControlGenericList< T, M, C >:



Public Member Functions

- **SimulationControlGenericList** (M model, GetterGeneric< [List](#)< T > * > getter, AdderGeneric< T > adder, RemoverGeneric< T > remover, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=true, bool isClass=false, bool isEnum=false)
- virtual std::string [getValue](#) () const override
- virtual void [setValue](#) (std::string value, bool remove=false) override
- virtual [List](#)< std::string > * [getStrValues](#) () override
- virtual bool [supportsListEditor](#) () const override

Public Member Functions inherited from [SimulationControl](#)

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string [show](#) () const
- bool [isReadOnly](#) () const
- virtual [List](#)< [SimulationControl](#) * > * [getChildSimulationControls](#) (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool [hasObjectInstance](#) () const
- virtual bool [ensureObjectInstance](#) ()
- virtual bool [isModelDataDefinitionReference](#) () const
- virtual [ModelDataDefinition](#) * [getReferencedModelDataDefinition](#) () const
- virtual bool [validateProposedValue](#) (const std::string &value, std::string &errorMessage) const
- virtual [SimulationControlEditorHint](#) [preferredEditorHint](#) () const
- void [setPreferredEditorHint](#) ([SimulationControlEditorHint](#) hint)
- virtual [ModelDataDefinition](#) * [getListElementModelDataDefinition](#) (int index) const
- virtual bool [supportsInlineExpansion](#) () const
- virtual bool [supportsExistingObjectSelection](#) () const
- virtual bool [supportsObjectCreation](#) () const
- virtual bool [supportsNewListElementCreation](#) () const

- virtual [List](#)< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual [List](#)< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- [SimulationControlEditorHint](#) **_preferredEditorHint** = [SimulationControlEditorHint](#)::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

15.803.1 Member Function Documentation

15.803.1.1 getStrValues()

```
template<typename T , typename M , typename C >
virtual List< std::string > * SimulationControlGenericList< T, M, C >::getStrValues ( ) [inline],
[override], [virtual]
```

Reimplemented from [SimulationResponse](#).

15.803.1.2 getValue()

```
template<typename T , typename M , typename C >
virtual std::string SimulationControlGenericList< T, M, C >::getValue ( ) const [inline],
[override], [virtual]
```

Implements [SimulationResponse](#).

15.803.1.3 setValue()

```
template<typename T , typename M , typename C >
virtual void SimulationControlGenericList< T, M, C >::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

Implements [SimulationControl](#).

15.803.1.4 supportsListEditor()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericList< T, M, C >::supportsListEditor ( ) const [inline],
[override], [virtual]
```

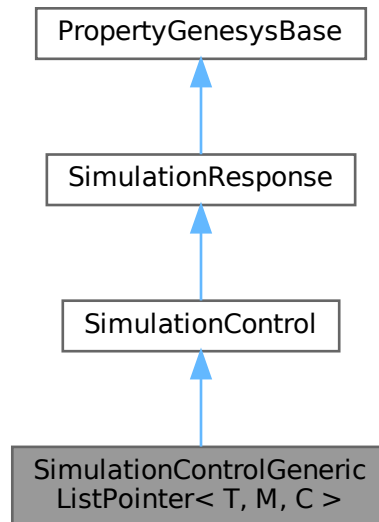
Reimplemented from [SimulationControl](#).

The documentation for this class was generated from the following file:

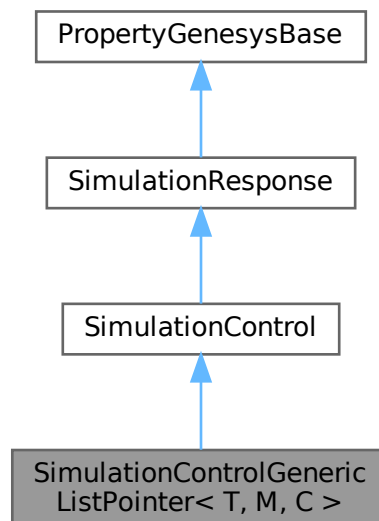
- kernel/simulator/SimulationControlAndResponse.h

15.804 `SimulationControlGenericListPointer< T, M, C >` Class Template Reference

Inheritance diagram for `SimulationControlGenericListPointer< T, M, C >`:



Collaboration diagram for `SimulationControlGenericListPointer< T, M, C >`:



Public Types

- using **Creator** = std::function< T(M, const std::string &)>
- using **TypedCreator** = std::function< T(M)>
- using **ElementTypeNamesGetter** = std::function< std::vector< std::string >()>
- using **CurrentElementTypeGetter** = std::function< std::string()>
- using **CurrentElementTypeSetter** = std::function< bool(const std::string &)>
- using **TypedNamedCreator** = std::function< T(M, const std::string &, const std::string &)>
- using **ListElementDataDefinitionGetter** = std::function< [ModelDataDefinition](#) *(int)>

Public Member Functions

- **SimulationControlGenericListPointer** (M model, GetterGeneric< [List](#)< T > * > getter, AdderGeneric< T > adder, RemoverGeneric< T > remover, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=true, bool isClass=true, bool isEnum=false, Creator creator=nullptr, TypedCreator typedCreator=nullptr, ElementTypeNamesGetter elementTypeNamesGetter=nullptr, CurrentElementTypeGetter currentElementTypeGetter=nullptr, CurrentElementTypeSetter currentElementTypeSetter=nullptr, TypedNamedCreator typedNamedCreator=nullptr, ListElementDataDefinitionGetter listElementDataDefinitionGetter=nullptr)
- virtual std::string [getValue](#) () const override
- virtual void [setValue](#) (std::string value, bool remove=false) override
- virtual bool [supportsListEditor](#) () const override
- virtual bool [supportsNewListElementCreation](#) () const override
- virtual [List](#)< std::string > * [getCreatableListElementTypes](#) () override
- virtual std::string [getCurrentListElementType](#) () const override
- virtual bool [setCurrentListElementType](#) (const std::string &typeName) override
- virtual bool [createNewListElement](#) (const std::string &value="") override
- virtual bool [createNewListElementOfType](#) (const std::string &typeName, const std::string &value="") override
- virtual [List](#)< [SimulationControl](#) * > * [getChildSimulationControls](#) (int index=0) override
Returns nested SimulationControls exposed by this control.
- virtual [ModelDataDefinition](#) * [getListElementModelDataDefinition](#) (int index) const override
- virtual [List](#)< std::string > * [getStrValues](#) () override

Public Member Functions inherited from [SimulationControl](#)

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string [show](#) () const
- bool [isReadOnly](#) () const
- virtual bool [hasObjectInstance](#) () const
- virtual bool [ensureObjectInstance](#) ()
- virtual bool [isModelDataDefinitionReference](#) () const
- virtual [ModelDataDefinition](#) * [getReferencedModelDataDefinition](#) () const
- virtual bool [validateProposedValue](#) (const std::string &value, std::string &errorMessage) const
- virtual [SimulationControlEditorHint](#) [preferredEditorHint](#) () const
- void [setPreferredEditorHint](#) ([SimulationControlEditorHint](#) hint)
- virtual bool [supportsInlineExpansion](#) () const
- virtual bool [supportsExistingObjectSelection](#) () const
- virtual bool [supportsObjectCreation](#) () const
- virtual bool [isInlineObjectProperty](#) () const
- virtual bool [createObjectInstance](#) (const std::string &value="")
- virtual [List](#)< std::string > * [getCreatableReferenceTypes](#) ()
- virtual std::string [getCurrentReferenceType](#) () const
- virtual bool [setCurrentReferenceType](#) (const std::string &typeName)
- virtual bool [createObjectInstanceOfType](#) (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * [getEditableChildSimulationControls](#) (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- SimulationControlEditorHint **_preferredEditorHint** = SimulationControlEditorHint::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

15.804.1 Member Function Documentation

15.804.1.1 createNewListElement()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericListPointer< T, M, C >::createNewListElement (
    const std::string & value = "" ) [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

15.804.1.2 createNewListElementOfType()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericListPointer< T, M, C >::createNewListElementOfType (
    const std::string & typeName,
    const std::string & value = "" ) [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

15.804.1.3 getChildSimulationControls()

```
template<typename T , typename M , typename C >
virtual List< SimulationControl * > * SimulationControlGenericListPointer< T, M, C >::get↔
ChildSimulationControls (
    int index = 0 ) [inline], [override], [virtual]
```

Returns nested SimulationControls exposed by this control.

This is the kernel-facing API for traversing child controls belonging to inline objects, references, or list elements represented by a control.

Reimplemented from [SimulationControl](#).

15.804.1.4 getCreatableListElementTypes()

```
template<typename T , typename M , typename C >
virtual List< std::string > * SimulationControlGenericListPointer< T, M, C >::getCreatable↔
ListElementTypes ( ) [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

15.804.1.5 getCurrentListElementType()

```
template<typename T , typename M , typename C >
virtual std::string SimulationControlGenericListPointer< T, M, C >::getCurrentListElementType
( ) const [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

15.804.1.6 getListElementModelDataDefinition()

```
template<typename T , typename M , typename C >
virtual ModelDataDefinition * SimulationControlGenericListPointer< T, M, C >::getListElement↔
ModelDataDefinition (
    int index ) const [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

15.804.1.7 getStrValues()

```
template<typename T , typename M , typename C >
virtual List< std::string > * SimulationControlGenericListPointer< T, M, C >::getStrValues (
) [inline], [override], [virtual]
```

Reimplemented from [SimulationResponse](#).

15.804.1.8 getValue()

```
template<typename T , typename M , typename C >
virtual std::string SimulationControlGenericListPointer< T, M, C >::getValue ( ) const [inline],
[override], [virtual]
```

Implements [SimulationResponse](#).

15.804.1.9 setCurrentListElementType()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericListPointer< T, M, C >::setCurrentListElementType (
    const std::string & typeName ) [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

15.804.1.10 setValue()

```
template<typename T , typename M , typename C >
virtual void SimulationControlGenericListPointer< T, M, C >::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

Implements [SimulationControl](#).

15.804.1.11 supportsListEditor()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericListPointer< T, M, C >::supportsListEditor ( ) const
[inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

15.804.1.12 supportsNewListElementCreation()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericListPointer< T, M, C >::supportsNewListElementCreation (
) const [inline], [override], [virtual]
```

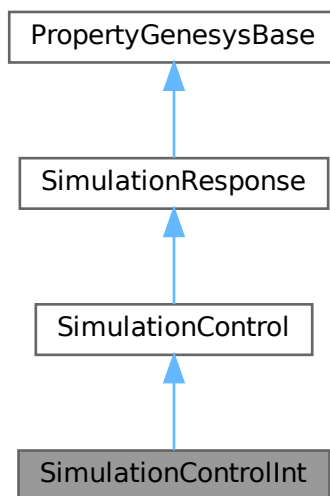
Reimplemented from [SimulationControl](#).

The documentation for this class was generated from the following file:

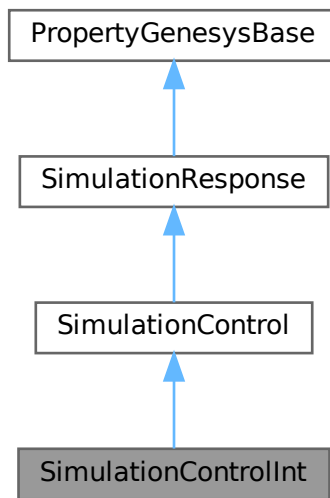
- kernel/simulator/SimulationControlAndResponse.h

15.805 SimulationControlInt Class Reference

Inheritance diagram for SimulationControlInt:



Collaboration diagram for SimulationControlInt:



Public Member Functions

- **SimulationControlInt** (GetterInt getter, SetterInt setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- virtual std::string **getValue** () const override
- virtual void **setValue** (std::string value, bool remove=false) override

Public Member Functions inherited from SimulationControl

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual List< SimulationControl * > * **getChildSimulationControls** (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool **hasObjectInstance** () const
- virtual bool **ensureObjectInstance** ()
- virtual bool **isModelDataDefinitionReference** () const
- virtual ModelDataDefinition * **getReferencedModelDataDefinition** () const
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const
- virtual SimulationControlEditorHint **preferredEditorHint** () const
- void **setPreferredEditorHint** (SimulationControlEditorHint hint)
- virtual ModelDataDefinition * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsInlineExpansion** () const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsExistingObjectSelection** () const
- virtual bool **supportsObjectCreation** () const
- virtual bool **supportsNewListElementCreation** () const
- virtual List< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)

- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual [List](#)< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual [List](#)< std::string > * **getStrValues** ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- [SimulationControlEditorHint](#) **_preferredEditorHint** = [SimulationControlEditorHint](#)::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

15.805.1 Member Function Documentation

15.805.1.1 getValue()

virtual std::string SimulationControlInt::getValue () const [inline], [override], [virtual]
Implements [SimulationResponse](#).

15.805.1.2 setValue()

virtual void SimulationControlInt::setValue (
std::string value,
bool remove = false) [inline], [override], [virtual]

Implements [SimulationControl](#).

The documentation for this class was generated from the following file:

- kernel/simulator/SimulationControlAndResponse.h

15.806 SimulationController Class Reference

Controller that validates simulation command readiness for [MainWindow](#) wrappers.

```
#include <applications/gui/genesys/controllers/SimulationController.h>
```

Public Member Functions

- [SimulationController](#) (QWidget *ownerWidget, [Simulator](#) *simulator)
Creates a controller bound to GUI owner and simulator.
- bool [hasCurrentModelSimulation](#) () const
Checks whether there is a valid current model with simulation object.
- [ModelSimulation](#) * [currentSimulation](#) () const
Returns current simulation object when available.
- bool [ensureReady](#) (bool checkModel, bool modelChecked, const std::function< bool()> &runModelCheck, const std::function< bool()> &syncModelFromText) const
Validates whether GUI can execute a simulation command now.

15.806.1 Detailed Description

Controller that validates simulation command readiness for [MainWindow](#) wrappers.

This controller is part of the incremental extraction that keeps [MainWindow](#) as a compatibility façade while moving simulation-command orchestration behind smaller collaborators. It centralizes precondition checks shared by start/step/pause/resume/stop flows, including model availability and model-text synchronization guards.

Responsibilities:

- verify that the current model and its [ModelSimulation](#) are available;
- execute callback-based readiness checks delegated by [MainWindow](#);
- return a single command-ready decision used by higher-level command controllers.

Boundaries:

- it does not own simulation state or lifecycle (owned by [Simulator](#));
- it does not execute GUI commands directly (handled by [SimulationCommandController](#));
- it does not update widgets, tabs, or trace outputs.

Dependencies are injected narrowly (owner widget + simulator + callbacks) to preserve existing ownership and enable incremental compatibility without changing slot signatures.

15.806.2 Constructor & Destructor Documentation

15.806.2.1 SimulationController()

```
SimulationController::SimulationController (
    QWidget * ownerWidget,
    Simulator * simulator )
```

Creates a controller bound to GUI owner and simulator.

Parameters

<i>ownerWidget</i>	Widget used to show warning dialogs.
<i>simulator</i>	Simulator facade used by GUI.

15.806.3 Member Function Documentation

15.806.3.1 currentSimulation()

```
ModelSimulation * SimulationController::currentSimulation ( ) const
```

Returns current simulation object when available.

Returns

Pointer to current [ModelSimulation](#) or nullptr when unavailable.

15.806.3.2 ensureReady()

```
bool SimulationController::ensureReady (
    bool checkModel,
    bool modelChecked,
    const std::function< bool()> & runModelCheck,
    const std::function< bool()> & syncModelFromText ) const
```

Validates whether GUI can execute a simulation command now.

Validation pipeline: 1) Ensure current model/simulation exists; 2) Optionally execute model check callback when model is not checked yet; 3) Execute model synchronization callback (text -> model) and ensure simulation still exists.

Parameters

<i>checkModel</i>	If true, model-check callback may be executed.
<i>modelChecked</i>	Indicates current model check state from MainWindow .
<i>runModelCheck</i>	Callback that executes model check when needed.
<i>syncModelFromText</i>	Callback that synchronizes textual model representation.

Returns

true if command can proceed safely.

Todo Replace callback-based contract by dedicated interfaces to improve testability.

15.806.3.3 hasCurrentModelSimulation()

```
bool SimulationController::hasCurrentModelSimulation ( ) const
```

Checks whether there is a valid current model with simulation object.

Returns

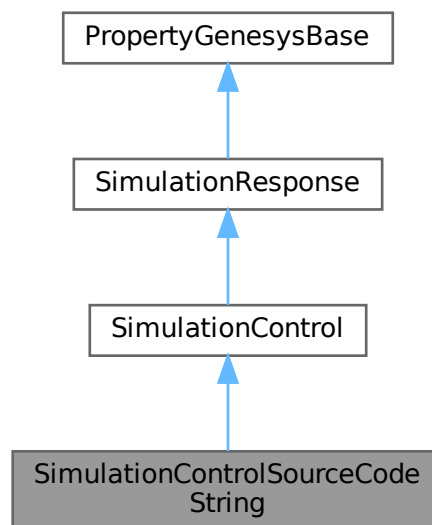
true when simulation commands can access `current() -> getSimulation()` safely.

The documentation for this class was generated from the following files:

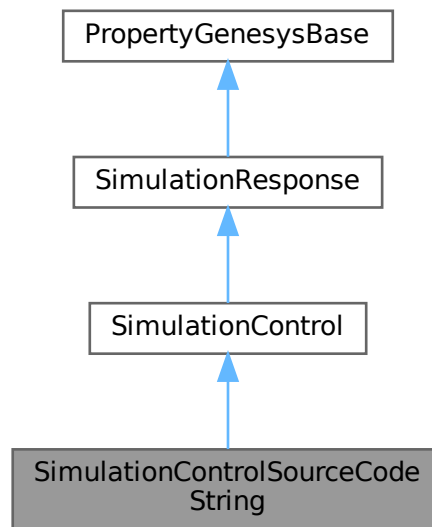
- applications/gui/genesys/controllers/SimulationController.h
- applications/gui/genesys/controllers/SimulationController.cpp

15.807 SimulationControlSourceCodeString Class Reference

Inheritance diagram for SimulationControlSourceCodeString:



Collaboration diagram for SimulationControlSourceCodeString:



Public Member Functions

- **SimulationControlSourceCodeString** (GetterSourceCodeString getter, SetterSourceCodeString setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false, ValidatorString validator=nullptr)
- virtual std::string **getValue** () const override
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const override
- virtual void **setValue** (std::string value, bool remove=false) override

Public Member Functions inherited from **SimulationControl**

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual **List**< **SimulationControl** * > * **getChildSimulationControls** (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool **hasObjectInstance** () const
- virtual bool **ensureObjectInstance** ()
- virtual bool **isModelDataDefinitionReference** () const
- virtual **ModelDataDefinition** * **getReferencedModelDataDefinition** () const
- virtual **SimulationControlEditorHint** **preferredEditorHint** () const
- void **setPreferredEditorHint** (**SimulationControlEditorHint** hint)
- virtual **ModelDataDefinition** * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsInlineExpansion** () const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsExistingObjectSelection** () const
- virtual bool **supportsObjectCreation** () const
- virtual bool **supportsNewListElementCreation** () const
- virtual **List**< std::string > * **getCreatableListElementTypes** ()

- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual [List](#)< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual [List](#)< std::string > * **getStrValues** ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- [SimulationControlEditorHint](#) **_preferredEditorHint** = [SimulationControlEditorHint](#)::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

15.807.1 Member Function Documentation

15.807.1.1 `getValue()`

```
virtual std::string SimulationControlSourceCodeString::getValue ( ) const [inline], [override], [virtual]
```

Implements [SimulationResponse](#).

15.807.1.2 `setValue()`

```
virtual void SimulationControlSourceCodeString::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

Implements [SimulationControl](#).

15.807.1.3 `validateProposedValue()`

```
virtual bool SimulationControlSourceCodeString::validateProposedValue (
    const std::string & value,
    std::string & errorMessage ) const [inline], [override], [virtual]
```

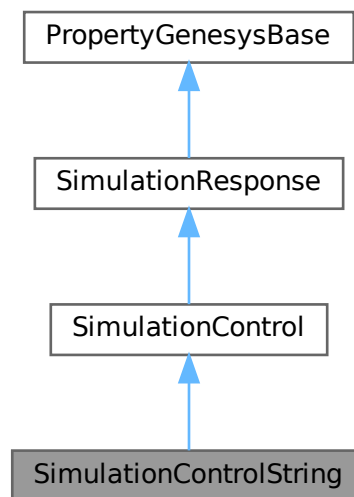
Reimplemented from [SimulationControl](#).

The documentation for this class was generated from the following file:

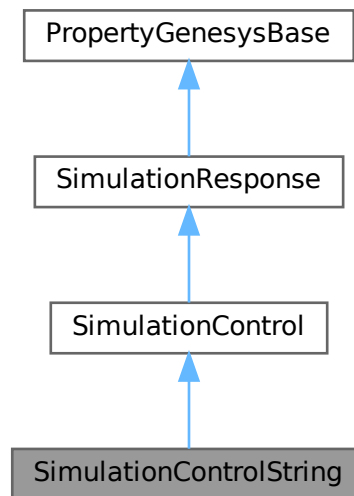
- kernel/simulator/SimulationControlAndResponse.h

15.808 SimulationControlString Class Reference

Inheritance diagram for SimulationControlString:



Collaboration diagram for SimulationControlString:



Public Member Functions

- **SimulationControlString** (GetString getter, SetterString setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false, ValidatorString validator=nullptr)
- virtual std::string **getValue** () const override
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const override
- virtual void **setValue** (std::string value, bool remove=false) override

Public Member Functions inherited from **SimulationControl**

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual **List**< **SimulationControl** * > * **getChildSimulationControls** (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool **hasObjectInstance** () const
- virtual bool **ensureObjectInstance** ()
- virtual bool **isModelDataDefinitionReference** () const
- virtual **ModelDataDefinition** * **getReferencedModelDataDefinition** () const
- virtual **SimulationControlEditorHint** **preferredEditorHint** () const
- void **setPreferredEditorHint** (**SimulationControlEditorHint** hint)
- virtual **ModelDataDefinition** * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsInlineExpansion** () const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsExistingObjectSelection** () const
- virtual bool **supportsObjectCreation** () const
- virtual bool **supportsNewListElementCreation** () const
- virtual **List**< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const

- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual [List](#)< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual [List](#)< std::string > * **getStrValues** ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- [SimulationControlEditorHint](#) **_preferredEditorHint** = [SimulationControlEditorHint](#)::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

15.808.1 Member Function Documentation

15.808.1.1 `getValue()`

`virtual std::string SimulationControlString::getValue ( ) const [inline], [override], [virtual]`
Implements [SimulationResponse](#).

15.808.1.2 `setValue()`

`virtual void SimulationControlString::setValue (`
 `std::string value,`
 `bool remove = false ) [inline], [override], [virtual]`
Implements [SimulationControl](#).

15.808.1.3 `validateProposedValue()`

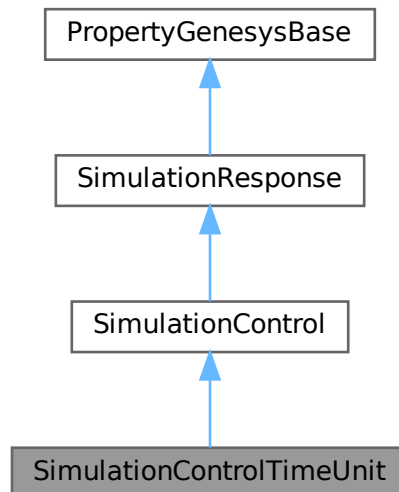
`virtual bool SimulationControlString::validateProposedValue (`
 `const std::string & value,`
 `std::string & errorMessage ) const [inline], [override], [virtual]`
Reimplemented from [SimulationControl](#).

The documentation for this class was generated from the following file:

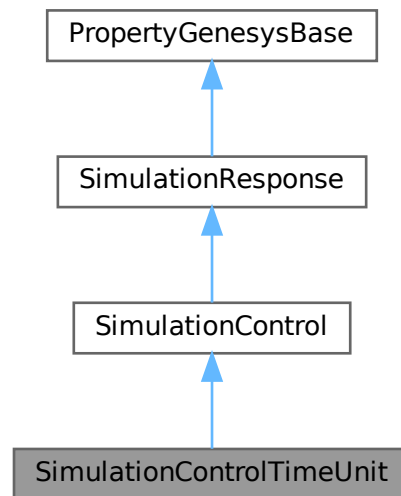
- `kernel/simulator/SimulationControlAndResponse.h`

15.809 SimulationControlTimeUnit Class Reference

Inheritance diagram for `SimulationControlTimeUnit`:



Collaboration diagram for SimulationControlTimeUnit:



Public Member Functions

- **SimulationControlTimeUnit** (GetterTimeUnit getter, SetterTimeUnit setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- virtual std::string [getValue](#) () const override
- virtual void [setValue](#) (std::string value, bool remove=false) override

Public Member Functions inherited from [SimulationControl](#)

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string [show](#) () const
- bool [isReadOnly](#) () const
- virtual [List](#)< [SimulationControl](#) * > * [getChildSimulationControls](#) (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool [hasObjectInstance](#) () const
- virtual bool [ensureObjectInstance](#) ()
- virtual bool [isModelDataDefinitionReference](#) () const
- virtual [ModelDataDefinition](#) * [getReferencedModelDataDefinition](#) () const
- virtual bool [validateProposedValue](#) (const std::string &value, std::string &errorMessage) const
- virtual [SimulationControlEditorHint](#) [preferredEditorHint](#) () const
- void [setPreferredEditorHint](#) ([SimulationControlEditorHint](#) hint)
- virtual [ModelDataDefinition](#) * [getListElementModelDataDefinition](#) (int index) const
- virtual bool [supportsInlineExpansion](#) () const
- virtual bool [supportsListEditor](#) () const
- virtual bool [supportsExistingObjectSelection](#) () const
- virtual bool [supportsObjectCreation](#) () const
- virtual bool [supportsNewListElementCreation](#) () const
- virtual [List](#)< std::string > * [getCreatableListElementTypes](#) ()
- virtual std::string [getCurrentListElementType](#) () const

- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual [List](#)< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual [List](#)< std::string > * **getStrValues** ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- [SimulationControlEditorHint](#) **_preferredEditorHint** = [SimulationControlEditorHint::Default](#)

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

15.809.1 Member Function Documentation

15.809.1.1 `getValue()`

```
virtual std::string SimulationControlTimeUnit::getValue ( ) const [inline], [override], [virtual]
```

Implements [SimulationResponse](#).

15.809.1.2 `setValue()`

```
virtual void SimulationControlTimeUnit::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

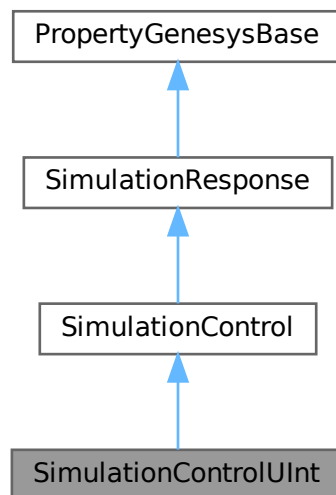
Implements [SimulationControl](#).

The documentation for this class was generated from the following file:

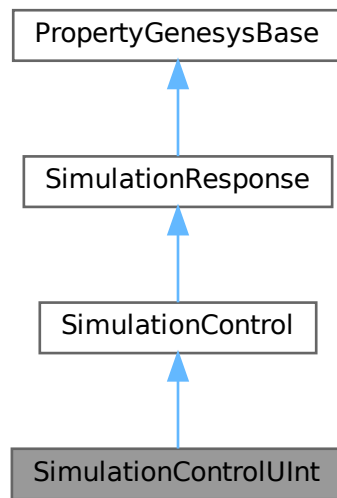
- `kernel/simulator/SimulationControlAndResponse.h`

15.810 SimulationControlUInt Class Reference

Inheritance diagram for SimulationControlUInt:



Collaboration diagram for SimulationControlUInt:



Public Member Functions

- **SimulationControlUInt** (GetterUInt getter, SetterUInt setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- virtual std::string **getValue** () const override
- virtual void **setValue** (std::string value, bool remove=false) override

Public Member Functions inherited from **SimulationControl**

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual List< SimulationControl * > * **getChildSimulationControls** (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool **hasObjectInstance** () const
- virtual bool **ensureObjectInstance** ()
- virtual bool **isModelDataDefinitionReference** () const
- virtual ModelDataDefinition * **getReferencedModelDataDefinition** () const
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const
- virtual SimulationControlEditorHint **preferredEditorHint** () const
- void **setPreferredEditorHint** (SimulationControlEditorHint hint)
- virtual ModelDataDefinition * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsInlineExpansion** () const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsExistingObjectSelection** () const
- virtual bool **supportsObjectCreation** () const
- virtual bool **supportsNewListElementCreation** () const
- virtual List< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)

- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual [List](#)< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual [List](#)< std::string > * **getStrValues** ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- [SimulationControlEditorHint](#) **_preferredEditorHint** = [SimulationControlEditorHint](#)::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

15.810.1 Member Function Documentation

15.810.1.1 `getValue()`

`virtual std::string SimulationControlUInt::getValue ( ) const [inline], [override], [virtual]`
Implements [SimulationResponse](#).

15.810.1.2 `setValue()`

`virtual void SimulationControlUInt::setValue (`
 `std::string value,`
 `bool remove = false ) [inline], [override], [virtual]`

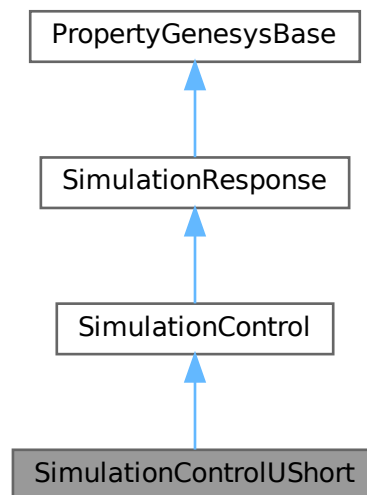
Implements [SimulationControl](#).

The documentation for this class was generated from the following file:

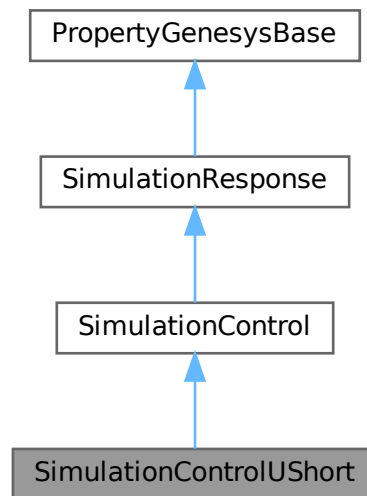
- `kernel/simulator/SimulationControlAndResponse.h`

15.811 SimulationControlUShort Class Reference

Inheritance diagram for SimulationControlUShort:



Collaboration diagram for SimulationControlUShort:



Public Member Functions

- **SimulationControlUShort** (GetterUShort getter, SetterUShort setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- virtual std::string **getValue** () const override
- virtual void **setValue** (std::string value, bool remove=false) override

Public Member Functions inherited from **SimulationControl**

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual List< SimulationControl * > * **getChildSimulationControls** (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool **hasObjectInstance** () const
- virtual bool **ensureObjectInstance** ()
- virtual bool **isModelDataDefinitionReference** () const
- virtual ModelDataDefinition * **getReferencedModelDataDefinition** () const
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const
- virtual SimulationControlEditorHint **preferredEditorHint** () const
- void **setPreferredEditorHint** (SimulationControlEditorHint hint)
- virtual ModelDataDefinition * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsInlineExpansion** () const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsExistingObjectSelection** () const
- virtual bool **supportsObjectCreation** () const
- virtual bool **supportsNewListElementCreation** () const
- virtual List< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const

- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual [List](#)< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual [List](#)< std::string > * **getStrValues** ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- [SimulationControlEditorHint](#) **_preferredEditorHint** = [SimulationControlEditorHint](#)::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

15.811.1 Member Function Documentation

15.811.1.1 `getValue()`

virtual std::string SimulationControlUShort::getValue () const [inline], [override], [virtual]
Implements [SimulationResponse](#).

15.811.1.2 `setValue()`

virtual void SimulationControlUShort::setValue (
std::string value,
bool remove = false) [inline], [override], [virtual]

Implements [SimulationControl](#).

The documentation for this class was generated from the following file:

- kernel/simulator/SimulationControlAndResponse.h

15.812 SimulationEvent Class Reference

```
#include <kernel/simulator/OnEventManager.h>
```

Public Member Functions

- void **setSimulatedTime** (double simulatedTime)
- double **getSimulatedTime** () const
- void **setStopRequested** (bool stopRequested)
- bool **isStopRequested** () const
- void **setPauseRequested** (bool pauseRequested)
- bool **isPauseRequested** () const
- void **setPaused** (bool Paused)
- bool **isPaused** () const
- void **setRunning** (bool Running)
- bool **isRunning** () const
- void **setCustomObject** (void *customObject)
- void * **getCustomObject** () const
- void **setCurrentReplicationNumber** (unsigned int currentReplicationNumber)
- unsigned int **getCurrentReplicationNumber** () const
- void **setCurrentEvent** ([Event](#) *currentEvent)
- [Event](#) * **getCurrentEvent** () const
- void **setEntityCreated** ([Entity](#) *entityCreated)
- [Entity](#) * **getEntityCreated** () const
- void **setDestinationComponent** ([ModelComponent](#) *destinationComponent)
- [ModelComponent](#) * **getDestinationComponent** () const
- void **setEntityMoveTimeDelay** (double entityMoveTimeDelay)
- double **getEntityMoveTimeDelay** () const

Friends

- class **ModelSimulation**

15.812.1 Detailed Description

Stores an event that happened on a specific replication

The documentation for this class was generated from the following file:

- kernel/simulator/OnEventManager.h

15.813 SimulationEventController Class Reference

Controller for simulation event callbacks, UI updates, and handler wiring.

```
#include <applications/gui/genesys/controllers/SimulationEventController.h>
```

Classes

- struct [Callbacks](#)

Public Member Functions

- **SimulationEventController** ([Simulator](#) *simulator, [ModelGraphicsScene](#) *scene, [ModelGraphicsView](#) *graphicsView, [QLabel](#) *replicationLabel, [QProgressBar](#) *simulationProgressBar, [QTableWidget](#) *simulationEventsTable, [QTableWidget](#) *entitiesTable, [QTableWidget](#) *variablesTable, [QTextEdit](#) *simulationText, [QTextEdit](#) *reportsText, [QTabWidget](#) *centralTabWidget, [QAction](#) *activateGraphicalSimulation, [QAction](#) *animationEnabledAction, bool *modelChecked, int tabCentralReportsIndex, [Callbacks](#) callbacks)
Creates the simulation-event bridge used by [MainWindow](#) compatibility handlers.
- void **onModelCheckSuccessHandler** ([ModelEvent](#) *re) const
Handles model-check success and updates delegated GUI state.
- void **onReplicationStartHandler** ([SimulationEvent](#) *re) const
Handles replication-start event and updates simulation/report panes.
- void **onSimulationStartHandler** ([SimulationEvent](#) *re) const
Handles simulation-start event and resets run-scoped UI state.
- void **onSimulationPausedHandler** ([SimulationEvent](#) *re) const
Handles simulation pause event and refreshes action availability.
- void **onSimulationResumeHandler** ([SimulationEvent](#) *re) const
Handles simulation resume event and resumes graphical simulation synchronization.
- void **onSimulationEndHandler** ([SimulationEvent](#) *re) const
Handles simulation end event and performs delegated cleanup/refresh.
- void **onReplicationEndHandler** ([SimulationEvent](#) *re) const
Handles replication end event and forces progress display completion.
- void **onProcessEventHandler** ([SimulationEvent](#) *re) const
Handles process-event updates for progress tables and scene refresh hooks.
- void **onEntityCreateHandler** ([SimulationEvent](#) *re) const
Handles entity-create events for compatibility callback wiring.
- void **onEntityRemoveHandler** ([SimulationEvent](#) *re) const
Handles entity-remove events for compatibility callback wiring.
- void **onMoveEntityEvent** ([SimulationEvent](#) *re) const
Handles entity-move event updates used by graphical simulation animation.
- void **onAfterProcessEvent** ([SimulationEvent](#) *re) const
Handles after-process event updates used by animation/frame synchronization.
- void **setOnEventHandlers** ([MainWindow](#) *owner) const
Registers kernel on-event callbacks through [MainWindow](#) compatibility façade wrappers.

15.813.1 Detailed Description

Controller for simulation event callbacks, UI updates, and handler wiring.

This controller concentrates event-reaction logic previously embedded in [MainWindow](#) and keeps compatibility by exposing methods invoked from [MainWindow](#) wrappers. It also provides the event-registration entry point used to wire kernel [OnEventManager](#) callbacks to those wrappers without changing the existing signal/slot surface.

Responsibilities:

- react to model-check, replication, simulation, process, and entity events;
- update progress/status widgets and debug tables through injected UI dependencies;

- trigger delegated refresh callbacks for actions, debug panes, and scene synchronization;
- register handlers through [MainWindow](#) compatibility façade ownership.

Boundaries:

- it does not own [MainWindow](#) or widget lifetime;
- it does not execute simulation commands (handled by command controllers);
- it does not perform persistence/export/model-language synchronization.

The documentation for this class was generated from the following files:

- applications/gui/genesys/controllers/SimulationEventController.h
- applications/gui/genesys/controllers/SimulationEventController.cpp

15.814 SimulationExperiment Class Reference

Represents a higher-level simulation experiment definition.

```
#include <kernel/simulator/SimulationExperiment.h>
```

15.814.1 Detailed Description

Represents a higher-level simulation experiment definition.

Historically, GenESyS started with experiment control concentrated in [ModelSimulation](#), mainly through replication count and replication length.

[SimulationExperiment](#) was introduced later as part of an unfinished effort to move richer experiment-design concepts into the simulation kernel, including experiment sets, scenarios and [Process](#) Analyzer-like workflows.

This class is still under development and should currently be treated as a placeholder for future experiment-layer behavior.

The documentation for this class was generated from the following files:

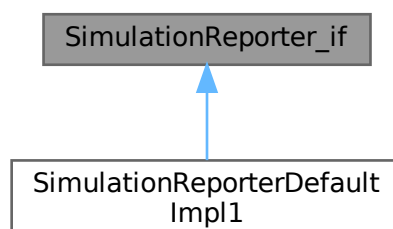
- kernel/simulator/SimulationExperiment.h
- kernel/simulator/SimulationExperiment.cpp

15.815 SimulationReporter_if Class Reference

Interface for reporting simulation results and experiment-facing outputs.

```
#include <kernel/simulator/SimulationReporter_if.h>
```

Inheritance diagram for SimulationReporter_if:



Public Member Functions

- virtual void [showReplicationStatistics](#) ()=0
showReplicationStatistics
- virtual void [showSimulationStatistics](#) ()=0
showSimulationStatistics
- virtual void [showSimulationResponses](#) ()=0
showSimulationResponses
- virtual void [showSimulationControls](#) ()=0
showSimulationControls

15.815.1 Detailed Description

Interface for reporting simulation results and experiment-facing outputs.

Reporter implementations define how replication/simulation summaries, responses and controls are presented to users or external consumers.

15.815.2 Member Function Documentation**15.815.2.1 showReplicationStatistics()**

```
virtual void SimulationReporter_if::showReplicationStatistics ( ) [pure virtual]
```

`showReplicationStatistics`

Shows detailed statistics for the current/finished replication.

Implemented in [SimulationReporterDefaultImpl1](#).

15.815.2.2 showSimulationControls()

```
virtual void SimulationReporter_if::showSimulationControls ( ) [pure virtual]
```

`showSimulationControls`

Shows monitored/configurable simulation controls (inputs).

Implemented in [SimulationReporterDefaultImpl1](#).

15.815.2.3 showSimulationResponses()

```
virtual void SimulationReporter_if::showSimulationResponses ( ) [pure virtual]
```

`showSimulationResponses`

Shows configured simulation responses (outputs) for experimental analysis.

Implemented in [SimulationReporterDefaultImpl1](#).

15.815.2.4 showSimulationStatistics()

```
virtual void SimulationReporter_if::showSimulationStatistics ( ) [pure virtual]
```

`showSimulationStatistics`

Shows aggregated statistics for the full simulation.

Implemented in [SimulationReporterDefaultImpl1](#).

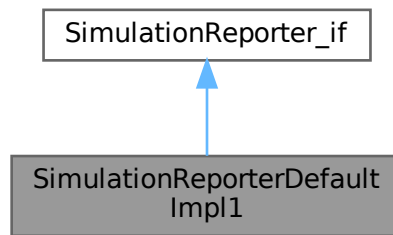
The documentation for this class was generated from the following file:

- `kernel/simulator/SimulationReporter_if.h`

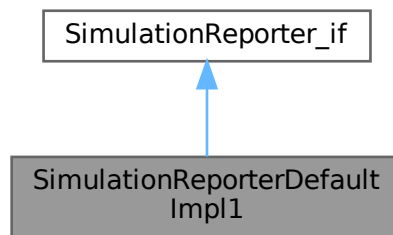
15.816 SimulationReporterDefaultImpl1 Class Reference

```
#include <kernel/simulator/SimulationReporterDefaultImpl1.h>
```

Inheritance diagram for SimulationReporterDefaultImpl1:



Collaboration diagram for SimulationReporterDefaultImpl1:



Public Member Functions

- **SimulationReporterDefaultImpl1** ([ModelSimulation](#) *simulation, [Model](#) *model, [List](#)< [ModelDataDefinition](#) * > *statsCountersSimulation)
- virtual void [showReplicationStatistics](#) () override
showReplicationStatistics
- virtual void [showSimulationStatistics](#) () override
showSimulationStatistics
- virtual void [showSimulationResponses](#) () override
showSimulationResponses
- virtual void [showSimulationControls](#) () override
showSimulationControls

15.816.1 Detailed Description

Class that implements [SimulationReporter_if](#) interface and is responsible for building and showing replication and simulation reports

15.816.2 Member Function Documentation

15.816.2.1 showReplicationStatistics()

```
void SimulationReporterDefaultImpl1::showReplicationStatistics ( ) [override], [virtual]
```

showReplicationStatistics

Shows detailed statistics for the current/finished replication.

Implements [SimulationReporter_if](#).

15.816.2.2 showSimulationControls()

```
void SimulationReporterDefaultImpl1::showSimulationControls ( ) [override], [virtual]
```

showSimulationControls

Shows monitored/configurable simulation controls (inputs).

Implements [SimulationReporter_if](#).

15.816.2.3 showSimulationResponses()

```
void SimulationReporterDefaultImpl1::showSimulationResponses ( ) [override], [virtual]
```

showSimulationResponses

Shows configured simulation responses (outputs) for experimental analysis.

Implements [SimulationReporter_if](#).

15.816.2.4 showSimulationStatistics()

```
void SimulationReporterDefaultImpl1::showSimulationStatistics ( ) [override], [virtual]
```

showSimulationStatistics

Shows aggregated statistics for the full simulation.

Implements [SimulationReporter_if](#).

The documentation for this class was generated from the following files:

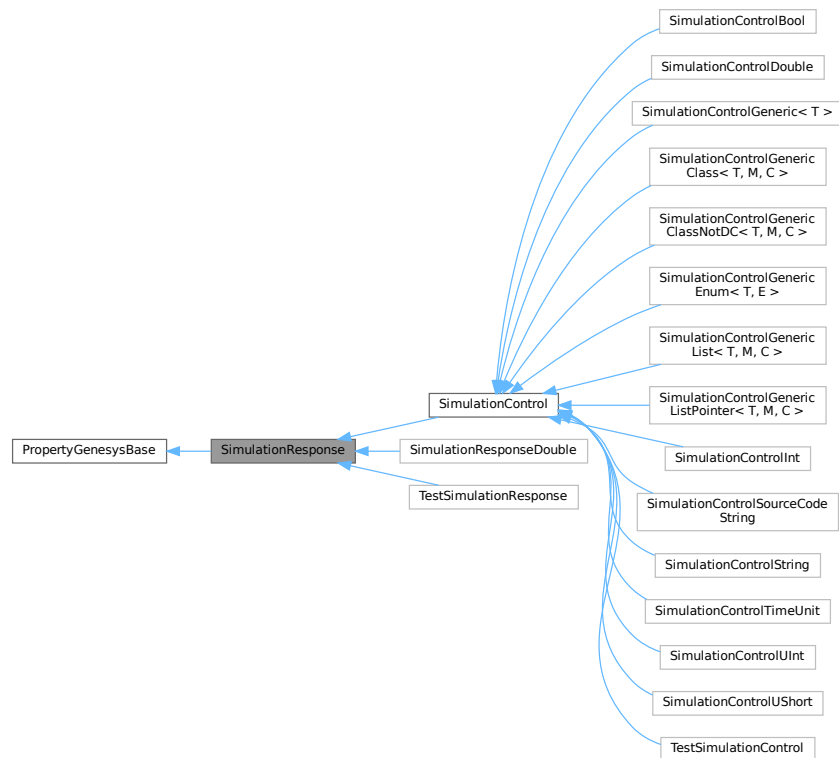
- kernel/simulator/SimulationReporterDefaultImpl1.h
- kernel/simulator/SimulationReporterDefaultImpl1.cpp

15.817 SimulationResponse Class Reference

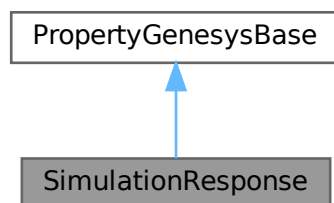
Read-only kernel-side simulation response abstraction.

```
#include <kernel/simulator/SimulationControlAndResponse.h>
```

Inheritance diagram for `SimulationResponse`:



Collaboration diagram for `SimulationResponse`:



Public Member Functions

- **`SimulationResponse`** (`std::string` className, `std::string` elementName, `std::string` propertyName, `std::string` whatsThis="", `bool` isList=false, `bool` isClass=false, `bool` isEnum=false)
- `std::string` **`show`** () const
- virtual `std::string` **`getValue`** () const =0
- virtual `List< std::string > *` **`getStrValues`** ()

Public Member Functions inherited from `PropertyGenesysBase`

- **`PropertyGenesysBase`** (`std::string` className, `std::string` elementName, `std::string` propertyName, `std::string` whatsThis="")

- ```

::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)

```
- `std::string whatsThis () const`
  - `std::string getClassname () const`
  - `std::string getElementName () const`
  - `std::string getName () const`
  - `std::string propertyType () const`
  - `bool getIsList () const`
  - `bool getIsClass () const`
  - `bool getIsEnum () const`
  - `void setWhatsThis (const std::string text)`
  - `void setName (std::string name)`

#### Additional Inherited Members

#### Protected Attributes inherited from [PropertyGenesysBase](#)

- `std::string _elementName`
- `std::string _className`
- `std::string _propertyName`
- `std::string _propertyType`
- `std::string _whatsThis`
- `bool _isList`
- `bool _isClass`
- `bool _isEnum`

#### 15.817.1 Detailed Description

Read-only kernel-side simulation response abstraction.

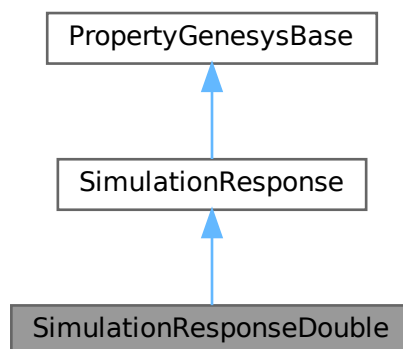
This is the intended kernel-level base for generic getter-based access to observable model attributes, statistics and experiment responses.

The documentation for this class was generated from the following file:

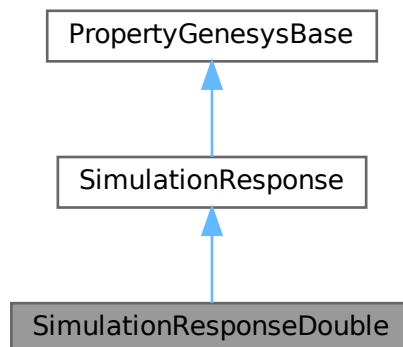
- `kernel/simulator/SimulationControlAndResponse.h`

## 15.818 SimulationResponseDouble Class Reference

Inheritance diagram for SimulationResponseDouble:



Collaboration diagram for SimulationResponseDouble:



### Public Member Functions

- **SimulationResponseDouble** (GetterDouble getter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- virtual std::string [getValue](#) () const override

### Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual [List](#)< std::string > \* **getStrValues** ()

### Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

### Additional Inherited Members

### Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **\_elementName**
- std::string **\_className**
- std::string **\_propertyName**
- std::string **\_propertyType**



- std::string **\_whatsThis**
- bool **\_isList**
- bool **\_isClass**
- bool **\_isEnum**

## 15.818.1 Member Function Documentation

### 15.818.1.1 getValue()

virtual std::string SimulationResponseDouble::getValue ( ) const [inline], [override], [virtual]  
Implements [SimulationResponse](#).

The documentation for this class was generated from the following file:

- kernel/simulator/SimulationControlAndResponse.h

## 15.819 SimulationResultsDataset Struct Reference

### Public Member Functions

- std::string [formatKindName](#) ( ) const
- bool [isRecordFormat](#) ( ) const
- std::vector< double > [values](#) ( ) const
- std::vector< unsigned int > [replications](#) ( ) const
- std::vector< double > [valuesForReplication](#) (unsigned int replication) const

### Public Attributes

- std::string **datasetName**
- std::string **randomVariableName**
- std::string **description**
- std::string **variableType** = "Continuous numeric"
- std::string **source**
- std::string **sourceDescription**
- std::string **expression**
- std::string **expressionName**
- std::string **formatVersion**
- SimulationResultsDatasetFormat **formatKind** = SimulationResultsDatasetFormat::Unknown
- bool **recordFile** = false
- bool **timeDependent** = false
- std::vector< [SimulationResultsObservation](#) > **observations**
- std::vector< std::string > **previewLines**

## 15.819.1 Member Function Documentation

### 15.819.1.1 formatKindName()

std::string SimulationResultsDataset::formatKindName ( ) const  
Returns the display name for the detected dataset format.

### 15.819.1.2 isRecordFormat()

bool SimulationResultsDataset::isRecordFormat ( ) const  
Returns true when this dataset was imported from a Genesys [Record](#) file.

### 15.819.1.3 replications()

std::vector< unsigned int > SimulationResultsDataset::replications ( ) const  
Returns the distinct replication identifiers present in the observations.

#### 15.819.1.4 values()

`std::vector< double > SimulationResultsDataset::values ( ) const`  
Returns the observation values without replication or time metadata.

#### 15.819.1.5 valuesForReplication()

`std::vector< double > SimulationResultsDataset::valuesForReplication ( unsigned int replication ) const`

Returns all observation values for a single replication.

The documentation for this struct was generated from the following files:

- tools/Statistics/SimulationResultsDataset.h
- tools/Statistics/SimulationResultsDataset.cpp

## 15.820 SimulationResultsDatasetParser Class Reference

### Static Public Member Functions

- static bool [loadFromTextFile](#) (const std::string &fileName, [SimulationResultsDataset](#) \*dataset, std::string \*errorMessage=nullptr)

### 15.820.1 Member Function Documentation

#### 15.820.1.1 loadFromTextFile()

```
bool SimulationResultsDatasetParser::loadFromTextFile (
 const std::string & fileName,
 SimulationResultsDataset * dataset,
 std::string * errorMessage = nullptr) [static]
```

Loads a simulation-results dataset from a text file.

The parser intentionally detects formats in this order: GUI tabular, Genesys [Record](#), then raw numeric. Once a format is detected, no residual heuristic reinterprets the same file as another format.

The documentation for this class was generated from the following files:

- tools/Statistics/SimulationResultsDataset.h
- tools/Statistics/SimulationResultsDataset.cpp

## 15.821 SimulationResultsObservation Struct Reference

### Public Attributes

- unsigned int **replication** = 1
- double **time** = 0.0
- bool **hasTime** = false
- double **value** = 0.0
- unsigned int **sourceLine** = 0

The documentation for this struct was generated from the following file:

- tools/Statistics/SimulationResultsDataset.h

## 15.822 SimulationScenario Class Reference

Represents a candidate simulation scenario within the unfinished experiment layer.

```
#include <kernel/simulator/SimulationScenario.h>
```

## Public Member Functions

- bool **startSimulation** ([Simulator](#) \*sim, std::string &errorMessage)
- std::list< std::pair< std::string, double > \* > \* [getResponseValues](#) () const
- std::list< std::pair< std::string, double > \* > \* **getControlValues** () const
- double **getResponseValue** (const std::string &responseName)
- void **setModelFilename** (std::string \_modelFilename)
- std::string **getModelFilename** () const
- void **setScenarioName** (std::string \_name)
- std::string **getScenarioName** () const
- void **setScenarioDescription** (std::string \_scenarioDescription)
- std::string **getScenarioDescription** () const
- std::list< std::string > \* **getSelectedControls** () const
- double **getControlValue** (const std::string &controlName)
- std::list< std::string > \* **getSelectedResponses** () const
- void **setSelectedControls** (std::list< std::string > \*selectedControls)
- void **setControl** (std::string name, double value) const

### 15.822.1 Detailed Description

Represents a candidate simulation scenario within the unfinished experiment layer.

Represents a scenario where a specific model (defined my ModelFilename) will be simulated. To each scenario will be associated a set of [SimulationControl](#) and [SimulationResponse](#), and their values are set to the scenario by the ProcessAnalyser.

The original GenESyS execution flow relied on [ModelSimulation](#) for the basic experimental controls already supported by the kernel.

[SimulationScenario](#) was introduced later to support richer scenario-based experimentation, potentially including alternative models, modified controls and future [Process](#) Analyzer-like workflows.

This class is still under development and should currently be treated as an evolving kernel-side abstraction for experiment scenarios.

### 15.822.2 Member Function Documentation

#### 15.822.2.1 getResponseValues()

```
std::list< std::pair< std::string, double > * > * SimulationScenario::getResponseValues ()
const
```

The final result of the simulationScenario

The documentation for this class was generated from the following files:

- kernel/simulator/SimulationScenario.h
- kernel/simulator/SimulationScenario.cpp

## 15.823 SimulatorSessionService::SimulationStatusResult Struct Reference

Captures the current simulation execution state.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

## Public Attributes

- bool **success** = false
- bool **invalidToken** = false
- bool **hasCurrentModel** = false
- bool **isRunning** = false
- bool **isPaused** = false
- double **simulatedTime** = 0.0

- unsigned int **currentReplicationNumber** = 0
- unsigned int **numberOfReplications** = 0
- double **replicationLength** = 0.0
- double **warmUpPeriod** = 0.0
- bool **pauseOnEvent** = false
- bool **pauseOnReplication** = false
- bool **initializeStatistics** = false
- bool **initializeSystem** = false

### 15.823.1 Detailed Description

Captures the current simulation execution state.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

## 15.824 Simulator Class Reference

```
#include <kernel/simulator/Simulator.h>
```

### Public Member Functions

- [Simulator](#) ()
- std::string [getVersion](#) () const  
*getVersion*
- unsigned int [getVersionNumber](#) () const  
*getVersionNumber*
- std::string [getName](#) () const  
*getName*
- [LicenceManager](#) \* [getLicenceManager](#) () const  
*getLicence*
- [PluginManager](#) \* [getPluginManager](#) () const  
*getPlugins*
- [ModelManager](#) \* [getModelManager](#) () const  
*getModelManager*
- [TraceManager](#) \* [getTraceManager](#) () const  
*getTraceManager*
- [ParserManager](#) \* [getParserManager](#) () const  
*getParserManager*
- [ExperimentManager](#) \* [getExperimentManager](#) () const  
*getExperimentManager*

### Friends

- class **PluginManager**

### 15.824.1 Detailed Description

The main class of the Genesys KERNEL simulation. It gives access to simulation models and tools. Simulation is the top level class and is supposed to be available to application as a dynamic linked library.

## 15.824.2 Constructor & Destructor Documentation

### 15.824.2.1 Simulator()

```
Simulator::Simulator ()
```

[Simulator](#) constructor basically constructs inner classes (managers) and shows basic information on the console. This is the only method on the entire simulator that directly plots something to the output

## 15.824.3 Member Function Documentation

### 15.824.3.1 getExperimentManager()

```
ExperimentManager * Simulator::getExperimentManager () const
getExperimentManager
```

Returns

### 15.824.3.2 getLicenceManager()

```
LicenceManager * Simulator::getLicenceManager () const
getLicence
```

Returns

### 15.824.3.3 getModelManager()

```
ModelManager * Simulator::getModelManager () const
getModelManager
```

Returns

### 15.824.3.4 getName()

```
std::string Simulator::getName () const
getName
```

Returns

### 15.824.3.5 getParserManager()

```
ParserManager * Simulator::getParserManager () const
getParserManager
```

Returns

### 15.824.3.6 getPluginManager()

```
PluginManager * Simulator::getPluginManager () const
getPlugins
```

Returns

**15.824.3.7 getTraceManager()**

```
TraceManager * Simulator::getTraceManager () const
getTraceManager
```

Returns

**15.824.3.8 getVersion()**

```
std::string Simulator::getVersion () const
getVersion
```

Returns

**15.824.3.9 getVersionNumber()**

```
unsigned int Simulator::getVersionNumber () const
getVersionNumber
```

Returns

The documentation for this class was generated from the following files:

- kernel/simulator/Simulator.h
- kernel/simulator/Simulator.cpp
- tests/unit/test\_simulator\_support.cpp
- tests/unit/test\_support\_experimentmanager.cpp
- tests/unit/test\_support\_modelmanager.cpp

**15.825 SimulatorFacade Class Reference**

High-level non-owning facade over [Simulator](#) and its main composed services.

```
#include <kernel/simulator/SimulatorFacade.h>
```

**Public Member Functions**

- **SimulatorFacade** ([Simulator](#) \*simulator)
- std::string **getVersion** () const
- unsigned int **getVersionNumber** () const
- std::string **getName** () const
- const std::string **licenceShow** () const
- const std::string **licenceShowLimits** () const
- const std::string **licenceShowActivationCode** () const
- bool **licenceLookforActivationCode** ()
- bool **licenceInsertActivationCode** ()
- void **licenceRemoveActivationCode** ()
- unsigned int **licenceGetModelComponentsLimit** ()
- unsigned int **licenceGetModelDatasLimit** ()
- unsigned int **licenceGetEntityLimit** ()
- unsigned int **licenceGetHostsLimit** ()
- unsigned int **licenceGetThreadsLimit** ()
- std::string **showPlugins** ()
- [List](#)< [Plugin](#) \* > \* **completePluginsFieldsAndTemplates** ()

- bool **pluginCheck** (const std::string &dynamicLibraryFilename)
- [SystemDependencyCheckResult](#) **pluginCheckSystemDependencies** (const std::string &dynamicLibraryFilename)
- [List](#)< std::string > \* **discoverPluginFileNames** () const
- [List](#)< [PluginLoadIssue](#) > \* **getPluginLoadIssues** () const
- void **clearPluginLoadIssues** ()
- void **clearPluginLoadIssue** (const std::string &dynamicLibraryFilename)
- [Plugin](#) \* **insertPlugin** (const std::string &dynamicLibraryFilename)
- [Plugin](#) \* **insertPlugin** (const std::string &dynamicLibraryFilename, const [PluginInsertionOptions](#) &options)
- bool **removePlugin** (const std::string &dynamicLibraryFilename)
- bool **removePlugin** ([Plugin](#) \*plugin)
- [Plugin](#) \* **findPlugin** (const std::string &pluginTypeName)
- std::vector< std::string > **getDataDefinitionPluginTypeNames** () const
- std::string **pluginSourceIncludePathFor** (const std::string &pluginTypeName)
- [List](#)< [Plugin](#) \* > \* **autoInsertPlugins** (const std::string &pluginsListFilename, const bool lookForPluginsIfFilenameNotFound=true)
- [List](#)< [Plugin](#) \* > \* **autoInsertPlugins** (const std::string &pluginsListFilename, const bool lookForPluginsIfFilenameNotFound, const [PluginInsertionOptions](#) &options)
- [List](#)< [Plugin](#) \* > \* **autoInsertPlugins** ()
- [List](#)< [Plugin](#) \* > \* **autoInsertPlugins** (const [PluginInsertionOptions](#) &options)
- [Plugin](#) \* **firstPlugin** ()
- [Plugin](#) \* **nextPlugin** ()
- [Plugin](#) \* **lastPlugin** ()
- unsigned int **pluginCount** ()
- [Plugin](#) \* **getPluginAtRank** (unsigned int rank)
- [ModelDataDefinition](#) \* **newInstance** (const std::string &pluginTypeName, [Model](#) \*model, std::string name="")
- template<typename T >  
T \* **newInstance** ([Model](#) \*model, std::string name="")
- [Model](#) \* **newModel** ()
- void **insertModel** ([Model](#) \*model)
- void **removeModel** ([Model](#) \*model)
- bool **setCurrentModel** ([Model](#) \*model)
- bool **saveModel** (std::string filename)
- [Model](#) \* **loadModel** (std::string filename)
- [Model](#) \* **createModelFromLanguage** (std::string modelSpecification)
- unsigned int **modelCount** ()
- std::vector< [Model](#) \* > **models** () const
- bool **hasModel** ([Model](#) \*model) const
- int **indexOfModel** ([Model](#) \*model) const
- [Model](#) \* **modelAt** (unsigned int index) const
- int **currentModelIndex** () const
- [Model](#) \* **firstModel** ()
- [Model](#) \* **lastModel** ()
- [Model](#) \* **currentModel** ()
- [Model](#) \* **nextModel** ()
- [Model](#) \* **previousModel** ()
- bool **canGoNextModel** () const
- bool **canGoPreviousModel** () const
- void **addTraceHandler** (traceListener traceListener)
- void **addTraceReportHandler** (traceListener traceReportListener)
- void **addTraceSimulationHandler** (traceSimulationListener traceSimulationListener)
- void **addTraceErrorHandler** (traceErrorListener traceErrorListener)
- template<typename Class >  
void **addTraceHandler** (Class \*object, void(Class::\*function)([TraceEvent](#)))

- `template<typename Class >`  
`void addTraceErrorHandler (Class *object, void(Class::*function)(TraceErrorEvent))`
- `template<typename Class >`  
`void addTraceReportHandler (Class *object, void(Class::*function)(TraceEvent))`
- `template<typename Class >`  
`void addTraceSimulationHandler (Class *object, void(Class::*function)(TraceSimulationEvent))`
- `void addTraceSimulationExceptionRuleModelData (void *thisobject)`
- `void beginTraceShutdown ()`
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed, bool showAnyway=false)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed, bool showAnyway=false)`
- `List< std::string > * traceGetErrorMessages () const`
- `void setTraceLevel (TraceManager::Level traceLevel)`
- `TraceManager::Level getTraceLevel () const`
- `void setTraceSimulationRuleAllAllowed (bool traceSimulationRuleAllAllowed)`
- `bool isTraceSimulationRuleAllAllowed () const`
- `ParserManager::GenerateNewParserResult parserGenerateNewParser (ParserChangesInformation *changes)`
- `bool parserConnectNewParser (ParserManager::NewParser newParser)`
- `SimulationExperiment * newSimulationExperiment ()`
- `void insertSimulationExperiment (SimulationExperiment *experiment)`
- `void removeSimulationExperiment (SimulationExperiment *experiment)`
- `void setCurrentSimulationExperiment (SimulationExperiment *experiment)`
- `bool saveSimulationExperiment (std::string filename)`
- `bool loadSimulationExperiment (std::string filename)`
- `unsigned int simulationExperimentCount ()`
- `SimulationExperiment * firstSimulationExperiment ()`
- `SimulationExperiment * currentSimulationExperiment ()`
- `SimulationExperiment * nextSimulationExperiment ()`
- `List< SimulationExperiment * > * getSimulationExperiments () const`
- `bool modelSave (std::string filename)`
- `bool modelLoad (std::string filename)`
- `bool modelCheck ()`
- `void modelClear ()`
- `std::string modelShowLanguage ()`
- `void modelShow ()`
- `bool modelInsert (ModelDataDefinition *elemOrComp)`
- `void modelRemove (ModelDataDefinition *elemOrComp)`
- `std::list< ModelDataDefinition * > modelCollectDataDefinitionsRemovedWith (const std::list< ModelDataDefinition * > &roots) const`
- `Entity * modelCreateEntity (std::string name, bool insertIntoModel=true)`
- `void modelRemoveEntity (Entity *entity)`
- `void modelSendEntityToComponent (Entity *entity, Connection *connection, double timeDelay=0.0)`
- `void modelSendEntityToComponent (Entity *entity, ModelComponent *component, double timeDelay=0.0, unsigned int componentinputPortNumber=0)`
- `double modelParseExpression (const std::string expression)`
- `double modelParseExpression (const std::string expression, bool &success, std::string &errorMessage)`
- `bool modelCheckExpression (const std::string expression, const std::string expressionName, std::string &errorMessage)`



- void **modelCheckReferencesToDataDefinitions** (std::string expression, std::map< std::string, std::list< std::string > \* > \*referencedDataDefinitions)
- Util::identification **modelGetId** () const
- bool **modelHasChanged** () const
- void **modelSetHasChanged** (bool hasChanged)
- [List](#)< [Event](#) \* > \* **modelGetFutureEvents** () const
- [List](#)< [SimulationControl](#) \* > \* **modelGetControls** () const
- [List](#)< [SimulationResponse](#) \* > \* **modelGetResponses** () const
- [Persistence\\_if](#) \* **modelGetPersistence** () const
- unsigned int **modelGetLevel** () const
- std::string **infoShow** ()
- void **infoSetName** (std::string name)
- std::string **infoGetName** () const
- void **infoSetAnalystName** (std::string analystName)
- std::string **infoGetAnalystName** () const
- void **infoSetDescription** (std::string description)
- std::string **infoGetDescription** () const
- void **infoSetProjectTitle** (std::string projectTitle)
- std::string **infoGetProjectTitle** () const
- void **infoSetVersion** (std::string version)
- std::string **infoGetVersion** () const
- void **infoLoadInstance** ([PersistenceRecord](#) \*fields)
- void **infoSaveInstance** ([PersistenceRecord](#) \*fields)
- bool **infoHasChanged** () const
- void **infoSetHasChanged** (bool hasChanged)
- std::string **simShow** ()
- void **simStart** ()
- void **simPause** ()
- void **simStep** ()
- void **simStop** ()
- void **simSetNumberOfReplications** (unsigned int numberOfReplications)
- unsigned int **simGetNumberOfReplications** () const
- void **simSetReplicationLength** (double replicationLength, [Util::TimeUnit](#) replicationLengthTimeUnit=[Util::TimeUnit::unknown](#))
- double **simGetReplicationLength** () const
- void **simSetReplicationLengthTimeUnit** ([Util::TimeUnit](#) replicationLengthTimeUnit)
- [Util::TimeUnit](#) **simGetReplicationLengthTimeUnit** () const
- void **simSetReplicationReportBaseTimeUnit** ([Util::TimeUnit](#) replicationReportBaseTimeUnit)
- [Util::TimeUnit](#) **simGetReplicationBaseTimeUnit** () const
- void **simSetWarmUpPeriod** (double warmUpPeriod, [Util::TimeUnit](#) warmUpPeriodTimeUnit=[Util::TimeUnit::unknown](#))
- double **simGetWarmUpPeriod** () const
- void **simSetWarmUpPeriodTimeUnit** ([Util::TimeUnit](#) warmUpPeriodTimeUnit)
- [Util::TimeUnit](#) **simGetWarmUpPeriodTimeUnit** () const
- void **simSetTerminatingCondition** (std::string terminatingCondition)
- std::string **simGetTerminatingCondition** () const
- void **simSetPauseOnEvent** (bool pauseOnEvent)
- bool **simIsPauseOnEvent** () const
- void **simSetStepByStep** (bool stepByStep)
- bool **simIsStepByStep** () const
- void **simSetInitializeStatistics** (bool initializeStatistics)
- bool **simIsInitializeStatistics** () const
- void **simSetInitializeSystem** (bool initializeSystem)
- bool **simIsInitializeSystem** () const
- void **simSetPauseOnReplication** (bool pauseBetweenReplications)

- bool **simIsPauseOnReplication** () const
- void **simSetReporter** ([SimulationReporter\\_if](#) \*simulationReporter)
- [SimulationReporter\\_if](#) \* **simGetReporter** () const
- double **simGetSimulatedTime** () const
- bool **simIsRunning** () const
- bool **simIsPaused** () const
- unsigned int **simGetCurrentReplicationNumber** () const
- void **simSetShowReportsAfterReplication** (bool showReportsAfterReplication)
- bool **simIsShowReportsAfterReplication** () const
- void **simSetShowReportsAfterSimulation** (bool showReportsAfterSimulation)
- bool **simIsShowReportsAfterSimulation** () const
- [List](#)< double > \* **simGetBreakpointsOnTime** () const
- [List](#)< [Entity](#) \* > \* **simGetBreakpointsOnEntity** () const
- [List](#)< [ModelComponent](#) \* > \* **simGetBreakpointsOnComponent** () const
- const [List](#)< [ModelDataDefinition](#) \* > \* **simGetSimulationStatisticsAggregates** () const
- void **simLoadInstance** ([PersistenceRecord](#) \*fields)
- void **simSaveInstance** ([PersistenceRecord](#) \*fields, bool saveDefaults)
- [Event](#) \* **simGetCurrentEvent** () const
- void **simSetShowSimulationResponsesInReport** (bool showSimulationResponsesInReport)
- bool **simIsShowSimulationResponsesInReport** () const
- void **simSetShowSimulationControlsInReport** (bool showSimulationControlsInReport)
- bool **simIsShowSimulationControlsInReport** () const
- bool **dataInsert** ([ModelDataDefinition](#) \*anElement)
- void **dataRemove** ([ModelDataDefinition](#) \*anElement)
- bool **dataInsert** (std::string datadefinitionTypename, [ModelDataDefinition](#) \*anElement)
- void **dataRemove** (std::string datadefinitionTypename, [ModelDataDefinition](#) \*anElement)
- bool **dataCheck** (std::string datadefinitionTypename, [ModelDataDefinition](#) \*anElement, std::string expressionName, std::string &errorMessage)
- bool **dataCheck** (std::string datadefinitionTypename, std::string elementName, std::string expressionName, bool mandatory, std::string &errorMessage)
- void **dataClear** ()
- [ModelDataDefinition](#) \* **dataGetDataDefinition** (std::string datadefinitionTypename, Util::identification id)
- [ModelDataDefinition](#) \* **dataGetDataDefinition** (std::string datadefinitionTypename, std::string name)
- unsigned int **dataGetNumberOfDataDefinitions** (std::string datadefinitionTypename)
- unsigned int **dataGetNumberOfDataDefinitions** ()
- int **dataGetRankOf** (std::string datadefinitionTypename, std::string name)
- std::list< std::string > **dataGetDataDefinitionClassnames** () const
- [List](#)< [ModelDataDefinition](#) \* > \* **dataGetDataDefinitionList** (std::string datadefinitionTypename) const
- void **dataShow** ()
- bool **dataHasChanged** () const
- void **dataSetHasChanged** (bool hasChanged)
- bool **componentInsert** ([ModelComponent](#) \*comp)
- void **componentRemove** ([ModelComponent](#) \*comp)
- [ModelComponent](#) \* **componentFind** (std::string name)
- [ModelComponent](#) \* **componentFind** (Util::identification id)
- void **componentClear** ()
- unsigned int **componentGetNumberOfComponents** ()
- std::list< [ModelComponent](#) \* >::iterator **componentBegin** ()
- std::list< [ModelComponent](#) \* >::iterator **componentEnd** ()
- [ModelComponent](#) \* **componentFront** ()
- [ModelComponent](#) \* **componentNext** ()
- bool **componentHasChanged** () const
- void **componentSetHasChanged** (bool hasChanged)
- std::list< [SourceModelComponent](#) \* > \* **componentGetSourceComponents** ()
- std::list< [ModelComponent](#) \* > \* **componentGetTransferInComponents** ()

- `std::list< ModelComponent * > * componentGetAllComponents ()` const
- `void eventAddOnModelCheckSucessHandler (modelEventHandler EventHandler)`
- `void eventAddOnModelLoadHandler (modelEventHandler EventHandler)`
- `void eventAddOnModelSaveHandler (modelEventHandler EventHandler)`
- `void eventAddOnReplicationStartHandler (simulationEventHandler EventHandler)`
- `void eventAddOnReplicationStepHandler (simulationEventHandler EventHandler)`
- `void eventAddOnReplicationEndHandler (simulationEventHandler EventHandler)`
- `void eventAddOnProcessEventHandler (simulationEventHandler EventHandler)`
- `void eventAddOnAfterProcessEventHandler (simulationEventHandler EventHandler)`
- `void eventAddOnEntityCreateHandler (simulationEventHandler EventHandler)`
- `void eventAddOnEntityMoveHandler (simulationEventHandler EventHandler)`
- `void eventAddOnEntityRemoveHandler (simulationEventHandler EventHandler)`
- `void eventAddOnSimulationStartHandler (simulationEventHandler EventHandler)`
- `void eventAddOnSimulationPausedHandler (simulationEventHandler EventHandler)`
- `void eventAddOnSimulationResumeHandler (simulationEventHandler EventHandler)`
- `void eventAddOnSimulationEndHandler (simulationEventHandler EventHandler)`
- `void eventAddOnBreakpointHandler (simulationEventHandler EventHandler)`
- `template<typename Class >`  
`void eventAddOnModelCheckSuccessHandler (Class *object, void(Class::*function)(ModelEvent *))`
- `template<typename Class >`  
`void eventAddOnModelLoadHandler (Class *object, void(Class::*function)(ModelEvent *))`
- `template<typename Class >`  
`void eventAddOnModelSaveHandler (Class *object, void(Class::*function)(ModelEvent *))`
- `template<typename Class >`  
`void eventAddOnReplicationStartHandler (Class *object, void(Class::*function)(SimulationEvent *))`
- `template<typename Class >`  
`void eventAddOnReplicationStepHandler (Class *object, void(Class::*function)(SimulationEvent *))`
- `template<typename Class >`  
`void eventAddOnReplicationEndHandler (Class *object, void(Class::*function)(SimulationEvent *))`
- `template<typename Class >`  
`void eventAddOnProcessEventHandler (Class *object, void(Class::*function)(SimulationEvent *))`
- `template<typename Class >`  
`void eventAddOnAfterProcessEventHandler (Class *object, void(Class::*function)(SimulationEvent *))`
- `template<typename Class >`  
`void eventAddOnEntityCreateHandler (Class *object, void(Class::*function)(SimulationEvent *))`
- `template<typename Class >`  
`void eventAddOnEntityMoveHandler (Class *object, void(Class::*function)(SimulationEvent *))`
- `template<typename Class >`  
`void eventAddOnEntityRemoveHandler (Class *object, void(Class::*function)(SimulationEvent *))`
- `template<typename Class >`  
`void eventAddOnSimulationStartHandler (Class *object, void(Class::*function)(SimulationEvent *))`
- `template<typename Class >`  
`void eventAddOnSimulationPausedHandler (Class *object, void(Class::*function)(SimulationEvent *))`
- `template<typename Class >`  
`void eventAddOnSimulationResumeHandler (Class *object, void(Class::*function)(SimulationEvent *))`
- `template<typename Class >`  
`void eventAddOnSimulationEndHandler (Class *object, void(Class::*function)(SimulationEvent *))`
- `template<typename Class >`  
`void eventAddOnBreakpointHandler (Class *object, void(Class::*function)(SimulationEvent *))`
- `void eventNotifyModelCheckSuccessHandlers (ModelEvent *se)`
- `void eventNotifyModelLoadHandlers (ModelEvent *se)`
- `void eventNotifyModelSaveHandlers (ModelEvent *se)`
- `void eventNotifyReplicationStartHandlers (SimulationEvent *se)`
- `void eventNotifyReplicationStepHandlers (SimulationEvent *se)`
- `void eventNotifyReplicationEndHandlers (SimulationEvent *se)`
- `void eventNotifyProcessEventHandlers (SimulationEvent *se)`

- void **eventNotifyAfterProcessEventHandlers** ([SimulationEvent](#) \*se)
- void **eventNotifyEntityCreateHandlers** ([SimulationEvent](#) \*se)
- void **eventNotifyEntityMoveHandlers** ([SimulationEvent](#) \*se)
- void **eventNotifyEntityRemoveHandlers** ([SimulationEvent](#) \*se)
- void **eventNotifySimulationStartHandlers** ([SimulationEvent](#) \*se)
- void **eventNotifySimulationPausedHandlers** ([SimulationEvent](#) \*se)
- void **eventNotifySimulationResumeHandlers** ([SimulationEvent](#) \*se)
- void **eventNotifySimulationEndHandlers** ([SimulationEvent](#) \*se)
- void **eventNotifyBreakpointHandlers** ([SimulationEvent](#) \*se)

#### Static Public Member Functions

- static std::string **traceConvertEnumToStr** (TraceManager::Level level)

### 15.825.1 Detailed Description

High-level non-owning facade over [Simulator](#) and its main composed services.

Construct with a non-owning pointer to an existing [Simulator](#) instance. All public methods delegate through the private helpers; none of those helpers are exposed publicly.

The documentation for this class was generated from the following file:

- kernel/simulator/SimulatorFacade.h

## 15.826 SimulatorSessionService::SimulatorInfoResult Struct Reference

Describes simulator identity metadata for a token-scoped session.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

#### Public Attributes

- std::string **name**
- std::string **versionName**
- unsigned int **versionNumber**

### 15.826.1 Detailed Description

Describes simulator identity metadata for a token-scoped session.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

## 15.827 SimulatorSessionService Class Reference

Encapsulates session-scoped simulator operations exposed by the Web API.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

#### Classes

- struct [CreateSessionResult](#)  
*Contains session creation identifiers returned to API clients.*
- struct [ModelImportResult](#)  
*Contains model import output and error state.*
- struct [ModelInfoResult](#)  
*Describes the currently active model information for a session.*
- struct [ModelPersistenceResult](#)  
*Contains save/load model operation output and error state.*

- struct [SimulationActionResult](#)  
*Contains the result of a simulation action such as run or step.*
- struct [SimulationConfigInput](#)  
*Defines user-provided simulation parameters to apply to the current model.*
- struct [SimulationConfigResult](#)  
*Contains the result of applying simulation configuration.*
- struct [SimulationStatusResult](#)  
*Captures the current simulation execution state.*
- struct [SimulatorInfoResult](#)  
*Describes simulator identity metadata for a token-scoped session.*
- struct [WorkerCapabilitiesResult](#)  
*Describes currently supported worker features.*
- struct [WorkerInfoResult](#)  
*Describes worker identity metadata for distributed discovery.*
- struct [WorkerJobCreationResult](#)  
*Contains worker job creation output and error state.*
- struct [WorkerJobInfoResult](#)  
*Describes worker job metadata exposed by worker stage 3 endpoints.*
- struct [WorkerJobQueryResult](#)  
*Contains worker job lookup output and error state.*
- struct [WorkerJobResultInfo](#)  
*Describes terminal worker job result data exposed by worker stage 5 endpoint.*
- struct [WorkerJobResultQueryResult](#)  
*Contains worker job terminal result lookup output and error state.*
- struct [WorkerJobRunResult](#)  
*Contains worker job run output and error state.*

## Public Types

- enum class [PersistenceError](#) {  
**None** , **InvalidToken** , **InvalidFilename** , **MissingCurrentModel** ,  
**FileNotFound** , **OperationFailed** }  
*Enumerates persistence-specific errors for save/load model operations.*
- enum class [ModelImportError](#) {  
**None** , **InvalidToken** , **EmptySpecification** , **InvalidSpecification** ,  
**OperationFailed** }  
*Enumerates model import errors for worker language-ingress operations.*
- enum class [WorkerJobError](#) {  
**None** , **InvalidToken** , **MissingCurrentModel** , **JobNotFound** ,  
**AccessDenied** , **OperationFailed** , **ResultNotReady** }  
*Enumerates worker job operation errors.*

## Public Member Functions

- [SimulatorSessionService](#) ([SessionManager](#) &sessionManager, [WorkerJobManager](#) &workerJobManager)  
*Builds a service bound to the provided session manager.*
- [CreateSessionResult](#) [createSession](#) ()  
*Creates a new authenticated session.*
- bool [tryGetSimulatorInfo](#) (const std::string &accessToken, [SimulatorInfoResult](#) &outInfo)  
*Fetches simulator metadata using a session token.*
- [WorkerInfoResult](#) [getWorkerInfo](#) () const  
*Fetches public worker identity metadata.*
- [WorkerCapabilitiesResult](#) [getWorkerCapabilities](#) () const

- Fetches worker capability flags for the current implementation state.*
- `bool tryCreateModel` (const std::string &accessToken, [ModelInfoResult](#) &outInfo)  
*Creates a new model in the token-scoped simulator session.*
- `bool tryGetCurrentModelInfo` (const std::string &accessToken, [ModelInfoResult](#) &outInfo)  
*Returns metadata for the current model in a session.*
- `SimulationStatusResult getSimulationStatus` (const std::string &accessToken)  
*Returns simulation status for the session's current model.*
- `SimulationConfigResult configureSimulation` (const std::string &accessToken, const [SimulationConfigInput](#) &input)  
*Applies simulation configuration to the current model.*
- `SimulationActionResult runSimulation` (const std::string &accessToken)  
*Executes a synchronous simulation run.*
- `SimulationActionResult stepSimulation` (const std::string &accessToken)  
*Executes one synchronous simulation step.*
- `ModelPersistenceResult saveCurrentModel` (const std::string &accessToken, const std::string &filename)  
*Saves the current model into the session workspace.*
- `ModelPersistenceResult loadModel` (const std::string &accessToken, const std::string &filename)  
*Loads a model from the session workspace by filename.*
- `ModelImportResult importModelFromLanguage` (const std::string &accessToken, const std::string &model← Specification)  
*Imports a model from plain text model language into the token-scoped session.*
- `WorkerJobCreationResult createWorkerJob` (const std::string &accessToken)  
*Creates a queued worker job for the token-scoped session model snapshot.*
- `WorkerJobQueryResult getWorkerJob` (const std::string &accessToken, const std::string &jobId)  
*Returns worker job metadata for a token-scoped session.*
- `WorkerJobRunResult runWorkerJob` (const std::string &accessToken, const std::string &jobId)  
*Executes a worker job synchronously in the token-scoped simulator session.*
- `WorkerJobResultQueryResult getWorkerJobResult` (const std::string &accessToken, const std::string &jobId)  
*Returns persisted terminal result data for a token-scoped worker job.*

### 15.827.1 Detailed Description

Encapsulates session-scoped simulator operations exposed by the Web API.

### 15.827.2 Constructor & Destructor Documentation

#### 15.827.2.1 SimulatorSessionService()

```
SimulatorSessionService::SimulatorSessionService (
 SessionManager & sessionManager,
 WorkerJobManager & workerJobManager) [explicit]
```

Builds a service bound to the provided session manager.  
Wires session lookup with per-session worker job storage.

#### Parameters

|                               |                                                  |
|-------------------------------|--------------------------------------------------|
| <code>sessionManager</code>   | Session manager used to resolve active sessions. |
| <code>workerJobManager</code> | In-memory worker job storage manager.            |

## 15.827.3 Member Function Documentation

### 15.827.3.1 configureSimulation()

```
SimulatorSessionService::SimulationConfigResult SimulatorSessionService::configureSimulation (
 const std::string & accessToken,
 const SimulationConfigInput & input)
```

Applies simulation configuration to the current model.

Applies user-provided simulation parameters to the current model.

#### Parameters

|                    |                                         |
|--------------------|-----------------------------------------|
| <i>accessToken</i> | Bearer token associated with a session. |
| <i>input</i>       | Configuration values to apply.          |

#### Returns

Structured configuration result and resulting status.

### 15.827.3.2 createSession()

```
SimulatorSessionService::CreateSessionResult SimulatorSessionService::createSession ()
```

Creates a new authenticated session.

Creates one authenticated session and returns its public identifiers.

#### Returns

Session identifiers and token for API usage.

### 15.827.3.3 createWorkerJob()

```
SimulatorSessionService::WorkerJobCreationResult SimulatorSessionService::createWorkerJob (
 const std::string & accessToken)
```

Creates a queued worker job for the token-scoped session model snapshot.

Creates a queued worker job by snapshotting the current model.

#### Parameters

|                    |                                         |
|--------------------|-----------------------------------------|
| <i>accessToken</i> | Bearer token associated with a session. |
|--------------------|-----------------------------------------|

#### Returns

Worker job creation output and error state.

### 15.827.3.4 getSimulationStatus()

```
SimulatorSessionService::SimulationStatusResult SimulatorSessionService::getSimulationStatus (
 const std::string & accessToken)
```

Returns simulation status for the session's current model.

Returns the current simulation status for a session token.

#### Parameters

|                    |                                         |
|--------------------|-----------------------------------------|
| <i>accessToken</i> | Bearer token associated with a session. |
|--------------------|-----------------------------------------|



**Returns**

Structured simulation status query result.

**15.827.3.5 getWorkerCapabilities()**

```
SimulatorSessionService::WorkerCapabilitiesResult SimulatorSessionService::getWorkerCapabilities
() const
```

Fetches worker capability flags for the current implementation state.

Advertises the runtime capabilities supported by this worker flavor.

**Returns**

Worker capability flags.

**15.827.3.6 getWorkerInfo()**

```
SimulatorSessionService::WorkerInfoResult SimulatorSessionService::getWorkerInfo () const
```

Fetches public worker identity metadata.

Returns the fixed worker identity metadata used for discovery.

**Returns**

Worker identity information for discovery endpoints.

**15.827.3.7 getWorkerJob()**

```
SimulatorSessionService::WorkerJobQueryResult SimulatorSessionService::getWorkerJob (
 const std::string & accessToken,
 const std::string & jobId)
```

Returns worker job metadata for a token-scoped session.

Retrieves worker-job metadata when the job belongs to the session.

**Parameters**

|                    |                                         |
|--------------------|-----------------------------------------|
| <i>accessToken</i> | Bearer token associated with a session. |
| <i>jobId</i>       | Worker job identifier to query.         |

**Returns**

Worker job lookup output and error state.

**15.827.3.8 getWorkerJobResult()**

```
SimulatorSessionService::WorkerJobResultQueryResult SimulatorSessionService::getWorkerJob↵
Result (
 const std::string & accessToken,
 const std::string & jobId)
```

Returns persisted terminal result data for a token-scoped worker job.

Retrieves the terminal result for a completed worker job.

**Parameters**

|                    |                                         |
|--------------------|-----------------------------------------|
| <i>accessToken</i> | Bearer token associated with a session. |
| <i>jobId</i>       | Worker job identifier to query.         |



**Returns**

Worker job terminal result lookup output and error state.

**15.827.3.9 importModelFromLanguage()**

```
SimulatorSessionService::ModelImportResult SimulatorSessionService::importModelFromLanguage (
 const std::string & accessToken,
 const std::string & modelSpecification)
```

Imports a model from plain text model language into the token-scoped session.

Imports a model from textual Genesys language.

**Parameters**

|                           |                                         |
|---------------------------|-----------------------------------------|
| <i>accessToken</i>        | Bearer token associated with a session. |
| <i>modelSpecification</i> | Plain text model language content.      |

**Returns**

Import output and error state.

**15.827.3.10 loadModel()**

```
SimulatorSessionService::ModelPersistenceResult SimulatorSessionService::loadModel (
 const std::string & accessToken,
 const std::string & filename)
```

Loads a model from the session workspace by filename.

Loads a saved model file from the session workspace.

**Parameters**

|                    |                                         |
|--------------------|-----------------------------------------|
| <i>accessToken</i> | Bearer token associated with a session. |
| <i>filename</i>    | Safe basename to load.                  |

**Returns**

Persistence output and error state.

**15.827.3.11 runSimulation()**

```
SimulatorSessionService::SimulationActionResult SimulatorSessionService::runSimulation (
 const std::string & accessToken)
```

Executes a synchronous simulation run.

Starts the current simulation for the token-scoped session.

**Parameters**

|                    |                                         |
|--------------------|-----------------------------------------|
| <i>accessToken</i> | Bearer token associated with a session. |
|--------------------|-----------------------------------------|

**Returns**

Structured action result and updated simulation status.

**15.827.3.12 runWorkerJob()**

```
SimulatorSessionService::WorkerJobRunResult SimulatorSessionService::runWorkerJob (
```

```
const std::string & accessToken,
const std::string & jobId)
```

Executes a worker job synchronously in the token-scoped simulator session.  
 Executes a queued worker job synchronously from its stored snapshot.

#### Parameters

|                    |                                         |
|--------------------|-----------------------------------------|
| <i>accessToken</i> | Bearer token associated with a session. |
| <i>jobId</i>       | Worker job identifier to execute.       |

#### Returns

Worker job run output and error state.

### 15.827.3.13 saveCurrentModel()

```
SimulatorSessionService::ModelPersistenceResult SimulatorSessionService::saveCurrentModel (
 const std::string & accessToken,
 const std::string & filename)
```

Saves the current model into the session workspace.  
 Persists the current model into the session workspace.

#### Parameters

|                    |                                         |
|--------------------|-----------------------------------------|
| <i>accessToken</i> | Bearer token associated with a session. |
| <i>filename</i>    | Safe basename to use for persistence.   |

#### Returns

Persistence output and error state.

### 15.827.3.14 stepSimulation()

```
SimulatorSessionService::SimulationActionResult SimulatorSessionService::stepSimulation (
 const std::string & accessToken)
```

Executes one synchronous simulation step.  
 Advances the current simulation by one step.

#### Parameters

|                    |                                         |
|--------------------|-----------------------------------------|
| <i>accessToken</i> | Bearer token associated with a session. |
|--------------------|-----------------------------------------|

#### Returns

Structured action result and updated simulation status.

### 15.827.3.15 tryCreateModel()

```
bool SimulatorSessionService::tryCreateModel (
 const std::string & accessToken,
 ModelInfoResult & outInfo)
```

Creates a new model in the token-scoped simulator session.  
 Creates a new kernel model in the current session.

#### Parameters

|                    |                                         |
|--------------------|-----------------------------------------|
| <i>accessToken</i> | Bearer token associated with a session. |
| <i>outInfo</i>     | Receives model information on success.  |

#### Returns

True when the model was created and output was populated.

#### 15.827.3.16 tryGetCurrentModelInfo()

```
bool SimulatorSessionService::tryGetCurrentModelInfo (
 const std::string & accessToken,
 ModelInfoResult & outInfo)
```

Returns metadata for the current model in a session.

Returns metadata for the current model if one exists.

#### Parameters

|                    |                                         |
|--------------------|-----------------------------------------|
| <i>accessToken</i> | Bearer token associated with a session. |
| <i>outInfo</i>     | Receives current model data.            |

#### Returns

True when token is valid and query completed.

#### 15.827.3.17 tryGetSimulatorInfo()

```
bool SimulatorSessionService::tryGetSimulatorInfo (
 const std::string & accessToken,
 SimulatorInfoResult & outInfo)
```

Fetches simulator metadata using a session token.

Resolves the simulator metadata visible to a session token.

#### Parameters

|                    |                                                     |
|--------------------|-----------------------------------------------------|
| <i>accessToken</i> | Bearer token associated with a session.             |
| <i>outInfo</i>     | Receives simulator identity information on success. |

#### Returns

True when token is valid and metadata was populated.

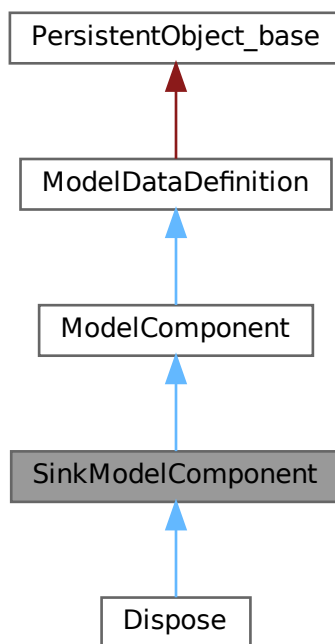
The documentation for this class was generated from the following files:

- applications/worker/service/SimulatorSessionService.h
- applications/worker/service/SimulatorSessionService.cpp

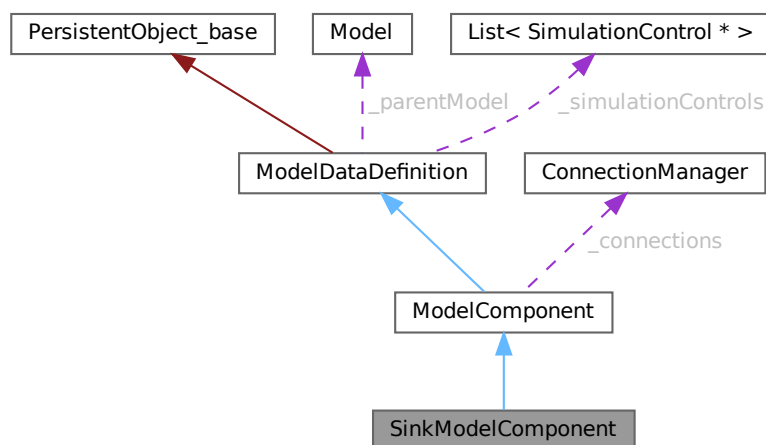
## 15.828 SinkModelComponent Class Reference

```
#include <kernel/simulator/SinkModelComponent.h>
```

Inheritance diagram for SinkModelComponent:



Collaboration diagram for SinkModelComponent:



### Public Member Functions

- `SinkModelComponent` (`Model` \*model, std::string componentTypename, std::string name="")

## Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** \*model, std::string componentTypename, std::string name="")  
*constructors*
- virtual ~**ModelComponent** ()  
*Releases component-owned resources and unregisters it from the model.*
- **ConnectionManager** \* **getConnectionManager** () const  
*new public user methods for this component*
- void **connectTo** (**ModelComponent** \*component, unsigned int inputPortNumber=0)  
*Connects this component to another component.*
- bool **hasBreakpointAt** ()  
*Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string \_description)  
*Updates the free-form description shown by the GUI.*
- std::string **getDescription** () const  
*Returns the free-form description associated with this component.*
- void **remove** (**ModelComponent** \*comp)  
*Removes a connected component from this component.*
- virtual std::string **show** () override  
*virtual public methods*

## Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** \*model, std::string datadefinitionTypename, std::string name="", bool insert↵  
IntoModel=true)  
*Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()  
*Releases resources owned by this data definition.*
- Util::identification **getId** () const  
*Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)  
*Updates the logical name of this instance.*
- const std::string & **getName** () const  
*Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)  
*Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const  
*Returns the user-visible label of this instance.*
- const std::string & **getClassname** () const  
*Returns the runtime class name used by the plugin system.*
- **Model** \* **getParentModel** () const  
*Returns the parent model that owns this definition.*
- bool **isReportStatistics** () const  
*Indicates whether this data definition contributes report statistics.*
- void **setReportStatistics** (bool reportStatistics)  
*Enables or disables report-statistics generation for this definition.*
- **ModelDataDefinition** \* **getInternalData** (const std::string &name) const  
*Looks up a named internal child data definition.*
- std::map< std::string, **ModelDataDefinition** \* > \* **getInternalData** () const  
*Returns the full map of internal child data definitions.*
- std::map< std::string, **ModelDataDefinition** \* > \* **getAttachedData** () const  
*Returns the full map of attached data definitions.*

- bool [hasChanged](#) () const  
*Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))  
*Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const  
*Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int \_modelLevel)  
*Sets the model nesting level for this definition.*
- [List< SimulationControl \\* > \\* getSimulationControls](#) () const  
*Returns the simulation controls owned by this data definition.*
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const  
*Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)  
*Updates the trace level used by this definition.*
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelspecific, bool traceLevelSpecific←  
Enabled=true)  
*Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const  
*Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)  
*Enables or disables the specific trace level.*

### Protected Member Functions

- virtual bool [\\_loadInstance](#) ([PersistenceRecord](#) \*fields) override  
*virtual protected method that must be overridden*
- virtual void [\\_saveInstance](#) ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*
- virtual bool [\\_check](#) (std::string &errorMessage) override  
*could be overridden by derived classes*

### Protected Member Functions inherited from [ModelComponent](#)

- virtual void [\\_onDispatchEvent](#) ([Entity](#) \*entity, unsigned int inputPortNumber)=0  
*Handles a dispatched event for this component type.*

### Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [\\_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) \* [\\_getParserChangesInformation](#) ()
- virtual void [\\_initBetweenReplications](#) ()
- virtual void [\\_addSimulationControl](#) ([SimulationControl](#) \*control)
- virtual void [\\_createInternalAndAttachedData](#) ()
- virtual void [\\_createInternalStatisticReporters](#) ()
- virtual void [\\_createAttachedAttributes](#) ()
- virtual void [\\_createNonEditableDataDefinitions](#) ()
- virtual void [\\_createEditableDataDefinitions](#) ()
- void [\\_templateCreateInternalStatisticReporters](#) ()
- void [\\_templateCreateAttachedAttributes](#) ()
- void [\\_templateCreateNonEditableDataDefinitions](#) ()
- void [\\_templateCreateEditableDataDefinitions](#) ()
- void [\\_internaStatisticReportersClear](#) ()  
*legacy helper methods used by the granular create\* hooks*

- void **\_internalDataClear** ()
- void **\_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) \*child)
- void **\_internalDataRemove** (const std::string &key)
- void **\_attachedDataClear** ()
- void **\_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_attachedDataRemove** (const std::string &key)
- void **\_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **\_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_statisticReporterRemove** (const std::string &key)
- void **\_statisticReportersClear** ()
- void **\_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **\_mandatoryAttachedAttributesClear** ()
- void **\_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryEditableDataDefinitionsClear** ()
- void **\_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalEditableDataDefinitionsClear** ()
- void **\_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryNonEditableDataDefinitionsClear** ()
- void **\_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalNonEditableDataDefinitionsClear** ()
- void **\_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)  
*method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))  
*not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1\\_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2\\_results](#))
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))
- void **traceSimulation** (void \*thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))  
*Traces text to the simulation output.*
- void **traceSimulation** (void \*thisobject, [TraceManager::Level](#) level, const std::string &text)

### Additional Inherited Members

### Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) \* **LoadInstance** ([Model](#) \*model, [PersistenceRecord](#) \*fields)  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool **Check** ([ModelComponent](#) \*component)  
*Validates a component instance.*
- static void **CreateInternalData** ([ModelComponent](#) \*component)  
*Creates internal data required by a component instance.*
- static void **SaveInstance** ([PersistenceRecord](#) \*fields, [ModelComponent](#) \*component)  
*Serializes a component instance into a persistence record.*
- static void **DispatchEvent** ([Event](#) \*event)  
*Dispatches a simulation event to the target component.*

## Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool [Check](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*
- static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

## Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) \* [\\_connections](#) = new [ConnectionManager](#)()  
*virtual protected methods that could be overridden by derived classes, if needed*

## Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification [\\_id](#)
- std::string [\\_typename](#)
- bool [\\_reportStatistics](#)
- bool [\\_hasChanged](#)
- unsigned int [\\_modelLevel](#) = 0
- [Model](#) \* [\\_parentModel](#)
- [List](#)< [SimulationControl](#) \* > \* [\\_simulationControls](#) = new [List](#)<[SimulationControl](#)\*>()
- [TraceManager](#)::Level [\\_traceLevelSpecific](#)
- bool [\\_traceLevelSpecificEnabled](#) = false

### 15.828.1 Detailed Description

This class is the basis for any component representing the end of a process flow, such as a [Dispose](#). It can remove entities from the system and collect statistics.

### 15.828.2 Member Function Documentation

#### 15.828.2.1 [\\_check\(\)](#)

```
bool SinkModelComponent::_check (
 std::string & errorMessage) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [Dispose](#).

#### 15.828.2.2 [\\_loadInstance\(\)](#)

```
bool SinkModelComponent::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

Reimplemented in [Dispose](#).



### 15.828.2.3 \_saveInstance()

```
void SinkModelComponent::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

#### Parameters

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [ModelComponent](#).

Reimplemented in [Dispose](#).

The documentation for this class was generated from the following files:

- kernel/simulator/SinkModelComponent.h
- kernel/simulator/SinkModelComponent.cpp

## 15.829 yy::genesyspp\_parser::stack< T, S >::slice Class Reference

Present a slice of the top of a stack.

```
#include <parser/GenesysParser.h>
```

#### Public Member Functions

- **slice** (const stack &stack, index\_type range) YY\_NOEXCEPT
- const T & **operator[]** (index\_type i) const

### 15.829.1 Detailed Description

```
template<typename T, typename S = std::vector<T>>
```

```
class yy::genesyspp_parser::stack< T, S >::slice
```

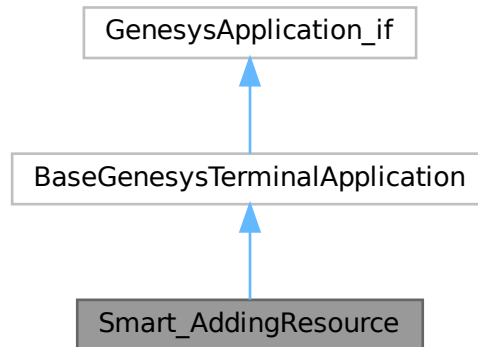
Present a slice of the top of a stack.

The documentation for this class was generated from the following file:

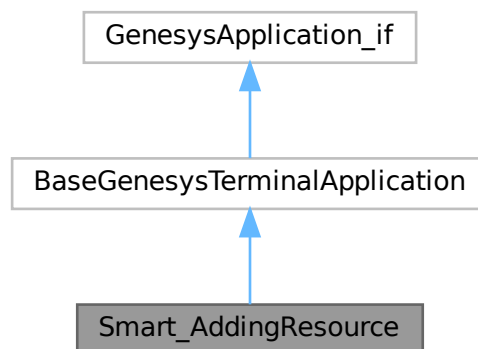
- parser/[GenesysParser.h](#)

## 15.830 Smart\_AddingResource Class Reference

Inheritance diagram for Smart\_AddingResource:



Collaboration diagram for Smart\_AddingResource:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.830.1 Member Function Documentation

### 15.830.1.1 main()

```
int Smart_AddingResource::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

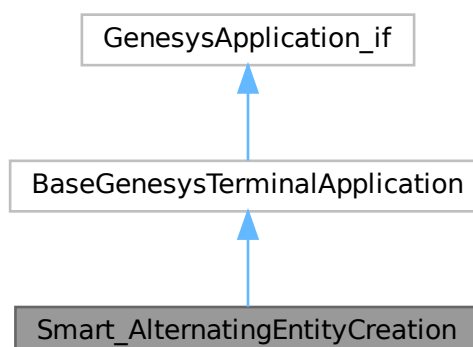
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

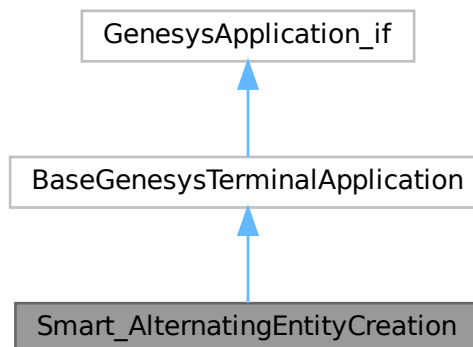
- applications/modelSpecific/arenaSmarts/Smart\_AddingResource.h
- applications/modelSpecific/arenaSmarts/Smart\_AddingResource.cpp

## 15.831 Smart\_AlternatingEntityCreation Class Reference

Inheritance diagram for Smart\_AlternatingEntityCreation:



Collaboration diagram for Smart\_AlternatingEntityCreation:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.831.1 Member Function Documentation

### 15.831.1.1 main()

```
int Smart_AlternatingEntityCreation::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

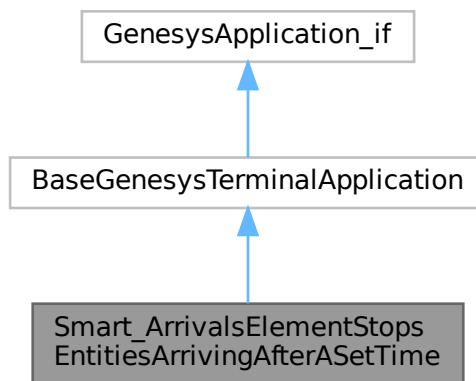
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

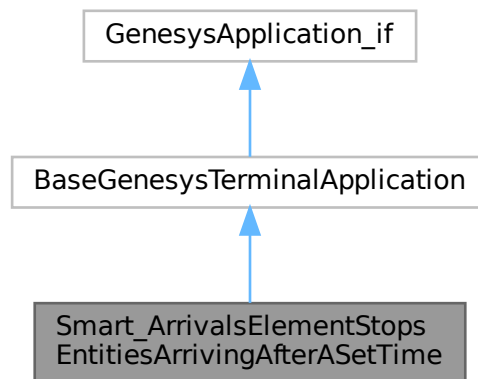
- applications/modelSpecific/arenaSmarts/Smart\_AlternatingEntityCreation.h
- applications/modelSpecific/arenaSmarts/Smart\_AlternatingEntityCreation.cpp

## 15.832 Smart\_ArrivalsElementStopsEntitiesArrivingAfterASetTime Class Reference

Inheritance diagram for Smart\_ArrivalsElementStopsEntitiesArrivingAfterASetTime:



Collaboration diagram for Smart\_ArrivalsElementStopsEntitiesArrivingAfterASetTime:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.832.1 Member Function Documentation

### 15.832.1.1 main()

```
int Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

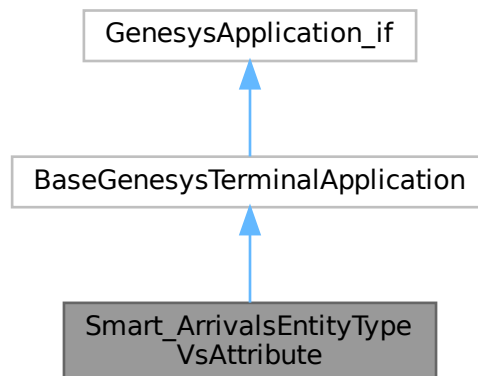
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

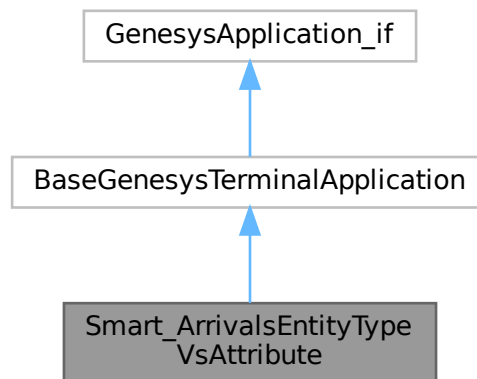
- applications/modelSpecific/arenaSmarts/Smart\_ArrivalsElementStopsEntitiesArrivingAfterASetTime.h
- applications/modelSpecific/arenaSmarts/Smart\_ArrivalsElementStopsEntitiesArrivingAfterASetTime.cpp

## 15.833 Smart\_ArrivalsEntityTypeVsAttribute Class Reference

Inheritance diagram for Smart\_ArrivalsEntityTypeVsAttribute:



Collaboration diagram for Smart\_ArrivalsEntityTypeVsAttribute:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)



## 15.833.1 Member Function Documentation

### 15.833.1.1 main()

```
int Smart_ArrivalsEntityTypeVsAttribute::main (
 int argc,
 char ** argv) [override], [virtual]
```

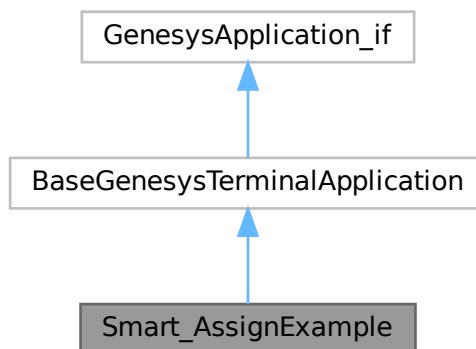
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

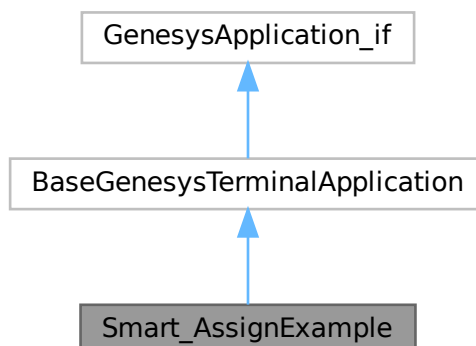
- applications/modelSpecific/arenaSmarts/Smart\_ArrivalsEntityTypeVsAttribute.h
- applications/modelSpecific/arenaSmarts/Smart\_ArrivalsEntityTypeVsAttribute.cpp

## 15.834 Smart\_AssignExample Class Reference

Inheritance diagram for Smart\_AssignExample:



Collaboration diagram for Smart\_AssignExample:



## Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

## Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

## Additional Inherited Members

## Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.834.1 Member Function Documentation

### 15.834.1.1 [main\(\)](#)

```
int Smart_AssignExample::main (
 int argc,
 char ** argv) [override], [virtual]
```

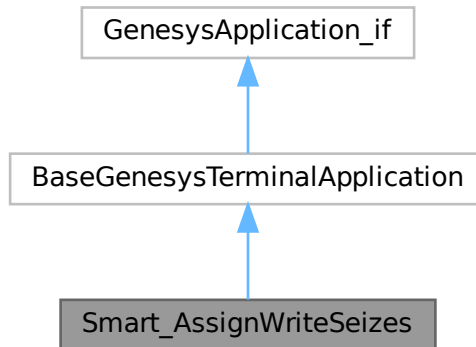
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

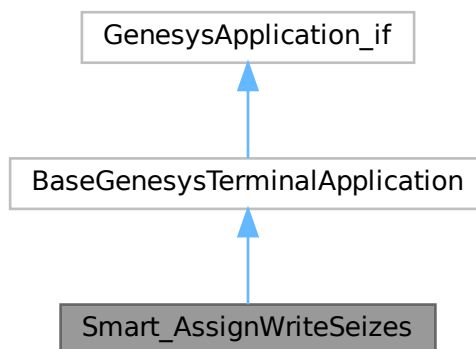
- applications/modelSpecific/arenaSmarts/Smart\_AssignExample.h
- applications/modelSpecific/arenaSmarts/Smart\_AssignExample.cpp

## 15.835 Smart\_AssignWriteSeizes Class Reference

Inheritance diagram for Smart\_AssignWriteSeizes:



Collaboration diagram for Smart\_AssignWriteSeizes:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.835.1 Member Function Documentation

### 15.835.1.1 main()

```
int Smart_AssignWriteSeizes::main (
 int argc,
 char ** argv) [override], [virtual]
```

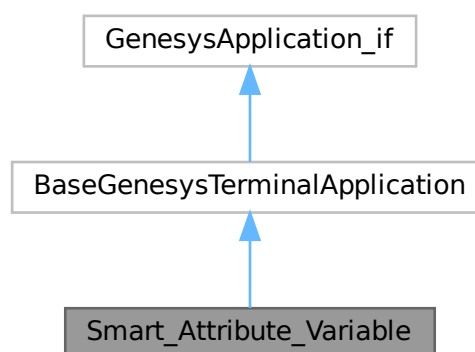
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

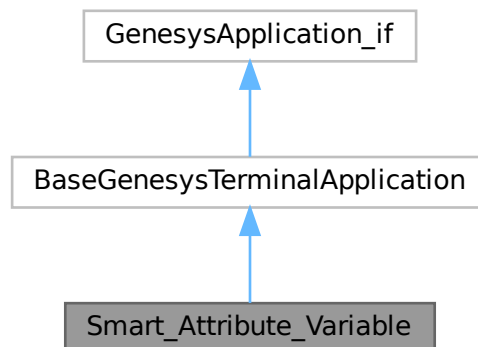
- applications/modelSpecific/smarts/Logic/Smart\_AssignWriteSeizes.h
- applications/modelSpecific/smarts/Logic/Smart\_AssignWriteSeizes.cpp

## 15.836 Smart\_Attribute\_Variable Class Reference

Inheritance diagram for Smart\_Attribute\_Variable:



Collaboration diagram for Smart\_Attribute\_Variable:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.836.1 Member Function Documentation

### 15.836.1.1 main()

```
int Smart_Attribute_Variable::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

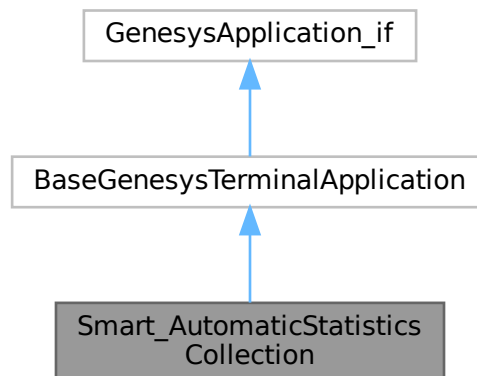
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

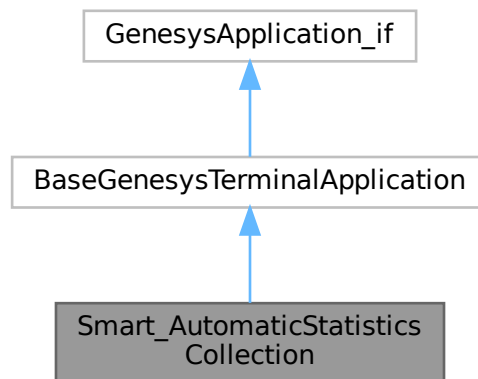
- applications/modelSpecific/smarts/Logic/Smart\_Attribute\_Variable.h
- applications/modelSpecific/smarts/Logic/Smart\_Attribute\_Variable.cpp

## 15.837 Smart\_AutomaticStatisticsCollection Class Reference

Inheritance diagram for Smart\_AutomaticStatisticsCollection:



Collaboration diagram for Smart\_AutomaticStatisticsCollection:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.837.1 Member Function Documentation

### 15.837.1.1 main()

```
int Smart_AutomaticStatisticsCollection::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

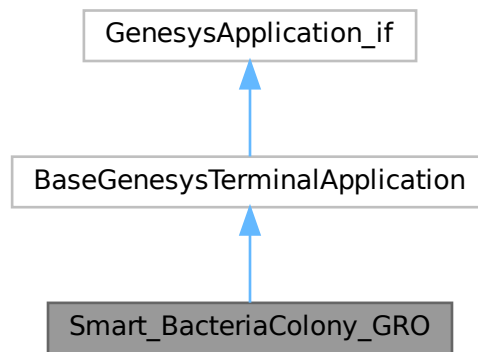
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

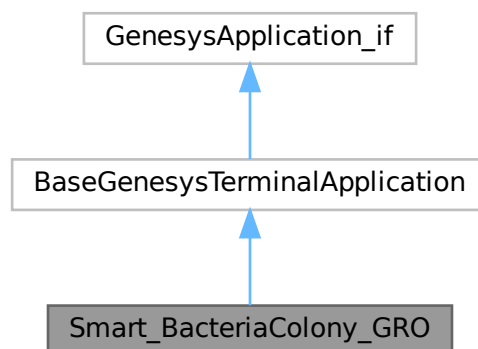
- applications/modelSpecific/arenaSmarts/Smart\_AutomaticStatisticsCollection.h
- applications/modelSpecific/arenaSmarts/Smart\_AutomaticStatisticsCollection.cpp

## 15.838 Smart\_BacteriaColony\_GRO Class Reference

Inheritance diagram for Smart\_BacteriaColony\_GRO:



Collaboration diagram for Smart\_BacteriaColony\_GRO:





## Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

## Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

## Additional Inherited Members

## Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.838.1 Member Function Documentation

### 15.838.1.1 [main\(\)](#)

```
int Smart_BacteriaColony_GRO::main (
 int argc,
 char ** argv) [override], [virtual]
```

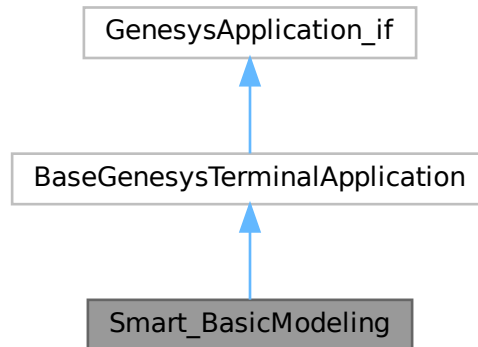
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

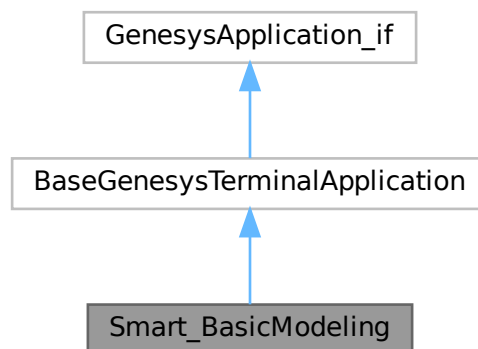
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart\_BacteriaColony\_GRO.h
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart\_BacteriaColony\_GRO.cpp

## 15.839 Smart\_BasicModeling Class Reference

Inheritance diagram for Smart\_BasicModeling:



Collaboration diagram for Smart\_BasicModeling:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.839.1 Member Function Documentation

### 15.839.1.1 main()

```
int Smart_BasicModeling::main (
 int argc,
 char ** argv) [override], [virtual]
```

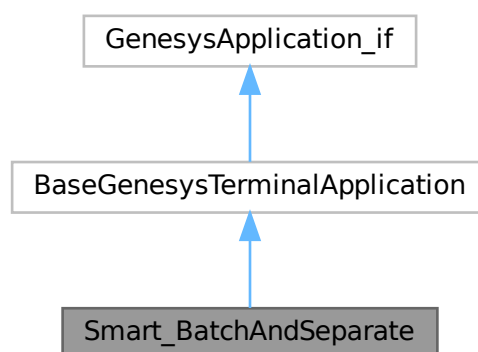
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

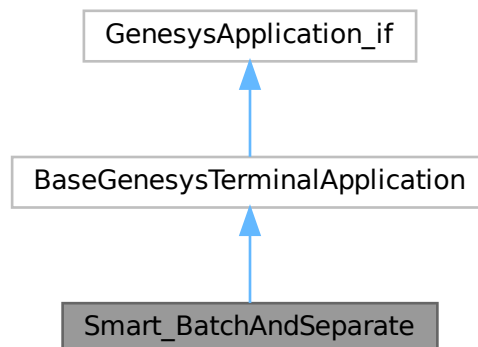
- applications/modelSpecific/arenaSmarts/Smart\_BasicModeling.h
- applications/modelSpecific/arenaSmarts/Smart\_BasicModeling.cpp

## 15.840 Smart\_BatchAndSeparate Class Reference

Inheritance diagram for Smart\_BatchAndSeparate:



Collaboration diagram for Smart\_BatchAndSeparate:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.840.1 Member Function Documentation

### 15.840.1.1 main()

```
int Smart_BatchAndSeparate::main (
 int argc,
 char ** argv) [override], [virtual]
```

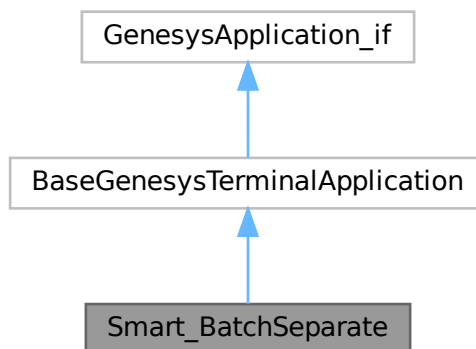
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

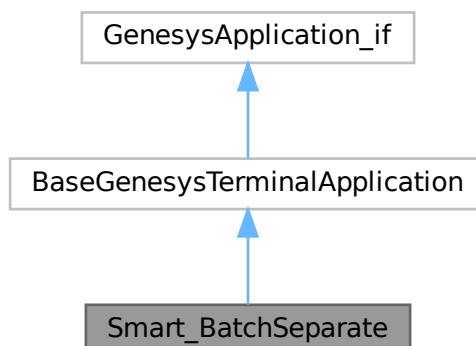
- applications/modelSpecific/arenaSmarts/Smart\_BatchAndSeparate.h
- applications/modelSpecific/arenaSmarts/Smart\_BatchAndSeparate.cpp

## 15.841 Smart\_BatchSeparate Class Reference

Inheritance diagram for Smart\_BatchSeparate:



Collaboration diagram for Smart\_BatchSeparate:



## Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

## Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

## Additional Inherited Members

## Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.841.1 Member Function Documentation

### 15.841.1.1 [main\(\)](#)

```
int Smart_BatchSeparate::main (
 int argc,
 char ** argv) [override], [virtual]
```

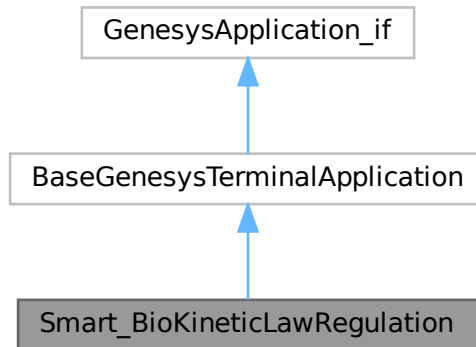
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

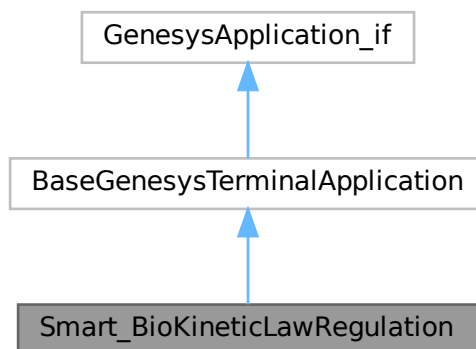
- applications/modelSpecific/smarts/Grouping/Smart\_BatchSeparate.h
- applications/modelSpecific/smarts/Grouping/Smart\_BatchSeparate.cpp

## 15.842 Smart\_BioKineticLawRegulation Class Reference

Inheritance diagram for Smart\_BioKineticLawRegulation:



Collaboration diagram for Smart\_BioKineticLawRegulation:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.842.1 Member Function Documentation

### 15.842.1.1 main()

```
int Smart_BioKineticLawRegulation::main (
 int argc,
 char ** argv) [override], [virtual]
```

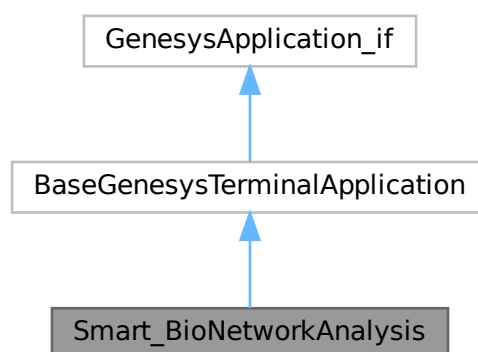
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/smarts/BiochemicalSimulation/Smart\_BioKineticLawRegulation.h
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart\_BioKineticLawRegulation.cpp

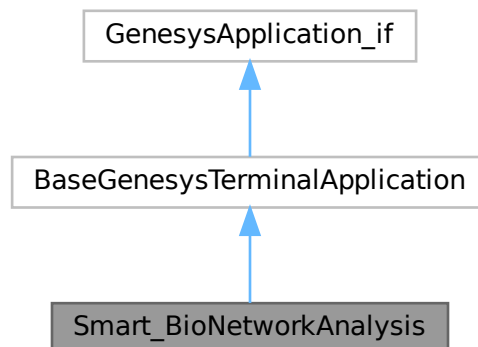
## 15.843 Smart\_BioNetworkAnalysis Class Reference

Inheritance diagram for Smart\_BioNetworkAnalysis:





Collaboration diagram for Smart\_BioNetworkAnalysis:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.843.1 Member Function Documentation

### 15.843.1.1 main()

```
int Smart_BioNetworkAnalysis::main (
 int argc,
 char ** argv) [override], [virtual]
```

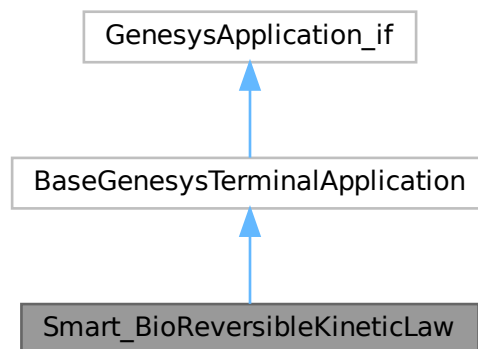
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

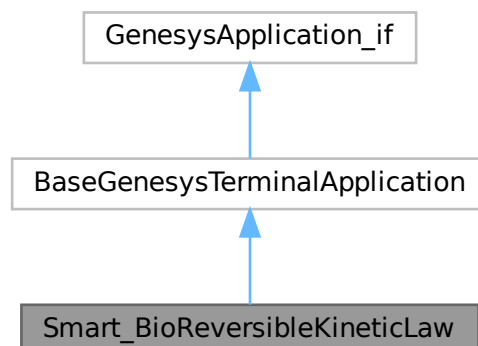
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart\_BioNetworkAnalysis.h
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart\_BioNetworkAnalysis.cpp

## 15.844 Smart\_BioReversibleKineticLaw Class Reference

Inheritance diagram for Smart\_BioReversibleKineticLaw:



Collaboration diagram for Smart\_BioReversibleKineticLaw:



## Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

## Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

## Additional Inherited Members

## Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.844.1 Member Function Documentation

### 15.844.1.1 [main\(\)](#)

```
int Smart_BioReversibleKineticLaw::main (
 int argc,
 char ** argv) [override], [virtual]
```

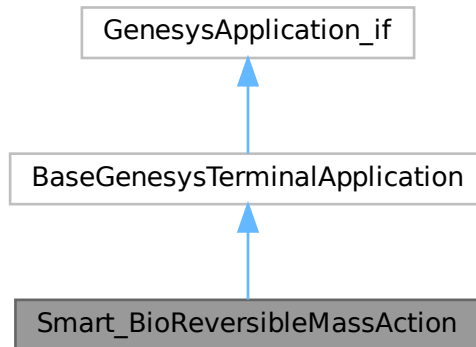
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

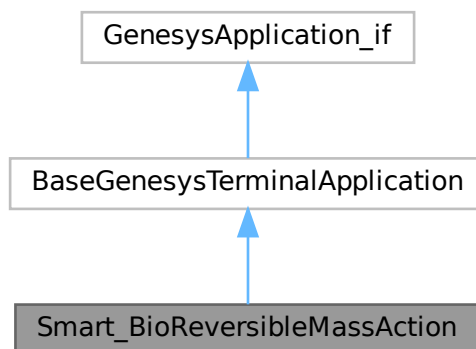
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart\_BioReversibleKineticLaw.h
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart\_BioReversibleKineticLaw.cpp

## 15.845 Smart\_BioReversibleMassAction Class Reference

Inheritance diagram for Smart\_BioReversibleMassAction:



Collaboration diagram for Smart\_BioReversibleMassAction:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.845.1 Member Function Documentation

### 15.845.1.1 main()

```
int Smart_BioReversibleMassAction::main (
 int argc,
 char ** argv) [override], [virtual]
```

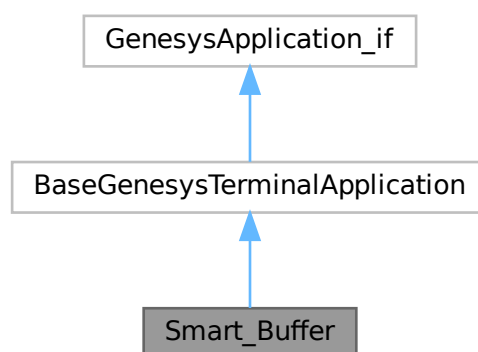
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

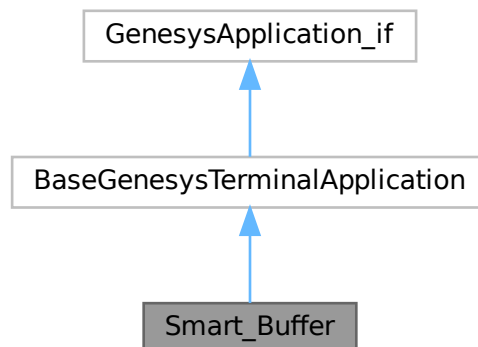
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart\_BioReversibleMassAction.h
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart\_BioReversibleMassAction.cpp

## 15.846 Smart\_Buffer Class Reference

Inheritance diagram for Smart\_Buffer:



Collaboration diagram for Smart\_Buffer:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.846.1 Member Function Documentation

### 15.846.1.1 main()

```
int Smart_Buffer::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

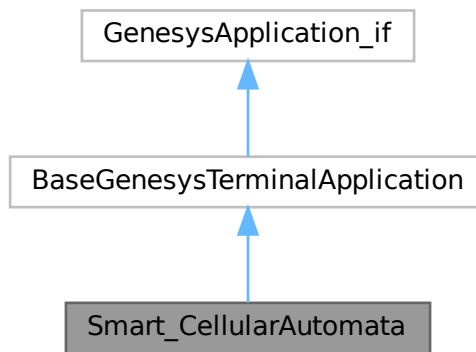
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

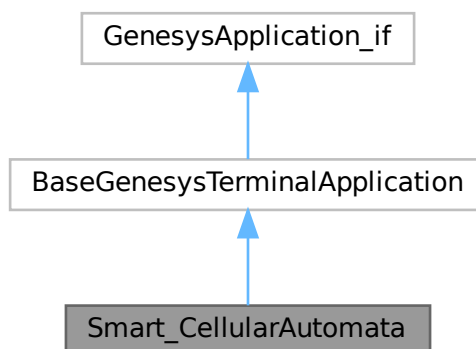
- applications/modelSpecific/smarts/DiscreteProcessing/Smart\_Buffer.h
- applications/modelSpecific/smarts/DiscreteProcessing/Smart\_Buffer.cpp

## 15.847 Smart\_CellularAutomata Class Reference

Inheritance diagram for Smart\_CellularAutomata:



Collaboration diagram for Smart\_CellularAutomata:



## Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

## Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

## Additional Inherited Members

## Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.847.1 Member Function Documentation

### 15.847.1.1 [main\(\)](#)

```
int Smart_CellularAutomata::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

Reimplemented from [BaseGenesysTerminalApplication](#).

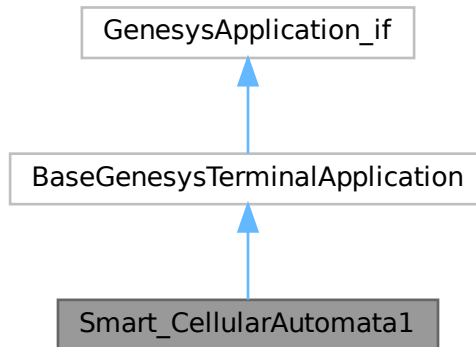
The documentation for this class was generated from the following files:

- applications/modelSpecific/smarts/ModalModel/Smart\_CellularAutomata.h
- applications/modelSpecific/smarts/ModalModel/Smart\_CellularAutomata.cpp

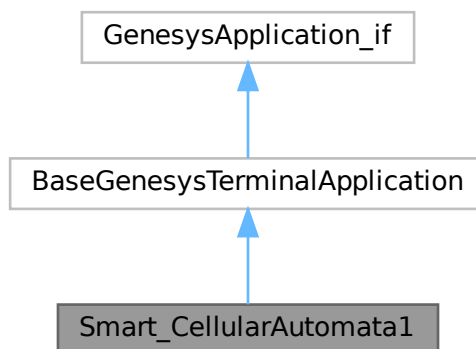


## 15.848 Smart\_CellularAutomata1 Class Reference

Inheritance diagram for Smart\_CellularAutomata1:



Collaboration diagram for Smart\_CellularAutomata1:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.848.1 Member Function Documentation

### 15.848.1.1 main()

```
int Smart_CellularAutomata1::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

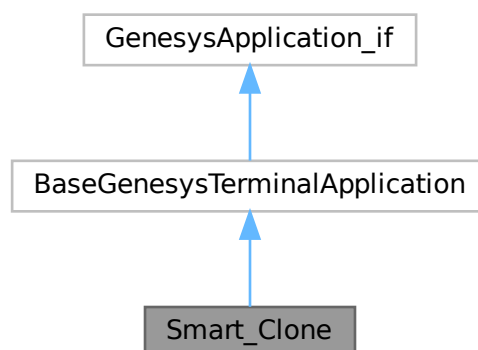
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

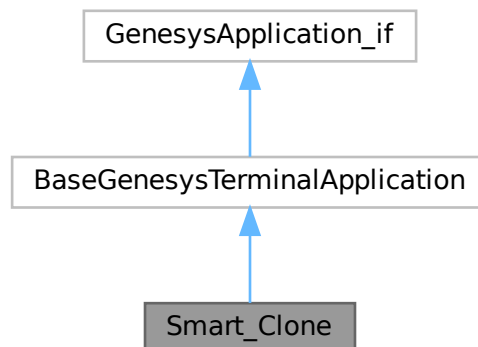
- applications/modelSpecific/smarts/ModalModel/Smart\_CellularAutomata1.h
- applications/modelSpecific/smarts/ModalModel/Smart\_CellularAutomata1.cpp

## 15.849 Smart\_Clone Class Reference

Inheritance diagram for Smart\_Clone:



Collaboration diagram for Smart\_Clone:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.849.1 Member Function Documentation

### 15.849.1.1 main()

```
int Smart_Clone::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

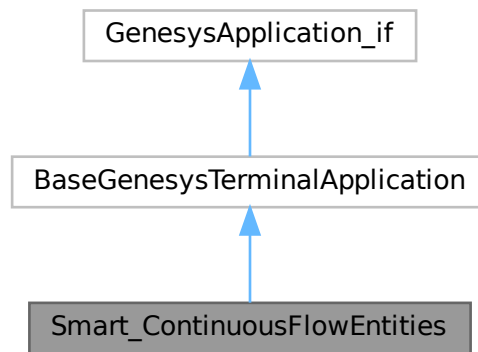
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

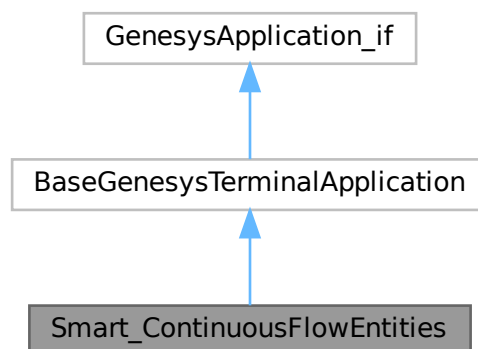
- applications/modelSpecific/smarts/DiscreteProcessing/Smart\_Clone.h
- applications/modelSpecific/smarts/DiscreteProcessing/Smart\_Clone.cpp

## 15.850 Smart\_ContinuousFlowEntities Class Reference

Inheritance diagram for Smart\_ContinuousFlowEntities:



Collaboration diagram for Smart\_ContinuousFlowEntities:



**Public Member Functions**

- virtual int [main](#) (int argc, char \*\*argv) override

**Public Member Functions inherited from [BaseGenesysTerminalApplication](#)**

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

**Additional Inherited Members****Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)**

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

**15.850.1 Member Function Documentation****15.850.1.1 main()**

```
int Smart_ContinuousFlowEntities::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

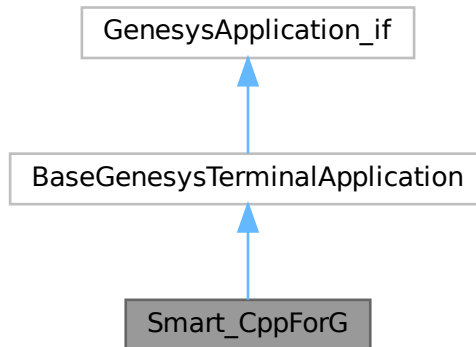
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

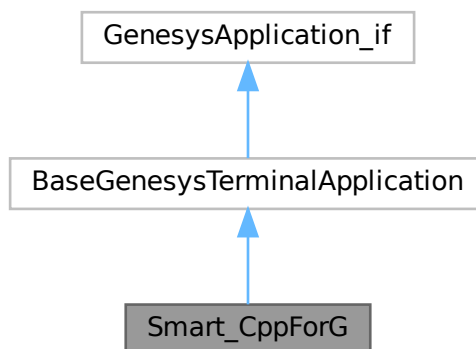
- applications/modelSpecific/arenaSmarts/Smart\_ContinuousFlowEntities.h
- applications/modelSpecific/arenaSmarts/Smart\_ContinuousFlowEntities.cpp

## 15.851 Smart\_CppForG Class Reference

Inheritance diagram for Smart\_CppForG:



Collaboration diagram for Smart\_CppForG:



### Public Member Functions

- virtual int `main` (int argc, char \*\*argv) override

### Public Member Functions inherited from `BaseGenesysTerminalApplication`

- virtual void `Simulate` (`Simulator` \*genesys, `Model` \*model, `PluginManager` \*plugins)
- void `setDefaultTraceHandlers` (`TraceManager` \*tm)
- void `setDefaultEventHandlers` (`OnEventManager` \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from `BaseGenesysTerminalApplication`

- virtual void `traceHandler` (`TraceEvent` e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.851.1 Member Function Documentation

### 15.851.1.1 main()

```
int Smart_CppForG::main (
 int argc,
 char ** argv) [override], [virtual]
```

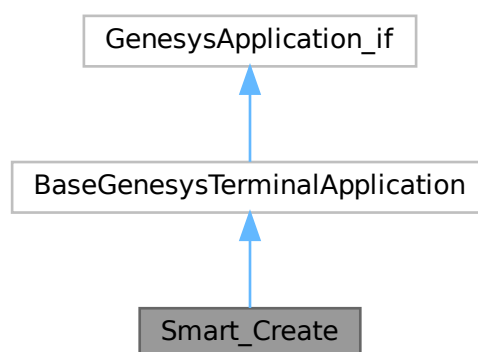
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

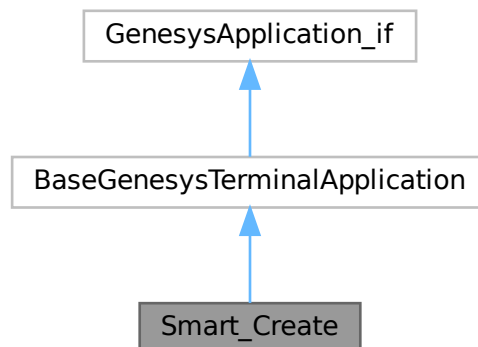
- applications/modelSpecific/smarts/ExternalIntegration/Smart\_CppForG.h
- applications/modelSpecific/smarts/ExternalIntegration/Smart\_CppForG.cpp

## 15.852 Smart\_Create Class Reference

Inheritance diagram for Smart\_Create:



Collaboration diagram for Smart\_Create:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)



## 15.852.1 Member Function Documentation

### 15.852.1.1 main()

```
int Smart_Create::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

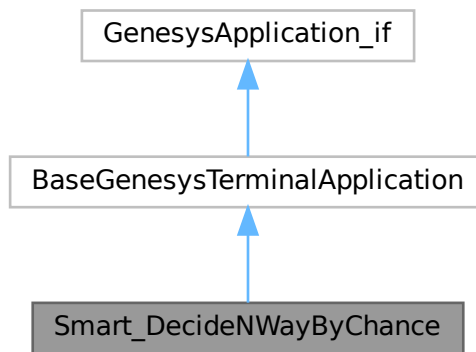
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

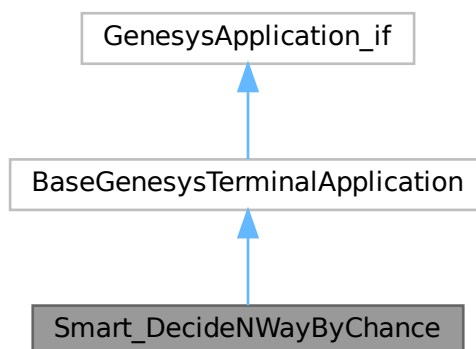
- applications/modelSpecific/arenaSmarts/Smart\_Create.h
- applications/modelSpecific/arenaSmarts/Smart\_Create.cpp

## 15.853 Smart\_DecideNWayByChance Class Reference

Inheritance diagram for Smart\_DecideNWayByChance:



Collaboration diagram for Smart\_DecideNWayByChance:



## Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

## Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

## Additional Inherited Members

## Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.853.1 Member Function Documentation

### 15.853.1.1 [main\(\)](#)

```
int Smart_DecideNWayByChance::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It create a new simulator instance, builds a simulation model and then simulate that model.

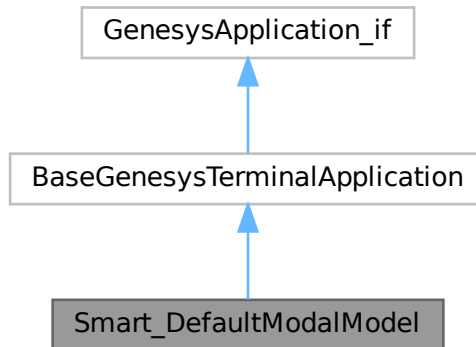
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

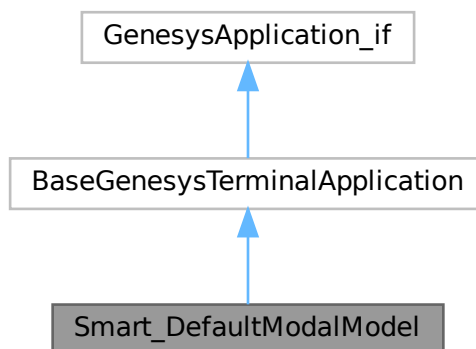
- applications/modelSpecific/arenaSmarts/Smart\_DecideNWayByChance.h
- applications/modelSpecific/arenaSmarts/Smart\_DecideNWayByChance.cpp

## 15.854 Smart\_DefaultModalModel Class Reference

Inheritance diagram for Smart\_DefaultModalModel:



Collaboration diagram for Smart\_DefaultModalModel:



### Public Member Functions

- virtual int `main` (int argc, char \*\*argv) override

### Public Member Functions inherited from `BaseGenesysTerminalApplication`

- virtual void `Simulate` (`Simulator` \*genesys, `Model` \*model, `PluginManager` \*plugins)
- void `setDefaultTraceHandlers` (`TraceManager` \*tm)
- void `setDefaultEventHandlers` (`OnEventManager` \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from `BaseGenesysTerminalApplication`

- virtual void `traceHandler` (`TraceEvent` e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.854.1 Member Function Documentation

### 15.854.1.1 main()

```
int Smart_DefaultModalModel::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

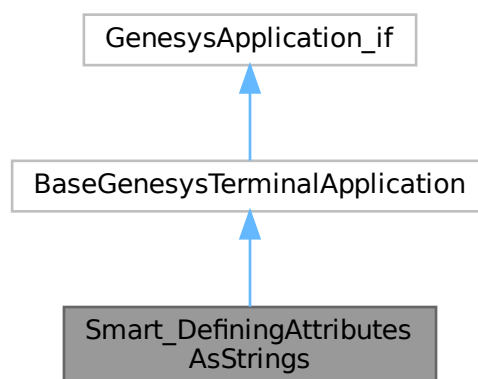
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

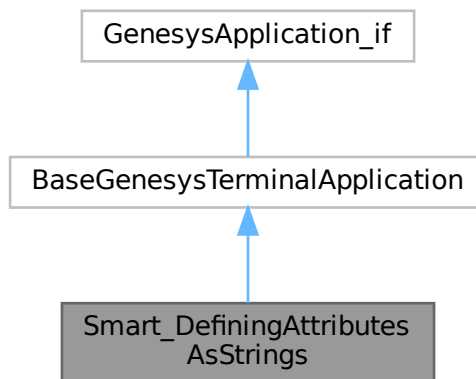
- applications/modelSpecific/smarts/ModalModel/Smart\_DefaultModalModel.h
- applications/modelSpecific/smarts/ModalModel/Smart\_DefaultModalModel.cpp

## 15.855 Smart\_DefiningAttributesAsStrings Class Reference

Inheritance diagram for Smart\_DefiningAttributesAsStrings:



Collaboration diagram for Smart\_DefiningAttributesAsStrings:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.855.1 Member Function Documentation

### 15.855.1.1 main()

```
int Smart_DefiningAttributesAsStrings::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

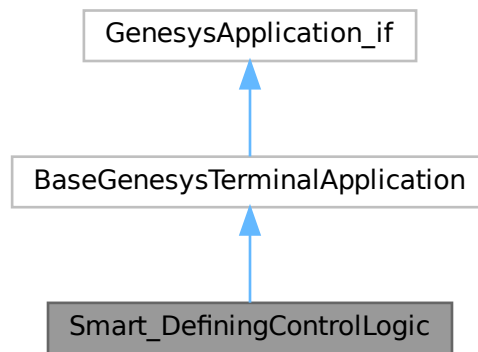
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

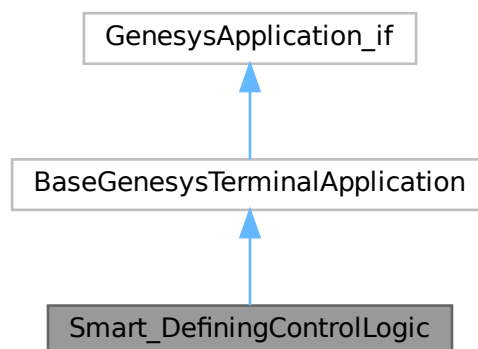
- applications/modelSpecific/arenaSmarts/Smart\_DefiningAttributesAsStrings.h
- applications/modelSpecific/arenaSmarts/Smart\_DefiningAttributesAsStrings.cpp

## 15.856 Smart\_DefiningControlLogic Class Reference

Inheritance diagram for Smart\_DefiningControlLogic:



Collaboration diagram for Smart\_DefiningControlLogic:



**Public Member Functions**

- virtual int [main](#) (int argc, char \*\*argv) override

**Public Member Functions inherited from [BaseGenesysTerminalApplication](#)**

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

**Additional Inherited Members****Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)**

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

**15.856.1 Member Function Documentation****15.856.1.1 main()**

```
int Smart_DefiningControlLogic::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

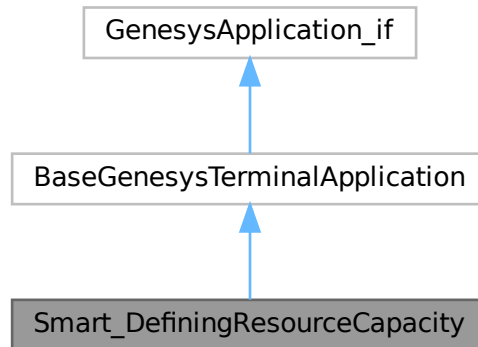
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

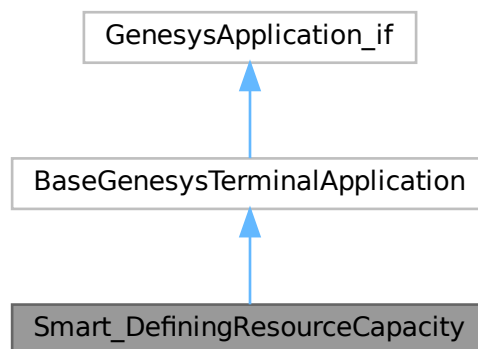
- applications/modelSpecific/arenaSmarts/Smart\_DefiningControlLogic.h
- applications/modelSpecific/arenaSmarts/Smart\_DefiningControlLogic.cpp

## 15.857 Smart\_DefiningResourceCapacity Class Reference

Inheritance diagram for Smart\_DefiningResourceCapacity:



Collaboration diagram for Smart\_DefiningResourceCapacity:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)



- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.857.1 Member Function Documentation

### 15.857.1.1 main()

```
int Smart_DefiningResourceCapacity::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

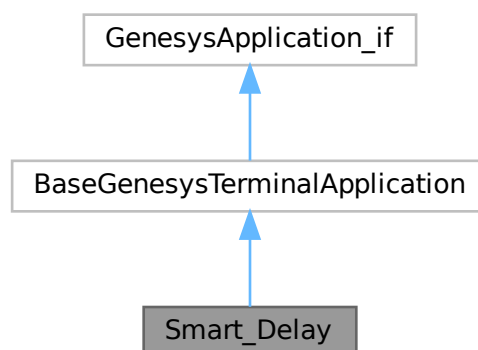
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

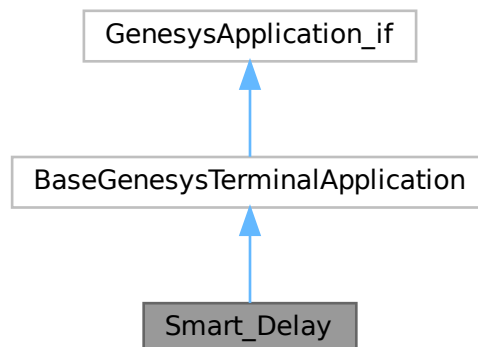
- applications/modelSpecific/arenaSmarts/Smart\_DefiningResourceCapacity.h
- applications/modelSpecific/arenaSmarts/Smart\_DefiningResourceCapacity.cpp

## 15.858 Smart\_Delay Class Reference

Inheritance diagram for Smart\_Delay:



Collaboration diagram for Smart\_Delay:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.858.1 Member Function Documentation

### 15.858.1.1 main()

```
int Smart_Delay::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

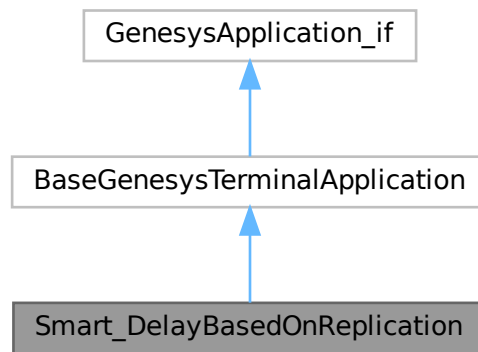
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/smarts/DiscreteProcessing/Smart\_Delay.h
- applications/modelSpecific/smarts/DiscreteProcessing/Smart\_Delay.cpp

## 15.859 Smart\_DelayBasedOnReplication Class Reference

Inheritance diagram for Smart\_DelayBasedOnReplication:





- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.859.1 Member Function Documentation

### 15.859.1.1 main()

```
int Smart_DelayBasedOnReplication::main (
 int argc,
 char ** argv) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

### 15.859.1.2 TestEventHandler()

```
void Smart_DelayBasedOnReplication::TestEventHandler (
 SimulationEvent * re)
```

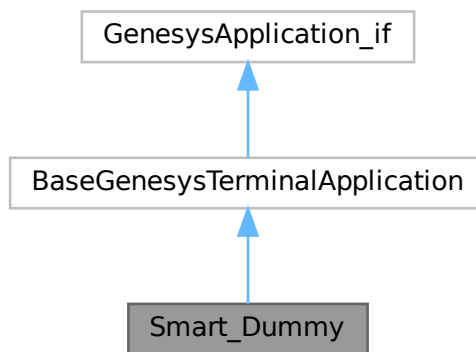
This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

The documentation for this class was generated from the following files:

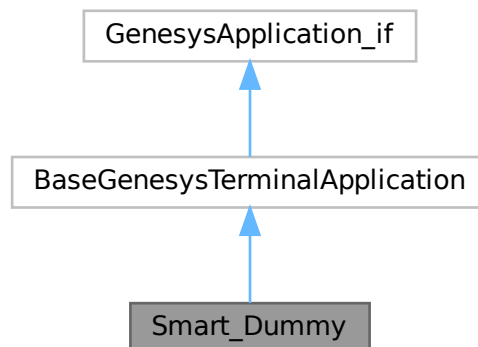
- applications/modelSpecific/arenaSmarts/Smart\_DelayBasedOnReplication.h
- applications/modelSpecific/arenaSmarts/Smart\_DelayBasedOnReplication.cpp

## 15.860 Smart\_Dummy Class Reference

Inheritance diagram for Smart\_Dummy:



Collaboration diagram for Smart\_Dummy:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.860.1 Member Function Documentation

### 15.860.1.1 main()

```
int Smart_Dummy::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

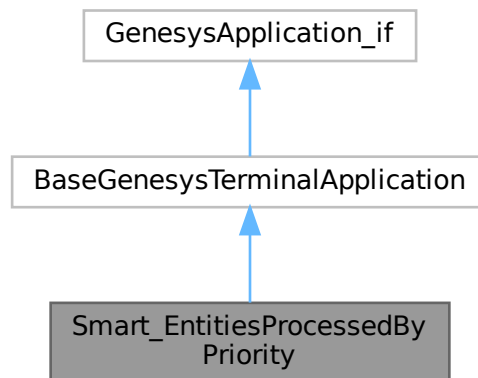
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

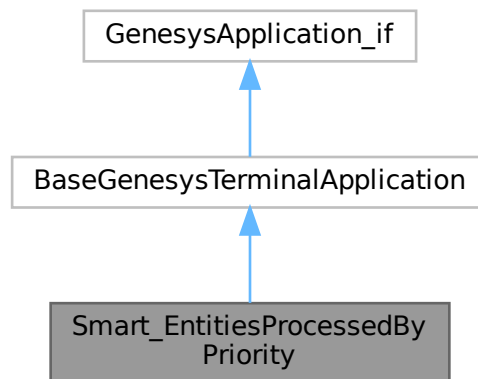
- applications/modelSpecific/smarts/ModalModel/Smart\_Dummy.h
- applications/modelSpecific/smarts/ModalModel/Smart\_Dummy.cpp

## 15.861 Smart\_EntitiesProcessedByPriority Class Reference

Inheritance diagram for Smart\_EntitiesProcessedByPriority:



Collaboration diagram for Smart\_EntitiesProcessedByPriority:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)



## 15.861.1 Member Function Documentation

### 15.861.1.1 main()

```
int Smart_EntitiesProcessedByPriority::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

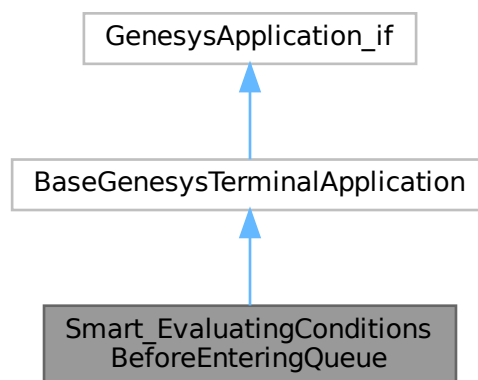
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

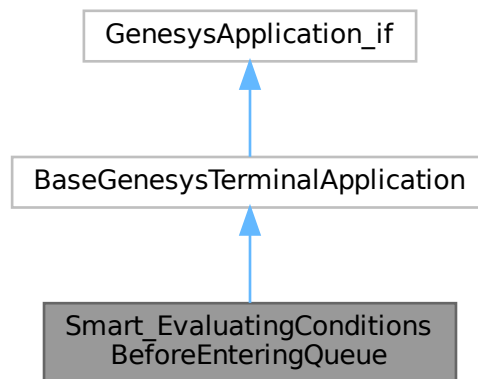
- applications/modelSpecific/arenaSmarts/Smart\_EntitiesProcessedByPriority.h
- applications/modelSpecific/arenaSmarts/Smart\_EntitiesProcessedByPriority.cpp

## 15.862 Smart\_EvaluatingConditionsBeforeEnteringQueue Class Reference

Inheritance diagram for Smart\_EvaluatingConditionsBeforeEnteringQueue:



Collaboration diagram for Smart\_EvaluatingConditionsBeforeEnteringQueue:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.862.1 Member Function Documentation

### 15.862.1.1 main()

```
int Smart_EvaluatingConditionsBeforeEnteringQueue::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

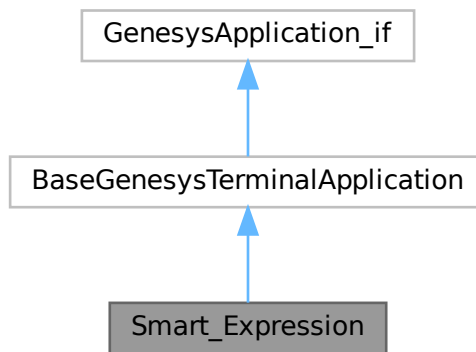
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

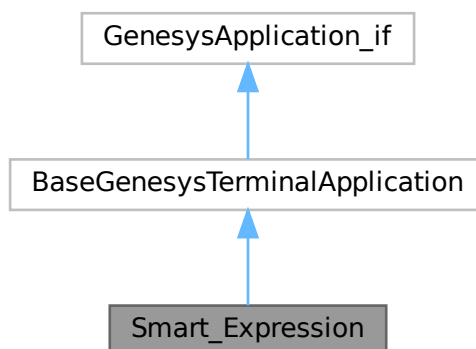
- applications/modelSpecific/arenaSmarts/Smart\_EvaluatingConditionsBeforeEnteringQueue.h
- applications/modelSpecific/arenaSmarts/Smart\_EvaluatingConditionsBeforeEnteringQueue.cpp

## 15.863 Smart\_Expression Class Reference

Inheritance diagram for Smart\_Expression:



Collaboration diagram for Smart\_Expression:



## Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

## Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

## Additional Inherited Members

## Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.863.1 Member Function Documentation

### 15.863.1.1 [main\(\)](#)

```
int Smart_Expression::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

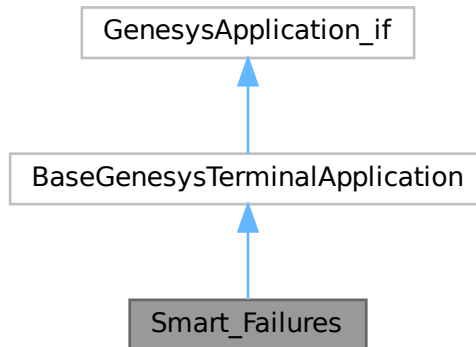
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

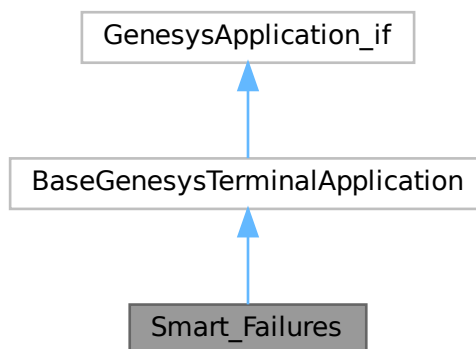
- applications/modelSpecific/arenaSmarts/Smart\_Expression.h
- applications/modelSpecific/arenaSmarts/Smart\_Expression.cpp

## 15.864 Smart\_Failures Class Reference

Inheritance diagram for Smart\_Failures:



Collaboration diagram for Smart\_Failures:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.864.1 Member Function Documentation

### 15.864.1.1 main()

```
int Smart_Failures::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

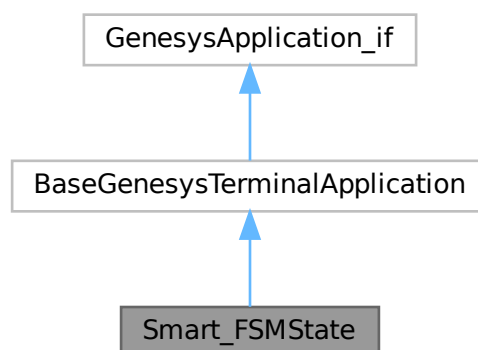
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

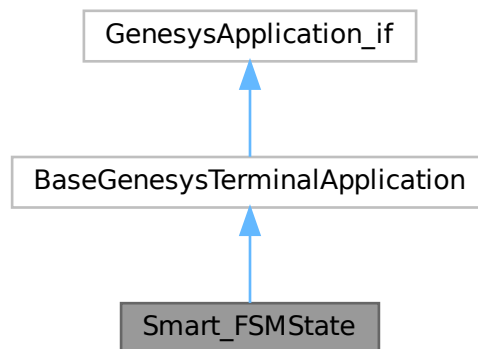
- applications/modelSpecific/smarts/DiscreteProcessing/Smart\_Failures.h
- applications/modelSpecific/smarts/DiscreteProcessing/Smart\_Failures.cpp

## 15.865 Smart\_FSMState Class Reference

Inheritance diagram for Smart\_FSMState:



Collaboration diagram for Smart\_FSMState:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.865.1 Member Function Documentation

### 15.865.1.1 main()

```
int Smart_FSMState::main (
 int argc,
 char ** argv) [override], [virtual]
```

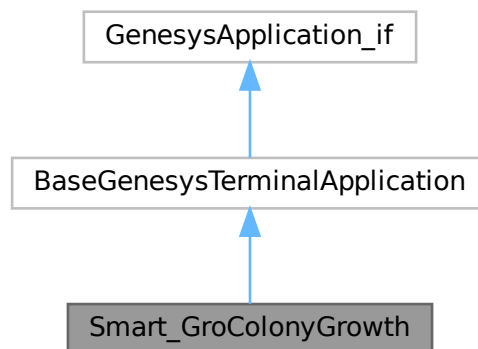
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

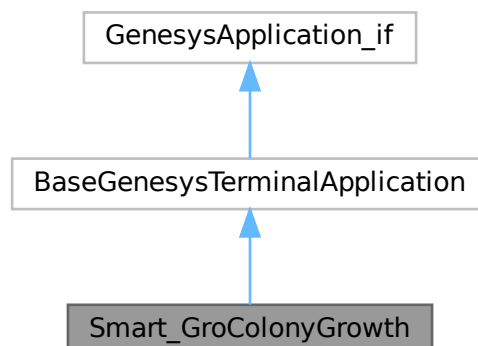
- applications/modelSpecific/smarts/ModalModel/Smart\_FSMState.h
- applications/modelSpecific/smarts/ModalModel/Smart\_FSMState.cpp

## 15.866 Smart\_GroColonyGrowth Class Reference

Inheritance diagram for Smart\_GroColonyGrowth:



Collaboration diagram for Smart\_GroColonyGrowth:





**Public Member Functions**

- virtual int [main](#) (int argc, char \*\*argv) override

**Public Member Functions inherited from [BaseGenesysTerminalApplication](#)**

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

**Additional Inherited Members****Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)**

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

**15.866.1 Member Function Documentation****15.866.1.1 main()**

```
int Smart_GroColonyGrowth::main (
 int argc,
 char ** argv) [override], [virtual]
```

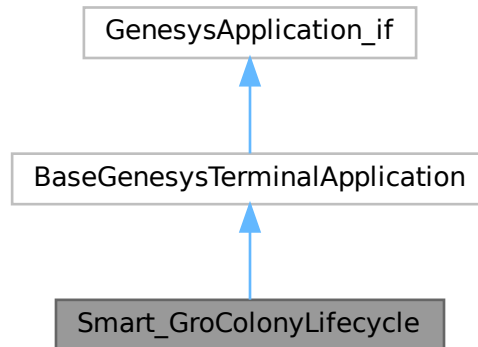
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

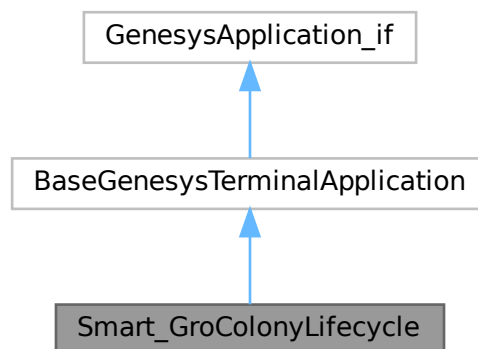
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart\_GroColonyGrowth.h
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart\_GroColonyGrowth.cpp

## 15.867 Smart\_GroColonyLifecycle Class Reference

Inheritance diagram for Smart\_GroColonyLifecycle:



Collaboration diagram for Smart\_GroColonyLifecycle:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.867.1 Member Function Documentation

### 15.867.1.1 main()

```
int Smart_GroColonyLifecycle::main (
 int argc,
 char ** argv) [override], [virtual]
```

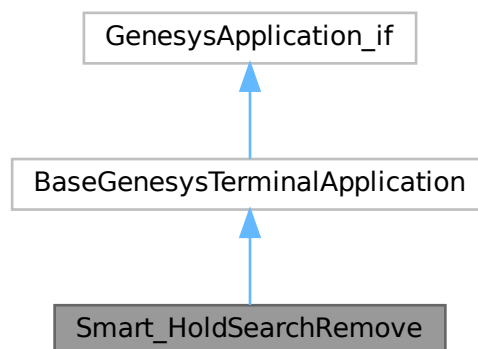
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

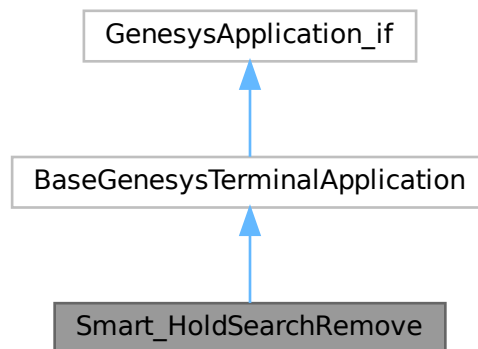
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart\_GroColonyLifecycle.h
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart\_GroColonyLifecycle.cpp

## 15.868 Smart\_HoldSearchRemove Class Reference

Inheritance diagram for Smart\_HoldSearchRemove:



Collaboration diagram for Smart\_HoldSearchRemove:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.868.1 Member Function Documentation

### 15.868.1.1 main()

```
int Smart_HoldSearchRemove::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

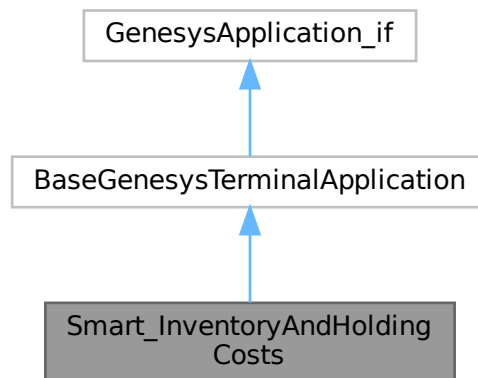
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

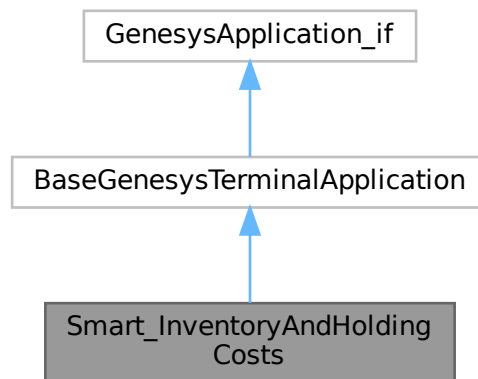
- applications/modelSpecific/smarts/DiscreteProcessing/Smart\_HoldSearchRemove.h
- applications/modelSpecific/smarts/DiscreteProcessing/Smart\_HoldSearchRemove.cpp

## 15.869 Smart\_InventoryAndHoldingCosts Class Reference

Inheritance diagram for Smart\_InventoryAndHoldingCosts:



Collaboration diagram for Smart\_InventoryAndHoldingCosts:



#### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

#### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

#### Additional Inherited Members

#### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.869.1 Member Function Documentation

### 15.869.1.1 main()

```
int Smart_InventoryAndHoldingCosts::main (
 int argc,
 char ** argv) [override], [virtual]
```

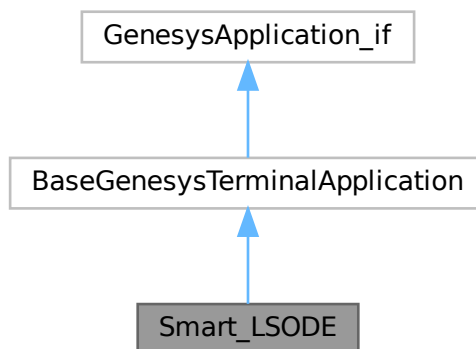
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

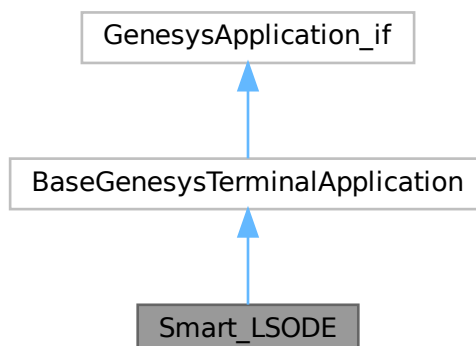
- applications/modelSpecific/arenaSmarts/Smart\_InventoryAndHoldingCosts.h
- applications/modelSpecific/arenaSmarts/Smart\_InventoryAndHoldingCosts.cpp

## 15.870 Smart\_LCODE Class Reference

Inheritance diagram for Smart\_LCODE:



Collaboration diagram for Smart\_LCODE:



## Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

## Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

## Additional Inherited Members

## Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.870.1 Member Function Documentation

### 15.870.1.1 [main\(\)](#)

```
int Smart_LSODE::main (
 int argc,
 char ** argv) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

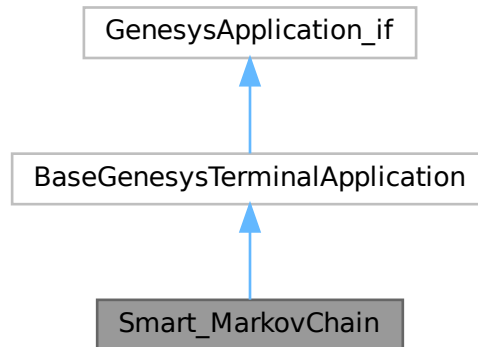
The documentation for this class was generated from the following files:

- applications/modelSpecific/smarts/Continuous/Smart\_LSODE.h
- applications/modelSpecific/smarts/Continuous/Smart\_LSODE.cpp

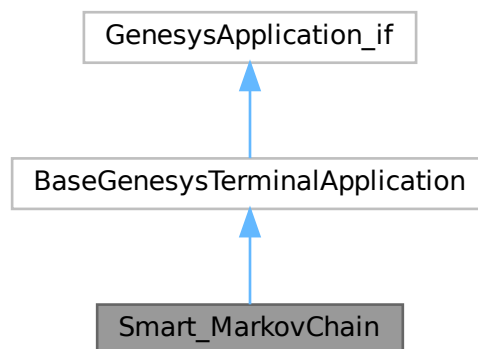


## 15.871 Smart\_MarkovChain Class Reference

Inheritance diagram for Smart\_MarkovChain:



Collaboration diagram for Smart\_MarkovChain:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.871.1 Member Function Documentation

### 15.871.1.1 main()

```
int Smart_MarkovChain::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

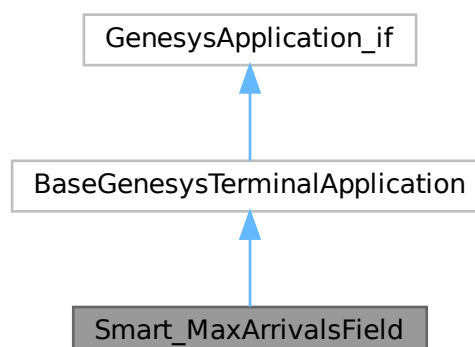
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

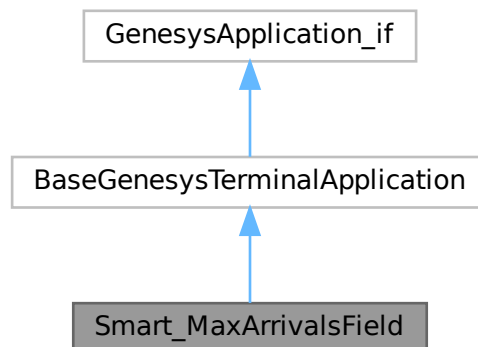
- applications/modelSpecific/smarts/AnalyticalModeling/Smart\_MarkovChain.h
- applications/modelSpecific/smarts/AnalyticalModeling/Smart\_MarkovChain.cpp

## 15.872 Smart\_MaxArrivalsField Class Reference

Inheritance diagram for Smart\_MaxArrivalsField:



Collaboration diagram for Smart\_MaxArrivalsField:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.872.1 Member Function Documentation

### 15.872.1.1 main()

```
int Smart_MaxArrivalsField::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

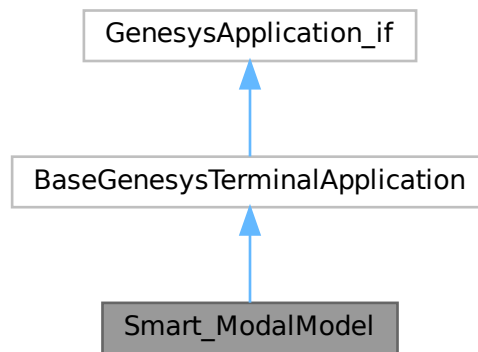
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

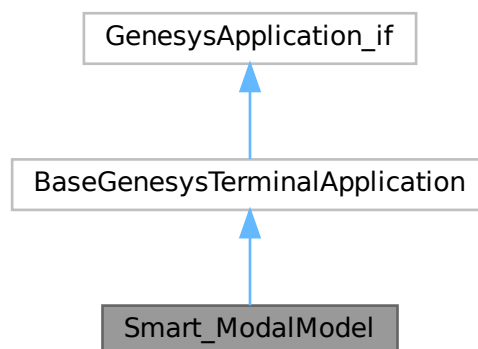
- applications/modelSpecific/arenaSmarts/Smart\_MaxArrivalsField.h
- applications/modelSpecific/arenaSmarts/Smart\_MaxArrivalsField.cpp

## 15.873 Smart\_ModalModel Class Reference

Inheritance diagram for Smart\_ModalModel:



Collaboration diagram for Smart\_ModalModel:



**Public Member Functions**

- virtual int [main](#) (int argc, char \*\*argv) override

**Public Member Functions inherited from [BaseGenesysTerminalApplication](#)**

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

**Additional Inherited Members****Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)**

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

**15.873.1 Member Function Documentation****15.873.1.1 main()**

```
int Smart_ModalModel::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

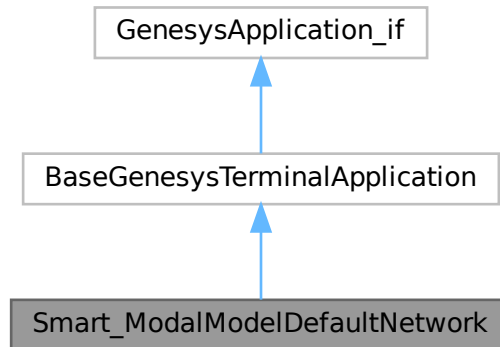
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

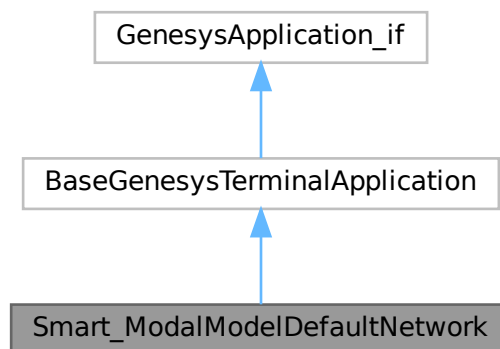
- applications/modelSpecific/smarts/ModalModel/Smart\_Modalmodel.h
- applications/modelSpecific/smarts/ModalModel/Smart\_Modalmodel.cpp

## 15.874 Smart\_ModelModelDefaultNetwork Class Reference

Inheritance diagram for Smart\_ModelModelDefaultNetwork:



Collaboration diagram for Smart\_ModelModelDefaultNetwork:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.874.1 Member Function Documentation

### 15.874.1.1 main()

```
int Smart_ModalModelDefaultNetwork::main (
 int argc,
 char ** argv) [override], [virtual]
```

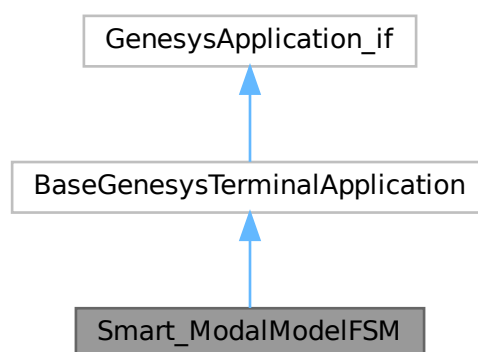
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

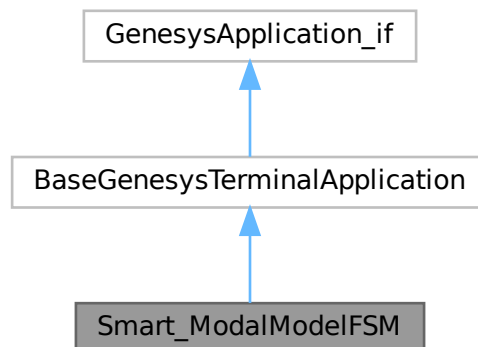
- applications/modelSpecific/smarts/ModalModel/Smart\_ModalModelDefaultNetwork.h
- applications/modelSpecific/smarts/ModalModel/Smart\_ModalModelDefaultNetwork.cpp

## 15.875 Smart\_ModalModelFSM Class Reference

Inheritance diagram for Smart\_ModalModelFSM:



Collaboration diagram for Smart\_ModalModelFSM:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)



## 15.875.1 Member Function Documentation

### 15.875.1.1 main()

```
int Smart_ModalModelFSM::main (
 int argc,
 char ** argv) [override], [virtual]
```

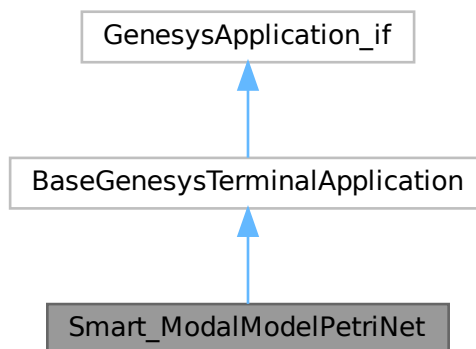
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

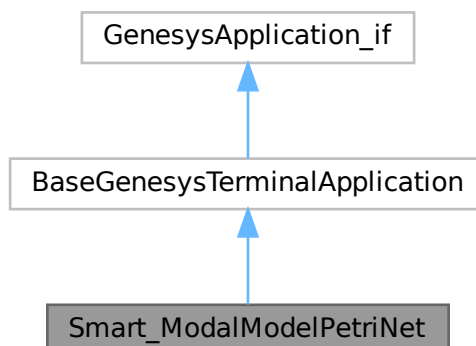
- applications/modelSpecific/smarts/ModalModel/Smart\_ModalModelFSM.h
- applications/modelSpecific/smarts/ModalModel/Smart\_ModalModelFSM.cpp

## 15.876 Smart\_ModalModelPetriNet Class Reference

Inheritance diagram for Smart\_ModalModelPetriNet:



Collaboration diagram for Smart\_ModalModelPetriNet:



## Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

## Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

## Additional Inherited Members

## Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.876.1 Member Function Documentation

### 15.876.1.1 [main\(\)](#)

```
int Smart_ModalModelPetriNet::main (
 int argc,
 char ** argv) [override], [virtual]
```

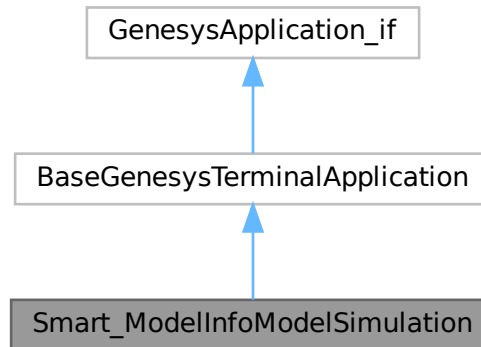
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

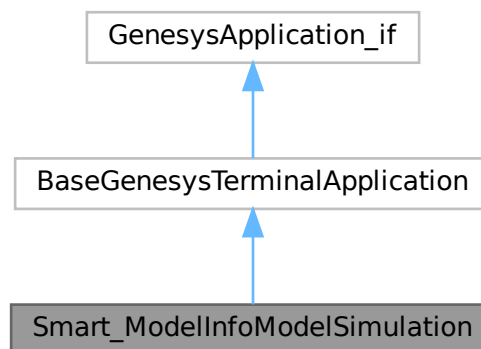
- applications/modelSpecific/smarts/ModalModel/Smart\_ModalModelPetriNet.h
- applications/modelSpecific/smarts/ModalModel/Smart\_ModalModelPetriNet.cpp

## 15.877 Smart\_ModelInfoModelSimulation Class Reference

Inheritance diagram for Smart\_ModelInfoModelSimulation:



Collaboration diagram for Smart\_ModelInfoModelSimulation:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.877.1 Member Function Documentation

### 15.877.1.1 main()

```
int Smart_ModelInfoModelSimulation::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

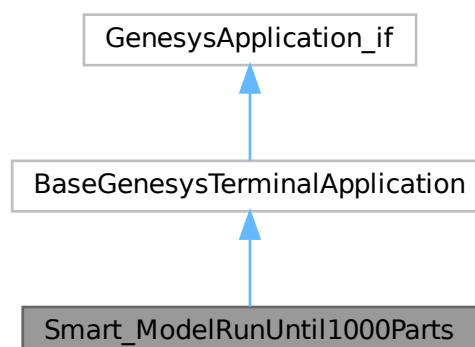
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

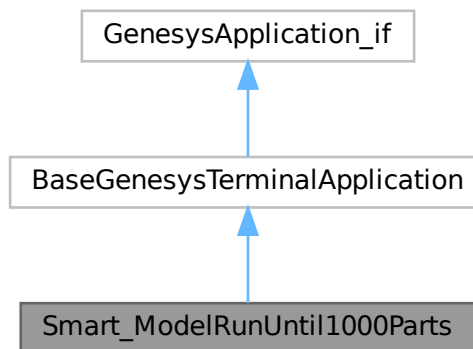
- applications/modelSpecific/smarts/Logic/Smart\_ModelInfoModelSimulation.h
- applications/modelSpecific/smarts/Logic/Smart\_ModelInfoModelSimulation.cpp

## 15.878 Smart\_ModelRunUntil1000Parts Class Reference

Inheritance diagram for Smart\_ModelRunUntil1000Parts:



Collaboration diagram for Smart\_ModelRunUntil1000Parts:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.878.1 Member Function Documentation

### 15.878.1.1 main()

```
int Smart_ModelRunUntil1000Parts::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It create a new simulator instance, builds a simulation model and then simulate that model.

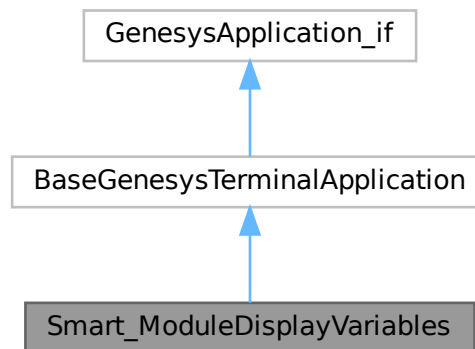
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

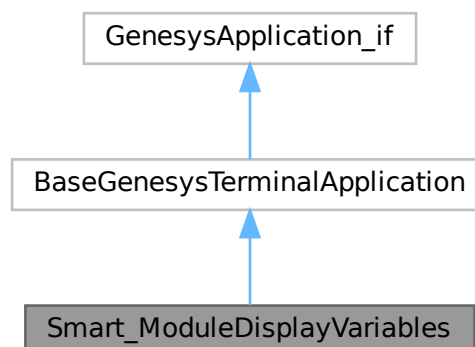
- applications/modelSpecific/arenaSmarts/Smart\_ModelRunUntil1000Parts.h
- applications/modelSpecific/arenaSmarts/Smart\_ModelRunUntil1000Parts.cpp

## 15.879 Smart\_ModuleDisplayVariables Class Reference

Inheritance diagram for Smart\_ModuleDisplayVariables:



Collaboration diagram for Smart\_ModuleDisplayVariables:



## Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

## Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

## Additional Inherited Members

## Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.879.1 Member Function Documentation

### 15.879.1.1 [main\(\)](#)

```
int Smart_ModuleDisplayVariables::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

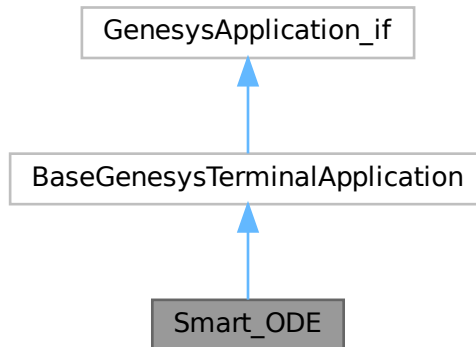
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

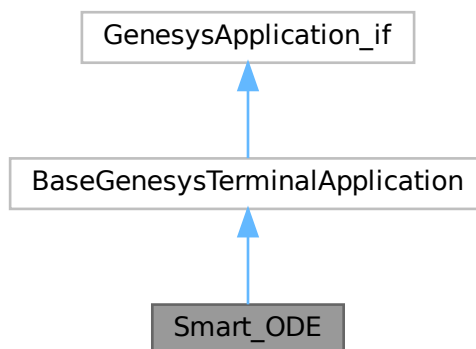
- applications/modelSpecific/arenaSmarts/Smart\_ModuleDisplayVariables.h
- applications/modelSpecific/arenaSmarts/Smart\_ModuleDisplayVariables.cpp

## 15.880 Smart\_ODE Class Reference

Inheritance diagram for Smart\_ODE:



Collaboration diagram for Smart\_ODE:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)



- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.880.1 Member Function Documentation

### 15.880.1.1 main()

```
int Smart_ODE::main (
 int argc,
 char ** argv) [override], [virtual]
```

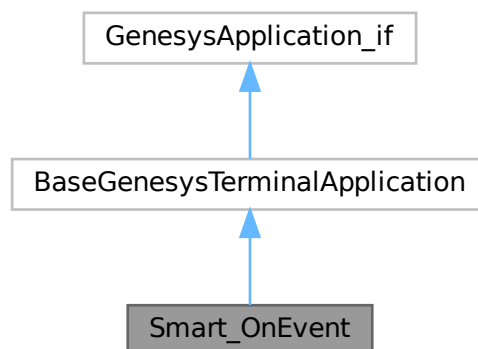
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

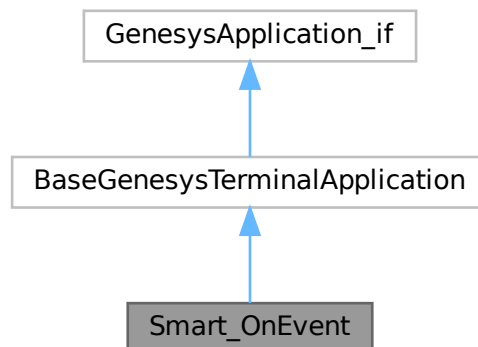
- applications/modelSpecific/smarts/Continuous/Smart\_ODE.h
- applications/modelSpecific/smarts/Continuous/Smart\_ODE.cpp

## 15.881 Smart\_OnEvent Class Reference

Inheritance diagram for Smart\_OnEvent:



Collaboration diagram for Smart\_OnEvent:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override
- virtual void [onBreakpointHandler](#) (SimulationEvent \*re) override
- virtual void [onEntityCreateHandler](#) (SimulationEvent \*re) override
- virtual void [onEntityMoveHandler](#) (SimulationEvent \*re) override
- virtual void [onSimulationStartHandler](#) (SimulationEvent \*re) override
- virtual void [onReplicationStartHandler](#) (SimulationEvent \*re) override
- virtual void [onReplicationStepHandler](#) (SimulationEvent \*re) override
- virtual void [onProcessEventHandler](#) (SimulationEvent \*re) override
- virtual void [onReplicationEndHandler](#) (SimulationEvent \*re) override
- virtual void [onSimulationEndHandler](#) (SimulationEvent \*re) override
- virtual void [onSimulationPausedHandler](#) (SimulationEvent \*re) override
- virtual void [onSimulationResumeHandler](#) (SimulationEvent \*re) override
- virtual void [onEntityRemoveHandler](#) (SimulationEvent \*re) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** (Simulator \*genesys, Model \*model, PluginManager \*plugins)
- void **setDefaultTraceHandlers** (TraceManager \*tm)
- void [setDefaultEventHandlers](#) (OnEventManager \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** (TraceEvent e)
- virtual void **traceErrorHandler** (TraceErrorEvent e)
- virtual void **traceReportHandler** (TraceEvent e)
- virtual void **traceSimulationHandler** (TraceSimulationEvent e)
- virtual void **onAfterProcessEventHandler** (SimulationEvent \*re)

## 15.881.1 Member Function Documentation

### 15.881.1.1 main()

```
int Smart_OnEvent::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

Reimplemented from [BaseGenesysTerminalApplication](#).

### 15.881.1.2 onBreakpointHandler()

```
void Smart_OnEvent::onBreakpointHandler (
 SimulationEvent * re) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

### 15.881.1.3 onEntityCreateHandler()

```
void Smart_OnEvent::onEntityCreateHandler (
 SimulationEvent * re) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

### 15.881.1.4 onEntityMoveHandler()

```
void Smart_OnEvent::onEntityMoveHandler (
 SimulationEvent * re) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

### 15.881.1.5 onEntityRemoveHandler()

```
void Smart_OnEvent::onEntityRemoveHandler (
 SimulationEvent * re) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

### 15.881.1.6 onProcessEventHandler()

```
void Smart_OnEvent::onProcessEventHandler (
 SimulationEvent * re) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

### 15.881.1.7 onReplicationEndHandler()

```
void Smart_OnEvent::onReplicationEndHandler (
 SimulationEvent * re) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

### 15.881.1.8 onReplicationStartHandler()

```
void Smart_OnEvent::onReplicationStartHandler (
 SimulationEvent * re) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

### 15.881.1.9 onReplicationStepHandler()

```
void Smart_OnEvent::onReplicationStepHandler (
 SimulationEvent * re) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

**15.881.1.10 onSimulationEndHandler()**

```
void Smart_OnEvent::onSimulationEndHandler (
 SimulationEvent * re) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

**15.881.1.11 onSimulationPausedHandler()**

```
void Smart_OnEvent::onSimulationPausedHandler (
 SimulationEvent * re) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

**15.881.1.12 onSimulationResumeHandler()**

```
void Smart_OnEvent::onSimulationResumeHandler (
 SimulationEvent * re) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

**15.881.1.13 onSimulationStartHandler()**

```
void Smart_OnEvent::onSimulationStartHandler (
 SimulationEvent * re) [override], [virtual]
```

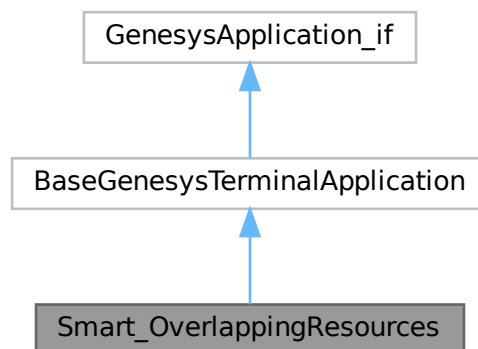
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

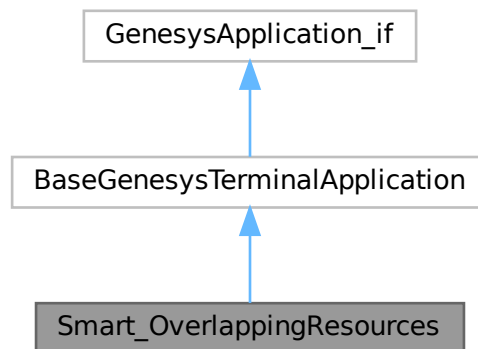
- applications/modelSpecific/smarts/Synchronization/Smart\_OnEvent.h
- applications/modelSpecific/smarts/Synchronization/Smart\_OnEvent.cpp

**15.882 Smart\_OverlappingResources Class Reference**

Inheritance diagram for Smart\_OverlappingResources:



Collaboration diagram for Smart\_OverlappingResources:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.882.1 Member Function Documentation

### 15.882.1.1 main()

```
int Smart_OverlappingResources::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

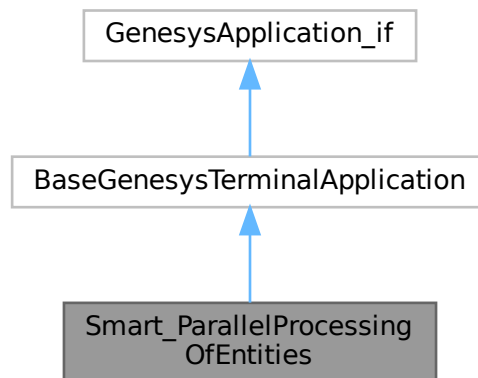
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

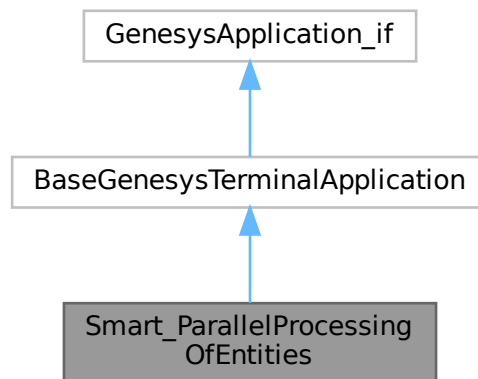
- applications/modelSpecific/arenaSmarts/Smart\_OverlappingResources.h
- applications/modelSpecific/arenaSmarts/Smart\_OverlappingResources.cpp

## 15.883 Smart\_ParallelProcessingOfEntities Class Reference

Inheritance diagram for Smart\_ParallelProcessingOfEntities:



Collaboration diagram for Smart\_ParallelProcessingOfEntities:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.883.1 Member Function Documentation

### 15.883.1.1 main()

```
int Smart_ParallelProcessingOfEntities::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

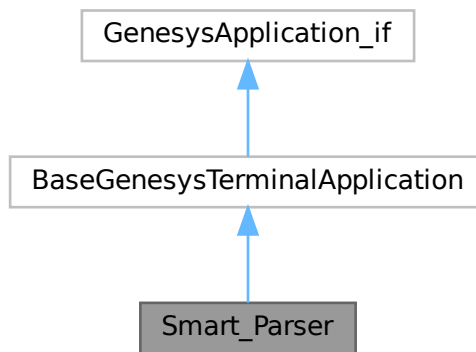
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

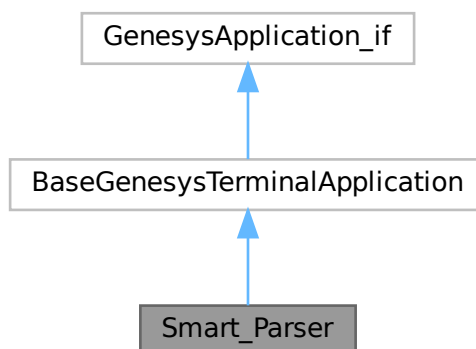
- applications/modelSpecific/arenaSmarts/Smart\_ParallelProcessingOfEntities.h
- applications/modelSpecific/arenaSmarts/Smart\_ParallelProcessingOfEntities.cpp

## 15.884 Smart\_Parser Class Reference

Inheritance diagram for Smart\_Parser:



Collaboration diagram for Smart\_Parser:





## Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

## Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

## Additional Inherited Members

## Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.884.1 Member Function Documentation

### 15.884.1.1 [main\(\)](#)

```
int Smart_Parser::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

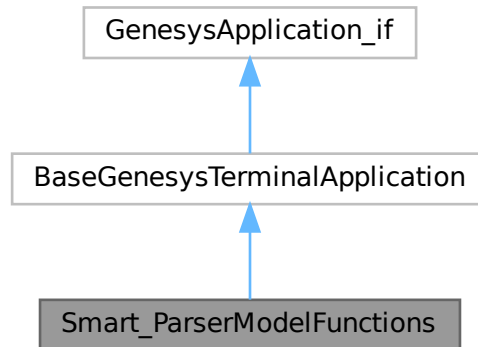
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

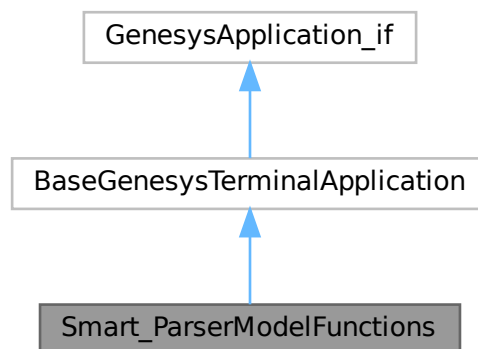
- applications/modelSpecific/smarts/Logic/Smart\_Parser.h
- applications/modelSpecific/smarts/Logic/Smart\_Parser.cpp

## 15.885 Smart\_ParserModelFunctions Class Reference

Inheritance diagram for Smart\_ParserModelFunctions:



Collaboration diagram for Smart\_ParserModelFunctions:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.885.1 Member Function Documentation

### 15.885.1.1 main()

```
int Smart_ParserModelFunctions::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

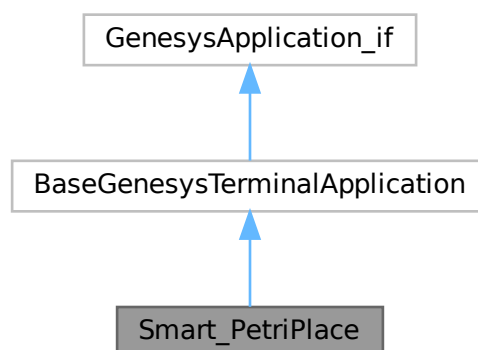
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

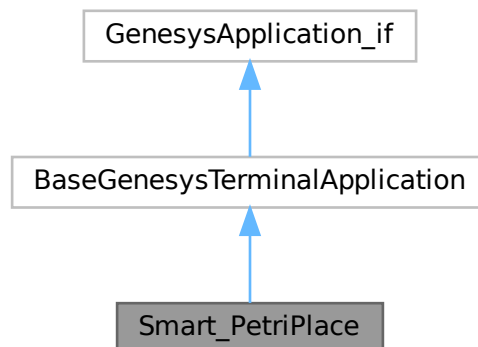
- applications/modelSpecific/smarts/Logic/Smart\_ParserModelFunctions.h
- applications/modelSpecific/smarts/Logic/Smart\_ParserModelFunctions.cpp

## 15.886 Smart\_PetriPlace Class Reference

Inheritance diagram for Smart\_PetriPlace:



Collaboration diagram for Smart\_PetriPlace:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.886.1 Member Function Documentation

### 15.886.1.1 main()

```
int Smart_PetriPlace::main (
 int argc,
 char ** argv) [override], [virtual]
```

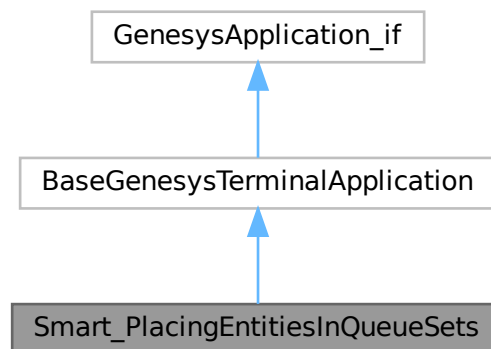
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

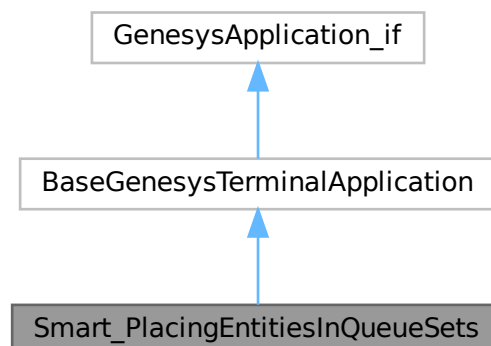
- applications/modelSpecific/smarts/ModalModel/Smart\_PetriPlace.h
- applications/modelSpecific/smarts/ModalModel/Smart\_PetriPlace.cpp

## 15.887 Smart\_PlacingEntitiesInQueueSets Class Reference

Inheritance diagram for Smart\_PlacingEntitiesInQueueSets:



Collaboration diagram for Smart\_PlacingEntitiesInQueueSets:



## Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

## Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

## Additional Inherited Members

## Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.887.1 Member Function Documentation

### 15.887.1.1 [main\(\)](#)

```
int Smart_PlacingEntitiesInQueueSets::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

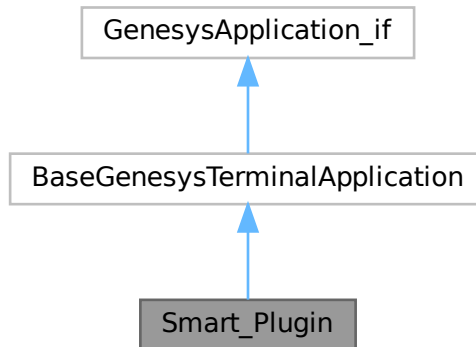
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

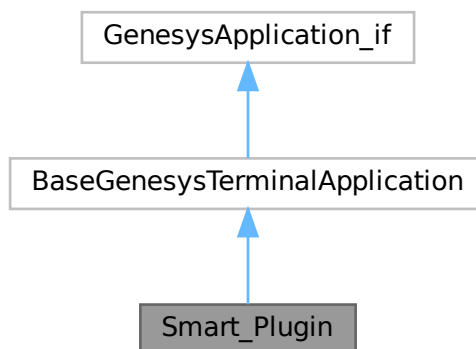
- applications/modelSpecific/arenaSmarts/Smart\_PlacingEntitiesInQueueSets.h
- applications/modelSpecific/arenaSmarts/Smart\_PlacingEntitiesInQueueSets.cpp

## 15.888 Smart\_Plugin Class Reference

Inheritance diagram for Smart\_Plugin:



Collaboration diagram for Smart\_Plugin:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.888.1 Member Function Documentation

### 15.888.1.1 main()

```
int Smart_Plugin::main (
 int argc,
 char ** argv) [override], [virtual]
```

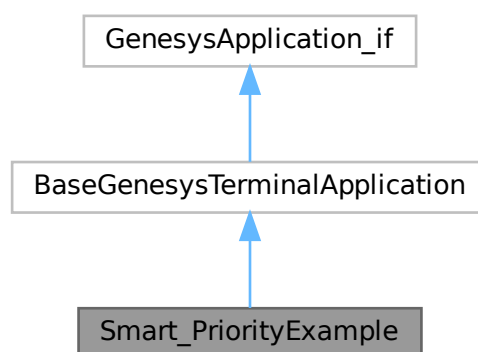
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/smarts/Logic/Smart\_Plugin.h
- applications/modelSpecific/smarts/Logic/Smart\_Plugin.cpp

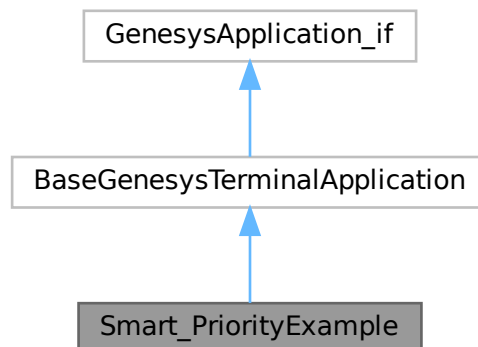
## 15.889 Smart\_PriorityExample Class Reference

Inheritance diagram for Smart\_PriorityExample:





Collaboration diagram for Smart\_PriorityExample:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.889.1 Member Function Documentation

### 15.889.1.1 main()

```
int Smart_PriorityExample::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

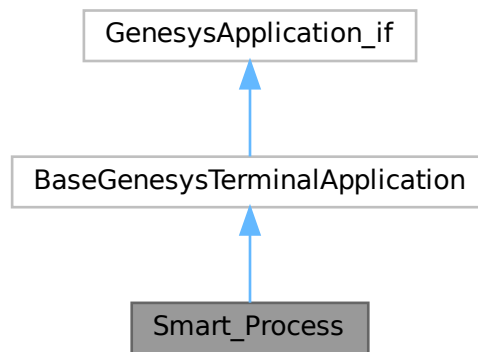
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

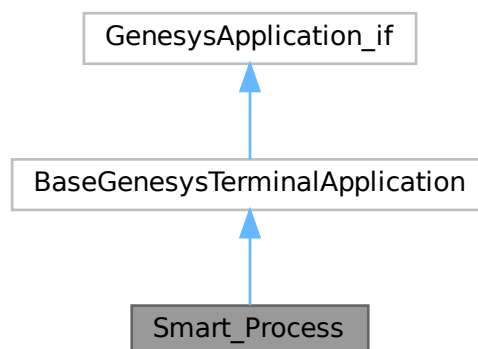
- applications/modelSpecific/arenaSmarts/Smart\_PriorityExample.h
- applications/modelSpecific/arenaSmarts/Smart\_PriorityExample.cpp

## 15.890 Smart\_Process Class Reference

Inheritance diagram for Smart\_Process:



Collaboration diagram for Smart\_Process:



**Public Member Functions**

- virtual int [main](#) (int argc, char \*\*argv) override

**Public Member Functions inherited from [BaseGenesysTerminalApplication](#)**

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

**Additional Inherited Members****Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)**

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

**15.890.1 Member Function Documentation****15.890.1.1 [main\(\)](#)**

```
int Smart_Process::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

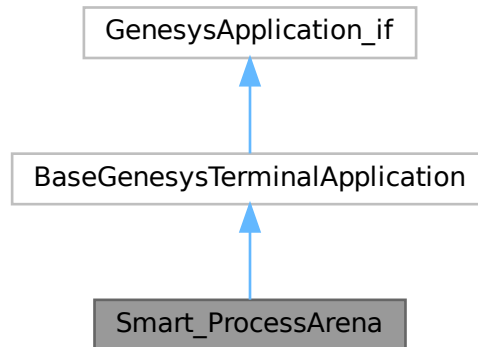
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

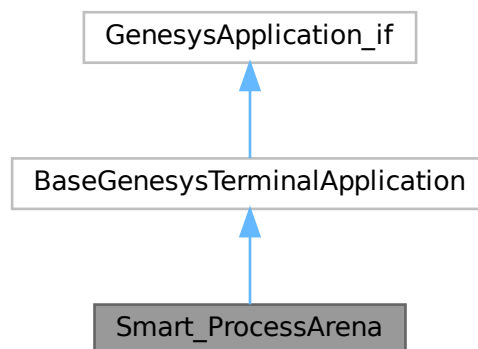
- applications/modelSpecific/smarts/DiscreteProcessing/Smart\_Process.h
- applications/modelSpecific/smarts/DiscreteProcessing/Smart\_Process.cpp

## 15.891 Smart\_ProcessArena Class Reference

Inheritance diagram for Smart\_ProcessArena:



Collaboration diagram for Smart\_ProcessArena:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.891.1 Member Function Documentation

### 15.891.1.1 main()

```
int Smart_ProcessArena::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

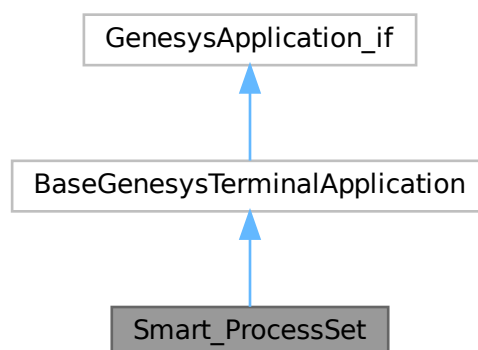
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

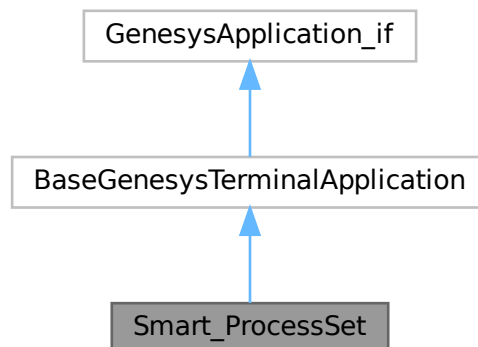
- applications/modelSpecific/arenaSmarts/Smart\_ProcessArena.h
- applications/modelSpecific/arenaSmarts/Smart\_ProcessArena.cpp

## 15.892 Smart\_ProcessSet Class Reference

Inheritance diagram for Smart\_ProcessSet:



Collaboration diagram for Smart\_ProcessSet:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.892.1 Member Function Documentation

### 15.892.1.1 main()

```
int Smart_ProcessSet::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

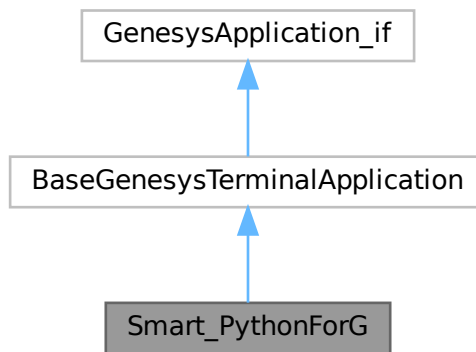
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

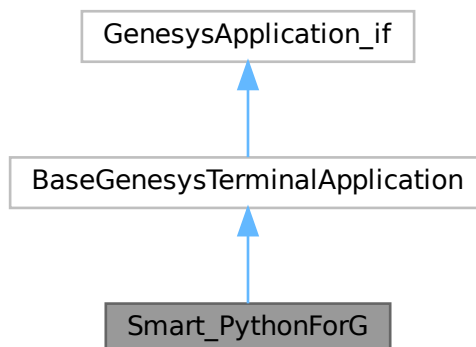
- applications/modelSpecific/smarts/DiscreteProcessing/Smart\_ProcessSet.h
- applications/modelSpecific/smarts/DiscreteProcessing/Smart\_ProcessSet.cpp

## 15.893 Smart\_PythonForG Class Reference

Inheritance diagram for Smart\_PythonForG:



Collaboration diagram for Smart\_PythonForG:



## Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

## Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

## Additional Inherited Members

## Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.893.1 Member Function Documentation

### 15.893.1.1 [main\(\)](#)

```
int Smart_PythonForG::main (
 int argc,
 char ** argv) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/smarts/ExternalIntegration/Smart\_PythonForG.h
- applications/modelSpecific/smarts/ExternalIntegration/Smart\_PythonForG.cpp

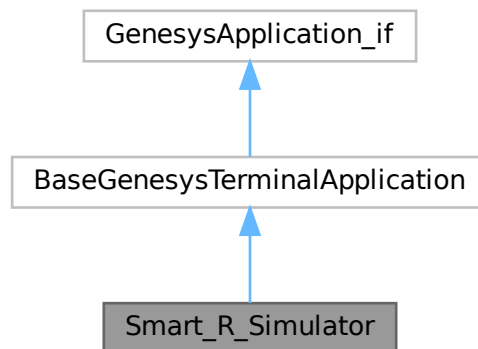
## 15.894 [Smart\\_R\\_Simulator](#) Class Reference

Builds a minimal [Create](#) -> [RSimulator](#) -> [Dispose](#) model.

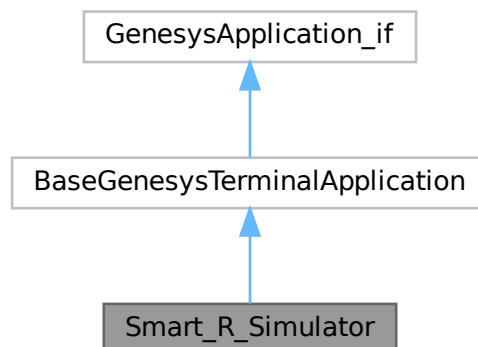
```
#include <applications/modelSpecific/smarts/ExternalIntegration/Smart_R_↵
Simulator.h>
```



Inheritance diagram for Smart\_R\_Simulator:



Collaboration diagram for Smart\_R\_Simulator:



### Public Member Functions

- **Smart\_R\_Simulator** ()  
*Creates the smart R simulator example.*
- virtual int **main** (int argc, char \*\*argv) override  
*Builds, saves and simulates the example model.*

### Public Member Functions inherited from **BaseGenesysTerminalApplication**

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

## Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

### 15.894.1 Detailed Description

Builds a minimal [Create](#) -> [R Simulator](#) -> [Dispose](#) model.

The example exercises the component-level R integration and is intended as a compact smoke model for [R Simulator](#) and its internal [R Simulator Runner](#).

### 15.894.2 Member Function Documentation

#### 15.894.2.1 main()

```
int Smart_R_Simulator::main (
 int argc,
 char ** argv) [override], [virtual]
```

Builds, saves and simulates the example model.

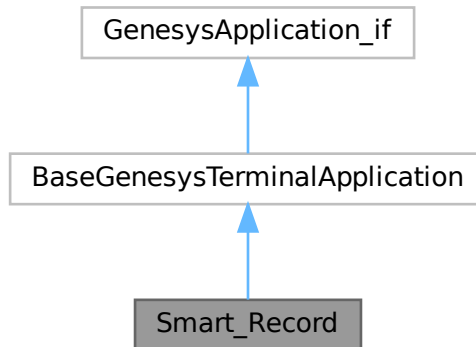
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

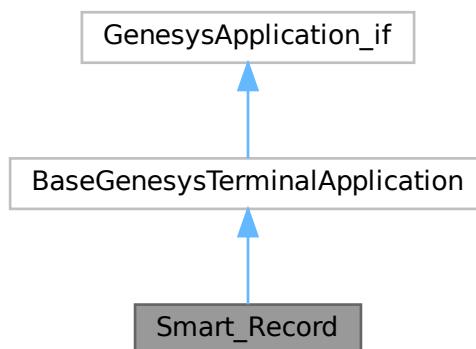
- applications/modelSpecific/smarts/ExternalIntegration/Smart\_R\_Simulator.h
- applications/modelSpecific/smarts/ExternalIntegration/Smart\_R\_Simulator.cpp

## 15.895 Smart\_Record Class Reference

Inheritance diagram for Smart\_Record:



Collaboration diagram for Smart\_Record:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.895.1 Member Function Documentation

### 15.895.1.1 main()

```
int Smart_Record::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

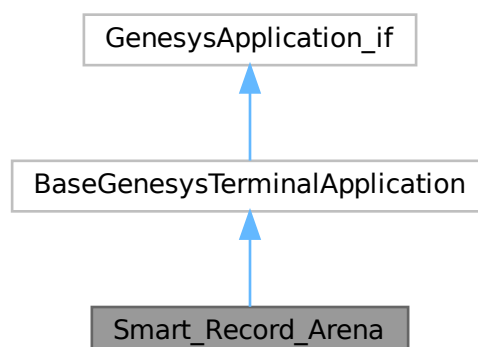
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

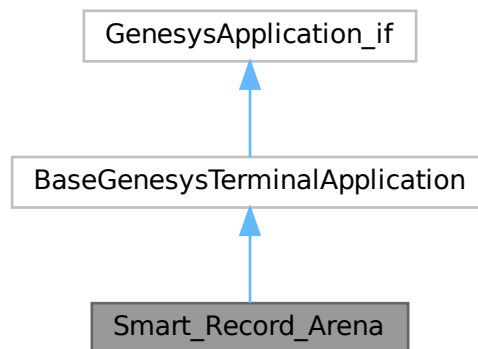
- applications/modelSpecific/smarts/InputOutput/Smart\_Record.h
- applications/modelSpecific/smarts/InputOutput/Smart\_Record.cpp

## 15.896 Smart\_Record\_Arena Class Reference

Inheritance diagram for Smart\_Record\_Arena:



Collaboration diagram for Smart\_Record\_Arena:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.896.1 Member Function Documentation

### 15.896.1.1 main()

```
int Smart_Record_Arena::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

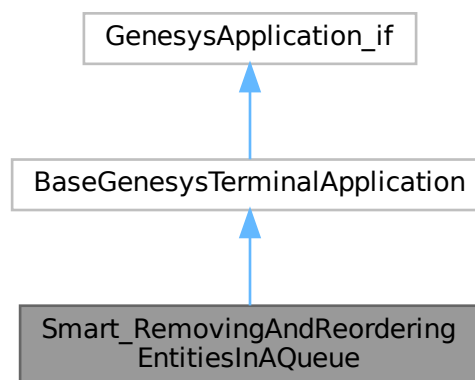
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

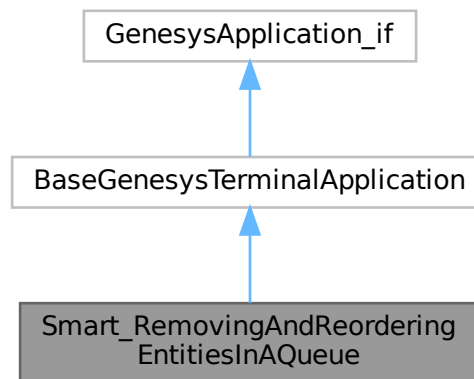
- applications/modelSpecific/arenaSmarts/Smart\_Record\_Arena.h
- applications/modelSpecific/arenaSmarts/Smart\_Record\_Arena.cpp

## 15.897 Smart\_RemovingAndReorderingEntitiesInAQueue Class Reference

Inheritance diagram for Smart\_RemovingAndReorderingEntitiesInAQueue:



Collaboration diagram for Smart\_RemovingAndReorderingEntitiesInAQueue:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.897.1 Member Function Documentation

### 15.897.1.1 main()

```
int Smart_RemovingAndReorderingEntitiesInAQueue::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

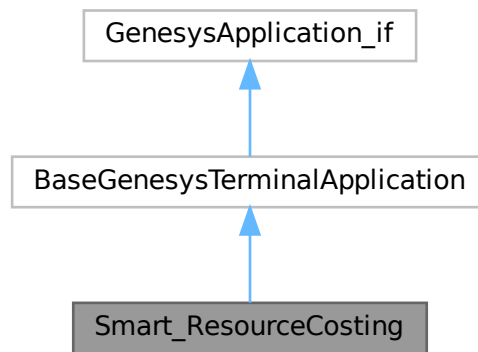
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

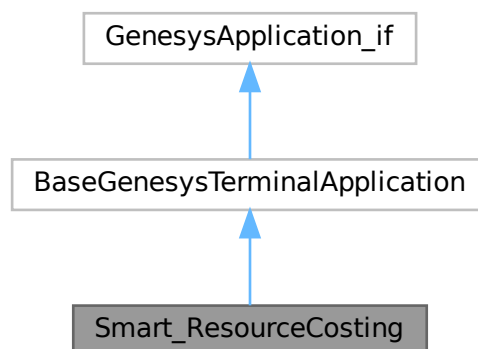
- applications/modelSpecific/arenaSmarts/Smart\_RemovingAndReorderingEntitiesInAQueue.h
- applications/modelSpecific/arenaSmarts/Smart\_RemovingAndReorderingEntitiesInAQueue.cpp

## 15.898 Smart\_ResourceCosting Class Reference

Inheritance diagram for Smart\_ResourceCosting:



Collaboration diagram for Smart\_ResourceCosting:





## Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

## Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

## Additional Inherited Members

## Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.898.1 Member Function Documentation

### 15.898.1.1 [main\(\)](#)

```
int Smart_ResourceCosting::main (
 int argc,
 char ** argv) [override], [virtual]
```

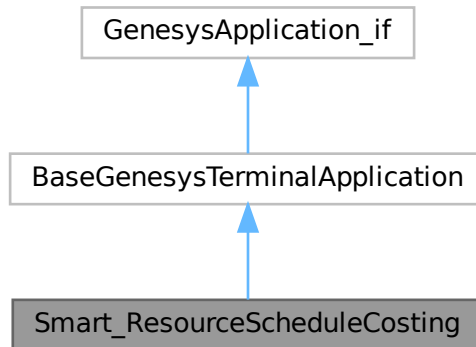
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

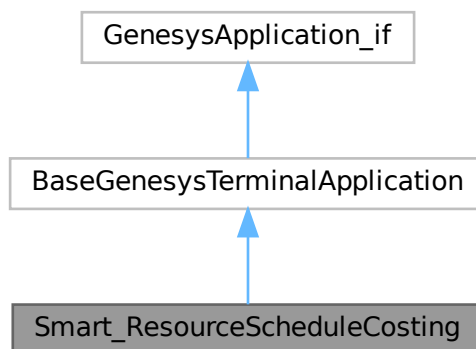
- applications/modelSpecific/arenaSmarts/Smart\_ResourceCosting.h
- applications/modelSpecific/arenaSmarts/Smart\_ResourceCosting.cpp

## 15.899 Smart\_ResourceScheduleCosting Class Reference

Inheritance diagram for Smart\_ResourceScheduleCosting:



Collaboration diagram for Smart\_ResourceScheduleCosting:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.899.1 Member Function Documentation

### 15.899.1.1 main()

```
int Smart_ResourceScheduleCosting::main (
 int argc,
 char ** argv) [override], [virtual]
```

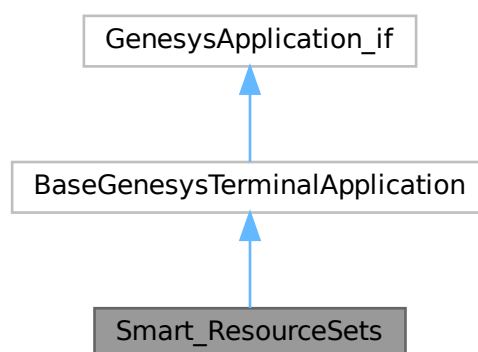
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

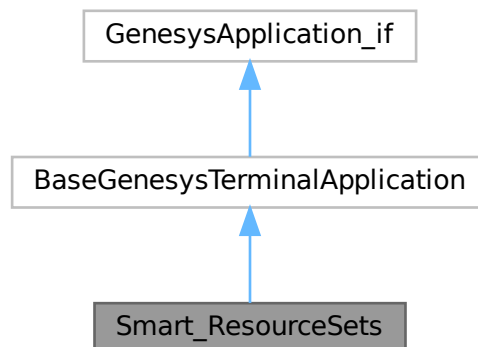
- applications/modelSpecific/arenaSmarts/Smart\_ResourceScheduleCosting.h
- applications/modelSpecific/arenaSmarts/Smart\_ResourceScheduleCosting.cpp

## 15.900 Smart\_ResourceSets Class Reference

Inheritance diagram for Smart\_ResourceSets:



Collaboration diagram for Smart\_ResourceSets:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.900.1 Member Function Documentation

### 15.900.1.1 main()

```
int Smart_ResourceSets::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It create a new simulator instance, builds a simulation model and then simulate that model.

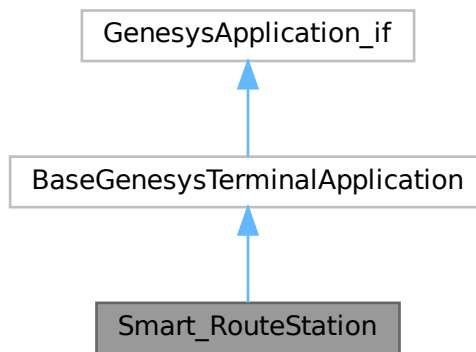
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

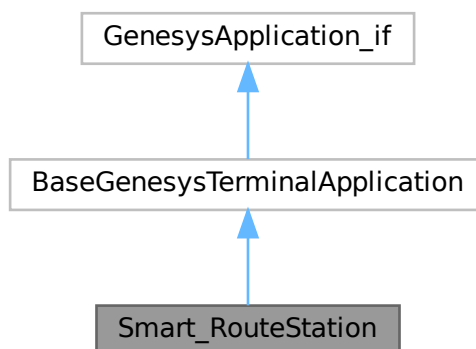
- applications/modelSpecific/arenaSmarts/Smart\_ResourceSets.h
- applications/modelSpecific/arenaSmarts/Smart\_ResourceSets.cpp

## 15.901 Smart\_RouteStation Class Reference

Inheritance diagram for Smart\_RouteStation:



Collaboration diagram for Smart\_RouteStation:



## Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

## Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

## Additional Inherited Members

## Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.901.1 Member Function Documentation

### 15.901.1.1 [main\(\)](#)

```
int Smart_RouteStation::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

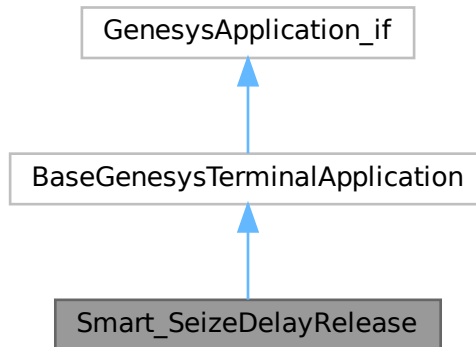
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

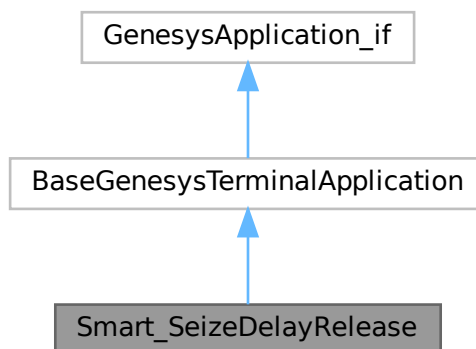
- applications/modelSpecific/smarts/MaterialHandling/Smart\_RouteStation.h
- applications/modelSpecific/smarts/MaterialHandling/Smart\_RouteStation.cpp

## 15.902 Smart\_SeizeDelayRelease Class Reference

Inheritance diagram for Smart\_SeizeDelayRelease:



Collaboration diagram for Smart\_SeizeDelayRelease:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.902.1 Member Function Documentation

### 15.902.1.1 main()

```
int Smart_SeizeDelayRelease::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

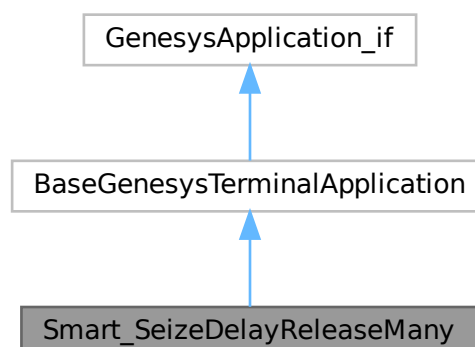
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/smarts/DiscreteProcessing/Smart\_SeizeDelayRelease.h
- applications/modelSpecific/smarts/DiscreteProcessing/Smart\_SeizeDelayRelease.cpp

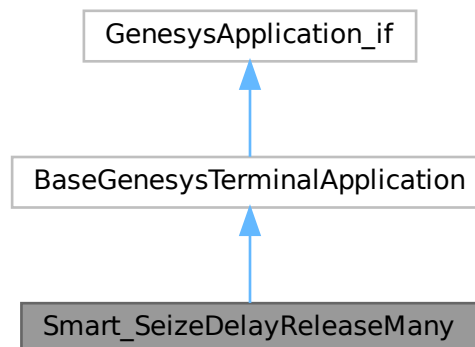
## 15.903 Smart\_SeizeDelayReleaseMany Class Reference

Inheritance diagram for Smart\_SeizeDelayReleaseMany:





Collaboration diagram for Smart\_SeizeDelayReleaseMany:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.903.1 Member Function Documentation

### 15.903.1.1 main()

```
int Smart_SeizeDelayReleaseMany::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

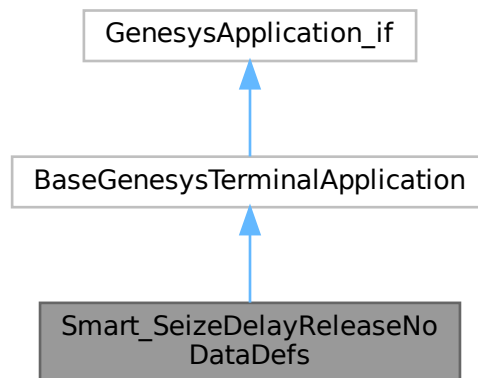
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

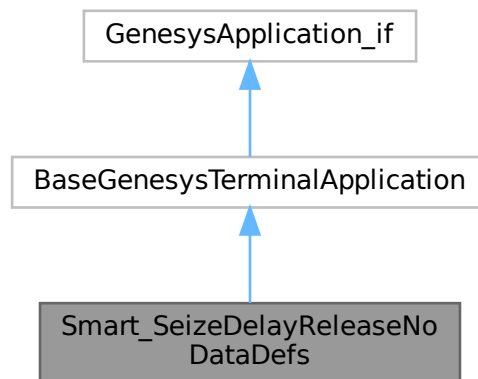
- applications/modelSpecific/smarts/DiscreteProcessing/Smart\_SeizeDelayReleaseMany.h
- applications/modelSpecific/smarts/DiscreteProcessing/Smart\_SeizeDelayReleaseMany.cpp

## 15.904 Smart\_SeizeDelayReleaseNoDataDefs Class Reference

Inheritance diagram for Smart\_SeizeDelayReleaseNoDataDefs:



Collaboration diagram for Smart\_SeizeDelayReleaseNoDataDefs:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.904.1 Member Function Documentation

### 15.904.1.1 main()

```
int Smart_SeizeDelayReleaseNoDataDefs::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

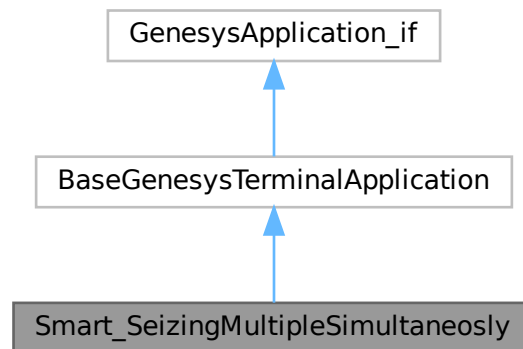
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

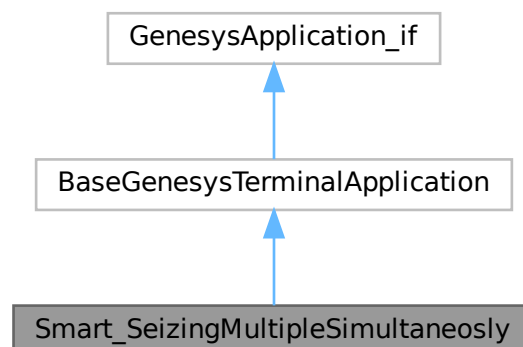
- applications/modelSpecific/smarts/DiscreteProcessing/Smart\_SeizeDelayReleaseNoDataDefs.h
- applications/modelSpecific/smarts/DiscreteProcessing/Smart\_SeizeDelayReleaseNoDataDefs.cpp

## 15.905 Smart\_SeizingMultipleSimultaneosly Class Reference

Inheritance diagram for Smart\_SeizingMultipleSimultaneosly:



Collaboration diagram for Smart\_SeizingMultipleSimultaneosly:



**Public Member Functions**

- virtual int [main](#) (int argc, char \*\*argv) override

**Public Member Functions inherited from [BaseGenesysTerminalApplication](#)**

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

**Additional Inherited Members****Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)**

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

**15.905.1 Member Function Documentation****15.905.1.1 main()**

```
int Smart_SeizingMultipleSimultaneosly::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

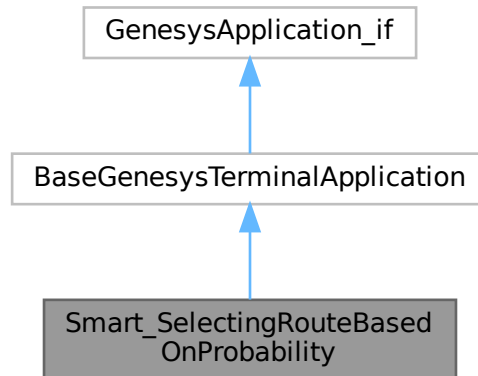
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

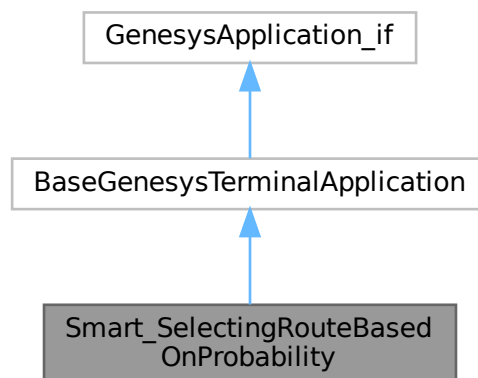
- applications/modelSpecific/arenaSmarts/Smart\_SeizingMultipleSimultaneosly.h
- applications/modelSpecific/arenaSmarts/Smart\_SeizingMultipleSimultaneosly.cpp

## 15.906 Smart\_SelectingRouteBasedOnProbability Class Reference

Inheritance diagram for Smart\_SelectingRouteBasedOnProbability:



Collaboration diagram for Smart\_SelectingRouteBasedOnProbability:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

**Additional Inherited Members****Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)**

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

**15.906.1 Member Function Documentation****15.906.1.1 main()**

```
int Smart_SelectingRouteBasedOnProbability::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

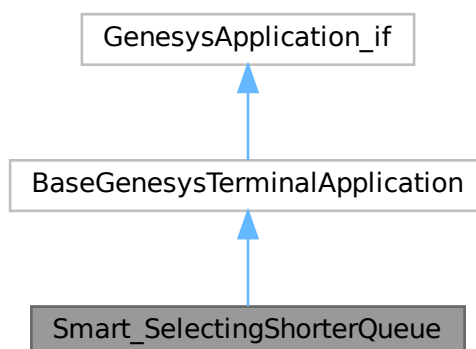
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

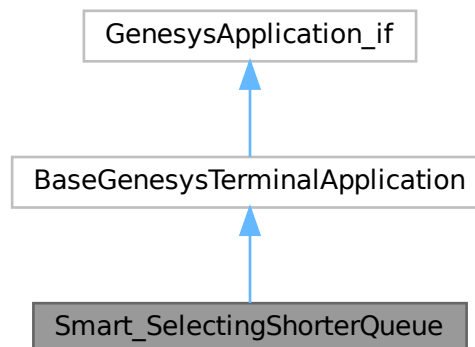
- applications/modelSpecific/arenaSmarts/Smart\_SelectingRouteBasedOnProbability.h
- applications/modelSpecific/arenaSmarts/Smart\_SelectingRouteBasedOnProbability.cpp

**15.907 Smart\_SelectingShorterQueue Class Reference**

Inheritance diagram for Smart\_SelectingShorterQueue:



Collaboration diagram for Smart\_SelectingShorterQueue:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)



## 15.907.1 Member Function Documentation

### 15.907.1.1 main()

```
int Smart_SelectingShorterQueue::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

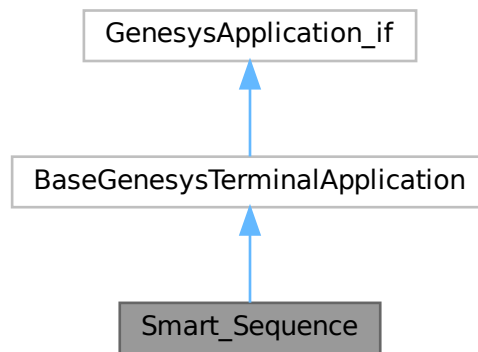
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

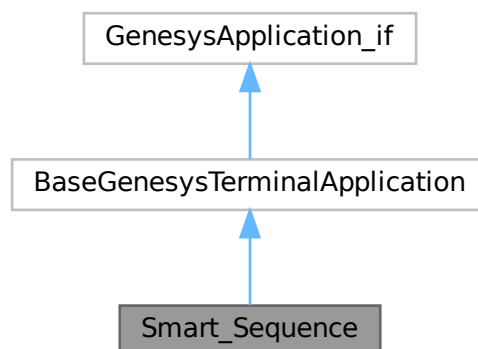
- applications/modelSpecific/arenaSmarts/Smart\_SelectingShorterQueue.h
- applications/modelSpecific/arenaSmarts/Smart\_SelectingShorterQueue.cpp

## 15.908 Smart\_Sequence Class Reference

Inheritance diagram for Smart\_Sequence:



Collaboration diagram for Smart\_Sequence:



## Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

## Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

## Additional Inherited Members

## Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.908.1 Member Function Documentation

### 15.908.1.1 [main\(\)](#)

```
int Smart_Sequence::main (
 int argc,
 char ** argv) [override], [virtual]
```

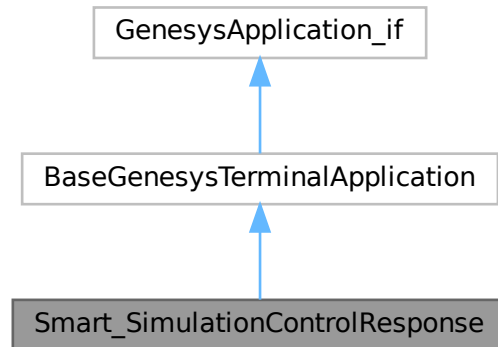
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

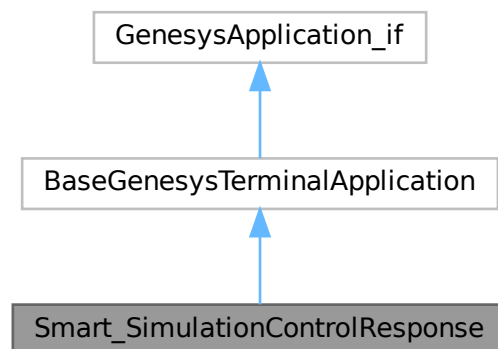
- applications/modelSpecific/smarts/MaterialHandling/Smart\_Sequence.h
- applications/modelSpecific/smarts/MaterialHandling/Smart\_Sequence.cpp

## 15.909 Smart\_SimulationControlResponse Class Reference

Inheritance diagram for Smart\_SimulationControlResponse:



Collaboration diagram for Smart\_SimulationControlResponse:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.909.1 Member Function Documentation

### 15.909.1.1 main()

```
int Smart_SimulationControlResponse::main (
 int argc,
 char ** argv) [override], [virtual]
```

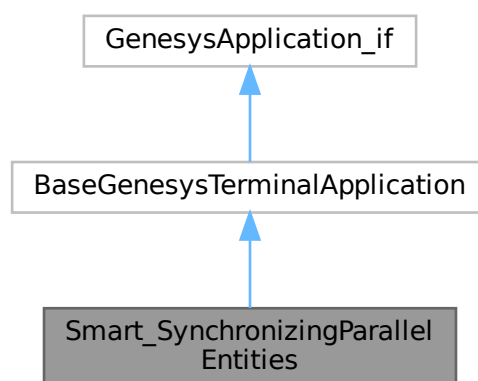
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

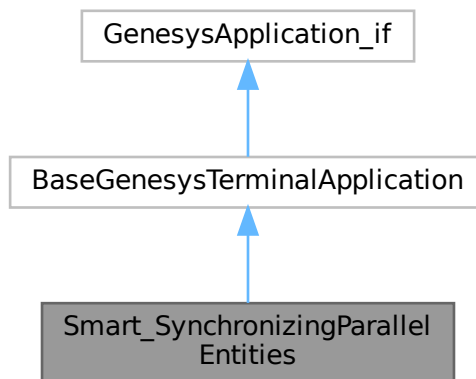
- applications/modelSpecific/smarts/Synchronization/Smart\_SimulationControlResponse.h
- applications/modelSpecific/smarts/Synchronization/Smart\_SimulationControlResponse.cpp

## 15.910 Smart\_SynchronizingParallelEntities Class Reference

Inheritance diagram for Smart\_SynchronizingParallelEntities:



Collaboration diagram for Smart\_SynchronizingParallelEntities:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.910.1 Member Function Documentation

### 15.910.1.1 main()

```
int Smart_SynchronizingParallelEntities::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

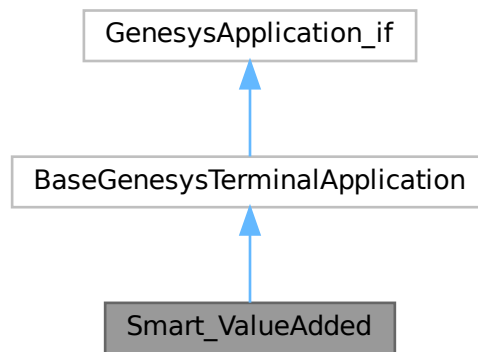
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

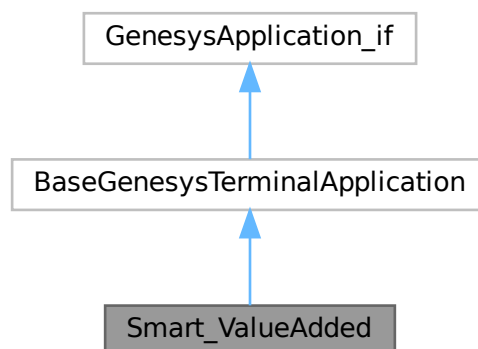
- applications/modelSpecific/arenaSmarts/Smart\_SynchronizingParallelEntities.h
- applications/modelSpecific/arenaSmarts/Smart\_SynchronizingParallelEntities.cpp

## 15.911 Smart\_ValueAdded Class Reference

Inheritance diagram for Smart\_ValueAdded:



Collaboration diagram for Smart\_ValueAdded:



**Public Member Functions**

- virtual int [main](#) (int argc, char \*\*argv) override

**Public Member Functions inherited from [BaseGenesysTerminalApplication](#)**

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

**Additional Inherited Members****Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)**

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

**15.911.1 Member Function Documentation****15.911.1.1 main()**

```
int Smart_ValueAdded::main (
 int argc,
 char ** argv) [override], [virtual]
```

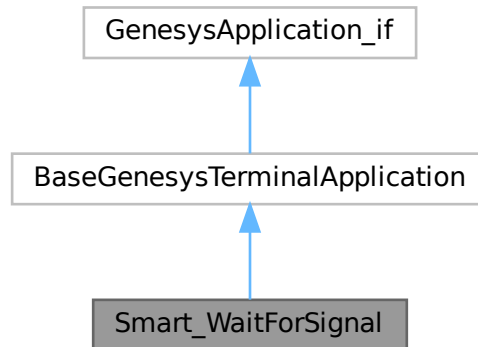
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

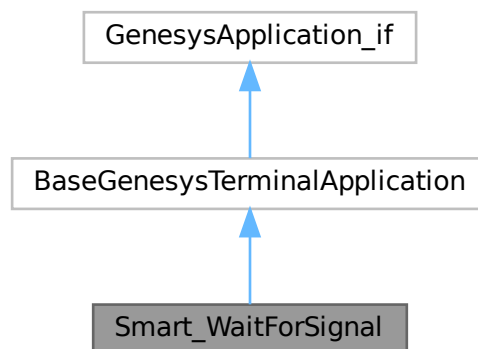
- applications/modelSpecific/arenaSmarts/Smart\_ValueAdded.h
- applications/modelSpecific/arenaSmarts/Smart\_ValueAdded.cpp

## 15.912 Smart\_WaitForSignal Class Reference

Inheritance diagram for Smart\_WaitForSignal:



Collaboration diagram for Smart\_WaitForSignal:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)



- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.912.1 Member Function Documentation

### 15.912.1.1 main()

```
int Smart_WaitForSignal::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

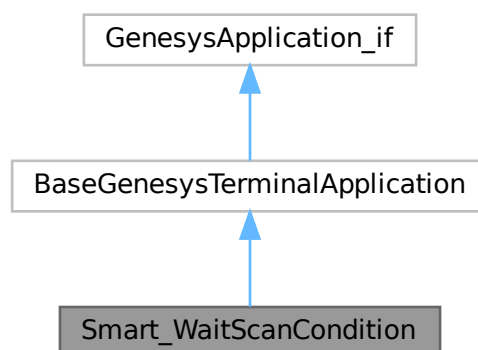
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

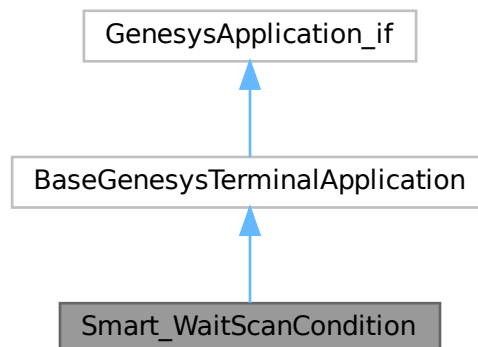
- applications/modelSpecific/arenaSmarts/Smart\_WaitForSignal.h
- applications/modelSpecific/arenaSmarts/Smart\_WaitForSignal.cpp

## 15.913 Smart\_WaitScanCondition Class Reference

Inheritance diagram for Smart\_WaitScanCondition:



Collaboration diagram for Smart\_WaitScanCondition:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

### 15.913.1 Member Function Documentation

#### 15.913.1.1 main()

```
int Smart_WaitScanCondition::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

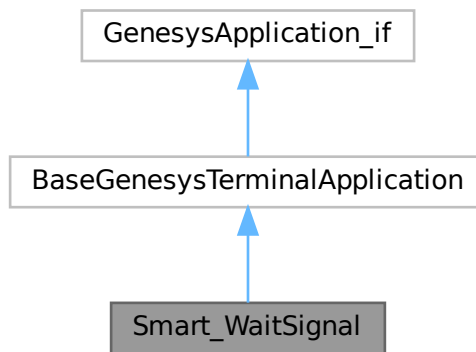
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

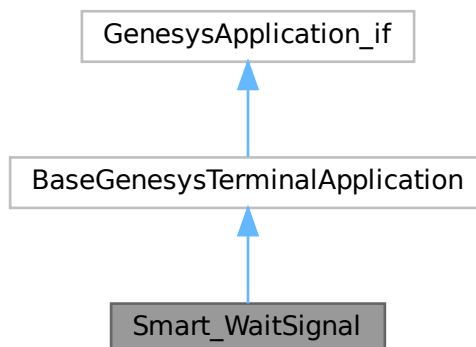
- applications/modelSpecific/smarts/Synchronization/Smart\_WaitScanCondition.h
- applications/modelSpecific/smarts/Synchronization/Smart\_WaitScanCondition.cpp

## 15.914 Smart\_WaitSignal Class Reference

Inheritance diagram for Smart\_WaitSignal:



Collaboration diagram for Smart\_WaitSignal:



## Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

## Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

## Additional Inherited Members

## Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.914.1 Member Function Documentation

### 15.914.1.1 [main\(\)](#)

```
int Smart_WaitSignal::main (
 int argc,
 char ** argv) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/smarts/Synchronization/Smart\_WaitSignal.h
- applications/modelSpecific/smarts/Synchronization/Smart\_WaitSignal.cpp

## 15.915 WebWorkerRuntime::Snapshot Struct Reference

Captures a thread-safe snapshot of the worker state.

```
#include <applications/worker/service/WebWorkerRuntime.h>
```

### Public Attributes

- bool **isRunning** = false
- bool **isStarting** = false
- bool **isStopping** = false
- bool **hasFailed** = false
- unsigned short **port** = 8080
- unsigned long **maxRequests** = 0
- unsigned long **servedRequests** = 0
- unsigned long **activeRequests** = 0
- std::string **statusMessage**
- std::string **lastError**
- std::vector< std::string > **recentEvents**
- std::vector< std::string > **recentErrors**

#### 15.915.1 Detailed Description

Captures a thread-safe snapshot of the worker state.

The documentation for this struct was generated from the following file:

- applications/worker/service/WebWorkerRuntime.h

## 15.916 MetabolicFluxBalanceSolver::Solution Struct Reference

### Public Attributes

- bool **feasible** = false
- double **objectiveValue** = 0.0
- std::vector< double > **fluxes**
- std::string **errorMessage**

The documentation for this struct was generated from the following file:

- tools/Biochemical/MetabolicFluxBalanceSolver.h

## 15.917 Optimizer\_if::SolutionSummary Struct Reference

Captures one candidate or best-known optimization solution.

```
#include <tools/Optimization/Optimizer_if.h>
```

### Public Attributes

- unsigned int **iteration** = 0
- unsigned int **simulationNumber** = 0
- bool **feasible** = false
- double **aggregatedObjectiveValue** = 0.0
- std::string **description**

#### 15.917.1 Detailed Description

Captures one candidate or best-known optimization solution.

A solution stores the controls/responses snapshot produced by one simulation scenario and whether all constraints were satisfied.

The documentation for this struct was generated from the following file:

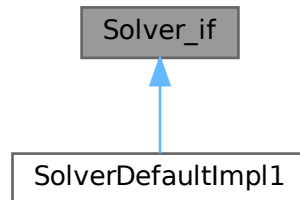
- tools/Optimization/Optimizer\_if.h

## 15.918 Solver\_if Class Reference

Legacy compatibility interface for numerical routines.

```
#include <tools/Continuous/Solver_if.h>
```

Inheritance diagram for Solver\_if:



### Public Member Functions

- virtual void **setPrecision** (double e)=0
- virtual double **getPrecision** ()=0
- virtual void **setMaxSteps** (double steps)=0
- virtual double **getMaxSteps** ()=0
- virtual double **integrate** (double min, double max, double(\*f)(double, double), double p2)=0
- virtual double **integrate** (double min, double max, double(\*f)(double, double, double), double p2, double p3)=0
- virtual double **integrate** (double min, double max, double(\*f)(double, double, double, double), double p2, double p3, double p4)=0
- virtual double **integrate** (double min, double max, double(\*f)(double, double, double, double, double), double p2, double p3, double p4, double p5)=0
- virtual double **derivate** (double initPoint, double initValue, double(\*f)(double, double), double p2)=0
- virtual double **derivate** (double initPoint, double initValue, double(\*f)(double, double, double), double p2, double p3)=0
- virtual double **derivate** (double initPoint, double initValue, double(\*f)(double, double, double, double), double p2, double p3, double p4)=0
- virtual double **derivate** (double initPoint, double initValue, double(\*f)(double, double, double, double, double), double p2, double p3, double p4, double p5)=0

### 15.918.1 Detailed Description

Legacy compatibility interface for numerical routines.

Architectural role:

- Historically groups two responsibilities: 1) Numerical quadrature/integration. 2) Numerical advancement/derivation for ODE-like flows.

Mathematical meaning:

- Function signatures use the first functional parameter as time  $t$ .
- Parameters p2..p5 represent model parameters passed through old-style function pointers.

Limitations:

- The p2/p3/p4/p5 model reflects a historical constraint and is not the preferred long-term abstraction.

Planned evolution:

- Split into focused interfaces (quadrature, root finding, ODE systems and ODE solvers) while preserving this API for backward compatibility.

The documentation for this class was generated from the following file:

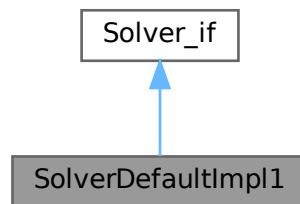
- tools/Continuous/Solver\_if.h

## 15.919 SolverDefaultImpl1 Class Reference

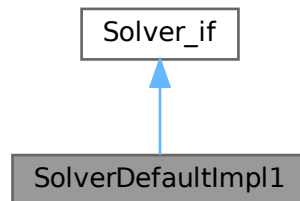
Legacy default implementation of [Solver\\_if](#).

```
#include <tools/Continuous/SolverDefaultImpl1.h>
```

Inheritance diagram for SolverDefaultImpl1:



Collaboration diagram for SolverDefaultImpl1:



### Public Member Functions

- **SolverDefaultImpl1** (double precision=1e-6, unsigned int steps=1e3)
- virtual void [setPrecision](#) (double e) override
- virtual double [getPrecision](#) () override
- virtual void [setMaxSteps](#) (double steps) override
- virtual double [getMaxSteps](#) () override
- virtual double [integrate](#) (double min, double max, double(\*f)(double, double), double p2) override
- virtual double [integrate](#) (double min, double max, double(\*f)(double, double, double), double p2, double p3) override
- virtual double [integrate](#) (double min, double max, double(\*f)(double, double, double, double), double p2, double p3, double p4) override
- virtual double [integrate](#) (double min, double max, double(\*f)(double, double, double, double, double), double p2, double p3, double p4, double p5) override

- virtual double [derivate](#) (double initPoint, double initValue, double(\*)(double, double), double p2) override
- virtual double [derivate](#) (double initPoint, double initValue, double(\*)(double, double, double), double p2, double p3) override
- virtual double [derivate](#) (double initPoint, double initValue, double(\*)(double, double, double, double), double p2, double p3, double p4) override
- virtual double [derivate](#) (double initPoint, double initValue, double(\*)(double, double, double, double, double), double p2, double p3, double p4, double p5) override

### 15.919.1 Detailed Description

Legacy default implementation of [Solver\\_if](#).

Supported behavior:

- Numerical quadrature through composite Simpson 1/3 with a positive even number of subintervals.

Unsupported behavior:

- The historical `derivate()` signatures do not expose a defined step-size contract and are not used by current repository clients. They fail explicitly instead of returning silent or undefined numerical results.

Planned evolution:

- Split the historical interface into focused quadrature and ODE abstractions while preserving compatibility where a valid contract exists.

## 15.919.2 Member Function Documentation

### 15.919.2.1 `derivate()` [1/4]

```
double SolverDefaultImpl1::derivate (
 double initPoint,
 double initValue,
 double(*) (double, double) f,
 double p2) [override], [virtual]
```

Implements [Solver\\_if](#).

### 15.919.2.2 `derivate()` [2/4]

```
double SolverDefaultImpl1::derivate (
 double initPoint,
 double initValue,
 double(*) (double, double, double) f,
 double p2,
 double p3) [override], [virtual]
```

Implements [Solver\\_if](#).

### 15.919.2.3 `derivate()` [3/4]

```
double SolverDefaultImpl1::derivate (
 double initPoint,
 double initValue,
 double(*) (double, double, double, double) f,
 double p2,
 double p3,
 double p4) [override], [virtual]
```

Implements [Solver\\_if](#).

### 15.919.2.4 `derivate()` [4/4]

```
double SolverDefaultImpl1::derivate (
 double initPoint,
```



```
double initValue,
double(*) (double, double, double, double, double) f,
double p2,
double p3,
double p4,
double p5) [override], [virtual]
```

Implements [Solver\\_if](#).

#### 15.919.2.5 getMaxSteps()

```
double SolverDefaultImpl1::getMaxSteps () [override], [virtual]
```

Implements [Solver\\_if](#).

#### 15.919.2.6 getPrecision()

```
double SolverDefaultImpl1::getPrecision () [override], [virtual]
```

Implements [Solver\\_if](#).

#### 15.919.2.7 integrate() [1/4]

```
double SolverDefaultImpl1::integrate (
 double min,
 double max,
 double(*) (double, double) f,
 double p2) [override], [virtual]
```

Implements [Solver\\_if](#).

#### 15.919.2.8 integrate() [2/4]

```
double SolverDefaultImpl1::integrate (
 double min,
 double max,
 double(*) (double, double, double) f,
 double p2,
 double p3) [override], [virtual]
```

Implements [Solver\\_if](#).

#### 15.919.2.9 integrate() [3/4]

```
double SolverDefaultImpl1::integrate (
 double min,
 double max,
 double(*) (double, double, double, double) f,
 double p2,
 double p3,
 double p4) [override], [virtual]
```

Implements [Solver\\_if](#).

#### 15.919.2.10 integrate() [4/4]

```
double SolverDefaultImpl1::integrate (
 double min,
 double max,
 double(*) (double, double, double, double, double) f,
 double p2,
 double p3,
 double p4,
 double p5) [override], [virtual]
```

Implements [Solver\\_if](#).

**15.919.2.11 setMaxSteps()**

```
void SolverDefaultImpl1::setMaxSteps (
 double steps) [override], [virtual]
```

Implements [Solver\\_if](#).

**15.919.2.12 setPrecision()**

```
void SolverDefaultImpl1::setPrecision (
 double e) [override], [virtual]
```

Implements [Solver\\_if](#).

The documentation for this class was generated from the following files:

- tools/Continuous/SolverDefaultImpl1.h
- tools/Continuous/SolverDefaultImpl1.cpp

**15.920 SortFile Class Reference**

Utility class that sorts numeric values persisted in a data file.

```
#include <kernel/statistics/SorttFile.h>
```

**Public Member Functions**

- **SortFile ()**  
*Creates an unbound sorter (filename must be configured before use).*
- **bool sort ()**  
*Executes in-file sorting for the configured dataset.*
- **void setDataFilename (std::string filename)**  
*Sets the target data filename to be sorted.*

**15.920.1 Detailed Description**

Utility class that sorts numeric values persisted in a data file.

Used by file-based statistics implementations to support quantile/median computations that require ordered samples.

**15.920.2 Member Function Documentation****15.920.2.1 setDataFilename()**

```
void SortFile::setDataFilename (
 std::string filename)
```

Sets the target data filename to be sorted.

**Parameters**

|                 |                                |
|-----------------|--------------------------------|
| <i>filename</i> | Path to the backing data file. |
|-----------------|--------------------------------|

**15.920.2.2 sort()**

```
bool SortFile::sort ()
```

Executes in-file sorting for the configured dataset.

**Returns**

True if sorting operation completes successfully.

The documentation for this class was generated from the following files:

- kernel/statistics/SorttFile.h
- kernel/statistics/SorttFile.cpp

## 15.921 SourceCodeString Class Reference

### Public Member Functions

- **SourceCodeString** (const std::string &value)
- **SourceCodeString** (std::string &&value)
- const std::string & **str** () const
- **operator std::string** () const

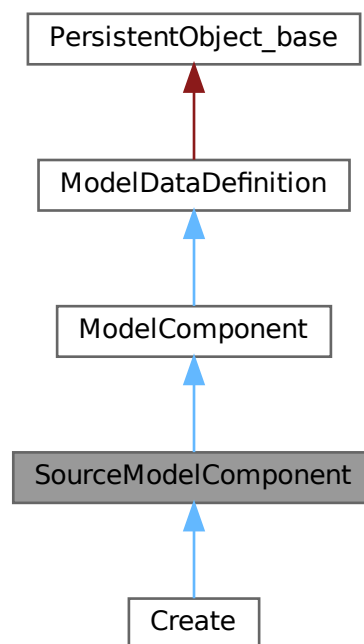
The documentation for this class was generated from the following file:

- kernel/simulator/SimulationControlAndResponse.h

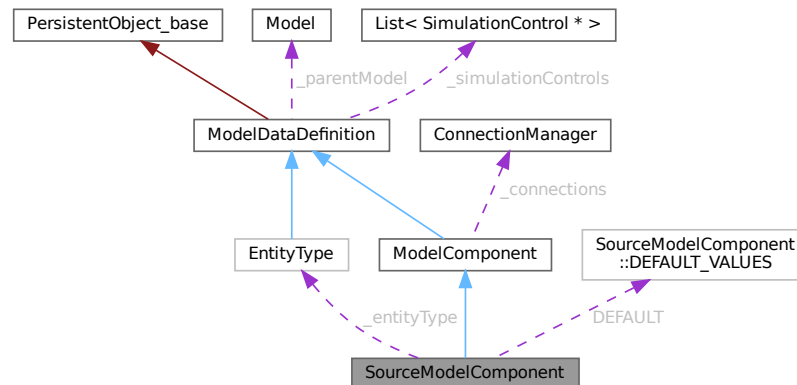
## 15.922 SourceModelComponent Class Reference

```
#include <kernel/simulator/SourceModelComponent.h>
```

Inheritance diagram for SourceModelComponent:



Collaboration diagram for SourceModelComponent:



## Classes

- struct [DEFAULT\\_VALUES](#)

## Public Member Functions

- **SourceModelComponent** ([Model](#) \*model, std::string componentTypename, std::string name="")
- void [setFirstCreation](#) (double \_firstCreation)  
    *setFirstCreation*
- double [getFirstCreation](#) () const  
    *getFirstCreation*
- void [setEntityType](#) ([EntityType](#) \*\_entityType)  
    *setEntityType*
- void [setEntityTypename](#) (std::string entityTypename)  
    *setEntityTypename*
- [EntityType](#) \* [getEntityType](#) () const  
    *getEntityType*
- void [setTimeUnit](#) ([Util::TimeUnit](#) \_timeUnit)  
    *setTimeUnit*
- [Util::TimeUnit](#) [getTimeUnit](#) () const  
    *getTimeUnit*
- void [setTimeBetweenCreationsExpression](#) (std::string \_timeBetweenCreations, [Util::TimeUnit](#) \_timeUnit=Util::TimeUnit::unknown)  
    *setTimeBetweenCreationsExpression*
- std::string [getTimeBetweenCreationsExpression](#) () const  
    *getTimeBetweenCreationsExpression*
- void [setMaxCreations](#) (unsigned long \_maxCreations)  
    *setMaxCreations*
- void [setMaxCreations](#) (std::string \_maxCreationsExpression)  
    *setMaxCreations*
- std::string [getMaxCreations](#) () const  
    *getMaxCreations*
- unsigned int [getEntitiesCreated](#) () const  
    *getEntitiesCreated*

- void [setEntitiesCreated](#) (unsigned int \_entitiesCreated)  
*setEntitiesCreated*
- void [setEntitiesPerCreation](#) (unsigned int \_entitiesPerCreation)  
*setEntitiesPerCreation*
- unsigned int [getEntitiesPerCreation](#) () const  
*getEntitiesPerCreation*
- virtual std::string [show](#) () override  
*virtual public methods*

### Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) \*model, std::string componentTypename, std::string name="")  
*constructors*
- virtual ~[ModelComponent](#) ()  
*Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) \* [getConnectionManager](#) () const  
*new public user methods for this component*
- void [connectTo](#) ([ModelComponent](#) \*component, unsigned int inputPortNumber=0)  
*Connects this component to another component.*
- bool [hasBreakpointAt](#) ()  
*Indicates whether a simulation breakpoint is set on this component.*
- void [setDescription](#) (std::string \_description)  
*Updates the free-form description shown by the GUI.*
- std::string [getDescription](#) () const  
*Returns the free-form description associated with this component.*
- void [remove](#) ([ModelComponent](#) \*comp)  
*Removes a connected component from this component.*

### Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) \*model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)  
*Creates a model data definition and optionally inserts it into the model.*
- virtual ~[ModelDataDefinition](#) ()  
*Releases resources owned by this data definition.*
- Util::identification [getId](#) () const  
*Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)  
*Updates the logical name of this instance.*
- const std::string & [getName](#) () const  
*Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)  
*Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const  
*Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const  
*Returns the runtime class name used by the plugin system.*
- [Model](#) \* [getParentModel](#) () const  
*Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const  
*Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)

- Enables or disables report-statistics generation for this definition.*

  - `ModelDataDefinition * getInternalData (const std::string &name) const`  
*Looks up a named internal child data definition.*
  - `std::map< std::string, ModelDataDefinition * > * getInternalData () const`  
*Returns the full map of internal child data definitions.*
  - `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`  
*Returns the full map of attached data definitions.*
  - `bool hasChanged () const`  
*Indicates whether this instance has been modified since the last reset.*
  - `void setHasChanged (bool hasChanged)`  
*Updates the changed flag for this data definition.*
  - `unsigned int getLevel () const`  
*Returns the level in the model hierarchy where this definition lives.*
  - `void setModelLevel (unsigned int _modelLevel)`  
*Sets the model nesting level for this definition.*
  - `List< SimulationControl * > * getSimulationControls () const`  
*Returns the simulation controls owned by this data definition.*
  - `TraceManager::Level getTraceLevelSpecific () const`  
*Returns the trace level that applies specifically to this definition.*
  - `void setTraceLevelSpecific (TraceManager::Level level)`  
*Updates the trace level used by this definition.*
  - `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific←Enabled=true)`  
*Defines the specific trace level and optionally enables it immediately.*
  - `bool isTraceLevelSpecificEnabled () const`  
*Indicates whether the specific trace level is enabled.*
  - `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`  
*Enables or disables the specific trace level.*

### Protected Member Functions

- `virtual bool _loadInstance (PersistenceRecord *fields) override`  
*virtual protected method that must be overridden*
- `virtual void _saveInstance (PersistenceRecord *fields, bool saveDefaultValues) override`  
*Saves object fields into a serialization structure.*
- `virtual void _initBetweenReplications () override`
- `virtual bool _check (std::string &errorMessage) override`  
*could be overridden by derived classes*
- `virtual void _createEditableDataDefinitions ()`
- `virtual void _createAttachedAttributes ()`

### Protected Member Functions inherited from `ModelComponent`

- `virtual void _onDispatchEvent (Entity *entity, unsigned int inputPortNumber)=0`  
*Handles a dispatched event for this component type.*

## Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [\\_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) \* [\\_getParserChangesInformation](#) ()
- virtual void [\\_addSimulationControl](#) ([SimulationControl](#) \*control)
- virtual void [\\_createInternalAndAttachedData](#) ()
- virtual void [\\_createInternalStatisticReporters](#) ()
- virtual void [\\_createNonEditableDataDefinitions](#) ()
- void [\\_templateCreateInternalStatisticReporters](#) ()
- void [\\_templateCreateAttachedAttributes](#) ()
- void [\\_templateCreateNonEditableDataDefinitions](#) ()
- void [\\_templateCreateEditableDataDefinitions](#) ()
- void [\\_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create\* hooks*
- void [\\_internalDataClear](#) ()
- void [\\_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*child)
- void [\\_internalDataRemove](#) (const std::string &key)
- void [\\_attachedDataClear](#) ()
- void [\\_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_attachedDataRemove](#) (const std::string &key)
- void [\\_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [\\_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_statisticReporterRemove](#) (const std::string &key)
- void [\\_statisticReportersClear](#) ()
- void [\\_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [\\_mandatoryAttachedAttributesClear](#) ()
- void [\\_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_mandatoryEditableDataDefinitionsClear](#) ()
- void [\\_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_optionalEditableDataDefinitionsClear](#) ()
- void [\\_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [\\_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_optionalNonEditableDataDefinitionsClear](#) ()
- void [\\_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1\\_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2\\_results](#))
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))
- void [traceSimulation](#) (void \*thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))
- Traces text to the simulation output.*
- void **traceSimulation** (void \*thisobject, [TraceManager::Level](#) level, const std::string &text)

### Protected Attributes

- `EntityType * _entityType` = nullptr
- const struct `SourceModelComponent::DEFAULT_VALUES` **DEFAULT**
- double `_firstCreation` = `DEFAULT.firstCreation`
- unsigned int `_entitiesPerCreation` = `DEFAULT.entitiesPerCreation`
- std::string `_maxCreationsExpression` = `DEFAULT.maxCreationsExpression`
- std::string `_timeBetweenCreationsExpression` = `DEFAULT.timeBetweenCreationsExpression`
- `Util::TimeUnit` `_timeBetweenCreationsTimeUnit` = `DEFAULT.timeBetweenCreationsTimeUnit`
- unsigned int `_entitiesCreatedSoFar` = 0

### Protected Attributes inherited from `ModelComponent`

- `ConnectionManager * _connections` = new `ConnectionManager()`  
*virtual protected methods that could be overridden by derived classes, if needed*

### Protected Attributes inherited from `ModelDataDefinition`

- `Util::identification` `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- `Model * _parentModel`
- `List< SimulationControl * > * _simulationControls` = new `List<SimulationControl*>()`
- `TraceManager::Level` `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

### Additional Inherited Members

### Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent * LoadInstance` (`Model *model`, `PersistenceRecord *fields`)  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool `Check` (`ModelComponent *component`)  
*Validates a component instance.*
- static void `CreateInternalData` (`ModelComponent *component`)  
*Creates internal data required by a component instance.*
- static void `SaveInstance` (`PersistenceRecord *fields`, `ModelComponent *component`)  
*Serializes a component instance into a persistence record.*
- static void `DispatchEvent` (`Event *event`)  
*Dispatches a simulation event to the target component.*

### Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition * LoadInstance` (`Model *model`, `PersistenceRecord *fields`, bool `insertIntoModel`)  
*Loads a model data definition instance from serialized fields.*
- static `ModelDataDefinition * NewInstance` (`Model *model`, std::string `name=""`)  
*Creates a new instance of a data definition using the plugin registry.*
- static void `SaveInstance` (`PersistenceRecord *fields`, `ModelDataDefinition *modeldatum`)  
*Serializes a data definition instance into a persistence record.*
- static bool `Check` (`ModelDataDefinition *modeldatum`, std::string `&errorMessage`)  
*Validates a data definition instance.*
- static void `InitBetweenReplications` (`ModelDataDefinition *modeldatum`)



*Resets simulation-time state between replications.*

- static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

### 15.922.1 Detailed Description

A source component implements the base for inserting entities into the model when its simulation is initialized. During the initialization, the new and empty future events list is populated by events of creating entities and sending them to the source components existing in the model

### 15.922.2 Member Function Documentation

#### 15.922.2.1 [\\_check\(\)](#)

```
bool SourceModelComponent::_check (
 std::string & errorMessage) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [Create](#).

#### 15.922.2.2 [\\_createAttachedAttributes\(\)](#)

```
void SourceModelComponent::_createAttachedAttributes () [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

#### 15.922.2.3 [\\_createEditableDataDefinitions\(\)](#)

```
void SourceModelComponent::_createEditableDataDefinitions () [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

#### 15.922.2.4 [\\_initBetweenReplications\(\)](#)

```
void SourceModelComponent::_initBetweenReplications () [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

#### 15.922.2.5 [\\_loadInstance\(\)](#)

```
bool SourceModelComponent::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

Reimplemented in [Create](#).

#### 15.922.2.6 [\\_saveInstance\(\)](#)

```
void SourceModelComponent::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

#### Parameters

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [ModelComponent](#).

Reimplemented in [Create](#).

**15.922.2.7 getEntitiesCreated()**

```
unsigned int SourceModelComponent::getEntitiesCreated () const
getEntitiesCreated
```

Returns

**15.922.2.8 getEntitiesPerCreation()**

```
unsigned int SourceModelComponent::getEntitiesPerCreation () const
getEntitiesPerCreation
```

Returns

**15.922.2.9 getEntityType()**

```
EntityType * SourceModelComponent::getEntityType () const
getEntityType
```

Returns

**15.922.2.10 getFirstCreation()**

```
double SourceModelComponent::getFirstCreation () const
getFirstCreation
```

Returns

**15.922.2.11 getMaxCreations()**

```
std::string SourceModelComponent::getMaxCreations () const
getMaxCreations
```

Returns

**15.922.2.12 getTimeBetweenCreationsExpression()**

```
std::string SourceModelComponent::getTimeBetweenCreationsExpression () const
getTimeBetweenCreationsExpression
```

Returns

**15.922.2.13 getTimeUnit()**

```
Util::TimeUnit SourceModelComponent::getTimeUnit () const
getTimeUnit
```

Returns

**15.922.2.14 setEntitiesCreated()**

```
void SourceModelComponent::setEntitiesCreated (
 unsigned int _entitiesCreated)
setEntitiesCreated
```

**Parameters**

|                         |  |
|-------------------------|--|
| <i>_entitiesCreated</i> |  |
|-------------------------|--|

**15.922.2.15 setEntitiesPerCreation()**

```
void SourceModelComponent::setEntitiesPerCreation (
 unsigned int _entitiesPerCreation)
setEntitiesPerCreation
```

**Parameters**

|                             |  |
|-----------------------------|--|
| <i>_entitiesPerCreation</i> |  |
|-----------------------------|--|

**15.922.2.16 setEntityType()**

```
void SourceModelComponent::setEntityType (
 EntityType * _entityType)
setEntityType
```

**Parameters**

|                    |  |
|--------------------|--|
| <i>_entityType</i> |  |
|--------------------|--|

**15.922.2.17 setEntityTypeName()**

```
void SourceModelComponent::setEntityTypeName (
 std::string entityTypeName)
setEntityTypeName
```

**Parameters**

|                       |  |
|-----------------------|--|
| <i>entityTypeName</i> |  |
|-----------------------|--|

**15.922.2.18 setFirstCreation()**

```
void SourceModelComponent::setFirstCreation (
 double _firstCreation)
setFirstCreation
```

**Parameters**

|                       |  |
|-----------------------|--|
| <i>_firstCreation</i> |  |
|-----------------------|--|

**15.922.2.19 setMaxCreations() [1/2]**

```
void SourceModelComponent::setMaxCreations (
 std::string _maxCreationsExpression)
```

setMaxCreations

Parameters

|                                      |  |
|--------------------------------------|--|
| <code>_maxCreationsExpression</code> |  |
|--------------------------------------|--|

#### 15.922.2.20 setMaxCreations() [2/2]

```
void SourceModelComponent::setMaxCreations (
 unsigned long _maxCreations)
```

setMaxCreations

Parameters

|                            |  |
|----------------------------|--|
| <code>_maxCreations</code> |  |
|----------------------------|--|

#### 15.922.2.21 setTimeBetweenCreationsExpression()

```
void SourceModelComponent::setTimeBetweenCreationsExpression (
 std::string _timeBetweenCreations,
 Util::TimeUnit _timeUnit = Util::TimeUnit::unknown)
```

setTimeBetweenCreationsExpression

Parameters

|                                    |  |
|------------------------------------|--|
| <code>_timeBetweenCreations</code> |  |
| <code>_timeUnit</code>             |  |

#### 15.922.2.22 setTimeUnit()

```
void SourceModelComponent::setTimeUnit (
 Util::TimeUnit _timeUnit)
```

setTimeUnit

Parameters

|                        |  |
|------------------------|--|
| <code>_timeUnit</code> |  |
|------------------------|--|

#### 15.922.2.23 show()

```
std::string SourceModelComponent::show () [override], [virtual]
virtual public methods
```

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

Reimplemented in [Create](#).

The documentation for this class was generated from the following files:

- kernel/simulator/SourceModelComponent.h
- kernel/simulator/SourceModelComponent.cpp

## 15.923 SparseValueStore Class Reference

Sparse N-dimensional numeric store using GenESyS textual index keys.

```
#include <kernel/simulator/essentialPlugins/SparseValueStore.h>
```

### Public Member Functions

- double **value** (const std::string &index="") const  
*Reads a sparse position, returning 0.0 when the position is absent.*
- void **setValue** (double **value**, const std::string &index="")  
*Writes a sparse position, creating it when needed.*
- void **setValues** (const std::map< std::string, double > &**values**)  
*Replaces all sparse values with a copy of the provided values.*
- void **copyFrom** (const [SparseValueStore](#) &other)  
*Replaces this store with a deep copy of another store.*
- void **insertDimensionSize** (unsigned int size)  
*Inserts one dimension size at the end of the dimension list.*
- std::list< unsigned int > \* **dimensionSizes** ()  
*Returns mutable dimension sizes for compatibility with legacy callers.*
- const std::list< unsigned int > \* **dimensionSizes** () const  
*Returns immutable dimension sizes.*
- std::map< std::string, double > \* **values** ()  
*Returns mutable sparse values for compatibility with legacy callers.*
- const std::map< std::string, double > \* **values** () const  
*Returns immutable sparse values.*
- void **clear** ()  
*Clears dimensions and values.*
- void **loadDimensions** ([PersistenceRecord](#) \*fields)  
*Loads dimensions using the legacy fields dimensions/dimension[i].*
- void **saveDimensions** ([PersistenceRecord](#) \*fields, bool saveDefaultValues) const  
*Saves dimensions using the legacy fields dimensions/dimension[i].*
- void **loadValues** ([PersistenceRecord](#) \*fields, const std::string &countField, const std::string &positionPrefix, const std::string &valuePrefix)  
*Loads sparse values using the provided count/position/value field prefixes.*
- void **saveValues** ([PersistenceRecord](#) \*fields, const std::string &countField, const std::string &positionPrefix, const std::string &valuePrefix, bool saveDefaultValues) const  
*Saves sparse values using the provided count/position/value field prefixes.*
- std::string **showValues** () const  
*Returns a compact textual representation used by show/debug paths.*

### Static Public Member Functions

- static std::string **makeIndexKey** (const std::vector< unsigned int > &indexes)  
*Builds the textual key used by legacy sparse matrices from index values.*
- static std::string **makeIndexKeyFromDoubles** (const std::vector< double > &indexes)  
*Builds the textual key used by legacy sparse matrices from expression values.*
- static std::string **appendIndexKey** (const std::string &currentKey, unsigned int index)  
*Appends one index to an existing textual sparse key.*
- static std::string **appendIndexKeyFromDouble** (const std::string &currentKey, double index)  
*Appends one expression value to an existing textual sparse key.*
- static std::string **normalizeIndexKey** (const std::string &index)  
*Returns a normalized copy of an already textual key.*

### 15.923.1 Detailed Description

Sparse N-dimensional numeric store using GenESyS textual index keys.

The store preserves the existing [Variable](#) semantics: the scalar position is represented by an empty key, multidimensional indexes are represented as comma-separated unsigned integers such as "1,2,3", and any missing position reads back as 0.0. It is intentionally small so [Variable](#), [Attribute](#) and [Entity](#) can share the same behavior without changing the persistence format.

The documentation for this class was generated from the following file:

- kernel/simulator/essentialPlugins/SparseValueStore.h

## 15.924 MassActionOdeSystem::Species Struct Reference

### Public Attributes

- std::string **name**
- double **initialAmount** = 0.0
- bool **boundaryCondition** = false
- bool **constant** = false

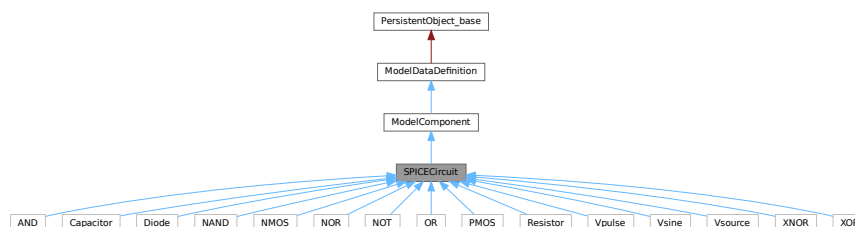
The documentation for this struct was generated from the following file:

- tools/Biochemical/MassActionOdeSystem.h

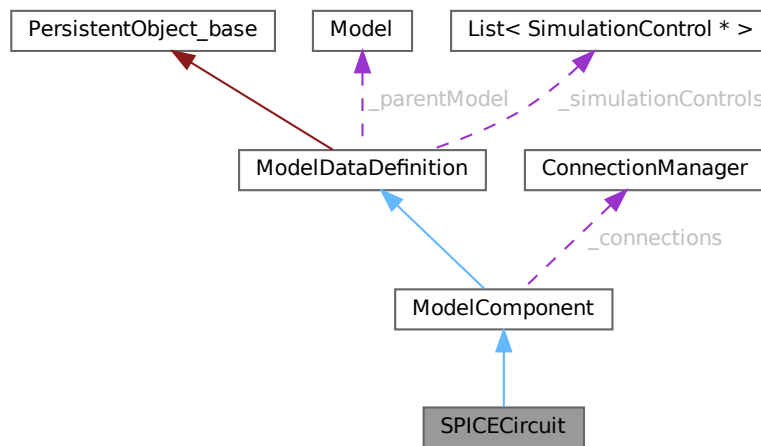
## 15.925 SPICECircuit Class Reference

```
#include <plugins/components/ElectronicsSimulation/SPICECircuit.h>
```

Inheritance diagram for SPICECircuit:



Collaboration diagram for SPICECircuit:



### Public Member Functions

- **SPICECircuit** ([Model](#) \*model, std::string name="")  
*constructors*
- void **setRunner** ([SPICERunner](#) \*runner)  
*new public user methods for this components*
- [SPICERunner](#) \* **getRunner** ()
- virtual std::string **show** () override  
*virtual public methods*

### Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) \*model, std::string componentTypename, std::string name="")  
*constructors*
- virtual ~**ModelComponent** ()  
*Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) \* **getConnectionManager** () const  
*new public user methods for this component*
- void **connectTo** ([ModelComponent](#) \*component, unsigned int inputPortNumber=0)  
*Connects this component to another component.*
- bool **hasBreakpointAt** ()  
*Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string \_description)  
*Updates the free-form description shown by the GUI.*
- std::string **getDescription** () const  
*Returns the free-form description associated with this component.*
- void **remove** ([ModelComponent](#) \*comp)  
*Removes a connected component from this component.*

## Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** \*model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
 

*Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
 

*Releases resources owned by this data definition.*
- Util::identification **getId** () const
 

*Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
 

*Updates the logical name of this instance.*
- const std::string & **getName** () const
 

*Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)
 

*Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const
 

*Returns the user-visible label of this instance.*
- const std::string & **getClassname** () const
 

*Returns the runtime class name used by the plugin system.*
- **Model** \* **getParentModel** () const
 

*Returns the parent model that owns this definition.*
- bool **isReportStatistics** () const
 

*Indicates whether this data definition contributes report statistics.*
- void **setReportStatistics** (bool reportStatistics)
 

*Enables or disables report-statistics generation for this definition.*
- **ModelDataDefinition** \* **getInternalData** (const std::string &name) const
 

*Looks up a named internal child data definition.*
- std::map< std::string, **ModelDataDefinition** \* > \* **getInternalData** () const
 

*Returns the full map of internal child data definitions.*
- std::map< std::string, **ModelDataDefinition** \* > \* **getAttachedData** () const
 

*Returns the full map of attached data definitions.*
- bool **hasChanged** () const
 

*Indicates whether this instance has been modified since the last reset.*
- void **setHasChanged** (bool **hasChanged**)
 

*Updates the changed flag for this data definition.*
- unsigned int **getLevel** () const
 

*Returns the level in the model hierarchy where this definition lives.*
- void **setModelLevel** (unsigned int \_modelLevel)
 

*Sets the model nesting level for this definition.*
- **List**< **SimulationControl** \* > \* **getSimulationControls** () const
 

*Returns the simulation controls owned by this data definition.*
- TraceManager::Level **getTraceLevelSpecific** () const
 

*Returns the trace level that applies specifically to this definition.*
- void **setTraceLevelSpecific** (TraceManager::Level level)
 

*Updates the trace level used by this definition.*
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)
 

*Defines the specific trace level and optionally enables it immediately.*
- bool **isTraceLevelSpecificEnabled** () const
 

*Indicates whether the specific trace level is enabled.*
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)
 

*Enables or disables the specific trace level.*



### Static Public Member Functions

- static [PluginInformation](#) \* **GetPluginInformation** ()  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static [ModelComponent](#) \* **LoadInstance** ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* **NewInstance** ([Model](#) \*model, std::string name="")
- static std::vector< std::string > **split** (std::string word, char split\_char)  
*Externalize function GetPluginInformation to be accessible through dynamic linked library.*

### Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) \* **LoadInstance** ([Model](#) \*model, [PersistenceRecord](#) \*fields)  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool **Check** ([ModelComponent](#) \*component)  
*Validates a component instance.*
- static void **CreateInternalData** ([ModelComponent](#) \*component)  
*Creates internal data required by a component instance.*
- static void **SaveInstance** ([PersistenceRecord](#) \*fields, [ModelComponent](#) \*component)  
*Serializes a component instance into a persistence record.*
- static void **DispatchEvent** ([Event](#) \*event)  
*Dispatches a simulation event to the target component.*

### Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* **LoadInstance** ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* **NewInstance** ([Model](#) \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void **SaveInstance** ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool **Check** ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void **InitBetweenReplications** ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*
- static void **CreateInternalData** ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

### Protected Member Functions

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the\_params={}, std::string the\_model="", std::string the\_model\_file="")  
*virtual protected method that must be overridden*
- void **UpdateConnections** ()
- virtual void **Build** ()
- virtual bool **\_loadInstance** ([PersistenceRecord](#) \*fields) override  
*virtual protected method that must be overridden*
- virtual void **\_saveInstance** ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*
- virtual void **\_onDispatchEvent** ([Entity](#) \*entity, unsigned int inputPortNumber) override  
*This method is only for ModelComponents, not ModelDataElements.*
- virtual void **\_createNonEditableDataDefinitions** () override  
*virtual protected methods that could be overridden by derived classes, if needed*

## Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [\\_getSaveDefaultsOption](#) ()
- virtual bool [\\_check](#) (std::string &errorMessage)  
*could be overridden by derived classes*
- virtual [ParserChangesInformation](#) \* [\\_getParserChangesInformation](#) ()
- virtual void [\\_initBetweenReplications](#) ()
- virtual void [\\_addSimulationControl](#) ([SimulationControl](#) \*control)
- virtual void [\\_createInternalAndAttachedData](#) ()
- virtual void [\\_createInternalStatisticReporters](#) ()
- virtual void [\\_createAttachedAttributes](#) ()
- virtual void [\\_createEditableDataDefinitions](#) ()
- void [\\_templateCreateInternalStatisticReporters](#) ()
- void [\\_templateCreateAttachedAttributes](#) ()
- void [\\_templateCreateNonEditableDataDefinitions](#) ()
- void [\\_templateCreateEditableDataDefinitions](#) ()
- void [\\_internalStatisticReportersClear](#) ()  
*legacy helper methods used by the granular create\* hooks*
- void [\\_internalDataClear](#) ()
- void [\\_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*child)
- void [\\_internalDataRemove](#) (const std::string &key)
- void [\\_attachedDataClear](#) ()
- void [\\_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_attachedDataRemove](#) (const std::string &key)
- void [\\_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [\\_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_statisticReporterRemove](#) (const std::string &key)
- void [\\_statisticReportersClear](#) ()
- void [\\_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [\\_mandatoryAttachedAttributesClear](#) ()
- void [\\_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_mandatoryEditableDataDefinitionsClear](#) ()
- void [\\_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_optionalEditableDataDefinitionsClear](#) ()
- void [\\_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [\\_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_optionalNonEditableDataDefinitionsClear](#) ()
- void [\\_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)  
*method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))  
*not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1\\_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2\\_results](#))
- void [traceSimulation](#) (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))
- void [traceSimulation](#) (void \*thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))  
*Traces text to the simulation output.*
- void [traceSimulation](#) (void \*thisobject, [TraceManager::Level](#) level, const std::string &text)

**Protected Attributes**

- unsigned int **id**
- std::function< void()> **build**

**Protected Attributes inherited from [ModelComponent](#)**

- [ConnectionManager](#) \* **\_connections** = new [ConnectionManager](#)()  
*virtual protected methods that could be overridden by derived classes, if needed*

**Protected Attributes inherited from [ModelDataDefinition](#)**

- Util::identification **\_id**
- std::string **\_typename**
- bool **\_reportStatistics**
- bool **\_hasChanged**
- unsigned int **\_modelLevel** = 0
- [Model](#) \* **\_parentModel**
- List< [SimulationControl](#) \* > \* **\_simulationControls** = new List<[SimulationControl](#)\*>()
- TraceManager::Level **\_traceLevelSpecific**
- bool **\_traceLevelSpecificEnabled** = false

**15.925.1 Detailed Description**

This component ...

**15.925.2 Member Function Documentation****15.925.2.1 [\\_createNonEditableDataDefinitions\(\)](#)**

```
void SPICECircuit::_createNonEditableDataDefinitions () [override], [protected], [virtual]
```

virtual protected methods that could be overridden by derived classes, if needed

This method is called by ModelChecker during model check. The component should check itself to verify if user parameters are ok (ex: correct syntax for the parser) and everything in its parameters allow the model too run without errors in this component

This method returns all changes in the parser that are needed by plugins of this ModelDatas. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dinamically linked to to simulator kernel to reflect the changes

This method is called by [ModelSimulation](#) when initiating the replication. The model should set all value for a new replication (Ex: setting back to 0 any internal counter, clearing lists, etc.

Reimplemented from [ModelDataDefinition](#).

**15.925.2.2 [\\_loadInstance\(\)](#)**

```
bool SPICECircuit::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

Reimplemented in [Resistor](#), [Vsource](#), [Vpulse](#), [Vsine](#), [Capacitor](#), [Diode](#), [PMOS](#), [NMOS](#), [NOT](#), [NAND](#), [AND](#), [NOR](#), [OR](#), [XOR](#), and [XNOR](#).

**15.925.2.3 [\\_onDispatchEvent\(\)](#)**

```
void SPICECircuit::_onDispatchEvent (
 Entity * entity,
 unsigned int inputPortNumber) [override], [protected], [virtual]
```

This method is only for ModelComponents, not ModelDataElements.

Implements [ModelComponent](#).

#### 15.925.2.4 `_saveInstance()`

```
void SPICECircuit::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

##### Parameters

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [ModelComponent](#).

Reimplemented in [Resistor](#), [Vsource](#), [Vpulse](#), [Vsine](#), [Capacitor](#), [Diode](#), [PMOS](#), [NMOS](#), [NOT](#), [NAND](#), [AND](#), [NOR](#), [OR](#), [XOR](#), and [XNOR](#).

#### 15.925.2.5 `show()`

```
std::string SPICECircuit::show () [override], [virtual]
```

virtual public methods

Reimplemented from [ModelComponent](#).

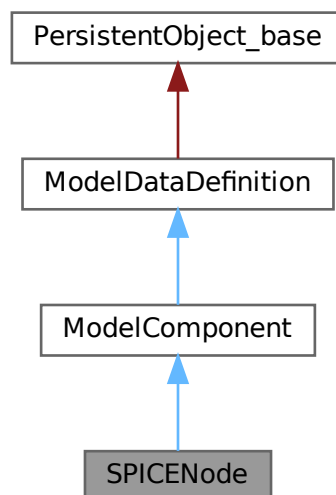
The documentation for this class was generated from the following files:

- `plugins/components/ElectronicsSimulation/SPICECircuit.h`
- `plugins/components/ElectronicsSimulation/SPICECircuit.cpp`

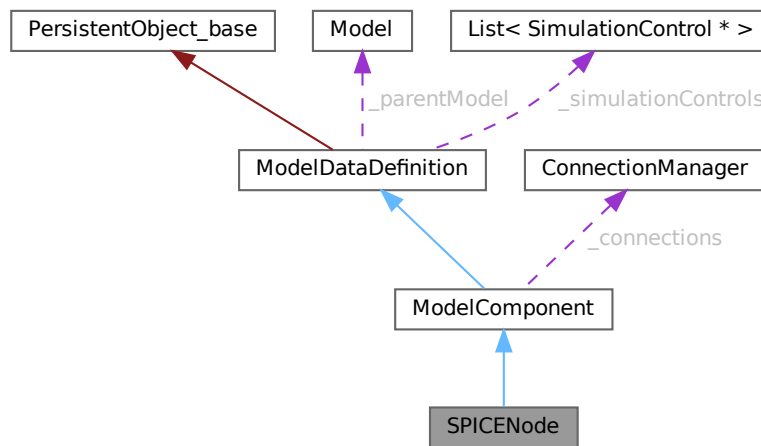
## 15.926 SPICENode Class Reference

```
#include <plugins/components/ElectronicsSimulation/SPICENode.h>
```

Inheritance diagram for SPICENode:



Collaboration diagram for SPICENode:



### Public Member Functions

- **SPICENode** ([Model](#) \*model, std::string name="")  
*constructors*
- std::string **getNodeName** ()  
*new public user methods for this component*
- virtual std::string **show** () override  
*virtual public methods*
- **operator std::string** () const

### Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) \*model, std::string componentTypename, std::string name="")  
*constructors*
- virtual **~ModelComponent** ()  
*Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) \* **getConnectionManager** () const  
*new public user methods for this component*
- void **connectTo** ([ModelComponent](#) \*component, unsigned int inputPortNumber=0)  
*Connects this component to another component.*
- bool **hasBreakpointAt** ()  
*Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string \_description)  
*Updates the free-form description shown by the GUI.*
- std::string **getDescription** () const  
*Returns the free-form description associated with this component.*
- void **remove** ([ModelComponent](#) \*comp)  
*Removes a connected component from this component.*

## Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** \*model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
 

*Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
 

*Releases resources owned by this data definition.*
- Util::identification **getId** () const
 

*Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
 

*Updates the logical name of this instance.*
- const std::string & **getName** () const
 

*Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)
 

*Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const
 

*Returns the user-visible label of this instance.*
- const std::string & **getClassname** () const
 

*Returns the runtime class name used by the plugin system.*
- **Model** \* **getParentModel** () const
 

*Returns the parent model that owns this definition.*
- bool **isReportStatistics** () const
 

*Indicates whether this data definition contributes report statistics.*
- void **setReportStatistics** (bool reportStatistics)
 

*Enables or disables report-statistics generation for this definition.*
- **ModelDataDefinition** \* **getInternalData** (const std::string &name) const
 

*Looks up a named internal child data definition.*
- std::map< std::string, **ModelDataDefinition** \* > \* **getInternalData** () const
 

*Returns the full map of internal child data definitions.*
- std::map< std::string, **ModelDataDefinition** \* > \* **getAttachedData** () const
 

*Returns the full map of attached data definitions.*
- bool **hasChanged** () const
 

*Indicates whether this instance has been modified since the last reset.*
- void **setHasChanged** (bool **hasChanged**)
 

*Updates the changed flag for this data definition.*
- unsigned int **getLevel** () const
 

*Returns the level in the model hierarchy where this definition lives.*
- void **setModelLevel** (unsigned int \_modelLevel)
 

*Sets the model nesting level for this definition.*
- List< **SimulationControl** \* > \* **getSimulationControls** () const
 

*Returns the simulation controls owned by this data definition.*
- TraceManager::Level **getTraceLevelSpecific** () const
 

*Returns the trace level that applies specifically to this definition.*
- void **setTraceLevelSpecific** (TraceManager::Level level)
 

*Updates the trace level used by this definition.*
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)
 

*Defines the specific trace level and optionally enables it immediately.*
- bool **isTraceLevelSpecificEnabled** () const
 

*Indicates whether the specific trace level is enabled.*
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)
 

*Enables or disables the specific trace level.*

### Static Public Member Functions

- static [PluginInformation](#) \* **GetPluginInformation** ()  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static [ModelComponent](#) \* **LoadInstance** ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* **NewInstance** ([Model](#) \*model, std::string name="")

### Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) \* **LoadInstance** ([Model](#) \*model, [PersistenceRecord](#) \*fields)  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool **Check** ([ModelComponent](#) \*component)  
*Validates a component instance.*
- static void **CreateInternalData** ([ModelComponent](#) \*component)  
*Creates internal data required by a component instance.*
- static void **SaveInstance** ([PersistenceRecord](#) \*fields, [ModelComponent](#) \*component)  
*Serializes a component instance into a persistence record.*
- static void **DispatchEvent** ([Event](#) \*event)  
*Dispatches a simulation event to the target component.*

### Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* **LoadInstance** ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* **NewInstance** ([Model](#) \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void **SaveInstance** ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool **Check** ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void **InitBetweenReplications** ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*
- static void **CreateInternalData** ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

### Protected Member Functions

- virtual bool **\_loadInstance** ([PersistenceRecord](#) \*fields) override  
*virtual protected method that must be overridden*
- virtual void **\_saveInstance** ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*
- virtual void **\_onDispatchEvent** ([Entity](#) \*entity, unsigned int inputPortNumber) override  
*This method is only for ModelComponents, not ModelDataElements.*

## Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [\\_getSaveDefaultsOption](#) ()
- virtual bool [\\_check](#) (std::string &errorMessage)  
*could be overridden by derived classes*
- virtual [ParserChangesInformation](#) \* [\\_getParserChangesInformation](#) ()
- virtual void [\\_initBetweenReplications](#) ()
- virtual void [\\_addSimulationControl](#) ([SimulationControl](#) \*control)
- virtual void [\\_createInternalAndAttachedData](#) ()
- virtual void [\\_createInternalStatisticReporters](#) ()
- virtual void [\\_createAttachedAttributes](#) ()
- virtual void [\\_createNonEditableDataDefinitions](#) ()
- virtual void [\\_createEditableDataDefinitions](#) ()
- void [\\_templateCreateInternalStatisticReporters](#) ()
- void [\\_templateCreateAttachedAttributes](#) ()
- void [\\_templateCreateNonEditableDataDefinitions](#) ()
- void [\\_templateCreateEditableDataDefinitions](#) ()
- void [\\_internalStatisticReportersClear](#) ()  
*legacy helper methods used by the granular create\* hooks*
- void [\\_internalDataClear](#) ()
- void [\\_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*child)
- void [\\_internalDataRemove](#) (const std::string &key)
- void [\\_attachedDataClear](#) ()
- void [\\_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_attachedDataRemove](#) (const std::string &key)
- void [\\_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [\\_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_statisticReporterRemove](#) (const std::string &key)
- void [\\_statisticReportersClear](#) ()
- void [\\_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [\\_mandatoryAttachedAttributesClear](#) ()
- void [\\_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_mandatoryEditableDataDefinitionsClear](#) ()
- void [\\_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_optionalEditableDataDefinitionsClear](#) ()
- void [\\_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [\\_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_optionalNonEditableDataDefinitionsClear](#) ()
- void [\\_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)  
*method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))  
*not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1\\_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2\\_results](#))
- void [traceSimulation](#) (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))
- void [traceSimulation](#) (void \*thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))  
*Traces text to the simulation output.*
- void [traceSimulation](#) (void \*thisobject, [TraceManager::Level](#) level, const std::string &text)



## Additional Inherited Members

### Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) \* `_connections` = new [ConnectionManager](#)()  
*virtual protected methods that could be overridden by derived classes, if needed*

### Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) \* `_parentModel`
- [List](#)< [SimulationControl](#) \* > \* `_simulationControls` = new [List](#)<[SimulationControl](#)\*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

## 15.926.1 Detailed Description

This component ...

## 15.926.2 Member Function Documentation

### 15.926.2.1 `_loadInstance()`

```
bool SPICENode::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

virtual protected method that must be overridden  
Reimplemented from [ModelComponent](#).

### 15.926.2.2 `_onDispatchEvent()`

```
void SPICENode::_onDispatchEvent (
 Entity * entity,
 unsigned int inputPortNumber) [override], [protected], [virtual]
```

This method is only for ModelComponents, not ModelDataElements.  
Implements [ModelComponent](#).

### 15.926.2.3 `_saveInstance()`

```
void SPICENode::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

#### Parameters

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [ModelComponent](#).

### 15.926.2.4 `show()`

```
std::string SPICENode::show () [override], [virtual]
```

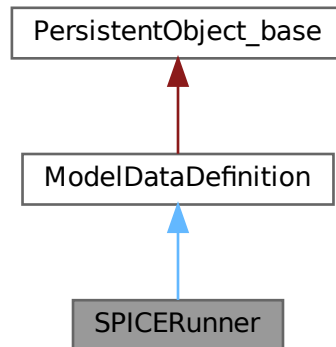
virtual public methods  
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

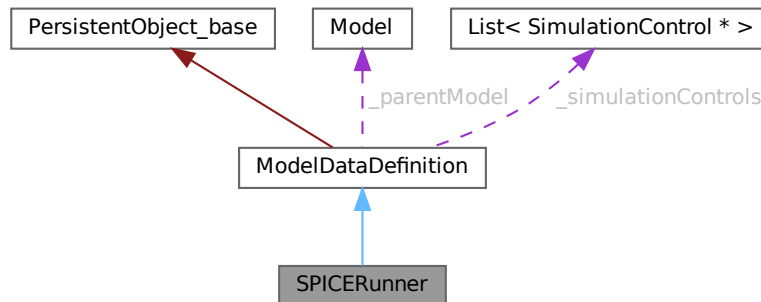
- plugins/components/ElectronicsSimulation/SPICENode.h
- plugins/components/ElectronicsSimulation/SPICENode.cpp

## 15.927 SPICERunner Class Reference

Inheritance diagram for SPICERunner:



Collaboration diagram for SPICERunner:



### Public Member Functions

- **SPICERunner** ([Model](#) \*model, std::string name="")  
*constructors*
- std::string **CompileSpiceFile** ()  
*new public user methods for this component*
- void **SendComponent** (std::string \*instance, std::string subcircuit="", std::string model="")
- void **SendCallback** (std::function< void()> callback)
- void **ConfigSim** (double duration, double step)
- void **Run** ()

- void **PlotV** (std::string net)
- template<typename... Args>  
void **PlotV** (std::string net, Args... args)
- void **PlotVRelative** (std::string comparison\_base, std::string net)
- template<typename... Args>  
void **PlotVRelative** (std::string comparison\_base, std::string net, Args... args)
- void **PlotI** (std::string net)
- template<typename... Args>  
void **PlotI** (std::string net, Args... args)
- void **PlotIRelative** (std::string comparison\_base, std::string net)
- template<typename... Args>  
void **PlotIRelative** (std::string comparison\_base, std::string net, Args... args)
- double \* **MeasurePeak** (std::string label, std::string peak, std::string quantity, std::string node, float start, float finish)
- double \* **MeasureTrigTarg** (std::string label, std::string trig, float trig\_value, std::string trig\_inclin, std::string targ, float targ\_value, std::string targ\_inclin)
- void **ParseOutput** ()
- void **setRunnerCommand** (std::string runnerCommand)
- std::string **getRunnerCommand** () const
- void **setModelsPath** (std::string modelsPath)
- std::string **getModelsPath** () const
- void **setWorkingInputFilename** (std::string workingInputFilename)
- std::string **getWorkingInputFilename** () const
- void **setWorkingOutputFilename** (std::string workingOutputFilename)
- std::string **getWorkingOutputFilename** () const
- void **setWorkingDirectory** (std::string workingDirectory)
- std::string **getWorkingDirectory** () const
- std::string **getLastRunCommand** () const
- virtual std::string **show** () override

*virtual public methods*

## Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) \*model, std::string datadefinitionType, std::string name="", bool insert↵ IntoModel=true)  
*Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()  
*Releases resources owned by this data definition.*
- Util::identification [getId](#) () const  
*Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)  
*Updates the logical name of this instance.*
- const std::string & [getName](#) () const  
*Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)  
*Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const  
*Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const  
*Returns the runtime class name used by the plugin system.*
- [Model](#) \* [getParentModel](#) () const  
*Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const

- Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)  
*Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) \* [getInternalData](#) (const std::string &name) const  
*Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getInternalData](#) () const  
*Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getAttachedData](#) () const  
*Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const  
*Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool hasChanged)  
*Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const  
*Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int \_modelLevel)  
*Sets the model nesting level for this definition.*
- List< [SimulationControl](#) \* > \* [getSimulationControls](#) () const  
*Returns the simulation controls owned by this data definition.*
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const  
*Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)  
*Updates the trace level used by this definition.*
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)  
*Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const  
*Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)  
*Enables or disables the specific trace level.*

### Static Public Member Functions

- static [PluginInformation](#) \* [GetPluginInformation](#) ()  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")
- static std::vector< std::string > [split](#) (std::string word, char split\_char)

### Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool [Check](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*

- static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

### Protected Member Functions

- virtual bool [\\_loadInstance](#) ([PersistenceRecord](#) \*fields) override  
*virtual protected method that must be overridden*
- virtual void [\\_saveInstance](#) ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*
- virtual bool [\\_check](#) (std::string &errorMessage) override  
*virtual protected methods that could be overridden by derived classes, if needed*

### Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [\\_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) \* [\\_getParserChangesInformation](#) ()
- virtual void [\\_initBetweenReplications](#) ()
- virtual void [\\_addSimulationControl](#) ([SimulationControl](#) \*control)
- virtual void [\\_createInternalAndAttachedData](#) ()
- virtual void [\\_createInternalStatisticReporters](#) ()
- virtual void [\\_createAttachedAttributes](#) ()
- virtual void [\\_createNonEditableDataDefinitions](#) ()
- virtual void [\\_createEditableDataDefinitions](#) ()
- void [\\_templateCreateInternalStatisticReporters](#) ()
- void [\\_templateCreateAttachedAttributes](#) ()
- void [\\_templateCreateNonEditableDataDefinitions](#) ()
- void [\\_templateCreateEditableDataDefinitions](#) ()
- void [\\_internalStatisticReportersClear](#) ()  
*legacy helper methods used by the granular create\* hooks*
- void [\\_internalDataClear](#) ()
- void [\\_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*child)
- void [\\_internalDataRemove](#) (const std::string &key)
- void [\\_attachedDataClear](#) ()
- void [\\_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_attachedDataRemove](#) (const std::string &key)
- void [\\_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [\\_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_statisticReporterRemove](#) (const std::string &key)
- void [\\_statisticReportersClear](#) ()
- void [\\_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [\\_mandatoryAttachedAttributesClear](#) ()
- void [\\_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_mandatoryEditableDataDefinitionsClear](#) ()
- void [\\_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_optionalEditableDataDefinitionsClear](#) ()
- void [\\_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [\\_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)

- void **\_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalNonEditableDataDefinitionsClear** ()
- void **\_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)  
*method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)  
*not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- void [traceSimulation](#) (void \*thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)  
*Traces text to the simulation output.*
- void **traceSimulation** (void \*thisobject, TraceManager::Level level, const std::string &text)

## Additional Inherited Members

### Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **\_id**
- std::string **\_typename**
- bool **\_reportStatistics**
- bool **\_hasChanged**
- unsigned int **\_modelLevel** = 0
- [Model](#) \* **\_parentModel**
- [List](#)< [SimulationControl](#) \* > \* **\_simulationControls** = new [List](#)<[SimulationControl](#)\*>()
- TraceManager::Level **\_traceLevelSpecific**
- bool **\_traceLevelSpecificEnabled** = false

## 15.927.1 Member Function Documentation

### 15.927.1.1 \_check()

```
bool SPICERunner::_check (
 std::string & errorMessage) [override], [protected], [virtual]
```

virtual protected methods that could be overridden by derived classes, if needed

This method is called by ModelChecker during model check. The component should check itself to verify if user parameters are ok (ex: correct syntax for the parser) and everything in its parameters allow the model too run without errors in this component

Reimplemented from [ModelDataDefinition](#).

### 15.927.1.2 \_loadInstance()

```
bool SPICERunner::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelDataDefinition](#).

### 15.927.1.3 \_saveInstance()

```
void SPICERunner::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

## Parameters

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [ModelDataDefinition](#).

**15.927.1.4 show()**

```
std::string SPICERunner::show () [override], [virtual]
```

virtual public methods

Reimplemented from [ModelDataDefinition](#).

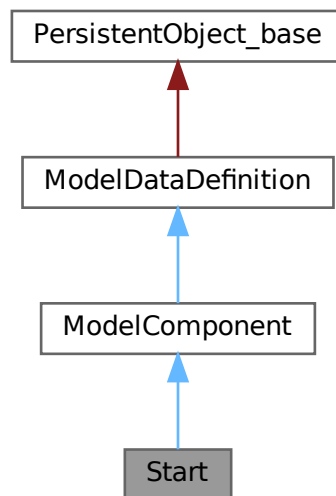
The documentation for this class was generated from the following files:

- plugins/data/ExternalIntegration/SPICERunner.h
- plugins/data/ExternalIntegration/SPICERunner.cpp

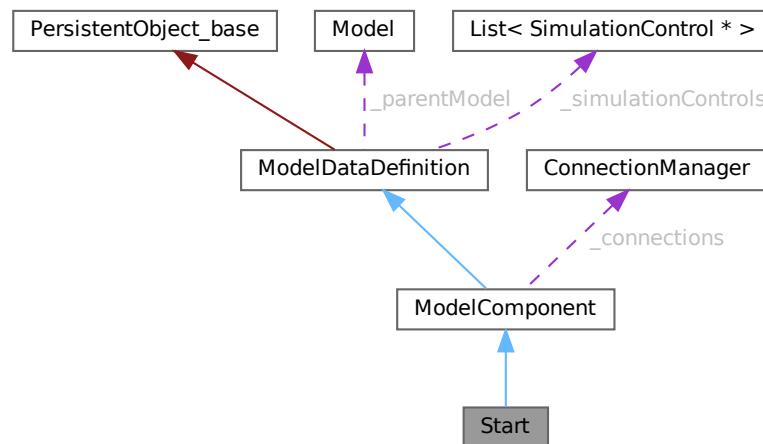
## 15.928 Start Class Reference

```
#include <plugins/components/MaterialHandling/Start.h>
```

Inheritance diagram for Start:



Collaboration diagram for Start:



### Public Member Functions

- **Start** ([Model](#) \*model, std::string name="")
  - virtual std::string [show](#) () override
- virtual public methods*

### Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) \*model, std::string componentTypename, std::string name="")  
*constructors*
- virtual ~**ModelComponent** ()  
*Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) \* [getConnectionManager](#) () const  
*new public user methods for this component*
- void [connectTo](#) ([ModelComponent](#) \*component, unsigned int inputPortNumber=0)  
*Connects this component to another component.*
- bool [hasBreakpointAt](#) ()  
*Indicates whether a simulation breakpoint is set on this component.*
- void [setDescription](#) (std::string \_description)  
*Updates the free-form description shown by the GUI.*
- std::string [getDescription](#) () const  
*Returns the free-form description associated with this component.*
- void [remove](#) ([ModelComponent](#) \*comp)  
*Removes a connected component from this component.*

### Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) \*model, std::string datadefinitionTypename, std::string name="", bool insert↔ IntoModel=true)  
*Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()  
*Releases resources owned by this data definition.*



- Util::identification [getId](#) () const  
*Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)  
*Updates the logical name of this instance.*
- const std::string & [getName](#) () const  
*Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)  
*Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const  
*Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const  
*Returns the runtime class name used by the plugin system.*
- [Model](#) \* [getParentModel](#) () const  
*Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const  
*Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)  
*Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) \* [getInternalData](#) (const std::string &name) const  
*Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getInternalData](#) () const  
*Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getAttachedData](#) () const  
*Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const  
*Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))  
*Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const  
*Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int \_modelLevel)  
*Sets the model nesting level for this definition.*
- [List](#)< [SimulationControl](#) \* > \* [getSimulationControls](#) () const  
*Returns the simulation controls owned by this data definition.*
- [TraceManager](#)::Level [getTraceLevelSpecific](#) () const  
*Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) ([TraceManager](#)::Level level)  
*Updates the trace level used by this definition.*
- void [defineTraceLevelSpecific](#) ([TraceManager](#)::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)  
*Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const  
*Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)  
*Enables or disables the specific trace level.*

### Static Public Member Functions

- static [PluginInformation](#) \* [GetPluginInformation](#) ()
- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")

## Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool [Check](#) ([ModelComponent](#) \*component)  
*Validates a component instance.*
- static void [CreateInternalData](#) ([ModelComponent](#) \*component)  
*Creates internal data required by a component instance.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelComponent](#) \*component)  
*Serializes a component instance into a persistence record.*
- static void [DispatchEvent](#) ([Event](#) \*event)  
*Dispatches a simulation event to the target component.*

## Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool [Check](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*
- static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

## Protected Member Functions

- virtual void [\\_onDispatchEvent](#) ([Entity](#) \*entity, unsigned int inputPortNumber) override  
*Handles a dispatched event for this component type.*
- virtual bool [\\_loadInstance](#) ([PersistenceRecord](#) \*fields) override  
*virtual protected method that must be overridden*
- virtual void [\\_saveInstance](#) ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*
- virtual bool [\\_check](#) (std::string &errorMessage) override  
*could be overridden by derived classes*

## Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [\\_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) \* [\\_getParserChangesInformation](#) ()
- virtual void [\\_initBetweenReplications](#) ()
- virtual void [\\_addSimulationControl](#) ([SimulationControl](#) \*control)
- virtual void [\\_createInternalAndAttachedData](#) ()
- virtual void [\\_createInternalStatisticReporters](#) ()
- virtual void [\\_createAttachedAttributes](#) ()
- virtual void [\\_createNonEditableDataDefinitions](#) ()
- virtual void [\\_createEditableDataDefinitions](#) ()
- void [\\_templateCreateInternalStatisticReporters](#) ()

- void **\_templateCreateAttachedAttributes** ()
- void **\_templateCreateNonEditableDataDefinitions** ()
- void **\_templateCreateEditableDataDefinitions** ()
- void **\_internaStatisticReportersClear** ()
- legacy helper methods used by the granular create\* hooks*
- void **\_internalDataClear** ()
- void **\_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) \*child)
- void **\_internalDataRemove** (const std::string &key)
- void **\_attachedDataClear** ()
- void **\_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_attachedDataRemove** (const std::string &key)
- void **\_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **\_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_statisticReporterRemove** (const std::string &key)
- void **\_statisticReportersClear** ()
- void **\_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **\_mandatoryAttachedAttributesClear** ()
- void **\_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryEditableDataDefinitionsClear** ()
- void **\_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalEditableDataDefinitionsClear** ()
- void **\_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryNonEditableDataDefinitionsClear** ()
- void **\_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalNonEditableDataDefinitionsClear** ()
- void **\_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
- method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- void **traceSimulation** (void \*thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- Traces text to the simulation output.*
- void **traceSimulation** (void \*thisobject, TraceManager::Level level, const std::string &text)

### Additional Inherited Members

### Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) \* **\_connections** = new [ConnectionManager](#)()
- virtual protected methods that could be overridden by derived classes, if needed*

## Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) \* `_parentModel`
- [List](#)< [SimulationControl](#) \* > \* `_simulationControls` = new [List](#)<[SimulationControl](#)\*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

### 15.928.1 Detailed Description

[Start](#) module DESCRIPTION The [Start](#) module changes the status of a conveyor from inactive to active. The conveyor may have been deactivated from either the [Stop](#) module or by initially being set to inactive at the start of the simulation. The velocity of the conveyor may be changed permanently when the conveyor is started. TYPICAL USES [Start](#) a bottling conveyor after scheduled maintenance [Start](#) a baggage claim conveyor when bags have arrived PROMPTS Prompt Description Name Unique name of the module that will be displayed in the flowchart. Conveyor Name Name of the conveyor to start. Velocity Speed of the conveyor once it begins to operate. This value will change the speed of the conveyor permanently, until it is changed in another module. Units Velocity time units.

### 15.928.2 Member Function Documentation

#### 15.928.2.1 `_check()`

```
bool Start::_check (
 std::string & errorMessage) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

#### 15.928.2.2 `_loadInstance()`

```
bool Start::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

#### 15.928.2.3 `_onDispatchEvent()`

```
void Start::_onDispatchEvent (
 Entity * entity,
 unsigned int inputPortNumber) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for [ModelComponents](#), not for [ModelDataElements](#).

#### Parameters

|                        |                                         |
|------------------------|-----------------------------------------|
| <i>entity</i>          | <a href="#">Entity</a> being processed. |
| <i>inputPortNumber</i> | Input port where the entity arrived.    |

Implements [ModelComponent](#).

#### 15.928.2.4 `_saveInstance()`

```
void Start::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

## Parameters

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [ModelComponent](#).

**15.928.2.5 show()**

```
std::string Start::show () [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

## Returns

Human-readable description used for tracing and debugging.

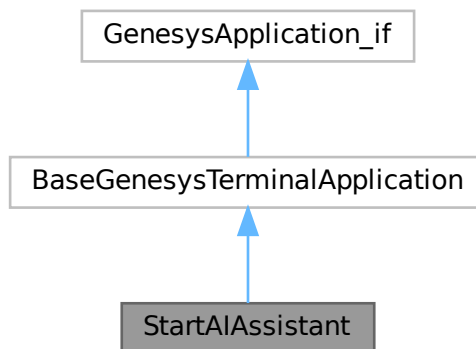
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

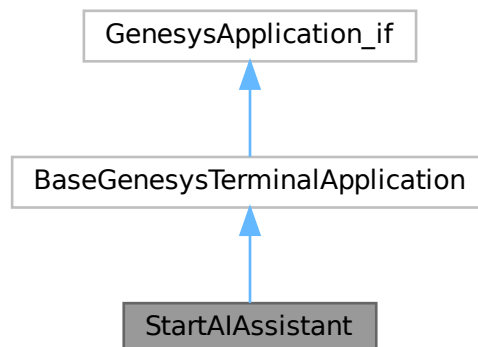
- plugins/components/MaterialHandling/Start.h
- plugins/components/MaterialHandling/Start.cpp

**15.929 StartAIAssistant Class Reference**

Inheritance diagram for StartAIAssistant:



Collaboration diagram for StartAIAssistant:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

## 15.929.1 Member Function Documentation

### 15.929.1.1 main()

```
int StartAIAssistant::main (
 int argc,
 char ** argv) [override], [virtual]
```

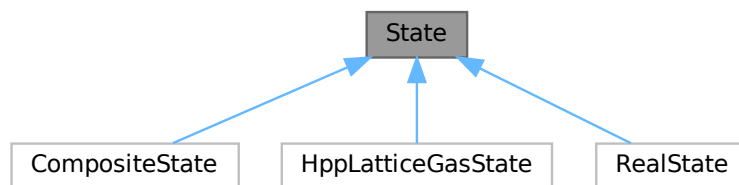
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/smarts/AI/StartAIAssistant.h
- applications/modelSpecific/smarts/AI/StartAIAssistant.cpp

## 15.930 State Class Reference

Inheritance diagram for State:



### Public Member Functions

- **State** (long value)
- **State** (const [State](#) &orig)
- virtual std::unique\_ptr< [State](#) > **clone** () const
- virtual bool **equals** (const [State](#) &other) const
- virtual std::string **toString** () const
- virtual std::string **show** () const
- virtual std::string **typeName** () const
- virtual long **getValue** () const
- virtual void **setValue** (long value)
- template<typename T >  
const T \* **as** () const noexcept
- template<typename T >  
T \* **as** () noexcept

### Protected Attributes

- long **\_value**

The documentation for this class was generated from the following file:

- plugins/components/ModalModel/CellularAutomata/State.h

## 15.931 GroProgramAst::Statement Struct Reference

### Public Attributes

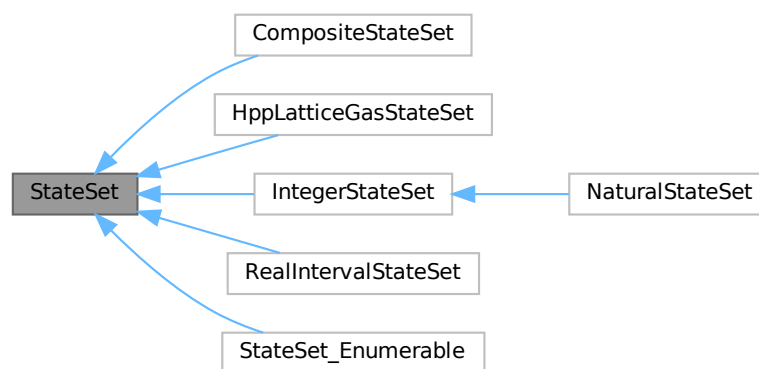
- `std::string sourceText = ""`

The documentation for this struct was generated from the following file:

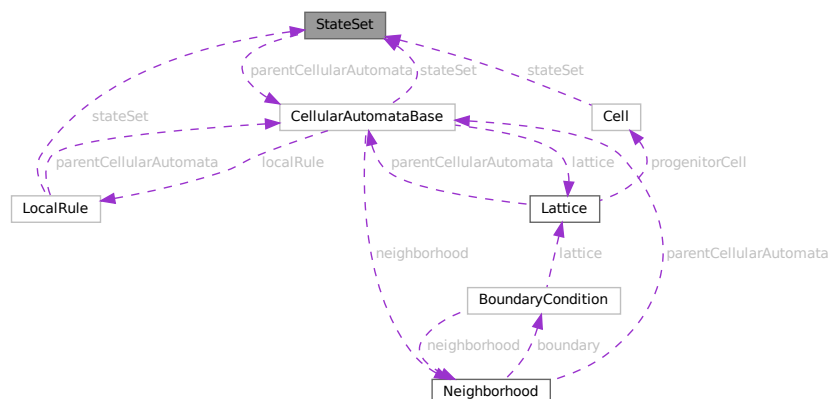
- `plugins/data/BiochemicalSimulation/GroProgramAst.h`

## 15.932 StateSet Class Reference

Inheritance diagram for StateSet:



Collaboration diagram for StateSet:



### Public Member Functions

- **StateSet** (`CellularAutomataBase` \*parentCellularAutomata)
- **StateSet** (const `StateSet` &orig)
- virtual bool **contains** (const `State` &state) const
- virtual `std::unique_ptr< State >` **createDefaultState** () const



- virtual std::unique\_ptr< [State](#) > **parseState** (const std::string &text) const
- virtual bool **isFinite** () const
- virtual bool **isDiscrete** () const
- virtual bool **isContinuous** () const
- virtual bool **isComposite** () const
- virtual std::vector< std::unique\_ptr< [State](#) > > **enumerateStates** () const
- virtual std::string **name** () const

### Protected Attributes

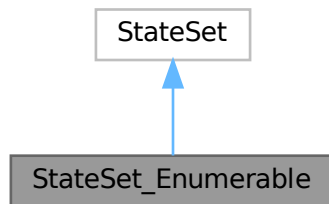
- [CellularAutomataBase](#) \* **parentCellularAutomata**

The documentation for this class was generated from the following files:

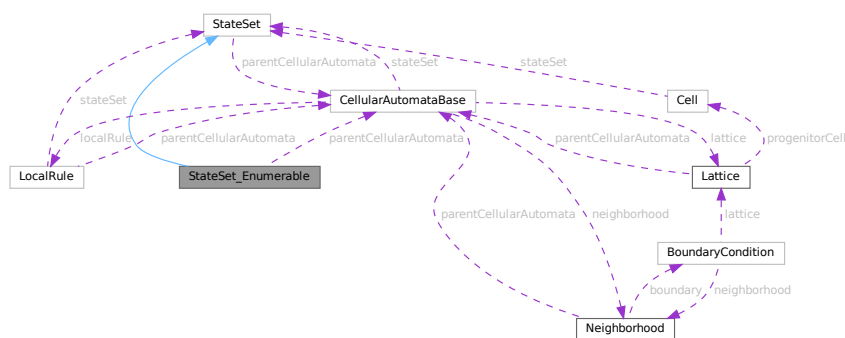
- plugins/components/ModalModel/CellularAutomata/StateSet.h
- plugins/components/ModalModel/CellularAutomata/StateSet.cpp

## 15.933 StateSet\_Enumerable Class Reference

Inheritance diagram for StateSet\_Enumerable:



Collaboration diagram for StateSet\_Enumerable:



### Public Member Functions

- **StateSet\_Enumerable** ([CellularAutomataBase](#) \*parentCellularAutomata, std::vector< [State](#) \* > states={})
- **StateSet\_Enumerable** (const [StateSet\\_Enumerable](#) &orig)
- virtual std::string **show** ()

- bool **contains** (const [State](#) &state) const override
- std::unique\_ptr< [State](#) > **createDefaultState** () const override
- std::unique\_ptr< [State](#) > **parseState** (const std::string &text) const override
- bool **isFinite** () const override
- bool **isDiscrete** () const override
- std::vector< std::unique\_ptr< [State](#) > > **enumerateStates** () const override
- std::string **name** () const override
- unsigned int **size** ()
- void **addState** ([State](#) \*state)
- void **addState** (const [State](#) &state)
- unsigned int **getStatesSize** ()
- [State](#) \* **getState** (unsigned int rank)
- const [State](#) \* **getState** (unsigned int rank) const
- [State](#) \* **getState** (std::string name)
- void **setStates** (std::vector< [State](#) \* > states)
- std::vector< [State](#) \* > **getStates** () const

## Public Member Functions inherited from [StateSet](#)

- **StateSet** ([CellularAutomataBase](#) \*parentCellularAutomata)
- **StateSet** (const [StateSet](#) &orig)
- virtual bool **isContinuous** () const
- virtual bool **isComposite** () const

## Protected Attributes

- [CellularAutomataBase](#) \* **parentCellularAutomata**
- std::vector< std::unique\_ptr< [State](#) > > **ownedStates**
- std::vector< [State](#) \* > **states**

## Protected Attributes inherited from [StateSet](#)

- [CellularAutomataBase](#) \* **parentCellularAutomata**

## 15.933.1 Member Function Documentation

### 15.933.1.1 contains()

```
bool StateSet_Enumerable::contains (
 const State & state) const [override], [virtual]
```

Reimplemented from [StateSet](#).

### 15.933.1.2 createDefaultState()

```
std::unique_ptr< State > StateSet_Enumerable::createDefaultState () const [override], [virtual]
```

Reimplemented from [StateSet](#).

### 15.933.1.3 enumerateStates()

```
std::vector< std::unique_ptr< State > > StateSet_Enumerable::enumerateStates () const [override], [virtual]
```

Reimplemented from [StateSet](#).

### 15.933.1.4 isDiscrete()

```
bool StateSet_Enumerable::isDiscrete () const [override], [virtual]
```

Reimplemented from [StateSet](#).

#### 15.933.1.5 isFinite()

```
bool StateSet_Enumerable::isFinite () const [override], [virtual]
```

Reimplemented from [StateSet](#).

#### 15.933.1.6 name()

```
std::string StateSet_Enumerable::name () const [override], [virtual]
```

Reimplemented from [StateSet](#).

#### 15.933.1.7 parseState()

```
std::unique_ptr< State > StateSet_Enumerable::parseState (
```

```
 const std::string & text) const [override], [virtual]
```

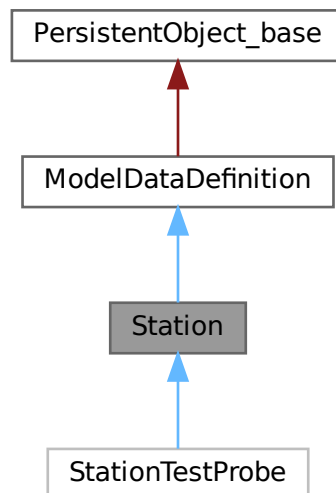
Reimplemented from [StateSet](#).

The documentation for this class was generated from the following files:

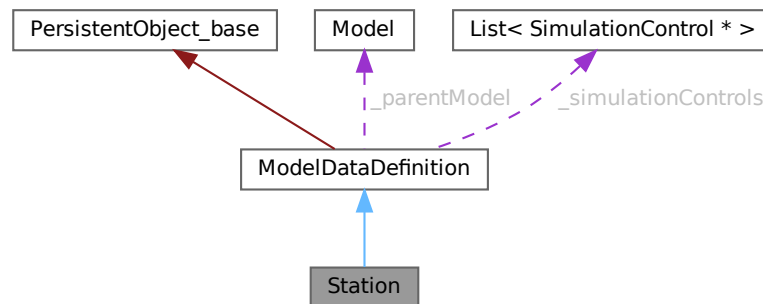
- plugins/components/ModalModel/CellularAutomata/StateSet\_Enumerable.h
- plugins/components/ModalModel/CellularAutomata/StateSet\_Enumerable.cpp

## 15.934 Station Class Reference

Inheritance diagram for Station:



Collaboration diagram for Station:



### Public Member Functions

- **Station** ([Model](#) \*model, std::string name="")
- virtual std::string **show** () override  
*Returns a textual representation of this data definition.*
- void **initBetweenReplications** ()
- void **enter** ([Entity](#) \*entity)
- void **leave** ([Entity](#) \*entity)
- void **setEnterIntoStationComponent** ([ModelComponent](#) \*\_enterIntoStationComponent)
- [ModelComponent](#) \* **getEnterIntoStationComponent** () const

### Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) \*model, std::string datadefinitionTypename, std::string name="", bool insert↔IntoModel=true)  
*Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()  
*Releases resources owned by this data definition.*
- Util::identification **getId** () const  
*Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)  
*Updates the logical name of this instance.*
- const std::string & **getName** () const  
*Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)  
*Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const  
*Returns the user-visible label of this instance.*
- const std::string & **getClassname** () const  
*Returns the runtime class name used by the plugin system.*
- [Model](#) \* **getParentModel** () const  
*Returns the parent model that owns this definition.*
- bool **isReportStatistics** () const  
*Indicates whether this data definition contributes report statistics.*
- void **setReportStatistics** (bool reportStatistics)  
*Enables or disables report-statistics generation for this definition.*

- [ModelDataDefinition](#) \* [getInternalData](#) (const std::string &name) const  
*Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getInternalData](#) () const  
*Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getAttachedData](#) () const  
*Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const  
*Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))  
*Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const  
*Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int \_modelLevel)  
*Sets the model nesting level for this definition.*
- List< [SimulationControl](#) \* > \* [getSimulationControls](#) () const  
*Returns the simulation controls owned by this data definition.*
- TraceManager::Level [getTraceLevelSpecific](#) () const  
*Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) (TraceManager::Level level)  
*Updates the trace level used by this definition.*
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)  
*Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const  
*Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)  
*Enables or disables the specific trace level.*

### Static Public Member Functions

- static [PluginInformation](#) \* [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")

### Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool [Check](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*
- static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

### Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) \*fields) override  
*must be overridden by derived classes*
- virtual void `_saveInstance` ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*
- virtual bool `_check` (std::string &errorMessage) override  
*could be overridden by derived classes*
- virtual void `_initBetweenReplications` () override
- virtual void `_createInternalStatisticReporters` () override
- virtual void `_createAttachedAttributes` () override

### Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) \* `_getParserChangesInformation` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) \*control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()  
*legacy helper methods used by the granular create\* hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) \*child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)  
*method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void `trace` (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))

*not just an easy access to trace manager, but wrappers to check if specific trace level applies*

- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- void [traceSimulation](#) (void \*thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)

*Traces text to the simulation output.*

- void **traceSimulation** (void \*thisobject, TraceManager::Level level, const std::string &text)

## Friends

- class **StationTestProbe**

## Additional Inherited Members

### Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **\_id**
- std::string **\_typename**
- bool **\_reportStatistics**
- bool **\_hasChanged**
- unsigned int **\_modelLevel** = 0
- [Model](#) \* **\_parentModel**
- [List](#)< [SimulationControl](#) \* > \* **\_simulationControls** = new [List](#)<[SimulationControl](#)\*>()
- TraceManager::Level **\_traceLevelSpecific**
- bool **\_traceLevelSpecificEnabled** = false

## 15.934.1 Member Function Documentation

### 15.934.1.1 **\_check()**

```
bool Station::_check (
 std::string & errorMessage) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

### 15.934.1.2 **\_createAttachedAttributes()**

```
void Station::_createAttachedAttributes () [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

### 15.934.1.3 **\_createInternalStatisticReporters()**

```
void Station::_createInternalStatisticReporters () [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

### 15.934.1.4 **\_initBetweenReplications()**

```
void Station::_initBetweenReplications () [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

### 15.934.1.5 **\_loadInstance()**

```
bool Station::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

**15.934.1.6 \_saveInstance()**

```
void Station::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

**Parameters**

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [ModelDataDefinition](#).

**15.934.1.7 show()**

```
std::string Station::show () [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

**Returns**

Human-readable summary used by traces and debug output.

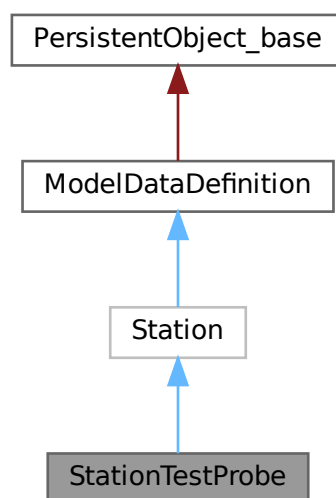
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- plugins/data/MaterialHandling/Station.h
- plugins/data/MaterialHandling/Station.cpp

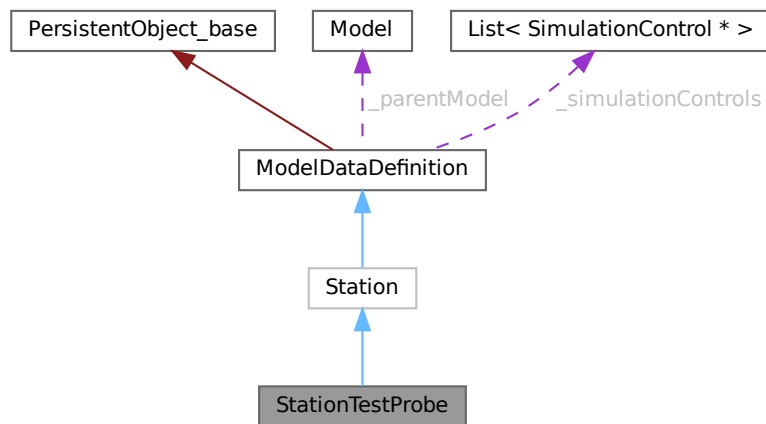
**15.935 StationTestProbe Class Reference**

Inheritance diagram for StationTestProbe:





Collaboration diagram for StationTestProbe:



### Public Member Functions

- **StationTestProbe** ([Model](#) \*model, const std::string &name="")
- void **CreateInternalAndAttachedDataProbe** ()
- void **InitBetweenReplicationsProbe** ()
- [StatisticsCollector](#) \* **NumberInStationCollectorProbe** () const
- [StatisticsCollector](#) \* **TimeInStationCollectorProbe** () const
- unsigned int **NumberInStationProbe** () const

### Public Member Functions inherited from [Station](#)

- **Station** ([Model](#) \*model, std::string name="")
- virtual std::string **show** () override  
*Returns a textual representation of this data definition.*
- void **initBetweenReplications** ()
- void **enter** ([Entity](#) \*entity)
- void **leave** ([Entity](#) \*entity)
- void **setEnterIntoStationComponent** ([ModelComponent](#) \* \_enterIntoStationComponent)
- [ModelComponent](#) \* **getEnterIntoStationComponent** () const

### Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) \*model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)  
*Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()  
*Releases resources owned by this data definition.*
- Util::identification **getId** () const  
*Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)  
*Updates the logical name of this instance.*
- const std::string & **getName** () const  
*Returns the logical name of this instance.*

- void [setLabel](#) (const std::string &label)  
*Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const  
*Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const  
*Returns the runtime class name used by the plugin system.*
- [Model](#) \* [getParentModel](#) () const  
*Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const  
*Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)  
*Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) \* [getInternalData](#) (const std::string &name) const  
*Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getInternalData](#) () const  
*Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getAttachedData](#) () const  
*Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const  
*Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))  
*Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const  
*Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int \_modelLevel)  
*Sets the model nesting level for this definition.*
- [List](#)< [SimulationControl](#) \* > \* [getSimulationControls](#) () const  
*Returns the simulation controls owned by this data definition.*
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const  
*Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)  
*Updates the trace level used by this definition.*
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)  
*Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const  
*Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)  
*Enables or disables the specific trace level.*

### Additional Inherited Members

### Static Public Member Functions inherited from [Station](#)

- static [PluginInformation](#) \* [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")

## Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool [Check](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*
- static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

## Protected Member Functions inherited from [Station](#)

- virtual bool [\\_loadInstance](#) ([PersistenceRecord](#) \*fields) override  
*must be overridden by derived classes*
- virtual void [\\_saveInstance](#) ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*
- virtual bool [\\_check](#) (std::string &errorMessage) override  
*could be overridden by derived classes*
- virtual void [\\_initBetweenReplications](#) () override
- virtual void [\\_createInternalStatisticReporters](#) () override
- virtual void [\\_createAttachedAttributes](#) () override

## Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [\\_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) \* [\\_getParserChangesInformation](#) ()
- virtual void [\\_addSimulationControl](#) ([SimulationControl](#) \*control)
- virtual void [\\_createInternalAndAttachedData](#) ()
- virtual void [\\_createNonEditableDataDefinitions](#) ()
- virtual void [\\_createEditableDataDefinitions](#) ()
- void [\\_templateCreateInternalStatisticReporters](#) ()
- void [\\_templateCreateAttachedAttributes](#) ()
- void [\\_templateCreateNonEditableDataDefinitions](#) ()
- void [\\_templateCreateEditableDataDefinitions](#) ()
- void [\\_internalStatisticReportersClear](#) ()  
*legacy helper methods used by the granular create\* hooks*
- void [\\_internalDataClear](#) ()
- void [\\_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*child)
- void [\\_internalDataRemove](#) (const std::string &key)
- void [\\_attachedDataClear](#) ()
- void [\\_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_attachedDataRemove](#) (const std::string &key)
- void [\\_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [\\_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_statisticReporterRemove](#) (const std::string &key)
- void [\\_statisticReportersClear](#) ()
- void [\\_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)

- void **\_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **\_mandatoryAttachedAttributesClear** ()
- void **\_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryEditableDataDefinitionsClear** ()
- void **\_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalEditableDataDefinitionsClear** ()
- void **\_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryNonEditableDataDefinitionsClear** ()
- void **\_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalNonEditableDataDefinitionsClear** ()
- void **\_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)  
*method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)  
*not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- void [traceSimulation](#) (void \*thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)  
*Traces text to the simulation output.*
- void **traceSimulation** (void \*thisobject, TraceManager::Level level, const std::string &text)

## Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **\_id**
- std::string **\_typename**
- bool **\_reportStatistics**
- bool **\_hasChanged**
- unsigned int **\_modelLevel** = 0
- [Model](#) \* **\_parentModel**
- [List](#)< [SimulationControl](#) \* > \* **\_simulationControls** = new [List](#)<[SimulationControl](#)\*>()
- TraceManager::Level **\_traceLevelSpecific**
- bool **\_traceLevelSpecificEnabled** = false

The documentation for this class was generated from the following file:

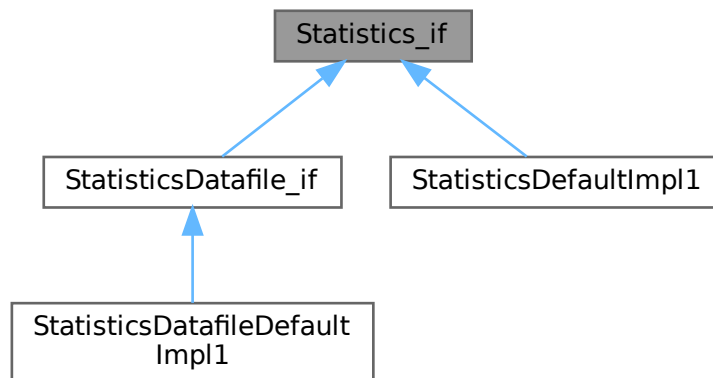
- tests/unit/test\_simulator\_runtime.cpp

## 15.936 Statistics\_if Class Reference

Interface for statistical synthesis over values provided by a [Collector\\_if](#).

```
#include <kernel/statistics/Statistics_if.h>
```

Inheritance diagram for Statistics\_if:



### Public Member Functions

- virtual [Collector\\_if](#) \* [getCollector](#) () const =0  
*Returns the data collector associated with these statistics.*
- virtual void [setCollector](#) ([Collector\\_if](#) \*collector)=0  
*Sets the collector used as source for statistical calculations.*
- virtual unsigned int [numElements](#) ()=0  
*numElements*
- virtual double [min](#) ()=0  
*min*
- virtual double [max](#) ()=0  
*max*
- virtual double [average](#) ()=0  
*average*
- virtual double [variance](#) ()=0  
*variance*
- virtual double [stddeviation](#) ()=0  
*stddeviation*
- virtual double [variationCoef](#) ()=0  
*variationCoef*
- virtual double [halfWidthConfidenceInterval](#) ()=0  
*halfWidthConfidenceInterval*
- virtual double [confidenceLevel](#) ()=0  
*confidenceLevel*
- virtual unsigned int [newSampleSize](#) (double halfWidth)=0  
*newSampleSize*
- virtual void [setConfidenceLevel](#) (double confidencelevel)=0  
*setConfidenceLevel*

### 15.936.1 Detailed Description

Interface for statistical synthesis over values provided by a [Collector\\_if](#).

Implementations may compute metrics incrementally (online) or based on persisted datasets, but must expose a consistent API for descriptive and inferential measures.

## 15.936.2 Member Function Documentation

### 15.936.2.1 average()

```
virtual double Statistics_if::average () [pure virtual]
```

average

Returns

Returns the sample mean of observations.

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

### 15.936.2.2 confidenceLevel()

```
virtual double Statistics_if::confidenceLevel () [pure virtual]
```

confidenceLevel

Returns

Returns the confidence level used for interval estimates.

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

### 15.936.2.3 getCollector()

```
virtual Collector_if * Statistics_if::getCollector () const [pure virtual]
```

Returns the data collector associated with these statistics.

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

### 15.936.2.4 halfWidthConfidenceInterval()

```
virtual double Statistics_if::halfWidthConfidenceInterval () [pure virtual]
```

halfWidthConfidenceInterval

Returns

Returns the half-width of the current confidence interval.

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

### 15.936.2.5 max()

```
virtual double Statistics_if::max () [pure virtual]
```

max

Returns

Returns the maximum observed value.

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

### 15.936.2.6 min()

```
virtual double Statistics_if::min () [pure virtual]
```

min

Returns

Returns the minimum observed value.

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

### 15.936.2.7 newSampleSize()

```
virtual unsigned int Statistics_if::newSampleSize (
 double halfWidth) [pure virtual]
newSampleSize
```

#### Parameters

|                  |  |
|------------------|--|
| <i>halfWidth</i> |  |
|------------------|--|

#### Returns

Estimates the sample size needed to reach a target half-width.  
Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

### 15.936.2.8 numElements()

```
virtual unsigned int Statistics_if::numElements () [pure virtual]
numElements
```

#### Returns

Returns the number of observations considered in calculations.  
Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

### 15.936.2.9 setCollector()

```
virtual void Statistics_if::setCollector (
 Collector_if * collector) [pure virtual]
```

Sets the collector used as source for statistical calculations.  
Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

### 15.936.2.10 setConfidenceLevel()

```
virtual void Statistics_if::setConfidenceLevel (
 double confidencelevel) [pure virtual]
setConfidenceLevel
```

#### Parameters

|                        |  |
|------------------------|--|
| <i>confidencelevel</i> |  |
|------------------------|--|

Sets the confidence level used in inferential calculations.  
Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

### 15.936.2.11 stddeviation()

```
virtual double Statistics_if::stddeviation () [pure virtual]
stddeviation
```

#### Returns

Returns the sample standard deviation.  
Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

### 15.936.2.12 variance()

```
virtual double Statistics_if::variance () [pure virtual]
variance
```

#### Returns

Returns the sample variance of observations.

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

### 15.936.2.13 variationCoef()

```
virtual double Statistics_if::variationCoef () [pure virtual]
variationCoef
```

#### Returns

Returns the coefficient of variation (standard deviation / mean).

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

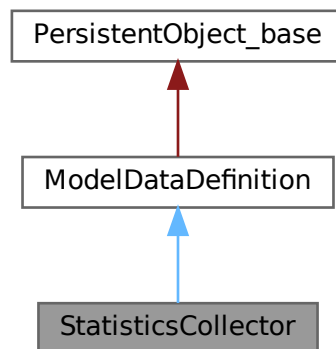
The documentation for this class was generated from the following file:

- kernel/statistics/Statistics\_if.h

## 15.937 StatisticsCollector Class Reference

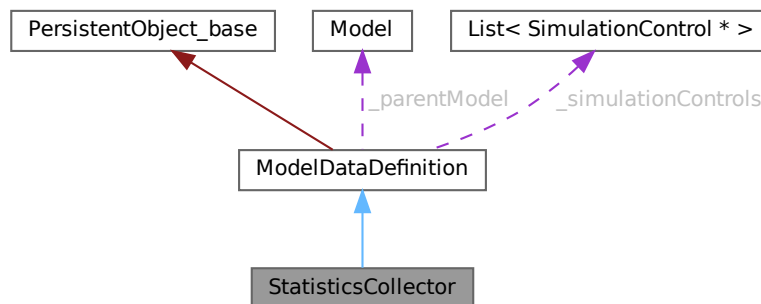
```
#include <kernel/simulator/essentialPlugins/StatisticsCollector.h>
```

Inheritance diagram for StatisticsCollector:





Collaboration diagram for StatisticsCollector:



### Public Member Functions

- **StatisticsCollector** ([Model](#) \*model, std::string name="", [ModelDataDefinition](#) \*parent=nullptr, bool insert↔ IntoModel=true)
- virtual std::string [show](#) () override  
*Returns a textual representation of this data definition.*
- [ModelDataDefinition](#) \* [getParent](#) () const  
*getParent*
- [Statistics\\_if](#) \* [getStatistics](#) () const  
*getStatistics*

### Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) \*model, std::string datadefinitionTypename, std::string name="", bool insert↔ IntoModel=true)  
*Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()  
*Releases resources owned by this data definition.*
- Util::identification [getId](#) () const  
*Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)  
*Updates the logical name of this instance.*
- const std::string & [getName](#) () const  
*Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)  
*Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const  
*Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const  
*Returns the runtime class name used by the plugin system.*
- [Model](#) \* [getParentModel](#) () const  
*Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const  
*Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)

- Enables or disables report-statistics generation for this definition.*
- **ModelDataDefinition \* getInternalData** (const std::string &name) const  
*Looks up a named internal child data definition.*
- std::map< std::string, **ModelDataDefinition \* > \* getInternalData** () const  
*Returns the full map of internal child data definitions.*
- std::map< std::string, **ModelDataDefinition \* > \* getAttachedData** () const  
*Returns the full map of attached data definitions.*
- bool **hasChanged** () const  
*Indicates whether this instance has been modified since the last reset.*
- void **setHasChanged** (bool hasChanged)  
*Updates the changed flag for this data definition.*
- unsigned int **getLevel** () const  
*Returns the level in the model hierarchy where this definition lives.*
- void **setModelLevel** (unsigned int \_modelLevel)  
*Sets the model nesting level for this definition.*
- **List< SimulationControl \* > \* getSimulationControls** () const  
*Returns the simulation controls owned by this data definition.*
- **TraceManager::Level getTraceLevelSpecific** () const  
*Returns the trace level that applies specifically to this definition.*
- void **setTraceLevelSpecific** (**TraceManager::Level** level)  
*Updates the trace level used by this definition.*
- void **defineTraceLevelSpecific** (**TraceManager::Level** traceLevelSpecific, bool traceLevelSpecific←  
Enabled=true)  
*Defines the specific trace level and optionally enables it immediately.*
- bool **isTraceLevelSpecificEnabled** () const  
*Indicates whether the specific trace level is enabled.*
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)  
*Enables or disables the specific trace level.*

### Static Public Member Functions

- static **PluginInformation \* GetPluginInformation** ()
- static **ModelDataDefinition \* LoadInstance** (**Model** \*model, **PersistenceRecord** \*fields)
- static **ModelDataDefinition \* NewInstance** (**Model** \*model, std::string name="")

### Static Public Member Functions inherited from **ModelDataDefinition**

- static **ModelDataDefinition \* LoadInstance** (**Model** \*model, **PersistenceRecord** \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static **ModelDataDefinition \* NewInstance** (**Model** \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void **SaveInstance** (**PersistenceRecord** \*fields, **ModelDataDefinition** \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool **Check** (**ModelDataDefinition** \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void **InitBetweenReplications** (**ModelDataDefinition** \*modeldatum)  
*Resets simulation-time state between replications.*
- static void **CreateInternalData** (**ModelDataDefinition** \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool **CreateRelatedDataElements** (**ModelDataDefinition** \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

### Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) \*fields) override  
*must be overridden by derived classes*
- virtual void `_saveInstance` ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*
- virtual bool `_check` (std::string &errorMessage) override  
*could be overridden by derived classes*

### Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) \* `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) \*control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()  
*legacy helper methods used by the granular create\* hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) \*child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)  
*method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void `trace` (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))

*not just an easy access to trace manager, but wrappers to check if specific trace level applies*

- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- void [traceSimulation](#) (void \*thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)

*Traces text to the simulation output.*

- void **traceSimulation** (void \*thisobject, TraceManager::Level level, const std::string &text)

## Additional Inherited Members

## Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **\_id**
- std::string **\_typename**
- bool **\_reportStatistics**
- bool **\_hasChanged**
- unsigned int **\_modelLevel** = 0
- [Model](#) \* **\_parentModel**
- [List](#)< [SimulationControl](#) \* > \* **\_simulationControls** = new [List](#)<[SimulationControl](#)\*>()
- TraceManager::Level **\_traceLevelSpecific**
- bool **\_traceLevelSpecificEnabled** = false

## 15.937.1 Detailed Description

The [StatisticsCollector](#) is the [ModelDataDefinition](#) responsible for collecting data from the model (using the Collector) and simultaneously keeping statistics updated (using the Statistics)

## 15.937.2 Member Function Documentation

### 15.937.2.1 [\\_check\(\)](#)

```
bool StatisticsCollector::_check (
 std::string & errorMessage) [override], [protected], [virtual]
```

could be overridden by derived classes  
Reimplemented from [ModelDataDefinition](#).

### 15.937.2.2 [\\_loadInstance\(\)](#)

```
bool StatisticsCollector::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

must be overridden by derived classes  
Reimplemented from [ModelDataDefinition](#).

### 15.937.2.3 [\\_saveInstance\(\)](#)

```
void StatisticsCollector::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

#### Parameters

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [ModelDataDefinition](#).

#### 15.937.2.4 getParent()

```
ModelDataDefinition * StatisticsCollector::getParent () const
getParent
```

Returns

#### 15.937.2.5 getStatistics()

```
Statistics_if * StatisticsCollector::getStatistics () const
getStatistics
```

Returns

#### 15.937.2.6 show()

```
std::string StatisticsCollector::show () [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

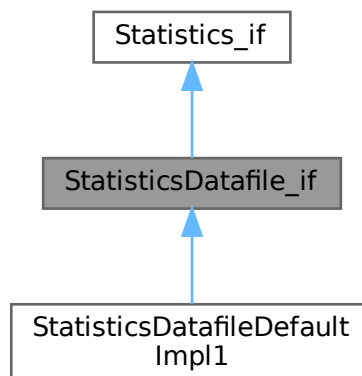
- kernel/simulator/essentialPlugins/StatisticsCollector.h
- kernel/simulator/essentialPlugins/StatisticsCollector.cpp

## 15.938 StatisticsDatafile\_if Class Reference

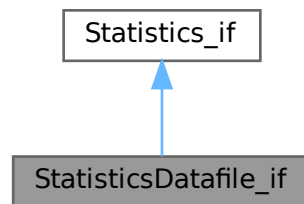
Extension of [Statistics\\_if](#) for metrics that require full data access.

```
#include <kernel/statistics/StatisticsDataFile_if.h>
```

Inheritance diagram for StatisticsDatafile\_if:



Collaboration diagram for StatisticsDatafile\_if:



### Public Member Functions

- virtual double `mode` ()=0  
*mode*
- virtual double `mediane` ()=0  
*mediane*
- virtual double `quartil` (unsigned short num)=0  
*quartil*
- virtual double `decil` (unsigned short num)=0  
*decil*
- virtual double `centil` (unsigned short num)=0  
*centil*
- virtual void `setHistogramNumClasses` (unsigned short num)=0  
*setHistogramNumClasses*
- virtual unsigned short `histogramNumClasses` ()=0  
*histogramNumClasses*
- virtual double `histogramClassLowerLimit` (unsigned short classNum)=0  
*histogramClassLowerLimit*
- virtual unsigned int `histogramClassFrequency` (unsigned short classNum)=0  
*histogramClassFrequency*

### Public Member Functions inherited from `Statistics_if`

- virtual `Collector_if` \* `getCollector` () const =0  
*Returns the data collector associated with these statistics.*
- virtual void `setCollector` (`Collector_if` \*collector)=0  
*Sets the collector used as source for statistical calculations.*
- virtual unsigned int `numElements` ()=0  
*numElements*
- virtual double `min` ()=0  
*min*
- virtual double `max` ()=0  
*max*
- virtual double `average` ()=0  
*average*
- virtual double `variance` ()=0  
*variance*

- virtual double `stddeviation` ()=0  
*stddeviation*
- virtual double `variationCoef` ()=0  
*variationCoef*
- virtual double `halfWidthConfidenceInterval` ()=0  
*halfWidthConfidenceInterval*
- virtual double `confidenceLevel` ()=0  
*confidenceLevel*
- virtual unsigned int `newSampleSize` (double halfWidth)=0  
*newSampleSize*
- virtual void `setConfidenceLevel` (double confidencelevel)=0  
*setConfidenceLevel*

### 15.938.1 Detailed Description

Extension of [Statistics\\_if](#) for metrics that require full data access.

This interface targets collector implementations backed by data files, enabling order-based measures (median, quantiles) and histogram-related operations.

### 15.938.2 Member Function Documentation

#### 15.938.2.1 centil()

```
virtual double StatisticsDatafile_if::centil (
 unsigned short num) [pure virtual]
```

centil

Parameters

|            |  |
|------------|--|
| <i>num</i> |  |
|------------|--|

Returns

Returns the requested percentile (1 to 99).

Implemented in [StatisticsDatafileDefaultImpl1](#).

#### 15.938.2.2 decil()

```
virtual double StatisticsDatafile_if::decil (
 unsigned short num) [pure virtual]
```

decil

Parameters

|            |  |
|------------|--|
| <i>num</i> |  |
|------------|--|

Returns

Returns the requested decile (1 to 9).

Implemented in [StatisticsDatafileDefaultImpl1](#).

#### 15.938.2.3 histogramClassFrequency()

```
virtual unsigned int StatisticsDatafile_if::histogramClassFrequency (
 unsigned short classNum) [pure virtual]
```

histogramClassFrequency

#### Parameters

|                 |  |
|-----------------|--|
| <i>classNum</i> |  |
|-----------------|--|

#### Returns

Returns the absolute frequency of the requested histogram class.  
Implemented in [StatisticsDatafileDefaultImpl1](#).

#### 15.938.2.4 histogramClassLowerLimit()

```
virtual double StatisticsDatafile_if::histogramClassLowerLimit (
 unsigned short classNum) [pure virtual]
```

histogramClassLowerLimit

#### Parameters

|                 |  |
|-----------------|--|
| <i>classNum</i> |  |
|-----------------|--|

#### Returns

Returns the lower bound of the histogram class at index *classNum*.  
Implemented in [StatisticsDatafileDefaultImpl1](#).

#### 15.938.2.5 histogramNumClasses()

```
virtual unsigned short StatisticsDatafile_if::histogramNumClasses () [pure virtual]
```

histogramNumClasses

#### Returns

Returns the configured number of histogram classes.  
Implemented in [StatisticsDatafileDefaultImpl1](#).

#### 15.938.2.6 mediane()

```
virtual double StatisticsDatafile_if::mediane () [pure virtual]
```

mediane

#### Returns

Returns the sample median.  
Implemented in [StatisticsDatafileDefaultImpl1](#).

#### 15.938.2.7 mode()

```
virtual double StatisticsDatafile_if::mode () [pure virtual]
```

mode

#### Returns

Returns the mode of the sample stored in file.  
Implemented in [StatisticsDatafileDefaultImpl1](#).



**15.938.2.8** `quartil()`

```
virtual double StatisticsDatafile_if::quartil (
 unsigned short num) [pure virtual]
```

`quartil`**Parameters**

|                  |  |
|------------------|--|
| <code>num</code> |  |
|------------------|--|

**Returns**

Returns the requested quartile (1 to 3).  
Implemented in [StatisticsDatafileDefaultImpl1](#).

**15.938.2.9** `setHistogramNumClasses()`

```
virtual void StatisticsDatafile_if::setHistogramNumClasses (
 unsigned short num) [pure virtual]
```

`setHistogramNumClasses`**Parameters**

|                  |  |
|------------------|--|
| <code>num</code> |  |
|------------------|--|

Sets the number of classes used in the histogram.  
Implemented in [StatisticsDatafileDefaultImpl1](#).  
The documentation for this class was generated from the following file:

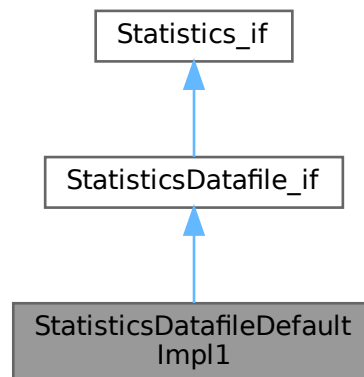
- `kernel/statistics/StatisticsDataFile_if.h`

**15.939 StatisticsDatafileDefaultImpl1 Class Reference**

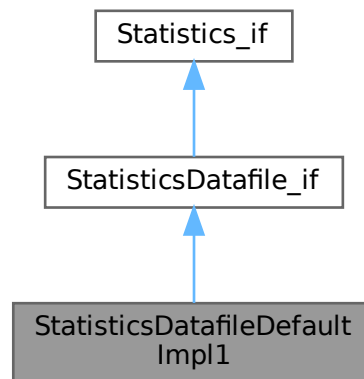
Default implementation of [StatisticsDatafile\\_if](#) for file-backed samples.

```
#include <kernel/statistics/StatisticsDataFileDefaultImpl.h>
```

Inheritance diagram for StatisticsDatafileDefaultImpl1:



Collaboration diagram for StatisticsDatafileDefaultImpl1:



## Public Member Functions

- **StatisticsDatafileDefaultImpl1 ()**  
*Creates statistics object with default collector implementation.*
- virtual **~StatisticsDatafileDefaultImpl1 ()**  
*Releases owned collector/sorter resources when configured as owner.*
- virtual **Collector\_if \* getCollector ()** const override  
*Returns the collector currently used as data source.*
- virtual void **setCollector (Collector\_if \*collector)** override  
*Sets a collector to be used as data source for calculations.*
- virtual unsigned int **numElements ()** override  
*Returns number of observations available in the collector.*
- virtual double **min ()** override  
*Returns minimum observed value.*
- virtual double **max ()** override  
*Returns maximum observed value.*
- virtual double **average ()** override  
*Returns arithmetic mean.*
- virtual double **variance ()** override  
*Returns sample variance.*
- virtual double **stddeviation ()** override  
*Returns sample standard deviation.*
- virtual double **variationCoef ()** override  
*Returns coefficient of variation.*
- virtual double **halfWidthConfidenceInterval ()** override  
*Returns current confidence interval half-width.*
- virtual double **confidenceLevel ()** override  
*Returns configured confidence level.*
- virtual unsigned int **newSampleSize (double halfWidth)** override  
*Estimates sample size needed for a desired half-width.*
- virtual void **setConfidenceLevel (double confidencelevel)** override

- Sets confidence level used in inferential calculations.*
- virtual double [mode](#) () override  
*Returns sample mode.*
- virtual double [mediane](#) () override  
*Returns sample median.*
- virtual double [quartil](#) (unsigned short num) override  
*Returns requested quartile.*
- virtual double [decil](#) (unsigned short num) override  
*Returns requested decile.*
- virtual double [centil](#) (unsigned short num) override  
*Returns requested percentile.*
- virtual void [setHistogramNumClasses](#) (unsigned short num) override  
*Sets number of histogram classes.*
- virtual unsigned short [histogramNumClasses](#) () override  
*Returns current number of histogram classes.*
- virtual double [histogramClassLowerLimit](#) (unsigned short classNum) override  
*Returns lower limit of histogram class `classNum`.*
- virtual unsigned int [histogramClassFrequency](#) (unsigned short classNum) override  
*Returns absolute frequency of histogram class `classNum`.*

### 15.939.1 Detailed Description

Default implementation of [StatisticsDatafile\\_if](#) for file-backed samples.

Computes descriptive/inferential statistics from a [CollectorDatafile\\_if](#) and caches results to avoid recomputation until source data changes.

### 15.939.2 Member Function Documentation

#### 15.939.2.1 [average\(\)](#)

```
double StatisticsDatafileDefaultImpl1::average () [override], [virtual]
```

Returns arithmetic mean.

Implements [Statistics\\_if](#).

#### 15.939.2.2 [centil\(\)](#)

```
double StatisticsDatafileDefaultImpl1::centil (
 unsigned short num) [override], [virtual]
```

Returns requested percentile.

Implements [StatisticsDatafile\\_if](#).

#### 15.939.2.3 [confidenceLevel\(\)](#)

```
double StatisticsDatafileDefaultImpl1::confidenceLevel () [override], [virtual]
```

Returns configured confidence level.

Implements [Statistics\\_if](#).

#### 15.939.2.4 [decil\(\)](#)

```
double StatisticsDatafileDefaultImpl1::decil (
 unsigned short num) [override], [virtual]
```

Returns requested decile.

Implements [StatisticsDatafile\\_if](#).

#### 15.939.2.5 getCollector()

```
Collector_if * StatisticsDatafileDefaultImpl1::getCollector () const [override], [virtual]
```

Returns the collector currently used as data source.

Implements [Statistics\\_if](#).

#### 15.939.2.6 halfWidthConfidenceInterval()

```
double StatisticsDatafileDefaultImpl1::halfWidthConfidenceInterval () [override], [virtual]
```

Returns current confidence interval half-width.

Implements [Statistics\\_if](#).

#### 15.939.2.7 histogramClassFrequency()

```
unsigned int StatisticsDatafileDefaultImpl1::histogramClassFrequency (
 unsigned short classNum) [override], [virtual]
```

Returns absolute frequency of histogram class `classNum`.

Implements [StatisticsDatafile\\_if](#).

#### 15.939.2.8 histogramClassLowerLimit()

```
double StatisticsDatafileDefaultImpl1::histogramClassLowerLimit (
 unsigned short classNum) [override], [virtual]
```

Returns lower limit of histogram class `classNum`.

Implements [StatisticsDatafile\\_if](#).

#### 15.939.2.9 histogramNumClasses()

```
unsigned short StatisticsDatafileDefaultImpl1::histogramNumClasses () [override], [virtual]
```

Returns current number of histogram classes.

Implements [StatisticsDatafile\\_if](#).

#### 15.939.2.10 max()

```
double StatisticsDatafileDefaultImpl1::max () [override], [virtual]
```

Returns maximum observed value.

Implements [Statistics\\_if](#).

#### 15.939.2.11 mediane()

```
double StatisticsDatafileDefaultImpl1::mediane () [override], [virtual]
```

Returns sample median.

Implements [StatisticsDatafile\\_if](#).

#### 15.939.2.12 min()

```
double StatisticsDatafileDefaultImpl1::min () [override], [virtual]
```

Returns minimum observed value.

Implements [Statistics\\_if](#).

#### 15.939.2.13 mode()

```
double StatisticsDatafileDefaultImpl1::mode () [override], [virtual]
```

Returns sample mode.

Implements [StatisticsDatafile\\_if](#).

**15.939.2.14 newSampleSize()**

```
unsigned int StatisticsDatafileDefaultImpl1::newSampleSize (
 double halfWidth) [override], [virtual]
```

Estimates sample size needed for a desired half-width.

Implements [Statistics\\_if](#).

**15.939.2.15 numElements()**

```
unsigned int StatisticsDatafileDefaultImpl1::numElements () [override], [virtual]
```

Returns number of observations available in the collector.

Implements [Statistics\\_if](#).

**15.939.2.16 quartil()**

```
double StatisticsDatafileDefaultImpl1::quartil (
 unsigned short num) [override], [virtual]
```

Returns requested quartile.

Implements [StatisticsDatafile\\_if](#).

**15.939.2.17 setCollector()**

```
void StatisticsDatafileDefaultImpl1::setCollector (
 Collector_if * collector) [override], [virtual]
```

Sets a collector to be used as data source for calculations.

Implements [Statistics\\_if](#).

**15.939.2.18 setConfidenceLevel()**

```
void StatisticsDatafileDefaultImpl1::setConfidenceLevel (
 double confidencelevel) [override], [virtual]
```

Sets confidence level used in inferential calculations.

Implements [Statistics\\_if](#).

**15.939.2.19 setHistogramNumClasses()**

```
void StatisticsDatafileDefaultImpl1::setHistogramNumClasses (
 unsigned short num) [override], [virtual]
```

Sets number of histogram classes.

Implements [StatisticsDatafile\\_if](#).

**15.939.2.20 stddeviation()**

```
double StatisticsDatafileDefaultImpl1::stddeviation () [override], [virtual]
```

Returns sample standard deviation.

Implements [Statistics\\_if](#).

**15.939.2.21 variance()**

```
double StatisticsDatafileDefaultImpl1::variance () [override], [virtual]
```

Returns sample variance.

Implements [Statistics\\_if](#).

**15.939.2.22 variationCoef()**

```
double StatisticsDatafileDefaultImpl1::variationCoef () [override], [virtual]
```

Returns coefficient of variation.

Implements [Statistics\\_if](#).

The documentation for this class was generated from the following files:

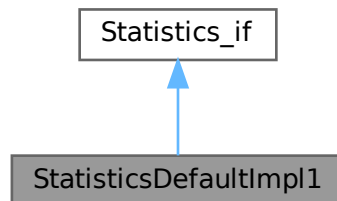
- kernel/statistics/StatisticsDataFileDefaultImpl.h
- kernel/statistics/StatisticsDataFileDefaultImpl.cpp

## 15.940 StatisticsDefaultImpl1 Class Reference

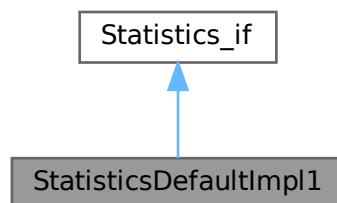
Default online statistics implementation over a [Collector\\_if](#) source.

```
#include <kernel/statistics/StatisticsDefaultImpl1.h>
```

Inheritance diagram for StatisticsDefaultImpl1:



Collaboration diagram for StatisticsDefaultImpl1:



### Public Member Functions

- [StatisticsDefaultImpl1](#) ()  
*Creates statistics object using configured default collector implementation.*
- [StatisticsDefaultImpl1](#) ([Collector\\_if](#) \*collector)  
*Creates statistics object bound to an external collector.*
- virtual [Collector\\_if](#) \* [getCollector](#) () const override  
*Returns current collector data source.*
- virtual void [setCollector](#) ([Collector\\_if](#) \*collector) override  
*Sets collector source and reconnects internal callbacks.*
- virtual unsigned int [numElements](#) () override  
*Returns number of observed samples.*
- virtual double [min](#) () override  
*Returns minimum observed value.*
- virtual double [max](#) () override

- Returns maximum observed value.*
- virtual double [average](#) () override
- Returns arithmetic mean.*
- virtual double [variance](#) () override
- Returns sample variance.*
- virtual double [stddeviation](#) () override
- Returns sample standard deviation.*
- virtual double [variationCoef](#) () override
- Returns coefficient of variation.*
- virtual double [halfWidthConfidenceInterval](#) () override
- Returns confidence interval half-width.*
- virtual double [confidenceLevel](#) () override
- Returns configured confidence level.*
- virtual unsigned int [newSampleSize](#) (double halfWidth) override
- Estimates sample size required for desired half-width.*
- virtual void [setConfidenceLevel](#) (double confidencelevel) override
- Sets confidence level used in inferential formulas.*

### 15.940.1 Detailed Description

Default online statistics implementation over a [Collector\\_if](#) source.  
Updates core metrics incrementally through collector callbacks, avoiding complete rescans for every query.

### 15.940.2 Constructor & Destructor Documentation

#### 15.940.2.1 StatisticsDefaultImpl1()

`StatisticsDefaultImpl1::StatisticsDefaultImpl1 ( )`  
Creates statistics object using configured default collector implementation.  
When constructor is invoked without a `Collector`, it is taken from `Traits<Statistics_if>::CollectorImplementation` configuration

### 15.940.3 Member Function Documentation

#### 15.940.3.1 average()

`double StatisticsDefaultImpl1::average ( ) [override], [virtual]`  
Returns arithmetic mean.  
Implements [Statistics\\_if](#).

#### 15.940.3.2 confidenceLevel()

`double StatisticsDefaultImpl1::confidenceLevel ( ) [override], [virtual]`  
Returns configured confidence level.  
Implements [Statistics\\_if](#).

#### 15.940.3.3 getCollector()

`Collector_if * StatisticsDefaultImpl1::getCollector ( ) const [override], [virtual]`  
Returns current collector data source.  
Implements [Statistics\\_if](#).

#### 15.940.3.4 halfWidthConfidenceInterval()

`double StatisticsDefaultImpl1::halfWidthConfidenceInterval ( ) [override], [virtual]`  
Returns confidence interval half-width.  
Implements [Statistics\\_if](#).

#### 15.940.3.5 max()

```
double StatisticsDefaultImpl1::max () [override], [virtual]
```

Returns maximum observed value.

Implements [Statistics\\_if](#).

#### 15.940.3.6 min()

```
double StatisticsDefaultImpl1::min () [override], [virtual]
```

Returns minimum observed value.

Implements [Statistics\\_if](#).

#### 15.940.3.7 newSampleSize()

```
unsigned int StatisticsDefaultImpl1::newSampleSize (
 double halfWidth) [override], [virtual]
```

Estimates sample size required for desired half-width.

Implements [Statistics\\_if](#).

#### 15.940.3.8 numElements()

```
unsigned int StatisticsDefaultImpl1::numElements () [override], [virtual]
```

Returns number of observed samples.

Implements [Statistics\\_if](#).

#### 15.940.3.9 setCollector()

```
void StatisticsDefaultImpl1::setCollector (
 Collector_if * collector) [override], [virtual]
```

Sets collector source and reconnects internal callbacks.

Implements [Statistics\\_if](#).

#### 15.940.3.10 setConfidenceLevel()

```
void StatisticsDefaultImpl1::setConfidenceLevel (
 double confidencelevel) [override], [virtual]
```

Sets confidence level used in inferential formulas.

Implements [Statistics\\_if](#).

#### 15.940.3.11 stddeviation()

```
double StatisticsDefaultImpl1::stddeviation () [override], [virtual]
```

Returns sample standard deviation.

Implements [Statistics\\_if](#).

#### 15.940.3.12 variance()

```
double StatisticsDefaultImpl1::variance () [override], [virtual]
```

Returns sample variance.

Implements [Statistics\\_if](#).

#### 15.940.3.13 variationCoef()

```
double StatisticsDefaultImpl1::variationCoef () [override], [virtual]
```

Returns coefficient of variation.

Implements [Statistics\\_if](#).

The documentation for this class was generated from the following files:

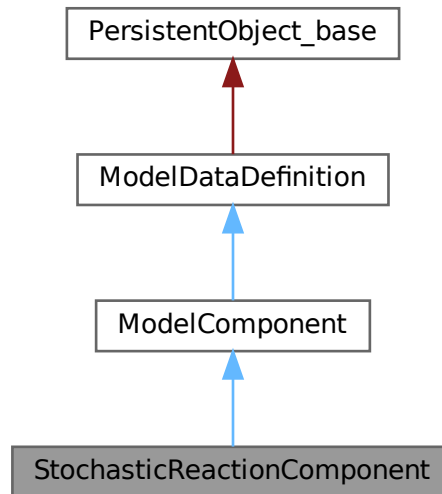
- kernel/statistics/StatisticsDefaultImpl1.h
- kernel/statistics/StatisticsDefaultImpl1.cpp



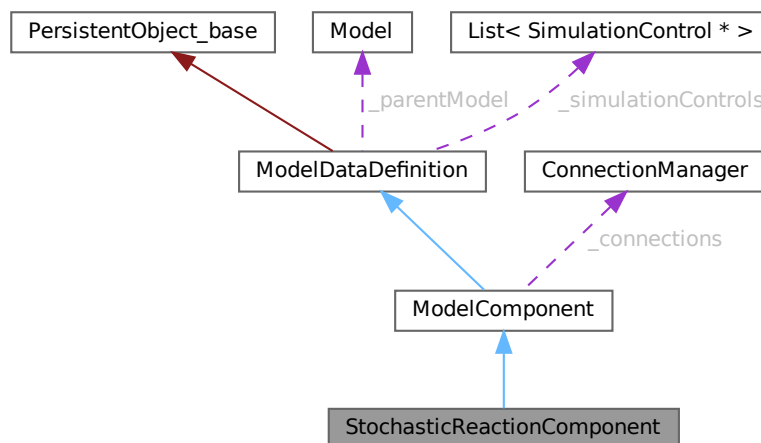
## 15.941 StochasticReactionComponent Class Reference

```
#include <plugins/components/WholeCellModeling/StochasticReactionComponent.h>
```

Inheritance diagram for StochasticReactionComponent:



Collaboration diagram for StochasticReactionComponent:



### Public Member Functions

- **StochasticReactionComponent** (`Model` \*model, std::string name="")
- void **setWholeCellState** (`WholeCellState` \*state)
- `WholeCellState` \* **getWholeCellState** () const

- void **setTimeWindow** (double timeWindow)
- double **getTimeWindow** () const
- void **setAdvanceWholeCellClock** (bool advance)
- bool **getAdvanceWholeCellClock** () const
- void **setRandomSeed** (unsigned int seed)
- unsigned int **getRandomSeed** () const
- int **getLastReactionCount** () const
- double **getLastSimulatedTime** () const
- virtual std::string **show** () override

*virtual public methods*

## Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** \*model, std::string componentName, std::string name="")  
*constructors*
- virtual ~**ModelComponent** ()  
*Releases component-owned resources and unregisters it from the model.*
- **ConnectionManager** \* **getConnectionManager** () const  
*new public user methods for this component*
- void **connectTo** (**ModelComponent** \*component, unsigned int inputPortNumber=0)  
*Connects this component to another component.*
- bool **hasBreakpointAt** ()  
*Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string \_description)  
*Updates the free-form description shown by the GUI.*
- std::string **getDescription** () const  
*Returns the free-form description associated with this component.*
- void **remove** (**ModelComponent** \*comp)  
*Removes a connected component from this component.*

## Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** \*model, std::string datadefinitionTypename, std::string name="", bool insert↵  
IntoModel=true)  
*Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()  
*Releases resources owned by this data definition.*
- Util::identification **getId** () const  
*Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)  
*Updates the logical name of this instance.*
- const std::string & **getName** () const  
*Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)  
*Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const  
*Returns the user-visible label of this instance.*
- const std::string & **getClassname** () const  
*Returns the runtime class name used by the plugin system.*
- **Model** \* **getParentModel** () const  
*Returns the parent model that owns this definition.*
- bool **isReportStatistics** () const

- Indicates whether this data definition contributes report statistics.*
  - void [setReportStatistics](#) (bool reportStatistics)
  - Enables or disables report-statistics generation for this definition.*
  - [ModelDataDefinition](#) \* [getInternalData](#) (const std::string &name) const
  - Looks up a named internal child data definition.*
  - std::map< std::string, [ModelDataDefinition](#) \* > \* [getInternalData](#) () const
  - Returns the full map of internal child data definitions.*
  - std::map< std::string, [ModelDataDefinition](#) \* > \* [getAttachedData](#) () const
  - Returns the full map of attached data definitions.*
  - bool [hasChanged](#) () const
  - Indicates whether this instance has been modified since the last reset.*
  - void [setHasChanged](#) (bool hasChanged)
  - Updates the changed flag for this data definition.*
  - unsigned int [getLevel](#) () const
  - Returns the level in the model hierarchy where this definition lives.*
  - void [setModelLevel](#) (unsigned int \_modelLevel)
  - Sets the model nesting level for this definition.*
  - List< [SimulationControl](#) \* > \* [getSimulationControls](#) () const
  - Returns the simulation controls owned by this data definition.*
  - [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
  - Returns the trace level that applies specifically to this definition.*
  - void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
  - Updates the trace level used by this definition.*
  - void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
  - Defines the specific trace level and optionally enables it immediately.*
  - bool [isTraceLevelSpecificEnabled](#) () const
  - Indicates whether the specific trace level is enabled.*
  - void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
  - Enables or disables the specific trace level.*

### Static Public Member Functions

- static [PluginInformation](#) \* [GetPluginInformation](#) ()
- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")

### Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool [Check](#) ([ModelComponent](#) \*component)
- Validates a component instance.*
- static void [CreateInternalData](#) ([ModelComponent](#) \*component)
- Creates internal data required by a component instance.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelComponent](#) \*component)
- Serializes a component instance into a persistence record.*
- static void [DispatchEvent](#) ([Event](#) \*event)
- Dispatches a simulation event to the target component.*

## Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool [Check](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*
- static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

## Protected Member Functions

- virtual bool [\\_loadInstance](#) ([PersistenceRecord](#) \*fields) override  
*virtual protected method that must be overridden*
- virtual void [\\_saveInstance](#) ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*
- virtual bool [\\_check](#) (std::string &errorMessage) override  
*could be overridden by derived classes*
- virtual void [\\_initBetweenReplications](#) () override
- virtual void [\\_onDispatchEvent](#) ([Entity](#) \*entity, unsigned int inputPortNumber) override  
*Handles a dispatched event for this component type.*
- virtual void [\\_createEditableDataDefinitions](#) () override

## Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [\\_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) \* [\\_getParserChangesInformation](#) ()
- virtual void [\\_addSimulationControl](#) ([SimulationControl](#) \*control)
- virtual void [\\_createInternalAndAttachedData](#) ()
- virtual void [\\_createInternalStatisticReporters](#) ()
- virtual void [\\_createAttachedAttributes](#) ()
- virtual void [\\_createNonEditableDataDefinitions](#) ()
- void [\\_templateCreateInternalStatisticReporters](#) ()
- void [\\_templateCreateAttachedAttributes](#) ()
- void [\\_templateCreateNonEditableDataDefinitions](#) ()
- void [\\_templateCreateEditableDataDefinitions](#) ()
- void [\\_internalStatisticReportersClear](#) ()  
*legacy helper methods used by the granular create\* hooks*
- void [\\_internalDataClear](#) ()
- void [\\_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*child)
- void [\\_internalDataRemove](#) (const std::string &key)
- void [\\_attachedDataClear](#) ()
- void [\\_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_attachedDataRemove](#) (const std::string &key)
- void [\\_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [\\_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_statisticReporterRemove](#) (const std::string &key)

- void **\_statisticReportersClear** ()
- void **\_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **\_mandatoryAttachedAttributesClear** ()
- void **\_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryEditableDataDefinitionsClear** ()
- void **\_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalEditableDataDefinitionsClear** ()
- void **\_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryNonEditableDataDefinitionsClear** ()
- void **\_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalNonEditableDataDefinitionsClear** ()
- void **\_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
 

*method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
 

*not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- void **traceSimulation** (void \*thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
 

*Traces text to the simulation output.*
- void **traceSimulation** (void \*thisobject, TraceManager::Level level, const std::string &text)

### Additional Inherited Members

### Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) \* **\_connections** = new [ConnectionManager](#)()
 

*virtual protected methods that could be overridden by derived classes, if needed*

### Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **\_id**
- std::string **\_typename**
- bool **\_reportStatistics**
- bool **\_hasChanged**
- unsigned int **\_modelLevel** = 0
- [Model](#) \* **\_parentModel**
- [List](#)< [SimulationControl](#) \* > \* **\_simulationControls** = new [List](#)<[SimulationControl](#)\*>()
- TraceManager::Level **\_traceLevelSpecific**
- bool **\_traceLevelSpecificEnabled** = false

### 15.941.1 Detailed Description

Gillespie Direct Method SSA runner for whole-cell stochastic simulations.

On each entity arrival the component runs the Direct Method (Gillespie 1977) for a configurable time window [0, timeWindow] using all [StochasticReactionRule](#) definitions registered in the model. Species counts are read from and written back to the referenced [WholeCellState](#) (if set) or directly to [MolecularSpecies](#) objects in the model.

Contract (Q7b): Input: current molecule counts from [WholeCellState](#) (or [MolecularSpecies](#))

- resource budget from [WholeCellState](#) Output: updated counts written back to [WholeCellState](#); entity forwarded

[Resource](#) cap: reactions whose firing would deplete a species below its resource budget floor are skipped, matching the fair-allocation contract of the WholeCell M. genitalium model (Karr et al. 2012).

### 15.941.2 Member Function Documentation

#### 15.941.2.1 `_check()`

```
bool StochasticReactionComponent::_check (
 std::string & errorMessage) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

#### 15.941.2.2 `_createEditableDataDefinitions()`

```
void StochasticReactionComponent::_createEditableDataDefinitions () [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

#### 15.941.2.3 `_initBetweenReplications()`

```
void StochasticReactionComponent::_initBetweenReplications () [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

#### 15.941.2.4 `_loadInstance()`

```
bool StochasticReactionComponent::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

#### 15.941.2.5 `_onDispatchEvent()`

```
void StochasticReactionComponent::_onDispatchEvent (
 Entity * entity,
 unsigned int inputPortNumber) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

#### Parameters

|                        |                                         |
|------------------------|-----------------------------------------|
| <i>entity</i>          | <a href="#">Entity</a> being processed. |
| <i>inputPortNumber</i> | Input port where the entity arrived.    |

Implements [ModelComponent](#).

#### 15.941.2.6 `_saveInstance()`

```
void StochasticReactionComponent::_saveInstance (
 PersistenceRecord * fields,
```

```
bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

#### Parameters

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [ModelComponent](#).

#### 15.941.2.7 show()

```
std::string StochasticReactionComponent::show () [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

#### Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

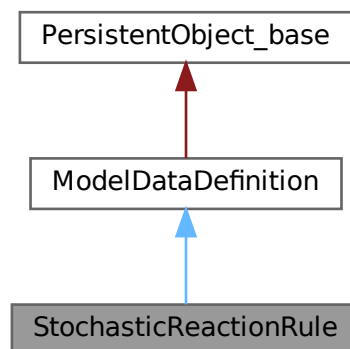
The documentation for this class was generated from the following files:

- plugins/components/WholeCellModeling/StochasticReactionComponent.h
- plugins/components/WholeCellModeling/StochasticReactionComponent.cpp

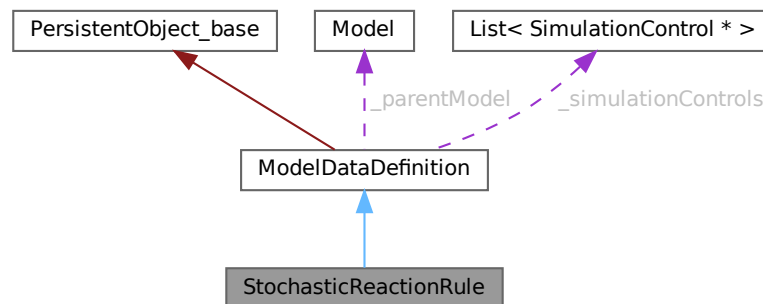
## 15.942 StochasticReactionRule Class Reference

```
#include <plugins/data/WholeCellModeling/StochasticReactionRule.h>
```

Inheritance diagram for StochasticReactionRule:



Collaboration diagram for StochasticReactionRule:



## Classes

- struct [StoichiometricTerm](#)

## Public Member Functions

- **StochasticReactionRule** ([Model](#) \*model, std::string name="")
- void **addReactant** (std::string speciesName, int stoichiometry=1)
- void **addProduct** (std::string speciesName, int stoichiometry=1)
- void **clearReactants** ()
- void **clearProducts** ()
- const std::vector< [StoichiometricTerm](#) > & **getReactants** () const
- const std::vector< [StoichiometricTerm](#) > & **getProducts** () const
- void **setRateConstant** (double rateConstant)
- double **getRateConstant** () const
- void **setCompartment** (std::string compartment)
- std::string **getCompartment** () const
- double **computePropensity** (const std::map< std::string, int > &counts) const
- virtual std::string **show** () override

*Returns a textual representation of this data definition.*

## Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) \*model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
- *Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- *Releases resources owned by this data definition.*
- Util::identification **getId** () const
- *Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
- *Updates the logical name of this instance.*
- const std::string & **getName** () const
- *Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)
- *Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const



- Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const  
*Returns the runtime class name used by the plugin system.*
- [Model](#) \* [getParentModel](#) () const  
*Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const  
*Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)  
*Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) \* [getInternalData](#) (const std::string &name) const  
*Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getInternalData](#) () const  
*Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getAttachedData](#) () const  
*Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const  
*Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))  
*Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const  
*Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int \_modelLevel)  
*Sets the model nesting level for this definition.*
- [List](#)< [SimulationControl](#) \* > \* [getSimulationControls](#) () const  
*Returns the simulation controls owned by this data definition.*
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const  
*Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)  
*Updates the trace level used by this definition.*
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)  
*Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const  
*Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)  
*Enables or disables the specific trace level.*

### Static Public Member Functions

- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [PluginInformation](#) \* [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")

### Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool [Check](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)

*Validates a data definition instance.*

- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*
- static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

### Protected Member Functions

- virtual bool [\\_loadInstance](#) ([PersistenceRecord](#) \*fields) override  
*must be overridden by derived classes*
- virtual void [\\_saveInstance](#) ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*
- virtual bool [\\_check](#) (std::string &errorMessage) override  
*could be overridden by derived classes*

### Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [\\_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) \* [\\_getParserChangesInformation](#) ()
- virtual void [\\_initBetweenReplications](#) ()
- virtual void [\\_addSimulationControl](#) ([SimulationControl](#) \*control)
- virtual void [\\_createInternalAndAttachedData](#) ()
- virtual void [\\_createInternalStatisticReporters](#) ()
- virtual void [\\_createAttachedAttributes](#) ()
- virtual void [\\_createNonEditableDataDefinitions](#) ()
- virtual void [\\_createEditableDataDefinitions](#) ()
- void [\\_templateCreateInternalStatisticReporters](#) ()
- void [\\_templateCreateAttachedAttributes](#) ()
- void [\\_templateCreateNonEditableDataDefinitions](#) ()
- void [\\_templateCreateEditableDataDefinitions](#) ()
- void [\\_internalStatisticReportersClear](#) ()  
*legacy helper methods used by the granular create\* hooks*
- void [\\_internalDataClear](#) ()
- void [\\_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*child)
- void [\\_internalDataRemove](#) (const std::string &key)
- void [\\_attachedDataClear](#) ()
- void [\\_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_attachedDataRemove](#) (const std::string &key)
- void [\\_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [\\_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_statisticReporterRemove](#) (const std::string &key)
- void [\\_statisticReportersClear](#) ()
- void [\\_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [\\_mandatoryAttachedAttributesClear](#) ()
- void [\\_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_mandatoryEditableDataDefinitionsClear](#) ()
- void [\\_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_optionalEditableDataDefinitionsClear](#) ()
- void [\\_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)

- void **\_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryNonEditableDataDefinitionsClear** ()
- void **\_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalNonEditableDataDefinitionsClear** ()
- void **\_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)  
*method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)  
*not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- void **traceSimulation** (void \*thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)  
*Traces text to the simulation output.*
- void **traceSimulation** (void \*thisobject, TraceManager::Level level, const std::string &text)

### Additional Inherited Members

### Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **\_id**
- std::string **\_typename**
- bool **\_reportStatistics**
- bool **\_hasChanged**
- unsigned int **\_modelLevel** = 0
- [Model](#) \* **\_parentModel**
- [List](#)< [SimulationControl](#) \* > \* **\_simulationControls** = new [List](#)<[SimulationControl](#)\*>()
- TraceManager::Level **\_traceLevelSpecific**
- bool **\_traceLevelSpecificEnabled** = false

## 15.942.1 Detailed Description

Stochastic reaction rule for Gillespie SSA whole-cell simulations.

Reactants and products are named by [MolecularSpecies](#) with integer stoichiometry. The propensity is computed as  $k * \text{combinatorial}(\text{counts})$ , where the combinatorial term is the product of  $C(x_i, n_i)$  for each reactant species  $i$  with stoichiometry  $n_i$ .

The rate constant  $k$  uses stochastic (microscopic) units:  $s^{-1}$  for unimolecular,  $\text{molecules}^{-1} * s^{-1}$  for bimolecular reactions.

## 15.942.2 Member Function Documentation

### 15.942.2.1 **\_check()**

```
bool StochasticReactionRule::_check (
 std::string & errorMessage) [override], [protected], [virtual]
```

could be overridden by derived classes  
 Reimplemented from [ModelDataDefinition](#).

### 15.942.2.2 **\_loadInstance()**

```
bool StochasticReactionRule::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

must be overridden by derived classes  
 Reimplemented from [ModelDataDefinition](#).

**15.942.2.3 \_saveInstance()**

```
void StochasticReactionRule::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

**Parameters**

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [ModelDataDefinition](#).

**15.942.2.4 show()**

```
std::string StochasticReactionRule::show () [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

**Returns**

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

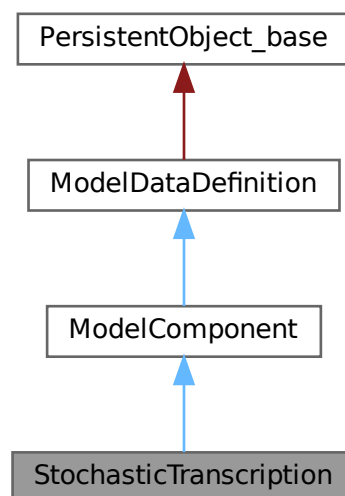
The documentation for this class was generated from the following files:

- plugins/data/WholeCellModeling/StochasticReactionRule.h
- plugins/data/WholeCellModeling/StochasticReactionRule.cpp

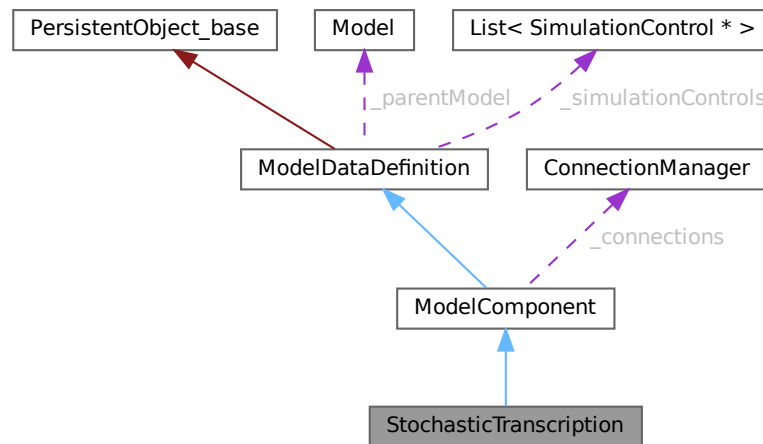
**15.943 StochasticTranscription Class Reference**

```
#include <plugins/components/WholeCellModeling/StochasticTranscription.h>
```

Inheritance diagram for StochasticTranscription:



Collaboration diagram for StochasticTranscription:



### Public Member Functions

- **StochasticTranscription** ([Model](#) \*model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) \*state)
- [WholeCellState](#) \* **getWholeCellState** () const
- void **setElongationRate** (double rate)
- double **getElongationRate** () const
- void **setMeanGeneLength** (double length)
- double **getMeanGeneLength** () const
- void **setBindingProbability** (double prob)
- double **getBindingProbability** () const
- void **setTimeWindow** (double window)
- double **getTimeWindow** () const
- void **setMRNAspeciesPrefix** (std::string prefix)
- std::string **getMRNAspeciesPrefix** () const
- void **setRnapCountKey** (std::string key)
- std::string **getRnapCountKey** () const
- void **setRandomSeed** (unsigned int seed)
- unsigned int **getRandomSeed** () const
- int **getLastSynthesizedCount** () const
- virtual std::string **show** () override

*virtual public methods*

### Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) \*model, std::string componentTypename, std::string name="")  
*constructors*
- virtual ~[ModelComponent](#) ()  
*Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) \* **getConnectionManager** () const  
*new public user methods for this component*
- void **connectTo** ([ModelComponent](#) \*component, unsigned int inputPortNumber=0)  
*Connects this component to another component.*

- bool [hasBreakpointAt](#) ()  
*Indicates whether a simulation breakpoint is set on this component.*
- void [setDescription](#) (std::string \_description)  
*Updates the free-form description shown by the GUI.*
- std::string [getDescription](#) () const  
*Returns the free-form description associated with this component.*
- void [remove](#) (ModelComponent \*comp)  
*Removes a connected component from this component.*

## Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) \*model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)  
*Creates a model data definition and optionally inserts it into the model.*
- virtual ~[ModelDataDefinition](#) ()  
*Releases resources owned by this data definition.*
- Util::identification [getId](#) () const  
*Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)  
*Updates the logical name of this instance.*
- const std::string & [getName](#) () const  
*Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)  
*Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const  
*Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const  
*Returns the runtime class name used by the plugin system.*
- [Model](#) \* [getParentModel](#) () const  
*Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const  
*Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)  
*Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) \* [getInternalData](#) (const std::string &name) const  
*Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getInternalData](#) () const  
*Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getAttachedData](#) () const  
*Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const  
*Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))  
*Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const  
*Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int \_modelLevel)  
*Sets the model nesting level for this definition.*
- List< [SimulationControl](#) \* > \* [getSimulationControls](#) () const  
*Returns the simulation controls owned by this data definition.*
- TraceManager::Level [getTraceLevelSpecific](#) () const

- Returns the trace level that applies specifically to this definition.*

  - void [setTraceLevelSpecific](#) (TraceManager::Level level)

*Updates the trace level used by this definition.*
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelspecific, bool traceLevelSpecific←Enabled=true)

*Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const

*Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)

*Enables or disables the specific trace level.*

### Static Public Member Functions

- static [PluginInformation](#) \* [GetPluginInformation](#) ()
- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")

### Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)

*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool [Check](#) ([ModelComponent](#) \*component)

*Validates a component instance.*
- static void [CreateInternalData](#) ([ModelComponent](#) \*component)

*Creates internal data required by a component instance.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelComponent](#) \*component)

*Serializes a component instance into a persistence record.*
- static void [DispatchEvent](#) ([Event](#) \*event)

*Dispatches a simulation event to the target component.*

### Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)

*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")

*Creates a new instance of a data definition using the plugin registry.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)

*Serializes a data definition instance into a persistence record.*
- static bool [Check](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)

*Validates a data definition instance.*
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) \*modeldatum)

*Resets simulation-time state between replications.*
- static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)

*Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)

*Creates related data and validates the instance in a single call.*

## Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) \*fields) override  
*virtual protected method that must be overridden*
- virtual void `_saveInstance` ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*
- virtual bool `_check` (std::string &errorMessage) override  
*could be overridden by derived classes*
- virtual void `_initBetweenReplications` () override
- virtual void `_onDispatchEvent` ([Entity](#) \*entity, unsigned int inputPortNumber) override  
*Handles a dispatched event for this component type.*
- virtual void `_createEditableDataDefinitions` () override

## Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) \* `_getParserChangesInformation` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) \*control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()  
*legacy helper methods used by the granular create\* hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) \*child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)



method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)  
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- void **traceSimulation** (void \*thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)  
Traces text to the simulation output.
- void **traceSimulation** (void \*thisobject, TraceManager::Level level, const std::string &text)

### Additional Inherited Members

### Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) \* \_connections = new [ConnectionManager](#)()  
virtual protected methods that could be overridden by derived classes, if needed

### Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification\_id
- std::string \_typename
- bool \_reportStatistics
- bool \_hasChanged
- unsigned int \_modelLevel = 0
- [Model](#) \* \_parentModel
- [List](#)< [SimulationControl](#) \* > \* \_simulationControls = new [List](#)<[SimulationControl](#)\*>()
- TraceManager::Level \_traceLevelSpecific
- bool \_traceLevelSpecificEnabled = false

## 15.943.1 Detailed Description

Stochastic mRNA synthesis via Poisson tau-leaping for whole-cell simulations.

Models the M. genitalium Transcription process (Karr et al. 2012):  $\lambda_i = (\text{elongationRate} / \text{meanGeneLength}) \times \text{bindingProbability} \times \text{RNAP\_free} \times \Delta t$   $N_{\text{synth}_i} \sim \text{Poisson}(\lambda_i)$

This is a tau-leaping approximation (not full Gillespie) appropriate for transcription because RNAP count (~200) acting over many genes (~473) with a 1–10 s window produces moderate propensities where the Poisson approximation is valid. Full Gillespie is implemented in [StochasticReactionComponent](#) for individual reactions with low copy numbers.

Parameters loaded from [WholeCellState](#) (set by loadParameters()): param.rnaPolymeraseElongationRate — 50 nt/s (from parameters.json)

mRNA species in [WholeCellState](#) are identified by the prefix set in mRNASpeciesPrefix (default "mRNA\_"). RNAP free count is read from the [WholeCellState](#) molecule count with key rnapCountKey (default "RNAP\_free").

## 15.943.2 Member Function Documentation

### 15.943.2.1 \_check()

```
bool StochasticTranscription::_check (
 std::string & errorMessage) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

**15.943.2.2 \_createEditableDataDefinitions()**

```
void StochasticTranscription::_createEditableDataDefinitions () [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

**15.943.2.3 \_initBetweenReplications()**

```
void StochasticTranscription::_initBetweenReplications () [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

**15.943.2.4 \_loadInstance()**

```
bool StochasticTranscription::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

**15.943.2.5 \_onDispatchEvent()**

```
void StochasticTranscription::_onDispatchEvent (
 Entity * entity,
 unsigned int inputPortNumber) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

**Parameters**

|                        |                                         |
|------------------------|-----------------------------------------|
| <i>entity</i>          | <a href="#">Entity</a> being processed. |
| <i>inputPortNumber</i> | Input port where the entity arrived.    |

Implements [ModelComponent](#).

**15.943.2.6 \_saveInstance()**

```
void StochasticTranscription::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

**Parameters**

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [ModelComponent](#).

**15.943.2.7 show()**

```
std::string StochasticTranscription::show () [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

**Returns**

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

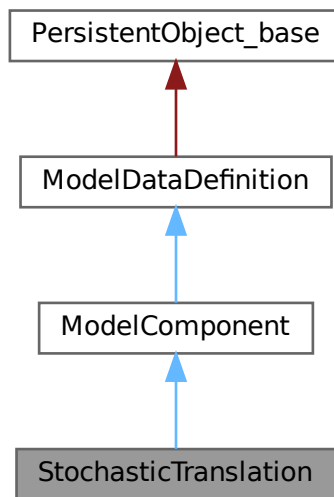
- `plugins/components/WholeCellModeling/StochasticTranscription.h`

- `plugins/components/WholeCellModeling/StochasticTranscription.cpp`

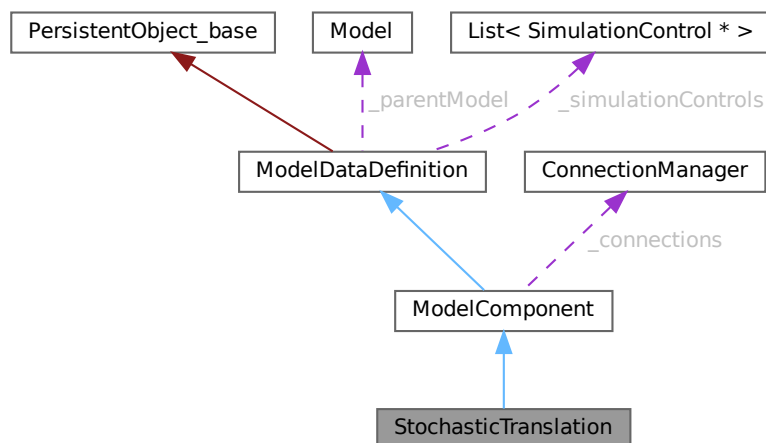
## 15.944 StochasticTranslation Class Reference

```
#include <plugins/components/WholeCellModeling/StochasticTranslation.h>
```

Inheritance diagram for StochasticTranslation:



Collaboration diagram for StochasticTranslation:



### Public Member Functions

- **StochasticTranslation** (`Model` \*model, `std::string` name="")
- void **setWholeCellState** (`WholeCellState` \*state)

- [WholeCellState](#) \* **getWholeCellState** () const
- void **setElongationRate** (double rate)
- double **getElongationRate** () const
- void **setMeanProteinLength** (double length)
- double **getMeanProteinLength** () const
- void **setTimeWindow** (double window)
- double **getTimeWindow** () const
- void **setMRNAspeciesPrefix** (std::string prefix)
- std::string **getMRNAspeciesPrefix** () const
- void **setProteinSpeciesPrefix** (std::string prefix)
- std::string **getProteinSpeciesPrefix** () const
- void **setRibosomeCountKey** (std::string key)
- std::string **getRibosomeCountKey** () const
- void **setRandomSeed** (unsigned int seed)
- unsigned int **getRandomSeed** () const
- int **getLastSynthesizedCount** () const
- virtual std::string **show** () override

*virtual public methods*

## Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) \*model, std::string componentTypename, std::string name="")  
*constructors*
- virtual ~[ModelComponent](#) ()  
*Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) \* **getConnectionManager** () const  
*new public user methods for this component*
- void **connectTo** ([ModelComponent](#) \*component, unsigned int inputPortNumber=0)  
*Connects this component to another component.*
- bool **hasBreakpointAt** ()  
*Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string \_description)  
*Updates the free-form description shown by the GUI.*
- std::string **getDescription** () const  
*Returns the free-form description associated with this component.*
- void **remove** ([ModelComponent](#) \*comp)  
*Removes a connected component from this component.*

## Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) \*model, std::string datadefinitionTypename, std::string name="", bool insert↔IntoModel=true)  
*Creates a model data definition and optionally inserts it into the model.*
- virtual ~[ModelDataDefinition](#) ()  
*Releases resources owned by this data definition.*
- Util::identification **getId** () const  
*Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)  
*Updates the logical name of this instance.*
- const std::string & **getName** () const  
*Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)  
*Updates the user-visible label of this instance.*

- const std::string & [getLabel](#) () const  
*Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const  
*Returns the runtime class name used by the plugin system.*
- [Model](#) \* [getParentModel](#) () const  
*Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const  
*Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)  
*Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) \* [getInternalData](#) (const std::string &name) const  
*Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getInternalData](#) () const  
*Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getAttachedData](#) () const  
*Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const  
*Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))  
*Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const  
*Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int \_modelLevel)  
*Sets the model nesting level for this definition.*
- [List](#)< [SimulationControl](#) \* > \* [getSimulationControls](#) () const  
*Returns the simulation controls owned by this data definition.*
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const  
*Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)  
*Updates the trace level used by this definition.*
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelspecific, bool traceLevelSpecificEnabled=true)  
*Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const  
*Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)  
*Enables or disables the specific trace level.*

### Static Public Member Functions

- static [PluginInformation](#) \* [GetPluginInformation](#) ()
- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")

### Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool [Check](#) ([ModelComponent](#) \*component)  
*Validates a component instance.*
- static void [CreateInternalData](#) ([ModelComponent](#) \*component)

- Creates internal data required by a component instance.*
  - static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelComponent](#) \*component)
  - Serializes a component instance into a persistence record.*
  - static void [DispatchEvent](#) ([Event](#) \*event)
  - Dispatches a simulation event to the target component.*

### Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)
  - Loads a model data definition instance from serialized fields.*
  - static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")
  - Creates a new instance of a data definition using the plugin registry.*
  - static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)
  - Serializes a data definition instance into a persistence record.*
  - static bool [Check](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)
  - Validates a data definition instance.*
  - static void [InitBetweenReplications](#) ([ModelDataDefinition](#) \*modeldatum)
  - Resets simulation-time state between replications.*
  - static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)
  - Creates internal and attached data required by this instance.*
  - static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)
  - Creates related data and validates the instance in a single call.*

### Protected Member Functions

- virtual bool [\\_loadInstance](#) ([PersistenceRecord](#) \*fields) override
  - virtual protected method that must be overridden*
  - virtual void [\\_saveInstance](#) ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override
  - Saves object fields into a serialization structure.*
  - virtual bool [\\_check](#) (std::string &errorMessage) override
  - could be overridden by derived classes*
  - virtual void [\\_initBetweenReplications](#) () override
  - virtual void [\\_onDispatchEvent](#) ([Entity](#) \*entity, unsigned int inputPortNumber) override
  - Handles a dispatched event for this component type.*
  - virtual void [\\_createEditableDataDefinitions](#) () override

### Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [\\_getSaveDefaultsOption](#) ()
  - virtual [ParserChangesInformation](#) \* [\\_getParserChangesInformation](#) ()
  - virtual void [\\_addSimulationControl](#) ([SimulationControl](#) \*control)
  - virtual void [\\_createInternalAndAttachedData](#) ()
  - virtual void [\\_createInternalStatisticReporters](#) ()
  - virtual void [\\_createAttachedAttributes](#) ()
  - virtual void [\\_createNonEditableDataDefinitions](#) ()
  - void [\\_templateCreateInternalStatisticReporters](#) ()
  - void [\\_templateCreateAttachedAttributes](#) ()
  - void [\\_templateCreateNonEditableDataDefinitions](#) ()
  - void [\\_templateCreateEditableDataDefinitions](#) ()
  - void [\\_internalStatisticReportersClear](#) ()
  - legacy helper methods used by the granular create\* hooks*
  - void [\\_internalDataClear](#) ()
  - void [\\_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*child)

- void **\_internalDataRemove** (const std::string &key)
- void **\_attachedDataClear** ()
- void **\_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_attachedDataRemove** (const std::string &key)
- void **\_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **\_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_statisticReporterRemove** (const std::string &key)
- void **\_statisticReportersClear** ()
- void **\_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **\_mandatoryAttachedAttributesClear** ()
- void **\_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryEditableDataDefinitionsClear** ()
- void **\_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalEditableDataDefinitionsClear** ()
- void **\_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryNonEditableDataDefinitionsClear** ()
- void **\_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalNonEditableDataDefinitionsClear** ()
- void **\_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)  
*method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)  
*not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- void **traceSimulation** (void \*thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)  
*Traces text to the simulation output.*
- void **traceSimulation** (void \*thisobject, TraceManager::Level level, const std::string &text)

### Additional Inherited Members

### Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) \* **\_connections** = new [ConnectionManager](#)()  
*virtual protected methods that could be overridden by derived classes, if needed*

### Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **\_id**
- std::string **\_typename**
- bool **\_reportStatistics**
- bool **\_hasChanged**
- unsigned int **\_modelLevel** = 0
- [Model](#) \* **\_parentModel**
- [List](#)< [SimulationControl](#) \* > \* **\_simulationControls** = new [List](#)<[SimulationControl](#)\*>()
- TraceManager::Level **\_traceLevelSpecific**
- bool **\_traceLevelSpecificEnabled** = false

### 15.944.1 Detailed Description

Stochastic protein synthesis via Poisson tau-leaping for whole-cell simulations.

Models the M. genitalium Translation process (Karr et al. 2012):  $\lambda_i = (\text{ribosomeElongationRate} / \text{meanProteinLength}) \times \text{mRNA\_count}_i \times \text{ribosome\_free} / \max(1, \text{total\_mRNA}) \times \Delta t$   $N_{\text{protein}_i} \sim \text{Poisson}(\lambda_i)$

Each mRNA template (prefix `proteinSpeciesPrefix`) is matched to its corresponding mRNA species (prefix `mRNA`SpeciesPrefix) in [WholeCellState](#). Free ribosome count is read from [WholeCellState](#) with key `ribosomeCountKey`.

Parameters loaded from [WholeCellState](#) (set by `loadParameters()`): `param.ribosomeElongationRate` — 16 AA/s (from `parameters.json`)

mRNA degradation is not modeled here; use a separate [StochasticReactionRule](#) for mRNA decay (first-order, rate  $\sim 1/\text{mRNA\_halflife}$ ).

### 15.944.2 Member Function Documentation

#### 15.944.2.1 `_check()`

```
bool StochasticTranslation::_check (
 std::string & errorMessage) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

#### 15.944.2.2 `_createEditableDataDefinitions()`

```
void StochasticTranslation::_createEditableDataDefinitions () [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

#### 15.944.2.3 `_initBetweenReplications()`

```
void StochasticTranslation::_initBetweenReplications () [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

#### 15.944.2.4 `_loadInstance()`

```
bool StochasticTranslation::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

#### 15.944.2.5 `_onDispatchEvent()`

```
void StochasticTranslation::_onDispatchEvent (
 Entity * entity,
 unsigned int inputPortNumber) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for [ModelComponents](#), not for [ModelDataElements](#).

#### Parameters

|                        |                                         |
|------------------------|-----------------------------------------|
| <i>entity</i>          | <a href="#">Entity</a> being processed. |
| <i>inputPortNumber</i> | Input port where the entity arrived.    |

Implements [ModelComponent](#).

#### 15.944.2.6 `_saveInstance()`

```
void StochasticTranslation::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```



Saves object fields into a serialization structure.

## Parameters

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [ModelComponent](#).

**15.944.2.7 show()**

```
std::string StochasticTranslation::show () [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

## Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/WholeCellModeling/StochasticTranslation.h
- plugins/components/WholeCellModeling/StochasticTranslation.cpp

**15.945 BioReaction::StoichiometricTerm Struct Reference**

## Public Attributes

- std::string **speciesName**
- double **stoichiometry** = 1.0

The documentation for this struct was generated from the following file:

- plugins/data/BiochemicalSimulation/BioReaction.h

**15.946 MassActionOdeSystem::StoichiometricTerm Struct Reference**

## Public Attributes

- unsigned int **speciesIndex** = 0
- double **stoichiometry** = 1.0

The documentation for this struct was generated from the following file:

- tools/Biochemical/MassActionOdeSystem.h

**15.947 MetabolicReaction::StoichiometricTerm Struct Reference**

## Public Attributes

- std::string **speciesName**
- double **stoichiometry** = 1.0

The documentation for this struct was generated from the following file:

- plugins/data/BiochemicalSimulation/MetabolicReaction.h

## 15.948 StochasticReactionRule::StoichiometricTerm Struct Reference

### Public Attributes

- std::string **speciesName**
- int **stoichiometry** = 1

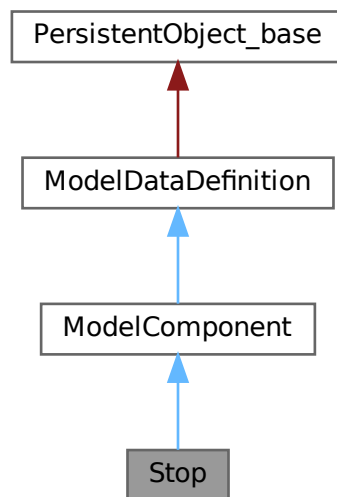
The documentation for this struct was generated from the following file:

- plugins/data/WholeCellModeling/StochasticReactionRule.h

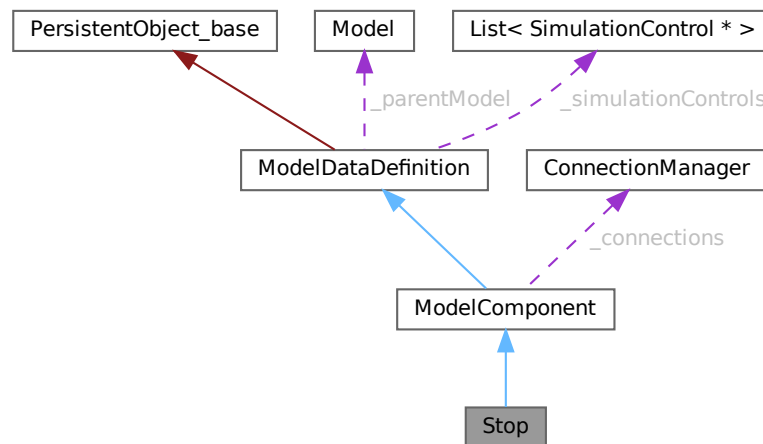
## 15.949 Stop Class Reference

```
#include <plugins/components/MaterialHandling/Stop.h>
```

Inheritance diagram for Stop:



Collaboration diagram for Stop:



### Public Member Functions

- **Stop** ([Model](#) \*model, std::string name="")
  - virtual std::string [show](#) () override
- virtual public methods*

### Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) \*model, std::string componentTypename, std::string name="")  
*constructors*
- virtual ~**ModelComponent** ()  
*Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) \* [getConnectionManager](#) () const  
*new public user methods for this component*
- void [connectTo](#) ([ModelComponent](#) \*component, unsigned int inputPortNumber=0)  
*Connects this component to another component.*
- bool [hasBreakpointAt](#) ()  
*Indicates whether a simulation breakpoint is set on this component.*
- void [setDescription](#) (std::string \_description)  
*Updates the free-form description shown by the GUI.*
- std::string [getDescription](#) () const  
*Returns the free-form description associated with this component.*
- void [remove](#) ([ModelComponent](#) \*comp)  
*Removes a connected component from this component.*

### Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) \*model, std::string datadefinitionTypename, std::string name="", bool insert↔ IntoModel=true)  
*Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()  
*Releases resources owned by this data definition.*

- Util::identification [getId](#) () const  
*Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)  
*Updates the logical name of this instance.*
- const std::string & [getName](#) () const  
*Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)  
*Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const  
*Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const  
*Returns the runtime class name used by the plugin system.*
- Model \* [getParentModel](#) () const  
*Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const  
*Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)  
*Enables or disables report-statistics generation for this definition.*
- ModelDataDefinition \* [getInternalData](#) (const std::string &name) const  
*Looks up a named internal child data definition.*
- std::map< std::string, ModelDataDefinition \* > \* [getInternalData](#) () const  
*Returns the full map of internal child data definitions.*
- std::map< std::string, ModelDataDefinition \* > \* [getAttachedData](#) () const  
*Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const  
*Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))  
*Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const  
*Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int \_modelLevel)  
*Sets the model nesting level for this definition.*
- List< SimulationControl \* > \* [getSimulationControls](#) () const  
*Returns the simulation controls owned by this data definition.*
- TraceManager::Level [getTraceLevelSpecific](#) () const  
*Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) (TraceManager::Level level)  
*Updates the trace level used by this definition.*
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific←  
Enabled=true)  
*Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const  
*Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)  
*Enables or disables the specific trace level.*

### Static Public Member Functions

- static PluginInformation \* [GetPluginInformation](#) ()
- static ModelComponent \* [LoadInstance](#) (Model \*model, PersistenceRecord \*fields)
- static ModelDataDefinition \* [NewInstance](#) (Model \*model, std::string name="")

## Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool [Check](#) ([ModelComponent](#) \*component)  
*Validates a component instance.*
- static void [CreateInternalData](#) ([ModelComponent](#) \*component)  
*Creates internal data required by a component instance.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelComponent](#) \*component)  
*Serializes a component instance into a persistence record.*
- static void [DispatchEvent](#) ([Event](#) \*event)  
*Dispatches a simulation event to the target component.*

## Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool [Check](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*
- static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

## Protected Member Functions

- virtual void [\\_onDispatchEvent](#) ([Entity](#) \*entity, unsigned int inputPortNumber) override  
*Handles a dispatched event for this component type.*
- virtual bool [\\_loadInstance](#) ([PersistenceRecord](#) \*fields) override  
*virtual protected method that must be overridden*
- virtual void [\\_saveInstance](#) ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*
- virtual bool [\\_check](#) (std::string &errorMessage) override  
*could be overridden by derived classes*

## Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [\\_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) \* [\\_getParserChangesInformation](#) ()
- virtual void [\\_initBetweenReplications](#) ()
- virtual void [\\_addSimulationControl](#) ([SimulationControl](#) \*control)
- virtual void [\\_createInternalAndAttachedData](#) ()
- virtual void [\\_createInternalStatisticReporters](#) ()
- virtual void [\\_createAttachedAttributes](#) ()
- virtual void [\\_createNonEditableDataDefinitions](#) ()
- virtual void [\\_createEditableDataDefinitions](#) ()
- void [\\_templateCreateInternalStatisticReporters](#) ()

- void **\_templateCreateAttachedAttributes** ()
- void **\_templateCreateNonEditableDataDefinitions** ()
- void **\_templateCreateEditableDataDefinitions** ()
- void **\_internaStatisticReportersClear** ()
- legacy helper methods used by the granular create\* hooks*
- void **\_internalDataClear** ()
- void **\_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) \*child)
- void **\_internalDataRemove** (const std::string &key)
- void **\_attachedDataClear** ()
- void **\_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_attachedDataRemove** (const std::string &key)
- void **\_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **\_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_statisticReporterRemove** (const std::string &key)
- void **\_statisticReportersClear** ()
- void **\_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **\_mandatoryAttachedAttributesClear** ()
- void **\_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryEditableDataDefinitionsClear** ()
- void **\_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalEditableDataDefinitionsClear** ()
- void **\_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryNonEditableDataDefinitionsClear** ()
- void **\_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalNonEditableDataDefinitionsClear** ()
- void **\_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
- method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- void **traceSimulation** (void \*thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- Traces text to the simulation output.*
- void **traceSimulation** (void \*thisobject, TraceManager::Level level, const std::string &text)

### Additional Inherited Members

### Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) \* **\_connections** = new [ConnectionManager](#)()
- virtual protected methods that could be overridden by derived classes, if needed*

## Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) \* `_parentModel`
- [List](#)< [SimulationControl](#) \* > \* `_simulationControls` = new [List](#)<[SimulationControl](#)\*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

### 15.949.1 Detailed Description

[Stop](#) module DESCRIPTION The [Stop](#) module sets the operational status of a conveyor to inactive. The conveyor may have been activated from either the [Start](#) module or by initially being set to active at the start of the simulation. When the entity enters the [Stop](#) module, the conveyor will stop immediately, regardless of the type of conveyor or the number of entities currently on the conveyor. TYPICAL USES [Stop](#) a baggage conveyor after a pre-determined amount of time [Stop](#) a conveyor for scheduled maintenance PROMPTS Prompt Description Name Unique name of the module that will be displayed in the flowchart. Conveyor Name Name of the conveyor to stop.

### 15.949.2 Member Function Documentation

#### 15.949.2.1 `_check()`

```
bool Stop::_check (
 std::string & errorMessage) [override], [protected], [virtual]
```

could be overridden by derived classes  
Reimplemented from [ModelDataDefinition](#).

#### 15.949.2.2 `_loadInstance()`

```
bool Stop::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

virtual protected method that must be overridden  
Reimplemented from [ModelComponent](#).

#### 15.949.2.3 `_onDispatchEvent()`

```
void Stop::_onDispatchEvent (
 Entity * entity,
 unsigned int inputPortNumber) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

#### Parameters

|                        |                                         |
|------------------------|-----------------------------------------|
| <i>entity</i>          | <a href="#">Entity</a> being processed. |
| <i>inputPortNumber</i> | Input port where the entity arrived.    |

Implements [ModelComponent](#).

#### 15.949.2.4 `_saveInstance()`

```
void Stop::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.



## Parameters

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [ModelComponent](#).

**15.949.2.5 show()**

```
std::string Stop::show () [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

## Returns

Human-readable description used for tracing and debugging.

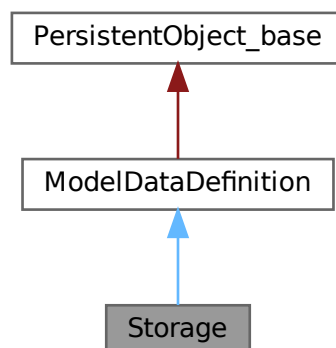
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

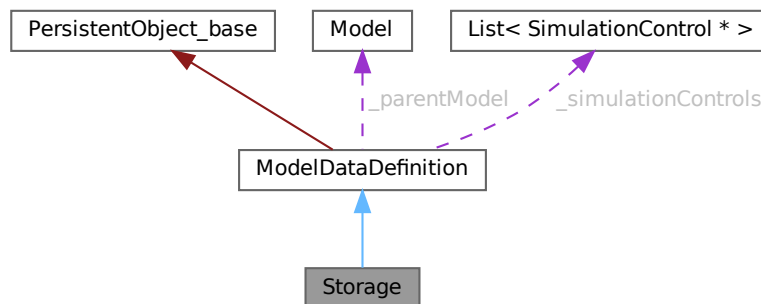
- plugins/components/MaterialHandling/Stop.h
- plugins/components/MaterialHandling/Stop.cpp

## 15.950 Storage Class Reference

Inheritance diagram for Storage:



Collaboration diagram for Storage:



### Public Member Functions

- **Storage** ([Model](#) \*model, std::string name="")
- virtual std::string **show** () override  
*Returns a textual representation of this data definition.*
- void **setTotalArea** (double \_totalArea)
- double **getTotalArea** () const
- void **setCapacity** (unsigned int \_capacity)
- unsigned int **getCapacity** () const
- void **setUnitsPerArea** (double \_unitsPerArea)
- double **getUnitsPerArea** () const

### Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) \*model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)  
*Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()  
*Releases resources owned by this data definition.*
- Util::identification **getId** () const  
*Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)  
*Updates the logical name of this instance.*
- const std::string & **getName** () const  
*Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)  
*Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const  
*Returns the user-visible label of this instance.*
- const std::string & **getClassname** () const  
*Returns the runtime class name used by the plugin system.*
- [Model](#) \* **getParentModel** () const  
*Returns the parent model that owns this definition.*
- bool **isReportStatistics** () const  
*Indicates whether this data definition contributes report statistics.*
- void **setReportStatistics** (bool reportStatistics)

- Enables or disables report-statistics generation for this definition.*
- `ModelDataDefinition * getInternalData (const std::string &name) const`  
*Looks up a named internal child data definition.*
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`  
*Returns the full map of internal child data definitions.*
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`  
*Returns the full map of attached data definitions.*
- `bool hasChanged () const`  
*Indicates whether this instance has been modified since the last reset.*
- `void setHasChanged (bool hasChanged)`  
*Updates the changed flag for this data definition.*
- `unsigned int getLevel () const`  
*Returns the level in the model hierarchy where this definition lives.*
- `void setModelLevel (unsigned int _modelLevel)`  
*Sets the model nesting level for this definition.*
- `List< SimulationControl * > * getSimulationControls () const`  
*Returns the simulation controls owned by this data definition.*
- `TraceManager::Level getTraceLevelSpecific () const`  
*Returns the trace level that applies specifically to this definition.*
- `void setTraceLevelSpecific (TraceManager::Level level)`  
*Updates the trace level used by this definition.*
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific←Enabled=true)`  
*Defines the specific trace level and optionally enables it immediately.*
- `bool isTraceLevelSpecificEnabled () const`  
*Indicates whether the specific trace level is enabled.*
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`  
*Enables or disables the specific trace level.*

### Static Public Member Functions

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static PluginInformation * GetPluginInformation ()`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

### Static Public Member Functions inherited from `ModelDataDefinition`

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`  
*Loads a model data definition instance from serialized fields.*
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`  
*Creates a new instance of a data definition using the plugin registry.*
- `static void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`  
*Serializes a data definition instance into a persistence record.*
- `static bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`  
*Validates a data definition instance.*
- `static void InitBetweenReplications (ModelDataDefinition *modeldatum)`  
*Resets simulation-time state between replications.*
- `static void CreateInternalData (ModelDataDefinition *modeldatum)`  
*Creates internal and attached data required by this instance.*
- `static bool CreateRelatedDataElements (ModelDataDefinition *modeldatum, std::string &errorMessage)`  
*Creates related data and validates the instance in a single call.*

## Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) \*fields) override  
*must be overridden by derived classes*
- virtual void `_saveInstance` ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*
- virtual bool `_check` (std::string &errorMessage) override  
*could be overridden by derived classes*
- virtual [ParserChangesInformation](#) \* `_getParserChangesInformation` () override

## Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) \*control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()  
*legacy helper methods used by the granular create\* hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) \*child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)  
*method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void `trace` (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))

*not just an easy access to trace manager, but wrappers to check if specific trace level applies*

- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- void [traceSimulation](#) (void \*thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)

*Traces text to the simulation output.*

- void **traceSimulation** (void \*thisobject, TraceManager::Level level, const std::string &text)

## Additional Inherited Members

### Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **\_id**
- std::string **\_typename**
- bool **\_reportStatistics**
- bool **\_hasChanged**
- unsigned int **\_modelLevel** = 0
- [Model](#) \* **\_parentModel**
- [List](#)< [SimulationControl](#) \* > \* **\_simulationControls** = new [List](#)<[SimulationControl](#)\*>()
- TraceManager::Level **\_traceLevelSpecific**
- bool **\_traceLevelSpecificEnabled** = false

## 15.950.1 Member Function Documentation

### 15.950.1.1 [\\_check\(\)](#)

```
bool Storage::_check (
 std::string & errorMessage) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

### 15.950.1.2 [\\_getParserChangesInformation\(\)](#)

```
ParserChangesInformation * Storage::_getParserChangesInformation () [override], [protected],
[virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelData. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dynamically linked to simulator kernel to reflect the changes

Reimplemented from [ModelDataDefinition](#).

### 15.950.1.3 [\\_loadInstance\(\)](#)

```
bool Storage::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

### 15.950.1.4 [\\_saveInstance\(\)](#)

```
void Storage::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

## Parameters

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Persist [Storage](#) parameters using symmetric keys from `_loadInstance`.

Reimplemented from [ModelDataDefinition](#).

**15.950.1.5 show()**

```
std::string Storage::show () [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

## Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

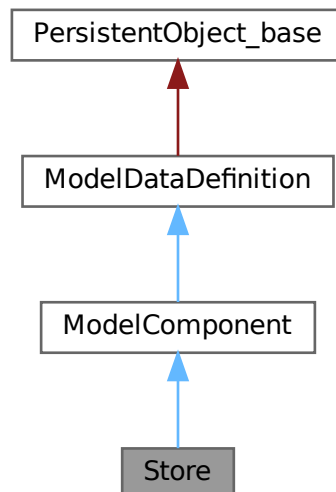
The documentation for this class was generated from the following files:

- `plugins/data/MaterialHandling/Storage.h`
- `plugins/data/MaterialHandling/Storage.cpp`

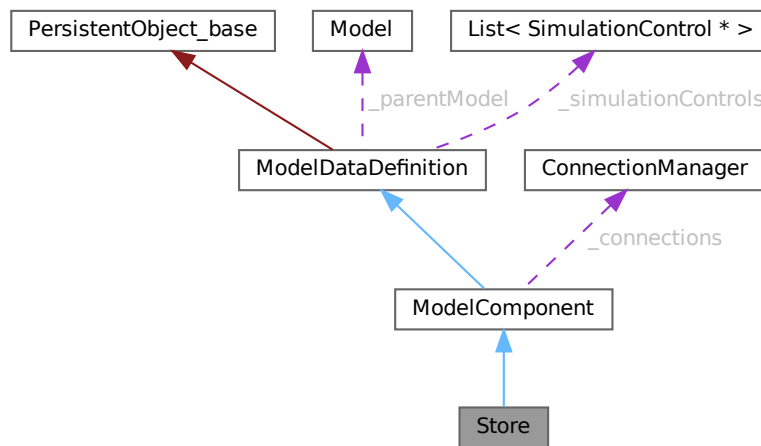
**15.951 Store Class Reference**

```
#include <plugins/components/MaterialHandling/Store.h>
```

Inheritance diagram for Store:



Collaboration diagram for Store:



### Public Member Functions

- **Store** (**Model** \*model, std::string name="")
  - virtual std::string **show** () override
- virtual public methods*

### Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** \*model, std::string componentTypename, std::string name="")
- constructors*
- virtual ~**ModelComponent** ()
- Releases component-owned resources and unregisters it from the model.*
- **ConnectionManager** \* **getConnectionManager** () const
- new public user methods for this component*
- void **connectTo** (**ModelComponent** \*component, unsigned int inputPortNumber=0)
- Connects this component to another component.*
- bool **hasBreakpointAt** ()
- Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string \_description)
- Updates the free-form description shown by the GUI.*
- std::string **getDescription** () const
- Returns the free-form description associated with this component.*
- void **remove** (**ModelComponent** \*comp)
- Removes a connected component from this component.*

### Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** \*model, std::string datadefinitionTypename, std::string name="", bool insert↔ IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- Releases resources owned by this data definition.*

- Util::identification [getId](#) () const  
*Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)  
*Updates the logical name of this instance.*
- const std::string & [getName](#) () const  
*Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)  
*Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const  
*Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const  
*Returns the runtime class name used by the plugin system.*
- [Model](#) \* [getParentModel](#) () const  
*Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const  
*Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)  
*Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) \* [getInternalData](#) (const std::string &name) const  
*Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getInternalData](#) () const  
*Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getAttachedData](#) () const  
*Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const  
*Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))  
*Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const  
*Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int \_modelLevel)  
*Sets the model nesting level for this definition.*
- [List](#)< [SimulationControl](#) \* > \* [getSimulationControls](#) () const  
*Returns the simulation controls owned by this data definition.*
- [TraceManager](#)::Level [getTraceLevelSpecific](#) () const  
*Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) ([TraceManager](#)::Level level)  
*Updates the trace level used by this definition.*
- void [defineTraceLevelSpecific](#) ([TraceManager](#)::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)  
*Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const  
*Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)  
*Enables or disables the specific trace level.*

### Static Public Member Functions

- static [PluginInformation](#) \* [GetPluginInformation](#) ()
- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")



## Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool [Check](#) ([ModelComponent](#) \*component)  
*Validates a component instance.*
- static void [CreateInternalData](#) ([ModelComponent](#) \*component)  
*Creates internal data required by a component instance.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelComponent](#) \*component)  
*Serializes a component instance into a persistence record.*
- static void [DispatchEvent](#) ([Event](#) \*event)  
*Dispatches a simulation event to the target component.*

## Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool [Check](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*
- static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

## Protected Member Functions

- virtual void [\\_onDispatchEvent](#) ([Entity](#) \*entity, unsigned int inputPortNumber) override  
*Handles a dispatched event for this component type.*
- virtual bool [\\_loadInstance](#) ([PersistenceRecord](#) \*fields) override  
*virtual protected method that must be overridden*
- virtual void [\\_saveInstance](#) ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*
- virtual bool [\\_check](#) (std::string &errorMessage) override  
*could be overridden by derived classes*

## Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [\\_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) \* [\\_getParserChangesInformation](#) ()
- virtual void [\\_initBetweenReplications](#) ()
- virtual void [\\_addSimulationControl](#) ([SimulationControl](#) \*control)
- virtual void [\\_createInternalAndAttachedData](#) ()
- virtual void [\\_createInternalStatisticReporters](#) ()
- virtual void [\\_createAttachedAttributes](#) ()
- virtual void [\\_createNonEditableDataDefinitions](#) ()
- virtual void [\\_createEditableDataDefinitions](#) ()
- void [\\_templateCreateInternalStatisticReporters](#) ()

- void **\_templateCreateAttachedAttributes** ()
- void **\_templateCreateNonEditableDataDefinitions** ()
- void **\_templateCreateEditableDataDefinitions** ()
- void **\_internaStatisticReportersClear** ()
- legacy helper methods used by the granular create\* hooks*
- void **\_internalDataClear** ()
- void **\_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) \*child)
- void **\_internalDataRemove** (const std::string &key)
- void **\_attachedDataClear** ()
- void **\_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_attachedDataRemove** (const std::string &key)
- void **\_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **\_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_statisticReporterRemove** (const std::string &key)
- void **\_statisticReportersClear** ()
- void **\_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **\_mandatoryAttachedAttributesClear** ()
- void **\_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryEditableDataDefinitionsClear** ()
- void **\_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalEditableDataDefinitionsClear** ()
- void **\_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryNonEditableDataDefinitionsClear** ()
- void **\_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalNonEditableDataDefinitionsClear** ()
- void **\_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
- method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- void **traceSimulation** (void \*thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- Traces text to the simulation output.*
- void **traceSimulation** (void \*thisobject, TraceManager::Level level, const std::string &text)

## Additional Inherited Members

## Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) \* **\_connections** = new [ConnectionManager](#)()
- virtual protected methods that could be overridden by derived classes, if needed*

## Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **\_id**
- std::string **\_typename**
- bool **\_reportStatistics**
- bool **\_hasChanged**
- unsigned int **\_modelLevel** = 0
- [Model](#) \* **\_parentModel**
- [List](#)< [SimulationControl](#) \* > \* **\_simulationControls** = new [List](#)<[SimulationControl](#)\*>()
- TraceManager::Level **\_traceLevelSpecific**
- bool **\_traceLevelSpecificEnabled** = false

### 15.951.1 Detailed Description

[Store](#) module DESCRIPTION The [Store](#) module adds an entity to storage. The [Unstore](#) module may then be used to remove the entity from the storage. When an entity arrives at the [Store](#) module, the storage specified is incremented, and the entity immediately moves to the next module in the model. Storages are useful for displaying entity animation while an entity undergoes processing in other modules. Additionally, statistics may be kept on the number of entities in storage. TYPICAL USES Animating a part through a number of delay operations (load, setup, process, unload) Tracking the number of customers within a grocery store (place in storage upon entry) PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Type Method of specifying the storage name as a [Storage](#), [Set](#), [Attribute](#), or Expression. [Storage](#) Name Name of the storage to which the entity will be added. Applies only when the Type is [Storage](#). [Set](#) Name Name of the storage set from which the storage is to be selected. Applies only when the Type is [Set](#). [Set](#) Index Index into the defined storage set that contains the desired storage name. Applies only when the Type is [Set](#). [Attribute](#) Name of the attribute whose value contains the storage. Applies only when the Type is [Attribute](#). Expression Expression that is evaluated to the storage into which the entity is placed. Applies only when the Type is Expression.

### 15.951.2 Member Function Documentation

#### 15.951.2.1 [\\_check\(\)](#)

```
bool Store::_check (
 std::string & errorMessage) [override], [protected], [virtual]
```

could be overridden by derived classes

Validate minimal consistency for a pass-through [Store](#) component.

Reimplemented from [ModelDataDefinition](#).

#### 15.951.2.2 [\\_loadInstance\(\)](#)

```
bool Store::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Load Store-specific persistent fields.

[Store](#) currently has no extra persisted attributes beyond [ModelComponent](#). Keep the template commands below as a guide for future extensions.

Reimplemented from [ModelComponent](#).

#### 15.951.2.3 [\\_onDispatchEvent\(\)](#)

```
void Store::_onDispatchEvent (
 Entity * entity,
 unsigned int inputPortNumber) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for [ModelComponents](#), not for [ModelDataElements](#).

#### Parameters

|                        |                                         |
|------------------------|-----------------------------------------|
| <i>entity</i>          | <a href="#">Entity</a> being processed. |
| <i>inputPortNumber</i> | Input port where the entity arrived.    |

Implements [ModelComponent](#).

#### 15.951.2.4 \_saveInstance()

```
void Store::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

##### Parameters

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Save Store-specific persistent fields.

Reimplemented from [ModelComponent](#).

#### 15.951.2.5 show()

```
std::string Store::show () [override], [virtual]
virtual public methods
```

Returns a textual representation of this component.

##### Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/MaterialHandling/Store.h
- plugins/components/MaterialHandling/Store.cpp

## 15.952 testing::internal::internal\_stream\_operator\_without\_lexical\_name\_lookup::StreamPrinter Struct Reference

### Static Public Member Functions

- `template<typename T, typename = typename std::enable_if<!std::is_member_pointer<T>::value>::type, typename = decltype(std::declval<std::ostream&>() << std::declval<const T&>())>`  
static void **PrintValue** (const T &value, ::std::ostream \*os)

The documentation for this struct was generated from the following file:

- gtest/gtest/gtest-printers.h

## 15.953 testing::internal::String Class Reference

### Static Public Member Functions

- static const char \* **CloneCString** (const char \*c\_str)
- static bool **CStringEquals** (const char \*lhs, const char \*rhs)
- static std::string **ShowWideCString** (const wchar\_t \*wide\_c\_str)
- static bool **WideCStringEquals** (const wchar\_t \*lhs, const wchar\_t \*rhs)
- static bool **CaseInsensitiveCStringEquals** (const char \*lhs, const char \*rhs)
- static bool **CaseInsensitiveWideCStringEquals** (const wchar\_t \*lhs, const wchar\_t \*rhs)
- static bool **EndsWithCaseInsensitive** (const std::string &str, const std::string &suffix)
- static std::string **FormatIntWidth2** (int value)
- static std::string **FormatIntWidthN** (int value, int width)

- static std::string **FormatHexInt** (int value)
- static std::string **FormatHexUInt32** (uint32\_t value)
- static std::string **FormatByte** (unsigned char value)

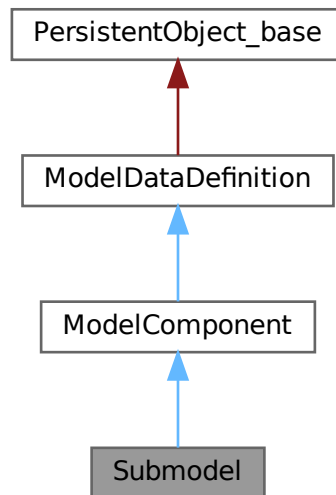
The documentation for this class was generated from the following files:

- gtest/gtest/internal/gtest-string.h
- gtest/src/gtest.cc

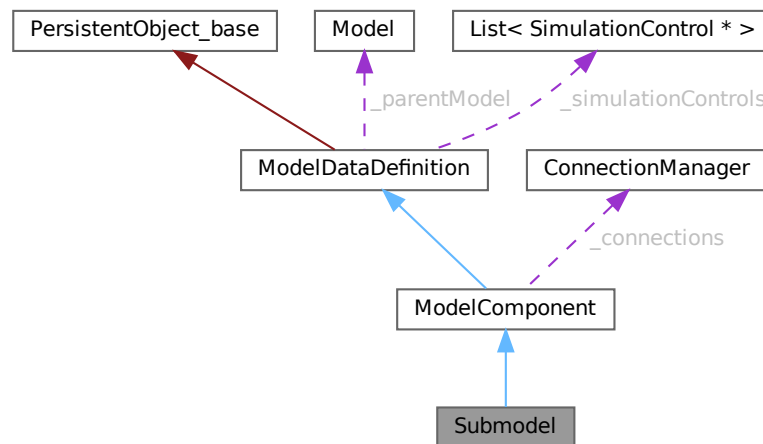
## 15.954 Submodel Class Reference

```
#include <plugins/components/ModalModel/Submodel.h>
```

Inheritance diagram for Submodel:



Collaboration diagram for Submodel:



### Public Member Functions

- **Submodel** ([Model](#) \*model, std::string name="")
  - virtual std::string [show](#) () override
- virtual public methods*

### Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) \*model, std::string componentTypename, std::string name="")
- constructors*
- virtual ~**ModelComponent** ()
- Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) \* [getConnectionManager](#) () const
- new public user methods for this component*
- void [connectTo](#) ([ModelComponent](#) \*component, unsigned int inputPortNumber=0)
- Connects this component to another component.*
- bool [hasBreakpointAt](#) ()
- Indicates whether a simulation breakpoint is set on this component.*
- void [setDescription](#) (std::string \_description)
- Updates the free-form description shown by the GUI.*
- std::string [getDescription](#) () const
- Returns the free-form description associated with this component.*
- void [remove](#) ([ModelComponent](#) \*comp)
- Removes a connected component from this component.*

### Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) \*model, std::string datadefinitionTypename, std::string name="", bool insert↔ IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- Releases resources owned by this data definition.*

- Util::identification [getId](#) () const  
*Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)  
*Updates the logical name of this instance.*
- const std::string & [getName](#) () const  
*Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)  
*Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const  
*Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const  
*Returns the runtime class name used by the plugin system.*
- Model \* [getParentModel](#) () const  
*Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const  
*Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)  
*Enables or disables report-statistics generation for this definition.*
- ModelDataDefinition \* [getInternalData](#) (const std::string &name) const  
*Looks up a named internal child data definition.*
- std::map< std::string, ModelDataDefinition \* > \* [getInternalData](#) () const  
*Returns the full map of internal child data definitions.*
- std::map< std::string, ModelDataDefinition \* > \* [getAttachedData](#) () const  
*Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const  
*Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool hasChanged)  
*Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const  
*Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int \_modelLevel)  
*Sets the model nesting level for this definition.*
- List< SimulationControl \* > \* [getSimulationControls](#) () const  
*Returns the simulation controls owned by this data definition.*
- TraceManager::Level [getTraceLevelSpecific](#) () const  
*Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) (TraceManager::Level level)  
*Updates the trace level used by this definition.*
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)  
*Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const  
*Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)  
*Enables or disables the specific trace level.*

### Static Public Member Functions

- static PluginInformation \* [GetPluginInformation](#) ()
- static ModelComponent \* [LoadInstance](#) (Model \*model, PersistenceRecord \*fields)
- static ModelDataDefinition \* [NewInstance](#) (Model \*model, std::string name="")

## Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool [Check](#) ([ModelComponent](#) \*component)  
*Validates a component instance.*
- static void [CreateInternalData](#) ([ModelComponent](#) \*component)  
*Creates internal data required by a component instance.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelComponent](#) \*component)  
*Serializes a component instance into a persistence record.*
- static void [DispatchEvent](#) ([Event](#) \*event)  
*Dispatches a simulation event to the target component.*

## Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool [Check](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*
- static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

## Protected Member Functions

- virtual void [\\_onDispatchEvent](#) ([Entity](#) \*entity, unsigned int inputPortNumber) override  
*Handles a dispatched event for this component type.*
- virtual bool [\\_loadInstance](#) ([PersistenceRecord](#) \*fields) override  
*virtual protected method that must be overridden*
- virtual void [\\_saveInstance](#) ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*
- virtual bool [\\_check](#) (std::string &errorMessage) override  
*could be overridden by derived classes*

## Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [\\_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) \* [\\_getParserChangesInformation](#) ()
- virtual void [\\_initBetweenReplications](#) ()
- virtual void [\\_addSimulationControl](#) ([SimulationControl](#) \*control)
- virtual void [\\_createInternalAndAttachedData](#) ()
- virtual void [\\_createInternalStatisticReporters](#) ()
- virtual void [\\_createAttachedAttributes](#) ()
- virtual void [\\_createNonEditableDataDefinitions](#) ()
- virtual void [\\_createEditableDataDefinitions](#) ()
- void [\\_templateCreateInternalStatisticReporters](#) ()



- void **\_templateCreateAttachedAttributes** ()
- void **\_templateCreateNonEditableDataDefinitions** ()
- void **\_templateCreateEditableDataDefinitions** ()
- void **\_internaStatisticReportersClear** ()
- legacy helper methods used by the granular create\* hooks*
- void **\_internalDataClear** ()
- void **\_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) \*child)
- void **\_internalDataRemove** (const std::string &key)
- void **\_attachedDataClear** ()
- void **\_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_attachedDataRemove** (const std::string &key)
- void **\_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **\_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_statisticReporterRemove** (const std::string &key)
- void **\_statisticReportersClear** ()
- void **\_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **\_mandatoryAttachedAttributesClear** ()
- void **\_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryEditableDataDefinitionsClear** ()
- void **\_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalEditableDataDefinitionsClear** ()
- void **\_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryNonEditableDataDefinitionsClear** ()
- void **\_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalNonEditableDataDefinitionsClear** ()
- void **\_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
- method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1\\_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2\\_results](#))
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))
- void [traceSimulation](#) (void \*thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))
- Traces text to the simulation output.*
- void **traceSimulation** (void \*thisobject, [TraceManager::Level](#) level, const std::string &text)

### Additional Inherited Members

### Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) \* **\_connections** = new [ConnectionManager](#)()
- virtual protected methods that could be overridden by derived classes, if needed*

## Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **\_id**
- std::string **\_typename**
- bool **\_reportStatistics**
- bool **\_hasChanged**
- unsigned int **\_modelLevel** = 0
- [Model](#) \* **\_parentModel**
- [List](#)< [SimulationControl](#) \* > \* **\_simulationControls** = new [List](#)<[SimulationControl](#)\*>()
- TraceManager::Level **\_traceLevelSpecific**
- bool **\_traceLevelSpecificEnabled** = false

### 15.954.1 Detailed Description

This component ...

### 15.954.2 Member Function Documentation

#### 15.954.2.1 `_check()`

```
bool Submodel::_check (
 std::string & errorMessage) [override], [protected], [virtual]
```

could be overridden by derived classes  
Reimplemented from [ModelDataDefinition](#).

#### 15.954.2.2 `_loadInstance()`

```
bool Submodel::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

virtual protected method that must be overridden  
Reimplemented from [ModelComponent](#).

#### 15.954.2.3 `_onDispatchEvent()`

```
void Submodel::_onDispatchEvent (
 Entity * entity,
 unsigned int inputPortNumber) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.  
This method is only for ModelComponents, not for ModelDataElements.

#### Parameters

|                        |                                         |
|------------------------|-----------------------------------------|
| <i>entity</i>          | <a href="#">Entity</a> being processed. |
| <i>inputPortNumber</i> | Input port where the entity arrived.    |

Implements [ModelComponent](#).

#### 15.954.2.4 `_saveInstance()`

```
void Submodel::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

#### Parameters

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [ModelComponent](#).

#### 15.954.2.5 show()

```
std::string Submodel::show () [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

##### Returns

Human-readable description used for tracing and debugging.

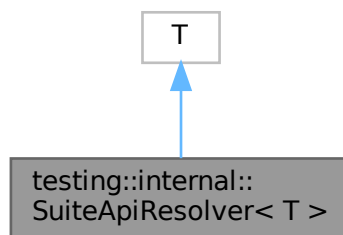
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

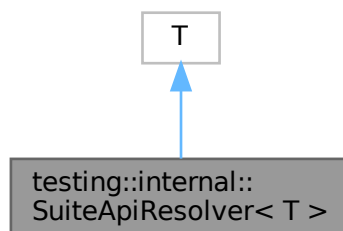
- plugins/components/ModalModel/Submodel.h
- plugins/components/ModalModel/Submodel.cpp

## 15.955 testing::internal::SuiteApiResolver< T > Struct Template Reference

Inheritance diagram for testing::internal::SuiteApiResolver< T >:



Collaboration diagram for testing::internal::SuiteApiResolver< T >:



### Public Types

- using **Test** = typename std::conditional< sizeof(T) !=0, [::testing::Test](#), void >::type

### Static Public Member Functions

- static SetUpTearDownSuiteFuncType **GetSetUpCaseOrSuite** (const char \*filename, int line\_num)
- static SetUpTearDownSuiteFuncType **GetTearDownCaseOrSuite** (const char \*filename, int line\_num)

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-internal.h

## 15.956 yy::genesyspp\_parser::symbol\_kind Struct Reference

Symbol kinds.

```
#include <parser/GenesysParser.h>
```

### Public Types

- enum [symbol\\_kind\\_type](#) {  
[YYNTOKENS](#) = 95, [S\\_YYEMPTY](#) = -2, [S\\_YYEOF](#) = 0, [S\\_YYerror](#) = 1,  
[S\\_YYUNDEF](#) = 2, [S\\_NUMD](#) = 3, [S\\_NUMH](#) = 4, [S\\_CTEZERO](#) = 5,  
[S\\_oLE](#) = 6, [S\\_oGE](#) = 7, [S\\_oEQ](#) = 8, [S\\_oNE](#) = 9,  
[S\\_oAND](#) = 10, [S\\_oOR](#) = 11, [S\\_oNAND](#) = 12, [S\\_oXOR](#) = 13,  
[S\\_oNOT](#) = 14, [S\\_fSIN](#) = 15, [S\\_fCOS](#) = 16, [S\\_fROUND](#) = 17,  
[S\\_fMOD](#) = 18, [S\\_fTRUNC](#) = 19, [S\\_fFRAC](#) = 20, [S\\_fEXP](#) = 21,  
[S\\_fSQRT](#) = 22, [S\\_fLOG](#) = 23, [S\\_fLN](#) = 24, [S\\_mathMIN](#) = 25,  
[S\\_mathMAX](#) = 26, [S\\_fVAL](#) = 27, [S\\_fEVAL](#) = 28, [S\\_fLENG](#) = 29,  
[S\\_fRND1](#) = 30, [S\\_fEXPO](#) = 31, [S\\_fNORM](#) = 32, [S\\_fUNIF](#) = 33,  
[S\\_fWEIB](#) = 34, [S\\_fLOGN](#) = 35, [S\\_fGAMM](#) = 36, [S\\_fERLA](#) = 37,  
[S\\_fTRIA](#) = 38, [S\\_fBETA](#) = 39, [S\\_fDISC](#) = 40, [S\\_fTNOW](#) = 41,  
[S\\_fTFIN](#) = 42, [S\\_fMAXREP](#) = 43, [S\\_fNUMREP](#) = 44, [S\\_fIDENT](#) = 45,  
[S\\_simulEntitiesWIP](#) = 46, [S\\_cIF](#) = 47, [S\\_cELSE](#) = 48, [S\\_cFOR](#) = 49,  
[S\\_cTO](#) = 50, [S\\_cDO](#) = 51, [S\\_ATTRIB](#) = 52, [S\\_CSTAT](#) = 53,  
[S\\_COUNTER](#) = 54, [S\\_SIMRESP](#) = 55, [S\\_SIMCTRL](#) = 56, [S\\_fTAVG](#) = 57,  
[S\\_fCOUNT](#) = 58, [S\\_ILLEGAL](#) = 59, [S\\_RESOURCE](#) = 60, [S\\_fNR](#) = 61,  
[S\\_fMR](#) = 62, [S\\_fIRF](#) = 63, [S\\_fRESSEIZES](#) = 64, [S\\_fSTATE](#) = 65,  
[S\\_fSETSUM](#) = 66, [S\\_fRESUTIL](#) = 67, [S\\_QUEUE](#) = 68, [S\\_fNQ](#) = 69,  
[S\\_fFIRSTINQ](#) = 70, [S\\_fLASTINQ](#) = 71, [S\\_fSAQUE](#) = 72, [S\\_fAQUE](#) = 73,  
[S\\_fENTATRANK](#) = 74, [S\\_SET](#) = 75, [S\\_fNUMSET](#) = 76, [S\\_VARI](#) = 77,  
[S\\_FORM](#) = 78, [S\\_fNUMGR](#) = 79, [S\\_fATRGR](#) = 80, [S\\_LPAREN](#) = 81,  
[S\\_RPAREN](#) = 82, [S\\_LBRACKET](#) = 83, [S\\_RBRACKET](#) = 84, [S\\_PLUS](#) = 85,  
[S\\_MINUS](#) = 86, [S\\_STAR](#) = 87, [S\\_POWER](#) = 88, [S\\_SLASH](#) = 89,  
[S\\_LESS](#) = 90, [S\\_GREATER](#) = 91, [S\\_ASSIGN](#) = 92, [S\\_COMMA](#) = 93,  
[S\\_94\\_USER](#) = 94, [S\\_YYACCEPT](#) = 95, [S\\_input](#) = 96, [S\\_expression](#) = 97,  
[S\\_logicalOr](#) = 98, [S\\_logicalXor](#) = 99, [S\\_logicalAnd](#) = 100, [S\\_logicalNot](#) = 101,  
[S\\_relational](#) = 102, [S\\_additive](#) = 103, [S\\_multiplicative](#) = 104, [S\\_power](#) = 105,  
[S\\_unary](#) = 106, [S\\_primary](#) = 107, [S\\_number](#) = 108, [S\\_command](#) = 109,  
[S\\_commandIF](#) = 110, [S\\_commandFOR](#) = 111, [S\\_function](#) = 112, [S\\_kernelFunction](#) = 113,  
[S\\_elementFunction](#) = 114, [S\\_trigonFunction](#) = 115, [S\\_mathFunction](#) = 116, [S\\_probFunction](#) = 117,  
[S\\_userFunction](#) = 118, [S\\_listaparm](#) = 119, [S\\_indexList](#) = 120, [S\\_illegal](#) = 121,  
[S\\_attribute](#) = 122, [S\\_simulationResponse](#) = 123, [S\\_simulationControl](#) = 124, [S\\_variable](#) = 125,  
[S\\_formula](#) = 126, [S\\_assignment](#) = 127, [S\\_pluginFunction](#) = 128 }

### 15.956.1 Detailed Description

Symbol kinds.

### 15.956.2 Member Enumeration Documentation

#### 15.956.2.1 symbol\_kind\_type

```
enum yy::genesyspp_parser::symbol_kind::symbol_kind_type
```

## Enumerator

|           |                   |
|-----------|-------------------|
| YYNTOKENS | Number of tokens. |
|-----------|-------------------|

The documentation for this struct was generated from the following file:

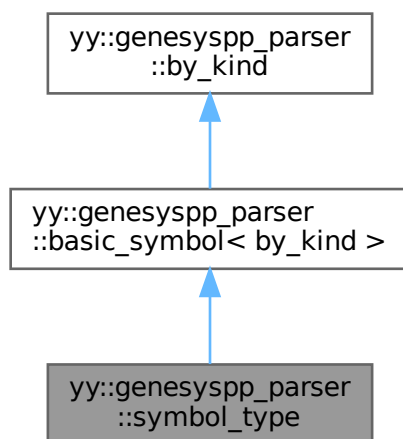
- parser/[GenesysParser.h](#)

## 15.957 yy::genesyspp\_parser::symbol\_type Struct Reference

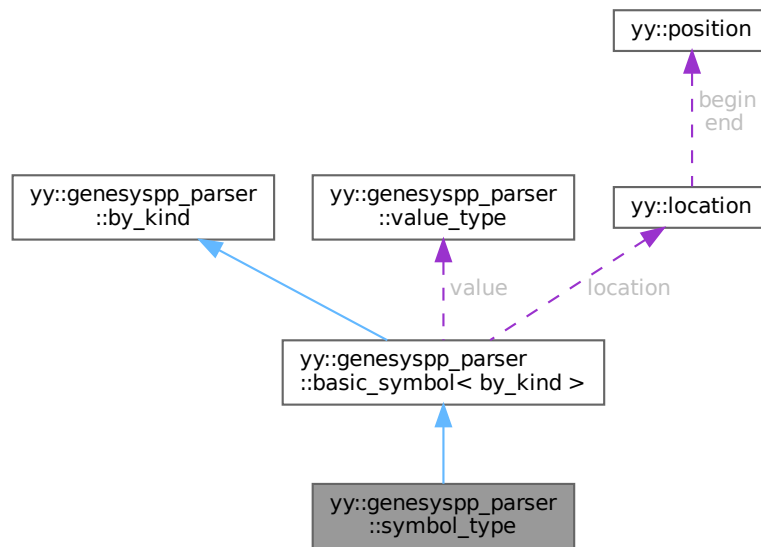
"External" symbols: returned by the scanner.

```
#include <parser/GenesysParser.h>
```

Inheritance diagram for yy::genesyspp\_parser::symbol\_type:



Collaboration diagram for `yy::genesyspp_parser::symbol_type`:



## Public Types

- typedef `basic_symbol< by_kind > super_type`  
*Superclass.*

## Public Types inherited from `yy::genesyspp_parser::basic_symbol< by_kind >`

- typedef `by_kind super_type`  
*Alias to Base.*

## Public Types inherited from `yy::genesyspp_parser::by_kind`

- typedef `token_kind_type kind_type`  
*The symbol kind as needed by the constructor.*

## Public Member Functions

- `symbol_type () YY_NOEXCEPT`  
*Empty symbol.*
- `symbol_type (int tok, const location_type &l)`  
*Constructor for valueless symbols, and symbols from each type.*
- `symbol_type (int tok, const obj_t &v, const location_type &l)`

## Public Member Functions inherited from `yy::genesyspp_parser::basic_symbol< by_kind >`

- `basic_symbol () YY_NOEXCEPT`  
*Default constructor.*
- `basic_symbol (const basic_symbol &that)`  
*Copy constructor.*

- **basic\_symbol** (typename Base::kind\_type t, const [location\\_type](#) &l)  
*Constructors for typed symbols.*
- **basic\_symbol** (typename Base::kind\_type t, const [obj\\_t](#) &v, const [location\\_type](#) &l)
- **basic\_symbol** (typename Base::kind\_type t, const std::string &v, const [location\\_type](#) &l)
- **~basic\_symbol** ()  
*Destroy the symbol.*
- void **clear** () YY\_NOEXCEPT  
*Destroy contents, and record that is empty.*
- std::string **name** () const YY\_NOEXCEPT  
*The user-facing name of this symbol.*
- [symbol\\_kind\\_type](#) **type\_get** () const YY\_NOEXCEPT  
*Backward compatibility (Bison 3.6).*
- bool **empty** () const YY\_NOEXCEPT  
*Whether empty.*
- void **move** ([basic\\_symbol](#) &s)  
*Destructive move, s is emptied into this.*

### Public Member Functions inherited from [yy::genesyspp\\_parser::by\\_kind](#)

- **by\_kind** () YY\_NOEXCEPT  
*Default constructor.*
- **by\_kind** (const [by\\_kind](#) &that) YY\_NOEXCEPT  
*Copy constructor.*
- **by\_kind** ([kind\\_type](#) t) YY\_NOEXCEPT  
*Constructor from (external) token numbers.*
- void **clear** () YY\_NOEXCEPT  
*[Record](#) that this symbol is empty.*
- void **move** ([by\\_kind](#) &that)  
*Steal the symbol kind from that.*
- [symbol\\_kind\\_type](#) **kind** () const YY\_NOEXCEPT
- [symbol\\_kind\\_type](#) **type\_get** () const YY\_NOEXCEPT  
*Backward compatibility (Bison 3.6).*

### Additional Inherited Members

### Public Attributes inherited from [yy::genesyspp\\_parser::basic\\_symbol](#)< [by\\_kind](#) >

- [value\\_type](#) **value**  
*The semantic value.*
- [location\\_type](#) **location**  
*The location.*

### Public Attributes inherited from [yy::genesyspp\\_parser::by\\_kind](#)

- [symbol\\_kind\\_type](#) **kind\_**

## 15.957.1 Detailed Description

"External" symbols: returned by the scanner.

The documentation for this struct was generated from the following file:

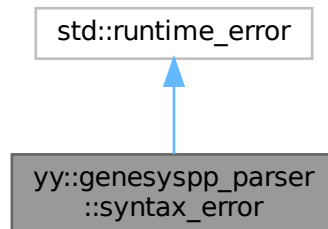
- parser/[GenesysParser.h](#)

## 15.958 yy::genesyspp\_parser::syntax\_error Struct Reference

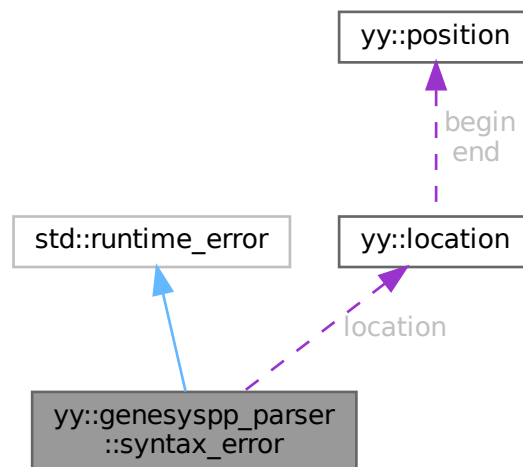
Syntax errors thrown from user actions.

```
#include <parser/GenesysParser.h>
```

Inheritance diagram for yy::genesyspp\_parser::syntax\_error:



Collaboration diagram for yy::genesyspp\_parser::syntax\_error:



### Public Member Functions

- **syntax\_error** (const [location\\_type](#) &l, const std::string &m)
- **syntax\_error** (const [syntax\\_error](#) &s)

### Public Attributes

- [location\\_type](#) **location**

### 15.958.1 Detailed Description

Syntax errors thrown from user actions.



The documentation for this struct was generated from the following files:

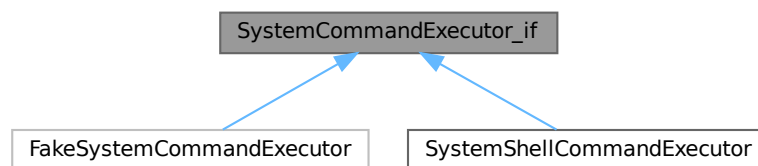
- parser/[GenesysParser.h](#)
- parser/[GenesysParser.cpp](#)

## 15.959 SystemCommandExecutor\_if Class Reference

Minimal command execution interface used to make system dependency checks testable.

```
#include <kernel/simulator/SystemDependencyResolver.h>
```

Inheritance diagram for SystemCommandExecutor\_if:



### Public Member Functions

- virtual `~SystemCommandExecutor_if()`=default  
*Destroys the command executor abstraction.*
- virtual `SystemCommandResult run` (const std::string &command)=0  
*Runs a shell command and returns its observable result.*

### 15.959.1 Detailed Description

Minimal command execution interface used to make system dependency checks testable.

### 15.959.2 Member Function Documentation

#### 15.959.2.1 run()

```
virtual SystemCommandResult SystemCommandExecutor_if::run (
 const std::string & command) [pure virtual]
```

Runs a shell command and returns its observable result.

Implemented in [SystemShellCommandExecutor](#), and [FakeSystemCommandExecutor](#).

The documentation for this class was generated from the following file:

- kernel/simulator/SystemDependencyResolver.h

## 15.960 SystemCommandResult Struct Reference

Captures the result of executing a system command used by dependency checks.

```
#include <kernel/simulator/SystemDependencyResolver.h>
```

### Public Member Functions

- bool `succeeded` () const  
*Returns true when the command started and exited successfully.*

## Public Attributes

- bool **started** = false  
*True when the command process was started by the executor.*
- int **exitCode** = -1  
*Process exit code, or -1 when the command could not be started.*
- std::string **output** = ""  
*Combined standard output and diagnostic text produced by the command.*
- std::string **errorMessage** = ""  
*Internal executor error message, when starting or collecting the command failed.*

### 15.960.1 Detailed Description

Captures the result of executing a system command used by dependency checks.  
The documentation for this struct was generated from the following files:

- kernel/simulator/SystemDependencyResolver.h
- kernel/simulator/SystemDependencyResolver.cpp

## 15.961 SystemDependency Class Reference

### Public Types

- enum class **OS** {  
    **Any** , **Linux** , **Windows** , **MacOS** ,  
    **Unknown** }

### Public Member Functions

- **SystemDependency** (OS os, std::string name, std::string installCommand="", std::string checkCommand="")
- OS **getOS** () const
- std::string **getName** () const
- std::string **getInstallCommand** () const
- std::string **getCheckCommand** () const
- std::string **show** () const

### Static Public Member Functions

- static std::string **osToString** (OS os)

The documentation for this class was generated from the following files:

- kernel/simulator/PluginInformation.h
- kernel/simulator/PluginInformation.cpp

## 15.962 SystemDependencyCheckEntry Class Reference

One dependency entry produced by a system dependency preflight.

```
#include <kernel/simulator/SystemDependencyResolver.h>
```

### Public Types

- enum class **Status** { **Satisfied** , **Missing** , **NotVerifiable** , **IgnoredDifferentOS** }  
*Classification assigned to one declared dependency after preflight.*

### Public Member Functions

- **SystemDependencyCheckEntry** ([SystemDependency](#) dependency, [Status](#) status, [SystemCommandResult](#) checkResult={}, std::string message="")  
*Builds a preflight entry for one declared dependency.*
- const [SystemDependency](#) & **dependency** () const  
*Returns the dependency declaration evaluated by this entry.*
- [Status](#) **status** () const  
*Returns the preflight status for this dependency.*
- const [SystemCommandResult](#) & **checkResult** () const  
*Returns the result of the check command, when one was executed.*
- const std::string & **message** () const  
*Returns human-readable diagnostic context for this entry.*
- bool **blocksInsertion** () const  
*Returns true when this entry should prevent plugin insertion.*
- bool **canAttemptInstall** () const  
*Returns true when this missing dependency has an installation command.*
- std::string **diagnosticText** () const  
*Returns a multi-line diagnostic with check/install commands and command results.*

### Static Public Member Functions

- static std::string **statusToString** ([Status](#) status)  
*Converts a status value into a stable textual representation.*

### 15.962.1 Detailed Description

One dependency entry produced by a system dependency preflight.  
The documentation for this class was generated from the following files:

- kernel/simulator/SystemDependencyResolver.h
- kernel/simulator/SystemDependencyResolver.cpp

## 15.963 SystemDependencyCheckResult Class Reference

Full preflight result for the system dependencies declared by one plugin.  
`#include <kernel/simulator/SystemDependencyResolver.h>`

### Public Member Functions

- void **add** ([SystemDependencyCheckEntry](#) entry)  
*Adds one dependency evaluation entry.*
- const std::list< [SystemDependencyCheckEntry](#) > & **entries** () const  
*Returns every evaluated dependency entry, including ignored entries.*
- bool **canInsertPlugin** () const  
*Returns true when no applicable dependency blocks insertion.*
- bool **hasBlockingEntries** () const  
*Returns true when at least one dependency blocks insertion.*
- bool **canAttemptInstallForAllMissing** () const  
*Returns true when every blocking missing dependency has an installation command.*
- std::string **summary** () const  
*Returns a compact diagnostic summary suitable for trace logs.*
- std::string **diagnosticText** (bool includeSatisfied=false) const  
*Returns a multi-line diagnostic suitable for terminal, GUI and trace details.*

### 15.963.1 Detailed Description

Full preflight result for the system dependencies declared by one plugin.  
The documentation for this class was generated from the following files:

- kernel/simulator/SystemDependencyResolver.h
- kernel/simulator/SystemDependencyResolver.cpp

## 15.964 SystemDependencyInstallEntry Class Reference

One command result from an attempted system dependency installation.  
`#include <kernel/simulator/SystemDependencyResolver.h>`

### Public Member Functions

- **SystemDependencyInstallEntry** ([SystemDependency dependency](#), [SystemCommandResult installResult](#))  
*Builds an installation result for a single dependency.*
- const [SystemDependency](#) & **dependency** () const  
*Returns the dependency whose install command was executed.*
- const [SystemCommandResult](#) & **installResult** () const  
*Returns the result produced by the install command.*
- std::string **diagnosticText** () const  
*Returns command, exit code and captured output for trace or GUI feedback.*

### 15.964.1 Detailed Description

One command result from an attempted system dependency installation.  
The documentation for this class was generated from the following files:

- kernel/simulator/SystemDependencyResolver.h
- kernel/simulator/SystemDependencyResolver.cpp

## 15.965 SystemDependencyInstallResult Class Reference

Aggregates commands executed while trying to satisfy missing system dependencies.  
`#include <kernel/simulator/SystemDependencyResolver.h>`

### Public Member Functions

- void **add** ([SystemDependencyInstallEntry entry](#))  
*Adds one installation command result.*
- const std::list< [SystemDependencyInstallEntry](#) > & **entries** () const  
*Returns all attempted installation command results.*
- bool **succeeded** () const  
*Returns true when every attempted installation command succeeded.*
- std::string **summary** () const  
*Returns a compact diagnostic summary suitable for trace logs.*
- std::string **diagnosticText** () const  
*Returns detailed diagnostic text for every executed install command.*

### 15.965.1 Detailed Description

Aggregates commands executed while trying to satisfy missing system dependencies.  
The documentation for this class was generated from the following files:

- kernel/simulator/SystemDependencyResolver.h
- kernel/simulator/SystemDependencyResolver.cpp

## 15.966 SystemDependencyResolver Class Reference

Evaluates and optionally installs plugin system dependencies.

```
#include <kernel/simulator/SystemDependencyResolver.h>
```

### Static Public Member Functions

- static SystemDependency::OS **currentOS** ()  
*Returns the operating system represented by the current executable.*
- static bool **appliesToCurrentOS** (const SystemDependency &dependency)  
*Returns true when a dependency declaration applies to the current OS.*
- static SystemDependencyCheckResult **evaluate** (const std::list< SystemDependency > \*dependencies, SystemCommandExecutor\_if &executor)  
*Evaluates all dependency declarations using check commands when available.*
- static SystemDependencyInstallResult **installMissingDependencies** (const SystemDependencyCheckResult &checkResult, SystemCommandExecutor\_if &executor)  
*Executes install commands for missing dependencies from a preflight result.*

### 15.966.1 Detailed Description

Evaluates and optionally installs plugin system dependencies.

The resolver deliberately contains no user-interface code. It only decides whether dependencies apply to the current OS, runs declared check/install commands through an executor, and reports diagnostics to the caller.

The documentation for this class was generated from the following files:

- kernel/simulator/SystemDependencyResolver.h
- kernel/simulator/SystemDependencyResolver.cpp

## 15.967 SystemPreferences Class Reference

Persistent application preferences used by the Qt GUI.

```
#include <applications/gui/genesys/systempreferences.h>
```

### Public Types

- enum class StartupModelMode : unsigned short { **NoModel** = 0 , **NewModel** = 1 , **OpenSpecificModel** = 2 , **OpenLastModel** = 3 }  
*Startup model selection remembered in the preferences file.*
- enum class VisualTheme { **Light** , **Dark** }  
*Global visual style used by the GUI.*
- enum class InterfaceStyle { **Classic** , **Modern** }  
*Interface density/style preset used by the Qt application.*
- enum class AutomaticPositioningStrategy { **Legacy** = 0 , **Centered** = 1 }  
*Automatic positioning policy for graphical elements.*

### Static Public Member Functions

- static bool **load** ()  
*Loads preferences from the configuration file.*
- static bool **save** ()  
*Saves the current preferences to disk.*
- static void **resetToDefaults** ()  
*Restores every preference to its built-in default value.*
- static QString **configFilePath** ()  
*Returns the full path to the active preferences file.*

- static bool **startMaximized** ()  
*Returns whether the application should start maximized.*
- static void **setStartMaximized** (bool newStartMaximized)  
*Updates the start-maximized preference.*
- static bool **autoLoadPlugins** ()  
*Returns whether plug-ins are loaded automatically at startup.*
- static void **setAutoLoadPlugins** (bool newAutoLoadPlugins)  
*Updates the automatic plug-in loading preference.*
- static [StartupModelMode](#) **startupModelMode** ()  
*Returns the startup model mode.*
- static void **setStartupModelMode** ([StartupModelMode](#) newStartupModelMode)  
*Updates the startup model mode.*
- static unsigned short **modelAtStart** ()  
*Returns the configured model index for the startup flow.*
- static void **setModelAtStart** (unsigned short newModelAtStart)  
*Updates the configured model index for the startup flow.*
- static std::string **modelfilename** ()  
*Returns the filename requested for startup loading.*
- static void **setModelfilename** (const std::string &newModelfilename)  
*Updates the filename requested for startup loading.*
- static std::string **lastModelFilename** ()  
*Returns the last model filename successfully opened by the GUI.*
- static void **setLastModelFilename** (const std::string &newLastModelFilename)  
*Updates the last model filename successfully opened by the GUI.*
- static std::vector< std::string > **recentModelFiles** ()  
*Returns the list of recently opened model files.*
- static void **setRecentModelFiles** (const std::vector< std::string > &[recentModelFiles](#))  
*Replaces the recent model file list.*
- static void [pushRecentModelFile](#) (const std::string &modelFilename)  
*Pushes a file to the recent model list with limit handling.*
- static unsigned int **recentModelFilesLimit** ()  
*Returns the maximum number of recent model entries to keep.*
- static void **setRecentModelFilesLimit** (unsigned int [recentModelFilesLimit](#))  
*Updates the maximum number of recent model entries.*
- static bool **checkSystemPackagesAtStart** ()  
*Returns whether system packages should be checked at startup.*
- static void **setCheckSystemPackagesAtStart** (bool newCheckSystemPackagesAtStart)  
*Updates the system-package check preference.*
- static TraceManager::Level **traceLevel** ()  
*Returns the current trace verbosity level.*
- static void **setTraceLevel** (TraceManager::Level newTraceLevel)  
*Updates the trace verbosity level.*
- static [VisualTheme](#) **visualTheme** ()  
*Returns the selected visual theme.*
- static void **setVisualTheme** ([VisualTheme](#) newVisualTheme)  
*Updates the selected visual theme.*
- static std::string **visualThemeName** ()  
*Returns the persisted name of the selected visual theme.*
- static [InterfaceStyle](#) **interfaceStyle** ()  
*Returns the selected interface style preset.*
- static void **setInterfaceStyle** ([InterfaceStyle](#) newInterfaceStyle)

- Updates the selected interface style preset.*
- static std::string **interfaceStyleName** ()  
*Returns the persisted name of the selected interface style.*
- static int **applicationFontSize** ()  
*Returns the application font size used by the GUI.*
- static void **setApplicationFontSize** (int newApplicationFontSize)  
*Updates the application font size used by the GUI.*
- static bool **diagramUsesThemeColors** ()  
*Returns whether diagram widgets should derive colors from the theme.*
- static void **setDiagramUsesThemeColors** (bool newDiagramUsesThemeColors)  
*Updates whether diagram widgets should derive colors from the theme.*
- static [AutomaticPositioningStrategy](#) **automaticPositioningStrategy** ()  
*Returns the automatic placement strategy used by the diagram canvas.*
- static void **setAutomaticPositioningStrategy** ([AutomaticPositioningStrategy](#) strategy)  
*Updates the automatic placement strategy used by the diagram canvas.*
- static unsigned int **canvasBackgroundColor** ()  
*Returns the RGB color used for the main canvas background.*
- static unsigned int **canvasDisabledBackgroundColor** ()  
*Returns the RGB color used for the disabled canvas background.*
- static unsigned int **gridColor** ()  
*Returns the RGB color used for the diagram grid.*

### 15.967.1 Detailed Description

Persistent application preferences used by the Qt GUI.

The class behaves like a singleton configuration store backed by a local preferences file. All access is static so the GUI can query and update the current runtime settings without manually passing a preferences object around.

### 15.967.2 Member Function Documentation

#### 15.967.2.1 load()

```
bool SystemPreferences::load () [static]
```

Loads preferences from the configuration file.

Loads GUI preferences from disk and normalizes invalid data.

#### 15.967.2.2 pushRecentModelFile()

```
void SystemPreferences::pushRecentModelFile (
 const std::string & modelName) [static]
```

Pushes a file to the recent model list with limit handling.

Pushes a model file to the top of the recent-file list.

#### 15.967.2.3 save()

```
bool SystemPreferences::save () [static]
```

Saves the current preferences to disk.

Serializes the current GUI preferences to the preferences file.

The documentation for this class was generated from the following files:

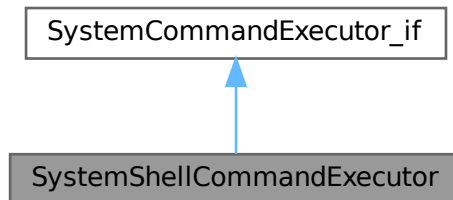
- applications/gui/genesys/systempreferences.h
- applications/gui/genesys/systempreferences.cpp

## 15.968 SystemShellCommandExecutor Class Reference

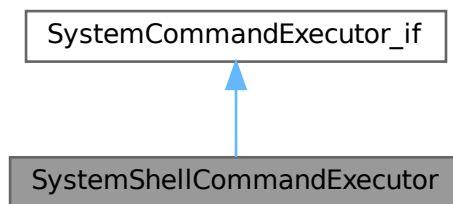
Shell-based executor used by production plugin dependency preflight.

```
#include <kernel/simulator/SystemDependencyResolver.h>
```

Inheritance diagram for SystemShellCommandExecutor:



Collaboration diagram for SystemShellCommandExecutor:



### Public Member Functions

- [SystemCommandResult run](#) (const std::string &command) override  
*Executes a command through the host shell and captures combined output.*

### Public Member Functions inherited from [SystemCommandExecutor\\_if](#)

- virtual `~SystemCommandExecutor_if()`=default  
*Destroys the command executor abstraction.*

#### 15.968.1 Detailed Description

Shell-based executor used by production plugin dependency preflight.

#### 15.968.2 Member Function Documentation

##### 15.968.2.1 run()

```
SystemCommandResult SystemShellCommandExecutor::run (
 const std::string & command) [override], [virtual]
```

Executes a command through the host shell and captures combined output.



Implements [SystemCommandExecutor\\_if](#).

The documentation for this class was generated from the following files:

- kernel/simulator/SystemDependencyResolver.h
- kernel/simulator/SystemDependencyResolver.cpp

## 15.969 testing::internal::Templates< Head\_, Tail\_ > Struct Template Reference

### Public Types

- using **Head** = [TemplateSel](#)< Head\_ >
- using **Tail** = [Templates](#)< Tail\_... >

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-type-util.h

## 15.970 testing::internal::Templates< Head\_ > Struct Template Reference

### Public Types

- using **Head** = [TemplateSel](#)< Head\_ >
- using **Tail** = [None](#)

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-type-util.h

## 15.971 testing::internal::TemplateSel< Tmpl > Struct Template Reference

### Classes

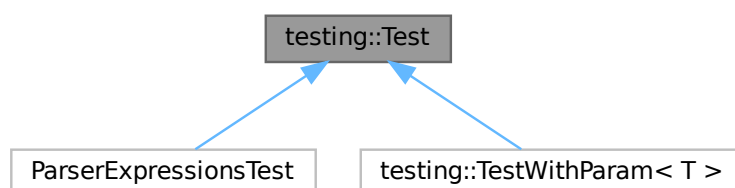
- struct [Bind](#)

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-type-util.h

## 15.972 testing::Test Class Reference

Inheritance diagram for testing::Test:



**Static Public Member Functions**

- static void **SetUpTestSuite** ()
- static void **TearDownTestSuite** ()
- static void **TearDownTestCase** ()
- static void **SetUpTestCase** ()
- static bool **HasFatalFailure** ()
- static bool **HasNonfatalFailure** ()
- static bool **IsSkipped** ()
- static bool **HasFailure** ()
- static void **RecordProperty** (const std::string &key, const std::string &value)
- static void **RecordProperty** (const std::string &key, int value)

**Protected Member Functions**

- virtual void **SetUp** ()
- virtual void **TearDown** ()

**Friends**

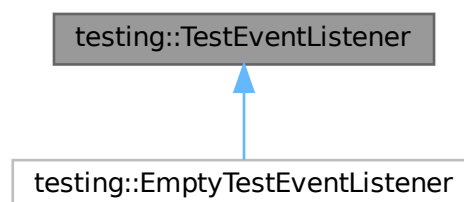
- class **TestInfo**

The documentation for this class was generated from the following files:

- gtest/gtest/gtest.h
- gtest/src/gtest.cc

**15.973 testing::TestEventListener Class Reference**

Inheritance diagram for testing::TestEventListener:

**Public Member Functions**

- virtual void **OnTestProgramStart** (const [UnitTest](#) &unit\_test)=0
- virtual void **OnTestIterationStart** (const [UnitTest](#) &unit\_test, int iteration)=0
- virtual void **OnEnvironmentsSetUpStart** (const [UnitTest](#) &unit\_test)=0
- virtual void **OnEnvironmentsSetUpEnd** (const [UnitTest](#) &unit\_test)=0
- virtual void **OnTestSuiteStart** (const [TestSuite](#) &)
- virtual void **OnTestCaseStart** (const [TestCase](#) &)
- virtual void **OnTestStart** (const [TestInfo](#) &test\_info)=0
- virtual void **OnTestDisabled** (const [TestInfo](#) &)
- virtual void **OnTestPartResult** (const TestPartResult &test\_part\_result)=0
- virtual void **OnTestEnd** (const [TestInfo](#) &test\_info)=0

- virtual void **OnTestSuiteEnd** (const [TestSuite](#) &)
- virtual void **OnTestCaseEnd** (const [TestCase](#) &)
- virtual void **OnEnvironmentsTearDownStart** (const [UnitTest](#) &unit\_test)=0
- virtual void **OnEnvironmentsTearDownEnd** (const [UnitTest](#) &unit\_test)=0
- virtual void **OnTestIterationEnd** (const [UnitTest](#) &unit\_test, int iteration)=0
- virtual void **OnTestProgramEnd** (const [UnitTest](#) &unit\_test)=0

The documentation for this class was generated from the following file:

- gtest/gtest/gtest.h

## 15.974 testing::TestEventListeners Class Reference

### Public Member Functions

- void **Append** ([TestEventListener](#) \*listener)
- [TestEventListener](#) \* **Release** ([TestEventListener](#) \*listener)
- [TestEventListener](#) \* **default\_result\_printer** () const
- [TestEventListener](#) \* **default\_xml\_generator** () const

### Friends

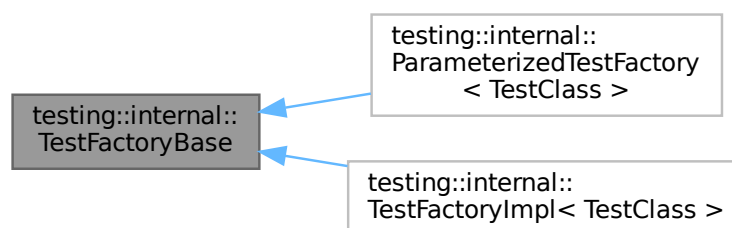
- class **TestSuite**
- class **TestInfo**
- class **internal::DefaultGlobalTestPartResultReporter**
- class **internal::NoExecDeathTest**
- class **internal::TestEventListenersAccessor**
- class **internal::UnitTestImpl**

The documentation for this class was generated from the following files:

- gtest/gtest/gtest.h
- gtest/src/gtest.cc

## 15.975 testing::internal::TestFactoryBase Class Reference

Inheritance diagram for testing::internal::TestFactoryBase:



### Public Member Functions

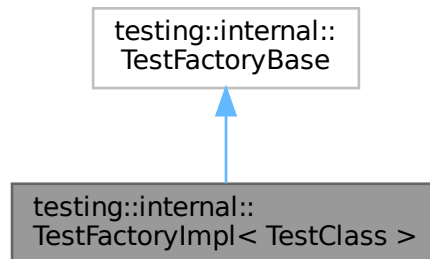
- virtual [Test](#) \* **CreateTest** ()=0

The documentation for this class was generated from the following file:

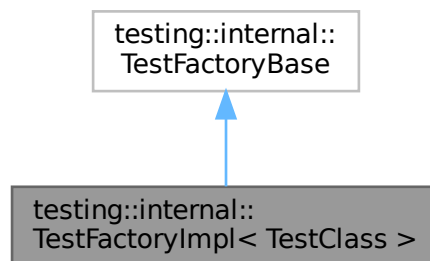
- gtest/gtest/internal/gtest-internal.h

## 15.976 testing::internal::TestFactoryImpl< TestClass > Class Template Reference

Inheritance diagram for testing::internal::TestFactoryImpl< TestClass >:



Collaboration diagram for testing::internal::TestFactoryImpl< TestClass >:



### Public Member Functions

- [Test \\* CreateTest](#) () override

### 15.976.1 Member Function Documentation

#### 15.976.1.1 CreateTest()

```
template<class TestClass >
Test * testing::internal::TestFactoryImpl< TestClass >::CreateTest () [inline], [override],
[virtual]
```

Implements [testing::internal::TestFactoryBase](#).

The documentation for this class was generated from the following file:

- gtest/gtest/internal/gtest-internal.h

## 15.977 testing::TestInfo Class Reference

### Public Member Functions

- const char \* **test\_suite\_name** () const
- const char \* **test\_case\_name** () const
- const char \* **name** () const
- const char \* **type\_param** () const
- const char \* **value\_param** () const
- const char \* **file** () const
- int **line** () const
- bool **is\_in\_another\_shard** () const
- bool **should\_run** () const
- bool **is\_reportable** () const
- const [TestResult](#) \* **result** () const

### Friends

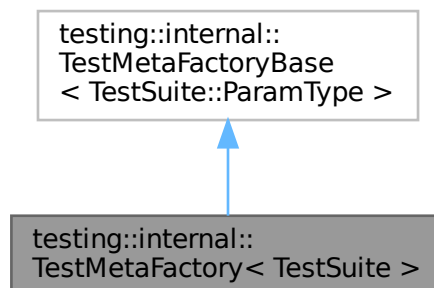
- class **Test**
- class **TestSuite**
- class **internal::UnitTestImpl**
- class **internal::StreamingListenerTest**
- [TestInfo](#) \* **internal::MakeAndRegisterTestInfo** (const char \*test\_suite\_name, const char \*name, const char \*type\_param, const char \*value\_param, [internal::CodeLocation](#) code\_location, internal::Type↵  
Id fixture\_class\_id, internal::SetUpTestSuiteFunc set\_up\_tc, internal::TearDownTestSuiteFunc tear\_down\_tc, [internal::TestFactoryBase](#) \*factory)

The documentation for this class was generated from the following files:

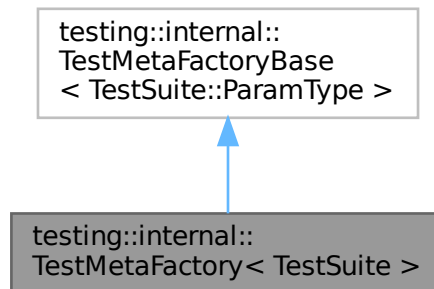
- gtest/gtest/gtest.h
- gtest/src/gtest.cc

## 15.978 testing::internal::TestMetaFactory< TestSuite > Class Template Reference

Inheritance diagram for testing::internal::TestMetaFactory< TestSuite >:



Collaboration diagram for `testing::internal::TestMetaFactory< TestSuite >`:



### Public Types

- using **ParamType** = typename TestSuite::ParamType

### Public Member Functions

- [TestFactoryBase](#) \* [CreateTestFactory](#) (ParamType parameter) override

## 15.978.1 Member Function Documentation

### 15.978.1.1 CreateTestFactory()

```
template<class TestSuite >
TestFactoryBase * testing::internal::TestMetaFactory< TestSuite >::CreateTestFactory (
 ParamType parameter) [inline], [override], [virtual]
```

Implements [testing::internal::TestMetaFactoryBase< TestSuite::ParamType >](#).

The documentation for this class was generated from the following file:

- gtest/gtest/internal/gtest-param-util.h

## 15.979 testing::internal::TestMetaFactoryBase< ParamType > Class Template Reference

### Public Member Functions

- virtual [TestFactoryBase](#) \* **CreateTestFactory** (ParamType parameter)=0

The documentation for this class was generated from the following file:

- gtest/gtest/internal/gtest-param-util.h

## 15.980 testing::TestParamInfo< ParamType > Struct Template Reference

### Public Member Functions

- **TestParamInfo** (const ParamType &a\_param, size\_t an\_index)

### Public Attributes

- ParamType **param**
- size\_t **index**

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-param-util.h

## 15.981 testing::TestProperty Class Reference

### Public Member Functions

- **TestProperty** (const std::string &a\_key, const std::string &a\_value)
- const char \* **key** () const
- const char \* **value** () const
- void **SetValue** (const std::string &new\_value)

The documentation for this class was generated from the following file:

- gtest/gtest/gtest.h

## 15.982 testing::internal::TestPropertyKeys Class Reference

### Public Member Functions

- **TestPropertyKeys** (const std::string &key)
- bool **operator()** (const [TestProperty](#) &test\_property) const

The documentation for this class was generated from the following file:

- gtest/src/gtest-internal-inl.h

## 15.983 HypothesisTester\_if::TestResult Class Reference

Result of a parametric hypothesis test.

```
#include <tools/Statistics/HypothesisTester_if.h>
```

### Public Member Functions

- **TestResult** (double pvalue, bool rejectH0, double acceptanceInferiorLimit, double acceptanceSuperiorLimit, double testStat)
- bool **rejectH0** () const
- bool **acceptH0** () const
- double **pValue** () const
- double **testStat** () const
- double **acceptanceInferiorLimit** () const
- double **acceptanceSuperiorLimit** () const

### 15.983.1 Detailed Description

Result of a parametric hypothesis test.

Contains p-value, reject/accept decision for H0, acceptance bounds for the test statistic and observed statistic value.

The documentation for this class was generated from the following file:

- tools/Statistics/HypothesisTester\_if.h

## 15.984 testing::TestResult Class Reference

### Public Member Functions

- int **total\_part\_count** () const
- int **test\_property\_count** () const
- bool **Passed** () const
- bool **Skipped** () const
- bool **Failed** () const
- bool **HasFatalFailure** () const
- bool **HasNonfatalFailure** () const
- TimeInMillis **elapsed\_time** () const
- TimeInMillis **start\_timestamp** () const
- const TestPartResult & **GetTestPartResult** (int i) const
- const [TestProperty](#) & **GetTestProperty** (int i) const

### Friends

- class **TestInfo**
- class **TestSuite**
- class **UnitTest**
- class **internal::DefaultGlobalTestPartResultReporter**
- class **internal::ExecDeathTest**
- class **internal::TestResultAccessor**
- class **internal::UnitTestImpl**
- class **internal::WindowsDeathTest**
- class **internal::FuchsiaDeathTest**

The documentation for this class was generated from the following files:

- gtest/gtest/gtest.h
- gtest/src/gtest.cc

## 15.985 testing::internal::TestResultAccessor Class Reference

### Static Public Member Functions

- static void **RecordProperty** ([TestResult](#) \*test\_result, const std::string &xml\_element, const [TestProperty](#) &property)
- static void **ClearTestPartResults** ([TestResult](#) \*test\_result)
- static const std::vector< testing::TestPartResult > & **test\_part\_results** (const [TestResult](#) &test\_result)

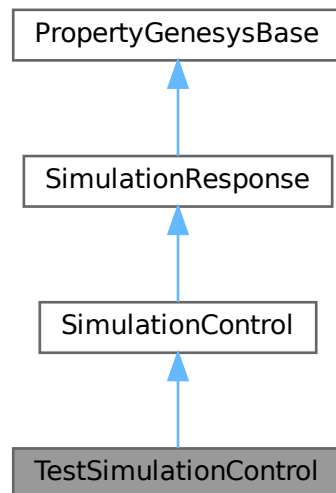
The documentation for this class was generated from the following file:

- gtest/src/gtest-internal-inl.h

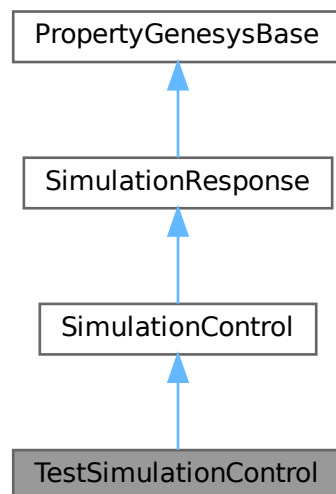


## 15.986 TestSimulationControl Class Reference

Inheritance diagram for TestSimulationControl:



Collaboration diagram for TestSimulationControl:



### Public Member Functions

- **TestSimulationControl** (std::function< std::string()> getter, std::function< void(std::string)> setter)
- std::string **getValue** () const override
- void **setValue** (std::string value, bool remove=false) override

## Public Member Functions inherited from [SimulationControl](#)

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual [List](#)< [SimulationControl](#) \* > \* **getChildSimulationControls** (int index=0)  
*Returns nested SimulationControls exposed by this control.*
- virtual bool **hasObjectInstance** () const
- virtual bool **ensureObjectInstance** ()
- virtual bool **isModelDataDefinitionReference** () const
- virtual [ModelDataDefinition](#) \* **getReferencedModelDataDefinition** () const
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const
- virtual [SimulationControlEditorHint](#) **preferredEditorHint** () const
- void **setPreferredEditorHint** ([SimulationControlEditorHint](#) hint)
- virtual [ModelDataDefinition](#) \* **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsInlineExpansion** () const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsExistingObjectSelection** () const
- virtual bool **supportsObjectCreation** () const
- virtual bool **supportsNewListElementCreation** () const
- virtual [List](#)< std::string > \* **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual [List](#)< std::string > \* **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) \* > \* **getEditableChildSimulationControls** (int index=0)

## Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual [List](#)< std::string > \* **getStrValues** ()

## Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

### Additional Inherited Members

### Protected Member Functions inherited from [SimulationControl](#)

- void `_ensureWritable` (const char \*operation) const

### Protected Attributes inherited from [SimulationControl](#)

- bool `_readonly` = false
- SimulationControlEditorHint `_preferredEditorHint` = SimulationControlEditorHint::Default

### Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string `_elementName`
- std::string `_className`
- std::string `_propertyName`
- std::string `_propertyType`
- std::string `_whatsThis`
- bool `_isList`
- bool `_isClass`
- bool `_isEnum`

## 15.986.1 Member Function Documentation

### 15.986.1.1 `getValue()`

std::string TestSimulationControl::getValue ( ) const [inline], [override], [virtual]  
Implements [SimulationResponse](#).

### 15.986.1.2 `setValue()`

```
void TestSimulationControl::setValue (
 std::string value,
 bool remove = false) [inline], [override], [virtual]
```

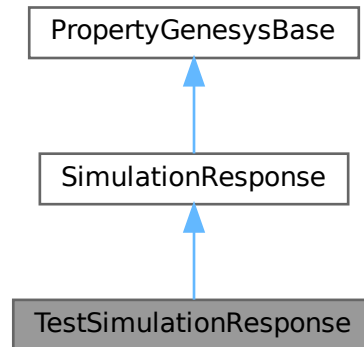
Implements [SimulationControl](#).

The documentation for this class was generated from the following file:

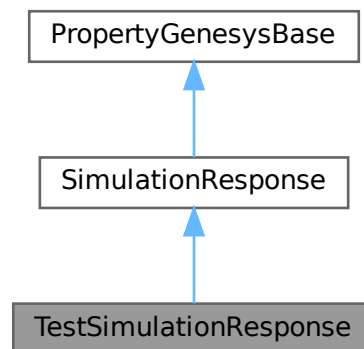
- tests/unit/test\_simulator\_support.cpp

## 15.987 TestSimulationResponse Class Reference

Inheritance diagram for TestSimulationResponse:



Collaboration diagram for TestSimulationResponse:



### Public Member Functions

- **TestSimulationResponse** (std::function< std::string()> getter, bool isEnum=false)
- std::string [getValue](#) () const override

### Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual [List](#)< std::string > \* **getStrValues** ()

## Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

## Additional Inherited Members

## Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **\_elementName**
- std::string **\_className**
- std::string **\_propertyName**
- std::string **\_propertyType**
- std::string **\_whatsThis**
- bool **\_isList**
- bool **\_isClass**
- bool **\_isEnum**

## 15.987.1 Member Function Documentation

### 15.987.1.1 getValue()

std::string TestSimulationResponse::getValue ( ) const [inline], [override], [virtual]

Implements [SimulationResponse](#).

The documentation for this class was generated from the following file:

- tests/unit/test\_simulator\_support.cpp

## 15.988 testing::TestSuite Class Reference

### Public Member Functions

- **TestSuite** (const char \*name, const char \*a\_type\_param, internal::SetUpTestSuiteFunc set\_up\_tc, internal::TearDownTestSuiteFunc tear\_down\_tc)
- const char \* **name** () const
- const char \* **type\_param** () const
- bool **should\_run** () const
- int **successful\_test\_count** () const
- int **skipped\_test\_count** () const
- int **failed\_test\_count** () const
- int **reportable\_disabled\_test\_count** () const
- int **disabled\_test\_count** () const
- int **reportable\_test\_count** () const
- int **test\_to\_run\_count** () const
- int **total\_test\_count** () const
- bool **Passed** () const
- bool **Failed** () const

- `TimeInMillis` **elapsed\_time** () const
- `TimeInMillis` **start\_timestamp** () const
- const `TestInfo` \* **GetTestInfo** (int i) const
- const `TestResult` & **ad\_hoc\_test\_result** () const

#### Friends

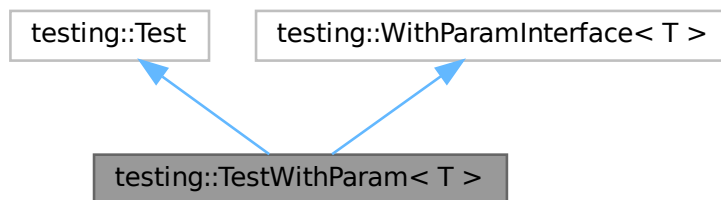
- class `Test`
- class `internal::UnitTestImpl`

The documentation for this class was generated from the following files:

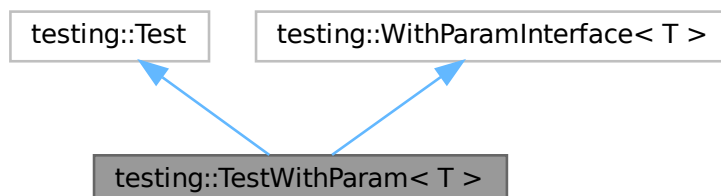
- `gtest/gtest/gtest.h`
- `gtest/src/gtest.cc`

## 15.989 `testing::TestWithParam< T >` Class Template Reference

Inheritance diagram for `testing::TestWithParam< T >`:



Collaboration diagram for `testing::TestWithParam< T >`:



#### Additional Inherited Members

Public Types inherited from `testing::WithParamInterface< T >`

- typedef `T` `ParamType`

**Static Public Member Functions inherited from [testing::Test](#)**

- static void **SetUpTestSuite** ()
- static void **TearDownTestSuite** ()
- static void **TearDownTestCase** ()
- static void **SetUpTestCase** ()
- static bool **HasFatalFailure** ()
- static bool **HasNonfatalFailure** ()
- static bool **IsSkipped** ()
- static bool **HasFailure** ()
- static void **RecordProperty** (const std::string &key, const std::string &value)
- static void **RecordProperty** (const std::string &key, int value)

**Static Public Member Functions inherited from [testing::WithParamInterface< T >](#)**

- static const ParamType & **GetParam** ()

**Protected Member Functions inherited from [testing::Test](#)**

- virtual void **SetUp** ()
- virtual void **TearDown** ()

The documentation for this class was generated from the following file:

- gtest/gtest/gtest.h

**15.990 testing::internal::ThreadLocal< T > Class Template Reference****Public Member Functions**

- **ThreadLocal** (const T &value)
- T \* **pointer** ()
- const T \* **pointer** () const
- const T & **get** () const
- void **set** (const T &value)

The documentation for this class was generated from the following file:

- gtest/gtest/internal/gtest-port.h

**15.991 QCPAxisPainterPrivate::TickLabelData Struct Reference****Public Attributes**

- QString **basePart**
- QString **expPart**
- QString **suffixPart**
- QRect **baseBounds**
- QRect **expBounds**
- QRect **suffixBounds**
- QRect **totalBounds**
- QRect **rotatedTotalBounds**
- QFont **baseFont**
- QFont **expFont**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/qcustomplot.h

## 15.992 Token Struct Reference

### Public Types

- enum **Kind** {  
**eof**, **null**, **f**, **t**,  
**number**, **string**, **lbracket**, **rbracket**,  
**lbrace**, **rbrace**, **comma**, **colon** }

### Public Attributes

- enum Token::Kind **kind**
- std::string **str** {}

The documentation for this struct was generated from the following file:

- kernel/simulator/persistence/JsonSerializer.cpp

## 15.993 yy::genesyspp\_parser::token Struct Reference

[Token](#) kinds.

```
#include <parser/GenesysParser.h>
```

### Public Types

- enum **token\_kind\_type** {  
**YYEMPTY** = -2, **END** = 0, **YError** = 256, **YYUNDEF** = 257 ,  
**NUMD** = 258, **NUMH** = 259, **CTEZERO** = 260, **oLE** = 261 ,  
**oGE** = 262, **oEQ** = 263, **oNE** = 264, **oAND** = 265 ,  
**oOR** = 266, **oNAND** = 267, **oXOR** = 268, **oNOT** = 269 ,  
**fSIN** = 270, **fCOS** = 271, **fROUND** = 272, **fMOD** = 273 ,  
**fTRUNC** = 274, **fFRAC** = 275, **fEXP** = 276, **fSQRT** = 277 ,  
**fLOG** = 278, **fLN** = 279, **mathMIN** = 280, **mathMAX** = 281 ,  
**fVAL** = 282, **fEVAL** = 283, **fLENG** = 284, **fRND1** = 285 ,  
**fEXPO** = 286, **fNORM** = 287, **fUNIF** = 288, **fWEIB** = 289 ,  
**fLOGN** = 290, **fGAMM** = 291, **fERLA** = 292, **fTRIA** = 293 ,  
**fBETA** = 294, **fDISC** = 295, **fTNOW** = 296, **fTFIN** = 297 ,  
**fMAXREP** = 298, **fNUMREP** = 299, **fIDENT** = 300, **simulEntitiesWIP** = 301 ,  
**cIF** = 302, **cELSE** = 303, **cFOR** = 304, **cTO** = 305 ,  
**cDO** = 306, **ATTRIB** = 307, **CSTAT** = 308, **COUNTER** = 309 ,  
**SIMRESP** = 310, **SIMCTRL** = 311, **fAVG** = 312, **fCOUNT** = 313 ,  
**ILLEGAL** = 314, **RESOURCE** = 315, **fNR** = 316, **fMR** = 317 ,  
**fIRF** = 318, **fRESSEIZES** = 319, **fSTATE** = 320, **fSETSUM** = 321 ,  
**fRESUTIL** = 322, **QUEUE** = 323, **fNQ** = 324, **fFIRSTINQ** = 325 ,  
**fLASTINQ** = 326, **fSAQUE** = 327, **fAQUE** = 328, **fENTATRANK** = 329 ,  
**SET** = 330, **fNUMSET** = 331, **VARI** = 332, **FORM** = 333 ,  
**fNUMGR** = 334, **fATRGR** = 335, **LPAREN** = 336, **RPAREN** = 337 ,  
**LBRACKET** = 338, **RBRACKET** = 339, **PLUS** = 340, **MINUS** = 341 ,  
**STAR** = 342, **POWER** = 343, **SLASH** = 344, **LESS** = 345 ,  
**GREATER** = 346, **ASSIGN** = 347, **COMMA** = 348 }
- typedef token\_kind\_type **yytokentype**  
*Backward compatibility alias (Bison 3.6).*

### 15.993.1 Detailed Description

[Token](#) kinds.

The documentation for this struct was generated from the following file:

- parser/[GenesysParser.h](#)



## 15.994 TokenService Class Reference

### Public Member Functions

- std::string **generateSessionId** ()
- std::string **generateAccessToken** ()

The documentation for this class was generated from the following files:

- applications/worker/auth/TokenService.h
- applications/worker/auth/TokenService.cpp

## 15.995 TraceConsoleController Class Reference

Controller that routes simulator trace events to GUI text widgets.

```
#include <applications/gui/genesys/controllers/TraceConsoleController.h>
```

### Public Member Functions

- **TraceConsoleController** (QTextEdit \*console, QTextEdit \*simulationText, QTextEdit \*reportsText, std::function< bool()> traceWidgetsEnabled=[])  
*Creates the trace-output bridge used by [MainWindow](#) trace wrappers.*
- void **simulatorTraceHandler** ([TraceEvent](#) e) const  
*Renders generic traces to the main console pane.*
- void **simulatorTraceErrorHandler** ([TraceErrorEvent](#) e) const  
*Renders error traces to the main console pane.*
- void **simulatorTraceSimulationHandler** ([TraceSimulationEvent](#) e) const  
*Renders simulation traces to the simulation output pane.*
- void **simulatorTraceReportsHandler** ([TraceEvent](#) e) const  
*Renders reports traces to the reports output pane.*

### 15.995.1 Detailed Description

Controller that routes simulator trace events to GUI text widgets.

This controller is the Phase-4 extraction for trace presentation. [MainWindow](#) keeps compatibility handlers registered in the kernel trace manager and delegates rendering to this class, reducing direct UI formatting logic in the façade. Responsibilities:

- render regular/error traces to the main console pane;
- render simulation traces to the simulation output pane;
- render report traces to the reports pane.

Boundaries:

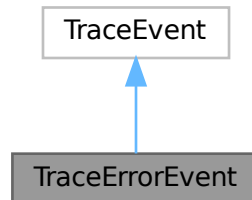
- it does not subscribe handlers by itself (registration stays in [MainWindow](#)/event layer);
- it does not change simulation flow, model state, or controller orchestration;
- it acts as a UI bridge, not as a domain service.

The documentation for this class was generated from the following files:

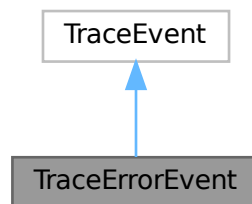
- applications/gui/genesys/controllers/TraceConsoleController.h
- applications/gui/genesys/controllers/TraceConsoleController.cpp

## 15.996 **TraceErrorEvent** Class Reference

Inheritance diagram for TraceErrorEvent:



Collaboration diagram for TraceErrorEvent:



### Public Member Functions

- **TraceErrorEvent** (const std::string &text, const std::exception &e)
- **TraceErrorEvent** (const std::string &text, TraceManager::Level level)
- const std::string & **getExceptionMessage** () const

### Public Member Functions inherited from **TraceEvent**

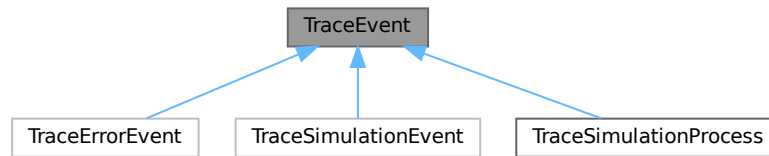
- **TraceEvent** (TraceManager::Level level, const std::string &text)
- **TraceEvent** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- TraceManager::Level **getTracelevel** () const
- const std::string & **getText** () const

The documentation for this class was generated from the following file:

- kernel/simulator/TraceManager.h

## 15.997 TraceEvent Class Reference

Inheritance diagram for TraceEvent:



### Public Member Functions

- **TraceEvent** (TraceManager::Level level, const std::string &text)
- **TraceEvent** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- TraceManager::Level **getTracelevel** () const
- const std::string & **getText** () const

The documentation for this class was generated from the following file:

- kernel/simulator/TraceManager.h

## 15.998 testing::internal::TraceInfo Struct Reference

### Public Attributes

- const char \* **file**
- int **line**
- std::string **message**

The documentation for this struct was generated from the following file:

- gtest/src/gtest-internal-inl.h

## 15.999 TraceManager Class Reference

```
#include <kernel/simulator/TraceManager.h>
```

### Public Types

- enum class **Level** : int {  
**L0\_noTraces** = 0 , **L1\_errorFatal** = 1 , **L2\_results** = 2 , **L3\_errorRecover** = 3 ,  
**L4\_warning** = 4 , **L5\_event** = 5 , **L6\_arrival** = 6 , **L7\_internal** = 7 ,  
**L8\_detailed** = 8 , **L9\_mostDetailed** = 9 , **num\_elements** = 10 }

### Public Member Functions

- **TraceManager** ([Simulator](#) \*simulator)
- void **addTraceHandler** (traceListener traceListener)  
*addTraceHandler*
- void **addTraceReportHandler** (traceListener traceReportListener)  
*addTraceReportHandler*
- void **addTraceSimulationHandler** (traceSimulationListener traceSimulationListener)

- addTraceSimulationHandler*
- void [addTraceErrorHandler](#) (traceErrorListener traceErrorListener)
  - addTraceErrorHandler*
- template<typename Class >
  - void [addTraceHandler](#) (Class \*object, void(Class::\*function)([TraceEvent](#)))
  - addTraceHandler*
- template<typename Class >
  - void [addTraceErrorHandler](#) (Class \*object, void(Class::\*function)([TraceErrorEvent](#)))
  - addTraceErrorHandler*
- template<typename Class >
  - void [addTraceReportHandler](#) (Class \*object, void(Class::\*function)([TraceEvent](#)))
  - addTraceReportHandler*
- template<typename Class >
  - void [addTraceSimulationHandler](#) (Class \*object, void(Class::\*function)([TraceSimulationEvent](#)))
  - addTraceSimulationHandler*
- void **addTraceSimulationExceptionRuleModelData** (void \*thisobject)
- void **trace** (TraceManager::Level level, const std::string &text)
  - Deprecated.*
- void **traceSimulation** (void \*thisobject, TraceManager::Level level, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text)
  - Deprecated.*
- void **traceSimulation** (void \*thisobject, TraceManager::Level level, const std::string &text)
  - Deprecated.*
- void **beginShutdown** ()
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
  - Trace to a general output, used when not simulating (eg: adding plugins, checking the model, etc)*
- void **traceError** (const std::string &text, const std::exception &e)
  - Trace to the error output, used in every situation an error happens (simulating, report, general)*
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)
  - Trace to the error output, used in every situation an error happens (simulating, report, general)*
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)
  - Trace to the report output, used only when generating the simulation report.*
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed, bool showAnyway=false)
  - Trace to the simulation output, used only when simulation is running (eg: components or dataElements inform something)*
- void **traceSimulation** (void \*thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed, bool showAnyway=false)
  - Trace to the simulation output, used only when simulation is running (eg: components or dataElements inform something)*
- [List](#)< std::string > \* [errorMessages](#) () const
  - errorMessages*
- void [setTraceLevel](#) (TraceManager::Level \_traceLevel)
  - setTraceLevel*
- TraceManager::Level [getTraceLevel](#) () const
  - getTraceLevel*
- [Simulator](#) \* [getParentSimulator](#) () const
  - getParentSimulator*
- void [setTraceSimulationRuleAllAllowed](#) (bool \_traceSimulationRuleAllAllowed)
  - setTraceSimulationRuleAllAllowed*
- bool [isTraceSimulationRuleAllAllowed](#) () const
  - isTraceSimulationRuleAllAllowed*

## Static Public Member Functions

- static std::string **convertEnumToStr** (Level level)

### 15.999.1 Detailed Description

The [TraceManager](#) is used to trace back model simulation information and track/debug the simulation. It works as the model simulation output (cout) and allows external methods to hook up such output as listeners.

### 15.999.2 Member Function Documentation

#### 15.999.2.1 addTraceErrorHandler() [1/2]

```
template<typename Class >
void TraceManager::addTraceErrorHandler (
 Class * object,
 void(Class::*)(TraceErrorEvent) function)
addTraceErrorHandler
```

##### Parameters

|               |                                                                 |
|---------------|-----------------------------------------------------------------|
| <i>object</i> | Add a method to by notified/invoked then a traceError is called |
|---------------|-----------------------------------------------------------------|

#### 15.999.2.2 addTraceErrorHandler() [2/2]

```
void TraceManager::addTraceErrorHandler (
 traceErrorListener traceErrorListener)
addTraceErrorHandler
```

##### Parameters

|                           |                                                                   |
|---------------------------|-------------------------------------------------------------------|
| <i>traceErrorListener</i> | Add a function to by notified/invoked then a traceError is called |
|---------------------------|-------------------------------------------------------------------|

#### 15.999.2.3 addTraceHandler() [1/2]

```
template<typename Class >
void TraceManager::addTraceHandler (
 Class * object,
 void(Class::*)(TraceEvent) function)
addTraceHandler
```

##### Parameters

|               |                                                            |
|---------------|------------------------------------------------------------|
| <i>object</i> | Add a method to by notified/invoked then a trace is called |
|---------------|------------------------------------------------------------|

#### 15.999.2.4 addTraceHandler() [2/2]

```
void TraceManager::addTraceHandler (
 traceListener traceListener)
addTraceHandler
```

##### Parameters

|                      |                                                              |
|----------------------|--------------------------------------------------------------|
| <i>traceListener</i> | Add a function to by notified/invoked then a trace is called |
|----------------------|--------------------------------------------------------------|

**15.999.2.5 addTraceReportHandler()** [1/2]

```
template<typename Class >
void TraceManager::addTraceReportHandler (
 Class * object,
 void(Class::*)(TraceEvent) function)
addTraceReportHandler
```

**Parameters**

|               |                                                                  |
|---------------|------------------------------------------------------------------|
| <i>object</i> | Add a method to by notified/invoked then a traceReport is called |
|---------------|------------------------------------------------------------------|

**15.999.2.6 addTraceReportHandler()** [2/2]

```
void TraceManager::addTraceReportHandler (
 traceListener traceReportListener)
addTraceReportHandler
```

**Parameters**

|                            |                                                                    |
|----------------------------|--------------------------------------------------------------------|
| <i>traceReportListener</i> | Add a function to by notified/invoked then a traceReport is called |
|----------------------------|--------------------------------------------------------------------|

**15.999.2.7 addTraceSimulationHandler()** [1/2]

```
template<typename Class >
void TraceManager::addTraceSimulationHandler (
 Class * object,
 void(Class::*)(TraceSimulationEvent) function)
addTraceSimulationHandler
```

**Parameters**

|               |                                                                      |
|---------------|----------------------------------------------------------------------|
| <i>object</i> | Add a method to by notified/invoked then a traceSimulation is called |
|---------------|----------------------------------------------------------------------|

**15.999.2.8 addTraceSimulationHandler()** [2/2]

```
void TraceManager::addTraceSimulationHandler (
 traceSimulationListener traceSimulationListener)
addTraceSimulationHandler
```

**Parameters**

|                                |                                                                        |
|--------------------------------|------------------------------------------------------------------------|
| <i>traceSimulationListener</i> | Add a function to by notified/invoked then a traceSimulation is called |
|--------------------------------|------------------------------------------------------------------------|

**15.999.2.9 errorMessages()**

```
List< std::string > * TraceManager::errorMessages () const
errorMessages
```

**Returns****15.999.2.10 getParentSimulator()**

```
Simulator * TraceManager::getParentSimulator () const
```

getParentSimulator

Returns

#### 15.999.2.11 getTraceLevel()

```
TraceManager::Level TraceManager::getTraceLevel () const
getTraceLevel
```

Returns

#### 15.999.2.12 isTraceSimulationRuleAllAllowed()

```
bool TraceManager::isTraceSimulationRuleAllAllowed () const
isTraceSimulationRuleAllAllowed
```

Returns

#### 15.999.2.13 setTraceLevel()

```
void TraceManager::setTraceLevel (
 TraceManager::Level _traceLevel)
setTraceLevel
```

Parameters

|                    |  |
|--------------------|--|
| <i>_traceLevel</i> |  |
|--------------------|--|

#### 15.999.2.14 setTraceSimulationRuleAllAllowed()

```
void TraceManager::setTraceSimulationRuleAllAllowed (
 bool _traceSimulationRuleAllAllowed)
setTraceSimulationRuleAllAllowed
```

Parameters

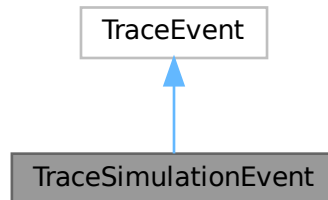
|                                       |  |
|---------------------------------------|--|
| <i>_traceSimulationRuleAllAllowed</i> |  |
|---------------------------------------|--|

The documentation for this class was generated from the following files:

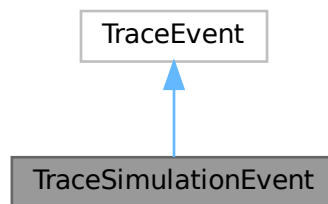
- kernel/simulator/TraceManager.h
- kernel/simulator/TraceManager.cpp

## 15.1000 TraceSimulationEvent Class Reference

Inheritance diagram for TraceSimulationEvent:



Collaboration diagram for TraceSimulationEvent:



### Public Member Functions

- **TraceSimulationEvent** (TraceManager::Level level, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text)
- [ModelComponent](#) \* **getComponent** () const
- [Entity](#) \* **getEntity** () const
- double **getTime** () const

### Public Member Functions inherited from [TraceEvent](#)

- **TraceEvent** (TraceManager::Level level, const std::string &text)
- **TraceEvent** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- TraceManager::Level **getTracelevel** () const
- const std::string & **getText** () const

The documentation for this class was generated from the following file:

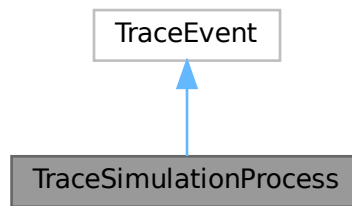
- kernel/simulator/TraceManager.h

## 15.1001 TraceSimulationProcess Class Reference

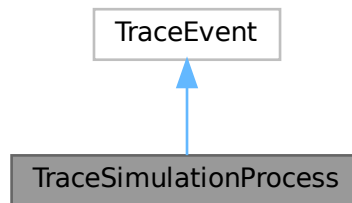
```
#include <kernel/simulator/TraceManager.h>
```



Inheritance diagram for TraceSimulationProcess:



Collaboration diagram for TraceSimulationProcess:



### Public Member Functions

- **TraceSimulationProcess** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_↔ detailed)

### Public Member Functions inherited from [TraceEvent](#)

- **TraceEvent** (TraceManager::Level level, const std::string &text)
- **TraceEvent** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- TraceManager::Level **getTracelevel** () const
- const std::string & **getText** () const

### 15.1001.1 Detailed Description

Events related to simulation "process" (usually process analyser), associated to entire replication or simulation events (begin/end/pause of replication/simulation) @ToDo: (importante): CLASS **NOT FULLY IMPLEMENTED** (to be implemented for process analyser)

The documentation for this class was generated from the following file:

- kernel/simulator/TraceManager.h

## 15.1002 TraitsGUI< T > Struct Template Reference

GUI-specific trait constants used to tune default rendering behavior.

```
#include <applications/gui/genesys/TraitsGUI.h>
```

### 15.1002.1 Detailed Description

```
template<class T>
struct TraitsGUI< T >
```

GUI-specific trait constants used to tune default rendering behavior.

These specializations keep visual defaults close to the legacy Qt interface without scattering magic numbers across the graphics and editor code.

The documentation for this struct was generated from the following file:

- applications/gui/genesys/TraitsGUI.h

### 15.1003 TraitsGUI< GAssociation > Struct Reference

#### Static Public Attributes

- static const uint64\_t **foreColor** = 0x808080FF
- static constexpr const int **margin** = 2
- static constexpr const int **selectionWidth** = 8

The documentation for this struct was generated from the following file:

- applications/gui/genesys/TraitsGUI.h

### 15.1004 TraitsGUI< GComponentPort > Struct Reference

#### Static Public Attributes

- static constexpr const int **width** = 20
- static constexpr const int **penWidth** = 1
- static constexpr const int **raise** = 3
- static constexpr const int **margin** = 2
- static const uint64\_t **connectedBackgroundColor** = 0x2020FF80
- static const uint64\_t **unconnectedBackgroundColor** = 0xFF202040
- static const uint64\_t **borderColor** = 0x000000FF
- static const uint64\_t **pathRaised** = 0x808080FF
- static const uint64\_t **pathStunken** = 0xFFFFFFFF
- static const uint64\_t **textColor** = 0xFFFFFFFF
- static const uint64\_t **textShadowColor** = 0x000000FF
- static const uint64\_t **selectionSquaresColor** = 0x000000FF

The documentation for this struct was generated from the following file:

- applications/gui/genesys/TraitsGUI.h

### 15.1005 TraitsGUI< GConnection > Struct Reference

#### Static Public Attributes

- static const uint64\_t **foreColor** = 0x000000FF
- static constexpr const int **margin** = 2
- static constexpr const int **selectionWidth** = 8

The documentation for this struct was generated from the following file:

- applications/gui/genesys/TraitsGUI.h

## 15.1006 TraitsGUI< GMainWindow > Struct Reference

### Static Public Attributes

- static const bool **startMaximized** = true
- static const int **zoomButtonChange** = 30
- static const uint64\_t **consoleTextColor** = 0x008000FF

The documentation for this struct was generated from the following file:

- applications/gui/genesys/TraitsGUI.h

## 15.1007 TraitsGUI< GModelComponent > Struct Reference

### Static Public Attributes

- static constexpr const int **opacity** = 256.0 \* 0.67
- static constexpr const int **colorIncrease** = 256.0 \* 0.2
- static constexpr const int **lighter** = 133
- static constexpr const int **width** = 150
- static constexpr const int **penWidth** = 1
- static constexpr const int **raise** = 5
- static constexpr const int **margin** = 8
- static constexpr const int **selectionWidth** = 8
- static constexpr const float **stretchPos** = 0.5
- static constexpr const float **stretch** = 0.0
- static constexpr const float **heightProportion** = 0.67
- static const uint64\_t **breakpointColor** = 0xFF0000FF
- static const uint64\_t **borderColor** = 0x000000FF
- static const uint64\_t **pathRaised** = 0xFFFFFFFF
- static const uint64\_t **pathStunken** = 0x808080FF
- static const uint64\_t **textColor** = 0xFFFFFFFF
- static const uint64\_t **textShadowColor** = 0x000000FF
- static const uint64\_t **selectionSquaresColor** = 0x000000FF

The documentation for this struct was generated from the following file:

- applications/gui/genesys/TraitsGUI.h

## 15.1008 TraitsGUI< GModelDataDefinition > Struct Reference

### Static Public Attributes

- static constexpr const int **opacity** = 256 \* 0.33
- static constexpr const int **nonEditableOpacity** = 58
- static constexpr const int **selectedNonEditableOpacity** = 72
- static constexpr const int **width** = 100
- static constexpr const int **penWidth** = 1
- static constexpr const int **raise** = 3
- static constexpr const int **margin** = 5
- static constexpr const int **selectionWidth** = 8
- static constexpr const float **stretchPos** = 0.5
- static constexpr const float **stretch** = 0.0
- static constexpr const float **heightProportion** = 0.50
- static const uint64\_t **breakpointColor** = 0xFF0000FF
- static const uint64\_t **borderColor** = 0x000000FF
- static const uint64\_t **pathRaised** = 0x808080FF

- static const uint64\_t **pathStunken** = 0xFFFFFFFF
- static const uint64\_t **textColor** = 0x0
- static const uint64\_t **textShadowColor** = 0x000000FF
- static const uint64\_t **selectionSquaresColor** = 0x000000FF

The documentation for this struct was generated from the following file:

- applications/gui/genesys/TraitsGUI.h

## 15.1009 TraitsGUI< GModelGraphicPic > Struct Reference

### Static Public Attributes

- static const int **nodeComponentFontSize** = 12
- static const uint64\_t **nodeComponentFontColor** = 0x000000FF
- static const uint64\_t **nodeComponentFillColor** = 0x80A040FF
- static const int **edgeComponentFontSize** = 7
- static const uint64\_t **edgeComponentEdgeColor** = 0x000000FF
- static const uint64\_t **edgeComponentFontColor** = 0x000000FF
- static const int **nodeDatadefInternalFontSize** = 8
- static const uint64\_t **nodeDatadefInternalColor** = 0xA0A0A0FF
- static const uint64\_t **nodeDatadefInternalFontColor** = 0xA0A0A0FF
- static const int **nodeDatadefAttachedFontSize** = 8
- static const uint64\_t **nodeDatadefAttachedFontColor** = 0x808080FF

The documentation for this struct was generated from the following file:

- applications/gui/genesys/TraitsGUI.h

## 15.1010 TraitsGUI< GScene > Struct Reference

### Static Public Attributes

- static constexpr const int **gridInterval** = 20
- static constexpr const int **gridPenWidth** = 1
- static const uint64\_t **gridColor** = 0xC0C0C0E0

The documentation for this struct was generated from the following file:

- applications/gui/genesys/TraitsGUI.h

## 15.1011 TraitsGUI< GView > Struct Reference

### Static Public Attributes

- static constexpr const int **sceneCenter** = 25000
- static constexpr const int **sceneDistanceCenter** = 25000
- static const uint64\_t **backgroundEnabledColor** = 0xFFFF8040
- static const uint64\_t **backgroundDisabledColor** = 0xC0C0C020

The documentation for this struct was generated from the following file:

- applications/gui/genesys/TraitsGUI.h

## 15.1012 TraitsKernel< T > Struct Template Reference

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

## 15.1013 TraitsKernel< Collector\_if > Struct Reference

### Public Types

- typedef [CollectorDatafileDefaultImpl1](#) **Implementation**
- typedef double **DataType**

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

## 15.1014 TraitsKernel< Model > Struct Reference

### Public Types

- typedef [StatisticsDefaultImpl1](#) **StatisticsCollector\_StatisticsImplementation**
- typedef [CollectorDefaultImpl1](#) **StatisticsCollector\_CollectorImplementation**

### Static Public Attributes

- static const TraceManager::Level **traceLevel** = TraceManager::Level::L5\_event

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

## 15.1015 TraitsKernel< ModelChecker\_if > Struct Reference

### Public Types

- typedef [ModelCheckerDefaultImpl1](#) **Implementation**

### Static Public Attributes

- static const TraceManager::Level **traceLevel** = TraceManager::Level::L2\_results

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

## 15.1016 TraitsKernel< ModelComponent > Struct Reference

### Static Public Attributes

- static constexpr bool **reportStatistics** = true
- static const TraceManager::Level **traceLevel** = TraceManager::Level::L2\_results

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

## 15.1017 TraitsKernel< ModelDataDefinition > Struct Reference

### Static Public Attributes

- static constexpr bool **reportStatistics** = true
- static const TraceManager::Level **traceLevel** = TraceManager::Level::L2\_results

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

## 15.1018 TraitsKernel< Parser\_if > Struct Reference

### Public Types

- typedef [ParserDefaultImpl2](#) **Implementation**

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

## 15.1019 TraitsKernel< Persistence\_if > Struct Reference

### Public Types

- typedef [PersistencePartialLoadImpl1](#) **Implementation**

### Static Public Attributes

- static const TraceManager::Level **traceLevel** = TraceManager::Level::L2\_results

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

## 15.1020 TraitsKernel< PluginConnector\_if > Struct Reference

### Public Types

- typedef [PluginConnectorDummyImpl1](#) **Implementation**

### Static Public Attributes

- static const TraceManager::Level **traceLevel** = TraceManager::Level::L4\_warning

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

## 15.1021 TraitsKernel< Sampler\_if > Struct Reference

### Public Types

- typedef [SamplerDefaultImpl1](#) **Implementation**
- typedef [SamplerDefaultImpl1::DefaultImpl1RNG\\_Parameters](#) **Parameters**

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

## 15.1022 TraitsKernel< SimulationReporter\_if > Struct Reference

### Public Types

- typedef [SimulationReporterDefaultImpl1](#) **Implementation**
- typedef [Counter](#) **CounterImplementation**

### Static Public Attributes

- static const TraceManager::Level **traceLevel** = TraceManager::Level::L2\_results

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

## 15.1023 TraitsKernel< Simulator > Struct Reference

### Static Public Attributes

- static const TraceManager::Level **traceLevel** = TraceManager::Level::L9\_mostDetailed

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

## 15.1024 TraitsKernel< Statistics\_if > Struct Reference

### Public Types

- typedef double **DataType**

### Static Public Attributes

- static constexpr double **SignificanceLevel** = 0.05

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

## 15.1025 TraitsKernel< StatisticsDatafile\_if > Struct Reference

### Public Types

- typedef [StatisticsDatafileDefaultImpl1](#) **Implementation**
- typedef [CollectorDatafileDefaultImpl1](#) **CollectorImplementation**

### Static Public Attributes

- static constexpr double **SignificanceLevel** = 0.05

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

## 15.1026 TraitsTools< T > Struct Template Reference

Registry-like traits to bind abstractions to concrete tools classes.

```
#include <tools/TraitsTools.h>
```

### 15.1026.1 Detailed Description

**template<typename T>**

**struct TraitsTools< T >**

Registry-like traits to bind abstractions to concrete tools classes.

Architectural role:

- Central selection point for concrete implementations used by this package.

Implementation status:

- Currently covers only a subset of tool abstractions.

Planned evolution:

- Extend bindings as new stable interfaces receive concrete implementations.

- New abstractions without safe concrete implementations should remain documented-only for now to avoid build instability.

The documentation for this struct was generated from the following file:

- tools/TraitsTools.h

## 15.1027 TraitsTools< AIAssistant\_if > Struct Reference

```
#include <tools/TraitsTools.h>
```

### Public Types

- typedef [AIAssistantDefaultImpl](#) Implementation

### 15.1027.1 Detailed Description

Configure the AI Assistant to be used

The documentation for this struct was generated from the following file:

- tools/TraitsTools.h

## 15.1028 TraitsTools< Fitter\_if > Struct Reference

```
#include <tools/TraitsTools.h>
```

### Public Types

- typedef [FitterDefaultImpl](#) Implementation

### 15.1028.1 Detailed Description

Configure the Fitter to be used

The documentation for this struct was generated from the following file:

- tools/TraitsTools.h

## 15.1029 TraitsTools< HypothesisTester\_if > Struct Reference

```
#include <tools/TraitsTools.h>
```

### Public Types

- typedef [HypothesisTesterDefaultImpl1](#) Implementation

### Static Public Attributes

- static constexpr unsigned int **ConfidenceLevel** = 95

### 15.1029.1 Detailed Description

Configure the Hypothesis Tester to be used

The documentation for this struct was generated from the following file:

- tools/TraitsTools.h

## 15.1030 TraitsTools< Solver\_if > Struct Reference

```
#include <tools/TraitsTools.h>
```



**Public Types**

- typedef [SolverDefaultImpl1](#) Implementation

**Static Public Attributes**

- static constexpr double **Precision** = 1e-5
- static constexpr unsigned int **MaxSteps** = 1e2

**15.1030.1 Detailed Description**

Configure the Solver to be used

The documentation for this struct was generated from the following file:

- tools/TraitsTools.h

**15.1031 testing::internal::TrueWithString Struct Reference****Public Member Functions**

- **TrueWithString** (const char \*str)
- **TrueWithString** (const std::string &str)
- **operator bool** () const

**Public Attributes**

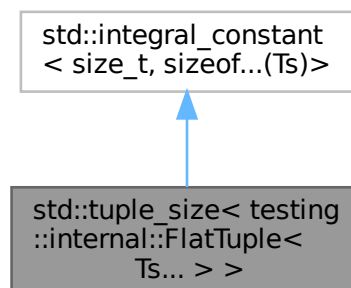
- std::string **value**

The documentation for this struct was generated from the following file:

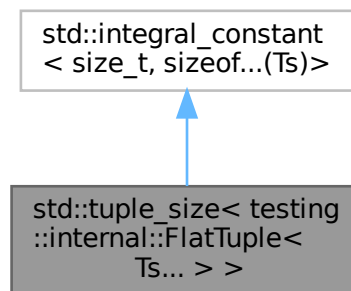
- gtest/gtest/internal/gtest-internal.h

**15.1032 std::tuple\_size< testing::internal::FlatTuple< Ts... > > Struct Template Reference**

Inheritance diagram for std::tuple\_size< testing::internal::FlatTuple< Ts... > >:



Collaboration diagram for `std::tuple_size< testing::internal::FlatTuple< Ts... > >`:



The documentation for this struct was generated from the following file:

- `gtest/gtest/internal/gtest-internal.h`

### 15.1033 `testing::internal::TypepeldHelper< T >` Class Template Reference

#### Static Public Attributes

- static bool **dummy\_** = false

The documentation for this class was generated from the following file:

- `gtest/gtest/internal/gtest-internal.h`

### 15.1034 `testing::internal::TypeParameterizedTest< Fixture, TestSel, Types >` Class Template Reference

#### Static Public Member Functions

- static bool **Register** (const char \*prefix, const [CodeLocation](#) &code\_location, const char \*case\_name, const char \*test\_names, int index, const std::vector< std::string > &type\_names=GenerateNames< [DefaultNameGenerator](#), [Types](#) >())

The documentation for this class was generated from the following file:

- `gtest/gtest/internal/gtest-internal.h`

### 15.1035 `testing::internal::TypeParameterizedTest< Fixture, TestSel, internal::None >` Class Template Reference

#### Static Public Member Functions

- static bool **Register** (const char \*, const [CodeLocation](#) &, const char \*, const char \*, int, const std::vector< std::string > &=std::vector< std::string >())

The documentation for this class was generated from the following file:

- `gtest/gtest/internal/gtest-internal.h`

## 15.1036 testing::internal::TypeParameterizedTestSuite< Fixture, Tests, Types > Class Template Reference

### Static Public Member Functions

- static bool **Register** (const char \*prefix, [CodeLocation](#) code\_location, const TypedTestSuitePState \*state, const char \*case\_name, const char \*test\_names, const std::vector< std::string > &type\_names=GenerateNames< [DefaultNameGenerator](#), [Types](#) >())

The documentation for this class was generated from the following file:

- gtest/gtest/internal/gtest-internal.h

## 15.1037 testing::internal::TypeParameterizedTestSuite< Fixture, internal::None, Types > Class Template Reference

### Static Public Member Functions

- static bool **Register** (const char \*, const [CodeLocation](#) &, const TypedTestSuitePState \*, const char \*, const char \*, const std::vector< std::string > &=std::vector< std::string >())

The documentation for this class was generated from the following file:

- gtest/gtest/internal/gtest-internal.h

## 15.1038 testing::internal::TypeParameterizedTestSuiteRegistry Class Reference

### Public Member Functions

- void **RegisterTestSuite** (const char \*test\_suite\_name, [CodeLocation](#) code\_location)
- void **RegisterInstantiation** (const char \*test\_suite\_name)
- void **CheckForInstantiations** ()

The documentation for this class was generated from the following files:

- gtest/gtest/internal/gtest-param-util.h
- gtest/src/gtest.cc

## 15.1039 testing::internal::Types< Head\_, Tail\_ > Struct Template Reference

### Public Types

- using **Head** = Head\_
- using **Tail** = [Types](#)< Tail\_... >

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-type-util.h

## 15.1040 testing::internal::Types< Head\_ > Struct Template Reference

### Public Types

- using **Head** = Head\_
- using **Tail** = [None](#)

The documentation for this struct was generated from the following file:

- gtest/gtest/internal/gtest-type-util.h

## 15.1041 testing::internal::TypeWithSize< size > Class Template Reference

### Public Types

- using **UInt** = void

The documentation for this class was generated from the following file:

- gtest/gtest/internal/gtest-port.h

## 15.1042 testing::internal::TypeWithSize< 4 > Class Reference

### Public Types

- using **Int** = std::int32\_t
- using **UInt** = std::uint32\_t

The documentation for this class was generated from the following file:

- gtest/gtest/internal/gtest-port.h

## 15.1043 testing::internal::TypeWithSize< 8 > Class Reference

### Public Types

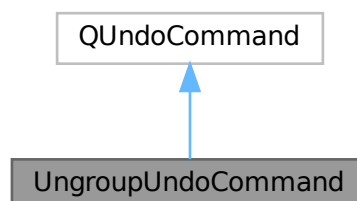
- using **Int** = std::int64\_t
- using **UInt** = std::uint64\_t

The documentation for this class was generated from the following file:

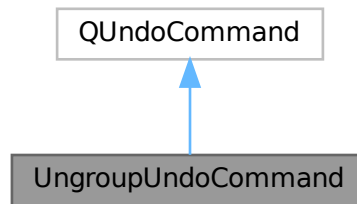
- gtest/gtest/internal/gtest-port.h

## 15.1044 UngroupUndoCommand Class Reference

Inheritance diagram for UngroupUndoCommand:



Collaboration diagram for UngroupUndoCommand:



### Public Member Functions

- **UngroupUndoCommand** (QGraphicsItemGroup \*group, [ModelGraphicsScene](#) \*scene, QUndoCommand \*parent=nullptr)
- void **undo** () override
- void **redo** () override

The documentation for this class was generated from the following files:

- applications/gui/genesys/actions/UngroupUndoCommand.h
- applications/gui/genesys/actions/UngroupUndoCommand.cpp

## 15.1045 testing::UnitTest Class Reference

### Public Member Functions

- int **Run** () GTEST\_MUST\_USE\_RESULT\_
- const char \* **original\_working\_dir** () const
- const [TestSuite](#) \* **current\_test\_suite** () const GTEST\_LOCK\_EXCLUDED\_(mutex\_)
- const [TestCase](#) \* **current\_test\_case** () const GTEST\_LOCK\_EXCLUDED\_(mutex\_)
- const [TestInfo](#) \* **current\_test\_info** () const GTEST\_LOCK\_EXCLUDED\_(mutex\_)
- int **random\_seed** () const
- [internal::ParameterizedTestSuiteRegistry](#) & **parameterized\_test\_registry** () GTEST\_LOCK\_EXCLUDED\_(mutex\_)
- int **successful\_test\_suite\_count** () const
- int **failed\_test\_suite\_count** () const
- int **total\_test\_suite\_count** () const
- int **test\_suite\_to\_run\_count** () const
- int **successful\_test\_case\_count** () const
- int **failed\_test\_case\_count** () const
- int **total\_test\_case\_count** () const
- int **test\_case\_to\_run\_count** () const
- int **successful\_test\_count** () const
- int **skipped\_test\_count** () const
- int **failed\_test\_count** () const
- int **reportable\_disabled\_test\_count** () const
- int **disabled\_test\_count** () const
- int **reportable\_test\_count** () const
- int **total\_test\_count** () const
- int **test\_to\_run\_count** () const

- `TimeInMillis` **start\_timestamp** () const
- `TimeInMillis` **elapsed\_time** () const
- `bool` **Passed** () const
- `bool` **Failed** () const
- `const` `TestSuite` \* **GetTestSuite** (int i) const
- `const` `TestCase` \* **GetTestCase** (int i) const
- `const` `TestResult` & **ad\_hoc\_test\_result** () const
- `TestEventListeners` & **listeners** ()

#### Static Public Member Functions

- `static` `UnitTest` \* **GetInstance** ()

#### Friends

- `class` **ScopedTrace**
- `class` **Test**
- `class` **internal::AssertHelper**
- `class` **internal::StreamingListenerTest**
- `class` **internal::UnitTestRecordPropertyTestHelper**
- `Environment` \* **AddGlobalTestEnvironment** (`Environment` \*env)
- `std::set< std::string >` \* **internal::GetIgnoredParameterizedTestSuites** ()
- `internal::UnitTestImpl` \* **internal::GetUnitTestImpl** ()
- `void` **internal::ReportFailureInUnknownLocation** (`TestPartResult::Type` result\_type, const `std::string` &message)

The documentation for this class was generated from the following files:

- `gtest/gtest/gtest.h`
- `gtest/src/gtest.cc`

## 15.1046 testing::internal::UnitTestImpl Class Reference

#### Public Types

- `enum` **ReactionToSharding** { **HONOR\_SHARDING\_PROTOCOL** , **IGNORE\_SHARDING\_PROTOCOL** }

#### Public Member Functions

- `UnitTestImpl` (`UnitTest` \*parent)
- `TestPartResultReporterInterface` \* **GetGlobalTestPartResultReporter** ()
- `void` **SetGlobalTestPartResultReporter** (`TestPartResultReporterInterface` \*reporter)
- `TestPartResultReporterInterface` \* **GetTestPartResultReporterForCurrentThread** ()
- `void` **SetTestPartResultReporterForCurrentThread** (`TestPartResultReporterInterface` \*reporter)
- `int` **successful\_test\_suite\_count** () const
- `int` **failed\_test\_suite\_count** () const
- `int` **total\_test\_suite\_count** () const
- `int` **test\_suite\_to\_run\_count** () const
- `int` **successful\_test\_count** () const
- `int` **skipped\_test\_count** () const
- `int` **failed\_test\_count** () const
- `int` **reportable\_disabled\_test\_count** () const
- `int` **disabled\_test\_count** () const
- `int` **reportable\_test\_count** () const
- `int` **total\_test\_count** () const
- `int` **test\_to\_run\_count** () const
- `TimeInMillis` **start\_timestamp** () const

- `TimeInMillis` **elapsed\_time** () const
- `bool` **Passed** () const
- `bool` **Failed** () const
- `const TestSuite *` **GetTestSuite** (int i) const
- `const TestCase *` **GetTestCase** (int i) const
- `TestSuite *` **GetMutableSuiteCase** (int i)
- `TestEventListeners *` **listeners** ()
- `TestResult *` **current\_test\_result** ()
- `const TestResult *` **ad\_hoc\_test\_result** () const
- `void` **set\_os\_stack\_trace\_getter** (`OsStackTraceGetterInterface *`getter)
- `OsStackTraceGetterInterface *` **os\_stack\_trace\_getter** ()
- `std::string` **CurrentOsStackTraceExceptTop** (int skip\_count) `GTEST_NO_INLINE_ GTEST_NO_TAIL_↵`  
`CALL_`
- `TestSuite *` **GetTestSuite** (const char \*test\_suite\_name, const char \*type\_param, `internal::SetUpTestSuite↵`  
`Func` set\_up\_tc, `internal::TearDownTestSuiteFunc` tear\_down\_tc)
- `TestCase *` **GetTestCase** (const char \*test\_case\_name, const char \*type\_param, `internal::SetUpTestSuite↵`  
`Func` set\_up\_tc, `internal::TearDownTestSuiteFunc` tear\_down\_tc)
- `void` **AddTestInfo** (`internal::SetUpTestSuiteFunc` set\_up\_tc, `internal::TearDownTestSuiteFunc` tear\_down\_tc,  
`TestInfo *`test\_info)
- `internal::ParameterizedTestSuiteRegistry` & **parameterized\_test\_registry** ()
- `std::set< std::string > *` **ignored\_parameterized\_test\_suites** ()
- `internal::TypeParameterizedTestSuiteRegistry` & **type\_parameterized\_test\_registry** ()
- `void` **set\_current\_test\_suite** (`TestSuite *`a\_current\_test\_suite)
- `void` **set\_current\_test\_info** (`TestInfo *`a\_current\_test\_info)
- `void` **RegisterParameterizedTests** ()
- `bool` **RunAllTests** ()
- `void` **ClearNonAdHocTestResult** ()
- `void` **ClearAdHocTestResult** ()
- `void` **RecordProperty** (const `TestProperty` &test\_property)
- `int` **FilterTests** (`ReactionToSharding` shard\_tests)
- `void` **ListTestsMatchingFilter** ()
- `const TestSuite *` **current\_test\_suite** () const
- `TestInfo *` **current\_test\_info** ()
- `const TestInfo *` **current\_test\_info** () const
- `std::vector< Environment * > &` **environments** ()
- `std::vector< TraceInfo > &` **gtest\_trace\_stack** ()
- `const std::vector< TraceInfo > &` **gtest\_trace\_stack** () const
- `void` **ConfigureXmlOutput** ()
- `void` **PostFlagParsingInit** ()
- `int` **random\_seed** () const
- `internal::Random *` **random** ()
- `void` **ShuffleTests** ()
- `void` **UnshuffleTests** ()
- `bool` **catch\_exceptions** () const

## Friends

- `class` `::testing::UnitTest`

The documentation for this class was generated from the following files:

- `gtest/src/gtest-internal-inl.h`
- `gtest/src/gtest.cc`

## 15.1047 testing::internal::UnitOptions Class Reference

### Static Public Member Functions

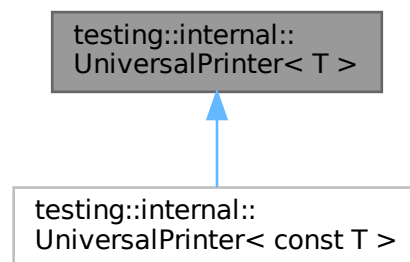
- static std::string **GetOutputFormat** ()
- static std::string **GetAbsolutePathToOutputFile** ()
- static bool **FilterMatchesTest** (const std::string &test\_suite\_name, const std::string &test\_name)
- static bool **MatchesFilter** (const std::string &name, const char \*filter)

The documentation for this class was generated from the following files:

- gtest/src/gtest-internal-inl.h
- gtest/src/gtest.cc

## 15.1048 testing::internal::UniversalPrinter< T > Class Template Reference

Inheritance diagram for testing::internal::UniversalPrinter< T >:



### Static Public Member Functions

- static void **Print** (const T &value, ::std::ostream \*os)

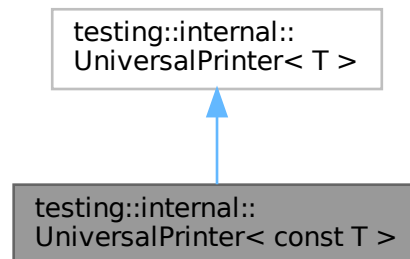
The documentation for this class was generated from the following file:

- gtest/gtest/gtest-printers.h

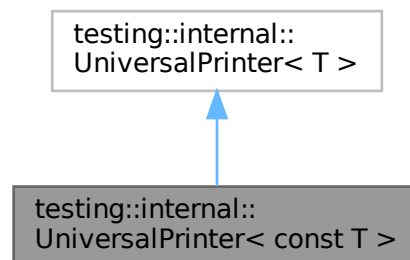


## 15.1049 testing::internal::UniversalPrinter< const T > Class Template Reference

Inheritance diagram for testing::internal::UniversalPrinter< const T >:



Collaboration diagram for testing::internal::UniversalPrinter< const T >:



### Additional Inherited Members

Static Public Member Functions inherited from [testing::internal::UniversalPrinter< T >](#)

- static void **Print** (const T &value, ::std::ostream \*os)

The documentation for this class was generated from the following file:

- gtest/gtest/gtest-printers.h

## 15.1050 testing::internal::UniversalPrinter< T & > Class Template Reference

### Static Public Member Functions

- static void **Print** (const T &value, ::std::ostream \*os)

The documentation for this class was generated from the following file:

- gtest/gtest/gtest-printers.h

## 15.1051 `testing::internal::UniversalPrinter< T[N]>` Class Template Reference

### Static Public Member Functions

- static void **Print** (const T(&a)[N], ::std::ostream \*os)

The documentation for this class was generated from the following file:

- gtest/gtest/gtest-printers.h

## 15.1052 `testing::internal::UniversalTersePrinter< T >` Class Template Reference

### Static Public Member Functions

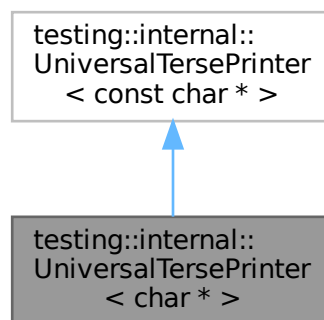
- static void **Print** (const T &value, ::std::ostream \*os)

The documentation for this class was generated from the following file:

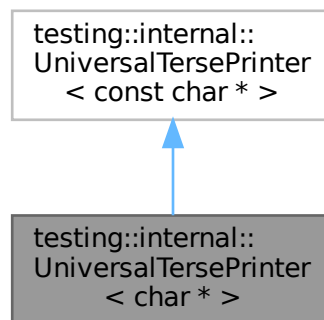
- gtest/gtest/gtest-printers.h

## 15.1053 `testing::internal::UniversalTersePrinter< char * >` Class Reference

Inheritance diagram for `testing::internal::UniversalTersePrinter< char * >`:



Collaboration diagram for testing::internal::UniversalTersePrinter< char \* >:



#### Additional Inherited Members

#### Static Public Member Functions inherited from

[testing::internal::UniversalTersePrinter< const char \\* >](#)

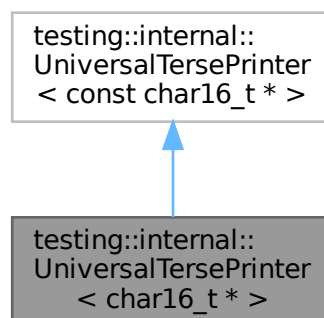
- static void **Print** (const char \*str, ::std::ostream \*os)

The documentation for this class was generated from the following file:

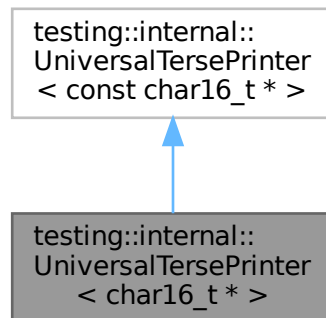
- gtest/gtest/gtest-printers.h

## 15.1054 testing::internal::UniversalTersePrinter< char16\_t \* > Class Reference

Inheritance diagram for testing::internal::UniversalTersePrinter< char16\_t \* >:



Collaboration diagram for `testing::internal::UniversalTersePrinter< char16_t * >`:



#### Additional Inherited Members

#### Static Public Member Functions inherited from

[`testing::internal::UniversalTersePrinter< const char16\_t \* >`](#)

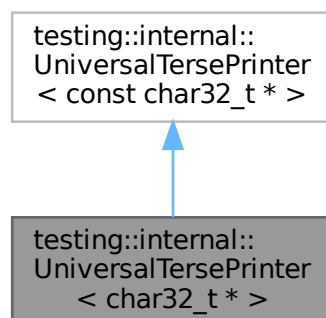
- static void **Print** (const char16\_t \*str, ::std::ostream \*os)

The documentation for this class was generated from the following file:

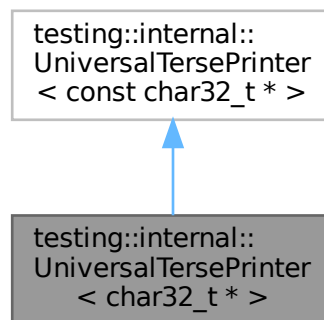
- `gtest/gtest/gtest-printers.h`

## 15.1055 `testing::internal::UniversalTersePrinter< char32_t * >` Class Reference

Inheritance diagram for `testing::internal::UniversalTersePrinter< char32_t * >`:



Collaboration diagram for testing::internal::UniversalTersePrinter< char32\_t \* >:



#### Additional Inherited Members

#### Static Public Member Functions inherited from

[testing::internal::UniversalTersePrinter< const char32\\_t \\* >](#)

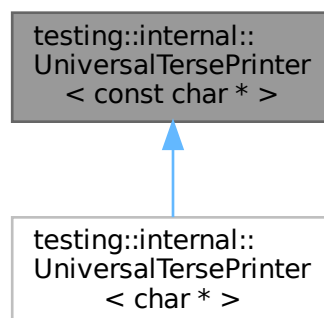
- static void **Print** (const char32\_t \*str, ::std::ostream \*os)

The documentation for this class was generated from the following file:

- gtest/gtest/gtest-printers.h

## 15.1056 testing::internal::UniversalTersePrinter< const char \* > Class Reference

Inheritance diagram for testing::internal::UniversalTersePrinter< const char \* >:



#### Static Public Member Functions

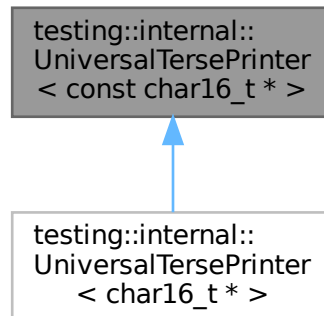
- static void **Print** (const char \*str, ::std::ostream \*os)

The documentation for this class was generated from the following file:

- gtest/gtest/gtest-printers.h

## 15.1057 testing::internal::UniversalTersePrinter< const char16\_t \* > Class Reference

Inheritance diagram for testing::internal::UniversalTersePrinter< const char16\_t \* >:



### Static Public Member Functions

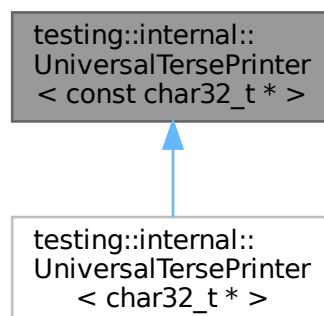
- static void **Print** (const char16\_t \*str, ::std::ostream \*os)

The documentation for this class was generated from the following file:

- gtest/gtest/gtest-printers.h

## 15.1058 testing::internal::UniversalTersePrinter< const char32\_t \* > Class Reference

Inheritance diagram for testing::internal::UniversalTersePrinter< const char32\_t \* >:



**Static Public Member Functions**

- static void **Print** (const char32\_t \*str, ::std::ostream \*os)

The documentation for this class was generated from the following file:

- gtest/gtest/gtest-printers.h

## 15.1059 testing::internal::UniversalTersePrinter< T & > Class Template Reference

**Static Public Member Functions**

- static void **Print** (const T &value, ::std::ostream \*os)

The documentation for this class was generated from the following file:

- gtest/gtest/gtest-printers.h

## 15.1060 testing::internal::UniversalTersePrinter< T[N]> Class Template Reference

**Static Public Member Functions**

- static void **Print** (const T(&value)[N], ::std::ostream \*os)

The documentation for this class was generated from the following file:

- gtest/gtest/gtest-printers.h

## 15.1061 testing::internal::UniversalTersePrinter< wchar\_t \* > Class Reference

**Static Public Member Functions**

- static void **Print** (wchar\_t \*str, ::std::ostream \*os)

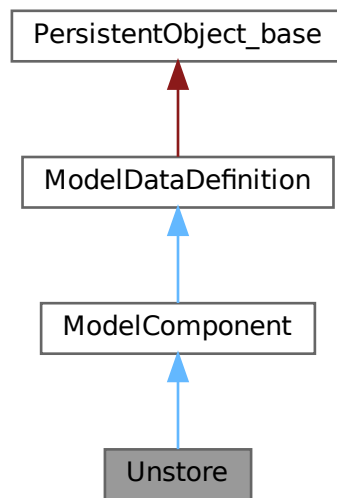
The documentation for this class was generated from the following file:

- gtest/gtest/gtest-printers.h

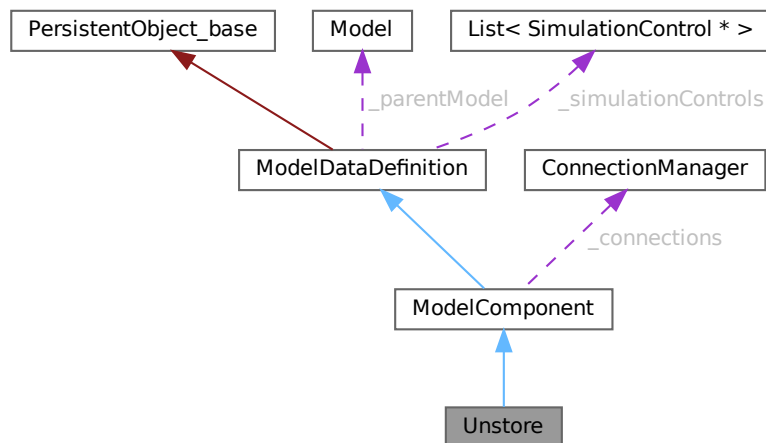
## 15.1062 Unstore Class Reference

```
#include <plugins/components/MaterialHandling/Unstore.h>
```

Inheritance diagram for Unstore:



Collaboration diagram for Unstore:



### Public Member Functions

- **Unstore** ([Model](#) \*model, std::string name="")
  - virtual std::string [show](#) () override
- virtual public methods*

### Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) \*model, std::string componentTypename, std::string name="")



*constructors*

- virtual **~ModelComponent** ()  
*Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) \* [getConnectionManager](#) () const  
*new public user methods for this component*
- void [connectTo](#) ([ModelComponent](#) \*component, unsigned int inputPortNumber=0)  
*Connects this component to another component.*
- bool [hasBreakpointAt](#) ()  
*Indicates whether a simulation breakpoint is set on this component.*
- void [setDescription](#) (std::string \_description)  
*Updates the free-form description shown by the GUI.*
- std::string [getDescription](#) () const  
*Returns the free-form description associated with this component.*
- void [remove](#) ([ModelComponent](#) \*comp)  
*Removes a connected component from this component.*

## Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) \*model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)  
*Creates a model data definition and optionally inserts it into the model.*
- virtual **~ModelDataDefinition** ()  
*Releases resources owned by this data definition.*
- Util::identification [getId](#) () const  
*Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)  
*Updates the logical name of this instance.*
- const std::string & [getName](#) () const  
*Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)  
*Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const  
*Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const  
*Returns the runtime class name used by the plugin system.*
- [Model](#) \* [getParentModel](#) () const  
*Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const  
*Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)  
*Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) \* [getInternalData](#) (const std::string &name) const  
*Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getInternalData](#) () const  
*Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getAttachedData](#) () const  
*Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const  
*Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))  
*Updates the changed flag for this data definition.*

- unsigned int [getLevel](#) () const  
*Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int \_modelLevel)  
*Sets the model nesting level for this definition.*
- [List](#)< [SimulationControl](#) \* > \* [getSimulationControls](#) () const  
*Returns the simulation controls owned by this data definition.*
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const  
*Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)  
*Updates the trace level used by this definition.*
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecific←Enabled=true)  
*Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const  
*Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)  
*Enables or disables the specific trace level.*

### Static Public Member Functions

- static [PluginInformation](#) \* [GetPluginInformation](#) ()
- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")

### Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool [Check](#) ([ModelComponent](#) \*component)  
*Validates a component instance.*
- static void [CreateInternalData](#) ([ModelComponent](#) \*component)  
*Creates internal data required by a component instance.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelComponent](#) \*component)  
*Serializes a component instance into a persistence record.*
- static void [DispatchEvent](#) ([Event](#) \*event)  
*Dispatches a simulation event to the target component.*

### Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool [Check](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*
- static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

### Protected Member Functions

- virtual void `_onDispatchEvent` (`Entity` \*entity, unsigned int inputPortNumber) override  
*Handles a dispatched event for this component type.*
- virtual bool `_loadInstance` (`PersistenceRecord` \*fields) override  
*virtual protected method that must be overridden*
- virtual void `_saveInstance` (`PersistenceRecord` \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*
- virtual bool `_check` (std::string &errorMessage) override  
*could be overridden by derived classes*

### Protected Member Functions inherited from `ModelDataDefinition`

- bool `_getSaveDefaultsOption` ()
- virtual `ParserChangesInformation` \* `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` (`SimulationControl` \*control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()  
*legacy helper methods used by the granular create\* hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, `ModelDataDefinition` \*child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, `ModelDataDefinition` \*data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, `ModelDataDefinition` \*data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, `ModelDataDefinition` \*data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` \*data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` \*data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` \*data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` \*data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

*method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)  
*not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- void [traceSimulation](#) (void \*thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)  
*Traces text to the simulation output.*
- void **traceSimulation** (void \*thisobject, TraceManager::Level level, const std::string &text)

### Additional Inherited Members

### Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) \* **\_connections** = new [ConnectionManager](#)()  
*virtual protected methods that could be overridden by derived classes, if needed*

### Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **\_id**
- std::string **\_typename**
- bool **\_reportStatistics**
- bool **\_hasChanged**
- unsigned int **\_modelLevel** = 0
- [Model](#) \* **\_parentModel**
- [List](#)< [SimulationControl](#) \* > \* **\_simulationControls** = new [List](#)<[SimulationControl](#)\*>()
- TraceManager::Level **\_traceLevelSpecific**
- bool **\_traceLevelSpecificEnabled** = false

## 15.1062.1 Detailed Description

[Unstore](#) module DESCRIPTION The [Unstore](#) module removes an entity from storage. When an entity arrives at the [Unstore](#) module, the storage specified is decreased and the entity immediately moves to the next module in the model. TYPICAL USES Removing the entity from an animation location when processing is complete Tracking the number of customers within a grocery store (unstore upon exit) PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Type Method of specifying the storage name as a [Storage](#), [Set](#), [Attribute](#), or Expression. Default will remove an entity from the last storage that it entered. [Storage](#) Name Name of the storage to which the entity will be added. Applies only when the Type is [Storage](#). [Set](#) Name Name of the storage set from which the entity is to be selected. Applies only when the Type is [Set](#). [Set](#) Index Index into the defined storage set that contains the desired storage name. Applies only when the Type is [Set](#). [Attribute](#) Name of the attribute whose value contains the storage. Applies only when the Type is [Attribute](#). Expression Expression that is evaluated to the storage into which the entity is placed. Applies only when the Type is Expression.

## 15.1062.2 Member Function Documentation

### 15.1062.2.1 [\\_check\(\)](#)

```
bool Unstore::_check (
 std::string & errorMessage) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

### 15.1062.2.2 `_loadInstance()`

```
bool Unstore::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

### 15.1062.2.3 `_onDispatchEvent()`

```
void Unstore::_onDispatchEvent (
 Entity * entity,
 unsigned int inputPortNumber) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

#### Parameters

|                        |                                         |
|------------------------|-----------------------------------------|
| <i>entity</i>          | <a href="#">Entity</a> being processed. |
| <i>inputPortNumber</i> | Input port where the entity arrived.    |

Implements [ModelComponent](#).

### 15.1062.2.4 `_saveInstance()`

```
void Unstore::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

#### Parameters

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [ModelComponent](#).

### 15.1062.2.5 `show()`

```
std::string Unstore::show () [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

#### Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- `plugins/components/MaterialHandling/Unstore.h`
- `plugins/components/MaterialHandling/Unstore.cpp`

## 15.1063 Util Class Reference

### Classes

- class [CommandResult](#)

## Public Types

- enum class [TimeUnit](#) : int {  
**unknown** = 0 , **picosecond** = 1 , **nanosecond** = 2 , **microsecond** = 3 ,  
**millisecond** = 4 , **second** = 5 , **minute** = 6 , **hour** = 7 ,  
**day** = 8 , **week** = 9 , **num\_elements** = 10 }  
*The TimeUnit enum.*
- enum class **AllocationType** : int {  
**ValueAdded** = 0 , **NonValueAdded** = 1 , **Transfer** = 2 , **Wait** = 3 ,  
**Others** = 4 , **num\_elements** = 5 }
- enum class **TimeFormat** : unsigned int { **twelve** = 12 , **twentyFour** = 24 }
- typedef unsigned long **identification**
- typedef unsigned int **rank**

## Static Public Member Functions

- static std::string [StrTimeUnitShort](#) (Util::TimeUnit timeUnit)  
*Returns the short textual symbol for a time unit.*
- static std::string [StrTimeUnitLong](#) (Util::TimeUnit timeUnit)  
*Returns the long textual label for a time unit.*
- static std::string [convertEnumToStr](#) (Util::TimeUnit timeUnit)  
*Converts a time unit enum value to its canonical string representation.*
- static std::string [StrAllocation](#) (Util::AllocationType allocation)  
*Returns the string representation for an allocation category.*
- static std::string [convertEnumToStr](#) (Util::AllocationType allocation)  
*Converts an allocation category enum value to a string.*
- static void [SetIndent](#) (const unsigned short indent)  
*Sets the indentation level used by [Indent\(\)](#).*
- static void **IncIndent** ()  
*Increments the current indentation level.*
- static void **DecIndent** ()  
*Decrements the current indentation level.*
- static void [SepKeyVal](#) (std::string str, std::string &key, std::string &value)  
*Splits a key-value string around the first = character.*
- static std::string [Indent](#) ()  
*Returns the current indentation prefix.*
- static std::string [SetW](#) (std::string text, unsigned short width)  
*Pads or truncates text to a fixed width.*
- static std::string [StrTruncfInt](#) (double value)  
*Formats a numeric value without losing integer semantics.*
- static std::string [StrTruncfInt](#) (std::string strValue)  
*Removes a trailing .000000 suffix when present.*
- static std::string [Trim](#) (std::string str)  
*Trims leading and trailing whitespace.*
- static std::string [StrReplace](#) (std::string text, std::string searchFor, std::string replaceBy)  
*Replaces all occurrences of one substring with another.*
- static std::string [StrReplaceSpecialChars](#) (std::string text)  
*Sanitizes a text for identifier-like usage.*
- static std::string [StrIndex](#) (int index)  
*Formats an integer index in bracket notation.*
- static void [Trimwithin](#) (std::string &str)  
*Removes all spaces from a string in place.*
- static std::string [Map2Str](#) (std::map< std::string, std::string > \*mapss)

- Serializes a string-to-string map to a compact text form.*
- static std::string [Map2str](#) (std::map< std::string, double > \*mapss)  
*Serializes a string-to-double map to a compact text form.*
- static std::string [List2str](#) (std::list< unsigned int > \*list)  
*Serializes a list of unsigned integers to a compact text form.*
- static Util::identification [GenerateNewId](#) ()  
*Generates a globally unique identifier.*
- static Util::identification [GenerateNewIdOfType](#) (std::string objtype)  
*Generates the next identifier for a given runtime type name.*
- static Util::identification [GetLastIdOfType](#) (std::string objtype)  
*Returns the most recently generated identifier for a type.*
- static void [ResetIdOfType](#) (std::string objtype)  
*Resets the identifier counter for a specific type.*
- static void [ResetAllIds](#) ()  
*Resets all identifier counters maintained by the utility class.*
- static double [TimeUnitConvert](#) (Util::TimeUnit timeUnit1, Util::TimeUnit timeUnit2)  
*Computes the multiplicative factor between two time units.*
- static char [DirSeparator](#) ()  
*Returns the platform directory separator character.*
- static std::string [FilenameFromFullFilename](#) (const std::string &s)  
*Extracts the file name from a full path.*
- static void [FileDelete](#) (const std::string &filename)  
*Deletes a file if it exists.*
- static std::string [PathFromFullFilename](#) (const std::string &s)  
*Extracts the directory path from a full filename.*
- static std::string [RunningPath](#) ()  
*Returns the directory that contains the running binary.*
- static std::vector< std::string > [ListFiles](#) (std::string dir, std::string fileFilter="", mode\_t attribFilter=S\_IFREG &S\_IFDIR)  
*Lists files in a directory using an optional substring filter.*
- static bool [FileExists](#) (const std::string &name)  
*Checks whether a file can be opened for reading.*
- static Util::CommandResult [ExecuteCommand](#) (std::string command)  
*Executes a shell command and captures the result.*
- template<typename T >  
static std::string [TypeOf](#) ()  
*Returns the compiler type name used for T.*
- template<class T >  
static Util::identification [GenerateNewIdOfType](#) ()  
*Generates the next identifier in the namespace of T.*

## 15.1063.1 Member Function Documentation

### 15.1063.1.1 convertEnumToStr() [1/2]

```
std::string Util::convertEnumToStr (
 Util::AllocationType allocation) [static]
```

Converts an allocation category enum value to a string.

#### Parameters

|                   |                                 |
|-------------------|---------------------------------|
| <i>allocation</i> | Allocation category to convert. |
|-------------------|---------------------------------|

**Returns**

Canonical string representation for the enum value.

**15.1063.1.2 convertEnumToStr() [2/2]**

```
std::string Util::convertEnumToStr (
 Util::TimeUnit timeUnit) [static]
```

Converts a time unit enum value to its canonical string representation.

**Parameters**

|                 |                       |
|-----------------|-----------------------|
| <i>timeUnit</i> | Time unit to convert. |
|-----------------|-----------------------|

**Returns**

Canonical string representation for the enum value.

**15.1063.1.3 DirSeparator()**

```
char Util::DirSeparator () [static]
```

Returns the platform directory separator character.

**Returns**

Directory separator for the current platform.

**15.1063.1.4 ExecuteCommand()**

```
Util::CommandResult Util::ExecuteCommand (
 std::string command) [static]
```

Executes a shell command and captures the result.

**Parameters**

|                |                      |
|----------------|----------------------|
| <i>command</i> | Command line to run. |
|----------------|----------------------|

**Returns**

Command execution outcome and captured output buffers.

**15.1063.1.5 FileDelete()**

```
void Util::FileDelete (
 const std::string & filename) [static]
```

Deletes a file if it exists.

**Parameters**

|                 |                             |
|-----------------|-----------------------------|
| <i>filename</i> | Path to the file to delete. |
|-----------------|-----------------------------|

**15.1063.1.6 FileExists()**

```
bool Util::FileExists (
 const std::string & name) [static]
```

Checks whether a file can be opened for reading.



## Parameters

|             |                    |
|-------------|--------------------|
| <i>name</i> | File path to test. |
|-------------|--------------------|

## Returns

true if the file exists and can be opened.

**15.1063.1.7 FilenameFromFullFilename()**

```
std::string Util::FilenameFromFullFilename (
 const std::string & s) [static]
```

Extracts the file name from a full path.

## Parameters

|          |                 |
|----------|-----------------|
| <i>s</i> | Full file path. |
|----------|-----------------|

## Returns

Trailing file name portion.

**15.1063.1.8 GenerateNewId()**

```
Util::identification Util::GenerateNewId () [static]
```

Generates a globally unique identifier.

## Returns

Next process-wide identifier value.

**15.1063.1.9 GenerateNewIdOfType() [1/2]**

```
template<class T >
static Util::identification Util::GenerateNewIdOfType () [inline], [static]
```

Generates the next identifier in the namespace of T.

## Template Parameters

|          |                                                       |
|----------|-------------------------------------------------------|
| <i>T</i> | Type whose namespace should receive a new identifier. |
|----------|-------------------------------------------------------|

## Returns

Next sequential identifier for the type.

**15.1063.1.10 GenerateNewIdOfType() [2/2]**

```
Util::identification Util::GenerateNewIdOfType (
 std::string objtype) [static]
```

Generates the next identifier for a given runtime type name.

## Parameters

|                |                                         |
|----------------|-----------------------------------------|
| <i>objtype</i> | Type name used as identifier namespace. |
|----------------|-----------------------------------------|

**Returns**

Next identifier within the requested type namespace.

**15.1063.1.11 GetLastIdOfType()**

```
Util::identification Util::GetLastIdOfType (
 std::string objtype) [static]
```

Returns the most recently generated identifier for a type.

**Parameters**

|                |                                         |
|----------------|-----------------------------------------|
| <i>objtype</i> | Type name used as identifier namespace. |
|----------------|-----------------------------------------|

**Returns**

Last identifier value for the requested type.

**15.1063.1.12 Indent()**

```
std::string Util::Indent () [static]
```

Returns the current indentation prefix.

**Returns**

A string made of repeated indentation markers.

**15.1063.1.13 List2str()**

```
std::string Util::List2str (
 std::list< unsigned int > * list) [static]
```

Serializes a list of unsigned integers to a compact text form.

**Parameters**

|             |                    |
|-------------|--------------------|
| <i>list</i> | List to serialize. |
|-------------|--------------------|

**Returns**

One-line representation of the list contents.

**15.1063.1.14 ListFiles()**

```
std::vector< std::string > Util::ListFiles (
 std::string dir,
 std::string fileFilter = "",
 mode_t attribFilter = S_IFREG & S_IFDIR) [static]
```

Lists files in a directory using an optional substring filter.

**Parameters**

|                     |                                                      |
|---------------------|------------------------------------------------------|
| <i>dir</i>          | Directory to scan.                                   |
| <i>fileFilter</i>   | Optional substring filter for file names.            |
| <i>attribFilter</i> | Attribute mask applied to <code>stat</code> results. |

**Returns**

Matching file names found in the directory.

**15.1063.1.15 Map2str() [1/2]**

```
std::string Util::Map2str (
 std::map< std::string, double > * mapss) [static]
```

Serializes a string-to-double map to a compact text form.

**Parameters**

|              |                   |
|--------------|-------------------|
| <i>mapss</i> | Map to serialize. |
|--------------|-------------------|

**Returns**

One-line representation of the map contents.

**15.1063.1.16 Map2str() [2/2]**

```
std::string Util::Map2str (
 std::map< std::string, std::string > * mapss) [static]
```

Serializes a string-to-string map to a compact text form.

**Parameters**

|              |                   |
|--------------|-------------------|
| <i>mapss</i> | Map to serialize. |
|--------------|-------------------|

**Returns**

One-line representation of the map contents.

**15.1063.1.17 PathFromFullFilename()**

```
std::string Util::PathFromFullFilename (
 const std::string & s) [static]
```

Extracts the directory path from a full filename.

**Parameters**

|          |                 |
|----------|-----------------|
| <i>s</i> | Full file path. |
|----------|-----------------|

**Returns**

Directory portion of the path.

**15.1063.1.18 ResetIdOfType()**

```
void Util::ResetIdOfType (
 std::string objtype) [static]
```

Resets the identifier counter for a specific type.

**Parameters**

|                |                     |
|----------------|---------------------|
| <i>objtype</i> | Type name to reset. |
|----------------|---------------------|

### 15.1063.1.19 RunningPath()

```
std::string Util::RunningPath () [static]
```

Returns the directory that contains the running binary.

#### Returns

Current executable directory.

### 15.1063.1.20 SepKeyVal()

```
void Util::SepKeyVal (
 std::string str,
 std::string & key,
 std::string & value) [static]
```

Splits a key-value string around the first = character.

#### Parameters

|              |                                    |
|--------------|------------------------------------|
| <i>str</i>   | Source text in the form key=value. |
| <i>key</i>   | Output key portion.                |
| <i>value</i> | Output value portion.              |

### 15.1063.1.21 SetIndent()

```
void Util::SetIndent (
 const unsigned short indent) [static]
```

Sets the indentation level used by [Indent \(\)](#).

#### Parameters

|               |                        |
|---------------|------------------------|
| <i>indent</i> | New indentation depth. |
|---------------|------------------------|

### 15.1063.1.22 SetW()

```
std::string Util::SetW (
 std::string text,
 unsigned short width) [static]
```

Pads or truncates text to a fixed width.

#### Parameters

|              |                 |
|--------------|-----------------|
| <i>text</i>  | Text to adjust. |
| <i>width</i> | Target width.   |

#### Returns

Text resized to the requested width.

### 15.1063.1.23 StrAllocation()

```
std::string Util::StrAllocation (
 Util::AllocationType allocation) [static]
```

Returns the string representation for an allocation category.

## Parameters

|                   |                                 |
|-------------------|---------------------------------|
| <i>allocation</i> | Allocation category to convert. |
|-------------------|---------------------------------|

## Returns

Allocation label used in reports and logs.

**15.1063.1.24 StrIndex()**

```
std::string Util::StrIndex (
 int index) [static]
```

Formats an integer index in bracket notation.

## Parameters

|              |                  |
|--------------|------------------|
| <i>index</i> | Index to format. |
|--------------|------------------|

## Returns

Text such as [4].

**15.1063.1.25 StrReplace()**

```
std::string Util::StrReplace (
 std::string text,
 std::string searchFor,
 std::string replaceBy) [static]
```

Replaces all occurrences of one substring with another.

## Parameters

|                  |                       |
|------------------|-----------------------|
| <i>text</i>      | Input text.           |
| <i>searchFor</i> | Substring to replace. |
| <i>replaceBy</i> | Replacement text.     |

## Returns

Updated text.

**15.1063.1.26 StrReplaceSpecialChars()**

```
std::string Util::StrReplaceSpecialChars (
 std::string text) [static]
```

Sanitizes a text for identifier-like usage.

## Parameters

|             |             |
|-------------|-------------|
| <i>text</i> | Input text. |
|-------------|-------------|

## Returns

Text with spaces and selected punctuation normalized.

**15.1063.1.27 StrTimeUnitLong()**

```
std::string Util::StrTimeUnitLong (
 Util::TimeUnit timeUnit) [static]
```

Returns the long textual label for a time unit.

**Parameters**

|                 |                       |
|-----------------|-----------------------|
| <i>timeUnit</i> | Time unit to convert. |
|-----------------|-----------------------|

**Returns**

Long label such as `millisecond` or `hour`.

**15.1063.1.28 StrTimeUnitShort()**

```
std::string Util::StrTimeUnitShort (
 Util::TimeUnit timeUnit) [static]
```

Returns the short textual symbol for a time unit.

**Parameters**

|                 |                       |
|-----------------|-----------------------|
| <i>timeUnit</i> | Time unit to convert. |
|-----------------|-----------------------|

**Returns**

Short symbol such as `ms` or `h`.

**15.1063.1.29 StrTruncIfInt() [1/2]**

```
std::string Util::StrTruncIfInt (
 double value) [static]
```

Formats a numeric value without losing integer semantics.

**Parameters**

|              |                  |
|--------------|------------------|
| <i>value</i> | Value to format. |
|--------------|------------------|

**Returns**

A string that preserves the decimal point for exact integers.

**15.1063.1.30 StrTruncIfInt() [2/2]**

```
std::string Util::StrTruncIfInt (
 std::string strValue) [static]
```

Removes a trailing `.000000` suffix when present.

**Parameters**

|                 |                                     |
|-----------------|-------------------------------------|
| <i>strValue</i> | String representation to normalize. |
|-----------------|-------------------------------------|

**Returns**

Normalized string value.

### 15.1063.1.31 TimeUnitConvert()

```
double Util::TimeUnitConvert (
 Util::TimeUnit timeUnit1,
 Util::TimeUnit timeUnit2) [static]
```

Computes the multiplicative factor between two time units.

#### Parameters

|                  |              |
|------------------|--------------|
| <i>timeUnit1</i> | Source unit. |
| <i>timeUnit2</i> | Target unit. |

#### Returns

Conversion factor from `timeUnit1` to `timeUnit2`.

### 15.1063.1.32 Trim()

```
std::string Util::Trim (
 std::string str) [static]
```

Trims leading and trailing whitespace.

#### Parameters

|            |               |
|------------|---------------|
| <i>str</i> | Input string. |
|------------|---------------|

#### Returns

Trimmed string.

### 15.1063.1.33 Trimwithin()

```
void Util::Trimwithin (
 std::string & str) [static]
```

Removes all spaces from a string in place.

#### Parameters

|            |                      |
|------------|----------------------|
| <i>str</i> | String to normalize. |
|------------|----------------------|

### 15.1063.1.34 TypeOf()

```
template<typename T >
static std::string Util::TypeOf () [inline], [static]
```

Returns the compiler type name used for `T`.

#### Template Parameters

|          |                  |
|----------|------------------|
| <i>T</i> | Type to inspect. |
|----------|------------------|

#### Returns

Compiler-provided type name, normalized when possible.

The documentation for this class was generated from the following files:

- kernel/util/Util.h
- kernel/util/Util.cpp

## 15.1064 yy::genesyspp\_parser::value\_type Class Reference

```
#include <parser/GenesysParser.h>
```

### Public Types

- typedef [value\\_type](#) **self\_type**  
*Type of \*this.*

### Public Member Functions

- **value\_type** () YY\_NOEXCEPT  
*Empty construction.*
- template<typename T >  
**value\_type** (YY\_RVREF(T) t)  
*Construct and fill.*
- ~**value\_type** () YY\_NOEXCEPT  
*Destruction, allowed only if empty.*
- template<typename T >  
T & **emplace** ()  
*Instantiate an empty T in here.*
- template<typename T >  
T & **emplace** (const T &t)  
*Instantiate a T in here from t.*
- template<typename T >  
T & **build** ()
- template<typename T >  
T & **build** (const T &t)
- template<typename T >  
T & **as** () YY\_NOEXCEPT  
*Accessor to a built T.*
- template<typename T >  
const T & **as** () const YY\_NOEXCEPT  
*Const accessor to a built T (for printer).*
- template<typename T >  
void **swap** ([self\\_type](#) &that) YY\_NOEXCEPT
- template<typename T >  
void **move** ([self\\_type](#) &that)
- template<typename T >  
void **copy** (const [self\\_type](#) &that)  
*Copy the content of that to this.*
- template<typename T >  
void **destroy** ()  
*Destroy the stored T.*

### 15.1064.1 Detailed Description

A buffer to store and retrieve objects.

Sort of a variant, but does not keep track of the nature of the stored data, since that knowledge is available via the current parser state.



## 15.1064.2 Member Function Documentation

### 15.1064.2.1 build() [1/2]

```
template<typename T >
T & yy::genesyspp_parser::value_type::build () [inline]
```

Instantiate an empty *T* in here. Obsolete, use `emplace`.

### 15.1064.2.2 build() [2/2]

```
template<typename T >
T & yy::genesyspp_parser::value_type::build (
 const T & t) [inline]
```

Instantiate a *T* in here from *t*. Obsolete, use `emplace`.

### 15.1064.2.3 move()

```
template<typename T >
void yy::genesyspp_parser::value_type::move (
 self_type & that) [inline]
```

Move the content of *that* to this.  
Destroys *that*.

### 15.1064.2.4 swap()

```
template<typename T >
void yy::genesyspp_parser::value_type::swap (
 self_type & that) [inline]
```

Swap the content with *that*, of same type.

Both variants must be built beforehand, because swapping the actual data requires reading it (with `as()`), and this is not possible on unconstructed variants: it would require some dynamic testing, which should not be the variant's responsibility. Swapping between built and (possibly) non-built is done with `self_type::move()`.

The documentation for this class was generated from the following file:

- parser/[GenesysParser.h](#)

## 15.1065 testing::internal::ValueArray< Ts > Class Template Reference

### Public Member Functions

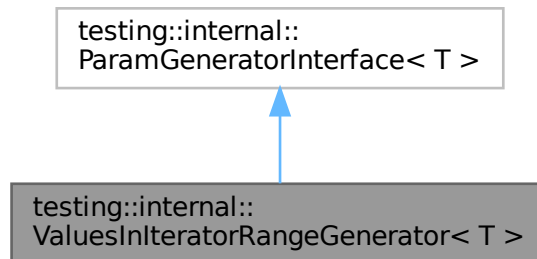
- **ValueArray** (Ts... v)
- `template<typename T >`  
**operator ParamGenerator< T > ()** const

The documentation for this class was generated from the following file:

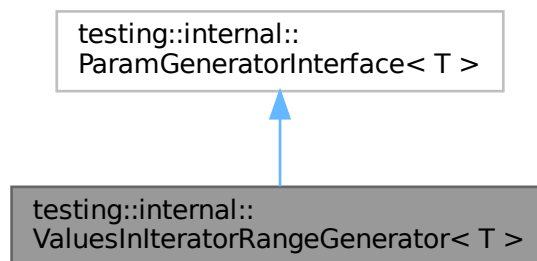
- gtest/gtest/internal/gtest-param-util.h

## 15.1066 testing::internal::ValuesInIteratorRangeGenerator< T > Class Template Reference

Inheritance diagram for testing::internal::ValuesInIteratorRangeGenerator< T >:



Collaboration diagram for testing::internal::ValuesInIteratorRangeGenerator< T >:



### Public Member Functions

- `template<typename ForwardIterator >`  
**ValuesInIteratorRangeGenerator** (ForwardIterator begin, ForwardIterator end)
- `ParamIteratorInterface< T > * Begin ()` const override
- `ParamIteratorInterface< T > * End ()` const override

### Additional Inherited Members

### Public Types inherited from `testing::internal::ParamGeneratorInterface< T >`

- `typedef T ParamType`

## 15.1066.1 Member Function Documentation

### 15.1066.1.1 Begin()

```
template<typename T >
```

```
ParamIteratorInterface< T > * testing::internal::ValuesInIteratorRangeGenerator< T >::Begin (
) const [inline], [override], [virtual]
Implements testing::internal::ParamGeneratorInterface< T >.
```

### 15.1066.1.2 End()

```
template<typename T >
ParamIteratorInterface< T > * testing::internal::ValuesInIteratorRangeGenerator< T >::End ()
const [inline], [override], [virtual]
Implements testing::internal::ParamGeneratorInterface< T >.
```

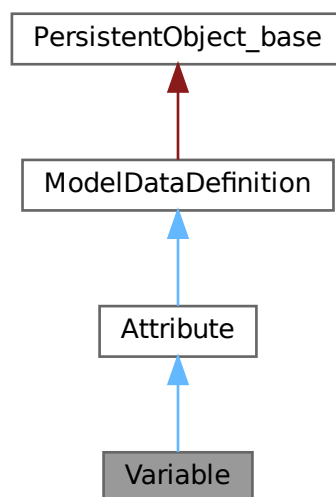
The documentation for this class was generated from the following file:

- gtest/gtest/internal/gtest-param-util.h

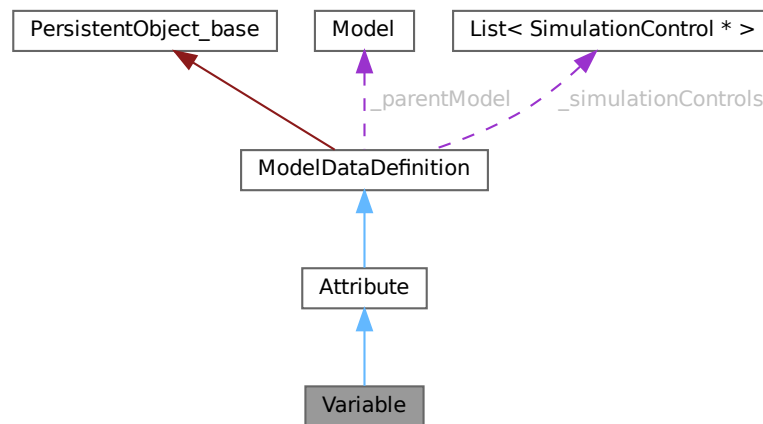
## 15.1067 Variable Class Reference

```
#include <plugins/data/Logic/Variable.h>
```

Inheritance diagram for Variable:



Collaboration diagram for Variable:



## Public Member Functions

- **Variable** (`Model *model`, `std::string name=""`)
- virtual `std::string show ()` override  
*Returns a textual representation of this data definition.*
- double **getValue** (`std::string index=""`)  
*Reads the current sparse value, returning 0.0 when the index is absent.*
- void **setValue** (`double value`, `std::string index=""`)  
*Writes the current sparse value at the scalar or indexed position.*
- void **copyValuesFrom** (`const Variable &source`)  
*Copies the full runtime sparse store from another variable.*
- `std::string getInitialValuesText ()` const override  
*Returns the initial values serialized in bracket notation.*
- void **setInitialValuesText** (`std::string valuesText`) override  
*Parses bracket notation and replaces both initial and runtime stores.*
- void **insertDimentionSize** (`unsigned int size`)  
*Appends one dimension size to the variable definition.*
- `std::map< std::string, double > * getValues ()` const  
*Returns mutable current sparse values for compatibility with existing callers.*
- `SparseValueStore * getValueStore ()`  
*Returns current value store used by this variable.*
- `ModelDataDefinition * get_scope ()`
- void **set\_scope** (`ModelDataDefinition *const scope`)

## Public Member Functions inherited from Attribute

- **Attribute** (`Model *model`, `std::string name=""`, `std::string dataDefinitionTypeName=Util::TypeOf< Attribute >()`)  
*Creates an entity attribute definition in the model.*
- virtual `~Attribute ()` override  
*Releases sparse initial values owned by this definition.*
- double **getInitialValue** (`std::string index=""`)

- Reads the initial sparse value, returning 0.0 when the index is absent.*
- void **setInitialValue** (double value, std::string index="")  
*Writes the initial sparse value at the scalar or indexed position.*
- void **setInitialValues** (const std::vector< std::pair< std::string, double > > values)  
*Replaces/extends initial sparse values from textual index/value pairs.*
- void **copyInitialValuesFrom** (const [Attribute](#) &source)  
*Copies the full initial sparse store from another attribute.*
- void **insertDimentionSize** (unsigned int size)  
*Appends one dimension size to the attribute definition.*
- std::list< unsigned int > \* **getDimensionSizes** () const  
*Returns dimension sizes for compatibility with existing callers.*
- std::map< std::string, double > \* **getInitialValues** () const  
*Returns mutable initial sparse values for compatibility with existing callers.*
- [SparseValueStore](#) \* **getInitialValueStore** ()  
*Returns the sparse initial value store used by this attribute.*
- bool **isInitialValuesTextValid** () const  
*Returns whether the last textual initial value is syntactically valid.*

## Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) \*model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)  
*Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()  
*Releases resources owned by this data definition.*
- Util::identification [getId](#) () const  
*Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)  
*Updates the logical name of this instance.*
- const std::string & [getName](#) () const  
*Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)  
*Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const  
*Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const  
*Returns the runtime class name used by the plugin system.*
- [Model](#) \* [getParentModel](#) () const  
*Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const  
*Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)  
*Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) \* [getInternalData](#) (const std::string &name) const  
*Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getInternalData](#) () const  
*Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getAttachedData](#) () const  
*Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const  
*Indicates whether this instance has been modified since the last reset.*

- void [setHasChanged](#) (bool [hasChanged](#))  
*Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const  
*Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int [\\_modelLevel](#))  
*Sets the model nesting level for this definition.*
- [List](#)< [SimulationControl](#) \* > \* [getSimulationControls](#) () const  
*Returns the simulation controls owned by this data definition.*
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const  
*Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)  
*Updates the trace level used by this definition.*
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) [traceLevelspecific](#), bool [traceLevelSpecificEnabled](#)=true)  
*Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const  
*Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool [traceLevelSpecificEnabled](#))  
*Enables or disables the specific trace level.*

#### Static Public Member Functions

- static [PluginInformation](#) \* [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")

#### Static Public Member Functions inherited from [Attribute](#)

- static [PluginInformation](#) \* [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")

#### Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool [Check](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*
- static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

### Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) \*fields) override  
*must be overridden by derived classes*
- virtual void `_saveInstance` ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*
- virtual bool `_check` (std::string &errorMessage) override  
*could be overridden by derived classes*
- virtual void `_initBetweenReplications` () override

### Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) \* `_getParserChangesInformation` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) \*control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()  
*legacy helper methods used by the granular create\* hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) \*child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) \*data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)  
*method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void `trace` (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))

*not just an easy access to trace manager, but wrappers to check if specific trace level applies*

- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- void [traceSimulation](#) (void \*thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)

*Traces text to the simulation output.*

- void **traceSimulation** (void \*thisobject, TraceManager::Level level, const std::string &text)

## Additional Inherited Members

## Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **\_id**
- std::string **\_typename**
- bool **\_reportStatistics**
- bool **\_hasChanged**
- unsigned int **\_modelLevel** = 0
- [Model](#) \* **\_parentModel**
- [List](#)< [SimulationControl](#) \* > \* **\_simulationControls** = new [List](#)<[SimulationControl](#)\*>()
- TraceManager::Level **\_traceLevelSpecific**
- bool **\_traceLevelSpecificEnabled** = false

## 15.1067.1 Detailed Description

[Variable](#) module DESCRIPTION This data module is used to define a variable's dimension and values. You can reference variables in other modules (for example, the [Decide](#) module), reassign new values to variables with the [Assign](#) module, and use variables in any expression. You can use an external data file to specify variable values, and you can specify the variable's initial values in the [Variable](#) module. If you use both methods, the values are read at different times, depending on the options you specify, including the [File](#) Read Time, the Clear Option, and the replication parameters you specify in the Run Setup dialog box. For more information, see the online Help. There are three methods for manually editing the Initial Values of a [Variable](#) module: Using the standard spreadsheet interface. In the module spreadsheet, right-click on the Initial Values cell and select the Edit via spreadsheet menu item. The values for two-dimensional arrays should be entered one column at a time. Array elements not explicitly assigned are assumed to have the last entered value. Using the module dialog box. In the module spreadsheet, right-click on any cell and select the Edit via dialog menu item. The values for two-dimensional arrays should be entered one column at a time. Array elements not explicitly assigned are assumed to have the last entered value. Using the two-dimensional (2-D) spreadsheet interface. In the module spreadsheet, click on the Initial Values cell. TYPICAL USES Number of documents processed per hour Serial number to assign to parts for unique identification Space available in a facility PROMPTS Prompt Description Name The unique name of the variable being defined. Rows Number of rows in a one- or two-dimensional variable. Columns Number of columns in a two-dimensional variable. Report Statistics Check box for determining whether or not statistics will be collected. This field is visible when the rows and columns are not specified (that is, for single variables). Data Type The data type of the values stored in the variable. Valid types are Real and String. The default type is Real. Clear Option Defines the time (if at all) when the value(s) of the variable is reset to the initial value(s) specified. Specifying Statistics resets this variable to its initial value(s) whenever statistics are cleared. Specifying System resets this variable to its initial value(s) whenever the system is cleared. Specifying None indicates that this variable is never reset to its initial value(s), except prior to the first replication. [File](#) Name Name of the file from which to read the variable's value or values. You can use any file access type supported by Arena except sequential text files and Lotus spreadsheet (.wks) files. If the file name you specify has not been created yet, Arena will create it, but you must edit the file to specify the file access type, path, and recordset (if required). Recordset Name of the recordset in the specified file from which to read values. This field is available only if you specify a [File](#) Name for a file that has been set up with a file access type, path, and recordset. Arena uses the Rows and Columns properties to determine the amount of data to read from the recordset. A recordset is required for all file types except .xml. The recordset size must be equal to or greater than the number of rows and columns specified for the variable. [File](#) Read Time Specifies when to read the values from the file into the variable. If you select PreCheck, the values for the variable are read while the model is



still in Edit mode (prior to the model being checked and compiled). If you select BeginSimulation, values are read when the model is compiled, prior to the first replication. If you select BeginReplication, values are read prior to each replication. Initial Values Lists the initial value or values of the variable. You can assign new values to the variable at different stages of the model by using the [Assign](#) module. Initial Value [Variable](#) value at the start of the simulation.

## 15.1067.2 Member Function Documentation

### 15.1067.2.1 `_check()`

```
bool Variable::_check (
 std::string & errorMessage) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [Attribute](#).

### 15.1067.2.2 `_initBetweenReplications()`

```
void Variable::_initBetweenReplications () [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

### 15.1067.2.3 `_loadInstance()`

```
bool Variable::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [Attribute](#).

### 15.1067.2.4 `_saveInstance()`

```
void Variable::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

#### Parameters

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [Attribute](#).

### 15.1067.2.5 `getInitialValuesText()`

```
std::string Variable::getInitialValuesText () const [override], [virtual]
```

Returns the initial values serialized in bracket notation.

Reimplemented from [Attribute](#).

### 15.1067.2.6 `setInitialValuesText()`

```
void Variable::setInitialValuesText (
 std::string valuesText) [override], [virtual]
```

Parses bracket notation and replaces both initial and runtime stores.

Reimplemented from [Attribute](#).

### 15.1067.2.7 `show()`

```
std::string Variable::show () [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

### Returns

Human-readable summary used by traces and debug output.

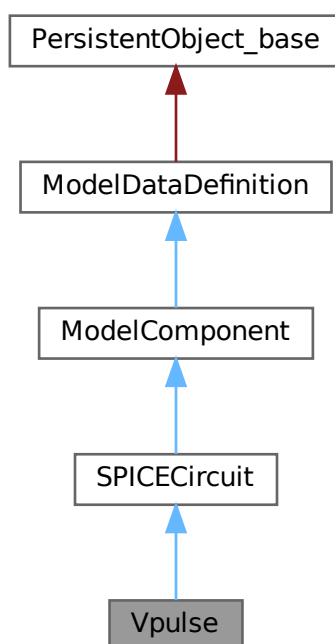
Reimplemented from [Attribute](#).

The documentation for this class was generated from the following files:

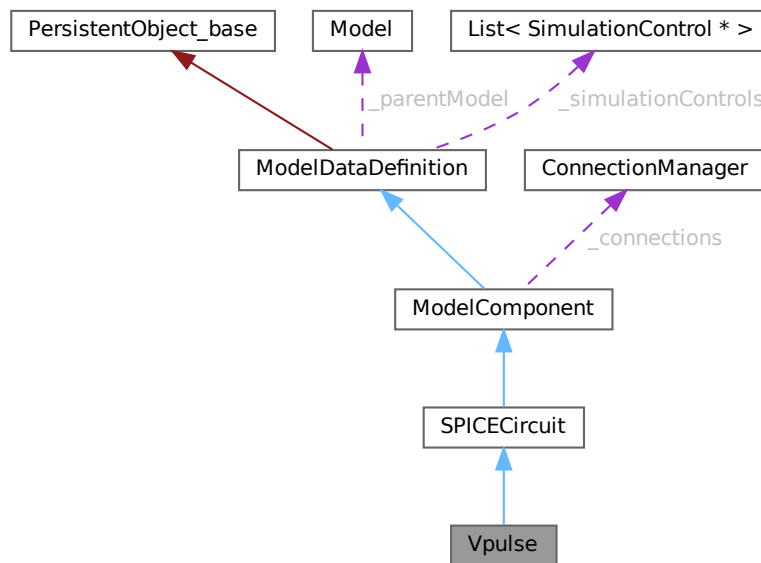
- plugins/data/Logic/Variable.h
- plugins/data/Logic/Variable.cpp

## 15.1068 Vpulse Class Reference

Inheritance diagram for Vpulse:



Collaboration diagram for Vpulse:



### Public Member Functions

- **Vpulse** ([Model](#) \*model, std::string name="")
- void **Build** () override
- void **setVoltage** (float voltage)
- void **setFreq** (float freq)
- void **setSlope** (float slope)

### Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) \*model, std::string name="")  
*constructors*
- void **setRunner** ([SPICERunner](#) \*runner)  
*new public user methods for this components*
- [SPICERunner](#) \* **getRunner** ()
- virtual std::string **show** () override  
*virtual public methods*

### Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) \*model, std::string componentName, std::string name="")  
*constructors*
- virtual ~**ModelComponent** ()  
*Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) \* **getConnectionManager** () const  
*new public user methods for this component*
- void **connectTo** ([ModelComponent](#) \*component, unsigned int inputPortNumber=0)  
*Connects this component to another component.*
- bool **hasBreakpointAt** ()

- Indicates whether a simulation breakpoint is set on this component.*
- void [setDescription](#) (std::string \_description)  
*Updates the free-form description shown by the GUI.*
- std::string [getDescription](#) () const  
*Returns the free-form description associated with this component.*
- void [remove](#) (ModelComponent \*comp)  
*Removes a connected component from this component.*

## Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) (Model \*model, std::string datadefinitionType, std::string name="", bool insert↵  
 IntoModel=true)  
*Creates a model data definition and optionally inserts it into the model.*
- virtual ~[ModelDataDefinition](#) ()  
*Releases resources owned by this data definition.*
- Util::identification [getId](#) () const  
*Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)  
*Updates the logical name of this instance.*
- const std::string & [getName](#) () const  
*Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)  
*Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const  
*Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const  
*Returns the runtime class name used by the plugin system.*
- Model \* [getParentModel](#) () const  
*Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const  
*Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)  
*Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) \* [getInternalData](#) (const std::string &name) const  
*Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getInternalData](#) () const  
*Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getAttachedData](#) () const  
*Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const  
*Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool hasChanged)  
*Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const  
*Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int \_modelLevel)  
*Sets the model nesting level for this definition.*
- List< [SimulationControl](#) \* > \* [getSimulationControls](#) () const  
*Returns the simulation controls owned by this data definition.*
- TraceManager::Level [getTraceLevelSpecific](#) () const  
*Returns the trace level that applies specifically to this definition.*

- void [setTraceLevelSpecific](#) (TraceManager::Level level)  
*Updates the trace level used by this definition.*
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)  
*Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const  
*Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)  
*Enables or disables the specific trace level.*

### Static Public Member Functions

- static [PluginInformation](#) \* [GetPluginInformation](#) ()
- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")

### Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation](#) \* [GetPluginInformation](#) ()  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")
- static std::vector< std::string > [split](#) (std::string word, char split\_char)  
*Externalize function GetPluginInformation to be accessible through dynamic linked library.*

### Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool [Check](#) ([ModelComponent](#) \*component)  
*Validates a component instance.*
- static void [CreateInternalData](#) ([ModelComponent](#) \*component)  
*Creates internal data required by a component instance.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelComponent](#) \*component)  
*Serializes a component instance into a persistence record.*
- static void [DispatchEvent](#) ([Event](#) \*event)  
*Dispatches a simulation event to the target component.*

### Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool [Check](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*
- static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

### Protected Member Functions

- virtual bool [\\_loadInstance](#) ([PersistenceRecord](#) \*fields) override  
*virtual protected method that must be overridden*
- virtual void [\\_saveInstance](#) ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*

### Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the\_params={}, std::string the\_model="", std::string the\_model\_file="")  
*virtual protected method that must be overridden*
- void **UpdateConnections** ()
- virtual void [\\_onDispatchEvent](#) ([Entity](#) \*entity, unsigned int inputPortNumber) override  
*This method is only for ModelComponents, not ModelDataElements.*
- virtual void [\\_createNonEditableDataDefinitions](#) () override  
*virtual protected methods that could be overridden by derived classes, if needed*

### Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [\\_getSaveDefaultsOption](#) ()
- virtual bool [\\_check](#) (std::string &errorMessage)  
*could be overridden by derived classes*
- virtual [ParserChangesInformation](#) \* [\\_getParserChangesInformation](#) ()
- virtual void [\\_initBetweenReplications](#) ()
- virtual void [\\_addSimulationControl](#) ([SimulationControl](#) \*control)
- virtual void [\\_createInternalAndAttachedData](#) ()
- virtual void [\\_createInternalStatisticReporters](#) ()
- virtual void [\\_createAttachedAttributes](#) ()
- virtual void [\\_createEditableDataDefinitions](#) ()
- void [\\_templateCreateInternalStatisticReporters](#) ()
- void [\\_templateCreateAttachedAttributes](#) ()
- void [\\_templateCreateNonEditableDataDefinitions](#) ()
- void [\\_templateCreateEditableDataDefinitions](#) ()
- void [\\_internalStatisticReportersClear](#) ()  
*legacy helper methods used by the granular create\* hooks*
- void [\\_internalDataClear](#) ()
- void [\\_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*child)
- void [\\_internalDataRemove](#) (const std::string &key)
- void [\\_attachedDataClear](#) ()
- void [\\_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_attachedDataRemove](#) (const std::string &key)
- void [\\_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [\\_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_statisticReporterRemove](#) (const std::string &key)
- void [\\_statisticReportersClear](#) ()
- void [\\_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [\\_mandatoryAttachedAttributesClear](#) ()
- void [\\_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_mandatoryEditableDataDefinitionsClear](#) ()
- void [\\_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_optionalEditableDataDefinitionRemove](#) (const std::string &key)

- void **\_optionalEditableDataDefinitionsClear** ()
- void **\_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryNonEditableDataDefinitionsClear** ()
- void **\_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalNonEditableDataDefinitionsClear** ()
- void **\_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)  
*method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)  
*not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- void [traceSimulation](#) (void \*thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)  
*Traces text to the simulation output.*
- void **traceSimulation** (void \*thisobject, TraceManager::Level level, const std::string &text)

### Additional Inherited Members

### Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

### Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) \* **\_connections** = new [ConnectionManager](#)()  
*virtual protected methods that could be overridden by derived classes, if needed*

### Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **\_id**
- std::string **\_typename**
- bool **\_reportStatistics**
- bool **\_hasChanged**
- unsigned int **\_modelLevel** = 0
- [Model](#) \* **\_parentModel**
- [List](#)< [SimulationControl](#) \* > \* **\_simulationControls** = new [List](#)<[SimulationControl](#)\*>()
- TraceManager::Level **\_traceLevelSpecific**
- bool **\_traceLevelSpecificEnabled** = false

## 15.1068.1 Member Function Documentation

### 15.1068.1.1 **\_loadInstance()**

```
bool Vpulse::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

**15.1068.1.2 \_saveInstance()**

```
void Vpulse::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

**Parameters**

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [SPICECircuit](#).

**15.1068.1.3 Build()**

```
void Vpulse::Build () [override], [virtual]
```

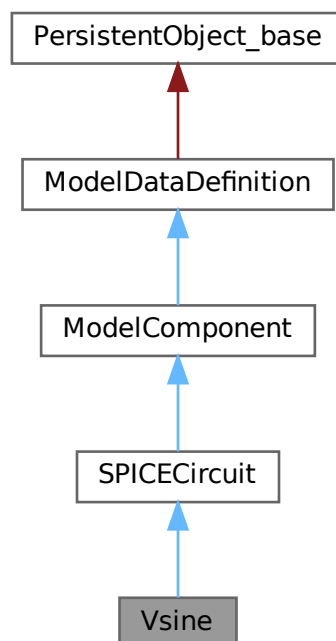
Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

- plugins/components/ElectronicsSimulation/SPICECircuit.h
- plugins/components/ElectronicsSimulation/SPICECircuit.cpp

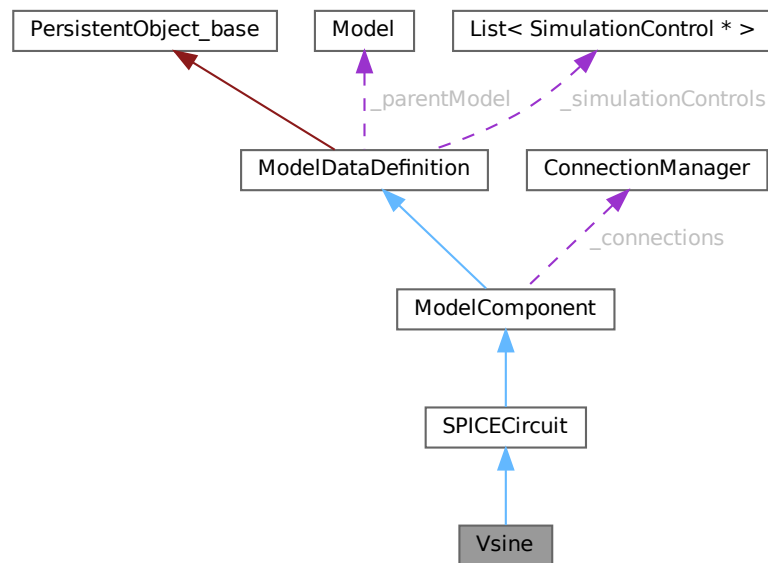
**15.1069 Vsine Class Reference**

Inheritance diagram for Vsine:





Collaboration diagram for Vsine:



### Public Member Functions

- **Vsine** ([Model](#) \*model, std::string name="")
- void **Build** () override
- void **setVoltage** (float voltage)
- void **setFreq** (float freq)

### Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) \*model, std::string name="")  
*constructors*
- void **setRunner** ([SPICERunner](#) \*runner)  
*new public user methods for this components*
- [SPICERunner](#) \* **getRunner** ()
- virtual std::string **show** () override  
*virtual public methods*

### Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) \*model, std::string componentName, std::string name="")  
*constructors*
- virtual ~**ModelComponent** ()  
*Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) \* **getConnectionManager** () const  
*new public user methods for this component*
- void **connectTo** ([ModelComponent](#) \*component, unsigned int inputPortNumber=0)  
*Connects this component to another component.*
- bool **hasBreakpointAt** ()  
*Indicates whether a simulation breakpoint is set on this component.*

- void [setDescription](#) (std::string \_description)  
*Updates the free-form description shown by the GUI.*
- std::string [getDescription](#) () const  
*Returns the free-form description associated with this component.*
- void [remove](#) ([ModelComponent](#) \*comp)  
*Removes a connected component from this component.*

## Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) \*model, std::string datatypeName, std::string name="", bool insert↵ IntoModel=true)  
*Creates a model data definition and optionally inserts it into the model.*
- virtual ~[ModelDataDefinition](#) ()  
*Releases resources owned by this data definition.*
- Util::identification [getId](#) () const  
*Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)  
*Updates the logical name of this instance.*
- const std::string & [getName](#) () const  
*Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)  
*Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const  
*Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const  
*Returns the runtime class name used by the plugin system.*
- [Model](#) \* [getParentModel](#) () const  
*Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const  
*Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)  
*Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) \* [getInternalData](#) (const std::string &name) const  
*Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getInternalData](#) () const  
*Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getAttachedData](#) () const  
*Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const  
*Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))  
*Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const  
*Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int \_modelLevel)  
*Sets the model nesting level for this definition.*
- List< [SimulationControl](#) \* > \* [getSimulationControls](#) () const  
*Returns the simulation controls owned by this data definition.*
- TraceManager::Level [getTraceLevelSpecific](#) () const  
*Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) (TraceManager::Level level)

*Updates the trace level used by this definition.*

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific←  
Enabled=true)

*Defines the specific trace level and optionally enables it immediately.*

- bool [isTraceLevelSpecificEnabled](#) () const  
*Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)  
*Enables or disables the specific trace level.*

### Static Public Member Functions

- static [PluginInformation](#) \* [GetPluginInformation](#) ()
- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")

### Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation](#) \* [GetPluginInformation](#) ()  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")
- static std::vector< std::string > [split](#) (std::string word, char split\_char)  
*Externalize function GetPluginInformation to be accessible through dynamic linked library.*

### Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool [Check](#) ([ModelComponent](#) \*component)  
*Validates a component instance.*
- static void [CreateInternalData](#) ([ModelComponent](#) \*component)  
*Creates internal data required by a component instance.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelComponent](#) \*component)  
*Serializes a component instance into a persistence record.*
- static void [DispatchEvent](#) ([Event](#) \*event)  
*Dispatches a simulation event to the target component.*

### Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool [Check](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*
- static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

### Protected Member Functions

- virtual bool [\\_loadInstance](#) ([PersistenceRecord](#) \*fields) override  
*virtual protected method that must be overridden*
- virtual void [\\_saveInstance](#) ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*

### Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the\_params={}, std::string the\_model="", std::string the\_model\_file="")  
*virtual protected method that must be overridden*
- void **UpdateConnections** ()
- virtual void [\\_onDispatchEvent](#) ([Entity](#) \*entity, unsigned int inputPortNumber) override  
*This method is only for ModelComponents, not ModelDataElements.*
- virtual void [\\_createNonEditableDataDefinitions](#) () override  
*virtual protected methods that could be overridden by derived classes, if needed*

### Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [\\_getSaveDefaultsOption](#) ()
- virtual bool [\\_check](#) (std::string &errorMessage)  
*could be overridden by derived classes*
- virtual [ParserChangesInformation](#) \* [\\_getParserChangesInformation](#) ()
- virtual void [\\_initBetweenReplications](#) ()
- virtual void [\\_addSimulationControl](#) ([SimulationControl](#) \*control)
- virtual void [\\_createInternalAndAttachedData](#) ()
- virtual void [\\_createInternalStatisticReporters](#) ()
- virtual void [\\_createAttachedAttributes](#) ()
- virtual void [\\_createEditableDataDefinitions](#) ()
- void [\\_templateCreateInternalStatisticReporters](#) ()
- void [\\_templateCreateAttachedAttributes](#) ()
- void [\\_templateCreateNonEditableDataDefinitions](#) ()
- void [\\_templateCreateEditableDataDefinitions](#) ()
- void [\\_internalStatisticReportersClear](#) ()  
*legacy helper methods used by the granular create\* hooks*
- void [\\_internalDataClear](#) ()
- void [\\_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*child)
- void [\\_internalDataRemove](#) (const std::string &key)
- void [\\_attachedDataClear](#) ()
- void [\\_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_attachedDataRemove](#) (const std::string &key)
- void [\\_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [\\_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_statisticReporterRemove](#) (const std::string &key)
- void [\\_statisticReportersClear](#) ()
- void [\\_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [\\_mandatoryAttachedAttributesClear](#) ()
- void [\\_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_mandatoryEditableDataDefinitionsClear](#) ()
- void [\\_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_optionalEditableDataDefinitionRemove](#) (const std::string &key)

- void **\_optionalEditableDataDefinitionsClear** ()
- void **\_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryNonEditableDataDefinitionsClear** ()
- void **\_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalNonEditableDataDefinitionsClear** ()
- void **\_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)  
*method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)  
*not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- void [traceSimulation](#) (void \*thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)  
*Traces text to the simulation output.*
- void **traceSimulation** (void \*thisobject, TraceManager::Level level, const std::string &text)

#### Additional Inherited Members

#### Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

#### Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) \* **\_connections** = new [ConnectionManager](#)()  
*virtual protected methods that could be overridden by derived classes, if needed*

#### Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **\_id**
- std::string **\_typename**
- bool **\_reportStatistics**
- bool **\_hasChanged**
- unsigned int **\_modelLevel** = 0
- [Model](#) \* **\_parentModel**
- [List](#)< [SimulationControl](#) \* > \* **\_simulationControls** = new [List](#)<[SimulationControl](#)\*>()
- TraceManager::Level **\_traceLevelSpecific**
- bool **\_traceLevelSpecificEnabled** = false

### 15.1069.1 Member Function Documentation

#### 15.1069.1.1 **\_loadInstance()**

```
bool Vsine::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

**15.1069.1.2 \_saveInstance()**

```
void Vsine::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

**Parameters**

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [SPICECircuit](#).

**15.1069.1.3 Build()**

```
void Vsine::Build () [override], [virtual]
```

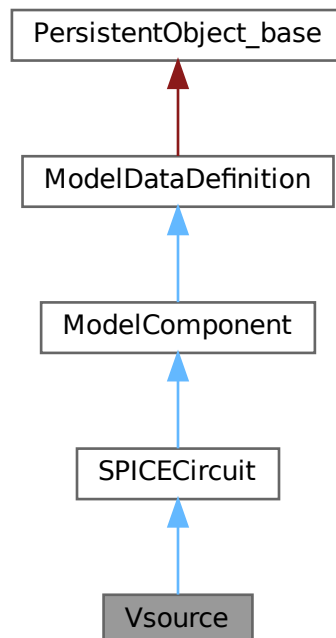
Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

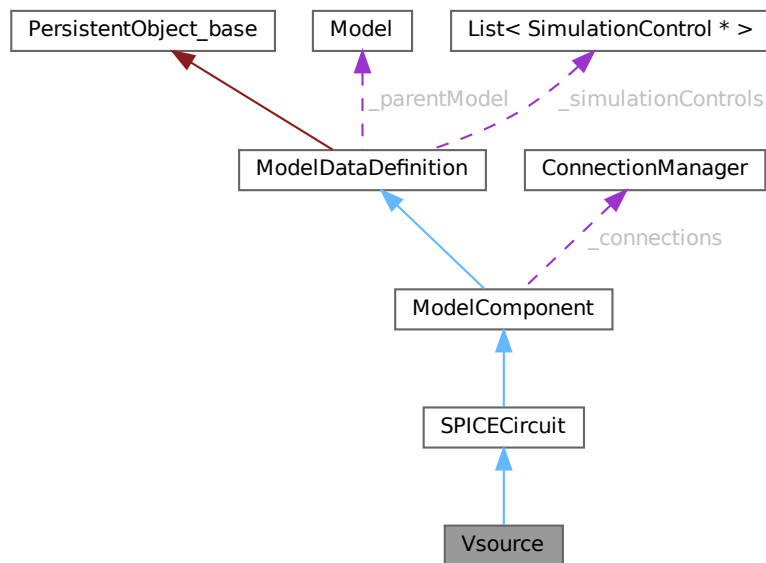
- plugins/components/ElectronicsSimulation/SPICECircuit.h
- plugins/components/ElectronicsSimulation/SPICECircuit.cpp

**15.1070 Vsource Class Reference**

Inheritance diagram for Vsource:



Collaboration diagram for Vsource:



### Public Member Functions

- **Vsource** ([Model](#) \*model, std::string name="")
- void **Build** () override
- void **setVoltage** (float voltage)

### Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) \*model, std::string name="")  
*constructors*
- void **setRunner** ([SPICERunner](#) \*runner)  
*new public user methods for this components*
- [SPICERunner](#) \* **getRunner** ()
- virtual std::string **show** () override  
*virtual public methods*

### Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) \*model, std::string componentTypename, std::string name="")  
*constructors*
- virtual ~**ModelComponent** ()  
*Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) \* **getConnectionManager** () const  
*new public user methods for this component*
- void **connectTo** ([ModelComponent](#) \*component, unsigned int inputPortNumber=0)  
*Connects this component to another component.*
- bool **hasBreakpointAt** ()  
*Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string \_description)

*Updates the free-form description shown by the GUI.*

- `std::string getDescription () const`

*Returns the free-form description associated with this component.*

- `void remove (ModelComponent *comp)`

*Removes a connected component from this component.*

## Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** \*model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

*Creates a model data definition and optionally inserts it into the model.*

- virtual ~**ModelDataDefinition** ()

*Releases resources owned by this data definition.*

- Util::identification getId () const

*Returns the unique identifier of this instance.*

- void setName (const std::string &name)

*Updates the logical name of this instance.*

- const std::string & getName () const

*Returns the logical name of this instance.*

- void setLabel (const std::string &label)

*Updates the user-visible label of this instance.*

- const std::string & getLabel () const

*Returns the user-visible label of this instance.*

- const std::string & getClassname () const

*Returns the runtime class name used by the plugin system.*

- **Model** \* getParentModel () const

*Returns the parent model that owns this definition.*

- bool isReportStatistics () const

*Indicates whether this data definition contributes report statistics.*

- void setReportStatistics (bool reportStatistics)

*Enables or disables report-statistics generation for this definition.*

- **ModelDataDefinition** \* getInternalData (const std::string &name) const

*Looks up a named internal child data definition.*

- std::map< std::string, **ModelDataDefinition** \* > \* getInternalData () const

*Returns the full map of internal child data definitions.*

- std::map< std::string, **ModelDataDefinition** \* > \* getAttachedData () const

*Returns the full map of attached data definitions.*

- bool hasChanged () const

*Indicates whether this instance has been modified since the last reset.*

- void setHasChanged (bool hasChanged)

*Updates the changed flag for this data definition.*

- unsigned int getLevel () const

*Returns the level in the model hierarchy where this definition lives.*

- void setModelLevel (unsigned int \_modelLevel)

*Sets the model nesting level for this definition.*

- List< **SimulationControl** \* > \* getSimulationControls () const

*Returns the simulation controls owned by this data definition.*

- TraceManager::Level getTraceLevelSpecific () const

*Returns the trace level that applies specifically to this definition.*

- void setTraceLevelSpecific (TraceManager::Level level)

*Updates the trace level used by this definition.*



- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↔ Enabled=true)  
*Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const  
*Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)  
*Enables or disables the specific trace level.*

### Static Public Member Functions

- static [PluginInformation](#) \* [GetPluginInformation](#) ()
- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")

### Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation](#) \* [GetPluginInformation](#) ()  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")
- static std::vector< std::string > [split](#) (std::string word, char split\_char)  
*Externalize function GetPluginInformation to be accessible through dynamic linked library.*

### Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool [Check](#) ([ModelComponent](#) \*component)  
*Validates a component instance.*
- static void [CreateInternalData](#) ([ModelComponent](#) \*component)  
*Creates internal data required by a component instance.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelComponent](#) \*component)  
*Serializes a component instance into a persistence record.*
- static void [DispatchEvent](#) ([Event](#) \*event)  
*Dispatches a simulation event to the target component.*

### Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool [Check](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*
- static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

### Protected Member Functions

- virtual bool [\\_loadInstance](#) ([PersistenceRecord](#) \*fields) override  
*virtual protected method that must be overridden*
- virtual void [\\_saveInstance](#) ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*

### Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the\_params={}, std::string the\_model="", std::string the\_model\_file="")  
*virtual protected method that must be overridden*
- void **UpdateConnections** ()
- virtual void [\\_onDispatchEvent](#) ([Entity](#) \*entity, unsigned int inputPortNumber) override  
*This method is only for ModelComponents, not ModelDataElements.*
- virtual void [\\_createNonEditableDataDefinitions](#) () override  
*virtual protected methods that could be overridden by derived classes, if needed*

### Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [\\_getSaveDefaultsOption](#) ()
- virtual bool [\\_check](#) (std::string &errorMessage)  
*could be overridden by derived classes*
- virtual [ParserChangesInformation](#) \* [\\_getParserChangesInformation](#) ()
- virtual void [\\_initBetweenReplications](#) ()
- virtual void [\\_addSimulationControl](#) ([SimulationControl](#) \*control)
- virtual void [\\_createInternalAndAttachedData](#) ()
- virtual void [\\_createInternalStatisticReporters](#) ()
- virtual void [\\_createAttachedAttributes](#) ()
- virtual void [\\_createEditableDataDefinitions](#) ()
- void [\\_templateCreateInternalStatisticReporters](#) ()
- void [\\_templateCreateAttachedAttributes](#) ()
- void [\\_templateCreateNonEditableDataDefinitions](#) ()
- void [\\_templateCreateEditableDataDefinitions](#) ()
- void [\\_internalStatisticReportersClear](#) ()  
*legacy helper methods used by the granular create\* hooks*
- void [\\_internalDataClear](#) ()
- void [\\_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*child)
- void [\\_internalDataRemove](#) (const std::string &key)
- void [\\_attachedDataClear](#) ()
- void [\\_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_attachedDataRemove](#) (const std::string &key)
- void [\\_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [\\_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_statisticReporterRemove](#) (const std::string &key)
- void [\\_statisticReportersClear](#) ()
- void [\\_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [\\_mandatoryAttachedAttributesClear](#) ()
- void [\\_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_mandatoryEditableDataDefinitionsClear](#) ()
- void [\\_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_optionalEditableDataDefinitionRemove](#) (const std::string &key)

- void **\_optionalEditableDataDefinitionsClear** ()
- void **\_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryNonEditableDataDefinitionsClear** ()
- void **\_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalNonEditableDataDefinitionsClear** ()
- void **\_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)  
*method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)  
*not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- void [traceSimulation](#) (void \*thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)  
*Traces text to the simulation output.*
- void **traceSimulation** (void \*thisobject, TraceManager::Level level, const std::string &text)

#### Additional Inherited Members

#### Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

#### Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) \* **\_connections** = new [ConnectionManager](#)()  
*virtual protected methods that could be overridden by derived classes, if needed*

#### Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **\_id**
- std::string **\_typename**
- bool **\_reportStatistics**
- bool **\_hasChanged**
- unsigned int **\_modelLevel** = 0
- [Model](#) \* **\_parentModel**
- [List](#)< [SimulationControl](#) \* > \* **\_simulationControls** = new [List](#)<[SimulationControl](#)\*>()
- TraceManager::Level **\_traceLevelSpecific**
- bool **\_traceLevelSpecificEnabled** = false

### 15.1070.1 Member Function Documentation

#### 15.1070.1.1 **\_loadInstance()**

```
bool Vsource::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

**15.1070.1.2 \_saveInstance()**

```
void Vsource::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

**Parameters**

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [SPICECircuit](#).

**15.1070.1.3 Build()**

```
void Vsource::Build () [override], [virtual]
```

Reimplemented from [SPICECircuit](#).

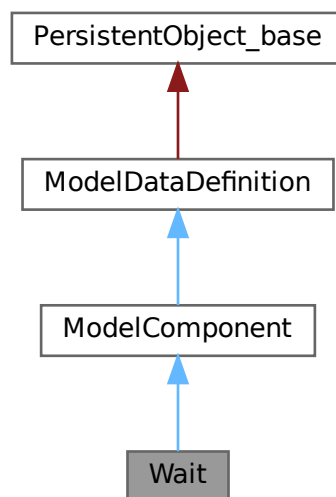
The documentation for this class was generated from the following files:

- plugins/components/ElectronicsSimulation/SPICECircuit.h
- plugins/components/ElectronicsSimulation/SPICECircuit.cpp

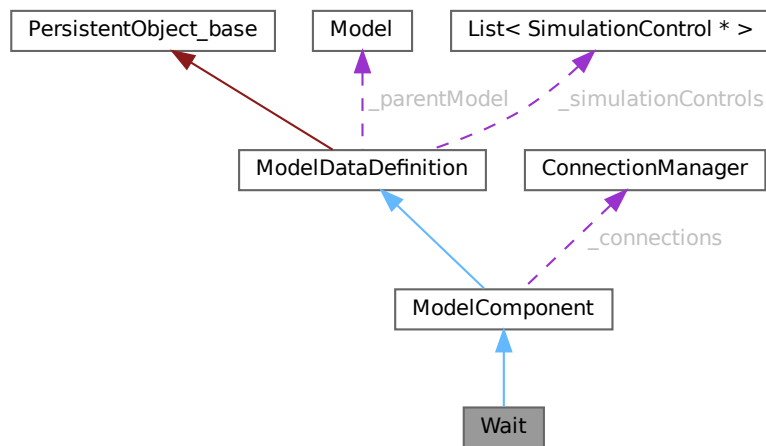
**15.1071 Wait Class Reference**

```
#include <plugins/components/Synchronization/Wait.h>
```

Inheritance diagram for Wait:



Collaboration diagram for Wait:



## Public Types

- enum class **WaitType** : int { **WaitForSignal** = 0 , **InfiniteHold** = 1 , **ScanForCondition** = 2 , num\_elements = 3 }

## Public Member Functions

- Wait** ([Model](#) \*model, std::string name="")
- virtual std::string **show** () override
- virtual public methods*
- void **setSignalData** ([SignalData](#) \*signal)
- void **setWaitType** (WaitType \_watType)
- WaitType **getWaitType** () const
- void **setCondition** (std::string \_condition)
- std::string **getCondition** () const
- [Queue](#) \* **getQueue** () const
- void **setQueue** ([Queue](#) \*queue)
- std::string **getlimitExpression** () const
- void **setLimitExpression** (const std::string &newLimitExpression)

## Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) \*model, std::string componentTypename, std::string name="")
- constructors*
- virtual ~**ModelComponent** ()
- Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) \* **getConnectionManager** () const
- new public user methods for this component*
- void **connectTo** ([ModelComponent](#) \*component, unsigned int inputPortNumber=0)
- Connects this component to another component.*
- bool **hasBreakpointAt** ()
- Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string \_description)

*Updates the free-form description shown by the GUI.*

- `std::string getDescription () const`

*Returns the free-form description associated with this component.*

- `void remove (ModelComponent *comp)`

*Removes a connected component from this component.*

## Public Member Functions inherited from **ModelDataDefinition**

- `ModelDataDefinition (Model *model, std::string datadefinitionTypename, std::string name="", bool insertIntoModel=true)`

*Creates a model data definition and optionally inserts it into the model.*

- `virtual ~ModelDataDefinition ()`

*Releases resources owned by this data definition.*

- `Util::identification getId () const`

*Returns the unique identifier of this instance.*

- `void setName (const std::string &name)`

*Updates the logical name of this instance.*

- `const std::string & getName () const`

*Returns the logical name of this instance.*

- `void setLabel (const std::string &label)`

*Updates the user-visible label of this instance.*

- `const std::string & getLabel () const`

*Returns the user-visible label of this instance.*

- `const std::string & getClassname () const`

*Returns the runtime class name used by the plugin system.*

- `Model * getParentModel () const`

*Returns the parent model that owns this definition.*

- `bool isReportStatistics () const`

*Indicates whether this data definition contributes report statistics.*

- `void setReportStatistics (bool reportStatistics)`

*Enables or disables report-statistics generation for this definition.*

- `ModelDataDefinition * getInternalData (const std::string &name) const`

*Looks up a named internal child data definition.*

- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`

*Returns the full map of internal child data definitions.*

- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`

*Returns the full map of attached data definitions.*

- `bool hasChanged () const`

*Indicates whether this instance has been modified since the last reset.*

- `void setHasChanged (bool hasChanged)`

*Updates the changed flag for this data definition.*

- `unsigned int getLevel () const`

*Returns the level in the model hierarchy where this definition lives.*

- `void setModelLevel (unsigned int _modelLevel)`

*Sets the model nesting level for this definition.*

- `List< SimulationControl * > * getSimulationControls () const`

*Returns the simulation controls owned by this data definition.*

- `TraceManager::Level getTraceLevelSpecific () const`

*Returns the trace level that applies specifically to this definition.*

- `void setTraceLevelSpecific (TraceManager::Level level)`

*Updates the trace level used by this definition.*

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)  
*Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const  
*Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)  
*Enables or disables the specific trace level.*

### Static Public Member Functions

- static std::string [convertEnumToStr](#) (WaitType type)
- static [PluginInformation](#) \* [GetPluginInformation](#) ()
- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")

### Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool [Check](#) ([ModelComponent](#) \*component)  
*Validates a component instance.*
- static void [CreateInternalData](#) ([ModelComponent](#) \*component)  
*Creates internal data required by a component instance.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelComponent](#) \*component)  
*Serializes a component instance into a persistence record.*
- static void [DispatchEvent](#) ([Event](#) \*event)  
*Dispatches a simulation event to the target component.*

### Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool [Check](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*
- static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

### Protected Member Functions

- virtual bool [\\_loadInstance](#) ([PersistenceRecord](#) \*fields) override  
*virtual protected method that must be overridden*
- virtual void [\\_saveInstance](#) ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*
- virtual void [\\_onDispatchEvent](#) ([Entity](#) \*entity, unsigned int inputPortNumber) override

*Handles a dispatched event for this component type.*

- virtual bool `_check` (std::string &errorMessage) override  
*could be overridden by derived classes*
- virtual void `_initBetweenReplications` () override
- virtual void `_createNonEditableDataDefinitions` () override
- virtual void `_createEditableDataDefinitions` () override

## Protected Member Functions inherited from `ModelDataDefinition`

- bool `_getSaveDefaultsOption` ()
- virtual `ParserChangesInformation` \* `_getParserChangesInformation` ()
- virtual void `_addSimulationControl` (`SimulationControl` \*control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()  
*legacy helper methods used by the granular create\* hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, `ModelDataDefinition` \*child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, `ModelDataDefinition` \*data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, `ModelDataDefinition` \*data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, `ModelDataDefinition` \*data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` \*data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` \*data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` \*data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` \*data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)  
*method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void `trace` (const std::string &text, `TraceManager::Level` level=`TraceManager::Level::L8_detailed`)  
*not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void `traceError` (const std::string &text, const std::exception &e)
- void `traceError` (const std::string &text, `TraceManager::Level` level=`TraceManager::Level::L1_errorFatal`)
- void `traceReport` (const std::string &text, `TraceManager::Level` level=`TraceManager::Level::L2_results`)



- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- void [traceSimulation](#) (void \*thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)

*Traces text to the simulation output.*

- void **traceSimulation** (void \*thisobject, TraceManager::Level level, const std::string &text)

### Additional Inherited Members

### Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) \* [\\_connections](#) = new [ConnectionManager](#)()
- virtual protected methods that could be overridden by derived classes, if needed*

### Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification [\\_id](#)
- std::string [\\_typename](#)
- bool [\\_reportStatistics](#)
- bool [\\_hasChanged](#)
- unsigned int [\\_modelLevel](#) = 0
- [Model](#) \* [\\_parentModel](#)
- [List](#)< [SimulationControl](#) \* > \* [\\_simulationControls](#) = new [List](#)<[SimulationControl](#)\*>()
- TraceManager::Level [\\_traceLevelSpecific](#)
- bool [\\_traceLevelSpecificEnabled](#) = false

## 15.1071.1 Detailed Description

[Wait](#) module DESCRIPTION This module will hold an entity in a queue to wait for a signal, wait for a specified condition to become true (scan), or be held infinitely (to be removed later with the [Remove](#) module). If the entity is holding for a signal, the [Signal](#) module is used elsewhere in the model to allow the entity to move on to the next module. If the entity is holding for a given condition to be true, the entity will remain at the module (either in a defined or internal queue) until the condition(s) becomes true. When the entity is in an infinite hold, the [Remove](#) module is used elsewhere in the model to allow the entity to continue processing. TYPICAL USES [Waiting](#) for a traffic light to turn green Holding a part for authorization Checking the status of a machine or operator to continue a process PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Type Indicates the reasoning for holding the entity within a specified or internal queue. [Wait](#) for [Signal](#) will hold the entity until a signal of the same value is received. Scan for Condition will hold the entity until the specified condition becomes true. Infinite [Wait](#) will hold the entity until it is removed from the queue by a [Remove](#) module. [Wait](#) for Value [Signal](#) code for the waiting entity. Applies only when Type is [Wait](#) for [Signal](#). Limit Maximum number of waiting entities that will be released upon receipt of a signal. Applies only when Type is [Wait](#) for [Signal](#). Condition Specifies the condition that will be evaluated to hold the entity at the module. If the condition is evaluated to true, the entity leaves the module immediately. If the condition is false, the entity will wait in the associated queue until the condition becomes true. Applies only when Type is Scan for Condition. [Queue](#) Type Determines the type of queue used to hold the entities. If [Queue](#) is selected, the queue name is specified. If [Set](#) is selected, the queue set and member in the set are specified. If Internal is selected, an internal queue is used to hold all waiting entities. [Attribute](#) and Expression are additional methods for defining the queue to be used. [Queue](#) Name This field is visible only if [Queue](#) Type is [Queue](#), and it defines the symbol name of the queue. [Set](#) Name This field is visible only if [Queue](#) Type is [Set](#), and it defines the queue set that contains the queue being referenced. [Set](#) Index This field is visible only if [Queue](#) Type is [Set](#), and it defines the index into the queue set. Note that this is the index into the set and not the name of the queue in the set. For example, the only valid entry for a queue set containing three members is an expression that evaluates to 1, 2, or 3. [Attribute](#) This field is visible only if [Queue](#) Type is [Attribute](#). The attribute entered in this field will be evaluated to indicate which queue is to be used. Expression This field is visible only if [Queue](#) Type is Expression. The expression entered in this field will be evaluated to indicate which queue is to be used.

## 15.1071.2 Member Function Documentation

### 15.1071.2.1 `_check()`

```
bool Wait::_check (
 std::string & errorMessage) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

### 15.1071.2.2 `_createEditableDataDefinitions()`

```
void Wait::_createEditableDataDefinitions () [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

### 15.1071.2.3 `_createNonEditableDataDefinitions()`

```
void Wait::_createNonEditableDataDefinitions () [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

### 15.1071.2.4 `_initBetweenReplications()`

```
void Wait::_initBetweenReplications () [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

### 15.1071.2.5 `_loadInstance()`

```
bool Wait::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

### 15.1071.2.6 `_onDispatchEvent()`

```
void Wait::_onDispatchEvent (
 Entity * entity,
 unsigned int inputPortNumber) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

#### Parameters

|                        |                                         |
|------------------------|-----------------------------------------|
| <i>entity</i>          | <a href="#">Entity</a> being processed. |
| <i>inputPortNumber</i> | Input port where the entity arrived.    |

Implements [ModelComponent](#).

### 15.1071.2.7 `_saveInstance()`

```
void Wait::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

#### Parameters

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [ModelComponent](#).

**15.1071.2.8 show()**

```
std::string Wait::show () [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

**Returns**

Human-readable description used for tracing and debugging.

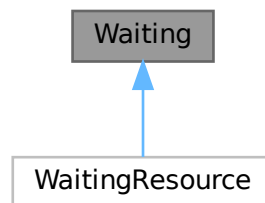
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/Synchronization/Wait.h
- plugins/components/Synchronization/Wait.cpp

**15.1072 Waiting Class Reference**

Inheritance diagram for Waiting:

**Public Member Functions**

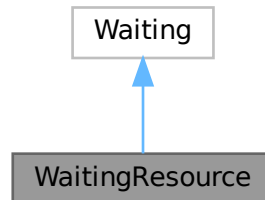
- **Waiting** ([Entity](#) \*entity, double timeStartedWaiting, [ModelComponent](#) \*thisComponent, unsigned int thisComponentOutputPort=0)
- virtual std::string **show** ( )
- double **getTimeStartedWaiting** ( ) const
- [ModelComponent](#) \* **geComponent** ( ) const
- [Entity](#) \* **getEntity** ( ) const
- unsigned int **geComponentOutputPort** ( ) const
- unsigned long long **getArrivalOrder** ( ) const
- void **setArrivalOrder** (unsigned long long arrivalOrder)

The documentation for this class was generated from the following file:

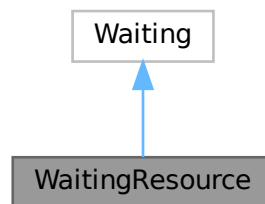
- plugins/data/DiscreteProcessing/Queue.h

## 15.1073 WaitingResource Class Reference

Inheritance diagram for WaitingResource:



Collaboration diagram for WaitingResource:



### Public Member Functions

- **WaitingResource** ([Entity](#) \*entity, double timeStartedWaiting, unsigned int quantity, [ModelComponent](#) \*thisComponent, unsigned int thisComponentOutputPort=0)
- **WaitingResource** (const [WaitingResource](#) &orig)
- virtual std::string **show** () override
- unsigned int **getQuantity** () const

### Public Member Functions inherited from [Waiting](#)

- **Waiting** ([Entity](#) \*entity, double timeStartedWaiting, [ModelComponent](#) \*thisComponent, unsigned int thisComponentOutputPort=0)
- double **getTimeStartedWaiting** () const
- [ModelComponent](#) \* **geComponent** () const
- [Entity](#) \* **getEntity** () const
- unsigned int **geComponentOutputPort** () const
- unsigned long long **getArrivalOrder** () const
- void **setArrivalOrder** (unsigned long long arrivalOrder)

### 15.1073.1 Member Function Documentation

#### 15.1073.1.1 show()

virtual std::string WaitingResource::show ( ) [inline], [override], [virtual]

Reimplemented from [Waiting](#).

The documentation for this class was generated from the following file:

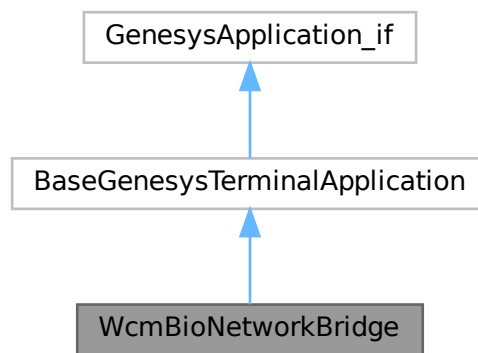
- plugins/components/DiscreteProcessing/Seize.h

## 15.1074 WcmBioNetworkBridge Class Reference

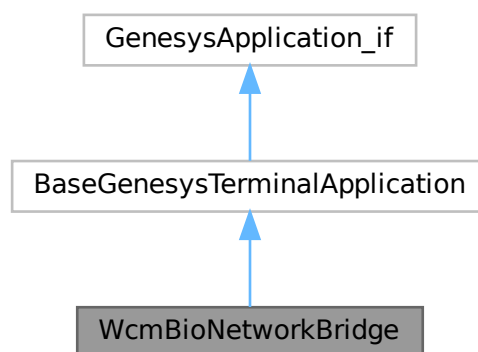
Demonstrates a minimal deterministic/stochastic whole-cell bridge.

```
#include <applications/modelSpecific/wcm/WcmBioNetworkBridge.h>
```

Inheritance diagram for WcmBioNetworkBridge:



Collaboration diagram for WcmBioNetworkBridge:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

## Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

## Additional Inherited Members

## Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

### 15.1074.1 Detailed Description

Demonstrates a minimal deterministic/stochastic whole-cell bridge.

The example builds a small deterministic [BioNetwork](#) for nutrient conversion, projects its final state into a [WholeCellState](#), and then runs stochastic transcription, translation, and degradation components. It is a didactic integration test for GenESyS abstractions, not a complete organism model.

### 15.1074.2 Member Function Documentation

#### 15.1074.2.1 main()

```
int WcmBioNetworkBridge::main (
 int argc,
 char ** argv) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

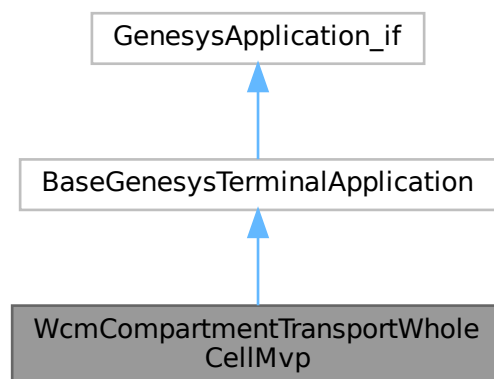
- applications/modelSpecific/wcm/WcmBioNetworkBridge.h
- applications/modelSpecific/wcm/WcmBioNetworkBridge.cpp

## 15.1075 WcmCompartmentTransportWholeCellMvp Class Reference

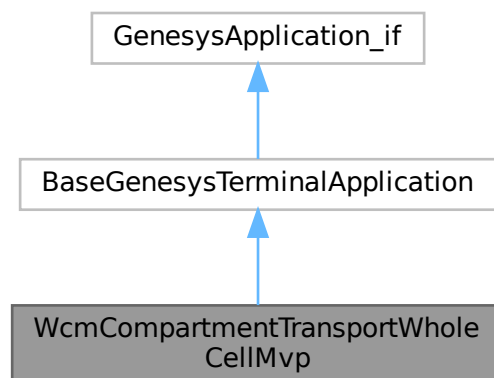
Whole-cell MVP with compartment-aware metabolic transport.

```
#include <applications/modelSpecific/wcm/WcmCompartmentTransportWholeCell↵
Mvp.h>
```

Inheritance diagram for WcmCompartmentTransportWholeCellMvp:



Collaboration diagram for WcmCompartmentTransportWholeCellMvp:



#### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

#### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) \*oem)

#### Additional Inherited Members

#### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)

- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

### 15.1075.1 Detailed Description

Whole-cell MVP with compartment-aware metabolic transport.

This example extends the flux-balance whole-cell path with a tiny extracellular-to-cytosol transport chain. It demonstrates how projected metabolic fluxes can feed:

- global energy proxies;
- compartment-scoped metabolite pools; and
- pathway activity checkpoints.

The biological scope is still didactic. The purpose is to validate the architecture for multi-compartment hybrid whole-cell simulations.

### 15.1075.2 Member Function Documentation

#### 15.1075.2.1 main()

```
int WcmCompartmentTransportWholeCellMvp::main (
 int argc,
 char ** argv) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/wcm/WcmCompartmentTransportWholeCellMvp.h
- applications/modelSpecific/wcm/WcmCompartmentTransportWholeCellMvp.cpp

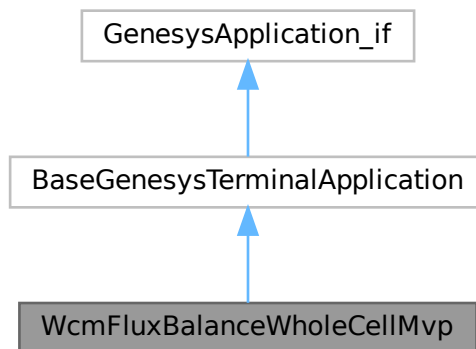
## 15.1076 WcmFluxBalanceWholeCellMvp Class Reference

Whole-cell MVP driven by flux-balance metabolism.

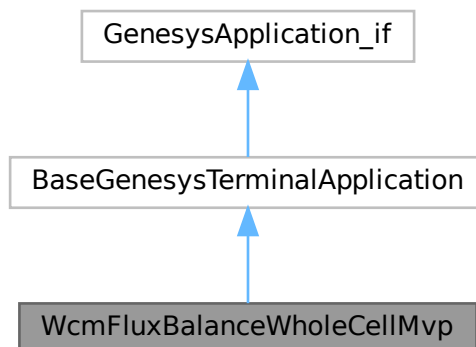
```
#include <applications/modelSpecific/wcm/WcmFluxBalanceWholeCellMvp.h>
```



Inheritance diagram for WcmFluxBalanceWholeCellMvp:



Collaboration diagram for WcmFluxBalanceWholeCellMvp:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)

- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

### 15.1076.1 Detailed Description

Whole-cell MVP driven by flux-balance metabolism.

This example demonstrates a second metabolic integration path for GenESyS whole-cell models. Instead of using the deterministic [BioNetwork](#) runner, it executes a small FBA problem through [MetabolicFluxBalance](#), projects selected fluxes into [WholeCellState](#), and then runs expression, growth, lifecycle checkpointing, fate routing, and division.

### 15.1076.2 Member Function Documentation

#### 15.1076.2.1 main()

```
int WcmFluxBalanceWholeCellMvp::main (
 int argc,
 char ** argv) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

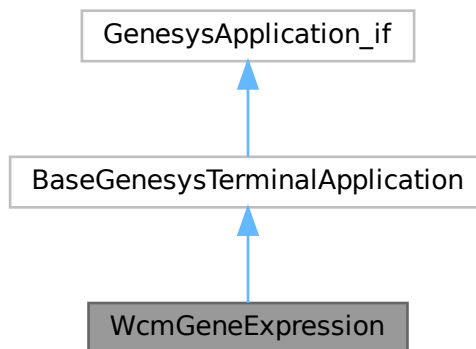
The documentation for this class was generated from the following files:

- applications/modelSpecific/wcm/WcmFluxBalanceWholeCellMvp.h
- applications/modelSpecific/wcm/WcmFluxBalanceWholeCellMvp.cpp

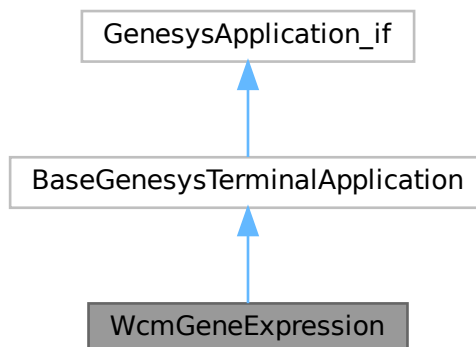
## 15.1077 WcmGeneExpression Class Reference

```
#include <applications/modelSpecific/wcm/WcmGeneExpression.h>
```

Inheritance diagram for WcmGeneExpression:



Collaboration diagram for WcmGeneExpression:



### Public Member Functions

- virtual int `main` (int argc, char \*\*argv) override

### Public Member Functions inherited from `BaseGenesysTerminalApplication`

- virtual void `Simulate` (`Simulator` \*genesys, `Model` \*model, `PluginManager` \*plugins)
- void `setDefaultTraceHandlers` (`TraceManager` \*tm)
- void `setDefaultEventHandlers` (`OnEventManager` \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from `BaseGenesysTerminalApplication`

- virtual void `traceHandler` (`TraceEvent` e)
- virtual void `traceErrorHandler` (`TraceErrorEvent` e)

- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

### 15.1077.1 Detailed Description

Central dogma simulation: stochastic transcription + translation + SSA degradation.

Models 4 genes using two complementary approximations:

- Poisson tau-leaping for transcription ([StochasticTranscription](#)) and translation ([StochasticTranslation](#)) — valid for high-copy-number RNAP/ribosome
- Gillespie Direct Method SSA for low-copy-number mRNA and protein degradation

Biological parameters (M. genitalium ballpark): RNAP\_free = 200 molecules ribosome\_free = 400 molecules 4 genes: geneA (metabolic), geneB (regulatory), geneC (structural), geneD (transport) Transcription: elongRate=50 nt/s, meanGeneLen=900 nt, bindProb=0.25,  $\Delta t=60$  s Translation: elongRate=16 AA/s, meanProtLen=300 AA,  $\Delta t=60$  s mRNA decay:  $k = 0.01/s$  (mean lifetime  $\approx 100$  s) Protein decay:  $k = 0.001/s$  (mean lifetime  $\approx 1000$  s  $\approx 16$  min)

Simulation: 10 replications  $\times$  3600 s, 60 s time steps.

Run via GENESYS\_TERMINAL\_EXAMPLE: `cmake -DGENESYS_TERMINAL_EXAMPLE=wcm/WcmGeneExpression.cpp ...`

### 15.1077.2 Member Function Documentation

#### 15.1077.2.1 main()

```
int WcmGeneExpression::main (
 int argc,
 char ** argv) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

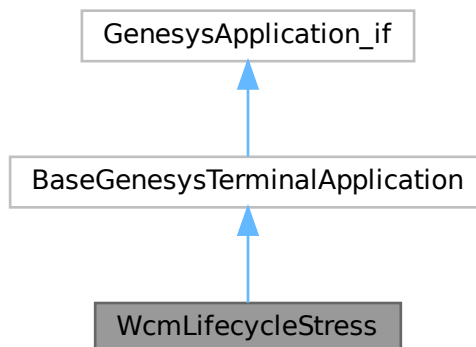
- applications/modelSpecific/wcm/WcmGeneExpression.h
- applications/modelSpecific/wcm/WcmGeneExpression.cpp

## 15.1078 WcmLifecycleStress Class Reference

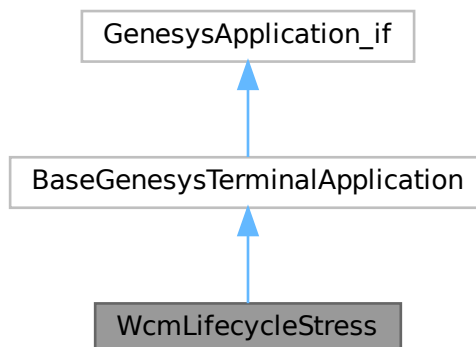
Demonstrates starvation and death routing in a whole-cell stress scenario.

```
#include <applications/modelSpecific/wcm/WcmLifecycleStress.h>
```

Inheritance diagram for WcmLifecycleStress:



Collaboration diagram for WcmLifecycleStress:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)

- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

### 15.1078.1 Detailed Description

Demonstrates starvation and death routing in a whole-cell stress scenario.

This example focuses on lifecycle checkpointing rather than biosynthesis. It uses a low-energy whole-cell state plus the [CellCycleCheckpointComponent](#) and [CellFateDecisionComponent](#) to show the transition: newborn -> starved -> dead.

### 15.1078.2 Member Function Documentation

#### 15.1078.2.1 main()

```
int WcmLifecycleStress::main (
 int argc,
 char ** argv) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

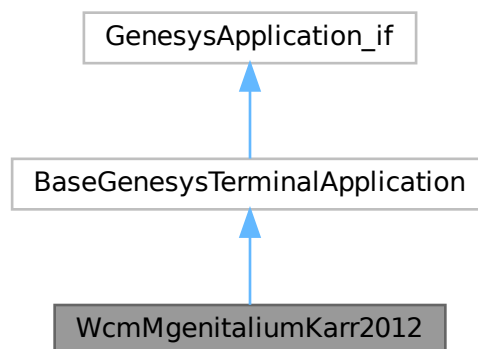
The documentation for this class was generated from the following files:

- applications/modelSpecific/wcm/WcmLifecycleStress.h
- applications/modelSpecific/wcm/WcmLifecycleStress.cpp

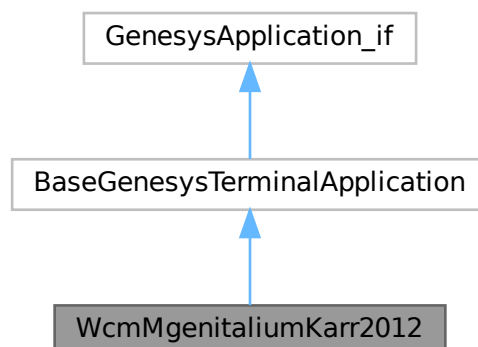
## 15.1079 WcmMgenitaliumKarr2012 Class Reference

```
#include <applications/modelSpecific/wcm/WcmMgenitaliumKarr2012.h>
```

Inheritance diagram for WcmMgenitaliumKarr2012:



Collaboration diagram for WcmMgenitaliumKarr2012:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)

- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

### 15.1079.1 Detailed Description

Whole-cell simulation analog of the M. genitalium model (Karr et al. 2012).

Reference: Karr JR, Sanghvi JC, Macklin DN, et al. "A Whole-Cell Computational Model Predicts Phenotype from Genotype." *Cell* 150(2):389–401, 2012. doi:10.1016/j.cell.2012.05.044

Architecture: This model uses all four GenESyS WholeCellModeling plugins in sequence:

SimClock → Transcription → Translation → MetabolicSSA → CellDivision |— port 0 (no division) → [Dispose](#) |— port 1 (division) → RecordDivision → [Dispose](#)

[StochasticTranscription](#): Poisson tau-leaping for 10 representative M. genitalium genes [StochasticTranslation](#): Poisson tau-leaping mapping mRNA\_ → prot\_ species [StochasticReactionComponent](#): Gillespie SSA for ATP/GTP metabolism + FtsZ ring polymerization [CellDivisionEvent](#): Triggers when FtsZ ring reaches 35% completion (350/1000 units) (mass-only trigger disabled; FtsZ-driven as in Karr et al.)

Biologically realistic parameters (from CovertLab/WholeCell parameters.json, MIT): RNAP\_free = 200 molecules (Karr et al. 2012, Supplementary Table S2) ribosome\_free = 400 molecules Transcription rate = 50 nt/s Translation rate = 16 AA/s Mean gene length = 900 nt Mean protein len = 300 AA [Cell](#) cycle ≈ 600 min (modeled as 600 × 60 s steps)

10 representative genes (selected from M. genitalium 473-gene model): MG\_001 — metabolic (glycolytic enzyme) MG\_006 — ATP synthase subunit (energy metabolism) MG\_055 — DNA polymerase III subunit alpha (DNA replication) MG\_139 — RNAP beta subunit (transcription machinery) MG\_175 — 30S ribosomal protein S1 (translation machinery) MG\_196 — FtsZ (cell division; drives the division trigger) MG\_203 — RNAP sigma factor (transcription regulation) MG\_339 — phosphoglycerate mutase (central metabolism) MG\_398 — pyruvate kinase (central metabolism) MG\_462 — ribosomal protein L7/L12 (translation machinery)

Simulation: 3 replications × 36000 s (10 h ≈ 1 M. genitalium cell cycle), 60 s steps.

Run via GENESYS\_TERMINAL\_EXAMPLE: cmake -DGENESYS\_TERMINAL\_EXAMPLE=wcm/WcmMgenitaliumKarr2012.cpp ...

### 15.1079.2 Member Function Documentation

#### 15.1079.2.1 main()

```
int WcmMgenitaliumKarr2012::main (
 int argc,
 char ** argv) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/wcm/WcmMgenitaliumKarr2012.h
- applications/modelSpecific/wcm/WcmMgenitaliumKarr2012.cpp

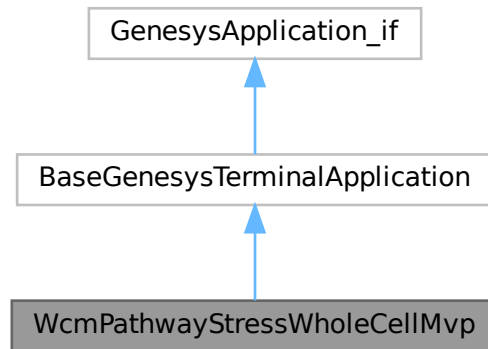


## 15.1080 WcmPathwayStressWholeCellMvp Class Reference

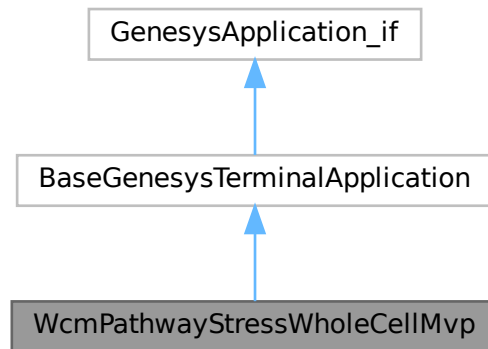
Whole-cell stress example driven by sustained low pathway activity.

```
#include <applications/modelSpecific/wcm/WcmPathwayStressWholeCellMvp.h>
```

Inheritance diagram for WcmPathwayStressWholeCellMvp:



Collaboration diagram for WcmPathwayStressWholeCellMvp:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

## Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

### 15.1080.1 Detailed Description

Whole-cell stress example driven by sustained low pathway activity.

This example keeps metabolism active but configures a monitored pathway threshold above the achievable FBA objective. The result is an arrest-then- death sequence driven by [PathwayStressResponseComponent](#).

### 15.1080.2 Member Function Documentation

#### 15.1080.2.1 main()

```
int WcmPathwayStressWholeCellMvp::main (
 int argc,
 char ** argv) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

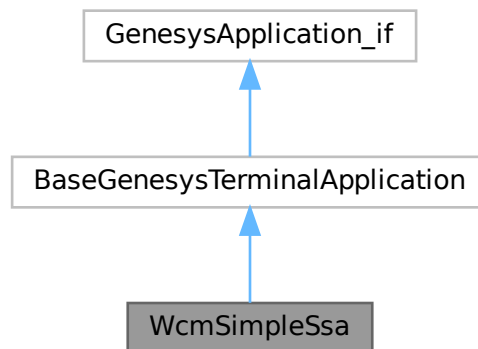
The documentation for this class was generated from the following files:

- applications/modelSpecific/wcm/WcmPathwayStressWholeCellMvp.h
- applications/modelSpecific/wcm/WcmPathwayStressWholeCellMvp.cpp

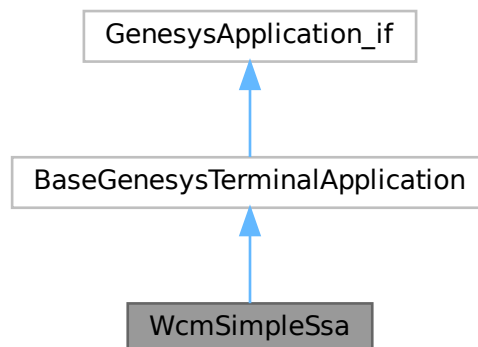
## 15.1081 WcmSimpleSsa Class Reference

```
#include <applications/modelSpecific/wcm/WcmSimpleSsa.h>
```

Inheritance diagram for WcmSimpleSsa:



Collaboration diagram for WcmSimpleSsa:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)

- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

### 15.1081.1 Detailed Description

Minimal whole-cell modeling example using the Gillespie Direct Method SSA.

Models stochastic gene expression of a single gene via 4 reactions: R1: transcription — DNA + RNAP → DNA + RNAP + mRNA\_geneA (k = 0.02/s) R2: mRNA degradation — mRNA\_geneA → ∅ (k = 0.01/s) R3: translation — mRNA\_geneA → mRNA\_geneA + prot\_geneA (k = 0.10/s) R4: protein degradation — prot\_geneA → ∅ (k = 0.001/s)

Steady-state analytic predictions: mRNA\_geneA ≈ 2 molecules (k\_tx / k\_deg\_mRNA = 0.02 / 0.01) prot\_geneA ≈ 200 molecules (k\_tl \* mRNA\_ss / k\_deg\_prot = 0.1 \* 2 / 0.001)

Simulation runs 30 replications × 3600 s each (1 hour of biological time) in 60 s Gillespie windows triggered by periodic clock entities.

Run via GENESYS\_TERMINAL\_EXAMPLE: cmake -DGENESYS\_TERMINAL\_EXAMPLE=wcm/WcmSimpleSsa.cpp ...

### 15.1081.2 Member Function Documentation

#### 15.1081.2.1 main()

```
int WcmSimpleSsa::main (
 int argc,
 char ** argv) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

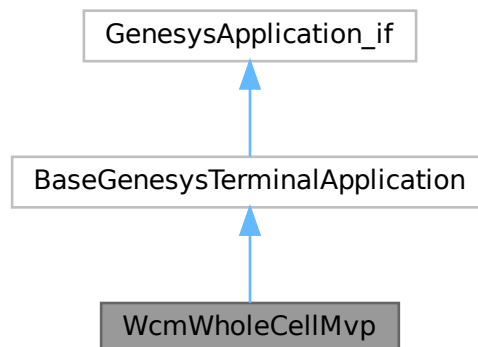
- applications/modelSpecific/wcm/WcmSimpleSsa.h
- applications/modelSpecific/wcm/WcmSimpleSsa.cpp

## 15.1082 WcmWholeCellMvp Class Reference

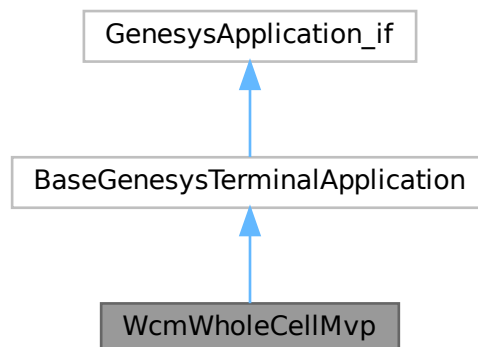
Minimal whole-cell MVP combining metabolism, expression, growth, and division.

```
#include <applications/modelSpecific/wcm/WcmWholeCellMvp.h>
```

Inheritance diagram for WcmWholeCellMvp:



Collaboration diagram for WcmWholeCellMvp:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)

- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

### 15.1082.1 Detailed Description

Minimal whole-cell MVP combining metabolism, expression, growth, and division. This example is intentionally small and didactic. It combines:

- deterministic biochemical metabolism through [BioNetwork](#);
- runtime projection into [WholeCellState](#);
- stochastic transcription and translation;
- energy-gated mass growth;
- mass-triggered cell division.

The result is not a calibrated organism model. It is a compact whole-cell analog intended to validate the GenESyS architecture for hybrid deterministic/stochastic cell simulations.

### 15.1082.2 Member Function Documentation

#### 15.1082.2.1 main()

```
int WcmWholeCellMvp::main (
 int argc,
 char ** argv) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

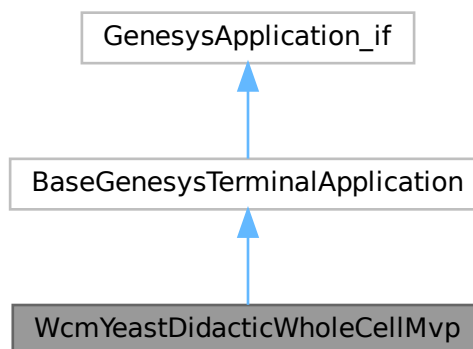
- applications/modelSpecific/wcm/WcmWholeCellMvp.h
- applications/modelSpecific/wcm/WcmWholeCellMvp.cpp

## 15.1083 WcmYeastDidacticWholeCellMvp Class Reference

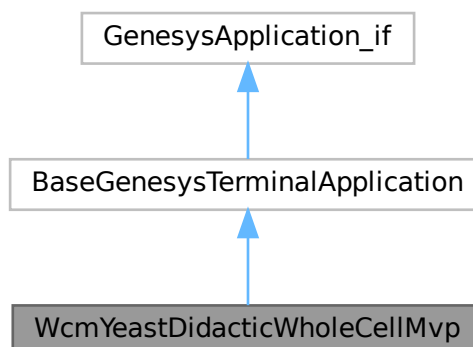
Didactic yeast-oriented whole-cell MVP built on GEM-inspired metabolism.

```
#include <applications/modelSpecific/wcm/WcmYeastDidacticWholeCellMvp.h>
```

Inheritance diagram for WcmYeastDidacticWholeCellMvp:



Collaboration diagram for WcmYeastDidacticWholeCellMvp:



### Public Member Functions

- virtual int `main` (int argc, char \*\*argv) override

### Public Member Functions inherited from `BaseGenesysTerminalApplication`

- virtual void `Simulate` (`Simulator` \*genesys, `Model` \*model, `PluginManager` \*plugins)
- void `setDefaultTraceHandlers` (`TraceManager` \*tm)
- void `setDefaultEventHandlers` (`OnEventManager` \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from `BaseGenesysTerminalApplication`

- virtual void `traceHandler` (`TraceEvent` e)
- virtual void `traceErrorHandler` (`TraceErrorEvent` e)

- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

### 15.1083.1 Detailed Description

Didactic yeast-oriented whole-cell MVP built on GEM-inspired metabolism.

This example is intentionally small and pedagogical. It combines a compact FBA network inspired by *Saccharomyces cerevisiae* carbon metabolism with whole-cell state projection, compartment exchange, stochastic expression, growth, lifecycle checkpointing, fate routing, and division.

It is not a canonical or validated yeast whole-cell model. It should be interpreted as a GenESyS architecture example informed by yeast-GEM style metabolic concepts, not as a public operational WCM for *S. cerevisiae*.

### 15.1083.2 Member Function Documentation

#### 15.1083.2.1 main()

```
int WcmYeastDidacticWholeCellMvp::main (
 int argc,
 char ** argv) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/wcm/WcmYeastDidacticWholeCellMvp.h
- applications/modelSpecific/wcm/WcmYeastDidacticWholeCellMvp.cpp

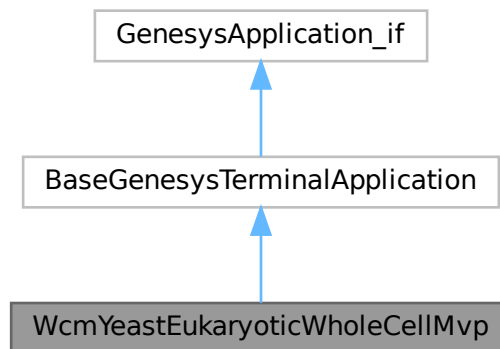
## 15.1084 WcmYeastEukaryoticWholeCellMvp Class Reference

Didactic eukaryotic whole-cell MVP inspired by budding yeast.

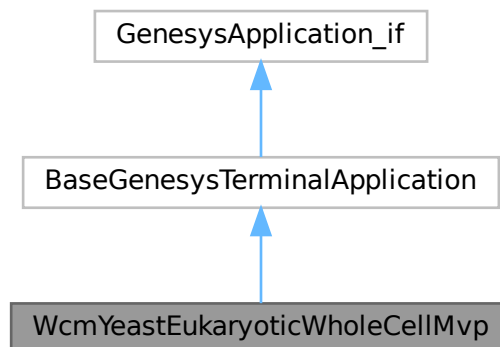
```
#include <applications/modelSpecific/wcm/WcmYeastEukaryoticWholeCellMvp.h>
```



Inheritance diagram for WcmYeastEukaryoticWholeCellMvp:



Collaboration diagram for WcmYeastEukaryoticWholeCellMvp:



### Public Member Functions

- virtual int [main](#) (int argc, char \*\*argv) override

### Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) \*genesys, [Model](#) \*model, [PluginManager](#) \*plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) \*tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) \*oem)

### Additional Inherited Members

### Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)

- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) \*re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) \*re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) \*re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) \*re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) \*re)

### 15.1084.1 Detailed Description

Didactic eukaryotic whole-cell MVP inspired by budding yeast.  
This example combines:

- a compact GEM-inspired FBA core;
- explicit biological compartments;
- compartment exchange and organelle energy proxies;
- stochastic expression and growth; and
- a simplified eukaryotic cell-cycle coordinator.

It is a didactic whole-cell example for *Saccharomyces cerevisiae*-inspired architecture. It is not a canonical public yeast whole-cell model.

### 15.1084.2 Member Function Documentation

#### 15.1084.2.1 main()

```
int WcmYeastEukaryoticWholeCellMvp::main (
 int argc,
 char ** argv) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

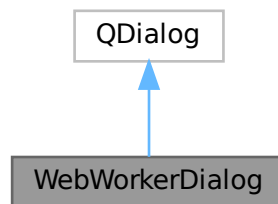
- applications/modelSpecific/wcm/WcmYeastEukaryoticWholeCellMvp.h
- applications/modelSpecific/wcm/WcmYeastEukaryoticWholeCellMvp.cpp

## 15.1085 WebWorkerDialog Class Reference

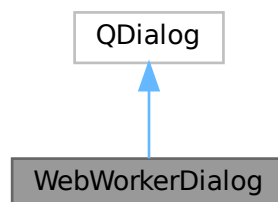
Lightweight control window for the embedded GenESyS web worker.

```
#include <applications/gui/httpworker/WebWorkerDialog.h>
```

Inheritance diagram for WebWorkerDialog:



Collaboration diagram for WebWorkerDialog:



### Public Member Functions

- **WebWorkerDialog** (QWidget \*parent=nullptr, bool allowClose=false)

### Protected Member Functions

- void **closeEvent** (QCloseEvent \*event) override
- void **showEvent** (QShowEvent \*event) override

### 15.1085.1 Detailed Description

Lightweight control window for the embedded GenESyS web worker.  
The documentation for this class was generated from the following files:

- applications/gui/httpworker/WebWorkerDialog.h
- applications/gui/httpworker/WebWorkerDialog.cpp

## 15.1086 WebWorkerRuntime Class Reference

Owns the embedded worker runtime used by the GUI and the standalone worker app.  
`#include <applications/worker/service/WebWorkerRuntime.h>`

## Classes

- struct [Snapshot](#)  
*Captures a thread-safe snapshot of the worker state.*

## Public Member Functions

- **WebWorkerRuntime** (unsigned short defaultPort=8080, unsigned long maxRequests=0)  
*Builds the reusable worker runtime with a default port and request cap.*
- **~WebWorkerRuntime** ()  
*Stops the worker and releases background resources.*
- bool [start](#) ()  
*Starts the worker if it is not already running.*
- void **stop** ()  
*Requests a graceful shutdown and waits for the worker thread to finish.*
- void **wait** ()  
*Waits for the worker thread to complete if it is running.*
- bool [restart](#) ()  
*Restarts the worker using the currently configured settings.*
- bool **isRunning** () const  
*Returns whether the worker thread is currently serving requests.*
- [Snapshot](#) **snapshot** () const  
*Returns a snapshot of the current worker lifecycle and counters.*
- void **setConfiguredPort** (unsigned short port)  
*Updates the configured listening port used by the next start.*
- unsigned short **configuredPort** () const  
*Returns the listening port configured for the next start.*
- void **setConfiguredMaxRequests** (unsigned long maxRequests)  
*Updates the configured request cap used by the next start.*
- unsigned long **configuredMaxRequests** () const  
*Returns the configured request cap used by the next start.*

## 15.1086.1 Detailed Description

Owns the embedded worker runtime used by the GUI and the standalone worker app.

The runtime wires the HTTP server, routing stack, and simulator-backed session services into a reusable lifecycle object. GUI code can start, stop, and inspect the worker without embedding HTTP details in the main window.

## 15.1086.2 Member Function Documentation

### 15.1086.2.1 restart()

```
bool WebWorkerRuntime::restart ()
```

Restarts the worker using the currently configured settings.

#### Returns

True when the worker was scheduled to start again.

**15.1086.2.2 start()**

```
bool WebWorkerRuntime::start ()
```

Starts the worker if it is not already running.

**Returns**

True when a new background worker thread was launched.

The documentation for this class was generated from the following files:

- applications/worker/service/WebWorkerRuntime.h
- applications/worker/service/WebWorkerRuntime.cpp

**15.1087 WholeCellParameterReader Class Reference**

```
#include <tools/Biochemical/WholeCellParameterReader.h>
```

**Static Public Member Functions**

- static std::map< std::string, double > **read** (const std::string &filePath)

**15.1087.1 Detailed Description**

Minimal JSON reader for the CovertLab/WholeCell M. genitalium parameter files.

Reads parameters.json and fixedConstants.json (MIT license) which have the two-level structure: {"processes": {"ProcessName": {"key": value, ...}, ...}, "states": {"StateName": {"key": value, ...}, ...}}

Scalar double values are extracted into a flat map keyed by "processes.ProcessName.key" or "states.StateName.key".

Arrays and nested objects are skipped. Non-numeric leaf values are ignored. No external JSON library is required.

Usage: auto params = WholeCellParameterReader::read("data/parameters.json"); double rate = params.count("processes.Transcription.rnaPolymeraseElongationRate") ? params.at("processes.Transcription.rnaPolymeraseElongationRate") : 50.0;

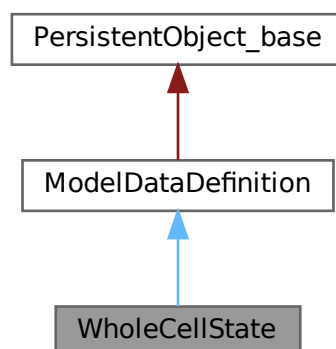
The documentation for this class was generated from the following file:

- tools/Biochemical/WholeCellParameterReader.h

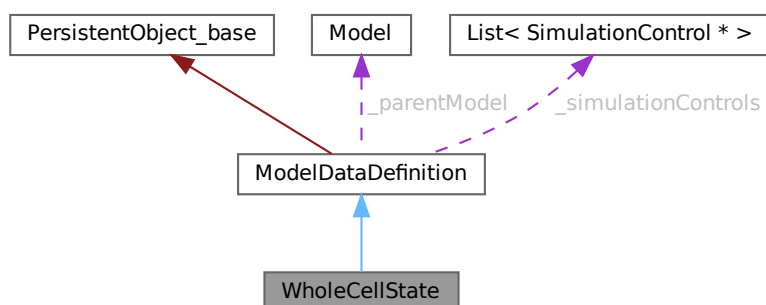
**15.1088 WholeCellState Class Reference**

```
#include <plugins/data/WholeCellModeling/WholeCellState.h>
```

Inheritance diagram for WholeCellState:



Collaboration diagram for WholeCellState:



## Public Member Functions

- **WholeCellState** ([Model](#) \*model, std::string name="")
- void **setMoleculeCount** (const std::string &speciesName, int count)
- int **getMoleculeCount** (const std::string &speciesName) const
- bool **hasMoleculeCount** (const std::string &speciesName) const
- const std::map< std::string, int > & **getMoleculeCounts** () const
- void **setMetaboliteAmount** (const std::string &metaboliteName, double amount)
- double **getMetaboliteAmount** (const std::string &metaboliteName) const
- const std::map< std::string, double > & **getMetabolitePool** () const
- void **setCompartmentMetaboliteAmount** (const std::string &compartmentName, const std::string &metaboliteName, double amount)
- double **getCompartmentMetaboliteAmount** (const std::string &compartmentName, const std::string &metaboliteName) const
- bool **hasCompartmentMetaboliteAmount** (const std::string &compartmentName, const std::string &metaboliteName) const
- const std::map< std::string, double > & **getCompartmentMetabolitePool** () const
- void **setPathwayActivity** (const std::string &pathwayName, double activity)
- double **getPathwayActivity** (const std::string &pathwayName) const
- bool **hasPathwayActivity** (const std::string &pathwayName) const
- const std::map< std::string, double > & **getPathwayActivities** () const
- void **setResourceBudget** (const std::string &speciesName, int budget)
- int **getResourceBudget** (const std::string &speciesName) const
- void **clearResourceBudget** ()
- void **setCellVolume** (double volume)
- double **getCellVolume** () const
- void **setCellMass** (double mass)
- double **getCellMass** () const
- void **setCurrentTime** (double time)
- double **getCurrentTime** () const
- void **setStepCount** (int stepCount)
- int **getStepCount** () const
- void **incrementStep** ()
- void **setLifecyclePhase** (const std::string &phase)
- std::string **getLifecyclePhase** () const
- void **setGenerationCount** (int generationCount)
- int **getGenerationCount** () const
- void **setViable** (bool viable)

- bool **isViable** () const
- void **setLastDivisionTime** (double time)
- double **getLastDivisionTime** () const
- bool **loadFixedConstants** (const std::string &jsonPath)
- bool **loadParameters** (const std::string &jsonPath)
- void **setFixedConstantsPath** (std::string path)
- std::string **getFixedConstantsPath** () const
- void **setParametersPath** (std::string path)
- std::string **getParametersPath** () const
- virtual std::string **show** () override

*Returns a textual representation of this data definition.*

## Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) \*model, std::string datatypeName, std::string name="", bool insert↵ IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- virtual ~[ModelDataDefinition](#) ()
- Releases resources owned by this data definition.*
- Util::identification [getId](#) () const
- Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)
- Updates the logical name of this instance.*
- const std::string & [getName](#) () const
- Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
- Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const
- Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const
- Returns the runtime class name used by the plugin system.*
- [Model](#) \* [getParentModel](#) () const
- Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const
- Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)
- Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) \* [getInternalData](#) (const std::string &name) const
- Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getInternalData](#) () const
- Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getAttachedData](#) () const
- Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const
- Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))
- Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const
- Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int \_modelLevel)
- Sets the model nesting level for this definition.*
- [List](#)< [SimulationControl](#) \* > \* [getSimulationControls](#) () const

- *Returns the simulation controls owned by this data definition.*
- `TraceManager::Level` [getTraceLevelSpecific](#) () const  
*Returns the trace level that applies specifically to this definition.*
- `void` [setTraceLevelSpecific](#) (`TraceManager::Level` level)  
*Updates the trace level used by this definition.*
- `void` [defineTraceLevelSpecific](#) (`TraceManager::Level` traceLevelspecific, `bool` traceLevelSpecific←  
Enabled=true)  
*Defines the specific trace level and optionally enables it immediately.*
- `bool` [isTraceLevelSpecificEnabled](#) () const  
*Indicates whether the specific trace level is enabled.*
- `void` [setTraceLevelSpecificEnabled](#) (`bool` traceLevelSpecificEnabled)  
*Enables or disables the specific trace level.*

### Static Public Member Functions

- `static` [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- `static` [PluginInformation](#) \* [GetPluginInformation](#) ()
- `static` [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, `std::string` name="")

### Static Public Member Functions inherited from [ModelDataDefinition](#)

- `static` [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields, `bool` insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- `static` [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, `std::string` name="")  
*Creates a new instance of a data definition using the plugin registry.*
- `static void` [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- `static bool` [Check](#) ([ModelDataDefinition](#) \*modeldatum, `std::string` &errorMessage)  
*Validates a data definition instance.*
- `static void` [InitBetweenReplications](#) ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*
- `static void` [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- `static bool` [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, `std::string` &errorMessage)  
*Creates related data and validates the instance in a single call.*

### Protected Member Functions

- `virtual bool` [\\_loadInstance](#) ([PersistenceRecord](#) \*fields) override  
*must be overridden by derived classes*
- `virtual void` [\\_saveInstance](#) ([PersistenceRecord](#) \*fields, `bool` saveDefaultValues) override  
*Saves object fields into a serialization structure.*
- `virtual bool` [\\_check](#) (`std::string` &errorMessage) override  
*could be overridden by derived classes*
- `virtual void` [\\_initBetweenReplications](#) () override



## Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [\\_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) \* [\\_getParserChangesInformation](#) ()
- virtual void [\\_addSimulationControl](#) ([SimulationControl](#) \*control)
- virtual void [\\_createInternalAndAttachedData](#) ()
- virtual void [\\_createInternalStatisticReporters](#) ()
- virtual void [\\_createAttachedAttributes](#) ()
- virtual void [\\_createNonEditableDataDefinitions](#) ()
- virtual void [\\_createEditableDataDefinitions](#) ()
- void [\\_templateCreateInternalStatisticReporters](#) ()
- void [\\_templateCreateAttachedAttributes](#) ()
- void [\\_templateCreateNonEditableDataDefinitions](#) ()
- void [\\_templateCreateEditableDataDefinitions](#) ()
- void [\\_internalStatisticReportersClear](#) ()
  - legacy helper methods used by the granular create\* hooks*
- void [\\_internalDataClear](#) ()
- void [\\_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*child)
- void [\\_internalDataRemove](#) (const std::string &key)
- void [\\_attachedDataClear](#) ()
- void [\\_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_attachedDataRemove](#) (const std::string &key)
- void [\\_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [\\_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_statisticReporterRemove](#) (const std::string &key)
- void [\\_statisticReportersClear](#) ()
- void [\\_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [\\_mandatoryAttachedAttributesClear](#) ()
- void [\\_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_mandatoryEditableDataDefinitionsClear](#) ()
- void [\\_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_optionalEditableDataDefinitionsClear](#) ()
- void [\\_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [\\_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_optionalNonEditableDataDefinitionsClear](#) ()
- void [\\_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
  - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))
  - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1\\_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2\\_results](#))
- void [traceSimulation](#) (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))
- void [traceSimulation](#) (void \*thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))
  - Traces text to the simulation output.*
- void [traceSimulation](#) (void \*thisobject, [TraceManager::Level](#) level, const std::string &text)

## Additional Inherited Members

### Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **\_id**
- std::string **\_typename**
- bool **\_reportStatistics**
- bool **\_hasChanged**
- unsigned int **\_modelLevel** = 0
- [Model](#) \* **\_parentModel**
- [List](#)< [SimulationControl](#) \* > \* **\_simulationControls** = new [List](#)<[SimulationControl](#)\*>()
- TraceManager::Level **\_traceLevelSpecific**
- bool **\_traceLevelSpecificEnabled** = false

### 15.1088.1 Detailed Description

Shared state container for whole-cell model simulations.

[WholeCellState](#) is the GenESyS analog to the Vivarium [Store](#): a single [ModelDataDefinition](#) instance shared by all WCM component plugins in a simulation step. It holds:

- discrete molecule counts (mRNA, proteins) updated by SSA
- continuous metabolite pool (ATP, amino acids) updated by FBA
- resource budget allocated by fair-allocation before each step
- cell geometry (volume, mass) for propensity scaling

JSON loading (Q5a) reads fixedConstants.json and parameters.json from the CovertLab/WholeCell M. genitalium repository (MIT license).

### 15.1088.2 Member Function Documentation

#### 15.1088.2.1 `_check()`

```
bool WholeCellState::_check (
 std::string & errorMessage) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

#### 15.1088.2.2 `_initBetweenReplications()`

```
void WholeCellState::_initBetweenReplications () [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

#### 15.1088.2.3 `_loadInstance()`

```
bool WholeCellState::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

#### 15.1088.2.4 `_saveInstance()`

```
void WholeCellState::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

#### Parameters

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [ModelDataDefinition](#).

### 15.1088.2.5 show()

```
std::string WholeCellState::show () [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

#### Returns

Human-readable summary used by traces and debug output.

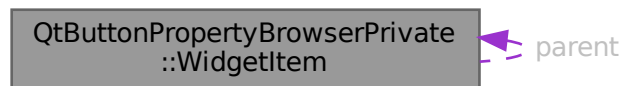
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- plugins/data/WholeCellModeling/WholeCellState.h
- plugins/data/WholeCellModeling/WholeCellState.cpp

## 15.1089 QPushButtonPropertyBrowserPrivate::WidgetItem Struct Reference

Collaboration diagram for QPushButtonPropertyBrowserPrivate::WidgetItem:



#### Public Attributes

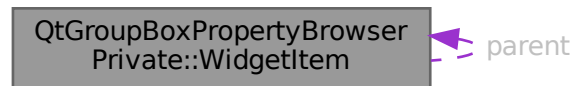
- QWidget \* **widget** {nullptr}
- QLabel \* **label** {nullptr}
- QLabel \* **widgetLabel** {nullptr}
- QToolButton \* **button** {nullptr}
- QWidget \* **container** {nullptr}
- QGridLayout \* **layout** {nullptr}
- [WidgetItem](#) \* **parent** {nullptr}
- QList< [WidgetItem](#) \* > **children**
- bool **expanded** {false}

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtbuttonpropertybrowser.cpp

## 15.1090 QtGroupBoxPropertyBrowserPrivate::WidgetItem Struct Reference

Collaboration diagram for QtGroupBoxPropertyBrowserPrivate::WidgetItem:



### Public Attributes

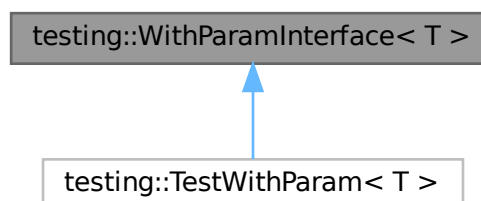
- QWidget \* **widget** {nullptr}
- QLabel \* **label** {nullptr}
- QLabel \* **widgetLabel** {nullptr}
- QGroupBox \* **groupBox** {nullptr}
- QGridLayout \* **layout** {nullptr}
- QFrame \* **line** {nullptr}
- [WidgetItem](#) \* **parent** {nullptr}
- QList< [WidgetItem](#) \* > **children**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtgroupboxpropertybrowser.cpp

## 15.1091 testing::WithParamInterface< T > Class Template Reference

Inheritance diagram for testing::WithParamInterface< T >:



### Public Types

- typedef T **ParamType**

### Static Public Member Functions

- static const ParamType & **GetParam** ()

### Friends

- template<class TestClass >  
class **internal::ParameterizedTestFactory**

The documentation for this class was generated from the following file:

- gtest/gtest/gtest.h

## 15.1092 SimulatorSessionService::WorkerCapabilitiesResult Struct Reference

Describes currently supported worker features.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

### Public Attributes

- bool **supportsSessionApi** = false
- bool **supportsSessionScopedSimulator** = false
- bool **supportsModelCreation** = false
- bool **supportsModelPersistence** = false
- bool **supportsSimulationStatus** = false
- bool **supportsSimulationConfig** = false
- bool **supportsSynchronousRun** = false
- bool **supportsSynchronousStep** = false
- bool **supportsDistributedJobs** = false
- bool **supportsJobPolling** = false
- bool **supportsBackgroundExecution** = false
- bool **supportsModelUpload** = false
- bool **supportsStreamingEvents** = false
- bool **supportsJobResultRetrieval** = false

### 15.1092.1 Detailed Description

Describes currently supported worker features.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

## 15.1093 SimulatorSessionService::WorkerInfoResult Struct Reference

Describes worker identity metadata for distributed discovery.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

### Public Attributes

- std::string **role**
- std::string **application**
- std::string **apiFamily**
- std::string **apiVersion**
- std::string **simulatorName**
- std::string **simulatorVersionName**
- unsigned int **simulatorVersionNumber** = 0

### 15.1093.1 Detailed Description

Describes worker identity metadata for distributed discovery.

The documentation for this struct was generated from the following file:

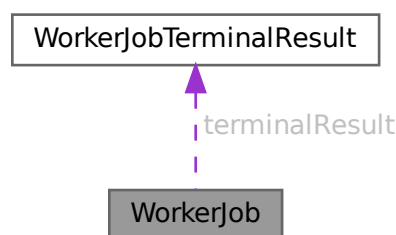
- applications/worker/service/SimulatorSessionService.h

### 15.1094 WorkerJob Struct Reference

Stores metadata for a worker job snapshot owned by the web worker.

```
#include <applications/worker/worker/WorkerJob.h>
```

Collaboration diagram for WorkerJob:



#### Public Attributes

- std::string **jobId**  
*Stable job identifier returned to API clients.*
- std::string **sessionId**  
*Session that owns the job snapshot and execution context.*
- WorkerJobState **state** = WorkerJobState::Queued  
*Current lifecycle state of the job.*
- std::string **snapshotFilename**  
*Snapshot filename stored in the session workspace.*
- std::string **createdMarker**  
*Monotonic creation marker used to preserve job ordering.*
- std::string **message**  
*Human-readable status or error text.*
- bool **hasTerminalResult** = false  
*True when terminalResult contains valid completion data.*
- [WorkerJobTerminalResult](#) **terminalResult**  
*Final simulation summary written when the job finishes.*

### 15.1094.1 Detailed Description

Stores metadata for a worker job snapshot owned by the web worker.

The documentation for this struct was generated from the following file:

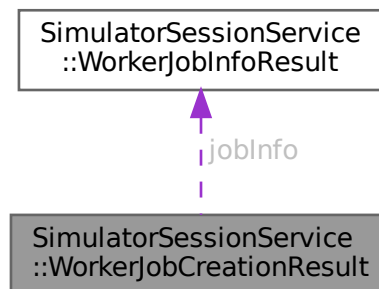
- applications/worker/worker/WorkerJob.h

## 15.1095 SimulatorSessionService::WorkerJobCreationResult Struct Reference

Contains worker job creation output and error state.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Collaboration diagram for SimulatorSessionService::WorkerJobCreationResult:



### Public Attributes

- bool **success** = false
- [WorkerJobError](#) **error** = WorkerJobError::None
- [WorkerJobInfoResult](#) **jobInfo**

### 15.1095.1 Detailed Description

Contains worker job creation output and error state.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

## 15.1096 SimulatorSessionService::WorkerJobInfoResult Struct Reference

Describes worker job metadata exposed by worker stage 3 endpoints.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

### Public Attributes

- std::string **jobId**
- std::string **sessionId**
- WorkerJobState **state** = WorkerJobState::Queued
- std::string **snapshotFilename**
- std::string **createdMarker**
- std::string **message**

### 15.1096.1 Detailed Description

Describes worker job metadata exposed by worker stage 3 endpoints.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

## 15.1097 WorkerJobManager Class Reference

Owns worker job records in memory and assigns unique identifiers.

```
#include <applications/worker/worker/WorkerJobManager.h>
```

### Public Member Functions

- [WorkerJob createQueuedJob](#) (const std::string &sessionId)  
*Creates and stores a queued worker job with generated identifiers.*
- bool [setSnapshotFilename](#) (const std::string &jobId, const std::string &snapshotFilename)  
*Replaces the snapshot filename for an existing job.*
- bool [setState](#) (const std::string &jobId, WorkerJobState state)  
*Updates the lifecycle state for an existing job.*
- bool [setMessage](#) (const std::string &jobId, const std::string &message)  
*Replaces the message text for an existing job.*
- bool [setTerminalResult](#) (const std::string &jobId, const [WorkerJobTerminalResult](#) &terminalResult)  
*Persists the terminal simulation summary for an existing job.*
- std::optional< [WorkerJob](#) > [getJob](#) (const std::string &jobId) const  
*Fetches a previously stored job record.*

### 15.1097.1 Detailed Description

Owns worker job records in memory and assigns unique identifiers.

The manager is intentionally simple: it keeps all job metadata in memory and provides synchronized read/write helpers for the web API and background jobs.

### 15.1097.2 Member Function Documentation

#### 15.1097.2.1 createQueuedJob()

```
WorkerJob WorkerJobManager::createQueuedJob (
 const std::string & sessionId)
```

Creates and stores a queued worker job with generated identifiers.

Creates and stores a queued worker job with a monotonic identifier.

#### Parameters

|                  |                                   |
|------------------|-----------------------------------|
| <i>sessionId</i> | Identifier of the owning session. |
|------------------|-----------------------------------|

#### Returns

Newly stored worker job metadata.

#### 15.1097.2.2 getJob()

```
std::optional< WorkerJob > WorkerJobManager::getJob (
 const std::string & jobId) const
```

Fetches a previously stored job record.

Retrieves a copy of a job record when it exists.

#### Parameters

|              |                 |
|--------------|-----------------|
| <i>jobId</i> | Job identifier. |
|--------------|-----------------|



**Returns**

Copy of the job when found.

**15.1097.2.3 setMessage()**

```
bool WorkerJobManager::setMessage (
 const std::string & jobId,
 const std::string & message)
```

Replaces the message text for an existing job.

Replaces the human-readable message for a stored job.

**Parameters**

|                |                                         |
|----------------|-----------------------------------------|
| <i>jobId</i>   | Job identifier to update.               |
| <i>message</i> | Human-readable status or error message. |

**Returns**

True when the job exists and was updated.

**15.1097.2.4 setSnapshotFilename()**

```
bool WorkerJobManager::setSnapshotFilename (
 const std::string & jobId,
 const std::string & snapshotFilename)
```

Replaces the snapshot filename for an existing job.

Updates the snapshot filename for a stored job.

**Parameters**

|                         |                                                   |
|-------------------------|---------------------------------------------------|
| <i>jobId</i>            | Job identifier to update.                         |
| <i>snapshotFilename</i> | Snapshot filename persisted in session workspace. |

**Returns**

True when the job exists and was updated.

**15.1097.2.5 setState()**

```
bool WorkerJobManager::setState (
 const std::string & jobId,
 WorkerJobState state)
```

Updates the lifecycle state for an existing job.

Updates the lifecycle state for a stored job.

**Parameters**

|              |                           |
|--------------|---------------------------|
| <i>jobId</i> | Job identifier to update. |
| <i>state</i> | New worker job state.     |

**Returns**

True when the job exists and was updated.

### 15.1097.2.6 setTerminalResult()

```
bool WorkerJobManager::setTerminalResult (
 const std::string & jobId,
 const WorkerJobTerminalResult & terminalResult)
```

Persists the terminal simulation summary for an existing job.  
Stores the terminal simulation result and marks it available.

#### Parameters

|                       |                                     |
|-----------------------|-------------------------------------|
| <i>jobId</i>          | Job identifier to update.           |
| <i>terminalResult</i> | Terminal simulation summary values. |

#### Returns

True when the job exists and was updated.

The documentation for this class was generated from the following files:

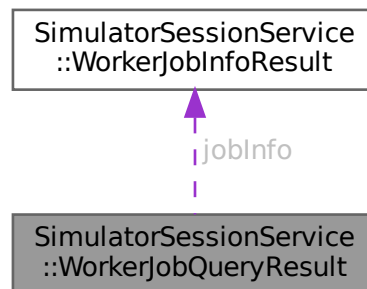
- applications/worker/worker/WorkerJobManager.h
- applications/worker/worker/WorkerJobManager.cpp

## 15.1098 SimulatorSessionService::WorkerJobQueryResult Struct Reference

Contains worker job lookup output and error state.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Collaboration diagram for SimulatorSessionService::WorkerJobQueryResult:



#### Public Attributes

- bool **success** = false
- [WorkerJobError](#) **error** = WorkerJobError::None
- [WorkerJobInfoResult](#) **jobInfo**

### 15.1098.1 Detailed Description

Contains worker job lookup output and error state.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

## 15.1099 SimulatorSessionService::WorkerJobResultInfo Struct Reference

Describes terminal worker job result data exposed by worker stage 5 endpoint.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

### Public Attributes

- std::string **jobId**
- WorkerJobState **state** = WorkerJobState::Queued
- std::string **message**
- double **simulatedTime** = 0.0
- unsigned int **currentReplicationNumber** = 0
- unsigned int **numberOfReplications** = 0
- double **replicationLength** = 0.0
- double **warmUpPeriod** = 0.0
- bool **hasIsPaused** = false
- bool **isPaused** = false

### 15.1099.1 Detailed Description

Describes terminal worker job result data exposed by worker stage 5 endpoint.

The documentation for this struct was generated from the following file:

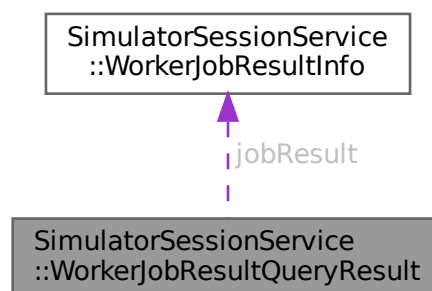
- applications/worker/service/SimulatorSessionService.h

## 15.1100 SimulatorSessionService::WorkerJobResultQueryResult Struct Reference

Contains worker job terminal result lookup output and error state.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Collaboration diagram for SimulatorSessionService::WorkerJobResultQueryResult:



### Public Attributes

- bool **success** = false
- WorkerJobError **error** = WorkerJobError::None
- WorkerJobResultInfo **jobResult**

### 15.1100.1 Detailed Description

Contains worker job terminal result lookup output and error state.

The documentation for this struct was generated from the following file:

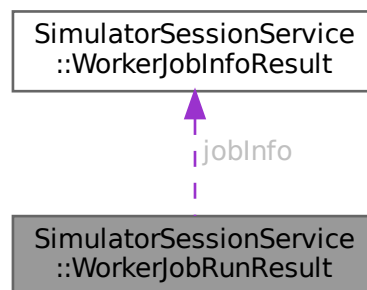
- applications/worker/service/SimulatorSessionService.h

## 15.1101 SimulatorSessionService::WorkerJobRunResult Struct Reference

Contains worker job run output and error state.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Collaboration diagram for SimulatorSessionService::WorkerJobRunResult:



### Public Attributes

- bool **success** = false
- [WorkerJobError](#) **error** = WorkerJobError::None
- [WorkerJobInfoResult](#) **jobInfo**

### 15.1101.1 Detailed Description

Contains worker job run output and error state.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

## 15.1102 WorkerJobTerminalResult Struct Reference

Stores the terminal simulation summary persisted for a finished or failed worker job.

```
#include <applications/worker/worker/WorkerJob.h>
```

### Public Attributes

- double **simulatedTime** = 0.0  
*Simulated time reached when the job completed.*
- unsigned int **currentReplicationNumber** = 0  
*Current replication number at completion.*
- unsigned int **numberOfReplications** = 0

*Total number of replications configured for the run.*

- double **replicationLength** = 0.0

*Replication length used by the run.*

- double **warmUpPeriod** = 0.0

*Warm-up period used by the run.*

- std::optional< bool > **isPaused**

*Pause state when the simulator explicitly reported it.*

### 15.1102.1 Detailed Description

Stores the terminal simulation summary persisted for a finished or failed worker job.

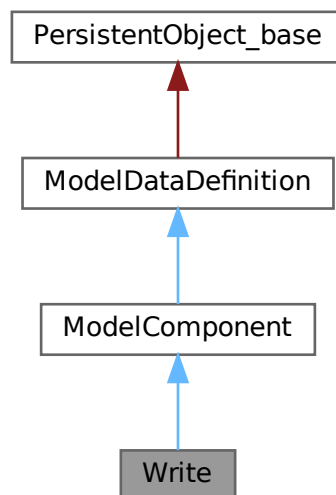
The documentation for this struct was generated from the following file:

- applications/worker/worker/WorkerJob.h

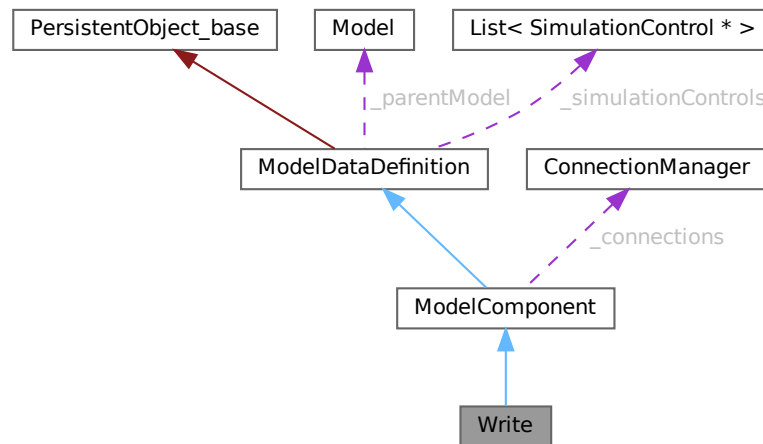
## 15.1103 Write Class Reference

```
#include <plugins/components/InputOutput/Write.h>
```

Inheritance diagram for Write:



Collaboration diagram for Write:



## Public Types

- enum class **WriteToType** : int { **SCREEN** = 0 , **FILE** = 1 , **num\_elements** = 2 }

## Public Member Functions

- **Write** ([Model](#) \*model, std::string name="")
- void **insertText** (std::list< std::string > texts)
- void **setFilename** (std::string \_filename)
- std::string **filename** () const
- void **setWriteToType** (WriteToType \_writeToType)
- Write::WriteToType **writeToType** () const
- virtual std::string **show** () override

*virtual public methods*

## Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) \*model, std::string componentTypename, std::string name="")  
*constructors*
- virtual ~**ModelComponent** ()  
*Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) \* **getConnectionManager** () const  
*new public user methods for this component*
- void **connectTo** ([ModelComponent](#) \*component, unsigned int inputPortNumber=0)  
*Connects this component to another component.*
- bool **hasBreakpointAt** ()  
*Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string \_description)  
*Updates the free-form description shown by the GUI.*
- std::string **getDescription** () const  
*Returns the free-form description associated with this component.*
- void **remove** ([ModelComponent](#) \*comp)  
*Removes a connected component from this component.*

## Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** \*model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
 

*Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
 

*Releases resources owned by this data definition.*
- Util::identification **getId** () const
 

*Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
 

*Updates the logical name of this instance.*
- const std::string & **getName** () const
 

*Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)
 

*Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const
 

*Returns the user-visible label of this instance.*
- const std::string & **getClassname** () const
 

*Returns the runtime class name used by the plugin system.*
- **Model** \* **getParentModel** () const
 

*Returns the parent model that owns this definition.*
- bool **isReportStatistics** () const
 

*Indicates whether this data definition contributes report statistics.*
- void **setReportStatistics** (bool reportStatistics)
 

*Enables or disables report-statistics generation for this definition.*
- **ModelDataDefinition** \* **getInternalData** (const std::string &name) const
 

*Looks up a named internal child data definition.*
- std::map< std::string, **ModelDataDefinition** \* > \* **getInternalData** () const
 

*Returns the full map of internal child data definitions.*
- std::map< std::string, **ModelDataDefinition** \* > \* **getAttachedData** () const
 

*Returns the full map of attached data definitions.*
- bool **hasChanged** () const
 

*Indicates whether this instance has been modified since the last reset.*
- void **setHasChanged** (bool **hasChanged**)
 

*Updates the changed flag for this data definition.*
- unsigned int **getLevel** () const
 

*Returns the level in the model hierarchy where this definition lives.*
- void **setModelLevel** (unsigned int \_modelLevel)
 

*Sets the model nesting level for this definition.*
- **List**< **SimulationControl** \* > \* **getSimulationControls** () const
 

*Returns the simulation controls owned by this data definition.*
- TraceManager::Level **getTraceLevelSpecific** () const
 

*Returns the trace level that applies specifically to this definition.*
- void **setTraceLevelSpecific** (TraceManager::Level level)
 

*Updates the trace level used by this definition.*
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)
 

*Defines the specific trace level and optionally enables it immediately.*
- bool **isTraceLevelSpecificEnabled** () const
 

*Indicates whether the specific trace level is enabled.*
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)
 

*Enables or disables the specific trace level.*

### Static Public Member Functions

- static std::string **convertEnumToStr** (WriteToType type)
- static [PluginInformation](#) \* **GetPluginInformation** ()
- static [ModelComponent](#) \* **LoadInstance** ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* **NewInstance** ([Model](#) \*model, std::string name="")

### Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) \* **LoadInstance** ([Model](#) \*model, [PersistenceRecord](#) \*fields)  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool **Check** ([ModelComponent](#) \*component)  
*Validates a component instance.*
- static void **CreateInternalData** ([ModelComponent](#) \*component)  
*Creates internal data required by a component instance.*
- static void **SaveInstance** ([PersistenceRecord](#) \*fields, [ModelComponent](#) \*component)  
*Serializes a component instance into a persistence record.*
- static void **DispatchEvent** ([Event](#) \*event)  
*Dispatches a simulation event to the target component.*

### Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* **LoadInstance** ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* **NewInstance** ([Model](#) \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void **SaveInstance** ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool **Check** ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void **InitBetweenReplications** ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*
- static void **CreateInternalData** ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

### Protected Member Functions

- virtual void **\_onDispatchEvent** ([Entity](#) \*entity, unsigned int inputPortNumber) override  
*Handles a dispatched event for this component type.*
- virtual bool **\_loadInstance** ([PersistenceRecord](#) \*fields) override  
*virtual protected method that must be overridden*
- virtual void **\_saveInstance** ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*
- virtual bool **\_check** (std::string &errorMessage) override  
*could be overridden by derived classes*
- virtual void **\_initBetweenReplications** () override



## Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [\\_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) \* [\\_getParserChangesInformation](#) ()
- virtual void [\\_addSimulationControl](#) ([SimulationControl](#) \*control)
- virtual void [\\_createInternalAndAttachedData](#) ()
- virtual void [\\_createInternalStatisticReporters](#) ()
- virtual void [\\_createAttachedAttributes](#) ()
- virtual void [\\_createNonEditableDataDefinitions](#) ()
- virtual void [\\_createEditableDataDefinitions](#) ()
- void [\\_templateCreateInternalStatisticReporters](#) ()
- void [\\_templateCreateAttachedAttributes](#) ()
- void [\\_templateCreateNonEditableDataDefinitions](#) ()
- void [\\_templateCreateEditableDataDefinitions](#) ()
- void [\\_internalStatisticReportersClear](#) ()
  - legacy helper methods used by the granular create\* hooks*
- void [\\_internalDataClear](#) ()
- void [\\_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*child)
- void [\\_internalDataRemove](#) (const std::string &key)
- void [\\_attachedDataClear](#) ()
- void [\\_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_attachedDataRemove](#) (const std::string &key)
- void [\\_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [\\_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_statisticReporterRemove](#) (const std::string &key)
- void [\\_statisticReportersClear](#) ()
- void [\\_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [\\_mandatoryAttachedAttributesClear](#) ()
- void [\\_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_mandatoryEditableDataDefinitionsClear](#) ()
- void [\\_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_optionalEditableDataDefinitionsClear](#) ()
- void [\\_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [\\_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_optionalNonEditableDataDefinitionsClear](#) ()
- void [\\_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
  - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))
  - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1\\_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2\\_results](#))
- void [traceSimulation](#) (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))
- void [traceSimulation](#) (void \*thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))
  - Traces text to the simulation output.*
- void [traceSimulation](#) (void \*thisobject, [TraceManager::Level](#) level, const std::string &text)

## Additional Inherited Members

### Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) \* `_connections` = new [ConnectionManager](#)()  
*virtual protected methods that could be overridden by derived classes, if needed*

### Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) \* `_parentModel`
- [List](#)< [SimulationControl](#) \* > \* `_simulationControls` = new [List](#)<[SimulationControl](#)\*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

## 15.1103.1 Detailed Description

This component ...

## 15.1103.2 Member Function Documentation

### 15.1103.2.1 `_check()`

```
bool Write::_check (
 std::string & errorMessage) [override], [protected], [virtual]
```

could be overridden by derived classes  
Reimplemented from [ModelDataDefinition](#).

### 15.1103.2.2 `_initBetweenReplications()`

```
void Write::_initBetweenReplications () [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

### 15.1103.2.3 `_loadInstance()`

```
bool Write::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

virtual protected method that must be overridden  
Reimplemented from [ModelComponent](#).

### 15.1103.2.4 `_onDispatchEvent()`

```
void Write::_onDispatchEvent (
 Entity * entity,
 unsigned int inputPortNumber) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

#### Parameters

|                        |                                         |
|------------------------|-----------------------------------------|
| <i>entity</i>          | <a href="#">Entity</a> being processed. |
| <i>inputPortNumber</i> | Input port where the entity arrived.    |

Implements [ModelComponent](#).

**15.1103.2.5 \_saveInstance()**

```
void Write::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

**Parameters**

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [ModelComponent](#).

**15.1103.2.6 show()**

```
std::string Write::show () [override], [virtual]
virtual public methods
```

Returns a textual representation of this component.

**Returns**

Human-readable description used for tracing and debugging.

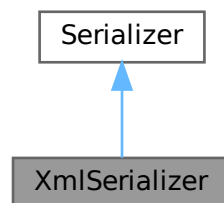
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

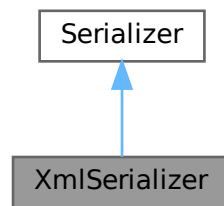
- plugins/components/InputOutput/Write.h
- plugins/components/InputOutput/Write.cpp

**15.1104 XmlSerializer Class Reference**

Inheritance diagram for XmlSerializer:



Collaboration diagram for XmlSerializer:



### Public Member Functions

- **XmlSerializer** ([Model](#) \*model)
- [PersistenceRecord](#) \* **newPersistenceRecord** () override  
*Creates an empty persistence record.*
- bool **dump** (std::ostream &output) override  
*Writes formatted contents to a given output stream.*
- bool **load** (std::istream &input) override  
*Attempts to load contents from a given input stream.*
- bool **get** (const std::string &name, [PersistenceRecord](#) \*entry) override  
*Tries to fetch an entry with the given label as a key.*
- bool **put** (const std::string name, const std::string type, const Util::identification id, [PersistenceRecord](#) \*fields) override  
*Upserts an entry into this structure.*
- int **for\_each** (std::function< int(const std::string &) > delegate) override  
*Iterates over this structure's entries, ordered by ID.*

## 15.1104.1 Member Function Documentation

### 15.1104.1.1 dump()

```
bool XmlSerializer::dump (
 std::ostream & output) [override], [virtual]
```

Writes formatted contents to a given output stream.

#### Parameters

|               |                            |
|---------------|----------------------------|
| <i>output</i> | output stream to write to. |
|---------------|----------------------------|

#### Returns

whether the current state of this structure's contents was deemed valid for serialization.

Implements [Serializer](#).

### 15.1104.1.2 for\_each()

```
int XmlSerializer::for_each (
 std::function< int(const std::string &) > delegate) [override], [virtual]
```

Iterates over this structure's entries, ordered by ID.

## Parameters

|                 |                                                                                              |
|-----------------|----------------------------------------------------------------------------------------------|
| <i>delegate</i> | delegate which is applied to every entry-identifying key, until it returns a non-zero value. |
|-----------------|----------------------------------------------------------------------------------------------|

## Returns

either zero, or the first non-zero value returned by the delegate.

Implements [Serializer](#).

**15.1104.1.3 get()**

```
bool XmlSerializer::get (
 const std::string & name,
 PersistenceRecord * entry) [override], [virtual]
```

Tries to fetch an entry with the given label as a key.

## Parameters

|              |                                                                            |
|--------------|----------------------------------------------------------------------------|
| <i>name</i>  | a label which uniquely identifies an entry.                                |
| <i>entry</i> | (out-param) a pointer to a caller-owned entry, which is filled on success. |

## Returns

whether the entry was found and the out-param was actually written to.

Implements [Serializer](#).

**15.1104.1.4 load()**

```
bool XmlSerializer::load (
 std::istream & input) [override], [virtual]
```

Attempts to load contents from a given input stream.

## Returns

whether the input was well-formatted and loading was successful.

Implements [Serializer](#).

**15.1104.1.5 newPersistenceRecord()**

```
PersistenceRecord * XmlSerializer::newPersistenceRecord () [override], [virtual]
```

Creates an empty persistence record.

Although the returned record can be freed normally with `delete`, care must be taken to only use records which come from a implementation, when configuring that implementation's contents.

Implements [Serializer](#).

**15.1104.1.6 put()**

```
bool XmlSerializer::put (
 const std::string name,
 const std::string type,
 const Util::identification id,
 PersistenceRecord * fields) [override], [virtual]
```

Upserts an entry into this structure.

## Parameters

|             |                                              |
|-------------|----------------------------------------------|
| <i>name</i> | a label which uniquely identifies the entry. |
|-------------|----------------------------------------------|

## Parameters

|               |                                                                             |
|---------------|-----------------------------------------------------------------------------|
| <i>type</i>   | a type name.                                                                |
| <i>id</i>     | a component/datadef ID number.                                              |
| <i>fields</i> | a persistence record for the entry, whose contents will be copied if valid. |

## Returns

whether the entry was valid and was actually copied into this structure.

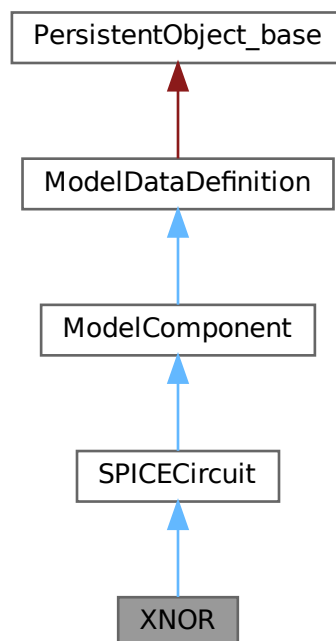
Implements [Serializer](#).

The documentation for this class was generated from the following files:

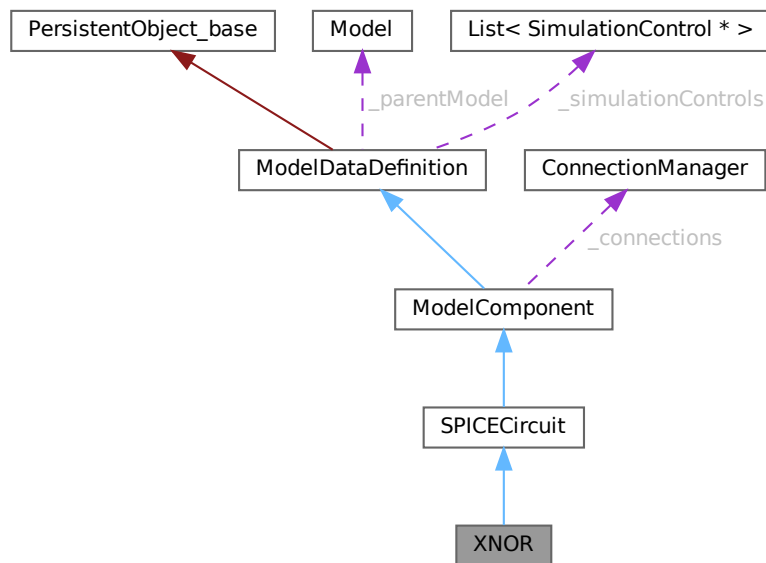
- kernel/simulator/persistence/XmlSerializer.h
- kernel/simulator/persistence/XmlSerializer.cpp

## 15.1105 XNOR Class Reference

Inheritance diagram for XNOR:



Collaboration diagram for XNOR:



### Public Member Functions

- **XNOR** ([Model](#) \*model, std::string name="")
- void **Build** () override
- void **setPMOSWidth** (float PMOSWidth)
- void **setPMOSLength** (float length)
- void **setNMOSWidth** (float NMOSWidth)
- void **setNMOSLength** (float NMOSLength)
- void **setElectricalModel** (std::string electricalModel)

### Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) \*model, std::string name="")

*constructors*

- void **setRunner** ([SPICERunner](#) \*runner)

*new public user methods for this components*

- [SPICERunner](#) \* **getRunner** ()
- virtual std::string **show** () override

*virtual public methods*

### Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) \*model, std::string componentTypename, std::string name="")

*constructors*

- virtual ~**ModelComponent** ()

*Releases component-owned resources and unregisters it from the model.*

- [ConnectionManager](#) \* **getConnectionManager** () const

*new public user methods for this component*

- void **connectTo** ([ModelComponent](#) \*component, unsigned int inputPortNumber=0)

- Connects this component to another component.*

  - bool [hasBreakpointAt](#) ()
  - Indicates whether a simulation breakpoint is set on this component.*
  - void [setDescription](#) (std::string \_description)
  - Updates the free-form description shown by the GUI.*
  - std::string [getDescription](#) () const
  - Returns the free-form description associated with this component.*
  - void [remove](#) ([ModelComponent](#) \*comp)
  - Removes a connected component from this component.*

## Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) \*model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- virtual ~[ModelDataDefinition](#) ()
- Releases resources owned by this data definition.*
- Util::identification [getId](#) () const
- Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)
- Updates the logical name of this instance.*
- const std::string & [getName](#) () const
- Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
- Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const
- Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const
- Returns the runtime class name used by the plugin system.*
- [Model](#) \* [getParentModel](#) () const
- Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const
- Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)
- Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) \* [getInternalData](#) (const std::string &name) const
- Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getInternalData](#) () const
- Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getAttachedData](#) () const
- Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const
- Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))
- Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const
- Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int \_modelLevel)
- Sets the model nesting level for this definition.*
- List< [SimulationControl](#) \* > \* [getSimulationControls](#) () const
- Returns the simulation controls owned by this data definition.*



- `TraceManager::Level` `getTraceLevelSpecific ()` const  
*Returns the trace level that applies specifically to this definition.*
- void `setTraceLevelSpecific` (`TraceManager::Level` level)  
*Updates the trace level used by this definition.*
- void `defineTraceLevelSpecific` (`TraceManager::Level` traceLevelspecific, bool traceLevelSpecific, bool Enabled=true)  
*Defines the specific trace level and optionally enables it immediately.*
- bool `isTraceLevelSpecificEnabled` () const  
*Indicates whether the specific trace level is enabled.*
- void `setTraceLevelSpecificEnabled` (bool traceLevelSpecificEnabled)  
*Enables or disables the specific trace level.*

### Static Public Member Functions

- static `PluginInformation` \* `GetPluginInformation` ()
- static `ModelComponent` \* `LoadInstance` (`Model` \*model, `PersistenceRecord` \*fields)
- static `ModelDataDefinition` \* `NewInstance` (`Model` \*model, std::string name="")

### Static Public Member Functions inherited from `SPICECircuit`

- static `PluginInformation` \* `GetPluginInformation` ()  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static `ModelComponent` \* `LoadInstance` (`Model` \*model, `PersistenceRecord` \*fields)
- static `ModelDataDefinition` \* `NewInstance` (`Model` \*model, std::string name="")
- static std::vector< std::string > `split` (std::string word, char split\_char)  
*Externalize function GetPluginInformation to be accessible through dynamic linked library.*

### Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent` \* `LoadInstance` (`Model` \*model, `PersistenceRecord` \*fields)  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool `Check` (`ModelComponent` \*component)  
*Validates a component instance.*
- static void `CreateInternalData` (`ModelComponent` \*component)  
*Creates internal data required by a component instance.*
- static void `SaveInstance` (`PersistenceRecord` \*fields, `ModelComponent` \*component)  
*Serializes a component instance into a persistence record.*
- static void `DispatchEvent` (`Event` \*event)  
*Dispatches a simulation event to the target component.*

### Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition` \* `LoadInstance` (`Model` \*model, `PersistenceRecord` \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static `ModelDataDefinition` \* `NewInstance` (`Model` \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void `SaveInstance` (`PersistenceRecord` \*fields, `ModelDataDefinition` \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool `Check` (`ModelDataDefinition` \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void `InitBetweenReplications` (`ModelDataDefinition` \*modeldatum)

*Resets simulation-time state between replications.*

- static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

### Protected Member Functions

- virtual bool [\\_loadInstance](#) ([PersistenceRecord](#) \*fields) override  
*virtual protected method that must be overridden*
- virtual void [\\_saveInstance](#) ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*

### Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the\_params={}, std::string the\_model="", std::string the\_model\_file="")  
*virtual protected method that must be overridden*
- void **UpdateConnections** ()
- virtual void [\\_onDispatchEvent](#) ([Entity](#) \*entity, unsigned int inputPortNumber) override  
*This method is only for ModelComponents, not ModelDataElements.*
- virtual void [\\_createNonEditableDataDefinitions](#) () override  
*virtual protected methods that could be overridden by derived classes, if needed*

### Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [\\_getSaveDefaultsOption](#) ()
- virtual bool [\\_check](#) (std::string &errorMessage)  
*could be overridden by derived classes*
- virtual [ParserChangesInformation](#) \* [\\_getParserChangesInformation](#) ()
- virtual void [\\_initBetweenReplications](#) ()
- virtual void [\\_addSimulationControl](#) ([SimulationControl](#) \*control)
- virtual void [\\_createInternalAndAttachedData](#) ()
- virtual void [\\_createInternalStatisticReporters](#) ()
- virtual void [\\_createAttachedAttributes](#) ()
- virtual void [\\_createEditableDataDefinitions](#) ()
- void [\\_templateCreateInternalStatisticReporters](#) ()
- void [\\_templateCreateAttachedAttributes](#) ()
- void [\\_templateCreateNonEditableDataDefinitions](#) ()
- void [\\_templateCreateEditableDataDefinitions](#) ()
- void [\\_internalStatisticReportersClear](#) ()  
*legacy helper methods used by the granular create\* hooks*
- void [\\_internalDataClear](#) ()
- void [\\_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*child)
- void [\\_internalDataRemove](#) (const std::string &key)
- void [\\_attachedDataClear](#) ()
- void [\\_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_attachedDataRemove](#) (const std::string &key)
- void [\\_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [\\_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_statisticReporterRemove](#) (const std::string &key)
- void [\\_statisticReportersClear](#) ()
- void [\\_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryAttachedAttributeRemove](#) (const std::string &key)

- void **\_mandatoryAttachedAttributesClear** ()
- void **\_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryEditableDataDefinitionsClear** ()
- void **\_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalEditableDataDefinitionsClear** ()
- void **\_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_mandatoryNonEditableDataDefinitionsClear** ()
- void **\_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) \*data)
- void **\_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **\_optionalNonEditableDataDefinitionsClear** ()
- void **\_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)  
*method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)  
*not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)
- void [traceSimulation](#) (void \*thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)  
*Traces text to the simulation output.*
- void **traceSimulation** (void \*thisobject, TraceManager::Level level, const std::string &text)

### Additional Inherited Members

### Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

### Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) \* **\_connections** = new [ConnectionManager](#)()  
*virtual protected methods that could be overridden by derived classes, if needed*

### Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **\_id**
- std::string **\_typename**
- bool **\_reportStatistics**
- bool **\_hasChanged**
- unsigned int **\_modelLevel** = 0
- [Model](#) \* **\_parentModel**
- [List](#)< [SimulationControl](#) \* > \* **\_simulationControls** = new [List](#)<[SimulationControl](#)\*>()
- TraceManager::Level **\_traceLevelSpecific**
- bool **\_traceLevelSpecificEnabled** = false

## 15.1105.1 Member Function Documentation

### 15.1105.1.1 `_loadInstance()`

```
bool XNOR::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

### 15.1105.1.2 `_saveInstance()`

```
void XNOR::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

#### Parameters

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [SPICECircuit](#).

### 15.1105.1.3 `Build()`

```
void XNOR::Build () [override], [virtual]
```

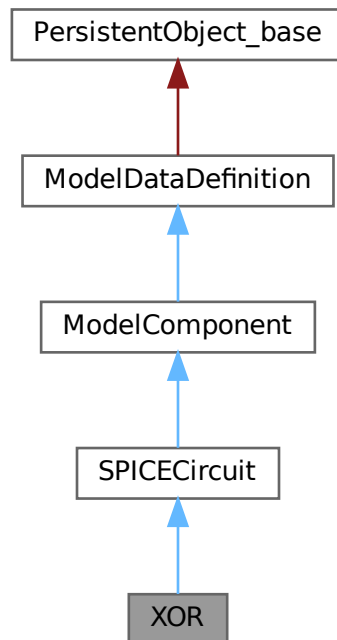
Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

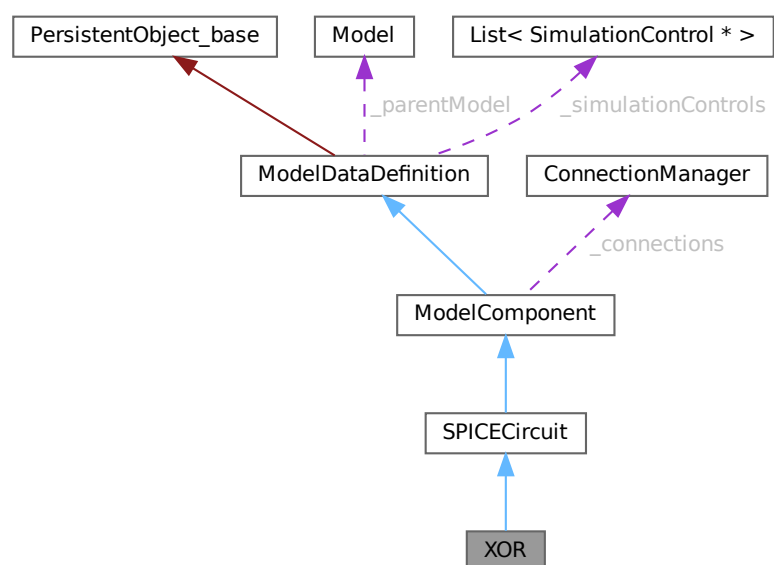
- `plugins/components/ElectronicsSimulation/SPICECircuit.h`
- `plugins/components/ElectronicsSimulation/SPICECircuit.cpp`

## 15.1106 XOR Class Reference

Inheritance diagram for XOR:



Collaboration diagram for XOR:



## Public Member Functions

- **XOR** ([Model](#) \*model, std::string name="")
- void **Build** () override
- void **setPMOSWidth** (float PMOSWidth)
- void **setPMOSLength** (float length)
- void **setNMOSWidth** (float NMOSWidth)
- void **setNMOSLength** (float NMOSLength)
- void **setElectricalModel** (std::string electricalModel)

## Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) \*model, std::string name="")

*constructors*

- void **setRunner** ([SPICERunner](#) \*runner)  
*new public user methods for this components*
- [SPICERunner](#) \* **getRunner** ()
- virtual std::string **show** () override  
*virtual public methods*

## Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) \*model, std::string componentTypename, std::string name="")

*constructors*

- virtual ~**ModelComponent** ()  
*Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) \* **getConnectionManager** () const  
*new public user methods for this component*
- void **connectTo** ([ModelComponent](#) \*component, unsigned int inputPortNumber=0)  
*Connects this component to another component.*
- bool **hasBreakpointAt** ()  
*Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string \_description)  
*Updates the free-form description shown by the GUI.*
- std::string **getDescription** () const  
*Returns the free-form description associated with this component.*
- void **remove** ([ModelComponent](#) \*comp)  
*Removes a connected component from this component.*

## Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) \*model, std::string datadefinitionTypename, std::string name="", bool insert↵  
IntoModel=true)  
*Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()  
*Releases resources owned by this data definition.*
- Util::identification **getId** () const  
*Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)  
*Updates the logical name of this instance.*
- const std::string & **getName** () const  
*Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)

- Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const  
*Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const  
*Returns the runtime class name used by the plugin system.*
- [Model](#) \* [getParentModel](#) () const  
*Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const  
*Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)  
*Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) \* [getInternalData](#) (const std::string &name) const  
*Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getInternalData](#) () const  
*Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) \* > \* [getAttachedData](#) () const  
*Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const  
*Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))  
*Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const  
*Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int \_modelLevel)  
*Sets the model nesting level for this definition.*
- List< [SimulationControl](#) \* > \* [getSimulationControls](#) () const  
*Returns the simulation controls owned by this data definition.*
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const  
*Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)  
*Updates the trace level used by this definition.*
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)  
*Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const  
*Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)  
*Enables or disables the specific trace level.*

### Static Public Member Functions

- static [PluginInformation](#) \* [GetPluginInformation](#) ()
- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")

### Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation](#) \* [GetPluginInformation](#) ()  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")
- static std::vector< std::string > [split](#) (std::string word, char split\_char)  
*Externalize function GetPluginInformation to be accessible through dynamic linked library.*

## Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields)  
*static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool [Check](#) ([ModelComponent](#) \*component)  
*Validates a component instance.*
- static void [CreateInternalData](#) ([ModelComponent](#) \*component)  
*Creates internal data required by a component instance.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelComponent](#) \*component)  
*Serializes a component instance into a persistence record.*
- static void [DispatchEvent](#) ([Event](#) \*event)  
*Dispatches a simulation event to the target component.*

## Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) \* [LoadInstance](#) ([Model](#) \*model, [PersistenceRecord](#) \*fields, bool insertIntoModel)  
*Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) \* [NewInstance](#) ([Model](#) \*model, std::string name="")  
*Creates a new instance of a data definition using the plugin registry.*
- static void [SaveInstance](#) ([PersistenceRecord](#) \*fields, [ModelDataDefinition](#) \*modeldatum)  
*Serializes a data definition instance into a persistence record.*
- static bool [Check](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Validates a data definition instance.*
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) \*modeldatum)  
*Resets simulation-time state between replications.*
- static void [CreateInternalData](#) ([ModelDataDefinition](#) \*modeldatum)  
*Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) \*modeldatum, std::string &errorMessage)  
*Creates related data and validates the instance in a single call.*

## Protected Member Functions

- virtual bool [\\_loadInstance](#) ([PersistenceRecord](#) \*fields) override  
*virtual protected method that must be overridden*
- virtual void [\\_saveInstance](#) ([PersistenceRecord](#) \*fields, bool saveDefaultValues) override  
*Saves object fields into a serialization structure.*

## Protected Member Functions inherited from [SPICECircuit](#)

- void [BuildCircuit](#) (std::string description, unsigned int id, std::vector< std::string > the\_params={}, std::string the\_model="", std::string the\_model\_file="")  
*virtual protected method that must be overridden*
- void [UpdateConnections](#) ()
- virtual void [\\_onDispatchEvent](#) ([Entity](#) \*entity, unsigned int inputPortNumber) override  
*This method is only for ModelComponents, not ModelDataElements.*
- virtual void [\\_createNonEditableDataDefinitions](#) () override  
*virtual protected methods that could be overridden by derived classes, if needed*



## Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [\\_getSaveDefaultsOption](#) ()
- virtual bool [\\_check](#) (std::string &errorMessage)  
*could be overridden by derived classes*
- virtual [ParserChangesInformation](#) \* [\\_getParserChangesInformation](#) ()
- virtual void [\\_initBetweenReplications](#) ()
- virtual void [\\_addSimulationControl](#) ([SimulationControl](#) \*control)
- virtual void [\\_createInternalAndAttachedData](#) ()
- virtual void [\\_createInternalStatisticReporters](#) ()
- virtual void [\\_createAttachedAttributes](#) ()
- virtual void [\\_createEditableDataDefinitions](#) ()
- void [\\_templateCreateInternalStatisticReporters](#) ()
- void [\\_templateCreateAttachedAttributes](#) ()
- void [\\_templateCreateNonEditableDataDefinitions](#) ()
- void [\\_templateCreateEditableDataDefinitions](#) ()
- void [\\_internalStatisticReportersClear](#) ()  
*legacy helper methods used by the granular create\* hooks*
- void [\\_internalDataClear](#) ()
- void [\\_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*child)
- void [\\_internalDataRemove](#) (const std::string &key)
- void [\\_attachedDataClear](#) ()
- void [\\_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_attachedDataRemove](#) (const std::string &key)
- void [\\_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [\\_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_statisticReporterRemove](#) (const std::string &key)
- void [\\_statisticReportersClear](#) ()
- void [\\_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [\\_mandatoryAttachedAttributesClear](#) ()
- void [\\_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_mandatoryEditableDataDefinitionsClear](#) ()
- void [\\_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_optionalEditableDataDefinitionsClear](#) ()
- void [\\_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [\\_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) \*data)
- void [\\_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [\\_optionalNonEditableDataDefinitionsClear](#) ()
- void [\\_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)  
*method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))  
*not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1\\_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2\\_results](#))
- void **traceSimulation** (void \*thisobject, double time, [Entity](#) \*entity, [ModelComponent](#) \*component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))
- void [traceSimulation](#) (void \*thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8\\_detailed](#))  
*Traces text to the simulation output.*
- void **traceSimulation** (void \*thisobject, [TraceManager::Level](#) level, const std::string &text)

## Additional Inherited Members

### Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

### Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) \* **\_connections** = new [ConnectionManager](#)()  
*virtual protected methods that could be overridden by derived classes, if needed*

### Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **\_id**
- std::string **\_typename**
- bool **\_reportStatistics**
- bool **\_hasChanged**
- unsigned int **\_modelLevel** = 0
- [Model](#) \* **\_parentModel**
- [List](#)< [SimulationControl](#) \* > \* **\_simulationControls** = new [List](#)<[SimulationControl](#)\*>()
- [TraceManager](#)::Level **\_traceLevelSpecific**
- bool **\_traceLevelSpecificEnabled** = false

## 15.1106.1 Member Function Documentation

### 15.1106.1.1 [\\_loadInstance\(\)](#)

```
bool XOR::_loadInstance (
 PersistenceRecord * fields) [override], [protected], [virtual]
```

virtual protected method that must be overridden  
Reimplemented from [SPICECircuit](#).

### 15.1106.1.2 [\\_saveInstance\(\)](#)

```
void XOR::_saveInstance (
 PersistenceRecord * fields,
 bool saveDefaultValues) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

#### Parameters

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>fields</i>            | a table-like structure used for serialization.                                |
| <i>saveDefaultValues</i> | indicates whether or not to save fields which are set to their default value. |

Reimplemented from [SPICECircuit](#).

### 15.1106.1.3 [Build\(\)](#)

```
void XOR::Build () [override], [virtual]
```

Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

- plugins/components/ElectronicsSimulation/SPICECircuit.h
- plugins/components/ElectronicsSimulation/SPICECircuit.cpp

## 15.1107 yy\_buffer\_state Struct Reference

### Public Attributes

- FILE \* **yy\_input\_file**
- char \* **yy\_ch\_buf**
- char \* **yy\_buf\_pos**
- int **yy\_buf\_size**
- int **yy\_n\_chars**
- int **yy\_is\_our\_buffer**
- int **yy\_is\_interactive**
- int **yy\_at\_bol**
- int **yy\_bs\_lineno**
- int **yy\_bs\_column**
- int **yy\_fill\_buffer**
- int **yy\_buffer\_status**

### 15.1107.1 Member Data Documentation

#### 15.1107.1.1 yy\_bs\_column

```
int yy_buffer_state::yy_bs_column
```

The column count.

#### 15.1107.1.2 yy\_bs\_lineno

```
int yy_buffer_state::yy_bs_lineno
```

The line count.

The documentation for this struct was generated from the following file:

- parser/Genesys++-scanner.cpp

## 15.1108 yy\_trans\_info Struct Reference

### Public Attributes

- flex\_int32\_t **yy\_verify**
- flex\_int32\_t **yy\_nxt**

The documentation for this struct was generated from the following file:

- parser/Genesys++-scanner.cpp



# Chapter 16

## File Documentation

### 16.1 BaseGenesysTerminalApplication.h

```
00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: BaseGenesysTerminalApplication.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 16:25
00012 */
00013
00014 #ifndef BaseGenesysTerminalApplication_H
00015 #define BaseGenesysTerminalApplication_H
00016
00017 #include "GenesysApplication_if.h"
00018 #include "kernel/simulator/TraceManager.h"
00019 #include "kernel/simulator/OnEventManager.h"
00020 #include "kernel/simulator/PluginManager.h"
00021 #include "../kernel/simulator/model/Model.h"
00022
00023 #include <thread>
00024
00025 class BaseGenesysTerminalApplication : public GenesysApplication_if {
00026 public:
00027 BaseGenesysTerminalApplication();
00028 virtual ~BaseGenesysTerminalApplication() = default;
00029 public:
00030 virtual int main(int argc, char** argv) override;
00031 virtual void Simulate(Simulator* genesys, Model* model, PluginManager *plugins);
00032 public:
00033 void setDefaultTraceHandlers(TraceManager* tm);
00034 void setDefaultEventHandlers(OnEventManager* oem);
00035 protected:
00036 // default Trace Handlers
00037 virtual void traceHandler(TraceEvent e);
00038 virtual void traceErrorHandler(TraceErrorEvent e);
00039 virtual void traceReportHandler(TraceEvent e);
00040 virtual void traceSimulationHandler(TraceSimulationEvent e);
00041 // default Event Handlers
00042 virtual void onAfterProcessEventHandler(SimulationEvent* re);
00043 virtual void onBreakpointHandler(SimulationEvent* re);
00044 virtual void onEntityCreateHandler(SimulationEvent* re);
00045 virtual void onEntityMoveHandler(SimulationEvent* re);
00046 virtual void onSimulationStartHandler(SimulationEvent* re);
00047 virtual void onReplicationStepHandler(SimulationEvent* re);
00048 virtual void onReplicationStartHandler(SimulationEvent* re);
00049 virtual void onProcessEventHandler(SimulationEvent* re);
00050 virtual void onReplicationEndHandler(SimulationEvent* re);
00051 virtual void onSimulationEndHandler(SimulationEvent* re);
00052 virtual void onSimulationPausedHandler(SimulationEvent* re);
00053 virtual void onSimulationResumeHandler(SimulationEvent* re);
00054 virtual void onEntityRemoveHandler(SimulationEvent* re);
00055 private:
00056
00057 };
00058
00059 #endif /* BaseGenesysTerminalApplication_H */
00060
```

## 16.2 GenesysApplication\_if.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: GenesysApplication_if.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 2 de Outubro de 2018, 19:14
00012 */
00013
00014 #ifndef GENESYSAPPLICATION_IF_H
00015 #define GENESYSAPPLICATION_IF_H
00016
00017 class GenesysApplication_if {
00018 public:
00019 virtual ~GenesysApplication_if() = default;
00020 virtual int main(int argc, char** argv) = 0;
00021 };
00022
00023 #endif /* GENESYSAPPLICATION_IF_H */

```

## 16.3 AIAssistantWindow.h

```

00001 #ifndef AIASSISTANTWINDOW_H
00002 #define AIASSISTANTWINDOW_H
00003
00004 #include "../tools/AIAssistant/AIAssistantDefaultImpl.h"
00005 #include "../tools/AIAssistant/AISecretStore.h"
00006 #include "../kernel/simulator/SimulatorFacade.h"
00007
00008 #include <QMainWindow>
00009
00010 class QAction;
00011 class QCheckBox;
00012 class QComboBox;
00013 class QDoubleSpinBox;
00014 class QLabel;
00015 class QLineEdit;
00016 class QPlainTextEdit;
00017 class QPushButton;
00018 class QSpinBox;
00019 class QTabWidget;
00020 class QTableWidget;
00021 class QTextBrowser;
00022 class Simulator;
00023
00024 class AIAssistantWindow : public QMainWindow {
00025 Q_OBJECT
00026 public:
00027 explicit AIAssistantWindow(Simulator* simulator, QWidget* parent = nullptr);
00028 ~AIAssistantWindow() override;
00029
00030 private:
00031 void buildMenus();
00032 void buildWorkspace();
00033 void connectActions();
00034
00035 QWidget* buildConfigTab();
00036 QWidget* buildPromptTab();
00037 QWidget* buildExecutionTab();
00038 QWidget* buildAuditTab();
00039
00040 void applyConfigurationToBackend();
00041 void runAnalyzePrompt();
00042 void runCreateModelPlan();
00043 void runBuildModel();
00044 void runConfigureSimulation();
00045 void runSimulation();
00046 void runCollectResults();
00047 void runFullPipeline();
00048 void saveApiKeyToKeyring();
00049 void loadApiKeyFromKeyring();
00050 void refreshAuditTable();
00051 void exportAuditLog();
00052 void appendLog(const QString& text);
00053 void setStatus(const QString& text);
00054 void setRunning(bool running);
00055 AIAssistantRequest buildRequest() const;
00056 QString providerKeyringAccount() const;

```

```

00057
00058 private:
00059 Simulator* _simulator = nullptr;
00060 SimulatorFacade _facade;
00061 AIAssistantDefaultImpl _assistant;
00062
00063 // Configuration tab
00064 QComboBox* _providerCombo = nullptr;
00065 QLineEdit* _modelNameEdit = nullptr;
00066 QLineEdit* _apiKeyEdit = nullptr;
00067 QPushButton* _apiKeyToggleBtn = nullptr;
00068 QPushButton* _saveKeyringBtn = nullptr;
00069 QPushButton* _loadKeyringBtn = nullptr;
00070 QLineEdit* _apiKeyEnvVarEdit = nullptr;
00071 QLineEdit* _baseUrlEdit = nullptr;
00072 QSpinBox* _timeoutSpin = nullptr;
00073 QDoubleSpinBox* _temperatureSpin = nullptr;
00074 QCheckBox* _allowNetworkChk = nullptr;
00075 QCheckBox* _allowMutationChk = nullptr;
00076 QCheckBox* _allowExecutionChk = nullptr;
00077 QCheckBox* _allowFilesystemChk = nullptr;
00078 QCheckBox* _sandboxChk = nullptr;
00079 QCheckBox* _dryRunChk = nullptr;
00080
00081 // Prompt & Plan tab
00082 QPlainTextEdit* _modelDescEdit = nullptr;
00083 QPlainTextEdit* _experimentDescEdit = nullptr;
00084 QPlainTextEdit* _userPromptEdit = nullptr;
00085 QPushButton* _analyzeBtn = nullptr;
00086 QPushButton* _planBtn = nullptr;
00087 QTextBrowser* _planOutput = nullptr;
00088
00089 // Execution tab
00090 QPushButton* _buildModelBtn = nullptr;
00091 QPushButton* _configureSimBtn = nullptr;
00092 QPushButton* _runSimBtn = nullptr;
00093 QPushButton* _collectResultsBtn = nullptr;
00094 QPushButton* _fullPipelineBtn = nullptr;
00095 QTextBrowser* _executionLog = nullptr;
00096 QLabel* _statusLabel = nullptr;
00097
00098 // Audit log tab
00099 QTableWidgetItem* _auditTable = nullptr;
00100 QPushButton* _refreshAuditBtn = nullptr;
00101 QPushButton* _exportAuditBtn = nullptr;
00102 QLabel* _auditLogPathLabel = nullptr;
00103
00104 QTabWidget* _tabs = nullptr;
00105
00106 // Actions
00107 QAction* _applyConfigAction = nullptr;
00108 QAction* _clearLogAction = nullptr;
00109 QAction* _refreshAuditAction = nullptr;
00110 };
00111
00112 #endif // AIASSISTANTWINDOW_H

```

## 16.4 AddUndoCommand.h

```

00001 #ifndef ADDUNDOCOMMAND_H
00002 #define ADDUNDOCOMMAND_H
00003
00004 #include <QUndoCommand>
00005 #include "graphicals/ModelGraphicsScene.h"
00006 #include "graphicals/GraphicalConnection.h"
00007 #include "graphicals/GraphicalModelComponent.h"
00008
00009 struct ComponentItem {
00010 GraphicalModelComponent *graphicalComponent;
00011 QPointF initialPosition;
00012 QList<GraphicalConnection *> inputConnections;
00013 QList<GraphicalConnection *> outputConnections;
00014 };
00015
00016 struct GroupItem {
00017 QGraphicsItemGroup *group;
00018 QList<ComponentItem> myComponentItems;
00019 QPointF initialPosition;
00020 };
00021
00022 struct DrawingItem {
00023 QGraphicsItem *myDrawingItem;
00024 QPointF initialPosition;
00025 };
00026

```

```

00027
00028 class AddUndoCommand: public QUndoCommand {
00029 public:
00030 AddUndoCommand(QGraphicsItem *item, ModelGraphicsScene *scene, QUndoCommand *parent = nullptr);
00031 ~AddUndoCommand();
00032
00033 void undo() override;
00034 void redo() override;
00035
00036 private:
00037 ComponentItem _myComponentItem;
00038 GraphicalConnection *_myConnectionItem;
00039 QGraphicsItem *_myDrawingItem;
00040 ModelGraphicsScene *_myGraphicsScene;
00041 bool _firstExecution;
00042 };
00043
00044 #endif // ADDUNDOCOMMAND_H

```

## 16.5 DeleteUndoCommand.h

```

00001 #ifndef DELETEUNDOCOMMAND_H
00002 #define DELETEUNDOCOMMAND_H
00003
00004 #include <QUndoCommand>
00005 #include <string>
00006 #include "AddUndoCommand.h"
00007 #include "graphicals/ModelGraphicsScene.h"
00008 #include "graphicals/GraphicalConnection.h"
00009 #include "graphicals/GraphicalModelDataDefinition.h"
00010
00011 struct DataDefinitionItem {
00012 GraphicalModelDataDefinition *graphicalDataDefinition;
00013 QPointF initialPosition;
00014 bool initiallySelected;
00015 };
00016
00017 struct DataDefinitionRelationItem {
00018 ModelDataDefinition *owner;
00019 ModelDataDefinition *dataDefinition;
00020 std::string key;
00021 bool internal;
00022 };
00023
00024 class DeleteUndoCommand : public QUndoCommand
00025 {
00026 public:
00027 explicit DeleteUndoCommand(QList<QGraphicsItem *> items, ModelGraphicsScene *scene, QUndoCommand
00028 *parent = nullptr);
00029 ~DeleteUndoCommand();
00030
00031 void undo() override;
00032 void redo() override;
00033
00034 private:
00035 void captureDataDefinitionsRemovedWithSelectedItems();
00036 void captureDataDefinitionRelations();
00037 void removeDataDefinitionRelations();
00038 void restoreDataDefinitionRelations();
00039 void removeDataDefinitionsFromModel();
00040 void restoreDataDefinitionsToModel();
00041 void detachDataDefinitionItems();
00042 void restoreDataDefinitionItems();
00043
00044 QList<ComponentItem> *_myComponentItems;
00045 QList<GraphicalConnection *> *_myConnectionItems;
00046 QList<DrawingItem> *_myDrawingItems;
00047 QList<GroupItem> *_myGroupItems;
00048 QList<DataDefinitionItem> *_myDataDefinitionItems;
00049 QList<DataDefinitionRelationItem> *_myDataDefinitionRelationItems;
00050 QList<ModelDataDefinition *> *_myDataDefinitions;
00051 ModelGraphicsScene *_myGraphicsScene;
00052 };
00053 #endif // DELETEUNDOCOMMAND_H

```

## 16.6 GroupUndoCommand.h

```

00001 #ifndef GROUPUNDOCOMMAND_H
00002 #define GROUPUNDOCOMMAND_H
00003
00004 #include <QUndoCommand>

```



```

00005 #include "graphicals/ModelGraphicsScene.h"
00006 #include "graphicals/GraphicalModelComponent.h"
00007
00008 class GroupUndoCommand: public QUndoCommand {
00009 public:
00010 GroupUndoCommand(QList<GraphicalModelComponent *> graphicalComponents, ModelGraphicsScene *scene,
00011 QUndoCommand *parent = nullptr);
00012 ~GroupUndoCommand();
00013 void undo() override;
00014 void redo() override;
00015
00016 private:
00017 QList<GraphicalModelComponent *> _myGraphicalComponents;
00018 QGraphicsItemGroup *_group;
00019 ModelGraphicsScene *_myGraphicsScene;
00020 QPointF _initialPosition;
00021 };
00022
00023 #endif // GROUPUNDOCOMMAND_H

```

## 16.7 MoveUndoCommand.h

```

00001 #ifndef MOVEUNDOCOMMAND_H
00002 #define MOVEUNDOCOMMAND_H
00003
00004 #include <QUndoCommand>
00005 #include "graphicals/ModelGraphicsScene.h"
00006
00007 class MoveUndoCommand : public QUndoCommand
00008 {
00009 public:
00010 MoveUndoCommand(QList<QGraphicsItem *> item, ModelGraphicsScene *scene, QList<QPointF> &oldPos,
00011 QList<QPointF> &newPos, QUndoCommand *parent = nullptr);
00012 ~MoveUndoCommand();
00013 void undo() override;
00014 void redo() override;
00015
00016 //bool mergeWith(const QUndoCommand *command) override;
00017
00018 private:
00019 QList<QGraphicsItem*> _myGraphicalItem;
00020 ModelGraphicsScene *_myGraphicsScene;
00021 QList<QPointF> _myOldPos;
00022 QList<QPointF> _myNewPos;
00023 };
00024
00025 #endif // MOVEUNDOCOMMAND_H

```

## 16.8 PasteUndoCommand.h

```

00001 #ifndef PASTEUNDOCOMMAND_H
00002 #define PASTEUNDOCOMMAND_H
00003
00004 #include <QUndoCommand>
00005 #include "AddUndoCommand.h"
00006 #include "graphicals/ModelGraphicsScene.h"
00007 #include "graphicals/GraphicalConnection.h"
00008
00009
00010 class PasteUndoCommand : public QUndoCommand
00011 {
00012 public:
00013 explicit PasteUndoCommand(QList<GraphicalModelComponent *> *graphicalComponents,
00014 QList<GraphicalConnection *> *connections, QList<QGraphicsItemGroup *> *groups, QList<QGraphicsItem *>
00015 *drawing, ModelGraphicsScene *scene, QUndoCommand *parent = nullptr);
00016 ~PasteUndoCommand();
00017 void undo() override;
00018 void redo() override;
00019 private:
00020 QList<ComponentItem> *_myComponentItems;
00021 QList<GraphicalConnection *> *_myConnectionItems;
00022 QList<DrawingItem> *_myDrawingItems;
00023 QList<GroupItem> *_myGroupItems;
00024 ModelGraphicsScene *_myGraphicsScene;
00025 };
00026
00027 #endif // PASTEUNDOCOMMAND_H

```

## 16.9 UngroupUndoCommand.h

```

00001 #ifndef UNGROUPUNDOCOMMAND_H
00002 #define UNGROUPUNDOCOMMAND_H
00003
00004
00005 #include <QUndoCommand>
00006 #include "graphics/ModelGraphicsScene.h"
00007
00008 class UngroupUndoCommand: public QUndoCommand {
00009 public:
00010 UngroupUndoCommand(QGraphicsItemGroup* group, ModelGraphicsScene *scene, QUndoCommand *parent =
00011 nullptr);
00012 ~UngroupUndoCommand();
00013 void undo() override;
00014 void redo() override;
00015 private:
00016 QGraphicsItemGroup *_group;
00017 QList<GraphicalModelComponent *> *_myComponentsGroup;
00018 ModelGraphicsScene *_myGraphicsScene;
00019 QPointF _initialPosition;
00020 };
00021
00022
00023 #endif // UNGROUPUNDOCOMMAND_H

```

## 16.10 AnimationCounter.h

```

00001 #ifndef ANIMATIONCOUNTER_H
00002 #define ANIMATIONCOUNTER_H
00003
00004 #include <QGraphicsRectItem>
00005 #include <QPainter>
00006 #include <QGraphicsSceneMouseEvent>
00007
00008 #include "../kernel/simulator/essentialPlugins/Counter.h"
00009 class AnimationCounter : public QGraphicsRectItem {
00010 public:
00011 // Construtor
00012 AnimationCounter();
00013
00014 // Sobrecarga do método paint
00015 void paint(QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget = nullptr)
00016 override;
00017
00018 // Getters
00019 double getValue();
00020 Counter *getCounter();
00021
00022 // Setters
00023 void setValue(double value);
00024 void setCounter(Counter *myCounter);
00025 void setIdCounter(int id);
00026 void setNameCounter(std::string name);
00027
00028 // Outros
00029 void startDrawing(QGraphicsSceneMouseEvent *event); // Inicia o desenho da tela
00030 void continueDrawing(QGraphicsSceneMouseEvent *event); // Continua o desenho na tela
00031 void stopDrawing(QGraphicsSceneMouseEvent *event); // Finaliza o desenho na tela
00032 void adjustSizeAndPosition(QGraphicsSceneMouseEvent *event); // Ajusta posição e dimensão do
00033 retângulo no final
00034 bool isDrawingInialized(); // Diz se o contador começou a ser desenhada
00035 bool isDrawingFinalized(); // Diz se o contador terminou de ser desenhada
00036 void setWhenLoaded(QList<Counter *> *counters); // seta o contador quando o modelo é carregado
00037 private:
00038 double _value = 0.0;
00039 QPointF _startPoint = QPointF(0, 0);
00040 bool _isResizing = false;
00041 bool _isDrawingInialized = false;
00042 bool _isDrawingFinalized = false;
00043 Counter *_counter = nullptr;
00044 int _idCounter = -1;
00045 std::string _counterName = "None";
00046 };
00047 #endif // ANIMATIONCOUNTER_H

```

## 16.11 AnimationPlaceholder.h

```

00001 #ifndef ANIMATIONPLACEHOLDER_H

```

```

00002 #define ANIMATIONPLACEHOLDER_H
00003
00004 #include <QGraphicsRectItem>
00005 #include <QGraphicsSceneMouseEvent>
00006 #include <QPainter>
00007 #include <QVector>
00008 #include <QString>
00009 #include <limits>
00010
00011 class Model;
00012
00013 class AnimationPlaceholder : public QGraphicsRectItem {
00014 public:
00015 explicit AnimationPlaceholder(const QString& animationType);
00016
00017 void paint(QPainter* painter, const QStyleOptionGraphicsItem* option, QWidget* widget = nullptr)
00018 override;
00019
00019 QString getAnimationType() const;
00020 QString getTargetName() const;
00021 void setTargetName(const QString& targetName);
00022
00023 void startDrawing(QGraphicsSceneMouseEvent* event);
00024 void continueDrawing(QGraphicsSceneMouseEvent* event);
00025 void stopDrawing(QGraphicsSceneMouseEvent* event);
00026 void adjustSizeAndPosition(QGraphicsSceneMouseEvent* event);
00027 bool isDrawingInialized() const;
00028 bool isDrawingFinalized() const;
00029
00030 private:
00031 QString _animationType;
00032 QString _targetName;
00033 QPointF _startPoint = QPointF(0, 0);
00034 bool _isResizing = false;
00035 bool _isDrawingInialized = false;
00036 bool _isDrawingFinalized = false;
00037 };
00038
00039 class AnimationAttribute : public AnimationPlaceholder {
00040 public:
00041 AnimationAttribute();
00042 };
00043
00044 class AnimationEntity : public AnimationPlaceholder {
00045 public:
00046 AnimationEntity();
00047 };
00048
00049 class AnimationEvent : public AnimationPlaceholder {
00050 public:
00051 AnimationEvent();
00052 };
00053
00054 class AnimationExpression : public AnimationPlaceholder {
00055 public:
00056 AnimationExpression();
00057 };
00058
00059 class AnimationPlot : public AnimationPlaceholder {
00060 public:
00061 enum class PlotType {
00062 Line = 0,
00063 Bar = 1
00064 };
00065
00066 struct DataSeries {
00067 QString label;
00068 QString expression;
00069 QColor color = QColor(31, 119, 180);
00070 qreal lineWidth = 2.0;
00071 QVector<double> values;
00072 };
00073
00074 AnimationPlot();
00075
00076 QString getTitle() const;
00077 void setTitle(const QString& title);
00078 QString getYAxisTitle() const;
00079 void setYAxisTitle(const QString& title);
00080 QString getXAxisTitle() const;
00081 void setXAxisTitle(const QString& title);
00082 PlotType getPlotType() const;
00083 void setPlotType(PlotType type);
00084 QString getDatasetsText() const;
00085 void setDatasetsText(const QString& datasetsText);
00086 bool getShowGridLines() const;
00087 void setShowGridLines(bool show);

```

```

00088 bool getShowTicks() const;
00089 void setShowTicks(bool show);
00090 double getXAxisMin() const;
00091 void setXAxisMin(double value);
00092 double getXAxisMax() const;
00093 void setXAxisMax(double value);
00094 double getYAxisMin() const;
00095 void setYAxisMin(double value);
00096 double getYAxisMax() const;
00097 void setYAxisMax(double value);
00098 void clearValues();
00099 void appendSample(Model* model);
00100
00101 void paint(QPainter* painter, const QStyleOptionGraphicsItem* option, QWidget* widget = nullptr)
00102 override;
00103 private:
00104 static QVector<DataSeries> parseDatasetsText(const QString& datasetsText);
00105 static QString datasetsTextFromSeries(const QVector<DataSeries>& series);
00106
00107 QString _title = "Plot";
00108 QString _xAxisTitle = "Time";
00109 QString _yAxisTitle = "Value";
00110 PlotType _plotType = PlotType::Line;
00111 bool _showGridLines = true;
00112 bool _showTicks = true;
00113 double _xAxisMin = std::numeric_limits<double>::quiet_NaN();
00114 double _xAxisMax = std::numeric_limits<double>::quiet_NaN();
00115 double _yAxisMin = std::numeric_limits<double>::quiet_NaN();
00116 double _yAxisMax = std::numeric_limits<double>::quiet_NaN();
00117 QVector<DataSeries> _series;
00118 };
00119
00120 class AnimationQueueDisplay : public AnimationPlaceholder {
00121 public:
00122 AnimationQueueDisplay();
00123 };
00124
00125 class AnimationResource : public AnimationPlaceholder {
00126 public:
00127 AnimationResource();
00128 };
00129
00130 class AnimationStation : public AnimationPlaceholder {
00131 public:
00132 AnimationStation();
00133 };
00134
00135 class AnimationStatistics : public AnimationPlaceholder {
00136 public:
00137 AnimationStatistics();
00138 };
00139
00140 #endif // ANIMATIONPLACEHOLDER_H

```

## 16.12 AnimationQueue.h

```

00001 #ifndef ANIMATIONQUEUE_H
00002 #define ANIMATIONQUEUE_H
00003
00004 #include "graphicals/GraphicalModelComponent.h"
00005
00006 class ModelGraphicsScene;
00007
00008 class AnimationQueue
00009 {
00010 public:
00011 // Construtor
00012 AnimationQueue(ModelGraphicsScene* myScene, ModelComponent* component);
00013 // Destrutor
00014 ~AnimationQueue();
00015
00016 // Getters
00017 GraphicalModelComponent* getGraphicalComponent() const;
00018
00019 // quando uma entidade esta movendo em relacao a um componente que tem fila, adiciona a animação
00020 void addAnimationQueue(bool visivible);
00021 // quando uma entidade esta movendo de um componente que tem fila, remove a animação
00022 void removeAnimationQueue();
00023
00024 // faz a animação de acordo com o tamanho da fila, mas não ficou muito condizente com a
 // visualização da animação
00025 // verifica se precisa adicionar animação e adiciona se for o caso (observando valores da fila)
00026 void verifyAddAnimationQueue(bool visivible);
00027 // verifica se precisa remover animação e remove se for o caso (observando valores da fila)

```

```

00028 void verifyRemoveAnimationQueue();
00029
00030 // retorna a posição em que a animação da fila será desenhada
00031 QPointF calculatePositionImageQueue(unsigned int indexQueue, unsigned int sizeQueue, unsigned int
width, unsigned int height);
00032
00033 private:
00034 ModelGraphicsScene* _myScene;
00035 GraphicalModelComponent* _graphicalComponent;
00036 };
00037
00038 #endif // ANIMATIONQUEUE_H

```

## 16.13 AnimationTimer.h

```

00001 #ifndef ANIMATIONTIMER_H
00002 #define ANIMATIONTIMER_H
00003
00004 #include <QGraphicsRectItem>
00005 #include <QPainter>
00006 #include <QGraphicsSceneMouseEvent>
00007 #include "kernel/util/Util.h"
00008
00009 class ModelGraphicsScene;
00010
00011 class AnimationTimer : public QGraphicsRectItem {
00012 public:
00013 // Construtor
00014 AnimationTimer(ModelGraphicsScene* myScene);
00015
00016 // Sobrecarga do método paint
00017 void paint(QPainter* painter, const QStyleOptionGraphicsItem* option, QWidget* widget = nullptr)
override;
00018
00019 // Getters
00020 double getTime();
00021 unsigned int getInitialHours();
00022 unsigned int getInitialMinutes();
00023 unsigned int getInitialSeconds();
00024 unsigned int getHours();
00025 unsigned int getMinutes();
00026 unsigned int getSeconds();
00027 Util::TimeFormat getTimeFormat();
00028
00029 // Setters
00030 void setTime(double value);
00031 static void setConversionFactorToSeconds(double factor);
00032 void setInitialHours(unsigned int initialHours);
00033 void setInitialMinutes(unsigned int initialMinutes);
00034 void setInitialSeconds(unsigned int initialSeconds);
00035 void setHours(unsigned int hours);
00036 void setMinutes(unsigned int minutes);
00037 void setSeconds(unsigned int seconds);
00038 void setTimeFormat(Util::TimeFormat timeFormat);
00039
00040 // Outros
00041 void startDrawing(QGraphicsSceneMouseEvent* event); // Inicia o desenho da tela
00042 void continueDrawing(QGraphicsSceneMouseEvent* event); // Continua o desenho na tela
00043 void stopDrawing(QGraphicsSceneMouseEvent* event); // Finaliza o desenho na tela
00044 void adjustSizeAndPosition(QGraphicsSceneMouseEvent* event); // Ajusta posição e dimensão do
retângulo no final
00045 void convertSeconds(); // converte o tempo recebido em segundos para horas, dias e segundos
00046 bool isDrawingInialized(); // Diz se o relógio começou a ser desenhado
00047 bool isDrawingFinalized(); // Diz se o relógio terminou de ser desenhado
00048
00049 private:
00050 ModelGraphicsScene* _myScene;
00051 double _time = 0.0;
00052 static double _conversionFactorToSeconds;
00053 QPointF _startPoint = QPointF(0, 0);
00054 unsigned int _initialHours = 0;
00055 unsigned int _initialMinutes = 0;
00056 unsigned int _initialSeconds = 0;
00057 unsigned int _hours = 0;
00058 unsigned int _minutes = 0;
00059 unsigned int _seconds = 0;
00060 unsigned int _microseconds = 0;
00061 Util::TimeFormat _timeFormat = Util::TimeFormat(24);
00062 bool _showMicroseconds = true;
00063 bool _isResizing = false;
00064 bool _isDrawingInialized = false;
00065 bool _isDrawingFinalized = false;
00066 };
00067
00068 #endif // ANIMATIONTIMER_H

```

## 16.14 AnimationTransition.h

```

00001 #ifndef ANIMATIONTRANSITION_H
00002 #define ANIMATIONTRANSITION_H
00003
00004 #include <QList>
00005 #include <QPainterPath>
00006 #include <QPointF>
00007 #include <QTimer>
00008 #include <QVariantAnimation>
00009 #include <QElapsedTimer>
00010
00011 #include "graphicals/GraphicalImageAnimation.h"
00012 #include "graphicals/GraphicalModelComponent.h"
00013 #include "graphicals/GraphicalConnection.h"
00014
00015 // Em segundos
00016 #define TEMPO_EXECUCAO_ANIMACAO 2.5
00017
00018 class ModelGraphicsScene;
00019
00020 class AnimationTransition : public QVariantAnimation
00021 {
00022 public:
00023 // Construtor
00024 AnimationTransition(ModelGraphicsScene* myScene, ModelComponent* graphicalStartComponent,
00025 ModelComponent* graphicalEndComponent, bool viewSimulation);
00026 // Destrutor
00027 ~AnimationTransition();
00028
00029 // Getters
00030 GraphicalModelComponent* getGraphicalStartComponent() const;
00031 GraphicalModelComponent* getGraphicalEndComponent() const;
00032 GraphicalConnection* getGraphicalConnection() const;
00033 static double getTimeExecution();
00034 QList<QPointF> getPointsForAnimation() const;
00035 GraphicalImageAnimation* getImageAnimation() const;
00036 unsigned int getPortNumber() const;
00037 // Expose whether transition has all runtime prerequisites to safely animate.
00038 bool isReadyToRun() const;
00039
00040 // Setters
00041 void setImageAnimation(GraphicalImageAnimation* imageAnimation);
00042 static void setTimeExecution(double timeExecution);
00043 static void setPause(bool pause);
00044 static void setRunning(bool running);
00045
00046 // Inicia a animação
00047 void startAnimation();
00048 // Finaliza a animação
00049 void stopAnimation();
00050 // Reinicia a animação
00051 void restartAnimation();
00052 // Configuração informações para a animação
00053 void configureAnimation();
00054 // Atualiza a duração da animação se ela for alterada em tempo de execução
00055 void updateDurationIfNeeeded();
00056 // Conecta o sinal valueChanged da animação ao slot onAnimationValueChanged
00057 void connectValueChangedSignal();
00058 // Conecta o sinal finished da animação ao slot onAnimationFinished
00059 void connectFinishedSignal();
00060
00061 public slots:
00062 // Slot para ser conectado ao sinal valueChanged
00063 void onAnimationValueChanged(const QVariant& value);
00064 // Slot que será conectado a um sinal
00065 void onAnimationFinished();
00066
00067 private:
00068 ModelGraphicsScene* _myScene;
00069 GraphicalModelComponent* _graphicalStartComponent;
00070 GraphicalModelComponent* _graphicalEndComponent;
00071 GraphicalConnection* _graphicalConnection;
00072 static double _timeExecution;
00073 static double _oldTimeExecution;
00074 static bool _pause;
00075 static bool _running;
00076 QList<QPointF> _pointsForAnimation;
00077 QPainterPath _pathForAnimation;
00078 GraphicalImageAnimation* _imageAnimation;
00079 unsigned int _portNumber;
00080 qreal _currentProgress;
00081 bool _viewSimulation;
00082 // Track terminal lifecycle paths to keep cleanup idempotent.
00083 bool _isStopping;
00084 // Track whether finished callback was already handled.
00085 bool _isFinishedHandled;

```

```

00085 bool _usesPathAnimation;
00086 };
00087
00088 #endif // ANIMATIONTRANSITION_H

```

## 16.15 AnimationVariable.h

```

00001 #ifndef ANIMATIONVARIABLE_H
00002 #define ANIMATIONVARIABLE_H
00003
00004 #include <QString>
00005 #include <QList>
00006 #include <QVector>
00007 #include <QGraphicsRectItem>
00008 #include <QGraphicsSceneMouseEvent>
00009 #include <QPainter>
00010 #include <string>
00011
00012 #include "../kernel/simulator/essentialPlugins/Attribute.h"
00013 #include "../kernel/simulator/essentialPlugins/Entity.h"
00014 #include "../kernel/simulator/model/Model.h"
00015 #include "../kernel/simulator/model/ModelDataDefinition.h"
00016 #include "../kernel/simulator/model/ModelDataManager.h"
00017 #include "../plugins/data/Logic/Variable.h"
00018
00019 class AnimationVariable : public QGraphicsRectItem {
00020 public:
00021 AnimationVariable();
00022
00023 void paint(QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget = nullptr)
00024 override;
00025
00026 double getValue();
00027 ModelDataDefinition *getDataDefinition() const;
00028 Variable *getVariable();
00029 Attribute *getAttribute();
00030 QString getTargetName() const;
00031 int getTargetId() const;
00032 unsigned int getDimensionCount() const;
00033 unsigned int getDimensionSize(unsigned int dimensionIndex) const;
00034 unsigned int getSliceIndex(unsigned int dimensionIndex) const;
00035
00036 void setValue(double value);
00037 void setDataDefinition(ModelDataDefinition *definition);
00038 void setVariable(Variable *variable);
00039 void setAttribute(Attribute *attribute);
00040 void setModel(Model *model);
00041 void setTargetName(std::string name);
00042 void setIdVariable(int id);
00043 void setSliceIndex(unsigned int dimensionIndex, unsigned int value);
00044 void setWhenLoaded(QList<ModelDataDefinition *> *definitions);
00045 void refreshValue(Entity *entity = nullptr);
00046
00047 void startDrawing(QGraphicsSceneMouseEvent *event);
00048 void continueDrawing(QGraphicsSceneMouseEvent *event);
00049 void stopDrawing(QGraphicsSceneMouseEvent *event);
00050 void adjustSizeAndPosition(QGraphicsSceneMouseEvent *event);
00051 bool isDrawingInicalized();
00052 bool isDrawingFinalized();
00053 private:
00054 double _value = 0.0;
00055 Model *_model = nullptr;
00056 ModelDataDefinition *_dataDefinition = nullptr;
00057 QString _targetName = "None";
00058 int _targetId = -1;
00059 QVector<unsigned int> _sliceIndexes;
00060 QVector<double> _cachedValues;
00061 QVector<unsigned int> _cachedShape;
00062 QString _cachedTypeName;
00063 QString _cachedTitle;
00064 QPointF _startPoint = QPointF(0, 0);
00065 bool _isResizing = false;
00066 bool _isDrawingInicalized = false;
00067 bool _isDrawingFinalized = false;
00068 };
00069
00070 #endif // ANIMATIONVARIABLE_H

```

## 16.16 BaseMVController.h

```

00001 #pragma once

```

```

00002
00003 #include <QMainWindow>
00004 #include <QObject>
00005 #include <QWidget>
00006 #include "kernel/simulator/Simulator.h"
00007 #include "ui_mainwindow.h"
00008
00016 class BaseMVController
00017 {
00018 public:
00019 BaseMVController();
00020 protected: // View (UI) and Model (Simulator) main elements to join to the Controller
00022 Ui::MainWindow *ui;
00024 Simulator* simulator;
00025 };

```

## 16.17 CellularAutomataDemoBuilder.h

```

00001 #pragma once
00002
00003 /*
00004
00005 • Para que a GUI use um novo autômato celular com nova regra local,
00006 você mexe em 3 camadas:
00007
00008 1) Implementar a regra no plugin
00009
00010 - Crie a classe da regra em source/plugins/components/ModalModel/
00011 CellularAutomata/.
00012 - Normalmente isso significa algo como:
00013 - LocalRule_MyRule.h
00014 - opcionalmente LocalRule_MyRule.cpp
00015 - A regra precisa herdar de LocalRule e sobrescrever
00016 applyRule(Cell* cell).
00017 - Se a regra precisar de vizinhança específica, ela vai ler
00018 cell->getNeighbors().
00019
00020 2) Registrar essa regra no builder da GUI
00021
00022 - O ponto de montagem do demo fica em:
00023 - source/applications/gui/genesys/cellular_automata/
00024 CellularAutomataDemoBuilder.h
00025 - source/applications/gui/genesys/cellular_automata/
00026 CellularAutomataDemoBuilder.cpp
00027 - É aqui que a GUI escolhe qual regra concreta instanciar.
00028 - Você precisa:
00029 - incluir o header da nova regra;
00030 - adicionar um valor novo no enum CellularAutomataRulePreset;
00031 - estender cellularAutomataRulePresetText(...);
00032 - atualizar makeLocalRule(...) para retornar sua classe.
00033
00034 3) Expor a escolha na janela
00035
00036 - A UI de seleção fica em:
00037 - source/applications/gui/genesys/cellular_automata/
00038 CellularAutomataViewerWindow.cpp
00039 - source/applications/gui/genesys/cellular_automata/
00040 CellularAutomataViewerWindow.h
00041 - Você precisa adicionar o novo preset ao combo de regra em
00042 _syncControlsFromController().
00043 - Se quiser salvar/restaurar essa escolha no JSON, ajuste também:
00044 - source/applications/gui/genesys/cellular_automata/
00045 CellularAutomataViewerController.cpp
00046
00047 Se a nova regra exigir outro tipo de vizinhança ou estado
00048
00049 - O ajuste continua na mesma área do builder:
00050 - CellularAutomataDemoBuilder.h/.cpp
00051 - Para vizinhança, hoje a GUI já suporta:
00052 - Moore
00053 - VonNeumann
00054 - Para tipo de estado, hoje ela expõe:
00055 - Binary
00056 - Enumerated
00057 - Numeric
00058 - Se sua regra precisar de um estado diferente, você precisa
00059 ampliar o preset de estado e o mapeamento de cores/menu.
00060
00061 Fluxo mínimo para fazer funcionar
00062
00063 1. Criar LocalRule_MyRule.
00064 2. Incluir essa classe em CellularAutomataDemoBuilder.h.
00065 3. Adicionar o preset no enum CellularAutomataRulePreset.
00066 4. Implementar a criação da regra em makeLocalRule(...).
00067 5. Adicionar o item no combo da janela em

```



```

00068 CellularAutomataViewerWindow.cpp.
00069 6. Atualizar serialização em CellularAutomataViewerController.cpp
00070 se quiser persistência.
00071
00072 Arquivos que você provavelmente vai mexer
00073
00074 - source/plugins/components/ModalModel/CellularAutomata/
00075 LocalRule_MyRule.h
00076 - source/plugins/components/ModalModel/CellularAutomata/
00077 LocalRule_MyRule.cpp se necessário
00078 - source/applications/gui/genesys/cellular_automata/
00079 CellularAutomataDemoBuilder.h
00080 - source/applications/gui/genesys/cellular_automata/
00081 CellularAutomataDemoBuilder.cpp
00082 - source/applications/gui/genesys/cellular_automata/
00083 CellularAutomataViewerWindow.cpp
00084 - source/applications/gui/genesys/cellular_automata/
00085 CellularAutomataViewerController.cpp
00086
00087 */
00088
00089
00090
00091
00092 #include <memory>
00093 #include <vector>
00094
00095 #include <QSize>
00096
00097 #include "plugins/components/ModalModel/CellularAutomata/Boundary_Closed.h"
00098 #include "plugins/components/ModalModel/CellularAutomata/Boundary_Fixed.h"
00099 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00100 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomata_Classic.h"
00101 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00102 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00103 #include "plugins/components/ModalModel/CellularAutomata/LocalRule_GameOfLife.h"
00104 #include "plugins/components/ModalModel/CellularAutomata/LocalRule_FlorestalFire.h"
00105 #include "plugins/components/ModalModel/CellularAutomata/LocalRule_Growty.h"
00106 #include "plugins/components/ModalModel/CellularAutomata/LocalRule_HppLatticeGas.h"
00107 #include "plugins/components/ModalModel/CellularAutomata/Neighborhood_Moore.h"
00108 #include "plugins/components/ModalModel/CellularAutomata/Neighborhood_VonNeumann.h"
00109 #include "plugins/components/ModalModel/CellularAutomata/StateSet_Enumerable.h"
00110
00111 enum class CellularAutomataRulePreset {
00112 GameOfLife = 0,
00113 Growty = 1,
00114 Identity = 2,
00115 ForestFire = 3,
00116 HppLatticeGas = 4,
00117 SandRock = 5
00118 };
00119
00120 enum class CellularAutomataNeighborhoodPreset {
00121 Moore = 0,
00122 VonNeumann = 1
00123 };
00124
00125 enum class CellularAutomataBoundaryPreset {
00126 Closed = 0,
00127 Fixed = 1
00128 };
00129
00130 enum class CellularAutomataStatePreset {
00131 Binary = 0,
00132 Enumerated = 1,
00133 Numeric = 2,
00134 HppLatticeGas = 3
00135 };
00136
00137 struct CellularAutomataDemoSettings {
00138 QSize latticeSize = QSize{100, 100};
00139 CellularAutomataRulePreset rulePreset = CellularAutomataRulePreset::SandRock;
00140 CellularAutomataNeighborhoodPreset neighborhoodPreset = CellularAutomataNeighborhoodPreset::Moore;
00141 CellularAutomataBoundaryPreset boundaryPreset = CellularAutomataBoundaryPreset::Fixed;
00142 CellularAutomataStatePreset statePreset = CellularAutomataStatePreset::Enumerated;
00143 };
00144
00145 QString cellularAutomataRulePresetText(CellularAutomataRulePreset preset);
00146 QString cellularAutomataNeighborhoodPresetText(CellularAutomataNeighborhoodPreset preset);
00147 QString cellularAutomataBoundaryPresetText(CellularAutomataBoundaryPreset preset);
00148 QString cellularAutomataStatePresetText(CellularAutomataStatePreset preset);
00149
00150 class LocalRule_CopyCurrentState final : public LocalRule {
00151 public:
00152 explicit LocalRule_CopyCurrentState(CellularAutomataBase* parentCellularAutomata)
00153 : LocalRule(parentCellularAutomata) {
00154 }

```

```

00155
00156 void applyRule(Cell* cell) override {
00157 if (cell != nullptr) {
00158 cell->setNextState(cell->getCurrentState());
00159 }
00160 }
00161 };
00162
00163 struct CellularAutomataDemoModel {
00164 explicit CellularAutomataDemoModel(CellularAutomataDemoSettings settings = {});
00165
00166 CellularAutomataDemoSettings settings;
00167 std::vector<State> ownedStates;
00168 std::vector<State*> statePointers;
00169 std::unique_ptr<CellularAutomata_Classic> automaton;
00170 std::unique_ptr<Lattice> lattice;
00171 std::unique_ptr<StateSet> stateSet;
00172 std::unique_ptr<BoundaryCondition> boundary;
00173 std::unique_ptr<Neighborhood> neighborhood;
00174 std::unique_ptr<LocalRule> localRule;
00175 std::unique_ptr<LocalRule_CopyCurrentState> identityLocalRule;
00176
00177 int columns() const;
00178 int rows() const;
00179 bool isValidPosition(int x, int y) const;
00180 Cell* cellAt(int x, int y) const;
00181 long stateAt(int x, int y) const;
00182 QString stateTextAt(int x, int y) const;
00183 bool setStateAt(int x, int y, long value);
00184 void fill(long value);
00185 void seedDefaultPattern();
00186 std::vector<long> availablePaintStates() const;
00187 };
00188
00189 class CellularAutomataDemoBuilder {
00190 public:
00191 static std::unique_ptr<CellularAutomataDemoModel> buildDemo(CellularAutomataDemoSettings settings
00192 = {});
00193 };

```

## 16.18 CellularAutomataLatticeWidget.h

```

00001 #pragma once
00002
00003 #include <QPoint>
00004 #include <QRectF>
00005 #include <QWidget>
00006
00007 class CellularAutomataViewerController;
00008
00009 class CellularAutomataLatticeWidget : public QWidget {
00010 public:
00011 explicit CellularAutomataLatticeWidget(QWidget* parent = nullptr);
00012
00013 void setController(CellularAutomataViewerController* controller);
00014
00015 protected:
00016 void paintEvent(QPaintEvent* event) override;
00017 void mousePressEvent(QMouseEvent* event) override;
00018 void mouseMoveEvent(QMouseEvent* event) override;
00019 void mouseReleaseEvent(QMouseEvent* event) override;
00020 void contextMenuEvent(QContextMenuEvent* event) override;
00021 QSize minimumSizeHint() const override;
00022
00023 private:
00024 bool _ensureSelectedPaintState();
00025 QPoint _cellAtPosition(const QPoint& widgetPosition) const;
00026 QRectF _gridRect() const;
00027 void _paintCellState(const QPoint& cellPosition);
00028 void _showStateContextMenu(const QPoint& globalPosition, const QPoint& cellPosition);
00029
00030 private:
00031 CellularAutomataViewerController* _controller = nullptr;
00032 bool _dragging = false;
00033 QPoint _lastPaintedCell = QPoint{-1, -1};
00034 };

```

## 16.19 CellularAutomataStateColorMap.h

```

00001 #pragma once
00002
00003 #include <QColor>

```

```

00004 #include <QString>
00005 #include <functional>
00006
00007 class CellularAutomataStateColorMap {
00008 public:
00009 static QColor colorForState(long state) {
00010 if (state <= 0) {
00011 return QColor(248, 248, 248);
00012 }
00013 if (state == 1) {
00014 return QColor(28, 30, 36);
00015 }
00016
00017 const int hue = static_cast<int>((state * 47) % 360);
00018 return QColor::fromHsv(hue, 180, 215);
00019 }
00020
00021 static QColor colorForStateText(const QString& stateText, long fallbackState) {
00022 if (stateText.isEmpty() || stateText == QString::number(fallbackState)) {
00023 return colorForState(fallbackState);
00024 }
00025 const size_t hash = std::hash<std::string>{}(stateText.toStdString());
00026 return QColor::fromHsv(static_cast<int>(hash % 360), 170, 220);
00027 }
00028
00029 static QColor backgroundColor() {
00030 return QColor(238, 239, 242);
00031 }
00032
00033 static QColor gridLineColor() {
00034 return QColor(190, 194, 201);
00035 }
00036
00037 static QColor emptyTextColor() {
00038 return QColor(110, 115, 124);
00039 }
00040 };

```

## 16.20 CellularAutomataViewerController.h

```

00001 #pragma once
00002
00003 #include <memory>
00004 #include <optional>
00005 #include <vector>
00006
00007 #include <QSize>
00008 #include <QString>
00009 #include <QStringList>
00010 #include <QTimer>
00011 #include <QObject>
00012
00013 #include "CellularAutomataDemoBuilder.h"
00014
00015 class CellularAutomataViewerController : public QObject {
00016 Q_OBJECT
00017 public:
00018 explicit CellularAutomataViewerController(QObject* parent = nullptr);
00019
00020 CellularAutomataDemoModel* model() const;
00021 CellularAutomataDemoSettings settings() const;
00022 QSize latticeSize() const;
00023 bool hasDemo() const;
00024 unsigned int simulatedTime() const;
00025 bool isRunning() const;
00026 QString configurationSummary() const;
00027 std::vector<long> availablePaintStates() const;
00028
00029 long cellState(int x, int y) const;
00030 bool setCellState(int x, int y, long value);
00031 void fill(long value);
00032
00033 bool hasSelectedPaintState() const;
00034 std::optional<long> selectedPaintState() const;
00035 QString selectedPaintStateText() const;
00036 void setSelectedPaintState(long state);
00037 void clearSelectedPaintState();
00038
00039 int autoStepIntervalMs() const;
00040 void setAutoStepIntervalMs(int intervalMs);
00041
00042 void rebuildDemo(CellularAutomataDemoSettings settings = {});
00043 void setSettings(CellularAutomataDemoSettings settings);
00044 void setRulePreset(CellularAutomataRulePreset preset);
00045 void setNeighborhoodPreset(CellularAutomataNeighborhoodPreset preset);

```

```

00046 void setBoundaryPreset(CellularAutomataBoundaryPreset preset);
00047 void setStatePreset(CellularAutomataStatePreset preset);
00048 void resetToInitialState();
00049 void step();
00050 void start();
00051 void pause();
00052 void stop();
00053
00054 bool saveConfiguration(const QString& filePath, QString* errorMessage = nullptr) const;
00055 bool loadConfiguration(const QString& filePath, QString* errorMessage = nullptr);
00056
00057 signals:
00058 void modelChanged();
00059 void settingsChanged();
00060 void timeChanged(unsigned int time);
00061 void runningChanged(bool running);
00062 void paintStateChanged();
00063 void statusMessage(const QString& message);
00064
00065 private slots:
00066 void _onTimerTimeout();
00067
00068 private:
00069 void _resetRuntimeState();
00070
00071 private:
00072 std::unique_ptr<CellularAutomataDemoModel> _demo;
00073 CellularAutomataDemoSettings _settings;
00074 QTimer _timer;
00075 unsigned int _simulatedTime = 0;
00076 bool _running = false;
00077 int _autoStepIntervalMs = 0;
00078 std::optional<long> _selectedPaintState;
00079 };

```

## 16.21 CellularAutomataViewerWindow.h

```

00001 #pragma once
00002
00003 #include <QMainWindow>
00004 #include <QString>
00005
00006 class QAction;
00007 class QComboBox;
00008 class QLabel;
00009 class CellularAutomataLatticeWidget;
00010 class CellularAutomataViewerController;
00011
00012 class CellularAutomataViewerWindow : public QMainWindow {
00013 public:
00014 explicit CellularAutomataViewerWindow(QWidget* parent = nullptr);
00015
00016 private:
00017 void _updateTimeLabel(unsigned int time);
00018 void _updateRunningState(bool running);
00019 void _updatePaintStateLabel();
00020 void _updateConfigurationLabel();
00021 void _updateStatusMessage(const QString& message);
00022 void _refreshUi();
00023 void _syncControlsFromController();
00024 void _onRulePresetChanged(int index);
00025 void _onNeighborhoodPresetChanged(int index);
00026 void _onBoundaryPresetChanged(int index);
00027 void _onStatePresetChanged(int index);
00028 bool _promptSavePath(QString* path) const;
00029 bool _promptLoadPath(QString* path) const;
00030
00031 private:
00032 CellularAutomataViewerController* _controller = nullptr;
00033 CellularAutomataLatticeWidget* _latticeWidget = nullptr;
00034 QComboBox* _ruleCombo = nullptr;
00035 QComboBox* _neighborhoodCombo = nullptr;
00036 QComboBox* _boundaryCombo = nullptr;
00037 QComboBox* _stateCombo = nullptr;
00038 QLabel* _timeLabel = nullptr;
00039 QLabel* _runningLabel = nullptr;
00040 QLabel* _paintStateLabel = nullptr;
00041 QLabel* _configurationLabel = nullptr;
00042 QLabel* _statusLabel = nullptr;
00043 QAction* _startAction = nullptr;
00044 QAction* _stepAction = nullptr;
00045 QAction* _continueAction = nullptr;
00046 QAction* _pauseAction = nullptr;
00047 QAction* _stopAction = nullptr;
00048 QAction* _restartAction = nullptr;

```

```

00049 QAction* _saveAction = nullptr;
00050 QAction* _loadAction = nullptr;
00051 };

```

## 16.22 CodeEditor.h

```

00001 #ifndef CODEEDITOR_H
00002 #define CODEEDITOR_H
00003
00004 #include <QPlainTextEdit>
00005
00006 class CodeEditor : public QPlainTextEdit {
00007 Q_OBJECT
00008
00009 public:
00010 CodeEditor(QWidget *parent = nullptr);
00011
00012 void lineNumberAreaPaintEvent(QPaintEvent *event);
00013 int lineNumberAreaWidth();
00014
00015 protected:
00016 void resizeEvent(QResizeEvent *event) override;
00017
00018 private slots:
00019 void updateLineNumberAreaWidth(int newBlockCount);
00020 void highlightCurrentLine();
00021 void updateLineNumberArea(const QRect &rect, int dy);
00022
00023 private:
00024 QWidget *lineNumberArea;
00025 };
00026
00027 #endif // CODEEDITOR_H

```

## 16.23 LineNumberArea.h

```

00001 #ifndef LINENUMBERAREA_H
00002 #define LINENUMBERAREA_H
00003
00004 #include <QWidget>
00005 #include "CodeEditor.h"
00006
00007 class LineNumberArea : public QWidget {
00008 public:
00009 LineNumberArea(CodeEditor *editor) : QWidget(editor), codeEditor(editor) {
00010 }
00011
00012 QSize sizeHint() const override {
00013 return QSize(codeEditor->lineNumberAreaWidth(), 0);
00014 }
00015
00016 protected:
00017
00018 void paintEvent(QPaintEvent *event) override {
00019 codeEditor->lineNumberAreaPaintEvent(event);
00020 }
00021
00022 private:
00023 CodeEditor *codeEditor;
00024 };
00025
00026 #endif // LINENUMBERAREA_H

```

## 16.24 DialogUtilityController.h

```

00001 #ifndef DIALOGUTILITYCONTROLLER_H
00002 #define DIALOGUTILITYCONTROLLER_H
00003
00004 #include <functional>
00005
00006 class MainWindow;
00007 class Simulator;
00008 class ModelGraphicsView;
00009 class ModelGraphicsScene;
00010 class QString;
00011
00012 namespace Ui {
00013 class MainWindow;

```

```

00014 }
00015
00016 // Document the dialog and utility controller extracted from remaining MainWindow actions.
00034 class DialogUtilityController {
00035 public:
00037 DialogUtilityController(MainWindow* ownerWidget,
00038 Simulator* simulator,
00039 Ui::MainWindow* ui,
00040 ModelGraphicsView* graphicsView,
00041 std::function<void()> showMessageNotImplemented,
00042 std::function<void(bool)> actualizeDebugBreakpoints,
00043 std::function<bool()> createModelImage,
00044 std::function<void()> actualizeActions,
00045 std::function<void()> actualizeTabPanels,
00046 std::function<void()> reloadPluginCatalog,
00047 std::function<ModelGraphicsScene*>()> currentScene,
00048 double& optimizerPrecision,
00049 unsigned int& optimizerMaxSteps,
00050 bool& parallelizationEnabled,
00051 int& parallelizationThreads,
00052 int& parallelizationBatchSize,
00053 QString& lastDataAnalyzerPath);
00054
00056 void onActionAboutAboutTriggered();
00058 void onActionAboutLicenceTriggered();
00060 void onActionAboutGetInvolvedTriggered();
00062 void onActionAboutReportIssueTriggered();
00064 void onActionEditFindTriggered();
00066 void onActionEditReplaceTriggered();
00068 void onActionToolsParserGrammarCheckerTriggered();
00070 void onActionToolsOptimizatorTriggered();
00072 void onActionToolsAIAssistantTriggered();
00074 void onActionToolsDataAnalyzerTriggered();
00076 void onActionViewConfigureTriggered();
00078 void onActionSimulatorPreferencesTriggered();
00080 void onActionSimulatorsPluginManagerTriggered(bool showProblemPlugins = false);
00082 void onPushButtonBreakpointInsertClicked();
00084 void onPushButtonBreakpointRemoveClicked();
00086 void onPushButtonExportClicked();
00088 void onActionParallelizationTriggered();
00089
00090 private:
00091 MainWindow* _ownerWidget;
00092 Simulator* _simulator;
00093 Ui::MainWindow* _ui;
00094 ModelGraphicsView* _graphicsView;
00095 std::function<void()> _showMessageNotImplemented;
00096 std::function<void(bool)> _actualizeDebugBreakpoints;
00097 std::function<bool()> _createModelImage;
00098 std::function<void()> _actualizeActions;
00099 std::function<void()> _actualizeTabPanels;
00100 std::function<void()> _reloadPluginCatalog;
00101 std::function<ModelGraphicsScene*>()> _currentScene;
00102 double& _optimizerPrecision;
00103 unsigned int& _optimizerMaxSteps;
00104 bool& _parallelizationEnabled;
00105 int& _parallelizationThreads;
00106 int& _parallelizationBatchSize;
00107 QString& _lastDataAnalyzerPath;
00108 };
00109
00110 #endif // DIALOGUTILITYCONTROLLER_H

```

## 16.25 EditCommandController.h

```

00001 #ifndef EDITCOMMANDCONTROLLER_H
00002 #define EDITCOMMANDCONTROLLER_H
00003
00004 #include <QList>
00005 #include <QGraphicsItem>
00006 #include <functional>
00007
00008 class Simulator;
00009 class ModelGraphicsView;
00010 class GraphicalModelComponent;
00011 class GraphicalConnection;
00012 class QGraphicsItemGroup;
00013
00014 // Document edit command orchestration extracted from MainWindow wrappers.
00032 class EditCommandController {
00033 public:
00035 EditCommandController(Simulator* simulator,
00036 ModelGraphicsView* graphicsView,
00037 std::function<void()> actualizeActions,
00038 bool* cut,

```

```

00039 QList<GraphicalModelComponent*>*& gmcCopies,
00040 QList<GraphicalConnection*>*& portsCopies,
00041 QList<QGraphicsItem*>*& drawCopy,
00042 QList<QGraphicsItemGroup*>*& groupCopy);
00043
00045 void onActionEditUndoTriggered() const;
00047 void onActionEditRedoTriggered() const;
00049 void onActionEditCutTriggered() const;
00051 void onActionEditCopyTriggered() const;
00053 void onActionEditPasteTriggered() const;
00055 void onActionEditDeleteTriggered() const;
00057 void onActionEditGroupTriggered() const;
00059 void onActionEditUngroupTriggered() const;
00061 void onActionViewGroupTriggered() const;
00063 void onActionViewUngroupTriggered() const;
00064
00066 void saveItemForCopy(QList<GraphicalModelComponent*>*& gmcList, QList<GraphicalConnection*>*&
connList) const;
00068 void helpCopy() const;
00069
00070 private:
00071 // Resolve current scene from the injected graphics view.
00072 class ModelGraphicsScene* scene() const;
00073
00074 private:
00075 Simulator* _simulator;
00076 ModelGraphicsView* _graphicsView;
00077 std::function<void()> _actualizeActions;
00078 bool* _cut;
00079 QList<GraphicalModelComponent*>*& _gmcCopies;
00080 QList<GraphicalConnection*>*& _portsCopies;
00081 QList<QGraphicsItem*>*& _drawCopy;
00082 QList<QGraphicsItemGroup*>*& _groupCopy;
00083 };
00084
00085 #endif // EDITCOMMANDCONTROLLER_H

```

## 16.26 GraphicalContextMenuController.h

```

00001 #ifndef GRAPHICALCONTEXTMENUCONTROLLER_H
00002 #define GRAPHICALCONTEXTMENUCONTROLLER_H
00003
00004 #include <functional>
00005
00006 class QContextMenuEvent;
00007 class QGraphicsItem;
00008 class QMenu;
00009 class QAction;
00010 class ModelGraphicsView;
00011 class ModelGraphicsScene;
00012 class ModelDataDefinition;
00013
00014 namespace Ui {
00015 class MainWindow;
00016 }
00017
00024 class GraphicalContextMenuController {
00025 public:
00029 GraphicalContextMenuController(ModelGraphicsView* graphicsView,
00030 Ui::MainWindow* ui,
00031 QAction* openSubmodelAction,
00032 std::function<ModelGraphicsScene*>()> currentScene,
00033 std::function<void()> actualizeActions);
00034
00038 void handleGraphicsViewContextMenu(QContextMenuEvent* event);
00039
00040 private:
00041 enum class ContextKind {
00042 Background,
00043 SingleComponent,
00044 SingleDataDefinition,
00045 SingleConnection,
00046 SingleDiagramConnection,
00047 SingleGroup,
00048 MultiSelection,
00049 GenericItem
00050 };
00051
00055 QGraphicsItem* semanticItemFor(QGraphicsItem* item) const;
00056
00060 void synchronizeSelectionForContext(QGraphicsItem* item) const;
00061
00065 ContextKind classifyContext(QGraphicsItem* item) const;
00066
00070 ModelDataDefinition* modelDataDefinitionFor(QGraphicsItem* item) const;

```

```

00071
00075 void populateBackgroundMenu(QMenu* menu) const;
00076
00080 void populateItemMenu(QMenu* menu, ContextKind contextKind, QGraphicsItem* clickedItem) const;
00081
00085 void addEditActions(QMenu* menu) const;
00086
00090 void addShowMenu(QMenu* menu) const;
00091
00095 void addZoomMenu(QMenu* menu) const;
00096
00100 void addTraceLevelMenu(QMenu* menu, QGraphicsItem* clickedItem) const;
00101
00105 void addBioReactionLinkMenu(QMenu* menu) const;
00106
00110 void addDrawMenu(QMenu* menu) const;
00111
00115 void addAnimateMenu(QMenu* menu) const;
00116
00120 void addAlignMenu(QMenu* menu) const;
00121
00122 ModelGraphicsView* _graphicsView;
00123 Ui::MainWindow* _ui;
00124 QAction* _openSubmodelAction;
00125 std::function<ModelGraphicsScene*>() _currentScene;
00126 std::function<void()> _actualizeActions;
00127 };
00128
00129 #endif // GRAPHICALCONTEXTMENUCONTROLLER_H

```

## 16.27 ModelInspectorController.h

```

00001 #ifndef MODELINSPECTORCONTROLLER_H
00002 #define MODELINSPECTORCONTROLLER_H
00003
00004 #include <QTreeWidget>
00005
00006 class Simulator;
00007 class ModelGraphicsView;
00008 class ModelDataDefinition;
00009 class GraphicalModelDataDefinition;
00010
00011 // Document the controller boundary for model-inspection and tree/scene synchronization.
00030 class ModelInspectorController {
00031 public:
00032 struct DataDefinitionSelection {
00033 bool handled = false;
00034 ModelDataDefinition* dataDefinition = nullptr;
00035 GraphicalModelDataDefinition* graphicalDataDefinition = nullptr;
00036 };
00037
00039 ModelInspectorController(Simulator* simulator,
00040 QTreeWidget* componentsTree,
00041 QTreeWidget* dataDefinitionsTree,
00042 ModelGraphicsView* graphicsView);
00043
00045 void actualizeModelComponents(bool force) const;
00047 void actualizeModelDataDefinitions(bool force) const;
00049 void beginDataDefinitionNameEdit(QTreeWidgetItem* item, int column) const;
00051 void applyDataDefinitionNameChange(QTreeWidgetItem* item, int column) const;
00053 void syncSelectedComponentTreeItemToScene() const;
00055 DataDefinitionSelection syncSelectedDataDefinitionTreeItemToScene() const;
00056
00057 private:
00058 // Keep simulator access scoped to model-inspection responsibilities.
00059 Simulator* _simulator;
00060 // Keep direct access to the Components tree used by this controller.
00061 QTreeWidget* _componentsTree;
00062 // Keep direct access to the Data Definitions tree used by this controller.
00063 QTreeWidget* _dataDefinitionsTree;
00064 // Keep direct access to the graphics view for selection synchronization.
00065 ModelGraphicsView* _graphicsView;
00066 };
00067
00068 #endif // MODELINSPECTORCONTROLLER_H

```

## 16.28 ModelLifecycleController.h

```

00001 #ifndef MODELLEIFECYCLECONTROLLER_H
00002 #define MODELLEIFECYCLECONTROLLER_H
00003
00004 #include <functional>

```



```

00005 #include <string>
00006
00007 class QFile;
00008 class QString;
00009 class QWidget;
00010 class Simulator;
00011 class Model;
00012
00013 namespace Ui {
00014 class MainWindow;
00015 }
00016
00017 // Document model lifecycle orchestration delegated by MainWindow compatibility wrappers.
00036 class ModelLifecycleController {
00037 public:
00038 // Group callback dependencies to keep constructor arguments focused and explicit.
00039 struct Callbacks {
00040 std::function<void(const std::string&)> insertCommandInConsole;
00041 std::function<void(Model*)> initUiForNewModel;
00042 std::function<void()> actualizeActions;
00043 std::function<void()> actualizeTabPanels;
00044 std::function<void(bool)> actualizeModelTextHasChanged;
00045 std::function<bool()> checkModel;
00046 std::function<bool()> setSimulationModelBasedOnText;
00047 std::function<void()> clearModelEditors;
00048 std::function<bool(QString)> saveGraphicalModel;
00049 std::function<bool(QFile*, QString)> saveTextModel;
00050 std::function<Model*(std::string)> loadGraphicalModel;
00051 std::function<void()> connectSceneSignals;
00052 std::function<void(const char*)> disconnectSceneSignals;
00053 };
00054
00056 ModelLifecycleController(QWidget* ownerWidget,
00057 Simulator* simulator,
00058 Ui::MainWindow* ui,
00059 QString* modelFilename,
00060 bool* textModelHasChanged,
00061 bool* graphicalModelHasChanged,
00062 bool* closingApproved,
00063 bool* loaded,
00064 bool& parallelizationEnabled,
00065 int& parallelizationThreads,
00066 int& parallelizationBatchSize,
00067 Callbacks callbacks);
00068
00070 void onActionModelNewTriggered() const;
00072 void onActionModelOpenTriggered() const;
00074 bool openModelFile(const QString& fileName) const;
00076 void onActionModelSaveTriggered() const;
00078 void onActionModelCloseTriggered() const;
00080 void onActionModelInformationTriggered() const;
00082 void onActionModelCheckTriggered() const;
00084 void onActionSimulationConfigureTriggered() const;
00086 void onActionSimulatorExitTriggered() const;
00088 bool hasPendingModelChanges() const;
00090 bool confirmApplicationExit() const;
00091
00092 private:
00093 bool openModelFileInternal(const QString& fileName, bool showDialogs) const;
00094
00095 // Keep QObject ownership separate while allowing dialog parenting.
00096 QWidget* _ownerWidget;
00097 Simulator* _simulator;
00098 Ui::MainWindow* _ui;
00099 QString* _modelFilename;
00100 bool* _textModelHasChanged;
00101 bool* _graphicalModelHasChanged;
00102 bool* _closingApproved;
00103 bool* _loaded;
00104 bool& _parallelizationEnabled;
00105 int& _parallelizationThreads;
00106 int& _parallelizationBatchSize;
00107 Callbacks _callbacks;
00108 };
00109
00110 #endif // MODELLIFECYCLECONTROLLER_H

```

## 16.29 PluginCatalogController.h

```

00001 #ifndef PLUGINCATALOGCONTROLLER_H
00002 #define PLUGINCATALOGCONTROLLER_H
00003
00004 #include <map>
00005 #include <string>
00006 #include <vector>

```

```

00007
00008 #include <QColor>
00009
00010 class Plugin;
00011 class Simulator;
00012 class QPlainTextEdit;
00013 class QPoint;
00014 class QString;
00015 class QTreeWidget;
00016 class QTreeWidgetItem;
00017
00018 // Document the plugin-catalog controller extracted from MainWindow UI code.
00036 class PluginCatalogController {
00037 public:
00039 PluginCatalogController(Simulator* simulator,
00040 QTreeWidget* pluginsTree,
00041 QPlainTextEdit* modelTextEditor,
00042 std::map<std::string, QColor>* pluginCategoryColor);
00043
00045 void insertPluginUI(Plugin* plugin) const;
00047 void reloadFromPluginManager() const;
00049 void applyCategoryExpansionPolicy() const;
00051 void showContextMenu(const QPoint& pos) const;
00053 void insertFakePlugins() const;
00055 void handlePluginItemDoubleClicked(QTreeWidgetItem* item, int column) const;
00057 void handlePluginItemClicked(QTreeWidgetItem* item, int column) const;
00059 void moveTopLevelCategory(QTreeWidgetItem* item, int delta) const;
00060
00061 private:
00062 static QString topLevelCategory(const QString& categoryPath);
00063 QTreeWidgetItem* ensureCategoryPath(const QString& categoryPath) const;
00064 QTreeWidgetItem* ensureCategoryChild(QTreeWidgetItem* parent, const QString& segment) const;
00065 QTreeWidgetItem* findDirectChild(QTreeWidgetItem* parent, const QString& text) const;
00066 QColor backgroundColorForCategory(const QString& categoryPath) const;
00067 void rememberTopLevelCategory(const QString& topLevelCategory) const;
00068 void applyTopLevelCategoryOrder() const;
00069 void applyExpansionPolicyRecursive(QTreeWidgetItem* item) const;
00070 void registerCategoryColorPrefixes(const QString& categoryPath, const QColor& categoryColor)
00071 const;
00072 private:
00073 Simulator* _simulator;
00074 QTreeWidget* _pluginsTree;
00075 QPlainTextEdit* _modelTextEditor;
00076 std::map<std::string, QColor>* _pluginCategoryColor;
00077 mutable std::vector<std::string> _topLevelCategoryOrder;
00078 };
00079
00080 #endif /* PLUGINCATALOGCONTROLLER_H */

```

## 16.30 PropertyEditorController.h

```

00001 #ifndef PROPERTYEDITORCONTROLLER_H
00002 #define PROPERTYEDITORCONTROLLER_H
00003
00004 #include <functional>
00005 #include <map>
00006
00007 class ObjectPropertyBrowser;
00008 class ModelGraphicsView;
00009 class PropertyEditorGenesys;
00010 class SimulationControl;
00011 class DataComponentProperty;
00012 class DataComponentEditor;
00013 class ComboBoxEnum;
00014 class ModelDataDefinition;
00015 class QObject;
00016
00017 // Document the property-editor controller and post-commit synchronization pipeline.
00035 class PropertyEditorController {
00036 public:
00038 PropertyEditorController(ObjectPropertyBrowser* propertyBrowser,
00039 ModelGraphicsView* graphicsView,
00040 PropertyEditorGenesys* propertyGenesys,
00041 std::map<SimulationControl*, DataComponentProperty*>* propertyList,
00042 std::map<SimulationControl*, DataComponentEditor*>* propertyEditorUI,
00043 std::map<SimulationControl*, ComboBoxEnum*>* propertyBox,
00044 std::function<void()> actualizeModelSimLanguage,
00045 std::function<void(bool)> actualizeModelComponents,
00046 std::function<void(bool)> actualizeModelDataDefinitions,
00047 std::function<void()> actualizeModelCppCode,
00048 std::function<bool()> createModelImage,
00049 std::function<void()> actualizeTabPanels,
00050 std::function<void()> actualizeActions);
00051

```

```

00053 void sceneSelectionChanged() const;
00055 void onPropertyEditorModelChanged() const;
00057 void clearPropertyEditorSelection() const;
00059 void bindDataDefinitionFromInspector(ModelDataDefinition* dataDefinition, QObject* graphicalObject
= nullptr) const;
00061 bool isPostCommitPipelineActive() const;
00062
00063 private:
00064 // Keep direct access to the property browser used by this controller.
00065 ObjectPropertyBrowser* _propertyBrowser;
00066 // Keep direct access to the graphical view used for selection inspection.
00067 ModelGraphicsView* _graphicsView;
00068 // Keep direct access to property-editor dependencies required for activation.
00069 PropertyEditorGenesys* _propertyGenesys;
00070 std::map<SimulationControl*, DataComponentProperty*>* _propertyList;
00071 std::map<SimulationControl*, DataComponentEditor*>* _propertyEditorUI;
00072 std::map<SimulationControl*, ComboBoxEnum*>* _propertyBox;
00073
00074 // Keep narrow callbacks for MainWindow-owned update methods.
00075 std::function<void()> _actualizeModelSimLanguage;
00076 std::function<void(bool)> _actualizeModelComponents;
00077 std::function<void(bool)> _actualizeModelDataDefinitions;
00078 std::function<void()> _actualizeModelCppCode;
00079 std::function<bool()> _createModelImage;
00080 std::function<void()> _actualizeTabPanels;
00081 std::function<void()> _actualizeActions;
00082
00083 // Coalesce and serialize heavy refresh requests after property commits.
00084 mutable bool _isGlobalRefreshQueued = false;
00085 mutable bool _isGlobalRefreshRunning = false;
00086 mutable bool _pendingGlobalRefresh = false;
00087 // Coalesce deferred selection synchronization while commit/refresh is active.
00088 mutable bool _isDeferredSelectionSyncScheduled = false;
00089
00090 void _runGlobalRefresh() const;
00091 };
00092
00093 #endif // PROPERTYEDITORCONTROLLER_H

```

## 16.31 SceneToolController.h

```

00001 #ifndef SCENETOOLCONTROLLER_H
00002 #define SCENETOOLCONTROLLER_H
00003
00004 #include <functional>
00005 #include <QRectF>
00006
00007 class ModelGraphicsView;
00008 class ModelGraphicsScene;
00009 class QAction;
00010
00011 namespace Ui {
00012 class MainWindow;
00013 }
00014
00015 // Document the scene tooling controller that receives delegated UI commands.
00032 class SceneToolController {
00033 public:
00037 SceneToolController(ModelGraphicsView* graphicsView,
00038 Ui::MainWindow* ui,
00039 std::function<ModelGraphicsScene*>()> currentScene,
00040 std::function<bool()> createModelImage,
00041 std::function<void()> unselectDrawIcons,
00042 std::function<bool()> checkSelectedDrawIcons,
00043 std::function<void(double)> gentleZoom,
00044 std::function<void()> actualizeActions,
00045 std::function<void()> actualizeTabPanels,
00046 int& zoomValue,
00047 bool& firstClickShowConnection);
00048
00050 void onActionShowGridTriggered();
00052 void onActionShowRuleTriggered();
00054 void onActionShowGuidesTriggered();
00056 void onActionShowSnapTriggered();
00058 void onActionZoomInTriggered();
00060 void onActionZoomOutTriggered();
00062 void onActionZoomAllTriggered();
00064 void onActionZoomActualSizeTriggered();
00066 void syncZoomWidgetsForCurrentScale();
00068 void onHorizontalSliderZoomGraphicalValueChanged(int value);
00070 void onActionDrawLineTriggered();
00072 void onActionDrawRectangleTriggered();
00074 void onActionDrawEllipseTriggered();
00076 void onActionDrawTextTriggered();
00078 void onActionDrawPolygonTriggered();

```

```

00080 void onActionAnimateSimulatedTimeTriggered();
00082 void onActionAnimateVariableTriggered();
00084 void onActionAnimateCounterTriggered();
00086 void onActionAnimateAttributeTriggered();
00088 void onActionAnimateEntityTriggered();
00090 void onActionAnimateEventTriggered();
00092 void onActionAnimateExpressionTriggered();
00094 void onActionAnimatePlotTriggered();
00096 void onActionAnimateQueueTriggered();
00098 void onActionAnimateResourceTriggered();
00100 void onActionAnimateStationTriggered();
00102 void onActionAnimateStatisticsTriggered();
00104 void onActionConnectTriggered();
00106 void onActionArrangeLeftTriggered();
00108 void onActionArrangeRightTriggered();
00110 void onActionArrangeTopTriggered();
00112 void onActionArrangeBottomTriggered();
00114 void onActionArrangeCenterTriggered();
00116 void onActionArrangeMiddleTriggered();
00118 void onActionActivateGraphicalSimulationTriggered();
00120 void onHorizontalSliderAnimationSpeedValueChanged(int value);
00122 void onActionSelectAllTriggered();
00124 void onActionShowInternalElementsTriggered();
00126 void onActionShowEditableElementsTriggered();
00128 void onActionShowAttachedElementsTriggered();
00130 void onCheckBoxShowElementsStateChanged(int arg1);
00132 void onCheckBoxShowInternalsStateChanged(int arg1);
00134 void onCheckBoxShowEditableElementsStateChanged(int arg1);
00136 void onActionShowRecursiveElementsTriggered();
00138 void onCheckBoxShowRecursiveStateChanged(int arg1);
00140 void onCheckBoxShowLevelsStateChanged(int arg1);
00142 void onActionGModelShowConnectTriggered();
00143
00144 private:
00145 QRectF _zoomableModelBounds() const;
00146 double _currentViewScale() const;
00147 double _minimumZoomScale() const;
00148 double _maximumZoomScale() const;
00149 double _zoomScaleFromSliderValue(int value) const;
00150 int _sliderValueForZoomScale(double scale) const;
00151 void _syncZoomWidgetsForCurrentScale();
00152
00153 void activateAnimationDrawingTool(QAction* action, void (ModelGraphicsScene::*drawingFunction)());
00154
00155 ModelGraphicsView* _graphicsView;
00156 Ui::MainWindow* _ui;
00157 std::function<ModelGraphicsScene*>()> _currentScene;
00158 std::function<bool()> _createModelImage;
00159 std::function<void()> _unselectDrawIcons;
00160 std::function<bool()> _checkSelectedDrawIcons;
00161 std::function<void(double)> _gentleZoom;
00162 std::function<void()> _actualizeActions;
00163 std::function<void()> _actualizeTabPanels;
00164 int& _zoomValue;
00165 bool& _firstClickShowConnection;
00166 };
00167
00168 #endif // SCENETOOLCONTROLLER_H

```

## 16.32 SimulationCommandController.h

```

00001 #ifndef SIMULATIONCOMMANDCONTROLLER_H
00002 #define SIMULATIONCOMMANDCONTROLLER_H
00003
00004 #include <functional>
00005 #include <string>
00006
00007 class SimulationController;
00008
00009 // Document simulation command orchestration delegated by MainWindow slots.
00027 class SimulationCommandController {
00028 public:
00032 SimulationCommandController(
00033 SimulationController* simulationController,
00034 const std::function<void(const std::string&);>& insertCommandInConsole,
00035 const std::function<void()>& actualizeActions,
00036 const std::function<bool()>& checkModel,
00037 const std::function<bool()>& setSimulationModelBasedOnText,
00038 const std::function<void()>& prepareNewSimulationOutputs);
00039
00041 void onActionSimulationStartTriggered(bool modelChecked) const;
00043 void onActionSimulationStepTriggered(bool modelChecked) const;
00045 void onActionSimulationPauseTriggered() const;
00047 void onActionSimulationResumeTriggered(bool modelChecked) const;
00049 void onActionSimulationStopTriggered() const;

```

```

00050
00051 private:
00052 SimulationController* _simulationController;
00053 std::function<void(const std::string&)> _insertCommandInConsole;
00054 std::function<void()> _actualizeActions;
00055 std::function<bool()> _checkModel;
00056 std::function<bool()> _setSimulationModelBasedOnText;
00057 std::function<void()> _prepareNewSimulationOutputs;
00058 };
00059
00060 #endif // SIMULATIONCOMMANDCONTROLLER_H

```

## 16.33 SimulationController.h

```

00001 #ifndef SIMULATIONCONTROLLER_H
00002 #define SIMULATIONCONTROLLER_H
00003
00004 #include <functional>
00005
00006 class QWidget;
00007 class Simulator;
00008 class ModelSimulation;
00009
00010 // Document the simulation precondition controller role in the refactored GUI architecture.
00032 class SimulationController {
00033 public:
00039 SimulationController(QWidget* ownerWidget, Simulator* simulator);
00040
00045 bool hasCurrentModelSimulation() const;
00046
00051 ModelSimulation* currentSimulation() const;
00052
00069 bool ensureReady(bool checkModel,
00070 bool modelChecked,
00071 const std::function<bool()>& runModelCheck,
00072 const std::function<bool()>& syncModelFromText) const;
00073
00074 private:
00075 QWidget* _ownerWidget;
00076 Simulator* _simulator;
00077 };
00078
00079 #endif // SIMULATIONCONTROLLER_H

```

## 16.34 SimulationEventController.h

```

00001 #ifndef SIMULATIONEVENTCONTROLLER_H
00002 #define SIMULATIONEVENTCONTROLLER_H
00003
00004 #include <functional>
00005
00006 #include "../kernel/simulator/model/ModelSimulation.h"
00007 #include "kernel/simulator/OnEventManager.h"
00008 #include "kernel/simulator/Simulator.h"
00009
00010 class MainWindow;
00011 class ModelGraphicsScene;
00012 class ModelGraphicsView;
00013 class QLabel;
00014 class QProgressBar;
00015 class QTableWidgetItem;
00016 class QTextEdit;
00017 class QTabWidget;
00018 class QAction;
00019
00020 // Document the controller that bridges kernel simulation events to GUI updates.
00040 class SimulationEventController {
00041 public:
00042 // Group callback dependencies to keep constructor explicit and narrow.
00043 struct Callbacks {
00044 std::function<void()> actualizeActions;
00045 std::function<void(SimulationEvent*)> actualizeSimulationEvents;
00046 std::function<void(bool)> actualizeDebugEntities;
00047 std::function<void(bool)> actualizeDebugVariables;
00048 std::function<void(SimulationEvent*)> actualizeGraphicalModel;
00049 };
00050
00052 SimulationEventController(Simulator* simulator,
00053 ModelGraphicsScene* scene,
00054 ModelGraphicsView* graphicsView,
00055 QLabel* replicationLabel,
00056 QProgressBar* simulationProgressBar,

```

```

00057 QTableWidget* simulationEventsTable,
00058 QTableWidget* entitiesTable,
00059 QTableWidget* variablesTable,
00060 QTextEdit* simulationText,
00061 QTextEdit* reportsText,
00062 QTabWidget* centralTabWidget,
00063 QAction* activateGraphicalSimulation,
00064 QAction* animationEnabledAction,
00065 bool* modelChecked,
00066 int tabCentralReportsIndex,
00067 Callbacks callbacks);
00068
00070 void onModelCheckSuccessHandler(ModelEvent* re) const;
00072 void onReplicationStartHandler(SimulationEvent* re) const;
00074 void onSimulationStartHandler(SimulationEvent* re) const;
00076 void onSimulationPausedHandler(SimulationEvent* re) const;
00078 void onSimulationResumeHandler(SimulationEvent* re) const;
00080 void onSimulationEndHandler(SimulationEvent* re) const;
00082 void onReplicationEndHandler(SimulationEvent* re) const;
00084 void onProcessEventHandler(SimulationEvent* re) const;
00086 void onEntityCreateHandler(SimulationEvent* re) const;
00088 void onEntityRemoveHandler(SimulationEvent* re) const;
00090 void onMoveEntityEvent(SimulationEvent* re) const;
00092 void onAfterProcessEvent(SimulationEvent* re) const;
00094 void setOnEventHandlers(MainWindow* owner) const;
00095
00096 private:
00097 // Keep simulator access scoped to simulation-event responsibilities.
00098 Simulator* _simulator;
00099 // Keep scene dependency for animation-related handlers.
00100 ModelGraphicsScene* _scene;
00101 // Keep graphics-view dependency for legacy compatibility and potential scene access.
00102 ModelGraphicsView* _graphicsView;
00103 // Keep replication label dependency for replication-start updates.
00104 QLabel* _replicationLabel;
00105 // Keep simulation progress bar dependency for process-event updates.
00106 QProgressBar* _simulationProgressBar;
00107 // Keep simulation-events table dependency for replication-start updates.
00108 QTableWidget* _simulationEventsTable;
00109 // Keep entities table dependency for simulation-start resets.
00110 QTableWidget* _entitiesTable;
00111 // Keep variables table dependency for simulation-start resets.
00112 QTableWidget* _variablesTable;
00113 // Keep simulation text dependency for simulation-start resets.
00114 QTextEdit* _simulationText;
00115 // Keep reports text dependency for simulation-start resets.
00116 QTextEdit* _reportsText;
00117 // Keep central-tab dependency for simulation-end tab switch.
00118 QTabWidget* _centralTabWidget;
00119 // Keep action dependency used by graphical simulation toggle.
00120 QAction* _activateGraphicalSimulation;
00121 // Keep action dependency used by the global Simulation/Animation runtime gate.
00122 QAction* _animationEnabledAction;
00123 // Keep mutable model-check flag dependency used at simulation end.
00124 bool* _modelChecked;
00125 // Keep tab index dependency used for report-tab activation.
00126 int _tabCentralReportsIndex;
00127 // Keep callback dependencies for delegated MainWindow operations.
00128 Callbacks _callbacks;
00129 // Throttle GUI event processing when animation/output updates are suppressed.
00130 mutable unsigned int _suppressedUiProgressEvents = 0;
00131
00132 bool animationsEnabled() const;
00133 void processSuppressedUiProgressEvents() const;
00134 };
00135
00136 #endif // SIMULATIONEVENTCONTROLLER_H

```

## 16.35 TraceConsoleController.h

```

00001 #ifndef TRACECONSOLECONTROLLER_H
00002 #define TRACECONSOLECONTROLLER_H
00003
00004 #include "kernel/simulator/TraceManager.h"
00005
00006 #include <functional>
00007
00008 class QTextEdit;
00009
00010 // Document the trace rendering bridge used by MainWindow simulator callbacks.
00028 class TraceConsoleController {
00029 public:
00031 TraceConsoleController(QTextEdit* console,
00032 QTextEdit* simulationText,
00033 QTextEdit* reportsText,

```

```

00034 std::function<bool()> traceWidgetsEnabled = {});
00035
00037 void simulatorTraceHandler(TraceEvent e) const;
00039 void simulatorTraceErrorHandler(TraceErrorEvent e) const;
00041 void simulatorTraceSimulationHandler(TraceSimulationEvent e) const;
00043 void simulatorTraceReportsHandler(TraceEvent e) const;
00044
00045 private:
00046 // Keep the console trace sink used by normal/error traces.
00047 QTextEdit* _console;
00048 // Keep the simulation trace sink used by simulation traces.
00049 QTextEdit* _simulationText;
00050 // Keep the reports trace sink used by reports traces.
00051 QTextEdit* _reportsText;
00052 // Runtime gate used to avoid expensive trace-widget churn during non-animated simulations.
00053 std::function<bool()> _traceWidgetsEnabled;
00054
00055 bool shouldRenderTraceWidgets() const;
00056 };
00057
00058 #endif // TRACECONSOLECONTROLLER_H

```

## 16.36 dialogBreakpoint.h

```

00001 /*
00002 * The MIT License
00003 *
00004 * Copyright 2022 rlcancian.
00005 *
00006 * Permission is hereby granted, free of charge, to any person obtaining a copy
00007 * of this software and associated documentation files (the "Software"), to deal
00008 * in the Software without restriction, including without limitation the rights
00009 * to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00010 * copies of the Software, and to permit persons to whom the Software is
00011 * furnished to do so, subject to the following conditions:
00012 *
00013 * The above copyright notice and this permission notice shall be included in
00014 * all copies or substantial portions of the Software.
00015 *
00016 * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00017 * IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00018 * FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00019 * AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00020 * LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00021 * OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
00022 * THE SOFTWARE.
00023 */
00024
00025 /*
00026 * File: dialogBreakpoint.h
00027 * Author: rlcancian
00028 *
00029 * Created on 6 de fevereiro de 2022, 14:24
00030 */
00031
00032 #ifndef _DIALOGBREAKPOINT_H
00033 #define _DIALOGBREAKPOINT_H
00034
00035 #include "ui_dialogBreakpoint.h"
00036 #include "kernel/simulator/Simulator.h"
00037
00038 class dialogBreakpoint : public QDialog {
00039
00040 Q_OBJECT
00041 public:
00042 class MVCResult {
00043 public:
00044
00045 MVCResult(std::string type, std::string on) {
00046 this->type = type;
00047 this->on = on;
00048 }
00049 std::string type;
00050 std::string on;
00051 };
00052 public:
00053 dialogBreakpoint();
00054 virtual ~dialogBreakpoint();
00055 public:
00056 void setMVCModel(Simulator* simulator);
00057 dialogBreakpoint::MVCResult* getMVCResult();
00058 private slots:
00059 void on_comboBox_Type_activated(const QString &arg1);
00060 void on_comboBox_On_activated(const QString &arg1);
00061 void on_buttonBox_accepted();

```

```

00062 private:
00063 Ui::dialogBreakpoint widget;
00064 Simulator* simulator;
00065 };
00066
00067 #endif /* _DIALOGBREAKPOINT_H */

```

## 16.37 DialogFind.h

```

00001 #ifndef DIALOGFIND_H
00002 #define DIALOGFIND_H
00003
00004 #include <QDialog>
00005 #include <QLabel>
00006 #include <QHBoxLayout>
00007 #include <QLineEdit>
00008 #include <QPushButton>
00009 #include <QMessageBox>
00010 #include "../graphicals/GraphicalModelComponent.h"
00011 #include "graphicals/ModelGraphicsScene.h"
00012 #include <QGraphicsItem>
00013 #include <QList>
00014
00015
00016 class QLineEdit;
00017 class QPushButton;
00018
00019 class FindLineEdit : public QLineEdit {
00020 Q_OBJECT
00021
00022 public:
00023 FindLineEdit(QWidget *parent = nullptr) : QLineEdit(parent) {
00024 connect(this, &QLineEdit::textChanged, this, &FindLineEdit::onTextChanged);
00025 }
00026
00027 private slots:
00028 void onTextChanged(const QString &text) {
00029 qDebug() << "Texto atual: " << text;
00030
00031 if (text.length() < previousText.length()) {
00032 this->setStyleSheet("color: black");
00033 }
00034
00035 previousText = text;
00036 }
00037
00038 private:
00039 QString previousText;
00040 };
00041
00042
00043 class DialogFind : public QDialog
00044 {
00045 Q_OBJECT
00046
00047 public:
00048 DialogFind(QWidget *parent = nullptr, ModelGraphicsScene * scene = nullptr);
00049 QString getFindText();
00050
00051 private slots:
00052 void findClicked();
00053 void nextClicked();
00054 void previousClicked();
00055
00056 private:
00057 QPushButton *findButton;
00058 QPushButton *nextButton;
00059 QPushButton *previousButton;
00060 FindLineEdit *lineEdit;
00061 bool clicked;
00062 int counter;
00063 QList<QGraphicsItem*> * finded = new QList<QGraphicsItem*>();
00064 ModelGraphicsScene * scene;
00065
00066 };
00067
00068 #endif // DIALOGFIND_H
00069
00070

```



## 16.38 Dialogmodelinformation.h

```

00001 #ifndef DIALOGMODELINFORMATION_H
00002 #define DIALOGMODELINFORMATION_H
00003
00004 #include "../kernel/simulator/model/ModelInfo.h"
00005 #include <QDialog>
00006 #include <string>
00007
00008 namespace Ui {
00009 class DialogModelInformation;
00010 }
00011
00012 class DialogModelInformation : public QDialog
00013 {
00014 Q_OBJECT
00015
00016 public:
00017 explicit DialogModelInformation(QWidget *parent = nullptr);
00018 ~DialogModelInformation();
00019 void setModelInfo(ModelInfo* modelInfo);
00020
00021 public slots:
00022 void accept() override;
00023 void reject() override;
00024
00025 private:
00026 void _loadModelInfo();
00027 bool _hasPendingChanges() const;
00028 std::string _projectTitleFromUi() const;
00029 std::string _nameFromUi() const;
00030 std::string _versionFromUi() const;
00031 std::string _descriptionFromUi() const;
00032 std::string _analystNameFromUi() const;
00033
00034 Ui::DialogModelInformation *ui;
00035 ModelInfo* _modelInfo = nullptr;
00036 std::string _originalProjectTitle;
00037 std::string _originalName;
00038 std::string _originalVersion;
00039 std::string _originalDescription;
00040 std::string _originalAnalystName;
00041 };
00042
00043 #endif // DIALOGMODELINFORMATION_H

```

## 16.39 dialogpluginmanager.h

```

00001 #ifndef DIALOGPLUGINMANAGER_H
00002 #define DIALOGPLUGINMANAGER_H
00003
00004 #include <QDialog>
00005
00006 #include <functional>
00007 #include <string>
00008 #include <vector>
00009
00010 class Plugin;
00011 class PluginInformation;
00012 class PluginLoadIssue;
00013 class SystemDependencyCheckResult;
00014 class Simulator;
00015 class QTableWidgetItem;
00016 class QTableWidgetItem;
00017
00018 namespace Ui {
00019 class DialogPluginManager;
00020 }
00021
00022 class DialogPluginManager : public QDialog {
00023 Q_OBJECT
00024
00025 public:
00026 explicit DialogPluginManager(QWidget* parent = nullptr);
00027 ~DialogPluginManager();
00028 void setSimulator(Simulator* simulator);
00029 void setPluginCatalogRefreshCallback(std::function<void()> refreshCallback);
00030 void showProblemPluginsTab();
00031
00032 private slots:
00033 void on_pushButtonBrowseAutoload_clicked();
00034 void on_pushButtonAutoLoadNow_clicked();
00035 void on_pushButtonCheck_clicked();
00036 void on_pushButtonInsert_clicked();

```

```

00038 void on_pushButtonResolveSelected_clicked();
00039 void on_pushButtonRemove_clicked();
00040 void on_pushButtonRefresh_clicked();
00041
00042 private:
00043 void _refreshPluginTable();
00044 void _configureLoadedPluginTable();
00045 void _configurePluginIssuesTable();
00046 void _refreshPluginCatalog();
00047 void _appendPluginIssueRow(const PluginLoadIssue& issue);
00048 Plugin* _selectedPlugin() const;
00049 std::vector<PluginLoadIssue> _selectedPluginLoadIssues() const;
00050 bool _isKernelPlugin(const Plugin* plugin) const;
00051 bool _isPluginUsedByCurrentModel(const Plugin* plugin) const;
00052 bool _choosePluginFilename(std::string& filename) const;
00053 QString _formatDynamicDependencyTableText(const PluginInformation* info) const;
00054 QString _formatSystemDependencyTableText(const PluginInformation* info) const;
00055 QString _formatSystemDependencyPreflight(const SystemDependencyCheckResult& result) const;
00056 QString _formatFields(const PluginInformation* info) const;
00057 bool _confirmSystemDependencyInstallation(const SystemDependencyCheckResult& result) const;
00058 bool _runInstallCommandsForIssue(const PluginLoadIssue& issue, QString* feedback);
00059 bool _runInstallCommandInteractive(const QString& command, QString* feedback);
00060 bool _terminalCommandForScript(const QString& script, QString* program, QStringList* arguments)
00061 const;
00062 void _configureCommonTable(QTableWidget* table) const;
00063 void _showOperationResult(const QString& title, const QString& message) const;
00064
00065 Ui::DialogPluginManager* ui;
00066 Simulator* _simulator = nullptr;
00067 std::function<void()> _refreshPluginCatalogCallback;
00068 };
00069
00070 #endif // DIALOGPLUGINMANAGER_H

```

## 16.40 DialogReportIssue.h

```

00001 #ifndef DIALOGREPORTISSUE_H
00002 #define DIALOGREPORTISSUE_H
00003
00004 #include <QDialog>
00005
00006 #include "../services/IssueReportRelayService.h"
00007
00008 class QCheckBox;
00009 class QComboBox;
00010 class QLabel;
00011 class QLineEdit;
00012 class QPlainTextEdit;
00013 class QPushButton;
00014
00015 class DialogReportIssue : public QDialog {
00016 public:
00017 explicit DialogReportIssue(QWidget* parent = nullptr);
00018
00019 IssueReportDraft draft() const;
00020 void setRelayConfigurationStatus(const QString& endpoint, bool configured, const QString& detail);
00021 private:
00022 void _updateSendButton();
00023
00024 QComboBox* _categoryCombo = nullptr;
00025 QLineEdit* _titleEdit = nullptr;
00026 QPlainTextEdit* _summaryEdit = nullptr;
00027 QPlainTextEdit* _stepsEdit = nullptr;
00028 QPlainTextEdit* _expectedEdit = nullptr;
00029 QPlainTextEdit* _observedEdit = nullptr;
00030 QPlainTextEdit* _additionalContextEdit = nullptr;
00031 QLineEdit* _contactEdit = nullptr;
00032 QCheckBox* _includeScreenshotCheck = nullptr;
00033 QCheckBox* _includeLogTailCheck = nullptr;
00034 QCheckBox* _includeModelTextCheck = nullptr;
00035 QLabel* _relayStatusLabel = nullptr;
00036 QPushButton* _sendButton = nullptr;
00037 bool _relayConfigured = true;
00038 };
00039
00040 #endif // DIALOGREPORTISSUE_H

```

## 16.41 DialogSelectCounter.h

```

00001 #ifndef DIALOGSELECTCOUNTER_H
00002 #define DIALOGSELECTCOUNTER_H

```

```

00003
00004 #include <QDialog>
00005 #include <QComboBox>
00006 #include <QPushButton>
00007
00008 #include "../kernel/simulator/essentialPlugins/Counter.h"
00009 class DialogSelectCounter : public QDialog {
00010 Q_OBJECT
00011
00012 public:
00013 explicit DialogSelectCounter(QWidget *parent = nullptr);
00014
00015 Counter* selectedIndex() const;
00016 void setCounterItems(QList<Counter *> *counters, Counter *counter);
00017
00018 private slots:
00019 void onOkClicked();
00020 void onCancelClicked();
00021
00022 private:
00023 QComboBox *comboBox;
00024 QPushButton *okButton;
00025 QPushButton *cancelButton;
00026 QMap<int, Counter*> indexToCounter;
00027 };
00028
00029 #endif // DIALOGSELECTCOUNTER_H

```

## 16.42 DialogSelectVariable.h

```

00001 #ifndef DIALOGSELECTVARIABLE_H
00002 #define DIALOGSELECTVARIABLE_H
00003
00004 #include <QDialog>
00005 #include <QComboBox>
00006 #include <QPushButton>
00007 #include <QList>
00008 #include <QMap>
00009
00010 class ModelDataDefinition;
00011
00012 class DialogSelectVariable : public QDialog {
00013 Q_OBJECT
00014
00015 public:
00016 explicit DialogSelectVariable(QWidget *parent = nullptr);
00017
00018 ModelDataDefinition* selectedIndex() const;
00019 void setVariableItems(QList<ModelDataDefinition *> *definitions, ModelDataDefinition *definition);
00020
00021 private slots:
00022 void onOkClicked();
00023 void onCancelClicked();
00024
00025 private:
00026 QComboBox *comboBox;
00027 QPushButton *okButton;
00028 QPushButton *cancelButton;
00029 QMap<int, ModelDataDefinition*> indexToVariable;
00030 };
00031
00032 #endif // DIALOGSELECTVARIABLE_H

```

## 16.43 dialogsimulationconfigure.h

```

00001 #ifndef DIALOGSIMULATIONCONFIGURE_H
00002 #define DIALOGSIMULATIONCONFIGURE_H
00003
00004 #include "../kernel/simulator/model/ModelSimulation.h"
00005 #include <QDialog>
00006 #include <string>
00007
00008 class QComboBox;
00009 class ExperimentManager;
00010 class SimulationReporter_if;
00011
00012 namespace Ui {
00013 class DialogSimulationConfigure;
00014 }
00015
00016 class DialogSimulationConfigure : public QDialog
00017 {

```

```

00018 Q_OBJECT
00019
00020 public:
00021 explicit DialogSimulationConfigure(QWidget *parent = nullptr);
00022 ~DialogSimulationConfigure();
00023 void setModelSimulation(ModelSimulation* modelSimulation);
00024 void setExperimentManager(ExperimentManager* experimentManager);
00025 void setParallelizationSettings(bool* enabled, int* localThreads, int* batchSize);
00026
00027 public slots:
00028 void accept() override;
00029 void reject() override;
00030
00031 private:
00032 struct SimulationConfiguration {
00033 unsigned int numberOfReplications = 1;
00034 Util::TimeUnit replicationBaseTimeUnit = Util::TimeUnit::unknown;
00035 double replicationLength = 0.0;
00036 Util::TimeUnit replicationLengthTimeUnit = Util::TimeUnit::unknown;
00037 double warmUpPeriod = 0.0;
00038 Util::TimeUnit warmUpPeriodTimeUnit = Util::TimeUnit::unknown;
00039 std::string terminatingCondition;
00040 bool initializeSystem = true;
00041 bool initializeStatistics = true;
00042 bool stepByStep = false;
00043 bool pauseOnEvent = false;
00044 bool pauseOnReplication = false;
00045 bool parallelizationEnabled = false;
00046 int parallelizationThreads = 1;
00047 int parallelizationBatchSize = 100;
00048 bool distributedParallelizationEnabled = false;
00049 std::string distributedCoordinatorUrl;
00050 std::string distributedToken;
00051 };
00052
00053 void _populateTimeUnitComboBoxes();
00054 void _loadModelSimulation();
00055 void _loadSimulationReporter();
00056 void _loadExperimentManager();
00057 void _loadParallelizationSettings();
00058 SimulationConfiguration _configurationFromUi() const;
00059 bool _hasPendingChanges() const;
00060 Util::TimeUnit _timeUnitFromComboBox(const QComboBox* comboBox) const;
00061 void _setTimeUnitComboBoxIndex(QComboBox* comboBox, Util::TimeUnit timeUnit);
00062 void _applyIfChanged(const SimulationConfiguration& edited);
00063
00064 Ui::DialogSimulationConfigure *ui;
00065 ModelSimulation* _modelSimulation = nullptr;
00066 SimulationReporter_if* _simulationReporter = nullptr;
00067 ExperimentManager* _experimentManager = nullptr;
00068 bool* _parallelizationEnabled = nullptr;
00069 int* _parallelizationThreads = nullptr;
00070 int* _parallelizationBatchSize = nullptr;
00071 SimulationConfiguration _originalConfiguration;
00072 };
00073
00074 #endif // DIALOGSIMULATIONCONFIGURE_H

```

## 16.44 dialogsistempreferences.h

```

00001 #ifndef DIALOGSYSTEMPREFERENCES_H
00002 #define DIALOGSYSTEMPREFERENCES_H
00003
00004 #include <QDialog>
00005
00006 class Simulator;
00007
00008 namespace Ui {
00009 class DialogSystemPreferences;
00010 }
00011
00012 class DialogSystemPreferences : public QDialog
00013 {
00014 Q_OBJECT
00015
00016 public:
00017 explicit DialogSystemPreferences(Simulator* simulator = nullptr, QWidget *parent = nullptr);
00018 ~DialogSystemPreferences();
00019
00020 private:
00021 void loadPreferencesIntoUi();
00022 void savePreferencesFromUi();
00023 void populateTraceLevels();
00024 void browseSpecificModelFile();
00025

```

```

00026 Ui::DialogSystemPreferences *ui;
00027 Simulator* _simulator = nullptr;
00028 };
00029
00030 #endif // DIALOGSYSTEMPREFERENCES_H

```

## 16.45 DialogTimerConfigure.h

```

00001 #ifndef DIALOGTIMERCONFIGURE_H
00002 #define DIALOGTIMERCONFIGURE_H
00003
00004 #include <QDialog>
00005 #include <QPushButton>
00006
00007 #include "../animations/AnimationTimer.h"
00008
00009 namespace Ui {
00010 class DialogTimerConfigure;
00011 }
00012
00013 class DialogTimerConfigure : public QDialog
00014 {
00015 Q_OBJECT
00016
00017 public:
00018 explicit DialogTimerConfigure(QWidget *parent = nullptr);
00019 ~DialogTimerConfigure();
00020
00021 // recebe o Timer
00022 void setAnimationTimer(AnimationTimer *animationTimer);
00023 // atualiza os valores no Timer
00024 void updateAnimationTimer();
00025
00026 private slots:
00027 // slot para tratar o clique no botão "OK"
00028 void on_buttonBox_accepted();
00029
00030 // slot para tratar mudança no formato de hora para 12 horas
00031 void on_radioButtonTwelve_clicked(bool checked);
00032
00033 // slot para tratar mudança no formato de hora para 24 horas
00034 void on_radioButtonTwentyFour_clicked(bool checked);
00035
00036 private:
00037 Ui::DialogTimerConfigure *ui;
00038 AnimationTimer *_animationTimer;
00039 };
00040
00041 #endif // DIALOGTIMERCONFIGURE_H

```

## 16.46 GuiExtensionConfigurationDialog.h

```

00001 #ifndef GUIEXTENSIONCONFIGURATIONDIALOG_H
00002 #define GUIEXTENSIONCONFIGURATIONDIALOG_H
00003
00004 #include "GuiExtensionContracts.h"
00005
00006 #include <QDialog>
00007 #include <QJsonObject>
00008
00009 #include <functional>
00010 #include <vector>
00011
00012 class QTableWidgetItem;
00013 class QPushButton;
00014
00015 class GuiExtensionConfigurationDialog : public QDialog {
00016 public:
00017 GuiExtensionConfigurationDialog(
00018 QWidget* parent,
00019 std::vector<const GuiExtensionPlugin*> plugins,
00020 std::function<void()> onSaved = {});
00021
00022 private:
00023 void reloadFromDisk();
00024 void saveToDisk();
00025 QString _configPath() const;
00026 QJsonObject _loadConfigRoot() const;
00027 bool _writeConfigRoot(const QJsonObject& root) const;
00028 void _populateFromConfig(const QJsonObject& root);
00029 void _configureTable();
00030 static std::string _normalizeId(const std::string& text);

```

```

00031
00032 private:
00033 std::vector<const GuiExtensionPlugin*> _plugins;
00034 std::function<void()> _onSaved;
00035 QTableWidgetItem* _table = nullptr;
00036 QPushButton* _reloadButton = nullptr;
00037 QPushButton* _saveButton = nullptr;
00038 QPushButton* _closeButton = nullptr;
00039 };
00040
00041 #endif // GUIEXTENSIONCONFIGURATIONDIALOG_H

```

## 16.47 GuiExtensionContracts.h

```

00001 #ifndef GUIEXTENSIONCONTRACTS_H
00002 #define GUIEXTENSIONCONTRACTS_H
00003
00004 #include <functional>
00005 #include <string>
00006 #include <vector>
00007
00008 class Simulator;
00009 class QMainWindow;
00010 class QWidget;
00011 class ModelGraphicsView;
00012 class ModelGraphicsScene;
00013
00014 namespace Ui {
00015 class MainWindow;
00016 }
00017
00018 struct GuiExtensionRuntimeContext {
00019 Simulator* simulator = nullptr;
00020 QMainWindow* mainWindow = nullptr;
00021 Ui::MainWindow* ui = nullptr;
00022 ModelGraphicsView* graphicsView = nullptr;
00023 ModelGraphicsScene* graphicsScene = nullptr;
00024 };
00025
00026 struct GuiActionContribution {
00027 std::string id;
00028 std::string text;
00029 std::string menuPath;
00030 std::string toolBarId;
00031 std::string statusTip;
00032 std::string shortcut;
00033 std::function<bool(const GuiExtensionRuntimeContext*)> isVisible;
00034 std::function<bool(const GuiExtensionRuntimeContext*)> isEnabled;
00035 std::function<void(const GuiExtensionRuntimeContext*)> trigger;
00036 };
00037
00038 struct GuiDockContribution {
00039 std::string id;
00040 std::string title;
00041 std::string areaHint;
00042 std::function<bool(const GuiExtensionRuntimeContext*)> isVisible;
00043 std::function<QWidget*(const GuiExtensionRuntimeContext*)> createWidget;
00044 };
00045
00046 struct GuiWindowContribution {
00047 std::string id;
00048 std::string menuPath;
00049 std::string text;
00050 std::function<bool(const GuiExtensionRuntimeContext*)> isVisible;
00051 std::function<void(const GuiExtensionRuntimeContext*)> open;
00052 };
00053
00054 class GuiExtensionRegistry {
00055 public:
00056 void addAction(GuiActionContribution contribution) {
00057 _actions.push_back(std::move(contribution));
00058 }
00059
00060 void addDock(GuiDockContribution contribution) {
00061 _docks.push_back(std::move(contribution));
00062 }
00063
00064 void addWindow(GuiWindowContribution contribution) {
00065 _windows.push_back(std::move(contribution));
00066 }
00067
00068 const std::vector<GuiActionContribution>& actions() const {
00069 return _actions;
00070 }
00071 }

```

```

00072 const std::vector<GuiDockContribution>& docks() const {
00073 return _docks;
00074 }
00075
00076 const std::vector<GuiWindowContribution>& windows() const {
00077 return _windows;
00078 }
00079
00080 void clear() {
00081 _actions.clear();
00082 _docks.clear();
00083 _windows.clear();
00084 }
00085
00086 private:
00087 std::vector<GuiActionContribution> _actions;
00088 std::vector<GuiDockContribution> _docks;
00089 std::vector<GuiWindowContribution> _windows;
00090 };
00091
00092 class GuiExtensionPlugin {
00093 public:
00094 virtual ~GuiExtensionPlugin() = default;
00095
00096 virtual std::string extensionId() const = 0;
00097 virtual std::string displayName() const = 0;
00098 virtual std::string version() const = 0;
00099 virtual std::vector<std::string> requiredModelPlugins() const = 0;
00100 virtual void registerContributions(GuiExtensionRegistry* registry) const = 0;
00101 };
00102
00103 #endif /* GUIEXTENSIONCONTRACTS_H */

```

## 16.48 GuiExtensionManager.h

```

00001 #ifndef GUIEXTENSIONMANAGER_H
00002 #define GUIEXTENSIONMANAGER_H
00003
00004 #include "GuiExtensionContracts.h"
00005
00006 #include <string>
00007 #include <vector>
00008
00009 class QAction;
00010 class QDockWidget;
00011 class QMainWindow;
00012 class QMenu;
00013 class QToolBar;
00014
00015 class GuiExtensionManager {
00016 public:
00017 explicit GuiExtensionManager(QMainWindow* mainWindow);
00018 ~GuiExtensionManager();
00019
00020 void setPlugins(std::vector<const GuiExtensionPlugin*> plugins);
00021 void setLoadedModelPluginIds(std::vector<std::string> modelPluginIds);
00022 void rebuild(const GuiExtensionRuntimeContext& context);
00023 void clear();
00024
00025 private:
00026 QMenu* _resolveMenuPath(const std::string& menuPath);
00027 QToolBar* _resolveToolBar(const std::string& toolBarId) const;
00028 void _applyActionContribution(const GuiActionContribution& contribution, const
GuiExtensionRuntimeContext& context);
00029 void _applyWindowContribution(const GuiWindowContribution& contribution, const
GuiExtensionRuntimeContext& context);
00030 void _applyDockContribution(const GuiDockContribution& contribution, const
GuiExtensionRuntimeContext& context);
00031 bool _isPluginDependenciesSatisfied(const GuiExtensionPlugin* plugin) const;
00032
00033 private:
00034 QMainWindow* _mainWindow = nullptr;
00035 std::vector<const GuiExtensionPlugin*> _plugins;
00036 std::vector<std::string> _loadedModelPluginIds;
00037 std::vector<QAction*> _createdActions;
00038 std::vector<QAction*> _createdMenuActions;
00039 std::vector<QDockWidget*> _createdDocks;
00040 };
00041
00042 #endif /* GUIEXTENSIONMANAGER_H */

```

## 16.49 GuiExtensionPluginCatalog.h

```

00001 #ifndef GUIEXTENSIONPLUGINCATALOG_H
00002 #define GUIEXTENSIONPLUGINCATALOG_H
00003
00004 #include "GuiExtensionContracts.h"
00005
00006 #include <string>
00007 #include <vector>
00008
00009 struct GuiExtensionPresentationMetadata {
00010 std::string category;
00011 std::string group;
00012 int priority = 0;
00013 };
00014
00015 class GuiExtensionPluginCatalog {
00016 public:
00017 static const std::vector<const GuiExtensionPlugin*>& plugins();
00018 static std::vector<const GuiExtensionPlugin*> resolvedPlugins();
00019 static GuiExtensionPresentationMetadata presentationMetadata(const std::string& extensionId);
00020 static void registerPlugin(const GuiExtensionPlugin* plugin);
00021 };
00022
00023 class GuiExtensionPluginAutoRegistrar {
00024 public:
00025 explicit GuiExtensionPluginAutoRegistrar(const GuiExtensionPlugin* plugin);
00026 };
00027
00028 #define REGISTER_GUI_EXTENSION_PLUGIN(PluginType) \
00029 namespace { \
00030 static const PluginType _instance_##PluginType; \
00031 static const GuiExtensionPluginAutoRegistrar \
00032 _registrar_##PluginType(&_instance_##PluginType); \
00033 } \
00034
00035 #endif /* GUIEXTENSIONPLUGINCATALOG_H */

```

## 16.50 GraphicalAnimateExpression.h

```

00001 #ifndef GRAPHICALANIMATEEXPRESSION_H
00002 #define GRAPHICALANIMATEEXPRESSION_H
00003
00004 #include <QGraphicsObject>
00005
00006 class GraphicalAnimateExpression : public QGraphicsObject {
00007 public:
00008 GraphicalAnimateExpression();
00009 };
00010
00011 #endif // GRAPHICALANIMATEEXPRESSION_H

```

## 16.51 GraphicalAssociation.h

```

00001 #ifndef GRAPHICALASSOCIATION_H
00002 #define GRAPHICALASSOCIATION_H
00003
00004
00005 class GraphicalAssociation
00006 {
00007 public:
00008 GraphicalAssociation();
00009 };
00010
00011 #endif // GRAPHICALASSOCIATION_H

```

## 16.52 GraphicalComponentPort.h

```

00001 #ifndef GRAPHICALCOMPONENTPORT_H
00002 #define GRAPHICALCOMPONENTPORT_H
00003
00004 #include <QGraphicsItem>
00005 #include <QGraphicsObject>
00006 #include <QPen>
00007 #include <QBrush>
00008 #include "../kernel/simulator/model/ModelComponent.h"
00009 #include "TraitsGUI.h"
00010 // #include "GraphicalConnection.h"
00011

```



```

00012 class GraphicalModelComponent;
00013 class GraphicalConnection;
00014
00015 class GraphicalComponentPort : public QGraphicsObject {
00016 public:
00024 GraphicalComponentPort(GraphicalModelComponent* componentGraph, bool isInputPort, unsigned int
portNum = 0, QGraphicsItem *parent = nullptr);
00025
00026 protected: // virtual functions
00027 //virtual void mousePressEvent(QGraphicsSceneMouseEvent *mouseEvent);
00028 //virtual void mouseReleaseEvent(QGraphicsSceneMouseEvent *mouseEvent);
00029 //virtual void mouseMoveEvent(QGraphicsSceneMouseEvent *mouseEvent);
00031 virtual QVariant itemChange(QGraphicsItem::GraphicsItemChange change, const QVariant &value)
override;
00032 public: //overrides
00034 QRectF boundingRect() const override;
00036 void paint(QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget) override;
00038 qreal width() const;
00040 qreal height() const;
00041 public:
00043 void addGraphicalConnection(GraphicalConnection* connection);
00045 void removeGraphicalConnection(GraphicalConnection* connection);
00046
00047 public: // sets and gets
00049 unsigned int portNum() const;
00051 bool isInputPort() const;
00053 GraphicalModelComponent *graphicalComponent() const;
00055 QList<GraphicalConnection*>*getConnections() const;
00056 private:
00057 QColor myrgba(uint64_t color); // TODO: Should NOT be here, but in UtilGUI.h, but then it
generates multiple definitions error
00058 private:
00059 qreal _width = TraitsGUI<GComponentPort>::width;//20;
00060 qreal _height = _width;
00061 qreal _margin = TraitsGUI<GComponentPort>::margin; //2
00062 int _raise = TraitsGUI<GComponentPort>::raise; //3;
00063 bool _isInputPort;
00064 unsigned int _portNum;
00065 GraphicalModelComponent* _componentGraph;
00066 QList<GraphicalConnection*>* _connections = new QList<GraphicalConnection*>();
00067 };
00068
00069 #endif // GRAPHICALCOMPONENTPORT_H

```

## 16.53 GraphicalConnection.h

```

00001 #ifndef GRAPHICALCONNECTION_H
00002 #define GRAPHICALCONNECTION_H
00003
00004 #include <QGraphicsObject>
00005
00006 #include <QGraphicsItem>
00007 #include <QGraphicsObject>
00008 #include <QPointer>
00009 #include <QPainterPath>
00010 #include <QPen>
00011 #include <QBrush>
00012 #include "GraphicalConnectionStyle.h"
00013 #include "GraphicalComponentPort.h"
00014 // #include "kernel/simulator/ModelComponent.h"
00015 // #include "kernel/simulator/Plugin.h"
00016 #include "kernel/simulator/ConnectionManager.h"
00017
00029 class GraphicalConnection : public QGraphicsObject {
00030 public:
00031 enum class ConnectionType : int {
00032 HORIZONTAL = 1, VERTICAL = 2, DIRECT = 3, USERDEFINED = 4
00033 };
00034
00035 public:
00037 GraphicalConnection(GraphicalComponentPort* sourceGraphicalPort, GraphicalComponentPort*
destinationGraphicalPort, unsigned int portSourceConnection = 0, unsigned int
portDestinationConnection = 0, QColor color = Qt::black, QGraphicsItem *parent = nullptr);
00039 GraphicalConnection(const GraphicalConnection& orig);
00041 virtual ~GraphicalConnection();
00042 public:
00044 QRectF boundingRect() const override;
00046 QPainterPath shape() const override;
00048 void paint(QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget) override;
00050 Connection* getSource() const;
00052 Connection* getDestination() const;
00054 GraphicalComponentPort* getSourceGraphicalPort();
00056 GraphicalComponentPort* getDestinationGraphicalPort();
00058 GraphicalModelComponent* getSourceGraphicalComponent() const;
00060 GraphicalModelComponent* getDestinationGraphicalComponent() const;

```

```

00062 void updateDimensionsAndPosition();
00064 GraphicalConnection::ConnectionType connectionType() const;
00066 unsigned int getPortSourceConnection() const;
00068 unsigned int getPortDestinationConnection() const;
00070 void setConnectionType(GraphicalConnection::ConnectionType newConnectionType);
00072 QList<QPointF> getPoints() const;
00074 bool usesCurvedStyle() const;
00076 QPainterPath animationPathForImage(qreal imageWidth, qreal imageHeight) const;
00077
00078 protected: // virtual
00079 virtual bool sceneEvent(QEvent *event) override;
00080 //virtual void hoverEnterEvent(QGraphicsSceneHoverEvent * event)
00081 //virtual void hoverLeaveEvent(QGraphicsSceneHoverEvent * event)
00082 //virtual void hoverMoveEvent(QGraphicsSceneHoverEvent * event)
00083 //virtual void inputMethodEvent(QInputMethodEvent * event)
00084 //virtual QVariant inputMethodQuery(Qt::InputMethodQuery query) const
00085 //virtual QVariant itemChange(GraphicsItemChange change, const QVariant & value)
00086 //virtual void keyPressEvent(QKeyEvent * event)
00087 //virtual void keyReleaseEvent(QKeyEvent * event)
00088 //virtual void mouseDoubleClickEvent(QGraphicsSceneMouseEvent * event);
00089 //virtual void mouseMoveEvent(QGraphicsSceneMouseEvent * event);
00090 //virtual void mousePressEvent(QGraphicsSceneMouseEvent * event);
00091 //virtual void mouseReleaseEvent(QGraphicsSceneMouseEvent * event)
00092 private:
00093 bool canRefreshGeometry() const;
00094 QPainterPath connectionPath() const;
00095 GraphicalConnectionStyle::RouteType routeTypeForStyle() const;
00096 qreal _width = 0.0;
00097 qreal _height = 0.0;
00098 QPointF _sourcePointLocal;
00099 QPointF _destinationPointLocal;
00100 unsigned int _margin = TraitsGUI<GConnection>::margin;//2;
00101 unsigned int _selWidth = TraitsGUI<GConnection>::selectionWidth;//8;
00102 ConnectionType _connectionType = ConnectionType::HORIZONTAL;
00103 Connection* _sourceConnection;
00104 Connection* _destinationConnection;
00105 QPointer<GraphicalComponentPort> _sourceGraphicalPort;
00106 QPointer<GraphicalComponentPort> _destinationGraphicalPort;
00107 unsigned int _portSourceConnection;
00108 unsigned int _portDestinationConnection;
00109 QColor _color;
00110 QList<QPointF> _points;
00111 };
00112 #endif // GRAPHICALCONNECTION_H

```

## 16.54 GraphicalConnectionStyle.h

```

00001 #ifndef GRAPHICALCONNECTIONSTYLE_H
00002 #define GRAPHICALCONNECTIONSTYLE_H
00003
00004 #include <QPainterPath>
00005 #include <QPointF>
00006
00007 class GraphicalConnectionStyle {
00008 public:
00009 enum class RouteType {
00010 Horizontal,
00011 Vertical,
00012 Direct,
00013 UserDefined
00014 };
00015
00016 static bool usesModernCurves();
00017 static QPainterPath modelConnectionPath(const QPointF& source,
00018 const QPointF& destination,
00019 RouteType routeType,
00020 bool modernCurves);
00021 static QPainterPath diagramConnectionPath(const QPointF& source,
00022 const QPointF& destination,
00023 bool modernCurves);
00024 static QPointF pointAtProgress(const QPainterPath& path, qreal progress);
00025
00026 private:
00027 GraphicalConnectionStyle() = default;
00028 };
00029
00030 #endif // GRAPHICALCONNECTIONSTYLE_H

```

## 16.55 GraphicalDiagramConnection.h

```

00001 #ifndef GRAPHICALDIAGRAMCONNECTION_H
00002 #define GRAPHICALDIAGRAMCONNECTION_H

```

```

00003
00004 #include <QGraphicsItem>
00005 #include <QGraphicsObject>
00006 #include <QPen>
00007 #include <QBrush>
00008 #include <QPainter>
00009 #include <QPainterPath>
00010 #include <cmath>
00011 #include <QGraphicsLineItem>
00012 #include "GraphicalConnectionStyle.h"
00013 #include "../kernel/simulator/model/ModelComponent.h"
00014 #include "../kernel/simulator/model/ModelDataDefinition.h"
00015 #include "GraphicalModelComponent.h"
00016 #include "GraphicalModelDataDefinition.h"
00017 #include "TraitsGUI.h"
00018 #include "UtilGUI.h"
00019
00020 class GraphicalDiagramConnection : public QGraphicsLineItem {
00021 public:
00022 enum class ConnectionType{
00023 ATTACHED, INTERNAL
00024 };
00025
00026 public:
00027 GraphicalDiagramConnection(QGraphicsItem* dataDefinition, QGraphicsItem* linkedTo, ConnectionType
type);
00028 void refreshGeometry();
00029
00030 QRectF boundingRect() const override;
00031 QPainterPath shape() const override;
00032 void paint(QPainter* painter, const QStyleOptionGraphicsItem* option, QWidget* widget = nullptr)
override;
00033
00034 QGraphicsItem* getDataDefinition();
00035 QGraphicsItem* getLinkedDataDefinition();
00036 ConnectionType getConnectionType();
00037
00038 private:
00039 ConnectionType _type;
00040 QGraphicsItem* _item1;
00041 QGraphicsItem* _item2;
00042 QPainterPath connectionPath() const;
00043 };
00044
00045
00046 #endif // GRAPHICALDIAGRAMCONNECTION_H

```

## 16.56 GraphicalImageAnimation.h

```

00001 #ifndef GRAPHICALIMAGEANIMATION_H
00002 #define GRAPHICALIMAGEANIMATION_H
00003
00004 #include <QGraphicsPixmapItem>
00005 #include <QPixmap>
00006
00007 class GraphicalImageAnimation : public QGraphicsPixmapItem {
00008 public:
00009 // Contrutor
00010 GraphicalImageAnimation(const QPointF startPoint, unsigned int width, unsigned int height, const
QString imageName);
00011 // Destrutor
00012 ~GraphicalImageAnimation();
00013
00014 // Getters
00015 QString getImagePath() const;
00016 QString getImageName() const;
00017 QPointF getStartPoint() const;
00018 unsigned int getWidth() const;
00019 unsigned int getHeight() const;
00020
00021 // Setters
00022 void setImageName(const QString &imageName);
00023 void setStartPoint(const QPointF &startPoint);
00024 void setWidth(unsigned int imageWidth);
00025 void setHeight(unsigned int imageHeight);
00026
00027 // Atualiza informações da imagem
00028 void updateImage();
00029
00030 private:
00031 // Builds an in-memory marker image when the file-based image cannot be loaded.
00032 QPixmap buildFallbackPixmap() const;
00033
00034 // Use a Qt resource base path to avoid working-directory dependent file paths.
00035 QString _resourceBasePath = ":/animations/";

```

```

00036
00037 // Atributos
00038 QString _imagePath;
00039 QPointF _startPoint;
00040 unsigned int _width;
00041 unsigned int _height;
00042 QString _imageName;
00043 };
00044
00045 #endif // GRAPHICALIMAGEANIMATION_H

```

## 16.57 GraphicalModelComponent.h

```

00001 #ifndef GRAPHICALMODELCOMPONENT_H
00002 #define GRAPHICALMODELCOMPONENT_H
00003
00004 /*
00005 * File: ModelComponentGraphicItem.h
00006 * Author: rlcancian
00007 *
00008 * Created on 16 de fevereiro de 2022, 11:41
00009 */
00010
00011 #include <QGraphicsItem>
00012 #include <QGraphicsObject>
00013 #include <QPen>
00014 #include <QBrush>
00015 #include "../kernel/simulator/model/ModelComponent.h"
00016 #include "../kernel/simulator/model/ModelDataDefinition.h"
00017 #include "../kernel/simulator/essentialPlugins/Attribute.h"
00018 #include "../kernel/simulator/essentialPlugins/Counter.h"
00019 #include "../kernel/simulator/PropertyGenesys.h"
00020 #include "../kernel/simulator/Simulator.h"
00021 #include "../plugins/data/Logic/Variable.h"
00022 #include "kernel/simulator/Plugin.h"
00023 #include "plugins/data/DiscreteProcessing/Queue.h"
00024 #include "graphicals/GraphicalImageAnimation.h"
00025 #include "GraphicalComponentPort.h"
00026 #include "GraphicalModelDataDefinition.h"
00027 #include "TraitsGUI.h"
00028
00035 class GraphicalModelComponent : public GraphicalModelDataDefinition {
00036 public:
00038 GraphicalModelComponent(Plugin* plugin, ModelComponent* component, QPointF position, QColor color
00039 = Qt::blue, QGraphicsItem *parent = nullptr);
00040 GraphicalModelComponent(const GraphicalModelComponent& orig);
00042 virtual ~GraphicalModelComponent();
00043 public:
00045 QRectF boundingRect() const override;
00047 void paint(QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget) override;
00049 ModelComponent* getComponent() const;
00051 qreal getHeight() const;
00053 QPointF getOldPosition() const;
00055 void setOldPosition(QPointF oldPosition);
00057 QList<GraphicalComponentPort *> getGraphicalInputPorts() const;
00059 QList<GraphicalComponentPort *> getGraphicalOutputPorts() const;
00061 QColor getColor() const;
00063 void setColor(const QColor& color);
00065 QList<ModelDataDefinition *> *getInternalData() const;
00067 QList<ModelDataDefinition *> *getAttachedData() const;
00069 EntityType* getEntityType() const;
00071 void setEntityType(EntityType *entityType);
00073 unsigned int getOccupiedInputPorts() const;
00075 unsigned int getOccupiedOutputPorts() const;
00077 void setOccupiedInputPorts(unsigned int value);
00079 void setOccupiedOutputPorts(unsigned int value);
00080 private:
00081 QColor myrgba(uint64_t color); // TODO: Should NOT be here, but in UtilGUI.h, but then it
00082 generates multiple definitions error
00083 protected: // virtual
00084 virtual bool sceneEvent(QEvent *event) override;
00085 QVariant itemChange(GraphicsItemChange change, const QVariant& value) override;
00086 GraphicalModelItemRenderContext renderContext() const override;
00087 //virtual void hoverEnterEvent(QGraphicsSceneHoverEvent * event)
00088 //virtual void hoverLeaveEvent(QGraphicsSceneHoverEvent * event)
00089 //virtual void hoverMoveEvent(QGraphicsSceneHoverEvent * event)
00090 //virtual void inputMethodEvent(QInputMethodEvent * event)
00091 //virtual QVariant inputMethodQuery(Qt::InputMethodQuery query) const
00092 //virtual void itemChange(GraphicsItemChange change, const QVariant & value)
00093 //virtual void keyPressEvent(QKeyEvent * event)
00094 //virtual void keyReleaseEvent(QKeyEvent * event)
00095 // virtual void mouseDoubleClickEvent(QGraphicsSceneMouseEvent * event);
00096 // virtual void mouseMoveEvent(QGraphicsSceneMouseEvent * event);
00097 // virtual void mousePressEvent(QGraphicsSceneMouseEvent * event);
00098 //virtual void mouseReleaseEvent(QGraphicsSceneMouseEvent * event)

```

```

00098 protected:
00099 qreal _width = TraitsGUI<GModelComponent>::width; //150;
00100 qreal _height = _width * TraitsGUI<GModelComponent>::heightProportion; //0.67;
00101 unsigned int _margin = TraitsGUI<GModelComponent>::margin; //8;
00102 unsigned int _selWidth = TraitsGUI<GModelComponent>::selectionWidth; //8;
00103 ModelComponent* _component;
00104 QColor _color;
00105 QPointF _oldPosition;
00106 qreal _stretchPosTop = TraitsGUI<GModelComponent>::stretchPos; //0.5;
00107 qreal _stretchPosBottom = TraitsGUI<GModelComponent>::stretchPos; //0.5;
00108 qreal _stretchPosLeft = TraitsGUI<GModelComponent>::stretchPos; //0.5;
00109 qreal _stretchPosRigth = TraitsGUI<GModelComponent>::stretchPos; //0.5;
00110 qreal _stretchRigth = TraitsGUI<GModelComponent>::stretch; //0;
00111 qreal _stretchLeft = TraitsGUI<GModelComponent>::stretch; //0;
00112 qreal _stretchRigthMidle = TraitsGUI<GModelComponent>::stretch; //0;
00113 qreal _stretchLeftMidle = TraitsGUI<GModelComponent>::stretch; //0;
00114 qreal _stretchTop = TraitsGUI<GModelComponent>::stretch; //0;
00115 qreal _stretchBottom = TraitsGUI<GModelComponent>::stretch; //0;
00116 qreal _stretchTopMidle = TraitsGUI<GModelComponent>::stretch; //0;
00117 qreal _stretchBottomMidle = TraitsGUI<GModelComponent>::stretch; //0;
00118 private:
00119 QList<GraphicalComponentPort*> _graphicalInputPorts = QList<GraphicalComponentPort*>();
00120 QList<GraphicalComponentPort*> _graphicalOutputPorts = QList<GraphicalComponentPort*>();
00121 QList<ModelDataDefinition *> _internalData = new QList<ModelDataDefinition*>();
00122 QList<ModelDataDefinition *> _attachedData = new QList<ModelDataDefinition*>();
00123 EntityType* _entityType = nullptr;
00124 unsigned int _ocupiedInputPorts = 0;
00125 unsigned int _ocupiedOutputPorts = 0;
00126
00127 private:
00128 QString _animationImageName = "default.png";
00129 // Map que mapeia o ponteiro da fila do componente para seu índice (caso tiver mais de uma) e
 tamanho (usado para animação)
00130 QMap<Queue *, QPair<unsigned int, unsigned int>*> _mapQueue = new QMap<Queue *, QPair<unsigned int,
 unsigned int>*>();
00131 // Irá conter referência para as imagens usadas para mostrar a fila do componente
00132 QList<QList<GraphicalImageAnimation *>*> _imagesQueue = new QList<QList<GraphicalImageAnimation
 >>();
00133 bool _hasQueue = false;
00134
00135 public:
00136 QString getAnimationImageName();
00137 void setAnimationImageName(QString name);
00138 QMap<Queue *, QPair<unsigned int, unsigned int>*> getMapQueue();
00139 QList<QList<GraphicalImageAnimation *>*> getImagesQueue();
00140
00141 void verifyQueue();
00142 bool hasQueue();
00143 static bool compareQueuesById(const Queue* a, const Queue* b);
00144 unsigned int getIndexQueue(Queue *queue);
00145 unsigned int getSizeQueue(Queue *queue);
00146 void populateMapQueue(QList<Queue *> queues);
00147 bool insertImageQueue(Queue *queue, GraphicalImageAnimation *image);
00148 QList<GraphicalImageAnimation *> removeImageQueue(Queue *queue, unsigned int quantityRemoved);
00149 bool insertImageQueue(GraphicalImageAnimation *image);
00150 GraphicalImageAnimation *removeImageQueue();
00151 void actualizeMapQueue(Queue *queue);
00152 void visivibleImageQueue(bool visivible);
00153 void clearQueues();
00154 };
00155
00156 #endif /* MODELCOMPONENTGRAPHICITEM_H */

```

## 16.58 GraphicalModelDataDefinition.h

```

00001 #ifndef GRAPHICALMODELDATADEFINITION_H
00002 #define GRAPHICALMODELDATADEFINITION_H
00003
00004 /*
00005 * File: ModelDataDefinitionGraphicItem.h
00006 * Author: rlcancian
00007 *
00008 * Created on 16 de fevereiro de 2022, 11:41
00009 */
00010
00011 #include <QGraphicsItem>
00012 #include <QGraphicsObject>
00013 #include <QPainterPath>
00014 #include <QPen>
00015 #include <QBrush>
00016 #include "kernel/simulator/Plugin.h"
00017 #include "graphicals/GraphicalModelItemRenderStrategy.h"
00018 #include "TraitsGUI.h"
00019
00020 class GraphicalModelDataDefinition : public QGraphicsObject {

```

```

00027 public:
00029 GraphicalModelDataDefinition(Plugin* plugin, ModelDataDefinition* element, QPointF position,
 QColor color = Qt::blue, QGraphicsItem *parent = nullptr);
00031 GraphicalModelDataDefinition(const GraphicalModelDataDefinition& orig);
00033 virtual ~GraphicalModelDataDefinition();
00034 public:
00036 QRectF boundingRect() const override;
00038 QPainterPath shape() const override;
00040 void paint(QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget) override;
00042 ModelDataDefinition* getDataDefinition() const;
00044 bool isEditableInPropertyEditor() const;
00046 void setEditableInPropertyEditor(bool editable);
00048 QPointF getOldPosition() const;
00050 void setOldPosition(qreal x, qreal y);
00052 QColor getColor() const;
00054 void setColor(QColor newColor);
00056 qreal getHeight() const;
00057
00058 protected:
00059 QColor myrgba(uint64_t color); // TODO: Should NOT be here, but in UtilGUI.h, but then it
 generates multiple definitions error
00060 virtual GraphicalModelItemRenderContext renderContext() const;
00061 protected: // virtual
00062 virtual bool sceneEvent(QEvent *event) override;
00063 //virtual void hoverEnterEvent(QGraphicsSceneHoverEvent * event)
00064 //virtual void hoverLeaveEvent(QGraphicsSceneHoverEvent * event)
00065 //virtual void hoverMoveEvent(QGraphicsSceneHoverEvent * event)
00066 //virtual void inputMethodEvent(QInputMethodEvent * event)
00067 //virtual QVariant inputMethodQuery(Qt::InputMethodQuery query) const
00068 //virtual QVariant itemChange(GraphicsItemChange change, const QVariant & value)
00069 //virtual void keyPressEvent(QKeyEvent * event)
00070 //virtual void keyReleaseEvent(QKeyEvent * event)
00071 //virtual void mouseDoubleClickEvent(QGraphicsSceneMouseEvent * event);
00072 //virtual void mouseMoveEvent(QGraphicsSceneMouseEvent * event);
00073 //virtual void mousePressEvent(QGraphicsSceneMouseEvent * event);
00074 //virtual void mouseReleaseEvent(QGraphicsSceneMouseEvent * event)
00075 protected:
00076 qreal _width = TraitsGUI<GModelDataDefinition>::width; //150;
00077 qreal _height = _width * TraitsGUI<GModelDataDefinition>::heightProportion; //0.67;
00078 unsigned int _margin = TraitsGUI<GModelDataDefinition>::margin; //8;
00079 unsigned int _selWidth = TraitsGUI<GModelDataDefinition>::selectionWidth; //8;
00080 ModelDataDefinition* _element;
00081 bool _editableInPropertyEditor = false;
00082 QColor _color;
00083 QPointF _oldPosition;
00084 qreal _stretchPosTop = TraitsGUI<GModelDataDefinition>::stretchPos; //0.5;
00085 qreal _stretchPosBottom = TraitsGUI<GModelDataDefinition>::stretchPos; //0.5;
00086 qreal _stretchPosLeft = TraitsGUI<GModelDataDefinition>::stretchPos; //0.5;
00087 qreal _stretchPosRigth = TraitsGUI<GModelDataDefinition>::stretchPos; //0.5;
00088 qreal _stretchRigth = TraitsGUI<GModelDataDefinition>::stretch; //0;
00089 qreal _stretchLeft = TraitsGUI<GModelDataDefinition>::stretch; //0;
00090 qreal _stretchRigthMidle = TraitsGUI<GModelDataDefinition>::stretch; //0;
00091 qreal _stretchLeftMidle = TraitsGUI<GModelDataDefinition>::stretch; //0;
00092 qreal _stretchTop = TraitsGUI<GModelDataDefinition>::stretch; //0;
00093 qreal _stretchBottom = TraitsGUI<GModelDataDefinition>::stretch; //0;
00094 qreal _stretchTopMidle = TraitsGUI<GModelDataDefinition>::stretch; //0;
00095 qreal _stretchBottomMidle = TraitsGUI<GModelDataDefinition>::stretch; //0;
00096 private:
00097 };
00098
00099 #endif /* GRAPHICALMODELDATADEFINITION_H */

```

## 16.59 GraphicalModelItemRenderStrategy.h

```

00001 #ifndef GRAPHICALMODELITEMRENDERSTRATEGY_H
00002 #define GRAPHICALMODELITEMRENDERSTRATEGY_H
00003
00004 #include <cstdint>
00005
00006 #include <QColor>
00007 #include <QPainterPath>
00008 #include <QRectF>
00009 #include <QString>
00010
00011 class QPainter;
00012
00013 struct GraphicalModelItemRenderContext {
00014 enum class ItemKind {
00015 DataDefinition,
00016 Component
00017 };
00018
00019 ItemKind kind = ItemKind::DataDefinition;
00020 QRectF bounds;
00021 QColor fillColor;

```

```

00022 QString primaryText;
00023 QString secondaryText;
00024 QString tertiaryText;
00025 QString semanticClassName;
00026 QString semanticSubtype;
00027 bool selected = false;
00028 bool breakpoint = false;
00029
00030 qreal width = 0.0;
00031 qreal height = 0.0;
00032 int penWidth = 1;
00033 int raise = 0;
00034 int margin = 0;
00035 int selectionWidth = 0;
00036
00037 qreal stretchPosTop = 0.5;
00038 qreal stretchPosBottom = 0.5;
00039 qreal stretchPosLeft = 0.5;
00040 qreal stretchPosRight = 0.5;
00041 qreal stretchRight = 0.0;
00042 qreal stretchLeft = 0.0;
00043 qreal stretchRightMiddle = 0.0;
00044 qreal stretchLeftMiddle = 0.0;
00045 qreal stretchTop = 0.0;
00046 qreal stretchBottom = 0.0;
00047 qreal stretchTopMiddle = 0.0;
00048 qreal stretchBottomMiddle = 0.0;
00049
00050 uint64_t borderColor = 0;
00051 uint64_t raisedColor = 0;
00052 uint64_t sunkenColor = 0;
00053 uint64_t textColor = 0;
00054 uint64_t textShadowColor = 0;
00055 uint64_t selectionColor = 0;
00056 uint64_t breakpointColor = 0;
00057 };
00058
00059 class GraphicalModelItemRenderStrategy {
00060 public:
00061 virtual ~GraphicalModelItemRenderStrategy() = default;
00062
00063 virtual const char* name() const = 0;
00064 virtual void paint(QPainter* painter, const GraphicalModelItemRenderContext& context) const = 0;
00065 virtual QPainterPath shape(const GraphicalModelItemRenderContext& context) const = 0;
00066 };
00067
00068 class GraphicalModelItemRenderer {
00069 public:
00070 static void paint(QPainter* painter, const GraphicalModelItemRenderContext& context);
00071 static QPainterPath shape(const GraphicalModelItemRenderContext& context);
00072 static const GraphicalModelItemRenderStrategy& currentStrategy();
00073 };
00074
00075 #endif // GRAPHICALMODELITEMRENDERSTRATEGY_H

```

## 16.60 ModelGraphicsScene.h

```

00001 /*
00002 * The MIT License
00003 *
00004 * Copyright 2022 rlcancian.
00005 *
00006 * Permission is hereby granted, free of charge, to any person obtaining a copy
00007 * of this software and associated documentation files (the "Software"), to deal
00008 * in the Software without restriction, including without limitation the rights
00009 * to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00010 * copies of the Software, and to permit persons to whom the Software is
00011 * furnished to do so, subject to the following conditions:
00012 *
00013 * The above copyright notice and this permission notice shall be included in
00014 * all copies or substantial portions of the Software.
00015 *
00016 * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00017 * IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00018 * FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00019 * AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00020 * LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00021 * OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
00022 * THE SOFTWARE.
00023 */
00024
00025 /*
00026 * File: ModelGraphicsScene.h
00027 * Author: rlcancian
00028 *

```

```

00029 * Created on 16 de fevereiro de 2022, 09:52
00030 */
00031
00032 #ifndef MODELGRAPHICSSCENE_H
00033 #define MODELGRAPHICSSCENE_H
00034
00035 #include <QGraphicsScene>
00036 #include <QGraphicsProxyWidget>
00037 #include <QGraphicsRectItem>
00038 #include <QTreeWidgetItem>
00039 #include <QUndoStack>
00040 #include <QAction>
00041 #include <QEventLoop>
00042 #include "graphicals/GraphicalModelComponent.h"
00043 #include "graphicals/GraphicalComponentPort.h"
00044 #include "graphicals/GraphicalDiagramConnection.h"
00045 #include "propertyeditor/DataComponentProperty.h"
00046 #include "propertyeditor/DataComponentEditor.h"
00047 #include "propertyeditor/ComboBoxEnum.h"
00048 #include "../kernel/simulator/model/ModelComponent.h"
00049 #include "kernel/simulator/Simulator.h"
00050 #include "kernel/simulator/Plugin.h"
00051 #include "animations/AnimationTransition.h"
00052 #include "animations/AnimationVariable.h"
00053 #include "animations/AnimationCounter.h"
00054 #include "animations/AnimationTimer.h"
00055 #include "animations/AnimationPlaceholder.h"
00056 #include "../kernel/simulator/essentialPlugins/Counter.h"
00057 #include "../kernel/simulator/essentialPlugins/Attribute.h"
00058 #include "kernel/simulator/PropertyGenesys.h"
00059 #include "../kernel/simulator/model/ModelDataDefinition.h"
00060 #include "../plugins/data/Logic/Variable.h"
00061
00062 class GraphicalModelEvent {
00063 public:
00064
00065 enum class EventType : int {
00066 CREATE = 1,
00067 REMOVE = 2,
00068 EDIT = 3,
00069 CLONE = 4,
00070 OTHER = 9
00071 };
00072
00073 enum class EventObjectType : int {
00074 COMPONENT = 1,
00075 DATADEFINITION = 2,
00076 CONNECTION = 3,
00077 DRAWING = 4,
00078 ANIMATION = 5,
00079 PLOT = 6,
00080 OTHER = 9
00081 };
00082
00083 public:
00084
00085 GraphicalModelEvent(GraphicalModelEvent::EventType eventType, GraphicalModelEvent::EventObjectType
eventObjectType, QGraphicsItem* item) {
00086 this->eventType = eventType;
00087 this->eventObjectType = eventObjectType;
00088 this->item = item;
00089 }
00090
00091 GraphicalModelEvent::EventType eventType;
00092 GraphicalModelEvent::EventObjectType eventObjectType;
00093 QGraphicsItem* item;
00094 };
00095
00096 class ModelGraphicsScene : public QGraphicsScene {
00097 public:
00098 ModelGraphicsScene(qreal x, qreal y, qreal width, qreal height, QObject *parent = nullptr);
00099 // Disable copy construction to keep scene ownership state unique.
00100 ModelGraphicsScene(const ModelGraphicsScene& orig) = delete;
00101 // Disable copy assignment to prevent shallow copies of GUI-owned resources.
00102 ModelGraphicsScene& operator=(const ModelGraphicsScene& other) = delete;
00103 virtual ~ModelGraphicsScene();
00104 public: // editing graphic model
00105 enum DrawingMode{
00106 NONE, LINE, TEXT, RECTANGLE, ELLIPSE, POLYGON, POLYGON_POINTS, POLYGON_FINISHED,
00107 COUNTER, VARIABLE, TIMER,
00108 ATTRIBUTE, ENTITY, EVENT, EXPRESSION, PLOT, QUEUE_PLACEHOLDER, RESOURCE, STATION, STATISTICS
00109 };
00110
00111 GraphicalModelComponent* addGraphicalModelComponent(Plugin* plugin, ModelComponent* component,
QPointF position, QColor color = Qt::blue, bool notify = false, GraphicalModelComponent*
autoConnectSource = nullptr);
00112
00113 GraphicalConnection* addGraphicalConnection(GraphicalComponentPort* sourcePort,
GraphicalComponentPort* destinationPort, unsigned int portSourceConnection, unsigned int
portDestinationConnection, bool notify = false);
00114
00115 GraphicalModelDataDefinition* addGraphicalModelDataDefinition(Plugin* plugin, ModelDataDefinition*

```



```

 element, QPointF position, QColor color = Qt::blue);
00126 GraphicalDiagramConnection* addGraphicalDiagramConnection(QGraphicsItem* dataDefinition,
 QGraphicsItem* linkedTo, GraphicalDiagramConnection::ConnectionType type);
00128 void initializeAnimationDrawing(QGraphicsSceneMouseEvent *mouseEvent);
00130 void continueAnimationDrawing(QGraphicsSceneMouseEvent *mouseEvent);
00132 void finishAnimationDrawing(QGraphicsSceneMouseEvent *mouseEvent);
00134 void startTextEditing();
00136 void removeComponent(GraphicalModelComponent* gmc, bool notify = false);
00138 void clearConnectionsComponent(GraphicalModelComponent* gmc);
00140 void clearInputConnectionsComponent(GraphicalModelComponent* graphicalComponent);
00142 void clearOutputConnectionsComponent(GraphicalModelComponent* graphicalComponent);
00144 void clearPorts(GraphicalConnection* connection, GraphicalModelComponent *source,
 GraphicalModelComponent *destination);
00146 void connectComponents(GraphicalConnection* connection, GraphicalModelComponent *source = nullptr,
 GraphicalModelComponent *destination = nullptr, bool notify = false);
00148 bool connectSource(GraphicalConnection* connection, GraphicalModelComponent *source = nullptr);
00150 bool connectDestination(GraphicalConnection* connection, GraphicalModelComponent *destination =
 nullptr);
00152 void redoConnections(GraphicalModelComponent *graphicalComponent, QList<GraphicalConnection *>
 *inputConnections, QList<GraphicalConnection *> *outputConnections);
00154 void removeComponentInModel(GraphicalModelComponent* gmc);
00156 void insertComponent(GraphicalModelComponent* gmc, QList<GraphicalConnection *> *inputConnections,
 QList<GraphicalConnection *> *outputConnections, bool addGMC = true, bool addAllGMC = true, bool
 notify = false);
00158 void removeGraphicalConnection(GraphicalConnection* graphicalConnection, GraphicalModelComponent
 *source, GraphicalModelComponent *destination, bool notify = false);
00160 void removeConnectionInModel(GraphicalConnection* graphicalConnection, GraphicalModelComponent
 *source);
00162 void addGeometry(QPointF endPoint, bool moving);
00164 void addDrawing(QGraphicsItem * item, bool notify = false);
00166 bool addDrawingGeometry(QGraphicsItem * item);
00168 bool addDrawingAnimation(QGraphicsItem * item);
00170 void removeGraphicalModelDataDefinition(GraphicalModelDataDefinition* gmdd);
00172 void detachGraphicalModelDataDefinition(GraphicalModelDataDefinition* gmdd);
00174 void restoreGraphicalModelDataDefinition(GraphicalModelDataDefinition* gmdd);
00176 void removeGraphicalDiagramConnection(GraphicalDiagramConnection* connection);
00178 void clearGraphicalModelDataDefinitions();
00180 void clearGraphicalDiagramConnections();
00182 void sanitizeGraphicalDataDefinitionsBookkeeping();
00184 void setDiagramLayerState(bool diagramCreated, bool visible);
00185 // Return only items that can be directly manipulated by user edit commands.
00187 QList<QGraphicsItem*> userOperableItems(const QList<QGraphicsItem*>& items) const;
00188 // Filter out non-deletable items from user-triggered delete flows.
00190 QList<QGraphicsItem*> userDeletableItems(const QList<QGraphicsItem*>& items) const;
00191 // Keep internal data-definition initial grouping opt-in during model rebuild only.
00193 void ensureInitialInternalDataDefinitionGrouping(GraphicalModelDataDefinition* dataDefinition,
 GraphicalModelComponent* component);
00194 // Allow serializer to disable automatic initial grouping while persisted GUI state is being
 restored.
00196 void setPersistedGuiRestoreInProgress(bool restoring);
00198 bool isPersistedGuiRestoreInProgress() const;
00200 void removeDrawing(QGraphicsItem * item, bool notify = false);
00201 bool removeDrawingGeometry(QGraphicsItem * item);
00202 bool removeDrawingAnimation(QGraphicsItem * item);
00204 void removeGroup(QGraphicsItemGroup* group, bool notify = false);
00206 void clearGraphicalModelComponents();
00208 void clearGraphicalModelConnections();
00210 void groupComponents(bool notify = false); // tenta agrupar (verifica se sao
 ModelGraphicalComponents)
00212 void groupModelComponents(QList<GraphicalModelComponent *> *graphicalComponents,
 QGraphicsItemGroup *group); // agrupa componentes
00214 void ungroupComponents(bool notify = false);
00216 void ungroupModelComponents(QGraphicsItemGroup *group);
00218 void notifyGraphicalModelChange(GraphicalModelEvent::EventType eventType,
 GraphicalModelEvent::EventObjectType eventObjectType, QGraphicsItem *item);
00219 // Return model component items by value to avoid heap ownership transfer.
00221 QList<GraphicalModelComponent*> graphicalModelComponentItems();
00223 GraphicalModelComponent* findGraphicalModelComponent(Util::identification id);
00225 GraphicalModelDataDefinition* findGraphicalModelDataDefinition(ModelDataDefinition*
 dataDefinition);
00227 GraphicalModelComponent* resolveSourceComponent(GraphicalConnection* connection) const;
00229 GraphicalModelComponent* resolveDestinationComponent(GraphicalConnection* connection) const;
00230 public:
00231 struct GRID {
00232 unsigned int interval;
00233 QPen pen;
00234 std::list<QGraphicsLineItem *> *lines;
00235 bool visible;
00236 void clear();
00237 };
00239 GRID *grid();
00241 void showGrid();
00242 // Aplica o estado visual do grid de forma determinística sem alternância implícita.
00244 void setGridVisible(bool visible);
00245 // Informa o estado visual atual do grid para sincronização com QAction.
00247 bool isGridVisible() const;
00249 void snapItemsToGrid();

```

```

00251 void actualizeDiagramArrows();
00253 void showDiagrams();
00255 void hideDiagrams();
00257 bool existDiagram();
00259 bool visibleDiagram();
00261 void destroyDiagram();
00263 void createDiagrams();
00265 QUndoStack* getUndoStack();
00267 Simulator* getSimulator();
00269 void setUndoStack(QUndoStack* undo);
00271 void beginConnection();
00273 void setSimulator(Simulator *simulator);
00275 void setPropertyEditor(PropertyEditorGenesys *propEditor);
00277 void setPropertyList(std::map<SimulationControl*, DataComponentProperty*>* propList);
00279 void setPropertyEditorUI(std::map<SimulationControl*, DataComponentEditor*>* propEditorUI);
00281 void setComboBox(std::map<SimulationControl*, ComboBoxEnum*>* propCombo);
00283 void setObjectBeingDragged(QTreeWidgetItem* objectBeingDragged);
00285 void setRestoringPersistedGuiLayout(bool restoring);
00287 bool isRestoringPersistedGuiLayout() const;
00289 void setParentWidget(QWidget *parentWidget);
00291 unsigned short connectingStep() const;
00293 void setConnectingStep(unsigned short connectingStep);
00295 void setSnapToGrid(bool activated);
00297 bool getSnapToGrid();
00299 void arranjeModels(int direction);
00301 void setDrawingMode(DrawingMode drawingMode);
00303 void clearDrawingMode();
00305 void setGraphicalComponentPort(GraphicalComponentPort * in);
00307 DrawingMode getDrawingMode();
00309 void setAction(QAction* action);
00311 QList<GraphicalModelComponent*> *getAllComponents();
00313 QList<GraphicalConnection*> *getAllConnections();
00315 QList<GraphicalModelDataDefinition*> *getAllDataDefinitions();
00317 QList<GraphicalDiagramConnection*> *getAllGraphicalDiagramsConnections();
00319 QMap<QGraphicsItemGroup *, QList<GraphicalModelComponent *> *getListComponentGroup();
00321 void insertComponentGroup(QGraphicsItemGroup *group, QList<GraphicalModelComponent *>
componentsGroup);
00323 void insertOldPositionItem(QGraphicsItem *item, QPointF position);
00325 QPointF getOldPositionItem(QGraphicsItem *item) const;
00327 QList<AnimationTransition *> *getAnimationsTransition();
00329 QList<AnimationCounter *> *getAnimationsCounter();
00331 QList<AnimationVariable *> *getAnimationsVariable();
00333 QList<AnimationTimer *> *getAnimationsTimer();
00335 QList<AnimationPlaceholder *> *getAnimationsPlaceholder();
00337 QMap<Event *, QList<AnimationTransition *> *> *getAnimationPaused();
00339 bool checkIgnoreEvent();
00341 void clearAnimations();
00343 void clearAnimationsTransition();
00345 void clearAnimationsCounter();
00347 void clearAnimationsVariable();
00349 void clearAnimationsTimer();
00351 void clearAnimationsPlaceholder();
00353 void clearAnimationsQueue();
00355 void animatePlot();
00357 QList<QString> *getImagesAnimation();
00359 void drawingCounter();
00361 void drawingVariable();
00363 void drawingTimer();
00365 void drawingAttribute();
00367 void drawingEntity();
00369 void drawingEvent();
00371 void drawingExpression();
00373 void drawingPlot();
00375 void drawingQueue();
00377 void drawingResource();
00379 void drawingStation();
00381 void drawingStatistics();
00383 void clearAnimationsValues();
00385 void setCounters();
00387 void setVariables();
00388 // TODO: Funções abaixo são usadas para reaver os dataDefinitions do componente nos
dataDefinitions do modelo dos componentes deletados, "checados" e reinseridos (Control Z de um delete,
por exemplo).
00389 // O kernel não trata este caso, ale acusa erro, pois não encontra os dataDefinitions do
componente nos dataDefinitions do modelo, pois ele os remove como "órfãos" e não os reinsere quando
voltados ao modelo.
00391 void insertRestoredDataDefinitions(bool loaded);
00393 void saveDataDefinitions();
00394 // ----- //
00396 void requestGraphicalDataDefinitionsSync();
00398 void scheduleGraphicalDataDefinitionsSync();
00400 bool isGraphicalDataDefinitionsSyncInProgress() const;
00402 void setShowStatisticsDataDefinitions(bool show);
00404 bool showStatisticsDataDefinitions() const;
00406 void setShowEditableDataDefinitions(bool show);
00408 bool showEditableDataDefinitions() const;
00410 void setShowSharedDataDefinitions(bool show);

```

```

00412 bool showSharedDataDefinitions() const;
00414 void setShowRecursiveDataDefinitions(bool show);
00416 bool showRecursiveDataDefinitions() const;
00424 void setModelLevelFilter(unsigned int modelLevel);
00426 void clearModelLevelFilter();
00428 bool hasModelLevelFilter() const;
00430 unsigned int modelLevelFilter() const;
00432 void setShowInternalDataDefinitions(bool show);
00434 bool showInternalDataDefinitions() const;
00436 void setShowAttachedDataDefinitions(bool show);
00438 bool showAttachedDataDefinitions() const;
00440 void setConnectionGeometryUpdatesBlocked(bool blocked);
00442 bool areConnectionGeometryUpdatesBlocked() const;
00443
00444 public:
00446 QList<QGraphicsItem*>*getGraphicalModelDataDefinitions() const;
00448 QList<QGraphicsItem*>*getGraphicalModelComponents() const;
00450 QList<QGraphicsItem*>*getGraphicalConnections() const;
00452 QList<QGraphicsItem*>*getGraphicalGeometries() const;
00454 QList<QGraphicsItem*>*getGraphicalAnimations() const;
00456 QList<QGraphicsItem*>*getGraphicalEntities() const;
00458 QList<QGraphicsItem*>*getGraphicalDiagramsConnections() const;
00460 QList<QGraphicsItemGroup*>*getGraphicalGroups() const;
00461
00462 public:
00464 void animateTransition(ModelComponent *source, ModelComponent *destination, bool viewSimulation,
Event *event);
00466 void runAnimateTransition(AnimationTransition *animationTransition, Event *event, bool restart =
false);
00468 void handleAnimationStateChanged(QAbstractAnimation::State newState, QEventLoop* loop, Event*
event, AnimationTransition* animationTransition);
00470 void animateQueueInsert(ModelComponent *component, bool visivable);
00472 void animateQueueRemove(ModelComponent *component);
00474 void animateCounter();
00476 void animateVariable(Entity* entity = nullptr);
00478 void animateTimer(double time);
00479
00480 protected: // virtual functions
00481 virtual void contextMenuEvent(QGraphicsSceneContextMenuEvent *contextMenuEvent);
00482 virtual void dragEnterEvent(QGraphicsSceneDragDropEvent *event);
00483 virtual void dragLeaveEvent(QGraphicsSceneDragDropEvent *event);
00484 virtual void dragMoveEvent(QGraphicsSceneDragDropEvent *event);
00485 //virtual void drawBackground(QPainter *painter, const QRectF &rect);
00486 //virtual void drawForeground(QPainter *painter, const QRectF &rect);
00487 virtual void dropEvent(QGraphicsSceneDragDropEvent *event);
00488 virtual void focusInEvent(QFocusEvent *focusEvent);
00489 virtual void focusOutEvent(QFocusEvent *focusEvent);
00490 //virtual void helpEvent(QGraphicsSceneHelpEvent *helpEvent);
00491 //virtual void inputMethodEvent(QInputMethodEvent *event);
00492 virtual void keyPressEvent(QKeyEvent *keyEvent);
00493 virtual void keyReleaseEvent(QKeyEvent *keyEvent);
00494 virtual void mouseDoubleClickEvent(QGraphicsSceneMouseEvent *mouseEvent);
00495 virtual void mouseMoveEvent(QGraphicsSceneMouseEvent *mouseEvent);
00496 virtual void mousePressEvent(QGraphicsSceneMouseEvent *mouseEvent);
00497 virtual void mouseReleaseEvent(QGraphicsSceneMouseEvent *mouseEvent);
00498 virtual void wheelEvent(QGraphicsSceneWheelEvent *wheelEvent);
00499
00500 private:
00501 bool tryCreateConnection(GraphicalComponentPort* source, GraphicalComponentPort* destination, bool
notify = true);
00502 GraphicalComponentPort* firstAvailableOutputPort(GraphicalModelComponent* component) const;
00503 GraphicalComponentPort* firstInputPort(GraphicalModelComponent* component) const;
00504 void resetConnectingState();
00505 private:
00506 GRID _grid;
00507 Simulator* _simulator = nullptr;
00508 PropertyEditorGenesys* _propertyEditor = nullptr;
00509 std::map<SimulationControl*, DataComponentProperty*>* _propertyList = nullptr;
00510 std::map<SimulationControl*, DataComponentEditor*>* _propertyEditorUI = nullptr;
00511 std::map<SimulationControl*, ComboBoxEnum*>* _propertyBox = nullptr;
00512 QTreeWidgetItem* _objectBeingDragged = nullptr;
00513 // Initialize the parent widget pointer to a known null state.
00514 QWidget* _parentWidget = nullptr;
00515 QList<GraphicalModelComponent*> _allGraphicalModelComponents;
00516 QList<GraphicalConnection*> _allGraphicalConnections;
00517 QList<GraphicalModelDataDefinition*> _allGraphicalModelDataDefinitions;
00518 QList<GraphicalDiagramConnection*> _allGraphicalDiagramConnections;
00519 QUndoStack *_undoStack = nullptr;
00520 QMap<QGraphicsItemGroup *, QList<GraphicalModelComponent *> > _listComponentsGroup;
00521 QMap<QGraphicsItem *, QPointF> _oldPositionsItems;
00522 QList<Counter *> *_counters = new QList<Counter *>();
00523 QList<ModelDataDefinition *> *_variables = new QList<ModelDataDefinition *>();
00524 private:
00525 DrawingMode _drawingMode = NONE;
00526 QGraphicsRectItem* _currentRectangle = nullptr;
00527 QGraphicsLineItem* _currentLine = nullptr;
00528 QGraphicsPolygonItem* _currentPolygon = nullptr;

```

```

00529 QGraphicsEllipseItem* _currentEllipse = nullptr;
00530 QPolygonF _currentPolygonPoints;
00531 QPointF _drawingStartPoint;
00532 QAction* _currentAction = nullptr;
00533 bool _drawing = false;
00534 bool _diagram = false;
00535 bool _visibleDiagram = false;
00536 bool _persistedGuiRestoreInProgress = false;
00537 unsigned short _connectingStep = 0; //0:nothing, 1:waiting click on source or destination, 2:
 click on source, 3: click on destination
00538 bool _controlIsPressed = false;
00539 bool _shiftSelectionInProgress = false;
00540 QGraphicsItem* _shiftClickedSelectableItem = nullptr;
00541 QList<QGraphicsItem*> _shiftSelectionBeforeClick;
00542 bool _snapToGrid = false;
00543 // Initialize the source port pointer before connection drawing starts.
00544 GraphicalComponentPort* _sourceGraphicalComponentPort = nullptr;
00545 // Initialize the destination port pointer before connection drawing starts.
00546 GraphicalComponentPort* _destinationGraphicalComponentPort = nullptr;
00547 // Initialize the current counter drawing pointer to avoid indeterminate access.
00548 AnimationCounter* _currentCounter = nullptr;
00549 // Initialize the current variable drawing pointer to avoid indeterminate access.
00550 AnimationVariable* _currentVariable = nullptr;
00551 // Initialize the current timer drawing pointer to avoid indeterminate access.
00552 AnimationTimer* _currentTimer = nullptr;
00553 AnimationPlaceholder* _currentPlaceholderAnimation = nullptr;
00554 QMap<Event*, QList<AnimationTransition*>*> _animationPaused = new QMap<Event*,
 QList<AnimationTransition*>*>();
00555
00556 private:
00557 QList<QString>* _imagesAnimation = new QList<QString>;
00558 QList<AnimationTransition*>* _animationsTransition = new QList<AnimationTransition*>();
00559 QList<AnimationCounter*>* _animationsCounter = new QList<AnimationCounter*>();
00560 QList<AnimationVariable*>* _animationsVariable = new QList<AnimationVariable*>();
00561 QList<AnimationTimer*>* _animationsTimer = new QList<AnimationTimer*>();
00562 QList<AnimationPlaceholder*>* _animationsPlaceholder = new QList<AnimationPlaceholder*>();
00563 private:
00564 // IMPORTANT. MUST BE CONSISTENT WITH SIMULATOR->MODEL
00565 QList<QGraphicsItem*>* _graphicalModelComponents = new QList<QGraphicsItem*>();
00566 QList<QGraphicsItem*>* _graphicalModelDataDefinitions = new QList<QGraphicsItem*>();
00567 QList<QGraphicsItem*>* _graphicalConnections = new QList<QGraphicsItem*>();
00568 QList<QGraphicsItem*>* _graphicalDiagramConnections = new QList<QGraphicsItem*>();
00569 QList<QGraphicsItem*>* _graphicalAssociations = new QList<QGraphicsItem*>();
00570 QList<QGraphicsItem*>* _graphicalGeometries = new QList<QGraphicsItem*>();
00571 QList<QGraphicsItem*>* _graphicalAnimations = new QList<QGraphicsItem*>();
00572 QList<QGraphicsItem*>* _graphicalEntities = new QList<QGraphicsItem*>();
00573 QList<QGraphicsItemGroup*>* _graphicalGroups = new QList<QGraphicsItemGroup*>();
00574 bool _restoringPersistedGuiLayout = false;
00575 bool _graphicalDataDefinitionsSyncPending = false;
00576 bool _graphicalDataDefinitionsSyncInProgress = false;
00577 bool _graphicalDataDefinitionsSyncDeferredDuringRestore = false;
00578 bool _connectionGeometryUpdatesBlocked = false;
00579 bool _showStatisticsDataDefinitions = true;
00580 bool _showEditableDataDefinitions = true;
00581 bool _showSharedDataDefinitions = true;
00582 bool _showRecursiveDataDefinitions = true;
00583 // Rendering scope used by root/submodel tabs. The filter is intentionally scene-local so
00584 // static graphical synchronization calls can keep the same scope as the active view.
00585 bool _hasModelLevelFilter = false;
00586 unsigned int _modelLevelFilter = 0;
00587 };
00588
00589 #endif /* MODELGRAPHICSSCENE_H */

```

## 16.61 ModelGraphicsView.h

```

00001 /*
00002 * The MIT License
00003 *
00004 * Copyright 2022 rlcancian.
00005 *
00006 * Permission is hereby granted, free of charge, to any person obtaining a copy
00007 * of this software and associated documentation files (the "Software"), to deal
00008 * in the Software without restriction, including without limitation the rights
00009 * to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00010 * copies of the Software, and to permit persons to whom the Software is
00011 * furnished to do so, subject to the following conditions:
00012 *
00013 * The above copyright notice and this permission notice shall be included in
00014 * all copies or substantial portions of the Software.
00015 *
00016 * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00017 * IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00018 * FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00019 * AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER

```

```

00020 * LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00021 * OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
00022 * THE SOFTWARE.
00023 */
00024
00025 /*
00026 * File: QModelGraphicView.h
00027 * Author: rlcancian
00028 *
00029 * Created on 15 de fevereiro de 2022, 21:12
00030 */
00031
00032 #ifndef QMODELGRAPHICVIEW_H
00033 #define QMODELGRAPHICVIEW_H
00034
00035 #include <QGraphicsView>
00036 #include <Qt>
00037 #include <QColor>
00038 #include <QStyle>
00039 #include <QGraphicsSceneMouseEvent>
00040 #include <QContextMenuEvent>
00041 #include <QPainter>
00042 #include "graphicals/ModelGraphicsScene.h"
00043 #include "propertyeditor/DataComponentProperty.h"
00044 #include "propertyeditor/DataComponentEditor.h"
00045 #include "propertyeditor/ComboBoxEnum.h"
00046 #include "../kernel/simulator/model/ModelComponent.h"
00047 #include "kernel/simulator/Simulator.h"
00048 #include "kernel/simulator/PropertyGenesys.h"
00049 // #include "kernel/simulator/Plugin.h"
00050
00051 class ModelGraphicsView : public QGraphicsView {
00052 public:
00060 ModelGraphicsView(QWidget *parent = nullptr);
00062 ModelGraphicsView(const ModelGraphicsView& orig);
00064 virtual ~ModelGraphicsView();
00065 public: // editing graphic model
00066 // TODO: AddGraphicalModelComponent should be only on scene
00067 //GraphicalModelComponent* addGraphicalModelComponent(Plugin* plugin, ModelComponent* component,
00068 QPointF position);
00069 //bool removeGraphicalModelComponent(GraphicalModelComponent* gmc);
00069 //bool addGraphicalConnection(GraphicalComponentPort* sourcePort, GraphicalComponentPort*
00070 destinationPort);
00070 //bool removeGraphicalConnection(GraphicalConnection* gc);
00071 //bool addDrawing();
00072 //bool removeDrawing();
00073 //bool addAnimation();
00074 //bool removeAnimation();
00076 ModelGraphicsScene* getScene();
00077 public:
00079 void showGrid();
00081 void clear();
00083 void beginConnection();
00085 void selectModelComponent(ModelComponent* component);
00087 void setSimulator(Simulator* simulator);
00089 void setPropertyEditor(PropertyEditorGenesys* propEditor);
00091 void setPropertyList(std::map<SimulationControl*, DataComponentProperty*>* propList);
00093 void setPropertyEditorUI(std::map<SimulationControl*, DataComponentEditor*>* propEditorUI);
00095 void setComboBox(std::map<SimulationControl*, ComboBoxEnum*>* propBox);
00097 void setEnabled(bool enabled);
00099 QList<QGraphicsItem*> selectedItems();
00100 public: // events and notifications
00101
00102 template<typename Class> void setSceneMouseEventHandler(Class * object, void
00103 (Class::*function)(QGraphicsSceneMouseEvent*)) {
00104 sceneMouseEventHandlerMethod handlerMethod = std::bind(function, object,
00105 std::placeholders::_1);
00106 this->_sceneMouseEventHandler = handlerMethod;
00107 }
00107 template<typename Class> void setGraphicalModelEventHandler(Class * object, void
00108 (Class::*function)(const GraphicalModelEvent&)) {
00109 sceneGraphicalModelEventHandlerMethod handlerMethod = std::bind(function, object,
00110 std::placeholders::_1);
00111 this->_sceneGraphicalModelEventHandler = handlerMethod;
00112 }
00112 template <typename Class> void setSceneWheelInEventHandler(Class *object, void
00113 (Class::*function)()) {
00114 sceneWheelEventHandlerMethod handlerMethod = std::bind(function, object);
00115 this->_sceneWheelInEventHandler = handlerMethod;
00116 }
00117 template <typename Class> void setSceneWheelOutEventHandler(Class *object, void
00118 (Class::*function)()) {
00119 sceneWheelEventHandlerMethod handlerMethod = std::bind(function, object);
00120 this->_sceneWheelOutEventHandler = handlerMethod;
00121 }
00122 };

```

```

00120 }
00121
00122 template <typename Class> void setContextMenuEventHandler(Class* object, void
(Class::*(function)) (QContextMenuEvent*)) {
00123 // Keep context-menu construction outside the view so menus can reuse MainWindow actions.
00124 contextMenuEventHandlerMethod handlerMethod = std::bind(function, object,
std::placeholders::_1);
00125 this->_contextMenuEventHandler = handlerMethod;
00126 }
00127
00129 void notifySceneMouseEventHandler(QGraphicsSceneMouseEvent* mouseEvent);
00130 void notifySceneWheelInEventHandler();
00131 void notifySceneWheelOutEventHandler();
00133 void notifySceneGraphicalModelEventHandler(const GraphicalModelEvent& modelGraphicsEvent);
00135 void setCanNotifyGraphicalModelEventHandlers(bool can);
00137 void clearEventHandlers();
00139 void setParentWidget(QWidget *parentWidget);
00141 // Enables or disables ruler rendering over the graphics viewport.
00142 void setRuleVisible(bool visible);
00144 // Informs if ruler rendering is active for action synchronization.
00145 bool isRuleVisible() const;
00147 // Enables or disables guide line rendering over the graphics viewport.
00148 void setGuidesVisible(bool visible);
00150 // Informs if guide line rendering is active for action synchronization.
00151 bool isGuidesVisible() const;
00153 protected:// slots:
00154 void changed(const QList<QRectF> ®ion);
00155 void focusItemChanged(QGraphicsItem *newFocusItem, QGraphicsItem *oldFocusItem, Qt::FocusReason
reason);
00157 void sceneRectChanged(const QRectF &rect);
00158 void selectionChanged();
00159 protected: // virtual functions
00160 virtual void contextMenuEvent(QContextMenuEvent *event) override;
00161 virtual void dragEnterEvent(QDragEnterEvent *event) override;
00162 //virtual void dragLeaveEvent(QDragLeaveEvent *event) override;
00163 //virtual void dragMoveEvent(QDragMoveEvent *event) override;
00164 //virtual void dropEvent(QDropEvent *event) override;
00165 //virtual bool event(QEvent *event) override;
00166 //virtual void focusInEvent(QFocusEvent *event) override;
00167 //virtual bool focusNextPrevChild(bool next) override;
00168 //virtual void focusOutEvent(QFocusEvent *event) override;
00169 //virtual void inputMethodEvent(QInputMethodEvent *event) override;
00170 virtual void keyPressEvent(QKeyEvent *event) override;
00171 virtual void keyReleaseEvent(QKeyEvent *event) override;
00172 //virtual void mouseDoubleClickEvent(QMouseEvent *event) override;
00173 //virtual void mouseMoveEvent(QMouseEvent *event) override;
00174 //virtual void mousePressEvent(QMouseEvent *event) override;
00175 //virtual void mouseReleaseEvent(QMouseEvent *event) override;
00176 //virtual void paintEvent(QPaintEvent *event) override;
00177 //virtual void resizeEvent(QResizeEvent *event) override;
00178 //virtual void scrollContentsBy(int dx, int dy) override;
00179 //virtual void showEvent(QShowEvent *event) override;
00180 //virtual bool viewportEvent(QEvent *event) override;
00181 virtual void wheelEvent(QWheelEvent *event) override;
00182 // Draws optional rulers and guides using the current visible scene rectangle.
00183 virtual void drawForeground(QPainter *painter, const QRectF &rect) override;
00184 private:
00185 typedef std::function<void(QGraphicsSceneMouseEvent*) > sceneMouseEventHandlerMethod;
00186 typedef std::function<void()> sceneWheelEventHandlerMethod;
00187 typedef std::function<void(const GraphicalModelEvent&)> sceneGraphicalModelEventHandlerMethod;
00188 typedef std::function<void(QContextMenuEvent*)> contextMenuEventHandlerMethod;
00189 sceneMouseEventHandlerMethod _sceneMouseEventHandler;
00190 sceneWheelEventHandlerMethod _sceneWheelInEventHandler;
00191 sceneWheelEventHandlerMethod _sceneWheelOutEventHandler;
00192 sceneGraphicalModelEventHandlerMethod _sceneGraphicalModelEventHandler;
00193 contextMenuEventHandlerMethod _contextMenuEventHandler;
00194 Simulator* _simulator = nullptr;
00195 PropertyEditorGenesys* _propertyEditor = nullptr;
00196 std::map<SimulationControl*, DataComponentProperty*>* _propertyList = nullptr;
00197 std::map<SimulationControl*, DataComponentEditor*>* _propertyEditorUI = nullptr;
00198 std::map<SimulationControl*, ComboBoxEnum*>* _propertyBox = nullptr;
00199 QWidget* _parentWidget;
00200 bool _notifyGraphicalModelEventHandlers = true;
00201 bool _ruleVisible = false;
00202 bool _guidesVisible = false;
00203 };
00204
00205 #endif /* QMODELGRAPHICVIEW_H */

```

## 16.62 GuiCrashDiagnostics.h

```

00001 #ifndef GUICRASHDIAGNOSTICS_H
00002 #define GUICRASHDIAGNOSTICS_H
00003
00011 namespace GuiCrashDiagnostics {

```



```

00012
00016 void installFatalSignalHandlers();
00017
00018 } // namespace GuiCrashDiagnostics
00019
00020 #endif // GUICRASHDIAGNOSTICS_H

```

## 16.63 GuiScopeTrace.h

```

00001 #ifndef GUISCOPETRACE_H
00002 #define GUISCOPETRACE_H
00003
00004 #include <chrono>
00005
00013 class GuiScopeTrace {
00014 public:
00020 GuiScopeTrace(const char* methodName, const void* thisPtr);
00024 ~GuiScopeTrace();
00025
00026 private:
00028 const char* _methodName;
00030 const void* _thisPtr;
00032 int _depth;
00034 std::chrono::steady_clock::time_point _start;
00035 };
00036
00037 #endif // GUISCOPETRACE_H

```

## 16.64 guithememanager.h

```

00001 #ifndef GUITHEMEMANAGER_H
00002 #define GUITHEMEMANAGER_H
00003
00004 class QApplication;
00005 class ModelGraphicsView;
00006
00013 class GuiThemeManager {
00014 public:
00019 static void applyApplicationTheme(QApplication* application);
00024 static void applyModelGraphicsTheme(ModelGraphicsView* graphicsView);
00025
00026 private:
00027 GuiThemeManager() = default;
00028 };
00029
00030 #endif // GUITHEMEMANAGER_H

```

## 16.65 applications/gui/genesys/mainwindow.cpp File Reference

Composition-root partition of [MainWindow](#) implementation.

```

#include "mainwindow.h"
#include "ui_mainwindow.h"
#include "kernel/simulator/SinkModelComponent.h"
#include "../kernel/simulator/essentialPlugins/Attribute.h"
#include "../kernel/simulator/essentialPlugins/Counter.h"
#include "../kernel/simulator/essentialPlugins/StatisticsCollector.h"
#include "kernel/simulator/PluginManager.h"
#include "kernel/simulator/Plugin.h"
#include "../kernel/simulator/model/ModelComponent.h"
#include "../kernel/simulator/model/ModelComponentManager.h"
#include "../kernel/simulator/model/ModelDataManager.h"
#include "graphicals/ModelGraphicsScene.h"
#include "TraitsGUI.h"
#include "graphicals/GraphicalConnection.h"
#include "graphicals/GraphicalModelDataDefinition.h"
#include "controllers/SimulationController.h"
#include "controllers/ModelInspectorController.h"
#include "controllers/TraceConsoleController.h"
#include "controllers/SimulationEventController.h"

```

```
#include "controllers/PluginCatalogController.h"
#include "controllers/PropertyEditorController.h"
#include "controllers/ModelLifecycleController.h"
#include "controllers/SimulationCommandController.h"
#include "controllers/EditCommandController.h"
#include "controllers/SceneToolController.h"
#include "controllers/GraphicalContextMenuController.h"
#include "controllers/DialogUtilityController.h"
#include "extensions/GuiExtensionContracts.h"
#include "extensions/GuiExtensionManager.h"
#include "extensions/GuiExtensionPluginCatalog.h"
#include "services/ModelLanguageSynchronizer.h"
#include "services/GraphvizModelExporter.h"
#include "services/CppModelExporter.h"
#include "services/GraphicalModelSerializer.h"
#include "services/GraphicalModelBuilder.h"
#include "UtilGUI.h"
#include "guithememanager.h"
#include "propertyeditor/qtpropertybrowser/qttreepropertybrowser.h"
#include "animations/AnimationVariable.h"
#include "systempreferences.h"
#include "../.../plugins/data/Logic/Variable.h"
#include <string>
#include <fstream>
#include <sstream>
#include <cstdlib>
#include <algorithm>
#include <cmath>
#include <limits>
#include <QMessageBox>
#include <QTextStream>
#include <QFileDialog>
#include <QFileInfo>
#include <QDateTime>
#include <QEventLoop>
#include <QTemporaryFile>
#include <Qt>
#include <QGraphicsPixmapItem>
#include <QKeySequence>
#include <QPropertyAnimation>
#include <QtWidgets/qgraphicsitem.h>
#include <QScreen>
#include <QDebug>
#include <QRegularExpression>
#include <QRandomGenerator>
#include <QAction>
#include <QFrame>
#include <QLabel>
#include <QMap>
#include <QMenu>
#include <QPainter>
#include <QScrollArea>
#include <QSignalBlocker>
#include <QSizePolicy>
#include <QSplitter>
#include <QTabBar>
#include <QTabWidget>
#include <QTreeWidget>
```



```
#include <QTimer>
#include <QToolBar>
#include <QVBoxLayout>
```

### 16.65.1 Detailed Description

Composition-root partition of [MainWindow](#) implementation.

This file contains central [MainWindow](#) construction, initialization, and wiring logic, including creation of extracted controllers/services and baseline UI setup. It does not define a new class; it is a physical partition of the same [MainWindow](#) implementation used as a compatibility façade in the incremental refactoring.

## 16.66 mainwindow.h

```
00001 #ifndef MAINWINDOW_H
00002 #define MAINWINDOW_H
00003
00004 #include <QMainWindow>
00005 #include <QListWidget>
00006 #include <QTreeWidgetItem>
00007 #include <QGraphicsItem>
00008 #include <QUndoView>
00009 #include <QMetaObject>
00010 #include <memory>
00011 #include <map>
00012
00013 #include "propertyeditor/DataComponentProperty.h"
00014 #include "propertyeditor/DataComponentEditor.h"
00015 #include "propertyeditor/ComboBoxEnum.h"
00016
00017 #include "kernel/simulator/Simulator.h"
00018 #include "kernel/simulator/PropertyGenesys.h"
00019 #include "kernel/simulator/TraceManager.h"
00020 #include "graphicals/ModelGraphicsScene.h"
00021
00022 QT_BEGIN_NAMESPACE
00023 namespace Ui {
00024 class MainWindow;
00025 }
00026
00027 QT_END_NAMESPACE
00028
00029 class ModellLanguageSynchronizer;
00030 class GraphvizModelExporter;
00031 class CppModelExporter;
00032 class GraphicalModelSerializer;
00033 class GraphicalModelBuilder;
00034 class ModelInspectorController;
00035 class TraceConsoleController;
00036 class SimulationEventController;
00037 class PluginCatalogController;
00038 class PropertyEditorController;
00039 class ModellLifecycleController;
00040 class SimulationCommandController;
00041 class EditCommandController;
00042 class SceneToolController;
00043 class GraphicalContextMenuController;
00044 class DialogUtilityController;
00045 class GuiExtensionManager;
00046 class QAction;
00047 class QMenu;
00048 class QTabWidget;
00049 class ModelGraphicsView;
00050 class ModelComponent;
00051 class ModelDataDefinition;
00052
00053 // Document MainWindow as composition root and incremental compatibility façade.
00073 class MainWindow : public QMainWindow {
00074 Q_OBJECT
00075
00076 // Allow the Phase 4 event controller to register private compatibility handlers.
00077 friend class SimulationEventController;
00078
00079 public:
00081 MainWindow(QWidget *parent = nullptr);
00083 ~MainWindow();
00085 ModelGraphicsScene* myScene() const;
00086
00087 public: // to notify changes
00089 bool graphicalModelHasChanged() const;
```

```

00091 void setGraphicalModelHasChanged(bool graphicalModelHasChanged);
00093 void unselectDrawIcons();
00095 bool checkSelectedDrawIcons();
00097 void refreshGuiExtensions();
00099 void refreshPluginCatalog();
00100
00101 private slots:
00102 // actions
00103 void on_actionEditUndo_triggered();
00104 void on_actionEditRedo_triggered();
00105 void on_actionEditFind_triggered();
00106 // void on_actionReplace_triggered(); // old name (without menu namespace)
00107 void on_actionEditReplace_triggered();
00108 void on_actionEditCut_triggered();
00109 void on_actionEditCopy_triggered();
00110 void on_actionEditPaste_triggered();
00111 void on_actionEditDelete_triggered();
00112 void on_actionEditGroup_triggered();
00113 void on_actionEditUngroup_triggered();
00114 void on_actionViewGroup_triggered();
00115 void on_actionViewUngroup_triggered();
00116
00117 void on_actionShowGrid_triggered();
00118 void on_actionShowRule_triggered();
00119 void on_actionShowGuides_triggered();
00120 void on_actionViewConfigure_triggered();
00121
00122 void on_actionZoom_In_triggered();
00123 void on_actionZoom_Out_triggered();
00124 void on_actionZoom_All_triggered();
00125 void on_actionZoom_OneToOne_triggered();
00126
00127 void on_actionDrawLine_triggered();
00128 void on_actionDrawRectangle_triggered();
00129 void on_actionDrawEllipse_triggered();
00130 void on_actionDrawText_triggered();
00131 void on_actionDrawPolygon_triggered();
00132
00133 void on_actionAnimateSimulatedTime_triggered();
00134 void on_actionAnimateVariable_triggered();
00135 void on_actionAnimateExpression_triggered();
00136 void on_actionAnimateResource_triggered();
00137 void on_actionAnimateQueue_triggered();
00138 void on_actionAnimateStation_triggered();
00139 void on_actionAnimateCounter_triggered();
00140 void on_actionAnimateEntity_triggered();
00141 void on_actionAnimateEvent_triggered();
00142 void on_actionAnimateAttribute_triggered();
00143 void on_actionAnimateStatistics_triggered();
00144 void on_actionAnimatePlot_triggered();
00145
00146 void on_actionSimulationStop_triggered();
00148 void on_actionSimulationStart_triggered();
00150 void on_actionSimulationStep_triggered();
00152 void on_actionSimulationPause_triggered();
00154 void on_actionSimulationResume_triggered();
00156 void on_actionSimulationConfigure_triggered();
00157
00158 void on_actionAboutAbout_triggered();
00159 void on_actionAboutLicence_triggered();
00160 void on_actionAboutGetInvolved_triggered();
00161
00162 void on_actionAlignMiddle_triggered();
00163 void on_actionAlignTop_triggered();
00164 void on_actionAlignRight_triggered();
00165 void on_actionAlignCenter_triggered();
00166 void on_actionAlignLeft_triggered();
00167
00168 void on_actionToolsParserGrammarChecker_triggered();
00169 void on_actionToolsWebWorker_triggered();
00170 // Legacy slot: there is no matching QAction in mainwindow.ui at this moment.
00171 void on_actionToolsExperimentation_triggered();
00172 void on_actionToolsOptimizator_triggered();
00173 void on_actionToolsAIAssistant_triggered();
00174 void on_actionToolsDataAnalyzer_triggered();
00175
00176 void on_actionSimulatorsPluginManager_triggered();
00177 void on_actionSimulatorExit_triggered();
00178 void on_actionSimulatorPreferences_triggered();
00179
00181 void on_actionModelNew_triggered();
00183 void on_actionModelOpen_triggered();
00185 void on_actionModelSave_triggered();
00187 void on_actionModelClose_triggered();
00189 void on_actionAnimation_triggered();
00191 void on_actionModelInformation_triggered();
00193 void on_actionModelCheck_triggered();

```

```

00194
00195 void on_actionGModelComponentBreakpoint_triggered();
00196 void on_actionShowInternalElements_triggered();
00197 void on_actionShowEditableElements_triggered();
00198 void on_actionShowAttachedElements_triggered();
00199 void on_actionShowRecursiveElements_triggered();
00200
00201
00202 // widget events
00203 //void on_textCodeEdit_Model_textChanged();
00204 void on_tabWidget_Model_tabBarClicked(int index);
00205 void on_tabWidget_Debug_currentChanged(int index);
00206 void on_tabWidgetCentral_currentChanged(int index);
00207 void on_tabWidgetCentral_tabBarClicked(int index);
00208 void on_tabWidgetModel_currentChanged(int index);
00209 void on_tabWidgetSimulation_currentChanged(int index);
00210 void on_tabWidgetReports_currentChanged(int index);
00211 void on_tabWidgetModelLanguages_currentChanged(int index);
00212
00213 void on_treeWidget_Plugins_itemDoubleClicked(QTreeWidgetItem *item, int column);
00214 void on_treeWidgetComponents_itemSelectionChanged();
00215 void on_treeWidgetDataDefinitions_itemSelectionChanged();
00216 void on_treeWidget_Plugins_itemClicked(QTreeWidgetItem *item, int column);
00217
00218 void on_horizontalSlider_Zoom_valueChanged(int value);
00219 void on_horizontalSlider_ZoomGraphical_valueChanged(int value);
00220
00221 void on_checkBox_ShowElements_stateChanged(int arg1);
00222 void on_checkBox_ShowInternals_stateChanged(int arg1);
00223 void on_checkBox_ShowEditableElements_stateChanged(int arg1);
00224 void on_checkBox_ShowRecursive_stateChanged(int arg1);
00225 void on_checkBox_ShowLevels_stateChanged(int arg1);
00226
00227 void on_pushButton_Breakpoint_Insert_clicked();
00228 void on_pushButton_Breakpoint_Remove_clicked();
00229 void on_pushButton_Export_clicked();
00230
00232 void on_graphicsView_rubberBandChanged(const QRect &viewportRect, const QPointF &fromScenePoint,
const QPointF &toScenePoint);
00234 void on_TextCodeEditor_textChanged();
00235
00236 void on_treeWidgetDataDefinitions_itemDoubleClicked(QTreeWidgetItem *item, int column);
00237 void on_treeWidgetDataDefinitions_itemChanged(QTreeWidgetItem *item, int column);
00238 void on_actionArranjeLeft_triggered();
00239 void on_actionArranjeRight_triggered();
00240 void on_actionArranjeTop_triggered();
00241 void on_actionArranjeBototm_triggered();
00242 void on_actionArranjeCenter_triggered();
00243 void on_actionArranjeMiddle_triggered();
00244 void on_actionShowSnap_triggered();
00245 void on_actionGModelShowConnect_triggered(bool checked = false);
00246
00247 void on_actionActivateGraphicalSimulation_triggered();
00248 void on_horizontalSliderAnimationSpeed_valueChanged(int value);
00249 void on_actionSelectAll_triggered();
00250
00251 void on_actionParallelization_triggered();
00252
00253 void on_horizontalSlider_ZoomGraphical_actionTriggered(int action);
00255 void _onPropertyEditorModelChanged();
00256
00257 protected:
00258 void closeEvent(QCloseEvent *event) override;
00259
00260 private: // VIEW
00261
00262 private: // trace handlers
00264 void _simulatorTraceHandler(TraceEvent e);
00266 void _simulatorTraceErrorHandler(TraceErrorEvent e);
00268 void _simulatorTraceSimulationHandler(TraceSimulationEvent e);
00270 void _simulatorTraceReportsHandler(TraceEvent e);
00271 private: // simulator event handlers
00273 void _onModelCheckSuccessHandler(ModelEvent* re);
00275 void _onReplicationStartHandler(SimulationEvent* re);
00277 void _onReplicationEndHandler(SimulationEvent* re);
00279 void _onSimulationStartHandler(SimulationEvent* re);
00281 void _onSimulationPausedHandler(SimulationEvent* re);
00283 void _onSimulationResumeHandler(SimulationEvent* re);
00285 void _onSimulationEndHandler(SimulationEvent* re);
00287 void _onProcessEventHandler(SimulationEvent* re);
00289 void _onEntityCreateHandler(SimulationEvent* re);
00291 void _onEntityRemoveHandler(SimulationEvent* re);
00293 void _onMoveEntityEvent(SimulationEvent * re);
00295 void _onAfterProcessEvent(SimulationEvent * re);
00296 private: // model Graphics View handlers
00298 void _onSceneMouseEvent(QGraphicsSceneMouseEvent* mouseEvent);
00300 void _onSceneWheelInEvent();

```

```

00302 void _onSceneWheelOutEvent();
00304 void _onSceneGraphicalModelEvent(const GraphicalModelEvent& event);
00305 private: // QGraphicsScene Slots
00307 void sceneChanged(const QList<QRectF> ®ion);
00309 void sceneFocusItemChanged(QGraphicsItem *newFocusItem, QGraphicsItem *oldFocusItem,
Qt::FocusReason reason);
00310 //void sceneRectChanged(const QRectF &rect);
00312 void sceneSelectionChanged();
00313 private: // Similar to QGraphicsScene Slots
00315 void sceneGraphicalModelChanged();
00316 private: // simulator related
00318 void _setOnEventHandlers();
00320 bool _ensureSimulationReady(bool checkModel = true);
00322 bool _hasCurrentModelSimulation() const;
00324 void _insertPluginUI(Plugin* plugin);
00326 void _insertFakePlugins();
00328 bool _setSimulationModelBasedOnText();
00330 bool _createModelImage();
00332 std::string _adjustDotName(std::string name);
00334 void _insertTextInDot(std::string text, unsigned int compLevel, unsigned int compRank,
std::map<unsigned int, std::map<unsigned int, std::list<std::string>*>*> dotmap, bool isNode =
false);
00336 void _recursiveCreateModelGraphicPicture(ModelDataDefinition* componentOrData,
std::list<ModelDataDefinition*>* visited, std::map<unsigned int, std::map<unsigned int,
std::list<std::string>*>*> dotmap);
00338 void _actualizeModelCppCode();
00340 std::string _addCppCodeLine(std::string line, unsigned int indent = 0);
00341 private: // view
00343 void _initModelGraphicsView();
00345 void _connectSceneSignals();
00347 void _disconnectSceneSignals(const char* context);
00349 void _initializeModelTabs();
00351 void _configureModelGraphicsView(ModelGraphicsView* graphicsView);
00353 ModelGraphicsView* _createModelGraphicsView(QWidget* parent);
00355 void _activateModelGraphicsView(ModelGraphicsView* graphicsView, bool rebuildControllers = true);
00357 ModelGraphicsView* _ensureModelTab(Model* model);
00359 void _removeModelTab(Model* model);
00361 void _updateModelTabs();
00363 void _activateModelTab(int index);
00365 void _activatePreviousModel();
00367 void _activateNextModel();
00369 void _rebuildViewDependentControllers();
00371 void _refreshGuiExtensions();
00373 void _initUiForNewModel(Model* m);
00375 void _actualizeActions();
00377 void _actualizeUndo();
00379 void _actualizeTabPanels();
00381 void _actualizeModelSimLanguage();
00383 void _actualizeModelTextHasChanged(bool hasChanged);
00385 void _actualizeSimulationEvents(SimulationEvent * re);
00387 void _actualizeDebugVariables(bool force);
00389 void _actualizeDebugEntities(bool force);
00391 void _actualizeDebugBreakpoints(bool force);
00393 void _prepareReportsResultsTable();
00395 void _clearReportsResultsTable();
00397 void _actualizeReportsResultsTable();
00399 void _prepareReportsPlots();
0401 void _clearReportsPlots();
0403 void _actualizeReportsPlots();
0405 void _showReplicationReportIfSuppressed();
0407 void _showSimulationReportIfSuppressed();
0409 void _actualizeModelComponents(bool force);
0411 void _actualizeModelDataDefinitions(bool force);
0413 void _actualizeGraphicalModel(SimulationEvent * re);
0415 void _insertCommandInConsole(std::string text);
0417 void _clearModelEditors();
0419 void _gentle_zoom(double factor);
0421 void _showMessageNotImplemented();
0423 void _recursivelyGenerateGraphicalModelFromModel(ModelComponent* component, List<ModelComponent*>*
visited, std::map<ModelComponent*, GraphicalModelComponent*>* map, int *x, int *y, int *ymax, int
sequenceIndex);
0425 void _generateGraphicalModelFromModel();
0427 void saveItemForCopy(QList<GraphicalModelComponent*> * gmcList, QList<GraphicalConnection*> *
connList);
0428 //bool _checkStartSimulation();
0429 private: // opened-model document state
0431 enum class GraphicalModelViewContextKind {
0432 RootModel,
0433 Submodel
0434 };
0435
0436 struct GraphicalModelViewContext {
0437 GraphicalModelViewContextKind kind = GraphicalModelViewContextKind::RootModel;
0438 Model* rootModel = nullptr;
0439 unsigned int modelLevel = 0;
0440 ModelComponent* ownerComponent = nullptr;
0441 ModelDataDefinition* ownerDataDefinition = nullptr;

```

```

00450 ModelGraphicsView* graphicsView = nullptr;
00451 QString explicitTitle;
00452 };
00453
00455 GraphicalModelViewContext* _activeViewContext() const;
00457 GraphicalModelViewContext* _contextForView(ModelGraphicsView* graphicsView) const;
00459 GraphicalModelViewContext* _rootContextForModel(Model* model) const;
00461 GraphicalModelViewContext* _ensureRootModelViewContext(Model* model, ModelGraphicsView*
graphicsView);
00463 ModelGraphicsView* _ensureSubmodelViewContext(ModelDataDefinition* owner);
00465 void _closeSubmodelViewContext(ModelGraphicsView* graphicsView);
00467 void _removeSubmodelViewContextsForModel(Model* model);
00469 void _removeViewContext(ModelGraphicsView* graphicsView);
00471 QString _viewContextTitle(const GraphicalModelViewContext& context) const;
00473 bool _viewContextBelongsToOpenModel(const GraphicalModelViewContext& context) const;
00475 ModelDataDefinition* _selectedSubmodelOwner() const;
00477 bool _canOpenSubmodelFor(ModelDataDefinition* owner) const;
00479 void _openSelectedSubmodel();
00480 void _refreshRecentModelsMenu();
00481 bool _openRecentModelFile(const QString& fileName);
00483 void _syncCurrentModelDocumentState();
00485 void _restoreModelDocumentState(Model* model);
00487 QString _modelDisplayName(Model* model) const;
00489 QString _modelSaveBaseFilename(QString filename) const;
00491 bool _modelHasPendingChanges(Model* model) const;
00493 bool _saveCurrentModel(bool promptForFilename = true);
00495 bool _saveModelWithActivation(Model* model);
00497 bool _confirmCloseModel(Model* model);
00499 bool _closeModel(Model* model);
00501 void _clearModelGraphicsViewForClose(ModelGraphicsView* graphicsView);
00502 private: // graphical model persistence
00504 bool _saveGraphicalModel(QString filename);
00506 bool _saveTextModel(QFile *saveFile, QString data);
00508 Model* _loadGraphicalModel(std::string filename);
00509 private: //???
00511 void _helpCopy();
00512 private: // interface and model main elements to join
00513 Ui::MainWindow *ui;
00514 Simulator* simulator;
00515 // Hosts one graphical scene/view per opened kernel model.
00516 QTabWidget* _modelGraphicsTabs = nullptr;
00517 // Maps opened kernel models to their graphical view so tab switching preserves each scene.
00518 std::map<Model*, ModelGraphicsView*> _modelGraphicsViews;
00519 // Reverse lookup used by tab activation without relying on QVariant pointer casts.
00520 std::map<ModelGraphicsView*, Model*> _modelsByGraphicsView;
00521 // Owns graphical contexts by view; currently every context is root-model, future ones may be
submodels.
00522 std::map<ModelGraphicsView*, std::unique_ptr<GraphicalModelViewContext>
_viewContextsByGraphicsView;
00523 // Fast lookup for the root graphical context of each opened kernel model.
00524 std::map<Model*, GraphicalModelViewContext*> _rootViewContextsByModel;
00525 // Keeps each opened model's last graphical/text filename independent while switching tabs.
00526 std::map<Model*, QString> _modelFileNames;
00527 // Caches the editable Simulang text per opened model so tab switches do not discard unsaved text.
00528 std::map<Model*, QString> _modelTextContents;
00529 // Tracks text-editor dirty state per opened model while legacy flags mirror only the active
model.
00530 std::map<Model*, bool> _modelTextHasChanged;
00531 // Tracks graphical dirty state per opened model while legacy flags mirror only the active model.
00532 std::map<Model*, bool> _modelGraphicalHasChanged;
00533 // Tracks tabs being changed programmatically to avoid re-entrant model switching.
00534 bool _changingModelTabProgrammatically = false;
00535 // Runtime menu actions for navigating the opened-model list.
00536 QAction* _actionModelPrevious = nullptr;
00537 QAction* _actionModelNext = nullptr;
00538 // Runtime action that opens a level-scoped graphical tab from the selected owner item.
00539 QAction* _actionOpenSelectedSubmodel = nullptr;
00540 QMenu* _menuOpenRecent = nullptr;
00541 QList<QAction*> _recentModelActions;
00542 // Keep core simulation command gateway owned by MainWindow composition root.
00543 std::unique_ptr<class SimulationController> _simulationController;
00544 // Keep phase-ordered services to make composition dependencies explicit in Phase 12.
00545 // Synchronize textual model language with the kernel model manager.
00546 std::unique_ptr<ModelLanguageSynchronizer> _modelLanguageSynchronizer;
00547 // Generate Graphviz DOT and rendered model images for the GUI pane.
00548 std::unique_ptr<GraphvizModelExporter> _graphvizModelExporter;
00549 // Generate C++ model representation text for the code viewer pane.
00550 std::unique_ptr<CppModelExporter> _cppModelExporter;
00551 // Persist and restore graphical .gui state through the Phase 2 serializer service.
00552 std::unique_ptr<GraphicalModelSerializer> _graphicalModelSerializer;
00553 // Rebuild graphical components and links through the Phase 2 builder service.
00554 std::unique_ptr<GraphicalModelBuilder> _graphicalModelBuilder;
00555 // Keep phase-ordered controllers to preserve the final compatibility façade surface.
00556 // Add the Phase 3 model-inspector controller owned by MainWindow.
00557 std::unique_ptr<ModelInspectorController> _modelInspectorController;
00558 // Add the Phase 4 trace controller owned by MainWindow.
00559 std::unique_ptr<TraceConsoleController> _traceConsoleController;

```

```

00560 // Add the Phase 4 simulation-event controller owned by MainWindow.
00561 std::unique_ptr<SimulationEventController> _simulationEventController;
00562 // Add the Phase 5 plugin-catalog controller owned by MainWindow.
00563 std::unique_ptr<PluginCatalogController> _pluginCatalogController;
00564 // Add the Phase 6 property-editor controller owned by MainWindow.
00565 std::unique_ptr<PropertyEditorController> _propertyEditorController;
00566 // Add the Phase 7 model-lifecycle controller owned by MainWindow.
00567 std::unique_ptr<ModelLifecycleController> _modelLifecycleController;
00568 // Add the Phase 8 simulation-command controller owned by MainWindow.
00569 std::unique_ptr<SimulationCommandController> _simulationCommandController;
00570 // Add the Phase 9 edit-command controller owned by MainWindow.
00571 std::unique_ptr<EditCommandController> _editCommandController;
00572 // Add the Phase 10 scene-tool controller owned by MainWindow.
00573 std::unique_ptr<SceneToolController> _sceneToolController;
00574 // Add the graphical context-menu controller owned by MainWindow.
00575 std::unique_ptr<GraphicalContextMenuController> _graphicalContextMenuController;
00576 // Add the Phase 11 dialog-utility controller owned by MainWindow.
00577 std::unique_ptr<DialogUtilityController> _dialogUtilityController;
00578 // Keeps runtime GUI extension contributions isolated from core static menus/toolbars.
00579 std::unique_ptr<GuiExtensionManager> _guiExtensionManager;
00580 PropertyEditorGenesys* propertyGenesys;
00581 std::map<SimulationControl*, DataComponentProperty*>* propertyList;
00582 std::map<SimulationControl*, DataComponentEditor*>* propertyEditorUI;
00583 std::map<SimulationControl*, ComboBoxEnum*>* propertyBox;
00584 private: // attributes to be saved and loaded withing the graphical model
00585 int _zoomValue; // todo should be set for each open graphical model, such as view rect, etc
00586 private: // misc useful
00587 bool _check(bool success = true);
00588 bool _hasPendingModelChanges() const;
00589 bool _confirmApplicationExit();
00590 bool _textModelHasChanged = false;
00591 bool _graphicalModelHasChanged = false;
00592 bool _modelWasOpened = false;
00593 // Avoids duplicated prompts when exit is approved and Qt emits close events during shutdown.
00594 bool _closingApproved = false;
00595 QString _autoLoadPluginsFilename = "autoloadplugins.txt";
00596 bool _checkItemsScene();
00597 QString _modelFilename;
00598 std::map<std::string /*category*/, QColor*>* _pluginCategoryColor = new
00599 std::map<std::string, QColor*>();
00600 // TODO 1: Faz parte do mecanismo de restaurar dataDefinitions deletados do modelo e que são
00601 restaurados com um Control Z
00602 // Caso: Ao adicionar um Create no modelo e dar um check() o EntityType será criado,
00603 // mas ao deletar o Create, dar outro check() e em seguida dar um Control Z (voltando o Create no
00604 modelo) e checar novamente, o EntityType não é restaurado
00605 // na lista de dataDefinitions do modelo, apesar de ainda existir no Create.
00606 // Neste caso, ocorre um erro acusando que o EntityType do Create não está no modelo.
00607 // Isso se dá ao fato de que no Kernel ele verifica se _entityType = nullptr para criar um novo e
00608 reinserir no modelo, mas não trata o caso dele não ser nullptr
00609 // e não estar nos dataDefinitions do modelo, que é o que ocorre na situação que foi descrita.
00610 // Esta função foi feita para tratar esse caso, assim como é feito com
00611 insertRestoredDataDefinitions e saveDataDefinitions em ModelGraphicScene
00612 void setStatisticsCollector();
00613
00614 bool _cut;
00615 QList<GraphicalModelComponent*> * _gmc_copies = new QList<GraphicalModelComponent*>();
00616 QList<GraphicalConnection*> * _ports_copies = new QList<GraphicalConnection*>();
00617 QList<QGraphicsItem *> * _draw_copy = new QList<QGraphicsItem *>();
00618 QList<QGraphicsItemGroup *> * _group_copy = new QList<QGraphicsItemGroup *>();
00619
00620 struct COPY {
00621 GraphicalModelComponent * old;
00622 GraphicalModelComponent * copy;
00623 };
00624
00625 bool _firstClickShowConnection = true;
00626 private:
00627
00628 const struct TABINDEXES_STRUC {
00629 const int TabCentralModelIndex = 0;
00630 const int TabCentralSimulationIndex = 1;
00631 const int TabCentralReportsIndex = 2;
00632 //
00633 const int TabModelSimLangIndex = 0;
00634 const int TabModelCppCodeIndex = 1;
00635 const int TabModelDiagramIndex = 2;
00636 const int TabModelComponentsIndex = 3;
00637 //
00638 const int TabModelDataDefinitionsIndex = 4;
00639 const int TabSimulationBreakpointsIndex = 0;
00640 const int TabSimulationVariablesIndex = 1;
00641 const int TabSimulationEntitiesIndex = 2;
00642 const int TabSimulationTracesIndex = 3;
00643 const int TabSimulationEventsIndex = 4;
00644 //
00645 const int TabReportReportIndex = 0;

```

```

00646 const int TabReportResultIndex = 1;
00647 const int TabReportPlotIndex = 2;
00648 } CONST;
00649
00650 QUndoView *undoView = nullptr;
00651 bool _graphicalSimulation = false;
00652 bool _modelCheked = false;
00653 bool _loaded = false;
00654 bool _shuttingDown = false;
00655 // Persists lightweight tool settings introduced in Phase 5 dialogs.
00656 double _optimizerPrecision = 1e-6;
00657 unsigned int _optimizerMaxSteps = 1000;
00658 // Persists minimal parallel execution preferences configured in GUI.
00659 bool _parallelizationEnabled = false;
00660 int _parallelizationThreads = 1;
00661 int _parallelizationBatchSize = 100;
00662 // Remembers the last dataset location used by the data analyzer workflow.
00663 QString _lastDataAnalyzerPath;
00664 QMetaObject::Connection _sceneChangedConnection;
00665 QMetaObject::Connection _sceneFocusItemChangedConnection;
00666 QMetaObject::Connection _sceneSelectionChangedConnection;
00667 bool _isDeferredPropertyEditorModelChangedScheduled = false;
00668 //CodeEditor* textCodeEdit_Model;
00669 };
00670 #endif // MAINWINDOW_H

```

## 16.67 applications/gui/genesys/mainwindow\_controller.cpp File Reference

Controller-delegation partition of [MainWindow](#) implementation.

```

#include "mainwindow.h"
#include "ui_mainwindow.h"
#include "controllers/ModelInspectorController.h"
#include "controllers/PropertyEditorController.h"
#include "controllers/PluginCatalogController.h"
#include "controllers/ModelLifecycleController.h"
#include "controllers/SimulationCommandController.h"
#include "controllers/EditCommandController.h"
#include "controllers/SceneToolController.h"
#include "controllers/DialogUtilityController.h"
#include "dialogs/dialogBreakpoint.h"
#include "dialogs/Dialogmodelinformation.h"
#include "dialogs/dialogsimulationconfigure.h"
#include "dialogs/dialogpluginmanager.h"
#include "dialogs/dialogsystempreferences.h"
#include "dialogs/DialogFind.h"
#include "controllers/SimulationController.h"
#include "services/ToolLauncher.h"
#include <string>
#include <fstream>
#include <memory>
#include <algorithm>
#include <cmath>
#include <cstdlib>
#include <QCoreApplication>
#include <QMessageBox>
#include <QTextStream>
#include <QFileDialog>
#include <QGraphicsScene>
#include <QDateTime>
#include <QEventLoop>
#include <QTemporaryFile>
#include <Qt>
#include <QGraphicsPixmapItem>
#include <QPropertyAnimation>

```



```

#include <QtWidgets/qgraphicsitem.h>
#include <QScreen>
#include <QDebug>
#include <QRegularExpression>
#include <QRandomGenerator>
#include <QSignalBlocker>
#include <QDialog>
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QFormLayout>
#include <QLineEdit>
#include <QPushButton>
#include <QDialogButtonBox>
#include <QLabel>
#include <QCheckBox>
#include <QSpinBox>
#include <QDoubleSpinBox>
#include <QPlainTextEdit>
#include <QTextEdit>
#include <QStatusBar>
#include <QImage>
#include <QPainter>
#include <QFileInfo>
#include "../kernel/simulator/model/ModelSimulation.h"
#include "../tools/Continuous/SolverDefaultImpl1.h"

```

### 16.67.1 Detailed Description

Controller-delegation partition of [MainWindow](#) implementation.

This file groups menu/toolbar/widget slot wrappers that delegate behavior to extracted controllers (for example lifecycle, simulation command, edit, scene tool, and dialog utility controllers). It does not define a new class; it is a thematic partition of [MainWindow](#).

## 16.68 applications/gui/genesys/mainwindow\_modelrepresentations.cpp

### File Reference

[Model](#) representation/persistence wrapper partition of [MainWindow](#) implementation.

```

#include "mainwindow.h"
#include "ui_mainwindow.h"
#include "controllers/ModelInspectorController.h"
#include "services/ModelLanguageSynchronizer.h"
#include "services/GraphvizModelExporter.h"
#include "services/CppModelExporter.h"
#include "services/GraphicalModelSerializer.h"
#include "services/GraphicalModelBuilder.h"
#include "kernel/simulator/SinkModelComponent.h"
#include <string>
#include <fstream>
#include <sstream>
#include <cstdlib>
#include <cctype>
#include <QMessageBox>
#include <QTextStream>
#include <QFileDialog>
#include <QDateTime>
#include <QEventLoop>

```



```

#include <QTemporaryFile>
#include <Qt>
#include <QGraphicsPixmapItem>
#include <QPropertyAnimation>
#include <QtWidgets/qgraphicsitem.h>
#include <QScreen>
#include <QDebug>
#include <QRegularExpression>
#include <QRandomGenerator>
#include <QScrollBar>
#include <QSignalBlocker>
#include <QTimer>
#include <QUrl>

```

Include dependency graph for mainwindow\_modelrepresentations.cpp:



### 16.68.1 Detailed Description

**Model** representation/persistence wrapper partition of **MainWindow** implementation.

This file groups **MainWindow** wrappers related to textual synchronization, Graphviz and C++ exports, graphical persistence/serialization, and graphical model reconstruction through extracted services/controllers. It does not define a new class; it is a thematic partition of **MainWindow**.

## 16.69 applications/gui/genesys/mainwindow\_scene.cpp File Reference

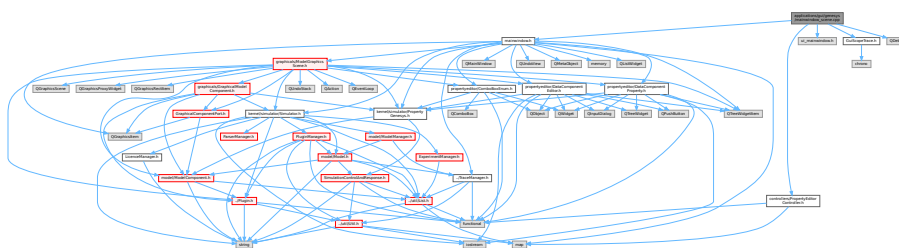
Scene callback and scene synchronization partition of **MainWindow** implementation.

```

#include "mainwindow.h"
#include "ui_mainwindow.h"
#include "controllers/PropertyEditorController.h"
#include "GuiScopeTrace.h"
#include <QDebug>

```

Include dependency graph for mainwindow\_scene.cpp:



### 16.69.1 Detailed Description

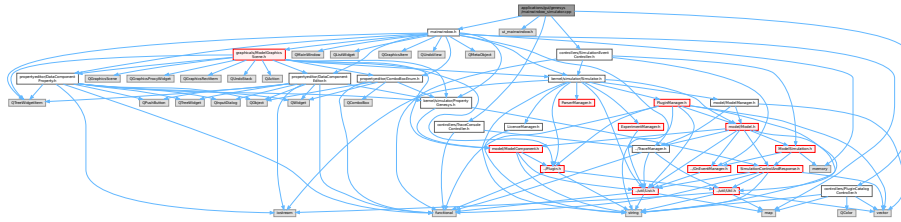
Scene callback and scene synchronization partition of **MainWindow** implementation.

This file concentrates scene-related callbacks, selection/focus/zoom bridging, and wrappers that synchronize scene state with UI elements such as actions and the property editor controller. It does not define a new class; it is a thematic partition of **MainWindow**.

## 16.70 applications/gui/genesys/mainwindow\_simulator.cpp File Reference

[Simulator](#) trace/event wrapper partition of [MainWindow](#) implementation.

```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include "controllers/TraceConsoleController.h"
#include "controllers/SimulationEventController.h"
#include "controllers/PluginCatalogController.h"
Include dependency graph for mainwindow_simulator.cpp:
```



### 16.70.1 Detailed Description

[Simulator](#) trace/event wrapper partition of [MainWindow](#) implementation.

This file centralizes [MainWindow](#) compatibility wrappers for simulator traces and simulation event callbacks, delegating rendering and reactions to [TraceConsoleController](#) and [SimulationEventController](#) (plus plugin-catalog related delegation where applicable). It does not define a new class; it is a thematic partition of [MainWindow](#).

## 16.71 MovedGuiCompatibilityIncludes.h

```
00001 #ifndef MOVED_GUI_COMPATIBILITY_INCLUDES_H
00002 #define MOVED_GUI_COMPATIBILITY_INCLUDES_H
00003
00004 // Temporary compatibility header for the physical GUI source-tree move.
00005 // Some legacy source files still contain multiple includes on one line after
00006 // the automated path adjustment used during the salvage move. The first include
00007 // on those lines is still seen by the preprocessor, but subsequent includes are
00008 // ignored as extra tokens. This header keeps the moved GUI buildable while those
00009 // source files are repaired incrementally.
00010
00011 #include "../kernel/simulator/Simulator.h"
00012 #include "../kernel/simulator/Plugin.h"
00013 #include "../kernel/simulator/PropertyGenesys.h"
00014 #include "../kernel/simulator/SimulationControlAndResponse.h"
00015 #include "../kernel/simulator/LicenceManager.h"
00016 #include "../kernel/simulator/SimulatorFacade.h"
00017
00018 #include "../kernel/simulator/model/Model.h"
00019 #include "../kernel/simulator/model/ModelManager.h"
00020 #include "../kernel/simulator/model/ModelSimulation.h"
00021 #include "../kernel/simulator/model/ModelComponent.h"
00022 #include "../kernel/simulator/model/ModelDataDefinition.h"
00023 #include "../kernel/simulator/model/ModelDataManager.h"
00024
00025 #include "../kernel/simulator/essentialPlugins/Attribute.h"
00026 #include "../kernel/simulator/essentialPlugins/Counter.h"
00027 #include "../kernel/simulator/essentialPlugins/Entity.h"
00028
00029 #include "../plugins/data/Logic/Variable.h"
00030 #include "../plugins/data/DiscreteProcessing/Queue.h"
00031
00032 #include "../tools/Statistics/FitterDefaultImpl.h"
00033 #include "../tools/Statistics/HypothesisTesterDefaultImpl.h"
00034 #include "../tools/Optimization/OptimizerDefaultImpl.h"
00035 #include "../tools/Continuous/SolverDefaultImpl.h"
00036
00037 #endif // MOVED_GUI_COMPATIBILITY_INCLUDES_H
```

## 16.72 ComboBoxEnum.h

```

00001 #ifndef COMBOBOXENUM_H
00002 #define COMBOBOXENUM_H
00003
00004 #include <functional>
00005
00006 #include <QObject>
00007 #include <QWidget>
00008 #include <QComboBox>
00009
00010 #include "kernel/simulator/PropertyGenesys.h"
00011
00012 class ComboBoxEnum : public QObject {
00013 public:
00014 using AfterChange = std::function<void()>;
00015
00016 ComboBoxEnum(
00017 PropertyEditorGenesys* editor,
00018 SimulationControl* property,
00019 AfterChange afterChange = {}
00020);
00021
00022 ~ComboBoxEnum() override = default;
00023
00024 public:
00025 void open_box();
00026 void configure_options(SimulationControl* property);
00027
00028 private Q_SLOTS:
00029 void changeValue();
00030
00031 private:
00032 QWidget* _window = nullptr;
00033 QComboBox* _comboBox = nullptr;
00034
00035 PropertyEditorGenesys* _editor = nullptr;
00036 SimulationControl* _property = nullptr;
00037 AfterChange _afterChange;
00038 };
00039
00040 #endif // COMBOBOXENUM_H

```

## 16.73 DataComponentEditor.h

```

00001 #ifndef DATACOMPONENTEDITOR_H
00002 #define DATACOMPONENTEDITOR_H
00003
00004 #include <functional>
00005 #include <iostream>
00006
00007 #include <QObject>
00008 #include <QWidget>
00009 #include <QTreeWidget>
00010 #include <QTreeWidgetItem>
00011 #include <QPushButton>
00012 #include <QInputDialog>
00013
00014 #include "kernel/simulator/PropertyGenesys.h"
00015
00016 class DataComponentProperty;
00017 class ComboBoxEnum;
00018
00019 class DataComponentEditor : public QObject {
00020 public:
00021 using AfterChange = std::function<void()>;
00022
00023 DataComponentEditor(
00024 PropertyEditorGenesys* editor,
00025 SimulationControl* property,
00026 AfterChange afterChange = {}
00027);
00028
00029 DataComponentEditor(
00030 PropertyEditorGenesys* editor,
00031 List<SimulationControl*>* properties,
00032 AfterChange afterChange = {}
00033);
00034
00035 ~DataComponentEditor() override = default;
00036
00037 public:
00038 void open_window(SimulationControl* property);
00039 void open_window(List<SimulationControl*>* properties);

```

```

00040
00041 void configure_properties(SimulationControl* property);
00042 void configure_properties(List<SimulationControl*>* properties);
00043
00044 private:
00045 void _notifyChanged();
00046
00047 void editProperty(SimulationControl* property);
00048 void editProperty(List<SimulationControl*>* properties);
00049
00050 private:
00051 QWidget* _window = nullptr;
00052 QTreeWidget* _view = nullptr;
00053 QPushButton* _edit = nullptr;
00054 QInputDialog* _newValue = nullptr;
00055
00056 PropertyEditorGenesys* _editor = nullptr;
00057 AfterChange _afterChange;
00058 };
00059
00060 #endif // DATACOMPONENTEDITOR_H

```

## 16.74 DataComponentProperty.h

```

00001 #ifndef DATACOMPONENTPROPERTY_H
00002 #define DATACOMPONENTPROPERTY_H
00003
00004 #include <functional>
00005 #include <iostream>
00006
00007 #include <QObject>
00008 #include <QWidget>
00009 #include <QTreeWidget>
00010 #include <QTreeWidgetItem>
00011 #include <QPushButton>
00012 #include <QInputDialog>
00013
00014 #include "kernel/simulator/PropertyGenesys.h"
00015
00016 class DataComponentEditor;
00017 class ComboBoxEnum;
00018
00019 class DataComponentProperty : public QObject {
00020 public:
00021 using AfterChange = std::function<void()>;
00022
00023 DataComponentProperty(
00024 PropertyEditorGenesys* editor,
00025 SimulationControl* property,
00026 bool necessaryConfig,
00027 AfterChange afterChange = {}
00028);
00029
00030 ~DataComponentProperty() override = default;
00031
00032 public:
00033 void open_window();
00034 void config_values();
00035 bool isInList(const std::string& value) const;
00036
00037 private:
00038 void _notifyChanged();
00039
00040 private Q_SLOTS:
00041 void addElement();
00042 void removeElement();
00043 void editProperty();
00044
00045 private:
00046 QWidget* _window = nullptr;
00047 QTreeWidget* _view = nullptr;
00048 QPushButton* _add = nullptr;
00049 QPushButton* _remove = nullptr;
00050 QPushButton* _edit = nullptr;
00051 QInputDialog* _confirmation = nullptr;
00052
00053 PropertyEditorGenesys* _editor = nullptr;
00054 SimulationControl* _property = nullptr;
00055 AfterChange _afterChange;
00056 };
00057
00058 #endif // DATACOMPONENTPROPERTY_H

```

## 16.75 ObjectPropertyBrowser.h

```

00001 #ifndef OBJECTPROPERTYBROWSER_H
00002 #define OBJECTPROPERTYBROWSER_H
00003
00004 #include <map>
00005 #include <vector>
00006 #include <set>
00007 #include <functional>
00008
00009 #include <QObject>
00010 #include <QBrush>
00011 #include <QPen>
00012 #include <QPointer>
00013 #include <QMap>
00014 #include <QSet>
00015 #include <QStringList>
00016 #include <QContextMenuEvent>
00017 #include <QGraphicsItem>
00018 #include <QKeyEvent>
00019 #include <QMouseEvent>
00020 #include <QVariant>
00021
00022 #include "qtpropertybrowser/qttreepropertybrowser.h"
00023 #include "qtpropertybrowser/qtvariantproperty.h"
00024 #include "qtpropertybrowser/qtpropertymanager.h"
00025 #include "qtpropertybrowser/qteditorfactory.h"
00026
00027 #include "DataComponentProperty.h"
00028 #include "DataComponentEditor.h"
00029 #include "ComboBoxEnum.h"
00030
00031 #include "../kernel/simulator/model/ModelDataDefinition.h"
00032 #include "kernel/simulator/PropertyGenesys.h"
00033 #include "kernel/simulator/GenesysPropertyIntrospection.h"
00034
00035 class ObjectPropertyBrowser : public QtTreePropertyBrowser {
00036 Q_OBJECT
00037
00038 public:
00039 explicit ObjectPropertyBrowser(QWidget* parent = nullptr);
00040 using ModelChangedCallback = std::function<void()>;
00041
00042 void setActiveObject(
00043 QObject* obj,
00044 ModelDataDefinition* mdd = nullptr,
00045 const QSet<QString>& graphicallyRepresentedModelObjects = {},
00046 const QSet<QString>& editableModelObjects = {},
00047 PropertyEditorGenesys* peg = nullptr,
00048 std::map<SimulationControl*, DataComponentProperty*>* pl = nullptr,
00049 std::map<SimulationControl*, DataComponentEditor*>* peUI = nullptr,
00050 std::map<SimulationControl*, ComboBoxEnum*>* pb = nullptr
00051);
00052 void setActiveGraphicsItem(QGraphicsItem* item);
00053
00054 void clearCurrentlyConnectedObject();
00055 void setModelChangedCallback(ModelChangedCallback callback);
00056 bool isCommitPipelineBusy() const;
00057
00058 private:
00059 enum class ActiveMode {
00060 None,
00061 KernelObject,
00062 GraphicsItem
00063 };
00064
00065 enum class BindingKind {
00066 Kernel,
00067 GraphicsVariant,
00068 GraphicsEnum
00069 };
00070
00071 struct Binding {
00072 BindingKind kind = BindingKind::Kernel;
00073 ModelDataDefinition* owner = nullptr;
00074 SimulationControl* control = nullptr;
00075 GenesysPropertyDescriptor descriptor;
00076 bool isObjectSelector = false;
00077 bool isModelObjectAction = false;
00078 bool isObjectListAction = false;
00079 bool isMultilineTextAction = false;
00080 QGraphicsItem* graphicsItem = nullptr;
00081 bool graphicsRequiresCommit = false;
00082 std::function<void(const QVariant&)> graphicsVariantSetter;
00083 std::function<void(int)> graphicsEnumSetter;
00084 };
00085

```

```

00086 struct ModelObjectRelation {
00087 bool exists = false;
00088 bool internal = false;
00089 std::string key;
00090 ModelDataDefinition* dataDefinition = nullptr;
00091 };
00092
00093 struct ObjectListAction {
00094 enum class Kind {
00095 None,
00096 CreateNew,
00097 SetCurrentType
00098 };
00099
00100 Kind kind = Kind::None;
00101 std::string typeName;
00102 };
00103
00104 struct ModelObjectAction {
00105 enum class Kind {
00106 None,
00107 SetReference,
00108 CreateNew,
00109 SelectAndEdit,
00110 RemoveReference
00111 };
00112
00113 Kind kind = Kind::None;
00114 std::string objectValue;
00115 std::string typeName;
00116 };
00117
00118 private:
00119 // Ensure the property browser infrastructure is created exactly once.
00120 void _ensureBrowserInfrastructure();
00121 void _notifyModelChangeApplied();
00122 void _clearAll();
00123 void _rebuildProperties();
00124 void _populateKernelProperties(ModelDataDefinition* mdd);
00125 void _populateGraphicsProperties(QGraphicsItem* item);
00126 void _appendDescriptorRecursively(
00127 QtProperty* parent,
00128 SimulationControl* control,
00129 std::set<const SimulationControl*>& recursionPath,
00130 int depth = 0
00131);
00132
00133 QtProperty* _createLeafProperty(const GenesysPropertyDescriptor& desc);
00134 QtProperty* _createObjectListProperty(
00135 SimulationControl* control,
00136 const GenesysPropertyDescriptor& desc,
00137 std::set<const SimulationControl*>& recursionPath,
00138 int depth
00139);
00140 QtProperty* _createModelObjectActionProperty(const GenesysPropertyDescriptor& desc);
00141 bool _isObjectListProperty(const GenesysPropertyDescriptor& desc) const;
00142 bool _isKernelEditingEnabled(const GenesysPropertyDescriptor& desc) const;
00143 bool _hasGraphicalRepresentation(const GenesysPropertyDescriptor& desc) const;
00144 bool _isModelObjectActionProperty(const GenesysPropertyDescriptor& desc, SimulationControl*
control) const;
00145 ModelObjectRelation _relationForDataDefinition(ModelDataDefinition* dataDefinition) const;
00146 ModelDataDefinition* _referencedModelDataDefinition(const Binding& binding) const;
00147 QString _modelObjectTypeName(const GenesysPropertyDescriptor& desc) const;
00148 QString _defaultModelObjectName(const GenesysPropertyDescriptor& desc, const std::string&
concreteTypeName = "") const;
00149 bool _isRegisteredModelDataDefinition(ModelDataDefinition* dataDefinition) const;
00150 void _materializeAffectedModelDataDefinitions(ModelDataDefinition* referencedDataDefinition =
nullptr) const;
00151 bool _createNewListElementForProperty(QtProperty* property, const std::string& typeName = "");
00152 bool _setCurrentListElementTypeForProperty(QtProperty* property, const std::string& typeName);
00153 bool _createModelObjectForProperty(QtProperty* property, const std::string& typeName = "");
00154 bool _setModelObjectReferenceForProperty(QtProperty* property, const QString& objectName);
00155 bool _selectModelObjectForProperty(QtProperty* property);
00156 bool _removeModelObjectReferenceForProperty(QtProperty* property);
00157 bool _deleteModelObjectForProperty(QtProperty* property);
00158 bool _applyObjectListSelection(QtProperty* property, int value);
00159 bool _applyModelObjectReferenceSelection(QtProperty* property, int value);
00160 void _synchronizeGraphicalModelDataDefinitionsNow() const;
00161 QtVariantProperty* _createGraphicsVariantProperty(
00162 const QString& name,
00163 int variantType,
00164 const QVariant& value,
00165 std::function<void(const QVariant&)> setter,
00166 bool requiresCommit = false,
00167 bool enabled = true
00168);
00169 QtProperty* _createGraphicsEnumProperty(

```

```

00170 const QString& name,
00171 const QStringList& choices,
00172 int value,
00173 std::function<void(int)> setter,
00174 bool enabled = true
00175);
00176 void _appendCommonGraphicsProperties(QtProperty* group, QGraphicsItem* item);
00177 void _appendPenProperties(QtProperty* group, const QPen& pen, std::function<void(const QPen&)>
setter);
00178 void _appendBrushProperties(QtProperty* group, const QBrush& brush, std::function<void(const
QBrush&)> setter);
00179 void _notifyGraphicsChangeApplied(QGraphicsItem* item);
00180 QVariant _toVariant(const GenesysPropertyDescriptor& desc) const;
00181 std::string _fromVariant(const GenesysPropertyDescriptor& desc, const QVariant& value) const;
00182 int _enumIndexFor(const GenesysPropertyDescriptor& desc) const;
00183 QStringList _toQStringList(const std::vector<std::string>& values) const;
00184 bool _hasSpecializedEditor(const GenesysPropertyDescriptor& desc) const;
00185 QString _specializedEditorActionText(const GenesysPropertyDescriptor& desc) const;
00186
00187 bool _openSpecializedEditorForCurrentItem();
00188 bool _openSpecializedEditor(QtProperty* property);
00189 bool _openTextDialogEditor(const Binding& binding, bool compactEditor);
00190 bool _openMultilineTextDialogEditor(const Binding& binding);
00191 bool _createObjectForProperty(QtProperty* property);
00192
00193 protected:
00194 void keyPressEvent(QKeyEvent* event) override;
00195 void contextMenuEvent(QContextMenuEvent* event) override;
00196 void mouseDoubleClickEvent(QMouseEvent* event) override;
00197
00198 private:
00199 // Guard against reentrant callback chains during property rebuild and notification.
00200 bool _isRebuildingProperties = false;
00201 // Guard against nested model-change notifications triggered by editor callbacks.
00202 bool _isNotifyingModelChange = false;
00203 // Keep track of pending rebuild requests raised during guarded execution.
00204 bool _pendingRebuild = false;
00205 // Coalesce queued rebuild dispatches while one deferred rebuild request is already scheduled.
00206 bool _isDeferredRebuildScheduled = false;
00207 // Coalesce queued model-change callbacks while one deferred dispatch is already scheduled.
00208 bool _isDeferredModelChangedScheduled = false;
00209 // Prevent recursive handling when a multiline text action resets its combo-box selection.
00210 bool _isHandlingMultilineTextAction = false;
00211
00212 private:
00213 // Execute property rebuild with reentrancy suppression and deferred retry support.
00214 void _rebuildPropertiesGuarded();
00215 // Request a deferred rebuild after current signal/slot stack finishes.
00216 void _scheduleDeferredRebuild();
00217 // Request a deferred model-changed callback after current signal/slot stack finishes.
00218 void _scheduleDeferredModelChangedCallback();
00219 // Check whether active editor bindings are currently valid before mutating model state.
00220 bool _hasValidActiveBindingContext(QtProperty* property = nullptr) const;
00221
00222 private:
00223 // Track the selected graphical object safely in case Qt destroys it during callbacks.
00224 QPointer<QObject> _graphicalObject = nullptr;
00225 QGraphicsItem* _graphicalItem = nullptr;
00226 ModelDataDefinition* _modelObject = nullptr;
00227 ActiveMode _activeMode = ActiveMode::None;
00228 QSet<QString> _graphicallyRepresentedModelObjects;
00229 QSet<QString> _editableModelObjects;
00230 bool _activeKernelObjectReadOnly = false;
00231 PropertyEditorGenesys* _propertyEditor = nullptr;
00232
00233 std::map<SimulationControl*, DataComponentProperty*> _propertyList = nullptr;
00234 std::map<SimulationControl*, DataComponentEditor*> _propertyEditorUI = nullptr;
00235 std::map<SimulationControl*, ComboBoxEnum*> _propertyBox = nullptr;
00236
00237 QtVariantPropertyManager* _variantManager = nullptr;
00238 QtGroupPropertyManager* _groupManager = nullptr;
00239 QtEnumPropertyManager* _enumManager = nullptr;
00240
00241 QtVariantEditorFactory* _variantFactory = nullptr;
00242 QtEnumEditorFactory* _enumFactory = nullptr;
00243 // Track whether factories were already bound to managers in the browser.
00244 bool _browserInfrastructureBound = false;
00245
00246 QMap<QtProperty*, Binding> _bindings;
00247 QMap<QtProperty*, QStringList> _enumNames;
00248 QMap<QtProperty*, std::vector<ObjectListAction>> _objectListActions;
00249 QMap<QtProperty*, std::vector<ModelObjectAction>> _modelObjectActions;
00250 QSet<QtProperty*> _pendingCommittedProperties;
00251 QMap<QtProperty*, QVariant> _pendingCommittedValues;
00252 ModelChangedCallback _modelChangedCallback;
00253
00254 private:

```

```

00255 bool _requiresCommitConfirmation(const Binding& binding) const;
00256 bool _applyVariantChange(QtProperty* property, const QVariant& value, bool committed);
00257 bool _applyGraphicsVariantChange(QtProperty* property, const QVariant& value, bool committed);
00258 bool _applyGraphicsEnumChange(QtProperty* property, int value);
00259
00260 private slots:
00261 void valueChanged(QtProperty *property, const QVariant &value);
00262 void enumValueChanged(QtProperty *property, int value);
00263 void onVariantEditorCommitted(QtProperty* property, const QVariant& committedValue = QVariant());
00264
00265 public slots:
00266 void objectUpdated();
00267 };
00268
00269 #endif // OBJECTPROPERTYBROWSER_H

```

## 16.76 PropertyEditor.h

```

00001 #ifndef PROPERTYEDITOR_H
00002 #define PROPERTYEDITOR_H
00003
00004 #include <QTreeWidget>
00005 #include <QObject>
00006
00007 class ModelDataDefinition;
00008
00009 class PropertyEditor : public QTreeWidget {
00010 public:
00011 PropertyEditor(QWidget *parent = nullptr);
00012 public:
00013 void setModelBlock(ModelDataDefinition* modelblock);
00014 };
00015
00016 #endif // PROPERTYEDITOR_H

```

## 16.77 qtbuttonpropertybrowser.h

```

00001 /*****
00002 **
00003 ** Copyright (C) 2016 The Qt Company Ltd.
00004 ** Contact: https://www.qt.io/licensing/
00005 **
00006 ** This file is part of the tools applications of the Qt Toolkit.
00007 **
00008 ** $QT_BEGIN_LICENSE:LGPL$
00009 ** Commercial License Usage
00010 ** Licensees holding valid commercial Qt licenses may use this file in
00011 ** accordance with the commercial license agreement provided with the
00012 ** Software or, alternatively, in accordance with the terms contained in
00013 ** a written agreement between you and The Qt Company. For licensing terms
00014 ** and conditions see https://www.qt.io/terms-conditions. For further
00015 ** information use the contact form at https://www.qt.io/contact-us.
00016 **
00017 ** GNU Lesser General Public License Usage
00018 ** Alternatively, this file may be used under the terms of the GNU Lesser
00019 ** General Public License version 3 as published by the Free Software
00020 ** Foundation and appearing in the file LICENSE.LGPL3 included in the
00021 ** packaging of this file. Please review the following information to
00022 ** ensure the GNU Lesser General Public License version 3 requirements
00023 ** will be met: https://www.gnu.org/licenses/lgpl-3.0.html.
00024 **
00025 ** GNU General Public License Usage
00026 ** Alternatively, this file may be used under the terms of the GNU
00027 ** General Public License version 2.0 or (at your option) the GNU General
00028 ** Public license version 3 or any later version approved by the KDE Free
00029 ** Qt Foundation. The licenses are as published by the Free Software
00030 ** Foundation and appearing in the file LICENSE.GPL2 and LICENSE.GPL3
00031 ** included in the packaging of this file. Please review the following
00032 ** information to ensure the GNU General Public License requirements will
00033 ** be met: https://www.gnu.org/licenses/gpl-2.0.html and
00034 ** https://www.gnu.org/licenses/gpl-3.0.html.
00035 **
00036 ** $QT_END_LICENSE$
00037 **
00038 *****/
00039
00040 #ifndef QTBUTTONPROPERTYBROWSER_H
00041 #define QTBUTTONPROPERTYBROWSER_H
00042
00043 #include "qtpropertybrowser.h"
00044
00045 QT_BEGIN_NAMESPACE

```



```

00046
00047 class QtButtonPropertyBrowserPrivate;
00048
00049 class QT_QTPROPERTYBROWSER_EXPORT QtButtonPropertyBrowser : public QtAbstractPropertyBrowser
00050 {
00051 Q_OBJECT
00052 public:
00053
00054 QtButtonPropertyBrowser(QWidget *parent = 0);
00055 ~QtButtonPropertyBrowser();
00056
00057 void setExpanded(QtBrowserItem *item, bool expanded);
00058 bool isExpanded(QtBrowserItem *item) const;
00059
00060 Q_SIGNALS:
00061
00062 void collapsed(QtBrowserItem *item);
00063 void expanded(QtBrowserItem *item);
00064
00065 protected:
00066 void itemInserted(QtBrowserItem *item, QtBrowserItem *afterItem) override;
00067 void itemRemoved(QtBrowserItem *item) override;
00068 void itemChanged(QtBrowserItem *item) override;
00069
00070 private:
00071
00072 QScopedPointer<QtButtonPropertyBrowserPrivate> d_ptr;
00073 Q_DECLARE_PRIVATE(QtButtonPropertyBrowser)
00074 Q_DISABLE_COPY_MOVE(QtButtonPropertyBrowser)
00075 Q_PRIVATE_SLOT(d_func(), void slotUpdate())
00076 Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed())
00077 Q_PRIVATE_SLOT(d_func(), void slotToggled(bool))
00078
00079 };
00080
00081 QT_END_NAMESPACE
00082
00083 #endif

```

## 16.78 qteditorfactory.h

```

00001 /*****
00002 **
00003 ** Copyright (C) 2016 The Qt Company Ltd.
00004 ** Contact: https://www.qt.io/licensing/
00005 **
00006 ** This file is part of the tools applications of the Qt Toolkit.
00007 **
00008 ** $QT_BEGIN_LICENSE:LGPL$
00009 ** Commercial License Usage
00010 ** Licensees holding valid commercial Qt licenses may use this file in
00011 ** accordance with the commercial license agreement provided with the
00012 ** Software or, alternatively, in accordance with the terms contained in
00013 ** a written agreement between you and The Qt Company. For licensing terms
00014 ** and conditions see https://www.qt.io/terms-conditions. For further
00015 ** information use the contact form at https://www.qt.io/contact-us.
00016 **
00017 ** GNU Lesser General Public License Usage
00018 ** Alternatively, this file may be used under the terms of the GNU Lesser
00019 ** General Public License version 3 as published by the Free Software
00020 ** Foundation and appearing in the file LICENSE.LGPL3 included in the
00021 ** packaging of this file. Please review the following information to
00022 ** ensure the GNU Lesser General Public License version 3 requirements
00023 ** will be met: https://www.gnu.org/licenses/lgpl-3.0.html.
00024 **
00025 ** GNU General Public License Usage
00026 ** Alternatively, this file may be used under the terms of the GNU
00027 ** General Public License version 2.0 or (at your option) the GNU General
00028 ** Public license version 3 or any later version approved by the KDE Free
00029 ** Qt Foundation. The licenses are as published by the Free Software
00030 ** Foundation and appearing in the file LICENSE.GPL2 and LICENSE.GPL3
00031 ** included in the packaging of this file. Please review the following
00032 ** information to ensure the GNU General Public License requirements will
00033 ** be met: https://www.gnu.org/licenses/gpl-2.0.html and
00034 ** https://www.gnu.org/licenses/gpl-3.0.html.
00035 **
00036 ** $QT_END_LICENSE$
00037 **
00038 *****/
00039
00040 #ifndef QTEDITORFACTORY_H
00041 #define QTEDITORFACTORY_H
00042
00043 #include "qtpropertymanager.h"
00044

```

```

00045 QT_BEGIN_NAMESPACE
00046
00047 class QRegularExpression;
00048
00049 class QtSpinBoxFactoryPrivate;
00050
00051 class QT_QTPROPERTYBROWSER_EXPORT QtSpinBoxFactory : public
 QtAbstractEditorFactory<QtIntPropertyManager>
00052 {
00053 Q_OBJECT
00054 public:
00055 QtSpinBoxFactory(QObject *parent = 0);
00056 ~QtSpinBoxFactory();
00057 protected:
00058 void connectPropertyManager(QtIntPropertyManager *manager) override;
00059 QWidget *createEditor(QtIntPropertyManager *manager, QtProperty *property,
00060 QWidget *parent) override;
00061 void disconnectPropertyManager(QtIntPropertyManager *manager) override;
00062 private:
00063 QScopedPointer<QtSpinBoxFactoryPrivate> d_ptr;
00064 Q_DECLARE_PRIVATE(QtSpinBoxFactory)
00065 Q_DISABLE_COPY_MOVE(QtSpinBoxFactory)
00066 Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, int))
00067 Q_PRIVATE_SLOT(d_func(), void slotRangeChanged(QtProperty *, int, int))
00068 Q_PRIVATE_SLOT(d_func(), void slotSingleStepChanged(QtProperty *, int))
00069 Q_PRIVATE_SLOT(d_func(), void slotSetValue(int))
00070 Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00071 };
00072
00073 class QtSliderFactoryPrivate;
00074
00075 class QT_QTPROPERTYBROWSER_EXPORT QtSliderFactory : public
 QtAbstractEditorFactory<QtIntPropertyManager>
00076 {
00077 Q_OBJECT
00078 public:
00079 QtSliderFactory(QObject *parent = 0);
00080 ~QtSliderFactory();
00081 protected:
00082 void connectPropertyManager(QtIntPropertyManager *manager) override;
00083 QWidget *createEditor(QtIntPropertyManager *manager, QtProperty *property,
00084 QWidget *parent) override;
00085 void disconnectPropertyManager(QtIntPropertyManager *manager) override;
00086 private:
00087 QScopedPointer<QtSliderFactoryPrivate> d_ptr;
00088 Q_DECLARE_PRIVATE(QtSliderFactory)
00089 Q_DISABLE_COPY_MOVE(QtSliderFactory)
00090 Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, int))
00091 Q_PRIVATE_SLOT(d_func(), void slotRangeChanged(QtProperty *, int, int))
00092 Q_PRIVATE_SLOT(d_func(), void slotSingleStepChanged(QtProperty *, int))
00093 Q_PRIVATE_SLOT(d_func(), void slotSetValue(int))
00094 Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00095 };
00096
00097 class QtScrollBarFactoryPrivate;
00098
00099 class QT_QTPROPERTYBROWSER_EXPORT QtScrollBarFactory : public
 QtAbstractEditorFactory<QtIntPropertyManager>
00100 {
00101 Q_OBJECT
00102 public:
00103 QtScrollBarFactory(QObject *parent = 0);
00104 ~QtScrollBarFactory();
00105 protected:
00106 void connectPropertyManager(QtIntPropertyManager *manager) override;
00107 QWidget *createEditor(QtIntPropertyManager *manager, QtProperty *property,
00108 QWidget *parent) override;
00109 void disconnectPropertyManager(QtIntPropertyManager *manager) override;
00110 private:
00111 QScopedPointer<QtScrollBarFactoryPrivate> d_ptr;
00112 Q_DECLARE_PRIVATE(QtScrollBarFactory)
00113 Q_DISABLE_COPY_MOVE(QtScrollBarFactory)
00114 Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, int))
00115 Q_PRIVATE_SLOT(d_func(), void slotRangeChanged(QtProperty *, int, int))
00116 Q_PRIVATE_SLOT(d_func(), void slotSingleStepChanged(QtProperty *, int))
00117 Q_PRIVATE_SLOT(d_func(), void slotSetValue(int))
00118 Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00119 };
00120
00121 class QtCheckBoxFactoryPrivate;
00122
00123 class QT_QTPROPERTYBROWSER_EXPORT QtCheckBoxFactory : public
 QtAbstractEditorFactory<QtBoolPropertyManager>
00124 {
00125 Q_OBJECT
00126 public:
00127 QtCheckBoxFactory(QObject *parent = 0);

```

```

00128 ~QtCheckBoxFactory();
00129 protected:
00130 void connectPropertyManager(QtBoolPropertyManager *manager) override;
00131 QWidget *createEditor(QtBoolPropertyManager *manager, QtProperty *property,
00132 QWidget *parent) override;
00133 void disconnectPropertyManager(QtBoolPropertyManager *manager) override;
00134 private:
00135 QScopedPointer<QtCheckBoxFactoryPrivate> d_ptr;
00136 Q_DECLARE_PRIVATE(QtCheckBoxFactory)
00137 Q_DISABLE_COPY_MOVE(QtCheckBoxFactory)
00138 Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, bool))
00139 Q_PRIVATE_SLOT(d_func(), void slotSetValues(bool))
00140 Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00141 };
00142
00143 class QtDoubleSpinBoxFactoryPrivate;
00144
00145 class QT_QTPROPERTYBROWSER_EXPORT QtDoubleSpinBoxFactory : public
 QtAbstractEditorFactory<QtDoublePropertyManager>
00146 {
00147 Q_OBJECT
00148 public:
00149 QtDoubleSpinBoxFactory(QObject *parent = 0);
00150 ~QtDoubleSpinBoxFactory();
00151 protected:
00152 void connectPropertyManager(QtDoublePropertyManager *manager) override;
00153 QWidget *createEditor(QtDoublePropertyManager *manager, QtProperty *property,
00154 QWidget *parent) override;
00155 void disconnectPropertyManager(QtDoublePropertyManager *manager) override;
00156 private:
00157 QScopedPointer<QtDoubleSpinBoxFactoryPrivate> d_ptr;
00158 Q_DECLARE_PRIVATE(QtDoubleSpinBoxFactory)
00159 Q_DISABLE_COPY_MOVE(QtDoubleSpinBoxFactory)
00160 Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, double))
00161 Q_PRIVATE_SLOT(d_func(), void slotRangeChanged(QtProperty *, double, double))
00162 Q_PRIVATE_SLOT(d_func(), void slotSingleStepChanged(QtProperty *, double))
00163 Q_PRIVATE_SLOT(d_func(), void slotDecimalsChanged(QtProperty *, int))
00164 Q_PRIVATE_SLOT(d_func(), void slotSetValues(double))
00165 Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00166 };
00167
00168 class QtLineEditFactoryPrivate;
00169
00170 class QT_QTPROPERTYBROWSER_EXPORT QtLineEditFactory : public
 QtAbstractEditorFactory<QtStringPropertyManager>
00171 {
00172 Q_OBJECT
00173 public:
00174 QtLineEditFactory(QObject *parent = 0);
00175 ~QtLineEditFactory();
00176 protected:
00177 void connectPropertyManager(QtStringPropertyManager *manager) override;
00178 QWidget *createEditor(QtStringPropertyManager *manager, QtProperty *property,
00179 QWidget *parent) override;
00180 void disconnectPropertyManager(QtStringPropertyManager *manager) override;
00181 private:
00182 QScopedPointer<QtLineEditFactoryPrivate> d_ptr;
00183 Q_DECLARE_PRIVATE(QtLineEditFactory)
00184 Q_DISABLE_COPY_MOVE(QtLineEditFactory)
00185 Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, const QString &))
00186 Q_PRIVATE_SLOT(d_func(), void slotRegExpChanged(QtProperty *, const QRegularExpression &))
00187 Q_PRIVATE_SLOT(d_func(), void slotSetValues(const QString &))
00188 Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00189 };
00190
00191 class QtDateEditFactoryPrivate;
00192
00193 class QT_QTPROPERTYBROWSER_EXPORT QtDateEditFactory : public
 QtAbstractEditorFactory<QtDatePropertyManager>
00194 {
00195 Q_OBJECT
00196 public:
00197 QtDateEditFactory(QObject *parent = 0);
00198 ~QtDateEditFactory();
00199 protected:
00200 void connectPropertyManager(QtDatePropertyManager *manager) override;
00201 QWidget *createEditor(QtDatePropertyManager *manager, QtProperty *property,
00202 QWidget *parent) override;
00203 void disconnectPropertyManager(QtDatePropertyManager *manager) override;
00204 private:
00205 QScopedPointer<QtDateEditFactoryPrivate> d_ptr;
00206 Q_DECLARE_PRIVATE(QtDateEditFactory)
00207 Q_DISABLE_COPY_MOVE(QtDateEditFactory)
00208 Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, QDate))
00209 Q_PRIVATE_SLOT(d_func(), void slotRangeChanged(QtProperty *, QDate, QDate))
00210 Q_PRIVATE_SLOT(d_func(), void slotSetValues(QDate))
00211 Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))

```

```

00212 };
00213
00214 class QtTimeEditFactoryPrivate;
00215
00216 class QT_QTPROPERTYBROWSER_EXPORT QtTimeEditFactory : public
 QtAbstractEditorFactory<QtTimePropertyManager>
00217 {
00218 Q_OBJECT
00219 public:
00220 QtTimeEditFactory(QObject *parent = 0);
00221 ~QtTimeEditFactory();
00222 protected:
00223 void connectPropertyManager(QtTimePropertyManager *manager) override;
00224 QWidget *createEditor(QtTimePropertyManager *manager, QtProperty *property,
00225 QWidget *parent) override;
00226 void disconnectPropertyManager(QtTimePropertyManager *manager) override;
00227 private:
00228 QScopedPointer<QtTimeEditFactoryPrivate> d_ptr;
00229 Q_DECLARE_PRIVATE(QtTimeEditFactory)
00230 Q_DISABLE_COPY_MOVE(QtTimeEditFactory)
00231 Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, QTime))
00232 Q_PRIVATE_SLOT(d_func(), void slotSetValue(QTime))
00233 Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00234 };
00235
00236 class QtDateTimeEditFactoryPrivate;
00237
00238 class QT_QTPROPERTYBROWSER_EXPORT QtDateTimeEditFactory : public
 QtAbstractEditorFactory<QtDateTimePropertyManager>
00239 {
00240 Q_OBJECT
00241 public:
00242 QtDateTimeEditFactory(QObject *parent = 0);
00243 ~QtDateTimeEditFactory();
00244 protected:
00245 void connectPropertyManager(QtDateTimePropertyManager *manager) override;
00246 QWidget *createEditor(QtDateTimePropertyManager *manager, QtProperty *property,
00247 QWidget *parent) override;
00248 void disconnectPropertyManager(QtDateTimePropertyManager *manager) override;
00249 private:
00250 QScopedPointer<QtDateTimeEditFactoryPrivate> d_ptr;
00251 Q_DECLARE_PRIVATE(QtDateTimeEditFactory)
00252 Q_DISABLE_COPY_MOVE(QtDateTimeEditFactory)
00253 Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, const QDateTime &))
00254 Q_PRIVATE_SLOT(d_func(), void slotSetValue(const QDateTime &))
00255 Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00256 };
00257
00258 class QtKeySequenceEditorFactoryPrivate;
00259
00260 class QT_QTPROPERTYBROWSER_EXPORT QtKeySequenceEditorFactory : public
 QtAbstractEditorFactory<QtKeySequencePropertyManager>
00261 {
00262 Q_OBJECT
00263 public:
00264 QtKeySequenceEditorFactory(QObject *parent = 0);
00265 ~QtKeySequenceEditorFactory();
00266 protected:
00267 void connectPropertyManager(QtKeySequencePropertyManager *manager) override;
00268 QWidget *createEditor(QtKeySequencePropertyManager *manager, QtProperty *property,
00269 QWidget *parent) override;
00270 void disconnectPropertyManager(QtKeySequencePropertyManager *manager) override;
00271 private:
00272 QScopedPointer<QtKeySequenceEditorFactoryPrivate> d_ptr;
00273 Q_DECLARE_PRIVATE(QtKeySequenceEditorFactory)
00274 Q_DISABLE_COPY_MOVE(QtKeySequenceEditorFactory)
00275 Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, const QKeySequence &))
00276 Q_PRIVATE_SLOT(d_func(), void slotSetValue(const QKeySequence &))
00277 Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00278 };
00279
00280 class QtCharEditorFactoryPrivate;
00281
00282 class QT_QTPROPERTYBROWSER_EXPORT QtCharEditorFactory : public
 QtAbstractEditorFactory<QtCharPropertyManager>
00283 {
00284 Q_OBJECT
00285 public:
00286 QtCharEditorFactory(QObject *parent = 0);
00287 ~QtCharEditorFactory();
00288 protected:
00289 void connectPropertyManager(QtCharPropertyManager *manager) override;
00290 QWidget *createEditor(QtCharPropertyManager *manager, QtProperty *property,
00291 QWidget *parent) override;
00292 void disconnectPropertyManager(QtCharPropertyManager *manager) override;
00293 private:
00294 QScopedPointer<QtCharEditorFactoryPrivate> d_ptr;

```

```

00295 Q_DECLARE_PRIVATE(QtCharEditorFactory)
00296 Q_DISABLE_COPY_MOVE(QtCharEditorFactory)
00297 Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, const QChar &))
00298 Q_PRIVATE_SLOT(d_func(), void slotSetValue(const QChar &))
00299 Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00300 };
00301
00302 class QtEnumEditorFactoryPrivate;
00303
00304 class QT_QTPROPERTYBROWSER_EXPORT QtEnumEditorFactory : public
 QtAbstractEditorFactory<QtEnumPropertyManager>
00305 {
00306 Q_OBJECT
00307 public:
00308 QtEnumEditorFactory(QObject *parent = 0);
00309 ~QtEnumEditorFactory();
00310 protected:
00311 void connectPropertyManager(QtEnumPropertyManager *manager) override;
00312 QWidget *createEditor(QtEnumPropertyManager *manager, QtProperty *property,
00313 QWidget *parent) override;
00314 void disconnectPropertyManager(QtEnumPropertyManager *manager) override;
00315 private:
00316 QScopedPointer<QtEnumEditorFactoryPrivate> d_ptr;
00317 Q_DECLARE_PRIVATE(QtEnumEditorFactory)
00318 Q_DISABLE_COPY_MOVE(QtEnumEditorFactory)
00319 Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, int))
00320 Q_PRIVATE_SLOT(d_func(), void slotEnumNamesChanged(QtProperty *,
00321 const QStringList &))
00322 Q_PRIVATE_SLOT(d_func(), void slotEnumIconsChanged(QtProperty *,
00323 const QMap<int, QIcon> &))
00324 Q_PRIVATE_SLOT(d_func(), void slotSetValue(int))
00325 Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00326 };
00327
00328 class QtCursorEditorFactoryPrivate;
00329
00330 class QT_QTPROPERTYBROWSER_EXPORT QtCursorEditorFactory : public
 QtAbstractEditorFactory<QtCursorPropertyManager>
00331 {
00332 Q_OBJECT
00333 public:
00334 QtCursorEditorFactory(QObject *parent = 0);
00335 ~QtCursorEditorFactory();
00336 protected:
00337 void connectPropertyManager(QtCursorPropertyManager *manager) override;
00338 QWidget *createEditor(QtCursorPropertyManager *manager, QtProperty *property,
00339 QWidget *parent) override;
00340 void disconnectPropertyManager(QtCursorPropertyManager *manager) override;
00341 private:
00342 QScopedPointer<QtCursorEditorFactoryPrivate> d_ptr;
00343 Q_DECLARE_PRIVATE(QtCursorEditorFactory)
00344 Q_DISABLE_COPY_MOVE(QtCursorEditorFactory)
00345 Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, const QCursor &))
00346 Q_PRIVATE_SLOT(d_func(), void slotEnumChanged(QtProperty *, int))
00347 Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00348 };
00349
00350 class QtColorEditorFactoryPrivate;
00351
00352 class QT_QTPROPERTYBROWSER_EXPORT QtColorEditorFactory : public
 QtAbstractEditorFactory<QtColorPropertyManager>
00353 {
00354 Q_OBJECT
00355 public:
00356 QtColorEditorFactory(QObject *parent = 0);
00357 ~QtColorEditorFactory();
00358 protected:
00359 void connectPropertyManager(QtColorPropertyManager *manager) override;
00360 QWidget *createEditor(QtColorPropertyManager *manager, QtProperty *property,
00361 QWidget *parent) override;
00362 void disconnectPropertyManager(QtColorPropertyManager *manager) override;
00363 private:
00364 QScopedPointer<QtColorEditorFactoryPrivate> d_ptr;
00365 Q_DECLARE_PRIVATE(QtColorEditorFactory)
00366 Q_DISABLE_COPY_MOVE(QtColorEditorFactory)
00367 Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, const QColor &))
00368 Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00369 Q_PRIVATE_SLOT(d_func(), void slotSetValue(const QColor &))
00370 };
00371
00372 class QtFontEditorFactoryPrivate;
00373
00374 class QT_QTPROPERTYBROWSER_EXPORT QtFontEditorFactory : public
 QtAbstractEditorFactory<QtFontPropertyManager>
00375 {
00376 Q_OBJECT
00377 public:

```

```

00378 QFontEditorFactory(QObject *parent = 0);
00379 ~QFontEditorFactory();
00380 protected:
00381 void connectPropertyManager(QFontPropertyManager *manager) override;
00382 QWidget *createEditor(QFontPropertyManager *manager, QtProperty *property,
00383 QWidget *parent) override;
00384 void disconnectPropertyManager(QFontPropertyManager *manager) override;
00385 private:
00386 QScopedPointer<QFontEditorFactoryPrivate> d_ptr;
00387 Q_DECLARE_PRIVATE(QFontEditorFactory)
00388 Q_DISABLE_COPY_MOVE(QFontEditorFactory)
00389 Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, const QFont &))
00390 Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00391 Q_PRIVATE_SLOT(d_func(), void slotSetValue(const QFont &))
00392 };
00393
00394 QT_END_NAMESPACE
00395
00396 #endif

```

## 16.79 qtgroupboxpropertybrowser.h

```

00001 /*****
00002 **
00003 ** Copyright (C) 2016 The Qt Company Ltd.
00004 ** Contact: https://www.qt.io/licensing/
00005 **
00006 ** This file is part of the tools applications of the Qt Toolkit.
00007 **
00008 ** $QT_BEGIN_LICENSE:LGPL$
00009 ** Commercial License Usage
00010 ** Licensees holding valid commercial Qt licenses may use this file in
00011 ** accordance with the commercial license agreement provided with the
00012 ** Software or, alternatively, in accordance with the terms contained in
00013 ** a written agreement between you and The Qt Company. For licensing terms
00014 ** and conditions see https://www.qt.io/terms-conditions. For further
00015 ** information use the contact form at https://www.qt.io/contact-us.
00016 **
00017 ** GNU Lesser General Public License Usage
00018 ** Alternatively, this file may be used under the terms of the GNU Lesser
00019 ** General Public License version 3 as published by the Free Software
00020 ** Foundation and appearing in the file LICENSE.LGPL3 included in the
00021 ** packaging of this file. Please review the following information to
00022 ** ensure the GNU Lesser General Public License version 3 requirements
00023 ** will be met: https://www.gnu.org/licenses/lgpl-3.0.html.
00024 **
00025 ** GNU General Public License Usage
00026 ** Alternatively, this file may be used under the terms of the GNU
00027 ** General Public License version 2.0 or (at your option) the GNU General
00028 ** Public license version 3 or any later version approved by the KDE Free
00029 ** Qt Foundation. The licenses are as published by the Free Software
00030 ** Foundation and appearing in the file LICENSE.GPL2 and LICENSE.GPL3
00031 ** included in the packaging of this file. Please review the following
00032 ** information to ensure the GNU General Public License requirements will
00033 ** be met: https://www.gnu.org/licenses/gpl-2.0.html and
00034 ** https://www.gnu.org/licenses/gpl-3.0.html.
00035 **
00036 ** $QT_END_LICENSE$
00037 **
00038 *****/
00039
00040 #ifndef QTGROUPBOXPROPERTYBROWSER_H
00041 #define QTGROUPBOXPROPERTYBROWSER_H
00042
00043 #include "qtpropertybrowser.h"
00044
00045 QT_BEGIN_NAMESPACE
00046
00047 class QtGroupBoxPropertyBrowserPrivate;
00048
00049 class QT_QTPROPERTYBROWSER_EXPORT QtGroupBoxPropertyBrowser : public QtAbstractPropertyBrowser
00050 {
00051 Q_OBJECT
00052 public:
00053 QtGroupBoxPropertyBrowser(QWidget *parent = 0);
00054 ~QtGroupBoxPropertyBrowser();
00055
00056 protected:
00057 void itemInserted(QtBrowserItem *item, QtBrowserItem *afterItem) override;
00058 void itemRemoved(QtBrowserItem *item) override;
00059 void itemChanged(QtBrowserItem *item) override;
00060
00061 private:
00062
00063

```

```

00064 QScopedPointer<QtGroupBoxPropertyBrowserPrivate> d_ptr;
00065 Q_DECLARE_PRIVATE(QtGroupBoxPropertyBrowser)
00066 Q_DISABLE_COPY_MOVE(QtGroupBoxPropertyBrowser)
00067 Q_PRIVATE_SLOT(d_func(), void slotUpdate())
00068 Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed())
00069
00070 };
00071
00072 QT_END_NAMESPACE
00073
00074 #endif

```

## 16.80 qtpropertybrowser.h

```

00001 /*****
00002 **
00003 ** Copyright (C) 2016 The Qt Company Ltd.
00004 ** Contact: https://www.qt.io/licensing/
00005 **
00006 ** This file is part of the tools applications of the Qt Toolkit.
00007 **
00008 ** $QT_BEGIN_LICENSE:LGPL$
00009 ** Commercial License Usage
00010 ** Licensees holding valid commercial Qt licenses may use this file in
00011 ** accordance with the commercial license agreement provided with the
00012 ** Software or, alternatively, in accordance with the terms contained in
00013 ** a written agreement between you and The Qt Company. For licensing terms
00014 ** and conditions see https://www.qt.io/terms-conditions. For further
00015 ** information use the contact form at https://www.qt.io/contact-us.
00016 **
00017 ** GNU Lesser General Public License Usage
00018 ** Alternatively, this file may be used under the terms of the GNU Lesser
00019 ** General Public License version 3 as published by the Free Software
00020 ** Foundation and appearing in the file LICENSE.LGPL3 included in the
00021 ** packaging of this file. Please review the following information to
00022 ** ensure the GNU Lesser General Public License version 3 requirements
00023 ** will be met: https://www.gnu.org/licenses/lgpl-3.0.html.
00024 **
00025 ** GNU General Public License Usage
00026 ** Alternatively, this file may be used under the terms of the GNU
00027 ** General Public License version 2.0 or (at your option) the GNU General
00028 ** Public License version 3 or any later version approved by the KDE Free
00029 ** Qt Foundation. The licenses are as published by the Free Software
00030 ** Foundation and appearing in the file LICENSE.GPL2 and LICENSE.GPL3
00031 ** included in the packaging of this file. Please review the following
00032 ** information to ensure the GNU General Public License requirements will
00033 ** be met: https://www.gnu.org/licenses/gpl-2.0.html and
00034 ** https://www.gnu.org/licenses/gpl-3.0.html.
00035 **
00036 ** $QT_END_LICENSE$
00037 **
00038 *****/
00039
00040 #ifndef QTPROPERTYBROWSER_H
00041 #define QTPROPERTYBROWSER_H
00042
00043 #include <QtWidgets/QWidget>
00044 #include <QtCore/QSet>
00045
00046 QT_BEGIN_NAMESPACE
00047
00048 #if defined(Q_OS_WIN)
00049 # if !defined(QT_QTPROPERTYBROWSER_EXPORT) && !defined(QT_QTPROPERTYBROWSER_IMPORT)
00050 # define QT_QTPROPERTYBROWSER_EXPORT
00051 # elif defined(QT_QTPROPERTYBROWSER_IMPORT)
00052 # if defined(QT_QTPROPERTYBROWSER_EXPORT)
00053 # undef QT_QTPROPERTYBROWSER_EXPORT
00054 # endif
00055 # define QT_QTPROPERTYBROWSER_EXPORT __declspec(dllimport)
00056 # elif defined(QT_QTPROPERTYBROWSER_EXPORT)
00057 # undef QT_QTPROPERTYBROWSER_EXPORT
00058 # define QT_QTPROPERTYBROWSER_EXPORT __declspec(dllexport)
00059 # endif
00060 #else
00061 # define QT_QTPROPERTYBROWSER_EXPORT
00062 #endif
00063
00064 class QtAbstractPropertyManager;
00065 class QtPropertyPrivate;
00066
00067 class QT_QTPROPERTYBROWSER_EXPORT QtProperty
00068 {
00069 public:
00070 virtual ~QtProperty();
00071

```



```

00072 QList<QtProperty *> subProperties() const;
00073
00074 QtAbstractPropertyManager *propertyManager() const;
00075
00076 QString toolTip() const { return valueToolTip(); } // Compatibility
00077 QString valueToolTip() const;
00078 QString descriptionToolTip() const;
00079 QString statusTip() const;
00080 QString whatsThis() const;
00081 QString propertyName() const;
00082 bool isEnabled() const;
00083 bool isModified() const;
00084
00085 bool hasValue() const;
00086 QIcon valueIcon() const;
00087 QString valueText() const;
00088
00089 void setToolTip(const QString &text) { setValueToolTip(text); } // Compatibility
00090 void setValueToolTip(const QString &text);
00091 void setDescriptionToolTip(const QString &text);
00092 void setStatusTip(const QString &text);
00093 void setWhatsThis(const QString &text);
00094 void setPropertyName(const QString &text);
00095 void setEnabled(bool enable);
00096 void setModified(bool modified);
00097
00098 void addSubProperty(QtProperty *property);
00099 void insertSubProperty(QtProperty *property, QtProperty *afterProperty);
00100 void removeSubProperty(QtProperty *property);
00101 protected:
00102 explicit QtProperty(QtAbstractPropertyManager *manager);
00103 void propertyChanged();
00104 private:
00105 friend class QtAbstractPropertyManager;
00106 QScopedPointer<QtPropertyPrivate> d_ptr;
00107 };
00108
00109 class QtAbstractPropertyManagerPrivate;
00110
00111 class QT_QTPROPERTYBROWSER_EXPORT QtAbstractPropertyManager : public QObject
00112 {
00113 Q_OBJECT
00114 public:
00115
00116 explicit QtAbstractPropertyManager(QObject *parent = 0);
00117 ~QtAbstractPropertyManager();
00118
00119 QSet<QtProperty *> properties() const;
00120 void clear() const;
00121
00122 QtProperty *addProperty(const QString &name = QString());
00123 Q_SIGNALS:
00124
00125 void propertyInserted(QtProperty *property,
00126 QtProperty *parent, QtProperty *after);
00127 void propertyChanged(QtProperty *property);
00128 void propertyRemoved(QtProperty *property, QtProperty *parent);
00129 void propertyDestroyed(QtProperty *property);
00130 protected:
00131 virtual bool hasValue(const QtProperty *property) const;
00132 virtual QIcon valueIcon(const QtProperty *property) const;
00133 virtual QString valueText(const QtProperty *property) const;
00134 virtual void initializeProperty(QtProperty *property) = 0;
00135 virtual void uninitializeProperty(QtProperty *property);
00136 virtual QtProperty *createProperty();
00137 private:
00138 friend class QtProperty;
00139 QScopedPointer<QtAbstractPropertyManagerPrivate> d_ptr;
00140 Q_DECLARE_PRIVATE(QtAbstractPropertyManager)
00141 Q_DISABLE_COPY_MOVE(QtAbstractPropertyManager)
00142 };
00143
00144 class QT_QTPROPERTYBROWSER_EXPORT QtAbstractEditorFactoryBase : public QObject
00145 {
00146 Q_OBJECT
00147 public:
00148 virtual QWidget *createEditor(QtProperty *property, QWidget *parent) = 0;
00149 protected:
00150 explicit QtAbstractEditorFactoryBase(QObject *parent = 0)
00151 : QObject(parent) {}
00152
00153 virtual void breakConnection(QtAbstractPropertyManager *manager) = 0;
00154 protected Q_SLOTS:
00155 virtual void managerDestroyed(QObject *manager) = 0;
00156
00157 friend class QtAbstractPropertyBrowser;
00158 };

```



```

00159
00160 template <class PropertyManager>
00161 class QtAbstractEditorFactory : public QtAbstractEditorFactoryBase
00162 {
00163 public:
00164 explicit QtAbstractEditorFactory(QObject *parent) : QtAbstractEditorFactoryBase(parent) {}
00165 QWidget *createEditor(QtProperty *property, QWidget *parent) override
00166 {
00167 for (PropertyManager *manager : qAsConst(m_managers)) {
00168 if (manager == property->propertyManager()) {
00169 return createEditor(manager, property, parent);
00170 }
00171 }
00172 return 0;
00173 }
00174 void addPropertyManager(PropertyManager *manager)
00175 {
00176 if (m_managers.contains(manager))
00177 return;
00178 m_managers.insert(manager);
00179 connectPropertyManager(manager);
00180 connect(manager, SIGNAL(destroyed(QObject *)),
00181 this, SLOT(managerDestroyed(QObject *)));
00182 }
00183 void removePropertyManager(PropertyManager *manager)
00184 {
00185 if (!m_managers.contains(manager))
00186 return;
00187 disconnect(manager, SIGNAL(destroyed(QObject *)),
00188 this, SLOT(managerDestroyed(QObject *)));
00189 disconnectPropertyManager(manager);
00190 m_managers.remove(manager);
00191 }
00192 QSet<PropertyManager *> propertyManagers() const
00193 {
00194 return m_managers;
00195 }
00196 PropertyManager *propertyManager(QtProperty *property) const
00197 {
00198 QtAbstractPropertyManager *manager = property->propertyManager();
00199 for (PropertyManager *m : qAsConst(m_managers)) {
00200 if (m == manager) {
00201 return m;
00202 }
00203 }
00204 return 0;
00205 }
00206 protected:
00207 virtual void connectPropertyManager(PropertyManager *manager) = 0;
00208 virtual QWidget *createEditor(PropertyManager *manager, QtProperty *property,
00209 QWidget *parent) = 0;
00210 virtual void disconnectPropertyManager(PropertyManager *manager) = 0;
00211 virtual void managerDestroyed(QObject *manager) override
00212 {
00213 for (PropertyManager *m : qAsConst(m_managers)) {
00214 if (m == manager) {
00215 m_managers.remove(m);
00216 return;
00217 }
00218 }
00219 }
00220 private:
00221 void breakConnection(QtAbstractPropertyManager *manager) override
00222 {
00223 for (PropertyManager *m : qAsConst(m_managers)) {
00224 if (m == manager) {
00225 removePropertyManager(m);
00226 return;
00227 }
00228 }
00229 }
00230 private:
00231 QSet<PropertyManager *> m_managers;
00232 friend class QtAbstractPropertyEditor;
00233 };
00234
00235 class QtAbstractPropertyBrowser;
00236 class QtBrowserItemPrivate;
00237
00238 class QT_QTPROPERTYBROWSER_EXPORT QtBrowserItem
00239 {
00240 public:
00241 QtProperty *property() const;
00242 QtBrowserItem *parent() const;
00243 QList<QtBrowserItem *> children() const;
00244 QtAbstractPropertyBrowser *browser() const;
00245 private:

```

```

00246 explicit QtBrowserItem(QtAbstractPropertyBrowser *browser, QtProperty *property, QtBrowserItem
 *parent);
00247 ~QtBrowserItem();
00248 QScopedPointer<QtBrowserItemPrivate> d_ptr;
00249 friend class QtAbstractPropertyBrowserPrivate;
00250 };
00251
00252 class QtAbstractPropertyBrowserPrivate;
00253
00254 class QT_QTPROPERTYBROWSER_EXPORT QtAbstractPropertyBrowser : public QWidget
00255 {
00256 Q_OBJECT
00257 public:
00258 explicit QtAbstractPropertyBrowser(QWidget *parent = 0);
00259 ~QtAbstractPropertyBrowser();
00260
00261 QList<QtProperty *> properties() const;
00262 QList<QtBrowserItem *> items(QtProperty *property) const;
00263 QtBrowserItem *topLevelItem(QtProperty *property) const;
00264 QList<QtBrowserItem *> topLevelItems() const;
00265 void clear();
00266
00267 template <class PropertyManager>
00268 void setFactoryForManager(PropertyManager *manager,
00269 QtAbstractEditorFactory<PropertyManager> *factory) {
00270 QtAbstractPropertyManager *abstractManager = manager;
00271 QtAbstractEditorFactoryBase *abstractFactory = factory;
00272
00273 if (addFactory(abstractManager, abstractFactory))
00274 factory->addPropertyManager(manager);
00275 }
00276
00277 void unsetFactoryForManager(QtAbstractPropertyManager *manager);
00278
00279 QtBrowserItem *currentItem() const;
00280 void setCurrentItem(QtBrowserItem *);
00281
00282 Q_SIGNALS:
00283 void currentItemChanged(QtBrowserItem *);
00284
00285 public Q_SLOTS:
00286
00287 QtBrowserItem *addProperty(QtProperty *property);
00288 QtBrowserItem *insertProperty(QtProperty *property, QtProperty *afterProperty);
00289 void removeProperty(QtProperty *property);
00290
00291 protected:
00292
00293 virtual void itemInserted(QtBrowserItem *item, QtBrowserItem *afterItem) = 0;
00294 virtual void itemRemoved(QtBrowserItem *item) = 0;
00295 // can be tooltip, statustip, whatsthis, name, icon, text.
00296 virtual void itemChanged(QtBrowserItem *item) = 0;
00297
00298 virtual QWidget *createEditor(QtProperty *property, QWidget *parent);
00299 private:
00300
00301 bool addFactory(QtAbstractPropertyManager *abstractManager,
00302 QtAbstractEditorFactoryBase *abstractFactory);
00303
00304 QScopedPointer<QtAbstractPropertyBrowserPrivate> d_ptr;
00305 Q_DECLARE_PRIVATE(QtAbstractPropertyBrowser)
00306 Q_DISABLE_COPY_MOVE(QtAbstractPropertyBrowser)
00307 Q_PRIVATE_SLOT(d_func(), void slotPropertyInserted(QtProperty *,
00308 QtProperty *, QtProperty *))
00309 Q_PRIVATE_SLOT(d_func(), void slotPropertyRemoved(QtProperty *,
00310 QtProperty *))
00311 Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty *))
00312 Q_PRIVATE_SLOT(d_func(), void slotPropertyDataChanged(QtProperty *))
00313
00314 };
00315
00316 QT_END_NAMESPACE
00317
00318 #endif // QTPROPERTYBROWSER_H

```

## 16.81 qtpropertybrowserutils\_p.h

```

00001 /*****
00002 **
00003 ** Copyright (C) 2016 The Qt Company Ltd.
00004 ** Contact: https://www.qt.io/licensing/
00005 **
00006 ** This file is part of the tools applications of the Qt Toolkit.
00007 **

```

```

00008 ** $QT_BEGIN_LICENSE:LGPL$
00009 ** Commercial License Usage
00010 ** Licensees holding valid commercial Qt licenses may use this file in
00011 ** accordance with the commercial license agreement provided with the
00012 ** Software or, alternatively, in accordance with the terms contained in
00013 ** a written agreement between you and The Qt Company. For licensing terms
00014 ** and conditions see https://www.qt.io/terms-conditions. For further
00015 ** information use the contact form at https://www.qt.io/contact-us.
00016 **
00017 ** GNU Lesser General Public License Usage
00018 ** Alternatively, this file may be used under the terms of the GNU Lesser
00019 ** General Public License version 3 as published by the Free Software
00020 ** Foundation and appearing in the file LICENSE.LGPL3 included in the
00021 ** packaging of this file. Please review the following information to
00022 ** ensure the GNU Lesser General Public License version 3 requirements
00023 ** will be met: https://www.gnu.org/licenses/lgpl-3.0.html.
00024 **
00025 ** GNU General Public License Usage
00026 ** Alternatively, this file may be used under the terms of the GNU
00027 ** General Public License version 2.0 or (at your option) the GNU General
00028 ** Public License version 3 or any later version approved by the KDE Free
00029 ** Qt Foundation. The licenses are as published by the Free Software
00030 ** Foundation and appearing in the file LICENSE.GPL2 and LICENSE.GPL3
00031 ** included in the packaging of this file. Please review the following
00032 ** information to ensure the GNU General Public License requirements will
00033 ** be met: https://www.gnu.org/licenses/gpl-2.0.html and
00034 ** https://www.gnu.org/licenses/gpl-3.0.html.
00035 **
00036 ** $QT_END_LICENSE$
00037 **
00038 *****/
00039 //
00040 // W A R N I N G
00041 // -----
00042 //
00043 //
00044 // This file is not part of the Qt API. It exists for the convenience
00045 // of Qt Designer. This header
00046 // file may change from version to version without notice, or even be removed.
00047 //
00048 // We mean it.
00049 //
00050
00051 #ifndef QTPROPERTYBROWSERUTILS_H
00052 #define QTPROPERTYBROWSERUTILS_H
00053
00054 #include <QtCore/QMap>
00055 #include <QtGui/QIcon>
00056 #include <QtWidgets/QWidget>
00057 #include <QtWidgets/QPushButton>
00058 #include <QtWidgets/QDateEdit>
00059 #include <QtWidgets/QSpinBox>
00060 #include <QtCore/QDateTime>
00061 #include <QtCore/QStringList>
00062
00063 QT_BEGIN_NAMESPACE
00064
00065 class QMouseEvent;
00066 class QCheckBox;
00067 class QLineEdit;
00068
00069 class QtCursorDatabase
00070 {
00071 public:
00072 QtCursorDatabase();
00073 void clear();
00074
00075 QStringList cursorShapeNames() const;
00076 QMap<int, QIcon> cursorShapeIcons() const;
00077 QString cursorToShapeName(const QCursor& cursor) const;
00078 QIcon cursorToShapeIcon(const QCursor& cursor) const;
00079 int cursorToValue(const QCursor& cursor) const;
00080 #ifndef QT_NO_CURSOR
00081 QCursor valueToCursor(int value) const;
00082 #endif
00083 private:
00084 void appendCursor(Qt::CursorShape shape, const QString& name, const QIcon& icon);
00085 QStringList m_cursorNames;
00086 QMap<int, QIcon> m_cursorIcons;
00087 QMap<int, Qt::CursorShape> m_valueToCursorShape;
00088 QMap<Qt::CursorShape, int> m_cursorShapeToValue;
00089 };
00090
00091 class QtPropertyBrowserUtils
00092 {
00093 public:
00094 static QPixmap brushValuePixmap(const QBrush& b);

```

```

00095 static QIcon brushValueIcon(const QBrush& b);
00096 static QString colorValueText(const QColor& c);
00097 static QPixmap fontValuePixmap(const QFont& f);
00098 static QIcon fontValueIcon(const QFont& f);
00099 static QString fontValueText(const QFont& f);
00100 static QString dateFormat();
00101 static QString timeFormat();
00102 static QString dateTimeFormat();
00103 };
00104
00105 #pragma region QtBoolEdit
00106
00107 class QtBoolEdit : public QWidget {
00108 Q_OBJECT
00109 public:
00110 QtBoolEdit(QWidget* parent = 0);
00111
00112 bool textVisible() const { return m_textVisible; }
00113 void setTextVisible(bool textVisible);
00114
00115 Qt::CheckState checkState() const;
00116 void setCheckState(Qt::CheckState state);
00117
00118 bool isChecked() const;
00119 void setChecked(bool c);
00120 void setInitialChecked(bool c);
00121
00122 bool blockCheckBoxSignals(bool block);
00123
00124 Q_SIGNALS:
00125 void toggled(bool);
00126
00127 protected:
00128 void mousePressEvent(QMouseEvent* event) override;
00129
00130 private:
00131 QCheckBox* m_checkBox;
00132 QPushButton* m_button;
00133 bool m_textVisible;
00134 bool m_initialChecked;
00135 };
00136 #pragma endregion
00137
00138 #pragma region QtIntEdit
00139
00140 class QtIntEdit : public QWidget {
00141 Q_OBJECT
00142 public:
00143 QtIntEdit(QWidget* parent = Q_NULLPTR);
00144
00145 int value();
00146 void setValue(int);
00147 void setInitialValue(int);
00148 void setSingleStep(int);
00149 void setRange(int, int);
00150 void setKeyboardTracking(bool);
00151
00152 Q_SIGNALS:
00153 void valueChanged(int);
00154
00155 private:
00156 QSpinBox* m_spinbox;
00157 QPushButton* m_button;
00158 int m_initialvalue;
00159 };
00160 #pragma endregion
00161
00162 #pragma region QtDoubleEdit
00163
00164 class QtDoubleEdit : public QWidget {
00165 Q_OBJECT
00166 public:
00167 QtDoubleEdit(QWidget* parent = Q_NULLPTR);
00168
00169 int value();
00170 void setValue(double);
00171 void setInitialValue(double);
00172 void setDecimals(double);
00173 void setSingleStep(double);
00174 void setRange(double, double);
00175 void setKeyboardTracking(bool);
00176
00177 Q_SIGNALS:
00178 void valueChanged(double);
00179
00180 private:

```

```
00182 QDoubleSpinBox* m_doublespinbox;
00183 QPushButton* m_button;
00184 double m_initialvalue;
00185 };
00186
00187 #pragma endregion
00188
00189 #pragma region QtStringEdit
00190
00191 class QtStringEdit : public QWidget {
00192 Q_OBJECT
00193 public:
00194 QtStringEdit(QWidget* parent = Q_NULLPTR);
00195
00196 QString text();
00197 void setText(QString);
00198 void setInitialText(QString);
00199 const QValidator* validator();
00200 void setValidator(QValidator*);
00201
00202 Q_SIGNALS:
00203 void textChanged(QString);
00204 void textEdited(QString);
00205
00206 private:
00207 QLineEdit* m_lineedit;
00208 QPushButton* m_button;
00209 QString m_initialvalue;
00210 };
00211
00212 #pragma endregion
00213
00214 #pragma region QtDateEdit
00215
00216 class QtDateEdit : public QWidget {
00217 Q_OBJECT
00218 public:
00219 QtDateEdit(QWidget* parent = Q_NULLPTR);
00220
00221 QDate date();
00222 void setDate(QDate);
00223 void setInitialDate(QDate);
00224 void setDateRange(QDate, QDate);
00225 void setDisplayFormat(QString);
00226 void setCalendarPopup(bool);
00227
00228 Q_SIGNALS:
00229 void dateChanged(QDate);
00230
00231 private:
00232 QDateEdit* m_dateedit;
00233 QPushButton* m_button;
00234 QDate m_initialvalue;
00235 };
00236
00237 #pragma endregion
00238
00239 #pragma region QtTimeEdit
00240
00241 class QtTimeEdit : public QWidget {
00242 Q_OBJECT
00243 public:
00244 QtTimeEdit(QWidget* parent = Q_NULLPTR);
00245
00246 QTime time();
00247 void setTime(QTime);
00248 void setInitialTime(QTime);
00249 void setTimeRange(QTime, QTime);
00250 void setDisplayFormat(QString);
00251 void setCalendarPopup(bool);
00252
00253 Q_SIGNALS:
00254 void timeChanged(QTime);
00255
00256 private:
00257 QTimeEdit* m_timeedit;
00258 QPushButton* m_button;
00259 QTime m_initialvalue;
00260 };
00261
00262 #pragma endregion
00263
00264 #pragma region QDateTimeEdit
00265
00266 class QDateTimeEdit : public QWidget {
00267 Q_OBJECT
00268 public:
```

```

00269 QDateTimeEdit(QWidget* parent = Q_NULLPTR);
00270
00271 QDateTime dateTime();
00272 void setDateTime(const QDateTime&);
00273 void setInitialDateTime(const QDateTime&);
00274 void setDateTimeRange(const QDateTime&, const QDateTime&);
00275 void setDisplayFormat(const QString&);
00276 void setCalendarPopup(bool);
00277
00278 Q_SIGNALS:
00279 void dateTimeChanged(const QDateTime&);
00280
00281 private:
00282 QDateTimeEdit* m_timeedit;
00283 QPushButton* m_button;
00284 QDateTime m_initialvalue;
00285 };
00286
00287 #pragma endregion
00288
00289
00290 QT_END_NAMESPACE
00291
00292 #endif

```

## 16.82 qtpropertymanager.h

```

00001 /*****
00002 **
00003 ** Copyright (C) 2016 The Qt Company Ltd.
00004 ** Contact: https://www.qt.io/licensing/
00005 **
00006 ** This file is part of the tools applications of the Qt Toolkit.
00007 **
00008 ** $QT_BEGIN_LICENSE:LGPL$
00009 ** Commercial License Usage
00010 ** Licensees holding valid commercial Qt licenses may use this file in
00011 ** accordance with the commercial license agreement provided with the
00012 ** Software or, alternatively, in accordance with the terms contained in
00013 ** a written agreement between you and The Qt Company. For licensing terms
00014 ** and conditions see https://www.qt.io/terms-conditions. For further
00015 ** information use the contact form at https://www.qt.io/contact-us.
00016 **
00017 ** GNU Lesser General Public License Usage
00018 ** Alternatively, this file may be used under the terms of the GNU Lesser
00019 ** General Public License version 3 as published by the Free Software
00020 ** Foundation and appearing in the file LICENSE.LGPL3 included in the
00021 ** packaging of this file. Please review the following information to
00022 ** ensure the GNU Lesser General Public License version 3 requirements
00023 ** will be met: https://www.gnu.org/licenses/lgpl-3.0.html.
00024 **
00025 ** GNU General Public License Usage
00026 ** Alternatively, this file may be used under the terms of the GNU
00027 ** General Public License version 2.0 or (at your option) the GNU General
00028 ** Public License version 3 or any later version approved by the KDE Free
00029 ** Qt Foundation. The licenses are as published by the Free Software
00030 ** Foundation and appearing in the file LICENSE.GPL2 and LICENSE.GPL3
00031 ** included in the packaging of this file. Please review the following
00032 ** information to ensure the GNU General Public License requirements will
00033 ** be met: https://www.gnu.org/licenses/gpl-2.0.html and
00034 ** https://www.gnu.org/licenses/gpl-3.0.html.
00035 **
00036 ** $QT_END_LICENSE$
00037 **
00038 *****/
00039
00040 #ifndef QTPROPERTYMANAGER_H
00041 #define QTPROPERTYMANAGER_H
00042
00043 #include "qtpropertybrowser.h"
00044
00045 QT_BEGIN_NAMESPACE
00046
00047 class QDate;
00048 class QTime;
00049 class QDateTime;
00050 class QLocale;
00051 class QRegularExpression;
00052
00053 #pragma region QtGroupPropertyManager
00054
00055 class QT_QTPROPERTYBROWSER_EXPORT QtGroupPropertyManager : public QtAbstractPropertyManager
00056 {
00057 Q_OBJECT
00058 public:

```

```

00059 QtGroupPropertyManager(QObject* parent = 0);
00060 ~QtGroupPropertyManager();
00061
00062 protected:
00063 bool hasValue(const QtProperty* property) const override;
00064
00065 void initializeProperty(QtProperty* property) override;
00066 void uninitializedProperty(QtProperty* property) override;
00067 };
00068
00069 #pragma endregion
00070
00071 #pragma region QtIntPropertyManager
00072
00073 class QtIntPropertyManagerPrivate;
00074
00075 class QT_QTPROPERTYBROWSER_EXPORT QtIntPropertyManager : public QtAbstractPropertyManager
00076 {
00077 Q_OBJECT
00078 public:
00079 QtIntPropertyManager(QObject* parent = 0);
00080 ~QtIntPropertyManager();
00081
00082 int value(const QtProperty* property) const;
00083 int initialValue(const QtProperty* property) const;
00084 int minimum(const QtProperty* property) const;
00085 int maximum(const QtProperty* property) const;
00086 int singleStep(const QtProperty* property) const;
00087
00088 public Q_SLOTS:
00089 void setValue(QtProperty* property, int val);
00090 void setValueOnly(QtProperty* property, int val);
00091 void setMinimum(QtProperty* property, int minVal);
00092 void setMaximum(QtProperty* property, int maxVal);
00093 void setRange(QtProperty* property, int minVal, int maxVal);
00094 void setSingleStep(QtProperty* property, int step);
00095 Q_SIGNALS:
00096 void valueChanged(QtProperty* property, int val);
00097 void rangeChanged(QtProperty* property, int minVal, int maxVal);
00098 void singleStepChanged(QtProperty* property, int step);
00099 protected:
00100 QString valueText(const QtProperty* property) const override;
00101 void initializeProperty(QtProperty* property) override;
00102 void uninitializedProperty(QtProperty* property) override;
00103 private:
00104 QScopedPointer<QtIntPropertyManagerPrivate> d_ptr;
00105 Q_DECLARE_PRIVATE(QtIntPropertyManager)
00106 Q_DISABLE_COPY_MOVE(QtIntPropertyManager)
00107 };
00108 #pragma endregion
00109
00110 #pragma region QtBoolPropertyManager
00111
00112 class QtBoolPropertyManagerPrivate;
00113
00114 class QT_QTPROPERTYBROWSER_EXPORT QtBoolPropertyManager : public QtAbstractPropertyManager
00115 {
00116 Q_OBJECT
00117 public:
00118 QtBoolPropertyManager(QObject* parent = 0);
00119 ~QtBoolPropertyManager();
00120
00121 bool value(const QtProperty* property) const;
00122 bool initialValue(const QtProperty* property) const;
00123
00124 public Q_SLOTS:
00125 void setValue(QtProperty* property, bool val);
00126 void setValueOnly(QtProperty* property, bool val);
00127 Q_SIGNALS:
00128 void valueChanged(QtProperty* property, bool val);
00129 protected:
00130 QString valueText(const QtProperty* property) const override;
00131 QIcon valueIcon(const QtProperty* property) const override;
00132 void initializeProperty(QtProperty* property) override;
00133 void uninitializedProperty(QtProperty* property) override;
00134 private:
00135 QScopedPointer<QtBoolPropertyManagerPrivate> d_ptr;
00136 Q_DECLARE_PRIVATE(QtBoolPropertyManager)
00137 Q_DISABLE_COPY_MOVE(QtBoolPropertyManager)
00138 };
00139 #pragma endregion
00140
00141 #pragma region QtDoublePropertyManager
00142
00143 class QtDoublePropertyManagerPrivate;
00144
00145

```

```

00146 class QT_QTPROPERTYBROWSER_EXPORT QtDoublePropertyManager : public QtAbstractPropertyManager
00147 {
00148 Q_OBJECT
00149 public:
00150 QtDoublePropertyManager(QObject* parent = 0);
00151 ~QtDoublePropertyManager();
00152
00153 double value(const QtProperty* property) const;
00154 double initialValue(const QtProperty* property) const;
00155 double minimum(const QtProperty* property) const;
00156 double maximum(const QtProperty* property) const;
00157 double singleStep(const QtProperty* property) const;
00158 int decimals(const QtProperty* property) const;
00159
00160 public Q_SLOTS:
00161 void setValue(QtProperty* property, double val);
00162 void setValueOnly(QtProperty* property, double val);
00163 void setMinimum(QtProperty* property, double minVal);
00164 void setMaximum(QtProperty* property, double maxVal);
00165 void setRange(QtProperty* property, double minVal, double maxVal);
00166 void setSingleStep(QtProperty* property, double step);
00167 void setDecimals(QtProperty* property, int prec);
00168 Q_SIGNALS:
00169 void valueChanged(QtProperty* property, double val);
00170 void rangeChanged(QtProperty* property, double minVal, double maxVal);
00171 void singleStepChanged(QtProperty* property, double step);
00172 void decimalsChanged(QtProperty* property, int prec);
00173 protected:
00174 QString valueText(const QtProperty* property) const override;
00175 void initializeProperty(QtProperty* property) override;
00176 void uninitializeProperty(QtProperty* property) override;
00177 private:
00178 QScopedPointer<QtDoublePropertyManagerPrivate> d_ptr;
00179 Q_DECLARE_PRIVATE(QtDoublePropertyManager)
00180 Q_DISABLE_COPY_MOVE(QtDoublePropertyManager)
00181 };
00182
00183 #pragma endregion
00184
00185 #pragma region QtStringPropertyManager
00186
00187 class QtStringPropertyManagerPrivate;
00188
00189 class QT_QTPROPERTYBROWSER_EXPORT QtStringPropertyManager : public QtAbstractPropertyManager
00190 {
00191 Q_OBJECT
00192 public:
00193 QtStringPropertyManager(QObject* parent = 0);
00194 ~QtStringPropertyManager();
00195
00196 QString value(const QtProperty* property) const;
00197 QString initialValue(const QtProperty* property) const;
00198 QRegularExpression regExp(const QtProperty* property) const;
00199
00200 public Q_SLOTS:
00201 void setValue(QtProperty* property, const QString& val);
00202 void setValueOnly(QtProperty* property, const QString& val);
00203 void setRegExp(QtProperty* property, const QRegularExpression& regExp);
00204 Q_SIGNALS:
00205 void valueChanged(QtProperty* property, const QString& val);
00206 void regExpChanged(QtProperty* property, const QRegularExpression& regExp);
00207 protected:
00208 QString valueText(const QtProperty* property) const override;
00209 void initializeProperty(QtProperty* property) override;
00210 void uninitializeProperty(QtProperty* property) override;
00211 private:
00212 QScopedPointer<QtStringPropertyManagerPrivate> d_ptr;
00213 Q_DECLARE_PRIVATE(QtStringPropertyManager)
00214 Q_DISABLE_COPY_MOVE(QtStringPropertyManager)
00215 };
00216
00217 #pragma endregion
00218
00219 #pragma region QtDatePropertyManager
00220
00221 class QtDatePropertyManagerPrivate;
00222
00223 class QT_QTPROPERTYBROWSER_EXPORT QtDatePropertyManager : public QtAbstractPropertyManager
00224 {
00225 Q_OBJECT
00226 public:
00227 QtDatePropertyManager(QObject* parent = 0);
00228 ~QtDatePropertyManager();
00229
00230 QDate value(const QtProperty* property) const;
00231 QDate initialValue(const QtProperty* property) const;
00232 QDate minimum(const QtProperty* property) const;

```



```

00233 QDate maximum(const QtProperty* property) const;
00234
00235 public Q_SLOTS:
00236 void setValue(QtProperty* property, QDate val);
00237 void setValueOnly(QtProperty* property, QDate val);
00238 void setMinimum(QtProperty* property, QDate minVal);
00239 void setMaximum(QtProperty* property, QDate maxVal);
00240 void setRange(QtProperty* property, QDate minVal, QDate maxVal);
00241 Q_SIGNALS:
00242 void valueChanged(QtProperty* property, QDate val);
00243 void rangeChanged(QtProperty* property, QDate minVal, QDate maxVal);
00244 protected:
00245 QString valueText(const QtProperty* property) const override;
00246 void initializeProperty(QtProperty* property) override;
00247 void uninitializedProperty(QtProperty* property) override;
00248 private:
00249 QScopedPointer<QtDatePropertyManagerPrivate> d_ptr;
00250 Q_DECLARE_PRIVATE(QtDatePropertyManager)
00251 Q_DISABLE_COPY_MOVE(QtDatePropertyManager)
00252 };
00253
00254 #pragma endregion
00255
00256 #pragma region QtTimePropertyManager
00257
00258 class QtTimePropertyManagerPrivate;
00259
00260 class QT_QTPROPERTYBROWSER_EXPORT QtTimePropertyManager : public QtAbstractPropertyManager
00261 {
00262 Q_OBJECT
00263 public:
00264 QtTimePropertyManager(QObject* parent = 0);
00265 ~QtTimePropertyManager();
00266
00267 QTime value(const QtProperty* property) const;
00268 QTime initialValue(const QtProperty* property) const;
00269
00270 public Q_SLOTS:
00271 void setValue(QtProperty* property, QTime val);
00272 void setValueOnly(QtProperty* property, QTime val);
00273 Q_SIGNALS:
00274 void valueChanged(QtProperty* property, QTime val);
00275 protected:
00276 QString valueText(const QtProperty* property) const override;
00277 void initializeProperty(QtProperty* property) override;
00278 void uninitializedProperty(QtProperty* property) override;
00279 private:
00280 QScopedPointer<QtTimePropertyManagerPrivate> d_ptr;
00281 Q_DECLARE_PRIVATE(QtTimePropertyManager)
00282 Q_DISABLE_COPY_MOVE(QtTimePropertyManager)
00283 };
00284
00285 #pragma endregion
00286
00287 #pragma region QDateTimePropertyManager
00288
00289 class QDateTimePropertyManagerPrivate;
00290
00291 class QT_QTPROPERTYBROWSER_EXPORT QDateTimePropertyManager : public QtAbstractPropertyManager
00292 {
00293 Q_OBJECT
00294 public:
00295 QDateTimePropertyManager(QObject* parent = 0);
00296 ~QDateTimePropertyManager();
00297
00298 QDateTime value(const QtProperty* property) const;
00299 QDateTime initialValue(const QtProperty* property) const;
00300
00301 public Q_SLOTS:
00302 void setValue(QtProperty* property, const QDateTime& val);
00303 void setValueOnly(QtProperty* property, const QDateTime& val);
00304 Q_SIGNALS:
00305 void valueChanged(QtProperty* property, const QDateTime& val);
00306 protected:
00307 QString valueText(const QtProperty* property) const override;
00308 void initializeProperty(QtProperty* property) override;
00309 void uninitializedProperty(QtProperty* property) override;
00310 private:
00311 QScopedPointer<QDateTimePropertyManagerPrivate> d_ptr;
00312 Q_DECLARE_PRIVATE(QDateTimePropertyManager)
00313 Q_DISABLE_COPY_MOVE(QDateTimePropertyManager)
00314 };
00315
00316 #pragma endregion
00317
00318 #pragma region QtKeySequencePropertyManager
00319

```

```

00320 class QtKeySequencePropertyManagerPrivate;
00321
00322 class QT_QTPROPERTYBROWSER_EXPORT QtKeySequencePropertyManager : public QtAbstractPropertyManager
00323 {
00324 Q_OBJECT
00325 public:
00326 QtKeySequencePropertyManager(QObject* parent = 0);
00327 ~QtKeySequencePropertyManager();
00328
00329 QKeySequence value(const QtProperty* property) const;
00330
00331 public Q_SLOTS:
00332 void setValue(QtProperty* property, const QKeySequence& val);
00333 Q_SIGNALS:
00334 void valueChanged(QtProperty* property, const QKeySequence& val);
00335 protected:
00336 QString valueText(const QtProperty* property) const override;
00337 void initializeProperty(QtProperty* property) override;
00338 void uninitializedProperty(QtProperty* property) override;
00339 private:
00340 QScopedPointer<QtKeySequencePropertyManagerPrivate> d_ptr;
00341 Q_DECLARE_PRIVATE(QtKeySequencePropertyManager)
00342 Q_DISABLE_COPY_MOVE(QtKeySequencePropertyManager)
00343 };
00344
00345 #pragma endregion
00346
00347 #pragma region QtCharPropertyManager
00348
00349 class QtCharPropertyManagerPrivate;
00350
00351 class QT_QTPROPERTYBROWSER_EXPORT QtCharPropertyManager : public QtAbstractPropertyManager
00352 {
00353 Q_OBJECT
00354 public:
00355 QtCharPropertyManager(QObject* parent = 0);
00356 ~QtCharPropertyManager();
00357
00358 QChar value(const QtProperty* property) const;
00359
00360 public Q_SLOTS:
00361 void setValue(QtProperty* property, const QChar& val);
00362 Q_SIGNALS:
00363 void valueChanged(QtProperty* property, const QChar& val);
00364 protected:
00365 QString valueText(const QtProperty* property) const override;
00366 void initializeProperty(QtProperty* property) override;
00367 void uninitializedProperty(QtProperty* property) override;
00368 private:
00369 QScopedPointer<QtCharPropertyManagerPrivate> d_ptr;
00370 Q_DECLARE_PRIVATE(QtCharPropertyManager)
00371 Q_DISABLE_COPY_MOVE(QtCharPropertyManager)
00372 };
00373
00374 #pragma endregion
00375
00376 #pragma region QtLocalePropertyManager
00377
00378 class QtEnumPropertyManager;
00379 class QtLocalePropertyManagerPrivate;
00380
00381 class QT_QTPROPERTYBROWSER_EXPORT QtLocalePropertyManager : public QtAbstractPropertyManager
00382 {
00383 Q_OBJECT
00384 public:
00385 QtLocalePropertyManager(QObject* parent = 0);
00386 ~QtLocalePropertyManager();
00387
00388 QtEnumPropertyManager* subEnumPropertyManager() const;
00389
00390 QLocale value(const QtProperty* property) const;
00391
00392 public Q_SLOTS:
00393 void setValue(QtProperty* property, const QLocale& val);
00394 Q_SIGNALS:
00395 void valueChanged(QtProperty* property, const QLocale& val);
00396 protected:
00397 QString valueText(const QtProperty* property) const override;
00398 void initializeProperty(QtProperty* property) override;
00399 void uninitializedProperty(QtProperty* property) override;
00400 private:
00401 QScopedPointer<QtLocalePropertyManagerPrivate> d_ptr;
00402 Q_DECLARE_PRIVATE(QtLocalePropertyManager)
00403 Q_DISABLE_COPY_MOVE(QtLocalePropertyManager)
00404 Q_PRIVATE_SLOT(d_func(), void slotEnumChanged(QtProperty*, int))
00405 Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty*))
00406 };

```

```

00407
00408 #pragma endregion
00409
00410 #pragma region QtPointPropertyManager
00411
00412 class QtPointPropertyManagerPrivate;
00413
00414 class QT_QTPROPERTYBROWSER_EXPORT QtPointPropertyManager : public QtAbstractPropertyManager
00415 {
00416 Q_OBJECT
00417 public:
00418 QtPointPropertyManager(QObject* parent = 0);
00419 ~QtPointPropertyManager();
00420
00421 QtIntPropertyManager* subIntPropertyManager() const;
00422
00423 QPoint value(const QtProperty* property) const;
00424
00425 public Q_SLOTS:
00426 void setValue(QtProperty* property, const QPoint& val);
00427 Q_SIGNALS:
00428 void valueChanged(QtProperty* property, const QPoint& val);
00429 protected:
00430 QString valueText(const QtProperty* property) const override;
00431 void initializeProperty(QtProperty* property) override;
00432 void uninitializeProperty(QtProperty* property) override;
00433 private:
00434 QScopedPointer<QtPointPropertyManagerPrivate> d_ptr;
00435 Q_DECLARE_PRIVATE(QtPointPropertyManager)
00436 Q_DISABLE_COPY_MOVE(QtPointPropertyManager)
00437 Q_PRIVATE_SLOT(d_func(), void slotIntChanged(QtProperty*, int))
00438 Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty*))
00439 };
00440
00441 #pragma endregion
00442
00443 #pragma region QtPointFPropertyManager
00444
00445 class QtPointFPropertyManagerPrivate;
00446
00447 class QT_QTPROPERTYBROWSER_EXPORT QtPointFPropertyManager : public QtAbstractPropertyManager
00448 {
00449 Q_OBJECT
00450 public:
00451 QtPointFPropertyManager(QObject* parent = 0);
00452 ~QtPointFPropertyManager();
00453
00454 QtDoublePropertyManager* subDoublePropertyManager() const;
00455
00456 QPointF value(const QtProperty* property) const;
00457 int decimals(const QtProperty* property) const;
00458
00459 public Q_SLOTS:
00460 void setValue(QtProperty* property, const QPointF& val);
00461 void setDecimals(QtProperty* property, int prec);
00462 Q_SIGNALS:
00463 void valueChanged(QtProperty* property, const QPointF& val);
00464 void decimalsChanged(QtProperty* property, int prec);
00465 protected:
00466 QString valueText(const QtProperty* property) const override;
00467 void initializeProperty(QtProperty* property) override;
00468 void uninitializeProperty(QtProperty* property) override;
00469 private:
00470 QScopedPointer<QtPointFPropertyManagerPrivate> d_ptr;
00471 Q_DECLARE_PRIVATE(QtPointFPropertyManager)
00472 Q_DISABLE_COPY_MOVE(QtPointFPropertyManager)
00473 Q_PRIVATE_SLOT(d_func(), void slotDoubleChanged(QtProperty*, double))
00474 Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty*))
00475 };
00476
00477 #pragma endregion
00478
00479 #pragma region QSizePropertyManager
00480
00481 class QSizePropertyManagerPrivate;
00482
00483 class QT_QTPROPERTYBROWSER_EXPORT QSizePropertyManager : public QtAbstractPropertyManager
00484 {
00485 Q_OBJECT
00486 public:
00487 QSizePropertyManager(QObject* parent = 0);
00488 ~QSizePropertyManager();
00489
00490 QtIntPropertyManager* subIntPropertyManager() const;
00491
00492 QSize value(const QtProperty* property) const;
00493 QSize minimum(const QtProperty* property) const;

```

```

00494 QSize maximum(const QtProperty* property) const;
00495
00496 public Q_SLOTS:
00497 void setValue(QtProperty* property, const QSize& val);
00498 void setMinimum(QtProperty* property, const QSize& minVal);
00499 void setMaximum(QtProperty* property, const QSize& maxVal);
00500 void setRange(QtProperty* property, const QSize& minVal, const QSize& maxVal);
00501 Q_SIGNALS:
00502 void valueChanged(QtProperty* property, const QSize& val);
00503 void rangeChanged(QtProperty* property, const QSize& minVal, const QSize& maxVal);
00504 protected:
00505 QString valueText(const QtProperty* property) const override;
00506 void initializeProperty(QtProperty* property) override;
00507 void uninitializedProperty(QtProperty* property) override;
00508 private:
00509 QScopedPointer<QtSizePropertyManagerPrivate> d_ptr;
00510 Q_DECLARE_PRIVATE(QtSizePropertyManager)
00511 Q_DISABLE_COPY_MOVE(QtSizePropertyManager)
00512 Q_PRIVATE_SLOT(d_func(), void slotIntChanged(QtProperty*, int))
00513 Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty*))
00514 };
00515
00516 #pragma endregion
00517
00518 #pragma region QtSizeFPropertyManager
00519
00520 class QtSizeFPropertyManagerPrivate;
00521
00522 class QT_QTPROPERTYBROWSER_EXPORT QtSizeFPropertyManager : public QtAbstractPropertyManager
00523 {
00524 Q_OBJECT
00525 public:
00526 QtSizeFPropertyManager(QObject* parent = 0);
00527 ~QtSizeFPropertyManager();
00528
00529 QtDoublePropertyManager* subDoublePropertyManager() const;
00530
00531 QSizeF value(const QtProperty* property) const;
00532 QSizeF minimum(const QtProperty* property) const;
00533 QSizeF maximum(const QtProperty* property) const;
00534 int decimals(const QtProperty* property) const;
00535
00536 public Q_SLOTS:
00537 void setValue(QtProperty* property, const QSizeF& val);
00538 void setMinimum(QtProperty* property, const QSizeF& minVal);
00539 void setMaximum(QtProperty* property, const QSizeF& maxVal);
00540 void setRange(QtProperty* property, const QSizeF& minVal, const QSizeF& maxVal);
00541 void setDecimals(QtProperty* property, int prec);
00542 Q_SIGNALS:
00543 void valueChanged(QtProperty* property, const QSizeF& val);
00544 void rangeChanged(QtProperty* property, const QSizeF& minVal, const QSizeF& maxVal);
00545 void decimalsChanged(QtProperty* property, int prec);
00546 protected:
00547 QString valueText(const QtProperty* property) const override;
00548 void initializeProperty(QtProperty* property) override;
00549 void uninitializedProperty(QtProperty* property) override;
00550 private:
00551 QScopedPointer<QtSizeFPropertyManagerPrivate> d_ptr;
00552 Q_DECLARE_PRIVATE(QtSizeFPropertyManager)
00553 Q_DISABLE_COPY_MOVE(QtSizeFPropertyManager)
00554 Q_PRIVATE_SLOT(d_func(), void slotDoubleChanged(QtProperty*, double))
00555 Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty*))
00556 };
00557
00558 #pragma endregion
00559
00560 #pragma region QtRectPropertyManager
00561
00562 class QtRectPropertyManagerPrivate;
00563
00564 class QT_QTPROPERTYBROWSER_EXPORT QtRectPropertyManager : public QtAbstractPropertyManager
00565 {
00566 Q_OBJECT
00567 public:
00568 QtRectPropertyManager(QObject* parent = 0);
00569 ~QtRectPropertyManager();
00570
00571 QtIntPropertyManager* subIntPropertyManager() const;
00572
00573 QRect value(const QtProperty* property) const;
00574 QRect constraint(const QtProperty* property) const;
00575
00576 public Q_SLOTS:
00577 void setValue(QtProperty* property, const QRect& val);
00578 void setConstraint(QtProperty* property, const QRect& constraint);
00579 Q_SIGNALS:
00580 void valueChanged(QtProperty* property, const QRect& val);

```

```

00581 void constraintChanged(QtProperty* property, const QRect& constraint);
00582 protected:
00583 QString valueText(const QtProperty* property) const override;
00584 void initializeProperty(QtProperty* property) override;
00585 void uninitializedProperty(QtProperty* property) override;
00586 private:
00587 QScopedPointer<QtRectPropertyManagerPrivate> d_ptr;
00588 Q_DECLARE_PRIVATE(QtRectPropertyManager)
00589 Q_DISABLE_COPY_MOVE(QtRectPropertyManager)
00590 Q_PRIVATE_SLOT(d_func(), void slotIntChanged(QtProperty*, int))
00591 Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty*))
00592 };
00593
00594 #pragma endregion
00595
00596 #pragma region QtRectFPropertyManager
00597
00598 class QtRectFPropertyManagerPrivate;
00599
00600 class QT_QTPROPERTYBROWSER_EXPORT QtRectFPropertyManager : public QtAbstractPropertyManager
00601 {
00602 Q_OBJECT
00603 public:
00604 QtRectFPropertyManager(QObject* parent = 0);
00605 ~QtRectFPropertyManager();
00606
00607 QtDoublePropertyManager* subDoublePropertyManager() const;
00608
00609 QRectF value(const QtProperty* property) const;
00610 QRectF constraint(const QtProperty* property) const;
00611 int decimals(const QtProperty* property) const;
00612
00613 public Q_SLOTS:
00614 void setValue(QtProperty* property, const QRectF& val);
00615 void setConstraint(QtProperty* property, const QRectF& constraint);
00616 void setDecimals(QtProperty* property, int prec);
00617 Q_SIGNALS:
00618 void valueChanged(QtProperty* property, const QRectF& val);
00619 void constraintChanged(QtProperty* property, const QRectF& constraint);
00620 void decimalsChanged(QtProperty* property, int prec);
00621 protected:
00622 QString valueText(const QtProperty* property) const override;
00623 void initializeProperty(QtProperty* property) override;
00624 void uninitializedProperty(QtProperty* property) override;
00625 private:
00626 QScopedPointer<QtRectFPropertyManagerPrivate> d_ptr;
00627 Q_DECLARE_PRIVATE(QtRectFPropertyManager)
00628 Q_DISABLE_COPY_MOVE(QtRectFPropertyManager)
00629 Q_PRIVATE_SLOT(d_func(), void slotDoubleChanged(QtProperty*, double))
00630 Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty*))
00631 };
00632
00633 #pragma endregion
00634
00635 #pragma region QtEnumPropertyManager
00636
00637 class QtEnumPropertyManagerPrivate;
00638
00639 class QT_QTPROPERTYBROWSER_EXPORT QtEnumPropertyManager : public QtAbstractPropertyManager
00640 {
00641 Q_OBJECT
00642 public:
00643 QtEnumPropertyManager(QObject* parent = 0);
00644 ~QtEnumPropertyManager();
00645
00646 int value(const QtProperty* property) const;
00647 QStringList enumNames(const QtProperty* property) const;
00648 QMap<int, QIcon> enumIcons(const QtProperty* property) const;
00649
00650 public Q_SLOTS:
00651 void setValue(QtProperty* property, int val);
00652 void setEnumNames(QtProperty* property, const QStringList& names);
00653 void setEnumIcons(QtProperty* property, const QMap<int, QIcon>& icons);
00654 Q_SIGNALS:
00655 void valueChanged(QtProperty* property, int val);
00656 void enumNamesChanged(QtProperty* property, const QStringList& names);
00657 void enumIconsChanged(QtProperty* property, const QMap<int, QIcon>& icons);
00658 protected:
00659 QString valueText(const QtProperty* property) const override;
00660 QIcon valueIcon(const QtProperty* property) const override;
00661 void initializeProperty(QtProperty* property) override;
00662 void uninitializedProperty(QtProperty* property) override;
00663 private:
00664 QScopedPointer<QtEnumPropertyManagerPrivate> d_ptr;
00665 Q_DECLARE_PRIVATE(QtEnumPropertyManager)
00666 Q_DISABLE_COPY_MOVE(QtEnumPropertyManager)
00667 };

```

```

00668
00669 #pragma endregion
00670
00671 #pragma region QtFlagPropertyManager
00672
00673 class QtFlagPropertyManagerPrivate;
00674
00675 class QT_QTPROPERTYBROWSER_EXPORT QtFlagPropertyManager : public QtAbstractPropertyManager
00676 {
00677 Q_OBJECT
00678 public:
00679 QtFlagPropertyManager(QObject* parent = 0);
00680 ~QtFlagPropertyManager();
00681
00682 QtBoolPropertyManager* subBoolPropertyManager() const;
00683
00684 int value(const QtProperty* property) const;
00685 QStringList flagNames(const QtProperty* property) const;
00686
00687 public Q_SLOTS:
00688 void setValue(QtProperty* property, int val);
00689 void setFlagNames(QtProperty* property, const QStringList& names);
00690 Q_SIGNALS:
00691 void valueChanged(QtProperty* property, int val);
00692 void flagNamesChanged(QtProperty* property, const QStringList& names);
00693 protected:
00694 QString valueText(const QtProperty* property) const override;
00695 void initializeProperty(QtProperty* property) override;
00696 void uninitializeProperty(QtProperty* property) override;
00697 private:
00698 QScopedPointer<QtFlagPropertyManagerPrivate> d_ptr;
00699 Q_DECLARE_PRIVATE(QtFlagPropertyManager)
00700 Q_DISABLE_COPY_MOVE(QtFlagPropertyManager)
00701 Q_PRIVATE_SLOT(d_func(), void slotBoolChanged(QtProperty*, bool))
00702 Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty*))
00703 };
00704
00705 #pragma endregion
00706
00707 #pragma region QtSizePolicyPropertyManager
00708
00709 class QtSizePolicyPropertyManagerPrivate;
00710
00711 class QT_QTPROPERTYBROWSER_EXPORT QtSizePolicyPropertyManager : public QtAbstractPropertyManager
00712 {
00713 Q_OBJECT
00714 public:
00715 QtSizePolicyPropertyManager(QObject* parent = 0);
00716 ~QtSizePolicyPropertyManager();
00717
00718 QtIntPropertyManager* subIntPropertyManager() const;
00719 QtEnumPropertyManager* subEnumPropertyManager() const;
00720
00721 QSizePolicy value(const QtProperty* property) const;
00722
00723 public Q_SLOTS:
00724 void setValue(QtProperty* property, const QSizePolicy& val);
00725 Q_SIGNALS:
00726 void valueChanged(QtProperty* property, const QSizePolicy& val);
00727 protected:
00728 QString valueText(const QtProperty* property) const override;
00729 void initializeProperty(QtProperty* property) override;
00730 void uninitializeProperty(QtProperty* property) override;
00731 private:
00732 QScopedPointer<QtSizePolicyPropertyManagerPrivate> d_ptr;
00733 Q_DECLARE_PRIVATE(QtSizePolicyPropertyManager)
00734 Q_DISABLE_COPY_MOVE(QtSizePolicyPropertyManager)
00735 Q_PRIVATE_SLOT(d_func(), void slotIntChanged(QtProperty*, int))
00736 Q_PRIVATE_SLOT(d_func(), void slotEnumChanged(QtProperty*, int))
00737 Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty*))
00738 };
00739
00740 #pragma endregion
00741
00742 #pragma region QtFontPropertyManager
00743
00744 class QtFontPropertyManagerPrivate;
00745
00746 class QT_QTPROPERTYBROWSER_EXPORT QtFontPropertyManager : public QtAbstractPropertyManager
00747 {
00748 Q_OBJECT
00749 public:
00750 QtFontPropertyManager(QObject* parent = 0);
00751 ~QtFontPropertyManager();
00752
00753 QtIntPropertyManager* subIntPropertyManager() const;
00754 QtEnumPropertyManager* subEnumPropertyManager() const;

```

```

00755 QtBoolPropertyManager* subBoolPropertyManager() const;
00756
00757 QFont value(const QtProperty* property) const;
00758
00759 public Q_SLOTS:
00760 void setValue(QtProperty* property, const QFont& val);
00761 Q_SIGNALS:
00762 void valueChanged(QtProperty* property, const QFont& val);
00763 protected:
00764 QString valueText(const QtProperty* property) const override;
00765 QIcon valueIcon(const QtProperty* property) const override;
00766 void initializeProperty(QtProperty* property) override;
00767 void uninitializedProperty(QtProperty* property) override;
00768 private:
00769 QScopedPointer<QtFontPropertyManagerPrivate> d_ptr;
00770 Q_DECLARE_PRIVATE(QtFontPropertyManager)
00771 Q_DISABLE_COPY_MOVE(QtFontPropertyManager)
00772 Q_PRIVATE_SLOT(d_func(), void slotIntChanged(QtProperty*, int))
00773 Q_PRIVATE_SLOT(d_func(), void slotEnumChanged(QtProperty*, int))
00774 Q_PRIVATE_SLOT(d_func(), void slotBoolChanged(QtProperty*, bool))
00775 Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty*))
00776 Q_PRIVATE_SLOT(d_func(), void slotFontDatabaseChanged())
00777 Q_PRIVATE_SLOT(d_func(), void slotFontDatabaseDelayedChange())
00778 };
00779
00780 #pragma endregion
00781
00782 #pragma region QColorPropertyManager
00783
00784 class QColorPropertyManagerPrivate;
00785
00786 class QT_QTPROPERTYBROWSER_EXPORT QColorPropertyManager : public QtAbstractPropertyManager
00787 {
00788 Q_OBJECT
00789 public:
00790 QColorPropertyManager(QObject* parent = 0);
00791 ~QColorPropertyManager();
00792
00793 QtIntPropertyManager* subIntPropertyManager() const;
00794
00795 QColor value(const QtProperty* property) const;
00796
00797 public Q_SLOTS:
00798 void setValue(QtProperty* property, const QColor& val);
00799 Q_SIGNALS:
00800 void valueChanged(QtProperty* property, const QColor& val);
00801 protected:
00802 QString valueText(const QtProperty* property) const override;
00803 QIcon valueIcon(const QtProperty* property) const override;
00804 void initializeProperty(QtProperty* property) override;
00805 void uninitializedProperty(QtProperty* property) override;
00806 private:
00807 QScopedPointer<QtColorPropertyManagerPrivate> d_ptr;
00808 Q_DECLARE_PRIVATE(QtColorPropertyManager)
00809 Q_DISABLE_COPY_MOVE(QtColorPropertyManager)
00810 Q_PRIVATE_SLOT(d_func(), void slotIntChanged(QtProperty*, int))
00811 Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty*))
00812 };
00813
00814 #pragma endregion
00815
00816 #pragma region QtCursorPropertyManager
00817
00818 class QtCursorPropertyManagerPrivate;
00819
00820 class QT_QTPROPERTYBROWSER_EXPORT QtCursorPropertyManager : public QtAbstractPropertyManager
00821 {
00822 Q_OBJECT
00823 public:
00824 QtCursorPropertyManager(QObject* parent = 0);
00825 ~QtCursorPropertyManager();
00826
00827 #ifndef QT_NO_CURSOR
00828 QCursor value(const QtProperty* property) const;
00829 #endif
00830
00831 public Q_SLOTS:
00832 void setValue(QtProperty* property, const QCursor& val);
00833 Q_SIGNALS:
00834 void valueChanged(QtProperty* property, const QCursor& val);
00835 protected:
00836 QString valueText(const QtProperty* property) const override;
00837 QIcon valueIcon(const QtProperty* property) const override;
00838 void initializeProperty(QtProperty* property) override;
00839 void uninitializedProperty(QtProperty* property) override;
00840 private:
00841 QScopedPointer<QtCursorPropertyManagerPrivate> d_ptr;

```



```

00842 Q_DECLARE_PRIVATE(QtCursorPropertyManager)
00843 Q_DISABLE_COPY_MOVE(QtCursorPropertyManager)
00844 };
00845
00846 #pragma endregion
00847
00848 QT_END_NAMESPACE
00849
00850 #endif

```

## 16.83 qttreepropertybrowser.h

```

00001 /*****
00002 **
00003 ** Copyright (C) 2016 The Qt Company Ltd.
00004 ** Contact: https://www.qt.io/licensing/
00005 **
00006 ** This file is part of the tools applications of the Qt Toolkit.
00007 **
00008 ** $QT_BEGIN_LICENSE:LGPL$
00009 ** Commercial License Usage
00010 ** Licensees holding valid commercial Qt licenses may use this file in
00011 ** accordance with the commercial license agreement provided with the
00012 ** Software or, alternatively, in accordance with the terms contained in
00013 ** a written agreement between you and The Qt Company. For licensing terms
00014 ** and conditions see https://www.qt.io/terms-conditions. For further
00015 ** information use the contact form at https://www.qt.io/contact-us.
00016 **
00017 ** GNU Lesser General Public License Usage
00018 ** Alternatively, this file may be used under the terms of the GNU Lesser
00019 ** General Public License version 3 as published by the Free Software
00020 ** Foundation and appearing in the file LICENSE.LGPL3 included in the
00021 ** packaging of this file. Please review the following information to
00022 ** ensure the GNU Lesser General Public License version 3 requirements
00023 ** will be met: https://www.gnu.org/licenses/lgpl-3.0.html.
00024 **
00025 ** GNU General Public License Usage
00026 ** Alternatively, this file may be used under the terms of the GNU
00027 ** General Public License version 2.0 or (at your option) the GNU General
00028 ** Public License version 3 or any later version approved by the KDE Free
00029 ** Qt Foundation. The licenses are as published by the Free Software
00030 ** Foundation and appearing in the file LICENSE.GPL2 and LICENSE.GPL3
00031 ** included in the packaging of this file. Please review the following
00032 ** information to ensure the GNU General Public License requirements will
00033 ** be met: https://www.gnu.org/licenses/gpl-2.0.html and
00034 ** https://www.gnu.org/licenses/gpl-3.0.html.
00035 **
00036 ** $QT_END_LICENSE$
00037 **
00038 *****/
00039
00040 #ifndef QTTREEPROPERTYBROWSER_H
00041 #define QTTREEPROPERTYBROWSER_H
00042
00043 #include "qtpropertybrowser.h"
00044
00045 QT_BEGIN_NAMESPACE
00046
00047 class QTreeWidgetItem;
00048 class QtTreePropertyBrowserPrivate;
00049
00050 class QT_QTPROPERTYBROWSER_EXPORT QtTreePropertyBrowser : public QtAbstractPropertyBrowser
00051 {
00052 Q_OBJECT
00053 Q_PROPERTY(int indentation READ indentation WRITE setIndentation)
00054 Q_PROPERTY(bool rootIsDecorated READ rootIsDecorated WRITE setRootIsDecorated)
00055 Q_PROPERTY(bool alternatingRowColors READ alternatingRowColors WRITE setAlternatingRowColors)
00056 Q_PROPERTY(bool headerVisible READ isHeaderVisible WRITE setHeaderVisible)
00057 Q_PROPERTY(ResizeMode resizeMode READ resizeMode WRITE setResizeMode)
00058 Q_PROPERTY(int splitterPosition READ splitterPosition WRITE setSplitterPosition)
00059 Q_PROPERTY(bool propertiesWithoutValueMarked READ propertiesWithoutValueMarked WRITE
setPropertiesWithoutValueMarked)
00060 public:
00061
00062 enum ResizeMode
00063 {
00064 Interactive,
00065 Stretch,
00066 Fixed,
00067 ResizeToContents
00068 };
00069 Q_ENUM(ResizeMode)
00070
00071 QtTreePropertyBrowser(QWidget *parent = 0);
00072 ~QtTreePropertyBrowser();

```



```

00073
00074 int indentation() const;
00075 void setIndentation(int i);
00076
00077 bool rootIsDecorated() const;
00078 void setRootIsDecorated(bool show);
00079
00080 bool alternatingRowColors() const;
00081 void setAlternatingRowColors(bool enable);
00082
00083 bool isHeaderVisible() const;
00084 void setHeaderVisible(bool visible);
00085
00086 ResizeMode resizeMode() const;
00087 void setResizeMode(ResizeMode mode);
00088
00089 int splitterPosition() const;
00090 void setSplitterPosition(int position);
00091
00092 void setExpanded(QTreeWidgetItem *item, bool expanded);
00093 bool isExpanded(QTreeWidgetItem *item) const;
00094
00095 bool isItemVisible(QTreeWidgetItem *item) const;
00096 void setItemVisible(QTreeWidgetItem *item, bool visible);
00097
00098 void setBackgroundColor(QTreeWidgetItem *item, const QColor &color);
00099 QColor backgroundColor(QTreeWidgetItem *item) const;
00100 QColor calculatedBackgroundColor(QTreeWidgetItem *item) const;
00101
00102 void setPropertiesWithoutValueMarked(bool mark);
00103 bool propertiesWithoutValueMarked() const;
00104
00105 void editItem(QTreeWidgetItem *item);
00106
00107 Q_SIGNALS:
00108
00109 void collapsed(QTreeWidgetItem *item);
00110 void expanded(QTreeWidgetItem *item);
00111
00112 protected:
00113 void itemInserted(QTreeWidgetItem *item, QTreeWidgetItem *afterItem) override;
00114 void itemRemoved(QTreeWidgetItem *item) override;
00115 void itemChanged(QTreeWidgetItem *item) override;
00116
00117 private:
00118
00119 QScopedPointer<QtTreePropertyBrowserPrivate> d_ptr;
00120 Q_DECLARE_PRIVATE(QtTreePropertyBrowser)
00121 Q_DISABLE_COPY_MOVE(QtTreePropertyBrowser)
00122
00123 Q_PRIVATE_SLOT(d_func(), void slotCollapsed(const QModelIndex &))
00124 Q_PRIVATE_SLOT(d_func(), void slotExpanded(const QModelIndex &))
00125 Q_PRIVATE_SLOT(d_func(), void slotCurrentBrowserItemChanged(QTreeWidgetItem *))
00126 Q_PRIVATE_SLOT(d_func(), void slotCurrentTreeItemChanged(QTreeWidgetItem *, QTreeWidgetItem *))
00127
00128 };
00129
00130 QT_END_NAMESPACE
00131
00132 #endif

```

## 16.84 qtvARIANTPROPERTY.H

```

00001 /*****
00002 **
00003 ** Copyright (C) 2016 The Qt Company Ltd.
00004 ** Contact: https://www.qt.io/licensing/
00005 **
00006 ** This file is part of the tools applications of the Qt Toolkit.
00007 **
00008 ** $QT_BEGIN_LICENSE:LGPL$
00009 ** Commercial License Usage
00010 ** Licensees holding valid commercial Qt licenses may use this file in
00011 ** accordance with the commercial license agreement provided with the
00012 ** Software or, alternatively, in accordance with the terms contained in
00013 ** a written agreement between you and The Qt Company. For licensing terms
00014 ** and conditions see https://www.qt.io/terms-conditions. For further
00015 ** information use the contact form at https://www.qt.io/contact-us.
00016 **
00017 ** GNU Lesser General Public License Usage
00018 ** Alternatively, this file may be used under the terms of the GNU Lesser
00019 ** General Public License version 3 as published by the Free Software
00020 ** Foundation and appearing in the file LICENSE.LGPL3 included in the
00021 ** packaging of this file. Please review the following information to
00022 ** ensure the GNU Lesser General Public License version 3 requirements

```

```

00023 ** will be met: https://www.gnu.org/licenses/lgpl-3.0.html.
00024 **
00025 ** GNU General Public License Usage
00026 ** Alternatively, this file may be used under the terms of the GNU
00027 ** General Public License version 2.0 or (at your option) the GNU General
00028 ** Public license version 3 or any later version approved by the KDE Free
00029 ** Qt Foundation. The licenses are as published by the Free Software
00030 ** Foundation and appearing in the file LICENSE.GPL2 and LICENSE.GPL3
00031 ** included in the packaging of this file. Please review the following
00032 ** information to ensure the GNU General Public License requirements will
00033 ** be met: https://www.gnu.org/licenses/gpl-2.0.html and
00034 ** https://www.gnu.org/licenses/gpl-3.0.html.
00035 **
00036 ** $QT_END_LICENSE$
00037 **
00038 *****/
00039
00040 #ifndef QTVARIANTPROPERTY_H
00041 #define QTVARIANTPROPERTY_H
00042
00043 #include "qtpropertybrowser.h"
00044 #include <QtCore/QString>
00045 #include <QtGui/QIcon>
00046
00047 QT_BEGIN_NAMESPACE
00048
00049 class QRegularExpression;
00050
00051 typedef QMap<int, QIcon> QtIconMap;
00052
00053 class QtVariantPropertyManager;
00054
00055 class QT_QTPROPERTYBROWSER_EXPORT QtVariantProperty : public QtProperty
00056 {
00057 public:
00058 ~QtVariantProperty();
00059 QVariant value() const;
00060 QVariant attributeValue(const QString &attribute) const;
00061 int valueType() const;
00062 int propertyType() const;
00063
00064 void setValue(const QVariant &value);
00065 void setAttribute(const QString &attribute, const QVariant &value);
00066 protected:
00067 QtVariantProperty(QtVariantPropertyManager *manager);
00068 private:
00069 friend class QtVariantPropertyManager;
00070 QScopedPointer<class QtVariantPropertyPrivate> d_ptr;
00071 };
00072
00073 class QT_QTPROPERTYBROWSER_EXPORT QtVariantPropertyManager : public QtAbstractPropertyManager
00074 {
00075 Q_OBJECT
00076 public:
00077 QtVariantPropertyManager(QObject *parent = 0);
00078 ~QtVariantPropertyManager();
00079
00080 virtual QtVariantProperty *addProperty(int propertyType, const QString &name = QString());
00081
00082 int propertyType(const QtProperty *property) const;
00083 int valueType(const QtProperty *property) const;
00084 QtVariantProperty *variantProperty(const QtProperty *property) const;
00085
00086 virtual bool isPropertyTypeSupported(int propertyType) const;
00087 virtual int valueType(int propertyType) const;
00088 virtual QStringList attributes(int propertyType) const;
00089 virtual int attributeType(int propertyType, const QString &attribute) const;
00090
00091 virtual QVariant value(const QtProperty *property) const;
00092 virtual QVariant attributeValue(const QtProperty *property, const QString &attribute) const;
00093
00094 static int enumTypeId();
00095 static int flagTypeId();
00096 static int groupTypeId();
00097 static int iconMapTypeId();
00098 public Q_SLOTS:
00099 virtual void setValue(QtProperty *property, const QVariant &val);
00100 virtual void setAttribute(QtProperty *property,
00101 const QString &attribute, const QVariant &value);
00102 Q_SIGNALS:
00103 void valueChanged(QtProperty *property, const QVariant &val);
00104 void attributeChanged(QtProperty *property,
00105 const QString &attribute, const QVariant &val);
00106 protected:
00107 bool hasValue(const QtProperty *property) const override;
00108 QString valueText(const QtProperty *property) const override;
00109 QIcon valueIcon(const QtProperty *property) const override;

```

```

00110 void initializeProperty(QtProperty *property) override;
00111 void uninitializeProperty(QtProperty *property) override;
00112 QtProperty *createProperty() override;
00113 private:
00114 QScopedPointer<class QtVariantPropertyManagerPrivate> d_ptr;
00115 Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, int))
00116 Q_PRIVATE_SLOT(d_func(), void slotRangeChanged(QtProperty *, int, int))
00117 Q_PRIVATE_SLOT(d_func(), void slotSingleStepChanged(QtProperty *, int))
00118 Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, double))
00119 Q_PRIVATE_SLOT(d_func(), void slotRangeChanged(QtProperty *, double, double))
00120 Q_PRIVATE_SLOT(d_func(), void slotSingleStepChanged(QtProperty *, double))
00121 Q_PRIVATE_SLOT(d_func(), void slotDecimalsChanged(QtProperty *, int))
00122 Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, bool))
00123 Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QString &))
00124 Q_PRIVATE_SLOT(d_func(), void slotRegExpChanged(QtProperty *, const QRegularExpression &))
00125 Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, QDate))
00126 Q_PRIVATE_SLOT(d_func(), void slotRangeChanged(QtProperty *, QDate, QDate))
00127 Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, QTime))
00128 Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QDateTime &))
00129 Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QKeySequence &))
00130 Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QChar &))
00131 Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QLocale &))
00132 Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QPoint &))
00133 Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QPointF &))
00134 Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QSize &))
00135 Q_PRIVATE_SLOT(d_func(), void slotRangeChanged(QtProperty *, const QSize &, const QSize &))
00136 Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QSizeF &))
00137 Q_PRIVATE_SLOT(d_func(), void slotRangeChanged(QtProperty *, const QSizeF &, const QSizeF &))
00138 Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QRect &))
00139 Q_PRIVATE_SLOT(d_func(), void slotConstraintChanged(QtProperty *, const QRect &))
00140 Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QRectF &))
00141 Q_PRIVATE_SLOT(d_func(), void slotConstraintChanged(QtProperty *, const QRectF &))
00142 Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QColor &))
00143 Q_PRIVATE_SLOT(d_func(), void slotEnumNamesChanged(QtProperty *, const QStringList &))
00144 Q_PRIVATE_SLOT(d_func(), void slotEnumIconsChanged(QtProperty *, const QMap<int, QIcon> &))
00145 Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QSizePolicy &))
00146 Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QFont &))
00147 Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QCursor &))
00148 Q_PRIVATE_SLOT(d_func(), void slotFlagNamesChanged(QtProperty *, const QStringList &))
00149
00150 Q_PRIVATE_SLOT(d_func(), void slotPropertyInserted(QtProperty *, QtProperty *, QtProperty *))
00151 Q_PRIVATE_SLOT(d_func(), void slotPropertyRemoved(QtProperty *, QtProperty *))
00152 Q_DECLARE_PRIVATE(QtVariantPropertyManager)
00153 Q_DISABLE_COPY_MOVE(QtVariantPropertyManager)
00154 };
00155
00156 class QT_QTPROPERTYBROWSER_EXPORT QtVariantEditorFactory : public
QtAbstractEditorFactory<QtVariantPropertyManager>
00157 {
00158 Q_OBJECT
00159 public:
00160 QtVariantEditorFactory(QObject *parent = 0);
00161 ~QtVariantEditorFactory();
00162 protected:
00163 void connectPropertyManager(QtVariantPropertyManager *manager) override;
00164 QWidget *createEditor(QtVariantPropertyManager *manager, QtProperty *property,
00165 QWidget *parent) override;
00166 void disconnectPropertyManager(QtVariantPropertyManager *manager) override;
00167 private:
00168 QScopedPointer<class QtVariantEditorFactoryPrivate> d_ptr;
00169 Q_DECLARE_PRIVATE(QtVariantEditorFactory)
00170 Q_DISABLE_COPY_MOVE(QtVariantEditorFactory)
00171 };
00172
00173 QT_END_NAMESPACE
00174
00175 Q_DECLARE_METATYPE(QIcon)
00176 Q_DECLARE_METATYPE(QtIconMap)
00177 #endif

```

## 16.85 qcustomplot.h

```

00001 /*****
00002 **
00003 ** QCustomPlot, an easy to use, modern plotting widget for Qt
00004 ** Copyright (C) 2011-2021 Emanuel Eichhammer
00005 **
00006 ** This program is free software: you can redistribute it and/or modify
00007 ** it under the terms of the GNU General Public License as published by
00008 ** the Free Software Foundation, either version 3 of the License, or
00009 ** (at your option) any later version.
00010 **
00011 ** This program is distributed in the hope that it will be useful,
00012 ** but WITHOUT ANY WARRANTY; without even the implied warranty of
00013 ** MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the

```

```
00014 ** GNU General Public License for more details. **
00015 ** **
00016 ** You should have received a copy of the GNU General Public License
00017 ** along with this program. If not, see http://www.gnu.org/licenses/. **
00018 ** **
00019 *****
00020 ** Author: Emanuel Eichhammer **
00021 ** Website/Contact: http://www.qcustomplot.com/ **
00022 ** Date: 29.03.21 **
00023 ** Version: 2.1.0 **
00024 *****/
00025
00026 // https://www.qcustomplot.com/documentation/index.html
00027
00028 #ifndef QCUSTOMPLOT_H
00029 #define QCUSTOMPLOT_H
00030
00031 #include <QtCore/qglobal.h>
00032
00033 // some Qt version/configuration dependent macros to include or exclude certain code paths:
00034 #ifdef QCUSTOMPLOT_USE_OPENGL
00035 #if QT_VERSION < QT_VERSION_CHECK(5, 0, 0)
00036 #define QCP_OPENGL_PBUFFER
00037 #else
00038 #define QCP_OPENGL_FBO
00039 #endif
00040 #if QT_VERSION >= QT_VERSION_CHECK(5, 3, 0)
00041 #define QCP_OPENGL_OFFSCREENSURFACE
00042 #endif
00043 #endif
00044
00045 #if QT_VERSION >= QT_VERSION_CHECK(5, 4, 0)
00046 #define QCP_DEVICEPIXELRATIO_SUPPORTED
00047 #if QT_VERSION >= QT_VERSION_CHECK(5, 6, 0)
00048 #define QCP_DEVICEPIXELRATIO_FLOAT
00049 #endif
00050 #endif
00051
00052 #include <QtCore/QObject>
00053 #include <QtCore/QPointer>
00054 #include <QtCore/QSharedPointer>
00055 #include <QtCore/Qtimer>
00056 #include <QtGui/QPainter>
00057 #include <QtGui/QPainterPath>
00058 #include <QtGui/QPaintEvent>
00059 #include <QtGui/QMouseEvent>
00060 #include <QtGui/QWheelEvent>
00061 #include <QtGui/QPixmap>
00062 #include <QtCore/QVector>
00063 #include <QtCore/QString>
00064 #include <QtCore/QDateTime>
00065 #include <QtCore/QMultiMap>
00066 #include <QtCore/QFlags>
00067 #include <QtCore/QDebug>
00068 #include <QtCore/QStack>
00069 #include <QtCore/QCache>
00070 #include <QtCore/QMargins>
00071 #include <qmath.h>
00072 #include <limits>
00073 #include <algorithm>
00074 #ifdef QCP_OPENGL_FBO
00075 #include <QtGui/QOpenGLContext>
00076 #if QT_VERSION < QT_VERSION_CHECK(6, 0, 0)
00077 #include <QtGui/QOpenGLFramebufferObject>
00078 #else
00079 #include <QOpenGLFramebufferObject>
00080 #include <QOpenGLPaintDevice>
00081 #endif
00082 #endif
00083 #ifdef QCP_OPENGL_OFFSCREENSURFACE
00084 #include <QtGui/QOffscreenSurface>
00085 #include <QtGui/QWindow>
00086 #endif
00087 #endif
00088 #ifdef QCP_OPENGL_PBUFFER
00089 #include <QtOpenGL/QGLPixelBuffer>
00090 #endif
00091 #if QT_VERSION < QT_VERSION_CHECK(5, 0, 0)
00092 #include <qnumeric.h>
00093 #include <QtGui/QWidget>
00094 #include <QtGui/QPrinter>
00095 #include <QtGui/QPrintEngine>
00096 #else
00097 #include <QtNumeric>
00098 #include <QtWidgets/QWidget>
00099 #include <QtPrintSupport/QtPrintSupport>
00100 #endif
```

```

00101 #if QT_VERSION >= QT_VERSION_CHECK(4, 8, 0)
00102 #include <QtCore/QElapsedTimer>
00103 #endif
00104 #if QT_VERSION >= QT_VERSION_CHECK(5, 2, 0)
00105 #include <QtCore/QTimeZone>
00106 #endif
00107
00108 class QCPPainter;
00109 class QCustomPlot;
00110 class QCPLayerable;
00111 class QCPLayoutElement;
00112 class QCPLayout;
00113 class QCPAxis;
00114 class QCPAxisRect;
00115 class QCPAxisPainterPrivate;
00116 class QCPAbstractPlottable;
00117 class QCPGraph;
00118 class QCPAbstractItem;
00119 class QCPPlottableInterfaceID;
00120 class QCPLegend;
00121 class QCPItemPosition;
00122 class QCPLayer;
00123 class QCPAbstractLegendItem;
00124 class QCPSelectionRect;
00125 class QCPColorMap;
00126 class QCPColorScale;
00127 class QCPBars;
00128 class QCPPolarAxisRadial;
00129 class QCPPolarAxisAngular;
00130 class QCPPolarGrid;
00131 class QCPPolarGraph;
00132
00133 /* including file 'src/global.h' */
00134 /* modified 2021-03-29T02:30:44, size 16981 */
00135
00136 #define QCUSTOMPLOT_VERSION_STR "2.1.0"
00137 #define QCUSTOMPLOT_VERSION 0x020100
00138
00139 // decl definitions for shared library compilation/usage:
00140 #if defined(QT_STATIC_BUILD)
00141 #define QCP_LIB_DECL
00142 #elif defined(QCUSTOMPLOT_COMPILE_LIBRARY)
00143 #define QCP_LIB_DECL Q_DECL_EXPORT
00144 #elif defined(QCUSTOMPLOT_USE_LIBRARY)
00145 #define QCP_LIB_DECL Q_DECL_IMPORT
00146 #else
00147 #define QCP_LIB_DECL
00148 #endif
00149
00150 // define empty macro for Q_DECL_OVERRIDE if it doesn't exist (Qt < 5)
00151 #ifndef Q_DECL_OVERRIDE
00152 #define Q_DECL_OVERRIDE
00153 #endif
00154
00155 #ifndef Q_MOC_RUN
00156 namespace QCP {
00157 #else
00158 #endif
00159
00160 class QCP { // when in moc-run, make it look like a class, so we get Q_GADGET, Q_ENUMS/Q_FLAGS
00161 features in namespace
00162
00163 Q_GADGET
00164 Q_ENUMS(ExportPen)
00165 Q_ENUMS(ResolutionUnit)
00166 Q_ENUMS(SignDomain)
00167 Q_ENUMS(MarginSide)
00168 Q_FLAGS(MarginSides)
00169 Q_ENUMS(AntialiasedElement)
00170 Q_FLAGS(AntialiasedElements)
00171 Q_ENUMS(PlottingHint)
00172 Q_FLAGS(PlottingHints)
00173 Q_ENUMS(Interaction)
00174 Q_FLAGS(Interactions)
00175 Q_ENUMS(SelectionRectMode)
00176 Q_ENUMS(SelectionType)
00177
00178 public:
00179 #endif
00180
00181 enum ResolutionUnit {
00182 ruDotsPerMeter
00183 , ruDotsPerCentimeter
00184 , ruDotsPerInch
00185 };
00186
00187 enum ExportPen {
00188 epNoCosmetic
00189 , epAllowCosmetic
00190 };

```

```

00204 };
00205
00213 enum SignDomain {
00214 sdNegative
00215 , sdBoth
00216 , sdPositive
00217 };
00218
00224 enum MarginSide {
00225 msLeft = 0x01
00226 , msRight = 0x02
00227 , msTop = 0x04
00228 , msBottom = 0x08
00229 , msAll = 0xFF
00230 , msNone = 0x00
00231 };
00232
00233 Q_DECLARE_FLAGS(MarginSides, MarginSide)
00234
00235
00244 enum AntialiasedElement {
00245 aeAxes = 0x0001
00246 , aeGrid = 0x0002
00247 , aeSubGrid = 0x0004
00248 , aeLegend = 0x0008
00249 , aeLegendItems = 0x0010
00250 , aePlottables = 0x0020
00251 , aeItems = 0x0040
00252 , aeScatters = 0x0080
00253 , aeFills = 0x0100
00254 , aeZeroLine = 0x0200
00255 , aeOther = 0x8000
00256 , aeAll = 0xFFFF
00257 , aeNone = 0x0000
00258 };
00259
00260 Q_DECLARE_FLAGS(AntialiasedElements, AntialiasedElement)
00261
00262
00267 enum PlottingHint {
00268 phNone = 0x000
00269 , phFastPolylines = 0x001
00271 , phImmediateRefresh = 0x002
00273 , phCacheLabels = 0x004
00274 };
00275
00276 Q_DECLARE_FLAGS(PlottingHints, PlottingHint)
00277
00278
00285 enum Interaction {
00286 iNone = 0x000
00287 , iRangeDrag = 0x001
00288 , iRangeZoom = 0x002
00289 , iMultiSelect = 0x004
00290 , iSelectPlottables = 0x008
00291 , iSelectAxes = 0x010
00292 , iSelectLegend = 0x020
00293 , iSelectItems = 0x040
00294 , iSelectOther = 0x080
00295 , iSelectPlottablesBeyondAxisRect = 0x100
00296 };
00297
00298 Q_DECLARE_FLAGS(Interactions, Interaction)
00299
00300
00305 enum SelectionRectMode {
00306 srmNone
00307 , srmZoom
00308 , srmSelect
00309 , srmCustom
00310 };
00311
00330 enum SelectionType {
00331 stNone
00332 , stWhole
00333 , stSingleData
00334 , stDataRange
00335 , stMultipleDataRanges
00336 };
00337
00344 inline bool isValidData(double value) {
00345 return !qIsNaN(value) || qIsInf(value);
00346 }
00347
00353 inline bool isValidData(double value1, double value2) {
00354 return isValidData(value1) || isValidData(value2);
00355 }

```

```

00356
00363 inline void setMarginValue(QMargins &margins, QCP::MarginSide side, int value) {
00364 switch (side) {
00365 case QCP::msLeft: margins.setLeft(value);
00366 break;
00367 case QCP::msRight: margins.setRight(value);
00368 break;
00369 case QCP::msTop: margins.setTop(value);
00370 break;
00371 case QCP::msBottom: margins.setBottom(value);
00372 break;
00373 case QCP::msAll: margins = QMargins(value, value, value, value);
00374 break;
00375 default: break;
00376 }
00377 }
00378
00386 inline int getMarginValue(const QMargins &margins, QCP::MarginSide side) {
00387 switch (side) {
00388 case QCP::msLeft: return margins.left();
00389 case QCP::msRight: return margins.right();
00390 case QCP::msTop: return margins.top();
00391 case QCP::msBottom: return margins.bottom();
00392 default: break;
00393 }
00394 return 0;
00395 }
00396
00397 extern const QMetaObject staticMetaObject; // in moc-run we create a static meta object for QCP
00398 "fake" object. This line is the link to it via QCP::staticMetaObject in normal operation as namespace
00399 } // end of namespace QCP
00400
00401
00402 Q_DECLARE_OPERATORS_FOR_FLAGS(QCP::AntialiasedElements)
00403 Q_DECLARE_OPERATORS_FOR_FLAGS(QCP::PlottingHints)
00404 Q_DECLARE_OPERATORS_FOR_FLAGS(QCP::MarginSides)
00405 Q_DECLARE_OPERATORS_FOR_FLAGS(QCP::Interactions)
00406 Q_DECLARE_METATYPE(QCP::ExportPen)
00407 Q_DECLARE_METATYPE(QCP::ResolutionUnit)
00408 Q_DECLARE_METATYPE(QCP::SignDomain)
00409 Q_DECLARE_METATYPE(QCP::MarginSide)
00410 Q_DECLARE_METATYPE(QCP::AntialiasedElement)
00411 Q_DECLARE_METATYPE(QCP::PlottingHint)
00412 Q_DECLARE_METATYPE(QCP::Interaction)
00413 Q_DECLARE_METATYPE(QCP::SelectionRectMode)
00414 Q_DECLARE_METATYPE(QCP::SelectionType)
00415
00416 /* end of 'src/global.h' */
00417
00418
00419 /* including file 'src/vector2d.h' */
00420 /* modified 2021-03-29T02:30:44, size 4988 */
00421
00422 class QCP_LIB_DECL QCPVector2D {
00423 public:
00424 QCPVector2D();
00425 QCPVector2D(double x, double y);
00426 QCPVector2D(const QPoint &point);
00427 QCPVector2D(const QPointF &point);
00428
00429 // getters:
00430
00431 double x() const {
00432 return mX;
00433 }
00434
00435 double y() const {
00436 return mY;
00437 }
00438
00439 double &rx() {
00440 return mX;
00441 }
00442
00443 double &ry() {
00444 return mY;
00445 }
00446
00447 // setters:
00448
00449 void setX(double x) {
00450 mX = x;
00451 }
00452
00453 void setY(double y) {
00454 mY = y;

```

```

00455 }
00456
00457 // non-virtual methods:
00458
00459 double length() const {
00460 return qSqrt(mX * mX + mY * mY);
00461 }
00462
00463 double lengthSquared() const {
00464 return mX * mX + mY*mY;
00465 }
00466
00467 double angle() const {
00468 return qAtan2(mY, mX);
00469 }
00470
00471 QPoint toPoint() const {
00472 return QPoint(int(mX), int(mY));
00473 }
00474
00475 QPointF toPointF() const {
00476 return QPointF(mX, mY);
00477 }
00478
00479 bool isNull() const {
00480 return qIsNull(mX) && qIsNull(mY);
00481 }
00482 void normalize();
00483 QCPVector2D normalized() const;
00484
00485 QCPVector2D perpendicular() const {
00486 return QCPVector2D(-mY, mX);
00487 }
00488
00489 double dot(const QCPVector2D &vec) const {
00490 return mX * vec.mX + mY * vec.mY;
00491 }
00492 double distanceSquaredToLine(const QCPVector2D &start, const QCPVector2D &end) const;
00493 double distanceSquaredToLine(const QLineF &line) const;
00494 double distanceToStraightLine(const QCPVector2D &base, const QCPVector2D &direction) const;
00495
00496 QCPVector2D &operator*=(double factor);
00497 QCPVector2D &operator/=(double divisor);
00498 QCPVector2D &operator+=(const QCPVector2D &vector);
00499 QCPVector2D &operator-=(const QCPVector2D &vector);
00500
00501 private:
00502 // property members:
00503 double mX, mY;
00504
00505 friend inline const QCPVector2D operator*(double factor, const QCPVector2D &vec);
00506 friend inline const QCPVector2D operator*(const QCPVector2D &vec, double factor);
00507 friend inline const QCPVector2D operator/(const QCPVector2D &vec, double divisor);
00508 friend inline const QCPVector2D operator+(const QCPVector2D &vec1, const QCPVector2D &vec2);
00509 friend inline const QCPVector2D operator-(const QCPVector2D &vec1, const QCPVector2D &vec2);
00510 friend inline const QCPVector2D operator-(const QCPVector2D &vec);
00511 };
00512 Q_DECLARE_TYPEINFO(QCPVector2D, Q_MOVABLE_TYPE);
00513
00514 inline const QCPVector2D operator*(double factor, const QCPVector2D &vec) {
00515 return QCPVector2D(vec.mX*factor, vec.mY * factor);
00516 }
00517
00518 inline const QCPVector2D operator*(const QCPVector2D &vec, double factor) {
00519 return QCPVector2D(vec.mX*factor, vec.mY * factor);
00520 }
00521
00522 inline const QCPVector2D operator/(const QCPVector2D &vec, double divisor) {
00523 return QCPVector2D(vec.mX / divisor, vec.mY / divisor);
00524 }
00525
00526 inline const QCPVector2D operator+(const QCPVector2D &vec1, const QCPVector2D &vec2) {
00527 return QCPVector2D(vec1.mX + vec2.mX, vec1.mY + vec2.mY);
00528 }
00529
00530 inline const QCPVector2D operator-(const QCPVector2D &vec1, const QCPVector2D &vec2) {
00531 return QCPVector2D(vec1.mX - vec2.mX, vec1.mY - vec2.mY);
00532 }
00533
00534 inline const QCPVector2D operator-(const QCPVector2D &vec) {
00535 return QCPVector2D(-vec.mX, -vec.mY);
00536 }
00537
00542 inlineQDebug operator<<(QDebug d, const QCPVector2D &vec) {
00543 d.nospace() << "QCPVector2D(" << vec.x() << ", " << vec.y() << ")";
00544 return d.space();
00545 }

```



```

00546
00547 /* end of 'src/vector2d.h' */
00548
00549
00550 /* including file 'src/painter.h' */
00551
00552 /* modified 2021-03-29T02:30:44, size 4035 */
00553
00554 class QCP_LIB_DECL QCPPainter : public QPainter {
00555
00556 Q_GADGET
00557 public:
00558 enum PainterMode {
00559 pmDefault = 0x00
00560 , pmVectorized = 0x01
00561 , pmNoCaching = 0x02
00562 , pmNonCosmetic = 0x04
00563 };
00564 Q_ENUMS(PainterMode)
00565 Q_FLAGS(PainterModes)
00566 Q_DECLARE_FLAGS(PainterModes, PainterMode)
00567
00568 QCPPainter();
00569 explicit QCPPainter(QPaintDevice *device);
00570
00571 // getters:
00572
00573 bool antialiasing() const {
00574 return testRenderHint(QPainter::Antialiasing);
00575 }
00576
00577 PainterModes modes() const {
00578 return mModes;
00579 }
00580
00581 // setters:
00582 void setAntialiasing(bool enabled);
00583 void setMode(PainterMode mode, bool enabled = true);
00584 void setModes(PainterModes modes);
00585
00586 // methods hiding non-virtual base class functions (QPainter bug workarounds):
00587 bool begin(QPaintDevice *device);
00588 void setPen(const QPen &pen);
00589 void setPen(const QColor &color);
00590 void setPen(Qt::PenStyle penStyle);
00591 void drawLine(const QLineF &line);
00592
00593 void drawLine(const QPointF &p1, const QPointF &p2) {
00594 drawLine(QLineF(p1, p2));
00595 }
00596 void save();
00597 void restore();
00598
00599 // non-virtual methods:
00600 void makeNonCosmetic();
00601
00602 protected:
00603 // property members:
00604 PainterModes mModes;
00605 bool mIsAntialiasing;
00606
00607 // non-property members:
00608 QStack<bool> mAntialiasingStack;
00609 };
00610
00611 Q_DECLARE_OPERATORS_FOR_FLAGS(QCPPainter::PainterModes)
00612 Q_DECLARE_METATYPE(QCPPainter::PainterMode)
00613
00614 /* end of 'src/painter.h' */
00615
00616
00617 /* including file 'src/paintbuffer.h' */
00618 /* modified 2021-03-29T02:30:44, size 5006 */
00619
00620
00621 class QCP_LIB_DECL QCPAbstractPaintBuffer {
00622 public:
00623 explicit QCPAbstractPaintBuffer(const QSize &size, double devicePixelRatio);
00624 virtual ~QCPAbstractPaintBuffer();
00625
00626 // getters:
00627
00628 QSize size() const {
00629 return mSize;
00630 }
00631
00632 bool invalidated() const {
00633 return mInvalidated;
00634 }
00635
00636

```

```

00637 }
00638
00639 double devicePixelRatio() const {
00640 return mDevicePixelRatio;
00641 }
00642
00643 // setters:
00644 void setSize(const QSize &size);
00645 void setInvalidated(bool invalidated = true);
00646 void setDevicePixelRatio(double ratio);
00647
00648 // introduced virtual methods:
00649 virtual QCPPainter *startPainting() = 0;
00650
00651 virtual void donePainting() {
00652 }
00653 virtual void draw(QCPPainter *painter) const = 0;
00654 virtual void clear(const QColor &color) = 0;
00655
00656 protected:
00657 // property members:
00658 QSize mSize;
00659 double mDevicePixelRatio;
00660
00661 // non-property members:
00662 bool mInvalidated;
00663
00664 // introduced virtual methods:
00665 virtual void reallocateBuffer() = 0;
00666 };
00667
00668 class QCP_LIB_DECL QCPPaintBufferPixmap : public QCPAbstractPaintBuffer {
00669 public:
00670 explicit QCPPaintBufferPixmap(const QSize &size, double devicePixelRatio);
00671 virtual ~QCPPaintBufferPixmap() Q_DECL_OVERRIDE;
00672
00673 // reimplemented virtual methods:
00674 virtual QCPPainter *startPainting() Q_DECL_OVERRIDE;
00675 virtual void draw(QCPPainter *painter) const Q_DECL_OVERRIDE;
00676 void clear(const QColor &color) Q_DECL_OVERRIDE;
00677
00678 protected:
00679 // non-property members:
00680 QPixmap mBuffer;
00681
00682 // reimplemented virtual methods:
00683 virtual void reallocateBuffer() Q_DECL_OVERRIDE;
00684 };
00685
00686 #ifndef QCP_OPENGL_PBUFFER
00687
00688 class QCP_LIB_DECL QCPPaintBufferGLPbuffer : public QCPAbstractPaintBuffer {
00689 public:
00690 explicit QCPPaintBufferGLPbuffer(const QSize &size, double devicePixelRatio, int multisamples);
00691 virtual ~QCPPaintBufferGLPbuffer() Q_DECL_OVERRIDE;
00692
00693 // reimplemented virtual methods:
00694 virtual QCPPainter *startPainting() Q_DECL_OVERRIDE;
00695 virtual void draw(QCPPainter *painter) const Q_DECL_OVERRIDE;
00696 void clear(const QColor &color) Q_DECL_OVERRIDE;
00697
00698 protected:
00699 // non-property members:
00700 QGLPixelBuffer *mGLPbuffer;
00701 int mMultisamples;
00702
00703 // reimplemented virtual methods:
00704 virtual void reallocateBuffer() Q_DECL_OVERRIDE;
00705 };
00706 #endif // QCP_OPENGL_PBUFFER
00707
00708 #ifndef QCP_OPENGL_FBO
00709
00710 class QCP_LIB_DECL QCPPaintBufferGLFbo : public QCPAbstractPaintBuffer {
00711 public:
00712 explicit QCPPaintBufferGLFbo(const QSize &size, double devicePixelRatio,
00713 QWeakPointer<QOpenGLContext> glContext, QWeakPointer<QOpenGLPaintDevice> glPaintDevice);
00714 virtual ~QCPPaintBufferGLFbo() Q_DECL_OVERRIDE;
00715
00716 // reimplemented virtual methods:
00717 virtual QCPPainter *startPainting() Q_DECL_OVERRIDE;
00718 virtual void donePainting() Q_DECL_OVERRIDE;
00719 virtual void draw(QCPPainter *painter) const Q_DECL_OVERRIDE;
00720 void clear(const QColor &color) Q_DECL_OVERRIDE;
00721
00722
```

```

00723 protected:
00724 // non-property members:
00725 QWeakPointer<QOpenGLContext> mGlcContext;
00726 QWeakPointer<QOpenGLPaintDevice> mGlpaintDevice;
00727 QOpenGLFramebufferObject *mGlfFramebuffer;
00728
00729 // reimplemented virtual methods:
00730 virtual void reallocateBuffer() Q_DECL_OVERRIDE;
00731 };
00732 #endif // QCP_OPENGL_FBO
00733
00734 /* end of 'src/paintbuffer.h' */
00735
00736 /* including file 'src/layer.h' */
00737
00738 /* modified 2021-03-29T02:30:44, size 7038 */
00739
00740
00741 class QCP_LIB_DECL QCPLayer : public QObject {
00742
00743 Q_OBJECT
00744 Q_PROPERTY(QCustomPlot* parentPlot READ parentPlot)
00745 Q_PROPERTY(QString name READ name)
00746 Q_PROPERTY(int index READ index)
00747 Q_PROPERTY(QList<QCPLayerable*> children READ children)
00748 Q_PROPERTY(bool visible READ visible WRITE setVisible)
00749 Q_PROPERTY(LayerMode mode READ mode WRITE setMode)
00750 public:
00751
00752 enum LayerMode {
00753 lmLogical
00754 , lmBuffered
00755 };
00756 Q_ENUMS(LayerMode)
00757
00758 QCPLayer(QCustomPlot* parentPlot, const QString &layerName);
00759 virtual ~QCPLayer();
00760
00761 // getters:
00762
00763 QCustomPlot *parentPlot() const {
00764 return mParentPlot;
00765 }
00766
00767 QString name() const {
00768 return mName;
00769 }
00770
00771 int index() const {
00772 return mIndex;
00773 }
00774
00775 QList<QCPLayerable*> children() const {
00776 return mChildren;
00777 }
00778
00779 bool visible() const {
00780 return mVisible;
00781 }
00782
00783 LayerMode mode() const {
00784 return mMode;
00785 }
00786
00787 // setters:
00788 void setVisible(bool visible);
00789 void setMode(LayerMode mode);
00790
00791 // non-virtual methods:
00792 void replot();
00793
00794 protected:
00795 // property members:
00796 QCustomPlot *mParentPlot;
00797 QString mName;
00798 int mIndex;
00799 QList<QCPLayerable*> mChildren;
00800 bool mVisible;
00801 LayerMode mMode;
00802
00803 // non-property members:
00804 QWeakPointer<QCPAbstractPaintBuffer> mPaintBuffer;
00805
00806 // non-virtual methods:
00807 void draw(QCPPainter *painter);
00808 void drawToPaintBuffer();
00809 void addChild(QCPLayerable *layerable, bool prepend);

```

```

00819 void removeChild(QCPLayerable *layerable);
00820
00821 private:
00822 Q_DISABLE_COPY(QCPLayer)
00823
00824 friend class QCustomPlot;
00825 friend class QCPLayerable;
00826 };
00827
00828 Q_DECLARE_METATYPE(QCPLayer::LayerMode)
00829
00830 class QCP_LIB_DECL QCPLayerable : public QObject {
00831 Q_OBJECT
00832 Q_PROPERTY(bool visible READ visible WRITE setVisible)
00833 Q_PROPERTY(QCustomPlot* parentPlot READ parentPlot)
00834 Q_PROPERTY(QCPLayerable* parentLayerable READ parentLayerable)
00835 Q_PROPERTY(QCPLayer* layer READ layer WRITE setLayer NOTIFY layerChanged)
00836 Q_PROPERTY(bool antialiased READ antialiased WRITE setAntialiased)
00837 public:
00838 QCPLayerable(QCustomPlot *plot, QString targetLayer = QString(), QCPLayerable *parentLayerable =
00839 nullptr);
00840 virtual ~QCPLayerable();
00841
00842 // getters:
00843
00844 bool visible() const {
00845 return mVisible;
00846 }
00847
00848 QCustomPlot *parentPlot() const {
00849 return mParentPlot;
00850 }
00851
00852 QCPLayerable *parentLayerable() const {
00853 return mParentLayerable.data();
00854 }
00855
00856 QCPLayer *layer() const {
00857 return mLayer;
00858 }
00859
00860 bool antialiased() const {
00861 return mAntialiased;
00862 }
00863
00864 // setters:
00865 void setVisible(bool on);
00866 Q_SLOT bool setLayer(QCPLayer *layer);
00867 bool setLayer(const QString &layerName);
00868 void setAntialiased(bool enabled);
00869
00870 // introduced virtual methods:
00871 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
00872 const;
00873
00874 // non-property methods:
00875 bool realVisibility() const;
00876
00877 signals:
00878 void layerChanged(QCPLayer *newLayer);
00879
00880 protected:
00881 // property members:
00882 bool mVisible;
00883 QCustomPlot *mParentPlot;
00884 QPointer<QCPLayerable> mParentLayerable;
00885 QCPLayer *mLayer;
00886 bool mAntialiased;
00887
00888 // introduced virtual methods:
00889 virtual void parentPlotInitialized(QCustomPlot *parentPlot);
00890 virtual QCP::Interaction selectionCategory() const;
00891 virtual QRect clipRect() const;
00892 virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const = 0;
00893 virtual void draw(QCPPainter *painter) = 0;
00894 // selection events:
00895 virtual void selectEvent(QMouseEvent *event, bool additive, const QVariant &details, bool
00896 *selectionStateChanged);
00897 virtual void deselectEvent(bool *selectionStateChanged);
00898 // low-level mouse events:
00899 virtual void mousePressEvent(QMouseEvent *event, const QVariant &details);
00900 virtual void mouseMoveEvent(QMouseEvent *event, const QPointF &startPos);
00901 virtual void mouseReleaseEvent(QMouseEvent *event, const QPointF &startPos);
00902 virtual void mouseDoubleClickEvent(QMouseEvent *event, const QVariant &details);
00903 virtual void wheelEvent(QWheelEvent *event);
00904
00905 // non-property methods:

```

```

00905 void initializeParentPlot(QCustomPlot *parentPlot);
00906 void setParentLayerable(QCPLayerable* parentLayerable);
00907 bool moveToLayer(QCPLayer *layer, bool prepend);
00908 void applyAntialiasingHint(QCPPainter *painter, bool localAntialiased, QCP::AntialiasedElement
overrideElement) const;
00909
00910 private:
00911 Q_DISABLE_COPY(QCPLayerable)
00912
00913 friend class QCustomPlot;
00914 friend class QCPLayer;
00915 friend class QCPAxisRect;
00916 };
00917
00918 /* end of 'src/layer.h' */
00919
00920
00921 /* including file 'src/axis/range.h' */
00922
00923 /* modified 2021-03-29T02:30:44, size 5280 */
00924
00925 class QCP_LIB_DECL QCPRange {
00926 public:
00927 double lower, upper;
00928
00929 QCPRange();
00930 QCPRange(double lower, double upper);
00931
00932 bool operator==(const QCPRange& other) const {
00933 return lower == other.lower && upper == other.upper;
00934 }
00935
00936 bool operator!=(const QCPRange& other) const {
00937 return !(*this == other);
00938 }
00939
00940 QCPRange &operator+=(const double& value) {
00941 lower += value;
00942 upper += value;
00943 return *this;
00944 }
00945
00946 QCPRange &operator-=(const double& value) {
00947 lower -= value;
00948 upper -= value;
00949 return *this;
00950 }
00951
00952 QCPRange &operator*=(const double& value) {
00953 lower *= value;
00954 upper *= value;
00955 return *this;
00956 }
00957
00958 QCPRange &operator/=(const double& value) {
00959 lower /= value;
00960 upper /= value;
00961 return *this;
00962 }
00963 friend inline const QCPRange operator+(const QCPRange&, double);
00964 friend inline const QCPRange operator+(double, const QCPRange&);
00965 friend inline const QCPRange operator-(const QCPRange& range, double value);
00966 friend inline const QCPRange operator*(const QCPRange& range, double value);
00967 friend inline const QCPRange operator*(double value, const QCPRange& range);
00968 friend inline const QCPRange operator/(const QCPRange& range, double value);
00969
00970 double size() const {
00971 return upper - lower;
00972 }
00973
00974 double center() const {
00975 return (upper + lower)*0.5;
00976 }
00977
00978 void normalize() {
00979 if (lower > upper) qSwap(lower, upper);
00980 }
00981 void expand(const QCPRange &otherRange);
00982 void expand(double includeCoord);
00983 QCPRange expanded(const QCPRange &otherRange) const;
00984 QCPRange expanded(double includeCoord) const;
00985 QCPRange bounded(double lowerBound, double upperBound) const;
00986 QCPRange sanitizedForLogScale() const;
00987 QCPRange sanitizedForLinScale() const;
00988
00989 bool contains(double value) const {
00990 return value >= lower && value <= upper;

```

```

00991 }
00992
00993 static bool validRange(double lower, double upper);
00994 static bool validRange(const QCPRange &range);
00995 static const double minRange;
00996 static const double maxRange;
00997
00998 };
00999 Q_DECLARE_TYPEINFO(QCPRange, Q_MOVABLE_TYPE);
01000
01005 inline QDebug operator<<(QDebug d, const QCPRange &range) {
01006 d.nospace() << "QCPRange(" << range.lower << ", " << range.upper << ")";
01007 return d.space();
01008 }
01009
01013 inline const QCPRange operator+(const QCPRange& range, double value) {
01014 QCPRange result(range);
01015 result += value;
01016 return result;
01017 }
01018
01022 inline const QCPRange operator+(double value, const QCPRange& range) {
01023 QCPRange result(range);
01024 result += value;
01025 return result;
01026 }
01027
01031 inline const QCPRange operator-(const QCPRange& range, double value) {
01032 QCPRange result(range);
01033 result -= value;
01034 return result;
01035 }
01036
01040 inline const QCPRange operator*(const QCPRange& range, double value) {
01041 QCPRange result(range);
01042 result *= value;
01043 return result;
01044 }
01045
01049 inline const QCPRange operator*(double value, const QCPRange& range) {
01050 QCPRange result(range);
01051 result *= value;
01052 return result;
01053 }
01054
01058 inline const QCPRange operator/(const QCPRange& range, double value) {
01059 QCPRange result(range);
01060 result /= value;
01061 return result;
01062 }
01063
01064 /* end of 'src/axis/range.h' */
01065
01066
01067 /* including file 'src/selection.h' */
01068
01069 /* modified 2021-03-29T02:30:44, size 8569 */
01070
01071 class QCP_LIB_DECL QCPDataRange {
01072 public:
01073 QCPDataRange();
01074 QCPDataRange(int begin, int end);
01075
01076 bool operator==(const QCPDataRange& other) const {
01077 return mBegin == other.mBegin && mEnd == other.mEnd;
01078 }
01079
01080 bool operator!=(const QCPDataRange& other) const {
01081 return !(*this == other);
01082 }
01083
01084 // getters:
01085
01086 int begin() const {
01087 return mBegin;
01088 }
01089
01090 int end() const {
01091 return mEnd;
01092 }
01093
01094 int size() const {
01095 return mEnd - mBegin;
01096 }
01097
01098 int length() const {
01099 return size();

```

```

01100 }
01101
01102 // setters:
01103
01104 void setBegin(int begin) {
01105 mBegin = begin;
01106 }
01107
01108 void setEnd(int end) {
01109 mEnd = end;
01110 }
01111
01112 // non-property methods:
01113
01114 bool isValid() const {
01115 return (mEnd >= mBegin) && (mBegin >= 0);
01116 }
01117
01118 bool isEmpty() const {
01119 return length() == 0;
01120 }
01121
01122 QCPDataRange bounded(const QCPDataRange &other) const;
01123 QCPDataRange expanded(const QCPDataRange &other) const;
01124 QCPDataRange intersection(const QCPDataRange &other) const;
01125
01126 QCPDataRange adjusted(int changeBegin, int changeEnd) const {
01127 return QCPDataRange(mBegin + changeBegin, mEnd + changeEnd);
01128 }
01129
01130 bool intersects(const QCPDataRange &other) const;
01131 bool contains(const QCPDataRange &other) const;
01132
01133 private:
01134 // property members:
01135 int mBegin, mEnd;
01136 };
01137 Q_DECLARE_TYPEINFO(QCPDataRange, Q_MOVABLE_TYPE);
01138
01139 class QCP_LIB_DECL QCPDataSelection {
01140 public:
01141 explicit QCPDataSelection();
01142 explicit QCPDataSelection(const QCPDataRange &range);
01143
01144 bool operator==(const QCPDataSelection& other) const;
01145
01146 bool operator!=(const QCPDataSelection& other) const {
01147 return !(*this == other);
01148 }
01149
01150 QCPDataSelection &operator+=(const QCPDataSelection& other);
01151 QCPDataSelection &operator+=(const QCPDataRange& other);
01152 QCPDataSelection &operator-=(const QCPDataSelection& other);
01153 QCPDataSelection &operator-=(const QCPDataRange& other);
01154 friend inline const QCPDataSelection operator+(const QCPDataSelection& a, const QCPDataSelection&
01155 b);
01156 friend inline const QCPDataSelection operator+(const QCPDataRange& a, const QCPDataSelection& b);
01157 friend inline const QCPDataSelection operator+(const QCPDataSelection& a, const QCPDataRange& b);
01158 friend inline const QCPDataSelection operator+(const QCPDataRange& a, const QCPDataRange& b);
01159 friend inline const QCPDataSelection operator-(const QCPDataSelection& a, const QCPDataSelection&
01160 b);
01161 friend inline const QCPDataSelection operator-(const QCPDataRange& a, const QCPDataSelection& b);
01162 friend inline const QCPDataSelection operator-(const QCPDataSelection& a, const QCPDataRange& b);
01163 friend inline const QCPDataSelection operator-(const QCPDataRange& a, const QCPDataRange& b);
01164
01165 // getters:
01166
01167 int dataRangeCount() const {
01168 return mDataRanges.size();
01169 }
01170
01171 int dataPointCount() const;
01172 QCPDataRange dataRange(int index = 0) const;
01173
01174 QList<QCPDataRange> dataRanges() const {
01175 return mDataRanges;
01176 }
01177
01178 QCPDataRange span() const;
01179
01180 // non-property methods:
01181 void addDataRange(const QCPDataRange &dataRange, bool simplify = true);
01182 void clear();
01183
01184 bool isEmpty() const {
01185 return mDataRanges.isEmpty();
01186 }
01187
01188 void simplify();
01189 void enforceType(QCP::SelectionType type);
01190 bool contains(const QCPDataSelection &other) const;
01191 QCPDataSelection intersection(const QCPDataRange &other) const;

```

```

01185 QCPDataSelection intersection(const QCPDataSelection &other) const;
01186 QCPDataSelection inverse(const QCPDataRange &outerRange) const;
01187
01188 private:
01189 // property members:
01190 QList<QCPDataRange> mDataRanges;
01191
01192 inline static bool lessThanDataRangeBegin(const QCPDataRange &a, const QCPDataRange &b) {
01193 return a.begin() < b.begin();
01194 }
01195 };
01196
01197 Q_DECLARE_METATYPE(QCPDataSelection)
01198
01199
01200
01201 inline const QCPDataSelection operator+(const QCPDataSelection& a, const QCPDataSelection& b) {
01202 QCPDataSelection result(a);
01203 result += b;
01204 return result;
01205 }
01206
01207 inline const QCPDataSelection operator+(const QCPDataRange& a, const QCPDataSelection& b) {
01208 QCPDataSelection result(a);
01209 result += b;
01210 return result;
01211 }
01212
01213 inline const QCPDataSelection operator+(const QCPDataSelection& a, const QCPDataRange& b) {
01214 QCPDataSelection result(a);
01215 result += b;
01216 return result;
01217 }
01218
01219 inline const QCPDataSelection operator+(const QCPDataRange& a, const QCPDataRange& b) {
01220 QCPDataSelection result(a);
01221 result += b;
01222 return result;
01223 }
01224
01225 inline const QCPDataSelection operator-(const QCPDataSelection& a, const QCPDataSelection& b) {
01226 QCPDataSelection result(a);
01227 result -= b;
01228 return result;
01229 }
01230
01231 inline const QCPDataSelection operator-(const QCPDataRange& a, const QCPDataSelection& b) {
01232 QCPDataSelection result(a);
01233 result -= b;
01234 return result;
01235 }
01236
01237 inline const QCPDataSelection operator-(const QCPDataSelection& a, const QCPDataRange& b) {
01238 QCPDataSelection result(a);
01239 result -= b;
01240 return result;
01241 }
01242
01243 inline const QCPDataSelection operator-(const QCPDataRange& a, const QCPDataRange& b) {
01244 QCPDataSelection result(a);
01245 result -= b;
01246 return result;
01247 }
01248
01249 inline QDebug operator<<(QDebug d, const QCPDataRange &dataRange) {
01250 d.nospace() << "QCPDataRange(" << dataRange.begin() << ", " << dataRange.end() << ")";
01251 return d;
01252 }
01253
01254 inline QDebug operator<<(QDebug d, const QCPDataSelection &selection) {
01255 d.nospace() << "QCPDataSelection(";
01256 for (int i = 0; i < selection.dataRangeCount(); ++i) {
01257 if (i != 0)
01258 d << ", ";
01259 d << selection.dataRange(i);
01260 }
01261 d << ")";
01262 return d;
01263 }
01264
01265
01266
01267
01268
01269
01270
01271
01272
01273
01274
01275
01276
01277
01278
01279
01280
01281
01282
01283
01284
01285
01286
01287
01288
01289
01290
01291
01292
01293
01294
01295
01296
01297
01298
01299
01300
01301
01302 /* end of 'src/selection.h' */
01303
01304
01305 /* including file 'src/selectionrect.h' */
01306

```



```

01307 /* modified 2021-03-29T02:30:44, size 3354 */
01308
01309 class QCP_LIB_DECL QCPSelectionRect : public QCPLayerable {
01310 Q_OBJECT
01311 public:
01312 explicit QCPSelectionRect(QCustomPlot *parentPlot);
01313 virtual ~QCPSelectionRect() Q_DECL_OVERRIDE;
01314
01315 // getters:
01316
01317 QRect rect() const {
01318 return mRect;
01319 }
01320 QCPRange range(const QCPAxis *axis) const;
01321
01322 QPen pen() const {
01323 return mPen;
01324 }
01325
01326 QBrush brush() const {
01327 return mBrush;
01328 }
01329
01330 bool isActive() const {
01331 return mActive;
01332 }
01333
01334 // setters:
01335 void setPen(const QPen &pen);
01336 void setBrush(const QBrush &brush);
01337
01338 // non-property methods:
01339 Q_SLOT void cancel();
01340
01341 signals:
01342 void started(QMouseEvent *event);
01343 void changed(const QRect &rect, QMouseEvent *event);
01344 void canceled(const QRect &rect, QInputEvent *event);
01345 void accepted(const QRect &rect, QMouseEvent *event);
01346
01347 protected:
01348 // property members:
01349 QRect mRect;
01350 QPen mPen;
01351 QBrush mBrush;
01352 // non-property members:
01353 bool mActive;
01354
01355 // introduced virtual methods:
01356 virtual void startSelection(QMouseEvent *event);
01357 virtual void moveSelection(QMouseEvent *event);
01358 virtual void endSelection(QMouseEvent *event);
01359 virtual void keyPressEvent(QKeyEvent *event);
01360
01361 // reimplemented virtual methods
01362 virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
01363 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
01364
01365 friend class QCustomPlot;
01366 };
01367
01368 /* end of 'src/selectionrect.h' */
01369
01370 /* including file 'src/layout.h' */
01371
01372 /* modified 2021-03-29T02:30:44, size 14279 */
01373
01374
01375 class QCP_LIB_DECL QCPMarginGroup : public QObject {
01376 Q_OBJECT
01377 public:
01378 explicit QCPMarginGroup(QCustomPlot *parentPlot);
01379 virtual ~QCPMarginGroup();
01380
01381 // non-virtual methods:
01382
01383 QList<QCPLayoutElement*> elements(QCP::MarginSide side) const {
01384 return mChildren.value(side);
01385 }
01386 bool isEmpty() const;
01387 void clear();
01388
01389 protected:
01390 // non-property members:
01391 QCustomPlot *mParentPlot;
01392 QHash<QCP::MarginSide, QList<QCPLayoutElement*> > mChildren;
01393

```

```

01394 // introduced virtual methods:
01395 virtual int commonMargin(QCP::MarginSide side) const;
01396
01397 // non-virtual methods:
01398 void addChild(QCP::MarginSide side, QCPLayoutElement *element);
01399 void removeChild(QCP::MarginSide side, QCPLayoutElement *element);
01400
01401 private:
01402 Q_DISABLE_COPY(QCPMarginGroup)
01403
01404 friend class QCPLayoutElement;
01405 };
01406
01407 class QCP_LIB_DECL QCPLayoutElement : public QCPLayerable {
01408
01409 Q_OBJECT
01410 Q_PROPERTY(QCPLayout* layout READ layout)
01411 Q_PROPERTY(QRect rect READ rect)
01412 Q_PROPERTY(QRect outerRect READ outerRect WRITE setOuterRect)
01413 Q_PROPERTY(QMargins margins READ margins WRITE setMargins)
01414 Q_PROPERTY(QMargins minimumMargins READ minimumMargins WRITE setMinimumMargins)
01415 Q_PROPERTY(QSize minimumSize READ minimumSize WRITE setMinimumSize)
01416 Q_PROPERTY(QSize maximumSize READ maximumSize WRITE setMaximumSize)
01417 Q_PROPERTY(SizeConstraintRect sizeConstraintRect READ sizeConstraintRect WRITE
01418 setSizeConstraintRect)
01419 public:
01420 enum UpdatePhase {
01421 upPreparation
01422 , upMargins
01423 , upLayout
01424 };
01425
01426 Q_ENUMS(UpdatePhase)
01427
01428 enum SizeConstraintRect {
01429 scrInnerRect
01430 , scrOuterRect
01431 };
01432
01433 Q_ENUMS(SizeConstraintRect)
01434
01435 explicit QCPLayoutElement(QCustomPlot *parentPlot = nullptr);
01436 virtual ~QCPLayoutElement() Q_DECL_OVERRIDE;
01437
01438 // getters:
01439
01440 QCPLayout *layout() const {
01441 return mParentLayout;
01442 }
01443
01444 QRect rect() const {
01445 return mRect;
01446 }
01447
01448 QRect outerRect() const {
01449 return mOuterRect;
01450 }
01451
01452 QMargins margins() const {
01453 return mMargins;
01454 }
01455
01456 QMargins minimumMargins() const {
01457 return mMinimumMargins;
01458 }
01459
01460 QCP::MarginSides autoMargins() const {
01461 return mAutoMargins;
01462 }
01463
01464 QSize minimumSize() const {
01465 return mMinimumSize;
01466 }
01467
01468 QSize maximumSize() const {
01469 return mMaximumSize;
01470 }
01471
01472 SizeConstraintRect sizeConstraintRect() const {
01473 return mSizeConstraintRect;
01474 }
01475
01476 QCPMarginGroup *marginGroup(QCP::MarginSide side) const {
01477 return mMarginGroups.value(side, nullptr);
01478 }
01479
01480 QHash<QCP::MarginSide, QCPMarginGroup*> marginGroups() const {

```

```

01493 return mMarginGroups;
01494 }
01495
01496 // setters:
01497 void setOuterRect(const QRect &rect);
01498 void setMargins(const QMargins &margins);
01499 void setMinimumMargins(const QMargins &margins);
01500 void setAutoMargins(QCP::MarginSides sides);
01501 void setMinimumSize(const QSize &size);
01502 void setMinimumSize(int width, int height);
01503 void setMaximumSize(const QSize &size);
01504 void setMaximumSize(int width, int height);
01505 void setSizeConstraintRect(SizeConstraintRect constraintRect);
01506 void setMarginGroup(QCP::MarginSides sides, QCPMarginGroup *group);
01507
01508 // introduced virtual methods:
01509 virtual void update(UpdatePhase phase);
01510 virtual QSize minimumOuterSizeHint() const;
01511 virtual QSize maximumOuterSizeHint() const;
01512 virtual QList<QCPLayoutElement*> elements(bool recursive) const;
01513
01514 // reimplemented virtual methods:
01515 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
01516
01517 protected:
01518 // property members:
01519 QCPLayout *mParentLayout;
01520 QSize mMinimumSize, mMaximumSize;
01521 SizeConstraintRect mSizeConstraintRect;
01522 QRect mRect, mOuterRect;
01523 QMargins mMargins, mMinimumMargins;
01524 QCP::MarginSides mAutoMargins;
01525 QHash<QCP::MarginSide, QCPMarginGroup*> mMarginGroups;
01526
01527 // introduced virtual methods:
01528 virtual int calculateAutoMargin(QCP::MarginSide side);
01529 virtual void layoutChanged();
01530
01531 // reimplemented virtual methods:
01532
01533 virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE {
01534 Q_UNUSED(painter)
01535 }
01536
01537 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE {
01538 Q_UNUSED(painter)
01539 }
01540 virtual void parentPlotInitialized(QCustomPlot *parentPlot) Q_DECL_OVERRIDE;
01541
01542 private:
01543 Q_DISABLE_COPY(QCPLayoutElement)
01544
01545 friend class QCustomPlot;
01546 friend class QCPLayout;
01547 friend class QCPMarginGroup;
01548 };
01549
01550 Q_DECLARE_METATYPE(QCPLayoutElement::UpdatePhase)
01551
01552
01553 class QCP_LIB_DECL QCPLayout : public QCPLayoutElement {
01554 Q_OBJECT
01555 public:
01556 explicit QCPLayout();
01557
01558 // reimplemented virtual methods:
01559 virtual void update(UpdatePhase phase) Q_DECL_OVERRIDE;
01560 virtual QList<QCPLayoutElement*> elements(bool recursive) const Q_DECL_OVERRIDE;
01561
01562 // introduced virtual methods:
01563 virtual int elementCount() const = 0;
01564 virtual QCPLayoutElement* elementAt(int index) const = 0;
01565 virtual QCPLayoutElement* takeAt(int index) = 0;
01566 virtual bool take(QCPLayoutElement* element) = 0;
01567 virtual void simplify();
01568
01569 // non-virtual methods:
01570 bool removeAt(int index);
01571 bool remove(QCPLayoutElement* element);
01572 void clear();
01573
01574 protected:
01575 // introduced virtual methods:
01576 virtual void updateLayout();
01577
01578 // non-virtual methods:

```

```

01579 void sizeConstraintsChanged() const;
01580 void adoptElement(QCPLLayoutElement *el);
01581 void releaseElement(QCPLLayoutElement *el);
01582 QVector<int> getSectionSizes(QVector<int> maxSizes, QVector<int> minSizes, QVector<double>
stretchFactors, int totalSize) const;
01583 static QSize getFinalMinimumOuterSize(const QCPLLayoutElement *el);
01584 static QSize getFinalMaximumOuterSize(const QCPLLayoutElement *el);
01585
01586 private:
01587 Q_DISABLE_COPY(QCPLLayout)
01588 friend class QCPLLayoutElement;
01589 };
01590
01591 class QCP_LIB_DECL QCPLLayoutGrid : public QCPLLayout {
01592
01593 Q_OBJECT
01594 Q_PROPERTY(int rowCount READ rowCount)
01595 Q_PROPERTY(int columnCount READ columnCount)
01596 Q_PROPERTY(QList<double> columnStretchFactors READ columnStretchFactors WRITE
setColumnStretchFactors)
01597 Q_PROPERTY(QList<double> rowStretchFactors READ rowStretchFactors WRITE setRowStretchFactors)
01598 Q_PROPERTY(int columnSpacing READ columnSpacing WRITE setColumnSpacing)
01599 Q_PROPERTY(int rowSpacing READ rowSpacing WRITE setRowSpacing)
01600 Q_PROPERTY(FillOrder fillOrder READ fillOrder WRITE setFillOrder)
01601 Q_PROPERTY(int wrap READ wrap WRITE setWrap)
01602 public:
01603 enum FillOrder {
01604 foRowsFirst
01605 , foColumnsFirst
01606 };
01607 Q_ENUMS(FillOrder)
01608
01609 explicit QCPLLayoutGrid();
01610 virtual ~QCPLLayoutGrid() Q_DECL_OVERRIDE;
01611
01612 // getters:
01613
01614 int rowCount() const {
01615 return mElements.size();
01616 }
01617
01618 int columnCount() const {
01619 return mElements.size() > 0 ? mElements.first().size() : 0;
01620 }
01621
01622 QList<double> columnStretchFactors() const {
01623 return mColumnStretchFactors;
01624 }
01625
01626 QList<double> rowStretchFactors() const {
01627 return mRowStretchFactors;
01628 }
01629
01630 int columnSpacing() const {
01631 return mColumnSpacing;
01632 }
01633
01634 int rowSpacing() const {
01635 return mRowSpacing;
01636 }
01637
01638 int wrap() const {
01639 return mWrap;
01640 }
01641
01642 FillOrder fillOrder() const {
01643 return mFillOrder;
01644 }
01645
01646 // setters:
01647 void setColumnStretchFactor(int column, double factor);
01648 void setColumnStretchFactors(const QList<double> &factors);
01649 void setRowStretchFactor(int row, double factor);
01650 void setRowStretchFactors(const QList<double> &factors);
01651 void setColumnSpacing(int pixels);
01652 void setRowSpacing(int pixels);
01653 void setWrap(int count);
01654 void setFillOrder(FillOrder order, bool rearrange = true);
01655
01656 // reimplemented virtual methods:
01657 virtual void updateLayout() Q_DECL_OVERRIDE;
01658
01659 virtual int elementCount() const Q_DECL_OVERRIDE {
01660 return rowCount() * columnCount();
01661 }
01662 virtual QCPLLayoutElement* elementAt(int index) const Q_DECL_OVERRIDE;

```

```

01673 virtual QCLayoutElement* takeAt(int index) Q_DECL_OVERRIDE;
01674 virtual bool take(QCLayoutElement* element) Q_DECL_OVERRIDE;
01675 virtual QList<QCLayoutElement*> elements(bool recursive) const Q_DECL_OVERRIDE;
01676 virtual void simplify() Q_DECL_OVERRIDE;
01677 virtual QSize minimumOuterSizeHint() const Q_DECL_OVERRIDE;
01678 virtual QSize maximumOuterSizeHint() const Q_DECL_OVERRIDE;
01679
01680 // non-virtual methods:
01681 QCLayoutElement *element(int row, int column) const;
01682 bool addElement(int row, int column, QCLayoutElement *element);
01683 bool addElement(QCLayoutElement *element);
01684 bool hasElement(int row, int column);
01685 void expandTo(int newRowCount, int newColumnCount);
01686 void insertRow(int newIndex);
01687 void insertColumn(int newIndex);
01688 int rowColToIndex(int row, int column) const;
01689 void indexToRowCol(int index, int &row, int &column) const;
01690
01691 protected:
01692 // property members:
01693 QList<QCLayoutElement*> mElements;
01694 QList<double> mColumnStretchFactors;
01695 QList<double> mRowStretchFactors;
01696 int mColumnSpacing, mRowSpacing;
01697 int mWrap;
01698 FillOrder mFillOrder;
01699
01700 // non-virtual methods:
01701 void getMinimumRowColSizes(QVector<int> *minColWidths, QVector<int> *minRowHeights) const;
01702 void getMaximumRowColSizes(QVector<int> *maxColWidths, QVector<int> *maxRowHeights) const;
01703
01704 private:
01705 Q_DISABLE_COPY(QCLayoutGrid)
01706 };
01707
01708 Q_DECLARE_METATYPE(QCLayoutGrid::FillOrder)
01709
01710 class QCP_LIB_DECL QCLayoutInset : public QCLayout {
01711 public:
01712 Q_OBJECT
01713 public:
01714 enum InsetPlacement {
01715 ipFree
01716 , ipBorderAligned
01717 };
01718 Q_ENUMS(InsetPlacement)
01719
01720 explicit QCLayoutInset();
01721 virtual ~QCLayoutInset() Q_DECL_OVERRIDE;
01722
01723 // getters:
01724 InsetPlacement insetPlacement(int index) const;
01725 Qt::Alignment insetAlignment(int index) const;
01726 QRectF insetRect(int index) const;
01727
01728 // setters:
01729 void setInsetPlacement(int index, InsetPlacement placement);
01730 void setInsetAlignment(int index, Qt::Alignment alignment);
01731 void setInsetRect(int index, const QRectF &rect);
01732
01733 // reimplemented virtual methods:
01734 virtual void updateLayout() Q_DECL_OVERRIDE;
01735 virtual int elementCount() const Q_DECL_OVERRIDE;
01736 virtual QCLayoutElement* elementAt(int index) const Q_DECL_OVERRIDE;
01737 virtual QCLayoutElement* takeAt(int index) Q_DECL_OVERRIDE;
01738 virtual bool take(QCLayoutElement* element) Q_DECL_OVERRIDE;
01739
01740 virtual void simplify() Q_DECL_OVERRIDE {
01741 }
01742 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
01743 const Q_DECL_OVERRIDE;
01744
01745 // non-virtual methods:
01746 void addElement(QCLayoutElement *element, Qt::Alignment alignment);
01747 void addElement(QCLayoutElement *element, const QRectF &rect);
01748
01749 protected:
01750 // property members:
01751 QList<QCLayoutElement*> mElements;
01752 QList<InsetPlacement> mInsetPlacement;
01753 QList<Qt::Alignment> mInsetAlignment;
01754 QList<QRectF> mInsetRect;
01755
01756 private:
01757 Q_DISABLE_COPY(QCLayoutInset)
01758 };

```



```

01863
01864
01867 enum AnchorReferenceType {
01868 artNormal
01869 , artTangent
01870 };
01871
01872 Q_ENUMS(AnchorReferenceType)
01873
01874
01877 enum AnchorSide {
01878 asLeft
01879 , asRight
01880 , asTop
01881 , asBottom
01882 , asTopLeft
01883 , asTopRight
01884 , asBottomRight
01885 , asBottomLeft
01886 };
01887 Q_ENUMS(AnchorSide)
01888
01889 explicit QCPLLabelPainterPrivate(QCustomPlot *parentPlot);
01890 virtual ~QCPLLabelPainterPrivate();
01891
01892 // setters:
01893 void setAnchorSide(AnchorSide side);
01894 void setAnchorMode(AnchorMode mode);
01895 void setAnchorReference(const QPointF &pixelPoint);
01896 void setAnchorReferenceType(AnchorReferenceType type);
01897 void setFont(const QFont &font);
01898 void setColor(const QColor &color);
01899 void setPadding(int padding);
01900 void setRotation(double rotation);
01901 void setSubstituteExponent(bool enabled);
01902 void setMultiplicationSymbol(QChar symbol);
01903 void setAbbreviateDecimalPowers(bool enabled);
01904 void setCacheSize(int labelCount);
01905
01906 // getters:
01907
01908 AnchorMode anchorMode() const {
01909 return mAnchorMode;
01910 }
01911
01912 AnchorSide anchorSide() const {
01913 return mAnchorSide;
01914 }
01915
01916 QPointF anchorReference() const {
01917 return mAnchorReference;
01918 }
01919
01920 AnchorReferenceType anchorReferenceType() const {
01921 return mAnchorReferenceType;
01922 }
01923
01924 QFont font() const {
01925 return mFont;
01926 }
01927
01928 QColor color() const {
01929 return mColor;
01930 }
01931
01932 int padding() const {
01933 return mPadding;
01934 }
01935
01936 double rotation() const {
01937 return mRotation;
01938 }
01939
01940 bool substituteExponent() const {
01941 return mSubstituteExponent;
01942 }
01943
01944 QChar multiplicationSymbol() const {
01945 return mMultiplicationSymbol;
01946 }
01947
01948 bool abbreviateDecimalPowers() const {
01949 return mAbbreviateDecimalPowers;
01950 }
01951 int cacheSize() const;
01952
01953 //virtual int size() const;

```

```

01954
01955 // non-property methods:
01956 void drawTickLabel(QCPPainter *painter, const QPointF &tickPos, const QString &text);
01957 void clearCache();
01958
01959 // constants that may be used with setMultiplicationSymbol:
01960 static const QChar SymbolDot;
01961 static const QChar SymbolCross;
01962
01963 protected:
01964
01965 struct CachedLabel {
01966 QPoint offset;
01967 QPixmap pixmap;
01968 };
01969
01970 struct LabelData {
01971 AnchorSide side;
01972 double rotation; // angle in degrees
01973 QTransform transform; // the transform about the label anchor which is at (0, 0). Does not
01974 contain final absolute x/y positioning on the plot/axis
01975 QString basePart, expPart, suffixPart;
01976 QRect baseBounds, expBounds, suffixBounds;
01977 QRect totalBounds; // is in a coordinate system where label top left is at (0, 0)
01978 QRect rotatedTotalBounds; // is in a coordinate system where the label anchor is at (0, 0)
01979 QFont baseFont, expFont;
01980 QColor color;
01981 };
01982
01983 // property members:
01984 AnchorMode mAnchorMode;
01985 AnchorSide mAnchorSide;
01986 QPointF mAnchorReference;
01987 AnchorReferenceType mAnchorReferenceType;
01988 QFont mFont;
01989 QColor mColor;
01990 int mPadding;
01991 double mRotation; // this is the rotation applied uniformly to all labels, not the heterogeneous
01992 rotation in amCircularRotated mode
01993 bool mSubstituteExponent;
01994 QChar mMultiplicationSymbol;
01995 bool mAbbreviateDecimalPowers;
01996 // non-property members:
01997 QCustomPlot *mParentPlot;
01998 QByteArray mLabelParameterHash; // to determine whether mLabelCache needs to be cleared due to
01999 changed parameters
02000 QCache<QString, CachedLabel> mLabelCache;
02001 QRect mAxisSelectionBox, mTickLabelsSelectionBox, mLabelSelectionBox;
02002 int mLetterCapHeight, mLetterDescent;
02003
02004 // introduced virtual methods:
02005 virtual void drawLabelMaybeCached(QCPPainter *painter, const QFont &font, const QColor &color,
02006 const QPointF &pos, AnchorSide side, double rotation, const QString &text);
02007 virtual QByteArray generateLabelParameterHash() const; // TODO: get rid of this in favor of
02008 invalidation flag upon setters?
02009
02010 // non-virtual methods:
02011 QPointF getAnchorPos(const QPointF &tickPos);
02012 void drawText(QCPPainter *painter, const QPointF &pos, const LabelData &labelData) const;
02013 LabelData getTickLabelData(const QFont &font, const QColor &color, double rotation, AnchorSide
02014 side, const QString &text) const;
02015 void applyAnchorTransform(LabelData &labelData) const;
02016 //void getMaxTickLabelSize(const QFont &font, const QString &text, QSize *tickLabelsSize) const;
02017 CachedLabel *createCachedLabel(const LabelData &labelData) const;
02018 QByteArray cacheKey(const QString &text, const QColor &color, double rotation, AnchorSide side)
02019 const;
02020 AnchorSide skewedAnchorSide(const QPointF &tickPos, double sideExpandHorz, double sideExpandVert)
02021 const;
02022 AnchorSide rotationCorrectedSide(AnchorSide side, double rotation) const;
02023 void analyzeFontMetrics();
02024 };
02025
02026 Q_DECLARE_METATYPE(QCPLabelPainterPrivate::AnchorMode)
02027 Q_DECLARE_METATYPE(QCPLabelPainterPrivate::AnchorSide)
02028
02029 /* end of 'src/axis/labelpainter.h' */
02030
02031 /* including file 'src/axis/axisticker.h' */
02032 /* modified 2021-03-29T02:30:44, size 4230 */
02033
02034 class QCP_LIB_DECL QCPAxisTicker {
02035 Q_GADGET
02036 public:
02037 enum TickStepStrategy {

```



```

02038 tssReadability
02039 , tssMeetTickCount
02040 };
02041 Q_ENUMS(TickStepStrategy)
02042
02043 QCPAxisTicker();
02044 virtual ~QCPAxisTicker();
02045
02046 // getters:
02047
02048 TickStepStrategy tickStepStrategy() const {
02049 return mTickStepStrategy;
02050 }
02051
02052 int tickCount() const {
02053 return mTickCount;
02054 }
02055
02056 double tickOrigin() const {
02057 return mTickOrigin;
02058 }
02059
02060 // setters:
02061 void setTickStepStrategy(TickStepStrategy strategy);
02062 void setTickCount(int count);
02063 void setTickOrigin(double origin);
02064
02065 // introduced virtual methods:
02066 virtual void generate(const QCPRange &range, const QLocale &locale, QChar formatChar, int
precision, QVector<double> &ticks, QVector<double> *subTicks, QVector<QString> *tickLabels);
02067
02068 protected:
02069 // property members:
02070 TickStepStrategy mTickStepStrategy;
02071 int mTickCount;
02072 double mTickOrigin;
02073
02074 // introduced virtual methods:
02075 virtual double getTickStep(const QCPRange &range);
02076 virtual int getSubTickCount(double tickStep);
02077 virtual QString getTickLabel(double tick, const QLocale &locale, QChar formatChar, int precision);
02078 virtual QVector<double> createTickVector(double tickStep, const QCPRange &range);
02079 virtual QVector<double> createSubTickVector(int subTickCount, const QVector<double> &ticks);
02080 virtual QVector<QString> createLabelVector(const QVector<double> &ticks, const QLocale &locale,
QChar formatChar, int precision);
02081
02082 // non-virtual methods:
02083 void trimTicks(const QCPRange &range, QVector<double> &ticks, bool keepOneOutlier) const;
02084 double pickClosest(double target, const QVector<double> &candidates) const;
02085 double getMantissa(double input, double *magnitude = nullptr) const;
02086 double cleanMantissa(double input) const;
02087
02088 private:
02089 Q_DISABLE_COPY(QCPAxisTicker)
02090
02091 };
02092
02093 Q_DECLARE_METATYPE(QCPAxisTicker::TickStepStrategy)
02094 Q_DECLARE_METATYPE(QSharedPointer<QCPAxisTicker>)
02095
02096 /* end of 'src/axis/axisticker.h' */
02097
02098
02099 /* including file 'src/axis/axistickerdatetime.h' */
02100 /* modified 2021-03-29T02:30:44, size 3600 */
02101
02102 class QCP_LIB_DECL QCPAxisTickerDateTime : public QCPAxisTicker {
02103 public:
02104 QCPAxisTickerDateTime();
02105
02106 // getters:
02107
02108 QString dateTimeFormat() const {
02109 return mDateTimeFormat;
02110 }
02111
02112 Qt::TimeSpec dateTimeSpec() const {
02113 return mDateTimeSpec;
02114 }
02115 #if QT_VERSION >= QT_VERSION_CHECK(5, 2, 0)
02116 QTimeZone timeZone() const {
02117 return mTimeZone;
02118 }
02119 #endif
02120 #endif
02121
02122 // setters:

```

```

02123 void setDateTimeFormat(const QString &format);
02124 void setTimeSpec(Qt::TimeSpec spec);
02125 #if QT_VERSION >= QT_VERSION_CHECK(5, 2, 0)
02126 void setTimeZone(const QTimeZone &zone);
02127 #endif
02128 void setTickOrigin(double origin); // hides base class method but calls baseclass implementation
 ("using" throws off IDEs and doxygen)
02129 void setTickOrigin(const QDateTime &origin);
02130
02131 // static methods:
02132 static QDateTime keyToDateTime(double key);
02133 static double dateTimeToKey(const QDateTime &dateTime);
02134 static double dateTimeToKey(const QDate &date, Qt::TimeSpec timeSpec = Qt::LocalTime);
02135
02136 protected:
02137 // property members:
02138 QString mDateTimeFormat;
02139 Qt::TimeSpec mDateTimeSpec;
02140 #if QT_VERSION >= QT_VERSION_CHECK(5, 2, 0)
02141 QTimeZone mTimeZone;
02142 #endif
02143 // non-property members:
02144
02145 enum DateStrategy {
02146 dsNone, dsUniformTimeInDay, dsUniformDayInMonth
02147 } mDateStrategy;
02148
02149 // reimplemented virtual methods:
02150 virtual double getTickStep(const QCPRange &range) Q_DECL_OVERRIDE;
02151 virtual int getSubTickCount(double tickStep) Q_DECL_OVERRIDE;
02152 virtual QString getTickLabel(double tick, const QLocale &locale, QChar formatChar, int precision)
 Q_DECL_OVERRIDE;
02153 virtual QVector<double> createTickVector(double tickStep, const QCPRange &range) Q_DECL_OVERRIDE;
02154 };
02155
02156 /* end of 'src/axis/axistickerdatetime.h' */
02157
02158
02159 /* including file 'src/axis/axistickertime.h' */
02160
02161 /* modified 2021-03-29T02:30:44, size 3542 */
02162
02163 class QCP_LIB_DECL QCPAxisTickerTime : public QCPAxisTicker {
02164
02165 Q_GADGET
02166 public:
02167 enum TimeUnit {
02168 tuMilliseconds
02169 , tuSeconds
02170 , tuMinutes
02171 , tuHours
02172 , tuDays
02173 };
02174 Q_ENUMS(TimeUnit)
02175
02176 QCPAxisTickerTime();
02177
02178 // getters:
02179
02180 QString timeFormat() const {
02181 return mTimeFormat;
02182 }
02183
02184 int fieldWidth(TimeUnit unit) const {
02185 return mFieldWidth.value(unit);
02186 }
02187
02188 // setters:
02189 void setTimeFormat(const QString &format);
02190 void setFieldWidth(TimeUnit unit, int width);
02191
02192 protected:
02193 // property members:
02194 QString mTimeFormat;
02195 QHash<TimeUnit, int> mFieldWidth;
02196
02197 // non-property members:
02198 TimeUnit mSmallestUnit, mBiggestUnit;
02199 QHash<TimeUnit, QString> mFormatPattern;
02200
02201 // reimplemented virtual methods:
02202 virtual double getTickStep(const QCPRange &range) Q_DECL_OVERRIDE;
02203 virtual int getSubTickCount(double tickStep) Q_DECL_OVERRIDE;
02204 virtual QString getTickLabel(double tick, const QLocale &locale, QChar formatChar, int precision)
 Q_DECL_OVERRIDE;
02205
02206 // non-virtual methods:

```

```

02212 void replaceUnit(QString &text, TimeUnit unit, int value) const;
02213 };
02214
02215 Q_DECLARE_METATYPE(QCPAxisTickerTime::TimeUnit)
02216
02217 /* end of 'src/axis/axistickertime.h' */
02218
02219
02220 /* including file 'src/axis/axistickerfixed.h' */
02221 /* modified 2021-03-29T02:30:44, size 3308 */
02222
02223 class QCP_LIB_DECL QCPAxisTickerFixed : public QCPAxisTicker {
02224 Q_GADGET
02225 public:
02226 enum ScaleStrategy {
02227 ssNone
02228 , ssMultiples
02229 , ssPowers
02230 };
02231 Q_ENUMS(ScaleStrategy)
02232
02233 QCPAxisTickerFixed();
02234
02235 // getters:
02236
02237 double tickStep() const {
02238 return mTickStep;
02239 }
02240
02241 ScaleStrategy scaleStrategy() const {
02242 return mScaleStrategy;
02243 }
02244
02245 // setters:
02246 void setTickStep(double step);
02247 void setScaleStrategy(ScaleStrategy strategy);
02248
02249 protected:
02250 // property members:
02251 double mTickStep;
02252 ScaleStrategy mScaleStrategy;
02253
02254 // reimplemented virtual methods:
02255 virtual double getTickStep(const QCPRange &range) Q_DECL_OVERRIDE;
02256 };
02257
02258 Q_DECLARE_METATYPE(QCPAxisTickerFixed::ScaleStrategy)
02259
02260 /* end of 'src/axis/axistickerfixed.h' */
02261
02262
02263 /* including file 'src/axis/axistickertext.h' */
02264 /* modified 2021-03-29T02:30:44, size 3090 */
02265
02266 class QCP_LIB_DECL QCPAxisTickerText : public QCPAxisTicker {
02267 public:
02268 QCPAxisTickerText();
02269
02270 // getters:
02271
02272 QMap<double, QString> &ticks() {
02273 return mTicks;
02274 }
02275
02276 int subTickCount() const {
02277 return mSubTickCount;
02278 }
02279
02280 // setters:
02281 void setTicks(const QMap<double, QString> &ticks);
02282 void setTicks(const QVector<double> &positions, const QVector<QString> &labels);
02283 void setSubTickCount(int subTicks);
02284
02285 // non-virtual methods:
02286 void clear();
02287 void addTick(double position, const QString &label);
02288 void addTicks(const QMap<double, QString> &ticks);
02289 void addTicks(const QVector<double> &positions, const QVector<QString> &labels);
02290
02291 protected:
02292 // property members:
02293 QMap<double, QString> mTicks;
02294 int mSubTickCount;
02295
02296 // reimplemented virtual methods:
02297 virtual double getTickStep(const QCPRange &range) Q_DECL_OVERRIDE;

```

```

02305 virtual int getSubTickCount(double tickStep) Q_DECL_OVERRIDE;
02306 virtual QString getTickLabel(double tick, const QLocale &locale, QChar formatChar, int precision)
02307 Q_DECL_OVERRIDE;
02308 virtual QVector<double> createTickVector(double tickStep, const QCPRange &range) Q_DECL_OVERRIDE;
02309 };
02310 /* end of 'src/axis/axistickertext.h' */
02311
02312
02313 /* including file 'src/axis/axistickerpi.h' */
02314
02315 /* modified 2021-03-29T02:30:44, size 3911 */
02316
02317 class QCP_LIB_DECL QCPAxisTickerPi : public QCPAxisTicker {
02318
02319 Q_GADGET
02320 public:
02321 enum FractionStyle {
02322 fsFloatingPoint
02323 , fsAsciiFractions
02324 , fsUnicodeFractions
02325 };
02326 Q_ENUMS(FractionStyle)
02327
02328 QCPAxisTickerPi();
02329
02330 // getters:
02331
02332 QString piSymbol() const {
02333 return mPiSymbol;
02334 }
02335
02336 double piValue() const {
02337 return mPiValue;
02338 }
02339
02340 bool periodicity() const {
02341 return mPeriodicity;
02342 }
02343
02344 FractionStyle fractionStyle() const {
02345 return mFractionStyle;
02346 }
02347
02348 // setters:
02349 void setPiSymbol(QString symbol);
02350 void setPiValue(double pi);
02351 void setPeriodicity(int multiplesOfPi);
02352 void setFractionStyle(FractionStyle style);
02353
02354 protected:
02355 // property members:
02356 QString mPiSymbol;
02357 double mPiValue;
02358 int mPeriodicity;
02359 FractionStyle mFractionStyle;
02360
02361 // non-property members:
02362 double mPiTickStep; // size of one tick step in units of mPiValue
02363
02364 // reimplemented virtual methods:
02365 virtual double getTickStep(const QCPRange &range) Q_DECL_OVERRIDE;
02366 virtual int getSubTickCount(double tickStep) Q_DECL_OVERRIDE;
02367 virtual QString getTickLabel(double tick, const QLocale &locale, QChar formatChar, int precision)
02368 Q_DECL_OVERRIDE;
02369
02370 // non-virtual methods:
02371 void simplifyFraction(int &numerator, int &denominator) const;
02372 QString fractionToString(int numerator, int denominator) const;
02373 QString unicodeFraction(int numerator, int denominator) const;
02374 QString unicodeSuperscript(int number) const;
02375 QString unicodeSubscript(int number) const;
02376 };
02377
02378 Q_DECLARE_METATYPE(QCPAxisTickerPi::FractionStyle)
02379
02380 /* end of 'src/axis/axistickerpi.h' */
02381
02382
02383 /* including file 'src/axis/axistickerlog.h' */
02384 /* modified 2021-03-29T02:30:44, size 2594 */
02385
02386 class QCP_LIB_DECL QCPAxisTickerLog : public QCPAxisTicker {
02387 public:
02388 QCPAxisTickerLog();
02389
02390 // getters:

```

```

02395
02396 double logBase() const {
02397 return mLogBase;
02398 }
02399
02400 int subTickCount() const {
02401 return mSubTickCount;
02402 }
02403
02404 // setters:
02405 void setLogBase(double base);
02406 void setSubTickCount(int subTicks);
02407
02408 protected:
02409 // property members:
02410 double mLogBase;
02411 int mSubTickCount;
02412
02413 // non-property members:
02414 double mLogBaseLnInv;
02415
02416 // reimplemented virtual methods:
02417 virtual int getSubTickCount(double tickStep) Q_DECL_OVERRIDE;
02418 virtual QVector<double> createTickVector(double tickStep, const QCPRange &range) Q_DECL_OVERRIDE;
02419 };
02420
02421 /* end of 'src/axis/axistickerlog.h' */
02422
02423
02424 /* including file 'src/axis/axis.h' */
02425
02426 /* modified 2021-03-29T02:30:44, size 20913 */
02427
02428 class QCP_LIB_DECL QCPGrid : public QCPLayerable {
02429 Q_OBJECT
02430 Q_PROPERTY(bool subGridVisible READ subGridVisible WRITE setSubGridVisible)
02431 Q_PROPERTY(bool antialiasedSubGrid READ antialiasedSubGrid WRITE setAntialiasedSubGrid)
02432 Q_PROPERTY(bool antialiasedZeroLine READ antialiasedZeroLine WRITE setAntialiasedZeroLine)
02433 Q_PROPERTY(QPen pen READ pen WRITE setPen)
02434 Q_PROPERTY(QPen subGridPen READ subGridPen WRITE setSubGridPen)
02435 Q_PROPERTY(QPen zeroLinePen READ zeroLinePen WRITE setZeroLinePen)
02436 public:
02437 explicit QCPGrid(QCPAxis *parentAxis);
02438
02439 // getters:
02440
02441 bool subGridVisible() const {
02442 return mSubGridVisible;
02443 }
02444
02445 bool antialiasedSubGrid() const {
02446 return mAntialiasedSubGrid;
02447 }
02448
02449 bool antialiasedZeroLine() const {
02450 return mAntialiasedZeroLine;
02451 }
02452
02453 QPen pen() const {
02454 return mPen;
02455 }
02456
02457 QPen subGridPen() const {
02458 return mSubGridPen;
02459 }
02460
02461 QPen zeroLinePen() const {
02462 return mZeroLinePen;
02463 }
02464
02465 // setters:
02466 void setSubGridVisible(bool visible);
02467 void setAntialiasedSubGrid(bool enabled);
02468 void setAntialiasedZeroLine(bool enabled);
02469 void setPen(const QPen &pen);
02470 void setSubGridPen(const QPen &pen);
02471 void setZeroLinePen(const QPen &pen);
02472
02473 protected:
02474 // property members:
02475 bool mSubGridVisible;
02476 bool mAntialiasedSubGrid, mAntialiasedZeroLine;
02477 QPen mPen, mSubGridPen, mZeroLinePen;
02478
02479 // non-property members:
02480 QCPAxis *mParentAxis;
02481
02482
02483

```

```

02484 // reimplemented virtual methods:
02485 virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
02486 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
02487
02488 // non-virtual methods:
02489 void drawGridLines(QCPPainter *painter) const;
02490 void drawSubGridLines(QCPPainter *painter) const;
02491
02492 friend class QCPAxis;
02493 };
02494
02495 class QCP_LIB_DECL QCPAxis : public QCPLayerable {
02496 Q_OBJECT
02497 Q_PROPERTY(AxisType axisType READ axisType)
02498 Q_PROPERTY(QCPAxisRect* axisRect READ axisRect)
02499 Q_PROPERTY(ScaleType scaleType READ scaleType WRITE setScaleType NOTIFY scaleTypeChanged)
02500 Q_PROPERTY(QCPRange range READ range WRITE setRange NOTIFY rangeChanged)
02501 Q_PROPERTY(bool rangeReversed READ rangeReversed WRITE setRangeReversed)
02502 Q_PROPERTY(QSharedPointer<QCPAxisTicker> ticker READ ticker WRITE setTicker)
02503 Q_PROPERTY(bool ticks READ ticks WRITE setTicks)
02504 Q_PROPERTY(bool tickLabels READ tickLabels WRITE setTickLabels)
02505 Q_PROPERTY(int tickLabelPadding READ tickLabelPadding WRITE setTickLabelPadding)
02506 Q_PROPERTY(QFont tickLabelFont READ tickLabelFont WRITE setTickLabelFont)
02507 Q_PROPERTY(QColor tickLabelColor READ tickLabelColor WRITE setTickLabelColor)
02508 Q_PROPERTY(double tickLabelRotation READ tickLabelRotation WRITE setTickLabelRotation)
02509 Q_PROPERTY(LabelSide tickLabelSide READ tickLabelSide WRITE setTickLabelSide)
02510 Q_PROPERTY(QString numberFormat READ numberFormat WRITE setNumberFormat)
02511 Q_PROPERTY(int numberPrecision READ numberPrecision WRITE setNumberPrecision)
02512 Q_PROPERTY(QVector<double> tickVector READ tickVector)
02513 Q_PROPERTY(QVector<QString> tickVectorLabels READ tickVectorLabels)
02514 Q_PROPERTY(int tickLengthIn READ tickLengthIn WRITE setTickLengthIn)
02515 Q_PROPERTY(int tickLengthOut READ tickLengthOut WRITE setTickLengthOut)
02516 Q_PROPERTY(bool subTicks READ subTicks WRITE setSubTicks)
02517 Q_PROPERTY(int subTickLengthIn READ subTickLengthIn WRITE setSubTickLengthIn)
02518 Q_PROPERTY(int subTickLengthOut READ subTickLengthOut WRITE setSubTickLengthOut)
02519 Q_PROPERTY(QPen basePen READ basePen WRITE setBasePen)
02520 Q_PROPERTY(QPen tickPen READ tickPen WRITE setTickPen)
02521 Q_PROPERTY(QPen subTickPen READ subTickPen WRITE setSubTickPen)
02522 Q_PROPERTY(QFont labelFont READ labelFont WRITE setLabelFont)
02523 Q_PROPERTY(QColor labelColor READ labelColor WRITE setLabelColor)
02524 Q_PROPERTY(QString label READ label WRITE setLabel)
02525 Q_PROPERTY(int labelPadding READ labelPadding WRITE setLabelPadding)
02526 Q_PROPERTY(int padding READ padding WRITE setPadding)
02527 Q_PROPERTY(int offset READ offset WRITE setOffset)
02528 Q_PROPERTY(SelectableParts selectedParts READ selectedParts WRITE setSelectedParts NOTIFY
02529 selectionChanged)
02530 Q_PROPERTY(SelectableParts selectableParts READ selectableParts WRITE setSelectedParts NOTIFY
02531 selectableChanged)
02532 Q_PROPERTY(QFont selectedTickLabelFont READ selectedTickLabelFont WRITE setSelectedTickLabelFont)
02533 Q_PROPERTY(QFont selectedLabelFont READ selectedLabelFont WRITE setSelectedLabelFont)
02534 Q_PROPERTY(QColor selectedTickLabelColor READ selectedTickLabelColor WRITE
02535 setSelectedTickLabelColor)
02536 Q_PROPERTY(QColor selectedLabelColor READ selectedLabelColor WRITE setSelectedLabelColor)
02537 Q_PROPERTY(QPen selectedBasePen READ selectedBasePen WRITE setSelectedBasePen)
02538 Q_PROPERTY(QPen selectedTickPen READ selectedTickPen WRITE setSelectedTickPen)
02539 Q_PROPERTY(QPen selectedSubTickPen READ selectedSubTickPen WRITE setSelectedSubTickPen)
02540 Q_PROPERTY(QCPLineEnding lowerEnding READ lowerEnding WRITE setLowerEnding)
02541 Q_PROPERTY(QCPLineEnding upperEnding READ upperEnding WRITE setUpperEnding)
02542 Q_PROPERTY(QCPGrid* grid READ grid)
02543 public:
02544 enum AxisType {
02545 atLeft = 0x01
02546 , atRight = 0x02
02547 , atTop = 0x04
02548 , atBottom = 0x08
02549 };
02550
02551 Q_ENUMS(AxisType)
02552 Q_FLAGS(AxisTypes)
02553 Q_DECLARE_FLAGS(AxisTypes, AxisType)
02554 enum LabelSide {
02555 lsInside
02556 , lsOutside
02557 };
02558 Q_ENUMS(LabelSide)
02559 enum ScaleType {
02560 stLinear
02561 , stLogarithmic
02562 };
02563 Q_ENUMS(ScaleType)
02564 enum SelectablePart {
02565 spNone = 0
02566 , spAxis = 0x001
02567 , spTickLabels = 0x002

```

```
02587 , spAxisLabel = 0x004
02588 };
02589 Q_ENUMS(SelectablePart)
02590 Q_FLAGS(SelectableParts)
02591 Q_DECLARE_FLAGS(SelectableParts, SelectablePart)
02592
02593 explicit QCPAxis(QCPAxisRect *parent, AxisType type);
02594 virtual ~QCPAxis() Q_DECL_OVERRIDE;
02595
02596 // getters:
02597
02598 AxisType axisType() const {
02599 return mAxisType;
02600 }
02601
02602 QCPAxisRect *axisRect() const {
02603 return mAxisRect;
02604 }
02605
02606 ScaleType scaleType() const {
02607 return mScaleType;
02608 }
02609
02610 const QCPRange range() const {
02611 return mRange;
02612 }
02613
02614 bool rangeReversed() const {
02615 return mRangeReversed;
02616 }
02617
02618 QSharedPointer<QCPAxisTicker> ticker() const {
02619 return mTicker;
02620 }
02621
02622 bool ticks() const {
02623 return mTicks;
02624 }
02625
02626 bool tickLabels() const {
02627 return mTickLabels;
02628 }
02629 int tickLabelPadding() const;
02630
02631 QFont tickLabelFont() const {
02632 return mTickLabelFont;
02633 }
02634
02635 QColor tickLabelColor() const {
02636 return mTickLabelColor;
02637 }
02638 double tickLabelRotation() const;
02639 LabelSide tickLabelSide() const;
02640 QString numberFormat() const;
02641
02642 int numberPrecision() const {
02643 return mNumberPrecision;
02644 }
02645
02646 QVector<double> tickVector() const {
02647 return mTickVector;
02648 }
02649
02650 QVector<QString> tickVectorLabels() const {
02651 return mTickVectorLabels;
02652 }
02653 int tickLengthIn() const;
02654 int tickLengthOut() const;
02655
02656 bool subTicks() const {
02657 return mSubTicks;
02658 }
02659 int subTickLengthIn() const;
02660 int subTickLengthOut() const;
02661
02662 QPen basePen() const {
02663 return mBasePen;
02664 }
02665
02666 QPen tickPen() const {
02667 return mTickPen;
02668 }
02669
02670 QPen subTickPen() const {
02671 return mSubTickPen;
02672 }
02673
```

```

02674 QFont labelFont() const {
02675 return mLabelFont;
02676 }
02677
02678 QColor labelColor() const {
02679 return mLabelColor;
02680 }
02681
02682 QString label() const {
02683 return mLabel;
02684 }
02685 int labelPadding() const;
02686
02687 int padding() const {
02688 return mPadding;
02689 }
02690 int offset() const;
02691
02692 SelectableParts selectedParts() const {
02693 return mSelectedParts;
02694 }
02695
02696 SelectableParts selectableParts() const {
02697 return mSelectableParts;
02698 }
02699
02700 QFont selectedTickLabelFont() const {
02701 return mSelectedTickLabelFont;
02702 }
02703
02704 QFont selectedLabelFont() const {
02705 return mSelectedLabelFont;
02706 }
02707
02708 QColor selectedTickLabelColor() const {
02709 return mSelectedTickLabelColor;
02710 }
02711
02712 QColor selectedLabelColor() const {
02713 return mSelectedLabelColor;
02714 }
02715
02716 QPen selectedBasePen() const {
02717 return mSelectedBasePen;
02718 }
02719
02720 QPen selectedTickPen() const {
02721 return mSelectedTickPen;
02722 }
02723
02724 QPen selectedSubTickPen() const {
02725 return mSelectedSubTickPen;
02726 }
02727 QCPLLineEnding lowerEnding() const;
02728 QCPLLineEnding upperEnding() const;
02729
02730 QCPGrid *grid() const {
02731 return mGrid;
02732 }
02733
02734 // setters:
02735 Q_SLOT void setScaleType(QCPAxis::ScaleType type);
02736 Q_SLOT void setRange(const QCPRange &range);
02737 void setRange(double lower, double upper);
02738 void setRange(double position, double size, Qt::AlignmentFlag alignment);
02739 void setRangeLower(double lower);
02740 void setRangeUpper(double upper);
02741 void setRangeReversed(bool reversed);
02742 void setTicker(QSharedPointer<QCPAxisTicker> ticker);
02743 void setTicks(bool show);
02744 void setTickLabels(bool show);
02745 void setTickLabelPadding(int padding);
02746 void setTickLabelFont(const QFont &font);
02747 void setTickLabelColor(const QColor &color);
02748 void setTickLabelRotation(double degrees);
02749 void setTickLabelSide(LabelSide side);
02750 void setNumberFormat(const QString &formatCode);
02751 void setNumberPrecision(int precision);
02752 void setTickLength(int inside, int outside = 0);
02753 void setTickLengthIn(int inside);
02754 void setTickLengthOut(int outside);
02755 void setSubTicks(bool show);
02756 void setSubTickLength(int inside, int outside = 0);
02757 void setSubTickLengthIn(int inside);
02758 void setSubTickLengthOut(int outside);
02759 void setBasePen(const QPen &pen);
02760 void setTickPen(const QPen &pen);

```



```

02761 void setSubTickPen(const QPen &pen);
02762 void setLabelFont(const QFont &font);
02763 void setLabelColor(const QColor &color);
02764 void setLabel(const QString &str);
02765 void setLabelPadding(int padding);
02766 void setPadding(int padding);
02767 void setOffset(int offset);
02768 void setSelectedTickLabelFont(const QFont &font);
02769 void setSelectedLabelFont(const QFont &font);
02770 void setSelectedTickLabelColor(const QColor &color);
02771 void setSelectedLabelColor(const QColor &color);
02772 void setSelectedBasePen(const QPen &pen);
02773 void setSelectedTickPen(const QPen &pen);
02774 void setSelectedSubTickPen(const QPen &pen);
02775 Q_SLOT void setSelectableParts(const QCPAxis::SelectableParts &selectableParts);
02776 Q_SLOT void setSelectedParts(const QCPAxis::SelectableParts &selectedParts);
02777 void setLowerEnding(const QCPLLineEnding &ending);
02778 void setUpperEnding(const QCPLLineEnding &ending);
02779
02780 // reimplemented virtual methods:
02781 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
02782
02783 // non-property methods:
02784
02785 Qt::Orientation orientation() const {
02786 return mOrientation;
02787 }
02788
02789 int pixelOrientation() const {
02790 return rangeReversed() != (orientation() == Qt::Vertical) ? -1 : 1;
02791 }
02792 void moveRange(double diff);
02793 void scaleRange(double factor);
02794 void scaleRange(double factor, double center);
02795 void setScaleRatio(const QCPAxis *otherAxis, double ratio = 1.0);
02796 void rescale(bool onlyVisiblePlottables = false);
02797 double pixelToCoord(double value) const;
02798 double coordToPixel(double value) const;
02799 SelectablePart getPartAt(const QPointF &pos) const;
02800 QList<QCPAbstractPlottable*> plottables() const;
02801 QList<QCPGraph*> graphs() const;
02802 QList<QCPAbstractItem*> items() const;
02803
02804 static AxisType marginSideToAxisType(QCP::MarginSide side);
02805
02806 static Qt::Orientation orientation(AxisType type) {
02807 return type == atBottom || type == atTop ? Qt::Horizontal : Qt::Vertical;
02808 }
02809 static AxisType opposite(AxisType type);
02810
02811 signals:
02812 void rangeChanged(const QCPRange &newRange);
02813 void rangeChanged(const QCPRange &newRange, const QCPRange &oldRange);
02814 void scaleTypeChanged(QCPAxis::ScaleType scaleType);
02815 void selectionChanged(const QCPAxis::SelectableParts &parts);
02816 void selectableChanged(const QCPAxis::SelectableParts &parts);
02817
02818 protected:
02819 // property members:
02820 // axis base:
02821 AxisType mAxisType;
02822 QCPAxisRect *mAxisRect;
02823 //int mOffset; // in QCPAxisPainter
02824 int mPadding;
02825 Qt::Orientation mOrientation;
02826 SelectableParts mSelectableParts, mSelectedParts;
02827 QPen mBasePen, mSelectedBasePen;
02828 //QCPLineEnding mLowerEnding, mUpperEnding; // in QCPAxisPainter
02829 // axis label:
02830 //int mLabelPadding; // in QCPAxisPainter
02831 QString mLabel;
02832 QFont mLabelFont, mSelectedLabelFont;
02833 QColor mLabelColor, mSelectedLabelColor;
02834 // tick labels:
02835 //int mTickLabelPadding; // in QCPAxisPainter
02836 bool mTickLabels;
02837 //double mTickLabelRotation; // in QCPAxisPainter
02838 QFont mTickLabelFont, mSelectedTickLabelFont;
02839 QColor mTickLabelColor, mSelectedTickLabelColor;
02840 int mNumberPrecision;
02841 QLatin1Char mNumberFormatChar;
02842 bool mNumberBeautifulPowers;
02843 //bool mNumberMultiplyCross; // QCPAxisPainter
02844 // ticks and subticks:
02845 bool mTicks;
02846 bool mSubTicks;

```

```

02847 //int mTickLengthIn, mTickLengthOut, mSubTickLengthIn, mSubTickLengthOut; // QCPAxisPainter
02848 QPen mTickPen, mSelectedTickPen;
02849 QPen mSubTickPen, mSelectedSubTickPen;
02850 // scale and range:
02851 QCPRange mRange;
02852 bool mRangeReversed;
02853 ScaleType mScaleType;
02854
02855 // non-property members:
02856 QCPGrid *mGrid;
02857 QCPAxisPainterPrivate *mAxisPainter;
02858 QSharedPointer<QCPAxisTicker> mTicker;
02859 QVector<double> mTickVector;
02860 QVector<QString> mTickVectorLabels;
02861 QVector<double> mSubTickVector;
02862 bool mCachedMarginValid;
02863 int mCachedMargin;
02864 bool mDragging;
02865 QCPRange mDragStartRange;
02866 QCP::AntialiasedElements mAADragBackup, mNotAADragBackup;
02867
02868 // introduced virtual methods:
02869 virtual int calculateMargin();
02870
02871 // reimplemented virtual methods:
02872 virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
02873 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
02874 virtual QCP::Interaction selectionCategory() const Q_DECL_OVERRIDE;
02875 // events:
02876 virtual void selectEvent(QMouseEvent *event, bool additive, const QVariant &details, bool
*selectionStateChanged) Q_DECL_OVERRIDE;
02877 virtual void deselectEvent(bool *selectionStateChanged) Q_DECL_OVERRIDE;
02878 // mouse events:
02879 virtual void mousePressEvent(QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE;
02880 virtual void mouseMoveEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
02881 virtual void mouseReleaseEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
02882 virtual void wheelEvent(QWheelEvent *event) Q_DECL_OVERRIDE;
02883
02884 // non-virtual methods:
02885 void setupTickVectors();
02886 QPen getBasePen() const;
02887 QPen getTickPen() const;
02888 QPen getSubTickPen() const;
02889 QFont getTickLabelFont() const;
02890 QFont getLabelFont() const;
02891 QColor getTickLabelColor() const;
02892 QColor getLabelColor() const;
02893
02894 private:
02895 Q_DISABLE_COPY(QCPAxis)
02896
02897 friend class QCustomPlot;
02898 friend class QCPGrid;
02899 friend class QCPAxisRect;
02900 };
02901
02902 Q_DECLARE_OPERATORS_FOR_FLAGS(QCPAxis::SelectableParts)
02903 Q_DECLARE_OPERATORS_FOR_FLAGS(QCPAxis::AxisTypes)
02904 Q_DECLARE_METATYPE(QCPAxis::AxisType)
02905 Q_DECLARE_METATYPE(QCPAxis::LabelSide)
02906 Q_DECLARE_METATYPE(QCPAxis::ScaleType)
02907 Q_DECLARE_METATYPE(QCPAxis::SelectablePart)
02908
02909
02910 class QCPAxisPainterPrivate {
02911 public:
02912 explicit QCPAxisPainterPrivate(QCustomPlot *parentPlot);
02913 virtual ~QCPAxisPainterPrivate();
02914
02915 virtual void draw(QCPPainter *painter);
02916 virtual int size();
02917 void clearCache();
02918
02919 QRect axisSelectionBox() const {
02920 return mAxisSelectionBox;
02921 }
02922
02923 QRect tickLabelsSelectionBox() const {
02924 return mTickLabelsSelectionBox;
02925 }
02926
02927 QRect labelSelectionBox() const {
02928 return mLabelSelectionBox;
02929 }
02930
02931 // public property members:
02932 QCPAxis::AxisType type;

```

```

02933 QPen basePen;
02934 QCPLLineEnding lowerEnding, upperEnding; // directly accessed by QCPAxis setters/getters
02935 int labelPadding; // directly accessed by QCPAxis setters/getters
02936 QFont labelFont;
02937 QColor labelColor;
02938 QString label;
02939 int tickLabelPadding; // directly accessed by QCPAxis setters/getters
02940 double tickLabelRotation; // directly accessed by QCPAxis setters/getters
02941 QCPAxis::LabelSide tickLabelSide; // directly accessed by QCPAxis setters/getters
02942 bool substituteExponent;
02943 bool numberMultiplyCross; // directly accessed by QCPAxis setters/getters
02944 int tickLengthIn, tickLengthOut, subTickLengthIn, subTickLengthOut; // directly accessed by
 QCPAxis setters/getters
02945 QPen tickPen, subTickPen;
02946 QFont tickLabelFont;
02947 QColor tickLabelColor;
02948 QRect axisRect, viewportRect;
02949 int offset; // directly accessed by QCPAxis setters/getters
02950 bool abbreviateDecimalPowers;
02951 bool reversedEndings;
02952
02953 QVector<double> subTickPositions;
02954 QVector<double> tickPositions;
02955 QVector<QString> tickLabels;
02956
02957 protected:
02958
02959 struct CachedLabel {
02960 QPointF offset;
02961 QPixmap pixmap;
02962 };
02963
02964 struct TickLabelData {
02965 QString basePart, expPart, suffixPart;
02966 QRect baseBounds, expBounds, suffixBounds, totalBounds, rotatedTotalBounds;
02967 QFont baseFont, expFont;
02968 };
02969
02970 QCustomPlot *mParentPlot;
02971 QByteArray mLabelParameterHash; // to determine whether mLabelCache needs to be cleared due to
 changed parameters
02972 QCache<QString, CachedLabel> mLabelCache;
02973 QRect mAxisSelectionBox, mTickLabelsSelectionBox, mLabelSelectionBox;
02974
02975 virtual QByteArray generateLabelParameterHash() const;
02976
02977 virtual void placeTickLabel(QCPPainter *painter, double position, int distanceToAxis, const
 QString &text, QSize *tickLabelsSize);
02978 virtual void drawTickLabel(QCPPainter *painter, double x, double y, const TickLabelData
 &labelData) const;
02979 virtual TickLabelData getTickLabelData(const QFont &font, const QString &text) const;
02980 virtual QPointF getTickLabelDrawOffset(const TickLabelData &labelData) const;
02981 virtual void getMaxTickLabelSize(const QFont &font, const QString &text, QSize *tickLabelsSize)
 const;
02982
02983 };
02984
02985 /* end of 'src/axis/axis.h' */
02986
02987 /* including file 'src/scatterstyle.h' */
02988 /* modified 2021-03-29T02:30:44, size 7275 */
02989
02990 class QCP_LIB_DECL QCPScatterStyle {
02991
02992 Q_GADGET
02993 public:
02994 enum ScatterProperty {
02995 spNone = 0x00
02996 , spPen = 0x01
02997 , spBrush = 0x02
02998 , spSize = 0x04
02999 , spShape = 0x08
03000 , spAll = 0xFF
03001 };
03002
03003 Q_ENUMS(ScatterProperty)
03004 Q_FLAGS(ScatterProperties)
03005 Q_DECLARE_FLAGS(ScatterProperties, ScatterProperty)
03006
03007 enum ScatterShape {
03008 ssNone
03009 , ssDot
03010 , ssCross
03011 , ssPlus
03012 , ssCircle
03013 , ssDisc
03014 };
03015
03016 Q_ENUMS(ScatterShape)

```

```

03029 , ssSquare
03030 , ssDiamond
03031 , ssStar
03032 , ssTriangle
03033 , ssTriangleInverted
03034 , ssCrossSquare
03035 , ssPlusSquare
03036 , ssCrossCircle
03037 , ssPlusCircle
03038 , ssPeace
03039 , ssPixmap
03040 , ssCustom
03041 };
03042 Q_ENUMS(ScatterShape)
03043
03044 QCPScatterStyle();
03045 QCPScatterStyle(ScatterShape shape, double size = 6);
03046 QCPScatterStyle(ScatterShape shape, const QColor &color, double size);
03047 QCPScatterStyle(ScatterShape shape, const QColor &color, const QColor &fill, double size);
03048 QCPScatterStyle(ScatterShape shape, const QPen &pen, const QBrush &brush, double size);
03049 QCPScatterStyle(const QPixmap &pixmap);
03050 QCPScatterStyle(const QPainterPath &customPath, const QPen &pen, const QBrush &brush =
Qt::NoBrush, double size = 6);
03051
03052 // getters:
03053
03054 double size() const {
03055 return mSize;
03056 }
03057
03058 ScatterShape shape() const {
03059 return mShape;
03060 }
03061
03062 QPen pen() const {
03063 return mPen;
03064 }
03065
03066 QBrush brush() const {
03067 return mBrush;
03068 }
03069
03070 QPixmap pixmap() const {
03071 return mPixmap;
03072 }
03073
03074 QPainterPath customPath() const {
03075 return mCustomPath;
03076 }
03077
03078 // setters:
03079 void setFromOther(const QCPScatterStyle &other, ScatterProperties properties);
03080 void setSize(double size);
03081 void setShape(ScatterShape shape);
03082 void setPen(const QPen &pen);
03083 void setBrush(const QBrush &brush);
03084 void setPixmap(const QPixmap &pixmap);
03085 void setCustomPath(const QPainterPath &customPath);
03086
03087 // non-property methods:
03088
03089 bool isNone() const {
03090 return mShape == ssNone;
03091 }
03092
03093 bool isPenDefined() const {
03094 return mPenDefined;
03095 }
03096 void undefinePen();
03097 void applyTo(QCPPainter *painter, const QPen &defaultPen) const;
03098 void drawShape(QCPPainter *painter, const QPointF &pos) const;
03099 void drawShape(QCPPainter *painter, double x, double y) const;
03100
03101 protected:
03102 // property members:
03103 double mSize;
03104 ScatterShape mShape;
03105 QPen mPen;
03106 QBrush mBrush;
03107 QPixmap mPixmap;
03108 QPainterPath mCustomPath;
03109
03110 // non-property members:
03111 bool mPenDefined;
03112 };
03113 Q_DECLARE_TYPEINFO(QCPScatterStyle, Q_MOVABLE_TYPE);
03114

```

```

03115 Q_DECLARE_OPERATORS_FOR_FLAGS(QCPScatterStyle::ScatterProperties)
03116 Q_DECLARE_METATYPE(QCPScatterStyle::ScatterProperty)
03117 Q_DECLARE_METATYPE(QCPScatterStyle::ScatterShape)
03118
03119 /* end of 'src/scatterstyle.h' */
03120
03121
03122 /* including file 'src/datacontainer.h' */
03123 /* modified 2021-03-29T02:30:44, size 34070 */
03124
03125
03130 template <class DataType>
03131 inline bool qcpLessThanSortKey(const DataType &a, const DataType &b) {
03132 return a.sortKey() < b.sortKey();
03133 }
03134
03135 template <class DataType>
03136 class QCPDataContainer // no QCP_LIB_DECL, template class ends up in header (cpp included below)
03137 {
03138 public:
03139 typedef typename QVector<DataType>::const_iterator const_iterator;
03140 typedef typename QVector<DataType>::iterator iterator;
03141
03142 QCPDataContainer();
03143
03144 // getters:
03145
03146 int size() const {
03147 return mData.size() - mPreallocSize;
03148 }
03149
03150 bool isEmpty() const {
03151 return size() == 0;
03152 }
03153
03154 bool autoSqueeze() const {
03155 return mAutoSqueeze;
03156 }
03157
03158 // setters:
03159 void setAutoSqueeze(bool enabled);
03160
03161 // non-virtual methods:
03162 void set(const QCPDataContainer<DataType> &data);
03163 void set(const QVector<DataType> &data, bool alreadySorted = false);
03164 void add(const QCPDataContainer<DataType> &data);
03165 void add(const QVector<DataType> &data, bool alreadySorted = false);
03166 void add(const DataType &data);
03167 void removeBefore(double sortKey);
03168 void removeAfter(double sortKey);
03169 void remove(double sortKeyFrom, double sortKeyTo);
03170 void remove(double sortKey);
03171 void clear();
03172 void sort();
03173 void squeeze(bool preAllocation = true, bool postAllocation = true);
03174
03175 const_iterator constBegin() const {
03176 return mData.constBegin() + mPreallocSize;
03177 }
03178
03179 const_iterator constEnd() const {
03180 return mData.constEnd();
03181 }
03182
03183 iterator begin() {
03184 return mData.begin() + mPreallocSize;
03185 }
03186
03187 iterator end() {
03188 return mData.end();
03189 }
03190
03191 const_iterator findBegin(double sortKey, bool expandedRange = true) const;
03192 const_iterator findEnd(double sortKey, bool expandedRange = true) const;
03193
03194 const_iterator at(int index) const {
03195 return constBegin() + qBound(0, index, size());
03196 }
03197 QCPRange keyRange(bool &foundRange, QCP::SignDomain signDomain = QCP::sdBoth);
03198 QCPRange valueRange(bool &foundRange, QCP::SignDomain signDomain = QCP::sdBoth, const QCPRange
&inKeyRange = QCPRange());
03199
03200 QCPDataRange dataRange() const {
03201 return QCPDataRange(0, size());
03202 }
03203 void limitIteratorsToDataRange(const_iterator &begin, const_iterator &end, const QCPDataRange
&dataRange) const;

```

```

03204 protected:
03205 // property members:
03206 bool mAutoSqueeze;
03207
03208 // non-property members:
03209 QVector<DataType> mData;
03210 int mPreallocSize;
03211 int mPreallocIteration;
03212
03213 // non-virtual methods:
03214 void preallocateGrow(int minimumPreallocSize);
03215 void performAutoSqueeze();
03216 };
03217
03218
03219
03220 // include implementation in header since it is a class template:
03221
03222
03223 /* start documentation of inline functions */
03224
03225 /* end documentation of inline functions */
03226
03227 template <class DataType>
03228 QCPDataContainer<DataType>::QCPDataContainer() :
03229 mAutoSqueeze(true),
03230 mPreallocSize(0),
03231 mPreallocIteration(0) {
03232 }
03233
03234 template <class DataType>
03235 void QCPDataContainer<DataType>::setAutoSqueeze(bool enabled) {
03236 if (mAutoSqueeze != enabled) {
03237 mAutoSqueeze = enabled;
03238 if (mAutoSqueeze)
03239 performAutoSqueeze();
03240 }
03241 }
03242
03243 template <class DataType>
03244 void QCPDataContainer<DataType>::set(const QCPDataContainer<DataType> &data) {
03245 clear();
03246 add(data);
03247 }
03248
03249 template <class DataType>
03250 void QCPDataContainer<DataType>::set(const QVector<DataType> &data, bool alreadySorted) {
03251 mData = data;
03252 mPreallocSize = 0;
03253 mPreallocIteration = 0;
03254 if (!alreadySorted)
03255 sort();
03256 }
03257
03258 template <class DataType>
03259 void QCPDataContainer<DataType>::add(const QCPDataContainer<DataType> &data) {
03260 if (data.isEmpty())
03261 return;
03262
03263 const int n = data.size();
03264 const int oldSize = size();
03265
03266 if (oldSize > 0 && !qcpLessThanSortKey<DataType>(*constBegin(), *(data.constEnd() - 1))) //
03267 prepend if new data keys are all smaller than or equal to existing ones
03268 {
03269 if (mPreallocSize < n)
03270 preallocateGrow(n);
03271 mPreallocSize -= n;
03272 std::copy(data.constBegin(), data.constEnd(), begin());
03273 } else // don't need to prepend, so append and merge if necessary
03274 {
03275 mData.resize(mData.size() + n);
03276 std::copy(data.constBegin(), data.constEnd(), end() - n);
03277 if (oldSize > 0 && !qcpLessThanSortKey<DataType>*(constEnd() - n - 1), *(constEnd() - n))) //
03278 if appended range keys aren't all greater than existing ones, merge the two partitions
03279 std::inplace_merge(begin(), end() - n, end(), qcpLessThanSortKey<DataType>);
03280 }
03281 }
03282
03283 template <class DataType>
03284 void QCPDataContainer<DataType>::add(const QVector<DataType> &data, bool alreadySorted) {
03285 if (data.isEmpty())
03286 return;
03287 if (isEmpty()) {
03288 set(data, alreadySorted);
03289 return;
03290 }
03291 }

```

```

03454 const int n = data.size();
03455 const int oldSize = size();
03456
03457 if (alreadySorted && oldSize > 0 && !qcpLessThanSortKey<DataType>(*constBegin(), *(data.constEnd()
- 1))) // prepend if new data is sorted and keys are all smaller than or equal to existing ones
03458 {
03459 if (mPreallocSize < n)
03460 preallocateGrow(n);
03461 mPreallocSize -= n;
03462 std::copy(data.constBegin(), data.constEnd(), begin());
03463 } else // don't need to prepend, so append and then sort and merge if necessary
03464 {
03465 mData.resize(mData.size() + n);
03466 std::copy(data.constBegin(), data.constEnd(), end() - n);
03467 if (!alreadySorted) // sort appended subrange if it wasn't already sorted
03468 std::sort(end() - n, end(), qcpLessThanSortKey<DataType>);
03469 if (oldSize > 0 && !qcpLessThanSortKey<DataType>(*(constEnd() - n - 1), *(constEnd() - n))) //
if appended range keys aren't all greater than existing ones, merge the two partitions
03470 std::inplace_merge(begin(), end() - n, end(), qcpLessThanSortKey<DataType>);
03471 }
03472 }
03473
03480 template <class DataType>
03481 void QCPDataContainer<DataType>::add(const DataType &data) {
03482 if (isEmpty() || !qcpLessThanSortKey<DataType>(data, *(constEnd() - 1))) // quickly handle appends
if new data key is greater or equal to existing ones
03483 {
03484 mData.append(data);
03485 } else if (qcpLessThanSortKey<DataType>(data, *constBegin())) // quickly handle prepends using
preallocated space
03486 {
03487 if (mPreallocSize < 1)
03488 preallocateGrow(1);
03489 --mPreallocSize;
03490 *begin() = data;
03491 } else // handle inserts, maintaining sorted keys
03492 {
03493 QCPDataContainer<DataType>::iterator insertionPoint = std::lower_bound(begin(), end(), data,
qcpLessThanSortKey<DataType>);
03494 mData.insert(insertionPoint, data);
03495 }
03496 }
03497
03503 template <class DataType>
03504 void QCPDataContainer<DataType>::removeBefore(double sortKey) {
03505 QCPDataContainer<DataType>::iterator it = begin();
03506 QCPDataContainer<DataType>::iterator itEnd = std::lower_bound(begin(), end(),
DataType::fromSortKey(sortKey), qcpLessThanSortKey<DataType>);
03507 mPreallocSize += int(itEnd - it); // don't actually delete, just add it to the preallocated block
(if it gets too large, squeeze will take care of it)
03508 if (mAutoSqueeze)
03509 performAutoSqueeze();
03510 }
03511
03517 template <class DataType>
03518 void QCPDataContainer<DataType>::removeAfter(double sortKey) {
03519 QCPDataContainer<DataType>::iterator it = std::upper_bound(begin(), end(),
DataType::fromSortKey(sortKey), qcpLessThanSortKey<DataType>);
03520 QCPDataContainer<DataType>::iterator itEnd = end();
03521 mData.erase(it, itEnd); // typically adds it to the postallocated block
03522 if (mAutoSqueeze)
03523 performAutoSqueeze();
03524 }
03525
03533 template <class DataType>
03534 void QCPDataContainer<DataType>::remove(double sortKeyFrom, double sortKeyTo) {
03535 if (sortKeyFrom >= sortKeyTo || isEmpty())
03536 return;
03537 QCPDataContainer<DataType>::iterator it = std::lower_bound(begin(), end(),
DataType::fromSortKey(sortKeyFrom), qcpLessThanSortKey<DataType>);
03538 QCPDataContainer<DataType>::iterator itEnd = std::upper_bound(it, end(),
DataType::fromSortKey(sortKeyTo), qcpLessThanSortKey<DataType>);
03539 mData.erase(it, itEnd);
03540 if (mAutoSqueeze)
03541 performAutoSqueeze();
03542 }
03543
03544 template <class DataType>
03545 void QCPDataContainer<DataType>::remove(double sortKey) {
03546 QCPDataContainer::iterator it = std::lower_bound(begin(), end(), DataType::fromSortKey(sortKey),
qcpLessThanSortKey<DataType>);
03547 if (it != end() && it->sortKey() == sortKey) {
03548 if (it == begin())
03549 ++mPreallocSize; // don't actually delete, just add it to the preallocated block (if it
gets too large, squeeze will take care of it)
03550 } else

```

```

03561 mData.erase(it);
03562 }
03563 if (mAutoSqueeze)
03564 performAutoSqueeze();
03565 }
03566
03572 template <class DataType>
03573 void QCPDataContainer<DataType>::clear() {
03574 mData.clear();
03575 mPreallocIteration = 0;
03576 mPreallocSize = 0;
03577 }
03578
03590 template <class DataType>
03591 void QCPDataContainer<DataType>::sort() {
03592 std::sort(begin(), end(), qcpLessThanSortKey<DataType>);
03593 }
03594
03605 template <class DataType>
03606 void QCPDataContainer<DataType>::squeeze(bool preAllocation, bool postAllocation) {
03607 if (preAllocation) {
03608 if (mPreallocSize > 0) {
03609 std::copy(begin(), end(), mData.begin());
03610 mData.resize(size());
03611 mPreallocSize = 0;
03612 }
03613 mPreallocIteration = 0;
03614 }
03615 if (postAllocation)
03616 mData.squeeze();
03617 }
03618
03634 template <class DataType>
03635 typename QCPDataContainer<DataType>::const_iterator QCPDataContainer<DataType>::findBegin(double
sortKey, bool expandedRange) const {
03636 if (isEmpty())
03637 return constEnd();
03638
03639 QCPDataContainer<DataType>::const_iterator it = std::lower_bound(constBegin(), constEnd(),
DataType::fromSortKey(sortKey), qcpLessThanSortKey<DataType>);
03640 if (expandedRange && it != constBegin()) // also covers it == constEnd case, and we know
--constEnd is valid because mData isn't empty
03641 --it;
03642 return it;
03643 }
03644
03660 template <class DataType>
03661 typename QCPDataContainer<DataType>::const_iterator QCPDataContainer<DataType>::findEnd(double
sortKey, bool expandedRange) const {
03662 if (isEmpty())
03663 return constEnd();
03664
03665 QCPDataContainer<DataType>::const_iterator it = std::upper_bound(constBegin(), constEnd(),
DataType::fromSortKey(sortKey), qcpLessThanSortKey<DataType>);
03666 if (expandedRange && it != constEnd())
03667 ++it;
03668 return it;
03669 }
03670
03686 template <class DataType>
03687 QCPRange QCPDataContainer<DataType>::keyRange(bool &foundRange, QCP::SignDomain signDomain) {
03688 if (isEmpty()) {
03689 foundRange = false;
03690 return QCPRange();
03691 }
03692 QCPRange range;
03693 bool haveLower = false;
03694 bool haveUpper = false;
03695 double current;
03696
03697 QCPDataContainer<DataType>::const_iterator it = constBegin();
03698 QCPDataContainer<DataType>::const_iterator itEnd = constEnd();
03699 if (signDomain == QCP::sdBoth) // range may be anywhere
03700 {
03701 if (DataType::sortKeyIsMainKey()) // if DataType is sorted by main key (e.g. QCPGraph, but not
QPCCurve), use faster algorithm by finding just first and last key with non-NaN value
03702 {
03703 while (it != itEnd) // find first non-nan going up from left
03704 {
03705 if (!qIsNaN(it->mainValue())) {
03706 range.lower = it->mainKey();
03707 haveLower = true;
03708 break;
03709 }
03710 ++it;
03711 }
03712 it = itEnd;

```



```

03713 while (it != constBegin()) // find first non-nan going down from right
03714 {
03715 --it;
03716 if (!qIsNaN(it->mainValue())) {
03717 range.upper = it->mainKey();
03718 haveUpper = true;
03719 break;
03720 }
03721 }
03722 } else // DataType is not sorted by main key, go through all data points and accordingly
expand range
03723 {
03724 while (it != itEnd) {
03725 if (!qIsNaN(it->mainValue())) {
03726 current = it->mainKey();
03727 if (current < range.lower || !haveLower) {
03728 range.lower = current;
03729 haveLower = true;
03730 }
03731 if (current > range.upper || !haveUpper) {
03732 range.upper = current;
03733 haveUpper = true;
03734 }
03735 }
03736 ++it;
03737 }
03738 }
03739 } else if (signDomain == QCP::sdNegative) // range may only be in the negative sign domain
03740 {
03741 while (it != itEnd) {
03742 if (!qIsNaN(it->mainValue())) {
03743 current = it->mainKey();
03744 if ((current < range.lower || !haveLower) && current < 0) {
03745 range.lower = current;
03746 haveLower = true;
03747 }
03748 if ((current > range.upper || !haveUpper) && current < 0) {
03749 range.upper = current;
03750 haveUpper = true;
03751 }
03752 }
03753 ++it;
03754 }
03755 } else if (signDomain == QCP::sdPositive) // range may only be in the positive sign domain
03756 {
03757 while (it != itEnd) {
03758 if (!qIsNaN(it->mainValue())) {
03759 current = it->mainKey();
03760 if ((current < range.lower || !haveLower) && current > 0) {
03761 range.lower = current;
03762 haveLower = true;
03763 }
03764 if ((current > range.upper || !haveUpper) && current > 0) {
03765 range.upper = current;
03766 haveUpper = true;
03767 }
03768 }
03769 ++it;
03770 }
03771 }
03772
03773 foundRange = haveLower && haveUpper;
03774 return range;
03775 }
03776
03793 template <class DataType>
03794 QCPRange QCPDataContainer<DataType>::valueRange(bool &foundRange, QCP::SignDomain signDomain, const
QCPRange &inKeyRange) {
03795 if (isEmpty()) {
03796 foundRange = false;
03797 return QCPRange();
03798 }
03799 QCPRange range;
03800 const bool restrictKeyRange = inKeyRange != QCPRange();
03801 bool haveLower = false;
03802 bool haveUpper = false;
03803 QCPRange current;
03804 QCPDataContainer<DataType>::const_iterator itBegin = constBegin();
03805 QCPDataContainer<DataType>::const_iterator itEnd = constEnd();
03806 if (DataType::sortKeyIsMainKey() && restrictKeyRange) {
03807 itBegin = findBegin(inKeyRange.lower, false);
03808 itEnd = findEnd(inKeyRange.upper, false);
03809 }
03810 if (signDomain == QCP::sdBoth) // range may be anywhere
03811 {
03812 for (QCPDataContainer<DataType>::const_iterator it = itBegin; it != itEnd; ++it) {
03813 if (restrictKeyRange && (it->mainKey() < inKeyRange.lower || it->mainKey() >

```

```

 inKeyRange.upper))
03814 continue;
03815 current = it->valueRange();
03816 if ((current.lower < range.lower || !haveLower) && !qIsNaN(current.lower)) {
03817 range.lower = current.lower;
03818 haveLower = true;
03819 }
03820 if ((current.upper > range.upper || !haveUpper) && !qIsNaN(current.upper)) {
03821 range.upper = current.upper;
03822 haveUpper = true;
03823 }
03824 }
03825 } else if (signDomain == QCP::sdNegative) // range may only be in the negative sign domain
03826 {
03827 for (QCPDataContainer<DataType>::const_iterator it = itBegin; it != itEnd; ++it) {
03828 if (restrictKeyRange && (it->mainKey() < inKeyRange.lower || it->mainKey() >
inKeyRange.upper))
03829 continue;
03830 current = it->valueRange();
03831 if ((current.lower < range.lower || !haveLower) && current.lower < 0 &&
!qIsNaN(current.lower)) {
03832 range.lower = current.lower;
03833 haveLower = true;
03834 }
03835 if ((current.upper > range.upper || !haveUpper) && current.upper < 0 &&
!qIsNaN(current.upper)) {
03836 range.upper = current.upper;
03837 haveUpper = true;
03838 }
03839 }
03840 } else if (signDomain == QCP::sdPositive) // range may only be in the positive sign domain
03841 {
03842 for (QCPDataContainer<DataType>::const_iterator it = itBegin; it != itEnd; ++it) {
03843 if (restrictKeyRange && (it->mainKey() < inKeyRange.lower || it->mainKey() >
inKeyRange.upper))
03844 continue;
03845 current = it->valueRange();
03846 if ((current.lower < range.lower || !haveLower) && current.lower > 0 &&
!qIsNaN(current.lower)) {
03847 range.lower = current.lower;
03848 haveLower = true;
03849 }
03850 if ((current.upper > range.upper || !haveUpper) && current.upper > 0 &&
!qIsNaN(current.upper)) {
03851 range.upper = current.upper;
03852 haveUpper = true;
03853 }
03854 }
03855 }
03856 foundRange = haveLower && haveUpper;
03857 return range;
03858 }
03859 }
03860
03861 template <class DataType>
03870 void QCPDataContainer<DataType>::limitIteratorsToDataRange(const_iterator &begin, const_iterator &end,
const QCPDataRange &dataRange) const {
03871 QCPDataRange iteratorRange(int(begin - constBegin()), int(end - constBegin()));
03872 iteratorRange = iteratorRange.bounded(dataRange.bounded(this->dataRange()));
03873 begin = constBegin() + iteratorRange.begin();
03874 end = constBegin() + iteratorRange.end();
03875 }
03876
03886 template <class DataType>
03887 void QCPDataContainer<DataType>::preallocateGrow(int minimumPreallocSize) {
03888 if (minimumPreallocSize <= mPreallocSize)
03889 return;
03890
03891 int newPreallocSize = minimumPreallocSize;
03892 newPreallocSize += (1u << qBound(4, mPreallocIteration + 4, 15)) - 12; // do 4 up to 32768-12
preallocation, doubling in each intermediate iteration
03893 ++mPreallocIteration;
03894
03895 int sizeDifference = newPreallocSize - mPreallocSize;
03896 mData.resize(mData.size() + sizeDifference);
03897 std::copy_backward(mData.begin() + mPreallocSize, mData.end() - sizeDifference, mData.end());
03898 mPreallocSize = newPreallocSize;
03899 }
03900
03915 template <class DataType>
03916 void QCPDataContainer<DataType>::performAutoSqueeze() {
03917 const int totalAlloc = mData.capacity();
03918 const int postAllocSize = totalAlloc - mData.size();
03919 const int usedSize = size();
03920 bool shrinkPostAllocation = false;
03921 bool shrinkPreAllocation = false;
03922 if (totalAlloc > 650000) // if allocation is larger, shrink earlier with respect to total used

```

```

 size
03923 {
03924 shrinkPostAllocation = postAllocSize > usedSize * 1.5; // QVector grow strategy is 2^n for
static data. Watch out not to oscillate!
03925 shrinkPreAllocation = mPreallocSize * 10 > usedSize;
03926 } else if (totalAlloc > 1000) // below 10 MiB raw data be generous with preallocated memory, below
1k points don't even bother
03927 {
03928 shrinkPostAllocation = postAllocSize > usedSize * 5;
03929 shrinkPreAllocation = mPreallocSize > usedSize * 1.5; // preallocation can grow into
postallocation, so can be smaller
03930 }
03931
03932 if (shrinkPreAllocation || shrinkPostAllocation)
03933 squeeze(shrinkPreAllocation, shrinkPostAllocation);
03934 }
03935
03936
03937 /* end of 'src/datacontainer.h' */
03938
03939
03940 /* including file 'src/plottable.h' */
03941
03942 /* modified 2021-03-29T02:30:44, size 8461 */
03943
03944 class QCP_LIB_DECL QCPSelectionDecorator {
03945 Q_GADGET
03946 public:
03947 QCPSelectionDecorator();
03948 virtual ~QCPSelectionDecorator();
03949
03950 // getters:
03951
03952 QPen pen() const {
03953 return mPen;
03954 }
03955
03956 QBrush brush() const {
03957 return mBrush;
03958 }
03959
03960 QCPScatterStyle scatterStyle() const {
03961 return mScatterStyle;
03962 }
03963
03964 QCPScatterStyle::ScatterProperties usedScatterProperties() const {
03965 return mUsedScatterProperties;
03966 }
03967
03968 // setters:
03969 void setPen(const QPen &pen);
03970 void setBrush(const QBrush &brush);
03971 void setScatterStyle(const QCPScatterStyle &scatterStyle, QCPScatterStyle::ScatterProperties
usedProperties = QCPScatterStyle::spPen);
03972 void setUsedScatterProperties(const QCPScatterStyle::ScatterProperties &properties);
03973
03974 // non-virtual methods:
03975 void applyPen(QCPPainter *painter) const;
03976 void applyBrush(QCPPainter *painter) const;
03977 QCPScatterStyle getFinalScatterStyle(const QCPScatterStyle &unselectedStyle) const;
03978
03979 // introduced virtual methods:
03980 virtual void copyFrom(const QCPSelectionDecorator *other);
03981 virtual void drawDecoration(QCPPainter *painter, QCPDataSelection selection);
03982
03983 protected:
03984 // property members:
03985 QPen mPen;
03986 QBrush mBrush;
03987 QCPScatterStyle mScatterStyle;
03988 QCPScatterStyle::ScatterProperties mUsedScatterProperties;
03989 // non-property members:
03990 QCPAbstractPlottable *mPlottable;
03991
03992 // introduced virtual methods:
03993 virtual bool registerWithPlottable(QCPAbstractPlottable *plottable);
03994
03995 private:
03996 Q_DISABLE_COPY(QCPSelectionDecorator)
03997 friend class QCPAbstractPlottable;
03998 };
03999
04000 Q_DECLARE_METATYPE(QCPSelectionDecorator*)
04001
04002
04003 class QCP_LIB_DECL QCPAbstractPlottable : public QCPLayerable {
04004 Q_OBJECT

```

```

04006 Q_PROPERTY(QString name READ name WRITE setName)
04007 Q_PROPERTY(bool antialiasedFill READ antialiasedFill WRITE setAntialiasedFill)
04008 Q_PROPERTY(bool antialiasedScatters READ antialiasedScatters WRITE setAntialiasedScatters)
04009 Q_PROPERTY(QPen pen READ pen WRITE setPen)
04010 Q_PROPERTY(QBrush brush READ brush WRITE setBrush)
04011 Q_PROPERTY(QCPAxis* keyAxis READ keyAxis WRITE setKeyAxis)
04012 Q_PROPERTY(QCPAxis* valueAxis READ valueAxis WRITE setValueAxis)
04013 Q_PROPERTY(QCP::SelectionType selectable READ selectable WRITE setSelectable NOTIFY
selectableChanged)
04014 Q_PROPERTY(QCPDataSelection selection READ selection WRITE setSelection NOTIFY selectionChanged)
04015 Q_PROPERTY(QCPSelectionDecorator* selectionDecorator READ selectionDecorator WRITE
setSelectionDecorator)
04017 public:
04018 QCPAbstractPlottable(QCPAxis *keyAxis, QCPAxis *valueAxis);
04019 virtual ~QCPAbstractPlottable() Q_DECL_OVERRIDE;
04020
04021 // getters:
04022
04023 QString name() const {
04024 return mName;
04025 }
04026
04027 bool antialiasedFill() const {
04028 return mAntialiasedFill;
04029 }
04030
04031 bool antialiasedScatters() const {
04032 return mAntialiasedScatters;
04033 }
04034
04035 QPen pen() const {
04036 return mPen;
04037 }
04038
04039 QBrush brush() const {
04040 return mBrush;
04041 }
04042
04043 QCPAxis *keyAxis() const {
04044 return mKeyAxis.data();
04045 }
04046
04047 QCPAxis *valueAxis() const {
04048 return mValueAxis.data();
04049 }
04050
04051 QCP::SelectionType selectable() const {
04052 return mSelectable;
04053 }
04054
04055 bool selected() const {
04056 return !mSelection.isEmpty();
04057 }
04058
04059 QCPDataSelection selection() const {
04060 return mSelection;
04061 }
04062
04063 QCPSelectionDecorator *selectionDecorator() const {
04064 return mSelectionDecorator;
04065 }
04066
04067 // setters:
04068 void setName(const QString &name);
04069 void setAntialiasedFill(bool enabled);
04070 void setAntialiasedScatters(bool enabled);
04071 void setPen(const QPen &pen);
04072 void setBrush(const QBrush &brush);
04073 void setKeyAxis(QCPAxis *axis);
04074 void setValueAxis(QCPAxis *axis);
04075 Q_SLOT void setSelectable(QCP::SelectionType selectable);
04076 Q_SLOT void setSelection(QCPDataSelection selection);
04077 void setSelectionDecorator(QCPSelectionDecorator *decorator);
04078
04079 // introduced virtual methods:
04080 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE = 0; // actually introduced in QCPLayerable as non-pure, but we want to force
reimplementation for plottables
04081
04082 virtual QCPPlottableInterface1D *interface1D() {
04083 return nullptr;
04084 }
04085 virtual QCPRange getKeyRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth) const =
0;
04086 virtual QCPRange getValueRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth, const
QCPRange &inKeyRange = QCPRange()) const = 0;
04087

```

```

04088 // non-property methods:
04089 void coordsToPixels(double key, double value, double &x, double &y) const;
04090 const QPointF coordsToPixels(double key, double value) const;
04091 void pixelsToCoords(double x, double y, double &key, double &value) const;
04092 void pixelsToCoords(const QPointF &pixelPos, double &key, double &value) const;
04093 void rescaleAxes(bool onlyEnlarge = false) const;
04094 void rescaleKeyAxis(bool onlyEnlarge = false) const;
04095 void rescaleValueAxis(bool onlyEnlarge = false, bool inKeyRange = false) const;
04096 bool addToLegend(QCPLegend *legend);
04097 bool addToLegend();
04098 bool removeFromLegend(QCPLegend *legend) const;
04099 bool removeFromLegend() const;
04100
04101 signals:
04102 void selectionChanged(bool selected);
04103 void selectionChanged(const QCPDataSelection &selection);
04104 void selectableChanged(QCP::SelectionType selectable);
04105
04106 protected:
04107 // property members:
04108 QString mName;
04109 bool mAntialiasedFill, mAntialiasedScatters;
04110 QPen mPen;
04111 QBrush mBrush;
04112 QPointer<QCPAxis> mKeyAxis, mValueAxis;
04113 QCP::SelectionType mSelectable;
04114 QCPDataSelection mSelection;
04115 QCPSelectionDecorator *mSelectionDecorator;
04116
04117 // reimplemented virtual methods:
04118 virtual QRect clipRect() const Q_DECL_OVERRIDE;
04119 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE = 0;
04120 virtual QCP::Interaction selectionCategory() const Q_DECL_OVERRIDE;
04121 void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
04122 // events:
04123 virtual void selectEvent(QMouseEvent *event, bool additive, const QVariant &details, bool
*selectionStateChanged) Q_DECL_OVERRIDE;
04124 virtual void deselectEvent(bool *selectionStateChanged) Q_DECL_OVERRIDE;
04125
04126 // introduced virtual methods:
04127 virtual void drawLegendIcon(QCPPainter *painter, const QRectF &rect) const = 0;
04128
04129 // non-virtual methods:
04130 void applyFillAntialiasingHint(QCPPainter *painter) const;
04131 void applyScattersAntialiasingHint(QCPPainter *painter) const;
04132
04133 private:
04134 Q_DISABLE_COPY(QCPAbstractPlottable)
04135
04136 friend class QCustomPlot;
04137 friend class QCPAxis;
04138 friend class QCPPlottableLegendItem;
04139 };
04140
04141
04142 /* end of 'src/plottable.h' */
04143
04144
04145 /* including file 'src/item.h' */
04146
04147 /* modified 2021-03-29T02:30:44, size 9425 */
04148
04149 class QCP_LIB_DECL QCPItemAnchor {
04150 Q_GADGET
04151 public:
04152 QCPItemAnchor(QCustomPlot *parentPlot, QCPAbstractItem *parentItem, const QString &name, int
anchorId = -1);
04153 virtual ~QCPItemAnchor();
04154
04155 // getters:
04156
04157 QString name() const {
04158 return mName;
04159 }
04160 virtual QPointF pixelPosition() const;
04161
04162 protected:
04163 // property members:
04164 QString mName;
04165
04166 // non-property members:
04167 QCustomPlot *mParentPlot;
04168 QCPAbstractItem *mParentItem;
04169 int mAnchorId;
04170 QSet<QCPItemPosition*> mChildrenX, mChildrenY;
04171
04172 // introduced virtual methods:

```

```

04173
04174 virtual QCPItemPosition *toQCPItemPosition() {
04175 return nullptr;
04176 }
04177
04178 // non-virtual methods:
04179 void addChildX(QCPItemPosition* pos); // called from pos when this anchor is set as parent
04180 void removeChildX(QCPItemPosition *pos); // called from pos when its parent anchor is reset or pos
04181 deleted
04182 void addChildY(QCPItemPosition* pos); // called from pos when this anchor is set as parent
04183 void removeChildY(QCPItemPosition *pos); // called from pos when its parent anchor is reset or pos
04184 deleted
04185
04186 private:
04187 Q_DISABLE_COPY(QCPItemAnchor)
04188 friend class QCPItemPosition;
04189 };
04190
04191 class QCP_LIB_DECL QCPItemPosition : public QCPItemAnchor {
04192 Q_GADGET
04193 public:
04194 enum PositionType {
04195 ptAbsolute
04196 , ptViewportRatio
04197 , ptAxisRectRatio
04198 , ptPlotCoords
04199 };
04200 Q_ENUMS(PositionType)
04201
04202 QCPItemPosition(QCustomPlot *parentPlot, QCPAbstractItem *parentItem, const QString &name);
04203 virtual ~QCPItemPosition() Q_DECL_OVERRIDE;
04204
04205 // getters:
04206
04207 PositionType type() const {
04208 return typeX();
04209 }
04210
04211 PositionType typeX() const {
04212 return mPositionTypeX;
04213 }
04214
04215 PositionType typeY() const {
04216 return mPositionTypeY;
04217 }
04218
04219 QCPItemAnchor *parentAnchor() const {
04220 return parentAnchorX();
04221 }
04222
04223 QCPItemAnchor *parentAnchorX() const {
04224 return mParentAnchorX;
04225 }
04226
04227 QCPItemAnchor *parentAnchorY() const {
04228 return mParentAnchorY;
04229 }
04230
04231 double key() const {
04232 return mKey;
04233 }
04234
04235 double value() const {
04236 return mValue;
04237 }
04238
04239 QPointF coords() const {
04240 return QPointF(mKey, mValue);
04241 }
04242
04243 QCPAxis *keyAxis() const {
04244 return mKeyAxis.data();
04245 }
04246
04247 QCPAxis *valueAxis() const {
04248 return mValueAxis.data();
04249 }
04250
04251 QCPAxisRect *axisRect() const;
04252 virtual QPointF pixelPosition() const Q_DECL_OVERRIDE;
04253
04254 // setters:
04255 void setType(PositionType type);
04256 void setTypeX(PositionType type);
04257 void setTypeY(PositionType type);
04258 bool setParentAnchor(QCPItemAnchor *parentAnchor, bool keepPixelPosition = false);

```

```

04268 bool setParentAnchorX(QCItemAnchor *parentAnchor, bool keepPixelPosition = false);
04269 bool setParentAnchorY(QCItemAnchor *parentAnchor, bool keepPixelPosition = false);
04270 void setCoords(double key, double value);
04271 void setCoords(const QPointF &pos);
04272 void setAxes(QCPAxis* keyAxis, QCPAxis* valueAxis);
04273 void setAxisRect(QCPAxisRect *axisRect);
04274 void setPixelPosition(const QPointF &pixelPosition);
04275
04276 protected:
04277 // property members:
04278 PositionType mPositionTypeX, mPositionTypeY;
04279 QPointer<QCPAxis> mKeyAxis, mValueAxis;
04280 QPointer<QCPAxisRect> mAxisRect;
04281 double mKey, mValue;
04282 QCItemAnchor *mParentAnchorX, *mParentAnchorY;
04283
04284 // reimplemented virtual methods:
04285
04286 virtual QCItemPosition *toQCItemPosition() Q_DECL_OVERRIDE {
04287 return this;
04288 }
04289
04290 private:
04291 Q_DISABLE_COPY(QCItemPosition)
04292
04293 };
04294
04295 Q_DECLARE_METATYPE(QCItemPosition::PositionType)
04296
04297 class QCP_LIB_DECL QCPAbstractItem : public QCPLayerable {
04298 Q_OBJECT
04299 Q_PROPERTY(bool clipToAxisRect READ clipToAxisRect WRITE setClipToAxisRect)
04300 Q_PROPERTY(QCPAxisRect* clipAxisRect READ clipAxisRect WRITE setClipAxisRect)
04301 Q_PROPERTY(bool selectable READ selectable WRITE setSelectable NOTIFY selectableChanged)
04302 Q_PROPERTY(bool selected READ selected WRITE setSelected NOTIFY selectionChanged)
04303 public:
04304 explicit QCPAbstractItem(QCustomPlot *parentPlot);
04305 virtual ~QCPAbstractItem() Q_DECL_OVERRIDE;
04306
04307 // getters:
04308
04309 bool clipToAxisRect() const {
04310 return mClipToAxisRect;
04311 }
04312 QCPAxisRect *clipAxisRect() const;
04313
04314 bool selectable() const {
04315 return mSelectable;
04316 }
04317
04318 bool selected() const {
04319 return mSelected;
04320 }
04321
04322 // setters:
04323 void setClipToAxisRect(bool clip);
04324 void setClipAxisRect(QCPAxisRect *rect);
04325 Q_SLOT void setSelectable(bool selectable);
04326 Q_SLOT void setSelected(bool selected);
04327
04328 // reimplemented virtual methods:
04329 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr) const Q_DECL_OVERRIDE = 0;
04330
04331 // non-virtual methods:
04332
04333 QList<QCItemPosition*> positions() const {
04334 return mPositions;
04335 }
04336
04337 QList<QCItemAnchor*> anchors() const {
04338 return mAnchors;
04339 }
04340
04341 QCItemPosition *position(const QString &name) const;
04342 QCItemAnchor *anchor(const QString &name) const;
04343 bool hasAnchor(const QString &name) const;
04344
04345 signals:
04346 void selectionChanged(bool selected);
04347 void selectableChanged(bool selectable);
04348
04349 protected:
04350 // property members:
04351 bool mClipToAxisRect;
04352 QPointer<QCPAxisRect> mClipAxisRect;
04353 QList<QCItemPosition*> mPositions;

```

```

04356 QList<QCPItemAnchor*> mAnchors;
04357 bool mSelectable, mSelected;
04358
04359 // reimplemented virtual methods:
04360 virtual QCP::Interaction selectionCategory() const Q_DECL_OVERRIDE;
04361 virtual QRect clipRect() const Q_DECL_OVERRIDE;
04362 virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
04363 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE = 0;
04364 // events:
04365 virtual void selectEvent(QMouseEvent *event, bool additive, const QVariant &details, bool
*selectionStateChanged) Q_DECL_OVERRIDE;
04366 virtual void deselectEvent(bool *selectionStateChanged) Q_DECL_OVERRIDE;
04367
04368 // introduced virtual methods:
04369 virtual QPointF anchorPixelPosition(int anchorId) const;
04370
04371 // non-virtual methods:
04372 double rectDistance(const QRectF &rect, const QPointF &pos, bool filledRect) const;
04373 QCPItemPosition *createPosition(const QString &name);
04374 QCPItemAnchor *createAnchor(const QString &name, int anchorId);
04375
04376 private:
04377 Q_DISABLE_COPY(QCPAbstractItem)
04378
04379 friend class QCustomPlot;
04380 friend class QCPItemAnchor;
04381 };
04382
04383 /* end of 'src/item.h' */
04384
04385
04386 /* including file 'src/core.h' */
04387
04388 /* modified 2021-03-29T02:30:44, size 19304 */
04389
04390 class QCP_LIB_DECL QCustomPlot : public QWidget {
04391
04392 Q_OBJECT
04393 Q_PROPERTY(QRect viewport READ viewport WRITE setViewport)
04394 Q_PROPERTY(QPixmap background READ background WRITE setBackground)
04395 Q_PROPERTY(bool backgroundScaled READ backgroundScaled WRITE setBackgroundScaled)
04396 Q_PROPERTY(Qt::AspectRatioMode backgroundScaledMode READ backgroundScaledMode WRITE
setBackgroundScaledMode)
04397 Q_PROPERTY(QCPLayoutGrid* plotLayout READ plotLayout)
04398 Q_PROPERTY(bool autoAddPlottableToLegend READ autoAddPlottableToLegend WRITE
setAutoAddPlottableToLegend)
04399 Q_PROPERTY(int selectionTolerance READ selectionTolerance WRITE setSelectionTolerance)
04400 Q_PROPERTY(bool noAntialiasingOnDrag READ noAntialiasingOnDrag WRITE setNoAntialiasingOnDrag)
04401 Q_PROPERTY(Qt::KeyboardModifier multiSelectModifier READ multiSelectModifier WRITE
setMultiSelectModifier)
04402 Q_PROPERTY(bool openGl READ openGl WRITE setOpenGl)
04403 public:
04404 enum LayerInsertMode {
04405 limBelow
04406 , limAbove
04407 };
04408 Q_ENUMS(LayerInsertMode)
04409
04410 enum RefreshPriority {
04411 rpImmediateRefresh
04412 , rpQueuedRefresh
04413 , rpRefreshHint
04414 , rpQueuedReplot
04415 };
04416 Q_ENUMS(RefreshPriority)
04417
04418 explicit QCustomPlot(QWidget *parent = nullptr);
04419 virtual ~QCustomPlot() Q_DECL_OVERRIDE;
04420
04421 // getters:
04422 QRect viewport() const {
04423 return mViewport;
04424 }
04425
04426 double bufferDevicePixelRatio() const {
04427 return mBufferDevicePixelRatio;
04428 }
04429
04430 QPixmap background() const {
04431 return mBackgroundPixmap;
04432 }
04433
04434 bool backgroundScaled() const {
04435 return mBackgroundScaled;
04436 }

```



```

04450 }
04451
04452 Qt::AspectRatioMode backgroundScaledMode() const {
04453 return mBackgroundScaledMode;
04454 }
04455
04456 QCPLayoutGrid *plotLayout() const {
04457 return mPlotLayout;
04458 }
04459
04460 QCP::AntialiasedElements antialiasedElements() const {
04461 return mAntialiasedElements;
04462 }
04463
04464 QCP::AntialiasedElements notAntialiasedElements() const {
04465 return mNotAntialiasedElements;
04466 }
04467
04468 bool autoAddPlottableToLegend() const {
04469 return mAutoAddPlottableToLegend;
04470 }
04471
04472 const QCP::Interactions interactions() const {
04473 return mInteractions;
04474 }
04475
04476 int selectionTolerance() const {
04477 return mSelectionTolerance;
04478 }
04479
04480 bool noAntialiasingOnDrag() const {
04481 return mNoAntialiasingOnDrag;
04482 }
04483
04484 QCP::PlottingHints plottingHints() const {
04485 return mPlottingHints;
04486 }
04487
04488 Qt::KeyboardModifier multiSelectModifier() const {
04489 return mMultiSelectModifier;
04490 }
04491
04492 QCP::SelectionRectMode selectionRectMode() const {
04493 return mSelectionRectMode;
04494 }
04495
04496 QCPSelectionRect *selectionRect() const {
04497 return mSelectionRect;
04498 }
04499
04500 bool openGl() const {
04501 return mOpenGl;
04502 }
04503
04504 // setters:
04505 void setViewport(const QRect &rect);
04506 void setBufferDevicePixelRatio(double ratio);
04507 void setBackground(const QPixmap &pm);
04508 void setBackground(const QPixmap &pm, bool scaled, Qt::AspectRatioMode mode =
Qt::KeepAspectRatioByExpanding);
04509 void setBackground(const QBrush &brush);
04510 void setBackgroundScaled(bool scaled);
04511 void setBackgroundScaledMode(Qt::AspectRatioMode mode);
04512 void setAntialiasedElements(const QCP::AntialiasedElements &antialiasedElements);
04513 void setAntialiasedElement(QCP::AntialiasedElement antialiasedElement, bool enabled = true);
04514 void setNotAntialiasedElements(const QCP::AntialiasedElements ¬AntialiasedElements);
04515 void setNotAntialiasedElement(QCP::AntialiasedElement notAntialiasedElement, bool enabled = true);
04516 void setAutoAddPlottableToLegend(bool on);
04517 void setInteractions(const QCP::Interactions &interactions);
04518 void setInteraction(const QCP::Interaction &interaction, bool enabled = true);
04519 void setSelectionTolerance(int pixels);
04520 void setNoAntialiasingOnDrag(bool enabled);
04521 void setPlottingHints(const QCP::PlottingHints &hints);
04522 void setPlottingHint(QCP::PlottingHint hint, bool enabled = true);
04523 void setMultiSelectModifier(Qt::KeyboardModifier modifier);
04524 void setSelectionRectMode(QCP::SelectionRectMode mode);
04525 void setSelectionRect(QCPSelectionRect *selectionRect);
04526 void setOpenGl(bool enabled, int multisampling = 16);
04527
04528 // non-property methods:
04529 // plottable interface:
04530 QCPAbstractPlottable *plottable(int index);
04531 QCPAbstractPlottable *plottable();
04532 bool removePlottable(QCPAbstractPlottable *plottable);
04533 bool removePlottable(int index);
04534 int clearPlottables();
04535 int plottableCount() const;

```

```

04536 QList<QCPAbstractPlottable*> selectedPlottables() const;
04537 template<class PlottableType>
04538 PlottableType *plottableAt(const QPointF &pos, bool onlySelectable = false, int *dataIndex =
nullptr) const;
04539 QCPAbstractPlottable *plottableAt(const QPointF &pos, bool onlySelectable = false, int *dataIndex
= nullptr) const;
04540 bool hasPlottable(QCPAbstractPlottable *plottable) const;
04541
04542 // specialized interface for QCPGraph:
04543 QCPGraph *graph(int index) const;
04544 QCPGraph *graph() const;
04545 QCPGraph *addGraph(QCPAxis *keyAxis = nullptr, QCPAxis *valueAxis = nullptr);
04546 bool removeGraph(QCPGraph *graph);
04547 bool removeGraph(int index);
04548 int clearGraphs();
04549 int graphCount() const;
04550 QList<QCPGraph*> selectedGraphs() const;
04551
04552 // item interface:
04553 QCPAbstractItem *item(int index) const;
04554 QCPAbstractItem *item() const;
04555 bool removeItem(QCPAbstractItem *item);
04556 bool removeItem(int index);
04557 int clearItems();
04558 int itemCount() const;
04559 QList<QCPAbstractItem*> selectedItems() const;
04560 template<class ItemType>
04561 ItemType *itemAt(const QPointF &pos, bool onlySelectable = false) const;
04562 QCPAbstractItem *itemAt(const QPointF &pos, bool onlySelectable = false) const;
04563 bool hasItem(QCPAbstractItem *item) const;
04564
04565 // layer interface:
04566 QCPLayer *layer(const QString &name) const;
04567 QCPLayer *layer(int index) const;
04568 QCPLayer *currentLayer() const;
04569 bool setCurrentLayer(const QString &name);
04570 bool setCurrentLayer(QCPLayer *layer);
04571 int layerCount() const;
04572 bool addLayer(const QString &name, QCPLayer *otherLayer = nullptr, LayerInsertMode insertMode =
limAbove);
04573 bool removeLayer(QCPLayer *layer);
04574 bool moveLayer(QCPLayer *layer, QCPLayer *otherLayer, LayerInsertMode insertMode = limAbove);
04575
04576 // axis rect/layout interface:
04577 int axisRectCount() const;
04578 QCPAxisRect* axisRect(int index = 0) const;
04579 QList<QCPAxisRect*> axisRects() const;
04580 QCPLayoutElement* layoutElementAt(const QPointF &pos) const;
04581 QCPAxisRect* axisRectAt(const QPointF &pos) const;
04582 Q_SLOT void rescaleAxes(bool onlyVisiblePlottables = false);
04583
04584 QList<QCPAxis*> selectedAxes() const;
04585 QList<QCPLegend*> selectedLegends() const;
04586 Q_SLOT void deselectAll();
04587
04588 bool savePdf(const QString &fileName, int width = 0, int height = 0, QCP::ExportPen exportPen =
QCP::epAllowCosmetic, const QString &pdfCreator = QString(), const QString &pdfTitle = QString());
04589 bool savePng(const QString &fileName, int width = 0, int height = 0, double scale = 1.0, int
quality = -1, int resolution = 96, QCP::ResolutionUnit resolutionUnit = QCP::ruDotsPerInch);
04590 bool saveJpg(const QString &fileName, int width = 0, int height = 0, double scale = 1.0, int
quality = -1, int resolution = 96, QCP::ResolutionUnit resolutionUnit = QCP::ruDotsPerInch);
04591 bool saveBmp(const QString &fileName, int width = 0, int height = 0, double scale = 1.0, int
resolution = 96, QCP::ResolutionUnit resolutionUnit = QCP::ruDotsPerInch);
04592 bool saveRastered(const QString &fileName, int width, int height, double scale, const char
*format, int quality = -1, int resolution = 96, QCP::ResolutionUnit resolutionUnit =
QCP::ruDotsPerInch);
04593 QPixmap toPixmap(int width = 0, int height = 0, double scale = 1.0);
04594 void toPainter(QCPPainter *painter, int width = 0, int height = 0);
04595 Q_SLOT void replot(QCustomPlot::RefreshPriority refreshPriority = QCustomPlot::rpRefreshHint);
04596 double replotTime(bool average = false) const;
04597
04598 QCPAxis *xAxis, *yAxis, *xAxis2, *yAxis2;
04599 QCPLegend *legend;
04600
04601 signals:
04602 void mouseDoubleClick(QMouseEvent *event);
04603 void mousePress(QMouseEvent *event);
04604 void mouseMove(QMouseEvent *event);
04605 void mouseRelease(QMouseEvent *event);
04606 void mouseWheel(QWheelEvent *event);
04607
04608 void plottableClick(QCPAbstractPlottable *plottable, int dataIndex, QMouseEvent *event);
04609 void plottableDoubleClick(QCPAbstractPlottable *plottable, int dataIndex, QMouseEvent *event);
04610 void itemClick(QCPAbstractItem *item, QMouseEvent *event);
04611 void itemDoubleClick(QCPAbstractItem *item, QMouseEvent *event);
04612 void axisClick(QCPAxis *axis, QCPAxis::SelectablePart part, QMouseEvent *event);
04613 void axisDoubleClick(QCPAxis *axis, QCPAxis::SelectablePart part, QMouseEvent *event);

```

```

04614 void legendClick(QCPLegend *legend, QCPAbstractLegendItem *item, QMouseEvent *event);
04615 void legendDoubleClick(QCPLegend *legend, QCPAbstractLegendItem *item, QMouseEvent *event);
04616
04617 void selectionChangedByUser();
04618 void beforeReplot();
04619 void afterLayout();
04620 void afterReplot();
04621
04622 protected:
04623 // property members:
04624 QRect mViewport;
04625 double mBufferDevicePixelRatio;
04626 QCPLayoutGrid *mPlotLayout;
04627 bool mAutoAddPlottableToLegend;
04628 QList<QCPAbstractPlottable*> mPlottables;
04629 QList<QCPGraph*> mGraphs; // extra list of plottables also in mPlottables that are of type
 QCPGraph
04630 QList<QCPAbstractItem*> mItems;
04631 QList<QCPLayer*> mLayers;
04632 QCP::AntialiasedElements mAntialiasedElements, mNotAntialiasedElements;
04633 QCP::Interactions mInteractions;
04634 int mSelectionTolerance;
04635 bool mNoAntialiasingOnDrag;
04636 QBrush mBackgroundBrush;
04637 QPixmap mBackgroundPixmap;
04638 QPixmap mScaledBackgroundPixmap;
04639 bool mBackgroundScaled;
04640 Qt::AspectRatioMode mBackgroundScaledMode;
04641 QCPLayer *mCurrentLayer;
04642 QCP::PlottingHints mPlottingHints;
04643 Qt::KeyboardModifier mMultiSelectModifier;
04644 QCP::SelectionRectMode mSelectionRectMode;
04645 QCPSelectionRect *mSelectionRect;
04646 bool mOpenGL;
04647
04648 // non-property members:
04649 QList<QSharedPointer<QCPAbstractPaintBuffer> > mPaintBuffers;
04650 QPoint mMousePressPos;
04651 bool mMouseHasMoved;
04652 QPointer<QCPLayerable> mMouseEventLayerable;
04653 QPointer<QCPLayerable> mMouseEventSignalLayerable;
04654 QVariant mMouseEventLayerableDetails;
04655 QVariant mMouseEventSignalLayerableDetails;
04656 bool mReplotting;
04657 bool mReplotQueued;
04658 double mReplotTime, mReplotTimeAverage;
04659 int mOpenGLMultisamples;
04660 QCP::AntialiasedElements mOpenGLAntialiasedElementsBackup;
04661 bool mOpenGLCacheLabelsBackup;
04662 #ifdef QCP_OPENGL_FBO
04663 QSharedPointer<QOpenGLContext> mGContext;
04664 QSharedPointer<QSurface> mGSurface;
04665 QSharedPointer<QOpenGLPaintDevice> mGPaintDevice;
04666 #endif
04667
04668 // reimplemented virtual methods:
04669 virtual QSize minimumSizeHint() const Q_DECL_OVERRIDE;
04670 virtual QSize sizeHint() const Q_DECL_OVERRIDE;
04671 virtual void paintEvent(QPaintEvent *event) Q_DECL_OVERRIDE;
04672 virtual void resizeEvent(QResizeEvent *event) Q_DECL_OVERRIDE;
04673 virtual void mouseDoubleClickEvent(QMouseEvent *event) Q_DECL_OVERRIDE;
04674 virtual void mousePressEvent(QMouseEvent *event) Q_DECL_OVERRIDE;
04675 virtual void mouseMoveEvent(QMouseEvent *event) Q_DECL_OVERRIDE;
04676 virtual void mouseReleaseEvent(QMouseEvent *event) Q_DECL_OVERRIDE;
04677 virtual void wheelEvent(QWheelEvent *event) Q_DECL_OVERRIDE;
04678
04679 // introduced virtual methods:
04680 virtual void draw(QCPPainter *painter);
04681 virtual void updateLayout();
04682 virtual void axisRemoved(QCPAxis *axis);
04683 virtual void legendRemoved(QCPLegend *legend);
04684 Q_SLOT virtual void processRectSelection(QRect rect, QMouseEvent *event);
04685 Q_SLOT virtual void processRectZoom(QRect rect, QMouseEvent *event);
04686 Q_SLOT virtual void processPointSelection(QMouseEvent *event);
04687
04688 // non-virtual methods:
04689 bool registerPlottable(QCPAbstractPlottable *plottable);
04690 bool registerGraph(QCPGraph *graph);
04691 bool registerItem(QCPAbstractItem *item);
04692 void updateLayerIndices() const;
04693 QCPLayerable *layerableAt(const QPointF &pos, bool onlySelectable, QVariant *selectionDetails =
 nullptr) const;
04694 QList<QCPLayerable*> layerableListAt(const QPointF &pos, bool onlySelectable, QList<QVariant>
 *selectionDetails = nullptr) const;
04695 void drawBackground(QCPPainter *painter);
04696 void setupPaintBuffers();
04697 QCPAbstractPaintBuffer *createPaintBuffer();

```

```

04698 bool hasInvalidatedPaintBuffers();
04699 bool setupOpenGL();
04700 void freeOpenGL();
04701
04702 friend class QCPLegend;
04703 friend class QCPAxis;
04704 friend class QCPLayer;
04705 friend class QCPAxisRect;
04706 friend class QCPAbstractPlottable;
04707 friend class QCPGraph;
04708 friend class QCPAbstractItem;
04709 };
04710
04711 Q_DECLARE_METATYPE(QCustomPlot::LayerInsertMode)
04712 Q_DECLARE_METATYPE(QCustomPlot::RefreshPriority)
04713
04714 // implementation of template functions:
04715
04735 template<class PlottableType>
04736 PlottableType *QCustomPlot::plottableAt(const QPointF &pos, bool onlySelectable, int *dataIndex) const
04737 {
04738 PlottableType *resultPlottable = 0;
04739 QVariant resultDetails;
04740 double resultDistance = mSelectionTolerance; // only regard clicks with distances smaller than
04741 mSelectionTolerance as selections, so initialize with that value
04742 foreach(QCPAbstractPlottable *plottable, mPlottables) {
04743 PlottableType *currentPlottable = qobject_cast<PlottableType*>(plottable);
04744 if (!currentPlottable || (onlySelectable && !currentPlottable->selectable())) // we could have
04745 also passed onlySelectable to the selectTest function, but checking here is faster, because we have
04746 access to QCPAbstractPlottable::selectable
04747 continue;
04748 if (currentPlottable->clipRect().contains(pos.toPoint())) // only consider clicks where the
04749 plottable is actually visible
04750 {
04751 QVariant details;
04752 double currentDistance = currentPlottable->selectTest(pos, false, dataIndex ? &details :
04753 nullptr);
04754 if (currentDistance >= 0 && currentDistance < resultDistance) {
04755 resultPlottable = currentPlottable;
04756 resultDetails = details;
04757 resultDistance = currentDistance;
04758 }
04759 }
04760 }
04761 if (resultPlottable && dataIndex) {
04762 QCPDataSelection sel = resultDetails.value<QCPDataSelection>();
04763 if (!sel.isEmpty())
04764 *dataIndex = sel.dataRange(0).begin();
04765 }
04766 return resultPlottable;
04767 }
04768
04778 template<class ItemType>
04779 ItemType *QCustomPlot::itemAt(const QPointF &pos, bool onlySelectable) const {
04780 ItemType *resultItem = 0;
04781 double resultDistance = mSelectionTolerance; // only regard clicks with distances smaller than
04782 mSelectionTolerance as selections, so initialize with that value
04783 foreach(QCPAbstractItem *item, mItems) {
04784 ItemType *currentItem = qobject_cast<ItemType*>(item);
04785 if (!currentItem || (onlySelectable && !currentItem->selectable())) // we could have also
04786 passed onlySelectable to the selectTest function, but checking here is faster, because we have access
04787 to QCPAbstractItem::selectable
04788 continue;
04789 if (!currentItem->clipToAxisRect() || currentItem->clipRect().contains(pos.toPoint())) // only
04790 consider clicks inside axis cliprect of the item if actually clipped to it
04791 {
04792 double currentDistance = currentItem->selectTest(pos, false);
04793 if (currentDistance >= 0 && currentDistance < resultDistance) {
04794 resultItem = currentItem;
04795 resultDistance = currentDistance;
04796 }
04797 }
04798 }
04799 return resultItem;
04800 }
04801
04802 /* end of 'src/core.h' */
04803
04804 /* including file 'src/plottableid.h' */

```

```

04806
04807 /* modified 2021-03-29T02:30:44, size 25638 */
04808
04809 class QCPPlottableInterface1D {
04810 public:
04811 virtual ~QCPPlottableInterface1D() = default;
04812 // introduced pure virtual methods:
04813 virtual int dataCount() const = 0;
04814 virtual double dataMainKey(int index) const = 0;
04815 virtual double dataSortKey(int index) const = 0;
04816 virtual double dataMainValue(int index) const = 0;
04817 virtual QCPRange dataValueRange(int index) const = 0;
04818 virtual QPointF dataPixelPosition(int index) const = 0;
04819 virtual bool sortKeyIsMainKey() const = 0;
04820 virtual QCPDataSelection selectTestRect(const QRectF &rect, bool onlySelectable) const = 0;
04821 virtual int findBegin(double sortKey, bool expandedRange = true) const = 0;
04822 virtual int findEnd(double sortKey, bool expandedRange = true) const = 0;
04823 };
04824
04825 template <class DataType>
04826 class QCPAbstractPlottable1D : public QCPAbstractPlottable, public QCPPlottableInterface1D // no
QCP_LIB_DECL, template class ends up in header (cpp included below)
04827 {
04828 // No Q_OBJECT macro due to template class
04829
04830 public:
04831 QCPAbstractPlottable1D(QCPAxis *keyAxis, QCPAxis *valueAxis);
04832 virtual ~QCPAbstractPlottable1D() Q_DECL_OVERRIDE;
04833
04834 // virtual methods of 1d plottable interface:
04835 virtual int dataCount() const Q_DECL_OVERRIDE;
04836 virtual double dataMainKey(int index) const Q_DECL_OVERRIDE;
04837 virtual double dataSortKey(int index) const Q_DECL_OVERRIDE;
04838 virtual double dataMainValue(int index) const Q_DECL_OVERRIDE;
04839 virtual QCPRange dataValueRange(int index) const Q_DECL_OVERRIDE;
04840 virtual QPointF dataPixelPosition(int index) const Q_DECL_OVERRIDE;
04841 virtual bool sortKeyIsMainKey() const Q_DECL_OVERRIDE;
04842 virtual QCPDataSelection selectTestRect(const QRectF &rect, bool onlySelectable) const
Q_DECL_OVERRIDE;
04843 virtual int findBegin(double sortKey, bool expandedRange = true) const Q_DECL_OVERRIDE;
04844 virtual int findEnd(double sortKey, bool expandedRange = true) const Q_DECL_OVERRIDE;
04845
04846 // reimplemented virtual methods:
04847 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
04848
04849 virtual QCPPlottableInterface1D *interface1D() Q_DECL_OVERRIDE {
04850 return this;
04851 }
04852
04853 protected:
04854 // property members:
04855 QSharedPointer<QCPDataContainer<DataType> > mDataContainer;
04856
04857 // helpers for subclasses:
04858 void getDataSegments(QList<QCPDataRange> &selectedSegments, QList<QCPDataRange>
&unselectedSegments) const;
04859 void drawPolyline(QCPPainter *painter, const QVector<QPointF> &lineData) const;
04860
04861 private:
04862 Q_DISABLE_COPY(QCPAbstractPlottable1D)
04863 };
04864
04865 // include implementation in header since it is a class template:
04866
04867 /* start documentation of pure virtual functions */
04868
04869 /* end documentation of pure virtual functions */
04870
04871 /* start documentation of inline functions */
04872
04873 /* end documentation of inline functions */
04874
04875 template <class DataType>
04876 QCPAbstractPlottable1D<DataType>::QCPAbstractPlottable1D(QCPAxis *keyAxis, QCPAxis *valueAxis) :
04877 QCPAbstractPlottable(keyAxis, valueAxis),
04878 mDataContainer(new QCPDataContainer<DataType>) {
04879 }
04880
04881 template <class DataType>
04882 QCPAbstractPlottable1D<DataType>::~~QCPAbstractPlottable1D() {
04883 }

```

```

05063
05064 template <class DataType>
05065 int QCPAbstractPlottable1D<DataType>::dataCount() const {
05066 return mDataContainer->size();
05067 }
05068
05069 template <class DataType>
05070 double QCPAbstractPlottable1D<DataType>::dataMainKey(int index) const {
05071 if (index >= 0 && index < mDataContainer->size()) {
05072 return (mDataContainer->constBegin() + index)->mainKey();
05073 } else {
05074 qDebug() << Q_FUNC_INFO << "Index out of bounds" << index;
05075 return 0;
05076 }
05077 }
05078
05079 template <class DataType>
05080 double QCPAbstractPlottable1D<DataType>::dataSortKey(int index) const {
05081 if (index >= 0 && index < mDataContainer->size()) {
05082 return (mDataContainer->constBegin() + index)->sortKey();
05083 } else {
05084 qDebug() << Q_FUNC_INFO << "Index out of bounds" << index;
05085 return 0;
05086 }
05087 }
05088
05089 template <class DataType>
05090 double QCPAbstractPlottable1D<DataType>::dataMainValue(int index) const {
05091 if (index >= 0 && index < mDataContainer->size()) {
05092 return (mDataContainer->constBegin() + index)->mainValue();
05093 } else {
05094 qDebug() << Q_FUNC_INFO << "Index out of bounds" << index;
05095 return 0;
05096 }
05097 }
05098
05099 template <class DataType>
05100 double QCPAbstractPlottable1D<DataType>::dataValueRange(int index) const {
05101 if (index >= 0 && index < mDataContainer->size()) {
05102 return (mDataContainer->constBegin() + index)->valueRange();
05103 } else {
05104 qDebug() << Q_FUNC_INFO << "Index out of bounds" << index;
05105 return QCPRange(0, 0);
05106 }
05107 }
05108
05109 template <class DataType>
05110 QPointF QCPAbstractPlottable1D<DataType>::dataPixelPosition(int index) const {
05111 if (index >= 0 && index < mDataContainer->size()) {
05112 const typename QCPDataContainer<DataType>::const_iterator it = mDataContainer->constBegin() +
05113 index;
05114 return coordsToPixels(it->mainKey(), it->mainValue());
05115 } else {
05116 qDebug() << Q_FUNC_INFO << "Index out of bounds" << index;
05117 return QPointF();
05118 }
05119 }
05120
05121 template <class DataType>
05122 bool QCPAbstractPlottable1D<DataType>::sortKeyIsMainKey() const {
05123 return DataType::sortKeyIsMainKey();
05124 }
05125
05126 template <class DataType>
05127 QCPDataSelection QCPAbstractPlottable1D<DataType>::selectTestRect(const QRectF &rect, bool
05128 onlySelectable) const {
05129 QCPDataSelection result;
05130 if ((onlySelectable && mSelectable == QCP::stNone) || mDataContainer->isEmpty())
05131 return result;
05132 if (!mKeyAxis || !mValueAxis)
05133 return result;
05134
05135 // convert rect given in pixels to ranges given in plot coordinates:
05136 double key1, value1, key2, value2;
05137 pixelsToCoords(rect.topLeft(), key1, value1);
05138 pixelsToCoords(rect.bottomRight(), key2, value2);
05139 QCPRange keyRange(key1, key2); // QCPRange normalizes internally so we don't have to care about
05140 whether key1 < key2
05141 QCPRange valueRange(value1, value2);
05142 typename QCPDataContainer<DataType>::const_iterator begin = mDataContainer->constBegin();
05143 typename QCPDataContainer<DataType>::const_iterator end = mDataContainer->constEnd();
05144 if (DataType::sortKeyIsMainKey()) // we can assume that data is sorted by main key, so can reduce
05145 the searched key interval:
05146 {
05147 begin = mDataContainer->findBegin(keyRange.lower, false);
05148 end = mDataContainer->findEnd(keyRange.upper, false);
05149 }

```

```

05174 if (begin == end)
05175 return result;
05176
05177 int currentSegmentBegin = -1; // -1 means we're currently not in a segment that's contained in
rect
05178 for (typename QCPDataContainer<DataType>::const_iterator it = begin; it != end; ++it) {
05179 if (currentSegmentBegin == -1) {
05180 if (valueRange.contains(it->mainValue()) && keyRange.contains(it->mainKey())) // start
segment
05181 currentSegmentBegin = int(it - mDataContainer->constBegin());
05182 } else if (!valueRange.contains(it->mainValue()) || !keyRange.contains(it->mainKey())) //
segment just ended
05183 {
05184 result.addDataRange(QCPDataRange(currentSegmentBegin, int(it -
mDataContainer->constBegin()), false);
05185 currentSegmentBegin = -1;
05186 }
05187 }
05188 // process potential last segment:
05189 if (currentSegmentBegin != -1)
05190 result.addDataRange(QCPDataRange(currentSegmentBegin, int(end -
mDataContainer->constBegin()), false);
05191
05192 result.simplify();
05193 return result;
05194 }
05195
05196 template <class DataType>
05197 int QCPAbstractPlottable1D<DataType>::findBegin(double sortKey, bool expandedRange) const {
05198 return int(mDataContainer->findBegin(sortKey, expandedRange) - mDataContainer->constBegin());
05199 }
05200
05201 template <class DataType>
05202 int QCPAbstractPlottable1D<DataType>::findEnd(double sortKey, bool expandedRange) const {
05203 return int(mDataContainer->findEnd(sortKey, expandedRange) - mDataContainer->constBegin());
05204 }
05205
05206 template <class DataType>
05207 double QCPAbstractPlottable1D<DataType>::selectTest(const QPointF &pos, bool onlySelectable, QVariant
*details) const {
05208 if ((onlySelectable && mSelectable == QCP::stNone) || mDataContainer->isEmpty())
05209 return -1;
05210 if (!mKeyAxis || !mValueAxis)
05211 return -1;
05212
05213 QCPDataSelection selectionResult;
05214 double minDistSqr = (std::numeric_limits<double>::max)();
05215 int minDistIndex = mDataContainer->size();
05216
05217 typename QCPDataContainer<DataType>::const_iterator begin = mDataContainer->constBegin();
05218 typename QCPDataContainer<DataType>::const_iterator end = mDataContainer->constEnd();
05219 if (DataType::sortKeyIsMainKey()) // we can assume that data is sorted by main key, so can reduce
the searched key interval:
05220 {
05221 // determine which key range comes into question, taking selection tolerance around pos into
account:
05222 double posKeyMin, posKeyMax, dummy;
05223 pixelsToCoords(pos - QPointF(mParentPlot->selectionTolerance(),
mParentPlot->selectionTolerance()), posKeyMin, dummy);
05224 pixelsToCoords(pos + QPointF(mParentPlot->selectionTolerance(),
mParentPlot->selectionTolerance()), posKeyMax, dummy);
05225 if (posKeyMin > posKeyMax)
05226 qSwap(posKeyMin, posKeyMax);
05227 begin = mDataContainer->findBegin(posKeyMin, true);
05228 end = mDataContainer->findEnd(posKeyMax, true);
05229 }
05230 if (begin == end)
05231 return -1;
05232 QCPRange keyRange(mKeyAxis->range());
05233 QCPRange valueRange(mValueAxis->range());
05234 for (typename QCPDataContainer<DataType>::const_iterator it = begin; it != end; ++it) {
05235 const double mainKey = it->mainKey();
05236 const double mainValue = it->mainValue();
05237 if (keyRange.contains(mainKey) && valueRange.contains(mainValue)) // make sure data point is
inside visible range, for speedup in cases where sort key isn't main key and we iterate over all
points
05238 {
05239 const double currentDistSqr = QCPVector2D(coordsToPixels(mainKey, mainValue) -
pos).lengthSquared();
05240 if (currentDistSqr < minDistSqr) {
05241 minDistSqr = currentDistSqr;
05242 minDistIndex = int(it - mDataContainer->constBegin());
05243 }
05244 }
05245 }
05246 if (minDistIndex != mDataContainer->size())
05247 selectionResult.addDataRange(QCPDataRange(minDistIndex, minDistIndex + 1), false);
05248 }

```



```

05264
05265 selectionResult.simplify();
05266 if (details)
05267 details->setValue(selectionResult);
05268 return qSqrt(minDistSqr);
05269 }
05270
05281 template <class DataType>
05282 void QCPAbstractPlottableID<DataType>::getDataSegments(QList<QCPDataRange> &selectedSegments,
05283 QList<QCPDataRange> &unselectedSegments) const {
05284 selectedSegments.clear();
05285 unselectedSegments.clear();
05286 if (mSelectable == QCP::stWhole) // stWhole selection type draws the entire plottable with
05287 selected style if mSelection isn't empty
05288 {
05289 if (selected())
05290 selectedSegments << QCPDataRange(0, dataCount());
05291 else
05292 unselectedSegments << QCPDataRange(0, dataCount());
05293 } else {
05294 QCPDataSelection sel(selection());
05295 sel.simplify();
05296 selectedSegments = sel.dataRanges();
05297 unselectedSegments = sel.inverse(QCPDataRange(0, dataCount())).dataRanges();
05298 }
05299 }
05300
05301 template <class DataType>
05302 void QCPAbstractPlottableID<DataType>::drawPolyline(QCPPainter *painter, const QVector<QPointF>
05303 &lineData) const {
05304 // if drawing lines in plot (instead of PDF), reduce 1px lines to cosmetic, because at least in
05305 // Qt6 drawing of "1px" width lines is much slower even though it has same appearance apart from
05306 // High-DPI. In High-DPI cases people must set a pen width slightly larger than 1.0 to get
05307 // correct DPI scaling of width, but of course with performance penalty.
05308 if (!painter->modes().testFlag(QCPPainter::pmVectorized) &&
05309 qFuzzyCompare(painter->pen().widthF(), 1.0)) {
05310 QPen newPen = painter->pen();
05311 newPen.setWidth(0);
05312 painter->setPen(newPen);
05313 }
05314
05315 // if drawing solid line and not in PDF, use much faster line drawing instead of polyline:
05316 if (mParentPlot->plottingHints().testFlag(QCP::phFastPolylines) &&
05317 painter->pen().style() == Qt::SolidLine &&
05318 !painter->modes().testFlag(QCPPainter::pmVectorized) &&
05319 !painter->modes().testFlag(QCPPainter::pmNoCaching)) {
05320 int i = 0;
05321 bool lastIsNaN = false;
05322 const int lineDataSize = lineData.size();
05323 while (i < lineDataSize && (qIsNaN(lineData.at(i).y()) || qIsNaN(lineData.at(i).x()))) // make
05324 sure first point is not NaN
05325 ++i;
05326 ++i; // because drawing works in 1 point retrospect
05327 while (i < lineDataSize) {
05328 if (!qIsNaN(lineData.at(i).y()) && !qIsNaN(lineData.at(i).x())) // NaNs create a gap in
05329 the line
05330 {
05331 if (!lastIsNaN)
05332 painter->drawLine(lineData.at(i - 1), lineData.at(i));
05333 else
05334 lastIsNaN = false;
05335 } else {
05336 lastIsNaN = true;
05337 ++i;
05338 }
05339 } else {
05340 int segmentStart = 0;
05341 int i = 0;
05342 const int lineDataSize = lineData.size();
05343 while (i < lineDataSize) {
05344 if (qIsNaN(lineData.at(i).y()) || qIsNaN(lineData.at(i).x()) ||
05345 qIsInf(lineData.at(i).y())) // NaNs create a gap in the line. Also filter Infs which make drawPolyline
05346 block
05347 {
05348 painter->drawPolyline(lineData.constData() + segmentStart, i - segmentStart); // i,
05349 because we don't want to include the current NaN point
05350 segmentStart = i + 1;
05351 }
05352 ++i;
05353 }
05354 // draw last segment:
05355 painter->drawPolyline(lineData.constData() + segmentStart, lineDataSize - segmentStart);
05356 }
05357 }
05358 }
05359 }
05360
05361
05362 /* end of 'src/plottableId.h' */

```



```

05363
05364
05365 /* including file 'src/colorgradient.h' */
05366
05367 /* modified 2021-03-29T02:30:44, size 7262 */
05368
05369 class QCP_LIB_DECL QCPColorGradient {
05370
05371 Q_GADGET
05372 public:
05373 enum ColorInterpolation {
05374 ciRGB
05375 , ciHSV
05376 };
05377
05378 Q_ENUMS(ColorInterpolation)
05379
05380 enum NanHandling {
05381 nhNone
05382 , nhLowestColor
05383 , nhHighestColor
05384 , nhTransparent
05385 , nhNanColor
05386 };
05387
05388 Q_ENUMS(NanHandling)
05389
05390 enum GradientPreset {
05391 gpGrayscale
05392 , gpHot
05393 , gpCold
05394 , gpNight
05395 , gpCandy
05396 , gpGeography
05397 , gpIon
05398 , gpThermal
05399 , gpPolar
05400 , gpSpectrum
05401 , gpJet
05402 , gpHues
05403 };
05404
05405 Q_ENUMS(GradientPreset)
05406
05407 QCPColorGradient();
05408 QCPColorGradient(GradientPreset preset);
05409 bool operator==(const QCPColorGradient &other) const;
05410
05411 bool operator!=(const QCPColorGradient &other) const {
05412 return !(*this == other);
05413 }
05414
05415 // getters:
05416
05417 int levelCount() const {
05418 return mLevelCount;
05419 }
05420
05421 QMap<double, QColor> colorStops() const {
05422 return mColorStops;
05423 }
05424
05425 ColorInterpolation colorInterpolation() const {
05426 return mColorInterpolation;
05427 }
05428
05429 NanHandling nanHandling() const {
05430 return mNanHandling;
05431 }
05432
05433 QColor nanColor() const {
05434 return mNanColor;
05435 }
05436
05437 bool periodic() const {
05438 return mPeriodic;
05439 }
05440
05441 // setters:
05442 void setLevelCount(int n);
05443 void setColorStops(const QMap<double, QColor> &colorStops);
05444 void setColorStopAt(double position, const QColor &color);
05445 void setColorInterpolation(ColorInterpolation interpolation);
05446 void setNanHandling(NanHandling handling);
05447 void setNanColor(const QColor &color);
05448 void setPeriodic(bool enabled);

```

```

05462
05463 // non-property methods:
05464 void colorize(const double *data, const QCPRange &range, QRgb *scanLine, int n, int
dataIndexFactor = 1, bool logarithmic = false);
05465 void colorize(const double *data, const unsigned char *alpha, const QCPRange &range, QRgb
*scanLine, int n, int dataIndexFactor = 1, bool logarithmic = false);
05466 QRgb color(double position, const QCPRange &range, bool logarithmic = false);
05467 void loadPreset(GradientPreset preset);
05468 void clearColorStops();
05469 QCPCColorGradient inverted() const;
05470
05471 protected:
05472 // property members:
05473 int mLevelCount;
05474 QMap<double, QColor> mColorStops;
05475 ColorInterpolation mColorInterpolation;
05476 NanHandling mNanHandling;
05477 QColor mNanColor;
05478 bool mPeriodic;
05479
05480 // non-property members:
05481 QVector<QRgb> mColorBuffer; // have colors premultiplied with alpha (for usage with
QImage::Format_ARGB32_Premultiplied)
05482 bool mColorBufferInvalidated;
05483
05484 // non-virtual methods:
05485 bool stopsUseAlpha() const;
05486 void updateColorBuffer();
05487 };
05488
05489 Q_DECLARE_METATYPE(QCPCColorGradient::ColorInterpolation)
05490 Q_DECLARE_METATYPE(QCPCColorGradient::NanHandling)
05491 Q_DECLARE_METATYPE(QCPCColorGradient::GradientPreset)
05492
05493 /* end of 'src/colorgradient.h' */
05494
05495
05496 /* including file 'src/selectiondecorator-bracket.h' */
05497 /* modified 2021-03-29T02:30:44, size 4458 */
05498
05499 class QCP_LIB_DECL QCPSelectionDecoratorBracket : public QCPSelectionDecorator {
05500
05501 Q_GADGET
05502 public:
05503
05504 enum BracketStyle {
05505 bsSquareBracket
05506 , bsHalfEllipse
05507 , bsEllipse
05508 , bsPlus
05509 , bsUserStyle
05510 };
05511 Q_ENUMS(BracketStyle)
05512
05513 QCPSelectionDecoratorBracket();
05514 virtual ~QCPSelectionDecoratorBracket() Q_DECL_OVERRIDE;
05515
05516 // getters:
05517
05518 QPen bracketPen() const {
05519 return mBracketPen;
05520 }
05521
05522 QBrush bracketBrush() const {
05523 return mBracketBrush;
05524 }
05525
05526 int bracketWidth() const {
05527 return mBracketWidth;
05528 }
05529
05530 int bracketHeight() const {
05531 return mBracketHeight;
05532 }
05533
05534 BracketStyle bracketStyle() const {
05535 return mBracketStyle;
05536 }
05537
05538 bool tangentToData() const {
05539 return mTangentToData;
05540 }
05541
05542 int tangentAverage() const {
05543 return mTangentAverage;
05544 }
05545
05546
05547
05548
05549
05550
05551

```

```

05552 // setters:
05553 void setBracketPen(const QPen &pen);
05554 void setBracketBrush(const QBrush &brush);
05555 void setBracketWidth(int width);
05556 void setBracketHeight(int height);
05557 void setBracketStyle(BracketStyle style);
05558 void setTangentToData(bool enabled);
05559 void setTangentAverage(int pointCount);
05560
05561 // introduced virtual methods:
05562 virtual void drawBracket(QCPPainter *painter, int direction) const;
05563
05564 // virtual methods:
05565 virtual void drawDecoration(QCPPainter *painter, QCPDataSelection selection) Q_DECL_OVERRIDE;
05566
05567 protected:
05568 // property members:
05569 QPen mBracketPen;
05570 QBrush mBracketBrush;
05571 int mBracketWidth;
05572 int mBracketHeight;
05573 BracketStyle mBracketStyle;
05574 bool mTangentToData;
05575 int mTangentAverage;
05576
05577 // non-virtual methods:
05578 double getTangentAngle(const QCPPlottableInterfaceID *interfaceId, int dataIndex, int direction)
05579 const;
05579 QPointF getPixelCoordinates(const QCPPlottableInterfaceID *interfaceId, int dataIndex) const;
05580
05581 };
05582
05583 Q_DECLARE_METATYPE(QCPSelectionDecoratorBracket::BracketStyle)
05584
05585 /* end of 'src/selectiondecorator-bracket.h' */
05586
05587
05588 /* including file 'src/layoutelements/layoutelement-axisrect.h' */
05589 /* modified 2021-03-29T02:30:44, size 7529 */
05590
05591 class QCP_LIB_DECL QCPAxisRect : public QCPLayoutElement {
05592 Q_OBJECT
05593 Q_PROPERTY(QPixmap background READ background WRITE setBackground)
05594 Q_PROPERTY(bool backgroundScaled READ backgroundScaled WRITE setBackgroundScaled)
05595 Q_PROPERTY(QCPAspectMode backgroundScaledMode READ backgroundScaledMode WRITE
05596 setBackgroundScaledMode)
05597 Q_PROPERTY(Qt::Orientations rangeDrag READ rangeDrag WRITE setRangeDrag)
05598 Q_PROPERTY(Qt::Orientations rangeZoom READ rangeZoom WRITE setRangeZoom)
05599 public:
05600 explicit QCPAxisRect(QCustomPlot *parentPlot, bool setupDefaultAxes = true);
05601 virtual ~QCPAxisRect() Q_DECL_OVERRIDE;
05602
05603 // getters:
05604
05605 QPixmap background() const {
05606 return mBackgroundPixmap;
05607 }
05608
05609 QBrush backgroundBrush() const {
05610 return mBackgroundBrush;
05611 }
05612
05613 bool backgroundScaled() const {
05614 return mBackgroundScaled;
05615 }
05616
05617 Qt::AspectRatioMode backgroundScaledMode() const {
05618 return mBackgroundScaledMode;
05619 }
05620
05621 Qt::Orientations rangeDrag() const {
05622 return mRangeDrag;
05623 }
05624
05625 Qt::Orientations rangeZoom() const {
05626 return mRangeZoom;
05627 }
05628
05629 QCPAxis *rangeDragAxis(Qt::Orientation orientation);
05630 QCPAxis *rangeZoomAxis(Qt::Orientation orientation);
05631 QList<QCPAxis*> rangeDragAxes(Qt::Orientation orientation);
05632 QList<QCPAxis*> rangeZoomAxes(Qt::Orientation orientation);
05633 double rangeZoomFactor(Qt::Orientation orientation);
05634
05635 // setters:
05636 void setBackground(const QPixmap &pm);
05637 void setBackground(const QPixmap &pm, bool scaled, Qt::AspectRatioMode mode =
05638 Qt::KeepAspectRatioByExpanding);

```

```

05638 void setBackground(const QBrush &brush);
05639 void setBackgroundScaled(bool scaled);
05640 void setBackgroundScaledMode(Qt::AspectRatioMode mode);
05641 void setRangeDrag(Qt::Orientations orientations);
05642 void setRangeZoom(Qt::Orientations orientations);
05643 void setRangeDragAxes(QCPAxis *horizontal, QCPAxis *vertical);
05644 void setRangeDragAxes(QList<QCPAxis*> axes);
05645 void setRangeDragAxes(QList<QCPAxis*> horizontal, QList<QCPAxis*> vertical);
05646 void setRangeZoomAxes(QCPAxis *horizontal, QCPAxis *vertical);
05647 void setRangeZoomAxes(QList<QCPAxis*> axes);
05648 void setRangeZoomAxes(QList<QCPAxis*> horizontal, QList<QCPAxis*> vertical);
05649 void setRangeZoomFactor(double horizontalFactor, double verticalFactor);
05650 void setRangeZoomFactor(double factor);
05651
05652 // non-property methods:
05653 int axisCount(QCPAxis::AxisType type) const;
05654 QCPAxis *axis(QCPAxis::AxisType type, int index = 0) const;
05655 QList<QCPAxis*> axes(QCPAxis::AxisTypes types) const;
05656 QList<QCPAxis*> axes() const;
05657 QCPAxis *addAxis(QCPAxis::AxisType type, QCPAxis *axis = nullptr);
05658 QList<QCPAxis*> addAxes(QCPAxis::AxisTypes types);
05659 bool removeAxis(QCPAxis *axis);
05660
05661 QCPLLayoutInset *insetLayout() const {
05662 return mInsetLayout;
05663 }
05664
05665 void zoom(const QRectF &pixelRect);
05666 void zoom(const QRectF &pixelRect, const QList<QCPAxis*> &affectedAxes);
05667 void setupFullAxesBox(bool connectRanges = false);
05668 QList<QCPAbstractPlottable*> plottables() const;
05669 QList<QCPGraph*> graphs() const;
05670 QList<QCPAbstractItem*> items() const;
05671
05672 // read-only interface imitating a QRect:
05673
05674 int left() const {
05675 return mRect.left();
05676 }
05677
05678 int right() const {
05679 return mRect.right();
05680 }
05681
05682 int top() const {
05683 return mRect.top();
05684 }
05685
05686 int bottom() const {
05687 return mRect.bottom();
05688 }
05689
05690 int width() const {
05691 return mRect.width();
05692 }
05693
05694 int height() const {
05695 return mRect.height();
05696 }
05697
05698 QSize size() const {
05699 return mRect.size();
05700 }
05701
05702 QPoint topLeft() const {
05703 return mRect.topLeft();
05704 }
05705
05706 QPoint topRight() const {
05707 return mRect.topRight();
05708 }
05709
05710 QPoint bottomLeft() const {
05711 return mRect.bottomLeft();
05712 }
05713
05714 QPoint bottomRight() const {
05715 return mRect.bottomRight();
05716 }
05717
05718 QPoint center() const {
05719 return mRect.center();
05720 }
05721
05722 // reimplemented virtual methods:
05723 virtual void update(UpdatePhase phase) Q_DECL_OVERRIDE;
05724 virtual QList<QCPLLayoutElement*> elements(bool recursive) const Q_DECL_OVERRIDE;

```

```

05725
05726 protected:
05727 // property members:
05728 QBrush mBackgroundBrush;
05729 QPixmap mBackgroundPixmap;
05730 QPixmap mScaledBackgroundPixmap;
05731 bool mBackgroundScaled;
05732 Qt::AspectRatioMode mBackgroundScaledMode;
05733 QCPLLayoutInset *mInsetLayout;
05734 Qt::Orientations mRangeDrag, mRangeZoom;
05735 QList<QPointer<QCPAxis> > mRangeDragHorzAxis, mRangeDragVertAxis;
05736 QList<QPointer<QCPAxis> > mRangeZoomHorzAxis, mRangeZoomVertAxis;
05737 double mRangeZoomFactorHorz, mRangeZoomFactorVert;
05738
05739 // non-property members:
05740 QList<QCPRange> mDragStartHorzRange, mDragStartVertRange;
05741 QCP::AntialiasedElements mAADragBackup, mNotAADragBackup;
05742 bool mDragging;
05743 QHash<QCPAxis::AxisType, QList<QCPAxis*> > mAxes;
05744
05745 // reimplemented virtual methods:
05746 virtual void applyDefaultAntialiasingHint(QCPLPainter *painter) const Q_DECL_OVERRIDE;
05747 virtual void draw(QCPLPainter *painter) Q_DECL_OVERRIDE;
05748 virtual int calculateAutoMargin(QCP::MarginSide side) Q_DECL_OVERRIDE;
05749 virtual void layoutChanged() Q_DECL_OVERRIDE;
05750 // events:
05751 virtual void mousePressEvent(QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE;
05752 virtual void mouseMoveEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
05753 virtual void mouseReleaseEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
05754 virtual void wheelEvent(QWheelEvent *event) Q_DECL_OVERRIDE;
05755
05756 // non-property methods:
05757 void drawBackground(QCPLPainter *painter);
05758 void updateAxesOffset(QCPAxis::AxisType type);
05759
05760 private:
05761 Q_DISABLE_COPY(QCPAxisRect)
05762
05763 friend class QCustomPlot;
05764 };
05765
05766
05767 /* end of 'src/layoutelements/layoutelement-axisrect.h' */
05768
05769
05770 /* including file 'src/layoutelements/layoutelement-legend.h' */
05771
05772 /* modified 2021-03-29T02:30:44, size 10425 */
05773
05774 class QCP_LIB_DECL QCPAbstractLegendItem : public QCPLLayoutElement {
05775 Q_OBJECT
05776 Q_PROPERTY(QCPLegend* parentLegend READ parentLegend)
05777 Q_PROPERTY(QFont font READ font WRITE setFont)
05778 Q_PROPERTY(QColor textColor READ textColor WRITE setTextColor)
05779 Q_PROPERTY(QFont selectedFont READ selectedFont WRITE setSelectedFont)
05780 Q_PROPERTY(QColor selectedTextColor READ selectedTextColor WRITE setSelectedTextColor)
05781 Q_PROPERTY(bool selectable READ selectable WRITE setSelectable NOTIFY selectionChanged)
05782 Q_PROPERTY(bool selected READ selected WRITE setSelected NOTIFY selectableChanged)
05783 public:
05784 explicit QCPAbstractLegendItem(QCPLegend *parent);
05785
05786 // getters:
05787
05788 QCPLegend *parentLegend() const {
05789 return mParentLegend;
05790 }
05791
05792 QFont font() const {
05793 return mFont;
05794 }
05795
05796 QColor textColor() const {
05797 return mTextColor;
05798 }
05799
05800 QFont selectedFont() const {
05801 return mSelectedFont;
05802 }
05803
05804 QColor selectedTextColor() const {
05805 return mSelectedTextColor;
05806 }
05807
05808 bool selectable() const {
05809 return mSelectable;
05810 }
05811
05812
05813

```

```

05814 bool selected() const {
05815 return mSelected;
05816 }
05817
05818 // setters:
05819 void setFont(const QFont &font);
05820 void setTextColor(const QColor &color);
05821 void setSelectedFont(const QFont &font);
05822 void setSelectedTextColor(const QColor &color);
05823 Q_SLOT void setSelectable(bool selectable);
05824 Q_SLOT void setSelected(bool selected);
05825
05826 // reimplemented virtual methods:
05827 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
05828 const Q_DECL_OVERRIDE;
05829
05829 signals:
05830 void selectionChanged(bool selected);
05831 void selectableChanged(bool selectable);
05832
05833 protected:
05834 // property members:
05835 QCPLegend *mParentLegend;
05836 QFont mFont;
05837 QColor mTextColor;
05838 QFont mSelectedFont;
05839 QColor mSelectedTextColor;
05840 bool mSelectable, mSelected;
05841
05842 // reimplemented virtual methods:
05843 virtual QCP::Interaction selectionCategory() const Q_DECL_OVERRIDE;
05844 virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
05845 virtual QRect clipRect() const Q_DECL_OVERRIDE;
05846 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE = 0;
05847 // events:
05848 virtual void selectEvent(QMouseEvent *event, bool additive, const QVariant &details, bool
05849 *selectionStateChanged) Q_DECL_OVERRIDE;
05850 virtual void deselectEvent(bool *selectionStateChanged) Q_DECL_OVERRIDE;
05851
05851 private:
05852 Q_DISABLE_COPY(QCPAbstractLegendItem)
05853
05854 friend class QCPLegend;
05855 };
05856
05857 class QCP_LIB_DECL QCPPlottableLegendItem : public QCPAbstractLegendItem {
05858 Q_OBJECT
05859 public:
05860 QCPPlottableLegendItem(QCPLegend *parent, QCPAbstractPlottable *plottable);
05861
05862 // getters:
05863
05864 QCPAbstractPlottable *plottable() {
05865 return mPlottable;
05866 }
05867
05868 protected:
05869 // property members:
05870 QCPAbstractPlottable *mPlottable;
05871
05872 // reimplemented virtual methods:
05873 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
05874 virtual QSize minimumOuterSizeHint() const Q_DECL_OVERRIDE;
05875
05876 // non-virtual methods:
05877 QPen getIconBorderPen() const;
05878 QColor getTextColor() const;
05879 QFont getFont() const;
05880 };
05881
05882 class QCP_LIB_DECL QCPLegend : public QCPLayoutGrid {
05883 Q_OBJECT
05884 Q_PROPERTY(QPen borderPen READ borderPen WRITE setBorderPen)
05885 Q_PROPERTY(QBrush brush READ brush WRITE setBrush)
05886 Q_PROPERTY(QFont font READ font WRITE setFont)
05887 Q_PROPERTY(QColor textColor READ textColor WRITE setTextColor)
05888 Q_PROPERTY(QSize iconSize READ iconSize WRITE setIconSize)
05889 Q_PROPERTY(int iconTextPadding READ iconTextPadding WRITE setIconTextPadding)
05890 Q_PROPERTY(QPen iconBorderPen READ iconBorderPen WRITE setIconBorderPen)
05891 Q_PROPERTY(SelectableParts selectableParts READ selectableParts WRITE setSelectableParts NOTIFY
05892 selectionChanged)
05893 Q_PROPERTY(SelectableParts selectedParts READ selectedParts WRITE setSelectedParts NOTIFY
05894 selectableChanged)
05895 Q_PROPERTY(QPen selectedBorderPen READ selectedBorderPen WRITE setSelectedBorderPen)
05896 Q_PROPERTY(QPen selectedIconBorderPen READ selectedIconBorderPen WRITE setSelectedIconBorderPen)
05897 Q_PROPERTY(QBrush selectedBrush READ selectedBrush WRITE setSelectedBrush)

```

```
05898 Q_PROPERTY(QFont selectedFont READ selectedFont WRITE setSelectedFont)
05899 Q_PROPERTY(QColor selectedTextColor READ selectedTextColor WRITE setSelectedTextColor)
05901 public:
05902 enum SelectablePart {
05903 spNone = 0x000
05904 , spLegendBox = 0x001
05905 , spItems = 0x002
05906 };
05907 Q_ENUMS(SelectablePart)
05908 Q_FLAGS(SelectableParts)
05909 Q_DECLARE_FLAGS(SelectableParts, SelectablePart)
05910
05911 explicit QCPLegend();
05912 virtual ~QCPLegend() Q_DECL_OVERRIDE;
05913
05914 // getters:
05915
05916 QPen borderPen() const {
05917 return mBorderPen;
05918 }
05919
05920 QBrush brush() const {
05921 return mBrush;
05922 }
05923
05924 QFont font() const {
05925 return mFont;
05926 }
05927
05928 QColor textColor() const {
05929 return mTextColor;
05930 }
05931
05932 QSize iconSize() const {
05933 return mIconSize;
05934 }
05935
05936 int iconTextPadding() const {
05937 return mIconTextPadding;
05938 }
05939
05940 QPen iconBorderPen() const {
05941 return mIconBorderPen;
05942 }
05943
05944 SelectableParts selectableParts() const {
05945 return mSelectableParts;
05946 }
05947
05948 SelectableParts selectedParts() const;
05949
05950 QPen selectedBorderPen() const {
05951 return mSelectedBorderPen;
05952 }
05953
05954 QPen selectedIconBorderPen() const {
05955 return mSelectedIconBorderPen;
05956 }
05957
05958 QBrush selectedBrush() const {
05959 return mSelectedBrush;
05960 }
05961
05962 QFont selectedFont() const {
05963 return mSelectedFont;
05964 }
05965
05966 QColor selectedTextColor() const {
05967 return mSelectedTextColor;
05968 }
05969
05970 // setters:
05971 void setBorderPen(const QPen &pen);
05972 void setBrush(const QBrush &brush);
05973 void setFont(const QFont &font);
05974 void setTextColor(const QColor &color);
05975 void setIconSize(const QSize &size);
05976 void setIconSize(int width, int height);
05977 void setIconTextPadding(int padding);
05978 void setIconBorderPen(const QPen &pen);
05979 Q_SLOT void setSelectableParts(const SelectableParts &selectableParts);
05980 Q_SLOT void setSelectedParts(const SelectableParts &selectedParts);
05981 void setSelectedBorderPen(const QPen &pen);
05982 void setSelectedIconBorderPen(const QPen &pen);
05983 void setSelectedBrush(const QBrush &brush);
05984 void setSelectedFont(const QFont &font);
05985 void setSelectedTextColor(const QColor &color);
05986
05987 05990
```

```

05991 // reimplemented virtual methods:
05992 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
05993
05994 // non-virtual methods:
05995 QCPAbstractLegendItem *item(int index) const;
05996 QCPPlottableLegendItem *itemWithPlottable(const QCPAbstractPlottable *plottable) const;
05997 int itemCount() const;
05998 bool hasItem(QCPAbstractLegendItem *item) const;
05999 bool hasItemWithPlottable(const QCPAbstractPlottable *plottable) const;
06000 bool addItem(QCPAbstractLegendItem *item);
06001 bool removeItem(int index);
06002 bool removeItem(QCPAbstractLegendItem *item);
06003 void clearItems();
06004 QList<QCPAbstractLegendItem*> selectedItems() const;
06005
06006 signals:
06007 void selectionChanged(QCPLegend::SelectableParts parts);
06008 void selectableChanged(QCPLegend::SelectableParts parts);
06009
06010 protected:
06011 // property members:
06012 QPen mBorderPen, mIconBorderPen;
06013 QBrush mBrush;
06014 QFont mFont;
06015 QColor mTextColor;
06016 QSize mIconSize;
06017 int mIconTextPadding;
06018 SelectableParts mSelectedParts, mSelectableParts;
06019 QPen mSelectedBorderPen, mSelectedIconBorderPen;
06020 QBrush mSelectedBrush;
06021 QFont mSelectedFont;
06022 QColor mSelectedTextColor;
06023
06024 // reimplemented virtual methods:
06025 virtual void parentPlotInitialized(QCustomPlot *parentPlot) Q_DECL_OVERRIDE;
06026 virtual QCP::Interaction selectionCategory() const Q_DECL_OVERRIDE;
06027 virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
06028 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
06029 // events:
06030 virtual void selectEvent(QMouseEvent *event, bool additive, const QVariant &details, bool
*selectionStateChanged) Q_DECL_OVERRIDE;
06031 virtual void deselectEvent(bool *selectionStateChanged) Q_DECL_OVERRIDE;
06032
06033 // non-virtual methods:
06034 QPen getBorderPen() const;
06035 QBrush getBrush() const;
06036
06037 private:
06038 Q_DISABLE_COPY(QCPLegend)
06039
06040 friend class QCustomPlot;
06041 friend class QCPAbstractLegendItem;
06042 };
06043
06044 Q_DECLARE_OPERATORS_FOR_FLAGS(QCPLegend::SelectableParts)
06045 Q_DECLARE_METATYPE(QCPLegend::SelectablePart)
06046
06047 /* end of 'src/layoutelements/layoutelement-legend.h' */
06048
06049
06050 /* including file 'src/layoutelements/layoutelement-textelement.h' */
06051 /* modified 2021-03-29T02:30:44, size 5359 */
06052
06053 class QCP_LIB_DECL QCPTextElement : public QCPLayoutElement {
06054 Q_OBJECT
06055 Q_PROPERTY(QString text READ text WRITE setText)
06056 Q_PROPERTY(QFont font READ font WRITE setFont)
06057 Q_PROPERTY(QColor textColor READ textColor WRITE setTextColor)
06058 Q_PROPERTY(QFont selectedFont READ selectedFont WRITE setSelectedFont)
06059 Q_PROPERTY(QColor selectedTextColor READ selectedTextColor WRITE setSelectedTextColor)
06060 Q_PROPERTY(bool selectable READ selectable WRITE setSelectable NOTIFY selectableChanged)
06061 Q_PROPERTY(bool selected READ selected WRITE setSelected NOTIFY selectionChanged)
06062 public:
06063 explicit QCPTextElement(QCustomPlot *parentPlot);
06064 QCPTextElement(QCustomPlot *parentPlot, const QString &text);
06065 QCPTextElement(QCustomPlot *parentPlot, const QString &text, double pointSize);
06066 QCPTextElement(QCustomPlot *parentPlot, const QString &text, const QString &fontFamily, double
pointSize);
06067 QCPTextElement(QCustomPlot *parentPlot, const QString &text, const QFont &font);
06068
06069 // getters:
06070
06071 QString text() const {
06072 return mText;
06073 }
06074
06075
06076

```



```

06077 int textFlags() const {
06078 return mTextFlags;
06079 }
06080
06081 QFont font() const {
06082 return mFont;
06083 }
06084
06085 QColor textColor() const {
06086 return mTextColor;
06087 }
06088
06089 QFont selectedFont() const {
06090 return mSelectedFont;
06091 }
06092
06093 QColor selectedTextColor() const {
06094 return mSelectedTextColor;
06095 }
06096
06097 bool selectable() const {
06098 return mSelectable;
06099 }
06100
06101 bool selected() const {
06102 return mSelected;
06103 }
06104
06105 // setters:
06106 void setText(const QString &text);
06107 void setTextFlags(int flags);
06108 void setFont(const QFont &font);
06109 void setTextColor(const QColor &color);
06110 void setSelectedFont(const QFont &font);
06111 void setSelectedTextColor(const QColor &color);
06112 Q_SLOT void setSelectable(bool selectable);
06113 Q_SLOT void setSelected(bool selected);
06114
06115 // reimplemented virtual methods:
06116 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
06117 virtual void mousePressEvent(QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE;
06118 virtual void mouseReleaseEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
06119 virtual void mouseDoubleClickEvent(QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE;
06120
06121 signals:
06122 void selectionChanged(bool selected);
06123 void selectableChanged(bool selectable);
06124 void clicked(QMouseEvent *event);
06125 void doubleClicked(QMouseEvent *event);
06126
06127 protected:
06128 // property members:
06129 QString mText;
06130 int mTextFlags;
06131 QFont mFont;
06132 QColor mTextColor;
06133 QFont mSelectedFont;
06134 QColor mSelectedTextColor;
06135 QRect mTextBoundingRect;
06136 bool mSelectable, mSelected;
06137
06138 // reimplemented virtual methods:
06139 virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
06140 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
06141 virtual QSize minimumOuterSizeHint() const Q_DECL_OVERRIDE;
06142 virtual QSize maximumOuterSizeHint() const Q_DECL_OVERRIDE;
06143 // events:
06144 virtual void selectEvent(QMouseEvent *event, bool additive, const QVariant &details, bool
*selectionStateChanged) Q_DECL_OVERRIDE;
06145 virtual void deselectEvent(bool *selectionStateChanged) Q_DECL_OVERRIDE;
06146
06147 // non-virtual methods:
06148 QFont mainFont() const;
06149 QColor mainTextColor() const;
06150
06151 private:
06152 Q_DISABLE_COPY(QCPLTextElement)
06153 };
06154
06155
06156
06157 /* end of 'src/layoutelements/layoutelement-textelement.h' */
06158
06159
06160 /* including file 'src/layoutelements/layoutelement-colorscale.h' */
06161

```

```

06162 /* modified 2021-03-29T02:30:44, size 5939 */
06163
06164
06165 class QCPColorScaleAxisRectPrivate : public QCPAxisRect {
06166 Q_OBJECT
06167 public:
06168 explicit QCPColorScaleAxisRectPrivate(QCPColorScale *parentColorScale);
06169 protected:
06170 QCPColorScale *mParentColorScale;
06171 QImage mGradientImage;
06172 bool mGradientImageInvalidated;
06173 // re-using some methods of QCPAxisRect to make them available to friend class QCPColorScale
06174 using QCPAxisRect::calculateAutoMargin;
06175 using QCPAxisRect::mousePressEvent;
06176 using QCPAxisRect::mouseMoveEvent;
06177 using QCPAxisRect::mouseReleaseEvent;
06178 using QCPAxisRect::wheelEvent;
06179 using QCPAxisRect::update;
06180 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
06181 void updateGradientImage();
06182 Q_SLOT void axisSelectionChanged(QCPAxis::SelectableParts selectedParts);
06183 Q_SLOT void axisSelectableChanged(QCPAxis::SelectableParts selectableParts);
06184 friend class QCPColorScale;
06185 };
06186
06187 class QCP_LIB_DECL QCPColorScale : public QCPLayoutElement {
06188 Q_OBJECT
06189 Q_PROPERTY(QCPAxis::AxisType type READ type WRITE setType)
06190 Q_PROPERTY(QCPRange dataRange READ dataRange WRITE setDataRange NOTIFY dataRangeChanged)
06191 Q_PROPERTY(QCPAxis::ScaleType dataScaleType READ dataScaleType WRITE setDataScaleType NOTIFY
06192 dataScaleTypeChanged)
06193 Q_PROPERTY(QCPColorGradient gradient READ gradient WRITE setGradient NOTIFY gradientChanged)
06194 Q_PROPERTY(QString label READ label WRITE setLabel)
06195 Q_PROPERTY(int barWidth READ barWidth WRITE setBarWidth)
06196 Q_PROPERTY(bool rangeDrag READ rangeDrag WRITE setRangeDrag)
06197 Q_PROPERTY(bool rangeZoom READ rangeZoom WRITE setRangeZoom)
06199 public:
06200 explicit QCPColorScale(QCustomPlot *parentPlot);
06201 virtual ~QCPColorScale() Q_DECL_OVERRIDE;
06202
06203 // getters:
06204
06205 QCPAxis *axis() const {
06206 return mColorAxis.data();
06207 }
06208
06209 QCPAxis::AxisType type() const {
06210 return mType;
06211 }
06212
06213 QCPRange dataRange() const {
06214 return mDataRange;
06215 }
06216
06217 QCPAxis::ScaleType dataScaleType() const {
06218 return mDataScaleType;
06219 }
06220
06221 QCPColorGradient gradient() const {
06222 return mGradient;
06223 }
06224 QString label() const;
06225
06226 int barWidth() const {
06227 return mBarWidth;
06228 }
06229 bool rangeDrag() const;
06230 bool rangeZoom() const;
06231
06232 // setters:
06233 void setType(QCPAxis::AxisType type);
06234 Q_SLOT void setDataRange(const QCPRange &dataRange);
06235 Q_SLOT void setDataScaleType(QCPAxis::ScaleType scaleType);
06236 Q_SLOT void setGradient(const QCPColorGradient &gradient);
06237 void setLabel(const QString &str);
06238 void setBarWidth(int width);
06239 void setRangeDrag(bool enabled);
06240 void setRangeZoom(bool enabled);
06241
06242 // non-property methods:
06243 QList<QCPColorMap*> colorMaps() const;
06244 void rescaleDataRange(bool onlyVisibleMaps);
06245
06246 // reimplemented virtual methods:
06247 virtual void update(UpdatePhase phase) Q_DECL_OVERRIDE;
06248
06249 signals:

```

```

06250 void dataRangeChanged(const QCPRange &newRange);
06251 void dataScaleTypeChanged(QCPAxis::ScaleType scaleType);
06252 void gradientChanged(const QCPColorGradient &newGradient);
06253
06254 protected:
06255 // property members:
06256 QCPAxis::AxisType mType;
06257 QCPRange mDataRange;
06258 QCPAxis::ScaleType mDataScaleType;
06259 QCPColorGradient mGradient;
06260 int mBarWidth;
06261
06262 // non-property members:
06263 QPointer<QCPColorScaleAxisRectPrivate> mAxisRect;
06264 QPointer<QCPAxis> mColorAxis;
06265
06266 // reimplemented virtual methods:
06267 virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
06268 // events:
06269 virtual void mousePressEvent(QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE;
06270 virtual void mouseMoveEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
06271 virtual void mouseReleaseEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
06272 virtual void wheelEvent(QWheelEvent *event) Q_DECL_OVERRIDE;
06273
06274 private:
06275 Q_DISABLE_COPY(QCPColorScale)
06276
06277 friend class QCPColorScaleAxisRectPrivate;
06278 };
06279
06280
06281 /* end of 'src/layoutelements/layoutelement-colorscale.h' */
06282
06283
06284 /* including file 'src/plottables/plottable-graph.h' */
06285
06286 /* modified 2021-03-29T02:30:44, size 9316 */
06287
06288 class QCP_LIB_DECL QCPGraphData {
06289 public:
06290 QCPGraphData();
06291 QCPGraphData(double key, double value);
06292
06293 inline double sortKey() const {
06294 return key;
06295 }
06296
06297 inline static QCPGraphData fromSortKey(double sortKey) {
06298 return QCPGraphData(sortKey, 0);
06299 }
06300
06301 inline static bool sortKeyIsMainKey() {
06302 return true;
06303 }
06304
06305 inline double mainKey() const {
06306 return key;
06307 }
06308
06309 inline double mainValue() const {
06310 return value;
06311 }
06312
06313 inline QCPRange valueRange() const {
06314 return QCPRange(value, value);
06315 }
06316
06317 double key, value;
06318 };
06319 Q_DECLARE_TYPEINFO(QCPGraphData, Q_PRIMITIVE_TYPE);
06320
06321 typedef QCPDataContainer<QCPGraphData> QCPGraphDataContainer;
06322
06323 class QCP_LIB_DECL QCPGraph : public QCPAbstractPlottable1D<QCPGraphData> {
06324
06325 Q_OBJECT
06326 Q_PROPERTY(LineStyle lineStyle READ lineStyle WRITE setLineStyle)
06327 Q_PROPERTY(QCPScatterStyle scatterStyle READ scatterStyle WRITE setScatterStyle)
06328 Q_PROPERTY(int scatterSkip READ scatterSkip WRITE setScatterSkip)
06329 Q_PROPERTY(QCPGraph* channelFillGraph READ channelFillGraph WRITE setChannelFillGraph)
06330 Q_PROPERTY(bool adaptiveSampling READ adaptiveSampling WRITE setAdaptiveSampling)
06331 public:
06332 enum LineStyle {
06333 lsNone
06334 , lsLine
06335 , lsStepLeft

```

```

06354 , lsStepRight
06355 , lsStepCenter
06356 , lsImpulse
06357 };
06358 Q_ENUMS(LineStyle)
06359
06360 explicit QCPGraph(QCPAxis *keyAxis, QCPAxis *valueAxis);
06361 virtual ~QCPGraph() Q_DECL_OVERRIDE;
06362
06363 // getters:
06364
06365 QSharedPointer<QCPGraphDataContainer> data() const {
06366 return mDataContainer;
06367 }
06368
06369 LineStyle lineStyle() const {
06370 return mLineStyle;
06371 }
06372
06373 QCPScatterStyle scatterStyle() const {
06374 return mScatterStyle;
06375 }
06376
06377 int scatterSkip() const {
06378 return mScatterSkip;
06379 }
06380
06381 QCPGraph *channelFillGraph() const {
06382 return mChannelFillGraph.data();
06383 }
06384
06385 bool adaptiveSampling() const {
06386 return mAdaptiveSampling;
06387 }
06388
06389 // setters:
06390 void setData(QSharedPointer<QCPGraphDataContainer> data);
06391 void setData(const QVector<double> &keys, const QVector<double> &values, bool alreadySorted =
false);
06392 void setLineStyle(LineStyle ls);
06393 void setScatterStyle(const QCPScatterStyle &style);
06394 void setScatterSkip(int skip);
06395 void setChannelFillGraph(QCPGraph *targetGraph);
06396 void setAdaptiveSampling(bool enabled);
06397
06398 // non-property methods:
06399 void addData(const QVector<double> &keys, const QVector<double> &values, bool alreadySorted =
false);
06400 void addData(double key, double value);
06401
06402 // reimplemented virtual methods:
06403 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
06404 virtual QCPRange getKeyRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth) const
Q_DECL_OVERRIDE;
06405 virtual QCPRange getValueRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth, const
QCPRange &inKeyRange = QCPRange()) const Q_DECL_OVERRIDE;
06406
06407 protected:
06408 // property members:
06409 LineStyle mLineStyle;
06410 QCPScatterStyle mScatterStyle;
06411 int mScatterSkip;
06412 QPointer<QCPGraph> mChannelFillGraph;
06413 bool mAdaptiveSampling;
06414
06415 // reimplemented virtual methods:
06416 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
06417 virtual void drawLegendIcon(QCPPainter *painter, const QRectF &rect) const Q_DECL_OVERRIDE;
06418
06419 // introduced virtual methods:
06420 virtual void drawFill(QCPPainter *painter, QVector<QPointF> *lines) const;
06421 virtual void drawScatterPlot(QCPPainter *painter, const QVector<QPointF> &scatters, const
QCPScatterStyle &style) const;
06422 virtual void drawLinePlot(QCPPainter *painter, const QVector<QPointF> &lines) const;
06423 virtual void drawImpulsePlot(QCPPainter *painter, const QVector<QPointF> &lines) const;
06424
06425 virtual void getOptimizedLineData(QVector<QCPGraphData> *lineData, const
QCPGraphDataContainer::const_iterator &begin, const QCPGraphDataContainer::const_iterator &end) const;
06426 virtual void getOptimizedScatterData(QVector<QCPGraphData> *scatterData,
QCPGraphDataContainer::const_iterator begin, QCPGraphDataContainer::const_iterator end) const;
06427
06428 // non-virtual methods:
06429 void getVisibleDataBounds(QCPGraphDataContainer::const_iterator &begin,
QCPGraphDataContainer::const_iterator &end, const QCPDataRange &rangeRestriction) const;
06430 void getLines(QVector<QPointF> *lines, const QCPDataRange &dataRange) const;
06431 void getScatters(QVector<QPointF> *scatters, const QCPDataRange &dataRange) const;

```

```

06432 QVector<QPointF> dataToLines(const QVector<QCPGraphData> &data) const;
06433 QVector<QPointF> dataToStepLeftLines(const QVector<QCPGraphData> &data) const;
06434 QVector<QPointF> dataToStepRightLines(const QVector<QCPGraphData> &data) const;
06435 QVector<QPointF> dataToStepCenterLines(const QVector<QCPGraphData> &data) const;
06436 QVector<QPointF> dataToImpulseLines(const QVector<QCPGraphData> &data) const;
06437 QVector<QCPDataRange> getNonNanSegments(const QVector<QPointF> *lineData, Qt::Orientation
keyOrientation) const;
06438 QVector<QPair<QCPDataRange, QCPDataRange> > getOverlappingSegments(QVector<QCPDataRange>
thisSegments, const QVector<QPointF> *thisData, QVector<QCPDataRange> otherSegments, const
VVector<QPointF> *otherData) const;
06439 bool segmentsIntersect(double aLower, double aUpper, double bLower, double bUpper, int
&bPrecedence) const;
06440 QPointF getFillBasePoint(QPointF matchingDataPoint) const;
06441 const QPolygonF getFillPolygon(const QVector<QPointF> *lineData, QCPDataRange segment) const;
06442 const QPolygonF getChannelFillPolygon(const QVector<QPointF> *thisData, QCPDataRange thisSegment,
const QVector<QPointF> *otherData, QCPDataRange otherSegment) const;
06443 int findIndexBelowX(const QVector<QPointF> *data, double x) const;
06444 int findIndexAboveX(const QVector<QPointF> *data, double x) const;
06445 int findIndexBelowY(const QVector<QPointF> *data, double y) const;
06446 int findIndexAboveY(const QVector<QPointF> *data, double y) const;
06447 double pointDistance(const QPointF &pixelPoint, QCPGraphDataContainer::const_iterator
&closestData) const;
06448
06449 friend class QCustomPlot;
06450 friend class QCPLegend;
06451 };
06452
06453 Q_DECLARE_METATYPE(QCPGraph::LineStyle)
06454
06455 /* end of 'src/plottables/plottable-graph.h' */
06456
06457
06458 /* including file 'src/plottables/plottable-curve.h' */
06459 /* modified 2021-03-29T02:30:44, size 7434 */
06460
06461 class QCP_LIB_DECL QCPCurveData {
06462 public:
06463 QCPCurveData();
06464 QCPCurveData(double t, double key, double value);
06465
06466 inline double sortKey() const {
06467 return t;
06468 }
06469
06470 inline static QCPCurveData fromSortKey(double sortKey) {
06471 return QCPCurveData(sortKey, 0, 0);
06472 }
06473
06474 inline static bool sortKeyIsMainKey() {
06475 return false;
06476 }
06477
06478 inline double mainKey() const {
06479 return key;
06480 }
06481
06482 inline double mainValue() const {
06483 return value;
06484 }
06485
06486 inline QCPRange valueRange() const {
06487 return QCPRange(value, value);
06488 }
06489
06490 double t, key, value;
06491 };
06492 Q_DECLARE_TYPEINFO(QCPCurveData, Q_PRIMITIVE_TYPE);
06493
06494
06495 typedef QCPDataContainer<QCPCurveData> QCPCurveDataContainer;
06506
06507 class QCP_LIB_DECL QCPCurve : public QCPAbstractPlottable1D<QCPCurveData> {
06508
06509 Q_OBJECT
06511 Q_PROPERTY(QCPScatterStyle scatterStyle READ scatterStyle WRITE setScatterStyle)
06512 Q_PROPERTY(int scatterSkip READ scatterSkip WRITE setScatterSkip)
06513 Q_PROPERTY(LineStyle lineStyle READ lineStyle WRITE setLineStyle)
06515 public:
06521 enum LineStyle {
06522 lsNone
06523 , lsLine
06524 };
06525 Q_ENUMS(LineStyle)
06526
06527 explicit QCPCurve(QCPAxis *keyAxis, QCPAxis *valueAxis);
06528 virtual ~QCPCurve() Q_DECL_OVERRIDE;
06529

```

```

06530 // getters:
06531
06532 QSharedPointer<QCPCurveDataContainer> data() const {
06533 return mDataContainer;
06534 }
06535
06536 QCPScatterStyle scatterStyle() const {
06537 return mScatterStyle;
06538 }
06539
06540 int scatterSkip() const {
06541 return mScatterSkip;
06542 }
06543
06544 LineStyle lineStyle() const {
06545 return mLineStyle;
06546 }
06547
06548 // setters:
06549 void setData(QSharedPointer<QCPCurveDataContainer> data);
06550 void setData(const QVector<double> &t, const QVector<double> &keys, const QVector<double> &values,
06551 bool alreadySorted = false);
06552 void setData(const QVector<double> &keys, const QVector<double> &values);
06553 void setScatterStyle(const QCPScatterStyle &style);
06554 void setScatterSkip(int skip);
06555 void setLineStyle(LineStyle style);
06556
06557 // non-property methods:
06558 void addData(const QVector<double> &t, const QVector<double> &keys, const QVector<double> &values,
06559 bool alreadySorted = false);
06560 void addData(const QVector<double> &keys, const QVector<double> &values);
06561 void addData(double t, double key, double value);
06562 void addData(double key, double value);
06563
06564 // reimplemented virtual methods:
06565 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
06566 const Q_DECL_OVERRIDE;
06567 virtual QCPRange getKeyRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth) const
06568 Q_DECL_OVERRIDE;
06569 virtual QCPRange getValueRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth, const
06570 QCPRange &inKeyRange = QCPRange()) const Q_DECL_OVERRIDE;
06571
06572 protected:
06573 // property members:
06574 QCPScatterStyle mScatterStyle;
06575 int mScatterSkip;
06576 LineStyle mLineStyle;
06577
06578 // reimplemented virtual methods:
06579 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
06580 virtual void drawLegendIcon(QCPPainter *painter, const QRectF &rect) const Q_DECL_OVERRIDE;
06581
06582 // introduced virtual methods:
06583 virtual void drawCurveLine(QCPPainter *painter, const QVector<QPointF> &lines) const;
06584 virtual void drawScatterPlot(QCPPainter *painter, const QVector<QPointF> &points, const
06585 QCPScatterStyle &style) const;
06586
06587 // non-virtual methods:
06588 void getCurveLines(QVector<QPointF> *lines, const QCPDataRange &dataRange, double penWidth) const;
06589 void getScatters(QVector<QPointF> *scatters, const QCPDataRange &dataRange, double scatterWidth)
06590 const;
06591 int getRegion(double key, double value, double keyMin, double valueMax, double keyMax, double
06592 valueMin) const;
06593 QPointF getOptimizedPoint(int otherRegion, double otherKey, double otherValue, double key, double
06594 value, double keyMin, double valueMax, double keyMax, double valueMin) const;
06595 QVector<QPointF> getOptimizedCornerPoints(int prevRegion, int currentRegion, double prevKey,
06596 double prevValue, double key, double value, double keyMin, double valueMax, double keyMax, double
06597 valueMin) const;
06598 bool mayTraverse(int prevRegion, int currentRegion) const;
06599 bool getTraverse(double prevKey, double prevValue, double key, double value, double keyMin, double
06600 valueMax, double keyMax, double valueMin, QPointF &crossA, QPointF &crossB) const;
06601 void getTraverseCornerPoints(int prevRegion, int currentRegion, double keyMin, double valueMax,
06602 double keyMax, double valueMin, QVector<QPointF> &beforeTraverse, QVector<QPointF> &afterTraverse)
06603 const;
06604 double pointDistance(const QPointF &pixelPoint, QCPCurveDataContainer::const_iterator
06605 &closestData) const;
06606
06607 friend class QCustomPlot;
06608 friend class QCPLegend;
06609 };
06610
06611 Q_DECLARE_METATYPE(QCPCurve::LineStyle)
06612
06613 /* end of 'src/plottables/plottable-curve.h' */
06614
06615 /* including file 'src/plottables/plottable-bars.h' */

```

```

06602 /* modified 2021-03-29T02:30:44, size 8955 */
06603
06604 class QCP_LIB_DECL QCPBarsGroup : public QObject {
06605
06606 Q_OBJECT
06607 Q_PROPERTY(SpacingType spacingType READ spacingType WRITE setSpacingType)
06608 Q_PROPERTY(double spacing READ spacing WRITE setSpacing)
06609 public:
06610 enum SpacingType {
06611 stAbsolute
06612 , stAxisRectRatio
06613 , stPlotCoords
06614 };
06615 Q_ENUMS(SpacingType)
06616
06617 explicit QCPBarsGroup(QCustomPlot *parentPlot);
06618 virtual ~QCPBarsGroup();
06619
06620 // getters:
06621
06622 SpacingType spacingType() const {
06623 return mSpacingType;
06624 }
06625
06626 double spacing() const {
06627 return mSpacing;
06628 }
06629
06630 // setters:
06631 void setSpacingType(SpacingType spacingType);
06632 void setSpacing(double spacing);
06633
06634 // non-virtual methods:
06635
06636 QList<QCPBars*> bars() const {
06637 return mBars;
06638 }
06639 QCPBars* bars(int index) const;
06640
06641 int size() const {
06642 return mBars.size();
06643 }
06644
06645 bool isEmpty() const {
06646 return mBars.isEmpty();
06647 }
06648 void clear();
06649
06650 bool contains(QCPBars *bars) const {
06651 return mBars.contains(bars);
06652 }
06653 void append(QCPBars *bars);
06654 void insert(int i, QCPBars *bars);
06655 void remove(QCPBars *bars);
06656
06657 protected:
06658 // non-property members:
06659 QCustomPlot *mParentPlot;
06660 SpacingType mSpacingType;
06661 double mSpacing;
06662 QList<QCPBars*> mBars;
06663
06664 // non-virtual methods:
06665 void registerBars(QCPBars *bars);
06666 void unregisterBars(QCPBars *bars);
06667
06668 // virtual methods:
06669 double keyPixelOffset(const QCPBars *bars, double keyCoord);
06670 double getPixelSpacing(const QCPBars *bars, double keyCoord);
06671
06672 private:
06673 Q_DISABLE_COPY(QCPBarsGroup)
06674
06675 friend class QCPBars;
06676 };
06677
06678 Q_DECLARE_METATYPE(QCPBarsGroup::SpacingType)
06679
06680 class QCP_LIB_DECL QCPBarsData {
06681 public:
06682 QCPBarsData();
06683 QCPBarsData(double key, double value);
06684
06685 inline double sortKey() const {
06686 return key;
06687 }
06688 }

```

```

06697
06698 inline static QCPBarsData fromSortKey(double sortKey) {
06699 return QCPBarsData(sortKey, 0);
06700 }
06701
06702 inline static bool sortKeyIsMainKey() {
06703 return true;
06704 }
06705
06706 inline double mainKey() const {
06707 return key;
06708 }
06709
06710 inline double mainValue() const {
06711 return value;
06712 }
06713
06714 inline QCPRange valueRange() const {
06715 return QCPRange(value, value);
06716 } // note that bar base value isn't held in each QCPBarsData and thus can't/shouldn't be returned
 here
06717
06718 double key, value;
06719 };
06720 Q_DECLARE_TYPEINFO(QCPBarsData, Q_PRIMITIVE_TYPE);
06721
06722
06723 typedef QCPDataContainer<QCPBarsData> QCPBarsDataContainer;
06724
06725 class QCP_LIB_DECL QCPBars : public QCPAbstractPlottable1D<QCPBarsData> {
06726
06727 Q_OBJECT
06728 Q_PROPERTY(double width READ width WRITE setWidth)
06729 Q_PROPERTY(WidthType widthType READ widthType WRITE setWidthType)
06730 Q_PROPERTY(QCPBarsGroup* barsGroup READ barsGroup WRITE setBarsGroup)
06731 Q_PROPERTY(double baseValue READ baseValue WRITE setBaseValue)
06732 Q_PROPERTY(double stackingGap READ stackingGap WRITE setStackingGap)
06733 Q_PROPERTY(QCPBars* barBelow READ barBelow)
06734 Q_PROPERTY(QCPBars* barAbove READ barAbove)
06735 public:
06736 enum WidthType {
06737 wtAbsolute
06738 , wtAxisRectRatio
06739 , wtPlotCoords
06740 };
06741 Q_ENUMS(WidthType)
06742
06743 explicit QCPBars(QCPAxis *keyAxis, QCPAxis *valueAxis);
06744 virtual ~QCPBars() Q_DECL_OVERRIDE;
06745
06746 // getters:
06747
06748 double width() const {
06749 return mWidth;
06750 }
06751
06752 WidthType widthType() const {
06753 return mWidthType;
06754 }
06755
06756 QCPBarsGroup *barsGroup() const {
06757 return mBarsGroup;
06758 }
06759
06760 double baseValue() const {
06761 return mBaseValue;
06762 }
06763
06764 double stackingGap() const {
06765 return mStackingGap;
06766 }
06767
06768 QCPBars *barBelow() const {
06769 return mBarBelow.data();
06770 }
06771
06772 QCPBars *barAbove() const {
06773 return mBarAbove.data();
06774 }
06775
06776 QSharedPointer<QCPBarsDataContainer> data() const {
06777 return mDataContainer;
06778 }
06779
06780 // setters:
06781 void setData(QSharedPointer<QCPBarsDataContainer> data);
06782 void setData(const QVector<double> &keys, const QVector<double> &values, bool alreadySorted =

```



```

 false);
06800 void setWidth(double width);
06801 void setWidthType(WidthType widthType);
06802 void setBarsGroup(QCPBarsGroup *barsGroup);
06803 void setBaseValue(double baseValue);
06804 void setStackingGap(double pixels);
06805
06806 // non-property methods:
06807 void addData(const QVector<double> &keys, const QVector<double> &values, bool alreadySorted =
false);
06808 void addData(double key, double value);
06809 void moveBelow(QCPBars *bars);
06810 void moveAbove(QCPBars *bars);
06811
06812 // reimplemented virtual methods:
06813 virtual QCPDataSelection selectTestRect(const QRectF &rect, bool onlySelectable) const
Q_DECL_OVERRIDE;
06814 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
06815 virtual QCPRange getKeyRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth) const
Q_DECL_OVERRIDE;
06816 virtual QCPRange getValueRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth, const
QCPRange &inKeyRange = QCPRange()) const Q_DECL_OVERRIDE;
06817 virtual QPointF dataPixelPosition(int index) const Q_DECL_OVERRIDE;
06818
06819 protected:
06820 // property members:
06821 double mWidth;
06822 WidthType mWidthType;
06823 QCPBarsGroup *mBarsGroup;
06824 double mBaseValue;
06825 double mStackingGap;
06826 QPointer<QCPBars> mBarBelow, mBarAbove;
06827
06828 // reimplemented virtual methods:
06829 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
06830 virtual void drawLegendIcon(QCPPainter *painter, const QRectF &rect) const Q_DECL_OVERRIDE;
06831
06832 // non-virtual methods:
06833 void getVisibleDataBounds(QCPBarsDataContainer::const_iterator &begin,
QCPBarsDataContainer::const_iterator &end) const;
06834 QRectF getBarRect(double key, double value) const;
06835 void getPixelWidth(double key, double &lower, double &upper) const;
06836 double getStackedBaseValue(double key, bool positive) const;
06837 static void connectBars(QCPBars* lower, QCPBars* upper);
06838
06839 friend class QCustomPlot;
06840 friend class QCPLegend;
06841 friend class QCPBarsGroup;
06842 };
06843
06844 Q_DECLARE_METATYPE(QCPBars::WidthType)
06845
06846 /* end of 'src/plottables/plottable-bars.h' */
06847
06848
06849 /* including file 'src/plottables/plottable-statisticalbox.h' */
06850 /* modified 2021-03-29T02:30:44, size 7522 */
06851
06852 class QCP_LIB_DECL QCPStatisticalBoxData {
06853 public:
06854 QCPStatisticalBoxData();
06855 QCPStatisticalBoxData(double key, double minimum, double lowerQuartile, double median, double
upperQuartile, double maximum, const QVector<double>& outliers = QVector<double>());
06856
06857 inline double sortKey() const {
06858 return key;
06859 }
06860
06861 inline static QCPStatisticalBoxData fromSortKey(double sortKey) {
06862 return QCPStatisticalBoxData(sortKey, 0, 0, 0, 0, 0, 0);
06863 }
06864
06865 inline static bool sortKeyIsMainKey() {
06866 return true;
06867 }
06868
06869 inline double mainKey() const {
06870 return key;
06871 }
06872
06873 inline double mainValue() const {
06874 return median;
06875 }
06876
06877 inline QCPRange valueRange() const {
06878 QCPRange result(minimum, maximum);

```

```

06879 for (QVector<double>::const_iterator it = outliers.constBegin(); it != outliers.constEnd();
++it)
06880 result.expand(*it);
06881 return result;
06882 }
06883
06884 double key, minimum, lowerQuartile, median, upperQuartile, maximum;
06885 QVector<double> outliers;
06886 };
06887 Q_DECLARE_TYPEINFO(QCPStatisticalBoxData, Q_MOVABLE_TYPE);
06888
06889
06900 typedef QCPDataContainer<QCPStatisticalBoxData> QCPStatisticalBoxDataContainer;
06901
06902 class QCP_LIB_DECL QCPStatisticalBox : public QCPAbstractPlottable1D<QCPStatisticalBoxData> {
06903 Q_OBJECT
06904 Q_PROPERTY(double width READ width WRITE setWidth)
06905 Q_PROPERTY(double whiskerWidth READ whiskerWidth WRITE setWhiskerWidth)
06906 Q_PROPERTY(QPen whiskerPen READ whiskerPen WRITE setWhiskerPen)
06907 Q_PROPERTY(QPen whiskerBarPen READ whiskerBarPen WRITE setWhiskerBarPen)
06908 Q_PROPERTY(bool whiskerAntialiased READ whiskerAntialiased WRITE setWhiskerAntialiased)
06909 Q_PROPERTY(QPen medianPen READ medianPen WRITE setMedianPen)
06910 Q_PROPERTY(QCPScatterStyle outlierStyle READ outlierStyle WRITE setOutlierStyle)
06911 public:
06912 explicit QCPStatisticalBox(QCPAxis *keyAxis, QCPAxis *valueAxis);
06913
06914 // getters:
06915
06916 QSharedPointer<QCPStatisticalBoxDataContainer> data() const {
06917 return mDataContainer;
06918 }
06919
06920 double width() const {
06921 return mWidth;
06922 }
06923
06924 double whiskerWidth() const {
06925 return mWhiskerWidth;
06926 }
06927
06928 QPen whiskerPen() const {
06929 return mWhiskerPen;
06930 }
06931
06932 QPen whiskerBarPen() const {
06933 return mWhiskerBarPen;
06934 }
06935
06936 bool whiskerAntialiased() const {
06937 return mWhiskerAntialiased;
06938 }
06939
06940 QPen medianPen() const {
06941 return mMedianPen;
06942 }
06943
06944 QCPScatterStyle outlierStyle() const {
06945 return mOutlierStyle;
06946 }
06947
06948 // setters:
06949 void setData(QSharedPointer<QCPStatisticalBoxDataContainer> data);
06950 void setData(const QVector<double> &keys, const QVector<double> &minimum, const QVector<double>
&lowerQuartile, const QVector<double> &median, const QVector<double> &upperQuartile, const
QVector<double> &maximum, bool alreadySorted = false);
06951 void setWidth(double width);
06952 void setWhiskerWidth(double width);
06953 void setWhiskerPen(const QPen &pen);
06954 void setWhiskerBarPen(const QPen &pen);
06955 void setWhiskerAntialiased(bool enabled);
06956 void setMedianPen(const QPen &pen);
06957 void setOutlierStyle(const QCPScatterStyle &style);
06958
06959 // non-property methods:
06960 void addData(const QVector<double> &keys, const QVector<double> &minimum, const QVector<double>
&lowerQuartile, const QVector<double> &median, const QVector<double> &upperQuartile, const
QVector<double> &maximum, bool alreadySorted = false);
06961 void addData(double key, double minimum, double lowerQuartile, double median, double
upperQuartile, double maximum, const QVector<double> &outliers = QVector<double>());
06962
06963 // reimplemented virtual methods:
06964 virtual QCPDataSelection selectTestRect(const QRectF &rect, bool onlySelectable) const
Q_DECL_OVERRIDE;
06965 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
06966 virtual QCPRange getKeyRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth) const
Q_DECL_OVERRIDE;

```

```

06969 virtual QCPRange getValueRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth, const
QCPRange &inKeyRange = QCPRange()) const Q_DECL_OVERRIDE;
06970
06971 protected:
06972 // property members:
06973 double mWidth;
06974 double mWhiskerWidth;
06975 QPen mWhiskerPen, mWhiskerBarPen;
06976 bool mWhiskerAntialiased;
06977 QPen mMedianPen;
06978 QCPScatterStyle mOutlierStyle;
06979
06980 // reimplemented virtual methods:
06981 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
06982 virtual void drawLegendIcon(QCPPainter *painter, const QRectF &rect) const Q_DECL_OVERRIDE;
06983
06984 // introduced virtual methods:
06985 virtual void drawStatisticalBox(QCPPainter *painter,
QCPStatisticalBoxDataContainer::const_iterator it, const QCPScatterStyle &outlierStyle) const;
06986
06987 // non-virtual methods:
06988 void getVisibleDataBounds(QCPStatisticalBoxDataContainer::const_iterator &begin,
QCPStatisticalBoxDataContainer::const_iterator &end) const;
06989 QRectF getQuartileBox(QCPStatisticalBoxDataContainer::const_iterator it) const;
06990 QVector<QLineF> getWhiskerBackboneLines(QCPStatisticalBoxDataContainer::const_iterator it) const;
06991 QVector<QLineF> getWhiskerBarLines(QCPStatisticalBoxDataContainer::const_iterator it) const;
06992
06993 friend class QCustomPlot;
06994 friend class QCPLegend;
06995 };
06996
06997 /* end of 'src/plottables/plottable-statisticalbox.h' */
06998
06999
07000 /* including file 'src/plottables/plottable-colormap.h' */
07001
07002 /* modified 2021-03-29T02:30:44, size 7092 */
07003
07004 class QCP_LIB_DECL QCPColorMapData {
07005 public:
07006 QCPColorMapData(int keySize, int valueSize, const QCPRange &keyRange, const QCPRange &valueRange);
07007 ~QCPColorMapData();
07008 QCPColorMapData(const QCPColorMapData &other);
07009 QCPColorMapData &operator=(const QCPColorMapData &other);
07010
07011 // getters:
07012
07013 int keySize() const {
07014 return mKeySize;
07015 }
07016
07017 int valueSize() const {
07018 return mValueSize;
07019 }
07020
07021 QCPRange keyRange() const {
07022 return mKeyRange;
07023 }
07024
07025 QCPRange valueRange() const {
07026 return mValueRange;
07027 }
07028
07029 QCPRange dataBounds() const {
07030 return mDataBounds;
07031 }
07032
07033 double data(double key, double value);
07034 double cell(int keyIndex, int valueIndex);
07035 unsigned char alpha(int keyIndex, int valueIndex);
07036
07037 // setters:
07038 void setSize(int keySize, int valueSize);
07039 void setKeySize(int keySize);
07040 void setValueSize(int valueSize);
07041 void setRange(const QCPRange &keyRange, const QCPRange &valueRange);
07042 void setKeyRange(const QCPRange &keyRange);
07043 void setValueRange(const QCPRange &valueRange);
07044 void setData(double key, double value, double z);
07045 void setCell(int keyIndex, int valueIndex, double z);
07046 void setAlpha(int keyIndex, int valueIndex, unsigned char alpha);
07047
07048 // non-property methods:
07049 void recalculateDataBounds();
07050 void clear();
07051 void clearAlpha();
07052 void fill(double z);
07053 void fillAlpha(unsigned char alpha);

```

```

07053
07054 bool isEmpty() const {
07055 return mIsEmpty;
07056 }
07057 void coordToCell(double key, double value, int *keyIndex, int *valueIndex) const;
07058 void cellToCoord(int keyIndex, int valueIndex, double *key, double *value) const;
07059
07060 protected:
07061 // property members:
07062 int mKeySize, mValueSize;
07063 QCPRange mKeyRange, mValueRange;
07064 bool mIsEmpty;
07065
07066 // non-property members:
07067 double *mData;
07068 unsigned char *mAlpha;
07069 QCPRange mDataBounds;
07070 bool mDataModified;
07071
07072 bool createAlpha(bool initializeOpaque = true);
07073
07074 friend class QCPCColorMap;
07075 };
07076
07077 class QCP_LIB_DECL QCPCColorMap : public QCPAbstractPlottable {
07078 Q_OBJECT
07079 Q_PROPERTY(QCPRange dataRange READ dataRange WRITE setDataRange NOTIFY dataRangeChanged)
07080 Q_PROPERTY(QCPAxis::ScaleType dataScaleType READ dataScaleType WRITE setDataScaleType NOTIFY
07081 dataScaleTypeChanged)
07082 Q_PROPERTY(QCPCColorGradient gradient READ gradient WRITE setGradient NOTIFY gradientChanged)
07083 Q_PROPERTY(bool interpolate READ interpolate WRITE setInterpolate)
07084 Q_PROPERTY(bool tightBoundary READ tightBoundary WRITE setTightBoundary)
07085 Q_PROPERTY(QCPCColorScale* colorScale READ colorScale WRITE setColorScale)
07086 public:
07087 explicit QCPCColorMap(QCPAxis *keyAxis, QCPAxis *valueAxis);
07088 virtual ~QCPCColorMap() Q_DECL_OVERRIDE;
07089
07090 // getters:
07091
07092 QCPCColorMapData *data() const {
07093 return mMapData;
07094 }
07095
07096 QCPRange dataRange() const {
07097 return mDataRange;
07098 }
07099
07100 QCPAxis::ScaleType dataScaleType() const {
07101 return mDataScaleType;
07102 }
07103
07104 bool interpolate() const {
07105 return mInterpolate;
07106 }
07107
07108 bool tightBoundary() const {
07109 return mTightBoundary;
07110 }
07111
07112 QCPCColorGradient gradient() const {
07113 return mGradient;
07114 }
07115
07116 QCPCColorScale *colorScale() const {
07117 return mColorScale.data();
07118 }
07119
07120 // setters:
07121 void setData(QCPCColorMapData *data, bool copy = false);
07122 Q_SLOT void setDataRange(const QCPRange &dataRange);
07123 Q_SLOT void setDataScaleType(QCPAxis::ScaleType scaleType);
07124 Q_SLOT void setGradient(const QCPCColorGradient &gradient);
07125 void setInterpolate(bool enabled);
07126 void setTightBoundary(bool enabled);
07127 void setColorScale(QCPCColorScale *colorScale);
07128
07129 // non-property methods:
07130 void rescaleDataRange(bool recalculateDataBounds = false);
07131 Q_SLOT void updateLegendIcon(Qt::TransformationMode transformMode = Qt::SmoothTransformation,
07132 const QSize &thumbSize = QSize(32, 18));
07133
07134 // reimplemented virtual methods:
07135 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
07136 const Q_DECL_OVERRIDE;
07137 virtual QCPRange getKeyRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth) const
07138 Q_DECL_OVERRIDE;
07139 virtual QCPRange getValueRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth, const

```

```

 QCPRange &inKeyRange = QCPRange()) const Q_DECL_OVERRIDE;
07138
07139 signals:
07140 void dataRangeChanged(const QCPRange &newRange);
07141 void dataScaleTypeChanged(QCPAxis::ScaleType scaleType);
07142 void gradientChanged(const QCPColorGradient &newGradient);
07143
07144 protected:
07145 // property members:
07146 QCPRange mDataRange;
07147 QCPAxis::ScaleType mDataScaleType;
07148 QCPColorMapData *mMapData;
07149 QCPColorGradient mGradient;
07150 bool mInterpolate;
07151 bool mTightBoundary;
07152 QPointer<QCPColorScale> mColorScale;
07153
07154 // non-property members:
07155 QImage mMapImage, mUndersampledMapImage;
07156 QPixmap mLegendIcon;
07157 bool mMapImageInvalidated;
07158
07159 // introduced virtual methods:
07160 virtual void updateMapImage();
07161
07162 // reimplemented virtual methods:
07163 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
07164 virtual void drawLegendIcon(QCPPainter *painter, const QRectF &rect) const Q_DECL_OVERRIDE;
07165
07166 friend class QCustomPlot;
07167 friend class QCPLegend;
07168 };
07169
07170 /* end of 'src/plottables/plottable-colormap.h' */
07171
07172
07173 /* including file 'src/plottables/plottable-financial.h' */
07174
07175 /* modified 2021-03-29T02:30:44, size 8644 */
07176
07177 class QCP_LIB_DECL QCPFinancialData {
07178 public:
07179 QCPFinancialData();
07180 QCPFinancialData(double key, double open, double high, double low, double close);
07181
07182 inline double sortKey() const {
07183 return key;
07184 }
07185
07186 inline static QCPFinancialData fromSortKey(double sortKey) {
07187 return QCPFinancialData(sortKey, 0, 0, 0, 0);
07188 }
07189
07190 inline static bool sortKeyIsMainKey() {
07191 return true;
07192 }
07193
07194 inline double mainKey() const {
07195 return key;
07196 }
07197
07198 inline double mainValue() const {
07199 return open;
07200 }
07201
07202 inline QCPRange valueRange() const {
07203 return QCPRange(low, high);
07204 } // open and close must lie between low and high, so we don't need to check them
07205
07206 double key, open, high, low, close;
07207 };
07208 Q_DECLARE_TYPEINFO(QCPFinancialData, Q_PRIMITIVE_TYPE);
07209
07210
07220 typedef QCPDataContainer<QCPFinancialData> QCPFinancialDataContainer;
07221
07222 class QCP_LIB_DECL QCPFinancial : public QCPAbstractPlottable1D<QCPFinancialData> {
07223 public:
07224 Q_OBJECT
07225 Q_PROPERTY(ChartStyle chartStyle READ chartStyle WRITE setChartStyle)
07226 Q_PROPERTY(double width READ width WRITE setWidth)
07227 Q_PROPERTY(WidthType widthType READ widthType WRITE setWidthType)
07228 Q_PROPERTY(bool twoColored READ twoColored WRITE setTwoColored)
07229 Q_PROPERTY(QBrush brushPositive READ brushPositive WRITE setBrushPositive)
07230 Q_PROPERTY(QBrush brushNegative READ brushNegative WRITE setBrushNegative)
07231 Q_PROPERTY(QPen penPositive READ penPositive WRITE setPenPositive)
07232 Q_PROPERTY(QPen penNegative READ penNegative WRITE setPenNegative)
07233

```

```

07235 public:
07242 enum WidthType {
07243 wtAbsolute
07244 , wtAxisRectRatio
07245 , wtPlotCoords
07246 };
07247
07248 Q_ENUMS(WidthType)
07249
07250 enum ChartStyle {
07255 csOhlc
07256 , csCandlestick
07257 };
07258
07259 Q_ENUMS(ChartStyle)
07260
07261 explicit QCPFinancial(QCPAxis *keyAxis, QCPAxis *valueAxis);
07262 virtual ~QCPFinancial() Q_DECL_OVERRIDE;
07263
07264 // getters:
07265
07266 QSharedPointer<QCPFinancialDataContainer> data() const {
07267 return mDataContainer;
07268 }
07269
07270 ChartStyle chartStyle() const {
07271 return mChartStyle;
07272 }
07273
07274 double width() const {
07275 return mWidth;
07276 }
07277
07278 WidthType widthType() const {
07279 return mWidthType;
07280 }
07281
07282 bool twoColored() const {
07283 return mTwoColored;
07284 }
07285
07286 QBrush brushPositive() const {
07287 return mBrushPositive;
07288 }
07289
07290 QBrush brushNegative() const {
07291 return mBrushNegative;
07292 }
07293
07294 QPen penPositive() const {
07295 return mPenPositive;
07296 }
07297
07298 QPen penNegative() const {
07299 return mPenNegative;
07300 }
07301
07302 // setters:
07303 void setData(QSharedPointer<QCPFinancialDataContainer> data);
07304 void setData(const QVector<double> &keys, const QVector<double> &open, const QVector<double>
&high, const QVector<double> &low, const QVector<double> &close, bool alreadySorted = false);
07305 void setChartStyle(ChartStyle style);
07306 void setWidth(double width);
07307 void setWidthType(WidthType widthType);
07308 void setTwoColored(bool twoColored);
07309 void setBrushPositive(const QBrush &brush);
07310 void setBrushNegative(const QBrush &brush);
07311 void setPenPositive(const QPen &pen);
07312 void setPenNegative(const QPen &pen);
07313
07314 // non-property methods:
07315 void addData(const QVector<double> &keys, const QVector<double> &open, const QVector<double>
&high, const QVector<double> &low, const QVector<double> &close, bool alreadySorted = false);
07316 void addData(double key, double open, double high, double low, double close);
07317
07318 // reimplemented virtual methods:
07319 virtual QCPDataSelection selectTestRect(const QRectF &rect, bool onlySelectable) const
Q_DECL_OVERRIDE;
07320 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
07321 virtual QCPRange getKeyRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth) const
Q_DECL_OVERRIDE;
07322 virtual QCPRange getValueRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth, const
QCPRange &inKeyRange = QCPRange()) const Q_DECL_OVERRIDE;
07323
07324 // static methods:
07325 static QCPFinancialDataContainer timeSeriesToOhlc(const QVector<double> &time, const

```

```

 QVector<double> &value, double timeBinSize, double timeBinOffset = 0);
07326
07327 protected:
07328 // property members:
07329 ChartStyle mChartStyle;
07330 double mWidth;
07331 WidthType mWidthType;
07332 bool mTwoColored;
07333 QBrush mBrushPositive, mBrushNegative;
07334 QPen mPenPositive, mPenNegative;
07335
07336 // reimplemented virtual methods:
07337 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
07338 virtual void drawLegendIcon(QCPPainter *painter, const QRectF &rect) const Q_DECL_OVERRIDE;
07339
07340 // non-virtual methods:
07341 void drawOhlcPlot(QCPPainter *painter, const QCPFinancialDataContainer::const_iterator &begin,
07342 const QCPFinancialDataContainer::const_iterator &end, bool isSelected);
07343 void drawCandlestickPlot(QCPPainter *painter, const QCPFinancialDataContainer::const_iterator
07344 &begin, const QCPFinancialDataContainer::const_iterator &end, bool isSelected);
07345 double getPixelWidth(double key, double keyPixel) const;
07346 double ohlcSelectTest(const QPointF &pos, const QCPFinancialDataContainer::const_iterator &begin,
07347 const QCPFinancialDataContainer::const_iterator &end, QCPFinancialDataContainer::const_iterator
07348 &closestDataPoint) const;
07349 double candlestickSelectTest(const QPointF &pos, const QCPFinancialDataContainer::const_iterator
07350 &begin, const QCPFinancialDataContainer::const_iterator &end,
07351 QCPFinancialDataContainer::const_iterator &closestDataPoint) const;
07352 void setVisibleDataBounds(QCPFinancialDataContainer::const_iterator &begin,
07353 QCPFinancialDataContainer::const_iterator &end) const;
07354 QRectF selectionHitBox(QCPFinancialDataContainer::const_iterator it) const;
07355
07356 friend class QCustomPlot;
07357 friend class QCPLegend;
07358 };
07359
07360 Q_DECLARE_METATYPE(QCPFinancial::ChartStyle)
07361
07362 /* end of 'src/plottables/plottable-financial.h' */
07363
07364 /* including file 'src/plottables/plottable-errorbar.h' */
07365 /* modified 2021-03-29T02:30:44, size 7749 */
07366
07367 class QCP_LIB_DECL QCPErrorsData {
07368 public:
07369 QCPErrorsData();
07370 explicit QCPErrorsData(double error);
07371 QCPErrorsData(double errorMinus, double errorPlus);
07372
07373 double errorMinus, errorPlus;
07374 };
07375 Q_DECLARE_TYPEINFO(QCPErrorsData, Q_PRIMITIVE_TYPE);
07376
07377 typedef QVector<QCPErrorsData> QCPErrorsDataContainer;
07378
07379 class QCP_LIB_DECL QCPErrors : public QCPAbstractPlottable, public QCPPlottableInterface1D {
07380 public:
07381 Q_OBJECT
07382 Q_PROPERTY(QSharedPointer<QCPErrorsDataContainer> data READ data WRITE setData)
07383 Q_PROPERTY(QCPAbstractPlottable* dataPlottable READ dataPlottable WRITE setDataPlottable)
07384 Q_PROPERTY(ErrorType errorType READ errorType WRITE setErrorType)
07385 Q_PROPERTY(double whiskerWidth READ whiskerWidth WRITE setWhiskerWidth)
07386 Q_PROPERTY(double symbolGap READ symbolGap WRITE setSymbolGap)
07387 public:
07388 enum ErrorType {
07389 etKeyError
07390 , etValueError
07391 };
07392 Q_ENUMS(ErrorType)
07393
07394 explicit QCPErrors(QCPAxis *keyAxis, QCPAxis *valueAxis);
07395 virtual ~QCPErrors() Q_DECL_OVERRIDE;
07396 // getters:
07397 QSharedPointer<QCPErrorsDataContainer> data() const {
07398 return mDataContainer;
07399 }
07400
07401 QCPAbstractPlottable *dataPlottable() const {
07402 return mDataPlottable.data();
07403 }
07404
07405 ErrorType errorType() const {
07406 return mErrorType;
07407 }

```

```

07428
07429 double whiskerWidth() const {
07430 return mWhiskerWidth;
07431 }
07432
07433 double symbolGap() const {
07434 return mSymbolGap;
07435 }
07436
07437 // setters:
07438 void setData(QSharedPointer<QCPErrorBarsDataContainer> data);
07439 void setData(const QVector<double> &error);
07440 void setData(const QVector<double> &errorMinus, const QVector<double> &errorPlus);
07441 void setDataPlottable(QCPAbstractPlottable* plottable);
07442 void setErrorType(ErrorType type);
07443 void setWhiskerWidth(double pixels);
07444 void setSymbolGap(double pixels);
07445
07446 // non-property methods:
07447 void addData(const QVector<double> &error);
07448 void addData(const QVector<double> &errorMinus, const QVector<double> &errorPlus);
07449 void addData(double error);
07450 void addData(double errorMinus, double errorPlus);
07451
07452 // virtual methods of 1d plottable interface:
07453 virtual int dataCount() const Q_DECL_OVERRIDE;
07454 virtual double dataMainKey(int index) const Q_DECL_OVERRIDE;
07455 virtual double dataSortKey(int index) const Q_DECL_OVERRIDE;
07456 virtual double dataMainValue(int index) const Q_DECL_OVERRIDE;
07457 virtual QCPRange dataValueRange(int index) const Q_DECL_OVERRIDE;
07458 virtual QPointF dataPixelPosition(int index) const Q_DECL_OVERRIDE;
07459 virtual bool sortKeyIsMainKey() const Q_DECL_OVERRIDE;
07460 virtual QCPDataSelection selectTestRect(const QRectF &rect, bool onlySelectable) const
07461 Q_DECL_OVERRIDE;
07462 virtual int findBegin(double sortKey, bool expandedRange = true) const Q_DECL_OVERRIDE;
07463 virtual int findEnd(double sortKey, bool expandedRange = true) const Q_DECL_OVERRIDE;
07464
07465 // reimplemented virtual methods:
07466 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
07467 const Q_DECL_OVERRIDE;
07468
07469 virtual QCPPlottableInterface1D *interface1D() Q_DECL_OVERRIDE {
07470 return this;
07471 }
07472
07473 protected:
07474 // property members:
07475 QSharedPointer<QCPErrorBarsDataContainer> mDataContainer;
07476 QPointer<QCPAbstractPlottable> mDataPlottable;
07477 ErrorType mErrorType;
07478 double mWhiskerWidth;
07479 double mSymbolGap;
07480
07481 // reimplemented virtual methods:
07482 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
07483 virtual void drawLegendIcon(QCPPainter *painter, const QRectF &rect) const Q_DECL_OVERRIDE;
07484 virtual QCPRange getKeyRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth) const
07485 Q_DECL_OVERRIDE;
07486 virtual QCPRange getValueRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth, const
07487 QCPRange &inKeyRange = QCPRange()) const Q_DECL_OVERRIDE;
07488
07489 // non-virtual methods:
07490 void getErrorBarLines(QCPErrorBarsDataContainer::const_iterator it, QVector<QLineF> &backbones,
07491 QVector<QLineF> &whiskers) const;
07492 void getVisibleDataBounds(QCPErrorBarsDataContainer::const_iterator &begin,
07493 QCPErrorBarsDataContainer::const_iterator &end, const QCPDataRange &rangeRestriction) const;
07494 double pointDistance(const QPointF &pixelPoint, QCPErrorBarsDataContainer::const_iterator
07495 &closestData) const;
07496 // helpers:
07497 void getDataSegments(QList<QCPDataRange> &selectedSegments, QList<QCPDataRange>
07498 &unselectedSegments) const;
07499 bool errorBarVisible(int index) const;
07500 bool rectIntersectsLine(const QRectF &pixelRect, const QLineF &line) const;
07501
07502 friend class QCustomPlot;
07503 friend class QCPLegend;
07504 };
07505
07506 /* end of 'src/plottables/plottable-errorbar.h' */
07507
07508 /* including file 'src/items/item-straightline.h' */
07509
07510 /* modified 2021-03-29T02:30:44, size 3137 */
07511
07512 class QCP_LIB_DECL QCPItemStraightLine : public QCPAbstractItem {
07513 Q_OBJECT

```



```

07508 Q_PROPERTY(QPen pen READ pen WRITE setPen)
07509 Q_PROPERTY(QPen selectedPen READ selectedPen WRITE setSelectedPen)
07511 public:
07512 explicit QCPItemStraightLine(QCustomPlot *parentPlot);
07513 virtual ~QCPItemStraightLine() Q_DECL_OVERRIDE;
07514
07515 // getters:
07516
07517 QPen pen() const {
07518 return mPen;
07519 }
07520
07521 QPen selectedPen() const {
07522 return mSelectedPen;
07523 }
07524
07525 // setters;
07526 void setPen(const QPen &pen);
07527 void setSelectedPen(const QPen &pen);
07528
07529 // reimplemented virtual methods:
07530 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
07531 const Q_DECL_OVERRIDE;
07532
07533 QCPItemPosition * const point1;
07534 QCPItemPosition * const point2;
07535 protected:
07536 // property members:
07537 QPen mPen, mSelectedPen;
07538
07539 // reimplemented virtual methods:
07540 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
07541
07542 // non-virtual methods:
07543 QLineF getRectClippedStraightLine(const QCPVector2D &base, const QCPVector2D &vec, const QRect
07544 &rect) const;
07545 QPen mainPen() const;
07546 };
07547 /* end of 'src/items/item-straightline.h' */
07548
07549
07550 /* including file 'src/items/item-line.h' */
07551
07552 /* modified 2021-03-29T02:30:44, size 3429 */
07553
07554 class QCP_LIB_DECL QCPItemLine : public QCPAbstractItem {
07555 Q_OBJECT
07556 Q_PROPERTY(QPen pen READ pen WRITE setPen)
07557 Q_PROPERTY(QPen selectedPen READ selectedPen WRITE setSelectedPen)
07558 Q_PROPERTY(QCPLineEnding head READ head WRITE setHead)
07559 Q_PROPERTY(QCPLineEnding tail READ tail WRITE setTail)
07560 public:
07561 explicit QCPItemLine(QCustomPlot *parentPlot);
07562 virtual ~QCPItemLine() Q_DECL_OVERRIDE;
07563
07564 // getters:
07565
07566 QPen pen() const {
07567 return mPen;
07568 }
07569
07570 QPen selectedPen() const {
07571 return mSelectedPen;
07572 }
07573
07574 QCPLineEnding head() const {
07575 return mHead;
07576 }
07577
07578 QCPLineEnding tail() const {
07579 return mTail;
07580 }
07581
07582 // setters;
07583 void setPen(const QPen &pen);
07584 void setSelectedPen(const QPen &pen);
07585 void setHead(const QCPLineEnding &head);
07586 void setTail(const QCPLineEnding &tail);
07587
07588 // reimplemented virtual methods:
07589 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
07590 const Q_DECL_OVERRIDE;
07591
07592 QCPItemPosition * const start;
07593 QCPItemPosition * const end;

```

```

07595
07596 protected:
07597 // property members:
07598 QPen mPen, mSelectedPen;
07599 QCPLLineEnding mHead, mTail;
07600
07601 // reimplemented virtual methods:
07602 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
07603
07604 // non-virtual methods:
07605 QLineF getRectClippedLine(const QCPVector2D &start, const QCPVector2D &end, const QRect &rect)
07606 const;
07607 QPen mainPen() const;
07608 };
07609
07610 /* end of 'src/items/item-line.h' */
07611
07612 /* including file 'src/items/item-curve.h' */
07613
07614 /* modified 2021-03-29T02:30:44, size 3401 */
07615
07616 class QCP_LIB_DECL QCPItemCurve : public QCPAbstractItem {
07617 Q_OBJECT
07618 Q_PROPERTY(QPen pen READ pen WRITE setPen)
07619 Q_PROPERTY(QPen selectedPen READ selectedPen WRITE setSelectedPen)
07620 Q_PROPERTY(QCPLLineEnding head READ head WRITE setHead)
07621 Q_PROPERTY(QCPLLineEnding tail READ tail WRITE setTail)
07622 public:
07623 explicit QCPItemCurve(QCustomPlot *parentPlot);
07624 virtual ~QCPItemCurve() Q_DECL_OVERRIDE;
07625
07626 // getters:
07627
07628 QPen pen() const {
07629 return mPen;
07630 }
07631
07632 QPen selectedPen() const {
07633 return mSelectedPen;
07634 }
07635
07636 QCPLLineEnding head() const {
07637 return mHead;
07638 }
07639
07640 QCPLLineEnding tail() const {
07641 return mTail;
07642 }
07643
07644 // setters:
07645 void setPen(const QPen &pen);
07646 void setSelectedPen(const QPen &pen);
07647 void setHead(const QCPLLineEnding &head);
07648 void setTail(const QCPLLineEnding &tail);
07649
07650 // reimplemented virtual methods:
07651 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
07652 const Q_DECL_OVERRIDE;
07653
07654 QCPItemPosition * const start;
07655 QCPItemPosition * const startDir;
07656 QCPItemPosition * const endDir;
07657 QCPItemPosition * const end;
07658
07659 protected:
07660 // property members:
07661 QPen mPen, mSelectedPen;
07662 QCPLLineEnding mHead, mTail;
07663
07664 // reimplemented virtual methods:
07665 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
07666
07667 // non-virtual methods:
07668 QPen mainPen() const;
07669 };
07670
07671 /* end of 'src/items/item-curve.h' */
07672
07673 /* including file 'src/items/item-rect.h' */
07674
07675 /* modified 2021-03-29T02:30:44, size 3710 */
07676
07677 class QCP_LIB_DECL QCPItemRect : public QCPAbstractItem {
07678 Q_OBJECT
07679 Q_PROPERTY(QPen pen READ pen WRITE setPen)

```

```

07683 Q_PROPERTY(QPen selectedPen READ selectedPen WRITE setSelectedPen)
07684 Q_PROPERTY(QBrush brush READ brush WRITE setBrush)
07685 Q_PROPERTY(QBrush selectedBrush READ selectedBrush WRITE setSelectedBrush)
07687 public:
07688 explicit QCPItemRect(QCustomPlot *parentPlot);
07689 virtual ~QCPItemRect() Q_DECL_OVERRIDE;
07690
07691 // getters:
07692
07693 QPen pen() const {
07694 return mPen;
07695 }
07696
07697 QPen selectedPen() const {
07698 return mSelectedPen;
07699 }
07700
07701 QBrush brush() const {
07702 return mBrush;
07703 }
07704
07705 QBrush selectedBrush() const {
07706 return mSelectedBrush;
07707 }
07708
07709 // setters;
07710 void setPen(const QPen &pen);
07711 void setSelectedPen(const QPen &pen);
07712 void setBrush(const QBrush &brush);
07713 void setSelectedBrush(const QBrush &brush);
07714
07715 // reimplemented virtual methods:
07716 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
07717
07718 QCPItemPosition * const topLeft;
07719 QCPItemPosition * const bottomRight;
07720 QCPItemAnchor * const top;
07721 QCPItemAnchor * const topRight;
07722 QCPItemAnchor * const right;
07723 QCPItemAnchor * const bottom;
07724 QCPItemAnchor * const bottomLeft;
07725 QCPItemAnchor * const left;
07726
07727 protected:
07728
07729 enum AnchorIndex {
07730 aiTop, aiTopRight, aiRight, aiBottom, aiBottomLeft, aiLeft
07731 };
07732
07733 // property members:
07734 QPen mPen, mSelectedPen;
07735 QBrush mBrush, mSelectedBrush;
07736
07737 // reimplemented virtual methods:
07738 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
07739 virtual QPointF anchorPixelPosition(int anchorId) const Q_DECL_OVERRIDE;
07740
07741 // non-virtual methods:
07742 QPen mainPen() const;
07743 QBrush mainBrush() const;
07744 };
07745
07746 /* end of 'src/items/item-rect.h' */
07747
07748
07749 /* including file 'src/items/item-text.h' */
07750
07751 /* modified 2021-03-29T02:30:44, size 5576 */
07752
07753 class QCP_LIB_DECL QCPItemText : public QCPAbstractItem {
07754 Q_OBJECT
07755 Q_PROPERTY(QColor color READ color WRITE setColor)
07756 Q_PROPERTY(QColor selectedColor READ selectedColor WRITE setSelectedColor)
07757 Q_PROPERTY(QPen pen READ pen WRITE setPen)
07758 Q_PROPERTY(QPen selectedPen READ selectedPen WRITE setSelectedPen)
07759 Q_PROPERTY(QBrush brush READ brush WRITE setBrush)
07760 Q_PROPERTY(QBrush selectedBrush READ selectedBrush WRITE setSelectedBrush)
07761 Q_PROPERTY(QFont font READ font WRITE setFont)
07762 Q_PROPERTY(QFont selectedFont READ selectedFont WRITE setSelectedFont)
07763 Q_PROPERTY(QString text READ text WRITE setText)
07764 Q_PROPERTY(Qt::Alignment positionAlignment READ positionAlignment WRITE setPositionAlignment)
07765 Q_PROPERTY(Qt::Alignment textAlignment READ textAlignment WRITE setTextAlignment)
07766 Q_PROPERTY(double rotation READ rotation WRITE setRotation)
07767 Q_PROPERTY(QMargins padding READ padding WRITE setPadding)
07768 public:
07769 explicit QCPItemText(QCustomPlot *parentPlot);

```

```

07772 virtual ~QCPItemText() Q_DECL_OVERRIDE;
07773
07774 // getters:
07775
07776 QColor color() const {
07777 return mColor;
07778 }
07779
07780 QColor selectedColor() const {
07781 return mSelectedColor;
07782 }
07783
07784 QPen pen() const {
07785 return mPen;
07786 }
07787
07788 QPen selectedPen() const {
07789 return mSelectedPen;
07790 }
07791
07792 QBrush brush() const {
07793 return mBrush;
07794 }
07795
07796 QBrush selectedBrush() const {
07797 return mSelectedBrush;
07798 }
07799
07800 QFont font() const {
07801 return mFont;
07802 }
07803
07804 QFont selectedFont() const {
07805 return mSelectedFont;
07806 }
07807
07808 QString text() const {
07809 return mText;
07810 }
07811
07812 Qt::Alignment positionAlignment() const {
07813 return mPositionAlignment;
07814 }
07815
07816 Qt::Alignment textAlignment() const {
07817 return mTextAlignment;
07818 }
07819
07820 double rotation() const {
07821 return mRotation;
07822 }
07823
07824 QMargins padding() const {
07825 return mPadding;
07826 }
07827
07828 // setters;
07829 void setColor(const QColor &color);
07830 void setSelectedColor(const QColor &color);
07831 void setPen(const QPen &pen);
07832 void setSelectedPen(const QPen &pen);
07833 void setBrush(const QBrush &brush);
07834 void setSelectedBrush(const QBrush &brush);
07835 void setFont(const QFont &font);
07836 void setSelectedFont(const QFont &font);
07837 void setText(const QString &text);
07838 void setPositionAlignment(Qt::Alignment alignment);
07839 void setTextAlignment(Qt::Alignment alignment);
07840 void setRotation(double degrees);
07841 void setPadding(const QMargins &padding);
07842
07843 // reimplemented virtual methods:
07844 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
07845 const Q_DECL_OVERRIDE;
07846
07847 QCPItemPosition * const position;
07848 QCPItemAnchor * const topLeft;
07849 QCPItemAnchor * const top;
07850 QCPItemAnchor * const topRight;
07851 QCPItemAnchor * const right;
07852 QCPItemAnchor * const bottomRight;
07853 QCPItemAnchor * const bottom;
07854 QCPItemAnchor * const bottomLeft;
07855 QCPItemAnchor * const left;
07856 protected:
07857

```

```

07858 enum AnchorIndex {
07859 aiTopLeft, aiTop, aiTopRight, aiRight, aiBottomRight, aiBottom, aiBottomLeft, aiLeft
07860 };
07861
07862 // property members:
07863 QColor mColor, mSelectedColor;
07864 QPen mPen, mSelectedPen;
07865 QBrush mBrush, mSelectedBrush;
07866 QFont mFont, mSelectedFont;
07867 QString mText;
07868 Qt::Alignment mPositionAlignment;
07869 Qt::Alignment mTextAlignment;
07870 double mRotation;
07871 QMargins mPadding;
07872
07873 // reimplemented virtual methods:
07874 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
07875 virtual QPointF anchorPixelPosition(int anchorId) const Q_DECL_OVERRIDE;
07876
07877 // non-virtual methods:
07878 QPointF getTextDrawPoint(const QPointF &pos, const QRectF &rect, Qt::Alignment positionAlignment)
07879 const;
07879 QFont mainFont() const;
07880 QColor mainColor() const;
07881 QPen mainPen() const;
07882 QBrush mainBrush() const;
07883 };
07884
07885 /* end of 'src/items/item-text.h' */
07886
07887
07888 /* including file 'src/items/item-ellipse.h' */
07889
07890 /* modified 2021-03-29T02:30:44, size 3890 */
07891
07892 class QCP_LIB_DECL QCPItemEllipse : public QCPAbstractItem {
07893 Q_OBJECT
07894 Q_PROPERTY(QPen pen READ pen WRITE setPen)
07895 Q_PROPERTY(QPen selectedPen READ selectedPen WRITE setSelectedPen)
07896 Q_PROPERTY(QBrush brush READ brush WRITE setBrush)
07897 Q_PROPERTY(QBrush selectedBrush READ selectedBrush WRITE setSelectedBrush)
07898 public:
07899 explicit QCPItemEllipse(QCustomPlot *parentPlot);
07900 virtual ~QCPItemEllipse() Q_DECL_OVERRIDE;
07901
07902 // getters:
07903
07904 QPen pen() const {
07905 return mPen;
07906 }
07907
07908 QPen selectedPen() const {
07909 return mSelectedPen;
07910 }
07911
07912 QBrush brush() const {
07913 return mBrush;
07914 }
07915
07916 QBrush selectedBrush() const {
07917 return mSelectedBrush;
07918 }
07919
07920 // setters:
07921 void setPen(const QPen &pen);
07922 void setSelectedPen(const QPen &pen);
07923 void setBrush(const QBrush &brush);
07924 void setSelectedBrush(const QBrush &brush);
07925
07926 // reimplemented virtual methods:
07927 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
07928 const Q_DECL_OVERRIDE;
07929
07930 QCPItemPosition * const topLeft;
07931 QCPItemPosition * const bottomRight;
07932 QCPItemAnchor * const topLeftRim;
07933 QCPItemAnchor * const top;
07934 QCPItemAnchor * const topRightRim;
07935 QCPItemAnchor * const right;
07936 QCPItemAnchor * const bottomRightRim;
07937 QCPItemAnchor * const bottom;
07938 QCPItemAnchor * const bottomLeftRim;
07939 QCPItemAnchor * const left;
07940 QCPItemAnchor * const center;
07941
07942 protected:
07943
07944

```

```

07945 enum AnchorIndex {
07946 aiTopLeftRim, aiTop, aiTopRightRim, aiRight, aiBottomRightRim, aiBottom, aiBottomLeftRim,
07947 aiLeft, aiCenter
07948 };
07949 // property members:
07950 QPen mPen, mSelectedPen;
07951 QBrush mBrush, mSelectedBrush;
07952
07953 // reimplemented virtual methods:
07954 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
07955 virtual QPointF anchorPixelPosition(int anchorId) const Q_DECL_OVERRIDE;
07956
07957 // non-virtual methods:
07958 QPen mainPen() const;
07959 QBrush mainBrush() const;
07960 };
07961
07962 /* end of 'src/items/item-ellipse.h' */
07963
07964
07965 /* including file 'src/items/item-pixmap.h' */
07966
07967 /* modified 2021-03-29T02:30:44, size 4407 */
07968
07969 class QCP_LIB_DECL QCPItemPixmap : public QCPAbstractItem {
07970 Q_OBJECT
07971 Q_PROPERTY(QPixmap pixmap READ pixmap WRITE setPixmap)
07972 Q_PROPERTY(bool scaled READ scaled WRITE setScaled)
07973 Q_PROPERTY(Qt::AspectRatioMode aspectRatioMode READ aspectRatioMode)
07974 Q_PROPERTY(Qt::TransformationMode transformationMode READ transformationMode)
07975 Q_PROPERTY(QPen pen READ pen WRITE setPen)
07976 Q_PROPERTY(QPen selectedPen READ selectedPen WRITE setSelectedPen)
07977 public:
07978 explicit QCPItemPixmap(QCustomPlot *parentPlot);
07979 virtual ~QCPItemPixmap() Q_DECL_OVERRIDE;
07980
07981 // getters:
07982
07983 QPixmap pixmap() const {
07984 return mPixmap;
07985 }
07986
07987 bool scaled() const {
07988 return mScaled;
07989 }
07990
07991 Qt::AspectRatioMode aspectRatioMode() const {
07992 return mAspectRatioMode;
07993 }
07994
07995 Qt::TransformationMode transformationMode() const {
07996 return mTransformationMode;
07997 }
07998
07999 QPen pen() const {
08000 return mPen;
08001 }
08002
08003 QPen selectedPen() const {
08004 return mSelectedPen;
08005 }
08006
08007 // setters:
08008 void setPixmap(const QPixmap &pixmap);
08009 void setScaled(bool scaled, Qt::AspectRatioMode aspectRatioMode = Qt::KeepAspectRatio,
08010 Qt::TransformationMode transformationMode = Qt::SmoothTransformation);
08011 void setPen(const QPen &pen);
08012 void setSelectedPen(const QPen &pen);
08013
08014 // reimplemented virtual methods:
08015 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
08016 const Q_DECL_OVERRIDE;
08017
08018 QCPItemPosition * const topLeft;
08019 QCPItemPosition * const bottomRight;
08020 QCPItemAnchor * const top;
08021 QCPItemAnchor * const topRight;
08022 QCPItemAnchor * const right;
08023 QCPItemAnchor * const bottom;
08024 QCPItemAnchor * const bottomLeft;
08025 QCPItemAnchor * const left;
08026
08027 protected:
08028
08029 enum AnchorIndex {
08030 aiTop, aiTopRight, aiRight, aiBottom, aiBottomLeft, aiLeft

```

```

08031 };
08032
08033 // property members:
08034 QPixmap mPixmap;
08035 QPixmap mScaledPixmap;
08036 bool mScaled;
08037 bool mScaledPixmapInvalidated;
08038 Qt::AspectRatioMode mAspectRatioMode;
08039 Qt::TransformationMode mTransformationMode;
08040 QPen mPen, mSelectedPen;
08041
08042 // reimplemented virtual methods:
08043 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
08044 virtual QPointF anchorPixelPosition(int anchorId) const Q_DECL_OVERRIDE;
08045
08046 // non-virtual methods:
08047 void updateScaledPixmap(QRect finalRect = QRect(), bool flipHorz = false, bool flipVert = false);
08048 QRect getFinalRect(bool *flippedHorz = nullptr, bool *flippedVert = nullptr) const;
08049 QPen mainPen() const;
08050 };
08051
08052 /* end of 'src/items/item-pixmap.h' */
08053
08054
08055 /* including file 'src/items/item-tracer.h' */
08056
08057 /* modified 2021-03-29T02:30:44, size 4811 */
08058
08059 class QCP_LIB_DECL QCPItemTracer : public QCPAbstractItem {
08060
08061 Q_OBJECT
08062 Q_PROPERTY(QPen pen READ pen WRITE setPen)
08063 Q_PROPERTY(QPen selectedPen READ selectedPen WRITE setSelectedPen)
08064 Q_PROPERTY(QBrush brush READ brush WRITE setBrush)
08065 Q_PROPERTY(QBrush selectedBrush READ selectedBrush WRITE setSelectedBrush)
08066 Q_PROPERTY(double size READ size WRITE setSize)
08067 Q_PROPERTY(TracerStyle style READ style WRITE setStyle)
08068 Q_PROPERTY(QCPGraph* graph READ graph WRITE setGraph)
08069 Q_PROPERTY(double graphKey READ graphKey WRITE setGraphKey)
08070 Q_PROPERTY(bool interpolating READ interpolating WRITE setInterpolating)
08071 public:
08072 enum TracerStyle {
08073 tsNone
08074 , tsPlus
08075 , tsCrosshair
08076 , tsCircle
08077 , tsSquare
08078 };
08079 Q_ENUMS(TracerStyle)
08080
08081 explicit QCPItemTracer(QCustomPlot *parentPlot);
08082 virtual ~QCPItemTracer() Q_DECL_OVERRIDE;
08083
08084 // getters:
08085
08086 QPen pen() const {
08087 return mPen;
08088 }
08089
08090 QPen selectedPen() const {
08091 return mSelectedPen;
08092 }
08093
08094 QBrush brush() const {
08095 return mBrush;
08096 }
08097
08098 QBrush selectedBrush() const {
08099 return mSelectedBrush;
08100 }
08101
08102 double size() const {
08103 return mSize;
08104 }
08105
08106 TracerStyle style() const {
08107 return mStyle;
08108 }
08109
08110 QCPGraph *graph() const {
08111 return mGraph;
08112 }
08113
08114 double graphKey() const {
08115 return mGraphKey;
08116 }
08117
08118 };
08119
08120
08121
08122
08123
08124

```

```

08125 bool interpolating() const {
08126 return mInterpolating;
08127 }
08128
08129 // setters;
08130 void setPen(const QPen &pen);
08131 void setSelectedPen(const QPen &pen);
08132 void setBrush(const QBrush &brush);
08133 void setSelectedBrush(const QBrush &brush);
08134 void setSize(double size);
08135 void setStyle(TracerStyle style);
08136 void setGraph(QCPGraph *graph);
08137 void setGraphKey(double key);
08138 void setInterpolating(bool enabled);
08139
08140 // reimplemented virtual methods:
08141 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
08142 const Q_DECL_OVERRIDE;
08143
08144 // non-virtual methods:
08145 void updatePosition();
08146
08147 QCPItemPosition * const position;
08148 protected:
08149 // property members:
08150 QPen mPen, mSelectedPen;
08151 QBrush mBrush, mSelectedBrush;
08152 double mSize;
08153 TracerStyle mStyle;
08154 QCPGraph *mGraph;
08155 double mGraphKey;
08156 bool mInterpolating;
08157
08158 // reimplemented virtual methods:
08159 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
08160
08161 // non-virtual methods:
08162 QPen mainPen() const;
08163 QBrush mainBrush() const;
08164 };
08165
08166 Q_DECLARE_METATYPE(QCPItemTracer::TracerStyle)
08167
08168 /* end of 'src/items/item-tracer.h' */
08169
08170
08171 /* including file 'src/items/item-bracket.h' */
08172 /* modified 2021-03-29T02:30:44, size 3991 */
08173
08174 class QCP_LIB_DECL QCPItemBracket : public QCPAbstractItem {
08175 public:
08176 Q_OBJECT
08177 Q_PROPERTY(QPen pen READ pen WRITE setPen)
08178 Q_PROPERTY(QPen selectedPen READ selectedPen WRITE setSelectedPen)
08179 Q_PROPERTY(double length READ length WRITE setLength)
08180 Q_PROPERTY(BracketStyle style READ style WRITE setStyle)
08181 public:
08182 enum BracketStyle {
08183 bsSquare
08184 , bsRound
08185 , bsCurly
08186 , bsCalligraphic
08187 };
08188 Q_ENUMS(BracketStyle)
08189
08190 explicit QCPItemBracket(QCustomPlot *parentPlot);
08191 virtual ~QCPItemBracket() Q_DECL_OVERRIDE;
08192
08193 // getters:
08194
08195 QPen pen() const {
08196 return mPen;
08197 }
08198
08199 QPen selectedPen() const {
08200 return mSelectedPen;
08201 }
08202
08203 double length() const {
08204 return mLength;
08205 }
08206
08207 BracketStyle style() const {
08208 return mStyle;
08209 }
08210
08211 };
08212
08213
08214
08215
08216
08217
08218

```



```

08219 // setters;
08220 void setPen(const QPen &pen);
08221 void setSelectedPen(const QPen &pen);
08222 void setLength(double length);
08223 void setStyle(BracketStyle style);
08224
08225 // reimplemented virtual methods:
08226 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
08227 const Q_DECL_OVERRIDE;
08228
08229 QCPItemPosition * const left;
08230 QCPItemPosition * const right;
08231 QCPItemAnchor * const center;
08232 protected:
08233 // property members:
08234
08235 enum AnchorIndex {
08236 aiCenter
08237 };
08238 QPen mPen, mSelectedPen;
08239 double mLength;
08240 BracketStyle mStyle;
08241
08242 // reimplemented virtual methods:
08243 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
08244 virtual QPointF anchorPixelPosition(int anchorId) const Q_DECL_OVERRIDE;
08245
08246 // non-virtual methods:
08247 QPen mainPen() const;
08248 };
08249
08250 Q_DECLARE_METATYPE(QCPItemBracket::BracketStyle)
08251
08252 /* end of 'src/items/item-bracket.h' */
08253
08254
08255 /* including file 'src/polar/radialaxis.h' */
08256 /* modified 2021-03-29T02:30:44, size 12227 */
08257
08258
08259 class QCP_LIB_DECL QCPPolarAxisRadial : public QCPLayerable {
08260
08261 Q_OBJECT
08262
08263 public:
08264 enum AngleReference {
08265 arAbsolute
08266 , arAngularAxis
08267 };
08268
08269 Q_ENUMS(AngleReference)
08270 enum ScaleType {
08271 stLinear
08272 , stLogarithmic
08273 };
08274
08275 Q_ENUMS(ScaleType)
08276 enum SelectablePart {
08277 spNone = 0
08278 , spAxis = 0x001
08279 , spTickLabels = 0x002
08280 , spAxisLabel = 0x004
08281 };
08282
08283 Q_ENUMS(SelectablePart)
08284 Q_FLAGS(SelectableParts)
08285 Q_DECLARE_FLAGS(SelectableParts, SelectablePart)
08286
08287 enum LabelMode {
08288 lmUpright
08289 , lmRotated
08290 };
08291
08292 Q_ENUMS(LabelMode)
08293
08294 explicit QCPPolarAxisRadial(QCPPolarAxisAngular *parent);
08295 virtual ~QCPPolarAxisRadial();
08296
08297 // getters:
08298
08299 bool rangeDrag() const {
08300 return mRangeDrag;
08301 }
08302
08303 bool rangeZoom() const {
08304 return mRangeZoom;
08305 }

```

```
08318
08319 double rangeZoomFactor() const {
08320 return mRangeZoomFactor;
08321 }
08322
08323 QCPPolarAxisAngular *angularAxis() const {
08324 return mAngularAxis;
08325 }
08326
08327 ScaleType scaleType() const {
08328 return mScaleType;
08329 }
08330
08331 const QCPRange range() const {
08332 return mRange;
08333 }
08334
08335 bool rangeReversed() const {
08336 return mRangeReversed;
08337 }
08338
08339 double angle() const {
08340 return mAngle;
08341 }
08342
08343 AngleReference angleReference() const {
08344 return mAngleReference;
08345 }
08346
08347 QSharedPointer<QCPAxisTicker> ticker() const {
08348 return mTicker;
08349 }
08350
08351 bool ticks() const {
08352 return mTicks;
08353 }
08354
08355 bool tickLabels() const {
08356 return mTickLabels;
08357 }
08358
08359 int tickLabelPadding() const {
08360 return mLabelPainter.padding();
08361 }
08362
08363 QFont tickLabelFont() const {
08364 return mTickLabelFont;
08365 }
08366
08367 QColor tickLabelColor() const {
08368 return mTickLabelColor;
08369 }
08370
08371 double tickLabelRotation() const {
08372 return mLabelPainter.rotation();
08373 }
08374 LabelMode tickLabelMode() const;
08375 QString numberFormat() const;
08376
08377 int numberPrecision() const {
08378 return mNumberPrecision;
08379 }
08380
08381 QVector<double> tickVector() const {
08382 return mTickVector;
08383 }
08384
08385 QVector<double> subTickVector() const {
08386 return mSubTickVector;
08387 }
08388
08389 QVector<QString> tickVectorLabels() const {
08390 return mTickVectorLabels;
08391 }
08392 int tickLengthIn() const;
08393 int tickLengthOut() const;
08394
08395 bool subTicks() const {
08396 return mSubTicks;
08397 }
08398 int subTickLengthIn() const;
08399 int subTickLengthOut() const;
08400
08401 QPen basePen() const {
08402 return mBasePen;
08403 }
08404
```

```

08405 QPen tickPen() const {
08406 return mTickPen;
08407 }
08408
08409 QPen subTickPen() const {
08410 return mSubTickPen;
08411 }
08412
08413 QFont labelFont() const {
08414 return mLabelFont;
08415 }
08416
08417 QColor labelColor() const {
08418 return mLabelColor;
08419 }
08420
08421 QString label() const {
08422 return mLabel;
08423 }
08424 int labelPadding() const;
08425
08426 SelectableParts selectedParts() const {
08427 return mSelectedParts;
08428 }
08429
08430 SelectableParts selectableParts() const {
08431 return mSelectableParts;
08432 }
08433
08434 QFont selectedTickLabelFont() const {
08435 return mSelectedTickLabelFont;
08436 }
08437
08438 QFont selectedLabelFont() const {
08439 return mSelectedLabelFont;
08440 }
08441
08442 QColor selectedTickLabelColor() const {
08443 return mSelectedTickLabelColor;
08444 }
08445
08446 QColor selectedLabelColor() const {
08447 return mSelectedLabelColor;
08448 }
08449
08450 QPen selectedBasePen() const {
08451 return mSelectedBasePen;
08452 }
08453
08454 QPen selectedTickPen() const {
08455 return mSelectedTickPen;
08456 }
08457
08458 QPen selectedSubTickPen() const {
08459 return mSelectedSubTickPen;
08460 }
08461
08462 // setters:
08463 void setRangeDrag(bool enabled);
08464 void setRangeZoom(bool enabled);
08465 void setRangeZoomFactor(double factor);
08466
08467 Q_SLOT void setScaleType(QCPPolarAxisRadial::ScaleType type);
08468 Q_SLOT void setRange(const QCPRange &range);
08469 void setRange(double lower, double upper);
08470 void setRange(double position, double size, Qt::AlignmentFlag alignment);
08471 void setRangeLower(double lower);
08472 void setRangeUpper(double upper);
08473 void setRangeReversed(bool reversed);
08474 void setAngle(double degrees);
08475 void setAngleReference(AngleReference reference);
08476 void setTicker(QSharedPointer<QCPAxisTicker> ticker);
08477 void setTicks(bool show);
08478 void setTickLabels(bool show);
08479 void setTickLabelPadding(int padding);
08480 void setTickLabelFont(const QFont &font);
08481 void setTickLabelColor(const QColor &color);
08482 void setTickLabelRotation(double degrees);
08483 void setTickLabelMode(LabelMode mode);
08484 void setNumberFormat(const QString &formatCode);
08485 void setNumberPrecision(int precision);
08486 void setTickLength(int inside, int outside = 0);
08487 void setTickLengthIn(int inside);
08488 void setTickLengthOut(int outside);
08489 void setSubTicks(bool show);
08490 void setSubTickLength(int inside, int outside = 0);
08491 void setSubTickLengthIn(int inside);

```

```

08492 void setSubTickLengthOut(int outside);
08493 void setBasePen(const QPen &pen);
08494 void setTickPen(const QPen &pen);
08495 void setSubTickPen(const QPen &pen);
08496 void setLabelFont(const QFont &font);
08497 void setLabelColor(const QColor &color);
08498 void setLabel(const QString &str);
08499 void setLabelPadding(int padding);
08500 void setSelectedTickLabelFont(const QFont &font);
08501 void setSelectedLabelFont(const QFont &font);
08502 void setSelectedTickLabelColor(const QColor &color);
08503 void setSelectedLabelColor(const QColor &color);
08504 void setSelectedBasePen(const QPen &pen);
08505 void setSelectedTickPen(const QPen &pen);
08506 void setSelectedSubTickPen(const QPen &pen);
08507 Q_SLOT void setSelectableParts(const QCPPolarAxisRadial::SelectableParts &selectableParts);
08508 Q_SLOT void setSelectedParts(const QCPPolarAxisRadial::SelectableParts &selectedParts);
08509
08510 // reimplemented virtual methods:
08511 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = 0) const
08512 Q_DECL_OVERRIDE;
08513
08514 // non-property methods:
08515 void moveRange(double diff);
08516 void scaleRange(double factor);
08517 void scaleRange(double factor, double center);
08518 void rescale(bool onlyVisiblePlottables = false);
08519 void pixelToCoord(QPointF pixelPos, double &angleCoord, double &radiusCoord) const;
08520 QPointF coordToPixel(double angleCoord, double radiusCoord) const;
08521 double coordToRadius(double coord) const;
08522 double radiusToCoord(double radius) const;
08523 SelectablePart getPartAt(const QPointF &pos) const;
08524
08525 signals:
08526 void rangeChanged(const QCPRange &newRange);
08527 void rangeChanged(const QCPRange &newRange, const QCPRange &oldRange);
08528 void scaleTypeChanged(QCPPolarAxisRadial::ScaleType scaleType);
08529 void selectionChanged(const QCPPolarAxisRadial::SelectableParts &parts);
08530 void selectableChanged(const QCPPolarAxisRadial::SelectableParts &parts);
08531
08532 protected:
08533 // property members:
08534 bool mRangeDrag;
08535 bool mRangeZoom;
08536 double mRangeZoomFactor;
08537
08538 // axis base:
08539 QCPPolarAxisAngular *mAngularAxis;
08540 double mAngle;
08541 AngleReference mAngleReference;
08542 SelectableParts mSelectableParts, mSelectedParts;
08543 QPen mBasePen, mSelectedBasePen;
08544
08545 // axis label:
08546 int mLabelPadding;
08547 QString mLabel;
08548 QFont mLabelFont, mSelectedLabelFont;
08549 QColor mLabelColor, mSelectedLabelColor;
08550
08551 // tick labels:
08552 //int mTickLabelPadding; in label painter
08553 bool mTickLabels;
08554 //double mTickLabelRotation; in label painter
08555 QFont mTickLabelFont, mSelectedTickLabelFont;
08556 QColor mTickLabelColor, mSelectedTickLabelColor;
08557 int mNumberPrecision;
08558 QLatin1Char mNumberFormatChar;
08559 bool mNumberBeautifulPowers;
08560 bool mNumberMultiplyCross;
08561
08562 // ticks and subticks:
08563 bool mTicks;
08564 bool mSubTicks;
08565 int mTickLengthIn, mTickLengthOut, mSubTickLengthIn, mSubTickLengthOut;
08566 QPen mTickPen, mSelectedTickPen;
08567 QPen mSubTickPen, mSelectedSubTickPen;
08568
08569 // scale and range:
08570 QCPRange mRange;
08571 bool mRangeReversed;
08572 ScaleType mScaleType;
08573
08574 // non-property members:
08575 QPointF mCenter;
08576 double mRadius;
08577 QSharedPointer<QCPAxisTicker> mTicker;
08578 QVector<double> mTickVector;
08579 QVector<QString> mTickVectorLabels;
08580 QVector<double> mSubTickVector;
08581 bool mDragging;
08582 QCPRange mDragStartRange;

```

```

08578 QCP::AntialiasedElements mAADragBackup, mNotAADragBackup;
08579 QCPLabelPainterPrivate mLabelPainter;
08580
08581 // reimplemented virtual methods:
08582 virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
08583 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
08584 virtual QCP::Interaction selectionCategory() const Q_DECL_OVERRIDE;
08585 // events:
08586 virtual void selectEvent(QMouseEvent *event, bool additive, const QVariant &details, bool
*selectionStateChanged) Q_DECL_OVERRIDE;
08587 virtual void deselectEvent(bool *selectionStateChanged) Q_DECL_OVERRIDE;
08588 // mouse events:
08589 virtual void mousePressEvent(QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE;
08590 virtual void mouseMoveEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
08591 virtual void mouseReleaseEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
08592 virtual void wheelEvent(QWheelEvent *event) Q_DECL_OVERRIDE;
08593
08594 // non-virtual methods:
08595 void updateGeometry(const QPointF ¢er, double radius);
08596 void setupTickVectors();
08597 QPen getBasePen() const;
08598 QPen getTickPen() const;
08599 QPen getSubTickPen() const;
08600 QFont getTickLabelFont() const;
08601 QFont getLabelFont() const;
08602 QColor getTickLabelColor() const;
08603 QColor getLabelColor() const;
08604
08605 private:
08606 Q_DISABLE_COPY(QCPPolarAxisRadial)
08607
08608 friend class QCustomPlot;
08609 friend class QCPPolarAxisAngular;
08610 };
08611
08612 Q_DECLARE_OPERATORS_FOR_FLAGS(QCPPolarAxisRadial::SelectableParts)
08613 Q_DECLARE_METATYPE(QCPPolarAxisRadial::AngleReference)
08614 Q_DECLARE_METATYPE(QCPPolarAxisRadial::ScaleType)
08615 Q_DECLARE_METATYPE(QCPPolarAxisRadial::SelectablePart)
08616
08617
08618
08619 /* end of 'src/polar/radialaxis.h' */
08620
08621
08622 /* including file 'src/polar/layoutelement-angularaxis.h' */
08623 /* modified 2021-03-29T02:30:44, size 13461 */
08624
08625 class QCP_LIB_DECL QCPPolarAxisAngular : public QCPLayoutElement {
08626
08627 Q_OBJECT
08628
08629 public:
08630 enum SelectablePart {
08631 spNone = 0
08632 , spAxis = 0x001
08633 , spTickLabels = 0x002
08634 , spAxisLabel = 0x004
08635 };
08636
08637 Q_ENUMS(SelectablePart)
08638 Q_FLAGS(SelectableParts)
08639 Q_DECLARE_FLAGS(SelectableParts, SelectablePart)
08640
08641 enum LabelMode {
08642 lmUpright
08643 , lmRotated
08644 };
08645 Q_ENUMS(LabelMode)
08646
08647 explicit QCPPolarAxisAngular(QCustomPlot *parentPlot);
08648 virtual ~QCPPolarAxisAngular();
08649
08650 // getters:
08651 QPixmap background() const {
08652 return mBackgroundPixmap;
08653 }
08654 QBrush backgroundBrush() const {
08655 return mBackgroundBrush;
08656 }
08657 bool backgroundScaled() const {
08658 return mBackgroundScaled;
08659 }

```

```
08672
08673 Qt::AspectRatioMode backgroundScaledMode() const {
08674 return mBackgroundScaledMode;
08675 }
08676
08677 bool rangeDrag() const {
08678 return mRangeDrag;
08679 }
08680
08681 bool rangeZoom() const {
08682 return mRangeZoom;
08683 }
08684
08685 double rangeZoomFactor() const {
08686 return mRangeZoomFactor;
08687 }
08688
08689 const QCPRange range() const {
08690 return mRange;
08691 }
08692
08693 bool rangeReversed() const {
08694 return mRangeReversed;
08695 }
08696
08697 double angle() const {
08698 return mAngle;
08699 }
08700
08701 QSharedPointer<QCPSAxisTicker> ticker() const {
08702 return mTicker;
08703 }
08704
08705 bool ticks() const {
08706 return mTicks;
08707 }
08708
08709 bool tickLabels() const {
08710 return mTickLabels;
08711 }
08712
08713 int tickLabelPadding() const {
08714 return mLabelPainter.padding();
08715 }
08716
08717 QFont tickLabelFont() const {
08718 return mTickLabelFont;
08719 }
08720
08721 QColor tickLabelColor() const {
08722 return mTickLabelColor;
08723 }
08724
08725 double tickLabelRotation() const {
08726 return mLabelPainter.rotation();
08727 }
08728 LabelMode tickLabelMode() const;
08729 QString numberFormat() const;
08730
08731 int numberPrecision() const {
08732 return mNumberPrecision;
08733 }
08734
08735 QVector<double> tickVector() const {
08736 return mTickVector;
08737 }
08738
08739 QVector<QString> tickVectorLabels() const {
08740 return mTickVectorLabels;
08741 }
08742
08743 int tickLengthIn() const {
08744 return mTickLengthIn;
08745 }
08746
08747 int tickLengthOut() const {
08748 return mTickLengthOut;
08749 }
08750
08751 bool subTicks() const {
08752 return mSubTicks;
08753 }
08754
08755 int subTickLengthIn() const {
08756 return mSubTickLengthIn;
08757 }
08758
```

```

08759 int subTickLengthOut() const {
08760 return mSubTickLengthOut;
08761 }
08762
08763 QPen basePen() const {
08764 return mBasePen;
08765 }
08766
08767 QPen tickPen() const {
08768 return mTickPen;
08769 }
08770
08771 QPen subTickPen() const {
08772 return mSubTickPen;
08773 }
08774
08775 QFont labelFont() const {
08776 return mLabelFont;
08777 }
08778
08779 QColor labelColor() const {
08780 return mLabelColor;
08781 }
08782
08783 QString label() const {
08784 return mLabel;
08785 }
08786
08787 int labelPadding() const {
08788 return mLabelPadding;
08789 }
08790
08791 SelectableParts selectedParts() const {
08792 return mSelectedParts;
08793 }
08794
08795 SelectableParts selectableParts() const {
08796 return mSelectableParts;
08797 }
08798
08799 QFont selectedTickLabelFont() const {
08800 return mSelectedTickLabelFont;
08801 }
08802
08803 QFont selectedLabelFont() const {
08804 return mSelectedLabelFont;
08805 }
08806
08807 QColor selectedTickLabelColor() const {
08808 return mSelectedTickLabelColor;
08809 }
08810
08811 QColor selectedLabelColor() const {
08812 return mSelectedLabelColor;
08813 }
08814
08815 QPen selectedBasePen() const {
08816 return mSelectedBasePen;
08817 }
08818
08819 QPen selectedTickPen() const {
08820 return mSelectedTickPen;
08821 }
08822
08823 QPen selectedSubTickPen() const {
08824 return mSelectedSubTickPen;
08825 }
08826
08827 QCPPolarGrid *grid() const {
08828 return mGrid;
08829 }
08830
08831 // setters:
08832 void setBackground(const QPixmap &pm);
08833 void setBackground(const QPixmap &pm, bool scaled, Qt::AspectRatioMode mode =
Qt::KeepAspectRatioByExpanding);
08834 void setBackground(const QBrush &brush);
08835 void setBackgroundScaled(bool scaled);
08836 void setBackgroundScaledMode(Qt::AspectRatioMode mode);
08837 void setRangeDrag(bool enabled);
08838 void setRangeZoom(bool enabled);
08839 void setRangeZoomFactor(double factor);
08840
08841 Q_SLOT void setRange(const QCPRange &range);
08842 void setRange(double lower, double upper);
08843 void setRange(double position, double size, Qt::AlignmentFlag alignment);
08844 void setRangeLower(double lower);

```

```

08845 void setRangeUpper(double upper);
08846 void setRangeReversed(bool reversed);
08847 void setAngle(double degrees);
08848 void setTicker(QSharedPointer<QCPAxisTicker> ticker);
08849 void setTicks(bool show);
08850 void setTickLabels(bool show);
08851 void setTickLabelPadding(int padding);
08852 void setTickLabelFont(const QFont &font);
08853 void setTickLabelColor(const QColor &color);
08854 void setTickLabelRotation(double degrees);
08855 void setTickLabelMode(LabelMode mode);
08856 void setNumberFormat(const QString &formatCode);
08857 void setNumberPrecision(int precision);
08858 void setTickLength(int inside, int outside = 0);
08859 void setTickLengthIn(int inside);
08860 void setTickLengthOut(int outside);
08861 void setSubTicks(bool show);
08862 void setSubTickLength(int inside, int outside = 0);
08863 void setSubTickLengthIn(int inside);
08864 void setSubTickLengthOut(int outside);
08865 void setBasePen(const QPen &pen);
08866 void setTickPen(const QPen &pen);
08867 void setSubTickPen(const QPen &pen);
08868 void setLabelFont(const QFont &font);
08869 void setLabelColor(const QColor &color);
08870 void setLabel(const QString &str);
08871 void setLabelPadding(int padding);
08872 void setLabelPosition(Qt::AlignmentFlag position);
08873 void setSelectedTickLabelFont(const QFont &font);
08874 void setSelectedLabelFont(const QFont &font);
08875 void setSelectedTickLabelColor(const QColor &color);
08876 void setSelectedLabelColor(const QColor &color);
08877 void setSelectedBasePen(const QPen &pen);
08878 void setSelectedTickPen(const QPen &pen);
08879 void setSelectedSubTickPen(const QPen &pen);
08880 Q_SLOT void setSelectableParts(const QCPPolarAxisAngular::SelectableParts &selectableParts);
08881 Q_SLOT void setSelectedParts(const QCPPolarAxisAngular::SelectableParts &selectedParts);
08882
08883 // reimplemented virtual methods:
08884 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = 0) const
08885 Q_DECL_OVERRIDE;
08886 virtual void update(UpdatePhase phase) Q_DECL_OVERRIDE;
08887 virtual QList<QCPLayoutElement*> elements(bool recursive) const Q_DECL_OVERRIDE;
08888
08889 // non-property methods:
08890 bool removeGraph(QCPPolarGraph *graph);
08891 int radialAxisCount() const;
08892 QCPPolarAxisRadial *radialAxis(int index = 0) const;
08893 QList<QCPPolarAxisRadial*> radialAxes() const;
08894 QCPPolarAxisRadial *addRadialAxis(QCPPolarAxisRadial *axis = 0);
08895 bool removeRadialAxis(QCPPolarAxisRadial *axis);
08896
08897 QCPLayoutInset *insetLayout() const {
08898 return mInsetLayout;
08899 }
08900
08901 QRegion exactClipRegion() const;
08902
08903 void moveRange(double diff);
08904 void scaleRange(double factor);
08905 void scaleRange(double factor, double center);
08906 void rescale(bool onlyVisiblePlottables = false);
08907
08908 double coordToAngleRad(double coord) const {
08909 return mAngleRad + (coord - mRange.lower) / mRange.size() * (mRangeReversed ? -2.0 * M_PI : 2.0
08910 * M_PI);
08911 } // mention in doc that return doesn't wrap
08912
08913 double angleRadToCoord(double angleRad) const {
08914 return mRange.lower + (angleRad - mAngleRad) / (mRangeReversed ? -2.0 * M_PI : 2.0 * M_PI) *
08915 mRange.size();
08916 }
08917 void pixelToCoord(QPointF pixelPos, double &angleCoord, double &radiusCoord) const;
08918 QPointF coordToPixel(double angleCoord, double radiusCoord) const;
08919 SelectablePart getPartAt(const QPointF &pos) const;
08920
08921 // read-only interface imitating a QRect:
08922 int left() const {
08923 return mRect.left();
08924 }
08925 int right() const {
08926 return mRect.right();
08927 }
08928 int top() const {
08929 return mRect.top();
08930 }

```



```

08929 }
08930
08931 int bottom() const {
08932 return mRect.bottom();
08933 }
08934
08935 int width() const {
08936 return mRect.width();
08937 }
08938
08939 int height() const {
08940 return mRect.height();
08941 }
08942
08943 QSize size() const {
08944 return mRect.size();
08945 }
08946
08947 QPoint topLeft() const {
08948 return mRect.topLeft();
08949 }
08950
08951 QPoint topRight() const {
08952 return mRect.topRight();
08953 }
08954
08955 QPoint bottomLeft() const {
08956 return mRect.bottomLeft();
08957 }
08958
08959 QPoint bottomRight() const {
08960 return mRect.bottomRight();
08961 }
08962
08963 QPointF center() const {
08964 return mCenter;
08965 }
08966
08967 double radius() const {
08968 return mRadius;
08969 }
08970
08971 signals:
08972 void rangeChanged(const QCPRange &newRange);
08973 void rangeChanged(const QCPRange &newRange, const QCPRange &oldRange);
08974 void selectionChanged(const QCPPolarAxisAngular::SelectableParts &parts);
08975 void selectableChanged(const QCPPolarAxisAngular::SelectableParts &parts);
08976
08977 protected:
08978 // property members:
08979 QBrush mBackgroundBrush;
08980 QPixmap mBackgroundPixmap;
08981 QPixmap mScaledBackgroundPixmap;
08982 bool mBackgroundScaled;
08983 Qt::AspectRatioMode mBackgroundScaledMode;
08984 QCPLayoutInset *mInsetLayout;
08985 bool mRangeDrag;
08986 bool mRangeZoom;
08987 double mRangeZoomFactor;
08988
08989 // axis base:
08990 double mAngle, mAngleRad;
08991 SelectableParts mSelectableParts, mSelectedParts;
08992 QPen mBasePen, mSelectedBasePen;
08993 // axis label:
08994 int mLabelPadding;
08995 QString mLabel;
08996 QFont mLabelFont, mSelectedLabelFont;
08997 QColor mLabelColor, mSelectedLabelColor;
08998 // tick labels:
08999 //int mTickLabelPadding; in label painter
09000 bool mTickLabels;
09001 //double mTickLabelRotation; in label painter
09002 QFont mTickLabelFont, mSelectedTickLabelFont;
09003 QColor mTickLabelColor, mSelectedTickLabelColor;
09004 int mNumberPrecision;
09005 QLatin1Char mNumberFormatChar;
09006 bool mNumberBeautifulPowers;
09007 bool mNumberMultiplyCross;
09008 // ticks and subticks:
09009 bool mTicks;
09010 bool mSubTicks;
09011 int mTickLengthIn, mTickLengthOut, mSubTickLengthIn, mSubTickLengthOut;
09012 QPen mTickPen, mSelectedTickPen;
09013 QPen mSubTickPen, mSelectedSubTickPen;
09014 // scale and range:
09015 QCPRange mRange;

```

```

09016 bool mRangeReversed;
09017
09018 // non-property members:
09019 QPointF mCenter;
09020 double mRadius;
09021 QList<QCPPolarAxisRadial*> mRadialAxes;
09022 QCPPolarGrid *mGrid;
09023 QList<QCPPolarGraph*> mGraphs;
09024 QSharedPointer<QCPAxisTicker> mTicker;
09025 QVector<double> mTickVector;
09026 QVector<QString> mTickVectorLabels;
09027 QVector<QPointF> mTickVectorCosSin;
09028 QVector<double> mSubTickVector;
09029 QVector<QPointF> mSubTickVectorCosSin;
09030 bool mDragging;
09031 QCPRange mDragAngularStart;
09032 QList<QCPRange> mDragRadialStart;
09033 QCP::AntialiasedElements mAADragBackup, mNotAADragBackup;
09034 QCPLabelPainterPrivate mLabelPainter;
09035
09036 // reimplemented virtual methods:
09037 virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
09038 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
09039 virtual QCP::Interaction selectionCategory() const Q_DECL_OVERRIDE;
09040 // events:
09041 virtual void mousePressEvent(QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE;
09042 virtual void mouseMoveEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
09043 virtual void mouseReleaseEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
09044 virtual void wheelEvent(QWheelEvent *event) Q_DECL_OVERRIDE;
09045
09046 // non-virtual methods:
09047 bool registerPolarGraph(QCPPolarGraph *graph);
09048 void drawBackground(QCPPainter *painter, const QPointF ¢er, double radius);
09049 void setupTickVectors();
09050 QPen getBasePen() const;
09051 QPen getTickPen() const;
09052 QPen getSubTickPen() const;
09053 QFont getTickLabelFont() const;
09054 QFont getLabelFont() const;
09055 QColor getTickLabelColor() const;
09056 QColor getLabelColor() const;
09057
09058 private:
09059 Q_DISABLE_COPY(QCPPolarAxisAngular)
09060
09061 friend class QCustomPlot;
09062 friend class QCPPolarGrid;
09063 friend class QCPPolarGraph;
09064 };
09065
09066 Q_DECLARE_OPERATORS_FOR_FLAGS(QCPPolarAxisAngular::SelectableParts)
09067 Q_DECLARE_METATYPE(QCPPolarAxisAngular::SelectablePart)
09068
09069 /* end of 'src/polar/layoutelement-angularaxis.h' */
09070
09071
09072 /* including file 'src/polar/polargrid.h' */
09073 /* modified 2021-03-29T02:30:44, size 4506 */
09074
09075 class QCP_LIB_DECL QCPPolarGrid : public QCPLayerable {
09076
09077 Q_OBJECT
09078
09079 public:
09080 enum GridType {
09081 gtAngular = 0x01
09082 , gtRadial = 0x02
09083 , gtAll = 0xFF
09084 , gtNone = 0x00
09085 };
09086 Q_ENUMS(GridType)
09087 Q_FLAGS(GridTypes)
09088 Q_DECLARE_FLAGS(GridTypes, GridType)
09089
09090 explicit QCPPolarGrid(QCPPolarAxisAngular *parentAxis);
09091
09092 // getters:
09093 QCPPolarAxisRadial *radialAxis() const {
09094 return mRadialAxis.data();
09095 }
09096
09097 GridTypes type() const {
09098 return mType;
09099 }
09100
09101 GridTypes subGridType() const {

```

```

09108 return mSubGridType;
09109 }
09110
09111 bool antialiasedSubGrid() const {
09112 return mAntialiasedSubGrid;
09113 }
09114
09115 bool antialiasedZeroLine() const {
09116 return mAntialiasedZeroLine;
09117 }
09118
09119 QPen angularPen() const {
09120 return mAngularPen;
09121 }
09122
09123 QPen angularSubGridPen() const {
09124 return mAngularSubGridPen;
09125 }
09126
09127 QPen radialPen() const {
09128 return mRadialPen;
09129 }
09130
09131 QPen radialSubGridPen() const {
09132 return mRadialSubGridPen;
09133 }
09134
09135 QPen radialZeroLinePen() const {
09136 return mRadialZeroLinePen;
09137 }
09138
09139 // setters:
09140 void setRadialAxis(QCPPolarAxisRadial *axis);
09141 void setType(GridTypes type);
09142 void setSubGridType(GridTypes type);
09143 void setAntialiasedSubGrid(bool enabled);
09144 void setAntialiasedZeroLine(bool enabled);
09145 void setAngularPen(const QPen &pen);
09146 void setAngularSubGridPen(const QPen &pen);
09147 void setRadialPen(const QPen &pen);
09148 void setRadialSubGridPen(const QPen &pen);
09149 void setRadialZeroLinePen(const QPen &pen);
09150
09151 protected:
09152 // property members:
09153 GridTypes mType;
09154 GridTypes mSubGridType;
09155 bool mAntialiasedSubGrid, mAntialiasedZeroLine;
09156 QPen mAngularPen, mAngularSubGridPen;
09157 QPen mRadialPen, mRadialSubGridPen, mRadialZeroLinePen;
09158
09159 // non-property members:
09160 QCPPolarAxisAngular *mParentAxis;
09161 QPointer<QCPPolarAxisRadial> mRadialAxis;
09162
09163 // reimplemented virtual methods:
09164 virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
09165 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
09166
09167 // non-virtual methods:
09168 void drawRadialGrid(QCPPainter *painter, const QPointF ¢er, const QVector<double> &coords,
09169 const QPen &pen, const QPen &zeroPen = Qt::NoPen);
09170 void drawAngularGrid(QCPPainter *painter, const QPointF ¢er, double radius, const
09171 QVector<QPointF> &ticksCosSin, const QPen &pen);
09172
09173 private:
09174 Q_DISABLE_COPY(QCPPolarGrid)
09175 };
09176
09177 Q_DECLARE_OPERATORS_FOR_FLAGS(QCPPolarGrid::GridTypes)
09178 Q_DECLARE_METATYPE(QCPPolarGrid::GridType)
09179
09180 /* end of 'src/polar/polargrid.h' */
09181
09182
09183 /* including file 'src/polar/polargraph.h' */
09184 /* modified 2021-03-29T02:30:44, size 9606 */
09185
09186
09187 class QCP_LIB_DECL QCPPolarLegendItem : public QCPAbstractLegendItem {
09188 Q_OBJECT
09189 public:
09190 QCPPolarLegendItem(QCPLegend *parent, QCPPolarGraph *graph);
09191
09192 // getters:

```

```

09193
09194 QCPPolarGraph *polarGraph() {
09195 return mPolarGraph;
09196 }
09197
09198 protected:
09199 // property members:
09200 QCPPolarGraph *mPolarGraph;
09201
09202 // reimplemented virtual methods:
09203 virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
09204 virtual QSize minimumOuterSizeHint() const Q_DECL_OVERRIDE;
09205
09206 // non-virtual methods:
09207 QPen getIconBorderPen() const;
09208 QColor getTextColor() const;
09209 QFont getFont() const;
09210 };
09211
09212 class QCP_LIB_DECL QCPPolarGraph : public QCPLayerable {
09213
09214 Q_OBJECT
09215
09216 public:
09217 enum LineStyle {
09218 lsNone
09219 , lsLine
09220 };
09221 Q_ENUMS(LineStyle)
09222
09223 QCPPolarGraph(QCPPolarAxisAngular *keyAxis, QCPPolarAxisRadial *valueAxis);
09224 virtual ~QCPPolarGraph();
09225
09226 // getters:
09227
09228 QString name() const {
09229 return mName;
09230 }
09231
09232 bool antialiasedFill() const {
09233 return mAntialiasedFill;
09234 }
09235
09236 bool antialiasedScatters() const {
09237 return mAntialiasedScatters;
09238 }
09239
09240 QPen pen() const {
09241 return mPen;
09242 }
09243
09244 QBrush brush() const {
09245 return mBrush;
09246 }
09247
09248 bool periodic() const {
09249 return mPeriodic;
09250 }
09251
09252 QCPPolarAxisAngular *keyAxis() const {
09253 return mKeyAxis.data();
09254 }
09255
09256 QCPPolarAxisRadial *valueAxis() const {
09257 return mValueAxis.data();
09258 }
09259
09260 QCP::SelectionType selectable() const {
09261 return mSelectable;
09262 }
09263
09264 bool selected() const {
09265 return !mSelection.isEmpty();
09266 }
09267
09268 QCPDataSelection selection() const {
09269 return mSelection;
09270 }
09271
09272 //QCPSelectionDecorator *selectionDecorator() const { return mSelectionDecorator; }
09273
09274 QSharedPointer<QCPGraphDataContainer> data() const {
09275 return mDataContainer;
09276 }
09277
09278 LineStyle lineStyle() const {
09279 return mLineStyle;
09280 }

```

```

09288
09289 QCPScatterStyle scatterStyle() const {
09290 return mScatterStyle;
09291 }
09292
09293 // setters:
09294 void setName(const QString &name);
09295 void setAntialiasedFill(bool enabled);
09296 void setAntialiasedScatters(bool enabled);
09297 void setPen(const QPen &pen);
09298 void setBrush(const QBrush &brush);
09299 void setPeriodic(bool enabled);
09300 void setKeyAxis(QCPPolarAxisAngular *axis);
09301 void setValueAxis(QCPPolarAxisRadial *axis);
09302 Q_SLOT void setSelectable(QCP::SelectionType selectable);
09303 Q_SLOT void setSelection(QCPDataSelection selection);
09304 //void setSelectionDecorator(QCPSelectionDecorator *decorator);
09305 void setData(QSharedPointer<QCPGraphDataContainer> data);
09306 void setData(const QVector<double> &keys, const QVector<double> &values, bool alreadySorted =
false);
09307 void setLineStyle(LineStyle ls);
09308 void setScatterStyle(const QCPScatterStyle &style);
09309
09310 // non-property methods:
09311 void addData(const QVector<double> &keys, const QVector<double> &values, bool alreadySorted =
false);
09312 void addData(double key, double value);
09313 void coordsToPixels(double key, double value, double &x, double &y) const;
09314 const QPointF coordsToPixels(double key, double value) const;
09315 void pixelsToCoords(double x, double y, double &key, double &value) const;
09316 void pixelsToCoords(const QPointF &pixelPos, double &key, double &value) const;
09317 void rescaleAxes(bool onlyEnlarge = false) const;
09318 void rescaleKeyAxis(bool onlyEnlarge = false) const;
09319 void rescaleValueAxis(bool onlyEnlarge = false, bool inKeyRange = false) const;
09320 bool addToLegend(QCPLegend *legend);
09321 bool addToLegend();
09322 bool removeFromLegend(QCPLegend *legend) const;
09323 bool removeFromLegend() const;
09324
09325 // introduced virtual methods:
09326 virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = 0) const;
 // actually introduced in QCPLayerable as non-pure, but we want to force reimplementation for
 plottables
09327
09328 virtual QCPPlottableInterface1D *interface1D() {
09329 return 0;
09330 } // TODO: return this later, when QCPAbstractPolarPlottable is created
09331 virtual QCPRange getKeyRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth) const;
09332 virtual QCPRange getValueRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth, const
QCPRange &inKeyRange = QCPRange()) const;
09333
09334 signals:
09335 void selectionChanged(bool selected);
09336 void selectionChanged(const QCPDataSelection &selection);
09337 void selectableChanged(QCP::SelectionType selectable);
09338
09339 protected:
09340 // property members:
09341 QSharedPointer<QCPGraphDataContainer> mDataContainer;
09342 LineStyle mLineStyle;
09343 QCPScatterStyle mScatterStyle;
09344 QString mName;
09345 bool mAntialiasedFill, mAntialiasedScatters;
09346 QPen mPen;
09347 QBrush mBrush;
09348 bool mPeriodic;
09349 QPointer<QCPPolarAxisAngular> mKeyAxis;
09350 QPointer<QCPPolarAxisRadial> mValueAxis;
09351 QCP::SelectionType mSelectable;
09352 QCPDataSelection mSelection;
09353 //QCPSelectionDecorator *mSelectionDecorator;
09354
09355 // introduced virtual methods (later reimplemented TODO from QCPAbstractPolarPlottable):
09356 virtual QRect clipRect() const;
09357 virtual void draw(QCPPainter *painter);
09358 virtual QCP::Interaction selectionCategory() const;
09359 void applyDefaultAntialiasingHint(QCPPainter *painter) const;
09360 // events:
09361 virtual void selectEvent(QMouseEvent *event, bool additive, const QVariant &details, bool
*selectionStateChanged);
09362 virtual void deselectEvent(bool *selectionStateChanged);
09363 // virtual drawing helpers:
09364 virtual void drawLinePlot(QCPPainter *painter, const QVector<QPointF> &lines) const;
09365 virtual void drawFill(QCPPainter *painter, QVector<QPointF> *lines) const;
09366 virtual void drawScatterPlot(QCPPainter *painter, const QVector<QPointF> &scatters, const
QCPScatterStyle &style) const;
09367

```

```

09368 // introduced virtual methods:
09369 virtual void drawLegendIcon(QCPPainter *painter, const QRectF &rect) const;
09370
09371 // non-virtual methods:
09372 void applyFillAntialiasingHint(QCPPainter *painter) const;
09373 void applyScattersAntialiasingHint(QCPPainter *painter) const;
09374 double pointDistance(const QPointF &pixelPoint, QCPGraphDataContainer::const_iterator
&closestData) const;
09375 // drawing helpers:
09376 virtual int dataCount() const;
09377 void getDataSegments(QList<QCPDataRange> &selectedSegments, QList<QCPDataRange>
&unselectedSegments) const;
09378 void drawPolyline(QCPPainter *painter, const QVector<QPointF> &lineData) const;
09379 void getVisibleDataBounds(QCPGraphDataContainer::const_iterator &begin,
QCPGraphDataContainer::const_iterator &end, const QCPDataRange &rangeRestriction) const;
09380 void getLines(QVector<QPointF> *lines, const QCPDataRange &dataRange) const;
09381 void getScatters(QVector<QPointF> *scatters, const QCPDataRange &dataRange) const;
09382 void getOptimizedLineData(QVector<QCPGraphData> *lineData, const
QCPGraphDataContainer::const_iterator &begin, const QCPGraphDataContainer::const_iterator &end) const;
09383 void getOptimizedScatterData(QVector<QCPGraphData> *scatterData,
QCPGraphDataContainer::const_iterator begin, QCPGraphDataContainer::const_iterator end) const;
09384 QVector<QPointF> dataToLines(const QVector<QCPGraphData> &data) const;
09385
09386 private:
09387 Q_DISABLE_COPY(QCPPolarGraph)
09388
09389 friend class QCPPolarLegendItem;
09390 };
09391
09392 /* end of 'src/polar/polargraph.h' */
09393
09394
09395 #endif // QCUSTOMPLOT_H
09396

```

## 16.86 CppModelExporter.h

```

00001 #ifndef CPPMODELEXPORTER_H
00002 #define CPPMODELEXPORTER_H
00003
00004 #include <string>
00005
00006 class QPlainTextEdit;
00007 class Simulator;
00008
00009 // Document the service that generates C++ model representation text.
00026 class CppModelExporter {
00027 public:
00029 CppModelExporter(Simulator* simulator, QPlainTextEdit* cppCodeEditor);
00030
00035 std::string addCppCodeLine(const std::string& line, unsigned int indent = 0) const;
00037 void actualizeModelCppCode() const;
00038
00039 private:
00040 Simulator* _simulator;
00041 QPlainTextEdit* _cppCodeEditor;
00042 };
00043
00044 #endif // CPPMODELEXPORTER_H

```

## 16.87 GraphicalAutomaticPositioningStrategy.h

```

00001 #ifndef GRAPHICALAUTOMATICPOSITIONINGSTRATEGY_H
00002 #define GRAPHICALAUTOMATICPOSITIONINGSTRATEGY_H
00003
00004 #include <QPointF>
00005 #include <QRectF>
00006 #include <QSizeF>
00007
00008 #include <memory>
00009
00010 class GraphicalAutomaticPositioningStrategy {
00011 public:
00019 virtual ~GraphicalAutomaticPositioningStrategy() = default;
00020
00022 virtual QPointF initialComponentLayoutOrigin() const = 0;
00023
00035 virtual QPointF dataDefinitionPosition(const QRectF& anchorBounds,
00036 const QSizeF& childSize,
00037 int index,
00038 int count,
00039 bool upperArc,

```

```

00040 int radialLayer) const = 0;
00041 };
00042
00046 class GraphicalAutomaticPositioningStrategyFactory {
00047 public:
00049 static std::unique_ptr<GraphicalAutomaticPositioningStrategy> createFromPreferences();
00050 };
00051
00052 #endif // GRAPHICALAUTOMATICPOSITIONINGSTRATEGY_H

```

## 16.88 GraphicalDataDefinitionLayout.h

```

00001 #ifndef GRAPHICALDATADEFINITIONLAYOUT_H
00002 #define GRAPHICALDATADEFINITIONLAYOUT_H
00003
00004 #include <QPointF>
00005 #include <QRectF>
00006 #include <QSizeF>
00007
00014 class GraphicalDataDefinitionLayout {
00015 public:
00027 static QPointF arcPosition(const QRectF& anchorBounds,
00028 const QSizeF& childSize,
00029 int index,
00030 int count,
00031 bool upperArc,
00032 int radialLayer = 0);
00033
00034 private:
00035 GraphicalDataDefinitionLayout() = default;
00036 };
00037
00038 #endif // GRAPHICALDATADEFINITIONLAYOUT_H

```

## 16.89 GraphicalModelBuilder.h

```

00001 #ifndef GRAPHICALMODELBUILDER_H
00002 #define GRAPHICALMODELBUILDER_H
00003
00004 #include <map>
00005 #include <string>
00006
00007 #include <QColor>
00008
00009 class Simulator;
00010 class QTextEdit;
00011 class ModelGraphicsView;
00012 class ModelGraphicsScene;
00013 class ModelComponent;
00014 class GraphicalModelComponent;
00015 class ModelDataDefinition;
00016 class GraphicalModelDataDefinition;
00017 template<typename T> class List;
00018
00019 // Document the service that rebuilds graphical scene artifacts from kernel models.
00037 class GraphicalModelBuilder {
00038 public:
00040 GraphicalModelBuilder(Simulator* simulator,
00041 ModelGraphicsView* graphicsView,
00042 ModelGraphicsScene* scene,
00043 std::map<std::string, QColor>* pluginCategoryColor,
00044 QTextEdit* console);
00045
00052 void setModelLevelFilter(unsigned int modelLevel);
00054 void clearModelLevelFilter();
00055
00059 void recursivelyGenerateGraphicalModelFromModel(ModelComponent* component,
00060 List<ModelComponent*>* visited,
00061 std::map<ModelComponent*,
00062 GraphicalModelComponent*>* map,
00063 int* x,
00064 int* y,
00065 int* ymax,
00066 int sequenceInLine);
00068 void generateGraphicalModelFromModel();
00070 static void synchronizeGraphicalDataDefinitionsLayer(Simulator* simulator, ModelGraphicsScene*
00071 scene);
00072 private:
00073 void rebuildGraphicalDataDefinitionsLayer(std::map<ModelComponent*, GraphicalModelComponent*>*
00074 componentMap);

```

```

00074 bool componentBelongsToActiveModelLevel(ModelComponent* component) const;
00075
00076 Simulator* _simulator;
00077 ModelGraphicsView* _graphicsView;
00078 ModelGraphicsScene* _scene;
00079 std::map<std::string, QColor>* _pluginCategoryColor;
00080 QTextEdit* _console;
00081 };
00082
00083 #endif // GRAPHICALMODELBUILDER_H

```

## 16.90 GraphicalModelSerializer.h

```

00001 #ifndef GRAPHICALMODELSerializer_H
00002 #define GRAPHICALMODELSerializer_H
00003
00004 #include <functional>
00005 #include <string>
00006
00007 class Simulator;
00008 class Model;
00009 class QWidget;
00010 class QFile;
00011 class QString;
00012 class QPlainTextEdit;
00013 class QTextEdit;
00014 class QSlider;
00015 class QAction;
00016 class ModelGraphicsView;
00017
00018 // Document persistence responsibilities for textual and graphical GUI model data.
00036 class GraphicalModelSerializer {
00037 public:
00039 GraphicalModelSerializer(Simulator* simulator,
00040 QWidget* ownerWidget,
00041 QPlainTextEdit* modelTextEditor,
00042 ModelGraphicsView* graphicsView,
00043 QSlider* zoomSlider,
00044 QAction* actionShowGrid,
00045 QAction* actionShowRule,
00046 QAction* actionShowSnap,
00047 QAction* actionShowGuides,
00048 QAction* actionShowInternalElements,
00049 QAction* actionShowEditableElements,
00050 QAction* actionShowAttachedElements,
00051 QAction* actionShowRecursiveElements,
00052 QTextEdit* console,
00053 QString* modelFilename,
00054 std::function<void()> clearModelEditors,
00055 std::function<void()> rebuildGraphicalModelFromModel,
00056 std::function<void()> applyShowInternalElements,
00057 std::function<void()> applyShowEditableElements,
00058 std::function<void()> applyShowAttachedElements);
00059
00064 bool saveTextModel(QFile* saveFile, const QString& data) const;
00069 bool saveGraphicalModel(const QString& filename) const;
00074 Model* loadGraphicalModel(const std::string& filename) const;
00075
00076 private:
00077 // Encode free text fields into a tab-safe persisted value.
00078 static QString encodeGuiText(const QString& text);
00079 // Decode tab-safe persisted text into original UTF-8 value.
00080 static QString decodeGuiText(const QString& text);
00081
00082 private:
00083 Simulator* _simulator;
00084 QWidget* _ownerWidget;
00085 QPlainTextEdit* _modelTextEditor;
00086 ModelGraphicsView* _graphicsView;
00087 QSlider* _zoomSlider;
00088 QAction* _actionShowGrid;
00089 QAction* _actionShowRule;
00090 QAction* _actionShowSnap;
00091 QAction* _actionShowGuides;
00092 QAction* _actionShowInternalElements;
00093 QAction* _actionShowEditableElements;
00094 QAction* _actionShowAttachedElements;
00095 QAction* _actionShowRecursiveElements;
00096 QTextEdit* _console;
00097 QString* _modelFilename;
00098 std::function<void()> _clearModelEditors;
00099 std::function<void()> _rebuildGraphicalModelFromModel;
00100 std::function<void()> _applyShowInternalElements;
00101 std::function<void()> _applyShowEditableElements;
00102 std::function<void()> _applyShowAttachedElements;

```



```

00103 };
00104
00105 #endif // GRAPHICALMODELSEIALIZER_H

```

## 16.91 GraphvizModelExporter.h

```

00001 #ifndef GRAPHVIZMODELEXPORTER_H
00002 #define GRAPHVIZMODELEXPORTER_H
00003
00004 #include <functional>
00005 #include <list>
00006 #include <map>
00007 #include <string>
00008
00009 class QLabel;
00010 class QCheckBox;
00011 class Simulator;
00012 class ModelDataDefinition;
00013
00014 // Document the service used to export graphical model representations via Graphviz.
00032 class GraphvizModelExporter {
00033 public:
00037 GraphvizModelExporter(Simulator* simulator,
00038 QLabel* modelGraphicLabel,
00039 QCheckBox* showStatistics,
00040 QCheckBox* showEditable,
00041 QCheckBox* showShared,
00042 QCheckBox* showRecursive,
00043 QCheckBox* showLevels,
00044 std::function<bool()> ensureModelSynchronized);
00045
00051 std::string adjustDotName(std::string name) const;
00052
00056 void insertTextInDot(std::string text,
00057 unsigned int compLevel,
00058 unsigned int compRank,
00059 std::map<unsigned int, std::map<unsigned int, std::list<std::string>*>*>
00060 dotmap,
00061 bool isNode = false) const;
00064 void recursiveCreateModelGraphicPicture(ModelDataDefinition* componentOrData,
00065 std::list<ModelDataDefinition*>* visited,
00066 std::map<unsigned int, std::map<unsigned int,
00067 std::list<std::string>*>*> dotmap) const;
00067
00072 bool createModelImage() const;
00073
00074 private:
00075 Simulator* _simulator;
00076 QLabel* _modelGraphicLabel;
00077 QCheckBox* _showStatistics;
00078 QCheckBox* _showEditable;
00079 QCheckBox* _showShared;
00080 QCheckBox* _showRecursive;
00081 QCheckBox* _showLevels;
00082 // Keep synchronization dependency narrow and explicit.
00083 std::function<bool()> _ensureModelSynchronized;
00084 };
00085
00086 #endif // GRAPHVIZMODELEXPORTER_H

```

## 16.92 IssueReportRelayService.h

```

00001 #ifndef ISSUEREPORTRELAYSERVICE_H
00002 #define ISSUEREPORTRELAYSERVICE_H
00003
00004 #include <QByteArray>
00005 #include <QString>
00006
00010 struct IssueReportDraft {
00011 QString categoryKey;
00012 QString categoryLabel;
00013 QString title;
00014 QString summary;
00015 QString reproductionSteps;
00016 QString expectedBehavior;
00017 QString observedBehavior;
00018 QString additionalContext;
00019 QString contactEmail;
00020 bool includeScreenshot = false;
00021 bool includeLogTail = true;
00022 bool includeModelText = false;
00023 };

```

```

00024
00028 struct IssueReportRuntimeContext {
00029 QString applicationName;
00030 QString applicationVersion;
00031 QString qtVersion;
00032 QString operatingSystem;
00033 QString currentModelName;
00034 QString currentModelSimulanguage;
00035 QString consoleTail;
00036 QString simulationTail;
00037 QString reportsTail;
00038 QString processLogTail;
00039 QByteArray screenshotPngBase64;
00040 };
00041
00045 struct IssueReportSubmissionResult {
00046 bool success = false;
00047 int httpStatus = 0;
00048 int issueNumber = 0;
00049 QString issueUrl;
00050 QString errorMessage;
00051 QString responseBody;
00052 };
00053
00067 class IssueReportRelayService {
00068 public:
00069 static QString configuredEndpoint();
00070 static QString optionalClientKey();
00071 static int requestTimeoutMs();
00072 static bool isEndpointConfigured(QString* reason = nullptr);
00073 static IssueReportSubmissionResult submit(const IssueReportDraft& draft,
00074 const IssueReportRuntimeContext& runtimeContext);
00075 };
00076
00077 #endif // ISSUEREPORTRELAYSERVICE_H

```

## 16.93 ModellLanguageSynchronizer.h

```

00001 #ifndef MODELLLANGUAGESYNCHRONIZER_H
00002 #define MODELLLANGUAGESYNCHRONIZER_H
00003
00004 #include <functional>
00005
00006 class QWidget;
00007 class QPlainTextEdit;
00008 class Simulator;
00009
00010 // Document the service boundary for textual model representation synchronization.
00028 class ModellLanguageSynchronizer {
00029 public:
00033 ModellLanguageSynchronizer(Simulator* simulator,
00034 QPlainTextEdit* modelTextEditor,
00035 bool* textModelHasChangedFlag,
00036 QWidget* ownerWidget,
00037 std::function<void()> onModelCreatedOrLoaded);
00038
00044 void actualizeModelSimLanguage() const;
00045
00050 bool setSimulationModelBasedOnText() const;
00051
00052 private:
00053 Simulator* _simulator;
00054 QPlainTextEdit* _modelTextEditor;
00055 bool* _textModelHasChangedFlag;
00056 QWidget* _ownerWidget;
00057 std::function<void()> _onModelCreatedOrLoaded;
00058 };
00059
00060 #endif // MODELLLANGUAGESYNCHRONIZER_H

```

## 16.94 ToolLauncher.h

```

00001 #pragma once
00002
00003 #include <QString>
00004 #include <QStringList>
00005
00006 class QWidget;
00007
00008 namespace ToolLauncher {
00009
00010 bool launchDetached(QWidget* parent,

```

```

00011 const QString& executableBaseName,
00012 const QString& displayName,
00013 const QStringList& arguments = {},
00014 const QStringList& relativeSearchDirs = {});
00015
00016 } // namespace ToolLauncher

```

## 16.95 systempreferences.h

```

00001 #ifndef SYSTEMPREFERENCES_H
00002 #define SYSTEMPREFERENCES_H
00003
00004 #include "kernel/simulator/TraceManager.h"
00005
00006 #include <QString>
00007 #include <string>
00008 #include <vector>
00009
00017 class SystemPreferences {
00018 public:
00022 enum class StartupModelMode : unsigned short {
00023 NoModel = 0,
00024 NewModel = 1,
00025 OpenSpecificModel = 2,
00026 OpenLastModel = 3
00027 };
00028
00032 enum class VisualTheme {
00033 Light,
00034 Dark
00035 };
00036
00040 enum class InterfaceStyle {
00041 Classic,
00042 Modern
00043 };
00044
00048 enum class AutomaticPositioningStrategy {
00049 Legacy = 0,
00050 Centered = 1
00051 };
00052
00054 static bool load();
00056 static bool save();
00058 static void resetToDefaults();
00060 static QString configFilePath();
00061
00063 static bool startMaximized();
00065 static void setStartMaximized(bool newStartMaximized);
00066
00068 static bool autoLoadPlugins();
00070 static void setAutoLoadPlugins(bool newAutoLoadPlugins);
00071
00073 static StartupModelMode startupModelMode();
00075 static void setStartupModelMode(StartupModelMode newStartupModelMode);
00077 static unsigned short modelAtStart();
00079 static void setModelAtStart(unsigned short newModelAtStart);
00080
00082 static std::string modelfilename();
00084 static void setModelfilename(const std::string &newModelfilename);
00086 static std::string lastModelFilename();
00088 static void setLastModelFilename(const std::string& newLastModelFilename);
00090 static std::vector<std::string> recentModelFiles();
00092 static void setRecentModelFiles(const std::vector<std::string>& recentModelFiles);
00094 static void pushRecentModelFile(const std::string& modelFilename);
00096 static unsigned int recentModelFilesLimit();
00098 static void setRecentModelFilesLimit(unsigned int recentModelFilesLimit);
00099
00101 static bool checkSystemPackagesAtStart();
00103 static void setCheckSystemPackagesAtStart(bool newCheckSystemPackagesAtStart);
00104
00106 static TraceManager::Level traceLevel();
00108 static void setTraceLevel(TraceManager::Level newTraceLevel);
00109
00111 static VisualTheme visualTheme();
00113 static void setVisualTheme(VisualTheme newVisualTheme);
00115 static std::string visualThemeName();
00116
00118 static InterfaceStyle interfaceStyle();
00120 static void setInterfaceStyle(InterfaceStyle newInterfaceStyle);
00122 static std::string interfaceStyleName();
00123
00125 static int applicationFontPointSize();
00127 static void setApplicationFontPointSize(int newApplicationFontPointSize);
00128

```

```

00130 static bool diagramUsesThemeColors();
00132 static void setDiagramUsesThemeColors(bool newDiagramUsesThemeColors);
00134 static AutomaticPositioningStrategy automaticPositioningStrategy();
00136 static void setAutomaticPositioningStrategy(AutomaticPositioningStrategy strategy);
00137
00139 static unsigned int canvasBackgroundColor();
00141 static unsigned int canvasDisabledBackgroundColor();
00143 static unsigned int gridColor();
00144
00145 private:
00146 SystemPreferences(){};
00147
00148 static QString defaultConfigDirectory();
00149 static StartupModelMode startupModelModeFromString(const QString& value);
00150 static QString startupModelModeToString(StartupModelMode value);
00151 static VisualTheme visualThemeFromString(const QString& value);
00152 static QString visualThemeToString(VisualTheme value);
00153 static InterfaceStyle interfaceStyleFromString(const QString& value);
00154 static QString interfaceStyleToString(InterfaceStyle value);
00155 static AutomaticPositioningStrategy automaticPositioningStrategyFromString(const QString& value);
00156 static QString automaticPositioningStrategyToString(AutomaticPositioningStrategy value);
00157 static TraceManager::Level traceLevelFromInt(int value);
00158
00159 static bool _startMaximized;
00160 static bool _autoLoadPlugins;
00161 static bool _checkSystemPackagesAtStart;
00162 static StartupModelMode _startupModelMode;
00163 static std::string _modelfilenameToOpen;
00164 static std::string _lastModelFilename;
00165 static std::vector<std::string> _recentModelFiles;
00166 static unsigned int _recentModelFilesLimit;
00167 static TraceManager::Level _traceLevel;
00168 static VisualTheme _visualTheme;
00169 static InterfaceStyle _interfaceStyle;
00170 static int _applicationFontPointSize;
00171 static bool _diagramUsesThemeColors;
00172 static AutomaticPositioningStrategy _automaticPositioningStrategy;
00173 };
00174
00175 #endif // SYSTEMPREFERENCES_H

```

## 16.96 DataAnalyzerWindow.h

```

00001 #ifndef DATAANALYZERWINDOW_H
00002 #define DATAANALYZERWINDOW_H
00003
00004 #include <QList>
00005 #include <QMainWindow>
00006 #include <QString>
00007 #include <QStringList>
00008
00009 class QGroupBox;
00010 class QCheckBox;
00011 class QComboBox;
00012 class QDoubleSpinBox;
00013 class QLabel;
00014 class QListWidget;
00015 class QPlainTextEdit;
00016 class QSpinBox;
00017 class QSplitter;
00018 class QTableWidget;
00019 class QTabWidget;
00020 class QTextEdit;
00021 class QTextBrowser;
00022 class Simulator;
00023
00024 class DataAnalyzerWindow : public QMainWindow {
00025 public:
00026 explicit DataAnalyzerWindow(Simulator* simulator, QString& lastDataAnalyzerPath, QWidget* parent =
 nullptr);
00027
00028 private:
00029 struct DataSummary {
00030 int count = 0;
00031 double min = 0.0;
00032 double max = 0.0;
00033 double mean = 0.0;
00034 double median = 0.0;
00035 QString mode;
00036 double firstQuartile = 0.0;
00037 double thirdQuartile = 0.0;
00038 double variance = 0.0;
00039 double stddev = 0.0;
00040 double cv = 0.0;
00041 double kurtosis = 0.0;

```

```

00042 };
00043
00044 struct DatasetObservation {
00045 int replication = 1;
00046 double time = 0.0;
00047 bool hasTime = false;
00048 double value = 0.0;
00049 };
00050
00051 struct DatasetDescriptor {
00052 QString datasetName;
00053 QString randomVariableName;
00054 QString description;
00055 QString variableType;
00056 QString source;
00057 QStringList previewLines;
00058 QList<double> values;
00059 bool raw = true;
00060 QList<DatasetObservation> observations;
00061 bool recordFile = false;
00062 bool timeDependent = false;
00063 QString formatKind;
00064 QString expression;
00065 QString expressionName;
00066 QString formatVersion;
00067 };
00068
00069 class HistogramPreview;
00070 class DoePreviewPlot;
00071
00072 void buildMenus();
00073 void buildWorkspace();
00074 void connectActions();
00075 void refreshModelResponses();
00076 void openDataset();
00077 void importSimulationResponsesSnapshot();
00078 void loadDatasetFromFile(const QString& fileName);
00079 void addDataset(const DatasetDescriptor& dataset);
00080 void filterDataset();
00081 void refreshDatasetList();
00082 void refreshScopeSelector();
00083 void showAnalysisSetupDialog(const QString& analysisName, int targetTabIndex);
00084 void refreshAnalysisViews();
00085 QList<double> scopedValues() const;
00086 QList<DatasetObservation> scopedObservations() const;
00087 QString scopedDatasetLabel() const;
00088 QString scopedDatasetDescription() const;
00089 QStringList scopedPreviewLines() const;
00090 void refreshStudySummaryView();
00091 void refreshDescriptiveView(const DataSummary& summary);
00092 void refreshDistributionFitView(const DataSummary& summary);
00093 void refreshInferenceView(const DataSummary& summary);
00094 void refreshReportView(const DataSummary& summary, const QString& fitConclusion);
00095 void refreshDoeDemoViews();
00096 void refreshDoePlanSummary();
00097 void exportDataset();
00098 void saveReport();
00099 void showSkeletonMessage(const QString& featureName);
00100
00101 static DataSummary summarizeValues(const QList<double>& values);
00102 static QList<double> sortedValues(const QList<double>& values);
00103 static double percentile(const QList<double>& sorted, double probability);
00104 static QString exactMode(const QList<double>& values);
00105 static QString formatNumber(double value);
00106
00107 private:
00108 Simulator* _simulator = nullptr;
00109 QString& _lastDataAnalyzerPath;
00110 QString _studyName;
00111 QList<DatasetDescriptor> _datasets;
00112 QString _lastFitConclusion;
00113
00114 QAction* _openDatasetAction = nullptr;
00115 QAction* _saveReportAction = nullptr;
00116 QAction* _importSimulationResponsesAction = nullptr;
00117 QAction* _refreshModelResponsesAction = nullptr;
00118 QAction* _exportDatasetAction = nullptr;
00119
00120 QListWidget* _datasetsList = nullptr;
00121 QListWidget* _analysisNavigator = nullptr;
00122 QTableWidget* _modelResponsesTable = nullptr;
00123 QTabWidget* _workspaceTabs = nullptr;
00124 QComboBox* _scopeCombo = nullptr;
00125 QLabel* _datasetSourceLabel = nullptr;
00126 QPlainTextEdit* _datasetPreview = nullptr;
00127 QTableWidget* _studySummaryTable = nullptr;
00128 QTableWidget* _centralTendencyTable = nullptr;

```

```

00129 QTableWidgetItem* _dispersionTable = nullptr;
00130 QTableWidgetItem* _rawDataTable = nullptr;
00131 QTableWidgetItem* _movingAverageTable = nullptr;
00132 HistogramPreview* _histogramPreview = nullptr;
00133 QTableWidgetItem* _fitTable = nullptr;
00134 QLabel* _fitConclusionLabel = nullptr;
00135 QTableWidgetItem* _inferenceTable = nullptr;
00136 QTableWidgetItem* _doeFactorsTable = nullptr;
00137 QTableWidgetItem* _doeResponsesTable = nullptr;
00138 QTableWidgetItem* _doeDesignTable = nullptr;
00139 QTableWidgetItem* _doeAnovaTable = nullptr;
00140 QTableWidgetItem* _doeOptimizationTable = nullptr;
00141 QTableWidgetItem* _doeAdviceTable = nullptr;
00142 QTableWidgetItem* _doeDesignQualityTable = nullptr;
00143 QTableWidgetItem* _doeDesirabilityTable = nullptr;
00144 QComboBox* _doeDesignFamilyCombo = nullptr;
00145 QComboBox* _doeModelOrderCombo = nullptr;
00146 QComboBox* _doePlotSelector = nullptr;
00147 QSpinBox* _doeReplicationsSpin = nullptr;
00148 QDoubleSpinBox* _doeAlphaSpin = nullptr;
00149 QCheckBox* _doeRandomizeCheck = nullptr;
00150 QLabel* _doeAdvisorLabel = nullptr;
00151 QLabel* _doePlanSummaryLabel = nullptr;
00152 DoePreviewPlot* _doePreviewPlot = nullptr;
00153 QTextBrowser* _startPage = nullptr;
00154 QTextEdit* _reportText = nullptr;
00155 };
00156
00157 #endif // DATAANALYZERWINDOW_H

```

## 16.97 ExpressionBuilder.h

```

00001 #ifndef EXPRESSIONBUILDER_H
00002 #define EXPRESSIONBUILDER_H
00003
00004 #include <QDialog>
00005 #include <QTreeWidget>
00006 #include <QListWidget>
00007 #include <QFormLayout>
00008 #include <QLineEdit>
00009 #include <QPushButton>
00010 #include <QClipboard>
00011 #include <QApplication>
00012 #include <QVBoxLayout>
00013 #include <QHBoxLayout>
00014 #include <QLabel>
00015 #include <QString>
00016 #include <QMap>
00017 #include <QList>
00018
00019 namespace Ui {
00020 class ExpressionBuilder;
00021 }
00022
00023 struct ExpressionParameter {
00024 QString name;
00025 QString defaultValue;
00026 QString description;
00027 };
00028
00029 struct ExpressionTemplate {
00030 QString name;
00031 QString pattern;
00032 QString description;
00033 QList<ExpressionParameter> parameters;
00034 };
00035
00036 struct ExpressionCategory {
00037 QString name;
00038 QList<ExpressionTemplate> expressions;
00039 };
00040
00041 class ExpressionBuilder : public QDialog
00042 {
00043 Q_OBJECT
00044
00045 public:
00046 explicit ExpressionBuilder(QWidget *parent = nullptr);
00047 ~ExpressionBuilder();
00048
00049 private slots:
00050 void on_categorySelected(QTreeWidgetItem *item, int column);
00051 void on_expressionSelected(QListWidgetItem *item);
00052 void on_copyToClipboard_clicked();
00053 void on_close_clicked();

```

```

00054
00055 private:
00056 Ui::ExpressionBuilder *ui;
00057
00058 // Expression data
00059 QList<ExpressionCategory> categories;
00060
00061 // UI Components
00062 QTreeWidget *treeCategories;
00063 QListWidget *listExpressions;
00064 QWidget *parameterWidget;
00065 QFormLayout *parameterLayout;
00066 QLabel *expressionPreview;
00067 QPushButton *btnCopyToClipboard;
00068 QPushButton *btnClose;
00069
00070 // Parameter inputs
00071 QMap<QString, QLineEdit*> parameterInputs;
00072
00073 // Current selection
00074 ExpressionTemplate *currentExpression;
00075
00076 // Initialize expression categories
00077 void initializeCategories();
00078 void updateParameterForm();
00079 void updateExpressionPreview();
00080 QString buildExpression();
00081
00082 // Extensibility: Register new categories and expressions
00083 void registerCategory(const ExpressionCategory &category);
00084 void registerExpression(const QString &categoryName, const ExpressionTemplate &expression);
00085 };
00086
00087 #endif // EXPRESSIONBUILDER_H

```

## 16.98 OptimizerWindow.h

```

00001 #ifndef OPTIMIZERWINDOW_H
00002 #define OPTIMIZERWINDOW_H
00003
00004 #include "tools/Optimization/OptimizerDefaultImpl1.h"
00005
00006 #include <QList>
00007 #include <QMainWindow>
00008 #include <QString>
00009 #include <QVector>
00010
00011 #include <vector>
00012
00013 class QAction;
00014 class QCheckBox;
00015 class QComboBox;
00016 class QDoubleSpinBox;
00017 class QLabel;
00018 class QPlainTextEdit;
00019 class QPushButton;
00020 class QSpinBox;
00021 class QTableWidget;
00022 class QTableWidgetItem;
00023 class QTabWidget;
00024 class QTextBrowser;
00025 class Simulator;
00026 class SimulationControl;
00027 class SimulationResponse;
00028
00029 class OptimizerWindow : public QMainWindow {
00030 public:
00031 explicit OptimizerWindow(Simulator* simulator, QWidget* parent = nullptr);
00032
00033 private:
00034 class ProgressPlot;
00035
00036 void buildMenus();
00037 void buildWorkspace();
00038 void connectActions();
00039 void refreshFromCurrentModel();
00040 void refreshControlsTable();
00041 void refreshResponsesTable();
00042 void refreshReport();
00043 void applyConfigurationToBackend();
00044 void refreshRunState();
00045 void addObjective();
00046 void removeSelectedObjective();
00047 void addConstraint();
00048 void removeSelectedConstraint();

```

```

00049 void configureTechnique();
00050 void checkReadiness();
00051 void startOptimization();
00052 void stepOptimization();
00053 void pauseOptimization();
00054 void resumeOptimization();
00055 void stopOptimization();
00056 void showSkeletonMessage(const QString& featureName);
00057
00058 static QString formatNumber(double value);
00059 static QString stateText(Optimizer_if::ExecutionState state);
00060 static QString techniqueDescription(const QString& technique);
00061 static bool stringIsNumeric(const QString& value);
00062 static QTableWidgetItem* readOnlyItem(const QString& text);
00063 static QTableWidgetItem* editableItem(const QString& text);
00064 static QTableWidgetItem* checkableItem(bool checked, bool enabled = true);
00065
00066 private:
00067 Simulator* _simulator = nullptr;
00068 OptimizerDefaultImpl1 _optimizer;
00069 std::vector<SimulationControl*> _controls;
00070 std::vector<SimulationResponse*> _responses;
00071 QVector<double> _progressValues;
00072
00073 QAction* _refreshModelAction = nullptr;
00074 QAction* _configureTechniqueAction = nullptr;
00075 QAction* _checkAction = nullptr;
00076 QAction* _startAction = nullptr;
00077 QAction* _stepAction = nullptr;
00078 QAction* _pauseAction = nullptr;
00079 QAction* _resumeAction = nullptr;
00080 QAction* _stopAction = nullptr;
00081
00082 QTabWidget* _tabs = nullptr;
00083 QLabel* _modelStatusLabel = nullptr;
00084 QLabel* _runStatusLabel = nullptr;
00085 QTableWidgetItem* _controlsTable = nullptr;
00086 QTableWidgetItem* _responsesTable = nullptr;
00087 QTableWidgetItem* _objectivesTable = nullptr;
00088 QTableWidgetItem* _constraintsTable = nullptr;
00089 QTableWidgetItem* _bestSolutionsTable = nullptr;
00090 QComboBox* _techniqueCombo = nullptr;
00091 QSpinBox* _maxIterationsSpin = nullptr;
00092 QSpinBox* _maxSimulationsSpin = nullptr;
00093 QSpinBox* _replicationsSpin = nullptr;
00094 QSpinBox* _bestSolutionsSpin = nullptr;
00095 QSpinBox* _randomSeedSpin = nullptr;
00096 QSpinBox* _populationSizeSpin = nullptr;
00097 QDoubleSpinBox* _mutationRateSpin = nullptr;
00098 QDoubleSpinBox* _crossoverRateSpin = nullptr;
00099 QDoubleSpinBox* _improvementToleranceSpin = nullptr;
00100 QDoubleSpinBox* _timeLimitSpin = nullptr;
00101 QPlainTextEdit* _techniqueNotes = nullptr;
00102 QTextBrowser* _overviewText = nullptr;
00103 QTextBrowser* _reportText = nullptr;
00104 ProgressPlot* _progressPlot = nullptr;
00105 };
00106
00107 #endif // OPTIMIZERWINDOW_H

```

## 16.99 TraitsGUI.h

```

00001 #ifndef TRAITSGUI_H
00002 #define TRAITSGUI_H
00003
00004 #include <stdint>
00005 #include <string>
00006
00013 template <class T>
00014 struct TraitsGUI {
00015 };
00016
00017 // include only features for this GUI, such as theme, default widgets, ...
00018
00019 class GMainWindow;
00020
00021 template <>
00022 struct TraitsGUI<GMainWindow> {
00023 static const bool startMaximized = true;
00024 static const int zoomButtonChange = 30;
00025 static const uint64_t consoleTextColor = 0x008000FF;
00026 };
00027
00028 class GModelGraphicPic;
00029

```



```

00030 template <>
00031 struct TraitsGUI<GModelGraphicPic> {
00032 static const int nodeComponentFontSize = 12;
00033 static const uint64_t nodeComponentFontColor = 0x000000FF;
00034 static const uint64_t nodeComponentFillColor = 0x80A040FF;
00035 static const int edgeComponentFontSize = 7;
00036 static const uint64_t edgeComponentEdgeColor = 0x000000FF;
00037 static const uint64_t edgeComponentFontColor = 0x000000FF;
00038 static const int nodeDatadefInternalFontSize = 8;
00039 static const uint64_t nodeDatadefInternalColor = 0xA0A0A0FF;
00040 static const uint64_t nodeDatadefInternalFontColor = 0xA0A0A0FF;
00041 static const int nodeDatadefAttachedFontSize = 8;
00042 static const uint64_t nodeDatadefAttachedFontColor = 0x808080FF;
00043 };
00044
00045 class GAssociation;
00046
00047 template <>
00048 struct TraitsGUI<GAssociation> {
00049 static const uint64_t foreColor = 0x808080FF;
00050 constexpr static const int margin = 2;
00051 constexpr static const int selectionWidth = 8;
00052 };
00053
00054 class GConnection;
00055
00056 template <>
00057 struct TraitsGUI<GConnection> {
00058 static const uint64_t foreColor = 0x000000FF;
00059 constexpr static const int margin = 2;
00060 constexpr static const int selectionWidth = 8;
00061 };
00062
00063 class GView;
00064
00065 template <>
00066 struct TraitsGUI<GView> {
00067 constexpr static const int sceneCenter = 25000;
00068 constexpr static const int sceneDistanceCenter = 25000;
00069 static const uint64_t backgroundEnabledColor = 0xFFFF8040;
00070 static const uint64_t backgroundDisabledColor = 0xC0C0C020;
00071 };
00072
00073 class GScene;
00074
00075 template <>
00076 struct TraitsGUI<GScene> {
00077 constexpr static const int gridInterval = 20;
00078 constexpr static const int gridPenWidth = 1;
00079 static const uint64_t gridColor = 0xC0C0C0E0;
00080 };
00081
00082 class GComponentPort;
00083
00084 template <>
00085 struct TraitsGUI<GComponentPort> {
00086 constexpr static const int width = 20;
00087 constexpr static const int penWidth = 1;
00088 constexpr static const int raise = 3;
00089 constexpr static const int margin = 2;
00090 static const uint64_t connectedBackgroundColor = 0x2020FF80;
00091 static const uint64_t unconnectedBackgroundColor = 0xFF202040;
00092 static const uint64_t borderColor = 0x000000FF;
00093 static const uint64_t pathRaised = 0x808080FF;
00094 static const uint64_t pathStunken = 0xFFFFFFFF;
00095 static const uint64_t textColor = 0xFFFFFFFF;
00096 static const uint64_t textShadowColor = 0x000000FF;
00097 static const uint64_t selectionSquaresColor = 0x000000FF;
00098 };
00099
00100 class GModelDataDefinition;
00101
00102 template <>
00103 struct TraitsGUI<GModelDataDefinition> {
00104 //<GraphicalModelComponent> {
00105 constexpr static const int opacity = 256 * 0.33;
00106 constexpr static const int nonEditableOpacity = 58;
00107 constexpr static const int selectedNonEditableOpacity = 72;
00108 constexpr static const int width = 100;
00109 constexpr static const int penWidth = 1;
00110 constexpr static const int raise = 3;
00111 constexpr static const int margin = 5;
00112 constexpr static const int selectionWidth = 8;
00113 constexpr static const float stretchPos = 0.5;
00114 constexpr static const float stretch = 0.0;
00115 constexpr static const float heightProportion = 0.50;
00116 static const uint64_t breakpointColor = 0xFF0000FF;

```

```

00117 static const uint64_t borderColor = 0x000000FF;
00118 static const uint64_t pathRaised = 0x808080FF;
00119 static const uint64_t pathStunken = 0xFFFFFFFF;
00120 static const uint64_t textColor = 0x0; // 0xFFFFFFFF;
00121 static const uint64_t textShadowColor = 0x000000FF;
00122 static const uint64_t selectionSquaresColor = 0x000000FF;
00123 };
00124
00125 class GModelComponent;
00126
00127 template <>
00128 struct TraitsGUI<GModelComponent> {
00129 //<GraphicalModelComponent> {
00130 constexpr static const int opacity = 256.0 * 0.67;
00131 constexpr static const int colorIncrease = 256.0 * 0.2;
00132 // 20% more some color component for sources, sinks, transfers
00133 constexpr static const int lighter = 133; // 33% lighter for some components
00134 constexpr static const int width = 150;
00135 constexpr static const int penWidth = 1;
00136 constexpr static const int raise = 5;
00137 constexpr static const int margin = 8;
00138 constexpr static const int selectionWidth = 8;
00139 constexpr static const float stretchPos = 0.5;
00140 constexpr static const float stretch = 0.0;
00141 constexpr static const float heightProportion = 0.67;
00142 static const uint64_t breakpointColor = 0xFF0000FF;
00143 static const uint64_t borderColor = 0x000000FF;
00144 static const uint64_t pathRaised = 0xFFFFFFFF;
00145 static const uint64_t pathStunken = 0x808080FF;
00146 static const uint64_t textColor = 0xFFFFFFFF;
00147 static const uint64_t textShadowColor = 0x000000FF;
00148 static const uint64_t selectionSquaresColor = 0x000000FF;
00149 };
00150
00151 #endif // TRAITSGUI_H

```

## 16.100 UtilGUI.h

```

00001 #ifndef UTILGUI_H
00002 #define UTILGUI_H
00003 #include <QColor>
00004 #include <stdint>
00005 #include <sstream>
00006 #include <string>
00007
00008 namespace UtilGUI {
00009
00017 inline QColor rgbaFromPacked(uint64_t color) {
00018 uint8_t r = (color & 0xFF000000) » 24;
00019 uint8_t g = (color & 0x00FF0000) » 16;
00020 uint8_t b = (color & 0x0000FF00) » 8;
00021 uint8_t a = (color & 0x000000FF);
00022 return QColor(r, g, b, a);
00023 }
00024
00032 inline std::string dotColorFromPacked(uint64_t color) {
00033 std::stringstream stream;
00034 stream << std::hex << "#" << color;
00035 return stream.str();
00036 }
00037
00038 }
00039
00040 #endif // UTILGUI_H

```

## 16.101 WebWorkerDialog.h

```

00001 #pragma once
00002
00003 #include <QDialog>
00004 #include <memory>
00005
00006 class QCloseEvent;
00007 class QShowEvent;
00008 class QPlainTextEdit;
00009 class QSpinBox;
00010 class QLabel;
00011 class WebWorkerRuntime;
00012
00016 class WebWorkerDialog : public QDialog {
00017 Q_OBJECT
00018

```

```

00019 public:
00020 explicit WebWorkerDialog(QWidget* parent = nullptr, bool allowClose = false);
00021 ~WebWorkerDialog() override;
00022
00023 protected:
00024 void closeEvent(QCloseEvent* event) override;
00025 void showEvent(QShowEvent* event) override;
00026
00027 private:
00028 void _refresh();
00029
00030 bool _allowClose = false;
00031 std::unique_ptr<WebWorkerRuntime> _runtime;
00032 QSpinBox* _portSpin = nullptr;
00033 QSpinBox* _maxRequestsSpin = nullptr;
00034 QLabel* _statusValue = nullptr;
00035 QLabel* _phaseValue = nullptr;
00036 QLabel* _urlValue = nullptr;
00037 QLabel* _servedValue = nullptr;
00038 QLabel* _activeValue = nullptr;
00039 QLabel* _stateValue = nullptr;
00040 QPlainTextEdit* _historyText = nullptr;
00041 QPlainTextEdit* _errorText = nullptr;
00042 };

```

## 16.102 Airport\_Extended1.h

```

00001 #ifndef AIRPORT_EXTENDED1_H
00002 #define AIRPORT_EXTENDED1_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00006 class Airport_Extended1 : public BaseGenesysTerminalApplication {
00007 public:
00008 Airport_Extended1();
00009 public:
00010 virtual int main(int argc, char** argv) override;
00011 };
00012
00013 #endif // SMART_FAILURES_H

```

## 16.103 AirportSecurityExample.h

```

00001 #pragma once
00002
00003 #include "../BaseGenesysTerminalApplication.h"
00004
00005 class AirportSecurityExample : public BaseGenesysTerminalApplication {
00006 public:
00007 AirportSecurityExample();
00008 virtual int main(int argc, char** argv) override;
00009 };

```

## 16.104 AirportSecurityExampleExtended.h

```

00001 #pragma once
00002
00003 #include "../BaseGenesysTerminalApplication.h"
00004
00005 class AirportSecurityExampleExtended : public BaseGenesysTerminalApplication {
00006 public:
00007 AirportSecurityExampleExtended();
00008 virtual int main(int argc, char** argv) override;
00009 };

```

## 16.105 Assembly\_Line.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_Delay.h
00009 * Author: rlcancian
00010 */

```

```

00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef ASSEMBLY_LINE_H
00015 #define ASSEMBLY_LINE_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Assembly_Line : public BaseGenesysTerminalApplication {
00020 public:
00021 Assembly_Line();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_DELAY_H */
00027

```

## 16.106 Banking\_Transactions.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_Dummy.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef BANKING_TRANSACTIONS_H
00015 #define BANKING_TRANSACTIONS_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Banking_Transactions : public BaseGenesysTerminalApplication {
00020 public:
00021 Banking_Transactions();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* BANKING_TRANSACTIONS_H */
00027

```

## 16.107 Example\_BasicOrderShipping.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Example_PublicTransport_SARA.h
00009 * Author: Bruno & Henrique
00010 *
00011 * Created 2022-2 DS3
00012 * */
00013
00014 #ifndef EXAMPLE_BASIC_ORDER_SHIPPING_H
00015 #define EXAMPLE_BASIC_ORDER_SHIPPING_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Example_BasicOrderShipping : public BaseGenesysTerminalApplication {
00020 public:
00021 Example_BasicOrderShipping();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 };
00027
00028 #endif /* EXAMPLE_BASIC_ORDER_SHIPPING_H */
00029

```

## 16.108 Example\_PortModel.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007
00008 #ifndef EXAMPLE_PORTMODEL_H
00009 #define EXAMPLE_PORTMODEL_H
00010
00011 #include "../BaseGenesysTerminalApplication.h"
00012
00013 class Example_PortModel : public BaseGenesysTerminalApplication {
00014 public:
00015 Example_PortModel();
00016 public:
00017 virtual int main(int argc, char** argv) override;
00018 };
00019
00020 #endif /* EXAMPLE_PORTMODEL_H */
00021

```

## 16.109 Example\_PublicTransport.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Example_PublicTransport.h
00009 * Author: Bruno & Henrique
00010 *
00011 * Created
00012 * */
00013
00014 #ifndef EXAMPLE_PUBLIC_TRANSPORT_H
00015 #define EXAMPLE_PUBLIC_TRANSPORT_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Example_PublicTransport : public BaseGenesysTerminalApplication {
00020 public:
00021 Example_PublicTransport();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024
00025
00026 };
00027
00028 #endif /* EXAMPLE_PUBLIC_TRANSPORT_H */
00029

```

## 16.110 Smart\_AddingResource.h

```

00001 /*
00002 * File: AddingResource.h
00003 * Author: genesys
00004 *
00005 * Created on 10 de dezembro de 2022, 17:57
00006 */
00007
00008 #ifndef ADDINGRESOURCE_H
00009 #define ADDINGRESOURCE_H
00010
00011 #include "../BaseGenesysTerminalApplication.h"
00012
00013 class Smart_AddingResource : public BaseGenesysTerminalApplication {
00014 public:
00015 Smart_AddingResource();
00016 public:
00017 virtual int main(int argc, char** argv) override;
00018 };
00019
00020 #endif /* ADDINGRESOURCE_H */
00021

```

## 16.111 Smart\_AlternatingEntityCreation.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_AlternatingEntityCreation.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_ALTERNATINGENTITYCREATION_H
00015 #define SMART_ALTERNATINGENTITYCREATION_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_AlternatingEntityCreation : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_AlternatingEntityCreation();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_ALTERNATINGENTITYCREATION_H */

```

## 16.112 Smart\_ArrivalsElementStopsEntitiesArrivingAfterASetTime.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime_H
00015 #define Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime : public BaseGenesysTerminalApplication
00020 {
00021 public:
00022 Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime();
00023 public:
00024 virtual int main(int argc, char** argv) override;
00025 };
00026 #endif /* Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime_H */

```

## 16.113 Smart\_ArrivalsEntityTypeVsAttribute.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/file.h to edit this template
00004 */
00005
00006 /*
00007 * File: Smart_ArrivalsEntityTypeVsAttribute.h
00008 * Author: genesys
00009 *
00010 * Created on 16 de dezembro de 2022, 15:22
00011 */
00012
00013 #ifndef SMART_ARRIVALSENTITYTYPEVSATTRIBUTE_H
00014 #define SMART_ARRIVALSENTITYTYPEVSATTRIBUTE_H
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class Smart_ArrivalsEntityTypeVsAttribute : public BaseGenesysTerminalApplication {
00019 public:
00020 Smart_ArrivalsEntityTypeVsAttribute();
00021 virtual int main(int argc, char** argv) override;

```

```

00022 };
00023
00024 #endif /* SMART_ARRIVALSENTITYTYPEVSATTRIBUTE_H */
00025

```

## 16.114 Smart\_AssignExample.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/file.h to edit this template
00004 */
00005
00006 /*
00007 * File: Smart_AssignExample.h
00008 * Author: genesys
00009 *
00010 * Created on 15 de dezembro de 2022, 23:46
00011 */
00012
00013 #ifndef SMART_ASSIGNEXAMPLE_H
00014 #define SMART_ASSIGNEXAMPLE_H
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class Smart_AssignExample : public BaseGenesysTerminalApplication {
00019 public:
00020 Smart_AssignExample();
00021 virtual int main(int argc, char** argv) override;
00022 };
00023
00024
00025 #endif /* SMART_ASSIGNEXAMPLE_H */
00026

```

## 16.115 Smart\_AutomaticStatisticsCollection.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_AutomaticStatisticsCollection.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef Smart_AutomaticStatisticsCollection_H
00015 #define Smart_AutomaticStatisticsCollection_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_AutomaticStatisticsCollection : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_AutomaticStatisticsCollection();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* Smart_AutomaticStatisticsCollection_H */

```

## 16.116 Smart\_BasicModeling.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: BasicExample.h
00008 * Author: genesys
00009 *
00010 * Created on 12 de dezembro de 2022, 12:38
00011 */
00012
00013 #ifndef SMART_BASICMODELING_H
00014 #define SMART_BASICMODELING_H

```

```

00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class Smart_BasicModeling : public BaseGenesysTerminalApplication {
00019 public:
00020 Smart_BasicModeling();
00021 virtual int main(int argc, char** argv) override;
00022 };
00023
00024
00025 #endif /* SMART_BASICMODELING_H */
00026

```

## 16.117 Smart\_BatchAndSeparate.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/file.h to edit this template
00004 */
00005
00006 /*
00007 * File: Smart_BatchAndSeparate.h
00008 * Author: genesys
00009 *
00010 * Created on 16 de dezembro de 2022, 00:26
00011 */
00012
00013 #ifndef SMART_BATCHANDSEPARATE_H
00014 #define SMART_BATCHANDSEPARATE_H
00015
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_BatchAndSeparate : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_BatchAndSeparate();
00022 virtual int main(int argc, char** argv) override;
00023 };
00024
00025 #endif /* SMART_BATCHANDSEPARATE_H */
00026

```

## 16.118 Smart\_ContinuousFlowEntities.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_ModellingLearningCurve.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_CONTINUOUSFLOWENTITIES_H
00015 #define SMART_CONTINUOUSFLOWENTITIES_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ContinuousFlowEntities : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_ContinuousFlowEntities();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_DUMMY_H */
00027

```

## 16.119 Smart\_Create.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */

```



```

00006
00007 /*
00008 * File: Smart_Create.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_CREATE_H
00015 #define SMART_CREATE_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Create : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_Create();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_CREATE_H */
00027

```

## 16.120 Smart\_DecideNWayByChance.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_DecideNWayByChance.h
00009 * Author: guifrrs and LuizValdiero
00010 *
00011 * Created on December 3, 2022, 8:04 PM
00012 */
00013
00014 #ifndef SMART_DECIDENWAYBYCHANCE_H
00015 #define SMART_DECIDENWAYBYCHANCE_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_DecideNWayByChance : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_DecideNWayByChance();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_DECIDENWAYBYCHANCE_H */
00027

```

## 16.121 Smart\_DefiningAttributesAsStrings.h

```

00001 /*
00002 * File: Smart_MaxArrivalsField.h
00003 * Author: genesys
00004 *
00005 * Created on 10 de dezembro de 2022, 15:34
00006 */
00007
00008 #ifndef SMART_DEFININGATTRIBUTESASSTRINGS_H
00009 #define SMART_DEFININGATTRIBUTESASSTRINGS_H
00010
00011 #include "../BaseGenesysTerminalApplication.h"
00012
00013 class Smart_DefiningAttributesAsStrings: public BaseGenesysTerminalApplication {
00014 public:
00015 Smart_DefiningAttributesAsStrings();
00016 public:
00017 virtual int main(int argc, char** argv) override;
00018 };
00019
00020 #endif /* SMART_DEFININGATTRIBUTESASSTRINGS_H */
00021

```

## 16.122 Smart\_DefiningControlLogic.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_DefiningControlLogic.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef Smart_DEFININGCONTROLLOGIC_H
00015 #define Smart_DEFININGCONTROLLOGIC_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018 #include "plugins/components/DiscreteProcessing/Delay.h"
00019
00020
00021 class Smart_DefiningControlLogic : public BaseGenesysTerminalApplication {
00022 public:
00023 Smart_DefiningControlLogic();
00024 public:
00025 virtual int main(int argc, char** argv) override;
00026 };
00027
00028 #endif /* Smart_DEFININGCONTROLLOGIC_H */

```

## 16.123 Smart\_DefiningResourceCapacity.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_DefiningResourceCapacity.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef Smart_DefiningResourceCapacity_H
00015 #define Smart_DefiningResourceCapacity_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_DefiningResourceCapacity : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_DefiningResourceCapacity();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* Smart_DefiningResourceCapacity_H */

```

## 16.124 Smart\_DelayBasedOnReplication.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_DelayBasedOnReplication.h
00009 * Author: João Janini
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef Smart_DELAYBASEDONREPLICATION_H
00015 #define Smart_DELAYBASEDONREPLICATION_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018 #include "plugins/components/DiscreteProcessing/Delay.h"
00019
00020

```

```

00021 class Smart_DelayBasedOnReplication : public BaseGenesysTerminalApplication {
00022 public:
00023 Smart_DelayBasedOnReplication();
00024 void TestEventHandler(SimulationEvent *re);
00025 public:
00026 virtual int main(int argc, char** argv) override;
00027 Delay* delay;
00028 int *proctimes;
00029 };
00030
00031 #endif /* Smart_DelayBasedOnReplication_H */

```

## 16.125 Smart\_EntitiesProcessedByPriority.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_EntitiesProcessedByPriority.h
00009 * Author: bepasar
00010 *
00011 * Created on 09/12/22
00012 */
00013
00014 #ifndef SMART_ENTITIESPROCESSEDBYPRIORITY_H
00015 #define SMART_ENTITIESPROCESSEDBYPRIORITY_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_EntitiesProcessedByPriority : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_EntitiesProcessedByPriority();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_ENTITIESPROCESSEDBYPRIORITY_H */
00027

```

## 16.126 Smart\_EvaluatingConditionsBeforeEnteringQueue.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_EvaluatingConditionsBeforeEnteringQueue.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef Smart_EvaluatingConditionsBeforeEnteringQueue_H
00015 #define Smart_EvaluatingConditionsBeforeEnteringQueue_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_EvaluatingConditionsBeforeEnteringQueue : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_EvaluatingConditionsBeforeEnteringQueue();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* Smart_EvaluatingConditionsBeforeEnteringQueue_H */

```

## 16.127 Smart\_Expression.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006

```

```

00007 /*
00008 * File: Smart_Expression.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_EXPRESSION_H
00015 #define SMART_EXPRESSION_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Expression : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_Expression();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_EXPRESSION_H */
00027

```

## 16.128 Smart\_InventoryAndHoldingCosts.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_Dummy.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_INVENTORYANDHOLDINGCOSTS_H
00015 #define SMART_INVENTORYANDHOLDINGCOSTS_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_InventoryAndHoldingCosts : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_InventoryAndHoldingCosts();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_INVENTORYANDHOLDINGCOSTS_H */
00027

```

## 16.129 Smart\_MaxArrivalsField.h

```

00001 /*
00002 * File: Smart_MaxArrivalsField.h
00003 * Author: genesys
00004 *
00005 * Created on 10 de dezembro de 2022, 15:34
00006 */
00007
00008 #ifndef SMART_MAXARRIVALSFIELD_H
00009 #define SMART_MAXARRIVALSFIELD_H
00010
00011 #include "../BaseGenesysTerminalApplication.h"
00012
00013 class Smart_MaxArrivalsField : public BaseGenesysTerminalApplication {
00014 public:
00015 Smart_MaxArrivalsField();
00016 public:
00017 virtual int main(int argc, char** argv) override;
00018 };
00019
00020 #endif /* SMART_MAXARRIVALSFIELD_H */
00021

```

## 16.130 Smart\_ModelRunUntil1000Parts.h

```

00001 /*

```

```

00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_ModelRunUntil1000Parts.h
00009 * Author: guifrrs and LuizValdiero
00010 *
00011 * Created on December 3, 2022, 8:04 PM
00012 */
00013
00014 #ifndef SMART_MODELRUNUNTIL1000PARTS_H
00015 #define SMART_MODELRUNUNTIL1000PARTS_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ModelRunUntil1000Parts : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_ModelRunUntil1000Parts();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_MODELRUNUNTIL1000PARTS_H */

```

## 16.131 Smart\_ModuleDisplayVariables.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_ModuleDisplayVariables.h
00009 * Author: bepasar
00010 *
00011 * Created on 06/12/22, 21:27
00012 */
00013
00014 #ifndef SMART_MODULEDISPLAYVARIABLES_H
00015 #define SMART_MODULEDISPLAYVARIABLES_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ModuleDisplayVariables : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_ModuleDisplayVariables();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_MODULEDISPLAYVARIABLES_H */
00027

```

## 16.132 Smart\_OverlappingResources.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007
00008 #ifndef SMART_OVERLAPPINGRESOURCES_H
00009 #define SMART_OVERLAPPINGRESOURCES_H
00010
00011 #include "../BaseGenesysTerminalApplication.h"
00012
00013 class Smart_OverlappingResources : public BaseGenesysTerminalApplication {
00014 public:
00015 Smart_OverlappingResources();
00016 public:
00017 virtual int main(int argc, char** argv) override;
00018 };
00019
00020 #endif /* SMART_OVERLAPPINGRESOURCES_H */
00021

```

## 16.133 Smart\_ParallelProcessingOfEntities.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_ParallelProcessingOfEntities.h
00009 * Author: Luiz e Guilherme
00010 *
00011 * Created on Dezembro de 2022
00012 */
00013
00014 #ifndef SMART_PARALLEL_PROCESSING_OF_ENTITIES_H
00015 #define SMART_PARALLEL_PROCESSING_OF_ENTITIES_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ParallelProcessingOfEntities : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_ParallelProcessingOfEntities();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_PARALLEL_PROCESSING_OF_ENTITIES_H */

```

## 16.134 Smart\_PlacingEntitiesInQueueSets.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_PlacingEntitiesInQueueSets.h
00009 * Author: Joçao Janini
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_PLACINGENTITIESINQUEUESETS_H
00015 #define SMART_PLACINGENTITIESINQUEUESETS_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_PlacingEntitiesInQueueSets : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_PlacingEntitiesInQueueSets();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_PLACINGENTITIESINQUEUESETS_H */

```

## 16.135 Smart\_PriorityExample.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_Dummy.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_PRIORITYEXAMPLE_H
00015 #define SMART_PRIORITYEXAMPLE_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_PriorityExample : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_PriorityExample();
00022 public:

```

```

00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_DUMMY_H */
00027

```

## 16.136 Smart\_ProcessArena.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_ProcessArena.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_PROCESSARENA_H
00015 #define SMART_PROCESSARENA_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ProcessArena : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_ProcessArena();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_PROCESSARENA_H */
00027

```

## 16.137 Smart\_Record\_Arena.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_Record_Arena.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef Smart_Record_Arena_ARENA_H
00015 #define Smart_Record_Arena_ARENA_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Record_Arena : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_Record_Arena();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* Smart_Record_Arena_ARENA_H */
00027

```

## 16.138 Smart\_RemovingAndReorderingEntitiesInAQueue.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007
00008 #ifndef SMART_REMOVINGANDREORDERINGENTITIESINAQUEUE_H
00009 #define SMART_REMOVINGANDREORDERINGENTITIESINAQUEUE_H
00010
00011 #include "../BaseGenesysTerminalApplication.h"

```

```

00012
00013 class Smart_RemovingAndReorderingEntitiesInAQueue : public BaseGenesysTerminalApplication {
00014 public:
00015 Smart_RemovingAndReorderingEntitiesInAQueue();
00016 public:
00017 virtual int main(int argc, char** argv) override;
00018 };
00019
00020 #endif /* SMART_REMOVINGANDREORDERINGENTITIESINAQUEUE_H */
00021

```

## 16.139 Smart\_ResourceCosting.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_Dummy.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_RESOURCECOSTING_H
00015 #define SMART_RESOURCECOSTING_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ResourceCosting : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_ResourceCosting();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_RESOURCECOSTING_H */
00027

```

## 16.140 Smart\_ResourceScheduleCosting.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_Dummy.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_RESOURCESCHEDULECOSTING_H
00015 #define SMART_RESOURCESCHEDULECOSTING_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ResourceScheduleCosting : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_ResourceScheduleCosting();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_RESOURCESCHEDULECOSTING_H */
00027

```

## 16.141 Smart\_ResourceSets.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006

```



```

00007 /*
00008 * File: Smart_ResourceSets.h
00009 * Author: guifrrs and LuizValdiero
00010 *
00011 * Created on December 3, 2022, 8:04 PM
00012 */
00013
00014 #ifndef SMART_RESOURCESETS_H
00015 #define SMART_RESOURCESETS_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ResourceSets : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_ResourceSets();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_RESOURCESETS_H */
00027

```

## 16.142 Smart\_SeizingMultipleSimultaneosly.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_SeizingMultipleSimultaneosly.h
00009 * Author: douglaspereira04
00010 *
00011 */
00012
00013 #ifndef SMART_SEIZINGMULTIPLESIMULTANEOSLY_H
00014 #define SMART_SEIZINGMULTIPLESIMULTANEOSLY_H
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class Smart_SeizingMultipleSimultaneosly : public BaseGenesysTerminalApplication {
00019 public:
00020 Smart_SeizingMultipleSimultaneosly();
00021 public:
00022 virtual int main(int argc, char** argv) override;
00023 };
00024 #endif /* SMART_SEIZINGMULTIPLESIMULTANEOSLY_H */

```

## 16.143 Smart\_SelectingRouteBasedOnProbability.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_WaitSignal.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_SELECTINGROUTEBASEDONPROBABILITY_H
00015 #define SMART_SELECTINGROUTEBASEDONPROBABILITY_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_SelectingRouteBasedOnProbability : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_SelectingRouteBasedOnProbability();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_SELECTINGROUTEBASEDONPROBABILITY_H */
00027

```

## 16.144 Smart\_SelectingShorterQueue.h

```

00001 /*
00002 * File: Smart_SelectingShorterQueue.h
00003 * Author: genesys
00004 *
00005 * Created on 10 de dezembro de 2022, 17:59
00006 */
00007
00008 #ifndef SMART_SELECTINGSHORTERQUEUE_H
00009 #define SMART_SELECTINGSHORTERQUEUE_H
00010
00011 #include "../BaseGenesysTerminalApplication.h"
00012
00013 class Smart_SelectingShorterQueue : public BaseGenesysTerminalApplication {
00014 public:
00015 Smart_SelectingShorterQueue();
00016 public:
00017 virtual int main(int argc, char** argv) override;
00018 };
00019
00020 #endif /* SMART_SELECTINGSHORTERQUEUE_H */
00021

```

## 16.145 Smart\_SynchronizingParallelEntities.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_SynchronizingParallelEntities.h
00009 * Author:
00010 *
00011 */
00012
00013 #ifndef SMART_SYNCHRONIZINGPARALLELENTITIES_H
00014 #define SMART_SYNCHRONIZINGPARALLELENTITIES_H
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class Smart_SynchronizingParallelEntities : public BaseGenesysTerminalApplication {
00019 public:
00020 Smart_SynchronizingParallelEntities();
00021 public:
00022 virtual int main(int argc, char** argv) override;
00023 };
00024 #endif /* SMART_SYNCHRONIZINGPARALLELENTITIES_H */
00025

```

## 16.146 Smart\_ValueAdded.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_Dummy.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_VALUEADDED_H
00015 #define SMART_VALUEADDED_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ValueAdded : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_ValueAdded();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_VALUEADDED_H */
00027

```

## 16.147 Smart\_WaitForSignal.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_WaitForSignal.h
00009 * Author: douglaspereira04
00010 *
00011 */
00012
00013 #ifndef SMART_WAITFORSIGNAL_H
00014 #define SMART_WAITFORSIGNAL_H
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class Smart_WaitForSignal : public BaseGenesysTerminalApplication {
00019 public:
00020 Smart_WaitForSignal();
00021 public:
00022 virtual int main(int argc, char** argv) override;
00023 };
00024 #endif /* SMART_WAITFORSIGNAL_H */

```

## 16.148 Book\_Cap02\_Example01.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: Book_Cap02_Example01.h
00008 * Author: rlcancian
00009 *
00010 * Created on 19 de outubro de 2021, 16:23
00011 */
00012
00013 #ifndef BOOK_CAP02_EXAMPLE01_H
00014 #define BOOK_CAP02_EXAMPLE01_H
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class Book_Cap02_Example01 : public BaseGenesysTerminalApplication {
00019 public:
00020 Book_Cap02_Example01();
00021 virtual int main(int argc, char** argv) override;
00022 };
00023
00024 #endif /* BOOK_CAP02_EXAMPLE01_H */
00025

```

## 16.149 StartAIAssistant.h

```

00001 #pragma once
00002
00003 #include "../BaseGenesysTerminalApplication.h"
00004
00005 class StartAIAssistant : public BaseGenesysTerminalApplication {
00006 public:
00007 StartAIAssistant();
00008 public:
00009 virtual int main(int argc, char** argv) override;
00010 };

```

## 16.150 Smart\_MarkovChain.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_MarkovChain.h
00009 * Author: rlcancian

```

```

00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_MARKOVCHAIN_H
00015 #define SMART_MARKOVCHAIN_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_MarkovChain : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_MarkovChain();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_MARKOVCHAIN_H */
00027

```

## 16.151 Smart\_BacteriaColony\_GRO.h

```

00001 /*
00002 * File: Smart_BacteriaColony_GRO.h
00003 * Author: GRO
00004 *
00005 * Created on 1 de Maio de 2026
00006 */
00007
00008 #ifndef SMART_BACTERIACOLONY_GRO_H
00009 #define SMART_BACTERIACOLONY_GRO_H
00010
00011 #include "../BaseGenesysTerminalApplication.h"
00012
00013 class Smart_BacteriaColony_GRO : public BaseGenesysTerminalApplication {
00014 public:
00015 Smart_BacteriaColony_GRO();
00016
00017 public:
00018 virtual int main(int argc, char** argv) override;
00019 };
00020
00021 #endif /* SMART_BACTERIACOLONY_GRO_H */

```

## 16.152 Smart\_BioKineticLawRegulation.h

```

00001 /*
00002 * File: Smart_BioKineticLawRegulation.h
00003 */
00004
00005 #ifndef SMART_BIOKINETICLAWREGULATION_H
00006 #define SMART_BIOKINETICLAWREGULATION_H
00007
00008 #include "../BaseGenesysTerminalApplication.h"
00009
00010 class Smart_BioKineticLawRegulation : public BaseGenesysTerminalApplication {
00011 public:
00012 Smart_BioKineticLawRegulation();
00013 public:
00014 virtual int main(int argc, char** argv) override;
00015 };
00016
00017 #endif /* SMART_BIOKINETICLAWREGULATION_H */

```

## 16.153 Smart\_BioNetworkAnalysis.h

```

00001 /*
00002 * File: Smart_BioNetworkAnalysis.h
00003 */
00004
00005 #ifndef SMART_BIONETWORKANALYSIS_H
00006 #define SMART_BIONETWORKANALYSIS_H
00007
00008 #include "../BaseGenesysTerminalApplication.h"
00009
00010 class Smart_BioNetworkAnalysis : public BaseGenesysTerminalApplication {
00011 public:
00012 Smart_BioNetworkAnalysis();
00013 public:
00014 virtual int main(int argc, char** argv) override;

```

```
00015 };
00016
00017 #endif /* SMART_BIONETWORKANALYSIS_H */
```

## 16.154 Smart\_BioReversibleKineticLaw.h

```
00001 /*
00002 * File: Smart_BioReversibleKineticLaw.h
00003 */
00004
00005 #ifndef SMART_BIOREVERSIBLEKINETICLAW_H
00006 #define SMART_BIOREVERSIBLEKINETICLAW_H
00007
00008 #include "../.../BaseGenesysTerminalApplication.h"
00009
00010 class Smart_BioReversibleKineticLaw : public BaseGenesysTerminalApplication {
00011 public:
00012 Smart_BioReversibleKineticLaw();
00013 public:
00014 virtual int main(int argc, char** argv) override;
00015 };
00016
00017 #endif /* SMART_BIOREVERSIBLEKINETICLAW_H */
```

## 16.155 Smart\_BioReversibleMassAction.h

```
00001 /*
00002 * File: Smart_BioReversibleMassAction.h
00003 */
00004
00005 #ifndef SMART_BIOREVERSIBLEMASSACTION_H
00006 #define SMART_BIOREVERSIBLEMASSACTION_H
00007
00008 #include "../.../BaseGenesysTerminalApplication.h"
00009
00010 class Smart_BioReversibleMassAction : public BaseGenesysTerminalApplication {
00011 public:
00012 Smart_BioReversibleMassAction();
00013 public:
00014 virtual int main(int argc, char** argv) override;
00015 };
00016
00017 #endif /* SMART_BIOREVERSIBLEMASSACTION_H */
```

## 16.156 Smart\_GroColonyGrowth.h

```
00001 /*
00002 * File: Smart_GroColonyGrowth.h
00003 * Author: GRO
00004 *
00005 * Created on 17 de Abril de 2026
00006 */
00007
00008 #ifndef SMART_GROCOLONYGROWTH_H
00009 #define SMART_GROCOLONYGROWTH_H
00010
00011 #include "../.../BaseGenesysTerminalApplication.h"
00012
00013 class Smart_GroColonyGrowth : public BaseGenesysTerminalApplication {
00014 public:
00015 Smart_GroColonyGrowth();
00016
00017 public:
00018 virtual int main(int argc, char** argv) override;
00019 };
00020
00021 #endif /* SMART_GROCOLONYGROWTH_H */
```

## 16.157 Smart\_GroColonyLifecycle.h

```
00001 /*
00002 * File: Smart_GroColonyLifecycle.h
00003 * Author: GRO
00004 *
00005 * Created on 17 de Abril de 2026
00006 */
00007
```

```

00008 #ifndef SMART_GROCOLONYLIFECYCLE_H
00009 #define SMART_GROCOLONYLIFECYCLE_H
00010
00011 #include "../././BaseGenesysTerminalApplication.h"
00012
00013 class Smart_GroColonyLifecycle : public BaseGenesysTerminalApplication {
00014 public:
00015 Smart_GroColonyLifecycle();
00016
00017 public:
00018 virtual int main(int argc, char** argv) override;
00019 };
00020
00021 #endif /* SMART_GROCOLONYLIFECYCLE_H */

```

## 16.158 Smart\_LCODE.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: Smart_ODE.h
00008 * Author: rlcancian
00009 *
00010 * Created on 11 de janeiro de 2022, 19:29
00011 */
00012
00013 #ifndef SMART_LCODE_H
00014 #define SMART_LCODE_H
00015
00016
00017 #include "../././BaseGenesysTerminalApplication.h"
00018
00019 class Smart_LCODE : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_LCODE();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_LCODE_H */
00027

```

## 16.159 Smart\_ODE.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: Smart_ODE.h
00008 * Author: rlcancian
00009 *
00010 * Created on 11 de janeiro de 2022, 19:29
00011 */
00012
00013 #ifndef SMART_ODE_H
00014 #define SMART_ODE_H
00015
00016
00017 #include "../././BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ODE : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_ODE();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_ODE_H */
00027

```

## 16.160 Smart\_Buffer.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.

```

```

00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_Buffer.h
00009 * Author: rlcancian
00010 *
00011 * Created on
00012 */
00013
00014 #ifndef SMART_BUFFER_H
00015 #define SMART_BUFFER_H
00016
00017 #include "../.../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Buffer : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_Buffer();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_BUFFER_H */
00027

```

## 16.161 Smart\_Clone.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_Clone.h
00009 * Author: rlcancian
00010 *
00011 * Created on 30 de Novembro de 2022, 19:25
00012 */
00013
00014 #ifndef SMART_CLONE_H
00015 #define SMART_CLONE_H
00016
00017 #include "../.../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Clone : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_Clone();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_CLONE_H */
00027

```

## 16.162 Smart\_Delay.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_Delay.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_DELAY_H
00015 #define SMART_DELAY_H
00016
00017 #include "../.../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Delay : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_Delay();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };

```

```
00025
00026 #endif /* SMART_DELAY_H */
00027
```

## 16.163 Smart\_Failures.h

```
00001 #ifndef SMART_FAILURES_H
00002 #define SMART_FAILURES_H
00003
00004 #include "../../BaseGenesysTerminalApplication.h"
00005
00006 class Smart_Failures : public BaseGenesysTerminalApplication {
00007 public:
00008 Smart_Failures();
00009 public:
00010 virtual int main(int argc, char** argv) override;
00011 };
00012
00013 #endif // SMART_FAILURES_H
```

## 16.164 Smart\_HoldSearchRemove.h

```
00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_HoldSearchRemove.h
00009 * Author: rlcancian
00010 *
00011 * Created on 01 de Dezembro de 2022, 09:31
00012 */
00013
00014 #ifndef SMART_HOLDSEARCHREMOVE_H
00015 #define SMART_HOLDSEARCHREMOVE_H
00016
00017 #include "../../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_HoldSearchRemove : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_HoldSearchRemove();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_HOLDSEARCHREMOVE_H */
00027
```

## 16.165 Smart\_Process.h

```
00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_Process.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_PROCESS_H
00015 #define SMART_PROCESS_H
00016
00017 #include "../../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Process : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_Process();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_PROCESS_H */
00027
```



## 16.166 Smart\_ProcessSet.h

```
00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_ProcessSet.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_PROCESSSET_H
00015 #define SMART_PROCESSSET_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ProcessSet : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_ProcessSet();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_PROCESSSET_H */
00027
```

## 16.167 Smart\_SeizeDelayRelease.h

```
00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_SeizeDelayRelease.h
00009 * Author: rlcancian
00010 *
00011 * Created on 24 de Setembro de 2019, 16:43
00012 */
00013
00014 #ifndef SMART_SEIZEDELAYRELEASE_H
00015 #define SMART_SEIZEDELAYRELEASE_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_SeizeDelayRelease : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_SeizeDelayRelease();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025 #endif /* SMART_SEIZEDELAYRELEASE_H */
00026
```

## 16.168 Smart\_SeizeDelayReleaseMany.h

```
00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_SeizeDelayReleaseMany.h
00009 * Author: rlcancian
00010 *
00011 * Created on 9 de abril de 2021, 18:09
00012 */
00013
00014 #ifndef MODEL_SEIZEDELAYRELEASEMANY_H
00015 #define MODEL_SEIZEDELAYRELEASEMANY_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_SeizeDelayReleaseMany : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_SeizeDelayReleaseMany();
00022 }
```

```

00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* MODEL_SEIZEDELAYRELEASEMANY_H */
00027

```

## 16.169 Smart\_SeizeDelayReleaseNoDataDefs.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: Smart_SeizeDelayReleaseNoDataDefs.h
00008 * Author: rlcancian
00009 *
00010 * Created on 23 de maio de 2022, 12:39
00011 */
00012
00013 #ifndef SMART_SEIZEDELAYRELEASENODATADEFS_H
00014 #define SMART_SEIZEDELAYRELEASENODATADEFS_H
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class Smart_SeizeDelayReleaseNoDataDefs : public BaseGenesysTerminalApplication {
00019 public:
00020 Smart_SeizeDelayReleaseNoDataDefs();
00021 public:
00022 virtual int main(int argc, char** argv) override;
00023 };
00024
00025 #endif /* SMART_SEIZEDELAYRELEASENODATADEFS_H */
00026

```

## 16.170 Smart\_CppForG.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: Smart_CppForG.h
00008 * Author: rlcancian
00009 *
00010 * Created on 11 de janeiro de 2022, 22:39
00011 */
00012
00013 #pragma once
00014
00015 #include "../BaseGenesysTerminalApplication.h"
00016
00017 class Smart_CppForG : public BaseGenesysTerminalApplication {
00018 public:
00019 Smart_CppForG();
00020 public:
00021 virtual int main(int argc, char** argv) override;
00022 };
00023

```

## 16.171 Smart\_PythonForG.h

```

00001 #ifndef SMART_PYTHONFORG_H
00002 #define SMART_PYTHONFORG_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00006 class Smart_PythonForG : public BaseGenesysTerminalApplication {
00007 public:
00008 Smart_PythonForG();
00009 virtual int main(int argc, char** argv) override;
00010 };
00011
00012 #endif /* SMART_PYTHONFORG_H */

```

## 16.172 Smart\_R\_Simulator.h

```

00001 /*
00002 * File: Smart_R_Simulator.h
00003 * Author: GenESyS
00004 *
00005 * Minimal smart model for the RSimulator component.
00006 */
00007
00008 #ifndef SMART_R_SIMULATOR_H
00009 #define SMART_R_SIMULATOR_H
00010
00011 #include "../BaseGenesysTerminalApplication.h"
00012
00019 class Smart_R_Simulator : public BaseGenesysTerminalApplication {
00020 public:
00022 Smart_R_Simulator();
00023
00024 public:
00026 virtual int main(int argc, char** argv) override;
00027 };
00028
00029 #endif /* SMART_R_SIMULATOR_H */

```

## 16.173 Smart\_BatchSeparate.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: Smart_BatchSeparate.h
00008 * Author: rlcancian
00009 *
00010 * Created on 15 de janeiro de 2022, 09:51
00011 */
00012
00013 #ifndef SMART_BATCHSEPARATE_H
00014 #define SMART_BATCHSEPARATE_H
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class Smart_BatchSeparate : public BaseGenesysTerminalApplication {
00019 public:
00020 Smart_BatchSeparate();
00021
00022 public:
00022 virtual int main(int argc, char** argv) override;
00023 };
00024
00025 #endif /* SMART_BATCHSEPARATE_H */
00026

```

## 16.174 Smart\_Record.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_Record.h
00009 * Author: rlcancian
00010 *
00011 * Created on
00012 */
00013
00014 #pragma once
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class Smart_Record : public BaseGenesysTerminalApplication {
00019 public:
00020 Smart_Record();
00021
00022 public:
00022 virtual int main(int argc, char** argv) override;
00023 };
00024
00025

```

## 16.175 Smart\_AssignWriteSeizes.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_AssignWriteSeizes.h
00009 * Author: rlcancian
00010 *
00011 * Created on 24 de Setembro de 2019, 20:56
00012 */
00013
00014 #ifndef SMART_ASSIGNWRITE3SEIZES_H
00015 #define SMART_ASSIGNWRITE3SEIZES_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_AssignWriteSeizes : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_AssignWriteSeizes();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_ASSIGNWRITE3SEIZES_H */
00027

```

## 16.176 Smart\_Attribute\_Variable.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_Attribute_Variable.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_ATTRIBUTE_VARIABLE_H
00015 #define SMART_ATTRIBUTE_VARIABLE_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Attribute_Variable : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_Attribute_Variable();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_ATTRIBUTE_VARIABLE_H */

```

## 16.177 Smart\_ModelInfoModelSimulation.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_ModelInfoModelSimulation.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:59
00012 */
00013
00014 #ifndef SMART_MODELINFOMODELSIMULATION_H
00015 #define SMART_MODELINFOMODELSIMULATION_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ModelInfoModelSimulation : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_ModelInfoModelSimulation();

```

```

00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_MODELINFOMODELSIMULATION_H */
00027

```

## 16.178 Smart\_Parser.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_Parser.h
00009 * Author: rlcancian
00010 *
00011 * Created on 5 de Janeiro de 2022, 07:21
00012 */
00013
00014 #ifndef SMART_PARSER_H
00015 #define SMART_PARSER_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Parser : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_Parser();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_PARSER_H */
00027

```

## 16.179 Smart\_ParserModelFunctions.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_ParserModelFunctions.h
00009 * Author: rlcancian
00010 *
00011 * Created on 5 de Janeiro de 2022, 07:21
00012 */
00013
00014 #ifndef SMART_PARSERMODELFUNCTIONS_H
00015 #define SMART_PARSERMODELFUNCTIONS_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ParserModelFunctions : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_ParserModelFunctions();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_PARSERMODELFUNCTIONS_H */
00027

```

## 16.180 Smart\_Plugin.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: Smart_Plugin.h
00008 * Author: rlcancian
00009 *
00010 * Created on 13 de janeiro de 2022, 17:47

```

```
00011 */
00012
00013 #ifndef SMART_PLUGIN_H
00014 #define SMART_PLUGIN_H
00015
00016 #include "../.../BaseGenesysTerminalApplication.h"
00017
00018 class Smart_Plugin : public BaseGenesysTerminalApplication {
00019 public:
00020 Smart_Plugin();
00021 public:
00022 virtual int main(int argc, char** argv) override;
00023 };
00024
00025 #endif /* SMART_PLUGIN_H */
00026
```

## 16.181 Smart\_RouteStation.h

```
00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_RouteStation.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_ROUTESTATION_H
00015 #define SMART_ROUTESTATION_H
00016
00017 #include "../.../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_RouteStation : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_RouteStation();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_ROUTESTATION_H */
00027
```

## 16.182 Smart\_Sequence.h

```
00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_Sequence.h
00009 * Author: rlcancian
00010 *
00011 * Created on 12 de março de 2021, 15:39
00012 */
00013
00014 #ifndef SMART_SEQUENCE_H
00015 #define SMART_SEQUENCE_H
00016
00017 #include "../.../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Sequence : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_Sequence();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_SEQUENCE_H */
00027
```

## 16.183 Smart\_CellularAutomata.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_CellularAutomata.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_CELLULARAUTOMATA_H
00015 #define SMART_CELLULARAUTOMATA_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_CellularAutomata : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_CellularAutomata();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_CELLULARAUTOMATA_H */
00027

```

## 16.184 Smart\_CellularAutomata1.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_CellularAutomata.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2025, 18:34
00012 */
00013
00014 #ifndef SMART_CELLULARAUTOMATA1_H
00015 #define SMART_CELLULARAUTOMATA1_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_CellularAutomata1 : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_CellularAutomata1();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_CELLULARAUTOMATA_H */

```

## 16.185 Smart\_DefaultModalModel.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007
00008 #pragma once
00009
00010 #include "../BaseGenesysTerminalApplication.h"
00011
00012 class Smart_DefaultModalModel : public BaseGenesysTerminalApplication {
00013 public:
00014 Smart_DefaultModalModel();
00015 public:
00016 virtual int main(int argc, char** argv) override;
00017 };
00018
00019

```

## 16.186 Smart\_Dummy.h

```
00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_Dummy.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_DUMMY_H
00015 #define SMART_DUMMY_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Dummy : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_Dummy();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_DUMMY_H */
00027
```

## 16.187 Smart\_FSMState.h

```
00001 #pragma once
00002
00003 #include "../BaseGenesysTerminalApplication.h"
00004
00005 class Smart_FSMState : public BaseGenesysTerminalApplication {
00006 public:
00007 Smart_FSMState();
00008 public:
00009 virtual int main(int argc, char** argv) override;
00010 };

```

## 16.188 Smart\_ModelModel.h

```
00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_ModelModel.h
00009 * Author: rlcancian
00010 *
00011 * Created on 30 de Setembro de 2025, 17:46
00012 */
00013
00014 #ifndef SMART_MODELMODEL_H
00015 #define SMART_MODELMODEL_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ModelModel : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_ModelModel();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_MODELMODEL_H */
00027
00028
```

## 16.189 Smart\_ModelModelDefaultNetwork.h

```
00001 #pragma once
00002
00003 #include "../BaseGenesysTerminalApplication.h"

```



```

00004
00005 class Smart_ModelModelDefaultNetwork : public BaseGenesysTerminalApplication {
00006 public:
00007 Smart_ModelModelDefaultNetwork();
00008 public:
00009 virtual int main(int argc, char** argv) override;
00010 };

```

## 16.190 Smart\_ModelModelFSM.h

```

00001 #pragma once
00002
00003 #include "../BaseGenesysTerminalApplication.h"
00004
00005 class Smart_ModelModelFSM : public BaseGenesysTerminalApplication {
00006 public:
00007 Smart_ModelModelFSM();
00008 public:
00009 virtual int main(int argc, char** argv) override;
00010 };

```

## 16.191 Smart\_ModelModelPetriNet.h

```

00001 #pragma once
00002
00003 #include "../BaseGenesysTerminalApplication.h"
00004
00005 class Smart_ModelModelPetriNet : public BaseGenesysTerminalApplication {
00006 public:
00007 Smart_ModelModelPetriNet();
00008 public:
00009 virtual int main(int argc, char** argv) override;
00010 };

```

## 16.192 Smart\_PetriPlace.h

```

00001 #pragma once
00002
00003 #include "../BaseGenesysTerminalApplication.h"
00004
00005 class Smart_PetriPlace : public BaseGenesysTerminalApplication {
00006 public:
00007 Smart_PetriPlace();
00008 public:
00009 virtual int main(int argc, char** argv) override;
00010 };

```

## 16.193 Smart\_OnEvent.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_OnEvent.h
00009 * Author: rlcancian
00010 *
00011 * Created on 5 de Janeiro de 2021, 18:00
00012 */
00013
00014 #ifndef SMART_ONEVENT_H
00015 #define SMART_ONEVENT_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_OnEvent : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_OnEvent();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 public:
00025 virtual void onBreakpointHandler(SimulationEvent* re) override;
00026 virtual void onEntityCreateHandler(SimulationEvent* re) override;
00027 virtual void onEntityMoveHandler(SimulationEvent* re) override;
00028 virtual void onSimulationStartHandler(SimulationEvent* re) override;

```

```

00029 virtual void onReplicationStartHandler(SimulationEvent* re) override;
00030 virtual void onReplicationStepHandler(SimulationEvent* re) override;
00031 virtual void onProcessEventHandler(SimulationEvent* re) override;
00032 virtual void onReplicationEndHandler(SimulationEvent* re) override;
00033 virtual void onSimulationEndHandler(SimulationEvent* re) override;
00034 virtual void onSimulationPausedHandler(SimulationEvent* re) override;
00035 virtual void onSimulationResumeHandler(SimulationEvent* re) override;
00036 virtual void onEntityRemoveHandler(SimulationEvent* re) override;
00037 };
00038
00039 #endif /* SMART_ONEVENT_H */

```

## 16.194 Smart\_SimulationControlResponse.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: Smart_SimulationControlResponse.h
00008 * Author: rlcancian
00009 *
00010 * Created on 29 de novembro de 2022, 21:07
00011 */
00012
00013 #ifndef SMART_SIMULATIONCONTROLRESPONSE_H
00014 #define SMART_SIMULATIONCONTROLRESPONSE_H
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class Smart_SimulationControlResponse : public BaseGenesysTerminalApplication {
00019 public:
00020 Smart_SimulationControlResponse();
00021 public:
00022 virtual int main(int argc, char** argv) override;
00023 };
00024
00025 #endif /* SMART_SIMULATIONCONTROLRESPONSE_H */
00026

```

## 16.195 Smart\_WaitScanCondition.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: Smart_WaitScanCondition.h
00008 * Author: rlcancian
00009 *
00010 * Created on 14 de junho de 2022, 13:00
00011 */
00012
00013 #ifndef SMART_WAITSCANCONDITION_H
00014 #define SMART_WAITSCANCONDITION_H
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class Smart_WaitScanCondition : public BaseGenesysTerminalApplication {
00019 public:
00020 Smart_WaitScanCondition();
00021 public:
00022 virtual int main(int argc, char** argv) override;
00023 };
00024
00025 #endif /* SMART_WAITSCANCONDITION_H */
00026

```

## 16.196 Smart\_WaitSignal.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*

```

```

00008 * File: Smart_WaitSignal.h
00009 * Author: rlcancian
00010 *
00011 * Created on 3 de Setembro de 2019, 18:34
00012 */
00013
00014 #ifndef SMART_WAIT SIGNAL_H
00015 #define SMART_WAIT SIGNAL_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_WaitSignal : public BaseGenesysTerminalApplication {
00020 public:
00021 Smart_WaitSignal();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_WAIT SIGNAL_H */
00027

```

## 16.197 AnElectronicAssemblyAndTestSystem.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: Smart_AnElectronicAssemblyAndTestSystem.h
00008 * Author: rlcancian
00009 *
00010 * Created on 6 de dezembro de 2021, 11:09
00011 */
00012
00013 #ifndef ANELECTRONICASSEMBLYANDTESTSYSTEM_H
00014 #define ANELECTRONICASSEMBLYANDTESTSYSTEM_H
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class AnElectronicAssemblyAndTestSystem : public BaseGenesysTerminalApplication {
00019 public:
00020 AnElectronicAssemblyAndTestSystem();
00021 public:
00022 virtual int main(int argc, char** argv) override;
00023 };
00024
00025
00026 #endif /* ANELECTRONICASSEMBLYANDTESTSYSTEM_H */
00027

```

## 16.198 BufferFIFO.h

```

00001 #pragma once
00002
00003 /*
00004 * File: BufferFIFO.h
00005 * Author: rlcancian
00006 *
00007 * Created on 13 de setembro de 2023, 18:03
00008 */
00009
00010 #include "../BaseGenesysTerminalApplication.h"
00011
00012 #include "kernel/simulator/Simulator.h"
00013 #include "plugins/components/Logic/Create.h"
00014 #include "plugins/components/DiscreteProcessing/Clone.h"
00015 #include "plugins/components/Synchronization/Wait.h"
00016 #include "plugins/components/Synchronization/Signal.h"
00017 #include "plugins/components/Logic/Dispose.h"
00018 #include "plugins/components/Decisions/Decide.h"
00019 #include "plugins/data/Synchronization/SignalData.h"
00020 #include "plugins/data/Logic/Variable.h"
00021
00022 class BufferFIFO : public BaseGenesysTerminalApplication {
00023 public:
00024 BufferFIFO() {}
00025 public:
00026 virtual int main(int argc, char** argv) {
00027 Simulator* genesys = new Simulator();
00028 this->setDefaultTraceHandlers(genesys->getTraceManager());
00029 genesys->getPluginManager()->autoInsertPlugins("autoloadplugins.txt");
00030 }
00031 };

```

```

00030 Model* model = genesys->getModelManager()->newModel();
00031 PluginManager* plugins = genesys->getPluginManager();
00032
00033 Create* create1 = plugins->newInstance<Create>(model);
00034 create1->setTimeBetweenCreationsExpression("1", Util::TimeUnit::hour);
00035
00036 Clone* clone1 = plugins->newInstance<Clone>(model);
00037 clone1->setNumClonesExpression("1");
00038
00039 create1->getConnectionManager()->insert(clone1);
00040
00041 SignalData* sigdata1 = plugins->newInstance<SignalData>(model);
00042 Queue* queue1 = plugins->newInstance<Queue>(model, "waitingQueue");
00043 Wait* wait1 = plugins->newInstance<Wait>(model);
00044 wait1->setWaitType(Wait::WaitType::WaitForSignal);
00045 wait1->setSignalData(sigdata1);
00046 wait1->setQueue(queue1);
00047
00048 clone1->getConnectionManager()->insert(wait1);
00049
00050 Dispose* dispose1 = plugins->newInstance<Dispose>(model);
00051
00052 wait1->getConnectionManager()->insert(dispose1);
00053
00054 Decide* decide1 = plugins->newInstance<Decide>(model);
00055 decide1->getConditions()->insert("nq(waitingQueue)<tamBuffer");
00056 Variable* var1 = plugins->newInstance<Variable>(model, "tamBuffer");
00057 var1->setInitialValue(5);
00058
00059 clone1->getConnectionManager()->insert(decide1);
00060
00061 Dispose* dispose2 = plugins->newInstance<Dispose>(model);
00062 dispose2->setReportStatistics(false);
00063
00064 decide1->getConnectionManager()->insert(dispose2);
00065
00066 Signal* signal1 = plugins->newInstance<Signal>(model);
00067 signal1->setSignalData(sigdata1);
00068
00069 decide1->getConnectionManager()->insert(signal1);
00070 signal1->getConnectionManager()->insert(dispose2);
00071
00072 ModelSimulation* sim = model->getSimulation();
00073 sim->setReplicationLength(50, Util::TimeUnit::hour);
00074 sim->setReplicationReportBaseTimeUnit(Util::TimeUnit::hour);
00075
00076 model->save("./model/teachingBufferFIFO.gen");
00077 model->getTracer()->setTraceLevel(TraceManager::Level::L9_mostDetailed);
00078
00079 do {
00080 sim->step();
00081 std::cout << "Press ENTER to continue...";
00082 std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00083 } while (sim->isPaused());
00084 delete genesys;
00085 return 0;
00086 }
00087 };
00088

```

## 16.199 ContinuousModel.h

```

00001 #pragma once
00002
00003 #include "../BaseGenesysTerminalApplication.h"
00004
00005 class ContinuousModel: public BaseGenesysTerminalApplication {
00006 public:
00007 ContinuousModel();
00008 public: // virtual
00009 virtual void Simulate(Simulator* genesys, Model* model, PluginManager *plugins) override;
00010 };
00011

```

## 16.200 FullSimulationOfComplexModel.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006

```

```

00007 /*
00008 * File: buildSimpleModel1.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 2 de Outubro de 2018, 19:18
00012 */
00013
00014 #ifndef COMPLEXMODEL01_H
00015 #define COMPLEXMODEL01_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class FullSimulationOfComplexModel : public BaseGenesysTerminalApplication {
00020 public:
00021 FullSimulationOfComplexModel();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* COMPLEXMODEL01_H */
00027

```

## 16.201 Half\_Adder.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 #ifndef HALF_ADDER_H
00008 #define HALF_ADDER_H
00009
00010 #include "../BaseGenesysTerminalApplication.h"
00011
00012 class Half_Adder : public BaseGenesysTerminalApplication {
00013 public:
00014 Half_Adder();
00015 public:
00016 virtual int main(int argc, char** argv) override;
00017 };
00018
00019 #endif /* HALF_ADDER_H */
00020

```

## 16.202 Loja01.h

```

00001 #pragma once
00002
00003
00004 /*
00005 * File: Loja01.h
00006 * Author: rlcancian
00007 *
00008 * Created on 13 de Novembro de 2024, 10:25
00009 */
00010 #include "../BaseGenesysTerminalApplication.h"
00011
00012 class Loja01 : public BaseGenesysTerminalApplication {
00013 public:
00014 Loja01();
00015 public:
00016 virtual int main(int argc, char** argv) override;
00017 };
00018
00019
00020

```

## 16.203 OperatingSystem02.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_SistemasOperacionais02.h
00009 * Author: rlcancian

```

```
00010 *
00011 * Created on 12 de abril de 2021, 21:54
00012 */
00013
00014 #ifndef OPERATINGSYSTEMSS02_H
00015 #define OPERATINGSYSTEMSS02_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class OperatingSystem02 : public BaseGenesysTerminalApplication {
00020 public:
00021 OperatingSystem02();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* OPERATINGSYSTEMSS02_H */
00027
```

## 16.204 OperatingSystem03.h

```
00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Smart_SistemaOperacional03.h
00009 * Author: rlcancian
00010 *
00011 * Created on 23 de abril de 2021, 14:03
00012 */
00013
00014 #ifndef OPERATINGSYSTEMS03_H
00015 #define OPERATINGSYSTEMS03_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class OperatingSystem03 : public BaseGenesysTerminalApplication {
00020 public:
00021 OperatingSystem03();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 private:
00025
00026 };
00027
00028 #endif /* OPERATINGSYSTEMS03_H */
00029
```

## 16.205 Rectifier.h

```
00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 #ifndef RECTIFIER_H
00008 #define RECTIFIER_H
00009
00010 #include "../BaseGenesysTerminalApplication.h"
00011
00012 class Rectifier : public BaseGenesysTerminalApplication {
00013 public:
00014 Rectifier();
00015 public:
00016 virtual int main(int argc, char** argv) override;
00017 };
00018
00019 #endif /* HALF_ADDER_H */
00020
```

## 16.206 WcmBioNetworkBridge.h

```
00001 #ifndef WCBIONETWORKBRIDGE_H
00002 #define WCBIONETWORKBRIDGE_H
00003
```

```

00004 #include "../BaseGenesysTerminalApplication.h"
00005
00014 class WcmBioNetworkBridge : public BaseGenesysTerminalApplication {
00015 public:
00016 WcmBioNetworkBridge();
00017 virtual int main(int argc, char** argv) override;
00018 };
00019
00020 #endif /* WCMBIONETWORKBRIDGE_H */

```

## 16.207 WcmCompartmentTransportWholeCellMvp.h

```

00001 #ifndef WCMCOMPARTMENTTRANSPORTWHOLECELLMVP_H
00002 #define WCMCOMPARTMENTTRANSPORTWHOLECELLMVP_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00019 class WcmCompartmentTransportWholeCellMvp : public BaseGenesysTerminalApplication {
00020 public:
00021 WcmCompartmentTransportWholeCellMvp();
00022 virtual int main(int argc, char** argv) override;
00023 };
00024
00025 #endif /* WCMCOMPARTMENTTRANSPORTWHOLECELLMVP_H */

```

## 16.208 WcmFluxBalanceWholeCellMvp.h

```

00001 #ifndef WCMFLUXBALANCEWHOLECELLMVP_H
00002 #define WCMFLUXBALANCEWHOLECELLMVP_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00015 class WcmFluxBalanceWholeCellMvp : public BaseGenesysTerminalApplication {
00016 public:
00017 WcmFluxBalanceWholeCellMvp();
00018 virtual int main(int argc, char** argv) override;
00019 };
00020
00021 #endif /* WCMFLUXBALANCEWHOLECELLMVP_H */

```

## 16.209 WcmGeneExpression.h

```

00001 #ifndef WCMGENEEXPRESSION_H
00002 #define WCMGENEEXPRESSION_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00028 class WcmGeneExpression : public BaseGenesysTerminalApplication {
00029 public:
00030 WcmGeneExpression();
00031 virtual int main(int argc, char** argv) override;
00032 };
00033
00034 #endif /* WCMGENEEXPRESSION_H */

```

## 16.210 WcmLifecycleStress.h

```

00001 #ifndef WCMLIFECYCLESTRESS_H
00002 #define WCMLIFECYCLESTRESS_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00014 class WcmLifecycleStress : public BaseGenesysTerminalApplication {
00015 public:
00016 WcmLifecycleStress();
00017 virtual int main(int argc, char** argv) override;
00018 };
00019
00020 #endif /* WCMLIFECYCLESTRESS_H */

```

## 16.211 WcmMgenitaliumKarr2012.h

```

00001 #ifndef WCMMGENITALIUMKARR2012_H
00002 #define WCMMGENITALIUMKARR2012_H

```

```

00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00053 class WcmMgenitaliumKarr2012 : public BaseGenesysTerminalApplication {
00054 public:
00055 WcmMgenitaliumKarr2012();
00056 virtual int main(int argc, char** argv) override;
00057 };
00058
00059 #endif /* WCMGENITALIUMKARR2012_H */

```

## 16.212 WcmPathwayStressWholeCellMvp.h

```

00001 #ifndef WCMPATHWAYSTRESSWHOLECELLMVP_H
00002 #define WCMPATHWAYSTRESSWHOLECELLMVP_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00013 class WcmPathwayStressWholeCellMvp : public BaseGenesysTerminalApplication {
00014 public:
00015 WcmPathwayStressWholeCellMvp();
00016 virtual int main(int argc, char** argv) override;
00017 };
00018
00019 #endif /* WCMPATHWAYSTRESSWHOLECELLMVP_H */

```

## 16.213 WcmSimpleSsa.h

```

00001 #ifndef WCMSIMPLESSA_H
00002 #define WCMSIMPLESSA_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00025 class WcmSimpleSsa : public BaseGenesysTerminalApplication {
00026 public:
00027 WcmSimpleSsa();
00028 virtual int main(int argc, char** argv) override;
00029 };
00030
00031 #endif /* WCMSIMPLESSA_H */

```

## 16.214 WcmWholeCellMvp.h

```

00001 #ifndef WCMWHOLECELLMVP_H
00002 #define WCMWHOLECELLMVP_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00020 class WcmWholeCellMvp : public BaseGenesysTerminalApplication {
00021 public:
00022 WcmWholeCellMvp();
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* WCMWHOLECELLMVP_H */

```

## 16.215 WcmYeastDidacticWholeCellMvp.h

```

00001 #ifndef WCMYEASTDIDACTICWHOLECELLMVP_H
00002 #define WCMYEASTDIDACTICWHOLECELLMVP_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00018 class WcmYeastDidacticWholeCellMvp : public BaseGenesysTerminalApplication {
00019 public:
00020 WcmYeastDidacticWholeCellMvp();
00021 virtual int main(int argc, char** argv) override;
00022 };
00023
00024 #endif /* WCMYEASTDIDACTICWHOLECELLMVP_H */

```

## 16.216 WcmYeastEukaryoticWholeCellMvp.h

```

00001 #ifndef WCMYEASTEUKARYOTICWHOLECELLMVP_H

```



```

00002 #define WCMYEASTEUKARYOTICWHOLECELLMVP_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00019 class WcmYeastEukaryoticWholeCellMvp : public BaseGenesysTerminalApplication {
00020 public:
00021 WcmYeastEukaryoticWholeCellMvp();
00022 public:
00023 virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* WCMYEASTEUKARYOTICWHOLECELLMVP_H */

```

## 16.217 GenesysShell.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: GenesysTerminalAppApp.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 23 de Maio de 2019, 13:02
00012 */
00013
00014 #ifndef GENESYSHELL_H
00015 #define GENESYSHELL_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018 #include "kernel/simulator/SimulatorFacade.h"
00019 #include "kernel/simulator/Simulator.h"
00020 #include "kernel/util/List.h"
00021 #include <memory>
00022 #include <regex>
00023 #include <fstream>
00024 #include <assert.h>
00025
00026 class GenesysShell : public BaseGenesysTerminalApplication {
00027 private:
00028 typedef std::function<void() > ExecuterMember;
00029
00030 template<typename Class>
00031 ExecuterMember DefineExecuterMember(Class * object, void (Class::*function)()) {
00032 return std::bind(function, object); //, std::placeholders::_1
00033 }
00034
00035 class ShellCommand {
00036 public:
00037
00038 ShellCommand(std::string shortname, std::string longname, std::string parameters, std::string
description, ExecuterMember executer) {
00039 this->description = description;
00040 this->longname = longname;
00041 this->parameters = parameters;
00042 this->shortname = shortname;
00043 this->executer = executer;
00044 }
00045 std::string shortname;
00046 std::string longname;
00047 std::string parameters;
00048 std::string description;
00049 ExecuterMember executer;
00050 };
00051 public:
00052 GenesysShell();
00053 GenesysShell(const GenesysShell& orig);
00054 virtual ~GenesysShell() = default;
00055 public:
00056 virtual int main(int argc, char** argv) override;
00057 public: // commands
00058 void cmdScript();
00059 void cmdExecuteScript();
00060 void cmdHelp();
00061 void cmdQuit();
00062 void cmdBash();
00063 void cmdFacade();
00064 void cmdLicence();
00065 void cmdInfo();
00066 void cmdSim();
00067 void cmdData();
00068 void cmdComponent();
00069 void cmdExperiment();

```

```

00070 void cmdParser();
00071 void cmdTraceLevel();
00072 void cmdPlugin();
00073 //void cmdPluginInfo();
00074 //void cmdPluginAdd();
00075 //void cmdPluginRemove();
00076 void cmdSimulation();
00077 //void cmdSimulationStep();
00078 //void cmdSimulationStop();
00079 //void cmdSimulationInfo();
00080 void cmdReplication();
00081 //void cmdShowReport();
00082 void cmdModel();
00083 //void cmdModelClose();
00084 void cmdModelLoad();
00085 //void cmdModelCheck();
00086 void cmdModelSave();
00087 //void cmdModelShow();
00088 //void cmdModelSetInfos();
00089 private:
00090 void defineCommands();
00091 void run(List<std::string>* arguments);
00092 void Trace(std::string message);
00093 //void tryExecuteCommand(std::string inputText, const char* shortPrefix, const char* longPrefix,
std::string separator);
00094 void tryExecuteCommand(std::string inputText);
00095 std::vector<std::string> split(std::string text, std::string separatorRegex);
00096 std::string joinArguments(std::size_t firstArgumentIndex = 1) const;
00097 std::string lowercase(std::string text) const;
00098 bool requireCurrentModel(const std::string& actionDescription);
00099 bool parseUnsigned(const std::string& text, unsigned int& value) const;
00100 bool parseInt(const std::string& text, int& value) const;
00101 bool parseDouble(const std::string& text, double& value) const;
00102 bool parseBool(const std::string& text, bool& value) const;
00103 bool parseTraceLevel(const std::string& text, TraceManager::Level& level) const;
00104 bool parseTimeUnit(const std::string& text, Util::TimeUnit& timeUnit) const;
00105 private:
00106 std::unique_ptr<Simulator> simulator;
00107 SimulatorFacade facade;
00108 Model* model = nullptr;
00109 std::vector<std::string> *_typedWords = new std::vector<std::string>();
00110 std::vector<std::string> *_history = new std::vector<std::string>();
00111 List<ShellCommand*> *_commands = new List<ShellCommand*>();
00112 std::string _prompt = "$genesys> ";
00113 bool _exitRequested = false;
00114 };
00115 #endif /* GENESYS_SHELL_H */

```

## 16.218 GenesysShell\_if.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: GenesysShell_if.h
00009 * Author: rlcancian
00010 *
00011 * Created on 24 de Maio de 2019, 11:02
00012 */
00013
00014 #ifndef GENESYS_SHELL_IF_H
00015 #define GENESYS_SHELL_IF_H
00016
00017 #include "GenesysApplication_if.h"
00018 #include "Util.h"
00019
00020 class GenesysShell_if : public GenesysApplication_if {
00021 public:
00022 virtual ~GenesysShell_if() = default;
00023 virtual void openModel(std::string filename) = 0;
00024 virtual void saveModelAs(std::string filename) = 0;
00025 virtual void saveModel() = 0;
00026 virtual void listElements() = 0;
00027 virtual void listComponents() = 0;
00028 virtual void listHosts() = 0;
00029 virtual void listPlugins() = 0;
00030 virtual void deleteTraceFiles() = 0;
00031 virtual void traceLevel(TraceManager::TraceLevel tracelevel) = 0;
00032 virtual void addPlugin(std::string filename) = 0;
00033 virtual void addFromFile(std::string filename) = 0;
00034 virtual void readCommandsFromFile(std::string filename) = 0;
00035 virtual void redirectTrace(std::string trace, std::string dest, std::string filename) = 0;

```

```

00036 virtual void closeModel() = 0;
00037 virtual void createModel() = 0;
00038 virtual void execLinuxCommand(std::string command) = 0;
00039 virtual void verboseMode(bool on) = 0;
00040 virtual void check() = 0;
00041 virtual void getGenesysInfo() = 0;
00042 virtual void getCommandLine() = 0;
00043 virtual void sendFile(std::string filename, std::string hostname, std::string portname) = 0;
00044 virtual void setActivationCode(std::string code) = 0;
00045 virtual void receiveFile(std::string filename) = 0;
00046 virtual void startSimulation() = 0;
00047 virtual void stepSimulation() = 0;
00048 virtual void stopSimulation() = 0;
00049 virtual void showInit() = 0;
00050 virtual void showHelp() = 0;
00051 virtual void showHostName() = 0;
00052 //virtual void execute() = 0;
00053 //property TraceHandler : TProcTrace write aTraceHandler;
00054 };
00055
00056 #endif /* GENESYSHELL_IF_H */

```

## 16.219 ApiRouter.h

```

00001 #pragma once
00002
00003 #include "../http/HttpRequest.h"
00004 #include "../http/HttpResponse.h"
00005 #include "../service/SimulatorSessionService.h"
00006
00007 #include <optional>
00008
00012 class ApiRouter {
00013 public:
00018 explicit ApiRouter(SimulatorSessionService& simulatorService);
00019
00025 HttpResponse handle(const HttpRequest& request) const;
00026
00027 private:
00028 SimulatorSessionService& _simulatorService;
00029
00037 static HttpResponse _jsonError(int status, const char* code, const char* message);
00043 static std::string _extractBearerToken(const HttpRequest& request);
00050 static bool _tryExtractFilenameFromBody(const std::string& body, std::string& outFilename);
00058 static bool _tryExtractUnsignedIntField(const std::string& body, const std::string& fieldName,
unsigned int& outValue);
00066 static bool _tryExtractDoubleField(const std::string& body, const std::string& fieldName, double&
outValue);
00074 static bool _tryExtractBooleanField(const std::string& body, const std::string& fieldName, bool&
outValue);
00080 static std::optional<SimulatorSessionService::SimulationConfigInput>
_parseSimulationConfigBody(const std::string& body);
00086 static std::string _simulationStatusDataJson(const
SimulatorSessionService::SimulationStatusResult& status);
00092 static HttpResponse _mapPersistenceError(const SimulatorSessionService::ModelPersistenceResult&
result);
00098 static HttpResponse _mapModelImportError(const SimulatorSessionService::ModelImportResult&
result);
00104 static std::string _escapeJson(const std::string& value);
00110 static std::string _modelInfoDataJson(const SimulatorSessionService::ModelInfoResult& info);
00116 static std::string _workerInfoDataJson(const SimulatorSessionService::WorkerInfoResult& info);
00122 static std::string _workerCapabilitiesDataJson(const
SimulatorSessionService::WorkerCapabilitiesResult& capabilities);
00129 static bool _tryExtractWorkerJobIdFromPath(const std::string& path, std::string& outJobId);
00136 static bool _tryExtractWorkerJobRunIdFromPath(const std::string& path, std::string& outJobId);
00143 static bool _tryExtractWorkerJobResultIdFromPath(const std::string& path, std::string& outJobId);
00149 static const char* _workerJobStateToString(WorkerJobState state);
00155 static std::string _workerJobDataJson(const SimulatorSessionService::WorkerJobInfoResult& job);
00161 static std::string _workerJobResultDataJson(const SimulatorSessionService::WorkerJobResultInfo&
result);
00168 static HttpResponse _mapWorkerJobError(SimulatorSessionService::WorkerJobError error, bool
includeMissingModelMessage);
00169 };

```

## 16.220 TokenService.h

```

00001 #pragma once
00002
00003 #include <cstdint>
00004 #include <string>
00005
00006 class TokenService {

```

```

00007 public:
00008 TokenService();
00009
00010 [[nodiscard]] std::string generateSessionId();
00011 [[nodiscard]] std::string generateAccessToken();
00012
00013 private:
00014 [[nodiscard]] std::string _generateHexToken(std::size_t bytes);
00015 };

```

## 16.221 BaseGenesysWebApplication.h

```

00001 #pragma once
00002
00003 #include "../GenesysApplication_if.h"
00004
00012 class BaseGenesysWebApplication : public GenesysApplication_if {
00013 public:
00020 int main(int argc, char** argv) override;
00021
00022 private:
00029 static unsigned short _readPort(int argc, char** argv);
00036 static unsigned long _readMaxRequests(int argc, char** argv);
00037 };

```

## 16.222 HttpRequest.h

```

00001 #pragma once
00002
00003 #include <algorithm>
00004 #include <cctype>
00005 #include <string>
00006 #include <unordered_map>
00007
00015 struct HttpRequest {
00017 std::string method;
00019 std::string path;
00021 std::string body;
00023 std::unordered_map<std::string, std::string> headers;
00024
00030 [[nodiscard]] std::string header(const std::string& name) const {
00031 std::string normalized = name;
00032 std::transform(normalized.begin(), normalized.end(), normalized.begin(), [](unsigned char c) {
00033 return static_cast<char>(std::tolower(c));
00034 });
00035 auto it = headers.find(normalized);
00036 if (it == headers.end()) {
00037 return {};
00038 }
00039 return it->second;
00040 }
00041 };

```

## 16.223 HttpResponse.h

```

00001 #pragma once
00002
00003 #include <string>
00004 #include <unordered_map>
00005
00012 struct HttpResponse {
00014 int status = 200;
00016 std::string contentType = "application/json";
00018 std::string body;
00020 std::unordered_map<std::string, std::string> headers;
00021 };

```

## 16.224 SimpleHttpServer.h

```

00001 #pragma once
00002
00003 #include "HttpRequest.h"
00004 #include "HttpResponse.h"
00005
00006 #include <atomic>
00007 #include <functional>
00008

```

```

00015 class SimpleHttpServer {
00016 public:
00017 using RequestHandler = std::function<HttpResponse(const HttpRequest&)>;
00018
00023 explicit SimpleHttpServer(unsigned short port);
00024
00031 bool serve(const RequestHandler& handler, unsigned long maxRequests = 0);
00032
00036 void requestStop();
00037
00038 private:
00039 unsigned short _port;
00040 std::atomic<bool> _stopRequested{false};
00041 };

```

## 16.225 SimulatorSessionService.h

```

00001 #pragma once
00002
00003 #include "../session/SessionManager.h"
00004 #include "../worker/WorkerJob.h"
00005 #include "../worker/WorkerJobManager.h"
00006
00007 #include <string>
00008
00012 class SimulatorSessionService {
00013 public:
00017 enum class PersistenceError {
00018 None,
00019 InvalidToken,
00020 InvalidFilename,
00021 MissingCurrentModel,
00022 FileNotFound,
00023 OperationFailed
00024 };
00025
00029 enum class ModelImportError {
00030 None,
00031 InvalidToken,
00032 EmptySpecification,
00033 InvalidSpecification,
00034 OperationFailed
00035 };
00036
00040 enum class WorkerJobError {
00041 None,
00042 InvalidToken,
00043 MissingCurrentModel,
00044 JobNotFound,
00045 AccessDenied,
00046 OperationFailed,
00047 ResultNotReady
00048 };
00049
00053 struct CreateSessionResult {
00054 std::string sessionId;
00055 std::string accessToken;
00056 };
00057
00061 struct SimulatorInfoResult {
00062 std::string name;
00063 std::string versionName;
00064 unsigned int versionNumber;
00065 };
00066
00070 struct WorkerInfoResult {
00071 std::string role;
00072 std::string application;
00073 std::string apiFamily;
00074 std::string apiVersion;
00075 std::string simulatorName;
00076 std::string simulatorVersionName;
00077 unsigned int simulatorVersionNumber = 0;
00078 };
00079
00083 struct WorkerCapabilitiesResult {
00084 bool supportsSessionApi = false;
00085 bool supportsSessionScopedSimulator = false;
00086 bool supportsModelCreation = false;
00087 bool supportsModelPersistence = false;
00088 bool supportsSimulationStatus = false;
00089 bool supportsSimulationConfig = false;
00090 bool supportsSynchronousRun = false;
00091 bool supportsSynchronousStep = false;
00092 bool supportsDistributedJobs = false;

```

```

00093 bool supportsJobPolling = false;
00094 bool supportsBackgroundExecution = false;
00095 bool supportsModelUpload = false;
00096 bool supportsStreamingEvents = false;
00097 bool supportsJobResultRetrieval = false;
00098 };
00099
00103 struct ModelInfoResult {
00104 bool exists = false;
00105 unsigned int modelId = 0;
00106 bool hasChanged = false;
00107 unsigned int level = 0;
00108 std::string name;
00109 std::string analystName;
00110 std::string projectTitle;
00111 std::string version;
00112 std::string description;
00113 unsigned int componentCount = 0;
00114 };
00115
00119 struct ModelPersistenceResult {
00120 bool success = false;
00121 PersistenceError error = PersistenceError::None;
00122 std::string filename;
00123 ModelInfoResult modelInfo;
00124 };
00125
00129 struct ModelImportResult {
00130 bool success = false;
00131 ModelImportError error = ModelImportError::None;
00132 ModelInfoResult modelInfo;
00133 };
00134
00138 struct WorkerJobInfoResult {
00139 std::string jobId;
00140 std::string sessionId;
00141 WorkerJobState state = WorkerJobState::Queued;
00142 std::string snapshotFilename;
00143 std::string createdMarker;
00144 std::string message;
00145 };
00146
00150 struct WorkerJobCreationResult {
00151 bool success = false;
00152 WorkerJobError error = WorkerJobError::None;
00153 WorkerJobInfoResult jobInfo;
00154 };
00155
00159 struct WorkerJobQueryResult {
00160 bool success = false;
00161 WorkerJobError error = WorkerJobError::None;
00162 WorkerJobInfoResult jobInfo;
00163 };
00164
00168 struct WorkerJobRunResult {
00169 bool success = false;
00170 WorkerJobError error = WorkerJobError::None;
00171 WorkerJobInfoResult jobInfo;
00172 };
00173
00177 struct WorkerJobResultInfo {
00178 std::string jobId;
00179 WorkerJobState state = WorkerJobState::Queued;
00180 std::string message;
00181 double simulatedTime = 0.0;
00182 unsigned int currentReplicationNumber = 0;
00183 unsigned int numberOfReplications = 0;
00184 double replicationLength = 0.0;
00185 double warmUpPeriod = 0.0;
00186 bool hasIsPaused = false;
00187 bool isPaused = false;
00188 };
00189
00193 struct WorkerJobResultQueryResult {
00194 bool success = false;
00195 WorkerJobError error = WorkerJobError::None;
00196 WorkerJobResultInfo jobResult;
00197 };
00198
00202 struct SimulationStatusResult {
00203 bool success = false;
00204 bool invalidToken = false;
00205 bool hasCurrentModel = false;
00206 bool isRunning = false;
00207 bool isPaused = false;
00208 double simulatedTime = 0.0;
00209 unsigned int currentReplicationNumber = 0;

```

```

00210 unsigned int numberOfReplications = 0;
00211 double replicationLength = 0.0;
00212 double warmUpPeriod = 0.0;
00213 bool pauseOnEvent = false;
00214 bool pauseOnReplication = false;
00215 bool initializeStatistics = false;
00216 bool initializeSystem = false;
00217 };
00218
00222 struct SimulationConfigInput {
00223 unsigned int numberOfReplications = 0;
00224 double replicationLength = 0.0;
00225 double warmUpPeriod = 0.0;
00226 bool pauseOnEvent = false;
00227 bool pauseOnReplication = false;
00228 bool initializeStatistics = false;
00229 bool initializeSystem = false;
00230 };
00231
00235 struct SimulationConfigResult {
00236 bool success = false;
00237 bool invalidToken = false;
00238 bool missingCurrentModel = false;
00239 SimulationStatusResult status;
00240 };
00241
00245 struct SimulationActionResult {
00246 bool success = false;
00247 bool invalidToken = false;
00248 bool missingCurrentModel = false;
00249 SimulationStatusResult status;
00250 };
00251
00257 explicit SimulatorSessionService(SessionManager& sessionManager, WorkerJobManager&
workerJobManager);
00258
00263 CreateSessionResult createSession();
00270 bool tryGetSimulatorInfo(const std::string& accessToken, SimulatorInfoResult& outInfo);
00275 WorkerInfoResult getWorkerInfo() const;
00280 WorkerCapabilitiesResult getWorkerCapabilities() const;
00287 bool tryCreateModel(const std::string& accessToken, ModelInfoResult& outInfo);
00294 bool tryGetCurrentModelInfo(const std::string& accessToken, ModelInfoResult& outInfo);
00300 SimulationStatusResult getSimulationStatus(const std::string& accessToken);
00307 SimulationConfigResult configureSimulation(const std::string& accessToken, const
SimulationConfigInput& input);
00313 SimulationActionResult runSimulation(const std::string& accessToken);
00319 SimulationActionResult stepSimulation(const std::string& accessToken);
00326 ModelPersistenceResult saveCurrentModel(const std::string& accessToken, const std::string&
filename);
00333 ModelPersistenceResult loadModel(const std::string& accessToken, const std::string& filename);
00340 ModelImportResult importModelFromLanguage(const std::string& accessToken, const std::string&
modelSpecification);
00346 WorkerJobCreationResult createWorkerJob(const std::string& accessToken);
00353 WorkerJobQueryResult getWorkerJob(const std::string& accessToken, const std::string& jobId);
00360 WorkerJobRunResult runWorkerJob(const std::string& accessToken, const std::string& jobId);
00367 WorkerJobResultQueryResult getWorkerJobResult(const std::string& accessToken, const std::string&
jobId);
00368
00369 private:
00375 static bool _isSafeFilename(const std::string& filename);
00376
00382 static WorkerJobInfoResult _toWorkerJobInfoResult(const WorkerJob& job);
00383
00389 static WorkerJobResultInfo _toWorkerJobResultInfo(const WorkerJob& job);
00390
00391 SessionManager& _sessionManager;
00392 WorkerJobManager& _workerJobManager;
00393 };

```

## 16.226 WebWorkerRuntime.h

```

00001 #pragma once
00002
00003 #include "api/ApiRouter.h"
00004 #include "auth/TokenService.h"
00005 #include "http/SimpleHttpServer.h"
00006 #include "kernel/simulator/Simulator.h"
00007 #include "service/SimulatorSessionService.h"
00008 #include "session/SessionManager.h"
00009 #include "worker/WorkerJobManager.h"
00010
00011 #include <atomic>
00012 #include <deque>
00013 #include <memory>
00014 #include <mutex>

```

```

00015 #include <string>
00016 #include <thread>
00017 #include <vector>
00018
00026 class WebWorkerRuntime {
00027 public:
00031 struct Snapshot {
00032 bool isRunning = false;
00033 bool isStarting = false;
00034 bool isStopping = false;
00035 bool hasFailed = false;
00036 unsigned short port = 8080;
00037 unsigned long maxRequests = 0;
00038 unsigned long servedRequests = 0;
00039 unsigned long activeRequests = 0;
00040 std::string statusMessage;
00041 std::string lastError;
00042 std::vector<std::string> recentEvents;
00043 std::vector<std::string> recentErrors;
00044 };
00045
00049 explicit WebWorkerRuntime(unsigned short defaultPort = 8080, unsigned long maxRequests = 0);
00053 ~WebWorkerRuntime();
00054
00059 bool start();
00063 void stop();
00067 void wait();
00072 bool restart();
00073
00077 bool isRunning() const;
00081 Snapshot snapshot() const;
00082
00086 void setConfiguredPort(unsigned short port);
00090 unsigned short configuredPort() const;
00091
00095 void setConfiguredMaxRequests(unsigned long maxRequests);
00099 unsigned long configuredMaxRequests() const;
00100
00101 private:
00105 void _run();
00109 void _fail(const std::string& errorMessage);
00113 void _recordEventLocked(const std::string& message);
00117 void _recordErrorLocked(const std::string& message);
00121 std::string _statusMessageLocked() const;
00125 static std::unique_ptr<Simulator> _createSimulator();
00126
00127 mutable std::mutex _mutex;
00128 std::atomic<bool> _stopRequested{false};
00129 std::atomic<bool> _threadRunning{false};
00130 std::atomic<bool> _starting{false};
00131 std::atomic<bool> _stopping{false};
00132 std::atomic<unsigned long> _servedRequests{0};
00133 std::atomic<unsigned long> _activeRequests{0};
00134 unsigned short _configuredPort = 8080;
00135 unsigned long _configuredMaxRequests = 0;
00136 std::string _lastError;
00137 std::string _statusMessage;
00138 std::deque<std::string> _recentEvents;
00139 std::deque<std::string> _recentErrors;
00140 std::thread _thread;
00141
00142 std::unique_ptr<TokenService> _tokenService;
00143 std::unique_ptr<SessionManager> _sessionManager;
00144 std::unique_ptr<WorkerJobManager> _workerJobManager;
00145 std::unique_ptr<SimulatorSessionService> _simulatorSessionService;
00146 std::unique_ptr<ApiRouter> _router;
00147 std::unique_ptr<SimpleHttpServer> _server;
00148 };

```

## 16.227 SessionContext.h

```

00001 #pragma once
00002
00003 #include "kernel/simulator/Simulator.h"
00004
00005 #include <filesystem>
00006 #include <memory>
00007 #include <mutex>
00008 #include <string>
00009
00017 struct SessionContext {
00019 std::string sessionId;
00021 std::string accessToken;
00023 std::filesystem::path workspacePath;
00025 std::unique_ptr<Simulator> simulator;

```



```
00027 mutable std::mutex mutex;
00028 };
```

## 16.228 SessionManager.h

```
00001 #pragma once
00002
00003 #include "SessionContext.h"
00004 #include "../auth/TokenService.h"
00005
00006 #include <functional>
00007 #include <filesystem>
00008 #include <memory>
00009 #include <mutex>
00010 #include <string>
00011 #include <unordered_map>
00012
00020 class SessionManager {
00021 public:
00022 using SimulatorFactory = std::function<std::unique_ptr<Simulator>()>;
00023
00024 SessionManager(TokenService& tokenService, SimulatorFactory simulatorFactory);
00030
00036 SessionContext* createSession();
00042 SessionContext* getSessionByToken(const std::string& token);
00043
00044 private:
00049 static std::filesystem::path _buildWorkspaceRoot();
00050
00051 TokenService& _tokenService;
00052 SimulatorFactory _simulatorFactory;
00053 std::filesystem::path _workspaceRoot;
00054 mutable std::mutex _sessionsMutex;
00055 std::unordered_map<std::string, std::unique_ptr<SessionContext> _sessionsByToken;
00056 };
```

## 16.229 WorkerJob.h

```
00001 #pragma once
00002
00003 #include <optional>
00004 #include <string>
00005
00009 enum class WorkerJobState {
00010 Queued,
00011 Running,
00012 Finished,
00013 Failed
00014 };
00015
00019 struct WorkerJobTerminalResult {
00021 double simulatedTime = 0.0;
00023 unsigned int currentReplicationNumber = 0;
00025 unsigned int numberOfReplications = 0;
00027 double replicationLength = 0.0;
00029 double warmUpPeriod = 0.0;
00031 std::optional<bool> isPaused;
00032 };
00033
00037 struct WorkerJob {
00039 std::string jobId;
00041 std::string sessionId;
00043 WorkerJobState state = WorkerJobState::Queued;
00045 std::string snapshotFilename;
00047 std::string createdMarker;
00049 std::string message;
00051 bool hasTerminalResult = false;
00053 WorkerJobTerminalResult terminalResult;
00054 };
```

## 16.230 WorkerJobManager.h

```
00001 #pragma once
00002
00003 #include "WorkerJob.h"
00004
00005 #include <mutex>
00006 #include <optional>
00007 #include <string>
```

```

00008 #include <unordered_map>
00009
00016 class WorkerJobManager {
00017 public:
00023 WorkerJob createQueuedJob(const std::string& sessionId);
00024
00031 bool setSnapshotFilename(const std::string& jobId, const std::string& snapshotFilename);
00038 bool setState(const std::string& jobId, WorkerJobState state);
00039
00046 bool setMessage(const std::string& jobId, const std::string& message);
00047
00054 bool setTerminalResult(const std::string& jobId, const WorkerJobTerminalResult& terminalResult);
00055
00061 std::optional<WorkerJob> getJob(const std::string& jobId) const;
00062
00063 private:
00069 static std::string _makeCreationMarker(unsigned long long numericId);
00070
00071 mutable std::mutex _mutex;
00072 std::unordered_map<std::string, WorkerJob> _jobs;
00073 unsigned long long _nextJobNumber = 1;
00074 };

```

## 16.231 gtest-assertion-result.h

```

00001 // Copyright 2005, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // The Google C++ Testing and Mocking Framework (Google Test)
00031 //
00032 // This file implements the AssertionResult type.
00033
00034 // IWYU pragma: private, include "gtest/gtest.h"
00035 // IWYU pragma: friend gtest/*.h
00036 // IWYU pragma: friend gmock/*.h
00037
00038 #ifndef GOOGLETEST_INCLUDE_GTEST_GTEST_ASSERTION_RESULT_H_
00039 #define GOOGLETEST_INCLUDE_GTEST_GTEST_ASSERTION_RESULT_H_
00040
00041 #include <memory>
00042 #include <ostream>
00043 #include <string>
00044 #include <type_traits>
00045
00046 #include "gtest/gtest-message.h"
00047 #include "gtest/internal/gtest-port.h"
00048
00049 GTEST_DISABLE_MSC_WARNINGS_PUSH_(4251 \
00050 /* class A needs to have dll-interface to be used by clients of class B */)
00051 namespace testing {
00052
00053 // A class for indicating whether an assertion was successful. When
00054 // the assertion wasn't successful, the AssertionResult object
00055 // remembers a non-empty message that describes how it failed.
00056 //
00057 // To create an instance of this class, use one of the factory functions
00058 // (AssertionSuccess() and AssertionFailure()).
00059 //
00060 // This class is never copied or assigned -- it is only constructed.

```

```

00061 // This class is useful for two purposes:
00062 // 1. Defining predicate functions to be used with Boolean test assertions
00063 // EXPECT_TRUE/EXPECT_FALSE and their ASSERT_ counterparts
00064 // 2. Defining predicate-format functions to be
00065 // used with predicate assertions (ASSERT_PRED_FORMAT*, etc).
00066 //
00067 // For example, if you define IsEven predicate:
00068 //
00069 // testing::AssertionResult IsEven(int n) {
00070 // if ((n % 2) == 0)
00071 // return testing::AssertionSuccess();
00072 // else
00073 // return testing::AssertionFailure() « n « " is odd";
00074 // }
00075 //
00076 // Then the failed expectation EXPECT_TRUE(IsEven(Fib(5)))
00077 // will print the message
00078 //
00079 // Value of: IsEven(Fib(5))
00080 // Actual: false (5 is odd)
00081 // Expected: true
00082 //
00083 // instead of a more opaque
00084 //
00085 // Value of: IsEven(Fib(5))
00086 // Actual: false
00087 // Expected: true
00088 //
00089 // in case IsEven is a simple Boolean predicate.
00090 //
00091 // If you expect your predicate to be reused and want to support informative
00092 // messages in EXPECT_FALSE and ASSERT_FALSE (negative assertions show up
00093 // about half as often as positive ones in our tests), supply messages for
00094 // both success and failure cases:
00095 //
00096 // testing::AssertionResult IsEven(int n) {
00097 // if ((n % 2) == 0)
00098 // return testing::AssertionSuccess() « n « " is even";
00099 // else
00100 // return testing::AssertionFailure() « n « " is odd";
00101 // }
00102 //
00103 // Then a statement EXPECT_FALSE(IsEven(Fib(6))) will print
00104 //
00105 // Value of: IsEven(Fib(6))
00106 // Actual: true (8 is even)
00107 // Expected: false
00108 //
00109 // NB: Predicates that support negative Boolean assertions have reduced
00110 // performance in positive ones so be careful not to use them in tests
00111 // that have lots (tens of thousands) of positive Boolean assertions.
00112 //
00113 // To use this class with EXPECT_PRED_FORMAT assertions such as:
00114 //
00115 // // Verifies that Foo() returns an even number.
00116 // EXPECT_PRED_FORMAT1(IsEven, Foo());
00117 //
00118 // you need to define:
00119 //
00120 // testing::AssertionResult IsEven(const char* expr, int n) {
00121 // if ((n % 2) == 0)
00122 // return testing::AssertionSuccess();
00123 // else
00124 // return testing::AssertionFailure()
00125 // « "Expected: " « expr « " is even\n Actual: it's " « n;
00126 // }
00127 //
00128 // If Foo() returns 5, you will see the following message:
00129 //
00130 // Expected: Foo() is even
00131 // Actual: it's 5
00132 //
00133
00134 class GTEST_API_ AssertionResult {
00135 public:
00136 // Copy constructor.
00137 // Used in EXPECT_TRUE/FALSE(assertion_result).
00138 AssertionResult(const AssertionResult& other);
00139
00140 // C4800 is a level 3 warning in Visual Studio 2015 and earlier.
00141 // This warning is not emitted in Visual Studio 2017.
00142 // This warning is off by default starting in Visual Studio 2019 but can be
00143 // enabled with command-line options.
00144 #if defined(_MSC_VER) && (_MSC_VER < 1910 || _MSC_VER >= 1920)
00145 GTEST_DISABLE_MSC_WARNINGS_PUSH_(4800 /* forcing value to bool */)
00146 #endif

```

```

00148
00149 // Used in the EXPECT_TRUE/FALSE(bool_expression).
00150 //
00151 // T must be contextually convertible to bool.
00152 //
00153 // The second parameter prevents this overload from being considered if
00154 // the argument is implicitly convertible to AssertionResult. In that case
00155 // we want AssertionResult's copy constructor to be used.
00156 template <typename T>
00157 explicit AssertionResult(
00158 const T& success,
00159 typename std::enable_if<
00160 !std::is_convertible<T, AssertionResult>::value>::type*
00161 /*enabler*/
00162 = nullptr)
00163 : success_(success) {
00164 }
00165
00166 #if defined(_MSC_VER) && (_MSC_VER < 1910 || _MSC_VER >= 1920)
00167
00168 GTEST_DISABLE_MSC_WARNINGS_POP_()
00169 #endif
00170
00171 // Assignment operator.
00172 AssertionResult& operator=(AssertionResult other) {
00173 swap(other);
00174 return *this;
00175 }
00176
00177 // Returns true if and only if the assertion succeeded.
00178
00179 operator bool() const {
00180 return success_;
00181 } // NOLINT
00182
00183 // Returns the assertion's negation. Used with EXPECT/ASSERT_FALSE.
00184 AssertionResult operator!() const;
00185
00186 // Returns the text streamed into this AssertionResult. Test assertions
00187 // use it when they fail (i.e., the predicate's outcome doesn't match the
00188 // assertion's expectation). When nothing has been streamed into the
00189 // object, returns an empty string.
00190
00191 const char* message() const {
00192 return message_.get() != nullptr ? message_>c_str() : "";
00193 }
00194 // Deprecated; please use message() instead.
00195
00196 const char* failure_message() const {
00197 return message();
00198 }
00199
00200 // Streams a custom failure message into this object.
00201
00202 template <typename T>
00203 AssertionResult& operator<<(const T& value) {
00204 AppendMessage(Message() << value);
00205 return *this;
00206 }
00207
00208 // Allows streaming basic output manipulators such as endl or flush into
00209 // this object.
00210
00211 AssertionResult& operator<<(
00212 ::std::ostream& (*basic_manipulator)(::std::ostream& stream)) {
00213 AppendMessage(Message() << basic_manipulator);
00214 return *this;
00215 }
00216
00217 private:
00218 // Appends the contents of message to message_.
00219
00220 void AppendMessage(const Message& a_message) {
00221 if (message_.get() == nullptr) message_.reset(new ::std::string);
00222 message_>append(a_message.GetString().c_str());
00223 }
00224
00225 // Swap the contents of this AssertionResult with other.
00226 void swap(AssertionResult& other);
00227
00228 // Stores result of the assertion predicate.
00229 bool success_;
00230 // Stores the message describing the condition in case the expectation
00231 // construct is not satisfied with the predicate's outcome.
00232 // Referenced via a pointer to avoid taking too much stack frame space
00233 // with test assertions.
00234 std::unique_ptr< ::std::string> message_;

```

```

00235 };
00236
00237 // Makes a successful assertion result.
00238 GTEST_API_ AssertionResult AssertionSuccess();
00239
00240 // Makes a failed assertion result.
00241 GTEST_API_ AssertionResult AssertionFailure();
00242
00243 // Makes a failed assertion result with the given failure message.
00244 // Deprecated; use AssertionFailure() « msg.
00245 GTEST_API_ AssertionResult AssertionFailure(const Message& msg);
00246
00247 } // namespace testing
00248
00249 GTEST_DISABLE_MSC_WARNINGS_POP() // 4251
00250
00251 #endif // GOOGLETEST_INCLUDE_GTEST_GTEST_ASSERTION_RESULT_H_

```

## 16.232 gtest-death-test.h

```

00001 // Copyright 2005, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // The Google C++ Testing and Mocking Framework (Google Test)
00031 //
00032 // This header file defines the public API for death tests. It is
00033 // #included by gtest.h so a user doesn't need to include this
00034 // directly.
00035
00036 // IWYU pragma: private, include "gtest/gtest.h"
00037 // IWYU pragma: friend gtest/*.h
00038 // IWYU pragma: friend gmock/*.h
00039
00040 #ifndef GOOGLETEST_INCLUDE_GTEST_GTEST_DEATH_TEST_H_
00041 #define GOOGLETEST_INCLUDE_GTEST_GTEST_DEATH_TEST_H_
00042
00043 #include "gtest/internal/gtest-death-test-internal.h"
00044
00045 // This flag controls the style of death tests. Valid values are "threadsafe",
00046 // meaning that the death test child process will re-execute the test binary
00047 // from the start, running only a single death test, or "fast",
00048 // meaning that the child process will execute the test logic immediately
00049 // after forking.
00050 GTEST_DECLARE_string_(death_test_style);
00051
00052 namespace testing {
00053
00054 #if GTEST_HAS_DEATH_TEST
00055
00056 namespace internal {
00057
00058 // Returns a Boolean value indicating whether the caller is currently
00059 // executing in the context of the death test child process. Tools such as
00060 // Valgrind heap checkers may need this to modify their behavior in death
00061 // tests. IMPORTANT: This is an internal utility. Using it may break the
00062 // implementation of death tests. User code MUST NOT use it.
00063 GTEST_API_ bool InDeathTestChild();
00064
00065 } // namespace internal
00066
00067 } // namespace testing

```

```

00066
00067 // The following macros are useful for writing death tests.
00068
00069 // Here's what happens when an ASSERT_DEATH* or EXPECT_DEATH* is
00070 // executed:
00071 //
00072 // 1. It generates a warning if there is more than one active
00073 // thread. This is because it's safe to fork() or clone() only
00074 // when there is a single thread.
00075 //
00076 // 2. The parent process clone()s a sub-process and runs the death
00077 // test in it; the sub-process exits with code 0 at the end of the
00078 // death test, if it hasn't exited already.
00079 //
00080 // 3. The parent process waits for the sub-process to terminate.
00081 //
00082 // 4. The parent process checks the exit code and error message of
00083 // the sub-process.
00084 //
00085 // Examples:
00086 //
00087 // ASSERT_DEATH(server.SendMessage(56, "Hello"), "Invalid port number");
00088 // for (int i = 0; i < 5; i++) {
00089 // EXPECT_DEATH(server.ProcessRequest(i),
00090 // "Invalid request .* in ProcessRequest()")
00091 // « "Failed to die on request " « i;
00092 // }
00093 //
00094 // ASSERT_EXIT(server.ExitNow(), ::testing::ExitedWithCode(0), "Exiting");
00095 //
00096 // bool KilledBySIGHUP(int exit_code) {
00097 // return WIFSIGNALED(exit_code) && WTERMSIG(exit_code) == SIGHUP;
00098 // }
00099 //
00100 // ASSERT_EXIT(client.HangUpServer(), KilledBySIGHUP, "Hanging up!");
00101 //
00102 // The final parameter to each of these macros is a matcher applied to any data
00103 // the sub-process wrote to stderr. For compatibility with existing tests, a
00104 // bare string is interpreted as a regular expression matcher.
00105 //
00106 // On the regular expressions used in death tests:
00107 //
00108 // On POSIX-compliant systems (*nix), we use the <regex.h> library,
00109 // which uses the POSIX extended regex syntax.
00110 //
00111 // On other platforms (e.g. Windows or Mac), we only support a simple regex
00112 // syntax implemented as part of Google Test. This limited
00113 // implementation should be enough most of the time when writing
00114 // death tests; though it lacks many features you can find in PCRE
00115 // or POSIX extended regex syntax. For example, we don't support
00116 // union ("x|y"), grouping ("(xy)"), brackets ("[xy]"), and
00117 // repetition count ("x{5,7}"), among others.
00118 //
00119 // Below is the syntax that we do support. We chose it to be a
00120 // subset of both PCRE and POSIX extended regex, so it's easy to
00121 // learn wherever you come from. In the following: 'A' denotes a
00122 // literal character, period (.), or a single \\ escape sequence;
00123 // 'x' and 'y' denote regular expressions; 'm' and 'n' are for
00124 // natural numbers.
00125 //
00126 // c matches any literal character c
00127 // \\d matches any decimal digit
00128 // \\D matches any character that's not a decimal digit
00129 // \\f matches \\f
00130 // \\n matches \\n
00131 // \\r matches \\r
00132 // \\s matches any ASCII whitespace, including \\n
00133 // \\S matches any character that's not a whitespace
00134 // \\t matches \\t
00135 // \\v matches \\v
00136 // \\w matches any letter, _, or decimal digit
00137 // \\W matches any character that \\w doesn't match
00138 // \\c matches any literal character c, which must be a punctuation
00139 // . matches any single character except \\n
00140 // A? matches 0 or 1 occurrences of A
00141 // A* matches 0 or many occurrences of A
00142 // A+ matches 1 or many occurrences of A
00143 // ^ matches the beginning of a string (not that of each line)
00144 // $ matches the end of a string (not that of each line)
00145 // xy matches x followed by y
00146 //
00147 // If you accidentally use PCRE or POSIX extended regex features
00148 // not implemented by us, you will get a run-time failure. In that
00149 // case, please try to rewrite your regular expression within the
00150 // above syntax.
00151 //
00152 // This implementation is *not* meant to be as highly tuned or robust

```

```

00153 // as a compiled regex library, but should perform well enough for a
00154 // death test, which already incurs significant overhead by launching
00155 // a child process.
00156 //
00157 // Known caveats:
00158 //
00159 // A "threadsafe" style death test obtains the path to the test
00160 // program from argv[0] and re-executes it in the sub-process. For
00161 // simplicity, the current implementation doesn't search the PATH
00162 // when launching the sub-process. This means that the user must
00163 // invoke the test program via a path that contains at least one
00164 // path separator (e.g. path/to/foo_test and
00165 // /absolute/path/to/bar_test are fine, but foo_test is not). This
00166 // is rarely a problem as people usually don't put the test binary
00167 // directory in PATH.
00168 //
00169 //
00170 // Asserts that a given `statement` causes the program to exit, with an
00171 // integer exit status that satisfies `predicate`, and emitting error output
00172 // that matches `matcher`.
00173 #define ASSERT_EXIT(statement, predicate, matcher) \
00174 GTEST_DEATH_TEST_(statement, predicate, matcher, GTEST_FATAL_FAILURE_)
00175 //
00176 // Like `ASSERT_EXIT`, but continues on to successive tests in the
00177 // test suite, if any:
00178 #define EXPECT_EXIT(statement, predicate, matcher) \
00179 GTEST_DEATH_TEST_(statement, predicate, matcher, GTEST_NONFATAL_FAILURE_)
00180 //
00181 // Asserts that a given `statement` causes the program to exit, either by
00182 // explicitly exiting with a nonzero exit code or being killed by a
00183 // signal, and emitting error output that matches `matcher`.
00184 #define ASSERT_DEATH(statement, matcher) \
00185 ASSERT_EXIT(statement, ::testing::internal::ExitedUnsuccessfully, matcher)
00186 //
00187 // Like `ASSERT_DEATH`, but continues on to successive tests in the
00188 // test suite, if any:
00189 #define EXPECT_DEATH(statement, matcher) \
00190 EXPECT_EXIT(statement, ::testing::internal::ExitedUnsuccessfully, matcher)
00191 //
00192 // Two predicate classes that can be used in {ASSERT,EXPECT}_EXIT*:
00193 //
00194 // Tests that an exit code describes a normal exit with a given exit code.
00195 //
00196 class GTEST_API_ ExitedWithCode {
00197 public:
00198 explicit ExitedWithCode(int exit_code);
00199 ExitedWithCode(const ExitedWithCode&) = default;
00200 void operator=(const ExitedWithCode& other) = delete;
00201 bool operator()(int exit_status) const;
00202 private:
00203 const int exit_code_;
00204 };
00205 //
00206 #if !GTEST_OS_WINDOWS && !GTEST_OS_FUCHSIA
00207 // Tests that an exit code describes an exit due to termination by a
00208 // given signal.
00209 //
00210 class GTEST_API_ KilledBySignal {
00211 public:
00212 explicit KilledBySignal(int signum);
00213 bool operator()(int exit_status) const;
00214 private:
00215 const int signum_;
00216 };
00217 #endif // !GTEST_OS_WINDOWS
00218 //
00219 // EXPECT_DEBUG_DEATH asserts that the given statements die in debug mode.
00220 // The death testing framework causes this to have interesting semantics,
00221 // since the sideeffects of the call are only visible in opt mode, and not
00222 // in debug mode.
00223 //
00224 // In practice, this can be used to test functions that utilize the
00225 // LOG(FATAL) macro using the following style:
00226 //
00227 // int DieInDebugOr12(int* sideeffect) {
00228 // if (sideeffect) {
00229 // *sideeffect = 12;
00230 // }
00231 // LOG(FATAL) << "death";
00232 // return 12;
00233 // }
00234 //
00235 // TEST(TestSuite, TestDieOr12WorksInDgbAndOpt) {
00236 // int sideeffect = 0;
00237 // // Only asserts in dbg.

```

```

00240 // EXPECT_DEBUG_DEATH(DieInDebugOr12(&sideeffect), "death");
00241 //
00242 // #ifdef NDEBUG
00243 // // opt-mode has sideeffect visible.
00244 // EXPECT_EQ(12, sideeffect);
00245 // #else
00246 // // dbg-mode no visible sideeffect.
00247 // EXPECT_EQ(0, sideeffect);
00248 // #endif
00249 // }
00250 //
00251 // This will assert that DieInDebugReturn12InOpt() crashes in debug
00252 // mode, usually due to a DCHECK or LOG(DFATAL), but returns the
00253 // appropriate fallback value (12 in this case) in opt mode. If you
00254 // need to test that a function has appropriate side-effects in opt
00255 // mode, include assertions against the side-effects. A general
00256 // pattern for this is:
00257 //
00258 // EXPECT_DEBUG_DEATH({
00259 // // Side-effects here will have an effect after this statement in
00260 // // opt mode, but none in debug mode.
00261 // EXPECT_EQ(12, DieInDebugOr12(&sideeffect));
00262 // }, "death");
00263 //
00264 #ifdef NDEBUG
00265
00266 #define EXPECT_DEBUG_DEATH(statement, regex) \
00267 GTEST_EXECUTE_STATEMENT_(statement, regex)
00268
00269 #define ASSERT_DEBUG_DEATH(statement, regex) \
00270 GTEST_EXECUTE_STATEMENT_(statement, regex)
00271
00272 #else
00273
00274 #define EXPECT_DEBUG_DEATH(statement, regex) EXPECT_DEATH(statement, regex)
00275
00276 #define ASSERT_DEBUG_DEATH(statement, regex) ASSERT_DEATH(statement, regex)
00277
00278 #endif // NDEBUG for EXPECT_DEBUG_DEATH
00279 #endif // GTEST_HAS_DEATH_TEST
00280
00281 // This macro is used for implementing macros such as
00282 // EXPECT_DEATH_IF_SUPPORTED and ASSERT_DEATH_IF_SUPPORTED on systems where
00283 // death tests are not supported. Those macros must compile on such systems
00284 // if and only if EXPECT_DEATH and ASSERT_DEATH compile with the same parameters
00285 // on systems that support death tests. This allows one to write such a macro on
00286 // a system that does not support death tests and be sure that it will compile
00287 // on a death-test supporting system. It is exposed publicly so that systems
00288 // that have death-tests with stricter requirements than GTEST_HAS_DEATH_TEST
00289 // can write their own equivalent of EXPECT_DEATH_IF_SUPPORTED and
00290 // ASSERT_DEATH_IF_SUPPORTED.
00291 //
00292 // Parameters:
00293 // statement - A statement that a macro such as EXPECT_DEATH would test
00294 // for program termination. This macro has to make sure this
00295 // statement is compiled but not executed, to ensure that
00296 // EXPECT_DEATH_IF_SUPPORTED compiles with a certain
00297 // parameter if and only if EXPECT_DEATH compiles with it.
00298 // regex - A regex that a macro such as EXPECT_DEATH would use to test
00299 // the output of statement. This parameter has to be
00300 // compiled but not evaluated by this macro, to ensure that
00301 // this macro only accepts expressions that a macro such as
00302 // EXPECT_DEATH would accept.
00303 // terminator - Must be an empty statement for EXPECT_DEATH_IF_SUPPORTED
00304 // and a return statement for ASSERT_DEATH_IF_SUPPORTED.
00305 // This ensures that ASSERT_DEATH_IF_SUPPORTED will not
00306 // compile inside functions where ASSERT_DEATH doesn't
00307 // compile.
00308 //
00309 // The branch that has an always false condition is used to ensure that
00310 // statement and regex are always compiled (and thus syntactically correct) but
00311 // never executed. The unreachable code macro protects the terminator
00312 // statement from generating an 'unreachable code' warning in case
00313 // statement unconditionally returns or throws. The Message constructor at
00314 // the end allows the syntax of streaming additional messages into the
00315 // macro, for compilational compatibility with EXPECT_DEATH/ASSERT_DEATH.
00316 #define GTEST_UNSUPPORTED_DEATH_TEST(statement, regex, terminator) \
00317 GTEST_AMBIGUOUS_ELSE_BLOCKER_ \
00318 if (::testing::internal::AlwaysTrue()) { \
00319 GTEST_LOG_(WARNING) << "Death tests are not supported on this platform.\n" \
00320 << "Statement '" #statement "' cannot be verified."; \
00321 } else if (::testing::internal::AlwaysFalse()) { \
00322 ::testing::internal::RE::PartialMatch(".", (regex)); \
00323 GTEST_SUPPRESS_UNREACHABLE_CODE_WARNING_BELOW_(statement); \
00324 terminator; \
00325 } else \
00326 ::testing::Message()

```



```

00327
00328 // EXPECT_DEATH_IF_SUPPORTED(statement, regex) and
00329 // ASSERT_DEATH_IF_SUPPORTED(statement, regex) expand to real death tests if
00330 // death tests are supported; otherwise they just issue a warning. This is
00331 // useful when you are combining death test assertions with normal test
00332 // assertions in one test.
00333 #if GTEST_HAS_DEATH_TEST
00334 #define EXPECT_DEATH_IF_SUPPORTED(statement, regex) \
00335 EXPECT_DEATH(statement, regex)
00336 #define ASSERT_DEATH_IF_SUPPORTED(statement, regex) \
00337 ASSERT_DEATH(statement, regex)
00338 #else
00339 #define EXPECT_DEATH_IF_SUPPORTED(statement, regex) \
00340 GTEST_UNSUPPORTED_DEATH_TEST(statement, regex,)
00341 #define ASSERT_DEATH_IF_SUPPORTED(statement, regex) \
00342 GTEST_UNSUPPORTED_DEATH_TEST(statement, regex, return)
00343 #endif
00344
00345 } // namespace testing
00346
00347 #endif // GOOGLETEST_INCLUDE_GTEST_GTEST_DEATH_TEST_H_

```

## 16.233 gtest-matchers.h

```

00001 // Copyright 2007, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // The Google C++ Testing and Mocking Framework (Google Test)
00031 //
00032 // This file implements just enough of the matcher interface to allow
00033 // EXPECT_DEATH and friends to accept a matcher argument.
00034
00035 // IWYU pragma: private, include "gtest/gtest.h"
00036 // IWYU pragma: friend gtest/*.h
00037 // IWYU pragma: friend gmock/*.h
00038
00039 #ifndef GOOGLETEST_INCLUDE_GTEST_GTEST_MATCHERS_H_
00040 #define GOOGLETEST_INCLUDE_GTEST_GTEST_MATCHERS_H_
00041
00042 #include <atomic>
00043 #include <memory>
00044 #include <ostream>
00045 #include <string>
00046 #include <type_traits>
00047
00048 #include "gtest/gtest-printers.h"
00049 #include "gtest/internal/gtest-internal.h"
00050 #include "gtest/internal/gtest-port.h"
00051
00052 // MSVC warning C5046 is new as of VS2017 version 15.8.
00053 #if defined(_MSC_VER) && _MSC_VER >= 1915
00054 #define GTEST_MAYBE_5046_ 5046
00055 #else
00056 #define GTEST_MAYBE_5046_
00057 #endif
00058
00059 GTEST_DISABLE_MSC_WARNINGS_PUSH_(
00060 4251 GTEST_MAYBE_5046_ /* class A needs to have dll-interface to be used by
00061 clients of class B */)

```

```

00062 /* Symbol involving type with internal linkage not defined */)
00063
00064 namespace testing {
00065
00066 // To implement a matcher Foo for type T, define:
00067 // 1. a class FooMatcherMatcher that implements the matcher interface:
00068 // using is_gtest_matcher = void;
00069 // bool MatchAndExplain(const T&, std::ostream*);
00070 // (MatchResultListener* can also be used instead of std::ostream*)
00071 // void DescribeTo(std::ostream*);
00072 // void DescribeNegationTo(std::ostream*);
00073 //
00074 // 2. a factory function that creates a Matcher<T> object from a
00075 // FooMatcherMatcher.
00076
00077 class MatchResultListener {
00078 public:
00079 // Creates a listener object with the given underlying ostream. The
00080 // listener does not own the ostream, and does not dereference it
00081 // in the constructor or destructor.
00082
00083 explicit MatchResultListener(::std::ostream* os) : stream_(os) {
00084 }
00085 virtual ~MatchResultListener() = 0; // Makes this class abstract.
00086
00087 // Streams x to the underlying ostream; does nothing if the ostream
00088 // is NULL.
00089
00090 template <typename T>
00091 MatchResultListener& operator<<(const T& x) {
00092 if (stream_ != nullptr) *stream_ << x;
00093 return *this;
00094 }
00095
00096 // Returns the underlying ostream.
00097
00098 ::std::ostream* stream() {
00099 return stream_;
00100 }
00101
00102 // Returns true if and only if the listener is interested in an explanation
00103 // of the match result. A matcher's MatchAndExplain() method can use
00104 // this information to avoid generating the explanation when no one
00105 // intends to hear it.
00106
00107 bool IsInterested() const {
00108 return stream_ != nullptr;
00109 }
00110
00111 private:
00112 ::std::ostream * const stream_;
00113
00114 MatchResultListener(const MatchResultListener&) = delete;
00115 MatchResultListener& operator=(const MatchResultListener&) = delete;
00116 };
00117
00118 inline MatchResultListener::~~MatchResultListener() {
00119 }
00120
00121 // An instance of a subclass of this knows how to describe itself as a
00122 // matcher.
00123
00124 class GTEST_API_ MatcherDescriberInterface {
00125 public:
00126
00127 virtual ~MatcherDescriberInterface() {
00128 }
00129
00130 // Describes this matcher to an ostream. The function should print
00131 // a verb phrase that describes the property a value matching this
00132 // matcher should have. The subject of the verb phrase is the value
00133 // being matched. For example, the DescribeTo() method of the Gt(7)
00134 // matcher prints "is greater than 7".
00135 virtual void DescribeTo(::std::ostream* os) const = 0;
00136
00137 // Describes the negation of this matcher to an ostream. For
00138 // example, if the description of this matcher is "is greater than
00139 // 7", the negated description could be "is not greater than 7".
00140 // You are not required to override this when implementing
00141 // MatcherInterface, but it is highly advised so that your matcher
00142 // can produce good error messages.
00143
00144 virtual void DescribeNegationTo(::std::ostream* os) const {
00145 *os << "not ";
00146 DescribeTo(os);
00147 *os << " ";
00148 }

```

```

00149 };
00150
00151 // The implementation of a matcher.
00152
00153 template <typename T>
00154 class MatcherInterface : public MatcherDescriberInterface {
00155 public:
00156 // Returns true if and only if the matcher matches x; also explains the
00157 // match result to 'listener' if necessary (see the next paragraph), in
00158 // the form of a non-restrictive relative clause ("which ...",
00159 // "whose ...", etc) that describes x. For example, the
00160 // MatchAndExplain() method of the Pointee(...) matcher should
00161 // generate an explanation like "which points to ...".
00162 //
00163 // Implementations of MatchAndExplain() should add an explanation of
00164 // the match result *if and only if* they can provide additional
00165 // information that's not already present (or not obvious) in the
00166 // print-out of x and the matcher's description. Whether the match
00167 // succeeds is not a factor in deciding whether an explanation is
00168 // needed, as sometimes the caller needs to print a failure message
00169 // when the match succeeds (e.g. when the matcher is used inside
00170 // Not()).
00171 //
00172 // For example, a "has at least 10 elements" matcher should explain
00173 // what the actual element count is, regardless of the match result,
00174 // as it is useful information to the reader; on the other hand, an
00175 // "is empty" matcher probably only needs to explain what the actual
00176 // size is when the match fails, as it's redundant to say that the
00177 // size is 0 when the value is already known to be empty.
00178 //
00179 // You should override this method when defining a new matcher.
00180 //
00181 // It's the responsibility of the caller (Google Test) to guarantee
00182 // that 'listener' is not NULL. This helps to simplify a matcher's
00183 // implementation when it doesn't care about the performance, as it
00184 // can talk to 'listener' without checking its validity first.
00185 // However, in order to implement dummy listeners efficiently,
00186 // listener->stream() may be NULL.
00187 virtual bool MatchAndExplain(T x, MatchResultListener* listener) const = 0;
00188
00189 // Inherits these methods from MatcherDescriberInterface:
00190 // virtual void DescribeTo(::std::ostream* os) const = 0;
00191 // virtual void DescribeNegationTo(::std::ostream* os) const;
00192 };
00193
00194 namespace internal {
00195
00196 struct AnyEq {
00197
00198 template <typename A, typename B>
00199 bool operator()(const A& a, const B& b) const {
00200 return a == b;
00201 }
00202 };
00203
00204 struct AnyNe {
00205
00206 template <typename A, typename B>
00207 bool operator()(const A& a, const B& b) const {
00208 return a != b;
00209 }
00210 };
00211
00212 struct AnyLt {
00213
00214 template <typename A, typename B>
00215 bool operator()(const A& a, const B& b) const {
00216 return a < b;
00217 }
00218 };
00219
00220 struct AnyGt {
00221
00222 template <typename A, typename B>
00223 bool operator()(const A& a, const B& b) const {
00224 return a > b;
00225 }
00226 };
00227
00228 struct AnyLe {
00229
00230 template <typename A, typename B>
00231 bool operator()(const A& a, const B& b) const {
00232 return a <= b;
00233 }
00234 };
00235

```

```

00236 struct AnyGe {
00237 template <typename A, typename B>
00238 bool operator()(const A& a, const B& b) const {
00239 return a >= b;
00240 }
00241 };
00242
00243 // A match result listener that ignores the explanation.
00244
00245 class DummyMatchResultListener : public MatchResultListener {
00246 public:
00247 DummyMatchResultListener() : MatchResultListener(nullptr) {}
00248
00249 private:
00250 DummyMatchResultListener(const DummyMatchResultListener&) = delete;
00251 DummyMatchResultListener& operator=(const DummyMatchResultListener&) = delete;
00252 };
00253
00254 // A match result listener that forwards the explanation to a given
00255 // ostream. The difference between this and MatchResultListener is
00256 // that the former is concrete.
00257
00258 class StreamMatchResultListener : public MatchResultListener {
00259 public:
00260 explicit StreamMatchResultListener(::std::ostream* os)
00261 : MatchResultListener(os) {}
00262
00263 private:
00264 StreamMatchResultListener(const StreamMatchResultListener&) = delete;
00265 StreamMatchResultListener& operator=(const StreamMatchResultListener&) =
00266 delete;
00267 };
00268
00269 struct SharedPayloadBase {
00270 std::atomic<int> ref{1};
00271
00272 void Ref() {
00273 ref.fetch_add(1, std::memory_order_relaxed);
00274 }
00275
00276 bool Unref() {
00277 return ref.fetch_sub(1, std::memory_order_acq_rel) == 1;
00278 }
00279 };
00280
00281 template <typename T>
00282 struct SharedPayload : SharedPayloadBase {
00283 explicit SharedPayload(const T& v) : value(v) {}
00284
00285 explicit SharedPayload(T&& v) : value(std::move(v)) {}
00286
00287 static void Destroy(SharedPayloadBase* shared) {
00288 delete static_cast<SharedPayload*>(shared);
00289 }
00290
00291 T value;
00292 };
00293
00294 // An internal class for implementing Matcher<T>, which will derive
00295 // from it. We put functionalities common to all Matcher<T>
00296 // specializations here to avoid code duplication.
00297
00298 template <typename T>
00299 class MatcherBase : private MatcherDescriberInterface {
00300 public:
00301 // Returns true if and only if the matcher matches x; also explains the
00302 // match result to 'listener'.
00303
00304 bool MatchAndExplain(const T& x, MatchResultListener* listener) const {
00305 GTEST_CHECK_(vtable_ != nullptr);
00306 return vtable_->match_and_explain(*this, x, listener);
00307 }
00308
00309 // Returns true if and only if this matcher matches x.
00310
00311 bool Matches(const T& x) const {
00312 DummyMatchResultListener dummy;
00313 return MatchAndExplain(x, &dummy);
00314 }
00315
00316 };

```

```

00323
00324 // Describes this matcher to an ostream.
00325
00326 void DescribeTo(::std::ostream* os) const final {
00327 GTEST_CHECK_(vtable_ != nullptr);
00328 vtable_->describe(*this, os, false);
00329 }
00330
00331 // Describes the negation of this matcher to an ostream.
00332
00333 void DescribeNegationTo(::std::ostream* os) const final {
00334 GTEST_CHECK_(vtable_ != nullptr);
00335 vtable_->describe(*this, os, true);
00336 }
00337
00338 // Explains why x matches, or doesn't match, the matcher.
00339
00340 void ExplainMatchResultTo(const T& x, ::std::ostream* os) const {
00341 StreamMatchResultListener listener(os);
00342 MatchAndExplain(x, &listener);
00343 }
00344
00345 // Returns the describer for this matcher object; retains ownership
00346 // of the describer, which is only guaranteed to be alive when
00347 // this matcher object is alive.
00348
00349 const MatcherDescriberInterface* GetDescriber() const {
00350 if (vtable_ == nullptr) return nullptr;
00351 return vtable_->get_describer(*this);
00352 }
00353
00354 protected:
00355
00356 MatcherBase() : vtable_(nullptr), buffer_() {
00357 }
00358
00359 // Constructs a matcher from its implementation.
00360
00361 template <typename U>
00362 explicit MatcherBase(const MatcherInterface<U>* impl)
00363 : vtable_(nullptr), buffer_() {
00364 Init(impl);
00365 }
00366
00367 template <typename M, typename = typename std::remove_reference<
00368 M>::type::is_gtest_matcher>
00369 MatcherBase(M&& m) : vtable_(nullptr), buffer_() { // NOLINT
00370 Init(std::forward<M>(m));
00371 }
00372
00373 MatcherBase(const MatcherBase& other)
00374 : vtable_(other.vtable_), buffer_(other.buffer_) {
00375 if (IsShared()) buffer_.shared->Ref();
00376 }
00377
00378 MatcherBase& operator=(const MatcherBase& other) {
00379 if (this == &other) return *this;
00380 Destroy();
00381 vtable_ = other.vtable_;
00382 buffer_ = other.buffer_;
00383 if (IsShared()) buffer_.shared->Ref();
00384 return *this;
00385 }
00386
00387 MatcherBase(MatcherBase&& other)
00388 : vtable_(other.vtable_), buffer_(other.buffer_) {
00389 other.vtable_ = nullptr;
00390 }
00391
00392 MatcherBase& operator=(MatcherBase&& other) {
00393 if (this == &other) return *this;
00394 Destroy();
00395 vtable_ = other.vtable_;
00396 buffer_ = other.buffer_;
00397 other.vtable_ = nullptr;
00398 return *this;
00399 }
00400
00401 ~MatcherBase() override {
00402 Destroy();
00403 }
00404
00405 private:
00406
00407 struct VTable {
00408 bool (*match_and_explain)(const MatcherBase&, const T&,
00409 MatchResultListener*);

```

```

00410 void (*describe)(const MatcherBase&, std::ostream*, bool negation);
00411 // Returns the captured object if it implements the interface, otherwise
00412 // returns the MatcherBase itself.
00413 const MatcherDescriberInterface* (*get_describer)(const MatcherBase&);
00414 // Called on shared instances when the reference count reaches 0.
00415 void (*shared_destroy)(SharedPayloadBase*);
00416 };
00417
00418 bool IsShared() const {
00419 return vtable_ != nullptr && vtable_>shared_destroy != nullptr;
00420 }
00421
00422 // If the implementation uses a listener, call that.
00423 template <typename P>
00424 static auto MatchAndExplainImpl(const MatcherBase& m, const T& value,
00425 MatchResultListener* listener)
00426 -> decltype(P::Get(m).MatchAndExplain(value, listener->stream())) {
00427 return P::Get(m).MatchAndExplain(value, listener->stream());
00428 }
00429
00430 template <typename P>
00431 static auto MatchAndExplainImpl(const MatcherBase& m, const T& value,
00432 MatchResultListener* listener)
00433 -> decltype(P::Get(m).MatchAndExplain(value, listener)) {
00434 return P::Get(m).MatchAndExplain(value, listener);
00435 }
00436
00437 template <typename P>
00438 static void DescribeImpl(const MatcherBase& m, std::ostream* os,
00439 bool negation) {
00440 if (negation) {
00441 P::Get(m).DescribeNegationTo(os);
00442 } else {
00443 P::Get(m).DescribeTo(os);
00444 }
00445 }
00446
00447 template <typename P>
00448 static const MatcherDescriberInterface* GetDescriberImpl(
00449 const MatcherBase& m) {
00450 // If the impl is a MatcherDescriberInterface, then return it.
00451 // Otherwise use MatcherBase itself.
00452 // This allows us to implement the GetDescriber() function without support
00453 // from the impl, but some users really want to get their impl back when
00454 // they call GetDescriber().
00455 // We use std::get on a tuple as a workaround of not having `if constexpr`.
00456 return std::get < (
00457 std::is_convertible<decltype(&P::Get(m)),
00458 const MatcherDescriberInterface*>::value
00459 ? 1
00460 : 0)>(std::make_tuple(&m, &P::Get(m)));
00461 }
00462
00463 template <typename P>
00464 const VTable* GetVTable() {
00465 static constexpr VTable kVTable = {&MatchAndExplainImpl<P>,
00466 &DescribeImpl<P>, &GetDescriberImpl<P>,
00467 P::shared_destroy};
00468 return &kVTable;
00469 }
00470
00471 union Buffer {
00472 // Add some types to give Buffer some common alignment/size use cases.
00473 void* ptr;
00474 double d;
00475 int64_t i;
00476 // And add one for the out-of-line cases.
00477 SharedPayloadBase* shared;
00478 };
00479
00480 void Destroy() {
00481 if (IsShared() && buffer_.shared->Unref()) {
00482 vtable_>shared_destroy(buffer_.shared);
00483 }
00484 }
00485
00486 template <typename M>
00487 static constexpr bool IsInlined() {
00488 return sizeof(M) <= sizeof(Buffer) && alignof(M) <= alignof(Buffer) &&
00489 std::is_trivially_copy_constructible<M>::value &&
00490 std::is_trivially_destructible<M>::value;
00491 }
00492
00493 template <typename M, bool = MatcherBase::IsInlined<M>()>
00494 struct ValuePolicy {
00495
00496 static const M& Get(const MatcherBase& m) {

```

```

00497 // When inlined along with Init, need to be explicit to avoid violating
00498 // strict aliasing rules.
00499 const M* ptr =
00500 static_cast<const M*> (static_cast<const void*> (&m.buffer_));
00501 return *ptr;
00502 }
00503
00504 static void Init(MatcherBase& m, M impl) {
00505 ::new (static_cast<void*> (&m.buffer_)) M(impl);
00506 }
00507 static constexpr auto shared_destroy = nullptr;
00508 };
00509
00510 template <typename M>
00511 struct ValuePolicy<M, false> {
00512 using Shared = SharedPayload<M>;
00513
00514 static const M& Get(const MatcherBase& m) {
00515 return static_cast<Shared*> (m.buffer_.shared)->value;
00516 }
00517
00518 template <typename Arg>
00519 static void Init(MatcherBase& m, Arg&& arg) {
00520 m.buffer_.shared = new Shared(std::forward<Arg>(arg));
00521 }
00522 static constexpr auto shared_destroy = &Shared::Destroy;
00523 };
00524
00525 template <typename U, bool B>
00526 struct ValuePolicy<const MatcherInterface<U>*, B> {
00527 using M = const MatcherInterface<U>;
00528 using Shared = SharedPayload<std::unique_ptr<M>>;
00529
00530 static const M& Get(const MatcherBase& m) {
00531 return *static_cast<Shared*> (m.buffer_.shared)->value;
00532 }
00533
00534 static void Init(MatcherBase& m, M* impl) {
00535 m.buffer_.shared = new Shared(std::unique_ptr<M>(impl));
00536 }
00537
00538 static constexpr auto shared_destroy = &Shared::Destroy;
00539 };
00540
00541 template <typename M>
00542 void Init(M&& m) {
00543 using MM = typename std::decay<M>::type;
00544 using Policy = ValuePolicy<MM>;
00545 vtable_ = GetVTable<Policy>();
00546 Policy::Init(*this, std::forward<M>(m));
00547 }
00548
00549 const VTable* vtable_;
00550 Buffer buffer_;
00551 };
00552
00553 } // namespace internal
00554
00555 // A Matcher<T> is a copyable and IMMUTABLE (except by assignment)
00556 // object that can check whether a value of type T matches. The
00557 // implementation of Matcher<T> is just a std::shared_ptr to const
00558 // MatcherInterface<T>. Don't inherit from Matcher!
00559
00560 template <typename T>
00561 class Matcher : public internal::MatcherBase<T> {
00562 public:
00563 // Constructs a null matcher. Needed for storing Matcher objects in STL
00564 // containers. A default-constructed matcher is not yet initialized. You
00565 // cannot use it until a valid value has been assigned to it.
00566
00567 explicit Matcher() {
00568 } // NOLINT
00569
00570 // Constructs a matcher from its implementation.
00571
00572 explicit Matcher(const MatcherInterface<const T*>* impl)
00573 : internal::MatcherBase<T>(impl) {
00574 }
00575
00576 template <typename U>
00577 explicit Matcher(
00578 const MatcherInterface<U>* impl,
00579 typename std::enable_if<!std::is_same<U, const U&::value>::type* =
00580 nullptr>
00581) : internal::MatcherBase<T>(impl) {
00582 }
00583

```

```

00584 template <typename M, typename = typename std::remove_reference<
00585 M>::type::is_gtest_matcher>
00586 Matcher(M&& m) : internal::MatcherBase<T>(std::forward<M>(m)) {
00587 } // NOLINT
00588
00589 // Implicit constructor here allows people to write
00590 // EXPECT_CALL(foo, Bar(5)) instead of EXPECT_CALL(foo, Bar(Eq(5))) sometimes
00591 Matcher(T value); // NOLINT
00592 };
00593
00594 // The following two specializations allow the user to write str
00595 // instead of Eq(str) and "foo" instead of Eq("foo") when a std::string
00596 // matcher is expected.
00597
00598 template <>
00599 class GTEST_API_ Matcher<const std::string&>
00600 : public internal::MatcherBase<const std::string&> {
00601 public:
00602 Matcher() {
00603 }
00604
00605 explicit Matcher(const MatcherInterface<const std::string&>* impl)
00606 : internal::MatcherBase<const std::string&>(impl) {
00607 }
00608
00609 template <typename M, typename = typename std::remove_reference<
00610 M>::type::is_gtest_matcher>
00611 Matcher(M&& m) // NOLINT
00612 : internal::MatcherBase<const std::string&>(std::forward<M>(m)) {
00613 }
00614
00615 // Allows the user to write str instead of Eq(str) sometimes, where
00616 // str is a std::string object.
00617 Matcher(const std::string& s); // NOLINT
00618
00619 // Allows the user to write "foo" instead of Eq("foo") sometimes.
00620 Matcher(const char* s); // NOLINT
00621 };
00622
00623 template <>
00624 class GTEST_API_ Matcher<std::string>
00625 : public internal::MatcherBase<std::string> {
00626 public:
00627 Matcher() {
00628 }
00629
00630 explicit Matcher(const MatcherInterface<const std::string&>* impl)
00631 : internal::MatcherBase<std::string>(impl) {
00632 }
00633
00634 explicit Matcher(const MatcherInterface<std::string>* impl)
00635 : internal::MatcherBase<std::string>(impl) {
00636 }
00637
00638 template <typename M, typename = typename std::remove_reference<
00639 M>::type::is_gtest_matcher>
00640 Matcher(M&& m) // NOLINT
00641 : internal::MatcherBase<std::string>(std::forward<M>(m)) {
00642 }
00643
00644 // Allows the user to write str instead of Eq(str) sometimes, where
00645 // str is a string object.
00646 Matcher(const std::string& s); // NOLINT
00647
00648 // Allows the user to write "foo" instead of Eq("foo") sometimes.
00649 Matcher(const char* s); // NOLINT
00650 };
00651
00652 #if GTEST_INTERNAL_HAS_STRING_VIEW
00653
00654 // The following two specializations allow the user to write str
00655 // instead of Eq(str) and "foo" instead of Eq("foo") when a absl::string_view
00656 // matcher is expected.
00657
00658 template <>
00659 class GTEST_API_ Matcher<const internal::StringView&>
00660 : public internal::MatcherBase<const internal::StringView&> {
00661 public:
00662 Matcher() {
00663 }
00664
00665 explicit Matcher(const MatcherInterface<const internal::StringView&>* impl)
00666 : internal::MatcherBase<const internal::StringView&>(impl) {
00667 }
00668
00669 explicit Matcher(const MatcherInterface<internal::StringView>* impl)
00670 : internal::MatcherBase<internal::StringView>(impl) {
00671 }
00672

```



```

00671 template <typename M, typename = typename std::remove_reference<
00672 M>::type::is_gtest_matcher>
00673 Matcher(M&& m) // NOLINT
00674 : internal::MatcherBase<const internal::StringView&>(std::forward<M>(m)) {
00675 }
00676
00677 // Allows the user to write str instead of Eq(str) sometimes, where
00678 // str is a std::string object.
00679 Matcher(const std::string& s); // NOLINT
00680
00681 // Allows the user to write "foo" instead of Eq("foo") sometimes.
00682 Matcher(const char* s); // NOLINT
00683
00684 // Allows the user to pass absl::string_views or std::string_views directly.
00685 Matcher(internal::StringView s); // NOLINT
00686 };
00687
00688 template <>
00689 class GTEST_API_ Matcher<internal::StringView>
00690 : public internal::MatcherBase<internal::StringView> {
00691 public:
00692 Matcher() {
00693 }
00694
00695 explicit Matcher(const MatcherInterface<const internal::StringView&>* impl)
00696 : internal::MatcherBase<internal::StringView>(impl) {
00697 }
00698
00699 explicit Matcher(const MatcherInterface<internal::StringView>* impl)
00700 : internal::MatcherBase<internal::StringView>(impl) {
00701 }
00702 }
00703
00704 template <typename M, typename = typename std::remove_reference<
00705 M>::type::is_gtest_matcher>
00706 Matcher(M&& m) // NOLINT
00707 : internal::MatcherBase<internal::StringView>(std::forward<M>(m)) {
00708 }
00709
00710 // Allows the user to write str instead of Eq(str) sometimes, where
00711 // str is a std::string object.
00712 Matcher(const std::string& s); // NOLINT
00713
00714 // Allows the user to write "foo" instead of Eq("foo") sometimes.
00715 Matcher(const char* s); // NOLINT
00716
00717 // Allows the user to pass absl::string_views or std::string_views directly.
00718 Matcher(internal::StringView s); // NOLINT
00719 };
00720 #ifdef GTEST_INTERNAL_HAS_STRING_VIEW
00721
00722 // Prints a matcher in a human-readable format.
00723
00724 template <typename T>
00725 std::ostream& operator<<(std::ostream& os, const Matcher<T>& matcher) {
00726 matcher.DescribeTo(&os);
00727 return os;
00728 }
00729
00730 // The PolymorphicMatcher class template makes it easy to implement a
00731 // polymorphic matcher (i.e. a matcher that can match values of more
00732 // than one type, e.g. Eq(n) and NotNull()).
00733 //
00734 // To define a polymorphic matcher, a user should provide an Impl
00735 // class that has a DescribeTo() method and a DescribeNegationTo()
00736 // method, and define a member function (or member function template)
00737 //
00738 // bool MatchAndExplain(const Value& value,
00739 // MatchResultListener* listener) const;
00740 //
00741 // See the definition of NotNull() for a complete example.
00742
00743 template <class Impl>
00744 class PolymorphicMatcher {
00745 public:
00746
00747 explicit PolymorphicMatcher(const Impl& an_impl) : impl_(an_impl) {
00748 }
00749
00750 // Returns a mutable reference to the underlying matcher
00751 // implementation object.
00752
00753 Impl& mutable_impl() {
00754 return impl_;
00755 }
00756
00757 // Returns an immutable reference to the underlying matcher

```

```

00758 // implementation object.
00759
00760 const Impl& impl() const {
00761 return impl_;
00762 }
00763
00764 template <typename T>
00765 operator Matcher<T>() const {
00766 return Matcher<T>(new MonomorphicImpl<const T&>(impl_));
00767 }
00768
00769 private:
00770
00771 template <typename T>
00772 class MonomorphicImpl : public MatcherInterface<T> {
00773 public:
00774
00775 explicit MonomorphicImpl(const Impl& impl) : impl_(impl) {
00776 }
00777
00778 void DescribeTo(::std::ostream* os) const override {
00779 impl_.DescribeTo(os);
00780 }
00781
00782 void DescribeNegationTo(::std::ostream* os) const override {
00783 impl_.DescribeNegationTo(os);
00784 }
00785
00786 bool MatchAndExplain(T x, MatchResultListener* listener) const override {
00787 return impl_.MatchAndExplain(x, listener);
00788 }
00789
00790 private:
00791 const Impl impl_;
00792 };
00793
00794 Impl impl_;
00795 };
00796
00797 // Creates a matcher from its implementation.
00798 // DEPRECATED: Especially in the generic code, prefer:
00799 // Matcher<T>(new MyMatcherImpl<const T&>(...));
00800 //
00801 // MakeMatcher may create a Matcher that accepts its argument by value, which
00802 // leads to unnecessary copies & lack of support for non-copyable types.
00803
00804 template <typename T>
00805 inline Matcher<T> MakeMatcher(const MatcherInterface<T>* impl) {
00806 return Matcher<T>(impl);
00807 }
00808
00809 // Creates a polymorphic matcher from its implementation. This is
00810 // easier to use than the PolymorphicMatcher<Impl> constructor as it
00811 // doesn't require you to explicitly write the template argument, e.g.
00812 //
00813 // MakePolymorphicMatcher(foo);
00814 // vs
00815 // PolymorphicMatcher<TypeOfFoo>(foo);
00816
00817 template <class Impl>
00818 inline PolymorphicMatcher<Impl> MakePolymorphicMatcher(const Impl& impl) {
00819 return PolymorphicMatcher<Impl>(impl);
00820 }
00821
00822 namespace internal {
00823 // Implements a matcher that compares a given value with a
00824 // pre-supplied value using one of the ==, <=, <, etc, operators. The
00825 // two values being compared don't have to have the same type.
00826 //
00827 // The matcher defined here is polymorphic (for example, Eq(5) can be
00828 // used to match an int, a short, a double, etc). Therefore we use
00829 // a template type conversion operator in the implementation.
00830 //
00831 // The following template definition assumes that the rhs parameter is
00832 // a "bare" type (i.e. neither 'const T' nor 'T&').
00833
00834 template <typename D, typename Rhs, typename Op>
00835 class ComparisonBase {
00836 public:
00837
00838 explicit ComparisonBase(const Rhs& rhs) : rhs_(rhs) {
00839 }
00840
00841 using is_gtest_matcher = void;
00842
00843 template <typename Lhs>
00844 bool MatchAndExplain(const Lhs& lhs, std::ostream*) const {

```

```

00845 return Op()(lhs, Unwrap(rhs_));
00846 }
00847
00848 void DescribeTo(std::ostream* os) const {
00849 *os << D::Desc() << " ";
00850 UniversalPrint(Unwrap(rhs_), os);
00851 }
00852
00853 void DescribeNegationTo(std::ostream* os) const {
00854 *os << D::NegatedDesc() << " ";
00855 UniversalPrint(Unwrap(rhs_), os);
00856 }
00857
00858 private:
00859
00860 template <typename T>
00861 static const T& Unwrap(const T& v) {
00862 return v;
00863 }
00864
00865 template <typename T>
00866 static const T& Unwrap(std::reference_wrapper<T> v) {
00867 return v;
00868 }
00869
00870 Rhs rhs_;
00871 };
00872
00873 template <typename Rhs>
00874 class EqMatcher : public ComparisonBase<EqMatcher<Rhs>, Rhs, AnyEq> {
00875 public:
00876
00877 explicit EqMatcher(const Rhs& rhs)
00878 : ComparisonBase<EqMatcher<Rhs>, Rhs, AnyEq>(rhs) {
00879 }
00880
00881 static const char* Desc() {
00882 return "is equal to";
00883 }
00884
00885 static const char* NegatedDesc() {
00886 return "isn't equal to";
00887 }
00888 };
00889
00890 template <typename Rhs>
00891 class NeMatcher : public ComparisonBase<NeMatcher<Rhs>, Rhs, AnyNe> {
00892 public:
00893
00894 explicit NeMatcher(const Rhs& rhs)
00895 : ComparisonBase<NeMatcher<Rhs>, Rhs, AnyNe>(rhs) {
00896 }
00897
00898 static const char* Desc() {
00899 return "isn't equal to";
00900 }
00901
00902 static const char* NegatedDesc() {
00903 return "is equal to";
00904 }
00905 };
00906
00907 template <typename Rhs>
00908 class LtMatcher : public ComparisonBase<LtMatcher<Rhs>, Rhs, AnyLt> {
00909 public:
00910
00911 explicit LtMatcher(const Rhs& rhs)
00912 : ComparisonBase<LtMatcher<Rhs>, Rhs, AnyLt>(rhs) {
00913 }
00914
00915 static const char* Desc() {
00916 return "is <";
00917 }
00918
00919 static const char* NegatedDesc() {
00920 return "isn't <";
00921 }
00922 };
00923
00924 template <typename Rhs>
00925 class GtMatcher : public ComparisonBase<GtMatcher<Rhs>, Rhs, AnyGt> {
00926 public:
00927
00928 explicit GtMatcher(const Rhs& rhs)
00929 : ComparisonBase<GtMatcher<Rhs>, Rhs, AnyGt>(rhs) {
00930 }
00931

```

```

00932 static const char* Desc() {
00933 return "is >";
00934 }
00935
00936 static const char* NegatedDesc() {
00937 return "isn't >";
00938 }
00939 };
00940
00941 template <typename Rhs>
00942 class LeMatcher : public ComparisonBase<LeMatcher<Rhs>, Rhs, AnyLe> {
00943 public:
00944
00945 explicit LeMatcher(const Rhs& rhs)
00946 : ComparisonBase<LeMatcher<Rhs>, Rhs, AnyLe>(rhs) {
00947 }
00948
00949 static const char* Desc() {
00950 return "is <=";
00951 }
00952
00953 static const char* NegatedDesc() {
00954 return "isn't <=";
00955 }
00956 };
00957
00958 template <typename Rhs>
00959 class GeMatcher : public ComparisonBase<GeMatcher<Rhs>, Rhs, AnyGe> {
00960 public:
00961
00962 explicit GeMatcher(const Rhs& rhs)
00963 : ComparisonBase<GeMatcher<Rhs>, Rhs, AnyGe>(rhs) {
00964 }
00965
00966 static const char* Desc() {
00967 return "is >=";
00968 }
00969
00970 static const char* NegatedDesc() {
00971 return "isn't >=";
00972 }
00973 };
00974
00975 template <typename T, typename = typename std::enable_if<
00976 std::is_constructible<std::string, T::value>::type>
00977 using StringLike = T;
00978
00979 // Implements polymorphic matchers MatchesRegex(regex) and
00980 // ContainsRegex(regex), which can be used as a Matcher<T> as long as
00981 // T can be converted to a string.
00982
00983 class MatchesRegexMatcher {
00984 public:
00985
00986 MatchesRegexMatcher(const RE* regex, bool full_match)
00987 : regex_(regex), full_match_(full_match) {
00988 }
00989
00990 #if GTEST_INTERNAL_HAS_STRING_VIEW
00991
00992 bool MatchAndExplain(const internal::StringView& s,
00993 MatchResultListener* listener) const {
00994 return MatchAndExplain(std::string(s), listener);
00995 }
00996 #endif // GTEST_INTERNAL_HAS_STRING_VIEW
00997
00998 // Accepts pointer types, particularly:
00999 // const char*
01000 // char*
01001 // const wchar_t*
01002 // wchar_t*
01003
01004 template <typename CharType>
01005 bool MatchAndExplain(CharType* s, MatchResultListener* listener) const {
01006 return s != nullptr && MatchAndExplain(std::string(s), listener);
01007 }
01008
01009 // Matches anything that can convert to std::string.
01010 //
01011 // This is a template, not just a plain function with const std::string&,
01012 // because absl::string_view has some interfering non-explicit constructors.
01013
01014 template <class MatcheeStringType>
01015 bool MatchAndExplain(const MatcheeStringType& s,
01016 MatchResultListener* /* listener */) const {
01017 const std::string & s2(s);
01018 return full_match_ ? RE::FullMatch(s2, *regex_)

```

```

01019 : RE::PartialMatch(s2, *regex_);
01020 }
01021
01022 void DescribeTo(::std::ostream* os) const {
01023 *os << (full_match_ ? "matches" : "contains") << " regular expression ";
01024 UniversalPrinter<std::string>::Print(regex_>pattern(), os);
01025 }
01026
01027 void DescribeNegationTo(::std::ostream* os) const {
01028 *os << "doesn't " << (full_match_ ? "match" : "contain")
01029 << " regular expression ";
01030 UniversalPrinter<std::string>::Print(regex_>pattern(), os);
01031 }
01032
01033 private:
01034 const std::shared_ptr<const RE> regex_;
01035 const bool full_match_;
01036 };
01037 } // namespace internal
01038
01039 // Matches a string that fully matches regular expression 'regex'.
01040 // The matcher takes ownership of 'regex'.
01041
01042 inline PolymorphicMatcher<internal::MatchesRegexMatcher> MatchesRegex(
01043 const internal::RE* regex) {
01044 return MakePolymorphicMatcher(internal::MatchesRegexMatcher(regex, true));
01045 }
01046
01047 template <typename T = std::string>
01048 PolymorphicMatcher<internal::MatchesRegexMatcher> MatchesRegex(
01049 const internal::StringLike<T>& regex) {
01050 return MatchesRegex(new internal::RE(std::string(regex)));
01051 }
01052
01053 // Matches a string that contains regular expression 'regex'.
01054 // The matcher takes ownership of 'regex'.
01055
01056 inline PolymorphicMatcher<internal::MatchesRegexMatcher> ContainsRegex(
01057 const internal::RE* regex) {
01058 return MakePolymorphicMatcher(internal::MatchesRegexMatcher(regex, false));
01059 }
01060
01061 template <typename T = std::string>
01062 PolymorphicMatcher<internal::MatchesRegexMatcher> ContainsRegex(
01063 const internal::StringLike<T>& regex) {
01064 return ContainsRegex(new internal::RE(std::string(regex)));
01065 }
01066
01067 // Creates a polymorphic matcher that matches anything equal to x.
01068 // Note: if the parameter of Eq() were declared as const T&, Eq("foo")
01069 // wouldn't compile.
01070
01071 template <typename T>
01072 inline internal::EqMatcher<T> Eq(T x) {
01073 return internal::EqMatcher<T>(x);
01074 }
01075
01076 // Constructs a Matcher<T> from a 'value' of type T. The constructed
01077 // matcher matches any value that's equal to 'value'.
01078
01079 template <typename T>
01080 Matcher<T>::Matcher(T value) {
01081 *this = Eq(value);
01082 }
01083
01084 // Creates a monomorphic matcher that matches anything with type Lhs
01085 // and equal to rhs. A user may need to use this instead of Eq(...)
01086 // in order to resolve an overloading ambiguity.
01087 //
01088 // TypedEq<T>(x) is just a convenient short-hand for Matcher<T>(Eq(x))
01089 // or Matcher<T>(x), but more readable than the latter.
01090 //
01091 // We could define similar monomorphic matchers for other comparison
01092 // operations (e.g. TypedLt, TypedGe, and etc), but decided not to do
01093 // it yet as those are used much less than Eq() in practice. A user
01094 // can always write Matcher<T>(Lt(5)) to be explicit about the type,
01095 // for example.
01096
01097 template <typename Lhs, typename Rhs>
01098 inline Matcher<Lhs> TypedEq(const Rhs& rhs) {
01099 return Eq(rhs);
01100 }
01101
01102 // Creates a polymorphic matcher that matches anything >= x.
01103
01104 template <typename Rhs>
01105 inline internal::GeMatcher<Rhs> Ge(Rhs x) {

```

```

01106 return internal::GeMatcher<Rhs>(x);
01107 }
01108
01109 // Creates a polymorphic matcher that matches anything > x.
01110
01111 template <typename Rhs>
01112 inline internal::GtMatcher<Rhs> Gt(Rhs x) {
01113 return internal::GtMatcher<Rhs>(x);
01114 }
01115
01116 // Creates a polymorphic matcher that matches anything <= x.
01117
01118 template <typename Rhs>
01119 inline internal::LeMatcher<Rhs> Le(Rhs x) {
01120 return internal::LeMatcher<Rhs>(x);
01121 }
01122
01123 // Creates a polymorphic matcher that matches anything < x.
01124
01125 template <typename Rhs>
01126 inline internal::LtMatcher<Rhs> Lt(Rhs x) {
01127 return internal::LtMatcher<Rhs>(x);
01128 }
01129
01130 // Creates a polymorphic matcher that matches anything != x.
01131
01132 template <typename Rhs>
01133 inline internal::NeMatcher<Rhs> Ne(Rhs x) {
01134 return internal::NeMatcher<Rhs>(x);
01135 }
01136 } // namespace testing
01137
01138 GTEST_DISABLE_MSC_WARNINGS_POP_() // 4251 5046
01139
01140 #endif // GOOGLETEST_INCLUDE_GTEST_GTEST_MATCHERS_H_

```

## 16.234 gtest-message.h

```

00001 // Copyright 2005, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // The Google C++ Testing and Mocking Framework (Google Test)
00031 //
00032 // This header file defines the Message class.
00033 //
00034 // IMPORTANT NOTE: Due to limitation of the C++ language, we have to
00035 // leave some internal implementation details in this header file.
00036 // They are clearly marked by comments like this:
00037 //
00038 // // INTERNAL IMPLEMENTATION - DO NOT USE IN A USER PROGRAM.
00039 //
00040 // Such code is NOT meant to be used by a user directly, and is subject
00041 // to CHANGE WITHOUT NOTICE. Therefore DO NOT DEPEND ON IT in a user
00042 // program!
00043
00044 // IWYU pragma: private, include "gtest/gtest.h"
00045 // IWYU pragma: friend gtest/*.h
00046 // IWYU pragma: friend gmock/*.h
00047

```

```

00048 #ifndef GOOGLETEST_INCLUDE_GTEST_GTEST_MESSAGE_H_
00049 #define GOOGLETEST_INCLUDE_GTEST_GTEST_MESSAGE_H_
00050
00051 #include <limits>
00052 #include <memory>
00053 #include <sstream>
00054
00055 #include "gtest/internal/gtest-port.h"
00056
00057 GTEST_DISABLE_MSC_WARNINGS_PUSH_(4251 \
00058 /* class A needs to have dll-interface to be used by clients of class B */)
00059
00060 // Ensures that there is at least one operator« in the global namespace.
00061 // See Message& operator«(...) below for why.
00062 void operator«(const testing::internal::Secret&, int);
00063
00064 namespace testing {
00065
00066 // The Message class works like an ostream repeater.
00067 //
00068 // Typical usage:
00069 //
00070 // 1. You stream a bunch of values to a Message object.
00071 // It will remember the text in a stringstream.
00072 // 2. Then you stream the Message object to an ostream.
00073 // This causes the text in the Message to be streamed
00074 // to the ostream.
00075 //
00076 // For example;
00077 //
00078 // testing::Message foo;
00079 // foo « 1 « " " != " « 2;
00080 // std::cout « foo;
00081 //
00082 // will print "1 != 2".
00083 //
00084 // Message is not intended to be inherited from. In particular, its
00085 // destructor is not virtual.
00086 //
00087 // Note that stringstream behaves differently in gcc and in MSVC. You
00088 // can stream a NULL char pointer to it in the former, but not in the
00089 // latter (it causes an access violation if you do). The Message
00090 // class hides this difference by treating a NULL char pointer as
00091 // "(null)".
00092
00093 class GTEST_API_ Message {
00094 private:
00095 // The type of basic IO manipulators (endl, ends, and flush) for
00096 // narrow streams.
00097 typedef std::ostream& (*BasicNarrowIoManip)(std::ostream&);
00098
00099 public:
00100 // Constructs an empty Message.
00101 Message();
00102
00103 // Copy constructor.
00104
00105 Message(const Message& msg) : ss_(new ::std::stringstream) { // NOLINT
00106 *ss_ « msg.GetString();
00107 }
00108
00109 // Constructs a Message from a C-string.
00110
00111 explicit Message(const char* str) : ss_(new ::std::stringstream) {
00112 *ss_ « str;
00113 }
00114
00115 // Streams a non-pointer value to this object.
00116
00117 template <typename T>
00118 inline Message& operator«(const T& val) {
00119 // Some libraries overload « for STL containers. These
00120 // overloads are defined in the global namespace instead of ::std.
00121 //
00122 // C++'s symbol lookup rule (i.e. Koenig lookup) says that these
00123 // overloads are visible in either the std namespace or the global
00124 // namespace, but not other namespaces, including the testing
00125 // namespace which Google Test's Message class is in.
00126 //
00127 // To allow STL containers (and other types that has a « operator
00128 // defined in the global namespace) to be used in Google Test
00129 // assertions, testing::Message must access the custom « operator
00130 // from the global namespace. With this using declaration,
00131 // overloads of « defined in the global namespace and those
00132 // visible via Koenig lookup are both exposed in this function.
00133 using ::operator«;
00134 *ss_ « val;

```

```

00135 return *this;
00136 }
00137
00138 // Streams a pointer value to this object.
00139 //
00140 // This function is an overload of the previous one. When you
00141 // stream a pointer to a Message, this definition will be used as it
00142 // is more specialized. (The C++ Standard, section
00143 // [temp.func.order].) If you stream a non-pointer, then the
00144 // previous definition will be used.
00145 //
00146 // The reason for this overload is that streaming a NULL pointer to
00147 // ostream is undefined behavior. Depending on the compiler, you
00148 // may get "0", "(nil)", "(null)", or an access violation. To
00149 // ensure consistent result across compilers, we always treat NULL
00150 // as "(null)".
00151
00152 template <typename T>
00153 inline Message& operator<<(T * const& pointer) { // NOLINT
00154 if (pointer == nullptr) {
00155 *ss_ << "(null)";
00156 } else {
00157 *ss_ << pointer;
00158 }
00159 return *this;
00160 }
00161
00162 // Since the basic IO manipulators are overloaded for both narrow
00163 // and wide streams, we have to provide this specialized definition
00164 // of operator <<, even though its body is the same as the
00165 // templatized version above. Without this definition, streaming
00166 // endl or other basic IO manipulators to Message will confuse the
00167 // compiler.
00168
00169 Message& operator<<(BasicNarrowIoManip val) {
00170 *ss_ << val;
00171 return *this;
00172 }
00173
00174 // Instead of 1/0, we want to see true/false for bool values.
00175
00176 Message& operator<<(bool b) {
00177 return *this << (b ? "true" : "false");
00178 }
00179
00180 // These two overloads allow streaming a wide C string to a Message
00181 // using the UTF-8 encoding.
00182 Message& operator<<(const wchar_t* wide_c_str);
00183 Message& operator<<(wchar_t* wide_c_str);
00184
00185 #if GTEST_HAS_STD_WSTRING
00186 // Converts the given wide string to a narrow string using the UTF-8
00187 // encoding, and streams the result to this Message object.
00188 Message& operator<<(const ::std::wstring& wstr);
00189 #endif // GTEST_HAS_STD_WSTRING
00190
00191 // Gets the text streamed to this object so far as an std::string.
00192 // Each '\0' character in the buffer is replaced with "\\0".
00193 //
00194 // INTERNAL IMPLEMENTATION - DO NOT USE IN A USER PROGRAM.
00195 std::string GetString() const;
00196
00197 private:
00198 // We'll hold the text streamed to this object here.
00199 const std::unique_ptr< ::std::stringstream> ss_;
00200
00201 // We declare (but don't implement) this to prevent the compiler
00202 // from implementing the assignment operator.
00203 void operator=(const Message&);
00204 };
00205
00206 // Streams a Message to an ostream.
00207
00208 inline std::ostream& operator<<(std::ostream& os, const Message& sb) {
00209 return os << sb.GetString();
00210 }
00211
00212 namespace internal {
00213
00214 // Converts a streamable value to an std::string. A NULL pointer is
00215 // converted to "(null)". When the input value is a ::string,
00216 // ::std::string, ::wstring, or ::std::wstring object, each NUL
00217 // character in it is replaced with "\\0".
00218
00219 template <typename T>
00220 std::string StreamableToString(const T& streamable) {
00221 return (Message() << streamable).GetString();
00222 }

```



```

00222 }
00223
00224 } // namespace internal
00225 } // namespace testing
00226
00227 GTEST_DISABLE_MSC_WARNINGS_POP_() // 4251
00228
00229 #endif // GOOGLETEST_INCLUDE_GTEST_GTEST_MESSAGE_H_

```

## 16.235 gtest-param-test.h

```

00001 // Copyright 2008, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // Macros and functions for implementing parameterized tests
00031 // in Google C++ Testing and Mocking Framework (Google Test)
00032
00033 // IWYU pragma: private, include "gtest/gtest.h"
00034 // IWYU pragma: friend gtest/*.h
00035 // IWYU pragma: friend gmock/*.h
00036
00037 #ifndef GOOGLETEST_INCLUDE_GTEST_GTEST_PARAM_TEST_H_
00038 #define GOOGLETEST_INCLUDE_GTEST_GTEST_PARAM_TEST_H_
00039
00040 // Value-parameterized tests allow you to test your code with different
00041 // parameters without writing multiple copies of the same test.
00042 //
00043 // Here is how you use value-parameterized tests:
00044 //
00045 // #if 0
00046 //
00047 // To write value-parameterized tests, first you should define a fixture
00048 // class. It is usually derived from testing::TestWithParam<T> (see below for
00049 // another inheritance scheme that's sometimes useful in more complicated
00050 // class hierarchies), where the type of your parameter values.
00051 // TestWithParam<T> is itself derived from testing::Test. T can be any
00052 // copyable type. If it's a raw pointer, you are responsible for managing the
00053 // lifespan of the pointed values.
00054
00055 class FooTest : public ::testing::TestWithParam<const char*> {
00056 // You can implement all the usual class fixture members here.
00057 };
00058
00059 // Then, use the TEST_P macro to define as many parameterized tests
00060 // for this fixture as you want. The _P suffix is for "parameterized"
00061 // or "pattern", whichever you prefer to think.
00062
00063 TEST_P(FooTest, DoesBlah) {
00064 // Inside a test, access the test parameter with the GetParam() method
00065 // of the TestWithParam<T> class:
00066 EXPECT_TRUE(foo.Blah(GetParam()));
00067 ...
00068 }
00069
00070 TEST_P(FooTest, HasBlahBlah) {
00071 ...
00072 }
00073
00074 // Finally, you can use INSTANTIATE_TEST_SUITE_P to instantiate the test

```

```

00075 // case with any set of parameters you want. Google Test defines a number
00076 // of functions for generating test parameters. They return what we call
00077 // (surprise!) parameter generators. Here is a summary of them, which
00078 // are all in the testing namespace:
00079 //
00080 //
00081 // Range(begin, end [, step]) - Yields values {begin, begin+step,
00082 // begin+step+step, ...}. The values do not
00083 // include end. step defaults to 1.
00084 // Values(v1, v2, ..., vN) - Yields values {v1, v2, ..., vN}.
00085 // ValuesIn(container) - Yields values from a C-style array, an STL
00086 // container, or an iterator range [begin, end).
00087 // Bool() - Yields sequence {false, true}.
00088 // Combine(g1, g2, ..., gN) - Yields all combinations (the Cartesian product
00089 // for the math savvy) of the values generated
00090 // by the N generators.
00091 //
00092 // For more details, see comments at the definitions of these functions below
00093 // in this file.
00094 //
00095 // The following statement will instantiate tests from the FooTest test suite
00096 // each with parameter values "meeny", "miny", and "moe".
00097
00098 INSTANTIATE_TEST_SUITE_P(InstantiationName,
00099 FooTest,
00100 Values("meeny", "miny", "moe"));
00101
00102 // To distinguish different instances of the pattern, (yes, you
00103 // can instantiate it more than once) the first argument to the
00104 // INSTANTIATE_TEST_SUITE_P macro is a prefix that will be added to the
00105 // actual test suite name. Remember to pick unique prefixes for different
00106 // instantiations. The tests from the instantiation above will have
00107 // these names:
00108 //
00109 // * InstantiationName/FooTest.DoesBlah/0 for "meeny"
00110 // * InstantiationName/FooTest.DoesBlah/1 for "miny"
00111 // * InstantiationName/FooTest.DoesBlah/2 for "moe"
00112 // * InstantiationName/FooTest.HasBlahBlah/0 for "meeny"
00113 // * InstantiationName/FooTest.HasBlahBlah/1 for "miny"
00114 // * InstantiationName/FooTest.HasBlahBlah/2 for "moe"
00115 //
00116 // You can use these names in --gtest_filter.
00117 //
00118 // This statement will instantiate all tests from FooTest again, each
00119 // with parameter values "cat" and "dog":
00120
00121 const char* pets[] = {"cat", "dog"};
00122 INSTANTIATE_TEST_SUITE_P(AnotherInstantiationName, FooTest, ValuesIn(pets));
00123
00124 // The tests from the instantiation above will have these names:
00125 //
00126 // * AnotherInstantiationName/FooTest.DoesBlah/0 for "cat"
00127 // * AnotherInstantiationName/FooTest.DoesBlah/1 for "dog"
00128 // * AnotherInstantiationName/FooTest.HasBlahBlah/0 for "cat"
00129 // * AnotherInstantiationName/FooTest.HasBlahBlah/1 for "dog"
00130 //
00131 // Please note that INSTANTIATE_TEST_SUITE_P will instantiate all tests
00132 // in the given test suite, whether their definitions come before or
00133 // AFTER the INSTANTIATE_TEST_SUITE_P statement.
00134 //
00135 // Please also note that generator expressions (including parameters to the
00136 // generators) are evaluated in InitGoogleTest(), after main() has started.
00137 // This allows the user on one hand, to adjust generator parameters in order
00138 // to dynamically determine a set of tests to run and on the other hand,
00139 // give the user a chance to inspect the generated tests with Google Test
00140 // reflection API before RUN_ALL_TESTS() is executed.
00141 //
00142 // You can see samples/sample7_unittest.cc and samples/sample8_unittest.cc
00143 // for more examples.
00144 //
00145 // In the future, we plan to publish the API for defining new parameter
00146 // generators. But for now this interface remains part of the internal
00147 // implementation and is subject to change.
00148 //
00149 //
00150 // A parameterized test fixture must be derived from testing::Test and from
00151 // testing::WithParamInterface<T>, where T is the type of the parameter
00152 // values. Inheriting from TestWithParam<T> satisfies that requirement because
00153 // TestWithParam<T> inherits from both Test and WithParamInterface. In more
00154 // complicated hierarchies, however, it is occasionally useful to inherit
00155 // separately from Test and WithParamInterface. For example:
00156
00157 class BaseTest : public ::testing::Test {
00158 // You can inherit all the usual members for a non-parameterized test
00159 // fixture here.
00160 };
00161

```

```

00162 class DerivedTest : public BaseTest, public ::testing::WithParamInterface<int> {
00163 // The usual test fixture members go here too.
00164 };
00165
00166 TEST_F(BaseTest, HasFoo) {
00167 // This is an ordinary non-parameterized test.
00168 }
00169
00170 TEST_P(DerivedTest, DoesBlah) {
00171 // GetParam works just the same here as if you inherit from TestWithParam.
00172 EXPECT_TRUE(foo.Blah(GetParam()));
00173 }
00174
00175 #endif // 0
00176
00177 #include <iterator>
00178 #include <utility>
00179
00180 #include "gtest/internal/gtest-internal.h"
00181 #include "gtest/internal/gtest-param-util.h"
00182 #include "gtest/internal/gtest-port.h"
00183
00184 namespace testing {
00185
00186 // Functions producing parameter generators.
00187 //
00188 // Google Test uses these generators to produce parameters for value-
00189 // parameterized tests. When a parameterized test suite is instantiated
00190 // with a particular generator, Google Test creates and runs tests
00191 // for each element in the sequence produced by the generator.
00192 //
00193 // In the following sample, tests from test suite FooTest are instantiated
00194 // each three times with parameter values 3, 5, and 8:
00195 //
00196 // class FooTest : public TestWithParam<int> { ... };
00197 //
00198 // TEST_P(FooTest, TestThis) {
00199 // }
00200 // TEST_P(FooTest, TestThat) {
00201 // }
00202 // INSTANTIATE_TEST_SUITE_P(TestSequence, FooTest, Values(3, 5, 8));
00203 //
00204 // Range() returns generators providing sequences of values in a range.
00205 //
00206 // Synopsis:
00207 // Range(start, end)
00208 // - returns a generator producing a sequence of values {start, start+1,
00209 // start+2, ..., }.
00210 // Range(start, end, step)
00211 // - returns a generator producing a sequence of values {start, start+step,
00212 // start+step+step, ..., }.
00213 // Notes:
00214 // * The generated sequences never include end. For example, Range(1, 5)
00215 // returns a generator producing a sequence {1, 2, 3, 4}. Range(1, 9, 2)
00216 // returns a generator producing {1, 3, 5, 7}.
00217 // * start and end must have the same type. That type may be any integral or
00218 // floating-point type or a user defined type satisfying these conditions:
00219 // * It must be assignable (have operator+=() defined).
00220 // * It must have operator+() (operator+(int-compatible type) for
00221 // two-operand version).
00222 // * It must have operator<() defined.
00223 // Elements in the resulting sequences will also have that type.
00224 // * Condition start < end must be satisfied in order for resulting sequences
00225 // to contain any elements.
00226 //
00227 //
00228 template <typename T, typename IncrementT>
00229 internal::ParamGenerator<T> Range(T start, T end, IncrementT step) {
00230 return internal::ParamGenerator<T>{
00231 new internal::RangeGenerator<T, IncrementT>(start, end, step)};
00232 }
00233
00234 template <typename T>
00235 internal::ParamGenerator<T> Range(T start, T end) {
00236 return Range(start, end, 1);
00237 }
00238
00239 // ValuesIn() function allows generation of tests with parameters coming from
00240 // a container.
00241 //
00242 // Synopsis:
00243 // ValuesIn(const T (&array)[N])
00244 // - returns a generator producing sequences with elements from
00245 // a C-style array.
00246 // ValuesIn(const Container& container)
00247 // - returns a generator producing sequences with elements from
00248 // an STL-style container.

```

```

00249 // ValuesIn(Iterator begin, Iterator end)
00250 // - returns a generator producing sequences with elements from
00251 // a range [begin, end) defined by a pair of STL-style iterators. These
00252 // iterators can also be plain C pointers.
00253 //
00254 // Please note that ValuesIn copies the values from the containers
00255 // passed in and keeps them to generate tests in RUN_ALL_TESTS().
00256 //
00257 // Examples:
00258 //
00259 // This instantiates tests from test suite StringTest
00260 // each with C-string values of "foo", "bar", and "baz":
00261 //
00262 // const char* strings[] = {"foo", "bar", "baz"};
00263 // INSTANTIATE_TEST_SUITE_P(StringSequence, StringTest, ValuesIn(strings));
00264 //
00265 // This instantiates tests from test suite StlStringTest
00266 // each with STL strings with values "a" and "b":
00267 //
00268 // ::std::vector< ::std::string> GetParameterStrings() {
00269 // ::std::vector< ::std::string> v;
00270 // v.push_back("a");
00271 // v.push_back("b");
00272 // return v;
00273 // }
00274 //
00275 // INSTANTIATE_TEST_SUITE_P(CharSequence,
00276 // StlStringTest,
00277 // ValuesIn(GetParameterStrings()));
00278 //
00279 //
00280 // This will also instantiate tests from CharTest
00281 // each with parameter values 'a' and 'b':
00282 //
00283 // ::std::list<char> GetParameterChars() {
00284 // ::std::list<char> list;
00285 // list.push_back('a');
00286 // list.push_back('b');
00287 // return list;
00288 // }
00289 // ::std::list<char> l = GetParameterChars();
00290 // INSTANTIATE_TEST_SUITE_P(CharSequence2,
00291 // CharTest,
00292 // ValuesIn(l.begin(), l.end()));
00293 //
00294 template <typename ForwardIterator>
00295 internal::ParamGenerator<
00296 typename std::iterator_traits<ForwardIterator>::value_type>
00297 ValuesIn(ForwardIterator begin, ForwardIterator end) {
00298 typedef typename std::iterator_traits<ForwardIterator>::value_type ParamType;
00299 return internal::ParamGenerator<ParamType>(
00300 new internal::ValuesInIteratorRangeGenerator<ParamType>(begin, end));
00301 }
00302
00303 template <typename T, size_t N>
00304 internal::ParamGenerator<T> ValuesIn(const T (&array)[N]) {
00305 return ValuesIn(array, array + N);
00306 }
00307
00308 template <class Container>
00309 internal::ParamGenerator<typename Container::value_type> ValuesIn(
00310 const Container& container) {
00311 return ValuesIn(container.begin(), container.end());
00312 }
00313
00314 // Values() allows generating tests from explicitly specified list of
00315 // parameters.
00316 //
00317 // Synopsis:
00318 // Values(T v1, T v2, ..., T vN)
00319 // - returns a generator producing sequences with elements v1, v2, ..., vN.
00320 //
00321 // For example, this instantiates tests from test suite BarTest each
00322 // with values "one", "two", and "three":
00323 //
00324 // INSTANTIATE_TEST_SUITE_P(NumSequence,
00325 // BarTest,
00326 // Values("one", "two", "three"));
00327 //
00328 // This instantiates tests from test suite BazTest each with values 1, 2, 3.5.
00329 // The exact type of values will depend on the type of parameter in BazTest.
00330 //
00331 // INSTANTIATE_TEST_SUITE_P(FloatingNumbers, BazTest, Values(1, 2, 3.5));
00332 //
00333 //
00334 template <typename... T>
00335 internal::ValueArray<T...> Values(T... v) {

```

```

00336 return internal::ValueArray<T...>(std::move(v)...);
00337 }
00338
00339 // Bool() allows generating tests with parameters in a set of {false, true}.
00340 //
00341 // Synopsis:
00342 // Bool()
00343 // - returns a generator producing sequences with elements {false, true}.
00344 //
00345 // It is useful when testing code that depends on Boolean flags. Combinations
00346 // of multiple flags can be tested when several Bool()'s are combined using
00347 // Combine() function.
00348 //
00349 // In the following example all tests in the test suite FlagDependentTest
00350 // will be instantiated twice with parameters false and true.
00351 //
00352 // class FlagDependentTest : public testing::TestWithParam<bool> {
00353 // virtual void SetUp() {
00354 // external_flag = GetParam();
00355 // }
00356 // }
00357 // INSTANTIATE_TEST_SUITE_P(BoolSequence, FlagDependentTest, Bool());
00358 //
00359 inline internal::ParamGenerator<bool> Bool() { return Values(false, true); }
00360
00361 // Combine() allows the user to combine two or more sequences to produce
00362 // values of a Cartesian product of those sequences' elements.
00363 //
00364 // Synopsis:
00365 // Combine(gen1, gen2, ..., genN)
00366 // - returns a generator producing sequences with elements coming from
00367 // the Cartesian product of elements from the sequences generated by
00368 // gen1, gen2, ..., genN. The sequence elements will have a type of
00369 // std::tuple<T1, T2, ..., TN> where T1, T2, ..., TN are the types
00370 // of elements from sequences produces by gen1, gen2, ..., genN.
00371 //
00372 // Example:
00373 //
00374 // This will instantiate tests in test suite AnimalTest each one with
00375 // the parameter values tuple("cat", BLACK), tuple("cat", WHITE),
00376 // tuple("dog", BLACK), and tuple("dog", WHITE):
00377 //
00378 // enum Color { BLACK, GRAY, WHITE };
00379 // class AnimalTest
00380 // : public testing::TestWithParam<std::tuple<const char*, Color> > {...};
00381 //
00382 // TEST_P(AnimalTest, AnimalLooksNice) {...}
00383 //
00384 // INSTANTIATE_TEST_SUITE_P(AnimalVariations, AnimalTest,
00385 // Combine(Values("cat", "dog"),
00386 // Values(BLACK, WHITE)));
00387 //
00388 // This will instantiate tests in FlagDependentTest with all variations of two
00389 // Boolean flags:
00390 //
00391 // class FlagDependentTest
00392 // : public testing::TestWithParam<std::tuple<bool, bool> > {
00393 // virtual void SetUp() {
00394 // // Assigns external_flag_1 and external_flag_2 values from the tuple.
00395 // std::tie(external_flag_1, external_flag_2) = GetParam();
00396 // }
00397 // };
00398 //
00399 // TEST_P(FlagDependentTest, TestFeature1) {
00400 // // Test your code using external_flag_1 and external_flag_2 here.
00401 // }
00402 // INSTANTIATE_TEST_SUITE_P(TwoBoolSequence, FlagDependentTest,
00403 // Combine(Bool(), Bool()));
00404 //
00405 template <typename... Generator>
00406 internal::CartesianProductHolder<Generator...> Combine(const Generator&... g) {
00407 return internal::CartesianProductHolder<Generator...>(g...);
00408 }
00409
00410 #define TEST_P(test_suite_name, test_name)
00411 class GTEST_TEST_CLASS_NAME_(test_suite_name, test_name)
00412 : public test_suite_name {
00413 public:
00414 GTEST_TEST_CLASS_NAME_(test_suite_name, test_name)() {}
00415 void TestBody() override;
00416 private:
00417 static int AddToRegistry() {
00418 ::testing::UnitTest::GetInstance()
00419 ->parameterized_test_registry()
00420 .GetTestSuitePatternHolder<test_suite_name>()
00421 GTEST_STRINGIFY_(test_suite_name),

```

```

00423 ::testing::internal::CodeLocation(__FILE__, __LINE__))
00424 ->AddTestPattern(
00425 GTEST_STRINGIFY_(test_suite_name), GTEST_STRINGIFY_(test_name),
00426 new ::testing::internal::TestMetaFactory<GTEST_TEST_CLASS_NAME_(
00427 test_suite_name, test_name)>()),
00428 ::testing::internal::CodeLocation(__FILE__, __LINE__));
00429 return 0;
00430 }
00431 static int gtest_registering_dummy_ GTEST_ATTRIBUTE_UNUSED_;
00432 GTEST_TEST_CLASS_NAME_(test_suite_name, test_name)
00433 (const GTEST_TEST_CLASS_NAME_(test_suite_name, test_name) &) = delete;
00434 GTEST_TEST_CLASS_NAME_(test_suite_name, test_name) & operator=(
00435 const GTEST_TEST_CLASS_NAME_(test_suite_name,
00436 test_name) &) = delete; /* NOLINT */
00437 };
00438 int GTEST_TEST_CLASS_NAME_(test_suite_name,
00439 test_name)::gtest_registering_dummy_ =
00440 GTEST_TEST_CLASS_NAME_(test_suite_name, test_name)::AddToRegistry();
00441 void GTEST_TEST_CLASS_NAME_(test_suite_name, test_name)::TestBody()
00442 {
00443 // The last argument to INSTITUTE_TEST_SUITE_P allows the user to specify
00444 // generator and an optional function or functor that generates custom test name
00445 // suffixes based on the test parameters. Such a function or functor should
00446 // accept one argument of type testing::TestParamInfo<class ParamType>, and
00447 // return std::string.
00448 //
00449 // testing::PrintToStringParamName is a builtin test suffix generator that
00450 // returns the value of testing::PrintToString(GetParam()).
00451 //
00452 // Note: test names must be non-empty, unique, and may only contain ASCII
00453 // alphanumeric characters or underscore. Because PrintToString adds quotes
00454 // to std::string and C strings, it won't work for these types.
00455 #define GTEST_EXPAND_(arg) arg
00456 #define GTEST_GET_FIRST_(first, ...) first
00457 #define GTEST_GET_SECOND_(first, second, ...) second
00458 #define INSTITUTE_TEST_SUITE_P(prefix, test_suite_name, ...)
00459 static ::testing::internal::ParamGenerator<test_suite_name::ParamType>
00460 gtest_##prefix##test_suite_name##_EvalGenerator_() {
00461 return GTEST_EXPAND_(GTEST_GET_FIRST_(__VA_ARGS__, DUMMY_PARAM_));
00462 }
00463 static ::std::string gtest_##prefix##test_suite_name##_EvalGenerateName_(
00464 const ::testing::TestParamInfo<test_suite_name::ParamType>& info) {
00465 if (::testing::internal::AlwaysFalse()) {
00466 ::testing::internal::TestNotEmpty(GTEST_EXPAND_(GTEST_GET_SECOND_(
00467 __VA_ARGS__,
00468 ::testing::internal::DefaultParamName<test_suite_name::ParamType>,
00469 DUMMY_PARAM_)));
00470 auto t = std::make_tuple(__VA_ARGS__);
00471 static_assert(std::tuple_size<decltype(t)>::value <= 2,
00472 "Too Many Args!");
00473 }
00474 return ((GTEST_EXPAND_(GTEST_GET_SECOND_(
00475 __VA_ARGS__,
00476 ::testing::internal::DefaultParamName<test_suite_name::ParamType>,
00477 DUMMY_PARAM_))))(info);
00478 }
00479 static int gtest_##prefix##test_suite_name##_dummy_
00480 GTEST_ATTRIBUTE_UNUSED_ =
00481 ::testing::UnitTest::GetInstance()
00482 ->parameterized_test_registry()
00483 .GetTestSuitePatternHolder<test_suite_name>(
00484 GTEST_STRINGIFY_(test_suite_name),
00485 ::testing::internal::CodeLocation(__FILE__, __LINE__))
00486 ->AddTestSuiteInstantiation(
00487 GTEST_STRINGIFY_(prefix),
00488 >est_##prefix##test_suite_name##_EvalGenerator_,
00489 >est_##prefix##test_suite_name##_EvalGenerateName_,
00490 __FILE__, __LINE__)
00491 // Allow Marking a Parameterized test class as not needing to be instantiated.
00492 #define GTEST_ALLOW_UNINSTANTIATED_PARAMETERIZED_TEST(T)
00493 namespace gtest_do_not_use_outside_namespace_scope {}
00494 static const ::testing::internal::MarkAsIgnored gtest_allow_ignore_##T(
00495 GTEST_STRINGIFY_(T))
00496 // Legacy API is deprecated but still available
00497 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00498 #define INSTITUTE_TEST_SUITE_P
00499 static_assert(::testing::internal::InstantiateTestCase_P_IsDeprecated(),
00500 "");
00501 INSTITUTE_TEST_SUITE_P
00502 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00503 } // namespace testing
00504

```

```
00510 #endif // GOOGLETEST_INCLUDE_GTEST_GTEST_PARAM_TEST_H_
```

## 16.236 gtest-printers.h

```
00001 // Copyright 2007, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // Google Test - The Google C++ Testing and Mocking Framework
00031 //
00032 // This file implements a universal value printer that can print a
00033 // value of any type T:
00034 //
00035 // void ::testing::internal::UniversalPrinter<T>::Print(value, ostream_ptr);
00036 //
00037 // A user can teach this function how to print a class type T by
00038 // defining either operator<<() or PrintTo() in the namespace that
00039 // defines T. More specifically, the FIRST defined function in the
00040 // following list will be used (assuming T is defined in namespace
00041 // foo):
00042 //
00043 // 1. foo::PrintTo(const T&, ostream*)
00044 // 2. operator<<(ostream&, const T&) defined in either foo or the
00045 // global namespace.
00046 //
00047 // However if T is an STL-style container then it is printed element-wise
00048 // unless foo::PrintTo(const T&, ostream*) is defined. Note that
00049 // operator<<() is ignored for container types.
00050 //
00051 // If none of the above is defined, it will print the debug string of
00052 // the value if it is a protocol buffer, or print the raw bytes in the
00053 // value otherwise.
00054 //
00055 // To aid debugging: when T is a reference type, the address of the
00056 // value is also printed; when T is a (const) char pointer, both the
00057 // pointer value and the NUL-terminated string it points to are
00058 // printed.
00059 //
00060 // We also provide some convenient wrappers:
00061 //
00062 // // Prints a value to a string. For a (const or not) char
00063 // // pointer, the NUL-terminated string (but not the pointer) is
00064 // // printed.
00065 // std::string ::testing::PrintToString(const T& value);
00066 //
00067 // // Prints a value tersely: for a reference type, the referenced
00068 // // value (but not the address) is printed; for a (const or not) char
00069 // // pointer, the NUL-terminated string (but not the pointer) is
00070 // // printed.
00071 // void ::testing::internal::UniversalTersePrint(const T& value, ostream*);
00072 //
00073 // // Prints value using the type inferred by the compiler. The difference
00074 // // from UniversalTersePrint() is that this function prints both the
00075 // // pointer and the NUL-terminated string for a (const or not) char pointer.
00076 // void ::testing::internal::UniversalPrint(const T& value, ostream*);
00077 //
00078 // // Prints the fields of a tuple tersely to a string vector, one
00079 // // element for each field. Tuple support must be enabled in
00080 // // gtest-port.h.
00081 // std::vector<string> UniversalTersePrintTupleFieldsToStrings(
```

```

00082 // const Tuple& value);
00083 //
00084 // Known limitation:
00085 //
00086 // The print primitives print the elements of an STL-style container
00087 // using the compiler-inferred type of *iter where iter is a
00088 // const_iterator of the container. When const_iterator is an input
00089 // iterator but not a forward iterator, this inferred type may not
00090 // match value_type, and the print output may be incorrect. In
00091 // practice, this is rarely a problem as for most containers
00092 // const_iterator is a forward iterator. We'll fix this if there's an
00093 // actual need for it. Note that this fix cannot rely on value_type
00094 // being defined as many user-defined container types don't have
00095 // value_type.
00096
00097 // IWYU pragma: private, include "gtest/gtest.h"
00098 // IWYU pragma: friend gtest/*.
00099 // IWYU pragma: friend gmock/*.
00100
00101 #ifndef GGOOGLETEST_INCLUDE_GTEST_GTEST_PRINTERS_H_
00102 #define GGOOGLETEST_INCLUDE_GTEST_GTEST_PRINTERS_H_
00103
00104 #include <functional>
00105 #include <memory>
00106 #include <ostream> // NOLINT
00107 #include <sstream>
00108 #include <string>
00109 #include <tuple>
00110 #include <type_traits>
00111 #include <utility>
00112 #include <vector>
00113
00114 #include "gtest/internal/gtest-internal.h"
00115 #include "gtest/internal/gtest-port.h"
00116
00117 namespace testing {
00118
00119 // Definitions in the internal* namespaces are subject to change without notice.
00120 // DO NOT USE THEM IN USER CODE!
00121 namespace internal {
00122
00123 template <typename T>
00124 void UniversalPrint(const T& value, ::std::ostream* os);
00125
00126 // Used to print an STL-style container when the user doesn't define
00127 // a PrintTo() for it.
00128
00129 struct ContainerPrinter {
00130
00131 template <typename T,
00132 typename = typename std::enable_if<
00133 (sizeof (IsContainerTest<T>(0)) == sizeof (IsContainer)) &&
00134 !IsRecursiveContainer<T>::value>::type>
00135 static void PrintValue(const T& container, std::ostream* os) {
00136 const size_t kMaxCount = 32; // The maximum number of elements to print.
00137 *os << '{';
00138 size_t count = 0;
00139 for (auto&& elem : container) {
00140 if (count > 0) {
00141 *os << ',';
00142 if (count == kMaxCount) { // Enough has been printed.
00143 *os << " ...";
00144 break;
00145 }
00146 }
00147 *os << ' ';
00148 // We cannot call PrintTo(elem, os) here as PrintTo() doesn't
00149 // handle `elem` being a native array.
00150 internal::UniversalPrint(elem, os);
00151 ++count;
00152 }
00153
00154 if (count > 0) {
00155 *os << ' ';
00156 }
00157 *os << '}';
00158 }
00159 };
00160
00161 // Used to print a pointer that is neither a char pointer nor a member
00162 // pointer, when the user doesn't define PrintTo() for it. (A member
00163 // variable pointer or member function pointer doesn't really point to
00164 // a location in the address space. Their representation is
00165 // implementation-defined. Therefore they will be printed as raw
00166 // bytes.)
00167
00168 struct FunctionPointerPrinter {

```



```

00169
00170 template <typename T, typename = typename std::enable_if<
00171 std::is_function<T>::value>::type>
00172 static void PrintValue(T* p, ::std::ostream* os) {
00173 if (p == nullptr) {
00174 *os << "NULL";
00175 } else {
00176 // T is a function type, so '*os << p' doesn't do what we want
00177 // (it just prints p as bool). We want to print p as a const
00178 // void*.
00179 *os << reinterpret_cast<const void*> (p);
00180 }
00181 }
00182 };
00183
00184 struct PointerPrinter {
00185
00186 template <typename T>
00187 static void PrintValue(T* p, ::std::ostream* os) {
00188 if (p == nullptr) {
00189 *os << "NULL";
00190 } else {
00191 // T is not a function type. We just call << to print p,
00192 // relying on ADL to pick up user-defined << for their pointer
00193 // types, if any.
00194 *os << p;
00195 }
00196 }
00197 };
00198
00199 namespace internal_stream_operator_without_lexical_name_lookup {
00200
00201 // The presence of an operator<< here will terminate lexical scope lookup
00202 // straight away (even though it cannot be a match because of its argument
00203 // types). Thus, the two operator<< calls in StreamPrinter will find only ADL
00204 // candidates.
00205
00206 struct LookupBlocker {
00207 };
00208 void operator<<(LookupBlocker, LookupBlocker);
00209
00210 struct StreamPrinter {
00211
00212 template <typename T,
00213 // Don't accept member pointers here. We'd print them via implicit
00214 // conversion to bool, which isn't useful.
00215 typename = typename std::enable_if<
00216 !std::is_member_pointer<T>::value>::type,
00217 // Only accept types for which we can find a streaming operator via
00218 // ADL (possibly involving implicit conversions).
00219 typename = decltype(std::declval<std::ostream&>()
00220 << std::declval<const T&>())>
00221 static void PrintValue(const T& value, ::std::ostream* os) {
00222 // Call streaming operator found by ADL, possibly with implicit conversions
00223 // of the arguments.
00224 *os << value;
00225 }
00226 };
00227
00228 } // namespace internal_stream_operator_without_lexical_name_lookup
00229
00230 struct ProtobufPrinter {
00231 // We print a protobuf using its ShortDebugString() when the string
00232 // doesn't exceed this many characters; otherwise we print it using
00233 // DebugString() for better readability.
00234 static const size_t kProtobufOneLinerMaxLength = 50;
00235
00236 template <typename T,
00237 typename = typename std::enable_if<
00238 internal::HasDebugStringAndShortDebugString<T>::value>::type>
00239 static void PrintValue(const T& value, ::std::ostream* os) {
00240 std::string pretty_str = value.ShortDebugString();
00241 if (pretty_str.length() > kProtobufOneLinerMaxLength) {
00242 pretty_str = "\n" + value.DebugString();
00243 }
00244 *os << ("<" + pretty_str + ">");
00245 }
00246 };
00247
00248 struct ConvertibleToIntegerPrinter {
00249 // Since T has no << operator or PrintTo() but can be implicitly
00250 // converted to BiggestInt, we print it as a BiggestInt.
00251 //
00252 // Most likely T is an enum type (either named or unnamed), in which
00253 // case printing it as an integer is the desired behavior. In case
00254 // T is not an enum, printing it as an integer is the best we can do
00255 // given that it has no user-defined printer.

```

```

00256 static void PrintValue(internal::BiggestInt value, ::std::ostream* os) {
00257 *os « value;
00258 }
00259 };
00260 };
00261
00262 struct ConvertibleToStringViewPrinter {
00263 #if GTEST_INTERNAL_HAS_STRING_VIEW
00264 static void PrintValue(internal::StringView value, ::std::ostream* os) {
00265 internal::UniversalPrint(value, os);
00266 }
00267 #endif
00268 };
00269 };
00270
00271 // Prints the given number of bytes in the given object to the given
00272 // ostream.
00273 GTEST_API_ void PrintBytesInObjectTo(const unsigned char* obj_bytes,
00274 size_t count, ::std::ostream* os);
00275
00276 struct RawBytesPrinter {
00277 // SFINAE on `sizeof` to make sure we have a complete type.
00278
00279 template <typename T, size_t = sizeof (T)>
00280 static void PrintValue(const T& value, ::std::ostream* os) {
00281 PrintBytesInObjectTo(
00282 static_cast<const unsigned char*> (
00283 // Load bearing cast to void* to support iOS
00284 reinterpret_cast<const void*> (std::addressof(value))),
00285 sizeof (value), os);
00286 }
00287 };
00288
00289 struct FallbackPrinter {
00290
00291 template <typename T>
00292 static void PrintValue(const T&, ::std::ostream* os) {
00293 *os « "(incomplete type)";
00294 }
00295 };
00296
00297 // Try every printer in order and return the first one that works.
00298
00299 template <typename T, typename E, typename Printer, typename... Printers>
00300 struct FindFirstPrinter : FindFirstPrinter<T, E, Printers...> {
00301 };
00302
00303 template <typename T, typename Printer, typename... Printers>
00304 struct FindFirstPrinter<
00305 T, decltype(Printer::PrintValue(std::declval<const T&>(), nullptr)),
00306 Printer, Printers...> {
00307 using type = Printer;
00308 };
00309
00310 // Select the best printer in the following order:
00311 // - Print containers (they have begin/end/etc).
00312 // - Print function pointers.
00313 // - Print object pointers.
00314 // - Use the stream operator, if available.
00315 // - Print protocol buffers.
00316 // - Print types convertible to BiggestInt.
00317 // - Print types convertible to StringView, if available.
00318 // - Fallback to printing the raw bytes of the object.
00319
00320 template <typename T>
00321 void PrintWithFallback(const T& value, ::std::ostream* os) {
00322 using Printer = typename FindFirstPrinter<
00323 T, void, ContainerPrinter, FunctionPointerPrinter, PointerPrinter,
00324 internal_stream_operator_without_lexical_name_lookup::StreamPrinter,
00325 ProtobufPrinter, ConvertibleToIntegerPrinter,
00326 ConvertibleToStringViewPrinter, RawBytesPrinter, FallbackPrinter>::type;
00327 Printer::PrintValue(value, os);
00328 }
00329
00330 // FormatForComparison<ToPrint, OtherOperand>::Format(value) formats a
00331 // value of type ToPrint that is an operand of a comparison assertion
00332 // (e.g. ASSERT_EQ). OtherOperand is the type of the other operand in
00333 // the comparison, and is used to help determine the best way to
00334 // format the value. In particular, when the value is a C string
00335 // (char pointer) and the other operand is an STL string object, we
00336 // want to format the C string as a string, since we know it is
00337 // compared by value with the string object. If the value is a char
00338 // pointer but the other operand is not an STL string object, we don't
00339 // know whether the pointer is supposed to point to a NUL-terminated
00340 // string, and thus want to print it as a pointer to be safe.
00341 //
00342 // INTERNAL IMPLEMENTATION - DO NOT USE IN A USER PROGRAM.

```

```

00343
00344 // The default case.
00345
00346 template <typename ToPrint, typename OtherOperand>
00347 class FormatForComparison {
00348 public:
00349
00350 static ::std::string Format(const ToPrint& value) {
00351 return ::testing::PrintToString(value);
00352 }
00353 };
00354
00355 // Array.
00356
00357 template <typename ToPrint, size_t N, typename OtherOperand>
00358 class FormatForComparison<ToPrint[N], OtherOperand> {
00359 public:
00360
00361 static ::std::string Format(const ToPrint* value) {
00362 return FormatForComparison<const ToPrint*, OtherOperand>::Format(value);
00363 }
00364 };
00365
00366 // By default, print C string as pointers to be safe, as we don't know
00367 // whether they actually point to a NUL-terminated string.
00368
00369 #define GTEST_IMPL_FORMAT_C_STRING_AS_POINTER(CharType) \
00370 template <typename OtherOperand> \
00371 class FormatForComparison<CharType*, OtherOperand> { \
00372 public: \
00373 static ::std::string Format(CharType* value) { \
00374 return ::testing::PrintToString(static_cast<const void*>(value)); \
00375 } \
00376 } \
00377
00378 GTEST_IMPL_FORMAT_C_STRING_AS_POINTER(char);
00379 GTEST_IMPL_FORMAT_C_STRING_AS_POINTER(const char);
00380 GTEST_IMPL_FORMAT_C_STRING_AS_POINTER(wchar_t);
00381 GTEST_IMPL_FORMAT_C_STRING_AS_POINTER(const wchar_t);
00382 #ifdef __cpp_lib_char8_t
00383 GTEST_IMPL_FORMAT_C_STRING_AS_POINTER(char8_t);
00384 GTEST_IMPL_FORMAT_C_STRING_AS_POINTER(const char8_t);
00385 #endif
00386 GTEST_IMPL_FORMAT_C_STRING_AS_POINTER(char16_t);
00387 GTEST_IMPL_FORMAT_C_STRING_AS_POINTER(const char16_t);
00388 GTEST_IMPL_FORMAT_C_STRING_AS_POINTER(char32_t);
00389 GTEST_IMPL_FORMAT_C_STRING_AS_POINTER(const char32_t);
00390
00391 #undef GTEST_IMPL_FORMAT_C_STRING_AS_POINTER_
00392
00393 // If a C string is compared with an STL string object, we know it's meant
00394 // to point to a NUL-terminated string, and thus can print it as a string.
00395
00396 #define GTEST_IMPL_FORMAT_C_STRING_AS_STRING(CharType, OtherStringType) \
00397 template <> \
00398 class FormatForComparison<CharType*, OtherStringType> { \
00399 public: \
00400 static ::std::string Format(CharType* value) { \
00401 return ::testing::PrintToString(value); \
00402 } \
00403 } \
00404
00405 GTEST_IMPL_FORMAT_C_STRING_AS_STRING(char, ::std::string);
00406 GTEST_IMPL_FORMAT_C_STRING_AS_STRING(const char, ::std::string);
00407 #ifdef __cpp_lib_char8_t
00408 GTEST_IMPL_FORMAT_C_STRING_AS_STRING(char8_t, ::std::u8string);
00409 GTEST_IMPL_FORMAT_C_STRING_AS_STRING(const char8_t, ::std::u8string);
00410 #endif
00411 GTEST_IMPL_FORMAT_C_STRING_AS_STRING(char16_t, ::std::u16string);
00412 GTEST_IMPL_FORMAT_C_STRING_AS_STRING(const char16_t, ::std::u16string);
00413 GTEST_IMPL_FORMAT_C_STRING_AS_STRING(char32_t, ::std::u32string);
00414 GTEST_IMPL_FORMAT_C_STRING_AS_STRING(const char32_t, ::std::u32string);
00415
00416 #if GTEST_HAS_STD_WSTRING
00417 GTEST_IMPL_FORMAT_C_STRING_AS_STRING(wchar_t, ::std::wstring);
00418 GTEST_IMPL_FORMAT_C_STRING_AS_STRING(const wchar_t, ::std::wstring);
00419 #endif
00420
00421 #undef GTEST_IMPL_FORMAT_C_STRING_AS_STRING_
00422
00423 // Formats a comparison assertion (e.g. ASSERT_EQ, EXPECT_LT, and etc)
00424 // operand to be used in a failure message. The type (but not value)
00425 // of the other operand may affect the format. This allows us to
00426 // print a char* as a raw pointer when it is compared against another
00427 // char* or void*, and print it as a C string when it is compared
00428 // against an std::string object, for example.
00429 //

```

```

00430 // INTERNAL IMPLEMENTATION - DO NOT USE IN A USER PROGRAM.
00431
00432 template <typename T1, typename T2>
00433 std::string FormatForComparisonFailureMessage(const T1& value,
00434 const T2& /* other_operand */) {
00435 return FormatForComparison<T1, T2>::Format(value);
00436 }
00437
00438 // UniversalPrinter<T>::Print(value, ostream_ptr) prints the given
00439 // value to the given ostream. The caller must ensure that
00440 // 'ostream_ptr' is not NULL, or the behavior is undefined.
00441 //
00442 // We define UniversalPrinter as a class template (as opposed to a
00443 // function template), as we need to partially specialize it for
00444 // reference types, which cannot be done with function templates.
00445 template <typename T>
00446 class UniversalPrinter;
00447
00448 // Prints the given value using the « operator if it has one;
00449 // otherwise prints the bytes in it. This is what
00450 // UniversalPrinter<T>::Print() does when PrintTo() is not specialized
00451 // or overloaded for type T.
00452 //
00453 // A user can override this behavior for a class type Foo by defining
00454 // an overload of PrintTo() in the namespace where Foo is defined. We
00455 // give the user this option as sometimes defining a « operator for
00456 // Foo is not desirable (e.g. the coding style may prevent doing it,
00457 // or there is already a « operator but it doesn't do what the user
00458 // wants).
00459
00460 template <typename T>
00461 void PrintTo(const T& value, ::std::ostream* os) {
00462 internal::PrintWithFallback(value, os);
00463 }
00464
00465 // The following list of PrintTo() overloads tells
00466 // UniversalPrinter<T>::Print() how to print standard types (built-in
00467 // types, strings, plain arrays, and pointers).
00468
00469 // Overloads for various char types.
00470 GTEST_API_ void PrintTo(unsigned char c, ::std::ostream* os);
00471 GTEST_API_ void PrintTo(signed char c, ::std::ostream* os);
00472
00473 inline void PrintTo(char c, ::std::ostream* os) {
00474 // When printing a plain char, we always treat it as unsigned. This
00475 // way, the output won't be affected by whether the compiler thinks
00476 // char is signed or not.
00477 PrintTo(static_cast<unsigned char>(c), os);
00478 }
00479
00480 // Overloads for other simple built-in types.
00481
00482 inline void PrintTo(bool x, ::std::ostream* os) {
00483 *os << (x ? "true" : "false");
00484 }
00485
00486 // Overload for wchar_t type.
00487 // Prints a wchar_t as a symbol if it is printable or as its internal
00488 // code otherwise and also as its decimal code (except for L'\0').
00489 // The L'\0' char is printed as "L'\0'". The decimal code is printed
00490 // as signed integer when wchar_t is implemented by the compiler
00491 // as a signed type and is printed as an unsigned integer when wchar_t
00492 // is implemented as an unsigned type.
00493 GTEST_API_ void PrintTo(wchar_t wc, ::std::ostream* os);
00494
00495 GTEST_API_ void PrintTo(char32_t c, ::std::ostream* os);
00496
00497 inline void PrintTo(char16_t c, ::std::ostream* os) {
00498 PrintTo(ImplicitCast_<char32_t>(c), os);
00499 }
00500 #ifdef __cpp_char8_t
00501
00502 inline void PrintTo(char8_t c, ::std::ostream* os) {
00503 PrintTo(ImplicitCast_<char32_t>(c), os);
00504 }
00505 #endif
00506
00507 // gcc/clang __{u,}int128_t
00508 #if defined(__SIZEOF_INT128__)
00509 GTEST_API_ void PrintTo(__uint128_t v, ::std::ostream* os);
00510 GTEST_API_ void PrintTo(__int128_t v, ::std::ostream* os);
00511 #endif // __SIZEOF_INT128__
00512
00513 // Overloads for C strings.
00514 GTEST_API_ void PrintTo(const char* s, ::std::ostream* os);
00515
00516 inline void PrintTo(char* s, ::std::ostream* os) {

```

```

00517 PrintTo(ImplicitCast_<const char*>(s), os);
00518 }
00519
00520 // signed/unsigned char is often used for representing binary data, so
00521 // we print pointers to it as void* to be safe.
00522
00523 inline void PrintTo(const signed char* s, ::std::ostream* os) {
00524 PrintTo(ImplicitCast_<const void*>(s), os);
00525 }
00526
00527 inline void PrintTo(signed char* s, ::std::ostream* os) {
00528 PrintTo(ImplicitCast_<const void*>(s), os);
00529 }
00530
00531 inline void PrintTo(const unsigned char* s, ::std::ostream* os) {
00532 PrintTo(ImplicitCast_<const void*>(s), os);
00533 }
00534
00535 inline void PrintTo(unsigned char* s, ::std::ostream* os) {
00536 PrintTo(ImplicitCast_<const void*>(s), os);
00537 }
00538 #ifdef __cpp_char8_t
00539 // Overloads for u8 strings.
00540 GTEST_API_ void PrintTo(const char8_t* s, ::std::ostream* os);
00541
00542 inline void PrintTo(char8_t* s, ::std::ostream* os) {
00543 PrintTo(ImplicitCast_<const char8_t*>(s), os);
00544 }
00545 #endif
00546 // Overloads for u16 strings.
00547 GTEST_API_ void PrintTo(const char16_t* s, ::std::ostream* os);
00548
00549 inline void PrintTo(char16_t* s, ::std::ostream* os) {
00550 PrintTo(ImplicitCast_<const char16_t*>(s), os);
00551 }
00552 // Overloads for u32 strings.
00553 GTEST_API_ void PrintTo(const char32_t* s, ::std::ostream* os);
00554
00555 inline void PrintTo(char32_t* s, ::std::ostream* os) {
00556 PrintTo(ImplicitCast_<const char32_t*>(s), os);
00557 }
00558
00559 // MSVC can be configured to define wchar_t as a typedef of unsigned
00560 // short. It defines _NATIVE_WCHAR_T_DEFINED when wchar_t is a native
00561 // type. When wchar_t is a typedef, defining an overload for const
00562 // wchar_t* would cause unsigned short* be printed as a wide string,
00563 // possibly causing invalid memory accesses.
00564 #if !defined(_MSC_VER) || defined(_NATIVE_WCHAR_T_DEFINED)
00565 // Overloads for wide C strings
00566 GTEST_API_ void PrintTo(const wchar_t* s, ::std::ostream* os);
00567
00568 inline void PrintTo(wchar_t* s, ::std::ostream* os) {
00569 PrintTo(ImplicitCast_<const wchar_t*>(s), os);
00570 }
00571 #endif
00572
00573 // Overload for C arrays. Multi-dimensional arrays are printed
00574 // properly.
00575
00576 // Prints the given number of elements in an array, without printing
00577 // the curly braces.
00578
00579 template <typename T>
00580 void PrintRawArrayTo(const T a[], size_t count, ::std::ostream* os) {
00581 UniversalPrint(a[0], os);
00582 for (size_t i = 1; i != count; i++) {
00583 *os << ", ";
00584 UniversalPrint(a[i], os);
00585 }
00586 }
00587
00588 // Overloads for ::std::string.
00589 GTEST_API_ void PrintStringTo(const ::std::string& s, ::std::ostream* os);
00590
00591 inline void PrintTo(const ::std::string& s, ::std::ostream* os) {
00592 PrintStringTo(s, os);
00593 }
00594
00595 // Overloads for ::std::u8string
00596 #ifdef __cpp_lib_char8_t
00597 GTEST_API_ void PrintU8StringTo(const ::std::u8string& s, ::std::ostream* os);
00598
00599 inline void PrintTo(const ::std::u8string& s, ::std::ostream* os) {
00600 PrintU8StringTo(s, os);
00601 }
00602 #endif
00603

```

```

00604 // Overloads for ::std::ul6string
00605 GTEST_API_ void PrintU16StringTo(const ::std::ul6string& s, ::std::ostream* os);
00606
00607 inline void PrintTo(const ::std::ul6string& s, ::std::ostream* os) {
00608 PrintU16StringTo(s, os);
00609 }
00610
00611 // Overloads for ::std::u32string
00612 GTEST_API_ void PrintU32StringTo(const ::std::u32string& s, ::std::ostream* os);
00613
00614 inline void PrintTo(const ::std::u32string& s, ::std::ostream* os) {
00615 PrintU32StringTo(s, os);
00616 }
00617
00618 // Overloads for ::std::wstring.
00619 #if GTEST_HAS_STD_WSTRING
00620 GTEST_API_ void PrintWideStringTo(const ::std::wstring& s, ::std::ostream* os);
00621
00622 inline void PrintTo(const ::std::wstring& s, ::std::ostream* os) {
00623 PrintWideStringTo(s, os);
00624 }
00625 #endif // GTEST_HAS_STD_WSTRING
00626
00627 #if GTEST_INTERNAL_HAS_STRING_VIEW
00628 // Overload for internal::StringView.
00629
00630 inline void PrintTo(internal::StringView sp, ::std::ostream* os) {
00631 PrintTo(::std::string(sp), os);
00632 }
00633 #endif // GTEST_INTERNAL_HAS_STRING_VIEW
00634
00635 inline void PrintTo(std::nullptr_t, ::std::ostream* os) {
00636 *os << "(nullptr)";
00637 }
00638
00639 #if GTEST_HAS_RTTI
00640
00641 inline void PrintTo(const std::type_info& info, std::ostream* os) {
00642 *os << internal::GetTypeName(info);
00643 }
00644 #endif // GTEST_HAS_RTTI
00645
00646 template <typename T>
00647 void PrintTo(std::reference_wrapper<T> ref, ::std::ostream* os) {
00648 UniversalPrinter<T>::Print(ref.get(), os);
00649 }
00650
00651 inline const void* VoidifyPointer(const void* p) {
00652 return p;
00653 }
00654
00655 inline const void* VoidifyPointer(volatile const void* p) {
00656 return const_cast<const void*>(p);
00657 }
00658
00659 template <typename T, typename Ptr>
00660 void PrintSmartPointer(const Ptr& ptr, std::ostream* os, char) {
00661 if (ptr == nullptr) {
00662 *os << "(nullptr)";
00663 } else {
00664 // We can't print the value. Just print the pointer..
00665 *os << "(" << (VoidifyPointer)(ptr.get()) << ")";
00666 }
00667 }
00668
00669 template <typename T, typename Ptr,
00670 typename = typename std::enable_if<!std::is_void<T>::value &&
00671 !std::is_array<T>::value>::type>
00672 void PrintSmartPointer(const Ptr& ptr, std::ostream* os, int) {
00673 if (ptr == nullptr) {
00674 *os << "(nullptr)";
00675 } else {
00676 *os << "(ptr = " << (VoidifyPointer)(ptr.get()) << ", value = ";
00677 UniversalPrinter<T>::Print(*ptr, os);
00678 *os << ")";
00679 }
00680 }
00681
00682 template <typename T, typename D>
00683 void PrintTo(const std::unique_ptr<T, D>& ptr, std::ostream* os) {
00684 (PrintSmartPointer<T>)(ptr, os, 0);
00685 }
00686
00687 template <typename T>
00688 void PrintTo(const std::shared_ptr<T>& ptr, std::ostream* os) {
00689 (PrintSmartPointer<T>)(ptr, os, 0);
00690 }

```

```

00691
00692 // Helper function for printing a tuple. T must be instantiated with
00693 // a tuple type.
00694
00695 template <typename T>
00696 void PrintTupleTo(const T& t, std::integral_constant<size_t, 0>,
00697 ::std::ostream*) {
00698 }
00699
00700 template <typename T, size_t I>
00701 void PrintTupleTo(const T& t, std::integral_constant<size_t, I>,
00702 ::std::ostream* os) {
00703 PrintTupleTo(t, std::integral_constant<size_t, I - 1>(), os);
00704 GTEST_INTENTIONAL_CONST_COND_PUSH_()
00705 if (I > 1) {
00706 GTEST_INTENTIONAL_CONST_COND_POP_()
00707 *os << ", ";
00708 }
00709 UniversalPrinter<typename std::tuple_element< I - 1, T>::type>::Print(
00710 std::get< I - 1>(t), os);
00711 }
00712
00713 template <typename... Types>
00714 void PrintTo(const ::std::tuple<Types...>& t, ::std::ostream* os) {
00715 *os << "(";
00716 PrintTupleTo(t, std::integral_constant<size_t, sizeof...(Types)>(), os);
00717 *os << ")";
00718 }
00719
00720 // Overload for std::pair.
00721
00722 template <typename T1, typename T2>
00723 void PrintTo(const ::std::pair<T1, T2>& value, ::std::ostream* os) {
00724 *os << '(';
00725 // We cannot use UniversalPrint(value.first, os) here, as T1 may be
00726 // a reference type. The same for printing value.second.
00727 UniversalPrinter<T1>::Print(value.first, os);
00728 *os << ", ";
00729 UniversalPrinter<T2>::Print(value.second, os);
00730 *os << ')';
00731 }
00732
00733 // Implements printing a non-reference type T by letting the compiler
00734 // pick the right overload of PrintTo() for T.
00735
00736 template <typename T>
00737 class UniversalPrinter {
00738 public:
00739 // MSVC warns about adding const to a function type, so we want to
00740 // disable the warning.
00741
00742 GTEST_DISABLE_MSC_WARNINGS_PUSH_(4180)
00743
00744 // Note: we deliberately don't call this PrintTo(), as that name
00745 // conflicts with ::testing::internal::PrintTo in the body of the
00746 // function.
00747 static void Print(const T& value, ::std::ostream* os) {
00748 // By default, ::testing::internal::PrintTo() is used for printing
00749 // the value.
00750 //
00751 // Thanks to Koenig look-up, if T is a class and has its own
00752 // PrintTo() function defined in its namespace, that function will
00753 // be visible here. Since it is more specific than the generic ones
00754 // in ::testing::internal, it will be picked by the compiler in the
00755 // following statement - exactly what we want.
00756 PrintTo(value, os);
00757 }
00758
00759 GTEST_DISABLE_MSC_WARNINGS_POP_()
00760 };
00761
00762 // Remove any const-qualifiers before passing a type to UniversalPrinter.
00763
00764 template <typename T>
00765 class UniversalPrinter<const T> : public UniversalPrinter<T> {
00766 };
00767
00768 #if GTEST_INTERNAL_HAS_ANY
00769
00770 // Printer for std::any / absl::any
00771
00772 template <>
00773 class UniversalPrinter<Any> {
00774 public:
00775
00776 static void Print(const Any& value, ::std::ostream* os) {
00777 if (value.has_value()) {

```

```

00778 *os << "value of type " << GetTypeName(value);
00779 } else {
00780 *os << "no value";
00781 }
00782 }
00783
00784 private:
00785
00786 static std::string GetTypeName(const Any& value) {
00787 #if GTEST_HAS_RTTI
00788 return internal::GetTypeName(value.type());
00789 #else
00790 static_cast<void>(value); // possibly unused
00791 return "<unknown_type>";
00792 #endif // GTEST_HAS_RTTI
00793 }
00794 };
00795
00796 #endif // GTEST_INTERNAL_HAS_ANY
00797
00798 #if GTEST_INTERNAL_HAS_OPTIONAL
00799
00800 // Printer for std::optional / absl::optional
00801
00802 template <typename T>
00803 class UniversalPrinter<Optional<T>>
00804 {
00805 public:
00806
00807 static void Print(const Optional<T>& value, ::std::ostream * os) {
00808 *os << ' (';
00809 if (!value) {
00810 *os << "nullopt";
00811 } else {
00812 UniversalPrint(*value, os);
00813 }
00814 *os << ')';
00815 }
00816 };
00817
00818 template <>
00819 class UniversalPrinter<decltype(Nullopt())> {
00820 public:
00821
00822 static void Print(decltype(Nullopt()), ::std::ostream* os) {
00823 *os << "(nullopt)";
00824 }
00825 };
00826
00827 #endif // GTEST_INTERNAL_HAS_OPTIONAL
00828
00829 #if GTEST_INTERNAL_HAS_VARIANT
00830
00831 // Printer for std::variant / absl::variant
00832
00833 template <typename... T>
00834 class UniversalPrinter<Variant<T...>>
00835 {
00836 public:
00837
00838 static void Print(const Variant<T...>& value, ::std::ostream * os) {
00839 *os << ' (';
00840 #if GTEST_HAS_ABSL
00841 absl::visit(Visitor{os, value.index()}, value);
00842 #else
00843 std::visit(Visitor{os, value.index()}, value);
00844 #endif // GTEST_HAS_ABSL
00845 *os << ')';
00846 }
00847
00848 private:
00849
00850 struct Visitor {
00851
00852 template <typename U>
00853 void operator()(const U& u) const {
00854 *os << "' " << GetTypeName<U>() << "(index = " << index
00855 << ")' with value ";
00856 UniversalPrint(u, os);
00857 }
00858 ::std::ostream* os;
00859 std::size_t index;
00860 };
00861 };
00862
00863 #endif // GTEST_INTERNAL_HAS_VARIANT
00864

```



```

00865 // UniversalPrintArray(begin, len, os) prints an array of 'len'
00866 // elements, starting at address 'begin'.
00867
00868 template <typename T>
00869 void UniversalPrintArray(const T* begin, size_t len, ::std::ostream* os) {
00870 if (len == 0) {
00871 *os << "{}";
00872 } else {
00873 *os << "{ ";
00874 const size_t kThreshold = 18;
00875 const size_t kChunkSize = 8;
00876 // If the array has more than kThreshold elements, we'll have to
00877 // omit some details by printing only the first and the last
00878 // kChunkSize elements.
00879 if (len <= kThreshold) {
00880 PrintRawArrayTo(begin, len, os);
00881 } else {
00882 PrintRawArrayTo(begin, kChunkSize, os);
00883 *os << ", ... ";
00884 PrintRawArrayTo(begin + len - kChunkSize, kChunkSize, os);
00885 }
00886 *os << " ";
00887 }
00888 }
00889 // This overload prints a (const) char array compactly.
00890 GTEST_API_ void UniversalPrintArray(const char* begin, size_t len,
00891 ::std::ostream* os);
00892
00893 #ifdef __cpp_char8_t
00894 // This overload prints a (const) char8_t array compactly.
00895 GTEST_API_ void UniversalPrintArray(const char8_t* begin, size_t len,
00896 ::std::ostream* os);
00897 #endif
00898
00899 // This overload prints a (const) char16_t array compactly.
00900 GTEST_API_ void UniversalPrintArray(const char16_t* begin, size_t len,
00901 ::std::ostream* os);
00902
00903 // This overload prints a (const) char32_t array compactly.
00904 GTEST_API_ void UniversalPrintArray(const char32_t* begin, size_t len,
00905 ::std::ostream* os);
00906
00907 // This overload prints a (const) wchar_t array compactly.
00908 GTEST_API_ void UniversalPrintArray(const wchar_t* begin, size_t len,
00909 ::std::ostream* os);
00910
00911 // Implements printing an array type T[N].
00912
00913 template <typename T, size_t N>
00914 class UniversalPrinter<T[N]> {
00915 public:
00916 // Prints the given array, omitting some elements when there are too
00917 // many.
00918
00919 static void Print(const T(&a)[N], ::std::ostream* os) {
00920 UniversalPrintArray(a, N, os);
00921 }
00922 };
00923
00924 // Implements printing a reference type T&.
00925
00926 template <typename T>
00927 class UniversalPrinter<T&> {
00928 public:
00929 // MSVC warns about adding const to a function type, so we want to
00930 // disable the warning.
00931
00932 GTEST_DISABLE_MSC_WARNINGS_PUSH_(4180)
00933
00934 static void Print(const T& value, ::std::ostream* os) {
00935 // Prints the address of the value. We use reinterpret_cast here
00936 // as static_cast doesn't compile when T is a function type.
00937 *os << "@" << reinterpret_cast<const void*>(&value) << " ";
00938
00939 // Then prints the value itself.
00940 UniversalPrint(value, os);
00941 }
00942
00943 GTEST_DISABLE_MSC_WARNINGS_POP_()
00944 };
00945
00946 // Prints a value tersely: for a reference type, the referenced value
00947 // (but not the address) is printed; for a (const) char pointer, the
00948 // NUL-terminated string (but not the pointer) is printed.
00949
00950 template <typename T>
00951 class UniversalTersePrinter {

```

```

00952 public:
00953
00954 static void Print(const T& value, ::std::ostream* os) {
00955 UniversalPrint(value, os);
00956 }
00957 };
00958
00959 template <typename T>
00960 class UniversalTersePrinter<T&> {
00961 public:
00962
00963 static void Print(const T& value, ::std::ostream* os) {
00964 UniversalPrint(value, os);
00965 }
00966 };
00967
00968 template <typename T, size_t N>
00969 class UniversalTersePrinter<T[N]> {
00970 public:
00971
00972 static void Print(const T(&value)[N], ::std::ostream* os) {
00973 UniversalPrinter < T[N]>::Print(value, os);
00974 }
00975 };
00976
00977 template <>
00978 class UniversalTersePrinter<const char*> {
00979 public:
00980
00981 static void Print(const char* str, ::std::ostream* os) {
00982 if (str == nullptr) {
00983 *os << "NULL";
00984 } else {
00985 UniversalPrint(std::string(str), os);
00986 }
00987 }
00988 };
00989
00990 template <>
00991 class UniversalTersePrinter<char*> : public UniversalTersePrinter<const char*> {
00992 };
00993
00994 #ifdef __cpp_char8_t
00995
00996 template <>
00997 class UniversalTersePrinter<const char8_t*> {
00998 public:
00999
01000 static void Print(const char8_t* str, ::std::ostream* os) {
01001 if (str == nullptr) {
01002 *os << "NULL";
01003 } else {
01004 UniversalPrint(::std::u8string(str), os);
01005 }
01006 }
01007 };
01008
01009 template <>
01010 class UniversalTersePrinter<char8_t*>
01011 : public UniversalTersePrinter<const char8_t*> {
01012 };
01013 #endif
01014
01015 template <>
01016 class UniversalTersePrinter<const char16_t*> {
01017 public:
01018
01019 static void Print(const char16_t* str, ::std::ostream* os) {
01020 if (str == nullptr) {
01021 *os << "NULL";
01022 } else {
01023 UniversalPrint(::std::u16string(str), os);
01024 }
01025 }
01026 };
01027
01028 template <>
01029 class UniversalTersePrinter<char16_t*>
01030 : public UniversalTersePrinter<const char16_t*> {
01031 };
01032
01033 template <>
01034 class UniversalTersePrinter<const char32_t*> {
01035 public:
01036
01037 static void Print(const char32_t* str, ::std::ostream* os) {
01038 if (str == nullptr) {

```

```

01039 *os << "NULL";
01040 } else {
01041 UniversalPrint(::std::u32string(str), os);
01042 }
01043 }
01044 };
01045
01046 template <>
01047 class UniversalTersePrinter<char32_t*>
01048 : public UniversalTersePrinter<const char32_t*> {
01049 };
01050
01051 #if GTEST_HAS_STD_WSTRING
01052
01053 template <>
01054 class UniversalTersePrinter<const wchar_t*> {
01055 public:
01056
01057 static void Print(const wchar_t* str, ::std::ostream* os) {
01058 if (str == nullptr) {
01059 *os << "NULL";
01060 } else {
01061 UniversalPrint(::std::wstring(str), os);
01062 }
01063 }
01064 };
01065 #endif
01066
01067 template <>
01068 class UniversalTersePrinter<wchar_t*> {
01069 public:
01070
01071 static void Print(wchar_t* str, ::std::ostream* os) {
01072 UniversalTersePrinter<const wchar_t*>::Print(str, os);
01073 }
01074 };
01075
01076 template <typename T>
01077 void UniversalTersePrint(const T& value, ::std::ostream* os) {
01078 UniversalTersePrinter<T>::Print(value, os);
01079 }
01080
01081 // Prints a value using the type inferred by the compiler. The
01082 // difference between this and UniversalTersePrint() is that for a
01083 // (const) char pointer, this prints both the pointer and the
01084 // NUL-terminated string.
01085
01086 template <typename T>
01087 void UniversalPrint(const T& value, ::std::ostream* os) {
01088 // A workaround for the bug in VC++ 7.1 that prevents us from instantiating
01089 // UniversalPrinter with T directly.
01090 typedef T T1;
01091 UniversalPrinter<T1>::Print(value, os);
01092 }
01093
01094 typedef ::std::vector<::std::string> Strings;
01095
01096 // Tersely prints the first N fields of a tuple to a string vector,
01097 // one element for each field.
01098
01099 template <typename Tuple>
01100 void TersePrintPrefixToStrings(const Tuple& t, std::integral_constant<size_t, 0>,
01101 Strings*) {
01102 }
01103
01104 template <typename Tuple, size_t I>
01105 void TersePrintPrefixToStrings(const Tuple& t,
01106 std::integral_constant<size_t, I>,
01107 Strings* strings) {
01108 TersePrintPrefixToStrings(t, std::integral_constant<size_t, I - 1>(),
01109 strings);
01110 ::std::stringstream ss;
01111 UniversalTersePrint(std::get < I - 1 > (t), &ss);
01112 strings->push_back(ss.str());
01113 }
01114
01115 // Prints the fields of a tuple tersely to a string vector, one
01116 // element for each field. See the comment before
01117 // UniversalTersePrint() for how we define "tersely".
01118
01119 template <typename Tuple>
01120 Strings UniversalTersePrintTupleFieldsToStrings(const Tuple& value) {
01121 Strings result;
01122 TersePrintPrefixToStrings(
01123 value, std::integral_constant<size_t, std::tuple_size<Tuple>::value>(),
01124 &result);
01125 return result;

```

```

01126 }
01127
01128 } // namespace internal
01129
01130 template <typename T>
01131 ::std::string PrintToString(const T& value) {
01132 ::std::stringstream ss;
01133 internal::UniversalTersePrinter<T>::Print(value, &ss);
01134 return ss.str();
01135 }
01136
01137 } // namespace testing
01138
01139 // Include any custom printer added by the local installation.
01140 // We must include this header at the end to make sure it can use the
01141 // declarations from this file.
01142 #include "gtest/internal/custom/gtest-printers.h"
01143
01144 #endif // GOOGLETEST_INCLUDE_GTEST_GTEST_PRINTERS_H_

```

## 16.237 gtest-printers.h

```

00001 // Copyright 2015, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029 //
00030 // This file provides an injection point for custom printers in a local
00031 // installation of gTest.
00032 // It will be included from gtest-printers.h and the overrides in this file
00033 // will be visible to everyone.
00034 //
00035 // Injection point for custom user configurations. See README for details
00036 //
00037 // ** Custom implementation starts here **
00038
00039 #ifndef GOOGLETEST_INCLUDE_GTEST_INTERNAL_CUSTOM_GTEST_PRINTERS_H_
00040 #define GOOGLETEST_INCLUDE_GTEST_INTERNAL_CUSTOM_GTEST_PRINTERS_H_
00041
00042 #endif // GOOGLETEST_INCLUDE_GTEST_INTERNAL_CUSTOM_GTEST_PRINTERS_H_

```

## 16.238 gtest-spi.h

```

00001 // Copyright 2007, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from

```

```

00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // Utilities for testing Google Test itself and code that uses Google Test
00031 // (e.g. frameworks built on top of Google Test).
00032
00033 #ifndef GOOGLETEST_INCLUDE_GTEST_GTEST_SPI_H_
00034 #define GOOGLETEST_INCLUDE_GTEST_GTEST_SPI_H_
00035
00036 #include "gtest/gtest.h"
00037
00038 GTEST_DISABLE_MSC_WARNINGS_PUSH_(4251 \
00039 /* class A needs to have dll-interface to be used by clients of class B */)
00040
00041 namespace testing {
00042
00043 // This helper class can be used to mock out Google Test failure reporting
00044 // so that we can test Google Test or code that builds on Google Test.
00045 //
00046 // An object of this class appends a TestPartResult object to the
00047 // TestPartResultArray object given in the constructor whenever a Google Test
00048 // failure is reported. It can either intercept only failures that are
00049 // generated in the same thread that created this object or it can intercept
00050 // all generated failures. The scope of this mock object can be controlled with
00051 // the second argument to the two arguments constructor.
00052
00053 class GTEST_API_ ScopedFakeTestPartResultReporter
00054 : public TestPartResultReporterInterface {
00055 public:
00056 // The two possible mocking modes of this object.
00057 enum InterceptMode {
00058 INTERCEPT_ONLY_CURRENT_THREAD, // Intercepts only thread local failures.
00059 INTERCEPT_ALL_THREADS // Intercepts all failures.
00060 };
00061
00062 // The c'tor sets this object as the test part result reporter used
00063 // by Google Test. The 'result' parameter specifies where to report the
00064 // results. This reporter will only catch failures generated in the current
00065 // thread. DEPRECATED
00066 explicit ScopedFakeTestPartResultReporter(TestPartResultArray* result);
00067
00068 // Same as above, but you can choose the interception scope of this object.
00069 ScopedFakeTestPartResultReporter(InterceptMode intercept_mode,
00070 TestPartResultArray* result);
00071
00072 // The d'tor restores the previous test part result reporter.
00073 ~ScopedFakeTestPartResultReporter() override;
00074
00075 // Appends the TestPartResult object to the TestPartResultArray
00076 // received in the constructor.
00077 //
00078 // This method is from the TestPartResultReporterInterface
00079 // interface.
00080 void ReportTestPartResult(const TestPartResult& result) override;
00081
00082 private:
00083 void Init();
00084
00085 const InterceptMode intercept_mode_;
00086 TestPartResultReporterInterface* old_reporter_;
00087 TestPartResultArray* const result_;
00088
00089 ScopedFakeTestPartResultReporter(const ScopedFakeTestPartResultReporter&) =
00090 delete;
00091 ScopedFakeTestPartResultReporter& operator=(
00092 const ScopedFakeTestPartResultReporter&) = delete;
00093 };
00094
00095 namespace internal {
00096
00097 // A helper class for implementing EXPECT_FATAL_FAILURE() and
00098 // EXPECT_NONFATAL_FAILURE(). Its destructor verifies that the given
00099 // TestPartResultArray contains exactly one failure that has the given
00100 // type and contains the given substring. If that's not the case, a
00101 // non-fatal failure will be generated.

```

```

00103
00104 class GTEST_API_ SingleFailureChecker {
00105 public:
00106 // The constructor remembers the arguments.
00107 SingleFailureChecker(const TestPartResultArray* results,
00108 TestPartResult::Type type, const std::string& substr);
00109 ~SingleFailureChecker();
00110
00111 private:
00112 const TestPartResultArray * const results_;
00113 const TestPartResult::Type type_;
00114 const std::string substr_;
00115
00116 SingleFailureChecker(const SingleFailureChecker&) = delete;
00117 SingleFailureChecker& operator=(const SingleFailureChecker&) = delete;
00118 };
00119
00120 } // namespace internal
00121
00122 } // namespace testing
00123
00124 GTEST_DISABLE_MSC_WARNINGS_POP_() // 4251
00125
00126 // A set of macros for testing Google Test assertions or code that's expected
00127 // to generate Google Test fatal failures (e.g. a failure from an ASSERT_EQ, but
00128 // not a non-fatal failure, as from EXPECT_EQ). It verifies that the given
00129 // statement will cause exactly one fatal Google Test failure with 'substr'
00130 // being part of the failure message.
00131 //
00132 // There are two different versions of this macro. EXPECT_FATAL_FAILURE only
00133 // affects and considers failures generated in the current thread and
00134 // EXPECT_FATAL_FAILURE_ON_ALL_THREADS does the same but for all threads.
00135 //
00136 // The verification of the assertion is done correctly even when the statement
00137 // throws an exception or aborts the current function.
00138 //
00139 // Known restrictions:
00140 // - 'statement' cannot reference local non-static variables or
00141 // non-static members of the current object.
00142 // - 'statement' cannot return a value.
00143 // - You cannot stream a failure message to this macro.
00144 //
00145 // Note that even though the implementations of the following two
00146 // macros are much alike, we cannot refactor them to use a common
00147 // helper macro, due to some peculiarity in how the preprocessor
00148 // works. The AcceptsMacroThatExpandsToUnprotectedComma test in
00149 // gtest_unittest.cc will fail to compile if we do that.
00150 #define EXPECT_FATAL_FAILURE(statement, substr)
00151 do {
00152 class GTestExpectFatalFailureHelper {
00153 public:
00154 static void Execute() { statement; }
00155 };
00156 ::testing::TestPartResultArray gtest_failures;
00157 ::testing::internal::SingleFailureChecker gtest_checker(
00158 >est_failures, ::testing::TestPartResult::kFatalFailure, (substr));
00159 {
00160 ::testing::ScopedFakeTestPartResultReporter gtest_reporter(
00161 ::testing::ScopedFakeTestPartResultReporter::
00162 INTERCEPT_ONLY_CURRENT_THREAD,
00163 >est_failures);
00164 GTestExpectFatalFailureHelper::Execute();
00165 }
00166 } while (::testing::internal::AlwaysFalse())
00167
00168 #define EXPECT_FATAL_FAILURE_ON_ALL_THREADS(statement, substr)
00169 do {
00170 class GTestExpectFatalFailureHelper {
00171 public:
00172 static void Execute() { statement; }
00173 };
00174 ::testing::TestPartResultArray gtest_failures;
00175 ::testing::internal::SingleFailureChecker gtest_checker(
00176 >est_failures, ::testing::TestPartResult::kFatalFailure, (substr));
00177 {
00178 ::testing::ScopedFakeTestPartResultReporter gtest_reporter(
00179 ::testing::ScopedFakeTestPartResultReporter::INTERCEPT_ALL_THREADS,
00180 >est_failures);
00181 GTestExpectFatalFailureHelper::Execute();
00182 }
00183 } while (::testing::internal::AlwaysFalse())
00184
00185 // A macro for testing Google Test assertions or code that's expected to
00186 // generate Google Test non-fatal failures (e.g. a failure from an EXPECT_EQ,
00187 // but not from an ASSERT_EQ). It asserts that the given statement will cause
00188 // exactly one non-fatal Google Test failure with 'substr' being part of the
00189 // failure message.

```

```

00190 //
00191 // There are two different versions of this macro. EXPECT_NONFATAL_FAILURE only
00192 // affects and considers failures generated in the current thread and
00193 // EXPECT_NONFATAL_FAILURE_ON_ALL_THREADS does the same but for all threads.
00194 //
00195 // 'statement' is allowed to reference local variables and members of
00196 // the current object.
00197 //
00198 // The verification of the assertion is done correctly even when the statement
00199 // throws an exception or aborts the current function.
00200 //
00201 // Known restrictions:
00202 // - You cannot stream a failure message to this macro.
00203 //
00204 // Note that even though the implementations of the following two
00205 // macros are much alike, we cannot refactor them to use a common
00206 // helper macro, due to some peculiarity in how the preprocessor
00207 // works. If we do that, the code won't compile when the user gives
00208 // EXPECT_NONFATAL_FAILURE() a statement that contains a macro that
00209 // expands to code containing an unprotected comma. The
00210 // AcceptsMacroThatExpandsToUnprotectedComma test in gtest_unittest.cc
00211 // catches that.
00212 //
00213 // For the same reason, we have to write
00214 // if (::testing::internal::AlwaysTrue()) { statement; }
00215 // instead of
00216 // GTEST_SUPPRESS_UNREACHABLE_CODE_WARNING_BELOW_(statement)
00217 // to avoid an MSVC warning on unreachable code.
00218 #define EXPECT_NONFATAL_FAILURE(statement, substr)
00219 do {
00220 ::testing::TestPartResultArray gtest_failures;
00221 ::testing::internal::SingleFailureChecker gtest_checker(
00222 >est_failures, ::testing::TestPartResult::kNonFatalFailure,
00223 (substr));
00224 {
00225 ::testing::ScopedFakeTestPartResultReporter gtest_reporter(
00226 ::testing::ScopedFakeTestPartResultReporter::
00227 INTERCEPT_ONLY_CURRENT_THREAD,
00228 >est_failures);
00229 if (::testing::internal::AlwaysTrue()) {
00230 statement;
00231 }
00232 }
00233 } while (::testing::internal::AlwaysFalse())
00234
00235 #define EXPECT_NONFATAL_FAILURE_ON_ALL_THREADS(statement, substr)
00236 do {
00237 ::testing::TestPartResultArray gtest_failures;
00238 ::testing::internal::SingleFailureChecker gtest_checker(
00239 >est_failures, ::testing::TestPartResult::kNonFatalFailure,
00240 (substr));
00241 {
00242 ::testing::ScopedFakeTestPartResultReporter gtest_reporter(
00243 ::testing::ScopedFakeTestPartResultReporter::INTERCEPT_ALL_THREADS,
00244 >est_failures);
00245 if (::testing::internal::AlwaysTrue()) {
00246 statement;
00247 }
00248 }
00249 } while (::testing::internal::AlwaysFalse())
00250
00251 #endif // GOOGLETEST_INCLUDE_GTEST_GTEST_SPI_H_

```

## 16.239 gtest-test-part.h

```

00001 // Copyright 2008, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR

```

```

00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // IWYU pragma: private, include "gtest/gtest.h"
00031 // IWYU pragma: friend gtest/*.
00032 // IWYU pragma: friend gmock/*.
00033
00034 #ifndef GOOGLETEST_INCLUDE_GTEST_GTEST_TEST_PART_H_
00035 #define GOOGLETEST_INCLUDE_GTEST_GTEST_TEST_PART_H_
00036
00037 #include <iosfwd>
00038 #include <vector>
00039
00040 #include "gtest/internal/gtest-internal.h"
00041 #include "gtest/internal/gtest-string.h"
00042
00043 GTEST_DISABLE_MSC_WARNINGS_PUSH_(4251 \
00044 /* class A needs to have dll-interface to be used by clients of class B */)
00045
00046 namespace testing {
00047
00048 // A copyable object representing the result of a test part (i.e. an
00049 // assertion or an explicit FAIL(), ADD_FAILURE(), or SUCCESS()).
00050 //
00051 // Don't inherit from TestPartResult as its destructor is not virtual.
00052
00053 class GTEST_API_ TestPartResult {
00054 public:
00055 // The possible outcomes of a test part (i.e. an assertion or an
00056 // explicit SUCCEED(), FAIL(), or ADD_FAILURE()).
00057
00058 enum Type {
00059 kSuccess, // Succeeded.
00060 kNonFatalFailure, // Failed but the test can continue.
00061 kFatalFailure, // Failed and the test should be terminated.
00062 kSkip // Skipped.
00063 };
00064
00065 // C'tor. TestPartResult does NOT have a default constructor.
00066 // Always use this constructor (with parameters) to create a
00067 // TestPartResult object.
00068
00069 TestPartResult(Type a_type, const char* a_file_name, int a_line_number,
00070 const char* a_message)
00071 : type_(a_type),
00072 file_name_(a_file_name == nullptr ? "" : a_file_name),
00073 line_number_(a_line_number),
00074 summary_(ExtractSummary(a_message)),
00075 message_(a_message) {}
00076
00077 // Gets the outcome of the test part.
00078
00079 Type type() const {
00080 return type_;
00081 }
00082
00083 // Gets the name of the source file where the test part took place, or
00084 // NULL if it's unknown.
00085
00086 const char* file_name() const {
00087 return file_name_.empty() ? nullptr : file_name_.c_str();
00088 }
00089
00090 // Gets the line in the source file where the test part took place,
00091 // or -1 if it's unknown.
00092
00093 int line_number() const {
00094 return line_number_;
00095 }
00096
00097 // Gets the summary of the failure message.
00098
00099 const char* summary() const {
00100 return summary_.c_str();
00101 }
00102
00103 // Gets the message associated with the test part.
00104
00105 const char* message() const {
00106 return message_.c_str();
00107 }
00107

```



```

00108 }
00109
00110 // Returns true if and only if the test part was skipped.
00111
00112 bool skipped() const {
00113 return type_ == kSkip;
00114 }
00115
00116 // Returns true if and only if the test part passed.
00117
00118 bool passed() const {
00119 return type_ == kSuccess;
00120 }
00121
00122 // Returns true if and only if the test part non-fatally failed.
00123
00124 bool nonfatally_failed() const {
00125 return type_ == kNonFatalFailure;
00126 }
00127
00128 // Returns true if and only if the test part fatally failed.
00129
00130 bool fatally_failed() const {
00131 return type_ == kFatalFailure;
00132 }
00133
00134 // Returns true if and only if the test part failed.
00135
00136 bool failed() const {
00137 return fatally_failed() || nonfatally_failed();
00138 }
00139
00140 private:
00141 Type type_;
00142
00143 // Gets the summary of the failure message by omitting the stack
00144 // trace in it.
00145 static std::string ExtractSummary(const char* message);
00146
00147 // The name of the source file where the test part took place, or
00148 // "" if the source file is unknown.
00149 std::string file_name_;
00150 // The line in the source file where the test part took place, or -1
00151 // if the line number is unknown.
00152 int line_number_;
00153 std::string summary_; // The test failure summary.
00154 std::string message_; // The test failure message.
00155 };
00156
00157 // Prints a TestPartResult object.
00158 std::ostream& operator<<(std::ostream& os, const TestPartResult& result);
00159
00160 // An array of TestPartResult objects.
00161 //
00162 // Don't inherit from TestPartResultArray as its destructor is not
00163 // virtual.
00164
00165 class GTEST_API_ TestPartResultArray {
00166 public:
00167
00168 TestPartResultArray() {
00169 }
00170
00171 // Appends the given TestPartResult to the array.
00172 void Append(const TestPartResult& result);
00173
00174 // Returns the TestPartResult at the given index (0-based).
00175 const TestPartResult& GetTestPartResult(int index) const;
00176
00177 // Returns the number of TestPartResult objects in the array.
00178 int size() const;
00179
00180 private:
00181 std::vector<TestPartResult> array_;
00182
00183 TestPartResultArray(const TestPartResultArray&) = delete;
00184 TestPartResultArray& operator=(const TestPartResultArray&) = delete;
00185 };
00186
00187 // This interface knows how to report a test part result.
00188
00189 class GTEST_API_ TestPartResultReporterInterface {
00190 public:
00191
00192 virtual ~TestPartResultReporterInterface() {
00193 }
00194

```

```

00195 virtual void ReportTestPartResult(const TestPartResult& result) = 0;
00196 };
00197
00198 namespace internal {
00199
00200 // This helper class is used by {ASSERT|EXPECT}_NO_FATAL_FAILURE to check if a
00201 // statement generates new fatal failures. To do so it registers itself as the
00202 // current test part result reporter. Besides checking if fatal failures were
00203 // reported, it only delegates the reporting to the former result reporter.
00204 // The original result reporter is restored in the destructor.
00205 // INTERNAL IMPLEMENTATION - DO NOT USE IN A USER PROGRAM.
00206
00207 class GTEST_API_ HasNewFatalFailureHelper
00208 : public TestPartResultReporterInterface {
00209 public:
00210 HasNewFatalFailureHelper();
00211 ~HasNewFatalFailureHelper() override;
00212 void ReportTestPartResult(const TestPartResult& result) override;
00213
00214 bool has_new_fatal_failure() const {
00215 return has_new_fatal_failure_;
00216 }
00217
00218 private:
00219 bool has_new_fatal_failure_;
00220 TestPartResultReporterInterface* original_reporter_;
00221
00222 HasNewFatalFailureHelper(const HasNewFatalFailureHelper&) = delete;
00223 HasNewFatalFailureHelper& operator=(const HasNewFatalFailureHelper&) = delete;
00224 };
00225
00226 } // namespace internal
00227
00228 } // namespace testing
00229
00230 GTEST_DISABLE_MSC_WARNINGS_POP_() // 4251
00231
00232 #endif // GOOGLETEST_INCLUDE_GTEST_GTEST_TEST_PART_H_

```

## 16.240 gtest-typed-test.h

```

00001 // Copyright 2008 Google Inc.
00002 // All Rights Reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // IWYU pragma: private, include "gtest/gtest.h"
00031 // IWYU pragma: friend gtest/*.h
00032 // IWYU pragma: friend gmock/*.h
00033
00034 #ifndef GOOGLETEST_INCLUDE_GTEST_GTEST_TYPED_TEST_H_
00035 #define GOOGLETEST_INCLUDE_GTEST_GTEST_TYPED_TEST_H_
00036
00037 // This header implements typed tests and type-parameterized tests.
00038
00039 // Typed (aka type-driven) tests repeat the same test for types in a
00040 // list. You must know which types you want to test with when writing
00041 // typed tests. Here's how you do it:
00042
00043 #if 0
00044

```

```

00045 // First, define a fixture class template. It should be parameterized
00046 // by a type. Remember to derive it from testing::Test.
00047
00048 template <typename T>
00049 class FooTest : public testing::Test {
00050 public:
00051 ...
00052 typedef std::list<T> List;
00053 static T shared_;
00054 T value_;
00055 };
00056
00057 // Next, associate a list of types with the test suite, which will be
00058 // repeated for each type in the list. The typedef is necessary for
00059 // the macro to parse correctly.
00060 typedef testing::Types<char, int, unsigned int> MyTypes;
00061 TYPED_TEST_SUITE(FooTest, MyTypes);
00062
00063 // If the type list contains only one type, you can write that type
00064 // directly without Types<...>:
00065 // TYPED_TEST_SUITE(FooTest, int);
00066
00067 // Then, use TYPED_TEST() instead of TEST_F() to define as many typed
00068 // tests for this test suite as you want.
00069
00070 TYPED_TEST(FooTest, DoesBlah) {
00071 // Inside a test, refer to the special name TypeParam to get the type
00072 // parameter. Since we are inside a derived class template, C++ requires
00073 // us to visit the members of FooTest via 'this'.
00074 TypeParam n = this->value_;
00075
00076 // To visit static members of the fixture, add the TestFixture::
00077 // prefix.
00078 n += TestFixture::shared_;
00079
00080 // To refer to typedefs in the fixture, add the "typename
00081 // TestFixture::" prefix.
00082 typename TestFixture::List values;
00083 values.push_back(n);
00084 ...
00085 }
00086
00087 TYPED_TEST(FooTest, HasPropertyA) {
00088 ...
00089 }
00090
00091 // TYPED_TEST_SUITE takes an optional third argument which allows to specify a
00092 // class that generates custom test name suffixes based on the type. This should
00093 // be a class which has a static template function GetName(int index) returning
00094 // a string for each type. The provided integer index equals the index of the
00095 // type in the provided type list. In many cases the index can be ignored.
00096 //
00097 // For example:
00098 // class MyTypeNames {
00099 // public:
00100 // template <typename T>
00101 // static std::string GetName(int) {
00102 // if (std::is_same<T, char>()) return "char";
00103 // if (std::is_same<T, int>()) return "int";
00104 // if (std::is_same<T, unsigned int>()) return "unsignedInt";
00105 // }
00106 // };
00107 // TYPED_TEST_SUITE(FooTest, MyTypes, MyTypeNames);
00108
00109 #endif // 0
00110
00111 // Type-parameterized tests are abstract test patterns parameterized
00112 // by a type. Compared with typed tests, type-parameterized tests
00113 // allow you to define the test pattern without knowing what the type
00114 // parameters are. The defined pattern can be instantiated with
00115 // different types any number of times, in any number of translation
00116 // units.
00117 //
00118 // If you are designing an interface or concept, you can define a
00119 // suite of type-parameterized tests to verify properties that any
00120 // valid implementation of the interface/concept should have. Then,
00121 // each implementation can easily instantiate the test suite to verify
00122 // that it conforms to the requirements, without having to write
00123 // similar tests repeatedly. Here's an example:
00124
00125 #if 0
00126
00127 // First, define a fixture class template. It should be parameterized
00128 // by a type. Remember to derive it from testing::Test.
00129
00130 template <typename T>
00131 class FooTest : public testing::Test {

```

```

00132 ...
00133 };
00134
00135 // Next, declare that you will define a type-parameterized test suite
00136 // (the _P suffix is for "parameterized" or "pattern", whichever you
00137 // prefer):
00138 TYPED_TEST_SUITE_P(FooTest);
00139
00140 // Then, use TYPED_TEST_P() to define as many type-parameterized tests
00141 // for this type-parameterized test suite as you want.
00142
00143 TYPED_TEST_P(FooTest, DoesBlah) {
00144 // Inside a test, refer to TypeParam to get the type parameter.
00145 TypeParam n = 0;
00146 ...
00147 }
00148
00149 TYPED_TEST_P(FooTest, HasPropertyA) {
00150 ...
00151 }
00152
00153 // Now the tricky part: you need to register all test patterns before
00154 // you can instantiate them. The first argument of the macro is the
00155 // test suite name; the rest are the names of the tests in this test
00156 // case.
00157 REGISTER_TYPED_TEST_SUITE_P(FooTest,
00158 DoesBlah, HasPropertyA);
00159
00160 // Finally, you are free to instantiate the pattern with the types you
00161 // want. If you put the above code in a header file, you can #include
00162 // it in multiple C++ source files and instantiate it multiple times.
00163 //
00164 // To distinguish different instances of the pattern, the first
00165 // argument to the INSTANTIATE_* macro is a prefix that will be added
00166 // to the actual test suite name. Remember to pick unique prefixes for
00167 // different instances.
00168 typedef testing::Types<char, int, unsigned int> MyTypes;
00169 INSTANTIATE_TYPED_TEST_SUITE_P(My, FooTest, MyTypes);
00170
00171 // If the type list contains only one type, you can write that type
00172 // directly without Types<...>:
00173 // INSTANTIATE_TYPED_TEST_SUITE_P(My, FooTest, int);
00174 //
00175 // Similar to the optional argument of TYPED_TEST_SUITE above,
00176 // INSTANTIATE_TEST_SUITE_P takes an optional fourth argument which allows to
00177 // generate custom names.
00178 // INSTANTIATE_TYPED_TEST_SUITE_P(My, FooTest, MyTypes, MyTypeNames);
00179
00180 #endif // 0
00181
00182 #include "gtest/internal/gtest-internal.h"
00183 #include "gtest/internal/gtest-port.h"
00184 #include "gtest/internal/gtest-type-util.h"
00185
00186 // Implements typed tests.
00187
00188 // INTERNAL IMPLEMENTATION - DO NOT USE IN USER CODE.
00189 //
00190 // Expands to the name of the typedef for the type parameters of the
00191 // given test suite.
00192 #define GTEST_TYPE_PARAMS_(TestSuiteName) gtest_type_params_##TestSuiteName##_
00193
00194 // Expands to the name of the typedef for the NameGenerator, responsible for
00195 // creating the suffixes of the name.
00196 #define GTEST_NAME_GENERATOR_(TestSuiteName) \
00197 gtest_type_params_##TestSuiteName##_NameGenerator
00198
00199 #define TYPED_TEST_SUITE(CaseName, Types, ...) \
00200 typedef ::testing::internal::GenerateTypeList<Types>::type \
00201 GTEST_TYPE_PARAMS_(CaseName); \
00202 typedef ::testing::internal::NameGeneratorSelector<__VA_ARGS__>::type \
00203 GTEST_NAME_GENERATOR_(CaseName)
00204
00205 #define TYPED_TEST(CaseName, TestName) \
00206 static_assert(sizeof(GTEST_STRINGIFY_(TestName)) > 1, \
00207 "test-name must not be empty"); \
00208 template <typename gtest_TypeParam_> \
00209 class GTEST_TEST_CLASS_NAME_(CaseName, TestName) \
00210 : public CaseName<gtest_TypeParam_> { \
00211 private: \
00212 typedef CaseName<gtest_TypeParam_> TestFixture; \
00213 typedef gtest_TypeParam_ TypeParam; \
00214 void TestBody() override; \
00215 }; \
00216 static bool gtest_##CaseName##_##TestName##_registered_ \
00217 GTEST_ATTRIBUTE_UNUSED_ = ::testing::internal::TypeParameterizedTest< \
00218 CaseName,

```

```

00219 ::testing::internal::TemplateSel<GTEST_TEST_CLASS_NAME_(CaseName, \
00220 TestName)>, \
00221 GTEST_TYPE_PARAMS_(\
00222 CaseName)>::Register("", \
00223 ::testing::internal::CodeLocation(\
00224 __FILE__, __LINE__), \
00225 GTEST_STRINGIFY_(CaseName), \
00226 GTEST_STRINGIFY_(TestName), 0, \
00227 ::testing::internal::GenerateNames< \
00228 GTEST_NAME_GENERATOR_(CaseName), \
00229 GTEST_TYPE_PARAMS_(CaseName)>()); \
00230 template <typename gtest_TypeParam> \
00231 void GTEST_TEST_CLASS_NAME_(CaseName, \
00232 TestName)<gtest_TypeParam>::TestBody() \
00233 \
00234 // Legacy API is deprecated but still available
00235 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00236 #define TYPED_TEST_CASE \
00237 static_assert(::testing::internal::TypedTestCaseIsDeprecated(), ""); \
00238 TYPED_TEST_SUITE \
00239 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00240 \
00241 // Implements type-parameterized tests.
00242 \
00243 // INTERNAL IMPLEMENTATION - DO NOT USE IN USER CODE.
00244 // \
00245 // Expands to the namespace name that the type-parameterized tests for \
00246 // the given type-parameterized test suite are defined in. The exact \
00247 // name of the namespace is subject to change without notice.
00248 #define GTEST_SUITE_NAMESPACE_(TestSuiteName) gtest_suite_##TestSuiteName##_
00249 \
00250 // INTERNAL IMPLEMENTATION - DO NOT USE IN USER CODE.
00251 // \
00252 // Expands to the name of the variable used to remember the names of \
00253 // the defined tests in the given test suite.
00254 #define GTEST_TYPED_TEST_SUITE_P_STATE_(TestSuiteName) \
00255 gtest_typed_test_suite_p_state_##TestSuiteName##_
00256 \
00257 // INTERNAL IMPLEMENTATION - DO NOT USE IN USER CODE DIRECTLY.
00258 // \
00259 // Expands to the name of the variable used to remember the names of \
00260 // the registered tests in the given test suite.
00261 #define GTEST_REGISTERED_TEST_NAMES_(TestSuiteName) \
00262 gtest_registered_test_names_##TestSuiteName##_
00263 \
00264 // The variables defined in the type-parameterized test macros are \
00265 // static as typically these macros are used in a .h file that can be \
00266 // #included in multiple translation units linked together.
00267 #define TYPED_TEST_SUITE_P(SuiteName) \
00268 static ::testing::internal::TypedTestSuitePState \
00269 GTEST_TYPED_TEST_SUITE_P_STATE_(SuiteName)
00270 \
00271 // Legacy API is deprecated but still available
00272 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00273 #define TYPED_TEST_CASE_P \
00274 static_assert(::testing::internal::TypedTestCaseP_IsDeprecated(), ""); \
00275 TYPED_TEST_SUITE_P \
00276 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00277 \
00278 #define TYPED_TEST_P(SuiteName, TestName) \
00279 namespace GTEST_SUITE_NAMESPACE_(SuiteName) { \
00280 template <typename gtest_TypeParam> \
00281 class TestName : public SuiteName<gtest_TypeParam> { \
00282 private: \
00283 typedef SuiteName<gtest_TypeParam> TestFixture; \
00284 typedef gtest_TypeParam TypeParam; \
00285 void TestBody() override; \
00286 }; \
00287 static bool gtest_##TestName##_defined_ GTEST_ATTRIBUTE_UNUSED_ = \
00288 GTEST_TYPED_TEST_SUITE_P_STATE_(SuiteName).AddTestName(\
00289 __FILE__, __LINE__, GTEST_STRINGIFY_(SuiteName), \
00290 GTEST_STRINGIFY_(TestName)); \
00291 } \
00292 template <typename gtest_TypeParam> \
00293 void GTEST_SUITE_NAMESPACE_(\
00294 SuiteName)::TestName<gtest_TypeParam>::TestBody() \
00295 \
00296 // Note: this won't work correctly if the trailing arguments are macros.
00297 #define REGISTER_TYPED_TEST_SUITE_P(SuiteName, ...) \
00298 namespace GTEST_SUITE_NAMESPACE_(SuiteName) { \
00299 typedef ::testing::internal::Templates<__VA_ARGS__> gtest_AllTests_; \
00300 } \
00301 static const char* const GTEST_REGISTERED_TEST_NAMES_(\
00302 SuiteName) GTEST_ATTRIBUTE_UNUSED_ = \
00303 GTEST_TYPED_TEST_SUITE_P_STATE_(SuiteName).VerifyRegisteredTestNames(\
00304 GTEST_STRINGIFY_(SuiteName), __FILE__, __LINE__, #__VA_ARGS__)
00305

```

```

00306 // Legacy API is deprecated but still available
00307 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00308 #define REGISTER_TYPED_TEST_CASE_P
00309 static_assert(::testing::internal::RegisterTypedTestCase_P_IsDeprecated(), \
00310 "");
00311 REGISTER_TYPED_TEST_SUITE_P
00312 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00313
00314 #define INSTATIATE_TYPED_TEST_SUITE_P(Prefix, SuiteName, Types, ...) \
00315 static_assert(sizeof(GTEST_STRINGIFY_(Prefix)) > 1, \
00316 "test-suit-prefix must not be empty"); \
00317 static bool gtest_##Prefix##_##SuiteName GTEST_ATTRIBUTE_UNUSED_ = \
00318 ::testing::internal::TypeParameterizedTestSuite< \
00319 SuiteName, GTEST_SUITE_NAMESPACE_(SuiteName)::gtest_AllTests_, \
00320 ::testing::internal::GenerateTypeList<Types>::type>:: \
00321 Register(GTEST_STRINGIFY_(Prefix), \
00322 ::testing::internal::CodeLocation(__FILE__, __LINE__), \
00323 >EST_TYPED_TEST_SUITE_P_STATE_(SuiteName), \
00324 GTEST_STRINGIFY_(SuiteName), \
00325 GTEST_REGISTERED_TEST_NAMES_(SuiteName), \
00326 ::testing::internal::GenerateNames< \
00327 ::testing::internal::NameGeneratorSelector< \
00328 __VA_ARGS__>::type, \
00329 ::testing::internal::GenerateTypeList<Types>::type>()) \
00330
00331 // Legacy API is deprecated but still available
00332 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00333 #define INSTATIATE_TYPED_TEST_CASE_P
00334 static_assert(\
00335 ::testing::internal::InstantiateTypedTestCase_P_IsDeprecated(), ""); \
00336 INSTATIATE_TYPED_TEST_SUITE_P
00337 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00338
00339 #endif // GOOGLETEST_INCLUDE_GTEST_GTEST_TYPED_TEST_H_

```

## 16.241 gtest.h

```

00001 // Copyright 2005, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // The Google C++ Testing and Mocking Framework (Google Test)
00031 //
00032 // This header file defines the public API for Google Test. It should be
00033 // included by any test program that uses Google Test.
00034 //
00035 // IMPORTANT NOTE: Due to limitation of the C++ language, we have to
00036 // leave some internal implementation details in this header file.
00037 // They are clearly marked by comments like this:
00038 //
00039 // // INTERNAL IMPLEMENTATION - DO NOT USE IN A USER PROGRAM.
00040 //
00041 // Such code is NOT meant to be used by a user directly, and is subject
00042 // to CHANGE WITHOUT NOTICE. Therefore DO NOT DEPEND ON IT in a user
00043 // program!
00044 //
00045 // Acknowledgment: Google Test borrowed the idea of automatic test
00046 // registration from Barthelemy Dagenais' (barthelemy@prologique.com)
00047 // easyUnit framework.
00048

```

```
00049 #ifndef GOOGLETEST_INCLUDE_GTEST_GTEST_H_
00050 #define GOOGLETEST_INCLUDE_GTEST_GTEST_H_
00051
00052 #include <cstdlib>
00053 #include <limits>
00054 #include <memory>
00055 #include <ostream>
00056 #include <type_traits>
00057 #include <vector>
00058
00059 #include "gtest/gtest-assertion-result.h"
00060 #include "gtest/gtest-death-test.h"
00061 #include "gtest/gtest-matchers.h"
00062 #include "gtest/gtest-message.h"
00063 #include "gtest/gtest-param-test.h"
00064 #include "gtest/gtest-printers.h"
00065 #include "gtest/gtest-test-part.h"
00066 #include "gtest/gtest-typed-test.h"
00067 #include "gtest/gtest_pred_impl.h"
00068 #include "gtest/gtest_prod.h"
00069 #include "gtest/internal/gtest-internal.h"
00070 #include "gtest/internal/gtest-string.h"
00071
00072 GTEST_DISABLE_MSC_WARNINGS_PUSH_(4251 \
00073 /* class A needs to have dll-interface to be used by clients of class B */)
00074
00075 // Declares the flags.
00076
00077 // This flag temporary enables the disabled tests.
00078 GTEST_DECLARE_bool_(also_run_disabled_tests);
00079
00080 // This flag brings the debugger on an assertion failure.
00081 GTEST_DECLARE_bool_(break_on_failure);
00082
00083 // This flag controls whether Google Test catches all test-thrown exceptions
00084 // and logs them as failures.
00085 GTEST_DECLARE_bool_(catch_exceptions);
00086
00087 // This flag enables using colors in terminal output. Available values are
00088 // "yes" to enable colors, "no" (disable colors), or "auto" (the default)
00089 // to let Google Test decide.
00090 GTEST_DECLARE_string_(color);
00091
00092 // This flag controls whether the test runner should continue execution past
00093 // first failure.
00094 GTEST_DECLARE_bool_(fail_fast);
00095
00096 // This flag sets up the filter to select by name using a glob pattern
00097 // the tests to run. If the filter is not given all tests are executed.
00098 GTEST_DECLARE_string_(filter);
00099
00100 // This flag controls whether Google Test installs a signal handler that dumps
00101 // debugging information when fatal signals are raised.
00102 GTEST_DECLARE_bool_(install_failure_signal_handler);
00103
00104 // This flag causes the Google Test to list tests. None of the tests listed
00105 // are actually run if the flag is provided.
00106 GTEST_DECLARE_bool_(list_tests);
00107
00108 // This flag controls whether Google Test emits a detailed XML report to a file
00109 // in addition to its normal textual output.
00110 GTEST_DECLARE_string_(output);
00111
00112 // This flags control whether Google Test prints only test failures.
00113 GTEST_DECLARE_bool_(brief);
00114
00115 // This flags control whether Google Test prints the elapsed time for each
00116 // test.
00117 GTEST_DECLARE_bool_(print_time);
00118
00119 // This flags control whether Google Test prints UTF8 characters as text.
00120 GTEST_DECLARE_bool_(print_utf8);
00121
00122 // This flag specifies the random number seed.
00123 GTEST_DECLARE_int32_(random_seed);
00124
00125 // This flag sets how many times the tests are repeated. The default value
00126 // is 1. If the value is -1 the tests are repeating forever.
00127 GTEST_DECLARE_int32_(repeat);
00128
00129 // This flag controls whether Google Test Environments are recreated for each
00130 // repeat of the tests. The default value is true. If set to false the global
00131 // test Environment objects are only set up once, for the first iteration, and
00132 // only torn down once, for the last.
00133 GTEST_DECLARE_bool_(recreate_environments_when_repeating);
00134
00135 // This flag controls whether Google Test includes Google Test internal
```

```

00136 // stack frames in failure stack traces.
00137 GTEST_DECLARE_bool_(show_internal_stack_frames);
00138
00139 // When this flag is specified, tests' order is randomized on every iteration.
00140 GTEST_DECLARE_bool_(shuffle);
00141
00142 // This flag specifies the maximum number of stack frames to be
00143 // printed in a failure message.
00144 GTEST_DECLARE_int32_(stack_trace_depth);
00145
00146 // When this flag is specified, a failed assertion will throw an
00147 // exception if exceptions are enabled, or exit the program with a
00148 // non-zero code otherwise. For use with an external test framework.
00149 GTEST_DECLARE_bool_(throw_on_failure);
00150
00151 // When this flag is set with a "host:port" string, on supported
00152 // platforms test results are streamed to the specified port on
00153 // the specified host machine.
00154 GTEST_DECLARE_string_(stream_result_to);
00155
00156 #if GTEST_USE_OWN_FLAGFILE_FLAG_
00157 GTEST_DECLARE_string_(flagfile);
00158 #endif // GTEST_USE_OWN_FLAGFILE_FLAG_
00159
00160 namespace testing {
00161
00162 // Silence C4100 (unreferenced formal parameter) and 4805
00163 // unsafe mix of type 'const int' and type 'const bool'
00164 #ifdef _MSC_VER
00165 #pragma warning(push)
00166 #pragma warning(disable : 4805)
00167 #pragma warning(disable : 4100)
00168 #endif
00169
00170 // The upper limit for valid stack trace depths.
00171 const int kMaxStackTraceDepth = 100;
00172
00173 namespace internal {
00174
00175 class AssertHelper;
00176 class DefaultGlobalTestPartResultReporter;
00177 class ExecDeathTest;
00178 class NoExecDeathTest;
00179 class FinalSuccessChecker;
00180 class GTestFlagSaver;
00181 class StreamingListenerTest;
00182 class TestResultAccessor;
00183 class TestEventListenersAccessor;
00184 class TestEventRepeater;
00185 class UnitTestRecordPropertyTestHelper;
00186 class WindowsDeathTest;
00187 class FuchsiaDeathTest;
00188 class UnitTestImpl* GetUnitTestImpl();
00189 void ReportFailureInUnknownLocation(TestPartResult::Type result_type,
00190 const std::string& message);
00191 std::set<std::string*> GetIgnoredParameterizedTestSuites();
00192
00193 } // namespace internal
00194
00195 // The friend relationship of some of these classes is cyclic.
00196 // If we don't forward declare them the compiler might confuse the classes
00197 // in friendship clauses with same named classes on the scope.
00198 class Test;
00199 class TestSuite;
00200
00201 // Old API is still available but deprecated
00202 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00203 using TestCase = TestSuite;
00204 #endif
00205 class TestInfo;
00206 class UnitTest;
00207
00208 // The abstract class that all tests inherit from.
00209 //
00210 // In Google Test, a unit test program contains one or many TestSuites, and
00211 // each TestSuite contains one or many Tests.
00212 //
00213 // When you define a test using the TEST macro, you don't need to
00214 // explicitly derive from Test - the TEST macro automatically does
00215 // this for you.
00216 //
00217 // The only time you derive from Test is when defining a test fixture
00218 // to be used in a TEST_F. For example:
00219 //
00220 // class FooTest : public testing::Test {
00221 // protected:
00222 // void SetUp() override { ... }

```



```

00223 // void TearDown() override { ... }
00224 // ...
00225 // };
00226 //
00227 // TEST_F(FooTest, Bar) { ... }
00228 // TEST_F(FooTest, Baz) { ... }
00229 //
00230 // Test is not copyable.
00231
00232 class GTEST_API_ Test {
00233 public:
00234 friend class TestInfo;
00235
00236 // The d'tor is virtual as we intend to inherit from Test.
00237 virtual ~Test();
00238
00239 // Sets up the stuff shared by all tests in this test suite.
00240 //
00241 // Google Test will call Foo::SetUpTestSuite() before running the first
00242 // test in test suite Foo. Hence a sub-class can define its own
00243 // SetUpTestSuite() method to shadow the one defined in the super
00244 // class.
00245
00246 static void SetUpTestSuite() {
00247 }
00248
00249 // Tears down the stuff shared by all tests in this test suite.
00250 //
00251 // Google Test will call Foo::TearDownTestSuite() after running the last
00252 // test in test suite Foo. Hence a sub-class can define its own
00253 // TearDownTestSuite() method to shadow the one defined in the super
00254 // class.
00255
00256 static void TearDownTestSuite() {
00257 }
00258
00259 // Legacy API is deprecated but still available. Use SetUpTestSuite and
00260 // TearDownTestSuite instead.
00261 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00262
00263 static void TearDownTestCase() {
00264 }
00265
00266 static void SetUpTestCase() {
00267 }
00268 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00269
00270 // Returns true if and only if the current test has a fatal failure.
00271 static bool HasFatalFailure();
00272
00273 // Returns true if and only if the current test has a non-fatal failure.
00274 static bool HasNonfatalFailure();
00275
00276 // Returns true if and only if the current test was skipped.
00277 static bool IsSkipped();
00278
00279 // Returns true if and only if the current test has a (either fatal or
00280 // non-fatal) failure.
00281
00282 static bool HasFailure() {
00283 return HasFatalFailure() || HasNonfatalFailure();
00284 }
00285
00286 // Logs a property for the current test, test suite, or for the entire
00287 // invocation of the test program when used outside of the context of a
00288 // test suite. Only the last value for a given key is remembered. These
00289 // are public static so they can be called from utility functions that are
00290 // not members of the test fixture. Calls to RecordProperty made during
00291 // lifespan of the test (from the moment its constructor starts to the
00292 // moment its destructor finishes) will be output in XML as attributes of
00293 // the <testcase> element. Properties recorded from fixture's
00294 // SetUpTestSuite or TearDownTestSuite are logged as attributes of the
00295 // corresponding <testsuite> element. Calls to RecordProperty made in the
00296 // global context (before or after invocation of RUN_ALL_TESTS and from
00297 // SetUp/TearDown method of Environment objects registered with Google
00298 // Test) will be output as attributes of the <testsuites> element.
00299 static void RecordProperty(const std::string& key, const std::string& value);
00300 static void RecordProperty(const std::string& key, int value);
00301
00302 protected:
00303 // Creates a Test object.
00304 Test();
00305
00306 // Sets up the test fixture.
00307 virtual void SetUp();
00308
00309 // Tears down the test fixture.

```

```

00310 virtual void TearDown();
00311
00312 private:
00313 // Returns true if and only if the current test has the same fixture class
00314 // as the first test in the current test suite.
00315 static bool HasSameFixtureClass();
00316
00317 // Runs the test after the test fixture has been set up.
00318 //
00319 // A sub-class must implement this to define the test logic.
00320 //
00321 // DO NOT OVERRIDE THIS FUNCTION DIRECTLY IN A USER PROGRAM.
00322 // Instead, use the TEST or TEST_F macro.
00323 virtual void TestBody() = 0;
00324
00325 // Sets up, executes, and tears down the test.
00326 void Run();
00327
00328 // Deletes self. We deliberately pick an unusual name for this
00329 // internal method to avoid clashing with names used in user TESTs.
00330
00331 void DeleteSelf_() {
00332 delete this;
00333 }
00334
00335 const std::unique_ptr<GTEST_FLAG_SAVER> gtest_flag_saver_;
00336
00337 // Often a user misspells SetUp() as Setup() and spends a long time
00338 // wondering why it is never called by Google Test. The declaration of
00339 // the following method is solely for catching such an error at
00340 // compile time:
00341 //
00342 // - The return type is deliberately chosen to be not void, so it
00343 // will be a conflict if void Setup() is declared in the user's
00344 // test fixture.
00345 //
00346 // - This method is private, so it will be another compiler error
00347 // if the method is called from the user's test fixture.
00348 //
00349 // DO NOT OVERRIDE THIS FUNCTION.
00350 //
00351 // If you see an error about overriding the following function or
00352 // about it being private, you have mis-spelled SetUp() as Setup().
00353
00354 struct Setup_should_be_spelled_SetUp {
00355 };
00356
00357 virtual Setup_should_be_spelled_SetUp* Setup() {
00358 return nullptr;
00359 }
00360
00361 // We disallow copying Tests.
00362 Test(const Test&) = delete;
00363 Test& operator=(const Test&) = delete;
00364 };
00365
00366 typedef internal::TimeInMillis TimeInMillis;
00367
00368 // A copyable object representing a user specified test property which can be
00369 // output as a key/value string pair.
00370 //
00371 // Don't inherit from TestProperty as its destructor is not virtual.
00372
00373 class TestProperty {
00374 public:
00375 // C'tor. TestProperty does NOT have a default constructor.
00376 // Always use this constructor (with parameters) to create a
00377 // TestProperty object.
00378
00379 TestProperty(const std::string& a_key, const std::string& a_value)
00380 : key_(a_key), value_(a_value) {
00381 }
00382
00383 // Gets the user supplied key.
00384
00385 const char* key() const {
00386 return key_.c_str();
00387 }
00388
00389 // Gets the user supplied value.
00390
00391 const char* value() const {
00392 return value_.c_str();
00393 }
00394
00395 // Sets a new value, overriding the one supplied in the constructor.
00396

```

```

00397 void SetValue(const std::string& new_value) {
00398 value_ = new_value;
00399 }
00400
00401 private:
00402 // The key supplied by the user.
00403 std::string key_;
00404 // The value supplied by the user.
00405 std::string value_;
00406 };
00407
00408 // The result of a single Test. This includes a list of
00409 // TestPartResults, a list of TestProperties, a count of how many
00410 // death tests there are in the Test, and how much time it took to run
00411 // the Test.
00412 //
00413 // TestResult is not copyable.
00414
00415 class GTEST_API_ TestResult {
00416 public:
00417 // Creates an empty TestResult.
00418 TestResult();
00419
00420 // D'tor. Do not inherit from TestResult.
00421 ~TestResult();
00422
00423 // Gets the number of all test parts. This is the sum of the number
00424 // of successful test parts and the number of failed test parts.
00425 int total_part_count() const;
00426
00427 // Returns the number of the test properties.
00428 int test_property_count() const;
00429
00430 // Returns true if and only if the test passed (i.e. no test part failed).
00431
00432 bool Passed() const {
00433 return !Skipped() && !Failed();
00434 }
00435
00436 // Returns true if and only if the test was skipped.
00437 bool Skipped() const;
00438
00439 // Returns true if and only if the test failed.
00440 bool Failed() const;
00441
00442 // Returns true if and only if the test fatally failed.
00443 bool HasFatalFailure() const;
00444
00445 // Returns true if and only if the test has a non-fatal failure.
00446 bool HasNonfatalFailure() const;
00447
00448 // Returns the elapsed time, in milliseconds.
00449
00450 TimeInMillis elapsed_time() const {
00451 return elapsed_time_;
00452 }
00453
00454 // Gets the time of the test case start, in ms from the start of the
00455 // UNIX epoch.
00456
00457 TimeInMillis start_timestamp() const {
00458 return start_timestamp_;
00459 }
00460
00461 // Returns the i-th test part result among all the results. i can range from 0
00462 // to total_part_count() - 1. If i is not in that range, aborts the program.
00463 const TestPartResult& GetTestPartResult(int i) const;
00464
00465 // Returns the i-th test property. i can range from 0 to
00466 // test_property_count() - 1. If i is not in that range, aborts the
00467 // program.
00468 const TestProperty& GetTestProperty(int i) const;
00469
00470 private:
00471 friend class TestInfo;
00472 friend class TestSuite;
00473 friend class UnitTest;
00474 friend class internal::DefaultGlobalTestPartResultReporter;
00475 friend class internal::ExecDeathTest;
00476 friend class internal::TestResultAccessor;
00477 friend class internal::UnitTestImpl;
00478 friend class internal::WindowsDeathTest;
00479 friend class internal::FuchsiaDeathTest;
00480
00481 // Gets the vector of TestPartResults.
00482
00483 const std::vector<TestPartResult>& test_part_results() const {

```

```

00484 return test_part_results_;
00485 }
00486
00487 // Gets the vector of TestProperties.
00488
00489 const std::vector<TestProperty>& test_properties() const {
00490 return test_properties_;
00491 }
00492
00493 // Sets the start time.
00494
00495 void set_start_timestamp(TimeInMillis start) {
00496 start_timestamp_ = start;
00497 }
00498
00499 // Sets the elapsed time.
00500
00501 void set_elapsed_time(TimeInMillis elapsed) {
00502 elapsed_time_ = elapsed;
00503 }
00504
00505 // Adds a test property to the list. The property is validated and may add
00506 // a non-fatal failure if invalid (e.g., if it conflicts with reserved
00507 // key names). If a property is already recorded for the same key, the
00508 // value will be updated, rather than storing multiple values for the same
00509 // key. xml_element specifies the element for which the property is being
00510 // recorded and is used for validation.
00511 void RecordProperty(const std::string& xml_element,
00512 const TestProperty& test_property);
00513
00514 // Adds a failure if the key is a reserved attribute of Google Test
00515 // testsuite tags. Returns true if the property is valid.
00516 // FIXME: Validate attribute names are legal and human readable.
00517 static bool ValidateTestProperty(const std::string& xml_element,
00518 const TestProperty& test_property);
00519
00520 // Adds a test part result to the list.
00521 void AddTestPartResult(const TestPartResult& test_part_result);
00522
00523 // Returns the death test count.
00524
00525 int death_test_count() const {
00526 return death_test_count_;
00527 }
00528
00529 // Increments the death test count, returning the new count.
00530
00531 int increment_death_test_count() {
00532 return ++death_test_count_;
00533 }
00534
00535 // Clears the test part results.
00536 void ClearTestPartResults();
00537
00538 // Clears the object.
00539 void Clear();
00540
00541 // Protects mutable state of the property vector and of owned
00542 // properties, whose values may be updated.
00543 internal::Mutex test_properties_mutex_;
00544
00545 // The vector of TestPartResults
00546 std::vector<TestPartResult> test_part_results_;
00547 // The vector of TestProperties
00548 std::vector<TestProperty> test_properties_;
00549 // Running count of death tests.
00550 int death_test_count_;
00551 // The start time, in milliseconds since UNIX Epoch.
00552 TimeInMillis start_timestamp_;
00553 // The elapsed time, in milliseconds.
00554 TimeInMillis elapsed_time_;
00555
00556 // We disallow copying TestResult.
00557 TestResult(const TestResult&) = delete;
00558 TestResult& operator=(const TestResult&) = delete;
00559 }; // class TestResult
00560
00561 // A TestInfo object stores the following information about a test:
00562 //
00563 // Test suite name
00564 // Test name
00565 // Whether the test should be run
00566 // A function pointer that creates the test object when invoked
00567 // Test result
00568 //
00569 // The constructor of TestInfo registers itself with the UnitTest
00570 // singleton such that the RUN_ALL_TESTS() macro knows which tests to

```

```

00571 // run.
00572
00573 class GTEST_API_ TestInfo {
00574 public:
00575 // Destructs a TestInfo object. This function is not virtual, so
00576 // don't inherit from TestInfo.
00577 ~TestInfo();
00578
00579 // Returns the test suite name.
00580
00581 const char* test_suite_name() const {
00582 return test_suite_name_.c_str();
00583 }
00584
00585 // Legacy API is deprecated but still available
00586 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00587 const char* test_case_name() const {
00588 return test_suite_name();
00589 }
00590 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00591 // Returns the test name.
00592
00593 const char* name() const {
00594 return name_.c_str();
00595 }
00596
00597 // Returns the name of the parameter type, or NULL if this is not a typed
00598 // or a type-parameterized test.
00599
00600 const char* type_param() const {
00601 if (type_param_.get() != nullptr) return type_param_->c_str();
00602 return nullptr;
00603 }
00604
00605 // Returns the text representation of the value parameter, or NULL if this
00606 // is not a value-parameterized test.
00607
00608 const char* value_param() const {
00609 if (value_param_.get() != nullptr) return value_param_->c_str();
00610 return nullptr;
00611 }
00612
00613 // Returns the file name where this test is defined.
00614
00615 const char* file() const {
00616 return location_.file.c_str();
00617 }
00618
00619 // Returns the line where this test is defined.
00620
00621 int line() const {
00622 return location_.line;
00623 }
00624
00625 // Return true if this test should not be run because it's in another shard.
00626
00627 bool is_in_another_shard() const {
00628 return is_in_another_shard_;
00629 }
00630
00631 // Returns true if this test should run, that is if the test is not
00632 // disabled (or it is disabled but the also_run_disabled_tests flag has
00633 // been specified) and its full name matches the user-specified filter.
00634 //
00635 // Google Test allows the user to filter the tests by their full names.
00636 // The full name of a test Bar in test suite Foo is defined as
00637 // "Foo.Bar". Only the tests that match the filter will run.
00638 //
00639 // A filter is a colon-separated list of glob (not regex) patterns,
00640 // optionally followed by a '-' and a colon-separated list of
00641 // negative patterns (tests to exclude). A test is run if it
00642 // matches one of the positive patterns and does not match any of
00643 // the negative patterns.
00644 //
00645 // For example, *A*:Foo.* is a filter that matches any string that
00646 // contains the character 'A' or starts with "Foo.".
00647
00648 bool should_run() const {
00649 return should_run_;
00650 }
00651
00652 // Returns true if and only if this test will appear in the XML report.
00653
00654 bool is_reportable() const {
00655 // The XML report includes tests matching the filter, excluding those

```

```

00658 // run in other shards.
00659 return matches_filter_ && !is_in_another_shard_;
00660 }
00661
00662 // Returns the result of the test.
00663
00664 const TestResult* result() const {
00665 return &result_;
00666 }
00667
00668 private:
00669 #if GTEST_HAS_DEATH_TEST
00670 friend class internal::DefaultDeathTestFactory;
00671 #endif // GTEST_HAS_DEATH_TEST
00672 friend class Test;
00673 friend class TestSuite;
00674 friend class internal::UnitTestImpl;
00675 friend class internal::StreamingListenerTest;
00676 friend TestInfo* internal::MakeAndRegisterTestInfo(
00677 const char* test_suite_name, const char* name, const char* type_param,
00678 const char* value_param, internal::CodeLocation code_location,
00679 internal::TypeId fixture_class_id, internal::SetUpTestSuiteFunc set_up_tc,
00680 internal::TearDownTestSuiteFunc tear_down_tc,
00681 internal::TestFactoryBase* factory);
00682
00683 // Constructs a TestInfo object. The newly constructed instance assumes
00684 // ownership of the factory object.
00685 TestInfo(const std::string& test_suite_name, const std::string& name,
00686 const char* a_type_param, // NULL if not a type-parameterized test
00687 const char* a_value_param, // NULL if not a value-parameterized test
00688 internal::CodeLocation a_code_location,
00689 internal::TypeId fixture_class_id,
00690 internal::TestFactoryBase* factory);
00691
00692 // Increments the number of death tests encountered in this test so
00693 // far.
00694
00695 int increment_death_test_count() {
00696 return result_.increment_death_test_count();
00697 }
00698
00699 // Creates the test object, runs it, records its result, and then
00700 // deletes it.
00701 void Run();
00702
00703 // Skip and records the test result for this object.
00704 void Skip();
00705
00706 static void ClearTestResult(TestInfo* test_info) {
00707 test_info->result_.Clear();
00708 }
00709
00710 // These fields are immutable properties of the test.
00711 const std::string test_suite_name_; // test suite name
00712 const std::string name_; // Test name
00713 // Name of the parameter type, or NULL if this is not a typed or a
00714 // type-parameterized test.
00715 const std::unique_ptr<const ::std::string> type_param_;
00716 // Text representation of the value parameter, or NULL if this is not a
00717 // value-parameterized test.
00718 const std::unique_ptr<const ::std::string> value_param_;
00719 internal::CodeLocation location_;
00720 const internal::TypeId fixture_class_id_; // ID of the test fixture class
00721 bool should_run_; // True if and only if this test should run
00722 bool is_disabled_; // True if and only if this test is disabled
00723 bool matches_filter_; // True if this test matches the
00724 // user-specified filter.
00725 bool is_in_another_shard_; // Will be run in another shard.
00726 internal::TestFactoryBase* const factory_; // The factory that creates
00727 // the test object
00728
00729 // This field is mutable and needs to be reset before running the
00730 // test for the second time.
00731 TestResult result_;
00732
00733 TestInfo(const TestInfo&) = delete;
00734 TestInfo& operator=(const TestInfo&) = delete;
00735 };
00736
00737 // A test suite, which consists of a vector of TestInfos.
00738 //
00739 // TestSuite is not copyable.
00740
00741 class GTEST_API_ TestSuite {
00742 public:
00743 // Creates a TestSuite with the given name.
00744 //

```

```

00745 // TestSuite does NOT have a default constructor. Always use this
00746 // constructor to create a TestSuite object.
00747 //
00748 // Arguments:
00749 //
00750 // name: name of the test suite
00751 // a_type_param: the name of the test's type parameter, or NULL if
00752 // this is not a type-parameterized test.
00753 // set_up_tc: pointer to the function that sets up the test suite
00754 // tear_down_tc: pointer to the function that tears down the test suite
00755 TestSuite(const char* name, const char* a_type_param,
00756 internal::SetUpTestSuiteFunc set_up_tc,
00757 internal::TearDownTestSuiteFunc tear_down_tc);
00758
00759 // Destructor of TestSuite.
00760 virtual ~TestSuite();
00761
00762 // Gets the name of the TestSuite.
00763
00764 const char* name() const {
00765 return name_.c_str();
00766 }
00767
00768 // Returns the name of the parameter type, or NULL if this is not a
00769 // type-parameterized test suite.
00770
00771 const char* type_param() const {
00772 if (type_param_.get() != nullptr) return type_param_->c_str();
00773 return nullptr;
00774 }
00775
00776 // Returns true if any test in this test suite should run.
00777
00778 bool should_run() const {
00779 return should_run_;
00780 }
00781
00782 // Gets the number of successful tests in this test suite.
00783 int successful_test_count() const;
00784
00785 // Gets the number of skipped tests in this test suite.
00786 int skipped_test_count() const;
00787
00788 // Gets the number of failed tests in this test suite.
00789 int failed_test_count() const;
00790
00791 // Gets the number of disabled tests that will be reported in the XML report.
00792 int reportable_disabled_test_count() const;
00793
00794 // Gets the number of disabled tests in this test suite.
00795 int disabled_test_count() const;
00796
00797 // Gets the number of tests to be printed in the XML report.
00798 int reportable_test_count() const;
00799
00800 // Get the number of tests in this test suite that should run.
00801 int test_to_run_count() const;
00802
00803 // Gets the number of all tests in this test suite.
00804 int total_test_count() const;
00805
00806 // Returns true if and only if the test suite passed.
00807
00808 bool Passed() const {
00809 return !Failed();
00810 }
00811
00812 // Returns true if and only if the test suite failed.
00813
00814 bool Failed() const {
00815 return failed_test_count() > 0 || ad_hoc_test_result().Failed();
00816 }
00817
00818 // Returns the elapsed time, in milliseconds.
00819
00820 TimeInMillis elapsed_time() const {
00821 return elapsed_time_;
00822 }
00823
00824 // Gets the time of the test suite start, in ms from the start of the
00825 // UNIX epoch.
00826
00827 TimeInMillis start_timestamp() const {
00828 return start_timestamp_;
00829 }
00830
00831 // Returns the i-th test among all the tests. i can range from 0 to

```

```

00832 // total_test_count() - 1. If i is not in that range, returns NULL.
00833 const TestInfo* GetTestInfo(int i) const;
00834
00835 // Returns the TestResult that holds test properties recorded during
00836 // execution of SetUpTestSuite and TearDownTestSuite.
00837
00838 const TestResult& ad_hoc_test_result() const {
00839 return ad_hoc_test_result_;
00840 }
00841
00842 private:
00843 friend class Test;
00844 friend class internal::UnitTestImpl;
00845
00846 // Gets the (mutable) vector of TestInfos in this TestSuite.
00847
00848 std::vector<TestInfo*>& test_info_list() {
00849 return test_info_list_;
00850 }
00851
00852 // Gets the (immutable) vector of TestInfos in this TestSuite.
00853
00854 const std::vector<TestInfo*>& test_info_list() const {
00855 return test_info_list_;
00856 }
00857
00858 // Returns the i-th test among all the tests. i can range from 0 to
00859 // total_test_count() - 1. If i is not in that range, returns NULL.
00860 TestInfo* GetMutableTestInfo(int i);
00861
00862 // Sets the should_run member.
00863
00864 void set_should_run(bool should) {
00865 should_run_ = should;
00866 }
00867
00868 // Adds a TestInfo to this test suite. Will delete the TestInfo upon
00869 // destruction of the TestSuite object.
00870 void AddTestInfo(TestInfo* test_info);
00871
00872 // Clears the results of all tests in this test suite.
00873 void ClearResult();
00874
00875 // Clears the results of all tests in the given test suite.
00876
00877 static void ClearTestSuiteResult(TestSuite* test_suite) {
00878 test_suite->ClearResult();
00879 }
00880
00881 // Runs every test in this TestSuite.
00882 void Run();
00883
00884 // Skips the execution of tests under this TestSuite
00885 void Skip();
00886
00887 // Runs SetUpTestSuite() for this TestSuite. This wrapper is needed
00888 // for catching exceptions thrown from SetUpTestSuite().
00889
00890 void RunSetUpTestSuite() {
00891 if (set_up_tc_ != nullptr) {
00892 (*set_up_tc_)();
00893 }
00894 }
00895
00896 // Runs TearDownTestSuite() for this TestSuite. This wrapper is
00897 // needed for catching exceptions thrown from TearDownTestSuite().
00898
00899 void RunTearDownTestSuite() {
00900 if (tear_down_tc_ != nullptr) {
00901 (*tear_down_tc_)();
00902 }
00903 }
00904
00905 // Returns true if and only if test passed.
00906
00907 static bool TestPassed(const TestInfo* test_info) {
00908 return test_info->should_run() && test_info->result()->Passed();
00909 }
00910
00911 // Returns true if and only if test skipped.
00912
00913 static bool TestSkipped(const TestInfo* test_info) {
00914 return test_info->should_run() && test_info->result()->Skipped();
00915 }
00916
00917 // Returns true if and only if test failed.
00918

```



```

00919 static bool TestFailed(const TestInfo* test_info) {
00920 return test_info->should_run() && test_info->result()->Failed();
00921 }
00922
00923 // Returns true if and only if the test is disabled and will be reported in
00924 // the XML report.
00925
00926 static bool TestReportableDisabled(const TestInfo* test_info) {
00927 return test_info->is_reportable() && test_info->is_disabled_;
00928 }
00929
00930 // Returns true if and only if test is disabled.
00931
00932 static bool TestDisabled(const TestInfo* test_info) {
00933 return test_info->is_disabled_;
00934 }
00935
00936 // Returns true if and only if this test will appear in the XML report.
00937
00938 static bool TestReportable(const TestInfo* test_info) {
00939 return test_info->is_reportable();
00940 }
00941
00942 // Returns true if the given test should run.
00943
00944 static bool ShouldRunTest(const TestInfo* test_info) {
00945 return test_info->should_run();
00946 }
00947
00948 // Shuffles the tests in this test suite.
00949 void ShuffleTests(internal::Random* random);
00950
00951 // Restores the test order to before the first shuffle.
00952 void UnshuffleTests();
00953
00954 // Name of the test suite.
00955 std::string name_;
00956 // Name of the parameter type, or NULL if this is not a typed or a
00957 // type-parameterized test.
00958 const std::unique_ptr<const ::std::string> type_param_;
00959 // The vector of TestInfos in their original order. It owns the
00960 // elements in the vector.
00961 std::vector<TestInfo> test_info_list_;
00962 // Provides a level of indirection for the test list to allow easy
00963 // shuffling and restoring the test order. The i-th element in this
00964 // vector is the index of the i-th test in the shuffled test list.
00965 std::vector<int> test_indices_;
00966 // Pointer to the function that sets up the test suite.
00967 internal::SetUpTestSuiteFunc set_up_tc_;
00968 // Pointer to the function that tears down the test suite.
00969 internal::TearDownTestSuiteFunc tear_down_tc_;
00970 // True if and only if any test in this test suite should run.
00971 bool should_run_;
00972 // The start time, in milliseconds since UNIX Epoch.
00973 TimeInMillis start_timestamp_;
00974 // Elapsed time, in milliseconds.
00975 TimeInMillis elapsed_time_;
00976 // Holds test properties recorded during execution of SetUpTestSuite and
00977 // TearDownTestSuite.
00978 TestResult ad_hoc_test_result_;
00979
00980 // We disallow copying TestSuites.
00981 TestSuite(const TestSuite&) = delete;
00982 TestSuite& operator=(const TestSuite&) = delete;
00983 };
00984
00985 // An Environment object is capable of setting up and tearing down an
00986 // environment. You should subclass this to define your own
00987 // environment(s).
00988 //
00989 // An Environment object does the set-up and tear-down in virtual
00990 // methods SetUp() and TearDown() instead of the constructor and the
00991 // destructor, as:
00992 //
00993 // 1. You cannot safely throw from a destructor. This is a problem
00994 // as in some cases Google Test is used where exceptions are enabled, and
00995 // we may want to implement ASSERT_* using exceptions where they are
00996 // available.
00997 // 2. You cannot use ASSERT_* directly in a constructor or
00998 // destructor.
00999
01000 class Environment {
01001 public:
01002 // The d'tor is virtual as we need to subclass Environment.
01003
01004 virtual ~Environment() {
01005

```

```

01006
01007 // Override this to define how to set up the environment.
01008
01009 virtual void SetUp() {
01010 }
01011
01012 // Override this to define how to tear down the environment.
01013
01014 virtual void TearDown() {
01015 }
01016
01017 private:
01018 // If you see an error about overriding the following function or
01019 // about it being private, you have mis-spelled SetUp() as Setup().
01020
01021 struct Setup_should_be_spelled_SetUp {
01022 };
01023
01024 virtual Setup_should_be_spelled_SetUp* Setup() {
01025 return nullptr;
01026 }
01027 };
01028
01029 #if GTEST_HAS_EXCEPTIONS
01030
01031 // Exception which can be thrown from TestEventListener::OnTestPartResult.
01032
01033 class GTEST_API_ AssertionException
01034 : public internal::GoogleTestFailureException {
01035 public:
01036
01037 explicit AssertionException(const TestPartResult& result)
01038 : GoogleTestFailureException(result) {
01039 }
01040 };
01041
01042 #endif // GTEST_HAS_EXCEPTIONS
01043
01044 // The interface for tracing execution of tests. The methods are organized in
01045 // the order the corresponding events are fired.
01046
01047 class TestEventListener {
01048 public:
01049
01050 virtual ~TestEventListener() {
01051 }
01052
01053 // Fired before any test activity starts.
01054 virtual void OnTestProgramStart(const UnitTest& unit_test) = 0;
01055
01056 // Fired before each iteration of tests starts. There may be more than
01057 // one iteration if GTEST_FLAG(repeat) is set. iteration is the iteration
01058 // index, starting from 0.
01059 virtual void OnTestIterationStart(const UnitTest& unit_test,
01060 int iteration) = 0;
01061
01062 // Fired before environment set-up for each iteration of tests starts.
01063 virtual void OnEnvironmentsSetUpStart(const UnitTest& unit_test) = 0;
01064
01065 // Fired after environment set-up for each iteration of tests ends.
01066 virtual void OnEnvironmentsSetUpEnd(const UnitTest& unit_test) = 0;
01067
01068 // Fired before the test suite starts.
01069
01070 virtual void OnTestSuiteStart(const TestSuite& /*test_suite*/) {
01071 }
01072
01073 // Legacy API is deprecated but still available
01074 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
01075
01076 virtual void OnTestCaseStart(const TestCase& /*test_case*/) {
01077 }
01078 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
01079
01080 // Fired before the test starts.
01081 virtual void OnTestStart(const TestInfo& test_info) = 0;
01082
01083 // Fired when a test is disabled
01084
01085 virtual void OnTestDisabled(const TestInfo& /*test_info*/) {
01086 }
01087
01088 // Fired after a failed assertion or a SUCCEED() invocation.
01089 // If you want to throw an exception from this function to skip to the next
01090 // TEST, it must be AssertionException defined above, or inherited from it.
01091 virtual void OnTestPartResult(const TestPartResult& test_part_result) = 0;
01092

```

```

01093 // Fired after the test ends.
01094 virtual void OnTestEnd(const TestInfo& test_info) = 0;
01095
01096 // Fired after the test suite ends.
01097
01098 virtual void OnTestSuiteEnd(const TestSuite& /*test_suite*/) {
01099 }
01100
01101 // Legacy API is deprecated but still available
01102 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
01103
01104 virtual void OnTestCaseEnd(const TestCase& /*test_case*/) {
01105 }
01106 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
01107
01108 // Fired before environment tear-down for each iteration of tests starts.
01109 virtual void OnEnvironmentsTearDownStart(const UnitTest& unit_test) = 0;
01110
01111 // Fired after environment tear-down for each iteration of tests ends.
01112 virtual void OnEnvironmentsTearDownEnd(const UnitTest& unit_test) = 0;
01113
01114 // Fired after each iteration of tests finishes.
01115 virtual void OnTestIterationEnd(const UnitTest& unit_test, int iteration) = 0;
01116
01117 // Fired after all test activities have ended.
01118 virtual void OnTestProgramEnd(const UnitTest& unit_test) = 0;
01119 };
01120
01121 // The convenience class for users who need to override just one or two
01122 // methods and are not concerned that a possible change to a signature of
01123 // the methods they override will not be caught during the build. For
01124 // comments about each method please see the definition of TestEventListener
01125 // above.
01126
01127 class EmptyTestEventListener : public TestEventListener {
01128 public:
01129
01130 void OnTestProgramStart(const UnitTest& /*unit_test*/) override {
01131 }
01132
01133 void OnTestIterationStart(const UnitTest& /*unit_test*/,
01134 int /*iteration*/) override {
01135 }
01136
01137 void OnEnvironmentsSetUpStart(const UnitTest& /*unit_test*/) override {
01138 }
01139
01140 void OnEnvironmentsSetUpEnd(const UnitTest& /*unit_test*/) override {
01141 }
01142
01143 void OnTestSuiteStart(const TestSuite& /*test_suite*/) override {
01144 }
01145 // Legacy API is deprecated but still available
01146 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
01147
01148 void OnTestCaseStart(const TestCase& /*test_case*/) override {
01149 }
01150 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
01151
01152 void OnTestStart(const TestInfo& /*test_info*/) override {
01153 }
01154
01155 void OnTestDisabled(const TestInfo& /*test_info*/) override {
01156 }
01157
01158 void OnTestPartResult(const TestPartResult& /*test_part_result*/) override {
01159 }
01160
01161 void OnTestEnd(const TestInfo& /*test_info*/) override {
01162 }
01163
01164 void OnTestSuiteEnd(const TestSuite& /*test_suite*/) override {
01165 }
01166 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
01167
01168 void OnTestCaseEnd(const TestCase& /*test_case*/) override {
01169 }
01170 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
01171
01172 void OnEnvironmentsTearDownStart(const UnitTest& /*unit_test*/) override {
01173 }
01174
01175 void OnEnvironmentsTearDownEnd(const UnitTest& /*unit_test*/) override {
01176 }
01177
01178 void OnTestIterationEnd(const UnitTest& /*unit_test*/,
01179 int /*iteration*/) override {

```

```

01180 }
01181
01182 void OnTestProgramEnd(const UnitTest& /*unit_test*/ override {
01183 }
01184 };
01185
01186 // TestEventListeners lets users add listeners to track events in Google Test.
01187
01188 class GTEST_API_ TestEventListeners {
01189 public:
01190 TestEventListeners();
01191 ~TestEventListeners();
01192
01193 // Appends an event listener to the end of the list. Google Test assumes
01194 // the ownership of the listener (i.e. it will delete the listener when
01195 // the test program finishes).
01196 void Append(TestEventListener* listener);
01197
01198 // Removes the given event listener from the list and returns it. It then
01199 // becomes the caller's responsibility to delete the listener. Returns
01200 // NULL if the listener is not found in the list.
01201 TestEventListener* Release(TestEventListener* listener);
01202
01203 // Returns the standard listener responsible for the default console
01204 // output. Can be removed from the listeners list to shut down default
01205 // console output. Note that removing this object from the listener list
01206 // with Release transfers its ownership to the caller and makes this
01207 // function return NULL the next time.
01208
01209 TestEventListener* default_result_printer() const {
01210 return default_result_printer_;
01211 }
01212
01213 // Returns the standard listener responsible for the default XML output
01214 // controlled by the --gtest_output=xml flag. Can be removed from the
01215 // listeners list by users who want to shut down the default XML output
01216 // controlled by this flag and substitute it with custom one. Note that
01217 // removing this object from the listener list with Release transfers its
01218 // ownership to the caller and makes this function return NULL the next
01219 // time.
01220
01221 TestEventListener* default_xml_generator() const {
01222 return default_xml_generator_;
01223 }
01224
01225 private:
01226 friend class TestSuite;
01227 friend class TestInfo;
01228 friend class internal::DefaultGlobalTestPartResultReporter;
01229 friend class internal::NoExecDeathTest;
01230 friend class internal::TestEventListenersAccessor;
01231 friend class internal::UnitTestImpl;
01232
01233 // Returns repeater that broadcasts the TestEventListener events to all
01234 // subscribers.
01235 TestEventListener* repeater();
01236
01237 // Sets the default_result_printer attribute to the provided listener.
01238 // The listener is also added to the listener list and previous
01239 // default_result_printer is removed from it and deleted. The listener can
01240 // also be NULL in which case it will not be added to the list. Does
01241 // nothing if the previous and the current listener objects are the same.
01242 void SetDefaultResultPrinter(TestEventListener* listener);
01243
01244 // Sets the default_xml_generator attribute to the provided listener. The
01245 // listener is also added to the listener list and previous
01246 // default_xml_generator is removed from it and deleted. The listener can
01247 // also be NULL in which case it will not be added to the list. Does
01248 // nothing if the previous and the current listener objects are the same.
01249 void SetDefaultXmlGenerator(TestEventListener* listener);
01250
01251 // Controls whether events will be forwarded by the repeater to the
01252 // listeners in the list.
01253 bool EventForwardingEnabled() const;
01254 void SuppressEventForwarding();
01255
01256 // The actual list of listeners.
01257 internal::TestEventRepeater* repeater_;
01258 // Listener responsible for the standard result output.
01259 TestEventListener* default_result_printer_;
01260 // Listener responsible for the creation of the XML output file.
01261 TestEventListener* default_xml_generator_;
01262
01263 // We disallow copying TestEventListeners.
01264 TestEventListeners(const TestEventListeners&) = delete;
01265 TestEventListeners& operator=(const TestEventListeners&) = delete;
01266 };

```

```

01267
01268 // A UnitTest consists of a vector of TestSuites.
01269 //
01270 // This is a singleton class. The only instance of UnitTest is
01271 // created when UnitTest::GetInstance() is first called. This
01272 // instance is never deleted.
01273 //
01274 // UnitTest is not copyable.
01275 //
01276 // This class is thread-safe as long as the methods are called
01277 // according to their specification.
01278
01279 class GTEST_API_ UnitTest {
01280 public:
01281 // Gets the singleton UnitTest object. The first time this method
01282 // is called, a UnitTest object is constructed and returned.
01283 // Consecutive calls will return the same object.
01284 static UnitTest* GetInstance();
01285
01286 // Runs all tests in this UnitTest object and prints the result.
01287 // Returns 0 if successful, or 1 otherwise.
01288 //
01289 // This method can only be called from the main thread.
01290 //
01291 // INTERNAL IMPLEMENTATION - DO NOT USE IN A USER PROGRAM.
01292 int Run() GTEST_MUST_USE_RESULT_;
01293
01294 // Returns the working directory when the first TEST() or TEST_F()
01295 // was executed. The UnitTest object owns the string.
01296 const char* original_working_dir() const;
01297
01298 // Returns the TestSuite object for the test that's currently running,
01299 // or NULL if no test is running.
01300 const TestSuite* current_test_suite() const GTEST_LOCK_EXCLUDED_(mutex_);
01301
01302 // Legacy API is still available but deprecated
01303 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
01304 const TestCase* current_test_case() const GTEST_LOCK_EXCLUDED_(mutex_);
01305 #endif
01306
01307 // Returns the TestInfo object for the test that's currently running,
01308 // or NULL if no test is running.
01309 const TestInfo* current_test_info() const GTEST_LOCK_EXCLUDED_(mutex_);
01310
01311 // Returns the random seed used at the start of the current test run.
01312 int random_seed() const;
01313
01314 // Returns the ParameterizedTestSuiteRegistry object used to keep track of
01315 // value-parameterized tests and instantiate and register them.
01316 //
01317 // INTERNAL IMPLEMENTATION - DO NOT USE IN A USER PROGRAM.
01318 internal::ParameterizedTestSuiteRegistry& parameterized_test_registry()
01319 GTEST_LOCK_EXCLUDED_(mutex_);
01320
01321 // Gets the number of successful test suites.
01322 int successful_test_suite_count() const;
01323
01324 // Gets the number of failed test suites.
01325 int failed_test_suite_count() const;
01326
01327 // Gets the number of all test suites.
01328 int total_test_suite_count() const;
01329
01330 // Gets the number of all test suites that contain at least one test
01331 // that should run.
01332 int test_suite_to_run_count() const;
01333
01334 // Legacy API is deprecated but still available
01335 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
01336 int successful_test_case_count() const;
01337 int failed_test_case_count() const;
01338 int total_test_case_count() const;
01339 int test_case_to_run_count() const;
01340 #endif
01341
01342 // Gets the number of successful tests.
01343 int successful_test_count() const;
01344
01345 // Gets the number of skipped tests.
01346 int skipped_test_count() const;
01347
01348 // Gets the number of failed tests.
01349 int failed_test_count() const;
01350
01351 // Gets the number of disabled tests that will be reported in the XML report.
01352 int reportable_disabled_test_count() const;
01353

```

```

01354 // Gets the number of disabled tests.
01355 int disabled_test_count() const;
01356
01357 // Gets the number of tests to be printed in the XML report.
01358 int reportable_test_count() const;
01359
01360 // Gets the number of all tests.
01361 int total_test_count() const;
01362
01363 // Gets the number of tests that should run.
01364 int test_to_run_count() const;
01365
01366 // Gets the time of the test program start, in ms from the start of the
01367 // UNIX epoch.
01368 TimeInMillis start_timestamp() const;
01369
01370 // Gets the elapsed time, in milliseconds.
01371 TimeInMillis elapsed_time() const;
01372
01373 // Returns true if and only if the unit test passed (i.e. all test suites
01374 // passed).
01375 bool Passed() const;
01376
01377 // Returns true if and only if the unit test failed (i.e. some test suite
01378 // failed or something outside of all tests failed).
01379 bool Failed() const;
01380
01381 // Gets the i-th test suite among all the test suites. i can range from 0 to
01382 // total_test_suite_count() - 1. If i is not in that range, returns NULL.
01383 const TestSuite* GetTestSuite(int i) const;
01384
01385 // Legacy API is deprecated but still available
01386 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
01387 const TestCase* GetTestCase(int i) const;
01388 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
01389
01390 // Returns the TestResult containing information on test failures and
01391 // properties logged outside of individual test suites.
01392 const TestResult& ad_hoc_test_result() const;
01393
01394 // Returns the list of event listeners that can be used to track events
01395 // inside Google Test.
01396 TestEventListeners& listeners();
01397
01398 private:
01399 // Registers and returns a global test environment. When a test
01400 // program is run, all global test environments will be set-up in
01401 // the order they were registered. After all tests in the program
01402 // have finished, all global test environments will be torn-down in
01403 // the *reverse* order they were registered.
01404 //
01405 // The UnitTest object takes ownership of the given environment.
01406 //
01407 // This method can only be called from the main thread.
01408 Environment* AddEnvironment(Environment* env);
01409
01410 // Adds a TestPartResult to the current TestResult object. All
01411 // Google Test assertion macros (e.g. ASSERT_TRUE, EXPECT_EQ, etc)
01412 // eventually call this to report their results. The user code
01413 // should use the assertion macros instead of calling this directly.
01414 void AddTestPartResult(TestPartResult::Type result_type,
01415 const char* file_name, int line_number,
01416 const std::string& message,
01417 const std::string& os_stack_trace)
01418 GTEST_LOCK_EXCLUDED_(mutex_);
01419
01420 // Adds a TestProperty to the current TestResult object when invoked from
01421 // inside a test, to current TestSuite's ad_hoc_test_result_ when invoked
01422 // from SetUpTestSuite or TearDownTestSuite, or to the global property set
01423 // when invoked elsewhere. If the result already contains a property with
01424 // the same key, the value will be updated.
01425 void RecordProperty(const std::string& key, const std::string& value);
01426
01427 // Gets the i-th test suite among all the test suites. i can range from 0 to
01428 // total_test_suite_count() - 1. If i is not in that range, returns NULL.
01429 TestSuite* GetMutableTestSuite(int i);
01430
01431 // Accessors for the implementation object.
01432
01433 internal::UnitTestImpl* impl() {
01434 return impl_;
01435 }
01436
01437 const internal::UnitTestImpl* impl() const {
01438 return impl_;
01439 }
01440

```

```

01441 // These classes and functions are friends as they need to access private
01442 // members of UnitTest.
01443 friend class ScopedTrace;
01444 friend class Test;
01445 friend class internal::AssertHelper;
01446 friend class internal::StreamingListenerTest;
01447 friend class internal::UnitTestRecordPropertyTestHelper;
01448 friend Environment* AddGlobalTestEnvironment(Environment* env);
01449 friend std::set<std::string>* internal::GetIgnoredParameterizedTestSuites();
01450 friend internal::UnitTestImpl* internal::GetUnitTestImpl();
01451 friend void internal::ReportFailureInUnknownLocation(
01452 TestPartResult::Type result_type, const std::string& message);
01453
01454 // Creates an empty UnitTest.
01455 UnitTest();
01456
01457 // D'tor
01458 virtual ~UnitTest();
01459
01460 // Pushes a trace defined by SCOPED_TRACE() on to the per-thread
01461 // Google Test trace stack.
01462 void PushGTestTrace(const internal::TraceInfo& trace)
01463 GTEST_LOCK_EXCLUDED_(mutex_);
01464
01465 // Pops a trace from the per-thread Google Test trace stack.
01466 void PopGTestTrace() GTEST_LOCK_EXCLUDED_(mutex_);
01467
01468 // Protects mutable state in *impl_. This is mutable as some const
01469 // methods need to lock it too.
01470 mutable internal::Mutex mutex_;
01471
01472 // Opaque implementation object. This field is never changed once
01473 // the object is constructed. We don't mark it as const here, as
01474 // doing so will cause a warning in the constructor of UnitTest.
01475 // Mutable state in *impl_ is protected by mutex_.
01476 internal::UnitTestImpl* impl_;
01477
01478 // We disallow copying UnitTest.
01479 UnitTest(const UnitTest&) = delete;
01480 UnitTest& operator=(const UnitTest&) = delete;
01481 };
01482
01483 // A convenient wrapper for adding an environment for the test
01484 // program.
01485 //
01486 // You should call this before RUN_ALL_TESTS() is called, probably in
01487 // main(). If you use gtest_main, you need to call this before main()
01488 // starts for it to take effect. For example, you can define a global
01489 // variable like this:
01490 //
01491 // testing::Environment* const foo_env =
01492 // testing::AddGlobalTestEnvironment(new FooEnvironment);
01493 //
01494 // However, we strongly recommend you to write your own main() and
01495 // call AddGlobalTestEnvironment() there, as relying on initialization
01496 // of global variables makes the code harder to read and may cause
01497 // problems when you register multiple environments from different
01498 // translation units and the environments have dependencies among them
01499 // (remember that the compiler doesn't guarantee the order in which
01500 // global variables from different translation units are initialized).
01501
01502 inline Environment* AddGlobalTestEnvironment(Environment* env) {
01503 return UnitTest::GetInstance()->AddEnvironment(env);
01504 }
01505
01506 // Initializes Google Test. This must be called before calling
01507 // RUN_ALL_TESTS(). In particular, it parses a command line for the
01508 // flags that Google Test recognizes. Whenever a Google Test flag is
01509 // seen, it is removed from argv, and *argc is decremented.
01510 //
01511 // No value is returned. Instead, the Google Test flag variables are
01512 // updated.
01513 //
01514 // Calling the function for the second time has no user-visible effect.
01515 GTEST_API_ void InitGoogleTest(int* argc, char** argv);
01516
01517 // This overloaded version can be used in Windows programs compiled in
01518 // UNICODE mode.
01519 GTEST_API_ void InitGoogleTest(int* argc, wchar_t** argv);
01520
01521 // This overloaded version can be used on Arduino/embedded platforms where
01522 // there is no argc/argv.
01523 GTEST_API_ void InitGoogleTest();
01524
01525 namespace internal {
01526
01527 // Separate the error generating code from the code path to reduce the stack

```

```

01528 // frame size of CmpHelperEQ. This helps reduce the overhead of some sanitizers
01529 // when calling EXPECT_* in a tight loop.
01530
01531 template <typename T1, typename T2>
01532 AssertionResult CmpHelperEQFailure(const char* lhs_expression,
01533 const char* rhs_expression, const T1& lhs,
01534 const T2& rhs) {
01535 return EqFailure(lhs_expression, rhs_expression,
01536 FormatForComparisonFailureMessage(lhs, rhs),
01537 FormatForComparisonFailureMessage(rhs, lhs), false);
01538 }
01539
01540 // This block of code defines operator==/!=
01541 // to block lexical scope lookup.
01542 // It prevents using invalid operator==/!= defined at namespace scope.
01543
01544 struct faketype {
01545 };
01546
01547 inline bool operator==(faketype, faketype) {
01548 return true;
01549 }
01550
01551 inline bool operator!=(faketype, faketype) {
01552 return false;
01553 }
01554
01555 // The helper function for {ASSERT|EXPECT}_EQ.
01556
01557 template <typename T1, typename T2>
01558 AssertionResult CmpHelperEQ(const char* lhs_expression,
01559 const char* rhs_expression, const T1& lhs,
01560 const T2& rhs) {
01561 if (lhs == rhs) {
01562 return AssertionSuccess();
01563 }
01564
01565 return CmpHelperEQFailure(lhs_expression, rhs_expression, lhs, rhs);
01566 }
01567
01568 class EqHelper {
01569 public:
01570 // This templated version is for the general case.
01571
01572 template <
01573 typename T1, typename T2,
01574 // Disable this overload for cases where one argument is a pointer
01575 // and the other is the null pointer constant.
01576 typename std::enable_if<!std::is_integral<T1>::value ||
01577 !std::is_pointer<T2>::value>::type* = nullptr>
01578 static AssertionResult Compare(const char* lhs_expression,
01579 const char* rhs_expression, const T1& lhs,
01580 const T2& rhs) {
01581 return CmpHelperEQ(lhs_expression, rhs_expression, lhs, rhs);
01582 }
01583
01584 // With this overloaded version, we allow anonymous enums to be used
01585 // in {ASSERT|EXPECT}_EQ when compiled with gcc 4, as anonymous
01586 // enums can be implicitly cast to BiggestInt.
01587 //
01588 // Even though its body looks the same as the above version, we
01589 // cannot merge the two, as it will make anonymous enums unhappy.
01590
01591 static AssertionResult Compare(const char* lhs_expression,
01592 const char* rhs_expression, BiggestInt lhs,
01593 BiggestInt rhs) {
01594 return CmpHelperEQ(lhs_expression, rhs_expression, lhs, rhs);
01595 }
01596
01597 template <typename T>
01598 static AssertionResult Compare(
01599 const char* lhs_expression, const char* rhs_expression,
01600 // Handle cases where '0' is used as a null pointer literal.
01601 std::nullptr_t /* lhs */, T* rhs) {
01602 // We already know that 'lhs' is a null pointer.
01603 return CmpHelperEQ(lhs_expression, rhs_expression, static_cast<T*>(nullptr),
01604 rhs);
01605 }
01606 };
01607
01608 // Separate the error generating code from the code path to reduce the stack
01609 // frame size of CmpHelperOP. This helps reduce the overhead of some sanitizers
01610 // when calling EXPECT_OP in a tight loop.
01611
01612 template <typename T1, typename T2>
01613 AssertionResult CmpHelperOpFailure(const char* expr1, const char* expr2,
01614 const T1& val1, const T2& val2,

```



```

01615 const char* op) {
01616 return AssertionFailure()
01617 « "Expected: (" « expr1 « ") " « op « " (" « expr2
01618 « "), actual: " « FormatForComparisonFailureMessage(val1, val2)
01619 « " vs " « FormatForComparisonFailureMessage(val2, val1);
01620 }
01621
01622 // A macro for implementing the helper functions needed to implement
01623 // ASSERT_?? and EXPECT_??. It is here just to avoid copy-and-paste
01624 // of similar code.
01625 //
01626 // INTERNAL IMPLEMENTATION - DO NOT USE IN A USER PROGRAM.
01627
01628 #define GTEST_IMPL_CMP_HELPER_(op_name, op)
01629 template <typename T1, typename T2>
01630 AssertionResult CmpHelper##op_name(const char* expr1, const char* expr2,
01631 const T1& val1, const T2& val2) {
01632 if (val1 op val2) {
01633 return AssertionSuccess();
01634 } else {
01635 return CmpHelperOpFailure(expr1, expr2, val1, val2, #op);
01636 }
01637 }
01638
01639 // INTERNAL IMPLEMENTATION - DO NOT USE IN A USER PROGRAM.
01640
01641 // Implements the helper function for {ASSERT|EXPECT}_NE
01642 GTEST_IMPL_CMP_HELPER_(NE, !=)
01643 // Implements the helper function for {ASSERT|EXPECT}_LE
01644 GTEST_IMPL_CMP_HELPER_(LE, <=)
01645 // Implements the helper function for {ASSERT|EXPECT}_LT
01646 GTEST_IMPL_CMP_HELPER_(LT, <)
01647 // Implements the helper function for {ASSERT|EXPECT}_GE
01648 GTEST_IMPL_CMP_HELPER_(GE, >=)
01649 // Implements the helper function for {ASSERT|EXPECT}_GT
01650 GTEST_IMPL_CMP_HELPER_(GT, >)
01651
01652 #undef GTEST_IMPL_CMP_HELPER_
01653
01654 // The helper function for {ASSERT|EXPECT}_STREQ.
01655 //
01656 // INTERNAL IMPLEMENTATION - DO NOT USE IN A USER PROGRAM.
01657 GTEST_API_ AssertionResult CmpHelperSTREQ(const char* s1_expression,
01658 const char* s2_expression,
01659 const char* s1, const char* s2);
01660
01661 // The helper function for {ASSERT|EXPECT}_STRCASEEQ.
01662 //
01663 // INTERNAL IMPLEMENTATION - DO NOT USE IN A USER PROGRAM.
01664 GTEST_API_ AssertionResult CmpHelperSTRCASEEQ(const char* s1_expression,
01665 const char* s2_expression,
01666 const char* s1, const char* s2);
01667
01668 // The helper function for {ASSERT|EXPECT}_STRNE.
01669 //
01670 // INTERNAL IMPLEMENTATION - DO NOT USE IN A USER PROGRAM.
01671 GTEST_API_ AssertionResult CmpHelperSTRNE(const char* s1_expression,
01672 const char* s2_expression,
01673 const char* s1, const char* s2);
01674
01675 // The helper function for {ASSERT|EXPECT}_STRCASENE.
01676 //
01677 // INTERNAL IMPLEMENTATION - DO NOT USE IN A USER PROGRAM.
01678 GTEST_API_ AssertionResult CmpHelperSTRCASENE(const char* s1_expression,
01679 const char* s2_expression,
01680 const char* s1, const char* s2);
01681
01682 // Helper function for *_STREQ on wide strings.
01683 //
01684 // INTERNAL IMPLEMENTATION - DO NOT USE IN A USER PROGRAM.
01685 GTEST_API_ AssertionResult CmpHelperWSTREQ(const char* s1_expression,
01686 const char* s2_expression,
01687 const wchar_t* s1, const wchar_t* s2);
01688
01689 // Helper function for *_STRNE on wide strings.
01690 //
01691 // INTERNAL IMPLEMENTATION - DO NOT USE IN A USER PROGRAM.
01692 GTEST_API_ AssertionResult CmpHelperWSTRNE(const char* s1_expression,
01693 const char* s2_expression,
01694 const wchar_t* s1, const wchar_t* s2);
01695
01696 } // namespace internal
01697
01698 // IsSubstring() and IsNotSubstring() are intended to be used as the
01699 // first argument to {EXPECT,ASSERT}_PRED_FORMAT2(), not by
01700 // themselves. They check whether needle is a substring of haystack
01701 // (NULL is considered a substring of itself only), and return an

```

```

01702 // appropriate error message when they fail.
01703 //
01704 // The {needle, haystack}_expr arguments are the stringified
01705 // expressions that generated the two real arguments.
01706 GTEST_API_ AssertionResult IsSubstring(const char* needle_expr,
01707 const char* haystack_expr,
01708 const char* needle,
01709 const char* haystack);
01710 GTEST_API_ AssertionResult IsSubstring(const char* needle_expr,
01711 const char* haystack_expr,
01712 const wchar_t* needle,
01713 const wchar_t* haystack);
01714 GTEST_API_ AssertionResult IsNotSubstring(const char* needle_expr,
01715 const char* haystack_expr,
01716 const char* needle,
01717 const char* haystack);
01718 GTEST_API_ AssertionResult IsNotSubstring(const char* needle_expr,
01719 const char* haystack_expr,
01720 const wchar_t* needle,
01721 const wchar_t* haystack);
01722 GTEST_API_ AssertionResult IsSubstring(const char* needle_expr,
01723 const char* haystack_expr,
01724 const ::std::string& needle,
01725 const ::std::string& haystack);
01726 GTEST_API_ AssertionResult IsNotSubstring(const char* needle_expr,
01727 const char* haystack_expr,
01728 const ::std::string& needle,
01729 const ::std::string& haystack);
01730
01731 #if GTEST_HAS_STD_WSTRING
01732 GTEST_API_ AssertionResult IsSubstring(const char* needle_expr,
01733 const char* haystack_expr,
01734 const ::std::wstring& needle,
01735 const ::std::wstring& haystack);
01736 GTEST_API_ AssertionResult IsNotSubstring(const char* needle_expr,
01737 const char* haystack_expr,
01738 const ::std::wstring& needle,
01739 const ::std::wstring& haystack);
01740 #endif // GTEST_HAS_STD_WSTRING
01741
01742 namespace internal {
01743
01744 // Helper template function for comparing floating-points.
01745 //
01746 // Template parameter:
01747 // RawType: the raw floating-point type (either float or double)
01748 //
01749 // INTERNAL IMPLEMENTATION - DO NOT USE IN A USER PROGRAM.
01750
01751 template <typename RawType>
01752 AssertionResult CmpHelperFloatingPointEQ(const char* lhs_expression,
01753 const char* rhs_expression,
01754 RawType lhs_value, RawType rhs_value) {
01755 const FloatingPoint<RawType> lhs(lhs_value), rhs(rhs_value);
01756
01757 if (lhs.AlmostEquals(rhs)) {
01758 return AssertionSuccess();
01759 }
01760
01761 ::std::stringstream lhs_ss;
01762 lhs_ss << std::setprecision(std::numeric_limits<RawType>::digits10 + 2)
01763 << lhs_value;
01764
01765 ::std::stringstream rhs_ss;
01766 rhs_ss << std::setprecision(std::numeric_limits<RawType>::digits10 + 2)
01767 << rhs_value;
01768
01769 return EqFailure(lhs_expression, rhs_expression,
01770 StringStreamToString(&lhs_ss), StringStreamToString(&rhs_ss),
01771 false);
01772 }
01773
01774 // Helper function for implementing ASSERT_NEAR.
01775 //
01776 // INTERNAL IMPLEMENTATION - DO NOT USE IN A USER PROGRAM.
01777 GTEST_API_ AssertionResult DoubleNearPredFormat(const char* expr1,
01778 const char* expr2,
01779 const char* abs_error_expr,
01780 double val1, double val2,
01781 double abs_error);
01782
01783 // INTERNAL IMPLEMENTATION - DO NOT USE IN USER CODE.
01784 // A class that enables one to stream messages to assertion macros
01785
01786 class GTEST_API_ AssertHelper {
01787 public:

```

```

01789 // Constructor.
01790 AssertHelper(TestPartResult::Type type, const char* file, int line,
01791 const char* message);
01792 ~AssertHelper();
01793
01794 // Message assignment is a semantic trick to enable assertion
01795 // streaming; see the GTEST_MESSAGE_ macro below.
01796 void operator=(const Message& message) const;
01797
01798 private:
01799 // We put our data in a struct so that the size of the AssertHelper class can
01800 // be as small as possible. This is important because gcc is incapable of
01801 // re-using stack space even for temporary variables, so every EXPECT_EQ
01802 // reserves stack space for another AssertHelper.
01803
01804 struct AssertHelperData {
01805
01806 AssertHelperData(TestPartResult::Type t, const char* srcfile, int line_num,
01807 const char* msg)
01808 : type(t), file(srcfile), line(line_num), message(msg) {}
01809
01810 TestPartResult::Type const type;
01811 const char* const file;
01812 int const line;
01813 std::string const message;
01814
01815 private:
01816 AssertHelperData(const AssertHelperData&) = delete;
01817 AssertHelperData& operator=(const AssertHelperData&) = delete;
01818 };
01819
01820 AssertHelperData * const data_;
01821
01822 AssertHelper(const AssertHelper&) = delete;
01823 AssertHelper& operator=(const AssertHelper&) = delete;
01824 };
01825
01826 } // namespace internal
01827
01828 // The pure interface class that all value-parameterized tests inherit from.
01829 // A value-parameterized class must inherit from both ::testing::Test and
01830 // ::testing::WithParamInterface. In most cases that just means inheriting
01831 // from ::testing::TestWithParam, but more complicated test hierarchies
01832 // may need to inherit from Test and WithParamInterface at different levels.
01833 //
01834 // This interface has support for accessing the test parameter value via
01835 // the GetParam() method.
01836 //
01837 // Use it with one of the parameter generator defining functions, like Range(),
01838 // Values(), ValuesIn(), Bool(), and Combine().
01839 //
01840 // class FooTest : public ::testing::TestWithParam<int> {
01841 // protected:
01842 // FooTest() {
01843 // // Can use GetParam() here.
01844 // }
01845 // ~FooTest() override {
01846 // // Can use GetParam() here.
01847 // }
01848 // void SetUp() override {
01849 // // Can use GetParam() here.
01850 // }
01851 // void TearDown() override {
01852 // // Can use GetParam() here.
01853 // }
01854 // };
01855 // TEST_P(FooTest, DoesBar) {
01856 // // Can use GetParam() method here.
01857 // Foo foo;
01858 // ASSERT_TRUE(foo.DoesBar(GetParam()));
01859 // }
01860 // INSTANTIATE_TEST_SUITE_P(OneToTenRange, FooTest, ::testing::Range(1, 10));
01861
01862 template <typename T>
01863 class WithParamInterface {
01864 public:
01865 typedef T ParamType;
01866
01867 virtual ~WithParamInterface() {}
01868
01869 // The current parameter value. Is also available in the test fixture's
01870 // constructor.
01871
01872 static const ParamType& GetParam() {
01873 GTEST_CHECK_(parameter_ != nullptr)

```

```

01876 « "GetParam() can only be called inside a value-parameterized test "
01877 « "-- did you intend to write TEST_P instead of TEST_F?";
01878 return *parameter_;
01879 }
01880
01881 private:
01882 // Sets parameter value. The caller is responsible for making sure the value
01883 // remains alive and unchanged throughout the current test.
01884
01885 static void SetParam(const ParamType* parameter) {
01886 parameter_ = parameter;
01887 }
01888
01889 // Static value used for accessing parameter during a test lifetime.
01890 static const ParamType* parameter_;
01891
01892 // TestClass must be a subclass of WithParamInterface<T> and Test.
01893 template <class TestClass>
01894 friend class internal::ParameterizedTestFactory;
01895 };
01896
01897 template <typename T>
01898 const T* WithParamInterface<T>::parameter_ = nullptr;
01899
01900 // Most value-parameterized classes can ignore the existence of
01901 // WithParamInterface, and can just inherit from ::testing::TestWithParam.
01902
01903 template <typename T>
01904 class TestWithParam : public Test, public WithParamInterface<T> {
01905 };
01906
01907 // Macros for indicating success/failure in test code.
01908
01909 // Skips test in runtime.
01910 // Skipping test aborts current function.
01911 // Skipped tests are neither successful nor failed.
01912 #define GTEST_SKIP() GTEST_SKIP_("")
01913
01914 // ADD_FAILURE unconditionally adds a failure to the current test.
01915 // SUCCEED generates a success - it doesn't automatically make the
01916 // current test successful, as a test is only successful when it has
01917 // no failure.
01918 //
01919 // EXPECT_* verifies that a certain condition is satisfied. If not,
01920 // it behaves like ADD_FAILURE. In particular:
01921 //
01922 // EXPECT_TRUE verifies that a Boolean condition is true.
01923 // EXPECT_FALSE verifies that a Boolean condition is false.
01924 //
01925 // FAIL and ASSERT_* are similar to ADD_FAILURE and EXPECT_*, except
01926 // that they will also abort the current function on failure. People
01927 // usually want the fail-fast behavior of FAIL and ASSERT_*, but those
01928 // writing data-driven tests often find themselves using ADD_FAILURE
01929 // and EXPECT_* more.
01930
01931 // Generates a nonfatal failure with a generic message.
01932 #define ADD_FAILURE() GTEST_NONFATAL_FAILURE_("Failed")
01933
01934 // Generates a nonfatal failure at the given source file location with
01935 // a generic message.
01936 #define ADD_FAILURE_AT(file, line) \
01937 GTEST_MESSAGE_AT_(file, line, "Failed", \
01938 ::testing::TestPartResult::kNonFatalFailure)
01939
01940 // Generates a fatal failure with a generic message.
01941 #define GTEST_FAIL() GTEST_FATAL_FAILURE_("Failed")
01942
01943 // Like GTEST_FAIL(), but at the given source file location.
01944 #define GTEST_FAIL_AT(file, line) \
01945 GTEST_MESSAGE_AT_(file, line, "Failed", \
01946 ::testing::TestPartResult::kFatalFailure)
01947
01948 // Define this macro to 1 to omit the definition of FAIL(), which is a
01949 // generic name and clashes with some other libraries.
01950 #if !GTEST_DONT_DEFINE_FAIL
01951 #define FAIL() GTEST_FAIL()
01952 #endif
01953
01954 // Generates a success with a generic message.
01955 #define GTEST_SUCCEED() GTEST_SUCCESS_("Succeeded")
01956
01957 // Define this macro to 1 to omit the definition of SUCCEED(), which
01958 // is a generic name and clashes with some other libraries.
01959 #if !GTEST_DONT_DEFINE_SUCCEED
01960 #define SUCCEED() GTEST_SUCCEED()
01961 #endif
01962

```

```

01963 // Macros for testing exceptions.
01964 //
01965 // * {ASSERT|EXPECT}_THROW(statement, expected_exception):
01966 // Tests that the statement throws the expected exception.
01967 // * {ASSERT|EXPECT}_NO_THROW(statement):
01968 // Tests that the statement doesn't throw any exception.
01969 // * {ASSERT|EXPECT}_ANY_THROW(statement):
01970 // Tests that the statement throws an exception.
01971
01972 #define EXPECT_THROW(statement, expected_exception) \
01973 GTEST_TEST_THROW(statement, expected_exception, GTEST_NONFATAL_FAILURE_)
01974 #define EXPECT_NO_THROW(statement) \
01975 GTEST_TEST_NO_THROW(statement, GTEST_NONFATAL_FAILURE_)
01976 #define EXPECT_ANY_THROW(statement) \
01977 GTEST_TEST_ANY_THROW(statement, GTEST_NONFATAL_FAILURE_)
01978 #define ASSERT_THROW(statement, expected_exception) \
01979 GTEST_TEST_THROW(statement, expected_exception, GTEST_FATAL_FAILURE_)
01980 #define ASSERT_NO_THROW(statement) \
01981 GTEST_TEST_NO_THROW(statement, GTEST_FATAL_FAILURE_)
01982 #define ASSERT_ANY_THROW(statement) \
01983 GTEST_TEST_ANY_THROW(statement, GTEST_FATAL_FAILURE_)
01984
01985 // Boolean assertions. Condition can be either a Boolean expression or an
01986 // AssertionResult. For more information on how to use AssertionResult with
01987 // these macros see comments on that class.
01988 #define GTEST_EXPECT_TRUE(condition) \
01989 GTEST_TEST_BOOLEAN_(condition, #condition, false, true, \
01990 GTEST_NONFATAL_FAILURE_)
01991 #define GTEST_EXPECT_FALSE(condition) \
01992 GTEST_TEST_BOOLEAN_(!(condition), #condition, true, false, \
01993 GTEST_NONFATAL_FAILURE_)
01994 #define GTEST_ASSERT_TRUE(condition) \
01995 GTEST_TEST_BOOLEAN_(condition, #condition, false, true, GTEST_FATAL_FAILURE_)
01996 #define GTEST_ASSERT_FALSE(condition) \
01997 GTEST_TEST_BOOLEAN_(!(condition), #condition, true, false, \
01998 GTEST_FATAL_FAILURE_)
01999
02000 // Define these macros to 1 to omit the definition of the corresponding
02001 // EXPECT or ASSERT, which clashes with some users' own code.
02002
02003 #if !GTEST_DONT_DEFINE_EXPECT_TRUE
02004 #define EXPECT_TRUE(condition) GTEST_EXPECT_TRUE(condition)
02005 #endif
02006
02007 #if !GTEST_DONT_DEFINE_EXPECT_FALSE
02008 #define EXPECT_FALSE(condition) GTEST_EXPECT_FALSE(condition)
02009 #endif
02010
02011 #if !GTEST_DONT_DEFINE_ASSERT_TRUE
02012 #define ASSERT_TRUE(condition) GTEST_ASSERT_TRUE(condition)
02013 #endif
02014
02015 #if !GTEST_DONT_DEFINE_ASSERT_FALSE
02016 #define ASSERT_FALSE(condition) GTEST_ASSERT_FALSE(condition)
02017 #endif
02018
02019 // Macros for testing equalities and inequalities.
02020 //
02021 // * {ASSERT|EXPECT}_EQ(v1, v2): Tests that v1 == v2
02022 // * {ASSERT|EXPECT}_NE(v1, v2): Tests that v1 != v2
02023 // * {ASSERT|EXPECT}_LT(v1, v2): Tests that v1 < v2
02024 // * {ASSERT|EXPECT}_LE(v1, v2): Tests that v1 <= v2
02025 // * {ASSERT|EXPECT}_GT(v1, v2): Tests that v1 > v2
02026 // * {ASSERT|EXPECT}_GE(v1, v2): Tests that v1 >= v2
02027 //
02028 // When they are not, Google Test prints both the tested expressions and
02029 // their actual values. The values must be compatible built-in types,
02030 // or you will get a compiler error. By "compatible" we mean that the
02031 // values can be compared by the respective operator.
02032 //
02033 // Note:
02034 //
02035 // 1. It is possible to make a user-defined type work with
02036 // {ASSERT|EXPECT}_??(), but that requires overloading the
02037 // comparison operators and is thus discouraged by the Google C++
02038 // Usage Guide. Therefore, you are advised to use the
02039 // {ASSERT|EXPECT}_TRUE() macro to assert that two objects are
02040 // equal.
02041 //
02042 // 2. The {ASSERT|EXPECT}_??() macros do pointer comparisons on
02043 // pointers (in particular, C strings). Therefore, if you use it
02044 // with two C strings, you are testing how their locations in memory
02045 // are related, not how their content is related. To compare two C
02046 // strings by content, use {ASSERT|EXPECT}_STR*().
02047 //
02048 // 3. {ASSERT|EXPECT}_EQ(v1, v2) is preferred to
02049 // {ASSERT|EXPECT}_TRUE(v1 == v2), as the former tells you

```

```

02050 // what the actual value is when it fails, and similarly for the
02051 // other comparisons.
02052 //
02053 // 4. Do not depend on the order in which {ASSERT|EXPECT}_??()
02054 // evaluate their arguments, which is undefined.
02055 //
02056 // 5. These macros evaluate their arguments exactly once.
02057 //
02058 // Examples:
02059 //
02060 // EXPECT_NE(Foo(), 5);
02061 // EXPECT_EQ(a_pointer, NULL);
02062 // ASSERT_LT(i, array_size);
02063 // ASSERT_GT(records.size(), 0) « "There is no record left.";
02064
02065 #define EXPECT_EQ(val1, val2) \
02066 EXPECT_PRED_FORMAT2(::testing::internal::EqHelper::Compare, val1, val2)
02067 #define EXPECT_NE(val1, val2) \
02068 EXPECT_PRED_FORMAT2(::testing::internal::CmpHelperNE, val1, val2)
02069 #define EXPECT_LE(val1, val2) \
02070 EXPECT_PRED_FORMAT2(::testing::internal::CmpHelperLE, val1, val2)
02071 #define EXPECT_LT(val1, val2) \
02072 EXPECT_PRED_FORMAT2(::testing::internal::CmpHelperLT, val1, val2)
02073 #define EXPECT_GE(val1, val2) \
02074 EXPECT_PRED_FORMAT2(::testing::internal::CmpHelperGE, val1, val2)
02075 #define EXPECT_GT(val1, val2) \
02076 EXPECT_PRED_FORMAT2(::testing::internal::CmpHelperGT, val1, val2)
02077
02078 #define GTEST_ASSERT_EQ(val1, val2) \
02079 ASSERT_PRED_FORMAT2(::testing::internal::EqHelper::Compare, val1, val2)
02080 #define GTEST_ASSERT_NE(val1, val2) \
02081 ASSERT_PRED_FORMAT2(::testing::internal::CmpHelperNE, val1, val2)
02082 #define GTEST_ASSERT_LE(val1, val2) \
02083 ASSERT_PRED_FORMAT2(::testing::internal::CmpHelperLE, val1, val2)
02084 #define GTEST_ASSERT_LT(val1, val2) \
02085 ASSERT_PRED_FORMAT2(::testing::internal::CmpHelperLT, val1, val2)
02086 #define GTEST_ASSERT_GE(val1, val2) \
02087 ASSERT_PRED_FORMAT2(::testing::internal::CmpHelperGE, val1, val2)
02088 #define GTEST_ASSERT_GT(val1, val2) \
02089 ASSERT_PRED_FORMAT2(::testing::internal::CmpHelperGT, val1, val2)
02090
02091 // Define macro GTEST_DONT_DEFINE_ASSERT_XY to 1 to omit the definition of
02092 // ASSERT_XY(), which clashes with some users' own code.
02093
02094 #if !GTEST_DONT_DEFINE_ASSERT_EQ
02095 #define ASSERT_EQ(val1, val2) GTEST_ASSERT_EQ(val1, val2)
02096 #endif
02097
02098 #if !GTEST_DONT_DEFINE_ASSERT_NE
02099 #define ASSERT_NE(val1, val2) GTEST_ASSERT_NE(val1, val2)
02100 #endif
02101
02102 #if !GTEST_DONT_DEFINE_ASSERT_LE
02103 #define ASSERT_LE(val1, val2) GTEST_ASSERT_LE(val1, val2)
02104 #endif
02105
02106 #if !GTEST_DONT_DEFINE_ASSERT_LT
02107 #define ASSERT_LT(val1, val2) GTEST_ASSERT_LT(val1, val2)
02108 #endif
02109
02110 #if !GTEST_DONT_DEFINE_ASSERT_GE
02111 #define ASSERT_GE(val1, val2) GTEST_ASSERT_GE(val1, val2)
02112 #endif
02113
02114 #if !GTEST_DONT_DEFINE_ASSERT_GT
02115 #define ASSERT_GT(val1, val2) GTEST_ASSERT_GT(val1, val2)
02116 #endif
02117
02118 // C-string Comparisons. All tests treat NULL and any non-NULL string
02119 // as different. Two NULLs are equal.
02120 //
02121 // * {ASSERT|EXPECT}_STREQ(s1, s2): Tests that s1 == s2
02122 // * {ASSERT|EXPECT}_STRNE(s1, s2): Tests that s1 != s2
02123 // * {ASSERT|EXPECT}_STRCASEEQ(s1, s2): Tests that s1 == s2, ignoring case
02124 // * {ASSERT|EXPECT}_STRCASENE(s1, s2): Tests that s1 != s2, ignoring case
02125 //
02126 // For wide or narrow string objects, you can use the
02127 // {ASSERT|EXPECT}_??() macros.
02128 //
02129 // Don't depend on the order in which the arguments are evaluated,
02130 // which is undefined.
02131 //
02132 // These macros evaluate their arguments exactly once.
02133
02134 #define EXPECT_STREQ(s1, s2) \
02135 EXPECT_PRED_FORMAT2(::testing::internal::CmpHelperSTREQ, s1, s2)
02136 #define EXPECT_STRNE(s1, s2) \

```

```

02137 EXPECT_PRED_FORMAT2(::testing::internal::CmpHelperSTRNE, s1, s2)
02138 #define EXPECT_STRCASEEQ(s1, s2) \
02139 EXPECT_PRED_FORMAT2(::testing::internal::CmpHelperSTRCASEEQ, s1, s2)
02140 #define EXPECT_STRCASENE(s1, s2) \
02141 EXPECT_PRED_FORMAT2(::testing::internal::CmpHelperSTRCASENE, s1, s2)
02142
02143 #define ASSERT_STREQ(s1, s2) \
02144 ASSERT_PRED_FORMAT2(::testing::internal::CmpHelperSTREQ, s1, s2)
02145 #define ASSERT_STRNE(s1, s2) \
02146 ASSERT_PRED_FORMAT2(::testing::internal::CmpHelperSTRNE, s1, s2)
02147 #define ASSERT_STRCASEEQ(s1, s2) \
02148 ASSERT_PRED_FORMAT2(::testing::internal::CmpHelperSTRCASEEQ, s1, s2)
02149 #define ASSERT_STRCASENE(s1, s2) \
02150 ASSERT_PRED_FORMAT2(::testing::internal::CmpHelperSTRCASENE, s1, s2)
02151
02152 // Macros for comparing floating-point numbers.
02153 //
02154 // * {ASSERT|EXPECT}_FLOAT_EQ(val1, val2):
02155 // Tests that two float values are almost equal.
02156 // * {ASSERT|EXPECT}_DOUBLE_EQ(val1, val2):
02157 // Tests that two double values are almost equal.
02158 // * {ASSERT|EXPECT}_NEAR(v1, v2, abs_error):
02159 // Tests that v1 and v2 are within the given distance to each other.
02160 //
02161 // Google Test uses ULP-based comparison to automatically pick a default
02162 // error bound that is appropriate for the operands. See the
02163 // FloatingPoint template class in gtest-internals.h if you are
02164 // interested in the implementation details.
02165
02166 #define EXPECT_FLOAT_EQ(val1, val2) \
02167 EXPECT_PRED_FORMAT2(::testing::internal::CmpHelperFloatingPointEQ<float>, \
02168 val1, val2)
02169
02170 #define EXPECT_DOUBLE_EQ(val1, val2) \
02171 EXPECT_PRED_FORMAT2(::testing::internal::CmpHelperFloatingPointEQ<double>, \
02172 val1, val2)
02173
02174 #define ASSERT_FLOAT_EQ(val1, val2) \
02175 ASSERT_PRED_FORMAT2(::testing::internal::CmpHelperFloatingPointEQ<float>, \
02176 val1, val2)
02177
02178 #define ASSERT_DOUBLE_EQ(val1, val2) \
02179 ASSERT_PRED_FORMAT2(::testing::internal::CmpHelperFloatingPointEQ<double>, \
02180 val1, val2)
02181
02182 #define EXPECT_NEAR(val1, val2, abs_error) \
02183 EXPECT_PRED_FORMAT3(::testing::internal::DoubleNearPredFormat, val1, val2, \
02184 abs_error)
02185
02186 #define ASSERT_NEAR(val1, val2, abs_error) \
02187 ASSERT_PRED_FORMAT3(::testing::internal::DoubleNearPredFormat, val1, val2, \
02188 abs_error)
02189
02190 // These predicate format functions work on floating-point values, and
02191 // can be used in {ASSERT|EXPECT}_PRED_FORMAT2*(), e.g.
02192 //
02193 // EXPECT_PRED_FORMAT2(testing::DoubleLE, Foo(), 5.0);
02194 //
02195 // Asserts that val1 is less than, or almost equal to, val2. Fails
02196 // otherwise. In particular, it fails if either val1 or val2 is NaN.
02197 GTEST_API_ AssertionResult FloatLE(const char* expr1, const char* expr2,
02198 float val1, float val2);
02199 GTEST_API_ AssertionResult DoubleLE(const char* expr1, const char* expr2,
02200 double val1, double val2);
02201
02202 #if GTEST_OS_WINDOWS
02203
02204 // Macros that test for HRESULT failure and success, these are only useful
02205 // on Windows, and rely on Windows SDK macros and APIs to compile.
02206 //
02207 // * {ASSERT|EXPECT}_HRESULT_{SUCCEEDED|FAILED}(expr)
02208 //
02209 // When expr unexpectedly fails or succeeds, Google Test prints the
02210 // expected result and the actual result with both a human-readable
02211 // string representation of the error, if available, as well as the
02212 // hex result code.
02213 #define EXPECT_HRESULT_SUCCEEDED(expr) \
02214 EXPECT_PRED_FORMAT1(::testing::internal::IsHRESULTSuccess, (expr))
02215
02216 #define ASSERT_HRESULT_SUCCEEDED(expr) \
02217 ASSERT_PRED_FORMAT1(::testing::internal::IsHRESULTSuccess, (expr))
02218
02219 #define EXPECT_HRESULT_FAILED(expr) \
02220 EXPECT_PRED_FORMAT1(::testing::internal::IsHRESULTFailure, (expr))
02221
02222 #define ASSERT_HRESULT_FAILED(expr) \
02223 ASSERT_PRED_FORMAT1(::testing::internal::IsHRESULTFailure, (expr))

```

```

02224
02225 #endif // GTEST_OS_WINDOWS
02226
02227 // Macros that execute statement and check that it doesn't generate new fatal
02228 // failures in the current thread.
02229 //
02230 // * {ASSERT|EXPECT}_NO_FATAL_FAILURE(statement);
02231 //
02232 // Examples:
02233 //
02234 // EXPECT_NO_FATAL_FAILURE(Process());
02235 // ASSERT_NO_FATAL_FAILURE(Process()) « "Process() failed";
02236 //
02237 #define ASSERT_NO_FATAL_FAILURE(statement) \
02238 GTEST_TEST_NO_FATAL_FAILURE_(statement, GTEST_FATAL_FAILURE_)
02239 #define EXPECT_NO_FATAL_FAILURE(statement) \
02240 GTEST_TEST_NO_FATAL_FAILURE_(statement, GTEST_NONFATAL_FAILURE_)
02241
02242 // Causes a trace (including the given source file path and line number,
02243 // and the given message) to be included in every test failure message generated
02244 // by code in the scope of the lifetime of an instance of this class. The effect
02245 // is undone with the destruction of the instance.
02246 //
02247 // The message argument can be anything streamable to std::ostream.
02248 //
02249 // Example:
02250 // testing::ScopedTrace trace("file.cc", 123, "message");
02251 //
02252
02253 class GTEST_API_ ScopedTrace {
02254 public:
02255 // The c'tor pushes the given source file location and message onto
02256 // a trace stack maintained by Google Test.
02257
02258 // Template version. Uses Message() to convert the values into strings.
02259 // Slow, but flexible.
02260
02261 template <typename T>
02262 ScopedTrace(const char* file, int line, const T& message) {
02263 PushTrace(file, line, (Message() « message).GetString());
02264 }
02265
02266 // Optimize for some known types.
02267
02268 ScopedTrace(const char* file, int line, const char* message) {
02269 PushTrace(file, line, message ? message : "(null)");
02270 }
02271
02272 ScopedTrace(const char* file, int line, const std::string& message) {
02273 PushTrace(file, line, message);
02274 }
02275
02276 // The d'tor pops the info pushed by the c'tor.
02277 //
02278 // Note that the d'tor is not virtual in order to be efficient.
02279 // Don't inherit from ScopedTrace!
02280 ~ScopedTrace();
02281
02282 private:
02283 void PushTrace(const char* file, int line, std::string message);
02284
02285 ScopedTrace(const ScopedTrace&) = delete;
02286 ScopedTrace& operator=(const ScopedTrace&) = delete;
02287 } GTEST_ATTRIBUTE_UNUSED_; // A ScopedTrace object does its job in its
02288 // c'tor and d'tor. Therefore it doesn't
02289 // need to be used otherwise.
02290
02291 // Causes a trace (including the source file path, the current line
02292 // number, and the given message) to be included in every test failure
02293 // message generated by code in the current scope. The effect is
02294 // undone when the control leaves the current scope.
02295 //
02296 // The message argument can be anything streamable to std::ostream.
02297 //
02298 // In the implementation, we include the current line number as part
02299 // of the dummy variable name, thus allowing multiple SCOPED_TRACE()'s
02300 // to appear in the same block - as long as they are on different
02301 // lines.
02302 //
02303 // Assuming that each thread maintains its own stack of traces.
02304 // Therefore, a SCOPED_TRACE() would (correctly) only affect the
02305 // assertions in its own thread.
02306 #define SCOPED_TRACE(message) \
02307 ::testing::ScopedTrace GTEST_CONCAT_TOKEN_(gtest_trace_, __LINE__) (\
02308 __FILE__, __LINE__, (message))
02309
02310 // Compile-time assertion for type equality.

```



```

02311 // StaticAssertTypeEq<type1, type2>() compiles if and only if type1 and type2
02312 // are the same type. The value it returns is not interesting.
02313 //
02314 // Instead of making StaticAssertTypeEq a class template, we make it a
02315 // function template that invokes a helper class template. This
02316 // prevents a user from misusing StaticAssertTypeEq<T1, T2> by
02317 // defining objects of that type.
02318 //
02319 // CAVEAT:
02320 //
02321 // When used inside a method of a class template,
02322 // StaticAssertTypeEq<T1, T2>() is effective ONLY IF the method is
02323 // instantiated. For example, given:
02324 //
02325 // template <typename T> class Foo {
02326 // public:
02327 // void Bar() { testing::StaticAssertTypeEq<int, T>(); }
02328 // };
02329 //
02330 // the code:
02331 //
02332 // void Test1() { Foo<bool> foo; }
02333 //
02334 // will NOT generate a compiler error, as Foo<bool>::Bar() is never
02335 // actually instantiated. Instead, you need:
02336 //
02337 // void Test2() { Foo<bool> foo; foo.Bar(); }
02338 //
02339 // to cause a compiler error.
02340
02341 template <typename T1, typename T2>
02342 constexpr bool StaticAssertTypeEq() noexcept {
02343 static_assert(std::is_same<T1, T2>::value, "T1 and T2 are not the same type");
02344 return true;
02345 }
02346
02347 // Defines a test.
02348 //
02349 // The first parameter is the name of the test suite, and the second
02350 // parameter is the name of the test within the test suite.
02351 //
02352 // The convention is to end the test suite name with "Test". For
02353 // example, a test suite for the Foo class can be named FooTest.
02354 //
02355 // Test code should appear between braces after an invocation of
02356 // this macro. Example:
02357 //
02358 // TEST(FooTest, InitializesCorrectly) {
02359 // Foo foo;
02360 // EXPECT_TRUE(foo.StatusIsOK());
02361 // }
02362
02363 // Note that we call GetTestTypeId() instead of GetTypeId<
02364 // ::testing::Test>() here to get the type ID of testing::Test. This
02365 // is to work around a suspected linker bug when using Google Test as
02366 // a framework on Mac OS X. The bug causes GetTypeId<
02367 // ::testing::Test>() to return different values depending on whether
02368 // the call is from the Google Test framework itself or from user test
02369 // code. GetTestTypeId() is guaranteed to always return the same
02370 // value, as it always calls GetTypeId<>() from the Google Test
02371 // framework.
02372 #define GTEST_TEST(test_suite_name, test_name) \
02373 GTEST_TEST_(test_suite_name, test_name, ::testing::Test, \
02374 ::testing::internal::GetTestTypeId())
02375
02376 // Define this macro to 1 to omit the definition of TEST(), which
02377 // is a generic name and clashes with some other libraries.
02378 #if !GTEST_DONT_DEFINE_TEST
02379 #define TEST(test_suite_name, test_name) GTEST_TEST(test_suite_name, test_name)
02380 #endif
02381
02382 // Defines a test that uses a test fixture.
02383 //
02384 // The first parameter is the name of the test fixture class, which
02385 // also doubles as the test suite name. The second parameter is the
02386 // name of the test within the test suite.
02387 //
02388 // A test fixture class must be declared earlier. The user should put
02389 // the test code between braces after using this macro. Example:
02390 //
02391 // class FooTest : public testing::Test {
02392 // protected:
02393 // void SetUp() override { b_.AddElement(3); }
02394 //
02395 // Foo a_;
02396 // Foo b_;
02397 // };

```

```

02398 //
02399 // TEST_F(FooTest, InitializesCorrectly) {
02400 // EXPECT_TRUE(a_.StatusIsOK());
02401 // }
02402 //
02403 // TEST_F(FooTest, ReturnsElementCountCorrectly) {
02404 // EXPECT_EQ(a_.size(), 0);
02405 // EXPECT_EQ(b_.size(), 1);
02406 // }
02407 #define GTEST_TEST_F(test_fixture, test_name) \
02408 GTEST_TEST_(test_fixture, test_name, test_fixture, \
02409 ::testing::internal::GetTypeId<test_fixture>())
02410 #if !GTEST_DONT_DEFINE_TEST_F
02411 #define TEST_F(test_fixture, test_name) GTEST_TEST_F(test_fixture, test_name)
02412 #endif
02413
02414 // Returns a path to temporary directory.
02415 // Tries to determine an appropriate directory for the platform.
02416 GTEST_API_ std::string TempDir();
02417
02418 #ifndef _MSC_VER
02419 #pragma warning(pop)
02420 #endif
02421
02422 // Dynamically registers a test with the framework.
02423 //
02424 // This is an advanced API only to be used when the `TEST` macros are
02425 // insufficient. The macros should be preferred when possible, as they avoid
02426 // most of the complexity of calling this function.
02427 //
02428 // The `factory` argument is a factory callable (move-constructible) object or
02429 // function pointer that creates a new instance of the Test object. It
02430 // handles ownership to the caller. The signature of the callable is
02431 // `Fixture*()`, where `Fixture` is the test fixture class for the test. All
02432 // tests registered with the same `test_suite_name` must return the same
02433 // fixture type. This is checked at runtime.
02434 //
02435 // The framework will infer the fixture class from the factory and will call
02436 // the `SetUpTestSuite` and `TearDownTestSuite` for it.
02437 //
02438 // Must be called before `RUN_ALL_TESTS()` is invoked, otherwise behavior is
02439 // undefined.
02440 //
02441 // Use case example:
02442 //
02443 // class MyFixture : public ::testing::Test {
02444 // public:
02445 // // All of these optional, just like in regular macro usage.
02446 // static void SetUpTestSuite() { ... }
02447 // static void TearDownTestSuite() { ... }
02448 // void SetUp() override { ... }
02449 // void TearDown() override { ... }
02450 // };
02451 //
02452 // class MyTest : public MyFixture {
02453 // public:
02454 // explicit MyTest(int data) : data_(data) {}
02455 // void TestBody() override { ... }
02456 // private:
02457 // int data_;
02458 // };
02459 //
02460 // void RegisterMyTests(const std::vector<int>& values) {
02461 // for (int v : values) {
02462 // ::testing::RegisterTest(
02463 // "MyFixture", ("Test" + std::to_string(v)).c_str(), nullptr,
02464 // std::to_string(v).c_str(),
02465 // __FILE__, __LINE__,
02466 // // Important to use the fixture type as the return type here.
02467 // [=]() -> MyFixture* { return new MyTest(v); });
02468 // }
02469 // }
02470 //
02471 // ...
02472 // int main(int argc, char** argv) {
02473 // ::testing::InitGoogleTest(&argc, argv);
02474 // std::vector<int> values_to_test = LoadValuesFromConfig();
02475 // RegisterMyTests(values_to_test);
02476 // ...
02477 // return RUN_ALL_TESTS();
02478 // }
02479 //
02480
02481 template <int&... ExplicitParameterBarrier, typename Factory>
02482 TestInfo* RegisterTest(const char* test_suite_name, const char* test_name,
02483 const char* type_param, const char* value_param,
02484 const char* file, int line, Factory factory) {

```

```

02485 using TestT = typename std::remove_pointer<decltype(factory())>::type;
02486
02487 class FactoryImpl : public internal::TestFactoryBase {
02488 public:
02489
02490 explicit FactoryImpl(Factory f) : factory_(std::move(f)) {
02491 }
02492
02493 Test* CreateTest() override {
02494 return factory_();
02495 }
02496
02497 private:
02498 Factory factory_;
02499 };
02500
02501 return internal::MakeAndRegisterTestInfo(
02502 test_suite_name, test_name, type_param, value_param,
02503 internal::CodeLocation(file, line), internal::GetTypeId<TestT>(),
02504 internal::SuiteApiResolver<TestT>::GetSetUpCaseOrSuite(file, line),
02505 internal::SuiteApiResolver<TestT>::GetTearDownCaseOrSuite(file, line),
02506 new FactoryImpl{std::move(factory)});
02507 }
02508
02509 } // namespace testing
02510
02511 // Use this function in main() to run all tests. It returns 0 if all
02512 // tests are successful, or 1 otherwise.
02513 //
02514 // RUN_ALL_TESTS() should be invoked after the command line has been
02515 // parsed by InitGoogleTest().
02516 //
02517 // This function was formerly a macro; thus, it is in the global
02518 // namespace and has an all-caps name.
02519 int RUN_ALL_TESTS() GTEST_MUST_USE_RESULT_;
02520
02521 inline int RUN_ALL_TESTS() {
02522 return ::testing::UnitTest::GetInstance()->Run();
02523 }
02524
02525 GTEST_DISABLE_MSC_WARNINGS_POP_() // 4251
02526
02527 #endif // GOOGLETEST_INCLUDE_GTEST_GTEST_H_

```

## 16.242 gtest.h

```

00001 // Copyright 2015, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029 //
00030 // Injection point for custom user configurations. See README for details
00031 //
00032 // ** Custom implementation starts here **
00033
00034 #ifndef GOOGLETEST_INCLUDE_GTEST_INTERNAL_CUSTOM_GTEST_H_
00035 #define GOOGLETEST_INCLUDE_GTEST_INTERNAL_CUSTOM_GTEST_H_
00036
00037 #endif // GOOGLETEST_INCLUDE_GTEST_INTERNAL_CUSTOM_GTEST_H_

```

## 16.243 gtest\_pred\_impl.h

```

00001 // Copyright 2006, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029 //
00030 // Implements a family of generic predicate assertion macros.
00031
00032 // IWYU pragma: private, include "gtest/gtest.h"
00033 // IWYU pragma: friend gtest/*.h
00034 // IWYU pragma: friend gmock/*.h
00035
00036 #ifndef GGOOGLETEST_INCLUDE_GTEST_GTEST_PRED_IMPL_H_
00037 #define GGOOGLETEST_INCLUDE_GTEST_GTEST_PRED_IMPL_H_
00038
00039 #include "gtest/gtest-assertion-result.h"
00040 #include "gtest/internal/gtest-internal.h"
00041 #include "gtest/internal/gtest-port.h"
00042
00043 namespace testing {
00044
00045 // This header implements a family of generic predicate assertion
00046 // macros:
00047 //
00048 // ASSERT_PRED_FORMAT1(pred_format, v1)
00049 // ASSERT_PRED_FORMAT2(pred_format, v1, v2)
00050 // ...
00051 //
00052 // where pred_format is a function or functor that takes n (in the
00053 // case of ASSERT_PRED_FORMATn) values and their source expression
00054 // text, and returns a testing::AssertionResult. See the definition
00055 // of ASSERT_EQ in gtest.h for an example.
00056 //
00057 // If you don't care about formatting, you can use the more
00058 // restrictive version:
00059 //
00060 // ASSERT_PRED1(pred, v1)
00061 // ASSERT_PRED2(pred, v1, v2)
00062 // ...
00063 //
00064 // where pred is an n-ary function or functor that returns bool,
00065 // and the values v1, v2, ..., must support the « operator for
00066 // streaming to std::ostream.
00067 //
00068 // We also define the EXPECT_* variations.
00069 //
00070 // For now we only support predicates whose arity is at most 5.
00071 // Please email googletestframework@googlegroups.com if you need
00072 // support for higher arities.
00073
00074 // GTEST_ASSERT_ is the basic statement to which all of the assertions
00075 // in this file reduce. Don't use this in your code.
00076
00077 #define GTEST_ASSERT_(expression, on_failure) \
00078 GTEST_AMBIGUOUS_ELSE_BLOCKER_ \
00079 if (const ::testing::AssertionResult gtest_ar = (expression)) \
00080 ; \
00081 else \
00082 on_failure(gtest_ar.failure_message())
00083
00084 // Helper function for implementing {EXPECT|ASSERT}_PRED1. Don't use
00085 // this in your code.

```

```

00086
00087 template <typename Pred, typename T1>
00088 AssertionResult AssertPred1Helper(const char* pred_text, const char* e1,
00089 Pred pred, const T1& v1) {
00090 if (pred(v1)) return AssertionSuccess();
00091
00092 return AssertionFailure()
00093 << pred_text << "(" << e1 << ") evaluates to false, where"
00094 << "\n"
00095 << e1 << " evaluates to " << ::testing::PrintToString(v1);
00096 }
00097
00098 // Internal macro for implementing {EXPECT|ASSERT}_PRED_FORMAT1.
00099 // Don't use this in your code.
00100 #define GTEST_PRED_FORMAT1(pred_format, v1, on_failure) \
00101 GTEST_ASSERT_(pred_format(#v1, v1), on_failure)
00102
00103 // Internal macro for implementing {EXPECT|ASSERT}_PRED1. Don't use
00104 // this in your code.
00105 #define GTEST_PRED1(pred, v1, on_failure) \
00106 GTEST_ASSERT_(:testing::AssertPred1Helper(#pred, #v1, pred, v1), on_failure)
00107
00108 // Unary predicate assertion macros.
00109 #define EXPECT_PRED_FORMAT1(pred_format, v1) \
00110 GTEST_PRED_FORMAT1(pred_format, v1, GTEST_NONFATAL_FAILURE_)
00111 #define EXPECT_PRED1(pred, v1) GTEST_PRED1(pred, v1, GTEST_NONFATAL_FAILURE_)
00112 #define ASSERT_PRED_FORMAT1(pred_format, v1) \
00113 GTEST_PRED_FORMAT1(pred_format, v1, GTEST_FATAL_FAILURE_)
00114 #define ASSERT_PRED1(pred, v1) GTEST_PRED1(pred, v1, GTEST_FATAL_FAILURE_)
00115
00116 // Helper function for implementing {EXPECT|ASSERT}_PRED2. Don't use
00117 // this in your code.
00118
00119 template <typename Pred, typename T1, typename T2>
00120 AssertionResult AssertPred2Helper(const char* pred_text, const char* e1,
00121 const char* e2, Pred pred, const T1& v1,
00122 const T2& v2) {
00123 if (pred(v1, v2)) return AssertionSuccess();
00124
00125 return AssertionFailure()
00126 << pred_text << "(" << e1 << ", " << e2
00127 << ") evaluates to false, where"
00128 << "\n"
00129 << e1 << " evaluates to " << ::testing::PrintToString(v1) << "\n"
00130 << e2 << " evaluates to " << ::testing::PrintToString(v2);
00131 }
00132
00133 // Internal macro for implementing {EXPECT|ASSERT}_PRED_FORMAT2.
00134 // Don't use this in your code.
00135 #define GTEST_PRED_FORMAT2(pred_format, v1, v2, on_failure) \
00136 GTEST_ASSERT_(pred_format(#v1, #v2, v1, v2), on_failure)
00137
00138 // Internal macro for implementing {EXPECT|ASSERT}_PRED2. Don't use
00139 // this in your code.
00140 #define GTEST_PRED2(pred, v1, v2, on_failure) \
00141 GTEST_ASSERT_(:testing::AssertPred2Helper(#pred, #v1, #v2, pred, v1, v2), \
00142 on_failure)
00143
00144 // Binary predicate assertion macros.
00145 #define EXPECT_PRED_FORMAT2(pred_format, v1, v2) \
00146 GTEST_PRED_FORMAT2(pred_format, v1, v2, GTEST_NONFATAL_FAILURE_)
00147 #define EXPECT_PRED2(pred, v1, v2) \
00148 GTEST_PRED2(pred, v1, v2, GTEST_NONFATAL_FAILURE_)
00149 #define ASSERT_PRED_FORMAT2(pred_format, v1, v2) \
00150 GTEST_PRED_FORMAT2(pred_format, v1, v2, GTEST_FATAL_FAILURE_)
00151 #define ASSERT_PRED2(pred, v1, v2) \
00152 GTEST_PRED2(pred, v1, v2, GTEST_FATAL_FAILURE_)
00153
00154 // Helper function for implementing {EXPECT|ASSERT}_PRED3. Don't use
00155 // this in your code.
00156
00157 template <typename Pred, typename T1, typename T2, typename T3>
00158 AssertionResult AssertPred3Helper(const char* pred_text, const char* e1,
00159 const char* e2, const char* e3, Pred pred,
00160 const T1& v1, const T2& v2, const T3& v3) {
00161 if (pred(v1, v2, v3)) return AssertionSuccess();
00162
00163 return AssertionFailure()
00164 << pred_text << "(" << e1 << ", " << e2 << ", " << e3
00165 << ") evaluates to false, where"
00166 << "\n"
00167 << e1 << " evaluates to " << ::testing::PrintToString(v1) << "\n"
00168 << e2 << " evaluates to " << ::testing::PrintToString(v2) << "\n"
00169 << e3 << " evaluates to " << ::testing::PrintToString(v3);
00170 }
00171
00172 // Internal macro for implementing {EXPECT|ASSERT}_PRED_FORMAT3.

```

```

00173 // Don't use this in your code.
00174 #define GTEST_PRED_FORMAT3(pred_format, v1, v2, v3, on_failure) \
00175 GTEST_ASSERT_(pred_format(#v1, #v2, #v3, v1, v2, v3), on_failure)
00176
00177 // Internal macro for implementing {EXPECT|ASSERT}_PRED3. Don't use
00178 // this in your code.
00179 #define GTEST_PRED3(pred, v1, v2, v3, on_failure) \
00180 GTEST_ASSERT_(\
00181 ::testing::AssertPred3Helper(#pred, #v1, #v2, #v3, pred, v1, v2, v3), \
00182 on_failure)
00183
00184 // Ternary predicate assertion macros.
00185 #define EXPECT_PRED_FORMAT3(pred_format, v1, v2, v3) \
00186 GTEST_PRED_FORMAT3(pred_format, v1, v2, v3, GTEST_NONFATAL_FAILURE_)
00187 #define EXPECT_PRED3(pred, v1, v2, v3) \
00188 GTEST_PRED3(pred, v1, v2, v3, GTEST_NONFATAL_FAILURE_)
00189 #define ASSERT_PRED_FORMAT3(pred_format, v1, v2, v3) \
00190 GTEST_PRED_FORMAT3(pred_format, v1, v2, v3, GTEST_FATAL_FAILURE_)
00191 #define ASSERT_PRED3(pred, v1, v2, v3) \
00192 GTEST_PRED3(pred, v1, v2, v3, GTEST_FATAL_FAILURE_)
00193
00194 // Helper function for implementing {EXPECT|ASSERT}_PRED4. Don't use
00195 // this in your code.
00196
00197 template <typename Pred, typename T1, typename T2, typename T3, typename T4>
00198 AssertionResult AssertPred4Helper(const char* pred_text, const char* e1,
00199 const char* e2, const char* e3,
00200 const char* e4, Pred pred, const T1& v1,
00201 const T2& v2, const T3& v3, const T4& v4) {
00202 if (pred(v1, v2, v3, v4)) return AssertionSuccess();
00203
00204 return AssertionFailure()
00205 << pred_text << "(" << e1 << ", " << e2 << ", " << e3 << ", " << e4
00206 << ") evaluates to false, where"
00207 << "\n"
00208 << e1 << " evaluates to " << ::testing::PrintToString(v1) << "\n"
00209 << e2 << " evaluates to " << ::testing::PrintToString(v2) << "\n"
00210 << e3 << " evaluates to " << ::testing::PrintToString(v3) << "\n"
00211 << e4 << " evaluates to " << ::testing::PrintToString(v4);
00212 }
00213
00214 // Internal macro for implementing {EXPECT|ASSERT}_PRED_FORMAT4.
00215 // Don't use this in your code.
00216 #define GTEST_PRED_FORMAT4(pred_format, v1, v2, v3, v4, on_failure) \
00217 GTEST_ASSERT_(pred_format(#v1, #v2, #v3, #v4, v1, v2, v3, v4), on_failure)
00218
00219 // Internal macro for implementing {EXPECT|ASSERT}_PRED4. Don't use
00220 // this in your code.
00221 #define GTEST_PRED4(pred, v1, v2, v3, v4, on_failure) \
00222 GTEST_ASSERT_(::testing::AssertPred4Helper(#pred, #v1, #v2, #v3, #v4, pred, \
00223 v1, v2, v3, v4), \
00224 on_failure)
00225
00226 // 4-ary predicate assertion macros.
00227 #define EXPECT_PRED_FORMAT4(pred_format, v1, v2, v3, v4) \
00228 GTEST_PRED_FORMAT4(pred_format, v1, v2, v3, v4, GTEST_NONFATAL_FAILURE_)
00229 #define EXPECT_PRED4(pred, v1, v2, v3, v4) \
00230 GTEST_PRED4(pred, v1, v2, v3, v4, GTEST_NONFATAL_FAILURE_)
00231 #define ASSERT_PRED_FORMAT4(pred_format, v1, v2, v3, v4) \
00232 GTEST_PRED_FORMAT4(pred_format, v1, v2, v3, v4, GTEST_FATAL_FAILURE_)
00233 #define ASSERT_PRED4(pred, v1, v2, v3, v4) \
00234 GTEST_PRED4(pred, v1, v2, v3, v4, GTEST_FATAL_FAILURE_)
00235
00236 // Helper function for implementing {EXPECT|ASSERT}_PRED5. Don't use
00237 // this in your code.
00238
00239 template <typename Pred, typename T1, typename T2, typename T3, typename T4,
00240 typename T5>
00241 AssertionResult AssertPred5Helper(const char* pred_text, const char* e1,
00242 const char* e2, const char* e3,
00243 const char* e4, const char* e5, Pred pred,
00244 const T1& v1, const T2& v2, const T3& v3,
00245 const T4& v4, const T5& v5) {
00246 if (pred(v1, v2, v3, v4, v5)) return AssertionSuccess();
00247
00248 return AssertionFailure()
00249 << pred_text << "(" << e1 << ", " << e2 << ", " << e3 << ", " << e4
00250 << ", " << e5 << ") evaluates to false, where"
00251 << "\n"
00252 << e1 << " evaluates to " << ::testing::PrintToString(v1) << "\n"
00253 << e2 << " evaluates to " << ::testing::PrintToString(v2) << "\n"
00254 << e3 << " evaluates to " << ::testing::PrintToString(v3) << "\n"
00255 << e4 << " evaluates to " << ::testing::PrintToString(v4) << "\n"
00256 << e5 << " evaluates to " << ::testing::PrintToString(v5);
00257 }
00258
00259 // Internal macro for implementing {EXPECT|ASSERT}_PRED_FORMAT5.

```

```

00260 // Don't use this in your code.
00261 #define GTEST_PRED_FORMAT5_(pred_format, v1, v2, v3, v4, v5, on_failure) \
00262 GTEST_ASSERT_(pred_format(#v1, #v2, #v3, #v4, #v5, v1, v2, v3, v4, v5), \
00263 on_failure)
00264
00265 // Internal macro for implementing {EXPECT|ASSERT}_PRED5. Don't use
00266 // this in your code.
00267 #define GTEST_PRED5_(pred, v1, v2, v3, v4, v5, on_failure) \
00268 GTEST_ASSERT_(:testing::AssertPred5Helper(#pred, #v1, #v2, #v3, #v4, #v5, \
00269 pred, v1, v2, v3, v4, v5), \
00270 on_failure)
00271
00272 // 5-ary predicate assertion macros.
00273 #define EXPECT_PRED_FORMAT5(pred_format, v1, v2, v3, v4, v5) \
00274 GTEST_PRED_FORMAT5_(pred_format, v1, v2, v3, v4, v5, GTEST_NONFATAL_FAILURE_)
00275 #define EXPECT_PRED5(pred, v1, v2, v3, v4, v5) \
00276 GTEST_PRED5_(pred, v1, v2, v3, v4, v5, GTEST_NONFATAL_FAILURE_)
00277 #define ASSERT_PRED_FORMAT5(pred_format, v1, v2, v3, v4, v5) \
00278 GTEST_PRED_FORMAT5_(pred_format, v1, v2, v3, v4, v5, GTEST_FATAL_FAILURE_)
00279 #define ASSERT_PRED5(pred, v1, v2, v3, v4, v5) \
00280 GTEST_PRED5_(pred, v1, v2, v3, v4, v5, GTEST_FATAL_FAILURE_)
00281
00282 } // namespace testing
00283
00284 #endif // GOOGLETEST_INCLUDE_GTEST_GTEST_PRED_IMPL_H_

```

## 16.244 gtest\_prod.h

```

00001 // Copyright 2006, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // Google C++ Testing and Mocking Framework definitions useful in production
00031 // code.
00032
00033 #ifndef GOOGLETEST_INCLUDE_GTEST_GTEST_PROD_H_
00034 #define GOOGLETEST_INCLUDE_GTEST_GTEST_PROD_H_
00035
00036 // When you need to test the private or protected members of a class,
00037 // use the FRIEND_TEST macro to declare your tests as friends of the
00038 // class. For example:
00039 //
00040 // class MyClass {
00041 // private:
00042 // void PrivateMethod();
00043 // FRIEND_TEST(MyClassTest, PrivateMethodWorks);
00044 // };
00045 //
00046 // class MyClassTest : public testing::Test {
00047 // // ...
00048 // };
00049 //
00050 // TEST_F(MyClassTest, PrivateMethodWorks) {
00051 // // Can call MyClass::PrivateMethod() here.
00052 // }
00053 //
00054 // Note: The test class must be in the same namespace as the class being tested.
00055 // For example, putting MyClassTest in an anonymous namespace will not work.
00056
00057 #define FRIEND_TEST(test_case_name, test_name) \

```

```
00058 friend class test_case_name##_##test_name##_Test
00059
00060 #endif // GOOGLETEST_INCLUDE_GTEST_GTEST_PROD_H_
```

## 16.245 gtest-port.h

```
00001 // Copyright 2015, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029 //
00030 // Injection point for custom user configurations. See README for details
00031 //
00032 // ** Custom implementation starts here **
00033
00034 #ifndef GOOGLETEST_INCLUDE_GTEST_INTERNAL_CUSTOM_GTEST_PORT_H_
00035 #define GOOGLETEST_INCLUDE_GTEST_INTERNAL_CUSTOM_GTEST_PORT_H_
00036
00037 #endif // GOOGLETEST_INCLUDE_GTEST_INTERNAL_CUSTOM_GTEST_PORT_H_
```

## 16.246 gtest-port.h

```
00001 // Copyright 2005, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029 //
00030 // Low-level types and utilities for porting Google Test to various
00031 // platforms. All macros ending with _ and symbols defined in an
00032 // internal namespace are subject to change without notice. Code
00033 // outside Google Test MUST NOT USE THEM DIRECTLY. Macros that don't
00034 // end with _ are part of Google Test's public API and can be used by
00035 // code outside Google Test.
00036 //
```



```

00037 // This file is fundamental to Google Test. All other Google Test source
00038 // files are expected to #include this. Therefore, it cannot #include
00039 // any other Google Test header.
00040
00041 // IWYU pragma: private, include "gtest/gtest.h"
00042 // IWYU pragma: friend gtest/*.
00043 // IWYU pragma: friend gmock/*.
00044
00045 #ifndef GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_PORT_H_
00046 #define GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_PORT_H_
00047
00048 // Environment-describing macros
00049 // -----
00050 //
00051 // Google Test can be used in many different environments. Macros in
00052 // this section tell Google Test what kind of environment it is being
00053 // used in, such that Google Test can provide environment-specific
00054 // features and implementations.
00055 //
00056 // Google Test tries to automatically detect the properties of its
00057 // environment, so users usually don't need to worry about these
00058 // macros. However, the automatic detection is not perfect.
00059 // Sometimes it's necessary for a user to define some of the following
00060 // macros in the build script to override Google Test's decisions.
00061 //
00062 // If the user doesn't define a macro in the list, Google Test will
00063 // provide a default definition. After this header is #included, all
00064 // macros in this list will be defined to either 1 or 0.
00065 //
00066 // Notes to maintainers:
00067 // - Each macro here is a user-tweakable knob; do not grow the list
00068 // lightly.
00069 // - Use #if to key off these macros. Don't use #ifdef or "#if
00070 // defined(...)", which will not work as these macros are ALWAYS
00071 // defined.
00072 //
00073 // GTEST_HAS_CLONE - Define it to 1/0 to indicate that clone(2)
00074 // is/isn't available.
00075 // GTEST_HAS_EXCEPTIONS - Define it to 1/0 to indicate that exceptions
00076 // are enabled.
00077 // GTEST_HAS_POSIX_RE - Define it to 1/0 to indicate that POSIX regular
00078 // expressions are/aren't available.
00079 // GTEST_HAS_PTHREAD - Define it to 1/0 to indicate that <pthread.h>
00080 // is/isn't available.
00081 // GTEST_HAS_RTTI - Define it to 1/0 to indicate that RTTI is/isn't
00082 // enabled.
00083 // GTEST_HAS_STD_WSTRING - Define it to 1/0 to indicate that
00084 // std::wstring does/doesn't work (Google Test can
00085 // be used where std::wstring is unavailable).
00086 // GTEST_HAS_SEH - Define it to 1/0 to indicate whether the
00087 // compiler supports Microsoft's "Structured
00088 // Exception Handling".
00089 // GTEST_HAS_STREAM_REDIRECTION
00090 // - Define it to 1/0 to indicate whether the
00091 // platform supports I/O stream redirection using
00092 // dup() and dup2().
00093 // GTEST_LINKED_AS_SHARED_LIBRARY
00094 // - Define to 1 when compiling tests that use
00095 // Google Test as a shared library (known as
00096 // DLL on Windows).
00097 // GTEST_CREATE_SHARED_LIBRARY
00098 // - Define to 1 when compiling Google Test itself
00099 // as a shared library.
00100 // GTEST_DEFAULT_DEATH_TEST_STYLE
00101 // - The default value of --gtest_death_test_style.
00102 // The legacy default has been "fast" in the open
00103 // source version since 2008. The recommended value
00104 // is "threadsafe", and can be set in
00105 // custom/gtest-port.h.
00106
00107 // Platform-indicating macros
00108 // -----
00109 //
00110 // Macros indicating the platform on which Google Test is being used
00111 // (a macro is defined to 1 if compiled on the given platform;
00112 // otherwise UNDEFINED -- it's never defined to 0). Google Test
00113 // defines these macros automatically. Code outside Google Test MUST
00114 // NOT define them.
00115 //
00116 // GTEST_OS_AIX - IBM AIX
00117 // GTEST_OS_CYGWIN - Cygwin
00118 // GTEST_OS_DRAGONFLY - DragonFlyBSD
00119 // GTEST_OS_FREEBSD - FreeBSD
00120 // GTEST_OS_FUCHSIA - Fuchsia
00121 // GTEST_OS_GNU_HURD - GNU/Hurd
00122 // GTEST_OS_GNU_KFREEBSD - GNU/kFreeBSD
00123 // GTEST_OS_HAIKU - Haiku

```

```

00124 // GTEST_OS_HPUX - HP-UX
00125 // GTEST_OS_LINUX - Linux
00126 // GTEST_OS_LINUX_ANDROID - Google Android
00127 // GTEST_OS_MAC - Mac OS X
00128 // GTEST_OS_IOS - iOS
00129 // GTEST_OS_NACL - Google Native Client (NaCl)
00130 // GTEST_OS_NETBSD - NetBSD
00131 // GTEST_OS_OPENBSD - OpenBSD
00132 // GTEST_OS_OS2 - OS/2
00133 // GTEST_OS_QNX - QNX
00134 // GTEST_OS_SOLARIS - Sun Solaris
00135 // GTEST_OS_WINDOWS - Windows (Desktop, MinGW, or Mobile)
00136 // GTEST_OS_WINDOWS_DESKTOP - Windows Desktop
00137 // GTEST_OS_WINDOWS_MINGW - MinGW
00138 // GTEST_OS_WINDOWS_MOBILE - Windows Mobile
00139 // GTEST_OS_WINDOWS_PHONE - Windows Phone
00140 // GTEST_OS_WINDOWS_RT - Windows Store App/WinRT
00141 // GTEST_OS_ZOS - z/OS
00142 //
00143 // Among the platforms, Cygwin, Linux, Mac OS X, and Windows have the
00144 // most stable support. Since core members of the Google Test project
00145 // don't have access to other platforms, support for them may be less
00146 // stable. If you notice any problems on your platform, please notify
00147 // googletestframework@googlegroups.com (patches for fixing them are
00148 // even more welcome!).
00149 //
00150 // It is possible that none of the GTEST_OS_* macros are defined.
00151 //
00152 // Feature-indicating macros
00153 // -----
00154 //
00155 // Macros indicating which Google Test features are available (a macro
00156 // is defined to 1 if the corresponding feature is supported;
00157 // otherwise UNDEFINED -- it's never defined to 0). Google Test
00158 // defines these macros automatically. Code outside Google Test MUST
00159 // NOT define them.
00160 //
00161 // These macros are public so that portable tests can be written.
00162 // Such tests typically surround code using a feature with an #if
00163 // which controls that code. For example:
00164 //
00165 // #if GTEST_HAS_DEATH_TEST
00166 // EXPECT_DEATH(DoSomethingDeadly());
00167 // #endif
00168 //
00169 // GTEST_HAS_DEATH_TEST - death tests
00170 // GTEST_HAS_TYPED_TEST - typed tests
00171 // GTEST_HAS_TYPED_TEST_P - type-parameterized tests
00172 // GTEST_IS_THREADSafe - Google Test is thread-safe.
00173 // GTEST_USES_RE2 - the RE2 regular expression library is used
00174 // GTEST_USES_POSIX_RE - enhanced POSIX regex is used. Do not confuse with
00175 // GTEST_HAS_POSIX_RE (see above) which users can
00176 // define themselves.
00177 // GTEST_USES_SIMPLE_RE - our own simple regex is used;
00178 // the above RE\b(s) are mutually exclusive.
00179 //
00180 // Misc public macros
00181 // -----
00182 //
00183 // GTEST_FLAG(flag_name) - references the variable corresponding to
00184 // the given Google Test flag.
00185 //
00186 // Internal utilities
00187 // -----
00188 //
00189 // The following macros and utilities are for Google Test's INTERNAL
00190 // use only. Code outside Google Test MUST NOT USE THEM DIRECTLY.
00191 //
00192 // Macros for basic C++ coding:
00193 // GTEST_AMBIGUOUS_ELSE_BLOCKER_ - for disabling a gcc warning.
00194 // GTEST_ATTRIBUTE_UNUSED_ - declares that a class' instances or a
00195 // variable don't have to be used.
00196 // GTEST_MUST_USE_RESULT_ - declares that a function's result must be used.
00197 // GTEST_INTENTIONAL_CONST_COND_PUSH_ - start code section where MSVC C4127 is
00198 // suppressed (constant conditional).
00199 // GTEST_INTENTIONAL_CONST_COND_POP_ - finish code section where MSVC C4127
00200 // is suppressed.
00201 // GTEST_INTERNAL_HAS_ANY - for enabling UniversalPrinter<std::any> or
00202 // UniversalPrinter<absl::any> specializations.
00203 // GTEST_INTERNAL_HAS_OPTIONAL - for enabling UniversalPrinter<std::optional>
00204 // or
00205 // UniversalPrinter<absl::optional>
00206 // specializations.
00207 // GTEST_INTERNAL_HAS_STRING_VIEW - for enabling Matcher<std::string_view> or
00208 // Matcher<absl::string_view>
00209 // specializations.
00210 // GTEST_INTERNAL_HAS_VARIANT - for enabling UniversalPrinter<std::variant> or

```

```

00211 // UniversalPrinter<absl::variant>
00212 // specializations.
00213 //
00214 // Synchronization:
00215 // Mutex, MutexLock, ThreadLocal, GetThreadCount()
00216 // - synchronization primitives.
00217 //
00218 // Regular expressions:
00219 // RE - a simple regular expression class using
00220 // 1) the RE2 syntax on all platforms when built with RE2
00221 // and Abseil as dependencies
00222 // 2) the POSIX Extended Regular Expression syntax on
00223 // UNIX-like platforms,
00224 // 3) A reduced regular exception syntax on other platforms,
00225 // including Windows.
00226 // Logging:
00227 // GTEST_LOG_() - logs messages at the specified severity level.
00228 // LogToStderr() - directs all log messages to stderr.
00229 // FlushInfoLog() - flushes informational log messages.
00230 //
00231 // Stdout and stderr capturing:
00232 // CaptureStdout() - starts capturing stdout.
00233 // GetCapturedStdout() - stops capturing stdout and returns the captured
00234 // string.
00235 // CaptureStderr() - starts capturing stderr.
00236 // GetCapturedStderr() - stops capturing stderr and returns the captured
00237 // string.
00238 //
00239 // Integer types:
00240 // TypeWithSize - maps an integer to a int type.
00241 // TimeInMillis - integers of known sizes.
00242 // BiggestInt - the biggest signed integer type.
00243 //
00244 // Command-line utilities:
00245 // GetInjectableArgs() - returns the command line as a vector of strings.
00246 //
00247 // Environment variable utilities:
00248 // GetEnv() - gets the value of an environment variable.
00249 // BoolFromGTestEnv() - parses a bool environment variable.
00250 // Int32FromGTestEnv() - parses an int32_t environment variable.
00251 // StringFromGTestEnv() - parses a string environment variable.
00252 //
00253 // Deprecation warnings:
00254 // GTEST_INTERNAL_DEPRECATED(message) - attribute marking a function as
00255 // deprecated; calling a marked function
00256 // should generate a compiler warning
00257 //
00258 #include <ctype.h> // for isspace, etc
00259 #include <stddef.h> // for ptrdiff_t
00260 #include <stdio.h>
00261 #include <stdlib.h>
00262 #include <string.h>
00263 //
00264 #include <cerrno>
00265 // #include <condition_variable> // Guarded by GTEST_IS_THREADSAFE below
00266 #include <cstdint>
00267 #include <iostream>
00268 #include <limits>
00269 #include <locale>
00270 #include <memory>
00271 #include <string>
00272 // #include <mutex> // Guarded by GTEST_IS_THREADSAFE below
00273 #include <tuple>
00274 #include <type_traits>
00275 #include <vector>
00276 //
00277 #ifndef _WIN32_WCE
00278 #include <sys/stat.h>
00279 #include <sys/types.h>
00280 #endif // !_WIN32_WCE
00281 //
00282 #if defined __APPLE__
00283 #include <AvailabilityMacros.h>
00284 #include <TargetConditionals.h>
00285 #endif
00286 //
00287 #include "gtest/internal/custom/gtest-port.h"
00288 #include "gtest/internal/gtest-port-arch.h"
00289 //
00290 #if GTEST_HAS_ABSL
00291 #include "absl/flags/declare.h"
00292 #include "absl/flags/flag.h"
00293 #include "absl/flags/reflection.h"
00294 #endif
00295 //
00296 #if !defined(GTEST_DEV_EMAIL_)
00297 #define GTEST_DEV_EMAIL_ "googletestframework@googlegroups.com"

```

```

00298 #define GTEST_FLAG_PREFIX_ "gtest_"
00299 #define GTEST_FLAG_PREFIX_DASH_ "gtest-"
00300 #define GTEST_FLAG_PREFIX_UPPER_ "GTEST_"
00301 #define GTEST_NAME_ "Google Test"
00302 #define GTEST_PROJECT_URL_ "https://github.com/google/googletest/"
00303 #endif // !defined(GTEST_DEV_EMAIL_)
00304
00305 #if !defined(GTEST_INIT_GOOGLE_TEST_NAME_)
00306 #define GTEST_INIT_GOOGLE_TEST_NAME_ "testing::InitGoogleTest"
00307 #endif // !defined(GTEST_INIT_GOOGLE_TEST_NAME_)
00308
00309 // Determines the version of gcc that is used to compile this.
00310 #ifdef __GNUC__
00311 // 40302 means version 4.3.2.
00312 #define GTEST_GCC_VER_ \
00313 (__GNUC__ * 10000 + __GNUC_MINOR__ * 100 + __GNUC_PATCHLEVEL__)
00314 #endif // __GNUC__
00315
00316 // Macros for disabling Microsoft Visual C++ warnings.
00317 //
00318 // GTEST_DISABLE_MSC_WARNINGS_PUSH_(4800 4385)
00319 // /* code that triggers warnings C4800 and C4385 */
00320 // GTEST_DISABLE_MSC_WARNINGS_POP_()
00321 #if defined(_MSC_VER)
00322 #define GTEST_DISABLE_MSC_WARNINGS_PUSH_(warnings) \
00323 __pragma(warning(push)) __pragma(warning(disable : warnings))
00324 #define GTEST_DISABLE_MSC_WARNINGS_POP_() __pragma(warning(pop))
00325 #else
00326 // Not all compilers are MSVC
00327 #define GTEST_DISABLE_MSC_WARNINGS_PUSH_(warnings)
00328 #define GTEST_DISABLE_MSC_WARNINGS_POP_()
00329 #endif
00330
00331 // Clang on Windows does not understand MSVC's pragma warning.
00332 // We need clang-specific way to disable function deprecation warning.
00333 #ifdef __clang__
00334 #define GTEST_DISABLE_MSC_DEPRECATED_PUSH_() \
00335 _Pragma("clang diagnostic push") \
00336 _Pragma("clang diagnostic ignored \"-Wdeprecated-declarations\"") \
00337 _Pragma("clang diagnostic ignored \"-Wdeprecated-implementations\"")
00338 #define GTEST_DISABLE_MSC_DEPRECATED_POP_() _Pragma("clang diagnostic pop")
00339 #else
00340 #define GTEST_DISABLE_MSC_DEPRECATED_PUSH_() \
00341 GTEST_DISABLE_MSC_WARNINGS_PUSH_(4996)
00342 #define GTEST_DISABLE_MSC_DEPRECATED_POP_() GTEST_DISABLE_MSC_WARNINGS_POP_()
00343 #endif
00344
00345 // Brings in definitions for functions used in the testing::internal::posix
00346 // namespace (read, write, close, chdir, isatty, stat). We do not currently
00347 // use them on Windows Mobile.
00348 #if GTEST_OS_WINDOWS
00349 #if !GTEST_OS_WINDOWS_MOBILE
00350 #include <direct.h>
00351 #include <io.h>
00352 #endif
00353 // In order to avoid having to include <windows.h>, use forward declaration
00354 #if GTEST_OS_WINDOWS_MINGW && !defined(__MINGW64_VERSION_MAJOR)
00355 // MinGW defined _CRITICAL_SECTION and _RTL_CRITICAL_SECTION as two
00356 // separate (equivalent) structs, instead of using typedef
00357 typedef struct _CRITICAL_SECTION GTEST_CRITICAL_SECTION;
00358 #else
00359 // Assume CRITICAL_SECTION is a typedef of _RTL_CRITICAL_SECTION.
00360 // This assumption is verified by
00361 // WindowsTypesTest.CRITICAL_SECTIONIs_RTL_CRITICAL_SECTION.
00362 typedef struct _RTL_CRITICAL_SECTION GTEST_CRITICAL_SECTION;
00363 #endif
00364 #elif GTEST_OS_XTENSA
00365 #include <unistd.h>
00366 // Xtensa toolchains define strcasecmp in the string.h header instead of
00367 // strings.h. string.h is already included.
00368 #else
00369 // This assumes that non-Windows OSes provide unistd.h. For OSes where this
00370 // is not the case, we need to include headers that provide the functions
00371 // mentioned above.
00372 #include <strings.h>
00373 #include <unistd.h>
00374 #endif // GTEST_OS_WINDOWS
00375
00376 #if GTEST_OS_LINUX_ANDROID
00377 // Used to define __ANDROID_API__ matching the target NDK API level.
00378 #include <android/api-level.h> // NOLINT
00379 #endif
00380
00381 // Defines this to true if and only if Google Test can use POSIX regular
00382 // expressions.
00383 #ifndef GTEST_HAS_POSIX_RE
00384 #if GTEST_OS_LINUX_ANDROID

```

```

00385 // On Android, <regex.h> is only available starting with Gingerbread.
00386 #define GTEST_HAS_POSIX_RE (__ANDROID_API__ >= 9)
00387 #else
00388 #define GTEST_HAS_POSIX_RE (!GTEST_OS_WINDOWS && !GTEST_OS_XTENSA)
00389 #endif
00390 #endif
00391
00392 // Select the regular expression implementation.
00393 #if GTEST_HAS_ABSL
00394 // When using Abseil, RE2 is required.
00395 #include "absl/strings/string_view.h"
00396 #include "re2/re2.h"
00397 #define GTEST_USES_RE2 1
00398 #elif GTEST_HAS_POSIX_RE
00399 #include <regex.h> // NOLINT
00400 #define GTEST_USES_POSIX_RE 1
00401 #else
00402 // Use our own simple regex implementation.
00403 #define GTEST_USES_SIMPLE_RE 1
00404 #endif
00405
00406 #ifndef GTEST_HAS_EXCEPTIONS
00407 // The user didn't tell us whether exceptions are enabled, so we need
00408 // to figure it out.
00409 #if defined(_MSC_VER) && defined(_CPPUNWIND)
00410 // MSVC defines _CPPUNWIND to 1 if and only if exceptions are enabled.
00411 #define GTEST_HAS_EXCEPTIONS 1
00412 #elif defined(__BORLANDC__)
00413 // C++Builder's implementation of the STL uses the _HAS_EXCEPTIONS
00414 // macro to enable exceptions, so we'll do the same.
00415 // Assumes that exceptions are enabled by default.
00416 #ifndef _HAS_EXCEPTIONS
00417 #define _HAS_EXCEPTIONS 1
00418 #endif // _HAS_EXCEPTIONS
00419 #define GTEST_HAS_EXCEPTIONS _HAS_EXCEPTIONS
00420 #elif defined(__clang__)
00421 // clang defines __EXCEPTIONS if and only if exceptions are enabled before clang
00422 // 220714, but if and only if cleanups are enabled after that. In Obj-C++ files,
00423 // there can be cleanups for ObjC exceptions which also need cleanups, even if
00424 // C++ exceptions are disabled. clang has __has_feature(cxx_exceptions) which
00425 // checks for C++ exceptions starting at clang r206352, but which checked for
00426 // cleanups prior to that. To reliably check for C++ exception availability with
00427 // clang, check for
00428 // __EXCEPTIONS && __has_feature(cxx_exceptions).
00429 #define GTEST_HAS_EXCEPTIONS (__EXCEPTIONS && __has_feature(cxx_exceptions))
00430 #elif defined(__GNUC__) && __EXCEPTIONS
00431 // gcc defines __EXCEPTIONS to 1 if and only if exceptions are enabled.
00432 #define GTEST_HAS_EXCEPTIONS 1
00433 #elif defined(__SUNPRO_CC)
00434 // Sun Pro CC supports exceptions. However, there is no compile-time way of
00435 // detecting whether they are enabled or not. Therefore, we assume that
00436 // they are enabled unless the user tells us otherwise.
00437 #define GTEST_HAS_EXCEPTIONS 1
00438 #elif defined(__IBMCPP__) && __EXCEPTIONS
00439 // xlC defines __EXCEPTIONS to 1 if and only if exceptions are enabled.
00440 #define GTEST_HAS_EXCEPTIONS 1
00441 #elif defined(__HP_aCC)
00442 // Exception handling is in effect by default in HP aCC compiler. It has to
00443 // be turned off by +noeh compiler option if desired.
00444 #define GTEST_HAS_EXCEPTIONS 1
00445 #else
00446 // For other compilers, we assume exceptions are disabled to be
00447 // conservative.
00448 #define GTEST_HAS_EXCEPTIONS 0
00449 #endif // defined(_MSC_VER) || defined(__BORLANDC__)
00450 #endif // GTEST_HAS_EXCEPTIONS
00451
00452 #ifndef GTEST_HAS_STD_WSTRING
00453 // The user didn't tell us whether ::std::wstring is available, so we need
00454 // to figure it out.
00455 // Cygwin 1.7 and below doesn't support ::std::wstring.
00456 // Solaris' libc++ doesn't support it either. Android has
00457 // no support for it at least as recent as Froyo (2.2).
00458 #define GTEST_HAS_STD_WSTRING \
00459 (!GTEST_OS_LINUX_ANDROID || GTEST_OS_CYGWIN || GTEST_OS_SOLARIS || \
00460 GTEST_OS_HAIKU || GTEST_OS_ESP32 || GTEST_OS_ESP8266 || GTEST_OS_XTENSA)
00461 #endif // GTEST_HAS_STD_WSTRING
00462
00463 // Determines whether RTTI is available.
00464 #ifndef GTEST_HAS_RTTI
00465 // The user didn't tell us whether RTTI is enabled, so we need to
00466 // figure it out.
00467 #if defined(_MSC_VER)
00468 #define GTEST_HAS_RTTI 1
00469 #elif defined(__CXXRTTI__)
00470 #define GTEST_HAS_RTTI 1
00471 #endif // _MSC_VER

```

```

00472 #define GTEST_HAS_RTTI 1
00473 #else
00474 #define GTEST_HAS_RTTI 0
00475 #endif
00476
00477 // Starting with version 4.3.2, gcc defines __GXX_RTTI if and only if RTTI is
00478 // enabled.
00479 #elif defined(__GNUC__)
00480
00481 #ifdef __GXX_RTTI
00482 // When building against STLport with the Android NDK and with
00483 // -frtti -fno-exceptions, the build fails at link time with undefined
00484 // references to __cxa_bad_typeid. Note sure if STL or toolchain bug,
00485 // so disable RTTI when detected.
00486 #if GTEST_OS_LINUX_ANDROID && defined(_STLPORT_MAJOR) && !defined(__EXCEPTIONS)
00487 #define GTEST_HAS_RTTI 0
00488 #else
00489 #define GTEST_HAS_RTTI 1
00490 #endif // GTEST_OS_LINUX_ANDROID && __STLPORT_MAJOR && !__EXCEPTIONS
00491 #else
00492 #define GTEST_HAS_RTTI 0
00493 #endif // __GXX_RTTI
00494
00495 // Clang defines __GXX_RTTI starting with version 3.0, but its manual recommends
00496 // using has_feature instead. has_feature(cxx_rtti) is supported since 2.7, the
00497 // first version with C++ support.
00498 #elif defined(__clang__)
00499
00500 #define GTEST_HAS_RTTI __has_feature(cxx_rtti)
00501
00502 // Starting with version 9.0 IBM Visual Age defines __RTTI_ALL__ to 1 if
00503 // both the typeid and dynamic_cast features are present.
00504 #elif defined(__IBMCPP__) && (__IBMCPP__ >= 900)
00505
00506 #ifdef __RTTI_ALL__
00507 #define GTEST_HAS_RTTI 1
00508 #else
00509 #define GTEST_HAS_RTTI 0
00510 #endif
00511
00512 #else
00513
00514 // For all other compilers, we assume RTTI is enabled.
00515 #define GTEST_HAS_RTTI 1
00516
00517 #endif // _MSC_VER
00518
00519 #endif // GTEST_HAS_RTTI
00520
00521 // It's this header's responsibility to #include <typeinfo> when RTTI
00522 // is enabled.
00523 #if GTEST_HAS_RTTI
00524 #include <typeinfo>
00525 #endif
00526
00527 // Determines whether Google Test can use the pthreads library.
00528 #ifndef GTEST_HAS_PTHREAD
00529 // The user didn't tell us explicitly, so we make reasonable assumptions about
00530 // which platforms have pthreads support.
00531 //
00532 // To disable threading support in Google Test, add -DGTEST_HAS_PTHREAD=0
00533 // to your compiler flags.
00534 #define GTEST_HAS_PTHREAD \
00535 (GTEST_OS_LINUX || GTEST_OS_MAC || GTEST_OS_HPUX || GTEST_OS_QNX || \
00536 GTEST_OS_FREEBSD || GTEST_OS_NACL || GTEST_OS_NETBSD || GTEST_OS_FUCHSIA || \
00537 GTEST_OS_DRAGONFLY || GTEST_OS_GNU_KFREEBSD || GTEST_OS_OPENBSD || \
00538 GTEST_OS_HAIKU || GTEST_OS_GNU_HURD)
00539 #endif // GTEST_HAS_PTHREAD
00540
00541 #if GTEST_HAS_PTHREAD
00542 // gtest-port.h guarantees to #include <pthread.h> when GTEST_HAS_PTHREAD is
00543 // true.
00544 #include <pthread.h> // NOLINT
00545
00546 // For timespec and nanosleep, used below.
00547 #include <time.h> // NOLINT
00548 #endif
00549
00550 // Determines whether clone(2) is supported.
00551 // Usually it will only be available on Linux, excluding
00552 // Linux on the Itanium architecture.
00553 // Also see http://linux.die.net/man/2/clone.
00554 #ifndef GTEST_HAS_CLONE
00555 // The user didn't tell us, so we need to figure it out.
00556
00557 #if GTEST_OS_LINUX && !defined(__ia64__)
00558 #if GTEST_OS_LINUX_ANDROID

```

```

00559 // On Android, clone() became available at different API levels for each 32-bit
00560 // architecture.
00561 #if defined(__LP64__) || (defined(__arm__) && __ANDROID_API__ >= 9) || \
00562 (defined(__mips__) && __ANDROID_API__ >= 12) || \
00563 (defined(__i386__) && __ANDROID_API__ >= 17)
00564 #define GTEST_HAS_CLONE 1
00565 #else
00566 #define GTEST_HAS_CLONE 0
00567 #endif
00568 #else
00569 #define GTEST_HAS_CLONE 1
00570 #endif
00571 #else
00572 #define GTEST_HAS_CLONE 0
00573 #endif // GTEST_OS_LINUX && !defined(__ia64__)
00574
00575 #endif // GTEST_HAS_CLONE
00576
00577 // Determines whether to support stream redirection. This is used to test
00578 // output correctness and to implement death tests.
00579 #ifndef GTEST_HAS_STREAM_REDIRECTION
00580 // By default, we assume that stream redirection is supported on all
00581 // platforms except known mobile ones.
00582 #if GTEST_OS_WINDOWS_MOBILE || GTEST_OS_WINDOWS_PHONE || \
00583 GTEST_OS_WINDOWS_RT || GTEST_OS_ESP8266 || GTEST_OS_XTENSA
00584 #define GTEST_HAS_STREAM_REDIRECTION 0
00585 #else
00586 #define GTEST_HAS_STREAM_REDIRECTION 1
00587 #endif // !GTEST_OS_WINDOWS_MOBILE
00588 #endif // GTEST_HAS_STREAM_REDIRECTION
00589
00590 // Determines whether to support death tests.
00591 // pops up a dialog window that cannot be suppressed programmatically.
00592 #if (GTEST_OS_LINUX || GTEST_OS_CYGWIN || GTEST_OS_SOLARIS || \
00593 (GTEST_OS_MAC && !GTEST_OS_IOS) || \
00594 (GTEST_OS_WINDOWS_DESKTOP && _MSC_VER) || GTEST_OS_WINDOWS_MINGW || \
00595 GTEST_OS_AIX || GTEST_OS_HPUX || GTEST_OS_OPENBSD || GTEST_OS_QNX || \
00596 GTEST_OS_FREEBSD || GTEST_OS_NETBSD || GTEST_OS_FUCHSIA || \
00597 GTEST_OS_DRAGONFLY || GTEST_OS_GNU_KFREEBSD || GTEST_OS_HAIKU || \
00598 GTEST_OS_GNU_HURD)
00599 #define GTEST_HAS_DEATH_TEST 1
00600 #endif
00601
00602 // Determines whether to support type-driven tests.
00603
00604 // Typed tests need <typeinfo> and variadic macros, which GCC, VC++ 8.0,
00605 // Sun Pro CC, IBM Visual Age, and HP aCC support.
00606 #if defined(__GNUC__) || defined(_MSC_VER) || defined(__SUNPRO_CC) || \
00607 defined(__IBMCPP__) || defined(__HP_aCC)
00608 #define GTEST_HAS_TYPED_TEST 1
00609 #define GTEST_HAS_TYPED_TEST_P 1
00610 #endif
00611
00612 // Determines whether the system compiler uses UTF-16 for encoding wide strings.
00613 #define GTEST_WIDE_STRING_USES_UTF16_ \
00614 (GTEST_OS_WINDOWS || GTEST_OS_CYGWIN || GTEST_OS_AIX || GTEST_OS_OS2)
00615
00616 // Determines whether test results can be streamed to a socket.
00617 #if GTEST_OS_LINUX || GTEST_OS_GNU_KFREEBSD || GTEST_OS_DRAGONFLY || \
00618 GTEST_OS_FREEBSD || GTEST_OS_NETBSD || GTEST_OS_OPENBSD || \
00619 GTEST_OS_GNU_HURD
00620 #define GTEST_CAN_STREAM_RESULTS_ 1
00621 #endif
00622
00623 // Defines some utility macros.
00624
00625 // The GNU compiler emits a warning if nested "if" statements are followed by
00626 // an "else" statement and braces are not used to explicitly disambiguate the
00627 // "else" binding. This leads to problems with code like:
00628 //
00629 // if (gate)
00630 // ASSERT_(condition) << "Some message";
00631 //
00632 // The "switch (0) case 0:" idiom is used to suppress this.
00633 #ifndef __INTEL_COMPILER
00634 #define GTEST_AMBIGUOUS_ELSE_BLOCKER_
00635 #else
00636 #define GTEST_AMBIGUOUS_ELSE_BLOCKER_ \
00637 switch (0) \
00638 case 0: \
00639 default: // NOLINT
00640 #endif
00641
00642 // Use this annotation at the end of a struct/class definition to
00643 // prevent the compiler from optimizing away instances that are never
00644 // used. This is useful when all interesting logic happens inside the
00645 // c'tor and / or d'tor. Example:

```

```

00646 //
00647 // struct Foo {
00648 // Foo() { ... }
00649 // } GTEST_ATTRIBUTE_UNUSED_;
00650 //
00651 // Also use it after a variable or parameter declaration to tell the
00652 // compiler the variable/parameter does not have to be used.
00653 #if defined(__GNUC__) && !defined(COMPILER_ICC)
00654 #define GTEST_ATTRIBUTE_UNUSED_ __attribute__((unused))
00655 #elif defined(__clang__)
00656 #if __has_attribute(unused)
00657 #define GTEST_ATTRIBUTE_UNUSED_ __attribute__((unused))
00658 #endif
00659 #endif
00660 #ifndef GTEST_ATTRIBUTE_UNUSED_
00661 #define GTEST_ATTRIBUTE_UNUSED_
00662 #endif
00663
00664 // Use this annotation before a function that takes a printf format string.
00665 #if (defined(__GNUC__) || defined(__clang__)) && !defined(COMPILER_ICC)
00666 #if defined(__MINGW_PRINTF_FORMAT)
00667 // MinGW has two different printf implementations. Ensure the format macro
00668 // matches the selected implementation. See
00669 // https://sourceforge.net/p/mingw-w64/wiki2/gnu%20printf/.
00670 #define GTEST_ATTRIBUTE_PRINTF_(string_index, first_to_check) \
00671 __attribute__((\
00672 __format__(__MINGW_PRINTF_FORMAT, string_index, first_to_check)))
00673 #else
00674 #define GTEST_ATTRIBUTE_PRINTF_(string_index, first_to_check) \
00675 __attribute__((__format__(__printf__, string_index, first_to_check)))
00676 #endif
00677 #else
00678 #define GTEST_ATTRIBUTE_PRINTF_(string_index, first_to_check)
00679 #endif
00680
00681 // Tell the compiler to warn about unused return values for functions declared
00682 // with this macro. The macro should be used on function declarations
00683 // following the argument list:
00684 //
00685 // Sprocket* AllocateSprocket() GTEST_MUST_USE_RESULT_;
00686 #if defined(__GNUC__) && !defined(COMPILER_ICC)
00687 #define GTEST_MUST_USE_RESULT_ __attribute__((warn_unused_result))
00688 #else
00689 #define GTEST_MUST_USE_RESULT_
00690 #endif // __GNUC__ && !COMPILER_ICC
00691
00692 // MS C++ compiler emits warning when a conditional expression is compile time
00693 // constant. In some contexts this warning is false positive and needs to be
00694 // suppressed. Use the following two macros in such cases:
00695 //
00696 // GTEST_INTENTIONAL_CONST_COND_PUSH_()
00697 // while (true) {
00698 // GTEST_INTENTIONAL_CONST_COND_POP_()
00699 // }
00700 #define GTEST_INTENTIONAL_CONST_COND_PUSH_() \
00701 GTEST_DISABLE_MSC_WARNINGS_PUSH_(4127)
00702 #define GTEST_INTENTIONAL_CONST_COND_POP_() GTEST_DISABLE_MSC_WARNINGS_POP_()
00703
00704 // Determine whether the compiler supports Microsoft's Structured Exception
00705 // Handling. This is supported by several Windows compilers but generally
00706 // does not exist on any other system.
00707 #ifndef GTEST_HAS_SEH
00708 // The user didn't tell us, so we need to figure it out.
00709
00710 #if defined(_MSC_VER) || defined(__BORLANDC__)
00711 // These two compilers are known to support SEH.
00712 #define GTEST_HAS_SEH 1
00713 #else
00714 // Assume no SEH.
00715 #define GTEST_HAS_SEH 0
00716 #endif
00717
00718 #endif // GTEST_HAS_SEH
00719
00720 #ifndef GTEST_IS_THREADSAFE
00721
00722 #define GTEST_IS_THREADSAFE \
00723 (GTEST_HAS_MUTEX_AND_THREAD_LOCAL_ || \
00724 (GTEST_OS_WINDOWS && !GTEST_OS_WINDOWS_PHONE && !GTEST_OS_WINDOWS_RT) || \
00725 GTEST_HAS_PTHREAD)
00726
00727 #endif // GTEST_IS_THREADSAFE
00728
00729 #if GTEST_IS_THREADSAFE
00730 // Some platforms don't support including these threading related headers.
00731 #include <condition_variable> // NOLINT
00732 #include <mutex> // NOLINT

```



```

00733 #endif // GTEST_IS_THREADSafe
00734
00735 // GTEST_API_ qualifies all symbols that must be exported. The definitions below
00736 // are guarded by #ifndef to give embedders a chance to define GTEST_API_ in
00737 // gtest/internal/custom/gtest-port.h
00738 #ifndef GTEST_API_
00739
00740 #ifdef _MSC_VER
00741 #if GTEST_LINKED_AS_SHARED_LIBRARY
00742 #define GTEST_API_ __declspec(dllimport)
00743 #elif GTEST_CREATE_SHARED_LIBRARY
00744 #define GTEST_API_ __declspec(dllexport)
00745 #endif
00746 #elif __GNUC__ >= 4 || defined(__clang__)
00747 #define GTEST_API_ __attribute__((visibility("default")))
00748 #endif // _MSC_VER
00749
00750 #endif // GTEST_API_
00751
00752 #ifndef GTEST_API_
00753 #define GTEST_API_
00754 #endif // GTEST_API_
00755
00756 #ifndef GTEST_DEFAULT_DEATH_TEST_STYLE
00757 #define GTEST_DEFAULT_DEATH_TEST_STYLE "fast"
00758 #endif // GTEST_DEFAULT_DEATH_TEST_STYLE
00759
00760 #ifdef __GNUC__
00761 // Ask the compiler to never inline a given function.
00762 #define GTEST_NO_INLINE_ __attribute__((noinline))
00763 #else
00764 #define GTEST_NO_INLINE_
00765 #endif
00766
00767 #if defined(__clang__)
00768 // Nested ifs to avoid triggering MSVC warning.
00769 #if __has_attribute(disable_tail_calls)
00770 // Ask the compiler not to perform tail call optimization inside
00771 // the marked function.
00772 #define GTEST_NO_TAIL_CALL_ __attribute__((disable_tail_calls))
00773 #endif
00774 #elif __GNUC__
00775 #define GTEST_NO_TAIL_CALL_ \
00776 __attribute__((optimize("no-optimize-sibling-calls")))
00777 #else
00778 #define GTEST_NO_TAIL_CALL_
00779 #endif
00780
00781 // _LIBCPP_VERSION is defined by the libc++ library from the LLVM project.
00782 #if !defined(GTEST_HAS_CXXABI_H_)
00783 #if defined(__GLIBCXX__) || (defined(_LIBCPP_VERSION) && !defined(_MSC_VER))
00784 #define GTEST_HAS_CXXABI_H_ 1
00785 #else
00786 #define GTEST_HAS_CXXABI_H_ 0
00787 #endif
00788 #endif
00789
00790 // A function level attribute to disable checking for use of uninitialized
00791 // memory when built with MemorySanitizer.
00792 #if defined(__clang__)
00793 #if __has_feature(memory_sanitizer)
00794 #define GTEST_ATTRIBUTE_NO_SANITIZE_MEMORY_ __attribute__((no_sanitize_memory))
00795 #else
00796 #define GTEST_ATTRIBUTE_NO_SANITIZE_MEMORY_
00797 #endif // __has_feature(memory_sanitizer)
00798 #else
00799 #define GTEST_ATTRIBUTE_NO_SANITIZE_MEMORY_
00800 #endif // __clang__
00801
00802 // A function level attribute to disable AddressSanitizer instrumentation.
00803 #if defined(__clang__)
00804 #if __has_feature(address_sanitizer)
00805 #define GTEST_ATTRIBUTE_NO_SANITIZE_ADDRESS_ \
00806 __attribute__((no_sanitize_address))
00807 #else
00808 #define GTEST_ATTRIBUTE_NO_SANITIZE_ADDRESS_
00809 #endif // __has_feature(address_sanitizer)
00810 #else
00811 #define GTEST_ATTRIBUTE_NO_SANITIZE_ADDRESS_
00812 #endif // __clang__
00813
00814 // A function level attribute to disable HWAddressSanitizer instrumentation.
00815 #if defined(__clang__)
00816 #if __has_feature(hwaddress_sanitizer)
00817 #define GTEST_ATTRIBUTE_NO_SANITIZE_HWADDRESS_ \
00818 __attribute__((no_sanitize("hwaddress")))
00819 #else

```

```

00820 #define GTEST_ATTRIBUTE_NO_SANITIZE_HWADDRESS_
00821 #endif // __has_feature(hwaddress_sanitizer)
00822 #else
00823 #define GTEST_ATTRIBUTE_NO_SANITIZE_HWADDRESS_
00824 #endif // __clang__
00825
00826 // A function level attribute to disable ThreadSanitizer instrumentation.
00827 #if defined(__clang__)
00828 #if __has_feature(thread_sanitizer)
00829 #define GTEST_ATTRIBUTE_NO_SANITIZE_THREAD_ __attribute__((no_sanitize_thread))
00830 #else
00831 #define GTEST_ATTRIBUTE_NO_SANITIZE_THREAD_
00832 #endif // __has_feature(thread_sanitizer)
00833 #else
00834 #define GTEST_ATTRIBUTE_NO_SANITIZE_THREAD_
00835 #endif // __clang__
00836
00837 namespace testing {
00838
00839 class Message;
00840
00841 // Legacy imports for backwards compatibility.
00842 // New code should use std:: names directly.
00843 using std::get;
00844 using std::make_tuple;
00845 using std::tuple;
00846 using std::tuple_element;
00847 using std::tuple_size;
00848
00849 namespace internal {
00850
00851 // A secret type that Google Test users don't know about. It has no
00852 // definition on purpose. Therefore it's impossible to create a
00853 // Secret object, which is what we want.
00854 class Secret;
00855
00856 // A helper for suppressing warnings on constant condition. It just
00857 // returns 'condition'.
00858 GTEST_API_ bool IsTrue(bool condition);
00859
00860 // Defines RE.
00861
00862 #if GTEST_USES_RE2
00863
00864 // This is almost `using RE = ::RE2`, except it is copy-constructible, and it
00865 // needs to disambiguate the `std::string`, `absl::string_view`, and `const
00866 // char*` constructors.
00867
00868 class GTEST_API_ RE {
00869 public:
00870
00871 RE(absl::string_view regex) : regex_(regex) {
00872 } // NOLINT
00873
00874 RE(const char* regex) : RE(absl::string_view(regex)) {
00875 } // NOLINT
00876
00877 RE(const std::string& regex) : RE(absl::string_view(regex)) {
00878 } // NOLINT
00879
00880 RE(const RE& other) : RE(other.pattern()) {
00881 }
00882
00883 const std::string& pattern() const {
00884 return regex_.pattern();
00885 }
00886
00887 static bool FullMatch(absl::string_view str, const RE& re) {
00888 return RE2::FullMatch(str, re.regex_);
00889 }
00890
00891 static bool PartialMatch(absl::string_view str, const RE& re) {
00892 return RE2::PartialMatch(str, re.regex_);
00893 }
00894
00895 private:
00896 RE2 regex_;
00897 };
00898
00899 #elif GTEST_USES_POSIX_RE || GTEST_USES_SIMPLE_RE
00900
00901 // A simple C++ wrapper for <regex.h>. It uses the POSIX Extended
00902 // Regular Expression syntax.
00903
00904 class GTEST_API_ RE {
00905 public:
00906 // A copy constructor is required by the Standard to initialize object

```

```

00907 // references from r-values.
00908
00909 RE(const RE& other) {
00910 Init(other.pattern());
00911 }
00912
00913 // Constructs an RE from a string.
00914
00915 RE(const ::std::string& regex) {
00916 Init(regex.c_str());
00917 } // NOLINT
00918
00919 RE(const char* regex) {
00920 Init(regex);
00921 } // NOLINT
00922 ~RE();
00923
00924 // Returns the string representation of the regex.
00925
00926 const char* pattern() const {
00927 return pattern_;
00928 }
00929
00930 // FullMatch(str, re) returns true if and only if regular expression re
00931 // matches the entire str.
00932 // PartialMatch(str, re) returns true if and only if regular expression re
00933 // matches a substring of str (including str itself).
00934
00935 static bool FullMatch(const ::std::string& str, const RE& re) {
00936 return FullMatch(str.c_str(), re);
00937 }
00938
00939 static bool PartialMatch(const ::std::string& str, const RE& re) {
00940 return PartialMatch(str.c_str(), re);
00941 }
00942
00943 static bool FullMatch(const char* str, const RE& re);
00944 static bool PartialMatch(const char* str, const RE& re);
00945
00946 private:
00947 void Init(const char* regex);
00948 const char* pattern_;
00949 bool is_valid_;
00950
00951 #if GTEST_USES_POSIX_RE
00952
00953 regex_t full_regex_; // For FullMatch().
00954 regex_t partial_regex_; // For PartialMatch().
00955
00956 #else // GTEST_USES_SIMPLE_RE
00957
00958 const char* full_pattern_; // For FullMatch();
00959
00960 #endif
00961 };
00962
00963 #endif // ::testing::internal::RE implementation
00964
00965 // Formats a source file path and a line number as they would appear
00966 // in an error message from the compiler used to compile this code.
00967 GTEST_API_ ::std::string FormatFileLocation(const char* file, int line);
00968
00969 // Formats a file location for compiler-independent XML output.
00970 // Although this function is not platform dependent, we put it next to
00971 // FormatFileLocation in order to contrast the two functions.
00972 GTEST_API_ ::std::string FormatCompilerIndependentFileLocation(const char* file,
00973 int line);
00974
00975 // Defines logging utilities:
00976 // GTEST_LOG_(severity) - logs messages at the specified severity level. The
00977 // message itself is streamed into the macro.
00978 // LogToStderr() - directs all log messages to stderr.
00979 // FlushInfoLog() - flushes informational log messages.
00980
00981 enum GTestLogSeverity {
00982 GTEST_INFO, GTEST_WARNING, GTEST_ERROR, GTEST_FATAL
00983 };
00984
00985 // Formats log entry severity, provides a stream object for streaming the
00986 // log message, and terminates the message with a newline when going out of
00987 // scope.
00988
00989 class GTEST_API_ GTestLog {
00990 public:
00991 GTestLog(GTestLogSeverity severity, const char* file, int line);
00992
00993 // Flushes the buffers and, if severity is GTEST_FATAL, aborts the program.

```

```

00994 ~GTestLog();
00995
00996 ::std::ostream& GetStream() {
00997 return ::std::cerr;
00998 }
00999
01000 private:
01001 const GTestLogSeverity severity_;
01002
01003 GTestLog(const GTestLog&) = delete;
01004 GTestLog& operator=(const GTestLog&) = delete;
01005 };
01006
01007 #if !defined(GTEST_LOG_)
01008
01009 #define GTEST_LOG_(severity) \
01010 ::testing::internal::GTestLog(::testing::internal::GTEST_##severity, \
01011 __FILE__, __LINE__) \
01012 .GetStream()
01013
01014 inline void LogToStderr() {
01015 }
01016
01017 inline void FlushInfoLog() {
01018 fflush(nullptr);
01019 }
01020
01021 #endif // !defined(GTEST_LOG_)
01022
01023 #if !defined(GTEST_CHECK_)
01024 // INTERNAL IMPLEMENTATION - DO NOT USE.
01025 //
01026 // GTEST_CHECK_ is an all-mode assert. It aborts the program if the condition
01027 // is not satisfied.
01028 // Synopsis:
01029 // GTEST_CHECK_(boolean_condition);
01030 // or
01031 // GTEST_CHECK_(boolean_condition) << "Additional message";
01032 //
01033 // This checks the condition and if the condition is not satisfied
01034 // it prints message about the condition violation, including the
01035 // condition itself, plus additional message streamed into it, if any,
01036 // and then it aborts the program. It aborts the program irrespective of
01037 // whether it is built in the debug mode or not.
01038 #define GTEST_CHECK_(condition) \
01039 GTEST_AMBIGUOUS_ELSE_BLOCKER_ \
01040 if (::testing::internal::IsTrue(condition)) \
01041 ; \
01042 else \
01043 GTEST_LOG_(FATAL) << "Condition " #condition " failed. "
01044 #endif // !defined(GTEST_CHECK_)
01045
01046 // An all-mode assert to verify that the given POSIX-style function
01047 // call returns 0 (indicating success). Known limitation: this
01048 // doesn't expand to a balanced 'if' statement, so enclose the macro
01049 // in {} if you need to use it as the only statement in an 'if'
01050 // branch.
01051 #define GTEST_CHECK_POSIX_SUCCESS_(posix_call) \
01052 if (const int gtest_error = (posix_call)) \
01053 GTEST_LOG_(FATAL) << #posix_call << "failed with error " << gtest_error
01054
01055 // Transforms "T" into "const T&" according to standard reference collapsing
01056 // rules (this is only needed as a backport for C++98 compilers that do not
01057 // support reference collapsing). Specifically, it transforms:
01058 //
01059 // char ==> const char&
01060 // const char ==> const char&
01061 // char& ==> char&
01062 // const char& ==> const char&
01063 //
01064 // Note that the non-const reference will not have "const" added. This is
01065 // standard, and necessary so that "T" can always bind to "const T&".
01066
01067 template <typename T>
01068 struct ConstRef {
01069 typedef const T& type;
01070 };
01071
01072 template <typename T>
01073 struct ConstRef<T&> {
01074 typedef T& type;
01075 };
01076
01077 // The argument T must depend on some template parameters.
01078 #define GTEST_REFERENCE_TO_CONST_(T) \
01079 typename ::testing::internal::ConstRef<T>::type
01080

```

```

01081 // INTERNAL IMPLEMENTATION - DO NOT USE IN USER CODE.
01082 //
01083 // Use ImplicitCast_ as a safe version of static_cast for upcasting in
01084 // the type hierarchy (e.g. casting a Foo* to a SuperclassOfFoo* or a
01085 // const Foo*). When you use ImplicitCast_, the compiler checks that
01086 // the cast is safe. Such explicit ImplicitCast_s are necessary in
01087 // surprisingly many situations where C++ demands an exact type match
01088 // instead of an argument type convertible to a target type.
01089 //
01090 // The syntax for using ImplicitCast_ is the same as for static_cast:
01091 //
01092 // ImplicitCast_<ToType>(expr)
01093 //
01094 // ImplicitCast_ would have been part of the C++ standard library,
01095 // but the proposal was submitted too late. It will probably make
01096 // its way into the language in the future.
01097 //
01098 // This relatively ugly name is intentional. It prevents clashes with
01099 // similar functions users may have (e.g., implicit_cast). The internal
01100 // namespace alone is not enough because the function can be found by ADL.
01101
01102 template <typename To>
01103 inline To ImplicitCast_(To x) {
01104 return x;
01105 }
01106
01107 // When you upcast (that is, cast a pointer from type Foo to type
01108 // SuperclassOfFoo), it's fine to use ImplicitCast_<>, since upcasts
01109 // always succeed. When you downcast (that is, cast a pointer from
01110 // type Foo to type SubclassOfFoo), static_cast<> isn't safe, because
01111 // how do you know the pointer is really of type SubclassOfFoo? It
01112 // could be a bare Foo, or of type DifferentSubclassOfFoo. Thus,
01113 // when you downcast, you should use this macro. In debug mode, we
01114 // use dynamic_cast<> to double-check the downcast is legal (we die
01115 // if it's not). In normal mode, we do the efficient static_cast<>
01116 // instead. Thus, it's important to test in debug mode to make sure
01117 // the cast is legal!
01118 // This is the only place in the code we should use dynamic_cast<>.
01119 // In particular, you SHOULDN'T be using dynamic_cast<> in order to
01120 // do RTTI (eg code like this:
01121 // if (dynamic_cast<Subclass1>(foo)) HandleASubclass1Object(foo);
01122 // if (dynamic_cast<Subclass2>(foo)) HandleASubclass2Object(foo);
01123 // You should design the code some other way not to need this.
01124 //
01125 // This relatively ugly name is intentional. It prevents clashes with
01126 // similar functions users may have (e.g., down_cast). The internal
01127 // namespace alone is not enough because the function can be found by ADL.
01128
01129 template <typename To, typename From> // use like this: DownCast_<T*>(foo);
01130 inline To DownCast_(From* f) { // so we only accept pointers
01131 // Ensures that To is a sub-type of From *. This test is here only
01132 // for compile-time type checking, and has no overhead in an
01133 // optimized build at run-time, as it will be optimized away
01134 // completely.
01135 GTEST_INTENTIONAL_CONST_COND_PUSH()
01136 if (false) {
01137 GTEST_INTENTIONAL_CONST_COND_POP()
01138 const To to = nullptr;
01139 ::testing::internal::ImplicitCast_<From*>(to);
01140 }
01141
01142 #if GTEST_HAS_RTTI
01143 // RTTI: debug mode only!
01144 GTEST_CHECK_(f == nullptr || dynamic_cast<To>(f) != nullptr);
01145 #endif
01146 return static_cast<To>(f);
01147 }
01148
01149 // Downcasts the pointer of type Base to Derived.
01150 // Derived must be a subclass of Base. The parameter MUST
01151 // point to a class of type Derived, not any subclass of it.
01152 // When RTTI is available, the function performs a runtime
01153 // check to enforce this.
01154
01155 template <class Derived, class Base>
01156 Derived* CheckedDowncastToActualType(Base* base) {
01157 #if GTEST_HAS_RTTI
01158 GTEST_CHECK_(typeid(*base) == typeid(Derived));
01159 #endif
01160
01161 #if GTEST_HAS_DOWNCAST_
01162 return ::down_cast<Derived*>(base);
01163 #elif GTEST_HAS_RTTI
01164 return dynamic_cast<Derived*>(base); // NOLINT
01165 #else
01166 return static_cast<Derived*>(base); // Poor man's downcast.
01167 #endif

```

```

01168 }
01169
01170 #if GTEST_HAS_STREAM_REDIRECTION
01171
01172 // Defines the stderr capturer:
01173 // CaptureStdout - starts capturing stdout.
01174 // GetCapturedStdout - stops capturing stdout and returns the captured string.
01175 // CaptureStderr - starts capturing stderr.
01176 // GetCapturedStderr - stops capturing stderr and returns the captured string.
01177 //
01178 GTEST_API_ void CaptureStdout();
01179 GTEST_API_ std::string GetCapturedStdout();
01180 GTEST_API_ void CaptureStderr();
01181 GTEST_API_ std::string GetCapturedStderr();
01182
01183 #endif // GTEST_HAS_STREAM_REDIRECTION
01184 // Returns the size (in bytes) of a file.
01185 GTEST_API_ size_t GetFileSize(FILE* file);
01186
01187 // Reads the entire content of a file as a string.
01188 GTEST_API_ std::string ReadEntireFile(FILE* file);
01189
01190 // All command line arguments.
01191 GTEST_API_ std::vector<std::string> GetArgvs();
01192
01193 #if GTEST_HAS_DEATH_TEST
01194
01195 std::vector<std::string> GetInjectableArgvs();
01196 // Deprecated: pass the args vector by value instead.
01197 void SetInjectableArgvs(const std::vector<std::string>* new_argvs);
01198 void SetInjectableArgvs(const std::vector<std::string>& new_argvs);
01199 void ClearInjectableArgvs();
01200
01201 #endif // GTEST_HAS_DEATH_TEST
01202
01203 // Defines synchronization primitives.
01204 #if GTEST_IS_THREADSAFE
01205
01206 #if GTEST_OS_WINDOWS
01207 // Provides leak-safe Windows kernel handle ownership.
01208 // Used in death tests and in threading support.
01209
01210 class GTEST_API_ AutoHandle {
01211 public:
01212 // Assume that Win32 HANDLE type is equivalent to void*. Doing so allows us to
01213 // avoid including <windows.h> in this header file. Including <windows.h> is
01214 // undesirable because it defines a lot of symbols and macros that tend to
01215 // conflict with client code. This assumption is verified by
01216 // WindowsTypesTest.HANDLEIsVoidStar.
01217 typedef void* Handle;
01218 AutoHandle();
01219 explicit AutoHandle(Handle handle);
01220
01221 ~AutoHandle();
01222
01223 Handle Get() const;
01224 void Reset();
01225 void Reset(Handle handle);
01226
01227 private:
01228 // Returns true if and only if the handle is a valid handle object that can be
01229 // closed.
01230 bool IsCloseable() const;
01231
01232 Handle handle_;
01233
01234 AutoHandle(const AutoHandle&) = delete;
01235 AutoHandle& operator=(const AutoHandle&) = delete;
01236 };
01237 #endif
01238
01239 #if GTEST_HAS_NOTIFICATION_
01240 // Notification has already been imported into the namespace.
01241 // Nothing to do here.
01242 #else
01243 #endif
01244
01245 GTEST_DISABLE_MSC_WARNINGS_PUSH_(4251 \
01246 /* class A needs to have dll-interface to be used by clients of class B */)
01247
01248 // Allows a controller thread to pause execution of newly created
01249 // threads until notified. Instances of this class must be created
01250 // and destroyed in the controller thread.
01251 //
01252 // This class is only for testing Google Test's own constructs. Do not
01253 // use it in user tests, either directly or indirectly.
01254 // TODO(b/203539622): Replace unconditionally with absl::Notification.

```

```

01255 class GTEST_API_ Notification {
01256 public:
01257
01258 Notification() : notified_(false) {
01259 }
01260 Notification(const Notification&) = delete;
01261 Notification& operator=(const Notification&) = delete;
01262
01263 // Notifies all threads created with this notification to start. Must
01264 // be called from the controller thread.
01265
01266 void Notify() {
01267 std::lock_guard<std::mutex> lock(mu_);
01268 notified_ = true;
01269 cv_.notify_all();
01270 }
01271
01272 // Blocks until the controller thread notifies. Must be called from a test
01273 // thread.
01274
01275 void WaitForNotification() {
01276 std::unique_lock<std::mutex> lock(mu_);
01277 cv_.wait(lock, [this]() {
01278 return notified_; });
01279 }
01280
01281 private:
01282 std::mutex mu_;
01283 std::condition_variable cv_;
01284 bool notified_;
01285 };
01286
01287 GTEST_DISABLE_MSC_WARNINGS_POP_() // 4251
01288 #endif // GTEST_HAS_NOTIFICATION_
01289
01290 // On MinGW, we can have both GTEST_OS_WINDOWS and GTEST_HAS_PTHREAD
01291 // defined, but we don't want to use MinGW's pthreads implementation, which
01292 // has conformance problems with some versions of the POSIX standard.
01293 #if GTEST_HAS_PTHREAD && !GTEST_OS_WINDOWS_MINGW
01294
01295 // As a C-function, ThreadFuncWithCLinkage cannot be templated itself.
01296 // Consequently, it cannot select a correct instantiation of ThreadWithParam
01297 // in order to call its Run(). Introducing ThreadWithParamBase as a
01298 // non-templated base class for ThreadWithParam allows us to bypass this
01299 // problem.
01300 class ThreadWithParamBase {
01301 public:
01302
01303 virtual ~ThreadWithParamBase() {}
01304
01305 virtual void Run() = 0;
01306 };
01307
01308 // pthread_create() accepts a pointer to a function type with the C linkage.
01309 // According to the Standard (7.5/1), function types with different linkages
01310 // are different even if they are otherwise identical. Some compilers (for
01311 // example, SunStudio) treat them as different types. Since class methods
01312 // cannot be defined with C-linkage we need to define a free C-function to
01313 // pass into pthread_create().
01314
01315 extern "C" inline void* ThreadFuncWithCLinkage(void* thread) {
01316 static_cast<ThreadWithParamBase*>(thread)->Run();
01317 return nullptr;
01318 }
01319
01320 // Helper class for testing Google Test's multi-threading constructs.
01321 // To use it, write:
01322 //
01323 // void ThreadFunc(int param) { /* Do things with param */ }
01324 // Notification thread_can_start;
01325 // ...
01326 // // The thread_can_start parameter is optional; you can supply NULL.
01327 // ThreadWithParam<int> thread(&ThreadFunc, 5, &thread_can_start);
01328 // thread_can_start.Notify();
01329 //
01330 // These classes are only for testing Google Test's own constructs. Do
01331 // not use them in user tests, either directly or indirectly.
01332
01333 template <typename T>
01334 class ThreadWithParam : public ThreadWithParamBase {
01335 public:
01336 typedef void UserThreadFunc(T);
01337
01338 ThreadWithParam(UserThreadFunc* func, T param, Notification* thread_can_start)
01339 : func_(func),
01340 param_(param),
01341 thread_can_start_(thread_can_start),

```

```

01342 finished_(false) {
01343 ThreadWithParamBase * const base = this;
01344 // The thread can be created only after all fields except thread_
01345 // have been initialized.
01346 GTEST_CHECK_POSIX_SUCCESS_(
01347 pthread_create(&thread_, nullptr, &ThreadFuncWithCLinkage, base));
01348 }
01349
01350 ~ThreadWithParam() override {
01351 Join();
01352 }
01353
01354 void Join() {
01355 if (!finished_) {
01356 GTEST_CHECK_POSIX_SUCCESS_(pthread_join(thread_, nullptr));
01357 finished_ = true;
01358 }
01359 }
01360
01361 void Run() override {
01362 if (thread_can_start_ != nullptr) thread_can_start_>WaitForNotification();
01363 func_(param_);
01364 }
01365
01366 private:
01367 UserThreadFunc * const func_; // User-supplied thread function.
01368 const T param_; // User-supplied parameter to the thread function.
01369 // When non-NULL, used to block execution until the controller thread
01370 // notifies.
01371 Notification * const thread_can_start_;
01372 bool finished_; // true if and only if we know that the thread function has
01373 // finished.
01374 pthread_t thread_; // The native thread object.
01375
01376 ThreadWithParam(const ThreadWithParam&) = delete;
01377 ThreadWithParam& operator=(const ThreadWithParam&) = delete;
01378 };
01379 #endif // !GTEST_OS_WINDOWS && GTEST_HAS_PTHREAD ||
01380 // GTEST_HAS_MUTEX_AND_THREAD_LOCAL_
01381
01382 #if GTEST_HAS_MUTEX_AND_THREAD_LOCAL_
01383 // Mutex and ThreadLocal have already been imported into the namespace.
01384 // Nothing to do here.
01385
01386 #elif GTEST_OS_WINDOWS && !GTEST_OS_WINDOWS_PHONE && !GTEST_OS_WINDOWS_RT
01387
01388 // Mutex implements mutex on Windows platforms. It is used in conjunction
01389 // with class MutexLock:
01390 //
01391 // Mutex mutex;
01392 // ...
01393 // MutexLock lock(&mutex); // Acquires the mutex and releases it at the
01394 // // end of the current scope.
01395 //
01396 // A static Mutex *must* be defined or declared using one of the following
01397 // macros:
01398 // GTEST_DEFINE_STATIC_MUTEX_(g_some_mutex);
01399 // GTEST_DECLARE_STATIC_MUTEX_(g_some_mutex);
01400 //
01401 // (A non-static Mutex is defined/declared in the usual way).
01402
01403 class GTEST_API_ Mutex {
01404 public:
01405
01406 enum MutexType {
01407 kStatic = 0, kDynamic = 1
01408 };
01409 // We rely on kStaticMutex being 0 as it is to what the linker initializes
01410 // type_ in static mutexes. critical_section_ will be initialized lazily
01411 // in ThreadSafeLazyInit().
01412
01413 enum StaticConstructorSelector {
01414 kStaticMutex = 0
01415 };
01416
01417 // This constructor intentionally does nothing. It relies on type_ being
01418 // statically initialized to 0 (effectively setting it to kStatic) and on
01419 // ThreadSafeLazyInit() to lazily initialize the rest of the members.
01420
01421 explicit Mutex(StaticConstructorSelector /*dummy*/) {
01422 }
01423
01424 Mutex();
01425 ~Mutex();
01426
01427 void Lock();
01428
```



```

01429 void Unlock();
01430
01431 // Does nothing if the current thread holds the mutex. Otherwise, crashes
01432 // with high probability.
01433 void AssertHeld();
01434
01435 private:
01436 // Initializes owner_thread_id_ and critical_section_ in static mutexes.
01437 void ThreadSafeLazyInit();
01438
01439 // Per https://blogs.msdn.microsoft.com/oldnewthing/20040223-00/?p=40503,
01440 // we assume that 0 is an invalid value for thread IDs.
01441 unsigned int owner_thread_id_;
01442
01443 // For static mutexes, we rely on these members being initialized to zeros
01444 // by the linker.
01445 MutexType type_;
01446 long critical_section_init_phase_; // NOLINT
01447 GTEST_CRITICAL_SECTION* critical_section_;
01448
01449 Mutex(const Mutex&) = delete;
01450 Mutex& operator=(const Mutex&) = delete;
01451 };
01452
01453 #define GTEST_DECLARE_STATIC_MUTEX_(mutex) \
01454 extern ::testing::internal::Mutex mutex
01455
01456 #define GTEST_DEFINE_STATIC_MUTEX_(mutex) \
01457 ::testing::internal::Mutex mutex(::testing::internal::Mutex::kStaticMutex)
01458
01459 // We cannot name this class MutexLock because the ctor declaration would
01460 // conflict with a macro named MutexLock, which is defined on some
01461 // platforms. That macro is used as a defensive measure to prevent against
01462 // inadvertent misuses of MutexLock like "MutexLock(&mu)" rather than
01463 // "MutexLock l(&mu)". Hence the typedef trick below.
01464
01465 class GTestMutexLock {
01466 public:
01467
01468 explicit GTestMutexLock(Mutex* mutex) : mutex_(mutex) {
01469 mutex_>Lock();
01470 }
01471
01472 ~GTestMutexLock() {
01473 mutex_>Unlock();
01474 }
01475
01476 private:
01477 Mutex * const mutex_;
01478
01479 GTestMutexLock(const GTestMutexLock&) = delete;
01480 GTestMutexLock& operator=(const GTestMutexLock&) = delete;
01481 };
01482
01483 typedef GTestMutexLock MutexLock;
01484
01485 // Base class for ValueHolder<T>. Allows a caller to hold and delete a value
01486 // without knowing its type.
01487
01488 class ThreadLocalValueHolderBase {
01489 public:
01490
01491 virtual ~ThreadLocalValueHolderBase() {
01492 }
01493 };
01494
01495 // Provides a way for a thread to send notifications to a ThreadLocal
01496 // regardless of its parameter type.
01497
01498 class ThreadLocalBase {
01499 public:
01500 // Creates a new ValueHolder<T> object holding a default value passed to
01501 // this ThreadLocal<T>'s constructor and returns it. It is the caller's
01502 // responsibility not to call this when the ThreadLocal<T> instance already
01503 // has a value on the current thread.
01504 virtual ThreadLocalValueHolderBase* NewValueForCurrentThread() const = 0;
01505
01506 protected:
01507
01508 ThreadLocalBase() {
01509 }
01510
01511 virtual ~ThreadLocalBase() {
01512 }
01513
01514 private:
01515 ThreadLocalBase(const ThreadLocalBase&) = delete;

```

```

01516 ThreadLocalBase& operator=(const ThreadLocalBase&) = delete;
01517 };
01518
01519 // Maps a thread to a set of ThreadLocals that have values instantiated on that
01520 // thread and notifies them when the thread exits. A ThreadLocal instance is
01521 // expected to persist until all threads it has values on have terminated.
01522
01523 class GTEST_API_ ThreadLocalRegistry {
01524 public:
01525 // Registers thread_local_instance as having value on the current thread.
01526 // Returns a value that can be used to identify the thread from other threads.
01527 static ThreadLocalValueHolderBase* GetValueOnCurrentThread(
01528 const ThreadLocalBase* thread_local_instance);
01529
01530 // Invoked when a ThreadLocal instance is destroyed.
01531 static void OnThreadLocalDestroyed(
01532 const ThreadLocalBase* thread_local_instance);
01533 };
01534
01535 class GTEST_API_ ThreadWithParamBase {
01536 public:
01537 void Join();
01538
01539 protected:
01540
01541 class Runnable {
01542 public:
01543 virtual ~Runnable() {}
01544 virtual void Run() = 0;
01545 };
01546
01547 ThreadWithParamBase(Runnable* runnable, Notification* thread_can_start);
01548 virtual ~ThreadWithParamBase();
01549
01550 private:
01551 AutoHandle thread_;
01552 };
01553
01554 // Helper class for testing Google Test's multi-threading constructs.
01555
01556 template <typename T>
01557 class ThreadWithParam : public ThreadWithParamBase {
01558 public:
01559 typedef void UserThreadFunc(T);
01560
01561 ThreadWithParam(UserThreadFunc* func, T param, Notification* thread_can_start)
01562 : ThreadWithParamBase(new RunnableImpl(func, param), thread_can_start) {}
01563
01564 virtual ~ThreadWithParam() {}
01565
01566 private:
01567 class RunnableImpl : public Runnable {
01568 public:
01569 RunnableImpl(UserThreadFunc* func, T param) : func_(func), param_(param) {}
01570
01571 virtual ~RunnableImpl() {}
01572
01573 virtual void Run() {
01574 func_(param_);
01575 }
01576
01577 private:
01578 UserThreadFunc * const func_;
01579 const T param_;
01580
01581 RunnableImpl(const RunnableImpl&) = delete;
01582 RunnableImpl& operator=(const RunnableImpl&) = delete;
01583 };
01584
01585 ThreadWithParam(const ThreadWithParam&) = delete;
01586 ThreadWithParam& operator=(const ThreadWithParam&) = delete;
01587 };
01588
01589 // Implements thread-local storage on Windows systems.
01590 //
01591 // Thread 1
01592 // ThreadLocal<int> tl(100); // 100 is the default value for each thread.
01593 //
01594 // Thread 2

```

```

01603 // t1.set(150); // Changes the value for thread 2 only.
01604 // EXPECT_EQ(150, t1.get());
01605 //
01606 // // Thread 1
01607 // EXPECT_EQ(100, t1.get()); // In thread 1, t1 has the original value.
01608 // t1.set(200);
01609 // EXPECT_EQ(200, t1.get());
01610 //
01611 // The template type argument T must have a public copy constructor.
01612 // In addition, the default ThreadLocal constructor requires T to have
01613 // a public default constructor.
01614 //
01615 // The users of a ThreadLocal instance have to make sure that all but one
01616 // threads (including the main one) using that instance have exited before
01617 // destroying it. Otherwise, the per-thread objects managed for them by the
01618 // ThreadLocal instance are not guaranteed to be destroyed on all platforms.
01619 //
01620 // Google Test only uses global ThreadLocal objects. That means they
01621 // will die after main() has returned. Therefore, no per-thread
01622 // object managed by Google Test will be leaked as long as all threads
01623 // using Google Test have exited when main() returns.
01624
01625 template <typename T>
01626 class ThreadLocal : public ThreadLocalBase {
01627 public:
01628
01629 ThreadLocal() : default_factory_(new DefaultValueHolderFactory()) {
01630 }
01631
01632 explicit ThreadLocal(const T& value)
01633 : default_factory_(new InstanceValueHolderFactory(value)) {
01634 }
01635
01636 ~ThreadLocal() override {
01637 ThreadLocalRegistry::OnThreadLocalDestroyed(this);
01638 }
01639
01640 T* pointer() {
01641 return GetOrCreateValue();
01642 }
01643
01644 const T* pointer() const {
01645 return GetOrCreateValue();
01646 }
01647
01648 const T& get() const {
01649 return *pointer();
01650 }
01651
01652 void set(const T& value) {
01653 *pointer() = value;
01654 }
01655
01656 private:
01657 // Holds a value of T. Can be deleted via its base class without the caller
01658 // knowing the type of T.
01659
01660 class ValueHolder : public ThreadLocalValueHolderBase {
01661 public:
01662
01663 ValueHolder() : value_() {
01664 }
01665
01666 explicit ValueHolder(const T& value) : value_(value) {
01667 }
01668
01669 T* pointer() {
01670 return &value_;
01671 }
01672
01673 private:
01674 T value_;
01675 ValueHolder(const ValueHolder&) = delete;
01676 ValueHolder& operator=(const ValueHolder&) = delete;
01677 };
01678
01679 T* GetOrCreateValue() const {
01680 return static_cast<ValueHolder*> (
01681 ThreadLocalRegistry::GetValueOnCurrentThread(this))
01682 ->pointer();
01683 }
01684
01685 ThreadLocalValueHolderBase* NewValueForCurrentThread() const override {
01686 return default_factory_->MakeNewHolder();
01687 }
01688
01689 class ValueHolderFactory {

```

```

01690 public:
01691
01692 ValueHolderFactory() {
01693 }
01694
01695 virtual ~ValueHolderFactory() {
01696 }
01697 virtual ValueHolder* MakeNewHolder() const = 0;
01698
01699 private:
01700 ValueHolderFactory(const ValueHolderFactory&) = delete;
01701 ValueHolderFactory& operator=(const ValueHolderFactory&) = delete;
01702 };
01703
01704 class DefaultValueHolderFactory : public ValueHolderFactory {
01705 public:
01706
01707 DefaultValueHolderFactory() {
01708 }
01709
01710 ValueHolder* MakeNewHolder() const override {
01711 return new ValueHolder();
01712 }
01713
01714 private:
01715 DefaultValueHolderFactory(const DefaultValueHolderFactory&) = delete;
01716 DefaultValueHolderFactory& operator=(const DefaultValueHolderFactory&) =
01717 delete;
01718 };
01719
01720 class InstanceValueHolderFactory : public ValueHolderFactory {
01721 public:
01722
01723 explicit InstanceValueHolderFactory(const T& value) : value_(value) {
01724 }
01725
01726 ValueHolder* MakeNewHolder() const override {
01727 return new ValueHolder(value_);
01728 }
01729
01730 private:
01731 const T value_; // The value for each thread.
01732
01733 InstanceValueHolderFactory(const InstanceValueHolderFactory&) = delete;
01734 InstanceValueHolderFactory& operator=(const InstanceValueHolderFactory&) =
01735 delete;
01736 };
01737
01738 std::unique_ptr<ValueHolderFactory> default_factory_;
01739
01740 ThreadLocal(const ThreadLocal&) = delete;
01741 ThreadLocal& operator=(const ThreadLocal&) = delete;
01742 };
01743
01744 #elif GTEST_HAS_PTHREAD
01745
01746 // MutexBase and Mutex implement mutex on pthreads-based platforms.
01747
01748 class MutexBase {
01749 public:
01750 // Acquires this mutex.
01751
01752 void Lock() {
01753 GTEST_CHECK_POSIX_SUCCESS_(pthread_mutex_lock(&mutex_));
01754 owner_ = pthread_self();
01755 has_owner_ = true;
01756 }
01757
01758 // Releases this mutex.
01759
01760 void Unlock() {
01761 // Since the lock is being released the owner_ field should no longer be
01762 // considered valid. We don't protect writing to has_owner_ here, as it's
01763 // the caller's responsibility to ensure that the current thread holds the
01764 // mutex when this is called.
01765 has_owner_ = false;
01766 GTEST_CHECK_POSIX_SUCCESS_(pthread_mutex_unlock(&mutex_));
01767 }
01768
01769 // Does nothing if the current thread holds the mutex. Otherwise, crashes
01770 // with high probability.
01771
01772 void AssertHeld() const {
01773 GTEST_CHECK_(has_owner_ && pthread_equal(owner_, pthread_self()))
01774 << "The current thread is not holding the mutex @" << this;
01775 }
01776

```

```

01777 // A static mutex may be used before main() is entered. It may even
01778 // be used before the dynamic initialization stage. Therefore we
01779 // must be able to initialize a static mutex object at link time.
01780 // This means MutexBase has to be a POD and its member variables
01781 // have to be public.
01782 public:
01783 pthread_mutex_t mutex_; // The underlying pthread mutex.
01784 // has_owner_ indicates whether the owner_ field below contains a valid thread
01785 // ID and is therefore safe to inspect (e.g., to use in pthread_equal()). All
01786 // accesses to the owner_ field should be protected by a check of this field.
01787 // An alternative might be to memset() owner_ to all zeros, but there's no
01788 // guarantee that a zero'd pthread_t is necessarily invalid or even different
01789 // from pthread_self().
01790 bool has_owner_;
01791 pthread_t owner_; // The thread holding the mutex.
01792 };
01793
01794 // Forward-declares a static mutex.
01795 #define GTEST_DECLARE_STATIC_MUTEX_(mutex) \
01796 extern ::testing::internal::MutexBase mutex
01797
01798 // Defines and statically (i.e. at link time) initializes a static mutex.
01799 // The initialization list here does not explicitly initialize each field,
01800 // instead relying on default initialization for the unspecified fields. In
01801 // particular, the owner_ field (a pthread_t) is not explicitly initialized.
01802 // This allows initialization to work whether pthread_t is a scalar or struct.
01803 // The flag -Wmissing-field-initializers must not be specified for this to work.
01804 #define GTEST_DEFINE_STATIC_MUTEX_(mutex) \
01805 ::testing::internal::MutexBase mutex = {PTHREAD_MUTEX_INITIALIZER, false, 0}
01806
01807 // The Mutex class can only be used for mutexes created at runtime. It
01808 // shares its API with MutexBase otherwise.
01809
01810 class Mutex : public MutexBase {
01811 public:
01812 Mutex() {
01813 GTEST_CHECK_POSIX_SUCCESS_(pthread_mutex_init(&mutex_, nullptr));
01814 has_owner_ = false;
01815 }
01816
01817 ~Mutex() {
01818 GTEST_CHECK_POSIX_SUCCESS_(pthread_mutex_destroy(&mutex_));
01819 }
01820
01821 private:
01822 Mutex(const Mutex&) = delete;
01823 Mutex& operator=(const Mutex&) = delete;
01824 };
01825
01826 // We cannot name this class MutexLock because the ctor declaration would
01827 // conflict with a macro named MutexLock, which is defined on some
01828 // platforms. That macro is used as a defensive measure to prevent against
01829 // inadvertent misuses of MutexLock like "MutexLock(&mu)" rather than
01830 // "MutexLock l(&mu)". Hence the typedef trick below.
01831
01832 class GTestMutexLock {
01833 public:
01834 explicit GTestMutexLock(MutexBase* mutex) : mutex_(mutex) {
01835 mutex_>Lock();
01836 }
01837
01838 ~GTestMutexLock() {
01839 mutex_>Unlock();
01840 }
01841
01842 private:
01843 MutexBase * const mutex_;
01844
01845 GTestMutexLock(const GTestMutexLock&) = delete;
01846 GTestMutexLock& operator=(const GTestMutexLock&) = delete;
01847 };
01848
01849 typedef GTestMutexLock MutexLock;
01850
01851 // Helpers for ThreadLocal.
01852
01853 // pthread_key_create() requires DeleteThreadLocalValue() to have
01854 // C-linkage. Therefore it cannot be templated to access
01855 // ThreadLocal<T>. Hence the need for class
01856 // ThreadLocalValueHolderBase.
01857
01858 class GTEST_API_ ThreadLocalValueHolderBase {
01859 public:
01860 virtual ~ThreadLocalValueHolderBase() {

```

```

01864 }
01865 };
01866
01867 // Called by pthread to delete thread-local data stored by
01868 // pthread_setspecific().
01869
01870 extern "C" inline void DeleteThreadLocalValue(void* value_holder) {
01871 delete static_cast<ThreadLocalValueHolderBase*> (value_holder);
01872 }
01873
01874 // Implements thread-local storage on pthreads-based systems.
01875
01876 template <typename T>
01877 class GTEST_API_ ThreadLocal {
01878 public:
01879 ThreadLocal()
01880 : key_(CreateKey()), default_factory_(new DefaultValueHolderFactory()) {
01881 }
01882
01883 explicit ThreadLocal(const T& value)
01884 : key_(CreateKey()),
01885 default_factory_(new InstanceValueHolderFactory(value)) {
01886 }
01887
01888 ~ThreadLocal() {
01889 // Destroys the managed object for the current thread, if any.
01890 DeleteThreadLocalValue(pthread_getspecific(key_));
01891
01892 // Releases resources associated with the key. This will *not*
01893 // delete managed objects for other threads.
01894 GTEST_CHECK_POSIX_SUCCESS_(pthread_key_delete(key_));
01895 }
01896
01897 T* pointer() {
01898 return GetOrCreateValue();
01899 }
01900
01901 const T* pointer() const {
01902 return GetOrCreateValue();
01903 }
01904
01905 const T& get() const {
01906 return *pointer();
01907 }
01908
01909 void set(const T& value) {
01910 *pointer() = value;
01911 }
01912
01913 private:
01914 // Holds a value of type T.
01915
01916 class ValueHolder : public ThreadLocalValueHolderBase {
01917 public:
01918 ValueHolder() : value_() {
01919 }
01920
01921 explicit ValueHolder(const T& value) : value_(value) {
01922 }
01923
01924 T* pointer() {
01925 return &value_;
01926 }
01927
01928 private:
01929 T value_;
01930 ValueHolder(const ValueHolder&) = delete;
01931 ValueHolder& operator=(const ValueHolder&) = delete;
01932 };
01933
01934 static pthread_key_t CreateKey() {
01935 pthread_key_t key;
01936 // When a thread exits, DeleteThreadLocalValue() will be called on
01937 // the object managed for that thread.
01938 GTEST_CHECK_POSIX_SUCCESS_(
01939 pthread_key_create(&key, &DeleteThreadLocalValue));
01940 return key;
01941 }
01942
01943 T* GetOrCreateValue() const {
01944 ThreadLocalValueHolderBase * const holder =
01945 static_cast<ThreadLocalValueHolderBase*> (pthread_getspecific(key_));
01946 if (holder != nullptr) {
01947 return CheckedDowncastToActualType<ValueHolder>(holder)->pointer();
01948 }
01949 }
01950

```

```

01951
01952 ValueHolder * const new_holder = default_factory_>MakeNewHolder();
01953 ThreadLocalValueHolderBase * const holder_base = new_holder;
01954 GTEST_CHECK_POSIX_SUCCESS_(pthread_setspecific(key_, holder_base));
01955 return new_holder->pointer();
01956 }
01957
01958 class ValueHolderFactory {
01959 public:
01960 ValueHolderFactory() {}
01961
01962 virtual ~ValueHolderFactory() {}
01963
01964 virtual ValueHolder* MakeNewHolder() const = 0;
01965
01966 private:
01967 ValueHolderFactory(const ValueHolderFactory&) = delete;
01968 ValueHolderFactory& operator=(const ValueHolderFactory&) = delete;
01969 };
01970
01971 class DefaultValueHolderFactory : public ValueHolderFactory {
01972 public:
01973 DefaultValueHolderFactory() {}
01974
01975 ValueHolder* MakeNewHolder() const override {
01976 return new ValueHolder();
01977 }
01978
01979 private:
01980 DefaultValueHolderFactory(const DefaultValueHolderFactory&) = delete;
01981 DefaultValueHolderFactory& operator=(const DefaultValueHolderFactory&) =
01982 delete;
01983 };
01984
01985 class InstanceValueHolderFactory : public ValueHolderFactory {
01986 public:
01987 explicit InstanceValueHolderFactory(const T& value) : value_(value) {}
01988
01989 ValueHolder* MakeNewHolder() const override {
01990 return new ValueHolder(value_);
01991 }
01992
01993 private:
01994 const T value_; // The value for each thread.
01995
01996 InstanceValueHolderFactory(const InstanceValueHolderFactory&) = delete;
01997 InstanceValueHolderFactory& operator=(const InstanceValueHolderFactory&) =
01998 delete;
01999 };
02000
02001 // A key pthreads uses for looking up per-thread values.
02002 const pthread_key_t key_;
02003 std::unique_ptr<ValueHolderFactory> default_factory_;
02004
02005 ThreadLocal(const ThreadLocal&) = delete;
02006 ThreadLocal& operator=(const ThreadLocal&) = delete;
02007 };
02008
02009 #endif // GTEST_HAS_MUTEX_AND_THREAD_LOCAL_
02010
02011 #else // GTEST_IS_THREADS
02012
02013 // A dummy implementation of synchronization primitives (mutex, lock,
02014 // and thread-local variable). Necessary for compiling Google Test where
02015 // mutex is not supported - using Google Test in multiple threads is not
02016 // supported on such platforms.
02017
02018 class Mutex {
02019 public:
02020 Mutex() {}
02021
02022 void Lock() {}
02023
02024 void Unlock() {}
02025
02026 void AssertHeld() const {}
02027 };

```

```

02038 };
02039
02040 #define GTEST_DECLARE_STATIC_MUTEX_(mutex) \
02041 extern ::testing::internal::Mutex mutex
02042
02043 #define GTEST_DEFINE_STATIC_MUTEX_(mutex) ::testing::internal::Mutex mutex
02044
02045 // We cannot name this class MutexLock because the ctor declaration would
02046 // conflict with a macro named MutexLock, which is defined on some
02047 // platforms. That macro is used as a defensive measure to prevent against
02048 // inadvertent misuses of MutexLock like "MutexLock(&mu)" rather than
02049 // "MutexLock l(&mu)". Hence the typedef trick below.
02050
02051 class GTestMutexLock {
02052 public:
02053
02054 explicit GTestMutexLock(Mutex*) {
02055 } // NOLINT
02056 };
02057
02058 typedef GTestMutexLock MutexLock;
02059
02060 template <typename T>
02061 class GTEST_API_ ThreadLocal {
02062 public:
02063
02064 ThreadLocal() : value_() {
02065 }
02066
02067 explicit ThreadLocal(const T& value) : value_(value) {
02068 }
02069
02070 T* pointer() {
02071 return &value_;
02072 }
02073
02074 const T* pointer() const {
02075 return &value_;
02076 }
02077
02078 const T& get() const {
02079 return value_;
02080 }
02081
02082 void set(const T& value) {
02083 value_ = value;
02084 }
02085
02086 private:
02087 T value_;
02088 };
02089
02090 #endif // GTEST_IS_THREADS
02091
02092 // Returns the number of threads running in the process, or 0 to indicate that
02093 // we cannot detect it.
02094 GTEST_API_ size_t GetThreadCount();
02095
02096 #if GTEST_OS_WINDOWS
02097 #define GTEST_PATH_SEP_ "\\"
02098 #define GTEST_HAS_ALT_PATH_SEP_ 1
02099 #else
02100 #define GTEST_PATH_SEP_ "/"
02101 #define GTEST_HAS_ALT_PATH_SEP_ 0
02102 #endif // GTEST_OS_WINDOWS
02103
02104 // Utilities for char.
02105
02106 // isspace(int ch) and friends accept an unsigned char or EOF. char
02107 // may be signed, depending on the compiler (or compiler flags).
02108 // Therefore we need to cast a char to unsigned char before calling
02109 // isspace(), etc.
02110
02111 inline bool IsAlpha(char ch) {
02112 return isalpha(static_cast<unsigned char>(ch)) != 0;
02113 }
02114
02115 inline bool IsAlNum(char ch) {
02116 return isalnum(static_cast<unsigned char>(ch)) != 0;
02117 }
02118
02119 inline bool IsDigit(char ch) {
02120 return isdigit(static_cast<unsigned char>(ch)) != 0;
02121 }
02122
02123 inline bool IsLower(char ch) {
02124 return islower(static_cast<unsigned char>(ch)) != 0;

```



```

02125 }
02126
02127 inline bool IsSpace(char ch) {
02128 return isspace(static_cast<unsigned char> (ch)) != 0;
02129 }
02130
02131 inline bool IsUpper(char ch) {
02132 return isupper(static_cast<unsigned char> (ch)) != 0;
02133 }
02134
02135 inline bool IsXDigit(char ch) {
02136 return isxdigit(static_cast<unsigned char> (ch)) != 0;
02137 }
02138 #ifdef __cpp_char8_t
02139
02140 inline bool IsXDigit(char8_t ch) {
02141 return isxdigit(static_cast<unsigned char> (ch)) != 0;
02142 }
02143 #endif
02144
02145 inline bool IsXDigit(char16_t ch) {
02146 const unsigned char low_byte = static_cast<unsigned char> (ch);
02147 return ch == low_byte && isxdigit(low_byte) != 0;
02148 }
02149
02150 inline bool IsXDigit(char32_t ch) {
02151 const unsigned char low_byte = static_cast<unsigned char> (ch);
02152 return ch == low_byte && isxdigit(low_byte) != 0;
02153 }
02154
02155 inline bool IsXDigit(wchar_t ch) {
02156 const unsigned char low_byte = static_cast<unsigned char> (ch);
02157 return ch == low_byte && isxdigit(low_byte) != 0;
02158 }
02159
02160 inline char ToLower(char ch) {
02161 return static_cast<char> (tolower(static_cast<unsigned char> (ch)));
02162 }
02163
02164 inline char ToUpper(char ch) {
02165 return static_cast<char> (toupper(static_cast<unsigned char> (ch)));
02166 }
02167
02168 inline std::string StripTrailingSpaces(std::string str) {
02169 std::string::iterator it = str.end();
02170 while (it != str.begin() && IsSpace(*--it)) it = str.erase(it);
02171 return str;
02172 }
02173
02174 // The testing::internal::posix namespace holds wrappers for common
02175 // POSIX functions. These wrappers hide the differences between
02176 // Windows/MSVC and POSIX systems. Since some compilers define these
02177 // standard functions as macros, the wrapper cannot have the same name
02178 // as the wrapped function.
02179
02180 namespace posix {
02181
02182 // Functions with a different name on Windows.
02183
02184 #if GTEST_OS_WINDOWS
02185
02186 typedef struct _stat StatStruct;
02187
02188 #ifdef __BORLANDC__
02189
02190 inline int DoIsATTY(int fd) {
02191 return isatty(fd);
02192 }
02193
02194 inline int StrCaseCmp(const char* s1, const char* s2) {
02195 return stricmp(s1, s2);
02196 }
02197
02198 inline char* StrDup(const char* src) {
02199 return strdup(src);
02200 }
02201 #else // !__BORLANDC__
02202 #if GTEST_OS_WINDOWS_MOBILE || GTEST_OS_ZOS || GTEST_OS_IOS || \
02203 GTEST_OS_WINDOWS_PHONE || GTEST_OS_WINDOWS_RT || defined(ESP_PLATFORM)
02204
02205 inline int DoIsATTY(int /* fd */) {
02206 return 0;
02207 }
02208 #else
02209
02210 inline int DoIsATTY(int fd) {
02211 return _isatty(fd);

```

```

02212 }
02213 #endif // GTEST_OS_WINDOWS_MOBILE
02214
02215 inline int StrCaseCmp(const char* s1, const char* s2) {
02216 return _stricmp(s1, s2);
02217 }
02218
02219 inline char* StrDup(const char* src) {
02220 return _strdup(src);
02221 }
02222 #endif // __BORLANDC__
02223
02224 #if GTEST_OS_WINDOWS_MOBILE
02225
02226 inline int FileNo(FILE* file) {
02227 return reinterpret_cast<int> (_fileno(file));
02228 }
02229 // Stat(), RmDir(), and IsDir() are not needed on Windows CE at this
02230 // time and thus not defined there.
02231 #else
02232
02233 inline int FileNo(FILE* file) {
02234 return _fileno(file);
02235 }
02236
02237 inline int Stat(const char* path, StatStruct* buf) {
02238 return _stat(path, buf);
02239 }
02240
02241 inline int RmDir(const char* dir) {
02242 return _rmdir(dir);
02243 }
02244
02245 inline bool IsDir(const StatStruct& st) {
02246 return (_S_IFDIR & st.st_mode) != 0;
02247 }
02248 #endif // GTEST_OS_WINDOWS_MOBILE
02249
02250 #elif GTEST_OS_ESP8266
02251 typedef struct stat StatStruct;
02252
02253 inline int FileNo(FILE* file) {
02254 return fileno(file);
02255 }
02256
02257 inline int DoIsATTY(int fd) {
02258 return isatty(fd);
02259 }
02260
02261 inline int Stat(const char* path, StatStruct* buf) {
02262 // stat function not implemented on ESP8266
02263 return 0;
02264 }
02265
02266 inline int StrCaseCmp(const char* s1, const char* s2) {
02267 return strcasecmp(s1, s2);
02268 }
02269
02270 inline char* StrDup(const char* src) {
02271 return strdup(src);
02272 }
02273
02274 inline int RmDir(const char* dir) {
02275 return rmdir(dir);
02276 }
02277
02278 inline bool IsDir(const StatStruct& st) {
02279 return S_ISDIR(st.st_mode);
02280 }
02281 #else
02282
02283 typedef struct stat StatStruct;
02284
02285 inline int FileNo(FILE* file) {
02286 return fileno(file);
02287 }
02288
02289 inline int DoIsATTY(int fd) {
02290 return isatty(fd);
02291 }
02292
02293 inline int Stat(const char* path, StatStruct* buf) {
02294 return stat(path, buf);
02295 }
02296
02297 inline int StrCaseCmp(const char* s1, const char* s2) {

```

```

02299 return strcasecmp(s1, s2);
02300 }
02301
02302 inline char* StrDup(const char* src) {
02303 return strdup(src);
02304 }
02305
02306 inline int Rmdir(const char* dir) {
02307 return rmdir(dir);
02308 }
02309
02310 inline bool IsDir(const StatStruct& st) {
02311 return S_ISDIR(st.st_mode);
02312 }
02313
02314 #endif // GTEST_OS_WINDOWS
02315
02316 inline int IsATTY(int fd) {
02317 // DoIsATTY might change errno (for example ENOTTY in case you redirect stdout
02318 // to a file on Linux), which is unexpected, so save the previous value, and
02319 // restore it after the call.
02320 int savedErrno = errno;
02321 int isAttyValue = DoIsATTY(fd);
02322 errno = savedErrno;
02323
02324 return isAttyValue;
02325 }
02326
02327 // Functions deprecated by MSVC 8.0.
02328
02329 GTEST_DISABLE_MSC_DEPRECATED_PUSH_()
02330
02331 // ChDir(), FReopen(), FOpen(), Read(), Write(), Close(), and
02332 // StrError() aren't needed on Windows CE at this time and thus not
02333 // defined there.
02334
02335 #if !GTEST_OS_WINDOWS_MOBILE && !GTEST_OS_WINDOWS_PHONE && \
02336 !GTEST_OS_WINDOWS_RT && !GTEST_OS_ESP8266 && !GTEST_OS_XTENSA
02337 inline int ChDir(const char* dir) {
02338 return chdir(dir);
02339 }
02340 #endif
02341
02342 inline FILE* FOpen(const char* path, const char* mode) {
02343 #if GTEST_OS_WINDOWS && !GTEST_OS_WINDOWS_MINGW
02344 struct wchar_codecvt : public std::codecvt<wchar_t, char, std::mbstate_t> {
02345 };
02346 std::wstring_convert<wchar_codecvt> converter;
02347 std::wstring wide_path = converter.from_bytes(path);
02348 std::wstring wide_mode = converter.from_bytes(mode);
02349 return _wopen(wide_path.c_str(), wide_mode.c_str());
02350 #else
02351 // GTEST_OS_WINDOWS && !GTEST_OS_WINDOWS_MINGW
02352 return fopen(path, mode);
02353 #endif // GTEST_OS_WINDOWS && !GTEST_OS_WINDOWS_MINGW
02354 }
02355 #if !GTEST_OS_WINDOWS_MOBILE
02356
02357 inline FILE* FReopen(const char* path, const char* mode, FILE* stream) {
02358 return freopen(path, mode, stream);
02359 }
02360
02361 inline FILE* FOpen(int fd, const char* mode) {
02362 return fdopen(fd, mode);
02363 }
02364 #endif
02365
02366 inline int FClose(FILE* fp) {
02367 return fclose(fp);
02368 }
02369 #if !GTEST_OS_WINDOWS_MOBILE
02370
02371 inline int Read(int fd, void* buf, unsigned int count) {
02372 return static_cast<int>(read(fd, buf, count));
02373 }
02374
02375 inline int Write(int fd, const void* buf, unsigned int count) {
02376 return static_cast<int>(write(fd, buf, count));
02377 }
02378
02379 inline int Close(int fd) {
02380 return close(fd);
02381 }
02382
02383 inline const char* StrError(int errnum) {
02384 return strerror(errnum);
02385 }

```

```

02386 #endif
02387
02388 inline const char* GetEnv(const char* name) {
02389 #if GTEST_OS_WINDOWS_MOBILE || GTEST_OS_WINDOWS_PHONE || \
02390 GTEST_OS_WINDOWS_RT || GTEST_OS_ESP8266 || GTEST_OS_XTENSA
02391 // We are on an embedded platform, which has no environment variables.
02392 static_cast<void>(name); // To prevent 'unused argument' warning.
02393 return nullptr;
02394 #elif defined(__BORLANDC__) || defined(__SunOS_5_8) || defined(__SunOS_5_9)
02395 // Environment variables which we programmatically clear will be set to the
02396 // empty string rather than unset (NULL). Handle that case.
02397 const char* const env = getenv(name);
02398 return (env != nullptr && env[0] != '\0') ? env : nullptr;
02399 #else
02400 return getenv(name);
02401 #endif
02402 }
02403
02404 GTEST_DISABLE_MSC_DEPRECATED_POP_()
02405
02406 #if GTEST_OS_WINDOWS_MOBILE
02407 // Windows CE has no C library. The abort() function is used in
02408 // several places in Google Test. This implementation provides a reasonable
02409 // imitation of standard behaviour.
02410 [[noreturn]] void Abort();
02411 #else
02412 [[noreturn]] inline void Abort() {
02413 abort();
02414 }
02415 #endif // GTEST_OS_WINDOWS_MOBILE
02416
02417 } // namespace posix
02418
02419 // MSVC "deprecates" snprintf and issues warnings wherever it is used. In
02420 // order to avoid these warnings, we need to use _snprintf or _snprintf_s on
02421 // MSVC-based platforms. We map the GTEST_SNPRINTF_ macro to the appropriate
02422 // function in order to achieve that. We use macro definition here because
02423 // snprintf is a variadic function.
02424 #if _MSC_VER && !GTEST_OS_WINDOWS_MOBILE
02425 // MSVC 2005 and above support variadic macros.
02426 #define GTEST_SNPRINTF_(buffer, size, format, ...) \
02427 _snprintf_s(buffer, size, size, format, __VA_ARGS__)
02428 #elif defined(_MSC_VER)
02429 // Windows CE does not define _snprintf_s
02430 #define GTEST_SNPRINTF_ _snprintf
02431 #else
02432 #define GTEST_SNPRINTF_ snprintf
02433 #endif
02434
02435 // The biggest signed integer type the compiler supports.
02436 //
02437 // long long is guaranteed to be at least 64-bits in C++11.
02438 using BiggestInt = long long; // NOLINT
02439
02440 // The maximum number a BiggestInt can represent.
02441 constexpr BiggestInt kMaxBiggestInt = (std::numeric_limits<BiggestInt>::max)();
02442
02443 // This template class serves as a compile-time function from size to
02444 // type. It maps a size in bytes to a primitive type with that
02445 // size. e.g.
02446 //
02447 // TypeWithSize<4>::UInt
02448 //
02449 // is typedef-ed to be unsigned int (unsigned integer made up of 4
02450 // bytes).
02451 //
02452 // Such functionality should belong to STL, but I cannot find it
02453 // there.
02454 //
02455 // Google Test uses this class in the implementation of floating-point
02456 // comparison.
02457 //
02458 // For now it only handles UInt (unsigned int) as that's all Google Test
02459 // needs. Other types can be easily added in the future if need
02460 // arises.
02461
02462 template <size_t size>
02463 class TypeWithSize {
02464 public:
02465 // This prevents the user from using TypeWithSize<N> with incorrect
02466 // values of N.
02467 using UInt = void;
02468 };
02469
02470 // The specialization for size 4.
02471
02472 template <>

```

```

02473 class TypeWithSize<4> {
02474 public:
02475 using Int = std::int32_t;
02476 using UInt = std::uint32_t;
02477 };
02478
02479 // The specialization for size 8.
02480
02481 template <>
02482 class TypeWithSize<8> {
02483 public:
02484 using Int = std::int64_t;
02485 using UInt = std::uint64_t;
02486 };
02487
02488 // Integer types of known sizes.
02489 using TimeInMillis = int64_t; // Represents time in milliseconds.
02490
02491 // Utilities for command line flags and environment variables.
02492
02493 // Macro for referencing flags.
02494 #if !defined(GTEST_FLAG)
02495 #define GTEST_FLAG_NAME_(name) gtest_##name
02496 #define GTEST_FLAG(name) FLAGS_gtest_##name
02497 #endif // !defined(GTEST_FLAG)
02498
02499 // Pick a command line flags implementation.
02500 #if GTEST_HAS_ABSL
02501
02502 // Macros for defining flags.
02503 #define GTEST_DEFINE_bool_(name, default_val, doc) \
02504 ABSL_FLAG(bool, GTEST_FLAG_NAME_(name), default_val, doc)
02505 #define GTEST_DEFINE_int32_(name, default_val, doc) \
02506 ABSL_FLAG(int32_t, GTEST_FLAG_NAME_(name), default_val, doc)
02507 #define GTEST_DEFINE_string_(name, default_val, doc) \
02508 ABSL_FLAG(std::string, GTEST_FLAG_NAME_(name), default_val, doc)
02509
02510 // Macros for declaring flags.
02511 #define GTEST_DECLARE_bool_(name) \
02512 ABSL_DECLARE_FLAG(bool, GTEST_FLAG_NAME_(name))
02513 #define GTEST_DECLARE_int32_(name) \
02514 ABSL_DECLARE_FLAG(int32_t, GTEST_FLAG_NAME_(name))
02515 #define GTEST_DECLARE_string_(name) \
02516 ABSL_DECLARE_FLAG(std::string, GTEST_FLAG_NAME_(name))
02517
02518 #define GTEST_FLAG_SAVER_ ::absl::FlagSaver
02519
02520 #define GTEST_FLAG_GET(name) ::absl::GetFlag(GTEST_FLAG(name))
02521 #define GTEST_FLAG_SET(name, value) \
02522 (void)(::absl::SetFlag(>EST_FLAG(name), value))
02523 #define GTEST_USE_OWN_FLAGFILE_FLAG_ 0
02524
02525 #else // GTEST_HAS_ABSL
02526
02527 // Macros for defining flags.
02528 #define GTEST_DEFINE_bool_(name, default_val, doc) \
02529 namespace testing { \
02530 GTEST_API_ bool GTEST_FLAG(name) = (default_val); \
02531 } \
02532 static_assert(true, "no-op to require trailing semicolon")
02533 #define GTEST_DEFINE_int32_(name, default_val, doc) \
02534 namespace testing { \
02535 GTEST_API_ std::int32_t GTEST_FLAG(name) = (default_val); \
02536 } \
02537 static_assert(true, "no-op to require trailing semicolon")
02538 #define GTEST_DEFINE_string_(name, default_val, doc) \
02539 namespace testing { \
02540 GTEST_API_ std::string GTEST_FLAG(name) = (default_val); \
02541 } \
02542 static_assert(true, "no-op to require trailing semicolon")
02543
02544 // Macros for declaring flags.
02545 #define GTEST_DECLARE_bool_(name) \
02546 namespace testing { \
02547 GTEST_API_ extern bool GTEST_FLAG(name); \
02548 } \
02549 static_assert(true, "no-op to require trailing semicolon")
02550 #define GTEST_DECLARE_int32_(name) \
02551 namespace testing { \
02552 GTEST_API_ extern std::int32_t GTEST_FLAG(name); \
02553 } \
02554 static_assert(true, "no-op to require trailing semicolon")
02555 #define GTEST_DECLARE_string_(name) \
02556 namespace testing { \
02557 GTEST_API_ extern std::string GTEST_FLAG(name); \
02558 } \
02559 static_assert(true, "no-op to require trailing semicolon")

```

```

02560
02561 #define GTEST_FLAG_SAVER_ ::testing::internal::GTestFlagSaver
02562
02563 #define GTEST_FLAG_GET(name) ::testing::GTEST_FLAG(name)
02564 #define GTEST_FLAG_SET(name, value) (void)(::testing::GTEST_FLAG(name) = value)
02565 #define GTEST_USE_OWN_FLAGFILE_FLAG_ 1
02566
02567 #endif // GTEST_HAS_ABSL
02568
02569 // Thread annotations
02570 #if !defined(GTEST_EXCLUSIVE_LOCK_REQUIRED_)
02571 #define GTEST_EXCLUSIVE_LOCK_REQUIRED_(locks)
02572 #define GTEST_LOCK_EXCLUDED_(locks)
02573 #endif // !defined(GTEST_EXCLUSIVE_LOCK_REQUIRED_)
02574
02575 // Parses 'str' for a 32-bit signed integer. If successful, writes the result
02576 // to *value and returns true; otherwise leaves *value unchanged and returns
02577 // false.
02578 GTEST_API_ bool ParseInt32(const Message& src_text, const char* str,
02579 int32_t* value);
02580
02581 // Parses a bool/int32_t/string from the environment variable
02582 // corresponding to the given Google Test flag.
02583 bool BoolFromGTestEnv(const char* flag, bool default_val);
02584 GTEST_API_ int32_t Int32FromGTestEnv(const char* flag, int32_t default_val);
02585 std::string OutputFlagAlsoCheckEnvVar();
02586 const char* StringFromGTestEnv(const char* flag, const char* default_val);
02587
02588 } // namespace internal
02589 } // namespace testing
02590
02591 #if !defined(GTEST_INTERNAL_DEPRECATED)
02592
02593 // Internal Macro to mark an API deprecated, for googletest usage only
02594 // Usage: class GTEST_INTERNAL_DEPRECATED(message) MyClass or
02595 // GTEST_INTERNAL_DEPRECATED(message) <return_type> myFunction(); Every usage of
02596 // a deprecated entity will trigger a warning when compiled with
02597 // '-Wdeprecated-declarations' option (clang, gcc, any __GNUC__ compiler).
02598 // For msvc /W3 option will need to be used
02599 // Note that for 'other' compilers this macro evaluates to nothing to prevent
02600 // compilations errors.
02601 #if defined(_MSC_VER)
02602 #define GTEST_INTERNAL_DEPRECATED(message) __declspec(deprecated(message))
02603 #elif defined(__GNUC__)
02604 #define GTEST_INTERNAL_DEPRECATED(message) __attribute__((deprecated(message)))
02605 #else
02606 #define GTEST_INTERNAL_DEPRECATED(message)
02607 #endif
02608
02609 #endif // !defined(GTEST_INTERNAL_DEPRECATED)
02610
02611 #if GTEST_HAS_ABSL
02612 // Always use absl::any for UniversalPrinter<> specializations if googletest
02613 // is built with absl support.
02614 #define GTEST_INTERNAL_HAS_ANY 1
02615 #include "absl/types/any.h"
02616 namespace testing {
02617 namespace internal {
02618 using Any = ::absl::any;
02619 } // namespace internal
02620 } // namespace testing
02621 #else
02622 #ifndef __has_include
02623 #if __has_include(<any>) && __cplusplus >= 201703L
02624 // Otherwise for C++17 and higher use std::any for UniversalPrinter<>
02625 // specializations.
02626 #define GTEST_INTERNAL_HAS_ANY 1
02627 #include <any>
02628 namespace testing {
02629 namespace internal {
02630 using Any = ::std::any;
02631 } // namespace internal
02632 } // namespace testing
02633 // The case where absl is configured NOT to alias std::any is not
02634 // supported.
02635 #endif // __has_include(<any>) && __cplusplus >= 201703L
02636 #endif // __has_include
02637 #endif // GTEST_HAS_ABSL
02638
02639 #if GTEST_HAS_ABSL
02640 // Always use absl::optional for UniversalPrinter<> specializations if
02641 // googletest is built with absl support.
02642 #define GTEST_INTERNAL_HAS_OPTIONAL 1
02643 #include "absl/types/optional.h"
02644 namespace testing {
02645 namespace internal {
02646 template <typename T>

```

```

02647 using Optional = ::absl::optional<T>;
02648
02649 inline ::absl::nullopt_t Nullopt() {
02650 return ::absl::nullopt;
02651 }
02652 } // namespace internal
02653 } // namespace testing
02654 #else
02655 #ifdef __has_include
02656 #if __has_include(<optional>) && __cplusplus >= 201703L
02657 // Otherwise for C++17 and higher use std::optional for UniversalPrinter<>
02658 // specializations.
02659 #define GTEST_INTERNAL_HAS_OPTIONAL 1
02660 #include <optional>
02661 namespace testing {
02662 namespace internal {
02663 template <typename T>
02664 using Optional = ::std::optional<T>;
02665
02666 inline ::std::nullopt_t Nullopt() {
02667 return ::std::nullopt;
02668 }
02669 } // namespace internal
02670 } // namespace testing
02671 // The case where absl is configured NOT to alias std::optional is not
02672 // supported.
02673 #endif // __has_include(<optional>) && __cplusplus >= 201703L
02674 #endif // __has_include
02675 #endif // GTEST_HAS_ABSL
02676
02677 #if GTEST_HAS_ABSL
02678 // Always use absl::string_view for Matcher<> specializations if googletest
02679 // is built with absl support.
02680 #define GTEST_INTERNAL_HAS_STRING_VIEW 1
02681 #include "absl/strings/string_view.h"
02682 namespace testing {
02683 namespace internal {
02684 using StringView = ::absl::string_view;
02685 } // namespace internal
02686 } // namespace testing
02687 #else
02688 #ifdef __has_include
02689 #if __has_include(<string_view>) && __cplusplus >= 201703L
02690 // Otherwise for C++17 and higher use std::string_view for Matcher<>
02691 // specializations.
02692 #define GTEST_INTERNAL_HAS_STRING_VIEW 1
02693 #include <string_view>
02694 namespace testing {
02695 namespace internal {
02696 using StringView = ::std::string_view;
02697 } // namespace internal
02698 } // namespace testing
02699 // The case where absl is configured NOT to alias std::string_view is not
02700 // supported.
02701 #endif // __has_include(<string_view>) && __cplusplus >= 201703L
02702 #endif // __has_include
02703 #endif // GTEST_HAS_ABSL
02704
02705 #if GTEST_HAS_ABSL
02706 // Always use absl::variant for UniversalPrinter<> specializations if googletest
02707 // is built with absl support.
02708 #define GTEST_INTERNAL_HAS_VARIANT 1
02709 #include "absl/types/variant.h"
02710 namespace testing {
02711 namespace internal {
02712 template <typename... T>
02713 using Variant = ::absl::variant<T...>;
02714 } // namespace internal
02715 } // namespace testing
02716 #else
02717 #ifdef __has_include
02718 #if __has_include(<variant>) && __cplusplus >= 201703L
02719 // Otherwise for C++17 and higher use std::variant for UniversalPrinter<>
02720 // specializations.
02721 #define GTEST_INTERNAL_HAS_VARIANT 1
02722 #include <variant>
02723 namespace testing {
02724 namespace internal {
02725 template <typename... T>
02726 using Variant = ::std::variant<T...>;
02727 } // namespace internal
02728 } // namespace testing
02729 // The case where absl is configured NOT to alias std::variant is not supported.
02730 #endif // __has_include(<variant>) && __cplusplus >= 201703L
02731 #endif // __has_include
02732 #endif // GTEST_HAS_ABSL
02733

```

```
02734 #endif // GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_PORT_H_
```

## 16.247 gtest-death-test-internal.h

```
00001 // Copyright 2005, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // The Google C++ Testing and Mocking Framework (Google Test)
00031 //
00032 // This header file defines internal utilities needed for implementing
00033 // death tests. They are subject to change without notice.
00034
00035 // IWYU pragma: private, include "gtest/gtest.h"
00036 // IWYU pragma: friend gtest/*.h
00037 // IWYU pragma: friend gmock/*.h
00038
00039 #ifndef GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_DEATH_TEST_INTERNAL_H_
00040 #define GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_DEATH_TEST_INTERNAL_H_
00041
00042 #include <stdio.h>
00043
00044 #include <memory>
00045
00046 #include "gtest/gtest-matchers.h"
00047 #include "gtest/internal/gtest-internal.h"
00048
00049 GTEST_DECLARE_string_(internal_run_death_test);
00050
00051 namespace testing {
00052 namespace internal {
00053
00054 // Names of the flags (needed for parsing Google Test flags).
00055 const char kDeathTestStyleFlag[] = "death_test_style";
00056 const char kDeathTestUseFork[] = "death_test_use_fork";
00057 const char kInternalRunDeathTestFlag[] = "internal_run_death_test";
00058
00059 #if GTEST_HAS_DEATH_TEST
00060
00061 GTEST_DISABLE_MSC_WARNINGS_PUSH_(4251 \
00062 /* class A needs to have dll-interface to be used by clients of class B */)
00063
00064 // DeathTest is a class that hides much of the complexity of the
00065 // GTEST_DEATH_TEST_ macro. It is abstract; its static Create method
00066 // returns a concrete class that depends on the prevailing death test
00067 // style, as defined by the --gtest_death_test_style and/or
00068 // --gtest_internal_run_death_test flags.
00069
00070 // In describing the results of death tests, these terms are used with
00071 // the corresponding definitions:
00072 //
00073 // exit status: The integer exit information in the format specified
00074 // by wait(2)
00075 // exit code: The integer code passed to exit(3), _exit(2), or
00076 // returned from main()
00077 class GTEST_API_ DeathTest {
00078 public:
00079 // Create returns false if there was an error determining the
00080 // appropriate action to take for the current death test; for example,
00081 // if the gtest_death_test_style flag is set to an invalid value.
```



```

00082 // The LastMessage method will return a more detailed message in that
00083 // case. Otherwise, the DeathTest pointer pointed to by the "test"
00084 // argument is set. If the death test should be skipped, the pointer
00085 // is set to NULL; otherwise, it is set to the address of a new concrete
00086 // DeathTest object that controls the execution of the current test.
00087 static bool Create(const char* statement, Matcher<const std::string&> matcher,
00088 const char* file, int line, DeathTest** test);
00089 DeathTest();
00090
00091 virtual ~DeathTest() {
00092 }
00093
00094 // A helper class that aborts a death test when it's deleted.
00095
00096 class ReturnSentinel {
00097 public:
00098
00099 explicit ReturnSentinel(DeathTest* test) : test_(test) {
00100 }
00101
00102 ~ReturnSentinel() {
00103 test_->Abort(TEST_ENCOUNTERED_RETURN_STATEMENT);
00104 }
00105
00106 private:
00107 DeathTest * const test_;
00108 ReturnSentinel(const ReturnSentinel&) = delete;
00109 ReturnSentinel& operator=(const ReturnSentinel&) = delete;
00110 } GTEST_ATTRIBUTE_UNUSED_;
00111
00112 // An enumeration of possible roles that may be taken when a death
00113 // test is encountered. EXECUTE means that the death test logic should
00114 // be executed immediately. OVERSEE means that the program should prepare
00115 // the appropriate environment for a child process to execute the death
00116 // test, then wait for it to complete.
00117
00118 enum TestRole {
00119 OVERSEE_TEST, EXECUTE_TEST
00120 };
00121
00122 // An enumeration of the three reasons that a test might be aborted.
00123
00124 enum AbortReason {
00125 TEST_ENCOUNTERED_RETURN_STATEMENT,
00126 TEST_THREW_EXCEPTION,
00127 TEST_DID_NOT_DIE
00128 };
00129
00130 // Assumes one of the above roles.
00131 virtual TestRole AssumeRole() = 0;
00132
00133 // Waits for the death test to finish and returns its status.
00134 virtual int Wait() = 0;
00135
00136 // Returns true if the death test passed; that is, the test process
00137 // exited during the test, its exit status matches a user-supplied
00138 // predicate, and its stderr output matches a user-supplied regular
00139 // expression.
00140 // The user-supplied predicate may be a macro expression rather
00141 // than a function pointer or functor, or else Wait and Passed could
00142 // be combined.
00143 virtual bool Passed(bool exit_status_ok) = 0;
00144
00145 // Signals that the death test did not die as expected.
00146 virtual void Abort(AbortReason reason) = 0;
00147
00148 // Returns a human-readable outcome message regarding the outcome of
00149 // the last death test.
00150 static const char* LastMessage();
00151
00152 static void set_last_death_test_message(const std::string& message);
00153
00154 private:
00155 // A string containing a description of the outcome of the last death test.
00156 static std::string last_death_test_message_;
00157
00158 DeathTest(const DeathTest&) = delete;
00159 DeathTest& operator=(const DeathTest&) = delete;
00160 };
00161
00162 GTEST_DISABLE_MSC_WARNINGS_POP_() // 4251
00163
00164 // Factory interface for death tests. May be mocked out for testing.
00165 class DeathTestFactory {
00166 public:
00167
00168 virtual ~DeathTestFactory() {

```

```

00169 }
00170 virtual bool Create(const char* statement,
00171 Matcher<const std::string&> matcher, const char* file,
00172 int line, DeathTest** test) = 0;
00173 };
00174
00175 // A concrete DeathTestFactory implementation for normal use.
00176
00177 class DefaultDeathTestFactory : public DeathTestFactory {
00178 public:
00179 bool Create(const char* statement, Matcher<const std::string&> matcher,
00180 const char* file, int line, DeathTest** test) override;
00181 };
00182
00183 // Returns true if exit_status describes a process that was terminated
00184 // by a signal, or exited normally with a nonzero exit code.
00185 GTEST_API_ bool ExitedUnsuccessfully(int exit_status);
00186
00187 // A string passed to EXPECT_DEATH (etc.) is caught by one of these overloads
00188 // and interpreted as a regex (rather than an Eq matcher) for legacy
00189 // compatibility.
00190
00191 inline Matcher<const std::string&> MakeDeathTestMatcher(
00192 ::testing::internal::RE regex) {
00193 return ContainsRegex(regex.pattern());
00194 }
00195
00196 inline Matcher<const std::string&> MakeDeathTestMatcher(const char* regex) {
00197 return ContainsRegex(regex);
00198 }
00199
00200 inline Matcher<const std::string&> MakeDeathTestMatcher(
00201 const std::string& regex) {
00202 return ContainsRegex(regex);
00203 }
00204
00205 // If a Matcher<const std::string&> is passed to EXPECT_DEATH (etc.), it's
00206 // used directly.
00207
00208 inline Matcher<const std::string&> MakeDeathTestMatcher(
00209 Matcher<const std::string&> matcher) {
00210 return matcher;
00211 }
00212
00213 // Traps C++ exceptions escaping statement and reports them as test
00214 // failures. Note that trapping SEH exceptions is not implemented here.
00215 #if GTEST_HAS_EXCEPTIONS
00216 #define GTEST_EXECUTE_DEATH_TEST_STATEMENT_(statement, death_test)
00217 try {
00218 GTEST_SUPPRESS_UNREACHABLE_CODE_WARNING_BELOW_(statement);
00219 } catch (const std::exception& gtest_exception) {
00220 fprintf(
00221 stderr,
00222 "\n%s: Caught std::exception-derived exception escaping the "
00223 "death test statement. Exception message: %s\n",
00224 ::testing::internal::FormatFileLocation(__FILE__, __LINE__).c_str(),
00225 gtest_exception.what());
00226 fflush(stderr);
00227 death_test->Abort(::testing::internal::DeathTest::TEST_THREW_EXCEPTION);
00228 } catch (...) {
00229 death_test->Abort(::testing::internal::DeathTest::TEST_THREW_EXCEPTION);
00230 }
00231 #else
00232 #define GTEST_EXECUTE_DEATH_TEST_STATEMENT_(statement, death_test) \
00233 GTEST_SUPPRESS_UNREACHABLE_CODE_WARNING_BELOW_(statement)
00234 #endif
00235
00236 #endif
00237
00238 // This macro is for implementing ASSERT_DEATH*, EXPECT_DEATH*,
00239 // ASSERT_EXIT*, and EXPECT_EXIT*.
00240 #define GTEST_DEATH_TEST_(statement, predicate, regex_or_matcher, fail)
00241 GTEST_AMBIGUOUS_ELSE_BLOCKER_
00242 if (::testing::internal::AlwaysTrue()) {
00243 ::testing::internal::DeathTest* gtest_dt;
00244 if (!::testing::internal::DeathTest::Create(
00245 #statement,
00246 ::testing::internal::MakeDeathTestMatcher(regex_or_matcher),
00247 __FILE__, __LINE__, >est_dt)) {
00248 goto GTEST_CONCAT_TOKEN_(gtest_label_, __LINE__);
00249 }
00250 if (gtest_dt != nullptr) {
00251 std::unique_ptr< ::testing::internal::DeathTest> gtest_dt_ptr(gtest_dt);
00252 switch (gtest_dt->AssumeRole()) {
00253 case ::testing::internal::DeathTest::OVERSEE_TEST:
00254 if (!gtest_dt->Passed(predicate(gtest_dt->Wait()))) {
00255 goto GTEST_CONCAT_TOKEN_(gtest_label_, __LINE__);
00256 }
00257 }
00258 }
00259 }
00260 else {
00261 // This block is not executed.
00262 }
00263 }

```

```

00256 }
00257 break;
00258 case ::testing::internal::DeathTest::EXECUTE_TEST: {
00259 ::testing::internal::DeathTest::ReturnSentinel gtest_sentinel(
00260 gtest_dt);
00261 GTEST_EXECUTE_DEATH_TEST_STATEMENT_(statement, gtest_dt);
00262 gtest_dt->Abort(::testing::internal::DeathTest::TEST_DID_NOT_DIE);
00263 break;
00264 }
00265 }
00266 }
00267 } else
00268 GTEST_CONCAT_TOKEN_(gtest_label_, __LINE__)
00269 : fail(::testing::internal::DeathTest::LastMessage())
00270 // The symbol "fail" here expands to something into which a message
00271 // can be streamed.
00272
00273 // This macro is for implementing ASSERT/EXPECT_DEBUG_DEATH when compiled in
00274 // NDEBUG mode. In this case we need the statements to be executed and the macro
00275 // must accept a streamed message even though the message is never printed.
00276 // The regex object is not evaluated, but it is used to prevent "unused"
00277 // warnings and to avoid an expression that doesn't compile in debug mode.
00278 #define GTEST_EXECUTE_STATEMENT_(statement, regex_or_matcher) \
00279 GTEST_AMBIGUOUS_ELSE_BLOCKER_
00280 if (::testing::internal::AlwaysTrue()) {
00281 GTEST_SUPPRESS_UNREACHABLE_CODE_WARNING_BELOW_(statement);
00282 } else if (!::testing::internal::AlwaysTrue()) {
00283 ::testing::internal::MakeDeathTestMatcher(regex_or_matcher);
00284 } else
00285 ::testing::Message()
00286
00287 // A class representing the parsed contents of the
00288 // --gtest_internal_run_death_test flag, as it existed when
00289 // RUN_ALL_TESTS was called.
00290
00291 class InternalRunDeathTestFlag {
00292 public:
00293
00294 InternalRunDeathTestFlag(const std::string& a_file, int a_line, int an_index,
00295 int a_write_fd)
00296 : file_(a_file), line_(a_line), index_(an_index), write_fd_(a_write_fd) {}
00297
00298 ~InternalRunDeathTestFlag() {
00299 if (write_fd_ >= 0) posix::Close(write_fd_);
00300 }
00301
00302 const std::string& file() const {
00303 return file_;
00304 }
00305
00306 int line() const {
00307 return line_;
00308 }
00309
00310 int index() const {
00311 return index_;
00312 }
00313
00314 int write_fd() const {
00315 return write_fd_;
00316 }
00317
00318 private:
00319 std::string file_;
00320 int line_;
00321 int index_;
00322 int write_fd_;
00323
00324 InternalRunDeathTestFlag(const InternalRunDeathTestFlag&) = delete;
00325 InternalRunDeathTestFlag& operator=(const InternalRunDeathTestFlag&) = delete;
00326 };
00327
00328 // Returns a newly created InternalRunDeathTestFlag object with fields
00329 // initialized from the GTEST_FLAG(internal_run_death_test) flag if
00330 // the flag is specified; otherwise returns NULL.
00331 InternalRunDeathTestFlag* ParseInternalRunDeathTestFlag();
00332
00333 #endif // GTEST_HAS_DEATH_TEST
00334
00335 } // namespace internal
00336 } // namespace testing
00337
00338 #endif // GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_DEATH_TEST_INTERNAL_H_

```

## 16.248 gtest-filepath.h

```

00001 // Copyright 2008, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // Google Test filepath utilities
00031 //
00032 // This header file declares classes and functions used internally by
00033 // Google Test. They are subject to change without notice.
00034 //
00035 // This file is #included in gtest/internal/gtest-internal.h.
00036 // Do not include this header file separately!
00037
00038 // IWYU pragma: private, include "gtest/gtest.h"
00039 // IWYU pragma: friend gtest/*.h
00040 // IWYU pragma: friend gmock/*.h
00041
00042 #ifndef GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_FILEPATH_H_
00043 #define GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_FILEPATH_H_
00044
00045 #include "gtest/internal/gtest-string.h"
00046
00047 GTEST_DISABLE_MSC_WARNINGS_PUSH_(4251 \
00048 /* class A needs to have dll-interface to be used by clients of class B */)
00049
00050 namespace testing {
00051 namespace internal {
00052
00053 // FilePath - a class for file and directory pathname manipulation which
00054 // handles platform-specific conventions (like the pathname separator).
00055 // Used for helper functions for naming files in a directory for xml output.
00056 // Except for Set methods, all methods are const or static, which provides an
00057 // "immutable value object" -- useful for peace of mind.
00058 // A FilePath with a value ending in a path separator ("like/this/") represents
00059 // a directory, otherwise it is assumed to represent a file. In either case,
00060 // it may or may not represent an actual file or directory in the file system.
00061 // Names are NOT checked for syntax correctness -- no checking for illegal
00062 // characters, malformed paths, etc.
00063
00064 class GTEST_API_ FilePath {
00065 public:
00066
00067 FilePath() : pathname_("") {
00068 }
00069
00070 FilePath(const FilePath& rhs) : pathname_(rhs.pathname_) {
00071 }
00072
00073 explicit FilePath(const std::string& pathname) : pathname_(pathname) {
00074 Normalize();
00075 }
00076
00077 FilePath& operator=(const FilePath& rhs) {
00078 Set(rhs);
00079 return *this;
00080 }
00081
00082 void Set(const FilePath& rhs) {
00083 pathname_ = rhs.pathname_;
00084 }
00085

```

```

00086 const std::string& string() const {
00087 return pathname_;
00088 }
00089
00090 const char* c_str() const {
00091 return pathname_.c_str();
00092 }
00093
00094 // Returns the current working directory, or "" if unsuccessful.
00095 static FilePath GetCurrentDir();
00096
00097 // Given directory = "dir", base_name = "test", number = 0,
00098 // extension = "xml", returns "dir/test.xml". If number is greater
00099 // than zero (e.g., 12), returns "dir/test_12.xml".
00100 // On Windows platform, uses \ as the separator rather than /.
00101 static FilePath MakeFileName(const FilePath& directory,
00102 const FilePath& base_name, int number,
00103 const char* extension);
00104
00105 // Given directory = "dir", relative_path = "test.xml",
00106 // returns "dir/test.xml".
00107 // On Windows, uses \ as the separator rather than /.
00108 static FilePath ConcatPaths(const FilePath& directory,
00109 const FilePath& relative_path);
00110
00111 // Returns a pathname for a file that does not currently exist. The pathname
00112 // will be directory/base_name.extension or
00113 // directory/base_name_<number>.extension if directory/base_name.extension
00114 // already exists. The number will be incremented until a pathname is found
00115 // that does not already exist.
00116 // Examples: 'dir/foo_test.xml' or 'dir/foo_test_1.xml'.
00117 // There could be a race condition if two or more processes are calling this
00118 // function at the same time -- they could both pick the same filename.
00119 static FilePath GenerateUniqueFileName(const FilePath& directory,
00120 const FilePath& base_name,
00121 const char* extension);
00122
00123 // Returns true if and only if the path is "".
00124
00125 bool IsEmpty() const {
00126 return pathname_.empty();
00127 }
00128
00129 // If input name has a trailing separator character, removes it and returns
00130 // the name, otherwise return the name string unmodified.
00131 // On Windows platform, uses \ as the separator, other platforms use /.
00132 FilePath RemoveTrailingPathSeparator() const;
00133
00134 // Returns a copy of the FilePath with the directory part removed.
00135 // Example: FilePath("path/to/file").RemoveDirectoryName() returns
00136 // FilePath("file"). If there is no directory part ("just_a_file"), it returns
00137 // the FilePath unmodified. If there is no file part ("just_a_dir/") it
00138 // returns an empty FilePath ("").
00139 // On Windows platform, '\' is the path separator, otherwise it is '/'.
00140 FilePath RemoveDirectoryName() const;
00141
00142 // RemoveFileName returns the directory path with the filename removed.
00143 // Example: FilePath("path/to/file").RemoveFileName() returns "path/to/".
00144 // If the FilePath is "a_file" or "/a_file", RemoveFileName returns
00145 // FilePath(".") or, on Windows, FilePath(".\\"). If the filepath does
00146 // not have a file, like "just_a_dir/", it returns the FilePath unmodified.
00147 // On Windows platform, '\' is the path separator, otherwise it is '/'.
00148 FilePath RemoveFileName() const;
00149
00150 // Returns a copy of the FilePath with the case-insensitive extension removed.
00151 // Example: FilePath("dir/file.exe").RemoveExtension("EXE") returns
00152 // FilePath("dir/file"). If a case-insensitive extension is not
00153 // found, returns a copy of the original FilePath.
00154 FilePath RemoveExtension(const char* extension) const;
00155
00156 // Creates directories so that path exists. Returns true if successful or if
00157 // the directories already exist; returns false if unable to create
00158 // directories for any reason. Will also return false if the FilePath does
00159 // not represent a directory (that is, it doesn't end with a path separator).
00160 bool CreateDirectoriesRecursively() const;
00161
00162 // Create the directory so that path exists. Returns true if successful or
00163 // if the directory already exists; returns false if unable to create the
00164 // directory for any reason, including if the parent directory does not
00165 // exist. Not named "CreateDirectory" because that's a macro on Windows.
00166 bool CreateFolder() const;
00167
00168 // Returns true if FilePath describes something in the file-system,
00169 // either a file, directory, or whatever, and that something exists.
00170 bool FileOrDirectoryExists() const;
00171
00172 // Returns true if pathname describes a directory in the file-system

```

```

00173 // that exists.
00174 bool DirectoryExists() const;
00175
00176 // Returns true if FilePath ends with a path separator, which indicates that
00177 // it is intended to represent a directory. Returns false otherwise.
00178 // This does NOT check that a directory (or file) actually exists.
00179 bool IsDirectory() const;
00180
00181 // Returns true if pathname describes a root directory. (Windows has one
00182 // root directory per disk drive.)
00183 bool IsRootDirectory() const;
00184
00185 // Returns true if pathname describes an absolute path.
00186 bool IsAbsolutePath() const;
00187
00188 private:
00189 // Replaces multiple consecutive separators with a single separator.
00190 // For example, "bar///foo" becomes "bar/foo". Does not eliminate other
00191 // redundancies that might be in a pathname involving "." or "..".
00192 //
00193 // A pathname with multiple consecutive separators may occur either through
00194 // user error or as a result of some scripts or APIs that generate a pathname
00195 // with a trailing separator. On other platforms the same API or script
00196 // may NOT generate a pathname with a trailing "/". Then elsewhere that
00197 // pathname may have another "/" and pathname components added to it,
00198 // without checking for the separator already being there.
00199 // The script language and operating system may allow paths like "foo//bar"
00200 // but some of the functions in FilePath will not handle that correctly. In
00201 // particular, RemoveTrailingPathSeparator() only removes one separator, and
00202 // it is called in CreateDirectoriesRecursively() assuming that it will change
00203 // a pathname from directory syntax (trailing separator) to filename syntax.
00204 //
00205 // On Windows this method also replaces the alternate path separator '/' with
00206 // the primary path separator '\\', so that for example "bar\\\\"foo" becomes
00207 // "bar\\foo".
00208
00209 void Normalize();
00210
00211 // Returns a pointer to the last occurrence of a valid path separator in
00212 // the FilePath. On Windows, for example, both '/' and '\' are valid path
00213 // separators. Returns NULL if no path separator was found.
00214 const char* FindLastPathSeparator() const;
00215
00216 std::string pathname_;
00217 }; // class FilePath
00218
00219 } // namespace internal
00220 } // namespace testing
00221
00222 GTEST_DISABLE_MSC_WARNINGS_POP_() // 4251
00223
00224 #endif // GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_FILEPATH_H_

```

## 16.249 gtest-internal.h

```

00001 // Copyright 2005, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // The Google C++ Testing and Mocking Framework (Google Test)

```

```

00031 //
00032 // This header file declares functions and macros used internally by
00033 // Google Test. They are subject to change without notice.
00034
00035 // IWYU pragma: private, include "gtest/gtest.h"
00036 // IWYU pragma: friend gtest/*.
00037 // IWYU pragma: friend gmock/*.
00038
00039 #ifndef GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_INTERNAL_H_
00040 #define GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_INTERNAL_H_
00041
00042 #include "gtest/internal/gtest-port.h"
00043
00044 #if GTEST_OS_LINUX
00045 #include <stdlib.h>
00046 #include <sys/types.h>
00047 #include <sys/wait.h>
00048 #include <unistd.h>
00049 #endif // GTEST_OS_LINUX
00050
00051 #if GTEST_HAS_EXCEPTIONS
00052 #include <stdexcept>
00053 #endif
00054
00055 #include <ctype.h>
00056 #include <float.h>
00057 #include <string.h>
00058
00059 #include <cstdint>
00060 #include <functional>
00061 #include <iomanip>
00062 #include <limits>
00063 #include <map>
00064 #include <set>
00065 #include <string>
00066 #include <type_traits>
00067 #include <vector>
00068
00069 #include "gtest/gtest-message.h"
00070 #include "gtest/internal/gtest-filepath.h"
00071 #include "gtest/internal/gtest-string.h"
00072 #include "gtest/internal/gtest-type-util.h"
00073
00074 // Due to C++ preprocessor weirdness, we need double indirection to
00075 // concatenate two tokens when one of them is __LINE__. Writing
00076 //
00077 // foo ## __LINE__
00078 //
00079 // will result in the token foo__LINE__, instead of foo followed by
00080 // the current line number. For more details, see
00081 // http://www.parashift.com/c++-faq-lite/misc-technical-issues.html#faq-39.6
00082 #define GTEST_CONCAT_TOKEN_(foo, bar) GTEST_CONCAT_TOKEN_IMPL_(foo, bar)
00083 #define GTEST_CONCAT_TOKEN_IMPL_(foo, bar) foo##bar
00084
00085 // Stringifies its argument.
00086 // Work around a bug in visual studio which doesn't accept code like this:
00087 //
00088 // #define GTEST_STRINGIFY_(name) #name
00089 // #define MACRO(a, b, c) ... GTEST_STRINGIFY_(a) ...
00090 // MACRO(, x, y)
00091 //
00092 // Complaining about the argument to GTEST_STRINGIFY_ being empty.
00093 // This is allowed by the spec.
00094 #define GTEST_STRINGIFY_HELPER_(name, ...) #name
00095 #define GTEST_STRINGIFY_(...) GTEST_STRINGIFY_HELPER_(__VA_ARGS__,)
00096
00097 namespace proto2 {
00098 class MessageLite;
00099 }
00100
00101 namespace testing {
00102 // Forward declarations.
00103
00104 class AssertionResult; // Result of an assertion.
00105 class Message; // Represents a failure message.
00106 class Test; // Represents a test.
00107 class TestInfo; // Information about a test.
00108 class TestPartResult; // Result of a test part.
00109 class UnitTest; // A collection of test suites.
00110
00111 template <typename T>
00112 ::std::string PrintToString(const T& value);
00113
00114 namespace internal {
00115
00116 struct TraceInfo; // Information about a trace point.

```

```

00118 class TestInfoImpl; // Opaque implementation of TestInfo
00119 class UnitTestImpl; // Opaque implementation of UnitTest
00120
00121 // The text used in failure messages to indicate the start of the
00122 // stack trace.
00123 GTEST_API_ extern const char kStackTraceMarker[];
00124
00125 // An IgnoredValue object can be implicitly constructed from ANY value.
00126 class IgnoredValue {
00127 struct Sink {};
00128
00129 public:
00130 // This constructor template allows any value to be implicitly
00131 // converted to IgnoredValue. The object has no data member and
00132 // doesn't try to remember anything about the argument. We
00133 // deliberately omit the 'explicit' keyword in order to allow the
00134 // conversion to be implicit.
00135 // Disable the conversion if T already has a magical conversion operator.
00136 // Otherwise we get ambiguity.
00137 template <typename T,
00138 typename std::enable_if<!std::is_convertible<T, Sink>::value,
00139 int>::type = 0>
00140 IgnoredValue(const T& /* ignored */) {} // NOLINT(runtime/explicit)
00141 };
00142
00143 // Appends the user-supplied message to the Google-Test-generated message.
00144 GTEST_API_ std::string AppendUserMessage(const std::string& gtest_msg,
00145 const Message& user_msg);
00146
00147 #if GTEST_HAS_EXCEPTIONS
00148
00149 GTEST_DISABLE_MSC_WARNINGS_PUSH_(
00150 4275 /* an exported class was derived from a class that was not exported */)
00151
00152 // This exception is thrown by (and only by) a failed Google Test
00153 // assertion when GTEST_FLAG(throw_on_failure) is true (if exceptions
00154 // are enabled). We derive it from std::runtime_error, which is for
00155 // errors presumably detectable only at run time. Since
00156 // std::runtime_error inherits from std::exception, many testing
00157 // frameworks know how to extract and print the message inside it.
00158 class GTEST_API_ GoogleTestFailureException : public std::runtime_error {
00159 public:
00160 explicit GoogleTestFailureException(const TestPartResult& failure);
00161 };
00162
00163 GTEST_DISABLE_MSC_WARNINGS_POP_() // 4275
00164
00165 #endif // GTEST_HAS_EXCEPTIONS
00166
00167 namespace edit_distance {
00168 // Returns the optimal edits to go from 'left' to 'right'.
00169 // All edits cost the same, with replace having lower priority than
00170 // add/remove.
00171 // Simple implementation of the Wagner-Fischer algorithm.
00172 // See http://en.wikipedia.org/wiki/Wagner-Fischer_algorithm
00173 enum EditType { kMatch, kAdd, kRemove, kReplace };
00174 GTEST_API_ std::vector<EditType> CalculateOptimalEdits(
00175 const std::vector<size_t>& left, const std::vector<size_t>& right);
00176
00177 // Same as above, but the input is represented as strings.
00178 GTEST_API_ std::vector<EditType> CalculateOptimalEdits(
00179 const std::vector<std::string>& left,
00180 const std::vector<std::string>& right);
00181
00182 // Create a diff of the input strings in Unified diff format.
00183 GTEST_API_ std::string CreateUnifiedDiff(const std::vector<std::string>& left,
00184 const std::vector<std::string>& right,
00185 size_t context = 2);
00186 } // namespace edit_distance
00187
00188 // Constructs and returns the message for an equality assertion
00189 // (e.g. ASSERT_EQ, EXPECT_STREQ, etc) failure.
00190 //
00191 // The first four parameters are the expressions used in the assertion
00192 // and their values, as strings. For example, for ASSERT_EQ(foo, bar)
00193 // where foo is 5 and bar is 6, we have:
00194 //
00195 // expected_expression: "foo"
00196 // actual_expression: "bar"
00197 // expected_value: "5"
00198 // actual_value: "6"
00199 //
00200 // The ignoring_case parameter is true if and only if the assertion is a
00201 // *_STRCASEEQ*. When it's true, the string " (ignoring case)" will
00202 // be inserted into the message.
00203 //
00204 GTEST_API_ AssertionResult EqFailure(const char* expected_expression,

```



```

00205 const char* actual_expression,
00206 const std::string& expected_value,
00207 const std::string& actual_value,
00208 bool ignoring_case);
00209
00210 // Constructs a failure message for Boolean assertions such as EXPECT_TRUE.
00211 GTEST_API_ std::string GetBoolAssertionFailureMessage(
00212 const AssertionResult& assertion_result, const char* expression_text,
00213 const char* actual_predicate_value, const char* expected_predicate_value);
00214
00215 // This template class represents an IEEE floating-point number
00216 // (either single-precision or double-precision, depending on the
00217 // template parameters).
00218 //
00219 // The purpose of this class is to do more sophisticated number
00220 // comparison. (Due to round-off error, etc, it's very unlikely that
00221 // two floating-points will be equal exactly. Hence a naive
00222 // comparison by the == operation often doesn't work.)
00223 //
00224 // Format of IEEE floating-point:
00225 //
00226 // The most-significant bit being the leftmost, an IEEE
00227 // floating-point looks like
00228 //
00229 // sign_bit exponent_bits fraction_bits
00230 //
00231 // Here, sign_bit is a single bit that designates the sign of the
00232 // number.
00233 //
00234 // For float, there are 8 exponent bits and 23 fraction bits.
00235 //
00236 // For double, there are 11 exponent bits and 52 fraction bits.
00237 //
00238 // More details can be found at
00239 // http://en.wikipedia.org/wiki/IEEE_floating-point_standard.
00240 //
00241 // Template parameter:
00242 //
00243 // RawType: the raw floating-point type (either float or double)
00244 template <typename RawType>
00245 class FloatingPoint {
00246 public:
00247 // Defines the unsigned integer type that has the same size as the
00248 // floating point number.
00249 typedef typename TypeWithSize<sizeof(RawType)>::UInt Bits;
00250
00251 // Constants.
00252
00253 // # of bits in a number.
00254 static const size_t kBitCount = 8 * sizeof(RawType);
00255
00256 // # of fraction bits in a number.
00257 static const size_t kFractionBitCount =
00258 std::numeric_limits<RawType>::digits - 1;
00259
00260 // # of exponent bits in a number.
00261 static const size_t kExponentBitCount = kBitCount - 1 - kFractionBitCount;
00262
00263 // The mask for the sign bit.
00264 static const Bits kSignBitMask = static_cast<Bits>(1) << (kBitCount - 1);
00265
00266 // The mask for the fraction bits.
00267 static const Bits kFractionBitMask = ~static_cast<Bits>(0) >>
00268 (kExponentBitCount + 1);
00269
00270 // The mask for the exponent bits.
00271 static const Bits kExponentBitMask = ~(kSignBitMask | kFractionBitMask);
00272
00273 // How many ULP's (Units in the Last Place) we want to tolerate when
00274 // comparing two numbers. The larger the value, the more error we
00275 // allow. A 0 value means that two numbers must be exactly the same
00276 // to be considered equal.
00277 //
00278 // The maximum error of a single floating-point operation is 0.5
00279 // units in the last place. On Intel CPU's, all floating-point
00280 // calculations are done with 80-bit precision, while double has 64
00281 // bits. Therefore, 4 should be enough for ordinary use.
00282 //
00283 // See the following article for more details on ULP:
00284 // http://randomascii.wordpress.com/2012/02/25/comparing-floating-point-numbers-2012-edition/
00285 static const uint32_t kMaxUlp = 4;
00286
00287 // Constructs a FloatingPoint from a raw floating-point number.
00288 //
00289 // On an Intel CPU, passing a non-normalized NAN (Not a Number)
00290 // around may change its bits, although the new value is guaranteed
00291 // to be also a NAN. Therefore, don't expect this constructor to

```

```

00292 // preserve the bits in x when x is a NAN.
00293 explicit FloatingPoint(const RawType& x) { u_.value_ = x; }
00294
00295 // Static methods
00296
00297 // Reinterprets a bit pattern as a floating-point number.
00298 //
00299 // This function is needed to test the AlmostEquals() method.
00300 static RawType ReinterpretBits(const Bits bits) {
00301 FloatingPoint fp(0);
00302 fp.u_.bits_ = bits;
00303 return fp.u_.value_;
00304 }
00305
00306 // Returns the floating-point number that represent positive infinity.
00307 static RawType Infinity() { return ReinterpretBits(kExponentBitMask); }
00308
00309 // Returns the maximum representable finite floating-point number.
00310 static RawType Max();
00311
00312 // Non-static methods
00313
00314 // Returns the bits that represents this number.
00315 const Bits& bits() const { return u_.bits_; }
00316
00317 // Returns the exponent bits of this number.
00318 Bits exponent_bits() const { return kExponentBitMask & u_.bits_; }
00319
00320 // Returns the fraction bits of this number.
00321 Bits fraction_bits() const { return kFractionBitMask & u_.bits_; }
00322
00323 // Returns the sign bit of this number.
00324 Bits sign_bit() const { return kSignBitMask & u_.bits_; }
00325
00326 // Returns true if and only if this is NAN (not a number).
00327 bool is_nan() const {
00328 // It's a NAN if the exponent bits are all ones and the fraction
00329 // bits are not entirely zeros.
00330 return (exponent_bits() == kExponentBitMask) && (fraction_bits() != 0);
00331 }
00332
00333 // Returns true if and only if this number is at most kMaxUlp's away
00334 // from rhs. In particular, this function:
00335 //
00336 // - returns false if either number is (or both are) NAN.
00337 // - treats really large numbers as almost equal to infinity.
00338 // - thinks +0.0 and -0.0 are 0 DLP's apart.
00339 bool AlmostEquals(const FloatingPoint& rhs) const {
00340 // The IEEE standard says that any comparison operation involving
00341 // a NAN must return false.
00342 if (is_nan() || rhs.is_nan()) return false;
00343
00344 return DistanceBetweenSignAndMagnitudeNumbers(u_.bits_, rhs.u_.bits_) <=
00345 kMaxUlp;
00346 }
00347
00348 private:
00349 // The data type used to store the actual floating-point number.
00350 union FloatingPointUnion {
00351 RawType value_; // The raw floating-point number.
00352 Bits bits_; // The bits that represent the number.
00353 };
00354
00355 // Converts an integer from the sign-and-magnitude representation to
00356 // the biased representation. More precisely, let N be 2 to the
00357 // power of (kBitCount - 1), an integer x is represented by the
00358 // unsigned number x + N.
00359 //
00360 // For instance,
00361 //
00362 // -N + 1 (the most negative number representable using
00363 // sign-and-magnitude) is represented by 1;
00364 // 0 is represented by N; and
00365 // N - 1 (the biggest number representable using
00366 // sign-and-magnitude) is represented by 2N - 1.
00367 //
00368 // Read http://en.wikipedia.org/wiki/Signed_number_representations
00369 // for more details on signed number representations.
00370 static Bits SignAndMagnitudeToBiased(const Bits& sam) {
00371 if (kSignBitMask & sam) {
00372 // sam represents a negative number.
00373 return ~sam + 1;
00374 } else {
00375 // sam represents a positive number.
00376 return kSignBitMask | sam;
00377 }
00378 }

```

```

00379
00380 // Given two numbers in the sign-and-magnitude representation,
00381 // returns the distance between them as an unsigned number.
00382 static Bits DistanceBetweenSignAndMagnitudeNumbers(const Bits& sam1,
00383 const Bits& sam2) {
00384 const Bits biased1 = SignAndMagnitudeToBiased(sam1);
00385 const Bits biased2 = SignAndMagnitudeToBiased(sam2);
00386 return (biased1 >= biased2) ? (biased1 - biased2) : (biased2 - biased1);
00387 }
00388
00389 FloatingPointUnion u_;
00390 };
00391
00392 // We cannot use std::numeric_limits<T>::max() as it clashes with the max()
00393 // macro defined by <windows.h>.
00394 template <>
00395 inline float FloatingPoint<float>::Max() {
00396 return FLT_MAX;
00397 }
00398 template <>
00399 inline double FloatingPoint<double>::Max() {
00400 return DBL_MAX;
00401 }
00402
00403 // Typedefs the instances of the FloatingPoint template class that we
00404 // care to use.
00405 typedef FloatingPoint<float> Float;
00406 typedef FloatingPoint<double> Double;
00407
00408 // In order to catch the mistake of putting tests that use different
00409 // test fixture classes in the same test suite, we need to assign
00410 // unique IDs to fixture classes and compare them. The TypeId type is
00411 // used to hold such IDs. The user should treat TypeId as an opaque
00412 // type: the only operation allowed on TypeId values is to compare
00413 // them for equality using the == operator.
00414 typedef const void* TypeId;
00415
00416 template <typename T>
00417 class TypeIdHelper {
00418 public:
00419 // dummy_ must not have a const type. Otherwise an overly eager
00420 // compiler (e.g. MSVC 7.1 & 8.0) may try to merge
00421 // TypeIdHelper<T>::dummy_ for different Ts as an "optimization".
00422 static bool dummy_;
00423 };
00424
00425 template <typename T>
00426 bool TypeIdHelper<T>::dummy_ = false;
00427
00428 // GetTypeId<T>() returns the ID of type T. Different values will be
00429 // returned for different types. Calling the function twice with the
00430 // same type argument is guaranteed to return the same ID.
00431 template <typename T>
00432 TypeId GetTypeId() {
00433 // The compiler is required to allocate a different
00434 // TypeIdHelper<T>::dummy_ variable for each T used to instantiate
00435 // the template. Therefore, the address of dummy_ is guaranteed to
00436 // be unique.
00437 return &(TypeIdHelper<T>::dummy_);
00438 }
00439
00440 // Returns the type ID of ::testing::Test. Always call this instead
00441 // of GetTypeId< ::testing::Test>() to get the type ID of
00442 // ::testing::Test, as the latter may give the wrong result due to a
00443 // suspected linker bug when compiling Google Test as a Mac OS X
00444 // framework.
00445 GTEST_API_ TypeId GetTestTypeId();
00446
00447 // Defines the abstract factory interface that creates instances
00448 // of a Test object.
00449 class TestFactoryBase {
00450 public:
00451 virtual ~TestFactoryBase() {}
00452
00453 // Creates a test instance to run. The instance is both created and destroyed
00454 // within TestInfoImpl::Run()
00455 virtual Test* CreateTest() = 0;
00456
00457 protected:
00458 TestFactoryBase() {}
00459
00460 private:
00461 TestFactoryBase(const TestFactoryBase&) = delete;
00462 TestFactoryBase& operator=(const TestFactoryBase&) = delete;
00463 };
00464
00465 // This class provides implementation of TestFactoryBase interface.

```

```

00466 // It is used in TEST and TEST_F macros.
00467 template <class TestClass>
00468 class TestFactoryImpl : public TestFactoryBase {
00469 public:
00470 Test* CreateTest() override { return new TestClass; }
00471 };
00472
00473 #if GTEST_OS_WINDOWS
00474
00475 // Predicate-formatters for implementing the HRESULT checking macros
00476 // {ASSERT|EXPECT}_HRESULT_{SUCCEEDED|FAILED}
00477 // We pass a long instead of HRESULT to avoid causing an
00478 // include dependency for the HRESULT type.
00479 GTEST_API_ AssertionResult IsHRESULTSuccess(const char* expr,
00480 long hr); // NOLINT
00481 GTEST_API_ AssertionResult IsHRESULTFailure(const char* expr,
00482 long hr); // NOLINT
00483
00484 #endif // GTEST_OS_WINDOWS
00485
00486 // Types of SetUpTestSuite() and TearDownTestSuite() functions.
00487 using SetUpTestSuiteFunc = void (*)();
00488 using TearDownTestSuiteFunc = void (*)();
00489
00490 struct CodeLocation {
00491 CodeLocation(const std::string& a_file, int a_line)
00492 : file(a_file), line(a_line) {}
00493
00494 std::string file;
00495 int line;
00496 };
00497
00498 // Helper to identify which setup function for TestCase / TestSuite to call.
00499 // Only one function is allowed, either TestCase or TestSuite but not both.
00500
00501 // Utility functions to help SuiteApiResolver
00502 using SetUpTearDownSuiteFuncType = void (*)();
00503
00504 inline SetUpTearDownSuiteFuncType GetNotDefaultOrNull(
00505 SetUpTearDownSuiteFuncType a, SetUpTearDownSuiteFuncType def) {
00506 return a == def ? nullptr : a;
00507 }
00508
00509 template <typename T>
00510 // Note that SuiteApiResolver inherits from T because
00511 // SetUpTestSuite()/TearDownTestSuite() could be protected. This way
00512 // SuiteApiResolver can access them.
00513 struct SuiteApiResolver : T {
00514 // testing::Test is only forward declared at this point. So we make it a
00515 // dependent class for the compiler to be OK with it.
00516 using Test =
00517 typename std::conditional<sizeof(T) != 0, ::testing::Test, void>::type;
00518
00519 static SetUpTearDownSuiteFuncType GetSetUpCaseOrSuite(const char* filename,
00520 int line_num) {
00521 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00522 SetUpTearDownSuiteFuncType test_case_fp =
00523 GetNotDefaultOrNull(&T::SetUpTestCase, &Test::SetUpTestCase);
00524 SetUpTearDownSuiteFuncType test_suite_fp =
00525 GetNotDefaultOrNull(&T::SetUpTestSuite, &Test::SetUpTestSuite);
00526
00527 GTEST_CHECK_(!test_case_fp || !test_suite_fp)
00528 << "Test can not provide both SetUpTestSuite and SetUpTestCase, please "
00529 << "make sure there is only one present at "
00530 << filename << ":" << line_num;
00531
00532 return test_case_fp != nullptr ? test_case_fp : test_suite_fp;
00533 #else
00534 (void)(filename);
00535 (void)(line_num);
00536 return &T::SetUpTestSuite;
00537 #endif
00538 }
00539
00540 static SetUpTearDownSuiteFuncType GetTearDownCaseOrSuite(const char* filename,
00541 int line_num) {
00542 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00543 SetUpTearDownSuiteFuncType test_case_fp =
00544 GetNotDefaultOrNull(&T::TearDownTestCase, &Test::TearDownTestCase);
00545 SetUpTearDownSuiteFuncType test_suite_fp =
00546 GetNotDefaultOrNull(&T::TearDownTestSuite, &Test::TearDownTestSuite);
00547
00548 GTEST_CHECK_(!test_case_fp || !test_suite_fp)
00549 << "Test can not provide both TearDownTestSuite and TearDownTestCase,"
00550 << " please make sure there is only one present at "
00551 << filename << ":" << line_num;
00552

```

```

00553 return test_case_fp != nullptr ? test_case_fp : test_suite_fp;
00554 #else
00555 (void)(filename);
00556 (void)(line_num);
00557 return &T::TearDownTestSuite;
00558 #endif
00559 }
00560 };
00561
00562 // Creates a new TestInfo object and registers it with Google Test;
00563 // returns the created object.
00564 //
00565 // Arguments:
00566 //
00567 // test_suite_name: name of the test suite
00568 // name: name of the test
00569 // type_param: the name of the test's type parameter, or NULL if
00570 // this is not a typed or a type-parameterized test.
00571 // value_param: text representation of the test's value parameter,
00572 // or NULL if this is not a type-parameterized test.
00573 // code_location: code location where the test is defined
00574 // fixture_class_id: ID of the test fixture class
00575 // set_up_tc: pointer to the function that sets up the test suite
00576 // tear_down_tc: pointer to the function that tears down the test suite
00577 // factory: pointer to the factory that creates a test object.
00578 // The newly created TestInfo instance will assume
00579 // ownership of the factory object.
00580 GTEST_API_ TestInfo* MakeAndRegisterTestInfo(
00581 const char* test_suite_name, const char* name, const char* type_param,
00582 const char* value_param, CodeLocation code_location,
00583 TypeId fixture_class_id, SetUpTestSuiteFunc set_up_tc,
00584 TearDownTestSuiteFunc tear_down_tc, TestFactoryBase* factory);
00585
00586 // If *pstr starts with the given prefix, modifies *pstr to be right
00587 // past the prefix and returns true; otherwise leaves *pstr unchanged
00588 // and returns false. None of pstr, *pstr, and prefix can be NULL.
00589 GTEST_API_ bool SkipPrefix(const char* prefix, const char** pstr);
00590
00591 GTEST_DISABLE_MSC_WARNINGS_PUSH_(4251 \
00592 /* class A needs to have dll-interface to be used by clients of class B */)
00593
00594 // State of the definition of a type-parameterized test suite.
00595 class GTEST_API_ TypedTestSuitePState {
00596 public:
00597 TypedTestSuitePState() : registered_(false) {}
00598
00599 // Adds the given test name to defined_test_names_ and return true
00600 // if the test suite hasn't been registered; otherwise aborts the
00601 // program.
00602 bool AddTestName(const char* file, int line, const char* case_name,
00603 const char* test_name) {
00604 if (registered_) {
00605 fprintf(stderr,
00606 "%s Test %s must be defined before "
00607 "REGISTER_TYPED_TEST_SUITE_P(%s, ...).\n",
00608 FormatFileLocation(file, line).c_str(), test_name, case_name);
00609 fflush(stderr);
00610 posix::Abort();
00611 }
00612 registered_tests_.insert(
00613 ::std::make_pair(test_name, CodeLocation(file, line)));
00614 return true;
00615 }
00616
00617 bool TestExists(const std::string& test_name) const {
00618 return registered_tests_.count(test_name) > 0;
00619 }
00620
00621 const CodeLocation& GetCodeLocation(const std::string& test_name) const {
00622 RegisteredTestsMap::const_iterator it = registered_tests_.find(test_name);
00623 GTEST_CHECK_(it != registered_tests_.end());
00624 return it->second;
00625 }
00626
00627 // Verifies that registered_tests match the test names in
00628 // defined_test_names_; returns registered_tests if successful, or
00629 // aborts the program otherwise.
00630 const char* VerifyRegisteredTestNames(const char* test_suite_name,
00631 const char* file, int line,
00632 const char* registered_tests);
00633
00634 private:
00635 typedef ::std::map<std::string, CodeLocation, std::less<> > RegisteredTestsMap;
00636
00637 bool registered_;
00638 RegisteredTestsMap registered_tests_;
00639 };

```

```

00640
00641 // Legacy API is deprecated but still available
00642 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00643 using TypedTestCasePState = TypedTestSuitePState;
00644 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00645
00646 GTEST_DISABLE_MSC_WARNINGS_POP_() // 4251
00647
00648 // Skips to the first non-space char after the first comma in 'str';
00649 // returns NULL if no comma is found in 'str'.
00650 inline const char* SkipComma(const char* str) {
00651 const char* comma = strchr(str, ',');
00652 if (comma == nullptr) {
00653 return nullptr;
00654 }
00655 while (IsSpace(*(++comma))) {
00656 }
00657 return comma;
00658 }
00659
00660 // Returns the prefix of 'str' before the first comma in it; returns
00661 // the entire string if it contains no comma.
00662 inline std::string GetPrefixUntilComma(const char* str) {
00663 const char* comma = strchr(str, ',');
00664 return comma == nullptr ? str : std::string(str, comma);
00665 }
00666
00667 // Splits a given string on a given delimiter, populating a given
00668 // vector with the fields.
00669 void SplitString(const ::std::string& str, char delimiter,
00670 ::std::vector<::std::string>* dest);
00671
00672 // The default argument to the template below for the case when the user does
00673 // not provide a name generator.
00674 struct DefaultNameGenerator {
00675 template <typename T>
00676 static std::string GetName(int i) {
00677 return StreamableToString(i);
00678 }
00679 };
00680
00681 template <typename Provided = DefaultNameGenerator>
00682 struct NameGeneratorSelector {
00683 typedef Provided type;
00684 };
00685
00686 template <typename NameGenerator>
00687 void GenerateNamesRecursively(internal::None, std::vector<std::string>*, int) {}
00688
00689 template <typename NameGenerator, typename Types>
00690 void GenerateNamesRecursively(Types, std::vector<std::string>* result, int i) {
00691 result->push_back(NameGenerator::template GetName<typename Types::Head>(i));
00692 GenerateNamesRecursively<NameGenerator>(typename Types::Tail(), result,
00693 i + 1);
00694 }
00695
00696 template <typename NameGenerator, typename Types>
00697 std::vector<std::string> GenerateNames() {
00698 std::vector<std::string> result;
00699 GenerateNamesRecursively<NameGenerator>(Types(), &result, 0);
00700 return result;
00701 }
00702
00703 // TypeParameterizedTest<Fixture, TestSel, Types>::Register()
00704 // registers a list of type-parameterized tests with Google Test. The
00705 // return value is insignificant - we just need to return something
00706 // such that we can call this function in a namespace scope.
00707 //
00708 // Implementation note: The GTEST_TEMPLATE_ macro declares a template
00709 // template parameter. It's defined in gtest-type-util.h.
00710 template <GTEST_TEMPLATE_ Fixture, class TestSel, typename Types>
00711 class TypeParameterizedTest {
00712 public:
00713 // 'index' is the index of the test in the type list 'Types'
00714 // specified in INSTANTIATE_TYPED_TEST_SUITE_P(Prefix, TestSuite,
00715 // Types). Valid values for 'index' are [0, N - 1] where N is the
00716 // length of Types.
00717 static bool Register(const char* prefix, const CodeLocation& code_location,
00718 const char* case_name, const char* test_names, int index,
00719 const std::vector<std::string>& type_names =
00720 GenerateNames<DefaultNameGenerator, Types>()) {
00721 typedef typename Types::Head Type;
00722 typedef Fixture<Type> FixtureClass;
00723 typedef typename GTEST_BIND_(TestSel, Type) TestClass;
00724
00725 // First, registers the first type-parameterized test in the type
00726 // list.

```

```

00727 MakeAndRegisterTestInfo(
00728 (std::string(prefix) + (prefix[0] == '\\0' ? "" : "/") + case_name +
00729 "/" + type_names[static_cast<size_t>(index)]).
00730 .c_str(),
00731 StripTrailingSpaces(GetPrefixUntilComma(test_names)).c_str(),
00732 GetTypeName<Type>().c_str(),
00733 nullptr, // No value parameter.
00734 code_location, GetTypeId<FixtureClass>(),
00735 SuiteApiResolver<TestClass>::GetSetUpCaseOrSuite(
00736 code_location.file.c_str(), code_location.line),
00737 SuiteApiResolver<TestClass>::GetTearDownCaseOrSuite(
00738 code_location.file.c_str(), code_location.line),
00739 new TestFactoryImpl<TestClass>);
00740
00741 // Next, recurses (at compile time) with the tail of the type list.
00742 return TypeParameterizedTest<Fixture, TestSel,
00743 typename Types::Tail>::Register(prefix,
00744 code_location,
00745 case_name,
00746 test_names,
00747 index + 1,
00748 type_names);
00749 }
00750 };
00751
00752 // The base case for the compile time recursion.
00753 template <GTEST_TEMPLATE_ Fixture, class TestSel>
00754 class TypeParameterizedTest<Fixture, TestSel, internal::None> {
00755 public:
00756 static bool Register(const char* /*prefix*/, const CodeLocation&,
00757 const char* /*case_name*/, const char* /*test_names*/,
00758 int /*index*/,
00759 const std::vector<std::string>& =
00760 std::vector<std::string>() /*type_names*/) {
00761 return true;
00762 }
00763 };
00764
00765 GTEST_API_ void RegisterTypeParameterizedTestSuite(const char* test_suite_name,
00766 CodeLocation code_location);
00767 GTEST_API_ void RegisterTypeParameterizedTestSuiteInstantiation(
00768 const char* case_name);
00769
00770 // TypeParameterizedTestSuite<Fixture, Tests, Types>::Register()
00771 // registers *all combinations* of 'Tests' and 'Types' with Google
00772 // Test. The return value is insignificant - we just need to return
00773 // something such that we can call this function in a namespace scope.
00774 template <GTEST_TEMPLATE_ Fixture, typename Tests, typename Types>
00775 class TypeParameterizedTestSuite {
00776 public:
00777 static bool Register(const char* prefix, CodeLocation code_location,
00778 const TypedTestSuitePState* state, const char* case_name,
00779 const char* test_names,
00780 const std::vector<std::string>& type_names =
00781 GenerateNames<DefaultNameGenerator, Types>()) {
00782 RegisterTypeParameterizedTestSuiteInstantiation(case_name);
00783 std::string test_name =
00784 StripTrailingSpaces(GetPrefixUntilComma(test_names));
00785 if (!state->TestExists(test_name)) {
00786 fprintf(stderr, "Failed to get code location for test %s.%s at %s.",
00787 case_name, test_name.c_str(),
00788 FormatFileLocation(code_location.file.c_str(), code_location.line)
00789 .c_str());
00790 fflush(stderr);
00791 posix::Abort();
00792 }
00793 const CodeLocation& test_location = state->GetCodeLocation(test_name);
00794
00795 typedef typename Tests::Head Head;
00796
00797 // First, register the first test in 'Test' for each type in 'Types'.
00798 TypeParameterizedTest<Fixture, Head, Types>::Register(
00799 prefix, test_location, case_name, test_names, 0, type_names);
00800
00801 // Next, recurses (at compile time) with the tail of the test list.
00802 return TypeParameterizedTestSuite<Fixture, typename Tests::Tail,
00803 Types>::Register(prefix, code_location,
00804 state, case_name,
00805 SkipComma(test_names),
00806 type_names);
00807 }
00808 };
00809
00810 // The base case for the compile time recursion.
00811 template <GTEST_TEMPLATE_ Fixture, typename Types>
00812 class TypeParameterizedTestSuite<Fixture, internal::None, Types> {
00813 public:

```

```

00814 static bool Register(const char* /*prefix*/, const CodeLocation&,
00815 const TypedTestSuitePState* /*state*/,
00816 const char* /*case_name*/, const char* /*test_names*/,
00817 const std::vector<std::string>& =
00818 std::vector<std::string>() /*type_names*/) {
00819 return true;
00820 }
00821 };
00822
00823 // Returns the current OS stack trace as an std::string.
00824 //
00825 // The maximum number of stack frames to be included is specified by
00826 // the gtest_stack_trace_depth flag. The skip_count parameter
00827 // specifies the number of top frames to be skipped, which doesn't
00828 // count against the number of frames to be included.
00829 //
00830 // For example, if Foo() calls Bar(), which in turn calls
00831 // GetCurrentOsStackTraceExceptTop(..., 1), Foo() will be included in
00832 // the trace but Bar() and GetCurrentOsStackTraceExceptTop() won't.
00833 GTEST_API_ std::string GetCurrentOsStackTraceExceptTop(int skip_count);
00834
00835 // Helpers for suppressing warnings on unreachable code or constant
00836 // condition.
00837
00838 // Always returns true.
00839 GTEST_API_ bool AlwaysTrue();
00840
00841 // Always returns false.
00842 inline bool AlwaysFalse() { return !AlwaysTrue(); }
00843
00844 // Helper for suppressing false warning from Clang on a const char*
00845 // variable declared in a conditional expression always being NULL in
00846 // the else branch.
00847 struct GTEST_API_ ConstCharPtr {
00848 ConstCharPtr(const char* str) : value(str) {}
00849 operator bool() const { return true; }
00850 const char* value;
00851 };
00852
00853 // Helper for declaring std::string within 'if' statement
00854 // in pre C++17 build environment.
00855 struct TrueWithString {
00856 TrueWithString() = default;
00857 explicit TrueWithString(const char* str) : value(str) {}
00858 explicit TrueWithString(const std::string& str) : value(str) {}
00859 explicit operator bool() const { return true; }
00860 std::string value;
00861 };
00862
00863 // A simple Linear Congruential Generator for generating random
00864 // numbers with a uniform distribution. Unlike rand() and srand(), it
00865 // doesn't use global state (and therefore can't interfere with user
00866 // code). Unlike rand_r(), it's portable. An LCG isn't very random,
00867 // but it's good enough for our purposes.
00868 class GTEST_API_ Random {
00869 public:
00870 static const uint32_t kMaxRange = 1u << 31;
00871
00872 explicit Random(uint32_t seed) : state_(seed) {}
00873
00874 void Reseed(uint32_t seed) { state_ = seed; }
00875
00876 // Generates a random number from [0, range). Crashes if 'range' is
00877 // 0 or greater than kMaxRange.
00878 uint32_t Generate(uint32_t range);
00879
00880 private:
00881 uint32_t state_;
00882 Random(const Random&) = delete;
00883 Random& operator=(const Random&) = delete;
00884 };
00885
00886 // Turns const U&, U&, const U, and U all into U.
00887 #define GTEST_REMOVE_REFERENCE_AND_CONST_(T) \
00888 typename std::remove_const<typename std::remove_reference<T>::type>::type
00889
00890 // HasDebugStringAndShortDebugString<T>::value is a compile-time bool constant
00891 // that's true if and only if T has methods DebugString() and ShortDebugString()
00892 // that return std::string.
00893 template <typename T>
00894 class HasDebugStringAndShortDebugString {
00895 private:
00896 template <typename C>
00897 static auto CheckDebugString(C*) -> typename std::is_same<
00898 std::string, decltype(std::declval<const C>().DebugString())>::type;
00899 template <typename>
00900 static std::false_type CheckDebugString(...);

```



```

00901
00902 template <typename C>
00903 static auto CheckShortDebugString(C*) -> typename std::is_same<
00904 std::string, decltype(std::declval<const C>().ShortDebugString())>::type;
00905 template <typename>
00906 static std::false_type CheckShortDebugString(...);
00907
00908 using HasDebugStringType = decltype(CheckDebugString<T>(nullptr));
00909 using HasShortDebugStringType = decltype(CheckShortDebugString<T>(nullptr));
00910
00911 public:
00912 static constexpr bool value =
00913 HasDebugStringType::value && HasShortDebugStringType::value;
00914 };
00915
00916 template <typename T>
00917 constexpr bool HasDebugStringAndShortDebugString<T>::value;
00918
00919 // When the compiler sees expression IsContainerTest<C>(0), if C is an
00920 // STL-style container class, the first overload of IsContainerTest
00921 // will be viable (since both C::iterator* and C::const_iterator* are
00922 // valid types and NULL can be implicitly converted to them). It will
00923 // be picked over the second overload as 'int' is a perfect match for
00924 // the type of argument 0. If C::iterator or C::const_iterator is not
00925 // a valid type, the first overload is not viable, and the second
00926 // overload will be picked. Therefore, we can determine whether C is
00927 // a container class by checking the type of IsContainerTest<C>(0).
00928 // The value of the expression is insignificant.
00929 //
00930 // In C++11 mode we check the existence of a const_iterator and that an
00931 // iterator is properly implemented for the container.
00932 //
00933 // For pre-C++11 that we look for both C::iterator and C::const_iterator.
00934 // The reason is that C++ injects the name of a class as a member of the
00935 // class itself (e.g. you can refer to class iterator as either
00936 // 'iterator' or 'iterator::iterator'). If we look for C::iterator
00937 // only, for example, we would mistakenly think that a class named
00938 // iterator is an STL container.
00939 //
00940 // Also note that the simpler approach of overloading
00941 // IsContainerTest(typename C::const_iterator*) and
00942 // IsContainerTest(...) doesn't work with Visual Age C++ and Sun C++.
00943 typedef int IsContainer;
00944 template <class C,
00945 class Iterator = decltype(std::declval<const C&>().begin()),
00946 class = decltype(std::declval<const C&>().end()),
00947 class = decltype(++std::declval<Iterator&>()),
00948 class = decltype(*std::declval<Iterator>()),
00949 class = typename C::const_iterator>
00950 IsContainer IsContainerTest(int /* dummy */) {
00951 return 0;
00952 }
00953
00954 typedef char IsNotContainer;
00955 template <class C>
00956 IsNotContainer IsContainerTest(long /* dummy */) {
00957 return '\0';
00958 }
00959
00960 // Trait to detect whether a type T is a hash table.
00961 // The heuristic used is that the type contains an inner type `hasher` and does
00962 // not contain an inner type `reverse_iterator`.
00963 // If the container is iterable in reverse, then order might actually matter.
00964 template <typename T>
00965 struct IsHashTable {
00966 private:
00967 template <typename U>
00968 static char test(typename U::hasher*, typename U::reverse_iterator*);
00969 template <typename U>
00970 static int test(typename U::hasher*, ...);
00971 template <typename U>
00972 static char test(...);
00973
00974 public:
00975 static const bool value = sizeof(test<T>(nullptr, nullptr)) == sizeof(int);
00976 };
00977
00978 template <typename T>
00979 const bool IsHashTable<T>::value;
00980
00981 template <typename C,
00982 bool = sizeof(IsContainerTest<C>(0)) == sizeof(IsContainer)>
00983 struct IsRecursiveContainerImpl;
00984
00985 template <typename C>
00986 struct IsRecursiveContainerImpl<C, false> : public std::false_type {};
00987

```

```

00988 // Since the IsRecursiveContainerImpl depends on the IsContainerTest we need to
00989 // obey the same inconsistencies as the IsContainerTest, namely check if
00990 // something is a container is relying on only const_iterator in C++11 and
00991 // is relying on both const_iterator and iterator otherwise
00992 template <typename C>
00993 struct IsRecursiveContainerImpl<C, true> {
00994 using value_type = decltype(*std::declval<typename C::const_iterator>());
00995 using type =
00996 std::is_same<typename std::remove_const<
00997 typename std::remove_reference<value_type>::type>::type,
00998 C>;
00999 };
01000
01001 // IsRecursiveContainer<Type> is a unary compile-time predicate that
01002 // evaluates whether C is a recursive container type. A recursive container
01003 // type is a container type whose value_type is equal to the container type
01004 // itself. An example for a recursive container type is
01005 // boost::filesystem::path, whose iterator has a value_type that is equal to
01006 // boost::filesystem::path.
01007 template <typename C>
01008 struct IsRecursiveContainer : public IsRecursiveContainerImpl<C>::type {};
01009
01010 // Utilities for native arrays.
01011
01012 // ArrayEq() compares two k-dimensional native arrays using the
01013 // elements' operator==, where k can be any integer >= 0. When k is
01014 // 0, ArrayEq() degenerates into comparing a single pair of values.
01015
01016 template <typename T, typename U>
01017 bool ArrayEq(const T* lhs, size_t size, const U* rhs);
01018
01019 // This generic version is used when k is 0.
01020 template <typename T, typename U>
01021 inline bool ArrayEq(const T& lhs, const U& rhs) {
01022 return lhs == rhs;
01023 }
01024
01025 // This overload is used when k >= 1.
01026 template <typename T, typename U, size_t N>
01027 inline bool ArrayEq(const T (&lhs)[N], const U (&rhs)[N]) {
01028 return internal::ArrayEq(lhs, N, rhs);
01029 }
01030
01031 // This helper reduces code bloat. If we instead put its logic inside
01032 // the previous ArrayEq() function, arrays with different sizes would
01033 // lead to different copies of the template code.
01034 template <typename T, typename U>
01035 bool ArrayEq(const T* lhs, size_t size, const U* rhs) {
01036 for (size_t i = 0; i != size; i++) {
01037 if (!internal::ArrayEq(lhs[i], rhs[i])) return false;
01038 }
01039 return true;
01040 }
01041
01042 // Finds the first element in the iterator range [begin, end) that
01043 // equals elem. Element may be a native array type itself.
01044 template <typename Iter, typename Element>
01045 Iter ArrayAwareFind(Iter begin, Iter end, const Element& elem) {
01046 for (Iter it = begin; it != end; ++it) {
01047 if (internal::ArrayEq(*it, elem)) return it;
01048 }
01049 return end;
01050 }
01051
01052 // CopyArray() copies a k-dimensional native array using the elements'
01053 // operator==, where k can be any integer >= 0. When k is 0,
01054 // CopyArray() degenerates into copying a single value.
01055
01056 template <typename T, typename U>
01057 void CopyArray(const T* from, size_t size, U* to);
01058
01059 // This generic version is used when k is 0.
01060 template <typename T, typename U>
01061 inline void CopyArray(const T& from, U* to) {
01062 *to = from;
01063 }
01064
01065 // This overload is used when k >= 1.
01066 template <typename T, typename U, size_t N>
01067 inline void CopyArray(const T (&from)[N], U (*to)[N]) {
01068 internal::CopyArray(from, N, *to);
01069 }
01070
01071 // This helper reduces code bloat. If we instead put its logic inside
01072 // the previous CopyArray() function, arrays with different sizes
01073 // would lead to different copies of the template code.
01074 template <typename T, typename U>

```

```

01075 void CopyArray(const T* from, size_t size, U* to) {
01076 for (size_t i = 0; i != size; i++) {
01077 internal::CopyArray(from[i], to + i);
01078 }
01079 }
01080
01081 // The relation between an NativeArray object (see below) and the
01082 // native array it represents.
01083 // We use 2 different structs to allow non-copyable types to be used, as long
01084 // as RelationToSourceReference() is passed.
01085 struct RelationToSourceReference {};
01086 struct RelationToSourceCopy {};
01087
01088 // Adapts a native array to a read-only STL-style container. Instead
01089 // of the complete STL container concept, this adaptor only implements
01090 // members useful for Google Mock's container matchers. New members
01091 // should be added as needed. To simplify the implementation, we only
01092 // support Element being a raw type (i.e. having no top-level const or
01093 // reference modifier). It's the client's responsibility to satisfy
01094 // this requirement. Element can be an array type itself (hence
01095 // multi-dimensional arrays are supported).
01096 template <typename Element>
01097 class NativeArray {
01098 public:
01099 // STL-style container typedefs.
01100 typedef Element value_type;
01101 typedef Element* iterator;
01102 typedef const Element* const_iterator;
01103
01104 // Constructs from a native array. References the source.
01105 NativeArray(const Element* array, size_t count, RelationToSourceReference) {
01106 InitRef(array, count);
01107 }
01108
01109 // Constructs from a native array. Copies the source.
01110 NativeArray(const Element* array, size_t count, RelationToSourceCopy) {
01111 InitCopy(array, count);
01112 }
01113
01114 // Copy constructor.
01115 NativeArray(const NativeArray& rhs) {
01116 (this->*rhs.clone_)(rhs.array_, rhs.size_);
01117 }
01118
01119 ~NativeArray() {
01120 if (clone_ != &NativeArray::InitRef) delete[] array_;
01121 }
01122
01123 // STL-style container methods.
01124 size_t size() const { return size_; }
01125 const_iterator begin() const { return array_; }
01126 const_iterator end() const { return array_ + size_; }
01127 bool operator==(const NativeArray& rhs) const {
01128 return size() == rhs.size() && ArrayEq(begin(), size(), rhs.begin());
01129 }
01130
01131 private:
01132 static_assert(!std::is_const<Element>::value, "Type must not be const");
01133 static_assert(!std::is_reference<Element>::value,
01134 "Type must not be a reference");
01135
01136 // Initializes this object with a copy of the input.
01137 void InitCopy(const Element* array, size_t a_size) {
01138 Element* const copy = new Element[a_size];
01139 CopyArray(array, a_size, copy);
01140 array_ = copy;
01141 size_ = a_size;
01142 clone_ = &NativeArray::InitCopy;
01143 }
01144
01145 // Initializes this object with a reference of the input.
01146 void InitRef(const Element* array, size_t a_size) {
01147 array_ = array;
01148 size_ = a_size;
01149 clone_ = &NativeArray::InitRef;
01150 }
01151
01152 const Element* array_;
01153 size_t size_;
01154 void (NativeArray::*clone_)(const Element*, size_t);
01155 };
01156
01157 // Backport of std::index_sequence.
01158 template <size_t... Is>
01159 struct IndexSequence {
01160 using type = IndexSequence;
01161 };

```

```

01162
01163 // Double the IndexSequence, and one if plus_one is true.
01164 template <bool plus_one, typename T, size_t sizeofT>
01165 struct DoubleSequence;
01166 template <size_t... I, size_t sizeofT>
01167 struct DoubleSequence<true, IndexSequence<I...>, sizeofT> {
01168 using type = IndexSequence<I..., (sizeofT + I)..., 2 * sizeofT>;
01169 };
01170 template <size_t... I, size_t sizeofT>
01171 struct DoubleSequence<false, IndexSequence<I...>, sizeofT> {
01172 using type = IndexSequence<I..., (sizeofT + I)...>;
01173 };
01174
01175 // Backport of std::make_index_sequence.
01176 // It uses O(ln(N)) instantiation depth.
01177 template <size_t N>
01178 struct MakeIndexSequenceImpl
01179 : DoubleSequence<N % 2 == 1, typename MakeIndexSequenceImpl<N / 2>::type,
01180 N / 2>::type {};
01181
01182 template <>
01183 struct MakeIndexSequenceImpl<0> : IndexSequence<> {};
01184
01185 template <size_t N>
01186 using MakeIndexSequence = typename MakeIndexSequenceImpl<N>::type;
01187
01188 template <typename... T>
01189 using IndexSequenceFor = typename MakeIndexSequence<sizeof...(T)>::type;
01190
01191 template <size_t>
01192 struct Ignore {
01193 Ignore(...); // NOLINT
01194 };
01195
01196 template <typename>
01197 struct ElemFromListImpl;
01198 template <size_t... I>
01199 struct ElemFromListImpl<IndexSequence<I...>> {
01200 // We make Ignore a template to solve a problem with MSVC.
01201 // A non-template Ignore would work fine with 'decltype(Ignore(I))...', but
01202 // MSVC doesn't understand how to deal with that pack expansion.
01203 // Use '0 * I' to have a single instantiation of Ignore.
01204 template <typename R>
01205 static R Apply(Ignore<0 * I>..., R (*)(), ...);
01206 };
01207
01208 template <size_t N, typename... T>
01209 struct ElemFromList {
01210 using type =
01211 decltype(ElemFromListImpl<typename MakeIndexSequence<N>::type>::Apply(
01212 static_cast<T (*)()>(nullptr)...));
01213 };
01214
01215 struct FlatTupleConstructTag {};
01216
01217 template <typename... T>
01218 class FlatTuple;
01219
01220 template <typename Derived, size_t I>
01221 struct FlatTupleElemBase;
01222
01223 template <typename... T, size_t I>
01224 struct FlatTupleElemBase<FlatTuple<T...>, I> {
01225 using value_type = typename ElemFromList<I, T...>::type;
01226 FlatTupleElemBase() = default;
01227 template <typename Arg>
01228 explicit FlatTupleElemBase(FlatTupleConstructTag, Arg&& t)
01229 : value(std::forward<Arg>(t)) {}
01230 value_type value;
01231 };
01232
01233 template <typename Derived, typename Idx>
01234 struct FlatTupleBase;
01235
01236 template <size_t... Idx, typename... T>
01237 struct FlatTupleBase<FlatTuple<T...>, IndexSequence<Idx...>>
01238 : FlatTupleElemBase<FlatTuple<T...>, Idx>... {
01239 using Indices = IndexSequence<Idx...>;
01240 FlatTupleBase() = default;
01241 template <typename... Args>
01242 explicit FlatTupleBase(FlatTupleConstructTag, Args&&... args)
01243 : FlatTupleElemBase<FlatTuple<T...>, Idx>(FlatTupleConstructTag{},
01244 std::forward<Args>(args))... {}
01245
01246 template <size_t I>
01247 const typename ElemFromList<I, T...>::type& Get() const {
01248 return FlatTupleElemBase<FlatTuple<T...>, I>::value;

```

```

01249 }
01250
01251 template <size_t I>
01252 typename ElemFromList<I, T...>::type& Get() {
01253 return FlatTupleElemBase<FlatTuple<T...>, I>::value;
01254 }
01255
01256 template <typename F>
01257 auto Apply(F&& f) -> decltype(std::forward<F>(f) (this->Get<Idx>()...)) {
01258 return std::forward<F>(f) (Get<Idx>()...);
01259 }
01260
01261 template <typename F>
01262 auto Apply(F&& f) const -> decltype(std::forward<F>(f) (this->Get<Idx>()...)) {
01263 return std::forward<F>(f) (Get<Idx>()...);
01264 }
01265 };
01266
01267 // Analog to std::tuple but with different tradeoffs.
01268 // This class minimizes the template instantiation depth, thus allowing more
01269 // elements than std::tuple would. std::tuple has been seen to require an
01270 // instantiation depth of more than 10x the number of elements in some
01271 // implementations.
01272 // FlatTuple and ElemFromList are not recursive and have a fixed depth
01273 // regardless of T...
01274 // MakeIndexSequence, on the other hand, it is recursive but with an
01275 // instantiation depth of O(ln(N)).
01276 template <typename... T>
01277 class FlatTuple
01278 : private FlatTupleBase<FlatTuple<T...>,
01279 typename MakeIndexSequence<sizeof...(T)>::type> {
01280 using Indices = typename FlatTupleBase<
01281 FlatTuple<T...>, typename MakeIndexSequence<sizeof...(T)>::type>::Indices;
01282
01283 public:
01284 FlatTuple() = default;
01285 template <typename... Args>
01286 explicit FlatTuple(FlatTupleConstructTag tag, Args&&... args)
01287 : FlatTupleBase(tag, std::forward<Args>(args)...) {}
01288
01289 using FlatTuple::FlatTupleBase::Apply;
01290 using FlatTuple::FlatTupleBase::Get;
01291 };
01292
01293 // Utility functions to be called with static_assert to induce deprecation
01294 // warnings.
01295 GTEST_INTERNAL_DEPRECATED (
01296 "INSTITUTE_TEST_CASE_P is deprecated, please use "
01297 "INSTITUTE_TEST_SUITE_P")
01298 constexpr bool InstantiateTestCase_P_IsDeprecated() { return true; }
01299
01300 GTEST_INTERNAL_DEPRECATED (
01301 "TYPED_TEST_CASE_P is deprecated, please use "
01302 "TYPED_TEST_SUITE_P")
01303 constexpr bool TypedTestCase_P_IsDeprecated() { return true; }
01304
01305 GTEST_INTERNAL_DEPRECATED (
01306 "TYPED_TEST_CASE is deprecated, please use "
01307 "TYPED_TEST_SUITE")
01308 constexpr bool TypedTestCaseIsDeprecated() { return true; }
01309
01310 GTEST_INTERNAL_DEPRECATED (
01311 "REGISTER_TYPED_TEST_CASE_P is deprecated, please use "
01312 "REGISTER_TYPED_TEST_SUITE_P")
01313 constexpr bool RegisterTypedTestCase_P_IsDeprecated() { return true; }
01314
01315 GTEST_INTERNAL_DEPRECATED (
01316 "INSTITUTE_TYPED_TEST_CASE_P is deprecated, please use "
01317 "INSTITUTE_TYPED_TEST_SUITE_P")
01318 constexpr bool InstantiateTypedTestCase_P_IsDeprecated() { return true; }
01319
01320 } // namespace internal
01321 } // namespace testing
01322
01323 namespace std {
01324 // Some standard library implementations use `struct tuple_size` and some use
01325 // `class tuple_size`. Clang warns about the mismatch.
01326 // https://reviews.llvm.org/D55466
01327 #ifdef __clang__
01328 #pragma clang diagnostic push
01329 #pragma clang diagnostic ignored "-Wmismatched-tags"
01330 #endif
01331 template <typename... Ts>
01332 struct tuple_size<testing::internal::FlatTuple<Ts...>>
01333 : std::integral_constant<size_t, sizeof...(Ts)> {};
01334 #ifdef __clang__
01335 #pragma clang diagnostic pop

```

```

01336 #endif
01337 } // namespace std
01338
01339 #define GTEST_MESSAGE_AT_(file, line, message, result_type) \
01340 ::testing::internal::AssertHelper(result_type, file, line, message) = \
01341 ::testing::Message()
01342
01343 #define GTEST_MESSAGE_(message, result_type) \
01344 GTEST_MESSAGE_AT_(__FILE__, __LINE__, message, result_type)
01345
01346 #define GTEST_FATAL_FAILURE_(message) \
01347 return GTEST_MESSAGE_(message, ::testing::TestPartResult::kFatalFailure)
01348
01349 #define GTEST_NONFATAL_FAILURE_(message) \
01350 GTEST_MESSAGE_(message, ::testing::TestPartResult::kNonFatalFailure)
01351
01352 #define GTEST_SUCCESS_(message) \
01353 GTEST_MESSAGE_(message, ::testing::TestPartResult::kSuccess)
01354
01355 #define GTEST_SKIP_(message) \
01356 return GTEST_MESSAGE_(message, ::testing::TestPartResult::kSkip)
01357
01358 // Suppress MSVC warning 4072 (unreachable code) for the code following
01359 // statement if it returns or throws (or doesn't return or throw in some
01360 // situations).
01361 // NOTE: The "else" is important to keep this expansion to prevent a top-level
01362 // "else" from attaching to our "if".
01363 #define GTEST_SUPPRESS_UNREACHABLE_CODE_WARNING_BELOW_(statement) \
01364 if (::testing::internal::AlwaysTrue()) { \
01365 statement; \
01366 } else \
01367 static_assert(true, "") // User must have a semicolon after expansion.
01368
01369 #if GTEST_HAS_EXCEPTIONS
01370
01371 namespace testing {
01372 namespace internal {
01373
01374 class NeverThrown {
01375 public:
01376 const char* what() const noexcept {
01377 return "this exception should never be thrown";
01378 }
01379 };
01380
01381 } // namespace internal
01382 } // namespace testing
01383
01384 #if GTEST_HAS_RTTI
01385
01386 #define GTEST_EXCEPTION_TYPE_(e) ::testing::internal::GetTypeName(typeid(e))
01387
01388 #else // GTEST_HAS_RTTI
01389
01390 #define GTEST_EXCEPTION_TYPE_(e) \
01391 std::string { "an std::exception-derived error" }
01392
01393 #endif // GTEST_HAS_RTTI
01394
01395 #define GTEST_TEST_THROW_CATCH_STD_EXCEPTION_(statement, expected_exception) \
01396 catch (typename std::conditional< \
01397 std::is_same<typename std::remove_cv<typename std::remove_reference< \
01398 expected_exception>::type>::type, \
01399 std::exception>::value, \
01400 const ::testing::internal::NeverThrown&, const std::exception&>::type \
01401 e) { \
01402 gtest_msg.value = "Expected: " #statement \
01403 " throws an exception of type " #expected_exception \
01404 ".\n Actual: it throws "; \
01405 gtest_msg.value += GTEST_EXCEPTION_TYPE_(e); \
01406 gtest_msg.value += " with description \""; \
01407 gtest_msg.value += e.what(); \
01408 gtest_msg.value += "\"."; \
01409 goto GTEST_CONCAT_TOKEN_(gtest_label_testthrow_, __LINE__); \
01410 } \
01411
01412 #else // GTEST_HAS_EXCEPTIONS
01413
01414 #define GTEST_TEST_THROW_CATCH_STD_EXCEPTION_(statement, expected_exception)
01415
01416 #endif // GTEST_HAS_EXCEPTIONS
01417
01418 #define GTEST_TEST_THROW_(statement, expected_exception, fail) \
01419 GTEST_AMBIGUOUS_ELSE_BLOCKER_ \
01420 if (::testing::internal::TrueWithString gtest_msg{}) { \
01421 bool gtest_caught_expected = false; \
01422 try {

```

```

01423 GTEST_SUPPRESS_UNREACHABLE_CODE_WARNING_BELOW_(statement);
01424 } catch (expected_exception const&) {
01425 gtest_caught_expected = true;
01426 }
01427 GTEST_TEST_THROW_CATCH_STD_EXCEPTION_(statement, expected_exception)
01428 catch (...) {
01429 gtest_msg.value = "Expected: " #statement
01430 " throws an exception of type " #expected_exception
01431 ".\n Actual: it throws a different type.";
01432 goto GTEST_CONCAT_TOKEN_(gtest_label_testthrow_, __LINE__);
01433 }
01434 if (!gtest_caught_expected) {
01435 gtest_msg.value = "Expected: " #statement
01436 " throws an exception of type " #expected_exception
01437 ".\n Actual: it throws nothing.";
01438 goto GTEST_CONCAT_TOKEN_(gtest_label_testthrow_, __LINE__);
01439 }
01440 } else /*NOLINT*/
01441 GTEST_CONCAT_TOKEN_(gtest_label_testthrow_, __LINE__)
01442 : fail(gtest_msg.value.c_str())
01443
01444 #if GTEST_HAS_EXCEPTIONS
01445
01446 #define GTEST_TEST_NO_THROW_CATCH_STD_EXCEPTION_()
01447 catch (std::exception const& e) {
01448 gtest_msg.value = "it throws ";
01449 gtest_msg.value += GTEST_EXCEPTION_TYPE_(e);
01450 gtest_msg.value += " with description \"";
01451 gtest_msg.value += e.what();
01452 gtest_msg.value += "\".";
01453 goto GTEST_CONCAT_TOKEN_(gtest_label_testnothrow_, __LINE__);
01454 }
01455
01456 #else // GTEST_HAS_EXCEPTIONS
01457
01458 #define GTEST_TEST_NO_THROW_CATCH_STD_EXCEPTION_()
01459
01460 #endif // GTEST_HAS_EXCEPTIONS
01461
01462 #define GTEST_TEST_NO_THROW_(statement, fail)
01463 GTEST_AMBIGUOUS_ELSE_BLOCKER_
01464 if (::testing::internal::TrueWithString gtest_msg{}) {
01465 try {
01466 GTEST_SUPPRESS_UNREACHABLE_CODE_WARNING_BELOW_(statement);
01467 }
01468 GTEST_TEST_NO_THROW_CATCH_STD_EXCEPTION_()
01469 catch (...) {
01470 gtest_msg.value = "it throws.";
01471 goto GTEST_CONCAT_TOKEN_(gtest_label_testnothrow_, __LINE__);
01472 }
01473 } else
01474 GTEST_CONCAT_TOKEN_(gtest_label_testnothrow_, __LINE__)
01475 : fail(("Expected: " #statement " doesn't throw an exception.\n"
01476 " Actual: " +
01477 gtest_msg.value)
01478 .c_str())
01479
01480 #define GTEST_TEST_ANY_THROW_(statement, fail)
01481 GTEST_AMBIGUOUS_ELSE_BLOCKER_
01482 if (::testing::internal::AlwaysTrue()) {
01483 bool gtest_caught_any = false;
01484 try {
01485 GTEST_SUPPRESS_UNREACHABLE_CODE_WARNING_BELOW_(statement);
01486 } catch (...) {
01487 gtest_caught_any = true;
01488 }
01489 if (!gtest_caught_any) {
01490 goto GTEST_CONCAT_TOKEN_(gtest_label_testanythrow_, __LINE__);
01491 }
01492 } else
01493 GTEST_CONCAT_TOKEN_(gtest_label_testanythrow_, __LINE__)
01494 : fail("Expected: " #statement
01495 " throws an exception.\n"
01496 " Actual: it doesn't.")
01497
01498 // Implements Boolean test assertions such as EXPECT_TRUE. expression can be
01499 // either a boolean expression or an AssertionResult. text is a textual
01500 // representation of expression as it was passed into the EXPECT_TRUE.
01501 #define GTEST_TEST_BOOLEAN_(expression, text, actual, expected, fail)
01502 GTEST_AMBIGUOUS_ELSE_BLOCKER_
01503 if (const ::testing::AssertionResult gtest_ar_ =
01504 ::testing::AssertionResult(expression))
01505 ;
01506 else
01507 fail(::testing::internal::GetBoolAssertionFailureMessage(
01508 gtest_ar_, text, #actual, #expected)
01509 .c_str())

```

```

01510
01511 #define GTEST_TEST_NO_FATAL_FAILURE_(statement, fail)
01512 GTEST_AMBIGUOUS_ELSE_BLOCKER_
01513 if (::testing::internal::AlwaysTrue()) {
01514 ::testing::internal::HasNewFatalFailureHelper gtest_fatal_failure_checker;
01515 GTEST_SUPPRESS_UNREACHABLE_CODE_WARNING_BELOW_(statement);
01516 if (gtest_fatal_failure_checker.has_new_fatal_failure()) {
01517 goto GTEST_CONCAT_TOKEN_(gtest_label_testnofatal_, __LINE__);
01518 }
01519 } else
01520 GTEST_CONCAT_TOKEN_(gtest_label_testnofatal_, __LINE__)
01521 : fail("Expected: " #statement
01522 " doesn't generate new fatal "
01523 "failures in the current thread.\n"
01524 " Actual: it does.")
01525
01526 // Expands to the name of the class that implements the given test.
01527 #define GTEST_TEST_CLASS_NAME_(test_suite_name, test_name) \
01528 test_suite_name##_##test_name##_Test
01529
01530 // Helper macro for defining tests.
01531 #define GTEST_TEST_(test_suite_name, test_name, parent_class, parent_id)
01532 static_assert(sizeof(GTEST_STRINGIFY_(test_suite_name)) > 1,
01533 "test_suite_name must not be empty");
01534 static_assert(sizeof(GTEST_STRINGIFY_(test_name)) > 1,
01535 "test_name must not be empty");
01536 class GTEST_TEST_CLASS_NAME_(test_suite_name, test_name)
01537 : public parent_class {
01538 public:
01539 GTEST_TEST_CLASS_NAME_(test_suite_name, test_name)() = default;
01540 ~GTEST_TEST_CLASS_NAME_(test_suite_name, test_name)() override = default;
01541 GTEST_TEST_CLASS_NAME_(test_suite_name, test_name)
01542 (const GTEST_TEST_CLASS_NAME_(test_suite_name, test_name) &) = delete;
01543 GTEST_TEST_CLASS_NAME_(test_suite_name, test_name) & operator=(
01544 const GTEST_TEST_CLASS_NAME_(test_suite_name,
01545 test_name) &) = delete; /* NOLINT */
01546 GTEST_TEST_CLASS_NAME_(test_suite_name, test_name)
01547 (GTEST_TEST_CLASS_NAME_(test_suite_name, test_name) &&) noexcept = delete;
01548 GTEST_TEST_CLASS_NAME_(test_suite_name, test_name) & operator=(
01549 GTEST_TEST_CLASS_NAME_(test_suite_name,
01550 test_name) &&) noexcept = delete; /* NOLINT */
01551 private:
01552 void TestBody() override;
01553 static ::testing::TestInfo* const test_info_ GTEST_ATTRIBUTE_UNUSED_;
01554 };
01555
01556 ::testing::TestInfo* const GTEST_TEST_CLASS_NAME_(test_suite_name,
01557 test_name)::test_info_ =
01558 ::testing::internal::MakeAndRegisterTestInfo(
01559 #test_suite_name, #test_name, nullptr, nullptr,
01560 ::testing::internal::CodeLocation(__FILE__, __LINE__), (parent_id),
01561 ::testing::internal::SuiteApiResolver<
01562 parent_class>::GetSetUpCaseOrSuite(__FILE__, __LINE__),
01563 ::testing::internal::SuiteApiResolver<
01564 parent_class>::GetTearDownCaseOrSuite(__FILE__, __LINE__),
01565 new ::testing::internal::TestFactoryImpl<GTEST_TEST_CLASS_NAME_(
01566 test_suite_name, test_name)>());
01567 void GTEST_TEST_CLASS_NAME_(test_suite_name, test_name)::TestBody()
01568
01569 #endif // GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_INTERNAL_H_

```

## 16.250 gtest-param-util.h

```

00001 // Copyright 2008 Google Inc.
00002 // All Rights Reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT

```



```

00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // Type and function utilities for implementing parameterized tests.
00031
00032 // IWYU pragma: private, include "gtest/gtest.h"
00033 // IWYU pragma: friend gtest/*.
00034 // IWYU pragma: friend gmock/*.
00035
00036 #ifndef GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_PARAM_UTIL_H_
00037 #define GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_PARAM_UTIL_H_
00038
00039 #include <ctype.h>
00040
00041 #include <cassert>
00042 #include <iterator>
00043 #include <memory>
00044 #include <set>
00045 #include <tuple>
00046 #include <type_traits>
00047 #include <utility>
00048 #include <vector>
00049
00050 #include "gtest/gtest-printers.h"
00051 #include "gtest/gtest-test-part.h"
00052 #include "gtest/internal/gtest-internal.h"
00053 #include "gtest/internal/gtest-port.h"
00054
00055 namespace testing {
00056 // Input to a parameterized test name generator, describing a test parameter.
00057 // Consists of the parameter value and the integer parameter index.
00058
00059 template <class ParamType>
00060 struct TestParamInfo {
00061
00062 TestParamInfo(const ParamType& a_param, size_t an_index)
00063 : param(a_param), index(an_index) {}
00064 ParamType param;
00065 size_t index;
00066 };
00067
00068 // A builtin parameterized test name generator which returns the result of
00069 // testing::PrintToString.
00070
00071 struct PrintToStringParamName {
00072
00073 template <class ParamType>
00074 std::string operator()(const TestParamInfo<ParamType>& info) const {
00075 return PrintToString(info.param);
00076 }
00077 };
00078
00079 namespace internal {
00080
00081 // INTERNAL IMPLEMENTATION - DO NOT USE IN USER CODE.
00082 // Utility Functions
00083
00084 // Outputs a message explaining invalid registration of different
00085 // fixture class for the same test suite. This may happen when
00086 // TEST_P macro is used to define two tests with the same name
00087 // but in different namespaces.
00088 GTEST_API_ void ReportInvalidTestSuiteType(const char* test_suite_name,
00089 CodeLocation code_location);
00090
00091 template <typename>
00092 class ParamGeneratorInterface;
00093 template <typename>
00094 class ParamGenerator;
00095
00096 // Interface for iterating over elements provided by an implementation
00097 // of ParamGeneratorInterface<T>.
00098
00099 template <typename T>
00100 class ParamIteratorInterface {
00101 public:
00102 virtual ~ParamIteratorInterface() {}
00103
00104 // A pointer to the base generator instance.
00105 // Used only for the purposes of iterator comparison
00106 // to make sure that two iterators belong to the same generator.
00107 };
00108 }

```

```

00109 virtual const ParamGeneratorInterface<T>* BaseGenerator() const = 0;
00110 // Advances iterator to point to the next element
00111 // provided by the generator. The caller is responsible
00112 // for not calling Advance() on an iterator equal to
00113 // BaseGenerator()->End().
00114 virtual void Advance() = 0;
00115 // Clones the iterator object. Used for implementing copy semantics
00116 // of ParamIterator<T>.
00117 virtual ParamIteratorInterface* Clone() const = 0;
00118 // Dereferences the current iterator and provides (read-only) access
00119 // to the pointed value. It is the caller's responsibility not to call
00120 // Current() on an iterator equal to BaseGenerator()->End().
00121 // Used for implementing ParamGenerator<T>::operator*().
00122 virtual const T* Current() const = 0;
00123 // Determines whether the given iterator and other point to the same
00124 // element in the sequence generated by the generator.
00125 // Used for implementing ParamGenerator<T>::operator==(()).
00126 virtual bool Equals(const ParamIteratorInterface& other) const = 0;
00127 };
00128
00129 // Class iterating over elements provided by an implementation of
00130 // ParamGeneratorInterface<T>. It wraps ParamIteratorInterface<T>
00131 // and implements the const forward iterator concept.
00132
00133 template <typename T>
00134 class ParamIterator {
00135 public:
00136 typedef T value_type;
00137 typedef const T& reference;
00138 typedef ptrdiff_t difference_type;
00139
00140 // ParamIterator assumes ownership of the impl_ pointer.
00141
00142 ParamIterator(const ParamIterator& other) : impl_(other.impl_->Clone()) {}
00143
00144 ParamIterator& operator=(const ParamIterator& other) {
00145 if (this != &other) impl_.reset(other.impl_->Clone());
00146 return *this;
00147 }
00148
00149 const T& operator*() const {
00150 return *impl_->Current();
00151 }
00152
00153 const T* operator->() const {
00154 return impl_->Current();
00155 }
00156
00157 // Prefix version of operator++.
00158
00159 ParamIterator& operator++() {
00160 impl_->Advance();
00161 return *this;
00162 }
00163
00164 // Postfix version of operator++.
00165
00166 ParamIterator operator++(int /*unused*/) {
00167 ParamIteratorInterface<T>* clone = impl_->Clone();
00168 impl_->Advance();
00169 return ParamIterator(clone);
00170 }
00171
00172 bool operator==(const ParamIterator& other) const {
00173 return impl_.get() == other.impl_.get() || impl_->Equals(*other.impl_);
00174 }
00175
00176 bool operator!=(const ParamIterator& other) const {
00177 return !(*this == other);
00178 }
00179 private:
00180 friend class ParamGenerator<T>;
00181
00182 explicit ParamIterator(ParamIteratorInterface<T>* impl) : impl_(impl) {}
00183
00184 std::unique_ptr<ParamIteratorInterface<T> impl_;
00185 };
00186
00187 // ParamGeneratorInterface<T> is the binary interface to access generators
00188 // defined in other translation units.
00189
00190 template <typename T>
00191 class ParamGeneratorInterface {
00192 public:
00193 typedef T ParamType;
00194
00195 virtual ~ParamGeneratorInterface() {}

```

```

00196 }
00197
00198 // Generator interface definition
00199 virtual ParamIteratorInterface<T>* Begin() const = 0;
00200 virtual ParamIteratorInterface<T>* End() const = 0;
00201 };
00202
00203 // Wraps ParamGeneratorInterface<T> and provides general generator syntax
00204 // compatible with the STL Container concept.
00205 // This class implements copy initialization semantics and the contained
00206 // ParamGeneratorInterface<T> instance is shared among all copies
00207 // of the original object. This is possible because that instance is immutable.
00208
00209 template <typename T>
00210 class ParamGenerator {
00211 public:
00212 typedef ParamIterator<T> iterator;
00213
00214 explicit ParamGenerator(ParamGeneratorInterface<T>* impl) : impl_(impl) {
00215 }
00216
00217 ParamGenerator(const ParamGenerator& other) : impl_(other.impl_) {
00218 }
00219
00220 ParamGenerator& operator=(const ParamGenerator& other) {
00221 impl_ = other.impl_;
00222 return *this;
00223 }
00224
00225 iterator begin() const {
00226 return iterator(impl_>Begin());
00227 }
00228
00229 iterator end() const {
00230 return iterator(impl_>End());
00231 }
00232
00233 private:
00234 std::shared_ptr<const ParamGeneratorInterface<T> impl_;
00235 };
00236
00237 // Generates values from a range of two comparable values. Can be used to
00238 // generate sequences of user-defined types that implement operator+() and
00239 // operator<().
00240 // This class is used in the Range() function.
00241
00242 template <typename T, typename IncrementT>
00243 class RangeGenerator : public ParamGeneratorInterface<T> {
00244 public:
00245
00246 RangeGenerator(T begin, T end, IncrementT step)
00247 : begin_(begin),
00248 end_(end),
00249 step_(step),
00250 end_index_(CalculateEndIndex(begin, end, step)) {
00251 }
00252
00253 ~RangeGenerator() override {
00254 }
00255
00256 ParamIteratorInterface<T>* Begin() const override {
00257 return new Iterator(this, begin_, 0, step_);
00258 }
00259
00260 ParamIteratorInterface<T>* End() const override {
00261 return new Iterator(this, end_, end_index_, step_);
00262 }
00263
00264 private:
00265
00266 class Iterator : public ParamIteratorInterface<T> {
00267 public:
00268
00269 Iterator(const ParamGeneratorInterface<T>* base, T value, int index,
00270 IncrementT step)
00271 : base_(base), value_(value), index_(index), step_(step) {
00272 }
00273
00274 ~Iterator() override {
00275 }
00276
00277 const ParamGeneratorInterface<T>* BaseGenerator() const override {
00278 return base_;
00279 }
00280
00281 void Advance() override {
00282 value_ = static_cast<T> (value_ + step_);

```

```

00283 index_++;
00284 }
00285
00286 ParamIteratorInterface<T>* Clone() const override {
00287 return new Iterator(*this);
00288 }
00289
00290 const T* Current() const override {
00291 return &value_;
00292 }
00293
00294 bool Equals(const ParamIteratorInterface<T>& other) const override {
00295 // Having the same base generator guarantees that the other
00296 // iterator is of the same type and we can downcast.
00297 GTEST_CHECK_(BaseGenerator() == other.BaseGenerator())
00298 << "The program attempted to compare iterators "
00299 << "from different generators." << std::endl;
00300 const int other_index =
00301 CheckedDowncastToActualType<const Iterator>(&other)->index_;
00302 return index_ == other_index;
00303 }
00304
00305 private:
00306
00307 Iterator(const Iterator& other)
00308 : ParamIteratorInterface<T>(),
00309 base_(other.base_),
00310 value_(other.value_),
00311 index_(other.index_),
00312 step_(other.step_) {
00313 }
00314
00315 // No implementation - assignment is unsupported.
00316 void operator=(const Iterator& other);
00317
00318 const ParamGeneratorInterface<T> * const base_;
00319 T value_;
00320 int index_;
00321 const IncrementT step_;
00322 }; // class RangeGenerator::Iterator
00323
00324 static int CalculateEndIndex(const T& begin, const T& end,
00325 const IncrementT& step) {
00326 int end_index = 0;
00327 for (T i = begin; i < end; i = static_cast<T>(i + step)) end_index++;
00328 return end_index;
00329 }
00330
00331 // No implementation - assignment is unsupported.
00332 void operator=(const RangeGenerator& other);
00333
00334 const T begin_;
00335 const T end_;
00336 const IncrementT step_;
00337 // The index for the end() iterator. All the elements in the generated
00338 // sequence are indexed (0-based) to aid iterator comparison.
00339 const int end_index_;
00340 }; // class RangeGenerator
00341
00342 // Generates values from a pair of STL-style iterators. Used in the
00343 // ValuesIn() function. The elements are copied from the source range
00344 // since the source can be located on the stack, and the generator
00345 // is likely to persist beyond that stack frame.
00346
00347 template <typename T>
00348 class ValuesInIteratorRangeGenerator : public ParamGeneratorInterface<T> {
00349 public:
00350
00351 template <typename ForwardIterator>
00352 ValuesInIteratorRangeGenerator(ForwardIterator begin, ForwardIterator end)
00353 : container_(begin, end) {
00354 }
00355
00356 ~ValuesInIteratorRangeGenerator() override {
00357 }
00358
00359 ParamIteratorInterface<T>* Begin() const override {
00360 return new Iterator(this, container_.begin());
00361 }
00362
00363 ParamIteratorInterface<T>* End() const override {
00364 return new Iterator(this, container_.end());
00365 }
00366
00367 private:
00368 typedef typename T::std::vector<T> ContainerType;
00369

```

```

00370 class Iterator : public ParamIteratorInterface<T> {
00371 public:
00372
00373 Iterator(const ParamGeneratorInterface<T>* base,
00374 typename ContainerType::const_iterator iterator)
00375 : base_(base), iterator_(iterator) {}
00376
00377
00378 ~Iterator() override {}
00379
00380
00381 const ParamGeneratorInterface<T>* BaseGenerator() const override {
00382 return base_;
00383 }
00384
00385 void Advance() override {
00386 ++iterator_;
00387 value_.reset();
00388 }
00389
00390 ParamIteratorInterface<T>* Clone() const override {
00391 return new Iterator(*this);
00392 }
00393 // We need to use cached value referenced by iterator_ because *iterator_
00394 // can return a temporary object (and of type other than T), so just
00395 // having "return &*iterator_;" doesn't work.
00396 // value_ is updated here and not in Advance() because Advance()
00397 // can advance iterator_ beyond the end of the range, and we cannot
00398 // detect that fact. The client code, on the other hand, is
00399 // responsible for not calling Current() on an out-of-range iterator.
00400
00401 const T* Current() const override {
00402 if (value_.get() == nullptr) value_.reset(new T(*iterator_));
00403 return value_.get();
00404 }
00405
00406 bool Equals(const ParamIteratorInterface<T>& other) const override {
00407 // Having the same base generator guarantees that the other
00408 // iterator is of the same type and we can downcast.
00409 GTEST_CHECK_(BaseGenerator() == other.BaseGenerator())
00410 << "The program attempted to compare iterators "
00411 << "from different generators." << std::endl;
00412 return iterator_ ==
00413 CheckedDowncastToActualType<const Iterator>(&other)->iterator_;
00414 }
00415
00416 private:
00417
00418 Iterator(const Iterator& other)
00419 // The explicit constructor call suppresses a false warning
00420 // emitted by gcc when supplied with the -Wextra option.
00421 : ParamIteratorInterface<T>(),
00422 base_(other.base_),
00423 iterator_(other.iterator_) {}
00424
00425 const ParamGeneratorInterface<T>* const base_;
00426 typename ContainerType::const_iterator iterator_;
00427 // A cached value of *iterator_. We keep it here to allow access by
00428 // pointer in the wrapping iterator's operator->().
00429 // value_ needs to be mutable to be accessed in Current().
00430 // Use of std::unique_ptr helps manage cached value's lifetime,
00431 // which is bound by the lifespan of the iterator itself.
00432 mutable std::unique_ptr<const T> value_;
00433 }; // class ValuesInIteratorRangeGenerator::Iterator
00434
00435 // No implementation - assignment is unsupported.
00436 void operator=(const ValuesInIteratorRangeGenerator& other);
00437
00438 const ContainerType container_;
00439 }; // class ValuesInIteratorRangeGenerator
00440
00441 // INTERNAL IMPLEMENTATION - DO NOT USE IN USER CODE.
00442 //
00443 // Default parameterized test name generator, returns a string containing the
00444 // integer test parameter index.
00445
00446 template <class ParamType>
00447 std::string DefaultParamName(const TestParamInfo<ParamType>& info) {
00448 Message name_stream;
00449 name_stream << info.index;
00450 return name_stream.GetString();
00451 }
00452
00453 template <typename T = int>
00454 void TestNotEmpty() {
00455 static_assert(sizeof(T) == 0, "Empty arguments are not allowed.");
00456

```

```

00457 }
00458
00459 template <typename T = int>
00460 void TestNotEmpty(const T&) {
00461 }
00462
00463 // INTERNAL IMPLEMENTATION - DO NOT USE IN USER CODE.
00464 //
00465 // Stores a parameter value and later creates tests parameterized with that
00466 // value.
00467
00468 template <class TestClass>
00469 class ParameterizedTestFactory : public TestFactoryBase {
00470 public:
00471 typedef typename TestClass::ParamType ParamType;
00472
00473 explicit ParameterizedTestFactory(ParamType parameter)
00474 : parameter_(parameter) {
00475 }
00476
00477 Test* CreateTest() override {
00478 TestClass::SetParam(¶meter_);
00479 return new TestClass();
00480 }
00481
00482 private:
00483 const ParamType parameter_;
00484
00485 ParameterizedTestFactory(const ParameterizedTestFactory&) = delete;
00486 ParameterizedTestFactory& operator=(const ParameterizedTestFactory&) = delete;
00487 };
00488
00489 // INTERNAL IMPLEMENTATION - DO NOT USE IN USER CODE.
00490 //
00491 // TestMetaFactoryBase is a base class for meta-factories that create
00492 // test factories for passing into MakeAndRegisterTestInfo function.
00493
00494 template <class ParamType>
00495 class TestMetaFactoryBase {
00496 public:
00497 virtual ~TestMetaFactoryBase() {
00498 }
00499
00500 virtual TestFactoryBase* CreateTestFactory(ParamType parameter) = 0;
00501 };
00502
00503 // INTERNAL IMPLEMENTATION - DO NOT USE IN USER CODE.
00504 //
00505 // TestMetaFactory creates test factories for passing into
00506 // MakeAndRegisterTestInfo function. Since MakeAndRegisterTestInfo receives
00507 // ownership of test factory pointer, same factory object cannot be passed
00508 // into that method twice. But ParameterizedTestSuiteInfo is going to call
00509 // it for each Test/Parameter value combination. Thus it needs meta factory
00510 // creator class.
00511
00512 template <class TestSuite>
00513 class TestMetaFactory
00514 : public TestMetaFactoryBase<typename TestSuite::ParamType> {
00515 public:
00516 using ParamType = typename TestSuite::ParamType;
00517
00518 TestMetaFactory() {
00519 }
00520
00521 TestFactoryBase* CreateTestFactory(ParamType parameter) override {
00522 return new ParameterizedTestFactory<TestSuite>(parameter);
00523 }
00524
00525 private:
00526 TestMetaFactory(const TestMetaFactory&) = delete;
00527 TestMetaFactory& operator=(const TestMetaFactory&) = delete;
00528 };
00529
00530 // INTERNAL IMPLEMENTATION - DO NOT USE IN USER CODE.
00531 //
00532 // ParameterizedTestSuiteInfoBase is a generic interface
00533 // to ParameterizedTestSuiteInfo classes. ParameterizedTestSuiteInfoBase
00534 // accumulates test information provided by TEST_P macro invocations
00535 // and generators provided by INSTANTIATE_TEST_SUITE_P macro invocations
00536 // and uses that information to register all resulting test instances
00537 // in RegisterTests method. The ParameterizeTestSuiteRegistry class holds
00538 // a collection of pointers to the ParameterizedTestSuiteInfo objects
00539 // and calls RegisterTests() on each of them when asked.
00540
00541 class ParameterizedTestSuiteInfoBase {
00542 public:

```

```

00544
00545 virtual ~ParameterizedTestSuiteInfoBase() {
00546 }
00547
00548 // Base part of test suite name for display purposes.
00549 virtual const std::string& GetTestSuiteName() const = 0;
00550 // Test suite id to verify identity.
00551 virtual TypeId GetTestSuiteTypeId() const = 0;
00552 // UnitTest class invokes this method to register tests in this
00553 // test suite right before running them in RUN_ALL_TESTS macro.
00554 // This method should not be called more than once on any single
00555 // instance of a ParameterizedTestSuiteInfoBase derived class.
00556 virtual void RegisterTests() = 0;
00557
00558 protected:
00559
00560 ParameterizedTestSuiteInfoBase() {
00561 }
00562
00563 private:
00564 ParameterizedTestSuiteInfoBase(const ParameterizedTestSuiteInfoBase&) =
00565 delete;
00566 ParameterizedTestSuiteInfoBase& operator=(
00567 const ParameterizedTestSuiteInfoBase&) = delete;
00568 };
00569
00570 // INTERNAL IMPLEMENTATION - DO NOT USE IN USER CODE.
00571 //
00572 // Report a the name of a test_suite as safe to ignore
00573 // as the side effect of construction of this type.
00574
00575 struct GTEST_API_ MarkAsIgnored {
00576 explicit MarkAsIgnored(const char* test_suite);
00577 };
00578
00579 GTEST_API_ void InsertSyntheticTestCase(const std::string& name,
00580 CodeLocation location, bool has_test_p);
00581
00582 // INTERNAL IMPLEMENTATION - DO NOT USE IN USER CODE.
00583 //
00584 // ParameterizedTestSuiteInfo accumulates tests obtained from TEST_P
00585 // macro invocations for a particular test suite and generators
00586 // obtained from INSTANTIATE_TEST_SUITE_P macro invocations for that
00587 // test suite. It registers tests with all values generated by all
00588 // generators when asked.
00589
00590 template <class TestSuite>
00591 class ParameterizedTestSuiteInfo : public ParameterizedTestSuiteInfoBase {
00592 public:
00593 // ParamType and GeneratorCreationFunc are private types but are required
00594 // for declarations of public methods AddTestPattern() and
00595 // AddTestSuiteInstantiation().
00596 using ParamType = typename TestSuite::ParamType;
00597 // A function that returns an instance of appropriate generator type.
00598 typedef ParamGenerator<ParamType>(GeneratorCreationFunc) ();
00599 using ParamNameGeneratorFunc = std::string(const TestParamInfo<ParamType>&);
00600
00601 explicit ParameterizedTestSuiteInfo(const char* name,
00602 CodeLocation code_location)
00603 : test_suite_name_(name), code_location_(code_location) {
00604 }
00605
00606 // Test suite base name for display purposes.
00607
00608 const std::string& GetTestSuiteName() const override {
00609 return test_suite_name_;
00610 }
00611 // Test suite id to verify identity.
00612
00613 TypeId GetTestSuiteTypeId() const override {
00614 return GetTypeId<TestSuite>();
00615 }
00616 // TEST_P macro uses AddTestPattern() to record information
00617 // about a single test in a LocalTestInfo structure.
00618 // test_suite_name is the base name of the test suite (without invocation
00619 // prefix). test_base_name is the name of an individual test without
00620 // parameter index. For the test SequenceA/FooTest.DoBar/1 FooTest is
00621 // test suite base name and DoBar is test base name.
00622
00623 void AddTestPattern(const char* test_suite_name, const char* test_base_name,
00624 TestMetaFactoryBase<ParamType>* meta_factory,
00625 CodeLocation code_location) {
00626 tests_.push_back(std::shared_ptr<TestInfo>(new TestInfo(
00627 test_suite_name, test_base_name, meta_factory, code_location)));
00628 }
00629 // INSTANTIATE_TEST_SUITE_P macro uses AddGenerator() to record information
00630 // about a generator.

```

```

00631
00632 int AddTestSuiteInstantiation(const std::string& instantiation_name,
00633 GeneratorCreationFunc* func,
00634 ParamNameGeneratorFunc* name_func,
00635 const char* file, int line) {
00636 instantiations_.push_back(
00637 InstantiationInfo(instantiation_name, func, name_func, file, line));
00638 return 0; // Return value used only to run this method in namespace scope.
00639 }
00640 // UnitTest class invokes this method to register tests in this test suite
00641 // right before running tests in RUN_ALL_TESTS macro.
00642 // This method should not be called more than once on any single
00643 // instance of a ParameterizedTestSuiteInfoBase derived class.
00644 // UnitTest has a guard to prevent from calling this method more than once.
00645
00646 void RegisterTests() override {
00647 bool generated_instantiations = false;
00648
00649 for (typename TestInfoContainer::iterator test_it = tests_.begin();
00650 test_it != tests_.end(); ++test_it) {
00651 std::shared_ptr<TestInfo> test_info = *test_it;
00652 for (typename InstantiationContainer::iterator gen_it =
00653 instantiations_.begin();
00654 gen_it != instantiations_.end(); ++gen_it) {
00655 const std::string& instantiation_name = gen_it->name;
00656 ParamGenerator<ParamType> generator((*gen_it->generator)());
00657 ParamNameGeneratorFunc* name_func = gen_it->name_func;
00658 const char* file = gen_it->file;
00659 int line = gen_it->line;
00660
00661 std::string test_suite_name;
00662 if (!instantiation_name.empty())
00663 test_suite_name = instantiation_name + "/";
00664 test_suite_name += test_info->test_suite_base_name;
00665
00666 size_t i = 0;
00667 std::set<std::string> test_param_names;
00668 for (typename ParamGenerator<ParamType>::iterator param_it =
00669 generator.begin();
00670 param_it != generator.end(); ++param_it, ++i) {
00671 generated_instantiations = true;
00672
00673 Message test_name_stream;
00674
00675 std::string param_name =
00676 name_func(TestParamInfo<ParamType>(*param_it, i));
00677
00678 GTEST_CHECK_(IsValidParamName(param_name))
00679 << "Parameterized test name '" << param_name
00680 << "' is invalid, in " << file << " line " << line << std::endl;
00681
00682 GTEST_CHECK_(test_param_names.count(param_name) == 0)
00683 << "Duplicate parameterized test name '" << param_name << "', in "
00684 << file << " line " << line << std::endl;
00685
00686 test_param_names.insert(param_name);
00687
00688 if (!test_info->test_base_name.empty()) {
00689 test_name_stream << test_info->test_base_name << "/";
00690 }
00691 test_name_stream << param_name;
00692 MakeAndRegisterTestInfo(
00693 test_suite_name.c_str(), test_name_stream.GetString().c_str(),
00694 nullptr, // No type parameter.
00695 PrintToString(*param_it).c_str(), test_info->code_location,
00696 GetTestSuiteTypeId(),
00697 SuiteApiResolver<TestSuite>::GetSetUpCaseOrSuite(file, line),
00698 SuiteApiResolver<TestSuite>::GetTearDownCaseOrSuite(file, line),
00699 test_info->test_meta_factory->CreateTestFactory(*param_it));
00700 } // for param_it
00701 } // for gen_it
00702 } // for test_it
00703
00704 if (!generated_instantiations) {
00705 // There are no generators, or they all generate nothing ...
00706 InsertSyntheticTestCase(GetTestSuiteName(), code_location,
00707 !tests_.empty());
00708 }
00709 } // RegisterTests
00710
00711 private:
00712 // LocalTestInfo structure keeps information about a single test registered
00713 // with TEST_P macro.
00714
00715 struct TestInfo {
00716
00717 TestInfo(const char* a_test_suite_base_name, const char* a_test_base_name,

```



```

00718 TestMetaFactoryBase<ParamType>* a_test_meta_factory,
00719 CodeLocation a_code_location)
00720 : test_suite_base_name(a_test_suite_base_name),
00721 test_base_name(a_test_base_name),
00722 test_meta_factory(a_test_meta_factory),
00723 code_location(a_code_location) {
00724 }
00725
00726 const std::string test_suite_base_name;
00727 const std::string test_base_name;
00728 const std::unique_ptr<TestMetaFactoryBase<ParamType>> test_meta_factory;
00729 const CodeLocation code_location;
00730 };
00731 using TestInfoContainer = ::std::vector<std::shared_ptr<TestInfo>>;
00732 // Records data received from INSTANTIATE_TEST_SUITE_P macros:
00733 // <Instantiation name, Sequence generator creation function,
00734 // Name generator function, Source file, Source line>
00735
00736 struct InstantiationInfo {
00737
00738 InstantiationInfo(const std::string& name_in,
00739 GeneratorCreationFunc* generator_in,
00740 ParamNameGeneratorFunc* name_func_in, const char* file_in,
00741 int line_in)
00742 : name(name_in),
00743 generator(generator_in),
00744 name_func(name_func_in),
00745 file(file_in),
00746 line(line_in) {
00747 }
00748
00749 std::string name;
00750 GeneratorCreationFunc* generator;
00751 ParamNameGeneratorFunc* name_func;
00752 const char* file;
00753 int line;
00754 };
00755 typedef ::std::vector<InstantiationInfo> InstantiationContainer;
00756
00757 static bool IsValidParamName(const std::string& name) {
00758 // Check for empty string
00759 if (name.empty()) return false;
00760
00761 // Check for invalid characters
00762 for (std::string::size_type index = 0; index < name.size(); ++index) {
00763 if (!IsAlNum(name[index]) && name[index] != '_') return false;
00764 }
00765
00766 return true;
00767 }
00768
00769 const std::string test_suite_name_;
00770 CodeLocation code_location_;
00771 TestInfoContainer tests_;
00772 InstantiationContainer instantiations_;
00773
00774 ParameterizedTestSuiteInfo(const ParameterizedTestSuiteInfo&) = delete;
00775 ParameterizedTestSuiteInfo& operator=(const ParameterizedTestSuiteInfo&) =
00776 delete;
00777 }; // class ParameterizedTestSuiteInfo
00778
00779 // Legacy API is deprecated but still available
00780 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00781 template <class TestCase>
00782 using ParameterizedTestCaseInfo = ParameterizedTestSuiteInfo<TestCase>;
00783 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00784
00785 // INTERNAL IMPLEMENTATION - DO NOT USE IN USER CODE.
00786 //
00787 // ParameterizedTestSuiteRegistry contains a map of
00788 // ParameterizedTestSuiteInfoBase classes accessed by test suite names. TEST_P
00789 // and INSTANTIATE_TEST_SUITE_P macros use it to locate their corresponding
00790 // ParameterizedTestSuiteInfo descriptors.
00791
00792 class ParameterizedTestSuiteRegistry {
00793 public:
00794
00795 ParameterizedTestSuiteRegistry() {
00796 }
00797
00798 ~ParameterizedTestSuiteRegistry() {
00799 for (auto& test_suite_info : test_suite_infos_) {
00800 delete test_suite_info;
00801 }
00802 }
00803
00804 // Looks up or creates and returns a structure containing information about

```

```

00805 // tests and instantiations of a particular test suite.
00806
00807 template <class TestSuite>
00808 ParameterizedTestSuiteInfo<TestSuite>* GetTestSuitePatternHolder(
00809 const char* test_suite_name, CodeLocation code_location) {
00810 ParameterizedTestSuiteInfo<TestSuite>* typed_test_info = nullptr;
00811 for (auto& test_suite_info : test_suite_infos_) {
00812 if (test_suite_info->GetTestSuiteName() == test_suite_name) {
00813 if (test_suite_info->GetTestSuiteTypeId() != GetTypeId<TestSuite>()) {
00814 // Complain about incorrect usage of Google Test facilities
00815 // and terminate the program since we cannot guaranty correct
00816 // test suite setup and tear-down in this case.
00817 ReportInvalidTestSuiteType(test_suite_name, code_location);
00818 posix::Abort();
00819 } else {
00820 // At this point we are sure that the object we found is of the same
00821 // type we are looking for, so we downcast it to that type
00822 // without further checks.
00823 typed_test_info = CheckedDowncastToActualType<
00824 ParameterizedTestSuiteInfo < TestSuite > (test_suite_info);
00825 }
00826 break;
00827 }
00828 }
00829 if (typed_test_info == nullptr) {
00830 typed_test_info = new ParameterizedTestSuiteInfo<TestSuite>(
00831 test_suite_name, code_location);
00832 test_suite_infos_.push_back(typed_test_info);
00833 }
00834 return typed_test_info;
00835 }
00836
00837 void RegisterTests() {
00838 for (auto& test_suite_info : test_suite_infos_) {
00839 test_suite_info->RegisterTests();
00840 }
00841 }
00842 // Legacy API is deprecated but still available
00843 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00844
00845 template <class TestCase>
00846 ParameterizedTestCaseInfo<TestCase>* GetTestCasePatternHolder(
00847 const char* test_case_name, CodeLocation code_location) {
00848 return GetTestSuitePatternHolder<TestCase>(test_case_name, code_location);
00849 }
00850
00851 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00852
00853 private:
00854 using TestSuiteInfoContainer = ::std::vector<ParameterizedTestSuiteInfoBase*>;
00855
00856 TestSuiteInfoContainer test_suite_infos_;
00857
00858 ParameterizedTestSuiteRegistry(const ParameterizedTestSuiteRegistry&) =
00859 delete;
00860 ParameterizedTestSuiteRegistry& operator=(
00861 const ParameterizedTestSuiteRegistry&) = delete;
00862 };
00863
00864 // Keep track of what type-parameterized test suite are defined and
00865 // where as well as which are intatiated. This allows susequently
00866 // identifying suits that are defined but never used.
00867
00868 class TypeParameterizedTestSuiteRegistry {
00869 public:
00870 // Add a suite definition
00871 void RegisterTestSuite(const char* test_suite_name,
00872 CodeLocation code_location);
00873
00874 // Add an instantiation of a suit.
00875 void RegisterInstantiation(const char* test_suite_name);
00876
00877 // For each suit repored as defined but not reported as instantiation,
00878 // emit a test that reports that fact (configurably, as an error).
00879 void CheckForInstantiations();
00880
00881 private:
00882 struct TypeParameterizedTestSuiteInfo {
00883
00884 explicit TypeParameterizedTestSuiteInfo(CodeLocation c)
00885 : code_location(c), instantiated(false) {
00886 }
00887
00888 CodeLocation code_location;
00889 bool instantiated;
00890 };
00891 };

```

```

00892
00893 std::map<std::string, TypeParameterizedTestSuiteInfo> suites_;
00894 };
00895
00896 } // namespace internal
00897
00898 // Forward declarations of ValuesIn(), which is implemented in
00899 // include/gtest/gtest-param-test.h.
00900 template <class Container>
00901 internal::ParamGenerator<typename Container::value_type> ValuesIn(
00902 const Container& container);
00903
00904 namespace internal {
00905 // Used in the Values() function to provide polymorphic capabilities.
00906
00907 #ifdef _MSC_VER
00908 #pragma warning(push)
00909 #pragma warning(disable : 4100)
00910 #endif
00911
00912 template <typename... Ts>
00913 class ValueArray {
00914 public:
00915
00916 explicit ValueArray(Ts... v) : v_(FlatTupleConstructTag{}, std::move(v)...) {
00917 }
00918
00919 template <typename T>
00920 operator ParamGenerator<T>() const { // NOLINT
00921 return ValuesIn(MakeVector<T>(MakeIndexSequence<sizeof...(Ts)>()));
00922 }
00923
00924 private:
00925
00926 template <typename T, size_t... I>
00927 std::vector<T> MakeVector(IndexSequence<I...>) const {
00928 return std::vector<T>{static_cast<T>(v_.template Get<I>())...};
00929 }
00930
00931 FlatTuple<Ts...> v_;
00932 };
00933
00934 #ifdef _MSC_VER
00935 #pragma warning(pop)
00936 #endif
00937
00938 template <typename... T>
00939 class CartesianProductGenerator
00940 : public ParamGeneratorInterface<::std::tuple<T...>>
00941 {
00942 public:
00943 typedef ::std::tuple<T...> ParamType;
00944
00945 CartesianProductGenerator(const std::tuple<ParamGenerator<T>...>& g)
00946 : generators_(g) {
00947 }
00948
00949 ~CartesianProductGenerator() override {
00950 }
00951
00952 ParamIteratorInterface<ParamType>* Begin() const override {
00953 return new Iterator(this, generators_, false);
00954 }
00955
00956 ParamIteratorInterface<ParamType>* End() const override {
00957 return new Iterator(this, generators_, true);
00958 }
00959
00960 private:
00961 template <class I>
00962 class IteratorImpl;
00963 template <size_t... I>
00964 class IteratorImpl<IndexSequence<I...>>
00965 : public ParamIteratorInterface<ParamType>
00966 {
00967 public:
00968
00969 IteratorImpl(const ParamGeneratorInterface<ParamType>* base,
00970 const std::tuple<ParamGenerator<T>...>& generators,
00971 bool is_end)
00972 : base_(base),
00973 begin_(std::get<I>(generators).begin()...),
00974 end_(std::get<I>(generators).end()...),
00975 current_(is_end ? end_ : begin_) {
00976 ComputeCurrentValue();
00977 }
00978

```

```

00979 ~IteratorImpl() override {
00980 }
00981
00982 const ParamGeneratorInterface<ParamType>* BaseGenerator() const override {
00983 return base_;
00984 }
00985 // Advance should not be called on beyond-of-range iterators
00986 // so no component iterators must be beyond end of range, either.
00987
00988 void Advance() override {
00989 assert(!AtEnd());
00990 // Advance the last iterator.
00991 ++std::get<sizeof...(T) - 1>(current_);
00992 // if that reaches end, propagate that up.
00993 AdvanceIfEnd<sizeof...(T) - 1>();
00994 ComputeCurrentValue();
00995 }
00996
00997 ParamIteratorInterface<ParamType>* Clone() const override {
00998 return new IteratorImpl(*this);
00999 }
01000
01001 const ParamType * Current() const override {
01002 return current_value_.get();
01003 }
01004
01005 bool Equals(const ParamIteratorInterface<ParamType>& other) const override {
01006 // Having the same base generator guarantees that the other
01007 // iterator is of the same type and we can downcast.
01008 GTEST_CHECK_(BaseGenerator() == other.BaseGenerator())
01009 << "The program attempted to compare iterators "
01010 << "from different generators." << std::endl;
01011 const IteratorImpl* typed_other =
01012 CheckedDowncastToActualType<const IteratorImpl>(&other);
01013
01014 // We must report iterators equal if they both point beyond their
01015 // respective ranges. That can happen in a variety of fashions,
01016 // so we have to consult AtEnd().
01017 if (AtEnd() && typed_other->AtEnd()) return true;
01018
01019 bool same = true;
01020 bool dummy[] = {
01021 (same = same && std::get<I>(current_) ==
01022 std::get<I>(typed_other->current_))...
01023 };
01024 (void) dummy;
01025 return same;
01026 }
01027
01028 private:
01029
01030 template <size_t ThisI>
01031 void AdvanceIfEnd() {
01032 if (std::get<ThisI>(current_) != std::get<ThisI>(end_)) return;
01033
01034 bool last = ThisI == 0;
01035 if (last) {
01036 // We are done. Nothing else to propagate.
01037 return;
01038 }
01039
01040 constexpr size_t NextI = ThisI - (ThisI != 0);
01041 std::get<ThisI>(current_) = std::get<ThisI>(begin_);
01042 ++std::get<NextI>(current_);
01043 AdvanceIfEnd<NextI>();
01044 }
01045
01046 void ComputeCurrentValue() {
01047 if (!AtEnd())
01048 current_value_ = std::make_shared<ParamType>(*std::get<I>(current_)...);
01049 }
01050
01051 bool AtEnd() const {
01052 bool at_end = false;
01053 bool dummy[] = {
01054 (at_end = at_end || std::get<I>(current_) == std::get<I>(end_))...
01055 };
01056 (void) dummy;
01057 return at_end;
01058 }
01059
01060 const ParamGeneratorInterface<ParamType> * const base_;
01061 std::tuple<typename ParamGenerator<T>::iterator...> begin_;
01062 std::tuple<typename ParamGenerator<T>::iterator...> end_;
01063 std::tuple<typename ParamGenerator<T>::iterator...> current_;
01064 std::shared_ptr<ParamType> current_value_;
01065 };

```

```

01066
01067 using Iterator = IteratorImpl<typename MakeIndexSequence<sizeof...(T)>::type>;
01068
01069 std::tuple<ParamGenerator<T>...> generators_;
01070 };
01071
01072 template <class... Gen>
01073 class CartesianProductHolder {
01074 public:
01075
01076 CartesianProductHolder(const Gen&... g) : generators_(g...) {
01077 }
01078
01079 template <typename... T>
01080 operator ParamGenerator<::std::tuple<T...>() const {
01081 return ParamGenerator<::std::tuple<T...>(>
01082 new CartesianProductGenerator<T...>(generators_));
01083 }
01084
01085 private:
01086 std::tuple<Gen...> generators_;
01087 };
01088
01089 } // namespace internal
01090 } // namespace testing
01091
01092 #endif // GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_PARAM_UTIL_H_

```

## 16.251 gtest-port-arch.h

```

00001 // Copyright 2015, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // The Google C++ Testing and Mocking Framework (Google Test)
00031 //
00032 // This header file defines the GTEST_OS_* macro.
00033 // It is separate from gtest-port.h so that custom/gtest-port.h can include it.
00034
00035 #ifndef GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_PORT_ARCH_H_
00036 #define GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_PORT_ARCH_H_
00037
00038 // Determines the platform on which Google Test is compiled.
00039 #ifdef __CYGWIN__
00040 #define GTEST_OS_CYGWIN 1
00041 #elif defined(__MINGW__) || defined(__MINGW32__) || defined(__MINGW64__)
00042 #define GTEST_OS_WINDOWS_MINGW 1
00043 #define GTEST_OS_WINDOWS 1
00044 #elif defined __WIN32
00045 #define GTEST_OS_WINDOWS 1
00046 #elif defined _WIN32_WCE
00047 #define GTEST_OS_WINDOWS_MOBILE 1
00048 #elif defined(WINAPI_FAMILY)
00049 #include <winapifamily.h>
00050 #if WINAPI_FAMILY_PARTITION(WINAPI_PARTITION_DESKTOP)
00051 #define GTEST_OS_WINDOWS_DESKTOP 1
00052 #elif WINAPI_FAMILY_PARTITION(WINAPI_PARTITION_PHONE_APP)
00053 #define GTEST_OS_WINDOWS_PHONE 1
00054 #elif WINAPI_FAMILY_PARTITION(WINAPI_PARTITION_APP)
00055 #define GTEST_OS_WINDOWS_RT 1

```

```

00056 #elif WINAPI_FAMILY_PARTITION(WINAPI_PARTITION_TV_TITLE)
00057 #define GTEST_OS_WINDOWS_PHONE 1
00058 #define GTEST_OS_WINDOWS_TV_TITLE 1
00059 #else
00060 // WINAPI_FAMILY defined but no known partition matched.
00061 // Default to desktop.
00062 #define GTEST_OS_WINDOWS_DESKTOP 1
00063 #endif
00064 #else
00065 #define GTEST_OS_WINDOWS_DESKTOP 1
00066 #endif // _WIN32_WCE
00067 #elif defined __OS2__
00068 #define GTEST_OS_OS2 1
00069 #elif defined __APPLE__
00070 #define GTEST_OS_MAC 1
00071 #include <TargetConditionals.h>
00072 #if TARGET_OS_IPHONE
00073 #define GTEST_OS_IOS 1
00074 #endif
00075 #elif defined __DragonFly__
00076 #define GTEST_OS_DRAGONFLY 1
00077 #elif defined __FreeBSD__
00078 #define GTEST_OS_FREEBSD 1
00079 #elif defined __Fuchsia__
00080 #define GTEST_OS_FUCHSIA 1
00081 #elif defined(__GNU__)
00082 #define GTEST_OS_GNU_HURD 1
00083 #elif defined(__GLIBC__) && defined(__FreeBSD_kernel__)
00084 #define GTEST_OS_GNU_KFREEBSD 1
00085 #elif defined __linux__
00086 #define GTEST_OS_LINUX 1
00087 #if defined __ANDROID__
00088 #define GTEST_OS_LINUX_ANDROID 1
00089 #endif
00090 #elif defined __MVS__
00091 #define GTEST_OS_ZOS 1
00092 #elif defined(__sun) && defined(__SVR4)
00093 #define GTEST_OS_SOLARIS 1
00094 #elif defined(_AIX)
00095 #define GTEST_OS_AIX 1
00096 #elif defined(__hpux)
00097 #define GTEST_OS_HPUX 1
00098 #elif defined __native_client__
00099 #define GTEST_OS_NACL 1
00100 #elif defined __NetBSD__
00101 #define GTEST_OS_NETBSD 1
00102 #elif defined __OpenBSD__
00103 #define GTEST_OS_OPENBSD 1
00104 #elif defined __QNX__
00105 #define GTEST_OS_QNX 1
00106 #elif defined(__HAIKU__)
00107 #define GTEST_OS_HAIKU 1
00108 #elif defined ESP8266
00109 #define GTEST_OS_ESP8266 1
00110 #elif defined ESP32
00111 #define GTEST_OS_ESP32 1
00112 #elif defined(__XTENSA__)
00113 #define GTEST_OS_XTENSA 1
00114 #endif // __CYGWIN__
00115
00116 #endif // GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_PORT_ARCH_H_

```

## 16.252 gtest-string.h

```

00001 // Copyright 2005, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT

```

```

00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // The Google C++ Testing and Mocking Framework (Google Test)
00031 //
00032 // This header file declares the String class and functions used internally by
00033 // Google Test. They are subject to change without notice. They should not used
00034 // by code external to Google Test.
00035 //
00036 // This header file is #included by gtest-internal.h.
00037 // It should not be #included by other files.
00038
00039 // IWYU pragma: private, include "gtest/gtest.h"
00040 // IWYU pragma: friend gtest/*.h
00041 // IWYU pragma: friend gmock/*.h
00042
00043 #ifndef GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_STRING_H_
00044 #define GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_STRING_H_
00045
00046 #ifdef __BORLANDC__
00047 // string.h is not guaranteed to provide strcpy on C++ Builder.
00048 #include <mem.h>
00049 #endif
00050
00051 #include <string.h>
00052
00053 #include <stdint>
00054 #include <string>
00055
00056 #include "gtest/internal/gtest-port.h"
00057
00058 namespace testing {
00059 namespace internal {
00060
00061 // String - an abstract class holding static string utilities.
00062
00063 class GTEST_API_ String {
00064 public:
00065 // Static utility methods
00066
00067 // Clones a 0-terminated C string, allocating memory using new. The
00068 // caller is responsible for deleting the return value using
00069 // delete[]. Returns the cloned string, or NULL if the input is
00070 // NULL.
00071 //
00072 // This is different from strdup() in string.h, which allocates
00073 // memory using malloc().
00074 static const char* CloneCString(const char* c_str);
00075
00076 #if GTEST_OS_WINDOWS_MOBILE
00077 // Windows CE does not have the 'ANSI' versions of Win32 APIs. To be
00078 // able to pass strings to Win32 APIs on CE we need to convert them
00079 // to 'Unicode', UTF-16.
00080
00081 // Creates a UTF-16 wide string from the given ANSI string, allocating
00082 // memory using new. The caller is responsible for deleting the return
00083 // value using delete[]. Returns the wide string, or NULL if the
00084 // input is NULL.
00085 //
00086 // The wide string is created using the ANSI codepage (CP_ACP) to
00087 // match the behaviour of the ANSI versions of Win32 calls and the
00088 // C runtime.
00089 static LPCWSTR AnsiToUtf16(const char* c_str);
00090
00091 // Creates an ANSI string from the given wide string, allocating
00092 // memory using new. The caller is responsible for deleting the return
00093 // value using delete[]. Returns the ANSI string, or NULL if the
00094 // input is NULL.
00095 //
00096 // The returned string is created using the ANSI codepage (CP_ACP) to
00097 // match the behaviour of the ANSI versions of Win32 calls and the
00098 // C runtime.
00099 static const char* Utf16ToAnsi(LPCWSTR utf16_str);
00100 #endif
00101
00102 // Compares two C strings. Returns true if and only if they have the same
00103 // content.
00104 //
00105 // Unlike strcmp(), this function can handle NULL argument(s). A
00106 // NULL C string is considered different to any non-NULL C string,
00107 // including the empty string.
00108 static bool CStringEquals(const char* lhs, const char* rhs);

```

```

00109
00110 // Converts a wide C string to a String using the UTF-8 encoding.
00111 // NULL will be converted to "(null)". If an error occurred during
00112 // the conversion, "(failed to convert from wide string)" is
00113 // returned.
00114 static std::string ShowWideCString(const wchar_t* wide_c_str);
00115
00116 // Compares two wide C strings. Returns true if and only if they have the
00117 // same content.
00118 //
00119 // Unlike wcsncmp(), this function can handle NULL argument(s). A
00120 // NULL C string is considered different to any non-NULL C string,
00121 // including the empty string.
00122 static bool WideCStringEquals(const wchar_t* lhs, const wchar_t* rhs);
00123
00124 // Compares two C strings, ignoring case. Returns true if and only if
00125 // they have the same content.
00126 //
00127 // Unlike strcasecmp(), this function can handle NULL argument(s).
00128 // A NULL C string is considered different to any non-NULL C string,
00129 // including the empty string.
00130 static bool CaseInsensitiveCStringEquals(const char* lhs, const char* rhs);
00131
00132 // Compares two wide C strings, ignoring case. Returns true if and only if
00133 // they have the same content.
00134 //
00135 // Unlike wcsncasecmp(), this function can handle NULL argument(s).
00136 // A NULL C string is considered different to any non-NULL wide C string,
00137 // including the empty string.
00138 // NB: The implementations on different platforms slightly differ.
00139 // On windows, this method uses _wcsicmp which compares according to LC_CTYPE
00140 // environment variable. On GNU platform this method uses wcsncasecmp
00141 // which compares according to LC_CTYPE category of the current locale.
00142 // On MacOS X, it uses towlower, which also uses LC_CTYPE category of the
00143 // current locale.
00144 static bool CaseInsensitiveWideCStringEquals(const wchar_t* lhs,
00145 const wchar_t* rhs);
00146
00147 // Returns true if and only if the given string ends with the given suffix,
00148 // ignoring case. Any string is considered to end with an empty suffix.
00149 static bool EndsWithCaseInsensitive(const std::string& str,
00150 const std::string& suffix);
00151
00152 // Formats an int value as "%02d".
00153 static std::string FormatIntWidth2(int value); // "%02d" for width == 2
00154
00155 // Formats an int value to given width with leading zeros.
00156 static std::string FormatIntWidthN(int value, int width);
00157
00158 // Formats an int value as "%X".
00159 static std::string FormatHexInt(int value);
00160
00161 // Formats an int value as "%X".
00162 static std::string FormatHexUInt32(uint32_t value);
00163
00164 // Formats a byte as "%02X".
00165 static std::string FormatByte(unsigned char value);
00166
00167 private:
00168 String(); // Not meant to be instantiated.
00169 }; // class String
00170
00171 // Gets the content of the stringstream's buffer as an std::string. Each '\0'
00172 // character in the buffer is replaced with "\\0".
00173 GTEST_API_ std::string StringStreamToString(::std::stringstream* stream);
00174
00175 } // namespace internal
00176 } // namespace testing
00177
00178 #endif // GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_STRING_H_

```

## 16.253 gtest-type-util.h

```

00001 // Copyright 2008 Google Inc.
00002 // All Rights Reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the

```



```

00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // Type utilities needed for implementing typed and type-parameterized
00031 // tests.
00032
00033 // IWYU pragma: private, include "gtest/gtest.h"
00034 // IWYU pragma: friend gtest/*.
00035 // IWYU pragma: friend gmock/*.
00036
00037 #ifndef GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_TYPE_UTIL_H_
00038 #define GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_TYPE_UTIL_H_
00039
00040 #include "gtest/internal/gtest-port.h"
00041
00042 // #ifdef __GNUC__ is too general here. It is possible to use gcc without using
00043 // libstdc++ (which is where cxxabi.h comes from).
00044 #if GTEST_HAS_CXXABI_H_
00045 #include <cxxabi.h>
00046 #elif defined(__HP_aCC)
00047 #include <acxx_demangle.h>
00048 #endif // GTEST_HASH_CXXABI_H_
00049
00050 namespace testing {
00051 namespace internal {
00052
00053 // Canonicalizes a given name with respect to the Standard C++ Library.
00054 // This handles removing the inline namespace within `std` that is
00055 // used by various standard libraries (e.g., `std::__1`). Names outside
00056 // of namespace std are returned unmodified.
00057
00058 inline std::string CanonicalizeForStdLibVersioning(std::string s) {
00059 static const char prefix[] = "std::__";
00060 if (s.compare(0, strlen(prefix), prefix) == 0) {
00061 std::string::size_type end = s.find(":", strlen(prefix));
00062 if (end != s.npos) {
00063 // Erase everything between the initial `std` and the second `:`.
00064 s.erase(strlen("std"), end - strlen("std"));
00065 }
00066 }
00067 return s;
00068 }
00069
00070 #if GTEST_HAS_RTTI
00071 // GetTypeName(const std::type_info&) returns a human-readable name of type T.
00072
00073 inline std::string GetTypeName(const std::type_info& type) {
00074 const char* const name = type.name();
00075 #if GTEST_HAS_CXXABI_H_ || defined(__HP_aCC)
00076 int status = 0;
00077 // gcc's implementation of typeid(T).name() mangles the type name,
00078 // so we have to demangle it.
00079 #if GTEST_HAS_CXXABI_H_
00080 using abi::__cxa_demangle;
00081 #endif // GTEST_HAS_CXXABI_H_
00082 char* const readable_name = __cxa_demangle(name, nullptr, nullptr, &status);
00083 const std::string name_str(status == 0 ? readable_name : name);
00084 free(readable_name);
00085 return CanonicalizeForStdLibVersioning(name_str);
00086 #else
00087 return name;
00088 #endif // GTEST_HAS_CXXABI_H_ || __HP_aCC
00089 }
00090 #endif // GTEST_HAS_RTTI
00091
00092 // GetTypeName<T>() returns a human-readable name of type T if and only if
00093 // RTTI is enabled, otherwise it returns a dummy type name.
00094 // NB: This function is also used in Google Mock, so don't move it inside of
00095 // the typed-test-only section below.
00096
00097 template <typename T>
00098 std::string GetTypeName() {
00099 #if GTEST_HAS_RTTI

```

```

00100 return GetTypeName(typeid (T));
00101 #else
00102 return "<type>";
00103 #endif // GTEST_HAS_RTTI
00104 }
00105
00106 // A unique type indicating an empty node
00107
00108 struct None {
00109 };
00110
00111 #define GTEST_TEMPLATE_ \
00112 template <typename T> \
00113 class
00114
00115 // The template "selector" struct TemplateSel<Tmpl> is used to
00116 // represent Tmpl, which must be a class template with one type
00117 // parameter, as a type. TemplateSel<Tmpl>::Bind<T>::type is defined
00118 // as the type Tmpl<T>. This allows us to actually instantiate the
00119 // template "selected" by TemplateSel<Tmpl>.
00120 //
00121 // This trick is necessary for simulating typedef for class templates,
00122 // which C++ doesn't support directly.
00123
00124 template <GTEST_TEMPLATE_ Tmpl>
00125 struct TemplateSel {
00126
00127 template <typename T>
00128 struct Bind {
00129 typedef Tmpl<T> type;
00130 };
00131 };
00132
00133 #define GTEST_BIND_(TmplSel, T) TmplSel::template Bind<T>::type
00134
00135 template <GTEST_TEMPLATE_ Head_, GTEST_TEMPLATE_... Tail_>
00136 struct Templates {
00137 using Head = TemplateSel<Head_>;
00138 using Tail = Templates<Tail_...>;
00139 };
00140
00141 template <GTEST_TEMPLATE_ Head_>
00142 struct Templates<Head_> {
00143 using Head = TemplateSel<Head_>;
00144 using Tail = None;
00145 };
00146
00147 // Tuple-like type lists
00148
00149 template <typename Head_, typename... Tail_>
00150 struct Types {
00151 using Head = Head_;
00152 using Tail = Types<Tail_...>;
00153 };
00154
00155 template <typename Head_>
00156 struct Types<Head_> {
00157 using Head = Head_;
00158 using Tail = None;
00159 };
00160
00161 // Helper metafunctions to tell apart a single type from types
00162 // generated by ::testing::Types
00163
00164 template <typename... Ts>
00165 struct ProxyTypeList {
00166 using type = Types<Ts...>;
00167 };
00168
00169 template <typename>
00170 struct is_proxy_type_list : std::false_type {
00171 };
00172
00173 template <typename... Ts>
00174 struct is_proxy_type_list<ProxyTypeList<Ts...> : std::true_type
00175 {
00176 };
00177
00178 // Generator which conditionally creates type lists.
00179 // It recognizes if a requested type list should be created
00180 // and prevents creating a new type list nested within another one.
00181
00182 template <typename T>
00183 struct GenerateTypeList {
00184 private:
00185 using proxy = typename std::conditional<is_proxy_type_list<T>::value, T,
00186 ProxyTypeList<T>::type;

```

```

00187
00188 public:
00189 using type = typename proxy::type;
00190 };
00191
00192 } // namespace internal
00193
00194 template <typename... Ts>
00195 using Types = internal::ProxyTypeList<Ts...>;
00196
00197 } // namespace testing
00198
00199 #endif // GOOGLETEST_INCLUDE_GTEST_INTERNAL_GTEST_TYPE_UTIL_H_

```

## 16.254 gtest-assertion-result.cc

```

00001 // Copyright 2005, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // The Google C++ Testing and Mocking Framework (Google Test)
00031 //
00032 // This file defines the AssertionResult type.
00033
00034 #include "gtest/gtest-assertion-result.h"
00035
00036 #include <string>
00037 #include <utility>
00038
00039 #include "gtest/gtest-message.h"
00040
00041 namespace testing {
00042
00043 // AssertionResult constructors.
00044 // Used in EXPECT_TRUE/FALSE(assertion_result).
00045
00046 AssertionResult::AssertionResult(const AssertionResult& other)
00047 : success_(other.success_),
00048 message_(other.message_.get() != nullptr
00049 ? new ::std::string(*other.message_)
00050 : static_cast< ::std::string*> (nullptr)) {
00051 }
00052
00053 // Swaps two AssertionResults.
00054
00055 void AssertionResult::swap(AssertionResult& other) {
00056 using std::swap;
00057 swap(success_, other.success_);
00058 swap(message_, other.message_);
00059 }
00060
00061 // Returns the assertion's negation. Used with EXPECT/ASSERT_FALSE.
00062
00063 AssertionResult AssertionResult::operator!() const {
00064 AssertionResult negation(!success_);
00065 if (message_.get() != nullptr) negation « *message_;
00066 return negation;
00067 }
00068
00069 // Makes a successful assertion result.

```

```

00070
00071 AssertionResult AssertionSuccess() {
00072 return AssertionResult(true);
00073 }
00074
00075 // Makes a failed assertion result.
00076
00077 AssertionResult AssertionFailure() {
00078 return AssertionResult(false);
00079 }
00080
00081 // Makes a failed assertion result with the given failure message.
00082 // Deprecated; use AssertionFailure() « message.
00083
00084 AssertionResult AssertionFailure(const Message& message) {
00085 return AssertionFailure() « message;
00086 }
00087
00088 } // namespace testing

```

## 16.255 gtest-death-test.cc

```

00001 // Copyright 2005, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 //
00031 // This file implements death tests.
00032
00033 #include "gtest/gtest-death-test.h"
00034
00035 #include <functional>
00036 #include <utility>
00037
00038 #include "gtest/internal/custom/gtest.h"
00039 #include "gtest/internal/gtest-port.h"
00040
00041 #if GTEST_HAS_DEATH_TEST
00042
00043 #if GTEST_OS_MAC
00044 #include <crt_externs.h>
00045 #endif // GTEST_OS_MAC
00046
00047 #include <errno.h>
00048 #include <fcntl.h>
00049 #include <limits.h>
00050
00051 #if GTEST_OS_LINUX
00052 #include <signal.h>
00053 #endif // GTEST_OS_LINUX
00054
00055 #include <stdarg.h>
00056
00057 #if GTEST_OS_WINDOWS
00058 #include <windows.h>
00059 #else
00060 #include <sys/mman.h>
00061 #include <sys/wait.h>
00062 #endif // GTEST_OS_WINDOWS
00063

```

```

00064 #if GTEST_OS_QNX
00065 #include <spawn.h>
00066 #endif // GTEST_OS_QNX
00067
00068 #if GTEST_OS_FUCHSIA
00069 #include <lib/fdio/fd.h>
00070 #include <lib/fdio/io.h>
00071 #include <lib/fdio/spawn.h>
00072 #include <lib/zz/channel.h>
00073 #include <lib/zz/port.h>
00074 #include <lib/zz/process.h>
00075 #include <lib/zz/socket.h>
00076 #include <zircon/processargs.h>
00077 #include <zircon/syscalls.h>
00078 #include <zircon/syscalls/policy.h>
00079 #include <zircon/syscalls/port.h>
00080 #endif // GTEST_OS_FUCHSIA
00081
00082 #endif // GTEST_HAS_DEATH_TEST
00083
00084 #include "gtest/gtest-message.h"
00085 #include "gtest/internal/gtest-string.h"
00086 #include "src/gtest-internal-inl.h"
00087
00088 namespace testing {
00089
00090 // Constants.
00091
00092 // The default death test style.
00093 //
00094 // This is defined in internal/gtest-port.h as "fast", but can be overridden by
00095 // a definition in internal/custom/gtest-port.h. The recommended value, which is
00096 // used internally at Google, is "threadsafe".
00097 static const char kDefaultDeathTestStyle[] = GTEST_DEFAULT_DEATH_TEST_STYLE;
00098
00099 } // namespace testing
00100
00101 GTEST_DEFINE_string_(
00102 death_test_style,
00103 testing::internal::StringFromGTestEnv("death_test_style",
00104 testing::kDefaultDeathTestStyle),
00105 "Indicates how to run a death test in a forked child process: "
00106 "\"threadsafe\" (child process re-executes the test binary "
00107 "\"from the beginning, running only the specific death test) or "
00108 "\"fast\" (child process runs the death test immediately "
00109 "\"after forking).");
00110
00111 GTEST_DEFINE_bool_(
00112 death_test_use_fork,
00113 testing::internal::BoolFromGTestEnv("death_test_use_fork", false),
00114 "Instructs to use fork()/_exit() instead of clone() in death tests. "
00115 "Ignored and always uses fork() on POSIX systems where clone() is not "
00116 "implemented. Useful when running under valgrind or similar tools if "
00117 "those do not support clone(). Valgrind 3.3.1 will just fail if "
00118 "it sees an unsupported combination of clone() flags. "
00119 "It is not recommended to use this flag w/o valgrind though it will "
00120 "work in 99% of the cases. Once valgrind is fixed, this flag will "
00121 "most likely be removed.");
00122
00123 GTEST_DEFINE_string_(
00124 internal_run_death_test, "",
00125 "Indicates the file, line number, temporal index of "
00126 "the single death test to run, and a file descriptor to "
00127 "which a success code may be sent, all separated by "
00128 "the '|' characters. This flag is specified if and only if the "
00129 "current process is a sub-process launched for running a thread-safe "
00130 "death test. FOR INTERNAL USE ONLY.");
00131
00132 namespace testing {
00133
00134 #if GTEST_HAS_DEATH_TEST
00135
00136 namespace internal {
00137
00138 // Valid only for fast death tests. Indicates the code is running in the
00139 // child process of a fast style death test.
00140 #if !GTEST_OS_WINDOWS && !GTEST_OS_FUCHSIA
00141 static bool g_in_fast_death_test_child = false;
00142 #endif
00143
00144 // Returns a Boolean value indicating whether the caller is currently
00145 // executing in the context of the death test child process. Tools such as
00146 // Valgrind heap checkers may need this to modify their behavior in death
00147 // tests. IMPORTANT: This is an internal utility. Using it may break the
00148 // implementation of death tests. User code MUST NOT use it.
00149
00150 bool InDeathTestChild() {

```

```

00151 #if GTEST_OS_WINDOWS || GTEST_OS_FUCHSIA
00152
00153 // On Windows and Fuchsia, death tests are thread-safe regardless of the value
00154 // of the death_test_style flag.
00155 return !GTEST_FLAG_GET(internal_run_death_test).empty();
00156
00157 #else
00158
00159 if (GTEST_FLAG_GET(death_test_style) == "threadsafe")
00160 return !GTEST_FLAG_GET(internal_run_death_test).empty();
00161 else
00162 return g_in_fast_death_test_child;
00163 #endif
00164 }
00165
00166 } // namespace internal
00167
00168 // ExitedWithCode constructor.
00169
00170 ExitedWithCode::ExitedWithCode(int exit_code) : exit_code_(exit_code) {
00171 }
00172
00173 // ExitedWithCode function-call operator.
00174
00175 bool ExitedWithCode::operator()(int exit_status) const {
00176 #if GTEST_OS_WINDOWS || GTEST_OS_FUCHSIA
00177 return exit_status == exit_code_;
00178 #else
00179 return WIFEXITED(exit_status) && WEXITSTATUS(exit_status) == exit_code_;
00180 #endif
00181 }
00182
00183 #endif // GTEST_OS_WINDOWS || GTEST_OS_FUCHSIA
00184
00185 }
00186
00187 #if !GTEST_OS_WINDOWS && !GTEST_OS_FUCHSIA
00188 // KilledBySignal constructor.
00189
00190 KilledBySignal::KilledBySignal(int signum) : signum_(signum) {
00191 }
00192
00193 // KilledBySignal function-call operator.
00194
00195 bool KilledBySignal::operator()(int exit_status) const {
00196 #if defined(GTEST_KILLED_BY_SIGNAL_OVERRIDE_)
00197 {
00198 bool result;
00199 if (GTEST_KILLED_BY_SIGNAL_OVERRIDE_(signum_, exit_status, &result)) {
00200 return result;
00201 }
00202 }
00203 #endif // defined(GTEST_KILLED_BY_SIGNAL_OVERRIDE_)
00204 return WIFSIGNALED(exit_status) && WTERMSIG(exit_status) == signum_;
00205 }
00206 #endif // !GTEST_OS_WINDOWS && !GTEST_OS_FUCHSIA
00207
00208 namespace internal {
00209
00210 // Utilities needed for death tests.
00211
00212 // Generates a textual description of a given exit code, in the format
00213 // specified by wait(2).
00214
00215 static std::string ExitSummary(int exit_code) {
00216 Message m;
00217
00218 #if GTEST_OS_WINDOWS || GTEST_OS_FUCHSIA
00219 m << "Exited with exit status " << exit_code;
00220 #else
00221 if (WIFEXITED(exit_code)) {
00222 m << "Exited with exit status " << WEXITSTATUS(exit_code);
00223 } else if (WIFSIGNALED(exit_code)) {
00224 m << "Terminated by signal " << WTERMSIG(exit_code);
00225 }
00226 #endif
00227
00228 #ifdef WCOREDUMP
00229 if (WCOREDUMP(exit_code)) {
00230 m << " (core dumped)";
00231 }
00232 #endif
00233 }
00234 #endif // GTEST_OS_WINDOWS || GTEST_OS_FUCHSIA
00235
00236 return m.GetString();
00237 }

```

```

00238
00239 // Returns true if exit_status describes a process that was terminated
00240 // by a signal, or exited normally with a nonzero exit code.
00241
00242 bool ExitedUnsuccessfully(int exit_status) {
00243 return !ExitedWithCode(0) (exit_status);
00244 }
00245
00246 #if !GTEST_OS_WINDOWS && !GTEST_OS_FUCHSIA
00247 // Generates a textual failure message when a death test finds more than
00248 // one thread running, or cannot determine the number of threads, prior
00249 // to executing the given statement. It is the responsibility of the
00250 // caller not to pass a thread_count of 1.
00251
00252 static std::string DeathTestThreadWarning(size_t thread_count) {
00253 Message msg;
00254 msg << "Death tests use fork(), which is unsafe particularly"
00255 << " in a threaded context. For this test, " << GTEST_NAME_ << " ";
00256 if (thread_count == 0) {
00257 msg << "couldn't detect the number of threads.";
00258 } else {
00259 msg << "detected " << thread_count << " threads.";
00260 }
00261 msg << " See "
00262 << "https://github.com/google/googletest/blob/main/docs/"
00263 << "advanced.md#death-tests-and-threads"
00264 << " for more explanation and suggested solutions, especially if"
00265 << " this is the last message you see before your test times out.";
00266 return msg.GetString();
00267 }
00268 #endif // !GTEST_OS_WINDOWS && !GTEST_OS_FUCHSIA
00269
00270 // Flag characters for reporting a death test that did not die.
00271 static const char kDeathTestLived = 'L';
00272 static const char kDeathTestReturned = 'R';
00273 static const char kDeathTestThrew = 'T';
00274 static const char kDeathTestInternalError = 'I';
00275
00276 #if GTEST_OS_FUCHSIA
00277
00278 // File descriptor used for the pipe in the child process.
00279 static const int kFuchsiaReadPipeFd = 3;
00280
00281 #endif
00282
00283 // An enumeration describing all of the possible ways that a death test can
00284 // conclude. DIED means that the process died while executing the test
00285 // code; LIVED means that process lived beyond the end of the test code;
00286 // RETURNED means that the test statement attempted to execute a return
00287 // statement, which is not allowed; THREW means that the test statement
00288 // returned control by throwing an exception. IN_PROGRESS means the test
00289 // has not yet concluded.
00290
00291 enum DeathTestOutcome {
00292 IN_PROGRESS, DIED, LIVED, RETURNED, THREW
00293 };
00294
00295 // Routine for aborting the program which is safe to call from an
00296 // exec-style death test child process, in which case the error
00297 // message is propagated back to the parent process. Otherwise, the
00298 // message is simply printed to stderr. In either case, the program
00299 // then exits with status 1.
00300 [[noreturn]] static void DeathTestAbort(const std::string& message) {
00301 // On a POSIX system, this function may be called from a threadsafe-style
00302 // death test child process, which operates on a very small stack. Use
00303 // the heap for any additional non-minuscule memory requirements.
00304 const InternalRunDeathTestFlag * const flag =
00305 GetUnitTestImpl()->internal_run_death_test_flag();
00306 if (flag != nullptr) {
00307 FILE* parent = posix::FDOpen(flag->write_fd(), "w");
00308 fprintf(parent, "DeathTestInternalError, parent");
00309 fprintf(parent, "%s", message.c_str());
00310 fflush(parent);
00311 _exit(1);
00312 } else {
00313 fprintf(stderr, "%s", message.c_str());
00314 fflush(stderr);
00315 posix::Abort();
00316 }
00317 }
00318
00319 // A replacement for CHECK that calls DeathTestAbort if the assertion
00320 // fails.
00321 #define GTEST_DEATH_TEST_CHECK_(expression) \
00322 do { \
00323 if (!testing::internal::IsTrue(expression)) { \
00324 DeathTestAbort(::std::string("CHECK failed: File ") + __FILE__ + \

```

```

00325 ", line " +
00326 ::testing::internal::StreamableToString(__LINE__) +
00327 ": " + #expression);
00328 }
00329 } while (::testing::internal::AlwaysFalse())
00330
00331 // This macro is similar to GTEST_DEATH_TEST_CHECK_, but it is meant for
00332 // evaluating any system call that fulfills two conditions: it must return
00333 // -1 on failure, and set errno to EINTR when it is interrupted and
00334 // should be tried again. The macro expands to a loop that repeatedly
00335 // evaluates the expression as long as it evaluates to -1 and sets
00336 // errno to EINTR. If the expression evaluates to -1 but errno is
00337 // something other than EINTR, DeathTestAbort is called.
00338 #define GTEST_DEATH_TEST_CHECK_SYSCALL_(expression)
00339 do {
00340 int gtest_retval;
00341 do {
00342 gtest_retval = (expression);
00343 } while (gtest_retval == -1 && errno == EINTR);
00344 if (gtest_retval == -1) {
00345 DeathTestAbort(::std::string("CHECK failed: File ") + __FILE__ +
00346 ", line " +
00347 ::testing::internal::StreamableToString(__LINE__) +
00348 ": " + #expression + " != -1");
00349 }
00350 } while (::testing::internal::AlwaysFalse())
00351
00352 // Returns the message describing the last system error in errno.
00353
00354 std::string GetLastErrnoDescription() {
00355 return errno == 0 ? "" : posix::StrError(errno);
00356 }
00357
00358 // This is called from a death test parent process to read a failure
00359 // message from the death test child process and log it with the FATAL
00360 // severity. On Windows, the message is read from a pipe handle. On other
00361 // platforms, it is read from a file descriptor.
00362
00363 static void FailFromInternalError(int fd) {
00364 Message error;
00365 char buffer[256];
00366 int num_read;
00367
00368 do {
00369 while ((num_read = posix::Read(fd, buffer, 255)) > 0) {
00370 buffer[num_read] = '\0';
00371 error << buffer;
00372 }
00373 } while (num_read == -1 && errno == EINTR);
00374
00375 if (num_read == 0) {
00376 GTEST_LOG_(FATAL) << error.GetString();
00377 } else {
00378 const int last_error = errno;
00379 GTEST_LOG_(FATAL) << "Error while reading death test internal: "
00380 << GetLastErrnoDescription() << " [" << last_error << "]";
00381 }
00382 }
00383
00384 // Death test constructor. Increments the running death test count
00385 // for the current test.
00386
00387 DeathTest::DeathTest() {
00388 TestInfo * const info = GetUnitTestImpl()->current_test_info();
00389 if (info == nullptr) {
00390 DeathTestAbort(
00391 "Cannot run a death test outside of a TEST or "
00392 "TEST_F construct");
00393 }
00394 }
00395
00396 // Creates and returns a death test by dispatching to the current
00397 // death test factory.
00398
00399 bool DeathTest::Create(const char* statement,
00400 Matcher<const std::string> matcher, const char* file,
00401 int line, DeathTest** test) {
00402 return GetUnitTestImpl()->death_test_factory()->Create(
00403 statement, std::move(matcher), file, line, test);
00404 }
00405
00406 const char* DeathTest::LastMessage() {
00407 return last_death_test_message_.c_str();
00408 }
00409
00410 void DeathTest::set_last_death_test_message(const std::string& message) {
00411 last_death_test_message_ = message;

```



```

00412 }
00413
00414 std::string DeathTest::last_death_test_message_;
00415
00416 // Provides cross platform implementation for some death functionality.
00417
00418 class DeathTestImpl : public DeathTest {
00419 protected:
00420
00421 DeathTestImpl(const char* a_statement, Matcher<const std::string&> matcher)
00422 : statement_(a_statement),
00423 matcher_(std::move(matcher)),
00424 spawned_(false),
00425 status_(-1),
00426 outcome_(IN_PROGRESS),
00427 read_fd_(-1),
00428 write_fd_(-1) {
00429 }
00430
00431 // read_fd_ is expected to be closed and cleared by a derived class.
00432
00433 ~DeathTestImpl() override {
00434 GTEST_DEATH_TEST_CHECK_(read_fd_ == -1);
00435 }
00436
00437 void Abort(AbortReason reason) override;
00438 bool Passed(bool status_ok) override;
00439
00440 const char* statement() const {
00441 return statement_;
00442 }
00443
00444 bool spawned() const {
00445 return spawned_;
00446 }
00447
00448 void set_spawned(bool is_spawned) {
00449 spawned_ = is_spawned;
00450 }
00451
00452 int status() const {
00453 return status_;
00454 }
00455
00456 void set_status(int a_status) {
00457 status_ = a_status;
00458 }
00459
00460 DeathTestOutcome outcome() const {
00461 return outcome_;
00462 }
00463
00464 void set_outcome(DeathTestOutcome an_outcome) {
00465 outcome_ = an_outcome;
00466 }
00467
00468 int read_fd() const {
00469 return read_fd_;
00470 }
00471
00472 void set_read_fd(int fd) {
00473 read_fd_ = fd;
00474 }
00475
00476 int write_fd() const {
00477 return write_fd_;
00478 }
00479
00480 void set_write_fd(int fd) {
00481 write_fd_ = fd;
00482 }
00483
00484 // Called in the parent process only. Reads the result code of the death
00485 // test child process via a pipe, interprets it to set the outcome_
00486 // member, and closes read_fd_. Outputs diagnostics and terminates in
00487 // case of unexpected codes.
00488 void ReadAndInterpretStatusByte();
00489
00490 // Returns stderr output from the child process.
00491 virtual std::string GetErrorLogs();
00492
00493 private:
00494 // The textual content of the code this object is testing. This class
00495 // doesn't own this string and should not attempt to delete it.
00496 const char* const statement_;
00497 // A matcher that's expected to match the stderr output by the child process.
00498 Matcher<const std::string&> matcher_;

```

```

00499 // True if the death test child process has been successfully spawned.
00500 bool spawned_;
00501 // The exit status of the child process.
00502 int status_;
00503 // How the death test concluded.
00504 DeathTestOutcome outcome_;
00505 // Descriptor to the read end of the pipe to the child process. It is
00506 // always -1 in the child process. The child keeps its write end of the
00507 // pipe in write_fd_.
00508 int read_fd_;
00509 // Descriptor to the child's write end of the pipe to the parent process.
00510 // It is always -1 in the parent process. The parent keeps its end of the
00511 // pipe in read_fd_.
00512 int write_fd_;
00513 };
00514
00515 // Called in the parent process only. Reads the result code of the death
00516 // test child process via a pipe, interprets it to set the outcome_
00517 // member, and closes read_fd_. Outputs diagnostics and terminates in
00518 // case of unexpected codes.
00519
00520 void DeathTestImpl::ReadAndInterpretStatusByte() {
00521 char flag;
00522 int bytes_read;
00523
00524 // The read() here blocks until data is available (signifying the
00525 // failure of the death test) or until the pipe is closed (signifying
00526 // its success), so it's okay to call this in the parent before
00527 // the child process has exited.
00528 do {
00529 bytes_read = posix::Read(read_fd(), &flag, 1);
00530 } while (bytes_read == -1 && errno == EINTR);
00531
00532 if (bytes_read == 0) {
00533 set_outcome(DIED);
00534 } else if (bytes_read == 1) {
00535 switch (flag) {
00536 case kDeathTestReturned:
00537 set_outcome(RETURNED);
00538 break;
00539 case kDeathTestThrew:
00540 set_outcome(THREW);
00541 break;
00542 case kDeathTestLived:
00543 set_outcome(LIVED);
00544 break;
00545 case kDeathTestInternalError:
00546 FailFromInternalError(read_fd()); // Does not return.
00547 break;
00548 default:
00549 GTEST_LOG_(FATAL) << "Death test child process reported "
00550 << "unexpected status byte ("
00551 << static_cast<unsigned int>(flag) << ")";
00552 }
00553 } else {
00554 GTEST_LOG_(FATAL) << "Read from death test child process failed: "
00555 << GetLastErrorDescription();
00556 }
00557 GTEST_DEATH_TEST_CHECK_SYSCALL_(posix::Close(read_fd()));
00558 set_read_fd(-1);
00559 }
00560
00561 std::string DeathTestImpl::GetErrorLogs() {
00562 return GetCapturedStderr();
00563 }
00564
00565 // Signals that the death test code which should have exited, didn't.
00566 // Should be called only in a death test child process.
00567 // Writes a status byte to the child's status file descriptor, then
00568 // calls _exit(1).
00569
00570 void DeathTestImpl::Abort(AbortReason reason) {
00571 // The parent process considers the death test to be a failure if
00572 // it finds any data in our pipe. So, here we write a single flag byte
00573 // to the pipe, then exit.
00574 const char status_ch = reason == TEST_DID_NOT_DIE ? kDeathTestLived
00575 : reason == TEST_THREW_EXCEPTION ? kDeathTestThrew
00576 : kDeathTestReturned;
00577
00578 GTEST_DEATH_TEST_CHECK_SYSCALL_(posix::Write(write_fd(), &status_ch, 1));
00579 // We are leaking the descriptor here because on some platforms (i.e.,
00580 // when built as Windows DLL), destructors of global objects will still
00581 // run after calling _exit(). On such systems, write_fd_ will be
00582 // indirectly closed from the destructor of UnitTestImpl, causing double
00583 // close if it is also closed here. On debug configurations, double close
00584 // may assert. As there are no in-process buffers to flush here, we are
00585 // relying on the OS to close the descriptor after the process terminates

```

```

00586 // when the destructors are not run.
00587 _exit(1); // Exits w/o any normal exit hooks (we were supposed to crash)
00588 }
00589
00590 // Returns an indented copy of stderr output for a death test.
00591 // This makes distinguishing death test output lines from regular log lines
00592 // much easier.
00593
00594 static ::std::string FormatDeathTestOutput(const ::std::string& output) {
00595 ::std::string ret;
00596 for (size_t at = 0;;) {
00597 const size_t line_end = output.find('\n', at);
00598 ret += "[DEATH] ";
00599 if (line_end == ::std::string::npos) {
00600 ret += output.substr(at);
00601 break;
00602 }
00603 ret += output.substr(at, line_end + 1 - at);
00604 at = line_end + 1;
00605 }
00606 return ret;
00607 }
00608
00609 // Assesses the success or failure of a death test, using both private
00610 // members which have previously been set, and one argument:
00611 //
00612 // Private data members:
00613 // outcome: An enumeration describing how the death test
00614 // concluded: DIED, LIVED, THREW, or RETURNED. The death test
00615 // fails in the latter three cases.
00616 // status: The exit status of the child process. On *nix, it is in the
00617 // in the format specified by wait(2). On Windows, this is the
00618 // value supplied to the ExitProcess() API or a numeric code
00619 // of the exception that terminated the program.
00620 // matcher_: A matcher that's expected to match the stderr output by the child
00621 // process.
00622 //
00623 // Argument:
00624 // status_ok: true if exit_status is acceptable in the context of
00625 // this particular death test, which fails if it is false
00626 //
00627 // Returns true if and only if all of the above conditions are met. Otherwise,
00628 // the first failing condition, in the order given above, is the one that is
00629 // reported. Also sets the last death test message string.
00630
00631 bool DeathTestImpl::Passed(bool status_ok) {
00632 if (!spawned()) return false;
00633
00634 const std::string error_message = GetErrorLogs();
00635
00636 bool success = false;
00637 Message buffer;
00638
00639 buffer << "Death test: " << statement() << "\n";
00640 switch (outcome()) {
00641 case LIVED:
00642 buffer << " Result: failed to die.\n"
00643 << " Error msg:\n"
00644 << FormatDeathTestOutput(error_message);
00645 break;
00646 case THREW:
00647 buffer << " Result: threw an exception.\n"
00648 << " Error msg:\n"
00649 << FormatDeathTestOutput(error_message);
00650 break;
00651 case RETURNED:
00652 buffer << " Result: illegal return in test statement.\n"
00653 << " Error msg:\n"
00654 << FormatDeathTestOutput(error_message);
00655 break;
00656 case DIED:
00657 if (status_ok) {
00658 if (matcher_.Matches(error_message)) {
00659 success = true;
00660 } else {
00661 std::ostringstream stream;
00662 matcher_.DescribeTo(&stream);
00663 buffer << " Result: died but not with expected error.\n"
00664 << " Expected: " << stream.str() << "\n"
00665 << " Actual msg:\n"
00666 << FormatDeathTestOutput(error_message);
00667 }
00668 } else {
00669 buffer << " Result: died but not with expected exit code:\n"
00670 << " " << ExitSummary(status()) << "\n"
00671 << " Actual msg:\n"
00672 << FormatDeathTestOutput(error_message);

```

```

00673 }
00674 break;
00675 case IN_PROGRESS:
00676 default:
00677 GTEST_LOG_(FATAL)
00678 « "DeathTest::Passed somehow called before conclusion of test";
00679 }
00680
00681 DeathTest::set_last_death_test_message(buffer.GetString());
00682 return success;
00683 }
00684
00685 #if GTEST_OS_WINDOWS
00686 // WindowsDeathTest implements death tests on Windows. Due to the
00687 // specifics of starting new processes on Windows, death tests there are
00688 // always threadsafe, and Google Test considers the
00689 // --gtest_death_test_style=fast setting to be equivalent to
00690 // --gtest_death_test_style=threadsafes there.
00691 //
00692 // A few implementation notes: Like the Linux version, the Windows
00693 // implementation uses pipes for child-to-parent communication. But due to
00694 // the specifics of pipes on Windows, some extra steps are required:
00695 //
00696 // 1. The parent creates a communication pipe and stores handles to both
00697 // ends of it.
00698 // 2. The parent starts the child and provides it with the information
00699 // necessary to acquire the handle to the write end of the pipe.
00700 // 3. The child acquires the write end of the pipe and signals the parent
00701 // using a Windows event.
00702 // 4. Now the parent can release the write end of the pipe on its side. If
00703 // this is done before step 3, the object's reference count goes down to
00704 // 0 and it is destroyed, preventing the child from acquiring it. The
00705 // parent now has to release it, or read operations on the read end of
00706 // the pipe will not return when the child terminates.
00707 // 5. The parent reads child's output through the pipe (outcome code and
00708 // any possible error messages) from the pipe, and its stderr and then
00709 // determines whether to fail the test.
00710 //
00711 // Note: to distinguish Win32 API calls from the local method and function
00712 // calls, the former are explicitly resolved in the global namespace.
00713 //
00714
00715 class WindowsDeathTest : public DeathTestImpl {
00716 public:
00717 WindowsDeathTest(const char* a_statement, Matcher<const std::string&> matcher,
00718 const char* file, int line)
00719 : DeathTestImpl(a_statement, std::move(matcher)),
00720 file_(file),
00721 line_(line) {
00722 }
00723
00724 // All of these virtual functions are inherited from DeathTest.
00725 virtual int Wait();
00726 virtual TestRole AssumeRole();
00727
00728 private:
00729 // The name of the file in which the death test is located.
00730 const char* const file_;
00731 // The line number on which the death test is located.
00732 const int line_;
00733 // Handle to the write end of the pipe to the child process.
00734 AutoHandle write_handle_;
00735 // Child process handle.
00736 AutoHandle child_handle_;
00737 // Event the child process uses to signal the parent that it has
00738 // acquired the handle to the write end of the pipe. After seeing this
00739 // event the parent can release its own handles to make sure its
00740 // ReadFile() calls return when the child terminates.
00741 AutoHandle event_handle_;
00742 };
00743
00744 // Waits for the child in a death test to exit, returning its exit
00745 // status, or 0 if no child process exists. As a side effect, sets the
00746 // outcome data member.
00747
00748 int WindowsDeathTest::Wait() {
00749 if (!spawned()) return 0;
00750
00751 // Wait until the child either signals that it has acquired the write end
00752 // of the pipe or it dies.
00753 const HANDLE wait_handles[2] = {child_handle_.Get(), event_handle_.Get()};
00754 switch (::WaitForMultipleObjects(2, wait_handles,
00755 FALSE, // Waits for any of the handles.
00756 INFINITE)) {
00757 case WAIT_OBJECT_0:
00758 case WAIT_OBJECT_0 + 1:

```

```

00760 break;
00761 default:
00762 GTEST_DEATH_TEST_CHECK_(false); // Should not get here.
00763 }
00764
00765 // The child has acquired the write end of the pipe or exited.
00766 // We release the handle on our side and continue.
00767 write_handle_.Reset();
00768 event_handle_.Reset();
00769
00770 ReadAndInterpretStatusByte();
00771
00772 // Waits for the child process to exit if it haven't already. This
00773 // returns immediately if the child has already exited, regardless of
00774 // whether previous calls to WaitForMultipleObjects synchronized on this
00775 // handle or not.
00776 GTEST_DEATH_TEST_CHECK_(WAIT_OBJECT_0 ==
00777 ::WaitForSingleObject(child_handle_.Get(), INFINITE));
00778 DWORD status_code;
00779 GTEST_DEATH_TEST_CHECK_(
00780 ::GetExitCodeProcess(child_handle_.Get(), &status_code) != FALSE);
00781 child_handle_.Reset();
00782 set_status(static_cast<int> (status_code));
00783 return status();
00784 }
00785
00786 // The AssumeRole process for a Windows death test. It creates a child
00787 // process with the same executable as the current process to run the
00788 // death test. The child process is given the --gtest_filter and
00789 // --gtest_internal_run_death_test flags such that it knows to run the
00790 // current death test only.
00791
00792 DeathTest::TestRole WindowsDeathTest::AssumeRole() {
00793 const UnitTestImpl * const impl = GetUnitTestImpl();
00794 const InternalRunDeathTestFlag * const flag =
00795 impl->internal_run_death_test_flag();
00796 const TestInfo * const info = impl->current_test_info();
00797 const int death_test_index = info->result()->death_test_count();
00798
00799 if (flag != nullptr) {
00800 // ParseInternalRunDeathTestFlag() has performed all the necessary
00801 // processing.
00802 set_write_fd(flag->write_fd());
00803 return EXECUTE_TEST;
00804 }
00805
00806 // WindowsDeathTest uses an anonymous pipe to communicate results of
00807 // a death test.
00808 SECURITY_ATTRIBUTES handles_are_inheritable = {sizeof (SECURITY_ATTRIBUTES),
00809 nullptr, TRUE};
00810 HANDLE read_handle, write_handle;
00811 GTEST_DEATH_TEST_CHECK_(::CreatePipe(&read_handle, &write_handle,
00812 &handles_are_inheritable,
00813 0) // Default buffer size.
00814 != FALSE);
00815 set_read_fd(
00816 ::open_osfhandle(reinterpret_cast<intptr_t> (read_handle), O_RDONLY));
00817 write_handle_.Reset(write_handle);
00818 event_handle_.Reset(::CreateEvent(
00819 &handles_are_inheritable,
00820 TRUE, // The event will automatically reset to non-signaled state.
00821 FALSE, // The initial state is non-signalled.
00822 nullptr)); // The even is unnamed.
00823 GTEST_DEATH_TEST_CHECK_(event_handle_.Get() != nullptr);
00824 const std::string filter_flag = std::string("--") + GTEST_FLAG_PREFIX_ +
00825 "filter=" + info->test_suite_name() + "." +
00826 info->name();
00827 const std::string internal_flag =
00828 std::string("--") + GTEST_FLAG_PREFIX_ +
00829 "internal_run_death_test=" + file_ + "|" + StreamableToString(line_) +
00830 "|" + StreamableToString(death_test_index) + "|" +
00831 StreamableToString(static_cast<unsigned int> (::GetCurrentProcessId())) +
00832 // size_t has the same width as pointers on both 32-bit and 64-bit
00833 // Windows platforms.
00834 // See http://msdn.microsoft.com/en-us/library/tcxflw6.aspx.
00835 "|" + StreamableToString(reinterpret_cast<size_t> (write_handle)) + "|" +
00836 StreamableToString(reinterpret_cast<size_t> (event_handle_.Get()));
00837
00838 char executable_path[_MAX_PATH + 1]; // NOLINT
00839 GTEST_DEATH_TEST_CHECK_(
00840 _MAX_PATH + 1 != ::GetModuleFileNameA(nullptr,
00841 executable_path,
00842 _MAX_PATH));
00843
00844 std::string command_line = std::string(::GetCommandLineA()) + " " +
00845 filter_flag + " \"" + internal_flag + "\"";
00846
00847 DeathTest::set_last_death_test_message("");

```

```

00847
00848 CaptureStderr();
00849 // Flush the log buffers since the log streams are shared with the child.
00850 FlushInfoLog();
00851
00852 // The child process will share the standard handles with the parent.
00853 STARTUPINFOA startup_info;
00854 memset(&startup_info, 0, sizeof (STARTUPINFO));
00855 startup_info.dwFlags = STARTF_USESTDHANDLES;
00856 startup_info.hStdInput = ::GetStdHandle(STD_INPUT_HANDLE);
00857 startup_info.hStdOutput = ::GetStdHandle(STD_OUTPUT_HANDLE);
00858 startup_info.hStdError = ::GetStdHandle(STD_ERROR_HANDLE);
00859
00860 PROCESS_INFORMATION process_info;
00861 GTEST_DEATH_TEST_CHECK_(
00862 ::CreateProcessA(
00863 executable_path, const_cast<char*> (command_line.c_str()),
00864 nullptr, // Returned process handle is not inheritable.
00865 nullptr, // Returned thread handle is not inheritable.
00866 TRUE, // Child inherits all inheritable handles (for write_handle_).
00867 0x0, // Default creation flags.
00868 nullptr, // Inherit the parent's environment.
00869 UnitTest::GetInstance()->original_working_dir(), &startup_info,
00870 &process_info) != FALSE);
00871 child_handle_.Reset(process_info.hProcess);
00872 ::CloseHandle(process_info.hThread);
00873 set_spawned(true);
00874 return OVERSEE_TEST;
00875 }
00876
00877 #elif GTEST_OS_FUCHSIA
00878
00879 class FuchsiaDeathTest : public DeathTestImpl {
00880 public:
00881
00882 FuchsiaDeathTest(const char* a_statement, Matcher<const std::string&> matcher,
00883 const char* file, int line)
00884 : DeathTestImpl(a_statement, std::move(matcher)),
00885 file_(file),
00886 line_(line) {
00887 }
00888
00889 // All of these virtual functions are inherited from DeathTest.
00890 int Wait() override;
00891 TestRole AssumeRole() override;
00892 std::string GetErrorLogs() override;
00893
00894 private:
00895 // The name of the file in which the death test is located.
00896 const char* const file_;
00897 // The line number on which the death test is located.
00898 const int line_;
00899 // The stderr data captured by the child process.
00900 std::string captured_stderr_;
00901
00902 zx::process child_process_;
00903 zx::channel exception_channel_;
00904 zx::socket stderr_socket_;
00905 };
00906
00907 // Utility class for accumulating command-line arguments.
00908
00909 class Arguments {
00910 public:
00911
00912 Arguments() {
00913 args_.push_back(nullptr);
00914 }
00915
00916 ~Arguments() {
00917 for (std::vector<char*>::iterator i = args_.begin(); i != args_.end();
00918 ++i) {
00919 free(*i);
00920 }
00921 }
00922
00923 void AddArgument(const char* argument) {
00924 args_.insert(args_.end() - 1, posix::StrDup(argument));
00925 }
00926
00927 template <typename Str>
00928 void AddArguments(const std::vector<Str>& arguments) {
00929 for (typename std::vector<Str>::const_iterator i = arguments.begin();
00930 i != arguments.end(); ++i) {
00931 args_.insert(args_.end() - 1, posix::StrDup(i->c_str()));
00932 }
00933 }
00934 };

```

```

00934
00935 char* const* Argv() {
00936 return &args_[0];
00937 }
00938
00939 int size() {
00940 return static_cast<int> (args_.size()) - 1;
00941 }
00942
00943 private:
00944 std::vector<char*> args_;
00945 };
00946
00947 // Waits for the child in a death test to exit, returning its exit
00948 // status, or 0 if no child process exists. As a side effect, sets the
00949 // outcome data member.
00950
00951 int FuchsiaDeathTest::Wait() {
00952 const int kProcessKey = 0;
00953 const int kSocketKey = 1;
00954 const int kExceptionKey = 2;
00955
00956 if (!spawned()) return 0;
00957
00958 // Create a port to wait for socket/task/exception events.
00959 zx_status_t status_zx;
00960 zx::port port;
00961 status_zx = zx::port::create(0, &port);
00962 GTEST_DEATH_TEST_CHECK_(status_zx == ZX_OK);
00963
00964 // Register to wait for the child process to terminate.
00965 status_zx =
00966 child_process_.wait_async(port, kProcessKey, ZX_PROCESS_TERMINATED, 0);
00967 GTEST_DEATH_TEST_CHECK_(status_zx == ZX_OK);
00968
00969 // Register to wait for the socket to be readable or closed.
00970 status_zx = stderr_socket_.wait_async(
00971 port, kSocketKey, ZX_SOCKET_READABLE | ZX_SOCKET_PEER_CLOSED, 0);
00972 GTEST_DEATH_TEST_CHECK_(status_zx == ZX_OK);
00973
00974 // Register to wait for an exception.
00975 status_zx = exception_channel_.wait_async(port, kExceptionKey,
00976 ZX_CHANNEL_READABLE, 0);
00977 GTEST_DEATH_TEST_CHECK_(status_zx == ZX_OK);
00978
00979 bool process_terminated = false;
00980 bool socket_closed = false;
00981 do {
00982 zx_port_packet_t packet = {};
00983 status_zx = port.wait(zx::time::infinite(), &packet);
00984 GTEST_DEATH_TEST_CHECK_(status_zx == ZX_OK);
00985
00986 if (packet.key == kExceptionKey) {
00987 // Process encountered an exception. Kill it directly rather than
00988 // letting other handlers process the event. We will get a kProcessKey
00989 // event when the process actually terminates.
00990 status_zx = child_process_.kill();
00991 GTEST_DEATH_TEST_CHECK_(status_zx == ZX_OK);
00992 } else if (packet.key == kProcessKey) {
00993 // Process terminated.
00994 GTEST_DEATH_TEST_CHECK_(ZX_PKT_IS_SIGNAL_ONE(packet.type));
00995 GTEST_DEATH_TEST_CHECK_(packet.signal.observed & ZX_PROCESS_TERMINATED);
00996 process_terminated = true;
00997 } else if (packet.key == kSocketKey) {
00998 GTEST_DEATH_TEST_CHECK_(ZX_PKT_IS_SIGNAL_ONE(packet.type));
00999 if (packet.signal.observed & ZX_SOCKET_READABLE) {
01000 // Read data from the socket.
01001 constexpr size_t kBufferSize = 1024;
01002 do {
01003 size_t old_length = captured_stderr_.length();
01004 size_t bytes_read = 0;
01005 captured_stderr_.resize(old_length + kBufferSize);
01006 status_zx =
01007 stderr_socket_.read(0, &captured_stderr_.front() + old_length,
01008 kBufferSize, &bytes_read);
01009 captured_stderr_.resize(old_length + bytes_read);
01010 } while (status_zx == ZX_OK);
01011 if (status_zx == ZX_ERR_PEER_CLOSED) {
01012 socket_closed = true;
01013 } else {
01014 GTEST_DEATH_TEST_CHECK_(status_zx == ZX_ERR_SHOULD_WAIT);
01015 status_zx = stderr_socket_.wait_async(
01016 port, kSocketKey, ZX_SOCKET_READABLE | ZX_SOCKET_PEER_CLOSED, 0);
01017 GTEST_DEATH_TEST_CHECK_(status_zx == ZX_OK);
01018 }
01019 } else {
01020 GTEST_DEATH_TEST_CHECK_(packet.signal.observed & ZX_SOCKET_PEER_CLOSED);

```

```

01021 socket_closed = true;
01022 }
01023 }
01024 } while (!process_terminated && !socket_closed);
01025
01026 ReadAndInterpretStatusByte();
01027
01028 zx_info_process_t buffer;
01029 status_zx = child_process_.get_info(ZX_INFO_PROCESS, &buffer, sizeof (buffer),
01030 nullptr, nullptr);
01031 GTEST_DEATH_TEST_CHECK_(status_zx == ZX_OK);
01032
01033 GTEST_DEATH_TEST_CHECK_(buffer.flags & ZX_INFO_PROCESS_FLAG_EXITED);
01034 set_status(static_cast<int> (buffer.return_code));
01035 return status();
01036 }
01037
01038 // The AssumeRole process for a Fuchsia death test. It creates a child
01039 // process with the same executable as the current process to run the
01040 // death test. The child process is given the --gtest_filter and
01041 // --gtest_internal_run_death_test flags such that it knows to run the
01042 // current death test only.
01043
01044 DeathTest::TestRole FuchsiaDeathTest::AssumeRole() {
01045 const UnitTestImpl * const impl = GetUnitTestImpl();
01046 const InternalRunDeathTestFlag * const flag =
01047 impl->internal_run_death_test_flag();
01048 const TestInfo * const info = impl->current_test_info();
01049 const int death_test_index = info->result()->death_test_count();
01050
01051 if (flag != nullptr) {
01052 // ParseInternalRunDeathTestFlag() has performed all the necessary
01053 // processing.
01054 set_write_fd(kFuchsiaReadPipeFd);
01055 return EXECUTE_TEST;
01056 }
01057
01058 // Flush the log buffers since the log streams are shared with the child.
01059 FlushInfoLog();
01060
01061 // Build the child process command line.
01062 const std::string filter_flag = std::string("--") + GTEST_FLAG_PREFIX_ +
01063 "filter=" + info->test_suite_name() + "." +
01064 info->name();
01065 const std::string internal_flag = std::string("--") + GTEST_FLAG_PREFIX_ +
01066 kInternalRunDeathTestFlag + "=" + file_ +
01067 "|" + StreamableToString(line_) + "|" +
01068 StreamableToString(death_test_index);
01069 Arguments args;
01070 args.AddArguments(GetInjectableArgvs());
01071 args.AddArgument(filter_flag.c_str());
01072 args.AddArgument(internal_flag.c_str());
01073
01074 // Build the pipe for communication with the child.
01075 zx_status_t status;
01076 zx_handle_t child_pipe_handle;
01077 int child_pipe_fd;
01078 status = fdio_pipe_half(&child_pipe_fd, &child_pipe_handle);
01079 GTEST_DEATH_TEST_CHECK_(status == ZX_OK);
01080 set_read_fd(child_pipe_fd);
01081
01082 // Set the pipe handle for the child.
01083 fdio_spawn_action_t spawn_actions[2] = {};
01084 fdio_spawn_action_t* add_handle_action = &spawn_actions[0];
01085 add_handle_action->action = FDIO_SPAWN_ACTION_ADD_HANDLE;
01086 add_handle_action->h.id = PA_HND(PA_FD, kFuchsiaReadPipeFd);
01087 add_handle_action->h.handle = child_pipe_handle;
01088
01089 // Create a socket pair will be used to receive the child process' stderr.
01090 zx::socket stderr_producer_socket;
01091 status = zx::socket::create(0, &stderr_producer_socket, &stderr_socket_);
01092 GTEST_DEATH_TEST_CHECK_(status >= 0);
01093 int stderr_producer_fd = -1;
01094 status =
01095 fdio_fd_create(stderr_producer_socket.release(), &stderr_producer_fd);
01096 GTEST_DEATH_TEST_CHECK_(status >= 0);
01097
01098 // Make the stderr socket nonblocking.
01099 GTEST_DEATH_TEST_CHECK_(fcntl(stderr_producer_fd, F_SETFL, 0) == 0);
01100
01101 fdio_spawn_action_t* add_stderr_action = &spawn_actions[1];
01102 add_stderr_action->action = FDIO_SPAWN_ACTION_CLONE_FD;
01103 add_stderr_action->fd.local_fd = stderr_producer_fd;
01104 add_stderr_action->fd.target_fd = STDERR_FILENO;
01105
01106 // Create a child job.
01107 zx_handle_t child_job = ZX_HANDLE_INVALID;

```



```

01108 status = zx_job_create(zx_job_default(), 0, &child_job);
01109 GTEST_DEATH_TEST_CHECK_(status == ZX_OK);
01110 zx_policy_basic_t policy;
01111 policy.condition = ZX_POL_NEW_ANY;
01112 policy.policy = ZX_POL_ACTION_ALLOW;
01113 status = zx_job_set_policy(child_job, ZX_JOB_POL_RELATIVE, ZX_JOB_POL_BASIC,
01114 &policy, 1);
01115 GTEST_DEATH_TEST_CHECK_(status == ZX_OK);
01116
01117 // Create an exception channel attached to the |child_job|, to allow
01118 // us to suppress the system default exception handler from firing.
01119 status = zx_task_create_exception_channel(
01120 child_job, 0, exception_channel_.reset_and_get_address());
01121 GTEST_DEATH_TEST_CHECK_(status == ZX_OK);
01122
01123 // Spawn the child process.
01124 status = fdio_spawn_etc(child_job, FDIO_SPAWN_CLONE_ALL, args.Argv()[0],
01125 args.Argv(), nullptr, 2, spawn_actions,
01126 child_process_.reset_and_get_address(), nullptr);
01127 GTEST_DEATH_TEST_CHECK_(status == ZX_OK);
01128
01129 set_spawned(true);
01130 return OVERSEE_TEST;
01131 }
01132
01133 std::string FuchsiaDeathTest::GetErrorLogs() {
01134 return captured_stderr_;
01135 }
01136
01137 #else // We are neither on Windows, nor on Fuchsia.
01138
01139 // ForkingDeathTest provides implementations for most of the abstract
01140 // methods of the DeathTest interface. Only the AssumeRole method is
01141 // left undefined.
01142
01143 class ForkingDeathTest : public DeathTestImpl {
01144 public:
01145 ForkingDeathTest(const char* statement, Matcher<const std::string&> matcher);
01146
01147 // All of these virtual functions are inherited from DeathTest.
01148 int Wait() override;
01149
01150 protected:
01151
01152 void set_child_pid(pid_t child_pid) {
01153 child_pid_ = child_pid;
01154 }
01155
01156 private:
01157 // PID of child process during death test; 0 in the child process itself.
01158 pid_t child_pid_;
01159 };
01160
01161 // Constructs a ForkingDeathTest.
01162
01163 ForkingDeathTest::ForkingDeathTest(const char* a_statement,
01164 Matcher<const std::string&> matcher)
01165 : DeathTestImpl(a_statement, std::move(matcher)), child_pid_(-1) {
01166 }
01167
01168 // Waits for the child in a death test to exit, returning its exit
01169 // status, or 0 if no child process exists. As a side effect, sets the
01170 // outcome data member.
01171
01172 int ForkingDeathTest::Wait() {
01173 if (!spawned()) return 0;
01174
01175 ReadAndInterpretStatusByte();
01176
01177 int status_value;
01178 GTEST_DEATH_TEST_CHECK_SYSCALL_(waitpid(child_pid_, &status_value, 0));
01179 set_status(status_value);
01180 return status_value;
01181 }
01182
01183 // A concrete death test class that forks, then immediately runs the test
01184 // in the child process.
01185
01186 class NoExecDeathTest : public ForkingDeathTest {
01187 public:
01188
01189 NoExecDeathTest(const char* a_statement, Matcher<const std::string&> matcher)
01190 : ForkingDeathTest(a_statement, std::move(matcher)) {
01191 }
01192
01193 TestRole AssumeRole() override;
01194 };

```

```

01195 // The AssumeRole process for a fork-and-run death test. It implements a
01196 // straightforward fork, with a simple pipe to transmit the status byte.
01197
01198 DeathTest::TestRole NoExecDeathTest::AssumeRole() {
01199 const size_t thread_count = GetThreadCount();
01200 if (thread_count != 1) {
01201 GTEST_LOG_(WARNING) << DeathTestThreadWarning(thread_count);
01202 }
01203
01204 int pipe_fd[2];
01205 GTEST_DEATH_TEST_CHECK_(pipe(pipe_fd) != -1);
01206
01207 DeathTest::set_last_death_test_message("");
01208 CaptureStderr();
01209 // When we fork the process below, the log file buffers are copied, but the
01210 // file descriptors are shared. We flush all log files here so that closing
01211 // the file descriptors in the child process doesn't throw off the
01212 // synchronization between descriptors and buffers in the parent process.
01213 // This is as close to the fork as possible to avoid a race condition in case
01214 // there are multiple threads running before the death test, and another
01215 // thread writes to the log file.
01216 FlushInfoLog();
01217
01218 const pid_t child_pid = fork();
01219 GTEST_DEATH_TEST_CHECK_(child_pid != -1);
01220 set_child_pid(child_pid);
01221 if (child_pid == 0) {
01222 GTEST_DEATH_TEST_CHECK_SYSCALL_(close(pipe_fd[0]));
01223 set_write_fd(pipe_fd[1]);
01224 // Redirects all logging to stderr in the child process to prevent
01225 // concurrent writes to the log files. We capture stderr in the parent
01226 // process and append the child process' output to a log.
01227 LogToStderr();
01228 // Event forwarding to the listeners of event listener API must be shut
01229 // down in death test subprocesses.
01230 GetUnitTestImpl()->listeners()->SuppressEventForwarding();
01231 g_in_fast_death_test_child = true;
01232 return EXECUTE_TEST;
01233 } else {
01234 GTEST_DEATH_TEST_CHECK_SYSCALL_(close(pipe_fd[1]));
01235 set_read_fd(pipe_fd[0]);
01236 set_spawned(true);
01237 return OVERSEE_TEST;
01238 }
01239 }
01240
01241 // A concrete death test class that forks and re-executes the main
01242 // program from the beginning, with command-line flags set that cause
01243 // only this specific death test to be run.
01244
01245 class ExecDeathTest : public ForkingDeathTest {
01246 public:
01247 ExecDeathTest(const char* a_statement, Matcher<const std::string&> matcher,
01248 const char* file, int line)
01249 : ForkingDeathTest(a_statement, std::move(matcher),
01250 file_(file),
01251 line_(line)) {
01252 }
01253 TestRole AssumeRole() override;
01254
01255 private:
01256 static ::std::vector<std::string> GetArgsForDeathTestChildProcess() {
01257 ::std::vector<std::string> args = GetInjectableArgs();
01258 #if defined(GTEST_EXTRA_DEATH_TEST_COMMAND_LINE_ARGS_)
01259 ::std::vector<std::string> extra_args =
01260 GTEST_EXTRA_DEATH_TEST_COMMAND_LINE_ARGS_();
01261 args.insert(args.end(), extra_args.begin(), extra_args.end());
01262 #endif // defined(GTEST_EXTRA_DEATH_TEST_COMMAND_LINE_ARGS_)
01263 return args;
01264 }
01265 // The name of the file in which the death test is located.
01266 const char* const file_;
01267 // The line number on which the death test is located.
01268 const int line_;
01269 };
01270
01271 // Utility class for accumulating command-line arguments.
01272
01273 class Arguments {
01274 public:
01275 Arguments() {
01276 args_.push_back(nullptr);
01277 }
01278 };
01279
01280
01281

```

```

01282 ~Arguments() {
01283 for (std::vector<char*>::iterator i = args_.begin(); i != args_.end();
01284 ++i) {
01285 free(*i);
01286 }
01287 }
01288
01289 void AddArgument(const char* argument) {
01290 args_.insert(args_.end() - 1, posix::StrDup(argument));
01291 }
01292
01293 template <typename Str>
01294 void AddArguments(const ::std::vector<Str>& arguments) {
01295 for (typename ::std::vector<Str>::const_iterator i = arguments.begin();
01296 i != arguments.end(); ++i) {
01297 args_.insert(args_.end() - 1, posix::StrDup(i->c_str()));
01298 }
01299 }
01300
01301 char* const* Argv() {
01302 return &args_[0];
01303 }
01304
01305 private:
01306 std::vector<char*> args_;
01307 };
01308
01309 // A struct that encompasses the arguments to the child process of a
01310 // threadsafe-style death test process.
01311
01312 struct ExecDeathTestArgs {
01313 char* const* argv; // Command-line arguments for the child's call to exec
01314 int close_fd; // File descriptor to close; the read end of a pipe
01315 };
01316
01317 #if GTEST_OS_QNX
01318 extern "C" char** environ;
01319 #else
01320 // GTEST_OS_QNX
01321 // The main function for a threadsafe-style death test child process.
01322 // This function is called in a clone()-ed process and thus must avoid
01323 // any potentially unsafe operations like malloc or libc functions.
01324
01325 static int ExecDeathTestChildMain(void* child_arg) {
01326 ExecDeathTestArgs* const args = static_cast<ExecDeathTestArgs*>(child_arg);
01327 GTEST_DEATH_TEST_CHECK_SYSCALL(close(args->close_fd));
01328
01329 // We need to execute the test program in the same environment where
01330 // it was originally invoked. Therefore we change to the original
01331 // working directory first.
01332 const char* const original_dir =
01333 UnitTest::GetInstance()->original_working_dir();
01334 // We can safely call chdir() as it's a direct system call.
01335 if (chdir(original_dir) != 0) {
01336 DeathTestAbort(std::string("chdir(\"") + original_dir +
01337 "\") failed: " + GetLastErrnoDescription());
01338 return EXIT_FAILURE;
01339 }
01340
01341 // We can safely call execv() as it's almost a direct system call. We
01342 // cannot use execvp() as it's a libc function and thus potentially
01343 // unsafe. Since execv() doesn't search the PATH, the user must
01344 // invoke the test program via a valid path that contains at least
01345 // one path separator.
01346 execv(args->argv[0], args->argv);
01347 DeathTestAbort(std::string("execv(\"") + args->argv[0] + ", ...") in " +
01348 original_dir + " failed: " + GetLastErrnoDescription());
01349 return EXIT_FAILURE;
01350 }
01351 #endif // GTEST_OS_QNX
01352
01353 #if GTEST_HAS_CLONE
01354 // Two utility routines that together determine the direction the stack
01355 // grows.
01356 // This could be accomplished more elegantly by a single recursive
01357 // function, but we want to guard against the unlikely possibility of
01358 // a smart compiler optimizing the recursion away.
01359 // GTEST_NO_INLINE_ is required to prevent GCC 4.6 from inlining
01360 // StackLowerThanAddress into StackGrowsDown, which then doesn't give
01361 // correct answer.
01362 static void StackLowerThanAddress(const void* ptr,
01363 bool* result) GTEST_NO_INLINE_;
01364 // Make sure sanitizers do not tamper with the stack here.
01365 // Ideally, we want to use `__builtin_frame_address` instead of a local variable
01366 // address with sanitizer disabled, but it does not work when the
01367 // compiler optimizes the stack frame out, which happens on PowerPC targets.
01368 // HWAddressSanitizer add a random tag to the MSB of the local variable address,

```

```

01369 // making comparison result unpredictable.
01370
01371 GTEST_ATTRIBUTE_NO_SANITIZE_ADDRESS_
01372 GTEST_ATTRIBUTE_NO_SANITIZE_HWADDRESS_
01373 static void StackLowerThanAddress(const void* ptr, bool* result) {
01374 int dummy = 0;
01375 *result = std::less<const void*>()(&dummy, ptr);
01376 }
01377
01378 // Make sure AddressSanitizer does not tamper with the stack here.
01379
01380 GTEST_ATTRIBUTE_NO_SANITIZE_ADDRESS_
01381 GTEST_ATTRIBUTE_NO_SANITIZE_HWADDRESS_
01382 static bool StackGrowsDown() {
01383 int dummy = 0;
01384 bool result;
01385 StackLowerThanAddress(&dummy, &result);
01386 return result;
01387 }
01388 #endif // GTEST_HAS_CLONE
01389
01390 // Spawns a child process with the same executable as the current process in
01391 // a thread-safe manner and instructs it to run the death test. The
01392 // implementation uses fork(2) + exec. On systems where clone(2) is
01393 // available, it is used instead, being slightly more thread-safe. On QNX,
01394 // fork supports only single-threaded environments, so this function uses
01395 // spawn(2) there instead. The function dies with an error message if
01396 // anything goes wrong.
01397
01398 static pid_t ExecDeathTestSpawnChild(char* const* argv, int close_fd) {
01399 ExecDeathTestArgs args = {argv, close_fd};
01400 pid_t child_pid = -1;
01401
01402 #if GTEST_OS_QNX
01403 // Obtains the current directory and sets it to be closed in the child
01404 // process.
01405 const int cwd_fd = open(".", O_RDONLY);
01406 GTEST_DEATH_TEST_CHECK(cwd_fd != -1);
01407 GTEST_DEATH_TEST_CHECK_SYSCALL_(fcntl(cwd_fd, F_SETFD, FD_CLOEXEC));
01408 // We need to execute the test program in the same environment where
01409 // it was originally invoked. Therefore we change to the original
01410 // working directory first.
01411 const char* const original_dir =
01412 UnitTest::GetInstance()->original_working_dir();
01413 // We can safely call chdir() as it's a direct system call.
01414 if (chdir(original_dir) != 0) {
01415 DeathTestAbort(std::string("chdir(\"") + original_dir +
01416 "\") failed: " + GetLastErrnoDescription());
01417 return EXIT_FAILURE;
01418 }
01419
01420 int fd_flags;
01421 // Set close_fd to be closed after spawn.
01422 GTEST_DEATH_TEST_CHECK_SYSCALL_(fd_flags = fcntl(close_fd, F_GETFD));
01423 GTEST_DEATH_TEST_CHECK_SYSCALL_(
01424 fcntl(close_fd, F_SETFD, fd_flags | FD_CLOEXEC));
01425 struct inheritance inherit = {};
01426 // spawn is a system call.
01427 child_pid = spawn(args.argv[0], 0, nullptr, &inherit, args.argv, environ);
01428 // Restores the current working directory.
01429 GTEST_DEATH_TEST_CHECK_(fchdir(cwd_fd) != -1);
01430 GTEST_DEATH_TEST_CHECK_SYSCALL_(close(cwd_fd));
01431
01432 #else // GTEST_OS_QNX
01433 #if GTEST_OS_LINUX
01434 // When a SIGPROF signal is received while fork() or clone() are executing,
01435 // the process may hang. To avoid this, we ignore SIGPROF here and re-enable
01436 // it after the call to fork()/clone() is complete.
01437 struct sigaction saved_sigprof_action;
01438 struct sigaction ignore_sigprof_action;
01439 memset(&ignore_sigprof_action, 0, sizeof(ignore_sigprof_action));
01440 sigemptyset(&ignore_sigprof_action.sa_mask);
01441 ignore_sigprof_action.sa_handler = SIG_IGN;
01442 GTEST_DEATH_TEST_CHECK_SYSCALL_(
01443 sigaction(SIGPROF, &ignore_sigprof_action, &saved_sigprof_action));
01444 #endif // GTEST_OS_LINUX
01445
01446 #if GTEST_HAS_CLONE
01447 const bool use_fork = GTEST_FLAG_GET(death_test_use_fork);
01448
01449 if (!use_fork) {
01450 static const bool stack_grows_down = StackGrowsDown();
01451 const auto stack_size = static_cast<size_t>(getpagesize() * 2);
01452 // MMAP_ANONYMOUS is not defined on Mac, so we use MAP_ANON instead.
01453 void* const stack = mmap(nullptr, stack_size, PROT_READ | PROT_WRITE,
01454 MAP_ANON | MAP_PRIVATE, -1, 0);
01455 GTEST_DEATH_TEST_CHECK_(stack != MAP_FAILED);

```

```

01456
01457 // Maximum stack alignment in bytes: For a downward-growing stack, this
01458 // amount is subtracted from size of the stack space to get an address
01459 // that is within the stack space and is aligned on all systems we care
01460 // about. As far as I know there is no ABI with stack alignment greater
01461 // than 64. We assume stack and stack_size already have alignment of
01462 // kMaxStackAlignment.
01463 const size_t kMaxStackAlignment = 64;
01464 void* const stack_top =
01465 static_cast<char*> (stack) +
01466 (stack_grows_down ? stack_size - kMaxStackAlignment : 0);
01467 GTEST_DEATH_TEST_CHECK(
01468 static_cast<size_t> (stack_size) > kMaxStackAlignment &&
01469 reinterpret_cast<uintptr_t> (stack_top) % kMaxStackAlignment == 0);
01470
01471 child_pid = clone(&ExecDeathTestChildMain, stack_top, SIGCHLD, &args);
01472
01473 GTEST_DEATH_TEST_CHECK(munmap(stack, stack_size) != -1);
01474 }
01475 #else
01476 const bool use_fork = true;
01477 #endif // GTEST_HAS_CLONE
01478
01479 if (use_fork && (child_pid = fork()) == 0) {
01480 ExecDeathTestChildMain(&args);
01481 _exit(0);
01482 }
01483 #endif // GTEST_OS_QNX
01484 #if GTEST_OS_LINUX
01485 GTEST_DEATH_TEST_CHECK_SYSCALL(
01486 sigaction(SIGPROF, &saved_sigprof_action, nullptr));
01487 #endif // GTEST_OS_LINUX
01488
01489 GTEST_DEATH_TEST_CHECK(child_pid != -1);
01490 return child_pid;
01491 }
01492
01493 // The AssumeRole process for a fork-and-exec death test. It re-executes the
01494 // main program from the beginning, setting the --gtest_filter
01495 // and --gtest_internal_run_death_test flags to cause only the current
01496 // death test to be re-run.
01497
01498 DeathTest::TestRole ExecDeathTest::AssumeRole() {
01499 const UnitTestImpl* const impl = GetUnitTestImpl();
01500 const InternalRunDeathTestFlag* const flag =
01501 impl->internal_run_death_test_flag();
01502 const TestInfo* const info = impl->current_test_info();
01503 const int death_test_index = info->result()->death_test_count();
01504
01505 if (flag != nullptr) {
01506 set_write_fd(flag->write_fd());
01507 return EXECUTE_TEST;
01508 }
01509
01510 int pipe_fd[2];
01511 GTEST_DEATH_TEST_CHECK(pipe(pipe_fd) != -1);
01512 // Clear the close-on-exec flag on the write end of the pipe, lest
01513 // it be closed when the child process does an exec.
01514 GTEST_DEATH_TEST_CHECK(fcntl(pipe_fd[1], F_SETFD, 0) != -1);
01515
01516 const std::string filter_flag = std::string("--") + GTEST_FLAG_PREFIX_ +
01517 "filter=" + info->test_suite_name() + "." +
01518 info->name();
01519 const std::string internal_flag = std::string("--") + GTEST_FLAG_PREFIX_ +
01520 "internal_run_death_test=" + file_ + "|" +
01521 StreamableToString(line_) + "|" +
01522 StreamableToString(death_test_index) + "|" +
01523 StreamableToString(pipe_fd[1]);
01524 Arguments args;
01525 args.AddArguments(GetArgvsForDeathTestChildProcess());
01526 args.AddArgument(filter_flag.c_str());
01527 args.AddArgument(internal_flag.c_str());
01528
01529 DeathTest::set_last_death_test_message("");
01530
01531 CaptureStderr();
01532 // See the comment in NoExecDeathTest::AssumeRole for why the next line
01533 // is necessary.
01534 FlushInfoLog();
01535
01536 const pid_t child_pid = ExecDeathTestSpawnChild(args.Argv(), pipe_fd[0]);
01537 GTEST_DEATH_TEST_CHECK_SYSCALL(close(pipe_fd[1]));
01538 set_child_pid(child_pid);
01539 set_read_fd(pipe_fd[0]);
01540 set_spawned(true);
01541 return OVERSEE_TEST;
01542 }

```

```

01543
01544 #endif // !GTEST_OS_WINDOWS
01545
01546 // Creates a concrete DeathTest-derived class that depends on the
01547 // --gtest_death_test_style flag, and sets the pointer pointed to
01548 // by the "test" argument to its address. If the test should be
01549 // skipped, sets that pointer to NULL. Returns true, unless the
01550 // flag is set to an invalid value.
01551
01552 bool DefaultDeathTestFactory::Create(const char* statement,
01553 Matcher<const std::string&> matcher,
01554 const char* file, int line,
01555 DeathTest** test) {
01556 UnitTestImpl * const impl = GetUnitTestImpl();
01557 const InternalRunDeathTestFlag * const flag =
01558 impl->internal_run_death_test_flag();
01559 const int death_test_index =
01560 impl->current_test_info()->increment_death_test_count();
01561
01562 if (flag != nullptr) {
01563 if (death_test_index > flag->index()) {
01564 DeathTest::set_last_death_test_message(
01565 "Death test count (" + StreamableToString(death_test_index) +
01566 ") somehow exceeded expected maximum (" +
01567 StreamableToString(flag->index()) + ")");
01568 return false;
01569 }
01570
01571 if (!(flag->file() == file && flag->line() == line &&
01572 flag->index() == death_test_index)) {
01573 *test = nullptr;
01574 return true;
01575 }
01576 }
01577
01578 #if GTEST_OS_WINDOWS
01579 if (GTEST_FLAG_GET(death_test_style) == "threadsafe" ||
01580 GTEST_FLAG_GET(death_test_style) == "fast") {
01581 *test = new WindowsDeathTest(statement, std::move(matcher), file, line);
01582 }
01583 #elif GTEST_OS_FUCHSIA
01584 if (GTEST_FLAG_GET(death_test_style) == "threadsafe" ||
01585 GTEST_FLAG_GET(death_test_style) == "fast") {
01586 *test = new FuchsiaDeathTest(statement, std::move(matcher), file, line);
01587 }
01588 #else
01589 if (GTEST_FLAG_GET(death_test_style) == "threadsafe") {
01590 *test = new ExecDeathTest(statement, std::move(matcher), file, line);
01591 } else if (GTEST_FLAG_GET(death_test_style) == "fast") {
01592 *test = new NoExecDeathTest(statement, std::move(matcher));
01593 }
01594 #endif // GTEST_OS_WINDOWS
01595
01596 else { // NOLINT - this is more readable than unbalanced brackets inside #if.
01597 DeathTest::set_last_death_test_message("Unknown death test style \"" +
01598 GTEST_FLAG_GET(death_test_style) +
01599 "\" encountered");
01600 return false;
01601 }
01602
01603 return true;
01604 }
01605
01606 #if GTEST_OS_WINDOWS
01607 // Recreates the pipe and event handles from the provided parameters,
01608 // signals the event, and returns a file descriptor wrapped around the pipe
01609 // handle. This function is called in the child process only.
01610
01611 static int GetStatusFileDescriptor(unsigned int parent_process_id,
01612 size_t write_handle_as_size_t,
01613 size_t event_handle_as_size_t) {
01614 AutoHandle parent_process_handle(::OpenProcess(PROCESS_DUP_HANDLE,
01615 FALSE, // Non-inheritable.
01616 parent_process_id));
01617 if (parent_process_handle.Get() == INVALID_HANDLE_VALUE) {
01618 DeathTestAbort("Unable to open parent process " +
01619 StreamableToString(parent_process_id));
01620 }
01621
01622 GTEST_CHECK_(sizeof (HANDLE) <= sizeof (size_t));
01623
01624 const HANDLE write_handle = reinterpret_cast<HANDLE> (write_handle_as_size_t);

```

```

01630 HANDLE dup_write_handle;
01631
01632 // The newly initialized handle is accessible only in the parent
01633 // process. To obtain one accessible within the child, we need to use
01634 // DuplicateHandle.
01635 if (!::DuplicateHandle(parent_process_handle.Get(), write_handle,
01636 ::GetCurrentProcess(), &dup_write_handle,
01637 0x0, // Requested privileges ignored since
01638 // DUPLICATE_SAME_ACCESS is used.
01639 FALSE, // Request non-inheritable handler.
01640 DUPLICATE_SAME_ACCESS)) {
01641 DeathTestAbort("Unable to duplicate the pipe handle " +
01642 StreamableToString(write_handle_as_size_t) +
01643 " from the parent process " +
01644 StreamableToString(parent_process_id));
01645 }
01646
01647 const HANDLE event_handle = reinterpret_cast<HANDLE> (event_handle_as_size_t);
01648 HANDLE dup_event_handle;
01649
01650 if (!::DuplicateHandle(parent_process_handle.Get(), event_handle,
01651 ::GetCurrentProcess(), &dup_event_handle, 0x0, FALSE,
01652 DUPLICATE_SAME_ACCESS)) {
01653 DeathTestAbort("Unable to duplicate the event handle " +
01654 StreamableToString(event_handle_as_size_t) +
01655 " from the parent process " +
01656 StreamableToString(parent_process_id));
01657 }
01658
01659 const int write_fd =
01660 ::_open_osfhandle(reinterpret_cast<intptr_t> (dup_write_handle), O_APPEND);
01661 if (write_fd == -1) {
01662 DeathTestAbort("Unable to convert pipe handle " +
01663 StreamableToString(write_handle_as_size_t) +
01664 " to a file descriptor");
01665 }
01666
01667 // Signals the parent that the write end of the pipe has been acquired
01668 // so the parent can release its own write end.
01669 ::SetEvent(dup_event_handle);
01670
01671 return write_fd;
01672 }
01673 #endif // GTEST_OS_WINDOWS
01674
01675 // Returns a newly created InternalRunDeathTestFlag object with fields
01676 // initialized from the GTEST_FLAG(internal_run_death_test) flag if
01677 // the flag is specified; otherwise returns NULL.
01678
01679 InternalRunDeathTestFlag* ParseInternalRunDeathTestFlag() {
01680 if (GTEST_FLAG_GET(internal_run_death_test) == "") return nullptr;
01681
01682 // GTEST_HAS_DEATH_TEST implies that we have ::std::string, so we
01683 // can use it here.
01684 int line = -1;
01685 int index = -1;
01686 ::std::vector< ::std::string> fields;
01687 SplitString(GTEST_FLAG_GET(internal_run_death_test), '|', &fields);
01688 int write_fd = -1;
01689
01690 #if GTEST_OS_WINDOWS
01691 unsigned int parent_process_id = 0;
01692 size_t write_handle_as_size_t = 0;
01693 size_t event_handle_as_size_t = 0;
01694
01695 if (fields.size() != 6 || !ParseNaturalNumber(fields[1], &line) ||
01696 !ParseNaturalNumber(fields[2], &index) ||
01697 !ParseNaturalNumber(fields[3], &parent_process_id) ||
01698 !ParseNaturalNumber(fields[4], &write_handle_as_size_t) ||
01699 !ParseNaturalNumber(fields[5], &event_handle_as_size_t)) {
01700 DeathTestAbort("Bad --gtest_internal_run_death_test flag: " +
01701 GTEST_FLAG_GET(internal_run_death_test));
01702 }
01703 write_fd = GetStatusFileDescriptor(parent_process_id, write_handle_as_size_t,
01704 event_handle_as_size_t);
01705 #elif GTEST_OS_FUCHSIA
01706 if (fields.size() != 3 || !ParseNaturalNumber(fields[1], &line) ||
01707 !ParseNaturalNumber(fields[2], &index)) {
01708 DeathTestAbort("Bad --gtest_internal_run_death_test flag: " +
01709 GTEST_FLAG_GET(internal_run_death_test));
01710 }
01711 #else
01712
01713 #endif
01714 }
01715 #else
01716

```

```

01717 if (fields.size() != 4 || !ParseNaturalNumber(fields[1], &line) ||
01718 !ParseNaturalNumber(fields[2], &index) ||
01719 !ParseNaturalNumber(fields[3], &write_fd)) {
01720 DeathTestAbort("Bad --gtest_internal_run_death_test flag: " +
01721 GTEST_FLAG_GET(internal_run_death_test));
01722 }
01723 }
01724 #endif // GTEST_OS_WINDOWS
01725
01726 return new InternalRunDeathTestFlag(fields[0], line, index, write_fd);
01727 }
01728
01729 } // namespace internal
01730
01731 #endif // GTEST_HAS_DEATH_TEST
01732
01733 } // namespace testing

```

## 16.256 gtest-filepath.cc

```

00001 // Copyright 2008, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 #include "gtest/internal/gtest-filepath.h"
00031
00032 #include <stdlib.h>
00033
00034 #include "gtest/gtest-message.h"
00035 #include "gtest/internal/gtest-port.h"
00036
00037 #if GTEST_OS_WINDOWS_MOBILE
00038 #include <windows.h>
00039 #elif GTEST_OS_WINDOWS
00040 #include <direct.h>
00041 #include <io.h>
00042 #else
00043 #include <limits.h>
00044
00045 #include <climits> // Some Linux distributions define PATH_MAX here.
00046 #endif // GTEST_OS_WINDOWS_MOBILE
00047
00048 #include "gtest/internal/gtest-string.h"
00049
00050 #if GTEST_OS_WINDOWS
00051 #define GTEST_PATH_MAX_ _MAX_PATH
00052 #elif defined(PATH_MAX)
00053 #define GTEST_PATH_MAX_ PATH_MAX
00054 #elif defined(_XOPEN_PATH_MAX)
00055 #define GTEST_PATH_MAX_ _XOPEN_PATH_MAX
00056 #else
00057 #define GTEST_PATH_MAX_ _POSIX_PATH_MAX
00058 #endif // GTEST_OS_WINDOWS
00059
00060 namespace testing {
00061 namespace internal {
00062
00063 #if GTEST_OS_WINDOWS
00064 // On Windows, '\\' is the standard path separator, but many tools and the
00065 // Windows API also accept '/' as an alternate path separator. Unless otherwise

```



```

00066 // noted, a file path can contain either kind of path separators, or a mixture
00067 // of them.
00068 const char kPathSeparator = '\\';
00069 const char kAlternatePathSeparator = '/';
00070 const char kAlternatePathSeparatorString[] = "/";
00071 #if GTEST_OS_WINDOWS_MOBILE
00072 // Windows CE doesn't have a current directory. You should not use
00073 // the current directory in tests on Windows CE, but this at least
00074 // provides a reasonable fallback.
00075 const char kCurrentDirectoryString[] = "\\.";
00076 // Windows CE doesn't define INVALID_FILE_ATTRIBUTES
00077 const DWORD kInvalidFileAttributes = 0xffffffff;
00078 #else
00079 const char kCurrentDirectoryString[] = ".";
00080 #endif // GTEST_OS_WINDOWS_MOBILE
00081 #else
00082 const char kPathSeparator = '/';
00083 const char kCurrentDirectoryString[] = ".";
00084 #endif // GTEST_OS_WINDOWS
00085
00086 // Returns whether the given character is a valid path separator.
00087
00088 static bool IsPathSeparator(char c) {
00089 #if GTEST_HAS_ALT_PATH_SEP_
00090 return (c == kPathSeparator) || (c == kAlternatePathSeparator);
00091 #else
00092 return c == kPathSeparator;
00093 #endif
00094 }
00095
00096 // Returns the current working directory, or "" if unsuccessful.
00097
00098 FilePath FilePath::GetCurrentDir() {
00099 #if GTEST_OS_WINDOWS_MOBILE || GTEST_OS_WINDOWS_PHONE || \
00100 GTEST_OS_WINDOWS_RT || GTEST_OS_ESP8266 || GTEST_OS_ESP32 || \
00101 GTEST_OS_XTENSA
00102 // These platforms do not have a current directory, so we just return
00103 // something reasonable.
00104 return FilePath(kCurrentDirectoryString);
00105 #elif GTEST_OS_WINDOWS
00106 char cwd[GTEST_PATH_MAX + 1] = {'\0'};
00107 return FilePath(_getcwd(cwd, sizeof(cwd)) == nullptr ? "" : cwd);
00108 #else
00109 char cwd[GTEST_PATH_MAX + 1] = {'\0'};
00110 char* result = getcwd(cwd, sizeof(cwd));
00111 #if GTEST_OS_NACL
00112 // getcwd will likely fail in NaCl due to the sandbox, so return something
00113 // reasonable. The user may have provided a shim implementation for getcwd,
00114 // however, so fallback only when failure is detected.
00115 return FilePath(result == nullptr ? kCurrentDirectoryString : cwd);
00116 #endif // GTEST_OS_NACL
00117 return FilePath(result == nullptr ? "" : cwd);
00118 #endif // GTEST_OS_WINDOWS_MOBILE
00119 }
00120
00121 // Returns a copy of the FilePath with the case-insensitive extension removed.
00122 // Example: FilePath("dir/file.exe").RemoveExtension("EXE") returns
00123 // FilePath("dir/file"). If a case-insensitive extension is not
00124 // found, returns a copy of the original FilePath.
00125
00126 FilePath FilePath::RemoveExtension(const char* extension) const {
00127 const std::string dot_extension = std::string(".") + extension;
00128 if (String::EndsWithCaseInsensitive(pathname_, dot_extension)) {
00129 return FilePath(
00130 pathname_.substr(0, pathname_.length() - dot_extension.length()));
00131 }
00132 return *this;
00133 }
00134
00135 // Returns a pointer to the last occurrence of a valid path separator in
00136 // the FilePath. On Windows, for example, both '/' and '\' are valid path
00137 // separators. Returns NULL if no path separator was found.
00138
00139 const char* FilePath::FindLastPathSeparator() const {
00140 const char* const last_sep = strrchr(c_str(), kPathSeparator);
00141 #if GTEST_HAS_ALT_PATH_SEP_
00142 const char* const last_alt_sep = strrchr(c_str(), kAlternatePathSeparator);
00143 // Comparing two pointers of which only one is NULL is undefined.
00144 if (last_alt_sep != nullptr &&
00145 (last_sep == nullptr || last_alt_sep > last_sep)) {
00146 return last_alt_sep;
00147 }
00148 #endif
00149 return last_sep;
00150 }
00151
00152 // Returns a copy of the FilePath with the directory part removed.

```

```

00153 // Example: FilePath("path/to/file").RemoveDirectoryName() returns
00154 // FilePath("file"). If there is no directory part ("just_a_file"), it returns
00155 // the FilePath unmodified. If there is no file part ("just_a_dir/") it
00156 // returns an empty FilePath ("").
00157 // On Windows platform, '\\' is the path separator, otherwise it is '/'.
00158
00159 FilePath FilePath::RemoveDirectoryName() const {
00160 const char* const last_sep = FindLastPathSeparator();
00161 return last_sep ? FilePath(last_sep + 1) : *this;
00162 }
00163
00164 // RemoveFileName returns the directory path with the filename removed.
00165 // Example: FilePath("path/to/file").RemoveFileName() returns "path/to/".
00166 // If the FilePath is "a_file" or "/a_file", RemoveFileName returns
00167 // FilePath(".") or, on Windows, FilePath(".\\"). If the filepath does
00168 // not have a file, like "just/a/dir/", it returns the FilePath unmodified.
00169 // On Windows platform, '\\' is the path separator, otherwise it is '/'.
00170
00171 FilePath FilePath::RemoveFileName() const {
00172 const char* const last_sep = FindLastPathSeparator();
00173 std::string dir;
00174 if (last_sep) {
00175 dir = std::string(c_str(), static_cast<size_t> (last_sep + 1 - c_str()));
00176 } else {
00177 dir = kCurrentDirectoryString;
00178 }
00179 return FilePath(dir);
00180 }
00181
00182 // Helper functions for naming files in a directory for xml output.
00183
00184 // Given directory = "dir", base_name = "test", number = 0,
00185 // extension = "xml", returns "dir/test.xml". If number is greater
00186 // than zero (e.g., 12), returns "dir/test_12.xml".
00187 // On Windows platform, uses \ as the separator rather than /.
00188
00189 FilePath FilePath::MakeFileName(const FilePath& directory,
00190 const FilePath& base_name, int number,
00191 const char* extension) {
00192 std::string file;
00193 if (number == 0) {
00194 file = base_name.string() + "." + extension;
00195 } else {
00196 file =
00197 base_name.string() + "_" + StreamableToString(number) + "." + extension;
00198 }
00199 return ConcatPaths(directory, FilePath(file));
00200 }
00201
00202 // Given directory = "dir", relative_path = "test.xml", returns "dir/test.xml".
00203 // On Windows, uses \ as the separator rather than /.
00204
00205 FilePath FilePath::ConcatPaths(const FilePath& directory,
00206 const FilePath& relative_path) {
00207 if (directory.IsEmpty()) return relative_path;
00208 const FilePath dir(directory.RemoveTrailingPathSeparator());
00209 return FilePath(dir.string() + kPathSeparator + relative_path.string());
00210 }
00211
00212 // Returns true if pathname describes something findable in the file-system,
00213 // either a file, directory, or whatever.
00214
00215 bool FilePath::FileOrDirectoryExists() const {
00216 #if GTEST_OS_WINDOWS_MOBILE
00217 LPCWSTR unicode = String::AnsiToUtf16(pathname_.c_str());
00218 const DWORD attributes = GetFileAttributes(unicode);
00219 delete[] unicode;
00220 return attributes != kInvalidFileAttributes;
00221 #else
00222 posix::StatStruct file_stat{};
00223 return posix::Stat(pathname_.c_str(), &file_stat) == 0;
00224 #endif // GTEST_OS_WINDOWS_MOBILE
00225 }
00226
00227 // Returns true if pathname describes a directory in the file-system
00228 // that exists.
00229
00230 bool FilePath::DirectoryExists() const {
00231 bool result = false;
00232 #if GTEST_OS_WINDOWS
00233 // Don't strip off trailing separator if path is a root directory on
00234 // Windows (like "C:\\").
00235 const FilePath & path(IsRootDirectory() ? *this
00236 : RemoveTrailingPathSeparator());
00237 #else
00238 const FilePath & path(*this);
00239 #endif

```

```

00240
00241 #if GTEST_OS_WINDOWS_MOBILE
00242 LPCWSTR unicode = String::AnsiToUtf16(path.c_str());
00243 const DWORD attributes = GetFileAttributes(unicode);
00244 delete[] unicode;
00245 if ((attributes != kInvalidFileAttributes) &&
00246 (attributes & FILE_ATTRIBUTE_DIRECTORY)) {
00247 result = true;
00248 }
00249 #else
00250 posix::StatStruct file_stat{};
00251 result =
00252 posix::Stat(path.c_str(), &file_stat) == 0 && posix::IsDir(file_stat);
00253 #endif // GTEST_OS_WINDOWS_MOBILE
00254
00255 return result;
00256 }
00257
00258 // Returns true if pathname describes a root directory. (Windows has one
00259 // root directory per disk drive.)
00260
00261 bool FilePath::IsRootDirectory() const {
00262 #if GTEST_OS_WINDOWS
00263 return pathname_.length() == 3 && IsAbsolutePath();
00264 #else
00265 return pathname_.length() == 1 && IsPathSeparator(pathname_.c_str()[0]);
00266 #endif
00267 }
00268
00269 // Returns true if pathname describes an absolute path.
00270
00271 bool FilePath::IsAbsolutePath() const {
00272 const char* const name = pathname_.c_str();
00273 #if GTEST_OS_WINDOWS
00274 return pathname_.length() >= 3 &&
00275 ((name[0] >= 'a' && name[0] <= 'z') ||
00276 (name[0] >= 'A' && name[0] <= 'Z')) &&
00277 name[1] == ':' && IsPathSeparator(name[2]);
00278 #else
00279 return IsPathSeparator(name[0]);
00280 #endif
00281 }
00282
00283 // Returns a pathname for a file that does not currently exist. The pathname
00284 // will be directory/base_name.extension or
00285 // directory/base_name.<number>.extension if directory/base_name.extension
00286 // already exists. The number will be incremented until a pathname is found
00287 // that does not already exist.
00288 // Examples: 'dir/foo_test.xml' or 'dir/foo_test_1.xml'.
00289 // There could be a race condition if two or more processes are calling this
00290 // function at the same time -- they could both pick the same filename.
00291
00292 FilePath FilePath::GenerateUniqueFileName(const FilePath& directory,
00293 const FilePath& base_name,
00294 const char* extension) {
00295 FilePath full_pathname;
00296 int number = 0;
00297 do {
00298 full_pathname.Set(MakeFileName(directory, base_name, number++, extension));
00299 } while (full_pathname.FileOrDirectoryExists());
00300 return full_pathname;
00301 }
00302
00303 // Returns true if FilePath ends with a path separator, which indicates that
00304 // it is intended to represent a directory. Returns false otherwise.
00305 // This does NOT check that a directory (or file) actually exists.
00306
00307 bool FilePath::IsDirectory() const {
00308 return !pathname_.empty() &&
00309 IsPathSeparator(pathname_.c_str()[pathname_.length() - 1]);
00310 }
00311
00312 // Create directories so that path exists. Returns true if successful or if
00313 // the directories already exist; returns false if unable to create directories
00314 // for any reason.
00315
00316 bool FilePath::CreateDirectoriesRecursively() const {
00317 if (!this->IsDirectory()) {
00318 return false;
00319 }
00320
00321 if (pathname_.length() == 0 || this->DirectoryExists()) {
00322 return true;
00323 }
00324
00325 const FilePath parent(this->RemoveTrailingPathSeparator().RemoveFileName());
00326 return parent.CreateDirectoriesRecursively() && this->CreateFolder();

```

```

00327 }
00328
00329 // Create the directory so that path exists. Returns true if successful or
00330 // if the directory already exists; returns false if unable to create the
00331 // directory for any reason, including if the parent directory does not
00332 // exist. Not named "CreateDirectory" because that's a macro on Windows.
00333
00334 bool FilePath::CreateFolder() const {
00335 #if GTEST_OS_WINDOWS_MOBILE
00336 FilePath removed_sep(this->RemoveTrailingPathSeparator());
00337 LPCWSTR unicode = String::AnsiToUtf16(removed_sep.c_str());
00338 int result = CreateDirectory(unicode, nullptr) ? 0 : -1;
00339 delete[] unicode;
00340 #elif GTEST_OS_WINDOWS
00341 int result = _mkdir(pathname_.c_str());
00342 #elif GTEST_OS_ESP8266 || GTEST_OS_XTENSA
00343 // do nothing
00344 int result = 0;
00345 #else
00346 int result = mkdir(pathname_.c_str(), 0777);
00347 #endif // GTEST_OS_WINDOWS_MOBILE
00348
00349 if (result == -1) {
00350 return this->DirectoryExists(); // An error is OK if the directory exists.
00351 }
00352 return true; // No error.
00353 }
00354
00355 // If input name has a trailing separator character, remove it and return the
00356 // name, otherwise return the name string unmodified.
00357 // On Windows platform, uses \ as the separator, other platforms use /.
00358
00359 FilePath FilePath::RemoveTrailingPathSeparator() const {
00360 return IsDirectory() ? FilePath(pathname_.substr(0, pathname_.length() - 1))
00361 : *this;
00362 }
00363
00364 // Removes any redundant separators that might be in the pathname.
00365 // For example, "bar///foo" becomes "bar/foo". Does not eliminate other
00366 // redundancies that might be in a pathname involving "." or "..".
00367
00368 void FilePath::Normalize() {
00369 auto out = pathname_.begin();
00370
00371 for (const char character : pathname_) {
00372 if (!IsPathSeparator(character)) {
00373 *(out++) = character;
00374 } else if (out == pathname_.begin() || *std::prev(out) != kPathSeparator) {
00375 *(out++) = kPathSeparator;
00376 } else {
00377 continue;
00378 }
00379 }
00380
00381 pathname_.erase(out, pathname_.end());
00382 }
00383
00384 } // namespace internal
00385 } // namespace testing

```

## 16.257 gtest-internal-inl.h

```

00001 // Copyright 2005, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT

```

```

00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // Utility functions and classes used by the Google C++ testing framework.//
00031 // This file contains purely Google Test's internal implementation. Please
00032 // DO NOT #INCLUDE IT IN A USER PROGRAM.
00033
00034 #ifndef GOOGLETEST_SRC_GTEST_INTERNAL_INL_H_
00035 #define GOOGLETEST_SRC_GTEST_INTERNAL_INL_H_
00036
00037 #ifndef _WIN32_WCE
00038 #include <errno.h>
00039 #endif // !_WIN32_WCE
00040 #include <stddef.h>
00041 #include <stdlib.h> // For strtoll/_strtoul64/malloc/free.
00042 #include <string.h> // For memmove.
00043
00044 #include <algorithm>
00045 #include <cstdint>
00046 #include <memory>
00047 #include <string>
00048 #include <vector>
00049
00050 #include "gtest/internal/gtest-port.h"
00051
00052 #if GTEST_CAN_STREAM_RESULTS_
00053 #include <arpa/inet.h> // NOLINT
00054 #include <netdb.h> // NOLINT
00055 #endif
00056
00057 #if GTEST_OS_WINDOWS
00058 #include <windows.h> // NOLINT
00059 #endif // GTEST_OS_WINDOWS
00060
00061 #include "gtest/gtest-spi.h"
00062 #include "gtest/gtest.h"
00063
00064 GTEST_DISABLE_MSC_WARNINGS_PUSH_(4251 \
00065 /* class A needs to have dll-interface to be used by clients of class B */)
00066
00067 // Declares the flags.
00068 //
00069 // We don't want the users to modify this flag in the code, but want
00070 // Google Test's own unit tests to be able to access it. Therefore we
00071 // declare it here as opposed to in gtest.h.
00072 GTEST_DECLARE_bool_(death_test_use_fork);
00073
00074 namespace testing {
00075 namespace internal {
00076
00077 // The value of GetTestTypeId() as seen from within the Google Test
00078 // library. This is solely for testing GetTestTypeId().
00079 GTEST_API_ extern const TypeId kTestTypeIdInGoogleTest;
00080
00081 // A valid random seed must be in [1, kMaxRandomSeed].
00082 const int kMaxRandomSeed = 99999;
00083
00084 // g_help_flag is true if and only if the --help flag or an equivalent form
00085 // is specified on the command line.
00086 GTEST_API_ extern bool g_help_flag;
00087
00088 // Returns the current time in milliseconds.
00089 GTEST_API_ TimeInMillis GetTimeInMillis();
00090
00091 // Returns true if and only if Google Test should use colors in the output.
00092 GTEST_API_ bool ShouldUseColor(bool stdout_is_tty);
00093
00094 // Formats the given time in milliseconds as seconds.
00095 GTEST_API_ std::string FormatTimeInMillisAsSeconds(TimeInMillis ms);
00096
00097 // Converts the given time in milliseconds to a date string in the ISO 8601
00098 // format, without the timezone information. N.B.: due to the use the
00099 // non-reentrant localtime() function, this function is not thread safe. Do
00100 // not use it in any code that can be called from multiple threads.
00101 GTEST_API_ std::string FormatEpochTimeInMillisAsIso8601(TimeInMillis ms);
00102
00103 // Parses a string for an Int32 flag, in the form of "--flag=value".
00104 //
00105 // On success, stores the value of the flag in *value, and returns
00106 // true. On failure, returns false without changing *value.
00107 GTEST_API_ bool ParseFlag(const char* str, const char* flag, int32_t* value);
00108
00109 // Returns a random seed in range [1, kMaxRandomSeed] based on the
00110 // given --gtest_random_seed flag value.

```

```

00111
00112 inline int GetRandomSeedFromFlag(int32_t random_seed_flag) {
00113 const unsigned int raw_seed =
00114 (random_seed_flag == 0) ? static_cast<unsigned int> (GetTimeInMillis())
00115 : static_cast<unsigned int> (random_seed_flag);
00116
00117 // Normalizes the actual seed to range [1, kMaxRandomSeed] such that
00118 // it's easy to type.
00119 const int normalized_seed =
00120 static_cast<int> ((raw_seed - 1U) %
00121 static_cast<unsigned int> (kMaxRandomSeed)) +
00122 1;
00123 return normalized_seed;
00124 }
00125
00126 // Returns the first valid random seed after 'seed'. The behavior is
00127 // undefined if 'seed' is invalid. The seed after kMaxRandomSeed is
00128 // considered to be 1.
00129
00130 inline int GetNextRandomSeed(int seed) {
00131 GTEST_CHECK_(1 <= seed && seed <= kMaxRandomSeed)
00132 << "Invalid random seed " << seed << " - must be in [1, "
00133 << kMaxRandomSeed << "].";
00134 const int next_seed = seed + 1;
00135 return (next_seed > kMaxRandomSeed) ? 1 : next_seed;
00136 }
00137
00138 // This class saves the values of all Google Test flags in its c'tor, and
00139 // restores them in its d'tor.
00140
00141 class GTestFlagSaver {
00142 public:
00143 // The c'tor.
00144
00145 GTestFlagSaver() {
00146 also_run_disabled_tests_ = GTEST_FLAG_GET(also_run_disabled_tests);
00147 break_on_failure_ = GTEST_FLAG_GET(break_on_failure);
00148 catch_exceptions_ = GTEST_FLAG_GET(catch_exceptions);
00149 color_ = GTEST_FLAG_GET(color);
00150 death_test_style_ = GTEST_FLAG_GET(death_test_style);
00151 death_test_use_fork_ = GTEST_FLAG_GET(death_test_use_fork);
00152 fail_fast_ = GTEST_FLAG_GET(fail_fast);
00153 filter_ = GTEST_FLAG_GET(filter);
00154 internal_run_death_test_ = GTEST_FLAG_GET(internal_run_death_test);
00155 list_tests_ = GTEST_FLAG_GET(list_tests);
00156 output_ = GTEST_FLAG_GET(output);
00157 brief_ = GTEST_FLAG_GET(brief);
00158 print_time_ = GTEST_FLAG_GET(print_time);
00159 print_utf8_ = GTEST_FLAG_GET(print_utf8);
00160 random_seed_ = GTEST_FLAG_GET(random_seed);
00161 repeat_ = GTEST_FLAG_GET(repeat);
00162 recreate_environments_when_repeating_ =
00163 GTEST_FLAG_GET(recreate_environments_when_repeating);
00164 shuffle_ = GTEST_FLAG_GET(shuffle);
00165 stack_trace_depth_ = GTEST_FLAG_GET(stack_trace_depth);
00166 stream_result_to_ = GTEST_FLAG_GET(stream_result_to);
00167 throw_on_failure_ = GTEST_FLAG_GET(throw_on_failure);
00168 }
00169
00170 // The d'tor is not virtual. DO NOT INHERIT FROM THIS CLASS.
00171
00172 ~GTestFlagSaver() {
00173 GTEST_FLAG_SET(also_run_disabled_tests, also_run_disabled_tests_);
00174 GTEST_FLAG_SET(break_on_failure, break_on_failure_);
00175 GTEST_FLAG_SET(catch_exceptions, catch_exceptions_);
00176 GTEST_FLAG_SET(color, color_);
00177 GTEST_FLAG_SET(death_test_style, death_test_style_);
00178 GTEST_FLAG_SET(death_test_use_fork, death_test_use_fork_);
00179 GTEST_FLAG_SET(filter, filter_);
00180 GTEST_FLAG_SET(fail_fast, fail_fast_);
00181 GTEST_FLAG_SET(internal_run_death_test, internal_run_death_test_);
00182 GTEST_FLAG_SET(list_tests, list_tests_);
00183 GTEST_FLAG_SET(output, output_);
00184 GTEST_FLAG_SET(brief, brief_);
00185 GTEST_FLAG_SET(print_time, print_time_);
00186 GTEST_FLAG_SET(print_utf8, print_utf8_);
00187 GTEST_FLAG_SET(random_seed, random_seed_);
00188 GTEST_FLAG_SET(repeat, repeat_);
00189 GTEST_FLAG_SET(recreate_environments_when_repeating,
00190 recreate_environments_when_repeating_);
00191 GTEST_FLAG_SET(shuffle, shuffle_);
00192 GTEST_FLAG_SET(stack_trace_depth, stack_trace_depth_);
00193 GTEST_FLAG_SET(stream_result_to, stream_result_to_);
00194 GTEST_FLAG_SET(throw_on_failure, throw_on_failure_);
00195 }
00196 private:
00197

```

```

00198 // Fields for saving the original values of flags.
00199 bool also_run_disabled_tests_;
00200 bool break_on_failure_;
00201 bool catch_exceptions_;
00202 std::string color_;
00203 std::string death_test_style_;
00204 bool death_test_use_fork_;
00205 bool fail_fast_;
00206 std::string filter_;
00207 std::string internal_run_death_test_;
00208 bool list_tests_;
00209 std::string output_;
00210 bool brief_;
00211 bool print_time_;
00212 bool print_utf8_;
00213 int32_t random_seed_;
00214 int32_t repeat_;
00215 bool recreate_environments_when_repeating_;
00216 bool shuffle_;
00217 int32_t stack_trace_depth_;
00218 std::string stream_result_to_;
00219 bool throw_on_failure_;
00220 } GTEST_ATTRIBUTE_UNUSED_;
00221
00222 // Converts a Unicode code point to a narrow string in UTF-8 encoding.
00223 // code_point parameter is of type UInt32 because wchar_t may not be
00224 // wide enough to contain a code point.
00225 // If the code_point is not a valid Unicode code point
00226 // (i.e. outside of Unicode range U+0 to U+10FFFF) it will be converted
00227 // to "(Invalid Unicode 0xxxxxxx)".
00228 GTEST_API_ std::string CodePointToUtf8(uint32_t code_point);
00229
00230 // Converts a wide string to a narrow string in UTF-8 encoding.
00231 // The wide string is assumed to have the following encoding:
00232 // UTF-16 if sizeof(wchar_t) == 2 (on Windows, Cygwin)
00233 // UTF-32 if sizeof(wchar_t) == 4 (on Linux)
00234 // Parameter str points to a null-terminated wide string.
00235 // Parameter num_chars may additionally limit the number
00236 // of wchar_t characters processed. -1 is used when the entire string
00237 // should be processed.
00238 // If the string contains code points that are not valid Unicode code points
00239 // (i.e. outside of Unicode range U+0 to U+10FFFF) they will be output
00240 // as '(Invalid Unicode 0xxxxxxx)'. If the string is in UTF16 encoding
00241 // and contains invalid UTF-16 surrogate pairs, values in those pairs
00242 // will be encoded as individual Unicode characters from Basic Normal Plane.
00243 GTEST_API_ std::string WideStringToUtf8(const wchar_t* str, int num_chars);
00244
00245 // Reads the GTEST_SHARD_STATUS_FILE environment variable, and creates the file
00246 // if the variable is present. If a file already exists at this location, this
00247 // function will write over it. If the variable is present, but the file cannot
00248 // be created, prints an error and exits.
00249 void WriteToShardStatusFileIfNeeded();
00250
00251 // Checks whether sharding is enabled by examining the relevant
00252 // environment variable values. If the variables are present,
00253 // but inconsistent (e.g., shard_index >= total_shards), prints
00254 // an error and exits. If in_subprocess_for_death_test, sharding is
00255 // disabled because it must only be applied to the original test
00256 // process. Otherwise, we could filter out death tests we intended to execute.
00257 GTEST_API_ bool ShouldShard(const char* total_shards_str,
00258 const char* shard_index_str,
00259 bool in_subprocess_for_death_test);
00260
00261 // Parses the environment variable var as a 32-bit integer. If it is unset,
00262 // returns default_val. If it is not a 32-bit integer, prints an error and
00263 // aborts.
00264 GTEST_API_ int32_t Int32FromEnvOrDie(const char* env_var, int32_t default_val);
00265
00266 // Given the total number of shards, the shard index, and the test id,
00267 // returns true if and only if the test should be run on this shard. The test id
00268 // is some arbitrary but unique non-negative integer assigned to each test
00269 // method. Assumes that 0 <= shard_index < total_shards.
00270 GTEST_API_ bool ShouldRunTestOnShard(int total_shards, int shard_index,
00271 int test_id);
00272
00273 // STL container utilities.
00274
00275 // Returns the number of elements in the given container that satisfy
00276 // the given predicate.
00277
00278 template <class Container, typename Predicate>
00279 inline int CountIf(const Container& c, Predicate predicate) {
00280 // Implemented as an explicit loop since std::count_if() in libCstd on
00281 // Solaris has a non-standard signature.
00282 int count = 0;
00283 for (auto it = c.begin(); it != c.end(); ++it) {
00284 if (predicate(*it)) ++count;

```

```

00285 }
00286 return count;
00287 }
00288
00289 // Applies a function/functor to each element in the container.
00290
00291 template <class Container, typename Functor>
00292 void ForEach(const Container& c, Functor functor) {
00293 std::for_each(c.begin(), c.end(), functor);
00294 }
00295
00296 // Returns the i-th element of the vector, or default_value if i is not
00297 // in range [0, v.size()).
00298
00299 template <typename E>
00300 inline E GetElementOr(const std::vector<E>& v, int i, E default_value) {
00301 return (i < 0 || i >= static_cast<int>(v.size())) ? default_value
00302 : v[static_cast<size_t>(i)];
00303 }
00304
00305 // Performs an in-place shuffle of a range of the vector's elements.
00306 // 'begin' and 'end' are element indices as an STL-style range;
00307 // i.e. [begin, end) are shuffled, where 'end' == size() means to
00308 // shuffle to the end of the vector.
00309
00310 template <typename E>
00311 void ShuffleRange(internal::Random* random, int begin, int end,
00312 std::vector<E>* v) {
00313 const int size = static_cast<int>(v->size());
00314 GTEST_CHECK_(0 <= begin && begin <= size)
00315 << "Invalid shuffle range start " << begin << ": must be in range [0, "
00316 << size << "].";
00317 GTEST_CHECK_(begin <= end && end <= size)
00318 << "Invalid shuffle range finish " << end << ": must be in range ["
00319 << begin << ", " << size << "].";
00320
00321 // Fisher-Yates shuffle, from
00322 // http://en.wikipedia.org/wiki/Fisher-Yates_shuffle
00323 for (int range_width = end - begin; range_width >= 2; range_width--) {
00324 const int last_in_range = begin + range_width - 1;
00325 const int selected =
00326 begin +
00327 static_cast<int>(random->Generate(static_cast<uint32_t>(range_width)));
00328 std::swap((*v)[static_cast<size_t>(selected)],
00329 (*v)[static_cast<size_t>(last_in_range)]);
00330 }
00331 }
00332
00333 // Performs an in-place shuffle of the vector's elements.
00334
00335 template <typename E>
00336 inline void Shuffle(internal::Random* random, std::vector<E>* v) {
00337 ShuffleRange(random, 0, static_cast<int>(v->size()), v);
00338 }
00339
00340 // A function for deleting an object. Handy for being used as a
00341 // functor.
00342
00343 template <typename T>
00344 static void Delete(T* x) {
00345 delete x;
00346 }
00347
00348 // A predicate that checks the key of a TestProperty against a known key.
00349 //
00350 // TestPropertyKeyIs is copyable.
00351
00352 class TestPropertyKeyIs {
00353 public:
00354 // Constructor.
00355 //
00356 // TestPropertyKeyIs has NO default constructor.
00357
00358 explicit TestPropertyKeyIs(const std::string& key) : key_(key) {}
00359 }
00360
00361 // Returns true if and only if the test name of test property matches on key_.
00362
00363 bool operator()(const TestProperty& test_property) const {
00364 return test_property.key() == key_;
00365 }
00366
00367 private:
00368 std::string key_;
00369 };
00370
00371 // Class UnitTestOptions.

```



```

00372 //
00373 // This class contains functions for processing options the user
00374 // specifies when running the tests. It has only static members.
00375 //
00376 // In most cases, the user can specify an option using either an
00377 // environment variable or a command line flag. E.g. you can set the
00378 // test filter using either GTEST_FILTER or --gtest_filter. If both
00379 // the variable and the flag are present, the latter overrides the
00380 // former.
00381
00382 class GTEST_API_ UnitTestOptions {
00383 public:
00384 // Functions for processing the gtest_output flag.
00385
00386 // Returns the output format, or "" for normal printed output.
00387 static std::string GetOutputFormat();
00388
00389 // Returns the absolute path of the requested output file, or the
00390 // default (test_detail.xml in the original working directory) if
00391 // none was explicitly specified.
00392 static std::string GetAbsolutePathToOutputFile();
00393
00394 // Functions for processing the gtest_filter flag.
00395
00396 // Returns true if and only if the user-specified filter matches the test
00397 // suite name and the test name.
00398 static bool FilterMatchesTest(const std::string& test_suite_name,
00399 const std::string& test_name);
00400
00401 #if GTEST_OS_WINDOWS
00402 // Function for supporting the gtest_catch_exception flag.
00403
00404 // Returns EXCEPTION_EXECUTE_HANDLER if Google Test should handle the
00405 // given SEH exception, or EXCEPTION_CONTINUE_SEARCH otherwise.
00406 // This function is useful as an __except condition.
00407 static int GTestShouldProcessSEH(DWORD exception_code);
00408 #endif // GTEST_OS_WINDOWS
00409
00410 // Returns true if "name" matches the ':' separated list of glob-style
00411 // filters in "filter".
00412 static bool MatchesFilter(const std::string& name, const char* filter);
00413 };
00414
00415 // Returns the current application's name, removing directory path if that
00416 // is present. Used by UnitTestOptions::GetOutputFile.
00417 GTEST_API_ FilePath GetCurrentExecutableName();
00418
00419 // The role interface for getting the OS stack trace as a string.
00420
00421 class OsStackTraceGetterInterface {
00422 public:
00423
00424 OsStackTraceGetterInterface() {}
00425
00426 virtual ~OsStackTraceGetterInterface() {}
00427
00428 // Returns the current OS stack trace as an std::string. Parameters:
00429 // max_depth - the maximum number of stack frames to be included
00430 // in the trace.
00431 // skip_count - the number of top frames to be skipped; doesn't count
00432 // against max_depth.
00433 virtual std::string CurrentStackTrace(int max_depth, int skip_count) = 0;
00434
00435 // UponLeavingGTest() should be called immediately before Google Test calls
00436 // user code. It saves some information about the current stack that
00437 // CurrentStackTrace() will use to find and hide Google Test stack frames.
00438 virtual void UponLeavingGTest() = 0;
00439
00440 // This string is inserted in place of stack frames that are part of
00441 // Google Test's implementation.
00442 static const char* const kElidedFramesMarker;
00443
00444 private:
00445 OsStackTraceGetterInterface(const OsStackTraceGetterInterface&) = delete;
00446 OsStackTraceGetterInterface& operator=(const OsStackTraceGetterInterface&) =
00447 delete;
00448 };
00449
00450 // A working implementation of the OsStackTraceGetterInterface interface.
00451
00452 class OsStackTraceGetter : public OsStackTraceGetterInterface {
00453 public:
00454
00455 OsStackTraceGetter() {}

```

```

00459 }
00460
00461 std::string CurrentStackTrace(int max_depth, int skip_count) override;
00462 void UponLeavingGTest() override;
00463
00464 private:
00465 #if GTEST_HAS_ABSL
00466 Mutex mutex_; // Protects all internal state.
00467
00468 // We save the stack frame below the frame that calls user code.
00469 // We do this because the address of the frame immediately below
00470 // the user code changes between the call to UponLeavingGTest()
00471 // and any calls to the stack trace code from within the user code.
00472 void* caller_frame_ = nullptr;
00473 #endif // GTEST_HAS_ABSL
00474
00475 OsStackTraceGetter(const OsStackTraceGetter&) = delete;
00476 OsStackTraceGetter& operator=(const OsStackTraceGetter&) = delete;
00477 };
00478
00479 // Information about a Google Test trace point.
00480
00481 struct TraceInfo {
00482 const char* file;
00483 int line;
00484 std::string message;
00485 };
00486
00487 // This is the default global test part result reporter used in UnitTestImpl.
00488 // This class should only be used by UnitTestImpl.
00489
00490 class DefaultGlobalTestPartResultReporter
00491 : public TestPartResultReporterInterface {
00492 public:
00493 explicit DefaultGlobalTestPartResultReporter(UnitTestImpl* unit_test);
00494 // Implements the TestPartResultReporterInterface. Reports the test part
00495 // result in the current test.
00496 void ReportTestPartResult(const TestPartResult& result) override;
00497
00498 private:
00499 UnitTestImpl * const unit_test_;
00500
00501 DefaultGlobalTestPartResultReporter(
00502 const DefaultGlobalTestPartResultReporter&) = delete;
00503 DefaultGlobalTestPartResultReporter& operator=(
00504 const DefaultGlobalTestPartResultReporter&) = delete;
00505 };
00506
00507 // This is the default per thread test part result reporter used in
00508 // UnitTestImpl. This class should only be used by UnitTestImpl.
00509
00510 class DefaultPerThreadTestPartResultReporter
00511 : public TestPartResultReporterInterface {
00512 public:
00513 explicit DefaultPerThreadTestPartResultReporter(UnitTestImpl* unit_test);
00514 // Implements the TestPartResultReporterInterface. The implementation just
00515 // delegates to the current global test part result reporter of *unit_test_.
00516 void ReportTestPartResult(const TestPartResult& result) override;
00517
00518 private:
00519 UnitTestImpl * const unit_test_;
00520
00521 DefaultPerThreadTestPartResultReporter(
00522 const DefaultPerThreadTestPartResultReporter&) = delete;
00523 DefaultPerThreadTestPartResultReporter& operator=(
00524 const DefaultPerThreadTestPartResultReporter&) = delete;
00525 };
00526
00527 // The private implementation of the UnitTest class. We don't protect
00528 // the methods under a mutex, as this class is not accessible by a
00529 // user and the UnitTest class that delegates work to this class does
00530 // proper locking.
00531
00532 class GTEST_API_ UnitTestImpl {
00533 public:
00534 explicit UnitTestImpl(UnitTest* parent);
00535 virtual ~UnitTestImpl();
00536
00537 // There are two different ways to register your own TestPartResultReporter.
00538 // You can register your own reporter to listen either only for test results
00539 // from the current thread or for results from all threads.
00540 // By default, each per-thread test result reporter just passes a new
00541 // TestPartResult to the global test result reporter, which registers the
00542 // test part result for the currently running test.
00543
00544 // Returns the global test part result reporter.
00545 TestPartResultReporterInterface* GetGlobalTestPartResultReporter();

```

```

00546
00547 // Sets the global test part result reporter.
00548 void SetGlobalTestPartResultReporter(
00549 TestPartResultReporterInterface* reporter);
00550
00551 // Returns the test part result reporter for the current thread.
00552 TestPartResultReporterInterface* GetTestPartResultReporterForCurrentThread();
00553
00554 // Sets the test part result reporter for the current thread.
00555 void SetTestPartResultReporterForCurrentThread(
00556 TestPartResultReporterInterface* reporter);
00557
00558 // Gets the number of successful test suites.
00559 int successful_test_suite_count() const;
00560
00561 // Gets the number of failed test suites.
00562 int failed_test_suite_count() const;
00563
00564 // Gets the number of all test suites.
00565 int total_test_suite_count() const;
00566
00567 // Gets the number of all test suites that contain at least one test
00568 // that should run.
00569 int test_suite_to_run_count() const;
00570
00571 // Gets the number of successful tests.
00572 int successful_test_count() const;
00573
00574 // Gets the number of skipped tests.
00575 int skipped_test_count() const;
00576
00577 // Gets the number of failed tests.
00578 int failed_test_count() const;
00579
00580 // Gets the number of disabled tests that will be reported in the XML report.
00581 int reportable_disabled_test_count() const;
00582
00583 // Gets the number of disabled tests.
00584 int disabled_test_count() const;
00585
00586 // Gets the number of tests to be printed in the XML report.
00587 int reportable_test_count() const;
00588
00589 // Gets the number of all tests.
00590 int total_test_count() const;
00591
00592 // Gets the number of tests that should run.
00593 int test_to_run_count() const;
00594
00595 // Gets the time of the test program start, in ms from the start of the
00596 // UNIX epoch.
00597
00598 TimeInMillis start_timestamp() const {
00599 return start_timestamp_;
00600 }
00601
00602 // Gets the elapsed time, in milliseconds.
00603
00604 TimeInMillis elapsed_time() const {
00605 return elapsed_time_;
00606 }
00607
00608 // Returns true if and only if the unit test passed (i.e. all test suites
00609 // passed).
00610
00611 bool Passed() const {
00612 return !Failed();
00613 }
00614
00615 // Returns true if and only if the unit test failed (i.e. some test suite
00616 // failed or something outside of all tests failed).
00617
00618 bool Failed() const {
00619 return failed_test_suite_count() > 0 || ad_hoc_test_result()->Failed();
00620 }
00621
00622 // Gets the i-th test suite among all the test suites. i can range from 0 to
00623 // total_test_suite_count() - 1. If i is not in that range, returns NULL.
00624
00625 const TestSuite* GetTestSuite(int i) const {
00626 const int index = GetElementOr(test_suite_indices_, i, -1);
00627 return index < 0 ? nullptr : test_suites_[static_cast<size_t> (i)];
00628 }
00629
00630 // Legacy API is deprecated but still available
00631 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00632

```

```

00633 const TestCase* GetTestCase(int i) const {
00634 return GetTestSuite(i);
00635 }
00636 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00637
00638 // Gets the i-th test suite among all the test suites. i can range from 0 to
00639 // total_test_suite_count() - 1. If i is not in that range, returns NULL.
00640
00641 TestSuite* GetMutableSuiteCase(int i) {
00642 const int index = GetElementOr(test_suite_indices_, i, -1);
00643 return index < 0 ? nullptr : test_suites_[static_cast<size_t> (index)];
00644 }
00645
00646 // Provides access to the event listener list.
00647
00648 TestEventListeners* listeners() {
00649 return &listeners_;
00650 }
00651
00652 // Returns the TestResult for the test that's currently running, or
00653 // the TestResult for the ad hoc test if no test is running.
00654 TestResult* current_test_result();
00655
00656 // Returns the TestResult for the ad hoc test.
00657
00658 const TestResult* ad_hoc_test_result() const {
00659 return &ad_hoc_test_result_;
00660 }
00661
00662 // Sets the OS stack trace getter.
00663 //
00664 // Does nothing if the input and the current OS stack trace getter
00665 // are the same; otherwise, deletes the old getter and makes the
00666 // input the current getter.
00667 void set_os_stack_trace_getter(OsStackTraceGetterInterface* getter);
00668
00669 // Returns the current OS stack trace getter if it is not NULL;
00670 // otherwise, creates an OsStackTraceGetter, makes it the current
00671 // getter, and returns it.
00672 OsStackTraceGetterInterface* os_stack_trace_getter();
00673
00674 // Returns the current OS stack trace as an std::string.
00675 //
00676 // The maximum number of stack frames to be included is specified by
00677 // the gtest_stack_trace_depth flag. The skip_count parameter
00678 // specifies the number of top frames to be skipped, which doesn't
00679 // count against the number of frames to be included.
00680 //
00681 // For example, if Foo() calls Bar(), which in turn calls
00682 // CurrentOsStackTraceExceptTop(1), Foo() will be included in the
00683 // trace but Bar() and CurrentOsStackTraceExceptTop() won't.
00684 std::string CurrentOsStackTraceExceptTop(int skip_count)
00685 GTEST_NO_INLINE_ GTEST_NO_TAIL_CALL_;
00686
00687 // Finds and returns a TestSuite with the given name. If one doesn't
00688 // exist, creates one and returns it.
00689 //
00690 // Arguments:
00691 // test_suite_name: name of the test suite
00692 // type_param: the name of the test's type parameter, or NULL if
00693 // this is not a typed or a type-parameterized test.
00694 // set_up_tc: pointer to the function that sets up the test suite
00695 // tear_down_tc: pointer to the function that tears down the test suite
00696 TestSuite* GetTestSuite(const char* test_suite_name, const char* type_param,
00697 internal::SetUpTestSuiteFunc set_up_tc,
00698 internal::TearDownTestSuiteFunc tear_down_tc);
00699
00700 // Legacy API is deprecated but still available
00701 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00702 TestCase* GetTestCase(const char* test_case_name, const char* type_param,
00703 internal::SetUpTestSuiteFunc set_up_tc,
00704 internal::TearDownTestSuiteFunc tear_down_tc) {
00705 return GetTestSuite(test_case_name, type_param, set_up_tc, tear_down_tc);
00706 }
00707 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
00708
00709 // Adds a TestInfo to the unit test.
00710 //
00711 // Arguments:
00712 // set_up_tc: pointer to the function that sets up the test suite
00713 // tear_down_tc: pointer to the function that tears down the test suite
00714 // test_info: the TestInfo object
00715 void AddTestInfo(internal::SetUpTestSuiteFunc set_up_tc,

```

```

00720 internal::TearDownTestSuiteFunc tear_down_tc,
00721 TestInfo* test_info) {
00722 #if GTEST_HAS_DEATH_TEST
00723 // In order to support thread-safe death tests, we need to
00724 // remember the original working directory when the test program
00725 // was first invoked. We cannot do this in RUN_ALL_TESTS(), as
00726 // the user may have changed the current directory before calling
00727 // RUN_ALL_TESTS(). Therefore we capture the current directory in
00728 // AddTestInfo(), which is called to register a TEST or TEST_F
00729 // before main() is reached.
00730 if (original_working_dir_.IsEmpty()) {
00731 original_working_dir_.Set(FilePath::GetCurrentDir());
00732 GTEST_CHECK_(!original_working_dir_.IsEmpty())
00733 << "Failed to get the current working directory.";
00734 }
00735 #endif // GTEST_HAS_DEATH_TEST
00736
00737 GetTestSuite(test_info->test_suite_name(), test_info->type_param(),
00738 set_up_tc, tear_down_tc)
00739 ->AddTestInfo(test_info);
00740 }
00741
00742 // Returns ParameterizedTestSuiteRegistry object used to keep track of
00743 // value-parameterized tests and instantiate and register them.
00744
00745 internal::ParameterizedTestSuiteRegistry& parameterized_test_registry() {
00746 return parameterized_test_registry_;
00747 }
00748
00749 std::set<std::string>* ignored_parameterized_test_suites() {
00750 return &ignored_parameterized_test_suites_;
00751 }
00752
00753 // Returns TypeParameterizedTestSuiteRegistry object used to keep track of
00754 // type-parameterized tests and instantiations of them.
00755
00756 internal::TypeParameterizedTestSuiteRegistry&
00757 type_parameterized_test_registry() {
00758 return type_parameterized_test_registry_;
00759 }
00760
00761 // Sets the TestSuite object for the test that's currently running.
00762
00763 void set_current_test_suite(TestSuite* a_current_test_suite) {
00764 current_test_suite_ = a_current_test_suite;
00765 }
00766
00767 // Sets the TestInfo object for the test that's currently running. If
00768 // current_test_info is NULL, the assertion results will be stored in
00769 // ad_hoc_test_result_.
00770
00771 void set_current_test_info(TestInfo* a_current_test_info) {
00772 current_test_info_ = a_current_test_info;
00773 }
00774
00775 // Registers all parameterized tests defined using TEST_P and
00776 // INSTANTIATE_TEST_SUITE_P, creating regular tests for each test/parameter
00777 // combination. This method can be called more than once; it has guards
00778 // protecting from registering the tests more than once. If
00779 // value-parameterized tests are disabled, RegisterParameterizedTests is
00780 // present but does nothing.
00781 void RegisterParameterizedTests();
00782
00783 // Runs all tests in this UnitTest object, prints the result, and
00784 // returns true if all tests are successful. If any exception is
00785 // thrown during a test, this test is considered to be failed, but
00786 // the rest of the tests will still be run.
00787 bool RunAllTests();
00788
00789 // Clears the results of all tests, except the ad hoc tests.
00790
00791 void ClearNonAdHocTestResult() {
00792 ForEach(test_suites_, TestSuite::ClearTestSuiteResult);
00793 }
00794
00795 // Clears the results of ad-hoc test assertions.
00796
00797 void ClearAdHocTestResult() {
00798 ad_hoc_test_result_.Clear();
00799 }
00800
00801 // Adds a TestProperty to the current TestResult object when invoked in a
00802 // context of a test or a test suite, or to the global property set. If the
00803 // result already contains a property with the same key, the value will be
00804 // updated.
00805 void RecordProperty(const TestProperty& test_property);
00806

```

```

00807 enum ReactionToSharding {
00808 HONOR_SHARDING_PROTOCOL, IGNORE_SHARDING_PROTOCOL
00809 };
00810
00811 // Matches the full name of each test against the user-specified
00812 // filter to decide whether the test should run, then records the
00813 // result in each TestSuite and TestInfo object.
00814 // If shard_tests == HONOR_SHARDING_PROTOCOL, further filters tests
00815 // based on sharding variables in the environment.
00816 // Returns the number of tests that should run.
00817 int FilterTests(ReactionToSharding shard_tests);
00818
00819 // Prints the names of the tests matching the user-specified filter flag.
00820 void ListTestsMatchingFilter();
00821
00822 const TestSuite* current_test_suite() const {
00823 return current_test_suite_;
00824 }
00825
00826 TestInfo* current_test_info() {
00827 return current_test_info_;
00828 }
00829
00830 const TestInfo* current_test_info() const {
00831 return current_test_info_;
00832 }
00833
00834 // Returns the vector of environments that need to be set-up/torn-down
00835 // before/after the tests are run.
00836
00837 std::vector<Environment*>& environments() {
00838 return environments_;
00839 }
00840
00841 // Getters for the per-thread Google Test trace stack.
00842
00843 std::vector<TraceInfo>& gtest_trace_stack() {
00844 return *(gtest_trace_stack_.pointer());
00845 }
00846
00847 const std::vector<TraceInfo>& gtest_trace_stack() const {
00848 return gtest_trace_stack_.get();
00849 }
00850
00851 #if GTEST_HAS_DEATH_TEST
00852
00853 void InitDeathTestSubprocessControlInfo() {
00854 internal_run_death_test_flag_.reset(ParseInternalRunDeathTestFlag());
00855 }
00856 // Returns a pointer to the parsed --gtest_internal_run_death_test
00857 // flag, or NULL if that flag was not specified.
00858 // This information is useful only in a death test child process.
00859 // Must not be called before a call to InitGoogleTest.
00860
00861 const InternalRunDeathTestFlag* internal_run_death_test_flag() const {
00862 return internal_run_death_test_flag_.get();
00863 }
00864
00865 // Returns a pointer to the current death test factory.
00866
00867 internal::DeathTestFactory* death_test_factory() {
00868 return death_test_factory_.get();
00869 }
00870
00871 void SuppressTestEventsIfInSubprocess();
00872
00873 friend class ReplaceDeathTestFactory;
00874 #endif // GTEST_HAS_DEATH_TEST
00875
00876 // Initializes the event listener performing XML output as specified by
00877 // UnitTestOptions. Must not be called before InitGoogleTest.
00878 void ConfigureXmlOutput();
00879
00880 #if GTEST_CAN_STREAM_RESULTS_
00881 // Initializes the event listener for streaming test results to a socket.
00882 // Must not be called before InitGoogleTest.
00883 void ConfigureStreamingOutput();
00884 #endif
00885
00886 // Performs initialization dependent upon flag values obtained in
00887 // ParseGoogleTestFlagsOnly. Is called from InitGoogleTest after the call to
00888 // ParseGoogleTestFlagsOnly. In case a user neglects to call InitGoogleTest
00889 // this function is also called from RunAllTests. Since this function can be
00890 // called more than once, it has to be idempotent.
00891 void PostFlagParsingInit();
00892
00893 // Gets the random seed used at the start of the current test iteration.

```

```

00894
00895 int random_seed() const {
00896 return random_seed_;
00897 }
00898
00899 // Gets the random number generator.
00900
00901 internal::Random* random() {
00902 return &random_;
00903 }
00904
00905 // Shuffles all test suites, and the tests within each test suite,
00906 // making sure that death tests are still run first.
00907 void ShuffleTests();
00908
00909 // Restores the test suites and tests to their order before the first shuffle.
00910 void UnshuffleTests();
00911
00912 // Returns the value of GTEST_FLAG(catch_exceptions) at the moment
00913 // UnitTest::Run() starts.
00914
00915 bool catch_exceptions() const {
00916 return catch_exceptions_;
00917 }
00918
00919 private:
00920 friend class ::testing::UnitTest;
00921
00922 // Used by UnitTest::Run() to capture the state of
00923 // GTEST_FLAG(catch_exceptions) at the moment it starts.
00924
00925 void set_catch_exceptions(bool value) {
00926 catch_exceptions_ = value;
00927 }
00928
00929 // The UnitTest object that owns this implementation object.
00930 UnitTest * const parent_;
00931
00932 // The working directory when the first TEST() or TEST_F() was
00933 // executed.
00934 internal::FilePath original_working_dir_;
00935
00936 // The default test part result reporters.
00937 DefaultGlobalTestPartResultReporter default_global_test_part_result_reporter_;
00938 DefaultPerThreadTestPartResultReporter
00939 default_per_thread_test_part_result_reporter_;
00940
00941 // Points to (but doesn't own) the global test part result reporter.
00942 TestPartResultReporterInterface* global_test_part_result_reporter_;
00943
00944 // Protects read and write access to global_test_part_result_reporter_.
00945 internal::Mutex global_test_part_result_reporter_mutex_;
00946
00947 // Points to (but doesn't own) the per-thread test part result reporter.
00948 internal::ThreadLocal<TestPartResultReporterInterface*>
00949 per_thread_test_part_result_reporter_;
00950
00951 // The vector of environments that need to be set-up/torn-down
00952 // before/after the tests are run.
00953 std::vector<Environment*> environments_;
00954
00955 // The vector of TestSuites in their original order. It owns the
00956 // elements in the vector.
00957 std::vector<TestSuite*> test_suites_;
00958
00959 // Provides a level of indirection for the test suite list to allow
00960 // easy shuffling and restoring the test suite order. The i-th
00961 // element of this vector is the index of the i-th test suite in the
00962 // shuffled order.
00963 std::vector<int> test_suite_indices_;
00964
00965 // ParameterizedTestRegistry object used to register value-parameterized
00966 // tests.
00967 internal::ParameterizedTestSuiteRegistry parameterized_test_registry_;
00968 internal::TypeParameterizedTestSuiteRegistry
00969 type_parameterized_test_registry_;
00970
00971 // The set holding the name of parameterized
00972 // test suites that may go uninstantiated.
00973 std::set<std::string> ignored_parameterized_test_suites_;
00974
00975 // Indicates whether RegisterParameterizedTests() has been called already.
00976 bool parameterized_tests_registered_;
00977
00978 // Index of the last death test suite registered. Initially -1.
00979 int last_death_test_suite_;
00980

```

```

00981 // This points to the TestSuite for the currently running test. It
00982 // changes as Google Test goes through one test suite after another.
00983 // When no test is running, this is set to NULL and Google Test
00984 // stores assertion results in ad_hoc_test_result_. Initially NULL.
00985 TestSuite* current_test_suite_;
00986
00987 // This points to the TestInfo for the currently running test. It
00988 // changes as Google Test goes through one test after another. When
00989 // no test is running, this is set to NULL and Google Test stores
00990 // assertion results in ad_hoc_test_result_. Initially NULL.
00991 TestInfo* current_test_info_;
00992
00993 // Normally, a user only writes assertions inside a TEST or TEST_F,
00994 // or inside a function called by a TEST or TEST_F. Since Google
00995 // Test keeps track of which test is current running, it can
00996 // associate such an assertion with the test it belongs to.
00997 //
00998 // If an assertion is encountered when no TEST or TEST_F is running,
00999 // Google Test attributes the assertion result to an imaginary "ad hoc"
01000 // test, and records the result in ad_hoc_test_result_.
01001 TestResult ad_hoc_test_result_;
01002
01003 // The list of event listeners that can be used to track events inside
01004 // Google Test.
01005 TestEventListeners listeners_;
01006
01007 // The OS stack trace getter. Will be deleted when the UnitTest
01008 // object is destructed. By default, an OsStackTraceGetter is used,
01009 // but the user can set this field to use a custom getter if that is
01010 // desired.
01011 OsStackTraceGetterInterface* os_stack_trace_getter_;
01012
01013 // True if and only if PostFlagParsingInit() has been called.
01014 bool post_flag_parse_init_performed_;
01015
01016 // The random number seed used at the beginning of the test run.
01017 int random_seed_;
01018
01019 // Our random number generator.
01020 internal::Random random_;
01021
01022 // The time of the test program start, in ms from the start of the
01023 // UNIX epoch.
01024 TimeInMillis start_timestamp_;
01025
01026 // How long the test took to run, in milliseconds.
01027 TimeInMillis elapsed_time_;
01028
01029 #if GTEST_HAS_DEATH_TEST
01030 // The decomposed components of the gtest_internal_run_death_test flag,
01031 // parsed when RUN_ALL_TESTS is called.
01032 std::unique_ptr<InternalRunDeathTestFlag> internal_run_death_test_flag_;
01033 std::unique_ptr<internal::DeathTestFactory> death_test_factory_;
01034 #endif // GTEST_HAS_DEATH_TEST
01035
01036 // A per-thread stack of traces created by the SCOPED_TRACE() macro.
01037 internal::ThreadLocal<std::vector<TraceInfo> > gtest_trace_stack_;
01038
01039 // The value of GTEST_FLAG(catch_exceptions) at the moment RunAllTests()
01040 // starts.
01041 bool catch_exceptions_;
01042
01043 UnitTestImpl(const UnitTestImpl&) = delete;
01044 UnitTestImpl& operator=(const UnitTestImpl&) = delete;
01045 }; // class UnitTestImpl
01046
01047 // Convenience function for accessing the global UnitTest
01048 // implementation object.
01049
01050 inline UnitTestImpl* GetUnitTestImpl() {
01051 return UnitTest::GetInstance()->impl();
01052 }
01053
01054 #if GTEST_USES_SIMPLE_RE
01055
01056 // Internal helper functions for implementing the simple regular
01057 // expression matcher.
01058 GTEST_API_ bool IsInSet(char ch, const char* str);
01059 GTEST_API_ bool IsAsciiDigit(char ch);
01060 GTEST_API_ bool IsAsciiPunct(char ch);
01061 GTEST_API_ bool IsRepeat(char ch);
01062 GTEST_API_ bool IsAsciiWhiteSpace(char ch);
01063 GTEST_API_ bool IsAsciiWordChar(char ch);
01064 GTEST_API_ bool IsValidEscape(char ch);
01065 GTEST_API_ bool AtomMatchesChar(bool escaped, char pattern, char ch);
01066 GTEST_API_ bool ValidateRegex(const char* regex);
01067 GTEST_API_ bool MatchRegexAtHead(const char* regex, const char* str);

```



```

01068 GTEST_API_ bool MatchRepetitionAndRegexAtHead(bool escaped, char ch,
01069 char repeat, const char* regex,
01070 const char* str);
01071 GTEST_API_ bool MatchRegexAnywhere(const char* regex, const char* str);
01072
01073 #endif // GTEST_USES_SIMPLE_RE
01074
01075 // Parses the command line for Google Test flags, without initializing
01076 // other parts of Google Test.
01077 GTEST_API_ void ParseGoogleTestFlagsOnly(int* argc, char** argv);
01078 GTEST_API_ void ParseGoogleTestFlagsOnly(int* argc, wchar_t** argv);
01079
01080 #if GTEST_HAS_DEATH_TEST
01081
01082 // Returns the message describing the last system error, regardless of the
01083 // platform.
01084 GTEST_API_ std::string GetLastErrnoDescription();
01085
01086 // Attempts to parse a string into a positive integer pointed to by the
01087 // number parameter. Returns true if that is possible.
01088 // GTEST_HAS_DEATH_TEST implies that we have ::std::string, so we can use
01089 // it here.
01090
01091 template <typename Integer>
01092 bool ParseNaturalNumber(const ::std::string& str, Integer* number) {
01093 // Fail fast if the given string does not begin with a digit;
01094 // this bypasses strtoul's "optional leading whitespace and plus
01095 // or minus sign" semantics, which are undesirable here.
01096 if (str.empty() || !IsDigit(str[0])) {
01097 return false;
01098 }
01099 errno = 0;
01100
01101 char* end;
01102 // BiggestConvertible is the largest integer type that system-provided
01103 // string-to-number conversion routines can return.
01104 using BiggestConvertible = unsigned long long; // NOLINT
01105
01106 const BiggestConvertible parsed = strtoull(str.c_str(), &end, 10); // NOLINT
01107 const bool parse_success = *end == '\0' && errno == 0;
01108
01109 GTEST_CHECK_(sizeof (Integer) <= sizeof (parsed));
01110
01111 const Integer result = static_cast<Integer> (parsed);
01112 if (parse_success && static_cast<BiggestConvertible> (result) == parsed) {
01113 *number = result;
01114 return true;
01115 }
01116 return false;
01117 }
01118 #endif // GTEST_HAS_DEATH_TEST
01119
01120 // TestResult contains some private methods that should be hidden from
01121 // Google Test user but are required for testing. This class allow our tests
01122 // to access them.
01123 //
01124 // This class is supplied only for the purpose of testing Google Test's own
01125 // constructs. Do not use it in user tests, either directly or indirectly.
01126
01127 class TestResultAccessor {
01128 public:
01129
01130 static void RecordProperty(TestResult* test_result,
01131 const std::string& xml_element,
01132 const TestProperty& property) {
01133 test_result->RecordProperty(xml_element, property);
01134 }
01135
01136 static void ClearTestPartResults(TestResult* test_result) {
01137 test_result->ClearTestPartResults();
01138 }
01139
01140 static const std::vector<testing::TestPartResult>& test_part_results(
01141 const TestResult& test_result) {
01142 return test_result.test_part_results();
01143 }
01144 };
01145
01146 #if GTEST_CAN_STREAM_RESULTS_
01147
01148 // Streams test results to the given port on the given host machine.
01149
01150 class StreamingListener : public EmptyTestEventListener {
01151 public:
01152 // Abstract base class for writing strings to a socket.
01153
01154 class AbstractSocketWriter {

```

```

01155 public:
01156
01157 virtual ~AbstractSocketWriter() {
01158 }
01159
01160 // Sends a string to the socket.
01161 virtual void Send(const std::string& message) = 0;
01162
01163 // Closes the socket.
01164
01165 virtual void CloseConnection() {
01166 }
01167
01168 // Sends a string and a newline to the socket.
01169
01170 void SendLn(const std::string& message) {
01171 Send(message + "\n");
01172 }
01173 };
01174
01175 // Concrete class for actually writing strings to a socket.
01176
01177 class SocketWriter : public AbstractSocketWriter {
01178 public:
01179
01180 SocketWriter(const std::string& host, const std::string& port)
01181 : sockfd_(-1), host_name_(host), port_num_(port) {
01182 MakeConnection();
01183 }
01184
01185 ~SocketWriter() override {
01186 if (sockfd_ != -1) CloseConnection();
01187 }
01188
01189 // Sends a string to the socket.
01190
01191 void Send(const std::string& message) override {
01192 GTEST_CHECK_(sockfd_ != -1)
01193 « "Send() can be called only when there is a connection.";
01194
01195 const auto len = static_cast<size_t>(message.length());
01196 if (write(sockfd_, message.c_str(), len) != static_cast<ssize_t>(len)) {
01197 GTEST_LOG_(WARNING) « "stream_result_to: failed to stream to "
01198 « host_name_ « ":" « port_num_;
01199 }
01200 }
01201
01202 private:
01203 // Creates a client socket and connects to the server.
01204 void MakeConnection();
01205
01206 // Closes the socket.
01207
01208 void CloseConnection() override {
01209 GTEST_CHECK_(sockfd_ != -1)
01210 « "CloseConnection() can be called only when there is a connection.";
01211
01212 close(sockfd_);
01213 sockfd_ = -1;
01214 }
01215
01216 int sockfd_; // socket file descriptor
01217 const std::string host_name_;
01218 const std::string port_num_;
01219
01220 SocketWriter(const SocketWriter&) = delete;
01221 SocketWriter& operator=(const SocketWriter&) = delete;
01222 }; // class SocketWriter
01223
01224 // Escapes '=', '&', '%', and '\n' characters in str as "%xx".
01225 static std::string UrlEncode(const char* str);
01226
01227 StreamingListener(const std::string& host, const std::string& port)
01228 : socket_writer_(new SocketWriter(host, port)) {
01229 Start();
01230 }
01231
01232 explicit StreamingListener(AbstractSocketWriter* socket_writer)
01233 : socket_writer_(socket_writer) {
01234 Start();
01235 }
01236
01237 void OnTestProgramStart(const UnitTest& /* unit_test */) override {
01238 SendLn("event=TestProgramStart");
01239 }
01240
01241 void OnTestProgramEnd(const UnitTest& unit_test) override {

```

```

01242 // Note that Google Test current only report elapsed time for each
01243 // test iteration, not for the entire test program.
01244 SendLn("event=TestProgramEnd&passed=" + FormatBool(unit_test.Passed()));
01245
01246 // Notify the streaming server to stop.
01247 socket_writer_>CloseConnection();
01248 }
01249
01250 void OnTestIterationStart(const UnitTest& /* unit_test */,
01251 int iteration) override {
01252 SendLn("event=TestIterationStart&iteration=" +
01253 StreamableToString(iteration));
01254 }
01255
01256 void OnTestIterationEnd(const UnitTest& unit_test,
01257 int /* iteration */) override {
01258 SendLn("event=TestIterationEnd&passed=" + FormatBool(unit_test.Passed()) +
01259 "&elapsed_time=" + StreamableToString(unit_test.elapsed_time()) +
01260 "ms");
01261 }
01262
01263 // Note that "event=TestCaseStart" is a wire format and has to remain
01264 // "case" for compatibility
01265
01266 void OnTestSuiteStart(const TestSuite& test_suite) override {
01267 SendLn(std::string("event=TestCaseStart&name=") + test_suite.name());
01268 }
01269
01270 // Note that "event=TestCaseEnd" is a wire format and has to remain
01271 // "case" for compatibility
01272
01273 void OnTestSuiteEnd(const TestSuite& test_suite) override {
01274 SendLn("event=TestCaseEnd&passed=" + FormatBool(test_suite.Passed()) +
01275 "&elapsed_time=" + StreamableToString(test_suite.elapsed_time()) +
01276 "ms");
01277 }
01278
01279 void OnTestStart(const TestInfo& test_info) override {
01280 SendLn(std::string("event=TestStart&name=") + test_info.name());
01281 }
01282
01283 void OnTestEnd(const TestInfo& test_info) override {
01284 SendLn("event=TestEnd&passed=" +
01285 FormatBool((test_info.result()->Passed()) + "&elapsed_time=" +
01286 StreamableToString((test_info.result()->elapsed_time()) + "ms");
01287 }
01288
01289 void OnTestPartResult(const TestPartResult& test_part_result) override {
01290 const char* file_name = test_part_result.file_name();
01291 if (file_name == nullptr) file_name = "";
01292 SendLn("event=TestPartResult&file=" + UrlEncode(file_name) +
01293 "&line=" + StreamableToString(test_part_result.line_number()) +
01294 "&message=" + UrlEncode(test_part_result.message()));
01295 }
01296
01297 private:
01298 // Sends the given message and a newline to the socket.
01299
01300 void SendLn(const std::string& message) {
01301 socket_writer_>SendLn(message);
01302 }
01303
01304 // Called at the start of streaming to notify the receiver what
01305 // protocol we are using.
01306
01307 void Start() {
01308 SendLn("gtest_streaming_protocol_version=1.0");
01309 }
01310
01311 std::string FormatBool(bool value) {
01312 return value ? "1" : "0";
01313 }
01314
01315 const std::unique_ptr<AbstractSocketWriter> socket_writer_;
01316
01317 StreamingListener(const StreamingListener&) = delete;
01318 StreamingListener& operator=(const StreamingListener&) = delete;
01319 }; // class StreamingListener
01320
01321 #endif // GTEST_CAN_STREAM_RESULTS_
01322
01323 } // namespace internal
01324 } // namespace testing
01325
01326 GTEST_DISABLE_MSC_WARNINGS_POP_() // 4251
01327
01328 #endif // GOOGLETEST_SRC_GTEST_INTERNAL_INL_H_

```

## 16.258 gtest-matchers.cc

```

00001 // Copyright 2007, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // The Google C++ Testing and Mocking Framework (Google Test)
00031 //
00032 // This file implements just enough of the matcher interface to allow
00033 // EXPECT_DEATH and friends to accept a matcher argument.
00034
00035 #include "gtest/gtest-matchers.h"
00036
00037 #include <string>
00038
00039 #include "gtest/internal/gtest-internal.h"
00040 #include "gtest/internal/gtest-port.h"
00041
00042 namespace testing {
00043
00044 // Constructs a matcher that matches a const std::string& whose value is
00045 // equal to s.
00046
00047 Matcher<const std::string&>::Matcher(const std::string& s) {
00048 *this = Eq(s);
00049 }
00050
00051 // Constructs a matcher that matches a const std::string& whose value is
00052 // equal to s.
00053
00054 Matcher<const std::string&>::Matcher(const char* s) {
00055 *this = Eq(std::string(s));
00056 }
00057
00058 // Constructs a matcher that matches a std::string whose value is equal to
00059 // s.
00060
00061 Matcher<std::string>::Matcher(const std::string& s) {
00062 *this = Eq(s);
00063 }
00064
00065 // Constructs a matcher that matches a std::string whose value is equal to
00066 // s.
00067
00068 Matcher<std::string>::Matcher(const char* s) {
00069 *this = Eq(std::string(s));
00070 }
00071
00072 #if GTEST_INTERNAL_HAS_STRING_VIEW
00073 // Constructs a matcher that matches a const StringView& whose value is
00074 // equal to s.
00075
00076 Matcher<const internal::StringView&>::Matcher(const std::string& s) {
00077 *this = Eq(s);
00078 }
00079
00080 // Constructs a matcher that matches a const StringView& whose value is
00081 // equal to s.
00082
00083 Matcher<const internal::StringView&>::Matcher(const char* s) {
00084 *this = Eq(std::string(s));
00085 }

```

```

00086
00087 // Constructs a matcher that matches a const StringView& whose value is
00088 // equal to s.
00089
00090 Matcher<const internal::StringView>::Matcher(internal::StringView s) {
00091 *this = Eq(std::string(s));
00092 }
00093
00094 // Constructs a matcher that matches a StringView whose value is equal to
00095 // s.
00096
00097 Matcher<internal::StringView>::Matcher(const std::string& s) {
00098 *this = Eq(s);
00099 }
00100
00101 // Constructs a matcher that matches a StringView whose value is equal to
00102 // s.
00103
00104 Matcher<internal::StringView>::Matcher(const char* s) {
00105 *this = Eq(std::string(s));
00106 }
00107
00108 // Constructs a matcher that matches a StringView whose value is equal to
00109 // s.
00110
00111 Matcher<internal::StringView>::Matcher(internal::StringView s) {
00112 *this = Eq(std::string(s));
00113 }
00114 #endif // GTEST_INTERNAL_HAS_STRING_VIEW
00115
00116 } // namespace testing

```

## 16.259 gtest-port.cc

```

00001 // Copyright 2008, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 #include "gtest/internal/gtest-port.h"
00031
00032 #include <limits.h>
00033 #include <stdio.h>
00034 #include <stdlib.h>
00035 #include <string.h>
00036
00037 #include <cstdint>
00038 #include <fstream>
00039 #include <memory>
00040
00041 #if GTEST_OS_WINDOWS
00042 #include <io.h>
00043 #include <sys/stat.h>
00044 #include <windows.h>
00045
00046 #include <map> // Used in ThreadLocal.
00047 #ifdef _MSC_VER
00048 #include <crtDBG.h>
00049 #endif // _MSC_VER
00050 #else
00051 #include <unistd.h>

```

```

00052 #endif // GTEST_OS_WINDOWS
00053
00054 #if GTEST_OS_MAC
00055 #include <mach/mach_init.h>
00056 #include <mach/task.h>
00057 #include <mach/vm_map.h>
00058 #endif // GTEST_OS_MAC
00059
00060 #if GTEST_OS_DRAGONFLY || GTEST_OS_FREEBSD || GTEST_OS_GNU_KFREEBSD || \
00061 GTEST_OS_NETBSD || GTEST_OS_OPENBSD
00062 #include <sys/sysctl.h>
00063 #if GTEST_OS_DRAGONFLY || GTEST_OS_FREEBSD || GTEST_OS_GNU_KFREEBSD
00064 #include <sys/user.h>
00065 #endif
00066 #endif
00067
00068 #if GTEST_OS_QNX
00069 #include <devctl.h>
00070 #include <fcntl.h>
00071 #include <sys/procfs.h>
00072 #endif // GTEST_OS_QNX
00073
00074 #if GTEST_OS_AIX
00075 #include <procinfo.h>
00076 #include <sys/types.h>
00077 #endif // GTEST_OS_AIX
00078
00079 #if GTEST_OS_FUCHSIA
00080 #include <zircon/process.h>
00081 #include <zircon/syscalls.h>
00082 #endif // GTEST_OS_FUCHSIA
00083
00084 #include "gtest/gtest-message.h"
00085 #include "gtest/gtest-spi.h"
00086 #include "gtest/internal/gtest-internal.h"
00087 #include "gtest/internal/gtest-string.h"
00088 #include "src/gtest-internal-inl.h"
00089
00090 namespace testing {
00091 namespace internal {
00092
00093 #if GTEST_OS_LINUX || GTEST_OS_GNU_HURD
00094
00095 namespace {
00096
00097 template <typename T>
00098 T ReadProcFileField(const std::string& filename, int field) {
00099 std::string dummy;
00100 std::ifstream file(filename.c_str());
00101 while (field-- > 0) {
00102 file >> dummy;
00103 }
00104 T output = 0;
00105 file >> output;
00106 return output;
00107 }
00108 } // namespace
00109
00110 // Returns the number of active threads, or 0 when there is an error.
00111
00112 size_t GetThreadCount() {
00113 const std::string filename =
00114 (Message() << "/proc/" << getpid() << "/stat").GetString();
00115 return ReadProcFileField<size_t>(filename, 19);
00116 }
00117 #endif
00118 #elif GTEST_OS_MAC
00119
00120 size_t GetThreadCount() {
00121 const task_t task = mach_task_self();
00122 mach_msg_type_number_t thread_count;
00123 thread_act_array_t thread_list;
00124 const kern_return_t status = task_threads(task, &thread_list, &thread_count);
00125 if (status == KERN_SUCCESS) {
00126 // task_threads allocates resources in thread_list and we need to free them
00127 // to avoid leaks.
00128 vm_deallocate(task, reinterpret_cast<vm_address_t>(thread_list),
00129 sizeof(thread_t) * thread_count);
00130 return static_cast<size_t>(thread_count);
00131 } else {
00132 return 0;
00133 }
00134 }
00135 #endif
00136 #elif GTEST_OS_DRAGONFLY || GTEST_OS_FREEBSD || GTEST_OS_GNU_KFREEBSD || \
00137 GTEST_OS_NETBSD

```

```

00139 #if GTEST_OS_NETBSD
00140 #undef KERN_PROC
00141 #define KERN_PROC KERN_PROC2
00142 #define kinfo_proc kinfo_proc2
00143 #endif
00144
00145 #if GTEST_OS_DRAGONFLY
00146 #define KP_NLWP(kp) (kp.kp_nthreads)
00147 #elif GTEST_OS_FREEBSD || GTEST_OS_GNU_KFREEBSD
00148 #define KP_NLWP(kp) (kp.ki_numthreads)
00149 #elif GTEST_OS_NETBSD
00150 #define KP_NLWP(kp) (kp.p_nlwps)
00151 #endif
00152
00153 // Returns the number of threads running in the process, or 0 to indicate that
00154 // we cannot detect it.
00155
00156 size_t GetThreadCount() {
00157 int mib[] = {
00158 CTL_KERN,
00159 KERN_PROC,
00160 KERN_PROC_PID,
00161 getpid(),
00162 #if GTEST_OS_NETBSD
00163 sizeof (struct kinfo_proc),
00164 1,
00165 #endif
00166 };
00167 u_int miblen = sizeof (mib) / sizeof (mib[0]);
00168 struct kinfo_proc info;
00169 size_t size = sizeof (info);
00170 if (sysctl(mib, miblen, &info, &size, NULL, 0)) {
00171 return 0;
00172 }
00173 return static_cast<size_t> (KP_NLWP(info));
00174 }
00175 #elif GTEST_OS_OPENBSD
00176
00177 // Returns the number of threads running in the process, or 0 to indicate that
00178 // we cannot detect it.
00179
00180 size_t GetThreadCount() {
00181 int mib[] = {
00182 CTL_KERN,
00183 KERN_PROC,
00184 KERN_PROC_PID | KERN_PROC_SHOW_THREADS,
00185 getpid(),
00186 sizeof (struct kinfo_proc),
00187 0,
00188 };
00189 u_int miblen = sizeof (mib) / sizeof (mib[0]);
00190
00191 // get number of structs
00192 size_t size;
00193 if (sysctl(mib, miblen, NULL, &size, NULL, 0)) {
00194 return 0;
00195 }
00196
00197 mib[5] = static_cast<int> (size / static_cast<size_t> (mib[4]));
00198
00199 // populate array of structs
00200 struct kinfo_proc info[mib[5]];
00201 if (sysctl(mib, miblen, &info, &size, NULL, 0)) {
00202 return 0;
00203 }
00204
00205 // exclude empty members
00206 size_t nthreads = 0;
00207 for (size_t i = 0; i < size / static_cast<size_t> (mib[4]); i++) {
00208 if (info[i].p_tid != -1) nthreads++;
00209 }
00210 return nthreads;
00211 }
00212
00213 #elif GTEST_OS_QNX
00214
00215 // Returns the number of threads running in the process, or 0 to indicate that
00216 // we cannot detect it.
00217
00218 size_t GetThreadCount() {
00219 const int fd = open("/proc/self/as", O_RDONLY);
00220 if (fd < 0) {
00221 return 0;
00222 }
00223 procfs_info process_info;
00224 const int status =
00225 devctl(fd, DCMD_PROC_INFO, &process_info, sizeof (process_info), nullptr);

```

```

00226 close(fd);
00227 if (status == EOK) {
00228 return static_cast<size_t> (process_info.num_threads);
00229 } else {
00230 return 0;
00231 }
00232 }
00233
00234 #elif GTEST_OS_AIX
00235
00236 size_t GetThreadCount() {
00237 struct procentry64 entry;
00238 pid_t pid = getpid();
00239 int status = getprocs64(&entry, sizeof (entry), nullptr, 0, &pid, 1);
00240 if (status == 1) {
00241 return entry.pi_thcount;
00242 } else {
00243 return 0;
00244 }
00245 }
00246
00247 #elif GTEST_OS_FUCHSIA
00248
00249 size_t GetThreadCount() {
00250 int dummy_buffer;
00251 size_t avail;
00252 zx_status_t status =
00253 zx_object_get_info(zx_process_self(), ZX_INFO_PROCESS_THREADS,
00254 &dummy_buffer, 0, nullptr, &avail);
00255 if (status == ZX_OK) {
00256 return avail;
00257 } else {
00258 return 0;
00259 }
00260 }
00261
00262 #else
00263
00264 size_t GetThreadCount() {
00265 // There's no portable way to detect the number of threads, so we just
00266 // return 0 to indicate that we cannot detect it.
00267 return 0;
00268 }
00269
00270 #endif // GTEST_OS_LINUX
00271
00272 #if GTEST_IS_THREADSafe && GTEST_OS_WINDOWS
00273
00274 AutoHandle::AutoHandle() : handle_(INVALID_HANDLE_VALUE) {
00275 }
00276
00277 AutoHandle::AutoHandle(Handle handle) : handle_(handle) {
00278 }
00279
00280 AutoHandle::~AutoHandle() {
00281 Reset();
00282 }
00283
00284 AutoHandle::Handle AutoHandle::Get() const {
00285 return handle_;
00286 }
00287
00288 void AutoHandle::Reset() {
00289 Reset(INVALID_HANDLE_VALUE);
00290 }
00291
00292 void AutoHandle::Reset(HANDLE handle) {
00293 // Resetting with the same handle we already own is invalid.
00294 if (handle_ != handle) {
00295 if (IsCloseable()) {
00296 ::CloseHandle(handle_);
00297 }
00298 handle_ = handle;
00299 } else {
00300 GTEST_CHECK_(!IsCloseable())
00301 « "Resetting a valid handle to itself is likely a programmer error "
00302 "and thus not allowed.";
00303 }
00304 }
00305
00306 bool AutoHandle::IsCloseable() const {
00307 // Different Windows APIs may use either of these values to represent an
00308 // invalid handle.
00309 return handle_ != nullptr && handle_ != INVALID_HANDLE_VALUE;
00310 }
00311
00312 Mutex::Mutex()

```



```

00313 : owner_thread_id_(0),
00314 type_(kDynamic),
00315 critical_section_init_phase_(0),
00316 critical_section_(new CRITICAL_SECTION) {
00317 ::InitializeCriticalSection(critical_section_);
00318 }
00319
00320 Mutex::~Mutex() {
00321 // Static mutexes are leaked intentionally. It is not thread-safe to try
00322 // to clean them up.
00323 if (type_ == kDynamic) {
00324 ::DeleteCriticalSection(critical_section_);
00325 delete critical_section_;
00326 critical_section_ = nullptr;
00327 }
00328 }
00329
00330 void Mutex::Lock() {
00331 ThreadSafeLazyInit();
00332 ::EnterCriticalSection(critical_section_);
00333 owner_thread_id_ = ::GetCurrentThreadId();
00334 }
00335
00336 void Mutex::Unlock() {
00337 ThreadSafeLazyInit();
00338 // We don't protect writing to owner_thread_id_ here, as it's the
00339 // caller's responsibility to ensure that the current thread holds the
00340 // mutex when this is called.
00341 owner_thread_id_ = 0;
00342 ::LeaveCriticalSection(critical_section_);
00343 }
00344
00345 // Does nothing if the current thread holds the mutex. Otherwise, crashes
00346 // with high probability.
00347
00348 void Mutex::AssertHeld() {
00349 ThreadSafeLazyInit();
00350 GTEST_CHECK_(owner_thread_id_ == ::GetCurrentThreadId())
00351 « "The current thread is not holding the mutex @" « this;
00352 }
00353
00354 namespace {
00355
00356 #ifdef _MSC_VER
00357 // Use the RAII idiom to flag mem allocs that are intentionally never
00358 // deallocated. The motivation is to silence the false positive mem leaks
00359 // that are reported by the debug version of MS's CRT which can only detect
00360 // if an alloc is missing a matching deallocation.
00361 // Example:
00362 // MemoryIsNotDeallocated memory_is_not_deallocated;
00363 // critical_section_ = new CRITICAL_SECTION;
00364 //
00365
00366 class MemoryIsNotDeallocated {
00367 public:
00368 MemoryIsNotDeallocated() : old_crtdbg_flag_(0) {
00369 old_crtdbg_flag_ = _CrtSetDbgFlag(_CRTDBG_REPORT_FLAG);
00370 // Set heap allocation block type to _IGNORE_BLOCK so that MS debug CRT
00371 // doesn't report mem leak if there's no matching deallocation.
00372 (void) _CrtSetDbgFlag(old_crtdbg_flag_ & ~_CRTDBG_ALLOC_MEM_DF);
00373 }
00374
00375 ~MemoryIsNotDeallocated() {
00376 // Restore the original _CRTDBG_ALLOC_MEM_DF flag
00377 (void) _CrtSetDbgFlag(old_crtdbg_flag_);
00378 }
00379
00380 private:
00381 int old_crtdbg_flag_;
00382
00383 MemoryIsNotDeallocated(const MemoryIsNotDeallocated&) = delete;
00384 MemoryIsNotDeallocated& operator=(const MemoryIsNotDeallocated&) = delete;
00385 };
00386 #endif // _MSC_VER
00387
00388 } // namespace
00389
00390 // Initializes owner_thread_id_ and critical_section_ in static mutexes.
00391
00392 void Mutex::ThreadSafeLazyInit() {
00393 // Dynamic mutexes are initialized in the constructor.
00394 if (type_ == kStatic) {
00395 switch (
00396 ::InterlockedCompareExchange(&critical_section_init_phase_, 1L, 0L)) {
00397 case 0:
00398 // If critical_section_init_phase_ was 0 before the exchange, we
00399

```

```

00400 // are the first to test it and need to perform the initialization.
00401 owner_thread_id_ = 0;
00402 {
00403 // Use RAII to flag that following mem alloc is never deallocated.
00404 #ifdef _MSC_VER
00405 MemoryIsNotDeallocated memory_is_not_deallocated;
00406 #endif // _MSC_VER
00407 critical_section_ = new CRITICAL_SECTION;
00408 }
00409 ::InitializeCriticalSection(critical_section_);
00410 // Updates the critical_section_init_phase_ to 2 to signal
00411 // initialization complete.
00412 GTEST_CHECK_ (::InterlockedCompareExchange(&critical_section_init_phase_,
00413 2L, 1L) == 1L);
00414 break;
00415 case 1:
00416 // Somebody else is already initializing the mutex; spin until they
00417 // are done.
00418 while (::InterlockedCompareExchange(&critical_section_init_phase_, 2L,
00419 2L) != 2L) {
00420 // Possibly yields the rest of the thread's time slice to other
00421 // threads.
00422 ::Sleep(0);
00423 }
00424 break;
00425 case 2:
00426 break; // The mutex is already initialized and ready for use.
00427 default:
00428 GTEST_CHECK_(false)
00429 << "Unexpected value of critical_section_init_phase_ "
00430 << "while initializing a static mutex.";
00431 }
00432 }
00433 }
00434 }
00435 }
00436 namespace {
00437
00438 class ThreadWithParamSupport : public ThreadWithParamBase {
00439 public:
00440 static HANDLE CreateThread(Runnable* runnable,
00441 Notification* thread_can_start) {
00442 ThreadMainParam* param = new ThreadMainParam(runnable, thread_can_start);
00443 DWORD thread_id;
00444 HANDLE thread_handle = ::CreateThread(
00445 nullptr, // Default security.
00446 0, // Default stack size.
00447 &ThreadWithParamSupport::ThreadMain,
00448 param, // Parameter to ThreadMainStatic
00449 0x0, // Default creation flags.
00450 &thread_id); // Need a valid pointer for the call to work under Win98.
00451 GTEST_CHECK_(thread_handle != nullptr)
00452 << "CreateThread failed with error " << ::GetLastError() << ".";
00453 if (thread_handle == nullptr) {
00454 delete param;
00455 }
00456 return thread_handle;
00457 }
00458 private:
00459 struct ThreadMainParam {
00460 ThreadMainParam(Runnable* runnable, Notification* thread_can_start)
00461 : runnable_(runnable), thread_can_start_(thread_can_start) {
00462 }
00463 std::unique_ptr<Runnable> runnable_;
00464 // Does not own.
00465 Notification* thread_can_start_;
00466 };
00467 static DWORD WINAPI ThreadMain(void* ptr) {
00468 // Transfers ownership.
00469 std::unique_ptr<ThreadMainParam> param(static_cast<ThreadMainParam*>(ptr));
00470 if (param->thread_can_start_ != nullptr)
00471 param->thread_can_start_->WaitForNotification();
00472 param->runnable_->Run();
00473 return 0;
00474 }
00475 // Prohibit instantiation.
00476 ThreadWithParamSupport();
00477 ThreadWithParamSupport(const ThreadWithParamSupport&) = delete;
00478 ThreadWithParamSupport& operator=(const ThreadWithParamSupport&) = delete;

```

```

00487 };
00488
00489 } // namespace
00490
00491 ThreadWithParamBase::ThreadWithParamBase(Runnable* runnable,
00492 Notification* thread_can_start)
00493 : thread_(
00494 ThreadWithParamSupport::CreateThread(runnable, thread_can_start)) {
00495 }
00496
00497 ThreadWithParamBase::~ThreadWithParamBase() {
00498 Join();
00499 }
00500
00501 void ThreadWithParamBase::Join() {
00502 GTEST_CHECK_ (::WaitForSingleObject(thread_.Get(), INFINITE) == WAIT_OBJECT_0)
00503 « "Failed to join the thread with error " « ::GetLastError() « ".";
00504 }
00505
00506 // Maps a thread to a set of ThreadIdToThreadLocals that have values
00507 // instantiated on that thread and notifies them when the thread exits. A
00508 // ThreadLocal instance is expected to persist until all threads it has
00509 // values on have terminated.
00510
00511 class ThreadLocalRegistryImpl {
00512 public:
00513 // Registers thread_local_instance as having value on the current thread.
00514 // Returns a value that can be used to identify the thread from other threads.
00515
00516 static ThreadLocalValueHolderBase* GetValueOnCurrentThread(
00517 const ThreadLocalBase* thread_local_instance) {
00518 #ifdef _MSC_VER
00519 MemoryIsNotDeallocated memory_is_not_deallocated;
00520 #endif // _MSC_VER
00521 DWORD current_thread = ::GetCurrentThreadId();
00522 MutexLock lock(&mutex_);
00523 ThreadIdToThreadLocals * const thread_to_thread_locals =
00524 GetThreadLocalsMapLocked();
00525 ThreadIdToThreadLocals::iterator thread_local_pos =
00526 thread_to_thread_locals->find(current_thread);
00527 if (thread_local_pos == thread_to_thread_locals->end()) {
00528 thread_local_pos =
00529 thread_to_thread_locals
00530 ->insert(std::make_pair(current_thread, ThreadLocalValues()))
00531 .first;
00532 StartWatcherThreadFor(current_thread);
00533 }
00534 ThreadLocalValues& thread_local_values = thread_local_pos->second;
00535 ThreadLocalValues::iterator value_pos =
00536 thread_local_values.find(thread_local_instance);
00537 if (value_pos == thread_local_values.end()) {
00538 value_pos =
00539 thread_local_values
00540 .insert(std::make_pair(
00541 thread_local_instance,
00542 std::shared_ptr<ThreadLocalValueHolderBase>(
00543 thread_local_instance->NewValueForCurrentThread()))
00544 .first;
00545 }
00546 return value_pos->second.get();
00547 }
00548
00549 static void OnThreadLocalDestroyed(
00550 const ThreadLocalBase* thread_local_instance) {
00551 std::vector<std::shared_ptr<ThreadLocalValueHolderBase> > value_holders;
00552 // Clean up the ThreadLocalValues data structure while holding the lock, but
00553 // defer the destruction of the ThreadLocalValueHolderBases.
00554 {
00555 MutexLock lock(&mutex_);
00556 ThreadIdToThreadLocals * const thread_to_thread_locals =
00557 GetThreadLocalsMapLocked();
00558 for (ThreadIdToThreadLocals::iterator it =
00559 thread_to_thread_locals->begin();
00560 it != thread_to_thread_locals->end(); ++it) {
00561 ThreadLocalValues& thread_local_values = it->second;
00562 ThreadLocalValues::iterator value_pos =
00563 thread_local_values.find(thread_local_instance);
00564 if (value_pos != thread_local_values.end()) {
00565 value_holders.push_back(value_pos->second);
00566 thread_local_values.erase(value_pos);
00567 // This 'if' can only be successful at most once, so theoretically we
00568 // could break out of the loop here, but we don't bother doing so.
00569 }
00570 }
00571 }
00572 // Outside the lock, let the destructor for 'value_holders' deallocate the
00573 // ThreadLocalValueHolderBases.

```

```

00574 }
00575
00576 static void OnThreadExit(DWORD thread_id) {
00577 GTEST_CHECK_(thread_id != 0) « ::GetLastError();
00578 std::vector<std::shared_ptr<ThreadLocalValueHolderBase> > value_holders;
00579 // Clean up the ThreadIdToThreadLocals data structure while holding the
00580 // lock, but defer the destruction of the ThreadLocalValueHolderBases.
00581 {
00582 MutexLock lock(&mutex_);
00583 ThreadIdToThreadLocals * const thread_to_thread_locals =
00584 GetThreadLocalsMapLocked();
00585 ThreadIdToThreadLocals::iterator thread_local_pos =
00586 thread_to_thread_locals->find(thread_id);
00587 if (thread_local_pos != thread_to_thread_locals->end()) {
00588 ThreadLocalValues& thread_local_values = thread_local_pos->second;
00589 for (ThreadLocalValues::iterator value_pos =
00590 thread_local_values.begin();
00591 value_pos != thread_local_values.end(); ++value_pos) {
00592 value_holders.push_back(value_pos->second);
00593 }
00594 thread_to_thread_locals->erase(thread_local_pos);
00595 }
00596 }
00597 // Outside the lock, let the destructor for 'value_holders' deallocate the
00598 // ThreadLocalValueHolderBases.
00599 }
00600
00601 private:
00602 // In a particular thread, maps a ThreadLocal object to its value.
00603 typedef std::map<const ThreadLocalBase*,
00604 std::shared_ptr<ThreadLocalValueHolderBase> >
00605 ThreadLocalValues;
00606 // Stores all ThreadIdToThreadLocals having values in a thread, indexed by
00607 // thread's ID.
00608 typedef std::map<DWORD, ThreadLocalValues> ThreadIdToThreadLocals;
00609
00610 // Holds the thread id and thread handle that we pass from
00611 // StartWatcherThreadFor to WatcherThreadFunc.
00612 typedef std::pair<DWORD, HANDLE> ThreadIdAndHandle;
00613
00614 static void StartWatcherThreadFor(DWORD thread_id) {
00615 // The returned handle will be kept in thread_map and closed by
00616 // watcher_thread in WatcherThreadFunc.
00617 HANDLE thread =
00618 ::OpenThread(SYNCHRONIZE | THREAD_QUERY_INFORMATION, FALSE, thread_id);
00619 GTEST_CHECK_(thread != nullptr);
00620 // We need to pass a valid thread ID pointer into CreateThread for it
00621 // to work correctly under Win98.
00622 DWORD watcher_thread_id;
00623 HANDLE watcher_thread = ::CreateThread(
00624 nullptr, // Default security.
00625 0, // Default stack size
00626 &ThreadLocalRegistryImpl::WatcherThreadFunc,
00627 reinterpret_cast<LPVOID>(new ThreadIdAndHandle(thread_id, thread)),
00628 CREATE_SUSPENDED, &watcher_thread_id);
00629 GTEST_CHECK_(watcher_thread != nullptr)
00630 « "CreateThread failed with error " « ::GetLastError() « ".";
00631 // Give the watcher thread the same priority as ours to avoid being
00632 // blocked by it.
00633 ::SetThreadPriority(watcher_thread,
00634 ::GetThreadPriority(::GetCurrentThread()));
00635 ::ResumeThread(watcher_thread);
00636 ::CloseHandle(watcher_thread);
00637 }
00638
00639 // Monitors exit from a given thread and notifies those
00640 // ThreadIdToThreadLocals about thread termination.
00641
00642 static DWORD WINAPI WatcherThreadFunc(LPVOID param) {
00643 const ThreadIdAndHandle* tah =
00644 reinterpret_cast<const ThreadIdAndHandle*>(param);
00645 GTEST_CHECK_(::WaitForSingleObject(tah->second, INFINITE) == WAIT_OBJECT_0);
00646 OnThreadExit(tah->first);
00647 ::CloseHandle(tah->second);
00648 delete tah;
00649 return 0;
00650 }
00651
00652 // Returns map of thread local instances.
00653
00654 static ThreadIdToThreadLocals* GetThreadLocalsMapLocked() {
00655 mutex_.AssertHeld();
00656 #ifdef _MSC_VER
00657 MemoryIsNotDeallocated memory_is_not_deallocated;
00658 #endif // _MSC_VER
00659 static ThreadIdToThreadLocals* map = new ThreadIdToThreadLocals();
00660 return map;

```

```

00661 }
00662
00663 // Protects access to GetThreadLocalsMapLocked() and its return value.
00664 static Mutex mutex_;
00665 // Protects access to GetThreadMapLocked() and its return value.
00666 static Mutex thread_map_mutex_;
00667 };
00668
00669 Mutex ThreadLocalRegistryImpl::mutex_(Mutex::kStaticMutex); // NOLINT
00670 Mutex ThreadLocalRegistryImpl::thread_map_mutex_(
00671 Mutex::kStaticMutex); // NOLINT
00672
00673 ThreadLocalValueHolderBase* ThreadLocalRegistry::GetValueOnCurrentThread(
00674 const ThreadLocalBase* thread_local_instance) {
00675 return ThreadLocalRegistryImpl::GetValueOnCurrentThread(
00676 thread_local_instance);
00677 }
00678
00679 void ThreadLocalRegistry::OnThreadLocalDestroyed(
00680 const ThreadLocalBase* thread_local_instance) {
00681 ThreadLocalRegistryImpl::OnThreadLocalDestroyed(thread_local_instance);
00682 }
00683
00684 #endif // GTEST_IS_THREADSafe && GTEST_OS_WINDOWS
00685
00686 #if GTEST_USES_POSIX_RE
00687
00688 // Implements RE. Currently only needed for death tests.
00689
00690 RE::~RE() {
00691 if (is_valid_) {
00692 // regfree'ing an invalid regex might crash because the content
00693 // of the regex is undefined. Since the regex's are essentially
00694 // the same, one cannot be valid (or invalid) without the other
00695 // being so too.
00696 regfree(&partial_regex_);
00697 regfree(&full_regex_);
00698 }
00699 free(const_cast<char*>(pattern_));
00700 }
00701
00702 // Returns true if and only if regular expression re matches the entire str.
00703
00704 bool RE::FullMatch(const char* str, const RE& re) {
00705 if (!re.is_valid_) return false;
00706
00707 regmatch_t match;
00708 return regexec(&re.full_regex_, str, 1, &match, 0) == 0;
00709 }
00710
00711 // Returns true if and only if regular expression re matches a substring of
00712 // str (including str itself).
00713
00714 bool RE::PartialMatch(const char* str, const RE& re) {
00715 if (!re.is_valid_) return false;
00716
00717 regmatch_t match;
00718 return regexec(&re.partial_regex_, str, 1, &match, 0) == 0;
00719 }
00720
00721 // Initializes an RE from its string representation.
00722
00723 void RE::Init(const char* regex) {
00724 pattern_ = posix::StrDup(regex);
00725
00726 // Reserves enough bytes to hold the regular expression used for a
00727 // full match.
00728 const size_t full_regex_len = strlen(regex) + 10;
00729 char* const full_pattern = new char[full_regex_len];
00730
00731 snprintf(full_pattern, full_regex_len, "^(%s)$", regex);
00732 is_valid_ = regcomp(&full_regex_, full_pattern, REG_EXTENDED) == 0;
00733 // We want to call regcomp(&partial_regex_, ...) even if the
00734 // previous expression returns false. Otherwise partial_regex_ may
00735 // not be properly initialized can may cause trouble when it's
00736 // freed.
00737 //
00738 // Some implementation of POSIX regex (e.g. on at least some
00739 // versions of Cygwin) doesn't accept the empty string as a valid
00740 // regex. We change it to an equivalent form "()" to be safe.
00741 if (is_valid_) {
00742 const char* const partial_regex = (*regex == '\0') ? "()" : regex;
00743 is_valid_ = regcomp(&partial_regex_, partial_regex, REG_EXTENDED) == 0;
00744 }
00745 EXPECT_TRUE(is_valid_)
00746 « "Regular expression \"" « regex
00747 « "\" is not a valid POSIX Extended regular expression.";

```

```

00748
00749 delete[] full_pattern;
00750 }
00751
00752 #elif GTEST_USES_SIMPLE_RE
00753
00754 // Returns true if and only if ch appears anywhere in str (excluding the
00755 // terminating '\0' character).
00756
00757 bool IsInSet(char ch, const char* str) {
00758 return ch != '\0' && strchr(str, ch) != nullptr;
00759 }
00760
00761 // Returns true if and only if ch belongs to the given classification.
00762 // Unlike similar functions in <ctype.h>, these aren't affected by the
00763 // current locale.
00764
00765 bool IsAsciiDigit(char ch) {
00766 return '0' <= ch && ch <= '9';
00767 }
00768
00769 bool IsAsciiPunct(char ch) {
00770 return IsInSet(ch, "^-!\"#$%&'()*+,-./:;<=>?@[\\]_`{|}~");
00771 }
00772
00773 bool IsRepeat(char ch) {
00774 return IsInSet(ch, "?*+");
00775 }
00776
00777 bool IsAsciiWhiteSpace(char ch) {
00778 return IsInSet(ch, " \f\n\r\t\v");
00779 }
00780
00781 bool IsAsciiWordChar(char ch) {
00782 return ('a' <= ch && ch <= 'z') || ('A' <= ch && ch <= 'Z') ||
00783 ('0' <= ch && ch <= '9') || ch == '_';
00784 }
00785
00786 // Returns true if and only if "\\c" is a supported escape sequence.
00787
00788 bool IsValidEscape(char c) {
00789 return (IsAsciiPunct(c) || IsInSet(c, "dDfnrsStvwW"));
00790 }
00791
00792 // Returns true if and only if the given atom (specified by escaped and
00793 // pattern) matches ch. The result is undefined if the atom is invalid.
00794
00795 bool AtomMatchesChar(bool escaped, char pattern_char, char ch) {
00796 if (escaped) { // "\\p" where p is pattern_char.
00797 switch (pattern_char) {
00798 case 'd':
00799 return IsAsciiDigit(ch);
00800 case 'D':
00801 return !IsAsciiDigit(ch);
00802 case 'f':
00803 return ch == '\f';
00804 case 'n':
00805 return ch == '\n';
00806 case 'r':
00807 return ch == '\r';
00808 case 's':
00809 return IsAsciiWhiteSpace(ch);
00810 case 'S':
00811 return !IsAsciiWhiteSpace(ch);
00812 case 't':
00813 return ch == '\t';
00814 case 'v':
00815 return ch == '\v';
00816 case 'w':
00817 return IsAsciiWordChar(ch);
00818 case 'W':
00819 return !IsAsciiWordChar(ch);
00820 }
00821 return IsAsciiPunct(pattern_char) && pattern_char == ch;
00822 }
00823
00824 return (pattern_char == '.' && ch != '\n') || pattern_char == ch;
00825 }
00826
00827 // Helper function used by ValidateRegex() to format error messages.
00828
00829 static std::string FormatRegexSyntaxError(const char* regex, int index) {
00830 return (Message() << "Syntax error at index " << index
00831 << " in simple regular expression \"" << regex << "\": ")
00832 .GetString();
00833 }
00834

```

```

00835 // Generates non-fatal failures and returns false if regex is invalid;
00836 // otherwise returns true.
00837
00838 bool ValidateRegex(const char* regex) {
00839 if (regex == nullptr) {
00840 ADD_FAILURE() << "NULL is not a valid simple regular expression.";
00841 return false;
00842 }
00843
00844 bool is_valid = true;
00845
00846 // True if and only if ?, *, or + can follow the previous atom.
00847 bool prev_repeatable = false;
00848 for (int i = 0; regex[i]; i++) {
00849 if (regex[i] == '\\') { // An escape sequence
00850 i++;
00851 if (regex[i] == '\0') {
00852 ADD_FAILURE() << FormatRegexSyntaxError(regex, i - 1)
00853 << "\\ cannot appear at the end.";
00854 return false;
00855 }
00856
00857 if (!IsValidEscape(regex[i])) {
00858 ADD_FAILURE() << FormatRegexSyntaxError(regex, i - 1)
00859 << "invalid escape sequence \"" << regex[i] << "\".";
00860 is_valid = false;
00861 }
00862 prev_repeatable = true;
00863 } else { // Not an escape sequence.
00864 const char ch = regex[i];
00865
00866 if (ch == '^' && i > 0) {
00867 ADD_FAILURE() << FormatRegexSyntaxError(regex, i)
00868 << "'^' can only appear at the beginning.";
00869 is_valid = false;
00870 } else if (ch == '$' && regex[i + 1] != '\0') {
00871 ADD_FAILURE() << FormatRegexSyntaxError(regex, i)
00872 << "'$' can only appear at the end.";
00873 is_valid = false;
00874 } else if (IsInSet(ch, "()[]{}|")) {
00875 ADD_FAILURE() << FormatRegexSyntaxError(regex, i) << "'" << ch
00876 << "' is unsupported.";
00877 is_valid = false;
00878 } else if (IsRepeat(ch) && !prev_repeatable) {
00879 ADD_FAILURE() << FormatRegexSyntaxError(regex, i) << "'" << ch
00880 << "' can only follow a repeatable token.";
00881 is_valid = false;
00882 }
00883
00884 prev_repeatable = !IsInSet(ch, "^$?*+");
00885 }
00886 }
00887
00888 return is_valid;
00889 }
00890
00891 // Matches a repeated regex atom followed by a valid simple regular
00892 // expression. The regex atom is defined as c if escaped is false,
00893 // or \c otherwise. repeat is the repetition meta character (?, *,
00894 // or +). The behavior is undefined if str contains too many
00895 // characters to be indexable by size_t, in which case the test will
00896 // probably time out anyway. We are fine with this limitation as
00897 // std::string has it too.
00898
00899 bool MatchRepetitionAndRegexAtHead(bool escaped, char c, char repeat,
00900 const char* regex, const char* str) {
00901 const size_t min_count = (repeat == '+') ? 1 : 0;
00902 const size_t max_count = (repeat == '?') ? 1 : static_cast<size_t> (-1) - 1;
00903 // We cannot call numeric_limits::max() as it conflicts with the
00904 // max() macro on Windows.
00905
00906 for (size_t i = 0; i <= max_count; ++i) {
00907 // We know that the atom matches each of the first i characters in str.
00908 if (i >= min_count && MatchRegexAtHead(regex, str + i)) {
00909 // We have enough matches at the head, and the tail matches too.
00910 // Since we only care about *whether* the pattern matches str
00911 // (as opposed to *how* it matches), there is no need to find a
00912 // greedy match.
00913 return true;
00914 }
00915 if (str[i] == '\0' || !AtomMatchesChar(escaped, c, str[i])) return false;
00916 }
00917 return false;
00918 }
00919
00920 // Returns true if and only if regex matches a prefix of str. regex must
00921 // be a valid simple regular expression and not start with "^", or the

```

```

00922 // result is undefined.
00923
00924 bool MatchRegexAtHead(const char* regex, const char* str) {
00925 if (*regex == '\0') // An empty regex matches a prefix of anything.
00926 return true;
00927
00928 // "$" only matches the end of a string. Note that regex being
00929 // valid guarantees that there's nothing after "$" in it.
00930 if (*regex == '$') return *str == '\0';
00931
00932 // Is the first thing in regex an escape sequence?
00933 const bool escaped = *regex == '\\';
00934 if (escaped) ++regex;
00935 if (IsRepeat(regex[1])) {
00936 // MatchRepetitionAndRegexAtHead() calls MatchRegexAtHead(), so
00937 // here's an indirect recursion. It terminates as the regex gets
00938 // shorter in each recursion.
00939 return MatchRepetitionAndRegexAtHead(escaped, regex[0], regex[1], regex + 2,
00940 str);
00941 } else {
00942 // regex isn't empty, isn't "$", and doesn't start with a
00943 // repetition. We match the first atom of regex with the first
00944 // character of str and recurse.
00945 return (*str != '\0') && AtomMatchesChar(escaped, *regex, *str) &&
00946 MatchRegexAtHead(regex + 1, str + 1);
00947 }
00948 }
00949
00950 // Returns true if and only if regex matches any substring of str. regex must
00951 // be a valid simple regular expression, or the result is undefined.
00952 //
00953 // The algorithm is recursive, but the recursion depth doesn't exceed
00954 // the regex length, so we won't need to worry about running out of
00955 // stack space normally. In rare cases the time complexity can be
00956 // exponential with respect to the regex length + the string length,
00957 // but usually it's much faster (often close to linear).
00958
00959 bool MatchRegexAnywhere(const char* regex, const char* str) {
00960 if (regex == nullptr || str == nullptr) return false;
00961
00962 if (*regex == '^') return MatchRegexAtHead(regex + 1, str);
00963
00964 // A successful match can be anywhere in str.
00965 do {
00966 if (MatchRegexAtHead(regex, str)) return true;
00967 } while (*str++ != '\0');
00968 return false;
00969 }
00970
00971 // Implements the RE class.
00972
00973 RE::~RE() {
00974 free(const_cast<char*>(pattern_));
00975 free(const_cast<char*>(full_pattern_));
00976 }
00977
00978 // Returns true if and only if regular expression re matches the entire str.
00979
00980 bool RE::FullMatch(const char* str, const RE& re) {
00981 return re.is_valid_ && MatchRegexAnywhere(re.full_pattern_, str);
00982 }
00983
00984 // Returns true if and only if regular expression re matches a substring of
00985 // str (including str itself).
00986
00987 bool RE::PartialMatch(const char* str, const RE& re) {
00988 return re.is_valid_ && MatchRegexAnywhere(re.pattern_, str);
00989 }
00990
00991 // Initializes an RE from its string representation.
00992
00993 void RE::Init(const char* regex) {
00994 pattern_ = full_pattern_ = nullptr;
00995 if (regex != nullptr) {
00996 pattern_ = posix::StrDup(regex);
00997 }
00998
00999 is_valid_ = ValidateRegex(regex);
01000 if (!is_valid_) {
01001 // No need to calculate the full pattern when the regex is invalid.
01002 return;
01003 }
01004
01005 const size_t len = strlen(regex);
01006 // Reserves enough bytes to hold the regular expression used for a
01007 // full match: we need space to prepend a '^', append a '$', and
01008 // terminate the string with '\0'.

```



```

01009 char* buffer = static_cast<char*> (malloc(len + 3));
01010 full_pattern_ = buffer;
01011
01012 if (*regex != '^')
01013 *buffer++ = '^'; // Makes sure full_pattern_ starts with '^'.
01014
01015 // We don't use snprintf or strncpy, as they trigger a warning when
01016 // compiled with VC++ 8.0.
01017 memcpy(buffer, regex, len);
01018 buffer += len;
01019
01020 if (len == 0 || regex[len - 1] != '$')
01021 *buffer++ = '$'; // Makes sure full_pattern_ ends with '$'.
01022
01023 *buffer = '\\0';
01024 }
01025
01026 #endif // GTEST_USES_POSIX_RE
01027
01028 const char kUnknownFile[] = "unknown file";
01029
01030 // Formats a source file path and a line number as they would appear
01031 // in an error message from the compiler used to compile this code.
01032
01033 GTEST_API_::std::string FormatFileLocation(const char* file, int line) {
01034 const std::string file_name(file == nullptr ? kUnknownFile : file);
01035
01036 if (line < 0) {
01037 return file_name + ":";
01038 }
01039 #ifdef _MSC_VER
01040 return file_name + "(" + StreamableToString(line) + ")";
01041 #else
01042 return file_name + ":" + StreamableToString(line) + ":";
01043 #endif // _MSC_VER
01044 }
01045
01046 // Formats a file location for compiler-independent XML output.
01047 // Although this function is not platform dependent, we put it next to
01048 // FormatFileLocation in order to contrast the two functions.
01049 // Note that FormatCompilerIndependentFileLocation() does NOT append colon
01050 // to the file location it produces, unlike FormatFileLocation().
01051
01052 GTEST_API_::std::string FormatCompilerIndependentFileLocation(const char* file,
01053 int line) {
01054 const std::string file_name(file == nullptr ? kUnknownFile : file);
01055
01056 if (line < 0)
01057 return file_name;
01058 else
01059 return file_name + ":" + StreamableToString(line);
01060 }
01061
01062 GTestLog::GTestLog(GTestLogSeverity severity, const char* file, int line)
01063 : severity_(severity) {
01064 const char* const marker = severity == GTEST_INFO ? "[INFO]"
01065 : severity == GTEST_WARNING ? "[WARNING]"
01066 : severity == GTEST_ERROR ? "[ERROR]"
01067 : "[FATAL]";
01068 GetStream() << ::std::endl
01069 << marker << " " << FormatFileLocation(file, line).c_str()
01070 << ": ";
01071 }
01072
01073 // Flushes the buffers and, if severity is GTEST_FATAL, aborts the program.
01074
01075 GTestLog::~GTestLog() {
01076 GetStream() << ::std::endl;
01077 if (severity_ == GTEST_FATAL) {
01078 fflush(stderr);
01079 posix::Abort();
01080 }
01081 }
01082
01083 // Disable Microsoft deprecation warnings for POSIX functions called from
01084 // this class (creat, dup, dup2, and close)
01085
01086 GTEST_DISABLE_MSC_DEPRECATED_PUSH_()
01087
01088 #if GTEST_HAS_STREAM_REDIRECTION
01089 // Object that captures an output stream (stdout/stderr).
01090 class CapturedStream {
01091 public:
01092 // The ctor redirects the stream to a temporary file.
01093
01094 explicit CapturedStream(int fd) : fd_(fd), uncaptured_fd_(dup(fd)) {

```

```

01096 #if GTEST_OS_WINDOWS
01097 char temp_dir_path[MAX_PATH + 1] = {'\0'}; // NOLINT
01098 char temp_file_path[MAX_PATH + 1] = {'\0'}; // NOLINT
01099
01100 ::GetTempPathA(sizeof (temp_dir_path), temp_dir_path);
01101 const UINT success = ::GetTempFileNameA(temp_dir_path, "gtest_redir",
01102 0, // Generate unique file name.
01103 temp_file_path);
01104 GTEST_CHECK_(success != 0)
01105 « "Unable to create a temporary file in " « temp_dir_path;
01106 const int captured_fd = creat(temp_file_path, _S_IREAD | _S_IWRITE);
01107 GTEST_CHECK_(captured_fd != -1)
01108 « "Unable to open temporary file " « temp_file_path;
01109 filename_ = temp_file_path;
01110 #else
01111 // There's no guarantee that a test has write access to the current
01112 // directory, so we create the temporary file in a temporary directory.
01113 std::string name_template;
01114
01115 #if GTEST_OS_LINUX_ANDROID
01116 // Note: Android applications are expected to call the framework's
01117 // Context.getExternalStorageDirectory() method through JNI to get
01118 // the location of the world-writable SD Card directory. However,
01119 // this requires a Context handle, which cannot be retrieved
01120 // globally from native code. Doing so also precludes running the
01121 // code as part of a regular standalone executable, which doesn't
01122 // run in a Dalvik process (e.g. when running it through 'adb shell').
01123 //
01124 // The location /data/local/tmp is directly accessible from native code.
01125 // '/sdcard' and other variants cannot be relied on, as they are not
01126 // guaranteed to be mounted, or may have a delay in mounting.
01127 name_template = "/data/local/tmp/";
01128 #elif GTEST_OS_IOS
01129 char user_temp_dir[PATH_MAX + 1];
01130
01131 // Documented alternative to NSTemporaryDirectory() (for obtaining creating
01132 // a temporary directory) at
01133 //
01134 // https://developer.apple.com/library/archive/documentation/Security/Conceptual/SecureCodingGuide/Articles/RaceConditions
01135 //
01136 // _CS_DARWIN_USER_TEMP_DIR (as well as _CS_DARWIN_USER_CACHE_DIR) is not
01137 // documented in the confstr() man page at
01138 //
01139 // https://developer.apple.com/library/archive/documentation/System/Conceptual/ManPages_iPhoneOS/man3/confstr.3.html#//app
01140 // https://trac.webkit.org/changeset/262004/webkit
01141 // https://trac.webkit.org/changeset/263705/webkit
01142 //
01143 // The confstr() implementation falls back to getenv("TMPDIR"). See
01144 // https://opensource.apple.com/source/Libc/Libc-1439.100.3/gen/confstr.c.auto.html
01145 ::confstr(_CS_DARWIN_USER_TEMP_DIR, user_temp_dir, sizeof (user_temp_dir));
01146
01147 name_template = user_temp_dir;
01148 if (name_template.back() != GTEST_PATH_SEP_[0])
01149 name_template.push_back(GTEST_PATH_SEP_[0]);
01150 #else
01151 name_template = "/tmp/";
01152 #endif
01153
01154 name_template.append("gtest_captured_stream.XXXXXX");
01155
01156 // mkstemp() modifies the string bytes in place, and does not go beyond the
01157 // string's length. This results in well-defined behavior in C++17.
01158 //
01159 // The const_cast is needed below C++17. The constraints on std::string
01160 // implementations in C++11 and above make assumption behind the const_cast
01161 // fairly safe.
01162 const int captured_fd = ::mkstemp(const_cast<char*> (name_template.data()));
01163 if (captured_fd == -1) {
01164 GTEST_LOG_(WARNING)
01165 « "Failed to create tmp file " « name_template
01166 « " for test; does the test have access to the /tmp directory?";
01167 }
01168 filename_ = std::move(name_template);
01169 #endif // GTEST_OS_WINDOWS
01170 fflush(nullptr);
01171 dup2(captured_fd, fd_);
01172 close(captured_fd);
01173
01174 ~CapturedStream() {
01175 remove(filename_.c_str());
01176 }
01177
01178 std::string GetCapturedString() {
01179 if (uncaptured_fd_ != -1) {
01180 // Restores the original stream.
01181 fflush(nullptr);

```

```

01181 dup2(uncaptured_fd_, fd_);
01182 close(uncaptured_fd_);
01183 uncaptured_fd_ = -1;
01184 }
01185
01186 FILE * const file = posix::FOpen(filename_.c_str(), "r");
01187 if (file == nullptr) {
01188 GTEST_LOG_(FATAL) << "Failed to open tmp file " << filename_
01189 << " for capturing stream.";
01190 }
01191 const std::string content = ReadEntireFile(file);
01192 posix::FClose(file);
01193 return content;
01194 }
01195
01196 private:
01197 const int fd_; // A stream to capture.
01198 int uncaptured_fd_;
01199 // Name of the temporary file holding the stderr output.
01200 ::std::string filename_;
01201
01202 CapturedStream(const CapturedStream&) = delete;
01203 CapturedStream& operator=(const CapturedStream&) = delete;
01204 };
01205
01206 GTEST_DISABLE_MSC_DEPRECATED_POP_()
01207
01208 static CapturedStream* g_captured_stderr = nullptr;
01209 static CapturedStream* g_captured_stdout = nullptr;
01210
01211 // Starts capturing an output stream (stdout/stderr).
01212
01213 static void CaptureStream(int fd, const char* stream_name,
01214 CapturedStream** stream) {
01215 if (*stream != nullptr) {
01216 GTEST_LOG_(FATAL) << "Only one " << stream_name
01217 << " capturer can exist at a time.";
01218 }
01219 *stream = new CapturedStream(fd);
01220 }
01221
01222 // Stops capturing the output stream and returns the captured string.
01223
01224 static std::string GetCapturedStream(CapturedStream** captured_stream) {
01225 const std::string content = (*captured_stream)->GetCapturedString();
01226
01227 delete *captured_stream;
01228 *captured_stream = nullptr;
01229
01230 return content;
01231 }
01232
01233 #if defined(_MSC_VER) || defined(__BORLANDC__)
01234 // MSVC and C++Builder do not provide a definition of STDERR_FILENO.
01235 const int kStdOutFileno = 1;
01236 const int kStdErrFileno = 2;
01237 #else
01238 const int kStdOutFileno = STDOUT_FILENO;
01239 const int kStdErrFileno = STDERR_FILENO;
01240 #endif // defined(_MSC_VER) || defined(__BORLANDC__)
01241
01242 // Starts capturing stdout.
01243
01244 void CaptureStdout() {
01245 CaptureStream(kStdOutFileno, "stdout", &g_captured_stdout);
01246 }
01247
01248 // Starts capturing stderr.
01249
01250 void CaptureStderr() {
01251 CaptureStream(kStdErrFileno, "stderr", &g_captured_stderr);
01252 }
01253
01254 // Stops capturing stdout and returns the captured string.
01255
01256 std::string GetCapturedStdout() {
01257 return GetCapturedStream(&g_captured_stdout);
01258 }
01259
01260 // Stops capturing stderr and returns the captured string.
01261
01262 std::string GetCapturedStderr() {
01263 return GetCapturedStream(&g_captured_stderr);
01264 }
01265
01266 #endif // GTEST_HAS_STREAM_REDIRECTION
01267

```

```

01268 size_t GetFileSize(FILE* file) {
01269 fseek(file, 0, SEEK_END);
01270 return static_cast<size_t> (ftell(file));
01271 }
01272
01273 std::string ReadEntireFile(FILE* file) {
01274 const size_t file_size = GetFileSize(file);
01275 char* const buffer = new char[file_size];
01276
01277 size_t bytes_last_read = 0; // # of bytes read in the last fread()
01278 size_t bytes_read = 0; // # of bytes read so far
01279
01280 fseek(file, 0, SEEK_SET);
01281
01282 // Keeps reading the file until we cannot read further or the
01283 // pre-determined file size is reached.
01284 do {
01285 bytes_last_read =
01286 fread(buffer + bytes_read, 1, file_size - bytes_read, file);
01287 bytes_read += bytes_last_read;
01288 } while (bytes_last_read > 0 && bytes_read < file_size);
01289
01290 const std::string content(buffer, bytes_read);
01291 delete[] buffer;
01292
01293 return content;
01294 }
01295
01296 #if GTEST_HAS_DEATH_TEST
01297 static const std::vector<std::string>* g_injected_test_args =
01298 nullptr; // Owned.
01299
01300 std::vector<std::string> GetInjectableArgs() {
01301 if (g_injected_test_args != nullptr) {
01302 return *g_injected_test_args;
01303 }
01304 return GetArgs();
01305 }
01306
01307 void SetInjectableArgs(const std::vector<std::string>* new_args) {
01308 if (g_injected_test_args != new_args) delete g_injected_test_args;
01309 g_injected_test_args = new_args;
01310 }
01311
01312 void SetInjectableArgs(const std::vector<std::string>& new_args) {
01313 SetInjectableArgs(
01314 new std::vector<std::string>(new_args.begin(), new_args.end()));
01315 }
01316
01317 void ClearInjectableArgs() {
01318 delete g_injected_test_args;
01319 g_injected_test_args = nullptr;
01320 }
01321 #endif // GTEST_HAS_DEATH_TEST
01322
01323 #if GTEST_OS_WINDOWS_MOBILE
01324 namespace posix {
01325
01326 void Abort() {
01327 DebugBreak();
01328 TerminateProcess(GetCurrentProcess(), 1);
01329 }
01330 } // namespace posix
01331 #endif // GTEST_OS_WINDOWS_MOBILE
01332
01333 // Returns the name of the environment variable corresponding to the
01334 // given flag. For example, FlagToEnvVar("foo") will return
01335 // "GTEST_FOO" in the open-source version.
01336
01337 static std::string FlagToEnvVar(const char* flag) {
01338 const std::string full_flag =
01339 (Message() << GTEST_FLAG_PREFIX_ << flag).GetString();
01340
01341 Message env_var;
01342 for (size_t i = 0; i != full_flag.length(); i++) {
01343 env_var << ToUpper(full_flag.c_str()[i]);
01344 }
01345
01346 return env_var.GetString();
01347 }
01348
01349 // Parses 'str' for a 32-bit signed integer. If successful, writes
01350 // the result to *value and returns true; otherwise leaves *value
01351 // unchanged and returns false.
01352
01353 bool ParseInt32(const Message& src_text, const char* str, int32_t* value) {
01354 // Parses the environment variable as a decimal integer.

```

```

01355 char* end = nullptr;
01356 const long long_value = strtol(str, &end, 10); // NOLINT
01357
01358 // Has strtol() consumed all characters in the string?
01359 if (*end != '\0') {
01360 // No - an invalid character was encountered.
01361 Message msg;
01362 msg << "WARNING: " << src_text
01363 << " is expected to be a 32-bit integer, but actually"
01364 << " has value \"" << str << "\".\n";
01365 printf("%s", msg.GetString().c_str());
01366 fflush(stdout);
01367 return false;
01368 }
01369
01370 // Is the parsed value in the range of an int32_t?
01371 const auto result = static_cast<int32_t>(long_value);
01372 if (long_value == LONG_MAX || long_value == LONG_MIN ||
01373 // The parsed value overflows as a long. (strtol() returns
01374 // LONG_MAX or LONG_MIN when the input overflows.)
01375 result != long_value
01376 // The parsed value overflows as an int32_t.
01377) {
01378 Message msg;
01379 msg << "WARNING: " << src_text
01380 << " is expected to be a 32-bit integer, but actually"
01381 << " has value " << str << ", which overflows.\n";
01382 printf("%s", msg.GetString().c_str());
01383 fflush(stdout);
01384 return false;
01385 }
01386
01387 *value = result;
01388 return true;
01389 }
01390
01391 // Reads and returns the Boolean environment variable corresponding to
01392 // the given flag; if it's not set, returns default_value.
01393 //
01394 // The value is considered true if and only if it's not "0".
01395
01396 bool BoolFromGTestEnv(const char* flag, bool default_value) {
01397 #if defined(GTEST_GET_BOOL_FROM_ENV_)
01398 return GTEST_GET_BOOL_FROM_ENV_(flag, default_value);
01399 #else
01400 const std::string env_var = FlagToEnvVar(flag);
01401 const char* const string_value = posix::GetEnv(env_var.c_str());
01402 return string_value == nullptr ? default_value
01403 : strcmp(string_value, "0") != 0;
01404 #endif // defined(GTEST_GET_BOOL_FROM_ENV_)
01405 }
01406
01407 // Reads and returns a 32-bit integer stored in the environment
01408 // variable corresponding to the given flag; if it isn't set or
01409 // doesn't represent a valid 32-bit integer, returns default_value.
01410
01411 int32_t Int32FromGTestEnv(const char* flag, int32_t default_value) {
01412 #if defined(GTEST_GET_INT32_FROM_ENV_)
01413 return GTEST_GET_INT32_FROM_ENV_(flag, default_value);
01414 #else
01415 const std::string env_var = FlagToEnvVar(flag);
01416 const char* const string_value = posix::GetEnv(env_var.c_str());
01417 if (string_value == nullptr) {
01418 // The environment variable is not set.
01419 return default_value;
01420 }
01421
01422 int32_t result = default_value;
01423 if (!ParseInt32(Message() << "Environment variable " << env_var, string_value,
01424 &result)) {
01425 printf("The default value %s is used.\n",
01426 (Message() << default_value).GetString().c_str());
01427 fflush(stdout);
01428 return default_value;
01429 }
01430
01431 return result;
01432 #endif // defined(GTEST_GET_INT32_FROM_ENV_)
01433 }
01434
01435 // As a special case for the 'output' flag, if GTEST_OUTPUT is not
01436 // set, we look for XML_OUTPUT_FILE, which is set by the Bazel build
01437 // system. The value of XML_OUTPUT_FILE is a filename without the
01438 // "xml:" prefix of GTEST_OUTPUT.
01439 // Note that this is meant to be called at the call site so it does
01440 // not check that the flag is 'output'
01441 // In essence this checks an env variable called XML_OUTPUT_FILE

```

```

01442 // and if it is set we prepend "xml:" to its value, if it not set we return ""
01443
01444 std::string OutputFlagAlsoCheckEnvVar() {
01445 std::string default_value_for_output_flag = "";
01446 const char* xml_output_file_env = posix::GetEnv("XML_OUTPUT_FILE");
01447 if (nullptr != xml_output_file_env) {
01448 default_value_for_output_flag = std::string("xml:") + xml_output_file_env;
01449 }
01450 return default_value_for_output_flag;
01451 }
01452
01453 // Reads and returns the string environment variable corresponding to
01454 // the given flag; if it's not set, returns default_value.
01455
01456 const char* StringFromGTestEnv(const char* flag, const char* default_value) {
01457 #if defined(GTEST_GET_STRING_FROM_ENV_)
01458 return GTEST_GET_STRING_FROM_ENV_(flag, default_value);
01459 #else
01460 const std::string env_var = FlagToEnvVar(flag);
01461 const char* const value = posix::GetEnv(env_var.c_str());
01462 return value == nullptr ? default_value : value;
01463 #endif // defined(GTEST_GET_STRING_FROM_ENV_)
01464 }
01465 } // namespace internal
01466 } // namespace testing

```

## 16.260 gtest-printers.cc

```

00001 // Copyright 2007, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 // Google Test - The Google C++ Testing and Mocking Framework
00031 //
00032 // This file implements a universal value printer that can print a
00033 // value of any type T:
00034 //
00035 // void ::testing::internal::UniversalPrinter<T>::Print(value, ostream_ptr);
00036 //
00037 // It uses the « operator when possible, and prints the bytes in the
00038 // object otherwise. A user can override its behavior for a class
00039 // type Foo by defining either operator«(::std::ostream&, const Foo&)
00040 // or void PrintTo(const Foo&, ::std::ostream*) in the namespace that
00041 // defines Foo.
00042
00043 #include "gtest/gtest-printers.h"
00044
00045 #include <stdio.h>
00046
00047 #include <cctype>
00048 #include <cstdint>
00049 #include <wchar>
00050 #include <ostream> // NOLINT
00051 #include <string>
00052 #include <type_traits>
00053
00054 #include "gtest/internal/gtest-port.h"
00055 #include "src/gtest-internal-inl.h"
00056

```

```

00057 namespace testing {
00058
00059 namespace {
00060
00061 using ::std::ostream;
00062
00063 // Prints a segment of bytes in the given object.
00064
00065 GTEST_ATTRIBUTE_NO_SANITIZE_MEMORY_
00066 GTEST_ATTRIBUTE_NO_SANITIZE_ADDRESS_
00067 GTEST_ATTRIBUTE_NO_SANITIZE_HWADDRESS_
00068 GTEST_ATTRIBUTE_NO_SANITIZE_THREAD_
00069 void PrintByteSegmentInObjectTo(const unsigned char* obj_bytes, size_t start,
00070 size_t count, ostream* os) {
00071 char text[5] = "";
00072 for (size_t i = 0; i != count; i++) {
00073 const size_t j = start + i;
00074 if (i != 0) {
00075 // Organizes the bytes into groups of 2 for easy parsing by
00076 // human.
00077 if ((j % 2) == 0)
00078 *os << ' ';
00079 else
00080 *os << '-';
00081 }
00082 GTEST_SNPRINTF_(text, sizeof (text), "%02X", obj_bytes[j]);
00083 *os << text;
00084 }
00085 }
00086
00087 // Prints the bytes in the given value to the given ostream.
00088
00089 void PrintBytesInObjectToImpl(const unsigned char* obj_bytes, size_t count,
00090 ostream* os) {
00091 // Tells the user how big the object is.
00092 *os << count << "-byte object <";
00093
00094 const size_t kThreshold = 132;
00095 const size_t kChunkSize = 64;
00096 // If the object size is bigger than kThreshold, we'll have to omit
00097 // some details by printing only the first and the last kChunkSize
00098 // bytes.
00099 if (count < kThreshold) {
00100 PrintByteSegmentInObjectTo(obj_bytes, 0, count, os);
00101 } else {
00102 PrintByteSegmentInObjectTo(obj_bytes, 0, kChunkSize, os);
00103 *os << " ... ";
00104 // Rounds up to 2-byte boundary.
00105 const size_t resume_pos = (count - kChunkSize + 1) / 2 * 2;
00106 PrintByteSegmentInObjectTo(obj_bytes, resume_pos, count - resume_pos, os);
00107 }
00108 *os << ">";
00109 }
00110
00111 // Helpers for widening a character to char32_t. Since the standard does not
00112 // specify if char / wchar_t is signed or unsigned, it is important to first
00113 // convert it to the unsigned type of the same width before widening it to
00114 // char32_t.
00115
00116 template <typename CharType>
00117 char32_t ToChar32(CharType in) {
00118 return static_cast<char32_t> (
00119 static_cast<typename std::make_unsigned<CharType>::type> (in));
00120 }
00121 } // namespace
00122 } // namespace
00123
00124 namespace internal {
00125
00126 // Delegates to PrintBytesInObjectToImpl() to print the bytes in the
00127 // given object. The delegation simplifies the implementation, which
00128 // uses the << operator and thus is easier done outside of the
00129 // ::testing::internal namespace, which contains a << operator that
00130 // sometimes conflicts with the one in STL.
00131
00132 void PrintBytesInObjectTo(const unsigned char* obj_bytes, size_t count,
00133 ostream* os) {
00134 PrintBytesInObjectToImpl(obj_bytes, count, os);
00135 }
00136
00137 // Depending on the value of a char (or wchar_t), we print it in one
00138 // of three formats:
00139 // - as is if it's a printable ASCII (e.g. 'a', '2', ' '),
00140 // - as a hexadecimal escape sequence (e.g. '\x7F'), or
00141 // - as a special escape sequence (e.g. '\r', '\n').
00142
00143 enum CharFormat {

```

```

00144 kAsIs, kHexEscape, kSpecialEscape
00145 };
00146
00147 // Returns true if c is a printable ASCII character. We test the
00148 // value of c directly instead of calling isprint(), which is buggy on
00149 // Windows Mobile.
00150
00151 inline bool IsPrintableAscii(char32_t c) {
00152 return 0x20 <= c && c <= 0x7E;
00153 }
00154
00155 // Prints c (of type char, char8_t, char16_t, char32_t, or wchar_t) as a
00156 // character literal without the quotes, escaping it when necessary; returns how
00157 // c was formatted.
00158
00159 template <typename Char>
00160 static CharFormat PrintAsCharLiteralTo(Char c, ostream* os) {
00161 const char32_t u_c = ToChar32(c);
00162 switch (u_c) {
00163 case L'\0':
00164 *os << "\\0";
00165 break;
00166 case L'\'':
00167 *os << "\\'";
00168 break;
00169 case L'\\':
00170 *os << "\\\"";
00171 break;
00172 case L'\a':
00173 *os << "\\a";
00174 break;
00175 case L'\b':
00176 *os << "\\b";
00177 break;
00178 case L'\f':
00179 *os << "\\f";
00180 break;
00181 case L'\n':
00182 *os << "\\n";
00183 break;
00184 case L'\r':
00185 *os << "\\r";
00186 break;
00187 case L'\t':
00188 *os << "\\t";
00189 break;
00190 case L'\v':
00191 *os << "\\v";
00192 break;
00193 default:
00194 if (IsPrintableAscii(u_c)) {
00195 *os << static_cast<char>(c);
00196 return kAsIs;
00197 } else {
00198 ostream::fmtflags flags = os->flags();
00199 *os << "\\x" << std::hex << std::uppercase << static_cast<int>(u_c);
00200 os->flags(flags);
00201 return kHexEscape;
00202 }
00203 }
00204 return kSpecialEscape;
00205 }
00206
00207 // Prints a char32_t c as if it's part of a string literal, escaping it when
00208 // necessary; returns how c was formatted.
00209
00210 static CharFormat PrintAsStringLiteralTo(char32_t c, ostream* os) {
00211 switch (c) {
00212 case L'\'':
00213 *os << "'";
00214 return kAsIs;
00215 case L'\"':
00216 *os << "\\\"";
00217 return kSpecialEscape;
00218 default:
00219 return PrintAsCharLiteralTo(c, os);
00220 }
00221 }
00222
00223 static const char* GetCharWidthPrefix(char) {
00224 return "";
00225 }
00226
00227 static const char* GetCharWidthPrefix(signed char) {
00228 return "";
00229 }
00230

```



```

00231 static const char* GetCharWidthPrefix(unsigned char) {
00232 return "";
00233 }
00234
00235 #ifdef __cpp_char8_t
00236
00237 static const char* GetCharWidthPrefix(char8_t) {
00238 return "u8";
00239 }
00240 #endif
00241
00242 static const char* GetCharWidthPrefix(char16_t) {
00243 return "u";
00244 }
00245
00246 static const char* GetCharWidthPrefix(char32_t) {
00247 return "U";
00248 }
00249
00250 static const char* GetCharWidthPrefix(wchar_t) {
00251 return "L";
00252 }
00253
00254 // Prints a char c as if it's part of a string literal, escaping it when
00255 // necessary; returns how c was formatted.
00256
00257 static CharFormat PrintAsStringLiteralTo(char c, ostream* os) {
00258 return PrintAsStringLiteralTo(ToChar32(c), os);
00259 }
00260
00261 #ifdef __cpp_char8_t
00262
00263 static CharFormat PrintAsStringLiteralTo(char8_t c, ostream* os) {
00264 return PrintAsStringLiteralTo(ToChar32(c), os);
00265 }
00266 #endif
00267
00268 static CharFormat PrintAsStringLiteralTo(char16_t c, ostream* os) {
00269 return PrintAsStringLiteralTo(ToChar32(c), os);
00270 }
00271
00272 static CharFormat PrintAsStringLiteralTo(wchar_t c, ostream* os) {
00273 return PrintAsStringLiteralTo(ToChar32(c), os);
00274 }
00275
00276 // Prints a character c (of type char, char8_t, char16_t, char32_t, or wchar_t)
00277 // and its code. '\0' is printed as "'\\0'", other unprintable characters are
00278 // also properly escaped using the standard C++ escape sequence.
00279
00280 template <typename Char>
00281 void PrintCharAndCodeTo(Char c, ostream* os) {
00282 // First, print c as a literal in the most readable form we can find.
00283 *os << GetCharWidthPrefix(c) << "'";
00284 const CharFormat format = PrintAsCharLiteralTo(c, os);
00285 *os << "'";
00286
00287 // To aid user debugging, we also print c's code in decimal, unless
00288 // it's 0 (in which case c was printed as "'\\0'", making the code
00289 // obvious).
00290 if (c == 0) return;
00291 *os << " (" << static_cast<int>(c);
00292
00293 // For more convenience, we print c's code again in hexadecimal,
00294 // unless c was already printed in the form '\x##' or the code is in
00295 // [1, 9].
00296 if (format == kHexEscape || (1 <= c && c <= 9)) {
00297 // Do nothing.
00298 } else {
00299 *os << ", 0x" << String::FormatHexInt(static_cast<int>(c));
00300 }
00301 *os << ")";
00302 }
00303
00304 void PrintTo(unsigned char c, ::std::ostream* os) {
00305 PrintCharAndCodeTo(c, os);
00306 }
00307
00308 void PrintTo(signed char c, ::std::ostream* os) {
00309 PrintCharAndCodeTo(c, os);
00310 }
00311
00312 // Prints a wchar_t as a symbol if it is printable or as its internal
00313 // code otherwise and also as its code. L'\0' is printed as "L'\0'".
00314
00315 void PrintTo(wchar_t wc, ostream* os) {
00316 PrintCharAndCodeTo(wc, os);
00317 }

```

```

00318
00319 // TODO(dcheng): Consider making this delegate to PrintCharAndCodeTo() as well.
00320
00321 void PrintTo(char32_t c, ::std::ostream& os) {
00322 *os << std::hex << "U+" << std::uppercase << std::setfill('0') << std::setw(4)
00323 << static_cast<uint32_t>(c);
00324 }
00325
00326 // gcc/clang __{u,}int128_t
00327 #if defined(__SIZEOF_INT128__)
00328
00329 void PrintTo(__uint128_t v, ::std::ostream& os) {
00330 if (v == 0) {
00331 *os << "0";
00332 return;
00333 }
00334
00335 // Buffer large enough for ceil(log10(2^128))==39 and the null terminator
00336 char buf[40];
00337 char* p = buf + sizeof(buf);
00338
00339 // Some configurations have a __uint128_t, but no support for built in
00340 // division. Do manual long division instead.
00341
00342 uint64_t high = static_cast<uint64_t>(v >> 64);
00343 uint64_t low = static_cast<uint64_t>(v);
00344
00345 *--p = 0;
00346 while (high != 0 || low != 0) {
00347 uint64_t high_mod = high % 10;
00348 high = high / 10;
00349 // This is the long division algorithm specialized for a divisor of 10 and
00350 // only two elements.
00351 // Notable values:
00352 // 2^64 / 10 == 1844674407370955161
00353 // 2^64 % 10 == 6
00354 const uint64_t carry = 6 * high_mod + low % 10;
00355 low = low / 10 + high_mod * 1844674407370955161 + carry / 10;
00356
00357 char digit = static_cast<char>(carry % 10);
00358 *--p = '0' + digit;
00359 }
00360 *os << p;
00361 }
00362
00363 void PrintTo(__int128_t v, ::std::ostream& os) {
00364 __uint128_t uv = static_cast<__uint128_t>(v);
00365 if (v < 0) {
00366 *os << "-";
00367 uv = -uv;
00368 }
00369 PrintTo(uv, os);
00370 }
00371 #endif // __SIZEOF_INT128__
00372
00373 // Prints the given array of characters to the ostream. CharType must be either
00374 // char, char8_t, char16_t, char32_t, or wchar_t.
00375 // The array starts at begin, the length is len, it may include '\0' characters
00376 // and may not be NUL-terminated.
00377
00378 template <typename CharType>
00379 GTEST_ATTRIBUTE_NO_SANITIZE_MEMORY_ GTEST_ATTRIBUTE_NO_SANITIZE_ADDRESS_
00380 GTEST_ATTRIBUTE_NO_SANITIZE_HWADDRESS_
00381 GTEST_ATTRIBUTE_NO_SANITIZE_THREAD_ static CharFormat
00382 PrintCharsAsStringTo(const CharType* begin, size_t len, ostream& os) {
00383 const char* const quote_prefix = GetCharWidthPrefix(*begin);
00384 *os << quote_prefix << "\"";
00385 bool is_previous_hex = false;
00386 CharFormat print_format = kAsIs;
00387 for (size_t index = 0; index < len; ++index) {
00388 const CharType cur = begin[index];
00389 if (is_previous_hex && IsXDigit(cur)) {
00390 // Previous character is of '\x..' form and this character can be
00391 // interpreted as another hexadecimal digit in its number. Break string to
00392 // disambiguate.
00393 *os << "\" " << quote_prefix << "\"";
00394 is_previous_hex = PrintAsStringLiteralTo(cur, os) == kHexEscape;
00395 // Remember if any characters required hex escaping.
00396 if (is_previous_hex) {
00397 print_format = kHexEscape;
00398 }
00399 }
00400 }
00401 *os << "\"";
00402 return print_format;
00403 }
00404

```

```

00405 // Prints a (const) char/wchar_t array of 'len' elements, starting at address
00406 // 'begin'. CharType must be either char or wchar_t.
00407
00408 template <typename CharType>
00409 GTEST_ATTRIBUTE_NO_SANITIZE_MEMORY_ GTEST_ATTRIBUTE_NO_SANITIZE_ADDRESS_
00410 GTEST_ATTRIBUTE_NO_SANITIZE_HWADDRESS_
00411 GTEST_ATTRIBUTE_NO_SANITIZE_THREAD_ static void
00412 UniversalPrintCharArray(const CharType* begin, size_t len,
00413 ostream* os) {
00414 // The code
00415 // const char kFoo[] = "foo";
00416 // generates an array of 4, not 3, elements, with the last one being '\0'.
00417 // Therefore when printing a char array, we don't print the last element if
00418 // it's '\0', such that the output matches the string literal as it's
00419 // written in the source code.
00420 if (len > 0 && begin[len - 1] == '\0') {
00421 PrintCharsAsStringTo(begin, len - 1, os);
00422 return;
00423 }
00424
00425 // If, however, the last element in the array is not '\0', e.g.
00426 // const char kFoo[] = { 'f', 'o', 'o' };
00427 // we must print the entire array. We also print a message to indicate
00428 // that the array is not NUL-terminated.
00429 PrintCharsAsStringTo(begin, len, os);
00430 *os << " (no terminating NUL)";
00431 }
00432
00433 // Prints a (const) char array of 'len' elements, starting at address 'begin'.
00434
00435 void UniversalPrintArray(const char* begin, size_t len, ostream* os) {
00436 UniversalPrintCharArray(begin, len, os);
00437 }
00438
00439 #ifdef __cpp_char8_t
00440 // Prints a (const) char8_t array of 'len' elements, starting at address
00441 // 'begin'.
00442 void UniversalPrintArray(const char8_t* begin, size_t len, ostream* os) {
00443 UniversalPrintCharArray(begin, len, os);
00444 }
00445 #endif
00446
00447 // Prints a (const) char16_t array of 'len' elements, starting at address
00448 // 'begin'.
00449 void UniversalPrintArray(const char16_t* begin, size_t len, ostream* os) {
00450 UniversalPrintCharArray(begin, len, os);
00451 }
00452
00453 // Prints a (const) char32_t array of 'len' elements, starting at address
00454 // 'begin'.
00455 void UniversalPrintArray(const char32_t* begin, size_t len, ostream* os) {
00456 UniversalPrintCharArray(begin, len, os);
00457 }
00458
00459 // Prints a (const) wchar_t array of 'len' elements, starting at address
00460 // 'begin'.
00461 void UniversalPrintArray(const wchar_t* begin, size_t len, ostream* os) {
00462 UniversalPrintCharArray(begin, len, os);
00463 }
00464
00465 namespace {
00466
00467 // Prints a null-terminated C-style string to the ostream.
00468
00469 template <typename Char>
00470 void PrintCStringTo(const Char* s, ostream* os) {
00471 if (s == nullptr) {
00472 *os << "NULL";
00473 } else {
00474 *os << ImplicitCast_<const void*>(s) << " pointing to ";
00475 PrintCharsAsStringTo(s, std::char_traits<Char>::length(s), os);
00476 }
00477 }
00478 } // anonymous namespace
00479
00480 void PrintTo(const char* s, ostream* os) {
00481 PrintCStringTo(s, os);
00482 }
00483
00484 #ifdef __cpp_char8_t
00485
00486 void PrintTo(const char8_t* s, ostream* os) {
00487 PrintCStringTo(s, os);
00488 }
00489
00490 #endif
00491

```

```

00492 void PrintTo(const char8_t* s, ostream* os) {
00493 PrintCStringTo(s, os);
00494 }
00495 #endif
00496
00497 void PrintTo(const char16_t* s, ostream* os) {
00498 PrintCStringTo(s, os);
00499 }
00500
00501 void PrintTo(const char32_t* s, ostream* os) {
00502 PrintCStringTo(s, os);
00503 }
00504
00505 // MSVC compiler can be configured to define wchar_t as a typedef
00506 // of unsigned short. Defining an overload for const wchar_t* in that case
00507 // would cause pointers to unsigned shorts be printed as wide strings,
00508 // possibly accessing more memory than intended and causing invalid
00509 // memory accesses. MSVC defines _NATIVE_WCHAR_T_DEFINED symbol when
00510 // wchar_t is implemented as a native type.
00511 #if !defined(_MSC_VER) || defined(_NATIVE_WCHAR_T_DEFINED)
00512 // Prints the given wide C string to the ostream.
00513
00514 void PrintTo(const wchar_t* s, ostream* os) {
00515 PrintCStringTo(s, os);
00516 }
00517 #endif // wchar_t is native
00518
00519 namespace {
00520
00521 bool ContainsUnprintableControlCodes(const char* str, size_t length) {
00522 const unsigned char* s = reinterpret_cast<const unsigned char*> (str);
00523
00524 for (size_t i = 0; i < length; i++) {
00525 unsigned char ch = *s++;
00526 if (std::iscntrl(ch)) {
00527 switch (ch) {
00528 case '\t':
00529 case '\n':
00530 case '\r':
00531 break;
00532 default:
00533 return true;
00534 }
00535 }
00536 }
00537 return false;
00538 }
00539
00540 bool IsUTF8TrailByte(unsigned char t) {
00541 return 0x80 <= t && t <= 0xbf;
00542 }
00543
00544 bool IsValidUTF8(const char* str, size_t length) {
00545 const unsigned char* s = reinterpret_cast<const unsigned char*> (str);
00546
00547 for (size_t i = 0; i < length; i++) {
00548 unsigned char lead = s[i++];
00549
00550 if (lead <= 0x7f) {
00551 continue; // single-byte character (ASCII) 0..7F
00552 }
00553 if (lead < 0xc2) {
00554 return false; // trail byte or non-shortest form
00555 } else if (lead <= 0xdf && (i + 1) <= length && IsUTF8TrailByte(s[i])) {
00556 ++i; // 2-byte character
00557 } else if (0xe0 <= lead && lead <= 0xef && (i + 2) <= length &&
00558 IsUTF8TrailByte(s[i]) && IsUTF8TrailByte(s[i + 1]) &&
00559 // check for non-shortest form and surrogate
00560 (lead != 0xe0 || s[i] >= 0xa0) &&
00561 (lead != 0xed || s[i] < 0xa0)) {
00562 i += 2; // 3-byte character
00563 } else if (0xf0 <= lead && lead <= 0xf4 && (i + 3) <= length &&
00564 IsUTF8TrailByte(s[i]) && IsUTF8TrailByte(s[i + 1]) &&
00565 IsUTF8TrailByte(s[i + 2]) &&
00566 // check for non-shortest form
00567 (lead != 0xf0 || s[i] >= 0x90) &&
00568 (lead != 0xf4 || s[i] < 0x90)) {
00569 i += 3; // 4-byte character
00570 } else {
00571 return false;
00572 }
00573 }
00574 return true;
00575 }
00576
00577 void ConditionalPrintAsText(const char* str, size_t length, ostream* os) {
00578 if (!ContainsUnprintableControlCodes(str, length) &&

```

```

00579 IsValidUTF8(str, length)) {
00580 *os << "\n As Text: \"" << str << "\"";
00581 }
00582 }
00583
00584 } // anonymous namespace
00585
00586 void PrintStringTo(const ::std::string& s, ostream* os) {
00587 if (PrintCharsAsStringTo(s.data(), s.size(), os) == kHexEscape) {
00588 if (GTEST_FLAG_GET(print_utf8)) {
00589 ConditionalPrintAsText(s.data(), s.size(), os);
00590 }
00591 }
00592 }
00593
00594 #ifdef __cpp_char8_t
00595
00596 void PrintU8StringTo(const ::std::u8string& s, ostream* os) {
00597 PrintCharsAsStringTo(s.data(), s.size(), os);
00598 }
00599 #endif
00600
00601 void PrintU16StringTo(const ::std::u16string& s, ostream* os) {
00602 PrintCharsAsStringTo(s.data(), s.size(), os);
00603 }
00604
00605 void PrintU32StringTo(const ::std::u32string& s, ostream* os) {
00606 PrintCharsAsStringTo(s.data(), s.size(), os);
00607 }
00608
00609 #if GTEST_HAS_STD_WSTRING
00610
00611 void PrintWideStringTo(const ::std::wstring& s, ostream* os) {
00612 PrintCharsAsStringTo(s.data(), s.size(), os);
00613 }
00614 #endif // GTEST_HAS_STD_WSTRING
00615
00616 } // namespace internal
00617
00618 } // namespace testing

```

## 16.261 gtest-test-part.cc

```

00001 // Copyright 2008, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 //
00031 // The Google C++ Testing and Mocking Framework (Google Test)
00032
00033 #include "gtest/gtest-test-part.h"
00034
00035 #include "gtest/internal/gtest-port.h"
00036 #include "src/gtest-internal-inl.h"
00037
00038 namespace testing {
00039
00040 using internal::GetUnitTestImpl;
00041
00042 // Gets the summary of the failure message by omitting the stack trace

```

```

00043 // in it.
00044
00045 std::string TestPartResult::ExtractSummary(const char* message) {
00046 const char* const stack_trace = strstr(message, internal::kStackTraceMarker);
00047 return stack_trace == nullptr ? message : std::string(message, stack_trace);
00048 }
00049
00050 // Prints a TestPartResult object.
00051
00052 std::ostream& operator<<(std::ostream& os, const TestPartResult& result) {
00053 return os << internal::FormatFileLocation(result.file_name(),
00054 result.line_number())
00055 << " "
00056 << (result.type() == TestPartResult::kSuccess ? "Success"
00057 : result.type() == TestPartResult::kSkip ? "Skipped"
00058 : result.type() == TestPartResult::kFatalFailure
00059 ? "Fatal failure"
00060 : "Non-fatal failure")
00061 << ":\n"
00062 << result.message() << std::endl;
00063 }
00064
00065 // Appends a TestPartResult to the array.
00066
00067 void TestPartResultArray::Append(const TestPartResult& result) {
00068 array_.push_back(result);
00069 }
00070
00071 // Returns the TestPartResult at the given index (0-based).
00072
00073 const TestPartResult& TestPartResultArray::GetTestPartResult(int index) const {
00074 if (index < 0 || index >= size()) {
00075 printf("\nInvalid index (%d) into TestPartResultArray.\n", index);
00076 internal::posix::Abort();
00077 }
00078
00079 return array_[static_cast<size_t>(index)];
00080 }
00081
00082 // Returns the number of TestPartResult objects in the array.
00083
00084 int TestPartResultArray::size() const {
00085 return static_cast<int>(array_.size());
00086 }
00087
00088 namespace internal {
00089
00090 HasNewFatalFailureHelper::HasNewFatalFailureHelper()
00091 : has_new_fatal_failure_(false),
00092 original_reporter_(
00093 GetUnitTestImpl()->GetTestPartResultReporterForCurrentThread()) {
00094 GetUnitTestImpl()->SetTestPartResultReporterForCurrentThread(this);
00095 }
00096
00097 HasNewFatalFailureHelper::~HasNewFatalFailureHelper() {
00098 GetUnitTestImpl()->SetTestPartResultReporterForCurrentThread(
00099 original_reporter_);
00100 }
00101
00102 void HasNewFatalFailureHelper::ReportTestPartResult(
00103 const TestPartResult& result) {
00104 if (result.fatally_failed()) has_new_fatal_failure_ = true;
00105 original_reporter_->ReportTestPartResult(result);
00106 }
00107
00108 } // namespace internal
00109
00110 } // namespace testing

```

## 16.262 gtest-typed-test.cc

```

00001 // Copyright 2008 Google Inc.
00002 // All Rights Reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its

```

```

00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 #include "gtest/gtest-typed-test.h"
00031
00032 #include "gtest/gtest.h"
00033
00034 namespace testing {
00035 namespace internal {
00036
00037 // Skips to the first non-space char in str. Returns an empty string if str
00038 // contains only whitespace characters.
00039
00040 static const char* SkipSpaces(const char* str) {
00041 while (IsSpace(*str)) str++;
00042 return str;
00043 }
00044
00045 static std::vector<std::string> SplitIntoTestNames(const char* src) {
00046 std::vector<std::string> name_vec;
00047 src = SkipSpaces(src);
00048 for (; src != nullptr; src = SkipComma(src)) {
00049 name_vec.push_back(StripTrailingSpaces(GetPrefixUntilComma(src)));
00050 }
00051 return name_vec;
00052 }
00053
00054 // Verifies that registered_tests match the test names in
00055 // registered_tests_; returns registered_tests if successful, or
00056 // aborts the program otherwise.
00057
00058 const char* TypedTestSuitePState::VerifyRegisteredTestNames(
00059 const char* test_suite_name, const char* file, int line,
00060 const char* registered_tests) {
00061 RegisterTypeParameterizedTestSuite(test_suite_name, CodeLocation(file, line));
00062
00063 typedef RegisteredTestsMap::const_iterator RegisteredTestIter;
00064 registered_ = true;
00065
00066 std::vector<std::string> name_vec = SplitIntoTestNames(registered_tests);
00067
00068 Message errors;
00069
00070 std::set<std::string> tests;
00071 for (std::vector<std::string>::const_iterator name_it = name_vec.begin();
00072 name_it != name_vec.end(); ++name_it) {
00073 const std::string& name = *name_it;
00074 if (tests.count(name) != 0) {
00075 errors << "Test " << name << " is listed more than once.\n";
00076 continue;
00077 }
00078
00079 if (registered_tests_.count(name) != 0) {
00080 tests.insert(name);
00081 } else {
00082 errors << "No test named " << name
00083 << " can be found in this test suite.\n";
00084 }
00085 }
00086
00087 for (RegisteredTestIter it = registered_tests_.begin();
00088 it != registered_tests_.end(); ++it) {
00089 if (tests.count(it->first) == 0) {
00090 errors << "You forgot to list test " << it->first << ".\n";
00091 }
00092 }
00093
00094 const std::string& errors_str = errors.GetString();
00095 if (errors_str != "") {
00096 fprintf(stderr, "%s %s", FormatFileLocation(file, line).c_str(),
00097 errors_str.c_str());
00098 fflush(stderr);
00099 posix::Abort();
00100 }
00101 }

```

```

00102 return registered_tests;
00103 }
00104
00105 } // namespace internal
00106 } // namespace testing

```

## 16.263 gtest.cc

```

00001 // Copyright 2005, Google Inc.
00002 // All rights reserved.
00003 //
00004 // Redistribution and use in source and binary forms, with or without
00005 // modification, are permitted provided that the following conditions are
00006 // met:
00007 //
00008 // * Redistributions of source code must retain the above copyright
00009 // notice, this list of conditions and the following disclaimer.
00010 // * Redistributions in binary form must reproduce the above
00011 // copyright notice, this list of conditions and the following disclaimer
00012 // in the documentation and/or other materials provided with the
00013 // distribution.
00014 // * Neither the name of Google Inc. nor the names of its
00015 // contributors may be used to endorse or promote products derived from
00016 // this software without specific prior written permission.
00017 //
00018 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
00019 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
00020 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
00021 // A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
00022 // OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00023 // SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00024 // LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00025 // DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00026 // THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00027 // (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00028 // OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00029
00030 //
00031 // The Google C++ Testing and Mocking Framework (Google Test)
00032
00033 #include "gtest/gtest.h"
00034
00035 #include <ctype.h>
00036 #include <stdarg.h>
00037 #include <stdio.h>
00038 #include <stdlib.h>
00039 #include <time.h>
00040 #include <wchar.h>
00041 #include <wctype.h>
00042
00043 #include <algorithm>
00044 #include <chrono> // NOLINT
00045 #include <cmath>
00046 #include <cstdint>
00047 #include <initializer_list>
00048 #include <iomanip>
00049 #include <iterator>
00050 #include <limits>
00051 #include <list>
00052 #include <map>
00053 #include <ostream> // NOLINT
00054 #include <sstream>
00055 #include <unordered_set>
00056 #include <vector>
00057
00058 #include "gtest/gtest-assertion-result.h"
00059 #include "gtest/gtest-spi.h"
00060 #include "gtest/internal/custom/gtest.h"
00061
00062 #if GTEST_OS_LINUX
00063
00064 #include <fcntl.h> // NOLINT
00065 #include <limits.h> // NOLINT
00066 #include <sched.h> // NOLINT
00067 // Declares vsnprintf(). This header is not available on Windows.
00068 #include <strings.h> // NOLINT
00069 #include <sys/mman.h> // NOLINT
00070 #include <sys/time.h> // NOLINT
00071 #include <unistd.h> // NOLINT
00072
00073 #include <string>
00074
00075 #elif GTEST_OS_ZOS
00076 #include <sys/time.h> // NOLINT
00077

```



```

00078 // On z/OS we additionally need strings.h for strcasecmp.
00079 #include <strings.h> // NOLINT
00080
00081 #elif GTEST_OS_WINDOWS_MOBILE // We are on Windows CE.
00082
00083 #include <windows.h> // NOLINT
00084 #undef min
00085
00086 #elif GTEST_OS_WINDOWS // We are on Windows proper.
00087
00088 #include <windows.h> // NOLINT
00089 #undef min
00090
00091 #ifdef _MSC_VER
00092 #include <crtDBG.h> // NOLINT
00093 #endif
00094
00095 #include <io.h> // NOLINT
00096 #include <sys/stat.h> // NOLINT
00097 #include <sys/timeb.h> // NOLINT
00098 #include <sys/types.h> // NOLINT
00099
00100 #if GTEST_OS_WINDOWS_MINGW
00101 #include <sys/time.h> // NOLINT
00102 #endif // GTEST_OS_WINDOWS_MINGW
00103
00104 #else
00105
00106 // cpplint thinks that the header is already included, so we want to
00107 // silence it.
00108 #include <sys/time.h> // NOLINT
00109 #include <unistd.h> // NOLINT
00110
00111 #endif // GTEST_OS_LINUX
00112
00113 #if GTEST_HAS_EXCEPTIONS
00114 #include <stdexcept>
00115 #endif
00116
00117 #if GTEST_CAN_STREAM_RESULTS_
00118 #include <arpa/inet.h> // NOLINT
00119 #include <netdb.h> // NOLINT
00120 #include <sys/socket.h> // NOLINT
00121 #include <sys/types.h> // NOLINT
00122 #endif
00123
00124 #include "src/gtest-internal-inl.h"
00125
00126 #if GTEST_OS_WINDOWS
00127 #define vsnprintf _vsnprintf
00128 #endif // GTEST_OS_WINDOWS
00129
00130 #if GTEST_OS_MAC
00131 #ifndef GTEST_OS_IOS
00132 #include <crt_externs.h>
00133 #endif
00134 #endif
00135
00136 #if GTEST_HAS_ABSL
00137 #include "absl/debugging/failure_signal_handler.h"
00138 #include "absl/debugging/stacktrace.h"
00139 #include "absl/debugging/symbolize.h"
00140 #include "absl/flags/parse.h"
00141 #include "absl/flags/usage.h"
00142 #include "absl/strings/str_cat.h"
00143 #include "absl/strings/str_replace.h"
00144 #endif // GTEST_HAS_ABSL
00145
00146 namespace testing {
00147
00148 using internal::CountIf;
00149 using internal::ForEach;
00150 using internal::GetElementOr;
00151 using internal::Shuffle;
00152
00153 // Constants.
00154
00155 // A test whose test suite name or test name matches this filter is
00156 // disabled and not run.
00157 static const char kDisableTestFilter[] = "DISABLED_*:*/DISABLED_*";
00158
00159 // A test suite whose name matches this filter is considered a death
00160 // test suite and will be run before test suites whose name doesn't
00161 // match this filter.
00162 static const char kDeathTestSuiteFilter[] = "*DeathTest:*DeathTest/*";
00163
00164 // A test filter that matches everything.

```

```

00165 static const char kUniversalFilter[] = "*";
00166
00167 // The default output format.
00168 static const char kDefaultOutputFormat[] = "xml";
00169 // The default output file.
00170 static const char kDefaultOutputFile[] = "test_detail";
00171
00172 // The environment variable name for the test shard index.
00173 static const char kTestShardIndex[] = "GTEST_SHARD_INDEX";
00174 // The environment variable name for the total number of test shards.
00175 static const char kTestTotalShards[] = "GTEST_TOTAL_SHARDS";
00176 // The environment variable name for the test shard status file.
00177 static const char kTestShardStatusFile[] = "GTEST_SHARD_STATUS_FILE";
00178
00179 namespace internal {
00180
00181 // The text used in failure messages to indicate the start of the
00182 // stack trace.
00183 const char kStackTraceMarker[] = "\nStack trace:\n";
00184
00185 // g_help_flag is true if and only if the --help flag or an equivalent form
00186 // is specified on the command line.
00187 bool g_help_flag = false;
00188
00189 // Utility function to Open File for Writing
00190
00191 static FILE* OpenFileForWriting(const std::string& output_file) {
00192 FILE* fileout = nullptr;
00193 FilePath output_file_path(output_file);
00194 FilePath output_dir(output_file_path.RemoveFileName());
00195
00196 if (output_dir.CreateDirectoriesRecursively()) {
00197 fileout = posix::FOpen(output_file.c_str(), "w");
00198 }
00199 if (fileout == nullptr) {
00200 GTEST_LOG_(FATAL) << "Unable to open file \"" << output_file << "\"";
00201 }
00202 return fileout;
00203 }
00204 } // namespace internal
00205
00206 // Bazel passes in the argument to '--test_filter' via the TESTBRIDGE_TEST_ONLY
00207 // environment variable.
00208
00209 static const char* GetDefaultFilter() {
00210 const char* const testbridge_test_only =
00211 internal::posix::GetEnv("TESTBRIDGE_TEST_ONLY");
00212 if (testbridge_test_only != nullptr) {
00213 return testbridge_test_only;
00214 }
00215 return kUniversalFilter;
00216 }
00217
00218 // Bazel passes in the argument to '--test_runner_fail_fast' via the
00219 // TESTBRIDGE_TEST_RUNNER_FAIL_FAST environment variable.
00220
00221 static bool GetDefaultFailFast() {
00222 const char* const testbridge_test_runner_fail_fast =
00223 internal::posix::GetEnv("TESTBRIDGE_TEST_RUNNER_FAIL_FAST");
00224 if (testbridge_test_runner_fail_fast != nullptr) {
00225 return strcmp(testbridge_test_runner_fail_fast, "1") == 0;
00226 }
00227 return false;
00228 }
00229 } // namespace testing
00230
00231 } // namespace testing
00232
00233 GTEST_DEFINE_bool_(
00234 fail_fast,
00235 testing::internal::BoolFromGTestEnv("fail_fast",
00236 testing::GetDefaultFailFast()),
00237 "True if and only if a test failure should stop further test execution.");
00238
00239 GTEST_DEFINE_bool_(
00240 also_run_disabled_tests,
00241 testing::internal::BoolFromGTestEnv("also_run_disabled_tests", false),
00242 "Run disabled tests too, in addition to the tests normally being run.");
00243
00244 GTEST_DEFINE_bool_(
00245 break_on_failure,
00246 testing::internal::BoolFromGTestEnv("break_on_failure", false),
00247 "True if and only if a failed assertion should be a debugger "
00248 "break-point.");
00249
00250 GTEST_DEFINE_bool_(catch_exceptions,
00251 testing::internal::BoolFromGTestEnv("catch_exceptions",

```

```

00252 true),
00253 "True if and only if " GTEST_NAME_
00254 " should catch exceptions and treat them as test failures.");
00255
00256 GTEST_DEFINE_string_(
00257 color, testing::internal::StringFromGTestEnv("color", "auto"),
00258 "Whether to use colors in the output. Valid values: yes, no, "
00259 "and auto. 'auto' means to use colors if the output is "
00260 "being sent to a terminal and the TERM environment variable "
00261 "is set to a terminal type that supports colors.");
00262
00263 GTEST_DEFINE_string_(
00264 filter,
00265 testing::internal::StringFromGTestEnv("filter",
00266 testing::GetDefaultFilter()),
00267 "A colon-separated list of glob (not regex) patterns "
00268 "for filtering the tests to run, optionally followed by a "
00269 "'-' and a : separated list of negative patterns (tests to "
00270 "exclude). A test is run if it matches one of the positive "
00271 "patterns and does not match any of the negative patterns.");
00272
00273 GTEST_DEFINE_bool_(
00274 install_failure_signal_handler,
00275 testing::internal::BoolFromGTestEnv("install_failure_signal_handler",
00276 false),
00277 "If true and supported on the current platform, " GTEST_NAME_
00278 " should "
00279 "install a signal handler that dumps debugging information when fatal "
00280 "signals are raised.");
00281
00282 GTEST_DEFINE_bool_(list_tests, false, "List all tests without running them.");
00283
00284 // The net priority order after flag processing is thus:
00285 // --gtest_output command line flag
00286 // GTEST_OUTPUT environment variable
00287 // XML_OUTPUT_FILE environment variable
00288 // "
00289 GTEST_DEFINE_string_(
00290 output,
00291 testing::internal::StringFromGTestEnv(
00292 "output", testing::internal::OutputFlagAlsoCheckEnvVar().c_str()),
00293 "A format (defaults to 'xml' but can be specified to be 'json'), "
00294 "optionally followed by a colon and an output file name or directory. "
00295 "A directory is indicated by a trailing pathname separator. "
00296 "Examples: 'xml:filename.xml', 'xml:directoryname/'. "
00297 "If a directory is specified, output files will be created "
00298 "within that directory, with file-names based on the test "
00299 "executable's name and, if necessary, made unique by adding "
00300 "digits.");
00301
00302 GTEST_DEFINE_bool_(
00303 brief, testing::internal::BoolFromGTestEnv("brief", false),
00304 "True if only test failures should be displayed in text output.");
00305
00306 GTEST_DEFINE_bool_(print_time,
00307 testing::internal::BoolFromGTestEnv("print_time", true),
00308 "True if and only if " GTEST_NAME_
00309 " should display elapsed time in text output.");
00310
00311 GTEST_DEFINE_bool_(print_utf8,
00312 testing::internal::BoolFromGTestEnv("print_utf8", true),
00313 "True if and only if " GTEST_NAME_
00314 " prints UTF8 characters as text.");
00315
00316 GTEST_DEFINE_int32_(
00317 random_seed, testing::internal::Int32FromGTestEnv("random_seed", 0),
00318 "Random number seed to use when shuffling test orders. Must be in range "
00319 "[1, 99999], or 0 to use a seed based on the current time.");
00320
00321 GTEST_DEFINE_int32_(
00322 repeat, testing::internal::Int32FromGTestEnv("repeat", 1),
00323 "How many times to repeat each test. Specify a negative number "
00324 "for repeating forever. Useful for shaking out flaky tests.");
00325
00326 GTEST_DEFINE_bool_(
00327 recreate_environments_when_repeating,
00328 testing::internal::BoolFromGTestEnv("recreate_environments_when_repeating",
00329 false),
00330 "Controls whether global test environments are recreated for each repeat "
00331 "of the tests. If set to false the global test environments are only set "
00332 "up once, for the first iteration, and only torn down once, for the last. "
00333 "Useful for shaking out flaky tests with stable, expensive test "
00334 "environments. If --gtest_repeat is set to a negative number, meaning "
00335 "there is no last run, the environments will always be recreated to avoid "
00336 "leaks.");
00337
00338 GTEST_DEFINE_bool_(show_internal_stack_frames, false,

```

```

00339 "True if and only if " GTEST_NAME_
00340 " should include internal stack frames when "
00341 "printing test failure stack traces.");
00342
00343 GTEST_DEFINE_bool_(shuffle,
00344 testing::internal::BoolFromGTestEnv("shuffle", false),
00345 "True if and only if " GTEST_NAME_
00346 " should randomize tests' order on every run.");
00347
00348 GTEST_DEFINE_int32_(
00349 stack_trace_depth,
00350 testing::internal::Int32FromGTestEnv("stack_trace_depth",
00351 testing::kMaxStackTraceDepth),
00352 "The maximum number of stack frames to print when an "
00353 "assertion fails. The valid range is 0 through 100, inclusive.");
00354
00355 GTEST_DEFINE_string_(
00356 stream_result_to,
00357 testing::internal::StringFromGTestEnv("stream_result_to", ""),
00358 "This flag specifies the host name and the port number on which to stream "
00359 "test results. Example: \"localhost:555\". The flag is effective only on "
00360 "Linux.");
00361
00362 GTEST_DEFINE_bool_(
00363 throw_on_failure,
00364 testing::internal::BoolFromGTestEnv("throw_on_failure", false),
00365 "When this flag is specified, a failed assertion will throw an exception "
00366 "if exceptions are enabled or exit the program with a non-zero code "
00367 "otherwise. For use with an external test framework.");
00368
00369 #if GTEST_USE_OWN_FLAGFILE_FLAG_
00370 GTEST_DEFINE_string_(
00371 flagfile, testing::internal::StringFromGTestEnv("flagfile", ""),
00372 "This flag specifies the flagfile to read command-line flags from.");
00373 #endif // GTEST_USE_OWN_FLAGFILE_FLAG_
00374
00375 namespace testing {
00376 namespace internal {
00377
00378 const uint32_t Random::kMaxRange;
00379
00380 // Generates a random number from [0, range), using a Linear
00381 // Congruential Generator (LCG). Crashes if 'range' is 0 or greater
00382 // than kMaxRange.
00383
00384 uint32_t Random::Generate(uint32_t range) {
00385 // These constants are the same as are used in glibc's rand(3).
00386 // Use wider types than necessary to prevent unsigned overflow diagnostics.
00387 state_ = static_cast<uint32_t> (1103515245ULL * state_ + 12345U) % kMaxRange;
00388
00389 GTEST_CHECK_(range > 0) << "Cannot generate a number in the range [0, 0).";
00390 GTEST_CHECK_(range <= kMaxRange)
00391 << "Generation of a number in [0, " << range << ") was requested, "
00392 << "but this can only generate numbers in [0, " << kMaxRange << ").";
00393
00394 // Converting via modulus introduces a bit of downward bias, but
00395 // it's simple, and a linear congruential generator isn't too good
00396 // to begin with.
00397 return state_ % range;
00398 }
00399
00400 // GTestIsInitialized() returns true if and only if the user has initialized
00401 // Google Test. Useful for catching the user mistake of not initializing
00402 // Google Test before calling RUN_ALL_TESTS().
00403
00404 static bool GTestIsInitialized() {
00405 return GetArgvs().size() > 0;
00406 }
00407
00408 // Iterates over a vector of TestSuites, keeping a running sum of the
00409 // results of calling a given int-returning method on each.
00410 // Returns the sum.
00411
00412 static int SumOverTestSuiteList(const std::vector<TestSuite*> & case_list,
00413 int (TestSuite::*method)() const) {
00414 int sum = 0;
00415 for (size_t i = 0; i < case_list.size(); i++) {
00416 sum += (case_list[i]->*method)();
00417 }
00418 return sum;
00419 }
00420
00421 // Returns true if and only if the test suite passed.
00422
00423 static bool TestSuitePassed(const TestSuite* test_suite) {
00424 return test_suite->should_run() && test_suite->Passed();
00425 }
00426 }
}

```

```

00426
00427 // Returns true if and only if the test suite failed.
00428
00429 static bool TestSuiteFailed(const TestSuite* test_suite) {
00430 return test_suite->should_run() && test_suite->Failed();
00431 }
00432
00433 // Returns true if and only if test_suite contains at least one test that
00434 // should run.
00435
00436 static bool ShouldRunTestSuite(const TestSuite* test_suite) {
00437 return test_suite->should_run();
00438 }
00439
00440 // AssertHelper constructor.
00441
00442 AssertHelper::AssertHelper(TestPartResult::Type type, const char* file,
00443 int line, const char* message)
00444 : data_(new AssertHelperData(type, file, line, message)) {
00445 }
00446
00447 AssertHelper::~AssertHelper() {
00448 delete data_;
00449 }
00450
00451 // Message assignment, for assertion streaming support.
00452
00453 void AssertHelper::operator=(const Message& message) const {
00454 UnitTest::GetInstance()->AddTestPartResult(
00455 data_->type, data_->file, data_->line,
00456 AppendUserMessage(data_->message, message),
00457 UnitTest::GetInstance()->impl()->CurrentStackTraceExceptTop(1)
00458 // Skips the stack frame for this function itself.
00459); // NOLINT
00460 }
00461
00462 namespace {
00463
00464 // When TEST_P is found without a matching INSTANTIATE_TEST_SUITE_P
00465 // to creates test cases for it, a synthetic test case is
00466 // inserted to report either an error or a log message.
00467 //
00468 // This configuration bit will likely be removed at some point.
00469 constexpr bool kErrorOnUninstantiatedParameterizedTest = true;
00470 constexpr bool kErrorOnUninstantiatedTypeParameterizedTest = true;
00471
00472 // A test that fails at a given file/line location with a given message.
00473
00474 class FailureTest : public Test {
00475 public:
00476
00477 explicit FailureTest(const CodeLocation& loc, std::string error_message,
00478 bool as_error)
00479 : loc_(loc),
00480 error_message_(std::move(error_message)),
00481 as_error_(as_error) {
00482 }
00483
00484 void TestBody() override {
00485 if (as_error_) {
00486 AssertHelper(TestPartResult::kNonFatalFailure, loc_.file.c_str(),
00487 loc_.line, "") = Message() « error_message_;
00488 } else {
00489 std::cout « error_message_ « std::endl;
00490 }
00491 }
00492
00493 private:
00494 const CodeLocation loc_;
00495 const std::string error_message_;
00496 const bool as_error_;
00497 };
00498
00499 } // namespace
00500
00501 std::set<std::string>* GetIgnoredParameterizedTestSuites() {
00502 return UnitTest::GetInstance()->impl()->ignored_parameterized_test_suites();
00503 }
00504
00505 // Add a given test_suit to the list of them allow to go un-instantiated.
00506
00507 MarkAsIgnored::MarkAsIgnored(const char* test_suite) {
00508 GetIgnoredParameterizedTestSuites()->insert(test_suite);
00509 }
00510
00511 // If this parameterized test suite has no instantiations (and that
00512 // has not been marked as okay), emit a test case reporting that.

```

```

00513
00514 void InsertSyntheticTestCase(const std::string& name, CodeLocation location,
00515 bool has_test_p) {
00516 const auto& ignored = *GetIgnoredParameterizedTestSuites();
00517 if (ignored.find(name) != ignored.end()) return;
00518
00519 const char kMissingInstantiation[] = //
00520 " is defined via TEST_P, but never instantiated. None of the test cases "
00521 "will run. Either no INSTANTIATE_TEST_SUITE_P is provided or the only "
00522 "ones provided expand to nothing."
00523 "\n\n"
00524 "Ideally, TEST_P definitions should only ever be included as part of "
00525 "binaries that intend to use them. (As opposed to, for example, being "
00526 "placed in a library that may be linked in to get other utilities.);";
00527
00528 const char kMissingTestCase[] = //
00529 " is instantiated via INSTANTIATE_TEST_SUITE_P, but no tests are "
00530 "defined via TEST_P . No test cases will run."
00531 "\n\n"
00532 "Ideally, INSTANTIATE_TEST_SUITE_P should only ever be invoked from "
00533 "code that always depend on code that provides TEST_P. Failing to do "
00534 "so is often an indication of dead code, e.g. the last TEST_P was "
00535 "removed but the rest got left behind.";
00536
00537 std::string message =
00538 "Parameterized test suite " + name +
00539 (has_test_p ? kMissingInstantiation : kMissingTestCase) +
00540 "\n\n"
00541 "To suppress this error for this test suite, insert the following line "
00542 "(in a non-header) in the namespace it is defined in:"
00543 "\n\n"
00544 "GTEST_ALLOW_UNINSTANTIATED_PARAMETERIZED_TEST(" +
00545 name + ");";
00546
00547 std::string full_name = "UninstantiatedParameterizedTestSuite<" + name + ">";
00548 RegisterTest(//
00549 "GoogleTestVerification", full_name.c_str(),
00550 nullptr, // No type parameter.
00551 nullptr, // No value parameter.
00552 location.file.c_str(), location.line, [message, location] {
00553 return new FailureTest(location, message,
00554 kErrorOnUninstantiatedParameterizedTest);
00555 });
00556 }
00557
00558 void RegisterTypeParameterizedTestSuite(const char* test_suite_name,
00559 CodeLocation code_location) {
00560 GetUnitTestImpl()->type_parameterized_test_registry().RegisterTestSuite(
00561 test_suite_name, code_location);
00562 }
00563
00564 void RegisterTypeParameterizedTestSuiteInstantiation(const char* case_name) {
00565 GetUnitTestImpl()->type_parameterized_test_registry().RegisterInstantiation(
00566 case_name);
00567 }
00568
00569 void TypeParameterizedTestSuiteRegistry::RegisterTestSuite(
00570 const char* test_suite_name, CodeLocation code_location) {
00571 suites_.emplace(std::string(test_suite_name),
00572 TypeParameterizedTestSuiteInfo(code_location));
00573 }
00574
00575 void TypeParameterizedTestSuiteRegistry::RegisterInstantiation(
00576 const char* test_suite_name) {
00577 auto it = suites_.find(std::string(test_suite_name));
00578 if (it != suites_.end()) {
00579 it->second.instantiated = true;
00580 } else {
00581 GTEST_LOG_(ERROR) << "Unknown type parameterized test suit '"
00582 << test_suite_name << "'";
00583 }
00584 }
00585
00586 void TypeParameterizedTestSuiteRegistry::CheckForInstantiations() {
00587 const auto& ignored = *GetIgnoredParameterizedTestSuites();
00588 for (const auto& testcase : suites_) {
00589 if (testcase.second.instantiated) continue;
00590 if (ignored.find(testcase.first) != ignored.end()) continue;
00591
00592 std::string message =
00593 "Type parameterized test suite " + testcase.first +
00594 " is defined via REGISTER_TYPED_TEST_SUITE_P, but never instantiated "
00595 "via INSTANTIATE_TYPED_TEST_SUITE_P. None of the test cases will run."
00596 "\n\n"
00597 "Ideally, TYPED_TEST_P definitions should only ever be included as "
00598 "part of binaries that intend to use them. (As opposed to, for "
00599 "example, being placed in a library that may be linked in to get other "

```

```

00600 "utilities.)"
00601 "\n\n"
00602 "To suppress this error for this test suite, insert the following line "
00603 "(in a non-header) in the namespace it is defined in:"
00604 "\n\n"
00605 "GTEST_ALLOW_UNINSTANTIATED_PARAMETERIZED_TEST(" +
00606 testcase.first + ");";
00607
00608 std::string full_name =
00609 "UninstantiatedTypeParameterizedTestSuite<" + testcase.first + ">";
00610 RegisterTest(//
00611 "GoogleTestVerification", full_name.c_str(),
00612 nullptr, // No type parameter.
00613 nullptr, // No value parameter.
00614 testcase.second.code_location.file.c_str(),
00615 testcase.second.code_location.line, [message, testcase] {
00616 return new FailureTest(testcase.second.code_location, message,
00617 kErrorOnUninstantiatedTypeParameterizedTest);
00618 });
00619 }
00620 }
00621
00622 // A copy of all command line arguments. Set by InitGoogleTest().
00623 static ::std::vector<std::string> g_argvs;
00624
00625 ::std::vector<std::string> GetArgvs() {
00626 #if defined(GTEST_CUSTOM_GET_ARGVS_)
00627 // GTEST_CUSTOM_GET_ARGVS_() may return a container of std::string or
00628 // ::string. This code converts it to the appropriate type.
00629 const auto& custom = GTEST_CUSTOM_GET_ARGVS_();
00630 return ::std::vector<std::string>(custom.begin(), custom.end());
00631 #else // defined(GTEST_CUSTOM_GET_ARGVS_)
00632 return g_argvs;
00633 #endif // defined(GTEST_CUSTOM_GET_ARGVS_)
00634 }
00635
00636 // Returns the current application's name, removing directory path if that
00637 // is present.
00638
00639 FilePath GetCurrentExecutableName() {
00640 FilePath result;
00641
00642 #if GTEST_OS_WINDOWS || GTEST_OS_OS2
00643 result.Set(FilePath(GetArgvs()[0]).RemoveExtension("exe"));
00644 #else
00645 result.Set(FilePath(GetArgvs()[0]));
00646 #endif // GTEST_OS_WINDOWS
00647
00648 return result.RemoveDirectoryName();
00649 }
00650
00651 // Functions for processing the gtest_output flag.
00652
00653 // Returns the output format, or "" for normal printed output.
00654
00655 std::string UnitTestOptions::GetOutputFormat() {
00656 std::string s = GTEST_FLAG_GET(output);
00657 const char* const gtest_output_flag = s.c_str();
00658 const char* const colon = strchr(gtest_output_flag, ':');
00659 return (colon == nullptr)
00660 ? std::string(gtest_output_flag)
00661 : std::string(gtest_output_flag,
00662 static_cast<size_t>(colon - gtest_output_flag));
00663 }
00664
00665 // Returns the name of the requested output file, or the default if none
00666 // was explicitly specified.
00667
00668 std::string UnitTestOptions::GetAbsolutePathToOutputFile() {
00669 std::string s = GTEST_FLAG_GET(output);
00670 const char* const gtest_output_flag = s.c_str();
00671
00672 std::string format = GetOutputFormat();
00673 if (format.empty()) format = std::string(kDefaultOutputFormat);
00674
00675 const char* const colon = strchr(gtest_output_flag, ':');
00676 if (colon == nullptr)
00677 return internal::FilePath::MakeFileName(
00678 internal::FilePath(
00679 UnitTest::GetInstance()->original_working_dir()),
00680 internal::FilePath(kDefaultOutputFile), 0, format.c_str())
00681 .string();
00682
00683 internal::FilePath output_name(colon + 1);
00684 if (!output_name.IsAbsolutePath())
00685 output_name = internal::FilePath::ConcatPaths(
00686 internal::FilePath(UnitTest::GetInstance()->original_working_dir()),

```

```

00687 internal::FilePath(colon + 1));
00688
00689 if (!output_name.IsDirectory()) return output_name.string();
00690
00691 internal::FilePath result(internal::FilePath::GenerateUniqueFileName(
00692 output_name, internal::GetCurrentExecutableName(),
00693 GetOutputFormat().c_str());
00694 return result.string();
00695 }
00696
00697 // Returns true if and only if the wildcard pattern matches the string. Each
00698 // pattern consists of regular characters, single-character wildcards (?), and
00699 // multi-character wildcards (*).
00700 //
00701 // This function implements a linear-time string globbing algorithm based on
00702 // https://research.swtch.com/glob.
00703
00704 static bool PatternMatchesString(const std::string& name_str,
00705 const char* pattern, const char* pattern_end) {
00706 const char* name = name_str.c_str();
00707 const char* const name_begin = name;
00708 const char* const name_end = name + name_str.size();
00709
00710 const char* pattern_next = pattern;
00711 const char* name_next = name;
00712
00713 while (pattern < pattern_end || name < name_end) {
00714 if (pattern < pattern_end) {
00715 switch (*pattern) {
00716 default: // Match an ordinary character.
00717 if (name < name_end && *name == *pattern) {
00718 ++pattern;
00719 ++name;
00720 continue;
00721 }
00722 break;
00723 case '?': // Match any single character.
00724 if (name < name_end) {
00725 ++pattern;
00726 ++name;
00727 continue;
00728 }
00729 break;
00730 case '*':
00731 // Match zero or more characters. Start by skipping over the wildcard
00732 // and matching zero characters from name. If that fails, restart and
00733 // match one more character than the last attempt.
00734 pattern_next = pattern;
00735 name_next = name + 1;
00736 ++pattern;
00737 continue;
00738 }
00739 }
00740 // Failed to match a character. Restart if possible.
00741 if (name_begin < name_next && name_next <= name_end) {
00742 pattern = pattern_next;
00743 name = name_next;
00744 continue;
00745 }
00746 return false;
00747 }
00748 return true;
00749 }
00750
00751 namespace {
00752
00753 bool IsGlobPattern(const std::string& pattern) {
00754 return std::any_of(pattern.begin(), pattern.end(),
00755 [](const char c) {
00756 return c == '?' || c == '*'; });
00757 }
00758
00759 class UnitTestFilter {
00760 public:
00761 UnitTestFilter() = default;
00762
00763 // Constructs a filter from a string of patterns separated by `:`.
00764
00765 explicit UnitTestFilter(const std::string& filter) {
00766 // By design "" filter matches "" string.
00767 std::vector<std::string> all_patterns;
00768 SplitString(filter, ':', &all_patterns);
00769 const auto exact_match_patterns_begin = std::partition(
00770 all_patterns.begin(), all_patterns.end(), &IsGlobPattern);
00771
00772 glob_patterns_.reserve(static_cast<size_t> (
00773 std::distance(all_patterns.begin(), exact_match_patterns_begin)));

```



```

00774 std::move(all_patterns.begin(), exact_match_patterns_begin,
00775 std::inserter(glob_patterns_, glob_patterns_.begin()));
00776 std::move(
00777 exact_match_patterns_begin, all_patterns.end(),
00778 std::inserter(exact_match_patterns_, exact_match_patterns_.begin()));
00779 }
00780
00781 // Returns true if and only if name matches at least one of the patterns in
00782 // the filter.
00783
00784 bool MatchesName(const std::string& name) const {
00785 return exact_match_patterns_.count(name) > 0 ||
00786 std::any_of(glob_patterns_.begin(), glob_patterns_.end(),
00787 [&name](const std::string & pattern) {
00788 return PatternMatchesString(
00789 name, pattern.c_str(),
00790 pattern.c_str() + pattern.size());
00791 });
00792 }
00793
00794 private:
00795 std::vector<std::string> glob_patterns_;
00796 std::unordered_set<std::string> exact_match_patterns_;
00797 };
00798
00799 class PositiveAndNegativeUnitTestFixture {
00800 public:
00801 // Constructs a positive and a negative filter from a string. The string
00802 // contains a positive filter optionally followed by a '-' character and a
00803 // negative filter. In case only a negative filter is provided the positive
00804 // filter will be assumed "*".
00805 // A filter is a list of patterns separated by ':'.
00806
00807 explicit PositiveAndNegativeUnitTestFixture(const std::string& filter) {
00808 std::vector<std::string> positive_and_negative_filters;
00809
00810 // NOTE: `SplitString` always returns a non-empty container.
00811 SplitString(filter, '-', &positive_and_negative_filters);
00812 const auto& positive_filter = positive_and_negative_filters.front();
00813
00814 if (positive_and_negative_filters.size() > 1) {
00815 positive_filter_ = UnitTestFixture(
00816 positive_filter.empty() ? kUniversalFilter : positive_filter);
00817
00818 // TODO(b/214626361): Fail on multiple '-' characters
00819 // For the moment to preserve old behavior we concatenate the rest of the
00820 // string parts with '-' as separator to generate the negative filter.
00821 auto negative_filter_string = positive_and_negative_filters[1];
00822 for (std::size_t i = 2; i < positive_and_negative_filters.size(); i++)
00823 negative_filter_string =
00824 negative_filter_string + '-' + positive_and_negative_filters[i];
00825 negative_filter_ = UnitTestFixture(negative_filter_string);
00826 } else {
00827 // In case we don't have a negative filter and positive filter is ""
00828 // we do not use kUniversalFilter by design as opposed to when we have a
00829 // negative filter.
00830 positive_filter_ = UnitTestFixture(positive_filter);
00831 }
00832 }
00833
00834 // Returns true if and only if test name (this is generated by appending test
00835 // suite name and test name via a '.' character) matches the positive filter
00836 // and does not match the negative filter.
00837
00838 bool MatchesTest(const std::string& test_suite_name,
00839 const std::string& test_name) const {
00840 return MatchesName(test_suite_name + "." + test_name);
00841 }
00842
00843 // Returns true if and only if name matches the positive filter and does not
00844 // match the negative filter.
00845
00846 bool MatchesName(const std::string& name) const {
00847 return positive_filter_.MatchesName(name) &&
00848 !negative_filter_.MatchesName(name);
00849 }
00850
00851 private:
00852 UnitTestFixture positive_filter_;
00853 UnitTestFixture negative_filter_;
00854 };
00855
00856 // namespace
00857
00858 bool UnitTestOptions::MatchesFilter(const std::string& name_str,
00859 const char* filter) {
00860 return UnitTestFixture(filter).MatchesName(name_str);
00861 }

```

```

00861
00862 // Returns true if and only if the user-specified filter matches the test
00863 // suite name and the test name.
00864
00865 bool UnitTestOptions::FilterMatchesTest(const std::string& test_suite_name,
00866 const std::string& test_name) {
00867 // Split --gtest_filter at '-', if there is one, to separate into
00868 // positive filter and negative filter portions
00869 return PositiveAndNegativeUnitTestFilter(GTEST_FLAG_GET(filter))
00870 .MatchesTest(test_suite_name, test_name);
00871 }
00872
00873 #if GTEST_HAS_SEH
00874 // Returns EXCEPTION_EXECUTE_HANDLER if Google Test should handle the
00875 // given SEH exception, or EXCEPTION_CONTINUE_SEARCH otherwise.
00876 // This function is useful as an __except condition.
00877
00878 int UnitTestOptions::GTestShouldProcessSEH(DWORD exception_code) {
00879 // Google Test should handle a SEH exception if:
00880 // 1. the user wants it to, AND
00881 // 2. this is not a breakpoint exception, AND
00882 // 3. this is not a C++ exception (VC++ implements them via SEH,
00883 // apparently).
00884 //
00885 // SEH exception code for C++ exceptions.
00886 // (see http://support.microsoft.com/kb/185294 for more information).
00887 const DWORD kCxxExceptionCode = 0xe06d7363;
00888
00889 bool should_handle = true;
00890
00891 if (!GTEST_FLAG_GET(catch_exceptions))
00892 should_handle = false;
00893 else if (exception_code == EXCEPTION_BREAKPOINT)
00894 should_handle = false;
00895 else if (exception_code == kCxxExceptionCode)
00896 should_handle = false;
00897
00898 return should_handle ? EXCEPTION_EXECUTE_HANDLER : EXCEPTION_CONTINUE_SEARCH;
00899 }
00900 #endif // GTEST_HAS_SEH
00901
00902 } // namespace internal
00903
00904 // The c'tor sets this object as the test part result reporter used by
00905 // Google Test. The 'result' parameter specifies where to report the
00906 // results. Intercepts only failures from the current thread.
00907
00908 ScopedFakeTestPartResultReporter::ScopedFakeTestPartResultReporter(
00909 TestPartResultArray* result)
00910 : intercept_mode_(INTERCEPT_ONLY_CURRENT_THREAD), result_(result) {
00911 Init();
00912 }
00913
00914 // The c'tor sets this object as the test part result reporter used by
00915 // Google Test. The 'result' parameter specifies where to report the
00916 // results.
00917
00918 ScopedFakeTestPartResultReporter::ScopedFakeTestPartResultReporter(
00919 InterceptMode intercept_mode, TestPartResultArray* result)
00920 : intercept_mode_(intercept_mode), result_(result) {
00921 Init();
00922 }
00923
00924 void ScopedFakeTestPartResultReporter::Init() {
00925 internal::UnitTestImpl * const impl = internal::GetUnitTestImpl();
00926 if (intercept_mode_ == INTERCEPT_ALL_THREADS) {
00927 old_reporter_ = impl->GetGlobalTestPartResultReporter();
00928 impl->SetGlobalTestPartResultReporter(this);
00929 } else {
00930 old_reporter_ = impl->GetTestPartResultReporterForCurrentThread();
00931 impl->SetTestPartResultReporterForCurrentThread(this);
00932 }
00933 }
00934
00935 // The d'tor restores the test part result reporter used by Google Test
00936 // before.
00937
00938 ScopedFakeTestPartResultReporter::~~ScopedFakeTestPartResultReporter() {
00939 internal::UnitTestImpl * const impl = internal::GetUnitTestImpl();
00940 if (intercept_mode_ == INTERCEPT_ALL_THREADS) {
00941 impl->SetGlobalTestPartResultReporter(old_reporter_);
00942 } else {
00943 impl->SetTestPartResultReporterForCurrentThread(old_reporter_);
00944 }
00945 }
00946
00947 // Increments the test part result count and remembers the result.

```

```

00948 // This method is from the TestPartResultReporterInterface interface.
00949
00950 void ScopedFakeTestPartResultReporter::ReportTestPartResult(
00951 const TestPartResult& result) {
00952 result_>Append(result);
00953 }
00954
00955 namespace internal {
00956
00957 // Returns the type ID of ::testing::Test. We should always call this
00958 // instead of GetTypeId< ::testing::Test>() to get the type ID of
00959 // testing::Test. This is to work around a suspected linker bug when
00960 // using Google Test as a framework on Mac OS X. The bug causes
00961 // GetTypeId< ::testing::Test>() to return different values depending
00962 // on whether the call is from the Google Test framework itself or
00963 // from user test code. GetTestTypeId() is guaranteed to always
00964 // return the same value, as it always calls GetTypeId<>() from the
00965 // gtest.cc, which is within the Google Test framework.
00966
00967 TypeId GetTestTypeId() {
00968 return GetTypeId<Test>();
00969 }
00970
00971 // The value of GetTestTypeId() as seen from within the Google Test
00972 // library. This is solely for testing GetTestTypeId().
00973 extern const TypeId kTestTypeIdInGoogleTest = GetTestTypeId();
00974
00975 // This predicate-formatter checks that 'results' contains a test part
00976 // failure of the given type and that the failure message contains the
00977 // given substring.
00978
00979 static AssertionResult HasOneFailure(const char* /* results_expr */,
00980 const char* /* type_expr */,
00981 const char* /* substr_expr */,
00982 const TestPartResultArray& results,
00983 TestPartResult::Type type,
00984 const std::string& substr) {
00985 const std::string expected(type == TestPartResult::kFatalFailure
00986 ? "1 fatal failure"
00987 : "1 non-fatal failure");
00988 Message msg;
00989 if (results.size() != 1) {
00990 msg << "Expected: " << expected << "\n"
00991 << " Actual: " << results.size() << " failures";
00992 for (int i = 0; i < results.size(); i++) {
00993 msg << "\n" << results.GetTestPartResult(i);
00994 }
00995 return AssertionFailure() << msg;
00996 }
00997
00998 const TestPartResult& r = results.GetTestPartResult(0);
00999 if (r.type() != type) {
01000 return AssertionFailure() << "Expected: " << expected << "\n"
01001 << " Actual:\n"
01002 << r;
01003 }
01004
01005 if (strstr(r.message(), substr.c_str()) == nullptr) {
01006 return AssertionFailure()
01007 << "Expected: " << expected << " containing \"" << substr << "\"\n"
01008 << " Actual:\n"
01009 << r;
01010 }
01011
01012 return AssertionSuccess();
01013 }
01014
01015 // The constructor of SingleFailureChecker remembers where to look up
01016 // test part results, what type of failure we expect, and what
01017 // substring the failure message should contain.
01018
01019 SingleFailureChecker::SingleFailureChecker(const TestPartResultArray* results,
01020 TestPartResult::Type type,
01021 const std::string& substr)
01022 : results_(results), type_(type), substr_(substr) {}
01023
01024
01025 // The destructor of SingleFailureChecker verifies that the given
01026 // TestPartResultArray contains exactly one failure that has the given
01027 // type and contains the given substring. If that's not the case, a
01028 // non-fatal failure will be generated.
01029
01030 SingleFailureChecker::~SingleFailureChecker() {
01031 EXPECT_PRED_FORMAT3(HasOneFailure, *results_, type_, substr_);
01032 }
01033
01034 DefaultGlobalTestPartResultReporter::DefaultGlobalTestPartResultReporter(

```

```

01035 UnitTestImpl* unit_test)
01036 : unit_test_(unit_test) {
01037 }
01038
01039 void DefaultGlobalTestPartResultReporter::ReportTestPartResult (
01040 const TestPartResult& result) {
01041 unit_test_>current_test_result()->AddTestPartResult(result);
01042 unit_test_>listeners()->repeater()->OnTestPartResult(result);
01043 }
01044
01045 DefaultPerThreadTestPartResultReporter::DefaultPerThreadTestPartResultReporter (
01046 UnitTestImpl* unit_test)
01047 : unit_test_(unit_test) {
01048 }
01049
01050 void DefaultPerThreadTestPartResultReporter::ReportTestPartResult (
01051 const TestPartResult& result) {
01052 unit_test_>GetGlobalTestPartResultReporter()->ReportTestPartResult(result);
01053 }
01054
01055 // Returns the global test part result reporter.
01056
01057 TestPartResultReporterInterface*
01058 UnitTestImpl::GetGlobalTestPartResultReporter() {
01059 internal::MutexLock lock(&global_test_part_result_reporter_mutex_);
01060 return global_test_part_result_reporter_;
01061 }
01062
01063 // Sets the global test part result reporter.
01064
01065 void UnitTestImpl::SetGlobalTestPartResultReporter (
01066 TestPartResultReporterInterface* reporter) {
01067 internal::MutexLock lock(&global_test_part_result_reporter_mutex_);
01068 global_test_part_result_reporter_ = reporter;
01069 }
01070
01071 // Returns the test part result reporter for the current thread.
01072
01073 TestPartResultReporterInterface*
01074 UnitTestImpl::GetTestPartResultReporterForCurrentThread() {
01075 return per_thread_test_part_result_reporter_.get();
01076 }
01077
01078 // Sets the test part result reporter for the current thread.
01079
01080 void UnitTestImpl::SetTestPartResultReporterForCurrentThread (
01081 TestPartResultReporterInterface* reporter) {
01082 per_thread_test_part_result_reporter_.set(reporter);
01083 }
01084
01085 // Gets the number of successful test suites.
01086
01087 int UnitTestImpl::successful_test_suite_count() const {
01088 return CountIf(test_suites_, TestSuitePassed);
01089 }
01090
01091 // Gets the number of failed test suites.
01092
01093 int UnitTestImpl::failed_test_suite_count() const {
01094 return CountIf(test_suites_, TestSuiteFailed);
01095 }
01096
01097 // Gets the number of all test suites.
01098
01099 int UnitTestImpl::total_test_suite_count() const {
01100 return static_cast<int> (test_suites_.size());
01101 }
01102
01103 // Gets the number of all test suites that contain at least one test
01104 // that should run.
01105
01106 int UnitTestImpl::test_suite_to_run_count() const {
01107 return CountIf(test_suites_, ShouldRunTestSuite);
01108 }
01109
01110 // Gets the number of successful tests.
01111
01112 int UnitTestImpl::successful_test_count() const {
01113 return SumOverTestSuiteList(test_suites_, &TestSuite::successful_test_count);
01114 }
01115
01116 // Gets the number of skipped tests.
01117
01118 int UnitTestImpl::skipped_test_count() const {
01119 return SumOverTestSuiteList(test_suites_, &TestSuite::skipped_test_count);
01120 }
01121

```

```

01122 // Gets the number of failed tests.
01123
01124 int UnitTestImpl::failed_test_count() const {
01125 return SumOverTestSuiteList(test_suites_, &TestSuite::failed_test_count);
01126 }
01127
01128 // Gets the number of disabled tests that will be reported in the XML report.
01129
01130 int UnitTestImpl::reportable_disabled_test_count() const {
01131 return SumOverTestSuiteList(test_suites_,
01132 &TestSuite::reportable_disabled_test_count);
01133 }
01134
01135 // Gets the number of disabled tests.
01136
01137 int UnitTestImpl::disabled_test_count() const {
01138 return SumOverTestSuiteList(test_suites_, &TestSuite::disabled_test_count);
01139 }
01140
01141 // Gets the number of tests to be printed in the XML report.
01142
01143 int UnitTestImpl::reportable_test_count() const {
01144 return SumOverTestSuiteList(test_suites_, &TestSuite::reportable_test_count);
01145 }
01146
01147 // Gets the number of all tests.
01148
01149 int UnitTestImpl::total_test_count() const {
01150 return SumOverTestSuiteList(test_suites_, &TestSuite::total_test_count);
01151 }
01152
01153 // Gets the number of tests that should run.
01154
01155 int UnitTestImpl::test_to_run_count() const {
01156 return SumOverTestSuiteList(test_suites_, &TestSuite::test_to_run_count);
01157 }
01158
01159 // Returns the current OS stack trace as an std::string.
01160 //
01161 // The maximum number of stack frames to be included is specified by
01162 // the gtest_stack_trace_depth flag. The skip_count parameter
01163 // specifies the number of top frames to be skipped, which doesn't
01164 // count against the number of frames to be included.
01165 //
01166 // For example, if Foo() calls Bar(), which in turn calls
01167 // CurrentStackTraceExceptTop(1), Foo() will be included in the
01168 // trace but Bar() and CurrentStackTraceExceptTop() won't.
01169
01170 std::string UnitTestImpl::CurrentStackTraceExceptTop(int skip_count) {
01171 return os_stack_trace_getter()->CurrentStackTrace(
01172 static_cast<int>(GTEST_FLAG_GET(stack_trace_depth)), skip_count + 1
01173 // Skips the user-specified number of frames plus this function
01174 // itself.
01175); // NOLINT
01176 }
01177
01178 // A helper class for measuring elapsed times.
01179
01180 class Timer {
01181 public:
01182
01183 Timer() : start_(std::chrono::steady_clock::now()) {
01184 }
01185
01186 // Return time elapsed in milliseconds since the timer was created.
01187
01188 TimeInMillis Elapsed() {
01189 return std::chrono::duration_cast<std::chrono::milliseconds>(
01190 std::chrono::steady_clock::now() - start_)
01191 .count();
01192 }
01193
01194 private:
01195 std::chrono::steady_clock::time_point start_;
01196 };
01197
01198 // Returns a timestamp as milliseconds since the epoch. Note this time may jump
01199 // around subject to adjustments by the system, to measure elapsed time use
01200 // Timer instead.
01201
01202 TimeInMillis GetTimeInMillis() {
01203 return std::chrono::duration_cast<std::chrono::milliseconds>(
01204 std::chrono::system_clock::now() -
01205 std::chrono::system_clock::from_time_t(0))
01206 .count();
01207 }
01208

```

```

01209 // Utilities
01210
01211 // class String.
01212
01213 #if GTEST_OS_WINDOWS_MOBILE
01214 // Creates a UTF-16 wide string from the given ANSI string, allocating
01215 // memory using new. The caller is responsible for deleting the return
01216 // value using delete[]. Returns the wide string, or NULL if the
01217 // input is NULL.
01218
01219 LPCWSTR String::AnsiToUtf16(const char* ansi) {
01220 if (!ansi) return nullptr;
01221 const int length = strlen(ansi);
01222 const int unicode_length =
01223 MultiByteToWideChar(CP_ACP, 0, ansi, length, nullptr, 0);
01224 WCHAR* unicode = new WCHAR[unicode_length + 1];
01225 MultiByteToWideChar(CP_ACP, 0, ansi, length, unicode, unicode_length);
01226 unicode[unicode_length] = 0;
01227 return unicode;
01228 }
01229
01230 // Creates an ANSI string from the given wide string, allocating
01231 // memory using new. The caller is responsible for deleting the return
01232 // value using delete[]. Returns the ANSI string, or NULL if the
01233 // input is NULL.
01234
01235 const char* String::Utf16ToAnsi(LPCWSTR utf16_str) {
01236 if (!utf16_str) return nullptr;
01237 const int ansi_length = WideCharToMultiByte(CP_ACP, 0, utf16_str, -1, nullptr,
01238 0, nullptr, nullptr);
01239 char* ansi = new char[ansi_length + 1];
01240 WideCharToMultiByte(CP_ACP, 0, utf16_str, -1, ansi, ansi_length, nullptr,
01241 nullptr);
01242 ansi[ansi_length] = 0;
01243 return ansi;
01244 }
01245
01246 #endif // GTEST_OS_WINDOWS_MOBILE
01247
01248 // Compares two C strings. Returns true if and only if they have the same
01249 // content.
01250 //
01251 // Unlike strcmp(), this function can handle NULL argument(s). A NULL
01252 // C string is considered different to any non-NULL C string,
01253 // including the empty string.
01254
01255 bool String::CStringEquals(const char* lhs, const char* rhs) {
01256 if (lhs == nullptr) return rhs == nullptr;
01257
01258 if (rhs == nullptr) return false;
01259
01260 return strcmp(lhs, rhs) == 0;
01261 }
01262
01263 #if GTEST_HAS_STD_WSTRING
01264
01265 // Converts an array of wide chars to a narrow string using the UTF-8
01266 // encoding, and streams the result to the given Message object.
01267
01268 static void StreamWideCharsToMessage(const wchar_t* wstr, size_t length,
01269 Message* msg) {
01270 for (size_t i = 0; i != length; i++) { // NOLINT
01271 if (wstr[i] != L'\0') {
01272 *msg << WideStringToUtf8(wstr + i, static_cast<int>(length - i));
01273 while (i != length && wstr[i] != L'\0') i++;
01274 } else {
01275 *msg << '\0';
01276 i++;
01277 }
01278 }
01279 }
01280
01281 #endif // GTEST_HAS_STD_WSTRING
01282
01283 void SplitString(const ::std::string& str, char delimiter,
01284 ::std::vector< ::std::string>* dest) {
01285 ::std::vector< ::std::string> parsed;
01286 ::std::string::size_type pos = 0;
01287 while (::testing::internal::AlwaysTrue()) {
01288 const ::std::string::size_type colon = str.find(delimiter, pos);
01289 if (colon == ::std::string::npos) {
01290 parsed.push_back(str.substr(pos));
01291 break;
01292 } else {
01293 parsed.push_back(str.substr(pos, colon - pos));
01294 pos = colon + 1;
01295 }
01296 }

```

```

01296 }
01297 dest->swap(parsed);
01298 }
01299 } // namespace internal
01300
01301 // Constructs an empty Message.
01302 // We allocate the stringstream separately because otherwise each use of
01303 // ASSERT/EXPECT in a procedure adds over 200 bytes to the procedure's
01304 // stack frame leading to huge stack frames in some cases; gcc does not reuse
01305 // the stack space.
01306
01307 Message::Message() : ss_(new ::std::stringstream) {
01308 // By default, we want there to be enough precision when printing
01309 // a double to a Message.
01310 *ss_ < std::setprecision(std::numeric_limits<double>::digits10 + 2);
01311 }
01312
01313 // These two overloads allow streaming a wide C string to a Message
01314 // using the UTF-8 encoding.
01315
01316 Message& Message::operator<<(const wchar_t* wide_c_str) {
01317 return *this < internal::String::ShowWideCString(wide_c_str);
01318 }
01319
01320 Message& Message::operator<<(wchar_t* wide_c_str) {
01321 return *this < internal::String::ShowWideCString(wide_c_str);
01322 }
01323
01324 #if GTEST_HAS_STD_WSTRING
01325 // Converts the given wide string to a narrow string using the UTF-8
01326 // encoding, and streams the result to this Message object.
01327
01328 Message& Message::operator<<(const ::std::wstring& wstr) {
01329 internal::StreamWideCharsToMessage(wstr.c_str(), wstr.length(), this);
01330 return *this;
01331 }
01332 #endif // GTEST_HAS_STD_WSTRING
01333
01334 // Gets the text streamed to this object so far as an std::string.
01335 // Each '\0' character in the buffer is replaced with "\\0".
01336
01337 std::string Message::GetString() const {
01338 return internal::StringStreamToString(ss_.get());
01339 }
01340
01341 namespace internal {
01342 namespace edit_distance {
01343
01344 std::vector<EditType> CalculateOptimalEdits(const std::vector<size_t>& left,
01345 const std::vector<size_t>& right) {
01346 std::vector<std::vector<double>> > costs(
01347 left.size() + 1, std::vector<double>(right.size() + 1));
01348 std::vector<std::vector<EditType>> > best_move(
01349 left.size() + 1, std::vector<EditType>(right.size() + 1));
01350
01351 // Populate for empty right.
01352 for (size_t l_i = 0; l_i < costs.size(); ++l_i) {
01353 costs[l_i][0] = static_cast<double>(l_i);
01354 best_move[l_i][0] = kRemove;
01355 }
01356 // Populate for empty left.
01357 for (size_t r_i = 1; r_i < costs[0].size(); ++r_i) {
01358 costs[0][r_i] = static_cast<double>(r_i);
01359 best_move[0][r_i] = kAdd;
01360 }
01361
01362 for (size_t l_i = 0; l_i < left.size(); ++l_i) {
01363 for (size_t r_i = 0; r_i < right.size(); ++r_i) {
01364 if (left[l_i] == right[r_i]) {
01365 // Found a match. Consume it.
01366 costs[l_i + 1][r_i + 1] = costs[l_i][r_i];
01367 best_move[l_i + 1][r_i + 1] = kMatch;
01368 continue;
01369 }
01370
01371 const double add = costs[l_i + 1][r_i];
01372 const double remove = costs[l_i][r_i + 1];
01373 const double replace = costs[l_i][r_i];
01374 if (add < remove && add < replace) {
01375 costs[l_i + 1][r_i + 1] = add + 1;
01376 best_move[l_i + 1][r_i + 1] = kAdd;
01377 } else if (remove < add && remove < replace) {
01378 costs[l_i + 1][r_i + 1] = remove + 1;
01379 best_move[l_i + 1][r_i + 1] = kRemove;
01380 } else {
01381 costs[l_i + 1][r_i + 1] = replace + 1;
01382 best_move[l_i + 1][r_i + 1] = kReplace;
01383 }
01384 }
01385 }
01386 }
01387 }
01388 }

```

```

01383 // We make replace a little more expensive than add/remove to lower
01384 // their priority.
01385 costs[l_i + 1][r_i + 1] = replace + 1.00001;
01386 best_move[l_i + 1][r_i + 1] = kReplace;
01387 }
01388 }
01389 }
01390
01391 // Reconstruct the best path. We do it in reverse order.
01392 std::vector<EditType> best_path;
01393 for (size_t l_i = left.size(), r_i = right.size(); l_i > 0 || r_i > 0;) {
01394 EditType move = best_move[l_i][r_i];
01395 best_path.push_back(move);
01396 l_i -= move != kAdd;
01397 r_i -= move != kRemove;
01398 }
01399 std::reverse(best_path.begin(), best_path.end());
01400 return best_path;
01401 }
01402
01403 namespace {
01404
01405 // Helper class to convert string into ids with deduplication.
01406
01407 class InternalStrings {
01408 public:
01409
01410 size_t GetId(const std::string& str) {
01411 IdMap::iterator it = ids_.find(str);
01412 if (it != ids_.end()) return it->second;
01413 size_t id = ids_.size();
01414 return ids_[str] = id;
01415 }
01416
01417 private:
01418 typedef std::map<std::string, size_t> IdMap;
01419 IdMap ids_;
01420 };
01421 } // namespace
01422
01423 std::vector<EditType> CalculateOptimalEdits(
01424 const std::vector<std::string>& left,
01425 const std::vector<std::string>& right) {
01426 std::vector<size_t> left_ids, right_ids;
01427 {
01428 InternalStrings intern_table;
01429 for (size_t i = 0; i < left.size(); ++i) {
01430 left_ids.push_back(intern_table.GetId(left[i]));
01431 }
01432 for (size_t i = 0; i < right.size(); ++i) {
01433 right_ids.push_back(intern_table.GetId(right[i]));
01434 }
01435 }
01436 return CalculateOptimalEdits(left_ids, right_ids);
01437 }
01438
01439 namespace {
01440
01441 // Helper class that holds the state for one hunk and prints it out to the
01442 // stream.
01443 // It reorders adds/removes when possible to group all removes before all
01444 // adds. It also adds the hunk header before printint into the stream.
01445
01446 class Hunk {
01447 public:
01448
01449 Hunk(size_t left_start, size_t right_start)
01450 : left_start_(left_start),
01451 right_start_(right_start),
01452 adds_(),
01453 removes_(),
01454 common_() {
01455 }
01456
01457 void PushLine(char edit, const char* line) {
01458 switch (edit) {
01459 case ' ':
01460 ++common_;
01461 FlushEdits();
01462 hunk_.push_back(std::make_pair(' ', line));
01463 break;
01464 case '-':
01465 ++removes_;
01466 hunk_.removes_.push_back(std::make_pair('-', line));
01467 break;
01468 case '+':

```



```

01470 ++adds_;
01471 hunk_adds_.push_back(std::make_pair('+', line));
01472 break;
01473 }
01474 }
01475
01476 void PrintTo(std::ostream* os) {
01477 PrintHeader(os);
01478 FlushEdits();
01479 for (std::list<std::pair<char, const char*> >::const_iterator it =
01480 hunk_.begin();
01481 it != hunk_.end(); ++it) {
01482 *os << it->first << it->second << "\n";
01483 }
01484 }
01485
01486 bool has_edits() const {
01487 return adds_ || removes_;
01488 }
01489
01490 private:
01491
01492 void FlushEdits() {
01493 hunk_.splice(hunk_.end(), hunk_removes_);
01494 hunk_.splice(hunk_.end(), hunk_adds_);
01495 }
01496
01497 // Print a unified diff header for one hunk.
01498 // The format is
01499 // "@@ -<left_start>,<left_length> +<right_start>,<right_length> @@"
01500 // where the left/right parts are omitted if unnecessary.
01501
01502 void PrintHeader(std::ostream* ss) const {
01503 *ss << "@@ ";
01504 if (removes_) {
01505 *ss << "-" << left_start_ << "," << (removes_ + common_);
01506 }
01507 if (removes_ && adds_) {
01508 *ss << " ";
01509 }
01510 if (adds_) {
01511 *ss << "+" << right_start_ << "," << (adds_ + common_);
01512 }
01513 *ss << " @@\n";
01514 }
01515
01516 size_t left_start_, right_start_;
01517 size_t adds_, removes_, common_;
01518 std::list<std::pair<char, const char*> > hunk_, hunk_adds_, hunk_removes_;
01519 };
01520
01521 } // namespace
01522
01523 // Create a list of diff hunks in Unified diff format.
01524 // Each hunk has a header generated by PrintHeader above plus a body with
01525 // lines prefixed with ' ' for no change, '-' for deletion and '+' for
01526 // addition.
01527 // 'context' represents the desired unchanged prefix/suffix around the diff.
01528 // If two hunks are close enough that their contexts overlap, then they are
01529 // joined into one hunk.
01530
01531 std::string CreateUnifiedDiff(const std::vector<std::string>& left,
01532 const std::vector<std::string>& right,
01533 size_t context) {
01534 const std::vector<EditType> edits = CalculateOptimalEdits(left, right);
01535
01536 size_t l_i = 0, r_i = 0, edit_i = 0;
01537 std::stringstream ss;
01538 while (edit_i < edits.size()) {
01539 // Find first edit.
01540 while (edit_i < edits.size() && edits[edit_i] == kMatch) {
01541 ++l_i;
01542 ++r_i;
01543 ++edit_i;
01544 }
01545
01546 // Find the first line to include in the hunk.
01547 const size_t prefix_context = std::min(l_i, context);
01548 Hunk hunk(l_i - prefix_context + 1, r_i - prefix_context + 1);
01549 for (size_t i = prefix_context; i > 0; --i) {
01550 hunk.PushLine(' ', left[l_i - i].c_str());
01551 }
01552
01553 // Iterate the edits until we found enough suffix for the hunk or the input
01554 // is over.
01555 size_t n_suffix = 0;
01556 for (; edit_i < edits.size(); ++edit_i) {

```

```

01557 if (n_suffix >= context) {
01558 // Continue only if the next hunk is very close.
01559 auto it = edits.begin() + static_cast<int> (edit_i);
01560 while (it != edits.end() && *it == kMatch) ++it;
01561 if (it == edits.end() ||
01562 static_cast<size_t> (it - edits.begin()) - edit_i >= context) {
01563 // There is no next edit or it is too far away.
01564 break;
01565 }
01566 }
01567
01568 EditType edit = edits[edit_i];
01569 // Reset count when a non match is found.
01570 n_suffix = edit == kMatch ? n_suffix + 1 : 0;
01571
01572 if (edit == kMatch || edit == kRemove || edit == kReplace) {
01573 hunk.PushLine(edit == kMatch ? ' ' : '-', left[l_i].c_str());
01574 }
01575 if (edit == kAdd || edit == kReplace) {
01576 hunk.PushLine('+', right[r_i].c_str());
01577 }
01578
01579 // Advance indices, depending on edit type.
01580 l_i += edit != kAdd;
01581 r_i += edit != kRemove;
01582 }
01583
01584 if (!hunk.has_edits()) {
01585 // We are done. We don't want this hunk.
01586 break;
01587 }
01588
01589 hunk.PrintTo(&ss);
01590 }
01591 return ss.str();
01592 }
01593
01594 } // namespace edit_distance
01595
01596 namespace {
01597
01598 // The string representation of the values received in EqFailure() are already
01599 // escaped. Split them on escaped '\n' boundaries. Leave all other escaped
01600 // characters the same.
01601
01602 std::vector<std::string> SplitEscapedString(const std::string& str) {
01603 std::vector<std::string> lines;
01604 size_t start = 0, end = str.size();
01605 if (end > 2 && str[0] == '"' && str[end - 1] == '"') {
01606 ++start;
01607 --end;
01608 }
01609 bool escaped = false;
01610 for (size_t i = start; i + 1 < end; ++i) {
01611 if (escaped) {
01612 escaped = false;
01613 if (str[i] == '\n') {
01614 lines.push_back(str.substr(start, i - start - 1));
01615 start = i + 1;
01616 }
01617 } else {
01618 escaped = str[i] == '\\';
01619 }
01620 }
01621 lines.push_back(str.substr(start, end - start));
01622 return lines;
01623 }
01624
01625 } // namespace
01626
01627 // Constructs and returns the message for an equality assertion
01628 // (e.g. ASSERT_EQ, EXPECT_STREQ, etc) failure.
01629 //
01630 // The first four parameters are the expressions used in the assertion
01631 // and their values, as strings. For example, for ASSERT_EQ(foo, bar)
01632 // where foo is 5 and bar is 6, we have:
01633 //
01634 // lhs_expression: "foo"
01635 // rhs_expression: "bar"
01636 // lhs_value: "5"
01637 // rhs_value: "6"
01638 //
01639 // The ignoring_case parameter is true if and only if the assertion is a
01640 // *_STRCASEEQ*. When it's true, the string "Ignoring case" will
01641 // be inserted into the message.
01642
01643 AssertionResult EqFailure(const char* lhs_expression,

```

```

01644 const char* rhs_expression,
01645 const std::string& lhs_value,
01646 const std::string& rhs_value, bool ignoring_case) {
01647 Message msg;
01648 msg << "Expected equality of these values:";
01649 msg << "\n " << lhs_expression;
01650 if (lhs_value != lhs_expression) {
01651 msg << "\n Which is: " << lhs_value;
01652 }
01653 msg << "\n " << rhs_expression;
01654 if (rhs_value != rhs_expression) {
01655 msg << "\n Which is: " << rhs_value;
01656 }
01657
01658 if (ignoring_case) {
01659 msg << "\nIgnoring case";
01660 }
01661
01662 if (!lhs_value.empty() && !rhs_value.empty()) {
01663 const std::vector<std::string> lhs_lines = SplitEscapedString(lhs_value);
01664 const std::vector<std::string> rhs_lines = SplitEscapedString(rhs_value);
01665 if (lhs_lines.size() > 1 || rhs_lines.size() > 1) {
01666 msg << "\nWith diff:\n"
01667 << edit_distance::CreateUnifiedDiff(lhs_lines, rhs_lines);
01668 }
01669 }
01670
01671 return AssertionFailure() << msg;
01672 }
01673
01674 // Constructs a failure message for Boolean assertions such as EXPECT_TRUE.
01675
01676 std::string GetBoolAssertionFailureMessage(
01677 const AssertionResult& assertion_result, const char* expression_text,
01678 const char* actual_predicate_value, const char* expected_predicate_value) {
01679 const char* actual_message = assertion_result.message();
01680 Message msg;
01681 msg << "Value of: " << expression_text
01682 << "\n Actual: " << actual_predicate_value;
01683 if (actual_message[0] != '\0') msg << " (" << actual_message << ")";
01684 msg << "\nExpected: " << expected_predicate_value;
01685 return msg.GetString();
01686 }
01687
01688 // Helper function for implementing ASSERT_NEAR.
01689
01690 AssertionResult DoubleNearPredFormat(const char* expr1, const char* expr2,
01691 const char* abs_error_expr, double val1,
01692 double val2, double abs_error) {
01693 const double diff = fabs(val1 - val2);
01694 if (diff <= abs_error) return AssertionSuccess();
01695
01696 // Find the value which is closest to zero.
01697 const double min_abs = std::min(fabs(val1), fabs(val2));
01698 // Find the distance to the next double from that value.
01699 const double epsilon =
01700 nextafter(min_abs, std::numeric_limits<double>::infinity()) - min_abs;
01701 // Detect the case where abs_error is so small that EXPECT_NEAR is
01702 // effectively the same as EXPECT_EQUAL, and give an informative error
01703 // message so that the situation can be more easily understood without
01704 // requiring exotic floating-point knowledge.
01705 // Don't do an epsilon check if abs_error is zero because that implies
01706 // that an equality check was actually intended.
01707 if (!(std::isnan)(val1) && !(std::isnan)(val2) && abs_error > 0 &&
01708 abs_error < epsilon) {
01709 return AssertionFailure()
01710 << "The difference between " << expr1 << " and " << expr2 << " is "
01711 << diff << ", where\n"
01712 << expr1 << " evaluates to " << val1 << ",\n"
01713 << expr2 << " evaluates to " << val2 << ".\nThe abs_error parameter "
01714 << abs_error_expr << " evaluates to " << abs_error
01715 << " which is smaller than the minimum distance between doubles for "
01716 << "numbers of this magnitude which is "
01717 << epsilon
01718 << ", thus making this EXPECT_NEAR check equivalent to "
01719 << "EXPECT_EQUAL. Consider using EXPECT_DOUBLE_EQ instead.";
01720 }
01721 return AssertionFailure()
01722 << "The difference between " << expr1 << " and " << expr2 << " is "
01723 << diff << ", which exceeds " << abs_error_expr << ", where\n"
01724 << expr1 << " evaluates to " << val1 << ",\n"
01725 << expr2 << " evaluates to " << val2 << ", and\n"
01726 << abs_error_expr << " evaluates to " << abs_error << ".";
01727 }
01728
01729 // Helper template for implementing FloatLE() and DoubleLE().
01730

```

```

01731 template <typename RawType>
01732 AssertionResult FloatingPointLE(const char* expr1, const char* expr2,
01733 RawType val1, RawType val2) {
01734 // Returns success if val1 is less than val2,
01735 if (val1 < val2) {
01736 return AssertionSuccess();
01737 }
01738
01739 // or if val1 is almost equal to val2.
01740 const FloatingPoint<RawType> lhs(val1), rhs(val2);
01741 if (lhs.AlmostEquals(rhs)) {
01742 return AssertionSuccess();
01743 }
01744
01745 // Note that the above two checks will both fail if either val1 or
01746 // val2 is NaN, as the IEEE floating-point standard requires that
01747 // any predicate involving a NaN must return false.
01748
01749 ::std::stringstream val1_ss;
01750 val1_ss << std::setprecision(std::numeric_limits<RawType>::digits10 + 2)
01751 << val1;
01752
01753 ::std::stringstream val2_ss;
01754 val2_ss << std::setprecision(std::numeric_limits<RawType>::digits10 + 2)
01755 << val2;
01756
01757 return AssertionFailure()
01758 << "Expected: (" << expr1 << ") <= (" << expr2 << ")\\n"
01759 << " Actual: " << StringStreamToString(&val1_ss) << " vs "
01760 << StringStreamToString(&val2_ss);
01761 }
01762 } // namespace internal
01763
01764 // Asserts that val1 is less than, or almost equal to, val2. Fails
01765 // otherwise. In particular, it fails if either val1 or val2 is NaN.
01766
01767 AssertionResult FloatLE(const char* expr1, const char* expr2, float val1,
01768 float val2) {
01769 return internal::FloatingPointLE<float>(expr1, expr2, val1, val2);
01770 }
01771
01772 // Asserts that val1 is less than, or almost equal to, val2. Fails
01773 // otherwise. In particular, it fails if either val1 or val2 is NaN.
01774
01775 AssertionResult DoubleLE(const char* expr1, const char* expr2, double val1,
01776 double val2) {
01777 return internal::FloatingPointLE<double>(expr1, expr2, val1, val2);
01778 }
01779
01780 namespace internal {
01781
01782 // The helper function for {ASSERT|EXPECT}_STREQ.
01783
01784 AssertionResult CmpHelperSTREQ(const char* lhs_expression,
01785 const char* rhs_expression, const char* lhs,
01786 const char* rhs) {
01787 if (String::CStringEquals(lhs, rhs)) {
01788 return AssertionSuccess();
01789 }
01790
01791 return EqFailure(lhs_expression, rhs_expression, PrintToString(lhs),
01792 PrintToString(rhs), false);
01793 }
01794
01795 // The helper function for {ASSERT|EXPECT}_STRCASEEQ.
01796
01797 AssertionResult CmpHelperSTRCASEEQ(const char* lhs_expression,
01798 const char* rhs_expression, const char* lhs,
01799 const char* rhs) {
01800 if (String::CaseInsensitiveCStringEquals(lhs, rhs)) {
01801 return AssertionSuccess();
01802 }
01803
01804 return EqFailure(lhs_expression, rhs_expression, PrintToString(lhs),
01805 PrintToString(rhs), true);
01806 }
01807
01808 // The helper function for {ASSERT|EXPECT}_STRNE.
01809
01810 AssertionResult CmpHelperSTRNE(const char* s1_expression,
01811 const char* s2_expression, const char* s1,
01812 const char* s2) {
01813 if (!String::CStringEquals(s1, s2)) {
01814 return AssertionSuccess();
01815 } else {
01816 return AssertionFailure()
01817
```

```

01818 « "Expected: (" « s1_expression « ") != (" « s2_expression
01819 « "), actual: \"" « s1 « "\" vs \"" « s2 « "\"";
01820 }
01821 }
01822
01823 // The helper function for {ASSERT|EXPECT}_STRCASENE.
01824
01825 AssertionResult CmpHelperSTRCASENE(const char* s1_expression,
01826 const char* s2_expression, const char* s1,
01827 const char* s2) {
01828 if (!String::CaseInsensitiveCStringEquals(s1, s2)) {
01829 return AssertionSuccess();
01830 } else {
01831 return AssertionFailure()
01832 « "Expected: (" « s1_expression « ") != (" « s2_expression
01833 « ") (ignoring case), actual: \"" « s1 « "\" vs \"" « s2 « "\"";
01834 }
01835 }
01836
01837 } // namespace internal
01838
01839 namespace {
01840
01841 // Helper functions for implementing IsSubString() and IsNotSubString().
01842
01843 // This group of overloaded functions return true if and only if needle
01844 // is a substring of haystack. NULL is considered a substring of
01845 // itself only.
01846
01847 bool IsSubStringPred(const char* needle, const char* haystack) {
01848 if (needle == nullptr || haystack == nullptr) return needle == haystack;
01849
01850 return strstr(haystack, needle) != nullptr;
01851 }
01852
01853 bool IsSubStringPred(const wchar_t* needle, const wchar_t* haystack) {
01854 if (needle == nullptr || haystack == nullptr) return needle == haystack;
01855
01856 return wcsstr(haystack, needle) != nullptr;
01857 }
01858
01859 // StringType here can be either ::std::string or ::std::wstring.
01860
01861 template <typename StringType>
01862 bool IsSubStringPred(const StringType& needle, const StringType& haystack) {
01863 return haystack.find(needle) != StringType::npos;
01864 }
01865
01866 // This function implements either IsSubString() or IsNotSubString(),
01867 // depending on the value of the expected_to_be_substring parameter.
01868 // StringType here can be const char*, const wchar_t*, ::std::string,
01869 // or ::std::wstring.
01870
01871 template <typename StringType>
01872 AssertionResult IsSubStringImpl(bool expected_to_be_substring,
01873 const char* needle_expr,
01874 const char* haystack_expr,
01875 const StringType& needle,
01876 const StringType& haystack) {
01877 if (IsSubStringPred(needle, haystack) == expected_to_be_substring)
01878 return AssertionSuccess();
01879
01880 const bool is_wide_string = sizeof(needle[0]) > 1;
01881 const char* const begin_string_quote = is_wide_string ? "L\"" : "\"";
01882 return AssertionFailure()
01883 « "Value of: " « needle_expr « "\n"
01884 « " Actual: " « begin_string_quote « needle « "\"\n"
01885 « "Expected: " « (expected_to_be_substring ? "" : "not ")
01886 « "a substring of " « haystack_expr « "\n"
01887 « "Which is: " « begin_string_quote « haystack « "\"";
01888 }
01889
01890 } // namespace
01891
01892 // IsSubString() and IsNotSubString() check whether needle is a
01893 // substring of haystack (NULL is considered a substring of itself
01894 // only), and return an appropriate error message when they fail.
01895
01896 AssertionResult IsSubString(const char* needle_expr, const char* haystack_expr,
01897 const char* needle, const char* haystack) {
01898 return IsSubStringImpl(true, needle_expr, haystack_expr, needle, haystack);
01899 }
01900
01901 AssertionResult IsSubString(const char* needle_expr, const char* haystack_expr,
01902 const wchar_t* needle, const wchar_t* haystack) {
01903 return IsSubStringImpl(true, needle_expr, haystack_expr, needle, haystack);
01904 }

```

```

01905
01906 AssertionResult IsNotSubstring(const char* needle_expr,
01907 const char* haystack_expr, const char* needle,
01908 const char* haystack) {
01909 return IsSubstringImpl(false, needle_expr, haystack_expr, needle, haystack);
01910 }
01911
01912 AssertionResult IsNotSubstring(const char* needle_expr,
01913 const char* haystack_expr, const wchar_t* needle,
01914 const wchar_t* haystack) {
01915 return IsSubstringImpl(false, needle_expr, haystack_expr, needle, haystack);
01916 }
01917
01918 AssertionResult IsSubstring(const char* needle_expr, const char* haystack_expr,
01919 const ::std::string& needle,
01920 const ::std::string& haystack) {
01921 return IsSubstringImpl(true, needle_expr, haystack_expr, needle, haystack);
01922 }
01923
01924 AssertionResult IsNotSubstring(const char* needle_expr,
01925 const char* haystack_expr,
01926 const ::std::string& needle,
01927 const ::std::string& haystack) {
01928 return IsSubstringImpl(false, needle_expr, haystack_expr, needle, haystack);
01929 }
01930
01931 #if GTEST_HAS_STD_WSTRING
01932
01933 AssertionResult IsSubstring(const char* needle_expr, const char* haystack_expr,
01934 const ::std::wstring& needle,
01935 const ::std::wstring& haystack) {
01936 return IsSubstringImpl(true, needle_expr, haystack_expr, needle, haystack);
01937 }
01938
01939 AssertionResult IsNotSubstring(const char* needle_expr,
01940 const char* haystack_expr,
01941 const ::std::wstring& needle,
01942 const ::std::wstring& haystack) {
01943 return IsSubstringImpl(false, needle_expr, haystack_expr, needle, haystack);
01944 }
01945 #endif // GTEST_HAS_STD_WSTRING
01946
01947 namespace internal {
01948
01949 #if GTEST_OS_WINDOWS
01950
01951 namespace {
01952
01953 // Helper function for HRESULT{Success/Failure} predicates
01954
01955 AssertionResult HRESULTFailureHelper(const char* expr, const char* expected,
01956 long hr) { // NOLINT
01957 #if GTEST_OS_WINDOWS_MOBILE || GTEST_OS_WINDOWS_TV_TITLE
01958
01959 // Windows CE doesn't support FormatMessage.
01960 const char error_text[] = "";
01961
01962 #else
01963
01964 // Looks up the human-readable system message for the HRESULT code
01965 // and since we're not passing any params to FormatMessage, we don't
01966 // want inserts expanded.
01967 const DWORD kFlags =
01968 FORMAT_MESSAGE_FROM_SYSTEM | FORMAT_MESSAGE_IGNORE_INSERTS;
01969 const DWORD kBufSize = 4096;
01970 // Gets the system's human readable message string for this HRESULT.
01971 char error_text[kBufSize] = {'\0'};
01972 DWORD message_length = ::FormatMessageA(kFlags,
01973 0, // no source, we're asking system
01974 static_cast<DWORD>(hr), // the error
01975 0, // no line width restrictions
01976 error_text, // output buffer
01977 kBufSize, // buf size
01978 nullptr); // no arguments for inserts
01979 // Trims trailing white space (FormatMessage leaves a trailing CR-LF)
01980 for (; message_length && IsSpace(error_text[message_length - 1]);
01981 --message_length) {
01982 error_text[message_length - 1] = '\0';
01983 }
01984
01985 #endif // GTEST_OS_WINDOWS_MOBILE
01986
01987 const std::string error_hex("0x" + String::FormatHexInt(hr));
01988 return ::testing::AssertionFailure()
01989 << "Expected: " << expr << " " << expected << ".\n"
01990 << " Actual: " << error_hex << " " << error_text << "\n";
01991
01992 }

```

```

01992
01993 } // namespace
01994
01995 AssertionResult IsHRESULTSuccess(const char* expr, long hr) { // NOLINT
01996 if (SUCCEEDED(hr)) {
01997 return AssertionSuccess();
01998 }
01999 return HRESULTFailureHelper(expr, "succeeds", hr);
02000 }
02001
02002 AssertionResult IsHRESULTFailure(const char* expr, long hr) { // NOLINT
02003 if (FAILED(hr)) {
02004 return AssertionSuccess();
02005 }
02006 return HRESULTFailureHelper(expr, "fails", hr);
02007 }
02008
02009 #endif // GTEST_OS_WINDOWS
02010
02011 // Utility functions for encoding Unicode text (wide strings) in
02012 // UTF-8.
02013
02014 // A Unicode code-point can have up to 21 bits, and is encoded in UTF-8
02015 // like this:
02016 //
02017 // Code-point length Encoding
02018 // 0 - 7 bits 0xxxxxxx
02019 // 8 - 11 bits 110xxxxx 10xxxxxx
02020 // 12 - 16 bits 1110xxxx 10xxxxxx 10xxxxxx
02021 // 17 - 21 bits 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx
02022
02023 // The maximum code-point a one-byte UTF-8 sequence can represent.
02024 constexpr uint32_t kMaxCodePoint1 = (static_cast<uint32_t> (1) << 7) - 1;
02025
02026 // The maximum code-point a two-byte UTF-8 sequence can represent.
02027 constexpr uint32_t kMaxCodePoint2 = (static_cast<uint32_t> (1) << (5 + 6)) - 1;
02028
02029 // The maximum code-point a three-byte UTF-8 sequence can represent.
02030 constexpr uint32_t kMaxCodePoint3 =
02031 (static_cast<uint32_t> (1) << (4 + 2 * 6)) - 1;
02032
02033 // The maximum code-point a four-byte UTF-8 sequence can represent.
02034 constexpr uint32_t kMaxCodePoint4 =
02035 (static_cast<uint32_t> (1) << (3 + 3 * 6)) - 1;
02036
02037 // Chops off the n lowest bits from a bit pattern. Returns the n
02038 // lowest bits. As a side effect, the original bit pattern will be
02039 // shifted to the right by n bits.
02040
02041 inline uint32_t ChopLowBits(uint32_t* bits, int n) {
02042 const uint32_t low_bits = *bits & ((static_cast<uint32_t> (1) << n) - 1);
02043 *bits >>= n;
02044 return low_bits;
02045 }
02046
02047 // Converts a Unicode code point to a narrow string in UTF-8 encoding.
02048 // code_point parameter is of type uint32_t because wchar_t may not be
02049 // wide enough to contain a code point.
02050 // If the code_point is not a valid Unicode code point
02051 // (i.e. outside of Unicode range U+0 to U+10FFFF) it will be converted
02052 // to "(Invalid Unicode 0xXXXXXXX)".
02053
02054 std::string CodePointToUtf8(uint32_t code_point) {
02055 if (code_point > kMaxCodePoint4) {
02056 return "(Invalid Unicode 0x" + String::FormatHexUInt32(code_point) + ")";
02057 }
02058
02059 char str[5]; // Big enough for the largest valid code point.
02060 if (code_point <= kMaxCodePoint1) {
02061 str[1] = '\0';
02062 str[0] = static_cast<char> (code_point); // 0xxxxxxx
02063 } else if (code_point <= kMaxCodePoint2) {
02064 str[2] = '\0';
02065 str[1] = static_cast<char> (0x80 | ChopLowBits(&code_point, 6)); // 10xxxxxx
02066 str[0] = static_cast<char> (0xC0 | code_point); // 110xxxxx
02067 } else if (code_point <= kMaxCodePoint3) {
02068 str[3] = '\0';
02069 str[2] = static_cast<char> (0x80 | ChopLowBits(&code_point, 6)); // 10xxxxxx
02070 str[1] = static_cast<char> (0x80 | ChopLowBits(&code_point, 6)); // 10xxxxxx
02071 str[0] = static_cast<char> (0xE0 | code_point); // 1110xxxx
02072 } else { // code_point <= kMaxCodePoint4
02073 str[4] = '\0';
02074 str[3] = static_cast<char> (0x80 | ChopLowBits(&code_point, 6)); // 10xxxxxx
02075 str[2] = static_cast<char> (0x80 | ChopLowBits(&code_point, 6)); // 10xxxxxx
02076 str[1] = static_cast<char> (0x80 | ChopLowBits(&code_point, 6)); // 10xxxxxx
02077 str[0] = static_cast<char> (0xF0 | code_point); // 11110xxx
02078 }

```

```

02079 return str;
02080 }
02081
02082 // The following two functions only make sense if the system
02083 // uses UTF-16 for wide string encoding. All supported systems
02084 // with 16 bit wchar_t (Windows, Cygwin) do use UTF-16.
02085
02086 // Determines if the arguments constitute UTF-16 surrogate pair
02087 // and thus should be combined into a single Unicode code point
02088 // using CreateCodePointFromUtf16SurrogatePair.
02089
02090 inline bool IsUtf16SurrogatePair(wchar_t first, wchar_t second) {
02091 return sizeof(wchar_t) == 2 && (first & 0xFC00) == 0xD800 &&
02092 (second & 0xFC00) == 0xDC00;
02093 }
02094
02095 // Creates a Unicode code point from UTF16 surrogate pair.
02096
02097 inline uint32_t CreateCodePointFromUtf16SurrogatePair(wchar_t first,
02098 wchar_t second) {
02099 const auto first_u = static_cast<uint32_t> (first);
02100 const auto second_u = static_cast<uint32_t> (second);
02101 const uint32_t mask = (1 << 10) - 1;
02102 return (sizeof(wchar_t) == 2)
02103 ? (((first_u & mask) << 10) | (second_u & mask)) + 0x10000
02104 :
02105 // This function should not be called when the condition is
02106 // false, but we provide a sensible default in case it is.
02107 first_u;
02108 }
02109
02110 // Converts a wide string to a narrow string in UTF-8 encoding.
02111 // The wide string is assumed to have the following encoding:
02112 // UTF-16 if sizeof(wchar_t) == 2 (on Windows, Cygwin)
02113 // UTF-32 if sizeof(wchar_t) == 4 (on Linux)
02114 // Parameter str points to a null-terminated wide string.
02115 // Parameter num_chars may additionally limit the number
02116 // of wchar_t characters processed. -1 is used when the entire string
02117 // should be processed.
02118 // If the string contains code points that are not valid Unicode code points
02119 // (i.e. outside of Unicode range U+0 to U+10FFFF) they will be output
02120 // as '(Invalid Unicode 0xFFFFFFFF)'. If the string is in UTF16 encoding
02121 // and contains invalid UTF-16 surrogate pairs, values in those pairs
02122 // will be encoded as individual Unicode characters from Basic Normal Plane.
02123
02124 std::string WideStringToUtf8(const wchar_t* str, int num_chars) {
02125 if (num_chars == -1) num_chars = static_cast<int> (wcslen(str));
02126
02127 ::std::stringstream stream;
02128 for (int i = 0; i < num_chars; ++i) {
02129 uint32_t unicode_code_point;
02130
02131 if (str[i] == L'\0') {
02132 break;
02133 } else if (i + 1 < num_chars && IsUtf16SurrogatePair(str[i], str[i + 1])) {
02134 unicode_code_point =
02135 CreateCodePointFromUtf16SurrogatePair(str[i], str[i + 1]);
02136 i++;
02137 } else {
02138 unicode_code_point = static_cast<uint32_t> (str[i]);
02139 }
02140
02141 stream << CodePointToUtf8(unicode_code_point);
02142 }
02143 return StringStreamToString(&stream);
02144 }
02145
02146 // Converts a wide C string to an std::string using the UTF-8 encoding.
02147 // NULL will be converted to "(null)".
02148
02149 std::string String::ShowWideCString(const wchar_t* wide_c_str) {
02150 if (wide_c_str == nullptr) return "(null)";
02151
02152 return internal::WideStringToUtf8(wide_c_str, -1);
02153 }
02154
02155 // Compares two wide C strings. Returns true if and only if they have the
02156 // same content.
02157 //
02158 // Unlike wscmp(), this function can handle NULL argument(s). A NULL
02159 // C string is considered different to any non-NULL C string,
02160 // including the empty string.
02161
02162 bool String::WideCStringEquals(const wchar_t* lhs, const wchar_t* rhs) {
02163 if (lhs == nullptr) return rhs == nullptr;
02164
02165 if (rhs == nullptr) return false;

```



```

02166 return wcscmp(lhs, rhs) == 0;
02167 }
02168 }
02169
02170 // Helper function for *_STREQ on wide strings.
02171
02172 AssertionResult CmpHelperSTREQ(const char* lhs_expression,
02173 const char* rhs_expression, const wchar_t* lhs,
02174 const wchar_t* rhs) {
02175 if (String::WideCStringEquals(lhs, rhs)) {
02176 return AssertionSuccess();
02177 }
02178 return EqFailure(lhs_expression, rhs_expression, PrintToString(lhs),
02179 PrintToString(rhs), false);
02180 }
02181
02182 // Helper function for *_STRNE on wide strings.
02183
02184 AssertionResult CmpHelperSTRNE(const char* s1_expression,
02185 const char* s2_expression, const wchar_t* s1,
02186 const wchar_t* s2) {
02187 if (!String::WideCStringEquals(s1, s2)) {
02188 return AssertionSuccess();
02189 }
02190 return AssertionFailure()
02191 << "Expected: (" < s1_expression < ") != (" < s2_expression
02192 << "), actual: " < PrintToString(s1) < " vs " < PrintToString(s2);
02193 }
02194
02195 // Compares two C strings, ignoring case. Returns true if and only if they have
02196 // the same content.
02197 //
02198 // Unlike strcasecmp(), this function can handle NULL argument(s). A
02199 // NULL C string is considered different to any non-NULL C string,
02200 // including the empty string.
02201
02202 bool String::CaseInsensitiveCStringEquals(const char* lhs, const char* rhs) {
02203 if (lhs == nullptr) return rhs == nullptr;
02204 if (rhs == nullptr) return false;
02205 return posix::StrCaseCmp(lhs, rhs) == 0;
02206 }
02207
02208 // Compares two wide C strings, ignoring case. Returns true if and only if they
02209 // have the same content.
02210 //
02211 // Unlike wcscasecmp(), this function can handle NULL argument(s).
02212 // A NULL C string is considered different to any non-NULL wide C string,
02213 // including the empty string.
02214 // NB: The implementations on different platforms slightly differ.
02215 // On windows, this method uses _wcsicmp which compares according to LC_CTYPE
02216 // environment variable. On GNU platform this method uses wcscasecmp
02217 // which compares according to LC_CTYPE category of the current locale.
02218 // On MacOS X, it uses towlower, which also uses LC_CTYPE category of the
02219 // current locale.
02220
02221 bool String::CaseInsensitiveWideCStringEquals(const wchar_t* lhs,
02222 const wchar_t* rhs) {
02223 if (lhs == nullptr) return rhs == nullptr;
02224 if (rhs == nullptr) return false;
02225 if (lhs == nullptr) return rhs == nullptr;
02226 if (rhs == nullptr) return false;
02227 }
02228
02229 #if GTEST_OS_WINDOWS
02230 return _wcsicmp(lhs, rhs) == 0;
02231 #elif GTEST_OS_LINUX && !GTEST_OS_LINUX_ANDROID
02232 return wcscasecmp(lhs, rhs) == 0;
02233 #else
02234 // Android, Mac OS X and Cygwin don't define wcscasecmp.
02235 // Other unknown OSes may not define it either.
02236 wint_t left, right;
02237 do {
02238 left = towlower(static_cast<wint_t> (*lhs++));
02239 right = towlower(static_cast<wint_t> (*rhs++));
02240 } while (left && left == right);
02241 return left == right;
02242 #endif // OS selector
02243 }
02244
02245 // Returns true if and only if str ends with the given suffix, ignoring case.
02246 // Any string is considered to end with an empty suffix.
02247
02248 bool String::EndsWithCaseInsensitive(const std::string& str,
02249 const std::string& suffix) {
02250 const size_t str_len = str.length();
02251 const size_t suffix_len = suffix.length();
02252 return (str_len >= suffix_len) &&

```

```

02253 CaseInsensitiveCStringEquals(str.c_str() + str_len - suffix_len,
02254 suffix.c_str());
02255 }
02256
02257 // Formats an int value as "%02d".
02258
02259 std::string String::FormatIntWidth2(int value) {
02260 return FormatIntWidthN(value, 2);
02261 }
02262
02263 // Formats an int value to given width with leading zeros.
02264
02265 std::string String::FormatIntWidthN(int value, int width) {
02266 std::stringstream ss;
02267 ss < std::setfill('0') < std::setw(width) < value;
02268 return ss.str();
02269 }
02270
02271 // Formats an int value as "%X".
02272
02273 std::string String::FormatHexUInt32(uint32_t value) {
02274 std::stringstream ss;
02275 ss < std::hex < std::uppercase < value;
02276 return ss.str();
02277 }
02278
02279 // Formats an int value as "%X".
02280
02281 std::string String::FormatHexInt(int value) {
02282 return FormatHexUInt32(static_cast<uint32_t> (value));
02283 }
02284
02285 // Formats a byte as "%02X".
02286
02287 std::string String::FormatByte(unsigned char value) {
02288 std::stringstream ss;
02289 ss < std::setfill('0') < std::setw(2) < std::hex < std::uppercase
02290 < static_cast<unsigned int> (value);
02291 return ss.str();
02292 }
02293
02294 // Converts the buffer in a stringstream to an std::string, converting NUL
02295 // bytes to "\\0" along the way.
02296
02297 std::string StringStreamToString(::std::stringstream* ss) {
02298 const ::std::string& str = ss->str();
02299 const char* const start = str.c_str();
02300 const char* const end = start + str.length();
02301
02302 std::string result;
02303 result.reserve(static_cast<size_t> (2 * (end - start)));
02304 for (const char* ch = start; ch != end; ++ch) {
02305 if (*ch == '\\0') {
02306 result += "\\0"; // Replaces NUL with "\\0";
02307 } else {
02308 result += *ch;
02309 }
02310 }
02311
02312 return result;
02313 }
02314
02315 // Appends the user-supplied message to the Google-Test-generated message.
02316
02317 std::string AppendUserMessage(const std::string& gtest_msg,
02318 const Message& user_msg) {
02319 // Appends the user message if it's non-empty.
02320 const std::string user_msg_string = user_msg.GetString();
02321 if (user_msg_string.empty()) {
02322 return gtest_msg;
02323 }
02324 if (gtest_msg.empty()) {
02325 return user_msg_string;
02326 }
02327 return gtest_msg + "\\n" + user_msg_string;
02328 }
02329
02330 } // namespace internal
02331
02332 // class TestResult
02333
02334 // Creates an empty TestResult.
02335
02336 TestResult::TestResult()
02337 : death_test_count_(0), start_timestamp_(0), elapsed_time_(0) {
02338 }
02339

```

```

02340 // D'tor.
02341
02342 TestResult::~TestResult() {
02343 }
02344
02345 // Returns the i-th test part result among all the results. i can
02346 // range from 0 to total_part_count() - 1. If i is not in that range,
02347 // aborts the program.
02348
02349 const TestPartResult& TestResult::GetTestPartResult(int i) const {
02350 if (i < 0 || i >= total_part_count()) internal::posix::Abort();
02351 return test_part_results_.at(static_cast<size_t>(i));
02352 }
02353
02354 // Returns the i-th test property. i can range from 0 to
02355 // test_property_count() - 1. If i is not in that range, aborts the
02356 // program.
02357
02358 const TestProperty& TestResult::GetTestProperty(int i) const {
02359 if (i < 0 || i >= test_property_count()) internal::posix::Abort();
02360 return test_properties_.at(static_cast<size_t>(i));
02361 }
02362
02363 // Clears the test part results.
02364
02365 void TestResult::ClearTestPartResults() {
02366 test_part_results_.clear();
02367 }
02368
02369 // Adds a test part result to the list.
02370
02371 void TestResult::AddTestPartResult(const TestPartResult& test_part_result) {
02372 test_part_results_.push_back(test_part_result);
02373 }
02374
02375 // Adds a test property to the list. If a property with the same key as the
02376 // supplied property is already represented, the value of this test_property
02377 // replaces the old value for that key.
02378
02379 void TestResult::RecordProperty(const std::string& xml_element,
02380 const TestProperty& test_property) {
02381 if (!ValidateTestProperty(xml_element, test_property)) {
02382 return;
02383 }
02384 internal::MutexLock lock(&test_properties_mutex_);
02385 const std::vector<TestProperty>::iterator property_with_matching_key =
02386 std::find_if(test_properties_.begin(), test_properties_.end(),
02387 internal::TestPropertyKeyIs(test_property.key()));
02388 if (property_with_matching_key == test_properties_.end()) {
02389 test_properties_.push_back(test_property);
02390 return;
02391 }
02392 property_with_matching_key->SetValue(test_property.value());
02393 }
02394
02395 // The list of reserved attributes used in the <testsuites> element of XML
02396 // output.
02397 static const char* const kReservedTestSuitesAttributes[] = {
02398 "disabled", "errors", "failures", "name",
02399 "random_seed", "tests", "time", "timestamp"
02400 };
02401
02402 // The list of reserved attributes used in the <testsuite> element of XML
02403 // output.
02404 static const char* const kReservedTestSuiteAttributes[] = {
02405 "disabled", "errors", "failures", "name",
02406 "tests", "time", "timestamp", "skipped"
02407 };
02408
02409 // The list of reserved attributes used in the <testcase> element of XML output.
02410 static const char* const kReservedTestCaseAttributes[] = {
02411 "classname", "name", "status", "time",
02412 "type_param", "value_param", "file", "line"
02413 };
02414
02415 // Use a slightly different set for allowed output to ensure existing tests can
02416 // still RecordProperty("result") or "RecordProperty(timestamp)"
02417 static const char* const kReservedOutputTestCaseAttributes[] = {
02418 "classname", "name", "status", "time", "type_param",
02419 "value_param", "file", "line", "result", "timestamp"
02420 };
02421
02422 template <size_t kSize>
02423 std::vector<std::string> ArrayAsVector(const char* const (&array)[kSize]) {
02424 return std::vector<std::string>(array, array + kSize);
02425 }
02426

```

```

02427 static std::vector<std::string> GetReservedAttributesForElement(
02428 const std::string& xml_element) {
02429 if (xml_element == "testsuites") {
02430 return ArrayAsVector(kReservedTestSuitesAttributes);
02431 } else if (xml_element == "testsuite") {
02432 return ArrayAsVector(kReservedTestSuiteAttributes);
02433 } else if (xml_element == "testcase") {
02434 return ArrayAsVector(kReservedTestCaseAttributes);
02435 } else {
02436 GTEST_CHECK_(false) << "Unrecognized xml_element provided: " << xml_element;
02437 }
02438 // This code is unreachable but some compilers may not realizes that.
02439 return std::vector<std::string>();
02440 }
02441
02442 // TODO(jdesprez): Merge the two getReserved attributes once skip is improved
02443
02444 static std::vector<std::string> GetReservedOutputAttributesForElement(
02445 const std::string& xml_element) {
02446 if (xml_element == "testsuites") {
02447 return ArrayAsVector(kReservedTestSuitesAttributes);
02448 } else if (xml_element == "testsuite") {
02449 return ArrayAsVector(kReservedTestSuiteAttributes);
02450 } else if (xml_element == "testcase") {
02451 return ArrayAsVector(kReservedOutputTestCaseAttributes);
02452 } else {
02453 GTEST_CHECK_(false) << "Unrecognized xml_element provided: " << xml_element;
02454 }
02455 // This code is unreachable but some compilers may not realizes that.
02456 return std::vector<std::string>();
02457 }
02458
02459 static std::string FormatWordList(const std::vector<std::string>& words) {
02460 Message word_list;
02461 for (size_t i = 0; i < words.size(); ++i) {
02462 if (i > 0 && words.size() > 2) {
02463 word_list << ", ";
02464 }
02465 if (i == words.size() - 1) {
02466 word_list << "and ";
02467 }
02468 word_list << "'" << words[i] << "'";
02469 }
02470 return word_list.GetString();
02471 }
02472
02473 static bool ValidateTestPropertyName(
02474 const std::string& property_name,
02475 const std::vector<std::string>& reserved_names) {
02476 if (std::find(reserved_names.begin(), reserved_names.end(), property_name) !=
02477 reserved_names.end()) {
02478 ADD_FAILURE() << "Reserved key used in RecordProperty(): " << property_name
02479 << " (" << FormatWordList(reserved_names)
02480 << " are reserved by " << GTEST_NAME_ << ")";
02481 return false;
02482 }
02483 return true;
02484 }
02485
02486 // Adds a failure if the key is a reserved attribute of the element named
02487 // xml_element. Returns true if the property is valid.
02488
02489 bool TestResult::ValidateTestProperty(const std::string& xml_element,
02490 const TestProperty& test_property) {
02491 return ValidateTestPropertyName(test_property.key(),
02492 GetReservedAttributesForElement(xml_element));
02493 }
02494
02495 // Clears the object.
02496
02497 void TestResult::Clear() {
02498 test_part_results_.clear();
02499 test_properties_.clear();
02500 death_test_count_ = 0;
02501 elapsed_time_ = 0;
02502 }
02503
02504 // Returns true off the test part was skipped.
02505
02506 static bool TestPartSkipped(const TestPartResult& result) {
02507 return result.skipped();
02508 }
02509
02510 // Returns true if and only if the test was skipped.
02511
02512 bool TestResult::Skipped() const {
02513 return !Failed() && CountIf(test_part_results_, TestPartSkipped) > 0;

```

```

02514 }
02515
02516 // Returns true if and only if the test failed.
02517
02518 bool TestResult::Failed() const {
02519 for (int i = 0; i < total_part_count(); ++i) {
02520 if (GetTestPartResult(i).failed()) return true;
02521 }
02522 return false;
02523 }
02524
02525 // Returns true if and only if the test part fatally failed.
02526
02527 static bool TestPartFatallyFailed(const TestPartResult& result) {
02528 return result.fatally_failed();
02529 }
02530
02531 // Returns true if and only if the test fatally failed.
02532
02533 bool TestResult::HasFatalFailure() const {
02534 return CountIf(test_part_results_, TestPartFatallyFailed) > 0;
02535 }
02536
02537 // Returns true if and only if the test part non-fatally failed.
02538
02539 static bool TestPartNonfatallyFailed(const TestPartResult& result) {
02540 return result.nonfatally_failed();
02541 }
02542
02543 // Returns true if and only if the test has a non-fatal failure.
02544
02545 bool TestResult::HasNonfatalFailure() const {
02546 return CountIf(test_part_results_, TestPartNonfatallyFailed) > 0;
02547 }
02548
02549 // Gets the number of all test parts. This is the sum of the number
02550 // of successful test parts and the number of failed test parts.
02551
02552 int TestResult::total_part_count() const {
02553 return static_cast<int> (test_part_results_.size());
02554 }
02555
02556 // Returns the number of the test properties.
02557
02558 int TestResult::test_property_count() const {
02559 return static_cast<int> (test_properties_.size());
02560 }
02561
02562 // class Test
02563
02564 // Creates a Test object.
02565
02566 // The c'tor saves the states of all flags.
02567
02568 Test::Test() : gtest_flag_saver_(new GTEST_FLAG_SAVER_) {
02569 }
02570
02571 // The d'tor restores the states of all flags. The actual work is
02572 // done by the d'tor of the gtest_flag_saver_ field, and thus not
02573 // visible here.
02574
02575 Test::~Test() {
02576 }
02577
02578 // Sets up the test fixture.
02579 //
02580 // A sub-class may override this.
02581
02582 void Test::SetUp() {
02583 }
02584
02585 // Tears down the test fixture.
02586 //
02587 // A sub-class may override this.
02588
02589 void Test::TearDown() {
02590 }
02591
02592 // Allows user supplied key value pairs to be recorded for later output.
02593
02594 void Test::RecordProperty(const std::string& key, const std::string& value) {
02595 UnitTest::GetInstance()->RecordProperty(key, value);
02596 }
02597
02598 // Allows user supplied key value pairs to be recorded for later output.
02599
02600 void Test::RecordProperty(const std::string& key, int value) {

```

```

02601 Message value_message;
02602 value_message « value;
02603 RecordProperty(key, value_message.GetString().c_str());
02604 }
02605
02606 namespace internal {
02607
02608 void ReportFailureInUnknownLocation(TestPartResult::Type result_type,
02609 const std::string& message) {
02610 // This function is a friend of UnitTest and as such has access to
02611 // AddTestPartResult.
02612 UnitTest::GetInstance()->AddTestPartResult(
02613 result_type,
02614 nullptr, // No info about the source file where the exception occurred.
02615 -1, // We have no info on which line caused the exception.
02616 message,
02617 ""); // No stack trace, either.
02618 }
02619
02620 } // namespace internal
02621
02622 // Google Test requires all tests in the same test suite to use the same test
02623 // fixture class. This function checks if the current test has the
02624 // same fixture class as the first test in the current test suite. If
02625 // yes, it returns true; otherwise it generates a Google Test failure and
02626 // returns false.
02627
02628 bool Test::HasSameFixtureClass() {
02629 internal::UnitTestImpl * const impl = internal::GetUnitTestImpl();
02630 const TestSuite * const test_suite = impl->current_test_suite();
02631
02632 // Info about the first test in the current test suite.
02633 const TestInfo * const first_test_info = test_suite->test_info_list()[0];
02634 const internal::TypeId first_fixture_id = first_test_info->fixture_class_id_;
02635 const char* const first_test_name = first_test_info->name();
02636
02637 // Info about the current test.
02638 const TestInfo * const this_test_info = impl->current_test_info();
02639 const internal::TypeId this_fixture_id = this_test_info->fixture_class_id_;
02640 const char* const this_test_name = this_test_info->name();
02641
02642 if (this_fixture_id != first_fixture_id) {
02643 // Is the first test defined using TEST?
02644 const bool first_is_TEST = first_fixture_id == internal::GetTestTypeId();
02645 // Is this test defined using TEST?
02646 const bool this_is_TEST = this_fixture_id == internal::GetTestTypeId();
02647
02648 if (first_is_TEST || this_is_TEST) {
02649 // Both TEST and TEST_F appear in same test suite, which is incorrect.
02650 // Tell the user how to fix this.
02651
02652 // Gets the name of the TEST and the name of the TEST_F. Note
02653 // that first_is_TEST and this_is_TEST cannot both be true, as
02654 // the fixture IDs are different for the two tests.
02655 const char* const TEST_name =
02656 first_is_TEST ? first_test_name : this_test_name;
02657 const char* const TEST_F_name =
02658 first_is_TEST ? this_test_name : first_test_name;
02659
02660 ADD_FAILURE()
02661 « "All tests in the same test suite must use the same test fixture\n"
02662 « "class, so mixing TEST_F and TEST in the same test suite is\n"
02663 « "illegal. In test suite " « this_test_info->test_suite_name()
02664 « ",\n"
02665 « "test " « TEST_F_name « " is defined using TEST_F but\n"
02666 « "test " « TEST_name « " is defined using TEST. You probably\n"
02667 « "want to change the TEST to TEST_F or move it to another test\n"
02668 « "case.";
02669 } else {
02670 // Two fixture classes with the same name appear in two different
02671 // namespaces, which is not allowed. Tell the user how to fix this.
02672 ADD_FAILURE()
02673 « "All tests in the same test suite must use the same test fixture\n"
02674 « "class. However, in test suite "
02675 « this_test_info->test_suite_name() « ",\n"
02676 « "you defined test " « first_test_name « " and test "
02677 « this_test_name « "\n"
02678 « "using two different test fixture classes. This can happen if\n"
02679 « "the two classes are from different namespaces or translation\n"
02680 « "units and have the same name. You should probably rename one\n"
02681 « "of the classes to put the tests into different test suites.";
02682 }
02683 return false;
02684 }
02685
02686 return true;
02687 }

```

```

02688
02689 #if GTEST_HAS_SEH
02690
02691 // Adds an "exception thrown" fatal failure to the current test. This
02692 // function returns its result via an output parameter pointer because VC++
02693 // prohibits creation of objects with destructors on stack in functions
02694 // using __try (see error C2712).
02695
02696 static std::string* FormatSehExceptionMessage(DWORD exception_code,
02697 const char* location) {
02698 Message message;
02699 message << "SEH exception with code 0x" << std::setbase(16) << exception_code
02700 << std::setbase(10) << " thrown in " << location << ".";
02701
02702 return new std::string(message.GetString());
02703 }
02704
02705 #endif // GTEST_HAS_SEH
02706
02707 namespace internal {
02708
02709 #if GTEST_HAS_EXCEPTIONS
02710
02711 // Adds an "exception thrown" fatal failure to the current test.
02712
02713 static std::string FormatCxxExceptionMessage(const char* description,
02714 const char* location) {
02715 Message message;
02716 if (description != nullptr) {
02717 message << "C++ exception with description \"" << description << "\"";
02718 } else {
02719 message << "Unknown C++ exception";
02720 }
02721 message << " thrown in " << location << ".";
02722
02723 return message.GetString();
02724 }
02725
02726 static std::string PrintTestPartResultToString(
02727 const TestPartResult& test_part_result);
02728
02729 GoogleTestFailureException::GoogleTestFailureException(
02730 const TestPartResult& failure)
02731 : ::std::runtime_error(PrintTestPartResultToString(failure).c_str()) {}
02732 }
02733
02734 #endif // GTEST_HAS_EXCEPTIONS
02735
02736 // We put these helper functions in the internal namespace as IBM's xlc
02737 // compiler rejects the code if they were declared static.
02738
02739 // Runs the given method and handles SEH exceptions it throws, when
02740 // SEH is supported; returns the 0-value for type Result in case of an
02741 // SEH exception. (Microsoft compilers cannot handle SEH and C++
02742 // exceptions in the same function. Therefore, we provide a separate
02743 // wrapper function for handling SEH exceptions.)
02744
02745 template <class T, typename Result>
02746 Result HandleSehExceptionsInMethodIfSupported(T* object, Result (T::*method)(),
02747 const char* location) {
02748 #if GTEST_HAS_SEH
02749 __try
02750 {
02751 return (object->*method)();
02752 }
02753 __except(internal::UnitTestOptions::GTestShouldProcessSEH(// NOLINT
02754 GetExceptionCode())) {
02755 // We create the exception message on the heap because VC++ prohibits
02756 // creation of objects with destructors on stack in functions using __try
02757 // (see error C2712).
02758 std::string* exception_message =
02759 FormatSehExceptionMessage(GetExceptionCode(), location);
02760 internal::ReportFailureInUnknownLocation(TestPartResult::kFatalFailure,
02761 *exception_message);
02762 delete exception_message;
02763 return static_cast<Result>(0);
02764 }
02765 #else
02766 (void) location;
02767 return (object->*method)();
02768 #endif // GTEST_HAS_SEH
02769 }
02770
02771 // Runs the given method and catches and reports C++ and/or SEH-style
02772 // exceptions, if they are supported; returns the 0-value for type
02773 // Result in case of an SEH exception.

```

```

02775
02776 template <class T, typename Result>
02777 Result HandleExceptionsInMethodIfSupported(T* object, Result(T::*method)(),
02778 const char* location) {
02779 // NOTE: The user code can affect the way in which Google Test handles
02780 // exceptions by setting GTEST_FLAG(catch_exceptions), but only before
02781 // RUN_ALL_TESTS() starts. It is technically possible to check the flag
02782 // after the exception is caught and either report or re-throw the
02783 // exception based on the flag's value:
02784 //
02785 // try {
02786 // // Perform the test method.
02787 // } catch (...) {
02788 // if (GTEST_FLAG_GET(catch_exceptions))
02789 // // Report the exception as failure.
02790 // else
02791 // throw; // Re-throws the original exception.
02792 // }
02793 //
02794 // However, the purpose of this flag is to allow the program to drop into
02795 // the debugger when the exception is thrown. On most platforms, once the
02796 // control enters the catch block, the exception origin information is
02797 // lost and the debugger will stop the program at the point of the
02798 // re-throw in this function -- instead of at the point of the original
02799 // throw statement in the code under test. For this reason, we perform
02800 // the check early, sacrificing the ability to affect Google Test's
02801 // exception handling in the method where the exception is thrown.
02802 if (internal::GetUnitTestImpl()->catch_exceptions()) {
02803 #if GTEST_HAS_EXCEPTIONS
02804 try {
02805 return HandleSehExceptionsInMethodIfSupported(object, method, location);
02806 } catch (const AssertionException&) { // NOLINT
02807 // This failure was reported already.
02808 } catch (const internal::GoogleTestFailureException&) { // NOLINT
02809 // This exception type can only be thrown by a failed Google
02810 // Test assertion with the intention of letting another testing
02811 // framework catch it. Therefore we just re-throw it.
02812 throw;
02813 } catch (const std::exception& e) { // NOLINT
02814 internal::ReportFailureInUnknownLocation(
02815 TestPartResult::kFatalFailure,
02816 FormatCxxExceptionMessage(e.what(), location));
02817 } catch (...) { // NOLINT
02818 internal::ReportFailureInUnknownLocation(
02819 TestPartResult::kFatalFailure,
02820 FormatCxxExceptionMessage(nullptr, location));
02821 }
02822 return static_cast<Result> (0);
02823 #else
02824 return HandleSehExceptionsInMethodIfSupported(object, method, location);
02825 #endif // GTEST_HAS_EXCEPTIONS
02826 } else {
02827 return (object->*method)();
02828 }
02829 }
02830
02831 } // namespace internal
02832
02833 // Runs the test and updates the test result.
02834
02835 void Test::Run() {
02836 if (!HasSameFixtureClass()) return;
02837
02838 internal::UnitTestImpl* const impl = internal::GetUnitTestImpl();
02839 impl->os_stack_trace_getter()->UponLeavingGTest();
02840 internal::HandleExceptionsInMethodIfSupported(this, &Test::SetUp, "SetUp()");
02841 // We will run the test only if SetUp() was successful and didn't call
02842 // GTEST_SKIP().
02843 if (!HasFatalFailure() && !IsSkipped()) {
02844 impl->os_stack_trace_getter()->UponLeavingGTest();
02845 internal::HandleExceptionsInMethodIfSupported(this, &Test::TestBody,
02846 "the test body");
02847 }
02848
02849 // However, we want to clean up as much as possible. Hence we will
02850 // always call TearDown(), even if SetUp() or the test body has
02851 // failed.
02852 impl->os_stack_trace_getter()->UponLeavingGTest();
02853 internal::HandleExceptionsInMethodIfSupported(this, &Test::TearDown,
02854 "TearDown()");
02855 }
02856
02857 // Returns true if and only if the current test has a fatal failure.
02858
02859 bool Test::HasFatalFailure() {
02860 return internal::GetUnitTestImpl()->current_test_result()->HasFatalFailure();
02861 }

```



```

02862
02863 // Returns true if and only if the current test has a non-fatal failure.
02864
02865 bool Test::HasNonfatalFailure() {
02866 return internal::GetUnitTestImpl()
02867 ->current_test_result()
02868 ->HasNonfatalFailure();
02869 }
02870
02871 // Returns true if and only if the current test was skipped.
02872
02873 bool Test::IsSkipped() {
02874 return internal::GetUnitTestImpl()->current_test_result()->Skipped();
02875 }
02876
02877 // class TestInfo
02878
02879 // Constructs a TestInfo object. It assumes ownership of the test factory
02880 // object.
02881
02882 TestInfo::TestInfo(const std::string& a_test_suite_name,
02883 const std::string& a_name, const char* a_type_param,
02884 const char* a_value_param,
02885 internal::CodeLocation a_code_location,
02886 internal::TypeId fixture_class_id,
02887 internal::TestFactoryBase* factory)
02888 : test_suite_name_(a_test_suite_name),
02889 name_(a_name),
02890 type_param_(a_type_param ? new std::string(a_type_param) : nullptr),
02891 value_param_(a_value_param ? new std::string(a_value_param) : nullptr),
02892 location_(a_code_location),
02893 fixture_class_id_(fixture_class_id),
02894 should_run_(false),
02895 is_disabled_(false),
02896 matches_filter_(false),
02897 is_in_another_shard_(false),
02898 factory_(factory),
02899 result_() {
02900 }
02901
02902 // Destructs a TestInfo object.
02903
02904 TestInfo::~TestInfo() {
02905 delete factory_;
02906 }
02907
02908 namespace internal {
02909
02910 // Creates a new TestInfo object and registers it with Google Test;
02911 // returns the created object.
02912 //
02913 // Arguments:
02914 // test_suite_name: name of the test suite
02915 // name: name of the test
02916 // type_param: the name of the test's type parameter, or NULL if
02917 // this is not a typed or a type-parameterized test.
02918 // value_param: text representation of the test's value parameter,
02919 // or NULL if this is not a value-parameterized test.
02920 // code_location: code location where the test is defined
02921 // fixture_class_id: ID of the test fixture class
02922 // set_up_tc: pointer to the function that sets up the test suite
02923 // tear_down_tc: pointer to the function that tears down the test suite
02924 // factory: pointer to the factory that creates a test object.
02925 // The newly created TestInfo instance will assume
02926 // ownership of the factory object.
02927 //
02928
02929 TestInfo* MakeAndRegisterTestInfo(
02930 const char* test_suite_name, const char* name, const char* type_param,
02931 const char* value_param, CodeLocation code_location,
02932 TypeId fixture_class_id, SetUpTestSuiteFunc set_up_tc,
02933 TearDownTestSuiteFunc tear_down_tc, TestFactoryBase* factory) {
02934 TestInfo* test_info =
02935 new TestInfo(test_suite_name, name, type_param, value_param,
02936 code_location, fixture_class_id, factory);
02937 GetUnitTestImpl()->AddTestInfo(set_up_tc, tear_down_tc, test_info);
02938 return test_info;
02939 }
02940
02941 void ReportInvalidTestSuiteType(const char* test_suite_name,
02942 CodeLocation code_location) {
02943 Message errors;
02944 errors
02945 << "Attempted redefinition of test suite " << test_suite_name << ".\n"
02946 << "All tests in the same test suite must use the same test fixture\n"
02947 << "class. However, in test suite " << test_suite_name << ", you tried\n"
02948 << "to define a test using a fixture class different from the one\n"

```

```

02949 « "used earlier. This can happen if the two fixture classes are\n"
02950 « "from different namespaces and have the same name. You should\n"
02951 « "probably rename one of the classes to put the tests into different\n"
02952 « "test suites.";
02953
02954 GTEST_LOG(ERROR) « FormatFileLocation(code_location.file.c_str(),
02955 code_location.line)
02956 « " " « errors.GetString();
02957 }
02958 } // namespace internal
02959
02960 namespace {
02961
02962 // A predicate that checks the test name of a TestInfo against a known
02963 // value.
02964 //
02965 // This is used for implementation of the TestSuite class only. We put
02966 // it in the anonymous namespace to prevent polluting the outer
02967 // namespace.
02968 //
02969 // TestNameIs is copyable.
02970
02971 class TestNameIs {
02972 public:
02973 // Constructor.
02974 //
02975 // TestNameIs has NO default constructor.
02976
02977 explicit TestNameIs(const char* name) : name_(name) {
02978 }
02979
02980 // Returns true if and only if the test name of test_info matches name_.
02981
02982 bool operator()(const TestInfo* test_info) const {
02983 return test_info && test_info->name() == name_;
02984 }
02985
02986 private:
02987 std::string name_;
02988 };
02989 } // namespace
02990
02991 namespace internal {
02992
02993 // This method expands all parameterized tests registered with macros TEST_P
02994 // and INSTANTIATE_TEST_SUITE_P into regular tests and registers those.
02995 // This will be done just once during the program runtime.
02996
02997 void UnitTestImpl::RegisterParameterizedTests() {
02998 if (!parameterized_tests_registered_) {
02999 parameterized_test_registry_.RegisterTests();
03000 type_parameterized_test_registry_.CheckForInstantiations();
03001 parameterized_tests_registered_ = true;
03002 }
03003 }
03004 }
03005
03006 } // namespace internal
03007
03008 // Creates the test object, runs it, records its result, and then
03009 // deletes it.
03010
03011 void TestInfo::Run() {
03012 TestEventListener* repeater = UnitTest::GetInstance()->listeners().repeater();
03013 if (!should_run_) {
03014 if (is_disabled_ && matches_filter_) repeater->OnTestDisabled(*this);
03015 return;
03016 }
03017
03018 // Tells UnitTest where to store test result.
03019 internal::UnitTestImpl* const impl = internal::GetUnitTestImpl();
03020 impl->set_current_test_info(this);
03021
03022 // Notifies the unit test event listeners that a test is about to start.
03023 repeater->OnTestStart(*this);
03024 result_.set_start_timestamp(internal::GetTimeInMillis());
03025 internal::Timer timer;
03026 impl->os_stack_trace_getter()->UponLeavingGTest();
03027
03028 // Creates the test object.
03029 Test* const test = internal::HandleExceptionsInMethodIfSupported(
03030 factory_, &internal::TestFactoryBase::CreateTest,
03031 "the test fixture's constructor");
03032
03033 // Runs the test if the constructor didn't generate a fatal failure or invoke
03034 // GTEST_SKIP().
03035 // Note that the object will not be null

```

```

03036 if (!Test::HasFatalFailure() && !Test::IsSkipped()) {
03037 // This doesn't throw as all user code that can throw are wrapped into
03038 // exception handling code.
03039 test->Run();
03040 }
03041
03042 if (test != nullptr) {
03043 // Deletes the test object.
03044 impl->os_stack_trace_getter()->UponLeavingGTest();
03045 internal::HandleExceptionsInMethodIfSupported(
03046 test, &Test::DeleteSelf_, "the test fixture's destructor");
03047 }
03048
03049 result_.set_elapsed_time(timer.Elapsed());
03050
03051 // Notifies the unit test event listener that a test has just finished.
03052 repeater->OnTestEnd(*this);
03053
03054 // Tells UnitTest to stop associating assertion results to this
03055 // test.
03056 impl->set_current_test_info(nullptr);
03057 }
03058
03059 // Skip and records a skipped test result for this object.
03060
03061 void TestInfo::Skip() {
03062 if (!should_run_) return;
03063
03064 internal::UnitTestImpl * const impl = internal::GetUnitTestImpl();
03065 impl->set_current_test_info(this);
03066
03067 TestEventListener* repeater = UnitTest::GetInstance()->listeners().repeater();
03068
03069 // Notifies the unit test event listeners that a test is about to start.
03070 repeater->OnTestStart(*this);
03071
03072 const TestPartResult test_part_result =
03073 TestPartResult(TestPartResult::kSkip, this->file(), this->line(), "");
03074 impl->GetTestPartResultReporterForCurrentThread()->ReportTestPartResult(
03075 test_part_result);
03076
03077 // Notifies the unit test event listener that a test has just finished.
03078 repeater->OnTestEnd(*this);
03079 impl->set_current_test_info(nullptr);
03080 }
03081
03082 // class TestSuite
03083
03084 // Gets the number of successful tests in this test suite.
03085
03086 int TestSuite::successful_test_count() const {
03087 return CountIf(test_info_list_, TestPassed);
03088 }
03089
03090 // Gets the number of successful tests in this test suite.
03091
03092 int TestSuite::skipped_test_count() const {
03093 return CountIf(test_info_list_, TestSkipped);
03094 }
03095
03096 // Gets the number of failed tests in this test suite.
03097
03098 int TestSuite::failed_test_count() const {
03099 return CountIf(test_info_list_, TestFailed);
03100 }
03101
03102 // Gets the number of disabled tests that will be reported in the XML report.
03103
03104 int TestSuite::reportable_disabled_test_count() const {
03105 return CountIf(test_info_list_, TestReportableDisabled);
03106 }
03107
03108 // Gets the number of disabled tests in this test suite.
03109
03110 int TestSuite::disabled_test_count() const {
03111 return CountIf(test_info_list_, TestDisabled);
03112 }
03113
03114 // Gets the number of tests to be printed in the XML report.
03115
03116 int TestSuite::reportable_test_count() const {
03117 return CountIf(test_info_list_, TestReportable);
03118 }
03119
03120 // Get the number of tests in this test suite that should run.
03121
03122 int TestSuite::test_to_run_count() const {

```

```

03123 return CountIf(test_info_list_, ShouldRunTest);
03124 }
03125
03126 // Gets the number of all tests.
03127
03128 int TestSuite::total_test_count() const {
03129 return static_cast<int> (test_info_list_.size());
03130 }
03131
03132 // Creates a TestSuite with the given name.
03133 //
03134 // Arguments:
03135 //
03136 // a_name: name of the test suite
03137 // a_type_param: the name of the test suite's type parameter, or NULL if
03138 // this is not a typed or a type-parameterized test suite.
03139 // set_up_tc: pointer to the function that sets up the test suite
03140 // tear_down_tc: pointer to the function that tears down the test suite
03141
03142 TestSuite::TestSuite(const char* a_name, const char* a_type_param,
03143 internal::SetUpTestSuiteFunc set_up_tc,
03144 internal::TearDownTestSuiteFunc tear_down_tc)
03145 : name_(a_name),
03146 type_param(a_type_param ? new std::string(a_type_param) : nullptr),
03147 set_up_tc_(set_up_tc),
03148 tear_down_tc_(tear_down_tc),
03149 should_run_(false),
03150 start_timestamp_(0),
03151 elapsed_time_(0) {
03152 }
03153
03154 // Destructor of TestSuite.
03155
03156 TestSuite::~TestSuite() {
03157 // Deletes every Test in the collection.
03158 ForEach(test_info_list_, internal::Delete<TestInfo>);
03159 }
03160
03161 // Returns the i-th test among all the tests. i can range from 0 to
03162 // total_test_count() - 1. If i is not in that range, returns NULL.
03163
03164 const TestInfo* TestSuite::GetTestInfo(int i) const {
03165 const int index = GetElementOr(test_indices_, i, -1);
03166 return index < 0 ? nullptr : test_info_list_[static_cast<size_t> (index)];
03167 }
03168
03169 // Returns the i-th test among all the tests. i can range from 0 to
03170 // total_test_count() - 1. If i is not in that range, returns NULL.
03171
03172 TestInfo* TestSuite::GetMutableTestInfo(int i) {
03173 const int index = GetElementOr(test_indices_, i, -1);
03174 return index < 0 ? nullptr : test_info_list_[static_cast<size_t> (index)];
03175 }
03176
03177 // Adds a test to this test suite. Will delete the test upon
03178 // destruction of the TestSuite object.
03179
03180 void TestSuite::AddTestInfo(TestInfo* test_info) {
03181 test_info_list_.push_back(test_info);
03182 test_indices_.push_back(static_cast<int> (test_indices_.size()));
03183 }
03184
03185 // Runs every test in this TestSuite.
03186
03187 void TestSuite::Run() {
03188 if (!should_run_) return;
03189
03190 internal::UnitTestImpl* const impl = internal::GetUnitTestImpl();
03191 impl->set_current_test_suite(this);
03192
03193 TestEventListener* repeater = UnitTest::GetInstance()->listeners().repeater();
03194
03195 // Call both legacy and the new API
03196 repeater->OnTestSuiteStart(*this);
03197 // Legacy API is deprecated but still available
03198 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
03199 repeater->OnTestCaseStart(*this);
03200 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
03201
03202 impl->os_stack_trace_getter()->UponLeavingGTest();
03203 internal::HandleExceptionsInMethodIfSupported(
03204 this, &TestSuite::RunSetUpTestSuite, "SetUpTestSuite()");
03205
03206 const bool skip_all = ad_hoc_test_result().Failed();
03207
03208 start_timestamp_ = internal::GetTimeInMillis();
03209 internal::Timer timer;

```

```

03210 for (int i = 0; i < total_test_count(); i++) {
03211 if (skip_all) {
03212 GetMutableTestInfo(i)->Skip();
03213 } else {
03214 GetMutableTestInfo(i)->Run();
03215 }
03216 if (GTEST_FLAG_GET(fail_fast) &&
03217 GetMutableTestInfo(i)->result()->Failed()) {
03218 for (int j = i + 1; j < total_test_count(); j++) {
03219 GetMutableTestInfo(j)->Skip();
03220 }
03221 break;
03222 }
03223 }
03224 elapsed_time_ = timer.Elapsed();
03225
03226 impl->os_stack_trace_getter()->UponLeavingGTest();
03227 internal::HandleExceptionsInMethodIfSupported(
03228 this, &TestSuite::RunTearDownTestSuite, "TearDownTestSuite()");
03229
03230 // Call both legacy and the new API
03231 repeater->OnTestSuiteEnd(*this);
03232 // Legacy API is deprecated but still available
03233 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
03234 repeater->OnTestCaseEnd(*this);
03235 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
03236
03237 impl->set_current_test_suite(nullptr);
03238 }
03239
03240 // Skips all tests under this TestSuite.
03241
03242 void TestSuite::Skip() {
03243 if (!should_run_) return;
03244
03245 internal::UnitTestImpl * const impl = internal::GetUnitTestImpl();
03246 impl->set_current_test_suite(this);
03247
03248 TestEventListener* repeater = UnitTest::GetInstance()->listeners().repeater();
03249
03250 // Call both legacy and the new API
03251 repeater->OnTestSuiteStart(*this);
03252 // Legacy API is deprecated but still available
03253 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
03254 repeater->OnTestCaseStart(*this);
03255 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
03256
03257 for (int i = 0; i < total_test_count(); i++) {
03258 GetMutableTestInfo(i)->Skip();
03259 }
03260
03261 // Call both legacy and the new API
03262 repeater->OnTestSuiteEnd(*this);
03263 // Legacy API is deprecated but still available
03264 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
03265 repeater->OnTestCaseEnd(*this);
03266 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
03267
03268 impl->set_current_test_suite(nullptr);
03269 }
03270
03271 // Clears the results of all tests in this test suite.
03272
03273 void TestSuite::ClearResult() {
03274 ad_hoc_test_result_.Clear();
03275 ForEach(test_info_list_, TestInfo::ClearTestResult);
03276 }
03277
03278 // Shuffles the tests in this test suite.
03279
03280 void TestSuite::ShuffleTests(internal::Random* random) {
03281 Shuffle(random, &test_indices_);
03282 }
03283
03284 // Restores the test order to before the first shuffle.
03285
03286 void TestSuite::UnshuffleTests() {
03287 for (size_t i = 0; i < test_indices_.size(); i++) {
03288 test_indices_[i] = static_cast<int>(i);
03289 }
03290 }
03291
03292 // Formats a countable noun. Depending on its quantity, either the
03293 // singular form or the plural form is used. e.g.
03294 //
03295 // FormatCountableNoun(1, "formula", "formuli") returns "1 formula".
03296 // FormatCountableNoun(5, "book", "books") returns "5 books".

```

```

03297
03298 static std::string FormatCountableNoun(int count, const char* singular_form,
03299 const char* plural_form) {
03300 return internal::StreamableToString(count) + " " +
03301 (count == 1 ? singular_form : plural_form);
03302 }
03303
03304 // Formats the count of tests.
03305
03306 static std::string FormatTestCount(int test_count) {
03307 return FormatCountableNoun(test_count, "test", "tests");
03308 }
03309
03310 // Formats the count of test suites.
03311
03312 static std::string FormatTestSuiteCount(int test_suite_count) {
03313 return FormatCountableNoun(test_suite_count, "test suite", "test suites");
03314 }
03315
03316 // Converts a TestPartResult::Type enum to human-friendly string
03317 // representation. Both kNonFatalFailure and kFatalFailure are translated
03318 // to "Failure", as the user usually doesn't care about the difference
03319 // between the two when viewing the test result.
03320
03321 static const char* TestPartResultTypeToString(TestPartResult::Type type) {
03322 switch (type) {
03323 case TestPartResult::kSkip:
03324 return "Skipped\n";
03325 case TestPartResult::kSuccess:
03326 return "Success";
03327
03328 case TestPartResult::kNonFatalFailure:
03329 case TestPartResult::kFatalFailure:
03330 #ifdef _MSC_VER
03331 return "error: ";
03332 #else
03333 return "Failure\n";
03334 #endif
03335 default:
03336 return "Unknown result type";
03337 }
03338 }
03339
03340 namespace internal {
03341 namespace {
03342
03343 enum class GTestColor {
03344 kDefault, kRed, kGreen, kYellow
03345 };
03346 } // namespace
03347
03348 // Prints a TestPartResult to an std::string.
03349
03350 static std::string PrintTestPartResultToString(
03351 const TestPartResult& test_part_result) {
03352 return (Message() << internal::FormatFileLocation(
03353 test_part_result.file_name(),
03354 test_part_result.line_number())
03355 << " "
03356 << TestPartResultTypeToString(test_part_result.type())
03357 << test_part_result.message()
03358 << GetString());
03359 }
03360
03361 // Prints a TestPartResult.
03362
03363 static void PrintTestPartResult(const TestPartResult& test_part_result) {
03364 const std::string& result = PrintTestPartResultToString(test_part_result);
03365 printf("%s\n", result.c_str());
03366 fflush(stdout);
03367 // If the test program runs in Visual Studio or a debugger, the
03368 // following statements add the test part result message to the Output
03369 // window such that the user can double-click on it to jump to the
03370 // corresponding source code location; otherwise they do nothing.
03371 #if GTEST_OS_WINDOWS && !GTEST_OS_WINDOWS_MOBILE
03372 // We don't call OutputDebugString*() on Windows Mobile, as printing
03373 // to stdout is done by OutputDebugString() there already - we don't
03374 // want the same message printed twice.
03375 ::OutputDebugStringA(result.c_str());
03376 ::OutputDebugStringA("\n");
03377 #endif
03378 }
03379
03380 // class PrettyUnitTestResultPrinter
03381 #if GTEST_OS_WINDOWS && !GTEST_OS_WINDOWS_MOBILE && !GTEST_OS_WINDOWS_PHONE && \
03382 !GTEST_OS_WINDOWS_RT && !GTEST_OS_WINDOWS_MINGW
03383

```

```

03384 // Returns the character attribute for the given color.
03385
03386 static WORD GetColorAttribute(GTestColor color) {
03387 switch (color) {
03388 case GTestColor::kRed:
03389 return FOREGROUND_RED;
03390 case GTestColor::kGreen:
03391 return FOREGROUND_GREEN;
03392 case GTestColor::kYellow:
03393 return FOREGROUND_RED | FOREGROUND_GREEN;
03394 default:
03395 return 0;
03396 }
03397 }
03398
03399 static int GetBitOffset(WORD color_mask) {
03400 if (color_mask == 0) return 0;
03401
03402 int bitOffset = 0;
03403 while ((color_mask & 1) == 0) {
03404 color_mask >>= 1;
03405 ++bitOffset;
03406 }
03407 return bitOffset;
03408 }
03409
03410 static WORD GetNewColor(GTestColor color, WORD old_color_attrs) {
03411 // Let's reuse the BG
03412 static const WORD background_mask = BACKGROUND_BLUE | BACKGROUND_GREEN |
03413 BACKGROUND_RED | BACKGROUND_INTENSITY;
03414 static const WORD foreground_mask = FOREGROUND_BLUE | FOREGROUND_GREEN |
03415 FOREGROUND_RED | FOREGROUND_INTENSITY;
03416 const WORD existing_bg = old_color_attrs & background_mask;
03417
03418 WORD new_color =
03419 GetColorAttribute(color) | existing_bg | FOREGROUND_INTENSITY;
03420 static const int bg_bitOffset = GetBitOffset(background_mask);
03421 static const int fg_bitOffset = GetBitOffset(foreground_mask);
03422
03423 if (((new_color & background_mask) >> bg_bitOffset) ==
03424 ((new_color & foreground_mask) >> fg_bitOffset)) {
03425 new_color ^= FOREGROUND_INTENSITY; // invert intensity
03426 }
03427 return new_color;
03428 }
03429
03430 #else
03431
03432 // Returns the ANSI color code for the given color. GTestColor::kDefault is
03433 // an invalid input.
03434
03435 static const char* GetAnsiColorCode(GTestColor color) {
03436 switch (color) {
03437 case GTestColor::kRed:
03438 return "1";
03439 case GTestColor::kGreen:
03440 return "2";
03441 case GTestColor::kYellow:
03442 return "3";
03443 default:
03444 return nullptr;
03445 }
03446 }
03447
03448 #endif // GTEST_OS_WINDOWS && !GTEST_OS_WINDOWS_MOBILE
03449
03450 // Returns true if and only if Google Test should use colors in the output.
03451
03452 bool ShouldUseColor(bool stdout_is_tty) {
03453 std::string c = GTEST_FLAG_GET(color);
03454 const char* const gtest_color = c.c_str();
03455
03456 if (String::CaseInsensitiveCStringEquals(gtest_color, "auto")) {
03457 #if GTEST_OS_WINDOWS && !GTEST_OS_WINDOWS_MINGW
03458 // On Windows the TERM variable is usually not set, but the
03459 // console there does support colors.
03460 return stdout_is_tty;
03461 #else
03462 // On non-Windows platforms, we rely on the TERM variable.
03463 const char* const term = posix::GetEnv("TERM");
03464 const bool term_supports_color =
03465 String::CStringEquals(term, "xterm") ||
03466 String::CStringEquals(term, "xterm-color") ||
03467 String::CStringEquals(term, "xterm-256color") ||
03468 String::CStringEquals(term, "screen") ||
03469 String::CStringEquals(term, "screen-256color") ||
03470 String::CStringEquals(term, "tmux") ||

```

```

03471 String::CStringEquals(term, "tmux-256color") ||
03472 String::CStringEquals(term, "rxvt-unicode") ||
03473 String::CStringEquals(term, "rxvt-unicode-256color") ||
03474 String::CStringEquals(term, "linux") ||
03475 String::CStringEquals(term, "cygwin");
03476 return stdout_is_tty && term_supports_color;
03477 #endif // GTEST_OS_WINDOWS
03478 }
03479
03480 return String::CaseInsensitiveCStringEquals(gtest_color, "yes") ||
03481 String::CaseInsensitiveCStringEquals(gtest_color, "true") ||
03482 String::CaseInsensitiveCStringEquals(gtest_color, "t") ||
03483 String::CStringEquals(gtest_color, "1");
03484 // We take "yes", "true", "t", and "1" as meaning "yes". If the
03485 // value is neither one of these nor "auto", we treat it as "no" to
03486 // be conservative.
03487 }
03488
03489 // Helpers for printing colored strings to stdout. Note that on Windows, we
03490 // cannot simply emit special characters and have the terminal change colors.
03491 // This routine must actually emit the characters rather than return a string
03492 // that would be colored when printed, as can be done on Linux.
03493
03494 GTEST_ATTRIBUTE_PRINTF_(2, 3)
03495 static void ColoredPrintf(GTestColor color, const char* fmt, ...) {
03496 va_list args;
03497 va_start(args, fmt);
03498
03499 static const bool in_color_mode =
03500 ShouldUseColor(posix::IsATTY(posix::FileNo(stdout)) != 0);
03501 const bool use_color = in_color_mode && (color != GTestColor::kDefault);
03502
03503 if (!use_color) {
03504 vprintf(fmt, args);
03505 va_end(args);
03506 return;
03507 }
03508
03509 #if GTEST_OS_WINDOWS && !GTEST_OS_WINDOWS_MOBILE && !GTEST_OS_WINDOWS_PHONE && \
03510 !GTEST_OS_WINDOWS_RT && !GTEST_OS_WINDOWS_MINGW
03511 const HANDLE stdout_handle = GetStdHandle(STD_OUTPUT_HANDLE);
03512
03513 // Gets the current text color.
03514 CONSOLE_SCREEN_BUFFER_INFO buffer_info;
03515 GetConsoleScreenBufferInfo(stdout_handle, &buffer_info);
03516 const WORD old_color_attrs = buffer_info.wAttributes;
03517 const WORD new_color = GetNewColor(color, old_color_attrs);
03518
03519 // We need to flush the stream buffers into the console before each
03520 // SetConsoleTextAttribute call lest it affect the text that is already
03521 // printed but has not yet reached the console.
03522 fflush(stdout);
03523 SetConsoleTextAttribute(stdout_handle, new_color);
03524
03525 vprintf(fmt, args);
03526
03527 fflush(stdout);
03528 // Restores the text color.
03529 SetConsoleTextAttribute(stdout_handle, old_color_attrs);
03530 #else
03531 printf("\033[0;3%sm", GetAnsiColorCode(color));
03532 vprintf(fmt, args);
03533 printf("\033[m"); // Resets the terminal to default.
03534 #endif // GTEST_OS_WINDOWS && !GTEST_OS_WINDOWS_MOBILE
03535 va_end(args);
03536 }
03537
03538 // Text printed in Google Test's text output and --gtest_list_tests
03539 // output to label the type parameter and value parameter for a test.
03540 static const char kTypeParamLabel[] = "TypeParam";
03541 static const char kValueParamLabel[] = "GetParam()";
03542
03543 static void PrintFullTestCommentIfPresent(const TestInfo& test_info) {
03544 const char* const type_param = test_info.type_param();
03545 const char* const value_param = test_info.value_param();
03546
03547 if (type_param != nullptr || value_param != nullptr) {
03548 printf(", where ");
03549 if (type_param != nullptr) {
03550 printf("%s = %s", kTypeParamLabel, type_param);
03551 if (value_param != nullptr) printf(" and ");
03552 }
03553 if (value_param != nullptr) {
03554 printf("%s = %s", kValueParamLabel, value_param);
03555 }
03556 }
03557 }

```



```

03558
03559 // This class implements the TestEventListener interface.
03560 //
03561 // Class PrettyUnitTestResultPrinter is copyable.
03562
03563 class PrettyUnitTestResultPrinter : public TestEventListener {
03564 public:
03565
03566 PrettyUnitTestResultPrinter() {
03567 }
03568
03569 static void PrintTestName(const char* test_suite, const char* test) {
03570 printf("%s.%s", test_suite, test);
03571 }
03572
03573 // The following methods override what's in the TestEventListener class.
03574
03575 void OnTestProgramStart(const UnitTest& /*unit_test*/) override {
03576 }
03577 void OnTestIterationStart(const UnitTest& unit_test, int iteration) override;
03578 void OnEnvironmentsSetUpStart(const UnitTest& unit_test) override;
03579
03580 void OnEnvironmentsSetUpEnd(const UnitTest& /*unit_test*/) override {
03581 }
03582 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
03583 void OnTestCaseStart(const TestCase& test_case) override;
03584 #else
03585 void OnTestSuiteStart(const TestSuite& test_suite) override;
03586 #endif // OnTestCaseStart
03587
03588 void OnTestStart(const TestInfo& test_info) override;
03589 void OnTestDisabled(const TestInfo& test_info) override;
03590
03591 void OnTestPartResult(const TestPartResult& result) override;
03592 void OnTestEnd(const TestInfo& test_info) override;
03593 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
03594 void OnTestCaseEnd(const TestCase& test_case) override;
03595 #else
03596 void OnTestSuiteEnd(const TestSuite& test_suite) override;
03597 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
03598
03599 void OnEnvironmentsTearDownStart(const UnitTest& unit_test) override;
03600
03601 void OnEnvironmentsTearDownEnd(const UnitTest& /*unit_test*/) override {
03602 }
03603 void OnTestIterationEnd(const UnitTest& unit_test, int iteration) override;
03604
03605 void OnTestProgramEnd(const UnitTest& /*unit_test*/) override {
03606 }
03607
03608 private:
03609 static void PrintFailedTests(const UnitTest& unit_test);
03610 static void PrintFailedTestSuites(const UnitTest& unit_test);
03611 static void PrintSkippedTests(const UnitTest& unit_test);
03612 };
03613
03614 // Fired before each iteration of tests starts.
03615
03616 void PrettyUnitTestResultPrinter::OnTestIterationStart(
03617 const UnitTest& unit_test, int iteration) {
03618 if (GTEST_FLAG_GET(repeat) != 1)
03619 printf("\nRepeating all tests (iteration %d) . . .\n\n", iteration + 1);
03620
03621 std::string f = GTEST_FLAG_GET(filter);
03622 const char* const filter = f.c_str();
03623
03624 // Prints the filter if it's not *. This reminds the user that some
03625 // tests may be skipped.
03626 if (!String::CStringEquals(filter, kUniversalFilter)) {
03627 ColoredPrintf(GTestColor::kYellow, "Note: %s filter = %s\n", GTEST_NAME_,
03628 filter);
03629 }
03630
03631 if (internal::ShouldShard(kTestTotalShards, kTestShardIndex, false)) {
03632 const int32_t shard_index = Int32FromEnvOrDie(kTestShardIndex, -1);
03633 ColoredPrintf(GTestColor::kYellow, "Note: This is test shard %d of %s.\n",
03634 static_cast<int>(shard_index) + 1,
03635 internal::posix::GetEnv(kTestTotalShards));
03636 }
03637
03638 if (GTEST_FLAG_GET(shuffle)) {
03639 ColoredPrintf(GTestColor::kYellow,
03640 "Note: Randomizing tests' orders with a seed of %d .\n",
03641 unit_test.random_seed());
03642 }
03643
03644 ColoredPrintf(GTestColor::kGreen, "[=====] ");

```

```

03645 printf("Running %s from %s.\n",
03646 FormatTestCount(unit_test.test_to_run_count()).c_str(),
03647 FormatTestSuiteCount(unit_test.test_suite_to_run_count()).c_str());
03648 fflush(stdout);
03649 }
03650
03651 void PrettyUnitTestResultPrinter::OnEnvironmentsSetUpStart(
03652 const UnitTest& /*unit_test*/) {
03653 ColoredPrintf(GTestColor::kGreen, "[-----] ");
03654 printf("Global test environment set-up.\n");
03655 fflush(stdout);
03656 }
03657
03658 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
03659
03660 void PrettyUnitTestResultPrinter::OnTestCaseStart(const TestCase& test_case) {
03661 const std::string counts =
03662 FormatCountableNoun(test_case.test_to_run_count(), "test", "tests");
03663 ColoredPrintf(GTestColor::kGreen, "[-----] ");
03664 printf("%s from %s", counts.c_str(), test_case.name());
03665 if (test_case.type_param() == nullptr) {
03666 printf("\n");
03667 } else {
03668 printf(", where %s = %s\n", kTypeParamLabel, test_case.type_param());
03669 }
03670 fflush(stdout);
03671 }
03672 #else
03673
03674 void PrettyUnitTestResultPrinter::OnTestSuiteStart(
03675 const TestSuite& test_suite) {
03676 const std::string counts =
03677 FormatCountableNoun(test_suite.test_to_run_count(), "test", "tests");
03678 ColoredPrintf(GTestColor::kGreen, "[-----] ");
03679 printf("%s from %s", counts.c_str(), test_suite.name());
03680 if (test_suite.type_param() == nullptr) {
03681 printf("\n");
03682 } else {
03683 printf(", where %s = %s\n", kTypeParamLabel, test_suite.type_param());
03684 }
03685 fflush(stdout);
03686 }
03687 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
03688
03689 void PrettyUnitTestResultPrinter::OnTestStart(const TestInfo& test_info) {
03690 ColoredPrintf(GTestColor::kGreen, "[RUN] ");
03691 PrintTestName(test_info.test_suite_name(), test_info.name());
03692 printf("\n");
03693 fflush(stdout);
03694 }
03695
03696 void PrettyUnitTestResultPrinter::OnTestDisabled(const TestInfo& test_info) {
03697 ColoredPrintf(GTestColor::kYellow, "[DISABLED] ");
03698 PrintTestName(test_info.test_suite_name(), test_info.name());
03699 printf("\n");
03700 fflush(stdout);
03701 }
03702
03703 // Called after an assertion failure.
03704
03705 void PrettyUnitTestResultPrinter::OnTestPartResult(
03706 const TestPartResult& result) {
03707 switch (result.type()) {
03708 // If the test part succeeded, we don't need to do anything.
03709 case TestPartResult::kSuccess:
03710 return;
03711 default:
03712 // Print failure message from the assertion
03713 // (e.g. expected this and got that).
03714 PrintTestPartResult(result);
03715 fflush(stdout);
03716 }
03717 }
03718
03719 void PrettyUnitTestResultPrinter::OnTestEnd(const TestInfo& test_info) {
03720 if (test_info.result()->Passed()) {
03721 ColoredPrintf(GTestColor::kGreen, "[OK] ");
03722 } else if (test_info.result()->Skipped()) {
03723 ColoredPrintf(GTestColor::kGreen, "[SKIPPED] ");
03724 } else {
03725 ColoredPrintf(GTestColor::kRed, "[FAILED] ");
03726 }
03727 PrintTestName(test_info.test_suite_name(), test_info.name());
03728 if (test_info.result()->Failed()) PrintFullTestCommentIfPresent(test_info);
03729
03730 if (GTEST_FLAG_GET(print_time)) {
03731 printf(" (%s ms)\n",

```

```

03732 internal::StreamableToString(test_info.result()->elapsed_time())
03733 .c_str());
03734 } else {
03735 printf("\n");
03736 }
03737 fflush(stdout);
03738 }
03739
03740 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
03741
03742 void PrettyUnitTestResultPrinter::OnTestCaseEnd(const TestCase& test_case) {
03743 if (!GTEST_FLAG_GET(print_time)) return;
03744
03745 const std::string counts =
03746 FormatCountableNoun(test_case.test_to_run_count(), "test", "tests");
03747 ColoredPrintf(GTestColor::kGreen, "[-----] ");
03748 printf("%s from %s (%s ms total)\n\n", counts.c_str(), test_case.name(),
03749 internal::StreamableToString(test_case.elapsed_time()).c_str());
03750 fflush(stdout);
03751 }
03752 #else
03753
03754 void PrettyUnitTestResultPrinter::OnTestSuiteEnd(const TestSuite& test_suite) {
03755 if (!GTEST_FLAG_GET(print_time)) return;
03756
03757 const std::string counts =
03758 FormatCountableNoun(test_suite.test_to_run_count(), "test", "tests");
03759 ColoredPrintf(GTestColor::kGreen, "[-----] ");
03760 printf("%s from %s (%s ms total)\n\n", counts.c_str(), test_suite.name(),
03761 internal::StreamableToString(test_suite.elapsed_time()).c_str());
03762 fflush(stdout);
03763 }
03764 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
03765
03766 void PrettyUnitTestResultPrinter::OnEnvironmentsTearDownStart(
03767 const UnitTest& /*unit_test*/) {
03768 ColoredPrintf(GTestColor::kGreen, "[-----] ");
03769 printf("Global test environment tear-down\n");
03770 fflush(stdout);
03771 }
03772
03773 // Internal helper for printing the list of failed tests.
03774
03775 void PrettyUnitTestResultPrinter::PrintFailedTests(const UnitTest& unit_test) {
03776 const int failed_test_count = unit_test.failed_test_count();
03777 ColoredPrintf(GTestColor::kRed, "[FAILED] ");
03778 printf("%s, listed below:\n", FormatTestCount(failed_test_count).c_str());
03779
03780 for (int i = 0; i < unit_test.total_test_suite_count(); ++i) {
03781 const TestSuite& test_suite = *unit_test.GetTestSuite(i);
03782 if (!test_suite.should_run() || (test_suite.failed_test_count() == 0)) {
03783 continue;
03784 }
03785 for (int j = 0; j < test_suite.total_test_count(); ++j) {
03786 const TestInfo& test_info = *test_suite.GetTestInfo(j);
03787 if (!test_info.should_run() || !test_info.result()->Failed()) {
03788 continue;
03789 }
03790 ColoredPrintf(GTestColor::kRed, "[FAILED] ");
03791 printf("%s.%s", test_suite.name(), test_info.name());
03792 PrintFullTestCommentIfPresent(test_info);
03793 printf("\n");
03794 }
03795 }
03796 printf("\n%2d FAILED %s\n", failed_test_count,
03797 failed_test_count == 1 ? "TEST" : "TESTS");
03798 }
03799
03800 // Internal helper for printing the list of test suite failures not covered by
03801 // PrintFailedTests.
03802
03803 void PrettyUnitTestResultPrinter::PrintFailedTestSuites(
03804 const UnitTest& unit_test) {
03805 int suite_failure_count = 0;
03806 for (int i = 0; i < unit_test.total_test_suite_count(); ++i) {
03807 const TestSuite& test_suite = *unit_test.GetTestSuite(i);
03808 if (!test_suite.should_run()) {
03809 continue;
03810 }
03811 if (test_suite.ad_hoc_test_result().Failed()) {
03812 ColoredPrintf(GTestColor::kRed, "[FAILED] ");
03813 printf("%s: SetUpTestSuite or TearDownTestSuite\n", test_suite.name());
03814 ++suite_failure_count;
03815 }
03816 }
03817 if (suite_failure_count > 0) {
03818 printf("\n%2d FAILED TEST %s\n", suite_failure_count,

```

```

03819 suite_failure_count == 1 ? "SUITE" : "SUITES");
03820 }
03821 }
03822
03823 // Internal helper for printing the list of skipped tests.
03824
03825 void PrettyUnitTestResultPrinter::PrintSkippedTests(const UnitTest& unit_test) {
03826 const int skipped_test_count = unit_test.skipped_test_count();
03827 if (skipped_test_count == 0) {
03828 return;
03829 }
03830
03831 for (int i = 0; i < unit_test.total_test_suite_count(); ++i) {
03832 const TestSuite& test_suite = *unit_test.GetTestSuite(i);
03833 if (!test_suite.should_run() || (test_suite.skipped_test_count() == 0)) {
03834 continue;
03835 }
03836 for (int j = 0; j < test_suite.total_test_count(); ++j) {
03837 const TestInfo& test_info = *test_suite.GetTestInfo(j);
03838 if (!test_info.should_run() || !test_info.result() -> Skipped()) {
03839 continue;
03840 }
03841 ColoredPrintf(GTestColor::kGreen, "[SKIPPED] ");
03842 printf("%s.%s", test_suite.name(), test_info.name());
03843 printf("\n");
03844 }
03845 }
03846 }
03847
03848 void PrettyUnitTestResultPrinter::OnTestIterationEnd(const UnitTest& unit_test,
03849 int /*iteration*/) {
03850 ColoredPrintf(GTestColor::kGreen, "[=====] ");
03851 printf("%s from %s ran.",
03852 FormatTestCount(unit_test.test_to_run_count()).c_str(),
03853 FormatTestSuiteCount(unit_test.test_suite_to_run_count()).c_str());
03854 if (GTEST_FLAG_GET(print_time)) {
03855 printf(" (%s ms total)",
03856 internal::StreamableToString(unit_test.elapsed_time()).c_str());
03857 }
03858 printf("\n");
03859 ColoredPrintf(GTestColor::kGreen, "[PASSED] ");
03860 printf("%s.\n", FormatTestCount(unit_test.successful_test_count()).c_str());
03861
03862 const int skipped_test_count = unit_test.skipped_test_count();
03863 if (skipped_test_count > 0) {
03864 ColoredPrintf(GTestColor::kGreen, "[SKIPPED] ");
03865 printf("%s, listed below:\n", FormatTestCount(skipped_test_count).c_str());
03866 PrintSkippedTests(unit_test);
03867 }
03868
03869 if (!unit_test.Passed()) {
03870 PrintFailedTests(unit_test);
03871 PrintFailedTestSuites(unit_test);
03872 }
03873
03874 int num_disabled = unit_test.reportable_disabled_test_count();
03875 if (num_disabled && !GTEST_FLAG_GET(also_run_disabled_tests)) {
03876 if (unit_test.Passed()) {
03877 printf("\n"); // Add a spacer if no FAILURE banner is displayed.
03878 }
03879 ColoredPrintf(GTestColor::kYellow, " YOU HAVE %d DISABLED %s\n\n",
03880 num_disabled, num_disabled == 1 ? "TEST" : "TESTS");
03881 }
03882 // Ensure that Google Test output is printed before, e.g., heapchecker output.
03883 fflush(stdout);
03884 }
03885
03886 // End PrettyUnitTestResultPrinter
03887
03888 // This class implements the TestEventListener interface.
03889 //
03890 // Class BriefUnitTestResultPrinter is copyable.
03891
03892 class BriefUnitTestResultPrinter : public TestEventListener {
03893 public:
03894 BriefUnitTestResultPrinter() {
03895 }
03896
03897 static void PrintTestName(const char* test_suite, const char* test) {
03898 printf("%s.%s", test_suite, test);
03899 }
03900
03901 // The following methods override what's in the TestEventListener class.
03902
03903 void OnTestProgramStart(const UnitTest& /*unit_test*/) override {
03904 }
03905 }

```

```

03906
03907 void OnTestIterationStart(const UnitTest& /*unit_test*/,
03908 int /*iteration*/) override {
03909 }
03910
03911 void OnEnvironmentsSetUpStart(const UnitTest& /*unit_test*/) override {
03912 }
03913
03914 void OnEnvironmentsSetUpEnd(const UnitTest& /*unit_test*/) override {
03915 }
03916 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
03917
03918 void OnTestCaseStart(const TestCase& /*test_case*/) override {
03919 }
03920 #else
03921
03922 void OnTestSuiteStart(const TestSuite& /*test_suite*/) override {
03923 }
03924 #endif // OnTestCaseStart
03925
03926 void OnTestStart(const TestInfo& /*test_info*/) override {
03927 }
03928
03929 void OnTestDisabled(const TestInfo& /*test_info*/) override {
03930 }
03931
03932 void OnTestPartResult(const TestPartResult& result) override;
03933 void OnTestEnd(const TestInfo& test_info) override;
03934 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
03935
03936 void OnTestCaseEnd(const TestCase& /*test_case*/) override {
03937 }
03938 #else
03939
03940 void OnTestSuiteEnd(const TestSuite& /*test_suite*/) override {
03941 }
03942 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
03943
03944 void OnEnvironmentsTearDownStart(const UnitTest& /*unit_test*/) override {
03945 }
03946
03947 void OnEnvironmentsTearDownEnd(const UnitTest& /*unit_test*/) override {
03948 }
03949 void OnTestIterationEnd(const UnitTest& unit_test, int iteration) override;
03950
03951 void OnTestProgramEnd(const UnitTest& /*unit_test*/) override {
03952 }
03953 };
03954
03955 // Called after an assertion failure.
03956
03957 void BriefUnitTestResultPrinter::OnTestPartResult(
03958 const TestPartResult& result) {
03959 switch (result.type()) {
03960 // If the test part succeeded, we don't need to do anything.
03961 case TestPartResult::kSuccess:
03962 return;
03963 default:
03964 // Print failure message from the assertion
03965 // (e.g. expected this and got that).
03966 PrintTestPartResult(result);
03967 fflush(stdout);
03968 }
03969 }
03970
03971 void BriefUnitTestResultPrinter::OnTestEnd(const TestInfo& test_info) {
03972 if (test_info.result()->Failed()) {
03973 ColoredPrintf(GTestColor::kRed, "[FAILED] ");
03974 PrintTestName(test_info.test_suite_name(), test_info.name());
03975 PrintFullTestCommentIfPresent(test_info);
03976
03977 if (GTEST_FLAG_GET(print_time)) {
03978 printf(" (%s ms)\n",
03979 internal::StreamableToString(test_info.result()->elapsed_time())
03980 .c_str());
03981 } else {
03982 printf("\n");
03983 }
03984 fflush(stdout);
03985 }
03986 }
03987
03988 void BriefUnitTestResultPrinter::OnTestIterationEnd(const UnitTest& unit_test,
03989 int /*iteration*/) {
03990 ColoredPrintf(GTestColor::kGreen, "[=====] ");
03991 printf("%s from %s ran.",
03992 FormatTestCount(unit_test.test_to_run_count()).c_str(),

```

```

03993 FormatTestSuiteCount(unit_test.test_suite_to_run_count()).c_str());
03994 if (GTEST_FLAG_GET(print_time)) {
03995 printf(" (%s ms total)",
03996 internal::StreamableToString(unit_test.elapsed_time()).c_str());
03997 }
03998 printf("\n");
03999 ColoredPrintf(GTestColor::kGreen, "[PASSED] ");
04000 printf("%s.\n", FormatTestCount(unit_test.successful_test_count()).c_str());
04001
04002 const int skipped_test_count = unit_test.skipped_test_count();
04003 if (skipped_test_count > 0) {
04004 ColoredPrintf(GTestColor::kGreen, "[SKIPPED] ");
04005 printf("%s.\n", FormatTestCount(skipped_test_count).c_str());
04006 }
04007
04008 int num_disabled = unit_test.reportable_disabled_test_count();
04009 if (num_disabled && !GTEST_FLAG_GET(also_run_disabled_tests)) {
04010 if (unit_test.Passed()) {
04011 printf("\n"); // Add a spacer if no FAILURE banner is displayed.
04012 }
04013 ColoredPrintf(GTestColor::kYellow, " YOU HAVE %d DISABLED %s\n\n",
04014 num_disabled, num_disabled == 1 ? "TEST" : "TESTS");
04015 }
04016 // Ensure that Google Test output is printed before, e.g., heapchecker output.
04017 fflush(stdout);
04018 }
04019
04020 // End BriefUnitTestResultPrinter
04021
04022 // class TestEventRepeater
04023 //
04024 // This class forwards events to other event listeners.
04025
04026 class TestEventRepeater : public TestEventListener {
04027 public:
04028 TestEventRepeater() : forwarding_enabled_(true) {
04029 }
04030 ~TestEventRepeater() override;
04031 void Append(TestEventListener* listener);
04032 TestEventListener* Release(TestEventListener* listener);
04033
04034 // Controls whether events will be forwarded to listeners_. Set to false
04035 // in death test child processes.
04036 bool forwarding_enabled() const {
04037 return forwarding_enabled_;
04038 }
04039
04040 void set_forwarding_enabled(bool enable) {
04041 forwarding_enabled_ = enable;
04042 }
04043
04044 void OnTestProgramStart(const UnitTest& unit_test) override;
04045 void OnTestIterationStart(const UnitTest& unit_test, int iteration) override;
04046 void OnEnvironmentsSetUpStart(const UnitTest& unit_test) override;
04047 void OnEnvironmentsSetUpEnd(const UnitTest& unit_test) override;
04048 // Legacy API is deprecated but still available
04049 void OnTestSuiteStart(const TestSuite& parameter) override;
04050 void OnTestStart(const TestInfo& test_info) override;
04051 void OnTestDisabled(const TestInfo& test_info) override;
04052 void OnTestPartResult(const TestPartResult& result) override;
04053 void OnTestEnd(const TestInfo& test_info) override;
04054 // Legacy API is deprecated but still available
04055 void OnTestCaseStart(const TestCase& parameter) override;
04056 void OnTestSuiteEnd(const TestSuite& parameter) override;
04057 void OnEnvironmentsTearDownStart(const UnitTest& unit_test) override;
04058 void OnEnvironmentsTearDownEnd(const UnitTest& unit_test) override;
04059 void OnTestIterationEnd(const UnitTest& unit_test, int iteration) override;
04060 void OnTestProgramEnd(const UnitTest& unit_test) override;
04061
04062 private:
04063 // Controls whether events will be forwarded to listeners_. Set to false
04064 // in death test child processes.
04065 bool forwarding_enabled_;
04066 // The list of listeners that receive events.
04067 std::vector<TestEventListener*> listeners_;
04068
04069 TestEventRepeater(const TestEventRepeater&) = delete;
04070 TestEventRepeater& operator=(const TestEventRepeater&) = delete;
04071 };
04072
04073
04074
04075
04076
04077
04078
04079

```

```

04080 TestEventRepeater::~TestEventRepeater() {
04081 ForEach(listeners_, Delete<TestEventListener>);
04082 }
04083
04084 void TestEventRepeater::Append(TestEventListener* listener) {
04085 listeners_.push_back(listener);
04086 }
04087
04088 TestEventListener* TestEventRepeater::Release(TestEventListener* listener) {
04089 for (size_t i = 0; i < listeners_.size(); ++i) {
04090 if (listeners_[i] == listener) {
04091 listeners_.erase(listeners_.begin() + static_cast<int>(i));
04092 return listener;
04093 }
04094 }
04095
04096 return nullptr;
04097 }
04098
04099 // Since most methods are very similar, use macros to reduce boilerplate.
04100 // This defines a member that forwards the call to all listeners.
04101 #define GTEST_REPEATER_METHOD_(Name, Type) \
04102 void TestEventRepeater::Name(const Type& parameter) { \
04103 if (forwarding_enabled_) { \
04104 for (size_t i = 0; i < listeners_.size(); i++) { \
04105 listeners_[i]->Name(parameter); \
04106 } \
04107 } \
04108 }
04109 // This defines a member that forwards the call to all listeners in reverse
04110 // order.
04111 #define GTEST_REVERSE_REPEATER_METHOD_(Name, Type) \
04112 void TestEventRepeater::Name(const Type& parameter) { \
04113 if (forwarding_enabled_) { \
04114 for (size_t i = listeners_.size(); i != 0; i--) { \
04115 listeners_[i - 1]->Name(parameter); \
04116 } \
04117 } \
04118 }
04119
04120 GTEST_REPEATER_METHOD_(OnTestProgramStart, UnitTest)
04121 GTEST_REPEATER_METHOD_(OnEnvironmentsSetUpStart, UnitTest)
04122 // Legacy API is deprecated but still available
04123 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
04124 GTEST_REPEATER_METHOD_(OnTestCaseStart, TestSuite)
04125 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
04126 GTEST_REPEATER_METHOD_(OnTestSuiteStart, TestSuite)
04127 GTEST_REPEATER_METHOD_(OnTestStart, TestInfo)
04128 GTEST_REPEATER_METHOD_(OnTestDisabled, TestInfo)
04129 GTEST_REPEATER_METHOD_(OnTestPartResult, TestPartResult)
04130 GTEST_REPEATER_METHOD_(OnEnvironmentsTearDownStart, UnitTest)
04131 GTEST_REVERSE_REPEATER_METHOD_(OnEnvironmentsSetUpEnd, UnitTest)
04132 GTEST_REVERSE_REPEATER_METHOD_(OnEnvironmentsTearDownEnd, UnitTest)
04133 GTEST_REVERSE_REPEATER_METHOD_(OnTestEnd, TestInfo)
04134 // Legacy API is deprecated but still available
04135 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
04136 GTEST_REVERSE_REPEATER_METHOD_(OnTestCaseEnd, TestSuite)
04137 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
04138 GTEST_REVERSE_REPEATER_METHOD_(OnTestSuiteEnd, TestSuite)
04139 GTEST_REVERSE_REPEATER_METHOD_(OnTestProgramEnd, UnitTest)
04140
04141 #undef GTEST_REPEATER_METHOD_
04142 #undef GTEST_REVERSE_REPEATER_METHOD_
04143
04144 void TestEventRepeater::OnTestIterationStart(const UnitTest& unit_test,
04145 int iteration) {
04146 if (forwarding_enabled_) {
04147 for (size_t i = 0; i < listeners_.size(); i++) {
04148 listeners_[i]->OnTestIterationStart(unit_test, iteration);
04149 }
04150 }
04151 }
04152
04153 void TestEventRepeater::OnTestIterationEnd(const UnitTest& unit_test,
04154 int iteration) {
04155 if (forwarding_enabled_) {
04156 for (size_t i = listeners_.size(); i > 0; i--) {
04157 listeners_[i - 1]->OnTestIterationEnd(unit_test, iteration);
04158 }
04159 }
04160 }
04161
04162 // End TestEventRepeater
04163
04164 // This class generates an XML output file.
04165
04166 class XmlUnitTestResultPrinter : public EmptyTestEventListener {

```

```

04167 public:
04168 explicit XmlUnitTestResultPrinter(const char* output_file);
04169
04170 void OnTestIterationEnd(const UnitTest& unit_test, int iteration) override;
04171 void ListTestsMatchingFilter(const std::vector<TestSuite*>& test_suites);
04172
04173 // Prints an XML summary of all unit tests.
04174 static void PrintXmlTestsList(std::ostream* stream,
04175 const std::vector<TestSuite*>& test_suites);
04176
04177 private:
04178 // Is c a whitespace character that is normalized to a space character
04179 // when it appears in an XML attribute value?
04180
04181 static bool IsNormalizableWhitespace(unsigned char c) {
04182 return c == '\t' || c == '\n' || c == '\r';
04183 }
04184
04185 // May c appear in a well-formed XML document?
04186 // https://www.w3.org/TR/REC-xml/#charsets
04187
04188 static bool IsValidXmlCharacter(unsigned char c) {
04189 return IsNormalizableWhitespace(c) || c >= 0x20;
04190 }
04191
04192 // Returns an XML-escaped copy of the input string str. If
04193 // is_attribute is true, the text is meant to appear as an attribute
04194 // value, and normalizable whitespace is preserved by replacing it
04195 // with character references.
04196 static std::string EscapeXml(const std::string& str, bool is_attribute);
04197
04198 // Returns the given string with all characters invalid in XML removed.
04199 static std::string RemoveInvalidXmlCharacters(const std::string& str);
04200
04201 // Convenience wrapper around EscapeXml when str is an attribute value.
04202
04203 static std::string EscapeXmlAttribute(const std::string& str) {
04204 return EscapeXml(str, true);
04205 }
04206
04207 // Convenience wrapper around EscapeXml when str is not an attribute value.
04208
04209 static std::string EscapeXmlText(const char* str) {
04210 return EscapeXml(str, false);
04211 }
04212
04213 // Verifies that the given attribute belongs to the given element and
04214 // streams the attribute as XML.
04215 static void OutputXmlAttribute(std::ostream* stream,
04216 const std::string& element_name,
04217 const std::string& name,
04218 const std::string& value);
04219
04220 // Streams an XML CDATA section, escaping invalid CDATA sequences as needed.
04221 static void OutputXmlCDataSection(std::ostream* stream, const char* data);
04222
04223 // Streams a test suite XML stanza containing the given test result.
04224 //
04225 // Requires: result.Failed()
04226 static void OutputXmlTestSuiteForTestResult(std::ostream* stream,
04227 const TestResult& result);
04228
04229 // Streams an XML representation of a TestResult object.
04230 static void OutputXmlTestResult(std::ostream* stream,
04231 const TestResult& result);
04232
04233 // Streams an XML representation of a TestInfo object.
04234 static void OutputXmlTestInfo(std::ostream* stream,
04235 const char* test_suite_name,
04236 const TestInfo& test_info);
04237
04238 // Prints an XML representation of a TestSuite object
04239 static void PrintXmlTestSuite(std::ostream* stream,
04240 const TestSuite& test_suite);
04241
04242 // Prints an XML summary of unit_test to output stream out.
04243 static void PrintXmlUnitTest(std::ostream* stream,
04244 const UnitTest& unit_test);
04245
04246 // Produces a string representing the test properties in a result as space
04247 // delimited XML attributes based on the property key="value" pairs.
04248 // When the std::string is not empty, it includes a space at the beginning,
04249 // to delimit this attribute from prior attributes.
04250 static std::string TestPropertiesAsXmlAttributes(const TestResult& result);
04251
04252 // Streams an XML representation of the test properties of a TestResult
04253 // object.

```



```

04254 static void OutputXmlTestProperties(std::ostream* stream,
04255 const TestResult& result);
04256
04257 // The output file.
04258 const std::string output_file_;
04259
04260 XmlUnitTestResultPrinter(const XmlUnitTestResultPrinter&) = delete;
04261 XmlUnitTestResultPrinter& operator=(const XmlUnitTestResultPrinter&) = delete;
04262 };
04263
04264 // Creates a new XmlUnitTestResultPrinter.
04265
04266 XmlUnitTestResultPrinter::XmlUnitTestResultPrinter(const char* output_file)
04267 : output_file_(output_file) {
04268 if (output_file_.empty()) {
04269 GTEST_LOG_(FATAL) << "XML output file may not be null";
04270 }
04271 }
04272
04273 // Called after the unit test ends.
04274
04275 void XmlUnitTestResultPrinter::OnTestIterationEnd(const UnitTest& unit_test,
04276 int /*iteration*/) {
04277 FILE* xmlout = OpenFileForWriting(output_file_);
04278 std::stringstream stream;
04279 PrintXmlUnitTest(&stream, unit_test);
04280 fprintf(xmlout, "%s", StringStreamToString(&stream).c_str());
04281 fclose(xmlout);
04282 }
04283
04284 void XmlUnitTestResultPrinter::ListTestsMatchingFilter(
04285 const std::vector<TestSuite*>& test_suites) {
04286 FILE* xmlout = OpenFileForWriting(output_file_);
04287 std::stringstream stream;
04288 PrintXmlTestsList(&stream, test_suites);
04289 fprintf(xmlout, "%s", StringStreamToString(&stream).c_str());
04290 fclose(xmlout);
04291 }
04292
04293 // Returns an XML-escaped copy of the input string str. If is_attribute
04294 // is true, the text is meant to appear as an attribute value, and
04295 // normalizable whitespace is preserved by replacing it with character
04296 // references.
04297 //
04298 // Invalid XML characters in str, if any, are stripped from the output.
04299 // It is expected that most, if not all, of the text processed by this
04300 // module will consist of ordinary English text.
04301 // If this module is ever modified to produce version 1.1 XML output,
04302 // most invalid characters can be retained using character references.
04303
04304 std::string XmlUnitTestResultPrinter::EscapeXml(const std::string& str,
04305 bool is_attribute) {
04306 Message m;
04307
04308 for (size_t i = 0; i < str.size(); ++i) {
04309 const char ch = str[i];
04310 switch (ch) {
04311 case '<':
04312 m << "<";
04313 break;
04314 case '>':
04315 m << ">";
04316 break;
04317 case '&':
04318 m << "&";
04319 break;
04320 case '\\':
04321 if (is_attribute)
04322 m << "'";
04323 else
04324 m << "\\\"";
04325 break;
04326 case '\"':
04327 if (is_attribute)
04328 m << """;
04329 else
04330 m << "\"'";
04331 break;
04332 default:
04333 if (IsValidXmlCharacter(static_cast<unsigned char> (ch))) {
04334 if (is_attribute &&
04335 IsNormalizableWhitespace(static_cast<unsigned char> (ch)))
04336 m << "&#x" << String::FormatByte(static_cast<unsigned char> (ch))
04337 << ";";
04338 else
04339 m << ch;
04340 }

```

```

04341 break;
04342 }
04343 }
04344
04345 return m.GetString();
04346 }
04347
04348 // Returns the given string with all characters invalid in XML removed.
04349 // Currently invalid characters are dropped from the string. An
04350 // alternative is to replace them with certain characters such as . or ?.
04351
04352 std::string XmlUnitTestResultPrinter::RemoveInvalidXmlCharacters(
04353 const std::string& str) {
04354 std::string output;
04355 output.reserve(str.size());
04356 for (std::string::const_iterator it = str.begin(); it != str.end(); ++it)
04357 if (IsValidXmlCharacter(static_cast<unsigned char> (*it)))
04358 output.push_back(*it);
04359
04360 return output;
04361 }
04362
04363 // The following routines generate an XML representation of a UnitTest
04364 // object.
04365 //
04366 // This is how Google Test concepts map to the DTD:
04367 //
04368 // <testsuites name="AllTests"> <-- corresponds to a UnitTest object
04369 // <testsuite name="testcase-name"> <-- corresponds to a TestSuite object
04370 // <testcase name="test-name"> <-- corresponds to a TestInfo object
04371 // <failure message="...">...</failure>
04372 // <failure message="...">...</failure>
04373 // <failure message="...">...</failure>
04374 // <-- individual assertion failures
04375 // </testcase>
04376 // </testsuite>
04377 // </testsuites>
04378
04379 // Formats the given time in milliseconds as seconds.
04380
04381 std::string FormatTimeInMillisAsSeconds(TimeInMillis ms) {
04382 ::std::stringstream ss;
04383 ss << (static_cast<double> (ms) * 1e-3);
04384 return ss.str();
04385 }
04386
04387 static bool PortableLocaltime(time_t seconds, struct tm* out) {
04388 #if defined(_MSC_VER)
04389 return localtime_s(out, &seconds) == 0;
04390 #elif defined(__MINGW32__) || defined(__MINGW64__)
04391 // MINGW <time.h> provides neither localtime_r nor localtime_s, but uses
04392 // Windows' localtime(), which has a thread-local tm buffer.
04393 struct tm* tm_ptr = localtime(&seconds); // NOLINT
04394 if (tm_ptr == nullptr) return false;
04395 *out = *tm_ptr;
04396 return true;
04397 #elif defined(__STDC_LIB_EXT1__)
04398 // Uses localtime_s when available as localtime_r is only available from
04399 // C23 standard.
04400 return localtime_s(&seconds, out) != nullptr;
04401 #else
04402 return localtime_r(&seconds, out) != nullptr;
04403 #endif
04404 }
04405
04406 // Converts the given epoch time in milliseconds to a date string in the ISO
04407 // 8601 format, without the timezone information.
04408
04409 std::string FormatEpochTimeInMillisAsIso8601(TimeInMillis ms) {
04410 struct tm time_struct;
04411 if (!PortableLocaltime(static_cast<time_t> (ms / 1000), &time_struct))
04412 return "";
04413 // YYYY-MM-DDThh:mm:ss.sss
04414 return StreamableToString(time_struct.tm_year + 1900) + "-" +
04415 String::FormatIntWidth2(time_struct.tm_mon + 1) + "-" +
04416 String::FormatIntWidth2(time_struct.tm_mday) + "T" +
04417 String::FormatIntWidth2(time_struct.tm_hour) + ":" +
04418 String::FormatIntWidth2(time_struct.tm_min) + ":" +
04419 String::FormatIntWidth2(time_struct.tm_sec) + "." +
04420 String::FormatIntWidthN(static_cast<int> (ms % 1000), 3);
04421 }
04422
04423 // Streams an XML CDATA section, escaping invalid CDATA sequences as needed.
04424
04425 void XmlUnitTestResultPrinter::OutputXmlCDATASection(::std::ostream* stream,
04426 const char* data) {
04427 const char* segment = data;

```

```

04428 *stream << "<![CDATA[";
04429 for (;;) {
04430 const char* const next_segment = strstr(segment, "]]>");
04431 if (next_segment != nullptr) {
04432 stream->write(segment,
04433 static_cast<std::streamsize>(next_segment - segment));
04434 *stream << "]]>]]>><![CDATA[";
04435 segment = next_segment + strlen("]]>");
04436 } else {
04437 *stream << segment;
04438 break;
04439 }
04440 }
04441 *stream << "]]>";
04442 }
04443
04444 void XmlUnitTestResultPrinter::OutputXmlAttribute(
04445 std::ostream* stream, const std::string& element_name,
04446 const std::string& name, const std::string& value) {
04447 const std::vector<std::string>& allowed_names =
04448 GetReservedOutputAttributesForElement(element_name);
04449
04450 GTEST_CHECK_(std::find(allowed_names.begin(), allowed_names.end(), name) !=
04451 allowed_names.end())
04452 << "Attribute " << name << " is not allowed for element <" << element_name
04453 << ".";
04454
04455 *stream << " " << name << "=" << EscapeXmlAttribute(value) << "\"";
04456 }
04457
04458 // Streams a test suite XML stanza containing the given test result.
04459
04460 void XmlUnitTestResultPrinter::OutputXmlTestSuiteForTestResult(
04461 std::ostream* stream, const TestResult& result) {
04462 // Output the boilerplate for a minimal test suite with one test.
04463 *stream << " <testsuite";
04464 OutputXmlAttribute(stream, "testsuite", "name", "NonTestSuiteFailure");
04465 OutputXmlAttribute(stream, "testsuite", "tests", "1");
04466 OutputXmlAttribute(stream, "testsuite", "failures", "1");
04467 OutputXmlAttribute(stream, "testsuite", "disabled", "0");
04468 OutputXmlAttribute(stream, "testsuite", "skipped", "0");
04469 OutputXmlAttribute(stream, "testsuite", "errors", "0");
04470 OutputXmlAttribute(stream, "testsuite", "time",
04471 FormatTimeInMillisAsSeconds(result.elapsed_time()));
04472 OutputXmlAttribute(
04473 stream, "testsuite", "timestamp",
04474 FormatEpochTimeInMillisAsIso8601(result.start_timestamp()));
04475 *stream << ">";
04476
04477 // Output the boilerplate for a minimal test case with a single test.
04478 *stream << " <testcase";
04479 OutputXmlAttribute(stream, "testcase", "name", "");
04480 OutputXmlAttribute(stream, "testcase", "status", "run");
04481 OutputXmlAttribute(stream, "testcase", "result", "completed");
04482 OutputXmlAttribute(stream, "testcase", "classname", "");
04483 OutputXmlAttribute(stream, "testcase", "time",
04484 FormatTimeInMillisAsSeconds(result.elapsed_time()));
04485 OutputXmlAttribute(
04486 stream, "testcase", "timestamp",
04487 FormatEpochTimeInMillisAsIso8601(result.start_timestamp()));
04488
04489 // Output the actual test result.
04490 OutputXmlTestResult(stream, result);
04491
04492 // Complete the test suite.
04493 *stream << " </testsuite>\n";
04494 }
04495
04496 // Prints an XML representation of a TestInfo object.
04497
04498 void XmlUnitTestResultPrinter::OutputXmlTestInfo(std::ostream* stream,
04499 const char* test_suite_name,
04500 const TestInfo& test_info) {
04501 const TestResult& result = *test_info.result();
04502 const std::string kTestsuite = "testcase";
04503
04504 if (test_info.is_in_another_shard()) {
04505 return;
04506 }
04507
04508 *stream << " <testcase";
04509 OutputXmlAttribute(stream, kTestsuite, "name", test_info.name());
04510
04511 if (test_info.value_param() != nullptr) {
04512 OutputXmlAttribute(stream, kTestsuite, "value_param",
04513 test_info.value_param());
04514 }

```

```

04515 if (test_info.type_param() != nullptr) {
04516 OutputXmlAttribute(stream, kTestsuite, "type_param",
04517 test_info.type_param());
04518 }
04519
04520 OutputXmlAttribute(stream, kTestsuite, "file", test_info.file());
04521 OutputXmlAttribute(stream, kTestsuite, "line",
04522 StreamableToString(test_info.line()));
04523 if (GTEST_FLAG_GET(list_tests)) {
04524 *stream << " />\n";
04525 return;
04526 }
04527
04528 OutputXmlAttribute(stream, kTestsuite, "status",
04529 test_info.should_run() ? "run" : "notrun");
04530 OutputXmlAttribute(stream, kTestsuite, "result",
04531 test_info.should_run()
04532 ? (result.Skipped() ? "skipped" : "completed")
04533 : "suppressed");
04534 OutputXmlAttribute(stream, kTestsuite, "time",
04535 FormatTimeInMillisAsSeconds(result.elapsed_time()));
04536 OutputXmlAttribute(
04537 stream, kTestsuite, "timestamp",
04538 FormatEpochTimeInMillisAsIso8601(result.start_timestamp()));
04539 OutputXmlAttribute(stream, kTestsuite, "classname", test_suite_name);
04540
04541 OutputXmlTestResult(stream, result);
04542 }
04543
04544 void XmlUnitTestResultPrinter::OutputXmlTestResult(::std::ostream* stream,
04545 const TestResult& result) {
04546 int failures = 0;
04547 int skips = 0;
04548 for (int i = 0; i < result.total_part_count(); ++i) {
04549 const TestPartResult& part = result.GetTestPartResult(i);
04550 if (part.failed()) {
04551 if (++failures == 1 && skips == 0) {
04552 *stream << ">\n";
04553 }
04554 const std::string location =
04555 internal::FormatCompilerIndependentFileLocation(part.file_name(),
04556 part.line_number());
04557 const std::string summary = location + "\n" + part.summary();
04558 *stream << " <failure message=\"" << EscapeXmlAttribute(summary)
04559 << "\" type=\"" << " ">";
04560 const std::string detail = location + "\n" + part.message();
04561 OutputXmlCDataSection(stream, RemoveInvalidXmlCharacters(detail).c_str());
04562 *stream << "</failure>\n";
04563 } else if (part.skipped()) {
04564 if (++skips == 1 && failures == 0) {
04565 *stream << ">\n";
04566 }
04567 const std::string location =
04568 internal::FormatCompilerIndependentFileLocation(part.file_name(),
04569 part.line_number());
04570 const std::string summary = location + "\n" + part.summary();
04571 *stream << " <skipped message=\"" << " ">";
04572 const std::string detail = location + "\n" + part.message();
04573 OutputXmlCDataSection(stream, RemoveInvalidXmlCharacters(detail).c_str());
04574 *stream << "</skipped>\n";
04575 }
04576 }
04577 }
04578
04579 if (failures == 0 && skips == 0 && result.test_property_count() == 0) {
04580 *stream << " />\n";
04581 } else {
04582 if (failures == 0 && skips == 0) {
04583 *stream << ">\n";
04584 }
04585 OutputXmlTestProperties(stream, result);
04586 *stream << " </testcase>\n";
04587 }
04588 }
04589
04590 // Prints an XML representation of a TestSuite object
04591
04592 void XmlUnitTestResultPrinter::PrintXmlTestSuite(std::ostream* stream,
04593 const TestSuite& test_suite) {
04594 const std::string kTestsuite = "testsuite";
04595 *stream << " <" << kTestsuite;
04596 OutputXmlAttribute(stream, kTestsuite, "name", test_suite.name());
04597 OutputXmlAttribute(stream, kTestsuite, "tests",
04598 StreamableToString(test_suite.reportable_test_count()));
04599 if (!GTEST_FLAG_GET(list_tests)) {
04600 OutputXmlAttribute(stream, kTestsuite, "failures",
04601 StreamableToString(test_suite.failed_test_count()));
04602 }

```

```

04602 OutputXmlAttribute(
04603 stream, kTestsuite, "disabled",
04604 StreamableToString(test_suite.reportable_disabled_test_count()));
04605 OutputXmlAttribute(stream, kTestsuite, "skipped",
04606 StreamableToString(test_suite.skipped_test_count()));
04607
04608 OutputXmlAttribute(stream, kTestsuite, "errors", "0");
04609
04610 OutputXmlAttribute(stream, kTestsuite, "time",
04611 FormatTimeInMillisAsSeconds(test_suite.elapsed_time()));
04612 OutputXmlAttribute(
04613 stream, kTestsuite, "timestamp",
04614 FormatEpochTimeInMillisAsIso8601(test_suite.start_timestamp()));
04615 *stream << TestPropertiesAsXmlAttributes(test_suite.ad_hoc_test_result());
04616 }
04617 *stream << ">\n";
04618 for (int i = 0; i < test_suite.total_test_count(); ++i) {
04619 if (test_suite.GetTestInfo(i)->is_reportable())
04620 OutputXmlTestInfo(stream, test_suite.name(), *test_suite.GetTestInfo(i));
04621 }
04622 *stream << " </" << kTestsuite << ">\n";
04623 }
04624
04625 // Prints an XML summary of unit_test to output stream out.
04626
04627 void XmlUnitTestResultPrinter::PrintXmlUnitTest(std::ostream* stream,
04628 const UnitTest& unit_test) {
04629 const std::string kTestsuites = "testsuites";
04630
04631 *stream << "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n";
04632 *stream << "<" << kTestsuites;
04633
04634 OutputXmlAttribute(stream, kTestsuites, "tests",
04635 StreamableToString(unit_test.reportable_test_count()));
04636 OutputXmlAttribute(stream, kTestsuites, "failures",
04637 StreamableToString(unit_test.failed_test_count()));
04638 OutputXmlAttribute(
04639 stream, kTestsuites, "disabled",
04640 StreamableToString(unit_test.reportable_disabled_test_count()));
04641 OutputXmlAttribute(stream, kTestsuites, "errors", "0");
04642 OutputXmlAttribute(stream, kTestsuites, "time",
04643 FormatTimeInMillisAsSeconds(unit_test.elapsed_time()));
04644 OutputXmlAttribute(
04645 stream, kTestsuites, "timestamp",
04646 FormatEpochTimeInMillisAsIso8601(unit_test.start_timestamp()));
04647
04648 if (GTEST_FLAG_GET(shuffle)) {
04649 OutputXmlAttribute(stream, kTestsuites, "random_seed",
04650 StreamableToString(unit_test.random_seed()));
04651 }
04652 *stream << TestPropertiesAsXmlAttributes(unit_test.ad_hoc_test_result());
04653
04654 OutputXmlAttribute(stream, kTestsuites, "name", "AllTests");
04655 *stream << ">\n";
04656
04657 for (int i = 0; i < unit_test.total_test_suite_count(); ++i) {
04658 if (unit_test.GetTestSuite(i)->reportable_test_count() > 0)
04659 PrintXmlTestSuite(stream, *unit_test.GetTestSuite(i));
04660 }
04661
04662 // If there was a test failure outside of one of the test suites (like in a
04663 // test environment) include that in the output.
04664 if (unit_test.ad_hoc_test_result().Failed()) {
04665 OutputXmlTestSuiteForTestResult(stream, unit_test.ad_hoc_test_result());
04666 }
04667
04668 *stream << "</" << kTestsuites << ">\n";
04669 }
04670
04671 void XmlUnitTestResultPrinter::PrintXmlTestsList(
04672 std::ostream* stream, const std::vector<TestSuite*> test_suites) {
04673 const std::string kTestsuites = "testsuites";
04674
04675 *stream << "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n";
04676 *stream << "<" << kTestsuites;
04677
04678 int total_tests = 0;
04679 for (auto test_suite : test_suites) {
04680 total_tests += test_suite->total_test_count();
04681 }
04682 OutputXmlAttribute(stream, kTestsuites, "tests",
04683 StreamableToString(total_tests));
04684 OutputXmlAttribute(stream, kTestsuites, "name", "AllTests");
04685 *stream << ">\n";
04686
04687 for (auto test_suite : test_suites) {
04688 PrintXmlTestSuite(stream, *test_suite);
04689 }
04690 }

```

```

04689 }
04690 *stream << "</" << kTestsuites << ">\n";
04691 }
04692
04693 // Produces a string representing the test properties in a result as space
04694 // delimited XML attributes based on the property key="value" pairs.
04695
04696 std::string XmlUnitTestResultPrinter::TestPropertiesAsXmlAttributes(
04697 const TestResult& result) {
04698 Message attributes;
04699 for (int i = 0; i < result.test_property_count(); ++i) {
04700 const TestProperty& property = result.GetTestProperty(i);
04701 attributes << " " << property.key() << "="
04702 << "\"" << EscapeXmlAttribute(property.value()) << "\"";
04703 }
04704 return attributes.GetString();
04705 }
04706
04707 void XmlUnitTestResultPrinter::OutputXmlTestProperties(
04708 std::ostream* stream, const TestResult& result) {
04709 const std::string kProperties = "properties";
04710 const std::string kProperty = "property";
04711
04712 if (result.test_property_count() <= 0) {
04713 return;
04714 }
04715
04716 *stream << " <" << kProperties << ">\n";
04717 for (int i = 0; i < result.test_property_count(); ++i) {
04718 const TestProperty& property = result.GetTestProperty(i);
04719 *stream << " <" << kProperty;
04720 *stream << " name=\"\" << EscapeXmlAttribute(property.key()) << "\"";
04721 *stream << " value=\"\" << EscapeXmlAttribute(property.value()) << "\"";
04722 *stream << "/>\n";
04723 }
04724 *stream << " </" << kProperties << ">\n";
04725 }
04726
04727 // End XmlUnitTestResultPrinter
04728
04729 // This class generates an JSON output file.
04730
04731 class JsonUnitTestResultPrinter : public EmptyTestEventListener {
04732 public:
04733 explicit JsonUnitTestResultPrinter(const char* output_file);
04734
04735 void OnTestIterationEnd(const UnitTest& unit_test, int iteration) override;
04736
04737 // Prints an JSON summary of all unit tests.
04738 static void PrintJsonTestList(std::ostream* stream,
04739 const std::vector<TestSuite*>& test_suites);
04740
04741 private:
04742 // Returns an JSON-escaped copy of the input string str.
04743 static std::string EscapeJson(const std::string& str);
04744
04745 static void OutputJsonKey(std::ostream* stream,
04746 const std::string& element_name,
04747 const std::string& name, const std::string& value,
04748 const std::string& indent, bool comma = true);
04749 static void OutputJsonKey(std::ostream* stream,
04750 const std::string& element_name,
04751 const std::string& name, int value,
04752 const std::string& indent, bool comma = true);
04753
04754 // Streams a test suite JSON stanza containing the given test result.
04755 //
04756 // Requires: result.Failed()
04757 static void OutputJsonTestSuiteForTestResult(std::ostream* stream,
04758 const TestResult& result);
04759
04760 // Streams a JSON representation of a TestResult object.
04761 static void OutputJsonTestResult(std::ostream* stream,
04762 const TestResult& result);
04763
04764 // Streams a JSON representation of a TestInfo object.
04765 static void OutputJsonTestInfo(std::ostream* stream,
04766 const char* test_suite_name,
04767 const TestInfo& test_info);
04768
04769 // Prints a JSON representation of a TestSuite object
04770 static void PrintJsonTestSuite(std::ostream* stream,
04771 const TestSuite& test_suite);
04772
04773 // Prints a JSON summary of unit_test to output stream out.
04774 static void PrintJsonUnitTest(std::ostream* stream,
04775 const UnitTest& unit_test);
04776
04777

```

```

04778
04779 // Produces a string representing the test properties in a result as
04780 // a JSON dictionary.
04781 static std::string TestPropertiesAsJson(const TestResult& result,
04782 const std::string& indent);
04783
04784 // The output file.
04785 const std::string output_file_;
04786
04787 JsonUnitTestResultPrinter(const JsonUnitTestResultPrinter&) = delete;
04788 JsonUnitTestResultPrinter& operator=(const JsonUnitTestResultPrinter&) =
04789 delete;
04790 };
04791
04792 // Creates a new JsonUnitTestResultPrinter.
04793
04794 JsonUnitTestResultPrinter::JsonUnitTestResultPrinter(const char* output_file)
04795 : output_file_(output_file) {
04796 if (output_file_.empty()) {
04797 GTEST_LOG_(FATAL) << "JSON output file may not be null";
04798 }
04799 }
04800
04801 void JsonUnitTestResultPrinter::OnTestIterationEnd(const UnitTest& unit_test,
04802 int /*iteration*/) {
04803 FILE* jsonout = OpenFileForWriting(output_file_);
04804 std::stringstream stream;
04805 PrintJsonUnitTest(&stream, unit_test);
04806 fprintf(jsonout, "%s", StringStreamToString(&stream).c_str());
04807 fclose(jsonout);
04808 }
04809
04810 // Returns an JSON-escaped copy of the input string str.
04811
04812 std::string JsonUnitTestResultPrinter::EscapeJson(const std::string& str) {
04813 Message m;
04814
04815 for (size_t i = 0; i < str.size(); ++i) {
04816 const char ch = str[i];
04817 switch (ch) {
04818 case '\\':
04819 case '"':
04820 case '/':
04821 m << '\\' << ch;
04822 break;
04823 case '\b':
04824 m << "\\b";
04825 break;
04826 case '\t':
04827 m << "\\t";
04828 break;
04829 case '\n':
04830 m << "\\n";
04831 break;
04832 case '\f':
04833 m << "\\f";
04834 break;
04835 case '\r':
04836 m << "\\r";
04837 break;
04838 default:
04839 if (ch < ' ') {
04840 m << "\\u00" << String::FormatByte(static_cast<unsigned char>(ch));
04841 } else {
04842 m << ch;
04843 }
04844 break;
04845 }
04846 }
04847
04848 return m.GetString();
04849 }
04850
04851 // The following routines generate an JSON representation of a UnitTest
04852 // object.
04853
04854 // Formats the given time in milliseconds as seconds.
04855
04856 static std::string FormatTimeInMillisAsDuration(TimeInMillis ms) {
04857 std::stringstream ss;
04858 ss << (static_cast<double>(ms) * 1e-3) << "s";
04859 return ss.str();
04860 }
04861
04862 // Converts the given epoch time in milliseconds to a date string in the
04863 // RFC3339 format, without the timezone information.
04864

```

```

04865 static std::string FormatEpochTimeInMillisAsRFC3339(TimeInMillis ms) {
04866 struct tm time_struct;
04867 if (!PortableLocaltime(static_cast<time_t> (ms / 1000), &time_struct))
04868 return "";
04869 // YYYY-MM-DDThh:mm:ss
04870 return StreamableToString(time_struct.tm_year + 1900) + "-" +
04871 String::FormatIntWidth2(time_struct.tm_mon + 1) + "-" +
04872 String::FormatIntWidth2(time_struct.tm_mday) + "T" +
04873 String::FormatIntWidth2(time_struct.tm_hour) + ":" +
04874 String::FormatIntWidth2(time_struct.tm_min) + ":" +
04875 String::FormatIntWidth2(time_struct.tm_sec) + "Z";
04876 }
04877
04878 static inline std::string Indent(size_t width) {
04879 return std::string(width, ' ');
04880 }
04881
04882 void JsonUnitTestResultPrinter::OutputJsonKey(std::ostream* stream,
04883 const std::string& element_name,
04884 const std::string& name,
04885 const std::string& value,
04886 const std::string& indent,
04887 bool comma) {
04888 const std::vector<std::string>& allowed_names =
04889 GetReservedOutputAttributesForElement(element_name);
04890
04891 GTEST_CHECK_(std::find(allowed_names.begin(), allowed_names.end(), name) !=
04892 allowed_names.end())
04893 << "Key \"" < name < "\" is not allowed for value \"" < element_name
04894 << "\".";
04895
04896 *stream << indent << "\"" < name < "\": \"" < EscapeJson(value) < "\"";
04897 if (comma) *stream << ",\n";
04898 }
04899
04900 void JsonUnitTestResultPrinter::OutputJsonKey(
04901 std::ostream* stream, const std::string& element_name,
04902 const std::string& name, int value, const std::string& indent, bool comma) {
04903 const std::vector<std::string>& allowed_names =
04904 GetReservedOutputAttributesForElement(element_name);
04905
04906 GTEST_CHECK_(std::find(allowed_names.begin(), allowed_names.end(), name) !=
04907 allowed_names.end())
04908 << "Key \"" < name < "\" is not allowed for value \"" < element_name
04909 << "\".";
04910
04911 *stream << indent << "\"" < name < "\": " < StreamableToString(value);
04912 if (comma) *stream << ",\n";
04913 }
04914
04915 // Streams a test suite JSON stanza containing the given test result.
04916
04917 void JsonUnitTestResultPrinter::OutputJsonTestSuiteForTestResult(
04918 ::std::ostream* stream, const TestResult& result) {
04919 // Output the boilerplate for a new test suite.
04920 *stream << Indent(4) << "{\n";
04921 OutputJsonKey(stream, "testsuite", "name", "NonTestSuiteFailure", Indent(6));
04922 OutputJsonKey(stream, "testsuite", "tests", 1, Indent(6));
04923 if (!GTEST_FLAG_GET(list_tests)) {
04924 OutputJsonKey(stream, "testsuite", "failures", 1, Indent(6));
04925 OutputJsonKey(stream, "testsuite", "disabled", 0, Indent(6));
04926 OutputJsonKey(stream, "testsuite", "skipped", 0, Indent(6));
04927 OutputJsonKey(stream, "testsuite", "errors", 0, Indent(6));
04928 OutputJsonKey(stream, "testsuite", "time",
04929 FormatTimeInMillisAsDuration(result.elapsed_time()),
04930 Indent(6));
04931 OutputJsonKey(stream, "testsuite", "timestamp",
04932 FormatEpochTimeInMillisAsRFC3339(result.start_timestamp()),
04933 Indent(6));
04934 }
04935 *stream << Indent(6) << "\"testsuite\": {\n";
04936
04937 // Output the boilerplate for a new test case.
04938 *stream << Indent(8) << "{\n";
04939 OutputJsonKey(stream, "testcase", "name", "", Indent(10));
04940 OutputJsonKey(stream, "testcase", "status", "RUN", Indent(10));
04941 OutputJsonKey(stream, "testcase", "result", "COMPLETED", Indent(10));
04942 OutputJsonKey(stream, "testcase", "timestamp",
04943 FormatEpochTimeInMillisAsRFC3339(result.start_timestamp()),
04944 Indent(10));
04945 OutputJsonKey(stream, "testcase", "time",
04946 FormatTimeInMillisAsDuration(result.elapsed_time()),
04947 Indent(10));
04948 OutputJsonKey(stream, "testcase", "classname", "", Indent(10), false);
04949 *stream << TestPropertiesAsJson(result, Indent(10));
04950
04951 // Output the actual test result.

```



```

04952 OutputJsonTestResult(stream, result);
04953
04954 // Finish the test suite.
04955 *stream << "\n" << Indent(6) << "]\n" << Indent(4) << "}";
04956 }
04957
04958 // Prints a JSON representation of a TestInfo object.
04959
04960 void JsonUnitTestResultPrinter::OutputJsonTestInfo(::std::ostream* stream,
04961 const char* test_suite_name,
04962 const TestInfo& test_info) {
04963 const TestResult& result = *test_info.result();
04964 const std::string kTestsuite = "testcase";
04965 const std::string kIndent = Indent(10);
04966
04967 *stream << Indent(8) << "{\n";
04968 OutputJsonKey(stream, kTestsuite, "name", test_info.name(), kIndent);
04969
04970 if (test_info.value_param() != nullptr) {
04971 OutputJsonKey(stream, kTestsuite, "value_param", test_info.value_param(),
04972 kIndent);
04973 }
04974 if (test_info.type_param() != nullptr) {
04975 OutputJsonKey(stream, kTestsuite, "type_param", test_info.type_param(),
04976 kIndent);
04977 }
04978
04979 OutputJsonKey(stream, kTestsuite, "file", test_info.file(), kIndent);
04980 OutputJsonKey(stream, kTestsuite, "line", test_info.line(), kIndent, false);
04981 if (GTEST_FLAG_GET(list_tests)) {
04982 *stream << "\n" << Indent(8) << "}";
04983 return;
04984 } else {
04985 *stream << ",\n";
04986 }
04987
04988 OutputJsonKey(stream, kTestsuite, "status",
04989 test_info.should_run() ? "RUN" : "NOTRUN", kIndent);
04990 OutputJsonKey(stream, kTestsuite, "result",
04991 test_info.should_run()
04992 ? (result.Skipped() ? "SKIPPED" : "COMPLETED")
04993 : "SUPPRESSED",
04994 kIndent);
04995 OutputJsonKey(stream, kTestsuite, "timestamp",
04996 FormatEpochTimeInMillisAsRFC3339(result.start_timestamp()),
04997 kIndent);
04998 OutputJsonKey(stream, kTestsuite, "time",
04999 FormatTimeInMillisAsDuration(result.elapsed_time()), kIndent);
05000 OutputJsonKey(stream, kTestsuite, "classname", test_suite_name, kIndent,
05001 false);
05002 *stream << TestPropertiesAsJson(result, kIndent);
05003
05004 OutputJsonTestResult(stream, result);
05005 }
05006
05007 void JsonUnitTestResultPrinter::OutputJsonTestResult(::std::ostream* stream,
05008 const TestResult& result) {
05009 const std::string kIndent = Indent(10);
05010
05011 int failures = 0;
05012 for (int i = 0; i < result.total_part_count(); ++i) {
05013 const TestPartResult& part = result.GetTestPartResult(i);
05014 if (part.failed()) {
05015 *stream << ",\n";
05016 if (++failures == 1) {
05017 *stream << kIndent << "\n"
05018 << "failures"
05019 << "\n": [\n";
05020 }
05021 const std::string location =
05022 internal::FormatCompilerIndependentFileLocation(part.file_name(),
05023 part.line_number());
05024 const std::string message = EscapeJson(location + "\n" + part.message());
05025 *stream << kIndent << " {\n"
05026 << kIndent << " \"failure\": \"\n\" << message << "\",\n"
05027 << kIndent << " \"type\": \"\n\"
05028 << kIndent << " }";
05029 }
05030 }
05031
05032 if (failures > 0) *stream << "\n" << kIndent << "]";
05033 *stream << "\n" << Indent(8) << "}";
05034 }
05035
05036 // Prints an JSON representation of a TestSuite object
05037
05038 void JsonUnitTestResultPrinter::PrintJsonTestSuite(

```

```

05039 std::ostream* stream, const TestSuite& test_suite) {
05040 const std::string kTestsuite = "testsuite";
05041 const std::string kIndent = Indent(6);
05042
05043 *stream << Indent(4) << "{\n";
05044 OutputJsonKey(stream, kTestsuite, "name", test_suite.name(), kIndent);
05045 OutputJsonKey(stream, kTestsuite, "tests", test_suite.reportable_test_count(),
05046 kIndent);
05047 if (!GTEST_FLAG_GET(list_tests)) {
05048 OutputJsonKey(stream, kTestsuite, "failures",
05049 test_suite.failed_test_count(), kIndent);
05050 OutputJsonKey(stream, kTestsuite, "disabled",
05051 test_suite.reportable_disabled_test_count(), kIndent);
05052 OutputJsonKey(stream, kTestsuite, "errors", 0, kIndent);
05053 OutputJsonKey(
05054 stream, kTestsuite, "timestamp",
05055 FormatEpochTimeInMillisAsRFC3339(test_suite.start_timestamp()),
05056 kIndent);
05057 OutputJsonKey(stream, kTestsuite, "time",
05058 FormatTimeInMillisAsDuration(test_suite.elapsed_time()),
05059 kIndent, false);
05060 *stream << TestPropertiesAsJson(test_suite.ad_hoc_test_result(), kIndent)
05061 << ",\n";
05062 }
05063
05064 *stream << kIndent << "\"" << kTestsuite << "\": {\n";
05065
05066 bool comma = false;
05067 for (int i = 0; i < test_suite.total_test_count(); ++i) {
05068 if (test_suite.GetTestInfo(i)->is_reportable()) {
05069 if (comma) {
05070 *stream << ",\n";
05071 } else {
05072 comma = true;
05073 }
05074 OutputJsonTestInfo(stream, test_suite.name(), *test_suite.GetTestInfo(i));
05075 }
05076 }
05077 *stream << "\n" << kIndent << "]\n" << Indent(4) << "}";
05078 }
05079
05080 // Prints a JSON summary of unit_test to output stream out.
05081
05082 void JsonUnitTestResultPrinter::PrintJsonUnitTest(std::ostream* stream,
05083 const UnitTest& unit_test) {
05084 const std::string kTestsuites = "testsuites";
05085 const std::string kIndent = Indent(2);
05086 *stream << "{\n";
05087
05088 OutputJsonKey(stream, kTestsuites, "tests", unit_test.reportable_test_count(),
05089 kIndent);
05090 OutputJsonKey(stream, kTestsuites, "failures", unit_test.failed_test_count(),
05091 kIndent);
05092 OutputJsonKey(stream, kTestsuites, "disabled",
05093 unit_test.reportable_disabled_test_count(), kIndent);
05094 OutputJsonKey(stream, kTestsuites, "errors", 0, kIndent);
05095 if (GTEST_FLAG_GET(shuffle)) {
05096 OutputJsonKey(stream, kTestsuites, "random_seed", unit_test.random_seed(),
05097 kIndent);
05098 }
05099 OutputJsonKey(stream, kTestsuites, "timestamp",
05100 FormatEpochTimeInMillisAsRFC3339(unit_test.start_timestamp()),
05101 kIndent);
05102 OutputJsonKey(stream, kTestsuites, "time",
05103 FormatTimeInMillisAsDuration(unit_test.elapsed_time()), kIndent,
05104 false);
05105
05106 *stream << TestPropertiesAsJson(unit_test.ad_hoc_test_result(), kIndent)
05107 << ",\n";
05108
05109 OutputJsonKey(stream, kTestsuites, "name", "AllTests", kIndent);
05110 *stream << kIndent << "\"" << kTestsuites << "\": {\n";
05111
05112 bool comma = false;
05113 for (int i = 0; i < unit_test.total_test_suite_count(); ++i) {
05114 if (unit_test.GetTestSuite(i)->reportable_test_count() > 0) {
05115 if (comma) {
05116 *stream << ",\n";
05117 } else {
05118 comma = true;
05119 }
05120 PrintJsonTestSuite(stream, *unit_test.GetTestSuite(i));
05121 }
05122 }
05123
05124 // If there was a test failure outside of one of the test suites (like in a
05125 // test environment) include that in the output.

```

```

05126 if (unit_test.ad_hoc_test_result().Failed()) {
05127 OutputJsonTestSuiteForTestResult(stream, unit_test.ad_hoc_test_result());
05128 }
05129
05130 *stream << "\n"
05131 << kIndent << "]\n"
05132 << "]\n";
05133 }
05134
05135 void JsonUnitTestResultPrinter::PrintJsonTestList(
05136 std::ostream* stream, const std::vector<TestSuite*>& test_suites) {
05137 const std::string kTestsuites = "testsuites";
05138 const std::string kIndent = Indent(2);
05139 *stream << "\n";
05140 int total_tests = 0;
05141 for (auto test_suite : test_suites) {
05142 total_tests += test_suite->total_test_count();
05143 }
05144 OutputJsonKey(stream, kTestsuites, "tests", total_tests, kIndent);
05145
05146 OutputJsonKey(stream, kTestsuites, "name", "AllTests", kIndent);
05147 *stream << kIndent << "\"\" < kTestsuites << "\": [\n";
05148
05149 for (size_t i = 0; i < test_suites.size(); ++i) {
05150 if (i != 0) {
05151 *stream << ",\n";
05152 }
05153 PrintJsonTestSuite(stream, *test_suites[i]);
05154 }
05155
05156 *stream << "\n"
05157 << kIndent << "]\n"
05158 << "]\n";
05159 }
05160 // Produces a string representing the test properties in a result as
05161 // a JSON dictionary.
05162
05163 std::string JsonUnitTestResultPrinter::TestPropertiesAsJson(
05164 const TestResult& result, const std::string& indent) {
05165 Message attributes;
05166 for (int i = 0; i < result.test_property_count(); ++i) {
05167 const TestProperty& property = result.GetTestProperty(i);
05168 attributes << ",\n"
05169 << indent << "\"\" < property.key() << "\": "
05170 << "\"\" < EscapeJson(property.value()) << "\"\"";
05171 }
05172 return attributes.GetString();
05173 }
05174
05175 // End JsonUnitTestResultPrinter
05176
05177 #if GTEST_CAN_STREAM_RESULTS_
05178
05179 // Checks if str contains '=', '&', '%' or '\n' characters. If yes,
05180 // replaces them by "%xx" where xx is their hexadecimal value. For
05181 // example, replaces "=" with "%3D". This algorithm is O(strlen(str))
05182 // in both time and space -- important as the input str may contain an
05183 // arbitrarily long test failure message and stack trace.
05184
05185 std::string StreamingListener::UrlEncode(const char* str) {
05186 std::string result;
05187 result.reserve(strlen(str) + 1);
05188 for (char ch = *str; ch != '\0'; ch = ++str) {
05189 switch (ch) {
05190 case '%':
05191 case '=':
05192 case '&':
05193 case '\n':
05194 result.append("%" + String::FormatByte(static_cast<unsigned char>(ch)));
05195 break;
05196 default:
05197 result.push_back(ch);
05198 break;
05199 }
05200 }
05201 return result;
05202 }
05203
05204 void StreamingListener::SocketWriter::MakeConnection() {
05205 GTEST_CHECK_(sockfd_ == -1)
05206 << "MakeConnection() can't be called when there is already a connection.";
05207
05208 addrinfo hints;
05209 memset(&hints, 0, sizeof(hints));
05210 hints.ai_family = AF_UNSPEC; // To allow both IPv4 and IPv6 addresses.
05211 hints.ai_socktype = SOCK_STREAM;
05212 addrinfo* servinfo = nullptr;

```

```

05213
05214 // Use the getaddrinfo() to get a linked list of IP addresses for
05215 // the given host name.
05216 const int error_num =
05217 getaddrinfo(host_name_.c_str(), port_num_.c_str(), &hints, &servinfo);
05218 if (error_num != 0) {
05219 GTEST_LOG(WARNING) << "stream_result_to: getaddrinfo() failed: "
05220 << gai_strerror(error_num);
05221 }
05222
05223 // Loop through all the results and connect to the first we can.
05224 for (addrinfo* cur_addr = servinfo; sockfd_ == -1 && cur_addr != nullptr;
05225 cur_addr = cur_addr->ai_next) {
05226 sockfd_ = socket(cur_addr->ai_family, cur_addr->ai_socktype,
05227 cur_addr->ai_protocol);
05228 if (sockfd_ != -1) {
05229 // Connect the client socket to the server socket.
05230 if (connect(sockfd_, cur_addr->ai_addr, cur_addr->ai_addrlen) == -1) {
05231 close(sockfd_);
05232 sockfd_ = -1;
05233 }
05234 }
05235 }
05236
05237 freeaddrinfo(servinfo); // all done with this structure
05238
05239 if (sockfd_ == -1) {
05240 GTEST_LOG(WARNING) << "stream_result_to: failed to connect to "
05241 << host_name_ << ":" << port_num_;
05242 }
05243 }
05244
05245 // End of class Streaming Listener
05246 #endif // GTEST_CAN_STREAM_RESULTS__
05247
05248 // class OsStackTraceGetter
05249
05250 const char* const OsStackTraceGetterInterface::kElidedFramesMarker =
05251 "... " GTEST_NAME_ " internal frames ...";
05252
05253 std::string OsStackTraceGetter::CurrentStackTrace(int max_depth, int skip_count)
05254 GTEST_LOCK_EXCLUDED_(mutex_) {
05255 #if GTEST_HAS_ABSL
05256 std::string result;
05257
05258 if (max_depth <= 0) {
05259 return result;
05260 }
05261
05262 max_depth = std::min(max_depth, kMaxStackTraceDepth);
05263
05264 std::vector<void*> raw_stack(max_depth);
05265 // Skips the frames requested by the caller, plus this function.
05266 const int raw_stack_size =
05267 absl::GetStackTrace(&raw_stack[0], max_depth, skip_count + 1);
05268
05269 void* caller_frame = nullptr;
05270 {
05271 MutexLock lock(&mutex_);
05272 caller_frame = caller_frame_;
05273 }
05274
05275 for (int i = 0; i < raw_stack_size; ++i) {
05276 if (raw_stack[i] == caller_frame &&
05277 !GTEST_FLAG_GET(show_internal_stack_frames)) {
05278 // Add a marker to the trace and stop adding frames.
05279 absl::StrAppend(&result, kElidedFramesMarker, "\n");
05280 break;
05281 }
05282
05283 char tmp[1024];
05284 const char* symbol = "(unknown)";
05285 if (absl::Symbolize(raw_stack[i], tmp, sizeof(tmp))) {
05286 symbol = tmp;
05287 }
05288
05289 char line[1024];
05290 snprintf(line, sizeof(line), " %p: %s\n", raw_stack[i], symbol);
05291 result += line;
05292 }
05293
05294 return result;
05295 #else // !GTEST_HAS_ABSL
05296 // static_cast<void*> (max_depth);
05297 static_cast<void*> (skip_count);
05298 return "";
05299 #endif

```

```

05300 #endif // GTEST_HAS_ABSL
05301 }
05302
05303 void OsStackTraceGetter::UponLeavingGTest() GTEST_LOCK_EXCLUDED_(mutex_) {
05304 #if GTEST_HAS_ABSL
05305 void* caller_frame = nullptr;
05306 if (absl::GetStackTrace(&caller_frame, 1, 3) <= 0) {
05307 caller_frame = nullptr;
05308 }
05309
05310 MutexLock lock(&mutex_);
05311 caller_frame_ = caller_frame;
05312 #endif // GTEST_HAS_ABSL
05313 }
05314
05315 // A helper class that creates the premature-exit file in its
05316 // constructor and deletes the file in its destructor.
05317
05318 class ScopedPrematureExitFile {
05319 public:
05320
05321 explicit ScopedPrematureExitFile(const char* premature_exit_filepath)
05322 : premature_exit_filepath_(
05323 premature_exit_filepath ? premature_exit_filepath : "") {
05324 // If a path to the premature-exit file is specified...
05325 if (!premature_exit_filepath_.empty()) {
05326 // create the file with a single "0" character in it. I/O
05327 // errors are ignored as there's nothing better we can do and we
05328 // don't want to fail the test because of this.
05329 FILE* pfile = posix::FOpen(premature_exit_filepath_.c_str(), "w");
05330 fwrite("0", 1, 1, pfile);
05331 fclose(pfile);
05332 }
05333 }
05334
05335 ~ScopedPrematureExitFile() {
05336 #if !defined GTEST_OS_ESP8266
05337 if (!premature_exit_filepath_.empty()) {
05338 int retval = remove(premature_exit_filepath_.c_str());
05339 if (retval) {
05340 GTEST_LOG_(ERROR) << "Failed to remove premature exit filepath \""
05341 << premature_exit_filepath_ << "\" with error "
05342 << retval;
05343 }
05344 }
05345 #endif
05346 }
05347
05348 private:
05349 const std::string premature_exit_filepath_;
05350
05351 ScopedPrematureExitFile(const ScopedPrematureExitFile&) = delete;
05352 ScopedPrematureExitFile& operator=(const ScopedPrematureExitFile&) = delete;
05353 };
05354
05355 } // namespace internal
05356
05357 // class TestEventListeners
05358
05359 TestEventListeners::TestEventListeners()
05360 : repeater_(new internal::TestEventRepeater()),
05361 default_result_printer_(nullptr),
05362 default_xml_generator_(nullptr) {}
05363
05364 TestEventListeners::~TestEventListeners() {
05365 delete repeater_;
05366 }
05367
05368 // Returns the standard listener responsible for the default console
05369 // output. Can be removed from the listeners list to shut down default
05370 // console output. Note that removing this object from the listener list
05371 // with Release transfers its ownership to the user.
05372
05373 void TestEventListeners::Append(TestEventListener* listener) {
05374 repeater_->Append(listener);
05375 }
05376
05377 // Removes the given event listener from the list and returns it. It then
05378 // becomes the caller's responsibility to delete the listener. Returns
05379 // NULL if the listener is not found in the list.
05380
05381 TestEventListener* TestEventListeners::Release(TestEventListener* listener) {
05382 if (listener == default_result_printer_)
05383 default_result_printer_ = nullptr;
05384 else if (listener == default_xml_generator_)
05385 default_xml_generator_ = nullptr;
05386 }

```

```

05387 return repeater_>Release(listener);
05388 }
05389
05390 // Returns repeater that broadcasts the TestEventListener events to all
05391 // subscribers.
05392
05393 TestEventListener* TestEventListeners::repeater() {
05394 return repeater_;
05395 }
05396
05397 // Sets the default_result_printer attribute to the provided listener.
05398 // The listener is also added to the listener list and previous
05399 // default_result_printer is removed from it and deleted. The listener can
05400 // also be NULL in which case it will not be added to the list. Does
05401 // nothing if the previous and the current listener objects are the same.
05402
05403 void TestEventListeners::SetDefaultResultPrinter(TestEventListener* listener) {
05404 if (default_result_printer_ != listener) {
05405 // It is an error to pass this method a listener that is already in the
05406 // list.
05407 delete Release(default_result_printer_);
05408 default_result_printer_ = listener;
05409 if (listener != nullptr) Append(listener);
05410 }
05411 }
05412
05413 // Sets the default_xml_generator attribute to the provided listener. The
05414 // listener is also added to the listener list and previous
05415 // default_xml_generator is removed from it and deleted. The listener can
05416 // also be NULL in which case it will not be added to the list. Does
05417 // nothing if the previous and the current listener objects are the same.
05418
05419 void TestEventListeners::SetDefaultXmlGenerator(TestEventListener* listener) {
05420 if (default_xml_generator_ != listener) {
05421 // It is an error to pass this method a listener that is already in the
05422 // list.
05423 delete Release(default_xml_generator_);
05424 default_xml_generator_ = listener;
05425 if (listener != nullptr) Append(listener);
05426 }
05427 }
05428
05429 // Controls whether events will be forwarded by the repeater to the
05430 // listeners in the list.
05431
05432 bool TestEventListeners::EventForwardingEnabled() const {
05433 return repeater_>forwarding_enabled();
05434 }
05435
05436 void TestEventListeners::SuppressEventForwarding() {
05437 repeater_>set_forwarding_enabled(false);
05438 }
05439
05440 // class UnitTest
05441
05442 // Gets the singleton UnitTest object. The first time this method is
05443 // called, a UnitTest object is constructed and returned. Consecutive
05444 // calls will return the same object.
05445 //
05446 // We don't protect this under mutex_ as a user is not supposed to
05447 // call this before main() starts, from which point on the return
05448 // value will never change.
05449
05450 UnitTest* UnitTest::GetInstance() {
05451 // CodeGear C++Builder insists on a public destructor for the
05452 // default implementation. Use this implementation to keep good OO
05453 // design with private destructor.
05454
05455 #if defined(__BORLANDC__)
05456 static UnitTest* const instance = new UnitTest;
05457 return instance;
05458 #else
05459 static UnitTest instance;
05460 return &instance;
05461 #endif // defined(__BORLANDC__)
05462 }
05463
05464 // Gets the number of successful test suites.
05465
05466 int UnitTest::successful_test_suite_count() const {
05467 return impl()->successful_test_suite_count();
05468 }
05469
05470 // Gets the number of failed test suites.
05471
05472 int UnitTest::failed_test_suite_count() const {
05473 return impl()->failed_test_suite_count();

```

```

05474 }
05475
05476 // Gets the number of all test suites.
05477
05478 int UnitTest::total_test_suite_count() const {
05479 return impl()->total_test_suite_count();
05480 }
05481
05482 // Gets the number of all test suites that contain at least one test
05483 // that should run.
05484
05485 int UnitTest::test_suite_to_run_count() const {
05486 return impl()->test_suite_to_run_count();
05487 }
05488
05489 // Legacy API is deprecated but still available
05490 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
05491
05492 int UnitTest::successful_test_case_count() const {
05493 return impl()->successful_test_suite_count();
05494 }
05495
05496 int UnitTest::failed_test_case_count() const {
05497 return impl()->failed_test_suite_count();
05498 }
05499
05500 int UnitTest::total_test_case_count() const {
05501 return impl()->total_test_suite_count();
05502 }
05503
05504 int UnitTest::test_case_to_run_count() const {
05505 return impl()->test_suite_to_run_count();
05506 }
05507 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
05508
05509 // Gets the number of successful tests.
05510
05511 int UnitTest::successful_test_count() const {
05512 return impl()->successful_test_count();
05513 }
05514
05515 // Gets the number of skipped tests.
05516
05517 int UnitTest::skipped_test_count() const {
05518 return impl()->skipped_test_count();
05519 }
05520
05521 // Gets the number of failed tests.
05522
05523 int UnitTest::failed_test_count() const {
05524 return impl()->failed_test_count();
05525 }
05526
05527 // Gets the number of disabled tests that will be reported in the XML report.
05528
05529 int UnitTest::reportable_disabled_test_count() const {
05530 return impl()->reportable_disabled_test_count();
05531 }
05532
05533 // Gets the number of disabled tests.
05534
05535 int UnitTest::disabled_test_count() const {
05536 return impl()->disabled_test_count();
05537 }
05538
05539 // Gets the number of tests to be printed in the XML report.
05540
05541 int UnitTest::reportable_test_count() const {
05542 return impl()->reportable_test_count();
05543 }
05544
05545 // Gets the number of all tests.
05546
05547 int UnitTest::total_test_count() const {
05548 return impl()->total_test_count();
05549 }
05550
05551 // Gets the number of tests that should run.
05552
05553 int UnitTest::test_to_run_count() const {
05554 return impl()->test_to_run_count();
05555 }
05556
05557 // Gets the time of the test program start, in ms from the start of the
05558 // UNIX epoch.
05559
05560 internal::TimeInMillis UnitTest::start_timestamp() const {

```

```

05561 return impl()->start_timestamp();
05562 }
05563
05564 // Gets the elapsed time, in milliseconds.
05565
05566 internal::TimeInMillis UnitTest::elapsed_time() const {
05567 return impl()->elapsed_time();
05568 }
05569
05570 // Returns true if and only if the unit test passed (i.e. all test suites
05571 // passed).
05572
05573 bool UnitTest::Passed() const {
05574 return impl()->Passed();
05575 }
05576
05577 // Returns true if and only if the unit test failed (i.e. some test suite
05578 // failed or something outside of all tests failed).
05579
05580 bool UnitTest::Failed() const {
05581 return impl()->Failed();
05582 }
05583
05584 // Gets the i-th test suite among all the test suites. i can range from 0 to
05585 // total_test_suite_count() - 1. If i is not in that range, returns NULL.
05586
05587 const TestSuite* UnitTest::GetTestSuite(int i) const {
05588 return impl()->GetTestSuite(i);
05589 }
05590
05591 // Legacy API is deprecated but still available
05592 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
05593
05594 const TestCase* UnitTest::GetTestCase(int i) const {
05595 return impl()->GetTestCase(i);
05596 }
05597 #endif // GTEST_REMOVE_LEGACY_TEST_CASEAPI_
05598
05599 // Returns the TestResult containing information on test failures and
05600 // properties logged outside of individual test suites.
05601
05602 const TestResult& UnitTest::ad_hoc_test_result() const {
05603 return *impl()->ad_hoc_test_result();
05604 }
05605
05606 // Gets the i-th test suite among all the test suites. i can range from 0 to
05607 // total_test_suite_count() - 1. If i is not in that range, returns NULL.
05608
05609 TestSuite* UnitTest::GetMutableTestSuite(int i) {
05610 return impl()->GetMutableSuiteCase(i);
05611 }
05612
05613 // Returns the list of event listeners that can be used to track events
05614 // inside Google Test.
05615
05616 TestEventListeners& UnitTest::listeners() {
05617 return *impl()->listeners();
05618 }
05619
05620 // Registers and returns a global test environment. When a test
05621 // program is run, all global test environments will be set-up in the
05622 // order they were registered. After all tests in the program have
05623 // finished, all global test environments will be torn-down in the
05624 // *reverse* order they were registered.
05625 //
05626 // The UnitTest object takes ownership of the given environment.
05627 //
05628 // We don't protect this under mutex_, as we only support calling it
05629 // from the main thread.
05630
05631 Environment* UnitTest::AddEnvironment(Environment* env) {
05632 if (env == nullptr) {
05633 return nullptr;
05634 }
05635
05636 impl_>environments().push_back(env);
05637 return env;
05638 }
05639
05640 // Adds a TestPartResult to the current TestResult object. All Google Test
05641 // assertion macros (e.g. ASSERT_TRUE, EXPECT_EQ, etc) eventually call
05642 // this to report their results. The user code should use the
05643 // assertion macros instead of calling this directly.
05644
05645 void UnitTest::AddTestPartResult(TestPartResult::Type result_type,
05646 const char* file_name, int line_number,
05647 const std::string& message,

```



```

05648 const std::string& os_stack_trace)
05649 GTEST_LOCK_EXCLUDED_(mutex_) {
05650 Message msg;
05651 msg << message;
05652
05653 internal::MutexLock lock(&mutex_);
05654 if (impl_>gtest_trace_stack().size() > 0) {
05655 msg << "\n" << GTEST_NAME_ << " trace:";
05656
05657 for (size_t i = impl_>gtest_trace_stack().size(); i > 0; --i) {
05658 const internal::TraceInfo& trace = impl_>gtest_trace_stack()[i - 1];
05659 msg << "\n"
05660 << internal::FormatFileLocation(trace.file, trace.line) << " "
05661 << trace.message;
05662 }
05663 }
05664
05665 if (os_stack_trace.c_str() != nullptr && !os_stack_trace.empty()) {
05666 msg << internal::kStackTraceMarker << os_stack_trace;
05667 }
05668
05669 const TestPartResult result = TestPartResult(
05670 result_type, file_name, line_number, msg.GetString().c_str());
05671 impl_>GetTestPartResultReporterForCurrentThread()->ReportTestPartResult(
05672 result);
05673
05674 if (result_type != TestPartResult::kSuccess &&
05675 result_type != TestPartResult::kSkip) {
05676 // gtest_break_on_failure takes precedence over
05677 // gtest_throw_on_failure. This allows a user to set the latter
05678 // in the code (perhaps in order to use Google Test assertions
05679 // with another testing framework) and specify the former on the
05680 // command line for debugging.
05681 if (GTEST_FLAG_GET(break_on_failure)) {
05682 #if GTEST_OS_WINDOWS && !GTEST_OS_WINDOWS_PHONE && !GTEST_OS_WINDOWS_RT
05683 // Using DebugBreak on Windows allows gtest to still break into a debugger
05684 // when a failure happens and both the --gtest_break_on_failure and
05685 // the --gtest_catch_exceptions flags are specified.
05686 DebugBreak();
05687 #elif (!defined(__native_client__)) && \
05688 ((defined(__clang__) || defined(__GNUC__)) && \
05689 (defined(__x86_64__) || defined(__i386__)))
05690 // with clang/gcc we can achieve the same effect on x86 by invoking int3
05691 asm("int3");
05692 #else
05693 // Dereference nullptr through a volatile pointer to prevent the compiler
05694 // from removing. We use this rather than abort() or __builtin_trap() for
05695 // portability: some debuggers don't correctly trap abort().
05696 *static_cast<volatile int*>(nullptr) = 1;
05697 #endif // GTEST_OS_WINDOWS
05698 } else if (GTEST_FLAG_GET(throw_on_failure)) {
05699 #if GTEST_HAS_EXCEPTIONS
05700 throw internal::GoogleTestFailureException(result);
05701 #else
05702 // We cannot call abort() as it generates a pop-up in debug mode
05703 // that cannot be suppressed in VC 7.1 or below.
05704 exit(1);
05705 #endif
05706 }
05707 }
05708 }
05709
05710 // Adds a TestProperty to the current TestResult object when invoked from
05711 // inside a test, to current TestSuite's ad_hoc_test_result_ when invoked
05712 // from SetUpTestSuite or TearDownTestSuite, or to the global property set
05713 // when invoked elsewhere. If the result already contains a property with
05714 // the same key, the value will be updated.
05715
05716 void UnitTest::RecordProperty(const std::string& key,
05717 const std::string& value) {
05718 impl_>RecordProperty(TestProperty(key, value));
05719 }
05720
05721 // Runs all tests in this UnitTest object and prints the result.
05722 // Returns 0 if successful, or 1 otherwise.
05723 //
05724 // We don't protect this under mutex_, as we only support calling it
05725 // from the main thread.
05726
05727 int UnitTest::Run() {
05728 const bool in_death_test_child_process =
05729 GTEST_FLAG_GET(internal_run_death_test).length() > 0;
05730
05731 // Google Test implements this protocol for catching that a test
05732 // program exits before returning control to Google Test:
05733 //
05734 // 1. Upon start, Google Test creates a file whose absolute path

```

```

05735 // is specified by the environment variable
05736 // TEST_PREMATURE_EXIT_FILE.
05737 // 2. When Google Test has finished its work, it deletes the file.
05738 //
05739 // This allows a test runner to set TEST_PREMATURE_EXIT_FILE before
05740 // running a Google-Test-based test program and check the existence
05741 // of the file at the end of the test execution to see if it has
05742 // exited prematurely.
05743
05744 // If we are in the child process of a death test, don't
05745 // create/delete the premature exit file, as doing so is unnecessary
05746 // and will confuse the parent process. Otherwise, create/delete
05747 // the file upon entering/leaving this function. If the program
05748 // somehow exits before this function has a chance to return, the
05749 // premature-exit file will be left undeleted, causing a test runner
05750 // that understands the premature-exit-file protocol to report the
05751 // test as having failed.
05752 const internal::ScopedPrematureExitFile premature_exit_file(
05753 in_death_test_child_process
05754 ? nullptr
05755 : internal::posix::GetEnv("TEST_PREMATURE_EXIT_FILE"));
05756
05757 // Captures the value of GTEST_FLAG(catch_exceptions). This value will be
05758 // used for the duration of the program.
05759 impl()->set_catch_exceptions(GTEST_FLAG_GET(catch_exceptions));
05760
05761 #if GTEST_OS_WINDOWS
05762 // Either the user wants Google Test to catch exceptions thrown by the
05763 // tests or this is executing in the context of death test child
05764 // process. In either case the user does not want to see pop-up dialogs
05765 // about crashes - they are expected.
05766 if (impl()->catch_exceptions() || in_death_test_child_process) {
05767 #if !GTEST_OS_WINDOWS_MOBILE && !GTEST_OS_WINDOWS_PHONE && !GTEST_OS_WINDOWS_RT
05768 // SetErrorMode doesn't exist on CE.
05769 SetErrorMode(SEM_FAILCRITICALERRORS | SEM_NOALIGNMENTFAULTEXCEPT |
05770 SEM_NOGPFAULTERRORBOX | SEM_NOOPENFILEERRORBOX);
05771 #endif // !GTEST_OS_WINDOWS_MOBILE
05772
05773 #if (defined(_MSC_VER) || GTEST_OS_WINDOWS_MINGW) && !GTEST_OS_WINDOWS_MOBILE
05774 // Death test children can be terminated with _abort(). On Windows,
05775 // _abort() can show a dialog with a warning message. This forces the
05776 // abort message to go to stderr instead.
05777 _set_error_mode(_OUT_TO_STDERR);
05778 #endif
05779
05780 #if defined(_MSC_VER) && !GTEST_OS_WINDOWS_MOBILE
05781 // In the debug version, Visual Studio pops up a separate dialog
05782 // offering a choice to debug the aborted program. We need to suppress
05783 // this dialog or it will pop up for every EXPECT/ASSERT_DEATH statement
05784 // executed. Google Test will notify the user of any unexpected
05785 // failure via stderr.
05786 if (!GTEST_FLAG_GET(break_on_failure))
05787 _set_abort_behavior(
05788 0x0, // Clear the following flags:
05789 _WRITE_ABORT_MSG | _CALL_REPORTFAULT); // pop-up window, core dump.
05790
05791 // In debug mode, the Windows CRT can crash with an assertion over invalid
05792 // input (e.g. passing an invalid file descriptor). The default handling
05793 // for these assertions is to pop up a dialog and wait for user input.
05794 // Instead ask the CRT to dump such assertions to stderr non-interactively.
05795 if (!IsDebuggerPresent()) {
05796 (void) _CrtSetReportMode(_CRT_ASSERT,
05797 _CRTDBG_MODE_FILE | _CRTDBG_MODE_DEBUG);
05798 (void) _CrtSetReportFile(_CRT_ASSERT, _CRTDBG_FILE_STDERR);
05799 }
05800 #endif
05801 }
05802 #endif // GTEST_OS_WINDOWS
05803
05804 return internal::HandleExceptionsInMethodIfSupported(
05805 impl(), &internal::UnitTestImpl::RunAllTests,
05806 "auxiliary test code (environments or event listeners)")
05807 ? 0
05808 : 1;
05809 }
05810
05811 // Returns the working directory when the first TEST() or TEST_F() was
05812 // executed.
05813
05814 const char* UnitTest::original_working_dir() const {
05815 return impl()->original_working_dir_.c_str();
05816 }
05817
05818 // Returns the TestSuite object for the test that's currently running,
05819 // or NULL if no test is running.
05820
05821 const TestSuite* UnitTest::current_test_suite() const

```

```

05822 GTEST_LOCK_EXCLUDED_(mutex_) {
05823 internal::MutexLock lock(&mutex_);
05824 return impl_>current_test_suite();
05825 }
05826
05827 // Legacy API is still available but deprecated
05828 #ifndef GTEST_REMOVE_LEGACY_TEST_CASEAPI_
05829
05830 const TestCase* UnitTest::current_test_case() const
05831 GTEST_LOCK_EXCLUDED_(mutex_) {
05832 internal::MutexLock lock(&mutex_);
05833 return impl_>current_test_suite();
05834 }
05835 #endif
05836
05837 // Returns the TestInfo object for the test that's currently running,
05838 // or NULL if no test is running.
05839
05840 const TestInfo* UnitTest::current_test_info() const
05841 GTEST_LOCK_EXCLUDED_(mutex_) {
05842 internal::MutexLock lock(&mutex_);
05843 return impl_>current_test_info();
05844 }
05845
05846 // Returns the random seed used at the start of the current test run.
05847
05848 int UnitTest::random_seed() const {
05849 return impl_>random_seed();
05850 }
05851
05852 // Returns ParameterizedTestSuiteRegistry object used to keep track of
05853 // value-parameterized tests and instantiate and register them.
05854
05855 internal::ParameterizedTestSuiteRegistry&
05856 UnitTest::parameterized_test_registry() GTEST_LOCK_EXCLUDED_(mutex_) {
05857 return impl_>parameterized_test_registry();
05858 }
05859
05860 // Creates an empty UnitTest.
05861
05862 UnitTest::UnitTest() {
05863 impl_ = new internal::UnitTestImpl(this);
05864 }
05865
05866 // Destructor of UnitTest.
05867
05868 UnitTest::~UnitTest() {
05869 delete impl_;
05870 }
05871
05872 // Pushes a trace defined by SCOPED_TRACE() on to the per-thread
05873 // Google Test trace stack.
05874
05875 void UnitTest::PushGTestTrace(const internal::TraceInfo& trace)
05876 GTEST_LOCK_EXCLUDED_(mutex_) {
05877 internal::MutexLock lock(&mutex_);
05878 impl_>gtest_trace_stack().push_back(trace);
05879 }
05880
05881 // Pops a trace from the per-thread Google Test trace stack.
05882
05883 void UnitTest::PopGTestTrace() GTEST_LOCK_EXCLUDED_(mutex_) {
05884 internal::MutexLock lock(&mutex_);
05885 impl_>gtest_trace_stack().pop_back();
05886 }
05887
05888 namespace internal {
05889
05890 UnitTestImpl::UnitTestImpl(UnitTest* parent)
05891 : parent_(parent),
05892 GTEST_DISABLE_MSC_WARNINGS_PUSH_(4355 /* using this in initializer */)
05893 default_global_test_part_result_reporter_(this),
05894 default_per_thread_test_part_result_reporter_(this),
05895 GTEST_DISABLE_MSC_WARNINGS_POP_() global_test_part_result_reporter_(
05896 &default_global_test_part_result_reporter_),
05897 per_thread_test_part_result_reporter_(
05898 &default_per_thread_test_part_result_reporter_),
05899 parameterized_test_registry_(),
05900 parameterized_tests_registered_(false),
05901 last_death_test_suite_(-1),
05902 current_test_suite_(nullptr),
05903 current_test_info_(nullptr),
05904 ad_hoc_test_result_(),
05905 os_stack_trace_getter_(nullptr),
05906 post_flag_parse_init_performed_(false),
05907 random_seed_(0), // Will be overridden by the flag before first use.
05908 random_(0), // Will be reseeded before first use.

```

```

05909 start_timestamp_(0),
05910 elapsed_time_(0),
05911 #if GTEST_HAS_DEATH_TEST
05912 death_test_factory_(new DefaultDeathTestFactory),
05913 #endif
05914 // Will be overridden by the flag before first use.
05915 catch_exceptions_(false) {
05916 listeners()->SetDefaultResultPrinter(new PrettyUnitTestResultPrinter);
05917 }
05918
05919 UnitTestImpl::~UnitTestImpl() {
05920 // Deletes every TestSuite.
05921 ForEach(test_suites_, internal::Delete<TestSuite>);
05922
05923 // Deletes every Environment.
05924 ForEach(environments_, internal::Delete<Environment>);
05925
05926 delete os_stack_trace_getter_;
05927 }
05928
05929 // Adds a TestProperty to the current TestResult object when invoked in a
05930 // context of a test, to current test suite's ad_hoc_test_result when invoke
05931 // from SetUpTestSuite/TearDownTestSuite, or to the global property set
05932 // otherwise. If the result already contains a property with the same key,
05933 // the value will be updated.
05934
05935 void UnitTestImpl::RecordProperty(const TestProperty& test_property) {
05936 std::string xml_element;
05937 TestResult* test_result; // TestResult appropriate for property recording.
05938
05939 if (current_test_info_ != nullptr) {
05940 xml_element = "testcase";
05941 test_result = &(current_test_info_->result_);
05942 } else if (current_test_suite_ != nullptr) {
05943 xml_element = "testsuite";
05944 test_result = &(current_test_suite_->ad_hoc_test_result_);
05945 } else {
05946 xml_element = "testsuites";
05947 test_result = &ad_hoc_test_result_;
05948 }
05949 test_result->RecordProperty(xml_element, test_property);
05950 }
05951
05952 #if GTEST_HAS_DEATH_TEST
05953 // Disables event forwarding if the control is currently in a death test
05954 // subprocess. Must not be called before InitGoogleTest.
05955
05956 void UnitTestImpl::SuppressTestEventsIfInSubprocess() {
05957 if (internal_run_death_test_flag_.get() != nullptr)
05958 listeners()->SuppressEventForwarding();
05959 }
05960 #endif // GTEST_HAS_DEATH_TEST
05961
05962 // Initializes event listeners performing XML output as specified by
05963 // UnitTestOptions. Must not be called before InitGoogleTest.
05964
05965 void UnitTestImpl::ConfigureXmlOutput() {
05966 const std::string& output_format = UnitTestOptions::GetOutputFormat();
05967 if (output_format == "xml") {
05968 listeners()->SetDefaultXmlGenerator(new XmlUnitTestResultPrinter(
05969 UnitTestOptions::GetAbsolutePathToOutputFile().c_str()));
05970 } else if (output_format == "json") {
05971 listeners()->SetDefaultXmlGenerator(new JsonUnitTestResultPrinter(
05972 UnitTestOptions::GetAbsolutePathToOutputFile().c_str()));
05973 } else if (output_format != "") {
05974 GTEST_LOG_(WARNING) << "WARNING: unrecognized output format \""
05975 << output_format << "\" ignored.";
05976 }
05977 }
05978
05979 #if GTEST_CAN_STREAM_RESULTS_
05980 // Initializes event listeners for streaming test results in string form.
05981 // Must not be called before InitGoogleTest.
05982
05983 void UnitTestImpl::ConfigureStreamingOutput() {
05984 const std::string& target = GTEST_FLAG_GET(stream_result_to);
05985 if (!target.empty()) {
05986 const size_t pos = target.find(':');
05987 if (pos != std::string::npos) {
05988 listeners()->Append(
05989 new StreamingListener(target.substr(0, pos), target.substr(pos + 1)));
05990 } else {
05991 GTEST_LOG_(WARNING) << "unrecognized streaming target \"" << target
05992 << "\" ignored.";
05993 }
05994 }
05995 }

```

```

05996 #endif // GTEST_CAN_STREAM_RESULTS_
05997
05998 // Performs initialization dependent upon flag values obtained in
05999 // ParseGoogleTestFlagsOnly. Is called from InitGoogleTest after the call to
06000 // ParseGoogleTestFlagsOnly. In case a user neglects to call InitGoogleTest
06001 // this function is also called from RunAllTests. Since this function can be
06002 // called more than once, it has to be idempotent.
06003
06004 void UnitTestImpl::PostFlagParsingInit() {
06005 // Ensures that this function does not execute more than once.
06006 if (!post_flag_parse_init_performed_) {
06007 post_flag_parse_init_performed_ = true;
06008
06009 #if defined(GTEST_CUSTOM_TEST_EVENT_LISTENER_)
06010 // Register to send notifications about key process state changes.
06011 listeners()->Append(new GTEST_CUSTOM_TEST_EVENT_LISTENER_());
06012 #endif // defined(GTEST_CUSTOM_TEST_EVENT_LISTENER_)
06013
06014 #if GTEST_HAS_DEATH_TEST
06015 InitDeathTestSubprocessControlInfo();
06016 SuppressTestEventsIfInSubprocess();
06017 #endif // GTEST_HAS_DEATH_TEST
06018
06019 // Registers parameterized tests. This makes parameterized tests
06020 // available to the UnitTest reflection API without running
06021 // RUN_ALL_TESTS.
06022 RegisterParameterizedTests();
06023
06024 // Configures listeners for XML output. This makes it possible for users
06025 // to shut down the default XML output before invoking RUN_ALL_TESTS.
06026 ConfigureXmlOutput();
06027
06028 if (GTEST_FLAG_GET(brief)) {
06029 listeners()->SetDefaultResultPrinter(new BriefUnitTestResultPrinter);
06030 }
06031
06032 #if GTEST_CAN_STREAM_RESULTS_
06033 // Configures listeners for streaming test results to the specified server.
06034 ConfigureStreamingOutput();
06035 #endif // GTEST_CAN_STREAM_RESULTS_
06036
06037 #if GTEST_HAS_ABSL
06038 if (GTEST_FLAG_GET(install_failure_signal_handler)) {
06039 absl::FailureSignalHandlerOptions options;
06040 absl::InstallFailureSignalHandler(options);
06041 }
06042 #endif // GTEST_HAS_ABSL
06043 }
06044 }
06045
06046 // A predicate that checks the name of a TestSuite against a known
06047 // value.
06048 //
06049 // This is used for implementation of the UnitTest class only. We put
06050 // it in the anonymous namespace to prevent polluting the outer
06051 // namespace.
06052 //
06053 // TestSuiteNameIs is copyable.
06054
06055 class TestSuiteNameIs {
06056 public:
06057 // Constructor.
06058
06059 explicit TestSuiteNameIs(const std::string& name) : name_(name) {}
06060
06061 // Returns true if and only if the name of test_suite matches name_.
06062
06063 bool operator()(const TestSuite* test_suite) const {
06064 return test_suite != nullptr &&
06065 strcmp(test_suite->name(), name_.c_str()) == 0;
06066 }
06067
06068 private:
06069 std::string name_;
06070 };
06071
06072 // Finds and returns a TestSuite with the given name. If one doesn't
06073 // exist, creates one and returns it. It's the CALLER'S
06074 // RESPONSIBILITY to ensure that this function is only called WHEN THE
06075 // TESTS ARE NOT SHUFFLED.
06076 //
06077 // Arguments:
06078 // test_suite_name: name of the test suite
06079 // type_param: the name of the test suite's type parameter, or NULL if
06080 // this is not a typed or a type-parameterized test suite.

```

```

06083 // set_up_tc: pointer to the function that sets up the test suite
06084 // tear_down_tc: pointer to the function that tears down the test suite
06085
06086 TestSuite* UnitTestImpl::GetTestSuite(
06087 const char* test_suite_name, const char* type_param,
06088 internal::SetUpTestSuiteFunc set_up_tc,
06089 internal::TearDownTestSuiteFunc tear_down_tc) {
06090 // Can we find a TestSuite with the given name?
06091 const auto test_suite =
06092 std::find_if(test_suites_.rbegin(), test_suites_.rend(),
06093 TestSuiteNameIs(test_suite_name));
06094
06095 if (test_suite != test_suites_.rend()) return *test_suite;
06096
06097 // No. Let's create one.
06098 auto* const new_test_suite =
06099 new TestSuite(test_suite_name, type_param, set_up_tc, tear_down_tc);
06100
06101 const UnitTestFilter death_test_suite_filter(kDeathTestSuiteFilter);
06102 // Is this a death test suite?
06103 if (death_test_suite_filter.MatchesName(test_suite_name)) {
06104 // Yes. Inserts the test suite after the last death test suite
06105 // defined so far. This only works when the test suites haven't
06106 // been shuffled. Otherwise we may end up running a death test
06107 // after a non-death test.
06108 ++last_death_test_suite_;
06109 test_suites_.insert(test_suites_.begin() + last_death_test_suite_,
06110 new_test_suite);
06111 } else {
06112 // No. Appends to the end of the list.
06113 test_suites_.push_back(new_test_suite);
06114 }
06115
06116 test_suite_indices_.push_back(static_cast<int> (test_suite_indices_.size()));
06117 return new_test_suite;
06118 }
06119
06120 // Helpers for setting up / tearing down the given environment. They
06121 // are for use in the ForEach() function.
06122
06123 static void SetUpEnvironment(Environment* env) {
06124 env->SetUp();
06125 }
06126
06127 static void TearDownEnvironment(Environment* env) {
06128 env->TearDown();
06129 }
06130
06131 // Runs all tests in this UnitTest object, prints the result, and
06132 // returns true if all tests are successful. If any exception is
06133 // thrown during a test, the test is considered to be failed, but the
06134 // rest of the tests will still be run.
06135 //
06136 // When parameterized tests are enabled, it expands and registers
06137 // parameterized tests first in RegisterParameterizedTests().
06138 // All other functions called from RunAllTests() may safely assume that
06139 // parameterized tests are ready to be counted and run.
06140
06141 bool UnitTestImpl::RunAllTests() {
06142 // True if and only if Google Test is initialized before RUN_ALL_TESTS() is
06143 // called.
06144 const bool gtest_is_initialized_before_run_all_tests = GTestIsInitialized();
06145
06146 // Do not run any test if the --help flag was specified.
06147 if (g_help_flag) return true;
06148
06149 // Repeats the call to the post-flag parsing initialization in case the
06150 // user didn't call InitGoogleTest.
06151 PostFlagParsingInit();
06152
06153 // Even if sharding is not on, test runners may want to use the
06154 // GTEST_SHARD_STATUS_FILE to query whether the test supports the sharding
06155 // protocol.
06156 internal::WriteToShardStatusFileIfNeeded();
06157
06158 // True if and only if we are in a subprocess for running a thread-safe-style
06159 // death test.
06160 bool in_subprocess_for_death_test = false;
06161
06162 #if GTEST_HAS_DEATH_TEST
06163 in_subprocess_for_death_test =
06164 (internal_run_death_test_flag_.get() != nullptr);
06165 #if defined(GTEST_EXTRA_DEATH_TEST_CHILD_SETUP_)
06166 if (in_subprocess_for_death_test) {
06167 GTEST_EXTRA_DEATH_TEST_CHILD_SETUP_();
06168 }
06169 #endif // defined(GTEST_EXTRA_DEATH_TEST_CHILD_SETUP_)
06170 #endif

```

```

06170 #endif // GTEST_HAS_DEATH_TEST
06171
06172 const bool should_shard = ShouldShard(kTestTotalShards, kTestShardIndex,
06173 in_subprocess_for_death_test);
06174
06175 // Compares the full test names with the filter to decide which
06176 // tests to run.
06177 const bool has_tests_to_run =
06178 FilterTests(should_shard ? HONOR_SHARDING_PROTOCOL
06179 : IGNORE_SHARDING_PROTOCOL) > 0;
06180
06181 // Lists the tests and exits if the --gtest_list_tests flag was specified.
06182 if (GTEST_FLAG_GET(list_tests)) {
06183 // This must be called after FilterTests() has been called.
06184 ListTestsMatchingFilter();
06185 return true;
06186 }
06187
06188 random_seed_ = GetRandomSeedFromFlag(GTEST_FLAG_GET(random_seed));
06189
06190 // True if and only if at least one test has failed.
06191 bool failed = false;
06192
06193 TestEventListener* repeater = listeners()->repeater();
06194
06195 start_timestamp_ = GetTimeInMillis();
06196 repeater->OnTestProgramStart(*parent_);
06197
06198 // How many times to repeat the tests? We don't want to repeat them
06199 // when we are inside the subprocess of a death test.
06200 const int repeat = in_subprocess_for_death_test ? 1 : GTEST_FLAG_GET(repeat);
06201
06202 // Repeats forever if the repeat count is negative.
06203 const bool gtest_repeat_forever = repeat < 0;
06204
06205 // Should test environments be set up and torn down for each repeat, or only
06206 // set up on the first and torn down on the last iteration? If there is no
06207 // "last" iteration because the tests will repeat forever, always recreate the
06208 // environments to avoid leaks in case one of the environments is using
06209 // resources that are external to this process. Without this check there would
06210 // be no way to clean up those external resources automatically.
06211 const bool recreate_environments_when_repeating =
06212 GTEST_FLAG_GET(recreate_environments_when_repeating) ||
06213 gtest_repeat_forever;
06214
06215 for (int i = 0; gtest_repeat_forever || i != repeat; i++) {
06216 // We want to preserve failures generated by ad-hoc test
06217 // assertions executed before RUN_ALL_TESTS().
06218 ClearNonAdHocTestResult();
06219
06220 Timer timer;
06221
06222 // Shuffles test suites and tests if requested.
06223 if (has_tests_to_run && GTEST_FLAG_GET(shuffle)) {
06224 random()->Reseed(static_cast<uint32_t>(random_seed_));
06225 // This should be done before calling OnTestIterationStart(),
06226 // such that a test event listener can see the actual test order
06227 // in the event.
06228 ShuffleTests();
06229 }
06230
06231 // Tells the unit test event listeners that the tests are about to start.
06232 repeater->OnTestIterationStart(*parent_, i);
06233
06234 // Runs each test suite if there is at least one test to run.
06235 if (has_tests_to_run) {
06236 // Sets up all environments beforehand. If test environments aren't
06237 // recreated for each iteration, only do so on the first iteration.
06238 if (i == 0 || recreate_environments_when_repeating) {
06239 repeater->OnEnvironmentsSetUpStart(*parent_);
06240 ForEach(environments_, SetUpEnvironment);
06241 repeater->OnEnvironmentsSetUpEnd(*parent_);
06242 }
06243
06244 // Runs the tests only if there was no fatal failure or skip triggered
06245 // during global set-up.
06246 if (Test::IsSkipped()) {
06247 // Emit diagnostics when global set-up calls skip, as it will not be
06248 // emitted by default.
06249 TestResult& test_result =
06250 *internal::GetUnitTestImpl()->current_test_result();
06251 for (int j = 0; j < test_result.total_part_count(); ++j) {
06252 const TestPartResult& test_part_result =
06253 test_result.GetTestPartResult(j);
06254 if (test_part_result.type() == TestPartResult::kSkip) {
06255 const std::string& result = test_part_result.message();
06256 printf("%s\n", result.c_str());

```

```

06257 }
06258 }
06259 fflush(stdout);
06260 } else if (!Test::HasFatalFailure()) {
06261 for (int test_index = 0; test_index < total_test_suite_count();
06262 test_index++) {
06263 GetMutableSuiteCase(test_index)->Run();
06264 if (GTEST_FLAG_GET(fail_fast) &&
06265 GetMutableSuiteCase(test_index)->Failed()) {
06266 for (int j = test_index + 1; j < total_test_suite_count(); j++) {
06267 GetMutableSuiteCase(j)->Skip();
06268 }
06269 break;
06270 }
06271 }
06272 } else if (Test::HasFatalFailure()) {
06273 // If there was a fatal failure during the global setup then we know we
06274 // aren't going to run any tests. Explicitly mark all of the tests as
06275 // skipped to make this obvious in the output.
06276 for (int test_index = 0; test_index < total_test_suite_count();
06277 test_index++) {
06278 GetMutableSuiteCase(test_index)->Skip();
06279 }
06280 }
06281
06282 // Tears down all environments in reverse order afterwards. If test
06283 // environments aren't recreated for each iteration, only do so on the
06284 // last iteration.
06285 if (i == repeat - 1 || recreate_environments_when_repeating) {
06286 repeater->OnEnvironmentsTearDownStart(*parent_);
06287 std::for_each(environments_.rbegin(), environments_.rend(),
06288 TearDownEnvironment);
06289 repeater->OnEnvironmentsTearDownEnd(*parent_);
06290 }
06291 }
06292
06293 elapsed_time_ = timer.Elapsed();
06294
06295 // Tells the unit test event listener that the tests have just finished.
06296 repeater->OnTestIterationEnd(*parent_, i);
06297
06298 // Gets the result and clears it.
06299 if (!Passed()) {
06300 failed = true;
06301 }
06302
06303 // Restores the original test order after the iteration. This
06304 // allows the user to quickly repro a failure that happens in the
06305 // N-th iteration without repeating the first (N - 1) iterations.
06306 // This is not enclosed in "if (GTEST_FLAG(shuffle)) { ... }", in
06307 // case the user somehow changes the value of the flag somewhere
06308 // (it's always safe to unshuffle the tests).
06309 UnshuffleTests();
06310
06311 if (GTEST_FLAG_GET(shuffle)) {
06312 // Picks a new random seed for each iteration.
06313 random_seed_ = GetNextRandomSeed(random_seed_);
06314 }
06315 }
06316
06317 repeater->OnTestProgramEnd(*parent_);
06318
06319 if (!gtest_is_initialized_before_run_all_tests) {
06320 ColoredPrintf(
06321 GTestColor::kRed,
06322 "\nIMPORTANT NOTICE - DO NOT IGNORE:\n"
06323 "This test program did NOT call " GTEST_INIT_GOOGLE_TEST_NAME_
06324 "() before calling RUN_ALL_TESTS(). This is INVALID. Soon " GTEST_NAME_
06325 " will start to enforce the valid usage. "
06326 "Please fix it ASAP, or IT WILL START TO FAIL.\n"); // NOLINT
06327 #if GTEST_FOR_GOOGLE_
06328 ColoredPrintf(GTestColor::kRed,
06329 "For more details, see http://wiki/Main/ValidGUnitMain.\n");
06330 #endif // GTEST_FOR_GOOGLE_
06331 }
06332
06333 return !failed;
06334 }
06335
06336 // Reads the GTEST_SHARD_STATUS_FILE environment variable, and creates the file
06337 // if the variable is present. If a file already exists at this location, this
06338 // function will write over it. If the variable is present, but the file cannot
06339 // be created, prints an error and exits.
06340
06341 void WriteToShardStatusFileIfNeeded() {
06342 const char* const test_shard_file = posix::GetEnv(kTestShardStatusFile);
06343 if (test_shard_file != nullptr) {

```



```

06344 FILE * const file = posix::FOpen(test_shard_file, "w");
06345 if (file == nullptr) {
06346 ColoredPrintf(GTestColor::kRed,
06347 "Could not write to the test shard status file \"%s\" "
06348 "specified by the %s environment variable.\n",
06349 test_shard_file, kTestShardStatusFile);
06350 fflush(stdout);
06351 exit(EXIT_FAILURE);
06352 }
06353 fclose(file);
06354 }
06355 }
06356
06357 // Checks whether sharding is enabled by examining the relevant
06358 // environment variable values. If the variables are present,
06359 // but inconsistent (i.e., shard_index >= total_shards), prints
06360 // an error and exits. If in_subprocess_for_death_test, sharding is
06361 // disabled because it must only be applied to the original test
06362 // process. Otherwise, we could filter out death tests we intended to execute.
06363
06364 bool ShouldShard(const char* total_shards_env, const char* shard_index_env,
06365 bool in_subprocess_for_death_test) {
06366 if (in_subprocess_for_death_test) {
06367 return false;
06368 }
06369
06370 const int32_t total_shards = Int32FromEnvOrDie(total_shards_env, -1);
06371 const int32_t shard_index = Int32FromEnvOrDie(shard_index_env, -1);
06372
06373 if (total_shards == -1 && shard_index == -1) {
06374 return false;
06375 } else if (total_shards == -1 && shard_index != -1) {
06376 const Message msg = Message() << "Invalid environment variables: you have "
06377 << kTestShardIndex << " = " << shard_index
06378 << ", but have left " << kTestTotalShards
06379 << " unset.\n";
06380 ColoredPrintf(GTestColor::kRed, "%s", msg.GetString().c_str());
06381 fflush(stdout);
06382 exit(EXIT_FAILURE);
06383 } else if (total_shards != -1 && shard_index == -1) {
06384 const Message msg = Message()
06385 << "Invalid environment variables: you have "
06386 << kTestTotalShards << " = " << total_shards
06387 << ", but have left " << kTestShardIndex << " unset.\n";
06388 ColoredPrintf(GTestColor::kRed, "%s", msg.GetString().c_str());
06389 fflush(stdout);
06390 exit(EXIT_FAILURE);
06391 } else if (shard_index < 0 || shard_index >= total_shards) {
06392 const Message msg =
06393 Message() << "Invalid environment variables: we require 0 <= "
06394 << kTestShardIndex << " < " << kTestTotalShards
06395 << ", but you have " << kTestShardIndex << " = " << shard_index
06396 << ", " << kTestTotalShards << " = " << total_shards << ".\n";
06397 ColoredPrintf(GTestColor::kRed, "%s", msg.GetString().c_str());
06398 fflush(stdout);
06399 exit(EXIT_FAILURE);
06400 }
06401
06402 return total_shards > 1;
06403 }
06404
06405 // Parses the environment variable var as an Int32. If it is unset,
06406 // returns default_val. If it is not an Int32, prints an error
06407 // and aborts.
06408
06409 int32_t Int32FromEnvOrDie(const char* var, int32_t default_val) {
06410 const char* str_val = posix::GetEnv(var);
06411 if (str_val == nullptr) {
06412 return default_val;
06413 }
06414
06415 int32_t result;
06416 if (!ParseInt32(Message() << "The value of environment variable " << var,
06417 str_val, &result)) {
06418 exit(EXIT_FAILURE);
06419 }
06420 return result;
06421 }
06422
06423 // Given the total number of shards, the shard index, and the test id,
06424 // returns true if and only if the test should be run on this shard. The test id
06425 // is some arbitrary but unique non-negative integer assigned to each test
06426 // method. Assumes that 0 <= shard_index < total_shards.
06427
06428 bool ShouldRunTestOnShard(int total_shards, int shard_index, int test_id) {
06429 return (test_id % total_shards) == shard_index;
06430 }

```

```

06431
06432 // Compares the name of each test with the user-specified filter to
06433 // decide whether the test should be run, then records the result in
06434 // each TestSuite and TestInfo object.
06435 // If shard_tests == true, further filters tests based on sharding
06436 // variables in the environment - see
06437 // https://github.com/google/googletest/blob/main/docs/advanced.md
06438 // . Returns the number of tests that should run.
06439
06440 int UnitTestImpl::FilterTests(ReactionToSharding shard_tests) {
06441 const int32_t total_shards = shard_tests == HONOR_SHARDING_PROTOCOL
06442 ? Int32FromEnvOrDie(kTestTotalShards, -1)
06443 : -1;
06444 const int32_t shard_index = shard_tests == HONOR_SHARDING_PROTOCOL
06445 ? Int32FromEnvOrDie(kTestShardIndex, -1)
06446 : -1;
06447
06448 const PositiveAndNegativeUnitTestFixture gtest_flag_filter(
06449 GTEST_FLAG_GET(filter));
06450 const UnitTestFixture disable_test_filter(kDisableTestFixture);
06451 // num_runnable_tests are the number of tests that will
06452 // run across all shards (i.e., match filter and are not disabled).
06453 // num_selected_tests are the number of tests to be run on
06454 // this shard.
06455 int num_runnable_tests = 0;
06456 int num_selected_tests = 0;
06457 for (auto* test_suite : test_suites_) {
06458 const std::string& test_suite_name = test_suite->name();
06459 test_suite->set_should_run(false);
06460
06461 for (size_t j = 0; j < test_suite->test_info_list().size(); j++) {
06462 TestInfo * const test_info = test_suite->test_info_list()[j];
06463 const std::string test_name(test_info->name());
06464 // A test is disabled if test suite name or test name matches
06465 // kDisableTestFixture.
06466 const bool is_disabled =
06467 disable_test_filter.MatchesName(test_suite_name) ||
06468 disable_test_filter.MatchesName(test_name);
06469 test_info->is_disabled_ = is_disabled;
06470
06471 const bool matches_filter =
06472 gtest_flag_filter.MatchesTest(test_suite_name, test_name);
06473 test_info->matches_filter_ = matches_filter;
06474
06475 const bool is_runnable =
06476 (GTEST_FLAG_GET(also_run_disabled_tests) || !is_disabled) &&
06477 matches_filter;
06478
06479 const bool is_in_another_shard =
06480 shard_tests != IGNORE_SHARDING_PROTOCOL &&
06481 !ShouldRunTestOnShard(total_shards, shard_index, num_runnable_tests);
06482 test_info->is_in_another_shard_ = is_in_another_shard;
06483 const bool is_selected = is_runnable && !is_in_another_shard;
06484
06485 num_runnable_tests += is_runnable;
06486 num_selected_tests += is_selected;
06487
06488 test_info->should_run_ = is_selected;
06489 test_suite->set_should_run(test_suite->should_run() || is_selected);
06490 }
06491 }
06492 return num_selected_tests;
06493 }
06494
06495 // Prints the given C-string on a single line by replacing all '\n'
06496 // characters with string "\\n". If the output takes more than
06497 // max_length characters, only prints the first max_length characters
06498 // and "...".
06499
06500 static void PrintOnOneLine(const char* str, int max_length) {
06501 if (str != nullptr) {
06502 for (int i = 0; *str != '\0'; ++str) {
06503 if (i >= max_length) {
06504 printf("...");
06505 break;
06506 }
06507 if (*str == '\n') {
06508 printf("\\n");
06509 i += 2;
06510 } else {
06511 printf("%c", *str);
06512 ++i;
06513 }
06514 }
06515 }
06516 }
06517

```

```

06518 // Prints the names of the tests matching the user-specified filter flag.
06519
06520 void UnitTestImpl::ListTestsMatchingFilter() {
06521 // Print at most this many characters for each type/value parameter.
06522 const int kMaxParamLength = 250;
06523
06524 for (auto* test_suite : test_suites_) {
06525 bool printed_test_suite_name = false;
06526
06527 for (size_t j = 0; j < test_suite->test_info_list().size(); j++) {
06528 const TestInfo * const test_info = test_suite->test_info_list()[j];
06529 if (test_info->matches_filter_) {
06530 if (!printed_test_suite_name) {
06531 printed_test_suite_name = true;
06532 printf("%s.", test_suite->name());
06533 if (test_suite->type_param() != nullptr) {
06534 printf(" # %s = ", kTypeParamLabel);
06535 // We print the type parameter on a single line to make
06536 // the output easy to parse by a program.
06537 PrintOnOneLine(test_suite->type_param(), kMaxParamLength);
06538 }
06539 printf("\n");
06540 }
06541 printf(" %s", test_info->name());
06542 if (test_info->value_param() != nullptr) {
06543 printf(" # %s = ", kValueParamLabel);
06544 // We print the value parameter on a single line to make the
06545 // output easy to parse by a program.
06546 PrintOnOneLine(test_info->value_param(), kMaxParamLength);
06547 }
06548 printf("\n");
06549 }
06550 }
06551 }
06552 fflush(stdout);
06553 const std::string& output_format = UnitTestOptions::GetOutputFormat();
06554 if (output_format == "xml" || output_format == "json") {
06555 FILE* fileout = OpenFileForWriting(
06556 UnitTestOptions::GetAbsolutePathToOutputFile().c_str());
06557 std::stringstream stream;
06558 if (output_format == "xml") {
06559 XmlUnitTestResultPrinter(
06560 UnitTestOptions::GetAbsolutePathToOutputFile().c_str())
06561 .PrintXmlTestsList(&stream, test_suites_);
06562 } else if (output_format == "json") {
06563 JsonUnitTestResultPrinter(
06564 UnitTestOptions::GetAbsolutePathToOutputFile().c_str())
06565 .PrintJsonTestsList(&stream, test_suites_);
06566 }
06567 fprintf(fileout, "%s", StreamToString(stream).c_str());
06568 fclose(fileout);
06569 }
06570 }
06571
06572 // Sets the OS stack trace getter.
06573 //
06574 // Does nothing if the input and the current OS stack trace getter are
06575 // the same; otherwise, deletes the old getter and makes the input the
06576 // current getter.
06577
06578 void UnitTestImpl::set_os_stack_trace_getter(
06579 OsStackTraceGetterInterface* getter) {
06580 if (os_stack_trace_getter_ != getter) {
06581 delete os_stack_trace_getter_;
06582 os_stack_trace_getter_ = getter;
06583 }
06584 }
06585
06586 // Returns the current OS stack trace getter if it is not NULL;
06587 // otherwise, creates an OsStackTraceGetter, makes it the current
06588 // getter, and returns it.
06589
06590 OsStackTraceGetterInterface* UnitTestImpl::os_stack_trace_getter() {
06591 if (os_stack_trace_getter_ == nullptr) {
06592 #ifdef GTEST_OS_STACK_TRACE_GETTER_
06593 os_stack_trace_getter_ = new GTEST_OS_STACK_TRACE_GETTER_;
06594 #else
06595 os_stack_trace_getter_ = new OsStackTraceGetter;
06596 #endif // GTEST_OS_STACK_TRACE_GETTER_
06597 }
06598 return os_stack_trace_getter_;
06599 }
06600
06601 // Returns the most specific TestResult currently running.
06602
06603 TestResult* UnitTestImpl::current_test_result() {

```

```

06605 if (current_test_info_ != nullptr) {
06606 return ¤t_test_info_>result_;
06607 }
06608 if (current_test_suite_ != nullptr) {
06609 return ¤t_test_suite_>ad_hoc_test_result_;
06610 }
06611 return &ad_hoc_test_result_;
06612 }
06613
06614 // Shuffles all test suites, and the tests within each test suite,
06615 // making sure that death tests are still run first.
06616
06617 void UnitTestImpl::ShuffleTests() {
06618 // Shuffles the death test suites.
06619 ShuffleRange(random(), 0, last_death_test_suite_ + 1, &test_suite_indices_);
06620
06621 // Shuffles the non-death test suites.
06622 ShuffleRange(random(), last_death_test_suite_ + 1,
06623 static_cast<int>(test_suites_.size()), &test_suite_indices_);
06624
06625 // Shuffles the tests inside each test suite.
06626 for (auto& test_suite : test_suites_) {
06627 test_suite->ShuffleTests(random());
06628 }
06629 }
06630
06631 // Restores the test suites and tests to their order before the first shuffle.
06632
06633 void UnitTestImpl::UnshuffleTests() {
06634 for (size_t i = 0; i < test_suites_.size(); i++) {
06635 // Unshuffles the tests in each test suite.
06636 test_suites_[i]->UnshuffleTests();
06637 // Resets the index of each test suite.
06638 test_suite_indices_[i] = static_cast<int>(i);
06639 }
06640 }
06641
06642 // Returns the current OS stack trace as an std::string.
06643 //
06644 // The maximum number of stack frames to be included is specified by
06645 // the gtest_stack_trace_depth flag. The skip_count parameter
06646 // specifies the number of top frames to be skipped, which doesn't
06647 // count against the number of frames to be included.
06648 //
06649 // For example, if Foo() calls Bar(), which in turn calls
06650 // GetCurrentOsStackTraceExceptTop(..., 1), Foo() will be included in
06651 // the trace but Bar() and GetCurrentOsStackTraceExceptTop() won't.
06652
06653 GTEST_NO_INLINE_ GTEST_NO_TAIL_CALL_ std::string
06654 GetCurrentOsStackTraceExceptTop(int skip_count) {
06655 // We pass skip_count + 1 to skip this wrapper function in addition
06656 // to what the user really wants to skip.
06657 return GetUnitTestImpl()->CurrentOsStackTraceExceptTop(skip_count + 1);
06658 }
06659
06660 // Used by the GTEST_SUPPRESS_UNREACHABLE_CODE_WARNING_BELOW_ macro to
06661 // suppress unreachable code warnings.
06662 namespace {
06663
06664 class ClassUniqueToAlwaysTrue {
06665 };
06666 } // namespace
06667
06668 bool IsTrue(bool condition) {
06669 return condition;
06670 }
06671
06672 bool AlwaysTrue() {
06673 #if GTEST_HAS_EXCEPTIONS
06674 // This condition is always false so AlwaysTrue() never actually throws,
06675 // but it makes the compiler think that it may throw.
06676 if (IsTrue(false)) throw ClassUniqueToAlwaysTrue();
06677 #endif // GTEST_HAS_EXCEPTIONS
06678 return true;
06679 }
06680
06681 // If *pstr starts with the given prefix, modifies *pstr to be right
06682 // past the prefix and returns true; otherwise leaves *pstr unchanged
06683 // and returns false. None of pstr, *pstr, and prefix can be NULL.
06684
06685 bool SkipPrefix(const char* prefix, const char** pstr) {
06686 const size_t prefix_len = strlen(prefix);
06687 if (strncmp(*pstr, prefix, prefix_len) == 0) {
06688 *pstr += prefix_len;
06689 return true;
06690 }
06691 return false;

```

```

06692 }
06693
06694 // Parses a string as a command line flag. The string should have
06695 // the format "--flag=value". When def_optional is true, the "=value"
06696 // part can be omitted.
06697 //
06698 // Returns the value of the flag, or NULL if the parsing failed.
06699
06700 static const char* ParseFlagValue(const char* str, const char* flag_name,
06701 bool def_optional) {
06702 // str and flag must not be NULL.
06703 if (str == nullptr || flag_name == nullptr) return nullptr;
06704
06705 // The flag must start with "--" followed by GTEST_FLAG_PREFIX_.
06706 const std::string flag_str =
06707 std::string("--") + GTEST_FLAG_PREFIX_ + flag_name;
06708 const size_t flag_len = flag_str.length();
06709 if (strncmp(str, flag_str.c_str(), flag_len) != 0) return nullptr;
06710
06711 // Skips the flag name.
06712 const char* flag_end = str + flag_len;
06713
06714 // When def_optional is true, it's OK to not have a "=value" part.
06715 if (def_optional && (flag_end[0] == '\0')) {
06716 return flag_end;
06717 }
06718
06719 // If def_optional is true and there are more characters after the
06720 // flag name, or if def_optional is false, there must be a '=' after
06721 // the flag name.
06722 if (flag_end[0] != '=') return nullptr;
06723
06724 // Returns the string after "=".
06725 return flag_end + 1;
06726 }
06727
06728 // Parses a string for a bool flag, in the form of either
06729 // "--flag=value" or "--flag".
06730 //
06731 // In the former case, the value is taken as true as long as it does
06732 // not start with '0', 'f', or 'F'.
06733 //
06734 // In the latter case, the value is taken as true.
06735 //
06736 // On success, stores the value of the flag in *value, and returns
06737 // true. On failure, returns false without changing *value.
06738
06739 static bool ParseFlag(const char* str, const char* flag_name, bool* value) {
06740 // Gets the value of the flag as a string.
06741 const char* const value_str = ParseFlagValue(str, flag_name, true);
06742
06743 // Aborts if the parsing failed.
06744 if (value_str == nullptr) return false;
06745
06746 // Converts the string value to a bool.
06747 *value = !(*value_str == '0' || *value_str == 'f' || *value_str == 'F');
06748 return true;
06749 }
06750
06751 // Parses a string for an int32_t flag, in the form of "--flag=value".
06752 //
06753 // On success, stores the value of the flag in *value, and returns
06754 // true. On failure, returns false without changing *value.
06755
06756 bool ParseFlag(const char* str, const char* flag_name, int32_t* value) {
06757 // Gets the value of the flag as a string.
06758 const char* const value_str = ParseFlagValue(str, flag_name, false);
06759
06760 // Aborts if the parsing failed.
06761 if (value_str == nullptr) return false;
06762
06763 // Sets *value to the value of the flag.
06764 return ParseInt32(Message() << "The value of flag --" << flag_name, value_str,
06765 value);
06766 }
06767
06768 // Parses a string for a string flag, in the form of "--flag=value".
06769 //
06770 // On success, stores the value of the flag in *value, and returns
06771 // true. On failure, returns false without changing *value.
06772
06773 template <typename String>
06774 static bool ParseFlag(const char* str, const char* flag_name, String* value) {
06775 // Gets the value of the flag as a string.
06776 const char* const value_str = ParseFlagValue(str, flag_name, false);
06777
06778 // Aborts if the parsing failed.

```

```

06779 if (value_str == nullptr) return false;
06780
06781 // Sets *value to the value of the flag.
06782 *value = value_str;
06783 return true;
06784 }
06785
06786 // Determines whether a string has a prefix that Google Test uses for its
06787 // flags, i.e., starts with GTEST_FLAG_PREFIX_ or GTEST_FLAG_PREFIX_DASH_.
06788 // If Google Test detects that a command line flag has its prefix but is not
06789 // recognized, it will print its help message. Flags starting with
06790 // GTEST_INTERNAL_PREFIX_ followed by "internal_" are considered Google Test
06791 // internal flags and do not trigger the help message.
06792
06793 static bool HasGoogleTestFlagPrefix(const char* str) {
06794 return (SkipPrefix("--", &str) || SkipPrefix("-", &str) ||
06795 SkipPrefix("/", &str)) &&
06796 !SkipPrefix(GTEST_FLAG_PREFIX_ "internal_", &str) &&
06797 (SkipPrefix(GTEST_FLAG_PREFIX_, &str) ||
06798 SkipPrefix(GTEST_FLAG_PREFIX_DASH_, &str));
06799 }
06800
06801 // Prints a string containing code-encoded text. The following escape
06802 // sequences can be used in the string to control the text color:
06803 //
06804 // @@ prints a single '@' character.
06805 // @R changes the color to red.
06806 // @G changes the color to green.
06807 // @Y changes the color to yellow.
06808 // @D changes to the default terminal text color.
06809 //
06810
06811 static void PrintColorEncoded(const char* str) {
06812 GTestColor color = GTestColor::kDefault; // The current color.
06813
06814 // Conceptually, we split the string into segments divided by escape
06815 // sequences. Then we print one segment at a time. At the end of
06816 // each iteration, the str pointer advances to the beginning of the
06817 // next segment.
06818 for (;;) {
06819 const char* p = strchr(str, '@');
06820 if (p == nullptr) {
06821 ColoredPrintf(color, "%s", str);
06822 return;
06823 }
06824
06825 ColoredPrintf(color, "%s", std::string(str, p).c_str());
06826
06827 const char ch = p[1];
06828 str = p + 2;
06829 if (ch == '@') {
06830 ColoredPrintf(color, "@");
06831 } else if (ch == 'D') {
06832 color = GTestColor::kDefault;
06833 } else if (ch == 'R') {
06834 color = GTestColor::kRed;
06835 } else if (ch == 'G') {
06836 color = GTestColor::kGreen;
06837 } else if (ch == 'Y') {
06838 color = GTestColor::kYellow;
06839 } else {
06840 --str;
06841 }
06842 }
06843 }
06844
06845 static const char kColorEncodedHelpMessage[] =
06846 "This program contains tests written using " GTEST_NAME_
06847 ". You can use the\n"
06848 "following command line flags to control its behavior:\n"
06849 "\n"
06850 "Test Selection:\n"
06851 " @G--" GTEST_FLAG_PREFIX_
06852 "list_tests@D\n"
06853 " List the names of all tests instead of running them. The name of\n"
06854 " TEST(Foo, Bar) is \"Foo.Bar\".\n"
06855 " @G--" GTEST_FLAG_PREFIX_
06856 "filter=@YPOSITIVE_PATTERNS\n"
06857 " [@G-@YNEGATIVE_PATTERNS]@D\n"
06858 " Run only the tests whose name matches one of the positive patterns "
06859 "but\n"
06860 " none of the negative patterns. '?' matches any single character; "
06861 "'*' \n"
06862 " matches any substring; ':' separates two patterns.\n"
06863 " @G--" GTEST_FLAG_PREFIX_
06864 "also_run_disabled_tests@D\n"
06865 " Run all disabled tests too.\n"

```

```

06866 "\n"
06867 "Test Execution:\n"
06868 " @G--" GTEST_FLAG_PREFIX_
06869 "repeat=@Y[COUNT]@D\n"
06870 " Run the tests repeatedly; use a negative count to repeat forever.\n"
06871 " @G--" GTEST_FLAG_PREFIX_
06872 "shuffle@D\n"
06873 " Randomize tests' orders on every iteration.\n"
06874 " @G--" GTEST_FLAG_PREFIX_
06875 "random_seed=@Y[NUMBER]@D\n"
06876 " Random number seed to use for shuffling test orders (between 1 and\n"
06877 " 99999, or 0 to use a seed based on the current time).\n"
06878 " @G--" GTEST_FLAG_PREFIX_
06879 "recreate_environments_when_repeating@D\n"
06880 " Sets up and tears down the global test environment on each repeat\n"
06881 " of the test.\n"
06882 "\n"
06883 "Test Output:\n"
06884 " @G--" GTEST_FLAG_PREFIX_
06885 "color=@Y(@Gyes@Y|@Gno@Y|@Gauto@Y)@D\n"
06886 " Enable/disable colored output. The default is @Gauto@D.\n"
06887 " @G--" GTEST_FLAG_PREFIX_
06888 "brief=1@D\n"
06889 " Only print test failures.\n"
06890 " @G--" GTEST_FLAG_PREFIX_
06891 "print_time=0@D\n"
06892 " Don't print the elapsed time of each test.\n"
06893 " @G--" GTEST_FLAG_PREFIX_
06894 "output=@Y(@Gjson@Y|@Gxml@Y) [@G:@YDIRECTORY_PATH@G" GTEST_PATH_SEP_
06895 "@Y|@G:@YFILE_PATH]@D\n"
06896 " Generate a JSON or XML report in the given directory or with the "
06897 " given\n"
06898 " file name. @YFILE_PATH@D defaults to @Gtest_detail.xml@D.\n"
06899 #if GTEST_CAN_STREAM_RESULTS_
06900 " @G--" GTEST_FLAG_PREFIX_
06901 "stream_result_to=@YHOST@G:@YPORT@D\n"
06902 " Stream test results to the given server.\n"
06903 #endif // GTEST_CAN_STREAM_RESULTS_
06904 "\n"
06905 "Assertion Behavior:\n"
06906 #if GTEST_HAS_DEATH_TEST && !GTEST_OS_WINDOWS
06907 " @G--" GTEST_FLAG_PREFIX_
06908 "death_test_style=@Y(@Gfast@Y|@Gthreadsafely@Y)@D\n"
06909 " Set the default death test style.\n"
06910 #endif // GTEST_HAS_DEATH_TEST && !GTEST_OS_WINDOWS
06911 " @G--" GTEST_FLAG_PREFIX_
06912 "break_on_failure@D\n"
06913 " Turn assertion failures into debugger break-points.\n"
06914 " @G--" GTEST_FLAG_PREFIX_
06915 "throw_on_failure@D\n"
06916 " Turn assertion failures into C++ exceptions for use by an external\n"
06917 " test framework.\n"
06918 " @G--" GTEST_FLAG_PREFIX_
06919 "catch_exceptions=0@D\n"
06920 " Do not report exceptions as test failures. Instead, allow them\n"
06921 " to crash the program or throw a pop-up (on Windows).\n"
06922 "\n"
06923 "Except for @G--" GTEST_FLAG_PREFIX_
06924 "list_tests@D, you can alternatively set "
06925 "the corresponding\n"
06926 "environment variable of a flag (all letters in upper-case). For example, "
06927 "to\n"
06928 "disable colored text output, you can either specify "
06929 "@G--" GTEST_FLAG_PREFIX_
06930 "color=no@D or set\n"
06931 "the @G" GTEST_FLAG_PREFIX_UPPER_
06932 "COLOR@D environment variable to @Gno@D.\n"
06933 "\n"
06934 "For more information, please read the " GTEST_NAME_
06935 " documentation at\n"
06936 "@G" GTEST_PROJECT_URL_ "@D. If you find a bug in " GTEST_NAME_
06937 "\n"
06938 "(not one in your own code or tests), please report it to\n"
06939 "@G<" GTEST_DEV_EMAIL_ ">@D.\n";
06940
06941 static bool ParseGoogleTestFlag(const char* const arg) {
06942 #define GTEST_INTERNAL_PARSE_FLAG(flag_name) \
06943 do { \
06944 auto value = GTEST_FLAG_GET(flag_name); \
06945 if (ParseFlag(arg, #flag_name, &value)) { \
06946 GTEST_FLAG_SET(flag_name, value); \
06947 return true; \
06948 } \
06949 } while (false)
06950
06951 GTEST_INTERNAL_PARSE_FLAG(also_run_disabled_tests);
06952 GTEST_INTERNAL_PARSE_FLAG(break_on_failure);

```

```

06953 GTEST_INTERNAL_PARSE_FLAG(catch_exceptions);
06954 GTEST_INTERNAL_PARSE_FLAG(color);
06955 GTEST_INTERNAL_PARSE_FLAG(death_test_style);
06956 GTEST_INTERNAL_PARSE_FLAG(death_test_use_fork);
06957 GTEST_INTERNAL_PARSE_FLAG(fail_fast);
06958 GTEST_INTERNAL_PARSE_FLAG(filter);
06959 GTEST_INTERNAL_PARSE_FLAG(internal_run_death_test);
06960 GTEST_INTERNAL_PARSE_FLAG(list_tests);
06961 GTEST_INTERNAL_PARSE_FLAG(output);
06962 GTEST_INTERNAL_PARSE_FLAG(brief);
06963 GTEST_INTERNAL_PARSE_FLAG(print_time);
06964 GTEST_INTERNAL_PARSE_FLAG(print_utf8);
06965 GTEST_INTERNAL_PARSE_FLAG(random_seed);
06966 GTEST_INTERNAL_PARSE_FLAG(repeat);
06967 GTEST_INTERNAL_PARSE_FLAG(recreate_environments_when_repeating);
06968 GTEST_INTERNAL_PARSE_FLAG(shuffle);
06969 GTEST_INTERNAL_PARSE_FLAG(stack_trace_depth);
06970 GTEST_INTERNAL_PARSE_FLAG(stream_result_to);
06971 GTEST_INTERNAL_PARSE_FLAG(throw_on_failure);
06972 return false;
06973 }
06974
06975 #if GTEST_USE_OWN_FLAGFILE_FLAG_
06976
06977 static void LoadFlagsFromFile(const std::string& path) {
06978 FILE* flagfile = posix::FOpen(path.c_str(), "r");
06979 if (!flagfile) {
06980 GTEST_LOG_(FATAL) << "Unable to open file \"" << GTEST_FLAG_GET(flagfile)
06981 << "\"";
06982 }
06983 std::string contents(ReadEntireFile(flagfile));
06984 posix::FClose(flagfile);
06985 std::vector<std::string> lines;
06986 SplitString(contents, '\n', &lines);
06987 for (size_t i = 0; i < lines.size(); ++i) {
06988 if (lines[i].empty()) continue;
06989 if (!ParseGoogleTestFlag(lines[i].c_str())) g_help_flag = true;
06990 }
06991 }
06992 #endif // GTEST_USE_OWN_FLAGFILE_FLAG_
06993
06994 // Parses the command line for Google Test flags, without initializing
06995 // other parts of Google Test. The type parameter CharType can be
06996 // instantiated to either char or wchar_t.
06997
06998 template <typename CharType>
06999 void ParseGoogleTestFlagsOnlyImpl(int* argc, CharType** argv) {
07000 std::string flagfile_value;
07001 for (int i = 1; i < *argc; i++) {
07002 const std::string arg_string = StreamableToString(argv[i]);
07003 const char* const arg = arg_string.c_str();
07004
07005 using internal::ParseFlag;
07006
07007 bool remove_flag = false;
07008 if (ParseGoogleTestFlag(arg)) {
07009 remove_flag = true;
07010 #if GTEST_USE_OWN_FLAGFILE_FLAG_
07011 } else if (ParseFlag(arg, "flagfile", &flagfile_value)) {
07012 GTEST_FLAG_SET(flagfile, flagfile_value);
07013 LoadFlagsFromFile(flagfile_value);
07014 remove_flag = true;
07015 #endif // GTEST_USE_OWN_FLAGFILE_FLAG_
07016 } else if (arg_string == "--help" || HasGoogleTestFlagPrefix(arg)) {
07017 // Both help flag and unrecognized Google Test flags (excluding
07018 // internal ones) trigger help display.
07019 g_help_flag = true;
07020 }
07021
07022 if (remove_flag) {
07023 // Shift the remainder of the argv list left by one. Note
07024 // that argv has (*argc + 1) elements, the last one always being
07025 // NULL. The following loop moves the trailing NULL element as
07026 // well.
07027 for (int j = i; j != *argc; j++) {
07028 argv[j] = argv[j + 1];
07029 }
07030
07031 // Decrements the argument count.
07032 (*argc)--;
07033
07034 // We also need to decrement the iterator as we just removed
07035 // an element.
07036 i--;
07037 }
07038 }
07039 }

```



```

07040 if (g_help_flag) {
07041 // We print the help here instead of in RUN_ALL_TESTS(), as the
07042 // latter may not be called at all if the user is using Google
07043 // Test with another testing framework.
07044 PrintColorEncoded(kColorEncodedHelpMessage);
07045 }
07046 }
07047
07048 // Parses the command line for Google Test flags, without initializing
07049 // other parts of Google Test.
07050
07051 void ParseGoogleTestFlagsOnly(int* argc, char** argv) {
07052 #if GTEST_HAS_ABSL
07053 if (*argc > 0) {
07054 // absl::ParseCommandLine() requires *argc > 0.
07055 auto positional_args = absl::flags_internal::ParseCommandLineImpl(
07056 *argc, argv, absl::flags_internal::ArgvListAction::kRemoveParsedArgs,
07057 absl::flags_internal::UsageFlagsAction::kHandleUsage,
07058 absl::flags_internal::OnUndefinedFlag::kReportUndefined);
07059 // Any command-line positional arguments not part of any command-line flag
07060 // (or arguments to a flag) are copied back out to argv, with the program
07061 // invocation name at position 0, and argc is resized. This includes
07062 // positional arguments after the flag-terminating delimiter '--'.
07063 // See https://abseil.io/docs/cpp/guides/flags.
07064 std::copy(positional_args.begin(), positional_args.end(), argv);
07065 if (static_cast<int>(positional_args.size()) < *argc) {
07066 argv[positional_args.size()] = nullptr;
07067 *argc = static_cast<int>(positional_args.size());
07068 }
07069 }
07070 #else
07071 ParseGoogleTestFlagsOnlyImpl(argc, argv);
07072 #endif
07073
07074 // Fix the value of *_NSGetArgc() on macOS, but if and only if
07075 // *_NSGetArgv() == argv
07076 // Only applicable to char** version of argv
07077 #if GTEST_OS_MAC
07078 #ifndef GTEST_OS_IOS
07079 if (*_NSGetArgv() == argv) {
07080 *_NSGetArgc() = *argc;
07081 }
07082 #endif
07083 #endif
07084 }
07085
07086 void ParseGoogleTestFlagsOnly(int* argc, wchar_t** argv) {
07087 ParseGoogleTestFlagsOnlyImpl(argc, argv);
07088 }
07089
07090 // The internal implementation of InitGoogleTest().
07091 //
07092 // The type parameter CharType can be instantiated to either char or
07093 // wchar_t.
07094
07095 template <typename CharType>
07096 void InitGoogleTestImpl(int* argc, CharType** argv) {
07097 // We don't want to run the initialization code twice.
07098 if (GTestIsInitialized()) return;
07099
07100 if (*argc <= 0) return;
07101
07102 g_argvs.clear();
07103 for (int i = 0; i != *argc; i++) {
07104 g_argvs.push_back(StreamableToString(argv[i]));
07105 }
07106
07107 #if GTEST_HAS_ABSL
07108 absl::InitializeSymbolizer(g_argvs[0].c_str());
07109
07110 // When using the Abseil Flags library, set the program usage message to the
07111 // help message, but remove the color-encoding from the message first.
07112 absl::SetProgramUsageMessage(absl::StrReplaceAll(
07113 kColorEncodedHelpMessage, {
07114 {"@D", ""},
07115 {"@R", ""},
07116 {"@G", ""},
07117 {"@Y", ""},
07118 {"@@", "@"}
07119 }));
07120 #endif // GTEST_HAS_ABSL
07121
07122 ParseGoogleTestFlagsOnly(argc, argv);
07123 GetUnitTestImpl()->PostFlagParsingInit();
07124 }
07125
07126 } // namespace internal

```

```

07127
07128 // Initializes Google Test. This must be called before calling
07129 // RUN_ALL_TESTS(). In particular, it parses a command line for the
07130 // flags that Google Test recognizes. Whenever a Google Test flag is
07131 // seen, it is removed from argv, and *argc is decremented.
07132 //
07133 // No value is returned. Instead, the Google Test flag variables are
07134 // updated.
07135 //
07136 // Calling the function for the second time has no user-visible effect.
07137
07138 void InitGoogleTest(int* argc, char** argv) {
07139 #if defined(GTEST_CUSTOM_INIT_GOOGLE_TEST_FUNCTION_)
07140 GTEST_CUSTOM_INIT_GOOGLE_TEST_FUNCTION_(argc, argv);
07141 #else // defined(GTEST_CUSTOM_INIT_GOOGLE_TEST_FUNCTION_)
07142 internal::InitGoogleTestImpl(argc, argv);
07143 #endif // defined(GTEST_CUSTOM_INIT_GOOGLE_TEST_FUNCTION_)
07144 }
07145
07146 // This overloaded version can be used in Windows programs compiled in
07147 // UNICODE mode.
07148
07149 void InitGoogleTest(int* argc, wchar_t** argv) {
07150 #if defined(GTEST_CUSTOM_INIT_GOOGLE_TEST_FUNCTION_)
07151 GTEST_CUSTOM_INIT_GOOGLE_TEST_FUNCTION_(argc, argv);
07152 #else // defined(GTEST_CUSTOM_INIT_GOOGLE_TEST_FUNCTION_)
07153 internal::InitGoogleTestImpl(argc, argv);
07154 #endif // defined(GTEST_CUSTOM_INIT_GOOGLE_TEST_FUNCTION_)
07155 }
07156
07157 // This overloaded version can be used on Arduino/embedded platforms where
07158 // there is no argc/argv.
07159
07160 void InitGoogleTest() {
07161 // Since Arduino doesn't have a command line, fake out the argc/argv arguments
07162 int argc = 1;
07163 const auto arg0 = "dummy";
07164 char* argv0 = const_cast<char*>(arg0);
07165 char** argv = &argv0;
07166
07167 #if defined(GTEST_CUSTOM_INIT_GOOGLE_TEST_FUNCTION_)
07168 GTEST_CUSTOM_INIT_GOOGLE_TEST_FUNCTION_(&argc, argv);
07169 #else // defined(GTEST_CUSTOM_INIT_GOOGLE_TEST_FUNCTION_)
07170 internal::InitGoogleTestImpl(&argc, argv);
07171 #endif // defined(GTEST_CUSTOM_INIT_GOOGLE_TEST_FUNCTION_)
07172 }
07173
07174 #if !defined(GTEST_CUSTOM_TMPDIR_FUNCTION_)
07175 // Return value of first environment variable that is set and contains
07176 // a non-empty string. If there are none, return the "fallback" string.
07177 // Since we like the temporary directory to have a directory separator suffix,
07178 // add it if not provided in the environment variable value.
07179
07180 static std::string GetTempDirFromEnv(
07181 std::initializer_list<const char*> environment_variables,
07182 const char* fallback, char separator) {
07183 for (const char* variable_name : environment_variables) {
07184 const char* value = internal::posix::GetEnv(variable_name);
07185 if (value != nullptr && value[0] != '\0') {
07186 if (value[strlen(value) - 1] != separator) {
07187 return std::string(value).append(1, separator);
07188 }
07189 return value;
07190 }
07191 }
07192 return fallback;
07193 }
07194 #endif
07195
07196 std::string TempDir() {
07197 #if defined(GTEST_CUSTOM_TMPDIR_FUNCTION_)
07198 return GTEST_CUSTOM_TMPDIR_FUNCTION_();
07199 #elif GTEST_OS_WINDOWS || GTEST_OS_WINDOWS_MOBILE
07200 return GetTempDirFromEnv({"TEST_TMPDIR", "TEMP"}, "\\temp\\", '\\');
07201 #elif GTEST_OS_LINUX_ANDROID
07202 return GetTempDirFromEnv({"TEST_TMPDIR", "TMPDIR"}, "/data/local/tmp/", '/');
07203 #else
07204 return GetTempDirFromEnv({"TEST_TMPDIR", "TMPDIR"}, "/tmp/", '/');
07205 #endif
07206 }
07207
07208 // Class ScopedTrace
07209
07210 // Pushes the given source file location and message onto a per-thread
07211 // trace stack maintained by Google Test.
07212
07213 void ScopedTrace::PushTrace(const char* file, int line, std::string message) {

```

```

07214 internal::TraceInfo trace;
07215 trace.file = file;
07216 trace.line = line;
07217 trace.message.swap(message);
07218
07219 UnitTest::GetInstance()->PushGTestTrace(trace);
07220 }
07221
07222 // Pops the info pushed by the c'tor.
07223
07224 ScopedTrace::~~ScopedTrace() GTEST_LOCK_EXCLUDED_(&UnitTest::mutex_) {
07225 UnitTest::GetInstance()->PopGTestTrace();
07226 }
07227
07228 } // namespace testing

```

## 16.264 ConnectionManager.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: ConnectionManager.h
00009 * Author: rlcancian
00010 *
00011 * Created on 1 de Julho de 2019, 18:39
00012 */
00013
00014 #ifndef CONNECTIONMANAGER_H
00015 #define CONNECTIONMANAGER_H
00016
00017 #include <utility>
00018 #include <map>
00019 #include "../util/List.h"
00020
00021 //namespace GenesysKernel {
00022
00023 class ModelComponent;
00024
00025 //typedef unsigned int ConnectionPort;
00026 struct ConnectionChannel { /*< actually a port is only a uint, but it could be more complex, including
00027 type, presence or absence of data, etc */
00028 unsigned int portNumber = 0;
00029 std::string portDescription = "";
00030 };
00031
00032 struct Connection {
00033 ModelComponent* component;
00034 ConnectionChannel channel;
00035 };
00036
00037 class ConnectionManager {
00038 public:
00039 ConnectionManager();
00040 virtual ~ConnectionManager();
00041 public:
00042 unsigned int size();
00043 Connection* getFrontConnection();
00044 Connection* getConnectionAtPort(unsigned int rank);
00045 void insert(ModelComponent* component, unsigned int inputPortNumber = 0);
00046 void insert(Connection* connection);
00047 void insertAtPort(unsigned int port, Connection* connection);
00048 void remove(Connection* connection);
00049 void removeAtPort(unsigned int port);
00050 std::map<unsigned int, Connection*> connections() const;
00051 //void setCurrentOutputConnections(unsigned int _currentOutputConnections);
00052 unsigned int getCurrentOutputConnectionsSize() const;
00053 void setMaxOutputConnections(unsigned int _maxOutputConnections);
00054 unsigned int getMaxOutputConnections() const;
00055 void setMinOutputConnections(unsigned int _minOutputConnections);
00056 unsigned int getMinOutputConnections() const;
00057 //void setCurrentInputConnections(unsigned int _currentInputConnections);
00058 unsigned int getCurrentInputConnectionsSize() const;
00059 void setMaxInputConnections(unsigned int _maxInputConnections);
00060 unsigned int getMaxInputConnections() const;
00061 void setMinInputConnections(unsigned int _minInputConnections);
00062 unsigned int getMinInputConnections() const;
00063 private:
00064 std::map<unsigned int, Connection*> _nextConnections = new std::map<unsigned int, Connection*>();
00065 unsigned int _minInputConnections = 0;
00066 unsigned int _maxInputConnections = 0;

```

```

00168 unsigned int _currentInputConnections = 0;
00169 unsigned int _minOutputConnections = 0;
00170 unsigned int _maxOutputConnections = 0;
00171 };
00172 //namespace\\}
00173 #endif /* CONNECTIONMANAGER_H */

```

## 16.265 DefineGetterSetter.h

```

00001 #ifndef DEFINEGETTERSETTER_H
00002 #define DEFINEGETTERSETTER_H
00003
00004 #endif

```

## 16.266 Attribute.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Attribute.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 25 de Setembro de 2018, 16:37
00012 */
00013
00014 #ifndef ATTRIBUTE_H
00015 #define ATTRIBUTE_H
00016
00017 #include <string>
00018 #include <list>
00019 #include <map>
00020 #include <utility>
00021 #include <vector>
00022 #include "../model/ModelDataDefinition.h"
00023 #include "../model/ModelDataManager.h"
00024 #include "../PluginInformation.h"
00025 #include "SparseValueStore.h"
00026
00027 //namespace GenesysKernel {
00028
00067 class Attribute : public ModelDataDefinition {
00068 public:
00070 Attribute(Model* model, std::string name = "", std::string dataDefinitionTypeName =
 Util::TypeOf<Attribute>());
00072 virtual ~Attribute() override;
00073 public:
00074 virtual std::string show() override;
00075 public: // public static methods
00076 static PluginInformation* GetPluginInformation();
00077 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00078 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00079 public:
00081 virtual std::string getInitialValuesText() const;
00083 virtual void setInitialValuesText(std::string valuesText);
00085 double getInitialValue(std::string index = "");
00087 void setInitialValue(double value, std::string index = "");
00089 void setInitialValues(const std::vector<std::pair<std::string,double>> values);
00091 void copyInitialValuesFrom(const Attribute& source);
00093 void insertDimentionSize(unsigned int size);
00095 std::list<unsigned int>* getDimensionSizes() const;
00097 std::map<std::string, double>* getInitialValues() const;
00099 SparseValueStore* getInitialValueStore();
00101 bool isInitialValuesTextValid() const;
00102 protected:
00103 virtual bool _loadInstance(PersistenceRecord *fields) override;
00104 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00105 protected:
00106 virtual bool _check(std::string& errorMessage) override;
00107 private:
00108 void _syncInitialValuesTextFromStore();
00109 SparseValueStore* _initialValues = new SparseValueStore();
00110 std::string _initialValuesText = "0";
00111 bool _initialValuesTextValid = true;
00112 std::string _initialValuesTextErrorMessage;
00113 };
00114 //namespace\\}
00115 #endif /* ATTRIBUTE_H */

```

## 16.267 Counter.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: CounterDefaultImpl1.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 29 de Maio de 2019, 11:24
00012 */
00013
00014 #ifndef COUNTERDEFAULTIMPL1_H
00015 #define COUNTERDEFAULTIMPL1_H
00016
00017 #include "../model/ModelDataDefinition.h"
00018 #include "../model/ModelDataManager.h"
00019 #include "../Plugin.h"
00020 //namespace GenesysKernel {
00021
00022 class Counter : public ModelDataDefinition {
00023 public:
00024 Counter(Model* model, std::string name = "", ModelDataDefinition* parent = nullptr);
00025 virtual ~Counter() override;
00026 public:
00027 virtual std::string show() override;
00028 public: // public static methods
00029 static PluginInformation* GetPluginInformation();
00030 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00031 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00032 public:
00033 void clear();
00034 void incCountValue(/*int*/double value = 1.0);
00035 double /*unsigned long*/ getCountValue() const;
00036 ModelDataDefinition* getParent() const;
00037 protected:
00038 virtual bool _loadInstance(PersistenceRecord *fields) override;
00039 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00040 protected:
00041 virtual bool _check(std::string& errorMessage) override;
00042 virtual void _initBetweenReplications() override;
00043 private:
00044 ModelDataDefinition* _parent;
00045 double /*unsigned long*/ _count = 0;
00046 SimulationResponse* _countValueResponse = nullptr;
00047 };
00048 //namespace\}
00049 #endif /* COUNTERDEFAULTIMPL1_H */

```

## 16.268 Entity.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Entity.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 21 de Junho de 2018, 16:30
00012 */
00013
00014 #ifndef ENTITY_H
00015 #define ENTITY_H
00016
00017 #include <string>
00018 #include <map>
00019
00020 #include "../util/Util.h"
00021 #include "../util/List.h"
00022 #include "../model/ModelDataDefinition.h"
00023 #include "EntityType.h"
00024 #include "SparseValueStore.h"
00025 //namespace GenesysKernel {
00026
00027 class Entity : public ModelDataDefinition {
00028 private: // no one can create or destroy entities directly. This can be done one through friend class
00029 Model
00030 Entity(Model* model, std::string name = "", bool insertIntoModel = true);

```

```

00081 virtual ~Entity();
00082 // friend Entity* Model::createEntity(std::string name, bool insertIntoModel); // It would be
 better, but Model is not known at this point of compilation
00083 friend class Model;
00084 public:
00085 virtual std::string show() override;
00086
00087 public: // g & s
00093 void setEntityTypeName(std::string entityTypeName); /*!< indirect access to EntityType
00098 std::string getEntityTypeName() const;
00104 void setEntityType(EntityType* entityType); /*!< direct access to EntityType
00109 EntityType* getEntityType() const;
00110 public:
00117 double getAttributeValue(std::string attributeName, std::string index="");
00124 double getAttributeValue(Util::identification attributeID, std::string index="");
00133 void setAttributeValue(std::string attributeName, double value, std::string index="", bool
 createIfNotFound=false);
00141 void setAttributeValue(Util::identification attributeID, double value, std::string index="");
00143 void copyAttributeValues(std::string destinationAttributeName, std::string sourceAttributeName);
00148 Util::identification entityNumber() const;
00149 protected:
00150 virtual bool _loadInstance(PersistenceRecord *fields) override;
00151 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00152 virtual bool _check(std::string& errorMessage) override;
00153 private:
00155 SparseValueStore* _ensureAttributeStore(unsigned int rank);
00156
00157 Util::identification _entityNumber;
00158 EntityType* _entityType = nullptr;
00159 List<SparseValueStore*>* _attributeValues = new List<SparseValueStore*>();
00160 };
00161 //namespace\\
00162 #endif /* ENTITY_H */

```

## 16.269 EntityType.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: EntityType.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 14 de Agosto de 2018, 19:24
00012 */
00013
00014 #ifndef ENTITYPE_H
00015 #define ENTITYPE_H
00016
00017 #include <string>
00018 #include "../model/ModelDataDefinition.h"
00019 #include "StatisticsCollector.h"
00020 #include "../model/ModelDataManager.h"
00021 #include "../Plugin.h"
00022
00023 // #include "Model.h"
00024 // namespace GenesysKernel {
00025
00026 class EntityType : public ModelDataDefinition {
00027 public:
00031 EntityType(Model* model, std::string name = "");
00032 virtual ~EntityType();
00033 public:
00034 virtual std::string show() override;
00035 public: //static
00036 static PluginInformation* GetPluginInformation();
00037 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00038 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00039 public: //get & set
00040 // @ToDo: (importante): Change categories Waiting, Others, Transfer, etc to customizable ones
00041 void setInitialWaitingCost(double _initialWaitingCost);
00042 double initialWaitingCost() const;
00043 void setInitialOtherCost(double _initialOtherCost);
00044 double initialOtherCost() const;
00045 void setInitialNVACost(double _initialNVACost);
00046 double initialNVACost() const;
00047 void setInitialVACost(double _initialVACost);
00048 double initialVACost() const;
00049 void setInitialPicture(std::string _initialPicture);
00050 std::string initialPicture() const;
00051 public: //get
00057 StatisticsCollector* addGetStatisticsCollector(std::string name);

```

```

00058
00059 protected: // must be overridden
00060 virtual bool _loadInstance(PersistenceRecord *fields) override;
00061 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00062 protected: // could be overridden
00063 //virtual bool _check(std::string& errorMessage) override;
00064 virtual void _initBetweenReplications() override;
00065 virtual void _createInternalStatisticReporters() override;
00066 private:
00067 void _initCostsAndStatistics();
00068 private:
00069
00070 const struct DEFAULT_VALUES {
00071 const std::string initialPicture = "report";
00072 const double initialCost = 0.0;
00073 } DEFAULT;
00074 std::string _initialPicture = DEFAULT.initialPicture;
00075 double _initialVACost = DEFAULT.initialCost;
00076 double _initialNVACost = DEFAULT.initialCost;
00077 double _initialOtherCost = DEFAULT.initialCost;
00078 double _initialWaitingCost = DEFAULT.initialCost;
00079 private: //l:n
00080 List<StatisticsCollector*> _statisticsCollectors = new List<StatisticsCollector*>();
00081 };
00082 //namespace\\
00083 #endif /* ENTITYTYPE_H */

```

## 16.270 SparseValueStore.h

```

00001 /*
00002 * File: SparseValueStore.h
00003 * Author: rafael.luiz.cancian
00004 *
00005 * Created on 16 de Abril de 2026
00006 */
00007
00008 #ifndef SPARSEVALUESTORE_H
00009 #define SPARSEVALUESTORE_H
00010
00011 #include "../Persistence.h"
00012 #include "../../util/Util.h"
00013
00014 #include <list>
00015 #include <map>
00016 #include <algorithm>
00017 #include <cctype>
00018 #include <sstream>
00019 #include <string>
00020 #include <utility>
00021 #include <vector>
00022
00023 class SparseValueStore {
00024 public:
00025 static std::string makeIndexKey(const std::vector<unsigned int>& indexes);
00026
00027 static std::string makeIndexKeyFromDoubles(const std::vector<double>& indexes);
00028
00029 static std::string appendIndexKey(const std::string& currentKey, unsigned int index);
00030
00031 static std::string appendIndexKeyFromDouble(const std::string& currentKey, double index);
00032
00033 static std::string normalizeIndexKey(const std::string& index);
00034
00035 double value(const std::string& index = "") const;
00036
00037 void setValue(double value, const std::string& index = "");
00038
00039 void setValues(const std::map<std::string, double>& values);
00040
00041 void copyFrom(const SparseValueStore& other);
00042
00043 void insertDimensionSize(unsigned int size);
00044
00045 std::list<unsigned int>* dimensionSizes();
00046
00047 const std::list<unsigned int>* dimensionSizes() const;
00048
00049 std::map<std::string, double>* values();
00050
00051 const std::map<std::string, double>* values() const;
00052
00053 void clear();
00054
00055 void loadDimensions(PersistenceRecord* fields);
00056

```

```

00083 void saveDimensions(PersistenceRecord* fields, bool saveDefaultValues) const;
00084
00085 void loadValues(PersistenceRecord* fields,
00086 const std::string& countField,
00087 const std::string& positionPrefix,
00088 const std::string& valuePrefix);
00089
00090 void saveValues(PersistenceRecord* fields,
00091 const std::string& countField,
00092 const std::string& positionPrefix,
00093 const std::string& valuePrefix,
00094 bool saveDefaultValues) const;
00095
00096 std::string showValues() const;
00097
00098 private:
00099 static std::string trimCopy(const std::string& text) {
00100 std::size_t first = 0;
00101 while (first < text.size() && std::isspace(static_cast<unsigned char>(text[first]))) {
00102 ++first;
00103 }
00104 std::size_t last = text.size();
00105 while (last > first && std::isspace(static_cast<unsigned char>(text[last - 1]))) {
00106 --last;
00107 }
00108 return text.substr(first, last - first);
00109 }
00110
00111 // Returns true when every comma-separated token in index consists of digits only
00112 // (or when index is empty, representing the scalar position).
00113 static bool isNumericKey(const std::string& index) {
00114 if (index.empty()) {
00115 return true;
00116 }
00117 for (char c : index) {
00118 if (!std::isdigit(static_cast<unsigned char>(c)) && c != ',' && c != ' ' && c != '\t') {
00119 return false;
00120 }
00121 }
00122 return true;
00123 }
00124
00125 static std::vector<unsigned int> parseIndex(const std::string& index) {
00126 std::vector<unsigned int> parsed;
00127 if (index.empty()) {
00128 return parsed;
00129 }
00130 std::stringstream input(index);
00131 std::string token;
00132 while (std::getline(input, token, ',')) {
00133 token = trimCopy(token);
00134 if (token.empty()) {
00135 continue;
00136 }
00137 parsed.push_back(static_cast<unsigned int>(std::stoul(token)));
00138 }
00139 return parsed;
00140 }
00141
00142 static std::size_t productOfDimensions(const std::list<unsigned int>& dimensions) {
00143 std::size_t product = 1u;
00144 for (unsigned int dimension : dimensions) {
00145 product *= static_cast<std::size_t>(dimension);
00146 }
00147 return product;
00148 }
00149
00150 static std::vector<std::size_t> stridesFor(const std::list<unsigned int>& dimensions) {
00151 std::vector<std::size_t> strides;
00152 strides.reserve(dimensions.size());
00153 std::size_t stride = 1u;
00154 for (auto it = dimensions.rbegin(); it != dimensions.rend(); ++it) {
00155 strides.push_back(stride);
00156 stride *= static_cast<std::size_t>(*it);
00157 }
00158 std::reverse(strides.begin(), strides.end());
00159 return strides;
00160 }
00161
00162 std::size_t linearIndex(const std::vector<unsigned int>& indexes) const {
00163 if (_dimensionSizes.empty()) {
00164 return indexes.empty() ? 0u : static_cast<std::size_t>(-1);
00165 }
00166 if (indexes.size() != _dimensionSizes.size()) {
00167 return static_cast<std::size_t>(-1);
00168 }
00169 const std::vector<std::size_t> strides = stridesFor(_dimensionSizes);

```



```

00173 std::size_t linear = 0u;
00174 auto dimIt = _dimensionSizes.begin();
00175 for (std::size_t i = 0; i < indexes.size(); ++i, ++dimIt) {
00176 if (indexes[i] >= *dimIt) {
00177 return static_cast<std::size_t>(-1);
00178 }
00179 linear += static_cast<std::size_t>(indexes[i]) * strides[i];
00180 }
00181 return linear;
00182 }
00183
00184 void rebuildDenseFromCache() {
00185 const std::size_t size = _dimensionSizes.empty() ? 1u : productOfDimensions(_dimensionSizes);
00186 _denseValues.assign(size, 0.0);
00187 for (const auto& entry : _values) {
00188 if (entry.first.empty()) {
00189 continue;
00190 }
00191 // Skip non-numeric string keys -- they live only in the sparse map.
00192 if (!isNumericKey(entry.first)) {
00193 continue;
00194 }
00195 const std::vector<unsigned int> indexes = parseIndex(entry.first);
00196 const std::size_t linear = linearIndex(indexes);
00197 if (linear != static_cast<std::size_t>(-1) && linear < _denseValues.size()) {
00198 _denseValues[linear] = entry.second;
00199 }
00200 }
00201 _denseActive = true;
00202 }
00203
00204 void syncCacheFromDense() {
00205 // Preserve non-numeric string keys across the rebuild.
00206 std::map<std::string, double> stringKeyed;
00207 for (const auto& entry : _values) {
00208 if (!entry.first.empty() && !isNumericKey(entry.first)) {
00209 stringKeyed.insert(entry);
00210 }
00211 }
00212 _values.clear();
00213 if (_hasScalarValue) {
00214 _values[""] = _scalarValue;
00215 }
00216 if (!_denseActive) {
00217 _values.insert(stringKeyed.begin(), stringKeyed.end());
00218 return;
00219 }
00220 if (_dimensionSizes.empty()) {
00221 if (!_denseValues.empty() && !_hasScalarValue) {
00222 _values[""] = _denseValues.front();
00223 }
00224 _values.insert(stringKeyed.begin(), stringKeyed.end());
00225 return;
00226 }
00227 std::vector<unsigned int> indexes;
00228 indexes.reserve(_dimensionSizes.size());
00229 const std::vector<std::size_t> strides = stridesFor(_dimensionSizes);
00230 for (std::size_t linear = 0; linear < _denseValues.size(); ++linear) {
00231 if (_denseValues[linear] == 0.0) {
00232 continue;
00233 }
00234 std::size_t remainder = linear;
00235 indexes.clear();
00236 for (std::size_t i = 0; i < strides.size(); ++i) {
00237 const unsigned int index = static_cast<unsigned int>(remainder / strides[i]);
00238 indexes.push_back(index);
00239 remainder %= strides[i];
00240 }
00241 _values[makeIndexKey(indexes)] = _denseValues[linear];
00242 }
00243 _values.insert(stringKeyed.begin(), stringKeyed.end());
00244 }
00245
00246 void ensureDenseForIndex(const std::vector<unsigned int>& indexes) {
00247 if (indexes.empty() && _dimensionSizes.empty()) {
00248 if (!_denseActive) {
00249 _denseActive = true;
00250 _denseValues.assign(1u, 0.0);
00251 }
00252 return;
00253 }
00254
00255 if (_dimensionSizes.empty()) {
00256 _dimensionSizes.clear();
00257 for (unsigned int index : indexes) {
00258 _dimensionSizes.push_back(index + 1u);
00259 }

```

```

00260 rebuildDenseFromCache();
00261 return;
00262 }
00263
00264 if (indexes.size() != _dimensionSizes.size()) {
00265 return;
00266 }
00267
00268 bool needsResize = false;
00269 auto dimIt = _dimensionSizes.begin();
00270 for (std::size_t i = 0; i < indexes.size(); ++i, ++dimIt) {
00271 if (indexes[i] >= *dimIt) {
00272 *dimIt = indexes[i] + 1u;
00273 needsResize = true;
00274 }
00275 }
00276 if (needsResize || !_denseActive) {
00277 rebuildDenseFromCache();
00278 }
00279 }
00280
00281 bool _denseActive = false;
00282 bool _hasScalarValue = false;
00283 double _scalarValue = 0.0;
00284 std::vector<double> _denseValues;
00285 std::list<unsigned int> _dimensionSizes;
00286 std::map<std::string, double> _values;
00287 };
00288
00289 inline std::string SparseValueStore::makeIndexKey(const std::vector<unsigned int>& indexes) {
00290 std::ostringstream key;
00291 bool first = true;
00292 for (const unsigned int index : indexes) {
00293 if (!first) {
00294 key << ",";
00295 }
00296 first = false;
00297 key << index;
00298 }
00299 return key.str();
00300 }
00301
00302 inline std::string SparseValueStore::makeIndexKeyFromDoubles(const std::vector<double>& indexes) {
00303 std::vector<unsigned int> unsignedIndexes;
00304 unsignedIndexes.reserve(indexes.size());
00305 for (const double index : indexes) {
00306 unsignedIndexes.push_back(static_cast<unsigned int>(index));
00307 }
00308 return makeIndexKey(unsignedIndexes);
00309 }
00310
00311 inline std::string SparseValueStore::appendIndexKey(const std::string& currentKey, unsigned int index)
00312 {
00313 const std::string nextPart = std::to_string(index);
00314 if (currentKey.empty()) {
00315 return nextPart;
00316 }
00317 return currentKey + "," + nextPart;
00318 }
00319
00320 inline std::string SparseValueStore::appendIndexKeyFromDouble(const std::string& currentKey, double
00321 index) {
00322 return appendIndexKey(currentKey, static_cast<unsigned int>(index));
00323 }
00324
00325 inline std::string SparseValueStore::normalizeIndexKey(const std::string& index) {
00326 return index;
00327 }
00328
00329 inline double SparseValueStore::value(const std::string& index) const {
00330 if (index.empty() && !_hasScalarValue) {
00331 return _scalarValue;
00332 }
00333 // Non-numeric string keys (e.g. "slot", "pi") bypass dense storage.
00334 if (!index.empty() && !isNumericKey(index)) {
00335 const auto it = _values.find(index);
00336 return (it != _values.end()) ? it->second : 0.0;
00337 }
00338 if (_denseActive) {
00339 const std::vector<unsigned int> indexes = parseIndex(index);
00340 const std::size_t linear = linearIndex(indexes);
00341 if (linear == static_cast<std::size_t>(-1) || linear >= _denseValues.size()) {
00342 // Dimension count mismatch or out of bounds -- fall back to sparse map.
00343 const auto it = _values.find(normalizeIndexKey(index));
00344 return (it != _values.end()) ? it->second : 0.0;
00345 }
00346 return _denseValues[linear];
00347 }

```

```

00345 }
00346 const auto it = _values.find(normalizeIndexKey(index));
00347 if (it == _values.end()) {
00348 return 0.0;
00349 }
00350 return it->second;
00351 }
00352
00353 inline void SparseValueStore::setValue(double value, const std::string& index) {
00354 if (index.empty()) {
00355 _scalarValue = value;
00356 _hasScalarValue = true;
00357 _values[""] = value;
00358 if (_denseActive && _dimensionSizes.empty()) {
00359 if (_denseValues.empty()) {
00360 _denseValues.assign(1u, 0.0);
00361 }
00362 _denseValues.front() = value;
00363 }
00364 return;
00365 }
00366 // Non-numeric string keys (e.g. "slot", "pi") are stored directly in the
00367 // sparse map without touching the dense storage or dimension metadata.
00368 if (!isNumericKey(index)) {
00369 _values[index] = value;
00370 return;
00371 }
00372 const std::vector<unsigned int> indexes = parseIndex(index);
00373 ensureDenseForIndex(indexes);
00374 if (_denseActive) {
00375 const std::size_t linear = linearIndex(indexes);
00376 if (linear != static_cast<std::size_t>(-1)) {
00377 if (linear >= _denseValues.size()) {
00378 _denseValues.resize(linear + 1u, 0.0);
00379 }
00380 _denseValues[linear] = value;
00381 syncCacheFromDense();
00382 return;
00383 }
00384 }
00385 _values[normalizeIndexKey(index)] = value;
00386 }
00387
00388 inline void SparseValueStore::setValues(const std::map<std::string, double>& values) {
00389 _values = values;
00390 const auto scalarIt = _values.find("");
00391 _hasScalarValue = scalarIt != _values.end();
00392 if (_hasScalarValue) {
00393 _scalarValue = scalarIt->second;
00394 }
00395 if (_dimensionSizes.empty()) {
00396 if (_values.size() == 1u && _hasScalarValue) {
00397 _denseActive = true;
00398 _denseValues.assign(1u, _scalarValue);
00399 } else {
00400 _denseActive = false;
00401 _denseValues.clear();
00402 }
00403 return;
00404 }
00405 rebuildDenseFromCache();
00406 }
00407
00408 inline void SparseValueStore::copyFrom(const SparseValueStore& other) {
00409 *this = other;
00410 }
00411
00412 inline void SparseValueStore::insertDimensionSize(unsigned int size) {
00413 _dimensionSizes.insert(_dimensionSizes.end(), size);
00414 rebuildDenseFromCache();
00415 }
00416
00417 inline std::list<unsigned int>* SparseValueStore::dimensionSizes() {
00418 return &_dimensionSizes;
00419 }
00420
00421 inline const std::list<unsigned int>* SparseValueStore::dimensionSizes() const {
00422 return &_dimensionSizes;
00423 }
00424
00425 inline std::map<std::string, double>* SparseValueStore::values() {
00426 return &_values;
00427 }
00428
00429 inline const std::map<std::string, double>* SparseValueStore::values() const {
00430 return &_values;
00431 }

```

```

00432
00433 inline void SparseValueStore::clear() {
00434 _dimensionSizes.clear();
00435 _values.clear();
00436 _hasScalarValue = false;
00437 _scalarValue = 0.0;
00438 _denseValues.clear();
00439 _denseActive = false;
00440 }
00441
00442 inline void SparseValueStore::loadDimensions(PersistenceRecord* fields) {
00443 _dimensionSizes.clear();
00444 if (fields == nullptr) {
00445 return;
00446 }
00447 const unsigned int count = fields->loadField("dimensions", 0u);
00448 for (unsigned int i = 0; i < count; i++) {
00449 insertDimensionSize(fields->loadField("dimension" + Util::StrIndex(i), 0u));
00450 }
00451 if (!_dimensionSizes.empty()) {
00452 rebuildDenseFromCache();
00453 }
00454 }
00455
00456 inline void SparseValueStore::saveDimensions(PersistenceRecord* fields, bool saveDefaultValues) const
00457 {
00458 if (fields == nullptr) {
00459 return;
00460 }
00461 unsigned int i = 0;
00462 fields->saveField("dimensions", static_cast<unsigned int>(_dimensionSizes.size()), 0u,
00463 saveDefaultValues);
00464 for (const unsigned int dimension : _dimensionSizes) {
00465 fields->saveField("dimension" + Util::StrIndex(i), dimension, 1u, saveDefaultValues);
00466 i++;
00467 }
00468 }
00469
00470 inline void SparseValueStore::loadValues(PersistenceRecord* fields,
00471 const std::string& countField,
00472 const std::string& positionPrefix,
00473 const std::string& valuePrefix) {
00474 _values.clear();
00475 if (fields == nullptr) {
00476 return;
00477 }
00478 const unsigned int count = fields->loadField(countField, 0u);
00479 for (unsigned int i = 0; i < count; i++) {
00480 const std::string position = fields->loadField(positionPrefix + Util::StrIndex(i), "");
00481 const double loadedValue = fields->loadField(valuePrefix + Util::StrIndex(i), 0.0);
00482 setValue(loadedValue, position);
00483 }
00484 }
00485
00486 inline void SparseValueStore::saveValues(PersistenceRecord* fields,
00487 const std::string& countField,
00488 const std::string& positionPrefix,
00489 const std::string& valuePrefix,
00490 bool saveDefaultValues) const {
00491 if (fields == nullptr) {
00492 return;
00493 }
00494 unsigned int i = 0;
00495 fields->saveField(countField, static_cast<unsigned int>(_values.size()), 0u, saveDefaultValues);
00496 for (const auto& value : _values) {
00497 fields->saveField(positionPrefix + Util::StrIndex(i), value.first, "0", saveDefaultValues);
00498 fields->saveField(valuePrefix + Util::StrIndex(i), value.second, 0.0, saveDefaultValues);
00499 i++;
00500 }
00501 }
00502
00503 inline std::string SparseValueStore::showValues() const {
00504 if (_denseActive && _values.empty() && !_denseValues.empty() && _dimensionSizes.empty()) {
00505 return "[" + std::to_string(_denseValues.front()) + "]";
00506 }
00507 std::string text = "{";
00508 for (const auto& value : _values) {
00509 text += value.first + "=" + Util::StrTruncIfInt(std::to_string(value.second)) + ", ";
00510 }
00511 if (!_values.empty()) {
00512 text = text.substr(0, text.length() - 2);
00513 }
00514 text += "}";
00515 return text;
00516 }
00517
00518 #endif /* SPARSEVALUESTORE_H */

```

## 16.271 StatisticsCollector.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: StatisticsCollector.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 30 de Agosto de 2018, 17:24
00012 */
00013
00014 #ifndef STATISTICSCOLLECTOR_H
00015 #define STATISTICSCOLLECTOR_H
00016
00017 #include "../model/ModelDataDefinition.h"
00018 #include "../statistics/Statistics_if.h"
00019 #include "../model/ModelDataManager.h"
00020 #include "../Plugin.h"
00021
00022 #include <vector>
00023
00024 //namespace GenesysKernel {
00025
00026 class StatisticsCollector : public ModelDataDefinition { //, public Statistics_if {
00027 public:
00028 StatisticsCollector(Model* model, std::string name = "", ModelDataDefinition* parent = nullptr,
00029 bool insertIntoModel = true);
00030 virtual ~StatisticsCollector() override;
00031 public:
00032 virtual std::string show() override;
00033 public:
00034 ModelDataDefinition* getParent() const;
00035 Statistics_if* getStatistics() const;
00036 public:
00037 static PluginInformation* GetPluginInformation();
00038 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00039 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00040 protected:
00041 virtual bool _loadInstance(PersistenceRecord *fields) override;
00042 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00043 protected:
00044 virtual bool _check(std::string& errorMessage) override;
00045 private:
00046 void _initStaticsAndCollector();
00047 void _addOwnedResponse(SimulationResponse* response);
00048 private:
00049 ModelDataDefinition* _parent;
00050 Statistics_if* _statistics = nullptr;
00051 std::vector<SimulationResponse*> _ownedResponses;
00052 };
00053 //namespace}
00054 #endif /* STATISTICSCOLLECTOR_H */

```

## 16.272 Event.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Event.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 21 de Junho de 2018, 19:41
00012 */
00013
00014 #ifndef EVENT_H
00015 #define EVENT_H
00016
00017 #include <string>
00018 #include <functional>
00019
00020 #include "model/ModelDataDefinition.h"
00021 #include "essentialPlugins/Entity.h"
00022 #include "model/ModelComponent.h"

```

```

00023
00024 //namespace GenesysKernel {
00025
00030 class Event {//: public ModelDataDefinition {
00031 public:
00032 Event(double time, Entity* entity, ModelComponent* component, unsigned int
componentinputPortNumber = 0);
00033 Event(double time, Entity* entity, Connection* connection);
00034 virtual ~Event() = default;
00035 public:
00040 double getTime() const;
00045 ModelComponent* getComponent() const;
00050 Entity* getEntity() const;
00055 unsigned int getComponentinputPortNumber() const;
00060 void setEntity(Entity* _entity);
00061 public: // virtual
00062 virtual std::string show();
00063 protected:
00064 double _time;
00065 private:
00066 Entity* _entity;
00067 ModelComponent* _component;
00068 unsigned int _componentinputPortNumber;
00069 };
00070
00071 //***** 2022/07/05
00072
00073 typedef std::function<void(void*) > InternalEventHandler;
00074
00075 class InternalEvent: public Event { // @ToDo: (importante): BAD SW ENGINEERING Fix it
00076 public:
00077 InternalEvent(double time, std::string description): Event(time, nullptr, nullptr, 0){
00078 _description = description;
00079 _time = time;
00080 }
00081 virtual ~InternalEvent() = default;
00082
00083 template<typename Class>
00084 void setEventHandler(Class* object, void (Class::function)(void*), void* parameter) {
00085 _object = object;
00086 _handler= std::bind(function, object, std::placeholders::_1);
00087 _parameter = parameter;
00088 }
00089
00090 virtual std::string show() override {
00091 std::string dataDefinitionName = static_cast<ModelDataDefinition*>(_object)->getName();
00092 std::string message = "time=" + std::to_string(_time) + //Util::StrTimeUnit(??)+
00093 ",description=\"\" + _description+\"\"+
00094 ",block=\"\" + dataDefinitionName+\"\";
00095 return message;
00096 }
00097
00098 const InternalEventHandler& eventHandler() const {
00099 return _handler;
00100 }
00101
00102 const std::string& description() const {
00103 return _description;
00104 }
00105
00106 void dispatchEvent() const {
00107 _handler(_parameter);
00108 }
00109
00110 void* object() const {
00111 return _object;
00112 }
00113
00114 void* parameter() const {
00115 return _parameter;
00116 }
00117
00118 private:
00119 InternalEventHandler _handler;
00120 std::string _description;
00121 void* _parameter;
00122 void* _object;
00123 };
00124 };
00125
00126 //namespace\\}
00127 #endif /* EVENT_H */

```

## 16.273 ExperimentManager.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: ExperimentManager.h
00009 * Author: Prof. Rafael Luiz Cancian, Dr. Eng.
00010 *
00011 * Created on 4 de maio de 2021, 11:43
00012 */
00013
00014 #ifndef EXPERIMENTMANAGER_H
00015 #define EXPERIMENTMANAGER_H
00016
00017 // #include "Simulator.h"
00018 #include "Persistence.h"
00019 // #include "SimulationControl.h"
00020 // #include "SimulationResponse.h"
00021 #include "SimulationExperiment.h"
00022 #include "../util/List.h"
00023
00024 class Simulator;
00025
00041 class ExperimentManager { // : PersistentObject_base {
00042 public:
00043
00044 /* class ModelSimulationInsOuts {
00045 public:
00046 ModelSimulationInsOuts() = default;
00047 struct ControlResponseInfos {
00048 std::string type;
00049 std::string parent;
00050 std::string name;
00051 double defaultValue;
00052 };
00053
00054 List<ControlResponseInfos*>* getResponsesInfos() const {
00055 return _responsesInfos;
00056 }
00057
00058 std::string getModelfilename() const {
00059 return _experimentfilename;
00060 }
00061
00062 List<ControlResponseInfos*>* getControlInfos() const {
00063 return _controlInfos;
00064 }
00065 private:
00066 std::string _modelfilename;
00067 List<ControlResponseInfos*>* _responsesInfos = new List<ControlResponseInfos*>();
00068 List<ControlResponseInfos*>* _controlInfos = new List<ControlResponseInfos*>();
00069 }; */
00070
00071 public:
00072 ExperimentManager(Simulator* simulator);
00073 // Release owned experiment instances and container storage during simulator shutdown.
00074 virtual ~ExperimentManager();
00075
00076 public:
00077 SimulationExperiment* newSimulationExperiment();
00078 void insert(SimulationExperiment* experiment);
00079 void remove(SimulationExperiment* experiment);
00080 void setCurrent(SimulationExperiment* experiment);
00081 // @ToDo: (importante): Implement persistence once SimulationExperiment serialization is defined.
00082 bool saveSimulationExperiment(std::string filename);
00083 // @ToDo: (importante): Implement loading once SimulationExperiment deserialization and scenario
 execution are defined.
00084 bool loadSimulationExperiment(std::string filename);
00085 unsigned int size();
00086 public:
00087 SimulationExperiment* front();
00088 SimulationExperiment* current();
00089 SimulationExperiment* next();
00090 //SimulationExperiment* end();
00091 public:
00092 List<SimulationExperiment*>* getExperiments() const;
00093 private:
00094 Simulator* _simulator;
00095 SimulationExperiment* _currentSimulationExperiment;
00096 List<SimulationExperiment*>* _experiments = new List<SimulationExperiment*>();
00097
00098 };
00099

```

```
00100 #endif /* EXPERIMENTMANAGER_H */
```

## 16.274 ExperimentManagerDefaultImpl1.h

```
00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: ProcessAnalyserDefaultImpl1.h
00009 * Author: rlcancian
00010 *
00011 * Created on 20 de Maio de 2019, 20:45
00012 */
00013
00014 #ifndef PROCESSANALYSERDEFAULTIMPL1_H
00015 #define PROCESSANALYSERDEFAULTIMPL1_H
00016
00017 #include "ExperimetManager_if.h"
00018 #include "SimulationScenario.h"
00019
00020 class ExperimentManagerDefaultImpl1 : public ExperimentManager_if {
00021 public:
00022 ExperimentManagerDefaultImpl1();
00023 virtual ~ExperimentManagerDefaultImpl1() = default;
00024 public:
00025 virtual List<SimulationScenario*>* getScenarios() const override;
00026 //virtual List<SimulationControl*>* getControls() const;
00027 //virtual List<SimulationResponse*>* getResponses() const;
00028 //virtual List<SimulationControl*>* extractControlsFromModel(std::string modelName) const;
00029 //virtual List<SimulationResponse*>* extractResponsesFromModel(std::string modelName) const;
00030 virtual void startSimulationOfScenario(SimulationScenario* scenario) override;
00031 virtual void startExperiment() override;
00032 virtual void stopExperiment() override;
00033 virtual void addTraceSimulationHandler(traceSimulationProcessListener
 traceSimulationProcessListener) override;
00034 private:
00035 //List<SimulationControl*>* _controls = new List<SimulationControl*>();
00036 };
00037
00038 #endif /* PROCESSANALYSERDEFAULTIMPL1_H */
```

## 16.275 ExperimetManager\_if.h

```
00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: ExperimentManager_if (old ProcessAnalyser_if.h)
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 10 de Outubro de 2018, 14:26
00012 */
00013
00014 #ifndef EXPERIMENTMANAGER_IF_H
00015 #define EXPERIMENTMANAGER_IF_H
00016
00017 #include "../util/List.h"
00018 #include "SimulationScenario.h"
00019 #include "PropertyGenesys.h"
00020 #include "TraceManager.h"
00021
00022 class ExperimentManager_if {
00023 public:
00024 virtual ~ExperimentManager_if() = default;
00025 virtual List<SimulationScenario*>* getScenarios() const = 0;
00026 //virtual List<SimulationControl*>* getControls() const = 0;
00027 //virtual List<SimulationResponse*>* getResponses() const = 0;
00028 //virtual List<SimulationControl*>* extractControlsFromModel(std::string modelName) const = 0;
00029 //virtual List<SimulationResponse*>* extractResponsesFromModel(std::string modelName) const = 0;
00030
00031 virtual void startSimulationOfScenario(SimulationScenario* scenario) = 0;
00032 virtual void startExperiment() = 0;
00033 virtual void stopExperiment() = 0;
00034 virtual void addTraceSimulationHandler(traceSimulationProcessListener
 traceSimulationProcessListener) = 0;
00035 };
00036
```



```

00046
00047 #endif /* EXPERIMENTMANAGER_IF_H */

```

## 16.276 GenesysPropertyIntrospection.h

```

00001 #pragma once
00002
00003 #include <string>
00004 #include <vector>
00005
00006 #include "SimulationControlAndResponse.h"
00007 #include "../util/Util.h"
00008
00009 enum class GenesysPropertyKind {
00010 Unknown,
00011 Boolean,
00012 Integer,
00013 UnsignedInteger,
00014 UnsignedShort,
00015 Double,
00016 String,
00017 TimeUnit,
00018 Enum,
00019 Object,
00020 List
00021 };
00022
00023 struct GenesysPropertyDescriptor {
00024 SimulationControl* control = nullptr;
00025
00026 std::string ownerClassName;
00027 std::string ownerElementName;
00028 std::string displayName;
00029 std::string technicalTypeName;
00030
00031 GenesysPropertyKind kind = GenesysPropertyKind::Unknown;
00032
00033 bool readOnly = true;
00034 bool isList = false;
00035 bool isClass = false;
00036 bool isEnum = false;
00037 bool isInlineObject = false;
00038 bool isModelDataDefinitionReference = false;
00039 bool supportsInlineExpansion = false;
00040 bool supportsListEditor = false;
00041 bool supportsExistingObjectSelection = false;
00042 bool supportsObjectCreation = false;
00043 bool supportsNewListElementCreation = false;
00044 SimulationControlEditorHint editorHint = SimulationControlEditorHint::Default;
00045
00046 std::string currentValue;
00047 std::vector<std::string> choices;
00048 std::string currentReferenceType;
00049 std::vector<std::string> creatableReferenceTypes;
00050 // For polymorphic list controls, this is the currently selected concrete element type.
00051 //
00052 // Example: a Set physically stores ModelDataDefinition* but may currently be a Set of Resource.
00053 // Empty means the list has not been typed yet.
00054 std::string currentListElementType;
00055 // For polymorphic list controls, these are the concrete element types that the kernel allows
00056 // the GUI to select/create. Empty means the list should use its legacy single-type behavior.
00057 std::vector<std::string> creatableListElementTypes;
00058 };
00059
00060 class GenesysPropertyIntrospection {
00061 public:
00062 static GenesysPropertyDescriptor describe(SimulationControl* control);
00063 static std::vector<GenesysPropertyDescriptor> describe(List<SimulationControl*>* controls);
00064
00065 static GenesysPropertyKind deduceKind(const SimulationControl* control);
00066
00067 static bool setValue(
00068 SimulationControl* control,
00069 const std::string& value,
00070 bool remove = false,
00071 std::string* errorMessage = nullptr
00072);
00073
00074 static std::vector<SimulationControl*> children(
00075 SimulationControl* control,
00076 int index = 0
00077);
00078 };

```

## 16.277 LicenceManager.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: LicenceManager.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 29 de Maio de 2019, 11:45
00012 */
00013
00014 #ifndef LICENCEMANAGER_H
00015 #define LICENCEMANAGER_H
00016
00017 #include <string>
00018
00019 //namespace GenesysKernel {
00020 class Simulator;
00021
00022 class LicenceManager {
00023 public:
00024 LicenceManager(Simulator* simulator);
00025 virtual ~LicenceManager() = default;
00026 const std::string showLicence() const;
00027 const std::string showLimits() const;
00028 const std::string showActivationCode() const;
00029 bool lookforActivationCode();
00030 bool insertActivationCode();
00031 void removeActivationCode();
00032 unsigned int getModelComponentsLimit();
00033 unsigned int getModelDatasLimit();
00034 unsigned int getEntityLimit();
00035 unsigned int getHostsLimit();
00036 unsigned int getThreadsLimit();
00037 private:
00038 void setDefaultLicenceAndLimits();
00039 private:
00040 Simulator* _simulator;
00041 std::string _licence;
00042 std::string _activationCode;
00043 unsigned int _components, _datadefinitions, _entities, _hosts, _threads;
00044 };
00045 //namespace\\}
00046 #endif /* LICENCEMANAGER_H */
00047

```

## 16.278 Model.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: SimulationModel.h
00009 * Author: Prof. Rafael Luiz Cancian, Dr. Eng.
00010 *
00011 * Created on 21 de Junho de 2018, 15:01
00012 */
00013
00014 #ifndef SIMULATIONMODEL_H
00015 #define SIMULATIONMODEL_H
00016
00017 #include <list>
00018 #include <string>
00019
00020 #include "../util/List.h"
00021 #include "ModelComponent.h"
00022 #include "../Event.h"
00023 #include "ModelChecker_if.h"
00024 #include "../Parser_if.h"
00025 #include "../persistence/Persistence_if.h"
00026 #include "ModelDataManager.h"
00027 #include "ModelComponentManager.h"
00028 #include "../TraceManager.h"
00029 #include "../OnEventManager.h"
00030 #include "ModelInfo.h"
00031 #include "ModelSimulation.h"
00032 #include "../SimulationControlAndResponse.h"

```

```

00033
00034 //namespace GenesysKernel {
00035 class Simulator;
00036
00037 class Model {
00038 public:
00039 Model(Simulator* simulator, unsigned int level = 0);
00040 virtual ~Model();
00041 public: // model control
00042 //void showReports();
00043 bool save(std::string filename);
00044 bool load(std::string filename);
00045 bool check();
00046 void clear();
00047 std::string showLanguage();
00048 void show();
00049 bool insert(ModelDataDefinition* elemOrComp);
00050 void remove(ModelDataDefinition* elemOrComp);
00051 std::list<ModelDataDefinition*> collectDataDefinitionsRemovedWith(const
std::list<ModelDataDefinition*>& roots) const;
00052 Entity* createEntity(std::string name, bool insertIntoModel = true);
00053 void removeEntity(Entity* entity); //, bool collectStatistics);
00054 void sendEntityToComponent(Entity* entity, Connection* connection, double timeDelay = 0.0);
00055 void sendEntityToComponent(Entity* entity, ModelComponent* component, double timeDelay = 0.0,
unsigned int componentInputPortNumber = 0);
00056 double parseExpression(const std::string expression);
00057 double parseExpression(const std::string expression, bool& success, std::string& errorMessage);
00058 bool checkExpression(const std::string expression, const std::string expressionName, std::string&
errorMessage);
00059 void checkReferencesToDataDefinitions(std::string expression, std::map<std::string,
std::list<std::string*>*> referencedDataDefinitions);
00060
00061 public: // only gets
00062 Util::identification getId() const;
00063 bool hasChanged() const;
00064 void setHasChanged(bool hasChanged);
00065 // 1:1
00066 OnEventManager* getOnEventManager() const;
00067 ModelDataManager* getDataManager() const;
00068 ModelComponentManager* getComponentManager() const;
00069 ModelInfo* getInfos() const;
00070 Simulator* getParentSimulator() const;
00071 ModelSimulation* getSimulation() const;
00072 // 1:n
00073 List<Event*>* getFutureEvents() const;
00074 List<SimulationControl*>* getControls() const;
00075 List<SimulationResponse*>* getResponses() const;
00076
00077 public: // gets and sets
00078 void setTracer(TraceManager* _traceManager);
00079 TraceManager* getTracer() const;
00080 Persistence_if* getPersistence() const;
00081 unsigned int getLevel() const;
00082
00083 private:
00084 void _showConnections() const;
00085 void _showComponents() const;
00086 void _showElements() const;
00087 void _showSimulationControls() const;
00088 void _showSimulationResponses() const;
00089 void _destroyFutureEvents();
00090 void _destroyTransientEntities();
00091 void _destroyComponents();
00092 void _destroyModelDataDefinitions();
00093 void _clearOrphanedDataDefinitions();
00094 void _createInternalDataDefinitions();
00095 private:
00096 bool _hasChanged = false;
00097 private: // read only public access (gets)
00098 Util::identification _id;
00099 unsigned int _level = 0;
00100 Simulator* _parentSimulator;
00101 // 1:1 (associted classes)
00102 TraceManager* _traceManager;
00103 OnEventManager* _eventManager;
00104 ModelDataManager* _modelDataManager;
00105 ModelComponentManager* _componentManager;
00106 ModelInfo* _modelInfo;
00107 ModelSimulation* _simulation;
00108 Persistence_if* _modelPersistence;
00109
00110 // 1:n
00111 //List<ModelComponent*>* _components;
00112 List<Event*>* _futureEvents;
00113 // for process analyser
00114 List<SimulationResponse*>* _responses;
00115 List<SimulationControl*>* _controls;

```

```

00316
00317 private: // no public access (no gets / sets)
00318 ModelChecker_if* _modelChecker;
00319 Parser_if* _parser;
00320 };
00321 //namespace\}
00322 #endif /* SIMULATIONMODEL_H */

```

## 16.279 ModelChecker\_if.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: ModelChecker_if.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 23 de Agosto de 2018, 15:48
00012 */
00013
00014 #ifndef MODELCHECKER_IF_H
00015 #define MODELCHECKER_IF_H
00016
00017 #include <typeinfo>
00018 #include <string>
00019
00020 // #include "Model.h"
00021 // class Model;
00022
00026 class ModelChecker_if {
00027 public:
00028 virtual ~ModelChecker_if() = default;
00033 virtual bool checkAll() = 0;
00038 virtual bool checkConnected() = 0;
00043 virtual bool checkSymbols() = 0;
00048 virtual bool checkActivationCode() = 0;
00053 virtual bool checkLimits() = 0;
00054 };
00055
00056 #endif /* MODELCHECKER_IF_H */

```

## 16.280 ModelCheckerDefaultImpl1.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: ModelCheckerDefaultImpl1.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 8 de Agosto de 2018, 18:44
00012 */
00013
00014 #ifndef MODELCHECKERDEFAULTIMPL1_H
00015 #define MODELCHECKERDEFAULTIMPL1_H
00016
00017 #include "ModelChecker_if.h"
00018 #include "Model.h"
00019 #include "../PluginManager.h"
00020
00021 // namespace GenesysKernel {
00022
00023 class ModelCheckerDefaultImpl1 : public ModelChecker_if {
00024 public:
00025 ModelCheckerDefaultImpl1(Model* model);
00026 virtual ~ModelCheckerDefaultImpl1() = default;
00027 public:
00028 virtual bool checkAll() override;
00029 virtual bool checkConnected() override;
00030 virtual bool checkSymbols() override;
00031 virtual bool checkActivationCode() override;
00032 virtual bool checkLimits() override;
00033 private:
00034 void _recursiveConnectedTo(PluginManager* pluginManager, ModelComponent* comp,
00035 List<ModelComponent*>* visited, List<ModelComponent*>* unconnected, bool* drenofound);
00035 void _showResult(bool result, std::string checking);

```

```

00036 private:
00037 Model* _model;
00038 };
00039 //namespace\}
00040 #endif /* MODELCHECKERDEFAULTIMPL1_H */
00041

```

## 16.281 ModelComponent.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Element.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 21 de Junho de 2018, 15:56
00012 */
00013
00014 #ifndef MODELCOMPONENT_H
00015 #define MODELCOMPONENT_H
00016
00017 #include <string>
00018 #include <list>
00019
00020 #include "../Plugin.h"
00021 #include "../util/List.h"
00022 #include "../essentialPlugins/Entity.h"
00023 #include "ModelDataDefinition.h"
00024 #include "../ConnectionManager.h"
00025 //namespace GenesysKernel {
00026
00027 class Model;
00028 class Event;
00029
00030 class ModelComponent : public ModelDataDefinition {
00031 public:
00032 ModelComponent(Model* model, std::string componentTypename, std::string name = "");
00033 virtual ~ModelComponent();
00034
00035 public:
00036 ConnectionManager* getConnectionManager() const;
00037 void connectTo(ModelComponent* component, unsigned int inputPortNumber = 0);
00038 bool hasBreakpointAt();
00039 void setDescription(std::string _description);
00040 std::string getDescription() const;
00041
00042 void remove(ModelComponent * comp);
00043 // ...
00044
00045 public:
00046 virtual std::string show() override;
00047
00048 public:
00049 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00050 // new static methods for all ModelComponents
00051 static bool Check(ModelComponent* component);
00052 static void CreateInternalData(ModelComponent* component);
00053 static void SaveInstance(PersistenceRecord *fields, ModelComponent* component);
00054 static void DispatchEvent(Event* event);
00055
00056 protected:
00057 virtual bool _loadInstance(PersistenceRecord *fields) override;
00058 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00059 // new virtual methods for all ModelComponents
00060 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) = 0;
00061
00062 protected:
00063 // virtual bool _check(std::string& errorMessage);
00064 // virtual ParserChangesInformation* _getParserChangesInformation();
00065 // virtual void _initBetweenReplications();
00066 // virtual void _createInternalAndAttachedData(); /* legacy combined lifecycle hook */
00067 // virtual void _addSimulationControl(SimulationControl* property);
00068
00069 protected: // new protected attributes for all ModelComponents
00070 ConnectionManager* _connections = new ConnectionManager();
00071
00072 private:
00073 // ...
00074
00075 private:
00076
00077

```

```

00164 // Default values for the attributes. Used on initing, loading and saving
00165 const struct DEFAULT_VALUES {
00166 const unsigned int nextSize = 1; // No need for attribute. Taken from
 _connections
00167 const unsigned int nextinputPortNumber = 0;
00168 const std::string description = "";
00169 } DEFAULT;
00170 std::string _description = DEFAULT.description;
00171
00172 private:
00173 //...
00174
00175 private:
00176 // ...
00177
00178 };
00179 //namespace\\}
00180 #endif /* MODELCOMPONENT_H */

```

## 16.282 ModelComponentManager.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: ModelComponentManager.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 28 de Maio de 2019, 10:41
00012 */
00013
00014 #ifndef COMPONENTMANAGER_H
00015 #define COMPONENTMANAGER_H
00016
00017 #include "ModelComponent.h"
00018 #include "../SourceModelComponent.h"
00019
00020 //namespace GenesysKernel {
00021 //class Model;
00022
00023 class ModelComponentManager {
00024 public:
00025 ModelComponentManager(Model* model);
00026 virtual ~ModelComponentManager();
00027 public:
00028 bool insert(ModelComponent* comp);
00029 void remove(ModelComponent* comp);
00030 ModelComponent* find(std::string name);
00031 ModelComponent* find(Util::identification id);
00032 void clear();
00033 public:
00034 unsigned int getNumberOfComponents();
00035 std::list<ModelComponent*>::iterator begin();
00036 std::list<ModelComponent*>::iterator end();
00037 ModelComponent* front();
00038 ModelComponent* next();
00039 bool hasChanged() const;
00040 void setHasChanged(bool _hasChanged);
00041 public:
00042 std::list<SourceModelComponent*>* getSourceComponents();
00043 std::list<ModelComponent*>* getTransferInComponents();
00044 std::list<ModelComponent*>* getAllComponents() const;
00045 private:
00046 List<ModelComponent*> _components;
00047 Model* _parentModel;
00048 bool _hasChanged = false;
00049 std::list<ModelComponent*>::iterator _componentIterator;
00050 };
00051 //namespace\\}
00052 #endif /* COMPONENTMANAGER_H */

```

## 16.283 ModelDataDefinition.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006

```

```

00007 /*
00008 * File: ModelDataDefinition.h
00009 * Author: Prof. Rafael Luiz Cancian, Dr. Eng.
00010 *
00011 * Created on 21 de Junho de 2018, 19:40
00012 */
00013
00014 #ifndef MODELELEMENT_H
00015 #define MODELELEMENT_H
00016
00017 #include <string>
00018 #include <list>
00019 #include <vector>
00020 #include <map>
00021
00022 #include "../util/List.h"
00023 #include "../ParserChangesInformation.h"
00024 #include "../Persistence.h"
00025 // #include "PropertyGenesys.h"
00026 #include "ModelDataDefinitionAssociations.h"
00027 #include "../SimulationControlAndResponse.h"
00028 #include "../TraceManager.h"
00029
00030 // namespace GenesysKernel {
00031 class Model;
00032
00033
00034 class ModelDataDefinition : PersistentObject_base {
00035 public:
00036 ModelDataDefinition(Model* model, std::string datadefinitionTypename, std::string name = "",
00037 bool insertIntoModel = true);
00038 // ModelDataDefinition(Model* model, std::string datadefinitionTypename, std::string name = "",
00039 bool insertIntoModel = true);
00040 // ModelDataDefinition(const ModelDataDefinition &orig);
00041 virtual ~ModelDataDefinition();
00042
00043 public: // get & set
00044 Util::identification getId() const;
00045 void setName(const std::string& name);
00046 const std::string& getName() const;
00047 void setLabel(const std::string& label);
00048 const std::string& getLabel() const;
00049 const std::string& getClassname() const;
00050 Model* getParentModel() const;
00051 bool isReportStatistics() const;
00052 void setReportStatistics(bool reportStatistics);
00053
00054 public:
00055 ModelDataDefinition* getInternalData(const std::string& name) const;
00056 std::map<std::string, ModelDataDefinition*> getInternalData() const;
00057 std::map<std::string, ModelDataDefinition*> getAttachedData() const;
00058 // ModelDataDefinition* getInternalData(std::string key) const;
00059 bool hasChanged() const;
00060 void setHasChanged(bool hasChanged);
00061 unsigned int getLevel() const;
00062 void setModelLevel(unsigned int _modelLevel);
00063 List<SimulationControl*> getSimulationControls() const;
00064 TraceManager::Level getTraceLevelSpecific() const;
00065 void setTraceLevelSpecific(TraceManager::Level level);
00066 void defineTraceLevelSpecific(TraceManager::Level traceLevelSpecific, bool
00067 traceLevelSpecificEnabled = true);
00068 bool isTraceLevelSpecificEnabled() const;
00069 void setTraceLevelSpecificEnabled(bool traceLevelSpecificEnabled);
00070
00071 public: // public static methods
00072 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields, bool
00073 insertIntoModel);
00074 // @ToDo: (pequena alteração): return ModelComponent* ?
00075 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00076 static void SaveInstance(PersistenceRecord* fields, ModelDataDefinition* modeldatum);
00077 static bool Check(ModelDataDefinition* modeldatum, std::string& errorMessage);
00078 static void InitBetweenReplications(ModelDataDefinition* modeldatum);
00079 // static PluginInformation* GetPluginInformation();
00080
00081 public:
00082 static void CreateInternalData(ModelDataDefinition* modeldatum);
00083 static bool CreateRelatedDataElements(ModelDataDefinition* modeldatum, std::string& errorMessage);
00084 /* This class method is responsible for invoking the protected method _initBetweenReplication(),
00085 which clears all statistics, attributes, counters and other stuff before starting a new replication */
00086
00087 public: // public virtual methods
00088 virtual std::string show();
00089 protected:
00090 bool _getSaveDefaultsOption();
00091
00092 protected:
00093 virtual bool _loadInstance(PersistenceRecord* fields) override;

```

```

00259 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00260
00261 protected:
00262 virtual bool _check(std::string& errorMessage);
00264 virtual ParserChangesInformation* _getParserChangesInformation();
00265 virtual void _initBetweenReplications();
00266 /*< A ModelDataDefinition or ModelComponent that includes (internal) ou refers to (attach) other
ModelDataDefinition must register them inside this method. */
00267 virtual void _addSimulationControl(SimulationControl* control);
00268 //virtual void _addSimulationResponse(SimulationControl* response);
00269
00270 /*@Todo: Fix this mess about diffent kind of internal and attached data related to report statistics,
00271 * mandatory and optional editable internal or attached dataelements that are properties,
00272 * and, I don't know, maybe other kind of internal and attached nedded stuff?
00273 */
00274 protected:
00276 virtual void _createInternalAndAttachedData(); // final; /*@Todo: will be final
00277 protected:
00278 virtual void _createInternalStatisticReporters();
00279 virtual void _createAttachedAttributes();
00280 virtual void _createNonEditableDataDefinitions();
00281 virtual void _createEditableDataDefinitions();
00282 protected:
00283 void _templateCreateInternalStatisticReporters();
00284 void _templateCreateAttachedAttributes();
00285 void _templateCreateNonEditableDataDefinitions();
00286 void _templateCreateEditableDataDefinitions();
00287 protected:
00288 void _internaStatisticReportersClear();
00289 void _internalDataClear();
00290 void _internalDataInsert(const std::string& key, ModelDataDefinition* child);
00291 void _internalDataRemove(const std::string& key);
00292 void _attachedDataClear();
00293 void _attachedDataInsert(const std::string& key, ModelDataDefinition* data);
00294 void _attachedDataRemove(const std::string& key);
00295 void _attachedAttributesInsert(const std::vector<std::string>& neededNames);
00296 void _statisticReporterInsert(const std::string& key, ModelDataDefinition* data);
00297 void _statisticReporterRemove(const std::string& key);
00298 void _statisticReportersClear();
00299 void _mandatoryAttachedAttributeInsert(const std::string& key, ModelDataDefinition* data);
00300 void _mandatoryAttachedAttributeRemove(const std::string& key);
00301 void _mandatoryAttachedAttributesClear();
00302 void _mandatoryEditableDataDefinitionInsert(const std::string& key, ModelDataDefinition* data);
00303 void _mandatoryEditableDataDefinitionRemove(const std::string& key);
00304 void _mandatoryEditableDataDefinitionsClear();
00305 void _optionalEditableDataDefinitionInsert(const std::string& key, ModelDataDefinition* data);
00306 void _optionalEditableDataDefinitionRemove(const std::string& key);
00307 void _optionalEditableDataDefinitionsClear();
00308 void _mandatoryNonEditableDataDefinitionInsert(const std::string& key, ModelDataDefinition* data);
00309 void _mandatoryNonEditableDataDefinitionRemove(const std::string& key);
00310 void _mandatoryNonEditableDataDefinitionsClear();
00311 void _optionalNonEditableDataDefinitionInsert(const std::string& key, ModelDataDefinition* data);
00312 void _optionalNonEditableDataDefinitionRemove(const std::string& key);
00313 void _optionalNonEditableDataDefinitionsClear();
00314 protected:
00316 void _checkCreateAttachedReferencedDataDefinition(const std::string& expression);
00317 //(std::map<std::string, std::list<std::string>*> referencedDataDefinitions);
00318
00319
00320 private:
00321 // name is now private. So changes in name must be throught setName, wich gives oportunity to
rename internalElements, SimulationControls and SimulationResponses
00322 std::string _name;
00323 std::string _label;
00324
00325 private:
00326 friend class ModelDataDefinitionAssociations;
00327 ModelDataDefinitionAssociations _dataAssociations;
00328
00329 protected:
00330 Util::identification _id;
00331 std::string _typename;
00332 bool _reportStatistics;
00333 bool _hasChanged;
00334 unsigned int _modelLevel = 0; // the ID of parent component (submodel or process, for now) in the
"superlevel"
00335 Model* _parentModel;
00336
00337 private:
00338 bool _checkSpecificTraceLevel(TraceManager::Level level);
00339
00340 protected:
00341 void trace(const std::string& text, TraceManager::Level level = TraceManager::Level::L8_detailed);
00342 void traceError(const std::string& text, const std::exception& e);
00343 void traceError(const std::string& text, TraceManager::Level level =
TraceManager::Level::L1_errorFatal);
00344 void traceReport(const std::string& text, TraceManager::Level level =

```



```

 TraceManager::Level::L2_results);
00345 void traceSimulation(void* thisobject, double time, Entity* entity, ModelComponent* component,
 const std::string& text,
00346 TraceManager::Level level = TraceManager::Level::L8_detailed);
00355 void traceSimulation(void* thisobject, const std::string& text,
00356 TraceManager::Level level = TraceManager::Level::L8_detailed);
00357 void traceSimulation(void* thisobject, TraceManager::Level level, const std::string& text);
00358
00359 protected:
00360 //List<SimulationControl*>* _simulationResponses = new List<SimulationControl*>();
00361 List<SimulationControl*>* _simulationControls = new List<SimulationControl*>();
00362 TraceManager::Level _traceLevelSpecific;
00363 bool _traceLevelSpecificEnabled = false;
00364 //PropertyListG* _propertiesG = new PropertyListG();
00365 };
00366
00367 //namespace\\}
00368
00369 #endif /* MODELELEMENT_H */

```

## 16.284 ModelDataDefinitionAssociations.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: ModelDataDefinitionAssociations.h
00009 * Author: Codex
00010 *
00011 * Created on 6 de Maio de 2026
00012 */
00013
00014 #ifndef MODELDATEDEFINITIONASSOCIATIONS_H
00015 #define MODELDATEDEFINITIONASSOCIATIONS_H
00016
00017 #include <map>
00018 #include <string>
00019 #include <vector>
00020
00021 class Attribute;
00022 class Model;
00023 class ModelDataDefinition;
00024
00032 class ModelDataDefinitionAssociations {
00033 public:
00034 explicit ModelDataDefinitionAssociations(ModelDataDefinition* owner);
00035 ~ModelDataDefinitionAssociations();
00036
00037 public:
00038 std::map<std::string, ModelDataDefinition*>* getInternalData() const;
00039 std::map<std::string, ModelDataDefinition*>* getAttachedData() const;
00040 ModelDataDefinition* getInternalData(const std::string& key) const;
00041
00042 void clearAll();
00043 void clearInternalData();
00044 void clearAttachedData();
00045
00046 void internalDataInsert(const std::string& key, ModelDataDefinition* data);
00047 void internalDataRemove(const std::string& key);
00048 void attachedDataInsert(const std::string& key, ModelDataDefinition* data);
00049 void attachedDataRemove(const std::string& key);
00050 void attachedAttributesInsert(const std::vector<std::string>& neededNames);
00051 void checkCreateAttachedReferencedDataDefinition(const std::string& expression);
00052
00053 void createInternalAndAttachedData();
00054 void createInternalStatisticReporters();
00055 void createAttachedAttributes();
00056 void createNonEditableDataDefinitions();
00057 void createEditableDataDefinitions();
00058
00059 void insertStatisticReporter(const std::string& key, ModelDataDefinition* data);
00060 void removeStatisticReporter(const std::string& key);
00061 void clearStatisticReporters();
00062
00063 void insertMandatoryAttachedAttribute(const std::string& key, ModelDataDefinition* data);
00064 void removeMandatoryAttachedAttribute(const std::string& key);
00065 void clearMandatoryAttachedAttributes();
00066
00067 void insertMandatoryEditableDataDefinition(const std::string& key, ModelDataDefinition* data);
00068 void removeMandatoryEditableDataDefinition(const std::string& key);
00069 void clearMandatoryEditableDataDefinitions();
00070

```

```

00071 void insertOptionalEditableDataDefinition(const std::string& key, ModelDataDefinition* data);
00072 void removeOptionalEditableDataDefinition(const std::string& key);
00073 void clearOptionalEditableDataDefinitions();
00074
00075 void insertMandatoryNonEditableDataDefinition(const std::string& key, ModelDataDefinition* data);
00076 void removeMandatoryNonEditableDataDefinition(const std::string& key);
00077 void clearMandatoryNonEditableDataDefinitions();
00078
00079 void insertOptionalNonEditableDataDefinition(const std::string& key, ModelDataDefinition* data);
00080 void removeOptionalNonEditableDataDefinition(const std::string& key);
00081 void clearOptionalNonEditableDataDefinitions();
00082
00083 private:
00084 void _deleteOwnedInternalData(ModelDataDefinition* data);
00085 void _rebuildLegacyViews();
00086
00087 private:
00088 ModelDataDefinition* _owner = nullptr;
00089 std::map<std::string, ModelDataDefinition*> _internalData;
00090 std::map<std::string, ModelDataDefinition*> _attachedData;
00091 std::map<std::string, ModelDataDefinition*> _statisticReporters;
00092 std::map<std::string, ModelDataDefinition*> _mandatoryAttachedAttributes;
00093 std::map<std::string, ModelDataDefinition*> _mandatoryEditableDataDefinitions;
00094 std::map<std::string, ModelDataDefinition*> _optionalEditableDataDefinitions;
00095 std::map<std::string, ModelDataDefinition*> _mandatoryNonEditableDataDefinitions;
00096 std::map<std::string, ModelDataDefinition*> _optionalNonEditableDataDefinitions;
00097 };
00098
00099 #endif /* MODELDATADEFINITIONASSOCIATIONS_H */

```

## 16.285 ModelDataManager.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: ElementManager.h
00009 * Author: rafael.luiiz.cancian
00010 *
00011 * Created on 7 de Novembro de 2018, 12:48
00012 */
00013
00014 #ifndef ELEMENTMANAGER_H
00015 #define ELEMENTMANAGER_H
00016
00017 #include <list>
00018 #include <map>
00019 #include "../util/List.h"
00020 #include "ModelDataDefinition.h"
00021
00022 //namespace GenesysKernel {
00023 class Model;
00024
00031 class ModelDataManager {
00032 public:
00034 ModelDataManager(Model* model);
00035 virtual ~ModelDataManager();
00036 public:
00042 bool insert(ModelDataDefinition* anElement);
00047 void remove(ModelDataDefinition* anElement);
00054 bool insert(std::string datadefinitionTypename, ModelDataDefinition* anElement);
00060 void remove(std::string datadefinitionTypename, ModelDataDefinition* anElement);
00069 bool check(std::string datadefinitionTypename, ModelDataDefinition* anElement, std::string
expressionName, std::string& errorMessage);
00079 bool check(std::string datadefinitionTypename, std::string elementName, std::string
expressionName, bool mandatory, std::string& errorMessage);
00083 void clear();
00084 public:
00091 ModelDataDefinition* getDataDefinition(std::string datadefinitionTypename, Util::identification
id);
00098 ModelDataDefinition* getDataDefinition(std::string datadefinitionTypename, std::string name);
00104 unsigned int getNumberOfDataDefinitions(std::string datadefinitionTypename);
00109 unsigned int getNumberOfDataDefinitions();
00116 int getRankOf(std::string datadefinitionTypename, std::string name);
00121 std::list<std::string> getDataDefinitionClassnames() const;
00122
00123 //private:
00124 public:
00125 // @ToDo: (importante): Should be removed and replaced by GetElement(elementType, rank)
00126 List<ModelDataDefinition*> getDataDefinitionList(std::string datadefinitionTypename) const;
00127 public:
00129 void show();

```

```

00131 Model* getParentModel() const;
00133 bool hasChanged() const;
00135 void setHasChanged(bool _hasChanged);
00136 private:
00137 std::map<std::string, List<ModelDataDefinition*>>* _datadefinitions;
00138 Model* _parentModel;
00139 bool _hasChanged = false;
00140 };
00141 //namespace\\
00142 #endif /* ELEMENTMANAGER_H */

```

## 16.286 ModellInfo.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: ModellInfo.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 7 de Novembro de 2018, 18:13
00012 */
00013
00014 #ifndef MODELINFO_H
00015 #define MODELINFO_H
00016
00017 #include <string>
00018 #include "../util/Util.h"
00019 #include "../Persistence.h"
00020
00021 //namespace GenesysKernel {
00022
00026 class ModellInfo {
00027 public:
00028 ModellInfo();
00029 virtual ~ModellInfo() = default;
00030 public:
00031 std::string show();
00032 public: // gets and sets
00033 void setName(std::string _name);
00042 std::string getName() const;
00047 void setAnalystName(std::string _analystName);
00052 std::string getAnalystName() const;
00057 void setDescription(std::string _description);
00062 std::string getDescription() const;
00067 void setProjectTitle(std::string _projectTitle);
00072 std::string getProjectTitle() const;
00077 void setVersion(std::string _version);
00082 std::string getVersion() const;
00083 public:
00088 void loadInstance(PersistenceRecord *fields);
00093 void saveInstance(PersistenceRecord *fields);
00098 bool hasChanged() const;
00100 void setHasChanged(bool hasChanged);
00101 private: // with public access (get & set)
00102 // model general information
00103 std::string _name;
00104 std::string _analystName = "";
00105 std::string _description = "";
00106 std::string _projectTitle = "";
00107 std::string _version = "1.0";
00108 bool _hasChanged = false;
00109 };
00110 //namespace\\
00111 #endif /* MODELINFO_H */

```

## 16.287 ModelManager.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: ModelManager.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 31 de Maio de 2019, 08:37

```

```

00012 */
00013
00014 #ifndef MODELMANAGER_H
00015 #define MODELMANAGER_H
00016
00017 #include "Model.h"
00018 #include "../TraceManager.h"
00019 #include <vector>
00020
00021 //namespace GenesysKernel {
00022
00026 class ModelManager {
00027 public:
00032 ModelManager(Simulator* simulator);
00036 virtual ~ModelManager();
00037 public:
00042 Model* newModel();
00047 void insert(Model* model);
00052 void remove(Model* model);
00058 bool setCurrent(Model* model);
00064 bool saveModel(std::string filename);
00070 Model* loadModel(std::string filename);
00076 Model* createFromLanguage(std::string modelSpecification);
00081 unsigned int size();
00082 public:
00087 std::vector<Model*> models() const;
00093 bool hasModel(Model* model) const;
00099 int indexOf(Model* model) const;
00105 Model* modelAt(unsigned int index) const;
00110 int currentIndex() const;
00115 Model* front();
00120 Model* last();
00125 Model* current();
00130 Model* next();
00135 Model* previous();
00140 bool canGoNext() const;
00145 bool canGoPrevious() const;
00146 //Model* end();
00147 private:
00148 List<Model*>* _models = new List<Model*>();
00149 Model* _currentModel = nullptr;
00150 private:
00151 Simulator* _simulator;
00152 };
00153 //namespace\\}
00154 #endif /* MODELMANAGER_H */

```

## 16.288 ModelSimulation.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: ModelSimulation.h
00009 * Author: Prof. Rafael Luiz Cancian, Dr. Eng.
00010 *
00011 * Created on 7 de Novembro de 2018, 18:04
00012 */
00013
00014 #ifndef MODELSIMULATION_H
00015 #define MODELSIMULATION_H
00016
00017 #include <chrono>
00018 #include <memory>
00019 #include "../SimulationControlAndResponse.h"
00020 #include "../Event.h"
00021 #include "../essentialPlugins/Entity.h"
00022 #include "ModelInfo.h"
00023 #include "../SimulationReporter_if.h"
00024 #include "../OnEventManager.h"
00025 // #include "Counter.h"
00026 // namespace GenesysKernel {
00027
00028 // #include "Model.h" // for friend functions
00029 class Model;
00030
00047 class ModelSimulation { // 202104 to be subjected to SimulationScenario
00048 public:
00049 ModelSimulation(Model* model);
00050 virtual ~ModelSimulation();
00051 public:
00052 std::string show();

```

```

00053 // @ToDo: (importante): Keep ModelSimulation as the stable
00054 // execution core while the higher-level experiment abstractions mature.
00055 // Any migration of responsibilities to SimulationExperiment/SimulationScenario
00056 // should preserve the already functional replication-based workflow here.
00057 public: // simulation control
00058 // @ToDo: (importante): Revisit how this class should interact with the unfinished higher-level
 experiment layer.
00059 void start();
00060 void pause();
00061 void step();
00062 void stop();
00063 public: // old modelInfos
00064 void setNumberOfReplications(unsigned int _numberOfReplications);
00065 unsigned int getNumberOfReplications() const;
00066 //void setReplicationLength(double _replicationLength);
00067 void setReplicationLength(double _replicationLength, Util::TimeUnit _replicationLengthTimeUnit =
 Util::TimeUnit::unknown);
00068 double getReplicationLength() const;
00069 void setReplicationLengthTimeUnit(Util::TimeUnit _replicationLengthTimeUnit);
00070 Util::TimeUnit getReplicationLengthTimeUnit() const;
00071 void setReplicationReportBaseTimeUnit(Util::TimeUnit _replicationReportBaseTimeUnit);
00072 Util::TimeUnit getReplicationBaseTimeUnit() const;
00073 //void setWarmUpPeriod(double warmUpPeriod);
00074 void setWarmUpPeriod(double warmUpPeriod, Util::TimeUnit warmUpPeriodTimeUnit =
 Util::TimeUnit::unknown);
00075 double getWarmUpPeriod() const;
00076 void setWarmUpPeriodTimeUnit(Util::TimeUnit _warmUpPeriodTimeUnit);
00077 Util::TimeUnit getWarmUpPeriodTimeUnit() const;
00078 void setTerminatingCondition(std::string _terminatingCondition);
00079 std::string getTerminatingCondition() const;
00080 public: // gets and sets
00081 void setPauseOnEvent(bool _pauseOnEvent);
00082 bool isPauseOnEvent() const;
00083 void setStepByStep(bool _stepByStep);
00084 bool isStepByStep() const;
00085 void setInitializeStatistics(bool _initializeStatistics);
00086 bool isInitializeStatistics() const;
00087 void setInitializeSystem(bool _initializeSystem);
00088 bool isInitializeSystem() const;
00089 void setPauseOnReplication(bool _pauseBetweenReplications);
00090 bool isPauseOnReplication() const;
00091 public: // gets and sets ModelSubParts
00092 void setReporter(SimulationReporter_if* _simulationReporter);
00093 SimulationReporter_if* getReporter() const;
00094 public: // only gets
00095 double getSimulatedTime() const;
00096 bool isRunning() const;
00097 bool isPaused() const;
00098 unsigned int getCurrentReplicationNumber() const;
00099 void setShowReportsAfterReplication(bool showReportsAfterReplication);
00100 bool isShowReportsAfterReplication() const;
00101 void setShowReportsAfterSimulation(bool showReportsAfterSimulation);
00102 bool isShowReportsAfterSimulation() const;
00103 List<double>* getBreakpointsOnTime() const;
00104 List<Entity*>* getBreakpointsOnEntity() const;
00105 List<ModelComponent*>* getBreakpointsOnComponent() const;
00106 const List<ModelDataDefinition*>* getSimulationStatisticsAggregates() const;
00107 public:
00108 void loadInstance(PersistenceRecord *fields);
00109 void saveInstance(PersistenceRecord *fields, bool saveDefaults);
00110 Event* getCurrentEvent() const;
00111 void setShowSimulationResponsesInReport(bool _showSimulationResponsesInReport);
00112 bool isShowSimulationResponsesInReport() const;
00113 void setShowSimulationControlsInReport(bool _showSimulationControlsInReport);
00114 bool isShowSimulationControlsInReport() const;
00115 /*
00116 * PRIVATE
00117 */
00118 private: // simulation control
00119 void _initSimulation();
00120 void _initReplication();
00121 void _clearStatistics();
00122 void _checkWarmUpTime(Event* nextEvent);
00123 void _stepSimulation();
00124 void _replicationEnded();
00125 void _simulationEnded();
00126 private:
00127 void _dispatchEvent(Event* event);
00128 bool _checkBreakpointAt(Event* event);
00129 bool _isReplicationEndCondition();
00130 void _actualizeSimulationStatistics();
00131 void _showSimulationHeader();
00132 void _traceReplicationEnded();
00133 private:
00134 std::unique_ptr<SimulationEvent> _createSimulationEvent(void* thiscustomObject = nullptr);
00135 //friend void Model::removeEntity(Entity* entity);
00136 //friend void Model::sendEntityToComponent(Entity* entity, ModelComponent* component, double

```

```

 timeDelay, unsigned int componentinputPortNumber);
00137 friend class Model;
00138 private:
00139 double _simulatedTime = 0.0;
00140 double _justTriggeredBreakpointsOnTime = 0.0;
00141 // @ToDo: (pequena alteração): list of double double _breakOnTimes;
00142 // @ToDo: (pequena alteração): list of modules _breakOnModules;
00143 bool _stepByStep = false;
00144 bool _pauseOnReplication = false;
00145 bool _pauseOnEvent = false;
00146 bool _initializeStatisticsBetweenReplications = true;
00147 bool _initializeSystem = true;
00148 bool _isRunning = false;
00149 bool _isPaused = false;
00150 bool _pauseRequested = false;
00151 bool _stopRequested = false;
00152 bool _simulationIsInitiated = false;
00153 bool _replicationIsInitiated = false;
00154 bool _hasChanged = false;
00155 private:
00156
00157 const struct DEFAULT_VALUES {
00158 const unsigned int numberOfReplications = 1;
00159 const double replicationLength = 60.0; // by default, 60s = 1min
00160 const Util::TimeUnit replicationLengthTimeUnit = Util::TimeUnit::second;
00161 const Util::TimeUnit replicationBaseTimeUnit = Util::TimeUnit::second;
00162 const Util::TimeUnit replicationReportBaseTimeUnit = Util::TimeUnit::second;
00163 const double warmUpPeriod = 0.0;
00164 const Util::TimeUnit warmUpPeriodTimeUnit = Util::TimeUnit::second;
00165 const std::string terminatingCondition = "";
00166 const bool initializeStatisticsBetweenReplications = true;
00167 const bool initializeSystem = true;
00168 const bool showReportsAfterSimulation = true;
00169 const bool showReportsAfterReplication = true;
00170 const bool showSimulationControlsInReport = false;
00171 const bool showSimulationResponsesInReport = false;
00172 } DEFAULT;
00173 unsigned int _numberOfReplications = DEFAULT.numberOfReplications;
00174 double _replicationLength = DEFAULT.replicationLength;
00175 Util::TimeUnit _replicationLengthTimeUnit = DEFAULT.replicationLengthTimeUnit;
00176 Util::TimeUnit _replicationBaseTimeUnit = DEFAULT.replicationReportBaseTimeUnit;
00177 double _warmUpPeriod = DEFAULT.warmUpPeriod;
00178 Util::TimeUnit _warmUpPeriodTimeUnit = DEFAULT.warmUpPeriodTimeUnit;
00179 std::string _terminatingCondition = DEFAULT.terminatingCondition;
00180 bool _showReportsAfterSimulation = DEFAULT.showReportsAfterSimulation;
00181 bool _showReportsAfterReplication = DEFAULT.showReportsAfterReplication;
00182 bool _showSimulationControlsInReport = DEFAULT.showSimulationControlsInReport;
00183 bool _showSimulationResponsesInReport = DEFAULT.showSimulationResponsesInReport;
00184 //
00185 double _replicationTimeScaleFactorToBase; // a scale that converts ReplicationLengthTimeUnit to
 ReplicationBaseTimeUnit. Future events are in "times" of unit ReplicationBaseTimeUnit
00186 std::chrono::system_clock::time_point _startRealSimulationTimeSimulation;
00187 std::chrono::system_clock::time_point _startRealSimulationTimeReplication;
00188 private:
00189 Event* _currentEvent = nullptr;
00190 unsigned int _currentReplicationNumber;
00191 private:
00192 const std::string _cte_stCountSimulNamePrefix = ""; //Simul.";
00193 private:
00194 Model* _model;
00195 ModelInfo* _info;
00196 SimulationReporter_if* _simulationReporter;
00197 bool _ownsSimulationReporter = true;
00198 List<SimulationControl*>* _ownedControls = new List<SimulationControl*>();
00199 // @ToDo: (importante): Change List below to a MAP, associating every CstatOuCounter in the
 replication to the equivalent in the simulation
00200 List<ModelDataDefinition*>* _cstatsAndCountersSimulation = new List<ModelDataDefinition*>();
00201 std::map<ModelDataDefinition*, ModelDataDefinition*>* _cstatsAndCountersMapSimulation = new
 std::map<ModelDataDefinition*, ModelDataDefinition*>();
00202 List<double*>* _breakpointsOnTime = new List<double*>();
00203 List<ModelComponent*>* _breakpointsOnComponent = new List<ModelComponent*>();
00204 List<Entity*>* _breakpointsOnEntity = new List<Entity*>();
00205 ModelComponent* _justTriggeredBreakpointsOnComponent = nullptr;
00206 Entity* _justTriggeredBreakpointsOnEntity = nullptr;
00207 };
00208 //namespace\\
00209 #endif /* MODELSIMULATION_H */

```

## 16.289 OnEventManager.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */

```

```

00006
00007 /*
00008 * File: OnEventManager.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 7 de Novembro de 2018, 12:28
00012 */
00013
00014 #ifndef ONEVENTMANAGER_H
00015 #define ONEVENTMANAGER_H
00016
00017 #include "../util/List.h"
00018 #include "Event.h"
00019
00020 //namespace GenesysKernel {
00021 /*
00022 * used to get and set values no matter the class (for process analyser)
00023 * should be a wait to invoke a getter or setter no matter the class (a pointer to a member function
 without specifying the class
00024 */
00025 //typedef double (*memberFunctionGetDoubleVarHandler)(); //template<> ... typedef double
 (T::*getDoubleVarHandler)() or something like that
00026 //typedef void (*memberFunctionSetDoubleVarHandler)(double);
00027
00028
00031 class ModelEvent {
00032 public:
00033 ModelEvent(Model* model){
00034 _model = model;
00035 };
00036 public:
00037 const Model* getModel() {
00038 return _model;
00039 }
00040 private:
00041 Model* _model;
00042 };
00043 typedef void (*modelEventHandler)(ModelEvent*);
00044 // for handlers that are class members (methods)
00045 typedef std::function<void(ModelEvent*) > modelEventHandlerMethod;
00046
00047
00050 class SimulationEvent {
00051 public:
00052
00053 void setSimulatedTime(double simulatedTime) {
00054 this->simulatedTime = simulatedTime;
00055 }
00056
00057 double getSimulatedTime() const {
00058 return simulatedTime;
00059 }
00060
00061 void setStopRequested(bool stopRequested) {
00062 this->stopRequested = stopRequested;
00063 }
00064
00065 bool isStopRequested() const {
00066 return stopRequested;
00067 }
00068
00069 void setPauseRequested(bool pauseRequested) {
00070 this->pauseRequested = pauseRequested;
00071 }
00072
00073 bool isPauseRequested() const {
00074 return pauseRequested;
00075 }
00076
00077 void setPaused(bool Paused) {
00078 _isPaused = Paused;
00079 }
00080
00081 bool isPaused() const {
00082 return _isPaused;
00083 }
00084
00085 void setRunning(bool Running) {
00086 _isRunning = Running;
00087 }
00088
00089 bool isRunning() const {
00090 return _isRunning;
00091 }
00092
00093 void setCustomObject(void* customObject) {
00094 this->customObject = customObject;

```

```

00095 }
00096
00097 void* getCustomObject() const {
00098 return customObject;
00099 }
00100
00101 void setCurrentReplicationNumber(unsigned int currentReplicationNumber) {
00102 this->currentReplicationNumber = currentReplicationNumber;
00103 }
00104
00105 unsigned int getCurrentReplicationNumber() const {
00106 return currentReplicationNumber;
00107 }
00108
00109 void setCurrentEvent(Event* currentEvent) {
00110 this->currentEvent = currentEvent;
00111 }
00112
00113 Event* getCurrentEvent() const {
00114 return currentEvent;
00115 }
00116 //void setCurrentInputPortNumber(unsigned int currentInputPortNumber) {
00117 // this->currentInputPortNumber = currentInputPortNumber;
00118 //}
00119 //unsigned int getCurrentInputPortNumber() const {
00120 // return currentInputPortNumber;
00121 //}
00122 //void setCurrentComponent(ModelComponent* currentComponent) {
00123 // this->currentComponent = currentComponent;
00124 //}
00125 //ModelComponent* getCurrentComponent() const {
00126 // return currentComponent;
00127 //}
00128 //void setCurrentEntity(Entity* currentEntity) {
00129 // this->currentEntity = currentEntity;
00130 //}
00131 //Entity* getCurrentEntity() const {
00132 // return currentEntity;
00133 //}
00134
00135 void setEntityCreated(Entity* entityCreated) {
00136 this->entityCreated = entityCreated;
00137 }
00138
00139 Entity* getEntityCreated() const {
00140 return entityCreated;
00141 }
00142
00143 void setDestinationComponent(ModelComponent* destinationComponent) {
00144 this->destinationComponent = destinationComponent;
00145 }
00146
00147 ModelComponent* getDestinationComponent() const {
00148 return destinationComponent;
00149 }
00150
00151 void setEntityMoveTimeDelay(double entityMoveTimeDelay) {
00152 this->entityMoveTimeDelay = entityMoveTimeDelay;
00153 }
00154
00155 double getEntityMoveTimeDelay() const {
00156 return entityMoveTimeDelay;
00157 }
00158 private:
00159 SimulationEvent() {
00160 }
00161 friend class ModelSimulation;
00162 private:
00163 Entity* entityCreated = nullptr;
00164 Event* currentEvent = nullptr;
00165 //Entity* currentEntity = nullptr;
00166 //unsigned int currentInputPortNumber = 0;
00167 //ModelComponent* currentComponent = nullptr;
00168 ModelComponent* destinationComponent = nullptr;
00169 void* customObject = nullptr;
00170 unsigned int currentReplicationNumber = 0;
00171 double simulatedTime = 0.0;
00172 double entityMoveTimeDelay = 0.0;
00173 bool _isRunning = false;
00174 bool _isPaused = false;
00175 bool pauseRequested = false;
00176 bool stopRequested = false;
00177 };
00178
00179 typedef void (*simulationEventHandler)(SimulationEvent*);
00180 // for handlers that are class members (methods)
00181 typedef std::function<void(SimulationEvent*)> simulationEventHandlerMethod;

```



```

00182
00187 class OnEventManager {
00188 public:
00189 OnEventManager();
00190 virtual ~OnEventManager();
00191 public: // event listeners (handlers)
00192 void addOnModelCheckSuccessHandler(modelEventHandler EventHandler);
00193 void addOnModelLoadHandler(modelEventHandler EventHandler);
00194 void addOnModelSaveHandler(modelEventHandler EventHandler);
00195 void addOnReplicationStartHandler(simulationEventHandler EventHandler);
00196 void addOnReplicationStepHandler(simulationEventHandler EventHandler);
00197 void addOnReplicationEndHandler(simulationEventHandler EventHandler);
00198 void addOnProcessEventHandler(simulationEventHandler EventHandler);
00199 void addOnAfterProcessEventHandler(simulationEventHandler EventHandler);
00200 void addOnEntityCreateHandler(simulationEventHandler EventHandler);
00201 void addOnEntityMoveHandler(simulationEventHandler EventHandler);
00202 void addOnEntityRemoveHandler(simulationEventHandler EventHandler);
00203 void addOnSimulationStartHandler(simulationEventHandler EventHandler);
00204 void addOnSimulationPausedHandler(simulationEventHandler EventHandler);
00205 void addOnSimulationResumeHandler(simulationEventHandler EventHandler);
00206 void addOnSimulationEndHandler(simulationEventHandler EventHandler);
00207 void addOnBreakpointHandler(simulationEventHandler EventHandler);
00208 public: // event listeners (method handlers)
00209 // for handlers that are class members (methods)
00210 template<typename Class> void addOnModelCheckSuccessHandler(Class * object, void
00211 (Class::*function) (ModelEvent*));
00211 template<typename Class> void addOnModelLoadHandler(Class * object, void
00212 (Class::*function) (ModelEvent*));
00212 template<typename Class> void addOnModelSaveHandler(Class * object, void
00213 (Class::*function) (ModelEvent*));
00213 template<typename Class> void addOnReplicationStartHandler(Class * object, void
00214 (Class::*function) (SimulationEvent*));
00214 template<typename Class> void addOnReplicationStepHandler(Class * object, void
00215 (Class::*function) (SimulationEvent*));
00215 template<typename Class> void addOnReplicationEndHandler(Class * object, void
00216 (Class::*function) (SimulationEvent*));
00216 template<typename Class> void addOnProcessEventHandler(Class * object, void
00217 (Class::*function) (SimulationEvent*));
00217 template<typename Class> void addOnAfterProcessEventHandler(Class * object, void
00218 (Class::*function) (SimulationEvent*));
00218 template<typename Class> void addOnEntityCreateHandler(Class * object, void
00219 (Class::*function) (SimulationEvent*));
00219 template<typename Class> void addOnEntityMoveHandler(Class * object, void
00220 (Class::*function) (SimulationEvent*));
00220 template<typename Class> void addOnEntityRemoveHandler(Class * object, void
00221 (Class::*function) (SimulationEvent*));
00221 template<typename Class> void addOnSimulationStartHandler(Class * object, void
00222 (Class::*function) (SimulationEvent*));
00222 template<typename Class> void addOnSimulationPausedHandler(Class * object, void
00223 (Class::*function) (SimulationEvent*));
00223 template<typename Class> void addOnSimulationResumeHandler(Class * object, void
00224 (Class::*function) (SimulationEvent*));
00224 template<typename Class> void addOnSimulationEndHandler(Class * object, void
00225 (Class::*function) (SimulationEvent*));
00225 template<typename Class> void addOnBreakpointHandler(Class * object, void
00226 (Class::*function) (SimulationEvent*));
00226
00227 public:
00228 void NotifyModelCheckSuccessHandlers(ModelEvent* se);
00229 void NotifyModelLoadHandlers(ModelEvent* se);
00230 void NotifyModelSaveHandlers(ModelEvent* se);
00231 void NotifyReplicationStartHandlers(SimulationEvent* se);
00232 void NotifyReplicationStepHandlers(SimulationEvent* se);
00233 void NotifyReplicationEndHandlers(SimulationEvent* se);
00234 void NotifyProcessEventHandlers(SimulationEvent* se);
00235 void NotifyAfterProcessEventHandlers(SimulationEvent* se);
00236 void NotifyEntityCreateHandlers(SimulationEvent* se);
00237 void NotifyEntityMoveHandlers(SimulationEvent* se);
00238 void NotifyEntityRemoveHandlers(SimulationEvent* se);
00239 void NotifySimulationStartHandlers(SimulationEvent* se);
00240 void NotifySimulationPausedHandlers(SimulationEvent* se);
00241 void NotifySimulationResumeHandlers(SimulationEvent* se);
00242 void NotifySimulationEndHandlers(SimulationEvent* se);
00243 void NotifyBreakpointHandlers(SimulationEvent* se);
00244
00245 private:
00246 void _NotifyHandlers(List<modelEventHandler>* list, ModelEvent* se);
00247 void _NotifyHandlers(List<simulationEventHandler>* list, SimulationEvent* se);
00248 void _NotifyHandlerMethods(List<modelEventHandlerMethod>* list, ModelEvent* se);
00249 void _NotifyHandlerMethods(List<simulationEventHandlerMethod>* list, SimulationEvent* se);
00250 void _addOnHandler(List<modelEventHandler>* list, modelEventHandler EventHandler);
00251 void _addOnHandler(List<simulationEventHandler>* list, simulationEventHandler EventHandler);
00252 private: // events listener
00253 List<modelEventHandler>* _onModelCheckSuccessHandlers = new List<modelEventHandler>();
00254 List<modelEventHandler>* _onModelLoadHandlers = new List<modelEventHandler>();
00255 List<modelEventHandler>* _onModelSaveHandlers = new List<modelEventHandler>();
00256 List<simulationEventHandler>* _onReplicationStartHandlers = new List<simulationEventHandler>();

```

```

00257 List<simulationEventHandler>* _onReplicationStepHandlers = new List<simulationEventHandler>();
00258 List<simulationEventHandler>* _onReplicationEndHandlers = new List<simulationEventHandler>();
00259 List<simulationEventHandler>* _onProcessEventHandlers = new List<simulationEventHandler>();
00260 List<simulationEventHandler>* _onAfterProcessEventHandlers = new List<simulationEventHandler>();
00261 List<simulationEventHandler>* _onEntityCreateHandlers = new List<simulationEventHandler>();
00262 List<simulationEventHandler>* _onEntityMoveHandlers = new List<simulationEventHandler>();
00263 List<simulationEventHandler>* _onEntityRemoveHandlers = new List<simulationEventHandler>();
00264 List<simulationEventHandler>* _onSimulationStartHandlers = new List<simulationEventHandler>();
00265 List<simulationEventHandler>* _onSimulationPausedHandlers = new List<simulationEventHandler>();
00266 List<simulationEventHandler>* _onSimulationResumeHandlers = new List<simulationEventHandler>();
00267 List<simulationEventHandler>* _onSimulationEndHandlers = new List<simulationEventHandler>();
00268 List<simulationEventHandler>* _onBreakpointHandlers = new List<simulationEventHandler>();
00269 private: // events listener for handlers that are class members (methods)
00270 List<modelEventHandlerMethod>* _onModelCheckSuccessHandlerMethods = new
List<modelEventHandlerMethod>();
00271 List<modelEventHandlerMethod>* _onModelLoadHandlerMethods = new List<modelEventHandlerMethod>();
00272 List<modelEventHandlerMethod>* _onModelSaveHandlerMethods = new List<modelEventHandlerMethod>();
00273 List<simulationEventHandlerMethod>* _onReplicationStartHandlerMethods = new
List<simulationEventHandlerMethod>();
00274 List<simulationEventHandlerMethod>* _onReplicationStepHandlerMethods = new
List<simulationEventHandlerMethod>();
00275 List<simulationEventHandlerMethod>* _onReplicationEndHandlerMethods = new
List<simulationEventHandlerMethod>();
00276 List<simulationEventHandlerMethod>* _onProcessEventHandlerMethods = new
List<simulationEventHandlerMethod>();
00277 List<simulationEventHandlerMethod>* _onAfterProcessEventHandlerMethods = new
List<simulationEventHandlerMethod>();
00278 List<simulationEventHandlerMethod>* _onEntityCreateHandlerMethods = new
List<simulationEventHandlerMethod>();
00279 List<simulationEventHandlerMethod>* _onEntityMoveHandlerMethods = new
List<simulationEventHandlerMethod>();
00280 List<simulationEventHandlerMethod>* _onEntityRemoveHandlerMethods = new
List<simulationEventHandlerMethod>();
00281 List<simulationEventHandlerMethod>* _onSimulationStartHandlerMethods = new
List<simulationEventHandlerMethod>();
00282 List<simulationEventHandlerMethod>* _onSimulationPausedHandlerMethods = new
List<simulationEventHandlerMethod>();
00283 List<simulationEventHandlerMethod>* _onSimulationResumeHandlerMethods = new
List<simulationEventHandlerMethod>();
00284 List<simulationEventHandlerMethod>* _onSimulationEndHandlerMethods = new
List<simulationEventHandlerMethod>();
00285 List<simulationEventHandlerMethod>* _onBreakpointHandlerMethods = new
List<simulationEventHandlerMethod>();
00286 };
00287
00288 //
00289 // implementation for template methods
00290 //
00291 // // @ToDo: (importante): (!!) Complicated: if handlerMethod already insert, should not insert it
again. Problem to solve <...> for function
00292 //if (_onProcessEventHandlerMethods->find(handlerMethod) ==
_onProcessEventHandlerMethods->list()->end())
00293 // trying unique to solve the issue
00294 //this->_onProcessEventHandlerMethods->list()->unique(); // does not work
00295 // @ToDo: (importante): probably to override == operator for type simulationEventHandlerMethod
00296 // ...
00297
00298 template<typename Class> void OnEventManager::addOnModelCheckSuccessHandler(Class * object, void
(Class::*function) (ModelEvent*)) {
00299 modelEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00300 this->_onModelCheckSuccessHandlerMethods->insert(handlerMethod);
00301 }
00302
00303 template<typename Class> void OnEventManager::addOnModelLoadHandler(Class * object, void
(Class::*function) (ModelEvent*)) {
00304 modelEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00305 this->_onModelLoadHandlerMethods->insert(handlerMethod);
00306 }
00307
00308 template<typename Class> void OnEventManager::addOnModelSaveHandler(Class * object, void
(Class::*function) (ModelEvent*)) {
00309 modelEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00310 this->_onModelSaveHandlerMethods->insert(handlerMethod);
00311 }
00312 template<typename Class> void OnEventManager::addOnReplicationStartHandler(Class * object, void
(Class::*function) (SimulationEvent*)) {
00313 simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00314 this->_onReplicationStartHandlerMethods->insert(handlerMethod);
00315 }
00316
00317 template<typename Class> void OnEventManager::addOnReplicationStepHandler(Class * object, void
(Class::*function) (SimulationEvent*)) {
00318 simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00319 this->_onReplicationStepHandlerMethods->insert(handlerMethod);
00320 }
00321
00322 template<typename Class> void OnEventManager::addOnReplicationEndHandler(Class * object, void

```

```

 (Class::*function)(SimulationEvent*)) {
00323 simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00324 this->_onReplicationEndHandlerMethods->insert(handlerMethod);
00325 }
00326
00327 template<typename Class> void OnEventManager::addOnProcessEventHandler(Class * object, void
 (Class::*function)(SimulationEvent*)) {
00328 simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00329 this->_onProcessEventHandlerMethods->insert(handlerMethod);
00330 }
00331
00332 template<typename Class> void OnEventManager::addOnAfterProcessEventHandler(Class * object, void
 (Class::*function)(SimulationEvent*)) {
00333 simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00334 this->_onAfterProcessEventHandlerMethods->insert(handlerMethod);
00335 }
00336
00337 template<typename Class> void OnEventManager::addOnEntityCreateHandler(Class * object, void
 (Class::*function)(SimulationEvent*)) {
00338 simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00339 this->_onEntityCreateHandlerMethods->insert(handlerMethod);
00340 }
00341
00342 template<typename Class> void OnEventManager::addOnEntityMoveHandler(Class * object, void
 (Class::*function)(SimulationEvent*)) {
00343 simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00344 this->_onEntityMoveHandlerMethods->insert(handlerMethod);
00345 }
00346
00347 template<typename Class> void OnEventManager::addOnEntityRemoveHandler(Class * object, void
 (Class::*function)(SimulationEvent*)) {
00348 simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00349 this->_onEntityRemoveHandlerMethods->insert(handlerMethod);
00350 }
00351
00352 template<typename Class> void OnEventManager::addOnSimulationStartHandler(Class * object, void
 (Class::*function)(SimulationEvent*)) {
00353 simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00354 this->_onSimulationStartHandlerMethods->insert(handlerMethod);
00355 }
00356
00357 template<typename Class> void OnEventManager::addOnSimulationPausedHandler(Class * object, void
 (Class::*function)(SimulationEvent*)) {
00358 simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00359 this->_onSimulationPausedHandlerMethods->insert(handlerMethod);
00360 }
00361
00362 template<typename Class> void OnEventManager::addOnSimulationResumeHandler(Class * object, void
 (Class::*function)(SimulationEvent*)) {
00363 simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00364 this->_onSimulationResumeHandlerMethods->insert(handlerMethod);
00365 }
00366
00367 template<typename Class> void OnEventManager::addOnSimulationEndHandler(Class * object, void
 (Class::*function)(SimulationEvent*)) {
00368 simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00369 this->_onSimulationEndHandlerMethods->insert(handlerMethod);
00370 }
00371
00372 template<typename Class> void OnEventManager::addOnBreakpointHandler(Class * object, void
 (Class::*function)(SimulationEvent*)) {
00373 simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00374 this->_onBreakpointHandlerMethods->insert(handlerMethod);
00375 }
00376
00377
00378 /*
00379 Private Sub ModelLogic_DocumentOpen()
00380 Private Sub ModelLogic_DocumentSave()
00381 Private Sub ModelLogic_OnClearStatistics()
00382 Private Sub ModelLogic_OnFileClose(ByVal filesElementNumber As Long, ByVal fileName As String)
00383 Private Sub ModelLogic_OnFileRead(ByVal filesElementNumber As Long, ByVal fileName As String, ByVal
 numberOfItems As Long, readBuffer() As Double)
00384 Private Sub ModelLogic_OnFileReadAsVariant(ByVal filesElementNumber As Long, ByVal fileName As String,
 ByVal numberOfItems As Long, readBuffer() As Variant)
00385 Private Sub ModelLogic_OnFileWrite(ByVal filesElementNumber As Long, ByVal fileName As String, ByVal
 numberOfItems As Long, writeBuffer() As Double)
00386 Private Sub ModelLogic_OnFileWriteAsVariant(ByVal filesElementNumber As Long, ByVal fileName As
 String, ByVal numberOfItems As Long, writeBuffer() As Variant)
00387 Private Sub ModelLogic_OnKeystroke(ByVal keyCode As Long)
00388 Private Function ModelLogic_RealTimeGetMaxTimeAdvance() As Long
00389 Private Function ModelLogic_RealTimeInitialize(ByVal processName As String, ByVal remoteProcessName As
 String) As Long
00390 Private Sub ModelLogic_RealTimeInitializeMaxTimeAdvance()
00391 Private Function ModelLogic_RealTimeReceive(message As String) As Long
00392 Private Function ModelLogic_RealTimeSend(ByVal message As String) As Long
00393 Private Function ModelLogic_RealTimeTerminate() As Long

```

```

00394 Private Sub ModelLogic_RunBegin()
00395 Private Sub ModelLogic_RunBeginReplication()
00396 Private Sub ModelLogic_RunBeginSimulation()
00397 Private Sub ModelLogic_RunBreak()
00398 Private Sub ModelLogic_RunEnd()
00399 Private Sub ModelLogic_RunEndReplication()
00400 Private Sub ModelLogic_RunEndSimulation()
00401 Private Sub ModelLogic_RunFastForward()
00402 Private Sub ModelLogic_RunPause()
00403 Private Sub ModelLogic_RunRestart()
00404 Private Sub ModelLogic_RunResume()
00405 Private Sub ModelLogic_RunStep()
00406 Private Sub ModelLogic_SimanError(ByVal routineName As String, msgType As Long, msgNum As Long)
00407 Private Sub ModelLogic_UserContinuousEquations()
00408 Private Function ModelLogic_UserFunction(ByVal entityID As Long, ByVal functionID As Long) As Double
00409 Private Function ModelLogic_UserRule(ByVal entityID As Long, ByVal ruleID As Long) As Double
00410 Private Sub VBA_Block_1_Fire()
00411 */
00412
00413 //namespace\\}
00414 #endif /* ONEVENTMANAGER_H */

```

## 16.290 Parser\_if.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Parser_if.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 23 de Agosto de 2018, 15:57
00012 */
00013
00014 #ifndef PARSER_IF_H
00015 #define PARSER_IF_H
00016
00017 #include <string>
00018 #include "../statistics/Sampler_if.h"
00019
00020 class genesyspp_driver;
00021
00022 class Parser_if {
00023 public:
00024 virtual ~Parser_if() = default;
00025 virtual double parse(const std::string expression) = 0; // may throw exception
00026 virtual double parse(const std::string expression, bool& success, std::string& errorMessage) = 0;
00027 // does not throw exceptions
00028 virtual std::string getErrorMessage() = 0; // to get error message in the case of thrown exception
00029 virtual void setSampler(Sampler_if* sampler) = 0;
00030 virtual Sampler_if* getSampler() const = 0;
00031 // @ToDo: (pequena alteração): Implement a method to get the TOKENS parsed. Example: If parsing
00032 "nq(queue1) < var1" return a list containing something like "fNQ tLPAR eQUEUE tRPAR tLESS eVARIABLE"
00033 virtual genesyspp_driver getParser() const = 0;
00034 };
00035
00036 #endif /* PARSER_IF_H */

```

## 16.291 ParserChangesInformation.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: ParserChangesInformation.h
00009 * Author: rlcancian
00010 *
00011 * Created on 11 de Setembro de 2019, 20:42
00012 */
00013
00014 #ifndef PARSERCHANGESINFORMATION_H
00015 #define PARSERCHANGESINFORMATION_H
00016
00017 #include <string>
00018 #include <list>
00019

```

```

00020 class ParserChangesInformation {
00021 public:
00022 ParserChangesInformation();
00023 virtual ~ParserChangesInformation() = default;
00024
00025 public: // gets and sets
00026 // @ToDo: (importante): Review the consistency of this class public API names.
00027 // There are mixed naming patterns such as getIncludes() and getFunctionProductions().
00028 // Any rename may break existing callers and should be planned carefully.
00029 std::string getIncludes() const;
00030 void setIncludes(const std::string &newIncludes);
00031
00032 std::string getTokens() const;
00033 void setTokens(const std::string &newTokens);
00034
00035 std::string getTypeObjs() const;
00036 void setTypeObjs(const std::string &newTypeObjs);
00037
00038 std::string getExpressions() const;
00039 void setExpressions(const std::string &newExpressions);
00040
00041 std::string getExpressionProductions() const;
00042 void setExpressionProductions(const std::string &newExpressionProductions);
00043
00044 std::string getAssignments() const;
00045 void setAssignments(const std::string &newAssignments);
00046
00047 // @ToDo: (importante): Confirm whether "Productions" should actually be "Production".
00048 // Do not rename it directly without mapping impacts on existing callers.
00049 std::string getFunctionProductions() const;
00050 void setFunctionProductions(const std::string &newFunctionProductions);
00051
00052 private:
00053 std::string _includes = "";
00054 std::string _tokens = "";
00055 std::string _typeObjs = "";
00056 std::string _expressions = "";
00057 std::string _expressionProductions = "";
00058 std::string _assignments = "";
00059 std::string _functionProductions = "";
00060
00061 };
00062
00063 #endif /* PARSECHANGESINFORMATION_H */

```

## 16.292 ParserDefaultImpl1.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: ParserDefaultImpl1.h
00009 * Author: rafael.lui.cancian
00010 *
00011 * Created on 2 de Agosto de 2018, 01:25
00012 */
00013
00014 #ifndef PARSEDEFAULTIMPL1_H
00015 #define PARSEDEFAULTIMPL1_H
00016
00017 #include "Parser_if.h"
00018 //namespace GenesysKernel {
00019 class Model;
00020
00021 class ParserDefaultImpl1 : public Parser_if {
00022 public:
00023 ParserDefaultImpl1(Model* model);
00024 virtual ~ParserDefaultImpl1() = default;
00025 public:
00026 virtual double parse(const std::string expression) override; // may throw exception
00027 virtual double parse(const std::string expression, bool& success, std::string& errorMessage)
00028 override;
00029 virtual std::string getErrorMessage() override;
00030 private:
00031 Model* _model;
00032 };
00033 //namespace\\}
00034 #endif /* PARSEDEFAULTIMPL1_H */

```

## 16.293 ParserDefaultImpl2.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: ParserDefaultImpl2.h
00009 * Author: Joao Ortigara (20180801-20181207) rafael.luiz.cancian (20181208-...(
00010 *
00011 *
00012 */
00013
00014 #ifndef PARSERDEFAULTIMPL2_H
00015 #define PARSERDEFAULTIMPL2_H
00016
00017 #include <string>
00018 #include "Parser_if.h"
00019 #include "model/Model.h"
00020 #include "../parser/Genesys++-driver.h"
00021
00022
00023 //namespace GenesysKernel {
00024
00025 class ParserDefaultImpl2 : public Parser_if {
00026 public:
00034 ParserDefaultImpl2(Model* model, Sampler_if* sampler, bool throws = false);
00042 virtual ~ParserDefaultImpl2();
00043 public:
00044 virtual double parse(const std::string expression) override; // may throw exception
00045 virtual double parse(const std::string expression, bool& success, std::string& errorMessage)
00046 override;
00046 virtual std::string getErrorMessage() override;
00054 virtual void setSampler(Sampler_if* _sampler) override;
00055 virtual Sampler_if* getSampler() const override;
00056 virtual genesyspp_driver getParser() const override;
00057 private:
00058 void _setSamplerInternal(Sampler_if* sampler, bool ownsSampler);
00059 private:
00060 Model* _model;
00061 genesyspp_driver _wrapper;
00062 bool _ownsSampler = false;
00063 };
00064 //namespace\}
00065 #endif /* PARSERDEFAULTIMPL2_H */

```

## 16.294 ParserManager.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: ParserManager.h
00009 * Author: rlcancian
00010 *
00011 * Created on 11 de Setembro de 2019, 20:34
00012 */
00013
00014 #ifndef PARSERMANAGER_H
00015 #define PARSERMANAGER_H
00016
00017 #include "ParserChangesInformation.h"
00018
00019 //namespace GenesysKernel {
00020
00021 class ParserManager {
00022 public:
00023
00027 struct NewParser {
00028 std::string bisonFilename;
00029 std::string flexFilename;
00030 std::string compiledParserFilename;
00031 };
00032
00036 struct GenerateNewParserResult {
00037 bool result = false;
00038 std::string bisonMessages;
00039 std::string lexMessages;
00040 std::string compilationMessages;

```

```

00041 NewParser newParser;
00042 };
00043 public:
00044 ParserManager();
00045 virtual ~ParserManager() = default;
00046 public:
00052 // @ToDo: (importante): Implement parser generation workflow and define the expected
 filesystem/toolchain contract.
00053 ParserManager::GenerateNewParserResult generateNewParser(ParserChangesInformation* changes);
00059 // @ToDo: (importante): Implement parser loading/activation and define ownership/lifetime rules
 for connected parsers.
00060 bool connectNewParser(ParserManager::NewParser newParser);
00061 private:
00062 };
00063 //namespace\\
00064 #endif /* PARSEMANAGER_H */

```

## 16.295 Persistence.h

```

00001 #ifndef PERSISTENCE_H
00002 #define PERSISTENCE_H
00003
00004 #include <string>
00005 #include <utility>
00006 #include <unordered_map>
00007 #include <memory>
00008
00009 #include "../util/Util.h"
00010 #include "persistence/Persistence_if.h"
00011
00012
00022 class PersistenceRecord {
00023 public:
00024 using Payload = std::string;
00026 struct Entry {
00027 enum Kind { text, numeric };
00028 std::string first;
00029 Payload second;
00030 Kind kind{Kind::text};
00031 };
00032 using Iterator = std::unordered_map<std::string, Entry>::iterator;
00033
00034 // this is the actual abstract interface
00035
00036 virtual ~PersistenceRecord();
00037
00038 virtual PersistenceRecord* newInstance();
00039
00040 virtual std::string loadField(const std::string& key, const std::string& defaultValue = "");
00041 virtual double loadField(const std::string& key, double defaultValue);
00042 virtual unsigned int loadField(const std::string& key, unsigned int defaultValue);
00043 virtual int loadField(const std::string& key, int defaultValue);
00044 virtual Util::TimeUnit loadField(const std::string& key, Util::TimeUnit defaultValue);
00045
00046 virtual void saveField(const std::string& key, const std::string& value, const std::string&
 defaultValue, const bool saveIfDefault = false);
00047 virtual void saveField(const std::string& key, const std::string& value);
00048 virtual void saveField(const std::string& key, double value, const double defaultValue, const bool
 saveIfDefault = false);
00049 virtual void saveField(const std::string& key, unsigned int value, const unsigned int
 defaultValue, const bool saveIfDefault = false);
00050 virtual void saveField(const std::string& key, unsigned int value);
00051 virtual void saveField(const std::string& key, int value, const int defaultValue, const bool
 saveIfDefault = false);
00052 virtual void saveField(const std::string& key, Util::TimeUnit value, const Util::TimeUnit
 defaultValue, const bool saveIfDefault = false);
00053
00054 // these are only needed because existing code does not abstract away the STL container API
00055 // still, subclasses implementing this interface may use them to insert post-processed entries
00056
00057 explicit PersistenceRecord(Persistence_if& config);
00058
00060 std::size_t size() const;
00061
00063 void insert(Entry entry);
00064
00066 void insert(Iterator it, Iterator end);
00067
00069 void erase(const std::string& key);
00070
00072 void clear();
00073
00075 Iterator find(const std::string& key);
00076
00078 Iterator begin();

```



```

00079
00081 Iterator end();
00082
00083 private:
00084 std::unordered_map<std::string, Entry> _fields;
00085 Persistence_if& _config;
00086 };
00087
00088
00094 class PersistentObject_base {
00095 public:
00096 virtual ~PersistentObject_base() {};
00097
00098 protected:
00106 virtual bool _loadInstance(PersistenceRecord *fields) = 0;
00107
00114 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues = false) = 0;
00115 };
00116
00117 #endif /* PERSISTENCE_H */

```

## 16.296 CppSerializer.h

```

00001 #ifndef CPPSERIALIZER_H
00002 #define CPPSERIALIZER_H
00003
00004 #include <unordered_map>
00005 #include<memory>
00006
00007 #include "Serializer.h"
00008 #include "../model/Model.h"
00009
00010 class CppSerializer : public Serializer {
00011 public:
00012 explicit CppSerializer(Model *model);
00013
00014 public: // Serializer interface
00015 PersistenceRecord* newPersistenceRecord() override;
00016 bool dump(std::ostream& output) override;
00017 bool load(std::istream& input) override;
00018 bool get(const std::string& name, PersistenceRecord *entry) override;
00019 bool put(const std::string name, const std::string type, const Util::identification id,
00020 PersistenceRecord *fields) override;
00021 int for_each(std::function<int(const std::string&) > delegate) override;
00022 private:
00023 Model *_model{};
00024 std::unordered_map<std::string, std::unique_ptr<PersistenceRecord> _metaobjects
00025 {
00026 };
00027 std::unordered_map<std::string, std::unique_ptr<PersistenceRecord> _components
00028 {
00029 };
00030 };
00031
00032 #endif // CPPSERIALIZER_H

```

## 16.297 GenSerializer.h

```

00001 #ifndef GENSERIALIZER_H
00002 #define GENSERIALIZER_H
00003
00004 #include <unordered_map>
00005 #include<memory>
00006
00007 #include "Serializer.h"
00008 #include "../model/Model.h"
00009
00010 class GenSerializer : public Serializer {
00011 public:
00012 explicit GenSerializer(Model *model);
00013
00014 public: // Serializer interface
00015 PersistenceRecord* newPersistenceRecord() override;
00016 bool dump(std::ostream& output) override;
00017 bool load(std::istream& input) override;
00018 bool get(const std::string& name, PersistenceRecord *entry) override;
00019 bool put(const std::string name, const std::string type, const Util::identification id,
00020 PersistenceRecord *fields) override;
00021 int for_each(std::function<int(const std::string&) > delegate) override;
00022 private:

```



```

00023 static std::string linearize(PersistenceRecord *fields);
00024 friend class PersistenceDefaultImpl2;
00025
00026 private:
00027 Model *_model{};
00028 std::unordered_map<std::string, std::unique_ptr<PersistenceRecord> _components
00029 {
00030 };
00031 };
00032
00033 #endif // GENSERIALIZER_H

```

## 16.298 JsonSerializer.h

```

00001 #ifndef JSONSERIALIZER_H
00002 #define JSONSERIALIZER_H
00003
00004 #include <unordered_map>
00005 #include <memory>
00006
00007 #include "Serializer.h"
00008 #include "../model/Model.h"
00009
00010 class JsonSerializer : public Serializer {
00011 public:
00012 explicit JsonSerializer(Model *model);
00013
00014 public: // Serializer interface
00015 PersistenceRecord* newPersistenceRecord() override;
00016 bool dump(std::ostream& output) override;
00017 bool load(std::istream& input) override;
00018 bool get(const std::string& name, PersistenceRecord *entry) override;
00019 bool put(const std::string name, const std::string type, const Util::identification id,
PersistenceRecord *fields) override;
00020 int for_each(std::function<int(const std::string&) > delegate) override;
00021
00022 private:
00023 Model *_model{};
00024 std::unordered_map<std::string, std::unique_ptr<PersistenceRecord> _metaobjects
00025 {
00026 };
00027 std::unordered_map<std::string, std::unique_ptr<PersistenceRecord> _components
00028 {
00029 };
00030 };
00031
00032 #endif // JSONSERIALIZER_H

```

## 16.299 Persistence\_if.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Persistence_if.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 24 de Agosto de 2018, 19:22
00012 */
00013
00014 #ifndef MODELPERSISTENCE_IF_H
00015 #define MODELPERSISTENCE_IF_H
00016
00017 #include <string>
00018 #include <map>
00019
00020
00021 // forward decl
00022 class PersistenceRecord;
00023
00024
00025 class Persistence_if {
00026 public:
00027 virtual ~Persistence_if() = default;
00028
00029 enum class Options : int {
00030 SAVEDEFAULTS = 1, HIDEIDKEY = 2, HIDETYPEKEY = 4, HIDENAMEKEY = 8, SORTALPHLY = 16
00031 };
00032
00033 public:

```

```

00041 virtual bool save(std::string filename) = 0;
00043 virtual bool load(std::string filename) = 0;
00044
00045 public:
00047 virtual bool hasChanged() = 0;
00049 virtual void setHasChanged(bool hasChanged) = 0;
00051 virtual bool getOption(Persistence_if::Options option) = 0;
00053 virtual void setOption(Persistence_if::Options option, bool value) = 0;
00054 public:
00056 virtual std::string getFormattedField(PersistenceRecord *fields) = 0;
00057 };
00058
00059 #endif /* MODELPERISTENCE_IF_H */

```

## 16.300 PersistenceDefaultImpl2.h

```

00001 #ifndef MODELPERISTENCEDEFAULTIMPL_H
00002 #define MODELPERISTENCEDEFAULTIMPL_H
00003
00004 #include "Persistence_if.h"
00005 #include "../model/Model.h"
00006
00007 class PersistenceDefaultImpl2 : public Persistence_if {
00008 public:
00009 PersistenceDefaultImpl2(Model* model);
00010
00011 public: // Persistence_if interface
00012 bool save(std::string filename) override;
00013 bool load(std::string filename) override;
00014 bool hasChanged() override;
00015 void setHasChanged(bool hasChanged) override;
00016 bool getOption(Persistence_if::Options option) override;
00017 void setOption(Persistence_if::Options option, bool value) override;
00018 std::string getFormattedField(PersistenceRecord *fields) override;
00019
00020 private:
00021 Model* _model;
00022 unsigned short _options{0};
00023 bool _dirty{false};
00024 };
00025
00026 #endif /* MODELPERISTENCEDEFAULTIMPL_H */

```

## 16.301 PersistencePartialLoadImpl1.h

```

00001 #ifndef MODELPERISTENCEPARTIALLOADIMPL1_H
00002 #define MODELPERISTENCEPARTIALLOADIMPL1_H
00003
00004 #include "PersistenceDefaultImpl2.h"
00005 #include "../model/Model.h"
00006
00007 class PersistencePartialLoadImpl1 : public PersistenceDefaultImpl2 {
00008 public:
00009 explicit PersistencePartialLoadImpl1(Model* model);
00010
00011 public:
00012 bool load(std::string filename) override;
00013
00014 private:
00015 Model* _model;
00016 };
00017
00018 #endif /* MODELPERISTENCEPARTIALLOADIMPL1_H */

```

## 16.302 Serializer.h

```

00001 #ifndef MODELSERIALIZER_H
00002 #define MODELSERIALIZER_H
00003
00004 #include <iostream>
00005 #include <functional>
00006
00007 #include "../Persistence.h"
00008 #include "../util/Util.h"
00009
00023 class Serializer {
00024 public:
00025 virtual ~Serializer();
00026

```

```

00033 virtual PersistenceRecord* newPersistenceRecord() = 0;
00034
00042 virtual bool dump(std::ostream& output) = 0;
00043
00049 virtual bool load(std::istream& input) = 0;
00050
00059 virtual bool get(const std::string& name, PersistenceRecord *entry) = 0;
00060
00071 virtual bool put(const std::string name, const std::string type, const Util::identification id,
PersistenceRecord *fields) = 0;
00072
00080 virtual int for_each(std::function<int(const std::string&) > delegate) = 0;
00081 };
00082
00083 #endif // MODELSerializer_H

```

## 16.303 XmlSerializer.h

```

00001 #ifndef XMLSERIALIZER_H
00002 #define XMLSERIALIZER_H
00003
00004 #include <unordered_map>
00005 #include<memory>
00006
00007 #include "Serializer.h"
00008 #include "../model/Model.h"
00009
00010 class XmlSerializer : public Serializer {
00011 public:
00012 explicit XmlSerializer(Model *model);
00013
00014 public: // Serializer interface
00015 PersistenceRecord* newPersistenceRecord() override;
00016 bool dump(std::ostream& output) override;
00017 bool load(std::istream& input) override;
00018 bool get(const std::string& name, PersistenceRecord *entry) override;
00019 bool put(const std::string name, const std::string type, const Util::identification id,
PersistenceRecord *fields) override;
00020 int for_each(std::function<int(const std::string&) > delegate) override;
00021
00022 private:
00023 Model *_model{};
00024 std::unordered_map<std::string, std::unique_ptr<PersistenceRecord> _metaobjects
00025 {
00026 };
00027 std::unordered_map<std::string, std::unique_ptr<PersistenceRecord> _components
00028 {
00029 };
00030 };
00031
00032 #endif // XMLSERIALIZER_H

```

## 16.304 Plugin.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Plugin.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 21 de Junho de 2018, 12:58
00012 */
00013
00014 #ifndef PLUGIN_H
00015 #define PLUGIN_H
00016
00017 #include <string>
00018 #include <functional>
00019 #include <list>
00020
00021 #include "../util/Util.h"
00022 #include "Persistence.h"
00023 #include "PluginInformation.h"
00024
00025 //namespace GenesysKernel {
00026
00031 class Plugin {
00032 public:

```

```

00037 Plugin(std::string filename_so_dll);
00042 Plugin(StaticGetPluginInformation getInformation); // @ToDo: (importante): temporary. Just while
compiled together
00046 virtual ~Plugin();
00047 public:
00052 std::string show();
00053 public:
00058 bool isValidPlugin() const;
00063 PluginInformation* getPluginInfo() const;
00072 ModelDataDefinition* loadNew(Model* model, PersistenceRecord *fields);
00079 bool loadAndInsertNew(Model* model, PersistenceRecord *fields);
00086 ModelDataDefinition* newInstance(Model* model, std::string name = "");
00087 private:
00088 ModelComponent* _loadNewComponent(Model* model, PersistenceRecord *fields);
00090 ModelDataDefinition* _loadNewElement(Model* model, PersistenceRecord *fields);
00091 private: // read only
00093 // Default invalid state avoids reading indeterminate validity after construction failures.
00094 bool _isValidPlugin = false;
00095 // Store plugin metadata pointer with a safe default for invalid plugin construction paths.
00096 PluginInformation* _pluginInfo = nullptr;
00097 private:
00098 StaticGetPluginInformation _StatMethodGetInformation;
00099 };
00100 //namespace\}
00101 #endif /* PLUGIN_H */

```

## 16.305 PluginConnector\_if.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: PluginConnector_if.h
00009 * Author: rlcancian
00010 *
00011 * Created on 9 de Setembro de 2019, 19:17
00012 */
00013
00014 #ifndef PLUGINCONNECTOR_IF_H
00015 #define PLUGINCONNECTOR_IF_H
00016
00017 #include<string>
00018 #include "Plugin.h"
00019 #include "../util/List.h"
00020
00021 //namespace GenesysKernel {
00022
00029 class PluginConnector_if {
00030 public:
00031 virtual ~PluginConnector_if() = default;
00038 virtual Plugin* check(const std::string dynamicLibraryFilename) = 0;
00045 virtual Plugin* connect(const std::string dynamicLibraryFilename) = 0;
00052 virtual List<std::string>* find() = 0;
00053
00060 virtual bool disconnect(const std::string dynamicLibraryFilename) = 0;
00067 virtual bool disconnect(Plugin* plugin) = 0;
00068 };
00069 //namespace\}
00070 #endif /* PLUGINCONNECTOR_IF_H */

```

## 16.306 PluginInformation.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: PluginInformation.h
00009 * Author: rlcancian
00010 *
00011 * Created on 9 de Setembro de 2019, 20:02
00012 */
00013
00014 #ifndef PLUGININFORMATION_H
00015 #define PLUGININFORMATION_H

```

```

00016
00017 #include <map>
00018 #include <list>
00019 #include <string>
00020
00021 //namespace GenesysKernel {
00022 class ModelDataDefinition;
00023 class ModelComponent;
00024 class Model;
00025 class ModelDataManager;
00026 class PersistenceRecord;
00027
00028 typedef ModelComponent* (*StaticLoaderComponentInstance)(Model*, PersistenceRecord*);
00029 typedef ModelDataDefinition* (*StaticLoaderDataDefinitionInstance)(Model*, PersistenceRecord*);
00030 typedef ModelDataDefinition* (*StaticConstructorDataDefinitionInstance)(Model*, std::string);
00031 class PluginInformation;
00032 typedef PluginInformation* (*StaticGetPluginInformation)();
00033
00034 class SystemDependency {
00035 public:
00036 enum class OS {
00037 Any,
00038 Linux,
00039 Windows,
00040 MacOS,
00041 Unknown
00042 };
00043
00044 SystemDependency() = default;
00045 SystemDependency(OS os, std::string name, std::string installCommand = "", std::string
checkCommand = "");
00046
00047 OS getOS() const;
00048 std::string getName() const;
00049 std::string getInstallCommand() const;
00050 std::string getCheckCommand() const;
00051 std::string show() const;
00052
00053 static std::string osToString(OS os);
00054
00055 private:
00056 OS _os = OS::Any;
00057 std::string _name = "";
00058 std::string _installCommand = "";
00059 std::string _checkCommand = "";
00060 };
00061
00062 class PluginInformation {
00063 public:
00064 PluginInformation(std::string pluginTypeName, StaticLoaderComponentInstance componentloader,
StaticConstructorDataDefinitionInstance elementConstructor);
00065 PluginInformation(std::string pluginTypeName, StaticLoaderDataDefinitionInstance elementloader,
StaticConstructorDataDefinitionInstance elementConstructor);
00066 // Release owned metadata containers allocated alongside plugin-information objects.
00067 virtual ~PluginInformation();
00068 public:
00069 // gets
00070 StaticLoaderDataDefinitionInstance getDataDefinitionLoader() const;
00071 StaticLoaderComponentInstance GetComponentLoader() const;
00072 StaticConstructorDataDefinitionInstance getDataDefinitionConstructor() const;
00073 bool isGenerateReport() const;
00074 bool isComponent() const;
00075 bool isSendTransfer() const;
00076 bool isReceiveTransfer() const;
00077 bool isSink() const;
00078 bool isSource() const;
00079 std::string getObservation() const;
00080 std::string getVersion() const;
00081 std::string getDate() const;
00082 std::string getAuthor() const;
00083 std::string getPluginTypeName() const;
00084 // sets
00085 void insertDynamicLibFileDependence(std::string filename);
00086 // @ToDo: (importante) Clarify ownership for the incoming list pointer.
00087 // This setter currently replaces the internal raw pointer without documenting transfer semantics.
00088 void setDynamicLibFilenameDependencies(std::list<std::string>* dynamicLibFilenameDependencies);
00089 std::list<std::string>* getDynamicLibFilenameDependencies() const;
00090 void insertSystemDependency(const SystemDependency& dependency);
00091 const std::list<SystemDependency>* getSystemDependencies() const;
00092 bool hasSystemDependencies() const;
00093 void setGenerateReport(bool generateReport);
00094 void setSendTransfer(bool sendTransfer);
00095 void setReceiveTransfer(bool receiveTransfer);
00096 void setSink(bool Sink);
00097 void setSource(bool Source);
00098 void setObservation(std::string observation);
00099 void setVersion(std::string version);

```

```

00100 void setDate(std::string date);
00101 void setAuthor(std::string author);
00102 void setMaximumOutputs(unsigned short _maximumOutputs);
00103 unsigned short getMaximumOutputs() const;
00104 void setMinimumOutputs(unsigned short _minimumOutputs);
00105 unsigned short getMinimumOutputs() const;
00106 void setMaximumInputs(unsigned short _maximumInputs);
00107 unsigned short getMaximumInputs() const;
00108 void setMinimumInputs(unsigned short _minimumInputs);
00109 unsigned short getMinimumInputs() const;
00110 void setDescriptionHelp(std::string _descriptionHelp);
00111 std::string getDescriptionHelp() const;
00112 // @ToDo: (importante): Clarify ownership for the incoming fields map.
00113 // This setter currently replaces the internal raw pointer without documenting lifetime
 expectations.
00114 void setFields(std::map<std::string, std::string>* _fiewlds);
00115 std::map<std::string, std::string>* getFields() const;
00116 void setLanguageTemplate(std::string _languageTemplate);
00117 std::string getLanguageTemplate() const;
00118 void setCategory(std::string _category);
00119 std::string getCategory() const;
00120 static std::string categoryFolderName(const std::string& category);
00121 public:
00122 private:
00123 std::string _author = "prof. Dr. Ing. Rafael Luiz Cancian";
00124 std::string _date = "01/08/2018";
00125 std::string _version = "0.9.1";
00126 std::string _category = "DiscreteProcessing";
00127 std::string _observation = "First implementation not fully completed nor tested. Use with
 caution.";
00128 std::string _descriptionHelp = "";
00129 std::string _languageTemplate = "";
00130 bool _isSource = false;
00131 bool _isSink = false;
00132 bool _receiveTransfer = false;
00133 bool _sendTransfer = false;
00134 bool _generateReport = false;
00135 std::list<std::string>* _dynamicLibFilenameDependencies = new std::list<std::string>();
00136 std::list<SystemDependency>* _systemDependencies = new std::list<SystemDependency>();
00137 std::map<std::string, std::string>* _fields = new std::map<std::string, std::string>();
00138 // set from constructor
00139 std::string _pluginTypeName;
00140 bool _isComponent = false;
00141 unsigned short _minimumInputs = 1;
00142 unsigned short _maximumInputs = 1;
00143 unsigned short _minimumOutputs = 1;
00144 unsigned short _maximumOutputs = 1;
00145 StaticLoaderComponentInstance _componentloader;
00146 StaticLoaderDataDefinitionInstance _elementloader;
00147 StaticConstructorDataDefinitionInstance _elementConstructor;
00148 };
00149 //namespace\}
00150
00151 #endif /* PLUGININFORMATION_H */

```

## 16.307 PluginManager.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: PluginManager.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 30 de Maio de 2019, 17:49
00012 */
00013
00014 #ifndef PLUGINMANAGER_H
00015 #define PLUGINMANAGER_H
00016
00017 #include <functional>
00018 #include <string>
00019 #include <vector>
00020
00021 #include "model/Model.h"
00022 #include "../util/List.h"
00023 #include "../util/Util.h"
00024 #include "Plugin.h"
00025 #include "PluginConnector_if.h"
00026 #include "SystemDependencyResolver.h"
00027
00028 class Simulator;

```

```

00029 class Model;
00030
00039 struct PluginInsertionOptions {
00041 std::function<bool(const SystemDependencyCheckResult&)> confirmSystemDependencyInstallation;
00042 };
00043
00051 class PluginLoadIssue {
00052 public:
00054 enum class Reason {
00055 InvalidPlugin,
00056 MissingSystemDependency,
00057 DynamicDependencyFailure,
00058 ConnectionFailure,
00059 InsertionFailure,
00060 Exception
00061 };
00062
00064 PluginLoadIssue();// fix // = default;
00066 PluginLoadIssue(std::string filename,
00067 std::string pluginTypeName,
00068 Reason reason,
00069 std::string message,
00070 SystemDependencyCheckResult systemDependencyResult = {}) noexcept;
00071
00073 const std::string& getFilename() const;
00075 const std::string& getPluginTypeName() const;
00077 Reason getReason() const;
00079 const std::string& getMessage() const;
00081 const SystemDependencyCheckResult& getSystemDependencyResult() const;
00083 bool hasSystemDependencyResult() const;
00085 static std::string reasonToString(Reason reason);
00087 std::string diagnosticText() const;
00088
00089 private:
00090 std::string _filename;
00091 std::string _pluginTypeName;
00092 Reason _reason;
00093 std::string _message;
00094 SystemDependencyCheckResult _systemDependencyResult;
00095 };
00096
00097 //namespace GenesysKernel {
00098
00106 class PluginManager {
00107 public:
00109 PluginManager(Simulator* simulator);
00116 PluginManager(Simulator* simulator, PluginConnector_if* pluginConnector, SystemCommandExecutor_if*
systemCommandExecutor);
00118 virtual ~PluginManager();
00119
00120 public:
00122 std::string show();
00123
00124 public:
00126 List<Plugin*>* completePluginsFieldsAndTemplates();
00127
00128 public:
00130 bool check(const std::string& dynamicLibraryFilename);
00132 SystemDependencyCheckResult checkSystemDependencies(const std::string& dynamicLibraryFilename);
00134 List<std::string*>* discoverPluginFileNames() const;
00136 List<PluginLoadIssue*>* getPluginLoadIssues() const;
00138 void clearPluginLoadIssues();
00140 void clearPluginLoadIssue(const std::string& dynamicLibraryFilename);
00142 Plugin* insert(const std::string& dynamicLibraryFilename);
00144 Plugin* insert(const std::string& dynamicLibraryFilename, const PluginInsertionOptions& options);
00146 bool remove(const std::string& dynamicLibraryFilename);
00148 bool remove(Plugin* plugin);
00150 Plugin* find(const std::string& pluginTypeName);
00152 std::vector<std::string> getDataDefinitionPluginTypeNames() const;
00154 std::string sourceIncludePathFor(const std::string& pluginTypeName);
00156 List<Plugin*>* autoInsertPlugins(const std::string& pluginsListFilename,
const bool lookForPluginsIfFilenameNotFound = true);
00158 List<Plugin*>* autoInsertPlugins(const std::string& pluginsListFilename,
const bool lookForPluginsIfFilenameNotFound,
const PluginInsertionOptions& options);
00160 List<Plugin*>* autoInsertPlugins();
00162 List<Plugin*>* autoInsertPlugins(const PluginInsertionOptions& options);
00164
00165 public:
00167 Plugin* front();
00169 Plugin* next();
00171 Plugin* last();
00173 unsigned int size();
00175 Plugin* getAtRank(unsigned int rank);
00177
00178 public:
00180 ModelDataDefinition* newInstance(const std::string& pluginTypeName, Model* model, std::string name

```

```

 = "");
00186
00187 template <typename T>
00188 T* newInstance(Model* model, std::string name = "") {
00189 name = Util::StrReplace(name, " ", "_");
00190 Plugin* plugin;
00191 std::string pluginTypeName = Util::TypeOf<T>();
00192 // @ToDo: (pequena alteração): Use Find method??
00193 for (unsigned short i = 0; i < _plugins->size(); i++) {
00194 plugin = _plugins->getAtRank(i);
00195 if (plugin->getPluginInfo()->getPluginTypeName() == pluginTypeName) {
00196 T* instance;
00197 StaticConstructorDataDefinitionInstance constructor = plugin->getPluginInfo()->
00198 getDataDefinitionConstructor();
00199 instance = static_cast<T*>(constructor(model, name));
00200 if (model != nullptr) {
00201 ModelDataDefinition::CreateInternalData(instance);
00202 }
00203 return instance;
00204 }
00205 }
00206 // Keep this header free of Simulator implementation details to avoid include cycles.
00207 return nullptr;
00208 }
00209 private:
00210 bool _insert(Plugin* plugin, const PluginInsertionOptions& options, const std::string&
00211 dynamicLibraryFilename);
00212 bool _preflightAndMaybeInstallSystemDependencies(PluginInformation* plugInfo,
00213 const PluginInsertionOptions& options,
00214 SystemDependencyCheckResult* blockingResult =
00215 nullptr,
00216 std::string* failureMessage = nullptr);
00217 void _recordLoadIssue(const PluginLoadIssue& issue);
00218 void _removeLoadIssue(const std::string& dynamicLibraryFilename, const std::string&
00219 pluginTypeName);
00220 void _insertDefaultKernelElements();
00221 List<Plugin*>* _autoFindPlugins(const PluginInsertionOptions& options);
00222 private:
00223 List<Plugin*>* _plugins;
00224 List<PluginLoadIssue*>* _pluginLoadIssues;
00225 Simulator* _simulator;
00226 PluginConnector_if* _pluginConnector;
00227 SystemCommandExecutor_if* _systemCommandExecutor;
00228 };
00229
00230 //namespace\\
00231 #endif /* PLUGINMANAGER_H */

```

## 16.308 PropertyGenesys.h

```

00001 #ifndef PROPERTY_H
00002 #define PROPERTY_H
00003
00004 #include "SimulationControlAndResponse.h"
00005 #include "model/ModelComponent.h"
00006
00007 class PropertyEditorGenesys {
00008 public:
00009 PropertyEditorGenesys() = default;
00010 virtual ~PropertyEditorGenesys() = default;
00011
00012 public:
00013 std::list<ModelComponent*> getElements() {
00014 return _elements;
00015 }
00016
00017 public:
00018 void changeSimulationControl(SimulationControl* control, const std::string& value, bool remove =
00019 false) {
00020 if (control != nullptr) {
00021 control->setValue(value, remove);
00022 }
00023 }
00024
00025 SimulationControl* findSimulationControl(const std::string& id, const std::string& attribute) {
00026 for (auto element : _elements) {
00027 if (std::to_string(element->getId()) == id) {
00028 for (auto control : *element->getSimulationControls()->list()) {
00029 if (control->getName() == attribute) {
00030 return control;
00031 }
00032 }
00033 }
00034 }
00035 }
00036 }

```



```

00032 }
00033 }
00034 return nullptr;
00035 }
00036
00037 // Legacy compatibility wrapper. Prefer changeSimulationControl.
00038 //void changeProperty(SimulationControl* control, const std::string& value, bool remove = false) {
00039 // changeSimulationControl(control, value, remove);
00040 //}
00041
00042 // Legacy compatibility wrapper. Prefer findSimulationControl.
00043 //SimulationControl* findProperty(const std::string& id, const std::string& attribute) {
00044 // return findSimulationControl(id, attribute);
00045 //}
00046
00047 void addElement(ModelComponent* component) {
00048 _elements.push_back(component);
00049 }
00050
00051 private:
00052 std::list<ModelComponent*> _elements;
00053 };
00054
00055 #endif

```

## 16.309 ScenarioExperiment\_if.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: ScenarioExperiment_if.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 10 de Outubro de 2018, 14:52
00012 */
00013
00014 #ifndef SCENARIOEXPERIMENT_IF_H
00015 #define SCENARIOEXPERIMENT_IF_H
00016
00017 class ScenarioExperiment_if {
00018 public:
00019 virtual ~ScenarioExperiment_if() = default;
00020 };
00021
00022 #endif /* SCENARIOEXPERIMENT_IF_H */

```

## 16.310 SimulationControlAndResponse.h

```

00001 #pragma once
00002
00003 #include <string>
00004 #include <sstream>
00005 #include <functional>
00006 #include <list>
00007 #include <vector>
00008 #include <stdexcept>
00009 #include <type_traits>
00010 #include "../util/Util.h"
00011 #include "../util/List.h"
00012
00013 //namespace GenesysKernel {
00014
00015 class ModelDataDefinition;
00016
00017 class SourceCodeString {
00018 public:
00019 SourceCodeString() = default;
00020 SourceCodeString(const std::string& value) : _value(value) {}
00021 SourceCodeString(std::string&& value) : _value(std::move(value)) {}
00022
00023 const std::string& str() const { return _value; }
00024 operator std::string() const { return _value; }
00025
00026 private:
00027 std::string _value;
00028 };
00029
00030 inline std::ostream& operator<<(std::ostream& os, const SourceCodeString& value) {

```

```

00031 os << value.str();
00032 return os;
00033 }
00034
00035
00047 class PropertyGenesysBase {
00048 public:
00049 PropertyGenesysBase(std::string className, std::string elementName, std::string propertyName,
00050 std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false) {
00051 _className=className;
00052 _elementName=elementName;
00053 _propertyName=propertyName;
00054 _whatsThis=whatsThis;
00055 _isList=isList;
00056 _isClass=isClass;
00057 _isEnum=isEnum;
00058 }
00059 virtual ~PropertyGenesysBase() = default;
00060 public:
00061 std::string whatsThis() const { return _whatsThis; }
00062 std::string getClassname() const { return _className; }
00063 std::string getElementName() const { return _elementName; }
00064 std::string getName() const { return _propertyName; }
00065 std::string propertyType() const { return _propertyType; }
00066 bool getIsList() const { return _isList; }
00067 bool getIsClass() const { return _isClass; }
00068 bool getIsEnum() const { return _isEnum; }
00069 void setWhatsThis(const std::string text);
00070 void setName(std::string name) { _propertyName = name; }
00071 protected:
00072 std::string _elementName;
00073 std::string _className;
00074 std::string _propertyName;
00075 std::string _propertyType;
00076 std::string _whatsThis;
00077 bool _isList;
00078 bool _isClass;
00079 bool _isEnum;
00080 };
00081
00082 class PropertyGenesys: public PropertyGenesysBase {
00083 public:
00084 PropertyGenesys(std::string className, std::string elementName, std::string propertyName,
00085 std::string whatsThis="", std::string propertyId="", std::string toolTip="", std::string statusTip="")
00086 : PropertyGenesysBase(className, elementName, propertyName, whatsThis){
00087 _propertyId=propertyId;
00088 _toolTip=toolTip;
00089 _statusTip=statusTip;
00090 }
00091 virtual ~PropertyGenesys() = default;
00092 public:
00093 std::list<PropertyGenesys*> subProperties() const { return _subProperties; };
00094 std::string toolTip() const { return _toolTip; }
00095 std::string statusTip() const { return _statusTip; }
00096 std::string propertyName() const { return _propertyName; }
00097 std::string propertyId() const { return _propertyId; }
00098 bool isEnabled() const { return _isEnabled; }
00099 bool isModified() const { return _isModified; }
00100 bool isSubProperty()const { return _isSubProperty; }
00101 bool hasValue() const { return _hasValue; }
00102 std::string valueText() const { return "?"; } // @ToDo: (importante): implement value text
00103 void setToolTip(const std::string text);
00104 void setStatusTip(const std::string text);
00105 void setWhatsThis(const std::string text);
00106 void setPropertyName(const std::string text);
00107 void setPropertyId(const std::string text);
00108 void setEnabled(bool enable);
00109 void setModified(bool modified);
00110 public:
00111 virtual bool compare(PropertyGenesys* otherProperty)const { return true; } // @ToDo: (importante):
00112 implement comparison
00113 public:
00114 void addSubProperty(PropertyGenesys *property) {}
00115 void insertSubProperty(PropertyGenesys *property, PropertyGenesys *afterProperty) {}
00116 void removeSubProperty(PropertyGenesys *property) {}
00117 protected:
00118 void propertyChanged() {}
00119 protected:
00120 //friend class QtAbstractPropertyManager;
00121 std::string _className;
00122 std::string _propertyName;
00123 std::string _whatsThis;
00124 std::string _propertyId;
00125 std::string _toolTip;
00126 std::string _statusTip;
00127 std::list<PropertyGenesys*> _subProperties;
00128 bool _isEnabled;

```

```

00126 bool _isModified;
00127 bool _hasValue;
00128 bool _isSubProperty;
00129 };
00130
00137 class SimulationResponse: public PropertyGenesysBase {
00138 public:
00139 SimulationResponse(std::string className, std::string elementName, std::string propertyName,
00140 std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
00141 : PropertyGenesysBase(className, elementName, propertyName, whatsThis, isList, isClass,
00142 isEnum) {
00143 }
00144 virtual ~SimulationResponse() = default;
00145 public:
00146 std::string show() const {
00147 std::string msg = "classname="+_className+ " , elementName="+_elementName+", name=\"\" +
00148 _propertyName + "\"";
00149 msg += " , value="+getValue();
00150 return msg;
00151 }
00152 virtual std::string getValue() const = 0;
00153 virtual List<std::string>* getStrValues() { return nullptr; };
00154 };
00155
00156 enum class SimulationControlEditorHint {
00157 Default,
00158 CodeEditor,
00159 MultiLineText
00160 };
00161
00162 class SimulationControl: public SimulationResponse {
00163 public:
00164 SimulationControl(std::string className, std::string elementName, std::string propertyName,
00165 std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
00166 : SimulationResponse(className, elementName, propertyName, whatsThis, isList, isClass,
00167 isEnum) {
00168 }
00169 std::string show() const {
00170 std::string msg = "classname="+_className+ " , elementName="+_elementName+",
00171 name=\"\"+_propertyName+\"\"";
00172 if (_readonly) {
00173 msg += " (readonly)";
00174 }
00175 //msg += " , type="+_propertyType;
00176 msg += " , value="+getValue();
00177 return msg;
00178 }
00179 bool isReadOnly() const { return _readonly; }
00180 public:
00181 virtual void setValue(std::string value, bool remove=false) = 0;
00182 virtual List<SimulationControl*>* getChildSimulationControls(int index=0) { return nullptr; };
00183 virtual bool hasObjectInstance() const { return true; };
00184 virtual bool ensureObjectInstance() { return hasObjectInstance(); };
00185 virtual bool isModelDataDefinitionReference() const { return false; };
00186 virtual ModelDataDefinition* getReferencedModelDataDefinition() const { return nullptr; };
00187 // This method validates a proposed string value before GUI editors commit it.
00188 virtual bool validateProposedValue(const std::string& value, std::string& errorMessage) const {
00189 (void)value;
00190 errorMessage.clear();
00191 return true;
00192 }
00193 // This method exposes the preferred editor affordance for GUI-side property editing.
00194 virtual SimulationControlEditorHint preferredEditorHint() const { return _preferredEditorHint; }
00195 void setPreferredEditorHint(SimulationControlEditorHint hint) { _preferredEditorHint = hint; }
00196 // This method exposes the ModelDataDefinition represented by a class-list row, when any.
00197 //
00198 // It lets generic tooling classify list members themselves as editable references without
00199 // knowing the concrete owner class. Non ModelDataDefinition lists keep the default nullptr.
00200 virtual ModelDataDefinition* getListElementModelDataDefinition(int index) const {
00201 (void)index;
00202 return nullptr;
00203 }
00204 // This method exposes whether the property should be rendered as an inline expandable object
00205 tree.
00206 virtual bool supportsInlineExpansion() const { return getIsClass() && !getIsList(); }
00207 // This method exposes whether the property should be edited by the dedicated list editor.
00208 virtual bool supportsListEditor() const { return getIsList(); }
00209 // This method exposes whether the property supports choosing an existing object instance.
00210 virtual bool supportsExistingObjectSelection() const { return false; }
00211 // This method exposes whether the property supports creating a new object instance.
00212 virtual bool supportsObjectCreation() const { return false; }
00213 // This method exposes whether the property supports typed creation of a new list element.
00214 virtual bool supportsNewListElementCreation() const { return false; }
00215 // This method exposes which concrete element types can be created for a polymorphic list.
00216 //
00217 // A nullptr return means the list has no typed-creation contract and should use the legacy
00218 // single "create new element" action. Callers own the returned List when it is not nullptr.

```

```

00224 virtual List<std::string>* getCreatableListElementTypes() { return nullptr; }
00225 // This method exposes the concrete type currently accepted by a polymorphic list.
00226 //
00227 // Empty string means no concrete type has been selected yet. Implementations may lock the
00228 // current type once the list has members, while still allowing it to be configured while empty.
00229 virtual std::string getCurrentListElementType() const { return ""; }
00230 // This method configures the concrete type currently accepted by a polymorphic list.
00231 //
00232 // The default implementation rejects the operation. Implementations that support it should
00233 // validate the requested type against getCreatableListElementTypes() and their own invariants.
00234 virtual bool setCurrentListElementType(const std::string& typeName) {
00235 (void)typeName;
00236 return false;
00237 }
00238 // This method exposes whether the property is an inline object (and not a ModelDataDefinition
reference).
00239 virtual bool isInlineObjectProperty() const {
00240 return supportsInlineExpansion() && !isModelDataDefinitionReference();
00241 }
00242 // This method provides an explicit object-creation operation for class-like properties.
00243 virtual bool createObjectInstance(const std::string& value = "") {
00244 (void)value;
00245 return false;
00246 }
00247 virtual List<std::string>* getCreatableReferenceTypes() { return nullptr; }
00248 virtual std::string getCurrentReferenceType() const { return ""; }
00249 virtual bool setCurrentReferenceType(const std::string& typeName) {
00250 (void)typeName;
00251 return false;
00252 }
00253 virtual bool createObjectInstanceOfType(const std::string& typeName, const std::string& value =
"") {
00254 if (typeName.empty()) {
00255 return createObjectInstance(value);
00256 }
00257 (void)value;
00258 return false;
00259 }
00260 // This method provides an explicit typed list-element creation operation for list-like
properties.
00261 virtual bool createNewListElement(const std::string& value = "") {
00262 (void)value;
00263 return false;
00264 }
00265 // This method creates a list element of a requested concrete type.
00266 //
00267 // The default keeps old controls working: when no concrete type is provided, it delegates to
00268 // createNewListElement(); otherwise the operation is unsupported.
00269 virtual bool createNewListElementOfType(const std::string& typeName, const std::string& value =
"") {
00270 if (typeName.empty()) {
00271 return createNewListElement(value);
00272 }
00273 (void)value;
00274 return false;
00275 }
00276 virtual List<SimulationControl*>* getEditableChildSimulationControls(int index=0) {
00277 if (supportsInlineExpansion() && !hasObjectInstance()) {
00278 if (!supportsObjectCreation()) {
00279 return nullptr;
00280 }
00281 if (!ensureObjectInstance()) {
00282 return nullptr;
00283 }
00284 }
00285 return getChildSimulationControls(index);
00286 }
00287 protected:
00288 void _ensureWritable(const char* operation) const {
00289 if (_readonly) {
00290 throw std::logic_error(std::string("Cannot ") + operation + " readonly SimulationControl
\"" + _propertyName + "\"");
00291 }
00292 }
00293 bool _readonly = false;
00294 SimulationControlEditorHint _preferredEditorHint = SimulationControlEditorHint::Default;
00295 };
00296 // -----
00297
00298 template<typename Class, typename T>
00299 std::function<T()> DefineSimulationGetter(Class* object, T(Class::*function)() const) {
00300 return std::bind(function, object);
00301 }
00302
00303 template<typename Class, typename T>
00304 std::function<T()> DefineSimulationGetter(Class* object, T(Class::*function)()) {

```

```

00306 return std::bind(function, object);
00307 }
00308
00309 template<typename Class, typename T>
00310 std::function<void(T)> DefineSimulationSetter(Class* object, void (Class::*function)(T)) {
00311 return std::bind(function, object, std::placeholders::_1);
00312 }
00313
00314 typedef std::function<std::string()> GetterString;
00315 typedef std::function<void(std::string)> SetterString;
00316 typedef std::function<bool(const std::string&, std::string&)> ValidatorString;
00317 class SimulationControlString: public SimulationControl {
00318 public:
00319 // SimulationControlString(GetterString getter, std::string className, std::string elementName,
00320 // std::string propertyName, std::string whatsThis="") : SimulationControl(className, elementName,
00321 // propertyName, whatsThis) {
00322 // SimulationControlString(getter, nullptr, className, propertyName, whatsThis);
00323 // }
00324 SimulationControlString(GetterString getter, SetterString setter, std::string className,
00325 std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool
00326 isClass=false, bool isEnum=false, ValidatorString validator=nullptr) : SimulationControl(className,
00327 elementName, propertyName, whatsThis, isList, isClass, isEnum) {
00328 _getter= getter;
00329 _setter = setter;
00330 _validator = validator;
00331 _readonly = setter == nullptr;
00332 _propertyType = Util::TypeOf<std::string>();
00333 }
00334 public:
00335 virtual std::string getValue() const override { return _getter(); }
00336 virtual bool validateProposedValue(const std::string& value, std::string& errorMessage) const
00337 override {
00338 if (!_validator) {
00339 errorMessage.clear();
00340 return true;
00341 }
00342 return _validator(value, errorMessage);
00343 }
00344 virtual void setValue(std::string value, bool remove=false) override {
00345 _ensureWritable("set value of");
00346 if (!_setter) {
00347 throw std::logic_error("SimulationControlString setter is not defined");
00348 }
00349 _setter(value);
00350 };
00351 private:
00352 GetterString _getter;
00353 SetterString _setter;
00354 ValidatorString _validator;
00355 };
00356
00357 typedef std::function<SourceCodeString()> GetterSourceCodeString;
00358 typedef std::function<void(SourceCodeString)> SetterSourceCodeString;
00359 class SimulationControlSourceCodeString: public SimulationControl {
00360 public:
00361 SimulationControlSourceCodeString(GetterSourceCodeString getter, SetterSourceCodeString setter,
00362 std::string className, std::string elementName, std::string propertyName, std::string whatsThis="",
00363 bool isList=false, bool isClass=false, bool isEnum=false, ValidatorString validator=nullptr) :
00364 SimulationControl(className, elementName, propertyName, whatsThis, isList, isClass, isEnum) {
00365 _getter = getter;
00366 _setter = setter;
00367 _validator = validator;
00368 _readonly = setter == nullptr;
00369 _propertyType = Util::TypeOf<SourceCodeString>();
00370 _preferredEditorHint = SimulationControlEditorHint::CodeEditor;
00371 }
00372 public:
00373 virtual std::string getValue() const override { return _getter().str(); }
00374 virtual bool validateProposedValue(const std::string& value, std::string& errorMessage) const
00375 override {
00376 if (!_validator) {
00377 errorMessage.clear();
00378 return true;
00379 }
00380 return _validator(value, errorMessage);
00381 }
00382 virtual void setValue(std::string value, bool remove=false) override {
00383 (void)remove;
00384 _ensureWritable("set value of");
00385 if (!_setter) {
00386 throw std::logic_error("SimulationControlSourceCodeString setter is not defined");
00387 }
00388 _setter(SourceCodeString(std::move(value)));
00389 };
00390 private:
00391 GetterSourceCodeString _getter;
00392 SetterSourceCodeString _setter;

```

```

00383 ValidatorString _validator;
00384 };
00385
00386
00387 // -----
00388
00389 typedef std::function<double()> GetterDouble;
00390 typedef std::function<void(double)> SetterDouble;
00391
00392 class SimulationResponseDouble: public SimulationResponse {
00393 public:
00394 SimulationResponseDouble(GetterDouble getter, std::string className, std::string elementName,
00395 std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool
00396 isEnum=false)
00397 : SimulationResponse(className, elementName, propertyName, whatsThis, isList, isClass, isEnum)
00398 {
00399 _getter = getter;
00400 _propertyType = Util::TypeOf<double>();
00401 }
00402 public:
00403 virtual std::string getValue() const override { return std::to_string(_getter()); }
00404 private:
00405 GetterDouble _getter;
00406 };
00407
00408 class SimulationControlDouble: public SimulationControl {
00409 public:
00410 // SimulationControlDouble(GetterDouble getter, std::string className, std::string elementName,
00411 // std::string propertyName, std::string whatsThis="") : SimulationControl(className, elementName,
00412 // propertyName, whatsThis) {
00413 // SimulationControlDouble(getter, nullptr, className, propertyName, whatsThis);
00414 // }
00415 SimulationControlDouble(GetterDouble getter, SetterDouble setter, std::string className,
00416 std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool
00417 isClass=false, bool isEnum=false) : SimulationControl(className, elementName, propertyName, whatsThis,
00418 isList, isClass, isEnum){
00419 _getter= getter;
00420 _setter = setter;
00421 _readonly = setter == nullptr;
00422 _propertyType = Util::TypeOf<double>();
00423 }
00424 public:
00425 virtual std::string getValue() const override { return std::to_string(_getter()); }
00426 virtual void setValue(std::string value, bool remove=false) override {
00427 _ensureWritable("set value of");
00428 if (!_setter) {
00429 throw std::logic_error("SimulationControlDouble setter is not defined");
00430 }
00431 _setter(std::stod(value));
00432 };
00433 private:
00434 GetterDouble _getter;
00435 SetterDouble _setter;
00436 };
00437
00438 // -----
00439
00440 typedef std::function<bool()> GetterBool;
00441 typedef std::function<void(bool)> SetterBool;
00442 class SimulationControlBool: public SimulationControl {
00443 public:
00444 SimulationControlBool(GetterBool getter, std::string className, std::string elementName,
00445 std::string propertyName, std::string whatsThis="") : SimulationControl(className, elementName,
00446 propertyName, whatsThis) {
00447 // SimulationControlBool(getter, nullptr, className, propertyName, whatsThis);
00448 // }
00449 SimulationControlBool(GetterBool getter, SetterBool setter, std::string className, std::string
00450 elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool
00451 isClass=false, bool isEnum=false) : SimulationControl(className, elementName, propertyName, whatsThis,
00452 isList, isClass, isEnum){
00453 _getter= getter;
00454 _setter = setter;
00455 _readonly = setter == nullptr;
00456 _propertyType = Util::TypeOf<bool>();
00457 }
00458 public:
00459 virtual std::string getValue() const override { return std::to_string(_getter()); }
00460 virtual void setValue(std::string value, bool remove=false) override {
00461 _ensureWritable("set value of");
00462 if (!_setter) {
00463 throw std::logic_error("SimulationControlBool setter is not defined");
00464 }
00465 std::istringstream auxStr(value);
00466 auxStr >> std::boolalpha;
00467 bool boolVal = false;
00468 if (!(auxStr >> boolVal)) {
00469 boolVal = std::stoi(value) != 0;
00470 }
00471 }
00472 };

```

```

00457 }
00458 _setter(boolVal);
00459 };
00460 private:
00461 GetterBool _getter;
00462 SetterBool _setter;
00463 };
00464
00465
00466 // -----
00467
00468 typedef std::function<unsigned int()> GetterUInt;
00469 typedef std::function<void(unsigned int)> SetterUInt;
00470 class SimulationControlUInt: public SimulationControl {
00471 public:
00472 // SimulationControlUInt(GetterUInt getter, std::string className, std::string elementName,
00473 // std::string propertyName, std::string whatsThis="") : SimulationControl(className, elementName,
00474 // propertyName, whatsThis) {
00475 // SimulationControlUInt(getter, nullptr, className, propertyName, whatsThis);
00476 // }
00477 SimulationControlUInt(GetterUInt getter, SetterUInt setter, std::string className, std::string
00478 elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool
00479 isClass=false, bool isEnum=false) : SimulationControl(className, elementName, propertyName, whatsThis,
00480 isList, isClass, isEnum){
00481 _getter= getter;
00482 _setter = setter;
00483 _readonly = setter == nullptr;
00484 _propertyType = Util::TypeOf<unsigned int>();
00485 }
00486 public:
00487 virtual std::string getValue() const override { return std::to_string(_getter()); }
00488 virtual void setValue(std::string value, bool remove=false) override {
00489 _ensureWritable("set value of");
00490 if (!_setter) {
00491 throw std::logic_error("SimulationControlUInt setter is not defined");
00492 }
00493 _setter(std::stoul(value));
00494 };
00495 private:
00496 GetterUInt _getter;
00497 SetterUInt _setter;
00498 };
00499
00500 // -----
00501
00502 typedef std::function<unsigned short()> GetterUShort;
00503 typedef std::function<void(unsigned short)> SetterUShort;
00504 class SimulationControlUShort: public SimulationControl {
00505 public:
00506 // SimulationControlUShort(GetterUShort getter, std::string className, std::string elementName,
00507 // std::string propertyName, std::string whatsThis="") : SimulationControl(className, elementName,
00508 // propertyName, whatsThis) {
00509 // SimulationControlUShort(getter, nullptr, className, propertyName, whatsThis);
00510 // }
00511 SimulationControlUShort(GetterUShort getter, SetterUShort setter, std::string className,
00512 std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool
00513 isClass=false, bool isEnum=false) : SimulationControl(className, elementName, propertyName, whatsThis,
00514 isList, isClass, isEnum){
00515 _getter= getter;
00516 _setter = setter;
00517 _readonly = setter == nullptr;
00518 _propertyType = Util::TypeOf<unsigned short>();
00519 }
00520 public:
00521 virtual std::string getValue() const override { return std::to_string(_getter()); }
00522 virtual void setValue(std::string value, bool remove=false) override {
00523 _ensureWritable("set value of");
00524 if (!_setter) {
00525 throw std::logic_error("SimulationControlUShort setter is not defined");
00526 }
00527 _setter(std::stoul(value));
00528 };
00529 private:
00530 GetterUShort _getter;
00531 SetterUShort _setter;
00532 };
00533
00534 // -----
00535
00536 typedef std::function<int()> GetterInt;
00537 typedef std::function<void(int)> SetterInt;
00538 class SimulationControlInt: public SimulationControl {
00539 public:
00540 // SimulationControlInt(GetterInt getter, std::string className, std::string elementName, std::string
00541 // propertyName, std::string whatsThis="") : SimulationControl(className, elementName, propertyName,
00542 // whatsThis) {

```

```

00532 // SimulationControlInt(getter, nullptr, className, propertyName, whatsThis);
00533 // }
00534 SimulationControlInt(GetterInt getter, SetterInt setter, std::string className, std::string
elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool
isClass=false, bool isEnum=false) : SimulationControl(className, elementName, propertyName, whatsThis,
isList, isClass, isEnum){
00535 _getter= getter;
00536 _setter = setter;
00537 _readonly = setter == nullptr;
00538 _propertyType = Util::TypeOf<int>();
00539 }
00540 public:
00541 virtual std::string getValue() const override { return std::to_string(_getter()); }
00542 virtual void setValue(std::string value, bool remove=false) override {
00543 _ensureWritable("set value of");
00544 if (!_setter) {
00545 throw std::logic_error("SimulationControlInt setter is not defined");
00546 }
00547 _setter(std::stoi(value));
00548 };
00549 private:
00550 GetterInt _getter;
00551 SetterInt _setter;
00552 };
00553
00554 // -----
00555
00556 typedef std::function<Util::TimeUnit()> GetterTimeUnit;
00557 typedef std::function<void(Util::TimeUnit)> SetterTimeUnit;
00558 class SimulationControlTimeUnit: public SimulationControl {
00559 public:
00560 // SimulationControlTimeUnit(GetterTimeUnit getter, std::string className, std::string elementName,
std::string propertyName, std::string whatsThis="") : SimulationControl(className, elementName,
propertyName, whatsThis) {
00561 // SimulationControlTimeUnit(getter, nullptr, className, propertyName, whatsThis);
00562 // }
00563 SimulationControlTimeUnit(GetterTimeUnit getter, SetterTimeUnit setter, std::string className,
std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool
isClass=false, bool isEnum=false) : SimulationControl(className, elementName, propertyName, whatsThis,
isList, isClass, isEnum){
00564 _getter= getter;
00565 _setter = setter;
00566 _readonly = setter == nullptr;
00567 _propertyType = Util::TypeOf<Util::TimeUnit>();
00568 }
00569 public:
00570 virtual std::string getValue() const override {
00571 int intVal = static_cast<int>(_getter());
00572 return std::to_string(intVal);
00573 }
00574 virtual void setValue(std::string value, bool remove=false) override {
00575 _ensureWritable("set value of");
00576 if (!_setter) {
00577 throw std::logic_error("SimulationControlTimeUnit setter is not defined");
00578 }
00579 int intVal = std::stoul(value);
00580 _setter(static_cast<Util::TimeUnit>(intVal));
00581 };
00582 private:
00583 GetterTimeUnit _getter;
00584 SetterTimeUnit _setter;
00585 };
00586
00587
00588 template <typename T>
00589 using GetterGeneric = std::function<T>();
00590 template <typename T>
00591 using SetterGeneric = std::function<void(T)>;
00592 template <typename T>
00593 using AdderGeneric = std::function<void(T)>;
00594 template <typename T>
00595 using RemoverGeneric = std::function<void(T)>;
00596
00597 template <typename T>
00598 class SimulationControlGeneric: public SimulationControl {
00599 public:
00600 SimulationControlGeneric(GetterGeneric<T> getter, SetterGeneric<T> setter, std::string className,
std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool
isClass=false, bool isEnum=false) : SimulationControl(className, elementName, propertyName, whatsThis,
isList, isClass, isEnum){
00601 _getter= getter;
00602 _setter = setter;
00603 _readonly = setter == nullptr;
00604 _propertyType = Util::TypeOf<T>();
00605 }
00606 public:
00607 virtual std::string getValue() const override {

```



```

00608 T tVal = static_cast<T>(_getter());
00609
00610 std::string strVal;
00611 std::ostringstream auxStr;
00612 auxStr << tVal;
00613 strVal = auxStr.str();
00614
00615 return strVal;
00616 }
00617
00618 virtual void setValue(std::string value, bool remove=false) override {
00619 _ensureWritable("set value of");
00620 if (!_setter) {
00621 throw std::logic_error("SimulationControlGeneric setter is not defined");
00622 }
00623 if constexpr (std::is_same_v<T, std::string>) {
00624 _setter(value);
00625 } else {
00626 std::istringstream auxStr(value);
00627 T tVal{};
00628 auxStr >> tVal;
00629 _setter(static_cast<T>(tVal));
00630 }
00631 };
00632
00633 private:
00634 GetterGeneric<T> _getter;
00635 SetterGeneric<T> _setter;
00636 };
00637
00638 // @ToDo: (pequena alteração): remove typename E
00639 template <typename T, typename E>
00640 class SimulationControlGenericEnum: public SimulationControl {
00641 public:
00642 SimulationControlGenericEnum(GetterGeneric<T> getter, SetterGeneric<T> setter, std::string
 className, std::string elementName, std::string propertyName, std::string whatsThis="", bool
 isList=false, bool isClass=false, bool isEnum=true) : SimulationControl(className, elementName,
 propertyName, whatsThis, isList, isClass, isEnum){
00643 static_assert(std::is_enum<T>::value, "SimulationControlGenericEnum requires T to be an
 enum");
00644 _getter= getter;
00645 _setter = setter;
00646 _readonly = setter == nullptr;
00647 _propertyType = Util::TypeOf<T>();
00648 _isEnum=true;
00649 }
00650 public:
00651 virtual std::string getValue() const override {
00652 int intVal = static_cast<int>(_getter());
00653 int max_i = static_cast<int>(T::num_elements);
00654 if (intVal < 0 || intVal >= max_i) {
00655 return "";
00656 }
00657 return E::convertEnumToStr(static_cast<T>(intVal));
00658 }
00659
00660 virtual void setValue(std::string value, bool remove=false) override {
00661 _ensureWritable("set value of");
00662 if (!_setter) {
00663 throw std::logic_error("SimulationControlGenericEnum setter is not defined");
00664 }
00665 int intVal = std::stoul(value);
00666 _setter(static_cast<T>(intVal));
00667 };
00668
00669 virtual List<std::string>* getStrValues() override {
00670 List<std::string>* strOptions = new List<std::string>();
00671 int max_i = static_cast<int>(T::num_elements);
00672 for (int i=0; i<max_i; i++) {
00673 std::string value = E::convertEnumToStr((T)i);
00674 strOptions->insert(value);
00675 }
00676 return strOptions;
00677 }
00678
00679 private:
00680 GetterGeneric<T> _getter;
00681 SetterGeneric<T> _setter;
00682 };
00683
00684 // @ToDo: (pequena alteração): remove typename C
00685 template <typename T, typename M, typename C>
00686 class SimulationControlGenericClass: public SimulationControl {
00687 public:
00688 using Creator = std::function<T(M, const std::string&)>;
00689 SimulationControlGenericClass(M model, GetterGeneric<T> getter, SetterGeneric<T> setter,
 std::string className, std::string elementName, std::string propertyName, std::string whatsThis="",

```

```

 bool isList=false, bool isClass=true, bool isEnum=false, Creator creator=nullptr) :
SimulationControl(className, elementName, propertyName, whatsThis, isList, isClass, isEnum){
00690 static_assert(std::is_pointer<T>::value, "SimulationControlGenericClass requires pointer type
T");
00691 _model = model;
00692 _getter= getter;
00693 _setter = setter;
00694 _creator = creator;
00695 _readonly = setter == nullptr;
00696 _propertyType = Util::TypeOf<C>();
00697 }
00698 public:
00699 virtual std::string getValue() const override {
00700 T tVal = static_cast<T>(_getter());
00701 std::string strVal;
00702
00703 if (tVal != nullptr) {
00704 strVal = tVal->getName();
00705 } else {
00706 strVal = "";
00707 };
00708
00709 return strVal;
00710 }
00711
00712 virtual void setValue(std::string value, bool remove=false) override {
00713 _ensureWritable("set value of");
00714 if (!_setter) {
00715 throw std::logic_error("SimulationControlGenericClass setter is not defined");
00716 }
00717 if (remove) {
00718 _setter(nullptr);
00719 return;
00720 }
00721 bool exists = false;
00722 // value.pop_back();
00723 T newVal;
00724
00725 for (auto modeldata : *_model->getDataManager()->getDataDefinitionList(_propertyType)->list())
 {
00726 if (modeldata->getName() == value) {
00727 exists = true;
00728 newVal = static_cast<T>(modeldata);
00729 break;
00730 };
00731 };
00732
00733 if (!exists) {
00734 if (_creator != nullptr) {
00735 newVal = _creator(_model, value);
00736 } else {
00737 newVal = new C(_model, value);
00738 }
00739 if (newVal == nullptr) {
00740 throw std::logic_error("SimulationControlGenericClass creator returned null");
00741 }
00742 _model->getDataManager()->insert(newVal);
00743 }
00744
00745 _setter(newVal);
00746 };
00747
00748 virtual bool isModelDataDefinitionReference() const override { return true; }
00749 virtual ModelDataDefinition* getReferencedModelDataDefinition() const override {
00750 return static_cast<T>(_getter());
00751 }
00752 // This method marks class references as inline-expandable in the universal contract.
00753 virtual bool supportsInlineExpansion() const override { return true; }
00754 // This method marks class references as selectable from existing ModelDataDefinition instances.
00755 virtual bool supportsExistingObjectSelection() const override { return true; }
00756 // This method marks class references as creatable when writable.
00757 virtual bool supportsObjectCreation() const override { return !_readonly; }
00758 // This method marks class references as non-inline object payloads.
00759 virtual bool isInlineObjectProperty() const override { return false; }
00760 // Property Editor advanced editing experiment, step 1:
00761 // ModelDataDefinition references must report whether the referenced object
00762 // actually exists. Previously the base implementation always returned true,
00763 // so the GUI could not distinguish "empty reference" from "loaded object".
00764 // Revert this override to restore the old always-available reference behavior.
00765 virtual bool hasObjectInstance() const override {
00766 return static_cast<T>(_getter()) != nullptr;
00767 }
00768 // Property Editor advanced editing experiment, step 1:
00769 // Do not auto-create ModelDataDefinition references while expanding the tree.
00770 // Unlike inline helper objects, Queue/Resource/Set references need an explicit
00771 // name or an existing target chosen by the user before creation is safe.
00772 // Revert this override if implicit expansion should create references again.

```

```

00773 virtual bool ensureObjectInstance() override {
00774 return hasObjectInstance();
00775 }
00776 // This method performs explicit object creation for class references.
00777 virtual bool createObjectInstance(const std::string& value = "") override {
00778 _ensureWritable("create instance for");
00779 std::string name = value;
00780 if (name.empty()) {
00781 name = getValue();
00782 }
00783 if (name.empty()) {
00784 return false;
00785 }
00786 setValue(name, false);
00787 return hasObjectInstance();
00788 }
00789
00790 virtual List<std::string>* getStrValues() override {
00791 List<std::string>* strOptions = new List<std::string>();
00792 for (auto modeldata : *_model->getDataManager()->getDataDefinitionList(_propertyType)->list())
00793 {
00794 if (modeldata != nullptr) {
00795 strOptions->insert(modeldata->getName());
00796 }
00797 }
00798 return strOptions;
00799 }
00800 virtual List<SimulationControl*>* getChildSimulationControls(int index=0) override {
00801 T tVal = static_cast<T>(_getter());
00802
00803 if (tVal) {
00804 return tVal->getSimulationControls();
00805 } else {
00806 return nullptr;
00807 }
00808 }
00809
00810 private:
00811 M _model;
00812 GetterGeneric<T> _getter;
00813 SetterGeneric<T> _setter;
00814 Creator _creator;
00815 };
00816
00817 // @ToDo: (pequena alteração): remove typename C
00818 template <typename T, typename M, typename C>
00819 class SimulationControlGenericClassNotDC: public SimulationControl {
00820 public:
00821 using Creator = std::function<T(M)>;
00822 using NamedCreator = std::function<T(M, const std::string&)>;
00823 SimulationControlGenericClassNotDC(M model, GetterGeneric<T> getter, SetterGeneric<T> setter,
std::string className, std::string elementName, std::string propertyName, std::string whatsThis="",
bool isList=false, bool isClass=true, bool isEnum=false, Creator creator=nullptr, NamedCreator
namedCreator=nullptr) : SimulationControl(className, elementName, propertyName, whatsThis, isList,
isClass, isEnum){
00824 static_assert(std::is_pointer<T>::value, "SimulationControlGenericClassNotDC requires pointer
type T");
00825 _model = model;
00826 _getter= getter;
00827 _setter = setter;
00828 _creator = creator;
00829 _namedCreator = namedCreator;
00830 _readonly = setter == nullptr;
00831 _propertyType = Util::TypeOf<C>();
00832 }
00833 public:
00834 virtual std::string getValue() const override {
00835 T tVal = static_cast<T>(_getter());
00836 std::string strVal;
00837
00838 if (tVal != nullptr) {
00839 strVal = tVal->getName();
00840 } else {
00841 strVal = "";
00842 };
00843
00844 return strVal;
00845 }
00846
00847 virtual void setValue(std::string value, bool remove=false) override {
00848 _ensureWritable("set value of");
00849 if (!_setter) {
00850 throw std::logic_error("SimulationControlGenericClassNotDC setter is not defined");
00851 }
00852 T newVal = _createNewInstance(value);
00853 }

```

```

00854 _setter(newVal);
00855 };
00856
00857 virtual bool hasObjectInstance() const override {
00858 return static_cast<T>(_getter()) != nullptr;
00859 }
00860 // This method marks inline class payloads as expandable in the universal contract.
00861 virtual bool supportsInlineExpansion() const override { return true; }
00862 // This method marks inline class payloads as creatable when writable.
00863 virtual bool supportsObjectCreation() const override { return !_readonly; }
00864 // This method marks inline class payloads as non-reference object payloads.
00865 virtual bool isInlineObjectProperty() const override { return true; }
00866
00867 virtual bool ensureObjectInstance() override {
00868 _ensureWritable("ensure instance of");
00869 if (!_setter) {
00870 throw std::logic_error("SimulationControlGenericClassNotDC setter is not defined");
00871 }
00872
00873 T current = static_cast<T>(_getter());
00874 if (current != nullptr) {
00875 return true;
00876 }
00877
00878 T newVal;
00879 newVal = _createNewInstance("");
00880 _setter(newVal);
00881 return static_cast<T>(_getter()) != nullptr;
00882 }
00883 // This method performs explicit object creation for non-DataManager classes.
00884 virtual bool createObjectInstance(const std::string& value = "") override {
00885 if (value.empty()) {
00886 return ensureObjectInstance();
00887 }
00888 setValue(value, false);
00889 return hasObjectInstance();
00890 }
00891
00892 virtual List<SimulationControl*>* getChildSimulationControls(int index=0) override {
00893 T tVal = static_cast<T>(_getter());
00894
00895 if (tVal) {
00896 return tVal->getSimulationControls();
00897 } else {
00898 return nullptr;
00899 }
00900 }
00901
00902 private:
00903 T _createNewInstance(const std::string& value) const {
00904 if (_namedCreator != nullptr) {
00905 return _namedCreator(_model, value);
00906 }
00907 if (_creator != nullptr) {
00908 return _creator(_model);
00909 }
00910 return new C(_model, value);
00911 }
00912
00913 private:
00914 M _model;
00915 GetterGeneric<T> _getter;
00916 SetterGeneric<T> _setter;
00917 Creator _creator;
00918 NamedCreator _namedCreator;
00919 };
00920
00921 template <typename T, typename M, typename C>
00922 class SimulationControlGenericList: public SimulationControl {
00923 public:
00924 SimulationControlGenericList(M model, GetterGeneric<List<T>*> getter, AdderGeneric<T> adder,
00925 RemoverGeneric<T> remover, std::string className, std::string elementName, std::string propertyName,
00926 std::string whatsThis="", bool isList=true, bool isClass=false, bool isEnum=false) :
00927 SimulationControl(className, elementName, propertyName, whatsThis, isList, isClass, isEnum){
00928 _model = model;
00929 _getter= getter;
00930 _adder = adder;
00931 _remover = remover;
00932 _readonly = adder == nullptr;
00933 _propertyType = Util::TypeOf<C>();
00934 }
00935 public:
00936 virtual std::string getValue() const override {
00937 List<T>* tVal = static_cast<List<T>*>(_getter());
00938 std::string strVal = std::to_string(tVal->size()) + " Items";
00939
00940 return strVal;

```

```

00938 }
00939
00940 virtual void setValue(std::string value, bool remove=false) override {
00941 _ensureWritable(remove ? "remove value from" : "add value to");
00942 T newVal{};
00943 if constexpr (std::is_same_v<T, std::string>) {
00944 newVal = value;
00945 } else {
00946 std::istringstream auxStr(value);
00947 auxStr >> newVal;
00948 newVal = static_cast<T>(newVal);
00949 }
00950
00951 if (remove) {
00952 if (!_remover) {
00953 throw std::logic_error("SimulationControlGenericList remover is not defined");
00954 }
00955 _remover(newVal);
00956 } else {
00957 if (!_adder) {
00958 throw std::logic_error("SimulationControlGenericList adder is not defined");
00959 }
00960 bool exists = false;
00961 List<T>* tVal = static_cast<List<T>*>(_getter());
00962 for (auto element : *tVal->list()) {
00963 std::ostringstream auxElement;
00964 auxElement << element;
00965 if (value == auxElement.str()) {
00966 exists = true;
00967 break;
00968 }
00969 }
00970
00971 if (!exists) {
00972 _adder(newVal);
00973 }
00974 }
00975 };
00976
00977 virtual List<std::string>* getStrValues() override {
00978 List<T>* tVal = static_cast<List<T>*>(_getter());
00979 List<std::string>* strOptions = new List<std::string>();
00980
00981 for (auto element : *tVal->list()) {
00982 std::string strVal;
00983 std::ostringstream auxStr;
00984 auxStr << element;
00985 strVal = auxStr.str();
00986
00987 strOptions->insert(strVal);
00988 }
00989 return strOptions;
00990 }
00991 // This method marks scalar lists as editable through the list editor in the universal contract.
00992 virtual bool supportsListEditor() const override { return true; }
00993
00994 private:
00995 M _model;
00996 GetterGeneric<List<T>*> _getter;
00997 AdderGeneric<T> _adder;
00998 RemoverGeneric<T> _remover;
00999 };
01000
01001 template <typename T, typename M, typename C>
01002 class SimulationControlGenericListPointer: public SimulationControl {
01003 public:
01004 using Creator = std::function<T(M, const std::string&)>;
01005 using TypedCreator = std::function<T(M)>;
01006 using ElementTypeNamesGetter = std::function<std::vector<std::string>()>;
01007 using CurrentElementTypeGetter = std::function<std::string()>;
01008 using CurrentElementTypeSetter = std::function<bool(const std::string&)>;
01009 using TypedNamedCreator = std::function<T(M, const std::string&, const std::string&)>;
01010 using ListElementDataDefinitionGetter = std::function<ModelDataDefinition*(int)>;
01011 SimulationControlGenericListPointer(M model, GetterGeneric<List<T>*> getter, AdderGeneric<T>
adder, RemoverGeneric<T> remover, std::string className, std::string elementName, std::string
propertyName, std::string whatsThis="", bool isList=true, bool isClass=true, bool isEnum=false,
Creator creator=nullptr, TypedCreator typedCreator=nullptr, ElementTypeNamesGetter
elementTypeNamesGetter=nullptr, CurrentElementTypeGetter currentElementTypeGetter=nullptr,
CurrentElementTypeSetter currentElementTypeSetter=nullptr, TypedNamedCreator
typedNamedCreator=nullptr, ListElementDataDefinitionGetter listElementDataDefinitionGetter=nullptr) :
SimulationControl(className, elementName, propertyName, whatsThis, isList, isClass, isEnum){
01012 static_assert(std::is_pointer<T>::value, "SimulationControlGenericListPointer requires pointer
type T");
01013 _model = model;
01014 _getter= getter;
01015 _adder = adder;
01016 _remover = remover;

```

```

01017 _creator = creator;
01018 _typedCreator = typedCreator;
01019 _elementTypeNamesGetter = elementTypeNamesGetter;
01020 _currentElementTypeGetter = currentElementTypeGetter;
01021 _currentElementTypeSetter = currentElementTypeSetter;
01022 _typedNamedCreator = typedNamedCreator;
01023 _listElementDataDefinitionGetter = listElementDataDefinitionGetter;
01024 _readonly = adder == nullptr;
01025 _propertyType = Util::TypeOf<C>();
01026 }
01027 public:
01028 virtual std::string getValue() const override {
01029 List<T>* tVal = static_cast<List<T>*>(_getter());
01030 std::string strVal = std::to_string(tVal->size()) + " Items";
01031
01032 return strVal;
01033 }
01034
01035 virtual void setValue(std::string value, bool remove=false) override {
01036 _ensureWritable(remove ? "remove value from" : "add value to");
01037 List<T>* tVal = static_cast<List<T>*>(_getter());
01038 T existingVal = nullptr;
01039 for (auto element : *tVal->list()) {
01040 if (element != nullptr && element->getName() == value) {
01041 existingVal = element;
01042 break;
01043 }
01044 }
01045
01046 if (remove) {
01047 if (!_remover) {
01048 throw std::logic_error("SimulationControlGenericListPointer remover is not
01049 defined");
01050 }
01051 if (existingVal != nullptr) {
01052 _remover(existingVal);
01053 }
01054 } else {
01055 if (!_adder) {
01056 throw std::logic_error("SimulationControlGenericListPointer adder is not
01057 defined");
01058 }
01059 if (existingVal == nullptr) {
01060 if (!createNewListElement(value)) {
01061 throw std::logic_error("SimulationControlGenericListPointer could not create a
01062 new list element");
01063 }
01064 }
01065 }
01066
01067 // This method marks pointer lists as editable through the list editor in the universal contract.
01068 virtual bool supportsListEditor() const override { return true; }
01069 // This method marks pointer lists as supporting explicit typed element creation when writable.
01070 virtual bool supportsNewListElementCreation() const override { return !_readonly; }
01071 // This method exposes concrete types for polymorphic pointer lists such as Set members.
01072 //
01073 // When no provider is configured, the list keeps the legacy single-type creation behavior.
01074 virtual List<std::string>* getCreatableListElementTypes() override {
01075 if (_elementTypeNamesGetter == nullptr) {
01076 return nullptr;
01077 }
01078 List<std::string>* types = new List<std::string>();
01079 for (const std::string& typeName : _elementTypeNamesGetter()) {
01080 if (!typeName.empty()) {
01081 types->insert(typeName);
01082 }
01083 }
01084 return types;
01085 }
01086 // This method reports the selected concrete type for typed polymorphic lists.
01087 virtual std::string getCurrentListElementType() const override {
01088 if (_currentElementTypeGetter != nullptr) {
01089 return _currentElementTypeGetter();
01090 }
01091 return "";
01092 }
01093 // This method lets the GUI configure the selected concrete type before creating members.
01094 virtual bool setCurrentListElementType(const std::string& typeName) override {
01095 if (_currentElementTypeSetter == nullptr) {
01096 return false;
01097 }
01098 return _currentElementTypeSetter(typeName);
01099 }
01100 // This method creates and inserts a typed list element using the configured creator or the
01101 // default constructor path.
01102 virtual bool createNewListElement(const std::string& value = "") override {
01103 if (_typedNamedCreator != nullptr) {

```

```

01100 return createNewListElementOfType(getCurrentListElementType(), value);
01101 }
01102 _ensureWritable("create list element for");
01103 if (!_adder) {
01104 throw std::logic_error("SimulationControlGenericListPointer adder is not defined");
01105 }
01106 T newVal = _createElement(value);
01107 if (newVal == nullptr) {
01108 return false;
01109 }
01110 _adder(newVal);
01111 return true;
01112 }
01113 // This method creates and inserts a member of a caller-selected concrete type.
01114 //
01115 // It is intentionally part of the control contract instead of GUI code so each kernel owner can
01116 // validate type compatibility, naming and model-registration rules in one place.
01117 virtual bool createNewListElementOfType(const std::string& typeName, const std::string& value =
""") override {
01118 _ensureWritable("create typed list element for");
01119 if (!_adder) {
01120 throw std::logic_error("SimulationControlGenericListPointer adder is not defined");
01121 }
01122 if (_typedNamedCreator == nullptr) {
01123 return SimulationControl::createNewListElementOfType(typeName, value);
01124 }
01125 std::string effectiveTypeName = typeName;
01126 if (effectiveTypeName.empty()) {
01127 effectiveTypeName = getCurrentListElementType();
01128 }
01129 if (effectiveTypeName.empty()) {
01130 return false;
01131 }
01132 if (_currentElementTypeSetter != nullptr && !_currentElementTypeSetter(effectiveTypeName)) {
01133 return false;
01134 }
01135 T newVal = _typedNamedCreator(_model, effectiveTypeName, value);
01136 if (newVal == nullptr) {
01137 return false;
01138 }
01139 _adder(newVal);
01140 return true;
01141 }
01142
01143 virtual List<SimulationControl*>* getChildSimulationControls(int index=0) override {
01144 List<T*> tVal = static_cast<List<T*>*>(_getter());
01145 T selectedElement = nullptr;
01146
01147 int current_index = 0;
01148 for (auto element : *tVal->list()) {
01149 if (current_index == index) {
01150 selectedElement = element;
01151 }
01152 current_index++;
01153 }
01154
01155 if (selectedElement) {
01156 if constexpr (requires(T element, M model) { element->ensureSimulationControls(model); })
01157 {
01158 selectedElement->ensureSimulationControls(_model);
01159 }
01160 return selectedElement->getSimulationControls();
01161 } else {
01162 return nullptr;
01163 }
01164 }
01165 virtual ModelDataDefinition* getListElementModelDataDefinition(int index) const override {
01166 if (_listElementDataDefinitionGetter == nullptr) {
01167 return nullptr;
01168 }
01169 return _listElementDataDefinitionGetter(index);
01170 }
01171
01172 virtual List<std::string*>* getStrValues() override {
01173 List<T*> tVal = static_cast<List<T*>*>(_getter());
01174 List<std::string*> strOptions = new List<std::string*>();
01175
01176 for (auto element : *tVal->list()) {
01177 std::string strVal;
01178 if (element != nullptr) {
01179 strVal = element->getName();
01180 } else {
01181 strVal = "";
01182 }
01183 }
01184 strOptions->insert(strVal);

```

```

01185 }
01186 return strOptions;
01187 }
01188
01189 private:
01190 T _createElement(const std::string& value) const {
01191 if (_typedCreator != nullptr) {
01192 return _typedCreator(_model);
01193 }
01194 if (_creator != nullptr) {
01195 return _creator(_model, value);
01196 }
01197 return new C(_model, value);
01198 }
01199
01200 private:
01201 M _model;
01202 GetterGeneric<List<T>*> _getter;
01203 AdderGeneric<T> _adder;
01204 RemoverGeneric<T> _remover;
01205 Creator _creator;
01206 TypedCreator _typedCreator;
01207 ElementTypeNamesGetter _elementTypeNamesGetter;
01208 CurrentElementTypeGetter _currentElementTypeGetter;
01209 CurrentElementTypeSetter _currentElementTypeSetter;
01210 TypedNamedCreator _typedNamedCreator;
01211 ListElementDataDefinitionGetter _listElementDataDefinitionGetter;
01212 };
01213
01214 //namespace\}

```

## 16.311 SimulationExperiment.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: SimulationExperiment.h
00009 * Author: Prof. Rafael Luiz Cancian, Dr. Eng.
00010 *
00011 * Created on 4 de maio de 2021, 15:10
00012 */
00013
00014 #ifndef SIMULATIONEXPERIMENT_H
00015 #define SIMULATIONEXPERIMENT_H
00016
00030 class SimulationExperiment {
00031 public:
00032 SimulationExperiment();
00033 virtual ~SimulationExperiment() = default;
00034 private:
00035
00036 };
00037
00038 #endif /* SIMULATIONEXPERIMENT_H */
00039

```

## 16.312 SimulationReporter\_if.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: SimulationReporter_if.h
00009 * Author: rafael.l Luiz Cancian
00010 *
00011 * Created on 8 de Agosto de 2018, 10:56
00012 */
00013
00014 #ifndef SIMULATIONREPORTER_IF_H
00015 #define SIMULATIONREPORTER_IF_H
00016
00017 #include "../util/List.h"
00018 // #include "StatisticsCollector.h"
00019
00026 class SimulationReporter_if {

```



```

00027 public:
00028 virtual ~SimulationReporter_if() = default;
00033 virtual void showReplicationStatistics() = 0;
00038 virtual void showSimulationStatistics() = 0;
00043 virtual void showSimulationResponses() = 0;
00048 virtual void showSimulationControls() = 0;
00049 //virtual void setFormat(std::string format)=0;
00050 };
00051
00052 #endif /* SIMULATIONREPORTER_IF_H */

```

## 16.313 SimulationReporterDefaultImpl1.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: SimulationReporterDefaultImpl1.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 8 de Agosto de 2018, 10:59
00012 */
00013
00014 #ifndef SIMULATIONREPORTERDEFAULTIMPL1_H
00015 #define SIMULATIONREPORTERDEFAULTIMPL1_H
00016
00017 #include "SimulationReporter_if.h"
00018 #include "model/ModelSimulation.h"
00019 #include "model/Model.h"
00020
00021 //namespace GenesysKernel {
00022
00026 class SimulationReporterDefaultImpl1 : public SimulationReporter_if {
00027 public:
00028 SimulationReporterDefaultImpl1(ModelSimulation* simulation, Model* model,
00029 List<ModelDataDefinition*>* statsCountersSimulation);
00029 virtual ~SimulationReporterDefaultImpl1() = default;
00030 public:
00031 virtual void showReplicationStatistics() override;
00032 virtual void showSimulationStatistics() override;
00033 virtual void showSimulationResponses() override;
00034 virtual void showSimulationControls() override;
00035 private:
00036 ModelSimulation* __simulation;
00037 Model* __model;
00038 private:
00039 List<ModelDataDefinition*>* __statsCountersSimulation;
00040 private:
00041 const unsigned short _w = 9;
00042 const unsigned short _nameW = 36;
00043 };
00044 //namespace\\}
00045 #endif /* SIMULATIONREPORTERDEFAULTIMPL1_H */
00046

```

## 16.314 SimulationScenario.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: SimulationScenario.h
00009 * Author: Prof. Rafael Luiz Cancian, Dr. Eng.
00010 *
00011 * Created on 10 de Outubro de 2018, 18:21
00012 */
00013
00014 #ifndef SIMULATIONSCENARIO_H
00015 #define SIMULATIONSCENARIO_H
00016
00017 #include <string>
00018 #include <list>
00019
00020 class Simulator;
00021
00038 class SimulationScenario {

```

```

00039 public:
00040 SimulationScenario();
00041 virtual ~SimulationScenario();
00042 public: // results
00043 bool startSimulation(Simulator *sim, std::string& errorMessage);
00044 std::list<std::pair<std::string, double>*>* getResponseValues() const;
00045 std::list<std::pair<std::string, double>*>* getControlValues() const;
00046 double getResponseValue(const std::string& responseName);
00047 public: // gets and sets
00048 void setModelFilename(std::string _modelFilename);
00049 std::string getModelFilename() const;
00050 void setScenarioName(std::string _name);
00051 std::string getScenarioName() const;
00052 void setScenarioDescription(std::string _scenarioDescription);
00053 std::string getScenarioDescription() const;
00054 std::list<std::string*>* getSelectedControls() const; // access to the list to insert or remove
00055 controls
00056 double getControlValue(const std::string& controlName);
00057 std::list<std::string*>* getSelectedResponses() const; // access to the list to insert or remove
00058 responses
00059 // @ToDo: (importante): Clarify ownership and replacement semantics for selected controls.
00060 // This setter currently allocates a new list and replaces the stored pointer.
00061 void setSelectedControls(std::list<std::string*>* selectedControls);
00062 void setControl(std::string name, double value) const;
00063 private:
00064 std::string _scenarioName;
00065 std::string _scenarioDescription;
00066 std::string _modelFilename;
00067 std::list<std::string*>* _selectedControls = new std::list<std::string*>();
00068 std::list<std::string*>* _selectedResponses = new std::list<std::string*>();
00069 std::list<std::pair<std::string, double>*>* _controlValues = new std::list<std::pair<std::string,
00070 double>*>();
00071 // @ToDo: (importante): Define initialization and ownership rules for response values.
00072 std::list<std::pair<std::string, double>*>* _responseValues = new std::list<std::pair<std::string,
00073 double>*>();
00074 };
00075
00076 #endif /* SIMULATIONSCENARIO_H */

```

## 16.315 Simulator.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Genesys.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 21 de Junho de 2018, 12:48
00012 */
00013
00014 #ifndef GENESYS_H
00015 #define GENESYS_H
00016
00017 #include <string>
00018 #include <iostream>
00019 #include "model/Model.h"
00020 #include "Plugin.h"
00021 #include "../util/List.h"
00022 #include "LicenceManager.h"
00023 #include "PluginManager.h"
00024 #include "model/ModelManager.h"
00025 // #include "ToolManager.h"
00026 #include "ParserManager.h"
00027 #include "ExperimentManager.h"
00028
00029 //namespace GenesysKernel {
00030
00031 class PluginManager;
00032
00033 /* https://www.doxygen.nl/manual/commands.html
00034 * Brief description \class \implements
00035 *
00036 * Detailed description
00037 * \param \return
00038 */
00039
00040 class Simulator {
00041 typedef void (*eventHandler)();
00042 public:
00043 Simulator();
00044 virtual ~Simulator();

```

```

00049 public: // only get
00054 std::string getVersion() const;
00059 unsigned int getVersionNumber() const;
00064 std::string getName() const;
00069 LicenceManager* getLicenceManager() const;
00074 PluginManager* getPluginManager() const;
00079 ModelManager* getModelManager() const;
00084 TraceManager* getTraceManager() const;
00089 ParserManager* getParserManager() const;
00094 ExperimentManager* getExperimentManager() const;
00095
00096 private:
00097 List<Plugin*>* _completePluginsFieldsAndTemplate();
00098 friend class PluginManager; // @ToDo: (pequena alteração): should be only member function
00099 PluginManager::completePluginsFieldsAndTemplate()
00100 private: // attributes 1:1 objects
00101 LicenceManager* _licenceManager;
00102 PluginManager* _pluginManager;
00103 ModelManager* _modelManager;
00104 TraceManager* _traceManager;
00105 ParserManager* _parserManager;
00106 ExperimentManager* _experimentManager;
00107
00108 private: // attributes 1:1 native
00109 const std::string _name = "GenESyS - GENeric and Expansible System Simulator";
00110 const std::string _versionName = "anaiera";
00111 const unsigned int _versionNumber = 260705;
00112 };
00113 //namespace\}
00114
00115 // passing a "C" class factory function which instantiates the class.
00116 typedef /*GenesysKernel::**/ Simulator* GenesysSimulator;
00117 #endif /* GENESYS_H */

```

## 16.316 SimulatorFacade.h

```

00001 /*
00002 * SimulatorFacade.h
00003 *
00004 * High-level non-owning facade over Simulator and its main composed services.
00005 *
00006 * This header delegates calls to Simulator, its managers, the current Model and
00007 * the current Model sub-services without forcing callers to navigate manually
00008 * through getPluginManager(), getModelManager(), getTraceManager(), etc.
00009 *
00010 * All methods tolerate a null _simulator or missing current model:
00011 * - pointer returns -> nullptr
00012 * - bool returns -> false
00013 * - numeric returns -> 0
00014 * - string returns -> ""
00015 * - void methods -> no-op
00016 *
00017 * Method prefixes resolve name conflicts between delegates:
00018 * (none) Simulator direct methods
00019 * licence* LicenceManager
00020 * plugin* PluginManager
00021 * model* ModelManager / Model
00022 * info* ModelInfo
00023 * sim* ModelSimulation
00024 * data* ModelDataManager
00025 * component* ModelComponentManager
00026 * event* OnEventManager
00027 * trace*, setTrace* TraceManager
00028 * parser* ParserManager
00029 * simulationExp* ExperimentManager
00030 */
00031
00032 #ifndef SIMULATORFACADE_H
00033 #define SIMULATORFACADE_H
00034
00035 #include "Simulator.h"
00036 #include "LicenceManager.h"
00037 #include "PluginManager.h"
00038 #include "TraceManager.h"
00039 #include "ParserManager.h"
00040 #include "ParserChangesInformation.h"
00041 #include "ExperimentManager.h"
00042 #include "SimulationExperiment.h"
00043 #include "OnEventManager.h"
00044 #include "model/ModelManager.h"
00045 #include "model/Model.h"
00046 #include "model/ModelInfo.h"
00047 #include "model/ModelSimulation.h"
00048 #include "model/ModelDataManager.h"

```

```

00049 #include "model/ModelComponentManager.h"
00050 #include "model/ModelComponent.h"
00051 #include "model/ModelDataDefinition.h"
00052
00060 class SimulatorFacade {
00061 public:
00062 explicit SimulatorFacade(Simulator* simulator) : _simulator(simulator) {}
00063 virtual ~SimulatorFacade() = default;
00064
00065 // -----
00066 // Simulator -- direct methods
00067 // -----
00068
00069 std::string getVersion() const {
00070 return _simulator != nullptr ? _simulator->getVersion() : "";
00071 }
00072
00073 unsigned int getVersionNumber() const {
00074 return _simulator != nullptr ? _simulator->getVersionNumber() : 0u;
00075 }
00076
00077 std::string getName() const {
00078 return _simulator != nullptr ? _simulator->getName() : "";
00079 }
00080
00081 // -----
00082 // LicenceManager
00083 // -----
00084
00085 const std::string licenceShow() const {
00086 LicenceManager* m = licenceManager_();
00087 return m != nullptr ? m->showLicence() : "";
00088 }
00089
00090 const std::string licenceShowLimits() const {
00091 LicenceManager* m = licenceManager_();
00092 return m != nullptr ? m->showLimits() : "";
00093 }
00094
00095 const std::string licenceShowActivationCode() const {
00096 LicenceManager* m = licenceManager_();
00097 return m != nullptr ? m->showActivationCode() : "";
00098 }
00099
00100 bool licenceLookforActivationCode() {
00101 LicenceManager* m = licenceManager_();
00102 return m != nullptr ? m->lookforActivationCode() : false;
00103 }
00104
00105 bool licenceInsertActivationCode() {
00106 LicenceManager* m = licenceManager_();
00107 return m != nullptr ? m->insertActivationCode() : false;
00108 }
00109
00110 void licenceRemoveActivationCode() {
00111 LicenceManager* m = licenceManager_();
00112 if (m != nullptr) m->removeActivationCode();
00113 }
00114
00115 unsigned int licenceGetModelComponentsLimit() {
00116 LicenceManager* m = licenceManager_();
00117 return m != nullptr ? m->getModelComponentsLimit() : 0u;
00118 }
00119
00120 unsigned int licenceGetModelDatasLimit() {
00121 LicenceManager* m = licenceManager_();
00122 return m != nullptr ? m->getModelDatasLimit() : 0u;
00123 }
00124
00125 unsigned int licenceGetEntityLimit() {
00126 LicenceManager* m = licenceManager_();
00127 return m != nullptr ? m->getEntityLimit() : 0u;
00128 }
00129
00130 unsigned int licenceGetHostsLimit() {
00131 LicenceManager* m = licenceManager_();
00132 return m != nullptr ? m->getHostsLimit() : 0u;
00133 }
00134
00135 unsigned int licenceGetThreadsLimit() {
00136 LicenceManager* m = licenceManager_();
00137 return m != nullptr ? m->getThreadsLimit() : 0u;
00138 }
00139
00140 // -----
00141 // PluginManager
00142 // -----

```

```

00143
00144 std::string showPlugins() {
00145 PluginManager* m = pluginManager_();
00146 return m != nullptr ? m->show() : "";
00147 }
00148
00149 List<Plugin*>* completePluginsFieldsAndTemplates() {
00150 PluginManager* m = pluginManager_();
00151 return m != nullptr ? m->completePluginsFieldsAndTemplates() : nullptr;
00152 }
00153
00154 bool pluginCheck(const std::string& dynamicLibraryFilename) {
00155 PluginManager* m = pluginManager_();
00156 return m != nullptr ? m->check(dynamicLibraryFilename) : false;
00157 }
00158
00159 SystemDependencyCheckResult pluginCheckSystemDependencies(const std::string&
dynamicLibraryFilename) {
00160 PluginManager* m = pluginManager_();
00161 return m != nullptr ? m->checkSystemDependencies(dynamicLibraryFilename) :
SystemDependencyCheckResult{};
00162 }
00163
00164 List<std::string*>* discoverPluginFileNames() const {
00165 PluginManager* m = pluginManager_();
00166 return m != nullptr ? m->discoverPluginFileNames() : nullptr;
00167 }
00168
00169 List<PluginLoadIssue*>* getPluginLoadIssues() const {
00170 PluginManager* m = pluginManager_();
00171 return m != nullptr ? m->getPluginLoadIssues() : nullptr;
00172 }
00173
00174 void clearPluginLoadIssues() {
00175 PluginManager* m = pluginManager_();
00176 if (m != nullptr) m->clearPluginLoadIssues();
00177 }
00178
00179 void clearPluginLoadIssue(const std::string& dynamicLibraryFilename) {
00180 PluginManager* m = pluginManager_();
00181 if (m != nullptr) m->clearPluginLoadIssue(dynamicLibraryFilename);
00182 }
00183
00184 Plugin* insertPlugin(const std::string& dynamicLibraryFilename) {
00185 PluginManager* m = pluginManager_();
00186 return m != nullptr ? m->insert(dynamicLibraryFilename) : nullptr;
00187 }
00188
00189 Plugin* insertPlugin(const std::string& dynamicLibraryFilename, const PluginInsertionOptions&
options) {
00190 PluginManager* m = pluginManager_();
00191 return m != nullptr ? m->insert(dynamicLibraryFilename, options) : nullptr;
00192 }
00193
00194 bool removePlugin(const std::string& dynamicLibraryFilename) {
00195 PluginManager* m = pluginManager_();
00196 return m != nullptr ? m->remove(dynamicLibraryFilename) : false;
00197 }
00198
00199 bool removePlugin(Plugin* plugin) {
00200 PluginManager* m = pluginManager_();
00201 return m != nullptr ? m->remove(plugin) : false;
00202 }
00203
00204 Plugin* findPlugin(const std::string& pluginTypeName) {
00205 PluginManager* m = pluginManager_();
00206 return m != nullptr ? m->find(pluginTypeName) : nullptr;
00207 }
00208
00209 std::vector<std::string> getDataDefinitionPluginTypeNames() const {
00210 PluginManager* m = pluginManager_();
00211 return m != nullptr ? m->getDataDefinitionPluginTypeNames() : std::vector<std::string>{};
00212 }
00213
00214 std::string pluginSourceIncludePathFor(const std::string& pluginTypeName) {
00215 PluginManager* m = pluginManager_();
00216 return m != nullptr ? m->sourceIncludePathFor(pluginTypeName) : "";
00217 }
00218
00219 List<Plugin*>* autoInsertPlugins(const std::string& pluginsListFilename, const bool
lookForPluginsIfFilenameNotFound = true) {
00220 PluginManager* m = pluginManager_();
00221 return m != nullptr ? m->autoInsertPlugins(pluginsListFilename,
lookForPluginsIfFilenameNotFound) : nullptr;
00222 }
00223
00224 List<Plugin*>* autoInsertPlugins(const std::string& pluginsListFilename, const bool

```

```

 lookForPluginsIfFilenameNotFound, const PluginInsertionOptions& options) {
00225 PluginManager* m = pluginManager_();
00226 return m != nullptr ? m->autoInsertPlugins(pluginListFilename,
 lookForPluginsIfFilenameNotFound, options) : nullptr;
00227 }
00228
00229 List<Plugin*>* autoInsertPlugins() {
00230 PluginManager* m = pluginManager_();
00231 return m != nullptr ? m->autoInsertPlugins() : nullptr;
00232 }
00233
00234 List<Plugin*>* autoInsertPlugins(const PluginInsertionOptions& options) {
00235 PluginManager* m = pluginManager_();
00236 return m != nullptr ? m->autoInsertPlugins(options) : nullptr;
00237 }
00238
00239 Plugin* firstPlugin() {
00240 PluginManager* m = pluginManager_();
00241 return m != nullptr ? m->front() : nullptr;
00242 }
00243
00244 Plugin* nextPlugin() {
00245 PluginManager* m = pluginManager_();
00246 return m != nullptr ? m->next() : nullptr;
00247 }
00248
00249 Plugin* lastPlugin() {
00250 PluginManager* m = pluginManager_();
00251 return m != nullptr ? m->last() : nullptr;
00252 }
00253
00254 unsigned int pluginCount() {
00255 PluginManager* m = pluginManager_();
00256 return m != nullptr ? m->size() : 0u;
00257 }
00258
00259 Plugin* getPluginAtRank(unsigned int rank) {
00260 PluginManager* m = pluginManager_();
00261 return m != nullptr ? m->getAtRank(rank) : nullptr;
00262 }
00263
00264 ModelDataDefinition* newInstance(const std::string& pluginTypename, Model* model, std::string name
= "") {
00265 PluginManager* m = pluginManager_();
00266 return m != nullptr ? m->newInstance(pluginTypename, model, name) : nullptr;
00267 }
00268
00269 template <typename T>
00270 T* newInstance(Model* model, std::string name = "") {
00271 PluginManager* m = pluginManager_();
00272 return m != nullptr ? m->newInstance<T>(model, name) : nullptr;
00273 }
00274
00275 // -----
00276 // ModelManager
00277 // -----
00278
00279 Model* newModel() {
00280 ModelManager* m = modelManager_();
00281 return m != nullptr ? m->newModel() : nullptr;
00282 }
00283
00284 void insertModel(Model* model) {
00285 ModelManager* m = modelManager_();
00286 if (m != nullptr) m->insert(model);
00287 }
00288
00289 void removeModel(Model* model) {
00290 ModelManager* m = modelManager_();
00291 if (m != nullptr) m->remove(model);
00292 }
00293
00294 bool setCurrentModel(Model* model) {
00295 ModelManager* m = modelManager_();
00296 return m != nullptr ? m->setCurrent(model) : false;
00297 }
00298
00299 bool saveModel(std::string filename) {
00300 ModelManager* m = modelManager_();
00301 return m != nullptr ? m->saveModel(filename) : false;
00302 }
00303
00304 Model* loadModel(std::string filename) {
00305 ModelManager* m = modelManager_();
00306 return m != nullptr ? m->loadModel(filename) : nullptr;
00307 }
00308

```

```

00309 Model* createModelFromLanguage(std::string modelSpecification) {
00310 ModelManager* m = modelManager_();
00311 return m != nullptr ? m->createFromLanguage(modelSpecification) : nullptr;
00312 }
00313
00314 unsigned int modelCount() {
00315 ModelManager* m = modelManager_();
00316 return m != nullptr ? m->size() : 0u;
00317 }
00318
00319 std::vector<Model*> models() const {
00320 ModelManager* m = modelManager_();
00321 return m != nullptr ? m->models() : std::vector<Model*>{};
00322 }
00323
00324 bool hasModel(Model* model) const {
00325 ModelManager* m = modelManager_();
00326 return m != nullptr ? m->hasModel(model) : false;
00327 }
00328
00329 int indexOfModel(Model* model) const {
00330 ModelManager* m = modelManager_();
00331 return m != nullptr ? m->indexOf(model) : -1;
00332 }
00333
00334 Model* modelAt(unsigned int index) const {
00335 ModelManager* m = modelManager_();
00336 return m != nullptr ? m->modelAt(index) : nullptr;
00337 }
00338
00339 int currentModelIndex() const {
00340 ModelManager* m = modelManager_();
00341 return m != nullptr ? m->currentIndex() : -1;
00342 }
00343
00344 Model* firstModel() {
00345 ModelManager* m = modelManager_();
00346 return m != nullptr ? m->front() : nullptr;
00347 }
00348
00349 Model* lastModel() {
00350 ModelManager* m = modelManager_();
00351 return m != nullptr ? m->last() : nullptr;
00352 }
00353
00354 Model* currentModel() {
00355 return currentModel_();
00356 }
00357
00358 Model* nextModel() {
00359 ModelManager* m = modelManager_();
00360 return m != nullptr ? m->next() : nullptr;
00361 }
00362
00363 Model* previousModel() {
00364 ModelManager* m = modelManager_();
00365 return m != nullptr ? m->previous() : nullptr;
00366 }
00367
00368 bool canGoNextModel() const {
00369 ModelManager* m = modelManager_();
00370 return m != nullptr ? m->canGoNext() : false;
00371 }
00372
00373 bool canGoPreviousModel() const {
00374 ModelManager* m = modelManager_();
00375 return m != nullptr ? m->canGoPrevious() : false;
00376 }
00377
00378 // -----
00379 // TraceManager
00380 // -----
00381
00382 static std::string traceConvertEnumToStr(TraceManager::Level level) {
00383 return TraceManager::convertEnumToStr(level);
00384 }
00385
00386 void addTraceHandler(traceListener traceListener) {
00387 TraceManager* m = traceManager_();
00388 if (m != nullptr) m->addTraceHandler(traceListener);
00389 }
00390
00391 void addTraceReportHandler(traceListener traceReportListener) {
00392 TraceManager* m = traceManager_();
00393 if (m != nullptr) m->addTraceReportHandler(traceReportListener);
00394 }
00395

```

```

00396 void addTraceSimulationHandler(traceSimulationListener traceSimulationListener) {
00397 TraceManager* m = traceManager_();
00398 if (m != nullptr) m->addTraceSimulationHandler(traceSimulationListener);
00399 }
00400
00401 void addTraceErrorHandler(traceErrorListener traceErrorListener) {
00402 TraceManager* m = traceManager_();
00403 if (m != nullptr) m->addTraceErrorHandler(traceErrorListener);
00404 }
00405
00406 template<typename Class>
00407 void addTraceHandler(Class* object, void (Class::*function)(TraceEvent)) {
00408 TraceManager* m = traceManager_();
00409 if (m != nullptr) m->addTraceHandler(object, function);
00410 }
00411
00412 template<typename Class>
00413 void addTraceErrorHandler(Class* object, void (Class::*function)(TraceErrorEvent)) {
00414 TraceManager* m = traceManager_();
00415 if (m != nullptr) m->addTraceErrorHandler(object, function);
00416 }
00417
00418 template<typename Class>
00419 void addTraceReportHandler(Class* object, void (Class::*function)(TraceEvent)) {
00420 TraceManager* m = traceManager_();
00421 if (m != nullptr) m->addTraceReportHandler(object, function);
00422 }
00423
00424 template<typename Class>
00425 void addTraceSimulationHandler(Class* object, void (Class::*function)(TraceSimulationEvent)) {
00426 TraceManager* m = traceManager_();
00427 if (m != nullptr) m->addTraceSimulationHandler(object, function);
00428 }
00429
00430 void addTraceSimulationExceptionRuleModelData(void* thisobject) {
00431 TraceManager* m = traceManager_();
00432 if (m != nullptr) m->addTraceSimulationExceptionRuleModelData(thisobject);
00433 }
00434
00435 void beginTraceShutdown() {
00436 TraceManager* m = traceManager_();
00437 if (m != nullptr) m->beginShutdown();
00438 }
00439
00440 void trace(const std::string& text, TraceManager::Level level = TraceManager::Level::L8_detailed)
00441 {
00442 TraceManager* m = traceManager_();
00443 if (m != nullptr) m->trace(text, level);
00444 }
00445
00446 void traceError(const std::string& text, const std::exception& e) {
00447 TraceManager* m = traceManager_();
00448 if (m != nullptr) m->traceError(text, e);
00449 }
00450
00451 void traceError(const std::string& text, TraceManager::Level level =
TraceManager::Level::L1_errorFatal) {
00452 TraceManager* m = traceManager_();
00453 if (m != nullptr) m->traceError(text, level);
00454 }
00455
00456 void traceReport(const std::string& text, TraceManager::Level level =
TraceManager::Level::L2_results) {
00457 TraceManager* m = traceManager_();
00458 if (m != nullptr) m->traceReport(text, level);
00459 }
00460
00461 void traceSimulation(void* thisobject, double time, Entity* entity, ModelComponent* component,
const std::string& text, TraceManager::Level level = TraceManager::Level::L8_detailed, bool showAnyway
= false) {
00462 TraceManager* m = traceManager_();
00463 if (m != nullptr) m->traceSimulation(thisobject, time, entity, component, text, level,
showAnyway);
00464 }
00465
00466 void traceSimulation(void* thisobject, const std::string& text, TraceManager::Level level =
TraceManager::Level::L8_detailed, bool showAnyway = false) {
00467 TraceManager* m = traceManager_();
00468 if (m != nullptr) m->traceSimulation(thisobject, text, level, showAnyway);
00469 }
00470
00471 List<std::string>* traceGetErrorMessages() const {
00472 TraceManager* m = traceManager_();
00473 return m != nullptr ? m->errorMessages() : nullptr;
00474 }
00475
00476 void setTraceLevel(TraceManager::Level traceLevel) {

```



```

00476 TraceManager* m = traceManager_();
00477 if (m != nullptr) m->setTraceLevel(traceLevel);
00478 }
00479
00480 TraceManager::Level getTraceLevel() const {
00481 TraceManager* m = traceManager_();
00482 return m != nullptr ? m->getTraceLevel() : TraceManager::Level::L0_noTraces;
00483 }
00484
00485 void setTraceSimulationRuleAllAllowed(bool traceSimulationRuleAllAllowed) {
00486 TraceManager* m = traceManager_();
00487 if (m != nullptr) m->setTraceSimulationRuleAllAllowed(traceSimulationRuleAllAllowed);
00488 }
00489
00490 bool isTraceSimulationRuleAllAllowed() const {
00491 TraceManager* m = traceManager_();
00492 return m != nullptr ? m->isTraceSimulationRuleAllAllowed() : false;
00493 }
00494
00495 // -----
00496 // ParserManager
00497 // -----
00498
00499 ParserManager::GenerateNewParserResult parserGenerateNewParser(ParserChangesInformation* changes)
00500 {
00501 ParserManager* m = parserManager_();
00502 return m != nullptr ? m->generateNewParser(changes) :
00503 ParserManager::GenerateNewParserResult{};
00504 }
00505
00506 bool parserConnectNewParser(ParserManager::NewParser newParser) {
00507 ParserManager* m = parserManager_();
00508 return m != nullptr ? m->connectNewParser(newParser) : false;
00509 }
00510
00511 // -----
00512 // ExperimentManager
00513 // -----
00514
00515 SimulationExperiment* newSimulationExperiment() {
00516 ExperimentManager* m = experimentManager_();
00517 return m != nullptr ? m->newSimulationExperiment() : nullptr;
00518 }
00519
00520 void insertSimulationExperiment(SimulationExperiment* experiment) {
00521 ExperimentManager* m = experimentManager_();
00522 if (m != nullptr) m->insert(experiment);
00523 }
00524
00525 void removeSimulationExperiment(SimulationExperiment* experiment) {
00526 ExperimentManager* m = experimentManager_();
00527 if (m != nullptr) m->remove(experiment);
00528 }
00529
00530 void setCurrentSimulationExperiment(SimulationExperiment* experiment) {
00531 ExperimentManager* m = experimentManager_();
00532 if (m != nullptr) m->setCurrent(experiment);
00533 }
00534
00535 bool saveSimulationExperiment(std::string filename) {
00536 ExperimentManager* m = experimentManager_();
00537 return m != nullptr ? m->saveSimulationExperiment(filename) : false;
00538 }
00539
00540 bool loadSimulationExperiment(std::string filename) {
00541 ExperimentManager* m = experimentManager_();
00542 return m != nullptr ? m->loadSimulationExperiment(filename) : false;
00543 }
00544
00545 unsigned int simulationExperimentCount() {
00546 ExperimentManager* m = experimentManager_();
00547 return m != nullptr ? m->size() : 0u;
00548 }
00549
00550 SimulationExperiment* firstSimulationExperiment() {
00551 ExperimentManager* m = experimentManager_();
00552 return m != nullptr ? m->front() : nullptr;
00553 }
00554
00555 SimulationExperiment* currentSimulationExperiment() {
00556 ExperimentManager* m = experimentManager_();
00557 return m != nullptr ? m->current() : nullptr;
00558 }
00559
00560 SimulationExperiment* nextSimulationExperiment() {
00561 ExperimentManager* m = experimentManager_();
00562 return m != nullptr ? m->next() : nullptr;

```

```

00561 }
00562
00563 List<SimulationExperiment*> getSimulationExperiments() const {
00564 ExperimentManager* m = experimentManager_();
00565 return m != nullptr ? m->getExperiments() : nullptr;
00566 }
00567
00568 // -----
00569 // Model -- operations on current model
00570 // -----
00571
00572 bool modelSave(std::string filename) {
00573 Model* model = currentModel_();
00574 return model != nullptr ? model->save(filename) : false;
00575 }
00576
00577 bool modelLoad(std::string filename) {
00578 Model* model = currentModel_();
00579 return model != nullptr ? model->load(filename) : false;
00580 }
00581
00582 bool modelCheck() {
00583 Model* model = currentModel_();
00584 return model != nullptr ? model->check() : false;
00585 }
00586
00587 void modelClear() {
00588 Model* model = currentModel_();
00589 if (model != nullptr) model->clear();
00590 }
00591
00592 std::string modelShowLanguage() {
00593 Model* model = currentModel_();
00594 return model != nullptr ? model->showLanguage() : "";
00595 }
00596
00597 void modelShow() {
00598 Model* model = currentModel_();
00599 if (model != nullptr) model->show();
00600 }
00601
00602 bool modelInsert(ModelDataDefinition* elemOrComp) {
00603 Model* model = currentModel_();
00604 return model != nullptr ? model->insert(elemOrComp) : false;
00605 }
00606
00607 void modelRemove(ModelDataDefinition* elemOrComp) {
00608 Model* model = currentModel_();
00609 if (model != nullptr) model->remove(elemOrComp);
00610 }
00611
00612 std::list<ModelDataDefinition*> modelCollectDataDefinitionsRemovedWith(const
std::list<ModelDataDefinition*>& roots) const {
00613 Model* model = currentModel_();
00614 return model != nullptr ? model->collectDataDefinitionsRemovedWith(roots) :
std::list<ModelDataDefinition*>{};
00615 }
00616
00617 Entity* modelCreateEntity(std::string name, bool insertIntoModel = true) {
00618 Model* model = currentModel_();
00619 return model != nullptr ? model->createEntity(name, insertIntoModel) : nullptr;
00620 }
00621
00622 void modelRemoveEntity(Entity* entity) {
00623 Model* model = currentModel_();
00624 if (model != nullptr) model->removeEntity(entity);
00625 }
00626
00627 void modelSendEntityToComponent(Entity* entity, Connection* connection, double timeDelay = 0.0) {
00628 Model* model = currentModel_();
00629 if (model != nullptr) model->sendEntityToComponent(entity, connection, timeDelay);
00630 }
00631
00632 void modelSendEntityToComponent(Entity* entity, ModelComponent* component, double timeDelay = 0.0,
unsigned int componentInputPortNumber = 0) {
00633 Model* model = currentModel_();
00634 if (model != nullptr) model->sendEntityToComponent(entity, component, timeDelay,
componentInputPortNumber);
00635 }
00636
00637 double modelParseExpression(const std::string expression) {
00638 Model* model = currentModel_();
00639 return model != nullptr ? model->parseExpression(expression) : 0.0;
00640 }
00641
00642 double modelParseExpression(const std::string expression, bool& success, std::string&
errorMessage) {

```

```

00643 Model* model = currentModel_();
00644 if (model != nullptr) return model->parseExpression(expression, success, errorMessage);
00645 success = false; errorMessage = ""; return 0.0;
00646 }
00647
00648 bool modelCheckExpression(const std::string expression, const std::string expressionName,
std::string& errorMessage) {
00649 Model* model = currentModel_();
00650 return model != nullptr ? model->checkExpression(expression, expressionName, errorMessage) :
false;
00651 }
00652
00653 void modelCheckReferencesToDataDefinitions(std::string expression, std::map<std::string,
std::list<std::string>>* referencedDataDefinitions) {
00654 Model* model = currentModel_();
00655 if (model != nullptr) model->checkReferencesToDataDefinitions(expression,
referencedDataDefinitions);
00656 }
00657
00658 Util::identification modelGetId() const {
00659 Model* model = currentModel_();
00660 return model != nullptr ? model->getId() : static_cast<Util::identification>(0);
00661 }
00662
00663 bool modelHasChanged() const {
00664 Model* model = currentModel_();
00665 return model != nullptr ? model->hasChanged() : false;
00666 }
00667
00668 void modelSetHasChanged(bool hasChanged) {
00669 Model* model = currentModel_();
00670 if (model != nullptr) model->setHasChanged(hasChanged);
00671 }
00672
00673 List<Event*>* modelGetFutureEvents() const {
00674 Model* model = currentModel_();
00675 return model != nullptr ? model->getFutureEvents() : nullptr;
00676 }
00677
00678 List<SimulationControl*>* modelGetControls() const {
00679 Model* model = currentModel_();
00680 return model != nullptr ? model->getControls() : nullptr;
00681 }
00682
00683 List<SimulationResponse*>* modelGetResponses() const {
00684 Model* model = currentModel_();
00685 return model != nullptr ? model->getResponses() : nullptr;
00686 }
00687
00688 Persistence_if* modelGetPersistence() const {
00689 Model* model = currentModel_();
00690 return model != nullptr ? model->getPersistence() : nullptr;
00691 }
00692
00693 unsigned int modelGetLevel() const {
00694 Model* model = currentModel_();
00695 return model != nullptr ? model->getLevel() : 0u;
00696 }
00697
00698 // -----
00699 // ModelInfo -- metadata of current model
00700 // -----
00701
00702 std::string infoShow() {
00703 ModelInfo* info = modelInfo_();
00704 return info != nullptr ? info->show() : "";
00705 }
00706
00707 void infoSetName(std::string name) {
00708 ModelInfo* info = modelInfo_();
00709 if (info != nullptr) info->setName(name);
00710 }
00711
00712 std::string infoGetName() const {
00713 ModelInfo* info = modelInfo_();
00714 return info != nullptr ? info->getName() : "";
00715 }
00716
00717 void infoSetAnalystName(std::string analystName) {
00718 ModelInfo* info = modelInfo_();
00719 if (info != nullptr) info->setAnalystName(analystName);
00720 }
00721
00722 std::string infoGetAnalystName() const {
00723 ModelInfo* info = modelInfo_();
00724 return info != nullptr ? info->getAnalystName() : "";
00725 }

```

```

00726
00727 void infoSetDescription(std::string description) {
00728 ModelInfo* info = modelInfo_();
00729 if (info != nullptr) info->setDescription(description);
00730 }
00731
00732 std::string infoGetDescription() const {
00733 ModelInfo* info = modelInfo_();
00734 return info != nullptr ? info->getDescription() : "";
00735 }
00736
00737 void infoSetProjectTitle(std::string projectTitle) {
00738 ModelInfo* info = modelInfo_();
00739 if (info != nullptr) info->setProjectTitle(projectTitle);
00740 }
00741
00742 std::string infoGetProjectTitle() const {
00743 ModelInfo* info = modelInfo_();
00744 return info != nullptr ? info->getProjectTitle() : "";
00745 }
00746
00747 void infoSetVersion(std::string version) {
00748 ModelInfo* info = modelInfo_();
00749 if (info != nullptr) info->setVersion(version);
00750 }
00751
00752 std::string infoGetVersion() const {
00753 ModelInfo* info = modelInfo_();
00754 return info != nullptr ? info->getVersion() : "";
00755 }
00756
00757 void infoLoadInstance(PersistenceRecord* fields) {
00758 ModelInfo* info = modelInfo_();
00759 if (info != nullptr) info->loadInstance(fields);
00760 }
00761
00762 void infoSaveInstance(PersistenceRecord* fields) {
00763 ModelInfo* info = modelInfo_();
00764 if (info != nullptr) info->saveInstance(fields);
00765 }
00766
00767 bool infoHasChanged() const {
00768 ModelInfo* info = modelInfo_();
00769 return info != nullptr ? info->hasChanged() : false;
00770 }
00771
00772 void infoSetHasChanged(bool hasChanged) {
00773 ModelInfo* info = modelInfo_();
00774 if (info != nullptr) info->setHasChanged(hasChanged);
00775 }
00776
00777 // -----
00778 // ModelSimulation -- simulation control on current model
00779 // -----
00780
00781 std::string simShow() {
00782 ModelSimulation* s = modelSimulation_();
00783 return s != nullptr ? s->show() : "";
00784 }
00785
00786 void simStart() {
00787 ModelSimulation* s = modelSimulation_();
00788 if (s != nullptr) s->start();
00789 }
00790
00791 void simPause() {
00792 ModelSimulation* s = modelSimulation_();
00793 if (s != nullptr) s->pause();
00794 }
00795
00796 void simStep() {
00797 ModelSimulation* s = modelSimulation_();
00798 if (s != nullptr) s->step();
00799 }
00800
00801 void simStop() {
00802 ModelSimulation* s = modelSimulation_();
00803 if (s != nullptr) s->stop();
00804 }
00805
00806 void simSetNumberOfReplications(unsigned int numberOfReplications) {
00807 ModelSimulation* s = modelSimulation_();
00808 if (s != nullptr) s->setNumberOfReplications(numberOfReplications);
00809 }
00810
00811 unsigned int simGetNumberOfReplications() const {
00812 ModelSimulation* s = modelSimulation_();

```

```

00813 return s != nullptr ? s->getNumberOfReplications() : 0u;
00814 }
00815
00816 void simSetReplicationLength(double replicationLength, Util::TimeUnit replicationLengthTimeUnit =
Util::TimeUnit::unknown) {
00817 ModelSimulation* s = modelSimulation_();
00818 if (s != nullptr) s->setReplicationLength(replicationLength, replicationLengthTimeUnit);
00819 }
00820
00821 double simGetReplicationLength() const {
00822 ModelSimulation* s = modelSimulation_();
00823 return s != nullptr ? s->getReplicationLength() : 0.0;
00824 }
00825
00826 void simSetReplicationLengthTimeUnit(Util::TimeUnit replicationLengthTimeUnit) {
00827 ModelSimulation* s = modelSimulation_();
00828 if (s != nullptr) s->setReplicationLengthTimeUnit(replicationLengthTimeUnit);
00829 }
00830
00831 Util::TimeUnit simGetReplicationLengthTimeUnit() const {
00832 ModelSimulation* s = modelSimulation_();
00833 return s != nullptr ? s->getReplicationLengthTimeUnit() : Util::TimeUnit::unknown;
00834 }
00835
00836 void simSetReplicationReportBaseTimeUnit(Util::TimeUnit replicationReportBaseTimeUnit) {
00837 ModelSimulation* s = modelSimulation_();
00838 if (s != nullptr) s->setReplicationReportBaseTimeUnit(replicationReportBaseTimeUnit);
00839 }
00840
00841 Util::TimeUnit simGetReplicationBaseTimeUnit() const {
00842 ModelSimulation* s = modelSimulation_();
00843 return s != nullptr ? s->getReplicationBaseTimeUnit() : Util::TimeUnit::unknown;
00844 }
00845
00846 void simSetWarmUpPeriod(double warmUpPeriod, Util::TimeUnit warmUpPeriodTimeUnit =
Util::TimeUnit::unknown) {
00847 ModelSimulation* s = modelSimulation_();
00848 if (s != nullptr) s->setWarmUpPeriod(warmUpPeriod, warmUpPeriodTimeUnit);
00849 }
00850
00851 double simGetWarmUpPeriod() const {
00852 ModelSimulation* s = modelSimulation_();
00853 return s != nullptr ? s->getWarmUpPeriod() : 0.0;
00854 }
00855
00856 void simSetWarmUpPeriodTimeUnit(Util::TimeUnit warmUpPeriodTimeUnit) {
00857 ModelSimulation* s = modelSimulation_();
00858 if (s != nullptr) s->setWarmUpPeriodTimeUnit(warmUpPeriodTimeUnit);
00859 }
00860
00861 Util::TimeUnit simGetWarmUpPeriodTimeUnit() const {
00862 ModelSimulation* s = modelSimulation_();
00863 return s != nullptr ? s->getWarmUpPeriodTimeUnit() : Util::TimeUnit::unknown;
00864 }
00865
00866 void simSetTerminatingCondition(std::string terminatingCondition) {
00867 ModelSimulation* s = modelSimulation_();
00868 if (s != nullptr) s->setTerminatingCondition(terminatingCondition);
00869 }
00870
00871 std::string simGetTerminatingCondition() const {
00872 ModelSimulation* s = modelSimulation_();
00873 return s != nullptr ? s->getTerminatingCondition() : "";
00874 }
00875
00876 void simSetPauseOnEvent(bool pauseOnEvent) {
00877 ModelSimulation* s = modelSimulation_();
00878 if (s != nullptr) s->setPauseOnEvent(pauseOnEvent);
00879 }
00880
00881 bool simIsPauseOnEvent() const {
00882 ModelSimulation* s = modelSimulation_();
00883 return s != nullptr ? s->isPauseOnEvent() : false;
00884 }
00885
00886 void simSetStepByStep(bool stepByStep) {
00887 ModelSimulation* s = modelSimulation_();
00888 if (s != nullptr) s->setStepByStep(stepByStep);
00889 }
00890
00891 bool simIsStepByStep() const {
00892 ModelSimulation* s = modelSimulation_();
00893 return s != nullptr ? s->isStepByStep() : false;
00894 }
00895
00896 void simSetInitializeStatistics(bool initializeStatistics) {
00897 ModelSimulation* s = modelSimulation_();

```

```

00898 if (s != nullptr) s->setInitializeStatistics(initializeStatistics);
00899 }
00900
00901 bool simIsInitializeStatistics() const {
00902 ModelSimulation* s = modelSimulation_();
00903 return s != nullptr ? s->isInitializeStatistics() : false;
00904 }
00905
00906 void simSetInitializeSystem(bool initializeSystem) {
00907 ModelSimulation* s = modelSimulation_();
00908 if (s != nullptr) s->setInitializeSystem(initializeSystem);
00909 }
00910
00911 bool simIsInitializeSystem() const {
00912 ModelSimulation* s = modelSimulation_();
00913 return s != nullptr ? s->isInitializeSystem() : false;
00914 }
00915
00916 void simSetPauseOnReplication(bool pauseBetweenReplications) {
00917 ModelSimulation* s = modelSimulation_();
00918 if (s != nullptr) s->setPauseOnReplication(pauseBetweenReplications);
00919 }
00920
00921 bool simIsPauseOnReplication() const {
00922 ModelSimulation* s = modelSimulation_();
00923 return s != nullptr ? s->isPauseOnReplication() : false;
00924 }
00925
00926 void simSetReporter(SimulationReporter_if* simulationReporter) {
00927 ModelSimulation* s = modelSimulation_();
00928 if (s != nullptr) s->setReporter(simulationReporter);
00929 }
00930
00931 SimulationReporter_if* simGetReporter() const {
00932 ModelSimulation* s = modelSimulation_();
00933 return s != nullptr ? s->getReporter() : nullptr;
00934 }
00935
00936 double simGetSimulatedTime() const {
00937 ModelSimulation* s = modelSimulation_();
00938 return s != nullptr ? s->getSimulatedTime() : 0.0;
00939 }
00940
00941 bool simIsRunning() const {
00942 ModelSimulation* s = modelSimulation_();
00943 return s != nullptr ? s->isRunning() : false;
00944 }
00945
00946 bool simIsPaused() const {
00947 ModelSimulation* s = modelSimulation_();
00948 return s != nullptr ? s->isPaused() : false;
00949 }
00950
00951 unsigned int simGetCurrentReplicationNumber() const {
00952 ModelSimulation* s = modelSimulation_();
00953 return s != nullptr ? s->getCurrentReplicationNumber() : 0u;
00954 }
00955
00956 void simSetShowReportsAfterReplication(bool showReportsAfterReplication) {
00957 ModelSimulation* s = modelSimulation_();
00958 if (s != nullptr) s->setShowReportsAfterReplication(showReportsAfterReplication);
00959 }
00960
00961 bool simIsShowReportsAfterReplication() const {
00962 ModelSimulation* s = modelSimulation_();
00963 return s != nullptr ? s->isShowReportsAfterReplication() : false;
00964 }
00965
00966 void simSetShowReportsAfterSimulation(bool showReportsAfterSimulation) {
00967 ModelSimulation* s = modelSimulation_();
00968 if (s != nullptr) s->setShowReportsAfterSimulation(showReportsAfterSimulation);
00969 }
00970
00971 bool simIsShowReportsAfterSimulation() const {
00972 ModelSimulation* s = modelSimulation_();
00973 return s != nullptr ? s->isShowReportsAfterSimulation() : false;
00974 }
00975
00976 List<double>* simGetBreakpointsOnTime() const {
00977 ModelSimulation* s = modelSimulation_();
00978 return s != nullptr ? s->getBreakpointsOnTime() : nullptr;
00979 }
00980
00981 List<Entity>* simGetBreakpointsOnEntity() const {
00982 ModelSimulation* s = modelSimulation_();
00983 return s != nullptr ? s->getBreakpointsOnEntity() : nullptr;
00984 }

```

```

00985
00986 List<ModelComponent*>* simGetBreakpointsOnComponent() const {
00987 ModelSimulation* s = modelSimulation_();
00988 return s != nullptr ? s->getBreakpointsOnComponent() : nullptr;
00989 }
00990
00991 const List<ModelDataDefinition*>* simGetSimulationStatisticsAggregates() const {
00992 ModelSimulation* s = modelSimulation_();
00993 return s != nullptr ? s->getSimulationStatisticsAggregates() : nullptr;
00994 }
00995
00996 void simLoadInstance(PersistenceRecord* fields) {
00997 ModelSimulation* s = modelSimulation_();
00998 if (s != nullptr) s->loadInstance(fields);
00999 }
01000
01001 void simSaveInstance(PersistenceRecord* fields, bool saveDefaults) {
01002 ModelSimulation* s = modelSimulation_();
01003 if (s != nullptr) s->saveInstance(fields, saveDefaults);
01004 }
01005
01006 Event* simGetCurrentEvent() const {
01007 ModelSimulation* s = modelSimulation_();
01008 return s != nullptr ? s->getCurrentEvent() : nullptr;
01009 }
01010
01011 void simSetShowSimulationResponsesInReport(bool showSimulationResponsesInReport) {
01012 ModelSimulation* s = modelSimulation_();
01013 if (s != nullptr) s->setShowSimulationResponsesInReport(showSimulationResponsesInReport);
01014 }
01015
01016 bool simIsShowSimulationResponsesInReport() const {
01017 ModelSimulation* s = modelSimulation_();
01018 return s != nullptr ? s->isShowSimulationResponsesInReport() : false;
01019 }
01020
01021 void simSetShowSimulationControlsInReport(bool showSimulationControlsInReport) {
01022 ModelSimulation* s = modelSimulation_();
01023 if (s != nullptr) s->setShowSimulationControlsInReport(showSimulationControlsInReport);
01024 }
01025
01026 bool simIsShowSimulationControlsInReport() const {
01027 ModelSimulation* s = modelSimulation_();
01028 return s != nullptr ? s->isShowSimulationControlsInReport() : false;
01029 }
01030
01031 // -----
01032 // ModelDataManager -- data definitions in current model
01033 // -----
01034
01035 bool dataInsert(ModelDataDefinition* anElement) {
01036 ModelDataManager* m = modelDataManager_();
01037 return m != nullptr ? m->insert(anElement) : false;
01038 }
01039
01040 void dataRemove(ModelDataDefinition* anElement) {
01041 ModelDataManager* m = modelDataManager_();
01042 if (m != nullptr) m->remove(anElement);
01043 }
01044
01045 bool dataInsert(std::string datadefinitionTypename, ModelDataDefinition* anElement) {
01046 ModelDataManager* m = modelDataManager_();
01047 return m != nullptr ? m->insert(datadefinitionTypename, anElement) : false;
01048 }
01049
01050 void dataRemove(std::string datadefinitionTypename, ModelDataDefinition* anElement) {
01051 ModelDataManager* m = modelDataManager_();
01052 if (m != nullptr) m->remove(datadefinitionTypename, anElement);
01053 }
01054
01055 bool dataCheck(std::string datadefinitionTypename, ModelDataDefinition* anElement, std::string
expressionName, std::string& errorMessage) {
01056 ModelDataManager* m = modelDataManager_();
01057 return m != nullptr ? m->check(datadefinitionTypename, anElement, expressionName,
errorMessage) : false;
01058 }
01059
01060 bool dataCheck(std::string datadefinitionTypename, std::string elementName, std::string
expressionName, bool mandatory, std::string& errorMessage) {
01061 ModelDataManager* m = modelDataManager_();
01062 return m != nullptr ? m->check(datadefinitionTypename, elementName, expressionName, mandatory,
errorMessage) : false;
01063 }
01064
01065 void dataClear() {
01066 ModelDataManager* m = modelDataManager_();
01067 if (m != nullptr) m->clear();

```

```

01068 }
01069
01070 ModelDataDefinition* dataGetDataDefinition(std::string datadefinitionTypename,
Util::identification id) {
01071 ModelDataManager* m = modelDataManager_();
01072 return m != nullptr ? m->getDataDefinition(datadefinitionTypename, id) : nullptr;
01073 }
01074
01075 ModelDataDefinition* dataGetDataDefinition(std::string datadefinitionTypename, std::string name) {
01076 ModelDataManager* m = modelDataManager_();
01077 return m != nullptr ? m->getDataDefinition(datadefinitionTypename, name) : nullptr;
01078 }
01079
01080 unsigned int dataGetNumberOfDataDefinitions(std::string datadefinitionTypename) {
01081 ModelDataManager* m = modelDataManager_();
01082 return m != nullptr ? m->getNumberOfDataDefinitions(datadefinitionTypename) : 0u;
01083 }
01084
01085 unsigned int dataGetNumberOfDataDefinitions() {
01086 ModelDataManager* m = modelDataManager_();
01087 return m != nullptr ? m->getNumberOfDataDefinitions() : 0u;
01088 }
01089
01090 int dataGetRankOf(std::string datadefinitionTypename, std::string name) {
01091 ModelDataManager* m = modelDataManager_();
01092 return m != nullptr ? m->getRankOf(datadefinitionTypename, name) : -1;
01093 }
01094
01095 std::list<std::string> dataGetDataDefinitionClassnames() const {
01096 ModelDataManager* m = modelDataManager_();
01097 return m != nullptr ? m->getDataDefinitionClassnames() : std::list<std::string>{};
01098 }
01099
01100 List<ModelDataDefinition*> dataGetDataDefinitionList(std::string datadefinitionTypename) const {
01101 ModelDataManager* m = modelDataManager_();
01102 return m != nullptr ? m->getDataDefinitionList(datadefinitionTypename) : nullptr;
01103 }
01104
01105 void dataShow() {
01106 ModelDataManager* m = modelDataManager_();
01107 if (m != nullptr) m->show();
01108 }
01109
01110 bool dataHasChanged() const {
01111 ModelDataManager* m = modelDataManager_();
01112 return m != nullptr ? m->hasChanged() : false;
01113 }
01114
01115 void dataSetHasChanged(bool hasChanged) {
01116 ModelDataManager* m = modelDataManager_();
01117 if (m != nullptr) m->setHasChanged(hasChanged);
01118 }
01119
01120 // -----
01121 // ModelComponentManager -- components in current model
01122 // -----
01123
01124 bool componentInsert(ModelComponent* comp) {
01125 ModelComponentManager* m = modelComponentManager_();
01126 return m != nullptr ? m->insert(comp) : false;
01127 }
01128
01129 void componentRemove(ModelComponent* comp) {
01130 ModelComponentManager* m = modelComponentManager_();
01131 if (m != nullptr) m->remove(comp);
01132 }
01133
01134 ModelComponent* componentFind(std::string name) {
01135 ModelComponentManager* m = modelComponentManager_();
01136 return m != nullptr ? m->find(name) : nullptr;
01137 }
01138
01139 ModelComponent* componentFind(Util::identification id) {
01140 ModelComponentManager* m = modelComponentManager_();
01141 return m != nullptr ? m->find(id) : nullptr;
01142 }
01143
01144 void componentClear() {
01145 ModelComponentManager* m = modelComponentManager_();
01146 if (m != nullptr) m->clear();
01147 }
01148
01149 unsigned int componentGetNumberOfComponents() {
01150 ModelComponentManager* m = modelComponentManager_();
01151 return m != nullptr ? m->getNumberOfComponents() : 0u;
01152 }
01153

```



```

01154 std::list<ModelComponent*>::iterator componentBegin() {
01155 ModelComponentManager* m = modelComponentManager_();
01156 static std::list<ModelComponent*> _empty;
01157 return m != nullptr ? m->begin() : _empty.begin();
01158 }
01159
01160 std::list<ModelComponent*>::iterator componentEnd() {
01161 ModelComponentManager* m = modelComponentManager_();
01162 static std::list<ModelComponent*> _empty;
01163 return m != nullptr ? m->end() : _empty.end();
01164 }
01165
01166 ModelComponent* componentFront() {
01167 ModelComponentManager* m = modelComponentManager_();
01168 return m != nullptr ? m->front() : nullptr;
01169 }
01170
01171 ModelComponent* componentNext() {
01172 ModelComponentManager* m = modelComponentManager_();
01173 return m != nullptr ? m->next() : nullptr;
01174 }
01175
01176 bool componentHasChanged() const {
01177 ModelComponentManager* m = modelComponentManager_();
01178 return m != nullptr ? m->hasChanged() : false;
01179 }
01180
01181 void componentSetHasChanged(bool hasChanged) {
01182 ModelComponentManager* m = modelComponentManager_();
01183 if (m != nullptr) m->setHasChanged(hasChanged);
01184 }
01185
01186 std::list<SourceModelComponent*>* componentGetSourceComponents() {
01187 ModelComponentManager* m = modelComponentManager_();
01188 return m != nullptr ? m->getSourceComponents() : nullptr;
01189 }
01190
01191 std::list<ModelComponent*>* componentGetTransferInComponents() {
01192 ModelComponentManager* m = modelComponentManager_();
01193 return m != nullptr ? m->getTransferInComponents() : nullptr;
01194 }
01195
01196 std::list<ModelComponent*>* componentGetAllComponents() const {
01197 ModelComponentManager* m = modelComponentManager_();
01198 return m != nullptr ? m->getAllComponents() : nullptr;
01199 }
01200
01201 // -----
01202 // OnEventManager -- event handlers on current model
01203 // -----
01204
01205 void eventAddOnModelCheckSucessHandler(modelEventHandler EventHandler) {
01206 OnEventManager* m = onEventManager_();
01207 if (m != nullptr) m->addOnModelCheckSucessHandler(EventHandler);
01208 }
01209
01210 void eventAddOnModelLoadHandler(modelEventHandler EventHandler) {
01211 OnEventManager* m = onEventManager_();
01212 if (m != nullptr) m->addOnModelLoadHandler(EventHandler);
01213 }
01214
01215 void eventAddOnModelSaveHandler(modelEventHandler EventHandler) {
01216 OnEventManager* m = onEventManager_();
01217 if (m != nullptr) m->addOnModelSaveHandler(EventHandler);
01218 }
01219
01220 void eventAddOnReplicationStartHandler(simulationEventHandler EventHandler) {
01221 OnEventManager* m = onEventManager_();
01222 if (m != nullptr) m->addOnReplicationStartHandler(EventHandler);
01223 }
01224
01225 void eventAddOnReplicationStepHandler(simulationEventHandler EventHandler) {
01226 OnEventManager* m = onEventManager_();
01227 if (m != nullptr) m->addOnReplicationStepHandler(EventHandler);
01228 }
01229
01230 void eventAddOnReplicationEndHandler(simulationEventHandler EventHandler) {
01231 OnEventManager* m = onEventManager_();
01232 if (m != nullptr) m->addOnReplicationEndHandler(EventHandler);
01233 }
01234
01235 void eventAddOnProcessEventHandler(simulationEventHandler EventHandler) {
01236 OnEventManager* m = onEventManager_();
01237 if (m != nullptr) m->addOnProcessEventHandler(EventHandler);
01238 }
01239
01240 void eventAddOnAfterProcessEventHandler(simulationEventHandler EventHandler) {

```

```

01241 OnEventManager* m = onEventManager_();
01242 if (m != nullptr) m->addOnAfterProcessEventHandler(EventHandler);
01243 }
01244
01245 void eventAddOnEntityCreateHandler(simulationEventHandler EventHandler) {
01246 OnEventManager* m = onEventManager_();
01247 if (m != nullptr) m->addOnEntityCreateHandler(EventHandler);
01248 }
01249
01250 void eventAddOnEntityMoveHandler(simulationEventHandler EventHandler) {
01251 OnEventManager* m = onEventManager_();
01252 if (m != nullptr) m->addOnEntityMoveHandler(EventHandler);
01253 }
01254
01255 void eventAddOnEntityRemoveHandler(simulationEventHandler EventHandler) {
01256 OnEventManager* m = onEventManager_();
01257 if (m != nullptr) m->addOnEntityRemoveHandler(EventHandler);
01258 }
01259
01260 void eventAddOnSimulationStartHandler(simulationEventHandler EventHandler) {
01261 OnEventManager* m = onEventManager_();
01262 if (m != nullptr) m->addOnSimulationStartHandler(EventHandler);
01263 }
01264
01265 void eventAddOnSimulationPausedHandler(simulationEventHandler EventHandler) {
01266 OnEventManager* m = onEventManager_();
01267 if (m != nullptr) m->addOnSimulationPausedHandler(EventHandler);
01268 }
01269
01270 void eventAddOnSimulationResumeHandler(simulationEventHandler EventHandler) {
01271 OnEventManager* m = onEventManager_();
01272 if (m != nullptr) m->addOnSimulationResumeHandler(EventHandler);
01273 }
01274
01275 void eventAddOnSimulationEndHandler(simulationEventHandler EventHandler) {
01276 OnEventManager* m = onEventManager_();
01277 if (m != nullptr) m->addOnSimulationEndHandler(EventHandler);
01278 }
01279
01280 void eventAddOnBreakpointHandler(simulationEventHandler EventHandler) {
01281 OnEventManager* m = onEventManager_();
01282 if (m != nullptr) m->addOnBreakpointHandler(EventHandler);
01283 }
01284
01285 template<typename Class>
01286 void eventAddOnModelCheckSuccessHandler(Class* object, void (Class::*function)(ModelEvent*)) {
01287 OnEventManager* m = onEventManager_();
01288 if (m != nullptr) m->addOnModelCheckSuccessHandler(object, function);
01289 }
01290
01291 template<typename Class>
01292 void eventAddOnModelLoadHandler(Class* object, void (Class::*function)(ModelEvent*)) {
01293 OnEventManager* m = onEventManager_();
01294 if (m != nullptr) m->addOnModelLoadHandler(object, function);
01295 }
01296
01297 template<typename Class>
01298 void eventAddOnModelSaveHandler(Class* object, void (Class::*function)(ModelEvent*)) {
01299 OnEventManager* m = onEventManager_();
01300 if (m != nullptr) m->addOnModelSaveHandler(object, function);
01301 }
01302
01303 template<typename Class>
01304 void eventAddOnReplicationStartHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01305 OnEventManager* m = onEventManager_();
01306 if (m != nullptr) m->addOnReplicationStartHandler(object, function);
01307 }
01308
01309 template<typename Class>
01310 void eventAddOnReplicationStepHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01311 OnEventManager* m = onEventManager_();
01312 if (m != nullptr) m->addOnReplicationStepHandler(object, function);
01313 }
01314
01315 template<typename Class>
01316 void eventAddOnReplicationEndHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01317 OnEventManager* m = onEventManager_();
01318 if (m != nullptr) m->addOnReplicationEndHandler(object, function);
01319 }
01320
01321 template<typename Class>
01322 void eventAddOnProcessEventHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01323 OnEventManager* m = onEventManager_();
01324 if (m != nullptr) m->addOnProcessEventHandler(object, function);
01325 }
01326
01327 template<typename Class>

```

```

01328 void eventAddOnAfterProcessEventHandler(Class* object, void (Class::*function)(SimulationEvent*))
01329 {
01330 OnEventManager* m = onEventManager_();
01331 if (m != nullptr) m->addOnAfterProcessEventHandler(object, function);
01332 }
01333
01334 template<typename Class>
01335 void eventAddOnEntityCreateHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01336 OnEventManager* m = onEventManager_();
01337 if (m != nullptr) m->addOnEntityCreateHandler(object, function);
01338 }
01339
01340 template<typename Class>
01341 void eventAddOnEntityMoveHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01342 OnEventManager* m = onEventManager_();
01343 if (m != nullptr) m->addOnEntityMoveHandler(object, function);
01344 }
01345
01346 template<typename Class>
01347 void eventAddOnEntityRemoveHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01348 OnEventManager* m = onEventManager_();
01349 if (m != nullptr) m->addOnEntityRemoveHandler(object, function);
01350 }
01351
01352 template<typename Class>
01353 void eventAddOnSimulationStartHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01354 OnEventManager* m = onEventManager_();
01355 if (m != nullptr) m->addOnSimulationStartHandler(object, function);
01356 }
01357
01358 template<typename Class>
01359 void eventAddOnSimulationPausedHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01360 OnEventManager* m = onEventManager_();
01361 if (m != nullptr) m->addOnSimulationPausedHandler(object, function);
01362 }
01363
01364 template<typename Class>
01365 void eventAddOnSimulationResumeHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01366 OnEventManager* m = onEventManager_();
01367 if (m != nullptr) m->addOnSimulationResumeHandler(object, function);
01368 }
01369
01370 template<typename Class>
01371 void eventAddOnSimulationEndHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01372 OnEventManager* m = onEventManager_();
01373 if (m != nullptr) m->addOnSimulationEndHandler(object, function);
01374 }
01375
01376 template<typename Class>
01377 void eventAddOnBreakpointHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01378 OnEventManager* m = onEventManager_();
01379 if (m != nullptr) m->addOnBreakpointHandler(object, function);
01380 }
01381
01382 void eventNotifyModelCheckSuccessHandlers(ModelEvent* se) {
01383 OnEventManager* m = onEventManager_();
01384 if (m != nullptr) m->NotifyModelCheckSuccessHandlers(se);
01385 }
01386
01387 void eventNotifyModelLoadHandlers(ModelEvent* se) {
01388 OnEventManager* m = onEventManager_();
01389 if (m != nullptr) m->NotifyModelLoadHandlers(se);
01390 }
01391
01392 void eventNotifyModelSaveHandlers(ModelEvent* se) {
01393 OnEventManager* m = onEventManager_();
01394 if (m != nullptr) m->NotifyModelSaveHandlers(se);
01395 }
01396
01397 void eventNotifyReplicationStartHandlers(SimulationEvent* se) {
01398 OnEventManager* m = onEventManager_();
01399 if (m != nullptr) m->NotifyReplicationStartHandlers(se);
01400 }
01401
01402 void eventNotifyReplicationStepHandlers(SimulationEvent* se) {
01403 OnEventManager* m = onEventManager_();
01404 if (m != nullptr) m->NotifyReplicationStepHandlers(se);
01405 }
01406
01407 void eventNotifyReplicationEndHandlers(SimulationEvent* se) {
01408 OnEventManager* m = onEventManager_();
01409 if (m != nullptr) m->NotifyReplicationEndHandlers(se);
01410 }
01411
01412 void eventNotifyProcessEventHandlers(SimulationEvent* se) {
01413 OnEventManager* m = onEventManager_();
01414 if (m != nullptr) m->NotifyProcessEventHandlers(se);
01415 }

```

```

01414 }
01415
01416 void eventNotifyAfterProcessEventHandlers(SimulationEvent* se) {
01417 OnEventManager* m = onEventManager_();
01418 if (m != nullptr) m->NotifyAfterProcessEventHandlers(se);
01419 }
01420
01421 void eventNotifyEntityCreateHandlers(SimulationEvent* se) {
01422 OnEventManager* m = onEventManager_();
01423 if (m != nullptr) m->NotifyEntityCreateHandlers(se);
01424 }
01425
01426 void eventNotifyEntityMoveHandlers(SimulationEvent* se) {
01427 OnEventManager* m = onEventManager_();
01428 if (m != nullptr) m->NotifyEntityMoveHandlers(se);
01429 }
01430
01431 void eventNotifyEntityRemoveHandlers(SimulationEvent* se) {
01432 OnEventManager* m = onEventManager_();
01433 if (m != nullptr) m->NotifyEntityRemoveHandlers(se);
01434 }
01435
01436 void eventNotifySimulationStartHandlers(SimulationEvent* se) {
01437 OnEventManager* m = onEventManager_();
01438 if (m != nullptr) m->NotifySimulationStartHandlers(se);
01439 }
01440
01441 void eventNotifySimulationPausedHandlers(SimulationEvent* se) {
01442 OnEventManager* m = onEventManager_();
01443 if (m != nullptr) m->NotifySimulationPausedHandlers(se);
01444 }
01445
01446 void eventNotifySimulationResumeHandlers(SimulationEvent* se) {
01447 OnEventManager* m = onEventManager_();
01448 if (m != nullptr) m->NotifySimulationResumeHandlers(se);
01449 }
01450
01451 void eventNotifySimulationEndHandlers(SimulationEvent* se) {
01452 OnEventManager* m = onEventManager_();
01453 if (m != nullptr) m->NotifySimulationEndHandlers(se);
01454 }
01455
01456 void eventNotifyBreakpointHandlers(SimulationEvent* se) {
01457 OnEventManager* m = onEventManager_();
01458 if (m != nullptr) m->NotifyBreakpointHandlers(se);
01459 }
01460
01461 private:
01462 Simulator* _simulator;
01463
01464 LicenceManager* licenceManager_() const {
01465 return _simulator != nullptr ? _simulator->getLicenceManager() : nullptr;
01466 }
01467
01468 PluginManager* pluginManager_() const {
01469 return _simulator != nullptr ? _simulator->getPluginManager() : nullptr;
01470 }
01471
01472 ModelManager* modelManager_() const {
01473 return _simulator != nullptr ? _simulator->getModelManager() : nullptr;
01474 }
01475
01476 TraceManager* traceManager_() const {
01477 return _simulator != nullptr ? _simulator->getTraceManager() : nullptr;
01478 }
01479
01480 ParserManager* parserManager_() const {
01481 return _simulator != nullptr ? _simulator->getParserManager() : nullptr;
01482 }
01483
01484 ExperimentManager* experimentManager_() const {
01485 return _simulator != nullptr ? _simulator->getExperimentManager() : nullptr;
01486 }
01487
01488 Model* currentModel_() const {
01489 ModelManager* m = modelManager_();
01490 return m != nullptr ? m->current() : nullptr;
01491 }
01492
01493 ModelInfo* modelInfo_() const {
01494 Model* model = currentModel_();
01495 return model != nullptr ? model->getInfos() : nullptr;
01496 }
01497
01498 ModelSimulation* modelSimulation_() const {
01499 Model* model = currentModel_();
01500 return model != nullptr ? model->getSimulation() : nullptr;

```

```

01501 }
01502
01503 ModelDataManager* modelDataManager_() const {
01504 Model* model = currentModel_();
01505 return model != nullptr ? model->getDataManager() : nullptr;
01506 }
01507
01508 ModelComponentManager* modelComponentManager_() const {
01509 Model* model = currentModel_();
01510 return model != nullptr ? model->getComponentManager() : nullptr;
01511 }
01512
01513 OnEventManager* onEventManager_() const {
01514 Model* model = currentModel_();
01515 return model != nullptr ? model->getOnEventManager() : nullptr;
01516 }
01517 };
01518
01519 #endif /* SIMULATORFACADE_H */

```

## 16.317 SinkModelComponent.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: SinkModelComponent.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 14 de Agosto de 2018, 14:29
00012 */
00013
00014 #ifndef SINKMODELCOMPONENT_H
00015 #define SINKMODELCOMPONENT_H
00016
00017 #include "model/ModelComponent.h"
00018
00019 //namespace GenesysKernel {
00020
00021 class SinkModelComponent : public ModelComponent {
00022 public:
00023 SinkModelComponent(Model* model, std::string componentTypename, std::string name = "");
00024 virtual ~SinkModelComponent() = default;
00025 public:
00026 protected:
00027 virtual bool _loadInstance(PersistenceRecord *fields) override;
00028 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00029 protected:
00030 //virtual void _initBetweenReplications();
00031 virtual bool _check(std::string& errorMessage) override;
00032 private:
00033 };
00034 //namespace\\}
00035 #endif /* SINKMODELCOMPONENT_H */
00036
00037
00038
00039
00040

```

## 16.318 SourceModelComponent.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: SourceModelComponent.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 21 de Junho de 2018, 19:50
00012 */
00013
00014 #ifndef SOURCEMODELCOMPONENT_H
00015 #define SOURCEMODELCOMPONENT_H
00016
00017 #include "model/ModelComponent.h"
00018 #include <string>
00019 #include <limits>
00020 //namespace GenesysKernel {
00021

```

```

00027 class SourceModelComponent : public ModelComponent {
00028 public:
00029 SourceModelComponent(Model* model, std::string componentTypename, std::string name = "");
00030 virtual ~SourceModelComponent() = default;
00031 public: // get & set
00036 void setFirstCreation(double _firstCreation);
00041 double getFirstCreation() const;
00046 void setEntityType(EntityType* _entityType);
00051 void setEntityTypeNames(std::string entityTypeNames);
00056 EntityType* getEntityType() const;
00061 void setTimeUnit(Util::TimeUnit _timeUnit);
00066 Util::TimeUnit getTimeUnit() const;
00072 void setTimeBetweenCreationsExpression(std::string _timeBetweenCreations, Util::TimeUnit
 _timeUnit=Util::TimeUnit::unknown);
00077 std::string getTimeBetweenCreationsExpression() const;
00082 void setMaxCreations(unsigned long _maxCreations);
00087 void setMaxCreations(std::string _maxCreationsExpression);
00092 std::string getMaxCreations() const;
00097 unsigned int getEntitiesCreated() const;
00102 void setEntitiesCreated(unsigned int _entitiesCreated);
00107 void setEntitiesPerCreation(unsigned int _entitiesPerCreation);
00112 unsigned int getEntitiesPerCreation() const;
00113 public:
00114 virtual std::string show() override;
00115
00116 protected:
00117 virtual bool _loadInstance(PersistenceRecord *fields) override;
00118 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00119 protected:
00120 virtual void _initBetweenReplications() override;
00121 virtual bool _check(std::string& errorMessage) override;
00122 protected:
00123 //virtual void _createInternalStatisticReporters();
00124 virtual void _createEditableDataDefinitions();
00125 virtual void _createAttachedAttributes();
00126 protected: // get & set
00127 EntityType* _entityType = nullptr;
00128
00129 const struct DEFAULT_VALUES {
00130 const double firstCreation = 0.0;
00131 const unsigned int entitiesPerCreation = 1;
00132 const std::string maxCreationsExpression = std::to_string(std::numeric_limits<unsigned
int>::max());
00133 const std::string timeBetweenCreationsExpression = "EXPO(1.0)";
00134 const Util::TimeUnit timeBetweenCreationsTimeUnit = Util::TimeUnit::second;
00135 //
00136 const std::string entityTypeNames = "entitytype";
00137 } DEFAULT;
00138 double _firstCreation = DEFAULT.firstCreation;
00139 unsigned int _entitiesPerCreation = DEFAULT.entitiesPerCreation;
00140 std::string _maxCreationsExpression = DEFAULT.maxCreationsExpression;
00141 std::string _timeBetweenCreationsExpression = DEFAULT.timeBetweenCreationsExpression;
00142 Util::TimeUnit _timeBetweenCreationsTimeUnit = DEFAULT.timeBetweenCreationsTimeUnit;
00143 unsigned int _entitiesCreatedSoFar = 0;
00144 };
00145 //namespace\\
00146 #endif /* SOURCEMODELCOMPONENT_H */

```

## 16.319 SystemDependencyResolver.h

```

00001 /*
00002 * File: SystemDependencyResolver.h
00003 * Author: rafael.luiiz.cancian
00004 *
00005 * Created on 16 de Abril de 2026
00006 */
00007
00008 #ifndef SYSTEMDEPENDENCYRESOLVER_H
00009 #define SYSTEMDEPENDENCYRESOLVER_H
00010
00011 #include "PluginInformation.h"
00012
00013 #include <list>
00014 #include <string>
00015
00019 struct SystemCommandResult {
00021 bool started = false;
00023 int exitCode = -1;
00025 std::string output = "";
00027 std::string errorMessage = "";
00028
00030 bool succeeded() const;
00031 };
00032
00036 class SystemCommandExecutor_if {

```

```

00037 public:
00039 virtual ~SystemCommandExecutor_if() = default;
00041 virtual SystemCommandResult run(const std::string& command) = 0;
00042 };
00043
00047 class SystemShellCommandExecutor : public SystemCommandExecutor_if {
00048 public:
00050 SystemCommandResult run(const std::string& command) override;
00051 };
00052
00056 class SystemDependencyCheckEntry {
00057 public:
00059 enum class Status {
00060 Satisfied,
00061 Missing,
00062 NotVerifiable,
00063 IgnoredDifferentOS
00064 };
00065
00067 SystemDependencyCheckEntry(SystemDependency dependency, Status status, SystemCommandResult
checkResult = {}, std::string message = "");
00068
00070 const SystemDependency& dependency() const;
00072 Status status() const;
00074 const SystemCommandResult& checkResult() const;
00076 const std::string& message() const;
00078 bool blocksInsertion() const;
00080 bool canAttemptInstall() const;
00082 std::string diagnosticText() const;
00084 static std::string statusToString(Status status);
00085
00086 private:
00087 SystemDependency _dependency;
00088 Status _status = Status::NotVerifiable;
00089 SystemCommandResult _checkResult;
00090 std::string _message = "";
00091 };
00092
00096 class SystemDependencyCheckResult {
00097 public:
00099 void add(SystemDependencyCheckEntry entry);
00101 const std::list<SystemDependencyCheckEntry>& entries() const;
00103 bool canInsertPlugin() const;
00105 bool hasBlockingEntries() const;
00107 bool canAttemptInstallForAllMissing() const;
00109 std::string summary() const;
00111 std::string diagnosticText(bool includeSatisfied = false) const;
00112
00113 private:
00114 std::list<SystemDependencyCheckEntry> _entries;
00115 };
00116
00120 class SystemDependencyInstallEntry {
00121 public:
00123 SystemDependencyInstallEntry(SystemDependency dependency, SystemCommandResult installResult);
00125 const SystemDependency& dependency() const;
00127 const SystemCommandResult& installResult() const;
00129 std::string diagnosticText() const;
00130
00131 private:
00132 SystemDependency _dependency;
00133 SystemCommandResult _installResult;
00134 };
00135
00139 class SystemDependencyInstallResult {
00140 public:
00142 void add(SystemDependencyInstallEntry entry);
00144 const std::list<SystemDependencyInstallEntry>& entries() const;
00146 bool succeeded() const;
00148 std::string summary() const;
00150 std::string diagnosticText() const;
00151
00152 private:
00153 std::list<SystemDependencyInstallEntry> _entries;
00154 };
00155
00163 class SystemDependencyResolver {
00164 public:
00166 static SystemDependency::OS currentOS();
00168 static bool appliesToCurrentOS(const SystemDependency& dependency);
00170 static SystemDependencyCheckResult evaluate(const std::list<SystemDependency>* dependencies,
SystemCommandExecutor_if& executor);
00173 static SystemDependencyInstallResult installMissingDependencies(const SystemDependencyCheckResult&
checkResult,
SystemCommandExecutor_if&
executor);
00175 };

```

```

00176
00177 #endif /* SYSTEMDEPENDENCYRESOLVER_H */

```

## 16.320 TraceManager.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: TraceManager.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 7 de Novembro de 2018, 11:59
00012 */
00013
00014 #ifndef TRACEMANAGER_H
00015 #define TRACEMANAGER_H
00016
00017 #include "../util/List.h"
00018 #include "../util/Util.h"
00019 #include <functional>
00020 //namespace GenesysKernel {
00021 class Model;
00022 class Entity;
00023 class ModelComponent;
00024 class Simulator;
00025
00026 class TraceEvent;
00027 class TraceErrorEvent;
00028 class TraceSimulationEvent;
00029 class TraceSimulationProcess;
00030
00031 // for handlers that are simple functions
00032 typedef void (*traceListener)(TraceEvent);
00033 typedef void (*traceErrorListener)(TraceErrorEvent);
00034 typedef void (*traceSimulationListener)(TraceSimulationEvent);
00035 typedef void (*traceSimulationProcessListener)(TraceSimulationProcess);
00036 // for handlers that are class members (methods)
00037 typedef std::function<void(TraceEvent)> traceListenerMethod;
00038 typedef std::function<void(TraceErrorEvent)> traceErrorListenerMethod;
00039 typedef std::function<void(TraceSimulationEvent)> traceSimulationListenerMethod;
00040 typedef std::function<void(TraceSimulationProcess)> traceSimulationProcessListenerMethod;
00041
00042 class TraceManager {
00043 public:
00044
00045 enum class Level : int {
00046 L0_noTraces = 0,
00047 L1_errorFatal = 1,
00048 L2_results = 2,
00049 L3_errorRecover = 3, // or progress
00050 L4_warning = 4,
00051 L5_event = 5,
00052 L6_arrival = 6,
00053 L7_internal = 7,
00054 L8_detailed = 8,
00055 L9_mostDetailed = 9,
00056 num_elements = 10
00057 };
00058
00059 public:
00060 static std::string convertEnumToStr(Level level);
00061 TraceManager(Simulator* simulator); // (Model* model);
00062 virtual ~TraceManager();
00063
00064 public: // add trace handlers
00065 // for handlers that are simple functions
00066 void addTraceHandler(traceListener traceListener);
00067 void addTraceReportHandler(traceListener traceReportListener);
00068 void addTraceSimulationHandler(traceSimulationListener traceSimulationListener);
00069 void addTraceErrorHandler(traceErrorListener traceErrorListener);
00070 // for handlers that are class members (methods)
00071 template<typename Class> void addTraceHandler(Class* object, void (Class::*function)(TraceEvent));
00072 template<typename Class> void addTraceErrorHandler(Class* object, void (Class::*function)(TraceErrorEvent));
00073 template<typename Class> void addTraceReportHandler(Class* object, void (Class::*function)(TraceEvent));
00074 template<typename Class> void addTraceSimulationHandler(Class* object, void (Class::*function)(TraceSimulationEvent));
00075
00076 public: // add ModelDataDefinition (or subclasses) allowed (or restrcted) to traceSimulation
00077 void addTraceSimulationExceptionRuleModelData(void* thisobject);
00078 public: // traces (invoke trace handlers)
00079 void trace(TraceManager::Level level, const std::string& text);

```



```

00114 void traceSimulation(void* thisobject, TraceManager::Level level, double time, Entity* entity,
ModelComponent* component, const std::string& text);
00115 void traceSimulation(void* thisobject, TraceManager::Level level, const std::string& text);
00116 public: // traces
00117 // Disable callbacks and clear handler lists before simulator teardown destroys GUI objects.
00118 void beginShutdown();
00119 void trace(const std::string& text, TraceManager::Level level = TraceManager::Level::L8_detailed);
00120 void traceError(const std::string& text, const std::exception& e);
00121 void traceError(const std::string& text, TraceManager::Level level =
TraceManager::Level::L1_errorFatal);
00122 void traceReport(const std::string& text, TraceManager::Level level =
TraceManager::Level::L2_results);
00123 void traceSimulation(void* thisobject, double time, Entity* entity, ModelComponent* component,
const std::string& text, TraceManager::Level level = TraceManager::Level::L8_detailed, bool showAnyway
= false);
00124 void traceSimulation(void* thisobject, const std::string& text, TraceManager::Level level =
TraceManager::Level::L8_detailed, bool showAnyway = false);
00125
00126 public:
00131 List<std::string>* errorMessages() const;
00136 void setTraceLevel(TraceManager::Level _traceLevel);
00141 TraceManager::Level getTraceLevel() const;
00146 Simulator* getParentSimulator() const;
00151 void setTraceSimulationRuleAllAllowed(bool _traceSimulationRuleAllAllowed);
00156 bool isTraceSimulationRuleAllAllowed() const;
00157
00158 private:
00159 //void _addHandler(List<traceListener>* list,)
00160 bool _traceConditionPassed(TraceManager::Level level);
00161 bool _traceSimulationConditionPassed(TraceManager::Level level, void* thisobject, bool showAnyway
= false);
00162 private: // trace listener
00163 // for handlers that are simple functions
00164 List<traceListener>* _traceHandlers;
00165 List<traceErrorListener>* _traceErrorHandlers;
00166 List<traceListener>* _traceReportHandlers;
00167 List<traceSimulationListener>* _traceSimulationHandlers;
00168 // for handlers that are class members (methods)
00169 List<traceListenerMethod>* _traceHandlersMethod;
00170 List<traceErrorListenerMethod>* _traceErrorHandlersMethod;
00171 List<traceListenerMethod>* _traceReportHandlersMethod;
00172 List<traceSimulationListenerMethod>* _traceSimulationHandlersMethod;
00173 private:
00174 List<void*>* _traceSimulationExceptionRule;
00175 private:
00176 Simulator* _simulator;
00177 private:
00178 TraceManager::Level _traceLevel; // = TraceManager::Level::L9_mostDetailed;
00179 bool _shuttingDown;
00180 double _lastTimeTraceSimulation;
00181 Util::identification _lastEntityTraceSimulation;
00182 Util::identification _lastModuleTraceSimulation;
00183 bool _traceSimulationRuleAllAllowed;
00184 List<std::string>* _errorMessages;
00185
00186 };
00187
00188 // implementation for template methods
00189
00190 template<typename Class> void TraceManager::addTraceHandler(Class* object, void
(Class::*function)(TraceEvent)) {
00191 this->_traceHandlersMethod->insert(std::bind(function, object, std::placeholders::_1));
00192 }
00193
00194 template<typename Class> void TraceManager::addTraceErrorHandler(Class* object, void
(Class::*function)(TraceErrorEvent)) {
00195 this->_traceErrorHandlersMethod->insert(std::bind(function, object, std::placeholders::_1));
00196 }
00197
00198 template<typename Class> void TraceManager::addTraceReportHandler(Class* object, void
(Class::*function)(TraceEvent)) {
00199 this->_traceReportHandlersMethod->insert(std::bind(function, object, std::placeholders::_1));
00200 }
00201
00202 template<typename Class> void TraceManager::addTraceSimulationHandler(Class* object, void
(Class::*function)(TraceSimulationEvent)) {
00203 this->_traceSimulationHandlersMethod->insert(std::bind(function, object, std::placeholders::_1));
00204 }
00205
00206 class TraceEvent {
00207 public:
00208
00209 TraceEvent(TraceManager::Level level, const std::string& text) {
00210 _tracelevel = level;
00211 _text = text;
00212 }
00213

```

```

00214 TraceEvent(const std::string& text, TraceManager::Level level = TraceManager::Level::L8_detailed)
00215 {
00216 _tracelevel = level;
00217 _text = text;
00218 }
00219 TraceManager::Level getTracelevel() const {
00220 return _tracelevel;
00221 }
00222 const std::string& getText() const {
00223 return _text;
00224 }
00225 private:
00226 TraceManager::Level _tracelevel;
00227 std::string _text;
00228 };
00229
00230 class TraceErrorEvent : public TraceEvent {
00231 public:
00232 TraceErrorEvent(const std::string& text, const std::exception& e) : TraceEvent(text,
00233 TraceManager::Level::L3_errorRecover), _exceptionMessage(e.what()) {
00234 }
00235 TraceErrorEvent(const std::string& text, TraceManager::Level level) : TraceEvent(text, level),
00236 _exceptionMessage("") {
00237 }
00238 const std::string& getExceptionMessage() const {
00239 return _exceptionMessage;
00240 }
00241 private:
00242 std::string _exceptionMessage;
00243 };
00244
00245 class TraceSimulationEvent : public TraceEvent {
00246 public:
00247 TraceSimulationEvent(TraceManager::Level level, double time, Entity* entity, ModelComponent*
00248 component, const std::string& text) : TraceEvent(text, level) {
00249 _time = time;
00250 _entity = entity;
00251 _component = component;
00252 }
00253 ModelComponent* getComponent() const {
00254 return _component;
00255 }
00256 Entity* getEntity() const {
00257 return _entity;
00258 }
00259 double getTime() const {
00260 return _time;
00261 }
00262 private:
00263 double _time;
00264 Entity* _entity;
00265 ModelComponent* _component;
00266 };
00267
00268 class TraceSimulationProcess : public TraceEvent {
00269 public:
00270 TraceSimulationProcess(const std::string& text, TraceManager::Level level =
00271 TraceManager::Level::L8_detailed) : TraceEvent(text, level) {
00272 }
00273 };
00274
00275 //namespace\\
00276 #endif /* TRACEMANAGER_H */

```

## 16.321 Collector\_if.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*

```

```

00008 * File: Collector_if.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 14 de Agosto de 2018, 14:16
00012 */
00013
00014 #ifndef COLLECTOR_IF_H
00015 #define COLLECTOR_IF_H
00016
00017 #include <string>
00018 #include <functional>
00019
00022 typedef std::function<void(double, double) > CollectorAddValueHandler;
00023
00024 template<typename Class>
00025 CollectorAddValueHandler setCollectorAddValueHandler(void (Class::*function)(double, double), Class *
object) {
00026 return std::bind(function, object, std::placeholders::_1, std::placeholders::_2);
00027 }
00028
00029 typedef std::function<void() > CollectorClearHandler;
00030
00031 template<typename Class>
00032 CollectorClearHandler setCollectorClearHandler(void (Class::*function)(), Class * object) {
00033 return std::bind(function, object);
00034 }
00035
00042 class Collector_if {
00043 public:
00044 virtual ~Collector_if() = default;
00049 virtual void clear() = 0;
00056 virtual void addValue(double value, double weight=1) = 0;
00062 virtual double getLastValue() = 0;
00068 virtual unsigned long numElements() = 0;
00069 public:
00075 virtual void setAddValueHandler(CollectorAddValueHandler addValueHandler) = 0;
00081 virtual void setClearHandler(CollectorClearHandler clearHandler) = 0;
00082 };
00083
00084 #endif /* COLLECTOR_IF_H */

```

## 16.322 CollectorDatafile\_if.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: CollectorDatafile_if.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 30 de Agosto de 2018, 16:45
00012 */
00013
00014 #ifndef COLLECTORDATAFILE_IF_H
00015 #define COLLECTORDATAFILE_IF_H
00016
00017 #include "Collector_if.h"
00018
00025 class CollectorDatafile_if : public Collector_if {
00026 public:
00027 virtual ~CollectorDatafile_if() = default;
00033 virtual double getValue(unsigned int rank) = 0;
00037 virtual void seekFirstValue() = 0;
00042 virtual double getNextValue() = 0;
00048 virtual std::string getDataFilename() = 0;
00054 virtual void setDataFilename(std::string filename) = 0;
00055 };
00056
00057 #endif /* COLLECTORDATAFILE_IF_H */

```

## 16.323 CollectorDatafileDefaultImpl.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*

```

```

00008 * File: CollectorDatafileDefaultImpl1.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 1 de Agosto de 2018, 20:58
00012 */
00013
00014 #ifndef COLLECTORDATAFILEDEFAULTIMPL1_H
00015 #define COLLECTORDATAFILEDEFAULTIMPL1_H
00016
00017 #include <limits>
00018 #include <string>
00019
00020 #include "CollectorDatafile_if.h"
00021
00022 class CollectorDatafileDefaultImpl1 : public CollectorDatafile_if {
00023 public:
00031 CollectorDatafileDefaultImpl1();
00032 virtual ~CollectorDatafileDefaultImpl1() = default;
00033 public: // inherited from Collector_if
00035 virtual void clear() override;
00037 virtual void addValue(double value, double weight=1) override;
00039 virtual double getLastValue() override;
00041 virtual unsigned long numElements() override;
00042 public:
00044 virtual double getValue(unsigned int num) override;
00046 virtual double getNextValue() override;
00048 virtual void seekFirstValue() override;
00050 virtual std::string getDataFilename() override;
00052 virtual void setDataFilename(std::string filename) override;
00053 public:
00055 virtual void setAddValueHandler(CollectorAddValueHandler addValueHandler) override;
00057 virtual void setClearHandler(CollectorClearHandler clearHandler) override;
00058 private:
00059 // Cache file metadata and sequential read position to avoid stale state.
00060 std::string _filename;
00061 double _lastValue = std::numeric_limits<double>::quiet_NaN();
00062 unsigned int _numElements = 0;
00063 unsigned int _nextReadIndex = 0;
00064 };
00065
00066 #endif /* COLLECTORDATAFILEDEFAULTIMPL1_H */

```

## 16.324 CollectorDefaultImpl1.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: CollectorDefaultImpl1.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 1 de Agosto de 2018, 20:43
00012 */
00013
00014 #ifndef COLLECTORDEFAULTIMPL1_H
00015 #define COLLECTORDEFAULTIMPL1_H
00016
00017 #include "Collector_if.h"
00018 //namespace GenesysKernel {
00019
00026 class CollectorDefaultImpl1 : public Collector_if {
00027 public:
00029 CollectorDefaultImpl1();
00030 virtual ~CollectorDefaultImpl1() = default;
00031 public:
00033 virtual void clear() override;
00035 virtual void addValue(double value, double weight=1) override;
00037 virtual double getLastValue() override;
00039 virtual unsigned long numElements() override;
00040 public:
00042 virtual void setAddValueHandler(CollectorAddValueHandler addValueHandler) override;
00044 virtual void setClearHandler(CollectorClearHandler clearHandler) override;
00045 private:
00046 double _lastValue;
00047 unsigned long _numElements = 0;
00048 CollectorAddValueHandler _addValueHandler = nullptr;
00049 CollectorClearHandler _clearHandler = nullptr;
00050 };
00051 //namespace\\}
00052 #endif /* COLLECTORDEFAULTIMPL1_H */

```

## 16.325 Sampler\_if.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Sampler_if.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 14 de Agosto de 2018, 13:20
00012 */
00013
00014 #ifndef Sampler_IF_H
00015 #define Sampler_IF_H
00016
00024 class Sampler_if {
00025 public:
00026 virtual ~Sampler_if() = default;
00027
00034 struct RNG_Parameters {
00035 virtual ~RNG_Parameters() = default;
00036 };
00037 public: // RNG
00042 virtual double random() = 0;
00043 public: // continuous probability distributions
00052 virtual double sampleBeta(double alpha, double beta, double infLimit, double supLimit) = 0;
00054 virtual double sampleBeta(double alpha, double beta) = 0;
00056 virtual double sampleErlang(double mean, int M, double offset = 0.0) = 0;
00058 virtual double sampleExponential(double mean, double offset = 0.0) = 0;
00059 //virtual double sampleGamma(double mean, double alpha, double offset=0.0) = 0;
00061 virtual double sampleGamma(double alpha, double beta, double offset = 0.0) = 0;
00063 virtual double sampleGumbell(double mode, double scale) = 0;
00065 virtual double sampleLogNormal(double mean, double stddev, double offset = 0.0) = 0;
00067 virtual double sampleNormal(double mean, double stddev) = 0;
00069 virtual double sampleTriangular(double min, double mode, double max) = 0;
00071 virtual double sampleUniform(double min, double max) = 0;
00073 virtual double sampleWeibull(double alpha, double scale) = 0;
00074 public: // discrete probability distributions
00075 // @ToDo: (pequena alteração): Poisson, si vous plait!!!!
00077 virtual double sampleBinomial(int trials, double p) = 0;
00079 virtual double sampleBernoulli(double p) = 0;
00081 virtual double sampleDiscrete(double prob, double value, ...) = 0;
00083 virtual double sampleDiscrete(double *prob, double *value, int size) = 0;
00085 virtual double sampleGeometric(double p) = 0;
00086 public:
00088 virtual void setRNGparameters(RNG_Parameters* param) = 0;
00090 virtual RNG_Parameters* getRNGparameters() const = 0;
00091 };
00092
00093 #endif /* Sampler_IF_H */

```

## 16.326 SamplerDefaultImpl1.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: SamplerDefaultImpl1.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 2 de Agosto de 2018, 01:10
00012 */
00013
00014 #ifndef SAMPLERDEFAULTIMPL1_H
00015 #define SAMPLERDEFAULTIMPL1_H
00016
00017 #include "Sampler_if.h"
00018 #include <stdint.h>
00019
00020 //namespace GenesysKernel {
00021
00028 class SamplerDefaultImpl1 : public Sampler_if {
00029 public:
00030
00032 struct DefaultImpl1RNG_Parameters : public RNG_Parameters {
00034 uint32_t seed = 16021974;
00036 uint32_t a = 279470273u; // multiplier
00038 uint32_t m = 0xffffffffb; // module

```

```

00040 uint32_t c = 0; // increment
00041 ~DefaultImpl1RNG_Parameters() = default;
00042 };
00043 public:
00044 SamplerDefaultImpl1();
00045 virtual ~SamplerDefaultImpl1();
00046 public: // RNG
00047 virtual double random() override;
00048 public: // continuous probability distributions
00049 virtual double sampleBeta(double alpha, double beta, double infLimit, double supLimit) override;
00050 virtual double sampleBeta(double alpha, double beta) override;
00051 virtual double sampleErlang(double mean, int M, double offset = 0.0) override;
00052 virtual double sampleExponential(double mean, double offset = 0.0) override;
00053 //virtual double sampleGamma(double mean, double alpha, double offset=0.0);
00054 virtual double sampleGamma(double alpha, double beta, double offset = 0.0) override;
00055 virtual double sampleGumbell(double mode, double scale) override;
00056 virtual double sampleLogNormal(double mean, double stddev, double offset = 0.0) override;
00057 virtual double sampleNormal(double mean, double stddev) override;
00058 virtual double sampleTriangular(double min, double mode, double max) override;
00059 virtual double sampleUniform(double min, double max) override;
00060 virtual double sampleWeibull(double alpha, double scale) override;
00061 public: // discrete probability distributions
00062 // @ToDo: (pequena alteração): Poisson, si vous plait!!!!
00063 virtual double sampleBinomial(int trials, double p) override;
00064 virtual double sampleBernoulli(double p) override;
00065 virtual double sampleDiscrete(double prob, double value, ...) override;
00066 virtual double sampleDiscrete(double *prob, double *value, int size) override;
00067 virtual double sampleGeometric(double p) override;
00068 public:
00069 void reset();
00070 public:
00071 virtual void setRNGparameters(RNG_Parameters* param) override;
00072 virtual RNG_Parameters* getRNGparameters() const override;
00073 private:
00074 RNG_Parameters* _param = new DefaultImpl1RNG_Parameters();
00075 bool _ownsParam = true;
00076 uint32_t _xi;
00077 bool _normalflag;
00078 double _lastnormal;
00079 };
00080
00081
00082 /*
00083 (*
00084
00085 Random generator for Borland Delphi 3.0
00086
00087 Copyright(c) 1998 Private enterprise - firm SOFTLAND, Ukraine
00088
00089 1. What is it?
00090
00091 TMultiRand is a component intended for generating random variates of
00092 different distributions:
00093
00094 Uniform
00095 Triangular
00096 Exponential
00097 Erlang
00098 Normal
00099 Gamma
00100 Beta
00101 Weibull
00102 Lognormal
00103
00104 2. Licensing.
00105
00106 a) This software is under Copyright(c) 1998 SOFTLAND,Ukraine
00107 b) This software is for free usage and distribution.
00108 c) The conditions of the distribution are
00109 - remaining the source code, comments and manual without changes
00110 - supplying the manual
00111 e) The user assumes all risks associated with this software.
00112 Softland is not responsible for any loss or damage incurred
00113 during the use of this software.
00114
00115 3. Manual
00116
00117 The manual for this component is in mrand.pdf
00118
00119 4. Where is it used?
00120
00121 This component is used in the discrete-event simulation system Delsi 1.0,
00122 which is implemented as a set of components for Borland Delphi.
00123
00124

```

```
00147 Find more info at http://www.softland.rovno.ua/delsi.htm
00148
00149 5. What is Softland ?
00150
00151 Softland is Ukrainian software firm.
00152
00153 Our skills:
00154
00155 - Delhpi, C++
00156 - SQL
00157 - Computer simulation of queuing systems
00158 - Web-design
00159
00160 We are looking for the remote development job in areas of
00161 general client/server applications and computer simulation.
00162 The rate of our service is extremely unexpensive.
00163
00164 Find more info at http://www.softland.rovno.ua
00165
00166 *)
00167
00168 unit Mrand;
00169
00170 interface
00171
00172 uses WinTypes, WinProcs, Classes, Dialogs;
00173
00174 const
00175
00176 MODULUS = 2147483647;
00177
00178 type
00179
00180 TMRand = class(TComponent)
00181 private
00182 FSeed: comp;
00183 FSeedInit: comp;
00184 FMultiplier: comp;
00185 NormalFlag: boolean;
00186 NormalResult: real;
00187 function Uni01: real;
00188 procedure SetSeed(Value: comp);
00189 function GetSeed: comp;
00190 procedure SetMultiplier(Value: comp);
00191 function GetMultiplier: comp;
00192 function GammaJonk(Alpha: real): real;
00193 public
00194 constructor Create(Owner: TComponent); override;
00195 procedure Reset;
00196 function Uniform(LowerBound: real; UpperBound: real): real;
00197 function Triangular(Min: real; Mode: real; Max: real): real;
00198 function Exponential(Mean: real): real;
00199 function Erlang(Mean: real; M: integer): real;
00200 function Normal(Mean: real; Deviation: real): real;
00201 function Gamma(Mean: real; Alpha: real): real;
00202 function Beta(ShapeAlpha: real; ShapeBeta: real;
00203 LowerBound: real; UpperBound: real): real;
00204 function Weibull(Alpha: real; Scale: real): real;
00205 function Lognormal(Mean: real; Deviation: real): real;
00206 published
00207 property Seed: comp read GetSeed write SetSeed;
00208 property Multiplier: comp read GetMultiplier write SetMultiplier;
00209 end;
00210
00211
00212 procedure Register;
00213
00214 implementation
00215
00216
00217 constructor TMRand.Create(Owner: TComponent);
00218 begin
00219 inherited Create(Owner);
00220 FSeed:=1000000000;
00221 FMultiplier:=950706376;
00222 NormalFlag:=True;
00223 end;
00224
00225 procedure TMRand.Reset;
00226 begin
00227 NormalFlag:=True;
00228 FSeed:=FSeedInit;
00229 end;
00230
00231
00232 procedure TMRand.SetSeed(Value: comp);
00233 begin
```

```
00234 if (Value<1) or (Value>2147483646) then
00235 begin
00236 MessageDlg(Name+'Seed must be in the range 2...2147483646',mtError,[mbOK],0);
00237 FSeed:=1000000000;
00238 end
00239 else
00240 FSeed:=Value;
00241 FSeedInit:=FSeed;
00242 end;
00243
00244 function TMRand.GetSeed: comp;
00245 begin
00246 if (FSeed<1) or (FSeed>2147483646) then
00247 begin
00248 FSeed:=1000000000;
00249 FSeedInit:=FSeed;
00250 end;
00251 Result:=FSeed;
00252 end;
00253
00254
00255 procedure TMRand.SetMultiplier(Value: comp);
00256 begin
00257 if (Value<2) or (Value>2147483646) then
00258 begin
00259 MessageDlg(Name+'Multiplier must be in the range 2...2147483646',mtError,[mbOK],0);
00260 FMultiplier:=950706376;
00261 end
00262 else
00263 FMultiplier:=Value;
00264 end;
00265
00266 function TMRand.GetMultiplier: comp;
00267 begin
00268 if (FMultiplier<2) or (FMultiplier>2147483646) then
00269 FMultiplier:=950706376;
00270 Result:=FMultiplier;
00271 end;
00272
00273 function TMRand.Uni01: real;
00274 begin
00275 FSeed:=FSeed*FMultiplier;
00276 FSeed:=FSeed-Int(FSeed/MODULUS)*MODULUS;
00277 Result:=FSeed/MODULUS;
00278 end;
00279
00280
00281 { Uniform distribution with the range LowerBound...UpperBound }
00282
00283 function TMRand.Uniform(LowerBound: real; UpperBound: real): real;
00284 begin
00285 if (LowerBound<0) or (UpperBound<0) or (UpperBound<LowerBound) then
00286 begin
00287 MessageDlg('Incorrect parameters of '+Name+'.Uniform',mtError,[mbOK],0);
00288 Result:=0.0;
00289 Exit;
00290 end;
00291 Result:=LowerBound+(UpperBound-LowerBound)*Uni01
00292 end;
00293
00294
00295 { Exponential distribution with mean "Mean" }
00296
00297 function TMRand.Exponential(Mean: real): real;
00298 begin
00299 if Mean>=0.0 then
00300 Result:=Mean*(-ln(Uni01))
00301 else
00302 begin
00303 MessageDlg('Incorrect parameter of '+Name+'.Exponential',mtError,[mbOK],0);
00304 Result:=0;
00305 end;
00306 end;
00307
00308
00309 { M-Erlang distribution with mean "Mean" }
00310
00311 function TMRand.Erlang(Mean: real; M: integer): real;
00312 var
00313 i: integer;
00314 P: real;
00315 begin
00316 if (Mean<0.0) or (M<=0) then
00317 begin
00318 MessageDlg('Incorrect parameters of '+Name+'.Erlang',mtError,[mbOK],0);
00319 Result:=0.0;
00320 Exit;
```



```

00321 end;
00322
00323 P:=1;
00324 for i:=1 to M do
00325 P:=P*Uni01;
00326 Result:=(Mean/M)*(-ln(P));
00327 end;
00328
00329
00330 { Normal distribution with mean "Mean" and deviation "Deviation" }
00331
00332 function TMRand.Normal(Mean: real; Deviation: real): real;
00333 var U1,U2: real;
00334 W,Y: real;
00335 begin
00336 if (Mean<=0.0) or (Deviation<=0.0) then
00337 begin
00338 MessageDlg('Incorrect parameter Deviation of '+Name+'.Normal',mtError,[mbOK],0);
00339 Result:=0.0;
00340 Exit;
00341 end;
00342
00343 if NormalFlag then
00344 begin
00345 repeat
00346 U1:=2*Uni01-1.0;
00347 U2:=2*Uni01-1.0;
00348 W:=U1*U1+U2*U2;
00349 until not (W>=1.0);
00350 Y:= sqrt((-2* ln (W)) / W);
00351 Result:=Mean+U1*Y*Deviation;
00352 NormalResult:=Mean+U2*Y*Deviation;
00353 NormalFlag:=False;
00354 end
00355 else
00356 begin
00357 Result:=NormalResult;
00358 NormalFlag:=True;
00359 end;
00360 end;
00361
00362
00363 { Gamma distribution for Alpha<1 and beta=1 }
00364
00365 function TMRand.GammaJonk(Alpha: real): real;
00366 var R,R1,R2,X,Y:real;
00367 begin
00368 repeat
00369 repeat
00370 R1:=Uni01;
00371 R2:=Uni01;
00372 until ((R1>1e-30) and (R2>1e-30));
00373
00374 if (ln(R2)/alpha<-1e3) then X:=0 else X:= exp(ln(R2)/alpha);
00375 if (ln(R1)/(1-alpha)<-1e3) then Y:=0 else Y:=exp(ln(R1)/(1-alpha));
00376
00377 until (X+Y<=1);
00378
00379 repeat
00380 R:=Uni01;
00381 until (R>1e-20);
00382
00383 Result:=-ln(R)*X/(Y+X);
00384 end;
00385
00386
00387 { Gamma distribution with the mean "Mean" and shape-parameter "Alpha" }
00388
00389 function TMRand.Gamma(Mean: real; Alpha: real): real;
00390 var
00391 i: integer;
00392 P: real;
00393 IntAlpha: integer;
00394 OstAlpha: real;
00395 begin
00396
00397 if (Mean<=0.0) or (Alpha<=0.0) then
00398 begin
00399 MessageDlg('Incorrect parameters of '+Name+'.Gamma',mtError,[mbOK],0);
00400 Result:=0.0;
00401 Exit;
00402 end;
00403
00404 if Alpha<1.0 then
00405 Result:=(Mean/Alpha)*GammaJonk(Alpha)
00406 else
00407 if Alpha=1.0 then

```

```

00408 Result:=Mean*(-ln(Uni01))
00409 else
00410 begin
00411 IntAlpha:=Round(Int(Alpha));
00412 OstAlpha:=Alpha-IntAlpha;
00413 repeat
00414 P:=1;
00415 for i:=1 to IntAlpha do
00416 P:=P*Uni01;
00417 until P>0;
00418 if OstAlpha>0 then
00419 Result:=(Mean/Alpha)*((-ln(P))+GammaJonk(OstAlpha))
00420 else
00421 Result:=(Mean/Alpha)*(-ln(P));
00422 repeat
00423 end;
00424 end;
00425
00426 { Beta distribution with alpha-parameter "ShapeAlpha,
00427 beta-parameter "ShapeBeta,
00428 lower bound "LowerBound and
00429 upper bound "UpperBound
00430 }
00431
00432 function TMRand.Beta(ShapeAlpha: real; ShapeBeta: real;
00433 LowerBound: real; UpperBound: real): real;
00434 var
00435 X,Y1,Y2: real;
00436 begin
00437 if (ShapeAlpha<=0.0) or (ShapeBeta<=0.0) or (LowerBound>UpperBound) or
00438 (LowerBound<0) or (UpperBound<0)
00439 then
00440 begin
00441 MessageDlg('Incorrect parameters of '+Name+'.Beta',mtError,[mbOK],0);
00442 Result:=0.0;
00443 Exit;
00444 end;
00445 repeat
00446 Y1:=Gamma(ShapeAlpha,ShapeAlpha);
00447 Y2:=Gamma(ShapeBeta,ShapeBeta);
00448 X:=Y1/(Y1+Y2);
00449 until (X>=0) and (X<=1.0);
00450 Result:= LowerBound + (UpperBound - LowerBound) * X;
00451 end;
00452
00453 { Weibull distribution with shape-parameter "Alpha" and scale "Scale" }
00454
00455 function TMRand.Weibull(Alpha: real; Scale: real): real;
00456 begin
00457 if (Alpha<=0.0) or (Scale<=0.0) then
00458 begin
00459 MessageDlg('Incorrect parameters of '+Name+'.Weibull',mtError,[mbOK],0);
00460 Result:=0.0;
00461 Exit;
00462 end;
00463 Result:= exp(ln(Scale*(-ln(Uni01)))/Alpha);
00464 end;
00465
00466 { Lognormal distribution with mean "Mean" and deviation "Deviation" }
00467
00468 function TMRand.Lognormal(Mean: real; Deviation: real): real;
00469 var
00470 MeanNorm,DispNorm: real;
00471 begin
00472 if (Mean<=0.0) or (Deviation<=0.0) then
00473 begin
00474 MessageDlg('Incorrect parameters of '+Name+'.Lognormal',mtError,[mbOK],0);
00475 Result:=0.0;
00476 Exit;
00477 end;
00478 DispNorm:=ln((Deviation*Deviation)/(Mean*Mean)+1.0);
00479 MeanNorm:=ln(Mean)-0.5*DispNorm;
00480 Result:=Exp(Normal(MeanNorm,SQRT(DispNorm)));
00481 end;
00482
00483 { Triangular distribution with minimal value "Min", maximal - "Max" and mode "Mode" }
00484
00485 function TMRand.Triangular(Min: real; Mode: real; Max: real): real;
00486 var

```

```

00495 Part1,Part2,Full,R: real;
00496 begin
00497
00498 if (Min>Mode) or (Max<Mode) or (Min>Max) or
00499 (Min<0) or (Max<0) or (Mode<0) then
00500 begin
00501 MessageDlg('Incorrect parameters of '+Name+'.Triangular',mtError,[mbOK],0);
00502 Result:=0.0;
00503 Exit;
00504 end;
00505
00506 Part1:=Mode-Min;
00507 Part2:=Max-Mode;
00508 Full:=Max-Min;
00509 R:=Uni01;
00510 if R<=Part1/Full then
00511 Result:=Min+sqrt(Part1*Full*R)
00512 else
00513 Result:=Max-sqrt(Part2*Full*(1.0-R));
00514 end;
00515
00516
00517 procedure Register;
00518 begin
00519 RegisterComponents('MRand', [TMRand]);
00520 end;
00521
00522
00523 end.
00524
00525 */
00526
00527
00528
00529 //namespace\\
00530 #endif /* SAMPLERDEFAULTIMPL1_H */

```

## 16.327 SorttFile.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: SorttFile.h
00008 * Author: rlcancian
00009 *
00010 * Created on 20 de novembro de 2019, 22:27
00011 */
00012
00013 #ifndef SORTTFILE_H
00014 #define SORTTFILE_H
00015
00016 #include <string>
00017
00024 class SortFile {
00025 public:
00026 SortFile();
00027 ~SortFile() = default;
00028 public:
00034 bool sort();
00039 void setDataFilename(std::string filename);
00040 private:
00042 void run(int start, int end);
00044 void swap(int position1, int position2);
00046 int partition(int start, int end);
00048 void addValue(double value, int position);
00050 double getValue(unsigned long position);
00051 private:
00052 std::string _filename;
00053 double _numElements = 0;
00054 };
00055
00056 #endif /* SORTTFILE_H */

```

## 16.328 Statistics\_if.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.

```

```

00005 */
00006
00007 /*
00008 * File: Statistics_if.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 14 de Agosto de 2018, 13:47
00012 */
00013
00014 #ifndef STATISTICS_IF_H
00015 #define STATISTICS_IF_H
00016
00017 #include <string>
00018 #include "Collector_if.h"
00019
00026 class Statistics_if {
00027 public:
00028 virtual ~Statistics_if() = default;
00030 virtual Collector_if* getCollector() const = 0;
00032 virtual void setCollector(Collector_if* collector) = 0;
00033 public:
00039 virtual unsigned int numElements() = 0;
00045 virtual double min() = 0;
00051 virtual double max() = 0;
00057 virtual double average() = 0;
00063 virtual double variance() = 0;
00069 virtual double stddeviation() = 0;
00075 virtual double variationCoef() = 0;
00081 virtual double halfWidthConfidenceInterval() = 0;
00087 virtual double confidenceLevel() = 0;
00094 virtual unsigned int newSampleSize(double halfWidth) = 0;
00100 virtual void setConfidenceLevel(double confidencelevel) = 0;
00101 };
00102
00103 #endif /* STATISTICS_IF_H */

```

## 16.329 StatisticsDataFile\_if.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: StatisticsDataFile.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 22 de Novembro de 2018, 01:16
00012 */
00013
00014 #ifndef STATISTICSDATAFILE_IF_H
00015 #define STATISTICSDATAFILE_IF_H
00016
00017 #include "CollectorDatafile_if.h"
00018 #include "Statistics_if.h"
00019
00026 class StatisticsDatafile_if : public Statistics_if {
00027 //public:
00028 // virtual CollectorDatafile_if* getCollector() = 0;
00029 // virtual void setCollector(Collector_if* collector) = 0;
00030 public:
00031 virtual ~StatisticsDatafile_if() = default;
00037 virtual double mode() = 0;
00043 virtual double mediane() = 0;
00050 virtual double quartil(unsigned short num) = 0;
00057 virtual double decil(unsigned short num) = 0;
00064 virtual double centil(unsigned short num) = 0;
00070 virtual void setHistogramNumClasses(unsigned short num) = 0;
00076 virtual unsigned short histogramNumClasses() = 0;
00083 virtual double histogramClassLowerLimit(unsigned short classNum) = 0;
00090 virtual unsigned int histogramClassFrequency(unsigned short classNum) = 0;
00091
00092 };
00093
00094 #endif /* STATISTICSDATAFILE_IF_H */

```

## 16.330 StatisticsDataFileDefaultImpl.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates

```

```

00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: StatisticsDataFileDefaultImpl.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 22 de Novembro de 2018, 01:24
00012 */
00013
00014 #ifndef STATISTICSDATAFILEDEFAULTIMPL_H
00015 #define STATISTICSDATAFILEDEFAULTIMPL_H
00016
00017 #include "StatisticsDataFile_if.h"
00018 #include "SortFile.h"
00019 #include <limits.h>
00020 #include <map>
00021 #include <string>
00022
00023 typedef double valueType;
00024
00031 class StatisticsDatafileDefaultImpl1 : public StatisticsDatafile_if {
00032 public:
00034 StatisticsDatafileDefaultImpl1();
00036 virtual ~StatisticsDatafileDefaultImpl1();
00037 public:
00039 virtual Collector_if* getCollector() const override;
00041 virtual void setCollector(Collector_if* collector) override;
00042 public:
00044 virtual unsigned int numElements() override;
00046 virtual double min() override;
00048 virtual double max() override;
00050 virtual double average() override;
00052 virtual double variance() override;
00054 virtual double stddeviation() override;
00056 virtual double variationCoef() override;
00058 virtual double halfWidthConfidenceInterval() override;
00060 virtual double confidenceLevel() override;
00062 virtual unsigned int newSampleSize(double halfWidth) override;
00064 virtual void setConfidenceLevel(double confidencelevel) override;
00065 public:
00067 virtual double mode() override;
00069 virtual double mediane() override;
00071 virtual double quartil(unsigned short num) override;
00073 virtual double decil(unsigned short num) override;
00075 virtual double centil(unsigned short num) override;
00077 virtual void setHistogramNumClasses(unsigned short num) override;
00079 virtual unsigned short histogramNumClasses() override;
00081 virtual double histogramClassLowerLimit(unsigned short classNum) override;
00083 virtual unsigned int histogramClassFrequency(unsigned short classNum) override;
00084 private:
00086 void _invalidateCachedResults();
00088 void _resetSortedFileState();
00090 void _sortFile();
00092 bool _hasNewValue();
00094 double _getNormalProbability(double confidenceLevel);
00095 private:
00096 CollectorDatafile_if* _collector;
00097 bool _ownsCollector = true;
00098 CollectorDatafile_if* _collectorSorted;
00099 bool _ownsCollectorSorted = true;
00100 //CollectorDatafile_if* _collector;
00101 SortFile * sort = new SortFile();
00102 bool _ownsSort = true;
00103 double _confidenceLevel = 0.95;
00104 unsigned long _numElements = 0;
00105 std::map<double, double> _z;
00106 bool _fileSorted = false;
00107 bool _fileSortedCreated = false;
00108 std::string _sortedSourceFilename;
00109 //unsigned long _numElements = 0;
00110 bool _numElementsCalculated = false;
00111 double _max = INT_MIN;
00112 bool _maxCalculated = false;
00113 double _min = INT_MAX;
00114 bool _minCalculated = false;
00115 bool _modeCalculated = false;
00116 double _mode = 0;
00117 bool _medianeCalculated = false;
00118 double _mediane = 0;
00119 double _average = 0;
00120 bool _averageCalculated = false;
00121 double _variance = 0;
00122 bool _varianceCalculated = false;
00123 double _sumSquareElements = 0;
00124 double _sumElements = 0;
00125 double _stddeviation = 0;

```

```

00126 bool _stddeviationCalculated = false;
00127 double _variationCoef = 0;
00128 bool _variationCoefCalculated = false;
00129 bool _halfWidthConfidenceIntervalCalculated = false;
00130 double _lastConfidenceLevel = 0;
00131 double _halfWidthConfidenceInterval = 0;
00132 bool _newSampleSizeCalculated = false;
00133 double _lastNewSampleSizeConfidenceLevel = 0;
00134 double _lastNewSampleSizeHalfWidth = 0;
00135 double _newSampleSize = 0;
00136 bool _quartilCalculated = false;
00137 unsigned short _lastQuartilNum = 0;
00138 double _quartil = 0;
00139 bool _decilCalculated = false;
00140 unsigned short _lastDecilNum = 0;
00141 double _decil = 0;
00142 bool _centilCalculated = false;
00143 unsigned short _lastCentilNum = 0;
00144 double _centil = 0;
00145 bool _histogramNumClassesCalculated = false;
00146 unsigned short _histogramNumClasses = 1;
00147 bool _histogramClassLowerLimitCalculated = false;
00148 unsigned short _lastClassNumHistogramClassLowerLimit = 0;
00149 double _histogramClassLowerLimit = 0;
00150 bool _histogramClassFrequencyCalculated = false;
00151 unsigned short _lastClassNumHistogramClassFrequency = 0;
00152 unsigned int _histogramClassFrequency = 0;
00153 bool _proportionCalculated = false;
00154 double _proportion = 0;
00155 };
00156
00157 #endif /* STATISTICSDEFAULTIMPL_H */

```

## 16.331 StatisticsDefaultImpl1.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: StatisticsDefaultImpl1.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 1 de Agosto de 2018, 21:03
00012 */
00013
00014 #ifndef STATISTICSDEFAULTIMPL1_H
00015 #define STATISTICSDEFAULTIMPL1_H
00016
00017 #include "Statistics_if.h"
00018 #include "Collector_if.h"
00019 #include <map>
00020 //namespace GenesysKernel {
00021
00022 class StatisticsDefaultImpl1 : public Statistics_if {
00023 public:
00024 StatisticsDefaultImpl1();
00025 StatisticsDefaultImpl1(Collector_if* collector);
00026 virtual ~StatisticsDefaultImpl1() override;
00027 public:
00028 virtual Collector_if* getCollector() const override;
00029 virtual void setCollector(Collector_if* collector) override;
00030 public:
00031 virtual unsigned int numElements() override;
00032 virtual double min() override;
00033 virtual double max() override;
00034 virtual double average() override;
00035 virtual double variance() override;
00036 virtual double stddeviation() override;
00037 virtual double variationCoef() override;
00038 virtual double halfWidthConfidenceInterval() override;
00039 virtual double confidenceLevel() override;
00040 virtual unsigned int newSampleSize(double halfWidth) override;
00041 virtual void setConfidenceLevel(double confidencelevel) override;
00042 private:
00043 void collectorAddHandler(double newValue, double newWeight);
00044 void collectorClearHandler();
00045 void _bindCollectorHandlers(Collector_if* collector);
00046 void _resetStateForCurrentCollector();
00047 void initStatistics();
00048 private:
00049 Collector_if* _collector;
00050 bool _ownsCollector = false;

```

```

00077 bool _onlineStateValid = true;
00078 unsigned long _elems;
00079 double _sumData;
00080 double _sumDataSquare;
00081 double _sumWeight;
00082 double _sumWeightSquare;
00083 double _min;
00084 double _max;
00085 double _average;
00086 double _variance;
00087 double _unweightedvariance;
00088 double _unbiasedVariance;
00089 double _stddeviation;
00090 double _variationCoef;
00091 double _confidenceLevel = 0.95;
00092 double _criticalTn_1 = 1.96;
00093 double _halfWidth;
00094 const std::map<double, double> _supportedConfidenceLevels{
00095 {0.50, 0.0},
00096 {0.80, 1.282},
00097 {0.90, 1.645},
00098 {0.95, 1.96},
00099 {0.975, 2.06},
00100 {0.98, 2.326},
00101 {0.99, 2.576},
00102 {0.995, 2.807}
00103 };
00104 };
00105 //namespace\\}
00106 #endif /* STATISTICSDEFAULTIMPL1_H */

```

## 16.332 TraitsKernel.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Traits.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 14 de Agosto de 2018, 19:36
00012 */
00013
00014
00015 #ifndef TRAITSKERNEL_H
00016 #define TRAITSKERNEL_H
00017
00018
00019 #include "simulator/persistence/PersistenceDefaultImpl2.h"
00020 #include "simulator/persistence/PersistencePartialLoadImpl1.h"
00021 #include "simulator/SimulationReporterDefaultImpl1.h"
00022 #include "simulator/essentialPlugins/Counter.h"
00023 #include "simulator/model/ModelCheckerDefaultImpl1.h"
00024 #include "simulator/ParserDefaultImpl2.h"
00025 #include "simulator/SimulationReporter_if.h"
00026
00027 #include "statistics/CollectorDatafileDefaultImpl1.h"
00028 #include "statistics/CollectorDefaultImpl1.h"
00029 #include "statistics/StatisticsDefaultImpl1.h"
00030 #include "statistics/StatisticsDataFileDefaultImpl.h"
00031 #include "statistics/SamplerDefaultImpl1.h"
00032
00033 #include "../plugins/PluginConnectorStaticImpl1.h"
00034 #include "../plugins/PluginConnectorDummyImpl1.h"
00035
00036
00037 //namespace GenesysKernel {
00038
00039 template <typename T>
00040 struct TraitsKernel {
00041 };
00042
00043 /*
00044 * Simulation and Simulation Parts
00045 */
00046
00047 template <>
00048 struct TraitsKernel<Simulator> {
00049 static const TraceManager::Level traceLevel = TraceManager::Level::L9_mostDetailed;
00050 };
00051
00052 template <>

```

```

00053 struct TraitsKernel<SimulationReporter_if> {
00054 typedef SimulationReporterDefaultImpl1 Implementation;
00055 typedef Counter CounterImplementation;
00056 static const TraceManager::Level traceLevel = TraceManager::Level::L2_results;
00057 };
00058
00059 template <>
00060 struct TraitsKernel<PluginConnector_if> {
00061 //typedef PluginConnectorStaticImpl1 Implementation;
00062 typedef PluginConnectorDummyImpl1 Implementation;
00063 //typedef PluginConnectorDynamicLibraryLoader Implementation;
00064 static const TraceManager::Level traceLevel = TraceManager::Level::L4_warning;
00065 };
00066
00067 template <>
00068 struct TraitsKernel<Parser_if> {
00069 typedef ParserDefaultImpl2 Implementation;
00070 };
00071
00072 /*
00073 * Model and Model Parts
00074 */
00075
00076 template <>
00077 struct TraitsKernel<Model> {
00078 typedef StatisticsDefaultImpl1 StatisticsCollector_StatisticsImplementation;
00079 typedef CollectorDefaultImpl1 StatisticsCollector_CollectorImplementation;
00080 static const TraceManager::Level traceLevel = TraceManager::Level::L5_event;
00081 };
00082
00083 template <>
00084 struct TraitsKernel<ModelComponent> {
00085 static constexpr bool reportStatistics = true;
00086 static const TraceManager::Level traceLevel = TraceManager::Level::L2_results;
00087 };
00088
00089 template <>
00090 struct TraitsKernel<ModelDataDefinition> {
00091 static constexpr bool reportStatistics = true;
00092 static const TraceManager::Level traceLevel = TraceManager::Level::L2_results;
00093 };
00094
00095 template <>
00096 struct TraitsKernel<ModelChecker_if> {
00097 typedef ModelCheckerDefaultImpl1 Implementation;
00098 static const TraceManager::Level traceLevel = TraceManager::Level::L2_results;
00099 };
00100
00101 template <>
00102 struct TraitsKernel<Persistence_if> {
00103 typedef PersistencePartialLoadImpl1 Implementation;
00104 static const TraceManager::Level traceLevel = TraceManager::Level::L2_results;
00105 };
00106
00107 /*
00108 * Statistics
00109 */
00110 template <>
00111 struct TraitsKernel<Statistics_if> {
00112 typedef double DataType;
00113 // @ToDo: (importante): not used yet. Change all classes that collect statistics to this type (so
00114 // classes that deal with erros and bit limits can be assigned to it
00115 static constexpr double SignificanceLevel = 0.05;
00116 };
00117
00118 template <>
00119 struct TraitsKernel<StatisticsDatafile_if> {
00120 typedef StatisticsDatafileDefaultImpl1 Implementation;
00121 typedef CollectorDatafileDefaultImpl1 CollectorImplementation;
00122 static constexpr double SignificanceLevel = 0.05;
00123 };
00124
00125 template <>
00126 struct TraitsKernel<Sampler_if> {
00127 typedef SamplerDefaultImpl1 Implementation;
00128 typedef SamplerDefaultImpl1::DefaultImpl1RNG_Parameters Parameters;
00129 };
00130
00131 template <>
00132 struct TraitsKernel<Collector_if> {
00133 typedef CollectorDatafileDefaultImpl1 Implementation;
00134 typedef double DataType;
00135 // @ToDo: (importante): not used yet. Change all classes that collect statistics to this type (so
00136 // classes that deal with erros and bit limits can be assigned to it
00137 };
00138
00139 //namespace\}

```



```

00138
00139 #endif /* TRAITSKERNEL_H */

```

## 16.333 Exact.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Exact.h
00009 * Author: rlcancian
00010 *
00011 * Created on 12 de agosto de 2021, 18:55
00012 */
00013
00014 #ifndef Exact_H
00015 #define Exact_H
00016
00017 #include <iostream>
00018 #include <iomanip>
00019
00020 class Exact {
00021 //friend ostream &operator<<(ostream&, const Exact &);
00022 //friend istream &operator>>(istream&, Exact &);
00023 public:
00024
00025 Exact() {
00026 };
00027
00028 Exact(int num, int den = 1) {
00029 _num = num;
00030 _den = den;
00031 };
00032
00033 const Exact &operator+(const Exact &right) {
00034 Exact e;
00035 e._num = this->_num * right._den + right._num * this->_den;
00036 e._den = this->_den * right._den;
00037 return &e;
00038 }
00039
00040 const Exact &operator+=(const Exact &right) {
00041 this->_num = this->_num * right._den + right._num * this->_den;
00042 this->_den *= right._den;
00043 }
00044
00045 const Exact &operator-(const Exact &right) {
00046 this->_num = this->_num * right._den - right._num * this->_den;
00047 this->_den *= right._den;
00048 }
00049
00050 const Exact &operator*=(const Exact &right) {
00051 this->_num *= right._num;
00052 this->_den *= right._den;
00053 }
00054
00055 const Exact &operator/=(const Exact &right) {
00056 this->_num *= right._den;
00057 this->_den *= right._num;
00058 }
00059
00060 const Exact &operator=(const Exact &right) {
00061 this->_num = right._num;
00062 this->_den = right._den;
00063 }
00064
00065 const Exact &operator=(const int &right) {
00066 this->_num = right;
00067 this->_den = 1;
00068 }
00069
00070 bool operator==(const Exact &right) {
00071 // verify proportionality is missing
00072 return (this->_num == right._num) && (this->_den == right._den);
00073 }
00074
00075 bool operator==(const double &right) {
00076 // verify proportionality is missing
00077 return (this->_num / this->_den) == right;
00078 }
00079
00080 bool operator<(const Exact &right) {

```

```

00081 // verify proportionality is missing
00082 return (this->_num / this->_den) < (right._num / right._den);
00083 }
00084
00085 bool operator<(const double &right) {
00086 // verify proportionality is missing
00087 return (this->_num / this->_den) < right;
00088 }
00089
00090 bool operator<=(const Exact &right) {
00091 // verify proportionality is missing
00092 return (this->_num / this->_den) <= (right._num / right._den);
00093 }
00094
00095 bool operator<=(const double &right) {
00096 // verify proportionality is missing
00097 return (this->_num / this->_den) <= right;
00098 }
00099
00100 bool operator>(const Exact &right) {
00101 // verify proportionality is missing
00102 return (this->_num / this->_den) > (right._num / right._den);
00103 }
00104
00105 bool operator>(const double &right) {
00106 // verify proportionality is missing
00107 return (this->_num / this->_den) > right;
00108 }
00109
00110 bool operator>=(const Exact &right) {
00111 // verify proportionality is missing
00112 return (this->_num / this->_den) >= (right._num / right._den);
00113 }
00114
00115 bool operator>=(const double &right) {
00116 // verify proportionality is missing
00117 return (this->_num / this->_den) >= right;
00118 }
00119
00120 void Simplify() {
00121 }
00122
00123 double toFloat() {
00124 return _num / _den;
00125 };
00126 protected:
00127 private:
00128 double _num, _den;
00129 };
00130
00131
00132 /*
00133 ostream &operator<<(ostream &output, const Exact &num) {
00134 output << num._num << "/" << num._den;
00135 return output;
00136 }
00137
00138 /*
00139 istream &operator>>(istream &input, Exact &num) {
00140 input >> num._num;
00141 input >> num._den;
00142 return input;
00143 }
00144 */
00145
00146
00147 #endif /* Exact_H */
00148

```

## 16.334 List.h

```

00001 #ifndef GENLIST_H
00002 #define GENLIST_H
00003 /*
00004 * To change this license header, choose License Headers in Project Properties.
00005 * To change this template file, choose Tools | Templates
00006 * and open the template in the editor.
00007 */
00008
00009 /*
00010 * File: List.h
00011 * Author: rafael.luiz.cancian
00012 *
00013 * Created on 21 de Junho de 2018, 12:55
00014 */

```

```

00015
00016
00017
00018 #include <string>
00019 #include <list>
00020 #include <map>
00021 #include <iterator>
00022 #include <functional>
00023 #include <algorithm>
00024
00025
00032 template <typename T>
00033 class List {
00034 public:
00035 using CompFuncnt = std::function<bool(const T, const T) >;
00036 public:
00037 List();
00038 List(const List<T>& origin);
00039 List<T>& operator=(const List<T>& origin);
00040 virtual ~List();
00041 public: // direct access to list
00042 unsigned int size();
00043 bool empty();
00044 void clear();
00045 void pop_front();
00046 template<class Compare>
00047 void sort(Compare comp);
00048 std::list<T>* list() const;
00049 public: // new methods
00050 T create();
00051 template<typename U>
00052 T create(U arg);
00053 std::string show();
00054 typename std::list<T>::iterator find(T element);
00055 //int rankOf(T modeldatum); //!< returns the position (1st position=0) of the modeldatum if found,
 or negative value if not found
00056 public: // improved (easier) methods
00057 void insert(T element);
00058 void remove(T element);
00059 void setAtRank(unsigned int rank, T element);
00060 T getAtRank(unsigned int rank);
00061 T next();
00062 T front();
00063 T last();
00064 T previous();
00065 T current(); // get current modeldatum on the list (the last used)
00066 void setSortFunc(CompFuncnt _sortFunc);
00067 protected:
00068 //std::map<Util::identification, T>* _map;
00069 std::list<T>* _list;
00070 CompFuncnt _sortFunc{[](const T, const T) {
00071 return false;
00072 }};
00073 typename std::list<T>::iterator _it;
00074 };
00075
00076 template <typename T>
00077 List<T>::List() {
00078 //_map = new std::map<Util::identification, T>();
00079 _list = new std::list<T>();
00080 _it = _list->begin();
00081 }
00082
00083 template <typename T>
00084 List<T>::List(const List<T>& origin) {
00085 _list = new std::list<T>(*origin._list);
00086 _sortFunc = origin._sortFunc;
00087 _it = _list->begin();
00088 }
00089
00090 template <typename T>
00091 List<T>& List<T>::operator=(const List<T>& origin) {
00092 if (this != &origin) {
00093 *_list = *origin._list;
00094 _sortFunc = origin._sortFunc;
00095 _it = _list->begin();
00096 }
00097 return *this;
00098 }
00099
00100 template <typename T>
00101 List<T>::~~List() {
00102 delete _list;
00103 }
00104
00105 template <typename T>
00106 std::list<T>* List<T>::list() const {

```

```

00129 return _list;
00130 }
00131
00132 template <typename T>
00133 unsigned int List<T>::size() {
00134 return _list->size();
00135 }
00136
00137 //template <typename T>
00138 //List<T>::List(const List& orig) {
00139 //}
00140
00141 //template <typename T>
00142 //List<T>::~~List() {
00143 //}
00144
00145 template <typename T>
00146 std::string List<T>::show() {
00147 int i = 0;
00148 std::string text = "{";
00149 for (typename std::list<T>::iterator it = _list->begin(); it != _list->end(); it++, i++) {
00150 text += "[" + std::to_string(i) + "]=(" + (*it)->show() + "),";
00151 }
00152 text += "}";
00153 return text;
00154 }
00155
00156 template <typename T>
00157 void List<T>::insert(T element) {
00158 const bool wasEmpty = _list->empty();
00159 _list->insert(std::upper_bound(_list->begin(), _list->end(), element, _sortFunc), element);
00160 if (wasEmpty) {
00161 _it = _list->begin();
00162 }
00163 }
00164
00165 template <typename T>
00166 bool List<T>::empty() {
00167 return _list->empty();
00168 }
00169
00170 template <typename T>
00171 void List<T>::pop_front() {
00172 if (_list->empty()) {
00173 return;
00174 }
00175 const bool cursorAtFront = _it == _list->begin();
00176 _list->pop_front();
00177 if (cursorAtFront) {
00178 _it = _list->begin(); // if it points to the removed modeldatum, then changes to begin
00179 }
00180 }
00181
00182 template <typename T>
00183 void List<T>::remove(T element) {
00184 // Reset the internal cursor after erasure to avoid dereferencing a stale iterator during
 recursive removals.
00185 _list->remove(element);
00186 _it = _list->begin();
00187 }
00188
00189 template <typename T>
00190 T List<T>::create() {
00191 return new T();
00192 }
00193
00194 template <typename T>
00195 void List<T>::clear() {
00196 _list->clear();
00197 _it = _list->begin();
00198 }
00199
00200 template <typename T>
00201 T List<T>::getAtRank(unsigned int rank) {
00202 unsigned int thisRank = 0;
00203 for (typename std::list<T>::iterator it = _list->begin(); it != _list->end(); it++) {
00204 if (rank == thisRank) {
00205 return (*it);
00206 } else {
00207 thisRank++;
00208 }
00209 }
00210 return T{};
00211 }
00212
00213 template <typename T>
00214 void List<T>::setAtRank(unsigned int rank, T element) {

```

```

00215 if (rank == _list->size()) {
00216 _list->insert(_list->end(), element);
00217 } else {
00218 unsigned int thisRank = 0;
00219 for (typename std::list<T>::iterator it = _list->begin(); it != _list->end(); it++) {
00220 if (rank == thisRank) {
00221 *it = element;
00222 return;
00223 } else {
00224 thisRank++;
00225 }
00226 }
00227 }
00228 }
00229
00230 template <typename T>
00231 T List<T>::next() {
00232 if (_list->empty() || _it == _list->end()) {
00233 return T{};
00234 }
00235 _it++;
00236 if (_it != _list->end())
00237 return (*_it);
00238 else
00239 return T{};
00240 }
00241 }
00242
00243 template <typename T>
00244 typename std::list<T>::iterator List<T>::find(const T element) {
00245 for (typename std::list<T>::iterator it = _list->begin(); it != _list->end(); it++) {
00246 if ((*it) == element) {
00247 return it;
00248 }
00249 }
00250 return _list->end();
00251 }
00252
00253 /*
00254 template <typename T>
00255 int List<T>::rankOf(T modeldatum) {
00256 int rank = 0;
00257 for (typename std::list<T>::iterator it = _list->begin(); it != _list->end(); it++) {
00258 if ((*it) == modeldatum) {
00259 return rank;
00260 } else
00261 rank++;
00262 }
00263 return -1; // not found -> negative rank
00264 }
00265 */
00266
00267 template <typename T>
00268 T List<T>::front() {
00269 if (_list->empty()) {
00270 return T{};
00271 }
00272 _it = _list->begin();
00273 //if (_it != _list->end())
00274 return (*_it);
00275 //else
00276 //return dynamic_cast<T>(nullptr);
00277 }
00278
00279 template <typename T>
00280 T List<T>::last() {
00281 if (_list->empty()) {
00282 return T{};
00283 }
00284 _it = _list->end();
00285 _it--;
00286 return (*_it);
00287 //else return nullptr;
00288 }
00289
00290 template <typename T>
00291 T List<T>::previous() {
00292 if (_list->empty() || _it == _list->begin()) {
00293 return T{};
00294 }
00295 _it--;
00296 return (*_it);
00297 }
00298
00299 template <typename T>
00300 T List<T>::current() {
00301 if (_list->empty() || _it == _list->end()) {

```

```

00302 return T{};
00303 }
00304 return (*_it);
00305 }
00306
00307 template <typename T>
00308 void List<T>::setSortFunc(CompFunc _sortFunc) {
00309 this->_sortFunc = _sortFunc;
00310 }
00311
00312 template <typename T>
00313 template<typename U>
00314 T List<T>::create(U arg) {
00315 return T(arg);
00316 }
00317
00318 template <typename T>
00319 template<class Compare>
00320 void List<T>::sort(Compare comp) {
00321 _list->sort(comp);
00322 }
00323
00324 #endif

```

## 16.335 ListObservable.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: TManager.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 21 de Junho de 2018, 12:55
00012 */
00013
00014 #ifndef LISTOBSERVABLE_H
00015 #define LISTOBSERVABLE_H
00016
00017 #include <string>
00018 #include <list>
00019 #include <map>
00020 #include <iterator>
00021 #include <functional>
00022 #include <algorithm>
00023 #include "Util.h"
00024 #include "../simulator/ModelElement.h"
00025
00026 #include "List.h"
00027
00028 template <typename T>
00029 class ListObservable : public List {
00030 public:
00031 using CompFunc = std::function<bool(const T, const T) >;
00032 public:
00033 ListObservable();
00034 virtual ~ListObservable() = default;
00035 public: // direct access to list
00036 void clear();
00037 void pop_front();
00038 public: // improved (easier) methods
00039 void insert(T element);
00040 void remove(T element);
00041 void setAtRank(unsigned int rank, T element);
00042 };
00043
00044 template <typename T>
00045 ListObservable<T>::ListObservable() {
00046 //_map = new std::map<Util::identification, T>();
00047 _list = new std::list<T>();
00048 _it = _list->begin();
00049 }
00050
00051 template <typename T>
00052 std::list<T>* ListObservable<T>::list() const {
00053 return _list;
00054 }
00055
00056 template <typename T>
00057 unsigned int ListObservable<T>::size() {
00058 return _list->size();
00059 }

```

```

00072
00073 //template <typename T>
00074 //ListObservable<T>::ListObservable(const ListObservable& orig) {
00075 //}
00076
00077 //template <typename T>
00078 //ListObservable<T>::~ListObservable() {
00079 //}
00080
00081 template <typename T>
00082 std::string ListObservable<T>::show() {
00083 int i = 0;
00084 std::string text = "{";
00085 for (typename std::list<T>::iterator it = _list->begin(); it != _list->end(); it++, i++) {
00086 text += "[" + std::to_string(i) + "]=(" + (*it)->show() + "),"";
00087 }
00088 text += "}";
00089 return text;
00090 }
00091
00092 template <typename T>
00093 void ListObservable<T>::insert(T element) {
00094 _list->insert(std::upper_bound(_list->begin(), _list->end(), element, _sortFunc), element);
00095 }
00096
00097 template <typename T>
00098 bool ListObservable<T>::empty() {
00099 return _list->empty();
00100 }
00101
00102 template <typename T>
00103 void ListObservable<T>::pop_front() {
00104 typename std::list<T>::iterator itTemp = _list->begin();
00105 _list->pop_front();
00106 if (_it == itTemp) { /* @ToDo: (importante): check this */
00107 _it = _list->begin(); // if it points to the removed element, then changes to begin
00108 }
00109 }
00110
00111 template <typename T>
00112 void ListObservable<T>::remove(T element) {
00113 _list->remove(element);
00114 if ((*_it) == element) { /* @ToDo: (importante): check this */
00115 _it = _list->begin(); // if it points to the removed element, then changes to begin
00116 }
00117 }
00118
00119 template <typename T>
00120 T ListObservable<T>::create() {
00121 return new T();
00122 }
00123
00124 template <typename T>
00125 void ListObservable<T>::clear() {
00126 _list->clear();
00127 }
00128
00129 template <typename T>
00130 T ListObservable<T>::getAtRank(unsigned int rank) {
00131 unsigned int thisRank = 0;
00132 for (typename std::list<T>::iterator it = _list->begin(); it != _list->end(); it++) {
00133 if (rank == thisRank) {
00134 return (*it);
00135 } else {
00136 thisRank++;
00137 }
00138 }
00139 return 0; /* @ToDo: (importante): Invalid return depends on T. If T is pointer, nullptr works
fine. If T is double, it does not. I just let (*it), buut it is not nice */
00140 }
00141
00142 template <typename T>
00143 void ListObservable<T>::setAtRank(unsigned int rank, T element) {
00144 if (rank == _list->size()) {
00145 _list->insert(_list->end(), element);
00146 } else {
00147 unsigned int thisRank = 0;
00148 for (typename std::list<T>::iterator it = _list->begin(); it != _list->end(); it++) {
00149 if (rank == thisRank) {
00150 *it = element;
00151 return;
00152 } else {
00153 thisRank++;
00154 }
00155 }
00156 }
00157 }

```

```

00158
00159 template <typename T>
00160 T ListObservable<T>::next() {
00161 _it++;
00162 if (_it != _list->end())
00163 return (*_it);
00164 else
00165 return nullptr;
00166 }
00167 }
00168
00169 template <typename T>
00170 typename std::list<T>::iterator ListObservable<T>::find(T element) {
00171 for (typename std::list<T>::iterator it = _list->begin(); it != _list->end(); it++) {
00172 if ((*it) == element) {
00173 return it;
00174 }
00175 }
00176 return _list->end(); /* @ToDo: (importante): check nullptr or invalid iterator when not found */
00177 //return nullptr;
00178 }
00179
00180 /*
00181 template <typename T>
00182 int ListObservable<T>::rankOf(T element) {
00183 int rank = 0;
00184 for (typename std::list<T>::iterator it = _list->begin(); it != _list->end(); it++) {
00185 if ((*it) == element) {
00186 return rank;
00187 } else
00188 rank++;
00189 }
00190 return -1; // not found -> negative rank
00191 }
00192 */
00193
00194 template <typename T>
00195 T ListObservable<T>::front() {
00196 _it = _list->begin();
00197 //if (_it != _list->end())
00198 return (*_it);
00199 //else
00200 //return dynamic_cast<T>(nullptr);
00201 }
00202
00203 template <typename T>
00204 T ListObservable<T>::last() {
00205 _it = _list->end();
00206 _it--;
00207 //if (_it != _list->end()) // @ToDo: (importante): CHECK!!!
00208 return (*_it);
00209 //else return nullptr;
00210 }
00211
00212 template <typename T>
00213 T ListObservable<T>::previous() {
00214 _it--; // @ToDo: (importante): CHECK!!!
00215 return (*_it);
00216 }
00217
00218 template <typename T>
00219 T ListObservable<T>::current() {
00220 /* @ToDo: (importante): To implement (i thing it's just to check). Must actualize _it on other
00221 methods when other elements are accessed */
00222 return (*_it);
00223 }
00224
00225 template <typename T>
00226 void ListObservable<T>::setSortFunc(CompFunc _sortFunc) {
00227 this->_sortFunc = _sortFunc;
00228 }
00229
00230 template <typename T>
00231 template<typename U>
00232 T ListObservable<T>::create(U arg) {
00233 return T(arg);
00234 }
00235
00236 template <typename T>
00237 template<class Compare>
00238 void ListObservable<T>::sort(Compare comp) {
00239 _list->sort(comp);
00240 }
00241 #endif /* LISTOBSERVABLE_H */

```



## 16.336 Util.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Util.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 21 de Junho de 2018, 13:02
00012 */
00013
00014
00015 //
00016 // VINCULADOR
00017 // SAIDA
00018 // ${CND_DISTDIR}/${CND_CONF}/${CND_PLATFORM}/genesysterminalapplication
00019 //
00020
00021
00022 #ifndef UTIL_H
00023 #define UTIL_H
00024
00025 #include <map>
00026 #include <typeinfo>
00027 #include <string>
00028 #include <list>
00029 #include <cctype>
00030 #include <algorithm>
00031 #include <cctype>
00032 #include <locale>
00033 #include <stdio.h>
00034 // dir
00035 #include <limits.h>
00036 #include <unistd.h>
00037 #include <vector>
00038 #include <dirent.h>
00039 #include <errno.h>
00040 #include <iostream>
00041 #include <sys/stat.h>
00042 #include <cstring>
00043
00044 #include <iostream>
00045 #include <string>
00046 //namespace GenesysKernel {
00047
00048 class Util {
00049 public:
00050 typedef unsigned long identification;
00051 typedef unsigned int rank;
00052
00053 // @ToDo: (pequena alteração): Should be insde ModelSimulation, where time goes on
00054
00055 enum class TimeUnit : int {
00056 unknown = 0,
00057 picosecond = 1,
00058 nanosecond = 2,
00059 microsecond = 3,
00060 millisecond = 4,
00061 second = 5,
00062 minute = 6,
00063 hour = 7,
00064 day = 8,
00065 week = 9,
00066 num_elements = 10
00067 };
00068
00069 static std::string StrTimeUnitShort(Util::TimeUnit timeUnit);
00070 static std::string StrTimeUnitLong(Util::TimeUnit timeUnit);
00071 static std::string convertEnumToStr(Util::TimeUnit timeUnit);
00072
00073 // @ToDo: (pequena alteração): check: here? Shouldn't it be on SimulationReport interface?
00074 enum class AllocationType : int {
00075 ValueAdded = 0, NonValueAdded = 1, Transfer = 2, Wait = 3, Others = 4, num_elements = 5
00076 };
00077
00078 static std::string StrAllocation(Util::AllocationType allocation);
00079 static std::string convertEnumToStr(Util::AllocationType allocation);
00080
00081 enum class TimeFormat : unsigned int {
00082 twelve = 12,
00083 twentyFour = 24,
00084 };
00085
00086
00087
00088
00089
00090
00091
00092
00093
00094
00095
00096
00097
00098
00099
00100
00101
00102
00103
00104
00105
00106
00107
00108
00109
00110
00111
00112
00113

```

```

00114 private:
00115 static unsigned int _S_indentation;
00116 static Util::identification _S_lastId;
00117 static std::map<std::string, Util::identification> _S_lastIdOfType;
00118 static std::map<std::string, std::string> _S_TypeOf;
00119
00120 public: // indentation and string
00121 static void SetIndent(const unsigned short indent);
00122 static void IncIndent();
00123 static void DecIndent();
00124 static void SepKeyVal(std::string str, std::string& key, std::string& value);
00125 static std::string Indent();
00126 static std::string SetW(std::string text, unsigned short width);
00127 static std::string StrTruncIfInt(double value);
00128 static std::string StrTruncIfInt(std::string strValue);
00129 static std::string Trim(std::string str);
00130 static std::string StrReplace(std::string text, std::string searchFor, std::string replaceBy);
00131 static std::string StrReplaceSpecialChars(std::string text);
00132 static std::string StrIndex(int index);
00133 //static char* Str2CharPtr(std::string str);
00134 static void Trimwithin(std::string& str);
00135
00136 public: // show strucutres
00137 static std::string Map2str(std::map<std::string, std::string>* mapss);
00138 static std::string Map2str(std::map<std::string, double>* mapss);
00139 static std::string List2str(std::list<unsigned int>* list);
00140
00141 public:
00142 // identitification // @ToDo: (importante): CHECK ALL, since some should be private and available
00143 to FRIEND classes in the kernel
00144 static Util::identification GenerateNewId();
00145 static Util::identification GenerateNewIdOfType(std::string objtype);
00146 static Util::identification GetLastIdOfType(std::string objtype);
00147 static void ResetIdOfType(std::string objtype);
00148 static void ResetAllIds();
00149
00150 public: // simulation support
00151 static double TimeUnitConvert(Util::TimeUnit timeUnit1, Util::TimeUnit timeUnit2);
00152
00153 public: // files
00154 static char DirSeparator();
00155 static std::string FilenameFromFullFilename(const std::string& s);
00156 static void FileDelete(const std::string& filename);
00157 static std::string PathFromFullFilename(const std::string& s);
00158 static std::string RunningPath();
00159 static std::vector<std::string> ListFiles(std::string dir, std::string fileFilter = "",
00160 mode_t attribFilter = S_IFREG & S_IFDIR);
00161 static bool FileExists(const std::string& name);
00162
00163 //public: // operating system specifics
00164 class CommandResult {
00165 public:
00166 CommandResult() {
00167 }
00168
00169 bool success = false;
00170 std::string commandStdOutput = "";
00171 std::string commandErrOutput = "";
00172 std::string workingDirectory = "./";
00173 std::string destinationPath = "./";
00174 };
00175
00176 static Util::CommandResult ExecuteCommand(std::string command);
00177
00178 public: // template implementations
00179
00180 template <typename T>
00181 static std::string TypeOf() {
00182 std::string name = typeid(T).name();
00183 std::map<std::string, std::string>::iterator it = _S_TypeOf.find(name);
00184 if (it != _S_TypeOf.end()) {
00185 return (*it).second;
00186 }
00187 else {
00188 std::string newname(name);
00189 while (std::isdigit(newname[0])) {
00190 newname.erase(0, 1);
00191 }
00192 _S_TypeOf.insert(std::pair<std::string, std::string>(name, newname));
00193 return newname;
00194 }
00195 }
00196
00197 template <class T>
00198 static Util::identification GenerateNewIdOfType() {
00199 std::string objtype = Util::TypeOf<T>();
00200 std::map<std::string, Util::identification>::iterator it =

```

```

 Util::_S_lastIdOfType.find(objtype);
00353 if (it == Util::_S_lastIdOfType.end()) {
00354 // a new one. create the pair
00355 Util::_S_lastIdOfType.insert(std::pair<std::string, Util::identification>(objtype, 0));
00356 it = Util::_S_lastIdOfType.find(objtype);
00357 }
00358 Util::identification next = (*it).second;
00359 next++;
00360 (*it).second = next;
00361 return (*it).second;
00362 }
00363
00364 private:
00365 Util();
00366 virtual ~Util() = default;
00367 };
00368
00369 //namespace\
00370 #endif /* UTIL_H */

```

## 16.337 Genesys++-driver.h

```

00001 #ifndef GENESYSPP_DRIVER_H
00002 #define GENESYSPP_DRIVER_H
00003
00004 #include <string>
00005 #include <map>
00006 #include "GenesysParser.h"
00007 #include "../kernel/simulator/model/Model.h"
00008 #include "kernel/simulator/SimulationControlAndResponse.h"
00009 #include "kernel/util/Util.h"
00010 #include "kernel/statistics/Sampler_if.h"
00011 // Tell Flex the lexer's prototype ...
00012
00013
00014 #define YY_DECL \
00015 yy::genesyspp_parser::symbol_type yylex (genesyspp_driver& driver)
00016
00017 /* # define YY_DECL \
00018 // yy::genesyspp_parser::symbol_type yylex (semantic_type* yylval, location_type* yylloc,
00019 // genesyspp_driver& driver)
00020 */
00021 // ... and declare it for the parser's sake.
00022 YY_DECL;
00023 // Conducting the whole scanning and parsing of Calc++.
00024
00025 //namespace GenesysKernel {
00026
00027 class genesyspp_driver {
00028 public:
00029 genesyspp_driver();
00030 genesyspp_driver(/*GenesysKernel::*Model* model, Sampler_if* sampler, bool throws = false);
00031 virtual ~genesyspp_driver();
00032 genesyspp_driver(const genesyspp_driver& other);
00033 genesyspp_driver& operator=(const genesyspp_driver& other);
00034 genesyspp_driver(genesyspp_driver&& other) noexcept;
00035 genesyspp_driver& operator=(genesyspp_driver&& other) noexcept;
00036 public:
00037 // Handling the scanner.
00038 void scan_begin_file();
00039 void scan_end_file();
00040 void scan_begin_str();
00041 void scan_end_str();
00042
00043 // Run the parser on file F.
00044 // Return 0 on success.
00045 int parse_file(const std::string& f);
00046 int parse_str(const std::string& str);
00047
00048 // Error handling.
00049 void error(const yy::location& l, const std::string& m);
00050 void error(const std::string& m);
00051
00052 double getResult();
00053 void setResult(double value);
00054
00055 bool getThrowsException();
00056 void setThrowsException(bool throws);
00057
00058 void setErrorMessage(std::string message);
00059 std::string getErrorMessage();
00060
00061 /*GenesysKernel::*Model* getModel();
00062 //Sampler_if* getProbs();

```

```

00063 std::string getFile();
00064 void setFile(std::string f);
00065
00066 std::string getStrToParse();
00067 void setStrToParse(std::string str);
00068
00069 public: // Sampler
00070 void setSampler(Sampler_if* _sampler);
00071 Sampler_if* getSampler() const;
00072
00073 public: // trying to get infos about ModelDataElements refered in expressions (so they are not
 considered orphans
00074 void setRegisterReferedDataElements(bool value);
00075 bool getRegisterReferedDataElements();
00076 std::map<std::string, std::list<std::string>*>* getReferedDataElements();
00077 void clearReferedDataElements();
00078 void addRefered(std::pair<std::string, std::string> referedElement);
00079
00080 public: // SimulationResponse / SimulationControl helpers for parser semantics
00081 SimulationResponse* findSimulationResponse(const std::string& name) const;
00082 SimulationControl* findSimulationControl(const std::string& name) const;
00083 double getSimulationResponseValueAsDouble(const std::string& name) const;
00084 double getSimulationControlValueAsDouble(const std::string& name) const;
00085 double stringToDoubleOrWarn(const std::string& sourceType, const std::string& symbolName, const
 std::string& valueText) const;
00086 void traceWarning(const std::string& message) const;
00087
00088 private:
00089 /*GenesysKernel::*Model* _model = nullptr;
00090 Sampler_if* _sampler = nullptr;
00091 std::map<std::string, std::list<std::string>*>* _referedDataElements = new std::map<std::string,
 std::list<std::string>*>(); // maps each dataelement class referenced to a list of referenced names
00092 bool _isRegisterReferedDataElements = false;
00093 private:
00094 double result = 0;
00095 std::string file;
00096 std::string str_to_parse;
00097 bool throwsException = true;
00098 std::string errorMessage;
00099 };
00100 */
00101 #endif // ! GENESYSPP_DRIVER_H

```

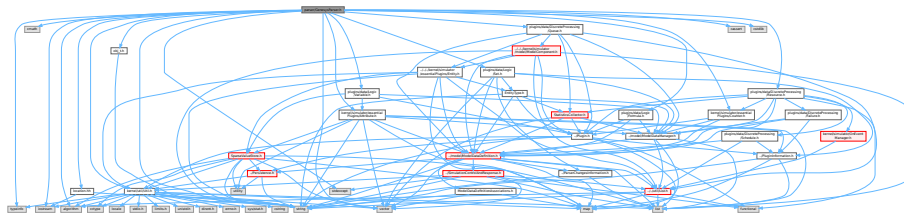
## 16.338 parser/GenesysParser.h File Reference

```

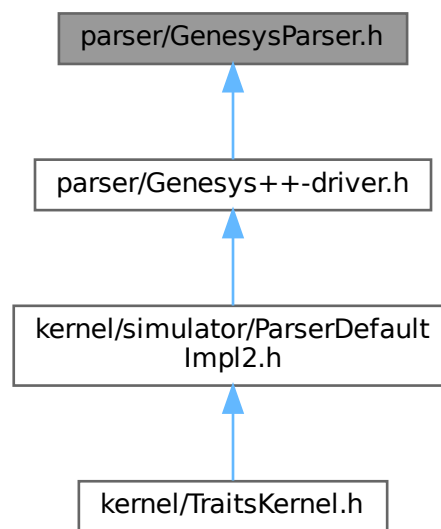
#include <string>
#include <cmath>
#include <algorithm>
#include "obj_t.h"
#include "kernel/util/Util.h"
#include "kernel/simulator/essentialPlugins/Attribute.h"
#include "kernel/simulator/essentialPlugins/Counter.h"
#include "plugins/data/Logic/Variable.h"
#include "plugins/data/DiscreteProcessing/Queue.h"
#include "plugins/data/Logic/Formula.h"
#include "plugins/data/DiscreteProcessing/Resource.h"
#include "plugins/data/Logic/Set.h"
#include <cassert>
#include <cstdlib>
#include <iostream>
#include <stdexcept>
#include <vector>
#include "location.hh"
#include <typeinfo>

```

Include dependency graph for GenesysParser.h:



This graph shows which files directly or indirectly include this file:



## Classes

- class [yy::genesyspp\\_parser](#)  
*A Bison parser.*
- class [yy::genesyspp\\_parser::value\\_type](#)
- struct [yy::genesyspp\\_parser::syntax\\_error](#)  
*Syntax errors thrown from user actions.*
- struct [yy::genesyspp\\_parser::token](#)  
*Token kinds.*
- struct [yy::genesyspp\\_parser::symbol\\_kind](#)  
*Symbol kinds.*
- struct [yy::genesyspp\\_parser::basic\\_symbol< Base >](#)
- struct [yy::genesyspp\\_parser::by\\_kind](#)  
*Type access provider for token (enum) based symbols.*
- struct [yy::genesyspp\\_parser::symbol\\_type](#)  
*"External" symbols: returned by the scanner.*
- class [yy::genesyspp\\_parser::context](#)
- class [yy::genesyspp\\_parser::stack< T, S >::slice](#)  
*Present a slice of the top of a stack.*

## Macros

- `#define YY_CPLUSPLUS 199711L`
- `#define YY_MOVE`
- `#define YY_MOVE_OR_COPY copy`
- `#define YY_MOVE_REF(Type) Type&`
- `#define YY_RVREF(Type) const Type&`
- `#define YY_COPY(Type) const Type&`
- `#define YY_NOEXCEPT`
- `#define YY_NOTHROW throw ()`
- `#define YY_CONSTEXPR`
- `#define YY_ASSERT assert`
- `#define YY_ATTRIBUTE_PURE`
- `#define YY_ATTRIBUTE_UNUSED`
- `#define YY_USE(E) ((void) (E))`
- `#define YY_INITIAL_VALUE(Value) Value`
- `#define YY_IGNORE_MAYBE_UNINITIALIZED_BEGIN`
- `#define YY_IGNORE_MAYBE_UNINITIALIZED_END`
- `#define YY_IGNORE_USELESS_CAST_BEGIN`
- `#define YY_IGNORE_USELESS_CAST_END`
- `#define YY_CAST(Type, Val) ((Type) (Val))`
- `#define YY_REINTERPRET_CAST(Type, Val) ((Type) (Val))`
- `#define YYDEBUG 1`

### 16.338.1 Detailed Description

Define the `yy::parser` class.

## 16.339 GenesysParser.h

[Go to the documentation of this file.](#)

```
00001 // A Bison parser, made by GNU Bison 3.8.2.
00002
00003 // Skeleton interface for Bison LALR(1) parsers in C++
00004
00005 // Copyright (C) 2002-2015, 2018-2021 Free Software Foundation, Inc.
00006
00007 // This program is free software: you can redistribute it and/or modify
00008 // it under the terms of the GNU General Public License as published by
00009 // the Free Software Foundation, either version 3 of the License, or
00010 // (at your option) any later version.
00011
00012 // This program is distributed in the hope that it will be useful,
00013 // but WITHOUT ANY WARRANTY; without even the implied warranty of
00014 // MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00015 // GNU General Public License for more details.
00016
00017 // You should have received a copy of the GNU General Public License
00018 // along with this program. If not, see <https://www.gnu.org/licenses/>.
00019
00020 // As a special exception, you may create a larger work that contains
00021 // part or all of the Bison parser skeleton and distribute that work
00022 // under terms of your choice, so long as that work isn't itself a
00023 // parser generator using the skeleton or a modified version thereof
00024 // as a parser skeleton. Alternatively, if you modify or redistribute
00025 // the parser skeleton itself, you may (at your option) remove this
00026 // special exception, which will cause the skeleton and the resulting
00027 // Bison output files to be licensed under the GNU General Public
00028 // License without this special exception.
00029
00030 // This special exception was added by the Free Software Foundation in
00031 // version 2.2 of Bison.
00032
00033
00039 // C++ LALR(1) parser skeleton written by Akim Demaille.
00040
00041 // DO NOT RELY ON FEATURES THAT ARE NOT DOCUMENTED in the manual,
00042 // especially those whose name start with YY_ or yy_. They are
00043 // private implementation details that can be changed or removed.
```

```

00044
00045 #ifndef YY_YG_GENESYSPARSER_H_INCLUDED
00046 # define YY_YG_GENESYSPARSER_H_INCLUDED
00047 // "%code requires" blocks.
00048 #line 9 "bisonparser.yy"
00049
00050 #include <string>
00051 #include <cmath>
00052 #include <algorithm>
00053 #include "obj_t.h"
00054 #include "kernel/util/Util.h"
00055 #include "kernel/simulator/essentialPlugins/Attribute.h"
00056 #include "kernel/simulator/essentialPlugins/Counter.h"
00057
00058 /****begin_Includes_plugins****/
00059
00061 #include "plugins/data/Logic/Variable.h"
00065 #include "plugins/data/DiscreteProcessing/Queue.h"
00069 #include "plugins/data/Logic/Formula.h"
00073 #include "plugins/data/DiscreteProcessing/Resource.h"
00077 #include "plugins/data/Logic/Set.h"
00080 /****end_Includes_plugins****/
00081
00082 #ifdef YYDEBUG
00083 yydebug = 1;
00084 #endif
00085
00086 class genesyspp_driver;
00087
00088 #line 89 "../GenesysParser.h"
00089
00090 # include <cassert>
00091 # include <cstdlib> // std::abort
00092 # include <iostream>
00093 # include <stdexcept>
00094 # include <string>
00095 # include <vector>
00096
00097 #if defined __cplusplus
00098 # define YY_CPLUSPLUS __cplusplus
00099 #else
00100 # define YY_CPLUSPLUS 199711L
00101 #endif
00102
00103 // Support move semantics when possible.
00104 #if 201103L <= YY_CPLUSPLUS
00105 # define YY_MOVE std::move
00106 # define YY_MOVE_OR_COPY move
00107 # define YY_MOVE_REF(Type) Type&&
00108 # define YY_RVREF(Type) Type&&
00109 # define YY_COPY(Type) Type
00110 #else
00111 # define YY_MOVE
00112 # define YY_MOVE_OR_COPY copy
00113 # define YY_MOVE_REF(Type) Type&
00114 # define YY_RVREF(Type) const Type&
00115 # define YY_COPY(Type) const Type&
00116 #endif
00117
00118 // Support noexcept when possible.
00119 #if 201103L <= YY_CPLUSPLUS
00120 # define YY_NOEXCEPT noexcept
00121 # define YY_NOTHROW
00122 #else
00123 # define YY_NOEXCEPT
00124 # define YY_NOTHROW throw ()
00125 #endif
00126
00127 // Support constexpr when possible.
00128 #if 201703 <= YY_CPLUSPLUS
00129 # define YY_CONSTEXPR constexpr
00130 #else
00131 # define YY_CONSTEXPR
00132 #endif
00133 # include "location.hh"
00134 #include <typeinfo>
00135 #ifndef YY_ASSERT
00136 # include <cassert>
00137 # define YY_ASSERT assert
00138 #endif
00139
00140
00141 #ifndef YY_ATTRIBUTE_PURE
00142 # if defined __GNUC__ && 2 < __GNUC__ + (96 <= __GNUC_MINOR__)
00143 # define YY_ATTRIBUTE_PURE __attribute__((__pure__))
00144 # else
00145 # define YY_ATTRIBUTE_PURE

```

```

00146 # endif
00147 #endif
00148
00149 #ifndef YY_ATTRIBUTE_UNUSED
00150 # if defined __GNUC__ && 2 < __GNUC__ + (7 <= __GNUC_MINOR__)
00151 # define YY_ATTRIBUTE_UNUSED __attribute__((__unused__))
00152 # else
00153 # define YY_ATTRIBUTE_UNUSED
00154 # endif
00155 #endif
00156
00157 /* Suppress unused-variable warnings by "using" E. */
00158 #if ! defined lint || defined __GNUC__
00159 # define YY_USE(E) ((void) (E))
00160 #else
00161 # define YY_USE(E) /* empty */
00162 #endif
00163
00164 /* Suppress an incorrect diagnostic about yylval being uninitialized. */
00165 #if defined __GNUC__ && ! defined __ICC && 406 <= __GNUC__ * 100 + __GNUC_MINOR__
00166 # if __GNUC__ * 100 + __GNUC_MINOR__ < 407
00167 # define YY_IGNORE_MAYBE_UNINITIALIZED_BEGIN \
00168 _Pragma ("GCC diagnostic push") \
00169 _Pragma ("GCC diagnostic ignored \\"-Wuninitialized\\")
00170 # else
00171 # define YY_IGNORE_MAYBE_UNINITIALIZED_BEGIN \
00172 _Pragma ("GCC diagnostic push") \
00173 _Pragma ("GCC diagnostic ignored \\"-Wuninitialized\\") \
00174 _Pragma ("GCC diagnostic ignored \\"-Wmaybe-uninitialized\\")
00175 # endif
00176 # define YY_IGNORE_MAYBE_UNINITIALIZED_END \
00177 _Pragma ("GCC diagnostic pop")
00178 #else
00179 # define YY_INITIAL_VALUE(Value) Value
00180 #endif
00181 #ifndef YY_IGNORE_MAYBE_UNINITIALIZED_BEGIN
00182 # define YY_IGNORE_MAYBE_UNINITIALIZED_BEGIN
00183 # define YY_IGNORE_MAYBE_UNINITIALIZED_END
00184 #endif
00185 #ifndef YY_INITIAL_VALUE
00186 # define YY_INITIAL_VALUE(Value) /* Nothing. */
00187 #endif
00188
00189 #if defined __cplusplus && defined __GNUC__ && ! defined __ICC && 6 <= __GNUC__
00190 # define YY_IGNORE_USELESS_CAST_BEGIN \
00191 _Pragma ("GCC diagnostic push") \
00192 _Pragma ("GCC diagnostic ignored \\"-Wuseless-cast\\")
00193 # define YY_IGNORE_USELESS_CAST_END \
00194 _Pragma ("GCC diagnostic pop")
00195 #endif
00196 #ifndef YY_IGNORE_USELESS_CAST_BEGIN
00197 # define YY_IGNORE_USELESS_CAST_BEGIN
00198 # define YY_IGNORE_USELESS_CAST_END
00199 #endif
00200
00201 #ifndef YY_CAST
00202 # ifdef __cplusplus
00203 # define YY_CAST(Type, Val) static_cast<Type> (Val)
00204 # define YY_REINTERPRET_CAST(Type, Val) reinterpret_cast<Type> (Val)
00205 # else
00206 # define YY_CAST(Type, Val) ((Type) (Val))
00207 # define YY_REINTERPRET_CAST(Type, Val) ((Type) (Val))
00208 # endif
00209 #endif
00210 #ifndef YY_NULLPTR
00211 # if defined __cplusplus
00212 # if 201103L <= __cplusplus
00213 # define YY_NULLPTR nullptr
00214 # else
00215 # define YY_NULLPTR 0
00216 # endif
00217 # else
00218 # define YY_NULLPTR ((void*)0)
00219 # endif
00220 #endif
00221
00222 /* Debug traces. */
00223 #ifndef YYDEBUG
00224 # define YYDEBUG 1
00225 #endif
00226
00227 namespace yy {
00228 #line 229 "../GenesysParser.h"
00229
00230
00231
00232

```



```

00234 class genesyspp_parser
00235 {
00236 public:
00237 #ifndef YYSTYPE
00238 # ifdef __GNUC__
00239 # pragma GCC message "bison: do not #define YYSTYPE in C++, use %define api.value.type"
00240 # endif
00241 typedef YYSTYPE value_type;
00242 #else
00243 class value_type
00244 {
00245 public:
00246 typedef value_type self_type;
00247
00248 value_type () YY_NOEXCEPT
00249 : yyraw_ ()
00250 , yytypeid_ (YY_NULLPTR)
00251 {}
00252
00253 template <typename T>
00254 value_type (YY_RVREF (T) t)
00255 : yytypeid_ (&typeid (T))
00256 {
00257 YY_ASSERT (sizeof (T) <= size);
00258 new (yyas_<T> ()) T (YY_MOVE (t));
00259 }
00260
00261 #if 201103L <= YY_CPLUSPLUS
00262 value_type (const self_type&) = delete;
00263 self_type& operator= (const self_type&) = delete;
00264 #endif
00265
00266 ~value_type () YY_NOEXCEPT
00267 {
00268 YY_ASSERT (!yytypeid_);
00269 }
00270
00271 # if 201103L <= YY_CPLUSPLUS
00272 template <typename T, typename... U>
00273 T&
00274 emplace (U&&... u)
00275 {
00276 YY_ASSERT (!yytypeid_);
00277 YY_ASSERT (sizeof (T) <= size);
00278 yytypeid_ = & typeid (T);
00279 return *new (yyas_<T> ()) T (std::forward <U>(u)...);
00280 }
00281 # else
00282 template <typename T>
00283 T&
00284 emplace ()
00285 {
00286 YY_ASSERT (!yytypeid_);
00287 YY_ASSERT (sizeof (T) <= size);
00288 yytypeid_ = & typeid (T);
00289 return *new (yyas_<T> ()) T ();
00290 }
00291
00292 template <typename T>
00293 T&
00294 emplace (const T& t)
00295 {
00296 YY_ASSERT (!yytypeid_);
00297 YY_ASSERT (sizeof (T) <= size);
00298 yytypeid_ = & typeid (T);
00299 return *new (yyas_<T> ()) T (t);
00300 }
00301 # endif
00302
00303 template <typename T>
00304 T&
00305 build ()
00306 {
00307 return emplace<T> ();
00308 }
00309
00310 template <typename T>
00311 T&
00312 build (const T& t)
00313 {
00314 return emplace<T> (t);
00315 }
00316
00317 template <typename T>
00318 T&
00319 as () YY_NOEXCEPT
00320 {

```

```

00340 YY_ASSERT (yytypeid_);
00341 YY_ASSERT (*yytypeid_ == typeid (T));
00342 YY_ASSERT (sizeof (T) <= size);
00343 return *yyas_<T> ();
00344 }
00345
00346 template <typename T>
00347 const T&
00348 as () const YY_NOEXCEPT
00349 {
00350 YY_ASSERT (yytypeid_);
00351 YY_ASSERT (*yytypeid_ == typeid (T));
00352 YY_ASSERT (sizeof (T) <= size);
00353 return *yyas_<T> ();
00354 }
00355
00356 template <typename T>
00357 void
00358 swap (self_type& that) YY_NOEXCEPT
00359 {
00360 YY_ASSERT (yytypeid_);
00361 YY_ASSERT (*yytypeid_ == *that.yytypeid_);
00362 std::swap (as<T> (), that.as<T> ());
00363 }
00364
00365 template <typename T>
00366 void
00367 move (self_type& that)
00368 {
00369 # if 201103L <= YY_CPLUSPLUS
00370 emplace<T> (std::move (that.as<T> ()));
00371 # else
00372 emplace<T> ();
00373 swap<T> (that);
00374 # endif
00375 that.destroy<T> ();
00376 }
00377
00378 # if 201103L <= YY_CPLUSPLUS
00379 template <typename T>
00380 void
00381 move (self_type&& that)
00382 {
00383 emplace<T> (std::move (that.as<T> ()));
00384 that.destroy<T> ();
00385 }
00386 #endif
00387
00388 template <typename T>
00389 void
00390 copy (const self_type& that)
00391 {
00392 emplace<T> (that.as<T> ());
00393 }
00394
00395 template <typename T>
00396 void
00397 destroy ()
00398 {
00399 as<T> ().~T ();
00400 yytypeid_ = YY_NULLPTR;
00401 }
00402
00403 private:
00404 #if YY_CPLUSPLUS < 201103L
00405 value_type (const self_type&);
00406 self_type& operator= (const self_type&);
00407 #endif
00408
00409 template <typename T>
00410 T*
00411 yyas_ () YY_NOEXCEPT
00412 {
00413 void *yyp = yyraw_;
00414 return static_cast<T*> (yyp);
00415 }
00416
00417 template <typename T>
00418 const T*
00419 yyas_ () const YY_NOEXCEPT
00420 {
00421 const void *yyp = yyraw_;
00422 return static_cast<const T*> (yyp);
00423 }
00424
00425 union union_type
00426 {

```

```
00447 // NUMD
00448 // NUMH
00449 // CTEZERO
00450 // oLE
00451 // oGE
00452 // oEQ
00453 // oNE
00454 // oAND
00455 // oOR
00456 // oNAND
00457 // oXOR
00458 // oNOT
00459 // fSIN
00460 // fCOS
00461 // fROUND
00462 // fMOD
00463 // fTRUNC
00464 // fFRAC
00465 // fEXP
00466 // fSQRT
00467 // fLOG
00468 // fLN
00469 // mathMIN
00470 // mathMAX
00471 // fVAL
00472 // fEVAL
00473 // fLENG
00474 // fRND1
00475 // fEXPO
00476 // fNORM
00477 // fUNIF
00478 // fWEIB
00479 // fLOGN
00480 // fGAMM
00481 // fERLA
00482 // fTRIA
00483 // fBETA
00484 // fDISC
00485 // fTNOW
00486 // fTFIN
00487 // fMAXREP
00488 // fNUMREP
00489 // fIDENT
00490 // simulEntitiesWIP
00491 // cIF
00492 // cELSE
00493 // cFOR
00494 // cTO
00495 // cDO
00496 // ATTRIB
00497 // CSTAT
00498 // COUNTER
00499 // SIMRESP
00500 // SIMCTRL
00501 // fTAVG
00502 // fCOUNT
00503 // ILLEGAL
00504 // RESOURCE
00505 // fNR
00506 // fMR
00507 // fIRF
00508 // fRESSEIZES
00509 // fSTATE
00510 // fSETSUM
00511 // fRESUTIL
00512 // QUEUE
00513 // fNQ
00514 // fFIRSTINQ
00515 // fLASTINQ
00516 // fSAQUE
00517 // fAQUE
00518 // fENTATRANK
00519 // SET
00520 // fNUMSET
00521 // VARI
00522 // FORM
00523 // fNUMGR
00524 // fATRGR
00525 // input
00526 // expression
00527 // logicalOr
00528 // logicalXor
00529 // logicalAnd
00530 // logicalNot
00531 // relational
00532 // additive
00533 // multiplicative
```

```

00534 // power
00535 // unary
00536 // primary
00537 // number
00538 // command
00539 // commandIF
00540 // commandFOR
00541 // function
00542 // kernelFunction
00543 // elementFunction
00544 // trigonFunction
00545 // mathFunction
00546 // probFunction
00547 // userFunction
00548 // listaparm
00549 // illegal
00550 // attribute
00551 // simulationResponse
00552 // simulationControl
00553 // variable
00554 // formula
00555 // assignment
00556 // pluginFunction
00557 char dummy1[sizeof (obj_t)];
00558
00559 // indexList
00560 char dummy2[sizeof (std::string)];
00561 };
00562
00564 enum { size = sizeof (union_type) };
00565
00567 union
00568 {
00570 long double yyalgn_me_;
00572 char yyraw_[size];
00573 };
00574
00576 const std::type_info *yytypeid;
00577 };
00578
00579 #endif
00581 typedef value_type semantic_type;
00582
00584 typedef location location_type;
00585
00587 struct syntax_error : std::runtime_error
00588 {
00589 syntax_error (const location_type& l, const std::string& m)
00590 : std::runtime_error (m)
00591 , location (l)
00592 {}
00593
00594 syntax_error (const syntax_error& s)
00595 : std::runtime_error (s.what ())
00596 , location (s.location)
00597 {}
00598
00599 ~syntax_error () YY_NOEXCEPT YY_NOTHROW;
00600
00601 location_type location;
00602 };
00603
00605 struct token
00606 {
00607 enum token_kind_type
00608 {
00609 YYEMPTY = -2,
00610 END = 0, // "end of file"
00611 YYerror = 256, // error
00612 YYUNDEF = 257, // "invalid token"
00613 NUMD = 258, // NUMD
00614 NUMH = 259, // NUMH
00615 CTEZERO = 260, // CTEZERO
00616 oLE = 261, // oLE
00617 oGE = 262, // oGE
00618 oEQ = 263, // oEQ
00619 oNE = 264, // oNE
00620 oAND = 265, // oAND
00621 oOR = 266, // oOR
00622 oNAND = 267, // oNAND
00623 oXOR = 268, // oXOR
00624 oNOT = 269, // oNOT
00625 fSIN = 270, // fSIN
00626 fCOS = 271, // fCOS
00627 fROUND = 272, // fROUND
00628 fMOD = 273, // fMOD
00629 fTRUNC = 274, // fTRUNC

```

```

00630 fFRAC = 275, // fFRAC
00631 fEXP = 276, // fEXP
00632 fSQRT = 277, // fSQRT
00633 fLOG = 278, // fLOG
00634 fLN = 279, // fLN
00635 mathMIN = 280, // mathMIN
00636 mathMAX = 281, // mathMAX
00637 fVAL = 282, // fVAL
00638 fEVAL = 283, // fEVAL
00639 fLENG = 284, // fLENG
00640 fRND1 = 285, // fRND1
00641 fEXPO = 286, // fEXPO
00642 fNORM = 287, // fNORM
00643 fUNIF = 288, // fUNIF
00644 fWEIB = 289, // fWEIB
00645 fLOGN = 290, // fLOGN
00646 fGAMM = 291, // fGAMM
00647 fERLA = 292, // fERLA
00648 fTRIA = 293, // fTRIA
00649 fBETA = 294, // fBETA
00650 fDISC = 295, // fDISC
00651 fTNOW = 296, // fTNOW
00652 fTFIN = 297, // fTFIN
00653 fMAXREP = 298, // fMAXREP
00654 fNUMREP = 299, // fNUMREP
00655 fIDENT = 300, // fIDENT
00656 simulEntitiesWIP = 301, // simulEntitiesWIP
00657 cIF = 302, // cIF
00658 cELSE = 303, // cELSE
00659 cFOR = 304, // cFOR
00660 cTO = 305, // cTO
00661 cDO = 306, // cDO
00662 ATRIB = 307, // ATRIB
00663 CSTAT = 308, // CSTAT
00664 COUNTER = 309, // COUNTER
00665 SIMRESP = 310, // SIMRESP
00666 SIMCTRL = 311, // SIMCTRL
00667 fTAVG = 312, // fTAVG
00668 fCOUNT = 313, // fCOUNT
00669 ILLEGAL = 314, // ILLEGAL
00670 RESOURCE = 315, // RESOURCE
00671 fNR = 316, // fNR
00672 fMR = 317, // fMR
00673 fIRF = 318, // fIRF
00674 fRESSEIZES = 319, // fRESSEIZES
00675 fSTATE = 320, // fSTATE
00676 fSETSUM = 321, // fSETSUM
00677 fRESUTIL = 322, // fRESUTIL
00678 QUEUE = 323, // QUEUE
00679 fNQ = 324, // fNQ
00680 fFIRSTINQ = 325, // fFIRSTINQ
00681 fLASTINQ = 326, // fLASTINQ
00682 fSAQUE = 327, // fSAQUE
00683 fAQUE = 328, // fAQUE
00684 fENTATRANK = 329, // fENTATRANK
00685 SET = 330, // SET
00686 fNUMSET = 331, // fNUMSET
00687 VARI = 332, // VARI
00688 FORM = 333, // FORM
00689 fNUMGR = 334, // fNUMGR
00690 fATRGR = 335, // fATRGR
00691 LPAREN = 336, // "("
00692 RPAREN = 337, // ")"
00693 LBRACKET = 338, // "["
00694 RBRACKET = 339, // "]"
00695 PLUS = 340, // "+"
00696 MINUS = 341, // "-"
00697 STAR = 342, // "*"
00698 POWER = 343, // "^"
00699 SLASH = 344, // "/"
00700 LESS = 345, // "<"
00701 GREATER = 346, // ">"
00702 ASSIGN = 347, // "="
00703 COMMA = 348, // ","
00704 };
00705 typedef token_kind_type yytokentype;
00706 };
00707
00708 typedef token::token_kind_type token_kind_type;
00709
00710 typedef token_kind_type token_type;
00711
00712 struct symbol_kind
00713 {
00714 enum symbol_kind_type
00715 {
00716 YYNTOKENS = 95,

```

```

00721 S_YYEMPTY = -2,
00722 S_YYEOF = 0, // "end of file"
00723 S_YYerror = 1, // error
00724 S_YYUNDEF = 2, // "invalid token"
00725 S_NUMD = 3, // NUMD
00726 S_NUMH = 4, // NUMH
00727 S_CTEZERO = 5, // CTEZERO
00728 S_oLE = 6, // oLE
00729 S_oGE = 7, // oGE
00730 S_oEQ = 8, // oEQ
00731 S_oNE = 9, // oNE
00732 S_oAND = 10, // oAND
00733 S_oOR = 11, // oOR
00734 S_oNAND = 12, // oNAND
00735 S_oXOR = 13, // oXOR
00736 S_oNOT = 14, // oNOT
00737 S_fSIN = 15, // fSIN
00738 S_fCOS = 16, // fCOS
00739 S_fROUND = 17, // fROUND
00740 S_fMOD = 18, // fMOD
00741 S_fTRUNC = 19, // fTRUNC
00742 S_fFRAC = 20, // fFRAC
00743 S_fEXP = 21, // fEXP
00744 S_fSQRT = 22, // fSQRT
00745 S_fLOG = 23, // fLOG
00746 S_fLN = 24, // fLN
00747 S_mathMIN = 25, // mathMIN
00748 S_mathMAX = 26, // mathMAX
00749 S_fVAL = 27, // fVAL
00750 S_fEVAL = 28, // fEVAL
00751 S_fLENG = 29, // fLENG
00752 S_fRND1 = 30, // fRND1
00753 S_fEXPO = 31, // fEXPO
00754 S_fNORM = 32, // fNORM
00755 S_fUNIF = 33, // fUNIF
00756 S_fWEIB = 34, // fWEIB
00757 S_fLOGN = 35, // fLOGN
00758 S_fGAMM = 36, // fGAMM
00759 S_fERLA = 37, // fERLA
00760 S_fTRIA = 38, // fTRIA
00761 S_fBETA = 39, // fBETA
00762 S_fDISC = 40, // fDISC
00763 S_fTNOW = 41, // fTNOW
00764 S_fTFIN = 42, // fTFIN
00765 S_fMAXREP = 43, // fMAXREP
00766 S_fNUMREP = 44, // fNUMREP
00767 S_fIDENT = 45, // fIDENT
00768 S_simulEntitiesWIP = 46, // simulEntitiesWIP
00769 S_cIF = 47, // cIF
00770 S_cELSE = 48, // cELSE
00771 S_cFOR = 49, // cFOR
00772 S_cTO = 50, // cTO
00773 S_cDO = 51, // cDO
00774 S_ATTRIB = 52, // ATTRIB
00775 S_CSTAT = 53, // CSTAT
00776 S_COUNTER = 54, // COUNTER
00777 S_SIMRESP = 55, // SIMRESP
00778 S_SIMCTRL = 56, // SIMCTRL
00779 S_fTAVG = 57, // fTAVG
00780 S_fCOUNT = 58, // fCOUNT
00781 S_ILLEGAL = 59, // ILLEGAL
00782 S_RESOURCE = 60, // RESOURCE
00783 S_fNR = 61, // fNR
00784 S_fMR = 62, // fMR
00785 S_fIRF = 63, // fIRF
00786 S_fRESSEIZES = 64, // fRESSEIZES
00787 S_fSTATE = 65, // fSTATE
00788 S_fSETSUM = 66, // fSETSUM
00789 S_fRESUTIL = 67, // fRESUTIL
00790 S_QUEUE = 68, // QUEUE
00791 S_fNQ = 69, // fNQ
00792 S_fFIRSTINQ = 70, // fFIRSTINQ
00793 S_fLASTINQ = 71, // fLASTINQ
00794 S_fSAQUE = 72, // fSAQUE
00795 S_fAQUE = 73, // fAQUE
00796 S_fENTATRANK = 74, // fENTATRANK
00797 S_SET = 75, // SET
00798 S_fNUMSET = 76, // fNUMSET
00799 S_VARI = 77, // VARI
00800 S_FORM = 78, // FORM
00801 S_fNUMGR = 79, // fNUMGR
00802 S_fATRGR = 80, // fATRGR
00803 S_LPAREN = 81, // "("
00804 S_RPAREN = 82, // ")"
00805 S_LBRACKET = 83, // "["
00806 S_RBRACKET = 84, // "]"
00807 S_PLUS = 85, // "+"

```

```

00808 S_MINUS = 86, // "-"
00809 S_STAR = 87, // "*"
00810 S_POWER = 88, // "^"
00811 S_SLASH = 89, // "/"
00812 S_LESS = 90, // "<"
00813 S_GREATER = 91, // ">"
00814 S_ASSIGN = 92, // "="
00815 S_COMMA = 93, // ","
00816 S_94_USER_ = 94, // "USER"
00817 S_YYACCEPT = 95, // $accept
00818 S_input = 96, // input
00819 S_expression = 97, // expression
00820 S_logicalOr = 98, // logicalOr
00821 S_logicalXor = 99, // logicalXor
00822 S_logicalAnd = 100, // logicalAnd
00823 S_logicalNot = 101, // logicalNot
00824 S_relational = 102, // relational
00825 S_additive = 103, // additive
00826 S_multiplicative = 104, // multiplicative
00827 S_power = 105, // power
00828 S_unary = 106, // unary
00829 S_primary = 107, // primary
00830 S_number = 108, // number
00831 S_command = 109, // command
00832 S_commandIF = 110, // commandIF
00833 S_commandFOR = 111, // commandFOR
00834 S_function = 112, // function
00835 S_kernelFunction = 113, // kernelFunction
00836 S_elementFunction = 114, // elementFunction
00837 S_trigonFunction = 115, // trigonFunction
00838 S_mathFunction = 116, // mathFunction
00839 S_probFunction = 117, // probFunction
00840 S_userFunction = 118, // userFunction
00841 S_listaparm = 119, // listaparm
00842 S_indexList = 120, // indexList
00843 S_illegal = 121, // illegal
00844 S_attribute = 122, // attribute
00845 S_simulationResponse = 123, // simulationResponse
00846 S_simulationControl = 124, // simulationControl
00847 S_variable = 125, // variable
00848 S_formula = 126, // formula
00849 S_assignment = 127, // assignment
00850 S_pluginFunction = 128 // pluginFunction
00851 };
00852 };
00853
00854 typedef symbol_kind::symbol_kind_type symbol_kind_type;
00855
00856 static const symbol_kind_type YYNTOKENS = symbol_kind::YYNTOKENS;
00857
00858 template <typename Base>
00859 struct basic_symbol : Base
00860 {
00861 typedef Base super_type;
00862
00863 basic_symbol () YY_NOEXCEPT
00864 : value ()
00865 , location ()
00866 {}
00867
00868 #if 201103L <= YY_CPLUSPLUS
00869 basic_symbol (basic_symbol&& that)
00870 : Base (std::move (that))
00871 , value ()
00872 , location (std::move (that.location))
00873 {
00874 switch (this->kind ())
00875 {
00876 case symbol_kind::S_NUMD: // NUMD
00877 case symbol_kind::S_NUMH: // NUMH
00878 case symbol_kind::S_CTEZERO: // CTEZERO
00879 case symbol_kind::S_oLE: // oLE
00880 case symbol_kind::S_oGE: // oGE
00881 case symbol_kind::S_oEQ: // oEQ
00882 case symbol_kind::S_oNE: // oNE
00883 case symbol_kind::S_oAND: // oAND
00884 case symbol_kind::S_oOR: // oOR
00885 case symbol_kind::S_oNAND: // oNAND
00886 case symbol_kind::S_oXOR: // oXOR
00887 case symbol_kind::S_oNOT: // oNOT
00888 case symbol_kind::S_fsIN: // fsIN
00889 case symbol_kind::S_fcOS: // fcOS
00890 case symbol_kind::S_fROUND: // fROUND
00891 case symbol_kind::S_fmOD: // fmOD
00892 case symbol_kind::S_fTRUNC: // fTRUNC
00893 case symbol_kind::S_fFRAC: // fFRAC
00894 case symbol_kind::S_fEXP: // fEXP

```

```

00906 case symbol_kind::S_fSQRT: // fSQRT
00907 case symbol_kind::S_fLOG: // fLOG
00908 case symbol_kind::S_fLN: // fLN
00909 case symbol_kind::S_mathMIN: // mathMIN
00910 case symbol_kind::S_mathMAX: // mathMAX
00911 case symbol_kind::S_fVAL: // fVAL
00912 case symbol_kind::S_fEVAL: // fEVAL
00913 case symbol_kind::S_fLENG: // fLENG
00914 case symbol_kind::S_frND1: // frND1
00915 case symbol_kind::S_fEXPO: // fEXPO
00916 case symbol_kind::S_fNORM: // fNORM
00917 case symbol_kind::S_fUNIF: // fUNIF
00918 case symbol_kind::S_fWEIB: // fWEIB
00919 case symbol_kind::S_fLOGN: // fLOGN
00920 case symbol_kind::S_fGAMM: // fGAMM
00921 case symbol_kind::S_fERLA: // fERLA
00922 case symbol_kind::S_fTRIA: // fTRIA
00923 case symbol_kind::S_fBETA: // fBETA
00924 case symbol_kind::S_fDISC: // fDISC
00925 case symbol_kind::S_ftNOW: // ftNOW
00926 case symbol_kind::S_ftFIN: // ftFIN
00927 case symbol_kind::S_fMAXREP: // fMAXREP
00928 case symbol_kind::S_fNUMREP: // fNUMREP
00929 case symbol_kind::S_fIDENT: // fIDENT
00930 case symbol_kind::S_simulEntitiesWIP: // simulEntitiesWIP
00931 case symbol_kind::S_cIF: // cIF
00932 case symbol_kind::S_cELSE: // cELSE
00933 case symbol_kind::S_cFOR: // cFOR
00934 case symbol_kind::S_cTO: // cTO
00935 case symbol_kind::S_cDO: // cDO
00936 case symbol_kind::S_ATTRIB: // ATTRIB
00937 case symbol_kind::S_CSTAT: // CSTAT
00938 case symbol_kind::S_COUNTER: // COUNTER
00939 case symbol_kind::S_SIMRESP: // SIMRESP
00940 case symbol_kind::S_SIMCTRL: // SIMCTRL
00941 case symbol_kind::S_ftAVG: // ftAVG
00942 case symbol_kind::S_fcOUNT: // fcOUNT
00943 case symbol_kind::S_ILLEGAL: // ILLEGAL
00944 case symbol_kind::S_RESOURCE: // RESOURCE
00945 case symbol_kind::S_fNR: // fNR
00946 case symbol_kind::S_fMR: // fMR
00947 case symbol_kind::S_fIRF: // fIRF
00948 case symbol_kind::S_fRESSEIZES: // fRESSEIZES
00949 case symbol_kind::S_fSTATE: // fSTATE
00950 case symbol_kind::S_fSETSUM: // fSETSUM
00951 case symbol_kind::S_fRESUTIL: // fRESUTIL
00952 case symbol_kind::S_QUEUE: // QUEUE
00953 case symbol_kind::S_fNQ: // fNQ
00954 case symbol_kind::S_ffIRSTINQ: // ffIRSTINQ
00955 case symbol_kind::S_fLASTINQ: // fLASTINQ
00956 case symbol_kind::S_fSAQUE: // fSAQUE
00957 case symbol_kind::S_fAQUE: // fAQUE
00958 case symbol_kind::S_fENTATRANK: // fENTATRANK
00959 case symbol_kind::S_SET: // SET
00960 case symbol_kind::S_fNUMSET: // fNUMSET
00961 case symbol_kind::S_VARI: // VARI
00962 case symbol_kind::S_FORM: // FORM
00963 case symbol_kind::S_fNUMGR: // fNUMGR
00964 case symbol_kind::S_fATRGR: // fATRGR
00965 case symbol_kind::S_input: // input
00966 case symbol_kind::S_expression: // expression
00967 case symbol_kind::S_logicalOr: // logicalOr
00968 case symbol_kind::S_logicalXor: // logicalXor
00969 case symbol_kind::S_logicalAnd: // logicalAnd
00970 case symbol_kind::S_logicalNot: // logicalNot
00971 case symbol_kind::S_relational: // relational
00972 case symbol_kind::S_additive: // additive
00973 case symbol_kind::S_multiplicative: // multiplicative
00974 case symbol_kind::S_power: // power
00975 case symbol_kind::S_unary: // unary
00976 case symbol_kind::S_primary: // primary
00977 case symbol_kind::S_number: // number
00978 case symbol_kind::S_command: // command
00979 case symbol_kind::S_commandIF: // commandIF
00980 case symbol_kind::S_commandFOR: // commandFOR
00981 case symbol_kind::S_function: // function
00982 case symbol_kind::S_kernelFunction: // kernelFunction
00983 case symbol_kind::S_elementFunction: // elementFunction
00984 case symbol_kind::S_trigonFunction: // trigonFunction
00985 case symbol_kind::S_mathFunction: // mathFunction
00986 case symbol_kind::S_probFunction: // probFunction
00987 case symbol_kind::S_userFunction: // userFunction
00988 case symbol_kind::S_listaparm: // listaparm
00989 case symbol_kind::S_illegal: // illegal
00990 case symbol_kind::S_attribute: // attribute
00991 case symbol_kind::S_simulationResponse: // simulationResponse
00992 case symbol_kind::S_simulationControl: // simulationControl

```



```

00993 case symbol_kind::S_variable: // variable
00994 case symbol_kind::S_formula: // formula
00995 case symbol_kind::S_assignment: // assignment
00996 case symbol_kind::S_pluginFunction: // pluginFunction
00997 value.move< obj_t > (std::move (that.value));
00998 break;
00999
01000 case symbol_kind::S_indexList: // indexList
01001 value.move< std::string > (std::move (that.value));
01002 break;
01003
01004 default:
01005 break;
01006 }
01007
01008 }
01009 #endif
01010
01011 basic_symbol (const basic_symbol& that);
01012
01013 #if 201103L <= YY_CPLUSPLUS
01014 basic_symbol (typename Base::kind_type t, location_type&& l)
01015 : Base (t)
01016 , location (std::move (l))
01017 {}
01018 #else
01019 basic_symbol (typename Base::kind_type t, const location_type& l)
01020 : Base (t)
01021 , location (l)
01022 {}
01023 #endif
01024
01025 #if 201103L <= YY_CPLUSPLUS
01026 basic_symbol (typename Base::kind_type t, obj_t&& v, location_type&& l)
01027 : Base (t)
01028 , value (std::move (v))
01029 , location (std::move (l))
01030 {}
01031 #else
01032 basic_symbol (typename Base::kind_type t, const obj_t& v, const location_type& l)
01033 : Base (t)
01034 , value (v)
01035 , location (l)
01036 {}
01037 #endif
01038
01039 #if 201103L <= YY_CPLUSPLUS
01040 basic_symbol (typename Base::kind_type t, std::string&& v, location_type&& l)
01041 : Base (t)
01042 , value (std::move (v))
01043 , location (std::move (l))
01044 {}
01045 #else
01046 basic_symbol (typename Base::kind_type t, const std::string& v, const location_type& l)
01047 : Base (t)
01048 , value (v)
01049 , location (l)
01050 {}
01051 #endif
01052
01053 ~basic_symbol ()
01054 {
01055 clear ();
01056 }
01057
01058 void clear () YY_NOEXCEPT
01059 {
01060 // User destructor.
01061 symbol_kind_type yykind = this->kind ();
01062 basic_symbol<Base>& yysym = *this;
01063 (void) yysym;
01064 switch (yykind)
01065 {
01066 default:
01067 break;
01068 }
01069
01070 // Value type destructor.
01071 switch (yykind)
01072 {
01073 case symbol_kind::S_NUMD: // NUMD
01074 case symbol_kind::S_NUMH: // NUMH
01075 case symbol_kind::S_CTEZERO: // CTEZERO
01076 case symbol_kind::S_oLE: // oLE
01077 case symbol_kind::S_oGE: // oGE

```

```
01084 case symbol_kind::S_oEQ: // oEQ
01085 case symbol_kind::S_oNE: // oNE
01086 case symbol_kind::S_oAND: // oAND
01087 case symbol_kind::S_oOR: // oOR
01088 case symbol_kind::S_oNAND: // oNAND
01089 case symbol_kind::S_oXOR: // oXOR
01090 case symbol_kind::S_oNOT: // oNOT
01091 case symbol_kind::S_fSIN: // fSIN
01092 case symbol_kind::S_fCOS: // fCOS
01093 case symbol_kind::S_fROUND: // fROUND
01094 case symbol_kind::S_fMOD: // fMOD
01095 case symbol_kind::S_fTRUNC: // fTRUNC
01096 case symbol_kind::S_fFRAC: // fFRAC
01097 case symbol_kind::S_fEXP: // fEXP
01098 case symbol_kind::S_fSQRT: // fSQRT
01099 case symbol_kind::S_fLOG: // fLOG
01100 case symbol_kind::S_fLN: // fLN
01101 case symbol_kind::S_mathMIN: // mathMIN
01102 case symbol_kind::S_mathMAX: // mathMAX
01103 case symbol_kind::S_fVAL: // fVAL
01104 case symbol_kind::S_fEVAL: // fEVAL
01105 case symbol_kind::S_fLENG: // fLENG
01106 case symbol_kind::S_fRND1: // fRND1
01107 case symbol_kind::S_fEXPO: // fEXPO
01108 case symbol_kind::S_fNORM: // fNORM
01109 case symbol_kind::S_fUNIF: // fUNIF
01110 case symbol_kind::S_fWEIB: // fWEIB
01111 case symbol_kind::S_fLOGN: // fLOGN
01112 case symbol_kind::S_fGAMM: // fGAMM
01113 case symbol_kind::S_fERLA: // fERLA
01114 case symbol_kind::S_fTRIA: // fTRIA
01115 case symbol_kind::S_fBETA: // fBETA
01116 case symbol_kind::S_fDISC: // fDISC
01117 case symbol_kind::S_fTNOW: // fTNOW
01118 case symbol_kind::S_fTFIN: // fTFIN
01119 case symbol_kind::S_fMAXREP: // fMAXREP
01120 case symbol_kind::S_fNUMREP: // fNUMREP
01121 case symbol_kind::S_fIDENT: // fIDENT
01122 case symbol_kind::S_simulEntitiesWIP: // simulEntitiesWIP
01123 case symbol_kind::S_cIF: // cIF
01124 case symbol_kind::S_cELSE: // cELSE
01125 case symbol_kind::S_cFOR: // cFOR
01126 case symbol_kind::S_cTO: // cTO
01127 case symbol_kind::S_cDO: // cDO
01128 case symbol_kind::S_ATTRIB: // ATTRIB
01129 case symbol_kind::S_CSTAT: // CSTAT
01130 case symbol_kind::S_COUNTER: // COUNTER
01131 case symbol_kind::S_SIMRESP: // SIMRESP
01132 case symbol_kind::S_SIMCTRL: // SIMCTRL
01133 case symbol_kind::S_fTAVG: // fTAVG
01134 case symbol_kind::S_fCOUNT: // fCOUNT
01135 case symbol_kind::S_ILLEGAL: // ILLEGAL
01136 case symbol_kind::S_RESOURCE: // RESOURCE
01137 case symbol_kind::S_fNR: // fNR
01138 case symbol_kind::S_fMR: // fMR
01139 case symbol_kind::S_fIRF: // fIRF
01140 case symbol_kind::S_fRESSEIZES: // fRESSEIZES
01141 case symbol_kind::S_fSTATE: // fSTATE
01142 case symbol_kind::S_fSETSUM: // fSETSUM
01143 case symbol_kind::S_fRESUTIL: // fRESUTIL
01144 case symbol_kind::S_QUEUE: // QUEUE
01145 case symbol_kind::S_fNQ: // fNQ
01146 case symbol_kind::S_fFIRSTINQ: // fFIRSTINQ
01147 case symbol_kind::S_fLASTINQ: // fLASTINQ
01148 case symbol_kind::S_fSAQUE: // fSAQUE
01149 case symbol_kind::S_fAQUE: // fAQUE
01150 case symbol_kind::S_fENTATRANK: // fENTATRANK
01151 case symbol_kind::S_SET: // SET
01152 case symbol_kind::S_fNUMSET: // fNUMSET
01153 case symbol_kind::S_VARI: // VARI
01154 case symbol_kind::S_FORM: // FORM
01155 case symbol_kind::S_fNUMGR: // fNUMGR
01156 case symbol_kind::S_fATRGR: // fATRGR
01157 case symbol_kind::S_input: // input
01158 case symbol_kind::S_expression: // expression
01159 case symbol_kind::S_logicalOr: // logicalOr
01160 case symbol_kind::S_logicalXor: // logicalXor
01161 case symbol_kind::S_logicalAnd: // logicalAnd
01162 case symbol_kind::S_logicalNot: // logicalNot
01163 case symbol_kind::S_relational: // relational
01164 case symbol_kind::S_additive: // additive
01165 case symbol_kind::S_multiplicative: // multiplicative
01166 case symbol_kind::S_power: // power
01167 case symbol_kind::S_unary: // unary
01168 case symbol_kind::S_primary: // primary
01169 case symbol_kind::S_number: // number
01170 case symbol_kind::S_command: // command
```

```

01171 case symbol_kind::S_commandIF: // commandIF
01172 case symbol_kind::S_commandFOR: // commandFOR
01173 case symbol_kind::S_function: // function
01174 case symbol_kind::S_kernelFunction: // kernelFunction
01175 case symbol_kind::S_elementFunction: // elementFunction
01176 case symbol_kind::S_trigonFunction: // trigonFunction
01177 case symbol_kind::S_mathFunction: // mathFunction
01178 case symbol_kind::S_probFunction: // probFunction
01179 case symbol_kind::S_userFunction: // userFunction
01180 case symbol_kind::S_listaparm: // listaparm
01181 case symbol_kind::S_illegal: // illegal
01182 case symbol_kind::S_attribute: // attribute
01183 case symbol_kind::S_simulationResponse: // simulationResponse
01184 case symbol_kind::S_simulationControl: // simulationControl
01185 case symbol_kind::S_variable: // variable
01186 case symbol_kind::S_formula: // formula
01187 case symbol_kind::S_assignment: // assignment
01188 case symbol_kind::S_pluginFunction: // pluginFunction
01189 value.template destroy< obj_t > ();
01190 break;
01191
01192 case symbol_kind::S_indexList: // indexList
01193 value.template destroy< std::string > ();
01194 break;
01195
01196 default:
01197 break;
01198 }
01199
01200 Base::clear ();
01201 }
01202
01204 std::string name () const YY_NOEXCEPT
01205 {
01206 return genesyspp_parser::symbol_name (this->kind ());
01207 }
01208
01210 symbol_kind_type type_get () const YY_NOEXCEPT;
01211
01213 bool empty () const YY_NOEXCEPT;
01214
01216 void move (basic_symbol& s);
01217
01219 value_type value;
01220
01222 location_type location;
01223
01224 private:
01225 #if YY_CPLUSPLUS < 201103L
01227 basic_symbol& operator= (const basic_symbol& that);
01228 #endif
01229 };
01230
01232 struct by_kind
01233 {
01235 typedef token_kind_type kind_type;
01236
01238 by_kind () YY_NOEXCEPT;
01239
01240 #if 201103L <= YY_CPLUSPLUS
01242 by_kind (by_kind&& that) YY_NOEXCEPT;
01243 #endif
01244
01246 by_kind (const by_kind& that) YY_NOEXCEPT;
01247
01249 by_kind (kind_type t) YY_NOEXCEPT;
01250
01251
01252 void clear () YY_NOEXCEPT;
01253
01255 void move (by_kind& that);
01256
01258 symbol_kind_type kind () const YY_NOEXCEPT;
01259
01261 symbol_kind_type type_get () const YY_NOEXCEPT;
01262
01264 symbol_kind_type kind_;
01265 };
01266
01268 typedef by_kind by_type;
01269
01271 struct symbol_type : basic_symbol<by_kind>
01272 {
01273 typedef basic_symbol<by_kind> super_type;
01274
01276 symbol_type () YY_NOEXCEPT {}

```

```

01282
01284 #if 201103L <= YY_CPLUSPLUS
01285 symbol_type (int tok, location_type l)
01286 : super_type (token_kind_type (tok), std::move (l))
01287 #else
01288 symbol_type (int tok, const location_type& l)
01289 : super_type (token_kind_type (tok), l)
01290 #endif
01291 {
01292 #if !defined _MSC_VER || defined __clang__
01293 YY_ASSERT (tok == token::END
01294 || (token::YError <= tok && tok <= token::YUNDEF)
01295 || (token::LPAREN <= tok && tok <= 349));
01296 #endif
01297 }
01298 #if 201103L <= YY_CPLUSPLUS
01299 symbol_type (int tok, obj_t v, location_type l)
01300 : super_type (token_kind_type (tok), std::move (v), std::move (l))
01301 #else
01302 symbol_type (int tok, const obj_t& v, const location_type& l)
01303 : super_type (token_kind_type (tok), v, l)
01304 #endif
01305 {
01306 #if !defined _MSC_VER || defined __clang__
01307 YY_ASSERT ((token::NUMD <= tok && tok <= token::FATRGR));
01308 #endif
01309 }
01310 };
01311
01313 genesyspp_parser (genesyspp_driver& driver_yyarg);
01314 virtual ~genesyspp_parser ();
01315
01316 #if 201103L <= YY_CPLUSPLUS
01318 genesyspp_parser (const genesyspp_parser&) = delete;
01320 genesyspp_parser& operator= (const genesyspp_parser&) = delete;
01321 #endif
01322
01325 int operator() ();
01326
01329 virtual int parse ();
01330
01331 #if YYDEBUG
01333 std::ostream& debug_stream () const YY_ATTRIBUTE_PURE;
01335 void set_debug_stream (std::ostream &);
01336
01338 typedef int debug_level_type;
01340 debug_level_type debug_level () const YY_ATTRIBUTE_PURE;
01342 void set_debug_level (debug_level_type l);
01343 #endif
01344
01348 virtual void error (const location_type& loc, const std::string& msg);
01349
01351 void error (const syntax_error& err);
01352
01355 static std::string symbol_name (symbol_kind_type yysymbol);
01356
01357 // Implementation of make_symbol for each token kind.
01358 #if 201103L <= YY_CPLUSPLUS
01359 static
01360 symbol_type
01361 make_END (location_type l)
01362 {
01363 return symbol_type (token::END, std::move (l));
01364 }
01365 #else
01366 static
01367 symbol_type
01368 make_END (const location_type& l)
01369 {
01370 return symbol_type (token::END, l);
01371 }
01372 #endif
01373 #if 201103L <= YY_CPLUSPLUS
01374 static
01375 symbol_type
01376 make_YError (location_type l)
01377 {
01378 return symbol_type (token::YError, std::move (l));
01379 }
01380 #else
01381 static
01382 symbol_type
01383 make_YError (const location_type& l)
01384 {
01385 return symbol_type (token::YError, l);
01386 }
01387 #endif

```

```
01388 #if 201103L <= YY_CPLUSPLUS
01389 static
01390 symbol_type
01391 make_YYUNDEF (location_type l)
01392 {
01393 return symbol_type (token::YYUNDEF, std::move (l));
01394 }
01395 #else
01396 static
01397 symbol_type
01398 make_YYUNDEF (const location_type& l)
01399 {
01400 return symbol_type (token::YYUNDEF, l);
01401 }
01402 #endif
01403 #if 201103L <= YY_CPLUSPLUS
01404 static
01405 symbol_type
01406 make_NUMD (obj_t v, location_type l)
01407 {
01408 return symbol_type (token::NUMD, std::move (v), std::move (l));
01409 }
01410 #else
01411 static
01412 symbol_type
01413 make_NUMD (const obj_t& v, const location_type& l)
01414 {
01415 return symbol_type (token::NUMD, v, l);
01416 }
01417 #endif
01418 #if 201103L <= YY_CPLUSPLUS
01419 static
01420 symbol_type
01421 make_NUMH (obj_t v, location_type l)
01422 {
01423 return symbol_type (token::NUMH, std::move (v), std::move (l));
01424 }
01425 #else
01426 static
01427 symbol_type
01428 make_NUMH (const obj_t& v, const location_type& l)
01429 {
01430 return symbol_type (token::NUMH, v, l);
01431 }
01432 #endif
01433 #if 201103L <= YY_CPLUSPLUS
01434 static
01435 symbol_type
01436 make_CTEZERO (obj_t v, location_type l)
01437 {
01438 return symbol_type (token::CTEZERO, std::move (v), std::move (l));
01439 }
01440 #else
01441 static
01442 symbol_type
01443 make_CTEZERO (const obj_t& v, const location_type& l)
01444 {
01445 return symbol_type (token::CTEZERO, v, l);
01446 }
01447 #endif
01448 #if 201103L <= YY_CPLUSPLUS
01449 static
01450 symbol_type
01451 make_oLE (obj_t v, location_type l)
01452 {
01453 return symbol_type (token::oLE, std::move (v), std::move (l));
01454 }
01455 #else
01456 static
01457 symbol_type
01458 make_oLE (const obj_t& v, const location_type& l)
01459 {
01460 return symbol_type (token::oLE, v, l);
01461 }
01462 #endif
01463 #if 201103L <= YY_CPLUSPLUS
01464 static
01465 symbol_type
01466 make_oGE (obj_t v, location_type l)
01467 {
01468 return symbol_type (token::oGE, std::move (v), std::move (l));
01469 }
01470 #else
01471 static
01472 symbol_type
01473 make_oGE (const obj_t& v, const location_type& l)
01474 {
```

```

01475 return symbol_type (token::oGE, v, l);
01476 }
01477 #endif
01478 #if 201103L <= YY_CPLUSPLUS
01479 static
01480 symbol_type
01481 make_oEQ (obj_t v, location_type l)
01482 {
01483 return symbol_type (token::oEQ, std::move (v), std::move (l));
01484 }
01485 #else
01486 static
01487 symbol_type
01488 make_oEQ (const obj_t& v, const location_type& l)
01489 {
01490 return symbol_type (token::oEQ, v, l);
01491 }
01492 #endif
01493 #if 201103L <= YY_CPLUSPLUS
01494 static
01495 symbol_type
01496 make_oNE (obj_t v, location_type l)
01497 {
01498 return symbol_type (token::oNE, std::move (v), std::move (l));
01499 }
01500 #else
01501 static
01502 symbol_type
01503 make_oNE (const obj_t& v, const location_type& l)
01504 {
01505 return symbol_type (token::oNE, v, l);
01506 }
01507 #endif
01508 #if 201103L <= YY_CPLUSPLUS
01509 static
01510 symbol_type
01511 make_oAND (obj_t v, location_type l)
01512 {
01513 return symbol_type (token::oAND, std::move (v), std::move (l));
01514 }
01515 #else
01516 static
01517 symbol_type
01518 make_oAND (const obj_t& v, const location_type& l)
01519 {
01520 return symbol_type (token::oAND, v, l);
01521 }
01522 #endif
01523 #if 201103L <= YY_CPLUSPLUS
01524 static
01525 symbol_type
01526 make_oOR (obj_t v, location_type l)
01527 {
01528 return symbol_type (token::oOR, std::move (v), std::move (l));
01529 }
01530 #else
01531 static
01532 symbol_type
01533 make_oOR (const obj_t& v, const location_type& l)
01534 {
01535 return symbol_type (token::oOR, v, l);
01536 }
01537 #endif
01538 #if 201103L <= YY_CPLUSPLUS
01539 static
01540 symbol_type
01541 make_oNAND (obj_t v, location_type l)
01542 {
01543 return symbol_type (token::oNAND, std::move (v), std::move (l));
01544 }
01545 #else
01546 static
01547 symbol_type
01548 make_oNAND (const obj_t& v, const location_type& l)
01549 {
01550 return symbol_type (token::oNAND, v, l);
01551 }
01552 #endif
01553 #if 201103L <= YY_CPLUSPLUS
01554 static
01555 symbol_type
01556 make_oXOR (obj_t v, location_type l)
01557 {
01558 return symbol_type (token::oXOR, std::move (v), std::move (l));
01559 }
01560 #else
01561 static

```

```
01562 symbol_type
01563 make_oXOR (const obj_t& v, const location_type& l)
01564 {
01565 return symbol_type (token::oXOR, v, l);
01566 }
01567 #endif
01568 #if 201103L <= YY_CPLUSPLUS
01569 static
01570 symbol_type
01571 make_oNOT (obj_t v, location_type l)
01572 {
01573 return symbol_type (token::oNOT, std::move (v), std::move (l));
01574 }
01575 #else
01576 static
01577 symbol_type
01578 make_oNOT (const obj_t& v, const location_type& l)
01579 {
01580 return symbol_type (token::oNOT, v, l);
01581 }
01582 #endif
01583 #if 201103L <= YY_CPLUSPLUS
01584 static
01585 symbol_type
01586 make_fSIN (obj_t v, location_type l)
01587 {
01588 return symbol_type (token::fSIN, std::move (v), std::move (l));
01589 }
01590 #else
01591 static
01592 symbol_type
01593 make_fSIN (const obj_t& v, const location_type& l)
01594 {
01595 return symbol_type (token::fSIN, v, l);
01596 }
01597 #endif
01598 #if 201103L <= YY_CPLUSPLUS
01599 static
01600 symbol_type
01601 make_fCOS (obj_t v, location_type l)
01602 {
01603 return symbol_type (token::fCOS, std::move (v), std::move (l));
01604 }
01605 #else
01606 static
01607 symbol_type
01608 make_fCOS (const obj_t& v, const location_type& l)
01609 {
01610 return symbol_type (token::fCOS, v, l);
01611 }
01612 #endif
01613 #if 201103L <= YY_CPLUSPLUS
01614 static
01615 symbol_type
01616 make_fROUND (obj_t v, location_type l)
01617 {
01618 return symbol_type (token::fROUND, std::move (v), std::move (l));
01619 }
01620 #else
01621 static
01622 symbol_type
01623 make_fROUND (const obj_t& v, const location_type& l)
01624 {
01625 return symbol_type (token::fROUND, v, l);
01626 }
01627 #endif
01628 #if 201103L <= YY_CPLUSPLUS
01629 static
01630 symbol_type
01631 make_fMOD (obj_t v, location_type l)
01632 {
01633 return symbol_type (token::fMOD, std::move (v), std::move (l));
01634 }
01635 #else
01636 static
01637 symbol_type
01638 make_fMOD (const obj_t& v, const location_type& l)
01639 {
01640 return symbol_type (token::fMOD, v, l);
01641 }
01642 #endif
01643 #if 201103L <= YY_CPLUSPLUS
01644 static
01645 symbol_type
01646 make_fTRUNC (obj_t v, location_type l)
01647 {
01648 return symbol_type (token::fTRUNC, std::move (v), std::move (l));
```

```
01649 }
01650 #else
01651 static
01652 symbol_type
01653 make_fTRUNC (const obj_t& v, const location_type& l)
01654 {
01655 return symbol_type (token::fTRUNC, v, l);
01656 }
01657 #endif
01658 #if 201103L <= YY_CPLUSPLUS
01659 static
01660 symbol_type
01661 make_fFRAC (obj_t v, location_type l)
01662 {
01663 return symbol_type (token::fFRAC, std::move (v), std::move (l));
01664 }
01665 #else
01666 static
01667 symbol_type
01668 make_fFRAC (const obj_t& v, const location_type& l)
01669 {
01670 return symbol_type (token::fFRAC, v, l);
01671 }
01672 #endif
01673 #if 201103L <= YY_CPLUSPLUS
01674 static
01675 symbol_type
01676 make_fEXP (obj_t v, location_type l)
01677 {
01678 return symbol_type (token::fEXP, std::move (v), std::move (l));
01679 }
01680 #else
01681 static
01682 symbol_type
01683 make_fEXP (const obj_t& v, const location_type& l)
01684 {
01685 return symbol_type (token::fEXP, v, l);
01686 }
01687 #endif
01688 #if 201103L <= YY_CPLUSPLUS
01689 static
01690 symbol_type
01691 make_fSQRT (obj_t v, location_type l)
01692 {
01693 return symbol_type (token::fSQRT, std::move (v), std::move (l));
01694 }
01695 #else
01696 static
01697 symbol_type
01698 make_fSQRT (const obj_t& v, const location_type& l)
01699 {
01700 return symbol_type (token::fSQRT, v, l);
01701 }
01702 #endif
01703 #if 201103L <= YY_CPLUSPLUS
01704 static
01705 symbol_type
01706 make_fLOG (obj_t v, location_type l)
01707 {
01708 return symbol_type (token::fLOG, std::move (v), std::move (l));
01709 }
01710 #else
01711 static
01712 symbol_type
01713 make_fLOG (const obj_t& v, const location_type& l)
01714 {
01715 return symbol_type (token::fLOG, v, l);
01716 }
01717 #endif
01718 #if 201103L <= YY_CPLUSPLUS
01719 static
01720 symbol_type
01721 make_fLN (obj_t v, location_type l)
01722 {
01723 return symbol_type (token::fLN, std::move (v), std::move (l));
01724 }
01725 #else
01726 static
01727 symbol_type
01728 make_fLN (const obj_t& v, const location_type& l)
01729 {
01730 return symbol_type (token::fLN, v, l);
01731 }
01732 #endif
01733 #if 201103L <= YY_CPLUSPLUS
01734 static
01735 symbol_type
```



```
01736 make_mathMIN (obj_t v, location_type l)
01737 {
01738 return symbol_type (token::mathMIN, std::move (v), std::move (l));
01739 }
01740 #else
01741 static
01742 symbol_type
01743 make_mathMIN (const obj_t& v, const location_type& l)
01744 {
01745 return symbol_type (token::mathMIN, v, l);
01746 }
01747 #endif
01748 #if 201103L <= YY_CPLUSPLUS
01749 static
01750 symbol_type
01751 make_mathMAX (obj_t v, location_type l)
01752 {
01753 return symbol_type (token::mathMAX, std::move (v), std::move (l));
01754 }
01755 #else
01756 static
01757 symbol_type
01758 make_mathMAX (const obj_t& v, const location_type& l)
01759 {
01760 return symbol_type (token::mathMAX, v, l);
01761 }
01762 #endif
01763 #if 201103L <= YY_CPLUSPLUS
01764 static
01765 symbol_type
01766 make_fVAL (obj_t v, location_type l)
01767 {
01768 return symbol_type (token::fVAL, std::move (v), std::move (l));
01769 }
01770 #else
01771 static
01772 symbol_type
01773 make_fVAL (const obj_t& v, const location_type& l)
01774 {
01775 return symbol_type (token::fVAL, v, l);
01776 }
01777 #endif
01778 #if 201103L <= YY_CPLUSPLUS
01779 static
01780 symbol_type
01781 make_fEVAL (obj_t v, location_type l)
01782 {
01783 return symbol_type (token::fEVAL, std::move (v), std::move (l));
01784 }
01785 #else
01786 static
01787 symbol_type
01788 make_fEVAL (const obj_t& v, const location_type& l)
01789 {
01790 return symbol_type (token::fEVAL, v, l);
01791 }
01792 #endif
01793 #if 201103L <= YY_CPLUSPLUS
01794 static
01795 symbol_type
01796 make_fLENG (obj_t v, location_type l)
01797 {
01798 return symbol_type (token::fLENG, std::move (v), std::move (l));
01799 }
01800 #else
01801 static
01802 symbol_type
01803 make_fLENG (const obj_t& v, const location_type& l)
01804 {
01805 return symbol_type (token::fLENG, v, l);
01806 }
01807 #endif
01808 #if 201103L <= YY_CPLUSPLUS
01809 static
01810 symbol_type
01811 make_fRND1 (obj_t v, location_type l)
01812 {
01813 return symbol_type (token::fRND1, std::move (v), std::move (l));
01814 }
01815 #else
01816 static
01817 symbol_type
01818 make_fRND1 (const obj_t& v, const location_type& l)
01819 {
01820 return symbol_type (token::fRND1, v, l);
01821 }
01822 #endif
```

```

01823 #if 201103L <= YY_CPLUSPLUS
01824 static
01825 symbol_type
01826 make_fEXPO (obj_t v, location_type l)
01827 {
01828 return symbol_type (token::fEXPO, std::move (v), std::move (l));
01829 }
01830 #else
01831 static
01832 symbol_type
01833 make_fEXPO (const obj_t& v, const location_type& l)
01834 {
01835 return symbol_type (token::fEXPO, v, l);
01836 }
01837 #endif
01838 #if 201103L <= YY_CPLUSPLUS
01839 static
01840 symbol_type
01841 make_fNORM (obj_t v, location_type l)
01842 {
01843 return symbol_type (token::fNORM, std::move (v), std::move (l));
01844 }
01845 #else
01846 static
01847 symbol_type
01848 make_fNORM (const obj_t& v, const location_type& l)
01849 {
01850 return symbol_type (token::fNORM, v, l);
01851 }
01852 #endif
01853 #if 201103L <= YY_CPLUSPLUS
01854 static
01855 symbol_type
01856 make_fUNIF (obj_t v, location_type l)
01857 {
01858 return symbol_type (token::fUNIF, std::move (v), std::move (l));
01859 }
01860 #else
01861 static
01862 symbol_type
01863 make_fUNIF (const obj_t& v, const location_type& l)
01864 {
01865 return symbol_type (token::fUNIF, v, l);
01866 }
01867 #endif
01868 #if 201103L <= YY_CPLUSPLUS
01869 static
01870 symbol_type
01871 make_fWEIB (obj_t v, location_type l)
01872 {
01873 return symbol_type (token::fWEIB, std::move (v), std::move (l));
01874 }
01875 #else
01876 static
01877 symbol_type
01878 make_fWEIB (const obj_t& v, const location_type& l)
01879 {
01880 return symbol_type (token::fWEIB, v, l);
01881 }
01882 #endif
01883 #if 201103L <= YY_CPLUSPLUS
01884 static
01885 symbol_type
01886 make_fLOGN (obj_t v, location_type l)
01887 {
01888 return symbol_type (token::fLOGN, std::move (v), std::move (l));
01889 }
01890 #else
01891 static
01892 symbol_type
01893 make_fLOGN (const obj_t& v, const location_type& l)
01894 {
01895 return symbol_type (token::fLOGN, v, l);
01896 }
01897 #endif
01898 #if 201103L <= YY_CPLUSPLUS
01899 static
01900 symbol_type
01901 make_fGAMM (obj_t v, location_type l)
01902 {
01903 return symbol_type (token::fGAMM, std::move (v), std::move (l));
01904 }
01905 #else
01906 static
01907 symbol_type
01908 make_fGAMM (const obj_t& v, const location_type& l)
01909 {

```

```
01910 return symbol_type (token::fGAMM, v, l);
01911 }
01912 #endif
01913 #if 201103L <= YY_CPLUSPLUS
01914 static
01915 symbol_type
01916 make_fERLA (obj_t v, location_type l)
01917 {
01918 return symbol_type (token::fERLA, std::move (v), std::move (l));
01919 }
01920 #else
01921 static
01922 symbol_type
01923 make_fERLA (const obj_t& v, const location_type& l)
01924 {
01925 return symbol_type (token::fERLA, v, l);
01926 }
01927 #endif
01928 #if 201103L <= YY_CPLUSPLUS
01929 static
01930 symbol_type
01931 make_fTRIA (obj_t v, location_type l)
01932 {
01933 return symbol_type (token::fTRIA, std::move (v), std::move (l));
01934 }
01935 #else
01936 static
01937 symbol_type
01938 make_fTRIA (const obj_t& v, const location_type& l)
01939 {
01940 return symbol_type (token::fTRIA, v, l);
01941 }
01942 #endif
01943 #if 201103L <= YY_CPLUSPLUS
01944 static
01945 symbol_type
01946 make_fBETA (obj_t v, location_type l)
01947 {
01948 return symbol_type (token::fBETA, std::move (v), std::move (l));
01949 }
01950 #else
01951 static
01952 symbol_type
01953 make_fBETA (const obj_t& v, const location_type& l)
01954 {
01955 return symbol_type (token::fBETA, v, l);
01956 }
01957 #endif
01958 #if 201103L <= YY_CPLUSPLUS
01959 static
01960 symbol_type
01961 make_fDISC (obj_t v, location_type l)
01962 {
01963 return symbol_type (token::fDISC, std::move (v), std::move (l));
01964 }
01965 #else
01966 static
01967 symbol_type
01968 make_fDISC (const obj_t& v, const location_type& l)
01969 {
01970 return symbol_type (token::fDISC, v, l);
01971 }
01972 #endif
01973 #if 201103L <= YY_CPLUSPLUS
01974 static
01975 symbol_type
01976 make_fTNOW (obj_t v, location_type l)
01977 {
01978 return symbol_type (token::fTNOW, std::move (v), std::move (l));
01979 }
01980 #else
01981 static
01982 symbol_type
01983 make_fTNOW (const obj_t& v, const location_type& l)
01984 {
01985 return symbol_type (token::fTNOW, v, l);
01986 }
01987 #endif
01988 #if 201103L <= YY_CPLUSPLUS
01989 static
01990 symbol_type
01991 make_fTFIN (obj_t v, location_type l)
01992 {
01993 return symbol_type (token::fTFIN, std::move (v), std::move (l));
01994 }
01995 #else
01996 static
```

```

01997 symbol_type
01998 make_fTFIN (const obj_t& v, const location_type& l)
01999 {
02000 return symbol_type (token::fTFIN, v, l);
02001 }
02002 #endif
02003 #if 201103L <= YY_CPLUSPLUS
02004 static
02005 symbol_type
02006 make_fMAXREP (obj_t v, location_type l)
02007 {
02008 return symbol_type (token::fMAXREP, std::move (v), std::move (l));
02009 }
02010 #else
02011 static
02012 symbol_type
02013 make_fMAXREP (const obj_t& v, const location_type& l)
02014 {
02015 return symbol_type (token::fMAXREP, v, l);
02016 }
02017 #endif
02018 #if 201103L <= YY_CPLUSPLUS
02019 static
02020 symbol_type
02021 make_fNUMREP (obj_t v, location_type l)
02022 {
02023 return symbol_type (token::fNUMREP, std::move (v), std::move (l));
02024 }
02025 #else
02026 static
02027 symbol_type
02028 make_fNUMREP (const obj_t& v, const location_type& l)
02029 {
02030 return symbol_type (token::fNUMREP, v, l);
02031 }
02032 #endif
02033 #if 201103L <= YY_CPLUSPLUS
02034 static
02035 symbol_type
02036 make_fIDENT (obj_t v, location_type l)
02037 {
02038 return symbol_type (token::fIDENT, std::move (v), std::move (l));
02039 }
02040 #else
02041 static
02042 symbol_type
02043 make_fIDENT (const obj_t& v, const location_type& l)
02044 {
02045 return symbol_type (token::fIDENT, v, l);
02046 }
02047 #endif
02048 #if 201103L <= YY_CPLUSPLUS
02049 static
02050 symbol_type
02051 make_simulEntitiesWIP (obj_t v, location_type l)
02052 {
02053 return symbol_type (token::simulEntitiesWIP, std::move (v), std::move (l));
02054 }
02055 #else
02056 static
02057 symbol_type
02058 make_simulEntitiesWIP (const obj_t& v, const location_type& l)
02059 {
02060 return symbol_type (token::simulEntitiesWIP, v, l);
02061 }
02062 #endif
02063 #if 201103L <= YY_CPLUSPLUS
02064 static
02065 symbol_type
02066 make_cIF (obj_t v, location_type l)
02067 {
02068 return symbol_type (token::cIF, std::move (v), std::move (l));
02069 }
02070 #else
02071 static
02072 symbol_type
02073 make_cIF (const obj_t& v, const location_type& l)
02074 {
02075 return symbol_type (token::cIF, v, l);
02076 }
02077 #endif
02078 #if 201103L <= YY_CPLUSPLUS
02079 static
02080 symbol_type
02081 make_cELSE (obj_t v, location_type l)
02082 {
02083 return symbol_type (token::cELSE, std::move (v), std::move (l));

```

```
02084 }
02085 #else
02086 static
02087 symbol_type
02088 make_cELSE (const obj_t& v, const location_type& l)
02089 {
02090 return symbol_type (token::cELSE, v, l);
02091 }
02092 #endif
02093 #if 201103L <= YY_CPLUSPLUS
02094 static
02095 symbol_type
02096 make_cFOR (obj_t v, location_type l)
02097 {
02098 return symbol_type (token::cFOR, std::move (v), std::move (l));
02099 }
02100 #else
02101 static
02102 symbol_type
02103 make_cFOR (const obj_t& v, const location_type& l)
02104 {
02105 return symbol_type (token::cFOR, v, l);
02106 }
02107 #endif
02108 #if 201103L <= YY_CPLUSPLUS
02109 static
02110 symbol_type
02111 make_cTO (obj_t v, location_type l)
02112 {
02113 return symbol_type (token::cTO, std::move (v), std::move (l));
02114 }
02115 #else
02116 static
02117 symbol_type
02118 make_cTO (const obj_t& v, const location_type& l)
02119 {
02120 return symbol_type (token::cTO, v, l);
02121 }
02122 #endif
02123 #if 201103L <= YY_CPLUSPLUS
02124 static
02125 symbol_type
02126 make_cDO (obj_t v, location_type l)
02127 {
02128 return symbol_type (token::cDO, std::move (v), std::move (l));
02129 }
02130 #else
02131 static
02132 symbol_type
02133 make_cDO (const obj_t& v, const location_type& l)
02134 {
02135 return symbol_type (token::cDO, v, l);
02136 }
02137 #endif
02138 #if 201103L <= YY_CPLUSPLUS
02139 static
02140 symbol_type
02141 make_ATTRIB (obj_t v, location_type l)
02142 {
02143 return symbol_type (token::ATTRIB, std::move (v), std::move (l));
02144 }
02145 #else
02146 static
02147 symbol_type
02148 make_ATTRIB (const obj_t& v, const location_type& l)
02149 {
02150 return symbol_type (token::ATTRIB, v, l);
02151 }
02152 #endif
02153 #if 201103L <= YY_CPLUSPLUS
02154 static
02155 symbol_type
02156 make_CSTAT (obj_t v, location_type l)
02157 {
02158 return symbol_type (token::CSTAT, std::move (v), std::move (l));
02159 }
02160 #else
02161 static
02162 symbol_type
02163 make_CSTAT (const obj_t& v, const location_type& l)
02164 {
02165 return symbol_type (token::CSTAT, v, l);
02166 }
02167 #endif
02168 #if 201103L <= YY_CPLUSPLUS
02169 static
02170 symbol_type
```

```

02171 make_COUNTER (obj_t v, location_type l)
02172 {
02173 return symbol_type (token::COUNTER, std::move (v), std::move (l));
02174 }
02175 #endif
02176 static
02177 symbol_type
02178 make_COUNTER (const obj_t& v, const location_type& l)
02179 {
02180 return symbol_type (token::COUNTER, v, l);
02181 }
02182 #endif
02183 #if 201103L <= YY_CPLUSPLUS
02184 static
02185 symbol_type
02186 make_SIMRESP (obj_t v, location_type l)
02187 {
02188 return symbol_type (token::SIMRESP, std::move (v), std::move (l));
02189 }
02190 #else
02191 static
02192 symbol_type
02193 make_SIMRESP (const obj_t& v, const location_type& l)
02194 {
02195 return symbol_type (token::SIMRESP, v, l);
02196 }
02197 #endif
02198 #if 201103L <= YY_CPLUSPLUS
02199 static
02200 symbol_type
02201 make_SIMCTRL (obj_t v, location_type l)
02202 {
02203 return symbol_type (token::SIMCTRL, std::move (v), std::move (l));
02204 }
02205 #else
02206 static
02207 symbol_type
02208 make_SIMCTRL (const obj_t& v, const location_type& l)
02209 {
02210 return symbol_type (token::SIMCTRL, v, l);
02211 }
02212 #endif
02213 #if 201103L <= YY_CPLUSPLUS
02214 static
02215 symbol_type
02216 make_fTAVG (obj_t v, location_type l)
02217 {
02218 return symbol_type (token::fTAVG, std::move (v), std::move (l));
02219 }
02220 #else
02221 static
02222 symbol_type
02223 make_fTAVG (const obj_t& v, const location_type& l)
02224 {
02225 return symbol_type (token::fTAVG, v, l);
02226 }
02227 #endif
02228 #if 201103L <= YY_CPLUSPLUS
02229 static
02230 symbol_type
02231 make_fCOUNT (obj_t v, location_type l)
02232 {
02233 return symbol_type (token::fCOUNT, std::move (v), std::move (l));
02234 }
02235 #else
02236 static
02237 symbol_type
02238 make_fCOUNT (const obj_t& v, const location_type& l)
02239 {
02240 return symbol_type (token::fCOUNT, v, l);
02241 }
02242 #endif
02243 #if 201103L <= YY_CPLUSPLUS
02244 static
02245 symbol_type
02246 make_ILLEGAL (obj_t v, location_type l)
02247 {
02248 return symbol_type (token::ILLEGAL, std::move (v), std::move (l));
02249 }
02250 #else
02251 static
02252 symbol_type
02253 make_ILLEGAL (const obj_t& v, const location_type& l)
02254 {
02255 return symbol_type (token::ILLEGAL, v, l);
02256 }
02257 #endif

```

```
02258 #if 201103L <= YY_CPLUSPLUS
02259 static
02260 symbol_type
02261 make_RESOURCE (obj_t v, location_type l)
02262 {
02263 return symbol_type (token::RESOURCE, std::move (v), std::move (l));
02264 }
02265 #else
02266 static
02267 symbol_type
02268 make_RESOURCE (const obj_t& v, const location_type& l)
02269 {
02270 return symbol_type (token::RESOURCE, v, l);
02271 }
02272 #endif
02273 #if 201103L <= YY_CPLUSPLUS
02274 static
02275 symbol_type
02276 make_fNR (obj_t v, location_type l)
02277 {
02278 return symbol_type (token::fNR, std::move (v), std::move (l));
02279 }
02280 #else
02281 static
02282 symbol_type
02283 make_fNR (const obj_t& v, const location_type& l)
02284 {
02285 return symbol_type (token::fNR, v, l);
02286 }
02287 #endif
02288 #if 201103L <= YY_CPLUSPLUS
02289 static
02290 symbol_type
02291 make_fMR (obj_t v, location_type l)
02292 {
02293 return symbol_type (token::fMR, std::move (v), std::move (l));
02294 }
02295 #else
02296 static
02297 symbol_type
02298 make_fMR (const obj_t& v, const location_type& l)
02299 {
02300 return symbol_type (token::fMR, v, l);
02301 }
02302 #endif
02303 #if 201103L <= YY_CPLUSPLUS
02304 static
02305 symbol_type
02306 make_fIRF (obj_t v, location_type l)
02307 {
02308 return symbol_type (token::fIRF, std::move (v), std::move (l));
02309 }
02310 #else
02311 static
02312 symbol_type
02313 make_fIRF (const obj_t& v, const location_type& l)
02314 {
02315 return symbol_type (token::fIRF, v, l);
02316 }
02317 #endif
02318 #if 201103L <= YY_CPLUSPLUS
02319 static
02320 symbol_type
02321 make_fRESSEIZES (obj_t v, location_type l)
02322 {
02323 return symbol_type (token::fRESSEIZES, std::move (v), std::move (l));
02324 }
02325 #else
02326 static
02327 symbol_type
02328 make_fRESSEIZES (const obj_t& v, const location_type& l)
02329 {
02330 return symbol_type (token::fRESSEIZES, v, l);
02331 }
02332 #endif
02333 #if 201103L <= YY_CPLUSPLUS
02334 static
02335 symbol_type
02336 make_fSTATE (obj_t v, location_type l)
02337 {
02338 return symbol_type (token::fSTATE, std::move (v), std::move (l));
02339 }
02340 #else
02341 static
02342 symbol_type
02343 make_fSTATE (const obj_t& v, const location_type& l)
02344 {
```

```

02345 return symbol_type (token::fSTATE, v, l);
02346 }
02347 #endif
02348 #if 201103L <= YY_CPLUSPLUS
02349 static
02350 symbol_type
02351 make_fSETSUM (obj_t v, location_type l)
02352 {
02353 return symbol_type (token::fSETSUM, std::move (v), std::move (l));
02354 }
02355 #else
02356 static
02357 symbol_type
02358 make_fSETSUM (const obj_t& v, const location_type& l)
02359 {
02360 return symbol_type (token::fSETSUM, v, l);
02361 }
02362 #endif
02363 #if 201103L <= YY_CPLUSPLUS
02364 static
02365 symbol_type
02366 make_fRESUTIL (obj_t v, location_type l)
02367 {
02368 return symbol_type (token::fRESUTIL, std::move (v), std::move (l));
02369 }
02370 #else
02371 static
02372 symbol_type
02373 make_fRESUTIL (const obj_t& v, const location_type& l)
02374 {
02375 return symbol_type (token::fRESUTIL, v, l);
02376 }
02377 #endif
02378 #if 201103L <= YY_CPLUSPLUS
02379 static
02380 symbol_type
02381 make_QUEUE (obj_t v, location_type l)
02382 {
02383 return symbol_type (token::QUEUE, std::move (v), std::move (l));
02384 }
02385 #else
02386 static
02387 symbol_type
02388 make_QUEUE (const obj_t& v, const location_type& l)
02389 {
02390 return symbol_type (token::QUEUE, v, l);
02391 }
02392 #endif
02393 #if 201103L <= YY_CPLUSPLUS
02394 static
02395 symbol_type
02396 make_fNQ (obj_t v, location_type l)
02397 {
02398 return symbol_type (token::fNQ, std::move (v), std::move (l));
02399 }
02400 #else
02401 static
02402 symbol_type
02403 make_fNQ (const obj_t& v, const location_type& l)
02404 {
02405 return symbol_type (token::fNQ, v, l);
02406 }
02407 #endif
02408 #if 201103L <= YY_CPLUSPLUS
02409 static
02410 symbol_type
02411 make_fFIRSTINQ (obj_t v, location_type l)
02412 {
02413 return symbol_type (token::fFIRSTINQ, std::move (v), std::move (l));
02414 }
02415 #else
02416 static
02417 symbol_type
02418 make_fFIRSTINQ (const obj_t& v, const location_type& l)
02419 {
02420 return symbol_type (token::fFIRSTINQ, v, l);
02421 }
02422 #endif
02423 #if 201103L <= YY_CPLUSPLUS
02424 static
02425 symbol_type
02426 make_fLASTINQ (obj_t v, location_type l)
02427 {
02428 return symbol_type (token::fLASTINQ, std::move (v), std::move (l));
02429 }
02430 #else
02431 static

```



```
02432 symbol_type
02433 make_fLASTINQ (const obj_t& v, const location_type& l)
02434 {
02435 return symbol_type (token::fLASTINQ, v, l);
02436 }
02437 #endif
02438 #if 201103L <= YY_CPLUSPLUS
02439 static
02440 symbol_type
02441 make_fSAQUE (obj_t v, location_type l)
02442 {
02443 return symbol_type (token::fSAQUE, std::move (v), std::move (l));
02444 }
02445 #else
02446 static
02447 symbol_type
02448 make_fSAQUE (const obj_t& v, const location_type& l)
02449 {
02450 return symbol_type (token::fSAQUE, v, l);
02451 }
02452 #endif
02453 #if 201103L <= YY_CPLUSPLUS
02454 static
02455 symbol_type
02456 make_fAQUE (obj_t v, location_type l)
02457 {
02458 return symbol_type (token::fAQUE, std::move (v), std::move (l));
02459 }
02460 #else
02461 static
02462 symbol_type
02463 make_fAQUE (const obj_t& v, const location_type& l)
02464 {
02465 return symbol_type (token::fAQUE, v, l);
02466 }
02467 #endif
02468 #if 201103L <= YY_CPLUSPLUS
02469 static
02470 symbol_type
02471 make_fENTATRANK (obj_t v, location_type l)
02472 {
02473 return symbol_type (token::fENTATRANK, std::move (v), std::move (l));
02474 }
02475 #else
02476 static
02477 symbol_type
02478 make_fENTATRANK (const obj_t& v, const location_type& l)
02479 {
02480 return symbol_type (token::fENTATRANK, v, l);
02481 }
02482 #endif
02483 #if 201103L <= YY_CPLUSPLUS
02484 static
02485 symbol_type
02486 make_SET (obj_t v, location_type l)
02487 {
02488 return symbol_type (token::SET, std::move (v), std::move (l));
02489 }
02490 #else
02491 static
02492 symbol_type
02493 make_SET (const obj_t& v, const location_type& l)
02494 {
02495 return symbol_type (token::SET, v, l);
02496 }
02497 #endif
02498 #if 201103L <= YY_CPLUSPLUS
02499 static
02500 symbol_type
02501 make_fNUMSET (obj_t v, location_type l)
02502 {
02503 return symbol_type (token::fNUMSET, std::move (v), std::move (l));
02504 }
02505 #else
02506 static
02507 symbol_type
02508 make_fNUMSET (const obj_t& v, const location_type& l)
02509 {
02510 return symbol_type (token::fNUMSET, v, l);
02511 }
02512 #endif
02513 #if 201103L <= YY_CPLUSPLUS
02514 static
02515 symbol_type
02516 make_VARI (obj_t v, location_type l)
02517 {
02518 return symbol_type (token::VARI, std::move (v), std::move (l));
```

```

02519 }
02520 #else
02521 static
02522 symbol_type
02523 make_VARI (const obj_t& v, const location_type& l)
02524 {
02525 return symbol_type (token::VARI, v, l);
02526 }
02527 #endif
02528 #if 201103L <= YY_CPLUSPLUS
02529 static
02530 symbol_type
02531 make_FORM (obj_t v, location_type l)
02532 {
02533 return symbol_type (token::FORM, std::move (v), std::move (l));
02534 }
02535 #else
02536 static
02537 symbol_type
02538 make_FORM (const obj_t& v, const location_type& l)
02539 {
02540 return symbol_type (token::FORM, v, l);
02541 }
02542 #endif
02543 #if 201103L <= YY_CPLUSPLUS
02544 static
02545 symbol_type
02546 make_fNUMGR (obj_t v, location_type l)
02547 {
02548 return symbol_type (token::fNUMGR, std::move (v), std::move (l));
02549 }
02550 #else
02551 static
02552 symbol_type
02553 make_fNUMGR (const obj_t& v, const location_type& l)
02554 {
02555 return symbol_type (token::fNUMGR, v, l);
02556 }
02557 #endif
02558 #if 201103L <= YY_CPLUSPLUS
02559 static
02560 symbol_type
02561 make_fATRGR (obj_t v, location_type l)
02562 {
02563 return symbol_type (token::fATRGR, std::move (v), std::move (l));
02564 }
02565 #else
02566 static
02567 symbol_type
02568 make_fATRGR (const obj_t& v, const location_type& l)
02569 {
02570 return symbol_type (token::fATRGR, v, l);
02571 }
02572 #endif
02573 #if 201103L <= YY_CPLUSPLUS
02574 static
02575 symbol_type
02576 make_LPAREN (location_type l)
02577 {
02578 return symbol_type (token::LPAREN, std::move (l));
02579 }
02580 #else
02581 static
02582 symbol_type
02583 make_LPAREN (const location_type& l)
02584 {
02585 return symbol_type (token::LPAREN, l);
02586 }
02587 #endif
02588 #if 201103L <= YY_CPLUSPLUS
02589 static
02590 symbol_type
02591 make_RPAREN (location_type l)
02592 {
02593 return symbol_type (token::RPAREN, std::move (l));
02594 }
02595 #else
02596 static
02597 symbol_type
02598 make_RPAREN (const location_type& l)
02599 {
02600 return symbol_type (token::RPAREN, l);
02601 }
02602 #endif
02603 #if 201103L <= YY_CPLUSPLUS
02604 static
02605 symbol_type

```

```
02606 make_LBRACKET (location_type l)
02607 {
02608 return symbol_type (token::LBRACKET, std::move (l));
02609 }
02610 #else
02611 static
02612 symbol_type
02613 make_LBRACKET (const location_type& l)
02614 {
02615 return symbol_type (token::LBRACKET, l);
02616 }
02617 #endif
02618 #if 201103L <= YY_CPLUSPLUS
02619 static
02620 symbol_type
02621 make_RBRACKET (location_type l)
02622 {
02623 return symbol_type (token::RBRACKET, std::move (l));
02624 }
02625 #else
02626 static
02627 symbol_type
02628 make_RBRACKET (const location_type& l)
02629 {
02630 return symbol_type (token::RBRACKET, l);
02631 }
02632 #endif
02633 #if 201103L <= YY_CPLUSPLUS
02634 static
02635 symbol_type
02636 make_PLUS (location_type l)
02637 {
02638 return symbol_type (token::PLUS, std::move (l));
02639 }
02640 #else
02641 static
02642 symbol_type
02643 make_PLUS (const location_type& l)
02644 {
02645 return symbol_type (token::PLUS, l);
02646 }
02647 #endif
02648 #if 201103L <= YY_CPLUSPLUS
02649 static
02650 symbol_type
02651 make_MINUS (location_type l)
02652 {
02653 return symbol_type (token::MINUS, std::move (l));
02654 }
02655 #else
02656 static
02657 symbol_type
02658 make_MINUS (const location_type& l)
02659 {
02660 return symbol_type (token::MINUS, l);
02661 }
02662 #endif
02663 #if 201103L <= YY_CPLUSPLUS
02664 static
02665 symbol_type
02666 make_STAR (location_type l)
02667 {
02668 return symbol_type (token::STAR, std::move (l));
02669 }
02670 #else
02671 static
02672 symbol_type
02673 make_STAR (const location_type& l)
02674 {
02675 return symbol_type (token::STAR, l);
02676 }
02677 #endif
02678 #if 201103L <= YY_CPLUSPLUS
02679 static
02680 symbol_type
02681 make_POWER (location_type l)
02682 {
02683 return symbol_type (token::POWER, std::move (l));
02684 }
02685 #else
02686 static
02687 symbol_type
02688 make_POWER (const location_type& l)
02689 {
02690 return symbol_type (token::POWER, l);
02691 }
02692 #endif
```

```

02693 #if 201103L <= YY_CPLUSPLUS
02694 static
02695 symbol_type
02696 make_SLASH (location_type l)
02697 {
02698 return symbol_type (token::SLASH, std::move (l));
02699 }
02700 #else
02701 static
02702 symbol_type
02703 make_SLASH (const location_type& l)
02704 {
02705 return symbol_type (token::SLASH, l);
02706 }
02707 #endif
02708 #if 201103L <= YY_CPLUSPLUS
02709 static
02710 symbol_type
02711 make_LESS (location_type l)
02712 {
02713 return symbol_type (token::LESS, std::move (l));
02714 }
02715 #else
02716 static
02717 symbol_type
02718 make_LESS (const location_type& l)
02719 {
02720 return symbol_type (token::LESS, l);
02721 }
02722 #endif
02723 #if 201103L <= YY_CPLUSPLUS
02724 static
02725 symbol_type
02726 make_GREATER (location_type l)
02727 {
02728 return symbol_type (token::GREATER, std::move (l));
02729 }
02730 #else
02731 static
02732 symbol_type
02733 make_GREATER (const location_type& l)
02734 {
02735 return symbol_type (token::GREATER, l);
02736 }
02737 #endif
02738 #if 201103L <= YY_CPLUSPLUS
02739 static
02740 symbol_type
02741 make_ASSIGN (location_type l)
02742 {
02743 return symbol_type (token::ASSIGN, std::move (l));
02744 }
02745 #else
02746 static
02747 symbol_type
02748 make_ASSIGN (const location_type& l)
02749 {
02750 return symbol_type (token::ASSIGN, l);
02751 }
02752 #endif
02753 #if 201103L <= YY_CPLUSPLUS
02754 static
02755 symbol_type
02756 make_COMMA (location_type l)
02757 {
02758 return symbol_type (token::COMMA, std::move (l));
02759 }
02760 #else
02761 static
02762 symbol_type
02763 make_COMMA (const location_type& l)
02764 {
02765 return symbol_type (token::COMMA, l);
02766 }
02767 #endif
02768
02769
02770 class context
02771 {
02772 public:
02773 context (const genesyspp_parser& yyparser, const symbol_type& yyla);
02774 const symbol_type& lookahead () const YY_NOEXCEPT { return yyla_; }
02775 symbol_kind_type token () const YY_NOEXCEPT { return yyla_.kind (); }
02776 const location_type& location () const YY_NOEXCEPT { return yyla_.location; }
02777
02778 int expected_tokens (symbol_kind_type yyarg[], int yyargn) const;
02782

```

```

02783 private:
02784 const genesyspp_parser& yyparser_;
02785 const symbol_type& yyla_;
02786 };
02787
02788 private:
02789 #if YY_CPLUSPLUS < 201103L
02791 genesyspp_parser (const genesyspp_parser&);
02793 genesyspp_parser& operator= (const genesyspp_parser&);
02794 #endif
02795
02796
02798 typedef short state_type;
02799
02801 int yy_syntax_error_arguments_ (const context& yyctx,
02802 symbol_kind_type yyarg[], int yyargn) const;
02803
02806 virtual std::string yysyntax_error_ (const context& yyctx) const;
02810 static state_type yy_lr_goto_state_ (state_type yystate, int yysym);
02811
02814 static bool yy_pact_value_is_default_ (int yyvalue) YY_NOEXCEPT;
02815
02818 static bool yy_table_value_is_error_ (int yyvalue) YY_NOEXCEPT;
02819
02820 static const short yypact_ninf_;
02821 static const signed char yytable_ninf_;
02822
02826 static symbol_kind_type yytranslate_ (int t) YY_NOEXCEPT;
02827
02829 static std::string yytnamerr_ (const char *yystr);
02830
02832 static const char* const yytname_[];
02833
02834
02835 // Tables.
02836 // YYPACT[STATE-NUM] -- Index in YYTABLE of the portion describing
02837 // STATE-NUM.
02838 static const short yypact_[];
02839
02840 // YYDEFACT[STATE-NUM] -- Default reduction number in state STATE-NUM.
02841 // Performed when YYTABLE does not specify something else to do. Zero
02842 // means the default is an error.
02843 static const signed char yydefact_[];
02844
02845 // YYPGOTO[NTERM-NUM].
02846 static const short yypgoto_[];
02847
02848 // YYDEFGOTO[NTERM-NUM].
02849 static const unsigned char yydefgoto_[];
02850
02851 // YYTABLE[YYPACT[STATE-NUM]] -- What to do in state STATE-NUM. If
02852 // positive, shift that token. If negative, reduce the rule whose
02853 // number is the opposite. If YYTABLE_NINF, syntax error.
02854 static const short yytable_[];
02855
02856 static const short yycheck_[];
02857
02858 // YYSTOS[STATE-NUM] -- The symbol kind of the accessing symbol of
02859 // state STATE-NUM.
02860 static const unsigned char yystos_[];
02861
02862 // YYR1[RULE-NUM] -- Symbol kind of the left-hand side of rule RULE-NUM.
02863 static const unsigned char yyr1_[];
02864
02865 // YYR2[RULE-NUM] -- Number of symbols on the right-hand side of rule RULE-NUM.
02866 static const signed char yyr2_[];
02867
02868
02869 #if YYDEBUG
02870 // YYRLINE[YYN] -- Source line where rule number YYN was defined.
02871 static const short yyrline_[];
02873 virtual void yy_reduce_print_ (int r) const;
02875 virtual void yy_stack_print_ () const;
02876
02878 int yydebug_;
02880 std::ostream* yycdebug_;
02881
02885 template <typename Base>
02886 void yy_print_ (std::ostream& yyo, const basic_symbol<Base>& yysym) const;
02887 #endif
02888
02893 template <typename Base>
02894 void yy_destroy_ (const char* yymsg, basic_symbol<Base>& yysym) const;
02895
02896 private:
02898 struct by_state
02899 {

```

```

02901 by_state () YY_NOEXCEPT;
02902
02904 typedef state_type kind_type;
02905
02907 by_state (kind_type s) YY_NOEXCEPT;
02908
02910 by_state (const by_state& that) YY_NOEXCEPT;
02911
02913 void clear () YY_NOEXCEPT;
02914
02916 void move (by_state& that);
02917
02920 symbol_kind_type kind () const YY_NOEXCEPT;
02921
02924 enum { empty_state = 0 };
02925
02928 state_type state;
02929 };
02930
02932 struct stack_symbol_type : basic_symbol<by_state>
02933 {
02935 typedef basic_symbol<by_state> super_type;
02937 stack_symbol_type ();
02939 stack_symbol_type (YY_RVREF (stack_symbol_type) that);
02941 stack_symbol_type (state_type s, YY_MOVE_REF (symbol_type) sym);
02942 #if YY_CPLUSPLUS < 201103L
02945 stack_symbol_type& operator= (stack_symbol_type& that);
02946
02949 stack_symbol_type& operator= (const stack_symbol_type& that);
02950 #endif
02951 };
02952
02954 template <typename T, typename S = std::vector<T> >
02955 class stack
02956 {
02957 public:
02958 // Hide our reversed order.
02959 typedef typename S::iterator iterator;
02960 typedef typename S::const_iterator const_iterator;
02961 typedef typename S::size_type size_type;
02962 typedef typename std::ptrdiff_t index_type;
02963
02964 stack (size_type n = 200) YY_NOEXCEPT
02965 : seq_ (n)
02966 {}
02967
02968 #if 201103L <= YY_CPLUSPLUS
02970 stack (const stack&) = delete;
02972 stack& operator= (const stack&) = delete;
02973 #endif
02974
02978 const T&
02979 operator[] (index_type i) const
02980 {
02981 return seq_[size_type (size () - 1 - i)];
02982 }
02983
02987 T&
02988 operator[] (index_type i)
02989 {
02990 return seq_[size_type (size () - 1 - i)];
02991 }
02992
02996 void
02997 push (YY_MOVE_REF (T) t)
02998 {
02999 seq_.push_back (T ());
03000 operator[] (0).move (t);
03001 }
03002
03004 void
03005 pop (std::ptrdiff_t n = 1) YY_NOEXCEPT
03006 {
03007 for (; 0 < n; --n)
03008 seq_.pop_back ();
03009 }
03010
03012 void
03013 clear () YY_NOEXCEPT
03014 {
03015 seq_.clear ();
03016 }
03017
03019 index_type
03020 size () const YY_NOEXCEPT
03021 {
03022 return index_type (seq_.size ());

```

```

03023 }
03024
03026 const_iterator
03027 begin () const YY_NOEXCEPT
03028 {
03029 return seq_.begin ();
03030 }
03031
03033 const_iterator
03034 end () const YY_NOEXCEPT
03035 {
03036 return seq_.end ();
03037 }
03038
03040 class slice
03041 {
03042 public:
03043 slice (const stack& stack, index_type range) YY_NOEXCEPT
03044 : stack_ (stack)
03045 , range_ (range)
03046 {}
03047
03048 const T&
03049 operator[] (index_type i) const
03050 {
03051 return stack_[range_ - i];
03052 }
03053
03054 private:
03055 const stack& stack_;
03056 index_type range_;
03057 };
03058
03059 private:
03060 #if YY_CPLUSPLUS < 201103L
03062 stack (const stack&);
03064 stack& operator= (const stack&);
03065 #endif
03067 S seq_;
03068 };
03069
03070
03072 typedef stack<stack_symbol_type> stack_type;
03073
03075 stack_type yystack_;
03076
03082 void yypush_ (const char* m, YY_MOVE_REF (stack_symbol_type) sym);
03083
03090 void yypush_ (const char* m, state_type s, YY_MOVE_REF (symbol_type) sym);
03091
03093 void yypop_ (int n = 1) YY_NOEXCEPT;
03094
03096 enum
03097 {
03098 yylast_ = 493,
03099 yynnts_ = 34,
03100 yyfinal_ = 142
03101 };
03102
03103
03104 // User arguments.
03105 genesyspp_driver& driver;
03106
03107 };
03108
03109 inline
03110 genesyspp_parser::symbol_kind_type
03111 genesyspp_parser::yytranslate_ (int t) YY_NOEXCEPT
03112 {
03113 // YYTRANSLATE[TOKEN-NUM] -- Symbol number corresponding to
03114 // TOKEN-NUM as returned by yylex.
03115 static
03116 const signed char
03117 translate_table[] =
03118 {
03119 0, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03120 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03121 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03122 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03123 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03124 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03125 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03126 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03127 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03128 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03129 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03130 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,

```

```

03131 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03132 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03133 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03134 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03135 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03136 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03137 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03138 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03139 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03140 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03141 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03142 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03143 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
03144 2, 2, 2, 2, 2, 2, 1, 2, 3, 4,
03145 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,
03146 15, 16, 17, 18, 19, 20, 21, 22, 23, 24,
03147 25, 26, 27, 28, 29, 30, 31, 32, 33, 34,
03148 35, 36, 37, 38, 39, 40, 41, 42, 43, 44,
03149 45, 46, 47, 48, 49, 50, 51, 52, 53, 54,
03150 55, 56, 57, 58, 59, 60, 61, 62, 63, 64,
03151 65, 66, 67, 68, 69, 70, 71, 72, 73, 74,
03152 75, 76, 77, 78, 79, 80, 81, 82, 83, 84,
03153 85, 86, 87, 88, 89, 90, 91, 92, 93, 94
03154 };
03155 // Last valid token kind.
03156 const int code_max = 349;
03157
03158 if (t <= 0)
03159 return symbol_kind::S_YYEOF;
03160 else if (t <= code_max)
03161 return static_cast<symbol_kind_type> (translate_table[t]);
03162 else
03163 return symbol_kind::S_YYUNDEF;
03164 }
03165
03166 // basic_symbol.
03167 template<typename Base>
03168 genesyspp_parser::basic_symbol<Base>::basic_symbol (const basic_symbol& that)
03169 : Base (that)
03170 , value ()
03171 , location (that.location)
03172 {
03173 switch (this->kind ())
03174 {
03175 case symbol_kind::S_NUMD: // NUMD
03176 case symbol_kind::S_NUMH: // NUMH
03177 case symbol_kind::S_CTEZERO: // CTEZERO
03178 case symbol_kind::S_oLE: // oLE
03179 case symbol_kind::S_oGE: // oGE
03180 case symbol_kind::S_oEQ: // oEQ
03181 case symbol_kind::S_oNE: // oNE
03182 case symbol_kind::S_oAND: // oAND
03183 case symbol_kind::S_oOR: // oOR
03184 case symbol_kind::S_oNAND: // oNAND
03185 case symbol_kind::S_oXOR: // oXOR
03186 case symbol_kind::S_oNOT: // oNOT
03187 case symbol_kind::S_fsIN: // fsIN
03188 case symbol_kind::S_fcOS: // fcOS
03189 case symbol_kind::S_fROUND: // fROUND
03190 case symbol_kind::S_fMOD: // fMOD
03191 case symbol_kind::S_fTRUNC: // fTRUNC
03192 case symbol_kind::S_ffRAC: // ffRAC
03193 case symbol_kind::S_fEXP: // fEXP
03194 case symbol_kind::S_fsQRT: // fsQRT
03195 case symbol_kind::S_fLOG: // fLOG
03196 case symbol_kind::S_fLN: // fLN
03197 case symbol_kind::S_mathMIN: // mathMIN
03198 case symbol_kind::S_mathMAX: // mathMAX
03199 case symbol_kind::S_fVAL: // fVAL
03200 case symbol_kind::S_fEVAL: // fEVAL
03201 case symbol_kind::S_fLENG: // fLENG
03202 case symbol_kind::S_frND1: // frND1
03203 case symbol_kind::S_fEXPO: // fEXPO
03204 case symbol_kind::S_fnORM: // fnORM
03205 case symbol_kind::S_funIF: // funIF
03206 case symbol_kind::S_fwEIB: // fwEIB
03207 case symbol_kind::S_fLOGN: // fLOGN
03208 case symbol_kind::S_fgAMM: // fgAMM
03209 case symbol_kind::S_feRLA: // feRLA
03210 case symbol_kind::S_ftRIA: // ftRIA
03211 case symbol_kind::S_fbETA: // fbETA
03212 case symbol_kind::S_fdISC: // fdISC
03213 case symbol_kind::S_ftNOW: // ftNOW
03214 case symbol_kind::S_ftFIN: // ftFIN
03215 case symbol_kind::S_fMAXREP: // fMAXREP
03216 case symbol_kind::S_fNUMREP: // fNUMREP
03217 case symbol_kind::S_fIDENT: // fIDENT

```



```

03218 case symbol_kind::S_simulEntitiesWIP: // simulEntitiesWIP
03219 case symbol_kind::S_cIF: // cIF
03220 case symbol_kind::S_cELSE: // cELSE
03221 case symbol_kind::S_cFOR: // cFOR
03222 case symbol_kind::S_cTO: // cTO
03223 case symbol_kind::S_cDO: // cDO
03224 case symbol_kind::S_ATRIB: // ATRIB
03225 case symbol_kind::S_CSTAT: // CSTAT
03226 case symbol_kind::S_COUNTER: // COUNTER
03227 case symbol_kind::S_SIMRESP: // SIMRESP
03228 case symbol_kind::S_SIMCTRL: // SIMCTRL
03229 case symbol_kind::S_ftAVG: // ftAVG
03230 case symbol_kind::S_fCOUNT: // fCOUNT
03231 case symbol_kind::S_ILLEGAL: // ILLEGAL
03232 case symbol_kind::S_RESOURCE: // RESOURCE
03233 case symbol_kind::S_fNR: // fNR
03234 case symbol_kind::S_fMR: // fMR
03235 case symbol_kind::S_fIRF: // fIRF
03236 case symbol_kind::S_fRESSEIZES: // fRESSEIZES
03237 case symbol_kind::S_fSTATE: // fSTATE
03238 case symbol_kind::S_fSETSUM: // fSETSUM
03239 case symbol_kind::S_fRESUTIL: // fRESUTIL
03240 case symbol_kind::S_QUEUE: // QUEUE
03241 case symbol_kind::S_fNQ: // fNQ
03242 case symbol_kind::S_ffIRSTINQ: // ffIRSTINQ
03243 case symbol_kind::S_fLASTINQ: // fLASTINQ
03244 case symbol_kind::S_fSAQUE: // fSAQUE
03245 case symbol_kind::S_fAQUE: // fAQUE
03246 case symbol_kind::S_fENTATRANK: // fENTATRANK
03247 case symbol_kind::S_SET: // SET
03248 case symbol_kind::S_fNUMSET: // fNUMSET
03249 case symbol_kind::S_VARI: // VARI
03250 case symbol_kind::S_FORM: // FORM
03251 case symbol_kind::S_fNUMGR: // fNUMGR
03252 case symbol_kind::S_fATRGR: // fATRGR
03253 case symbol_kind::S_input: // input
03254 case symbol_kind::S_expression: // expression
03255 case symbol_kind::S_logicalOr: // logicalOr
03256 case symbol_kind::S_logicalXor: // logicalXor
03257 case symbol_kind::S_logicalAnd: // logicalAnd
03258 case symbol_kind::S_logicalNot: // logicalNot
03259 case symbol_kind::S_relational: // relational
03260 case symbol_kind::S_additive: // additive
03261 case symbol_kind::S_multiplicative: // multiplicative
03262 case symbol_kind::S_power: // power
03263 case symbol_kind::S_unary: // unary
03264 case symbol_kind::S_primary: // primary
03265 case symbol_kind::S_number: // number
03266 case symbol_kind::S_command: // command
03267 case symbol_kind::S_commandIF: // commandIF
03268 case symbol_kind::S_commandFOR: // commandFOR
03269 case symbol_kind::S_function: // function
03270 case symbol_kind::S_kernelFunction: // kernelFunction
03271 case symbol_kind::S_elementFunction: // elementFunction
03272 case symbol_kind::S_trigonFunction: // trigonFunction
03273 case symbol_kind::S_mathFunction: // mathFunction
03274 case symbol_kind::S_probFunction: // probFunction
03275 case symbol_kind::S_userFunction: // userFunction
03276 case symbol_kind::S_listaparm: // listaparm
03277 case symbol_kind::S_illegal: // illegal
03278 case symbol_kind::S_attribute: // attribute
03279 case symbol_kind::S_simulationResponse: // simulationResponse
03280 case symbol_kind::S_simulationControl: // simulationControl
03281 case symbol_kind::S_variable: // variable
03282 case symbol_kind::S_formula: // formula
03283 case symbol_kind::S_assignment: // assignment
03284 case symbol_kind::S_pluginFunction: // pluginFunction
03285 value.copy< obj_t > (YY_MOVE (that.value));
03286 break;
03287
03288 case symbol_kind::S_indexList: // indexList
03289 value.copy< std::string > (YY_MOVE (that.value));
03290 break;
03291
03292 default:
03293 break;
03294 }
03295 }
03296 }
03297
03298
03299
03300
03301 template <typename Base>
03302 genesyspp_parser::symbol_kind_type
03303 genesyspp_parser::basic_symbol<Base>::type_get () const YY_NOEXCEPT
03304 {

```

```

03305 return this->kind ();
03306 }
03307
03308
03309 template <typename Base>
03310 bool
03311 genesyspp_parser::basic_symbol<Base>::empty () const YY_NOEXCEPT
03312 {
03313 return this->kind () == symbol_kind::S_YYEMPTY;
03314 }
03315
03316 template <typename Base>
03317 void
03318 genesyspp_parser::basic_symbol<Base>::move (basic_symbol& s)
03319 {
03320 super_type::move (s);
03321 switch (this->kind ())
03322 {
03323 case symbol_kind::S_NUMD: // NUMD
03324 case symbol_kind::S_NUMH: // NUMH
03325 case symbol_kind::S_CTEZERO: // CTEZERO
03326 case symbol_kind::S_oLE: // oLE
03327 case symbol_kind::S_oGE: // oGE
03328 case symbol_kind::S_oEQ: // oEQ
03329 case symbol_kind::S_oNE: // oNE
03330 case symbol_kind::S_oAND: // oAND
03331 case symbol_kind::S_oOR: // oOR
03332 case symbol_kind::S_oNAND: // oNAND
03333 case symbol_kind::S_oXOR: // oXOR
03334 case symbol_kind::S_oNOT: // oNOT
03335 case symbol_kind::S_fSIN: // fSIN
03336 case symbol_kind::S_fCOS: // fCOS
03337 case symbol_kind::S_fROUND: // fROUND
03338 case symbol_kind::S_fMOD: // fMOD
03339 case symbol_kind::S_fTRUNC: // fTRUNC
03340 case symbol_kind::S_fFRAC: // fFRAC
03341 case symbol_kind::S_fEXP: // fEXP
03342 case symbol_kind::S_fSQRT: // fSQRT
03343 case symbol_kind::S_fLOG: // fLOG
03344 case symbol_kind::S_fLN: // fLN
03345 case symbol_kind::S_mathMIN: // mathMIN
03346 case symbol_kind::S_mathMAX: // mathMAX
03347 case symbol_kind::S_fVAL: // fVAL
03348 case symbol_kind::S_fEVAL: // fEVAL
03349 case symbol_kind::S_fLENG: // fLENG
03350 case symbol_kind::S_fRND1: // fRND1
03351 case symbol_kind::S_fEXPO: // fEXPO
03352 case symbol_kind::S_fNORM: // fNORM
03353 case symbol_kind::S_fUNIF: // fUNIF
03354 case symbol_kind::S_fWEIB: // fWEIB
03355 case symbol_kind::S_fLOGN: // fLOGN
03356 case symbol_kind::S_fGAMM: // fGAMM
03357 case symbol_kind::S_fERLA: // fERLA
03358 case symbol_kind::S_fTRIA: // fTRIA
03359 case symbol_kind::S_fBETA: // fBETA
03360 case symbol_kind::S_fDISC: // fDISC
03361 case symbol_kind::S_fTNOW: // fTNOW
03362 case symbol_kind::S_fTFIN: // fTFIN
03363 case symbol_kind::S_fMAXREP: // fMAXREP
03364 case symbol_kind::S_fNUMREP: // fNUMREP
03365 case symbol_kind::S_fIDENT: // fIDENT
03366 case symbol_kind::S_simulEntitiesWIP: // simulEntitiesWIP
03367 case symbol_kind::S_cIF: // cIF
03368 case symbol_kind::S_cELSE: // cELSE
03369 case symbol_kind::S_cFOR: // cFOR
03370 case symbol_kind::S_cTO: // cTO
03371 case symbol_kind::S_cDO: // cDO
03372 case symbol_kind::S_ATTRIB: // ATTRIB
03373 case symbol_kind::S_CSTAT: // CSTAT
03374 case symbol_kind::S_COUNTER: // COUNTER
03375 case symbol_kind::S_SIMRESP: // SIMRESP
03376 case symbol_kind::S_SIMCTRL: // SIMCTRL
03377 case symbol_kind::S_fTAVG: // fTAVG
03378 case symbol_kind::S_fCOUNT: // fCOUNT
03379 case symbol_kind::S_ILLEGAL: // ILLEGAL
03380 case symbol_kind::S_RESOURCE: // RESOURCE
03381 case symbol_kind::S_fNR: // fNR
03382 case symbol_kind::S_fMR: // fMR
03383 case symbol_kind::S_fIRF: // fIRF
03384 case symbol_kind::S_fRESSEIZES: // fRESSEIZES
03385 case symbol_kind::S_fSTATE: // fSTATE
03386 case symbol_kind::S_fSETSUM: // fSETSUM
03387 case symbol_kind::S_fRESUTIL: // fRESUTIL
03388 case symbol_kind::S_QUEUE: // QUEUE
03389 case symbol_kind::S_fNQ: // fNQ
03390 case symbol_kind::S_fFIRSTINQ: // fFIRSTINQ
03391 case symbol_kind::S_fLASTINQ: // fLASTINQ

```

```

03392 case symbol_kind::S_fSAQUE: // fSAQUE
03393 case symbol_kind::S_fAQUE: // fAQUE
03394 case symbol_kind::S_fENTATRANK: // fENTATRANK
03395 case symbol_kind::S_SET: // SET
03396 case symbol_kind::S_fNUMSET: // fNUMSET
03397 case symbol_kind::S_VARI: // VARI
03398 case symbol_kind::S_FORM: // FORM
03399 case symbol_kind::S_fNUMGR: // fNUMGR
03400 case symbol_kind::S_fATRGR: // fATRGR
03401 case symbol_kind::S_input: // input
03402 case symbol_kind::S_expression: // expression
03403 case symbol_kind::S_logicalOr: // logicalOr
03404 case symbol_kind::S_logicalXor: // logicalXor
03405 case symbol_kind::S_logicalAnd: // logicalAnd
03406 case symbol_kind::S_logicalNot: // logicalNot
03407 case symbol_kind::S_relational: // relational
03408 case symbol_kind::S_additive: // additive
03409 case symbol_kind::S_multiplicative: // multiplicative
03410 case symbol_kind::S_power: // power
03411 case symbol_kind::S_unary: // unary
03412 case symbol_kind::S_primary: // primary
03413 case symbol_kind::S_number: // number
03414 case symbol_kind::S_command: // command
03415 case symbol_kind::S_commandIF: // commandIF
03416 case symbol_kind::S_commandFOR: // commandFOR
03417 case symbol_kind::S_function: // function
03418 case symbol_kind::S_kernelFunction: // kernelFunction
03419 case symbol_kind::S_elementFunction: // elementFunction
03420 case symbol_kind::S_trigonFunction: // trigonFunction
03421 case symbol_kind::S_mathFunction: // mathFunction
03422 case symbol_kind::S_probFunction: // probFunction
03423 case symbol_kind::S_userFunction: // userFunction
03424 case symbol_kind::S_listaparm: // listaparm
03425 case symbol_kind::S_illegal: // illegal
03426 case symbol_kind::S_attribute: // attribute
03427 case symbol_kind::S_simulationResponse: // simulationResponse
03428 case symbol_kind::S_simulationControl: // simulationControl
03429 case symbol_kind::S_variable: // variable
03430 case symbol_kind::S_formula: // formula
03431 case symbol_kind::S_assignment: // assignment
03432 case symbol_kind::S_pluginFunction: // pluginFunction
03433 value.move< obj_t > (YY_MOVE (s.value));
03434 break;
03435
03436 case symbol_kind::S_indexList: // indexList
03437 value.move< std::string > (YY_MOVE (s.value));
03438 break;
03439
03440 default:
03441 break;
03442 }
03443
03444 location = YY_MOVE (s.location);
03445 }
03446
03447 // by_kind.
03448 inline
03449 genesyspp_parser::by_kind::by_kind () YY_NOEXCEPT
03450 : kind_ (symbol_kind::S_YYEMPTY)
03451 {}
03452
03453 #if 201103L <= YY_CPLUSPLUS
03454 inline
03455 genesyspp_parser::by_kind::by_kind (by_kind&& that) YY_NOEXCEPT
03456 : kind_ (that.kind_)
03457 {
03458 that.clear ();
03459 }
03460 #endif
03461
03462 inline
03463 genesyspp_parser::by_kind::by_kind (const by_kind& that) YY_NOEXCEPT
03464 : kind_ (that.kind_)
03465 {}
03466
03467 inline
03468 genesyspp_parser::by_kind::by_kind (token_kind_type t) YY_NOEXCEPT
03469 : kind_ (yytranslate_ (t))
03470 {}
03471
03472
03473
03474 inline
03475 void
03476 genesyspp_parser::by_kind::clear () YY_NOEXCEPT
03477 {
03478 kind_ = symbol_kind::S_YYEMPTY;

```

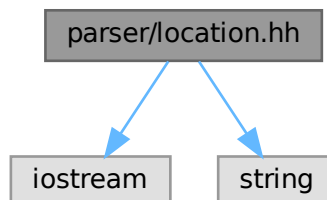
```
03479 }
03480
03481 inline
03482 void
03483 genesyspp_parser::by_kind::move (by_kind& that)
03484 {
03485 kind_ = that.kind_;
03486 that.clear ();
03487 }
03488
03489 inline
03490 genesyspp_parser::symbol_kind_type
03491 genesyspp_parser::by_kind::kind () const YY_NOEXCEPT
03492 {
03493 return kind_;
03494 }
03495
03496 inline
03497 genesyspp_parser::symbol_kind_type
03498 genesyspp_parser::by_kind::type_get () const YY_NOEXCEPT
03499 {
03500 return this->kind ();
03501 }
03502 }
03503
03504
03505 } // yy
03506 #line 3507 "../GenesysParser.h"
03507
03508
03509
03510
03511 #endif // !YY_YG_GENESYSPARSER_H_INCLUDED
```

## 16.340 parser/location.hh File Reference

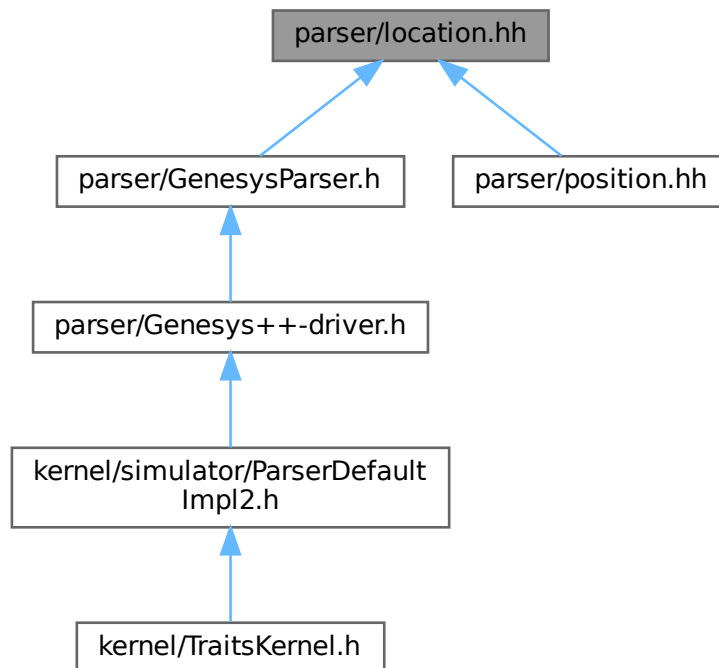
```
#include <iostream>
```

```
#include <string>
```

Include dependency graph for location.hh:



This graph shows which files directly or indirectly include this file:



## Classes

- class [yy::position](#)  
*A point in a source file.*
- class [yy::location](#)  
*Two points in a source file.*

## Macros

- `#define YY_NULLPTR ((void*)0)`

## Functions

- [position](#) & [yy::operator+=](#) ([position](#) &res, [position::counter\\_type](#) width)  
*Add width columns, in place.*
- [position](#) [yy::operator+](#) ([position](#) res, [position::counter\\_type](#) width)  
*Add width columns.*
- [position](#) & [yy::operator-=](#) ([position](#) &res, [position::counter\\_type](#) width)  
*Subtract width columns, in place.*
- [position](#) [yy::operator-](#) ([position](#) res, [position::counter\\_type](#) width)  
*Subtract width columns.*
- `template<typename YYChar >`  
`std::basic_ostream< YYChar > & yy::operator<< (std::basic_ostream< YYChar > &ostr, const position &pos)`  
*Intercept output stream redirection.*

- `location & yy::operator+= (location &res, const location &end)`  
*Join two locations, in place.*
- `location yy::operator+ (location res, const location &end)`  
*Join two locations.*
- `location & yy::operator+= (location &res, location::counter_type width)`  
*Add width columns to the end position, in place.*
- `location yy::operator+ (location res, location::counter_type width)`  
*Add width columns to the end position.*
- `location & yy::operator-= (location &res, location::counter_type width)`  
*Subtract width columns to the end position, in place.*
- `location yy::operator- (location res, location::counter_type width)`  
*Subtract width columns to the end position.*
- `template<typename YYChar >  
std::basic_ostream< YYChar > & yy::operator<< (std::basic_ostream< YYChar > &ostr, const location &loc)`  
*Intercept output stream redirection.*

### 16.340.1 Detailed Description

Define the `yy::location` class.

### 16.340.2 Function Documentation

#### 16.340.2.1 `operator<<()` [1/2]

```
template<typename YYChar >
std::basic_ostream< YYChar > & yy::operator<< (
 std::basic_ostream< YYChar > & ostr,
 const location & loc)
```

Intercept output stream redirection.

##### Parameters

|             |                                         |
|-------------|-----------------------------------------|
| <i>ostr</i> | the destination output stream           |
| <i>loc</i>  | a reference to the location to redirect |

Avoid duplicate information.

#### 16.340.2.2 `operator<<()` [2/2]

```
template<typename YYChar >
std::basic_ostream< YYChar > & yy::operator<< (
 std::basic_ostream< YYChar > & ostr,
 const position & pos)
```

Intercept output stream redirection.

##### Parameters

|             |                                         |
|-------------|-----------------------------------------|
| <i>ostr</i> | the destination output stream           |
| <i>pos</i>  | a reference to the position to redirect |

## 16.341 location.hh

[Go to the documentation of this file.](#)

```

00001 // A Bison parser, made by GNU Bison 3.8.2.
00002
00003 // Locations for Bison parsers in C++
00004
00005 // Copyright (C) 2002-2015, 2018-2021 Free Software Foundation, Inc.
00006
00007 // This program is free software: you can redistribute it and/or modify
00008 // it under the terms of the GNU General Public License as published by
00009 // the Free Software Foundation, either version 3 of the License, or
00010 // (at your option) any later version.
00011
00012 // This program is distributed in the hope that it will be useful,
00013 // but WITHOUT ANY WARRANTY; without even the implied warranty of
00014 // MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00015 // GNU General Public License for more details.
00016
00017 // You should have received a copy of the GNU General Public License
00018 // along with this program. If not, see <https://www.gnu.org/licenses/>.
00019
00020 // As a special exception, you may create a larger work that contains
00021 // part or all of the Bison parser skeleton and distribute that work
00022 // under terms of your choice, so long as that work isn't itself a
00023 // parser generator using the skeleton or a modified version thereof
00024 // as a parser skeleton. Alternatively, if you modify or redistribute
00025 // the parser skeleton itself, you may (at your option) remove this
00026 // special exception, which will cause the skeleton and the resulting
00027 // Bison output files to be licensed under the GNU General Public
00028 // License without this special exception.
00029
00030 // This special exception was added by the Free Software Foundation in
00031 // version 2.2 of Bison.
00032
00033 #ifndef YY_Y_LOCATION_HH_INCLUDED
00034 # define YY_Y_LOCATION_HH_INCLUDED
00035
00036 # include <iostream>
00037 # include <string>
00038
00039 # ifndef YY_NULLPTR
00040 # if defined __cplusplus
00041 # if 201103L <= __cplusplus
00042 # define YY_NULLPTR nullptr
00043 # else
00044 # define YY_NULLPTR 0
00045 # endif
00046 # else
00047 # define YY_NULLPTR ((void*)0)
00048 # endif
00049 # endif
00050
00051 namespace yy {
00052 #line 58 "../location.hh"
00053
00054 class position
00055 {
00056 public:
00057 typedef const std::string filename_type;
00058 typedef int counter_type;
00059
00060 explicit position (filename_type* f = YY_NULLPTR,
00061 counter_type l = 1,
00062 counter_type c = 1)
00063 : filename (f)
00064 , line (l)
00065 , column (c)
00066 {}
00067
00068 void initialize (filename_type* fn = YY_NULLPTR,
00069 counter_type l = 1,
00070 counter_type c = 1)
00071 {
00072 filename = fn;
00073 line = l;
00074 column = c;
00075 }
00076
00077 void lines (counter_type count = 1)
00078 {
00079 if (count)
00080 {
00081 column = 1;
00082 line = add_ (line, count, 1);
00083 }
00084 }
00085
00086 void columns (counter_type count = 1)

```

```

00102 {
00103 column = add_ (column, count, 1);
00104 }
00108 filename_type* filename;
00110 counter_type line;
00112 counter_type column;
00113
00114 private:
00116 static counter_type add_ (counter_type lhs, counter_type rhs, counter_type min)
00117 {
00118 return lhs + rhs < min ? min : lhs + rhs;
00119 }
00120 };
00121
00123 inline position&
00124 operator+= (position& res, position::counter_type width)
00125 {
00126 res.columns (width);
00127 return res;
00128 }
00129
00131 inline position
00132 operator+ (position res, position::counter_type width)
00133 {
00134 return res += width;
00135 }
00136
00138 inline position&
00139 operator-= (position& res, position::counter_type width)
00140 {
00141 return res += -width;
00142 }
00143
00145 inline position
00146 operator- (position res, position::counter_type width)
00147 {
00148 return res -= width;
00149 }
00150
00155 template <typename YYChar>
00156 std::basic_ostream<YYChar>&
00157 operator« (std::basic_ostream<YYChar>& ostr, const position& pos)
00158 {
00159 if (pos.filename)
00160 ostr « *pos.filename « " ";
00161 return ostr « pos.line « " " « pos.column;
00162 }
00163
00165 class location
00166 {
00167 public:
00169 typedef position::filename_type filename_type;
00171 typedef position::counter_type counter_type;
00172
00174 location (const position& b, const position& e)
00175 : begin (b)
00176 , end (e)
00177 {}
00178
00180 explicit location (const position& p = position ())
00181 : begin (p)
00182 , end (p)
00183 {}
00184
00186 explicit location (filename_type* f,
00187 counter_type l = 1,
00188 counter_type c = 1)
00189 : begin (f, l, c)
00190 , end (f, l, c)
00191 {}
00192
00193
00195 void initialize (filename_type* f = YY_NULLPTR,
00196 counter_type l = 1,
00197 counter_type c = 1)
00198 {
00199 begin.initialize (f, l, c);
00200 end = begin;
00201 }
00202
00205 public:
00207 void step ()
00208 {
00209 begin = end;
00210 }
00211
00213 void columns (counter_type count = 1)

```



```

00214 {
00215 end += count;
00216 }
00217
00219 void lines (counter_type count = 1)
00220 {
00221 end.lines (count);
00222 }
00226 public:
00228 position begin;
00230 position end;
00231 };
00232
00234 inline location&
00235 operator+= (location& res, const location& end)
00236 {
00237 res.end = end.end;
00238 return res;
00239 }
00240
00242 inline location
00243 operator+ (location res, const location& end)
00244 {
00245 return res += end;
00246 }
00247
00249 inline location&
00250 operator+= (location& res, location::counter_type width)
00251 {
00252 res.columns (width);
00253 return res;
00254 }
00255
00257 inline location
00258 operator+ (location res, location::counter_type width)
00259 {
00260 return res += width;
00261 }
00262
00264 inline location&
00265 operator-= (location& res, location::counter_type width)
00266 {
00267 return res += -width;
00268 }
00269
00271 inline location
00272 operator- (location res, location::counter_type width)
00273 {
00274 return res -= width;
00275 }
00276
00283 template <typename YYChar>
00284 std::basic_ostream<YYChar>&
00285 operator« (std::basic_ostream<YYChar>& ostr, const location& loc)
00286 {
00287 location::counter_type end_col
00288 = 0 < loc.end.column ? loc.end.column - 1 : 0;
00289 ostr « loc.begin;
00290 if (loc.end.filename
00291 && (!loc.begin.filename
00292 || *loc.begin.filename != *loc.end.filename))
00293 ostr « '-' « loc.end.filename « ':' « loc.end.line « '.' « end_col;
00294 else if (loc.begin.line < loc.end.line)
00295 ostr « '-' « loc.end.line « '.' « end_col;
00296 else if (loc.begin.column < end_col)
00297 ostr « '-' « end_col;
00298 return ostr;
00299 }
00300
00301 } // yy
00302 #line 303 "../location.hh"
00303
00304 #endif // !YY_YY_LOCATION_HH_INCLUDED

```

## 16.342 obj\_t.h

```

00001 #ifndef OBJ_T_H
00002 #define OBJ_T_H
00003 #include <string>
00004
00005 class obj_t {
00006 public:
00007 obj_t();
00008 virtual ~obj_t();
00009 obj_t(double v, std::string t);

```

```

00010 obj_t(double v, std::string t, unsigned long uid);
00011 double valor;
00012 std::string tipo;
00013 unsigned long id;
00014 bool isReference;
00015 bool isAttributeReference;
00016 };
00017
00018 #endif

```

## 16.343 position.hh

```

00001 // A Bison parser, made by GNU Bison 3.8.2.
00002
00003 // Starting with Bison 3.2, this file is useless: the structure it
00004 // used to define is now defined in "location.hh".
00005 //
00006 // To get rid of this file:
00007 // 1. add '%require "3.2"' (or newer) to your grammar file
00008 // 2. remove references to this file from your build system
00009 // 3. if you used to include it, include "location.hh" instead.
00010
00011 #include "location.hh"

```

## 16.344 stack.hh

```

00001 // A Bison parser, made by GNU Bison 3.8.2.
00002
00003 // Starting with Bison 3.2, this file is useless: the structure it
00004 // used to define is now defined with the parser itself.
00005 //
00006 // To get rid of this file:
00007 // 1. add '%require "3.2"' (or newer) to your grammar file
00008 // 2. remove references to this file from your build system.

```

## 16.345 AIAssistant.h

```

00001 #ifndef AIASSISTANTCOMPONENT_H
00002 #define AIASSISTANTCOMPONENT_H
00003
00004 #include "kernel/simulator/model/ModelComponent.h"
00005 #include "plugins/data/AI/AISupport.h"
00006
00007 #include <map>
00008
00009 class AIAssistant : public ModelComponent {
00010 public:
00011 AIAssistant(Model* model, std::string name = "");
00012 virtual ~AIAssistant() override = default;
00013
00014 static PluginInformation* GetPluginInformation();
00015 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00016 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00017
00018 void setAISupport(AISupport* support);
00019 AISupport* getAISupport() const;
00020 void setAISupportName(std::string value);
00021 std::string getAISupportName() const;
00022 void setPromptTemplate(std::string value);
00023 std::string getPromptTemplate() const;
00024 void setConversationScope(std::string value);
00025 std::string getConversationScope() const;
00026 void setConversationKeyExpression(std::string value);
00027 std::string getConversationKeyExpression() const;
00028 void setActionMappings(std::string value);
00029 std::string getActionMappings() const;
00030 void setDelayExpression(std::string value);
00031 std::string getDelayExpression() const;
00032 void setDefaultOutputPort(unsigned int value);
00033 unsigned int getDefaultOutputPort() const;
00034 void setErrorOutputPort(unsigned int value);
00035 unsigned int getErrorOutputPort() const;
00036 void setAllowDirectOutputPort(bool value);
00037 bool isAllowDirectOutputPort() const;
00038 void setMinimumConfidence(double value);
00039 double getMinimumConfidence() const;
00040 void setSaveConfidenceAttribute(std::string value);
00041 std::string getSaveConfidenceAttribute() const;
00042 void setSaveSuccessAttribute(std::string value);

```

```

00043 std::string getSaveSuccessAttribute() const;
00044 std::string getLastPrompt() const;
00045 std::string getLastResponse() const;
00046 std::string getLastError() const;
00047 bool renderPrompt(std::string& result, std::string& error);
00048
00049 virtual std::string show() override;
00050
00051 protected:
00052 virtual bool _loadInstance(PersistenceRecord* fields) override;
00053 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00054 virtual bool _check(std::string& errorMessage) override;
00055 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00056
00057 private:
00058 struct ParsedResponse {
00059 bool valid = false;
00060 std::string action;
00061 int outputPort = -1;
00062 double confidence = 1.0;
00063 std::string explanation;
00064 std::map<std::string, double> values;
00065 std::string error;
00066 };
00067
00068 AISupport* _resolveSupport();
00069 bool _renderPrompt(std::string& result, std::string& error);
00070 std::string _conversationKey(Entity* entity, std::string& error);
00071 ParsedResponse _parseResponse(const std::string& content) const;
00072 std::map<std::string, unsigned int> _parseActionMappings() const;
00073 unsigned int _selectPort(const ParsedResponse& response, bool& success) const;
00074 void _forward(Entity* entity, unsigned int port, double delay);
00075 void _fail(Entity* entity, const std::string& message);
00076
00077 const struct DEFAULT_VALUES {
00078 std::string aiSupportName = "";
00079 std::string promptTemplate = "Decide the route for this entity. Return only JSON with action,
confidence, explanation, and numeric values fields.";
00080 std::string conversationScope = "PerEntity";
00081 std::string conversationKeyExpression = "Entity.Id";
00082 std::string actionMappings = "route:0";
00083 std::string delayExpression = "0";
00084 unsigned int defaultOutputPort = 0;
00085 unsigned int errorOutputPort = 0;
00086 bool allowDirectOutputPort = false;
00087 double minimumConfidence = 0.0;
00088 std::string saveConfidenceAttribute = "";
00089 std::string saveSuccessAttribute = "";
00090 } DEFAULT;
00091
00092 AISupport* _aiSupport = nullptr;
00093 std::string _aiSupportName = DEFAULT.aiSupportName;
00094 std::string _promptTemplate = DEFAULT.promptTemplate;
00095 std::string _conversationScope = DEFAULT.conversationScope;
00096 std::string _conversationKeyExpression = DEFAULT.conversationKeyExpression;
00097 std::string _actionMappings = DEFAULT.actionMappings;
00098 std::string _delayExpression = DEFAULT.delayExpression;
00099 unsigned int _defaultOutputPort = DEFAULT.defaultOutputPort;
00100 unsigned int _errorOutputPort = DEFAULT.errorOutputPort;
00101 bool _allowDirectOutputPort = DEFAULT.allowDirectOutputPort;
00102 double _minimumConfidence = DEFAULT.minimumConfidence;
00103 std::string _saveConfidenceAttribute = DEFAULT.saveConfidenceAttribute;
00104 std::string _saveSuccessAttribute = DEFAULT.saveSuccessAttribute;
00105 std::string _lastPrompt;
00106 std::string _lastResponse;
00107 std::string _lastError;
00108 };
00109
00110 #endif

```

## 16.346 MarkovChain.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: MarkovChain.h
00009 * Author: rlcancian
00010 *
00011 * Created on 24 de Outubro de 2019, 18:26
00012 */
00013

```

```

00014 #ifndef MARKOVCHAIN_H
00015 #define MARKOVCHAIN_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "kernel/statistics/Sampler_if.h"
00019
00020 class MarkovChain : public ModelComponent {
00021 public: // constructors
00022 MarkovChain(Model* model, std::string name = "");
00023 virtual ~MarkovChain() override;
00024 public: // virtual
00025 virtual std::string show() override;
00026 public: // static
00027 static PluginInformation* GetPluginInformation();
00028 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00029 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00030 public: // get and set
00031 void setTransitionProbabilityMatrix(ModelDataDefinition* transitionMatrix);
00032 ModelDataDefinition* getTransitionMatrix() const;
00033 void setInitialDistribution(ModelDataDefinition* initialDistribution);
00034 ModelDataDefinition* getInitialDistribution() const;
00035 void setCurrentState(ModelDataDefinition* currentState);
00036 ModelDataDefinition* getCurrentState() const;
00037 protected: // virtual
00038 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00039 virtual bool _loadInstance(PersistenceRecord *fields) override;
00040 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00041 protected: // virtual
00042 virtual void _initBetweenReplications() override;
00043 virtual bool _check(std::string& errorMessage) override;
00044 // virtual void _createInternalAndAttachedData() override;
00045 protected:
00046 // virtual void _createInternalStatisticReporters() override;
00047 // virtual void _createNonEditableDataDefinitions() override;
00048 virtual void _createEditableDataDefinitions() override;
00049 // virtual void _createAttachedAttributes() override;
00050
00051 private: // methods
00052 unsigned int _drawNextState(Entity* entity, unsigned int currentState);
00053 double _readStateValue(Entity* entity) const;
00054 void _writeStateValue(Entity* entity, double value);
00055 double _readTransitionProbability(Entity* entity, unsigned int fromState, unsigned int toState)
00056 const;
00057 unsigned int _stateCount() const;
00057 private: // attributes 1:1
00058 ModelDataDefinition* _transitionProbMatrix = nullptr;
00059 ModelDataDefinition* _initialDistribution = nullptr;
00060 ModelDataDefinition* _currentState = nullptr;
00061 private: // attributes 1:n
00062 Sampler_if* _sampler = nullptr;
00063 };
00064
00065 #endif /* MARKOVCHAIN_H */

```

## 16.347 BacteriaColony.h

```

00001 /*
00002 * File: BacteriaColony.h
00003 * Author: rlcancian
00004 *
00005 * Created on 17 de Abril de 2022
00006 */
00007
00008 #ifndef BACTERIACOLONY_H
00009 #define BACTERIACOLONY_H
00010
00011 #include "../kernel/simulator/model/ModelComponent.h"
00012 #include "kernel/simulator/Plugin.h"
00013 #include "kernel/util/Util.h"
00014 #include "plugins/data/BiochemicalSimulation/BioNetwork.h"
00015 #include "plugins/data/BiochemicalSimulation/GroProgram.h"
00016 #include "plugins/data/BiochemicalSimulation/GroProgramRuntime.h"
00017 #include "plugins/data/BiochemicalSimulation/BacteriaSignalGrid.h"
00018
00019 #include <cstdint>
00020 #include <map>
00021 #include <string>
00022 #include <vector>
00023
00024 class BacteriaColony : public ModelComponent {
00025 public:
00026 struct BarrierSegment {
00027 double x1 = 0.0;
00028 double y1 = 0.0;
00029 double x2 = 0.0;

```

```

00038 double y2 = 0.0;
00039 };
00040
00041 struct BacteriumState {
00042 unsigned int id = 0;
00043 unsigned int parentId = 0;
00044 unsigned int generation = 0;
00045 std::string programName = "";
00046 std::vector<double> programArguments;
00047 unsigned int divisionCount = 0;
00048 double birthTime = 0.0;
00049 double lastUpdateTime = 0.0;
00050 double lastDivisionTime = 0.0;
00051 double positionX = 0.0;
00052 double positionY = 0.0;
00053 double directionRadians = 0.0;
00054 double volume = 1.0;
00055 double size = 1.0;
00056 double gfp = 0.0;
00057 double rfp = 0.0;
00058 double yfp = 0.0;
00059 double cfp = 0.0;
00060 double speed = 0.1;
00061 unsigned int tickCount = 0;
00062 unsigned int gridX = 0;
00063 unsigned int gridY = 0;
00064 bool hasExplicitGridPosition = false;
00065 bool justDivided = false;
00066 bool daughter = false;
00067 bool alive = true;
00068 std::map<std::string, double> runtimeVariables;
00069 };
00070
00071 public:
00072 BacteriaColony(Model* model, std::string name = "");
00073 virtual ~BacteriaColony() override = default;
00074
00075 public:
00076 virtual std::string show() override;
00077
00078 public: // static plugin interface
00079 static PluginInformation* GetPluginInformation();
00080 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00081 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00082
00083 public:
00084 void setGroProgram(GroProgram* groProgram);
00085 GroProgram* getGroProgram() const;
00086 void setSimulationStep(double simulationStep);
00087 double getSimulationStep() const;
00088 void setNumSteps(unsigned int numSteps);
00089 unsigned int getNumSteps() const;
00090 void setColonyTimeUnit(Util::TimeUnit colonyTimeUnit);
00091 Util::TimeUnit getColonyTimeUnit() const;
00092 double getColonyTime() const;
00093 void setInitialPopulation(unsigned int initialPopulation);
00094 unsigned int getInitialPopulation() const;
00095 unsigned int getPopulationSize() const;
00096 void setBioNetwork(BioNetwork* bioNetwork);
00097 BioNetwork* getBioNetwork() const;
00098 void setSignalGrid(BacteriaSignalGrid* signalGrid);
00099 BacteriaSignalGrid* getSignalGrid() const;
00100 std::size_t getInternalBacteriaCount() const;
00101 const BacteriumState& getBacteriumState(std::size_t index) const;
00102 double getBacteriumAge(std::size_t index) const;
00103 double getBacteriumPositionX(std::size_t index) const;
00104 double getBacteriumPositionY(std::size_t index) const;
00105 double getBacteriumDirectionRadians(std::size_t index) const;
00106 double getBacteriumVolume(std::size_t index) const;
00107 double getBacteriumSize(std::size_t index) const;
00108 double getBacteriumGfp(std::size_t index) const;
00109 double getBacteriumRfp(std::size_t index) const;
00110 double getBacteriumRuntimeVariableValue(std::size_t index, const std::string& variableName) const;
00111 bool hasBacteriumRuntimeVariable(std::size_t index, const std::string& variableName) const;
00112 double getRuntimeVariableValue(const std::string& variableName) const;
00113 bool hasRuntimeVariable(const std::string& variableName) const;
00114 double getSignalValueAt(unsigned int x, unsigned int y) const;
00115 std::vector<std::vector<double>> getSignalMatrix() const;
00116 std::string getSignalMatrixDump(std::size_t maxRows = 0, std::size_t maxColumns = 0) const;
00117 double getBacteriumLocalSignal(std::size_t index) const;
00118 void setChemostatMode(bool enabled);
00119 bool getChemostatMode() const;
00120 void addBarrier(double x1, double y1, double x2, double y2);
00121 const std::vector<BarrierSegment>& getBarriers() const;
00122 const std::vector<double>& getMappedCellValues() const;
00123 const std::string& getMappedCellExpression() const;
00124 void setGridWidth(unsigned int gridWidth);

```

```

00167 unsigned int getWidth() const;
00169 void setGridHeight(unsigned int gridHeight);
00171 unsigned int getHeight() const;
00173 GroProgramRuntime::ExecutionResult executeGroProgram();
00174
00175 protected:
00176 virtual bool _loadInstance(PersistenceRecord* fields) override;
00177 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00178 virtual bool _check(std::string& errorMessage) override;
00179 virtual void _initBetweenReplications() override;
00180 // virtual void _createInternalAndAttachedData() override;
00181 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00182
00183
00184 protected:
00185 // virtual void _createInternalStatisticReporters() override;
00186 // virtual void _createNonEditableDataDefinitions() override;
00187 // virtual void _createEditableDataDefinitions() override;
00188 virtual void _createAttachedAttributes() override;
00189
00190 private:
00191 const struct DEFAULT_VALUES {
00192 const std::string groProgramName = "";
00193 const std::string bioNetworkName = "";
00194 const std::string signalGridName = "";
00195 const double simulationStep = 0.01;
00196 const unsigned int numSteps = 100;
00197 const Util::TimeUnit colonyTimeUnit = Util::TimeUnit::second;
00198 const unsigned int initialPopulation = 1;
00199 const unsigned int gridWidth = 1;
00200 const unsigned int gridHeight = 1;
00201 } DEFAULT;
00202
00203 GroProgram* _groProgram = nullptr;
00204 BioNetwork* _bioNetwork = nullptr;
00205 BacteriaSignalGrid* _signalGrid = nullptr;
00206 double _simulationStep = DEFAULT.simulationStep;
00207 unsigned int _currentStep = 0;
00208 unsigned int _numSteps = DEFAULT.numSteps;
00209 Util::TimeUnit _colonyTimeUnit = DEFAULT.colonyTimeUnit;
00210 unsigned int _initialPopulation = DEFAULT.initialPopulation;
00211 unsigned int _populationSize = DEFAULT.initialPopulation;
00212 unsigned int _gridWidth = DEFAULT.gridWidth;
00213 unsigned int _gridHeight = DEFAULT.gridHeight;
00214 unsigned int _nextBacteriumId = 1;
00215 unsigned int _colonyTickCount = 0;
00216 bool _chemostatMode = false;
00217 std::vector<BacteriumState> _bacteria;
00218 std::map<std::string, double> _runtimeVariables;
00219 std::vector<BarrierSegment> _barriers;
00220 std::vector<double> _mappedCellValues;
00221 std::string _mappedCellExpression;
00222 std::vector<double> _signalField;
00223 struct GroSeedDefinition {
00224 unsigned int gridX = 0;
00225 unsigned int gridY = 0;
00226 std::string programName = "";
00227 std::vector<double> programArguments;
00228 };
00229 std::vector<GroSeedDefinition> _groSeedDefinitions;
00230 std::string _groSeedSignature = "";
00231 bool _groSeedsApplied = false;
00232
00233 private:
00234 // SignalGrid remains the authoritative source for spatial configuration
00235 // when attached; colony timing is always read from ModelSimulation.
00236 bool _usesSignalGridDimensions() const;
00237 void _synchronizeGridDimensionsFromSignalGrid();
00238 bool _resetRuntimeSignalField(std::string& errorMessage);
00239 bool _collectBioNetworkSpecies(std::vector<BioSpecies*>& species, std::string& errorMessage)
00240 const;
00241 bool _collectBioNetworkSpeciesByIdentifier(std::map<std::string, BioSpecies*>&
00242 speciesByIdentifier,
00243 std::string& errorMessage) const;
00244 void _appendBioNetworkContextVariables(GroProgramRuntimeState& runtimeState, std::string&
00245 errorMessage) const;
00246 bool _applyBioNetworkAssignments(const std::map<std::string, double>& assignedVariables,
00247 std::string& errorMessage);
00248 void _removeBioNetworkAssignmentVariables(std::map<std::string, double>& variables) const;
00249 bool _refreshRuntimeConfigurationFromGroProgram(std::string& errorMessage);
00250 void _ensureGroSeededBacteriaInitialized();
00251 void _rebuildInternalBacteriaFromGroSeeds();
00252 std::size_t _signalIndex(unsigned int x, unsigned int y) const;
00253 double _signalValueAt(unsigned int x, unsigned int y) const;
00254 void _setSignalValueAt(unsigned int x, unsigned int y, double value);
00255 void _addSignalAt(unsigned int x, unsigned int y, double value);
00256 std::vector<std::vector<double>> _buildSignalMatrix() const;

```

```

00253 std::string _buildSignalFieldDump(std::size_t maxRows = 0, std::size_t maxColumns = 0) const;
00254 void _captureSignalFieldSnapshot(GroProgramRuntime::ExecutionResult& result,
00255 std::size_t maxRows = 0,
00256 std::size_t maxColumns = 0) const;
00257 double _computeNeighborSignalSum(unsigned int x, unsigned int y) const;
00258 unsigned int _computeLocalBacteriaCount(unsigned int x, unsigned int y) const;
00259 void _applySignalFieldStep();
00260 void _applyBacteriumSignalMutations(const BacteriumState& bacterium,
00261 const std::vector<GroProgramRuntime::SignalMutation>&
mutations);
00262 void _rebuildInternalBacteria(unsigned int populationSize);
00263 void _resizeInternalBacteria(unsigned int populationSize);
00264 void _applyRuntimePopulationMutations(const std::vector<GroProgramRuntime::PopulationMutation>&
mutations,
 unsigned int finalPopulationSize);
00265 bool _applyColonyMutations(const std::vector<GroProgramRuntime::ColonyMutation>& mutations,
00266 GroProgramRuntime::ExecutionResult& result,
00267 bool allowStructureMutations,
00268 std::string& errorMessage);
00269 bool _executeSeededNamedGroPrograms(const GroProgramIr& ir, GroProgramRuntime::ExecutionResult&
result);
00270 bool _executeBacteriumScopedGroProgram(const GroProgramIr& ir, GroProgramRuntime::ExecutionResult&
result);
00271 bool _containsBacteriumScopedOnlyUnsupportedCommand(const std::vector<GroProgramIr::Command>&
commands,
 std::string& unsupportedCommand) const;
00272 std::size_t _findBacteriumIndexById(unsigned int bacteriumId) const;
00273 GroProgramRuntimeState _createBacteriumRuntimeState(const BacteriumState& bacterium,
00274 std::size_t bacteriumIndex) const;
00275 void _applyBacteriumScopedPopulationMutations(unsigned int bacteriumId,
00276 unsigned int parentGeneration,
00277 const
std::vector<GroProgramRuntime::PopulationMutation>& mutations,
 GroProgramRuntime::ExecutionResult& result);
00278 void _appendBacterium(unsigned int parentId = 0, unsigned int generation = 0,
00279 const std::string& programName = "");
00280 void _appendBacterium(const GroSeedDefinition& seedDefinition);
00281 void _removeBacteria(unsigned int amount);
00282 bool _removeBacteriumById(unsigned int bacteriumId);
00283 void _refreshBacteriaUpdateTime();
00284 void _applyBacteriumGrowth(BacteriumState& bacterium) const;
00285 void _rebuildBacteriaGridPositions();
00286 void _assignBacteriumGridPosition(BacteriumState& bacterium, std::size_t index) const;
00287 void _initializeBacteriumPhenotype(BacteriumState& bacterium, std::size_t index) const;
00288 void _syncBacteriumSpatialState(BacteriumState& bacterium, const GroProgramRuntimeState&
runtimeState) const;
00289 void _updateBacteriumSpatialMotion(BacteriumState& bacterium) const;
00290 void _setBacteriumPosition(BacteriumState& bacterium, double x, double y) const;
00291 double _clampPositionX(double value) const;
00292 double _clampPositionY(double value) const;
00293 void _bindProgramArguments(GroProgramRuntimeState& runtimeState,
00294 const std::vector<std::string>& parameterNames,
00295 const std::vector<double>& arguments) const;
00296 };
00297
00300 #endif /* BACTERIACOLONY_H */

```

## 16.348 BioRunnerCommand.h

```

00001 /*
00002 * File: BioRunnerCommand.h
00003 * Author: rlcancian
00004 *
00005 * Component that executes BioSimulatorRunner commands when entities arrive.
00006 */
00007
00008 #ifndef BIORUNNERCOMMAND_H
00009 #define BIORUNNERCOMMAND_H
00010
00011 #include <string>
00012
00013 #include "../kernel/simulator/model/ModelComponent.h"
00014 #include "kernel/simulator/Plugin.h"
00015 #include "plugins/data/BiochemicalSimulation/BioSimulatorRunner.h"
00016
00017 class BioRunnerCommand : public ModelComponent {
00018 public:
00019 BioRunnerCommand(Model* model, std::string name = "");
00020 virtual ~BioRunnerCommand() override = default;
00021
00022 public:
00023 static PluginInformation* GetPluginInformation();
00024 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00025 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00026 };

```

```

00034 public:
00035 void setRunner(BioSimulatorRunner* runner);
00036 BioSimulatorRunner* getRunner() const;
00037 void setCommand(std::string command);
00038 std::string getCommand() const;
00039 void setLastSucceeded(bool lastSucceeded);
00040 bool getLastSucceeded() const;
00041 void setLastStatus(std::string lastStatus);
00042 std::string getLastStatus() const;
00043 void setLastMessage(std::string lastMessage);
00044 std::string getLastMessage() const;
00045
00046 virtual std::string show() override;
00047
00048 protected:
00049 virtual bool _loadInstance(PersistenceRecord* fields) override;
00050 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00051 virtual bool _check(std::string& errorMessage) override;
00052 // virtual void _createInternalAndAttachedData() override;
00053 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00054
00055
00056 protected:
00057 // virtual void _createInternalStatisticReporters() override;
00058 // virtual void _createNonEditableDataDefinitions() override;
00059 virtual void _createEditableDataDefinitions() override;
00060 // virtual void _createAttachedAttributes() override;
00061
00062 private:
00063 void _forwardEntity(Entity* entity);
00064
00065 private:
00066 const struct DEFAULT_VALUES {
00067 std::string runnerName = "";
00068 std::string command = "validateModel()";
00069 bool lastSucceeded = false;
00070 std::string lastStatus = "";
00071 std::string lastMessage = "";
00072 } DEFAULT;
00073
00074 BioSimulatorRunner* _runner = nullptr;
00075 std::string _command = DEFAULT.command;
00076 bool _lastSucceeded = DEFAULT.lastSucceeded;
00077 std::string _lastStatus = DEFAULT.lastStatus;
00078 std::string _lastMessage = DEFAULT.lastMessage;
00079 };
00080
00081 #endif /* BIORUNNERCOMMAND_H */

```

## 16.349 BioSimulate.h

```

00001 /*
00002 * File: BioSimulate.h
00003 * Author: rlcancian
00004 *
00005 * Component that executes BioNetwork simulation when entities arrive.
00006 */
00007
00008 #ifndef BIOSIMULATE_H
00009 #define BIOSIMULATE_H
00010
00011 #include <string>
00012
00013 #include "../kernel/simulator/model/ModelComponent.h"
00014 #include "kernel/simulator/Plugin.h"
00015 #include "plugins/data/BiochemicalSimulation/BioNetwork.h"
00016
00024 class BioSimulate : public ModelComponent {
00025 public:
00026 BioSimulate(Model* model, std::string name = "");
00027 virtual ~BioSimulate() override = default;
00028
00029 public:
00030 static PluginInformation* GetPluginInformation();
00031 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00032 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00033
00034 public:
00035 void setBioNetwork(BioNetwork* bioNetwork);
00036 BioNetwork* getBioNetwork() const;
00037 void setUseNetworkTimeWindow(bool useNetworkTimeWindow);
00038 bool getUseNetworkTimeWindow() const;
00039 void setStartTime(double startTime);
00040 double getStartTime() const;
00041 void setStopTime(double stopTime);

```



```

00042 double getStopTime() const;
00043 void setStepSize(double stepSize);
00044 double getStepSize() const;
00045 void setLastSucceeded(bool lastSucceeded);
00046 bool getLastSucceeded() const;
00047 void setLastMessage(std::string lastMessage);
00048 std::string getLastMessage() const;
00049
00050 virtual std::string show() override;
00051
00052 protected:
00053 virtual bool _loadInstance(PersistenceRecord* fields) override;
00054 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00055 virtual bool _check(std::string& errorMessage) override;
00056 // virtual void _createInternalAndAttachedData() override;
00057 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00058
00059
00060 protected:
00061 // virtual void _createInternalStatisticReporters() override;
00062 // virtual void _createNonEditableDataDefinitions() override;
00063 virtual void _createEditableDataDefinitions() override;
00064 // virtual void _createAttachedAttributes() override;
00065
00066 private:
00067 void _forwardEntity(Entity* entity);
00068
00069 private:
00070 const struct DEFAULT_VALUES {
00071 std::string bioNetworkName = "";
00072 bool useNetworkTimeWindow = true;
00073 double startTime = 0.0;
00074 double stopTime = 1.0;
00075 double stepSize = 0.1;
00076 bool lastSucceeded = false;
00077 std::string lastMessage = "";
00078 } DEFAULT;
00079
00080 BioNetwork* _bioNetwork = nullptr;
00081 bool _useNetworkTimeWindow = DEFAULT.useNetworkTimeWindow;
00082 double _startTime = DEFAULT.startTime;
00083 double _stopTime = DEFAULT.stopTime;
00084 double _stepSize = DEFAULT.stepSize;
00085 bool _lastSucceeded = DEFAULT.lastSucceeded;
00086 std::string _lastMessage = DEFAULT.lastMessage;
00087 };
00088
00089 #endif /* BIOSIMULATE_H */

```

## 16.350 BioSteadyState.h

```

00001 /*
00002 * File: BioSteadyState.h
00003 * Author: rlcancian
00004 *
00005 * Component that evaluates BioNetwork steady-state status.
00006 */
00007
00008 #ifndef BIOSTEADYSTATE_H
00009 #define BIOSTEADYSTATE_H
00010
00011 #include <string>
00012
00013 #include "../kernel/simulator/model/ModelComponent.h"
00014 #include "kernel/simulator/Plugin.h"
00015 #include "plugins/data/BiochemicalSimulation/BioNetwork.h"
00016
00023 class BioSteadyState : public ModelComponent {
00024 public:
00025 BioSteadyState(Model* model, std::string name = "");
00026 virtual ~BioSteadyState() override = default;
00027
00028 public:
00029 static PluginInformation* GetPluginInformation();
00030 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00031 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00032
00033 public:
00034 void setBioNetwork(BioNetwork* bioNetwork);
00035 BioNetwork* getBioNetwork() const;
00036 void setTolerance(double tolerance);
00037 double getTolerance() const;
00038 void setRunSimulationBeforeCheck(bool runSimulationBeforeCheck);
00039 bool getRunSimulationBeforeCheck() const;
00040 void setLastSteady(bool lastSteady);

```

```

00041 bool getLastSteady() const;
00042 void setLastMaxAbsoluteDerivative(double lastMaxAbsoluteDerivative);
00043 double getLastMaxAbsoluteDerivative() const;
00044 void setLastMessage(std::string lastMessage);
00045 std::string getLastMessage() const;
00046
00047 virtual std::string show() override;
00048
00049 protected:
00050 virtual bool _loadInstance(PersistenceRecord* fields) override;
00051 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00052 virtual bool _check(std::string& errorMessage) override;
00053 // virtual void _createInternalAndAttachedData() override;
00054 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00055
00056
00057 protected:
00058 // virtual void _createInternalStatisticReporters() override;
00059 // virtual void _createNonEditableDataDefinitions() override;
00060 // virtual void _createEditableDataDefinitions() override;
00061 virtual void _createAttachedAttributes() override;
00062
00063 private:
00064 void _forwardEntity(Entity* entity);
00065
00066 private:
00067 const struct DEFAULT_VALUES {
00068 std::string bioNetworkName = "";
00069 double tolerance = 1e-6;
00070 bool runSimulationBeforeCheck = false;
00071 bool lastSteady = false;
00072 double lastMaxAbsoluteDerivative = 0.0;
00073 std::string lastMessage = "";
00074 } DEFAULT;
00075
00076 BioNetwork* _bioNetwork = nullptr;
00077 double _tolerance = DEFAULT.tolerance;
00078 bool _runSimulationBeforeCheck = DEFAULT.runSimulationBeforeCheck;
00079 bool _lastSteady = DEFAULT.lastSteady;
00080 double _lastMaxAbsoluteDerivative = DEFAULT.lastMaxAbsoluteDerivative;
00081 std::string _lastMessage = DEFAULT.lastMessage;
00082 };
00083
00084 #endif /* BIOSTEADYSTATE_H */

```

## 16.351 GeneticCircuitSimulate.h

```

00001 #ifndef GENETICCIRCUITSIMULATE_H
00002 #define GENETICCIRCUITSIMULATE_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelComponent.h"
00007 #include "kernel/simulator/Plugin.h"
00008 #include "plugins/data/BiochemicalSimulation/GeneticCircuit.h"
00009
00010 class GeneticCircuitPart;
00011
00012 struct GeneticCircuitSimulationSummary {
00013 bool succeeded = false;
00014 unsigned int sampleCount = 0;
00015 double totalExpression = 0.0;
00016 std::string message = "";
00017 };
00018
00022 class GeneticCircuitSimulate : public ModelComponent {
00023 public:
00024 GeneticCircuitSimulate(Model* model, std::string name = "");
00025 virtual ~GeneticCircuitSimulate() override = default;
00026
00027 public:
00028 static PluginInformation* GetPluginInformation();
00029 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00030 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00031
00032 public:
00033 void setGeneticCircuit(GeneticCircuit* geneticCircuit);
00034 GeneticCircuit* getGeneticCircuit() const;
00035 void setStartTime(double startTime);
00036 double getStartTime() const;
00037 void setStopTime(double stopTime);
00038 double getStopTime() const;
00039 void setStepSize(double stepSize);
00040 double getStepSize() const;
00041 void setApplyRegulation(bool applyRegulation);

```

```

00042 bool getApplyRegulation() const;
00043 void setLastSucceeded(bool lastSucceeded);
00044 bool getLastSucceeded() const;
00045 void setLastSampleCount(unsigned int lastSampleCount);
00046 unsigned int getLastSampleCount() const;
00047 void setLastTotalExpression(double lastTotalExpression);
00048 double getLastTotalExpression() const;
00049 void setLastMessage(std::string lastMessage);
00050 std::string getLastMessage() const;
00051 static bool simulateCircuit(Model* model, GeneticCircuit* geneticCircuit, double startTime, double
stopTime,
00052 double stepSize, bool applyRegulation,
GeneticCircuitsSimulationSummary* summary,
00053 std::string& errorMessage);
00054 bool simulate(std::string& errorMessage);
00055
00056 virtual std::string show() override;
00057
00058 protected:
00059 virtual bool _loadInstance(PersistenceRecord* fields) override;
00060 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00061 virtual bool _check(std::string& errorMessage) override;
00062 // virtual void _createInternalAndAttachedData() override;
00063 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00064
00065 protected:
00066 // virtual void _createInternalStatisticReporters() override;
00067 // virtual void _createNonEditableDataDefinitions() override;
00068 virtual void _createEditableDataDefinitions() override;
00069 // virtual void _createAttachedAttributes() override;
00070
00071 private:
00072 double _computeRegulationMultiplier(const GeneticCircuitPart* part) const;
00073 void _executeStep(double timeStep);
00074 void _forwardEntity(Entity* entity);
00075
00076 private:
00077 const struct DEFAULT_VALUES {
00078 std::string geneticCircuitName = "";
00079 double startTime = 0.0;
00080 double stopTime = 10.0;
00081 double stepSize = 1.0;
00082 bool applyRegulation = true;
00083 bool lastSucceeded = false;
00084 unsigned int lastSampleCount = 0;
00085 double lastTotalExpression = 0.0;
00086 std::string lastMessage = "";
00087 } DEFAULT;
00088
00089 GeneticCircuit* _geneticCircuit = nullptr;
00090 double _startTime = DEFAULT.startTime;
00091 double _stopTime = DEFAULT.stopTime;
00092 double _stepSize = DEFAULT.stepSize;
00093 bool _applyRegulation = DEFAULT.applyRegulation;
00094 bool _lastSucceeded = DEFAULT.lastSucceeded;
00095 unsigned int _lastSampleCount = DEFAULT.lastSampleCount;
00096 double _lastTotalExpression = DEFAULT.lastTotalExpression;
00097 std::string _lastMessage = DEFAULT.lastMessage;
00098 };
00099
00100 #endif /* GENETICCIRCUITSIMULATE_H */

```

## 16.352 GeneticExpressionStep.h

```

00001 #ifndef GENETICEXPRESSIONSTEP_H
00002 #define GENETICEXPRESSIONSTEP_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelComponent.h"
00007 #include "kernel/simulator/Plugin.h"
00008 #include "plugins/data/BiochemicalSimulation/GeneticCircuit.h"
00009
00010 class GeneticCircuitPart;
00011
00012 class GeneticExpressionStep : public ModelComponent {
00013 public:
00014 GeneticExpressionStep(Model* model, std::string name = "");
00015 virtual ~GeneticExpressionStep() override = default;
00016
00017 public:
00018 static PluginInformation* GetPluginInformation();
00019 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00020 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00021
00022 };

```

```

00025 public:
00026 void setGeneticCircuit(GeneticCircuit* geneticCircuit);
00027 GeneticCircuit* getGeneticCircuit() const;
00028 void setTimeStep(double timeStep);
00029 double getTimeStep() const;
00030 void setApplyRegulation(bool applyRegulation);
00031 bool getApplyRegulation() const;
00032 void setLastSucceeded(bool lastSucceeded);
00033 bool getLastSucceeded() const;
00034 void setLastTotalExpression(double lastTotalExpression);
00035 double getLastTotalExpression() const;
00036 void setLastMessage(std::string lastMessage);
00037 std::string getLastMessage() const;
00038
00039 virtual std::string show() override;
00040
00041 protected:
00042 virtual bool _loadInstance(PersistenceRecord* fields) override;
00043 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00044 virtual bool _check(std::string& errorMessage) override;
00045 // virtual void _createInternalAndAttachedData() override;
00046 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00047 protected:
00048 // virtual void _createInternalStatisticReporters() override;
00049 // virtual void _createNonEditableDataDefinitions() override;
00050 virtual void _createEditableDataDefinitions() override;
00051 // virtual void _createAttachedAttributes() override;
00052
00053 private:
00054 double _computeRegulationMultiplier(const GeneticCircuitPart* part) const;
00055 void _forwardEntity(Entity* entity);
00056
00057 private:
00058 const struct DEFAULT_VALUES {
00059 std::string geneticCircuitName = "";
00060 double timeStep = 1.0;
00061 bool applyRegulation = true;
00062 bool lastSucceeded = false;
00063 double lastTotalExpression = 0.0;
00064 std::string lastMessage = "";
00065 } DEFAULT;
00066
00067 GeneticCircuit* _geneticCircuit = nullptr;
00068 double _timeStep = DEFAULT.timeStep;
00069 bool _applyRegulation = DEFAULT.applyRegulation;
00070 bool _lastSucceeded = DEFAULT.lastSucceeded;
00071 double _lastTotalExpression = DEFAULT.lastTotalExpression;
00072 std::string _lastMessage = DEFAULT.lastMessage;
00073 };
00074
00075 #endif /* GENETICEXPRESSIONSTEP_H */

```

## 16.353 MetabolicFluxBalance.h

```

00001 #ifndef METABOLICFLUXBALANCE_H
00002 #define METABOLICFLUXBALANCE_H
00003
00004 #include <map>
00005 #include <string>
00006
00007 #include "../kernel/simulator/model/ModelComponent.h"
00008 #include "kernel/simulator/Plugin.h"
00009 #include "plugins/data/BiochemicalSimulation/MetabolicNetwork.h"
00010
00038 class MetabolicFluxBalance : public ModelComponent {
00039 public:
00040 MetabolicFluxBalance(Model* model, std::string name = "");
00041 virtual ~MetabolicFluxBalance() override = default;
00042
00043 public:
00044 static PluginInformation* GetPluginInformation();
00045 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00046 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00047
00048 public:
00049 void setMetabolicNetwork(MetabolicNetwork* metabolicNetwork);
00050 MetabolicNetwork* getMetabolicNetwork() const;
00051 void setObjectiveReactionName(std::string objectiveReactionName);
00052 std::string getObjectiveReactionName() const;
00053 void setLastSucceeded(bool lastSucceeded);
00054 bool getLastSucceeded() const;
00055 void setLastObjectiveValue(double lastObjectiveValue);
00056 double getLastObjectiveValue() const;
00057 void setLastMessage(std::string lastMessage);
00058 std::string getLastMessage() const;

```

```

00059 const std::map<std::string, double>& getLastFluxes() const;
00060
00061 virtual std::string show() override;
00062
00063 protected:
00064 virtual bool _loadInstance(PersistenceRecord* fields) override;
00065 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00066 virtual bool _check(std::string& errorMessage) override;
00067 // virtual void _createInternalAndAttachedData() override;
00068 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00069
00070 protected:
00071 // virtual void _createInternalStatisticReporters() override;
00072 // virtual void _createNonEditableDataDefinitions() override;
00073 virtual void _createEditableDataDefinitions() override;
00074 // virtual void _createAttachedAttributes() override;
00075
00076 private:
00077 void _forwardEntity(Entity* entity);
00078
00079 private:
00080 const struct DEFAULT_VALUES {
00081 std::string metabolicNetworkName = "";
00082 std::string objectiveReactionName = "";
00083 bool lastSucceeded = false;
00084 double lastObjectiveValue = 0.0;
00085 std::string lastMessage = "";
00086 } DEFAULT;
00087
00088 MetabolicNetwork* _metabolicNetwork = nullptr;
00089 std::string _objectiveReactionName = DEFAULT.objectiveReactionName;
00090 bool _lastSucceeded = DEFAULT.lastSucceeded;
00091 double _lastObjectiveValue = DEFAULT.lastObjectiveValue;
00092 std::string _lastMessage = DEFAULT.lastMessage;
00093 std::map<std::string, double> _lastFluxes;
00094 };
00095
00096 #endif /* METABOLICFLUXBALANCE_H */

```

## 16.354 ContinuousSystemComponent.h

```

00001 /*
00002 * File: ContinuousSystemComponent.h
00003 * Author: GenESys
00004 *
00005 * ModelComponent that drives continuous-time simulation by delegating
00006 * numerical integration to an ODESolver data definition.
00007 */
00008
00009 #ifndef CONTINUOUSSYSTEMCOMPONENT_H
00010 #define CONTINUOUSSYSTEMCOMPONENT_H
00011
00012 #include <string>
00013
00014 #include "kernel/simulator/model/ModelComponent.h"
00015 #include "plugins/data/Continuous/ODESolver.h"
00016
00026 class ContinuousSystemComponent : public ModelComponent {
00027 public: // constructors
00028 ContinuousSystemComponent(Model* model, std::string name = "");
00029 virtual ~ContinuousSystemComponent() override = default;
00030
00031 public: // virtual
00032 virtual std::string show() override;
00033
00034 public: // static -- required by the plugin framework
00035 static PluginInformation* GetPluginInformation();
00036 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00037 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00038 static void SaveInstance(PersistenceRecord* fields, ModelComponent* component);
00039 static bool Check(ModelComponent* component);
00040 static void CreateInternalData(ModelComponent* component);
00041 static void DispatchEvent(Event* event);
00042
00043 public: // getters & setters
00044 void setOdeSolver(ODESolver* solver);
00045 ODESolver* getOdeSolver() const;
00046 void setOdeSolverName(std::string name);
00047 std::string getOdeSolverName() const;
00048
00049 protected: // virtual -- must be overridden
00050 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00051 virtual bool _loadInstance(PersistenceRecord* fields) override;
00052 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00053 virtual bool _check(std::string& errorMessage) override;

```

```

00054 virtual void _createEditableDataDefinitions() override;
00055
00056 private:
00057 const struct DEFAULT_VALUES {
00058 const std::string odeSolverName = "";
00059 } DEFAULT;
00060
00061 ODESolver* _odeSolver = nullptr;
00062 std::string _odeSolverName = DEFAULT.odeSolverName;
00063 };
00064
00065 #endif /* CONTINUOUSYSTEMCOMPONENT_H */

```

## 16.355 DiffEquations.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: DiffEquations.h
00009 * Author: Prof. Rafael Luiz Cancian, Dr. Eng.
00010 *
00011 * Created on
00012 */
00013
00014 #pragma once
00015
00016 #include "../kernel/simulator/model/ModelComponent.h"
00017 #include "../data/ExternalIntegration/CppCompiler.h"
00018
00019 //extern "C" typedef void (*initBetweenReplications_t)(Model* model);
00020 extern "C" typedef void (*diffEquationsOnDispatchEvent_t)(double* stateVars, int n, double t0, double
tTarget, double step);
00021
00022 class DiffEquations : public ModelComponent {
00023 public:
00024 DiffEquations(Model* model, std::string name = "");
00025 virtual ~DiffEquations() override;
00026
00027 public:
00028 List<std::string> *getEquations() const;
00029 List<std::string> *getStateVariableNames() const;
00030 double getprecision() const;
00031 void setPrecision(double newPrecision);
00032 double getmaxSteps() const;
00033 void setMaxSteps(double newMaxSteps);
00034 double getfinalTime() const;
00035 void setFinalTime(double newFinalTime);
00036 List<double> *getvaluesOnFinalTime() const;
00037
00038 public:
00039 virtual std::string show() override;
00040
00041 public:
00042 static PluginInformation* GetPluginInformation();
00043 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00044 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00045
00046 protected:
00047 virtual bool _loadInstance(PersistenceRecord *fields) override;
00048 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00049 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00050
00051 protected:
00052 virtual bool _check(std::string& errorMessage) override;
00053 // virtual ParserChangesInformation* _getParserChangesInformation();
00054 //virtual void _initBetweenReplications();
00055 // virtual void _addSimulationControl(SimulationControl* property);
00056
00057 protected:
00058 // virtual void _createInternalStatisticReporters() override;
00059 // virtual void _createNonEditableDataDefinitions() override;
00060 virtual void _createEditableDataDefinitions() override;
00061 // virtual void _createAttachedAttributes() override;
00062
00063 private:
00064 List<std::string> *_equations = new List<std::string>();
00065 List<std::string> *_stateVariableNames = new List<std::string>();
00066 List<double> *_valuesOnFinalTime = new List<double>();
00067 // pointers to functions on a shared libraty

```

```

00077 diffEquationsOnDispatchEvent_t dispatchEvent_SharedLibHandler;
00078 //initBetweenReplications_t initBetweenReplications_SharedLibHandler;
00079 // ...
00080
00081 private:
00082
00083 // Default values for the attributes. Used on initing, loading and saving
00084 const struct DEFAULT_VALUES {
00085 const double precision = 1e-6;
00086 const double maxSteps = 1e3;
00087 const double finalTime = 1.0;
00088 } DEFAULT;
00089 double _precision = DEFAULT.precision;
00090 double _maxSteps = DEFAULT.maxSteps;
00091 double _finalTime = DEFAULT.finalTime;
00092
00093 private:
00094 CppCompiler *_cppCompiler = nullptr;
00095
00096 private:
00097
00098 };

```

## 16.356 DiffusionSimulate.h

```

00001 /*
00002 * File: DiffusionSimulate.h
00003 * Author: GenESyS DCS (Tema 8.2)
00004 *
00005 * Component that advances a DiffusionField (continuous N-D diffusion) when
00006 * entities arrive, mirroring the BioSimulate -> BioNetwork pattern.
00007 */
00008
00009 #ifndef DIFFUSIONSIMULATE_H
00010 #define DIFFUSIONSIMULATE_H
00011
00012 #include <string>
00013
00014 #include "../kernel/simulator/model/ModelComponent.h"
00015 #include "kernel/simulator/Plugin.h"
00016 #include "plugins/data/Continuous/DiffusionField.h"
00017
00018 class DiffusionSimulate : public ModelComponent {
00019 public:
00020 DiffusionSimulate(Model* model, std::string name = "");
00021 virtual ~DiffusionSimulate() override = default;
00022
00023 public:
00024 static PluginInformation* GetPluginInformation();
00025 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00026 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00027
00028 public:
00029 void setDiffusionField(DiffusionField* diffusionField);
00030 DiffusionField* getDiffusionField() const;
00031 void setUseFieldTimeWindow(bool useFieldTimeWindow);
00032 bool getUseFieldTimeWindow() const;
00033 void setStartTime(double startTime);
00034 double getStartTime() const;
00035 void setStopTime(double stopTime);
00036 double getStopTime() const;
00037 void setStepSize(double stepSize);
00038 double getStepSize() const;
00039 void setLastSucceeded(bool lastSucceeded);
00040 bool getLastSucceeded() const;
00041 void setLastMessage(std::string lastMessage);
00042 std::string getLastMessage() const;
00043
00044 virtual std::string show() override;
00045
00046 protected:
00047 virtual bool _loadInstance(PersistenceRecord* fields) override;
00048 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00049 virtual bool _check(std::string& errorMessage) override;
00050 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00051
00052 protected:
00053 virtual void _createEditableDataDefinitions() override;
00054
00055 private:
00056 void _forwardEntity(Entity* entity);
00057
00058 private:
00059 const struct DEFAULT_VALUES {
00060 std::string diffusionFieldName = "";

```

```

00072 bool useFieldTimeWindow = true;
00073 double startTime = 0.0;
00074 double stopTime = 1.0;
00075 double stepSize = 0.1;
00076 bool lastSucceeded = false;
00077 std::string lastMessage = "";
00078 } DEFAULT;
00079
00080 DiffusionField* _diffusionField = nullptr;
00081 bool _useFieldTimeWindow = DEFAULT.useFieldTimeWindow;
00082 double _startTime = DEFAULT.startTime;
00083 double _stopTime = DEFAULT.stopTime;
00084 double _stepSize = DEFAULT.stepSize;
00085 bool _lastSucceeded = DEFAULT.lastSucceeded;
00086 std::string _lastMessage = DEFAULT.lastMessage;
00087 };
00088
00089 #endif /* DIFFUSIONSIMULATE_H */

```

## 16.357 LODE.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: LODE.h
00009 * Author: rlcancian
00010 *
00011 * Created on 22 de Outubro de 2019, 22:28
00012 */
00013
00014 #ifndef LODE_H
00015 #define LODE_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "../data/Logic/Formula.h"
00019 #include "../data/Logic/Variable.h"
00020
00024 class LODE : public ModelComponent {
00025 public: // constructors
00026 LODE(Model* model, std::string name = "");
00027 virtual ~LODE() override;
00028 public: // virtual
00029 virtual std::string show() override;
00030 public: // static
00031 static PluginInformation* GetPluginInformation();
00032 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00033 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00034 public: // g&s
00035 void setTimeVariable(Variable* _timeVariable);
00036 Variable* getTimeVariable() const;
00037 void setStep(double _step);
00038 double getStep() const;
00039 void setVariable(Variable* _variables);
00040 Variable* getVariable() const;
00041 List<std::string>* getDiffEquations() const;
00042 void addDiffEquation(std::string newDiffEquation);
00043 void removeDiffEquation(std::string diffEquation);
00044 void setFilename(std::string filename);
00045 std::string getFileName() const;
00046 protected: // virtual
00047 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00048 virtual bool _loadInstance(PersistenceRecord *fields) override;
00049 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00050 protected: // virtual
00051 //virtual void _initBetweenReplications();
00052 // virtual void _createInternalStatisticReporters() override;
00053 // virtual void _createNonEditableDataDefinitions() override;
00054 // virtual void _createEditableDataDefinitions() override;
00055 // virtual void _createAttachedAttributes() override;
00056 virtual bool _check(std::string& errorMessage) override;
00057 //virtual void _createInternalAndAttachedData();
00058
00059 protected:
00060
00061 private: // methods
00062 bool _doStep();
00063 private: // attributes 1:1
00064
00065 const struct DEFAULT_VALUES {
00066 const std::string filename = "";
00067 const double step = 0.0;

```



```

00068 } DEFAULT;
00069 List<std::string>* _diffEquations = new List<std::string>();
00070 Variable* _variable = nullptr;
00071 Variable* _timeVariable = nullptr;
00072 std::string _filename = DEFAULT.filename;
00073 double _step = DEFAULT.step;
00074 private: // attributes 1:n
00075 };
00076
00077 #endif /* LSOE_H */

```

## 16.358 OLD\_ODEelement.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: ODE.h
00009 * Author: rlcancian
00010 *
00011 * Created on 13 de Junho de 2019, 19:12
00012 */
00013
00014 #ifndef OLD_ODEELEMENT_H
00015 #define OLD_ODEELEMENT_H
00016
00017 #include "../kernel/simulator/model/ModelDataDefinition.h"
00018 #include "kernel/simulator/model/ModelDataManager.h"
00019 #include "kernel/simulator/Plugin.h"
00020
00021 class ODEfunction {
00022 public:
00023 ODEfunction(std::string expression, double initialPoint, double initialValue) {
00024 this->expression = expression;
00025 this->initialPoint = initialPoint;
00026 this->initialValue = initialValue;
00027 }
00028 std::string expression;
00029 double initialPoint;
00030 double initialValue;
00031 };
00032
00033 class OLD_ODEelement : public ModelDataDefinition {
00034 public:
00035 OLD_ODEelement(Model* model, std::string name = "");
00036 virtual ~OLD_ODEelement() = default;
00037 public:
00038 virtual std::string show() override;
00039 public:
00040 static PluginInformation* GetPluginInformation();
00041 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00042 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00043 public:
00044 double solve();
00045 void setStepH(double _h);
00046 double getStepH() const;
00047 void setEndTime(double _endTime);
00048 double getEndTime() const;
00049 List<ODEfunction*>* getODEfunctions() const;
00050 void addODEfunction(ODEfunction* newFunction);
00051 void removeODEfunction(ODEfunction* function);
00052 protected: // must be overridden
00053 virtual bool _loadInstance(PersistenceRecord *fields) override;
00054 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00055 virtual bool _check(std::string& errorMessage) override;
00056
00057 protected:
00058 // virtual void _createInternalStatisticReporters() override;
00059 // virtual void _createNonEditableDataDefinitions() override;
00060 // virtual void _createEditableDataDefinitions() override;
00061 // virtual void _createAttachedAttributes() override;
00062
00063 private:
00064
00065 private:
00066 List<ODEfunction*>* _ODEfunctions = new List<ODEfunction*>();
00067 double _stepH = 0.1;
00068 double _endTime;
00069 };
00070
00071 #endif /* OLD_ODEELEMENT_H */

```

## 16.359 Decide.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Decide.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 9 de Agosto de 2018, 20:39
00012 */
00013
00014 #ifndef DECIDE_H
00015 #define DECIDE_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "../kernel/simulator/essentialPlugins/Counter.h"
00019
00074 class Decide : public ModelComponent {
00075 public:
00076 Decide(Model* model, std::string name = "");
00077 virtual ~Decide() override;
00078 public:
00079 List<std::string>* getConditions() const;
00080 void addConditions(std::string newCondition);
00081 void removeConditions(std::string condition);
00082 public:
00083 virtual std::string show() override;
00084 public:
00085 static PluginInformation* GetPluginInformation();
00086 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00087 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00088 protected:
00089 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00090 virtual bool _loadInstance(PersistenceRecord *fields) override;
00091 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00092 protected:
00093 //virtual void _initBetweenReplications();
00094 virtual bool _check(std::string& errorMessage) override;
00095 protected:
00096 virtual void _createInternalStatisticReporters() override;
00097 // virtual void _createNonEditableDataDefinitions() override;
00098 // virtual void _createEditableDataDefinitions() override;
00099 // virtual void _createAttachedAttributes() override;
00100
00101 protected:
00102
00103 private:
00104 List<std::string>* _conditions = new List<std::string>();
00105 List<Counter*>* _numberOuts = nullptr;
00106 private:
00107
00108 };
00109
00110 #endif /* DECIDE_H */

```

## 16.360 DropOff.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: DropOff.h
00009 * Author: rlcancian
00010 *
00011 * Created on 03 de Junho de 2019, 15:15
00012 */
00013
00014 #ifndef DROPOFF_H
00015 #define DROPOFF_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00041 class DropOff : public ModelComponent {
00042 public: // constructors
00043 DropOff(Model* model, std::string name = "");
00044 virtual ~DropOff() = default;
00045 public: // virtual

```

```

00046 virtual std::string show() override;
00047 public: // static
00048 static PluginInformation* GetPluginInformation();
00049 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00050 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00051 protected: // virtual
00052 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00053 virtual bool _loadInstance(PersistenceRecord *fields) override;
00054 protected: // virtual
00055 //virtual void _initBetweenReplications();
00056 // virtual void _createInternalStatisticReporters() override;
00057 // virtual void _createNonEditableDataDefinitions() override;
00058 // virtual void _createEditableDataDefinitions() override;
00059 // virtual void _createAttachedAttributes() override;
00060 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00061 virtual bool _check(std::string& errorMessage) override;
00062
00063 protected:
00064
00065 private: // methods
00066 private: // attributes 1:1
00067 private: // attributes 1:n
00068 };
00069
00070
00071 #endif /* DROPOFF_H */

```

## 16.361 Pickup.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Pickup.h
00009 * Author: rlcancian
00010 *
00011 * Created on 03 de Junho de 2019, 15:15
00012 */
00013
00014 #ifndef PICKUP_H
00015 #define PICKUP_H
00016
00017 #include "plugins/components/Decisions/Remove.h"
00018
00019 class Pickup : public Remove {
00020 public: // constructors
00021 Pickup(Model* model, std::string name = "");
00022 virtual ~Pickup() = default;
00023 public: // static
00024 static PluginInformation* GetPluginInformation();
00025 //static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00026 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00027 };
00028
00029 #endif /* PICKUP_H */
00030

```

## 16.362 Remove.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Remove.h
00009 * Author: rlcancian
00010 *
00011 * Created on 03 de Junho de 2019, 15:20
00012 */
00013
00014 #ifndef REMOVE_H
00015 #define REMOVE_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00019 class Remove : public ModelComponent {

```

```

00038 public:
00039
00040 enum class RemoveFromType : int {
00041 QUEUE = 0, ENTITYGROUP = 1, num_elements = 2
00042 };
00043 public:
00044 static std::string convertEnumToStr(RemoveFromType type);
00045 public: // constructors
00046 Remove(Model* model, std::string name = "");
00047 virtual ~Remove() = default;
00048 public: // virtual
00049 virtual std::string show() override;
00050 public:
00051 void setRemoveStartRank(std::string _removeFromRank);
00052 std::string getRemoveStartRank() const;
00053 void setRemoveEndRank(std::string _removeEndRank);
00054 std::string getRemoveEndRank() const;
00055 void setRemoveFromType(Remove::RemoveFromType _removeFromType);
00056 Remove::RemoveFromType getRemoveFromType() const;
00057 void setRemoveFrom(ModelDataDefinition* _removeFrom);
00058 ModelDataDefinition* getRemoveFrom() const;
00059
00060 public: // static
00061 static PluginInformation* GetPluginInformation();
00062 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00063 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00064 protected: // must be overridden
00065 virtual bool _loadInstance(PersistenceRecord *fields) override;
00066 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00067 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00068 protected: // could be overridden by derived classes
00069 virtual bool _check(std::string& errorMessage) override;
00070 //virtual ParserChangesInformation* _getParserChangesInformation();
00071 //virtual void _initBetweenReplications();
00072 protected:
00073 // virtual void _createInternalStatisticReporters() override;
00074 // virtual void _createNonEditableDataDefinitions() override;
00075 virtual void _createEditableDataDefinitions() override;
00076 // virtual void _createAttachedAttributes() override;
00077 // virtual void _createInternalAndAttachedData() override;
00078 //virtual void _addSimulationControl(SimulationControl* property);
00079
00080 protected:
00081
00082 private: // methods
00083 private: // attributes 1:1
00084
00085 const struct DEFAULT_VALUES {
00086 const Remove::RemoveFromType removeFromType = Remove::RemoveFromType::QUEUE;
00087 const std::string removeStartRank = "";
00088 const std::string removeEndRank = "";
00089 } DEFAULT;
00090 ModelDataDefinition* _removeFrom = nullptr;
00091 Remove::RemoveFromType _removeFromType = DEFAULT.removeFromType;
00092 std::string _removeStartRank = DEFAULT.removeStartRank;
00093 std::string _removeEndRank = DEFAULT.removeStartRank;
00094 //std::string _removeFromRank = DEFAULT.removeFromRank;
00095
00096 private: // attributes 1:n
00097 };
00098
00099 #endif /* REMOVE_H */

```

## 16.363 Search.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Search.h
00009 * Author: rlcancian
00010 *
00011 * Created on 03 de Junho de 2019, 15:20
00012 */
00013
00014 #ifndef SEARCH_H
00015 #define SEARCH_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00019 class Search : public ModelComponent {

```

```

00056 public:
00057 enum class SearchInType : int {
00058 QUEUE = 0, ENTITYGROUP = 1, num_elements = 2
00059 };
00060 public:
00061 static std::string convertEnumToStr(SearchInType type);
00062 public: // constructors
00063 Search(Model* model, std::string name = "");
00064 virtual ~Search() = default;
00065 public: // virtual
00066 virtual std::string show() override;
00067 public:
00068 void setSaveFounRankAttribute(std::string _saveFounRankAttribute);
00069 std::string getSaveFounRankAttribute() const;
00070 void setSearchCondition(std::string _searchCondition);
00071 std::string getSearchCondition() const;
00072 void setEndRank(std::string _endRank);
00073 std::string getEndRank() const;
00074 void setStartRank(std::string _startRank);
00075 std::string getStartRank() const;
00076 void setSearchInName(std::string searchInName);
00077 std::string getSearchInName() const;
00078 void setSearchIn(ModelDataDefinition* _searchIn);
00079 ModelDataDefinition* getSearchIn() const;
00080 void setSearchInType(Search::SearchInType _searchInType);
00081 Search::SearchInType getSearchInType() const;
00082 public: // static
00083 static PluginInformation* GetPluginInformation();
00084 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00085 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00086
00087 protected: // must be overridden
00088 virtual bool _loadInstance(PersistenceRecord* fields) override;
00089 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00090 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00091 protected: // could be overridden by derived classes
00092 virtual bool _check(std::string& errorMessage) override;
00093 //virtual ParserChangesInformation* _getParserChangesInformation();
00094 //virtual void _initBetweenReplications();
00095
00096 protected:
00097 // virtual void _createInternalStatisticReporters() override;
00098 // virtual void _createNonEditableDataDefinitions() override;
00099 virtual void _createEditableDataDefinitions() override;
00100 // virtual void _createAttachedAttributes() override;
00101 // virtual void _createInternalAndAttachedData() override;
00102 //virtual void _addSimulationControl(SimulationControl* property);
00103
00104
00105 protected:
00106
00107 private: // methods
00108 private: // attributes 1:1
00109
00110 const struct DEFAULT_VALUES {
00111 const SearchInType searchInType = SearchInType::QUEUE;
00112 const std::string startRank = "";
00113 const std::string endRank = "";
00114 const std::string searchCondition = "";
00115 const std::string saveFounRankAttribute = "";
00116 } DEFAULT;
00117 ModelDataDefinition* _searchIn = nullptr;
00118 Search::SearchInType _searchInType = DEFAULT.searchInType;
00119 std::string _startRank = DEFAULT.startRank;
00120 std::string _endRank = DEFAULT.endRank;
00121 std::string _searchCondition = DEFAULT.searchCondition;
00122 std::string _saveFounRankAttribute = DEFAULT.saveFounRankAttribute;
00123
00124 private: // attributes 1:n
00125 };
00126
00127
00128 #endif /* SEARCH_H */

```

## 16.364 QueueableItem.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: QueueableItem.h
00009 * Author: rlcancian
00010 *
00011 * Created on 23 de abril de 2021, 15:09

```

```

00012 */
00013
00014 #ifndef QUEUEABLEITEM_H
00015 #define QUEUEABLEITEM_H
00016
00017 #include "plugins/data/DiscreteProcessing/Queue.h"
00018 #include "../.../data/Logic/Set.h"
00019 #include "../.../kernel/simulator/model/ModelDataManager.h"
00020
00021 // @TODO should inhene from a common base to SeizeableItem
00022
00023 class QueueableItem {
00024 public:
00025
00026 enum class QueueableType : int {
00027 QUEUE = 0, SET = 1, num_elements = 2 //, HOLD = 2, num_elements = 3
00028 };
00029 public:
00030 static std::string convertEnumToStr(QueueableType type);
00031 public:
00032 QueueableItem(ModelDataDefinition* queueOrSet, QueueableItem::QueueableType queueableType =
QueueableItem::QueueableType::QUEUE, std::string index = "0");
00033 QueueableItem(Model* model, std::string queueName);
00034 public:
00035 bool loadInstance(PersistenceRecord *fields);
00036 void saveInstance(PersistenceRecord *fields, bool saveDefaultValues);
00037 public:
00038 std::string show();
00039 void setIndex(std::string index);
00040 std::string getIndex() const;
00041 std::string getQueueableName() const;
00042 std::string getName() const;
00043 void setQueue(Queue* resource);
00044 Queue* getQueue() const;
00045 void setSet(Set* set);
00046 Set* getSet() const;
00047 //
00048 void setQueueableType(QueueableType queueableType);
00049 QueueableType getQueueableType() const;
00050 ModelDataDefinition* getQueueable() const;
00051 void setElementManager(ModelDataManager* _modeldataManager);
00052 //void setComponentManager(ModelComponentManager* _componentManager);
00053 List<SimulationControl*>* getSimulationControls() const;
00054 void _addSimulationControl(SimulationControl* control);
00055
00056 std::string getTypeDC() {return _typeDC;};
00057 private:
00058 void _ensureSimulationControls(Model* model);
00059
00060 private:
00061
00062 const struct DEFAULT_VALUES {
00063 const std::string quantityExpression = "1";
00064 const QueueableItem::QueueableType queueableType = QueueableItem::QueueableType::QUEUE;
00065 const std::string saveAttribute = "";
00066 const std::string index = "0";
00067 } DEFAULT;
00068
00069 QueueableType _queueableType = DEFAULT.queueableType;
00070 ModelDataDefinition* _queueOrSet = nullptr;
00071 std::string _queueableName;
00072 std::string _index = DEFAULT.index;
00073 std::string _typeDC = Util::TypeOf<Queue>();
00074 private:
00075 //ModelComponentManager* _componentManager;
00076 ModelDataManager* _modeldataManager = nullptr;
00077 List<SimulationControl*>* _simulationControls = new List<SimulationControl*>();
00078 };
00079
00080
00081 #endif /* QUEUEABLEITEM_H */

```

## 16.365 SeizableItem.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: SeizableItem.h
00009 * Author: rlcancian
00010 *
00011 * Created on 10 de abril de 2021, 08:45

```

```

00012 */
00013
00014 #ifndef SEIZABLEITEM_H
00015 #define SEIZABLEITEM_H
00016
00017 #include "plugins/data/DiscreteProcessing/Resource.h"
00018 #include "../.../data/Logic/Set.h"
00019 #include "../.../kernel/simulator/model/ModelDataManager.h"
00020
00021 // @TODO should inherit from a common base to QueueableItem
00022
00023 class SeizableItem {
00024 public:
00025 enum class SelectionRule : int {
00026 CYCLICAL = 0, RANDOM = 1, SPECIFICMEMBER = 2, LARGESTREMAININGCAPACITY = 3, SMALLESTNUMBERBUSY
00027 = 4,
00028 PREFEREDORDER = 5, num_elements = 6
00029 };
00030 enum class SeizableType : int {
00031 RESOURCE = 0, SET = 1, num_elements = 2
00032 };
00033
00034 static std::string convertEnumToStr(SelectionRule rule);
00035 static std::string convertEnumToStr(SeizableType type);
00036
00037 public:
00038 SeizableItem(ModelDataDefinition* resourceOrSet, std::string quantityExpression = "1",
00039 SeizableItem::SelectionRule selectionRule =
00040 SeizableItem::SelectionRule::LARGESTREMAININGCAPACITY,
00041 std::string saveAttribute = "", std::string index = "0");
00042 SeizableItem(Model* model, std::string resourceName, std::string quantityExpression = "1",
00043 SeizableItem::SelectionRule selectionRule =
00044 SeizableItem::SelectionRule::LARGESTREMAININGCAPACITY,
00045 std::string saveAttribute = "", std::string index = "0");
00046 SeizableItem(SeizableItem* original);
00047
00048 public:
00049 bool loadInstance(PersistenceRecord* fields, unsigned int parentIndex);
00050 void saveInstance(PersistenceRecord* fields, unsigned int parentIndex, bool saveDefault);
00051 bool loadInstance(PersistenceRecord* fields);
00052 void saveInstance(PersistenceRecord* fields, bool saveDefaults);
00053
00054 public:
00055 std::string show();
00056 void setIndex(std::string index);
00057 std::string getIndex() const;
00058 void setSaveAttribute(std::string saveAttribute);
00059 std::string getSaveAttribute() const;
00060 void setSelectionRule(SelectionRule selectionRule);
00061 SelectionRule getSelectionRule() const;
00062 void setQuantityExpression(std::string quantityExpression);
00063 std::string getQuantityExpression() const;
00064 std::string getResourceName() const;
00065 std::string getName() const;
00066 void setResource(Resource* resource);
00067 Resource* getResource() const;
00068 void setSet(Set* set);
00069 Set* getSet() const;
00070 //void setSeizableType(SeizableType resourceType);
00071 SeizableType getSeizableType() const;
00072 void setLastMemberSeized(unsigned int lastMemberSeized);
00073 unsigned int getLastMemberSeized() const;
00074 ModelDataDefinition* getSeizable() const;
00075 void setElementManager(ModelDataManager* modelDataManager);
00076 void setLastPreferredOrder(unsigned int lastPreferredOrder);
00077 unsigned int getLastPreferredOrder() const;
00078 //void setComponentManager(ModelComponentManager* _componentManager);
00079 List<SimulationControl*>* getSimulationControls() const;
00080 void _addSimulationControl(SimulationControl* control);
00081
00082 std::string getTypeDC() { return _typeDC; };
00083
00084 private:
00085 void _ensureSimulationControls(Model* model);
00086
00087 private:
00088 const struct DEFAULT_VALUES {
00089 const std::string quantityExpression = "1";
00090 const SeizableItem::SeizableType seizableType = SeizableItem::SeizableType::RESOURCE;
00091 const SeizableItem::SelectionRule selectionRule =
00092 SeizableItem::SelectionRule::LARGESTREMAININGCAPACITY;
00093 const std::string saveAttribute = "";
00094 const std::string index = "0";
00095 } DEFAULT;
00096 SeizableType _seizableType = DEFAULT.seizableType;

```

```

00095 SelectionRule _selectionRule = DEFAULT.selectionRule;
00096 std::string _saveAttribute = DEFAULT.saveAttribute;
00097 std::string _index = DEFAULT.index;
00098 ModelDataDefinition* _resourceOrSet = nullptr;
00099 std::string _seizableName;
00100 std::string _quantityExpression;
00101 unsigned int _lastMemberSeized = 0;
00102 unsigned int _lastPreferredOrder = 0;
00103 std::string _typeDC = Util::TypeOf<Resource>();
00104
00105 private:
00106 ModelDataManager* _modelDataManager;
00107 List<SimulationControl*> _simulationControls = new List<SimulationControl*>();
00108 };
00109
00110 #endif /* SEIZABLEITEM_H */

```

## 16.366 Buffer.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Buffer.h
00009 * Author: Prof. Rafael Luiz Cancian, Dr. Eng.
00010 *
00011 * Created on
00012 */
00013
00014 #ifndef BUFFER_H
00015 #define BUFFER_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "../data/Synchronization/SignalData.h"
00019
00023 class Buffer : public ModelComponent {
00024 public:
00025 enum class AdvanceOn : int {
00026 NewArrivals = 0, Signal = 1, num_elements = 2
00027 };
00028 enum class ArrivalOnFullBufferRule : int {
00029 Dispose = 0, SendToBulkPort = 1, ReplaceLastPosition = 2, num_elements = 3
00030 };
00031 static std::string convertEnumToStr(AdvanceOn advance);
00032 static std::string convertEnumToStr(ArrivalOnFullBufferRule arrival);
00033 public: // constructors
00034 Buffer(Model* model, std::string name = "");
00035 virtual ~Buffer() override;
00036 public: // virtual
00037 virtual std::string show() override;
00038 public: // static
00039 static PluginInformation* GetPluginInformation();
00040 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00041 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00042 public:
00043 Buffer::ArrivalOnFullBufferRule getArrivalOnFullBufferRule() const;
00044 void setArrivalOnFullBufferRule(Buffer::ArrivalOnFullBufferRule newArrivalOnFullBufferRule);
00045 Buffer::AdvanceOn getAdvanceOn() const;
00046 void setAdvanceOn(Buffer::AdvanceOn newAdvanceOn);
00047 unsigned int getCapacity() const;
00048 void setCapacity(unsigned int newCapacity);
00049 SignalData* getSignal() const;
00050 void setSignal(SignalData* newSignal);
00051
00052 protected: // must be overridden
00053 virtual bool _loadInstance(PersistenceRecord* fields) override;
00054 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00055 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00056 protected: // could be overridden by derived classes
00057 virtual bool _check(std::string& errorMessage) override;
00059 virtual ParserChangesInformation* _getParserChangesInformation() override;
00060 virtual void _initBetweenReplications() override;
00062 // virtual void _createInternalAndAttachedData() override;
00063 virtual void _addSimulationControl(SimulationControl* property) override;
00064
00065 protected:
00066 // virtual void _createInternalStatisticReporters() override;
00067 // virtual void _createNonEditableDataDefinitions() override;
00068 virtual void _createEditableDataDefinitions() override;
00069 // virtual void _createAttachedAttributes() override;
00070
00071 private: // methods

```



```

00072 unsigned int _handlerForSignalDataEvent(SignalData* signalData);
00073 Entity* _advance(Entity* enteringEntity);
00074 private: // attributes 1:1
00075
00076 const struct DEFAULT_VALUES {
00077 const AdvanceOn advanceOn = AdvanceOn::NewArrivals;
00078 const ArrivalOnFullBufferRule arrivalOnFullBufferRule = ArrivalOnFullBufferRule::Dispose;
00079 const unsigned int capacity = 1;
00080 } DEFAULT;
00081 ArrivalOnFullBufferRule _arrivalOnFullBufferRule = DEFAULT.arrivalOnFullBufferRule;
00082 AdvanceOn _advanceOn = DEFAULT.advanceOn;
00083 unsigned int _capacity = DEFAULT.capacity;
00084 private:
00085 std::vector<Entity*>* _buffer = new std::vector<Entity*>;
00086 private: // attached
00087 SignalData* _attachedSignal = nullptr;
00088 SignalData* _signalWithRegisteredHandler = nullptr;
00089 };
00090
00091 #endif /* BUFFER_H */

```

## 16.367 Clone.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Clone.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 30 de Novembro de 2021, 18:50
00012 */
00013
00014 #ifndef CLONE_H
00015 #define CLONE_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "../kernel/simulator/essentialPlugins/Counter.h"
00019
00023 class Clone : public ModelComponent {
00024 public: // constructors
00025 Clone(Model* model, std::string name = "");
00026 virtual ~Clone() = default;
00027 public: // virtual
00028 virtual std::string show() override;
00029 public: //
00030 void setNumClonesExpression(std::string numClones);
00031 std::string getNumClonesExpression() const;
00032 public: // static
00033 static PluginInformation* GetPluginInformation();
00034 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00035 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00036 protected: // must be overridden
00037 virtual bool _loadInstance(PersistenceRecord *fields) override;
00038 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00039 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00040 protected: // could be overridden
00041 virtual bool _check(std::string& errorMessage) override;
00042 virtual void _initBetweenReplications() override;
00043 // virtual void _createInternalAndAttachedData() override;
00044 //virtual ParserChangesInformation* _getParserChangesInformation();
00045
00046 protected:
00047 virtual void _createInternalStatisticReporters() override;
00048 // virtual void _createNonEditableDataDefinitions() override;
00049 // virtual void _createEditableDataDefinitions() override;
00050 // virtual void _createAttachedAttributes() override;
00051
00052 private: // methods
00053 private: // attributes 1:1
00054
00055 const struct DEFAULT_VALUES {
00056 const std::string numClonesExpression = "1";
00057 } DEFAULT;
00058 std::string _numClonesExpression = DEFAULT.numClonesExpression;
00059 Counter* _counter = nullptr;
00060 private: // attributes 1:n
00061 };
00062
00063 #endif /* CLONE_H */

```

## 16.368 Delay.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Delay.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 21 de Junho de 2018, 19:49
00012 */
00013
00014 #ifndef DELAY_H
00015 #define DELAY_H
00016
00017 #include <string>
00018 #include <vector>
00019 #include "../kernel/simulator/model/ModelComponent.h"
00020 #include "kernel/simulator/Plugin.h"
00021
00022 class Delay : public ModelComponent {
00023 public:
00024 Delay(Model* model, std::string name = "");
00025 virtual ~Delay() = default;
00026 public:
00027 //void setDelayExpression(std::string _delayExpression);
00028 void setDelayExpression(std::string _delayExpression, Util::TimeUnit
00029 _delayTimeUnit=Util::TimeUnit::unknown);
00030 std::string delayExpression() const;
00031 void setDelay(double delay);
00032 double delay() const;
00033 void setDelayTimeUnit(Util::TimeUnit _delayTimeUnit);
00034 Util::TimeUnit delayTimeUnit() const;
00035 void setAllocation(Util::AllocationType allocation);
00036 Util::AllocationType getAllocation() const;
00037 public:
00038 virtual std::string show() override;
00039 public:
00040 static PluginInformation* GetPluginInformation();
00041 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00042 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00043 //public: // properties
00044 //using property_owner_t = Delay;
00045 //decl_property(DelayExpression, decl_get(std::string){return delayExpression();} void
00046 decl_set(std::string val){setDelayExpression(val);});
00047
00048 protected:
00049 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00050 virtual bool _loadInstance(PersistenceRecord *fields) override;
00051 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00052 protected:
00053 //virtual void _initBetweenReplications();
00054 virtual bool _check(std::string& errorMessage) override;
00055 protected:
00056 virtual void _createInternalStatisticReporters() override;
00057 // virtual void _createNonEditableDataDefinitions() override;
00058 // virtual void _createEditableDataDefinitions() override;
00059 virtual void _createAttachedAttributes() override;
00060 public:
00061 const struct DEFAULT_VALUES {
00062 const std::string delayExpression = "1.0";
00063 const Util::TimeUnit delayTimeUnit = Util::TimeUnit::second;
00064 const Util::AllocationType allocation = Util::AllocationType::Wait;
00065 } DEFAULT;
00066
00067 protected:
00068
00069 private:
00070 std::string _delayExpression = DEFAULT.delayExpression;
00071 Util::TimeUnit _delayTimeUnit = DEFAULT.delayTimeUnit;
00072 Util::AllocationType _allocation = DEFAULT.allocation;
00073 private:
00074 std::vector<std::string> _allAllocationAttachedAttributeName() const;
00075 std::string _allocationAttachedAttributeName(Util::AllocationType allocation) const;
00076 void _reconcileAllocationAttachedAttributes();
00077 void _initCStats();
00078 private: // inner internal elements
00079 friend class DelayProbe;
00080 StatisticsCollector* _cstatWaitTime = nullptr;
00081 };
00082 //enable_this_owner(Delay, DelayExpression);
00083
00084 #endif /* DELAY_H */

```

## 16.369 Process.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Process.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 22 de Maio de 2019, 18:41
00012 */
00013
00014 #ifndef PROCESS_H
00015 #define PROCESS_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "plugins/components/DiscreteProcessing/Seize.h"
00019 #include "plugins/components/DiscreteProcessing/Delay.h"
00020 #include "plugins/components/DiscreteProcessing/Release.h"
00021
00022 class Process : public ModelComponent {
00023 public: // constructors
00024 Process(Model* model, std::string name = "");
00025 virtual ~Process() = default;
00026 public: // virtual
00027 virtual std::string show() override;
00028 public: // static
00029 static PluginInformation* GetPluginInformation();
00030 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00031 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00032 public:
00033 void setPriority(unsigned short _priority);
00034 unsigned short getPriority() const;
00035 void setPriorityExpression(std::string priority);
00036 std::string getPriorityExpression() const;
00037 void setAllocationType(Util::AllocationType _allocationType);
00038 Util::AllocationType getAllocationType() const;
00039 List<SeizableItem*> getSeizeRequests() const;
00040 void addSeizeRequest(SeizableItem* newRequest);
00041 void removeSeizeRequest(SeizableItem* request);
00042 void setQueueableItem(QueueableItem* _queueableItem);
00043 QueueableItem* getQueueableItem() const;
00044 void setDelayExpression(std::string _delayExpression);
00045 void setDelayExpression(std::string _delayExpression, Util::TimeUnit _delayTimeUnit);
00046 std::string delayExpression() const;
00047 void setDelayTimeUnit(Util::TimeUnit _delayTimeUnit);
00048 Util::TimeUnit delayTimeUnit() const;
00049 protected: // virtual
00050 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00051 // virtual void _initBetweenReplications();
00052 virtual bool _loadInstance(PersistenceRecord *fields) override;
00053 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00054 protected: // virtual
00055 virtual bool _check(std::string& errorMessage) override;
00056 protected: // virtual
00057 // virtual void _createInternalStatisticReporters() override;
00058 virtual void _createNonEditableDataDefinitions() override;
00059 virtual void _createEditableDataDefinitions() override;
00060 // virtual void _createAttachedAttributes() override;
00061 // virtual void _createInternalAndAttachedData() override;
00062
00063 protected:
00064
00065 private: // methods
00066 void _adjustConnections();
00067 void _ensureInternalComponents();
00068 void _reconcileInternalComponents();
00069 bool _hasConsistentInternalChain(std::string& errorMessage) const;
00070 private: // attributes 1:1
00071 Seize* _seize = nullptr;
00072 Delay* _delay = nullptr;
00073 Release* _release = nullptr;
00074 bool _flagConstructing;
00075 //ModelComponent* _nextComponent = nullptr;
00076 //unsigned int _nextInput;
00077 private: // attributes 1:n
00078 };
00079
00080 #endif /* PROCESS_H */

```

## 16.370 Release.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Release.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 21 de Agosto de 2018, 16:17
00012 */
00013
00014 #ifndef RELEASE_H
00015 #define RELEASE_H
00016
00017 #include <string>
00018
00019 #include "../kernel/simulator/model/ModelComponent.h"
00020 #include "kernel/simulator/Plugin.h"
00021 #include "auxiliar/SeizableItem.h"
00022 #include "plugins/data/DiscreteProcessing/Resource.h"
00023
00024 class Release : public ModelComponent {
00025 public:
00026 Release(Model* model, std::string name = "");
00027 virtual ~Release() override;
00028 public:
00029 virtual std::string show() override;
00030 public: //static
00031 static PluginInformation* GetPluginInformation();
00032 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00033 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00034 public: // get & set
00035 void setPriority(unsigned short _priority);
00036 unsigned short priority() const;
00037 public: // gets
00038 List<SeizableItem*>* getReleaseRequests() const;
00039 void addReleaseRequests(SeizableItem* newRequest);
00040 void removeReleaseRequests(SeizableItem* request);
00041
00042 protected:
00043 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00044 virtual bool _loadInstance(PersistenceRecord *fields) override;
00045 virtual void _initBetweenReplications() override;
00046 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00047 virtual bool _check(std::string& errorMessage) override;
00048 // virtual void _createInternalAndAttachedData() override;
00049
00050 protected:
00051 // virtual void _createInternalStatisticReporters() override;
00052 // virtual void _createNonEditableDataDefinitions() override;
00053 virtual void _createEditableDataDefinitions() override;
00054 // virtual void _createAttachedAttributes() override;
00055
00056 private:
00057 Resource* _getResourceFromSeizableItem(SeizableItem* seizable, Entity* entity);
00058 private:
00059
00060 const struct DEFAULT_VALUES {
00061 const unsigned short priority = 0;
00062 const unsigned int releaseRequestSize = 1;
00063 } DEFAULT;
00064 unsigned short _priority = DEFAULT.priority;
00065 List<SeizableItem*>* _releaseRequests = new List<SeizableItem*>();
00066 };
00067
00068 #endif /* RELEASE_H */

```

## 16.371 Seize.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Seize.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 21 de Agosto de 2018, 16:17

```

```

00012 */
00013
00014 #ifndef SEIZE_H
00015 #define SEIZE_H
00016
00017 #include <string>
00018 #include "../kernel/simulator/model/ModelComponent.h"
00019 #include "../kernel/simulator/model/Model.h"
00020 #include "plugins/data/DiscreteProcessing/Resource.h"
00021 #include "plugins/data/DiscreteProcessing/Queue.h"
00022 #include "kernel/simulator/Plugin.h"
00023 #include "auxiliar/SeizableItem.h"
00024 #include "auxiliar/QueueableItem.h"
00025
00026 class WaitingResource : public Waiting {
00027 public:
00028
00029 WaitingResource(Entity* entity, double timeStartedWaiting, unsigned int quantity, ModelComponent*
thisComponent, unsigned int thisComponentOutputPort = 0) : Waiting(entity, timeStartedWaiting,
thisComponent, thisComponentOutputPort) {
00030 _quantity = quantity;
00031 }
00032
00033 WaitingResource(const WaitingResource& orig) : Waiting(orig) {
00034 }
00035
00036 virtual ~WaitingResource() = default;
00037 public:
00038
00039 virtual std::string show() override {
00040 return Waiting::show() +
00041 ",quantity=" + std::to_string(this->_quantity);
00042 }
00043 public:
00044
00045 unsigned int getQuantity() const {
00046 return _quantity;
00047 }
00048
00049 protected:
00050 private:
00051 unsigned int _quantity;
00052 };
00053
00054 class Seize : public ModelComponent {
00055 public:
00056 Seize(Model* model, std::string name = "");
00057 virtual ~Seize() override;
00058 public:
00059 virtual std::string show() override;
00060 public:
00061 static PluginInformation* GetPluginInformation();
00062 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00063 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00064 public: // get & set
00065 void setPriority(unsigned short _priority);
00066 unsigned short getPriority() const;
00067 void setAllocationType(Util::AllocationType _allocationType);
00068 Util::AllocationType getAllocationType() const;
00069 // indirect access to Queue* and Resource*
00070 //void setResourceName(std::string _resourceName) throw ();
00071 //std::string getResourceName() const;
00072 //void setQueueName(std::string queueName) throw ();
00073 //std::string getQueueName() const;
00074 void setQueue(Queue* queue);
00075 //Queue* getQueue() const;
00076 List<SeizableItem*> getSeizeRequests() const;
00077 void addRequest(SeizableItem* newRequest);
00078 void removeRequest(SeizableItem* request);
00079 void setQueueableItem(QueueableItem* _queueableItem);
00080 QueueableItem* getQueueableItem() const;
00081 void setPriorityExpression(std::string _priorityExpression);
00082 std::string getPriorityExpression() const;
00083 protected:
00084 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00085 virtual bool _loadInstance(PersistenceRecord *fields) override;
00086 virtual void _initBetweenReplications() override;
00087 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00088 virtual bool _check(std::string& errorMessage) override;
00089 protected:
00090 // virtual void _createInternalStatisticReporters() override;
00091 // virtual void _createNonEditableDataDefinitions() override;
00092 virtual void _createEditableDataDefinitions() override;
00093 // virtual void _createAttachedAttributes() override;
00094 // virtual void _createInternalAndAttachedData() override;
00095 private:
00096 void _handlerForResourceEvent(Resource* resource);

```

```

00171 Resource* _getResourceFromSeizableItem(SeizableItem* seizable, Entity* entity, unsigned
int*indexPtr);
00172 Queue* _getQueue() const;
00173 public:
00174
00175 const struct DEFAULT_VALUES {
00176 const Util::AllocationType allocationType = Util::AllocationType::Others;
00177 const unsigned short priority = 0;
00178 const std::string priorityExpression = "";
00179 const unsigned int seizeRequestSize = 1;
00180 } DEFAULT;
00181 private:
00182 Util::AllocationType _allocationType = DEFAULT.allocationType; // uint ? enum?
00183 unsigned short _priority = DEFAULT.priority;
00184 std::string _priorityExpression = DEFAULT.priorityExpression;
00185 QueueableItem* _queueableItem = nullptr; // usually has a queue, but not always (it could be a
hold or a set)
00186 List<SeizableItem*>* _seizeRequests = new List<SeizableItem*>();
00187 };
00188
00189 #endif /* SEIZE_H */

```

## 16.372 SPICECircuit.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: SPICECircuit.h
00009 * Author: bnmfw & joaomai
00010 *
00011 * Created on 10 de outubro de 2023, 23:00
00012 */
00013
00014 #pragma once
00015
00016 #include "../kernel/simulator/model/ModelComponent.h"
00017 #include "../data/ExternalIntegration/SPICERunner.h"
00018 #include "plugins/components/ElectronicsSimulation/SPICENode.h"
00019
00023 class SPICECircuit : public ModelComponent {
00024 public:
00025 SPICECircuit(Model* model, std::string name = "");
00026 virtual ~SPICECircuit() = default;
00027
00028 public:
00029 void setRunner(SPICERunner* runner);
00030 SPICERunner* getRunner();
00031
00032 public:
00033 virtual std::string show() override;
00034
00035 public:
00036 static PluginInformation* GetPluginInformation();
00037 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00038 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00039 // Added
00040 static std::vector<std::string> split(std::string word, char split_char);
00041
00042 protected:
00043 void BuildCircuit(std::string description, unsigned int id, std::vector<std::string> the_params =
{"", std::string the_model = "", std::string the_model_file = "");
00044 void UpdateConnections();
00045 virtual void Build() { id = counter++; build = [this]() {this->Build();}};
00046 virtual bool _loadInstance(PersistenceRecord *fields) override;
00047 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00048 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00049 // virtual void _createInternalAndAttachedData() override;
00050
00051
00052 protected:
00053 // virtual bool _check(std::string& errorMessage);
00054 // virtual ParserChangesInformation* _getParserChangesInformation();
00055 // virtual void _initBetweenReplications();
00056 // virtual void _createInternalStatisticReporters() override;
00057 // virtual void _createEditableDataDefinitions() override;
00058 virtual void _createNonEditableDataDefinitions() override;
00059 // virtual void _createAttachedAttributes() override;
00060 // virtual void _addSimulationControl(SimulationControl* property);
00061
00062 private:

```

```

00068 // Spice subcircuit declaration, if one exists
00069 std::string spice_template;
00070 // Name of component
00071 std::string spice_name;
00072 // Params of the component, such as resistance or voltage
00073 std::vector<std::string> params;
00074 // Pin map of the component
00075 std::vector<std::string> pins;
00076 // Instance of the component in spice
00077 std::string *spice_instance;
00078 // Model of the component, if one exists (the model name should also be the name of the .cir file)
00079 std::string model;
00080 std::string model_file;
00081 // Spice compiler
00082 SPICERunner *compiler = nullptr;
00083
00084 static unsigned int counter;
00085 bool plain_circuit;
00086 protected:
00087 unsigned int id;
00088 std::function<void()> build;
00089
00090 private:
00091
00093 const struct DEFAULT_VALUES {
00094 const std::string someString = "Test";
00095 const unsigned int someUInt = 1;
00096 } DEFAULT;
00097 std::string _someString = DEFAULT.someString;
00098 unsigned int _someUInt = DEFAULT.someUInt;
00099
00100 private:
00101
00102 private:
00103 // ...
00104 };
00105
00106 class Resistor : public SPICECircuit {
00107 public:
00108 Resistor(Model* model, std::string name = ""): SPICECircuit(model, name) {
00109 id = counter++;
00110 build = [this]() {this->Build();};
00111 }
00112 void Build() override;
00113 void setResistance(float resistance);
00114
00115 static PluginInformation* GetPluginInformation();
00116 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00117 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00118
00119 protected:
00120 virtual bool _loadInstance(PersistenceRecord *fields) override;
00121 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00122
00123 private:
00124 const struct DEFAULT_VALUES {
00125 const std::string someString = "Test";
00126 const unsigned int someUInt = 1;
00127 const float someResistance = 1000;
00128 } DEFAULT;
00129
00130 static unsigned int counter;
00131 float resistance = DEFAULT.someResistance;
00132 };
00133
00134 class Vsource : public SPICECircuit {
00135 public:
00136 Vsource(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
00137 [this]() {this->Build();};}
00138 void Build() override;
00139 void setVoltage(float voltage);
00140
00141 static PluginInformation* GetPluginInformation();
00142 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00143 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00144
00145 protected:
00146 virtual bool _loadInstance(PersistenceRecord *fields) override;
00147 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00148
00149 private:
00150 const struct DEFAULT_VALUES {
00151 const std::string someString = "Test";
00152 const unsigned int someUInt = 1;
00153 const float someVoltage = 5;
00154 } DEFAULT;

```

```

00155 float voltage = DEFAULT.someVoltage;
00156 static unsigned int counter;
00157 };
00158
00159 class Vpulse : public SPICECircuit {
00160 public:
00161 Vpulse(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
00162 [this]() {this->Build();};}
00163 void Build() override;
00164 void setVoltage(float voltage);
00165 void setFreq(float freq);
00166 void setSlope(float slope);
00167
00168 static PluginInformation* GetPluginInformation();
00169 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00170 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00171 protected:
00172 virtual bool _loadInstance(PersistenceRecord *fields) override;
00173 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00174
00175 private:
00176 const struct DEFAULT_VALUES {
00177 const std::string someString = "Test";
00178 const unsigned int someUInt = 1;
00179 const float someVoltage = 5;
00180 const float someFreq = 60;
00181 const float someSlope = 1e-11;
00182 } DEFAULT;
00183
00184 static unsigned int counter;
00185 float voltage = DEFAULT.someVoltage;
00186 float freq = DEFAULT.someFreq;
00187 float slope = DEFAULT.someSlope;
00188 };
00189
00190 class Vsine : public SPICECircuit {
00191 public:
00192 Vsine(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
00193 [this]() {this->Build();};}
00194 void Build() override;
00195 void setVoltage(float voltage);
00196 void setFreq(float freq);
00197
00198 static PluginInformation* GetPluginInformation();
00199 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00200 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00201 protected:
00202 virtual bool _loadInstance(PersistenceRecord *fields) override;
00203 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00204
00205 private:
00206 const struct DEFAULT_VALUES {
00207 const std::string someString = "Test";
00208 const unsigned int someUInt = 1;
00209 const float someVoltage = 5;
00210 const float someFreq = 60;
00211 } DEFAULT;
00212
00213 static unsigned int counter;
00214 float voltage = DEFAULT.someVoltage;
00215 float freq = DEFAULT.someFreq;
00216 };
00217
00218 class Capacitor : public SPICECircuit {
00219 public:
00220 Capacitor(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build
00221 = [this]() {this->Build();};}
00222 void setCapacitance(float capacitance);
00223 void Build() override;
00224
00225 static PluginInformation* GetPluginInformation();
00226 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00227 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00228 protected:
00229 virtual bool _loadInstance(PersistenceRecord *fields) override;
00230 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00231
00232 private:
00233 const struct DEFAULT_VALUES {
00234 const std::string someString = "Test";
00235 const unsigned int someUInt = 1;
00236 const float someCapacitance = 0.01;
00237 } DEFAULT;
00238

```



```

00239 static unsigned int counter;
00240 float capacitance = DEFAULT.someCapacitance;
00241 };
00242
00243 class Diode : public SPICECircuit {
00244 public:
00245 Diode(Model* model, std::string name = ""): SPICECircuit(model, name) {
00246 id = counter++; build = [this]() {this->Build();};
00247 }
00248 void Build() override;
00249 void setElectricalModel(std::string electricalModel);
00250
00251 static PluginInformation* GetPluginInformation();
00252 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00253 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00254
00255 protected:
00256 virtual bool _loadInstance(PersistenceRecord *fields) override;
00257 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00258
00259 private:
00260 const struct DEFAULT_VALUES {
00261 const std::string someString = "Test";
00262 const unsigned int someUInt = 1;
00263 const std::string someElectricalModel = "diode";
00264 } DEFAULT;
00265
00266 static unsigned int counter;
00267 std::string electricalModel = DEFAULT.someElectricalModel;
00268 };
00269
00270 class PMOS: public SPICECircuit {
00271 public:
00272 PMOS(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
00273 [this]() {this->Build();};}
00274 void Build() override;
00275 void setWidth(float width);
00276 void setLength(float length);
00277 void setElectricalModel(std::string electricalModel);
00278
00279 static PluginInformation* GetPluginInformation();
00280 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00281 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00282
00283 protected:
00284 virtual bool _loadInstance(PersistenceRecord *fields) override;
00285 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00286
00287 private:
00288 const struct DEFAULT_VALUES {
00289 const std::string someString = "Test";
00290 const unsigned int someUInt = 1;
00291 const float someWidth = 140e-9;
00292 const float someLength = 32e-9;
00293 const std::string someElectricalModel = "32nmCMOS";
00294 } DEFAULT;
00295
00296 static unsigned int counter;
00297 float width = DEFAULT.someWidth;
00298 float length = DEFAULT.someLength;
00299 std::string electricalModel = DEFAULT.someElectricalModel;
00300 };
00301
00302 class NMOS: public SPICECircuit {
00303 public:
00304 NMOS(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
00305 [this]() {this->Build();};}
00306 void Build() override;
00307 void setWidth(float width);
00308 void setLength(float length);
00309 void setElectricalModel(std::string electricalModel);
00310
00311 static PluginInformation* GetPluginInformation();
00312 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00313 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00314
00315 protected:
00316 virtual bool _loadInstance(PersistenceRecord *fields) override;
00317 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00318
00319 private:
00320 const struct DEFAULT_VALUES {
00321 const std::string someString = "Test";
00322 const unsigned int someUInt = 1;
00323 const float someWidth = 70e-9;
00324 const float someLength = 32e-9;
00325 const std::string someElectricalModel = "32nmCMOS";

```

```

00324 } DEFAULT;
00325
00326 static unsigned int counter;
00327 float width = DEFAULT.someWidth;
00328 float length = DEFAULT.someLength;
00329 std::string electricalModel = DEFAULT.someElectricalModel;
00330 };
00331
00332 class NOT : public SPICECircuit {
00333 public:
00334 NOT(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
00335 [this]() {this->Build();}};
00336 void Build() override;
00337 void setPMOSWidth(float PMOSWidth);
00338 void setPMOSLength(float length);
00339 void setNMOSWidth(float NMOSWidth);
00340 void setNMOSLength(float NMOSLength);
00341 void setElectricalModel(std::string electricalModel);
00342
00343 static PluginInformation* GetPluginInformation();
00344 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00345 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00346 protected:
00347 virtual bool _loadInstance(PersistenceRecord *fields) override;
00348 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00349 private:
00350 const struct DEFAULT_VALUES {
00351 const std::string someString = "Test";
00352 const unsigned int someUInt = 1;
00353 const float somePMOSWidth = 140e-9;
00354 const float somePMOSLength = 32e-9;
00355 const float someNMOSWidth = 70e-9;
00356 const float someNMOSLength = 32e-9;
00357 const std::string someElectricalModel = "32nmCMOS";
00358 } DEFAULT;
00359
00360 static unsigned int counter;
00361 float PMOSWidth = DEFAULT.somePMOSWidth;
00362 float PMOSLength = DEFAULT.somePMOSLength;
00363 float NMOSWidth = DEFAULT.someNMOSWidth;
00364 float NMOSLength = DEFAULT.someNMOSLength;
00365 std::string electricalModel = DEFAULT.someElectricalModel;
00366 };
00367
00368 class NAND : public SPICECircuit {
00369 public:
00370 NAND(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
00371 [this]() {this->Build();}};
00372 void Build() override;
00373 void setPMOSWidth(float PMOSWidth);
00374 void setPMOSLength(float length);
00375 void setNMOSWidth(float NMOSWidth);
00376 void setNMOSLength(float NMOSLength);
00377 void setElectricalModel(std::string electricalModel);
00378
00379 static PluginInformation* GetPluginInformation();
00380 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00381 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00382 protected:
00383 virtual bool _loadInstance(PersistenceRecord *fields) override;
00384 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00385 private:
00386 const struct DEFAULT_VALUES {
00387 const std::string someString = "Test";
00388 const unsigned int someUInt = 1;
00389 const float somePMOSWidth = 140e-9;
00390 const float somePMOSLength = 32e-9;
00391 const float someNMOSWidth = 70e-9;
00392 const float someNMOSLength = 32e-9;
00393 const std::string someElectricalModel = "32nmCMOS";
00394 } DEFAULT;
00395
00396 static unsigned int counter;
00397 float PMOSWidth = DEFAULT.somePMOSWidth;
00398 float PMOSLength = DEFAULT.somePMOSLength;
00399 float NMOSWidth = DEFAULT.someNMOSWidth;
00400 float NMOSLength = DEFAULT.someNMOSLength;
00401 std::string electricalModel = DEFAULT.someElectricalModel;
00402 };
00403
00404 class AND : public SPICECircuit {
00405 public:
00406 AND(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =

```

```

 [this]() {this->Build();};}
00409 void Build() override;
00410 void setPMOSWidth(float PMOSWidth);
00411 void setPMOSLength(float length);
00412 void setNMOSWidth(float NMOSWidth);
00413 void setNMOSLength(float NMOSLength);
00414 void setElectricalModel(std::string electricalModel);
00415
00416 static PluginInformation* GetPluginInformation();
00417 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00418 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00419
00420 protected:
00421 virtual bool _loadInstance(PersistenceRecord *fields) override;
00422 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00423
00424 private:
00425 const struct DEFAULT_VALUES {
00426 const std::string someString = "Test";
00427 const unsigned int someUInt = 1;
00428 const float somePMOSWidth = 140e-9;
00429 const float somePMOSLength = 32e-9;
00430 const float someNMOSWidth = 70e-9;
00431 const float someNMOSLength = 32e-9;
00432 const std::string someElectricalModel = "32nmCMOS";
00433 } DEFAULT;
00434
00435 static unsigned int counter;
00436 float PMOSWidth = DEFAULT.somePMOSWidth;
00437 float PMOSLength = DEFAULT.somePMOSLength;
00438 float NMOSWidth = DEFAULT.someNMOSWidth;
00439 float NMOSLength = DEFAULT.someNMOSLength;
00440 std::string electricalModel = DEFAULT.someElectricalModel;
00441 };
00442
00443 class NOR : public SPICECircuit {
00444 public:
00445 NOR(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
 [this]() {this->Build();};}
00446 void Build() override;
00447 void setPMOSWidth(float PMOSWidth);
00448 void setPMOSLength(float length);
00449 void setNMOSWidth(float NMOSWidth);
00450 void setNMOSLength(float NMOSLength);
00451 void setElectricalModel(std::string electricalModel);
00452
00453 static PluginInformation* GetPluginInformation();
00454 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00455 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00456
00457 protected:
00458 virtual bool _loadInstance(PersistenceRecord *fields) override;
00459 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00460
00461 private:
00462 const struct DEFAULT_VALUES {
00463 const std::string someString = "Test";
00464 const unsigned int someUInt = 1;
00465 const float somePMOSWidth = 140e-9;
00466 const float somePMOSLength = 32e-9;
00467 const float someNMOSWidth = 70e-9;
00468 const float someNMOSLength = 32e-9;
00469 const std::string someElectricalModel = "32nmCMOS";
00470 } DEFAULT;
00471
00472 static unsigned int counter;
00473 float PMOSWidth = DEFAULT.somePMOSWidth;
00474 float PMOSLength = DEFAULT.somePMOSLength;
00475 float NMOSWidth = DEFAULT.someNMOSWidth;
00476 float NMOSLength = DEFAULT.someNMOSLength;
00477 std::string electricalModel = DEFAULT.someElectricalModel;
00478 };
00479
00480 class OR : public SPICECircuit {
00481 public:
00482 OR(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
 [this]() {this->Build();};}
00483 void Build() override;
00484 void setPMOSWidth(float PMOSWidth);
00485 void setPMOSLength(float length);
00486 void setNMOSWidth(float NMOSWidth);
00487 void setNMOSLength(float NMOSLength);
00488 void setElectricalModel(std::string electricalModel);
00489
00490 static PluginInformation* GetPluginInformation();
00491 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00492 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");

```

```

00493
00494 protected:
00495 virtual bool _loadInstance(PersistenceRecord *fields) override;
00496 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00497
00498 private:
00499 const struct DEFAULT_VALUES {
00500 const std::string someString = "Test";
00501 const unsigned int someUInt = 1;
00502 const float somePMOSWidth = 140e-9;
00503 const float somePMOSLength = 32e-9;
00504 const float someNMOSWidth = 70e-9;
00505 const float someNMOSLength = 32e-9;
00506 const std::string someElectricalModel = "32nmCMOS";
00507 } DEFAULT;
00508
00509 static unsigned int counter;
00510 float PMOSWidth = DEFAULT.somePMOSWidth;
00511 float PMOSLength = DEFAULT.somePMOSLength;
00512 float NMOSWidth = DEFAULT.someNMOSWidth;
00513 float NMOSLength = DEFAULT.someNMOSLength;
00514 std::string electricalModel = DEFAULT.someElectricalModel;
00515 };
00516
00517 class XOR : public SPICECircuit {
00518 public:
00519 XOR(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
00520 [this]() {this->Build();}};
00521 void Build() override;
00522 void setPMOSWidth(float PMOSWidth);
00523 void setPMOSLength(float length);
00524 void setNMOSWidth(float NMOSWidth);
00525 void setNMOSLength(float NMOSLength);
00526 void setElectricalModel(std::string electricalModel);
00527
00528 static PluginInformation* GetPluginInformation();
00529 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00530 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00531
00532 protected:
00533 virtual bool _loadInstance(PersistenceRecord *fields) override;
00534 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00535
00536 private:
00537 const struct DEFAULT_VALUES {
00538 const std::string someString = "Test";
00539 const unsigned int someUInt = 1;
00540 const float somePMOSWidth = 140e-9;
00541 const float somePMOSLength = 32e-9;
00542 const float someNMOSWidth = 70e-9;
00543 const float someNMOSLength = 32e-9;
00544 const std::string someElectricalModel = "32nmCMOS";
00545 } DEFAULT;
00546
00547 static unsigned int counter;
00548 float PMOSWidth = DEFAULT.somePMOSWidth;
00549 float PMOSLength = DEFAULT.somePMOSLength;
00550 float NMOSWidth = DEFAULT.someNMOSWidth;
00551 float NMOSLength = DEFAULT.someNMOSLength;
00552 std::string electricalModel = DEFAULT.someElectricalModel;
00553 };
00554
00555 class XNOR : public SPICECircuit {
00556 public:
00557 XNOR(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
00558 [this]() {this->Build();}};
00559 void Build() override;
00560 void setPMOSWidth(float PMOSWidth);
00561 void setPMOSLength(float length);
00562 void setNMOSWidth(float NMOSWidth);
00563 void setNMOSLength(float NMOSLength);
00564 void setElectricalModel(std::string electricalModel);
00565
00566 static PluginInformation* GetPluginInformation();
00567 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00568 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00569
00570 protected:
00571 virtual bool _loadInstance(PersistenceRecord *fields) override;
00572 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00573
00574 private:
00575 const struct DEFAULT_VALUES {
00576 const std::string someString = "Test";
00577 const unsigned int someUInt = 1;
00578 const float somePMOSWidth = 140e-9;
00579 const float somePMOSLength = 32e-9;

```

```

00578 const float someNMOSWidth = 70e-9;
00579 const float someNMOSLength = 32e-9;
00580 const std::string someElectricalModel = "32nmCMOS";
00581 } DEFAULT;
00582
00583 static unsigned int counter;
00584 float PMOSWidth = DEFAULT.somePMOSWidth;
00585 float PMOSLength = DEFAULT.somePMOSLength;
00586 float NMOSWidth = DEFAULT.someNMOSWidth;
00587 float NMOSLength = DEFAULT.someNMOSLength;
00588 std::string electricalModel = DEFAULT.someElectricalModel;
00589 };

```

## 16.373 SPICENode.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: SPICENode.h
00009 * Author: bnmfw & joaomai
00010 *
00011 * Created on 10 de outubro de 2023, 23:00
00012 */
00013
00014 #pragma once
00015
00016 #include "../kernel/simulator/model/ModelComponent.h"
00017 #include "../data/ExternalIntegration/SPICERunner.h"
00018 #include "plugins/components/ElectronicsSimulation/SPICENode.h"
00019
00023 class SPICENode : public ModelComponent {
00024 public:
00025 SPICENode(Model* model, std::string name = "");
00026 virtual ~SPICENode() = default;
00027
00028 public:
00029 std::string getNodeName();
00030
00031 public:
00032 virtual std::string show() override;
00033
00034 public:
00035 static PluginInformation* GetPluginInformation();
00036 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00037 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00038 operator std::string() const;
00039
00040 protected:
00041 virtual bool _loadInstance(PersistenceRecord *fields) override;
00042 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00043 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00044
00045
00046 protected:
00047 // virtual bool _check(std::string& errorMessage);
00048 // virtual ParserChangesInformation* _getParserChangesInformation();
00049 // virtual void _initBetweenReplications();
00050 // virtual void _createInternalStatisticReporters() override;
00051 // virtual void _createNonEditableDataDefinitions() override;
00052 // virtual void _createEditableDataDefinitions() override;
00053 // virtual void _createAttachedAttributes() override;
00054 // virtual void _createInternalAndAttachedData(); /*< A ModelDataDefinition or ModelComponent that
00055 includes (internal) ou refers to (attach) other ModelDataDefinition must register them inside this
00056 method. */
00057 // virtual void _addSimulationControl(SimulationControl* property);
00058
00059
00060
00061
00062
00063 protected:
00064
00065 private:
00066 std::string node_name;
00067 static int std_node_count;
00068 static std::set<std::string> used_names;
00069
00070
00071 private:
00072
00073 const struct DEFAULT_VALUES {
00074 const std::string someString = "Test";
00075 const unsigned int someUInt = 1;
00076 } DEFAULT;
00077

```

```

00078 std::string _someString = DEFAULT.someString;
00079 unsigned int _someUint = DEFAULT.someUint;
00080
00081 private:
00082
00083 private:
00084 // ...
00085 };

```

## 16.374 CppForG.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: CppForG.h
00008 * Author: rlcancian
00009 *
00010 * Created on 11 de janeiro de 2022, 22:37
00011 */
00012
00013 #pragma once
00014
00015 #include "../kernel/simulator/model/ModelComponent.h"
00016 #include "kernel/simulator/Simulator.h"
00017 #include "../data/ExternalIntegration/CppCompiler.h"
00018
00019 extern "C" typedef void (*initBetweenReplications_t)(Model* model);
00020 extern "C" typedef void (*onDispatchEvent_t)(Simulator* simulator, Model* model, Entity* entity);
00021
00022 class CppForG : public ModelComponent {
00023 public: // constructors
00024 CppForG(Model* model, std::string name = "");
00025 virtual ~CppForG() = default;
00026 public: // virtual
00027 virtual std::string show() override;
00028 public: // static
00029 static PluginInformation* GetPluginInformation();
00030 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00031 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00032 public:
00033 void setCppCompiler(CppCompiler* _cppCompiler);
00034 CppCompiler* getCppCompiler() const;
00035 void setInitBetweenReplicationCode(std::string _initBetweenReplicationCode);
00036 std::string getInitBetweenReplicationCode() const;
00037 void setOnDispatchEventCode(std::string _onDispatchEventCode);
00038 std::string getOnDispatchEventCode() const;
00039 void setIncludesCode(std::string _includesCode);
00040 std::string getIncludesCode() const;
00041 protected: // must be overridden
00042 virtual bool _loadInstance(PersistenceRecord *fields) override;
00043 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00044 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00045 protected: // could be overridden by derived classes
00046 virtual bool _check(std::string& errorMessage) override;
00047 //virtual ParserChangesInformation* _getParserChangesInformation();
00048 virtual void _initBetweenReplications() override;
00049 // virtual void _createInternalAndAttachedData() override;
00050 //virtual void _addSimulationControl(SimulationControl* property);
00051 protected:
00052 // virtual void _createInternalStatisticReporters() override;
00053 virtual void _createNonEditableDataDefinitions() override;
00054 // virtual void _createAttachedAttributes() override;
00055
00056 private: // methods
00057
00058 private: // attributes 1:1
00059
00060 const struct DEFAULT_VALUES {
00061 std::string onDispatchEventCode = "";
00062 std::string initBetweenReplicationCode = "";
00063 std::string includesCode = "";
00064 } DEFAULT;
00065 std::string _onDispatchEventCode = DEFAULT.onDispatchEventCode;
00066 std::string _initBetweenReplicationCode = DEFAULT.initBetweenReplicationCode;
00067 std::string _includesCode = DEFAULT.includesCode;
00068 //
00069 std::string _sourceFilename;
00070 std::string _outputFilename;
00071 // pointers to functions on a shared library
00072 onDispatchEvent_t dispatchEvent_SharedLibHandler;
00073 initBetweenReplications_t initBetweenReplications_SharedLibHandler;
00074 private: // internal (TODO: should be Attached??)

```

```

00080 CppCompiler* _cppCompiler = nullptr;
00081 private: // attributes 1:n
00082 };

```

## 16.375 PythonForG.h

```

00001 #ifndef PYTHONFORG_H
00002 #define PYTHONFORG_H
00003
00004 #include "../kernel/simulator/model/ModelComponent.h"
00005 #include "../kernel/simulator/SimulationControlAndResponse.h"
00006 #include "plugins/data/ExternalIntegration/PythonRuntime.h"
00007
00008 class PythonForG : public ModelComponent {
00009 public:
00010 PythonForG(Model* model, std::string name = "");
00011 virtual ~PythonForG() override = default;
00012
00013 public:
00014 static PluginInformation* GetPluginInformation();
00015 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00016 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00017
00018 public:
00019 void setInitBetweenReplicationCode(std::string initBetweenReplicationCode);
00020 std::string getInitBetweenReplicationCode() const;
00021 void setInitBetweenReplicationCodeProperty(SourceCodeString initBetweenReplicationCode);
00022 SourceCodeString getInitBetweenReplicationCodeProperty() const;
00023 void setOnDispatchEventCode(std::string onDispatchEventCode);
00024 std::string getOnDispatchEventCode() const;
00025 void setOnDispatchEventCodeProperty(SourceCodeString onDispatchEventCode);
00026 SourceCodeString getOnDispatchEventCodeProperty() const;
00027 void setForwardEntityOnError(bool forwardEntityOnError);
00028 bool isForwardEntityOnError() const;
00029 PythonRuntime* getPythonRuntime() const;
00030
00031 virtual std::string show() override;
00032
00033 protected:
00034 virtual bool _loadInstance(PersistenceRecord* fields) override;
00035 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00036 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00037 virtual bool _check(std::string& errorMessage) override;
00038 virtual void _initBetweenReplications() override;
00039 virtual void _createNonEditableDataDefinitions() override;
00040
00041 private:
00042 bool _validateInitCode(const std::string& candidate, std::string& errorMessage) const;
00043 bool _validateDispatchCode(const std::string& candidate, std::string& errorMessage) const;
00044
00045 private:
00046 const struct DEFAULT_VALUES {
00047 std::string initBetweenReplicationCode = "";
00048 std::string onDispatchEventCode = "";
00049 bool forwardEntityOnError = true;
00050 } DEFAULT;
00051
00052 std::string _initBetweenReplicationCode = DEFAULT.initBetweenReplicationCode;
00053 std::string _onDispatchEventCode = DEFAULT.onDispatchEventCode;
00054 bool _forwardEntityOnError = DEFAULT.forwardEntityOnError;
00055 PythonRuntime* _pythonRuntime = nullptr;
00056 };
00057
00058 #endif /* PYTHONFORG_H */

```

## 16.376 RSimulator.h

```

00001 /*
00002 * File: RSimulator.h
00003 * Author: GenESys
00004 *
00005 * Component that executes R commands when entities arrive.
00006 */
00007
00008 #ifndef RSIMULATOR_H
00009 #define RSIMULATOR_H
00010
00011 #include <list>
00012 #include <string>
00013
00014 #include "../kernel/simulator/model/ModelComponent.h"
00015 #include "kernel/simulator/Plugin.h"

```

```

00016 #include "plugins/data/ExternalIntegration/RSimulatorRunner.h"
00017
00025 class RSimulator : public ModelComponent {
00026 public:
00028 RSimulator(Model* model, std::string name = "");
00030 virtual ~RSimulator() = default;
00031
00032 public:
00034 void setRExecutable(std::string rExecutable);
00036 std::string getRExecutable() const;
00038 void setWorkingDirectory(std::string workingDirectory);
00040 std::string getWorkingDirectory() const;
00042 void setPreludeScript(std::string preludeScript);
00044 std::string getPreludeScript() const;
00046 void insertCommand(std::string command);
00048 void setCommands(std::list<std::string> commands);
00050 List<std::string>* getCommands() const;
00052 void setCommandList(std::string commandList);
00054 std::string getCommandList() const;
00056 RSimulatorRunner* getRunner();
00058 virtual std::string show() override;
00059
00060 public:
00062 static PluginInformation* GetPluginInformation();
00064 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00066 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00067
00068 protected:
00070 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00072 virtual bool _loadInstance(PersistenceRecord *fields) override;
00074 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00076 virtual bool _check(std::string& errorMessage) override;
00078 // virtual void _createInternalAndAttachedData() override;
00079
00080
00081 protected:
00082 // virtual void _createInternalStatisticReporters() override;
00083 // virtual void _createNonEditableDataDefinitions() override;
00084 virtual void _createEditableDataDefinitions() override;
00085 // virtual void _createAttachedAttributes() override;
00086
00087 private:
00089 void _ensureRunner();
00091 void _syncRunnerConfiguration();
00093 bool _isNonEmptyCommand(const std::string& command) const;
00094
00095 private:
00096 const struct DEFAULT_VALUES {
00097 std::string rExecutable = "Rscript";
00098 std::string workingDirectory = "";
00099 std::string preludeScript = "";
00100 } DEFAULT;
00101
00102 std::string _rExecutable = DEFAULT.rExecutable;
00103 std::string _workingDirectory = DEFAULT.workingDirectory;
00104 std::string _preludeScript = DEFAULT.preludeScript;
00105 List<std::string>* _commands = new List<std::string>();
00106 RSimulatorRunner* _runner = nullptr;
00107 };
00108
00109 #endif /* RSIMULATOR_H */

```

## 16.377 Batch.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Batch.h
00009 * Author: rlcancian
00010 *
00011 * Created on 03 de Junho de 2019, 15:14
00012 */
00013
00014 #ifndef BATCH_H
00015 #define BATCH_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "plugins/data/DiscreteProcessing/Queue.h"
00019 #include "../data/Grouping/EntityGroup.h"
00053 class Batch : public ModelComponent {
00054 public:

```



```

00055
00056 enum class BatchType : int {
00057 Temporary = 0, Permanent = 1, num_elements = 2
00058 };
00059
00060 enum class Rule : int {
00061 Any = 0, ByAttribute = 1, num_elements = 2
00062 };
00063
00064 enum class GroupedAttribs : int {
00065 FirstEntity = 0, LastEntity = 1, SumAttributes = 2, num_elements = 3
00066 };
00067 public:
00068 static std::string convertEnumToStr(BatchType type);
00069 static std::string convertEnumToStr(Rule rule);
00070 static std::string convertEnumToStr(GroupedAttribs attribs);
00071 public: // constructors
00072 Batch(Model* model, std::string name = "");
00073 virtual ~Batch() = default;
00074 public: // virtual
00075 virtual std::string show() override;
00076 public: // static
00077 static PluginInformation* GetPluginInformation();
00078 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00079 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00080 public:
00081 void setGroupedEntityType(EntityType* groupedEntityType);
00082 void setGroupedEntityType(std::string groupedEntityType);
00083 EntityType* getGroupedEntityType() const;
00084 void setBatchType(Batch::BatchType batchType);
00085 Batch::BatchType getBatchType() const;
00086 void setAttributeName(std::string attributeName);
00087 std::string getAttributeName() const;
00088 void setBatchSize(std::string batchSize);
00089 std::string getBatchSize() const;
00090 void setRule(Batch::Rule _rule);
00091 Batch::Rule getRule() const;
00092 void setGroupedAttributes(Batch::GroupedAttribs _groupedAttributes);
00093 Batch::GroupedAttribs getGroupedAttributes() const;
00094 protected: // virtual should
00095 //virtual void _initBetweenReplications();
00096 // virtual void _createInternalStatisticReporters() override;
00097 virtual void _createNonEditableDataDefinitions() override;
00098 virtual void _createEditableDataDefinitions() override;
00099 virtual void _createAttachedAttributes() override;
00100 // virtual void _createInternalAndAttachedData() override;
00101 virtual bool _check(std::string& errorMessage) override;
00102 protected: // virtual must
00103 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00104 virtual bool _loadInstance(PersistenceRecord *fields) override;
00105 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00106
00107 protected:
00108
00109 private: // methods
00110 private: // attributes 1:1
00111
00112 const struct DEFAULT_VALUES {
00113 const Batch::BatchType batchType = Batch::BatchType::Temporary;
00114 const Batch::Rule rule = Batch::Rule::Any;
00115 const Batch::GroupedAttribs groupedAttributes = Batch::GroupedAttribs::FirstEntity;
00116 const std::string batchSize = "2";
00117 const std::string attributeName = "";
00118 } DEFAULT;
00119 Batch::BatchType _batchType = DEFAULT.batchType;
00120 Batch::Rule _rule = DEFAULT.rule;
00121 Batch::GroupedAttribs _groupedAttributes = DEFAULT.groupedAttributes;
00122 std::string _batchSize = DEFAULT.batchSize;
00123 std::string _attributeName = DEFAULT.attributeName;
00124 private: // attributes 1:1
00125 EntityType* _groupedEntityType = nullptr;
00126 EntityGroup* _entityGroup = nullptr;
00127 Queue* _queue = nullptr;
00128 private: // attributes 1:n
00129 // count number of batches?
00130 };
00131
00132
00133 #endif /* BATCH_H */

```

## 16.378 Separate.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates

```

```

00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Separate.h
00009 * Author: rlcancian
00010 *
00011 * Created on 03 de Junho de 2019, 15:14
00012 */
00013
00014 #ifndef SEPARATE_H
00015 #define SEPARATE_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00065 class Separate : public ModelComponent {
00066 public: // constructors
00067 Separate(Model* model, std::string name = "");
00068 virtual ~Separate() = default;
00069 public: // virtual
00070 virtual std::string show() override;
00071 public: // static
00072 static PluginInformation* GetPluginInformation();
00073 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00074 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00075 protected: // virtual
00076 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00077 virtual bool _loadInstance(PersistenceRecord *fields) override;
00078 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00079 protected: // virtual
00080 //virtual void _initBetweenReplications();
00081 // virtual void _createInternalStatisticReporters() override;
00082 // virtual void _createNonEditableDataDefinitions() override;
00083 // virtual void _createEditableDataDefinitions() override;
00084 // virtual void _createAttachedAttributes() override;
00085 virtual bool _check(std::string& errorMessage) override;
00086
00087 protected:
00088
00089 private: // methods
00090 private: // attributes 1:1
00091 private: // attributes 1:n
00092 };
00093
00094
00095 #endif /* SEPARATE_H */

```

## 16.379 Record.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Record.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 9 de Agosto de 2018, 13:52
00012 */
00013
00014 #ifndef RECORD_H
00015 #define RECORD_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include <string>
00019
00062 class Record : public ModelComponent {
00063 public:
00064 Record(Model* model, std::string name = "");
00065 virtual ~Record();
00066
00067 public:
00068 void setFilename(std::string filename);
00069 std::string getFileName() const;
00070 void setExpression(const std::string expression);
00071 std::string getExpression() const;
00072 void setExpressionName(std::string expressionName);
00073 std::string getExpressionName() const;
00074 void setDatasetName(std::string datasetName);
00075 std::string getDatasetName() const;
00076 void setRandomVariableName(std::string randomVariableName);
00077 std::string getRandomVariableName() const;
00078 void setVariableType(std::string variableType);

```

```

00091 std::string getVariableType() const;
00095 void setDatasetDescription(std::string description);
00096 std::string getDatasetDescription() const;
00097 StatisticsCollector* getCstatExpression() const;
00098 bool getTimeDependent() const;
00099 void setTimeDependent(bool timeDependent);
00100
00101 public:
00102 virtual std::string show() override;
00103
00104 public:
00105 static PluginInformation* GetPluginInformation();
00106 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00107 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00108
00109 protected:
00110 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00111 virtual bool _loadInstance(PersistenceRecord *fields) override;
00112 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00113
00114 protected:
00115 virtual void _initBetweenReplications() override;
00116 virtual bool _check(std::string& errorMessage) override;
00117 // virtual void _createInternalAndAttachedData() override;
00118
00119
00120 protected:
00121 virtual void _createInternalStatisticReporters() override;
00122 // virtual void _createNonEditableDataDefinitions() override;
00123 // virtual void _createEditableDataDefinitions() override;
00124 // virtual void _createAttachedAttributes() override;
00125
00126 private:
00127 const struct DEFAULT_VALUES {
00128 const bool timeDependent = false;
00129 const std::string expression = "";
00130 const std::string expressionName = "";
00131 const std::string datasetName = "";
00132 const std::string randomVariableName = "";
00133 const std::string variableType = "Continuous numeric";
00134 const std::string description = "";
00135 const std::string filename = "";
00136 } DEFAULT;
00137 bool _timeDependent = DEFAULT.timeDependent;
00138 std::string _expression = DEFAULT.expression;
00139 std::string _expressionName = DEFAULT.expressionName;
00140 std::string _datasetName = DEFAULT.datasetName;
00141 std::string _randomVariableName = DEFAULT.randomVariableName;
00142 std::string _variableType = DEFAULT.variableType;
00143 std::string _description = DEFAULT.description;
00144 std::string _filename = DEFAULT.filename;
00145 private:
00146 const std::string _separator = " ";
00147 StatisticsCollector* _cstatExpression = nullptr;
00148 /* @TODO: Create an internal class to agregate ExpressionStatisticsColelctor, and change Record to
got a list of it, so Record can record a set of expressions into a set of files */
00149 };
00150
00151 #endif /* RECORD_H */

```

## 16.380 Write.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Write.h
00009 * Author: rlcancian
00010 *
00011 * Created on 11 de Setembro de 2019, 13:06
00012 */
00013
00014 #ifndef WRITE_H
00015 #define WRITE_H
00016
00017 #include <list>
00018 #include <fstream>
00019 #include "../kernel/simulator/model/ModelComponent.h"
00020
00024 class Write : public ModelComponent {
00025 public:
00026 enum class WriteToType : int {

```

```

00027 SCREEN = 0, FILE = 1, num_elements = 2
00028 };
00029 public:
00030 static std::string convertEnumToStr(WriteToType type);
00031 public: // constructors
00032 Write(Model* model, std::string name = "");
00033 virtual ~Write() override;
00034
00035 public:
00036 void insertText(std::list<std::string> texts);
00037 void setFilename(std::string _filename);
00038 std::string filename() const;
00039 void setWriteToType(WriteToType _writeToType);
00040 Write::WriteToType writeToType() const;
00041
00042 public: // virtual
00043 virtual std::string show() override;
00044
00045 public: // static
00046 static PluginInformation* GetPluginInformation();
00047 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00048 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00049
00050 protected: // virtual
00051 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00052 virtual bool _loadInstance(PersistenceRecord *fields) override;
00053 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00054
00055 protected: // virtual
00056 virtual bool _check(std::string& errorMessage) override;
00057 virtual void _initBetweenReplications() override;
00058
00059
00060 protected:
00061 // virtual void _createInternalStatisticReporters() override;
00062 // virtual void _createNonEditableDataDefinitions() override;
00063 // virtual void _createEditableDataDefinitions() override;
00064 // virtual void _createAttachedAttributes() override;
00065
00066 private: // methods
00067 private: // attributes 1:1
00068 const struct DEFAULT_VALUES {
00069 const WriteToType writeToType = Write::WriteToType::SCREEN;
00070 const std::string filename = "";
00071 } DEFAULT;
00072 WriteToType _writeToType = DEFAULT.writeToType;
00073 std::string _filename = DEFAULT.filename;
00074 std::ofstream _savefile;
00075
00076 private: // attributes 1:n
00077 List<std::string>* _writeElements = new List<std::string>();
00078 };
00079
00080
00081 #endif /* WRITE_H */

```

## 16.381 Assign.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Assign.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 31 de Agosto de 2018, 10:10
00012 */
00013
00014 #ifndef ASSIGN_H
00015 #define ASSIGN_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "../kernel/simulator/model/Model.h"
00019 #include "kernel/simulator/Plugin.h"
00020 #include "../kernel/simulator/essentialPlugins/Attribute.h"
00021 #include "../data/Logic/AssignmentItem.h"
00022 #include "../data/Logic/Variable.h"
00023
00061 class Assign : public ModelComponent {
00062 public:
00063 Assign(Model* model, std::string name = "");
00064 virtual ~Assign() override;

```

```

00065 public:
00066 virtual std::string show() override;
00067 public:
00068 static PluginInformation* GetPluginInformation();
00069 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00070 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00071 public:
00072 List<Assignment*>* getAssignments() const;
00073 void addAssignment(Assignment* newAssignment);
00074 void removeAssignment(Assignment* assignment);
00075 protected: // must be overridden
00076 virtual bool _loadInstance(PersistenceRecord *fields) override;
00077 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00078 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00079 protected: // could be overridden by derived classes
00080 virtual bool _check(std::string& errorMessage) override;
00081 //virtual ParserChangesInformation* _getParserChangesInformation();
00082 //virtual void _initBetweenReplications();
00083 protected:
00084 // virtual void _createInternalStatisticReporters() override;
00085 // virtual void _createNonEditableDataDefinitions() override;
00086 virtual void _createEditableDataDefinitions() override;
00087 //virtual void _createAttachedAttributes() override;
00088 // virtual void _createInternalAndAttachedData() override;
00089 //virtual void _addSimulationControl(SimulationControl* property);
00090 protected:
00091 private:
00092 static std::string _destinationBaseName(const std::string& destination);
00093 std::string _destinationIndex(const std::string& destination);
00094 void _prepareAssignment(Assignment* assignment);
00095 private:
00096 const struct DEFAULT_VALUES {
00097 const unsigned int assignmentsSize = 1;
00098 } DEFAULT;
00099 List<Assignment*>* _assignments = new List<Assignment*>();
00100 };
00101 #endif /* ASSIGN_H */

```

## 16.382 Create.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Create.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 21 de Junho de 2018, 20:12
00012 */
00013
00014 #ifndef CREATE_H
00015 #define CREATE_H
00016
00017 #include <string>
00018 #include <limits>
00019 #include "kernel/simulator/SourceModelComponent.h"
00020 #include "../kernel/simulator/essentialPlugins/EntityType.h"
00021 #include "../kernel/simulator/essentialPlugins/Counter.h"
00022 #include "kernel/simulator/Plugin.h"
00023
00024 #include "../data/Logic/Formula.h"
00025 #include "plugins/data/DiscreteProcessing/Schedule.h"
00026
00027 class Create : public SourceModelComponent {
00028 public:
00029 Create(Model* model, std::string name = "");
00030 virtual ~Create() = default;
00031 public: // virtual
00032 virtual std::string show() override;
00033 public:
00034 void setTimeBetweenCreationsFormula(Formula* _timeBetweenCreationsFormula);
00035 Formula* getTimeBetweenCreationsFormula() const;
00036 void setTimeBetweenCreationsSchedule(Schedule* _timeBetweenCreationsSchedule);
00037 Schedule* getTimeBetweenCreationsSchedule() const;
00038 public: // static
00039 static PluginInformation* GetPluginInformation();
00040 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00041 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");

```

```

00085
00086 double testePropertyCreateDouble() const;
00087 void setTestePropertyCreateDouble(double newTestePropertyCreateDouble);
00088
00089 protected:
00090 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00091 virtual bool _loadInstance(PersistenceRecord *fields) override;
00092 virtual void _initBetweenReplications() override;
00093 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00094 virtual bool _check(std::string& errorMessage) override;
00095 virtual void _createInternalStatisticReporters() override;
00096 virtual void _createEditableDataDefinitions() override;
00097 private:
00098 double _lastArrival = -1.0;
00099 private: // internal elements
00100 Counter* _numberOut = nullptr; // internal modeldatum
00101 private: // attached elements
00102 Schedule* _timeBetweenCreationsSchedule = nullptr;
00103 Formula* _timeBetweenCreationsFormula = nullptr;
00104 };
00105
00106 #endif /* CREATE_H */

```

## 16.383 Dispose.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Dispose.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 21 de Junho de 2018, 20:13
00012 */
00013
00014 #ifndef DISPOSE_H
00015 #define DISPOSE_H
00016
00017 #include "kernel/simulator/SinkModelComponent.h"
00018 #include "../kernel/simulator/essentialPlugins/Counter.h"
00019 #include "kernel/simulator/Plugin.h"
00020
00038 class Dispose : public SinkModelComponent {
00039 public:
00040 Dispose(Model* model, std::string name = "");
00041 virtual ~Dispose() = default;
00042 public:
00043 virtual std::string show() override;
00044 public:
00045 static PluginInformation* GetPluginInformation();
00046 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00047 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00048 protected:
00049 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00050 virtual bool _loadInstance(PersistenceRecord *fields) override;
00051 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00052 protected:
00053 virtual void _initBetweenReplications() override;
00054 virtual bool _check(std::string& errorMessage) override;
00055 protected:
00056 virtual void _createInternalStatisticReporters() override;
00057 // virtual void _createNonEditableDataDefinitions() override;
00058 // virtual void _createEditableDataDefinitions() override;
00059 virtual void _createAttachedAttributes() override;
00060 protected:
00061 private: // internal elements
00062 Counter* _numberOut = nullptr;
00063 };
00064
00065 #endif /* DISPOSE_H */

```

## 16.384 Access.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006

```

```

00007 /*
00008 * File: Access.h
00009 * Author: rlcancian
00010 *
00011 * Created on 11 de Setembro de 2019, 13:14
00012 */
00013
00014 #ifndef ACCESS_H
00015 #define ACCESS_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00053 class Access : public ModelComponent {
00054 public: // constructors
00055 Access(Model* model, std::string name = "");
00056 virtual ~Access() = default;
00057 public: // virtual
00058 virtual std::string show() override;
00059 public: // static
00060 static PluginInformation* GetPluginInformation();
00061 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00062 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00063 protected: // virtual
00064 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00065 virtual bool _loadInstance(PersistenceRecord *fields) override;
00066 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00067 protected: // virtual
00068 //virtual void _initBetweenReplications();
00069 // virtual void _createInternalStatisticReporters() override;
00070 // virtual void _createNonEditableDataDefinitions() override;
00071 // virtual void _createEditableDataDefinitions() override;
00072 // virtual void _createAttachedAttributes() override;
00073 virtual bool _check(std::string& errorMessage) override;
00074
00075 protected:
00076
00077 private: // methods
00078 private: // attributes 1:1
00079 private: // attributes 1:n
00080 };
00081
00082
00083 #endif /* ACCESS_H */

```

## 16.385 Enter.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Enter.h
00009 * Author: rlcancian
00010 *
00011 * Created on 03 de Junho de 2019, 15:14
00012 */
00013
00014 #ifndef ENTER_H
00015 #define ENTER_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "../data/MaterialHandling/Station.h"
00019 #include "../kernel/simulator/essentialPlugins/Counter.h"
00020
00099 class Enter : public ModelComponent {
00100 public:
00101 Enter(Model* model, std::string name = "");
00102 virtual ~Enter() = default;
00103 public:
00104 virtual std::string show() override;
00105 public:
00106 static PluginInformation* GetPluginInformation();
00107 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00108 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00109 public:
00110 void setStation(Station* _station);
00111 void setStationName(std::string stationName);
00112 Station* getStation() const;
00113 protected:
00114 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00115 virtual bool _loadInstance(PersistenceRecord *fields) override;
00116 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00117 protected:

```

```

00118 //virtual void _initBetweenReplications();
00119 virtual bool _check(std::string& errorMessage) override;
00120
00121 virtual void _createInternalStatisticReporters() override;
00122 // virtual void _createNonEditableDataDefinitions() override;
00123 virtual void _createEditableDataDefinitions() override;
00124 // virtual void _createAttachedAttributes() override;
00125 // virtual void _createInternalAndAttachedData() override;
00126
00127 protected:
00128
00129 private: // association
00130 Station* _station = nullptr;
00131 private: // internal elements
00132 Counter* _numberIn = nullptr;
00133 };
00134
00135 #endif /* ENTER_H */

```

## 16.386 Exit.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Exit.h
00009 * Author: rlcancian
00010 *
00011 * Created on 11 de Setembro de 2019, 13:15
00012 */
00013
00014 #ifndef EXIT_H
00015 #define EXIT_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00037 class Exit : public ModelComponent {
00038 public: // constructors
00039 Exit(Model* model, std::string name = "");
00040 virtual ~Exit() = default;
00041 public: // virtual
00042 virtual std::string show() override;
00043 public: // static
00044 static PluginInformation* GetPluginInformation();
00045 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00046 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00047 protected: // virtual
00048 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00049 virtual bool _loadInstance(PersistenceRecord *fields) override;
00050 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00051 protected: // virtual
00052 //virtual void _initBetweenReplications();
00053 // virtual void _createInternalStatisticReporters() override;
00054 // virtual void _createNonEditableDataDefinitions() override;
00055 // virtual void _createEditableDataDefinitions() override;
00056 // virtual void _createAttachedAttributes() override;
00057 virtual bool _check(std::string& errorMessage) override;
00058
00059 protected:
00060
00061 private: // methods
00062 private: // attributes 1:1
00063 private: // attributes 1:n
00064 };
00065
00066
00067 #endif /* EXIT_H */

```

## 16.387 Leave.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Leave.h
00009 * Author: rlcancian

```



```

00010 *
00011 * Created on 03 de Junho de 2019, 15:14
00012 */
00013
00014 #ifndef LEAVE_H
00015 #define LEAVE_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "../data/MaterialHandling/Station.h"
00019 #include "../kernel/simulator/essentialPlugins/Counter.h"
00020
00094 class Leave : public ModelComponent {
00095 public:
00096 Leave(Model* model, std::string name = "");
00097 virtual ~Leave() = default;
00098 public:
00099 virtual std::string show() override;
00100 public:
00101 static PluginInformation* GetPluginInformation();
00102 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00103 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00104 public:
00105 void setStation(Station* _station);
00106 void setStationName(std::string stationName);
00107 Station* getStation() const;
00108 protected:
00109 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00110 virtual bool _loadInstance(PersistenceRecord *fields) override;
00111 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00112 protected:
00113 //virtual void _initBetweenReplications();
00114 protected:
00115 virtual void _createInternalStatisticReporters() override;
00116 // virtual void _createNonEditableDataDefinitions() override;
00117 virtual void _createEditableDataDefinitions() override;
00118 // virtual void _createAttachedAttributes() override;
00119 virtual bool _check(std::string& errorMessage) override;
00120 // virtual void _createInternalAndAttachedData() override;
00121
00122 protected:
00123
00124 private: // association
00125 Station* _station = nullptr;
00126 private: // internal elements
00127 Counter* _numberIn = nullptr;
00128 };
00129
00130 #endif /* LEAVE_H */

```

## 16.388 PickableStationItem.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: PickableStationItem.h
00008 * Author: Prof. Rafael Luiz Cancian, Dr. Eng.
00009 *
00010 * Created on 1 de dezembro de 2022, 14:00
00011 */
00012
00013 #ifndef PICKABLESTATIONITEM_H
00014 #define PICKABLESTATIONITEM_H
00015
00016 #include "../kernel/simulator/model/Model.h"
00017 #include "plugins/data/DiscreteProcessing/Queue.h"
00018 #include "plugins/data/DiscreteProcessing/Resource.h"
00019 #include "../data/MaterialHandling/Station.h"
00020
00021 class PickableStationItem {
00022 public:
00023 PickableStationItem(Model* model, std::string stationName, std::string queueName = "", std::string
resourceName = "");
00024 PickableStationItem(Model* model, std::string stationName, std::string expression);
00025 PickableStationItem(Station* station, Queue* queue);
00026 PickableStationItem(Station* station, Resource* resource);
00027 PickableStationItem(Station* station, std::string expression);
00028 PickableStationItem(Station* station);
00029 virtual ~PickableStationItem() = default;
00030 public:
00031 std::string getExpression() const;
00032 Queue* getQueue() const;
00033 Resource* getResource() const;

```

```

00034 Station* getStation() const;
00035 void setExpression(std::string _expression);
00036 void setQueue(Queue* _queue);
00037 void setResource(Resource* _resource);
00038 void setStation(Station* _station);
00039
00040 std::string getName();
00041 void _addSimulationControl(SimulationControl* control);
00042 List<SimulationControl*>* getSimulationControls() const;
00043
00044 std::string getTypeDC() {return _typeDC;};
00045 private:
00046 private:
00047 Station* _station = nullptr;
00048 Resource* _resource = nullptr;
00049 Queue* _queue = nullptr;
00050 std::string _expression = "";
00051 std::string _stationName = "";
00052
00053 std::string _typeDC = Util::TypeOf<Station>();
00054 List<SimulationControl*>* _simulationControls = new List<SimulationControl*>();
00055 };
00056
00057 #endif /* PICKABLESTATIONITEM_H */

```

## 16.389 PickStation.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: PickStation.h
00009 * Author: Prof. Rafael Luiz Cancian, Dr. Eng.
00010 *
00011 * Created on 11 de Setembro de 2019, 13:08
00012 */
00013
00014 #ifndef PICKSTATION_H
00015 #define PICKSTATION_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "../kernel/simulator/model/ModelDataDefinition.h"
00019 #include "kernel/util/List.h"
00020 #include "plugins/data/DiscreteProcessing/Queue.h"
00021 #include "plugins/data/DiscreteProcessing/Resource.h"
00022
00023 #include "PickableStationItem.h"
00024
00068 class PickStation : public ModelComponent {
00069 public:
00070
00071 enum class TestCondition : int {
00072 MINIMUM = 0, MAXIMUM = 1, num_elements = 2
00073 };
00074
00075 //enum class PickCondition : int {
00076 // NUMBER_BUSY_RESOURCE = 1, NUMBER_IN_QUEUE = 2, EXPRESSION = 3
00077 //};
00078 public:
00079 static std::string convertEnumToStr(TestCondition condition);
00080 public: // constructors
00081 PickStation(Model* model, std::string name = "");
00082 virtual ~PickStation() override;
00083 public: // virtual
00084 virtual std::string show() override;
00085 public:
00086 //void setPickCondition(PickStation::PickCondition _pickCondition);
00087 //PickStation::PickCondition getPickCondition() const;
00088 void setTestCondition(PickStation::TestCondition _testCondition);
00089 PickStation::TestCondition getTestCondition() const;
00090 void setSaveAttribute(std::string _saveAttribute);
00091 std::string getSaveAttribute() const;
00092 List<PickableStationItem*>* getPickableStationItems() const;
00093 void addPickableStationItem(PickableStationItem* newItem);
00094 void removePickableStationItem(PickableStationItem* item);
00095 void setPickConditionExpression(bool _pickConditionExpression);
00096 bool isPickConditionExpression() const;
00097 void setPickConditionNumberInQueue(bool _pickConditionNumberInQueue);
00098 bool isPickConditionNumberInQueue() const;
00099 void setPickConditionNumberBusyResource(bool _pickConditionNumberBusyResource);
00100 bool isPickConditionNumberBusyResource() const;
00101 public: // static

```

```

00102 static PluginInformation* GetPluginInformation();
00103 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00104 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00105
00106
00107 protected: // must be overridden
00108 virtual bool _loadInstance(PersistenceRecord *fields) override;
00109 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00110 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00111 protected: // could be overridden by derived classes
00112 virtual bool _check(std::string& errorMessage) override;
00113 virtual ParserChangesInformation* _getParserChangesInformation() override;
00114 virtual void _initBetweenReplications() override;
00115 // virtual void _createInternalAndAttachedData() override;
00116 virtual void _addSimulationControl(SimulationControl* property) override;
00117
00118
00119
00120
00121 protected:
00122 // virtual void _createInternalStatisticReporters() override;
00123 // virtual void _createNonEditableDataDefinitions() override;
00124 virtual void _createEditableDataDefinitions() override;
00125 // virtual void _createAttachedAttributes() override;
00126
00127 private: // methods
00128 private: // attributes 1:1
00129
00130 const struct DEFAULT_VALUES {
00131 const std::string saveAttribute = "";
00132 const PickStation::TestCondition testCondition = TestCondition::MINIMUM;
00133 bool pickConditionNumberBusyResource = false;
00134 bool pickConditionNumberInQueue = false;
00135 bool pickConditionExpression = true;
00136 //const PickStation::PickCondition pickCondition = PickCondition::NUMBER_BUSY_RESOURCE;
00137 } DEFAULT;
00138 std::string _saveAttribute = DEFAULT.saveAttribute;
00139 PickStation::TestCondition _testCondition = DEFAULT.testCondition;
00140 //PickStation::PickCondition _pickCondition = DEFAULT.pickCondition;
00141 bool _pickConditionNumberBusyResource = DEFAULT.pickConditionNumberBusyResource;
00142 bool _pickConditionNumberInQueue = DEFAULT.pickConditionNumberInQueue;
00143 bool _pickConditionExpression = DEFAULT.pickConditionExpression;
00144
00145 //DummyElement* _internalDataDefinition = nullptr;
00146 private: // attributes 1:n
00147 List<PickableStationItem*> _pickableStationItems = new List<PickableStationItem*>();
00148 };
00149
00150
00151 #endif /* PICKSTATION_H */

```

## 16.390 Route.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Route.h
00009 * Author: rlcancian
00010 *
00011 * Created on 03 de Junho de 2019, 15:15
00012 */
00013
00014 #ifndef ROUTE_H
00015 #define ROUTE_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "../kernel/simulator/essentialPlugins/Counter.h"
00019 #include "../data/MaterialHandling/Station.h"
00020 #include "../data/MaterialHandling/Sequence.h"
00021 #include "../data/Logic/Label.h"
00022
00023
00024
00025
00026
00027
00028
00029
00030
00031
00032
00033
00034
00035
00036
00037
00038
00039
00040
00041
00042
00043
00044
00045
00046
00047
00048
00049
00050
00051
00052
00053
00054
00055
00056
00057
00058
00059
00060
00061
00062
00063
00064
00065
00066
00067
00068
00069
00070
00071
00072
00073
00074
00075
00076
00077
00078
00079
00080
00081
00082
00083
00084
00085
00086
00087
00088
00089
00090
00091
00092
00093
00094
00095
00096
00097
00098
00099
00100
00101
00102
00103
00104
00105
00106
00107
00108
00109
00110
00111
00112
00113
00114
00115
00116
00117
00118
00119
00120
00121
00122
00123
00124
00125
00126
00127
00128
00129
00130
00131
00132
00133
00134
00135
00136
00137
00138
00139
00140
00141
00142
00143
00144
00145
00146
00147
00148
00149
00150
00151
00152
00153
00154
00155
00156
00157
00158
00159
00160
00161
00162
00163
00164
00165
00166
00167
00168
00169
00170
00171
00172
00173
00174
00175
00176
00177
00178
00179
00180
00181
00182
00183
00184
00185
00186
00187
00188
00189
00190
00191
00192
00193
00194
00195
00196
00197
00198
00199
00200
00201
00202
00203
00204
00205
00206
00207
00208
00209
00210
00211
00212
00213
00214
00215
00216
00217
00218
00219
00220
00221
00222
00223
00224
00225
00226
00227
00228
00229
00230
00231
00232
00233
00234
00235
00236
00237
00238
00239
00240
00241
00242
00243
00244
00245
00246
00247
00248
00249
00250
00251
00252
00253
00254
00255
00256
00257
00258
00259
00260
00261
00262
00263
00264
00265
00266
00267
00268
00269
00270
00271
00272
00273
00274
00275
00276
00277
00278
00279
00280
00281
00282
00283
00284
00285
00286
00287
00288
00289
00290
00291
00292
00293
00294
00295
00296
00297
00298
00299
00300
00301
00302
00303
00304
00305
00306
00307
00308
00309
00310
00311
00312
00313
00314
00315
00316
00317
00318
00319
00320
00321
00322
00323
00324
00325
00326
00327
00328
00329
00330
00331
00332
00333
00334
00335
00336
00337
00338
00339
00340
00341
00342
00343
00344
00345
00346
00347
00348
00349
00350
00351
00352
00353
00354
00355
00356
00357
00358
00359
00360
00361
00362
00363
00364
00365
00366
00367
00368
00369
00370
00371
00372
00373
00374
00375
00376
00377
00378
00379
00380
00381
00382
00383
00384
00385
00386
00387
00388
00389
00390
00391
00392
00393
00394
00395
00396
00397
00398
00399
00400
00401
00402
00403
00404
00405
00406
00407
00408
00409
00410
00411
00412
00413
00414
00415
00416
00417
00418
00419
00420
00421
00422
00423
00424
00425
00426
00427
00428
00429
00430
00431
00432
00433
00434
00435
00436
00437
00438
00439
00440
00441
00442
00443
00444
00445
00446
00447
00448
00449
00450
00451
00452
00453
00454
00455
00456
00457
00458
00459
00460
00461
00462
00463
00464
00465
00466
00467
00468
00469
00470
00471
00472
00473
00474
00475
00476
00477
00478
00479
00480
00481
00482
00483
00484
00485
00486
00487
00488
00489
00490
00491
00492
00493
00494
00495
00496
00497
00498
00499
00500
00501
00502
00503
00504
00505
00506
00507
00508
00509
00510
00511
00512
00513
00514
00515
00516
00517
00518
00519
00520
00521
00522
00523
00524
00525
00526
00527
00528
00529
00530
00531
00532
00533
00534
00535
00536
00537
00538
00539
00540
00541
00542
00543
00544
00545
00546
00547
00548
00549
00550
00551
00552
00553
00554
00555
00556
00557
00558
00559
00560
00561
00562
00563
00564
00565
00566
00567
00568
00569
00570
00571
00572
00573
00574
00575
00576
00577
00578
00579
00580
00581
00582
00583
00584
00585
00586
00587
00588
00589
00590
00591
00592
00593
00594
00595
00596
00597
00598
00599
00600
00601
00602
00603
00604
00605
00606
00607
00608
00609
00610
00611
00612
00613
00614
00615
00616
00617
00618
00619
00620
00621
00622
00623
00624
00625
00626
00627
00628
00629
00630
00631
00632
00633
00634
00635
00636
00637
00638
00639
00640
00641
00642
00643
00644
00645
00646
00647
00648
00649
00650
00651
00652
00653
00654
00655
00656
00657
00658
00659
00660
00661
00662
00663
00664
00665
00666
00667
00668
00669
00670
00671
00672
00673
00674
00675
00676
00677
00678
00679
00680
00681
00682
00683
00684
00685
00686
00687
00688
00689
00690
00691
00692
00693
00694
00695
00696
00697
00698
00699
00700
00701
00702
00703
00704
00705
00706
00707
00708
00709
00710
00711
00712
00713
00714
00715
00716
00717
00718
00719
00720
00721
00722
00723
00724
00725
00726
00727
00728
00729
00730
00731
00732
00733
00734
00735
00736
00737
00738
00739
00740
00741
00742
00743
00744
00745
00746
00747
00748
00749
00750
00751
00752
00753
00754
00755
00756
00757
00758
00759
00760
00761
00762
00763
00764
00765
00766
00767
00768
00769
00770
00771
00772
00773
00774
00775
00776
00777
00778
00779
00780
00781
00782
00783
00784
00785
00786
00787
00788
00789
00790
00791
00792
00793
00794
00795
00796
00797
00798
00799
00800
00801
00802
00803
00804
00805
00806
00807
00808
00809
00810
00811
00812
00813
00814
00815
00816
00817
00818
00819
00820
00821
00822
00823
00824
00825
00826
00827
00828
00829
00830
00831
00832
00833
00834
00835
00836
00837
00838
00839
00840
00841
00842
00843
00844
00845
00846
00847
00848
00849
00850
00851
00852
00853
00854
00855
00856
00857
00858
00859
00860
00861
00862
00863
00864
00865
00866
00867
00868
00869
00870
00871
00872
00873
00874
00875
00876
00877
00878
00879
00880
00881
00882
00883
00884
00885
00886
00887
00888
00889
00890
00891
00892
00893
00894
00895
00896
00897
00898
00899
00900
00901
00902
00903
00904
00905
00906
00907
00908
00909
00910
00911
00912
00913
00914
00915
00916
00917
00918
00919
00920
00921
00922
00923
00924
00925
00926
00927
00928
00929
00930
00931
00932
00933
00934
00935
00936
00937
00938
00939
00940
00941
00942
00943
00944
00945
00946
00947
00948
00949
00950
00951
00952
00953
00954
00955
00956
00957
00958
00959
00960
00961
00962
00963
00964
00965
00966
00967
00968
00969
00970
00971
00972
00973
00974
00975
00976
00977
00978
00979
00980
00981
00982
00983
00984
00985
00986
00987
00988
00989
00990
00991
00992
00993
00994
00995
00996
00997
00998
00999

```

```

00067 virtual std::string show() override;
00068 public:
00069 static PluginInformation* GetPluginInformation();
00070 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00071 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00072 public:
00073 void setStationName(std::string stationName);
00074 void setStation(Station* _station);
00075 Station* getStation() const;
00076 void setStationExpression(std::string _stationExpression);
00077 std::string getStationExpression() const;
00078 void setRouteTimeExpression(std::string _routeTimeExpression);
00079 void setRouteTimeExpression(std::string _routeTimeExpression, Util::TimeUnit _routeTimeTimeUnit);
00080 std::string getRouteTimeExpression() const;
00081 void setRouteTimeTimeUnit(Util::TimeUnit _routeTimeTimeUnit);
00082 Util::TimeUnit getRouteTimeTimeUnit() const;
00083 void setRouteDestinationType(DestinationType _routeDestinationType);
00084 Route::DestinationType getRouteDestinationType() const;
00085 void setLabel(Label* _label);
00086 Label* getLabel() const;
00087 protected:
00088 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00089 virtual bool _loadInstance(PersistenceRecord *fields) override;
00090 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00091 protected:
00092 //virtual void _initBetweenReplications();
00093 virtual bool _check(std::string& errorMessage) override;
00094
00095 protected:
00096 virtual void _createInternalStatisticReporters() override;
00097 // virtual void _createNonEditableDataDefinitions() override;
00098 virtual void _createEditableDataDefinitions() override;
00099 virtual void _createAttachedAttributes() override;
00100 // virtual void _createInternalAndAttachedData() override;
00101
00102 protected:
00103
00104 private:
00105
00106 const struct DEFAULT_VALUES {
00107 const std::string routeTimeExpression = "0.0";
00108 const std::string stationExpression = "";
00109 const Util::TimeUnit routeTimeTimeUnit = Util::TimeUnit::second;
00110 const Route::DestinationType routeDestinationType = DestinationType::Station;
00111 } DEFAULT;
00112 std::string _routeTimeExpression = DEFAULT.routeTimeExpression;
00113 std::string _stationExpression = DEFAULT.stationExpression;
00114 Util::TimeUnit _routeTimeTimeUnit = DEFAULT.routeTimeTimeUnit;
00115 Route::DestinationType _routeDestinationType = DEFAULT.routeDestinationType;
00116 private: // association
00117 Station* _station = nullptr;
00118 Label* _label = nullptr;
00119 private: // internal elements
00120 Counter* _numberIn = nullptr;
00121 };
00122
00123 #endif /* ROUTE_H */

```

## 16.391 Start.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Start.h
00009 * Author: rlcancian
00010 *
00011 * Created on 11 de Setembro de 2019, 13:15
00012 */
00013
00014 #ifndef START_H
00015 #define START_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00039 class Start : public ModelComponent {
00040 public: // constructors
00041 Start(Model* model, std::string name = "");
00042 virtual ~Start() = default;
00043 public: // virtual
00044 virtual std::string show() override;
00045 public: // static

```

```

00046 static PluginInformation* GetPluginInformation();
00047 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00048 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00049 protected: // virtual
00050 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00051 virtual bool _loadInstance(PersistenceRecord *fields) override;
00052 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00053 protected: // virtual
00054 //virtual void _initBetweenReplications();
00055 // virtual void _createInternalStatisticReporters() override;
00056 // virtual void _createNonEditableDataDefinitions() override;
00057 // virtual void _createEditableDataDefinitions() override;
00058 // virtual void _createAttachedAttributes() override;
00059 virtual bool _check(std::string& errorMessage) override;
00060
00061 protected:
00062
00063 private: // methods
00064 private: // attributes 1:1
00065 private: // attributes 1:n
00066 };
00067
00068
00069 #endif /* START_H */

```

## 16.392 Stop.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Stop.h
00009 * Author: rlcancian
00010 *
00011 * Created on 11 de Setembro de 2019, 13:15
00012 */
00013
00014 #ifndef STOP_H
00015 #define STOP_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00036 class Stop : public ModelComponent {
00037 public: // constructors
00038 Stop(Model* model, std::string name = "");
00039 virtual ~Stop() = default;
00040 public: // virtual
00041 virtual std::string show() override;
00042 public: // static
00043 static PluginInformation* GetPluginInformation();
00044 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00045 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00046 protected: // virtual
00047 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00048 virtual bool _loadInstance(PersistenceRecord *fields) override;
00049 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00050 protected: // virtual
00051 //virtual void _initBetweenReplications();
00052 // virtual void _createInternalStatisticReporters() override;
00053 // virtual void _createNonEditableDataDefinitions() override;
00054 // virtual void _createEditableDataDefinitions() override;
00055 // virtual void _createAttachedAttributes() override;
00056 virtual bool _check(std::string& errorMessage) override;
00057
00058 protected:
00059
00060 private: // methods
00061 private: // attributes 1:1
00062 private: // attributes 1:n
00063 };
00064
00065
00066 #endif /* STOP_H */

```

## 16.393 Store.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates

```

```

00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Store.h
00009 * Author: rlcancian
00010 *
00011 * Created on 11 de Setembro de 2019, 13:07
00012 */
00013
00014 #ifndef STORE_H
00015 #define STORE_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00050 class Store : public ModelComponent {
00051 public: // constructors
00052 Store(Model* model, std::string name = "");
00053 virtual ~Store() = default;
00054 public: // virtual
00055 virtual std::string show() override;
00056 public: // static
00057 static PluginInformation* GetPluginInformation();
00058 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00059 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00060 protected: // virtual
00061 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00062 virtual bool _loadInstance(PersistenceRecord *fields) override;
00063 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00064 protected: // virtual
00065 //virtual void _initBetweenReplications();
00066 // virtual void _createInternalStatisticReporters() override;
00067 // virtual void _createNonEditableDataDefinitions() override;
00068 // virtual void _createEditableDataDefinitions() override;
00069 // virtual void _createAttachedAttributes() override;
00070 virtual bool _check(std::string& errorMessage) override;
00071
00072 protected:
00073
00074 private: // methods
00075 private: // attributes 1:1
00076 private: // attributes 1:n
00077 };
00078
00079
00080 #endif /* STORE_H */

```

## 16.394 Unstore.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Unstore.h
00009 * Author: rlcancian
00010 *
00011 * Created on 11 de Setembro de 2019, 13:08
00012 */
00013
00014 #ifndef UNSTORE_H
00015 #define UNSTORE_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00045 class Unstore : public ModelComponent {
00046 public: // constructors
00047 Unstore(Model* model, std::string name = "");
00048 virtual ~Unstore() = default;
00049 public: // virtual
00050 virtual std::string show() override;
00051 public: // static
00052 static PluginInformation* GetPluginInformation();
00053 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00054 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00055 protected: // virtual
00056 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00057 virtual bool _loadInstance(PersistenceRecord *fields) override;
00058 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00059 protected: // virtual
00060 //virtual void _initBetweenReplications();
00061 // virtual void _createInternalStatisticReporters() override;
00062 // virtual void _createNonEditableDataDefinitions() override;

```

```

00063 // virtual void _createEditableDataDefinitions() override;
00064 // virtual void _createAttachedAttributes() override;
00065 virtual bool _check(std::string& errorMessage) override;
00066 protected:
00067
00068
00069 private: // methods
00070 private: // attributes 1:1
00071 private: // attributes 1:n
00072 };
00073
00074
00075 #endif /* UNSTORE_H */

```

## 16.395 Boundary\_Closed.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: Boundary_Closed.h
00008 * Author: cancian
00009 *
00010 * Created on 10 de abril de 2023, 16:22
00011 */
00012 #pragma once
00013
00014 #include "plugins/components/ModalModel/CellularAutomata/BoundaryCondition.h"
00015 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00016 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00017 #include "plugins/components/ModalModel/CellularAutomata/State.h"
00018
00019 class Boundary_Closed : public BoundaryCondition {
00020 public:
00021
00022 Boundary_Closed(Lattice* lattice = nullptr, Neighborhood* neighborhood = nullptr)
00023 : BoundaryCondition(lattice, neighborhood) {
00024 fixedCell = new Cell();
00025 }
00026
00027 Boundary_Closed(const Boundary_Closed& orig)
00028 : BoundaryCondition(orig) {
00029 }
00030 virtual ~Boundary_Closed() = default;
00031 public:
00032 virtual Cell* getNeighborCell(std::vector<int> cellNDimPosition, std::vector<int>
neighborNDimPosition) override {
00033 long neighborCellNumber = lattice->cellNDimPosition2Number(neighborNDimPosition);
00034 if (neighborCellNumber < 0) { // out of bounds
00035 int pos, maxPos;
00036 for (unsigned short dim = 0; dim < neighborNDimPosition.size(); dim++) {
00037 pos = neighborNDimPosition.at(dim);
00038 maxPos = (int) (lattice->getDimension(dim));
00039 if (pos < 0 || pos >= maxPos) { // out of bound
00040 // Keep periodic wrapping valid for any radius, including large negative offsets.
00041 pos = ((pos % maxPos) + maxPos) % maxPos;
00042 // position corrected
00043 neighborNDimPosition.at(dim) = pos;
00044 }
00045 }
00046 //std::cout << " bond fixed to " <<Util::showNDimPosition(&neighborNDimPosition);
00047 neighborCellNumber = lattice->cellNDimPosition2Number(neighborNDimPosition);
00048 }
00049 return lattice->getCell(neighborCellNumber);
00050 }
00051 public:
00052
00053 void setFixedState(const State& fixedState) {
00054 fixedCell->setCurrentState(fixedState);
00055 }
00056
00057 const State& getFixedState() const {
00058 return fixedCell->getCurrentState();
00059 }
00060 private:
00061 Cell* fixedCell;
00062 };

```

## 16.396 Boundary\_Fixed.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: Boundary_Fixed.h
00008 * Author: cancian
00009 *
00010 * Created on 6 de abril de 2023, 13:48
00011 */
00012 #pragma once
00013
00014 #include "plugins/components/ModalModel/CellularAutomata/BoundaryCondition.h"
00015 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00016 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00017 #include "plugins/components/ModalModel/CellularAutomata/State.h"
00018
00019 class Boundary_Fixed : public BoundaryCondition {
00020 public:
00021
00022 Boundary_Fixed(Lattice* lattice = nullptr, Neighborhood* neighborhood = nullptr)
00023 : BoundaryCondition(lattice, neighborhood) {
00024 fixedCell = new Cell();
00025 fixedCell->setCellNumber(-99);
00026 }
00027
00028 Boundary_Fixed(const Boundary_Fixed& orig)
00029 : BoundaryCondition(orig) {
00030 }
00031 virtual ~Boundary_Fixed() = default;
00032 public:
00033
00034 virtual Cell* getNeighborCell(std::vector<int> cellNDimPosition, std::vector<int>
neighborNDimPosition) override {
00035 long neighborCellNumber = lattice->cellNDimPosition2Number(neighborNDimPosition);
00036 if (neighborCellNumber < 0) { // out of bounds
00037 //std::cout << " bond fixed to fixedCell";
00038 return fixedCell;
00039 }
00040 return lattice->getCell(neighborCellNumber);
00041 }
00042 public:
00043
00044 void setFixedState(const State& fixedState) {
00045 fixedCell->setCurrentState(fixedState);
00046 }
00047
00048 const State& getFixedState() const {
00049 return fixedCell->getCurrentState();
00050 }
00051 private:
00052 Cell* fixedCell;
00053 };

```

## 16.397 BoundaryCondition.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00004 // #include "plugins/components/ModalModel/CellularAutomata/Neighborhood.h"
00005 #include <string>
00006 #include <vector>
00007
00008
00009 class Neighborhood;
00010
00011 class BoundaryCondition {
00012 public:
00013 BoundaryCondition(Lattice* lattice = nullptr, Neighborhood* neighborhood=nullptr){
00014 this->lattice = lattice;
00015 this->neighborhood = neighborhood;
00016 }
00017 BoundaryCondition(const BoundaryCondition& orig){}
00018 virtual ~BoundaryCondition()=default;
00019 public:
00020 virtual std::string show() {
00021 return "todo Boundary.show()";
00022 }
00023 virtual Cell* getNeighborCell(std::vector<int> cellPosition, std::vector<int>
neighborCellPosition) = 0;
00024 public:

```



```

00025 void setNeighborhood(Neighborhood* neighborhood){
00026 this->neighborhood = neighborhood;
00027 }
00028 Neighborhood* getNeighborhood() const{
00029 return neighborhood;
00030 }
00031 void setLattice(Lattice* lattice){
00032 this->lattice = lattice;
00033 }
00034 Lattice* getLattice() const{
00035 return lattice;
00036 }
00037 protected:
00038 Lattice* lattice;
00039 Neighborhood* neighborhood;
00040 private:
00041 };

```

## 16.398 Cell.h

```

00001 #pragma once
00002
00003
00004
00005 #include "plugins/components/ModalModel/CellularAutomata/State.h"
00006 #include "plugins/components/ModalModel/CellularAutomata/StateSet.h"
00007 #include <memory>
00008 #include <vector>
00009 #include <string>
00010
00011 class State;
00012
00013 class Cell {
00014 public: // constructors
00015 Cell(StateSet* stateSet = nullptr, long cellNum=0, std::vector<int> position={});
00016 Cell(long cellNum, std::vector<int> position);
00017 Cell(const Cell& orig);
00018 Cell& operator=(const Cell& orig);
00019 virtual ~Cell()=default;
00020 public: // methods
00021 virtual std::string show();
00022 bool updateState();
00023 bool isUpdatePending();
00024 virtual void draw();
00025 public: // getters
00026 void setCellNumber(long cellNum);
00027 long getCellNumber() const;
00028 void setPosition(std::vector<int> position);
00029 std::vector<int> getPosition() const;
00030 void setStateSet(StateSet* stateSet);
00031 StateSet* getStateSet() const;
00032 const State& getPreviousState() const;
00033 bool setCurrentState(const State& currentState);
00034 const State& getCurrentState() const;
00035 bool setNextState(const State& nextState);
00036 const State& getNextState() const;
00037 void setNeighbors(std::vector<Cell*> neighbors);
00038 std::vector<Cell*> getNeighbors() const;
00039 protected:
00040 bool acceptsState(const State& state) const;
00041 std::unique_ptr<State> previousState;
00042 std::unique_ptr<State> currentState;
00043 std::unique_ptr<State> nextState;
00044 StateSet* stateSet = nullptr;
00045 bool updatePending;
00046 std::vector<int> position;
00047 long cellNum;
00048 std::vector<Cell*> neighbors;
00049 protected: // for other CA types
00050 private:
00051 };

```

## 16.399 CellularAutomata.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00004 #include "plugins/components/ModalModel/CellularAutomata/StateSet.h"
00005 #include "plugins/components/ModalModel/CellularAutomata/Neighborhood.h"
00006 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00007 #include <string>
00008 #include <functional>

```

```

00009 #include <list>
00010 #include <thread>
00011
00012 class CellularAutomata;
00013
00014 class CellularAutomataEvent { // event for observers
00015 public:
00016 CellularAutomataEvent(CellularAutomata* ca) {
00017 this->ca = ca;
00018 }
00019 CellularAutomata* ca;
00020 };
00021
00022 typedef std::function<void(CellularAutomataEvent) > CellularAutomataEventObserver;
00023
00024 class CellularAutomata {
00025 public:
00026 CellularAutomata(std::string name = "", Lattice* lattice = nullptr, StateSet* stateSet = nullptr,
00027 Neighborhood* neighborhood = nullptr, LocalRule* localRule = nullptr);
00028 CellularAutomata(const CellularAutomata& orig);
00029 virtual ~CellularAutomata() = default;
00030 public:
00031 //virtual std::string show();
00032 virtual bool init();
00033 virtual void step();
00034 virtual void start();
00035 virtual void pause();
00036 template<typename Class> void addObserver(Class * object, void
00037 (Class::*function) (CellularAutomataEvent));
00038 public:
00039 std::string getName() const;
00040 void setName(const std::string name);
00041 void setLocalRule(LocalRule* localRule);
00042 LocalRule* getLocalRule() const;
00043 void setNeighborhood(Neighborhood* neighborhood);
00044 Neighborhood* getNeighborhood() const;
00045 void setStateSet(StateSet* stateSet);
00046 StateSet* getStateSet() const;
00047 void setLattice(Lattice* lattice);
00048 Lattice* getLattice() const;
00049 void setSimulationLength(unsigned int simulationLength);
00050 unsigned int getSimulationLength() const;
00051 unsigned int getSimulatedTime() const;
00052 bool isRunning() const;
00053 protected:
00054 void notifyObservers(CellularAutomataEvent event);
00055 virtual void applyLocalRule() = 0;
00056 protected:
00057 Lattice* lattice;
00058 StateSet* stateSet;
00059 Neighborhood* neighborhood;
00060 LocalRule* localRule;
00061 std::string name;
00062 unsigned int simulatedTime;
00063 unsigned int simulationLength;
00064 bool running;
00065 private:
00066 std::list<CellularAutomataEventObserver*> observers = new
00067 std::list<CellularAutomataEventObserver>();
00068 };
00069
00070 template<typename Class> void CellularAutomata::addObserver(Class * object, void
00071 (Class::*function) (CellularAutomataEvent)) {
00072 CellularAutomataEventObserver observer = std::bind(function, object, std::placeholders::_1);
00073 observers->emplace_back(observer);
00074 }

```

## 16.400 CellularAutomata\_1DTimed.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00004 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomataBase.h"
00005 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00006 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00007 #include "plugins/components/ModalModel/CellularAutomata/State.h"
00008
00009 class CellularAutomata_1DTimed: public CellularAutomataBase {
00010 public:
00011 CellularAutomata_1DTimed(Lattice* lattice = nullptr, StateSet* stateSet = nullptr, Neighborhood*
00012 neighborhood = nullptr, LocalRule* localRule = nullptr)
00013 :CellularAutomataBase(lattice, stateSet, neighborhood, localRule) { }
00014 CellularAutomata_1DTimed(const CellularAutomata_1DTimed& orig)
00015 : CellularAutomataBase(orig){ }
00016 virtual ~CellularAutomata_1DTimed() = default;

```

```

00017 protected:
00018 virtual void applyLocalRule() override {
00019 Cell* cell;
00020 int nextRow;
00021 int maxRows = std::min<int>(lattice->getDimension(1)-2, lattice->getDimension(1)-2); //
simulatedTime);
00022 for (int row=maxRows; row>=0; row--) {
00023 nextRow = row + 1;
00024 // move cell states to the next row
00025 for (int col=0; col<lattice->getDimension(0); col++) { // move to row below
00026 cell = lattice->getCell({col,row});
00027 lattice->setCellState({col,nextRow}, const_cast<State*>(&cell->getCurrentState()));
00028 }
00029 if (row==0) {
00030 // apply the same local rule for all unidimensional cells at the same time
00031 for (int col=0; col<lattice->getDimension(0); col++) {
00032 cell = lattice->getCell({col,row});
00033 localRule->applyRule(cell);
00034 }
00035 }
00036 }
00037 // alter changing the nextState of all unidimensional cells, update them (currentState =
nextState)
00038 for (int col=0; col<lattice->getDimension(0); col++) {
00039 cell = lattice->getCell({col,0});
00040 cell->updateState();
00041 }
00042 }
00043 protected:
00044 };

```

## 16.401 CellularAutomata\_Classic.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomataBase.h"
00004 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00005
00007 class CellularAutomata_Classic: public CellularAutomataBase {
00008 public:
00009 CellularAutomata_Classic(Lattice* lattice = nullptr, StateSet* stateSet = nullptr, Neighborhood*
neighborhood = nullptr, LocalRule* localRule = nullptr)
00010 :CellularAutomataBase(lattice, stateSet, neighborhood, localRule) { }
00011 CellularAutomata_Classic(const CellularAutomata_Classic& orig): CellularAutomataBase(orig){ }
00012 virtual ~CellularAutomata_Classic() = default;
00013 protected:
00014 virtual void applyLocalRule() override {
00015 Cell* cell;
00016 // apply the same local rule to all cells at the same time
00017 for (long cellNumber = 0; cellNumber < lattice->getCellsSize(); cellNumber++) {
00018 cell = lattice->getCell(cellNumber);
00019 localRule->applyRule(cell);
00020 }
00021 // alter changing the nextState of all cells, update them (currentState = nextState)
00022 for (long cellNumber = 0; cellNumber < lattice->getCellsSize(); cellNumber++) {
00023 cell = lattice->getCell(cellNumber);
00024 cell->updateState();
00025 }
00026 }
00027 };

```

## 16.402 CellularAutomataAllHeaders.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/BoundaryCondition.h"
00004 #include "plugins/components/ModalModel/CellularAutomata/Boundary_Closed.h"
00005 #include "plugins/components/ModalModel/CellularAutomata/Boundary_Fixed.h"
00006 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00007 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomata.h"
00008 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomataBase.h"
00009 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomata_1DTimed.h"
00010 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomata_Classic.h"
00011 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00012 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00013 #include "plugins/components/ModalModel/CellularAutomata/LocalRule_Elementary.h"
00014 #include "plugins/components/ModalModel/CellularAutomata/LocalRule_FlorestalFire.h"
00015 #include "plugins/components/ModalModel/CellularAutomata/LocalRule_GameOfLife.h"
00016 #include "plugins/components/ModalModel/CellularAutomata/LocalRule_Growty.h"
00017 #include "plugins/components/ModalModel/CellularAutomata/LocalRule_HppLatticeGas.h"
00018 #include "plugins/components/ModalModel/CellularAutomata/LocalRule_SandRock_Student.h"
00019 #include "plugins/components/ModalModel/CellularAutomata/LocalRule_ShowNeighborhood.h"

```

```

00020 #include "plugins/components/ModalModel/CellularAutomata/Neighborhood.h"
00021 #include "plugins/components/ModalModel/CellularAutomata/Neighborhood_Center.h"
00022 #include "plugins/components/ModalModel/CellularAutomata/Neighborhood_Moore.h"
00023 #include "plugins/components/ModalModel/CellularAutomata/Neighborhood_VonNeumann.h"
00024 #include "plugins/components/ModalModel/CellularAutomata/State.h"
00025 #include "plugins/components/ModalModel/CellularAutomata/StateSet.h"
00026 #include "plugins/components/ModalModel/CellularAutomata/StateSet_Enumerable.h"
00027 #include "plugins/components/ModalModel/CellularAutomata/State_Numerical.h"

```

## 16.403 CellularAutomataBase.h

```

00001 #pragma once
00002
00003
00004 #include <string>
00005 #include <functional>
00006 #include <list>
00007 #include <thread>
00008
00009 class Lattice;
00010 class StateSet;
00011 class Neighborhood;
00012 class LocalRule;
00013
00014
00015 class CellularAutomataBase {
00016 public:
00017 CellularAutomataBase(Lattice* lattice = nullptr, StateSet* stateSet = nullptr, Neighborhood*
neighborhood = nullptr, LocalRule* localRule = nullptr);
00018 CellularAutomataBase(const CellularAutomataBase& orig);
00019 virtual ~CellularAutomataBase() = default;
00020 public:
00021 //virtual std::string show();
00022 virtual bool init();
00023 virtual void step();
00024 public:
00025 void setLocalRule(LocalRule* localRule);
00026 LocalRule* getLocalRule() const;
00027 void setNeighborhood(Neighborhood* neighborhood);
00028 Neighborhood* getNeighborhood() const;
00029 void setStateSet(StateSet* stateSet);
00030 StateSet* getStateSet() const;
00031 void setLattice(Lattice* lattice);
00032 Lattice* getLattice() const;
00033 protected:
00034 virtual void applyLocalRule() = 0;
00035 protected:
00036 Lattice* lattice;
00037 StateSet* stateSet;
00038 Neighborhood* neighborhood;
00039 LocalRule* localRule;
00040 };

```

## 16.404 Lattice.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00004 #include <vector>
00005 #include <string>
00006
00007 class CellularAutomataBase;
00008
00009 enum class LatticeType : int {
00010 RETICULAR = 1, TRIANGULAR = 2, HEXAGONAL = 3, NETWORK = 4, USERDEFINED = 5
00011 };
00012
00013
00014 class Lattice {
00015 public:
00016 Lattice(CellularAutomataBase* parentCellularAutomata, Cell* progenitorCell = nullptr,
std::vector<unsigned short> dimensions={}, LatticeType latticeType = LatticeType::RETICULAR);
00017 Lattice(const Lattice& orig);
00018 virtual ~Lattice() = default;
00019 public:
00020 virtual std::string show();
00021 bool init();
00022 virtual long cellNDimPosition2Number(const std::vector<int> position);
00023 virtual std::vector<int> cellNumber2NDimPosition(const long cellNumber);
00024 Cell* getCell(const long cellNumber);
00025 Cell* getCell(const std::vector<int> position);
00026 void setCell(const std::vector<int> position, Cell* cell);

```

```

00028 bool setCellState(std::vector<int> position, State* state = nullptr, Cell* cell = nullptr);
00029 bool setCellState(long cellNumber, State* state = nullptr, Cell* cell = nullptr);
00030 public:
00031 void setProgenitorCell(Cell *progenitorCell);
00032 Cell* getProgenitorCell() const;
00033 bool setNumCells(unsigned short dimension, unsigned int numCells, bool createDimensions = true);
00034 unsigned short getNumDimensions();
00035 unsigned int getNumCell(unsigned short dimension);
00036 LatticeType getLatticeType() const;
00037 unsigned long getCellsSize() const;
00038 unsigned long getTotalCells() const;
00039 unsigned long calculateTotalCells();
00040 bool isInit() const;
00041 void setDimensions(std::vector<unsigned short> dimensions);
00042 std::vector<unsigned short> getDimensions() const;
00043 unsigned short getDimension(unsigned short dimensionNumber) const;
00044 std::vector<Cell*> getCells() const;
00045 protected:
00046 CellularAutomataBase* parentCellularAutomata;
00047 std::vector<Cell*> cells;
00048 std::vector<unsigned short> dimensions;
00049 Cell *progenitorCell;
00050 LatticeType latticeType;
00051 bool hasBeenInit;
00052 unsigned long totalCells;
00053 };

```

## 16.405 LocalRule.h

```

00001 #pragma once
00002 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00003 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomataBase.h"
00004
00005
00006 class LocalRule {
00007 public:
00008 LocalRule(CellularAutomataBase* parentCellularAutomata, StateSet* stateSet=nullptr){
00009 this->parentCellularAutomata = parentCellularAutomata;
00010 parentCellularAutomata->setLocalRule(this);
00011 if (stateSet != nullptr)
00012 this->stateSet = stateSet;
00013 else
00014 this->stateSet = parentCellularAutomata->getStateSet();
00015 }
00016 LocalRule(const LocalRule& orig){}
00017 virtual ~LocalRule()=default;
00018 public:
00019 virtual void applyRule(Cell* cell) = 0;
00020 public:
00021 void setStateSet(StateSet* stateSet){
00022 this->stateSet = stateSet;
00023 }
00024 protected:
00025 CellularAutomataBase* parentCellularAutomata;
00026 StateSet* stateSet;
00027 };

```

## 16.406 LocalRule\_Elementary.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00004 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00005 #include "plugins/components/ModalModel/CellularAutomata/StateSet_Enumerable.h"
00006 #include <cmath>
00007
00008 class LocalRule_Elementary: public LocalRule {
00009 public:
00010 LocalRule_Elementary(CellularAutomataBase* parentCellularAutomata, uint8_t ruleNumber=30,
00011 StateSet* stateSet=nullptr)
00012 :LocalRule(parentCellularAutomata, stateSet) {
00013 this->ruleNumber = ruleNumber;
00014 }
00015 LocalRule_Elementary(const LocalRule_Elementary& orig): LocalRule(orig) { }
00016 virtual ~LocalRule_Elementary()=default;
00017 public:
00018 virtual void applyRule(Cell* cell) override {
00019 long number = 0;
00020 int bit, power = 2;
00021 for (Cell* neigh : cell->getNeighbors()) {
00022 number += neigh->getCurrentState().getValue() * std::pow(2, power--);
00023 if (power==1)

```

```

00023 number += std::pow(2, power--)*cell->getCurrentState().getValue();
00024 }
00025 bit = (ruleNumber & (uint8_t)std::pow(2,number)) » number;
00026 cell->setNextState(State(bit));
00027 }
00028 public:
00029 uint8_t getRuleNumber() const { return ruleNumber; }
00030 void setRuleNumber(const uint8_t ruleNumber) {this->ruleNumber = ruleNumber;}
00031 private:
00032 uint8_t ruleNumber;
00033 };

```

## 16.407 LocalRule\_FlorestalFire.h

```

00001 #pragma once
00002
00003 #include <random>
00004 #include <vector>
00005
00006 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00007 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00008 #include "plugins/components/ModalModel/CellularAutomata/StateSet_Enumerable.h"
00009
00010 class LocalRule_FlorestalFire : public LocalRule {
00011 public:
00012
00013 LocalRule_FlorestalFire(CellularAutomataBase* parentCellularAutomata)
00014 : LocalRule(parentCellularAutomata) {
00015 }
00016
00017 LocalRule_FlorestalFire(const LocalRule_FlorestalFire& orig)
00018 : LocalRule(orig) {
00019 }
00020 virtual ~LocalRule_FlorestalFire() = default;
00021 public:
00022
00023 virtual void applyRule(Cell* cell) override {
00024 if (cell == nullptr || stateSet == nullptr) {
00025 return;
00026 }
00027
00028 auto* enumerableStateSet = dynamic_cast<StateSet_Enumerable*>(stateSet);
00029 if (enumerableStateSet == nullptr || enumerableStateSet->getStatesSize() < 4) {
00030 cell->setNextState(cell->getCurrentState());
00031 return;
00032 }
00033
00034 std::random_device randomDevice;
00035 std::default_random_engine engine(randomDevice());
00036 std::uniform_real_distribution<double> uniform(0.0, 1.0);
00037 std::vector<unsigned int> sumNeighStates(enumerableStateSet->getStatesSize(), 0);
00038
00039 for (Cell* neigh : cell->getNeighbors()) {
00040 const long stateValue = neigh->getCurrentState().getValue();
00041 if (stateValue >= 0 && static_cast<size_t>(stateValue) < sumNeighStates.size()) {
00042 sumNeighStates.at(static_cast<size_t>(stateValue))++;
00043 }
00044 }
00045
00046 const long currentState = cell->getCurrentState().getValue();
00047 cell->setNextState(cell->getCurrentState());
00048
00049 switch (currentState) {
00050 case 0: // soil
00051 if (sumNeighStates.size() > 1 && uniform(engine) < pSoil2Grass + sumNeighStates.at(1)
00052 * pSoil2Grass) {
00053 cell->setNextState(*enumerableStateSet->getState(1));
00054 }
00055 break;
00056 case 1: // grass
00057 if (sumNeighStates.size() > 2 && uniform(engine) < pGrass2Tree + sumNeighStates.at(2)
00058 * pGrass2Tree) {
00059 cell->setNextState(*enumerableStateSet->getState(2));
00060 }
00061 else if (sumNeighStates.size() > 3 &&
00062 uniform(engine) < pGrass2Fire + (sumNeighStates.at(3) > 0 ?
00063 pGrassPropagateFire : 0.0)) {
00064 cell->setNextState(*enumerableStateSet->getState(3));
00065 }
00066 break;
00067 case 2: // tree
00068 if (sumNeighStates.size() > 3 &&
00069 uniform(engine) < pTree2Fire + (sumNeighStates.at(3) > 0 ? pTreePropagateFire :
00070 0.0)) {
00071 cell->setNextState(*enumerableStateSet->getState(3));
00072 }
00073 break;
00074 default:
00075 cell->setNextState(*enumerableStateSet->getState(0));
00076 }
00077 }

```

```

00067 } else if (sumNeighStates.size() > 0 && uniform(engine) < pTree2Soil +
sumNeighStates.at(0) * pTree2Soil) {
00068 cell->setNextState(*enumerableStateSet->getState(0));
00069 }
00070 break;
00071 case 3: // fire
00072 cell->setNextState(*enumerableStateSet->getState(0));
00073 break;
00074 default:
00075 break;
00076 }
00077 }
00078 protected:
00079 private:
00080 double pSoil2Grass = 0.01;
00081 double pGrass2Tree = 0.001;
00082 double pGrass2Fire = 0.00001;
00083 double pTree2Fire = 0.00001;
00084 double pGrassPropagateFire = 0.05;
00085 double pTreePropagateFire = 0.5;
00086 double pTree2Soil = 0.0005;
00087 };

```

## 16.408 LocalRule\_GameOfLife.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00004 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00005 #include "plugins/components/ModalModel/CellularAutomata/State.h"
00006
00007 class LocalRule_GameOfLife: public LocalRule {
00008 public:
00009 LocalRule_GameOfLife(CellularAutomataBase* parentCellularAutomata)
00010 : LocalRule(parentCellularAutomata) { }
00011 LocalRule_GameOfLife(const LocalRule_GameOfLife& orig): LocalRule(orig) { }
00012 virtual ~LocalRule_GameOfLife()=default;
00013 public:
00014 virtual void applyRule(Cell* cell) override {
00015 unsigned int living = 0;
00016 for (Cell* neigh : cell->getNeighbors()) {
00017 living += neigh->getCurrentState().getValue();
00018 }
00019 if (cell->getCurrentState().getValue() == 1) {
00020 if (living < 2 || living > 3)
00021 cell->setNextState(State(0));
00022 else
00023 cell->setNextState(cell->getCurrentState());
00024 } else if (living == 3)
00025 cell->setNextState(State(1));
00026 else
00027 cell->setNextState(cell->getCurrentState());
00028 }
00029 };

```

## 16.409 LocalRule\_Growty.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00004 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00005
00006 class LocalRule_Growty : public LocalRule {
00007 public:
00008 LocalRule_Growty(CellularAutomataBase* parentCellularAutomata)
00009 : LocalRule(parentCellularAutomata) { }
00010 LocalRule_Growty(const LocalRule_Growty& orig): LocalRule(orig) { }
00011 virtual ~LocalRule_Growty() = default;
00012 public:
00013 virtual void applyRule(Cell* cell) override {
00014 unsigned int sum = 0;
00015 for (Cell* neigh : cell->getNeighbors()) {
00016 sum += neigh->getCurrentState().getValue();
00017 }
00018 if (sum <= 3 || sum == 5)
00019 cell->setNextState(State(0));
00020 else
00021 cell->setNextState(State(1));
00022 }
00023 };

```

## 16.410 LocalRule\_HppLatticeGas.h

```

00001 #pragma once
00002
00003 #include <iostream>
00004 #include <vector>
00005
00006 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00007 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00008 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00009 #include "plugins/components/ModalModel/CellularAutomata/State.h"
00010 #include "plugins/components/ModalModel/CellularAutomata/StateSet.h"
00011
00012 class LocalRule_HppLatticeGas : public LocalRule {
00013 public:
00014 // Simplified HPP lattice gas: particles stream on the four von Neumann
00015 // directions and head-on two-particle collisions rotate by 90 degrees.
00016 // Conceptual reference: local PDF "2018_Book_Adamatzky A. Cellular Automata...Systems Science 2ed
00017 // 2018.pdf",
00018 // physical systems / lattice-gas cellular automata discussion.
00019 explicit LocalRule_HppLatticeGas(CellularAutomataBase* parentCellularAutomata)
00020 : LocalRule(parentCellularAutomata) {
00021 }
00022
00023 void applyRule(Cell* cell) override {
00024 if (cell == nullptr || parentCellularAutomata == nullptr ||
00025 parentCellularAutomata->getLattice() == nullptr) {
00026 return;
00027 }
00028 if (stateSet != nullptr && dynamic_cast<HppLatticeGasStateSet*>(stateSet) == nullptr) {
00029 std::cerr << "LocalRule_HppLatticeGas requires HppLatticeGasStateSet." << std::endl;
00030 cell->setNextState(cell->getCurrentState());
00031 return;
00032 }
00033
00034 Lattice* lattice = parentCellularAutomata->getLattice();
00035 const std::vector<int> position = cell->getPosition();
00036 if (position.size() < 2 || lattice->getNumDimensions() < 2) {
00037 cell->setNextState(cell->getCurrentState());
00038 return;
00039 }
00040 // sem tempo para acessar por Von Neuman
00041 Cell* northCell = lattice->getCell(wrappedPosition({position[0], position[1] - 1}, lattice));
00042 Cell* eastCell = lattice->getCell(wrappedPosition({position[0] + 1, position[1]}, lattice));
00043 Cell* southCell = lattice->getCell(wrappedPosition({position[0], position[1] + 1}, lattice));
00044 Cell* westCell = lattice->getCell(wrappedPosition({position[0] - 1, position[1]}, lattice));
00045
00046 HppLatticeGasState incoming;
00047 incoming.set(HppLatticeGasState::North, hasDirection(southCell, HppLatticeGasState::North));
00048 incoming.set(HppLatticeGasState::East, hasDirection(westCell, HppLatticeGasState::East));
00049 incoming.set(HppLatticeGasState::South, hasDirection(northCell, HppLatticeGasState::South));
00050 incoming.set(HppLatticeGasState::West, hasDirection(eastCell, HppLatticeGasState::West));
00051
00052 const bool verticalCollision = incoming.has(HppLatticeGasState::North) &&
00053 incoming.has(HppLatticeGasState::South) &&
00054 !incoming.has(HppLatticeGasState::East) &&
00055 !incoming.has(HppLatticeGasState::West);
00056 const bool horizontalCollision = incoming.has(HppLatticeGasState::East) &&
00057 incoming.has(HppLatticeGasState::West) &&
00058 !incoming.has(HppLatticeGasState::North) &&
00059 !incoming.has(HppLatticeGasState::South);
00060
00061 if (verticalCollision) {
00062 cell->setNextState(HppLatticeGasState({{false, true, false, true}}));
00063 } else if (horizontalCollision) {
00064 cell->setNextState(HppLatticeGasState({{true, false, true, false}}));
00065 } else {
00066 cell->setNextState(incoming);
00067 }
00068 }
00069 private:
00070 static std::vector<int> wrappedPosition(std::vector<int> position, Lattice* lattice) {
00071 for (unsigned short dimension = 0; dimension < position.size(); ++dimension) {
00072 const int dimensionSize = static_cast<int>(lattice->getDimension(dimension));
00073 position[dimension] = ((position[dimension] % dimensionSize) + dimensionSize) %
00074 dimensionSize;
00075 }
00076 return position;
00077 }
00078
00079 static bool hasDirection(Cell* cell, HppLatticeGasState::Direction direction) {
00080 if (cell == nullptr) {
00081 return false;
00082 }
00083 const auto* hpp = cell->getCurrentState().as<HppLatticeGasState>();
00084 if (hpp != nullptr) {

```



```

00083 return hpp->has(direction);
00084 }
00085 return (cell->getCurrentState().getValue() & (1L « direction)) != 0;
00086 }
00087 };

```

## 16.411 LocalRule\_SandRock.h

```

00001 // Aluno: Matheus Antonio de Souza - 21203363
00002 #pragma once
00003
00004 #include <vector>
00005
00006 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00007 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00008 #include "plugins/components/ModalModel/CellularAutomata/StateSet_Enumerable.h"
00009
00010 class LocalRule_SandRock_Student : public LocalRule {
00011 public:
00012 LocalRule_SandRock_Student(CellularAutomataBase* parentCellularAutomata)
00013 : LocalRule(parentCellularAutomata) {
00014 }
00015
00016 LocalRule_SandRock_Student(const LocalRule_SandRock_Student& orig)
00017 : LocalRule(orig) {
00018 }
00019 virtual ~LocalRule_SandRock_Student() = default;
00020 static constexpr long EMPTY = 0;
00021 static constexpr long SAND = 1;
00022 static constexpr long ROCK = 2;
00023 public:
00024
00025 virtual void applyRule(Cell* cell) override {
00026 if (cell == nullptr || stateSet == nullptr) {
00027 return;
00028 }
00029
00030 auto* enumerableStateSet = dynamic_cast<StateSet_Enumerable*>(stateSet);
00031 if (enumerableStateSet == nullptr || enumerableStateSet->getStatesSize() < 3) {
00032 cell->setNextState(cell->getCurrentState());
00033 return;
00034 }
00035
00036 auto pos = cell->getPosition();
00037 // Cada célula possui uma preferência entre cair para esquerda ou para direita
00038 // Neste caso esta sendo utilizado um padrão xadrez que é visualmente agradável
00039 bool preferLeft = ((pos[0] + pos[1]) & 1) != 0;
00040
00041 const auto& nbrs = cell->getNeighbors();
00042 // 0 5 10 14 19
00043 // 1 6 11 15 20
00044 // 2 7 16 21
00045 // 3 8 12 17 22
00046 // 4 9 13 18 23
00047
00048 // nw2 nnw n2 nne ne2
00049 // wnw nw n ne ene
00050 // w2 w e e2
00051 // wsw sw s se ese
00052 // sw2 ssw s2 sse se2
00053
00054 long nw = nbrs[6]->getCurrentState().getValue();
00055 long w = nbrs[7]->getCurrentState().getValue();
00056 long sw = nbrs[8]->getCurrentState().getValue();
00057
00058 long n = nbrs[11]->getCurrentState().getValue();
00059 long s = nbrs[12]->getCurrentState().getValue();
00060
00061 long ne = nbrs[15]->getCurrentState().getValue();
00062 long e = nbrs[16]->getCurrentState().getValue();
00063 long se = nbrs[17]->getCurrentState().getValue();
00064
00065 const long wnw = nbrs[1]->getCurrentState().getValue();
00066 const long w2 = nbrs[2]->getCurrentState().getValue();
00067 const long wsw = nbrs[3]->getCurrentState().getValue();
00068
00069 const long ene = nbrs[20]->getCurrentState().getValue();
00070 const long e2 = nbrs[21]->getCurrentState().getValue();
00071 const long ese = nbrs[22]->getCurrentState().getValue();
00072
00073 const long currentState = cell->getCurrentState().getValue();
00074
00075 // A estratégia funciona da seguinte maneira para uma célula areia:
00076 // 1) Se a célula de baixo (s) estiver livre, a areia cai verticalmente (vira vazio)
00077 // 2) Se (s) está bloqueado, e preferLeft

```

```

00078 // - tenta ir para (sw).
00079 // - se falhar, tenta (se).
00080 // 3) Analogamente, em caso de !preferLeft
00081 // - tenta ir para (se).
00082 // - se falhar, tenta (sw).
00083 // 4) Não há movimentos válidos, permanece onde está
00084 //
00085 // Do ponto de vista de uma célula vazia, ela segue a mesma lógica observando os grãos de
areia de cima
00086 // que tentariam escorregar para sua posição, e se o grão cederia ou não a preferencia.
00087 // Isto garante a conservação de matéria. Pelo menos nos casos que testei.
00088 // Um comportamento que não consegui reproduzir é o de preencher completamente um recipiente
dependendo de
00089 // seu formato.
00090 switch (currentState) {
00091 case SAND: {
00092 // 1) Queda vertical tem prioridade máxima.
00093 if (s == EMPTY) {
00094 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00095 return;
00096 }
00097
00098 // 2) Tentativa de escorregamento diagonal.
00099 bool canSW = (sw == EMPTY) && (w != SAND && w != ROCK);
00100 bool canSE = (se == EMPTY) && (e != SAND && e != ROCK);
00101
00102 if (preferLeft) {
00103 if (canSW) {
00104 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00105 return;
00106 }
00107 // Fallback para SE: só escorrega para direita (se) se não ha uma areia a direita(e2)
com
00108 // preferencia para cair em SE. Por ser preferLeft ser definido em xadrez, sei que ela
00109 // também é preferLeft e cederá passagem para ela.
00110 if (canSE && !(e2 == SAND && ese != EMPTY)) {
00111 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00112 return;
00113 }
00114 } else {
00115 if (canSE) {
00116 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00117 return;
00118 }
00119 // Fallback para SW: análogo, cedo se w2 quer meu sw.
00120 if (canSW && !(w2 == SAND && wsw != EMPTY)) {
00121 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00122 return;
00123 }
00124 }
00125
00126 // Não consigo me mover: permaneço SAND.
00127 cell->setNextState(*enumerableStateSet->getState(SAND));
00128 break;
00129 }
00130 case EMPTY: {
00131 // 1) não cai verticalmente.
00132 if (n == SAND) {
00133 cell->setNextState(*enumerableStateSet->getState(SAND));
00134 return;
00135 }
00136
00137 // 2) se há rock em cima não tem como areia escorregar pra aqui
00138 if (n == ROCK) {
00139 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00140 return;
00141 }
00142
00143 // 3) escorregamento diagonal.
00144 // (Sabemos n != SAND, então diagonais não competem com queda vertical)
00145 bool nw_fills = false;
00146 if (nw == SAND && w != EMPTY) {
00147 if (preferLeft) {
00148 // por xadrez, nw prefere seu sw(w2)
00149 const bool canSW_of_nw = (w2 == EMPTY) && (wnw == EMPTY);
00150 //
00151 const bool fallback_safe = !(ne == SAND && e != EMPTY);
00152
00153 if (!canSW_of_nw && fallback_safe) {
00154 nw_fills = true;
00155 }
00156 } else {
00157 // nw prefere seu se, direção preferida.
00158 nw_fills = true;
00159 }
00160 }
00161

```

```

00162 // simétrico
00163 bool ne_fills = false;
00164 if (ne == SAND && e != EMPTY) {
00165 if (preferLeft) {
00166 ne_fills = true;
00167 } else {
00168 const bool canSE_of_ne = (e2 == EMPTY) && (ene == EMPTY);
00169 const bool fallback_safe = !(nw == SAND && w != EMPTY);
00170 if (!canSE_of_ne && fallback_safe) {
00171 ne_fills = true;
00172 }
00173 }
00174 }
00175
00176 if (nw_fills || ne_fills) {
00177 cell->setNextState(*enumerableStateSet->getState(SAND));
00178 } else {
00179 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00180 }
00181 break;
00182 }
00183 case ROCK: {
00184 // objeto imóvel, sempre é pedra
00185 cell->setNextState(*enumerableStateSet->getState(ROCK));
00186 break;
00187 }
00188 default:
00189 cell->setNextState(cell->getCurrentState());
00190 break;
00191 }
00192 }
00193 protected:
00194 private:
00195 };

```

## 16.412 LocalRule\_SandRock\_Student.h

```

00001 // Aluno: Matheus Antonio de Souza - 21203363
00002 #pragma once
00003
00004 #include <vector>
00005
00006 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00007 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00008 #include "plugins/components/ModalModel/CellularAutomata/StateSet_Enumerable.h"
00009
00010 class LocalRule_SandRock_Student : public LocalRule {
00011 public:
00012 LocalRule_SandRock_Student(CellularAutomataBase* parentCellularAutomata)
00013 : LocalRule(parentCellularAutomata) {
00014 }
00015
00016 LocalRule_SandRock_Student(const LocalRule_SandRock_Student& orig)
00017 : LocalRule(orig) {
00018 }
00019 virtual ~LocalRule_SandRock_Student() = default;
00020 static constexpr long EMPTY = 0;
00021 static constexpr long SAND = 1;
00022 static constexpr long ROCK = 2;
00023 public:
00024
00025 virtual void applyRule(Cell* cell) override {
00026 if (cell == nullptr || stateSet == nullptr) {
00027 return;
00028 }
00029
00030 auto* enumerableStateSet = dynamic_cast<StateSet_Enumerable*>(stateSet);
00031 if (enumerableStateSet == nullptr || enumerableStateSet->getStatesSize() < 3) {
00032 cell->setNextState(cell->getCurrentState());
00033 return;
00034 }
00035
00036 auto pos = cell->getPosition();
00037 // Cada célula possui uma preferência entre cair para esquerda ou para direita
00038 // Neste caso esta sendo utilizado um padrão xadrez que é visualmente agradável
00039 bool preferLeft = ((pos[0] + pos[1]) & 1) != 0;
00040
00041 const auto& nbrs = cell->getNeighbors();
00042 // 0 5 10 14 19
00043 // 1 6 11 15 20
00044 // 2 7 16 21
00045 // 3 8 12 17 22
00046 // 4 9 13 18 23
00047
00048 // nw2 nnw n2 nne ne2

```

```

00049 // wnw nw n ne ene
00050 // w2 w e e2
00051 // wsw sw s se ese
00052 // sw2 ssw s2 sse se2
00053
00054 long nw = nbrs[6]->getCurrentState().getValue();
00055 long w = nbrs[7]->getCurrentState().getValue();
00056 long sw = nbrs[8]->getCurrentState().getValue();
00057
00058 long n = nbrs[11]->getCurrentState().getValue();
00059 long s = nbrs[12]->getCurrentState().getValue();
00060
00061 long ne = nbrs[15]->getCurrentState().getValue();
00062 long e = nbrs[16]->getCurrentState().getValue();
00063 long se = nbrs[17]->getCurrentState().getValue();
00064
00065 const long wnw = nbrs[1]->getCurrentState().getValue();
00066 const long w2 = nbrs[2]->getCurrentState().getValue();
00067 const long wsw = nbrs[3]->getCurrentState().getValue();
00068
00069 const long ene = nbrs[20]->getCurrentState().getValue();
00070 const long e2 = nbrs[21]->getCurrentState().getValue();
00071 const long ese = nbrs[22]->getCurrentState().getValue();
00072
00073 const long currentState = cell->getCurrentState().getValue();
00074
00075 // A estratégia funciona da seguinte maneira para uma célula areia:
00076 // 1) Se a célula de baixo (s) estiver livre, a areia cai verticalmente (vira vazio)
00077 // 2) Se (s) está bloqueado, e preferLeft
00078 // - tenta ir para (sw).
00079 // - se falhar, tenta (se).
00080 // 3) Analogamente, em caso de !preferLeft
00081 // - tenta ir para (se).
00082 // - se falhar, tenta (sw).
00083 // 4) Não há movimentos válidos, permanece onde está
00084 //
00085 // Do ponto de vista de uma célula vazia, ela segue a mesma lógica observando os grãos de
areia de cima
00086 // que tentariam escorregar para sua posição, e se o grão cederia ou não a preferencia.
00087 // Isto garante a conservação de matéria. Pelo menos nos casos que testei.
00088 // Um comportamento que não consegui reproduzir é o de preencher completamente um recipiente
dependendo de
00089 // seu formato.
00090 switch (currentState) {
00091 case SAND: {
00092 // 1) Queda vertical tem prioridade máxima.
00093 if (s == EMPTY) {
00094 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00095 return;
00096 }
00097
00098 // 2) Tentativa de escorregamento diagonal.
00099 bool canSW = (sw == EMPTY) && (w != SAND && w != ROCK);
00100 bool canSE = (se == EMPTY) && (e != SAND && e != ROCK);
00101
00102 if (preferLeft) {
00103 if (canSW) {
00104 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00105 return;
00106 }
00107 // Fallback para SE: só escorrega para direita (se) se não ha uma areia a direita(e2)
com
00108 // preferencia para cair em SE. Por ser preferLeft ser definido em xadrez, sei que ela
00109 // também é preferLeft e cederá passagem para ela.
00110 if (canSE && !(e2 == SAND && ese != EMPTY)) {
00111 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00112 return;
00113 }
00114 } else {
00115 if (canSE) {
00116 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00117 return;
00118 }
00119 // Fallback para SW: análogo, cedo se w2 quer meu sw.
00120 if (canSW && !(w2 == SAND && wsw != EMPTY)) {
00121 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00122 return;
00123 }
00124 }
00125
00126 // Não consigo me mover: permaneço SAND.
00127 cell->setNextState(*enumerableStateSet->getState(SAND));
00128 break;
00129 }
00130 case EMPTY: {
00131 // 1) não cai verticalmente.
00132 if (n == SAND) {

```

```

00133 cell->setNextState(*enumerableStateSet->getState(SAND));
00134 return;
00135 }
00136
00137 // 2) se há rock em cima não tem como areia escorregar pra aqui
00138 if (n == ROCK) {
00139 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00140 return;
00141 }
00142
00143 // 3) escorregamento diagonal.
00144 // (Sabemos n != SAND, então diagonais não competem com queda vertical)
00145 bool nw_fills = false;
00146 if (nw == SAND && w != EMPTY) {
00147 if (preferLeft) {
00148 // por xadrez, nw prefere seu sw(w2)
00149 const bool canSW_of_nw = (w2 == EMPTY) && (wnw == EMPTY);
00150 //
00151 const bool fallback_safe = !(ne == SAND && e != EMPTY);
00152
00153 if (!canSW_of_nw && fallback_safe) {
00154 nw_fills = true;
00155 }
00156 } else {
00157 // nw prefere seu se, direção preferida.
00158 nw_fills = true;
00159 }
00160 }
00161
00162 // simétrico
00163 bool ne_fills = false;
00164 if (ne == SAND && e != EMPTY) {
00165 if (preferLeft) {
00166 ne_fills = true;
00167 } else {
00168 const bool canSE_of_ne = (e2 == EMPTY) && (ene == EMPTY);
00169 const bool fallback_safe = !(nw == SAND && w != EMPTY);
00170 if (!canSE_of_ne && fallback_safe) {
00171 ne_fills = true;
00172 }
00173 }
00174 }
00175
00176 if (nw_fills || ne_fills) {
00177 cell->setNextState(*enumerableStateSet->getState(SAND));
00178 } else {
00179 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00180 }
00181 break;
00182 }
00183 case ROCK: {
00184 // objeto imóvel, sempre é pedra
00185 cell->setNextState(*enumerableStateSet->getState(ROCK));
00186 break;
00187 }
00188 default:
00189 cell->setNextState(cell->getCurrentState());
00190 break;
00191 }
00192 }
00193 protected:
00194 private:
00195 };

```

## 16.413 LocalRule\_ShowNeighborhood.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00004 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00005 #include "plugins/components/ModalModel/CellularAutomata/StateSet_Enumerable.h"
00006
00007 class LocalRule_ShowNeighborhood : public LocalRule {
00008 public:
00009 LocalRule_ShowNeighborhood(CellularAutomataBase* parentCellularAutomata)
00010 : LocalRule(parentCellularAutomata) { }
00011 LocalRule_ShowNeighborhood(const LocalRule_ShowNeighborhood& orig): LocalRule(orig) {}
00012 virtual ~LocalRule_ShowNeighborhood() = default;
00013 public:
00014 virtual void applyRule(Cell* cell) {
00015 auto* enumerableStateSet = dynamic_cast<StateSet_Enumerable*>(stateSet);
00016 if (cell == nullptr || enumerableStateSet == nullptr || enumerableStateSet->getStatesSize() <
00017 3)
00018 return;
00019 if (cell->getCurrentState() == *enumerableStateSet->getState(1))

```

```

00019 cell->setNextState(cell->getCurrentState());
00020 else {
00021 for (Cell* n: cell->getNeighbors()) {
00022 if (n != nullptr && n->getCurrentState()==*enumerableStateSet->getState(1)) {
00023 cell->setNextState(*enumerableStateSet->getState(2));
00024 return;
00025 }
00026 }
00027 cell->setNextState(*enumerableStateSet->getState(0));
00028 }
00029 }
00030 };

```

## 16.414 Neighborhood.h

```

00001 #pragma once
00002
00003 #include <string>
00004 #include <utility>
00005 #include <vector>
00006
00007
00008 class Cell;
00009 #include "plugins/components/ModalModel/CellularAutomata/BoundaryCondition.h"
00010 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomataBase.h"
00011
00012 class Neighborhood {
00013 public:
00014 Neighborhood(CellularAutomataBase* parentCellularAutomata, unsigned short radius=1,
00015 BoundaryCondition* boundary= nullptr, bool includeCellItself=false){
00016 this->parentCellularAutomata = parentCellularAutomata;
00017 parentCellularAutomata->setNeighborhood(this);
00018 this->radius = radius;
00019 this->boundary = boundary;
00020 this->includeCellItself = includeCellItself;
00021 }
00022 Neighborhood(const Neighborhood& orig){}
00023 virtual ~Neighborhood()=default;
00024 public:
00025 virtual std::string show(){
00026 return "-";
00027 }
00028 virtual std::vector<Cell*> getNeighbors(Cell* cell) = 0;
00029 public:
00030 void setRadius(unsigned short radius) {
00031 this->radius = radius;
00032 }
00033 unsigned short getRadius() const{
00034 return radius;
00035 }
00036 void setBoundary(BoundaryCondition* boundary) {
00037 this->boundary = boundary;
00038 boundary->setNeighborhood(this);
00039 }
00040 BoundaryCondition* getBoundary() const{
00041 return boundary;
00042 }
00043 void setIncludeCellItself(bool includeCellItself){
00044 this->includeCellItself = includeCellItself;
00045 }
00046 bool getIncludeCellItself() const{
00047 return includeCellItself;
00048 }
00049 public:
00050 protected:
00051 Cell* getNeighborCell(std::vector<int> cellPosition, std::vector<std::pair<unsigned short, int>
00052 dimensionChanges){
00053 std::vector<int> neighborPosition;
00054 neighborPosition = cellPosition;
00055 for (std::pair<unsigned short, int> change : dimensionChanges) {
00056 neighborPosition.at(change.first) += change.second;
00057 }
00058 //std::cout << "Neig pos from " << Util::showNDimPosition(cellPosition) << " is "
00059 << Util::showNDimPosition(&neighborPosition);
00060 Cell* neighCell = boundary->getNeighborCell(cellPosition, neighborPosition);
00061 //std::cout << std::endl;
00062 return neighCell;
00063 }
00064 protected:
00065 std::string name;
00066 CellularAutomataBase* parentCellularAutomata;
00067 BoundaryCondition* boundary;
00068 unsigned short radius;
00069 bool includeCellItself;
00070 private:

```

```
00069 };
```

## 16.415 Neighborhood\_Center.h

```
00001 #pragma once
00002
00003 #include <string>
00004 #include <vector>
00005
00006 #include "plugins/components/ModalModel/CellularAutomata/Neighborhood.h"
00007 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00008 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomataBase.h"
00009
00010 class Neighborhood_Center: public Neighborhood {
00011 public:
00012 Neighborhood_Center(CellularAutomataBase* parentCellularAutomata, unsigned short radius=1,
00013 BoundaryCondition* boundary= nullptr, bool includeCellItself=false)
00014 :Neighborhood(parentCellularAutomata, radius, boundary, includeCellItself) {
00015 this->name = "Center";
00016 }
00017
00018 Neighborhood_Center(const Neighborhood_Center& orig)
00019 : Neighborhood(orig) { }
00020 virtual ~Neighborhood_Center()=default;
00021 public:
00022 virtual std::vector<Cell*> getNeighbors(Cell* cell) override {
00023 std::vector<Cell*> neighbors;
00024 //std::string m = "Center::getNeighbors(CellPos=" +
00025 Util::showNDimPosition(cell->getPosition())+"")=";
00026 //unsigned short numDimensions = parentCellularAutomata->getLattice()->getNumDimensions();
00027 //std::vector<int>* cellPosition = cell->getPosition();
00028 for (int r = 1; r <= radius; r++) {
00029 int d = 0;
00030 //for (int d = 0; r < numDimensions; d++) {
00031 neighbors.emplace_back(getNeighborCell(cell->getPosition(), {{d, -r}}));
00032 neighbors.emplace_back(getNeighborCell(cell->getPosition(), {{d, r}}));
00033 }
00034 if (includeCellItself)
00035 neighbors.emplace_back(cell);
00036 }
00037 return neighbors;
00038 }
00039 protected:
00040 private:
00041 };
```

## 16.416 Neighborhood\_Moore.h

```
00001 /*
00002 * File: Neighborhood_Moore.h
00003 * Author: cancian
00004 *
00005 * Created on 6 de abril de 2023, 14:27
00006 */
00007
00008 #pragma once
00009
00010 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00011 #include "plugins/components/ModalModel/CellularAutomata/Neighborhood.h"
00012 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00013
00014 #include <vector>
00015
00016 class Neighborhood_Moore : public Neighborhood {
00017 public:
00018 Neighborhood_Moore(CellularAutomataBase* parentCellularAutomata, unsigned short radius = 1,
00019 BoundaryCondition* boundary = nullptr)
00020 :Neighborhood(parentCellularAutomata, radius, boundary) { this->name = "Moore"; }
00021 Neighborhood_Moore(const Neighborhood_Moore& orig):Neighborhood(orig) { }
00022 virtual ~Neighborhood_Moore() = default;
00023 public:
00024 virtual std::string show() override {
00025 return "todo";
00026 }
00027 virtual std::vector<Cell*> getNeighbors(Cell* cell) override {
00028 std::vector<Cell*> neighbors;
00029 unsigned short numDimensions = parentCellularAutomata->getLattice()->getNumDimensions();
00030 std::vector<int> cellPosition = cell->getPosition();
00031
00032 std::vector<int> offset(numDimensions, 0);
00033 collectNeighbors(cellPosition, offset, 0, neighbors);
00034 }
```

```

00034 if (includeCellItself)
00035 neighbors.emplace_back(cell);
00036 return neighbors;
00037 }
00038 private:
00039 void collectNeighbors(const std::vector<int>& cellPosition, std::vector<int>& offset, unsigned
short dimension, std::vector<Cell*>& neighbors) {
00040 if (dimension == offset.size()) {
00041 bool isZeroOffset = true;
00042 std::vector<std::pair<unsigned short, int>> dimensionChanges;
00043 for (unsigned short dim = 0; dim < offset.size(); ++dim) {
00044 if (offset.at(dim) != 0) {
00045 isZeroOffset = false;
00046 dimensionChanges.emplace_back(dim, offset.at(dim));
00047 }
00048 }
00049 if (!isZeroOffset) {
00050 // Offsets are visited from dimension 0 to n-1.
00051 neighbors.emplace_back(getNeighborCell(cellPosition, dimensionChanges));
00052 }
00053 return;
00054 }
00055
00056 for (int delta = -static_cast<int>(radius); delta <= static_cast<int>(radius); ++delta) {
00057 offset.at(dimension) = delta;
00058 collectNeighbors(cellPosition, offset, dimension + 1, neighbors);
00059 }
00060 }
00061 };

```

## 16.417 Neighborhood\_VonNeumann.h

```

00001 /*
00002 * File: Neighborhood_VonNeumann.h
00003 * Author: cancian
00004 *
00005 * Created on 6 de abril de 2023, 14:27
00006 */
00007
00008 #pragma once
00009
00010 #include <string>
00011 #include <vector>
00012 #include <cstdlib>
00013
00014 #include "plugins/components/ModalModel/CellularAutomata/Neighborhood.h"
00015 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00016
00017 class Neighborhood_VonNeumann : public Neighborhood {
00018 public:
00019
00020 Neighborhood_VonNeumann(CellularAutomataBase* parentCellularAutomata, unsigned short radius = 1,
BoundaryCondition* boundary = nullptr)
:Neighborhood(parentCellularAutomata, radius, boundary) {
00021 this->name = "Von Neumann";
00022 }
00023
00024 Neighborhood_VonNeumann(const Neighborhood_VonNeumann& orig)
:Neighborhood(orig) {
00025 }
00026
00027 virtual ~Neighborhood_VonNeumann() = default;
00028 public:
00029
00030 virtual std::string show() override {
00031 return "-";
00032 }
00033
00034 virtual std::vector<Cell*> getNeighbors(Cell* cell) override {
00035 std::vector<Cell*> neighbors;
00036 unsigned short numDimensions = parentCellularAutomata->getLattice()->getNumDimensions();
00037 std::vector<int> cellPosition = cell->getPosition();
00038
00039 std::vector<int> offset(numDimensions, 0);
00040 collectNeighbors(cellPosition, offset, 0, neighbors);
00041
00042 if (includeCellItself)
00043 neighbors.emplace_back(cell);
00044 return neighbors;
00045 }
00046 private:
00047 void collectNeighbors(const std::vector<int>& cellPosition, std::vector<int>& offset, unsigned
short dimension, std::vector<Cell*>& neighbors) {
00048 if (dimension == offset.size()) {
00049 bool isZeroOffset = true;
00050 unsigned int manhattanDistance = 0;

```



```

00052 std::vector<std::pair<unsigned short, int>> dimensionChanges;
00053 for (unsigned short dim = 0; dim < offset.size(); ++dim) {
00054 int delta = offset.at(dim);
00055 manhattanDistance += static_cast<unsigned int>(std::abs(delta));
00056 if (delta != 0) {
00057 isZeroOffset = false;
00058 dimensionChanges.emplace_back(dim, delta);
00059 }
00060 }
00061 if (!isZeroOffset && manhattanDistance <= radius) {
00062 // Offsets are visited from dimension 0 to n-1.
00063 neighbors.emplace_back(getNeighborCell(cellPosition, dimensionChanges));
00064 }
00065 return;
00066 }
00067
00068 for (int delta = -static_cast<int>(radius); delta <= static_cast<int>(radius); ++delta) {
00069 offset.at(dimension) = delta;
00070 collectNeighbors(cellPosition, offset, dimension + 1, neighbors);
00071 }
00072 }
00073 };

```

## 16.418 State.h

```

00001 #pragma once
00002
00003 #include <array>
00004 #include <memory>
00005 #include <sstream>
00006 #include <string>
00007 #include <typeinfo>
00008 #include <utility>
00009 #include <vector>
00010
00011 /*
00012 template<class T>
00013 class State {
00014 public:
00015 State<T>() {}
00016 State<T>(T value) {_value=value;}
00017 State<T>(const State& orig){}
00018 virtual ~State<T>() = default;
00019 public:
00020 T getValue(){return _value;}
00021 void setValue(T value) {_value=value;}
00022 protected:
00023 T _value;
00024 private:
00025 };
00026 */
00027
00028 class State {
00029 public:
00030 State() : _value(0) {}
00031 State(long value) : _value(value) {}
00032 State(const State& orig) : _value(orig._value) {}
00033 virtual ~State() = default;
00034 public:
00035 virtual std::unique_ptr<State> clone() const { return std::make_unique<State>(*this); }
00036 virtual bool equals(const State& other) const { return typeName() == other.typeName() &&
00037 getValue() == other.getValue(); }
00037 virtual std::string toString() const { return std::to_string(_value); }
00038 virtual std::string show() const { return toString(); }
00039 virtual std::string typeName() const { return "State"; }
00040 virtual long getValue() const { return _value; }
00041 virtual void setValue(long value) { _value=value; }
00042
00043 template <typename T>
00044 const T* as() const noexcept { return dynamic_cast<const T*>(this); }
00045
00046 template <typename T>
00047 T* as() noexcept { return dynamic_cast<T*>(this); }
00048 protected:
00049 long _value;
00050 private:
00051 };
00052
00053 inline bool operator==(const State& left, const State& right) {
00054 return left.equals(right);
00055 }
00056
00057 inline bool operator!=(const State& left, const State& right) {
00058 return !(left == right);
00059 }

```

```

00060
00061 class RealState : public State {
00062 public:
00063 explicit RealState(double value = 0.0)
00064 : State(static_cast<long>(value)), _realValue(value) {
00065 }
00066
00067 std::unique_ptr<State> clone() const override { return std::make_unique<RealState>(*this); }
00068 bool equals(const State& other) const override {
00069 const auto* real = other.as<RealState>();
00070 return real != nullptr && _realValue == real->_realValue;
00071 }
00072 std::string toString() const override {
00073 std::ostringstream stream;
00074 stream << _realValue;
00075 return stream.str();
00076 }
00077 std::string typeName() const override { return "RealState"; }
00078 long getValue() const override { return static_cast<long>(_realValue); }
00079 void setValue(long value) override { _realValue = static_cast<double>(value); _value = value; }
00080 double realValue() const { return _realValue; }
00081 void setRealValue(double value) { _realValue = value; _value = static_cast<long>(value); }
00082 private:
00083 double _realValue = 0.0;
00084 };
00085
00086 class CompositeState : public State {
00087 public:
00088 using Field = std::pair<std::string, std::unique_ptr<State>>;
00089
00090 CompositeState() = default;
00091 CompositeState(const CompositeState& orig) : State(orig) {
00092 for (const auto& field : orig._fields) {
00093 _fields.emplace_back(field.first, field.second != nullptr ? field.second->clone() :
00094 nullptr);
00095 }
00096 }
00097 CompositeState& operator=(const CompositeState& orig) {
00098 if (this == &orig) {
00099 return *this;
00100 }
00101 _value = orig._value;
00102 _fields.clear();
00103 for (const auto& field : orig._fields) {
00104 _fields.emplace_back(field.first, field.second != nullptr ? field.second->clone() :
00105 nullptr);
00106 }
00107 return *this;
00108 }
00109 std::unique_ptr<State> clone() const override { return std::make_unique<CompositeState>(*this); }
00110 bool equals(const State& other) const override {
00111 const auto* composite = other.as<CompositeState>();
00112 if (composite == nullptr || _fields.size() != composite->_fields.size()) {
00113 return false;
00114 }
00115 for (size_t index = 0; index < _fields.size(); ++index) {
00116 const auto& left = _fields.at(index);
00117 const auto& right = composite->_fields.at(index);
00118 if (left.first != right.first || left.second == nullptr || right.second == nullptr ||
00119 *left.second != *right.second) {
00120 return false;
00121 }
00122 }
00123 return true;
00124 }
00125 std::string toString() const override {
00126 std::string text = "{";
00127 for (size_t index = 0; index < _fields.size(); ++index) {
00128 if (index > 0) {
00129 text += ",";
00130 }
00131 text += _fields.at(index).first + ":" + (_fields.at(index).second != nullptr ?
00132 _fields.at(index).second->toString() : "null");
00133 }
00134 return text + "}";
00135 }
00136 std::string typeName() const override { return "CompositeState"; }
00137 bool addField(std::string name, const State& state) {
00138 _fields.emplace_back(std::move(name), state.clone());
00139 return true;
00140 }
00141 const std::vector<Field>& fields() const { return _fields; }
00142 private:
00143 std::vector<Field> _fields;
00144 };

```

```

00143 class HppLatticeGasState : public State {
00144 public:
00145 enum Direction : size_t { North = 0, East = 1, South = 2, West = 3 };
00146
00147 HppLatticeGasState() = default;
00148 explicit HppLatticeGasState(std::array<bool, 4> occupations)
00149 : State(maskFromOccupations(occupations), _occupations(occupations) {
00150 }
00151
00152 std::unique_ptr<State> clone() const override { return
std::make_unique<HppLatticeGasState>(*this); }
00153 bool equals(const State& other) const override {
00154 const auto* hpp = other.as<HppLatticeGasState>();
00155 return hpp != nullptr && _occupations == hpp->_occupations;
00156 }
00157 std::string toString() const override {
00158 return std::string("{N:") + (_occupations[North] ? "1" : "0") +
00159 ",E:" + (_occupations[East] ? "1" : "0") +
00160 ",S:" + (_occupations[South] ? "1" : "0") +
00161 ",W:" + (_occupations[West] ? "1" : "0") + "}";
00162 }
00163 std::string typeName() const override { return "HppLatticeGasState"; }
00164 long getValue() const override { return maskFromOccupations(_occupations); }
00165 void setValue(long value) override {
00166 _occupations = {{
00167 (value & (1L << North)) != 0,
00168 (value & (1L << East)) != 0,
00169 (value & (1L << South)) != 0,
00170 (value & (1L << West)) != 0,
00171 }};
00172 _value = getValue();
00173 }
00174 bool has(Direction direction) const { return _occupations[direction]; }
00175 void set(Direction direction, bool occupied) { _occupations[direction] = occupied; _value =
getValue(); }
00176 const std::array<bool, 4>& occupations() const { return _occupations; }
00177 static long maskFromOccupations(const std::array<bool, 4>& occupations) {
00178 long mask = 0;
00179 for (size_t index = 0; index < occupations.size(); ++index) {
00180 if (occupations[index]) {
00181 mask |= (1L << index);
00182 }
00183 }
00184 return mask;
00185 }
00186 private:
00187 std::array<bool, 4> _occupations{{false, false, false, false}};
00188 };

```

## 16.419 State\_Numerical.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/State.h"
00004
00005 /*
00006 class State_Numerical: public State {
00007 public:
00008 State_Numerical(long value)
00009 : State(ColorCA(0,0,0), ' ', value, ""){
00010 this->longValue = value;
00011 }
00012 State_Numerical(const State_Numerical& orig)
00013 :State(orig) {
00014 }
00015 }
00016 virtual ~State_Numerical() = default;
00017 public:
00018 virtual ColorCA getColor() const override {
00019 int llongValue = 0;
00020 if (longValue)
00021 llongValue = 1<<9;
00022 uint8_t south = longValue&((1<<3)|(1<<5)|(1<<7))?0:255;
00023 uint8_t r = longValue&((1<<0)|(1<<4)|(1<<5))?0:south; // west : south
00024 uint8_t g = longValue&((1<<1)|(1<<6)|(1<<7))?0:south; // east : south
00025 uint8_t b = longValue&((1<<2)|(1<<4)|(1<<6))?0:south; // north : south
00026 //std::cout << longValue << " : " << (int)r << ", "<< (int)g << ", "<< (int)b << std::endl;
00027 return ColorCA(r,g,b);
00028 }
00029
00030 protected:
00031 private:
00032 };
00033 */

```

## 16.420 StateSet.h

```

00001 #pragma once
00002
00003 #include <limits>
00004 #include <memory>
00005 #include <optional>
00006 #include <sstream>
00007 #include <string>
00008 #include <vector>
00009 #include "plugins/components/ModalModel/CellularAutomata/State.h"
00010
00011 class CellularAutomataBase;
00012
00013 class StateSet {
00014 public:
00015 StateSet(CellularAutomataBase* parentCellularAutomata);
00016 StateSet(const StateSet& orig);
00017 virtual ~StateSet()=default;
00018 public:
00019 virtual bool contains(const State& state) const;
00020 virtual std::unique_ptr<State> createDefaultState() const;
00021 virtual std::unique_ptr<State> parseState(const std::string& text) const;
00022 virtual bool isFinite() const;
00023 virtual bool isDiscrete() const;
00024 virtual bool isContinuous() const;
00025 virtual bool isComposite() const;
00026 virtual std::vector<std::unique_ptr<State> enumerateStates() const;
00027 virtual std::string name() const;
00028 protected:
00029 CellularAutomataBase* parentCellularAutomata;
00030 private:
00031 };
00032
00033 class IntegerStateSet : public StateSet {
00034 public:
00035 IntegerStateSet(CellularAutomataBase* parentCellularAutomata,
00036 std::optional<long> minimum = std::nullopt,
00037 std::optional<long> maximum = std::nullopt)
00038 : StateSet(parentCellularAutomata), _minimum(minimum), _maximum(maximum) {
00039 }
00040
00041 bool contains(const State& state) const override {
00042 if (state.as<RealState>() != nullptr || state.as<CompositeState>() != nullptr ||
00043 state.as<HpplatticeGasState>() != nullptr) {
00044 return false;
00045 }
00046 const long value = state.getValue();
00047 return (!_minimum.has_value() || value >= *_minimum) && (!_maximum.has_value() || value <=
00048 *_maximum);
00049 }
00050 std::unique_ptr<State> createDefaultState() const override {
00051 return std::make_unique<State>(_minimum.value_or(0));
00052 }
00053 std::unique_ptr<State> parseState(const std::string& text) const override {
00054 try {
00055 auto state = std::make_unique<State>(std::stol(text));
00056 return contains(*state) ? std::move(state) : nullptr;
00057 } catch (...) {
00058 return nullptr;
00059 }
00060 }
00061 bool isFinite() const override { return _minimum.has_value() && _maximum.has_value(); }
00062 bool isDiscrete() const override { return true; }
00063 std::vector<std::unique_ptr<State> enumerateStates() const override {
00064 std::vector<std::unique_ptr<State> states;
00065 if (!isFinite()) {
00066 return states;
00067 }
00068 for (long value = *_minimum; value <= *_maximum; ++value) {
00069 states.emplace_back(std::make_unique<State>(value));
00070 if (value == std::numeric_limits<long>::max()) {
00071 break;
00072 }
00073 }
00074 return states;
00075 }
00076 std::string name() const override { return "IntegerStateSet"; }
00077 private:
00078 std::optional<long> _minimum;
00079 std::optional<long> _maximum;
00080 };
00081
00082 class NaturalStateSet : public IntegerStateSet {
00083 public:
00084 explicit NaturalStateSet(CellularAutomataBase* parentCellularAutomata, std::optional<long> maximum
00085 = std::nullopt)

```

```

00083 : IntegerStateSet(parentCellularAutomata, 0, maximum) {
00084 }
00085 std::string name() const override { return "NaturalStateSet"; }
00086 };
00087
00088 class RealIntervalStateSet : public StateSet {
00089 public:
00090 RealIntervalStateSet(CellularAutomataBase* parentCellularAutomata,
00091 std::optional<double> minimum = std::nullopt,
00092 std::optional<double> maximum = std::nullopt)
00093 : StateSet(parentCellularAutomata), _minimum(minimum), _maximum(maximum) {
00094 }
00095
00096 bool contains(const State& state) const override {
00097 const auto* real = state.as<RealState>();
00098 const double value = real != nullptr ? real->realValue() :
00099 static_cast<double>(state.getValue());
00099 return (!_minimum.has_value() || value >= *_minimum) && (!_maximum.has_value() || value <=
*_maximum);
00100 }
00101 std::unique_ptr<State> createDefaultState() const override {
00102 return std::make_unique<RealState>(_minimum.value_or(0.0));
00103 }
00104 std::unique_ptr<State> parseState(const std::string& text) const override {
00105 try {
00106 auto state = std::make_unique<RealState>(std::stod(text));
00107 return contains(*state) ? std::move(state) : nullptr;
00108 } catch (...) {
00109 return nullptr;
00110 }
00111 }
00112 bool isContinuous() const override { return true; }
00113 std::string name() const override { return "RealIntervalStateSet"; }
00114 private:
00115 std::optional<double> _minimum;
00116 std::optional<double> _maximum;
00117 };
00118
00119 class CompositeStateSet : public StateSet {
00120 public:
00121 explicit CompositeStateSet(CellularAutomataBase* parentCellularAutomata)
00122 : StateSet(parentCellularAutomata) {
00123 }
00124
00125 bool contains(const State& state) const override { return state.as<CompositeState>() != nullptr; }
00126 std::unique_ptr<State> createDefaultState() const override { return
std::make_unique<CompositeState>(); }
00127 bool isComposite() const override { return true; }
00128 std::string name() const override { return "CompositeStateSet"; }
00129 };
00130
00131 class HppLatticeGasStateSet : public StateSet {
00132 public:
00133 explicit HppLatticeGasStateSet(CellularAutomataBase* parentCellularAutomata)
00134 : StateSet(parentCellularAutomata) {
00135 }
00136
00137 bool contains(const State& state) const override { return state.as<HppLatticeGasState>() !=
nullptr; }
00138 std::unique_ptr<State> createDefaultState() const override { return
std::make_unique<HppLatticeGasState>(); }
00139 std::unique_ptr<State> parseState(const std::string& text) const override {
00140 try {
00141 auto state = std::make_unique<HppLatticeGasState>();
00142 state->setValue(std::stol(text));
00143 return state;
00144 } catch (...) {
00145 return nullptr;
00146 }
00147 }
00148 bool isFinite() const override { return true; }
00149 bool isDiscrete() const override { return true; }
00150 bool isComposite() const override { return true; }
00151 std::vector<std::unique_ptr<State>> enumerateStates() const override {
00152 std::vector<std::unique_ptr<State>> states;
00153 for (long mask = 0; mask < 16; ++mask) {
00154 auto state = std::make_unique<HppLatticeGasState>();
00155 state->setValue(mask);
00156 states.emplace_back(std::move(state));
00157 }
00158 return states;
00159 }
00160 std::string name() const override { return "HppLatticeGasStateSet"; }
00161 };

```

## 16.421 StateSet\_Enumerable.h

```

00001 #pragma once
00002
00003 #include <memory>
00004 #include <string>
00005 #include <vector>
00006 #include "plugins/components/ModalModel/CellularAutomata/State.h"
00007 #include "plugins/components/ModalModel/CellularAutomata/StateSet.h"
00008
00009 class CellularAutomataBase;
00010
00011 class StateSet_Enumerable: public StateSet{
00012 public:
00013 StateSet_Enumerable(CellularAutomataBase* parentCellularAutomata, std::vector<State*> states =
00014 {});
00015 StateSet_Enumerable(const StateSet_Enumerable& orig);
00016 virtual ~StateSet_Enumerable()=default;
00017 public:
00018 virtual std::string show();
00019 bool contains(const State& state) const override;
00020 std::unique_ptr<State> createDefaultState() const override;
00021 std::unique_ptr<State> parseState(const std::string& text) const override;
00022 bool isFinite() const override;
00023 bool isDiscrete() const override;
00024 std::vector<std::unique_ptr<State>> enumerateStates() const override;
00025 std::string name() const override;
00026 unsigned int size();
00027 void addState(State* state);
00028 void addState(const State& state);
00029 unsigned int getStatesSize();
00030 State* getState(unsigned int rank);
00031 const State* getState(unsigned int rank) const;
00032 State* getState(std::string name);
00033 void setStates(std::vector<State*> states);
00034 std::vector<State*> getStates() const;
00035 protected:
00036 CellularAutomataBase* parentCellularAutomata;
00037 std::vector<std::unique_ptr<State>> ownedStates;
00038 std::vector<State*> states;
00039 private:
00040 };

```

## 16.422 CellularAutomataComp.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Dummy.h
00009 * Author: rafael.luiiz.cancian
00010 *
00011 * Created on 22 de Maio de 2019, 18:41
00012 */
00013
00014 #pragma once
00015
00016 #include "../kernel/simulator/model/ModelComponent.h"
00017 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomataBase.h"
00018 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00019 #include "plugins/components/ModalModel/CellularAutomata/Neighborhood.h"
00020 #include "plugins/components/ModalModel/CellularAutomata/StateSet.h"
00021
00022 class BoundaryCondition;
00023
00024 class CellularAutomataComp : public ModelComponent {
00025 public:
00026 enum class CellularAutomataType : int {
00027 CLASSIC = 1, TIMED_1D = 2, ASYNCHRONOUS = 3, NONUNIFORMRULE = 4, NONUNIFORMNEIGHBOOR = 5,
00028 USERDEFINED = 6
00029 };
00030
00031 enum class LatticeType : int {
00032 RETICULAR = 1, TRIANGULAR = 2, HEXAGONAL = 3, NETWORK = 4, USERDEFINED = 5
00033 };
00034
00035 enum class NeighborhoodType : int {
00036 CENTERED = 1, BACKWARD = 2, FORWARD = 3, VONNEUMANN = 4, MOORE = 5, USERDEFINED = 6
00037 };
00038
00039 enum class BoundaryType : int {

```

```

00042 FIXED = 1, CLOSED = 2, REFLEXIVE = 3, ADIABATIC = 4, USERDEFINED = 5
00043 };
00044
00045 enum class StateSetType : int {
00046 ENUMERATED = 1, INTEGERBASED = 2, BITBASED = 3, DOUBLEBASED = 4, USERDEFINED = 5
00047 };
00048
00049 enum class LocalRuleType : int {
00050 ELEMENTAR_CA = 1, GAME_OF_LIFE = 2, BIASED_COMPETITION = 3, HPP = 4, SAND_ROCK = 5,
00051 USERDEFINED = 99
00052 };
00053
00054 enum class AutomataType : int {
00055 Temporary = 0, Permanent = 1
00056 };
00057 public:
00058 CellularAutomataComp(Model* model, std::string name = "");
00059 virtual ~CellularAutomataComp() = default;
00060
00061 public:
00062 CellularAutomataComp::CellularAutomataType getCellularAutomataType() const;
00063 void setCellularAutomataType(CellularAutomataComp::CellularAutomataType newCellularAutomataType);
00064
00065 CellularAutomataComp::LatticeType getLatticeType() const;
00066 void setLatticeType(CellularAutomataComp::LatticeType newLatticeStructure);
00067
00068 CellularAutomataComp::NeighborhoodType getNeighborhoodType() const;
00069 void setNeighborhoodType(CellularAutomataComp::NeighborhoodType newNeighborhood);
00070
00071 CellularAutomataComp::BoundaryType getBoundaryType() const;
00072 void setBoundaryType(CellularAutomataComp::BoundaryType newBoundary);
00073
00074 CellularAutomataComp::StateSetType getStateSetType() const;
00075 void setStateSetType(CellularAutomataComp::StateSetType newStateSetType);
00076
00077 //CellularAutomataBase *getcellularAutomata() const;
00078 Lattice *getlattice() const;
00079 Neighborhood *getNeighborhood() const;
00080 //BoundaryCondition *getBoundary() const;
00081 StateSet *getStateSet() const;
00082
00083 public:
00084 virtual std::string show() override;
00085
00086 public:
00087 static PluginInformation* GetPluginInformation();
00088 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00089 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00090
00091 CellularAutomataComp::LocalRuleType getlocalRuleType() const;
00092 void setLocalRuleType(CellularAutomataComp::LocalRuleType newLocalRuleType);
00093
00094 LocalRule *getlocalRule() const;
00095
00096 protected:
00097 virtual bool _loadInstance(PersistenceRecord *fields) override;
00098 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00099 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00100
00101
00102 protected:
00103 virtual bool _check(std::string* errorMessage);
00104 // virtual ParserChangesInformation* _getParserChangesInformation();
00105 virtual void _initBetweenReplications() override;
00106 // virtual void _createInternalAndAttachedData(); /*< A ModelDataDefinition or ModelComponent that
00107 includes (internal) ou refers to (attach) other ModelDataDefinition must register them inside this
00108 method. */
00109 // virtual void _addSimulationControl(SimulationControl* property);
00110
00111
00112
00113
00114
00115 protected:
00116 // virtual void _createInternalStatisticReporters() override;
00117 // virtual void _createNonEditableDataDefinitions() override;
00118 // virtual void _createEditableDataDefinitions() override;
00119 // virtual void _createAttachedAttributes() override;
00120
00121 private:
00122 // ...
00123
00124 private:
00125
00126 // Default values for the attributes. Used on initing, loading and saving
00127 const struct DEFAULT_VALUES {
00128 const CellularAutomataType cellularAutomataType = CellularAutomataType::CLASSIC;
00129 const LatticeType latticeType = LatticeType::RETICULAR;
00130 const NeighborhoodType neighborhoodType = NeighborhoodType::VONNEUMANN;

```

```

00131 const BoundaryType boundaryType = BoundaryType::FIXED;
00132 const StateSetType stateSetType = StateSetType::ENUMERATED;
00133 const LocalRuleType localRuleType = LocalRuleType::GAME_OF_LIFE;
00134 } DEFAULT;
00135 CellularAutomataComp::CellularAutomataType _cellularAutomataType = DEFAULT.cellularAutomataType;
00136 CellularAutomataComp::LatticeType _latticeType = DEFAULT.latticeType;
00137 CellularAutomataComp::NeighborhoodType _neighborhoodType = DEFAULT.neighborhoodType;
00138 CellularAutomataComp::BoundaryType _boundaryType = DEFAULT.boundaryType;
00139 CellularAutomataComp::StateSetType _stateSetType = DEFAULT.stateSetType;
00140 CellularAutomataComp::LocalRuleType _localRuleType = DEFAULT.localRuleType;
00141
00142 private:
00143 CellularAutomataBase* _cellularAutomata = nullptr;
00144 Lattice* _lattice = nullptr;
00145 Neighborhood* _neighborhood = nullptr;
00146 BoundaryCondition* _boundary = nullptr;
00147 StateSet* _stateSet = nullptr;
00148 LocalRule* _localRule = nullptr;
00149
00150 private:
00151
00152 private:
00153 // ...
00154 };

```

## 16.423 DefaultNode.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: DefaultNode.h
00009 * Author: Prof. Rafael Luiz Cancian, Dr. Eng.
00010 *
00011 * Created on 01 de Julho de 2025, 14:26
00012 */
00013
00014 #pragma once
00015
00016 #include "../kernel/simulator/model/ModelComponent.h"
00017 #include <functional>
00018 #include <string>
00019 // #include "plugins/data/Template/DummyElement.h"
00020
00021 class DefaultNode;
00022 class Model;
00023 class Entity;
00024
00025 class DefaultNodeTransition {
00026 public:
00027 enum class TransitionKind : int { DETERMINISTIC = 0, PROBABILISTIC = 1 };
00028
00029 public:
00030 DefaultNodeTransition(DefaultNode* source, DefaultNode* destination, std::string name = "");
00031 virtual ~DefaultNodeTransition() = default;
00032
00033 public:
00034 void setSource(DefaultNode* source);
00035 DefaultNode* getSource() const;
00036 void setDestination(DefaultNode* destination);
00037 DefaultNode* getDestination() const;
00038 void setName(std::string name);
00039 std::string getName() const;
00040 void setGuardExpression(std::string guardExpression);
00041 std::string getGuardExpression() const;
00042 void setOutputExpression(std::string outputExpression);
00043 std::string getOutputExpression() const;
00044 void setInputEvent(std::string inputEvent);
00045 std::string getInputEvent() const;
00046 void setPriority(unsigned int priority);
00047 unsigned int getPriority() const;
00048 void setProbability(double probability);
00049 double getProbability() const;
00050 void setTransitionKind(TransitionKind transitionKind);
00051 TransitionKind getTransitionKind() const;
00052
00053 public:
00054 virtual bool canFire(Model* model, Entity* entity) const;
00055 virtual void execute(Model* model, Entity* entity) const;
00056
00057 protected:

```



```

00059
00060 private:
00061 DefaultNode* _source = nullptr;
00062 DefaultNode* _destination = nullptr;
00063 std::string _name = "";
00064 std::string _guardExpression = "";
00065 std::string _outputExpression = "";
00066 std::string _inputEvent = "";
00067 unsigned int _priority = 0;
00068 double _probability = 1.0;
00069 TransitionKind _transitionKind = TransitionKind::DETERMINISTIC;
00070 };
00071
00072 class DefaultNode : public ModelComponent {
00073 public:
00074 DefaultNode(Model* model, std::string componentTypeName, std::string name);
00075 DefaultNode(Model* model, std::string name = "");
00076 virtual ~DefaultNode() = default;
00077
00078 public:
00079 void addTransition(DefaultNodeTransition* transition);
00080 void removeTransition(DefaultNodeTransition* transition);
00081 List<DefaultNodeTransition*>* getTransitions() const;
00082 void setInitialNode(bool initialNode);
00083 bool isInitialNode() const;
00084 void setFinalNode(bool finalNode);
00085 bool isFinalNode() const;
00086
00087 public:
00088 virtual std::string show() override;
00089
00090 public:
00091 static PluginInformation* GetPluginInformation();
00092 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00093 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00094
00095 protected:
00096 virtual bool _loadInstance(PersistenceRecord *fields) override;
00097 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00098 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00099
00100 protected:
00101 // virtual bool _check(std::string& errorMessage);
00102 // virtual ParserChangesInformation* _getParserChangesInformation();
00103 // virtual void _initBetweenReplications();
00104 // virtual void _createInternalStatisticReporters() override;
00105 // virtual void _createNonEditableDataDefinitions() override;
00106 // virtual void _createEditableDataDefinitions() override;
00107 // virtual void _createAttachedAttributes() override;
00108 // virtual void _createInternalAndAttachedData(); /*< A ModelDataDefinition or ModelComponent that
00109 includes (internal) ou refers to (attach) other ModelDataDefinition must register them inside this
00110 method. */
00111 // virtual void _addSimulationControl(SimulationControl* property);
00112
00113 private:
00114 // ...
00115
00116 private:
00117 const struct DEFAULT_VALUES {
00118 const bool initialNode = false;
00119 const bool finalNode = false;
00120 } DEFAULT;
00121 bool _initialNode = DEFAULT.initialNode;
00122 bool _finalNode = DEFAULT.finalNode;
00123 List<DefaultNodeTransition*>* _transitions = new List<DefaultNodeTransition*>();
00124
00125 private:
00126 //DummyElement* _internalDataDefinition = nullptr;
00127
00128 private:
00129 // ...
00130 };

```

## 16.424 DefaultTransitionExtensions.h

```

00001 #pragma once
00002
00003 #include "DefaultNode.h"
00004 #include "plugins/components/ModalModel/PetriPlace.h"
00005 #include <map>
00006
00007 class EFSMTransition : public DefaultNodeTransition {
00008 public:

```

```

00009 EFSMTransition(DefaultNode* source, DefaultNode* destination, std::string name = "");
00010 virtual ~EFSMTransition() = default;
00011
00012 public:
00013 void setTriggerEvent(std::string triggerEvent);
00014 std::string getTriggerEvent() const;
00015 void setProbabilityExpression(std::string probabilityExpression);
00016 std::string getProbabilityExpression() const;
00017
00018 public:
00019 virtual bool canFire(Model* model, Entity* entity) const override;
00020 virtual void execute(Model* model, Entity* entity) const override;
00021
00022 private:
00023 std::string _triggerEvent = "";
00024 std::string _probabilityExpression = "";
00025 };
00026
00027 class PetriTransition : public DefaultNodeTransition {
00028 public:
00029 PetriTransition(DefaultNode* source, DefaultNode* destination, std::string name = "");
00030 virtual ~PetriTransition() = default;
00031
00032 public:
00033 void setInputArcWeight(std::string color, unsigned int weight);
00034 void setOutputArcWeight(std::string color, unsigned int weight);
00035
00036 public:
00037 virtual bool canFire(Model* model, Entity* entity) const override;
00038 virtual void execute(Model* model, Entity* entity) const override;
00039
00040 private:
00041 std::map<std::string, unsigned int> _inputArcWeights;
00042 std::map<std::string, unsigned int> _outputArcWeights;
00043 };

```

## 16.425 FSMState.h

```

00001 #pragma once
00002
00003 #include "DefaultNode.h"
00004
00005 class FSMState : public DefaultNode {
00006 public:
00007 FSMState(Model* model, std::string name = "");
00008 virtual ~FSMState() = default;
00009
00010 public:
00011 void setEntryActionExpression(std::string expression);
00012 std::string getEntryActionExpression() const;
00013 void setExitActionExpression(std::string expression);
00014 std::string getExitActionExpression() const;
00015
00016 public: // static
00017 static PluginInformation* GetPluginInformation();
00018 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00019 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00020
00021 protected:
00022 virtual bool _loadInstance(PersistenceRecord *fields) override;
00023 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00024
00025 private:
00026 const struct DEFAULT_VALUES {
00027 const std::string entryActionExpression = "";
00028 const std::string exitActionExpression = "";
00029 } DEFAULT;
00030 std::string _entryActionExpression = DEFAULT.entryActionExpression;
00031 std::string _exitActionExpression = DEFAULT.exitActionExpression;
00032 };

```

## 16.426 ModalModelDefault.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: ModalModelDefault.h
00009 * Author: Prof. Rafael Luiz Cancian, Dr. Eng.

```

```

00010 *
00011 * Created on 01 de Julho de 2025, 14:26
00012 */
00013
00014 #pragma once
00015
00016 #include "../kernel/simulator/model/ModelComponent.h"
00017 #include "DefaultNode.h"
00021 class ModalModelDefault : public ModelComponent {
00022 public:
00023 ModalModelDefault(Model* model, std::string name = "");
00024 virtual ~ ModalModelDefault() = default;
00025
00026 public: // getters and setters
00027 virtual void setMaxTransitionsPerDispatch(unsigned int maxTransitionsPerDispatch);
00028 virtual unsigned int getMaxTransitionsPerDispatch() const;
00029 virtual List<DefaultNode*>* getNodes() const;
00030 virtual List<DefaultNodeTransition*>* getTransitions() const;
00031 DefaultNode* getEntryNode();
00032 void setEntryNode(DefaultNode* const entry_node);
00033 DefaultNode* getCurrentNode();
00034 std::string getTimeDelayExpressionPerDispatch();
00035 void setTimeDelayExpressionPerDispatch(const std::string time_delay_expression_per_dispatch);
00036 Util::TimeUnit getTimeDelayPerDispatchTimeUnit();
00037 void setTimeDelayPerDispatchTimeUnit(const Util::TimeUnit time_delay_per_dispatch_time_unit);
00038
00039 public:
00040 virtual void addNode(DefaultNode* node);
00041 virtual void removeNode(DefaultNode* node);
00042 virtual void addTransition(DefaultNodeTransition* transition);
00043 virtual void removeTransition(DefaultNodeTransition* transition);
00044 virtual void addOutputExpressionReference(ModelDataDefinition* expressionReference);
00045 virtual void removeOutputExpressionReference(DefaultNodeTransition* expressionReference);
00046
00047 public:
00048 virtual std::string show() override;
00049
00050 public:
00051 static PluginInformation* GetPluginInformation();
00052 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00053 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00054
00055 protected:
00056 virtual bool _loadInstance(PersistenceRecord* fields) override;
00057 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00058 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00059
00060 protected:
00062 virtual bool _check(std::string& errorMessage) override;
00064 // virtual ParserChangesInformation* _getParserChangesInformation();
00066 virtual void _initBetweenReplications() override;
00068 // virtual void _createInternalAndAttachedData() override;
00070 // virtual void _addSimulationControl(SimulationControl* property);
00071
00072 // sections bellow are *protected* since ModalModelDefault is intended to be extended
00073 protected:
00074
00076 const struct DEFAULT_VALUES {
00077 const std::string timeDelayExpressionPerDispatch = "0";
00078 const Util::TimeUnit timeDelayPerDispatchTimeUnit = Util::TimeUnit::second;
00079 const unsigned int maxTransitionsPerDispatch = 1;
00080 } DEFAULT;
00081 //std::string _entryNodeName = DEFAULT.entryNodeName;
00082 unsigned int _maxTransitionsPerDispatch = DEFAULT.maxTransitionsPerDispatch;
00083 std::string _timeDelayExpressionPerDispatch = DEFAULT.timeDelayExpressionPerDispatch;
00084 Util::TimeUnit _timeDelayPerDispatchTimeUnit = DEFAULT.timeDelayPerDispatchTimeUnit;
00085
00086 protected:
00087 DefaultNode* _currentNode = nullptr;
00088 DefaultNode* _entryNode = nullptr;
00089
00090 protected:
00091 List<DefaultNode*>* _nodes = new List<DefaultNode*>();
00092 List<DefaultNodeTransition*>* _transitions = new List<DefaultNodeTransition*>();
00093 //List<ModelDataDefinition*>* _expressionDataReferences = new List<ModelDataDefinition*>();
00094
00095 protected:
00096 // virtual void _createInternalStatisticReporters() override;
00097 // virtual void _createNonEditableDataDefinitions() override;
00098 // virtual void _createEditableDataDefinitions() override;
00099 virtual void _createAttachedAttributes() override;
00100
00101 private:
00102 // ...
00103 private:
00104 // ...
00105

```

```

00106 private:
00107 // ...
00108
00109
00110 };

```

## 16.427 ModalModelFSM.h

```

00001 #pragma once
00002
00003 #include "FSMState.h"
00004 #include "plugins/components/ModalModel/ModalModelDefault.h"
00005
00006 class ModalModelFSM : public ModalModelDefault {
00007 public:
00008 ModalModelFSM(Model* model, std::string name = "");
00009 virtual ~ModalModelFSM() = default;
00010 public:
00011
00012 public: // static
00013 static PluginInformation* GetPluginInformation();
00014 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00015 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00016
00017 protected:
00018 virtual bool _check(std::string& errorMessage) override;
00019 // virtual ParserChangesInformation* _getParserChangesInformation();
00020 virtual void _initBetweenReplications();
00021 // virtual void _addSimulationControl(SimulationControl* property);
00022
00023 private:
00024 FSMState* _fsmInitialState;
00025
00026 };
00027
00028
00029
00030 };

```

## 16.428 ModalModelPetriNet.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/ModalModelDefault.h"
00004
00005 class ModalModelPetriNet : public ModalModelDefault {
00006 public:
00007 ModalModelPetriNet(Model* model, std::string name = "");
00008 virtual ~ModalModelPetriNet() = default;
00009
00010 public: // static
00011 static PluginInformation* GetPluginInformation();
00012 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00013 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00014
00015 protected:
00016 virtual bool _check(std::string& errorMessage) override;
00017 };

```

## 16.429 PetriPlace.h

```

00001 #pragma once
00002
00003 #include "DefaultNode.h"
00004 #include <map>
00005
00006 class PetriPlace : public DefaultNode {
00007 public:
00008 PetriPlace(Model* model, std::string name = "");
00009 virtual ~PetriPlace() = default;
00010
00011 public:
00012 unsigned int getTokens(std::string color = "default") const;
00013 void addTokens(unsigned int quantity, std::string color = "default");
00014 bool removeTokens(unsigned int quantity, std::string color = "default");
00015 std::map<std::string, unsigned int>* getAllTokens();
00016
00017 public: // static
00018 static PluginInformation* GetPluginInformation();
00019 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00020 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00021
00022 protected:

```

```

00023 virtual bool _loadInstance(PersistenceRecord *fields) override;
00024 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00025
00026 private:
00027 std::map<std::string, unsigned int>* _tokensByColor = new std::map<std::string, unsigned int>();
00028 };

```

## 16.430 Submodel.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Submodel.h
00009 * Author: rlcancian
00010 *
00011 * Created on 11 de Setembro de 2019, 13:04
00012 */
00013
00014 #ifndef SUBMODEL_H
00015 #define SUBMODEL_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00022 class Submodel : public ModelComponent {
00023 public: // constructors
00024 Submodel(Model* model, std::string name = "");
00025 virtual ~Submodel() = default;
00026 public: // virtual
00027 virtual std::string show() override;
00028 public: // static
00029 static PluginInformation* GetPluginInformation();
00030 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00031 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00032 protected: // virtual
00033 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00034 virtual bool _loadInstance(PersistenceRecord *fields) override;
00035 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00036 protected: // virtual
00037 //virtual void _initBetweenReplications();
00038 // virtual void _createInternalStatisticReporters() override;
00039 // virtual void _createNonEditableDataDefinitions() override;
00040 // virtual void _createEditableDataDefinitions() override;
00041 // virtual void _createAttachedAttributes() override;
00042 virtual bool _check(std::string& errorMessage) override;
00043
00044 protected:
00045
00046 private: // methods
00047 private: // attributes 1:1
00048 private: // attributes 1:n
00049 };
00050
00051
00052 #endif /* SUBMODEL_H */

```

## 16.431 Match.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Match.h
00009 * Author: rlcancian
00010 *
00011 * Created on 03 de Junho de 2019, 15:14
00012 */
00013
00014 #ifndef MATCH_H
00015 #define MATCH_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "plugins/data/DiscreteProcessing/Queue.h"
00019
00048 class Match : public ModelComponent {
00049 public:

```

```

00050
00051 enum class Rule : int {
00052 Any = 0, ByAttribute = 1, num_elements = 2
00053 };
00054 public:
00055 static std::string convertEnumToStr(Rule rule);
00056 public: // constructors
00057 Match(Model* model, std::string name = "");
00058 virtual ~Match() override;
00059 public: // virtual
00060 virtual std::string show() override;
00061 public: // static
00062 static PluginInformation* GetPluginInformation();
00063 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00064 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00065 public: // virtual
00066 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00067 virtual bool _loadInstance(PersistenceRecord *fields) override;
00068 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00069 public:
00070 void setRule(Match::Rule _rule);
00071 Match::Rule getRule() const;
00072 void setAttributeName(std::string _attributeName);
00073 std::string getAttributeName() const;
00074 void setMatchSize(std::string _matchSize);
00075 std::string getMatchSize() const;
00076 void setNumberOfQueues(unsigned int _numberOfQueues);
00077 unsigned int getNumberOfQueues() const;
00078 protected: // virtual
00079 //virtual void _initBetweenReplications();
00080 // virtual void _createInternalStatisticReporters() override;
00081 // virtual void _createNonEditableDataDefinitions() override;
00082 virtual void _createEditableDataDefinitions() override;
00083 // virtual void _createAttachedAttributes() override;
00084 virtual bool _check(std::string& errorMessage) override;
00085 // virtual void _createInternalAndAttachedData() override;
00086
00087 protected:
00088
00089 private: // methods
00090 private: // attributes 1:1
00091
00092 const struct DEFAULT_VALUES {
00093 const Match::Rule rule = Match::Rule::Any;
00094 const unsigned int numberOfQueues = 2;
00095 const std::string matchSize = "1";
00096 const std::string attributeName = "";
00097 } DEFAULT;
00098 Match::Rule _rule = DEFAULT.rule;
00099 unsigned int _numberOfQueues = DEFAULT.numberOfQueues;
00100 std::string _matchSize = DEFAULT.matchSize;
00101 std::string _attributeName = DEFAULT.attributeName;
00102 private: // attributes 1:1
00103 std::map<Queue*, std::map<double, unsigned int>*>* _entitiesByAttrib = new std::map<Queue*,
std::map<double, unsigned int>*>();
00104 private: // attributes 1:n
00105 List<Queue*>* _queues = new List<Queue*>();
00106 };
00107
00108
00109 #endif /* MATCH_H */

```

## 16.432 Signal.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Signal.h
00009 * Author: rlcancian
00010 *
00011 * Created on 03 de Junho de 2019, 15:20
00012 */
00013
00014 #ifndef SIGNAL_H
00015 #define SIGNAL_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "../data/Synchronization/SignalData.h"
00019
00039 class Signal : public ModelComponent {
00040 public: // constructors

```

```

00041 Signal(Model* model, std::string name = "");
00042 virtual ~Signal() = default;
00043 public: // virtual
00044 virtual std::string show() override;
00045 public:
00046 void setSignalData(SignalData* signal);
00047 const std::string&limitExpression() const;
00048 void setLimitExpression(const std::string&newLimitExpression);
00049 public: // static
00050 static PluginInformation* GetPluginInformation();
00051 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00052 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00053
00054 protected: // must be overridden
00055 virtual bool _loadInstance(PersistenceRecord *fields) override;
00056 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00057 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00058 protected: // could be overridden .
00059 virtual bool _check(std::string& errorMessage) override;
00060 virtual void _initBetweenReplications() override;
00061 // virtual void _createInternalAndAttachedData() override;
00062 //virtual ParserChangesInformation* _getParserChangesInformation();
00063
00064 protected:
00065 // virtual void _createInternalStatisticReporters() override;
00066 // virtual void _createNonEditableDataDefinitions() override;
00067 virtual void _createEditableDataDefinitions() override;
00068 // virtual void _createAttachedAttributes() override;
00069
00070 private: // methods
00071 private: // attributes 1:1
00072 const struct DEFAULT_VALUES {
00073 const std::string limitExpression = "1";
00074 } DEFAULT;
00075 std::string _limitExpression = DEFAULT.limitExpression;
00076 unsigned int _signalsTriggered = 0;
00077 private: // attributes 1:n
00078 private: // attached
00079 SignalData* _signalData = nullptr;
00080
00081 };
00082
00083
00084 #endif /* SIGNAL_H */

```

## 16.433 Wait.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Wait.h
00009 * Author: rlcancian
00010 *
00011 * Created on 03 de Junho de 2019, 15:20
00012 */
00013
00014 #ifndef WAIT_H
00015 #define WAIT_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "plugins/data/DiscreteProcessing/Queue.h"
00019 #include "../data/Synchronization/SignalData.h"
00020 #include "kernel/simulator/OnEventManager.h"
00021
00022 class Wait : public ModelComponent {
00023 public:
00024
00025 enum class WaitType : int {
00026 WaitForSignal = 0, InfiniteHold = 1, ScanForCondition = 2, num_elements = 3
00027 };
00028 public:
00029 static std::string convertEnumToStr(WaitType type);
00030 public: // constructors
00031 Wait(Model* model, std::string name = "");
00032 virtual ~Wait() = default;
00033 public: // virtual
00034 virtual std::string show() override;
00035 public: //
00036 void setSignalData(SignalData* signal);
00037 void setWaitType(WaitType _waitType);
00038 Wait::WaitType getWaitType() const;

```

```

00095 void setCondition(std::string _condition);
00096 std::string getCondition() const;
00097 Queue* getQueue() const;
00098 void setQueue(Queue* queue);
00099 public: // static
00100 static PluginInformation* GetPluginInformation();
00101 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00102 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00103 std::string getLimitExpression() const;
00104 void setLimitExpression(const std::string &newLimitExpression);
00105
00106 protected: // must be overridden
00107 virtual bool _loadInstance(PersistenceRecord *fields) override;
00108 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00109 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00110 protected: // could be overridden
00111 virtual bool _check(std::string& errorMessage) override;
00112 virtual void _initBetweenReplications() override;
00113 // virtual void _createInternalAndAttachedData() override;
00114 //virtual ParserChangesInformation* _getParserChangesInformation();
00115
00116 protected:
00117 //virtual void _createInternalStatisticReporters() override;
00118 virtual void _createNonEditableDataDefinitions() override;
00119 virtual void _createEditableDataDefinitions() override;
00120 //virtual void _createAttachedAttributes() override;
00121
00122 private: // methods
00123 unsigned int _handlerForSignalDataEvent(SignalData* signalData);
00124 void _handlerForAfterProcessEventEvent(SimulationEvent* event);
00125 private: // attributes 1:1
00126
00127 const struct DEFAULT_VALUES {
00128 const WaitType waitType = Wait::WaitType::WaitForSignal;
00129 const std::string condition = "";
00130 const std::string limitExpression = "0";
00131 } DEFAULT;
00132 WaitType _waitType = DEFAULT.waitType;
00133 std::string _condition = DEFAULT.condition;
00134 std::string limitExpression = DEFAULT.limitExpression;
00135 private: // internal
00136 Queue *_queue = nullptr; // @TODO: It should be a QueueableItem, (Queue or Set)
00137 private: // attached
00138 SignalData* _signalData = nullptr;
00139 bool _isScanConditionHandlerRegistered = false; // local guard to avoid duplicate registration in
 repeated checks
00140 private: // attributes 1:n
00141 };
00142
00143
00144 #endif /* WAIT_H */

```

## 16.434 DummyComponent.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Dummy.h
00009 * Author: Prof. Rafael Luiz Cancian, Dr. Eng.
00010 *
00011 * Created on 22 de Maio de 2019, 18:41
00012 */
00013
00014 #pragma once
00015
00016 #include "../kernel/simulator/model/ModelComponent.h"
00017 #include "plugins/data/Template/DummyElement.h"
00018
00022 class DummyComponent : public ModelComponent {
00023 public:
00024 DummyComponent(Model* model, std::string name = "");
00025 virtual ~DummyComponent() = default;
00026
00027 public:
00028 // ...
00029
00030 public:
00031 virtual std::string show() override;
00032
00033 public:
00034 static PluginInformation* GetPluginInformation();

```



```

00035 static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00036 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00037
00038 protected:
00039 virtual bool _loadInstance(PersistenceRecord *fields) override;
00040 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00041 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00042
00043
00044 protected:
00045 // virtual bool _check(std::string& errorMessage);
00046 // virtual ParserChangesInformation* _getParserChangesInformation();
00047 // virtual void _initBetweenReplications();
00048 // virtual void _createInternalStatisticReporters() override;
00049 // virtual void _createNonEditableDataDefinitions() override;
00050 // virtual void _createEditableDataDefinitions() override;
00051 // virtual void _createAttachedAttributes() override;
00052 // virtual void _createInternalAndAttachedData(); /*< A ModelDataDefinition or ModelComponent that
includes (internal) ou refers to (attach) other ModelDataDefinition must register them inside this
method. */
00053 // virtual void _addSimulationControl(SimulationControl* property);
00054
00055
00056 protected:
00057
00058 private:
00059 // ...
00060
00061 private:
00062 const struct DEFAULT_VALUES {
00063 const std::string someString = "Test";
00064 const unsigned int someUInt = 1;
00065 } DEFAULT;
00066 std::string _someString = DEFAULT.someString;
00067 unsigned int _someUInt = DEFAULT.someUInt;
00068
00069 private:
00070 DummyElement* _internalDataDefinition = nullptr;
00071
00072 private:
00073 // ...
00074
00075 };

```

## 16.435 BioStateProjectionComponent.h

```

00001 #ifndef BIONETWORKPROJECTIONCOMPONENT_H
00002 #define BIONETWORKPROJECTIONCOMPONENT_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 #include "../kernel/simulator/model/ModelComponent.h"
00008 #include "kernel/simulator/Plugin.h"
00009 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00010
00011 class BioSpecies;
00012
00013 class BioStateProjectionComponent : public ModelComponent {
00014 public:
00015 enum class ProjectionKind {
00016 MoleculeCount,
00017 MetaboliteAmount
00018 };
00019
00020 struct Projection {
00021 std::string sourceSpeciesName;
00022 std::string targetKey;
00023 ProjectionKind kind = ProjectionKind::MoleculeCount;
00024 double scale = 1.0;
00025 double offset = 0.0;
00026 };
00027
00028 public:
00029 BioStateProjectionComponent(Model* model, std::string name = "");
00030 virtual ~BioStateProjectionComponent() override = default;
00031
00032 public:
00033 static PluginInformation* GetPluginInformation();
00034 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00035 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00036
00037 public:
00038 void setWholeCellState(WholeCellState* state);
00039 WholeCellState* getWholeCellState() const;

```

```

00049 void clearProjections();
00050 void addMoleculeProjection(const std::string& sourceSpeciesName, const std::string& targetKey =
00051 "",
00052 double scale = 1.0, double offset = 0.0);
00053 void addMetaboliteProjection(const std::string& sourceSpeciesName, const std::string& targetKey =
00054 "",
00055 double scale = 1.0, double offset = 0.0);
00056 unsigned int getProjectionCount() const;
00057 void setLastSucceeded(bool lastSucceeded);
00058 bool getLastSucceeded() const;
00059 void setLastMessage(std::string lastMessage);
00060 std::string getLastMessage() const;
00061 virtual std::string show() override;
00062 protected:
00063 virtual bool _loadInstance(PersistenceRecord* fields) override;
00064 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00065 virtual bool _check(std::string& errorMessage) override;
00066 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00067 virtual void _createEditableDataDefinitions() override;
00068 private:
00069 void _forwardEntity(Entity* entity);
00070 void _syncEditableDataDefinitions();
00071 private:
00072 const struct DEFAULT_VALUES {
00073 std::string wholeCellStateName = "";
00074 bool lastSucceeded = false;
00075 std::string lastMessage = "";
00076 } DEFAULT;
00077 WholeCellState* _wholeCellState = nullptr;
00078 std::vector<Projection> _projections;
00079 bool _lastSucceeded = DEFAULT.lastSucceeded;
00080 std::string _lastMessage = DEFAULT.lastMessage;
00081 };
00082
00083 #endif /* BIONETWORKPROJECTIONCOMPONENT_H */

```

## 16.436 CellCycleCheckpointComponent.h

```

00001 #ifndef CELLCYCLECHECKPOINTCOMPONENT_H
00002 #define CELLCYCLECHECKPOINTCOMPONENT_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelComponent.h"
00007 #include "kernel/simulator/Plugin.h"
00008 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00009
00010 class CellCycleCheckpointComponent : public ModelComponent {
00011 public:
00012 CellCycleCheckpointComponent(Model* model, std::string name = "");
00013 virtual ~CellCycleCheckpointComponent() override = default;
00014
00015 public:
00016 static PluginInformation* GetPluginInformation();
00017 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00018 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00019
00020 public:
00021 void setWholeCellState(WholeCellState* state);
00022 WholeCellState* getWholeCellState() const;
00023 void setDeltaT(double deltaT);
00024 double getDeltaT() const;
00025 void setAdvanceWholeCellClock(bool advance);
00026 bool getAdvanceWholeCellClock() const;
00027 void setEnergyMetaboliteKey(std::string key);
00028 std::string getEnergyMetaboliteKey() const;
00029 void setCompartmentEnergyRegion(std::string region);
00030 std::string getCompartmentEnergyRegion() const;
00031 void setCompartmentEnergyMetaboliteKey(std::string key);
00032 std::string getCompartmentEnergyMetaboliteKey() const;
00033 void setStarvationAtpThreshold(double threshold);
00034 double getStarvationAtpThreshold() const;
00035 void setCompartmentStarvationThreshold(double threshold);
00036 double getCompartmentStarvationThreshold() const;
00037 void setLethalStarvationSteps(unsigned int steps);
00038 unsigned int getLethalStarvationSteps() const;
00039 void setDivisionMassThreshold(double threshold);
00040 double getDivisionMassThreshold() const;
00041 void setFtsZRingKey(std::string key);
00042 std::string getFtsZRingKey() const;

```

```

00060 void setFtsZThreshold(double threshold);
00061 double getFtsZThreshold() const;
00062 void setCriticalPathwayActivityKey(std::string key);
00063 std::string getCriticalPathwayActivityKey() const;
00064 void setCriticalPathwayActivityThreshold(double threshold);
00065 double getCriticalPathwayActivityThreshold() const;
00066 unsigned int getStarvationStreak() const;
00067
00068 virtual std::string show() override;
00069
00070 protected:
00071 virtual bool _loadInstance(PersistenceRecord* fields) override;
00072 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00073 virtual bool _check(std::string& errorMessage) override;
00074 virtual void _initBetweenReplications() override;
00075 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00076
00077 protected:
00078 virtual void _createEditableDataDefinitions() override;
00079
00080 private:
00081 bool _isDivisionReady() const;
00082 void _updateLifecyclePhase();
00083 void _forwardEntity(Entity* entity);
00084
00085 private:
00086 const struct DEFAULT_VALUES {
00087 std::string wholeCellStateName = "";
00088 double deltaT = 60.0;
00089 bool advanceWholeCellClock = true;
00090 std::string energyMetaboliteKey = "ATP";
00091 std::string compartmentEnergyRegion = "";
00092 std::string compartmentEnergyMetaboliteKey = "";
00093 double starvationAtpThreshold = 0.25;
00094 double compartmentStarvationThreshold = 0.0;
00095 unsigned int lethalStarvationSteps = 0u;
00096 double divisionMassThreshold = 0.0;
00097 std::string ftsZRingKey = "FtsZ_ring_completion";
00098 double ftsZThreshold = 0.0;
00099 std::string criticalPathwayActivityKey = "";
00100 double criticalPathwayActivityThreshold = 0.0;
00101 } DEFAULT;
00102
00103 WholeCellState* _wholeCellState = nullptr;
00104 double _deltaT = DEFAULT.deltaT;
00105 bool _advanceWholeCellClock = DEFAULT.advanceWholeCellClock;
00106 std::string _energyMetaboliteKey = DEFAULT.energyMetaboliteKey;
00107 std::string _compartmentEnergyRegion = DEFAULT.compartmentEnergyRegion;
00108 std::string _compartmentEnergyMetaboliteKey = DEFAULT.compartmentEnergyMetaboliteKey;
00109 double _starvationAtpThreshold = DEFAULT.starvationAtpThreshold;
00110 double _compartmentStarvationThreshold = DEFAULT.compartmentStarvationThreshold;
00111 unsigned int _lethalStarvationSteps = DEFAULT.lethalStarvationSteps;
00112 double _divisionMassThreshold = DEFAULT.divisionMassThreshold;
00113 std::string _ftsZRingKey = DEFAULT.ftsZRingKey;
00114 double _ftsZThreshold = DEFAULT.ftsZThreshold;
00115 std::string _criticalPathwayActivityKey = DEFAULT.criticalPathwayActivityKey;
00116 double _criticalPathwayActivityThreshold = DEFAULT.criticalPathwayActivityThreshold;
00117 unsigned int _starvationStreak = 0u;
00118 };
00119
00120 #endif /* CELLCYCLECHECKPOINTCOMPONENT_H */

```

## 16.437 CellDivisionEvent.h

```

00001 #ifndef CELLDIVISIONEVENT_H
00002 #define CELLDIVISIONEVENT_H
00003
00004 #include <random>
00005 #include <string>
00006
00007 #include "../kernel/simulator/model/ModelComponent.h"
00008 #include "kernel/simulator/Plugin.h"
00009 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00010
00039 class CellDivisionEvent : public ModelComponent {
00040 public:
00041 CellDivisionEvent(Model* model, std::string name = "");
00042 virtual ~CellDivisionEvent() override = default;
00043
00044 public:
00045 static PluginInformation* GetPluginInformation();
00046 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00047 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00048
00049 public:

```

```

00050 void setWholeCellState(WholeCellState* state);
00051 WholeCellState* getWholeCellState() const;
00052 void setDivisionMassThreshold(double threshold);
00053 double getDivisionMassThreshold() const;
00054 void setFtsZThreshold(double threshold);
00055 double getFtsZThreshold() const;
00056 void setFtsZRingKey(std::string key);
00057 std::string getFtsZRingKey() const;
00058 void setRandomSeed(unsigned int seed);
00059 unsigned int getRandomSeed() const;
00060 bool getLastDivided() const;
00061 int getDivisionCount() const;
00062
00063 virtual std::string show() override;
00064
00065 protected:
00066 virtual bool _loadInstance(PersistenceRecord* fields) override;
00067 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00068 virtual bool _check(std::string& errorMessage) override;
00069 virtual void _initBetweenReplications() override;
00070 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00071
00072 protected:
00073 virtual void _createEditableDataDefinitions() override;
00074
00075 private:
00076 bool _checkDivisionCondition() const;
00077 void _applyDivision();
00078 void _forwardEntity(Entity* entity, unsigned int port);
00079
00080 private:
00081 const struct DEFAULT_VALUES {
00082 std::string wholeCellStateName = "";
00083 double divisionMassThreshold = 7.86e-15; // 2 x 3.93e-15 g (M. genitalium,
parameters.json)
00084 double ftsZThreshold = 0.0; // 0 = mass-only trigger; >0 requires FtsZ ring
00085 std::string ftsZRingKey = "FtsZ_ring_completion";
00086 unsigned int randomSeed = 44u;
00087 } DEFAULT;
00088
00089 WholeCellState* _wholeCellState = nullptr;
00090 double _divisionMassThreshold = DEFAULT.divisionMassThreshold;
00091 double _ftsZThreshold = DEFAULT.ftsZThreshold;
00092 std::string _ftsZRingKey = DEFAULT.ftsZRingKey;
00093 unsigned int _randomSeed = DEFAULT.randomSeed;
00094 bool _lastDivided = false;
00095 int _divisionCount = 0;
00096 std::mt19937 _rng(DEFAULT.randomSeed);
00097 };
00098
00099 #endif /* CELLDIVISIONEVENT_H */

```

## 16.438 CellFateDecisionComponent.h

```

00001 #ifndef CELLFATEDECISIONCOMPONENT_H
00002 #define CELLFATEDECISIONCOMPONENT_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelComponent.h"
00007 #include "kernel/simulator/Plugin.h"
00008 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00009
00024 class CellFateDecisionComponent : public ModelComponent {
00025 public:
00026 CellFateDecisionComponent(Model* model, std::string name = "");
00027 virtual ~CellFateDecisionComponent() override = default;
00028
00029 public:
00030 static PluginInformation* GetPluginInformation();
00031 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00032 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00033
00034 public:
00035 void setWholeCellState(WholeCellState* state);
00036 WholeCellState* getWholeCellState() const;
00037 void setDivisionReadyPhase(std::string phase);
00038 std::string getDivisionReadyPhase() const;
00039 void setStarvedPhase(std::string phase);
00040 std::string getStarvedPhase() const;
00041 void setArrestedPhase(std::string phase);
00042 std::string getArrestedPhase() const;
00043 void setDeadPhase(std::string phase);
00044 std::string getDeadPhase() const;
00045 unsigned int getLastRoutedPort() const;

```

```

00046
00047 virtual std::string show() override;
00048
00049 protected:
00050 virtual bool _loadInstance(PersistenceRecord* fields) override;
00051 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00052 virtual bool _check(std::string& errorMessage) override;
00053 virtual void _initBetweenReplications() override;
00054 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00055
00056 protected:
00057 virtual void _createEditableDataDefinitions() override;
00058
00059 private:
00060 unsigned int _selectOutputPort() const;
00061 void _forwardEntity(Entity* entity, unsigned int port);
00062
00063 private:
00064 const struct DEFAULT_VALUES {
00065 std::string wholeCellStateName = "";
00066 std::string divisionReadyPhase = "division_ready";
00067 std::string starvedPhase = "starved";
00068 std::string arrestedPhase = "arrested";
00069 std::string deadPhase = "dead";
00070 unsigned int lastRoutedPort = 0u;
00071 } DEFAULT;
00072
00073 WholeCellState* _wholeCellState = nullptr;
00074 std::string _divisionReadyPhase = DEFAULT.divisionReadyPhase;
00075 std::string _starvedPhase = DEFAULT.starvedPhase;
00076 std::string _arrestedPhase = DEFAULT.arrestedPhase;
00077 std::string _deadPhase = DEFAULT.deadPhase;
00078 unsigned int _lastRoutedPort = DEFAULT.lastRoutedPort;
00079 };
00080
00081 #endif /* CELLFATEDECISIONCOMPONENT_H */

```

## 16.439 CellGrowthComponent.h

```

00001 #ifndef CELLGROWTHCOMPONENT_H
00002 #define CELLGROWTHCOMPONENT_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelComponent.h"
00007 #include "kernel/simulator/Plugin.h"
00008 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00009
00022 class CellGrowthComponent : public ModelComponent {
00023 public:
00024 CellGrowthComponent(Model* model, std::string name = "");
00025 virtual ~CellGrowthComponent() override = default;
00026
00027 public:
00028 static PluginInformation* GetPluginInformation();
00029 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00030 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00031
00032 public:
00033 void setWholeCellState(WholeCellState* state);
00034 WholeCellState* getWholeCellState() const;
00035 void setGrowthRate(double rate);
00036 double getGrowthRate() const;
00037 void setEnergyMetaboliteKey(std::string key);
00038 std::string getEnergyMetaboliteKey() const;
00039 void setEnergyHalfSaturation(double value);
00040 double getEnergyHalfSaturation() const;
00041 void setDensity(double density);
00042 double getDensity() const;
00043 void setDeltaT(double dt);
00044 double getDeltaT() const;
00045
00046 virtual std::string show() override;
00047
00048 protected:
00049 virtual bool _loadInstance(PersistenceRecord* fields) override;
00050 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00051 virtual bool _check(std::string& errorMessage) override;
00052 virtual void _initBetweenReplications() override;
00053 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00054
00055 protected:
00056 virtual void _createEditableDataDefinitions() override;
00057
00058 private:

```

```

00059 void _forwardEntity(Entity* entity);
00060
00061 private:
00062 const struct DEFAULT_VALUES {
00063 std::string wholeCellStateName = "";
00064 double growthRate = 2.1393e-05; // /s -- M. genitalium (parameters.json)
00065 std::string energyMetaboliteKey = "";
00066 double energyHalfSaturation = 0.0; // 0 disables energy gating
00067 double density = 1100.0; // kg/m3 -- states.Geometry.density
00068 double deltaT = 60.0; // s -- must match clock step
00069 } DEFAULT;
00070
00071 WholeCellState* _wholeCellState = nullptr;
00072 double _growthRate = DEFAULT.growthRate;
00073 std::string _energyMetaboliteKey = DEFAULT.energyMetaboliteKey;
00074 double _energyHalfSaturation = DEFAULT.energyHalfSaturation;
00075 double _density = DEFAULT.density;
00076 double _deltaT = DEFAULT.deltaT;
00077 };
00078
00079 #endif /* CELLGROWTHCOMPONENT_H */

```

## 16.440 CompartmentExchangeComponent.h

```

00001 #ifndef COMPARTMENTEXCHANGECOMPONENT_H
00002 #define COMPARTMENTEXCHANGECOMPONENT_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 #include "../kernel/simulator/model/ModelComponent.h"
00008 #include "kernel/simulator/Plugin.h"
00009 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00010
00024 class CompartmentExchangeComponent : public ModelComponent {
00025 public:
00026 struct ExchangeRule {
00027 std::string label;
00028 std::string sourceRegion;
00029 std::string sourceMetaboliteKey;
00030 std::string targetRegion;
00031 std::string targetMetaboliteKey;
00032 double exchangeFraction = 0.0;
00033 double baselineAmount = 0.0;
00034 std::string driverPathwayKey;
00035 double driverScale = 1.0;
00036 double maxTransferAmount = 0.0;
00037 bool conserveMass = true;
00038 };
00039
00040 public:
00041 CompartmentExchangeComponent(Model* model, std::string name = "");
00042 virtual ~CompartmentExchangeComponent() override = default;
00043
00044 public:
00045 static PluginInformation* GetPluginInformation();
00046 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00047 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00048
00049 public:
00050 void setWholeCellState(WholeCellState* state);
00051 WholeCellState* getWholeCellState() const;
00052 void setSourceRegion(std::string region);
00053 std::string getSourceRegion() const;
00054 void setSourceMetaboliteKey(std::string key);
00055 std::string getSourceMetaboliteKey() const;
00056 void setTargetRegion(std::string region);
00057 std::string getTargetRegion() const;
00058 void setTargetMetaboliteKey(std::string key);
00059 std::string getTargetMetaboliteKey() const;
00060 void setExchangeFraction(double fraction);
00061 double getExchangeFraction() const;
00062 void setBaselineAmount(double amount);
00063 double getBaselineAmount() const;
00064 void setDriverPathwayKey(std::string key);
00065 std::string getDriverPathwayKey() const;
00066 void setDriverScale(double scale);
00067 double getDriverScale() const;
00068 void setMaxTransferAmount(double amount);
00069 double getMaxTransferAmount() const;
00070 void setConserveMass(bool conserve);
00071 bool getConserveMass() const;
00072 void clearExchangeRules();
00073 void addExchangeRule(const std::string& label,
00074 const std::string& sourceRegion,

```

```

00075 const std::string& sourceMetaboliteKey,
00076 const std::string& targetRegion,
00077 const std::string& targetMetaboliteKey,
00078 double exchangeFraction,
00079 double baselineAmount = 0.0,
00080 const std::string& driverPathwayKey = "",
00081 double driverScale = 1.0,
00082 double maxTransferAmount = 0.0,
00083 bool conserveMass = true);
00084 unsigned int getExchangeRuleCount() const;
00085 const std::vector<ExchangeRule>& getExchangeRules() const;
00086 double getLastTransferAmount() const;
00087
00088 virtual std::string show() override;
00089
00090 protected:
00091 virtual bool _loadInstance(PersistenceRecord* fields) override;
00092 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00093 virtual bool _check(std::string& errorMessage) override;
00094 virtual void _initBetweenReplications() override;
00095 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00096 virtual void _createEditableDataDefinitions() override;
00097
00098 private:
00099 ExchangeRule _buildLegacyRule() const;
00100 std::vector<ExchangeRule> _getEffectiveRules() const;
00101 bool _checkRule(const ExchangeRule& rule, std::string& errorMessage, unsigned int index) const;
00102 double _computeTransferAmount(const ExchangeRule& rule) const;
00103 void _applyRule(const ExchangeRule& rule);
00104 double _getPoolAmount(const std::string& region, const std::string& key) const;
00105 void _setPoolAmount(const std::string& region, const std::string& key, double amount);
00106 void _forwardEntity(Entity* entity);
00107
00108 private:
00109 const struct DEFAULT_VALUES {
00110 std::string wholeCellStateName = "";
00111 std::string sourceRegion = "";
00112 std::string sourceMetaboliteKey = "";
00113 std::string targetRegion = "";
00114 std::string targetMetaboliteKey = "";
00115 double exchangeFraction = 0.0;
00116 double baselineAmount = 0.0;
00117 std::string driverPathwayKey = "";
00118 double driverScale = 1.0;
00119 double maxTransferAmount = 0.0;
00120 bool conserveMass = true;
00121 double lastTransferAmount = 0.0;
00122 } DEFAULT;
00123
00124 WholeCellState* _wholeCellState = nullptr;
00125 std::string _sourceRegion = DEFAULT.sourceRegion;
00126 std::string _sourceMetaboliteKey = DEFAULT.sourceMetaboliteKey;
00127 std::string _targetRegion = DEFAULT.targetRegion;
00128 std::string _targetMetaboliteKey = DEFAULT.targetMetaboliteKey;
00129 double _exchangeFraction = DEFAULT.exchangeFraction;
00130 double _baselineAmount = DEFAULT.baselineAmount;
00131 std::string _driverPathwayKey = DEFAULT.driverPathwayKey;
00132 double _driverScale = DEFAULT.driverScale;
00133 double _maxTransferAmount = DEFAULT.maxTransferAmount;
00134 bool _conserveMass = DEFAULT.conserveMass;
00135 std::vector<ExchangeRule> _exchangeRules;
00136 double _lastTransferAmount = DEFAULT.lastTransferAmount;
00137 };
00138
00139 #endif /* COMPARTMENTEXCHANGECOMPONENT_H */

```

## 16.441 EukaryoticCellCycleComponent.h

```

00001 #ifndef EUKARYOTICCELLCYCLECOMPONENT_H
00002 #define EUKARYOTICCELLCYCLECOMPONENT_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelComponent.h"
00007 #include "kernel/simulator/Plugin.h"
00008 #include "plugins/data/WholeCellModeling/BioCompartment.h"
00009 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00010
00039 class EukaryoticCellCycleComponent : public ModelComponent {
00040 public:
00041 EukaryoticCellCycleComponent(Model* model, std::string name = "");
00042 virtual ~EukaryoticCellCycleComponent() override = default;
00043
00044 public:
00045 static PluginInformation* GetPluginInformation();

```



```

00046 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00047 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00048
00049 public:
00050 void setWholeCellState(WholeCellState* state);
00051 WholeCellState* getWholeCellState() const;
00052 void setDeltaT(double deltaT);
00053 double getDeltaT() const;
00054 void setAdvanceWholeCellClock(bool advance);
00055 bool getAdvanceWholeCellClock() const;
00056
00057 void setCytosolCompartmentName(std::string name);
00058 std::string getCytosolCompartmentName() const;
00059 void setNucleusCompartmentName(std::string name);
00060 std::string getNucleusCompartmentName() const;
00061 void setMitochondrionCompartmentName(std::string name);
00062 std::string getMitochondrionCompartmentName() const;
00063 void setBudCompartmentName(std::string name);
00064 std::string getBudCompartmentName() const;
00065
00066 void setEnergyMetaboliteKey(std::string key);
00067 std::string getEnergyMetaboliteKey() const;
00068 void setCytosolicEnergyMetaboliteKey(std::string key);
00069 std::string getCytosolicEnergyMetaboliteKey() const;
00070 void setMitochondrialEnergyMetaboliteKey(std::string key);
00071 std::string getMitochondrialEnergyMetaboliteKey() const;
00072 void setBudEnergyMetaboliteKey(std::string key);
00073 std::string getBudEnergyMetaboliteKey() const;
00074
00075 void setGrowthFluxKey(std::string key);
00076 std::string getGrowthFluxKey() const;
00077 void setRespirationFluxKey(std::string key);
00078 std::string getRespirationFluxKey() const;
00079
00080 void setBudProgressKey(std::string key);
00081 std::string getBudProgressKey() const;
00082 void setDnaReplicationProgressKey(std::string key);
00083 std::string getDnaReplicationProgressKey() const;
00084 void setSpindleAssemblyProgressKey(std::string key);
00085 std::string getSpindleAssemblyProgressKey() const;
00086 void setMitoticExitProgressKey(std::string key);
00087 std::string getMitoticExitProgressKey() const;
00088
00089 void setGlobalAtpThreshold(double threshold);
00090 double getGlobalAtpThreshold() const;
00091 void setCytosolicAtpThreshold(double threshold);
00092 double getCytosolicAtpThreshold() const;
00093 void setMitochondrialAtpThreshold(double threshold);
00094 double getMitochondrialAtpThreshold() const;
00095 void setBudAtpThreshold(double threshold);
00096 double getBudAtpThreshold() const;
00097 void setGrowthFluxThreshold(double threshold);
00098 double getGrowthFluxThreshold() const;
00099 void setRespirationFluxThreshold(double threshold);
00100 double getRespirationFluxThreshold() const;
00101
00102 void setBudProgressRate(double rate);
00103 double getBudProgressRate() const;
00104 void setDnaReplicationRate(double rate);
00105 double getDnaReplicationRate() const;
00106 void setSpindleAssemblyRate(double rate);
00107 double getSpindleAssemblyRate() const;
00108 void setMitoticExitRate(double rate);
00109 double getMitoticExitRate() const;
00110
00111 void setBudProgressThreshold(double threshold);
00112 double getBudProgressThreshold() const;
00113 void setDnaReplicationThreshold(double threshold);
00114 double getDnaReplicationThreshold() const;
00115 void setSpindleAssemblyThreshold(double threshold);
00116 double getSpindleAssemblyThreshold() const;
00117 void setMitoticExitThreshold(double threshold);
00118 double getMitoticExitThreshold() const;
00119
00120 virtual std::string show() override;
00121
00122 protected:
00123 virtual bool _loadInstance(PersistenceRecord* fields) override;
00124 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00125 virtual bool _check(std::string& errorMessage) override;
00126 virtual void _initBetweenReplications() override;
00127 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00128 virtual void _createEditableDataDefinitions() override;
00129
00130 private:
00131 double _normalizedSignal(double value, double threshold) const;
00132 double _averageSignal(double a, double b) const;

```



```

00133 double _advanceProgress(const std::string& key, double increment) const;
00134 void _updateLifecyclePhase();
00135 void _forwardEntity(Entity* entity);
00136 bool _validateReferencedCompartment(const std::string& name, std::string* errorMessage) const;
00137 BioCompartment* _findCompartment(const std::string& name) const;
00138
00139 private:
00140 const struct DEFAULT_VALUES {
00141 std::string wholeCellStateName = "";
00142 double deltaT = 60.0;
00143 bool advanceWholeCellClock = true;
00144 std::string cytosolCompartmentName = "cytosol";
00145 std::string nucleusCompartmentName = "nucleus";
00146 std::string mitochondrionCompartmentName = "mitochondria";
00147 std::string budCompartmentName = "bud";
00148 std::string energyMetaboliteKey = "ATP";
00149 std::string cytosolicEnergyMetaboliteKey = "ATP_c";
00150 std::string mitochondrialEnergyMetaboliteKey = "ATP_m";
00151 std::string budEnergyMetaboliteKey = "ATP_bud";
00152 std::string growthFluxKey = "biomass_flux";
00153 std::string respirationFluxKey = "respiration_flux";
00154 std::string budProgressKey = "bud_growth_progress";
00155 std::string dnaReplicationProgressKey = "dna_replication_progress";
00156 std::string spindleAssemblyProgressKey = "spindle_assembly_progress";
00157 std::string mitoticExitProgressKey = "mitotic_exit_progress";
00158 double globalAtpThreshold = 0.50;
00159 double cytosolicAtpThreshold = 0.40;
00160 double mitochondrialAtpThreshold = 0.35;
00161 double budAtpThreshold = 0.20;
00162 double growthFluxThreshold = 4.0;
00163 double respirationFluxThreshold = 1.0;
00164 double budProgressRate = 0.30;
00165 double dnaReplicationRate = 0.24;
00166 double spindleAssemblyRate = 0.22;
00167 double mitoticExitRate = 0.20;
00168 double budProgressThreshold = 1.0;
00169 double dnaReplicationThreshold = 1.0;
00170 double spindleAssemblyThreshold = 1.0;
00171 double mitoticExitThreshold = 1.0;
00172 } DEFAULT;
00173
00174 WholeCellState* _wholeCellState = nullptr;
00175 double _deltaT = DEFAULT.deltaT;
00176 bool _advanceWholeCellClock = DEFAULT.advanceWholeCellClock;
00177 std::string _cytosolCompartmentName = DEFAULT.cytosolCompartmentName;
00178 std::string _nucleusCompartmentName = DEFAULT.nucleusCompartmentName;
00179 std::string _mitochondrionCompartmentName = DEFAULT.mitochondrionCompartmentName;
00180 std::string _budCompartmentName = DEFAULT.budCompartmentName;
00181 std::string _energyMetaboliteKey = DEFAULT.energyMetaboliteKey;
00182 std::string _cytosolicEnergyMetaboliteKey = DEFAULT.cytosolicEnergyMetaboliteKey;
00183 std::string _mitochondrialEnergyMetaboliteKey = DEFAULT.mitochondrialEnergyMetaboliteKey;
00184 std::string _budEnergyMetaboliteKey = DEFAULT.budEnergyMetaboliteKey;
00185 std::string _growthFluxKey = DEFAULT.growthFluxKey;
00186 std::string _respirationFluxKey = DEFAULT.respirationFluxKey;
00187 std::string _budProgressKey = DEFAULT.budProgressKey;
00188 std::string _dnaReplicationProgressKey = DEFAULT.dnaReplicationProgressKey;
00189 std::string _spindleAssemblyProgressKey = DEFAULT.spindleAssemblyProgressKey;
00190 std::string _mitoticExitProgressKey = DEFAULT.mitoticExitProgressKey;
00191 double _globalAtpThreshold = DEFAULT.globalAtpThreshold;
00192 double _cytosolicAtpThreshold = DEFAULT.cytosolicAtpThreshold;
00193 double _mitochondrialAtpThreshold = DEFAULT.mitochondrialAtpThreshold;
00194 double _budAtpThreshold = DEFAULT.budAtpThreshold;
00195 double _growthFluxThreshold = DEFAULT.growthFluxThreshold;
00196 double _respirationFluxThreshold = DEFAULT.respirationFluxThreshold;
00197 double _budProgressRate = DEFAULT.budProgressRate;
00198 double _dnaReplicationRate = DEFAULT.dnaReplicationRate;
00199 double _spindleAssemblyRate = DEFAULT.spindleAssemblyRate;
00200 double _mitoticExitRate = DEFAULT.mitoticExitRate;
00201 double _budProgressThreshold = DEFAULT.budProgressThreshold;
00202 double _dnaReplicationThreshold = DEFAULT.dnaReplicationThreshold;
00203 double _spindleAssemblyThreshold = DEFAULT.spindleAssemblyThreshold;
00204 double _mitoticExitThreshold = DEFAULT.mitoticExitThreshold;
00205 };
00206
00207 #endif /* EUKARYOTICCELLCYCLECOMPONENT_H */

```

## 16.442 FtsZPolymerizationComponent.h

```

00001 #ifndef FTSZPOLYMERIZATIONCOMPONENT_H
00002 #define FTSZPOLYMERIZATIONCOMPONENT_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelComponent.h"
00007 #include "kernel/simulator/Plugin.h"

```

```

00008 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00009
00035 class FtsZPolymerizationComponent : public ModelComponent {
00036 public:
00037 FtsZPolymerizationComponent(Model* model, std::string name = "");
00038 virtual ~FtsZPolymerizationComponent() override = default;
00039
00040 public:
00041 static PluginInformation* GetPluginInformation();
00042 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00043 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00044
00045 public:
00046 void setWholeCellState(WholeCellState* state);
00047 WholeCellState* getWholeCellState() const;
00048 void setActivationFwd(double rate);
00049 double getActivationFwd() const;
00050 void setActivationRev(double rate);
00051 double getActivationRev() const;
00052 void setVolumeNorm(double norm);
00053 double getVolumeNorm() const;
00054 void setDeltaT(double dt);
00055 double getDeltaT() const;
00056 void setFtsZRingKey(std::string key);
00057 std::string getFtsZRingKey() const;
00058 void setFtsZMonomerKey(std::string key);
00059 std::string getFtsZMonomerKey() const;
00060
00061 virtual std::string show() override;
00062
00063 protected:
00064 virtual bool _loadInstance(PersistenceRecord* fields) override;
00065 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00066 virtual bool _check(std::string& errorMessage) override;
00067 virtual void _initBetweenReplications() override;
00068 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00069
00070 protected:
00071 virtual void _createEditableDataDefinitions() override;
00072
00073 private:
00074 void _forwardEntity(Entity* entity);
00075
00076 private:
00077 const struct DEFAULT_VALUES {
00078 std::string wholeCellStateName = "";
00079 double activationFwd = 1.1; // /s -- processes.FtsZPolymerization.activationFwd
00080 double activationRev = 0.01; // /s -- derived from Karr 2012 ratio
00081 double volumeNorm = 3.94e-5; // dimensionless -- calibrated for 35% at ~12000 s
00082 double deltaT = 60.0; // s -- must match clock step
00083 std::string ftsZRingKey = "FtsZ_ring_completion";
00084 std::string ftsZMonomerKey = "FtsZ_monomer";
00085 } DEFAULT;
00086
00087 WholeCellState* _wholeCellState = nullptr;
00088 double _activationFwd = DEFAULT.activationFwd;
00089 double _activationRev = DEFAULT.activationRev;
00090 double _volumeNorm = DEFAULT.volumeNorm;
00091 double _deltaT = DEFAULT.deltaT;
00092 std::string _ftsZRingKey = DEFAULT.ftsZRingKey;
00093 std::string _ftsZMonomerKey = DEFAULT.ftsZMonomerKey;
00094 int _nRef = 0; // reference monomer count captured on first step
00095 };
00096
00097 #endif /* FTSZPOLYMERIZATIONCOMPONENT_H */

```

## 16.443 MetabolicStateProjectionComponent.h

```

00001 #ifndef METABOLICSTATEPROJECTIONCOMPONENT_H
00002 #define METABOLICSTATEPROJECTIONCOMPONENT_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 #include "../kernel/simulator/model/ModelComponent.h"
00008 #include "kernel/simulator/Plugin.h"
00009 #include "plugins/components/BiochemicalSimulation/MetabolicFluxBalance.h"
00010 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00011
00027 class MetabolicStateProjectionComponent : public ModelComponent {
00028 public:
00029 enum class ProjectionKind {
00030 MetaboliteAmount,
00031 CompartmentMetaboliteAmount,
00032 PathwayActivity

```

```

00033 };
00034
00035 enum class ProjectionUpdateMode {
00036 Overwrite,
00037 Accumulate,
00038 Turnover
00039 };
00040
00041 struct Projection {
00042 std::string reactionName;
00043 std::string targetKey;
00044 ProjectionKind kind = ProjectionKind::MetaboliteAmount;
00045 ProjectionUpdateMode updateMode = ProjectionUpdateMode::Overwrite;
00046 double scale = 1.0;
00047 double offset = 0.0;
00048 double turnoverFraction = 0.0;
00049 std::string compartmentName;
00050 };
00051
00052 public:
00053 MetabolicStateProjectionComponent(Model* model, std::string name = "");
00054 virtual ~MetabolicStateProjectionComponent() override = default;
00055
00056 public:
00057 static PluginInformation* GetPluginInformation();
00058 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00059 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00060
00061 public:
00062 void setWholeCellState(WholeCellState* state);
00063 WholeCellState* getWholeCellState() const;
00064 void setFluxBalanceComponent(MetabolicFluxBalance* component);
00065 MetabolicFluxBalance* getFluxBalanceComponent() const;
00066 void setFluxBalanceComponentName(std::string componentName);
00067 std::string getFluxBalanceComponentName() const;
00068
00069 void clearObjectiveProjection();
00070 void setObjectiveAsMetabolite(const std::string& targetKey, double scale = 1.0, double offset =
00071 0.0,
00072 ProjectionUpdateMode updateMode = ProjectionUpdateMode::Overwrite,
00073 double turnoverFraction = 0.0);
00074 void setObjectiveAsCompartmentMetabolite(const std::string& componentName, const std::string&
00075 targetKey,
00076 double scale = 1.0, double offset = 0.0,
00077 ProjectionUpdateMode updateMode =
00078 ProjectionUpdateMode::Overwrite,
00079 double turnoverFraction = 0.0);
00080 bool hasObjectiveProjection() const;
00081
00082 void clearFluxProjections();
00083 void addMetaboliteProjection(const std::string& reactionName, const std::string& targetKey,
00084 double scale = 1.0, double offset = 0.0,
00085 ProjectionUpdateMode updateMode = ProjectionUpdateMode::Overwrite,
00086 double turnoverFraction = 0.0);
00087 void addCompartmentMetaboliteProjection(const std::string& reactionName, const std::string&
00088 compartmentName,
00089 const std::string& targetKey, double scale = 1.0, double
00090 offset = 0.0,
00091 ProjectionUpdateMode updateMode =
00092 ProjectionUpdateMode::Overwrite,
00093 double turnoverFraction = 0.0);
00094 void addPathwayProjection(const std::string& reactionName, const std::string& targetKey,
00095 double scale = 1.0, double offset = 0.0,
00096 ProjectionUpdateMode updateMode = ProjectionUpdateMode::Overwrite,
00097 double turnoverFraction = 0.0);
00098 unsigned int getProjectionCount() const;
00099
00100 void setLastSucceeded(bool lastSucceeded);
00101 bool getLastSucceeded() const;
00102 void setLastMessage(std::string lastMessage);
00103 std::string getLastMessage() const;
00104
00105 virtual std::string show() override;
00106
00107 protected:
00108 virtual bool _loadInstance(PersistenceRecord* fields) override;
00109 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00110 virtual bool _check(std::string& errorMessage) override;
00111 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00112 virtual void _createEditableDataDefinitions() override;
00113 private:

```

```

00112 void _forwardEntity(Entity* entity);
00113 void _syncEditableDataDefinitions();
00114 void _resolveFluxBalanceComponent();
00115 void _configureObjectiveProjection(ProjectionKind kind, const std::string& targetKey, const
std::string& compartmentName,
00116 double scale, double offset, ProjectionUpdateMode updateMode,
00117 double turnoverFraction);
00118 void _appendProjection(const Projection& projection);
00119 void _applyProjection(const Projection& projection, double value);
00120 std::string _resolveCompartmentName(const Projection& projection) const;
00121 double _resolveCurrentValue(const Projection& projection) const;
00122
00123 private:
00124 const struct DEFAULT_VALUES {
00125 std::string wholeCellStateName = "";
00126 std::string fluxBalanceComponentName = "";
00127 bool objectiveProjectionEnabled = false;
00128 std::string objectiveTargetKey = "";
00129 std::string objectiveKind = "MetaboliteAmount";
00130 std::string objectiveUpdateMode = "Overwrite";
00131 double objectiveScale = 1.0;
00132 double objectiveOffset = 0.0;
00133 double objectiveTurnoverFraction = 0.0;
00134 std::string objectiveCompartmentName = "";
00135 bool lastSucceeded = false;
00136 std::string lastMessage = "";
00137 } DEFAULT;
00138
00139 WholeCellState* _wholeCellState = nullptr;
00140 MetabolicFluxBalance* _fluxBalanceComponent = nullptr;
00141 std::string _fluxBalanceComponentName = DEFAULT.fluxBalanceComponentName;
00142 bool _objectiveProjectionEnabled = DEFAULT.objectiveProjectionEnabled;
00143 Projection _objectiveProjection;
00144 std::vector<Projection> _projections;
00145 bool _lastSucceeded = DEFAULT.lastSucceeded;
00146 std::string _lastMessage = DEFAULT.lastMessage;
00147 };
00148
00149 #endif /* METABOLICSTATEPROJECTIONCOMPONENT_H */

```

## 16.444 MetabolicSubmodelComponent.h

```

00001 #ifndef METABOLICSUBMODELCOMPONENT_H
00002 #define METABOLICSUBMODELCOMPONENT_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelComponent.h"
00007 #include "kernel/simulator/Plugin.h"
00008 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00009
00029 class MetabolicSubmodelComponent : public ModelComponent {
00030 public:
00031 MetabolicSubmodelComponent(Model* model, std::string name = "");
00032 virtual ~MetabolicSubmodelComponent() override = default;
00033
00034 public:
00035 static PluginInformation* GetPluginInformation();
00036 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00037 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00038
00039 public:
00040 void setWholeCellState(WholeCellState* state);
00041 WholeCellState* getWholeCellState() const;
00042 void setAtpYieldRate(double rate);
00043 double getAtpYieldRate() const;
00044 void setConsumptionFraction(double frac);
00045 double getConsumptionFraction() const;
00046 void setAtpPoolMax(double maxPool);
00047 double getAtpPoolMax() const;
00048 void setDeltaT(double dt);
00049 double getDeltaT() const;
00050
00051 virtual std::string show() override;
00052
00053 protected:
00054 virtual bool _loadInstance(PersistenceRecord* fields) override;
00055 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00056 virtual bool _check(std::string& errorMessage) override;
00057 virtual void _initBetweenReplications() override;
00058 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00059
00060 protected:
00061 virtual void _createEditableDataDefinitions() override;
00062

```

```

00063 private:
00064 void _forwardEntity(Entity* entity);
00065
00066 private:
00067 const struct DEFAULT_VALUES {
00068 std::string wholeCellStateName = "";
00069 // mmol ATP per fL per second -- calibrated for M. genitalium basal metabolism
00070 double atpYieldRate = 1.0e-3;
00071 double consumptionFraction = 0.1; // fraction of ATP pool consumed per step
00072 double atpPoolMax = 10.0; // mmol -- approximate saturation
00073 double deltaT = 60.0; // s
00074 } DEFAULT;
00075
00076 WholeCellState* _wholeCellState = nullptr;
00077 double _atpYieldRate = DEFAULT.atpYieldRate;
00078 double _consumptionFraction = DEFAULT.consumptionFraction;
00079 double _atpPoolMax = DEFAULT.atpPoolMax;
00080 double _deltaT = DEFAULT.deltaT;
00081 };
00082
00083 #endif /* METABOLICSUBMODELCOMPONENT_H */

```

## 16.445 PathwayStressResponseComponent.h

```

00001 #ifndef PATHWAYSTRESSRESPONSECOMPONENT_H
00002 #define PATHWAYSTRESSRESPONSECOMPONENT_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelComponent.h"
00007 #include "kernel/simulator/Plugin.h"
00008 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00009
00010 class PathwayStressResponseComponent : public ModelComponent {
00011 public:
00012 PathwayStressResponseComponent(Model* model, std::string name = "");
00013 virtual ~PathwayStressResponseComponent() override = default;
00014
00015 public:
00016 static PluginInformation* GetPluginInformation();
00017 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00018 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00019
00020 public:
00021 void setWholeCellState(WholeCellState* state);
00022 WholeCellState* getWholeCellState() const;
00023 void setMonitoredPathwayKey(std::string key);
00024 std::string getMonitoredPathwayKey() const;
00025 void setStressThreshold(double threshold);
00026 double getStressThreshold() const;
00027 void setArrestAfterSteps(unsigned int steps);
00028 unsigned int getArrestAfterSteps() const;
00029 void setDeathAfterSteps(unsigned int steps);
00030 unsigned int getDeathAfterSteps() const;
00031 void setArrestPhase(std::string phase);
00032 std::string getArrestPhase() const;
00033 void setDeadPhase(std::string phase);
00034 std::string getDeadPhase() const;
00035 void setRecoveryPhase(std::string phase);
00036 std::string getRecoveryPhase() const;
00037 void setResetStreakOnRecovery(bool reset);
00038 bool getResetStreakOnRecovery() const;
00039 unsigned int getStressStreak() const;
00040 double getLastObservedActivity() const;
00041
00042 virtual std::string show() override;
00043
00044 protected:
00045 virtual bool _loadInstance(PersistenceRecord* fields) override;
00046 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00047 virtual bool _check(std::string& errorMessage) override;
00048 virtual void _initBetweenReplications() override;
00049 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00050 virtual void _createEditableDataDefinitions() override;
00051
00052 private:
00053 void _forwardEntity(Entity* entity);
00054
00055 private:
00056 const struct DEFAULT_VALUES {
00057 std::string wholeCellStateName = "";
00058 std::string monitoredPathwayKey = "biomass_objective";
00059 double stressThreshold = 0.0;
00060 unsigned int arrestAfterSteps = 0u;
00061 unsigned int deathAfterSteps = 0u;
00062 } DEFAULT;

```

```

00078 std::string arrestPhase = "arrested";
00079 std::string deadPhase = "dead";
00080 std::string recoveryPhase = "growth";
00081 bool resetStreakOnRecovery = true;
00082 unsigned int stressStreak = 0u;
00083 double lastObservedActivity = 0.0;
00084 } DEFAULT;
00085
00086 WholeCellState* _wholeCellState = nullptr;
00087 std::string _monitoredPathwayKey = DEFAULT.monitoredPathwayKey;
00088 double _stressThreshold = DEFAULT.stressThreshold;
00089 unsigned int _arrestAfterSteps = DEFAULT.arrestAfterSteps;
00090 unsigned int _deathAfterSteps = DEFAULT.deathAfterSteps;
00091 std::string _arrestPhase = DEFAULT.arrestPhase;
00092 std::string _deadPhase = DEFAULT.deadPhase;
00093 std::string _recoveryPhase = DEFAULT.recoveryPhase;
00094 bool _resetStreakOnRecovery = DEFAULT.resetStreakOnRecovery;
00095 unsigned int _stressStreak = DEFAULT.stressStreak;
00096 double _lastObservedActivity = DEFAULT.lastObservedActivity;
00097 };
00098
00099 #endif /* PATHWAYSTRESSRESPONSECOMPONENT_H */

```

## 16.446 ResourceAllocationComponent.h

```

00001 #ifndef RESOURCEALLOCATIONCOMPONENT_H
00002 #define RESOURCEALLOCATIONCOMPONENT_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelComponent.h"
00007 #include "kernel/simulator/Plugin.h"
00008 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00009
00031 class ResourceAllocationComponent : public ModelComponent {
00032 public:
00033 ResourceAllocationComponent(Model* model, std::string name = "");
00034 virtual ~ResourceAllocationComponent() override = default;
00035
00036 public:
00037 static PluginInformation* GetPluginInformation();
00038 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00039 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00040
00041 public:
00042 void setWholeCellState(WholeCellState* state);
00043 WholeCellState* getWholeCellState() const;
00044 void setRnapCountKey(std::string key);
00045 std::string getRnapCountKey() const;
00046 void setRibosomeCountKey(std::string key);
00047 std::string getRibosomeCountKey() const;
00048 void setMRNASpeciesPrefix(std::string prefix);
00049 std::string getMRNASpeciesPrefix() const;
00050 void setProteinSpeciesPrefix(std::string prefix);
00051 std::string getProteinSpeciesPrefix() const;
00052
00053 virtual std::string show() override;
00054
00055 protected:
00056 virtual bool _loadInstance(PersistenceRecord* fields) override;
00057 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00058 virtual bool _check(std::string& errorMessage) override;
00059 virtual void _initBetweenReplications() override;
00060 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00061
00062 protected:
00063 virtual void _createEditableDataDefinitions() override;
00064
00065 private:
00066 void _forwardEntity(Entity* entity);
00067
00068 private:
00069 const struct DEFAULT_VALUES {
00070 std::string wholeCellStateName = "";
00071 std::string rnapCountKey = "RNAP_free";
00072 std::string ribosomeCountKey = "ribosome_free";
00073 std::string mrnasSpeciesPrefix = "mRNA_";
00074 std::string proteinSpeciesPrefix = "protein_";
00075 } DEFAULT;
00076
00077 WholeCellState* _wholeCellState = nullptr;
00078 std::string _rnapCountKey = DEFAULT.rnapCountKey;
00079 std::string _ribosomeCountKey = DEFAULT.ribosomeCountKey;
00080 std::string _mrnasSpeciesPrefix = DEFAULT.mrnasSpeciesPrefix;
00081 std::string _proteinSpeciesPrefix = DEFAULT.proteinSpeciesPrefix;

```

```

00082 };
00083
00084 #endif /* RESOURCEALLOCATIONCOMPONENT_H */

```

## 16.447 StochasticReactionComponent.h

```

00001 #ifndef STOCHASTICREACTIONCOMPONENT_H
00002 #define STOCHASTICREACTIONCOMPONENT_H
00003
00004 #include <map>
00005 #include <random>
00006 #include <string>
00007 #include <vector>
00008
00009 #include "../kernel/simulator/model/ModelComponent.h"
00010 #include "kernel/simulator/Plugin.h"
00011 #include "plugins/data/WholeCellModeling/StochasticReactionRule.h"
00012 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00013
00032 class StochasticReactionComponent : public ModelComponent {
00033 public:
00034 StochasticReactionComponent(Model* model, std::string name = "");
00035 virtual ~StochasticReactionComponent() override = default;
00036
00037 public:
00038 static PluginInformation* GetPluginInformation();
00039 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00040 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00041
00042 public:
00043 void setWholeCellState(WholeCellState* state);
00044 WholeCellState* getWholeCellState() const;
00045 void setTimeWindow(double timeWindow);
00046 double getTimeWindow() const;
00047 void setAdvanceWholeCellClock(bool advance);
00048 bool getAdvanceWholeCellClock() const;
00049 void setRandomSeed(unsigned int seed);
00050 unsigned int getRandomSeed() const;
00051 int getLastReactionCount() const;
00052 double getLastSimulatedTime() const;
00053
00054 virtual std::string show() override;
00055
00056 protected:
00057 virtual bool _loadInstance(PersistenceRecord* fields) override;
00058 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00059 virtual bool _check(std::string& errorMessage) override;
00060 virtual void _initBetweenReplications() override;
00061 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00062
00063 protected:
00064 virtual void _createEditableDataDefinitions() override;
00065
00066 private:
00067 struct GillespieState {
00068 std::map<std::string, int> counts;
00069 double simulatedTime = 0.0;
00070 int reactionCount = 0;
00071 };
00072
00073 GillespieState _runGillespie(
00074 const std::map<std::string, int>& initialCounts,
00075 const std::vector<StochasticReactionRule*>& rules,
00076 const std::map<std::string, int>& resourceBudget,
00077 double timeWindow);
00078
00079 void _collectCountsFromState(std::map<std::string, int>& counts) const;
00080 void _applyCountsToState(const std::map<std::string, int>& counts);
00081 std::vector<StochasticReactionRule*> _collectRules() const;
00082
00083 void _forwardEntity(Entity* entity);
00084
00085 private:
00086 const struct DEFAULT_VALUES {
00087 std::string wholeCellStateName = "";
00088 double timeWindow = 1.0;
00089 bool advanceWholeCellClock = true;
00090 unsigned int randomSeed = 42u;
00091 } DEFAULT;
00092
00093 WholeCellState* _wholeCellState = nullptr;
00094 double _timeWindow = DEFAULT.timeWindow;
00095 bool _advanceWholeCellClock = DEFAULT.advanceWholeCellClock;
00096 unsigned int _randomSeed = DEFAULT.randomSeed;
00097 int _lastReactionCount = 0;

```



```

00098 double _lastSimulatedTime = 0.0;
00099
00100 std::mt19937 _rng{DEFAULT.randomSeed};
00101 };
00102
00103 #endif /* STOCHASTICREACTIONCOMPONENT_H */

```

## 16.448 StochasticTranscription.h

```

00001 #ifndef STOCHASTICTRANSSCRIPTION_H
00002 #define STOCHASTICTRANSSCRIPTION_H
00003
00004 #include <random>
00005 #include <string>
00006
00007 #include "../kernel/simulator/model/ModelComponent.h"
00008 #include "kernel/simulator/Plugin.h"
00009 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00010
00031 class StochasticTranscription : public ModelComponent {
00032 public:
00033 StochasticTranscription(Model* model, std::string name = "");
00034 virtual ~StochasticTranscription() override = default;
00035
00036 public:
00037 static PluginInformation* GetPluginInformation();
00038 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00039 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00040
00041 public:
00042 void setWholeCellState(WholeCellState* state);
00043 WholeCellState* getWholeCellState() const;
00044 void setElongationRate(double rate);
00045 double getElongationRate() const;
00046 void setMeanGeneLength(double length);
00047 double getMeanGeneLength() const;
00048 void setBindingProbability(double prob);
00049 double getBindingProbability() const;
00050 void setTimeWindow(double window);
00051 double getTimeWindow() const;
00052 void setMRNASpeciesPrefix(std::string prefix);
00053 std::string getMRNASpeciesPrefix() const;
00054 void setRnapCountKey(std::string key);
00055 std::string getRnapCountKey() const;
00056 void setRandomSeed(unsigned int seed);
00057 unsigned int getRandomSeed() const;
00058 int getLastSynthesizedCount() const;
00059
00060 virtual std::string show() override;
00061
00062 protected:
00063 virtual bool _loadInstance(PersistenceRecord* fields) override;
00064 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00065 virtual bool _check(std::string& errorMessage) override;
00066 virtual void _initBetweenReplications() override;
00067 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00068
00069 protected:
00070 virtual void _createEditableDataDefinitions() override;
00071
00072 private:
00073 void _forwardEntity(Entity* entity);
00074
00075 private:
00076 const struct DEFAULT_VALUES {
00077 std::string wholeCellStateName = "";
00078 double elongationRate = 50.0; // nt/s -- M. genitalium RNA pol (parameters.json)
00079 double meanGeneLength = 900.0; // nt -- approximate M. genitalium average
00080 double bindingProbability = 1.0; // dimensionless; scale to calibrate output
00081 double timeWindow = 1.0; // s
00082 std::string mrnaSpeciesPrefix = "mRNA_";
00083 std::string rnapCountKey = "RNAP_free";
00084 unsigned int randomSeed = 42u;
00085 } DEFAULT;
00086
00087 WholeCellState* _wholeCellState = nullptr;
00088 double _elongationRate = DEFAULT.elongationRate;
00089 double _meanGeneLength = DEFAULT.meanGeneLength;
00090 double _bindingProbability = DEFAULT.bindingProbability;
00091 double _timeWindow = DEFAULT.timeWindow;
00092 std::string _mrnaSpeciesPrefix = DEFAULT.mrnaSpeciesPrefix;
00093 std::string _rnapCountKey = DEFAULT.rnapCountKey;
00094 unsigned int _randomSeed = DEFAULT.randomSeed;
00095 int _lastSynthesizedCount = 0;
00096 std::mt19937 _rng{DEFAULT.randomSeed};

```



```

00097 };
00098
00099 #endif /* STOCHASTICTRANSSCRIPTION_H */

```

## 16.449 StochasticTranslation.h

```

00001 #ifndef STOCHASTICTRANSLATION_H
00002 #define STOCHASTICTRANSLATION_H
00003
00004 #include <random>
00005 #include <string>
00006
00007 #include "../kernel/simulator/model/ModelComponent.h"
00008 #include "kernel/simulator/Plugin.h"
00009 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00010
00029 class StochasticTranslation : public ModelComponent {
00030 public:
00031 StochasticTranslation(Model* model, std::string name = "");
00032 virtual ~StochasticTranslation() override = default;
00033
00034 public:
00035 static PluginInformation* GetPluginInformation();
00036 static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00037 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00038
00039 public:
00040 void setWholeCellState(WholeCellState* state);
00041 WholeCellState* getWholeCellState() const;
00042 void setElongationRate(double rate);
00043 double getElongationRate() const;
00044 void setMeanProteinLength(double length);
00045 double getMeanProteinLength() const;
00046 void setTimeWindow(double window);
00047 double getTimeWindow() const;
00048 void setMRNASpeciesPrefix(std::string prefix);
00049 std::string getMRNASpeciesPrefix() const;
00050 void setProteinSpeciesPrefix(std::string prefix);
00051 std::string getProteinSpeciesPrefix() const;
00052 void setRibosomeCountKey(std::string key);
00053 std::string getRibosomeCountKey() const;
00054 void setRandomSeed(unsigned int seed);
00055 unsigned int getRandomSeed() const;
00056 int getLastSynthesizedCount() const;
00057
00058 virtual std::string show() override;
00059
00060 protected:
00061 virtual bool _loadInstance(PersistenceRecord* fields) override;
00062 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00063 virtual bool _check(std::string& errorMessage) override;
00064 virtual void _initBetweenReplications() override;
00065 virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00066
00067 protected:
00068 virtual void _createEditableDataDefinitions() override;
00069
00070 private:
00071 void _forwardEntity(Entity* entity);
00072
00073 private:
00074 const struct DEFAULT_VALUES {
00075 std::string wholeCellStateName = "";
00076 double elongationRate = 16.0; // AA/s -- M. genitalium ribosome (parameters.json)
00077 double meanProteinLength = 300.0; // AA -- approximate M. genitalium average
00078 double timeWindow = 1.0; // s
00079 std::string mrNASpeciesPrefix = "mRNA_";
00080 std::string proteinSpeciesPrefix = "prot_";
00081 std::string ribosomeCountKey = "ribosome_free";
00082 unsigned int randomSeed = 43u;
00083 } DEFAULT;
00084
00085 WholeCellState* _wholeCellState = nullptr;
00086 double _elongationRate = DEFAULT.elongationRate;
00087 double _meanProteinLength = DEFAULT.meanProteinLength;
00088 double _timeWindow = DEFAULT.timeWindow;
00089 std::string _mrNASpeciesPrefix = DEFAULT.mrNASpeciesPrefix;
00090 std::string _proteinSpeciesPrefix = DEFAULT.proteinSpeciesPrefix;
00091 std::string _ribosomeCountKey = DEFAULT.ribosomeCountKey;
00092 unsigned int _randomSeed = DEFAULT.randomSeed;
00093 int _lastSynthesizedCount = 0;
00094 std::mt19937 _rng(DEFAULT.randomSeed);
00095 };
00096
00097 #endif /* STOCHASTICTRANSLATION_H */

```

## 16.450 AISupport.h

```

00001 #ifndef AISUPPORT_H
00002 #define AISUPPORT_H
00003
00004 #include "kernel/simulator/model/ModelDataDefinition.h"
00005 #include "kernel/simulator/PluginInformation.h"
00006 #include "tools/AIAssistant/AIConversationService.h"
00007
00008 class AISupport : public ModelDataDefinition {
00009 public:
00010 AISupport(Model* model, std::string name = "");
00011 virtual ~AISupport() override = default;
00012
00013 static PluginInformation* GetPluginInformation();
00014 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00015 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00016
00017 AIConversationResponse converse(const std::string& conversationKey, const std::string& prompt);
00018 void clearConversation(const std::string& conversationKey);
00019 void clearAllConversations();
00020
00021 void setProvider(std::string value);
00022 std::string getProvider() const;
00023 void setModel(std::string value);
00024 std::string getModel() const;
00025 void setBaseUrl(std::string value);
00026 std::string getBaseUrl() const;
00027 void setApiKeyEnvironmentVariable(std::string value);
00028 std::string getApiKeyEnvironmentVariable() const;
00029 void setSystemInstructions(std::string value);
00030 std::string getSystemInstructions() const;
00031 void setTemperature(double value);
00032 double getTemperature() const;
00033 void setMaxOutputTokens(unsigned int value);
00034 unsigned int getMaxOutputTokens() const;
00035 void setTimeoutSeconds(unsigned int value);
00036 unsigned int getTimeoutSeconds() const;
00037 void setHighReasoningMode(bool value);
00038 bool isHighReasoningMode() const;
00039 void setConversationEnabled(bool value);
00040 bool isConversationEnabled() const;
00041 void setMaxHistoryMessages(unsigned int value);
00042 unsigned int getMaxHistoryMessages() const;
00043 void setClearHistoryBetweenReplications(bool value);
00044 bool isClearHistoryBetweenReplications() const;
00045 unsigned long getCallCount() const;
00046 unsigned long getFailureCount() const;
00047
00048 virtual std::string show() override;
00049
00050 protected:
00051 virtual bool _loadInstance(PersistenceRecord* fields) override;
00052 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00053 virtual bool _check(std::string& errorMessage) override;
00054 virtual void _initBetweenReplications() override;
00055
00056 private:
00057 void _configurationChanged();
00058 AIConversationConfiguration _makeConfiguration() const;
00059
00060 const struct DEFAULT_VALUES {
00061 std::string provider = "Local";
00062 std::string model = "llama3.2";
00063 std::string baseUrl = "";
00064 std::string apiKeyEnvironmentVariable = "";
00065 std::string systemInstructions = "You are a decision service inside a GenESys simulation.
Return only valid JSON.";
00066 double temperature = 0.2;
00067 unsigned int maxOutputTokens = 512;
00068 unsigned int timeoutSeconds = 120;
00069 bool highReasoningMode = false;
00070 bool conversationEnabled = true;
00071 unsigned int maxHistoryMessages = 20;
00072 bool clearHistoryBetweenReplications = true;
00073 } DEFAULT;
00074
00075 std::string _provider = DEFAULT.provider;
00076 std::string _model = DEFAULT.model;
00077 std::string _baseUrl = DEFAULT.baseUrl;
00078 std::string _apiKeyEnvironmentVariable = DEFAULT.apiKeyEnvironmentVariable;
00079 std::string _systemInstructions = DEFAULT.systemInstructions;
00080 double _temperature = DEFAULT.temperature;
00081 unsigned int _maxOutputTokens = DEFAULT.maxOutputTokens;
00082 unsigned int _timeoutSeconds = DEFAULT.timeoutSeconds;
00083 bool _highReasoningMode = DEFAULT.highReasoningMode;
00084 bool _conversationEnabled = DEFAULT.conversationEnabled;

```

```

00085 unsigned int _maxHistoryMessages = DEFAULT.maxHistoryMessages;
00086 bool _clearHistoryBetweenReplications = DEFAULT.clearHistoryBetweenReplications;
00087 unsigned long _callCount = 0;
00088 unsigned long _failureCount = 0;
00089 AIConversationService _service;
00090 };
00091
00092 #endif

```

## 16.451 BacteriaSignalGrid.h

```

00001 /*
00002 * File: BacteriaSignalGrid.h
00003 * Author: rlcancian
00004 *
00005 * Created on 28 de Abril de 2022
00006 */
00007
00008 #ifndef BACTERIASIGNALGRID_H
00009 #define BACTERIASIGNALGRID_H
00010
00011 #include "../kernel/simulator/model/ModelDataDefinition.h"
00012 #include "kernel/simulator/Plugin.h"
00013
00014 #include <string>
00015 #include <vector>
00016
00024 class BacteriaSignalGrid : public ModelDataDefinition {
00025 public:
00026 BacteriaSignalGrid(Model* model, std::string name = "");
00027 virtual ~BacteriaSignalGrid() override = default;
00028
00029 public:
00030 virtual std::string show() override;
00031
00032 public: // static plugin interface
00033 static PluginInformation* GetPluginInformation();
00034 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00035 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00036
00037 public:
00038 void setWidth(unsigned int width);
00039 unsigned int getWidth() const;
00040 void setHeight(unsigned int height);
00041 unsigned int getHeight() const;
00042 void setInitialSignal(double initialSignal);
00043 double getInitialSignal() const;
00044 void setDiffusionRate(double diffusionRate);
00045 double getDiffusionRate() const;
00046 void setDecayRate(double decayRate);
00047 double getDecayRate() const;
00048 void setInitialValues(std::string initialValues);
00049 std::string getInitialValues() const;
00050 unsigned int getCellCount() const;
00051 bool buildInitialField(std::vector<double>& values, std::string& errorMessage) const;
00052
00053 protected:
00054 virtual bool _loadInstance(PersistenceRecord* fields) override;
00055 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00056 virtual bool _check(std::string& errorMessage) override;
00057
00058 protected:
00059 // virtual void _createInternalStatisticReporters() override;
00060 // virtual void _createNonEditableDataDefinitions() override;
00061 // virtual void _createEditableDataDefinitions() override;
00062 // virtual void _createAttachedAttributes() override;
00063
00064 private:
00065 const struct DEFAULT_VALUES {
00066 const unsigned int width = 32;
00067 const unsigned int height = 32;
00068 const double initialSignal = 0.0;
00069 const double diffusionRate = 0.18;
00070 const double decayRate = 0.02;
00071 const std::string initialValues = "";
00072 } DEFAULT;
00073
00074 unsigned int _width = DEFAULT.width;
00075 unsigned int _height = DEFAULT.height;
00076 double _initialSignal = DEFAULT.initialSignal;
00077 double _diffusionRate = DEFAULT.diffusionRate;
00078 double _decayRate = DEFAULT.decayRate;
00079 std::string _initialValues = DEFAULT.initialValues;
00080 };
00081

```

```
00082 #endif /* BACTERIASIGNALGRID_H */
```

## 16.452 BioNetwork.h

```
00001 #ifndef BIONETWORK_H
00002 #define BIONETWORK_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 #include "../kernel/simulator/model/ModelDataDefinition.h"
00008 #include "kernel/simulator/PluginInformation.h"
00009 #include "tools/Biochemical/BioSimulationAnalysis.h"
00010 #include "tools/Biochemical/BioSimulationResult.h"
00011
00012 class BioReaction;
00013 class BioSpecies;
00014 class MassActionOdeSystem;
00015
00025 class BioNetwork : public ModelDataDefinition {
00026 public:
00027 BioNetwork(Model* model, std::string name = "");
00028 virtual ~BioNetwork() override = default;
00029
00030 public:
00031 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00032 static PluginInformation* GetPluginInformation();
00033 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00034
00035 public:
00036 void setStartTime(double startTime);
00037 double getStartTime() const;
00038 void setStopTime(double stopTime);
00039 double getStopTime() const;
00040 void setStepSize(double stepSize);
00041 double getStepSize() const;
00042 void setCurrentTime(double currentTime);
00043 double getCurrentTime() const;
00044 void setAutoSchedule(bool autoSchedule);
00045 bool getAutoSchedule() const;
00046 void setLastStatus(std::string lastStatus);
00047 std::string getLastStatus() const;
00048 void setLastErrorMessage(std::string lastErrorMessage);
00049 std::string getLastErrorMessage() const;
00050 void setLastResponsePayload(std::string lastResponsePayload);
00051 std::string getLastResponsePayload() const;
00052 void addSpecies(std::string speciesName);
00053 void addReaction(std::string reactionName);
00054 void clearSpecies();
00055 void clearReactions();
00056 const std::vector<std::string>& getSpeciesNames() const;
00057 const std::vector<std::string>& getReactionNames() const;
00058 const BioSimulationResult& getLastSimulationResult() const;
00059 bool getSpeciesTimeCourseDataset(const std::string& speciesName, SimulationResultsDataset*
dataset,
 std::string* errorMessage = nullptr) const;
00060 bool buildOdeSystemForAnalysis(MassActionOdeSystem* system, std::string& errorMessage) const;
00061 bool getStoichiometryMatrix(BioStoichiometryMatrix* matrix, std::string* errorMessage = nullptr)
const;
00062 bool getReactionRateTimeCourse(BioReactionRateTimeCourse* timeCourse, std::string* errorMessage =
nullptr) const;
00063 bool checkLastSampleSteadyState(double tolerance, BioSteadyStateCheck* check, std::string*
errorMessage = nullptr) const;
00064 bool scanLocalParameterSensitivity(double relativeStep, double absoluteStep, BioSensitivityScan*
scan,
 std::string* errorMessage = nullptr) const;
00065
00066 bool simulate(std::string& errorMessage);
00067 bool simulate(double startTime, double stopTime, double stepSize, std::string& errorMessage);
00068 bool advanceOneStep(std::string& errorMessage);
00069
00070 virtual std::string show() override;
00071
00072 protected:
00073 virtual bool _loadInstance(PersistenceRecord* fields) override;
00074 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00075 virtual bool _check(std::string& errorMessage) override;
00076 virtual void _initBetweenReplications() override;
00077
00078 protected:
00079 // virtual void _createInternalStatisticReporters() override;
00080 // virtual void _createNonEditableDataDefinitions() override;
00081 virtual void _createEditableDataDefinitions() override;
00082 // virtual void _createAttachedAttributes() override;
```

```

00093
00094 private:
00095 void handleInternalEvent(void* parameter);
00096 void scheduleNextInternalEvent();
00097 bool collectSpecies(std::vector<BioSpecies*>& species, std::string& errorMessage) const;
00098 bool collectReactions(std::vector<BioReaction*>& reactions, std::string& errorMessage) const;
00099 bool buildSystem(const std::vector<BioSpecies*>& species, const std::vector<BioReaction*>&
reactions, MassActionOdeSystem* system, std::string& errorMessage) const;
00100 void updatePayload(const std::vector<BioSpecies*>& species);
00101 void configureSimulationResult(const std::vector<BioSpecies*>& species);
00102 void appendSimulationResultSample(const std::vector<BioSpecies*>& species);
00103
00104 private:
00105 const struct DEFAULT_VALUES {
00106 double startTime = 0.0;
00107 double stopTime = 1.0;
00108 double stepSize = 0.1;
00109 double currentTime = 0.0;
00110 bool autoSchedule = false;
00111 std::string lastStatus = "Idle";
00112 std::string lastErrorMessage = "";
00113 std::string lastResponsePayload = "";
00114 } DEFAULT;
00115
00116 double _startTime = DEFAULT.startTime;
00117 double _stopTime = DEFAULT.stopTime;
00118 double _stepSize = DEFAULT.stepSize;
00119 double _currentTime = DEFAULT.currentTime;
00120 bool _autoSchedule = DEFAULT.autoSchedule;
00121 std::string _lastStatus = DEFAULT.lastStatus;
00122 std::string _lastErrorMessage = DEFAULT.lastErrorMessage;
00123 std::string _lastResponsePayload = DEFAULT.lastResponsePayload;
00124 std::vector<std::string> _speciesNames;
00125 std::vector<std::string> _reactionNames;
00126 BioSimulationResult _lastSimulationResult;
00127 };
00128
00129 #endif /* BIONETWORK_H */

```

## 16.453 BioParameter.h

```

00001 #ifndef BIOPARAMETER_H
00002 #define BIOPARAMETER_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelDataDefinition.h"
00007 #include "kernel/simulator/PluginInformation.h"
00008
00016 class BioParameter : public ModelDataDefinition {
00017 public:
00018 BioParameter(Model* model, std::string name = "");
00019 virtual ~BioParameter() override = default;
00020
00021 public:
00022 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00023 static PluginInformation* GetPluginInformation();
00024 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00025
00026 public:
00027 void setValue(double value);
00028 double getValue() const;
00029 void setUnit(std::string unit);
00030 std::string getUnit() const;
00031
00032 virtual std::string show() override;
00033
00034 protected:
00035 virtual bool _loadInstance(PersistenceRecord *fields) override;
00036 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00037 virtual bool _check(std::string& errorMessage) override;
00038
00039 protected:
00040 // virtual void _createInternalStatisticReporters() override;
00041 // virtual void _createNonEditableDataDefinitions() override;
00042 // virtual void _createEditableDataDefinitions() override;
00043 // virtual void _createAttachedAttributes() override;
00044
00046 private:
00047 const struct DEFAULT_VALUES {
00048 double value = 0.0;
00049 std::string unit = "";
00050 } DEFAULT;
00051

```

```

00052 double _value = DEFAULT.value;
00053 std::string _unit = DEFAULT.unit;
00054 };
00055
00056 #endif /* BIOPARAMETER_H */

```

## 16.454 BioReaction.h

```

00001 #ifndef BIOREACTION_H
00002 #define BIOREACTION_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 #include "../kernel/simulator/model/ModelDataDefinition.h"
00008 #include "kernel/simulator/PluginInformation.h"
00009
00023 class BioReaction : public ModelDataDefinition {
00024 public:
00025 struct StoichiometricTerm {
00026 std::string speciesName;
00027 double stoichiometry = 1.0;
00028 };
00029
00030 public:
00036 BioReaction(Model* model, std::string name = "");
00040 virtual ~BioReaction() override = default;
00041
00042 public:
00049 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00054 static PluginInformation* GetPluginInformation();
00061 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00062
00063 public:
00069 void addReactant(std::string speciesName, double stoichiometry = 1.0);
00075 void addProduct(std::string speciesName, double stoichiometry = 1.0);
00080 void addModifier(std::string speciesName);
00084 void clearReactants();
00088 void clearProducts();
00092 void clearModifiers();
00097 const std::vector<StoichiometricTerm>& getReactants() const;
00102 const std::vector<StoichiometricTerm>& getProducts() const;
00107 const std::vector<std::string>& getModifiers() const;
00112 void setRateConstant(double rateConstant);
00117 double getRateConstant() const;
00122 void setRateConstantParameterName(std::string rateConstantParameterName);
00127 std::string getRateConstantParameterName() const;
00132 double resolveRateConstant() const;
00137 void setReverseRateConstant(double reverseRateConstant);
00142 double getReverseRateConstant() const;
00147 void setReverseRateConstantParameterName(std::string reverseRateConstantParameterName);
00152 std::string getReverseRateConstantParameterName() const;
00157 double resolveReverseRateConstant() const;
00162 void setKineticLawExpression(std::string kineticLawExpression);
00167 std::string getKineticLawExpression() const;
00172 void setReverseKineticLawExpression(std::string reverseKineticLawExpression);
00177 std::string getReverseKineticLawExpression() const;
00182 void setReversible(bool reversible);
00187 bool isReversible() const;
00188
00193 virtual std::string show() override;
00194
00195 protected:
00196 virtual bool _loadInstance(PersistenceRecord *fields) override;
00197 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00198 virtual bool _check(std::string& errorMessage) override;
00199 // virtual void _createInternalAndAttachedData() override;
00200
00201
00202 protected:
00203 // virtual void _createInternalStatisticReporters() override;
00204 // virtual void _createNonEditableDataDefinitions() override;
00205 // virtual void _createEditableDataDefinitions() override;
00206 virtual void _createAttachedAttributes() override;
00207
00208 private:
00209 bool checkTerms(const std::vector<StoichiometricTerm>& terms, const std::string& side,
00210 std::string& errorMessage) const;
00210 bool checkModifiers(std::string& errorMessage) const;
00211 bool validateKineticLawExpression(std::string& errorMessage) const;
00212 bool validateKineticLawExpression(const std::string& expression, const std::string& label,
00213 std::string& errorMessage) const;
00213 bool hasParticipantSpecies(const std::string& speciesName) const;
00214 bool resolveKineticLawSymbol(const std::string& symbolName, double& value) const;
00215 void syncParticipantAttachedSpecies();

```

```

00216
00217 private:
00218 const struct DEFAULT_VALUES {
00219 double rateConstant = 0.0;
00220 std::string rateConstantParameterName = "";
00221 double reverseRateConstant = 0.0;
00222 std::string reverseRateConstantParameterName = "";
00223 std::string kineticLawExpression = "";
00224 std::string reverseKineticLawExpression = "";
00225 bool reversible = false;
00226 } DEFAULT;
00227
00228 std::vector<StoichiometricTerm> _reactants;
00229 std::vector<StoichiometricTerm> _products;
00230 std::vector<std::string> _modifiers;
00231 double _rateConstant = DEFAULT.rateConstant;
00232 std::string _rateConstantParameterName = DEFAULT.rateConstantParameterName;
00233 double _reverseRateConstant = DEFAULT.reverseRateConstant;
00234 std::string _reverseRateConstantParameterName = DEFAULT.reverseRateConstantParameterName;
00235 std::string _kineticLawExpression = DEFAULT.kineticLawExpression;
00236 std::string _reverseKineticLawExpression = DEFAULT.reverseKineticLawExpression;
00237 bool _reversible = DEFAULT.reversible;
00238 };
00239
00240 #endif /* BIOREACTION_H */

```

## 16.455 BioSBMLBridge.h

```

00001 #ifndef BIOSBMLBRIDGE_H
00002 #define BIOSBMLBRIDGE_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 class Model;
00008
00009 struct BioSBMLImportResult {
00010 std::string networkName;
00011 unsigned int speciesImported = 0;
00012 unsigned int parametersImported = 0;
00013 unsigned int reactionsImported = 0;
00014 std::vector<std::string> warnings;
00015 };
00016
00017 struct BioSBMLExportResult {
00018 std::string networkName;
00019 unsigned int speciesExported = 0;
00020 unsigned int parametersExported = 0;
00021 unsigned int reactionsExported = 0;
00022 std::vector<std::string> warnings;
00023 };
00024
00025 class BioSBMLBridge {
00026 public:
00027 static bool importFromString(Model* model,
00028 const std::string& sbmlText,
00029 const std::string& preferredNetworkName,
00030 BioSBMLImportResult* result,
00031 std::string& errorMessage);
00032
00033 static bool exportToString(Model* model,
00034 const std::string& requestedNetworkName,
00035 std::string* sbmlText,
00036 BioSBMLExportResult* result,
00037 std::string& errorMessage);
00038 };
00039
00040 #endif /* BIOSBMLBRIDGE_H */

```

## 16.456 BioSimulatorRunner.h

```

00001 /*
00002 * File: BioSimulatorRunner.h
00003 * Author: rlcancian
00004 *
00005 * Structural biochemical simulator runner data definition.
00006 */
00007
00008 #ifndef BIOSIMULATORRUNNER_H
00009 #define BIOSIMULATORRUNNER_H
00010
00011 #include <string>

```

```

00012
00013 #include "../kernel/simulator/model/ModelDataDefinition.h"
00014 #include "kernel/simulator/PluginInformation.h"
00015
00016 class BioSimulatorRunner : public ModelDataDefinition {
00017 public:
00018 BioSimulatorRunner(Model* model, std::string name = "");
00019 virtual ~BioSimulatorRunner() = default;
00020
00021 public:
00022 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00023 static PluginInformation* GetPluginInformation();
00024 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00025
00026 public:
00027 void setBackend(std::string backend);
00028 std::string getBackend() const;
00029 void setModelSourceType(std::string modelSourceType);
00030 std::string getModelSourceType() const;
00031 void setModelSource(std::string modelSource);
00032 std::string getModelSource() const;
00033 void setCommand(std::string command);
00034 std::string getCommand() const;
00035 void setLastStatus(std::string lastStatus);
00036 std::string getLastStatus() const;
00037 void setLastErrorMessage(std::string lastErrorMessage);
00038 std::string getLastErrorMessage() const;
00039 void setLastResponsePayload(std::string lastResponsePayload);
00040 std::string getLastResponsePayload() const;
00041 void setLastResponseFilename(std::string lastResponseFilename);
00042 std::string getLastResponseFilename() const;
00043 void setTargetBioNetworkName(std::string targetBioNetworkName);
00044 std::string getTargetBioNetworkName() const;
00045 void setWorkingDirectory(std::string workingDirectory);
00046 std::string getWorkingDirectory() const;
00047 void setWorkingInputFilename(std::string workingInputFilename);
00048 std::string getWorkingInputFilename() const;
00049 void setWorkingOutputFilename(std::string workingOutputFilename);
00050 std::string getWorkingOutputFilename() const;
00051 void setEndpointOrLibrary(std::string endpointOrLibrary);
00052 std::string getEndpointOrLibrary() const;
00053 void setTimeoutSeconds(unsigned int timeoutSeconds);
00054 unsigned int getTimeoutSeconds() const;
00055 void setAutoValidateModel(bool autoValidateModel);
00056 bool getAutoValidateModel() const;
00057
00058 bool executeCommand(std::string& errorMessage);
00059
00060 virtual std::string show() override;
00061
00062 protected:
00063 virtual bool _loadInstance(PersistenceRecord *fields) override;
00064 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00065 virtual bool _check(std::string& errorMessage) override;
00066 // virtual void _createInternalAndAttachedData() override;
00067
00068
00069 protected:
00070 // virtual void _createInternalStatisticReporters() override;
00071 // virtual void _createNonEditableDataDefinitions() override;
00072 // virtual void _createEditableDataDefinitions() override;
00073 // virtual void _createAttachedAttributes() override;
00074
00075 private:
00076 const struct DEFAULT_VALUES {
00077 std::string backend = "RoadRunnerEmbedded";
00078 std::string modelSourceType = "SBMLString";
00079 std::string modelSource = "";
00080 std::string command = "";
00081 std::string lastStatus = "Idle";
00082 std::string lastErrorMessage = "";
00083 std::string lastResponsePayload = "";
00084 std::string lastResponseFilename = "";
00085 std::string targetBioNetworkName = "";
00086 std::string workingDirectory = "";
00087 std::string workingInputFilename = "biosim_input.xml";
00088 std::string workingOutputFilename = "biosim_output.json";
00089 std::string endpointOrLibrary = "";
00090 unsigned int timeoutSeconds = 30;
00091 bool autoValidateModel = true;
00092 } DEFAULT;
00093
00094 std::string _backend = DEFAULT.backend;
00095 std::string _modelSourceType = DEFAULT.modelSourceType;
00096 std::string _modelSource = DEFAULT.modelSource;
00097 std::string _command = DEFAULT.command;
00098 std::string _lastStatus = DEFAULT.lastStatus;

```



```

00099 std::string _lastErrorMessage = DEFAULT.lastErrorMessage;
00100 std::string _lastResponsePayload = DEFAULT.lastResponsePayload;
00101 std::string _lastResponseFilename = DEFAULT.lastResponseFilename;
00102 std::string _targetBioNetworkName = DEFAULT.targetBioNetworkName;
00103 std::string _workingDirectory = DEFAULT.workingDirectory;
00104 std::string _workingInputFilename = DEFAULT.workingInputFilename;
00105 std::string _workingOutputFilename = DEFAULT.workingOutputFilename;
00106 std::string _endpointOrLibrary = DEFAULT.endpointOrLibrary;
00107 unsigned int _timeoutSeconds = DEFAULT.timeoutSeconds;
00108 bool _autoValidateModel = DEFAULT.autoValidateModel;
00109 };
00110
00111 #endif /* BIOSIMULATORRUNNER_H */

```

## 16.457 BioSpecies.h

```

00001 #ifndef BIOSPECIES_H
00002 #define BIOSPECIES_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelDataDefinition.h"
00007 #include "kernel/simulator/PluginInformation.h"
00008
00017 class BioSpecies : public ModelDataDefinition {
00018 public:
00019 BioSpecies(Model* model, std::string name = "");
00020 virtual ~BioSpecies() override = default;
00021
00022 public:
00023 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00024 static PluginInformation* GetPluginInformation();
00025 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00026
00027 public:
00028 void setInitialAmount(double initialAmount);
00029 double getInitialAmount() const;
00030 void setAmount(double amount);
00031 double getAmount() const;
00032 void setConstant(bool constant);
00033 bool isConstant() const;
00034 void setBoundaryCondition(bool boundaryCondition);
00035 bool isBoundaryCondition() const;
00036 void setUnit(std::string unit);
00037 std::string getUnit() const;
00038
00039 virtual std::string show() override;
00040
00041 protected:
00042 virtual bool _loadInstance(PersistenceRecord *fields) override;
00043 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00044 virtual bool _check(std::string& errorMessage) override;
00045 virtual void _initBetweenReplications() override;
00046
00047 protected:
00048 // virtual void _createInternalStatisticReporters() override;
00049 // virtual void _createNonEditableDataDefinitions() override;
00050 // virtual void _createEditableDataDefinitions() override;
00051 // virtual void _createAttachedAttributes() override;
00052
00053 private:
00054 const struct DEFAULT_VALUES {
00055 double initialAmount = 0.0;
00056 double amount = 0.0;
00057 bool constant = false;
00058 bool boundaryCondition = false;
00059 std::string unit = "";
00060 } DEFAULT;
00061
00062 double _initialAmount = DEFAULT.initialAmount;
00063 double _amount = DEFAULT.amount;
00064 bool _constant = DEFAULT.constant;
00065 bool _boundaryCondition = DEFAULT.boundaryCondition;
00066 std::string _unit = DEFAULT.unit;
00067 };
00068
00069
00070 #endif /* BIOSPECIES_H */

```

## 16.458 GeneticCircuit.h

```

00001 #ifndef GENETICCIRCUIT_H

```

```

00002 #define GENETICCIRCUIT_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 #include "../kernel/simulator/model/ModelDataDefinition.h"
00008 #include "kernel/simulator/PluginInformation.h"
00009
00017 class GeneticCircuit : public ModelDataDefinition {
00018 public:
00019 GeneticCircuit(Model* model, std::string name = "");
00020 virtual ~GeneticCircuit() override = default;
00021
00022 public:
00023 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00024 static PluginInformation* GetPluginInformation();
00025 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00026
00027 public:
00028 void addPart(std::string partName);
00029 void addRegulation(std::string regulationName);
00030 void clearParts();
00031 void clearRegulations();
00032 const std::vector<std::string>& getPartNames() const;
00033 const std::vector<std::string>& getRegulationNames() const;
00034 void setHostOrganism(std::string hostOrganism);
00035 std::string getHostOrganism() const;
00036 void setCompartment(std::string compartment);
00037 std::string getCompartment() const;
00038 void setEnabled(bool enabled);
00039 bool isEnabled() const;
00040
00041 virtual std::string show() override;
00042
00043 protected:
00044 virtual bool _loadInstance(PersistenceRecord* fields) override;
00045 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00046 virtual bool _check(std::string& errorMessage) override;
00047
00048 protected:
00049 // virtual void _createInternalStatisticReporters() override;
00050 // virtual void _createNonEditableDataDefinitions() override;
00051 // virtual void _createEditableDataDefinitions() override;
00052 // virtual void _createAttachedAttributes() override;
00053
00054 private:
00055 bool _checkPartNames(std::string& errorMessage) const;
00056 bool _checkRegulationNames(std::string& errorMessage) const;
00057
00058 private:
00059 const struct DEFAULT_VALUES {
00060 std::string hostOrganism = "";
00061 std::string compartment = "";
00062 bool enabled = true;
00063 } DEFAULT;
00064
00065 std::vector<std::string> _partNames;
00066 std::vector<std::string> _regulationNames;
00067 std::string _hostOrganism = DEFAULT.hostOrganism;
00068 std::string _compartment = DEFAULT.compartment;
00069 bool _enabled = DEFAULT.enabled;
00070 };
00071
00072 #endif /* GENETICCIRCUIT_H */

```

## 16.459 GeneticCircuitPart.h

```

00001 #ifndef GENETICCIRCUITPART_H
00002 #define GENETICCIRCUITPART_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelDataDefinition.h"
00007 #include "kernel/simulator/PluginInformation.h"
00008
00016 class GeneticCircuitPart : public ModelDataDefinition {
00017 public:
00018 GeneticCircuitPart(Model* model, std::string name = "");
00019 virtual ~GeneticCircuitPart() override = default;
00020
00021 public:
00022 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00023 static PluginInformation* GetPluginInformation();
00024 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00025

```

```

00026 public:
00027 void setPartType(std::string partType);
00028 std::string getPartType() const;
00029 void setSequence(std::string sequence);
00030 std::string getSequence() const;
00031 void setProductSpeciesName(std::string productSpeciesName);
00032 std::string getProductSpeciesName() const;
00033 void setCopyNumber(double copyNumber);
00034 double getCopyNumber() const;
00035 void setBasalExpressionRate(double basalExpressionRate);
00036 double getBasalExpressionRate() const;
00037 void setDegradationRate(double degradationRate);
00038 double getDegradationRate() const;
00039 void setEnabled(bool enabled);
00040 bool isEnabled() const;
00041
00042 virtual std::string show() override;
00043
00044 protected:
00045 virtual bool _loadInstance(PersistenceRecord* fields) override;
00046 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00047 virtual bool _check(std::string& errorMessage) override;
00048
00049 protected:
00050 // virtual void _createInternalStatisticReporters() override;
00051 // virtual void _createNonEditableDataDefinitions() override;
00052 // virtual void _createEditableDataDefinitions() override;
00053 // virtual void _createAttachedAttributes() override;
00054
00055 private:
00056 const struct DEFAULT_VALUES {
00057 std::string partType = "Promoter";
00058 std::string sequence = "";
00059 std::string productSpeciesName = "";
00060 double copyNumber = 1.0;
00061 double basalExpressionRate = 0.0;
00062 double degradationRate = 0.0;
00063 bool enabled = true;
00064 } DEFAULT;
00065
00066 std::string _partType = DEFAULT.partType;
00067 std::string _sequence = DEFAULT.sequence;
00068 std::string _productSpeciesName = DEFAULT.productSpeciesName;
00069 double _copyNumber = DEFAULT.copyNumber;
00070 double _basalExpressionRate = DEFAULT.basalExpressionRate;
00071 double _degradationRate = DEFAULT.degradationRate;
00072 bool _enabled = DEFAULT.enabled;
00073 };
00074
00075 #endif /* GENETICCIRCUITPART_H */

```

## 16.460 GeneticRegulation.h

```

00001 #ifndef GENETICREGULATION_H
00002 #define GENETICREGULATION_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelDataDefinition.h"
00007 #include "kernel/simulator/PluginInformation.h"
00008
00012 class GeneticRegulation : public ModelDataDefinition {
00013 public:
00014 GeneticRegulation(Model* model, std::string name = "");
00015 virtual ~GeneticRegulation() override = default;
00016
00017 public:
00018 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00019 static PluginInformation* GetPluginInformation();
00020 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00021
00022 public:
00023 void setRegulatorSpeciesName(std::string regulatorSpeciesName);
00024 std::string getRegulatorSpeciesName() const;
00025 void setTargetPartName(std::string targetPartName);
00026 std::string getTargetPartName() const;
00027 void setRegulationType(std::string regulationType);
00028 std::string getRegulationType() const;
00029 void setHillCoefficient(double hillCoefficient);
00030 double getHillCoefficient() const;
00031 void setDissociationConstant(double dissociationConstant);
00032 double getDissociationConstant() const;
00033 void setMaxFoldChange(double maxFoldChange);
00034 double getMaxFoldChange() const;
00035 void setLeakiness(double leakiness);

```

```

00036 double getLeakiness() const;
00037 void setEnabled(bool enabled);
00038 bool isEnabled() const;
00039
00040 virtual std::string show() override;
00041
00042 protected:
00043 virtual bool _loadInstance(PersistenceRecord* fields) override;
00044 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00045 virtual bool _check(std::string& errorMessage) override;
00046
00047 protected:
00048 // virtual void _createInternalStatisticReporters() override;
00049 // virtual void _createNonEditableDataDefinitions() override;
00050 // virtual void _createEditableDataDefinitions() override;
00051 // virtual void _createAttachedAttributes() override;
00052
00053 private:
00054 const struct DEFAULT_VALUES {
00055 std::string regulatorSpeciesName = "";
00056 std::string targetPartName = "";
00057 std::string regulationType = "Activation";
00058 double hillCoefficient = 1.0;
00059 double dissociationConstant = 1.0;
00060 double maxFoldChange = 1.0;
00061 double leakiness = 0.0;
00062 bool enabled = true;
00063 } DEFAULT;
00064
00065 std::string _regulatorSpeciesName = DEFAULT.regulatorSpeciesName;
00066 std::string _targetPartName = DEFAULT.targetPartName;
00067 std::string _regulationType = DEFAULT.regulationType;
00068 double _hillCoefficient = DEFAULT.hillCoefficient;
00069 double _dissociationConstant = DEFAULT.dissociationConstant;
00070 double _maxFoldChange = DEFAULT.maxFoldChange;
00071 double _leakiness = DEFAULT.leakiness;
00072 bool _enabled = DEFAULT.enabled;
00073 };
00074
00075 #endif /* GENETICREGULATION_H */

```

## 16.461 GroProgram.h

```

00001 /*
00002 * File: GroProgram.h
00003 * Author: rlcancian
00004 *
00005 * Created on 17 de Abril de 2022
00006 */
00007
00008 #ifndef GROPROGRAM_H
00009 #define GROPROGRAM_H
00010
00011 #include "../kernel/simulator/model/ModelDataDefinition.h"
00012 #include "kernel/simulator/Plugin.h"
00013 #include <string>
00014
00023 class GroProgram : public ModelDataDefinition {
00024 public:
00025 GroProgram(Model* model, std::string name = "");
00026 virtual ~GroProgram() override = default;
00027
00028 public:
00029 virtual std::string show() override;
00030
00031 public: // static plugin interface
00032 static PluginInformation* GetPluginInformation();
00033 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00034 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00035
00036 public:
00037 void setSourceCode(std::string sourceCode);
00038 void setSourceCodeProperty(SourceCodeString sourceCode);
00039 std::string getSourceCode() const;
00040 SourceCodeString getSourceCodeProperty() const;
00041 bool createDefaultGroProgram(const std::string& filename = "");
00042 bool validateSyntax(std::string& errorMessage) const;
00043
00044 protected:
00045 virtual bool _loadInstance(PersistenceRecord* fields) override;
00046 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00047 virtual bool _check(std::string& errorMessage) override;
00048
00049 protected:

```

```

00057 // virtual void _createInternalStatisticReporters() override;
00058 // virtual void _createNonEditableDataDefinitions() override;
00059 // virtual void _createEditableDataDefinitions() override;
00060 // virtual void _createAttachedAttributes() override;
00061
00062 private:
00063 bool _validateSourceCodeSyntax(const std::string& sourceCode, std::string& errorMessage) const;
00064
00065 private:
00066 const struct DEFAULT_VALUES {
00067 const std::string sourceCode = R"(
00068 include gro
00069 set ("dt", 0.1);
00070 set ("population_max", 2000);
00071 program p() := {
00072 skip();
00073 };
00074 program q() := {
00075 set ("ecoli_growth_rate", 0.1);
00076 set ("ecoli_division_size_mean", 2.0);
00077 set ("ecoli_division_size_variance", 0.2);
00078 };
00079 program r() := {
00080 selected : { message (1, toString (volume)) }
00081 };
00082 program s() := {
00083 set ("ecoli_division_size_mean", 1000); // essentially turns off gro's division machinery
00084 rate(1) & volume > 3.14 : {
00085 divide()
00086 }
00087 };
00088 ecoli ([x := 0, y := 0], program q()); // Try p(), q(), r() or s() here. They are all a bit
different.
00089
00090 //-----
00091 set ("dt", 0.1);
00092 set ("ecoli_growth_rate", 0.1);
00093
00094 s0 := signal (1, 0.2); // create two new signals
00095 s1 := signal (1, 0.2);
00096
00097 // These functions provide various growth rates in different states. a and b
00098 // are the intensities of signaling molecules.
00099 fun on a b . 0.1;
00100 fun off a b . 0.0;
00101 fun and a b . if a > 0.1 & b > 0.1 then 0.1 else 0.0 end;
00102
00103 //
00104 // The state program. Parameters are
00105 //
00106 // this: the state index
00107 // m_next, d_next: states to go to after division if mother or daughter cell
00108 // t_next: state to go to after timer is up
00109 // tf: timer length (use -1 for infinity)
00110 // gr: growth rate function
00111 // sigs: rates to emit signals
00112 //
00113 program state (this, m_next, d_next, t_next, tf, gr, sigs) := {
00114
00115 needs q, t;
00116 active := false;
00117 needs event;
00118
00119 true : { active := (q = this); }
00120
00121 !event & active & just_divided & !daughter & q != m_next : { q := m_next, t := 0, event := true }
00122 !event & active & just_divided & daughter & q != d_next : { q := d_next, t := 0, event := true }
00123 !event & active & tf > 0 & t > tf & q != t_next : { q := t_next, t := 0, event := true }
00124
00125 active : {
00126 set ("ecoli_growth_rate", gr (get_signal (0)) (get_signal (1))),
00127 emit_signal (0, sigs[0]),
00128 emit_signal (1, sigs[1])
00129 }
00130
00131 };
00132
00133 //
00134 // State machine program: coordinates states.
00135 //
00136 program sm() := {
00137
00138 q := 0;
00139 t := 0;
00140 event := false;
00141
00142 true : { t := t + dt, event := false }

```

```

00143
00144 };
00145
00146 //
00147 // The morphogenesis state machine example.
00148 //
00149 program p() := sm() + state (0, 1, 2, 0, -1, on, { 0, 0 }) sharing q, t, event
00150 + state (1, 1, 3, 5, 60, on, { 50, 0 }) sharing q, t, event
00151 + state (2, 2, 3, 6, 60, on, { 0, 50 }) sharing q, t, event
00152 + state (3, 3, 3, 3, -1, and, { 0, 0 }) sharing q, t, event
00153 + state (5, 5, 5, 7, 40, on, { 0, 0 }) sharing q, t, event
00154 + state (6, 6, 6, 8, 40, on, { 0, 0 }) sharing q, t, event
00155 + state (7, 7, 7, 7, 0, off, { 0, 0 }) sharing q, t, event
00156 + state (8, 8, 8, 8, 0, off, { 0, 0 }) sharing q, t, event;
00157
00158 // A helper function.
00159 fun test x y . if x = y then 1 else 0 end;
00160
00161 //
00162 // Makes cells in different states report different colors.
00163 //
00164 program report() := {
00165
00166 gfp := 0; rfp := 0; yfp := 0; cfp := 0;
00167 needs q;
00168
00169 true : {
00170 rfp := 100 * test (q) 3;
00171 yfp := 100 * (test (q) 1 + test (q) 2),
00172 cfp := 100 * (test (q) 5 + test (q) 7);
00173 gfp := 100 * (test (q) 6 + test (q) 8);
00174 }
00175
00176 selected : { message (2, toString ({ id, q })) }
00177 };
00178
00179 //
00180 // Hides the state variable in all() -- see below -- so that it is not cut in half when the
00181 // cell divides.
00182 //
00183 //
00184 program hide() := {
00185
00186 needs gfp;
00187 needs rfp;
00188 needs yfp;
00189 needs cfp;
00190
00191 skip();
00192 };
00193
00194 //
00195 // All the programs stuck together
00196 //
00197 //
00198 program all() := p() + report() sharing q, gfp, rfp, yfp, cfp + hide() sharing gfp, rfp, yfp, cfp;
00199
00200 //
00201 // The main program makes the simulation loop
00202 //
00203 program main() := {
00204
00205 t := 1;
00206
00207 true : { t := t + dt }
00208
00209 t > 120 : {
00210 reset(),
00211 ecoli ([], program all()),
00212 t := 0
00213 }
00214 };
00215
00216 // associate the program with a cell
00217 ecoli ([], program all());
00218
00219
00220
00221
00222);
00223
00224 } DEFAULT;
00225
00226 std::string _sourceCode = DEFAULT.sourceCode;
00227 };
00228
00229 #endif /* GROPROGRAM_H */

```

```

00230
00231
00232 /*
00233
00234
00235
00236 //
00237 // gro is protected by the UW OPEN SOURCE LICENSE, which is summarized here.
00238 // Please see the file LICENSE.txt for the complete license.
00239 //
00240 // THE SOFTWARE (AS DEFINED BELOW) AND HARDWARE DESIGNS (AS DEFINED BELOW) IS PROVIDED
00241 // UNDER THE TERMS OF THIS OPEN SOURCE LICENSE ("LICENSE"). THE SOFTWARE IS PROTECTED
00242 // BY COPYRIGHT AND/OR OTHER APPLICABLE LAW. ANY USE OF THIS SOFTWARE OTHER THAN AS
00243 // AUTHORIZED UNDER THIS LICENSE OR COPYRIGHT LAW IS PROHIBITED.
00244 //
00245 // BY EXERCISING ANY RIGHTS TO THE SOFTWARE AND/OR HARDWARE PROVIDED HERE, YOU ACCEPT AND
00246 // AGREE TO BE BOUND BY THE TERMS OF THIS LICENSE. TO THE EXTENT THIS LICENSE MAY BE
00247 // CONSIDERED A CONTRACT, THE UNIVERSITY OF WASHINGTON ("UW") GRANTS YOU THE RIGHTS
00248 // CONTAINED HERE IN CONSIDERATION OF YOUR ACCEPTANCE OF SUCH TERMS AND CONDITIONS.
00249 //
00250 // TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION
00251 //
00252
00253 include gro
00254
00255 set ("dt", 0.1);
00256 set ("ecoli_growth_rate", 0.1);
00257
00258 s0 := signal (1, 0.2); // create two new signals
00259 s1 := signal (1, 0.2);
00260
00261 // These functions provide various growth rates in different states. a and b
00262 // are the intensities of signaling molecules.
00263 fun on a b . 0.1;
00264 fun off a b . 0.0;
00265 fun and a b . if a > 0.1 & b > 0.1 then 0.1 else 0.0 end;
00266
00267 //
00268 // The state program. Parameters are
00269 //
00270 // this: the state index
00271 // m_next, d_next: states to go to after division if mother or daughter cell
00272 // t_next: state to go to after timer is up
00273 // tf: timer length (use -1 for infinity)
00274 // gr: growth rate function
00275 // sigs: rates to emit signals
00276 //
00277 program state (this, m_next, d_next, t_next, tf, gr, sigs) := {
00278
00279 needs q, t;
00280 active := false;
00281 needs event;
00282
00283 true : { active := (q = this); }
00284
00285 !event & active & just_divided & !daughter & q != m_next : { q := m_next, t := 0, event := true }
00286 !event & active & just_divided & daughter & q != d_next : { q := d_next, t := 0, event := true }
00287 !event & active & tf > 0 & t > tf & q != t_next : { q := t_next, t := 0, event := true }
00288
00289 active : {
00290 set ("ecoli_growth_rate", gr (get_signal (0)) (get_signal (1))),
00291 emit_signal (0, sigs[0]),
00292 emit_signal (1, sigs[1])
00293 }
00294
00295 };
00296
00297 //
00298 // State machine program: coordinates states.
00299 //
00300 program sm() := {
00301
00302 q := 0;
00303 t := 0;
00304 event := false;
00305
00306 true : { t := t + dt, event := false }
00307
00308 };
00309
00310 //
00311 // The morphogenesis state machine example.
00312 //
00313 program p() := sm() + state (0, 1, 2, 0, -1, on, { 0, 0 }) sharing q, t, event
00314 + state (1, 1, 3, 5, 60, on, { 50, 0 }) sharing q, t, event
00315 + state (2, 2, 3, 6, 60, on, { 0, 50 }) sharing q, t, event
00316 + state (3, 3, 3, 3, -1, and, { 0, 0 }) sharing q, t, event

```

```

00317 + state (5, 5, 5, 7, 40, on, { 0, 0 }) sharing q, t, event
00318 + state (6, 6, 6, 8, 40, on, { 0, 0 }) sharing q, t, event
00319 + state (7, 7, 7, 7, 0, off, { 0, 0 }) sharing q, t, event
00320 + state (8, 8, 8, 8, 0, off, { 0, 0 }) sharing q, t, event;
00321
00322 // A helper function.
00323 fun test x y . if x = y then 1 else 0 end;
00324
00325 //
00326 // Makes cells in different states report different colors.
00327 //
00328 program report() := {
00329
00330 gfp := 0; rfp := 0; yfp := 0; cfp := 0;
00331 needs q;
00332
00333 true : {
00334 rfp := 100 * test (q) 3;
00335 yfp := 100 * (test (q) 1 + test (q) 2),
00336 cfp := 100 * (test (q) 5 + test (q) 7);
00337 gfp := 100 * (test (q) 6 + test (q) 8);
00338 }
00339
00340 selected : { message (2, toString ({ id, q })) }
00341
00342 };
00343
00344 //
00345 // Hides the state variable in all() -- see below -- so that it is not cut in half when the
00346 // cell divides.
00347 //
00348 program hide() := {
00349
00350 needs gfp;
00351 needs rfp;
00352 needs yfp;
00353 needs cfp;
00354
00355 skip();
00356
00357 };
00358
00359 //
00360 // All the programs stuck together
00361 //
00362 program all() := p() + report() sharing q, gfp, rfp, yfp, cfp + hide() sharing gfp, rfp, yfp, cfp;
00363
00364 //
00365 // The main program makes the simulation loop
00366 //
00367 program main() := {
00368
00369 t := 1;
00370
00371 true : { t := t + dt }
00372
00373 t > 120 : {
00374 reset(),
00375 ecoli ([], program all()),
00376 t := 0
00377 }
00378
00379 };
00380
00381 // associate the program with a cell
00382 ecoli ([], program all());
00383 */
00384
00385 /*
00386 include gro
00387 set ("dt",0.1);
00388 s := signal (1,1.05);
00389 k0 := 2.5; // base rate of oscillation
00390 kb := 5; // cell-cell feedback strength
00391 tr := 10; // refractory period length
00392 se := 650; // signal emit magnitude
00393 GO := 0;
00394 WAIT := 1;
00395 program oscillator(g0) := {
00396 gfp := 0.5*volume*g0;
00397 p := [mode := GO, t := 0, x := g0];
00398 true : {gfp := 0.5*volume*(g0 + p.x/150)}
00399 // advance the oscillation phase
00400 p.mode = GO & rate (k0) : {p.x := p.x + 0.01*(150-p.x)}
00401 p.mode = GO & rate(kb*get_signal(s)) : {p.x := p.x + 0.01*(150-p.x)}
00402 // phase reset
00403 p.x > 100 : {

```



```

00404 p.x := 0,
00405 p.mode := WAIT,
00406 emit_signal(s, se)
00407 }
00408 // refractory period timer
00409 p.mode = WAIT : {p.t := p.t+dt}
00410 p.mode = WAIT & p.t > tr : {
00411 p.mode := GO,
00412 p.t := 0
00413 }
00414 };
00415 ecoli ([x:=0, y:=0], program oscillator(1));
00416 program main() := {
00417 //n := 0;
00418 //true : {snapshot("movie/oscillator/" <> toString(n) <> ".tif"), n:=n+1}
00419 skip();
00420 };
00421 */

```

## 16.462 GroProgramAst.h

```

00001 /*
00002 * File: GroProgramAst.h
00003 * Author: rlcancian
00004 *
00005 * Created on 17 de Abril de 2022
00006 */
00007
00008 #ifndef GROPROGRAMAST_H
00009 #define GROPROGRAMAST_H
00010
00011 #include <string>
00012 #include <vector>
00013
00021 struct GroProgramAst {
00022 enum class SourceForm {
00023 RawStatements,
00024 ProgramBlock
00025 };
00026
00027 struct Statement {
00028 std::string sourceText = "";
00029 };
00030
00031 struct NamedProgram {
00032 std::string name = "";
00033 std::vector<std::string> parameters;
00034 std::string bodySource = "";
00035 std::vector<Statement> statements;
00036 };
00037
00038 SourceForm sourceForm = SourceForm::RawStatements;
00039 std::string programName = "";
00040 std::vector<std::string> programParameters;
00041 std::string bodySource = "";
00042 std::vector<Statement> statements;
00043 std::vector<NamedProgram> namedPrograms;
00044
00045 bool isProgramBlock() const {
00046 return sourceForm == SourceForm::ProgramBlock;
00047 }
00048
00049 bool hasNamedPrograms() const {
00050 return !namedPrograms.empty();
00051 }
00052 };
00053
00054 #endif /* GROPROGRAMAST_H */

```

## 16.463 GroProgramCompiler.h

```

00001 /*
00002 * File: GroProgramCompiler.h
00003 * Author: rlcancian
00004 *
00005 * Created on 17 de Abril de 2022
00006 */
00007
00008 #ifndef GROPROGRAMCOMPILER_H
00009 #define GROPROGRAMCOMPILER_H
00010
00011 #include "plugins/data/BiochemicalSimulation/GroProgramAst.h"

```

```

00012 #include "plugins/data/BiochemicalSimulation/GroProgramIr.h"
00013
00017 class GroProgramCompiler {
00018 public:
00020 GroProgramIr compile(const GroProgramAst& ast) const;
00021 };
00022
00023 #endif /* GROPROGRAMCOMPILER_H */

```

## 16.464 GroProgramIr.h

```

00001 /*
00002 * File: GroProgramIr.h
00003 * Author: rlcancian
00004 *
00005 * Created on 17 de Abril de 2022
00006 */
00007
00008 #ifndef GROPROGRAMIR_H
00009 #define GROPROGRAMIR_H
00010
00011 #include "plugins/data/BiochemicalSimulation/GroProgramAst.h"
00012
00013 #include <map>
00014 #include <string>
00015 #include <vector>
00016
00024 struct GroProgramIr {
00025 struct Command {
00026 enum class Kind {
00027 RawStatement,
00028 FunctionCall,
00029 Assignment,
00030 IfStatement
00031 };
00032
00033 Kind kind = Kind::RawStatement;
00034 std::string sourceText = "";
00035 std::string functionName = "";
00036 std::vector<std::string> arguments;
00037 std::string assignmentTarget = "";
00038 std::string expressionText = "";
00039 bool assignmentOnlyIfUnset = false;
00040 std::vector<Command> thenCommands;
00041 std::vector<Command> elseCommands;
00042
00043 bool isFunctionCall() const {
00044 return kind == Kind::FunctionCall;
00045 }
00046
00047 bool isAssignment() const {
00048 return kind == Kind::Assignment;
00049 }
00050
00051 bool isIfStatement() const {
00052 return kind == Kind::IfStatement;
00053 }
00054 };
00055
00056 struct NamedProgramDefinition {
00057 std::vector<std::string> parameters;
00058 std::string sourceText = "";
00059 std::vector<Command> commands;
00060 };
00061
00062 GroProgramAst::SourceForm sourceForm = GroProgramAst::SourceForm::RawStatements;
00063 std::string programName = "";
00064 std::vector<Command> commands;
00065 std::map<std::string, NamedProgramDefinition> namedPrograms;
00066
00067 bool isProgramBlock() const {
00068 return sourceForm == GroProgramAst::SourceForm::ProgramBlock;
00069 }
00070
00071 bool hasNamedProgram(const std::string& name) const {
00072 return namedPrograms.find(name) != namedPrograms.end();
00073 }
00074 };
00075
00076 #endif /* GROPROGRAMIR_H */

```

## 16.465 GroProgramParser.h

```
00001 /*
00002 * File: GroProgramParser.h
00003 * Author: rlcancian
00004 *
00005 * Created on 17 de Abril de 2022
00006 */
00007
00008 #ifndef GROPROGRAMPARSER_H
00009 #define GROPROGRAMPARSER_H
00010
00011 #include "plugins/data/BiochemicalSimulation/GroProgramAst.h"
00012
00013 #include <string>
00014
00022 class GroProgramParser {
00023 public:
00024 struct Result {
00025 bool accepted = false;
00026 std::string errorMessage = "";
00027 GroProgramAst ast;
00028 };
00029
00030 public:
00032 Result parse(const std::string& sourceCode) const;
00033 };
00034
00035 #endif /* GROPROGRAMPARSER_H */
```

## 16.466 GroProgramRuntime.h

```
00001 /*
00002 * File: GroProgramRuntime.h
00003 * Author: rlcancian
00004 *
00005 * Created on 17 de Abril de 2022
00006 */
00007
00008 #ifndef GROPROGRAMRUNTIME_H
00009 #define GROPROGRAMRUNTIME_H
00010
00011 #include "plugins/data/BiochemicalSimulation/GroProgramIr.h"
00012
00013 #include <map>
00014 #include <string>
00015 #include <vector>
00016
00020 struct GroProgramRuntimeState {
00021 double colonyTime = 0.0;
00022 double simulationStep = 1.0;
00023 unsigned int populationSize = 1;
00024 unsigned int tickCount = 0;
00025 std::map<std::string, double> contextVariables;
00026 std::map<std::string, double> variables;
00027 };
00028
00037 class GroProgramRuntime {
00038 public:
00039 enum class PopulationMutationType {
00040 Grow,
00041 Divide,
00042 Die,
00043 SetPopulation
00044 };
00045
00046 enum class SignalMutationType {
00047 Emit,
00048 Consume,
00049 Set
00050 };
00051
00052 enum class MotionMutationType {
00053 Run,
00054 Tumble
00055 };
00056
00057 struct PopulationMutation {
00058 PopulationMutationType type = PopulationMutationType::Grow;
00059 unsigned int value = 0;
00060 unsigned int previousPopulationSize = 0;
00061 unsigned int resultingPopulationSize = 0;
00062 };
00063 }
```

```

00064 struct SignalMutation {
00065 SignalMutationType type = SignalMutationType::Emit;
00066 double value = 0.0;
00067 };
00068
00069 struct MotionMutation {
00070 MotionMutationType type = MotionMutationType::Run;
00071 double value = 0.0;
00072 double previousDirection = 0.0;
00073 double resultingDirection = 0.0;
00074 double previousSpeed = 0.0;
00075 double resultingSpeed = 0.0;
00076 };
00077
00078 struct ColonyMutation {
00079 enum class Type {
00080 Reset,
00081 SpawnSeed,
00082 SetChemostatMode,
00083 AddBarrier,
00084 MapToCells,
00085 SetSignalAt,
00086 SetSignalRect,
00087 SetSignalGridWidth,
00088 SetSignalGridHeight,
00089 GetSignalMatrix,
00090 DumpSignalField
00091 };
00092
00093 Type type = Type::Reset;
00094 std::vector<std::string> arguments;
00095 std::vector<double> numericArguments;
00096 std::string expressionText;
00097 std::size_t previewRows = 0;
00098 std::size_t previewColumns = 0;
00099 };
00100
00101 struct ExecutionResult {
00102 bool succeeded = true;
00103 std::string errorMessage = "";
00104 unsigned int executedCommands = 0;
00105 std::vector<PopulationMutation> populationMutations;
00106 std::vector<SignalMutation> signalMutations;
00107 std::vector<MotionMutation> motionMutations;
00108 std::vector<ColonyMutation> colonyMutations;
00109 std::vector<double> mappedCellValues;
00110 std::string mappedCellExpression;
00111 std::vector<double> signalMatrixValues;
00112 unsigned int signalMatrixWidth = 0;
00113 unsigned int signalMatrixHeight = 0;
00114 std::string signalFieldDump;
00115 std::vector<std::string> messages;
00116 std::map<std::string, double> assignedVariables;
00117 std::vector<std::string> unsupportedCommands;
00118 std::vector<std::string> skippedRawStatements;
00119 };
00120
00121 public:
00122 ExecutionResult execute(const GroProgramIr& ir, GroProgramRuntimeState& state) const;
00123 static bool evaluateExpression(const std::string& expressionText,
00124 const GroProgramRuntimeState& state,
00125 double& value,
00126 std::string& errorMessage);
00127 };
00128
00129 #endif /* GROPROGRAMRUNTIME_H */

```

## 16.467 MetabolicNetwork.h

```

00001 #ifndef METABOLICNETWORK_H
00002 #define METABOLICNETWORK_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 #include "../kernel/simulator/model/ModelDataDefinition.h"
00008 #include "kernel/simulator/PluginInformation.h"
00009
00016 class MetabolicNetwork : public ModelDataDefinition {
00017 public:
00018 MetabolicNetwork(Model* model, std::string name = "");
00019 virtual ~MetabolicNetwork() override = default;
00020
00021 public:
00022 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);

```

```

00023 static PluginInformation* GetPluginInformation();
00024 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00025
00026 public:
00027 void addReaction(std::string reactionName);
00028 void addExchangeSpecies(std::string speciesName);
00029 void clearReactions();
00030 void clearExchangeSpecies();
00031 const std::vector<std::string>& getReactionNames() const;
00032 const std::vector<std::string>& getExchangeSpeciesNames() const;
00033 void setObjectiveReactionName(std::string objectiveReactionName);
00034 std::string getObjectiveReactionName() const;
00035 void setObjectiveSense(std::string objectiveSense);
00036 std::string getObjectiveSense() const;
00037 void setCompartment(std::string compartment);
00038 std::string getCompartment() const;
00039 void setEnabled(bool enabled);
00040 bool isEnabled() const;
00041
00042 virtual std::string show() override;
00043
00044 protected:
00045 virtual bool _loadInstance(PersistenceRecord* fields) override;
00046 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00047 virtual bool _check(std::string& errorMessage) override;
00048 virtual void _createEditableDataDefinitions() override;
00049
00050 protected:
00051 // virtual void _createInternalStatisticReporters() override;
00052 // virtual void _createNonEditableDataDefinitions() override;
00053 // virtual void _createAttachedAttributes() override;
00054
00055 private:
00056 bool _checkReactionNames(std::string& errorMessage) const;
00057 bool _checkExchangeSpeciesNames(std::string& errorMessage) const;
00058
00059 private:
00060 const struct DEFAULT_VALUES {
00061 std::string objectiveReactionName = "";
00062 std::string objectiveSense = "Maximize";
00063 std::string compartment = "";
00064 bool enabled = true;
00065 } DEFAULT;
00066
00067 std::vector<std::string> _reactionNames;
00068 std::vector<std::string> _exchangeSpeciesNames;
00069 std::string _objectiveReactionName = DEFAULT.objectiveReactionName;
00070 std::string _objectiveSense = DEFAULT.objectiveSense;
00071 std::string _compartment = DEFAULT.compartment;
00072 bool _enabled = DEFAULT.enabled;
00073 };
00074
00075 #endif /* METABOLICNETWORK_H */

```

## 16.468 MetabolicReaction.h

```

00001 #ifndef METABOLICREACTION_H
00002 #define METABOLICREACTION_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 #include "../kernel/simulator/model/ModelDataDefinition.h"
00008 #include "kernel/simulator/PluginInformation.h"
00009
00013 class MetabolicReaction : public ModelDataDefinition {
00014 public:
00015 struct StoichiometricTerm {
00016 std::string speciesName;
00017 double stoichiometry = 1.0;
00018 };
00019
00020 public:
00021 MetabolicReaction(Model* model, std::string name = "");
00022 virtual ~MetabolicReaction() override = default;
00023
00024 public:
00025 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00026 static PluginInformation* GetPluginInformation();
00027 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00028
00029 public:
00030 void addReactant(std::string speciesName, double stoichiometry = 1.0);
00031 void addProduct(std::string speciesName, double stoichiometry = 1.0);
00032 void clearReactants();

```

```

00033 void clearProducts();
00034 const std::vector<StoichiometricTerm>& getReactants() const;
00035 const std::vector<StoichiometricTerm>& getProducts() const;
00036 void setLowerBound(double lowerBound);
00037 double getLowerBound() const;
00038 void setUpperBound(double upperBound);
00039 double getUpperBound() const;
00040 void setObjectiveCoefficient(double objectiveCoefficient);
00041 double getObjectiveCoefficient() const;
00042 void setReversible(bool reversible);
00043 bool isReversible() const;
00044 void setGeneRule(std::string geneRule);
00045 std::string getGeneRule() const;
00046
00047 virtual std::string show() override;
00048
00049 protected:
00050 virtual bool _loadInstance(PersistenceRecord* fields) override;
00051 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00052 virtual bool _check(std::string& errorMessage) override;
00053 virtual void _createEditableDataDefinitions() override;
00054
00055 protected:
00056 // virtual void _createInternalStatisticReporters() override;
00057 // virtual void _createNonEditableDataDefinitions() override;
00058 // virtual void _createAttachedAttributes() override;
00059
00060 private:
00061 bool _checkTerms(const std::vector<StoichiometricTerm>& terms, const std::string& side,
00062 std::string& errorMessage) const;
00063
00064 private:
00065 const struct DEFAULT_VALUES {
00066 double lowerBound = 0.0;
00067 double upperBound = 1000.0;
00068 double objectiveCoefficient = 0.0;
00069 bool reversible = false;
00070 std::string geneRule = "";
00071 } DEFAULT;
00072
00073 std::vector<StoichiometricTerm> _reactants;
00074 std::vector<StoichiometricTerm> _products;
00075 double _lowerBound = DEFAULT.lowerBound;
00076 double _upperBound = DEFAULT.upperBound;
00077 double _objectiveCoefficient = DEFAULT.objectiveCoefficient;
00078 bool _reversible = DEFAULT.reversible;
00079 std::string _geneRule = DEFAULT.geneRule;
00080 };
00081 #endif /* METABOLICREACTION_H */

```

## 16.469 DiffusionField.h

```

00001 #ifndef DIFFUSIONFIELD_H
00002 #define DIFFUSIONFIELD_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 #include "../kernel/simulator/model/ModelDataDefinition.h"
00008 #include "kernel/simulator/PluginInformation.h"
00009
00010 class DiffusionMethodOfLinesSystem;
00011
00012 // Plugin that solves the diffusion (heat) equation $du/dt = D \cdot \text{Laplacian}(u)$ in N
00013 // dimensions.
00014 //
00015 // It works in two steps. First the Method of Lines turns the PDE into a system
00016 // of ODEs (central differences in space - see DiffusionMethodOfLinesSystem).
00017 // Then it integrates that system in time with a solver from OdeSolverFactory, so
00018 // the user can pick RK4 or Dormand-Prince 5(4), exactly like in BioNetwork.
00019 //
00020 // N is set by the Dimensions control; the grid has PointsPerDimension points and
00021 // length DomainLength on each axis. The boundary is Dirichlet (fixed value) or
00022 // Neumann (no flux, conserves mass), and the starting field is one of a few
00023 // named shapes (sine mode, Gaussian bump, constant).
00024 //
00025 // The class follows the same layout as BioNetwork (controls, save/load, _check,
00026 // simulate / advanceOneStep) to stay consistent with the rest of the code. The
00027 // grid doesn't change during a run, so each step just rebuilds the small system
00028 // object and reuses the stored field. The solver is never hard-coded - it comes
00029 // from the OdeSolver name through the factory.
00030 //
00031 // Equation representation (design choice): the diffusion PDE is encoded as a
00032 // dedicated C++ class (DiffusionMethodOfLinesSystem), not as a parser expression

```

```

00033 // nor as dynamically compiled C++. This is a deliberate scope decision - a
00034 // specific, well-tested class for diffusion - leaving a general user-defined
00035 // equation mechanism (parser or CppCompiler) as a future extension.
00036 //
00037 // Time semantics: with AutoSchedule enabled, the field steps once per kernel
00038 // InternalEvent (see scheduleNextInternalEvent), so the continuous solve advances
00039 // in sync with the discrete-event clock; otherwise simulate() runs the whole time
00040 // window synchronously when driven by the DiffusionSimulate component.
00041 class DiffusionField : public ModelDataDefinition {
00042 public:
00043 DiffusionField(Model* model, std::string name = "");
00044 virtual ~DiffusionField() override = default;
00045 public:
00046 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00047 static PluginInformation* GetPluginInformation();
00048 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00049 public:
00050 // --- configuration ---
00051 void setDimensions(unsigned int dimensions);
00052 unsigned int getDimensions() const;
00053 void setPointsPerDimension(unsigned int pointsPerDimension);
00054 unsigned int getPointsPerDimension() const;
00055 void setDomainLength(double domainLength);
00056 double getDomainLength() const;
00057 void setDiffusionCoefficient(double diffusionCoefficient);
00058 double getDiffusionCoefficient() const;
00059 void setBoundaryCondition(std::string boundaryCondition);
00060 std::string getBoundaryCondition() const;
00061 void setInitialCondition(std::string initialCondition);
00062 std::string getInitialCondition() const;
00063 void setInitialParameter(double initialParameter);
00064 double getInitialParameter() const;
00065 void setStartTime(double startTime);
00066 double getStartTime() const;
00067 void setStopTime(double stopTime);
00068 double getStopTime() const;
00069 void setStepSize(double stepSize);
00070 double getStepSize() const;
00071 void setCurrentTime(double currentTime);
00072 double getCurrentTime() const;
00073 // When true, the field schedules itself on the kernel's event queue and
00074 // advances one step per InternalEvent, staying in sync with simulated time.
00075 void setAutoSchedule(bool autoSchedule);
00076 bool getAutoSchedule() const;
00077 void setOdeSolver(std::string odeSolver);
00078 std::string getOdeSolver() const;
00079 void setLastStatus(std::string lastStatus);
00080 std::string getLastStatus() const;
00081 void setLastErrorMessage(std::string lastErrorMessage);
00082 std::string getLastErrorMessage() const;
00083 // --- diagnostics (read-only outputs) ---
00084 double getTotalMass() const;
00085 double getMaxValue() const;
00086 double getL2Norm() const;
00087 const std::vector<double>& getField() const;
00088 double getFieldValue(const std::vector<unsigned int>& multiIndex) const;
00089 // --- execution ---
00090 bool simulate(std::string& errorMessage);
00091 bool simulate(double startTime, double stopTime, double stepSize, std::string& errorMessage);
00092 bool advanceOneStep(std::string& errorMessage);
00093 // --- display ---
00094 std::string showFieldAsAscii() const;
00095 virtual std::string show() override;
00096 protected:
00097 virtual bool _loadInstance(PersistenceRecord* fields) override;
00098 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00099 virtual bool _check(std::string& errorMessage) override;
00100 virtual void _initBetweenReplications() override;
00101 private:
00102 bool buildSystem(DiffusionMethodOfLinesSystem* system, std::string& errorMessage) const;
00103 void buildInitialField(const DiffusionMethodOfLinesSystem& system, std::vector<double>& field) const;
00104 void refreshDiagnostics(const DiffusionMethodOfLinesSystem& system);
00105 // Event-driven advance (mirrors BioNetwork): each InternalEvent advances one
00106 // step and reschedules the next, coupling the continuous solve to the kernel clock.
00107 void handleInternalEvent(void* parameter);
00108 void scheduleNextInternalEvent();
00109 private:

```

```

00120 const struct DEFAULT_VALUES {
00121 unsigned int dimensions = 2;
00122 unsigned int pointsPerDimension = 21;
00123 double domainLength = 1.0;
00124 double diffusionCoefficient = 0.1;
00125 std::string boundaryCondition = "Dirichlet";
00126 std::string initialCondition = "SineMode";
00127 double initialParameter = 1.0;
00128 double startTime = 0.0;
00129 double stopTime = 0.1;
00130 double stepSize = 0.01;
00131 double currentTime = 0.0;
00132 bool autoSchedule = false;
00133 std::string odeSolver = "RungeKutta4";
00134 std::string lastStatus = "Idle";
00135 std::string lastErrorMessage = "";
00136 } DEFAULT;
00137
00138 unsigned int _dimensions = DEFAULT.dimensions;
00139 unsigned int _pointsPerDimension = DEFAULT.pointsPerDimension;
00140 double _domainLength = DEFAULT.domainLength;
00141 double _diffusionCoefficient = DEFAULT.diffusionCoefficient;
00142 std::string _boundaryCondition = DEFAULT.boundaryCondition;
00143 std::string _initialCondition = DEFAULT.initialCondition;
00144 double _initialParameter = DEFAULT.initialParameter;
00145 double _startTime = DEFAULT.startTime;
00146 double _stopTime = DEFAULT.stopTime;
00147 double _stepSize = DEFAULT.stepSize;
00148 double _currentTime = DEFAULT.currentTime;
00149 bool _autoSchedule = DEFAULT.autoSchedule;
00150 std::string _odeSolver = DEFAULT.odeSolver;
00151 std::string _lastStatus = DEFAULT.lastStatus;
00152 std::string _lastErrorMessage = DEFAULT.lastErrorMessage;
00153
00154 std::vector<double> _field; // current flattened grid state
00155 double _lastTotalMass = 0.0;
00156 double _lastMaxValue = 0.0;
00157 double _lastL2Norm = 0.0;
00158 };
00159
00160 #endif /* DIFFUSIONFIELD_H */

```

## 16.470 ODESolver.h

```

00001 /*
00002 * File: ODESolver.h
00003 * Author: GenESys
00004 *
00005 * Created on 13 de maio de 2026
00006 */
00007
00008 #ifndef ODESOLVER_H
00009 #define ODESOLVER_H
00010
00011 #include <string>
00012 #include <vector>
00013
00014 #include "kernel/simulator/model/ModelDataDefinition.h"
00015 #include "kernel/simulator/PluginInformation.h"
00016 #include "tools/Continuous/OdeSolver_if.h"
00017 #include "tools/Continuous/OdeSystem_if.h"
00018 #include "tools/Continuous/RungeKutta4OdeSolver.h"
00019
00020 class ODESolver : public ModelDataDefinition {
00021 public:
00022 ODESolver(Model* model, std::string name = "");
00023 virtual ~ODESolver() = default;
00024 public: // static
00025 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00026 static PluginInformation* GetPluginInformation();
00027 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00028 static void SaveInstance(PersistenceRecord* fields, ModelDataDefinition* dataDef);
00029 static bool Check(ModelDataDefinition* dataDef, std::string& errorMessage);
00030 static void InitBetweenReplications(ModelDataDefinition* dataDef);
00031 static void CreateInternalData(ModelDataDefinition* dataDef);
00032 public:
00033 virtual std::string show() override;
00034 public: // gets & sets
00035 void setTimeVariableName(std::string timeVariableName);
00036 std::string getTimeVariableName() const;
00037 void setStateVariableNames(std::vector<std::string> stateVariableNames);
00038 std::vector<std::string> getStateVariableNames() const;
00039 void setEquationExpressions(std::vector<std::string> equationExpressions);
00040 std::vector<std::string> getEquationExpressions() const;
00041 void setStep(double step);

```



```

00042 double getStep() const;
00043 void setPrecision(double precision);
00044 double getPrecision() const;
00045 void setMaxSteps(int maxSteps);
00046 int getMaxSteps() const;
00047 void setCurrentTime(double currentTime);
00048 double getCurrentTime() const;
00049 void setStateValues(std::vector<double> stateValues);
00050 std::vector<double> getStateValues() const;
00051 void setInitialStateValues(std::vector<double> initialStateValues);
00052 std::vector<double> getInitialStateValues() const;
00053 void setAutoAdvance(bool autoAdvance);
00054 bool isAutoAdvance() const;
00055 void setOutputFile(std::string outputFile);
00056 std::string getOutputFile() const;
00057 public: // new methods
00058 void integrate(double targetTime);
00059 void resetState();
00060 double getStateValue(unsigned int index) const;
00061 void setStateValue(unsigned int index, double value);
00062 void synchronizeStateFromVariables();
00063 void writeStateToVariables() const;
00064
00065 protected: // must be overridden
00066 virtual bool _loadInstance(PersistenceRecord *fields) override;
00067 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00068 protected: // could be overridden
00069 virtual bool _check(std::string& errorMessage) override;
00070 virtual void _initBetweenReplications() override;
00071 virtual void _createEditableDataDefinitions() override;
00072
00073 private:
00074 const struct DEFAULT_VALUES {
00075 const std::string timeVariableName = "t";
00076 const double step = 0.01;
00077 const double precision = 1e-6;
00078 const int maxSteps = 100000;
00079 const double currentTime = 0.0;
00080 const bool autoAdvance = true;
00081 const std::string outputFile = "";
00082 } DEFAULT;
00083 std::string _timeVariableName = DEFAULT.timeVariableName;
00084 std::vector<std::string> _stateVariableNames;
00085 std::vector<std::string> _equationExpressions;
00086 double _step = DEFAULT.step;
00087 double _precision = DEFAULT.precision;
00088 int _maxSteps = DEFAULT.maxSteps;
00089 double _currentTime = DEFAULT.currentTime;
00090 bool _autoAdvance = DEFAULT.autoAdvance;
00091 std::string _outputFile = DEFAULT.outputFile;
00092 std::vector<double> _stateValues;
00093 std::vector<double> _initialStateValues;
00094 RungeKutta4OdeSolver _solver;
00095
00096 std::vector<double> _loadStateVector(PersistenceRecord* fields, const std::string& prefix) const;
00097 void _saveStateVector(PersistenceRecord* fields,
00098 const std::string& prefix,
00099 const std::vector<double>& values,
00100 bool saveDefaultValues) const;
00101 double _readVariableValue(const std::string& variableName) const;
00102 // Self-scheduling that couples the continuous integration to the discrete event calendar:
00103 // each internal event advances the ODE by one step and schedules the next, so the continuous
00104 // state evolves with the simulation clock instead of only when a discrete entity arrives.
00105 void _handleInternalEvent(void* parameter);
00106 void _scheduleNextInternalEvent();
00107 // Optional CSV dump of the trajectory: when _outputFile is set, one row per integration
00108 // step is written (t and each state variable), so any model can export its trajectory.
00109 void _writeOutputHeader() const;
00110 void _appendOutputRow() const;
00111 };
00112
00113 #endif /* ODESOLVER_H */

```

## 16.471 Failure.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Failure.h
00009 * Author: rlcancian
00010 */

```

```

00011 * Created on 20 de Failureembro de 2019, 20:07
00012 */
00013
00014 #ifndef FAILURE_H
00015 #define FAILURE_H
00016
00017
00018 #include "../kernel/simulator/model/ModelDataDefinition.h"
00019 #include "../kernel/simulator/model/ModelDataManager.h"
00020 #include "kernel/simulator/PluginInformation.h"
00021
00022 class Resource;
00023
00024 class Failure : public ModelDataDefinition {
00025 public:
00026 enum class FailureType : int {
00027 COUNT = 0, TIME = 1, num_elements = 2
00028 };
00029 enum class FailureRule : int {
00030 IGNORE = 0, PREEMPT = 1, WAIT = 2, num_elements = 3
00031 };
00032 public:
00033 static std::string convertEnumToStr(FailureType type);
00034 static std::string convertEnumToStr(FailureRule rule);
00035 public:
00036 Failure(Model* model, std::string name = "");
00037 virtual ~Failure() override;
00038 public: // static
00039 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00040 static PluginInformation* GetPluginInformation();
00041 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00042 public:
00043 virtual std::string show() override;
00044 void checkIsGoingToFailByCount(Resource* resource);
00045 public: // gets & sets
00046 void setFailureType(FailureType _failureType);
00047 FailureType getFailureType() const;
00048 void setCountExpression(std::string countExpression);
00049 std::string getCountExpression() const;
00050 void setDownTimeTimeUnit(Util::TimeUnit downTimeTimeUnit);
00051 Util::TimeUnit getDownTimeTimeUnit() const;
00052 void setDownTimeExpression(std::string downTimeExpression);
00053 std::string getDownTimeExpression() const;
00054 void setUpTimeTimeUnit(Util::TimeUnit upTimeTimeUnit);
00055 Util::TimeUnit getUpTimeTimeUnit() const;
00056 void setUpTimeExpression(std::string upTimeExpression);
00057 std::string getUpTimeExpression() const;
00058 void setFailureRule(FailureRule _failureRule);
00059 FailureRule getFailureRule() const;
00060
00061 List<Resource*>*falingResources() const;
00062 void addResource(Resource* newResource);
00063 void removeResource(Resource* resource);
00064
00065 protected: // must be overridden
00066 virtual bool _loadInstance(PersistenceRecord *fields) override;
00067 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00068 protected: // could be overridden
00069 virtual bool _check(std::string& errorMessage) override;
00070 virtual void _initBetweenReplications() override;
00071 //virtual ParserChangesInformation* _getParserChangesInformation();
00072 protected:
00073 // virtual void _createInternalStatisticReporters() override;
00074 // virtual void _createNonEditableDataDefinitions() override;
00075 // virtual void _createEditableDataDefinitions() override;
00076 // virtual void _createAttachedAttributes() override;
00077
00078 private: // simulation internal event handlers
00079 void _onFailureActiveEventHandler(void* resourcePtr);
00080 void _onFailureFailEventHandler(void* resourcePtr);
00081 private:
00082 void _scheduleActivation(Resource* resource);
00083 private:
00084 const struct DEFAULT_VALUES {
00085 const FailureType failureType = FailureType::TIME;
00086 const FailureRule failureRule = FailureRule::IGNORE;
00087 const std::string countExpression = "";
00088 const std::string upTimeExpression = "";
00089 const Util::TimeUnit upTimeTimeUnit = Util::TimeUnit::second;
00090 const std::string downTimeExpression = "";
00091 const Util::TimeUnit downTimeTimeUnit = Util::TimeUnit::second;
00092 } DEFAULT;
00093 FailureType _failureType = DEFAULT.failureType;
00094 FailureRule _failureRule = DEFAULT.failureRule;

```

```

00142 std::string _countExpression = DEFAULT.countExpression;
00143 std::string _upTimeExpression = DEFAULT.upTimeExpression;
00144 Util::TimeUnit _upTimeTimeUnit = DEFAULT.upTimeTimeUnit;
00145 std::string _downTimeExpression = DEFAULT.downTimeExpression;
00146 Util::TimeUnit _downTimeTimeUnit = DEFAULT.downTimeTimeUnit;
00147 private:
00148 std::map<Resource*, unsigned int>* _releaseCounts = new std::map<Resource*, unsigned int>();
00149 List<Resource*>* _falingResources = new List<Resource*>();
00150 };
00151
00152 #endif /* FAILURE_H */

```

## 16.472 Queue.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Queue.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 21 de Agosto de 2018, 17:12
00012 */
00013
00014 #ifndef QUEUE_H
00015 #define QUEUE_H
00016
00017 #include "../kernel/simulator/model/ModelDataDefinition.h"
00018 #include "kernel/util/List.h"
00019 #include "../kernel/simulator/essentialPlugins/Entity.h"
00020 #include "../kernel/simulator/model/ModelDataManager.h"
00021 #include "../kernel/simulator/essentialPlugins/StatisticsCollector.h"
00022 #include "kernel/simulator/Plugin.h"
00023 #include "../kernel/simulator/model/ModelComponent.h"
00024
00025 class Waiting {
00026 public:
00027 Waiting(Entity* entity, double timeStartedWaiting, ModelComponent* thisComponent, unsigned int
thisComponentOutputPort = 0) {
00028 _entity = entity;
00029 _thisComponent = thisComponent;
00030 _timeStartedWaiting = timeStartedWaiting;
00031 _thisComponentOutputPort = thisComponentOutputPort;
00032 }
00033
00034 virtual ~Waiting() = default;
00035 public:
00036 virtual std::string show() {
00037 return //ModelDataDefinition::show()+
00038 ",entity=" + std::to_string(_entity->getId()) +
00039 ",component=\"" + _thisComponent->getName() + "\"" +
00040 ",inputPort=\"" + std::to_string(_thisComponentOutputPort) + "\"" +
00041 ",timeStartedWaiting=" + std::to_string(_timeStartedWaiting);
00042 }
00043 public:
00044 double getTimeStartedWaiting() const {
00045 return _timeStartedWaiting;
00046 }
00047 ModelComponent* geComponent() const {
00048 return _thisComponent;
00049 }
00050 Entity* getEntity() const {
00051 return _entity;
00052 }
00053 unsigned int geComponentOutputPort() const {
00054 return _thisComponentOutputPort;
00055 }
00056 unsigned long long getArrivalOrder() const {
00057 return _arrivalOrder;
00058 }
00059 void setArrivalOrder(unsigned long long arrivalOrder) {
00060 _arrivalOrder = arrivalOrder;
00061 }
00062
00063 protected:
00064 private:
00065 Entity* _entity;
00066 ModelComponent* _thisComponent;
00067 double _timeStartedWaiting;
00068 unsigned int _thisComponentOutputPort;
00069 unsigned long long _arrivalOrder = 0;
00070 };

```

```

00071
00101 class Queue : public ModelDataDefinition {
00102 public:
00103
00104 enum class OrderRule : int {
00105 FIFO = 0, LIFO = 1, HIGHESTVALUE = 2, SMALLESTVALUE = 3, num_elements = 4
00106 };
00107 public:
00108 static std::string convertEnumToStr(OrderRule rule);
00109 public:
00110 Queue(Model* model, std::string name = "");
00111 virtual ~Queue();
00112 public:
00113 virtual std::string show() override;
00114 public: // static
00115 static PluginInformation* GetPluginInformation();
00116 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00117 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00118 public:
00119 void insertElement(Waiting* modeldatum);
00120 void removeElement(Waiting* modeldatum);
00121 unsigned int size();
00122 Waiting* first();
00123 Waiting* getAtRank(unsigned int rank);
00124 void setAttributeName(std::string _attributeName);
00125 std::string getAttributeName() const;
00126 void setOrderRule(OrderRule _orderRule);
00127 Queue::OrderRule getOrderRule() const;
00128 void setOrderRuleInt(int orderRule);
00129 int getOrderRuleInt() const;
00130 public: // to implement SIMAN functions
00131 double sumAttributesFromWaiting(Util::identification attributeID); // use to implement SIMAN SAQUE
00132 function
00133 double getAttributeFromWaitingRank(unsigned int rank, Util::identification attributeID);
00134 //public:
00135 // void initBetweenReplications();
00136 protected: // must be overridden
00137 virtual bool _loadInstance(PersistenceRecord *fields) override;
00138 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00139 protected: // could be overridden
00140 virtual bool _check(std::string& errorMessage) override;
00141 virtual void _initBetweenReplications() override;
00142 virtual ParserChangesInformation* _getParserChangesInformation() override;
00143 protected:
00144 virtual void _createInternalStatisticReporters() override;
00145 // virtual void _createNonEditableDataDefinitions() override;
00146 // virtual void _createEditableDataDefinitions() override;
00147 // virtual void _createAttachedAttributes() override;
00148 private:
00149 void _initCStats();
00150 void _configureListComparator();
00151 private:
00152 List<Waiting*>* _list = new List<Waiting*>();
00153 double _lastTimeNumberInQueueChanged = 0.0;
00154 unsigned long long _nextArrivalOrder = 0;
00155 private: //1:1
00156
00157 const struct DEFAULT_VALUES {
00158 const OrderRule orderRule = OrderRule::FIFO;
00159 const std::string attributeName = "";
00160 } DEFAULT;
00161 OrderRule _orderRule = DEFAULT.orderRule;
00162 std::string _attributeName = DEFAULT.attributeName;
00163 private: // inner internal elements
00164 StatisticsCollector* _cstatNumberInQueue = nullptr;
00165 StatisticsCollector* _cstatTimeInQueue = nullptr;
00166 };
00167
00168 #endif /* QUEUE_H */

```

## 16.473 Resource.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Resource.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 21 de Agosto de 2018, 16:52
00012 */

```

```

00013
00014 #ifndef RESOURCE_H
00015 #define RESOURCE_H
00016
00017 #include "../kernel/simulator/model/ModelDataDefinition.h"
00018 #include "../kernel/simulator/essentialPlugins/StatisticsCollector.h"
00019 #include "../kernel/simulator/model/ModelDataManager.h"
00020 #include "../kernel/simulator/essentialPlugins/Counter.h"
00021 #include "kernel/simulator/Plugin.h"
00022 #include "kernel/simulator/OnEventManager.h"
00023
00024 #include "plugins/data/DiscreteProcessing/Failure.h"
00025 #include "plugins/data/DiscreteProcessing/Schedule.h"
00026
00027 #include <functional>
00028
00029
00030
00031 class SeizableItem;
00032
00033
00034
00035
00036
00037
00038
00039
00040
00041
00042
00043
00044
00045
00046
00047
00048
00049
00050
00051
00052
00053
00054
00055
00056
00057
00058
00059
00060
00061
00062
00063
00064
00065
00066
00067
00068
00069
00070
00071
00072
00073
00074
00075
00076
00077
00078
00079
00080
00081
00082
00083
00084
00085
00086
00087
00088
00089
00090
00091
00092
00093
00094
00095
00096
00097
00098
00099
00100
00101
00102
00103
00104
00105
00106
00107
00108
00109
00110
00111
00112
00113
00114
00115
00116
00117
00118
00119
00120
00121
00122
00123
00124
00125
00126
00127
00128
00129
00130
00131
00132
00133
00134
00135
00136
00137
00138
00139
00140
00141
00142
00143
00144
00145
00146
00147
00148
00149
00150
00151
00152

```

```

00153
00154 private: //methods
00155 void _notifyReleaseEventHandlers();
00156 void _onReplicationEnd(SimulationEvent* se);
00157 void _fail();
00158 void _active();
00159 void _checkFailByCount();
00160 void _initStatisticsAndAccounting();
00161 friend class Failure;
00162 friend class ResourceTestProbe;
00163
00164 private:
00165
00166 const struct DEFAULT_VALUES {
00167 const unsigned int capacity = 1;
00168 const double cost = 1.0;
00169 const ResourceState resourceState = ResourceState::IDLE;
00170 } DEFAULT;
00171 unsigned int _capacity = DEFAULT.capacity;
00172 double _costBusyTimeUnit = DEFAULT.cost;
00173 double _costIdleTimeUnit = DEFAULT.cost;
00174 double _costPerUse = DEFAULT.cost;
00175 ResourceState _resourceState = DEFAULT.resourceState;
00176 private: // only gets
00177 unsigned int _numberBusy = 0;
00178 double _lastTimeSeized = 0.0; // @TODO: It won't work for resources with capacity>1, when not all
 capacity is seized and them some more are seized. Seized time of first units will be lost. I don't
 have a solution so far
00179 double _lastTimeReleased = 0.0;
00180 double _lastTimeFailed = 0.0;
00181 double _lastTimeCapacityEvaluated = 0.0;
00182 double _lastTimeAnythingNumberBusy = 0.0;
00183 double _lastTimeIdle = 0.0;
00184 double _lastTimeBusy = 0.0;
00185 double _sumNumberBusyOverTime = 0.0;
00186 double _sumCapacityOverTime = 0.0;
00187 bool _isActive = true;
00188 bool _replicationEndHandlerRegistered = false;
00189 private: // not gets nor sets
00190 unsigned int _originalCapacity; // used for failing purposes, when _capacity changes to 0
00191 private: //l:n
00192 List<SortedResourceEventHandler*> _resourceEventHandlers = new
 List<SortedResourceEventHandler*>();
00193 List<Failure*> _failures = new List<Failure*>();
00194 private: // attached elements
00195 Schedule* _capacitySchedule = nullptr;
00196 private: // internal elements
00197 StatisticsCollector* _cstatTimeSeized = nullptr;
00198 StatisticsCollector* _cstatTimeFailed = nullptr;
00199 StatisticsCollector* _cstatProportionSeized = nullptr;
00200 StatisticsCollector* _cstatCapacityUtilization = nullptr;
00201 Counter* _counterTotalTimeSeized = nullptr;
00202 Counter* _counterTotalTimeFailed = nullptr;
00203 Counter* _counterNumSeizes = nullptr;
00204 Counter* _counterNumReleases = nullptr;
00205 Counter* _counterTotalCostPerUse = nullptr;
00206 Counter* _counterTotalCostBusy = nullptr;
00207 Counter* _counterTotalCostIdle = nullptr;
00208 };
00209
00210 #endif /* RESOURCE_H */

```

## 16.474 Schedule.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: Schedule.h
00008 * Author: Prof. Rafael Luiz Cancian, Dr. Eng.
00009 *
00010 * Created on 14 de dezembro de 2022, 04:24
00011 */
00012
00013 #ifndef SCHEDULE_H
00014 #define SCHEDULE_H
00015
00016 #include "../kernel/simulator/model/ModelDataDefinition.h"
00017 #include "kernel/simulator/PluginInformation.h"
00018 #include "kernel/util/List.h"
00019
00020 class SchedulableItem {
00021 public:

```

```

00022
00023 enum class Rule : int {
00024 IGNORE = 1, PREEMPT = 2, WAIT = 3
00025 };
00026
00027 public:
00028
00029 SchedulableItem(std::string expression, double duration, SchedulableItem::Rule rule =
SchedulableItem::Rule::IGNORE) {
00030 this->expression = expression;
00031 this->duration = duration;
00032 this->rule = rule;
00033 }
00034 public:
00035
00036 void setDuration(double duration) {
00037 this->duration = duration;
00038 }
00039
00040 double getDuration() const {
00041 return duration;
00042 }
00043
00044 void setExpression(std::string expression) {
00045 this->expression = expression;
00046 }
00047
00048 std::string getExpression() const {
00049 return expression;
00050 }
00051
00052 void setRule(SchedulableItem::Rule rule) {
00053 this->rule = rule;
00054 }
00055
00056 SchedulableItem::Rule getRule() const {
00057 return rule;
00058 }
00059
00060 protected:
00061 private:
00062 std::string expression;
00063 double duration;
00064 SchedulableItem::Rule rule;
00065 };
00066
00067 //class SchedulableResourceItem : public SchedulableItem {
00068 //};
00069
00070 class Schedule : public ModelDataDefinition {
00071 public:
00072 Schedule(Model* model, std::string name = "");
00073 virtual ~Schedule() override;
00074 public: // static
00075 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00076 static PluginInformation* GetPluginInformation();
00077 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00078 public: // virtual
00079 virtual std::string show() override;
00080 public:
00081 std::string getExpression();
00082 List<SchedulableItem*>* getSchedulableItems() const;
00083 void setRepeatAfterLast(bool _repeatAfterLast);
00084 bool isRepeatAfterLast() const;
00085 protected: // must be overridden
00086 virtual bool _loadInstance(PersistenceRecord *fields) override;
00087 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00088 protected: // could be overridden by derived classes
00089 virtual bool _check(std::string& errorMessage) override;
00090 virtual ParserChangesInformation* _getParserChangesInformation() override;
00091 virtual void _initBetweenReplications() override;
00092 virtual void _addSimulationControl(SimulationControl* property) override;
00093 protected:
00094 // virtual void _createInternalStatisticReporters() override;
00095 // virtual void _createNonEditableDataDefinitions() override;
00096 // virtual void _createEditableDataDefinitions() override;
00097 // virtual void _createAttachedAttributes() override;
00098
00099 private:
00100
00101 const struct DEFAULT_VALUES {
00102 //const std::string someString = "Test";
00103 //const unsigned int someUInt = 1;
00104 bool repeatAfterLast = true;
00105 } DEFAULT;
00106 bool _repeatAfterLast = DEFAULT.repeatAfterLast;
00107 List<SchedulableItem*>* _schedulableItems = new List<SchedulableItem*>();

```

```

00109 //std::string _someString = DEFAULT.someString;
00110 //unsigned int _someUInt = DEFAULT.someUInt;
00111 };
00112
00113 #endif /* SCHEDULE_H */

```

## 16.475 CppCompiler.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: DynamicLinkedCode.h
00008 * Author: rlcancian
00009 *
00010 * Created on 11 de janeiro de 2022, 22:24
00011 */
00012
00013 #ifndef CPPCOMPILER_H
00014 #define CPPCOMPILER_H
00015
00016 #ifndef OPEN_MAX
00017 #define OPEN_MAX 256
00018 #endif
00019
00020 #include <string>
00021
00022 #include <sys/types.h>
00023 #include <limits.h>
00024 #include <sys/stat.h>
00025 // #include <stdio.h>
00026 #include <unistd.h>
00027 // #include <errno.h>
00028 #include <paths.h>
00029
00030 #include "../kernel/simulator/model/ModelDataDefinition.h"
00031 #include "kernel/simulator/PluginInformation.h"
00032
00033
00034 class CompilerThread {
00035 public:
00036 void operator()();
00037 };
00038
00039 class CppCompiler : public ModelDataDefinition {
00040 public:
00041
00042 class CompilationResult {
00043 public:
00044 CompilationResult() { }
00045 bool success = false;
00046 std::string compilationStdOutput = "";
00047 std::string compilationErrOutput = "";
00048 std::string destinationPath = "";
00049 unsigned int lineNumber = 0;
00050 };
00051
00052 public:
00053 CppCompiler(Model* model, std::string name = "");
00054 virtual ~CppCompiler() = default;
00055 public: // static
00056 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00057 static PluginInformation* GetPluginInformation();
00058 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00059 public:
00060 virtual std::string show() override;
00061 public: // gets & sets
00062 void setSourceFilename(std::string sourceFilename);
00063 std::string getSourceFilename() const;
00064 bool IsLibraryLoaded() const;
00065 void setFlagsExecutable(std::string _flagsExecutable);
00066 std::string getFlagsExecutable() const;
00067 void setLibraryLoaded(bool libraryLoaded);
00068 void setFlagsStaticLibrary(std::string _flagsStaticLibrary);
00069 std::string getFlagsStaticLibrary() const;
00070 void setFlagsDynamicLibrary(std::string _flagsDynamicLibrary);
00071 std::string getFlagsDynamicLibrary() const;
00072 void setFlagsGeneral(std::string _flagsGeneral);
00073 std::string getFlagsGeneral() const;
00074 void setCompilerCommand(std::string _compilerCommand);
00075 std::string getCompilerCommand() const;
00076 void setOutputDir(std::string _outputDir);
00077 std::string getOutputDir() const;

```



```

00078 void setTempDir(std::string _tempDir);
00079 std::string getTempDir() const;
00080 void setOutputFilename(std::string _outputFilename);
00081 std::string getOutputFilename() const;
00082 void setObjectFiles(std::string _objectFiles);
00083 std::string getObjectFiles() const;
00084 public: // new methods
00085 CompilationResult compileToExecutable();
00086 CompilationResult compileToDynamicLibrary();
00087 CompilationResult compileToStaticLibrary();
00088 bool loadLibrary(std::string& errorMessage);
00089 bool unloadLibrary();
00090 void* getDynamicLibraryHandler() const;
00091
00092 protected: // must be overridden
00093 virtual bool _loadInstance(PersistenceRecord *fields) override;
00094 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00095 protected: // could be overridden
00096 virtual bool _check(std::string& errorMessage) override;
00097 // virtual ParserChangesInformation* _getParserChangesInformation();
00098 virtual void _initBetweenReplications() override;
00099 // virtual void _createInternalAndAttachedData() override;
00100 protected:
00101 CompilationResult _invokeCompiler(std::string command); //system(command.c_str());
00102 std::string _read(std::string filename);
00103
00104 protected:
00105 // virtual void _createInternalStatisticReporters() override;
00106 // virtual void _createNonEditableDataDefinitions() override;
00107 // virtual void _createEditableDataDefinitions() override;
00108 // virtual void _createAttachedAttributes() override;
00109
00110 private:
00111
00112 const struct DEFAULT_VALUES {
00113 const std::string sourceFilename = "";
00114 const std::string tempDir = ".temp/";
00115 const std::string outputDir = ".temp/";
00116 const std::string outputFilename = "";
00117 const std::string compiler = "g++";
00118 const std::string flagsGeneral = "-v -w -std=c++14";
00119 const std::string flagsDynamicLibrary = "-shared -fPIC";
00120 const std::string flagsStaticLibrary = "-fPIC";
00121 const std::string flagsExecutable = "";
00122 const std::string objectFiles = "";
00123 } DEFAULT;
00124 std::string _sourceFilename = DEFAULT.sourceFilename;
00125 std::string _outputFilename = DEFAULT.outputFilename;
00126 std::string _tempDir = DEFAULT.tempDir;
00127 std::string _outputDir = DEFAULT.outputDir;
00128 std::string _compilerCommand = DEFAULT.compiler;
00129 std::string _flagsGeneral = DEFAULT.flagsGeneral;
00130 std::string _flagsDynamicLibrary = DEFAULT.flagsDynamicLibrary;
00131 std::string _flagsStaticLibrary = DEFAULT.flagsStaticLibrary;
00132 std::string _flagsExecutable = DEFAULT.flagsExecutable;
00133 std::string _objectFiles = DEFAULT.objectFiles;
00134 //
00135 bool _libraryLoaded = false;
00136 bool _compiledToDynamicLibrary = false;
00137 void* _dynamicLibraryHandle = nullptr;
00138 };
00139
00140 #endif /* CPPCOMPILER_H */

```

## 16.476 OctaveRunner.h

```

00001 /*
00002 * File: OctaveRunner.h
00003 * Author: rlcancian
00004 *
00005 * Octave integration data definition.
00006 */
00007
00008 #ifndef OCTAVERUNNER_H
00009 #define OCTAVERUNNER_H
00010
00011 #include <string>
00012
00013 #include "../kernel/simulator/model/ModelDataDefinition.h"
00014 #include "kernel/simulator/PluginInformation.h"
00015
00023 class OctaveRunner : public ModelDataDefinition {
00024 public:
00026 OctaveRunner(Model* model, std::string name = "");
00028 virtual ~OctaveRunner() = default;

```

```

00029
00030 public:
00032 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00034 static PluginInformation* GetPluginInformation();
00036 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00037
00038 public:
00040 void setOctaveExecutable(std::string octaveExecutable);
00042 std::string getOctaveExecutable() const;
00044 void setWorkingDirectory(std::string workingDirectory);
00046 std::string getWorkingDirectory() const;
00048 void setPreludeScript(std::string preludeScript);
00050 std::string getPreludeScript() const;
00052 void setCommand(std::string command);
00054 std::string getCommand() const;
00056 void setLastStatus(std::string lastStatus);
00058 std::string getLastStatus() const;
00060 void setLastExitCode(int lastExitCode);
00062 int getLastExitCode() const;
00064 void setLastStdout(std::string lastStdout);
00066 std::string getLastStdout() const;
00068 void setLastStderr(std::string lastStderr);
00070 std::string getLastStderr() const;
00072 void setLastResponsePayload(std::string lastResponsePayload);
00074 std::string getLastResponsePayload() const;
00076 void setLastScriptFilename(std::string lastScriptFilename);
00078 std::string getLastScriptFilename() const;
00080 void setLastResponseFilename(std::string lastResponseFilename);
00082 std::string getLastResponseFilename() const;
00083
00085 bool executeCommand(std::string& errorMessage);
00086
00088 virtual std::string show() override;
00089
00090 protected:
00092 virtual bool _loadInstance(PersistenceRecord *fields) override;
00094 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00096 virtual bool _check(std::string& errorMessage) override;
00098 // virtual void _createInternalAndAttachedData() override;
00099
00100
00101 protected:
00102 // virtual void _createInternalStatisticReporters() override;
00103 // virtual void _createNonEditableDataDefinitions() override;
00104 // virtual void _createEditableDataDefinitions() override;
00105 // virtual void _createAttachedAttributes() override;
00106
00107 private:
00108 const struct DEFAULT_VALUES {
00109 std::string octaveExecutable = "Ocavescript";
00110 std::string workingDirectory = "";
00111 std::string preludeScript = "";
00112 std::string command = "";
00113 std::string lastStatus = "Idle";
00114 int lastExitCode = -1;
00115 std::string lastStdout = "";
00116 std::string lastStderr = "";
00117 std::string lastResponsePayload = "";
00118 std::string lastScriptFilename = "";
00119 std::string lastResponseFilename = "";
00120 } DEFAULT;
00121
00122 std::string _octaveExecutable = DEFAULT.octaveExecutable;
00123 std::string _workingDirectory = DEFAULT.workingDirectory;
00124 std::string _preludeScript = DEFAULT.preludeScript;
00125 std::string _command = DEFAULT.command;
00126 std::string _lastStatus = DEFAULT.lastStatus;
00127 int _lastExitCode = DEFAULT.lastExitCode;
00128 std::string _lastStdout = DEFAULT.lastStdout;
00129 std::string _lastStderr = DEFAULT.lastStderr;
00130 std::string _lastResponsePayload = DEFAULT.lastResponsePayload;
00131 std::string _lastScriptFilename = DEFAULT.lastScriptFilename;
00132 std::string _lastResponseFilename = DEFAULT.lastResponseFilename;
00133 };
00134
00135 #endif /* OCTAVERUNNER_H */

```

## 16.477 PythonRuntime.h

```

00001 #ifndef PYTHONRUNTIME_H
00002 #define PYTHONRUNTIME_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelDataDefinition.h"

```

```

00007 #include "kernel/simulator/PluginInformation.h"
00008
00009 class Entity;
00010 class ModelComponent;
00011
00012 class PythonRuntime : public ModelDataDefinition {
00013 public:
00014 PythonRuntime(Model* model, std::string name = "");
00015 virtual ~PythonRuntime() override;
00016
00017 public:
00018 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00019 static PluginInformation* GetPluginInformation();
00020 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00021
00022 public:
00023 void setPythonExecutable(std::string pythonExecutable);
00024 std::string getPythonExecutable() const;
00025 void setFacadeObjectName(std::string facadeObjectName);
00026 std::string getFacadeObjectName() const;
00027 void setCaptureOutput(bool captureOutput);
00028 bool isCaptureOutput() const;
00029 void setLastStatus(std::string lastStatus);
00030 std::string getLastStatus() const;
00031 void setLastErrorMessage(std::string lastErrorMessage);
00032 std::string getLastErrorMessage() const;
00033 void setLastStdout(std::string lastStdout);
00034 std::string getLastStdout() const;
00035 void setLastStderr(std::string lastStderr);
00036 std::string getLastStderr() const;
00037
00038 bool validateHooks(const std::string& initCode,
00039 const std::string& onDispatchCode,
00040 std::string& errorMessage) const;
00041 bool executeInitHook(ModelComponent* component,
00042 const std::string& initCode,
00043 const std::string& onDispatchCode,
00044 std::string& errorMessage);
00045 bool executeOnDispatchHook(ModelComponent* component,
00046 Entity* entity,
00047 const std::string& initCode,
00048 const std::string& onDispatchCode,
00049 std::string& errorMessage);
00050 void resetExecutionState();
00051
00052 virtual std::string show() override;
00053
00054 protected:
00055 virtual bool _loadInstance(PersistenceRecord* fields) override;
00056 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00057 virtual bool _check(std::string& errorMessage) override;
00058
00059 private:
00060 bool _validateIdentifier(std::string& errorMessage) const;
00061 void _invalidatePreparedHooks();
00062 bool _prepareHooks(ModelComponent* component,
00063 const std::string& initCode,
00064 const std::string& onDispatchCode,
00065 std::string& errorMessage);
00066
00067 private:
00068 const struct DEFAULT_VALUES {
00069 std::string pythonExecutable = "python3";
00070 std::string facadeObjectName = "simulator";
00071 bool captureOutput = true;
00072 std::string lastStatus = "Idle";
00073 std::string lastErrorMessage = "";
00074 std::string lastStdout = "";
00075 std::string lastStderr = "";
00076 } DEFAULT;
00077
00078 std::string _pythonExecutable = DEFAULT.pythonExecutable;
00079 std::string _facadeObjectName = DEFAULT.facadeObjectName;
00080 bool _captureOutput = DEFAULT.captureOutput;
00081 std::string _lastStatus = DEFAULT.lastStatus;
00082 std::string _lastErrorMessage = DEFAULT.lastErrorMessage;
00083 std::string _lastStdout = DEFAULT.lastStdout;
00084 std::string _lastStderr = DEFAULT.lastStderr;
00085 std::string _preparedSignature = "";
00086
00087 #if GENESYS_HAS_PYTHON_INTEGRATION
00088 void* _preparedGlobals = nullptr;
00089 void* _preparedInitFunction = nullptr;
00090 void* _preparedDispatchFunction = nullptr;
00091 void* _preparedFacadeObject = nullptr;
00092 #endif
00093 };

```

```

00094
00095 #endif /* PYTHONRUNTIME_H */

```

## 16.478 RSimulatorRunner.h

```

00001 /*
00002 * File: RSimulatorRunner.h
00003 * Author: GenESyS
00004 *
00005 * R integration data definition.
00006 */
00007
00008 #ifndef RSIMULATORRUNNER_H
00009 #define RSIMULATORRUNNER_H
00010
00011 #include <string>
00012
00013 #include "../kernel/simulator/model/ModelDataDefinition.h"
00014 #include "kernel/simulator/PluginInformation.h"
00015
00023 class RSimulatorRunner : public ModelDataDefinition {
00024 public:
00026 RSimulatorRunner(Model* model, std::string name = "");
00028 virtual ~RSimulatorRunner() = default;
00029
00030 public:
00032 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00034 static PluginInformation* GetPluginInformation();
00036 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00037
00038 public:
00040 void setRExecutable(std::string rExecutable);
00042 std::string getRExecutable() const;
00044 void setWorkingDirectory(std::string workingDirectory);
00046 std::string getWorkingDirectory() const;
00048 void setPreludeScript(std::string preludeScript);
00050 std::string getPreludeScript() const;
00052 void setCommand(std::string command);
00054 std::string getCommand() const;
00056 void setLastStatus(std::string lastStatus);
00058 std::string getLastStatus() const;
00060 void setLastExitCode(int lastExitCode);
00062 int getLastExitCode() const;
00064 void setLastStdout(std::string lastStdout);
00066 std::string getLastStdout() const;
00068 void setLastStderr(std::string lastStderr);
00070 std::string getLastStderr() const;
00072 void setLastResponsePayload(std::string lastResponsePayload);
00074 std::string getLastResponsePayload() const;
00076 void setLastScriptFilename(std::string lastScriptFilename);
00078 std::string getLastScriptFilename() const;
00080 void setLastResponseFilename(std::string lastResponseFilename);
00082 std::string getLastResponseFilename() const;
00083
00085 bool executeCommand(std::string& errorMessage);
00086
00088 virtual std::string show() override;
00089
00090 protected:
00092 virtual bool _loadInstance(PersistenceRecord *fields) override;
00094 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00096 virtual bool _check(std::string& errorMessage) override;
00098 // virtual void _createInternalAndAttachedData() override;
00099
00100
00101 protected:
00102 // virtual void _createInternalStatisticReporters() override;
00103 // virtual void _createNonEditableDataDefinitions() override;
00104 // virtual void _createEditableDataDefinitions() override;
00105 // virtual void _createAttachedAttributes() override;
00106
00107 private:
00108 const struct DEFAULT_VALUES {
00109 std::string rExecutable = "Rscript";
00110 std::string workingDirectory = "";
00111 std::string preludeScript = "";
00112 std::string command = "";
00113 std::string lastStatus = "Idle";
00114 int lastExitCode = -1;
00115 std::string lastStdout = "";
00116 std::string lastStderr = "";
00117 std::string lastResponsePayload = "";
00118 std::string lastScriptFilename = "";
00119 std::string lastResponseFilename = "";
00120 } DEFAULT;

```

```

00121
00122 std::string _rExecutable = DEFAULT.rExecutable;
00123 std::string _workingDirectory = DEFAULT.workingDirectory;
00124 std::string _preludeScript = DEFAULT.preludeScript;
00125 std::string _command = DEFAULT.command;
00126 std::string _lastStatus = DEFAULT.lastStatus;
00127 int _lastExitCode = DEFAULT.lastExitCode;
00128 std::string _lastStdout = DEFAULT.lastStdout;
00129 std::string _lastStderr = DEFAULT.lastStderr;
00130 std::string _lastResponsePayload = DEFAULT.lastResponsePayload;
00131 std::string _lastScriptFilename = DEFAULT.lastScriptFilename;
00132 std::string _lastResponseFilename = DEFAULT.lastResponseFilename;
00133 };
00134
00135 #endif /* RSIMULATORRUNNER_H */

```

## 16.479 SPICERunner.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: SPICERunner.h
00008 * Author: bnmfw & joaomai
00009 *
00010 * Created on 10 de outubro de 2023, 23:00
00011 */
00012
00013 #ifndef SPICERUNNER_H
00014 #define SPICERUNNER_H
00015
00016 #include <iostream>
00017 #include <fstream>
00018 #include <sstream>
00019 #include <string>
00020 #include <vector>
00021 #include <map>
00022 #include <set>
00023 #include <algorithm>
00024 #include <functional>
00025
00026 #include "../kernel/simulator/model/ModelDataDefinition.h"
00027 #include "kernel/simulator/PluginInformation.h"
00028
00029 const std::vector<std::pair<std::string, float>
 spice_unit_map{{"T",1e12},{ "G",1e9},{ "Meg",1e6},{ "K",1e3},{ "",1e0},{ "m",1e-3},{ "u",1e-6},{ "n",1e-9},{ "p",1e-12},{ "f",1e-15}}
00030 inline std::string uc(double value){
00031 float precision = 0.9999;
00032 for (auto element: spice_unit_map){
00033 if (value/element.second < precision) continue;
00034 return std::to_string(value/element.second) + element.first + " ";
00035 }
00036 return std::to_string(value/1e-18) + "a ";
00037 };
00038
00039 class SPICERunner : public ModelDataDefinition {
00040 public:
00041 SPICERunner(Model* model, std::string name = "");
00042 virtual ~SPICERunner();
00043
00044 public:
00045 std::string CompileSpiceFile();
00046 void SendComponent(std::string *instance, std::string subcircuit = "", std::string model = "");
00047 void SendCallback(std::function<void()> callback);
00048 void ConfigSim(double duration, double step);
00049 void Run();
00050 void PlotV(std::string net);
00051 template<typename... Args> void PlotV(std::string net, Args... args);
00052 void PlotVRelative(std::string comparison_base, std::string net);
00053 template<typename... Args> void PlotVRelative(std::string comparison_base, std::string net,
 Args... args);
00054 void PlotI(std::string net);
00055 template<typename... Args> void PlotI(std::string net, Args... args);
00056 void PlotIRelative(std::string comparison_base, std::string net);
00057 template<typename... Args> void PlotIRelative(std::string comparison_base, std::string net,
 Args... args);
00058 double* MeasurePeak(std::string label, std::string peak, std::string quantity, std::string node,
 float start, float finish);
00059 double* MeasureTrigTarg(std::string label, std::string trig, float trig_value, std::string
 trig_inclin, std::string targ, float targ_value, std::string targ_inclin);
00060 void ParseOutput();
00061 void setRunnerCommand(std::string runnerCommand);
00062 std::string getRunnerCommand() const;

```

```

00063 void setModelsPath(std::string modelsPath);
00064 std::string getModelsPath() const;
00065 void setWorkingInputFilename(std::string workingInputFilename);
00066 std::string getWorkingInputFilename() const;
00067 void setWorkingOutputFilename(std::string workingOutputFilename);
00068 std::string getWorkingOutputFilename() const;
00069 void setWorkingDirectory(std::string workingDirectory);
00070 std::string getWorkingDirectory() const;
00071 std::string getLastRunCommand() const;
00072
00073 public:
00074 virtual std::string show() override;
00075
00076 public:
00077 static PluginInformation* GetPluginInformation();
00078 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00079 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00080 // Added
00081 static std::vector<std::string> split(std::string word, char split_char);
00082
00083 protected:
00084 virtual bool _loadInstance(PersistenceRecord *fields) override;
00085 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00086
00087 protected:
00088 virtual bool _check(std::string& errorMessage) override;
00089 // virtual ParserChangesInformation* _getParserChangesInformation();
00090 // virtual void _initBetweenReplications();
00091 // virtual void _createInternalStatisticReporters() override;
00092 // virtual void _createNonEditableDataDefinitions() override;
00093 // virtual void _createEditableDataDefinitions() override;
00094 // virtual void _createAttachedAttributes() override;
00095 // virtual void _createInternalAndAttachedData(); /*< A ModelDataDefinition or ModelComponent that
includes (internal) ou refers to (attach) other ModelDataDefinition must register them inside this
method. */
00101 // virtual void _addSimulationControl(SimulationControl* property);
00102
00103
00104 protected:
00105
00106 private:
00107 // ...
00108
00109 private:
00110 const struct DEFAULT_VALUES {
00111 const std::string runnerCommand = "ngspice";
00112 const std::string modelsPath = "./";
00113 const std::string workingInputFilename = "input.cir";
00114 const std::string workingOutputFilename = "output";
00115 const std::string workingDirectory = "";
00116 } DEFAULT;
00117 std::string _runnerCommand = DEFAULT.runnerCommand;
00118 std::string _modelsPath = DEFAULT.modelsPath;
00119 std::string _workingInputFilename = DEFAULT.workingInputFilename;
00120 std::string _workingOutputFilename = DEFAULT.workingOutputFilename;
00121 std::string _workingDirectory = DEFAULT.workingDirectory;
00122 std::string _lastRunCommand = "";
00123
00124 std::set<std::string> subcircuits;
00125 std::set<std::string> models;
00126
00127 std::vector<std::function<void()>> subscribers;
00128 std::vector<std::string*> instances;
00129
00130 // Plot
00131 std::vector<std::string> vplots;
00132 std::vector<std::string> iplots;
00133
00134 // Measure
00135 std::vector<std::string> measures;
00136 std::map<std::string, double*> promises;
00137
00138 std::string config;
00139
00140 private:
00141 // DummyElement* _internalDataDefinition = nullptr;
00142
00143 private:
00144 // ...
00145 };
00146
00147 #endif /* SPICERUNNER_H */

```

## 16.480 EntityGroup.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Group.h
00009 * Author: rlcancian
00010 *
00011 * Created on 12 de Junho de 2019, 19:00
00012 */
00013
00014 #ifndef ENTITYGROUP_H
00015 #define ENTITYGROUP_H
00016
00017 #include "../kernel/simulator/model/ModelDataDefinition.h"
00018 #include "../kernel/simulator/essentialPlugins/Entity.h"
00019 #include "kernel/util/List.h"
00020 #include <list>
00021 #include <map>
00022
00023 /*
00024 The EntityGroup represents a map of entities grouped together somehow.
00025 * The entities in this group are kept into a internal queue.
00026 * Usually one entity representing all grouped entities is generated by ModelComponents and the
 EntityGroup it represented is accessed through an attribute.
00027 * The Key of this map is the ID of the representative entity
00028 */
00029 class EntityGroup : public ModelDataDefinition {
00030 public:
00031 EntityGroup(Model* model, std::string name = "");
00032 virtual ~EntityGroup();
00033 public:
00034 virtual std::string show() override;
00035 public:
00036 static PluginInformation* GetPluginInformation();
00037 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00038 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00039 public:
00040 void insertElement(unsigned int idKey, Entity* modeldatum);
00041 void removeElement(unsigned int idKey, Entity* modeldatum);
00042 List<Entity*>* getGroup(unsigned int idKey);
00043 protected:
00044 virtual bool _loadInstance(PersistenceRecord *fields) override;
00045 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00046 virtual bool _check(std::string& errorMessage) override;
00047 virtual void _initBetweenReplications() override;
00048 // virtual void _createInternalAndAttachedData() override;
00049
00050 protected:
00051 virtual void _createInternalStatisticReporters() override;
00052 // virtual void _createNonEditableDataDefinitions() override;
00053 // virtual void _createEditableDataDefinitions() override;
00054 virtual void _createAttachedAttributes() override;
00055
00056 private:
00057 void _clearGroups();
00058 void _initCStats();
00059 private: //l:n
00060 std::map<unsigned int, List<Entity*>*>* _groupMap = new std::map<unsigned int, List<Entity*>*>();
00061 std::list<List<Entity*>*>* _detachedEmptyGroups;
00062 private: // child inner modeldatum
00063 StatisticsCollector* _cstatNumberInGroup = nullptr;
00064 };
00065
00066 #endif /* ENTITYGROUP_H */

```

## 16.481 File.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: File.h
00009 * Author: rlcancian
00010 *
00011 * Created on 20 de Fileembro de 2019, 20:07
00012 */

```

```

00013
00014 #ifndef FILE_H
00015 #define FILE_H
00016
00017
00018 #include "../kernel/simulator/model/ModelDataDefinition.h"
00019 #include "../kernel/simulator/model/ModelDataManager.h"
00020 // #include "ParserChangesInformation.h"
00021 #include "kernel/simulator/PluginInformation.h"
00022
00064 class File : public ModelDataDefinition {
00065 public:
00066 enum class AccessMode : unsigned int {
00067 Read = 0,
00068 Write = 1,
00069 Append = 2,
00070 ReadWrite = 3
00071 };
00072
00073 File(Model* model, std::string name = "");
00074 virtual ~File() = default;
00075 public: // static
00076 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00077 static PluginInformation* GetPluginInformation();
00078 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00079 public:
00080 void setSystemFilename(std::string systemFilename);
00081 std::string getSystemFilename() const;
00082 std::string getFilenameOnly() const;
00083 std::string getPathOnly() const;
00084
00085 void setAccessMode(AccessMode accessMode);
00086 AccessMode getAccessMode() const;
00087 void setAccessModeAsString(std::string accessMode);
00088 std::string getAccessModeAsString() const;
00089
00090 virtual std::string show() override;
00091
00092 protected: // must be overridden
00093 virtual bool _loadInstance(PersistenceRecord* fields) override;
00094 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00095 protected: // could be overridden
00096 virtual bool _check(std::string& errorMessage) override;
00097 virtual ParserChangesInformation* _getParserChangesInformation() override;
00098
00099
00100 protected:
00101 // virtual void _createInternalStatisticReporters() override;
00102 // virtual void _createNonEditableDataDefinitions() override;
00103 // virtual void _createEditableDataDefinitions() override;
00104 // virtual void _createAttachedAttributes() override;
00105
00106 private:
00107 static std::string _accessModeToString(AccessMode accessMode);
00108 static bool _stringToAccessMode(const std::string& accessMode, AccessMode* parsedAccessMode);
00109 static std::string _normalizePathSeparators(const std::string& filename);
00110
00111 private:
00112 const struct DEFAULT_VALUES {
00113 std::string systemFilename = "";
00114 AccessMode accessMode = AccessMode::Read;
00115 } DEFAULT;
00116 std::string _systemFilename = DEFAULT.systemFilename;
00117 AccessMode _accessMode = DEFAULT.accessMode;
00118 bool _accessModeWasInvalid = false;
00119 };
00120
00121 #endif /* FILE_H */

```

## 16.482 AssignmentItem.h

```

00001 /*
00002 * The MIT License
00003 *
00004 * Copyright 2022 rlcancian.
00005 *
00006 * Permission is hereby granted, free of charge, to any person obtaining a copy
00007 * of this software and associated documentation files (the "Software"), to deal
00008 * in the Software without restriction, including without limitation the rights
00009 * to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00010 * copies of the Software, and to permit persons to whom the Software is
00011 * furnished to do so, subject to the following conditions:
00012 *
00013 * The above copyright notice and this permission notice shall be included in
00014 * all copies or substantial portions of the Software.

```



```

00015 *
00016 * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00017 * IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00018 * FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00019 * AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00020 * LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00021 * OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
00022 * THE SOFTWARE.
00023 */
00024
00025 /*
00026 * File: AssignmentItem.h
00027 * Author: Prof. Rafael Luiz Cancian, Dr. Eng.
00028 *
00029 * Created on 2 de fevereiro de 2022, 14:12
00030 */
00031
00032 #ifndef ASSIGNMENTITEM_H
00033 #define ASSIGNMENTITEM_H
00034
00035 #include <functional>
00036 #include "../kernel/simulator/model/Model.h"
00037
00041 class Assignment {
00042 public:
00043 Assignment(Model* model, std::string destination, std::string expression="1", bool
isAttributeNotVariable = true);
00044 Assignment(std::string destination, std::string expression, bool isAttributeNotVariable = true);
00045 virtual ~Assignment();
00046 void setDestination(std::string _destination);
00047 std::string getDestination() const;
00048 void setExpression(std::string _expression);
00049 std::string getExpression() const;
00050 void setAttributeNotVariable(bool isAttributeNotVariable);
00051 bool isAttributeNotVariable() const;
00052 void setChangeCallback(std::function<void(Assignment*)> callback);
00053
00054 // TODO: don't have name
00055 std::string getName() const;
00056
00057 List<SimulationControl*> getSimulationControls() const;
00058 void ensureSimulationControls(Model* model);
00059 void _addSimulationControl(SimulationControl* control);
00060
00061 std::string getTypeDC() {return _typeDC;};
00062 public:
00063 bool loadInstance(PersistenceRecord *fields, unsigned int parentIndex);
00064 void saveInstance(PersistenceRecord *fields, unsigned int parentIndex, bool saveDefault);
00065 private:
00066 void _updateTypeDC();
00067 std::string _destination = "";
00068 std::string _expression = "";
00069 bool _isAttributeNotVariable = true;
00070 std::string _typeDC;
00071 std::function<void(Assignment*)> _changeCallback = nullptr;
00072
00073 List<SimulationControl*> _simulationControls = new List<SimulationControl*>();
00074 };
00075
00076
00077 #endif /* ASSIGNMENTITEM_H */

```

## 16.483 Formula.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Formula.h
00009 * Author: rlcancian
00010 *
00011 * Created on 20 de Junho de 2019, 00:56
00012 */
00013
00014 #ifndef FORMULA_H
00015 #define FORMULA_H
00016
00017 #include "../kernel/simulator/model/ModelDataDefinition.h"
00018 #include "../kernel/simulator/model/ModelDataManager.h"
00019 #include "kernel/simulator/Plugin.h"
00020 // #include "kernel/simulator/Parser_if.h"
00021

```

```

00022 class Formula : public ModelDataDefinition {
00023 public:
00029 Formula(Model* model, std::string name = "");
00033 virtual ~Formula() override;
00034 public: // virtual
00039 virtual std::string show() override;
00040 public:
00045 unsigned int size();
00051 void setExpression(std::string formulaExpression, std::string index = "");
00052 //void setExpression(std::string formulaExpression);
00058 std::string getExpression(std::string index = "");
00059 //std::string getExpression();
00060 //double getValue();
00066 double getValue(std::string index);
00067 public: // statics
00072 static PluginInformation* GetPluginInformation();
00079 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00086 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00087 protected: // must be overridden by derived classes
00088 virtual bool _loadInstance(PersistenceRecord *fields) override;
00089 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00090 protected: // could be overridden by derived classes
00091 virtual bool _check(std::string& errorMessage) override;
00092 //virtual void _addSimulationControl(SimulationControl* property);
00094 //virtual ParserChangesInformation* _getParserChangesInformation();
00095 //virtual void _initBetweenReplications();
00096 // virtual void _createInternalStatisticReporters() override;
00097 // virtual void _createNonEditableDataDefinitions() override;
00098 // virtual void _createEditableDataDefinitions() override;
00099 virtual void _createAttachedAttributes() override;
00100
00101 private:
00102 std::map<std::string, std::string>* _formulaExpressions = new std::map<std::string,
std::string>(); // map<index, formula>
00103 private:
00105 };
00106
00107 #endif /* FORMULA_H */

```

## 16.484 Label.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: Label.h
00008 * Author: rlcancian
00009 *
00010 * Created on 15 de janeiro de 2022, 10:13
00011 */
00012
00013 #ifndef LABEL_H
00014 #define LABEL_H
00015 #include "../kernel/simulator/model/ModelDataDefinition.h"
00016 #include "../kernel/simulator/essentialPlugins/Entity.h"
00017
00018 class Label : public ModelDataDefinition {
00019 public:
00020 Label(Model* model, std::string name = "");
00021 virtual ~Label() = default;
00022 public: // static
00023 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00024 static PluginInformation* GetPluginInformation();
00025 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00026 public:
00027 virtual std::string show() override;
00028 void setLabel(std::string _label);
00029 std::string getLabel() const;
00030 void setEnterIntoLabelComponent(ModelComponent* enteringLabelComponent);
00031 ModelComponent* getEnterIntoLabelComponent() const;
00032 void sendEntityToLabelComponent(Entity* entity, double timeDelay);
00033 protected: // must be overridden
00034 virtual bool _loadInstance(PersistenceRecord *fields) override;
00035 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00036 protected: // could be overridden
00037 virtual bool _check(std::string& errorMessage) override;
00038 //virtual ParserChangesInformation* _getParserChangesInformation();
00039 //virtual void _initBetweenReplications();
00040 // virtual void _createInternalStatisticReporters() override;
00041 // virtual void _createNonEditableDataDefinitions() override;
00042 // virtual void _createEditableDataDefinitions() override;
00043 // virtual void _createAttachedAttributes() override;
00044 //virtual void _createInternalAndAttachedData();

```

```

00045
00046 protected:
00047
00048 private:
00049 std::string _label;
00050 ModelComponent* _enteringLabelComponent = nullptr;
00051 };
00052
00053 #endif /* LABEL_H */

```

## 16.485 Set.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Set.h
00009 * Author: rlcancian
00010 *
00011 * Created on 03 de Junho de 2019, 15:11
00012 */
00013
00014 #ifndef SET_H
00015 #define SET_H
00016
00017 #include <vector>
00018
00019 #include "../kernel/simulator/model/ModelDataDefinition.h"
00020 #include "../kernel/simulator/model/ModelDataManager.h"
00021 #include "kernel/simulator/ParserChangesInformation.h"
00022 #include "kernel/simulator/PluginInformation.h"
00023 #include "../kernel/simulator/essentialPlugins/EntityType.h"
00024
00058 class Set : public ModelDataDefinition {
00059 public:
00060 Set(Model* model, std::string name = "", std::vector<std::string> allowedElementTypes = {});
00061 virtual ~Set() override;
00062 public: // static
00063 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00064 static PluginInformation* GetPluginInformation();
00065 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00066 public:
00067 virtual std::string show() override;
00068 public:
00069 //void setSetOfType(std::string _setOfType);
00070 std::string getSetOfType() const;
00071 bool setSetOfType(const std::string& setOfType);
00072 bool canChangeSetOfType() const;
00073 void setAllowedElementTypes(const std::vector<std::string>& allowedElementTypes);
00074 void addAllowedElementType(const std::string& allowedElementType);
00075 std::vector<std::string> getAllowedElementTypes() const;
00076 List<ModelDataDefinition*> getElementSet() const;
00077
00091
00092 void addElementSet(ModelDataDefinition* newElement);
00093 void removeElementSet(ModelDataDefinition* element);
00100 ModelDataDefinition* createElementSetOfType(const std::string& typeName, const std::string& name =
00101 "");
00102 protected: // must be overridden
00103 virtual bool _loadInstance(PersistenceRecord *fields) override;
00104 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00105 protected: // could be overridden
00106 virtual bool _check(std::string& errorMessage) override;
00107 virtual ParserChangesInformation* _getParserChangesInformation() override;
00108 protected:
00109 // virtual void _createInternalStatisticReporters() override;
00110 // virtual void _createNonEditableDataDefinitions() override;
00111 // virtual void _createEditableDataDefinitions() override;
00112 virtual void _createAttachedAttributes() override;
00113
00114 private:
00115 //ElementManager* _elems;
00116
00117 const struct DEFAULT_VALUES {
00118 const unsigned int membersSize = 0;
00119 const std::string setOfType = "";
00120 } DEFAULT;
00121 List<ModelDataDefinition*> _elementSet = new List<ModelDataDefinition*>();
00122 std::string _setOfType = DEFAULT.setOfType;
00123 std::vector<std::string> _allowedElementTypes;
00124 };
00125

```

```
00126 #endif /* SET_H */
```

## 16.486 Variable.h

```
00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Variable.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 4 de Setembro de 2018, 18:28
00012 */
00013
00014 #ifndef VARIABLE_H
00015 #define VARIABLE_H
00016
00017 #include "../kernel/simulator/essentialPlugins/Attribute.h"
00018 #include "../kernel/simulator/model/ModelDataManager.h"
00019 #include "kernel/simulator/Plugin.h"
00020 #include "../kernel/simulator/essentialPlugins/SparseValueStore.h"
00021
00091 class Variable : public Attribute {
00092 public:
00093 Variable(Model* model, std::string name = "");
00094 virtual ~Variable() override;
00095 public:
00096 virtual std::string show() override;
00097 public: //static
00098 static PluginInformation* GetPluginInformation();
00099 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00100 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00101 public:
00102 double getValue(std::string index="");
00103 void setValue(double value, std::string index="");
00104 void copyValuesFrom(const Variable& source);
00105 std::string getInitialValuesText() const override;
00106 void setInitialValuesText(std::string valuesText) override;
00107 void insertDimentionSize(unsigned int size);
00108 std::map<std::string, double> *getValues() const;
00109 SparseValueStore* getValueStore();
00110 ModelDataDefinition* get_scope();
00111 void set_scope(ModelDataDefinition* const scope);
00112
00121 protected:
00122 virtual bool _loadInstance(PersistenceRecord *fields) override;
00123 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00124 virtual bool _check(std::string& errorMessage) override;
00125 virtual void _initBetweenReplications() override;
00126
00127
00128 protected:
00129 // virtual void _createInternalStatisticReporters() override;
00130 // virtual void _createNonEditableDataDefinitions() override;
00131 // virtual void _createEditableDataDefinitions() override;
00132 // virtual void _createAttachedAttributes() override;
00133
00134 private:
00135 //const struct DEFAULT_VALUES { } DEFAULT;
00136 SparseValueStore* _values = new SparseValueStore();
00137 ModelDataDefinition* scope = nullptr; // not used so far
00138 };
00139
00140 #endif /* VARIABLE_H */
```

## 16.487 Sequence.h

```
00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Sequence.h
00009 * Author: rlcancian
00010 *
00011 * Created on 03 de Junho de 2019, 15:12
00012 */
```

```

00013
00014 #ifndef SEQUENCE_H
00015 #define SEQUENCE_H
00016
00017 #include "../kernel/simulator/model/ModelDataDefinition.h"
00018 #include "../kernel/simulator/model/ModelDataManager.h"
00019 #include "kernel/simulator/PluginInformation.h"
00020 #include "Station.h"
00021 #include "../Logic/Label.h"
00022 #include "../Logic/AssignmentItem.h"
00023
00024 class SequenceStep : public PersistentObject_base {
00025 public:
00026 SequenceStep(Station* station, std::list<Assignment*>* assignments = nullptr);
00027 SequenceStep(Label* label, std::list<Assignment*>* assignments = nullptr);
00028 SequenceStep(Model* model, std::string stationOrLabelName, bool isStation = true,
00029 std::list<Assignment*>* assignments = nullptr);
00029 virtual ~SequenceStep() override;
00030 public: // virtual
00031
00032 virtual bool _loadInstance(PersistenceRecord *fields, unsigned int parentIndex);
00033 virtual void _saveInstance(PersistenceRecord *fields, unsigned int parentIndex, bool
00034 saveDefaultValues);
00034 virtual bool _loadInstance(PersistenceRecord *fields) override;
00035 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00036
00037 public:
00038
00039 std::list<Assignment*>* getAssignments() const;
00040 void setStation(Station* _station);
00041 Station* getStation() const;
00042 void setElementManager(ModelDataManager* _modelDataManager);
00043 void setLabel(Label* _label);
00044 Label* getLabel() const;
00045
00046 protected:
00047 private:
00048
00049 const struct DEFAULT_VALUES {
00050 const unsigned int assignmentsSize = 0;
00051 } DEFAULT;
00052 Station* _station = nullptr;
00053 Label* _label = nullptr;
00054 std::list<Assignment*>* _assignments = new std::list<Assignment*>();
00055 private:
00056 ModelDataManager* _modelDataManager = nullptr;
00057 };
00058
00059 class Sequence : public ModelDataDefinition {
00060 public:
00061
00062 public:
00063 Sequence(Model* model, std::string name = "");
00064 virtual ~Sequence() override;
00065 public:
00066 virtual std::string show() override;
00067 public: // static
00068 static PluginInformation* GetPluginInformation();
00069 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00070 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00071 public:
00072 List<SequenceStep*>* getSteps() const;
00073 protected:
00074 virtual bool _loadInstance(PersistenceRecord *fields) override;
00075 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00076 virtual bool _check(std::string& errorMessage) override;
00077 protected:
00078 // virtual void _createInternalStatisticReporters() override;
00079 // virtual void _createNonEditableDataDefinitions() override;
00080 // virtual void _createEditableDataDefinitions() override;
00081 virtual void _createAttachedAttributes() override;
00082 private:
00083 List<SequenceStep*>* _steps = new List<SequenceStep*>();
00084 };
00085
00086 #endif /* SEQUENCE_H */

```

## 16.488 Station.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */

```

```

00006
00007 /*
00008 * File: Station.h
00009 * Author: rlcancian
00010 *
00011 * Created on 03 de Junho de 2019, 15:12
00012 */
00013
00014 #ifndef STATION_H
00015 #define STATION_H
00016
00017 #include "../kernel/simulator/model/ModelDataDefinition.h"
00018 #include "../kernel/simulator/model/ModelDataManager.h"
00019 #include "kernel/simulator/Plugin.h"
00020 #include "../kernel/simulator/essentialPlugins/Entity.h"
00021
00022 /*
00023 Station module
00024 DESCRIPTION
00025 The Station module defines a station (or a set of stations) corresponding to a physical
00026 or logical location where processing occurs. If the Station module defines a station
00027 set, it is effectively defining multiple processing locations.
00028 The station (or each station within the defined set) has a matching Activity Area that
00029 is used to report all times and costs accrued by the entities in this station. This
00030 Activity Area's name is the same as the station. If a parent Activity Area is defined,
00031 then it also accrues any times and costs by the entities in this station.
00032 TYPICAL USES
00033 * Defining a lathe area
00034 * Defining a set of toll booths
00035 * Defining a food preparation area
00036 PROMPTS
00037 Prompt Description
00038 Name Unique name of the module that will be displayed in the
00039 flowchart.
00040 Station Type Type of station being defined, either as an individual Station or a
00041 station Set.
00042 Station Name Name of the individual station.
00043 Set Name Name of the station set.
00044 Parent Activity Area Name of the Activity Area's parent.
00045 Associated Intersection Name of the intersection associated with this station in a guided
00046 transporter network.
00047 Report Statistics Specifies whether or not statistics will automatically be collected
00048 and stored in the report database for this station and its
00049 corresponding activity area.
00050 Save Attribute Attribute name used to store the index number into the station set
00051 of the member that is selected.
00052 Station Set Members Names of the stations that are members of this station set.
00053 Station Name A given station can only exist once within a model. Therefore, an
00054 individual station can only be the member of one station set, and
00055 that individual station may not be the name of a station in
00056 another module.
00057 Parent Activity Area Name of the Activity Area's parent for the station set member.
00058 Associated Intersection Name of the intersection associated with this station set in a
00059 guided transporter network.
00060 Report Statistics Specifies whether or not statistics will automatically be collected
00061 and stored in the report database for this station set member and
00062 its corresponding activity area.
00063 */
00064 class Station : public ModelDataDefinition {
00065 public:
00066 Station(Model* model, std::string name = "");
00067 virtual ~Station();
00068 public:
00069 virtual std::string show() override;
00070 public: // static
00071 static PluginInformation* GetPluginInformation();
00072 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00073 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00074 public:
00075 void initBetweenReplications();
00076 void enter(Entity* entity);
00077 void leave(Entity* entity);
00078 void setEnterIntoStationComponent(ModelComponent* _enterIntoStationComponent);
00079 ModelComponent* getEnterIntoStationComponent() const;
00080 protected:
00081 virtual bool _loadInstance(PersistenceRecord *fields) override;
00082 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00083 virtual bool _check(std::string& errorMessage) override;
00084 // virtual void _createInternalAndAttachedData() override;
00085 virtual void _initBetweenReplications() override;
00086
00087 protected:
00088 virtual void _createInternalStatisticReporters() override;
00089 // virtual void _createNonEditableDataDefinitions() override;
00090 // virtual void _createEditableDataDefinitions() override;
00091 virtual void _createAttachedAttributes() override;
00092

```

```

00093 private:
00094 void _initCStats();
00095 unsigned int _numberInStation = 0;
00096 ModelComponent* _enterIntoStationComponent = nullptr;
00097 private: // inner elements
00098 StatisticsCollector* _cstatNumberInStation = nullptr;
00099 StatisticsCollector* _cstatTimeInStation = nullptr;
00100 friend class StationTestProbe;
00101 };
00102
00103 #endif /* STATION_H */

```

## 16.489 Storage.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Storage.h
00009 * Author: rlcancian
00010 *
00011 * Created on 20 de Storageembro de 2019, 20:06
00012 */
00013
00014 #ifndef STORAGE_H
00015 #define STORAGE_H
00016
00017
00018 #include "../kernel/simulator/model/ModelDataDefinition.h"
00019 #include "../kernel/simulator/model/ModelDataManager.h"
00020 // #include "kernel/simulator/ParserChangesInformation.h"
00021 #include "kernel/simulator/PluginInformation.h"
00022
00023 /*
00024 Storage module
00025 DESCRIPTION
00026 The Storage module defines the name of a storage. Storages are automatically created
00027 by any module that references the storage so that this module is seldom needed. The
00028 only time this module is needed is when a storage is defined as a member of a storage
00029 set or specified using an attribute or expression.
00030 TYPICAL USES
00031 * Defining an animate storage for a set of storages
00032 PROMPTS
00033 Prompt Description
00034 Name The name of the storage set being defined. This name must be
00035 unique.
00036 */
00037 class Storage : public ModelDataDefinition {
00038 public:
00039 Storage(Model* model, std::string name = "");
00040 virtual ~Storage() = default;
00041 public: // static
00042 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00043 static PluginInformation* GetPluginInformation();
00044 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00045 public:
00046 virtual std::string show() override;
00047 void setTotalArea(double _totalArea);
00048 double getTotalArea() const;
00049 void setCapacity(unsigned int _capacity);
00050 unsigned int getCapacity() const;
00051 void setUnitsPerArea(double _unitsPerArea);
00052 double getUnitsPerArea() const;
00053
00054 protected: // must be overridden
00055 virtual bool _loadInstance(PersistenceRecord *fields) override;
00056 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00057 protected: // could be overridden
00058 virtual bool _check(std::string& errorMessage) override;
00059 virtual ParserChangesInformation* _getParserChangesInformation() override;
00060
00061 protected:
00062 // virtual void _createInternalStatisticReporters() override;
00063 // virtual void _createNonEditableDataDefinitions() override;
00064 // virtual void _createEditableDataDefinitions() override;
00065 // virtual void _createAttachedAttributes() override;
00066
00067 private:
00068
00069 const struct DEFAULT_VALUES {
00070 const double totalArea = 1;
00071 const unsigned int capacity = 10;

```

```

00072 const double unitsPerArea = 1;
00073 } DEFAULT;
00074 double _totalArea = DEFAULT.totalArea;
00075 unsigned int _capacity = DEFAULT.capacity;
00076 double _unitsPerArea = DEFAULT.unitsPerArea;
00077
00078 private:
00079 //@TODO: Add statisticCollector for ProportionOfStorageUsage
00080 };
00081 #endif /* STORAGE_H */

```

## 16.490 SignalData.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: SignalData.h
00008 * Author: rlcancian
00009 *
00010 * Created on 25 de maio de 2022, 14:37
00011 */
00012
00013 #ifndef SIGNALDATA_H
00014 #define SIGNALDATA_H
00015
00016 #include "../kernel/simulator/model/ModelDataDefinition.h"
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "kernel/util/List.h"
00019
00020 class SignalData : public ModelDataDefinition {
00021 public:
00022 typedef std::function<unsigned int(SignalData*)> SignalDataEventHandler; ///< Returns the total
 number of freed entities.
00023 typedef std::pair<SignalDataEventHandler, ModelComponent*> PairSignalDataEventHandler;
00024 template<typename Class>
00025 static SignalDataEventHandler SetSignalDataEventHandler(unsigned int
 (Class::*function)(SignalData*), Class * object) {
00026 return std::bind(function, object, std::placeholders::_1);
00027 }
00028 public:
00029 SignalData(Model* model, std::string name = "");
00030 virtual ~SignalData() override;
00031 public: // static
00032 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00033 static PluginInformation* GetPluginInformation();
00034 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00035 public: //virtual
00036 virtual std::string show() override;
00037 public:
00038 unsigned int generateSignal(double signalValue, unsigned int limit);
00039 void addSignalDataEventHandler(SignalDataEventHandler eventHandler, ModelComponent* component);
00040 void removeSignalDataEventHandler(ModelComponent* component);
00041 bool hasSignalDataEventHandler(ModelComponent* component) const;
00042 unsigned int remainsToLimit() const;
00043 void decreaseRemainLimit();
00044
00045 protected: // must be overridden
00046 virtual bool _loadInstance(PersistenceRecord *fields) override;
00047 virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00048 protected: // could be overridden
00049 virtual bool _check(std::string& errorMessage) override;
00050 virtual void _initBetweenReplications() override;
00051 //virtual void _createInternalAndAttachedData();
00052 //virtual ParserChangesInformation* _getParserChangesInformation();
00053
00054 protected:
00055 // virtual void _createInternalStatisticReporters() override;
00056 // virtual void _createNonEditableDataDefinitions() override;
00057 // virtual void _createEditableDataDefinitions() override;
00058 // virtual void _createAttachedAttributes() override;
00059
00060 private: // methods
00061 unsigned int _notifySignalDataEventHandlers();
00062 private: //1:1
00063 unsigned int _remainsToLimit = 0;
00064 private: //1:n
00065 List<PairSignalDataEventHandler*> _signalDataEventHandlers = new
 List<PairSignalDataEventHandler*>();
00066 };
00067
00068 #endif /* SIGNALDATA_H */

```



## 16.491 DummyElement.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: DummyElement.h
00008 * Author: Prof. Rafael Luiz Cancian, Dr. Eng.
00009 *
00010 * Created on 11 de janeiro de 2022, 22:26
00011 */
00012
00013 #ifndef DUMMYELEMENT_H
00014 #define DUMMYELEMENT_H
00015
00016 #include "../kernel/simulator/model/ModelDataDefinition.h"
00017 #include "kernel/simulator/PluginInformation.h"
00018
00019 class DummyElement : public ModelDataDefinition {
00020 public:
00021 DummyElement(Model* model, std::string name = "");
00022 virtual ~DummyElement() = default;
00023
00024 public:
00025 void setSomeString(const std::string& someString);
00026 std::string getSomeString() const;
00027 void setSomeUInt(unsigned int someUInt);
00028 unsigned int getSomeUInt() const;
00029
00030 public:
00031 virtual std::string show() override;
00032
00033 public:
00034 static PluginInformation* GetPluginInformation();
00035 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00036 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00037
00038 protected:
00039 virtual bool _loadInstance(PersistenceRecord* fields) override;
00040 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00041 virtual bool _check(std::string& errorMessage) override;
00042
00043 protected:
00044 // virtual bool _check(std::string& errorMessage);
00045 // virtual ParserChangesInformation* _getParserChangesInformation();
00046 // virtual void _initBetweenReplications();
00047 // virtual void _createInternalStatisticReporters() override;
00048 // virtual void _createNonEditableDataDefinitions() override;
00049 // virtual void _createEditableDataDefinitions() override;
00050 // virtual void _createAttachedAttributes() override;
00051 // virtual void _createInternalAndAttachedData(); /*< A ModelDataDefinition or ModelComponent that
00052 includes (internal) ou refers to (attach) other ModelDataDefinition must register them inside this
00053 method. */
00054 // virtual void _addSimulationControl(SimulationControl* property);
00055
00056 protected:
00057
00058 private:
00059 // ...
00060
00061 private:
00062 const struct DEFAULT_VALUES {
00063 const std::string someString = "Test";
00064 const unsigned int someUInt = 1;
00065 } DEFAULT;
00066 std::string _someString = DEFAULT.someString;
00067 unsigned int _someUInt = DEFAULT.someUInt;
00068
00069 private:
00070 // DummyElement* _internalDataDefinition = nullptr;
00071
00072 private:
00073 // ...
00074 };
00075
00076 #endif /* DUMMYELEMENT_H */

```

## 16.492 BioCompartment.h

```

00001 #ifndef BIOCOMPARTMENT_H
00002 #define BIOCOMPARTMENT_H

```

```

00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelDataDefinition.h"
00007 #include "kernel/simulator/PluginInformation.h"
00008
00025 class BioCompartment : public ModelDataDefinition {
00026 public:
00027 BioCompartment(Model* model, std::string name = "");
00028 virtual ~BioCompartment() override = default;
00029
00030 public:
00031 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00032 static PluginInformation* GetPluginInformation();
00033 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00034
00035 public:
00036 void setCompartmentType(std::string type);
00037 std::string getCompartmentType() const;
00038 void setParentCompartmentName(std::string parentName);
00039 std::string getParentCompartmentName() const;
00040 void setMembraneBounded(bool membraneBounded);
00041 bool isMembraneBounded() const;
00042 void setVolumeFraction(double volumeFraction);
00043 double getVolumeFraction() const;
00044 void setCopyNumber(unsigned int copyNumber);
00045 unsigned int getCopyNumber() const;
00046 void setRole(std::string role);
00047 std::string getRole() const;
00048
00049 virtual std::string show() override;
00050
00051 protected:
00052 virtual bool _loadInstance(PersistenceRecord* fields) override;
00053 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00054 virtual bool _check(std::string& errorMessage) override;
00055 virtual void _createEditableDataDefinitions() override;
00056
00057 private:
00058 const struct DEFAULT_VALUES {
00059 std::string compartmentType = "cytosol";
00060 std::string parentCompartmentName = "";
00061 bool membraneBounded = false;
00062 double volumeFraction = 1.0;
00063 unsigned int copyNumber = 1u;
00064 std::string role = "generic";
00065 } DEFAULT;
00066
00067 std::string _compartmentType = DEFAULT.compartmentType;
00068 std::string _parentCompartmentName = DEFAULT.parentCompartmentName;
00069 bool _membraneBounded = DEFAULT.membraneBounded;
00070 double _volumeFraction = DEFAULT.volumeFraction;
00071 unsigned int _copyNumber = DEFAULT.copyNumber;
00072 std::string _role = DEFAULT.role;
00073 };
00074
00075 #endif /* BIOCOMPARTMENT_H */

```

## 16.493 MolecularSpecies.h

```

00001 #ifndef MOLECULARSPECIES_H
00002 #define MOLECULARSPECIES_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelDataDefinition.h"
00007 #include "kernel/simulator/PluginInformation.h"
00008
00016 class MolecularSpecies : public ModelDataDefinition {
00017 public:
00018 MolecularSpecies(Model* model, std::string name = "");
00019 virtual ~MolecularSpecies() override = default;
00020
00021 public:
00022 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00023 static PluginInformation* GetPluginInformation();
00024 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00025
00026 public:
00027 void setInitialCount(int initialCount);
00028 int getInitialCount() const;
00029 void setCount(int count);
00030 int getCount() const;
00031 void setCompartment(std::string compartment);
00032 std::string getCompartment() const;

```

```

00033 void setUnit(std::string unit);
00034 std::string getUnit() const;
00035
00036 virtual std::string show() override;
00037
00038 protected:
00039 virtual bool _loadInstance(PersistenceRecord* fields) override;
00040 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00041 virtual bool _check(std::string& errorMessage) override;
00042 virtual void _initBetweenReplications() override;
00043
00044 private:
00045 const struct DEFAULT_VALUES {
00046 int initialCount = 0;
00047 int count = 0;
00048 std::string compartment = "cytoplasm";
00049 std::string unit = "molecules";
00050 } DEFAULT;
00051
00052 int _initialCount = DEFAULT.initialCount;
00053 int _count = DEFAULT.count;
00054 std::string _compartment = DEFAULT.compartment;
00055 std::string _unit = DEFAULT.unit;
00056 };
00057
00058 #endif /* MOLECULARSPECIES_H */

```

## 16.494 StochasticReactionRule.h

```

00001 #ifndef STOCHASTICREACTIONRULE_H
00002 #define STOCHASTICREACTIONRULE_H
00003
00004 #include <string>
00005 #include <vector>
00006 #include <map>
00007
00008 #include "../kernel/simulator/model/ModelDataDefinition.h"
00009 #include "kernel/simulator/PluginInformation.h"
00010
00022 class StochasticReactionRule : public ModelDataDefinition {
00023 public:
00024 struct StoichiometricTerm {
00025 std::string speciesName;
00026 int stoichiometry = 1;
00027 };
00028
00029 public:
00030 StochasticReactionRule(Model* model, std::string name = "");
00031 virtual ~StochasticReactionRule() override = default;
00032
00033 public:
00034 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00035 static PluginInformation* GetPluginInformation();
00036 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00037
00038 public:
00039 void addReactant(std::string speciesName, int stoichiometry = 1);
00040 void addProduct(std::string speciesName, int stoichiometry = 1);
00041 void clearReactants();
00042 void clearProducts();
00043 const std::vector<StoichiometricTerm>& getReactants() const;
00044 const std::vector<StoichiometricTerm>& getProducts() const;
00045
00046 void setRateConstant(double rateConstant);
00047 double getRateConstant() const;
00048 void setCompartment(std::string compartment);
00049 std::string getCompartment() const;
00050
00051 double computePropensity(const std::map<std::string, int>& counts) const;
00052
00053 virtual std::string show() override;
00054
00055 protected:
00056 virtual bool _loadInstance(PersistenceRecord* fields) override;
00057 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00058 virtual bool _check(std::string& errorMessage) override;
00059
00060 private:
00061 static double _combinatorial(int n, int k);
00062
00063 private:
00064 const struct DEFAULT_VALUES {
00065 double rateConstant = 0.0;
00066 std::string compartment = "cytoplasm";
00067 } DEFAULT;

```

```

00068
00069 std::vector<StoichiometricTerm> _reactants;
00070 std::vector<StoichiometricTerm> _products;
00071 double _rateConstant = DEFAULT.rateConstant;
00072 std::string _compartment = DEFAULT.compartment;
00073 };
00074
00075 #endif /* STOCHASTICREACTIONRULE_H */

```

## 16.495 WholeCellState.h

```

00001 #ifndef WHOLECELLSTATE_H
00002 #define WHOLECELLSTATE_H
00003
00004 #include <map>
00005 #include <string>
00006 #include <vector>
00007
00008 #include "../kernel/simulator/model/ModelDataDefinition.h"
00009 #include "kernel/simulator/PluginInformation.h"
00010
00025 class WholeCellState : public ModelDataDefinition {
00026 public:
00027 WholeCellState(Model* model, std::string name = "");
00028 virtual ~WholeCellState() override = default;
00029
00030 public:
00031 static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00032 static PluginInformation* GetPluginInformation();
00033 static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00034
00035 public:
00036 // Molecule count access (integer, for SSA)
00037 void setMoleculeCount(const std::string& speciesName, int count);
00038 int getMoleculeCount(const std::string& speciesName) const;
00039 bool hasMoleculeCount(const std::string& speciesName) const;
00040 const std::map<std::string, int>& getMoleculeCounts() const;
00041
00042 // Metabolite pool access (continuous, for FBA output)
00043 void setMetaboliteAmount(const std::string& metaboliteName, double amount);
00044 double getMetaboliteAmount(const std::string& metaboliteName) const;
00045 const std::map<std::string, double>& getMetabolitePool() const;
00046
00047 // Compartment-specific metabolite pool access (continuous, for compartment-aware metabolism)
00048 void setCompartmentMetaboliteAmount(const std::string& compartmentName, const std::string&
metaboliteName, double amount);
00049 double getCompartmentMetaboliteAmount(const std::string& compartmentName, const std::string&
metaboliteName) const;
00050 bool hasCompartmentMetaboliteAmount(const std::string& compartmentName, const std::string&
metaboliteName) const;
00051 const std::map<std::string, double>& getCompartmentMetabolitePool() const;
00052
00053 // Pathway activity access (continuous, for FBA objective/flux summaries)
00054 void setPathwayActivity(const std::string& pathwayName, double activity);
00055 double getPathwayActivity(const std::string& pathwayName) const;
00056 bool hasPathwayActivity(const std::string& pathwayName) const;
00057 const std::map<std::string, double>& getPathwayActivities() const;
00058
00059 // Resource budget access (integer, set by fair allocation before SSA)
00060 void setResourceBudget(const std::string& speciesName, int budget);
00061 int getResourceBudget(const std::string& speciesName) const;
00062 void clearResourceBudget();
00063
00064 // Cell geometry
00065 void setCellVolume(double volume);
00066 double getCellVolume() const;
00067 void setCellMass(double mass);
00068 double getCellMass() const;
00069
00070 // Simulation progress
00071 void setCurrentTime(double time);
00072 double getCurrentTime() const;
00073 void setStepCount(int stepCount);
00074 int getStepCount() const;
00075 void incrementStep();
00076
00077 // Lifecycle metadata
00078 void setLifecyclePhase(const std::string& phase);
00079 std::string getLifecyclePhase() const;
00080 void setGenerationCount(int generationCount);
00081 int getGenerationCount() const;
00082 void setViable(bool viable);
00083 bool isViable() const;
00084 void setLastDivisionTime(double time);
00085 double getLastDivisionTime() const;

```

```

00086
00087 // JSON loading from CovertLab/WholeCell data files (MIT)
00088 bool loadFixedConstants(const std::string& jsonPath);
00089 bool loadParameters(const std::string& jsonPath);
00090
00091 // JSON file paths (persisted so the model file remembers them)
00092 void setFixedConstantsPath(std::string path);
00093 std::string getFixedConstantsPath() const;
00094 void setParametersPath(std::string path);
00095 std::string getParametersPath() const;
00096
00097 virtual std::string show() override;
00098
00099 protected:
00100 virtual bool _loadInstance(PersistenceRecord* fields) override;
00101 virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00102 virtual bool _check(std::string& errorMessage) override;
00103 virtual void _initBetweenReplications() override;
00104
00105 private:
00106 const struct DEFAULT_VALUES {
00107 double cellVolume = 1.0e-15; // 1 fL, typical M. genitalium volume
00108 double cellMass = 1.0e-13; // ~100 fg, typical M. genitalium mass
00109 double currentTime = 0.0;
00110 int stepCount = 0;
00111 std::string lifecyclePhase = "newborn";
00112 int generationCount = 0;
00113 bool viable = true;
00114 double lastDivisionTime = -1.0;
00115 std::string fixedConstantsPath = "";
00116 std::string parametersPath = "";
00117 } DEFAULT;
00118
00119 std::map<std::string, int> _moleculeCounts;
00120 std::map<std::string, int> _initialMoleculeCounts;
00121 std::map<std::string, double> _metabolitePool;
00122 std::map<std::string, double> _initialMetabolitePool;
00123 std::map<std::string, double> _compartmentMetabolitePool;
00124 std::map<std::string, double> _initialCompartmentMetabolitePool;
00125 std::map<std::string, double> _pathwayActivities;
00126 std::map<std::string, double> _initialPathwayActivities;
00127 std::map<std::string, int> _resourceBudget;
00128
00129 double _cellVolume = DEFAULT.cellVolume;
00130 double _cellMass = DEFAULT.cellMass;
00131 double _initialCellVolume = DEFAULT.cellVolume;
00132 double _initialCellMass = DEFAULT.cellMass;
00133 double _currentTime = DEFAULT.currentTime;
00134 int _stepCount = DEFAULT.stepCount;
00135 std::string _lifecyclePhase = DEFAULT.lifecyclePhase;
00136 std::string _initialLifecyclePhase = DEFAULT.lifecyclePhase;
00137 int _generationCount = DEFAULT.generationCount;
00138 int _initialGenerationCount = DEFAULT.generationCount;
00139 bool _viable = DEFAULT.viable;
00140 bool _initialViable = DEFAULT.viable;
00141 double _lastDivisionTime = DEFAULT.lastDivisionTime;
00142 double _initialLastDivisionTime = DEFAULT.lastDivisionTime;
00143 std::string _fixedConstantsPath = DEFAULT.fixedConstantsPath;
00144 std::string _parametersPath = DEFAULT.parametersPath;
00145 };
00146
00147 #endif /* WHOLECELLSTATE_H */

```

## 16.496 PluginConnectorDummyImpl1.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: PluginConnectorDummyImpl1.h
00009 * Author: rlcancian
00010 *
00011 * Created on 9 de Setembro de 2019, 19:24
00012 */
00013
00014 #ifndef PLUGINCONNECTORDUMMYIMPL1_H
00015 #define PLUGINCONNECTORDUMMYIMPL1_H
00016
00017 #include "kernel/simulator/PluginConnector_if.h"
00018 #include "kernel/util/List.h"
00019 //namespace GenesysKernel {
00020

```

```

00027 class PluginConnectorDummyImpl1 : public PluginConnector_if {
00028 public:
00029 PluginConnectorDummyImpl1();
00030 virtual ~PluginConnectorDummyImpl1() = default;
00031 public:
00032 Plugin* check(const std::string dynamicLibraryFilename) override;
00043 Plugin* connect(const std::string dynamicLibraryFilename) override;
00048 List<std::string>* find() override;
00054 bool disconnect(const std::string dynamicLibraryFilename) override;
00060 bool disconnect(Plugin* plugin) override;
00061 private:
00062 void _connect(List<Plugin*>* plugins, Plugin* plugin);
00063 };
00064 //namespace\\}
00065 #endif /* PLUGINCONNECTORDUMMYIMPL1_H */

```

## 16.497 PluginConnectorStaticImpl1.h

```

00001 #ifndef PLUGINCONNECTORSTATICIMPL1_H
00002 #define PLUGINCONNECTORSTATICIMPL1_H
00003
00004 #include "kernel/simulator/PluginConnector_if.h"
00005
00012 class PluginConnectorStaticImpl1 : public PluginConnector_if {
00013 public:
00014 PluginConnectorStaticImpl1() = default;
00015 virtual ~PluginConnectorStaticImpl1() = default;
00016
00022 Plugin* check(const std::string dynamicLibraryFilename) override;
00028 Plugin* connect(const std::string dynamicLibraryFilename) override;
00033 List<std::string>* find() override;
00039 bool disconnect(const std::string dynamicLibraryFilename) override;
00045 bool disconnect(Plugin* plugin) override;
00046 };
00047
00048 #endif

```

## 16.498 AIAssistant\_if.h

```

00001 /*
00002 * File: AIAssistant_if.h
00003 * Author: Genesys Team
00004 *
00005 * Created on 28 de Maio de 2026
00006 */
00007
00008 #ifndef AIASSISTANT_IF_H
00009 #define AIASSISTANT_IF_H
00010
00011 #include "AIAuditLog.h"
00012
00013 #include <string>
00014 #include <vector>
00015
00016 // Non-owning handles to kernel objects.
00017 // Implementation files that call facade methods must include the full headers.
00018 class SimulatorFacade;
00019 class AIProviderClient_if;
00020
00021 // -----
00022 // Enumerations
00023 // -----
00024
00028 enum class AIProvider {
00029 Undefined,
00030 OpenAI,
00031 Anthropic,
00032 Local,
00033 CustomHttp
00034 };
00035
00039 enum class ExecutionState {
00040 NotConfigured,
00041 Ready,
00042 Running,
00043 Finished,
00044 Failed,
00045 Cancelled,
00046 NotImplemented
00047 };
00048
00057 enum class ReasoningEffort {

```

```

00058 Low,
00059 Medium,
00060 High
00061 };
00062
00063 // -----
00064 // Configuration structures
00065 // -----
00066
00084 struct AIAssistantConfiguration {
00085 AIProvider provider = AIProvider::Undefined;
00086 std::string providerName;
00087 std::string modelName;
00088 std::string baseUrl;
00089 std::string apiKeyEnvironmentVariable;
00090 std::string organizationId;
00091 std::string projectId;
00092 ReasoningEffort reasoningEffort = ReasoningEffort::Medium;
00093 double temperature = 0.2;
00094 unsigned int maxOutputTokens = 4096;
00095 unsigned int timeoutSeconds = 120;
00096 bool allowNetworkAccess = false;
00097 bool allowModelMutation = false;
00098 bool allowSimulationExecution = false;
00099 bool allowFileSystemWrites = false;
00100
00110 bool sandboxEnabled = false;
00111
00126 bool dryRun = false;
00127 };
00128
00129 // -----
00130 // Request / response structures
00131 // -----
00132
00142 struct AIAssistantRequest {
00143 std::string userPrompt;
00144 std::string systemContext;
00145 std::string modelDescription;
00146 std::string experimentDescription;
00147 std::string modelPlan;
00148 bool buildModel = false;
00149 bool configureSimulation = false;
00150 bool runSimulation = false;
00151 bool collectResults = false;
00152 };
00153
00166 struct AIAssistantResponse {
00167 bool success = false;
00168 ExecutionState state = ExecutionState::NotImplemented;
00169 std::string message;
00170 std::string generatedPlan;
00171 std::string generatedModelLanguage;
00172 std::string diagnostics;
00173 };
00174
00175 // -----
00176 // Knowledge base structures
00177 // -----
00178
00190 struct GenesysKnowledgeItem {
00191 std::string name;
00192 std::string category;
00193 std::string typeName;
00194 std::string description;
00195 std::string usage;
00196 std::string constraints;
00197 };
00198
00202 struct GenesysKnowledgeBaseSummary {
00203 unsigned int numberOfPlugins = 0;
00204 unsigned int numberOfComponents = 0;
00205 unsigned int numberOfDataDefinitions = 0;
00206 std::string summaryMarkdown;
00207 };
00208
00209 // -----
00210 // Interface
00211 // -----
00212
00237 class AIAssistant_if {
00238 public:
00239 virtual ~AIAssistant_if() = default;
00240
00241 // -----
00242 // Configuration
00243 // -----

```

```

00244
00252 virtual void setConfiguration(const AIAssistantConfiguration& configuration) = 0;
00253
00260 virtual AIAssistantConfiguration getConfiguration() const = 0;
00261
00263 virtual void setProvider(AIProvider provider) = 0;
00265 virtual AIProvider getProvider() const = 0;
00266
00268 virtual void setProviderName(const std::string& providerName) = 0;
00270 virtual std::string getProviderName() const = 0;
00271
00273 virtual void setModelName(const std::string& modelName) = 0;
00275 virtual std::string getModelName() const = 0;
00276
00284 virtual void setBaseUrl(const std::string& baseUrl) = 0;
00286 virtual std::string getBaseUrl() const = 0;
00287
00289 virtual void setReasoningEffort(ReasoningEffort effort) = 0;
00291 virtual ReasoningEffort getReasoningEffort() const = 0;
00292
00300 virtual void setTemperature(double temperature) = 0;
00302 virtual double getTemperature() const = 0;
00303
00305 virtual void setMaxOutputTokens(unsigned int maxOutputTokens) = 0;
00307 virtual unsigned int getMaxOutputTokens() const = 0;
00308
00310 virtual void setTimeoutSeconds(unsigned int timeoutSeconds) = 0;
00312 virtual unsigned int getTimeoutSeconds() const = 0;
00313
00314 // -----
00315 // Security and secrets
00316 // -----
00317
00330 virtual void setApiKey(const std::string& apiKey) = 0;
00331
00342 virtual void clearApiKey() = 0;
00343
00345 virtual bool hasApiKey() const = 0;
00346
00355 virtual std::string getMaskedApiKey() const = 0;
00356
00364 virtual void setApiKeyEnvironmentVariable(const std::string& variableName) = 0;
00366 virtual std::string getApiKeyEnvironmentVariable() const = 0;
00367
00378 virtual bool loadApiKeyFromEnvironment() = 0;
00379
00380 // -----
00381 // Integration with GenESyS
00382 // -----
00383
00392 virtual void setSimulatorFacade(SimulatorFacade* simulatorFacade) = 0;
00394 virtual SimulatorFacade* getSimulatorFacade() const = 0;
00396 virtual bool hasSimulatorFacade() const = 0;
00397
00398 // -----
00399 // Provider client
00400 // -----
00401
00414 virtual void setProviderClient(AIProviderClient_if* client) = 0;
00415
00419 virtual AIProviderClient_if* getProviderClient() const = 0;
00420
00422 virtual bool hasProviderClient() const = 0;
00423
00424 // -----
00425 // Knowledge base
00426 // -----
00427
00429 virtual void clearKnowledgeBase() = 0;
00430
00432 virtual void addKnowledgeItem(const GenesysKnowledgeItem& item) = 0;
00433
00435 virtual std::vector<GenesysKnowledgeItem> getKnowledgeItems() const = 0;
00436
00438 virtual GenesysKnowledgeBaseSummary getKnowledgeBaseSummary() const = 0;
00439
00447 virtual std::string exportKnowledgeBaseAsMarkdown() const = 0;
00448
00466 virtual bool refreshKnowledgeBaseFromSimulator(std::string* message = nullptr) = 0;
00467
00468 // -----
00469 // Workflow
00470 // -----
00471
00490 virtual bool checkReady(std::string* message = nullptr) const = 0;
00491
00498 virtual AIAssistantResponse analyzePrompt(const AIAssistantRequest& request) = 0;

```



```

00499
00506 virtual AIAssistantResponse createModelPlan(const AIAssistantRequest& request) = 0;
00507
00523 virtual AIAssistantResponse buildModel(const AIAssistantRequest& request) = 0;
00524
00539 virtual AIAssistantResponse configureSimulation(const AIAssistantRequest& request) = 0;
00540
00549 virtual AIAssistantResponse runSimulation(const AIAssistantRequest& request) = 0;
00550
00558 virtual AIAssistantResponse collectResults(const AIAssistantRequest& request) = 0;
00559
00568 virtual AIAssistantResponse execute(const AIAssistantRequest& request) = 0;
00569
00570 // -----
00571 // Diagnostics
00572 // -----
00573
00575 virtual ExecutionState getState() const = 0;
00576
00578 virtual std::string getLastError() const = 0;
00579
00586 virtual std::string getLastDiagnostics() const = 0;
00587
00589 virtual void clearDiagnostics() = 0;
00590
00591 // -----
00592 // Audit log
00593 // -----
00594
00602 virtual std::vector<AuditEntry> getAuditEntries(unsigned int maxEntries = 200) const = 0;
00603
00605 virtual std::string getAuditLogPath() const = 0;
00606
00612 virtual unsigned int exportAuditLog(const std::string& csvPath) const = 0;
00613 };
00614
00615 #endif /* AIASSISTANT_IF_H */

```

## 16.499 AIAssistantDefaultImpl.h

```

00001 /*
00002 * File: AIAssistantDefaultImpl.h
00003 * Author: Genesys Team
00004 *
00005 * Created on 28 de Maio de 2026
00006 */
00007
00008 #ifndef AIASSISTANTDEFAULTIMPL_H
00009 #define AIASSISTANTDEFAULTIMPL_H
00010
00011 #include "AIAssistant_if.h"
00012 #include "AIAuditLog.h"
00013 #include "AIProviderClient_if.h"
00014
00015 #include <memory>
00016 #include <string>
00017 #include <vector>
00018
00045 class AIAssistantDefaultImpl : public AIAssistant_if {
00046 public:
00047 AIAssistantDefaultImpl();
00048 virtual ~AIAssistantDefaultImpl();
00049
00050 // Not copyable: would propagate the in-memory API key and the non-owning
00051 // SimulatorFacade pointer without a clear ownership contract.
00052 AIAssistantDefaultImpl(const AIAssistantDefaultImpl&) = delete;
00053 AIAssistantDefaultImpl& operator=(const AIAssistantDefaultImpl&) = delete;
00054
00055 public:
00056 // -----
00057 // Configuration
00058 // -----
00059 virtual void setConfiguration(const AIAssistantConfiguration& configuration) override;
00060 virtual AIAssistantConfiguration getConfiguration() const override;
00061
00062 virtual void setProvider(AIProvider provider) override;
00063 virtual AIProvider getProvider() const override;
00064
00065 virtual void setProviderName(const std::string& providerName) override;
00066 virtual std::string getProviderName() const override;
00067
00068 virtual void setModelName(const std::string& modelName) override;
00069 virtual std::string getModelName() const override;
00070
00071 virtual void setBaseUrl(const std::string& baseUrl) override;

```

```

00072 virtual std::string getUrl() const override;
00073
00074 virtual void setReasoningEffort(ReasoningEffort effort) override;
00075 virtual ReasoningEffort getReasoningEffort() const override;
00076
00077 virtual void setTemperature(double temperature) override;
00078 virtual double getTemperature() const override;
00079
00080 virtual void setMaxOutputTokens(unsigned int maxOutputTokens) override;
00081 virtual unsigned int getMaxOutputTokens() const override;
00082
00083 virtual void setTimeoutSeconds(unsigned int timeoutSeconds) override;
00084 virtual unsigned int getTimeoutSeconds() const override;
00085
00086 // -----
00087 // Security and secrets
00088 // -----
00089 virtual void setApiKey(const std::string& apiKey) override;
00090 virtual void clearApiKey() override;
00091 virtual bool hasApiKey() const override;
00092 virtual std::string getMaskedApiKey() const override;
00093
00094 virtual void setApiKeyEnvironmentVariable(const std::string& variableName) override;
00095 virtual std::string getApiKeyEnvironmentVariable() const override;
00096 virtual bool loadApiKeyFromEnvironment() override;
00097
00098 // -----
00099 // Integration with GenESyS
00100 // -----
00101 virtual void setSimulatorFacade(SimulatorFacade* simulatorFacade) override;
00102 virtual SimulatorFacade* getSimulatorFacade() const override;
00103 virtual bool hasSimulatorFacade() const override;
00104
00105 // -----
00106 // Provider client
00107 // -----
00108 virtual void setProviderClient(AIProviderClient_if* client) override;
00109 virtual AIProviderClient_if* getProviderClient() const override;
00110 virtual bool hasProviderClient() const override;
00111
00112 bool createAndAttachProviderClient();
00113
00114 // -----
00115 // Knowledge base
00116 // -----
00117 virtual void clearKnowledgeBase() override;
00118 virtual void addKnowledgeItem(const GenesysKnowledgeItem& item) override;
00119 virtual std::vector<GenesysKnowledgeItem> getKnowledgeItems() const override;
00120 virtual GenesysKnowledgeBaseSummary getKnowledgeBaseSummary() const override;
00121 virtual std::string exportKnowledgeBaseAsMarkdown() const override;
00122 virtual bool refreshKnowledgeBaseFromSimulator(std::string* message = nullptr) override;
00123
00124 // -----
00125 // Workflow
00126 // -----
00127 virtual bool checkReady(std::string* message = nullptr) const override;
00128
00129 virtual AIAssistantResponse analyzePrompt(const AIAssistantRequest& request) override;
00130 virtual AIAssistantResponse createModelPlan(const AIAssistantRequest& request) override;
00131 virtual AIAssistantResponse buildModel(const AIAssistantRequest& request) override;
00132 virtual AIAssistantResponse configureSimulation(const AIAssistantRequest& request) override;
00133 virtual AIAssistantResponse runSimulation(const AIAssistantRequest& request) override;
00134 virtual AIAssistantResponse collectResults(const AIAssistantRequest& request) override;
00135 virtual AIAssistantResponse execute(const AIAssistantRequest& request) override;
00136
00137 // -----
00138 // Diagnostics
00139 // -----
00140 virtual ExecutionState getState() const override;
00141 virtual std::string getLastErrorMessage() const override;
00142 virtual std::string getLastDiagnostics() const override;
00143 virtual void clearDiagnostics() override;
00144
00145 // -----
00146 // Audit log
00147 // -----
00148 virtual std::vector<AuditEntry> getAuditEntries(unsigned int maxEntries) const override;
00149 virtual std::string getAuditLogPath() const override;
00150 virtual unsigned int exportAuditLog(const std::string& csvPath) const override;
00151
00152 private:
00153 // -----
00154 // Private helpers
00155 // -----
00156 bool _providerRequiresApiKey() const;
00157

```

```

00183 bool _providerRequiresBaseUrl() const;
00184
00191 AIAssistantResponse _notImplementedResponse(const std::string& operationName,
00192 unsigned int plannedStage) const;
00193
00199 void _updateState();
00200
00209 void _appendAudit(const std::string& operation,
00210 const AIAssistantResponse& resp,
00211 long long durationMs,
00212 const std::string& promptPreview);
00213
00214 private:
00215 // -----
00216 // Private data members
00217 // -----
00218
00219 AIAssistantConfiguration _config;
00220
00230 std::string _apiKey;
00231
00233 SimulatorFacade* _simulatorFacade = nullptr;
00234
00243 AIProviderClient_if* _providerClient = nullptr;
00244 bool _ownsProviderClient = false;
00245
00247 std::vector<GenesysKnowledgeItem> _knowledgeItems;
00248
00250 ExecutionState _state = ExecutionState::NotConfigured;
00251
00253 std::string _lastError;
00254
00256 std::string _lastDiagnostics;
00257
00259 AIAuditLog _auditLog;
00260 };
00261
00262 #endif /* AIASSISTANTDEFAULTIMPL_H */

```

## 16.500 AIAuditLog.h

```

00001 /*
00002 * File: AIAuditLog.h
00003 * Author: Genesys Team
00004 *
00005 * Created on 28 de Maio de 2026
00006 */
00007
00008 #ifndef AIAUDITLOG_H
00009 #define AIAUDITLOG_H
00010
00011 #include <chrono>
00012 #include <string>
00013 #include <vector>
00014
00022 struct AuditEntry {
00023 std::chrono::system_clock::time_point timestamp;
00024 std::string operation;
00025 std::string provider;
00026 std::string modelName;
00027 std::string promptPreview;
00028 bool success = false;
00029 bool dryRun = false;
00030 long long durationMs = 0;
00031 std::string errorSummary;
00032 };
00033
00042 class AIAuditLog {
00043 public:
00044 explicit AIAuditLog(const std::string& logFilePath = "");
00045 ~AIAuditLog() = default;
00046
00047 AIAuditLog(const AIAuditLog&) = delete;
00048 AIAuditLog& operator=(const AIAuditLog&) = delete;
00049
00051 void append(const AuditEntry& entry);
00052
00057 std::vector<AuditEntry> loadRecentEntries(unsigned int maxEntries = 200) const;
00058
00060 std::string logFilePath() const { return _logFilePath; }
00061
00066 unsigned int exportCsv(const std::string& csvPath) const;
00067
00069 static std::string defaultLogFilePath();
00070

```

```

00071 private:
00072 static std::string _encodeEntry(const AuditEntry& entry);
00073 static bool _decodeEntry(const std::string& line, AuditEntry& out);
00074 void _ensureDirectoryExists() const;
00075
00076 std::string _logFilePath;
00077 };
00078
00079 #endif /* AIAUDITLOG_H */

```

## 16.501 AIConversationService.h

```

00001 #ifndef AICONVERSATIONSERVICE_H
00002 #define AICONVERSATIONSERVICE_H
00003
00004 #include "AIProviderClient_if.h"
00005
00006 #include <map>
00007 #include <memory>
00008 #include <string>
00009 #include <vector>
00010
00011 enum class AIConversationProvider { OpenAI, Anthropic, Local };
00012
00013 struct AIConversationConfiguration {
00014 AIConversationProvider provider = AIConversationProvider::Local;
00015 std::string model = "llama3.2";
00016 std::string baseUrl;
00017 std::string apiKeyEnvironmentVariable;
00018 std::string systemInstructions;
00019 double temperature = 0.2;
00020 unsigned int maxOutputTokens = 512;
00021 unsigned int timeoutSeconds = 120;
00022 bool highReasoningMode = false;
00023 bool conversationEnabled = true;
00024 unsigned int maxHistoryMessages = 20;
00025 };
00026
00027 struct AIConversationResponse {
00028 bool success = false;
00029 std::string content;
00030 std::string errorMessage;
00031 int httpStatusCode = 0;
00032 unsigned int inputTokens = 0;
00033 unsigned int outputTokens = 0;
00034 };
00035
00036 class AIConversationService {
00037 public:
00038 AIConversationService() = default;
00039 ~AIConversationService() = default;
00040
00041 void setConfiguration(const AIConversationConfiguration& configuration);
00042 const AIConversationConfiguration& getConfiguration() const;
00043 bool initialize(std::string& errorMessage);
00044 void setProviderClient(std::unique_ptr<AIProviderClient_if> client);
00045 AIConversationResponse send(const std::string& conversationKey, const std::string& prompt);
00046 void clearConversation(const std::string& conversationKey);
00047 void clearAllConversations();
00048 bool isInitialized() const;
00049
00050 private:
00051 std::vector<ChatMessage> _buildMessages(const std::string& key, const std::string& prompt) const;
00052 void _appendHistory(const std::string& key, const ChatMessage& message);
00053
00054 AIConversationConfiguration _configuration;
00055 std::unique_ptr<AIProviderClient_if> _client;
00056 std::map<std::string, std::vector<ChatMessage> _histories;
00057 };
00058
00059 #endif

```

## 16.502 AIProviderClient\_if.h

```

00001 /*
00002 * File: AIProviderClient_if.h
00003 * Author: Genesys Team
00004 *
00005 * Created on 28 de Maio de 2026
00006 */
00007
00008 #ifndef AIPROVIDERCLIENT_IF_H

```

```

00009 #define AIPROVIDERCLIENT_IF_H
00010
00011 #include <string>
00012 #include <vector>
00013
00014 // -----
00015 // LLM chat structures
00016 // -----
00017
00027 struct ChatMessage {
00028 std::string role;
00029 std::string content;
00030 };
00031
00035 struct ProviderRequest {
00036 std::string model;
00037 std::vector<ChatMessage> messages;
00038 double temperature = 0.2;
00039 unsigned int maxTokens = 4096;
00040 bool highReasoningMode = false;
00041 };
00042
00046 struct ProviderResponse {
00047 bool success = false;
00048 std::string content;
00049 std::string errorMessage;
00050 int httpStatusCode = 0;
00051 unsigned int inputTokens = 0;
00052 unsigned int outputTokens = 0;
00053 };
00054
00055 // -----
00056 // Interface
00057 // -----
00058
00077 class AIProviderClient_if {
00078 public:
00079 virtual ~AIProviderClient_if() = default;
00080
00081 // -----
00082 // Provider identification
00083 // -----
00084
00086 virtual std::string getProviderName() const = 0;
00087
00095 virtual bool isAvailable() const = 0;
00096
00097 // -----
00098 // Configuration
00099 // -----
00100
00102 virtual void setBaseUrl(const std::string& url) = 0;
00104 virtual std::string getBaseUrl() const = 0;
00105
00107 virtual void setTimeoutSeconds(unsigned int seconds) = 0;
00109 virtual unsigned int getTimeoutSeconds() const = 0;
00110
00111 // -----
00112 // Secrets -- same security contract as AIAssistant_if
00113 // -----
00114
00122 virtual void setApiKey(const std::string& apiKey) = 0;
00123
00127 virtual void clearApiKey() = 0;
00128
00130 virtual bool hasApiKey() const = 0;
00131
00132 // -----
00133 // Request
00134 // -----
00135
00144 virtual ProviderResponse send(const ProviderRequest& request) = 0;
00145 };
00146
00147 #endif /* AIPROVIDERCLIENT_IF_H */

```

## 16.503 AISecretStore.h

```

00001 /*
00002 * File: AISecretStore.h
00003 * Author: Genesys Team
00004 *
00005 * Created on 28 de Maio de 2026
00006 */
00007

```

```

00008 #ifndef AISECRETSTORE_H
00009 #define AISECRETSTORE_H
00010
00011 #include <optional>
00012 #include <string>
00013
00038 class AISecretStore {
00039 public:
00040 AISecretStore() = delete;
00041
00049 static bool isAvailable();
00050
00056 static bool save(const std::string& service,
00057 const std::string& account,
00058 const std::string& secret);
00059
00064 static std::optional<std::string> load(const std::string& service,
00065 const std::string& account);
00066
00071 static bool remove(const std::string& service,
00072 const std::string& account);
00073
00074 private:
00075 static std::string _shellEscape(const std::string& s);
00076 static std::string _pipeRead(const std::string& cmd);
00077 };
00078
00079 #endif /* AISECRETSTORE_H */

```

## 16.504 AnthropicProviderClientImpl.h

```

00001 /*
00002 * File: AnthropicProviderClientImpl.h
00003 * Author: Genesys Team
00004 *
00005 * Created on 28 de Maio de 2026
00006 */
00007
00008 #ifndef ANTHROPICPROVIDERCLIENTIMPL_H
00009 #define ANTHROPICPROVIDERCLIENTIMPL_H
00010
00011 #include "HttpProviderClientBase.h"
00012
00035 class AnthropicProviderClientImpl : public HttpProviderClientBase {
00036 public:
00040 AnthropicProviderClientImpl();
00041 virtual ~AnthropicProviderClientImpl() = default;
00042
00043 // -----
00044 // Provider identification
00045 // -----
00046
00047 std::string getProviderName() const override;
00048
00049 bool isAvailable() const override;
00050
00051
00052 // -----
00053 // Anthropic-specific configuration
00054 // -----
00055
00064 void setApiVersion(const std::string& version);
00066 std::string getApiVersion() const;
00067
00075 void setThinkingBudgetTokens(unsigned int budget);
00077 unsigned int getThinkingBudgetTokens() const;
00078
00079 protected:
00080 std::string _buildRequestBody(const ProviderRequest& request) const override;
00081 ProviderResponse _parseResponseBody(const std::string& rawBody,
00082 int httpStatus) const override;
00083 std::string _authorizationHeader() const override;
00084 std::vector<std::string> _extraRequestHeaders() const override;
00085
00086 private:
00087 std::string _apiVersion = "2023-06-01";
00088 unsigned int _thinkingBudgetTokens = 10000;
00089 };
00090
00091 #endif /* ANTHROPICPROVIDERCLIENTIMPL_H */

```

## 16.505 HttpProviderClientBase.h

```

00001 /*
00002 * File: HttpProviderClientBase.h
00003 * Author: Genesys Team
00004 *
00005 * Created on 28 de Maio de 2026
00006 */
00007
00008 #ifndef HTTPPROVIDERCLIENTBASE_H
00009 #define HTTPPROVIDERCLIENTBASE_H
00010
00011 #include "AIPProviderClient_if.h"
00012
00013 #include <string>
00014 #include <vector>
00015
00049 class HttpProviderClientBase : public AIPProviderClient_if {
00050 public:
00051 HttpProviderClientBase();
00052 virtual ~HttpProviderClientBase();
00053
00054 // Not copyable: API key in member string, non-owning semantics.
00055 HttpProviderClientBase(const HttpProviderClientBase&) = delete;
00056 HttpProviderClientBase& operator=(const HttpProviderClientBase&) = delete;
00057
00058 // -----
00059 // AIPProviderClient_if -- common implementations
00060 // -----
00061
00062 bool isAvailable() const override;
00063
00064 void setBaseUrl(const std::string& url) override;
00065 std::string getBaseUrl() const override;
00066
00067 void setTimeoutSeconds(unsigned int seconds) override;
00068 unsigned int getTimeoutSeconds() const override;
00069
00070 void setApiKey(const std::string& apiKey) override;
00071 void clearApiKey() override;
00072 bool hasApiKey() const override;
00073
00087 ProviderResponse send(const ProviderRequest& request) override;
00088
00089 protected:
00090 // -----
00091 // Subclass hooks (pure virtual)
00092 // -----
00093
00095 virtual std::string _buildRequestBody(const ProviderRequest& request) const = 0;
00096
00104 virtual ProviderResponse _parseResponseBody(const std::string& rawBody,
00105 int httpStatus) const = 0;
00106
00118 virtual std::string _authorizationHeader() const = 0;
00119
00130 virtual std::vector<std::string> _extraRequestHeaders() const;
00131
00132 public:
00133 // -----
00134 // JSON utilities -- public static; usable by any caller that needs
00135 // lightweight JSON field extraction without a full parser dependency.
00136 // -----
00137
00139 static std::string _escapeJsonString(const std::string& s);
00140
00145 static std::string _extractJsonString(const std::string& json,
00146 const std::string& key);
00147
00152 static int _extractJsonInt(const std::string& json, const std::string& key);
00153
00158 static double _extractJsonDouble(const std::string& json, const std::string& key);
00159
00161 static std::string _buildMessagesArray(const std::vector<ChatMessage>& messages);
00162
00167 static std::string _extractFirstArrayElement(const std::string& json,
00168 const std::string& arrayKey);
00169
00170 protected:
00171 std::string _baseUrl;
00172 unsigned int _timeoutSeconds = 30;
00173
00183 std::string _apiKey;
00184 };
00185
00186 #endif /* HTTPPROVIDERCLIENTBASE_H */

```

## 16.506 LocalProviderClientImpl.h

```

00001 /*
00002 * File: LocalProviderClientImpl.h
00003 * Author: Genesys Team
00004 *
00005 * Created on 28 de Maio de 2026
00006 */
00007
00008 #ifndef LOCALPROVIDERCLIENTIMPL_H
00009 #define LOCALPROVIDERCLIENTIMPL_H
00010
00011 #include "HttpProviderClientBase.h"
00012
00013 class LocalProviderClientImpl : public HttpProviderClientBase {
00014 public:
00015 LocalProviderClientImpl();
00016 virtual ~LocalProviderClientImpl() = default;
00017
00018 // -----
00019 // Provider identification
00020 // -----
00021
00022 std::string getProviderName() const override;
00023
00024 bool isAvailable() const override;
00025
00026 protected:
00027 std::string _buildRequestBody(const ProviderRequest& request) const override;
00028 ProviderResponse _parseResponseBody(const std::string& rawBody,
00029 int httpStatus) const override;
00030
00031 std::string _authorizationHeader() const override;
00032 };
00033 #endif /* LOCALPROVIDERCLIENTIMPL_H */

```

## 16.507 OpenAIProviderClientImpl.h

```

00001 /*
00002 * File: OpenAIProviderClientImpl.h
00003 * Author: Genesys Team
00004 *
00005 * Created on 28 de Maio de 2026
00006 */
00007
00008 #ifndef OPENAIPROVIDERIMPL_H
00009 #define OPENAIPROVIDERIMPL_H
00010
00011 #include "HttpProviderClientBase.h"
00012
00013 class OpenAIProviderClientImpl : public HttpProviderClientBase {
00014 public:
00015 OpenAIProviderClientImpl();
00016 virtual ~OpenAIProviderClientImpl() = default;
00017
00018 // -----
00019 // Provider identification
00020 // -----
00021
00022 std::string getProviderName() const override;
00023
00024 bool isAvailable() const override;
00025
00026 // -----
00027 // Optional per-request headers
00028 // -----
00029
00030 void setOrganizationId(const std::string& orgId);
00031 std::string getOrganizationId() const;
00032
00033 void setProjectId(const std::string& projectId);
00034 std::string getProjectId() const;
00035
00036 protected:
00037 std::string _buildRequestBody(const ProviderRequest& request) const override;
00038 ProviderResponse _parseResponseBody(const std::string& rawBody,
00039 int httpStatus) const override;
00040 std::string _authorizationHeader() const override;
00041 std::vector<std::string> _extraRequestHeaders() const override;
00042 private:
00043 std::string _organizationId;
00044 std::string _projectId;

```



```

00077 };
00078
00079 #endif /* OPENAIPROVIDERIMPL_H */

```

## 16.508 BioKineticLawExpression.h

```

00001 #ifndef BIODIFFERENTIALALGEBRA_H
00002 #define BIODIFFERENTIALALGEBRA_H
00003
00004 #include <algorithm>
00005 #include <cctype>
00006 #include <cmath>
00007 #include <functional>
00008 #include <string>
00009 #include <vector>
00010
00020 class BioKineticLawExpression {
00021 public:
00022 using SymbolResolver = std::function<bool(const std::string& name, double& value)>;
00023
00024 bool evaluate(const std::string& expression, SymbolResolver resolver, double& value, std::string&
00025 errorMessage) const {
00026 Parser parser(expression, resolver, errorMessage);
00027 value = parser.parseExpression();
00028 parser.skipWhitespace();
00029 if (!errorMessage.empty()) {
00030 return false;
00031 }
00032 if (!parser.atEnd()) {
00033 errorMessage = "Unexpected token at position " + std::to_string(parser.position()) + ".";
00034 return false;
00035 }
00036 if (!std::isfinite(value)) {
00037 errorMessage = "Expression evaluated to a non-finite value.";
00038 return false;
00039 }
00040 return true;
00041 }
00042 private:
00043 class Parser {
00044 public:
00045 Parser(const std::string& expression, SymbolResolver resolver, std::string& errorMessage)
00046 : _expression(expression), _resolver(resolver), _errorMessage(errorMessage) {}
00047
00048 double parseExpression() {
00049 double value = parseTerm();
00050 while (_errorMessage.empty()) {
00051 skipWhitespace();
00052 if (consume('+')) {
00053 value += parseTerm();
00054 } else if (consume('-')) {
00055 value -= parseTerm();
00056 } else {
00057 break;
00058 }
00059 }
00060 return value;
00061 }
00062
00063 void skipWhitespace() {
00064 while (_position < _expression.size() && std::isspace(static_cast<unsigned
00065 char>(_expression[_position])) != 0) {
00066 ++_position;
00067 }
00068 }
00069
00070 bool atEnd() const {
00071 return _position >= _expression.size();
00072 }
00073
00074 unsigned int position() const {
00075 return static_cast<unsigned int>(_position);
00076 }
00077
00078 private:
00079 double parseTerm() {
00080 double value = parsePower();
00081 while (_errorMessage.empty()) {
00082 skipWhitespace();
00083 if (consume('*')) {
00084 value *= parsePower();
00085 } else if (consume('/')) {
00086 const double denominator = parsePower();

```

```

00087 if (denominator == 0.0) {
00088 _errorMessage = "Division by zero.";
00089 return 0.0;
00090 }
00091 value /= denominator;
00092 } else {
00093 break;
00094 }
00095 }
00096 return value;
00097 }
00098
00099 double parsePower() {
00100 double value = parseUnary();
00101 skipWhitespace();
00102 if (consume('^')) {
00103 value = std::pow(value, parsePower());
00104 }
00105 return value;
00106 }
00107
00108 double parseUnary() {
00109 skipWhitespace();
00110 if (consume('+')) {
00111 return parseUnary();
00112 }
00113 if (consume('-')) {
00114 return -parseUnary();
00115 }
00116 return parsePrimary();
00117 }
00118
00119 double parsePrimary() {
00120 skipWhitespace();
00121 if (_position >= _expression.size()) {
00122 _errorMessage = "Unexpected end of expression.";
00123 return 0.0;
00124 }
00125 if (consume('(')) {
00126 double value = parseExpression();
00127 skipWhitespace();
00128 if (!consume(')')) {
00129 _errorMessage = "Missing closing parenthesis.";
00130 return 0.0;
00131 }
00132 return value;
00133 }
00134 if (std::isdigit(static_cast<unsigned char>(_expression[_position])) != 0 ||
00135 _expression[_position] == '.') {
00136 return parseNumber();
00137 }
00138 if (isIdentifierStart(_expression[_position])) {
00139 const std::string identifier = parseIdentifier();
00140 skipWhitespace();
00141 if (consume('(')) {
00142 return parseFunction(identifier);
00143 }
00144 double value = 0.0;
00145 if (_resolver == nullptr || !_resolver(identifier, value)) {
00146 _errorMessage = "Unknown kinetic-law symbol \"" + identifier + "\".";
00147 return 0.0;
00148 }
00149 return value;
00150 }
00151 _errorMessage = "Unexpected token at position " + std::to_string(_position) + ".";
00152 return 0.0;
00153 }
00154
00155 double parseNumber() {
00156 const size_t start = _position;
00157 while (_position < _expression.size() &&
00158 (std::isdigit(static_cast<unsigned char>(_expression[_position])) != 0 ||
00159 _expression[_position] == '.')) {
00160 ++_position;
00161 }
00162 if (_position < _expression.size() && (_expression[_position] == 'e' ||
00163 _expression[_position] == 'E')) {
00164 ++_position;
00165 if (_position < _expression.size() && (_expression[_position] == '+' ||
00166 _expression[_position] == '-')) {
00167 ++_position;
00168 }
00169 while (_position < _expression.size() && std::isdigit(static_cast<unsigned
00170 char>(_expression[_position])) != 0) {
00171 ++_position;
00172 }
00173 }
00174 }

```

```

00169 try {
00170 return std::stod(_expression.substr(start, _position - start));
00171 } catch (...) {
00172 _errorMessage = "Invalid numeric literal at position " + std::to_string(start) + ".";
00173 return 0.0;
00174 }
00175 }
00176
00177 std::string parseIdentifier() {
00178 const size_t start = _position;
00179 ++_position;
00180 while (_position < _expression.size() && isIdentifierChar(_expression[_position])) {
00181 ++_position;
00182 }
00183 return _expression.substr(start, _position - start);
00184 }
00185
00186 double parseFunction(const std::string& name) {
00187 std::vector<double> args;
00188 skipWhitespace();
00189 if (!consume('(')) {
00190 while (_errorMessage.empty()) {
00191 args.push_back(parseExpression());
00192 skipWhitespace();
00193 if (consume(',')) {
00194 break;
00195 }
00196 if (!consume(')')) {
00197 _errorMessage = "Missing comma or closing parenthesis in function call.";
00198 return 0.0;
00199 }
00200 }
00201 }
00202 return applyFunction(name, args);
00203 }
00204
00205 double applyFunction(const std::string& name, const std::vector<double>& args) {
00206 if (name == "abs" && args.size() == 1) {
00207 return std::fabs(args[0]);
00208 }
00209 if (name == "exp" && args.size() == 1) {
00210 return std::exp(args[0]);
00211 }
00212 if (name == "log" && args.size() == 1) {
00213 if (args[0] <= 0.0) {
00214 _errorMessage = "log() requires a positive argument.";
00215 return 0.0;
00216 }
00217 return std::log(args[0]);
00218 }
00219 if (name == "sqrt" && args.size() == 1) {
00220 if (args[0] < 0.0) {
00221 _errorMessage = "sqrt() requires a non-negative argument.";
00222 return 0.0;
00223 }
00224 return std::sqrt(args[0]);
00225 }
00226 if (name == "min" && args.size() == 2) {
00227 return std::min(args[0], args[1]);
00228 }
00229 if (name == "max" && args.size() == 2) {
00230 return std::max(args[0], args[1]);
00231 }
00232 if (name == "pow" && args.size() == 2) {
00233 return std::pow(args[0], args[1]);
00234 }
00235 _errorMessage = "Unsupported function \"" + name + "\" or invalid argument count.";
00236 return 0.0;
00237 }
00238
00239 bool consume(char expected) {
00240 skipWhitespace();
00241 if (_position < _expression.size() && _expression[_position] == expected) {
00242 ++_position;
00243 return true;
00244 }
00245 return false;
00246 }
00247
00248 bool isIdentifierStart(char ch) const {
00249 return std::isalpha(static_cast<unsigned char>(ch)) != 0 || ch == '_';
00250 }
00251
00252 bool isIdentifierChar(char ch) const {
00253 return std::isalnum(static_cast<unsigned char>(ch)) != 0 || ch == '_' || ch == '.';
00254 }
00255

```

```

00256 private:
00257 const std::string& _expression;
00258 SymbolResolver _resolver;
00259 std::string& _errorMessage;
00260 size_t _position = 0;
00261 };
00262 };
00263
00264 #endif /* BIOKINETICLAWEXPRESSION_H */

```

## 16.509 BioSimulationAnalysis.h

```

00001 #ifndef BIOSIMULATIONANALYSIS_H
00002 #define BIOSIMULATIONANALYSIS_H
00003
00004 #include <algorithm>
00005 #include <cmath>
00006 #include <string>
00007 #include <vector>
00008
00009 #include "tools/Biochemical/BioSimulationResult.h"
00010 #include "tools/Biochemical/MassActionOdeSystem.h"
00011
00012 struct BioStoichiometryMatrix {
00013 std::vector<std::string> speciesNames;
00014 std::vector<std::string> reactionNames;
00015 std::vector<std::vector<double>> coefficients;
00016
00017 double coefficient(unsigned int speciesIndex, unsigned int reactionIndex) const {
00018 if (speciesIndex >= coefficients.size() || reactionIndex >= reactionNames.size()) {
00019 return 0.0;
00020 }
00021 return coefficients[speciesIndex][reactionIndex];
00022 }
00023 };
00024
00025 enum class BioReactionRateKind {
00026 Forward,
00027 Reverse,
00028 Net
00029 };
00030
00031 struct BioReactionRateSample {
00032 double time = 0.0;
00033 std::vector<double> forwardRates;
00034 std::vector<double> reverseRates;
00035 std::vector<double> netRates;
00036 };
00037
00038 struct BioReactionRateTimeCourse {
00039 std::vector<std::string> reactionNames;
00040 std::vector<BioReactionRateSample> samples;
00041
00042 bool hasReaction(const std::string& reactionName) const {
00043 return std::find(reactionNames.begin(), reactionNames.end(), reactionName) !=
00044 reactionNames.end();
00045 }
00046
00047 bool toDataset(const std::string& reactionName, BioReactionRateKind kind,
00048 SimulationResultsDataset* dataset,
00049 std::string* errorMessage = nullptr) const {
00050 if (dataset == nullptr) {
00051 if (errorMessage != nullptr) {
00052 *errorMessage = "Invalid dataset output parameter.";
00053 }
00054 return false;
00055 }
00056 const auto it = std::find(reactionNames.begin(), reactionNames.end(), reactionName);
00057 if (it == reactionNames.end()) {
00058 if (errorMessage != nullptr) {
00059 *errorMessage = "BioReactionRateTimeCourse does not contain reaction \"" +
00060 reactionName + "\".";
00061 }
00062 return false;
00063 }
00064 const unsigned int reactionIndex = static_cast<unsigned int>(it - reactionNames.begin());
00065 const std::string suffix = rateKindName(kind);
00066
00067 SimulationResultsDataset converted;
00068 converted.datasetName = reactionName + "." + suffix;
00069 converted.randomVariableName = reactionName + "." + suffix;
00070 converted.description = "Bio simulation reaction-rate time-course dataset.";
00071 converted.variableType = "Continuous numeric";
00072 converted.source = "BioReactionRateTimeCourse";

```

```

00071 converted.sourceDescription = "BioReactionRateTimeCourse";
00072 converted.expression = reactionName + "." + suffix;
00073 converted.expressionName = reactionName + "." + suffix;
00074 converted.formatKind = SimulationResultsDatasetFormat::GuiTabular;
00075 converted.recordFile = false;
00076 converted.timeDependent = true;
00077 for (unsigned int i = 0; i < samples.size(); ++i) {
00078 const std::vector<double>* values = valuesForKind(samples[i], kind);
00079 if (reactionIndex >= values->size()) {
00080 if (errorMessage != nullptr) {
00081 *errorMessage = "BioReactionRateTimeCourse sample is missing reaction \"" +
reactionName + "\".";
00082 }
00083 return false;
00084 }
00085 SimulationResultsObservation observation;
00086 observation.replication = 1;
00087 observation.time = samples[i].time;
00088 observation.hasTime = true;
00089 observation.value = (*values)[reactionIndex];
00090 observation.sourceLine = i + 1;
00091 converted.observations.push_back(observation);
00092 }
00093 *dataset = converted;
00094 return true;
00095 }
00096 }
00097
00098 private:
00099 static std::string rateKindName(BioReactionRateKind kind) {
00100 switch (kind) {
00101 case BioReactionRateKind::Forward:
00102 return "forwardRate";
00103 case BioReactionRateKind::Reverse:
00104 return "reverseRate";
00105 case BioReactionRateKind::Net:
00106 return "netRate";
00107 }
00108 return "netRate";
00109 }
00110
00111 static const std::vector<double>* valuesForKind(const BioReactionRateSample& sample,
BioReactionRateKind kind) {
00112 switch (kind) {
00113 case BioReactionRateKind::Forward:
00114 return &sample.forwardRates;
00115 case BioReactionRateKind::Reverse:
00116 return &sample.reverseRates;
00117 case BioReactionRateKind::Net:
00118 return &sample.netRates;
00119 }
00120 return &sample.netRates;
00121 }
00122 };
00123
00124 struct BioSpeciesDerivative {
00125 std::string speciesName;
00126 double derivative = 0.0;
00127 };
00128
00129 struct BioSteadyStateCheck {
00130 bool steady = false;
00131 double maxAbsoluteDerivative = 0.0;
00132 double tolerance = 0.0;
00133 std::vector<BioSpeciesDerivative> derivatives;
00134 };
00135
00136 struct BioParameterSensitivityEntry {
00137 std::string parameterName;
00138 double baseValue = 0.0;
00139 double delta = 0.0;
00140 double maxAbsoluteSensitivity = 0.0;
00141 std::vector<BioSpeciesDerivative> derivativeSensitivities;
00142 };
00143
00144 struct BioSensitivityScan {
00145 double time = 0.0;
00146 std::vector<std::string> speciesNames;
00147 std::vector<BioParameterSensitivityEntry> entries;
00148 };
00149
00150 class BioSimulationAnalysis {
00151 public:
00152 static BioStoichiometryMatrix buildStoichiometryMatrix(const MassActionOdeSystem& system) {
00153 BioStoichiometryMatrix matrix;
00154 matrix.speciesNames.reserve(system.species().size());
00155 matrix.reactionNames.reserve(system.reactions().size());

```

```

00156 matrix.coefficients.assign(system.species().size(),
std::vector<double>(system.reactions().size(), 0.0));
00157
00158 for (const MassActionOdeSystem::Species& species : system.species()) {
00159 matrix.speciesNames.push_back(species.name);
00160 }
00161 for (unsigned int reactionIndex = 0; reactionIndex < system.reactions().size();
++reactionIndex) {
00162 const MassActionOdeSystem::Reaction& reaction = system.reactions()[reactionIndex];
00163 matrix.reactionNames.push_back(reaction.name);
00164 for (const MassActionOdeSystem::StoichiometricTerm& reactant : reaction.reactants) {
00165 if (reactant.speciesIndex < matrix.coefficients.size()) {
00166 matrix.coefficients[reactant.speciesIndex][reactionIndex] -=
reactant.stoichiometry;
00167 }
00168 }
00169 for (const MassActionOdeSystem::StoichiometricTerm& product : reaction.products) {
00170 if (product.speciesIndex < matrix.coefficients.size()) {
00171 matrix.coefficients[product.speciesIndex][reactionIndex] += product.stoichiometry;
00172 }
00173 }
00174 }
00175 return matrix;
00176 }
00177
00178 static bool buildReactionRateTimeCourse(const MassActionOdeSystem& system, const
BioSimulationResult& result,
00179 BioReactionRateTimeCourse* timeCourse,
std::string* errorMessage = nullptr) {
00180
00181 if (timeCourse == nullptr) {
00182 if (errorMessage != nullptr) {
00183 *errorMessage = "Invalid reaction rate time-course output parameter.";
00184 }
00185 return false;
00186 }
00187
00188 BioReactionRateTimeCourse converted;
00189 converted.reactionNames.reserve(system.reactions().size());
00190 for (const MassActionOdeSystem::Reaction& reaction : system.reactions()) {
00191 converted.reactionNames.push_back(reaction.name);
00192 }
00193
00194 for (const BioSimulationSample& sample : result.getSamples()) {
00195 std::vector<double> y;
00196 if (!stateFromSample(system, sample, &y, errorMessage)) {
00197 return false;
00198 }
00199 BioReactionRateSample rateSample;
00200 rateSample.time = sample.time;
00201 rateSample.forwardRates.reserve(system.reactions().size());
00202 rateSample.reverseRates.reserve(system.reactions().size());
00203 rateSample.netRates.reserve(system.reactions().size());
00204 for (unsigned int reactionIndex = 0; reactionIndex < system.reactions().size();
++reactionIndex) {
00205 const double forwardRate = system.evaluateForwardRate(reactionIndex, y.data());
00206 const double reverseRate = system.evaluateReverseRate(reactionIndex, y.data());
00207 rateSample.forwardRates.push_back(forwardRate);
00208 rateSample.reverseRates.push_back(reverseRate);
00209 rateSample.netRates.push_back(forwardRate - reverseRate);
00210 }
00211 converted.samples.push_back(rateSample);
00212 }
00213
00214 *timeCourse = converted;
00215 return true;
00216 }
00217
00218 static bool checkSteadyState(const MassActionOdeSystem& system, const BioSimulationSample& sample,
double tolerance,
00219 BioSteadyStateCheck* check, std::string* errorMessage = nullptr) {
00220
00221 if (check == nullptr) {
00222 if (errorMessage != nullptr) {
00223 *errorMessage = "Invalid steady-state output parameter.";
00224 }
00225 return false;
00226 }
00227 if (tolerance < 0.0) {
00228 if (errorMessage != nullptr) {
00229 *errorMessage = "Steady-state tolerance must be non-negative.";
00230 }
00231 return false;
00232 }
00233
00234 std::vector<double> y;
00235 if (!stateFromSample(system, sample, &y, errorMessage)) {
00236 return false;
00237 }

```

```

00237 std::vector<double> dydt(system.dimension(), 0.0);
00238 system.evaluate(sample.time, y.data(), dydt.data());
00239
00240 BioSteadyStateCheck converted;
00241 converted.tolerance = tolerance;
00242 converted.derivatives.reserve(system.species().size());
00243 for (unsigned int i = 0; i < system.species().size(); ++i) {
00244 const double absoluteDerivative = std::abs(dydt[i]);
00245 converted.maxAbsoluteDerivative = std::max(converted.maxAbsoluteDerivative,
absoluteDerivative);
00246 converted.derivatives.push_back({system.species()[i].name, dydt[i]});
00247 }
00248 converted.steady = converted.maxAbsoluteDerivative <= tolerance;
00249
00250 *check = converted;
00251 return true;
00252 }
00253
00254 static bool scanLocalParameterSensitivity(const MassActionOdeSystem& system, const
BioSimulationSample& sample,
00255 double relativeStep, double absoluteStep,
BioSensitivityScan* scan,
00256 std::string* errorMessage = nullptr) {
00257 if (scan == nullptr) {
00258 if (errorMessage != nullptr) {
00259 *errorMessage = "Invalid sensitivity scan output parameter.";
00260 }
00261 return false;
00262 }
00263 if (relativeStep < 0.0 || absoluteStep <= 0.0) {
00264 if (errorMessage != nullptr) {
00265 *errorMessage = "Sensitivity scan steps must satisfy relativeStep >= 0 and
absoluteStep > 0.";
00266 }
00267 return false;
00268 }
00269
00270 std::vector<double> y;
00271 if (!stateFromSample(system, sample, &y, errorMessage)) {
00272 return false;
00273 }
00274 std::vector<double> baseDerivatives;
00275 evaluateDerivatives(system, sample.time, y, &baseDerivatives);
00276
00277 BioSensitivityScan converted;
00278 converted.time = sample.time;
00279 converted.speciesNames.reserve(system.species().size());
00280 for (const MassActionOdeSystem::Species& species : system.species()) {
00281 converted.speciesNames.push_back(species.name);
00282 }
00283
00284 const std::vector<ParameterTarget> targets = parameterTargets(system);
00285 for (const ParameterTarget& target : targets) {
00286 std::vector<MassActionOdeSystem::Reaction> reactions = system.reactions();
00287 double baseValue = 0.0;
00288 if (!applyPerturbation(target, relativeStep, absoluteStep, &reactions, &baseValue)) {
00289 continue;
00290 }
00291
00292 MassActionOdeSystem perturbed(system.species(), reactions);
00293 std::vector<double> perturbedDerivatives;
00294 evaluateDerivatives(perturbed, sample.time, y, &perturbedDerivatives);
00295
00296 const double delta = deltaForValue(baseValue, relativeStep, absoluteStep);
00297 BioParameterSensitivityEntry entry;
00298 entry.parameterName = target.name;
00299 entry.baseValue = baseValue;
00300 entry.delta = delta;
00301 entry.derivativeSensitivities.reserve(system.species().size());
00302 for (unsigned int i = 0; i < system.species().size(); ++i) {
00303 const double sensitivity = (perturbedDerivatives[i] - baseDerivatives[i]) / delta;
00304 entry.maxAbsoluteSensitivity = std::max(entry.maxAbsoluteSensitivity,
std::abs(sensitivity));
00305 entry.derivativeSensitivities.push_back({system.species()[i].name, sensitivity});
00306 }
00307 converted.entries.push_back(entry);
00308 }
00309
00310 *scan = converted;
00311 return true;
00312 }
00313
00314 private:
00315 enum class ParameterTargetKind {
00316 ForwardRateConstant,
00317 ReverseRateConstant,
00318 NamedParameter

```

```

00319 };
00320
00321 struct ParameterTarget {
00322 ParameterTargetKind kind = ParameterTargetKind::NamedParameter;
00323 unsigned int reactionIndex = 0;
00324 std::string name;
00325 };
00326
00327 static void evaluateDerivatives(const MassActionOdeSystem& system, double time, const
std::vector<double>& y,
00328 std::vector<double>* derivatives) {
00329 derivatives->assign(system.dimension(), 0.0);
00330 system.evaluate(time, y.data(), derivatives->data());
00331 }
00332
00333 static std::vector<ParameterTarget> parameterTargets(const MassActionOdeSystem& system) {
00334 std::vector<ParameterTarget> targets;
00335 std::vector<std::string> namedParameters;
00336 for (unsigned int reactionIndex = 0; reactionIndex < system.reactions().size();
++reactionIndex) {
00337 const MassActionOdeSystem::Reaction& reaction = system.reactions()[reactionIndex];
00338 if (reaction.kineticLawExpression.empty()) {
00339 targets.push_back({ParameterTargetKind::ForwardRateConstant, reactionIndex,
reaction.name + ".rateConstant"});
00340 }
00341 if (reaction.reversible && reaction.reverseKineticLawExpression.empty()) {
00342 targets.push_back({ParameterTargetKind::ReverseRateConstant, reactionIndex,
reaction.name + ".reverseRateConstant"});
00343 }
00344 for (const auto& parameter : reaction.parameters) {
00345 if (std::find(namedParameters.begin(), namedParameters.end(), parameter.first) ==
namedParameters.end()) {
00346 namedParameters.push_back(parameter.first);
00347 targets.push_back({ParameterTargetKind::NamedParameter, reactionIndex,
parameter.first});
00348 }
00349 }
00350 }
00351 return targets;
00352 }
00353
00354 static double deltaForValue(double value, double relativeStep, double absoluteStep) {
00355 return std::max(std::abs(value) * relativeStep, absoluteStep);
00356 }
00357
00358 static bool applyPerturbation(const ParameterTarget& target, double relativeStep, double
absoluteStep,
00359 std::vector<MassActionOdeSystem::Reaction>* reactions, double*
baseValue) {
00360 if (reactions == nullptr || baseValue == nullptr || target.reactionIndex >= reactions->size())
{
00361 return false;
00362 }
00363 if (target.kind == ParameterTargetKind::ForwardRateConstant) {
00364 *baseValue = (*reactions)[target.reactionIndex].rateConstant;
00365 (*reactions)[target.reactionIndex].rateConstant += deltaForValue(*baseValue, relativeStep,
absoluteStep);
00366 return true;
00367 }
00368 if (target.kind == ParameterTargetKind::ReverseRateConstant) {
00369 *baseValue = (*reactions)[target.reactionIndex].reverseRateConstant;
00370 (*reactions)[target.reactionIndex].reverseRateConstant += deltaForValue(*baseValue,
relativeStep, absoluteStep);
00371 return true;
00372 }
00373
00374 bool found = false;
00375 for (MassActionOdeSystem::Reaction& reaction : *reactions) {
00376 for (auto& parameter : reaction.parameters) {
00377 if (parameter.first == target.name) {
00378 if (!found) {
00379 *baseValue = parameter.second;
00380 }
00381 parameter.second += deltaForValue(parameter.second, relativeStep, absoluteStep);
00382 found = true;
00383 }
00384 }
00385 }
00386 return found;
00387 }
00388
00389 static bool stateFromSample(const MassActionOdeSystem& system, const BioSimulationSample& sample,
std::vector<double>* y, std::string* errorMessage) {
00390 if (y == nullptr) {
00391 if (errorMessage != nullptr) {
00392 *errorMessage = "Invalid state output parameter.";
00393 }
00394 }

```



```

00395 return false;
00396 }
00397 y->assign(system.species().size(), 0.0);
00398 for (unsigned int speciesIndex = 0; speciesIndex < system.species().size(); ++speciesIndex) {
00399 const std::string& speciesName = system.species()[speciesIndex].name;
00400 bool found = false;
00401 for (const BioSimulationSpeciesAmount& amount : sample.species) {
00402 if (amount.speciesName == speciesName) {
00403 (*y)[speciesIndex] = amount.amount;
00404 found = true;
00405 break;
00406 }
00407 }
00408 if (!found) {
00409 if (errorMessage != nullptr) {
00410 *errorMessage = "BioSimulationSample is missing species \"" + speciesName + "\".";
00411 }
00412 return false;
00413 }
00414 }
00415 return true;
00416 }
00417 };
00418
00419 #endif /* BIOSIMULATIONANALYSIS_H */

```

## 16.510 BioSimulationResult.h

```

00001 #ifndef BIOSIMULATIONRESULT_H
00002 #define BIOSIMULATIONRESULT_H
00003
00004 #include <algorithm>
00005 #include <string>
00006 #include <vector>
00007
00008 #include "tools/Statistics/SimulationResultsDataset.h"
00009
00010 struct BioSimulationSpeciesAmount {
00011 std::string speciesName;
00012 double amount = 0.0;
00013 };
00014
00015 struct BioSimulationSample {
00016 double time = 0.0;
00017 std::vector<BioSimulationSpeciesAmount> species;
00018 };
00019
00020 class BioSimulationResult {
00021 public:
00022 void clear() {
00023 _networkName.clear();
00024 _startTime = 0.0;
00025 _stopTime = 0.0;
00026 _stepSize = 0.0;
00027 _speciesNames.clear();
00028 _samples.clear();
00029 }
00030
00031 void configure(std::string networkName, double startTime, double stopTime, double stepSize,
00032 const std::vector<std::string>& speciesNames) {
00033 _networkName = networkName;
00034 _startTime = startTime;
00035 _stopTime = stopTime;
00036 _stepSize = stepSize;
00037 _speciesNames = speciesNames;
00038 _samples.clear();
00039 }
00040
00041 void appendSample(double time, const std::vector<BioSimulationSpeciesAmount>& speciesAmounts) {
00042 BioSimulationSample sample;
00043 sample.time = time;
00044 sample.species = speciesAmounts;
00045 _samples.push_back(sample);
00046 }
00047
00048 bool empty() const {
00049 return _samples.empty();
00050 }
00051
00052 unsigned int sampleCount() const {
00053 return static_cast<unsigned int>(_samples.size());
00054 }
00055
00056 const std::string& getNetworkName() const {
00057 return _networkName;

```

```

00058 }
00059
00060 double getStartTime() const {
00061 return _startTime;
00062 }
00063
00064 double getStopTime() const {
00065 return _stopTime;
00066 }
00067
00068 double getStepSize() const {
00069 return _stepSize;
00070 }
00071
00072 const std::vector<std::string>& getSpeciesNames() const {
00073 return _speciesNames;
00074 }
00075
00076 const std::vector<BioSimulationSample>& getSamples() const {
00077 return _samples;
00078 }
00079
00080 bool hasSpecies(const std::string& speciesName) const {
00081 return std::find(_speciesNames.begin(), _speciesNames.end(), speciesName) !=
00082 _speciesNames.end();
00083 }
00084
00085 bool toDataset(const std::string& speciesName, SimulationResultsDataset* dataset,
00086 std::string* errorMessage = nullptr) const {
00087 if (dataset == nullptr) {
00088 if (errorMessage != nullptr) {
00089 *errorMessage = "Invalid dataset output parameter.";
00090 }
00091 return false;
00092 }
00093 if (!hasSpecies(speciesName)) {
00094 if (errorMessage != nullptr) {
00095 *errorMessage = "BioSimulationResult does not contain species \"" + speciesName +
00096 "\".";
00097 }
00098 return false;
00099 }
00100 SimulationResultsDataset converted;
00101 converted.datasetName = _networkName + "." + speciesName;
00102 converted.randomVariableName = speciesName;
00103 converted.description = "Bio simulation species time-course dataset.";
00104 converted.variableType = "Continuous numeric";
00105 converted.source = _networkName;
00106 converted.sourceDescription = _networkName;
00107 converted.expression = speciesName;
00108 converted.expressionName = speciesName;
00109 converted.formatKind = SimulationResultsDatasetFormat::GuiTabular;
00110 converted.recordFile = false;
00111 converted.timeDependent = true;
00112 for (unsigned int i = 0; i < _samples.size(); ++i) {
00113 bool found = false;
00114 const double value = valueForSpecies(_samples[i], speciesName, &found);
00115 if (!found) {
00116 if (errorMessage != nullptr) {
00117 *errorMessage = "BioSimulationResult sample is missing species \"" + speciesName +
00118 "\".";
00119 }
00120 return false;
00121 }
00122 SimulationResultsObservation observation;
00123 observation.replication = 1;
00124 observation.time = _samples[i].time;
00125 observation.hasTime = true;
00126 observation.value = value;
00127 observation.sourceLine = i + 1;
00128 converted.observations.push_back(observation);
00129 }
00130 *dataset = converted;
00131 return true;
00132 }
00133
00134 std::vector<SimulationResultsDataset> toDatasets() const {
00135 std::vector<SimulationResultsDataset> datasets;
00136 for (const std::string& speciesName : _speciesNames) {
00137 SimulationResultsDataset dataset;
00138 if (toDataset(speciesName, &dataset)) {
00139 datasets.push_back(dataset);
00140 }
00141 }
00142 return datasets;

```

```

00142 }
00143
00144 private:
00145 double valueForSpecies(const BioSimulationSample& sample, const std::string& speciesName, bool*
found) const {
00146 for (const BioSimulationSpeciesAmount& amount : sample.species) {
00147 if (amount.speciesName == speciesName) {
00148 if (found != nullptr) {
00149 *found = true;
00150 }
00151 return amount.amount;
00152 }
00153 }
00154 if (found != nullptr) {
00155 *found = false;
00156 }
00157 return 0.0;
00158 }
00159
00160 private:
00161 std::string _networkName = "";
00162 double _startTime = 0.0;
00163 double _stopTime = 0.0;
00164 double _stepSize = 0.0;
00165 std::vector<std::string> _speciesNames;
00166 std::vector<BioSimulationSample> _samples;
00167 };
00168
00169 #endif /* BIOSIMULATIONRESULT_H */

```

## 16.511 GlpkFluxBalanceSolver.h

```

00001 #ifndef GLPKFLUXBALANCESOLVER_H
00002 #define GLPKFLUXBALANCESOLVER_H
00003
00004 #include "tools/Biochemical/MetabolicFluxBalanceSolver.h"
00005
00006 #ifdef GENESYS_HAVE_GLPK
00007
00008 #include <glpk.h>
00009 #include <cstring>
00010 #include <limits>
00011
00039 class GlpkFluxBalanceSolver {
00040 public:
00041 static MetabolicFluxBalanceSolver::Solution solve(const MetabolicFluxBalanceSolver::Problem&
problem) {
00042 MetabolicFluxBalanceSolver::Solution solution;
00043 const std::size_t nCols = problem.objective.size();
00044 const std::size_t nRows = problem.stoichiometry.size();
00045
00046 if (nCols == 0u) {
00047 solution.errorMessage = "Empty objective vector.";
00048 return solution;
00049 }
00050 if (problem.lowerBounds.size() != nCols || problem.upperBounds.size() != nCols) {
00051 solution.errorMessage = "Invalid flux bound vector dimensions.";
00052 return solution;
00053 }
00054
00055 glp_prob* lp = glp_create_prob();
00056 glp_set_prob_name(lp, "fba");
00057 glp_set_obj_dir(lp, problem.maximize ? GLP_MAX : GLP_MIN);
00058
00059 // Add constraint rows (one per metabolite: S_i * v = 0)
00060 if (nRows > 0u) {
00061 glp_add_rows(lp, static_cast<int>(nRows));
00062 for (std::size_t i = 0; i < nRows; ++i) {
00063 glp_set_row_bnds(lp, static_cast<int>(i) + 1, GLP_FX, 0.0, 0.0);
00064 }
00065 }
00066
00067 // Add flux variables (columns) with bounds and objective coefficients
00068 glp_add_cols(lp, static_cast<int>(nCols));
00069 for (std::size_t j = 0; j < nCols; ++j) {
00070 const int col = static_cast<int>(j) + 1;
00071 const double lb = problem.lowerBounds[j];
00072 const double ub = problem.upperBounds[j];
00073 const double kInf = std::numeric_limits<double>::infinity();
00074 if (lb == ub) {
00075 glp_set_col_bnds(lp, col, GLP_FX, lb, ub);
00076 } else if (lb <= -kInf && ub >= kInf) {
00077 glp_set_col_bnds(lp, col, GLP_FR, 0.0, 0.0);
00078 } else if (lb <= -kInf) {
00079 glp_set_col_bnds(lp, col, GLP_UP, 0.0, ub);

```

```

00080 } else if (ub >= kInf) {
00081 glp_set_col_bnds(lp, col, GLP_LO, lb, 0.0);
00082 } else {
00083 glp_set_col_bnds(lp, col, GLP_DB, lb, ub);
00084 }
00085 glp_set_obj_coef(lp, col, problem.objective[j]);
00086 }
00087
00088 // Load stoichiometry matrix as sparse triplets (GLPK uses 1-based indices)
00089 // Count non-zeros first
00090 std::size_t nnz = 0u;
00091 for (const auto& row : problem.stoichiometry) {
00092 for (double val : row) {
00093 if (val != 0.0) ++nnz;
00094 }
00095 }
00096
00097 if (nnz > 0u && nRows > 0u) {
00098 std::vector<int> ia(nnz + 1u);
00099 std::vector<int> ja(nnz + 1u);
00100 std::vector<double> ar(nnz + 1u);
00101 std::size_t k = 1u;
00102 for (std::size_t i = 0; i < nRows; ++i) {
00103 for (std::size_t j = 0; j < nCols && j < problem.stoichiometry[i].size(); ++j) {
00104 const double val = problem.stoichiometry[i][j];
00105 if (val != 0.0) {
00106 ia[k] = static_cast<int>(i) + 1;
00107 ja[k] = static_cast<int>(j) + 1;
00108 ar[k] = val;
00109 ++k;
00110 }
00111 }
00112 }
00113 glp_load_matrix(lp, static_cast<int>(nnz), ia.data(), ja.data(), ar.data());
00114 }
00115
00116 // Solve with simplex, messages suppressed
00117 glp_smpc parm;
00118 glp_init_smpc(&parm);
00119 parm.msg_lev = GLP_MSG_OFF;
00120 const int status = glp_simplex(lp, &parm);
00121
00122 if (status == 0 && glp_get_status(lp) == GLP_OPT) {
00123 solution.feasible = true;
00124 solution.objectiveValue = glp_get_obj_val(lp);
00125 solution.fluxes.resize(nCols);
00126 for (std::size_t j = 0; j < nCols; ++j) {
00127 solution.fluxes[j] = glp_get_col_prim(lp, static_cast<int>(j) + 1);
00128 }
00129 } else {
00130 const int glpStatus = glp_get_status(lp);
00131 if (glpStatus == GLP_INFEAS || glpStatus == GLP_NOFEAS) {
00132 solution.errorMessage = "GLPK: problem is infeasible.";
00133 } else if (glpStatus == GLP_UNBND) {
00134 solution.errorMessage = "GLPK: problem is unbounded.";
00135 } else {
00136 solution.errorMessage = "GLPK: simplex failed (status=" + std::to_string(status) + ",
00137 glpStatus=" + std::to_string(glpStatus) + ") .";
00138 }
00139 }
00140 glp_delete_prob(lp);
00141 return solution;
00142 }
00143 };
00144
00145 #endif /* GENESYS_HAVE_GLPK */
00146
00147 #endif /* GLPKFLUXBALANCESOLVER_H */

```

## 16.512 MassActionOdeSystem.h

```

00001 #ifndef MASSACTIONODESYSTEM_H
00002 #define MASSACTIONODESYSTEM_H
00003
00004 #include <algorithm>
00005 #include <cmath>
00006 #include <string>
00007 #include <utility>
00008 #include <vector>
00009
00010 #include "BioKineticLawExpression.h"
00011 #include "tools/Continuous/OdeSystem_if.h"
00012
00016 class MassActionOdeSystem : public OdeSystem_if {

```

```
00017 public:
00018 struct Species {
00019 std::string name;
00020 double initialAmount = 0.0;
00021 bool boundaryCondition = false;
00022 bool constant = false;
00023 };
00024
00025 struct StoichiometricTerm {
00026 unsigned int speciesIndex = 0;
00027 double stoichiometry = 1.0;
00028 };
00029
00030 struct Reaction {
00031 std::string name;
00032 double rateConstant = 0.0;
00033 std::vector<StoichiometricTerm> reactants;
00034 std::vector<StoichiometricTerm> products;
00035 std::string kineticLawExpression;
00036 std::vector<std::pair<std::string, double>> parameters;
00037 std::string reverseKineticLawExpression;
00038 double reverseRateConstant = 0.0;
00039 bool reversible = false;
00040 };
00041
00042 MassActionOdeSystem() = default;
00043
00044 MassActionOdeSystem(std::vector<Species> species, std::vector<Reaction> reactions)
00045 : _species(std::move(species)), _reactions(std::move(reactions)) {}
00046
00047 unsigned int dimension() const override {
00048 return static_cast<unsigned int>(_species.size());
00049 }
00050
00051 void evaluate(double, const double* y, double* dydt) const override {
00052 const unsigned int n = dimension();
00053 if (y == nullptr || dydt == nullptr) {
00054 return;
00055 }
00056
00057 for (unsigned int i = 0; i < n; ++i) {
00058 dydt[i] = 0.0;
00059 }
00060
00061 for (const Reaction& reaction : _reactions) {
00062 const double forwardRate = evaluateRate(reaction, reaction.kineticLawExpression,
00063 reaction.rateConstant, reaction.reactants, y);
00064 const double reverseRate = reaction.reversible ? evaluateRate(reaction,
00065 reaction.reverseKineticLawExpression, reaction.reverseRateConstant, reaction.products, y) : 0.0;
00066
00067 for (const StoichiometricTerm& reactant : reaction.reactants) {
00068 applyDerivative(reactant.speciesIndex, reactant.stoichiometry * (reverseRate -
00069 forwardRate), dydt);
00070 }
00071 for (const StoichiometricTerm& product : reaction.products) {
00072 applyDerivative(product.speciesIndex, product.stoichiometry * (forwardRate -
00073 reverseRate), dydt);
00074 }
00075 }
00076
00077 const std::vector<Species>& species() const {
00078 return _species;
00079 }
00080
00081 const std::vector<Reaction>& reactions() const {
00082 return _reactions;
00083 }
00084
00085 double evaluateForwardRate(unsigned int reactionIndex, const double* y) const {
00086 if (reactionIndex >= _reactions.size() || y == nullptr) {
00087 return 0.0;
00088 }
00089 const Reaction& reaction = _reactions[reactionIndex];
00090 return evaluateRate(reaction, reaction.kineticLawExpression, reaction.rateConstant,
00091 reaction.reactants, y);
00092 }
00093
00094 double evaluateReverseRate(unsigned int reactionIndex, const double* y) const {
00095 if (reactionIndex >= _reactions.size() || y == nullptr) {
00096 return 0.0;
00097 }
00098 const Reaction& reaction = _reactions[reactionIndex];
00099 if (!reaction.reversible) {
00100 return 0.0;
00101 }
00102 return evaluateRate(reaction, reaction.reverseKineticLawExpression, reaction.reverseRateConstant,
00103 reaction.products, y);
00104 }
00105 }
```

```

00099 return evaluateRate(reaction, reaction.reverseKineticLawExpression,
00100 reaction.reverseRateConstant, reaction.products, y);
00101 }
00102 double evaluateNetRate(unsigned int reactionIndex, const double* y) const {
00103 return evaluateForwardRate(reactionIndex, y) - evaluateReverseRate(reactionIndex, y);
00104 }
00105
00106 private:
00107 double evaluateRate(const Reaction& reaction, const std::string& kineticLawExpression, double
00108 rateConstant, const std::vector<StoichiometricTerm>& massActionTerms, const double* y) const {
00109 if (kineticLawExpression.empty()) {
00110 return evaluateMassActionRate(rateConstant, massActionTerms, y);
00111 }
00112 BioKineticLawExpression expression;
00113 double rate = 0.0;
00114 std::string errorMessage;
00115 const bool ok = expression.evaluate(kineticLawExpression,
00116 [this, &reaction, y](const std::string& name, double& value) {
00117 for (unsigned int i = 0; i < _species.size(); ++i) {
00118 if (_species[i].name == name) {
00119 value = std::max(0.0, y[i]);
00120 return true;
00121 }
00122 }
00123 for (const auto& parameter : reaction.parameters) {
00124 if (parameter.first == name) {
00125 value = parameter.second;
00126 return true;
00127 }
00128 }
00129 return false;
00130 },
00131 rate, errorMessage);
00132 return ok ? std::max(0.0, rate) : 0.0;
00133 }
00134
00135 double evaluateMassActionRate(double rateConstant, const std::vector<StoichiometricTerm>& terms,
00136 const double* y) const {
00137 double rate = rateConstant;
00138 for (const StoichiometricTerm& term : terms) {
00139 if (term.speciesIndex >= _species.size()) {
00140 return 0.0;
00141 }
00142 const double amount = std::max(0.0, y[term.speciesIndex]);
00143 rate *= std::pow(amount, term.stoichiometry);
00144 }
00145 return std::max(0.0, rate);
00146 }
00147
00148 void applyDerivative(unsigned int speciesIndex, double value, double* dydt) const {
00149 if (speciesIndex >= _species.size() || _species[speciesIndex].boundaryCondition ||
00150 _species[speciesIndex].constant) {
00151 return;
00152 }
00153 dydt[speciesIndex] += value;
00154 }
00155 private:
00156 std::vector<Species> _species;
00157 std::vector<Reaction> _reactions;
00158 };
00159 #endif /* MASSACTIONODESYSTEM_H */

```

## 16.513 MetabolicFluxBalanceSolver.h

```

00001 #ifndef METABOLICFLUXBALANCESOLVER_H
00002 #define METABOLICFLUXBALANCESOLVER_H
00003
00004 #include <algorithm>
00005 #include <cmath>
00006 #include <limits>
00007 #include <string>
00008 #include <vector>
00009
00023 class MetabolicFluxBalanceSolver {
00024 public:
00025 struct Problem {
00026 std::vector<std::vector<double>> stoichiometry;
00027 std::vector<double> lowerBounds;
00028 std::vector<double> upperBounds;
00029 std::vector<double> objective;
00030 bool maximize = true;

```

```

00031 };
00032
00033 struct Solution {
00034 bool feasible = false;
00035 double objectiveValue = 0.0;
00036 std::vector<double> fluxes;
00037 std::string errorMessage;
00038 };
00039
00040 public:
00041 static Solution solve(const Problem& problem) {
00042 Solution solution;
00043 const std::size_t variableCount = problem.objective.size();
00044 if (variableCount == 0u) {
00045 solution.errorMessage = "Empty objective vector.";
00046 return solution;
00047 }
00048 if (problem.lowerBounds.size() != variableCount || problem.upperBounds.size() !=
variableCount) {
00049 solution.errorMessage = "Invalid flux bound vector dimensions.";
00050 return solution;
00051 }
00052 for (const std::vector<double>& row : problem.stoichiometry) {
00053 if (row.size() != variableCount) {
00054 solution.errorMessage = "Invalid stoichiometry matrix dimensions.";
00055 return solution;
00056 }
00057 }
00058 for (std::size_t i = 0; i < variableCount; ++i) {
00059 if (problem.upperBounds[i] < problem.lowerBounds[i]) {
00060 solution.errorMessage = "Upper bound smaller than lower bound.";
00061 return solution;
00062 }
00063 }
00064
00065 std::vector<std::vector<double>> constraints = problem.stoichiometry;
00066 _removeRedundantRows(&constraints);
00067 const std::size_t rank = constraints.size();
00068 if (rank > variableCount) {
00069 solution.errorMessage = "Stoichiometry rank exceeds variable count.";
00070 return solution;
00071 }
00072
00073 std::vector<std::size_t> basis;
00074 for (std::size_t i = 0; i < rank; ++i) {
00075 basis.push_back(i);
00076 }
00077
00078 bool found = false;
00079 double bestObjective = problem.maximize ? -std::numeric_limits<double>::infinity()
: std::numeric_limits<double>::infinity();
00080 std::vector<double> bestFluxes(variableCount, 0.0);
00081
00082 do {
00083 if (!_isIndependentBasis(constraints, basis)) {
00084 continue;
00085 }
00086
00087 std::vector<std::size_t> nonBasis;
00088 for (std::size_t i = 0; i < variableCount; ++i) {
00089 if (std::find(basis.begin(), basis.end(), i) == basis.end()) {
00090 nonBasis.push_back(i);
00091 }
00092 }
00093
00094 const std::size_t combinations = std::size_t{1} << nonBasis.size();
00095 for (std::size_t mask = 0; mask < combinations; ++mask) {
00096 std::vector<double> fluxes(variableCount, 0.0);
00097 for (std::size_t k = 0; k < nonBasis.size(); ++k) {
00098 const std::size_t variableIndex = nonBasis[k];
00099 const bool chooseUpper = ((mask >> k) & 1u) != 0u;
00100 fluxes[variableIndex] = chooseUpper ? problem.upperBounds[variableIndex] :
problem.lowerBounds[variableIndex];
00101 }
00102
00103 std::vector<std::vector<double>> basisMatrix(rank, std::vector<double>(rank, 0.0));
00104 std::vector<double> rhs(rank, 0.0);
00105 for (std::size_t row = 0; row < rank; ++row) {
00106 for (std::size_t col = 0; col < rank; ++col) {
00107 basisMatrix[row][col] = constraints[row][basis[col]];
00108 }
00109 for (std::size_t variableIndex : nonBasis) {
00110 rhs[row] -= constraints[row][variableIndex] * fluxes[variableIndex];
00111 }
00112 }
00113
00114 std::vector<double> basisValues;

```

```

00116 if (!_solveLinearSystem(basisMatrix, rhs, &basisValues)) {
00117 continue;
00118 }
00119 for (std::size_t col = 0; col < rank; ++col) {
00120 fluxes[basis[col]] = basisValues[col];
00121 }
00122 if (!_withinBounds(fluxes, problem.lowerBounds, problem.upperBounds)) {
00123 continue;
00124 }
00125 if (!_satisfiesConstraints(constraints, fluxes)) {
00126 continue;
00127 }
00128
00129 const double objectiveValue = _dot(problem.objective, fluxes);
00130 if (!found ||
00131 (problem.maximize && objectiveValue > bestObjective + 1e-9) ||
00132 (!problem.maximize && objectiveValue < bestObjective - 1e-9)) {
00133 found = true;
00134 bestObjective = objectiveValue;
00135 bestFluxes = fluxes;
00136 }
00137 }
00138 } while (_nextBasis(variableCount, rank, &basis));
00139
00140 if (!found) {
00141 solution.errorMessage = "No feasible bounded flux solution found.";
00142 return solution;
00143 }
00144
00145 solution.feasible = true;
00146 solution.objectiveValue = bestObjective;
00147 solution.fluxes = std::move(bestFluxes);
00148 return solution;
00149 }
00150
00151 private:
00152 static void _removeRedundantRows(std::vector<std::vector<double>>* matrix) {
00153 if (matrix == nullptr) {
00154 return;
00155 }
00156 std::vector<std::vector<double>> filtered;
00157 for (const std::vector<double>& row : *matrix) {
00158 bool allZero = true;
00159 for (double value : row) {
00160 if (std::fabs(value) > 1e-9) {
00161 allZero = false;
00162 break;
00163 }
00164 }
00165 if (!allZero) {
00166 filtered.push_back(row);
00167 }
00168 }
00169
00170 std::vector<std::vector<double>> independent;
00171 for (const std::vector<double>& row : filtered) {
00172 std::vector<std::vector<double>> candidate = independent;
00173 candidate.push_back(row);
00174 if (_matrixRank(candidate) > independent.size()) {
00175 independent.push_back(row);
00176 }
00177 }
00178 *matrix = independent;
00179 }
00180
00181 static bool _isIndependentBasis(const std::vector<std::vector<double>>& constraints,
00182 const std::vector<std::size_t>& basis) {
00183 const std::size_t rank = constraints.size();
00184 if (basis.size() != rank) {
00185 return false;
00186 }
00187 if (rank == 0u) {
00188 return true;
00189 }
00190 std::vector<std::vector<double>> basisMatrix(rank, std::vector<double>(rank, 0.0));
00191 for (std::size_t row = 0; row < rank; ++row) {
00192 for (std::size_t col = 0; col < rank; ++col) {
00193 basisMatrix[row][col] = constraints[row][basis[col]];
00194 }
00195 }
00196 return _matrixRank(basisMatrix) == rank;
00197 }
00198
00199 static bool _solveLinearSystem(std::vector<std::vector<double>> matrix,
00200 std::vector<double> rhs,
00201 std::vector<double>* solution) {
00202 if (solution == nullptr) {

```



```

00203 return false;
00204 }
00205 const std::size_t n = matrix.size();
00206 if (rhs.size() != n) {
00207 return false;
00208 }
00209 for (const std::vector<double>& row : matrix) {
00210 if (row.size() != n) {
00211 return false;
00212 }
00213 }
00214
00215 for (std::size_t pivot = 0; pivot < n; ++pivot) {
00216 std::size_t pivotRow = pivot;
00217 double bestAbs = std::fabs(matrix[pivot][pivot]);
00218 for (std::size_t row = pivot + 1; row < n; ++row) {
00219 const double candidate = std::fabs(matrix[row][pivot]);
00220 if (candidate > bestAbs) {
00221 bestAbs = candidate;
00222 pivotRow = row;
00223 }
00224 }
00225 if (bestAbs <= 1e-9) {
00226 return false;
00227 }
00228 if (pivotRow != pivot) {
00229 std::swap(matrix[pivotRow], matrix[pivot]);
00230 std::swap(rhs[pivotRow], rhs[pivot]);
00231 }
00232
00233 const double pivotValue = matrix[pivot][pivot];
00234 for (std::size_t col = pivot; col < n; ++col) {
00235 matrix[pivot][col] /= pivotValue;
00236 }
00237 rhs[pivot] /= pivotValue;
00238
00239 for (std::size_t row = 0; row < n; ++row) {
00240 if (row == pivot) {
00241 continue;
00242 }
00243 const double factor = matrix[row][pivot];
00244 if (std::fabs(factor) <= 1e-12) {
00245 continue;
00246 }
00247 for (std::size_t col = pivot; col < n; ++col) {
00248 matrix[row][col] -= factor * matrix[pivot][col];
00249 }
00250 rhs[row] -= factor * rhs[pivot];
00251 }
00252 }
00253
00254 *solution = rhs;
00255 return true;
00256 }
00257
00258 static std::size_t _matrixRank(std::vector<std::vector<double>> matrix) {
00259 if (matrix.empty()) {
00260 return 0u;
00261 }
00262 const std::size_t rows = matrix.size();
00263 const std::size_t cols = matrix.front().size();
00264 std::size_t rank = 0u;
00265 std::size_t pivotRow = 0u;
00266 for (std::size_t col = 0; col < cols && pivotRow < rows; ++col) {
00267 std::size_t bestRow = pivotRow;
00268 double bestAbs = std::fabs(matrix[pivotRow][col]);
00269 for (std::size_t row = pivotRow + 1; row < rows; ++row) {
00270 const double candidate = std::fabs(matrix[row][col]);
00271 if (candidate > bestAbs) {
00272 bestAbs = candidate;
00273 bestRow = row;
00274 }
00275 }
00276 if (bestAbs <= 1e-9) {
00277 continue;
00278 }
00279 if (bestRow != pivotRow) {
00280 std::swap(matrix[bestRow], matrix[pivotRow]);
00281 }
00282 const double pivotValue = matrix[pivotRow][col];
00283 for (std::size_t activeCol = col; activeCol < cols; ++activeCol) {
00284 matrix[pivotRow][activeCol] /= pivotValue;
00285 }
00286 for (std::size_t row = pivotRow + 1; row < rows; ++row) {
00287 const double factor = matrix[row][col];
00288 if (std::fabs(factor) <= 1e-12) {
00289 continue;

```

```

00290 }
00291 for (std::size_t activeCol = col; activeCol < cols; ++activeCol) {
00292 matrix[row][activeCol] -= factor * matrix[pivotRow][activeCol];
00293 }
00294 }
00295 ++pivotRow;
00296 ++rank;
00297 }
00298 return rank;
00299 }
00300
00301 static bool _withinBounds(const std::vector<double>& fluxes,
00302 const std::vector<double>& lowerBounds,
00303 const std::vector<double>& upperBounds) {
00304 for (std::size_t i = 0; i < fluxes.size(); ++i) {
00305 if (fluxes[i] < lowerBounds[i] - 1e-7 || fluxes[i] > upperBounds[i] + 1e-7) {
00306 return false;
00307 }
00308 }
00309 return true;
00310 }
00311
00312 static bool _satisfiesConstraints(const std::vector<std::vector<double>&> constraints,
00313 const std::vector<double>& fluxes) {
00314 for (const std::vector<double>& row : constraints) {
00315 double total = 0.0;
00316 for (std::size_t i = 0; i < row.size(); ++i) {
00317 total += row[i] * fluxes[i];
00318 }
00319 if (std::fabs(total) > 1e-6) {
00320 return false;
00321 }
00322 }
00323 return true;
00324 }
00325
00326 static double _dot(const std::vector<double>& a, const std::vector<double>& b) {
00327 double value = 0.0;
00328 for (std::size_t i = 0; i < a.size(); ++i) {
00329 value += a[i] * b[i];
00330 }
00331 return value;
00332 }
00333
00334 static bool _nextBasis(std::size_t variableCount,
00335 std::size_t basisSize,
00336 std::vector<std::size_t*> basis) {
00337 if (basis == nullptr) {
00338 return false;
00339 }
00340 if (basisSize == 0u) {
00341 return false;
00342 }
00343 for (std::size_t i = basisSize; i-- > 0;) {
00344 const std::size_t maxValue = variableCount - basisSize + i;
00345 if ((*basis)[i] < maxValue) {
00346 ++(*basis)[i];
00347 for (std::size_t j = i + 1u; j < basisSize; ++j) {
00348 (*basis)[j] = (*basis)[j - 1u] + 1u;
00349 }
00350 return true;
00351 }
00352 }
00353 return false;
00354 }
00355 };
00356
00357 #endif /* METABOLICFLUXBALANCESOLVER_H */

```

## 16.514 WholeCellParameterReader.h

```

00001 #ifndef WHOLECELLPARAMETERREADER_H
00002 #define WHOLECELLPARAMETERREADER_H
00003
00004 #include <cctype>
00005 #include <fstream>
00006 #include <map>
00007 #include <sstream>
00008 #include <string>
00009
00030 class WholeCellParameterReader {
00031 public:
00032 static std::map<std::string, double> read(const std::string& filePath) {
00033 std::map<std::string, double> result;
00034 std::ifstream file(filePath);

```

```

00035 if (!file.is_open()) return result;
00036 std::ostringstream oss;
00037 oss << file.rdbuf();
00038 const std::string json = oss.str();
00039 _parseTopLevel(json, result);
00040 return result;
00041 }
00042
00043 private:
00044 static void _skipWhitespace(const std::string& s, std::size_t& pos) {
00045 while (pos < s.size() && std::isspace(static_cast<unsigned char>(s[pos]))) ++pos;
00046 }
00047
00048 static bool _parseString(const std::string& s, std::size_t& pos, std::string& out) {
00049 _skipWhitespace(s, pos);
00050 if (pos >= s.size() || s[pos] != '"') return false;
00051 ++pos;
00052 out.clear();
00053 while (pos < s.size() && s[pos] != '"') {
00054 if (s[pos] == '\\') {
00055 ++pos;
00056 if (pos < s.size()) { out += s[pos]; ++pos; }
00057 } else {
00058 out += s[pos++];
00059 }
00060 }
00061 if (pos < s.size()) ++pos;
00062 return true;
00063 }
00064
00065 static bool _parseDouble(const std::string& s, std::size_t& pos, double& out) {
00066 _skipWhitespace(s, pos);
00067 const std::size_t start = pos;
00068 if (pos < s.size() && (s[pos] == '-' || s[pos] == '+')) ++pos;
00069 bool hasDigit = false;
00070 while (pos < s.size() && std::isdigit(static_cast<unsigned char>(s[pos]))) { ++pos; hasDigit =
00071 true; }
00072 if (pos < s.size() && s[pos] == '.') {
00073 ++pos;
00074 while (pos < s.size() && std::isdigit(static_cast<unsigned char>(s[pos]))) { ++pos;
00075 hasDigit = true; }
00076 }
00077 if (pos < s.size() && (s[pos] == 'e' || s[pos] == 'E')) {
00078 ++pos;
00079 if (pos < s.size() && (s[pos] == '+' || s[pos] == '-')) ++pos;
00080 while (pos < s.size() && std::isdigit(static_cast<unsigned char>(s[pos]))) ++pos;
00081 }
00082 if (!hasDigit) return false;
00083 try { out = std::stod(s.substr(start, pos - start)); } catch (...) { return false; }
00084 return true;
00085 }
00086
00087 static void _skipValue(const std::string& s, std::size_t& pos) {
00088 _skipWhitespace(s, pos);
00089 if (pos >= s.size()) return;
00090 if (s[pos] == '"') {
00091 std::string dummy; _parseString(s, pos, dummy);
00092 } else if (s[pos] == '{') {
00093 int depth = 1; ++pos;
00094 while (pos < s.size() && depth > 0) {
00095 if (s[pos] == '{') ++depth;
00096 else if (s[pos] == '}') --depth;
00097 else if (s[pos] == '"') { std::string d; _parseString(s, pos, d); continue; }
00098 ++pos;
00099 }
00100 } else if (s[pos] == '[') {
00101 int depth = 1; ++pos;
00102 while (pos < s.size() && depth > 0) {
00103 if (s[pos] == '[') ++depth;
00104 else if (s[pos] == ']') --depth;
00105 else if (s[pos] == '"') { std::string d; _parseString(s, pos, d); continue; }
00106 ++pos;
00107 }
00108 } else {
00109 while (pos < s.size() && s[pos] != ',' && s[pos] != ']' && s[pos] != '}') ++pos;
00110 }
00111 }
00112
00113 // Parse {"key": value, ...} and call handler for each key-value pair
00114 static void _parseObject(const std::string& s, std::size_t& pos,
00115 const std::string& prefix,
00116 std::map<std::string, double>& out) {
00117 _skipWhitespace(s, pos);
00118 if (pos >= s.size() || s[pos] != '{') return;
00119 ++pos;
00120 while (pos < s.size()) {
00121 _skipWhitespace(s, pos);

```

```

00120 if (pos >= s.size() || s[pos] == '}') { if (pos < s.size()) ++pos; break; }
00121 if (s[pos] == ',') { ++pos; continue; }
00122 std::string key;
00123 if (!_parseString(s, pos, key)) { ++pos; continue; }
00124 _skipWhitespace(s, pos);
00125 if (pos < s.size() && s[pos] == ':') ++pos;
00126 _skipWhitespace(s, pos);
00127 const std::string fullKey = prefix.empty() ? key : prefix + "." + key;
00128 if (pos < s.size() && s[pos] == '{') {
00129 // nested object -- recurse one level
00130 _parseObject(s, pos, fullKey, out);
00131 } else if (pos < s.size() && s[pos] != '[' && s[pos] != '"') {
00132 // scalar value -- try as double
00133 double val = 0.0;
00134 const std::size_t before = pos;
00135 if (_parseDouble(s, pos, val)) {
00136 out[fullKey] = val;
00137 } else {
00138 pos = before;
00139 _skipValue(s, pos);
00140 }
00141 } else {
00142 // array or string -- skip
00143 _skipValue(s, pos);
00144 }
00145 }
00146 }
00147
00148 static void _parseTopLevel(const std::string& json, std::map<std::string, double>& out) {
00149 std::size_t pos = 0;
00150 _skipWhitespace(json, pos);
00151 if (pos >= json.size() || json[pos] != '{') return;
00152 ++pos;
00153 while (pos < json.size()) {
00154 _skipWhitespace(json, pos);
00155 if (json[pos] == '}') break;
00156 if (json[pos] == ',') { ++pos; continue; }
00157 std::string section;
00158 if (!_parseString(json, pos, section)) { ++pos; continue; }
00159 _skipWhitespace(json, pos);
00160 if (pos < json.size() && json[pos] == ':') ++pos;
00161 _parseObject(json, pos, section, out);
00162 }
00163 }
00164 };
00165
00166 #endif /* WHOLECELLPARAMETERREADER_H */

```

## 16.515 DiffusionMethodOfLinesSystem.h

```

00001 #ifndef DIFFUSIONMETHODOFLINESYSTEM_H
00002 #define DIFFUSIONMETHODOFLINESYSTEM_H
00003
00004 #include <cmath>
00005 #include <cstdint>
00006 #include <limits>
00007 #include <vector>
00008
00009 #include "OdeSystem_if.h"
00010
00011 class DiffusionMethodOfLinesSystem : public OdeSystem_if {
00012 public:
00013 static constexpr double kPi = 3.14159265358979323846;
00014
00015 enum class Boundary {
00016 Dirichlet,
00017 Neumann
00018 };
00019
00020 DiffusionMethodOfLinesSystem() = default;
00021
00022 DiffusionMethodOfLinesSystem(std::vector<unsigned int> shape,
00023 std::vector<double> spacing,
00024 double diffusionCoefficient,
00025 Boundary boundary)
00026 : _shape(std::move(shape)),
00027 _spacing(std::move(spacing)),
00028 _D(diffusionCoefficient),
00029 _boundary(boundary) {
00030 rebuild();
00031 }
00032
00033 unsigned int dimension() const override { return _totalNodes; }
00034
00035 void evaluate(double, const double* y, double* dydt) const override {

```

```

00036 if (!_valid || y == nullptr || dydt == nullptr) {
00037 return;
00038 }
00039 const unsigned int N = static_cast<unsigned int>(_shape.size());
00040 for (unsigned int node = 0; node < _totalNodes; ++node) {
00041 if (_boundary == Boundary::Dirichlet && isBoundaryNode(node)) {
00042 dydt[node] = 0.0;
00043 continue;
00044 }
00045 double laplacian = 0.0;
00046 for (unsigned int d = 0; d < N; ++d) {
00047 const unsigned int m = coordinate(node, d);
00048 const unsigned int last = _shape[d] - 1u;
00049 const std::size_t s = _stride[d];
00050 double secondDifference;
00051 if (m == 0u) {
00052 secondDifference = 2.0 * (y[node + s] - y[node]);
00053 } else if (m == last) {
00054 secondDifference = 2.0 * (y[node - s] - y[node]);
00055 } else {
00056 secondDifference = y[node + s] - 2.0 * y[node] + y[node - s];
00057 }
00058 laplacian += secondDifference * _invH2[d];
00059 }
00060 dydt[node] = _D * laplacian;
00061 }
00062 }
00063
00064 bool isValid() const { return _valid; }
00065 unsigned int totalNodes() const { return _totalNodes; }
00066 const std::vector<unsigned int>& shape() const { return _shape; }
00067 const std::vector<double>& spacing() const { return _spacing; }
00068 double diffusionCoefficient() const { return _D; }
00069 Boundary boundary() const { return _boundary; }
00070
00071 unsigned int coordinate(unsigned int node, unsigned int d) const {
00072 return static_cast<unsigned int>((node / _stride[d]) % _shape[d]);
00073 }
00074
00075 unsigned int nodeIndex(const std::vector<unsigned int>& multiIndex) const {
00076 std::size_t idx = 0;
00077 for (std::size_t d = 0; d < _shape.size(); ++d) {
00078 idx += static_cast<std::size_t>(multiIndex[d]) * _stride[d];
00079 }
00080 return static_cast<unsigned int>(idx);
00081 }
00082
00083 bool isBoundaryNode(unsigned int node) const {
00084 for (unsigned int d = 0; d < _shape.size(); ++d) {
00085 const unsigned int m = coordinate(node, d);
00086 if (m == 0u || m == _shape[d] - 1u) {
00087 return true;
00088 }
00089 }
00090 return false;
00091 }
00092
00093 void fillSineModes(const std::vector<unsigned int>& modes, double amplitude, double* y) const {
00094 const unsigned int N = static_cast<unsigned int>(_shape.size());
00095 for (unsigned int node = 0; node < _totalNodes; ++node) {
00096 double value = amplitude;
00097 for (unsigned int d = 0; d < N; ++d) {
00098 const unsigned int m = coordinate(node, d);
00099 const double L = (_shape[d] - 1u) * _spacing[d];
00100 const double x = m * _spacing[d];
00101 value *= std::sin(modes[d] * kPi * x / L);
00102 }
00103 y[node] = value;
00104 }
00105 }
00106
00107 void fillGaussian(double amplitude, double sigma, double* y) const {
00108 const unsigned int N = static_cast<unsigned int>(_shape.size());
00109 const double twoSigma2 = 2.0 * sigma * sigma;
00110 for (unsigned int node = 0; node < _totalNodes; ++node) {
00111 double r2 = 0.0;
00112 for (unsigned int d = 0; d < N; ++d) {
00113 const unsigned int m = coordinate(node, d);
00114 const double L = (_shape[d] - 1u) * _spacing[d];
00115 const double x = m * _spacing[d];
00116 const double dx = x - 0.5 * L;
00117 r2 += dx * dx;
00118 }
00119 y[node] = amplitude * std::exp(-r2 / twoSigma2);
00120 }
00121 }
00122

```

```

00123 double sineModeDecayRate(const std::vector<unsigned int>& modes) const {
00124 double lambda = 0.0;
00125 for (std::size_t d = 0; d < _shape.size(); ++d) {
00126 const double s = std::sin(modes[d] * kPi / (2.0 * (_shape[d] - 1u)));
00127 lambda += (4.0 / (_spacing[d] * _spacing[d])) * s * s;
00128 }
00129 return _D * lambda;
00130 }
00131
00132 double totalMass(const double* y) const {
00133 double mass = 0.0;
00134 double cellVolume = 1.0;
00135 for (double h : _spacing) {
00136 cellVolume *= h;
00137 }
00138 const unsigned int N = static_cast<unsigned int>(_shape.size());
00139 for (unsigned int node = 0; node < _totalNodes; ++node) {
00140 double weight = 1.0;
00141 for (unsigned int d = 0; d < N; ++d) {
00142 const unsigned int m = coordinate(node, d);
00143 if (m == 0u || m == _shape[d] - 1u) {
00144 weight *= 0.5;
00145 }
00146 }
00147 mass += weight * y[node];
00148 }
00149 return mass * cellVolume;
00150 }
00151
00152 double l2Norm(const double* y) const {
00153 double sum = 0.0;
00154 for (unsigned int i = 0; i < _totalNodes; ++i) {
00155 sum += y[i] * y[i];
00156 }
00157 return std::sqrt(sum);
00158 }
00159
00160 double maxValue(const double* y) const {
00161 double mx = -std::numeric_limits<double>::infinity();
00162 for (unsigned int i = 0; i < _totalNodes; ++i) {
00163 if (y[i] > mx) {
00164 mx = y[i];
00165 }
00166 }
00167 return mx;
00168 }
00169
00170 private:
00171 void rebuild() {
00172 _valid = false;
00173 _totalNodes = 0;
00174 _stride.clear();
00175 _invH2.clear();
00176 const std::size_t N = _shape.size();
00177 if (N == 0 || _spacing.size() != N || _D < 0.0) {
00178 return;
00179 }
00180 for (std::size_t d = 0; d < N; ++d) {
00181 if (_shape[d] < 3u || _spacing[d] <= 0.0) {
00182 return;
00183 }
00184 }
00185
00186 _stride.assign(N, 1);
00187 for (std::size_t d = N - 1; d-- > 0;) {
00188 _stride[d] = _stride[d + 1] * _shape[d + 1];
00189 }
00190 std::size_t total = 1;
00191 for (unsigned int n : _shape) {
00192 if (total > std::numeric_limits<std::size_t>::max() / n) {
00193 return;
00194 }
00195 total *= n;
00196 }
00197 if (total > std::numeric_limits<unsigned int>::max()) {
00198 return;
00199 }
00200 _totalNodes = static_cast<unsigned int>(total);
00201
00202 _invH2.assign(N, 0.0);
00203 for (std::size_t d = 0; d < N; ++d) {
00204 _invH2[d] = 1.0 / (_spacing[d] * _spacing[d]);
00205 }
00206 _valid = true;
00207 }
00208
00209 private:

```

```

00210 std::vector<unsigned int> _shape;
00211 std::vector<double> _spacing;
00212 double _D = 0.0;
00213 Boundary _boundary = Boundary::Dirichlet;
00214 bool _valid = false;
00215 unsigned int _totalNodes = 0;
00216 std::vector<std::size_t> _stride;
00217 std::vector<double> _invH2;
00218 };
00219
00220 #endif /* DIFFUSIONMETHODOFLINESSYSTEM_H */

```

## 16.516 DormandPrince54OdeSolver.h

```

00001 #ifndef DORMANDPRINCE54ODESOLVER_H
00002 #define DORMANDPRINCE54ODESOLVER_H
00003
00004 #include <algorithm>
00005 #include <cmath>
00006 #include <limits>
00007 #include <vector>
00008
00009 #include "OdeSolver_if.h"
00010
00011 class DormandPrince54OdeSolver : public OdeSolver_if {
00012 public:
00013 explicit DormandPrince54OdeSolver(double relativeTolerance = 1e-6,
00014 double absoluteTolerance = 1e-9,
00015 double safetyFactor = 0.9,
00016 double minScale = 0.2,
00017 double maxScale = 10.0,
00018 unsigned int maxSubsteps = 100000u)
00019 : _rtol(relativeTolerance),
00020 _atol(absoluteTolerance),
00021 _safety(safetyFactor),
00022 _minScale(minScale),
00023 _maxScale(maxScale),
00024 _maxSubsteps(maxSubsteps) {}
00025
00026 double getRelativeTolerance() const { return _rtol; }
00027 double getAbsoluteTolerance() const { return _atol; }
00028
00029 bool advance(const OdeSystem_if& system, double t0, double dt,
00030 const double* y0, double* y1) const override {
00031 const unsigned int n = system.dimension();
00032 if (n == 0 || y0 == nullptr || y1 == nullptr || dt < 0.0) {
00033 return false;
00034 }
00035 for (unsigned int i = 0; i < n; ++i) {
00036 y1[i] = y0[i];
00037 }
00038 if (dt == 0.0) {
00039 return true;
00040 }
00041
00042 std::vector<double> y(y0, y0 + n), yNext(n, 0.0), yErr(n, 0.0), tmp(n, 0.0);
00043 std::vector<double> k1(n, 0.0), k2(n, 0.0), k3(n, 0.0), k4(n, 0.0);
00044 std::vector<double> k5(n, 0.0), k6(n, 0.0), k7(n, 0.0);
00045 double t = t0;
00046 const double tEnd = t0 + dt;
00047 double h = dt;
00048 system.evaluate(t, y.data(), k1.data());
00049 bool firstSameAsLast = false;
00050
00051 unsigned int steps = 0;
00052 while (t < tEnd) {
00053 if (++steps > _maxSubsteps) {
00054 return false;
00055 }
00056 const double remaining = tEnd - t;
00057 if (remaining <= stepFloor(t)) {
00058 break;
00059 }
00060 if (h > remaining) {
00061 h = remaining;
00062 }
00063 if (firstSameAsLast) {
00064 k1 = k7;
00065 }
00066
00067 for (unsigned int i = 0; i < n; ++i) tmp[i] = y[i] + h * A21 * k1[i];
00068 system.evaluate(t + C2 * h, tmp.data(), k2.data());
00069 for (unsigned int i = 0; i < n; ++i) tmp[i] = y[i] + h * (A31 * k1[i] + A32 * k2[i]);
00070 system.evaluate(t + C3 * h, tmp.data(), k3.data());

```

```

00071 for (unsigned int i = 0; i < n; ++i) tmp[i] = y[i] + h * (A41 * k1[i] + A42 * k2[i] + A43
 * k3[i]);
00072 system.evaluate(t + C4 * h, tmp.data(), k4.data());
00073 for (unsigned int i = 0; i < n; ++i) tmp[i] = y[i] + h * (A51 * k1[i] + A52 * k2[i] + A53
 * k3[i] + A54 * k4[i]);
00074 system.evaluate(t + C5 * h, tmp.data(), k5.data());
00075 for (unsigned int i = 0; i < n; ++i) tmp[i] = y[i] + h * (A61 * k1[i] + A62 * k2[i] + A63
 * k3[i] + A64 * k4[i] + A65 * k5[i]);
00076 system.evaluate(t + C6 * h, tmp.data(), k6.data());
00077 for (unsigned int i = 0; i < n; ++i) yNext[i] = y[i] + h * (B5_1 * k1[i] + B5_3 * k3[i] +
 B5_4 * k4[i] + B5_5 * k5[i] + B5_6 * k6[i]);
00078 system.evaluate(t + h, yNext.data(), k7.data());
00079
00080 for (unsigned int i = 0; i < n; ++i) {
00081 yErr[i] = h * (E1 * k1[i] + E3 * k3[i] + E4 * k4[i] + E5 * k5[i] + E6 * k6[i] + E7 *
 k7[i]);
00082 }
00083
00084 double errNorm = 0.0;
00085 for (unsigned int i = 0; i < n; ++i) {
00086 if (!std::isfinite(yNext[i]) || !std::isfinite(yErr[i])) {
00087 return false;
00088 }
00089 const double sc = _atol + _rtol * std::max(std::fabs(y[i]), std::fabs(yNext[i]));
00090 const double ratio = (sc > 0.0) ? (yErr[i] / sc) : 0.0;
00091 errNorm += ratio * ratio;
00092 }
00093 errNorm = std::sqrt(errNorm / static_cast<double>(n));
00094
00095 if (errNorm <= 1.0) {
00096 t += h;
00097 y = yNext;
00098 firstSameAsLast = true;
00099 h *= scaleFactor(errNorm);
00100 } else {
00101 h *= scaleFactor(errNorm);
00102 firstSameAsLast = false;
00103 if (h < stepFloor(t)) {
00104 return false;
00105 }
00106 }
00107 }
00108
00109 for (unsigned int i = 0; i < n; ++i) {
00110 y1[i] = y[i];
00111 }
00112 return true;
00113 }
00114
00115 private:
00116 double scaleFactor(double errNorm) const {
00117 if (errNorm == 0.0) {
00118 return _maxScale;
00119 }
00120 return std::min(_maxScale, std::max(_minScale, _safety * std::pow(errNorm, -0.2)));
00121 }
00122
00123 static double stepFloor(double t) {
00124 return 16.0 * std::numeric_limits<double>::epsilon() * (std::fabs(t) + 1.0);
00125 }
00126
00127 private:
00128 double _rtol;
00129 double _atol;
00130 double _safety;
00131 double _minScale;
00132 double _maxScale;
00133 unsigned int _maxSubsteps;
00134
00135 static constexpr double C2 = 1.0 / 5.0;
00136 static constexpr double C3 = 3.0 / 10.0;
00137 static constexpr double C4 = 4.0 / 5.0;
00138 static constexpr double C5 = 8.0 / 9.0;
00139 static constexpr double C6 = 1.0;
00140 static constexpr double A21 = 1.0 / 5.0;
00141 static constexpr double A31 = 3.0 / 40.0;
00142 static constexpr double A32 = 9.0 / 40.0;
00143 static constexpr double A41 = 44.0 / 45.0;
00144 static constexpr double A42 = -56.0 / 15.0;
00145 static constexpr double A43 = 32.0 / 9.0;
00146 static constexpr double A51 = 19372.0 / 6561.0;
00147 static constexpr double A52 = -25360.0 / 2187.0;
00148 static constexpr double A53 = 64448.0 / 6561.0;
00149 static constexpr double A54 = -212.0 / 729.0;
00150 static constexpr double A61 = 9017.0 / 3168.0;
00151 static constexpr double A62 = -355.0 / 33.0;
00152 static constexpr double A63 = 46732.0 / 5247.0;

```



```

00153 static constexpr double A64 = 49.0 / 176.0;
00154 static constexpr double A65 = -5103.0 / 18656.0;
00155 static constexpr double B5_1 = 35.0 / 384.0;
00156 static constexpr double B5_3 = 500.0 / 1113.0;
00157 static constexpr double B5_4 = 125.0 / 192.0;
00158 static constexpr double B5_5 = -2187.0 / 6784.0;
00159 static constexpr double B5_6 = 11.0 / 84.0;
00160 static constexpr double E1 = 35.0 / 384.0 - 5179.0 / 57600.0;
00161 static constexpr double E3 = 500.0 / 1113.0 - 7571.0 / 16695.0;
00162 static constexpr double E4 = 125.0 / 192.0 - 393.0 / 640.0;
00163 static constexpr double E5 = -2187.0 / 6784.0 - (-92097.0 / 339200.0);
00164 static constexpr double E6 = 11.0 / 84.0 - 187.0 / 2100.0;
00165 static constexpr double E7 = -1.0 / 40.0;
00166 };
00167
00168 #endif /* DORMANDPRINCE54ODESOLVER_H */

```

## 16.517 OdeSolver\_if.h

```

00001 #ifndef ODESOLVER_IF_H
00002 #define ODESOLVER_IF_H
00003
00004 #include "OdeSystem_if.h"
00005
00013 class OdeSolver_if {
00014 public:
00015 virtual ~OdeSolver_if() = default;
00016 virtual bool advance(const OdeSystem_if& system, double t0, double dt, const double* y0, double*
 y1) const = 0;
00017 };
00018
00019 #endif /* ODESOLVER_IF_H */

```

## 16.518 OdeSolverFactory.h

```

00001 #ifndef ODESOLVERFACTORY_H
00002 #define ODESOLVERFACTORY_H
00003
00004 #include <functional>
00005 #include <map>
00006 #include <memory>
00007 #include <string>
00008 #include <vector>
00009
00010 #include "DormandPrince54OdeSolver.h"
00011 #include "OdeSolver_if.h"
00012 #include "RungeKutta4OdeSolver.h"
00013
00014 enum class OdeSolverType {
00015 RungeKutta4,
00016 DormandPrince54
00017 };
00018
00019 class OdeSolverFactory {
00020 public:
00021 using Creator = std::function<std::unique_ptr<OdeSolver_if>()>;
00022
00023 static const std::string& keyRungeKutta4() {
00024 static const std::string k = "RungeKutta4";
00025 return k;
00026 }
00027
00028 static const std::string& keyDormandPrince54() {
00029 static const std::string k = "DormandPrince54";
00030 return k;
00031 }
00032
00033 static const std::string& defaultKey() {
00034 return keyRungeKutta4();
00035 }
00036
00037 static std::unique_ptr<OdeSolver_if> create(OdeSolverType type) {
00038 switch (type) {
00039 case OdeSolverType::DormandPrince54:
00040 return std::make_unique<DormandPrince54OdeSolver>();
00041 case OdeSolverType::RungeKutta4:
00042 default:
00043 return std::make_unique<RungeKutta4OdeSolver>();
00044 }
00045 }
00046
00047 static std::unique_ptr<OdeSolver_if> create(const std::string& key) {

```

```

00048 auto& reg = registry();
00049 auto it = reg.find(key);
00050 if (it != reg.end()) {
00051 return it->second();
00052 }
00053 return std::make_unique<RungeKutta4OdeSolver>();
00054 }
00055
00056 static bool isRegistered(const std::string& key) {
00057 return registry().find(key) != registry().end();
00058 }
00059
00060 static std::vector<std::string> availableKeys() {
00061 std::vector<std::string> keys;
00062 keys.reserve(registry().size());
00063 for (const auto& entry : registry()) {
00064 keys.push_back(entry.first);
00065 }
00066 return keys;
00067 }
00068
00069 static bool registerCreator(const std::string& key, Creator creator) {
00070 return registry().emplace(key, std::move(creator)).second;
00071 }
00072
00073 static std::string toKey(OdeSolverType type) {
00074 switch (type) {
00075 case OdeSolverType::DormandPrince54:
00076 return keyDormandPrince54();
00077 case OdeSolverType::RungeKutta4:
00078 default:
00079 return keyRungeKutta4();
00080 }
00081 }
00082
00083 static bool fromKey(const std::string& key, OdeSolverType& out) {
00084 if (key == keyDormandPrince54()) {
00085 out = OdeSolverType::DormandPrince54;
00086 return true;
00087 }
00088 if (key == keyRungeKutta4()) {
00089 out = OdeSolverType::RungeKutta4;
00090 return true;
00091 }
00092 return false;
00093 }
00094
00095 private:
00096 static std::map<std::string, Creator>& registry() {
00097 static std::map<std::string, Creator> reg = [] {
00098 std::map<std::string, Creator> initial;
00099 initial.emplace(keyRungeKutta4(),
00100 [] { return std::make_unique<RungeKutta4OdeSolver>(); });
00101 initial.emplace(keyDormandPrince54(),
00102 [] { return std::make_unique<DormandPrince54OdeSolver>(); });
00103 return initial;
00104 }();
00105 return reg;
00106 }
00107 };
00108
00109 #endif /* ODESOLVERFACTORY_H */

```

## 16.519 OdeSystem\_if.h

```

00001 #ifndef ODESYSTEM_IF_H
00002 #define ODESYSTEM_IF_H
00003
00007 class OdeSystem_if {
00008 public:
00009 virtual ~OdeSystem_if() = default;
00010 virtual unsigned int dimension() const = 0;
00011 virtual void evaluate(double t, const double* y, double* dydt) const = 0;
00012 };
00013
00014 #endif /* ODESYSTEM_IF_H */

```

## 16.520 Quadrature\_if.h

```

00001 #ifndef QUADRATURE_IF_H
00002 #define QUADRATURE_IF_H
00003

```

```

00010 class Quadrature_if {
00011 public:
00012 virtual ~Quadrature_if() = default;
00013 virtual double integrate(double min, double max, double (*f)(double), double precision) const = 0;
00014 };
00015
00016 #endif /* QUADRATURE_IF_H */

```

## 16.521 RootFinder\_if.h

```

00001 #ifndef ROOTFINDER_IF_H
00002 #define ROOTFINDER_IF_H
00003
00007 class RootFinder_if {
00008 public:
00009 virtual ~RootFinder_if() = default;
00010 virtual double findRoot(double min, double max, double (*f)(double), double tolerance) const = 0;
00011 };
00012
00013 #endif /* ROOTFINDER_IF_H */

```

## 16.522 RungeKutta4OdeSolver.h

```

00001 #ifndef RUNGEKUTTA4ODESOLVER_H
00002 #define RUNGEKUTTA4ODESOLVER_H
00003
00004 #include <vector>
00005
00006 #include "OdeSolver_if.h"
00007
00011 class RungeKutta4OdeSolver : public OdeSolver_if {
00012 public:
00013 // Advances the ODE system by one step dt using classical RK4.
00014 // y0: state at t0 (input), y1: state at t0+dt (output). Returns false if inputs are invalid.
00015 bool advance(const OdeSystem_if& system, double t0, double dt, const double* y0, double* y1) const
00016 override {
00017 const unsigned int n = system.dimension();
00018 if (n == 0 || y0 == nullptr || y1 == nullptr || dt < 0.0) {
00019 return false;
00020 }
00021 std::vector<double> k1(n, 0.0);
00022 std::vector<double> k2(n, 0.0);
00023 std::vector<double> k3(n, 0.0);
00024 std::vector<double> k4(n, 0.0);
00025 std::vector<double> temp(n, 0.0);
00026
00027 // k1: slope at the beginning of the interval
00028 system.evaluate(t0, y0, k1.data());
00029 for (unsigned int i = 0; i < n; ++i) {
00030 temp[i] = y0[i] + 0.5 * dt * k1[i];
00031 }
00032
00033 // k2: slope at the midpoint, using k1 to estimate the state there
00034 system.evaluate(t0 + 0.5 * dt, temp.data(), k2.data());
00035 for (unsigned int i = 0; i < n; ++i) {
00036 temp[i] = y0[i] + 0.5 * dt * k2[i];
00037 }
00038
00039 // k3: slope at the midpoint again, using the improved k2 estimate
00040 system.evaluate(t0 + 0.5 * dt, temp.data(), k3.data());
00041 for (unsigned int i = 0; i < n; ++i) {
00042 temp[i] = y0[i] + dt * k3[i];
00043 }
00044
00045 // k4: slope at the end of the interval
00046 system.evaluate(t0 + dt, temp.data(), k4.data());
00047
00048 // weighted average: k1 and k4 count once, k2 and k3 count twice
00049 for (unsigned int i = 0; i < n; ++i) {
00050 y1[i] = y0[i] + (dt / 6.0) * (k1[i] + 2.0 * k2[i] + 2.0 * k3[i] + k4[i]);
00051 }
00052 return true;
00053 }
00054 };
00055
00056 #endif /* RUNGEKUTTA4ODESOLVER_H */

```

## 16.523 Solver\_if.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Integrator_if.h
00009 * Author: rafael.luiz.cancian
00010 *
00011 * Created on 14 de Agosto de 2018, 13:54
00012 */
00013
00014 #ifndef SOLVER_IF_H
00015 #define SOLVER_IF_H
00016
00017 class Solver_if {
00018 public:
00019 virtual ~Solver_if() = default;
00020 virtual void setPrecision(double e) = 0;
00021 virtual double getPrecision() = 0;
00022 virtual void setMaxSteps(double steps) = 0;
00023 virtual double getMaxSteps() = 0;
00024 virtual double integrate(double min, double max, double (*f)(double, double), double p2) = 0;
00025 virtual double integrate(double min, double max, double (*f)(double, double, double), double p2,
00026 double p3) = 0;
00027 virtual double integrate(double min, double max, double (*f)(double, double, double, double),
00028 double p2, double p3, double p4) = 0;
00029 virtual double integrate(double min, double max, double (*f)(double, double, double, double,
00030 double), double p2, double p3, double p4, double p5) = 0;
00031 virtual double derivate(double initPoint, double initValue, double (*f)(double, double), double
00032 p2) = 0;
00033 virtual double derivate(double initPoint, double initValue, double (*f)(double, double, double),
00034 double p2, double p3) = 0;
00035 virtual double derivate(double initPoint, double initValue, double (*f)(double, double, double,
00036 double), double p2, double p3, double p4) = 0;
00037 virtual double derivate(double initPoint, double initValue, double (*f)(double, double, double,
00038 double, double), double p2, double p3, double p4, double p5) = 0;
00039 };
00040
00041 #endif /* SOLVER_IF_H */

```

## 16.524 SolverDefaultImpl1.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: IntegratorDefaultImpl1.h
00008 * Author: rlcancian
00009 *
00010 * Created on 23 de novembro de 2021, 18:36
00011 */
00012
00013 #ifndef SOLVERDEFAULTIMPL1_IF_H
00014 #define SOLVERDEFAULTIMPL1_IF_H
00015
00016 #include "Solver_if.h"
00017
00018 class SolverDefaultImpl1 : public Solver_if {
00019 public:
00020 SolverDefaultImpl1(double precision = 1e-6, unsigned int steps = 1e3);
00021 virtual ~SolverDefaultImpl1() = default;
00022 public:
00023 virtual void setPrecision(double e) override;
00024 virtual double getPrecision() override;
00025 virtual void setMaxSteps(double steps) override;
00026 virtual double getMaxSteps() override;
00027 virtual double integrate(double min, double max, double (*f)(double, double), double p2) override;
00028 virtual double integrate(double min, double max, double (*f)(double, double, double), double p2,
00029 double p3) override;
00030 virtual double integrate(double min, double max, double (*f)(double, double, double, double),
00031 double p2, double p3, double p4) override;
00032 virtual double integrate(double min, double max, double (*f)(double, double, double, double,
00033 double), double p2, double p3, double p4, double p5) override;
00034 virtual double derivate(double initPoint, double initValue, double (*f)(double, double), double
00035 p2) override;
00036 virtual double derivate(double initPoint, double initValue, double (*f)(double, double, double),
00037 double p2, double p3) override;
00038 virtual double derivate(double initPoint, double initValue, double (*f)(double, double, double,
00039 double, double), double p2, double p3, double p4, double p5) override;

```

```

 double), double p2, double p3, double p4) override;
00050 virtual double derivate(double initPoint, double initValue, double (*f)(double, double, double,
 double, double), double p2, double p3, double p4, double p5) override;
00051 private:
00052 double _precision;
00053 unsigned int _numSteps;
00054 double _stepSize;
00055 };
00056
00057 #endif /* SOLVERDEFAULTIMPL1_IF_H */

```

## 16.525 FactorialDesign.h

```

00001 #ifndef FACTORIAL_DESIGN_H
00002 #define FACTORIAL_DESIGN_H
00003
00004 #include <vector>
00005 #include <string>
00006 #include <tuple>
00007
00008 // Class representing a Fractional Factorial Design of Experiments
00009 class FactorialDesign {
00010 public:
00011 // Design parameters
00012 int k; // Number of factors investigated
00013 int p; // Size of the fraction of the full factorial
00014 int r; // Number of replications
00015 int NUM_FACTORS; // Derived number of factors
00016 int NUM_ROWS; // Number of rows in the sign table
00017
00018 // Storage of experimental results
00019 std::vector<std::vector<int>> resultsMatrix;
00020
00021 // Constructor and main methods
00022 FactorialDesign(int k, int p, int r);
00023 bool checkFeasibility();
00024 void generateIndexCombinations();
00025 void createColumnLabels();
00026 void generateInputs();
00027 void calculateResultsMean();
00028 void createSignTable();
00029 void calculateStatistics();
00030 double linearRegression(const std::vector<int>& xs);
00031 void showSignTable();
00032 void showImpacts();
00033 void showLinearRegressionModel();
00034 void showResiduals();
00035
00036 // Getters
00037 const std::vector<std::vector<int>>& getIndexCombinations() const {
00038 return indexCombinations;
00039 }
00040
00041 const std::vector<std::string>& getColumnLabels() const {
00042 return columnLabels;
00043 }
00044
00045 const std::vector<std::vector<int>>& getInputs() const {
00046 return inputs;
00047 }
00048
00049 const std::vector<double>& getResultsMean() const {
00050 return resultsMean;
00051 }
00052
00053 const std::vector<std::tuple<int, std::vector<int>, std::vector<int>, double>>& getSignTable()
00054 const {
00055 return signTable;
00056 }
00057
00058 const std::vector<double>& getEffectSums() const {
00059 return effectSums;
00060 }
00061
00062 const std::vector<double>& getEffectAverages() const {
00063 return effectAverages;
00064 }
00065
00066 const std::vector<int>& getSquareSums() const {
00067 return squareSums;
00068 }
00069
00070 double getSquareSumError() const {
00071 return squareSumError;
00072 }

```

```

00072
00073 double getSquareSumTotal() const {
00074 return squareSumTotal;
00075 }
00076
00077 const std::vector<double>& getFactorVariations() const {
00078 return factorVariations;
00079 }
00080
00081 double getErrorVariation() const {
00082 return errorVariation;
00083 }
00084
00085 // Setters
00086 void setResultsMatrix(const std::vector<std::vector<int>>& matrix) {
00087 resultsMatrix = matrix;
00088 }
00089
00090 private:
00091 // Internal data structures
00092 std::vector<std::vector<int>> indexCombinations;
00093 std::vector<std::string> factorLabels;
00094 std::vector<std::string> columnLabels;
00095 std::vector<std::vector<int>> inputs;
00096 std::vector<double> resultsMean;
00097 std::vector<std::tuple<int, std::vector<int>, std::vector<int>, double>> signTable;
00098 std::vector<double> effectSums;
00099 std::vector<double> effectAverages;
00100 std::vector<int> squareSums;
00101 double squareSumTotal;
00102 std::vector<double> factorVariations;
00103 double squareSumError;
00104 double errorVariation;
00105
00106 // Helper method
00107 std::vector<std::vector<int>> generateCombinations(const std::vector<int>& elements, int length);
00108 };
00109
00110 #endif

```

## 16.526 Optimizer\_if.h

```

00001 /*
00002 * File: Optimizer_if.h
00003 * Author: Genesys Team
00004 *
00005 * Created on 12 de Abril de 2026
00006 */
00007
00008 #ifndef OPTIMIZER_IF_H
00009 #define OPTIMIZER_IF_H
00010
00011 #include <cstdint>
00012 #include <string>
00013 #include "kernel/util/List.h"
00014
00015 class Model;
00016 class SimulationControl;
00017 class SimulationResponse;
00018
00037 class Optimizer_if {
00038 public:
00039 virtual ~Optimizer_if() = default;
00040
00044 enum class ObjectiveSense {
00045 MINIMIZE,
00046 MAXIMIZE
00047 };
00048
00052 enum class ExecutionState {
00053 NOT_READY,
00054 READY,
00055 RUNNING,
00056 PAUSED,
00057 STOPPED,
00058 FINISHED,
00059 ERROR
00060 };
00061
00069 struct ObjectiveDefinition {
00070 std::string name;
00071 ObjectiveSense sense;
00072 std::string expression;
00073 double weight = 1.0;
00074 bool enabled = true;

```

```

00075 };
00076
00084 struct ConstraintDefinition {
00085 std::string name;
00086 std::string expression;
00087 bool enabled = true;
00088 };
00089
00093 struct OptimizationSettings {
00094 unsigned int maxIterations = 100;
00095 unsigned int maxSimulations = 0;
00096 unsigned int replicationsPerSolution = 1;
00097 unsigned int bestSolutionsToKeep = 10;
00098 unsigned int randomSeed = 0;
00099 double improvementTolerance = 0.0;
00100 double timeLimitSeconds = 0.0;
00101 };
00102
00110 struct SolutionSummary {
00111 unsigned int iteration = 0;
00112 unsigned int simulationNumber = 0;
00113 bool feasible = false;
00114 double aggregatedObjectiveValue = 0.0;
00115 std::string description;
00116 };
00117
00121 virtual void setModel(Model* model) = 0;
00122
00129 virtual bool setModelFilename(const std::string& filename) = 0;
00130
00134 virtual Model* getModel() const = 0;
00135
00139 virtual std::string getModelFilename() const = 0;
00140
00144 virtual List<SimulationControl*>* getAvailableControls() const = 0;
00145
00149 virtual List<SimulationResponse*>* getAvailableResponses() const = 0;
00150
00154 virtual void setSelectedControls(List<SimulationControl*>* controls) = 0;
00155
00159 virtual void setSelectedResponses(List<SimulationResponse*>* responses) = 0;
00160
00164 virtual List<SimulationControl*>* getSelectedControls() const = 0;
00165
00169 virtual List<SimulationResponse*>* getSelectedResponses() const = 0;
00170
00174 virtual void addObjective(const ObjectiveDefinition& objective) = 0;
00175
00179 virtual void clearObjectives() = 0;
00180
00184 virtual List<ObjectiveDefinition*>* getObjectives() const = 0;
00185
00189 virtual void addConstraint(const ConstraintDefinition& constraint) = 0;
00190
00194 virtual void clearConstraints() = 0;
00195
00199 virtual List<ConstraintDefinition*>* getConstraints() const = 0;
00200
00204 virtual void setSettings(const OptimizationSettings& settings) = 0;
00205
00209 virtual OptimizationSettings getSettings() const = 0;
00210
00217 virtual bool checkReady(std::string* message = nullptr) const = 0;
00218
00223 virtual bool start() = 0;
00224
00229 virtual bool step() = 0;
00230
00234 virtual bool pause() = 0;
00235
00239 virtual bool resume() = 0;
00240
00244 virtual bool stop() = 0;
00245
00249 virtual ExecutionState getExecutionState() const = 0;
00250
00254 virtual unsigned int getCurrentIteration() const = 0;
00255
00259 virtual unsigned int getTotalSimulations() const = 0;
00260
00264 virtual List<SolutionSummary*>* getBestSolutions() const = 0;
00265
00269 virtual const SolutionSummary* getCurrentBestSolution() const = 0;
00270
00274 virtual void resetResults() = 0;
00275 };
00276

```

```
00277 #endif /* OPTIMIZER_IF_H */
```

## 16.527 OptimizerDefaultImpl1.h

```
00001 /*
00002 * File: OptimizerDefaultImpl1.h
00003 * Author: Genesys Team
00004 *
00005 * Created on 12 de Abril de 2026
00006 */
00007
00008 #ifndef OPTIMIZERDEFAULTIMPL1_H
00009 #define OPTIMIZERDEFAULTIMPL1_H
00010
00011 #include "Optimizer_if.h"
00012 #include <string>
00013
00030 class OptimizerDefaultImpl1 : public Optimizer_if {
00031 public:
00032 OptimizerDefaultImpl1();
00033 virtual ~OptimizerDefaultImpl1();
00034
00035 OptimizerDefaultImpl1(const OptimizerDefaultImpl1&) = delete;
00036 OptimizerDefaultImpl1& operator=(const OptimizerDefaultImpl1&) = delete;
00037 OptimizerDefaultImpl1(OptimizerDefaultImpl1&&) = delete;
00038 OptimizerDefaultImpl1& operator=(OptimizerDefaultImpl1&&) = delete;
00039
00040 public:
00041 virtual void setModel(Model* model) override;
00042 virtual bool setModelFilename(const std::string& filename) override;
00043 virtual Model* getModel() const override;
00044 virtual std::string getModelFilename() const override;
00045
00046 virtual List<SimulationControl*>* getAvailableControls() const override;
00047 virtual List<SimulationResponse*>* getAvailableResponses() const override;
00048 virtual void setSelectedControls(List<SimulationControl*>* controls) override;
00049 virtual void setSelectedResponses(List<SimulationResponse*>* responses) override;
00050 virtual List<SimulationControl*>* getSelectedControls() const override;
00051 virtual List<SimulationResponse*>* getSelectedResponses() const override;
00052
00053 virtual void addObjective(const ObjectiveDefinition& objective) override;
00054 virtual void clearObjectives() override;
00055 virtual List<ObjectiveDefinition*>* getObjectives() const override;
00056
00057 virtual void addConstraint(const ConstraintDefinition& constraint) override;
00058 virtual void clearConstraints() override;
00059 virtual List<ConstraintDefinition*>* getConstraints() const override;
00060
00061 virtual void setSettings(const OptimizationSettings& settings) override;
00062 virtual OptimizationSettings getSettings() const override;
00063 virtual bool checkReady(std::string* message = nullptr) const override;
00064
00065 virtual bool start() override;
00066 virtual bool step() override;
00067 virtual bool pause() override;
00068 virtual bool resume() override;
00069 virtual bool stop() override;
00070
00071 virtual ExecutionState getExecutionState() const override;
00072 virtual unsigned int getCurrentIteration() const override;
00073 virtual unsigned int getTotalSimulations() const override;
00074 virtual List<SolutionSummary*>* getBestSolutions() const override;
00075 virtual const SolutionSummary* getCurrentBestSolution() const override;
00076 virtual void resetResults() override;
00077
00078 private:
00079 void _clearRuntimeState();
00080 void _refreshAvailableElementsFromModel();
00081 void _copyControlList(List<SimulationControl*>* source, List<SimulationControl*>* target);
00082 void _copyResponseList(List<SimulationResponse*>* source, List<SimulationResponse*>* target);
00083 bool _hasEnabledObjective() const;
00084 void _updateReadyState();
00085
00086 private:
00087 Model* _model = nullptr;
00088 std::string _modelFilename;
00089 List<SimulationControl*>* _availableControls = nullptr;
00090 List<SimulationResponse*>* _availableResponses = nullptr;
00091 List<SimulationControl*>* _selectedControls = nullptr;
00092 List<SimulationResponse*>* _selectedResponses = nullptr;
00093 List<ObjectiveDefinition*>* _objectives = nullptr;
00094 List<ConstraintDefinition*>* _constraints = nullptr;
00095 List<SolutionSummary*>* _bestSolutions = nullptr;
00096 OptimizationSettings _settings;
00097 ExecutionState _executionState = ExecutionState::NOT_READY;
```



```

00098 unsigned int _currentIteration = 0;
00099 unsigned int _totalSimulations = 0;
00100 std::string _lastErrorMessage;
00101 const SolutionSummary* _currentBestSolution = nullptr;
00102 };
00103
00104 #endif /* OPTIMIZERDEFAULTIMPL1_H */

```

## 16.528 ContinuousDistribution\_if.h

```

00001 #ifndef CONTINUOUSDISTRIBUTION_IF_H
00002 #define CONTINUOUSDISTRIBUTION_IF_H
00003
00004 #include "Distribution_if.h"
00005
00014 class ContinuousDistribution_if : public Distribution_if {
00015 public:
00016 virtual ~ContinuousDistribution_if() = default;
00017 virtual double pdf(double x) const = 0;
00018 virtual double cdf(double x) const = 0;
00019 virtual double quantile(double probability) const = 0;
00020 };
00021
00022 #endif /* CONTINUOUSDISTRIBUTION_IF_H */

```

## 16.529 DataAnalyser\_if.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/file.h to edit this template
00004 */
00005
00006 /*
00007 * File: DataAnalyser.h
00008 * Author: rlcancian
00009 *
00010 * Created on 17 de maio de 2022, 11:06
00011 */
00012
00013 #ifndef DATAANALYSERIF_H
00014 #define DATAANALYSERIF_H
00015
00016 #include <string>
00017 #include "Fitter_if.h"
00018 #include "HypothesisTester_if.h"
00019 #include "kernel/statistics/Sampler_if.h"
00020 #include "kernel/simulator/ExperimetManager_if.h"
00021
00042 class DataAnalyser_if {
00043 public:
00044 virtual ~DataAnalyser_if() = default;
00045 virtual bool loadDataSet(std::string datafilename) = 0;
00046 virtual bool saveDataSet(std::string datasetname) = 0;
00047 virtual void newDataSet(std::string datasetname, std::string datafilename) = 0;
00048 virtual Fitter_if* fitter() = 0;
00049 virtual Sampler_if* sampler() = 0;
00050 virtual ExperimentManager_if* experimenter() = 0;
00051 virtual HypothesisTester_if* tester() = 0;
00052 };
00053
00054
00055 #endif /* DATAANALYSERIF_H */

```

## 16.530 DataSet\_if.h

```

00001 #ifndef DATASET_IF_H
00002 #define DATASET_IF_H
00003
00004 #include <string>
00005
00016 class DataSet_if {
00017 public:
00018 virtual ~DataSet_if() = default;
00019 virtual unsigned int size() const = 0;
00020 virtual double value(unsigned int index) const = 0;
00021 virtual bool isLoaded() const = 0;
00022 virtual std::string sourceDescription() const = 0;
00023 };
00024

```

```
00025 #endif /* DATASET_IF_H */
```

## 16.531 DiscreteDistribution\_if.h

```
00001 #ifndef DISCRETEDISTRIBUTION_IF_H
00002 #define DISCRETEDISTRIBUTION_IF_H
00003
00004 #include "Distribution_if.h"
00005
00014 class DiscreteDistribution_if : public Distribution_if {
00015 public:
00016 virtual ~DiscreteDistribution_if() = default;
00017 virtual double pmf(unsigned int k) const = 0;
00018 virtual double cdf(unsigned int k) const = 0;
00019 virtual unsigned int quantile(double probability) const = 0;
00020 };
00021
00022 #endif /* DISCRETEDISTRIBUTION_IF_H */
```

## 16.532 Distribution\_if.h

```
00001 #ifndef DISTRIBUTION_IF_H
00002 #define DISTRIBUTION_IF_H
00003
00004 #include <string>
00005
00009 class Distribution_if {
00010 public:
00011 virtual ~Distribution_if() = default;
00012 virtual std::string name() const = 0;
00013 };
00014
00015 #endif /* DISTRIBUTION_IF_H */
```

## 16.533 Fitter\_if.h

```
00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: Fitter_if.h
00009 * Author: cancian
00010 *
00011 * Created on 14 de Agosto de 2018, 14:05
00012 */
00013
00014 #ifndef FITTER_IF_H
00015 #define FITTER_IF_H
00016
00017 #include <string>
00018
00051 class Fitter_if {
00052 public:
00053 virtual ~Fitter_if() = default;
00054 virtual bool isNormalDistributed(double confidencelevel) = 0;
00055 virtual void fitUniform(double *sqerror, double *min, double *max) = 0;
00056 virtual void fitTriangular(double *sqerror, double *min, double *mo, double *max) = 0;
00057 virtual void fitNormal(double *sqerror, double *avg, double *stddev) = 0;
00058 virtual void fitExpo(double *sqerror, double *avgl) = 0;
00059 virtual void fitErlang(double *sqerror, double *avg, double *m) = 0;
00060 virtual void fitBeta(double *sqerror, double *alpha, double *beta, double *infLimit, double
 *supLimit) = 0;
00061 virtual void fitWeibull(double *sqerror, double *alpha, double *scale) = 0;
00062 virtual void fitAll(double *sqerror, std::string *name) = 0;
00063 public:
00067 virtual void setDataFilename(std::string dataFilename) = 0;
00071 virtual std::string getDataFilename() = 0;
00072 };
00073
00074 #endif /* FITTER_IF_H */
```

## 16.534 FitterDefaultImpl.h

```
00001 #ifndef FITTERDEFAULTIMPL_H
```

```

00002 #define FITTERDEFAULTIMPL_H
00003
00004 #include "Fitter_if.h"
00005 #include "ProbabilityDistributionBase.h"
00006 #include "tools/Continuous/SolverDefaultImpl.h"
00007
00008 #include <algorithm>
00009 #include <cmath>
00010 #include <fstream>
00011 #include <functional>
00012 #include <limits>
00013 #include <numeric>
00014 #include <string>
00015 #include <vector>
00016
00036 class FitterDefaultImpl : public Fitter_if {
00037 public:
00038 FitterDefaultImpl() = default;
00039 virtual ~FitterDefaultImpl() = default;
00040
00041 public:
00042 virtual bool isNormalDistributed(double confidencelevel) override {
00043 double sqrrerror = std::numeric_limits<double>::infinity();
00044 double avg = _nan();
00045 double stddev = _nan();
00046 fitNormal(&sqrrerror, &avg, &stddev);
00047 if (!std::isfinite(sqrrerror)) {
00048 return false;
00049 }
00050 if (_count < 8) {
00051 return false;
00052 }
00053 double cl = confidencelevel;
00054 if (cl > 1.0) {
00055 cl /= 100.0;
00056 }
00057 cl = std::max(0.0, std::min(1.0, cl));
00058 const double normalizedSse = sqrrerror / static_cast<double>(_count);
00059 const double threshold = 0.008 + (1.0 - cl) * 0.02;
00060 return normalizedSse <= threshold;
00061 }
00062
00063 virtual void fitUniform(double *sqrrerror, double *min, double *max) override {
00064 if (!_ensureDataLoaded() || _count < 2 || !(_sampleMax > _sampleMin)) {
00065 _setFailure3(sqrrerror, min, max);
00066 return;
00067 }
00068 const double fittedMin = _sampleMin;
00069 const double fittedMax = _sampleMax;
00070
00071 if (min != nullptr) {
00072 *min = fittedMin;
00073 }
00074 if (max != nullptr) {
00075 *max = fittedMax;
00076 }
00077 _setSse(sqrrerror, [fittedMin, fittedMax](double x) {
00078 if (x <= fittedMin) {
00079 return 0.0;
00080 }
00081 if (x >= fittedMax) {
00082 return 1.0;
00083 }
00084 return (x - fittedMin) / (fittedMax - fittedMin);
00085 });
00086 }
00087
00088 virtual void fitTriangular(double *sqrrerror, double *min, double *mo, double *max) override {
00089 if (!_ensureDataLoaded() || _count < 2 || !(_sampleMax > _sampleMin)) {
00090 _setFailure4(sqrrerror, min, mo, max);
00091 return;
00092 }
00093
00094 const double fittedMin = _sampleMin;
00095 const double fittedMax = _sampleMax;
00096 const double range = fittedMax - fittedMin;
00097 double mode = 3.0 * _sampleMean - fittedMin - fittedMax;
00098 mode = std::max(fittedMin, std::min(fittedMax, mode));
00099
00100 const double eps = range * 1e-9;
00101 if (mode <= fittedMin) {
00102 mode = fittedMin + eps;
00103 }
00104 if (mode >= fittedMax) {
00105 mode = fittedMax - eps;
00106 }
00107 if (!(mode > fittedMin) || !(mode < fittedMax)) {

```

```

00108 _setFailure4(sqrerror, min, mo, max);
00109 return;
00110 }
00111
00112 if (min != nullptr) {
00113 *min = fittedMin;
00114 }
00115 if (mo != nullptr) {
00116 *mo = mode;
00117 }
00118 if (max != nullptr) {
00119 *max = fittedMax;
00120 }
00121
00122 _setSse(sqrerror, [fittedMin, mode, fittedMax](double x) {
00123 if (x <= fittedMin) {
00124 return 0.0;
00125 }
00126 if (x >= fittedMax) {
00127 return 1.0;
00128 }
00129 if (x <= mode) {
00130 const double num = (x - fittedMin) * (x - fittedMin);
00131 const double den = (fittedMax - fittedMin) * (mode - fittedMin);
00132 return num / den;
00133 }
00134 const double num = (fittedMax - x) * (fittedMax - x);
00135 const double den = (fittedMax - fittedMin) * (fittedMax - mode);
00136 return 1.0 - num / den;
00137 });
00138 }
00139
00140 virtual void fitNormal(double *sqrerror, double *avg, double *stddev) override {
00141 if (!_ensureDataLoaded() || _count < 2 || !(_sampleStddev > 0.0) ||
!std::isfinite(_sampleStddev)) {
00142 _setFailure3(sqrerror, avg, stddev);
00143 return;
00144 }
00145 if (avg != nullptr) {
00146 *avg = _sampleMean;
00147 }
00148 if (stddev != nullptr) {
00149 *stddev = _sampleStddev;
00150 }
00151 _setSse(sqrerror, [this](double x) {
00152 const double z = (x - _sampleMean) / (_sampleStddev * std::sqrt(2.0));
00153 return 0.5 * std::erfc(-z);
00154 });
00155 }
00156
00157 virtual void fitExpo(double *sqrerror, double *avg1) override {
00158 if (!_ensureDataLoaded() || !(_sampleMean > 0.0) || _hasNegativeData) {
00159 _setFailure2(sqrerror, avg1);
00160 return;
00161 }
00162 if (avg1 != nullptr) {
00163 *avg1 = _sampleMean;
00164 }
00165 _setSse(sqrerror, [this](double x) {
00166 if (x < 0.0) {
00167 return 0.0;
00168 }
00169 return 1.0 - std::exp(-x / _sampleMean);
00170 });
00171 }
00172
00173 virtual void fitErlang(double *sqrerror, double *avg, double *m) override {
00174 if (!_ensureDataLoaded() || !(_sampleMean > 0.0) || !(_sampleVariance > 0.0) ||
_hasNegativeData) {
00175 _setFailure3(sqrerror, avg, m);
00176 return;
00177 }
00178 long int mInt = static_cast<long int>(std::llround((_sampleMean * _sampleMean) /
_sampleVariance));
00179 if (mInt < 1) {
00180 mInt = 1;
00181 }
00182 const double scale = _sampleMean / static_cast<double>(mInt);
00183 if (!(scale > 0.0) || !std::isfinite(scale)) {
00184 _setFailure3(sqrerror, avg, m);
00185 return;
00186 }
00187 if (avg != nullptr) {
00188 *avg = _sampleMean;
00189 }
00190 if (m != nullptr) {
00191 *m = static_cast<double>(mInt);

```

```

00192 }
00193 _setSse(sqrerror, [mInt, scale](double x) {
00194 if (x < 0.0) {
00195 return 0.0;
00196 }
00197 const double z = x / scale;
00198 double sum = 1.0;
00199 double term = 1.0;
00200 for (long int k = 1; k < mInt; ++k) {
00201 term *= z / static_cast<double>(k);
00202 sum += term;
00203 }
00204 return 1.0 - std::exp(-z) * sum;
00205 });
00206 }
00207
00208 virtual void fitBeta(double *sqrerror, double *alpha, double *beta, double *infLimit, double
*supLimit) override {
00209 double fittedAlpha = _nan();
00210 double fittedBeta = _nan();
00211 double fittedInf = _nan();
00212 double fittedSup = _nan();
00213 if (!_estimateScaledBetaMoments(&fittedAlpha, &fittedBeta, &fittedInf, &fittedSup)) {
00214 _setFailure5(sqrerror, alpha, beta, infLimit, supLimit);
00215 return;
00216 }
00217
00218 if (alpha != nullptr) {
00219 *alpha = fittedAlpha;
00220 }
00221 if (beta != nullptr) {
00222 *beta = fittedBeta;
00223 }
00224 if (infLimit != nullptr) {
00225 *infLimit = fittedInf;
00226 }
00227 if (supLimit != nullptr) {
00228 *supLimit = fittedSup;
00229 }
00230
00231 _setSse(sqrerror, [this, fittedAlpha, fittedBeta, fittedInf, fittedSup](double x) {
00232 return _betaCdfScaled(x, fittedAlpha, fittedBeta, fittedInf, fittedSup);
00233 });
00234 }
00235
00236 virtual void fitWeibull(double *sqrerror, double *alpha, double *scale) override {
00237 if (!_ensureDataLoaded() || _count < 2 || _hasNegativeData || !(_sampleMean > 0.0) ||
!(_sampleVariance > 0.0)) {
00238 _setFailure3(sqrerror, alpha, scale);
00239 return;
00240 }
00241
00242 const double cv = _sampleStddev / _sampleMean;
00243 double fittedShape = _nan();
00244 if (!_estimateWeibullShapeFromCv(cv, &fittedShape)) {
00245 _setFailure3(sqrerror, alpha, scale);
00246 return;
00247 }
00248
00249 const double gammaTerm = std::tgamma(1.0 + 1.0 / fittedShape);
00250 const double fittedScale = _sampleMean / gammaTerm;
00251 if (!_isFinitePositive(fittedShape) || !_isFinitePositive(fittedScale)) {
00252 _setFailure3(sqrerror, alpha, scale);
00253 return;
00254 }
00255
00256 if (alpha != nullptr) {
00257 *alpha = fittedShape;
00258 }
00259 if (scale != nullptr) {
00260 *scale = fittedScale;
00261 }
00262
00263 _setSse(sqrerror, [this, fittedShape, fittedScale](double x) {
00264 return _weibullCdf(x, fittedShape, fittedScale);
00265 });
00266 }
00267
00268 virtual void fitAll(double *sqrerror, std::string *name) override {
00269 if (!_ensureDataLoaded()) {
00270 if (sqrerror != nullptr) {
00271 *sqrerror = std::numeric_limits<double>::infinity();
00272 }
00273 if (name != nullptr) {
00274 *name = "invalid-dataset";
00275 }
00276 return;

```

```

00277 }
00278
00279 double bestError = std::numeric_limits<double>::infinity();
00280 std::string bestName = "no-valid-fit";
00281
00282 auto consider = [&bestError, &bestName](double error, const std::string& candidate) {
00283 if (std::isfinite(error) && error < bestError) {
00284 bestError = error;
00285 bestName = candidate;
00286 }
00287 };
00288
00289 double error = std::numeric_limits<double>::infinity();
00290 double p1, p2, p3, p4;
00291
00292 fitUniform(&error, &p1, &p2);
00293 consider(error, "uniform");
00294
00295 fitTriangular(&error, &p1, &p2, &p3);
00296 consider(error, "triangular");
00297
00298 fitNormal(&error, &p1, &p2);
00299 consider(error, "normal");
00300
00301 fitExpo(&error, &p1);
00302 consider(error, "exponential");
00303
00304 fitErlang(&error, &p1, &p2);
00305 consider(error, "erlang");
00306
00307 fitBeta(&error, &p1, &p2, &p3, &p4);
00308 consider(error, "beta");
00309
00310 fitWeibull(&error, &p1, &p2);
00311 consider(error, "weibull");
00312
00313 if (sqrrerror != nullptr) {
00314 *sqrrerror = bestError;
00315 }
00316 if (name != nullptr) {
00317 *name = bestName;
00318 }
00319 }
00320
00321 virtual void setDataFilename(std::string dataFilename) override {
00322 if (_dataFilename != dataFilename) {
00323 _dataFilename = dataFilename;
00324 _invalidateCache();
00325 }
00326 }
00327
00328 virtual std::string getDataFilename() override {
00329 return _dataFilename;
00330 }
00331
00332 private:
00333 void _invalidateCache() {
00334 _cacheLoaded = false;
00335 _cacheUsable = false;
00336 _data.clear();
00337 _sortedData.clear();
00338 _count = 0;
00339 _sampleMin = _nan();
00340 _sampleMax = _nan();
00341 _sampleMean = _nan();
00342 _sampleVariance = _nan();
00343 _sampleStddev = _nan();
00344 _hasNegativeData = false;
00345 }
00346
00347 bool _ensureDataLoaded() {
00348 if (!_cacheLoaded) {
00349 _cacheUsable = _loadDataFromBinaryFile();
00350 _cacheLoaded = true;
00351 }
00352 return _cacheUsable;
00353 }
00354
00355 bool _loadDataFromBinaryFile() {
00356 _invalidateCache();
00357 if (_dataFilename.empty()) {
00358 return false;
00359 }
00360
00361 std::ifstream file(_dataFilename.c_str(), std::ios::in | std::ios::binary | std::ios::ate);
00362 if (!file.good()) {
00363 return false;

```

```

00364 }
00365
00366 const std::streamsize bytes = file.tellg();
00367 if (bytes <= 0 || (bytes % static_cast<std::streamsize>(sizeof(double))) != 0) {
00368 return false;
00369 }
00370
00371 file.seekg(0, std::ios::beg);
00372 const std::size_t n = static_cast<std::size_t>(bytes /
static_cast<std::streamsize>(sizeof(double)));
00373 std::vector<double> values(n, 0.0);
00374 file.read(reinterpret_cast<char*>(values.data()), bytes);
00375 if (!file || file.gcount() != bytes) {
00376 return false;
00377 }
00378
00379 for (double value : values) {
00380 if (!std::isfinite(value)) {
00381 return false;
00382 }
00383 }
00384
00385 _data = values;
00386 _sortedData = _data;
00387 std::sort(_sortedData.begin(), _sortedData.end());
00388 _count = _data.size();
00389 if (_count == 0) {
00390 return false;
00391 }
00392
00393 _sampleMin = _sortedData.front();
00394 _sampleMax = _sortedData.back();
00395 const double sum = std::accumulate(_data.begin(), _data.end(), 0.0);
00396 _sampleMean = sum / static_cast<double>(_count);
00397 _hasNegativeData = (_sampleMin < 0.0);
00398
00399 if (_count >= 2) {
00400 double sq = 0.0;
00401 for (double x : _data) {
00402 const double d = x - _sampleMean;
00403 sq += d * d;
00404 }
00405 _sampleVariance = sq / static_cast<double>(_count - 1U);
00406 _sampleStddev = std::sqrt(_sampleVariance);
00407 } else {
00408 _sampleVariance = _nan();
00409 _sampleStddev = _nan();
00410 }
00411 return true;
00412 }
00413
00414 void _setSse(double* sqerror, const std::function<double(double)>& cdf) {
00415 if (sqerror == nullptr) {
00416 return;
00417 }
00418 if (_sortedData.empty()) {
00419 *sqerror = std::numeric_limits<double>::infinity();
00420 return;
00421 }
00422 double sse = 0.0;
00423 const std::size_t n = _sortedData.size();
00424 for (std::size_t i = 0; i < n; ++i) {
00425 const double pi = (static_cast<double>(i) + 0.5) / static_cast<double>(n);
00426 double model = cdf(_sortedData[i]);
00427 if (!std::isfinite(model)) {
00428 *sqerror = std::numeric_limits<double>::infinity();
00429 return;
00430 }
00431 model = std::max(0.0, std::min(1.0, model));
00432 const double d = model - pi;
00433 sse += d * d;
00434 }
00435 *sqerror = sse;
00436 }
00437
00438 static double _clampUnitInterval(double value) {
00439 return std::max(0.0, std::min(1.0, value));
00440 }
00441
00442 static bool _isFinitePositive(double value) {
00443 return std::isfinite(value) && value > 0.0;
00444 }
00445
00446 static double _betaPdfSafe(double x, double alpha, double beta) {
00447 if (!(_isFinitePositive(alpha) && _isFinitePositive(beta))) {
00448 return std::numeric_limits<double>::quiet_NaN();
00449 }

```

```

00450 const double eps = 1e-12;
00451 const double xc = std::max(eps, std::min(1.0 - eps, x));
00452 const double pdf = ProbabilityDistributionBase::beta(xc, alpha, beta);
00453 if (!std::isfinite(pdf) || pdf < 0.0) {
00454 return std::numeric_limits<double>::quiet_NaN();
00455 }
00456 return pdf;
00457 }
00458
00459 double _betaCdfScaled(double x, double alpha, double beta, double infLimit, double supLimit) const
00460 {
00461 if (!(_isFinitePositive(alpha) && _isFinitePositive(beta))) {
00462 return _nan();
00463 }
00464 if (!std::isfinite(infLimit) || !std::isfinite(supLimit) || !(supLimit > infLimit)) {
00465 return _nan();
00466 }
00467 if (x <= infLimit) {
00468 return 0.0;
00469 }
00470 if (x >= supLimit) {
00471 return 1.0;
00472 }
00473 const double y = (x - infLimit) / (supLimit - infLimit);
00474 if (y <= 0.0) {
00475 return 0.0;
00476 }
00477 if (y >= 1.0) {
00478 return 1.0;
00479 }
00480 SolverDefaultImpl1 solver(1e-5, 2000);
00481 const double cdf = solver.integrate(0.0, y, &_betaPdfSafe, alpha, beta);
00482 if (!std::isfinite(cdf)) {
00483 return _nan();
00484 }
00485 return _clampUnitInterval(cdf);
00486 }
00487
00488 double _weibullCdf(double x, double alpha, double scale) const {
00489 if (!(_isFinitePositive(alpha) && _isFinitePositive(scale))) {
00490 return _nan();
00491 }
00492 if (x < 0.0) {
00493 return 0.0;
00494 }
00495 const double ratio = x / scale;
00496 const double p = std::pow(ratio, alpha);
00497 if (!std::isfinite(p)) {
00498 return _nan();
00499 }
00500 return _clampUnitInterval(1.0 - std::exp(-p));
00501 }
00502
00503 bool _estimateScaledBetaMoments(double* alpha, double* beta, double* infLimit, double* supLimit) {
00504 if (!_ensureDataLoaded() || _count < 2 || !(_sampleMax > _sampleMin) || !(_sampleVariance >
00505 0.0)) {
00506 return false;
00507 }
00508
00509 const double a = _sampleMin;
00510 const double b = _sampleMax;
00511 const double range = b - a;
00512 if (range > 0.0) {
00513 return false;
00514 }
00515
00516 const double eps = 1e-12;
00517 double sum = 0.0;
00518 for (double x : _data) {
00519 const double y = (x - a) / range;
00520 const double yc = std::max(eps, std::min(1.0 - eps, y));
00521 sum += yc;
00522 }
00523 const double n = static_cast<double>(_data.size());
00524 const double m = sum / n;
00525 if (!(m > 0.0 && m < 1.0) || !std::isfinite(m)) {
00526 return false;
00527 }
00528
00529 double varSum = 0.0;
00530 for (double x : _data) {
00531 const double y = (x - a) / range;
00532 const double yc = std::max(eps, std::min(1.0 - eps, y));
00533 const double d = yc - m;
00534 varSum += d * d;
00535 }
00536 const double v = varSum / static_cast<double>(_data.size() - 1U);

```



```

00535 if (!(v > 0.0) || !std::isfinite(v)) {
00536 return false;
00537 }
00538
00539 const double common = (m * (1.0 - m) / v) - 1.0;
00540 const double aShape = m * common;
00541 const double bShape = (1.0 - m) * common;
00542 if (!(common > 0.0) || !_isFinitePositive(aShape) || !_isFinitePositive(bShape)) {
00543 return false;
00544 }
00545
00546 if (alpha != nullptr) {
00547 *alpha = aShape;
00548 }
00549 if (beta != nullptr) {
00550 *beta = bShape;
00551 }
00552 if (infLimit != nullptr) {
00553 *infLimit = a;
00554 }
00555 if (supLimit != nullptr) {
00556 *supLimit = b;
00557 }
00558 return true;
00559 }
00560
00561 bool _estimateWeibullShapeFromCv(double cv, double* shape) const {
00562 if (!(cv > 0.0) || !std::isfinite(cv) || shape == nullptr) {
00563 return false;
00564 }
00565 const double cv2 = cv * cv;
00566 auto g = [cv2](double k) {
00567 const double g1 = std::tgamma(1.0 + 1.0 / k);
00568 const double g2 = std::tgamma(1.0 + 2.0 / k);
00569 if (!(std::isfinite(g1) && std::isfinite(g2) && g1 > 0.0)) {
00570 return std::numeric_limits<double>::quiet_NaN();
00571 }
00572 return (g2 / (g1 * g1)) - 1.0 - cv2;
00573 };
00574
00575 double lo = 0.1;
00576 double hi = 100.0;
00577 double flo = g(lo);
00578 double fhi = g(hi);
00579 if (!(std::isfinite(flo) && std::isfinite(fhi))) {
00580 return false;
00581 }
00582 if (flo * fhi > 0.0) {
00583 return false;
00584 }
00585
00586 for (int i = 0; i < 100; ++i) {
00587 const double mid = 0.5 * (lo + hi);
00588 const double fmid = g(mid);
00589 if (!std::isfinite(fmid)) {
00590 return false;
00591 }
00592 if (std::abs(fmid) < 1e-10 || (hi - lo) < 1e-8) {
00593 *shape = mid;
00594 return true;
00595 }
00596 if (flo * fmid > 0.0) {
00597 lo = mid;
00598 flo = fmid;
00599 } else {
00600 hi = mid;
00601 fhi = fmid;
00602 }
00603 }
00604
00605 *shape = 0.5 * (lo + hi);
00606 return _isFinitePositive(*shape);
00607 }
00608
00609 static double _nan() {
00610 return std::numeric_limits<double>::quiet_NaN();
00611 }
00612
00613 void _setFailure2(double* v1, double* v2) {
00614 if (v1 != nullptr) {
00615 *v1 = std::numeric_limits<double>::infinity();
00616 }
00617 if (v2 != nullptr) {
00618 *v2 = _nan();
00619 }
00620 }
00621

```

```

00622 void _setFailure3(double* v1, double* v2, double* v3) {
00623 _setFailure2(v1, v2);
00624 if (v3 != nullptr) {
00625 *v3 = _nan();
00626 }
00627 }
00628
00629 void _setFailure4(double* v1, double* v2, double* v3, double* v4) {
00630 _setFailure3(v1, v2, v3);
00631 if (v4 != nullptr) {
00632 *v4 = _nan();
00633 }
00634 }
00635
00636 void _setFailure5(double* v1, double* v2, double* v3, double* v4, double* v5) {
00637 _setFailure4(v1, v2, v3, v4);
00638 if (v5 != nullptr) {
00639 *v5 = _nan();
00640 }
00641 }
00642
00643 private:
00644 std::string _dataFilename = "";
00645
00646 bool _cacheLoaded = false;
00647 bool _cacheUsable = false;
00648 std::vector<double> _data;
00649 std::vector<double> _sortedData;
00650
00651 std::size_t _count = 0;
00652 double _sampleMin = _nan();
00653 double _sampleMax = _nan();
00654 double _sampleMean = _nan();
00655 double _sampleVariance = _nan();
00656 double _sampleStddev = _nan();
00657 bool _hasNegativeData = false;
00658 };
00659
00660 #endif /* FITTERDEFAULTIMPL_H */

```

## 16.535 FitterDummyImpl.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: FitterDummyImpl.h
00009 * Author: cancian
00010 *
00011 * Created on 23 de Agosto de 2018, 15:36
00012 */
00013
00014 #ifndef FITTERDUMMYIMPL_H
00015 #define FITTERDUMMYIMPL_H
00016
00017 #include "Fitter_if.h"
00018
00033 class FitterDummyImpl : public Fitter_if {
00034 public:
00035 FitterDummyImpl();
00036 FitterDummyImpl(const FitterDummyImpl& orig);
00037 ~FitterDummyImpl();
00038 public:
00039 bool isNormalDistributed(double confidencelevel);
00040 void fitUniform(double *sqrrerror, double *min, double *max);
00041 void fitTriangular(double *sqrrerror, double *min, double *mo, double *max);
00042 void fitNormal(double *sqrrerror, double *avg, double *stddev);
00043 void fitExpo(double *sqrrerror, double *avg1);
00044 void fitErlang(double *sqrrerror, double *avg, double *m);
00045 void fitBeta(double *sqrrerror, double *alpha, double *beta, double *infLimit, double *supLimit);
00046 void fitWeibull(double *sqrrerror, double *alpha, double *scale);
00047 void fitAll(double *sqrrerror, std::string *name);
00048 public:
00049 void setDataFilename(std::string dataFilename);
00050 std::string getDataFilename();
00051 private:
00052 std::string _dataFilename = "";
00053 };
00054
00055 #endif /* FITTERDUMMYIMPL_H */

```

## 16.536 HypothesisTester\_if.h

```

00001 /*
00002 * To change this license header, choose License Headers in Project Properties.
00003 * To change this template file, choose Tools | Templates
00004 * and open the template in the editor.
00005 */
00006
00007 /*
00008 * File: HypothesisTester_if.h
00009 * Author: rafael.luiiz.cancian
00010 *
00011 * Created on 23 de Agosto de 2018, 19:04
00012 */
00013
00014 #ifndef HYPOTHESISTESTER_IF_H
00015 #define HYPOTHESISTESTER_IF_H
00016
00017 #include <string>
00018 #include "kernel/statistics/StatisticsDataFile_if.h"
00019
00020 typedef bool (*checkProportionFunction)(double value);
00021
00022 class HypothesisTester_if {
00023 public:
00024 virtual ~HypothesisTester_if() = default;
00025
00026 class ConfidenceInterval {
00027 public:
00028 ConfidenceInterval(double inferiorLimit, double superiorLimit, double e0) {
00029 if (inferiorLimit <= superiorLimit) {
00030 _infLim = inferiorLimit;
00031 _supLim = superiorLimit;
00032 _e0 = e0;
00033 } else {
00034 _infLim = superiorLimit;
00035 _supLim = inferiorLimit;
00036 }
00037 }
00038
00039 double inferiorLimit() {
00040 return _infLim;
00041 }
00042
00043 double superiorLimit() {
00044 return _supLim;
00045 }
00046
00047 double halfWidth() {
00048 return _e0;
00049 }
00050
00051 private:
00052 double _infLim, _supLim, _e0;
00053 };
00054
00055 enum H1Comparison {
00056 DIFFERENT = 1,
00057 LESS_THAN = 2,
00058 GREATER_THAN = 3
00059 };
00060
00061 class TestResult {
00062 public:
00063 TestResult(double pvalue, bool rejectH0, double acceptanceInferiorLimit, double
00064 acceptanceSuperiorLimit, double testStat) {
00065 _pvalue = pvalue;
00066 _rejectH0 = rejectH0;
00067 _acceptanceInferiorLimit = acceptanceInferiorLimit;
00068 _acceptanceSuperiorLimit = acceptanceSuperiorLimit;
00069 _testStat = testStat;
00070 }
00071
00072 inline bool rejectH0() const {
00073 return _rejectH0;
00074 }
00075
00076 inline bool acceptH0() const {
00077 return !_rejectH0;
00078 }
00079
00080 inline double pValue() const {
00081 return _pvalue;
00082 }
00083 };
00084
00085 };
00086
00087
00088
00089
00090
00091
00092
00093
00094
00095
00096
00097
00098
00099
00100
00101
00102
00103
00104
00105
00106
00107
00108
00109
00110
00111

```

```

00112 inline double testStat() const {
00113 return _testStat;
00114 }
00115
00116 inline double acceptanceInferiorLimit() const {
00117 return _acceptanceInferiorLimit;
00118 }
00119
00120 inline double acceptanceSuperiorLimit() const {
00121 return _acceptanceSuperiorLimit;
00122 }
00123
00124 private:
00125 double _pvalue, _acceptanceInferiorLimit, _acceptanceSuperiorLimit, _testStat;
00126 bool _rejectH0;
00127 };
00128
00129
00130 public:
00131 // One-population confidence intervals.
00132 virtual HypothesisTester_if::ConfidenceInterval averageConfidenceInterval(double avg, double
stddev, unsigned int n, double confidenceLevel) = 0;
00133 virtual HypothesisTester_if::ConfidenceInterval proportionConfidenceInterval(double prop, unsigned
int n, double confidenceLevel) = 0;
00134 virtual HypothesisTester_if::ConfidenceInterval proportionConfidenceInterval(double prop, unsigned
int n, int N, double confidenceLevel) = 0;
00135 virtual HypothesisTester_if::ConfidenceInterval varianceConfidenceInterval(double var, unsigned
int n, double confidenceLevel) = 0;
00136 // Two-population confidence intervals.
00137 virtual HypothesisTester_if::ConfidenceInterval averageDifferenceConfidenceInterval(double avg1,
double stddev1, unsigned int n1, double avg2, double stddev2, unsigned int n2, double confidenceLevel)
= 0;
00138 virtual HypothesisTester_if::ConfidenceInterval proportionDifferenceConfidenceInterval(double
avg1, double stddev1, unsigned int n1, double avg2, double stddev2, unsigned int n2, double
confidenceLevel) = 0;
00139 virtual HypothesisTester_if::ConfidenceInterval varianceRatioConfidenceInterval(double var1,
unsigned int n1, double var2, unsigned int n2, double confidenceLevel) = 0;
00140 // File-based confidence intervals.
00141 virtual HypothesisTester_if::ConfidenceInterval averageConfidenceInterval(std::string
sampleDataFilename, double confidenceLevel) = 0;
00142 virtual HypothesisTester_if::ConfidenceInterval proportionConfidenceInterval(std::string
sampleDataFilename, checkProportionFunction function, double confidenceLevel) = 0;
00143 virtual HypothesisTester_if::ConfidenceInterval proportionConfidenceInterval(std::string
sampleDataFilename, checkProportionFunction function, double N, double confidenceLevel) = 0;
00144 virtual HypothesisTester_if::ConfidenceInterval varianceConfidenceInterval(std::string
sampleDataFilename, double confidenceLevel) = 0;
00145 // determine sample size based on confidence interval
00146 virtual unsigned int estimateSampleSize(double avg, double stddev, double desiredE0, double
confidenceLevel) = 0;
00147
00148 // One-population tests.
00149 virtual HypothesisTester_if::TestResult testAverage(double avg, double stddev, unsigned int n,
double avgSample, double confidenceLevel, HypothesisTester_if::H1Comparison comp) = 0;
00150 virtual HypothesisTester_if::TestResult testProportion(double prop, unsigned int n, double
proptest, double confidenceLevel, HypothesisTester_if::H1Comparison comp) = 0;
00151 virtual HypothesisTester_if::TestResult testVariance(double var, unsigned int n, double vartest,
double confidenceLevel, HypothesisTester_if::H1Comparison comp) = 0;
00152 // Two-population tests.
00153 virtual HypothesisTester_if::TestResult testAverage(double avg1, double stddev1, unsigned int n1,
double avg2, double stddev2, unsigned int n2, double confidenceLevel,
HypothesisTester_if::H1Comparison comp) = 0;
00154 virtual HypothesisTester_if::TestResult testProportion(double prop1, unsigned int n1, double
prop2, unsigned int n2, double confidenceLevel, HypothesisTester_if::H1Comparison comp) = 0;
00155 virtual HypothesisTester_if::TestResult testVariance(double var1, unsigned int n1, double var2,
unsigned int n2, double confidenceLevel, HypothesisTester_if::H1Comparison comp) = 0;
00156 // One-population file-based tests.
00157 virtual HypothesisTester_if::TestResult testAverage(std::string sampleDataFilename, double
avgSample, double confidenceLevel, HypothesisTester_if::H1Comparison comp) = 0;
00158 virtual HypothesisTester_if::TestResult testProportion(std::string sampleDataFilename,
checkProportionFunction function, double proptest, double confidenceLevel,
HypothesisTester_if::H1Comparison comp) = 0;
00159 virtual HypothesisTester_if::TestResult testVariance(std::string sampleDataFilename, double
vartest, double confidenceLevel, HypothesisTester_if::H1Comparison comp) = 0;
00160 // Two-population file-based tests.
00161 virtual HypothesisTester_if::TestResult testAverage(std::string firstSampleDataFilename,
std::string secondSampleDataFilename, double confidenceLevel, HypothesisTester_if::H1Comparison comp)
= 0;
00162 virtual HypothesisTester_if::TestResult testProportion(std::string firstSampleDataFilename,
std::string secondSampleDataFilename, checkProportionFunction function, double confidenceLevel,
HypothesisTester_if::H1Comparison comp) = 0;
00163 virtual HypothesisTester_if::TestResult testVariance(std::string firstSampleDataFilename,
std::string secondSampleDataFilename, double confidenceLevel, HypothesisTester_if::H1Comparison comp)
= 0;
00164 };
00165
00166 #endif /* HYPOTHESISTESTER_IF_H */

```

## 16.537 HypothesisTesterDefaultImpl1.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: HypothesisTesterDefaultImpl1.h
00008 * Author: rlcancian
00009 *
00010 * Created on 24 de novembro de 2021, 02:52
00011 */
00012
00013 #ifndef HYPOTHESISTESTERDEFAULTIMPL1_H
00014 #define HYPOTHESISTESTERDEFAULTIMPL1_H
00015
00016 #include "HypothesisTester_if.h"
00017
00033 class HypothesisTesterDefaultImpl1 : public HypothesisTester_if {
00034 public:
00035 HypothesisTesterDefaultImpl1();
00036 ~HypothesisTesterDefaultImpl1() = default;
00037 public:
00038 // confidence intervals of parameters
00039 HypothesisTester_if::ConfidenceInterval averageConfidenceInterval(double avg, double stddev,
00040 unsigned int n, double confidenceLevel) override;
00041 HypothesisTester_if::ConfidenceInterval proportionConfidenceInterval(double prop, unsigned int n,
00042 double confidenceLevel) override;
00043 HypothesisTester_if::ConfidenceInterval proportionConfidenceInterval(double prop, unsigned int n,
00044 int N, double confidenceLevel) override;
00045 HypothesisTester_if::ConfidenceInterval varianceConfidenceInterval(double var, unsigned int n,
00046 double confidenceLevel) override;
00047 // confidence intervals of differences of parameters
00048 HypothesisTester_if::ConfidenceInterval averageDifferenceConfidenceInterval(double avg1, double
00049 stddev1, unsigned int n1, double avg2, double stddev2, unsigned int n2, double confidenceLevel)
00050 override;
00051 HypothesisTester_if::ConfidenceInterval proportionDifferenceConfidenceInterval(double avg1, double
00052 stddev1, unsigned int n1, double avg2, double stddev2, unsigned int n2, double confidenceLevel)
00053 override;
00054 HypothesisTester_if::ConfidenceInterval varianceRatioConfidenceInterval(double var1, unsigned int
00055 n1, double var2, unsigned int n2, double confidenceLevel) override;
00056 // confidence intervals based on datafile
00057 HypothesisTester_if::ConfidenceInterval averageConfidenceInterval(std::string sampleDataFilename,
00058 double confidenceLevel) override;
00059 HypothesisTester_if::ConfidenceInterval proportionConfidenceInterval(std::string
00060 sampleDataFilename, checkProportionFunction function, double confidenceLevel) override;
00061 HypothesisTester_if::ConfidenceInterval proportionConfidenceInterval(std::string
00062 sampleDataFilename, checkProportionFunction function, double N, double confidenceLevel) override;
00063 HypothesisTester_if::ConfidenceInterval varianceConfidenceInterval(std::string sampleDataFilename,
00064 double confidenceLevel) override;
00065 // determine sample size based on confidence interval
00066 unsigned int estimateSampleSize(double avg, double stddev, double desiredE0, double
00067 confidenceLevel) override;
00068 // one population
00069 HypothesisTester_if::TestResult testAverage(double avg, double stddev, unsigned int n, double
00070 avgSample, double confidenceLevel, HypothesisTester_if::H1Comparison comp) override;
00071 HypothesisTester_if::TestResult testProportion(double prop, unsigned int n, double proptest,
00072 double confidenceLevel, HypothesisTester_if::H1Comparison comp) override;
00073 HypothesisTester_if::TestResult testVariance(double var, unsigned int n, double vartest, double
00074 confidenceLevel, HypothesisTester_if::H1Comparison comp) override;
00075 // two populations
00076 HypothesisTester_if::TestResult testAverage(double avg1, double stddev1, unsigned int n1, double
00077 avg2, double stddev2, unsigned int n2, double confidenceLevel, HypothesisTester_if::H1Comparison
00078 comp) override;
00079 HypothesisTester_if::TestResult testProportion(double prop1, unsigned int n1, double prop2,
00080 unsigned int n2, double confidenceLevel, HypothesisTester_if::H1Comparison comp) override;
00081 HypothesisTester_if::TestResult testVariance(double var1, unsigned int n1, double var2, unsigned
00082 int n2, double confidenceLevel, HypothesisTester_if::H1Comparison comp) override;
00083 // one population based on datafile
00084 HypothesisTester_if::TestResult testAverage(std::string sampleDataFilename, double avgSample,
00085 double confidenceLevel, HypothesisTester_if::H1Comparison comp) override;
00086 HypothesisTester_if::TestResult testProportion(std::string sampleDataFilename,
00087 checkProportionFunction function, double proptest, double confidenceLevel,
00088 HypothesisTester_if::H1Comparison comp) override;
00089 HypothesisTester_if::TestResult testVariance(std::string sampleDataFilename, double vartest,
00090 double confidenceLevel, HypothesisTester_if::H1Comparison comp) override;
00091 // two populations based on datafile
00092 HypothesisTester_if::TestResult testAverage(std::string firstSampleDataFilename, std::string
00093 secondSampleDataFilename, double confidenceLevel, HypothesisTester_if::H1Comparison comp) override;
00094 HypothesisTester_if::TestResult testProportion(std::string firstSampleDataFilename, std::string
00095 secondSampleDataFilename, checkProportionFunction function, double confidenceLevel,
00096 HypothesisTester_if::H1Comparison comp) override;
00097 HypothesisTester_if::TestResult testVariance(std::string firstSampleDataFilename, std::string
00098 secondSampleDataFilename, double confidenceLevel, HypothesisTester_if::H1Comparison comp) override;
00099 };
00100
00101

```

```
00072 #endif /* HYPOTHESISTESTERDEFAULTIMPL1_H */
```

## 16.538 ProbabilityDistribution.h

```
00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: ProbabilityDistribution.h
00008 * Author: rlcancian
00009 *
00010 * Created on 22 de novembro de 2021, 17:24
00011 */
00012
00013 #ifndef PROBABILITYDISTRIBUTION_H
00014 #define PROBABILITYDISTRIBUTION_H
00015
00016 #include "ProbabilityDistributionBase.h"
00017
00033 class ProbabilityDistribution : public ProbabilityDistributionBase {
00034 public:
00035 static double inverseChi2(double cumulativeProbability, double degreeFreedom);
00036 static double inverseFFisherSnedecor(double cumulativeProbability, double d1, double d2);
00037 static double inverseNormal(double cumulativeProbability, double mean, double stddev);
00038 static double inverseTStudent(double cumulativeProbability, double mean, double stddev, double
 degreeFreedom);
00039 private:
00040 static double _findInverseChi2(double a, double fa, double b, double fb, unsigned int recursions,
 double cumulativeProbability, double degreeFreedom);
00041 static double _findInverseFFisherSnedecor(double a, double fa, double b, double fb, unsigned int
 recursions, double cumulativeProbability, double d1, double d2);
00042 static double _findInverseNormal(double a, double fa, double b, double fb, unsigned int
 recursions, double cumulativeProbability, double mean, double stddev);
00043 static double _findInverseTStudent(double a, double fa, double b, double fb, unsigned int
 recursions, double cumulativeProbability, double mean, double stddev, double degreeFreedom);
00044 private:
00049 static std::map<std::string, double>* memory; // = std::map<std::string, double>();
00053 static Solver_if* integrator;
00054 };
00055 #endif /* PROBABILITYDISTRIBUTION_H */
```

## 16.539 ProbabilityDistributionBase.h

```
00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004 */
00005
00006 /*
00007 * File: BaseProbabilityDistribution.h
00008 * Author: rlcancian
00009 *
00010 * Created on 4 de agosto de 2022, 11:03
00011 */
00012
00013 #ifndef PROBABILITYDISTRIBUTIONBASE_H
00014 #define PROBABILITYDISTRIBUTIONBASE_H
00015
00016 #include "tools/Continuous/Solver_if.h"
00017 #include <string>
00018 #include <map>
00019
00031 class ProbabilityDistributionBase {
00032 public:
00034 static double beta(double x, double alpha, double beta);
00036 static double chi2(double x, double degreeFreedom);
00038 static double erlang(double x, double shape, double scale); // int M
00040 static double exponential(double x, double mean);
00042 static double fisherSnedecor(double x, double d1, double d2);
00044 static double gamma(double x, double shape, double scale);
00046 static double logNormal(double x, double mean, double stddev);
00048 static double normal(double x, double mean, double stddev);
00050 static double poisson(double x, double mean);
00052 static double triangular(double x, double min, double mode, double max);
00054 static double tStudent(double x, double mean, double stddev, double degreeFreedom);
00056 static double uniform(double x, double min, double max);
00058 static double weibull(double x, double shape, double scale);
00059 protected:
00061 static double _gammaFunction(double x);
00063 static double _betaFunction(double x, double y);
```

```

00065 static double _factorial(double x);
00066 };
00067
00068 #endif /* PROBABILITYDISTRIBUTIONBASE_H */

```

## 16.540 SimulationResultsDataset.h

```

00001 #ifndef SIMULATIONRESULTS DATASET_H
00002 #define SIMULATIONRESULTS DATASET_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 enum class SimulationResultsDatasetFormat {
00008 Unknown,
00009 RawNumeric,
00010 RecordLegacy,
00011 RecordEnriched,
00012 GuiTabular
00013 };
00014
00015 struct SimulationResultsObservation {
00016 unsigned int replication = 1;
00017 double time = 0.0;
00018 bool hasTime = false;
00019 double value = 0.0;
00020 unsigned int sourceLine = 0;
00021 };
00022
00023 struct SimulationResultsDataset {
00024 std::string datasetName;
00025 std::string randomVariableName;
00026 std::string description;
00027 std::string variableType = "Continuous numeric";
00028 std::string source;
00029 std::string sourceDescription;
00030 std::string expression;
00031 std::string expressionName;
00032 std::string formatVersion;
00033 SimulationResultsDatasetFormat formatKind = SimulationResultsDatasetFormat::Unknown;
00034 bool recordFile = false;
00035 bool timeDependent = false;
00036 std::vector<SimulationResultsObservation> observations;
00037 std::vector<std::string> previewLines;
00038
00042 std::string formatKindName() const;
00043
00047 bool isRecordFormat() const;
00048
00052 std::vector<double> values() const;
00053
00057 std::vector<unsigned int> replications() const;
00058
00062 std::vector<double> valuesForReplication(unsigned int replication) const;
00063 };
00064
00065 class SimulationResultsDatasetParser {
00066 public:
00074 static bool loadFromTextFile(const std::string& fileName, SimulationResultsDataset* dataset,
00075 std::string* errorMessage = nullptr);
00076 };
00077
00078 #endif /* SIMULATIONRESULTS DATASET_H */

```

## 16.541 TraitsTools.h

```

00001 /*
00002 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003 * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/file.h to edit this template
00004 */
00005
00006 /*
00007 * File: TraitsTools.h
00008 * Author: rlcancian
00009 *
00010 * Created on 3 de maio de 2022, 18:57
00011 */
00012
00013 #ifndef TRAITSTOOLS_H
00014 #define TRAITSTOOLS_H
00015
00016 // TOOLS

```

```
00017 #include "AIAssistant/AIAssistant_if.h"
00018 #include "AIAssistant/AIAssistantDefaultImpl.h"
00019 #include "Continuous/SolverDefaultImpl1.h"
00020 #include "Statistics/HypothesisTesterDefaultImpl1.h"
00021 #include "Statistics/HypothesisTester_if.h"
00022 #include "Statistics/FitterDefaultImpl.h"
00023 #include "Statistics/Fitter_if.h"
00024
00039 template <typename T>
00040 struct TraitsTools {
00041 };
00042
00046 template <> struct TraitsTools<Solver_if> {
00047 typedef SolverDefaultImpl1 Implementation;
00048 static constexpr double Precision = 1e-5;
00049 static constexpr unsigned int MaxSteps = 1e2;
00050 };
00051
00055 template <> struct TraitsTools<HypothesisTester_if> {
00056 typedef HypothesisTesterDefaultImpl1 Implementation;
00057 static constexpr unsigned int ConfidenceLevel = 95;
00058 };
00059
00063 template <> struct TraitsTools<Fitter_if> {
00064 typedef FitterDefaultImpl1 Implementation;
00065 };
00066
00070 template <> struct TraitsTools<AIAssistant_if> {
00071 typedef AIAssistantDefaultImpl1 Implementation;
00072 };
00073
00074 #endif /* TRAITSTOOLS_H */
```